Conteúdo da Documentação Python

- O que há de novo no Python
  - O que há de novo no Python 3.15
    - Resumo – Destaques da versão
    - Novas funcionalidades
      - **PEP 810**: Importações preguiçosas explícitas
      - **PEP 814**: Adiciona o tipo embutido frozendict
      - **PEP 661**: Adiciona o tipo embutido sentinela
      - **PEP 799**: Um pacote de perfilamento dedicado
      - Tachyon: Perfilador de amostragem estatística de alta frequência
      - **PEP 831**: Ponteiros de quadros habilitados por padrão
      - **PEP 798**: Desempacotamento em compreensões
      - **PEP 829**: Arquivos de configuração de inicialização de pacote
      - **PEP 803**: ABI Estável para construções com threads livres
      - **PEP 788**: Protegendo a API C de finalização do interpretador
      - Mensagens de erro melhoradas
    - Outras mudanças na linguagem
    - Console interativo padrão
    - Novos módulos
      - math.integer
    - Módulos melhorados
      - argparse
      - array
      - ast
      - asyncio
      - base64
      - binascii
      - calendar

- collections
- concurrent.futures
- contextlib
- ctypes
- dataclasses
- dbm
- difflib
- email
- faulthandler
- functools
- gc
- hashlib
- http.client
- http.server
- importlib.metadata
- inspect
- json
- locale
- math
- mimetypes
- mmap
- os
- os.path
- pathlib
- pdb
- pickle
- pickletools
- pprint
- re
- resource
- shelve
- socket
- sqlite3
- ssl
- subprocess
- symtable
- sys

    - sys.monitoring
    - tarfile
    - threading
    - timeit
    - tkinter
    - tokenize
    - tomllib
    - types
    - typing
    - unicodedata
    - unittest
    - urllib.parse
    - venv
    - avisos
    - wave
    - webbrowser
    - xml
    - xml.parsers.expat
    - zlib
- Otimizações
    - base64 e binascii
    - csv
    - Compilador JIT experimental atualizado
- Removidos
    - ast
    - collections.abc
    - ctypes
    - datetime
    - glob
    - http.server
    - importlib
    - importlib.resources
    - pathlib
    - platform
    - sre_*
    - sysconfig
    - threading

        - types
        - typing
        - wave
        - zipimport
    - Descontinuados
        - Novas descontinuações
        - Remoção pendente no Python 3.16
        - Remoção pendente no Python 3.17
        - Remoção pendente no Python 3.18
        - Remoção pendente no Python 3.19
        - Remoção pendente no Python 3.20
        - Remoção pendente em versões futuras
        - descontinuações suave
    - Alterações na API C
        - Novas funcionalidades
        - Alterações na API C
        - APIs C removidas
        - APIs C descontinuadas
    - Alterações na construção
    - Portando para o Python 3.15
- O que há de novo no Python 3.14
    - Resumo – Destaques da versão
    - Novas funcionalidades
        - **PEP 649** e **PEP 749**: avaliação adiada de anotações
        - **PEP 734**: múltiplos interpretadores na biblioteca padrão
        - **PEP 750**: literais de string template
        - **PEP 768**: interface segura para depurador externo
        - Um novo tipo de interpretador
        - Melhorias no modo com threads livres
        - Mensagens de erro melhoradas
        - **PEP 784**: suporte ao Zstandard na biblioteca padrão
        - Capacidades de introspecção assíncrona
        - Controle de avisos seguros simultâneos
    - Outras mudanças na linguagem
        - Embutidos
        - Linha de comando e ambiente

    - PEP 758: permite expressões `except` e `except*` sem parênteses
    - PEP 765: fluxo de controle em blocos `finally`
    - Coleta de lixo
    - Console interativo padrão
- Novos módulos
- Módulos melhorados
    - argparse
    - ast
    - asyncio
    - calendar
    - concurrent.futures
    - configparser
    - contextvars
    - ctypes
    - curses
    - datetime
    - decimal
    - difflib
    - dis
    - errno
    - faulthandler
    - fnmatch
    - fractions
    - functools
    - getopt
    - getpass
    - graphlib
    - heapq
    - hmac
    - http
    - imaplib
    - inspect
    - io
    - json
    - linecache
    - logging.handlers

    - math
    - mimetypes
    - multiprocessing
    - operador
    - os
    - os.path
    - pathlib
    - pdb
    - pickle
    - platform
    - pydoc
    - re
    - socket
    - ssl
    - struct
    - symtable
    - sys
    - sys.monitoring
    - sysconfig
    - tarfile
    - threading
    - tkinter
    - turtle
    - types
    - typing
    - unicodedata
    - unittest
    - urllib
    - uuid
    - webbrowser
    - zipfile
- Otimizações
    - asyncio
    - base64
    - bdb
    - difflib
    - gc

    - io
    - pathlib
    - pdb
    - textwrap
    - uuid
    - zlib
- Removidos
    - argparse
    - ast
    - asyncio
    - email
    - importlib.abc
    - itertools
    - pathlib
    - pkgutil
    - pty
    - sqlite3
    - urllib
- Descontinuados
    - Novas descontinuações
    - Remoção pendente no Python 3.15
    - Remoção pendente no Python 3.16
    - Remoção pendente no Python 3.17
    - Remoção pendente no Python 3.18
    - Remoção pendente no Python 3.19
    - Remoção pendente no Python 3.20
    - Remoção pendente em versões futuras
- Alterações de bytecode do CPython
    - Pseudoinstruções
- Alterações na API C
    - API C de configuração do Python
    - Novos recursos na API C
    - Mudanças na API C limitada
    - APIs C removidas
    - APIs C descontinuadas
        - Remoção pendente no Python 3.15
        - Remoção pendente no Python 3.16

        - Remoção pendente no Python 3.18
        - Remoção pendente em versões futuras
    - Alterações na construção
        - `build-details.json`
        - Descontinuação de assinaturas PGP
        - Python com threads livres é oficialmente suportado
        - Versões binárias para o compilador experimental just-in-time
    - Portando para o Python 3.14
        - Alterações na API Python
        - Mudanças em anotações (**PEP 649** e **PEP 749**)
            - Implicações para o código anotado
            - Implicações para leitores de `__annotations__`
            - Mudanças relacionadas
            - `from __future__ import annotations`
        - Alterações na API C
    - Alterações notáveis no 3.14.5
        - gc
- O que há de novo no Python 3.13
    - Resumo – Destaques da versão
    - Novas funcionalidades
        - Um melhor interpretador interativo
        - Mensagens de erro melhoradas
        - CPython com threads livres
        - Um compilador just-in-time (JIT) experimental
        - Definidas semânticas de mutação para `locals()`
        - Suporte para plataformas móveis
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
        - argparse
        - array
        - ast
        - asyncio
        - base64
        - compileall
        - concurrent.futures

- configparser
- copy
- ctypes
- dbm
- dis
- doctest
- email
- enum
- fractions
- glob
- importlib
- io
- ipaddress
- itertools
- marshal
- math
- mimetypes
- mmap
- multiprocessing
- os
- os.path
- pathlib
- pdb
- queue
- random
- re
- shutil
- site
- sqlite3
- ssl
- statistics
- subprocess
- sys
- tempfile
- time
- tkinter
- traceback

    - types
    - typing
    - unicodedata
    - venv
    - warnings
    - xml
    - zipimport
- Otimizações
- Módulos e APIs removidas
    - PEP 594: remove “baterias descarregadas” da biblioteca padrão
    - 2to3
    - embutidos
    - configparser
    - importlib.metadata
    - locale
    - opcode
    - optparse
    - pathlib
    - re
    - tkinter.tix
    - turtle
    - typing
    - unittest
    - urllib
    - webbrowser
- Novas descontinuações
    - Remoção pendente no Python 3.14
    - Remoção pendente no Python 3.15
    - Remoção pendente no Python 3.16
    - Remoção pendente no Python 3.17
    - Remoção pendente no Python 3.18
    - Remoção pendente no Python 3.19
    - Remoção pendente no Python 3.20
    - Remoção pendente em versões futuras
- Alterações de bytecode do CPython
- Alterações na API C

        - Novas funcionalidades
        - Alterações na API C
        - Mudanças na API C Limitada
        - APIs C removidas
        - APIs C descontinuadas
            - Remoção pendente no Python 3.14
            - Remoção pendente no Python 3.15
            - Remoção pendente no Python 3.16
            - Remoção pendente no Python 3.18
            - Remoção pendente em versões futuras
    - Mudanças na construção
    - Portando para o Python 3.13
        - Alterações na API Python
        - Alterações na API C
    - Mudanças em teste de regressão
- O que há de novo no Python 3.12
    - Resumo – Destaques da versão
    - Novas funcionalidades
        - PEP 695: Sintaxe do parâmetro de tipo
        - PEP 701: Formalização sintática de f-strings
        - PEP 684: Uma GIL por interpretador
        - PEP 669: Monitoramento de baixo impacto para CPython
        - PEP 688: Tornando o protocolo de buffer acessível no Python
        - PEP 709: Fazendo inline de compreensão
        - Mensagens de erro melhoradas
    - Novos recursos relacionados a dicas de tipo
        - PEP 692: Usando `TypedDict` para tipagem mais precisa de `**kwargs`
        - PEP 698: substitui decorador para tipagem estática
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
        - array
        - asyncio
        - calendar

    - csv
    - dis
    - fractions
    - importlib.resources
    - inspect
    - itertools
    - math
    - os
    - os.path
    - pathlib
    - platform
    - pdb
    - random
    - shutil
    - sqlite3
    - statistics
    - sys
    - tempfile
    - threading
    - tkinter
    - tokenize
    - types
    - typing
    - unicodedata
    - unittest
    - uuid
- Otimizações
- Alterações de bytecode do CPython
- Ferramentas e daemons
- Descontinuados
    - Remoção pendente no Python 3.13
    - Remoção pendente no Python 3.14
    - Remoção pendente no Python 3.15
    - Remoção pendente no Python 3.16
    - Remoção pendente no Python 3.17
    - Remoção pendente no Python 3.19
    - Remoção pendente no Python 3.20

        - Remoção pendente em versões futuras
    - Removidos
        - asynchat e asyncore
        - configparser
        - distutils
        - ensurepip
        - enum
        - ftplib
        - gzip
        - hashlib
        - importlib
        - imp
        - io
        - locale
        - smtpd
        - sqlite3
        - ssl
        - unittest
        - webbrowser
        - xml.etree.ElementTree
        - zipimport
        - Outros
    - Portando para Python 3.12
        - Alterações na API Python
    - Mudanças na construção
    - Alterações na API C
        - Novas funcionalidades
        - Portando para Python 3.12
        - Descontinuados
            - Remoção pendente no Python 3.14
            - Remoção pendente no Python 3.15
            - Remoção pendente no Python 3.16
            - Remoção pendente em versões futuras
        - Removidos
- O que há de novo no Python 3.11
    - Resumo – Destaques da versão
    - Novas funcionalidades

    - PEP 657: Localizações de erros refinadas em tracebacks
    - PEP 654: Grupos de exceção e `except*`
    - PEP 678: Exceções podem ser enriquecidas com notas
    - Melhorias no inicializador `py.exe` do Windows
- Novos recursos relacionados a dicas de tipo
    - PEP 646: Genéricos variádicos
    - PEP 655: Marcando itens `TypedDict` individuais como obrigatórios ou não obrigatórios
    - PEP 673: Tipo `Self`
    - PEP 675: Tipo de string literal arbitrário
    - PEP 681: Transformações de classe de dados
    - PEP 563 pode não ser o futuro
- Outras mudanças na linguagem
- Outras mudanças na implementação do CPython
- Novos módulos
- Módulos melhorados
    - asyncio
    - contextlib
    - dataclasses
    - datetime
    - enum
    - fcntl
    - fractions
    - functools
    - gzip
    - hashlib
    - IDLE e idlelib
    - inspect
    - locale
    - logging
    - math
    - operator
    - os
    - pathlib
    - re
    - shutil
    - socket

- sqlite3
- string
- sys
- sysconfig
- tempfile
- threading
- time
- tkinter
- traceback
- typing
- unicodedata
- unittest
- venv
- warnings
- zipfile
- Otimizações
- CPython mais rápido
  - Inicialização mais rápida
    - Importações congeladas/objetos de código estático
  - Tempo de execução mais rápido
    - Quadros Python menos custosos e mais preguiçosos
    - Chamadas de função Python em linha
    - PEP 659: Interpretador adaptativo especializado
  - Diversos
  - FAQ
    - Como devo escrever meu código para utilizar esses aceleradores?
    - O CPython 3.11 usará mais memória?
    - Não vejo nenhuma aceleração em minha carga de trabalho. Por que?
    - Existe um compilador JIT?
  - Sobre
- Alterações de bytecode do CPython
  - Novos opcodes
  - Opcodes substituídos
  - Opcodes alterados/removidos

- Descontinuados
  - Linguagem/Embutidos
  - Módulos
  - Biblioteca Padrão
- Pendente remoção no Python 3.12
- Removidos
- Portando para Python 3.11
- Mudanças na construção
- Alterações na API C
  - Novas funcionalidades
  - Portando para Python 3.11
  - Descontinuados
  - Pendente remoção no Python 3.12
  - Removidos
- Alterações notáveis no 3.11.4
  - tarfile
- Alterações notáveis no 3.11.5
  - OpenSSL

- O que há de novo no Python 3.10
  - Resumo – Destaques da versão
  - Novas funcionalidades
    - Gerenciadores de contexto entre parênteses
    - Melhores mensagens de erro
      - SyntaxErrors
      - IndentationErrors
      - AttributeErrors
      - NameErrors
    - PEP 626: Números de linha precisos para depuração e outras ferramentas
    - PEP 634: Correspondência de padrão estrutural
      - Sintaxe e operações
      - Abordagem declarativa
      - Padrão simples: corresponder a um literal
        - Comportamento sem o curinga
      - Padrões com uma literal e variável
      - Padrões e classes
        - Padrões com parâmetros posicionais

        - Padrões aninhados
        - Padrões complexos e o curinga
        - Guarda
        - Outros recursos-chave
    - `EncodingWarning` opcional e opção `encoding="locale"`
- Novos recursos relacionados a dicas de tipo
    - PEP 604: novo operador de união de tipo
    - PEP 612: variáveis de especificação de parâmetros
    - PEP 613: TypeAlias
    - PEP 647: guardas de tipo definidas pelo usuário
- Outras mudanças na linguagem
- Novos módulos
- Módulos melhorados
    - asyncio
    - argparse
    - array
    - asynchat, asyncore, smtpd
    - base64
    - bdb
    - bisect
    - codecs
    - collections.abc
    - contextlib
    - curses
    - dataclasses
        - __slots__
        - Campos somente-nomeados
    - distutils
    - doctest
    - encodings
    - enum
    - fileinput
    - faulthandler
    - gc
    - glob
    - hashlib

    - hmac
    - IDLE e idlelib
    - importlib.metadata
    - inspect
    - itertools
    - linecache
    - os
    - os.path
    - pathlib
    - platform
    - pprint
    - py_compile
    - pyclbr
    - shelve
    - statistics
    - site
    - socket
    - ssl
    - sqlite3
    - sys
    - _thread
    - threading
    - traceback
    - types
    - typing
    - unittest
    - urllib.parse
    - xml
    - zipimport
- Otimizações
- Descontinuados
- Removidos
- Portando para Python 3.10
    - Alterações na sintaxe Python
    - Alterações na API Python
    - Alterações na API C
- Alterações de bytecode do CPython

- Mudanças na construção
- Alterações na API C
  - PEP 652: Mantendo a ABI estável
  - Novas funcionalidades
  - Portando para Python 3.10
  - Descontinuados
  - Removidos
- Recursos de segurança notáveis no 3.10.7
- Recursos de segurança notáveis no 3.10.8
- Alterações notáveis no 3.10.12
  - tarfile
- O que há de novo no Python 3.9
  - Resumo – Destaques da versão
  - Você deve verificar DeprecationWarning no seu código
  - Novas funcionalidades
    - Operadores de mesclagem & atualização de dicionário
    - Novos métodos de strings para remover prefixos e sufixos
    - Type hinting genéricos nas coleções padrão
    - Novo analisador sintático
  - Outras mudanças na linguagem
  - Novos módulos
    - zoneinfo
    - graphlib
  - Módulos melhorados
    - ast
    - asyncio
    - compileall
    - concurrent.futures
    - curses
    - datetime
    - distutils
    - fcntl
    - ftplib
    - gc
    - hashlib
    - http

    - IDLE e idlelib
    - imaplib
    - importlib
    - inspect
    - ipaddress
    - math
    - multiprocessing
    - nntplib
    - os
    - pathlib
    - pdb
    - poplib
    - pprint
    - pydoc
    - random
    - signal
    - smtplib
    - socket
    - time
    - sys
    - tracemalloc
    - typing
    - unicodedata
    - venv
    - xml
- Otimizações
- Descontinuados
- Removidos
- Portando para Python 3.9
    - Alterações na API Python
    - Alterações na API C
    - Alterações de bytecode do CPython
- Mudanças na construção
- Alterações na API C
    - Novas funcionalidades
    - Portando para Python 3.9
    - Removidos

- Alterações notáveis no Python 3.9.1
  - typing
  - Suporte a macOS 11.0 (Big Sur) e Apple Silicon Mac
- Alterações notáveis no Python 3.9.2
  - collections.abc
  - urllib.parse
- Alterações notáveis no Python 3.9.3
- Alterações notáveis no Python 3.9.5
  - urllib.parse
- Recursos de segurança notáveis no 3.9.14
- Alterações notáveis no 3.9.17
  - tarfile

- O que há de novo no Python 3.8
  - Resumo – Destaques da versão
  - Novas funcionalidades
    - Expressões de atribuição
    - Parâmetros somente-posicionais
    - Cache do sistema de arquivos paralelo para arquivos de bytecode compilados
    - A compilação de depuração usa a mesma ABI da compilação de lançamento
    - As *f-strings* aceitam `=` para expressões auto-documentadas e depuração
    - PEP 578: Ganchos de auditoria em tempo de execução Python
    - PEP 587: Configuração de Inicialização do Python
    - PEP 590: Vectorcall: um protocolo de chamada rápida para CPython
    - Protocolo de Pickle 5 com buffers de dados fora da banda
  - Outras mudanças na linguagem
  - Novos módulos
  - Módulos melhorados
    - ast
    - asyncio
    - builtins
    - collections

- cProfile
- csv
- curses
- ctypes
- datetime
- functools
- gc
- gettext
- gzip
- IDLE e idlelib
- inspect
- io
- itertools
- json.tool
- logging
- math
- mmap
- multiprocessing
- os
- os.path
- pathlib
- pickle
- plistlib
- pprint
- py_compile
- shlex
- shutil
- socket
- ssl
- statistics
- sys
- tarfile
- threading
- tokenize
- tkinter
- time
- typing

        - unicodedata
        - unittest
        - venv
        - weakref
        - xml
        - xmlrpc
    - Otimizações
    - Mudanças a construções e API C
    - Descontinuados
    - Remoção de APIs e recursos
    - Portando para o Python 3.8
        - Alterações no comportamento do Python
        - Alterações na API Python
        - Alterações na API C
        - Alterações de bytecode do CPython
        - Ferramentas e daemons
    - Alterações notáveis no Python 3.8.1
    - Alterações notáveis no Python 3.8.2
    - Alterações notáveis no Python 3.8.3
    - Alterações notáveis no Python 3.8.8
    - Alterações notáveis no Python 3.8.9
    - Alterações notáveis no Python 3.8.10
        - Suporte a macOS 11.0 (Big Sur) e Apple Silicon Mac
    - Alterações notáveis no Python 3.8.10
        - urllib.parse
    - Alterações notáveis no Python 3.8.12
        - Alterações na API Python
    - Recursos de segurança notáveis no 3.8.14
    - Alterações notáveis no 3.8.17
        - tarfile
- O que há de novo no Python 3.7
    - Resumo – Destaques da versão
    - Novas funcionalidades
        - PEP 563: Avaliação Postergada de Anotações
        - PEP 538: Coerção de localidade C legada
        - PEP 540: Modo de tempo de execução UTF-8 forçado
        - PEP 553: `breakpoint()` embutida

    - PEP 539: Nova API C para armazenamento local de thread
    - PEP 562: Personalização de acesso aos atributos do módulo
    - PEP 564: Novas funções de tempo com resolução de nanossegundos
    - PEP 565: Mostra DeprecationWarning em `__main__`
    - PEP 560: Suporte básico para módulo `typing` e tipos genéricos
    - PEP 552: Arquivos .pyc baseados em hash
    - PEP 545: Traduções da documentação do Python
    - Modo de Desenvolvimento do Python (-X dev)
- Outras mudanças na linguagem
- Novos módulos
    - contextvars
    - dataclasses
    - importlib.resources
- Módulos melhorados
    - argparse
    - asyncio
    - binascii
    - calendar
    - collections
    - compileall
    - concurrent.futures
    - contextlib
    - cProfile
    - crypt
    - datetime
    - dbm
    - decimal
    - dis
    - distutils
    - enum
    - functools
    - gc
    - hmac

- http.client
- http.server
- idlelib e IDLE
- importlib
- io
- ipaddress
- itertools
- locale
- logging
- math
- mimetypes
- msilib
- multiprocessing
- os
- pathlib
- pdb
- py_compile
- pydoc
- queue
- re
- signal
- socket
- socketserver
- sqlite3
- ssl
- string
- subprocess
- sys
- time
- tkinter
- tracemalloc
- types
- unicodedata
- unittest
- unittest.mock
- urllib.parse
- uu

    - uuid
    - avisos
    - xml
    - xml.etree
    - xmlrpc.server
    - zipapp
    - zipfile
- Alterações na API C
- Mudanças na construção
- Otimizações
- Outras mudanças na implementação do CPython
- Comportamento do Python descontinuado
- Deprecated Python modules, functions and methods
    - aifc
    - asyncio
    - collections
    - dbm
    - enum
    - gettext
    - importlib
    - locale
    - macpath
    - threading
    - socket
    - ssl
    - sunau
    - sys
    - wave
- Deprecated functions and types of the C API
- Platform Support Removals
- Remoção de APIs e recursos
- Module Removals
- Windows-only Changes
- Porting to Python 3.7
    - Changes in Python Behavior
    - Alterações na API Python
    - Alterações na API C

        - Alterações de bytecode do CPython
        - Windows-only Changes
        - Other CPython implementation changes
    - Notable changes in Python 3.7.1
    - Notable changes in Python 3.7.2
    - Notable changes in Python 3.7.6
    - Notable changes in Python 3.7.10
    - Notable changes in Python 3.7.11
    - Notable security feature in 3.7.14
  - What's New In Python 3.6
    - Resumo – Destaques da versão
    - Novas funcionalidades
        - PEP 498: Formatted string literals
        - PEP 526: Syntax for variable annotations
        - PEP 515: Underscores in Numeric Literals
        - PEP 525: Asynchronous Generators
        - PEP 530: Asynchronous Comprehensions
        - PEP 487: Simpler customization of class creation
        - PEP 487: Descriptor Protocol Enhancements
        - PEP 519: Adding a file system path protocol
        - PEP 495: Local Time Disambiguation
        - PEP 529: Change Windows filesystem encoding to UTF-8
        - PEP 528: Change Windows console encoding to UTF-8
        - PEP 520: Preserving Class Attribute Definition Order
        - PEP 468: Preserving Keyword Argument Order
        - New dict implementation
        - PEP 523: Adding a frame evaluation API to CPython
        - PYTHONMALLOC environment variable
        - DTrace and SystemTap probing support
    - Outras mudanças na linguagem
    - Novos módulos
        - segredos
    - Módulos melhorados
        - array
        - ast
        - asyncio

- binascii
- cmath
- collections
- concurrent.futures
- contextlib
- datetime
- decimal
- distutils
- email
- encodings
- enum
- faulthandler
- fileinput
- hashlib
- http.client
- idlelib e IDLE
- importlib
- inspect
- json
- logging
- math
- multiprocessing
- os
- pathlib
- pdb
- pickle
- pickletools
- pydoc
- random
- re
- readline
- rlcompleter
- shlex
- site
- sqlite3
- socket
- socketserver

- ssl
- statistics
- struct
- subprocess
- sys
- telnetlib
- time
- timeit
- tkinter
- traceback
- tracemalloc
- typing
- unicodedata
- unittest.mock
- urllib.request
- urllib.robotparser
- venv
- avisos
- winreg
- winsound
- xmlrpc.client
- zipfile
- zlib

- Otimizações
- Mudanças a construções e API C
- Other Improvements
- Descontinuados
  - New Keywords
  - Deprecated Python behavior
  - Deprecated Python modules, functions and methods
    - asynchat
    - asyncore
    - dbm
    - distutils
    - grp
    - importlib
    - os

        - re
        - ssl
        - tkinter
        - venv
    - xml
    - Deprecated functions and types of the C API
    - Deprecated Build Options
  - Removidos
    - Remoção de APIs e recursos
  - Porting to Python 3.6
    - Changes in ‘python’ Command Behavior
    - Alterações na API Python
    - Alterações na API C
    - Alterações de bytecode do CPython
  - Notable changes in Python 3.6.2
    - Novo alvo de construção `make regen-all`
    - Remoção do alvo de construção `make touch`
  - Notable changes in Python 3.6.4
  - Notable changes in Python 3.6.5
  - Notable changes in Python 3.6.7
  - Notable changes in Python 3.6.10
  - Notable changes in Python 3.6.13
  - Notable changes in Python 3.6.14
- O que há de novo no Python 3.5
  - Resumo – Destaques da versão
  - Novas funcionalidades
    - PEP 492 - Coroutines with async and await syntax
    - PEP 465 - A dedicated infix operator for matrix multiplication
    - PEP 448 - Additional Unpacking Generalizations
    - PEP 461 - percent formatting support for bytes and bytearray
    - PEP 484 - Type Hints
    - PEP 471 - os.scandir() function – a better and faster directory iterator
    - PEP 475: Retry system calls failing with EINTR

    - PEP 479: Change StopIteration handling inside generators
    - PEP 485: A function for testing approximate equality
    - PEP 486: Make the Python Launcher aware of virtual environments
    - PEP 488: Elimination of PYO files
    - PEP 489: Multi-phase extension module initialization
- Outras mudanças na linguagem
- Novos módulos
    - typing
    - zipapp
- Módulos melhorados
    - argparse
    - asyncio
    - bz2
    - cgi
    - cmath
    - código
    - collections
    - collections.abc
    - compileall
    - concurrent.futures
    - configparser
    - contextlib
    - csv
    - curses
    - dbm
    - difflib
    - distutils
    - doctest
    - email
    - enum
    - faulthandler
    - functools
    - glob
    - gzip
    - heapq

- http
- http.client
- idlelib e IDLE
- imaplib
- imghdr
- importlib
- inspect
- io
- ipaddress
- json
- linecache
- locale
- logging
- lzma
- math
- multiprocessing
- operador
- os
- pathlib
- pickle
- poplib
- re
- readline
- selectors
- shutil
- signal
- smtpd
- smtplib
- sndhdr
- socket
- ssl
  - Підтримка BIO пам’яті
  - Application-Layer Protocol Negotiation Support
  - Other Changes
- sqlite3
- subprocess
- sys

        - sysconfig
        - tarfile
        - threading
        - time
        - timeit
        - tkinter
        - traceback
        - types
        - unicodedata
        - unittest
        - unittest.mock
        - urllib
        - wsgiref
        - xmlrpc
        - xml.sax
        - zipfile
    - Other module-level changes
    - Otimizações
    - Mudanças a construções e API C
    - Descontinuados
        - New Keywords
        - Comportamento do Python descontinuado
        - Unsupported Operating Systems
        - Deprecated Python modules, functions and methods
    - Removidos
        - Remoção de APIs e recursos
    - Porting to Python 3.5
        - Alterações no comportamento do Python
        - Alterações na API Python
        - Alterações na API C
    - Notable changes in Python 3.5.4
        - Novo alvo de construção `make regen-all`
        - Remoção do alvo de construção `make touch`
- O que há de novo no Python 3.4
    - Resumo – Destaques da versão
    - Novas funcionalidades

    - PEP 453: Explicit Bootstrapping of PIP in Python Installations
        - Bootstrapping pip By Default
        - Mudanças na documentação
    - PEP 446: Newly Created File Descriptors Are Non-Inheritable
    - Melhorias no manuseio de Codecs
    - PEP 451: A ModuleSpec Type for the Import System
    - Outras mudanças na linguagem
- Novos módulos
    - asyncio
    - ensurepip
    - enum
    - pathlib
    - selectors
    - statistics
    - tracemalloc
- Módulos melhorados
    - abc
    - aifc
    - argparse
    - audioop
    - base64
    - collections
    - colorsys
    - contextlib
    - dbm
    - dis
    - doctest
    - email
    - filecmp
    - functools
    - gc
    - glob
    - hashlib
    - hmac
    - html

- http
- idlelib e IDLE
- importlib
- inspect
- ipaddress
- logging
- marshal
- mmap
- multiprocessing
- operador
- os
- pdb
- pickle
- plistlib
- poplib
- pprint
- pty
- pydoc
- re
- resource
- select
- shelve
- shutil
- smtpd
- smtplib
- socket
- sqlite3
- ssl
- stat
- struct
- subprocess
- sunau
- sys
- tarfile
- textwrap
- threading
- traceback

        - types
        - urllib
        - unittest
        - venv
        - wave
        - weakref
        - xml.etree
        - zipfile
    - CPython Implementation Changes
        - PEP 445: Customization of CPython Memory Allocators
        - PEP 442: Safe Object Finalization
        - PEP 456: Secure and Interchangeable Hash Algorithm
        - PEP 436: Argument Clinic
        - Other Alterações a compilações e API C
        - Other Improvements
        - Otimizações Relevantes
    - Descontinuados
        - Deprecations in the Python API
        - Deprecated Features
    - Removidos
        - Operating Systems No Longer Supported
        - Remoção de APIs e recursos
        - Code Cleanups
    - Porting to Python 3.4
        - Changes in ‘python’ Command Behavior
        - Alterações na API Python
        - Alterações na API C
    - Alterado em 3.4.3
        - PEP 476: Habilitando verificação de certificado por padrão para clientes http stdlib
- O que há de novo no Python 3.3
    - Resumo – Destaques da versão
    - PEP 405: Ambientes Virtuais
    - PEP 420: Implicit Namespace Packages
    - PEP 3118: New memoryview implementation and buffer protocol documentation

    - Recursos
    - Mudanças na API
- PEP 393: Flexible String Representation
    - Funcionalidade
    - Desempenho e utilização de recursos
- PEP 397: Python Launcher for Windows
- PEP 3151: Reworking the OS and IO exception hierarchy
- PEP 380: Syntax for Delegating to a Subgenerator
- PEP 409: Suppressing exception context
- PEP 414: Explicit Unicode literals
- PEP 3155: Qualified name for classes and functions
- PEP 412: Key-Sharing Dictionary
- PEP 362: Function Signature Object
- PEP 421: Adding sys.implementation
    - SimpleNamespace
- Using importlib as the Implementation of Import
    - New APIs
    - Visible Changes
- Outras mudanças na linguagem
- A Finer-Grained Import Lock
- Builtin functions and types
- Novos módulos
    - faulthandler
    - ipaddress
    - lzma
- Módulos melhorados
    - abc
    - array
    - base64
    - binascii
    - bz2
    - codecs
    - collections
    - contextlib
    - crypt
    - curses
    - datetime

- decimal
  - Recursos
  - Mudanças na API
- email
  - Policy Framework
  - Provisional Policy with New Header API
  - Other API Changes
- ftplib
- functools
- gc
- hmac
- http
- html
- imaplib
- inspect
- io
- itertools
- logging
- math
- mmap
- multiprocessing
- nntplib
- os
- pdb
- pickle
- pydoc
- re
- sched
- select
- shlex
- shutil
- signal
- smtpd
- smtplib
- socket
- socketserver
- sqlite3

        - ssl
        - stat
        - struct
        - subprocess
        - sys
        - tarfile
        - tempfile
        - textwrap
        - threading
        - time
        - types
        - unittest
        - urllib
        - webbrowser
        - xml.etree.ElementTree
        - zlib
    - Otimizações
    - Mudanças a construções e API C
    - Descontinuados
        - Unsupported Operating Systems
        - Deprecated Python modules, functions and methods
        - Deprecated functions and types of the C API
        - Deprecated features
    - Porting to Python 3.3
        - Porting Python code
        - Porting C code
        - Building C extensions
        - Command Line Switch Changes
- O que há de novo no Python 3.2
    - PEP 384: Defining a Stable ABI
    - PEP 389: Argparse Command Line Parsing Module
    - PEP 391: Dictionary Based Configuration for Logging
    - PEP 3148: The `concurrent.futures` module
    - PEP 3147: PYC Repository Directories
    - PEP 3149: ABI Version Tagged .so Files
    - PEP 3333: Python Web Server Gateway Interface v1.0.1
    - Outras mudanças na linguagem

- New, Improved, and Deprecated Modules
  - email
  - elementtree
  - functools
  - itertools
  - collections
  - threading
  - datetime and time
  - math
  - abc
  - io
  - reprlib
  - logging
  - csv
  - contextlib
  - decimal and fractions
  - ftp
  - popen
  - select
  - gzip and zipfile
  - tarfile
  - hashlib
  - ast
  - os
  - shutil
  - sqlite3
  - html
  - socket
  - ssl
  - nntp
  - сертифікати
  - imaplib
  - http.client
  - unittest
  - random
  - poplib
  - asyncore

        - tempfile
        - inspect
        - pydoc
        - dis
        - dbm
        - ctypes
        - site
        - sysconfig
        - pdb
        - configparser
        - urllib.parse
        - mailbox
        - turtledemo
    - Multi-threading
    - Otimizações
    - Unicode
    - Codecs
    - Documentação
    - IDLE
    - Code Repository
    - Mudanças a construções e API C
    - Porting to Python 3.2
  - O que há de novo no Python 3.1
    - PEP 372: Dicionários ordenados
    - PEP 378: Especificador de formato para separador de milhares
    - Outras mudanças na linguagem
    - New, Improved, and Deprecated Modules
    - Otimizações
    - IDLE
    - Mudanças a construções e API C
    - Porting to Python 3.1
  - O que há de novo no Python 3.0
    - Obstáculos comuns
        - print é uma função
        - Visualizações e iteradores em vez de listas
        - Comparações de ordenação

        - Inteiros
        - Texto vs. Dados em vez de Unicode vs. 8 bits
    - Visão geral das alterações da sintaxe
        - Nova sintaxe
        - Sintaxe alterada
        - Sintaxe removida
    - Mudanças já presentes no Python 2.6
    - Mudanças na biblioteca
    - **PEP 3101**: Uma nova abordagem para formatação de strings
    - Mudanças para exceções
    - Outras mudanças diversas
        - Operadores e métodos especiais
        - Funções embutidas
    - Mudanças a construções e API C
    - Desempenho
    - Portando para o Python 3.0
- O que há de novo no Python 2.7
    - O futuro para o Python 2.x
    - Mudanças no tratamento de avisos de descontinuação
    - Recursos do Python 3.1
    - PEP 372: Adicionando um dicionário ordenado a coleções
    - PEP 378: Especificador de formato para separador de milhares
    - PEP 389: O módulo argparse para analisar linhas de comando
    - PEP 391: Configuração baseada em dicionário para logging
    - PEP 3106: Views de dicionário
    - PEP 3137: O objeto memoryview
    - Outras mudanças na linguagem
        - Alterações no interpretador
        - Otimizações
    - Módulos Novos ou Aprimorados
        - Novo módulo: importlib
        - Novo módulo: sysconfig
        - ttk: Widgets temáticos para Tk
        - Módulo atualizado: unittest
        - Módulo atualizado: ElementTree 1.3

- Mudanças a construções e API C
  - Capsules
  - Alterações específicas da porta: Windows
  - Alterações específicas da porta: Mac OS X
  - Alterações específicas da porta: FreeBSD
- Outras alterações e correções
- Portando para Python 2.7
- Novos recursos adicionados às versões de manutenção do Python 2.7
  - Duas novas variáveis de ambiente para modo de depuração
  - PEP 434: Exceção de aprimoramento do IDLE para todos os branches
  - PEP 466: Aprimoramentos de segurança de rede do Python 2.7
  - PEP 477: Backport do ensurepip (PEP 453) para o Python 2.7
    - Inicializando pip por padrão
    - Mudanças na documentação
  - PEP 476: Habilitando verificação de certificado por padrão para clientes http stdlib
  - PEP 493: Ferramentas de migração de verificação HTTPS para Python 2.7
  - Novo alvo de construção `make regen-all`
  - Remoção do alvo de construção `make touch`
- Reconhecimentos

- O que há de novo no Python 2.6
  - Python 3.0
  - Changes to the Development Process
    - New Issue Tracker: Roundup
    - New Documentation Format: reStructuredText Using Sphinx
  - PEP 343: The ‘with’ statement
    - Writing Context Managers
    - The contextlib module
  - PEP 366: Explicit Relative Imports From a Main Module
  - PEP 370: Per-user `site-packages` Directory

- PEP 371: The `multiprocessing` Package
- PEP 3101: Advanced String Formatting
- PEP 3105: `print` As a Function
- PEP 3110: Exception-Handling Changes
- PEP 3112: Byte Literals
- PEP 3116: New I/O Library
- PEP 3118: Revised Buffer Protocol
- PEP 3119: Abstract Base Classes
- PEP 3127: Integer Literal Support and Syntax
- PEP 3129: Class Decorators
- PEP 3141: A Type Hierarchy for Numbers
  - The `fractions` Module
- Outras mudanças na linguagem
  - Otimizações
  - Alterações no interpretador
- Módulos novos ou aprimorados
  - The `ast` module
  - The `future_builtins` module
  - The `json` module: JavaScript Object Notation
  - The `plistlib` module: A Property-List Parser
  - ctypes Enhancements
  - Improved SSL Support
- Deprecations and Removals
- Mudanças a construções e API C
  - Port-Specific Changes: Windows
  - Port-Specific Changes: Mac OS X
  - Port-Specific Changes: IRIX
- Porting to Python 2.6
- Reconhecimentos

- What's New in Python 2.5
  - PEP 308: Conditional Expressions
  - PEP 309: Partial Function Application
  - PEP 314: Metadata for Python Software Packages v1.1
  - PEP 328: Absolute and Relative Imports
  - PEP 338: Executing Modules as Scripts
  - PEP 341: Unified try/except/finally
  - PEP 342: New Generator Features

- PEP 343: The ‘with’ statement
  - Writing Context Managers
  - The contextlib module
- PEP 352: Exceptions as New-Style Classes
- PEP 353: Using ssize_t as the index type
- PEP 357: The ‘__index__’ method
- Outras mudanças na linguagem
  - Interactive Interpreter Changes
  - Otimizações
- New, Improved, and Removed Modules
  - The ctypes package
  - The ElementTree package
  - The hashlib package
  - The sqlite3 package
  - The wsgiref package
- Mudanças a construções e API C
  - Port-Specific Changes
- Porting to Python 2.5
- Reconhecimentos

- What’s New in Python 2.4
  - PEP 218: Built-In Set Objects
  - PEP 237: Unifying Long Integers and Integers
  - PEP 289: Generator Expressions
  - PEP 292: Simpler String Substitutions
  - PEP 318: Decorators for Functions and Methods
  - PEP 322: Reverse Iteration
  - PEP 324: New subprocess Module
  - PEP 327: Decimal Data Type
    - Why is Decimal needed?
    - The `Decimal` type
    - The `Context` type
  - PEP 328: Multi-line Imports
  - PEP 331: Locale-Independent Float/String Conversions
  - Outras mudanças na linguagem
    - Otimizações
  - New, Improved, and Deprecated Modules
    - cookielib

        - doctest
    - Mudanças a construções e API C
        - Port-Specific Changes
    - Porting to Python 2.4
    - Reconhecimentos
- What's New in Python 2.3
    - PEP 218: A Standard Set Datatype
    - PEP 255: Simple Generators
    - PEP 263: Source Code Encodings
    - PEP 273: Importing Modules from ZIP Archives
    - PEP 277: Unicode file name support for Windows NT
    - PEP 278: Universal Newline Support
    - PEP 279: enumerate()
    - PEP 282: The logging Package
    - PEP 285: A Boolean Type
    - PEP 293: Codec Error Handling Callbacks
    - PEP 301: Package Index and Metadata for Distutils
    - PEP 302: New Import Hooks
    - PEP 305: Comma-separated Files
    - PEP 307: Pickle Enhancements
    - Extended Slices
    - Outras mudanças na linguagem
        - String Changes
        - Otimizações
    - New, Improved, and Deprecated Modules
        - Date/Time Type
        - The optparse Module
    - Pymalloc: A Specialized Object Allocator
    - Mudanças a construções e API C
        - Port-Specific Changes
    - Outras alterações e correções
    - Porting to Python 2.3
    - Reconhecimentos
- What's New in Python 2.2
    - Introdução
    - PEPs 252 and 253: Type and Class Changes
        - Old and New Classes

        - Descriptors
        - Multiple Inheritance: The Diamond Rule
        - Attribute Access
        - Related Links
    - PEP 234: Iterators
    - PEP 255: Simple Generators
    - PEP 237: Unifying Long Integers and Integers
    - PEP 238: Changing the Division Operator
    - Unicode Changes
    - PEP 227: Nested Scopes
    - Módulos novos ou aprimorados
    - Interpreter Changes and Fixes
    - Outras alterações e correções
    - Reconhecimentos
- What's New in Python 2.1
    - Introdução
    - PEP 227: Nested Scopes
    - PEP 236: __future__ Directives
    - PEP 207: Rich Comparisons
    - PEP 230: Warning Framework
    - PEP 229: New Build System
    - PEP 205: Weak References
    - PEP 232: Function Attributes
    - PEP 235: Importing Modules on Case-Insensitive Platforms
    - PEP 217: Interactive Display Hook
    - PEP 208: New Coercion Model
    - PEP 241: Metadata in Python Packages
    - Módulos novos ou aprimorados
    - Outras alterações e correções
    - Reconhecimentos
- What's New in Python 2.0
    - Introdução
    - What About Python 1.6?
    - New Development Process
    - Unicode
    - Compreensões de lista
    - Augmented Assignment

- String Methods
- Garbage Collection of Cycles
- Other Core Changes
  - Minor Language Changes
  - Changes to Built-in Functions
- Porting to 2.0
- Extending/Embedding Changes
- Distutils: Making Modules Easy to Install
- XML Modules
  - SAX2 Support
  - DOM Support
  - Relationship to PyXML
- Module changes
- Novos módulos
- IDLE Improvements
- Deleted and Deprecated Modules
- Reconhecimentos

- Changelog
  - Python seguinte
    - Núcleo e embutidos
    - Biblioteca
    - Testes
    - Construção
    - Windows
    - IDLE
    - Ferramentas/Demos
    - API C
  - Python 3.15.0 final
  - Python 3.15.0 candidato a lançamento 3
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS

    - IDLE
    - API C
- Python 3.15.0 candidato a lançamento 2
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.15.0 candidato a lançamento 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - Ferramentas/Demos
    - API C
- Python 3.15.0 beta 4
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.15.0 beta 3
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes

    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.15.0 beta 2
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Testes
    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.15.0 beta 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.15.0 alpha 8
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C

- Python 3.15.0 alpha 7
  - Windows
  - Testes
  - Segurança
  - Biblioteca
  - Documentação
  - Núcleo e embutidos
  - API C
  - Construção
- Python 3.15.0 alpha 6
  - macOS
  - Windows
  - Testes
  - Segurança
  - Biblioteca
  - IDLE
  - Núcleo e embutidos
  - API C
  - Construção
- Python 3.15.0 alpha 5
  - Windows
  - Ferramentas/Demos
  - Testes
  - Biblioteca
  - Documentação
  - Núcleo e embutidos
  - API C
- Python 3.15.0 alpha 4
  - Testes
  - Biblioteca
  - Núcleo e embutidos
- Python 3.15.0 alpha 3
  - Ferramentas/Demos
  - Testes
  - Segurança
  - Biblioteca
  - Documentação

    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.15.0 alpha 2
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.15.0 alpha 1
    - macOS
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - IDLE
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca

    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.14.0 beta 1
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - IDLE
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca

    - API C
    - Construção
- Python 3.14.0 alpha 7
    - macOS
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.14.0 alpha 6
    - macOS
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.14.0 alpha 5
    - macOS
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - IDLE

    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - API C
    - Construção
- Python 3.14.0 alpha 4
    - macOS
    - Ferramentas/Demos
    - Testes
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.14.0 alpha 3
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção
- Python 3.14.0 alpha 2
    - Windows
    - Ferramentas/Demos
    - Testes

    - Segurança
    - Biblioteca
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - API C
    - Construção
- Python 3.14.0 alpha 1
    - macOS
    - Windows
    - Ferramentas/Demos
    - Testes
    - Segurança
    - Biblioteca
    - IDLE
    - Documentação
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - API C
    - Construção

- Python 3.13.0 beta 1
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Construção
  - Windows
  - macOS
  - IDLE
  - API C
- Python 3.13.0 alpha 6
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
  - Windows
  - API C
- Python 3.13.0 alpha 5
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
  - Windows
  - macOS
  - IDLE
  - Ferramentas/Demos

    - API C
- Python 3.13.0 alpha 4
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.13.0 alpha 3
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.13.0 alpha 2
    - Núcleo e embutidos
    - Biblioteca
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.13.0 alpha 1
    - Segurança
    - Núcleo e embutidos

    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.12.0 beta 1
    - Segurança
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.12.0 alpha 7
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Construção
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - Ferramentas/Demos
    - API C

- Python 3.12.0 alpha 6
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
  - Windows
  - macOS
  - API C
- Python 3.12.0 alpha 5
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
  - Windows
- Python 3.12.0 alpha 4
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
  - Windows
  - macOS
  - Ferramentas/Demos
  - API C
- Python 3.12.0 alpha 3
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - Núcleo e embutidos

    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.12.0 alpha 2
    - Segurança
    - Núcleo e embutidos
    - Construção
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - API C
- Python 3.12.0 alpha 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos

    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.11.0 beta 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Construção
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.11.0 alpha 7
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes

    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.11.0 alpha 6
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - IDLE
    - API C
- Python 3.11.0 alpha 5
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Construção
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.11.0 alpha 4
    - Núcleo e embutidos

    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - API C
- Python 3.11.0 alpha 3
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - API C
- Python 3.11.0 alpha 2
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.11.0 alpha 1
    - Segurança
    - Núcleo e embutidos

    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.10.0 beta 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes

    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.10.0 alpha 7
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - IDLE
    - API C
- Python 3.10.0 alpha 6
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.10.0 alpha 5
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos

    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.10.0 alpha 4
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.10.0 alpha 3
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE

    - Ferramentas/Demos
    - API C
- Python 3.10.0 alpha 2
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.10.0 alpha 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca

    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.9.0 beta 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - Ferramentas/Demos
    - API C
- Python 3.9.0 alpha 6
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS

    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.9.0 alpha 5
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.9.0 alpha 4
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - IDLE
    - API C
- Python 3.9.0 alpha 3
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Construção
    - IDLE
    - API C
- Python 3.9.0 alpha 2
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação

    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - API C
- Python 3.9.0 alpha 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.8.0 beta 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.8.0 alpha 4
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção

    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.8.0 alpha 3
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.8.0 alpha 2
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Windows
    - IDLE
- Python 3.8.0 alpha 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.7.0 final
    - Biblioteca

    - API C
- Python 3.7.0 release candidate 1
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Construção
    - Windows
    - IDLE
- Python 3.7.0 beta 5
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - macOS
    - IDLE
- Python 3.7.0 beta 4
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
- Python 3.7.0 beta 3
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos

    - API C
- Python 3.7.0 beta 2
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
- Python 3.7.0 beta 1
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - API C
- Python 3.7.0 alpha 4
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Windows
    - Ferramentas/Demos
    - API C
- Python 3.7.0 alpha 3
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS

    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.7.0 alpha 2
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Construção
    - IDLE
    - API C
- Python 3.7.0 alpha 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.6.6 final
- Python 3.6.6 release candidate 1
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.6.5 final
    - Testes
    - Construção
- Python 3.6.5 release candidate 1

    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.6.4 final
- Python 3.6.4 release candidate 1
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - macOS
    - IDLE
    - Ferramentas/Demos
    - API C
- Python 3.6.3 final
    - Biblioteca
    - Construção
- Python 3.6.3 release candidate 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - Documentação
    - Testes
    - Construção
    - Windows
    - IDLE
    - Ferramentas/Demos
- Python 3.6.2 final

- Python 3.6.2 release candidate 2
  - Segurança
- Python 3.6.2 release candidate 1
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - API C
  - Construção
  - Documentação
  - Ferramentas/Demos
  - Testes
  - Windows
- Python 3.6.1 final
  - Núcleo e embutidos
  - Construção
- Python 3.6.1 release candidate 1
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - Windows
  - API C
  - Documentação
  - Testes
  - Construção
- Python 3.6.0 final
- Python 3.6.0 release candidate 2
  - Núcleo e embutidos
  - Ferramentas/Demos
  - Windows
  - Construção
- Python 3.6.0 release candidate 1
  - Núcleo e embutidos
  - Biblioteca
  - API C
  - Documentação
  - Ferramentas/Demos

- Python 3.6.0 beta 4
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
- Python 3.6.0 beta 3
  - Núcleo e embutidos
  - Biblioteca
  - Windows
  - Construção
  - Testes
- Python 3.6.0 beta 2
  - Núcleo e embutidos
  - Biblioteca
  - Windows
  - API C
  - Construção
  - Testes
- Python 3.6.0 beta 1
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - API C
  - Testes
  - Construção
  - Ferramentas/Demos
  - Windows
- Python 3.6.0 alpha 4
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - Testes
  - Windows
  - Construção
- Python 3.6.0 alpha 3
  - Segurança

    - Núcleo e embutidos
    - Biblioteca
    - IDLE
    - API C
    - Construção
    - Ferramentas/Demos
    - Documentação
    - Testes
- Python 3.6.0 alpha 2
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - IDLE
    - Documentação
    - Testes
    - Windows
    - Construção
    - API C
    - Ferramentas/Demos
- Python 3.6.0 alpha 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
    - IDLE
    - Documentação
    - Testes
    - Construção
    - Windows
    - Ferramentas/Demos
    - API C
- Python 3.5.5 final
- Python 3.5.5 release candidate 1
    - Segurança
    - Núcleo e embutidos
    - Biblioteca
- Python 3.5.4 final
    - Biblioteca

- Python 3.5.4 release candidate 1
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - Documentação
  - Testes
  - Construção
  - Windows
  - API C
- Python 3.5.3 final
- Python 3.5.3 release candidate 1
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - API C
  - Documentação
  - Testes
  - Ferramentas/Demos
  - Windows
  - Construção
- Python 3.5.2 final
  - Núcleo e embutidos
  - Testes
  - IDLE
- Python 3.5.2 release candidate 1
  - Segurança
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - Documentação
  - Testes
  - Construção
  - Windows
  - Ferramentas/Demos
- Python 3.5.1 final
  - Núcleo e embutidos

    - Windows
- Python 3.5.1 release candidate 1
    - Núcleo e embutidos
    - Biblioteca
    - IDLE
    - Documentação
    - Testes
    - Construção
    - Windows
    - Ferramentas/Demos
- Python 3.5.0 final
    - Construção
- Python 3.5.0 release candidate 4
    - Biblioteca
    - Construção
- Python 3.5.0 release candidate 3
    - Núcleo e embutidos
    - Biblioteca
- Python 3.5.0 release candidate 2
    - Núcleo e embutidos
    - Biblioteca
- Python 3.5.0 release candidate 1
    - Núcleo e embutidos
    - Biblioteca
    - IDLE
    - Documentação
    - Testes
- Python 3.5.0 beta 4
    - Núcleo e embutidos
    - Biblioteca
    - Construção
- Python 3.5.0 beta 3
    - Núcleo e embutidos
    - Biblioteca
    - Testes
    - Documentação
    - Construção

- Python 3.5.0 beta 2
  - Núcleo e embutidos
  - Biblioteca
- Python 3.5.0 beta 1
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - Testes
  - Documentação
  - Ferramentas/Demos
- Python 3.5.0 alpha 4
  - Núcleo e embutidos
  - Biblioteca
  - Construção
  - Testes
  - Ferramentas/Demos
  - API C
- Python 3.5.0 alpha 3
  - Núcleo e embutidos
  - Biblioteca
  - Construção
  - Testes
  - Ferramentas/Demos
- Python 3.5.0 alpha 2
  - Núcleo e embutidos
  - Biblioteca
  - Construção
  - API C
  - Windows
- Python 3.5.0 alpha 1
  - Núcleo e embutidos
  - Biblioteca
  - IDLE
  - Construção
  - API C
  - Documentação
  - Testes

        - Ferramentas/Demos
        - Windows
- O tutorial do Python
    - 1. Abrindo seu apetite
    - 2. Utilizando o interpretador Python
        - 2.1. Chamando o interpretador
            - 2.1.1. Passagem de argumentos
            - 2.1.2. Modo interativo
        - 2.2. O interpretador e seu ambiente
            - 2.2.1. Edição de código-fonte
    - 3. Uma introdução informal ao Python
        - 3.1. Usando Python como uma calculadora
            - 3.1.1. Números
            - 3.1.2. Texto
            - 3.1.3. Listas
        - 3.2. Primeiros passos para a programação
    - 4. Mais ferramentas de controle de fluxo
        - 4.1. Instruções `if`
        - 4.2. Instruções `for`
        - 4.3. A função `range()`
        - 4.4. Instruções `break` e `continue`
        - 4.5. Cláusulas `else` em laços
        - 4.6. Instruções `pass`
        - 4.7. Instruções `match`
        - 4.8. Definindo funções
        - 4.9. Mais sobre definição de funções
            - 4.9.1. Argumentos com valor padrão
            - 4.9.2. Argumentos nomeados
            - 4.9.3. Parâmetros especiais
                - 4.9.3.1. Argumentos posicional-ou-nomeados
                - 4.9.3.2. Parâmetros somente-posicionais
                - 4.9.3.3. Argumentos somente-nomeados
                - 4.9.3.4. Exemplos de funções
                - 4.9.3.5. Recapitulando
            - 4.9.4. Listas de argumentos arbitrárias
            - 4.9.5. Desempacotando listas de argumentos
            - 4.9.6. Expressões lambda

        - 4.9.7. Strings de documentação
        - 4.9.8. Anotações de função
    - 4.10. Intermezzo: estilo de codificação
- 5. Estruturas de dados
    - 5.1. Mais sobre listas
        - 5.1.1. Usando listas como pilhas
        - 5.1.2. Usando listas como filas
        - 5.1.3. Compreensões de lista
        - 5.1.4. Compreensões de lista aninhadas
        - 5.1.5. Desempacotamento em Listas e Compreensões de Lista
    - 5.2. A instrução `del`
    - 5.3. Tuplas e Sequências
    - 5.4. Conjuntos
    - 5.5. Dicionários
    - 5.6. Técnicas de iteração
    - 5.7. Mais sobre condições
    - 5.8. Comparando sequências e outros tipos
- 6. Módulos
    - 6.1. Mais sobre módulos
        - 6.1.1. Executando módulos como scripts
        - 6.1.2. O caminho de busca dos módulos
        - 6.1.3. Arquivos Python “compilados”
    - 6.2. Módulos padrões
    - 6.3. A função `dir()`
    - 6.4. Pacotes
        - 6.4.1. Importando * de um pacote
        - 6.4.2. Referências em um mesmo pacote
- 7. Entrada e saída
    - 7.1. Formatação de saída mais elaborada
        - 7.1.1. Literais de string formatados
        - 7.1.2. O método format() de strings
        - 7.1.3. Formatação manual de strings
        - 7.1.4. Formatação de strings à moda antiga
    - 7.2. Lendo e escrevendo arquivos
        - 7.2.1. Métodos de objetos arquivo
        - 7.2.2. Gravando dados estruturados com `json`

- 8. Erros e exceções
  - 8.1. Erros de sintaxe
  - 8.2. Exceções
  - 8.3. Tratamento de exceções
  - 8.4. Levantando exceções
  - 8.5. Encadeamento de exceções
  - 8.6. Exceções definidas pelo usuário
  - 8.7. Definindo ações de limpeza
  - 8.8. Ações de limpeza predefinidas
  - 8.9. Criando e tratando várias exceções não relacionadas
  - 8.10. Enriquecendo exceções com notas
- 9. Classes
  - 9.1. Uma palavra sobre nomes e objetos
  - 9.2. Escopos e espaços de nomes do Python
    - 9.2.1. Exemplo de escopos e espaço de nomes
  - 9.3. Uma primeira olhada nas classes
    - 9.3.1. Sintaxe da definição de classe
    - 9.3.2. Objetos classe
    - 9.3.3. Objetos instância
    - 9.3.4. Objetos método
    - 9.3.5. Variáveis de classe e instância
  - 9.4. Observações aleatórias
  - 9.5. Herança
    - 9.5.1. Herança múltipla
  - 9.6. Variáveis privadas
  - 9.7. Curiosidades e conclusões
  - 9.8. Iteradores
  - 9.9. Geradores
  - 9.10. Expressões geradoras
- 10. Um breve passeio pela biblioteca padrão
  - 10.1. Interface com o sistema operacional
  - 10.2. Caracteres curinga
  - 10.3. Argumentos de linha de comando
  - 10.4. Redirecionamento de erros e encerramento do programa
  - 10.5. Reconhecimento de padrões em strings
  - 10.6. Matemática

        - 10.7. Acesso à internet
        - 10.8. Data e hora
        - 10.9. Compressão de dados
        - 10.10. Medição de desempenho
        - 10.11. Controle de qualidade
        - 10.12. Baterias incluídas
    - 11. Um breve passeio pela biblioteca padrão — parte II
        - 11.1. Formatando a saída
        - 11.2. Usando templates
        - 11.3. Trabalhando com formatos binários de dados
        - 11.4. Multi-threading
        - 11.5. Gerando logs
        - 11.6. Referências fracas
        - 11.7. Ferramentas para trabalhar com listas
        - 11.8. Aritmética decimal com ponto flutuante
    - 12. Ambientes virtuais e pacotes
        - 12.1. Introdução
        - 12.2. Criando ambientes virtuais
        - 12.3. Gerenciando pacotes com o pip
    - 13. E agora?
    - 14. Edição de entrada interativa e substituição de histórico
        - 14.1. Tab completion e histórico de edição
        - 14.2. Alternativas ao interpretador interativo
    - 15. Aritmética de ponto flutuante: problemas e limitações
        - 15.1. Erro de representação
    - 16. Anexo
        - 16.1. Modo interativo
            - 16.1.1. Tratamento de erros
            - 16.1.2. Scripts Python executáveis
            - 16.1.3. Arquivo de inicialização do modo interativo
            - 16.1.4. Módulos de customização
- Configurações e Uso do Python
    - 1. Linha de comando e ambiente
        - 1.1. Linha de comando
            - 1.1.1. Opções de interface
            - 1.1.2. Opções genéricas
            - 1.1.3. Opções diversas

- 1.1.4. Controlando cores
- 1.2. Variáveis de ambiente
  - 1.2.1. Variáveis de modo de depuração
- 2. Utilizando Python em plataformas Unix
  - 2.1. Obtendo e instalando a versão mais recente do Python
    - 2.1.1. No Linux
      - 2.1.1.1. Instalação do IDLE
    - 2.1.2. No FreeBSD e OpenBSD
  - 2.2. Compilando o Python
  - 2.3. Paths e arquivos relacionados com o Python
  - 2.4. Diversos
  - 2.5. OpenSSL personalizado
- 3. Configurando o Python
  - 3.1. Requisitos de construção
    - 3.1.1. Requisitos para módulos opcionais
  - 3.2. Arquivos gerados
    - 3.2.1. Script configure
  - 3.3. Opções de configuração
    - 3.3.1. Opções gerais
    - 3.3.2. Opções do compilador C
    - 3.3.3. Opções da ligação
    - 3.3.4. Opções para dependências de terceiros
    - 3.3.5. Opções de WebAssembly
    - 3.3.6. Opções de instalação
    - 3.3.7. Opções de desempenho
    - 3.3.8. Compilação de depuração do Python
    - 3.3.9. Opções de depuração
    - 3.3.10. Opções da ligação
    - 3.3.11. Opções da biblioteca
    - 3.3.12. Opções de segurança
    - 3.3.13. Opções do macOS
    - 3.3.14. Opções do iOS
    - 3.3.15. Opções de compilação cruzada
  - 3.4. Sistema de Construção Python
    - 3.4.1. Arquivos principais do sistema de construção
    - 3.4.2. Principais etapas de construção
    - 3.4.3. Alvos principais do Makefile

- 3.4.3.1. make
- 3.4.3.2. make platform
- 3.4.3.3. make profile-opt
- 3.4.3.4. make clean
- 3.4.3.5. make distclean
- 3.4.3.6. make install
- 3.4.3.7. make test
- 3.4.3.8. make ci
- 3.4.3.9. make buildbottest
- 3.4.3.10. make regen-all
- 3.4.4. Extensões C
- 3.5. Sinalizadores do compilador e do vinculador
  - 3.5.1. Sinalizadores do pré-processador
  - 3.5.2. Sinalizadores do compilador
  - 3.5.3. Sinalizadores do vinculador
- 4. Utilizando Python no Windows
  - 4.1. Gerenciador de instalação do Python
    - 4.1.1. Instalação
    - 4.1.2. Uso básico
    - 4.1.3. Comando help
    - 4.1.4. Listando ambientes de execução
    - 4.1.5. Instalando ambientes de execução
    - 4.1.6. Instalações offline
    - 4.1.7. Desinstalando ambientes de execução
    - 4.1.8. Configuração
    - 4.1.9. Linhas shebang
    - 4.1.10. Instalação avançada
    - 4.1.11. Configuração administrativa
    - 4.1.12. Instalando binários com threads livres
    - 4.1.13. Index signatures
    - 4.1.14. Proxy settings
    - 4.1.15. Solução de problemas
  - 4.2. O pacote embutível
    - 4.2.1. Aplicação Python
    - 4.2.2. Incorporando Python
  - 4.3. Os pacotes nuget.org
    - 4.3.1. Pacotes com threads livres

- 4.4. Pacotes Alternativos
- 4.5. Versões do Windows suportadas
- 4.6. Removendo a limitação de MAX_PATH
- 4.7. Modo UTF-8
- 4.8. Encontrando módulos
- 4.9. Módulos adicionais
  - 4.9.1. PyWin32
  - 4.9.2. cx_Freeze
- 4.10. Compilando Python no Windows
- 4.11. O instalador completo (descontinuado)
  - 4.11.1. Etapas de instalação
  - 4.11.2. Removendo a limitação de MAX_PATH
  - 4.11.3. Instalando sem UI
  - 4.11.4. Instalando sem download
  - 4.11.5. Modificando uma instalação
  - 4.11.6. Instalando binários com threads livres
- 4.12. Inicializador Python para Windows (descontinuado)
  - 4.12.1. Começando
    - 4.12.1.1. Pela linha de comando
    - 4.12.1.2. Ambientes virtuais
    - 4.12.1.3. Por um script
    - 4.12.1.4. Por associação de arquivos
  - 4.12.2. Linhas shebang
  - 4.12.3. Argumentos em linhas shebang
  - 4.12.4. Personalização
    - 4.12.4.1. Personalização via arquivos INI
    - 4.12.4.2. Personalizando versões padrão do Python
  - 4.12.5. Diagnósticos
  - 4.12.6. Teste de simulação
  - 4.12.7. Instalar sob demanda
  - 4.12.8. Códigos de retorno
- 5. Usando Python em macOS
  - 5.1. Usando Python para macOS do `python.org`
    - 5.1.1. Etapas de instalação
    - 5.1.2. Como executar um script Python
  - 5.2. Distribuições alternativas
  - 5.3. Instalar pacotes Python adicionais

        - 5.4. Programação de GUI
        - 5.5. Tópicos avançados
            - 5.5.1. Instalando binários com threads livres
            - 5.5.2. Instalação usando a linha de comando
            - 5.5.3. Distribuindo aplicações Python
            - 5.5.4. Conformidade com a App Store
        - 5.6. Outros recursos
    - 6. Utilizando Python no Android
        - 6.1. Adicionando Python a uma aplicação Android
        - 6.2. Construindo um pacote Python para Android
    - 7. Usando Python no iOS
        - 7.1. Python em tempo de execução no iOS
            - 7.1.1. Compatibilidade com a versão do iOS
            - 7.1.2. Identificação da plataforma
            - 7.1.3. Disponibilidade da biblioteca padrão
            - 7.1.4. Módulos de extensão binária
            - 7.1.5. Binários stub do compilador
        - 7.2. Instalando Python no iOS
            - 7.2.1. Ferramentas para construir aplicações de iOS
            - 7.2.2. Adicionando Python a um projeto iOS
            - 7.2.3. Testando um pacote Python
        - 7.3. Conformidade com a App Store
            - 7.3.1. Código incompatível na biblioteca padrão
            - 7.3.2. Manifestos de privacidade
    - 8. Editores e IDEs
        - 8.1. IDLE — editor e console Python
        - 8.2. Outros editores e IDEs
- A Referência da Linguagem Python
    - 1. Introdução
        - 1.1. Implementações Alternativas
        - 1.2. Notação
            - 1.2.1. Definições lexicais e sintáticas
    - 2. Análise léxica
        - 2.1. Estrutura das linhas
            - 2.1.1. Linhas lógicas
            - 2.1.2. Linhas físicas
            - 2.1.3. Comentários

- 2.1.4. Declarações de codificação
- 2.1.5. Junção de linha explícita
- 2.1.6. Junção de linha implícita
- 2.1.7. Linhas em branco
- 2.1.8. Indentação
- 2.1.9. Espaços em branco entre tokens
- 2.1.10. Marcador de fim
- 2.2. Outros tokens
- 2.3. Nomes (identificadores e palavras reservadas)
  - 2.3.1. Palavras reservadas
  - 2.3.2. Palavras reservadas contextuais
  - 2.3.3. Classes reservadas de identificadores
  - 2.3.4. Caracteres não ASCII em nomes
- 2.4. Literais
- 2.5. Literais de string e bytes
  - 2.5.1. Strings entre aspas triplas
  - 2.5.2. Prefixos de string
  - 2.5.3. Gramática formal
  - 2.5.4. Sequências de escape
    - 2.5.4.1. Fim de linha ignorado
    - 2.5.4.2. Caracteres escapados
    - 2.5.4.3. Caractere octal
    - 2.5.4.4. Caractere hexadecimal
    - 2.5.4.5. Caractere nomeado Unicode
    - 2.5.4.6. Caracteres hexadecimais Unicode
    - 2.5.4.7. Sequências de espace não reconhecidas
  - 2.5.5. Literais de bytes
  - 2.5.6. Literais de string brutas
  - 2.5.7. Literais de strings formatadas
  - 2.5.8. Literais de string template (t-strings)
  - 2.5.9. Gramática formal para f-strings
- 2.6. Literais numéricos
  - 2.6.1. Inteiros literais
  - 2.6.2. Literais de ponto flutuante
  - 2.6.3. Literais imaginários
- 2.7. Operadores e delimitadores

- 3. Modelo de dados

- 3.1. Objetos, valores e tipos
- 3.2. A hierarquia de tipos padrão
  - 3.2.1. None
  - 3.2.2. NotImplemented
  - 3.2.3. Ellipsis
  - 3.2.4. `numbers.Number`
    - 3.2.4.1. `numbers.Integral`
    - 3.2.4.2. `numbers.Real` (`float`)
    - 3.2.4.3. `numbers.Complex` (`complex`)
  - 3.2.5. Sequências
    - 3.2.5.1. Sequências imutáveis
    - 3.2.5.2. Sequências mutáveis
  - 3.2.6. Tipos de conjuntos
  - 3.2.7. Mapeamentos
    - 3.2.7.1. Dicionários
    - 3.2.7.2. Dicionários congelados
  - 3.2.8. Tipos chamáveis
    - 3.2.8.1. Funções definidas pelo usuário
      - 3.2.8.1.1. Atributos especiais de somente leitura
      - 3.2.8.1.2. Atributos especiais graváveis
    - 3.2.8.2. Métodos de instância
    - 3.2.8.3. Funções geradoras
    - 3.2.8.4. Funções de corrotina
    - 3.2.8.5. Funções geradoras assíncronas
    - 3.2.8.6. Funções embutidas
    - 3.2.8.7. Métodos embutidos
    - 3.2.8.8. Classes
    - 3.2.8.9. Instâncias de classe
  - 3.2.9. Módulos
    - 3.2.9.1. Atributos relacionados à importação em objetos de módulo
    - 3.2.9.2. Outros atributos graváveis em objetos de módulo
    - 3.2.9.3. Dicionários do módulo
  - 3.2.10. Classes personalizadas
    - 3.2.10.1. Atributos especiais

- 3.2.10.2. Métodos especiais
- 3.2.11. Instâncias de classe
  - 3.2.11.1. Atributos especiais
- 3.2.12. Objetos de E/S (também conhecidos como objetos arquivo)
- 3.2.13. Tipos internos
  - 3.2.13.1. Objetos código
    - 3.2.13.1.1. Atributos especiais de somente leitura
    - 3.2.13.1.2. Métodos de objetos código
  - 3.2.13.2. Objetos quadro
    - 3.2.13.2.1. Atributos especiais de somente leitura
    - 3.2.13.2.2. Atributos especiais graváveis
    - 3.2.13.2.3. Métodos de objetos quadro
  - 3.2.13.3. Objetos traceback
  - 3.2.13.4. Objetos slice
  - 3.2.13.5. Objetos método estático
  - 3.2.13.6. Objetos método de classe
- 3.3. Nomes de métodos especiais
  - 3.3.1. Personalização básica
  - 3.3.2. Personalizando o acesso aos atributos
    - 3.3.2.1. Personalizando acesso a atributos de módulos
    - 3.3.2.2. Implementando descritores
    - 3.3.2.3. Invocando descritores
    - 3.3.2.4. __slots__
  - 3.3.3. Personalizando a criação de classe
    - 3.3.3.1. Metaclasses
    - 3.3.3.2. Resolvendo entradas de MRO
    - 3.3.3.3. Determinando a metaclasse apropriada
    - 3.3.3.4. Preparando o espaço de nomes da classe
    - 3.3.3.5. Executando o corpo da classe
    - 3.3.3.6. Criando o objeto classe
    - 3.3.3.7. Usos para metaclasses
  - 3.3.4. Personalizando verificações de instância e subclasse

- 3.3.5. Emulando tipos genéricos
  - 3.3.5.1. O propósito de *__class_getitem__*
  - 3.3.5.2. *__class_getitem__* versus *__getitem__*
- 3.3.6. Emulando objetos chamáveis
- 3.3.7. Emulando tipos contêineres
- 3.3.8. Emulando tipos numéricos
- 3.3.9. Gerenciadores de contexto da instrução with
- 3.3.10. Customizando argumentos posicionais na classe correspondência de padrão
- 3.3.11. Emulando tipos buffer
- 3.3.12. Anotações
- 3.3.13. Pesquisa de método especial
- 3.4. Corrotinas
  - 3.4.1. Objetos aguardáveis
  - 3.4.2. Objetos corrotina
  - 3.4.3. Iteradores assíncronos
  - 3.4.4. Gerenciadores de contexto assíncronos

4. Modelo de execução
- 4.1. Estrutura de um programa
- 4.2. Nomeação e ligação
  - 4.2.1. Ligação de nomes
  - 4.2.2. Resolução de nomes
  - 4.2.3. Escopos de anotação
  - 4.2.4. Avaliação preguiçosa
  - 4.2.5. Builtins e execução restrita
  - 4.2.6. Interação com recursos dinâmicos
- 4.3. Exceções
- 4.4. Componentes do Ambiente de Execução
  - 4.4.1. Modelo Geral de Computação
  - 4.4.2. Modelo do Ambiente de Execução Python

5. O sistema de importação
- 5.1. `importlib`
- 5.2. Pacotes
  - 5.2.1. Pacotes regulares
  - 5.2.2. Pacotes de espaço de nomes
- 5.3. Caminho de busca
  - 5.3.1. O cache de módulos

- 5.3.2. Localizadores e carregadores
- 5.3.3. Ganchos de importação
- 5.3.4. O metacaminho
- 5.4. Carregando
  - 5.4.1. Carregadores
  - 5.4.2. Submódulos
  - 5.4.3. Especificações de módulo
  - 5.4.4. Atributo __path__ dos módulos
  - 5.4.5. Representações do módulo
  - 5.4.6. Invalidação de bytecode em cache
- 5.5. O localizador baseado no caminho
  - 5.5.1. Localizadores de entrada de caminho
  - 5.5.2. Protocolo do localizador de entrada de caminho
- 5.6. Substituindo o sistema de importação padrão
- 5.7. Importações relativas ao pacote
- 5.8. Considerações especiais para __main__
  - 5.8.1. __main__.__spec__
- 5.9. Referências

- 6. Expressões
  - 6.1. Conversões aritméticas
  - 6.2. Átomos
    - 6.2.1. Constantes embutidas
    - 6.2.2. Identificadores (Nomes)
      - 6.2.2.1. Desfiguração de nome privado
    - 6.2.3. Literais
      - 6.2.3.1. Identidade de objeto e literais
      - 6.2.3.2. Concatenação de literal de string
    - 6.2.4. Grupos de parênteses
    - 6.2.5. Exibições de contêiner
      - 6.2.5.1. Sintaxes de criação de lista
      - 6.2.5.2. Sintaxes de criação de conjunto
      - 6.2.5.3. Tuple displays
      - 6.2.5.4. Sintaxes de criação de dicionário
    - 6.2.6. Compreensões
      - 6.2.6.1. Filtering in comprehensions
      - 6.2.6.2. Complex comprehensions
      - 6.2.6.3. Unpacking in comprehensions

- 6.2.6.4. Asynchronous comprehensions
- 6.2.6.5. Gramática formal para compreensões
- 6.2.7. Expressões geradoras
- 6.2.8. Expressões yield
- 6.2.8.1. Métodos de iterador gerador
- 6.2.8.2. Exemplos
- 6.2.8.3. Funções geradoras assíncronas
- 6.2.8.4. Métodos geradores-iteradores assíncronos
- 6.3. Primárias
- 6.3.1. Referências de atributo
- 6.3.2. Subscriptions and slicings
- 6.3.2.1. Fatiamentos
- 6.3.2.2. Comma-separated subscripts
- 6.3.2.3. “Starred” subscriptions
- 6.3.2.4. Formal subscription grammar
- 6.3.3. Chamadas
- 6.4. Expressão await
- 6.5. O operador de potência
- 6.6. Operações aritméticas unárias e bit a bit
- 6.7. Operações binárias aritméticas
- 6.8. Operações de deslocamento
- 6.9. Operações binárias bit a bit
- 6.10. Comparações
- 6.10.1. Comparações de valor
- 6.10.2. Operações de teste de pertinência
- 6.10.3. Comparações de identidade
- 6.11. Operações booleanas
- 6.12. Expressões de atribuição
- 6.13. Expressões condicionais
- 6.14. Lambdas
- 6.15. Listas de expressões
- 6.15.1. Iterable unpacking
- 6.16. Ordem de avaliação
- 6.17. Precedência de operadores
- 7. Instruções simples
- 7.1. Instruções de expressão
- 7.2. Instruções de atribuição

- 7.2.1. Instruções de atribuição aumentada
- 7.2.2. instruções de atribuição anotado
- 7.3. A instrução `assert`
- 7.4. A instrução `pass`
- 7.5. A instrução `del`
- 7.6. A instrução `return`
- 7.7. A instrução `yield`
- 7.8. A instrução `raise`
- 7.9. A instrução `break`
- 7.10. A instrução `continue`
- 7.11. A instrução `import`
  - 7.11.1. Lazy imports
    - 7.11.1.1. Compatibility via `__lazy_modules__`
  - 7.11.2. Instruções future
- 7.12. A instrução `global`
- 7.13. A instrução `nonlocal`
- 7.14. A instrução `type`

8. Instruções compostas
- 8.1. A instrução `if`
- 8.2. A instrução `while`
- 8.3. A instrução `for`
- 8.4. A instrução `try`
  - 8.4.1. Cláusula `except`
  - 8.4.2. Cláusula `except*`
  - 8.4.3. Cláusula `else`
  - 8.4.4. Cláusula `finally`
- 8.5. A instrução `with`
- 8.6. A instrução `match`
  - 8.6.1. Visão Geral
  - 8.6.2. Guards
  - 8.6.3. Blocos irrefutáveis de case
  - 8.6.4. Padrões
    - 8.6.4.1. Padrões OR
    - 8.6.4.2. Padrões AS
    - 8.6.4.3. Padrões literais
    - 8.6.4.4. Padrões de captura
    - 8.6.4.5. Padrões curingas

- 8.6.4.6. Padrões de valor
- 8.6.4.7. Padrões de grupo
- 8.6.4.8. Padrões de sequência
- 8.6.4.9. Padrões de mapeamento
- 8.6.4.10. Padrões de classe
- 8.7. Definições de função
- 8.8. Definições de classe
  - 8.8.1. Herança múltipla
- 8.9. Corrotinas
  - 8.9.1. Definição de função de corrotina
  - 8.9.2. A instrução `async for`
  - 8.9.3. A instrução `async with`
- 8.10. Listas de parâmetros de tipo
  - 8.10.1. Funções genéricas
  - 8.10.2. Classes genéricas
  - 8.10.3. Apelidos de tipo genérico
- 8.11. Anotações
- 9. Componentes de Alto Nível
  - 9.1. Programas Python completos
  - 9.2. Entrada de arquivo
  - 9.3. Entrada interativa
  - 9.4. Entrada de expressão
- 10. Especificação completa da gramática
- Referência da bibliotecas embutidas do Python
  - Tipos embutidos
    - Teste do valor verdade
    - Operações booleanas — `and`, `or`, `not`
    - Comparações
    - Tipos numéricos — `int`, `float`, `complex`
      - Operações de bits em tipos inteiros
      - Métodos adicionais em tipos inteiros
      - Métodos adicionais em ponto flutuante
      - Métodos adicionais em números complexos
      - Hashing de tipos numéricos
    - Tipo booleano - `bool`
    - Tipos iteradores
      - Tipos geradores

- Tipos sequências — `list`, `tuple`, `range`
  - Operações comuns de sequências
  - Tipos sequência imutáveis
  - Tipos sequências mutáveis
  - Listas
  - Tuplas
  - Intervalos
- Resumo dos métodos de tipo de sequência binária e de texto
- Tipo sequência de texto — `str`
  - String Methods
  - Strings literais formatadas (f-strings)
    - Especificador de depuração
    - Especificador de conversão
    - Especificador de formatação
  - literais de strings templates (t-strings)
  - Formatação de string no estilo `printf`
- Tipos de sequência binária — `bytes`, `bytearray`, `memoryview`
  - Objetos bytes
  - Objetos Bytearray
  - Operações com Bytes e Bytearray
  - Estilo de Formatação de bytes `printf`-style
  - Visões de memória
- Tipo conjuntos — `set`, `frozenset`
- Tipos mapeamentos — `dict`, `frozendict`
  - Objetos de visão de dicionário
  - Dicionários congelados
- Tipos de Gerenciador de Contexto
- Tipos de anotação de tipo — Apelido genérico, União
  - Tipo Generic Alias
    - Classes genéricas padrão
    - Atributos especiais de objetos `GenericAlias`
  - Tipo União
- Outros tipos embutidos
  - Módulos
  - Classes e Instâncias de Classes
  - Funções

        - Métodos
        - Objetos código
        - Objetos tipo
        - O objeto nulo
        - O Objeto Ellipsis
        - O Objeto NotImplemented
        - Objetos Internos
    - Atributos Especiais
    - Limitação de comprimento de string na conversão para inteiro
        - APIs afetadas
        - Configurando o limiter
        - Configuração recomendada
- Constantes embutidas
    - Constantes adicionadas pelo módulo `site`
- Funções embutidas
- Exceções embutidas
    - Contexto da exceção
    - Herdando de exceções embutidas
    - Classes base
    - Exceções concretas
        - Exceções de sistema operacional
    - Avisos
    - Grupos de exceções
    - Hierarquia das exceções
- Garantias de segurança de thread
    - Níveis de segurança de threads
        - Incompatível
        - Compatível
        - Seguro em objetos distintos
        - Seguro em objetos compartilhados
        - Atômica
    - Segurança de threads para objetos lista
    - Segurança de threads para objetos dict
    - Segurança de thread para objetos conjunto
    - Segurança para thread para objetos bytearray
    - Segurança para thread para objetos memoryview

- Complexidade de tempo de operações em tipos embutidos
  - `list`
  - `tuple`
  - `dict`, `frozendict`
  - `set`, `frozenset`
  - `str`, `bytes`, `bytearray`
  - `memoryview`
  - `range`
  - Notas
- A biblioteca padrão do Python
  - Introdução
    - Observações sobre disponibilidade
      - Plataformas WebAssembly
      - Plataformas móveis
  - Serviços de Processamento de Texto
    - `string` — Operações comuns de strings
      - Constantes de strings
      - Formatação personalizada de strings
      - Sintaxe das strings de formato
        - Minilinguagem de especificação de formato
        - Exemplos de formato
      - Strings template ($-strings)
      - Funções auxiliares
    - `string.templatelib` — Suporte para literais de string template
      - Strings template
      - Tipos
      - Funções auxiliares
    - `re` — Operações com expressões regulares
      - Sintaxe de expressão regular
      - Conteúdo do módulo
        - Sinalizadores
        - Funções
        - Exceções
      - Regular expression objects
      - Match objects
      - Regular expression examples

        - Checking for a pair
        - Simulando scanf()
        - search() vs. prefixmatch()
        - prefixmatch() vs. match()
        - Making a phonebook
        - Text munging
        - Finding all adverbs
        - Finding all adverbs and their positions
        - Raw string notation
        - Writing a tokenizer
    - `difflib` — Helpers for computing deltas
        - Junk heuristic
        - The `difflib` algorithm
        - Diff generation
        - Junk definition functions
        - SequenceMatcher objects
        - Exemplos
        - SequenceMatcher examples
        - Differ example
        - A command-line interface to difflib
        - ndiff example
    - `textwrap` — Quebra automática e preenchimento de texto
    - `unicodedata` — Banco de dados de Unicode
    - `stringprep` — Preparação de string para internet
    - `readline` — Interface para o GNU readline
        - Arquivo init
        - Buffer de linha
        - Arquivo de histórico
        - Lista de histórico
        - Ganchos de inicialização
        - Autocomplemento
        - Exemplo
    - `rlcompleter` — Função de autocomplemento para GNU readline
  - Serviços de Dados Binários
    - `struct` — Interpreta bytes como dados binários empacotados

- Funções e exceções
- Format Strings
  - Byte Order, Size, and Alignment
  - Caracteres Formatados
  - Exemplos
- Applications
  - Native Formats
  - Standard Formats
- Classes
- `codecs` — Codec registry and base classes
  - Codec Base Classes
    - Error Handlers
    - Stateless Encoding and Decoding
    - Incremental Encoding and Decoding
      - IncrementalEncoder Objects
      - IncrementalDecoder Objects
    - Stream Encoding and Decoding
      - StreamWriter Objects
      - StreamReader Objects
      - StreamReaderWriter Objects
      - StreamRecoder Objects
  - Encodings and Unicode
  - Standard Encodings
  - Python Specific Encodings
    - Text Encodings
    - Binary Transforms
    - Standalone Codec Functions
    - Text Transforms
  - `encodings` — Encodings package
  - `encodings.idna` — Internationalized Domain Names in Applications
  - `encodings.mbcs` — Windows ANSI codepage
  - `encodings.utf_8_sig` — UTF-8 codec with BOM signature
- Tipos de Dados
  - `datetime` — Tipos básicos de data e hora
    - Aware and naive objects

- Constantes
- Available types
  - Common properties
  - Determining if an object is aware or naive
- `timedelta` objects
  - Examples of usage: `timedelta`
- Objetos `date`
  - Examples of usage: `date`
- `datetime` objects
  - Examples of usage: `datetime`
- Objetos `time`
  - Examples of usage: `time`
- `tzinfo` objects
- `timezone` objects
- `strftime()` and `strptime()` behavior
  - `strftime()` and `strptime()` format codes
  - Technical detail
- `zoneinfo` — Suporte a fuso horário da IANA
  - Usando `ZoneInfo`
  - Fontes de dados
    - Configurando as fontes de dados
      - Configuração em tempo de compilação
      - Configuração do ambiente
      - Configuração em tempo de execução
  - A classe `ZoneInfo`
    - String representations
    - Pickle serialization
  - Funções
  - Globals
  - Exceptions and warnings
- `calendar` — Funções gerais relacionadas ao calendário
  - Uso na linha de comando
- `collections` — Tipos de dados de contêineres
  - Objetos `ChainMap`
    - Exemplos e Receitas de `ChainMap`
  - Objetos `Counter`
  - Objetos `deque`

        - Receitas de `deque`
    - Objetos `defaultdict`
        - Exemplos de `defaultdict`
    - Função de fábrica para tuplas com campos nomeados `namedtuple()`
    - Objetos `OrderedDict`
        - Exemplos e receitas de `OrderedDict`
    - Objetos `UserDict`
    - Objetos `UserList`
    - Objetos `UserString`
- `collections.abc` — Classes Base Abstratas para Contêineres
    - Classes Base Abstratas de Coleções
    - Classes Base Abstrata de Coleções – Descrições Detalhadas
    - Exemplos e receitas
- `heapq` — Algoritmo de fila heap
    - Exemplos básicos
    - Other Applications
    - Notas de implementação da fila de prioridade
    - Teoria
- `bisect` — Algoritmo de bisseção de vetor
    - Observações sobre desempenho
    - Buscando em listas ordenadas
    - Exemplos
- `array`— Vetores eficientes de valores numéricos
- `weakref` — Referências fracas
    - Objetos referência fraca
    - Exemplo
    - Objetos finalizadores
    - Comparando finalizadores com métodos `__del__()`
- `types` — Criação de tipos dinâmicos e nomes para tipos embutidos
    - Criação de tipos dinâmicos
    - Tipos padrão do interpretador
    - Classes e funções de utilidades adicionais
    - Funções de utilidade de corrotina

- `copy` — Operações de cópia profunda e cópia rasa
- `pprint` — Impressão bonita de dados
  - Funções
  - Objetos PrettyPrinter
  - Exemplo
- `reprlib` — Implementação alternativa à `repr()`
  - Objetos Repr
  - Estendendo objetos Repr
- `enum` — Suporte a enumerações
  - Conteúdo do módulo
  - Data types
    - Nomes `__dunder__` suportados
    - Nomes `_sunder_` suportados
  - Utilities and decorators
  - Notas
- `graphlib` — Funcionalidade para operar com estruturas do tipo grafo
  - Exceções
- Módulos Matemáticos e Numéricos
  - `numbers` — Classes base abstratas numéricas
    - A torre numérica
    - Notas para implementadores de tipos
      - Adicionando mais ABCs numéricas
      - Implementando as operações aritméticas
  - `math` — Funções matemáticas
    - Arimética de ponto flutuante
    - Funções de manipulação de ponto flutuante
    - Funções de potências, exponenciais e logarítmicas
    - Funções de soma e produto
    - Conversão angular
    - Funções trigonométricas
    - Funções hiperbólicas
    - Funções especiais
    - Funções de teoria dos números
    - Constantes
  - `math.integer` — integer-specific mathematics functions
  - `cmath` — Funções matemáticas para números complexos

    - Conversões de e para coordenadas polares
    - Funções de potência e logarítmicas
    - Funções trigonométricas
    - Funções hiperbólicas
    - Funções de classificação
    - Constantes
- `decimal` — Aritmética de ponto fixo decimal e ponto flutuante
    - Tutorial de início rápido
    - Objetos de Decimal
        - Operandos lógicos
    - Objetos de contexto
    - Constantes
    - Modos de arredondamento
    - Sinais
    - Observações sobre ponto flutuante
        - Atenuando o erro de arredondamento com precisão maior
        - Valores especiais
    - Trabalhando com threads
    - Receitas
    - FAQ sobre Decimal
- `fractions` — Números racionais
- `random` — Gera números pseudoaleatórios
    - Funções de contabilidade
    - Funções para bytes
    - Funções para inteiros
    - Funções para sequências
    - Distribuições discretas
    - Distribuições com valor real
    - Gerador alternativo
    - Notas sobre reprodutibilidade
    - Exemplos
    - Receitas
    - Uso na linha de comando
    - Exemplos na linha de comando
- `statistics` — Funções estatísticas

        - Médias e medidas de valor central
        - Medidas de espalhamento
        - Estatísticas para relações entre duas entradas
        - Detalhes das funções
        - Exceções
        - Objetos `NormalDist`
        - Exemplos e receitas
            - Problemas clássicos de probabilidade
            - Entradas de Monte Carlo para simulações
            - Aproximação de distribuições binomiais
            - Classificador bayesiano ingênuo
    - Módulos de Programação Funcional
        - `itertools` — Funções que criam iteradores para laços eficientes
            - Funções do intertools
            - Receitas com itertools
        - `functools` — Funções e operações de ordem superior em objetos chamáveis
            - Objetos `partial`
        - `operator` — Operadores padrões como funções
            - Mapeando os operadores para suas respectivas funções
            - Operadores in-place
    - Acesso a arquivos e diretórios
        - `pathlib` — Caminhos do sistema de arquivos orientados a objetos
            - Uso básico
            - Exceções
            - Caminhos puros
                - Propriedades gerais
                - Operadores
                - Acessando partes individuais
                - Métodos e propriedades
            - Caminhos concretos
                - Analisando e gerando URIs
                - Expandindo e resolvendo caminhos
                - Consultando tipo de arquivo e status
                - Lendo e escrevendo arquivos

        - Lendo diretórios
        - Criando arquivos e diretórios
        - Copiando, movendo e excluindo
        - Permissões e propriedade
    - Linguagem de padrões
    - Comparação com o módulo `glob`
    - Comparação com os módulos `os` e `os.path`
        - Ferramentas correspondentes
    - Protocolos
- `os.path` — Manipulações comuns de nomes de caminhos
- `stat` — Interpretando resultados de `stat()`
- `filecmp` — Comparações de arquivos e diretórios
    - A classe `dircmp`
- `tempfile` — Gera arquivos temporários e diretórios
    - Exemplos
    - Deprecated functions and variables
- `glob` — Expansão de padrão de nome de arquivo no estilo Unix
    - Exemplos
- `fnmatch` — Correspondência de padrões de nome de arquivo Unix
- `linecache` — Acesso aleatório a linhas de texto
- `shutil` — Operações de arquivo de alto nível
    - Operações de diretório e arquivos
        - Operações de cópia eficientes dependentes da plataforma
        - Exemplo de copytree
        - exemplo rmtree
    - Operações de arquivamento
        - Exemplo de arquivo
        - Exemplo de arquivamento com *base_dir*
    - Consultando o tamanho do terminal de saída
- Persistência de Dados
    - `pickle` — Serialização de objetos Python
        - Relação com outros módulos Python
        - Comparação com `marshal`
        - Comparação com `json`

    - Formato de fluxo de dados
    - Interface do módulo
    - O que pode ser serializado e desserializado com pickle?
    - Serializando instâncias de classes com pickle
        - Persistência de objetos externos
        - Tabelas de despacho
        - Manipulação de objetos com estado
    - Redução personalizada para tipos, funções e outros objetos
    - Buffers fora da banda
        - API de provedor
        - API de consumidor
        - Exemplo
    - Restringindo globais
    - Desempenho
    - Exemplos
    - Interface de linha de comando
- `copyreg` — Registra funções de suporte `pickle`
    - Exemplo
- `shelve` — Python object persistence
    - Restrições
    - Exemplo
    - Exceções
- `marshal` — Serialização interna de objetos Python
- `dbm` — Interfaces para “banco de dados” Unix
    - `dbm.sqlite3` — SQLite backend for dbm
    - `dbm.gnu` — GNU database manager
    - `dbm.ndbm` — New Database Manager
    - `dbm.dumb` — Portable DBM implementation
- `sqlite3` —- interface DB-API 2.0 interface para bancos de dados SQLite
    - Tutorial
    - Referência
        - Funções do módulo
        - Constantes do módulo
        - Connection objects
        - Cursor objects

- Row objects
- Blob objects
- PrepareProtocol objects
- Exceções
- SQLite and Python types
- Default adapters and converters (deprecated)
- Interface de linha de comando
- Guias de como fazer
  - How to use placeholders to bind values in SQL queries
  - How to adapt custom Python types to SQLite values
    - How to write adaptable objects
    - How to register adapter callables
  - How to convert SQLite values to custom Python types
  - Adapter and converter recipes
  - How to use connection shortcut methods
  - How to use the connection context manager
  - How to work with SQLite URIs
  - How to create and use row factories
  - How to handle non-UTF-8 text encodings
- Explicação
  - Controle de transações
    - Controle de transações através do atributo `autocommit`
    - Controle de transação através do atributo `isolation_level`

- Compressão de Dados e Arquivamento
  - O pacote `compression`
  - `compression.zstd` — Compactação compatível com o formato Zstandard
    - Exceções
    - Lendo e escrevendo arquivos compactados
    - Compactando e descompactando dados na memória
    - Dicionários de Zstandard
    - Controle avançado de parâmetros

    - Diversos
    - Exemplos
- `zlib` — Compactação compatível com **gzip**
- `gzip` — Suporte para arquivos **gzip**
    - Exemplos de uso
    - Interface de linha de comando
        - Opções de linha de comando
- `bz2` — Suporte para compressão **bzip2**
    - (Des)compressão de arquivos
    - (Des)compressão incremental
    - (De)compressão de uma só vez (one-shot)
    - Exemplos de uso
- `lzma` — Compressão usando o algoritmo LZMA
    - Lendo e escrevendo arquivos comprimidos
    - Comprimindo e descomprimindo dados na memória
    - Diversos
    - Especificando cadeias de filtros personalizadas
    - Constantes
    - Exemplos
- `zipfile` — Trabalha com arquivos ZIP
    - Objetos ZipFile
    - Objetos Path
    - Objetos PyZipFile
    - Objetos ZipInfo
    - Interface de linha de comando
        - Opções de linha de comando
    - Armadilhas de descompressão
        - Do próprio arquivo
        - Limitações do sistema de arquivos
        - Limitações de recursos
        - Interrupção
        - Comportamentos padrão da extração
- `tarfile` — Ler e gravar arquivos do tipo tar
    - Objetos TarFile
    - Objetos TarInfo
    - Filtros de extração
        - Default named filters

        - Filter errors
        - Dicas para verificação adicional
        - Suporte a versões mais antigas do Python
        - Stateful extraction filter example
    - Interface de Linha de Comando
        - Opções de linha de comando
    - Exemplos
        - Reading examples
        - Writing examples
    - Formatos tar suportados
    - Problemas de Unicode
- Formatos de Arquivo
    - `csv` — Leitura e escrita de arquivos CSV
        - Conteúdo do módulo
        - Dialects and formatting parameters
        - Reader objects
        - Writer objects
        - Exemplos
    - `configparser` — Analisador sintático de arquivo de configuração
        - Início rápido
        - Tipos de dados suportados
        - Valores reservas
        - Estrutura dos arquivos INI
        - Seções não nomeadas
        - Interpolação de valores
        - Acesso através do protocolo de mapeamento
        - Personalizando o comportamento do analisador sintático
        - Exemplos de APIs legadas
        - Objetos ConfigParser
        - Objetos RawConfigParser
        - Exceções
    - `tomllib` — Analisa arquivos TOML
        - Exemplos
        - Tabela de conversão
        - Considerações sobre limites e interoperabilidade

- `netrc` — Arquivo de processamento netrc
  - Objetos netrc
- `plistlib` — Gera e analisa arquivos Apple `.plist`
  - Exemplos
- Serviços Criptográficos
  - `hashlib` — Resumos de mensagens e hashes seguros
    - Algoritmos de hash
    - Uso
    - Construtores
    - Atributos
    - Objetos hash
    - Resumos de comprimento variável de SHAKE
    - Hash de arquivo
    - Derivação de chave
    - BLAKE2
      - Criando objetos hash
      - Constantes
      - Exemplos
        - Simple hashing
        - Using different digest sizes
        - Keyed hashing
        - Randomized hashing
        - Personalization
        - Modo árvore
      - Credits
  - `hmac` — Código de autenticação de mensagem com chave hash
  - `secrets` — Gera números aleatórios seguros para gerenciar segredos
    - Números aleatórios
    - Gerando tokens
      - Quantos bytes os tokens devem usar?
    - Outras funções
    - Receitas e melhores práticas
- Serviços Genéricos do Sistema Operacional
  - `os` — Diversas interfaces de sistema operacional

- Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente
- Modo UTF-8 do Python
- Parâmetros de processo
- Criação de objetos arquivos
- Operações dos descritores de arquivos
  - Consultando o tamanho de um terminal
  - Herança de descritores de arquivos
- Arquivos e diretórios
  - Descritores de arquivo de temporizador
  - Atributos estendidos do Linux
- Gerenciamento de processo
- Interface do agendador
- Diversas informações de sistema
- Números aleatórios

- `io` — Ferramentas essenciais para trabalhar com fluxos
  - Visão Geral
    - E/S de texto
    - E/S binária
    - E/S bruta
  - Codificação do texto
    - Ativar EncodingWarning
  - Interface de alto nível do módulo
  - hierarquia de classe
    - I/O base classes
    - Raw file I/O
    - Buffered streams
    - E/S de texto
  - Static typing
  - Desempenho
    - E/S binária
    - E/S de texto
    - Multi-threading
    - Reentrancy
- `time` — Acesso e conversões de tempo
  - Funções
  - Constantes de ID de Relógio

    - Constantes de Fuso Horário
- `logging` — Recursos de registro de eventos para Python
    - Objetos Logger
    - Níveis de Logging
    - Objetos Handler
    - Formatter Objects
    - Filter Objects
    - LogRecord Objects
    - Atributos LogRecord
    - LoggerAdapter Objects
    - Thread Safety
    - Funções de Nível de Módulo
    - Module-Level Attributes
    - Integration with the warnings module
- `logging.config` — Configuração do módulo logging
    - Funções de configuração
    - Considerações de segurança
    - Esquema do Dicionário de Configuração
        - Detalhes do Esquema de Dicionário
        - Configuração Incremental
        - Conexões de objeto
        - Objetos definidos pelo usuário
        - Ordem de configuração de manipuladores
        - Acesso a objetos externos
        - Acesso a objetos internos
        - Resolução de importações e importadores personalizados
        - Configurando QueueHandler e QueueListener
    - Formato do arquivo de configuração
- `logging.handlers` — Manipuladores de logging
    - StreamHandler
    - FileHandler
    - NullHandler
    - WatchedFileHandler
    - BaseRotatingHandler
    - RotatingFileHandler
    - TimedRotatingFileHandler

    - SocketHandler
    - DatagramHandler
    - SysLogHandler
    - NTEventLogHandler
    - SMTPHandler
    - MemoryHandler
    - HTTPHandler
    - QueueHandler
    - QueueListener
- `platform` — Acesso aos dados de identificação da plataforma subjacente
    - Multiplataforma
    - Plataforma Windows
    - Plataforma macOS
    - Plataforma iOS
    - Plataformas Unix
    - Plataformas Linux
    - Plataforma Android
    - Uso na linha de comando
- `errno` — Símbolos padrão do sistema errno
- `ctypes` — Uma biblioteca de funções externas para Python
    - Tutorial do ctypes
        - Carregando bibliotecas de ligação dinâmica
        - Acessando funções de dlls carregadas
        - Chamando funções
        - Tipos de dados fundamentais
        - Chamando funções, continuação
        - Chamando funções variadas
        - Chamando funções com seus próprios tipos de dados personalizados
        - Especificando os tipos de argumentos necessários (protótipos de função)
        - Tipos de Retorno
        - Passando ponteiros (ou: passando parâmetros por referência)
        - Estruturas e uniões

- Layout, alinhamento e ordem de bytes de estrutura/união
- Campos de bit em estruturas e uniões
- Vetores
- Ponteiros
- Segurança de thread (Thread safety) sem o GIL
- Conversão de Tipos
- Tipos Incompletos
- Funções Callbacks
- Acessando valores exportados de dlls
- Surpresas
- Tipos de dados de tamanho variável

- Referência ctypes
  - Loading shared libraries
  - Encontrando bibliotecas compartilhadas
  - Listing loaded shared libraries
  - Foreign functions
  - Function prototypes
  - Funções utilitárias
  - Data types
  - Tipos de dados fundamentais
  - Tipos de dados estruturados
  - Arrays and pointers
  - Exceções

- Bibliotecas de interface de linha de comando
  - `argparse` — Analisador sintático para opções de linha de comando, argumentos e subcomandos
    - Objetos ArgumentParser
      - prog
      - usage
      - description
      - epilog
      - parents
      - formatter_class
      - prefix_chars
      - fromfile_prefix_chars
      - argument_default

    - allow_abbrev
    - conflict_handler
    - add_help
    - exit_on_error
    - suggest_on_error
    - color
- O método add_argument()
    - name ou flags
    - action
    - nargs
    - const
    - default
    - type
    - choices
    - required
    - help
    - metavar
    - dest
    - deprecated
    - Classes de ação
- O método parse_args()
    - Sintaxe de valores da opção
    - Argumentos inválidos
    - Argumentos contendo -
    - Abreviações de argumento (correspondência de prefixo)
    - Além do `sys.argv`
    - O objeto Namespace
- Outros utilitários
    - Subcomandos
    - Objetos FileType
    - Grupos de argumentos
    - Exclusão mútua
    - Padrões do analisador sintático
    - Imprimindo a ajuda
    - Análise parcial
    - Personalizando a análise de arquivos

- Métodos de saída
- Análise misturada
- Registrando ações e tipos personalizados
- Exceções
- Tutorial de Argparse
  - Conceitos
  - O básico
  - Apresentando os argumentos posicionais
  - Apresentando os argumentos opcionais
    - Opções curtas
  - Combinando argumentos posicionais e opcionais
  - Avançando um pouco mais
    - Especificando argumentos ambíguos
    - Opções conflitantes
  - Como traduzir a saída do argparse
  - Conversores de tipo personalizados
  - Conclusão
- Migrando código `optparse` para `argparse`
- `optparse` — Analisador sintático para opções de linha de comando
  - Escolhendo uma biblioteca de análise de linha de comando
  - Introdução
  - Contexto
    - Terminologia
    - Para que servem as opções?
    - Para que servem os argumentos posicionais?
  - Tutorial
    - Compreendendo as ações de opções
    - A ação store
    - Manipulando opções (sinalizadores) booleanas
    - Outras ações
    - Valores padrão
    - Gerando texto de ajuda
      - Agrupando opções
    - Exibindo uma string de versão

        - How `optparse` handles errors
        - Juntando tudo
    - Guia de referência
        - Criando o analisador sintático
        - Preenchendo o analisador sintático
        - Definindo as opções
        - Atributos da classe Option
        - Ações de opção padrão
        - Tipos de opção padrão
        - Análise de argumentos
        - Consultando e manipulando seu analisador sintático de opções
        - Conflitos entre opções
        - Limpeza
        - Outros métodos
    - Funções de retorno de opção
        - Definindo uma opção de retorno de chamada
        - Como os retornos de chamada são chamados
        - Levantando erros em uma função de retorno
        - Exemplo de função de retorno 1: retorno de chamada trivial
        - Exemplo de função de retorno 2: verificar a ordem das opções
        - Exemplo 3 de função de retorno: verificar a ordem das opções (generalizada)
        - Exemplo de função de retorno 4: verificar uma condição arbitrária
        - Exemplo de função de retorno 5: argumentos fixos
        - Exemplo de função de retorno 6: argumentos variáveis
    - Extending `optparse`
        - Adicionando novos tipos
        - Adicionando novas ações
    - Exceções
- `getpass` — Entrada de senha portátil
- `fileinput` — Itera sobre linhas de múltiplos fluxos de entrada

- `curses` — Gerenciador de terminal para visualizadores de células de caracteres.
  - Funções
  - Window objects
  - Constantes
- `curses.textpad` — Text input widget for curses programs
  - Textbox objects
- `curses.ascii` — Utilitários para caracteres ASCII
- `curses.panel` — Uma extensão de pilha de painéis para o curses
  - Funções
  - Panel objects
- `cmd` — Suporte para interpretadores de comando orientado a linhas
  - Objetos Cmd
  - Exemplo do Cmd
- Execução Concorrente
  - `threading` — Paralelismo baseado em threads
    - Introdução
    - GIL and performance considerations
    - Referência
      - Thread-local data
      - Thread objects
      - Lock objects
      - RLock objects
      - Condition objects
      - Semaphore objects
      - `Semaphore` example
      - Event objects
      - Timer objects
      - Barrier objects
    - Using locks, conditions, and semaphores in the `with` statement
    - Iterator synchronization
  - `multiprocessing` — Paralelismo baseado em processo
    - Introdução
      - A classe `Process`

        - Contextos e métodos de inicialização
        - Trocando objetos entre processos
        - Sincronização entre processos
        - Compartilhando estado entre processos
        - Usando um pool de workers
    - Referência
        - Global start method
        - `Process` e exceções
        - Encadeamentos e filas
        - Diversos
        - Objetos de conexão
        - Primitivas de sincronização
        - Objetos `ctypes` compartilhados
            - The `multiprocessing.sharedctypes` module
        - Gerenciadores
            - Customized managers
            - Using a remote manager
        - Objetos proxies
            - Limpeza
        - Process Pools
        - Listeners and Clients
            - Formatos de Endereços
        - Authentication keys
        - Gerando logs
        - The `multiprocessing.dummy` module
    - Programming guidelines
        - All start methods
        - The *spawn* and *forkserver* start methods
    - Exemplos
- `multiprocessing.shared_memory` — Memória compartilhada para acesso direto entre processos
- O pacote `concurrent`
- `concurrent.futures` — Launching parallel tasks
    - Executor Objects
    - ThreadPoolExecutor
        - Exemplo de ThreadPoolExecutor
    - InterpreterPoolExecutor

- ProcessPoolExecutor
  - ProcessPoolExecutor Example
- Future Objects
- Module Functions
- Exception classes

- `concurrent.interpreters` — Múltiplos interpretadores no mesmo processo
  - Detalhes chaves
  - Introdução
    - Multiple Interpreters and Isolation
    - Running in an Interpreter
    - Concurrency and Parallelism
    - Communication Between Interpreters
    - “Sharing” Objects
  - Referência
    - Objetos interpretador
    - Exceções
    - Communicating Between Interpreters
  - Uso básico
- `subprocess` — Gerenciamento de subprocessos
  - Using the `subprocess` Module
    - Argumentos usados frequentemente
    - Construtor Popen
    - Exceções
  - Considerações de Segurança
  - Objetos Popen
  - Windows Popen Helpers
    - Constantes do Windows
  - API de alto nível mais antiga
  - Replacing Older Functions with the `subprocess` Module
    - Replacing **/bin/sh** shell command substitution
    - Replacing shell pipeline
    - Substituindo `os.system()`
    - Replacing the `os.spawn` family
    - Replacing `os.popen()`
  - Legacy Shell Invocation Functions

- Notas
  - Timeout Behavior
  - Converter uma sequência de argumentos em uma string no Windows
  - Disable use of `posix_spawn()`
- `sched` — Agendador de eventos
  - Objetos Scheduler
- `queue` — Uma classe de fila sincronizada
  - Objetos Queue
    - Waiting for task completion
    - Terminating queues
  - Objetos SimpleQueue
- `contextvars` — Variáveis de contexto
  - Variáveis de contexto
  - Gerenciamento de contexto manual
  - Suporte a asyncio
- `_thread`— API de segmentação de baixo nível

- Comunicação em Rede e Interprocesso
  - `asyncio` — E/S assíncrona
    - Runners
      - Executando um programa asyncio
      - Runner context manager
      - Handling Keyboard Interruption
    - Coroutines and tasks
      - Corrotinas
      - Aguardáveis
      - Creating tasks
      - Task cancellation
      - Task groups
      - Dormindo
      - Running tasks concurrently
      - Eager task factory
      - Shielding from cancellation
      - Tempo limite
      - Waiting primitives
      - Running in threads
      - Scheduling from other threads

    - Introspecção
    - Task object
- Streams
    - StreamReader
    - StreamWriter
    - Exemplos
        - Cliente para eco TCP usando streams
        - Servidor eco TCP usando streams
        - Obtém headers HTTP
        - Registra um soquete aberto para aguardar por dados usando streams
- Synchronization Primitives
    - Lock
    - Evento
    - Condição
    - Semaphore
    - BoundedSemaphore
    - Barrier
- Subprocessos
    - Criando subprocessos
    - Constantes
    - Interagindo com subprocessos
        - Subprocesso e Threads
        - Exemplos
- Filas
    - Queue
    - Fila de prioridade
    - Filas LIFO (último a entrar, primeiro a sair)
    - Exceções
    - Exemplos
- Exceções
- Call graph introspection
    - Funções utilitárias de baixo nível
- Command-line introspection tools
    - Opções de linha de comando
- Laço de eventos
    - Event loop methods

- Executar e interromper o laço
- Agendando funções de retorno
- Agendando funções de retorno atrasadas
- Creating futures and tasks
- Abrindo conexões de rede
- Criando servidores de rede
- Transferindo arquivos
- TLS upgrade
- Observando descritores de arquivo
- Trabalhando com objetos soquete diretamente
- DNS
- Trabalhando com encadeamentos
- Sinais Unix
- Executando código em conjuntos de threads ou processos
- Error handling API
- Habilitando o modo de debug
- Running subprocesses
- Callback handles
- Server objects
- Event loop implementations
- Exemplos
- Hello World com call_soon()
- Exibe a data atual com call_later()
- Observa um descritor de arquivo por eventos de leitura
- Define tratadores de sinais para SIGINT e SIGTERM
- Futuros
- Funções Future
- Objeto Future
- Transports e Protocols
- Transportes
- Transports Hierarchy
- Base Transport
- Read-only Transports
- Write-only Transports

        - Transportes de datagrama
        - Transportes de Subprocesso
    - Protocolos
        - Protocolos de Base
        - Base Protocol
        - Streaming Protocols
        - Protocolos de Streaming Bufferizados
        - Protocolos de Datagramas
        - Protocolos de Subprocesso
    - Exemplos
        - TCP Echo Server
        - TCP Echo Client
        - UDP Echo Server
        - UDP Echo Client
        - Connecting Existing Sockets
        - loop.subprocess_exec() and SubprocessProtocol
- Políticas
    - Getting and Setting the Policy
    - Policy Objects
    - Custom Policies
- Suporte a plataformas
    - Todas as plataformas
    - Windows
        - Suporte para subprocesso no Windows
    - macOS
- Extensão
    - Escrevendo um laço de eventos personalizado
    - Construtores privados de Future e Task
    - Suporte ao ciclo de vida de tarefas
- Índice da API de alto nível
    - Tarefas
    - Filas
    - Subprocessos
    - Streams
    - Sincronização
    - Exceções

- Índice de APIs de baixo nível
  - Obtendo o laço de eventos
  - Métodos do laço de eventos
  - Transportes
  - Protocolos
  - Políticas de laço de eventos
- Desenvolvendo com asyncio
  - Modo de Depuração
  - Concorrência e Múltiplas Threads
  - Executando código bloqueante
  - Gerando logs
  - Detectar corrotinas nunca aguardadas
  - Detectar exceções nunca recuperadas
  - Asynchronous generators best practices
    - Close asynchronous generators explicitly
    - Create asynchronous generators only when the event loop is running
    - Avoid concurrent iteration and closure of the same generator
- asyncio and free-threaded Python
  - Thread safety considerations
  - Using asyncio with threads
  - Producer/consumer across threads
- `socket` — Interface de rede de baixo nível
  - Famílias de soquete
  - Conteúdo do módulo
    - Exceções
    - Constantes
    - Funções
      - Criação de sockets
      - Outras funções
  - Socket Objects
  - Notas sobre tempo limite de soquete
    - Timeouts and the `connect` method
    - Timeouts and the `accept` method
  - Exemplo
- `ssl` — Invólucro de TLS/SSL para objetos socket

- Funções, constantes e exceções.
  - Criação de socket
  - Criação de contexto
  - Signature algorithms
  - Exceções
  - Random generation
  - Manipulação de certificados
  - Constantes
- SSL sockets
- SSL contexts
- Certificados
  - Certificate chains
  - CA certificates
  - Combined key and certificate
  - Self-signed certificates
- Exemplos
  - Testing for SSL support
  - Client-side operation
  - Server-side operation
- Notes on non-blocking sockets
- Memory BIO support
- SSL session
- Considerações de segurança
  - Best defaults
  - Manual settings
    - Verifying certificates
    - Protocol versions
    - Cipher selection
  - Multi-processing
- TLS 1.3

- `select` — Waiting for I/O completion
  - `/dev/poll` polling objects
  - Edge and level trigger polling (epoll) objects
  - Polling objects
  - Kqueue objects
  - Kevent objects
- `selectors` — High-level I/O multiplexing

        - Introdução
        - Classes
        - Exemplos
    - `signal` — Define manipuladores para eventos assíncronos
        - Regras gerais
            - Execução de manipuladores de sinais Python
            - Sinais e threads
        - Conteúdo do módulo
        - Exemplos
        - Nota sobre SIGPIPE
        - Nota sobre manipuladores de sinais e exceções
    - `mmap` — Suporte a arquivos mapeados na memória
        - Constantes MADV_*
        - Constantes MAP_*
        - MS_* Constants
- Manuseio de Dados na Internet
    - `email` — Um e-mail e um pacote MIME manipulável
        - `email.message`: Representing an email message
        - `email.parser`: Parsing email messages
            - API do FeedParser
            - Parser API
            - Additional notes
        - `email.generator`: Geração de documentos MIME
        - `email.policy`: Policy Objects
        - `email.errors`: Classes de Exceção e Defeito.
        - `email.headerregistry`: Custom Header Objects
        - `email.contentmanager`: Gerenciando conteúdo MIME
            - Content Manager Instances
        - `email`: Exemplos
        - `email.message.Message`: Representing an email message using the `compat32` API
        - `email.mimeemail.mime`: Criando e-mail e objetos MIME do zero
        - `email.header`: Internationalized headers
        - `email.charset`: Representando conjuntos de caracteres
        - `email.encoders`: Codificadores
        - `email.utils`: Utilitários diversos

- `email.iterators`: Iteradores
- `json` — Codificador e decodificador JSON
  - Uso básico
  - Codificadores e decodificadores
  - Exceções
  - Conformidade e interoperabilidade entre padrões
    - Codificações de caracteres
    - Valores numéricos infinitos e NaN
    - Nomes repetidos dentro de um objeto
    - Valores não objeto e não vetor de nível superior
    - Limitações de implementação
  - Interface de linha de comando
    - Opções de linha de comando
- `mailbox` — Manipule caixas de entradas em vários formatos
  - Objetos `Mailbox`
    - `Maildir` objects
    - `mbox` objects
    - `MH` objects
    - `Babyl` objects
    - `MMDF` objects
  - `Message` objects
    - `MaildirMessage` objects
    - `mboxMessage` objects
    - `MHMessage` objects
    - `BabylMessage` objects
    - `MMDFMessage` objects
  - Exceções
  - Exemplos
- `mimetypes` — Mapeia nomes de arquivos para tipos MIME
  - Objetos MimeTypes
  - Uso na linha de comando
  - Exemplos na linha de comando
- `base64` — Base16, Base32, Base64, Base85 Data Encodings
  - RFC 4648 Encodings
  - Base85 Encodings
  - Legacy Interface
  - Considerações de Segurança

- **binascii** — Converte entre binário e ASCII
- **quopri** — Codifica e decodifica dados MIME imprimidos entre aspas
- Ferramentas de Processamento de Markup Estruturado
  - **html** — Suporte HTML (HyperText Markup Language)
  - **html.parser** — Analisador simples de HTML e XHTML
    - Exemplo de aplicação para análise de HTML
    - Métodos **HTMLParser**
    - Exemplos
  - **html.entities** — Definições de entidades gerais de HTML
  - Módulos de Processamento de XML
    - Segurança no XML
  - **xml.etree.ElementTree** — A API XML ElementTree
    - Tutorial
      - Árvore e elementos XML
      - Analisando XML
      - A API de pull para análise sem bloqueio
      - Encontrando elementos interessantes
      - Modificando um arquivo XML
      - Construindo documentos XML
      - Analisando XML com espaços de nomes
    - Suporte a XPath
      - Exemplo
      - Supported XPath syntax
    - Referência
      - Funções
    - Suporte a XInclude
      - Exemplo
    - Referência
      - Funções
      - Element Objects
      - ElementTree Objects
      - QName Objects
      - TreeBuilder Objects
      - Objetos XMLParser
      - XMLPullParser Objects
      - Exceções

- `xml.dom` — The Document Object Model API
  - Conteúdo do módulo
  - Objects in the DOM
    - DOMImplementation Objects
    - Objetos Node
    - Objetos NodeList
    - DocumentType Objects
    - Document Objects
    - Element Objects
    - Attr Objects
    - NamedNodeMap Objects
    - DocumentFragment Objects
    - CharacterData Objects
    - Comment Objects
    - Text and CDATASection Objects
    - Objetos ProcessingInstruction
    - Entity Objects
    - Notation Objects
    - Exceções
  - Conformance
    - Type Mapping
    - Accessor Methods
- `xml.dom.minidom` — Minimal DOM implementation
  - Objetos DOM
  - DOM Example
  - minidom e o padrão DOM
- `xml.dom.pulldom` — Suporte para construir árvores parciais de DOM
  - Objetos DOMEventStream
- `xml.sax` — Support for SAX2 parsers
  - SAXException Objects
- `xml.sax.handler` — Classes base para manipuladores de SAX
  - ContentHandler Objects
  - DTDHandler Objects
  - EntityResolver Objects
  - ErrorHandler Objects

        - LexicalHandler Objects
    - `xml.sax.saxutils` — Utilitários de SAX
    - `xml.sax.xmlreader` — Interface for XML parsers
        - XMLReader Objects
        - IncrementalParser Objects
        - Locator Objects
        - InputSource Objects
        - The `Attributes` Interface
        - The `AttributesNS` Interface
    - `xml.parsers.expat` — Fast XML parsing using Expat
        - Objetos XMLParser
        - Exceções ExpatError
        - Exemplo
        - Content Model Descriptions
        - Constantes de erro de expatriados
- Protocolos de Internet e Suporte
    - `webbrowser` — Controlador de navegador web conveniente
        - Interface de linha de comando
        - Objetos controladores de navegador
    - `wsgiref` — Implementação de referência e utilitários WSGI
        - `wsgiref.util` – WSGI environment utilities
        - `wsgiref.headers` – WSGI response header tools
        - `wsgiref.simple_server` – a simple WSGI HTTP server
        - `wsgiref.validate` — WSGI conformance checker
        - `wsgiref.handlers` – server/gateway base classes
        - `wsgiref.types` – WSGI types for static type checking
        - Exemplos
    - `urllib` — Módulos de manipulação de URL
    - `urllib.request` — Biblioteca extensível para abrir URLs
        - Objeto Request
        - OpenerDirector Objects
        - BaseHandler Objects
        - HTTPRedirectHandler Objects
        - HTTPCookieProcessor Objects
        - ProxyHandler Objects
        - HTTPPasswordMgr Objects

    - HTTPPasswordMgrWithPriorAuth Objects
    - AbstractBasicAuthHandler Objects
    - HTTPBasicAuthHandler Objects
    - ProxyBasicAuthHandler Objects
    - AbstractDigestAuthHandler Objects
    - HTTPDigestAuthHandler Objects
    - ProxyDigestAuthHandler Objects
    - HTTPHandler Objects
    - Objetos HTTPSHandler
    - FileHandler Objects
    - DataHandler Objects
    - FTPHandler Objects
    - CacheFTPHandler Objects
    - Objetos UnknownHandler
    - HTTPErrorProcessor Objects
    - Exemplos
    - Legacy interface
    - `urllib.request` Restrictions
- `urllib.response` — Response classes used by urllib
- `urllib.parse` — Analisa URLs para componentes
    - Análise de URL
    - Segurança ao analisar URLs
    - Analisando bytes codificados em ASCII
    - Structured Parse Results
    - URL Quoting
- `urllib.error` — Classes de exceção levantadas por urllib.request
- `urllib.robotparser` — Analisador sintático de robots.txt
- `http` — Módulos HTTP
    - Códigos de estado HTTP
    - Categoria de estado HTTP
    - Métodos HTTP
- `http.client` — HTTP protocol client
    - Objetos de HTTPConnection
    - Objetos HTTPResponse
    - Exemplos
    - HTTPMessage Objects

- `ftplib` — FTP protocol client
  - Referência
    - FTP objects
    - FTP_TLS objects
    - Module variables
- `poplib` — Cliente de protocolo POP3
  - Objetos POP3
  - Exemplo POP3
- `imaplib` — IMAP4 protocol client
  - Objetos IMAP4
  - Exemplo IMAP4
- `smtplib` — Cliente de protocolo SMTP
  - Objetos SMTP
  - Exemplo SMTP
- `uuid` — Objetos UUID conforme **RFC 9562**
  - Uso na linha de comando
  - Exemplo
  - Exemplo de linha de comando
- `socketserver` — Um framework para servidores de rede
  - Notas sobre criação do servidor
  - Objetos Server
  - Objetos manipulador de requisições
  - Exemplos
    - Exemplo de `socketserver.TCPServer`
    - Exemplo `socketserver.UDPServer`
    - Mixins assíncronos
- `http.server` — HTTP servers
  - Interface de linha de comando
  - Considerações de segurança
- `http.cookies` — HTTP state management
  - Objetos Cookie
  - Objetos Morsel
  - Exemplo
- `http.cookiejar` — Cookie handling for HTTP clients
  - CookieJar and FileCookieJar objects
  - FileCookieJar subclasses and co-operation with web browsers

- CookiePolicy objects
- DefaultCookiePolicy objects
- Cookie objects
- Exemplos

- `xmlrpc` — Módulos de servidor e cliente XMLRPC
- `xmlrpc.client` — Cliente XML-RPC
  - Objetos ServerProxy
  - Objetos DateTime
  - Objetos Binários
  - Objetos Fault
  - Objetos ProtocolError
  - Objetos MultiCall
  - Convenience Functions
  - Exemplo de uso do cliente
  - Example of Client and Server Usage
- `xmlrpc.server` — Servidores XML-RPC básicos
  - SimpleXMLRPCServer objects
    - SimpleXMLRPCServer example
  - CGIXMLRPCRequestHandler
  - Documentando servidor XMLRPC
  - DocXMLRPCServer objects
  - DocCGIXMLRPCRequestHandler
- `ipaddress` — Biblioteca de manipulação de IPv4/IPv6
  - Funções de fábrica de conveniência
  - Endereços IP
    - Objetos de endereço
    - Conversão para strings e inteiros
    - Operadores
      - Operadores de comparação
      - Operadores aritméticos
  - Definições de rede IP
    - Prefixo, máscara de rede e máscara de host
    - Objetos de rede
    - Operadores
      - Operadores lógicos
      - Iteração
      - Redes como contêineres de endereços

- Objetos de interface
  - Operadores
    - Operadores lógicos
- Outras funções de nível de módulo
- Exceções personalizadas

- Serviços Multimídia
  - `wave` — Read and write WAV files
    - Objetos Wave_read
    - Objetos Wave_write
  - `colorsys` — Conversões entre sistemas de cores
- Internacionalização
  - `gettext` — Serviços de internacionalização multilíngues
    - API do GNU **gettext**
    - API baseada em classe
      - A classe `NullTranslations`
      - A classe `GNUTranslations`
      - Suporte a catálogo de mensagens do Solaris
      - O construtor Catalog
    - Internacionalizando seus programas e módulos
      - Localizando seu módulo
      - Localizando sua aplicação
      - Alterando os idiomas durante o uso
      - Traduções adiadas
    - Reconhecimentos
  - `locale` — Serviços de internacionalização
    - Histórico, detalhes, dicas, dicas e advertências
    - Nomes de localidades
    - Para escritores de extensão e programas que incorporam Python
    - Acesso a catálogos de mensagens
- Interfaces gráficas de usuário com Tk
  - `tkinter` — Interface Python para Tcl/Tk
    - Arquitetura
    - Tkinter modules
    - Tkinter life preserver
      - A Hello World program
      - Important Tk concepts

        - Understanding how Tkinter wraps Tcl/Tk
        - Como é que eu…? Que opção faz…?
        - Navigating the Tcl/Tk reference manual
    - Modelo de threading
    - Handy reference
        - Setting options
        - Geometry management
        - Coupling widget variables
        - The window manager
        - Tk option data types
        - Bindings and events
        - The index parameter
        - Imagens
    - Referência
        - Base and mixin classes
        - Toplevel widgets
        - Widget classes
        - Variable classes
        - Image classes
        - Other classes
        - Module-level functions
        - File handlers
        - Constantes
- **`tkinter.colorchooser`** — Diálogo de escolha de cor
- **`tkinter.font`** — Invólucro de fontes Tkinter
- Tkinter dialogs
    - **`tkinter.simpledialog`** — Standard Tkinter input dialogs
    - **`tkinter.filedialog`** — File selection dialogs
        - Native load/save dialogs
    - **`tkinter.commondialog`** — Dialog window templates
    - **`tkinter.dialog`** — Classic Tk dialog boxes
- **`tkinter.messagebox`** — Prompts de mensagem do Tkinter
- **`tkinter.scrolledtext`** — Widget de texto com *scrollbar*
- **`tkinter.dnd`** — Suporte para arrastar e soltar
- **`tkinter.ttk`** — Widgets temáticos do Tk
    - Usando Ttk

- Ttk widgets
- Widget
  - Standard options
  - Scrollable widget options
  - Label options
  - Compatibility options
  - Widget states
  - ttk.Widget
- Combobox
  - Opções
  - Eventos virtuais
  - ttk.Combobox
- Spinbox
  - Opções
  - Eventos virtuais
  - ttk.Spinbox
- Notebook
  - Opções
  - Tab options
  - Tab identifiers
  - Eventos virtuais
  - ttk.Notebook
- Progressbar
  - Opções
  - ttk.Progressbar
- Separator
  - Opções
- Sizegrip
  - Platform-specific notes
  - Bugs
- Treeview
  - Opções
  - Item options
  - Tag options
  - Column identifiers
  - Eventos virtuais
  - ttk.Treeview

- Ttk styling
  - Layouts
- Additional widgets
- IDLE — Editor e console Python
  - Menus
    - Menu File (Shell e Editor)
    - Menu Edit (Shell e Editor)
    - Menu Format (somente na janela do Editor)
    - Menu Run (somente na janela do Editor)
    - Menu Shell (somente na janela do Shell)
    - Menu Debug (somente na janela do Shell)
    - Menu Options (Shell e Editor)
    - Menu Window (Shell e Editor)
    - Menu Help (Shell e Editor)
    - Menus de contexto
  - Edição e navegação
    - Janelas do Editor
    - Teclas de atalho
    - Indentação automática
    - Busca e substituição
    - Autocomplementos
    - Dicas de chamada
    - Formatação de bloco
    - Contexto de código
    - Janela Shell
    - Text colors
  - Startup and Code Execution
    - Uso na linha de comando
    - Startup failure
    - Running user code
    - User output in Shell
    - Developing tkinter applications
    - Running without a subprocess
  - Ajuda e preferências
    - Help sources
    - Configurando preferências
    - IDLE on macOS

- Extensions
- idlelib — implementation of IDLE application
- `turtle` — Gráficos tartaruga
  - Tutorial
    - Iniciando um ambiente da tartaruga
    - Desenho básico
      - Controle da Caneta
      - A posição da tartaruga
    - Criação de padrões algorítmicos
  - Referência
    - Métodos e funções Turtle
      - Movimento e desenho
      - Fala o estado de Turtle
      - Configurações de medida
      - Controle da Caneta
        - Estado do Desenho
        - Controle da Cor
        - Preenchimento
        - Mais sobre o Controle do Desenho
      - Estado da tartaruga
        - Visibilidade
        - Aparência
      - Eventos Utilizados
      - Métodos Especiais da Tartaruga
      - Formas compostas
    - Métodos e funções de janela
      - Controle da Janela
      - Controle da animação
      - Usando os eventos de tela
      - Métodos de entrada
      - Configurações e métodos especiais
      - Métodos apenas de janela
    - Classes Públicas
    - Exceções
  - Guias de como fazer
    - Inicie e encerre o preenchimento automaticamente
    - Use o espaço de nomes do módulo `turtle`

- Use o módulo gráfico de tartaruga dentro de um script com código específico
- Use o módulo gráficos da tartaruga orientado a objetos
- Como usar a Ajuda
- Tradução de docstrings em diferentes idiomas
- Como configurar Screen and Turtles
- Explicação
- `turtledemo` — Scripts de Demonstração

- Ferramentas de Desenvolvimento
  - `typing` — Suporte para dicas de tipo
    - Especificação para o sistema de tipos do Python
    - Apelidos de tipo
    - NewType
    - Anotações de objetos chamáveis
    - Genéricos
    - Anotando tuplas
    - O tipo de objetos de classe
    - Anotando geradores e corrotinas
    - Tipos genéricos definidos pelo usuário
    - O tipo `Any`
    - Subtipagem nominal vs estrutural
    - Conteúdo do módulo
      - Tipos primitivos especiais
        - Tipos especiais
        - Formas especiais
        - Criando tipos genéricos e apelidos de tipo
        - Outras diretivas especiais
      - Protocolos
      - ABCs e protocolos para trabalhar com E/S
      - Funções e decoradores
      - Auxiliares de introspecção
      - Constante
      - Apelidos descontinuados
        - Apelidos de tipos embutidos
        - Apelidos para tipos em `collections`
        - Apelidos de outros tipos concretos

- Apelidos de contêineres ABC em `collections.abc`.
- Apelidos para ABCs assíncronas em `collections.abc`
- Apelidos para outros ABCs em `collections.abc`.
- Apelidos de ABCs da `contextlib`
- Cronograma de descontinuação dos principais recursos
- `pydoc` — Gerador de documentação e sistema de ajuda online
- Modo de Desenvolvimento do Python
  - Efeitos do Modo de Desenvolvimento do Python
  - Exemplo de ResourceWarning
  - Exemplo de erro de descritor de arquivo inválido
- `doctest` — Teste exemplos interativos de Python
  - Uso simples: verificando exemplos em Docstrings
  - Utilização comum: Verificando exemplos em um arquivo texto
  - Uso na linha de comando
  - Como ele funciona
    - Quais docstrings são examinadas?
    - Como os exemplos de docstrings são reconhecidos?
    - Qual é o contexto de execução?
    - E quanto às exceções?
    - Flags opcionais
    - Directives
    - Avisos
  - Basic API
  - API do Unittest
  - Advanced API
    - DocTest Objects
    - Example Objects
    - DocTestFinder objects
    - DocTestParser objects
    - TestResults objects
    - DocTestRunner objects

        - OutputChecker objects
    - Depuração
    - Soapbox
- `unittest` — Framework de testes unitários
    - Exemplo Básico
    - Interface de Linha de Comando
        - Opções de linha de comando
    - Test Discovery
    - Organizando código teste
    - Reutilizando códigos de teste antigos
    - Ignorando testes e falhas esperadas
    - Distinguindo iterações de teste utilizando subtestes
    - Classes e funções
        - Casos de teste
        - Grouping tests
        - Carregando e executando testes
            - load_tests Protocol
    - Classes e Módulos de Definição de Contexto
        - setUpClass and tearDownClass
        - setUpModule and tearDownModule
    - Tratamento de sinal
- `unittest.mock` — mock object library
    - Guia Rápido
    - A classe Mock
        - Fazendo chamadas
        - Deletando Atributos
        - Nomes de Mock e o atributo *name*
        - Anexando Mocks como Atributos
    - Os criadores de patches
        - patch
        - patch.object
        - patch.dict
        - patch.multiple
        - métodos do patch: *start* e *stop*
        - patch de embutidos
        - TEST_PREFIX
        - Aninhando Decoradores Patch

- Onde fazer patch
- Patching Descriptors and Proxy Objects
- MagicMock and magic method support
  - Simulando Métodos Mágicos
  - Magic Mock
- Ajudantes
  - sentinel
  - DEFAULT
  - chamada
  - create_autospec
  - ANY
  - FILTER_DIR
  - mock_open
  - Especificação automática
  - Vedando mocks
- Order of precedence of `side_effect`, `return_value` and *wraps*

- `unittest.mock` — primeiros passos
  - Usando Mock
    - Mock patching methods
    - Mock for method calls on an object
    - Mocking classes
    - Nomeando os mocks
    - Tracking all calls
    - Setting return values and attributes
    - Raising exceptions with mocks
    - Side effect functions and iterables
    - Mocking asynchronous iterators
    - Mocking asynchronous context manager
    - Creating a mock from an existing object
    - Using side_effect to return per file content
  - Patch decorators
  - Further examples
    - Mocking chained calls
    - Partial mocking
    - Mocking a generator method
    - Applying the same patch to every test method

        - Mocking unbound methods
        - Checking multiple calls with mock
        - Coping with mutable arguments
        - Nesting patches
        - Mocking a dictionary with MagicMock
        - Mock subclasses and their attributes
        - Mocking imports with patch.dict
        - Tracking order of calls and less verbose call assertions
        - More complex argument matching
    - `test` — Pacote de Testes de Regressão do Python
        - Writing Unit Tests for the `test` package
        - Executando testes usando a interface de linha de comando
    - `test.support` — Utilities for the Python test suite
    - `test.support.socket_helper` — Utilities for socket tests
    - `test.support.script_helper` — Utilities for the Python execution tests
    - `test.support.bytecode_helper` — Support tools for testing correct bytecode generation
    - `test.support.threading_helper` — Utilities for threading tests
    - `test.support.os_helper` — Utilities for os tests
    - `test.support.import_helper` — Utilities for import tests
    - `test.support.warnings_helper` — Utilities for warnings tests
- Depuração e perfilamento
    - Tabela de eventos de auditoria
    - `bdb` — Framework do depurador
    - `faulthandler` — Dump the Python traceback
        - Dumping the traceback
        - Dumping the C stack
            - C Stack Compatibility
        - Fault handler state
        - Dumping the tracebacks after a timeout
        - Dumping the traceback on a user signal
        - Issue with file descriptors

- Exemplo
- `pdb` — O depurador do Python
  - Interface de linha de comando
  - Comandos de depuração
- `profiling` — Python profilers
  - Introduction to profiling
  - Choosing a profiler
    - When to use statistical sampling
    - When to use deterministic tracing
  - Quick start
    - Statistical profiling
    - Deterministic profiling
  - Understanding profile output
  - Legacy compatibility
    - `profiling.tracing` — Deterministic profiler
      - What is deterministic profiling?
      - Command-line interface
      - Programmatic usage examples
        - Basic profiling
        - Using the `Profile` class
      - Module reference
      - Using a custom timer
      - Limitations
    - `profiling.sampling` — Statistical profiler
      - What is statistical profiling?
        - How time is estimated
        - When to use a different approach
      - Quick examples
      - Commands
        - The `run` command
        - The `attach` command
        - The `dump` command
        - The `replay` command
        - Profiling in production
        - Platform requirements
        - Version compatibility
      - Sampling configuration

    - Sampling rate and duration
    - Thread selection
    - Blocking mode
    - Special frames
    - Opcode-aware profiling
    - Real-time statistics
    - Subprocess profiling
    - Sampling efficiency
- Profiling modes
    - Wall-clock mode
    - CPU mode
    - Comparing wall-clock and CPU profiles
    - GIL mode
    - Exception mode
- Output formats
    - pstats format
    - Collapsed stacks format
    - Flame graph format
        - Differential flame graphs
    - Gecko format
    - Heatmap format
    - Binary format
- Record and replay workflow
- Live mode
    - Keyboard commands
- Async-aware profiling
    - Async modes
    - Task markers and stack reconstruction
    - Option restrictions
- Command-line interface
    - Global options
    - Dump options
    - Sampling options
    - Mode options
    - Output options
    - pstats display options
    - Run command options

- `pstats` — Statistics for profilers
  - Reading and displaying profile data
    - Working with statistics
    - Filtering output
    - Analyzing call relationships
    - Combining multiple profiles
  - The `Stats` class
  - Interface de linha de comando
- `timeit` — Mede o tempo de execução de pequenos trechos de código
  - Basic examples
  - Python interface
  - Interface de linha de comando
  - Exemplos
- `trace` — Rastreia ou acompanha a execução de instruções Python
  - Uso na linha de comando
    - Opções principais
    - Modificadores
    - Filtros
  - Interface programática
- `tracemalloc` — Trace memory allocations
  - Exemplos
    - Exibe o top 10
    - Compute differences
    - Get the traceback of a memory block
    - Pretty top
      - Record the current and peak size of all traced memory blocks
  - API
    - Funções
    - DomainFilter
    - Filter
    - Frame
    - Snapshot
    - Statistic
    - StatisticDiff

- Trace
- Traceback
- Empacotamento e Distribuição de Software
  - `ensurepip` — Inicialização do instalador do `pip`
    - Interface de linha de comando
    - API do módulo
  - `venv` — Criação de ambientes virtuais
    - Criando ambientes virtuais
    - Como funcionam os venvs
    - API
    - Um exemplo de extensão de `EnvBuilder`
  - `zipapp` — Gerencia arquivos zip executáveis do Python
    - Exemplo básico
    - Interface de linha de comando
    - API do Python
    - Exemplos
    - Especificando o interpretador
    - Criando aplicações autônomas com zipapp
      - Advertência
    - O formato de arquivo da aplicação zip Python
- Serviços de Tempo de Execução Python
  - `sys` — Parâmetros e funções específicas do sistema
  - `sys.monitoring` — Monitoramento de eventos de execução
    - Identificadores de ferramenta
      - Registro e uso de ferramentas
    - Eventos
      - Eventos locais
      - Evento descontinuado
      - Eventos auxiliares
      - Outros eventos
      - O evento STOP_ITERATION
    - Ativação e desativação de eventos
      - Definir eventos globalmente
      - Eventos por objeto código
      - Desativação de eventos
    - Registro de funções de retorno de chamada
      - Argumentos da função de retorno de chamada

- `sysconfig` — Fornece acesso às informações de configuração do Python
  - Variáveis de configuração
  - Caminhos de instalação
  - Esquema de usuário
    - `posix_user`
    - `nt_user`
    - `osx_framework_user`
  - Esquema de home
    - `posix_home`
  - Esquema de prefixo
    - `posix_prefix`
    - `nt`
  - Funções de caminho de instalação
  - Outras funções
  - Uso na linha de comando
- `builtins` — Objetos embutidos
- `__main__` — Ambiente de código principal
  - `__name__ == '__main__'`
    - O que é o “ambiente de código principal”?
    - Uso idiomático
    - Considerações sobre pacotes
  - `__main__.py` em pacotes Python
    - Uso idiomático
  - `import __main__`
- `warnings` — Controle de avisos
  - Categorias de avisos
  - O filtro de avisos
    - Critérios de supressão de avisos repetidos
    - Descrevendo filtros de aviso
    - Filtro de avisos padrão
    - Overriding the default filter
  - Temporarily Suppressing Warnings
  - Testing Warnings
  - Updating Code For New Versions of Dependencies
  - Funções disponíveis
  - Gerenciadores de contexto disponíveis

    - Concurrent safety of Context Managers
- `dataclasses` — Data Classes
    - Conteúdo do módulo
    - Processamento pós-inicialização
    - Variáveis de classe
    - Variáveis de inicialização apenas
    - Frozen instances
    - Herança
    - Re-ordering of keyword-only parameters in `__init__()`
    - Funções padrão de fábrica
    - Valores padrão mutáveis
    - Descriptor-typed fields
- `contextlib` — Utilitários para contextos da instrução `with`
    - Utilitários
    - Exemplos e receitas
        - Supporting a variable number of context managers
        - Catching exceptions from `__enter__` methods
        - Cleaning up in an `__enter__` implementation
        - Replacing any use of `try-finally` and flag variables
        - Using a context manager as a function decorator
    - Single use, reusable and reentrant context managers
        - Reentrant context managers
        - Gerenciadores de contexto reutilizáveis
- `abc` — Classes base abstratas
- `atexit` — Manipuladores de saída
    - `atexit` Example
- `traceback` — Imprime ou recupera um traceback da pilha
    - Funções de nível de módulo
    - `TracebackException` Objects
    - `StackSummary` Objects
    - `FrameSummary` Objects
    - Examples of Using the Module-Level Functions
    - Examples of Using `TracebackException`
- `__future__` — Definições de instruções future
    - Conteúdo do módulo
- `gc` — Interface para o coletor de lixo

- `inspect` — Inspecione objetos vivos
  - Tipos e membros
  - Retrieving source code
  - Introspecting callables with the Signature object
  - Classes e funções
  - A pilha to interpretador
  - Fetching attributes statically
  - Current State of Generators, Coroutines, and Asynchronous Generators
  - Code Objects Bit Flags
  - Buffer flags
  - Interface de linha de comando
- `annotationlib` — Funcionalidade para introspecção de anotações
  - Semânticas de anotação
  - Classes
  - Funções
  - Receitas
    - Usando anotações em uma metaclasse
    - Creating a custom callable annotate function
  - Limitações do formato `STRING`
  - Limitações do formato `FORWARDREF`
  - Implicações de segurança de anotações de introspecção
- `site` — Gancho de configuração específico do site
  - Path extensions (`.pth` files)
  - Startup entry points (`.start` files)
  - Startup file examples
  - Migrating from `import` lines in `.pth` files to `.start` files
  - `sitecustomize`
  - `usercustomize`
  - Configuração Readline
  - Conteúdo do módulo
  - Interface de linha de comando
- Interpretadores Python Personalizados
  - `code` — Classes bases do interpretador
    - Objetos de interpretador interativo

        - Objetos de console Interativo
    - `codeop` — Compila código Python
- Importando módulos
    - `zipimport` - Importa módulos de arquivos Zip
        - Objetos zipimporter
        - Exemplos
    - `pkgutil` — Utilitário de extensão de pacote
    - `modulefinder` — Procura módulos usados por um script
        - Exemplo de uso de `ModuleFinder`
    - `runpy` — Localizando e executando módulos Python
    - `importlib` — A implementação de `import`
        - Introdução
        - Funções
        - `importlib.abc` – Abstract base classes related to import
        - `importlib.machinery` – Importers and path hooks
        - `importlib.util` – Utility code for importers
        - Exemplos
            - Importing programmatically
            - Checando se o módulo pode ser importado
            - Importa o arquivo de origem diretamente
            - Implementing lazy imports
            - Setting up an importer
            - Approximating `importlib.import_module()`
    - `importlib.resources` – Leitura, abertura e acesso a recursos de pacotes
        - API funcional
    - `importlib.resources.abc` – Classes base abstratas para recursos
    - `importlib.metadata` – Acessando metadados do pacote
        - Visão Geral
        - API funcional
            - Pontos de entrada
            - Metadados de distribuição
            - Versões de distribuição
            - Arquivos de distribuição
            - Requisitos de distribuição

        - Mapeando importação pra pacotes de distribuição
    - Distribuições
    - Descoberta de distribuição
    - Implementando provedores personalizados
        - Exemplo
  - A inicialização do caminho de pesquisa de módulos `sys.path`
    - Ambientes virtuais
    - Arquivos _pth
    - Python embarcado
- Serviços da Linguagem Python
  - `ast` — Árvores de sintaxe abstrata
    - Gramática abstrata
    - Classes de nós
        - Nós raízes
        - Literais
        - Variáveis
        - Expressões
            - Subscrição
            - Compreensões
        - Instruções
            - Importações
        - Fluxo de controle
        - Correspondência de padrões
        - Anotações de tipos
        - Parâmetros de tipo
        - Definições de função e classe
        - Async e await
    - Auxiliares de `ast`
    - Sinalizadores do compilador
    - Uso na linha de comando
  - `symtable` — Acesso a tabela de símbolos do compilador
    - Generating Symbol Tables
    - Examining Symbol Tables
    - Uso na linha de comando
  - `token` — Constantes usadas com árvores de análises do Python

- `keyword` — Testando palavras reservadas do Python
- `tokenize` — Tokenizador para código-fonte Python
  - Tokenizing input
  - Uso na linha de comando
  - Exemplos
- `tabnanny` — Detecção de indentação ambígua
- `pyclbr` — Suporte a navegador de módulos do Python
  - Objetos Function
  - Objetos Class
- `py_compile` — Compila arquivos fonte do Python
  - Interface de Linha de Comando
- `compileall` — Compilar bibliotecas do Python para bytecode
  - Uso na linha de comando
  - Funções públicas
- `dis` — Disassembler de bytecode do Python
  - Interface de linha de comando
  - Análise de bytecode
  - Funções de análise
  - Instruções em bytecode do Python
  - Opcode collections
- `pickletools` — Ferramentas para desenvolvedores pickle
  - Uso na linha de comando
    - Opções de linha de comando
  - Interface programática

- Serviços Específicos do MS Windows
  - `msvcrt` — Rotinas úteis do runtime MS VC++
    - Operações com arquivos
    - E/S de console
    - Outras funções
  - `winreg` — Acesso aos registros do Windows
    - Funções
    - Constantes
      - Constantes HKEY_*
      - Direitos de acesso
        - Específico de 64 bits
      - Tipos de valores

        - Objetos identificador de registro
    - `winsound` — Interface de reprodução de som para o Windows
        - Funções
        - Constantes
- Serviços específicos do Unix
    - `shlex` — Simple lexical analysis
        - shlex Objects
        - Regras de análise
        - Improved Compatibility with Shells
    - `posix` — As chamadas de sistema mais comuns do POSIX
        - Suporte a arquivos grandes
        - Conteúdo notável do módulo
    - `pwd` — A senha do banco de dados
    - `grp` — O banco de dados de grupos
    - `termios` — Controle de tty no estilo POSIX
        - Exemplo
    - `tty` — Funções de controle de terminal
    - `pty` — Utilitários de pseudoterminal
        - Exemplo
    - `fcntl` — as chamadas de sistema `fcntl` e `ioctl`
    - `resource` — Informação de uso de recursos
        - Limites de recursos
        - Uso de recursos
    - `syslog` — Rotinas da biblioteca syslog do Unix
        - Exemplos
            - Exemplo simples
- Interface de linha de comando (CLI) de módulos
- Módulos substituídos
    - `getopt` — Analisador sintático no estilo C para opções de linha de comando
    - `profile` — Pure Python profiler
        - Migration
        - `profile` and `profiling.tracing` module reference
        - Differences from `profiling.tracing`
        - What is deterministic profiling?
        - Limitações

            - Calibration
            - Using a custom timer
    - Módulos removidos
    - Considerações de segurança
- Estendendo e incorporando o interpretador Python
    - Ferramentas de terceiros recomendadas
    - Tutorial da API C
    - Guias para tópicos intermediários
    - Incorporando o tempo de execução do CPython em uma aplicação maior
- Manual de referência da API C/Python
    - Introdução
        - Compatibilidade com a versão da linguagem
        - Padrões de codificação
        - Arquivos de inclusão
            - System includes
        - Macros úteis
            - Docstring macros
            - General utility macros
                - Numeric utilities
                - Assertion utilities
                - Type size utilities
                - Macro definition utilities
            - Declaration utilities
            - Outdated macros
        - Objetos, tipos e contagens de referências
            - Contagens de referências
                - Detalhes da contagem de referências
            - Tipos
        - Exceções
        - Incorporando Python
        - Compilações de depuração
        - Ferramentas de terceiros recomendadas
    - C API and ABI Stability
        - API C Instável
        - Stable Application Binary Interfaces
            - Compiling for Stable ABI

        - Stable ABI Scope and Performance
        - Stable ABI Caveats
    - Considerações da plataforma
    - ABI Checking
    - Conteúdo da API Limitada
- A camada de nível muito alto
    - Símbolos iniciais disponíveis
    - Stack Effects
- Contagem de referências
- Tratamento de exceções
    - Impressão e limpeza
    - Levantando exceções
    - Emitindo avisos
    - Consultando o indicador de erro
    - Tratamento de sinal
    - Classes de exceção
    - Objeto Exceção
    - Objetos de exceção Unicode
    - Controle de recursão
    - Exception and warning types
        - Tipos de exceção
        - OSError aliases
        - Warning types
    - Tracebacks
- Definindo módulos de extensão
    - Extension export hook
    - Inicialização multifásica
    - Várias instâncias de módulo
    - `PyInit` function
        - Inicialização monofásica legada
- Definition slots
    - Convenience macros
    - Common slot IDs
- Utilitários
    - Utilitários do sistema operacional
    - Funções de sistema
    - Controle de processos

- Importando módulos
- Suporte a *marshalling* de dados
- Análise de argumentos e construção de valores
  - Análise de argumentos
    - Strings e buffers
    - Números
    - Outros objetos
    - Funções da API
  - Construindo valores
- Conversão e formação de strings
- Classificação e conversão de caracteres
- API do PyHash
- Reflexão
- Registro de codec e funções de suporte
  - API de pesquisa de codec
  - API de registro de tratamentos de erros de decodificação Unicode
  - Variáveis utilitárias de codec
- API C PyTime
  - Tipos
  - Funções de relógio
  - Funções de relógio brutas
  - Funções de conversão
- Suporte a Mapas do Perf

- Camada de Objetos Abstratos
  - Protocolo de objeto
  - Protocolo de chamada
    - O protocolo *tp_call*
    - O protocolo vectorcall
      - Controle de recursão
      - API de suporte à chamada de vetores
    - API de chamada de objetos
    - API de suporte a chamadas
  - Protocolo de número
  - Protocolo de sequência
  - Protocolo de mapeamento
  - Protocolo Iterador

- Protocolo de buffer
  - Estrutura de Buffer
  - Tipos de solicitação do buffer
    - campos independentes do pedido
    - apenas em formato
    - forma, avanços, suboffsets
    - requisições contíguas
    - requisições compostas
  - Vetores Complexos
    - Estilo NumPy: forma e avanços
    - Estilo-PIL: forma, avanços e suboffsets
  - Funções relacionadas ao Buffer
- Camada de Objetos Concretos
  - Objetos Fundamentais
    - Objetos tipo
      - Creating Heap-Allocated Types
        - Type slot IDs
      - Soft-deprecated API
    - O Objeto `None`
  - Objetos Numéricos
    - Objetos Inteiros
      - Export API
      - PyLongWriter API
      - Deprecated API
    - Objetos Booleanos
    - Objetos de ponto flutuante
      - As funções Pack e Unpack
        - Funções Pack
        - Funções de Unpack
    - Objetos números complexos
      - Números complexos como estruturas C.
  - Objetos Sequência
    - Objetos Bytes
    - PyBytesWriter
      - Create, Finish, Discard
      - High-level API
      - Getters

- Low-level API
- Objetos byte array
  - Macros para verificação de tipo
  - Funções diretas da API
  - Macros
- Objetos Unicode e Codecs
  - Objetos Unicode
    - Tipo Unicode
    - Propriedade de caracteres Unicode
    - Creating and accessing Unicode strings
    - Locale Encoding
    - File System Encoding
    - wchar_t Support
  - Built-in Codecs
    - Generic Codecs
    - UTF-8 Codecs
    - UTF-32 Codecs
    - UTF-16 Codecs
    - UTF-7 Codecs
    - Unicode-Escape Codecs
    - Raw-Unicode-Escape Codecs
    - Latin-1 Codecs
    - ASCII Codecs
    - Character Map Codecs
    - MBCS codecs for Windows
  - Methods and Slot Functions
  - PyUnicodeWriter
  - Deprecated API
- Objeto tupla
- Objetos sequência de estrutura
- Objeto List
- Coleções
  - Dictionary objects
    - Objetos de visão de dicionário
    - Frozen dictionary objects
    - Ordered dictionaries
  - Objeto Set

        - Deprecated API
- Objetos Função
    - Objetos Função
    - Objetos de Método de Instância
    - Objetos método
    - Objeto célula
    - Objetos código
    - Sinalizadores de objetos código
    - Informação adicional
- Outros Objetos
    - File objects
        - Soft-deprecated API
    - Objetos do Módulo
        - Module definition
            - Metadata slots
            - Feature slots
            - Creation and initialization slots
        - Module state
            - Slots for defining module state
            - Module token
        - Creating extension modules dynamically
        - Module definition struct
        - Support functions
            - Module lookup (single-phase initialization)
    - Objetos Iteradores
        - Objetos intervalo
        - Tipos de iteradores embutidos
        - Outros objetos iteradores
    - Objetos descritores
        - Descritores embutidos
    - Objetos Slice
        - Objeto Ellipsis
    - Objetos MemoryView
    - Objetos buffer serializados com pickle
    - Objetos referência fraca
    - Capsules
    - Objetos sentinel

- Objetos quadro
  - Frame locals proxies
  - Legacy local variable APIs
  - Internal frames
- Objetos Geradores
  - Objetos geradores assíncronos
  - Deprecated API
- Objetos corrotina
- Objetos de variáveis de contexto
- Objetos de indicação de tipos
- API C para módulos de extensão
  - API C de curses
  - Dados internos
  - Objetos DateTime
  - Dados internos
- Interpreter initialization and finalization
  - Before Python initialization
  - Variáveis de configuração global
  - Inicializando e encerrando o interpretador
  - Cautions regarding interpreter finalization
  - Interpreter views
  - Parâmetros de todo o processo
- Thread states and the global interpreter lock
  - Desvinculação do estado de thread do código de extensão
    - APIs
  - Using the C API from foreign threads
    - Reusing a thread state across repeated calls
    - Attaching/detaching thread states
  - GIL-state APIs
  - Cuidados com o uso de fork()
  - APIs de alto nível
  - APIs de baixo nível
- Asynchronous notifications
- Operating system thread APIs
- Primitivas de sincronização
  - Python critical section API
    - Low-level critical section API

    - Legacy locking APIs
- Thread-local storage support
    - Thread-specific storage API
    - Dynamic allocation
    - Métodos
    - Legacy APIs
- Multiple interpreters in a Python process
    - A per-interpreter GIL
    - Bugs and caveats
    - APIs de alto nível
    - APIs de baixo nível
    - Advanced debugger support
- Profiling and tracing
- Reference tracing
- Configuração de inicialização do Python
    - API C PyInitConfig
        - Exemplo
        - Criar configuração
        - Tratamento de erros
        - Opções de get
        - Opções de set
        - Módulo
        - Inicializar o Python
    - Opções de configuração
    - API de configuração Python em tempo de execução
    - API C PyConfig
        - Exemplo
        - PyWideStringList
        - PyStatus
        - PyPreConfig
        - Pré-inicializar Python com PyPreConfig
        - PyConfig
        - Initialization with PyConfig
        - Isolated Configuration
        - Configuração do Python
        - Python Path Configuration
    - Py_GetArgcArgv()

- Multi-Phase Initialization Private Provisional API
- Gerenciamento de Memória
  - Visão Geral
  - Allocator Domains
  - Raw Memory Interface
  - Interface da Memória
    - Apelidos descontinuados
  - Alocadores de objeto
  - Alocadores de memória padrão
  - Alocadores de memória
  - Debug hooks on the Python memory allocators
  - The pymalloc allocator
    - Customize pymalloc Arena Allocator
  - The mimalloc allocator
  - tracemalloc C API
  - Exemplos
- Suporte a implementação de Objetos
  - Alocando objetos na heap
    - Apelidos suavemente descontinuados
  - Ciclo de vida do objeto
    - Eventos de vida
    - Destruição de isolado cíclico
    - Funções
  - Estruturas comuns de objetos
    - Macros e tipos de objetos base
    - Implementing functions and methods
    - Accessing attributes of extension types
      - Member flags
      - Member types
      - Defining Getters and Setters
  - Estruturas de objetos tipo
    - Referências rápidas
      - “slots tp”
      - sub-slots
      - slot typedefs
    - PyTypeObject Definition
    - PyObject Slots

            - PyVarObject Slots
            - PyTypeObject Slots
            - Static Types
            - Tipos no heap
            - Number Object Structures
            - Mapping Object Structures
            - Sequence Object Structures
            - Buffer Object Structures
            - Async Object Structures
            - Slot Type typedefs
            - Exemplos
        - Suporte a Coleta Cíclica de Lixo
            - Traversal
                - Traversal-safe functions
                - “DuringGC” functions
            - Controlando o estado do coletor de lixo
            - Querying Garbage Collector State
    - API e Versionamento de ABI
        - Constantes de versão de tempo de construção
        - Versão de tempo de execução
        - Macros de empacotamento de bits
    - API C de monitoramento
    - Gerando eventos de execução
        - Gerenciando o Estado de um Monitoramento
- Instalando módulos Python
    - Termos chave
    - Uso básico
    - Como eu …?
        - … instalo pacotes apenas para o usuário atual?
        - … instalo pacotes científicos do Python?
        - … trabalho com várias versões do Python instaladas em paralelo?
    - Problemas comuns de instalação
        - Instalando no sistema Python no Linux
        - Pip não instalado
        - Instalando extensões binárias
- Python HOWTOs

- Perguntas Frequentes Sobre Python
  - Python Geral
    - Informações gerais
    - Python no mundo real
  - FAQ sobre programação
    - General questions
    - Core language
    - Números e strings
    - Desempenho
    - Sequences (tuples/lists)
    - Objetos
    - Módulos
  - FAQ sobre design e histórico
    - Por que o Python usa indentação para agrupamento de instruções?
    - Por que eu estou recebendo resultados estranhos com simples operações aritméticas?
    - Por que o cálculo de pontos flutuantes são tão imprecisos?
    - Por que strings do Python são imutáveis?
    - Por que o 'self' deve ser usado explicitamente em definições de método e chamadas?
    - Por que não posso usar uma atribuição em uma expressão?
    - Por que o Python usa métodos para algumas funcionalidades (ex: lista.index()) mas funções para outras (ex: len(lista))?
    - Por que o join() é um método de string em vez de ser um método de lista ou tupla?
    - Quão rápidas são as exceções?
    - Por que não existe uma instrução de switch ou case no Python?
    - Você não pode emular threads no interpretador em vez de confiar em uma implementação de thread específica do sistema operacional?
    - Por que expressões lambda não podem conter instruções?
    - O Python pode ser compilado para linguagem de máquina, C ou alguma outra linguagem?
    - Como o Python gerencia memória?

    - Por que o CPython não usa uma forma mais tradicional de esquema de coleta de lixo?
    - Por que toda memória não é liberada quando o CPython fecha?
    - Por que existem tipos de dados separados para tuplas e listas?
    - Como as listas são implementadas no CPython?
    - Como são os dicionários implementados no CPython?
    - Por que chaves de dicionário devem ser imutáveis?
    - Por que lista.sort() não retorna a lista ordenada?
    - Como você especifica e aplica um spec de interface no Python?
    - Por que não há goto?
    - Por que strings brutas (r-strings) não podem terminar com uma contrabarra?
    - Por que o Python não tem uma instrução “with” para atribuição de atributos?
    - Por que os geradores não suportam a instrução with?
    - Por que dois pontos são necessários para as instruções de if/while/def/class?
    - Por que o Python permite vírgulas ao final de listas e tuplas?
  - FAQ de Bibliotecas e Extensões
    - Questões gerais sobre bibliotecas
    - Tarefas comuns
    - Threads
    - Entrada e Saída
    - Programação Rede / Internet
    - Base de Dados
    - Matemáticos e Numéricos
  - FAQ sobre Extensão/Incorporação
    - Posso criar minhas próprias funções em C?
    - Posso criar minhas próprias funções em C++?
    - Escrever C é difícil; há alguma alternativa?
    - Como posso executar instruções arbitrárias de Python a partir de C?
    - Como posso executar e obter o resultado de uma expressão Python arbitrária a partir de C?

    - Como extraio valores em C a partir de um objeto Python?
    - Como posso utilizar Py_BuildValue() para criar uma tupla de comprimento arbitrário?
    - Como eu chamo um método de um objeto a partir do C?
    - Como posso capturar a saída da função PyErr_Print() (ou qualquer outra coisa que escreva para stdout/stderr)?
    - Como faço para acessar a partir do C um módulo escrito em Python?
    - Como posso interagir com objetos C++ a partir do Python?
    - Adicionei um módulo usando o arquivo de Setup e o make falha; por quê?
    - Como eu depuro uma extensão?
    - Quero compilar um módulo Python no meu sistema Linux, mas alguns arquivos estão faltando. Por quê?
    - Como posso distinguir “entrada incompleta” de “entrada inválida”?
    - Como encontro os símbolos __builtin_new ou __pure_virtual não-definidos no g++?
    - Posso criar uma classe de objetos com alguns métodos implementados em C e outros em Python (por exemplo, via herança)?
  - Python no Windows
    - Como faço para executar um programa Python no Windows?
    - Como eu faço para criar programas Python executáveis?
    - Por que Python às vezes demora tanto para iniciar?
    - Como eu faço para criar um executável a partir de um código Python?
    - Um arquivo `*.pyd` é o mesmo que um DLL?
    - Como eu posso embutir Python dentro de uma aplicação do Windows?
    - Como eu impeço editores de adicionarem tabulações na minha source do Python?
    - Como faço para verificar uma tecla pressionada sem bloquear?
    - Como resolvo o erro da api-ms-win-crt-runtime-l1-1-0.dll ausente?
  - FAQ da Interface Gráfica do Usuário

        - Perguntas Gerais sobre a GUI
        - Quais toolkits de GUI existem para o Python?
        - Perguntas do Tkinter
    - FAD de “Por que o Python está instalado em meu computador?”
        - O que é Python?
        - Porque Python está instalado em minha máquina?
        - Eu posso apagar o Python?
- Descontinuações
    - Remoção pendente no Python 3.16
    - Remoção pendente no Python 3.17
    - Remoção pendente no Python 3.18
    - Remoção pendente no Python 3.19
    - Remoção pendente no Python 3.20
    - Remoção pendente em versões futuras
    - descontinuações suave
    - Descontinuações na API C
        - Remoção pendente no Python 3.15
        - Remoção pendente no Python 3.16
        - Remoção pendente no Python 3.18
        - Remoção pendente no Python 3.19
        - Remoção pendente no Python 3.20
        - Remoção pendente em versões futuras
- Glossário
- Sobre esta documentação
    - Contribuidores da documentação do Python
- Lidando com bugs
    - Bugs na documentação
    - Usando o rastreador de problemas do Python
    - Começando a contribuir com o Python por conta própria
- Direitos autorais
- História e Licença
    - História do software
    - Termos e condições para acessar ou usar Python
        - PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2
        - ACORDO DE LICENCIAMENTO DA BEOPEN.COM PARA PYTHON 2.0

    - CONTRATO DE LICENÇA DA CNRI PARA O PYTHON 1.6.1
    - ACORDO DE LICENÇA DA CWI PARA PYTHON 0.9.0 A 1.2
    - ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON DOCUMENTATION
  - Licenças e Reconhecimentos para Software Incorporado
    - Mersenne Twister
    - Soquetes
    - Serviços de soquete assíncrono
    - Gerenciamento de cookies
    - Rastreamento de execução
    - Funções UUencode e UUdecode
    - Chamadas de procedimento remoto XML
    - test_epoll
    - kqueue de seleção
    - SipHash24
    - strtod e dtoa
    - OpenSSL
    - expat
    - libffi
    - zlib
    - cfuhash
    - libmpdec
    - Conjunto de testes C14N do W3C
    - mimalloc
    - asyncio
    - Global Unbounded Sequences (GUS)
    - Ligações Zstandard
    - Módulo de perfilamento
    - Pacotes Pixi
    - Banco de dados de caracteres Unicode

O que há de novo no Python

A série “O que há de novo no Python” é uma série de ensaios que nos guia através das mudanças mais importantes entre as principais versões do Python. Ela é imprescindível para aqueles que desejam manter-se atualizados a cada novo lançamento.

- O que há de novo no Python 3.15
  - Resumo – Destaques da versão
  - Novas funcionalidades
  - Outras mudanças na linguagem
  - Console interativo padrão
  - Novos módulos
  - Módulos melhorados
  - Otimizações
  - Removidos
  - Descontinuados
  - Alterações na API C
  - Alterações na construção
  - Portando para o Python 3.15
- O que há de novo no Python 3.14
  - Resumo – Destaques da versão
  - Novas funcionalidades
  - Outras mudanças na linguagem
  - Novos módulos
  - Módulos melhorados
  - Otimizações
  - Removidos
  - Descontinuados
  - Alterações de bytecode do CPython
  - Alterações na API C
  - Alterações na construção
  - Portando para o Python 3.14
  - Alterações notáveis no 3.14.5
- O que há de novo no Python 3.13

    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Módulos e APIs removidas
    - Novas descontinuações
    - Alterações de bytecode do CPython
    - Alterações na API C
    - Mudanças na construção
    - Portando para o Python 3.13
    - Mudanças em teste de regressão
- O que há de novo no Python 3.12
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Novos recursos relacionados a dicas de tipo
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Alterações de bytecode do CPython
    - Ferramentas e daemons
    - Descontinuados
    - Removidos
    - Portando para Python 3.12
    - Mudanças na construção
    - Alterações na API C
- O que há de novo no Python 3.11
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Novos recursos relacionados a dicas de tipo
    - Outras mudanças na linguagem
    - Outras mudanças na implementação do CPython
    - Novos módulos
    - Módulos melhorados
    - Otimizações

    - CPython mais rápido
    - Alterações de bytecode do CPython
    - Descontinuados
    - Pendente remoção no Python 3.12
    - Removidos
    - Portando para Python 3.11
    - Mudanças na construção
    - Alterações na API C
    - Alterações notáveis no 3.11.4
    - Alterações notáveis no 3.11.5
- O que há de novo no Python 3.10
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Novos recursos relacionados a dicas de tipo
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Descontinuados
    - Removidos
    - Portando para Python 3.10
    - Alterações de bytecode do CPython
    - Mudanças na construção
    - Alterações na API C
    - Recursos de segurança notáveis no 3.10.7
    - Recursos de segurança notáveis no 3.10.8
    - Alterações notáveis no 3.10.12
- O que há de novo no Python 3.9
    - Resumo – Destaques da versão
    - Você deve verificar DeprecationWarning no seu código
    - Novas funcionalidades
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Descontinuados
    - Removidos

    - Portando para Python 3.9
    - Mudanças na construção
    - Alterações na API C
    - Alterações notáveis no Python 3.9.1
    - Alterações notáveis no Python 3.9.2
    - Alterações notáveis no Python 3.9.3
    - Alterações notáveis no Python 3.9.5
    - Recursos de segurança notáveis no 3.9.14
    - Alterações notáveis no 3.9.17
- O que há de novo no Python 3.8
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Mudanças a construções e API C
    - Descontinuados
    - Remoção de APIs e recursos
    - Portando para o Python 3.8
    - Alterações notáveis no Python 3.8.1
    - Alterações notáveis no Python 3.8.2
    - Alterações notáveis no Python 3.8.3
    - Alterações notáveis no Python 3.8.8
    - Alterações notáveis no Python 3.8.9
    - Alterações notáveis no Python 3.8.10
    - Alterações notáveis no Python 3.8.10
    - Alterações notáveis no Python 3.8.12
    - Recursos de segurança notáveis no 3.8.14
    - Alterações notáveis no 3.8.17
- O que há de novo no Python 3.7
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Alterações na API C

    - Mudanças na construção
    - Otimizações
    - Outras mudanças na implementação do CPython
    - Comportamento do Python descontinuado
    - Deprecated Python modules, functions and methods
    - Deprecated functions and types of the C API
    - Platform Support Removals
    - Remoção de APIs e recursos
    - Module Removals
    - Windows-only Changes
    - Porting to Python 3.7
    - Notable changes in Python 3.7.1
    - Notable changes in Python 3.7.2
    - Notable changes in Python 3.7.6
    - Notable changes in Python 3.7.10
    - Notable changes in Python 3.7.11
    - Notable security feature in 3.7.14
- What’s New In Python 3.6
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Mudanças a construções e API C
    - Other Improvements
    - Descontinuados
    - Removidos
    - Porting to Python 3.6
    - Notable changes in Python 3.6.2
    - Notable changes in Python 3.6.4
    - Notable changes in Python 3.6.5
    - Notable changes in Python 3.6.7
    - Notable changes in Python 3.6.10
    - Notable changes in Python 3.6.13
    - Notable changes in Python 3.6.14
- O que há de novo no Python 3.5

    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Outras mudanças na linguagem
    - Novos módulos
    - Módulos melhorados
    - Other module-level changes
    - Otimizações
    - Mudanças a construções e API C
    - Descontinuados
    - Removidos
    - Porting to Python 3.5
    - Notable changes in Python 3.5.4
- O que há de novo no Python 3.4
    - Resumo – Destaques da versão
    - Novas funcionalidades
    - Novos módulos
    - Módulos melhorados
    - CPython Implementation Changes
    - Descontinuados
    - Removidos
    - Porting to Python 3.4
    - Alterado em 3.4.3
- O que há de novo no Python 3.3
    - Resumo – Destaques da versão
    - PEP 405: Ambientes Virtuais
    - PEP 420: Implicit Namespace Packages
    - PEP 3118: New memoryview implementation and buffer protocol documentation
    - PEP 393: Flexible String Representation
    - PEP 397: Python Launcher for Windows
    - PEP 3151: Reworking the OS and IO exception hierarchy
    - PEP 380: Syntax for Delegating to a Subgenerator
    - PEP 409: Suppressing exception context
    - PEP 414: Explicit Unicode literals
    - PEP 3155: Qualified name for classes and functions
    - PEP 412: Key-Sharing Dictionary
    - PEP 362: Function Signature Object

    - PEP 421: Adding sys.implementation
    - Using importlib as the Implementation of Import
    - Outras mudanças na linguagem
    - A Finer-Grained Import Lock
    - Builtin functions and types
    - Novos módulos
    - Módulos melhorados
    - Otimizações
    - Mudanças a construções e API C
    - Descontinuados
    - Porting to Python 3.3
- O que há de novo no Python 3.2
    - PEP 384: Defining a Stable ABI
    - PEP 389: Argparse Command Line Parsing Module
    - PEP 391: Dictionary Based Configuration for Logging
    - PEP 3148: The `concurrent.futures` module
    - PEP 3147: PYC Repository Directories
    - PEP 3149: ABI Version Tagged .so Files
    - PEP 3333: Python Web Server Gateway Interface v1.0.1
    - Outras mudanças na linguagem
    - New, Improved, and Deprecated Modules
    - Multi-threading
    - Otimizações
    - Unicode
    - Codecs
    - Documentação
    - IDLE
    - Code Repository
    - Mudanças a construções e API C
    - Porting to Python 3.2
- O que há de novo no Python 3.1
    - PEP 372: Dicionários ordenados
    - PEP 378: Especificador de formato para separador de milhares
    - Outras mudanças na linguagem
    - New, Improved, and Deprecated Modules
    - Otimizações
    - IDLE

    - Mudanças a construções e API C
    - Porting to Python 3.1
- O que há de novo no Python 3.0
    - Obstáculos comuns
    - Visão geral das alterações da sintaxe
    - Mudanças já presentes no Python 2.6
    - Mudanças na biblioteca
    - **PEP 3101**: Uma nova abordagem para formatação de strings
    - Mudanças para exceções
    - Outras mudanças diversas
    - Mudanças a construções e API C
    - Desempenho
    - Portando para o Python 3.0
- O que há de novo no Python 2.7
    - O futuro para o Python 2.x
    - Mudanças no tratamento de avisos de descontinuação
    - Recursos do Python 3.1
    - PEP 372: Adicionando um dicionário ordenado a coleções
    - PEP 378: Especificador de formato para separador de milhares
    - PEP 389: O módulo argparse para analisar linhas de comando
    - PEP 391: Configuração baseada em dicionário para logging
    - PEP 3106: Views de dicionário
    - PEP 3137: O objeto memoryview
    - Outras mudanças na linguagem
    - Módulos Novos ou Aprimorados
    - Mudanças a construções e API C
    - Outras alterações e correções
    - Portando para Python 2.7
    - Novos recursos adicionados às versões de manutenção do Python 2.7
    - Reconhecimentos
- O que há de novo no Python 2.6
    - Python 3.0
    - Changes to the Development Process
    - PEP 343: The ‘with’ statement
    - PEP 366: Explicit Relative Imports From a Main Module
    - PEP 370: Per-user `site-packages` Directory

    - PEP 371: The `multiprocessing` Package
    - PEP 3101: Advanced String Formatting
    - PEP 3105: `print` As a Function
    - PEP 3110: Exception-Handling Changes
    - PEP 3112: Byte Literals
    - PEP 3116: New I/O Library
    - PEP 3118: Revised Buffer Protocol
    - PEP 3119: Abstract Base Classes
    - PEP 3127: Integer Literal Support and Syntax
    - PEP 3129: Class Decorators
    - PEP 3141: A Type Hierarchy for Numbers
    - Outras mudanças na linguagem
    - Módulos novos ou aprimorados
    - Deprecations and Removals
    - Mudanças a construções e API C
    - Porting to Python 2.6
    - Reconhecimentos
- What's New in Python 2.5
    - PEP 308: Conditional Expressions
    - PEP 309: Partial Function Application
    - PEP 314: Metadata for Python Software Packages v1.1
    - PEP 328: Absolute and Relative Imports
    - PEP 338: Executing Modules as Scripts
    - PEP 341: Unified try/except/finally
    - PEP 342: New Generator Features
    - PEP 343: The 'with' statement
    - PEP 352: Exceptions as New-Style Classes
    - PEP 353: Using ssize_t as the index type
    - PEP 357: The '__index__' method
    - Outras mudanças na linguagem
    - New, Improved, and Removed Modules
    - Mudanças a construções e API C
    - Porting to Python 2.5
    - Reconhecimentos
- What's New in Python 2.4
    - PEP 218: Built-In Set Objects
    - PEP 237: Unifying Long Integers and Integers

    - PEP 289: Generator Expressions
    - PEP 292: Simpler String Substitutions
    - PEP 318: Decorators for Functions and Methods
    - PEP 322: Reverse Iteration
    - PEP 324: New subprocess Module
    - PEP 327: Decimal Data Type
    - PEP 328: Multi-line Imports
    - PEP 331: Locale-Independent Float/String Conversions
    - Outras mudanças na linguagem
    - New, Improved, and Deprecated Modules
    - Mudanças a construções e API C
    - Porting to Python 2.4
    - Reconhecimentos
- What's New in Python 2.3
    - PEP 218: A Standard Set Datatype
    - PEP 255: Simple Generators
    - PEP 263: Source Code Encodings
    - PEP 273: Importing Modules from ZIP Archives
    - PEP 277: Unicode file name support for Windows NT
    - PEP 278: Universal Newline Support
    - PEP 279: enumerate()
    - PEP 282: The logging Package
    - PEP 285: A Boolean Type
    - PEP 293: Codec Error Handling Callbacks
    - PEP 301: Package Index and Metadata for Distutils
    - PEP 302: New Import Hooks
    - PEP 305: Comma-separated Files
    - PEP 307: Pickle Enhancements
    - Extended Slices
    - Outras mudanças na linguagem
    - New, Improved, and Deprecated Modules
    - Pymalloc: A Specialized Object Allocator
    - Mudanças a construções e API C
    - Outras alterações e correções
    - Porting to Python 2.3
    - Reconhecimentos
- What's New in Python 2.2

    - Introdução
    - PEPs 252 and 253: Type and Class Changes
    - PEP 234: Iterators
    - PEP 255: Simple Generators
    - PEP 237: Unifying Long Integers and Integers
    - PEP 238: Changing the Division Operator
    - Unicode Changes
    - PEP 227: Nested Scopes
    - Módulos novos ou aprimorados
    - Interpreter Changes and Fixes
    - Outras alterações e correções
    - Reconhecimentos
- What's New in Python 2.1
    - Introdução
    - PEP 227: Nested Scopes
    - PEP 236: __future__ Directives
    - PEP 207: Rich Comparisons
    - PEP 230: Warning Framework
    - PEP 229: New Build System
    - PEP 205: Weak References
    - PEP 232: Function Attributes
    - PEP 235: Importing Modules on Case-Insensitive Platforms
    - PEP 217: Interactive Display Hook
    - PEP 208: New Coercion Model
    - PEP 241: Metadata in Python Packages
    - Módulos novos ou aprimorados
    - Outras alterações e correções
    - Reconhecimentos
- What's New in Python 2.0
    - Introdução
    - What About Python 1.6?
    - New Development Process
    - Unicode
    - Compreensões de lista
    - Augmented Assignment
    - String Methods
    - Garbage Collection of Cycles

- Other Core Changes
- Porting to 2.0
- Extending/Embedding Changes
- Distutils: Making Modules Easy to Install
- XML Modules
- Module changes
- Novos módulos
- IDLE Improvements
- Deleted and Deprecated Modules
- Reconhecimentos

O “Changelog” é uma versão HTML do arquivo construído [https://pypi.org/project/blurb/] do conteúdo da árvore de diretórios Misc/NEWS.d [https://github.com/python/cpython/tree/3.15/Misc/NEWS.d], que contém *todas* as mudanças não triviais do Python para a versão atual.

- Changelog
  - Python seguinte
  - Python 3.15.0 final
  - Python 3.15.0 candidato a lançamento 3
  - Python 3.15.0 candidato a lançamento 2
  - Python 3.15.0 candidato a lançamento 1
  - Python 3.15.0 beta 4
  - Python 3.15.0 beta 3
  - Python 3.15.0 beta 2
  - Python 3.15.0 beta 1
  - Python 3.15.0 alpha 8
  - Python 3.15.0 alpha 7
  - Python 3.15.0 alpha 6
  - Python 3.15.0 alpha 5
  - Python 3.15.0 alpha 4
  - Python 3.15.0 alpha 3
  - Python 3.15.0 alpha 2
  - Python 3.15.0 alpha 1
  - Python 3.14.0 beta 1
  - Python 3.14.0 alpha 7
  - Python 3.14.0 alpha 6

- Python 3.14.0 alpha 5
- Python 3.14.0 alpha 4
- Python 3.14.0 alpha 3
- Python 3.14.0 alpha 2
- Python 3.14.0 alpha 1
- Python 3.13.0 beta 1
- Python 3.13.0 alpha 6
- Python 3.13.0 alpha 5
- Python 3.13.0 alpha 4
- Python 3.13.0 alpha 3
- Python 3.13.0 alpha 2
- Python 3.13.0 alpha 1
- Python 3.12.0 beta 1
- Python 3.12.0 alpha 7
- Python 3.12.0 alpha 6
- Python 3.12.0 alpha 5
- Python 3.12.0 alpha 4
- Python 3.12.0 alpha 3
- Python 3.12.0 alpha 2
- Python 3.12.0 alpha 1
- Python 3.11.0 beta 1
- Python 3.11.0 alpha 7
- Python 3.11.0 alpha 6
- Python 3.11.0 alpha 5
- Python 3.11.0 alpha 4
- Python 3.11.0 alpha 3
- Python 3.11.0 alpha 2
- Python 3.11.0 alpha 1
- Python 3.10.0 beta 1
- Python 3.10.0 alpha 7
- Python 3.10.0 alpha 6
- Python 3.10.0 alpha 5
- Python 3.10.0 alpha 4
- Python 3.10.0 alpha 3
- Python 3.10.0 alpha 2
- Python 3.10.0 alpha 1
- Python 3.9.0 beta 1

- Python 3.9.0 alpha 6
- Python 3.9.0 alpha 5
- Python 3.9.0 alpha 4
- Python 3.9.0 alpha 3
- Python 3.9.0 alpha 2
- Python 3.9.0 alpha 1
- Python 3.8.0 beta 1
- Python 3.8.0 alpha 4
- Python 3.8.0 alpha 3
- Python 3.8.0 alpha 2
- Python 3.8.0 alpha 1
- Python 3.7.0 final
- Python 3.7.0 release candidate 1
- Python 3.7.0 beta 5
- Python 3.7.0 beta 4
- Python 3.7.0 beta 3
- Python 3.7.0 beta 2
- Python 3.7.0 beta 1
- Python 3.7.0 alpha 4
- Python 3.7.0 alpha 3
- Python 3.7.0 alpha 2
- Python 3.7.0 alpha 1
- Python 3.6.6 final
- Python 3.6.6 release candidate 1
- Python 3.6.5 final
- Python 3.6.5 release candidate 1
- Python 3.6.4 final
- Python 3.6.4 release candidate 1
- Python 3.6.3 final
- Python 3.6.3 release candidate 1
- Python 3.6.2 final
- Python 3.6.2 release candidate 2
- Python 3.6.2 release candidate 1
- Python 3.6.1 final
- Python 3.6.1 release candidate 1
- Python 3.6.0 final
- Python 3.6.0 release candidate 2

- Python 3.6.0 release candidate 1
- Python 3.6.0 beta 4
- Python 3.6.0 beta 3
- Python 3.6.0 beta 2
- Python 3.6.0 beta 1
- Python 3.6.0 alpha 4
- Python 3.6.0 alpha 3
- Python 3.6.0 alpha 2
- Python 3.6.0 alpha 1
- Python 3.5.5 final
- Python 3.5.5 release candidate 1
- Python 3.5.4 final
- Python 3.5.4 release candidate 1
- Python 3.5.3 final
- Python 3.5.3 release candidate 1
- Python 3.5.2 final
- Python 3.5.2 release candidate 1
- Python 3.5.1 final
- Python 3.5.1 release candidate 1
- Python 3.5.0 final
- Python 3.5.0 release candidate 4
- Python 3.5.0 release candidate 3
- Python 3.5.0 release candidate 2
- Python 3.5.0 release candidate 1
- Python 3.5.0 beta 4
- Python 3.5.0 beta 3
- Python 3.5.0 beta 2
- Python 3.5.0 beta 1
- Python 3.5.0 alpha 4
- Python 3.5.0 alpha 3
- Python 3.5.0 alpha 2
- Python 3.5.0 alpha 1

O que há de novo no Python 3.15

Editor:: Hugo van Kemenade

This article explains the new features in Python 3.15, compared to 3.14. Python 3.15 was released on 9 October 2026. For full details, see the changelog.

Ver também

PEP 790 [https://peps.python.org/pep-0790/] – Python 3.15 release schedule

Resumo – Destaques da versão

Python 3.15 is the latest stable release of the Python programming language, with a mix of changes to the language, the implementation, and the standard library. The biggest changes include lazy imports, frozendict and sentinel builtins, UTF-8 as the default encoding, unpacking in comprehensions, and a stable ABI for free-threaded builds.

As alterações na biblioteca incluem um novo pacote de perfilamento com o Tachyon — uma ferramenta de perfilamento baseada em amostragem estatística de alta frequência —, mais cores na saída da linha de comando, bem como as habituais descontinuações e remoções, além de melhorias na usabilidade e na correção.

Este artigo não tenta fornecer uma especificação completa de todos os novos recursos, mas fornece uma visão geral conveniente. Para detalhes completos, consulte a documentação, como Referência da Biblioteca e Referência da Linguagem. Para entender a implementação completa e a justificativa do design para uma mudança, consulte a PEP para um novo recurso específico; mas observe que as PEPs geralmente não são mantidas atualizadas depois que um recurso é totalmente implementado.

Consulte Portando para o Python 3.15 para obter orientações sobre a atualização a partir de versões anteriores do Python.

Melhorias no interpretador:

- **PEP 661** [https://peps.python.org/pep-0661/]: Adiciona o tipo embutido sentinela
- **PEP 686** [https://peps.python.org/pep-0686/]: Python agora usa UTF-8 como codificação padrão
- **PEP 798** [https://peps.python.org/pep-0798/]: Desempacotamento em compreensões
- **PEP 810** [https://peps.python.org/pep-0810/]: Importações preguiçosas explícitas para tempos de inicialização mais rápidos
- **PEP 814** [https://peps.python.org/pep-0814/]: Adiciona o tipo embutido frozendict
- **PEP 829** [https://peps.python.org/pep-0829/]: Arquivos de configuração de inicialização de pacote
- O compilador JIT experimental foi melhorado de forma significativa

- Mensagens de erro melhoradas

Melhorias significativas na biblioteca padrão:

- **PEP 799** [https://peps.python.org/pep-0799/]: Um pacote de perfilamento dedicado para organizar as ferramentas de perfilamento do Python
- **PEP 799** [https://peps.python.org/pep-0799/]: Tachyon: Perfilador de amostragem estatística de alta frequência
- Mais cores

Novos recursos de tipagem:

- **PEP 728** [https://peps.python.org/pep-0728/]: TypedDict com itens extras tipados
- **PEP 747** [https://peps.python.org/pep-0747/]: Anotando formulários de tipo com TypeForm
- **PEP 800** [https://peps.python.org/pep-0800/]: Bases disjuntas no sistema de tipos

Melhorias na API C:

- **PEP 782** [https://peps.python.org/pep-0782/]: Uma nova API C PyBytesWriter para criar um objeto bytes Python
- **PEP 788** [https://peps.python.org/pep-0788/]: Proteção contra finalização na API C
- **PEP 803** [https://peps.python.org/pep-0803/], **820** [https://peps.python.org/pep-0820/], **793** [https://peps.python.org/pep-0793/]: ABI estável para construções com threads livres e APIs C relacionadas

Alterações na construção:

- **PEP 831** [https://peps.python.org/pep-0831/]: Ponteiros de quadro estão habilitados por padrão para melhorar a observabilidade em nível de sistema

Alterações na versão:

- Os binários oficiais do Windows 64 bit agora usam um interpretador de chamada de cauda
- The official macOS binaries now install free-threading support by default

Novas funcionalidades

PEP 810 [https://peps.python.org/pep-0810/]: Importações preguiçosas explícitas

Aplicações Python de grande porte frequentemente sofrem com tempos de inicialização lentos. Um fator significativo que contribui para esse problema é o sistema de importação: quando um módulo é importado, o Python precisa localizar o arquivo, lê-lo do disco, compilá-lo para bytecode e executar todo o código de nível superior. Para aplicações com árvores de dependências complexas, esse processo pode levar segundos, mesmo quando a maior parte do código importado nunca é efetivamente utilizada durante uma execução específica.

Os desenvolvedores contornaram esse problema movendo as importações para dentro de funções, usando `importlib` para carregar módulos sob demanda ou reestruturando o código para evitar

dependências desnecessárias. Essas abordagens funcionam, mas tornam o código mais difícil de ler e manter, espalham as instruções de importação por toda a base de código e exigem disciplina para serem aplicadas de forma consistente.

O Python agora oferece uma solução mais limpa por meio de importações **`lazy`** explícitas, usando a nova palavra reservada contextual `lazy` ("preguiçosa"). Quando você marca uma importação como preguiçosa, o Python adia o carregamento do módulo até que o nome importado seja usado pela primeira vez. Isso oferece os benefícios de organização de declarar todas as importações no início do arquivo, pagando o custo de carregamento apenas pelos módulos que você realmente usa.

A palavra reservada `lazy` funciona tanto com instruções `import` quanto com instruções `from ... import`. Quando você escreve `lazy import módulo_pesado`, o Python não carrega o módulo imediatamente. Em vez disso, ele cria um objeto intermediário leve. O carregamento real do módulo ocorre de forma transparente quando você acessa o nome pela primeira vez:

```
lazy import json
lazy from pathlib import Path

print("Inicializando...")  # json e pathlib não totalmente importados ainda

dados = json.loads('{"chave": "valor"}')  # json é totalmente importado aqui
p = Path(".")  # pathlib é totalmente importado aqui
```

Esse mecanismo é particularmente útil para aplicações que importam muitos módulos no nível superior, mas que podem usar apenas um subconjunto deles em uma determinada execução. O carregamento diferido reduz a latência de inicialização sem exigir reestruturação do código ou importações condicionais espalhadas por toda a base de código.

Caso o carregamento de um módulo importado "preguiçosamente" falhe (por exemplo, se o módulo não existir), o Python levanta a exceção no momento do primeiro uso, e não no momento da importação. O traceback associado inclui tanto o local onde o nome foi acessado quanto a instrução de importação original, facilitando o diagnóstico e a depuração da falha.

Para casos em que você deseja habilitar o carregamento preguiçoso globalmente sem modificar o código-fonte, o Python fornece a opção de linha de comando **`-X lazy_imports`** e a variável de ambiente **`PYTHON_LAZY_IMPORTS`**. Ambas aceitam dois valores: `all` torna todas as importações preguiçosas por padrão e `normal` (o padrão) respeita a palavra reservada `lazy` no código-fonte. As funções **`sys.set_lazy_imports()`** e **`sys.get_lazy_imports()`** permitem alterar e consultar esse modo em tempo de execução.

Para um controle mais seletivo, **`sys.set_lazy_imports_filter()`** aceita um chamável que determina se um módulo específico deve ser carregado preguiçosamente. O filtro recebe três argumentos: o nome do módulo a ser importado (ou `None`), o nome do módulo importado e a lista "from" (ou `None` para importações regulares). Ele deve retornar `True` para permitir que a importação seja preguiçosa ou `False` para forçar o carregamento imediato. Isso permite padrões como fazer com que apenas os módulos da sua própria aplicação sejam carregados preguiçosamente, enquanto mantém as dependências de terceiros sendo carregadas imediatamente.

```
import sys

def filtro_do_meuapp(importing, imported, fromlist):
    return imported.startswith("meuapp.")
sys.set_lazy_imports_filter(filtro_do_meuapp)
sys.set_lazy_imports("all")

import meuapp.slow_module  # preguiçoso (corresponde ao filtro)
import json               # imediato (não corresponde ao filtro)
```

O próprio tipo proxy está disponível como **`types.LazyImportType`** para código que precisa detectar importações preguiçosas programaticamente.

Existem algumas restrições quanto ao uso da palavra reservada `lazy`. Importações preguiçosas são permitidas apenas no escopo do módulo; usar `lazy` dentro de uma função, corpo da classe ou bloco `try`/`except`/`finally` levanta **`SyntaxError`**. Importações com asterisco e importações de future não pode ser preguiçosas (`lazy from module import *` e `lazy from __future__ import ...` levantam **`SyntaxError`**).

Para código que não pode usar a palavra reservada `lazy` diretamente (por exemplo, ao dar suporte a versões do Python anteriores à 3.15, mas ainda usar importações preguiçosas na versão 3.15 ou superior), um módulo pode definir **`__lazy_modules__`** como um contêiner de strings com nomes de módulos totalmente qualificados. As instruções `import` regulares para esses módulos são então tratadas como preguiçosas, com a mesma semântica da palavra reservada `lazy`.

```
__lazy_modules__ = ["json", "pathlib"]

import json     # preguiçosa
import os       # ainda imediato
```

(Contribuição de Pablo Galindo Salgado e Dino Viehland em gh-142349 [https://github.com/python/cpython/issues/142349].)

Ver também

PEP 810 [https://peps.python.org/pep-0810/] para a especificação completa e justificativa.

PEP 814 [https://peps.python.org/pep-0814/]: Adiciona o tipo embutido frozendict

Um novo tipo imutável, **`frozendict`**, foi adicionado ao módulo **`builtins`**. Ele não permite modificações após a criação. Um **`frozendict`** não é uma subclasse de `dict`; ele herda diretamente de `object`. Um **`frozendict`** é hasheável desde que todas as suas chaves e valores sejam hasheáveis. Um **`frozendict`** preserva a ordem de inserção, mas a comparação não leva a ordem em consideração.

Por exemplo:

```
>>> a = frozendict(x=1, y=2)
>>> a
```

```
frozendict({'x': 1, 'y': 2})
>>> a['z'] = 3
Traceback (most recent call last):
  File "<python-input-2>", line 1, in <module>
    a['z'] = 3
    ~^^^^^
TypeError: 'frozendict' object does not support item assignment
>>> b = frozendict(y=2, x=1)
>>> hash(a) == hash(b)
True
>>> a == b
True
```

Os seguintes módulos da biblioteca padrão foram atualizados para aceitar **`frozendict`**: **`copy`**, **`decimal`**, **`json`**, **`marshal`**, **`plistlib`** (somente para serialização), **`pickle`**, **`pprint`** e **`xml.etree.ElementTree`**.

`eval()` e **`exec()`** aceitam **`frozendict`** para *globals*, e **`type()`** e **`str.maketrans()`** aceitam **`frozendict`** para *dict*.

Verificação de código para o tipo **`dict`** usando `isinstance(arg, dict)` pode ser atualizado para `isinstance(arg, (dict, frozendict))` para aceitar também o tipo **`frozendict`**, ou para `isinstance(arg, collections.abc.Mapping)` aceitar também outros tipos de mapeamentos como **`MappingProxyType`**.

(Contribuição de Victor Stinner e Donghee Na em gh-141510 [https://github.com/python/cpython/issues/141510].)

Ver também

PEP 814 [https://peps.python.org/pep-0814/] para a especificação completa e justificativa.

PEP 661 [https://peps.python.org/pep-0661/]: Adiciona o tipo embutido sentinela

Um novo tipo **`sentinel`** foi adicionado ao módulo **`builtins`** para criar valores sentinela únicos com uma representação concisa. Objetos sentinela preservam a identidade quando copiados, suportam uso em expressões de tipo com o operador `|` e podem ser serializados com pickle quando são importáveis por módulo e nome.

(Contribuição de Jelle Zijlstra em gh-148829 [https://github.com/python/cpython/issues/148829]; PEP 661 escrita por Tal Einat.)

Ver também

PEP 661 [https://peps.python.org/pep-0661/] para mais detalhes.

PEP 799 [https://peps.python.org/pep-0799/]: Um pacote de perfilamento dedicado

Um novo módulo `profiling` foi adicionado para organizar as ferramentas embutidas do Python para perfilamento em um único espaço de nomes coerente. Este módulo contém:

- `profiling.tracing`: rastreamento determinístico de chamada de função (realocado de `cProfile`).
- `profiling.sampling`: um novo perfilador de amostragem estatística (chamado Tachyon).

O módulo `cProfile` permanece como um apelido para fins de retrocompatibilidade. O módulo `profile` está descontinuado e será removido no Python 3.17.

(Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)

Ver também

PEP 799 [https://peps.python.org/pep-0799/] para mais detalhes.

Tachyon: Perfilador de amostragem estatística de alta frequência

Um novo perfilador de amostragem estatística (Tachyon) foi adicionado como `profiling.sampling`. Este perfilador permite a análise de desempenho com baixa sobrecarga de processos Python em execução, sem exigir modificação de código ou reinicialização do processo.

Ao contrário dos perfiladores determinísticos (como o `profiling.tracing`) que instrumentam cada chamada de função, o perfilador de amostragem captura periodicamente os rastreamentos de pilha dos processos em execução. Essa abordagem oferece sobrecarga praticamente zero, ao mesmo tempo que atinge taxas de amostragem de **até 1.000.000 Hz**, tornando-o o perfilador de amostragem mais rápido disponível para Python (na época de sua contribuição) e ideal para depurar problemas de desempenho em ambientes de produção. Essa capacidade é particularmente valiosa para depurar problemas de desempenho em sistemas de produção, onde as abordagens de perfilamento tradicionais seriam muito intrusivas.

Recursos chave incluem:

- **Perfilamento de sobrecarga zero**: Conecte-se a qualquer processo Python em execução sem afetar seu desempenho. Ideal para depuração em produção, onde você não pode se dar ao luxo de reiniciar ou tornar sua aplicação mais lenta.

- **Nenhuma modificação de código necessária**: Crie perfis de aplicações existentes sem reiniciá-las. Basta apontar o perfilador para um processo em execução pelo PID e começar a coletar dados.
- **Modos de alvos flexíveis**:
  - Perfile processos em execução por PID (`attach`) - anexar a aplicações já em execução
  - Execute e analise scripts diretamente (`run`) - analise desde o início da execução.
  - Execute e perfile módulos (`run -m`) - perfile pacotes executados como `python -m module`
  - Captura um snapshot único de um processo em execução (`dump`) - imprime uma pilha no estilo traceback de cada thread (ou de todas as tarefas asyncio com `--async-aware`). Útil para investigar processos travados.
- **Múltiplos modos de perfilamento**: Escolha o que medir com base na sua investigação de desempenho:
  - **Tempo real** (`--mode wall`, padrão): Mede o tempo real decorrido, incluindo E/S, esperas de rede e operações de bloqueio. Use isso para entender onde seu programa gasta tempo, inclusive quando está aguardando recursos externos.
  - **Tempo de CPU** (`--mode cpu`): Mede apenas o tempo de execução ativo da CPU, excluindo esperas de E/S e bloqueios. Use isso para identificar gargalos que consomem muita CPU e otimizar o trabalho computacional.
  - **Tempo de retenção da GIL** (`--mode gil`): Mede o tempo gasto mantendo a trava global do interpretador (Global Interpreter Lock - GIL) do Python em uso. Use isso para identificar quais threads dominam o uso da GIL em aplicações multithread.
  - **Tempo de tratamento de exceções** (`--mode exception`): Captura amostras apenas de threads com uma exceção ativa. Use isso para analisar a sobrecarga do tratamento de exceções.
- **Perfilamento consciente de threads**: Opção para criar perfil de todas as threads (`-a`) ou apenas da thread principal, essencial para entender o comportamento de aplicações multithread.
- **Múltiplos formatos de saída**: Escolha a visualização que melhor se adapta ao seu fluxo de trabalho:
  - `--pstats`: Estatísticas tabulares detalhadas compatíveis com **`pstats`**. Mostra o tempo de execução em nível de função com amostras diretas e cumulativas. Ideal para análises detalhadas e integração com ferramentas de perfilamento Python existentes.
  - `--collapsed`: Gera stack trace recolhidos (uma linha por pilha). Este formato foi especificamente projetado para a criação de flame graphs com ferramentas externas como os scripts FlameGraph de Brendan Gregg ou o SpeedScope.
  - `--flamegraph`: Gera um flame graph HTML interativo e independente usando D3.js. Abre diretamente no seu navegador para análise visual imediata. Os flame graphs mostram a hierarquia de chamadas, onde a largura representa o tempo gasto, facilitando a identificação de gargalos rapidamente.
  - `--gecko`: Gera um formato do Gecko Profiler compatível com o Firefox Profiler [https://profiler.firefox.com]. Envie a saída para o Firefox Profiler para uma análise avançada baseada em linha do tempo com recursos como gráficos de pilha, marcadores e atividade de rede.
  - `--heatmap`: Gera uma visualização interativa de mapa de calor em HTML com contagens de amostras em nível de linha. Cria um diretório com mapas de calor por arquivo, mostrando exatamente onde o tempo é gasto no nível do código-fonte.

- **Modo interativo ao vivo**: TUI de perfilador em tempo real com uma interface semelhante ao top (`--live`). Monitore o desempenho da sua aplicação enquanto ela está em execução, com classificação e filtragem interativas.
- **Perfilamento com reconhecimento de assincronismo**: Cria perfis de código assíncrono/await com reconstrução de pilha baseada em tarefas (`--async-aware`). Veja quais corrotinas estão consumindo tempo, com opções para mostrar apenas as tarefas em execução ou todas as tarefas, incluindo as que estão aguardando.
- **Perfilamento em nível de opcode**: Coleta informações de opcode de bytecode para perfilamento em nível de instrução (`--opcodes`). Mostra quais instruções de bytecode estão sendo executadas, incluindo especializações do interpretador adaptativo.

Consulte **`profiling.sampling`** para obter a documentação completa, incluindo todos os formatos de saída disponíveis, modos de criação de perfil e opções de configuração.

(Contribuição de Pablo Galindo e László Kiss Kollár em gh-135953 [https://github.com/python/cpython/issues/135953] e gh-138122 [https://github.com/python/cpython/issues/138122].)

PEP 831 [https://peps.python.org/pep-0831/]: Ponteiros de quadros habilitados por padrão

O CPython agora é construído com ponteiros de quadro por padrão em plataformas que os suportam. Isso utiliza os sinalizadores de compilação `-fno-omit-frame-pointer` e `-mno-omit-leaf-frame-pointer`, tornando o desenrolamento da pilha nativa mais rápido e confiável para perfiladores de sistema, depuradores, ferramentas de análise de falhas e ferramentas de observabilidade baseadas em eBPF.

Os sinalizadores são expostas através do **`sysconfig`**, portanto, módulos de extensão criados por ferramentas que consomem a configuração de compilação do Python herdam ponteiros de quadro por padrão. Essa propagação é intencional: o perfilamento misto de Python/nativo requer uma cadeia ininterrupta de ponteiros de quadro através do interpretador, módulos de extensão, aplicações incorporadas e bibliotecas nativas.

Importante

Backends de construção de terceiros e sistemas de construção nativos devem preservar esses sinalizadores ao consumirem os valores de **`sysconfig`** do Python. Sistemas de construção que compilam código C, C++, Rust ou outros códigos nativos sem herdar os sinalizadores do compilador Python devem habilitar sinalizadores equivalentes de ponteiro de quadro. Um único componente nativo compilado sem ponteiros de quadro pode interromper o desenrolamento da pilha para todo o processo Python.

(Contribuição de Pablo Galindo Salgado e Savannah Ostrowski em gh-149201 [https://github.com/python/cpython/issues/149201]; PEP 831 escrita por Pablo Galindo Salgado, Ken Jin, Savannah Ostrowski e Diego Russo.)

Ver também

PEP 831 [https://peps.python.org/pep-0831/] para mais detalhes.

PEP 798 [https://peps.python.org/pep-0798/]: Desempacotamento em compreensões

Compreensões de lista, conjunto e dicionário, bem como expressões geradoras, agora oferecem suporte a desempacotamento com `*` e `**`. Isso estende a sintaxe de desempacotamento de **PEP 448** [https://peps.python.org/pep-0448/] para compreensões, fornecendo uma nova sintaxe para combinar um número arbitrário de iteráveis ou dicionários em uma única estrutura plana. Essa nova sintaxe é uma alternativa direta a compreensões aninhadas, `itertools.chain()` e `itertools.chain.from_iterable()`. Por exemplo:

```
>>> lists = [[1, 2], [3, 4], [5]]
>>> [*L for L in lists]  # equivalent to [x for L in lists for x in L]
[1, 2, 3, 4, 5]

>>> sets = [{1, 2}, {2, 3}, {3, 4}]
>>> {*s for s in sets}  # equivalent to {x for s in sets for x in s}
{1, 2, 3, 4}

>>> dicts = [{'a': 1}, {'b': 2}, {'a': 3}]
>>> {**d for d in dicts}  # equivalent to {k: v for d in dicts for k,v in d.items(
{'a': 3, 'b': 2}
```

Expressões geradoras podem usar o desempacotamento de forma semelhante para produzir valores a partir de múltiplos iteráveis.

```
>>> gen = (*L for L in lists)  # equivalent to (x for L in lists for x in L)
>>> list(gen)
[1, 2, 3, 4, 5]
```

Essa mudança também se estende às expressões geradoras assíncronas, de modo que, por exemplo, `(*a async for a in agen())` equivale `(x async for a in agen() for x in a)`.

(Contribuição de Adam Hartz em gh-143055 [https://github.com/python/cpython/issues/143055].)

Ver também

PEP 798 [https://peps.python.org/pep-0798/] para mais detalhes.

PEP 829 [https://peps.python.org/pep-0829/]: Arquivos de configuração de inicialização de pacote

Carregados pelo módulo `site` quando `-S` não é fornecido, os arquivos .pth podem conter linhas que estendem `sys.path` e executam código arbitrário quando a linha começa com `import` (seguido por um espaço ou tabulação). Esta última funcionalidade pode ser problemática, pois é difícil saber exatamente o que é executado quando o Python é iniciado.

Como um passo para melhorar a capacidade de auditar o código executável antes da inicialização, o Python 3.15 introduz os arquivos .start, que contêm especificações de ponto de entrada no formato `pkg.mod:callable`, onde `pkg.mod` é o caminho de importação para a função chamável em questão. Quando o Python é iniciado, a função chamável é localizada e chamada sem argumentos.

As linhas `import` em arquivos `.pth` foram descontinuadas silenciosamente. Quando um arquivo `.start` correspondente for encontrado, as linhas `import` em arquivos `.pth` serão ignoradas. Não haverá alterações nas linhas de extensão **`sys.path`** em arquivos `.pth`.

O módulo **`site`** também fornece a classe **`site.StartupState`** para processar em lote a inicialização de múltiplos diretórios do site, garantindo que todas as extensões de caminho estáticas sejam aplicadas antes da execução de qualquer código de inicialização. **`site.main()`** utiliza implicitamente uma instância dessa classe para processar em lote todos os arquivos de configuração de inicialização durante a inicialização normal do interpretador. Chamadores precisando do mesmo comportamento de processamento em lote podem criar uma **`StartupState`** diretamente e controlá-la com **`addsitedir()`**, **`addusersitepackages()`** e **`addsitepackages()`**,, e então chamar **`process()`** uma única vez ao final do lote.

(Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641] e gh-150228 [https://github.com/python/cpython/issues/150228].)

PEP 803 [https://peps.python.org/pep-0803/]: ABI Estável para construções com threads livres

Extensões C que têm como alvo a ABI Estável agora podem ser compiladas para a nova *ABI Estável para Construções com Threads Livres* (também conhecida como `abi3t`), o que as torna compatíveis com as construções com threads livres do CPython. Isso geralmente requer algumas alterações não triviais no código-fonte; especificamente:

- A mudança para a API introduzida em **PEP 697** [https://peps.python.org/pep-0697/] (Python 3.12), como **`basicsize`** e **`PyObject_GetTypeData()`** negativos, em vez de tornar **`PyObject`** parte da estrutura da instância; e
- Mudança de uma função `PyInit_` para um novo gancho de exportação, **`PyModExport_*`**, introduzido para este propósito em **PEP 793** [https://peps.python.org/pep-0793/], com uma nova estrutura **`PySlot`** introduzida em **PEP 820** [https://peps.python.org/pep-0820/].

Note que a ABI Estável não oferece todas as funcionalidades que o CPython oferece. Extensões que não podem migrar para `abi3t` devem continuar a ser construir separadamente para a ABI Estável existente (`abi3`) e para a ABI específica da versão para threads livres (`cp315t`).

Stable ABI for Free-Threaded Builds should typically be selected in a build tool. Tools with `abi3t` support include meson-python, scikit-build-core, and Maturin; see Setting up the build for links to their documentation on `abi3t` builds. If your build tool does not support `abi3t`, compile for `cp315t` separately. If not using a build tool – or when writing such a tool – you can select `abi3t` by setting the macro **`Py_TARGET_ABI3T`** as discussed in Compiling for Stable ABI.

A practical migration guide for switching to `abi3t` is available. The community-maintained Python Free-Threading Guide [https://py-free-threading.github.io/abi3t/] provides examples of building and testing `abi3t` wheels.

Ver também

PEP 803 [https://peps.python.org/pep-0803/] para mais detalhes.

PEP 788 [https://peps.python.org/pep-0788/]: Protegendo a API C de finalização do interpretador

Na API C, a finalização do interpretador pode ser problemática para muitas extensões, pois a anexação de um estado de thread trava permanentemente a thread, resultando em impasses (deadlocks) e outros problemas espúrios. Além disso, historicamente tem sido impossível verificar com segurança se um interpretador está ativo antes de usá-lo, o que leva a falhas quando uma thread exclui um interpretador simultaneamente enquanto outra thread está tentando se anexar a ele.

Já existem diversos novos conjuntos de APIs para contornar esses problemas:

- Guardas do interpretador, que impedem um interpretador de ser finalizado.
- Visões do interpretador, que permitem acesso seguro para thread a um interpretador que pode estar sendo finalizado ou excluído simultaneamente.
- Novas APIs para anexar e desanexar automaticamente estados de thread que vêm com proteção embutida contra finalização.

Além disso, as APIs da família `PyGILState` (principalmente **`PyGILState_Ensure()`** e **`PyGILState_Release()`**) foram suavemente descontinuados. **Não** há planos para removê-las, e o código existente continuará funcionando, mas não haverá novas APIs `PyGILState` em versões futuras do Python.

(Contribuição de Peter Bierma em gh-149101 [https://github.com/python/cpython/issues/149101].)

Ver também

PEP 788 [https://peps.python.org/pep-0788/] para mais detalhes.

Mensagens de erro melhoradas

- O interpretador agora fornece sugestões mais úteis em exceções **`AttributeError`** ao acessar um atributo em um objeto que não existe, mas um atributo semelhante está disponível por meio de um de seus membros.

  Por exemplo, se o objeto tiver um atributo que exponha o nome solicitado, a mensagem de erro sugerirá acessá-lo por meio desse atributo interno:

```
from dataclasses import dataclass
from math import pi

@dataclass
class Circle:
   radius: float

   @property
   def area(self) -> float:
      return pi * self.radius**2

class Container:
   def __init__(self, inner: Circle) -> None:
      self.inner = inner

circle = Circle(radius=4.0)
container = Container(circle)
print(container.area)
```

Executar esse código agora produz uma sugestão mais clara:

```
Traceback (most recent call last):
  File "example.py", line 18, in <module>
    print(container.area)
          ^^^^^^^^^^^^^^
AttributeError: 'Container' object has no attribute 'area'. Did you mean '.inr
```

- Quando um erro `AttributeError` em um tipo embutido não possui uma correspondência próxima por meio da distância de Levenshtein, a mensagem de erro agora verifica uma tabela estática de nomes de métodos comuns de outras linguagens (JavaScript, Java, Ruby, C#) e sugere o equivalente em Python:

```
>>> [1, 2, 3].push(4)
Traceback (most recent call last):
...
AttributeError: 'list' object has no attribute 'push'. Did you mean '.append'?

>>> 'hello'.toUpperCase()
Traceback (most recent call last):
...
AttributeError: 'str' object has no attribute 'toUpperCase'. Did you mean '.up
```

Quando o equivalente em Python for uma construção da linguagem em vez de um método, a dica descreve a construção diretamente:

```
>>> {}.put("a", 1)
Traceback (most recent call last):
...
AttributeError: 'dict' object has no attribute 'put'. Use d[k] = v.
```

Quando um método mutável é chamado em um tipo imutável, a dica sugere a contraparte mutável:

```
>>> (1, 2, 3).append(4)
Traceback (most recent call last):
...
AttributeError: 'tuple' object has no attribute 'append'. Did you mean to use
```

Essas dicas também funcionam para subclasses de tipos predefinidos.

(Contribuição de Matt Van Horn em gh-146406 [https://github.com/python/cpython/issues/146406].)

- O interpretador agora tenta fornecer uma sugestão quando `delattr()` falha devido à ausência de um atributo. Quando um nome de atributo muito semelhante a um atributo existente é usado, o interpretador sugerirá o nome correto do atributo na mensagem de erro. Por exemplo:

```
>>> class A:
...     pass
>>> a = A()
>>> a.abcde = 1
>>> del a.abcdf
Traceback (most recent call last):
...
AttributeError: 'A' object has no attribute 'abcdf'. Did you mean: 'abcde'?
```

(Contribuição de Nikita Sobolev e Pranjal Prajapati em gh-136588 [https://github.com/python/cpython/issues/136588].)

- Diversas mensagens de erro que utilizavam incorretamente o termo “argumento” foram corrigidas. (Contribuição de Stan Ulbrych em gh-133382 [https://github.com/python/cpython/issues/133382].)

Outras mudanças na linguagem

- O Python agora utiliza UTF-8 [https://en.wikipedia.org/wiki/UTF-8] como codificação padrão, independentemente do ambiente do sistema. Isso significa que operações de E/S sem uma codificação explícita — por exemplo, `open('flying-circus.txt')` — utilizarão UTF-8. O UTF-8 é uma codificação de caracteres Unicode [https://home.unicode.org/] amplamente suportada que se tornou um padrão *de facto* para a representação de texto, abrangendo quase todas as páginas da web, muitos formatos de arquivo comuns, linguagens de programação e muito mais.

  Isso se aplica apenas quando nenhum argumento `encoding` é fornecido. Para garantir a melhor compatibilidade entre versões do Python, certifique-se de sempre fornecer um argumento `encoding` explícito. O aviso de codificação opcional pode ser usado para identificar códigos que possam ser afetados por essa mudança. O argumento especial `encoding='locale'` utiliza a codificação da localidade (*locale*) atual e é suportado desde o Python 3.10.

  Para manter o comportamento anterior, o modo UTF-8 do Python pode ser desativado com a variável de ambiente `PYTHONUTF8=0` ou com a opção de linha de comando `-X utf8=0`.

Contribuição de Adam Turner em gh-133711 [https://github.com/python/cpython/issues/133711]. PEP 686 escrita por Inada Naoki.)

Ver também

PEP 686 [https://peps.python.org/pep-0686/] para mais detalhes.

- A ajuda do interpretador (como `python --help`) agora é exibida com cores. Isso pode ser controlado por variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-148766 [https://github.com/python/cpython/issues/148766].)
- Exceções não levantáveis agora são destacadas com cores por padrão. Isso pode ser controlado por variáveis de ambiente. (Contribuição de Peter Bierma em gh-134170 [https://github.com/python/cpython/issues/134170].)
- Mais cores em argparse, ast, calendar, difflib, http.server, pickletools, autocompletar por tab do PyREPL, python –help, sqlite3, timeit, tokenize, exceções não levantáveis e na ajuda na interface de linha de comando da stdlib (ast, compileall, doctest, gzip, inspect, json.tool, pdb, profiling.sampling, random, regrtest, sqlite3, timeit, tokenize, trace, unittest, uuid, zipapp, zipfile).
- O método `__repr__()` de `ImportError` e de `ModuleNotFoundError` agora mostra “name” e “path” como `name=<name>` e `path=<path>` se eles foram fornecidos como argumentos nomeados em tempo de construção. (Contribuição de Serhiy Storchaka, Oleg Iarygin e Yoav Nir em gh-74185 [https://github.com/python/cpython/issues/74185].)
- Os descritores `__dict__` e `__weakref__` agora utilizam uma única instância de descritor por interpretador, compartilhada entre todos os tipos que deles necessitam. Isso acelera a criação de classes e ajuda a evitar ciclos de referência. (Contribuição de Petr Viktorin em gh-135228 [https://github.com/python/cpython/issues/135228].)
- A opção `-W` e a variável de ambiente `PYTHONWARNINGS` agora podem especificar expressões regulares em vez de strings literais para corresponder à mensagem de aviso e ao nome do módulo, caso o campo correspondente comece e termine com uma barra (`/`). (Contribuição de Serhiy Storchaka em gh-134716 [https://github.com/python/cpython/issues/134716].)
- Funções que recebem argumentos *timestamp* ou *timeout* agora aceitam quaisquer números reais (como `Decimal` e `Fraction`), e não apenas números inteiros ou de ponto flutuante, embora isso não melhore a precisão. (Contribuição de Serhiy Storchaka em gh-67795 [https://github.com/python/cpython/issues/67795].)
- Adicionado `bytearray.take_bytes(n=None, /)` para extrair bytes de um `bytearray` sem realizar cópia. Isso permite otimizar códigos que precisam retornar `bytes` após trabalharem com um buffer mutável de bytes, como em cenários de buffering de dados, análise de protocolos de rede, codificação, decodificação e compressão. Padrões de código comuns que podem ser otimizados com `take_bytes()` estão listados abaixo.

Refatorações sugeridas para otimização

Descrição	Antigo	Novo

Retorna `bytes` após trabalhar com `bytearray`	`def read() -> bytes:`
	`  buffer = bytearray(1024)`
	`  ...`
	`  return bytes(buffer)`
		`def read() -> bytes:`
		`  buffer = bytearray(1024)`
		`  ...`
		`  return buffer.take_bytes()`

Esvazia um buffer obtendo os bytes	`buffer = bytearray(1024)`
	`...`
	`data = bytes(buffer)`
	`buffer.clear()`
		`buffer = bytearray(1024)`
		`...`
		`data = buffer.take_bytes()`

Divide um buffer em um separador específico	`buffer = bytearray(b'abc\ndef')`
	`n = buffer.find(b'\n')`
	`data = bytes(buffer[:n + 1])`
	`del buffer[:n + 1]`
	`assert data == b'abc\n'`
	`assert buffer == bytearray(b'def')`
		`buffer = bytearray(b'abc\ndef')`
		`n = buffer.find(b'\n')`
		`data = buffer.take_bytes(n + 1)`

Divide um buffer em um separador específico; descarta após o separador	`buffer = bytearray(b'abc\ndef')`
	`n = buffer.find(b'\n')`
	`data = bytes(buffer[:n])`
	`buffer.clear()`
	`assert data == b'abc'`
	`assert len(buffer) == 0`
		`buffer = bytearray(b'abc\ndef')`
		`n = buffer.find(b'\n')`
		`buffer.resize(n)`
		`data = buffer.take_bytes()`

(Contribuição de Cody Maloney em gh-139871 [https://github.com/python/cpython/issues/139871].;)

- Muitas funções relacionadas à compilação ou análise sintática (*parsing*) de código Python, como `compile()`, `ast.parse()`, `symtable.symtable()` e `importlib.abc.InspectLoader.source_to_code()`, agora permitem que o nome do módulo seja passado. Isso é necessário para filtrar avisos de sintaxe pelo nome do módulo de forma inequívoca. (Contribuição de Serhiy Storchaka em gh-135801 [https://github.com/python/cpython/issues/135801].)

- Permitida a deinição de *__dict__* e *__weakref__* em `__slots__` para qualquer classe. (Contribuição de Serhiy Storchaka em gh-41779 [https://github.com/python/cpython/issues/41779].)

- Permitida a definição de `__slots__` para uma classe derivada de `tuple` (incluindo classes criadas por `collections.namedtuple()`). (Contribuição de Serhiy Storchaka em gh-41779 [https://github.com/python/cpython/issues/41779].)

- O tipo `slice` agora oferece suporte a subscrição, tornando-se um tipo genérico. (Contribuição de James Hilton-Balfe em gh-128335 [https://github.com/python/cpython/issues/128335].)

- A classe `memoryview` agora oferece suporte aos tipos C float complex e double complex: caracteres de formatação `'Zf'` e `'Zd'`, respectivamente. (Contribuição de Victor Stinner em gh-146151 [https://github.com/python/cpython/issues/146151] e gh-148675 [https://github.com/python/cpython/issues/148675].)

- Permite que o argumento *count* de `bytes.replace()` seja um argumento nomeado. (Contribuição de Stan Ulbrych em gh-147856 [https://github.com/python/cpython/issues/147856].)

- O sinal de mais unário agora é aceito em padrões literais de `match`, espelhando o suporte existente para o sinal de menos unário. (Contribuição de Bartosz Sławecki em gh-145239 [https://github.com/python/cpython/issues/145239].)

- O sistema de importação agora adquire travas por módulo em ordem hierárquica (pacotes pai antes de seus submódulos). Isso corrige um impasse (*deadlock*) de longa data no qual uma *thread* importando `pkg.sub` e outra importando `pkg.sub.mod` podiam bloquear uma à outra quando `pkg/sub/__init__.py` importava `pkg.sub.mod`. (Contribuição de Gregory P. Smith em gh-83065 [https://github.com/python/cpython/issues/83065].)

- Nomes de arquivo de extensões da ABI Estável que utilizam o sufixo `.so` agora podem incluir uma tupla *multiarch*; por exemplo, `foo.abi3-x86-64-linux-gnu.so`. Isso permite que extensões da ABI estável para múltiplas arquiteturas sejam instaladas simultaneamente no mesmo diretório sem entrarem em conflito entre si, ao contrário do que ocorre com extensões dinâmicas comuns. (Contribuição de Stefano Rivera em gh-122931 [https://github.com/python/cpython/issues/122931].)

- O atributo `__cached__` em módulos, que estava descontinuado desde a versão 3.13, não é mais definido nem levado em consideração pelo sistema de importação ou pela biblioteca padrão. Em vez disso, utilize `__spec__.cached`. (Contribuição de Brett Cannon em gh-97879 [https://github.com/python/cpython/issues/97879].)

  Observe que os atributos `__loader__` e `__package__` também estão descontinuado e com remoção programada.

Console interativo padrão

- O autocompletar via tecla Tab agora é colorido de acordo com o tipo de objeto, com base no fancycompleter [https://pypi.org/project/fancycompleter/]. Defina `PYTHON_BASIC_COMPLETER` para utilizar o `rlcompleter` como alternativa. As cores também podem ser controladas por meio de variáveis de ambiente. (Contribuição de Antonio Cuni e Pablo Galindo em gh-130472 [https://github.com/python/cpython/issues/130472].)

- O preenchimento via tecla Tab agora sugere atributos de módulos em instruções `from ... import`. Os atributos só podem ser sugeridos após a importação do módulo; por isso, módulos da stdlib são importados automaticamente, enquanto, para outros módulos, o recurso de preenchimento oferece a importação quando a tecla `Tab` é pressionada uma segunda vez. (Contribuição de Loïc Simon e Pablo Galindo em gh-140870 [https://github.com/python/cpython/issues/140870].)

Novos módulos

math.integer

Este módulo fornece acesso às funções matemáticas para argumentos inteiros (**PEP 791** [https://peps.python.org/pep-0791/]). (Contribuição de Serhiy Storchaka em gh-81313 [https://github.com/python/cpython/issues/81313].)

Módulos melhorados

argparse

- A ação `BooleanOptionalAction` agora oferece suporte a opções longas com um único traço e a caracteres de prefixo alternativos. (Contribuição de Serhiy Storchaka em gh-138525 [https://github.com/python/cpython/issues/138525].)
- O parâmetro *suggest_on_error* de `argparse.ArgumentParser` foi alterado para ter `True` como valor padrão. Isso habilita, por padrão, sugestões para argumentos digitados incorretamente. (Contribuição de Jakob Schluse em gh-140450 [https://github.com/python/cpython/issues/140450].)
- Adicionado suporte à marcação com crases (*backticks*) na descrição e no texto de *epilog* da classe `ArgumentParser` para destacar código em linha quando a saída colorida estiver habilitada. (Contribuição de Savannah Ostrowski em gh-142390 [https://github.com/python/cpython/issues/142390].)
- Foi estendida a marcação com crases simples para o texto do argumento `help` e adicionou-se suporte para crases duplas (estilo literal inline do RST). (Contribuição de Hugo van Kemenade em gh-149375 [https://github.com/python/cpython/issues/149375].)

array

- Suporte aos tipos C float complex e double complex: caracteres de formatação `'Zf'` e `'Zd'`, respectivamente. (Contribuição de Victor Stinner em gh-146151 [https://github.com/python/cpython/issues/146151] e gh-148675 [https://github.com/python/cpython/issues/148675].)
- Suporte a half-floats (formato de intercâmbio binário IEEE 754 de 16 bits): caractere de formatação `'e'`. (Contribuição de Sergey B Kirpichev em gh-146238 [https://github.com/python/cpython/issues/146238].)
- O tipo de `array.typecodes` mudou de `str` para `tuple` para oferecer suporte a códigos de tipo com mais de 1 caractere (`Zf` e `Zd`). (Contribuição de Victor Stinner em gh-148675 [https://github.com/python/cpython/issues/148675].)

ast

- Adiciona o parâmetro *color* a `dump()`. Se for `True`, a string retornada terá destaque de sintaxe usando sequências de escape ANSI. Se for `False` (o padrão), a saída colorida estará sempre desativada. (Contribuição de Stan Ulbrych em gh-148981 [https://github.com/python/cpython/issues/148981].)
- A saída da linha de comando agora conta com destaque de sintaxe por padrão. Isso pode ser controlado usando variáveis de ambiente. (Contribuição de Stan Ulbrych em gh-148981 [https://github.com/python/cpython/issues/148981].)

asyncio

- Adicionado `TaskGroup.cancel` para permitir o encerramento antecipado de um grupo de tarefas — por exemplo, quando o objetivo das tarefas tiver sido alcançado ou seus serviços não forem mais necessários. Anteriormente, isso envolvia código repetitivo (*boilerplate*) pouco intuitivo, como uma tarefa adicional que levantava uma exceção personalizada, a qual era então suprimida ao sair do grupo de tarefas. (Contribuição de John Belmonte em gh-127214 [https://github.com/python/cpython/issues/127214].)

base64

- Adiciona o parâmetro *pad* a `z85encode()`. (Contribuição de Hauke Dämpfling em gh-143103 [https://github.com/python/cpython/issues/143103].)
- Adicionado o parâmetro *padded* a `b32encode()`, `b32decode()`, `b32hexencode()`, `b32hexdecode()`, `b64encode()`, `b64decode()`, `urlsafe_b64encode()` e `urlsafe_b64decode()`. (Contribuição de Serhiy Storchaka em gh-73613 [https://github.com/python/cpython/issues/73613].)
- Adicionado o parâmetro *wrapcol* a `b16encode()`, `b32encode()`, `b32hexencode()`, `b64encode()`, `b85encode()` e `z85encode()`. (Contribuição de Serhiy Storchaka em gh-143214 [https://github.com/python/cpython/issues/143214] e gh-146431 [https://github.com/python/cpython/issues/146431].)
- Adicionado o parâmetro *ignorechars* em `b16decode()`, `b32decode()`, `b32hexdecode()`, `b64decode()`, `b85decode()` e `z85decode()`. (Contribuição de Serhiy Storchaka em gh-144001 [https://github.com/python/cpython/issues/144001] e gh-146431 [https://github.com/python/cpython/issues/146431].)
- Adicionado o parâmetro *canonical* a `b32decode()`, `b32hexdecode()`, `b64decode()`, `a85decode()`, `b85decode()` e `z85decode()` para rejeitar codificações o bits de preenchimento diferente de zero ou outras formas não canônicas. (Contribuição de Gregory P. Smith em gh-146311 [https://github.com/python/cpython/issues/146311].)

binascii

- Adicionadas as funções para codificação Base32:

  - `b2a_base32()` e `a2b_base32()`

  (Contribuição de James Seo em gh-146192 [https://github.com/python/cpython/issues/146192].)

- Adicionadas funções para codificação Ascii85, Base85 e Z85:
  - `b2a_ascii85()` e `a2b_ascii85()`
  - `b2a_base85()` e `a2b_base85()`

  (Contribuição de James Seo e Serhiy Storchaka em gh-101178 [https://github.com/python/cpython/issues/101178].)

- Adicionado o parâmetro *padded* em `b2a_base32()`, `a2b_base32()`, `b2a_base64()` e `a2b_base64()`. (Contribuição de Serhiy Storchaka em gh-73613 [https://github.com/python/cpython/issues/73613].)

- Adicionado o parâmetro *wrapcol* a `b2a_base64()`. (Contribuição de Serhiy Storchaka em gh-143214 [https://github.com/python/cpython/issues/143214].)

- Adicionado o parâmetro *alphabet* em `b2a_base64()` e `a2b_base64()`. (Contribuição de Serhiy Storchaka em gh-145980 [https://github.com/python/cpython/issues/145980].)

- Adicionado o parâmetro *ignorechars* em `a2b_hex()`, `unhexlify()` e `a2b_base64()`. (Contribuição de Serhiy Storchaka em gh-144001 [https://github.com/python/cpython/issues/144001] e gh-146431 [https://github.com/python/cpython/issues/146431].)

- Adicionado o parâmetro *canonical* em `a2b_base64()`, para rejeitar codificações com bits de preenchimento diferente de zero. (Contribuição de Gregory P. Smith em gh-146311 [https://github.com/python/cpython/issues/146311].)

calendar

- A saída de texto da linha de comando do módulo `calendar` tem mais cores. Isso pode ser controlado com as variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-148352 [https://github.com/python/cpython/issues/148352].)
- A saída HTML da interface de linha de comando do módulo `calendar` agora aceita a opção ano-mês: `python -m calendar -t html 2009 06`. (Contribuição de Pål Grønås Drange em gh-140212 [https://github.com/python/cpython/issues/140212].)
- As páginas de calendário geradas pela classe `calendar.HTMLCalendar` agora oferecem suporte ao modo escuro e foram migradas para o padrão HTML5 para melhorar a acessibilidade. (Contribuição de Jiahao Li e Hugo van Kemenade em gh-137634 [https://github.com/python/cpython/issues/137634].)

collections

- Adicionados `collections.Counter.__xor__()` e `collections.Counter.__ixor__()` para calcular a diferença simétrica entre objetos `Counter`. (Contribuição de Raymond Hettinger em gh-138682 [https://github.com/python/cpython/issues/138682].)

concurrent.futures

- Melhoria no relatório de erros quando um processo filho em um `concurrent.futures.ProcessPoolExecutor` é encerrado abruptamente. O traceback resultante agora informará o PID e o código de saída do processo encerrado. (Contribuição de Jonathan Berg em gh-139486 [https://github.com/python/cpython/issues/139486].)

contextlib

- Adicionado suporte para descritores arbitrários **`__enter__()`**, **`__exit__()`**, **`__aenter__()`** e **`__aexit__()`** em `ExitStack` e `contextlib.AsyncExitStack`, para consistência com as instruções `with` e `async with`. (Contribuição de Serhiy Storchaka em gh-144386 [https://github.com/python/cpython/issues/144386].)
- `ContextDecorator` e `AsyncContextDecorator` (e, consequentemente, `contextmanager()` e `asynccontextmanager()` quando usados como decoradores) agora detectam funções geradoras, funções de corrotina e funções geradoras assíncronas, mantendo o gerenciador de contexto aberto durante a iteração ou a operação de *await*. Anteriormente, o gerenciador de contexto era encerrado assim que o objeto gerador ou de corrotina era criado. (Contribuição de Alex Grönholm e Gregory P. Smith em gh-125862 [https://github.com/python/cpython/issues/125862].)

ctypes

- Altera o atributo `_type_` de `c_float_complex`, `c_double_complex` e `c_longdouble_complex` de `F`, `D` e `G` para `Zf`, `Zd` e `Zg` para compatibilidade com NumPy. (Contribuição de Victor Stinner em gh-148675 [https://github.com/python/cpython/issues/148675].)

dataclasses

- Anotações para métodos `__init__` gerados não mais incluem nomes de tipos internos. (Contribuição de David Ellis em gh-137530 [https://github.com/python/cpython/issues/137530].)

dbm

- Foram adicionados novos métodos **`reorganize()`** a `dbm.dumb` e `dbm.sqlite3` para recuperar espaço livre não utilizado, anteriormente ocupado por entradas excluídas. (Contribuição de Andrea Oliveri em gh-134004 [https://github.com/python/cpython/issues/134004].)

difflib

- Foi introduzido o parâmetro opcional *color* em `difflib.unified_diff()`, permitindo uma saída colorida semelhante à do **git diff**. Isso pode ser controlado por variáveis de ambiente. (Contribuição de Douglas Thor em gh-133725 [https://github.com/python/cpython/issues/133725].)
- Melhoria no estilo das páginas de diff em HTML geradas pela classe `difflib.HtmlDiff` e migração da saída para o padrão HTML5. (Contribuição de Jiahao Li em gh-134580 [https://github.com/python/cpython/issues/134580].)

email

- Geradores de e-mail agora levantam um erro quando um `EmailMessage` não pode ser achatado com precisão devido a um endereço de e-mail não ASCII (caixa de correio) em um cabeçalho de endereço. Opções para suporte à Internacionalização de Endereço de E-mail (EAI) são discutidas em `EmailPolicy.utf8`. (Contribuição de R David Murray e Mike Edmunds em gh-122540 [https://github.com/python/cpython/issues/122540].)

faulthandler

- Adicionado o parâmetro *max_threads* em `faulthandler.enable()`, `faulthandler.dump_traceback()`, `faulthandler.dump_traceback_later()` e `faulthandler.register()`. (Contribuição de Eric Froemling em gh-149085 [https://github.com/python/cpython/issues/149085].)

functools

- `singledispatchmethod()` agora oferece suporte a chamáveis que não são descritores. (Contribuição de Serhiy Storchaka em gh-140873 [https://github.com/python/cpython/issues/140873].)
- `singledispatchmethod()` agora despacha pelo segundo argumento se ele envolver um método regular e for chamado como um atributo de classe. (Contribuição de Bartosz Sławecki em gh-143535 [https://github.com/python/cpython/issues/143535].)

gc

- As versões 3.14.0 a 3.14.4 do Python incluíam um novo coletor de lixo incremental. No entanto, devido a diversos relatos [https://github.com/python/cpython/issues/142516] de pressão significativa sobre a memória em ambientes de produção, houve um retorno ao GC geracional da versão 3.13. Esse é o GC utilizado atualmente no Python 3.14.5 e versões posteriores, inclusive Python 3.15. (Contribuição de Sergey Miryanov em gh-148726 [https://github.com/python/cpython/issues/148726].)

hashlib

- Garante que funções de hash garantidas para estarem sempre *disponíveis* existam como atributos do `hashlib` mesmo que não funcionem em tempo de execução devido à ausência de implementações de backend. Por exemplo, `hashlib.md5` não levantará mais `AttributeError` se o OpenSSL não estiver disponível e o Python tiver sido construído sem suporte a MD5. (Contribuição de Bénédikt Tran em gh-136929 [https://github.com/python/cpython/issues/136929].)

http.client

- Um novo parâmetro somente-nomeado *max_response_headers* foi adicionado aos construtores do `HTTPConnection` e do `HTTPSConnection`. Este parâmetro substitui o número máximo padrão de cabeçalhos de resposta permitidos. (Contribuição de Alexander Enrique Urieles Nieto em gh-131724 [https://github.com/python/cpython/issues/131724].)

http.server

- O registro de logs de `BaseHTTPRequestHandler`, como usado pela interface de linha de comando, agora é colorido por padrão. Isso pode ser controlado com variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-146292 [https://github.com/python/cpython/issues/146292].)
- Adicionado `default_content_type` e a opção de linha de comando `--content-type` para permitir a personalização do cabeçalho `Content-Type` padrão para arquivos com extensões desconhecidas. (Contribuição de John Comeau e Hugo van Kemenade em gh-113471 [https://github.com/python/cpython/issues/113471].)
- Adiciona um novo argumento nomeado `extra_response_headers` ao `SimpleHTTPRequestHandler` para dar suporte a cabeçalhos personalizados em respostas HTTP. (Contribuição de Anton I. Sipos em gh-135057 [https://github.com/python/cpython/issues/135057].)
- Adiciona uma opção `-H/--header` à interface de linha de comando **python -m http.server** para dar suporte a cabeçalhos personalizados em respostas HTTP. (Contribuição de Anton I. Sipos em gh-135057 [https://github.com/python/cpython/issues/135057].)

importlib.metadata

- Anteriormente, ao acessar um diretório de metadados de distribuição que não contivesse um arquivo de metadados, `metadata()` e `Distribution.metadata` retornavam um objeto `PackageMetadata` vazio, como se o arquivo estivesse presente, mas vazio. Agora, uma exceção `MetadataNotFound` é levantada. Veja importlib_metadata#493 [https://github.com/python/importlib_metadata/issues/493] para contexto e justificativa e gh-143387 [https://github.com/python/cpython/issues/143387] para a justificativa quanto às preocupações de compatibilidade. (Contribuição de Jason R. Coombs.)

inspect

- Adiciona os parâmetros *inherit_class_doc* e *fallback_to_class_doc* para `getdoc()`. (Contribuição de Serhiy Storchaka em gh-132686 [https://github.com/python/cpython/issues/132686].)

json

- Adiciona o parâmetro *array_hook* às funções `load()` e `loads()`: permite uma função de retorno para tipos de array literal em JSON para personalizar listas Python no objeto decodificado resultante. Passar um `frozendict` combinado para o parâmetro *object_pairs_hook* e uma `tuple` para *array_hook* vai produzir uma estrutura Python imutável profundamente aninhada representando os dados JSON. (Contribuição de Joao S. O. Bueno em gh-146440 [https://github.com/python/cpython/issues/146440].)

locale

- `setlocale()` agora oferece suporte a códigos de idioma com modificadores `@`. Modificadores `@` não são mais removidos silenciosamente em `getlocale()`, mas sim incluídos no código do idioma. (Contribuição de Serhiy Storchaka em gh-137729 [https://github.com/python/cpython/issues/137729].)

- Remove a descontinuação da função `locale.getdefaultlocale()`. (Contribuição de Victor Stinner em gh-130796 [https://github.com/python/cpython/issues/130796].)

math

- Adiciona as funções `math.isnormal()` e `math.issubnormal()`. (Contribuição de Sergey B Kirpichev em gh-132908 [https://github.com/python/cpython/issues/132908].)
- Adiciona as funções `math.fmax()`, `math.fmin()` e `math.signbit()`. (Contribuição de Bénédikt Tran em gh-135853 [https://github.com/python/cpython/issues/135853].)

mimetypes

- Adiciona mais tipos MIME. (Contribuição de Benedikt Johannes, Charlie Lin, Foolbar, Gil Forcada e John Franey em gh-144217 [https://github.com/python/cpython/issues/144217], gh-145720 [https://github.com/python/cpython/issues/145720], gh-140937 [https://github.com/python/cpython/issues/140937], gh-139959 [https://github.com/python/cpython/issues/139959], gh-145698 [https://github.com/python/cpython/issues/145698], gh-145718 [https://github.com/python/cpython/issues/145718], gh-145918 [https://github.com/python/cpython/issues/145918] e gh-144213 [https://github.com/python/cpython/issues/144213].)
- Renomeia `application/x-texinfo` para `application/texinfo`. (Contribuição de Charlie Lin em gh-140165 [https://github.com/python/cpython/issues/140165].)
- Alterado o tipo MIME para arquivos `.ai` para `application/pdf`. (Contribuição de Stan Ulbrych em gh-141239 [https://github.com/python/cpython/issues/141239].)

mmap

- `mmap.mmap` agora tem um parâmetro *trackfd* no Windows; se for `False`, o manipulador de arquivo correspondente a *fileno* não será duplicado. (Contribuição de Serhiy Storchaka em gh-78502 [https://github.com/python/cpython/issues/78502].)
- Adicionado o método `mmap.mmap.set_name()` para anotar um mapeamento de memória anônimo se o kernel do Linux oferecer suporte a *PR_SET_VMA_ANON_NAME [https://manpages.debian.org/PR_SET_VMA(2const)]* (Linux 5.17 ou mais recente). (Contribuição de Donghee Na em gh-142419 [https://github.com/python/cpython/issues/142419].)

os

- Adiciona `os.statx()` nas versões 4.11 e posteriores do kernel do Linux com versões 2.28 e posteriores da glibc. (Contribuição de Jeffrey Bosboom e Victor Stinner em gh-83714 [https://github.com/python/cpython/issues/83714].)
- A função `os.makedirs()` agora tem um parâmetro *parent_mode* que permite especificar o modo para diretórios intermediários. Isso pode ser usado para corresponder ao comportamento do Python 3.6 e anterior passando `parent_mode=mode`. (Contribuição de Zackery Spytz e Gregory P. Smith em gh-86533 [https://github.com/python/cpython/issues/86533].)

os.path

- Adiciona suporte ao modo “tudo exceto o último” em `realpath()`. (Contribuição de Serhiy Storchaka em gh-71189 [https://github.com/python/cpython/issues/71189].)
- O parâmetro *strict* para `os.path.realpath()` aceita um novo valor, `os.path.ALLOW_MISSING`. Se usado, erros diferentes de `FileNotFoundError` serão levantados novamente; o caminho resultante pode estar ausente, mas estará livre de links simbólicos. (Contribuição de Petr Viktorin para **CVE 2025-4517** [https://www.cve.org/CVERecord?id=CVE-2025-4517].)

pathlib

- `pathlib.Path.mkdir()` agora tem um parâmetro *parent_mode* que permite especificar o modo para diretórios intermediários quando `parents=True`. (Contribuição de Gregory P. Smith em gh-86533 [https://github.com/python/cpython/issues/86533].)

pdb

- Usa o novo console interativo como o console de entrada padrão para o `pdb`. (Contribuição de Tian Gao em gh-145379 [https://github.com/python/cpython/issues/145379].)

pickle

- Adiciona suporte para serialização com pickle de métodos privados e classes aninhadas. (Contribuição de Zackery Spytz e Serhiy Storchaka em gh-77188 [https://github.com/python/cpython/issues/77188].)

pickletools

- A saída da interface de linha de comando do `pickletools` agora é colorida por padrão. Isso pode ser controlado com variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-149026 [https://github.com/python/cpython/issues/149026].)

pprint

- Adiciona um argumento nomeado *expand* para `pprint.pprint()`, `pprint.pformat()`, `pprint.pp()`. Se verdadeiro, a saída será formatada de forma semelhante ao `json.dumps()` com formatação legível quando *indent* for fornecido. (Contribuição de Stefan Todoran, Semyon Moroz e Hugo van Kemenade em gh-112632 [https://github.com/python/cpython/issues/112632].)
- Adiciona suporte a t-strings ao `pprint`. (Contribuição de Loïc Simon e Hugo van Kemenade em gh-134551 [https://github.com/python/cpython/issues/134551].)

re

- `re.prefixmatch()` e o correspondente `re.Pattern.prefixmatch()` foram adicionados como nomes alternativos e mais explícitos para as APIs existentes do `re.match()` e do `re.Pattern.match()` que agora estão suavemente descontinuadas. Elas foram concebidas para serem usadas a fim de atenuar a confusão sobre o que *match* significa, seguindo o

mantra do Zen do Python *"Explícito é melhor que implícito"*. A maioria das bibliotecas de expressões regulares de outras linguagens usa uma API chamada *match* para significar o que o Python sempre chamou de *search*. (Contribuição de Gregory P. Smith em gh-86519 [https://github.com/python/cpython/issues/86519].)

resource

- Adiciona novas constantes: `RLIMIT_NTHR`, `RLIMIT_UMTXP`, `RLIMIT_THREADS`, `RLIM_SAVED_CUR` e `RLIM_SAVED_MAX`. (Contribuição de Serhiy Storchaka em gh-137512 [https://github.com/python/cpython/issues/137512].)

shelve

- Adicionado novo método `reorganize()` ao `shelve` usado para recuperar espaço livre não utilizado anteriormente ocupado por entradas excluídas. (Contribuição de Andrea Oliveri em gh-134004 [https://github.com/python/cpython/issues/134004].)
- Adiciona suporte para funções personalizadas de serialização e desserialização no módulo `shelve`. (Contribuição de Furkan Onder em gh-99631 [https://github.com/python/cpython/issues/99631].)

socket

- Adiciona constantes para o protocolo ISO-TP CAN. (Contribuição de Patrick Menschel e Stefan Tatschner em gh-86819 [https://github.com/python/cpython/issues/86819].)

sqlite3

- A interface de linha de comando possui vários recursos novos:
  - Conclusão de palavras reservadas SQL com a tecla `Tab`. (Contribuição de Long Tan em gh-133393 [https://github.com/python/cpython/issues/133393].)
  - Prompts, mensagens de erro e textos de ajuda agora são coloridos. Isso está habilitado por padrão; veja Controlando cores para mais detalhes. (Contribuição de Stan Ulbrych e Łukasz Langa em gh-133461 [https://github.com/python/cpython/issues/133461].)
  - Conclusão de tabela, índice, gatilho, visão, coluna, função e esquema com a tecla `Tab`. (Contribuição de Long Tan em gh-136101 [https://github.com/python/cpython/issues/136101].)

ssl

- Indica por meio de `ssl.HAS_PSK_TLS13` se o módulo `ssl` oferece suporte a "PSKs externas" no TLSv1.3, conforme descrito na **RFC 9258** [https://datatracker.ietf.org/doc/html/rfc9258.html]. (Contribuição de Will Childs-Klein em gh-133624 [https://github.com/python/cpython/issues/133624].)

- Adicionados novos métodos para gerenciar grupos usados no acordo de chaves SSL:
  - `ssl.SSLContext.set_groups()` define os grupos permitidos para realizar acordo de chaves, estendendo o método anterior `ssl.SSLContext.set_ecdh_curve()`. Esta nova

API oferece a capacidade de listar múltiplos grupos e oferece suporte a grupos de campo fixo e pós-quânticos, além de curvas ECDH. Este método também pode ser usado para controlar quais compartilhamentos de chave são enviados no handshake TLS.
  - `ssl.SSLSocket.group()` retorna o grupo selecionado para realizar acordo de chaves na conexão atual após a conclusão do handshake TLS. Esta chamada requer OpenSSL 3.2 ou posterior.
  - `ssl.SSLContext.get_groups()` retorna uma lista de todos os grupos de acordo de chaves disponíveis compatíveis com as versões mínima e máxima do TLS atualmente configuradas no contexto. Esta chamada requer OpenSSL 3.5 ou posterior.

(Contribuição de Ron Frederick em gh-136306 [https://github.com/python/cpython/issues/136306].)

- Adicionado um novo método `ssl.SSLContext.set_ciphersuites()` para definir suítes de cifras TLS 1.3. Para o TLS 1.2 ou anterior, `ssl.SSLContext.set_ciphers()` deve continuar a ser usado. Ambas as chamadas podem ser feitas no mesmo contexto e a suíte de cifras selecionada dependerá da versão do TLS negociada quando uma conexão for realizada. (Contribuição de Ron Frederick em gh-137197 [https://github.com/python/cpython/issues/137197].)

- Adicionados novos métodos para gerenciar algoritmos de assinatura:

  - `ssl.get_sigalgs()` retorna uma lista de todos os algoritmos de assinatura TLS disponíveis. Esta chamada requer OpenSSL 3.4 ou posterior.
  - `ssl.SSLContext.set_client_sigalgs()` define os algoritmos de assinatura permitidos para autenticação do cliente baseada em certificado.
  - `ssl.SSLContext.set_server_sigalgs()` define os algoritmos de assinatura permitidos para o servidor concluir o handshake TLS.
  - `ssl.SSLSocket.client_sigalg()` retorna o algoritmo de assinatura selecionado para autenticação do cliente na conexão atual. Esta chamada requer OpenSSL 3.5 ou posterior.
  - `ssl.SSLSocket.server_sigalg()` retorna o algoritmo de assinatura selecionado para o servidor concluir o handshake TLS na conexão atual. Esta chamada requer OpenSSL 3.5 ou posterior.

(Contribuição de Ron Frederick em gh-138252 [https://github.com/python/cpython/issues/138252].)

subprocess

- `subprocess.Popen.wait()`: quando `timeout` não for `None` e a plataforma oferecer suporte, um mecanismo eficiente orientado a eventos é usado para aguardar o encerramento do processo:

  - Linux >= 5.3 usa `os.pidfd_open()` + `select.poll()`.
  - macOS e outras variantes BSD usam `select.kqueue()` + `KQ_FILTER_PROC` + `KQ_NOTE_EXIT`.
  - O Windows continua usando `WaitForSingleObject` (inalterado).

Se nenhum desses mecanismos estiver disponível, a função recorre ao laço de espera ativo tradicional (chamada não bloqueante e pequenas pausas). (Contribuição de Giampaolo Rodola em gh-83069 [https://github.com/python/cpython/issues/83069].)

symtable

- Adiciona os métodos `symtable.Function.get_cells()` e `symtable.Symbol.is_cell()`. (Contribuição de Yashp002 em gh-143504 [https://github.com/python/cpython/issues/143504].)

sys

- Adiciona o espaço de nomes `sys.abi_info` para melhorar o acesso às informações de ABI. (Contribuição de Klaus Zimmermann em gh-137476 [https://github.com/python/cpython/issues/137476].)

sys.monitoring

- Os outros eventos (`PY_THROW`, `PY_UNWIND`, `RAISE`, `EXCEPTION_HANDLED` e `RERAISE`) agora podem ser ativados e desativados por objeto de código. Retornar `DISABLE` a partir de uma função de retorno para um desses eventos desativa o evento para todo o objeto de código (para a ferramenta atual), em vez de levantar `ValueError` como nas versões anteriores. (Contribuição de Gabriele N. Tornetta em gh-146182 [https://github.com/python/cpython/issues/146182].)

tarfile

- `data_filter()` agora normaliza alvos de links simbólicos para evitar ataques de travessia de caminho. (Contribuição de Petr Viktorin em gh-127987 [https://github.com/python/cpython/issues/127987] e **CVE 2025-4138** [https://www.cve.org/CVERecord?id=CVE-2025-4138].)
- `extractall()` agora ignora a correção de atributos de diretório quando um diretório é removido ou substituído por outro tipo de arquivo. (Contribuição de Petr Viktorin em gh-127987 [https://github.com/python/cpython/issues/127987] e **CVE 2024-12718** [https://www.cve.org/CVERecord?id=CVE-2024-12718].)
- `extract()` e `extractall()` agora (re)aplicam o filtro de extração ao substituir um link (físico ou simbólico) por uma cópia de outro membro do arquivo e ao corrigir atributos de diretório. O primeiro levanta uma nova exceção, `LinkFallbackError`. (Contribuição de Petr Viktorin para **CVE 2025-4330** [https://www.cve.org/CVERecord?id=CVE-2025-4330] e **CVE 2024-12718** [https://www.cve.org/CVERecord?id=CVE-2024-12718].)
- `extract()` e `extractall()` não extraem mais membros rejeitados quando `errorlevel()` é zero. (Contribuição de Matt Prodani e Petr Viktorin em gh-112887 [https://github.com/python/cpython/issues/112887] e **CVE 2025-4435** [https://www.cve.org/CVERecord?id=CVE-2025-4435].)
- `extract()` e `extractall()` agora substituem barras por barras invertidas em alvos de links simbólicos no Windows para evitar a criação de links corrompidos. (Contribuição de Christoph Walcher em gh-57911 [https://github.com/python/cpython/issues/57911].)
- `gettarinfo()` agora substitui contrabarras por barras em alvos de links simbólicos no Windows para estar em conformidade com o padrão do formato tar. (Contribuição de

Daniele Nicolodi em gh-151669 [https://github.com/python/cpython/issues/151669].)

threading

- Adicionado **`serialize_iterator`**, **`synchronized_iterator()`** e **`concurrent_tee()`** para dar suporte ao acesso concorrente a geradores e iteradores. (Contribuição de Raymond Hettinger em gh-124397 [https://github.com/python/cpython/issues/124397].)

timeit

- A saída da interface de linha de comando do **`timeit`** agora é colorida por padrão. Isso pode ser controlado com variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-146609 [https://github.com/python/cpython/issues/146609].)
- A interface de linha de comando agora colore os tracebacks de erro por padrão. Isso pode ser controlado com variáveis de ambiente. (Contribuição de Yi Hong em gh-139374 [https://github.com/python/cpython/issues/139374].)
- Torna o tempo alvo de **`timeit.Timer.autorange()`** configurável e adiciona a opção `--target-time` à interface de linha de comando. (Contribuição de Alessandro Cucci e Miikka Koskinen em gh-80642 [https://github.com/python/cpython/issues/80642].)

tkinter

- O método **`tkinter.Text.search()`** agora oferece suporte a dois argumentos adicionais: *nolinestop*, que permite que a busca continue além dos limites da linha; e *strictlimits*, que restringe a busca ao intervalo especificado. (Contribuição de Rihaan Meher em gh-130848 [https://github.com/python/cpython/issues/130848].)
- Um novo método **`tkinter.Text.search_all()`** foi introduzido. Este método permite buscar por todas as correspondências de um padrão usando as opções `-all` e `-overlap` do Tcl. (Contribuição de Rihaan Meher em gh-130848 [https://github.com/python/cpython/issues/130848].)
- Adicionados os novos métodos **`pack_content()`**, **`place_content()`** e **`grid_content()`** que usam comandos Tk com novos nomes (introduzidos no Tk 8.6) em vez dos métodos **`*_slaves()`** que usam comandos Tk com nomes desatualizados. (Contribuição de Serhiy Storchaka em gh-143754 [https://github.com/python/cpython/issues/143754].)
- Adicionados os atributos do **`Event`** **`user_data`** para eventos virtuais do Tk e **`detail`** para os eventos `Enter`, `Leave`, `FocusIn`, `FocusOut` e `ConfigureRequest`. (Contribuição de Matthias Kievernagel e Serhiy Storchaka em gh-47655 [https://github.com/python/cpython/issues/47655].)

tokenize

- A saída da interface de linha de comando do **`tokenize`** agora é colorida por padrão. Isso pode ser controlado com variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-148991 [https://github.com/python/cpython/issues/148991].)

tomllib

- O módulo **`tomllib`** agora suporta o TOML 1.1.0. Esta é uma atualização compatível com versões anteriores, o que significa que todos os documentos válidos em TOML 1.0.0 são

analisados da mesma forma.

As alterações, de acordo com o changelog oficial do TOML [https://github.com/toml-lang/toml/blob/main/CHANGELOG.md], são:

- Permite quebras de linha e vírgulas finais em tabelas em linha.

  Anteriormente, uma tabela em linha tinha que estar em uma única linha e não podia terminar com uma vírgula final. Isso agora foi flexibilizado para que o seguinte seja válido:

  ```
  tbl = {
     key      = "a string",
     moar-tbl =  {
        key = 1,
     },
  }
  ```

- Adiciona a notação `\xHH` a strings básicas para pontos de código abaixo de 255 e o caractere de escape `\e` para o caractere de escape:

  ```
  null = "null byte: \x00; letter a: \x61"
  csi = "\e["
  ```

- Segundos em valores de data/hora e hora agora são opcionais. Os seguintes agora são válidos:

  ```
  dt = 2010-02-03 14:15
  t  = 14:15
  ```

(Contribuição de Taneli Hukkinen em gh-142956 [https://github.com/python/cpython/issues/142956].)

types

- Expõe o tipo de proxy write-through de **`locals()`** como **`types.FrameLocalsProxyType`**. Isso representa o tipo do atributo **`frame.f_locals`**, conforme descrito na **PEP 667** [https://peps.python.org/pep-0667/]. (Contribuição de Peter Bierma em gh-136492 [https://github.com/python/cpython/issues/136492].)

typing

- **PEP 747** [https://peps.python.org/pep-0747/]: Adiciona **`TypeForm`**, uma nova forma especial para anotar valores que são, eles próprios, expressões de tipo. `TypeForm[T]` significa “um objeto de forma de tipo descrevendo `T` (ou um tipo atribuível a `T`)”. Em tempo de execução, `TypeForm(x)` simplesmente retorna `x`, o que permite a anotação explícita de valores de forma de tipo sem alterar o comportamento.

  Isso ajuda bibliotecas que aceitam expressões de tipo fornecidas pelo usuário (por exemplo, `int`, `str | None`, classes **`TypedDict`** ou `list[int]`) a exporem assinaturas precisas:

```
from typing import Any, TypeForm

def cast[T](typ: TypeForm[T], value: Any) -> T: ...
```

(Contribuição de Jelle Zijlstra em gh-145033 [https://github.com/python/cpython/issues/145033].)

- **PEP 728** [https://peps.python.org/pep-0728/]: Adiciona suporte em `TypedDict` para os argumentos de classe *closed* e *extra_items*. Um `TypedDict` fechado não permite chaves extras além das especificadas no corpo da classe, enquanto um `TypedDict` com `extra_items` permite itens extras arbitrários onde os valores são do tipo especificado. (Contribuição de Angela Liss em gh-137840 [https://github.com/python/cpython/issues/137840].)
- Códigos como `class ExtraTypeVars(P1[S], Protocol[T, T2]): ...` agora levantam um `TypeError`, porque `S` não está listado nos parâmetros de `Protocol`. (Contribuição de Nikita Sobolev em gh-137191 [https://github.com/python/cpython/issues/137191].)
- Códigos como `class B2(A[T2], Protocol[T1, T2]): ...` agora tratam corretamente a ordem dos parâmetros de tipo: é `(T1, T2)`, e não `(T2, T1)` como era inferido incorretamente em tempo de execução anteriormente. (Contribuição de Nikita Sobolev em gh-137191 [https://github.com/python/cpython/issues/137191].)
- **PEP 800** [https://peps.python.org/pep-0800/]: Adiciona `@typing.disjoint_base`, um novo decorador que marca uma classe como uma base disjunta. Este é um recurso avançado destinado principalmente a permitir que verificadores de tipo reflitam com precisão a semântica em tempo de execução de tipos definidos como embutidos ou em extensões compiladas. Se uma classe `C` for uma base disjunta, então as classes filhas dessa classe não poderão herdar de outras bases disjuntas que não sejam classes pai ou filho de `C`. (Contribuição de Jelle Zijlstra em gh-148639 [https://github.com/python/cpython/issues/148639].)
- `TypeVarTuple` agora aceita os argumentos nomeados `bound`, `covariant`, `contravariant` e `infer_variance`, correspondendo à interface de `TypeVar` e `ParamSpec`. A semântica de `bound` permanece não definida na especificação.

unicodedata

- O banco de dados Unicode foi atualizado para Unicode 17.0.0. (Contribuição de Benjamin Peterson em gh-138706 [https://github.com/python/cpython/issues/138706]).
- Adiciona as funções `unicodedata.isxidstart()` e `unicodedata.isxidcontinue()` para verificar se um caractere pode iniciar ou continuar um identificador do Anexo do Padrão Unicode #31 [https://www.unicode.org/reports/tr31/]. (Contribuição de Stan Ulbrych em gh-129117 [https://github.com/python/cpython/issues/129117].)
- Adiciona a função `iter_graphemes()` para iterar sobre agrupamentos de grafemas de acordo com as regras definidas no Anexo do Padrão Unicode #29, "Segmentação de Texto Unicode" [https://www.unicode.org/reports/tr29/]. Adiciona as funções `grapheme_cluster_break()`, `indic_conjunct_break()` e `extended_pictographic()` para obter as propriedades do caractere relativas ao algoritmo acima. (Contribuição de Serhiy Storchaka e Guillaume Sanchez em gh-74902 [https://github.com/python/cpython/issues/74902].)
- Adiciona a função `block()` para retornar o bloco Unicode [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G64189] atribuído a um caractere. (Contribuição de Stan Ulbrych em gh-66802 [https://github.com/python/cpython/issues/66802].)

unittest

- `unittest.TestCase.assertLogs()` agora aceitará um formatador para controlar como as mensagens são formatadas. (Contribuição de Garry Cairns em gh-134567 [https://github.com/python/cpython/issues/134567].)
- `unittest.TestCase.assertWarns()` e `unittest.TestCase.assertWarnsRegex()` não descartam mais avisos que não correspondem à categoria ou expressão regular especificada. Gerenciadores de contexto aninhados agora são suportados. (Contribuição de Serhiy Storchaka em gh-143231 [https://github.com/python/cpython/issues/143231].)

urllib.parse

- Adiciona o parâmetro *missing_as_none* às funções `urlsplit()`, `urlparse()` e `urldefrag()`. Adiciona o parâmetro *keep_empty* às funções `urlunsplit()` e `urlunparse()`. Isso permite distinguir entre componentes URI vazios e não definidos e preservar componentes vazios. (Contribuição de Serhiy Storchaka em gh-67041 [https://github.com/python/cpython/issues/67041].)

venv

- Em plataformas POSIX, diretórios platlib serão criados se necessário ao criar ambientes virtuais, em vez de usar um link simbólico `lib64 -> lib`. Isso significa que purelib e platlib de ambientes virtuais não compartilham mais o mesmo diretório `lib` em plataformas onde `sys.platlibdir` não for igual a `lib`. (Contribuição de Rui Xi em gh-133951 [https://github.com/python/cpython/issues/133951].)

avisos

- Melhora a filtragem por módulo em `warnings.warn_explicit()` se nenhum argumento *module* for passado. Agora ele testa a expressão regular de módulo no filtro de avisos não apenas em relação ao nome do arquivo com `.py` removido, mas também em relação a nomes de módulos construídos a partir de diferentes diretórios pai do nome do arquivo (com `/__init__.py`, `.py` e, no Windows, `.pyw` removidos). (Contribuição de Serhiy Storchaka em gh-135801 [https://github.com/python/cpython/issues/135801].)

wave

- Adicionado suporte para áudio WAVE em ponto flutuante IEEE (`WAVE_FORMAT_IEEE_FLOAT`) em `wave`.
- Adicionados `wave.Wave_read.getformat()`, `wave.Wave_write.getformat()` e `wave.Wave_write.setformat()` para manipulação explícita do formato de quadro.
- `wave.Wave_write.setparams()` aceita tanto tuplas de 7 elementos incluindo `format` quanto tuplas de 6 elementos para retrocompatibilidade (recorrendo por padrão a `WAVE_FORMAT_PCM`).
- A saída `WAVE_FORMAT_IEEE_FLOAT` agora inclui um pedaço `fact`, conforme exigido para formatos WAVE não-PCM.

(Contribuição de Lionel Koenig e Michiel W. Beijen em gh-60729 [https://github.com/python/cpython/issues/60729].)

webbrowser

- No macOS, a nova classe `webbrowser.MacOS` abre URLs via **/usr/bin/open** em vez de construir e executar AppleScript via **osascript**. O navegador padrão é detectado a partir do arquivo de preferências do LaunchServices usando `plistlib`, com `com.apple.Safari` como alternativa em instalações novas. Para URLs não HTTP(S), **open -b <bundle-id>** é usado para rotear a URL através de um navegador em vez do manipulador de arquivos do SO, prevenindo ataques de injeção de arquivos. (Contribuição de Jeff Lyon em gh-137586 [https://github.com/python/cpython/issues/137586].)

xml

- Adiciona a função `xml.is_valid_name()` para verificar se uma string pode ser usada como um nome de elemento ou atributo em XML. (Contribuição de Serhiy Storchaka em gh-139489 [https://github.com/python/cpython/issues/139489].)
- Adiciona a função `xml.is_valid_text()` para verificar se uma string pode ser usada em um documento XML. (Contribuição de Serhiy Storchaka em gh-139489 [https://github.com/python/cpython/issues/139489].)

xml.parsers.expat

- Adiciona `SetAllocTrackerActivationThreshold()` e `SetAllocTrackerMaximumAmplification()` a objetos xmlparser para ajustar as proteções contra quantidades desproporcionais de uso de memória dinâmica de dentro de um analisador Expat. (Contribuição de Bénédikt Tran em gh-90949 [https://github.com/python/cpython/issues/90949].)
- Adiciona `SetBillionLaughsAttackProtectionActivationThreshold()` e `SetBillionLaughsAttackProtectionMaximumAmplification()` a objetos xmlparser para ajustar proteções contra ataques de billion laughs [https://en.wikipedia.org/wiki/Billion_laughs_attack]. (Contribuição de Bénédikt Tran em gh-90949 [https://github.com/python/cpython/issues/90949].)

zlib

- Permite combinar dois checksums Adler-32 via `adler32_combine()`. (Contribuição de Callum Attryde e Bénédikt Tran em gh-134635 [https://github.com/python/cpython/issues/134635].)
- Permite combinar dois checksums CRC-32 via `crc32_combine()`. (Contribuição de Bénédikt Tran em gh-134635 [https://github.com/python/cpython/issues/134635].)

Otimizações

- `mimalloc` agora é usado como o alocador padrão para alocações brutas de memória, como por meio de `PyMem_RawMalloc()`, para um melhor desempenho em construções com threads livres. (Contribuição de Kumar Aditya em gh-144914 [https://github.com/python/cpython/issues/144914].)

base64 e binascii

- A implementação base64 subjacente do CPython agora codifica 2x mais rápido e decodifica 3x mais rápido graças a otimizações simples de pipelining da CPU. (Contribuição de Gregory P. Smith e Serhiy Storchaka em gh-143262 [https://github.com/python/cpython/issues/143262].)
- A implementação para as codificações Ascii85, Base85 e Z85 foi reescrita em C. A codificação e decodificação estão agora duas ordens de grandeza mais rápidas e consomem duas ordens de grandeza menos memória. (Contribuição de James Seo e Serhiy Storchaka em gh-101178 [https://github.com/python/cpython/issues/101178].)
- A implementação para Base32 foi reescrita em C. A codificação e decodificação estão agora duas ordens de grandeza mais rápidas. (Contribuição de James Seo em gh-146192 [https://github.com/python/cpython/issues/146192].)

csv

- A detecção de delimitador do `csv.Sniffer.sniff()` está agora até 1,6x mais rápida. (Contribuição de Maurycy Pawłowski-Wieroński em gh-137628 [https://github.com/python/cpython/issues/137628].)

Compilador JIT experimental atualizado

Resultados da suíte de benchmarks pyperformance [https://github.com/python/pyperformance] relatam uma melhoria de desempenho na média geométrica de 7-8% [https://www.doesjitgobrrr.com/run/2026-09-25] para o JIT em relação ao interpretador CPython padrão construído com todas as otimizações habilitadas em Linux x86-64. No macOS AArch64, o JIT tem uma aceleração de 11-12% [https://www.doesjitgobrrr.com/run/2026-09-25] em relação ao interpretador com chamada de cauda (tail call) com todas as otimizações habilitadas. As acelerações para construções com JIT versus construções sem JIT variam de aproximadamente 15% de desaceleração até mais de 100% de aceleração (ignorando o microbenchmark de `unpack_sequence`) em sistemas Linux x86-64 e macOS AArch64.

As principais atualizações no JIT são:

- Dependência do LLVM 21 em tempo de construção
- Novo frontend de rastreamento (tracing)
- Alocação básica de registradores no JIT
- Mais otimizações no JIT
- Suporte a desbobinamento (unwinding) do GDB e `backtrace()` do GNU
- Melhor geração de código de máquina

Dependência do LLVM 21 em tempo de construção

O compilador JIT agora usa LLVM 21 para geração de stencils em tempo de constrção. Como sempre, o LLVM só é necessário ao compilar o CPython com o JIT habilitado; usuários finais que executam o Python não precisam do LLVM instalado. Instruções para instalar o LLVM podem ser encontradas na documentação do compilador JIT [https://github.com/python/cpython/blob/main/Tools/jit/README.md] para todas as plataformas suportadas. (Contribuição de Savannah Ostrowski em gh-140973 [https://github.com/python/cpython/issues/140973].)

Um novo frontend de rastreamento

O compilador JIT agora oferece suporte a significativamente mais operações de bytecode e fluxo de controle do que no Python 3.14, permitindo ganhos de velocidade em uma variedade maior de código. Por exemplo, a criação simples de objetos Python agora é compreendida pelo compilador JIT do 3.15. Operações sobrecarregadas e geradores também são parcialmente suportados. Isso foi tornado possível por um frontend de rastreamento JIT reformulado que registra os caminhos reais de execução pelo código, em vez de estimá-los como a implementação anterior fazia. (Contribuição de Ken Jin em gh-139109 [https://github.com/python/cpython/issues/139109]. Suporte para Windows adicionado por Mark Shannon em gh-141703 [https://github.com/python/cpython/issues/141703].)

Alocação básica de registradores no JIT

Uma forma básica de alocação de registradores foi adicionada ao otimizador do compilador JIT. Isso permite que o compilador JIT evite completamente certas operações de pilha e passe a operar em registradores. Isso permite que o JIT produza traços mais eficientes, evitando leituras e escritas na memória. (Contribuição de Mark Shannon em gh-135379 [https://github.com/python/cpython/issues/135379].)

Mais otimizações no JIT

Mais propagação de constantes [https://en.wikipedia.org/wiki/Constant_folding] é realizada agora. Isso significa que quando o compilador JIT detecta que determinado código do usuário resulta em constantes, o código pode ser simplificado pelo JIT. (Contribuição de Ken Jin e Savannah Ostrowski em gh-132732 [https://github.com/python/cpython/issues/132732].)

Contagens de referências são evitadas sempre que for seguro fazê-lo. Isso geralmente reduz o custo da maioria das operações no Python. (Contribuição de Ken Jin, Donghee Na, Zheao Li, Hai Zhu, Savannah Ostrowski, Reiden Ong, Noam Cohen, Tomas Roun, PuQing, Cajetan Rodrigues e Sacul em gh-134584 [https://github.com/python/cpython/issues/134584].)

Rastreando referências únicas para objetos, o otimizador JIT agora pode eliminar atualizações de contagem de referência e realizar operações no próprio local (in-place) em ints e floats. (Contribuição de Reiden Ong e Pieter Eendebak em gh-143414 [https://github.com/python/cpython/issues/143414] e gh-146306 [https://github.com/python/cpython/issues/146306].)

O otimizador JIT agora oferece suporte a significativamente mais operações do que no 3.14. (Contribuição de Kumar Aditya, Ken Jin, Jiahao Li e Sacul em gh-131798 [https://github.com/python/cpython/issues/131798].)

Suporte a desbobinamento (unwinding) do GDB e `backtrace()` do GNU

O compilador JIT agora publica informações de desbobinamento (unwind) do código de máquina gerado para a interface do GDB em plataformas ELF do Linux suportadas. Quando o registro de quadros do libgcc está disponível, as mesmas informações de desbobinamento também são registradas para os depuradores de pilha `backtrace()` do GNU. Isso permite que depuradores nativos, manipuladores de travamento (crash handlers) e ferramentas de diagnóstico que usam esses mecanismos façam o desbobinamento através de quadros do JIT em vez de parar no código

gerado. (Contribuição de Diego Russo e Pablo Galindo Salgado em gh-146071 [https://github.com/python/cpython/issues/146071] e gh-149104 [https://github.com/python/cpython/issues/149104].)

Melhor geração de código de máquina

O gerador de código de máquina do compilador JIT agora produz um código de máquina melhor para alvos x86-64 e macOS/Linux AArch64. Em geral, os usuários devem experimentar menor uso de memória para o código de máquina gerado e um código de máquina mais eficiente em comparação com o 3.14. (Contribuição de Brandt Bucher em gh-136528 [https://github.com/python/cpython/issues/136528] e gh-135905 [https://github.com/python/cpython/issues/135905]. Implementação para AArch64 foi uma contribuição de Mark Shannon em gh-139855 [https://github.com/python/cpython/issues/139855]. Otimizações adicionais para AArch64 foram uma contribuição de Mark Shannon e Diego Russo em gh-140683 [https://github.com/python/cpython/issues/140683] e gh-142305 [https://github.com/python/cpython/issues/142305].)

Manutenibilidade

As operações do otimizador JIT foram simplificadas. Isso foi tornado possível por uma refatoração das estruturas de dados do JIT. (Contribuição de Zhongtian Zheng em gh-148211 [https://github.com/python/cpython/issues/148211] e Hai Zhu em gh-143421 [https://github.com/python/cpython/issues/143421].)

Removidos

ast

- Os construtores de nós AST agora levantam um **`TypeError`** quando um argumento obrigatório é omitido ou quando um argumento nomeado que não mapeia para um campo no nó AST é passado. Esses casos levantavam anteriormente uma **`DeprecationWarning`** desde o Python 3.13. (Contribuição de Brian Schubert e Jelle Zijlstra em gh-137600 [https://github.com/python/cpython/issues/137600] e gh-105858 [https://github.com/python/cpython/issues/105858].)

collections.abc

- A classe **`collections.abc.ByteString`** foi removida de `collections.abc.__all__`. **`collections.abc.ByteString`** está descontinuado desde o Python 3.12 e está agendado para remoção no Python 3.17. (Contribuição de Alex Waygood em gh-118803 [https://github.com/python/cpython/issues/118803].)

ctypes

- Removida a função não documentada **`ctypes.SetPointerType()`**, que estava descontinuada desde o Python 3.13. (Contribuição de Bénédikt Tran em gh-133866 [https://github.com/python/cpython/issues/133866].)

datetime

- `strptime()` agora levanta `ValueError` quando a string de formatação contém `%d` (dia do mês) sem uma diretiva de ano. Isso estava descontinuado desde o Python 3.13. (Contribuição de Stan Ulbrych e Gregory P. Smith em gh-70647 [https://github.com/python/cpython/issues/70647].)

glob

- Removidas as funções não documentadas **`glob.glob0()`** e **`glob.glob1()`**, que estavam descontinuadas desde o Python 3.13. Use `glob.glob()` e passe um diretório para seu argumento *root_dir* em seu lugar. (Contribuição de Barney Gale em gh-137466 [https://github.com/python/cpython/issues/137466].)

http.server

- Removida a classe **`CGIHTTPRequestHandler`** e o sinalizador `--cgi` da interface de linha de comando **python -m http.server**. Elas foram descontinuadas no Python 3.13. (Contribuição de Bénédikt Tran em gh-133810 [https://github.com/python/cpython/issues/133810].)

importlib

- Os métodos `load_module()` de `Loader` e de suas subclasses foram removidos. O sistema de importação não os chamará mais quando estiverem definidos em subclasses personalizadas. Esse método estava descontinuado em favor de `exec_module()` desde o Python 3.4. (Contribuição de Brett Cannon em gh-97850 [https://github.com/python/cpython/issues/97850].)

importlib.resources

- Removido o parâmetro descontinuado `package` da função `importlib.resources.files()`. (Contribuição de Semyon Moroz em gh-138044 [https://github.com/python/cpython/issues/138044].)

pathlib

- Removido o método descontinuado **`pathlib.PurePath.is_reserved()`**. Use `os.path.isreserved()` para detectar caminhos reservados no Windows. (Contribuição de Nikita Sobolev em gh-133875 [https://github.com/python/cpython/issues/133875].)

platform

- Removida a função **`platform.java_ver()`**, que estava descontinuada desde o Python 3.13. (Contribuição de Alexey Makridenko em gh-133604 [https://github.com/python/cpython/issues/133604].)

sre_*

- Removidos os módulos **`sre_compile`**, **`sre_constants`** e **`sre_parse`**. (Contribuição de Stan Ulbrych em gh-135994 [https://github.com/python/cpython/issues/135994].)

sysconfig

- Removido o parâmetro *check_home* de `sysconfig.is_python_build()`. (Contribuição de Filipe Laíns em gh-92897 [https://github.com/python/cpython/issues/92897].)

threading

- Removido o suporte para argumentos nomeados e posicionais arbitrários na implementação em C de objetos `RLock`. Isso foi descontinuado no Python 3.14. (Contribuição de Bénédikt Tran em gh-134087 [https://github.com/python/cpython/issues/134087].)

types

- Removido **`codeobject.co_lnotab`** do `types.CodeType`, descontinuado desde o Python 3.12 pela **PEP 626** [https://peps.python.org/pep-0626/]. (Contribuição de Nikita Sobolev em gh-134690 [https://github.com/python/cpython/issues/134690].)

typing

- A classe `typing.ByteString` foi removida de `typing.__all__`. **`typing.ByteString`** está descontinuada desde o Python 3.9 e está agendada para remoção no Python 3.17. (Contribuição de Alex Waygood em gh-118803 [https://github.com/python/cpython/issues/118803].)
- A sintaxe de argumentos nomeados não documentada para criar classes `NamedTuple` (por exemplo, `Point = NamedTuple("Point", x=int, y=int)`) não é mais suportada. Use a sintaxe baseada em classe ou a sintaxe funcional em seu lugar. (Contribuição de Bénédikt Tran em gh-133817 [https://github.com/python/cpython/issues/133817].)
- O uso de `TD = TypedDict("TD")` ou `TD = TypedDict("TD", None)` para construir um `TypedDict` com zero campos não é mais suportado. Use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})` em seu lugar. (Contribuição de Bénédikt Tran em gh-133823 [https://github.com/python/cpython/issues/133823].)
- O decorador descontinuado **`@typing.no_type_check_decorator`** foi removido. (Contribuição de Nikita Sobolev em gh-133601 [https://github.com/python/cpython/issues/133601].)

wave

- Removidos os métodos `getmark()`, `setmark()` e `getmarkers()` das classes `Wave_read` e `Wave_write`, que estavam descontinuados desde o Python 3.13. (Contribuição de Bénédikt Tran em gh-133873 [https://github.com/python/cpython/issues/133873].)

zipimport

- Removido o método descontinuado **`zipimport.zipimporter.load_module()`**. Use `zipimport.zipimporter.exec_module()` em seu lugar. (Contribuição de Jiahao Li em gh-133656 [https://github.com/python/cpython/issues/133656].)

Descontinuados

Novas descontinuações

- **`ast`**:
  - A criação de instâncias de nós abstratos da AST (como **`ast.AST`** ou **`ast.expr`**) está descontinuada e levantará um erro no Python 3.20.

    (Contribuição de Brian Schubert em gh-116021 [https://github.com/python/cpython/issues/116021].)

- **`base64`**:
  - Aceitar os caracteres `+` e `/` com um alfabeto alternativo em **`b64decode()`** e **`urlsafe_b64decode()`** agora está descontinuado. Em versões futuras do Python, eles serão erros no modo estrito e descartados no modo não estrito. (Contribuição de Serhiy Storchaka em gh-125346 [https://github.com/python/cpython/issues/125346].)

- CLI:
  - Descontinua as opções de linha de comando **`-b`** e **`-bb`** e agenda-as para se tornarem sem efeito (no-ops) no Python 3.17. Essas eram principalmente auxiliares para a transição do Python 2 -> 3. A partir do Python 3.17, nenhuma **`BytesWarning`** será levantada para esses casos; use um verificador de tipo em seu lugar.

    (Contribuição de Nikita Sobolev em gh-136355 [https://github.com/python/cpython/issues/136355].)

- **`collections.abc`**:
  - As seguintes instruções agora fazem com que `DeprecationWarning`s sejam emitidos em tempo de execução:

    - `from collections.abc import ByteString`
    - `import collections.abc; collections.abc.ByteString`.

    `DeprecationWarning`s já eram emitidas se **`collections.abc.ByteString`** fosse herdada ou usada como o segundo argumento para **`isinstance()`** ou **`issubclass()`**, mas avisos não eram emitidos anteriormente se ela fosse meramente importada ou acessada a partir do módulo **`collections.abc`**. (Contribuição de Alex Waygood em gh-118803 [https://github.com/python/cpython/issues/118803].)

- **`hashlib`**:
  - Em construtores de funções de hash, como **`new()`** ou nos construtores diretos nomeados por hash como **`md5()`** e **`sha256()`**, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado chamado `data=` ou `string=` em várias implementações do **`hashlib`**.

    O suporte para o nome de argumento nomeado `string` agora está descontinuado e está programado para remoção no Python 3.19. Dê preferência a passar os dados iniciais como um argumento posicional para obter máxima retrocompatibilidade.

(Contribuição de Bénédikt Tran em gh-134978 [https://github.com/python/cpython/issues/134978].)

- **`http.cookies`**:
  - **`Morsel.js_output`** e **`BaseCookie.js_output`** estão descontinuados e serão removidos no Python 3.19. Use **`Morsel.output`** ou **`BaseCookie.output`** em seu lugar. (Contribuição de kishorhange111 em gh-148849 [https://github.com/python/cpython/issues/148849].)

- **`imaplib`**:
  - A alteração de **`IMAP4.file`** agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade agora não é utilizada e alterar seu valor *não* fecha explicitamente o arquivo atual. (Contribuição de Bénédikt Tran em gh-142307 [https://github.com/python/cpython/issues/142307].)

- **`profile`**:
  - O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)

- **`re`**:
  - **`re.match()`** e **`re.Pattern.match()`** estão agora suavemente descontinuados em favor das novas APIs **`re.prefixmatch()`** e **`re.Pattern.prefixmatch()`**, adicionadas como nomes alternativos e mais explícitos. Elas visam reduzir a confusão sobre o significado de *match*, seguindo o princípio do Zen do Python: *"Explícito é melhor que implícito"*. A maioria das bibliotecas de expressões regulares de outras linguagens utiliza uma API chamada *match* para a funcionalidade que o Python sempre chamou de *search*.

    Não planejamos remover o nome mais antigo **`match()`**, pois ele tem sido utilizado no código há mais de 30 anos. Códigos que oferecem suporte a versões mais antigas do Python devem continuar utilizando **`match()`**, enquanto novos códigos devem dar preferência a **`prefixmatch()`**. Veja prefixmatch() vs. match().

    (Contribuição de Gregory P. Smith em gh-86519 [https://github.com/python/cpython/issues/86519] e Hugo van Kemenade em gh-148100 [https://github.com/python/cpython/issues/148100].)

- **`struct`**:
  - Chamar `Struct.__new__()` sem um argumento obrigatório agora está descontinuado e será removido no Python 3.20. Chamar o método **`__init__()`** em um objeto **`Struct`** já inicializado está descontinuado e será removido no Python 3.20.

    (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os códigos de tipo `'F'` e `'D'` estão agora suavemente descontinuados em favor das formas de duas letras `'Zf'` e `'Zd'`. (Contribuição de Sergey B Kirpichev em gh-121249 [https://github.com/python/cpython/issues/121249].)

- **`typing`**:
  - As seguintes instruções agora fazem com que `DeprecationWarnings` sejam emitidos em tempo de execução:
    - `from typing import ByteString`
    - `import typing; typing.ByteString`.

    `DeprecationWarnings` já eram emitidas se **`typing.ByteString`** fosse herdada ou usada como o segundo argumento para **`isinstance()`** ou **`issubclass()`**, mas avisos não eram emitidos anteriormente se ela fosse meramente importada ou acessada a partir do módulo **`typing`**. (Contribuição de Alex Waygood em gh-118803 [https://github.com/python/cpython/issues/118803].)
  - É desencorajado realizar verificações com **`isinstance()`** e **`issubclass()`** em classes de protocolo que não foram explicitamente decoradas com **`runtime_checkable()`**, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso resultará em um **`TypeError`** no Python 3.20.

    (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

- **`webbrowser`**:
  - A classe **`webbrowser.MacOSXOSAScript`** está descontinuada em favor de **`webbrowser.MacOS`** e agendada para remoção no Python 3.17. (Contribuição de Jeff Lyon em gh-137586 [https://github.com/python/cpython/issues/137586].)

- `__version__`:
  - Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize **`sys.version_info`** em seu lugar.
    - **`argparse`**
    - **`csv`**
    - **`ctypes`**
    - **`ctypes.macholib`**
    - **`decimal`** (use **`decimal.SPEC_VERSION`**)
    - **`http.server`**
    - **`imaplib`**
    - **`ipaddress`**
    - **`json`**
    - **`logging`** (`__date__` também descontinuado)
    - **`optparse`**

- `pickle`
- `platform`
- `re`
- `socketserver`
- `tabnanny`
- `tarfile`
- `tkinter.font`
- `tkinter.ttk`
- `wsgiref.simple_server`
- `xml.etree.ElementTree`
- `xml.sax.expatreader`
- `xml.sax.handler`
- `zlib`

(Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

Remoção pendente no Python 3.16

- O sistema de importação:
  - A definição de `__loader__` em um módulo enquanto falha na definição de `__spec__.loader` está descontinuado. No Python 3.16, `__loader__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.
- `array`:
  - O código de formato `'u'` (`wchar_t`) foi descontinuado na documentação desde o Python 3.3 e em tempo de execução desde o Python 3.13. Use o código de formato `'w'` (`Py_UCS4`) para caracteres Unicode.
- `asyncio`:
  - `asyncio.iscoroutinefunction()` foi descontinuado e será removido no Python 3.16, use `inspect.iscoroutinefunction()` em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)
  - O sistema de políticas `asyncio` está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:
    - `asyncio.AbstractEventLoopPolicy`
    - `asyncio.DefaultEventLoopPolicy`
    - `asyncio.WindowsSelectorEventLoopPolicy`
    - `asyncio.WindowsProactorEventLoopPolicy`
    - `asyncio.get_event_loop_policy()`
    - `asyncio.set_event_loop_policy()`

Os usuários devem usar `asyncio.run()` ou `asyncio.Runner` com *loop_factory* para usar a implementação de loop de eventos desejada.

por exemplo, para usar `asyncio.SelectorEventLoop` no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventLoop)
```

(Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- `builtins`:
  - A inversão bit a bit em tipos booleanos, `~True` ou `~False` foi descontinuada desde o Python 3.12, pois produz resultados surpreendentes e não intuitivos (`-2` e `-1`). Use `not x` em vez disso para a negação lógica de um Booleano. No caso raro de você precisar da inversão bit a bit do inteiro subjacente, converta para `int` explicitamente (`~int(x)`).
- `functools`:
  - A chamada da implementação Python de `functools.reduce()` com *function* ou *sequence* como argumentos nomeados foi descontinuada desde o Python 3.14.
- `logging`:
  - O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)
- `mimetypes`:
  - Extensões válidas começam com um '.' ou estão vazias para `mimetypes.MimeTypes.add_type()`. Extensões sem ponto estão descontinuadas e levantarão uma exceção `ValueError` no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)
- `shutil`:
  - A exceção **`ExecError`** foi descontinuada desde o Python 3.14. Ela não foi usada por nenhuma função em **`shutil`** desde o Python 3.4, e agora é um alias de `RuntimeError`.
- `symtable`:
  - O método `Class.get_methods` foi descontinuado desde o Python 3.14.
- `sys`:

- A função `_enablelegacywindowsfsencoding()` foi descontinuada desde o Python 3.13. Use a variável de ambiente `PYTHONLEGACYWINDOWSFSENCODING`.

- `sysconfig`:
  - A função `sysconfig.expand_makefile_vars()` está descontinuada desde o Python 3.14. Em vez disso, use o argumento `vars` de `sysconfig.get_paths()`.

- `tarfile`:
  - O atributo não documentado e não utilizado `TarFile.tarfile` foi descontinuado desde o Python 3.13.

Remoção pendente no Python 3.17

- `datetime`:
  - `strptime()` chama usando uma string de formato contendo `%e` (dia do mês) sem um ano. Isso está descontinuado desde Python 3.15. (Contribuição de Stan Ulbrych em gh-70647 [https://github.com/python/cpython/issues/70647].)

- `collections.abc`:
  - `collections.abc.ByteString` está programado para remoção no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use `Buffer` ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de `bytes` e `bytearray`. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de `ByteString` nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como `memoryview`, também nunca foram entendidos como subtipos de `ByteString` (seja em tempo de execução ou por verificadores de tipo estático).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- `encodings`:
  - Passar nomes de *codificação* não ASCII para `encodings.normalize_encoding()` está descontinuado e sua remoção está prevista para o Python 3.17. (Contribuição de Stan Ulbrych em gh-136702 [https://github.com/python/cpython/issues/136702].)

- `profile`:

- O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)

- **`webbrowser`**:

  - **`webbrowser.MacOSXOSAScript`** está descontinuado em favor de **`webbrowser.MacOS`**. (gh-137586 [https://github.com/python/cpython/issues/137586])

- **`typing`**:

  - Antes do Python 3.14, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para retrocompatibilidade, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como **`typing.get_origin()`** e **`typing.get_args()`**, em vez de depender de detalhes de implementação privada.

  - **`typing.ByteString`**, descontinuada desde o Python 3.9, está programada para ser removida no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estático).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`tkinter`**:

  - Os métodos **`trace_variable()`**, **`trace()`** (um apelido de **`trace_variable()`**), **`trace_vdelete()`** e **`trace_vinfo()`** da classe **`tkinter.Variable`**, descontinuados desde o Python 3.14, têm remoção prevista para o Python 3.17. Em vez deles, utilize **`trace_add()`**, **`trace_remove()`** e **`trace_info()`**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

Remoção pendente no Python 3.18

- Não aceita mais um valor booleano quando um descritor de arquivo é esperado. (Contribuição de Serhiy Storchaka em gh-82626 [https://github.com/python/cpython/issues/82626].)

- **decimal**:
  - O especificador de formato de **Decimal** não padrão e não documentado `'N'`, que só é suportado na implementação C do módulo **decimal**, foi descontinuada desde o Python 3.13. (Contribuição de Serhiy Storchaka em gh-89902 [https://github.com/python/cpython/issues/89902].)
- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:
  - Linhas de `import` em arquivos *`name`*`.pth` são silenciosamente ignoradas.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

Remoção pendente no Python 3.19

- **ctypes**:
  - Troca implícita para o layout de estrutura compatível com MSVC definindo **_pack_**, mas não **_layout_** em plataformas não Windows.
- **hashlib**:
  - Em construtores de funções de hash, como **new()**, ou nos construtores nomeados diretamente pelo algoritmo de hash, como **md5()** e **sha256()**, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado `data=` ou `string=` em várias implementações do **hashlib**.

    O suporte ao nome do argumento nomeado `string` está agora descontinuado e previsto para ser removido no Python 3.19.

    Antes do Python 3.13, o parâmetro nomeado `string` não era suportado corretamente, dependendo da implementação de funções de hash do backend. Dê preferência a passar os dados iniciais como um argumento posicional para garantir a máxima compatibilidade com versões anteriores.
- **http.cookies**:
  - **http.cookies.Morsel.js_output()** está descontinuado e será removido no Python 3.19.
  - **http.cookies.BaseCookie.js_output()** está descontinuado e será removido no Python 3.19.
- **imaplib**:
  - A alteração de **IMAP4.file** agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade agora não é utilizada e alterar seu valor não fecha automaticamente o arquivo atual.

Antes do Python 3.14, essa propriedade era usada para implementar os métodos `read()` e `readline()` correspondentes para `IMAP4`, mas isso não ocorre mais desde então.

Remoção pendente no Python 3.20

- Chamar o método `__new__()` de `struct.Struct` sem o argumento *format* está descontinuado e será removido no Python 3.20. Chamar o método `__init__()` em objetos `Struct` já inicializados está descontinuado e será removido no Python 3.20.

  (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize `sys.version_info` em seu lugar.

  - `argparse`
  - `csv`
  - `ctypes`
  - `ctypes.macholib`
  - `decimal` (use `decimal.SPEC_VERSION`)
  - `http.server`
  - `imaplib`
  - `ipaddress`
  - `json`
  - `logging` (`__date__` também descontinuado)
  - `optparse`
  - `pickle`
  - `platform`
  - `re`
  - `socketserver`
  - `tabnanny`
  - `tarfile`
  - `tkinter.font`
  - `tkinter.ttk`
  - `wsgiref.simple_server`
  - `xml.etree.ElementTree`
  - `xml.sax.expatreader`
  - `xml.sax.handler`
  - `zlib`

  (Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

  - Avisos são gerados para linhas `import` encontradas em arquivos *`name`*`.pth`.

- Arquivos *name*`.pth` não são mais decodificados usando a codificação da localidade (*locale encoding*) por padrão. Eles **DEVEM** estar codificados em `utf-8-sig`.

(Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

- **`ast`**:
  - A criação de instâncias de nós abstratos da AST (como **`ast.AST`** ou **`ast.expr`**) está descontinuada e levantará um erro no Python 3.20.

- **`typing`**:
  - É desencorajado realizar verificações com **`isinstance()`** e **`issubclass()`** em classes de protocolo que não foram explicitamente decoradas com **`runtime_checkable()`**, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso resultará em um **`TypeError`** no Python 3.20.

    (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

Remoção pendente em versões futuras

As APIs a seguir serão removidas no futuro, embora atualmente não haja uma data agendada para sua remoção.

- **`argparse`**:
  - O aninhamento de grupos de argumentos e o aninhamento de grupos mutuamente exclusivos estão descontinuado.
  - A passagem do argumento nomeado não documentado *prefix_chars* para **`add_argument_group()`** agora está descontinuado.
  - O conversor de tipo **`argparse.FileType`** está descontinuado.
- **`builtins`**:
  - Geradores: a assinatura `throw(type, exc, tb)` e `athrow(type, exc, tb)` está descontinuada: use `throw(exc)` e `athrow(exc)`, a assinatura do argumento único.
  - Atualmente Python aceita literais numéricos imediatamente seguidos por palavras reservadas como, por exemplo, `0in x`, `1or x`, `0if 1else 2`. Ele permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). Um aviso de sintaxe é levantado se o literal numérico for imediatamente seguido por uma das palavras reservadas **`and`**, **`else`**, **`for`**, **`if`**, **`in`** , **`is`** e **`or`**. Em uma versão futura, será alterado para um erro de sintaxe. (gh-87999 [https://github.com/python/cpython/issues/87999])
  - Suporte para métodos `__index__()` e `__int__()` retornando tipo não-int: esses métodos serão necessários para retornar uma instância de uma subclasse estrita de **`int`**.
  - Suporte para o método `__float__()` retornando uma subclasse estrita de **`float`**: esses métodos serão necessários para retornar uma instância de **`float`**.
  - Suporte para o método `__complex__()` retornando uma subclasse estrita de **`complex`**: esses métodos serão necessários para retornar uma instância de **`complex`**.

    - Passar um número complexo como argumento *real* ou *imag* no construtor `complex()` agora está descontinuado; deve ser passado apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)
- `calendar`: as constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por `calendar.JANUARY` e `calendar.FEBRUARY`. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)
- `codecs`: use `open()` em vez de `codecs.open()`. (gh-133038 [https://github.com/python/cpython/issues/133038])
- **`codeobject.co_lnotab`**: use o método `codeobject.co_lines()`.
- `datetime`:
    - `utcnow()`: use `datetime.datetime.now(tz=datetime.UTC)`.
    - `utcfromtimestamp()`: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- `gettext`: o valor de plural deve ser um número inteiro.
- `importlib`:
    - O parâmetro *debug_override* de `cache_from_source()` foi descontinuado: em vez disso, use o parâmetro *optimization*.
- `importlib.metadata`:
    - Interface de tupla `EntryPoints`.
    - `None` implícito nos valores de retorno.
- `logging`: o método `warn()` foi descontinuado desde o Python 3.3, use `warning()`.
- `mailbox`: o uso da entrada StringIO e do modo de texto foi descontinuado; em vez disso, use BytesIO e o modo binário.
- `os`: chame `os.register_at_fork()` em um processo multithread.
- `os.path`: `os.path.commonprefix()` está descontinuada; use `os.path.commonpath()` para prefixos de caminho. A função **`os.path.commonprefix()`** está sendo descontinuada por possuir nome e módulo enganosos. O uso dessa função não é seguro para prefixos de caminho, apesar de ela estar incluída em um módulo voltado à manipulação de caminhos; isso significa que é fácil introduzir acidentalmente vulnerabilidades de *path traversal* (travessia de diretórios) em programas Python ao utilizá-la.
- **`pydoc.ErrorDuringImport`**: um valor de tupla para o parâmetro *exc_info* foi descontinuado, use uma instância de exceção.
- `re`: regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)
- `shutil`: o parâmetro *onerror* de `rmtree()` foi descontinuado no Python 3.12; use o parâmetro *onexc*.
- Protocolos e opções de `ssl`
    - `ssl.SSLContext` sem argumento de protocolo foi descontinuado.
    - `ssl.SSLContext`: `set_npn_protocols()` e **`selected_npn_protocol()`** foram descontinuados: use ALPN.
    - Opções de `ssl.OP_NO_SSL*`
    - Opções de `ssl.OP_NO_TLS*`
    - `ssl.PROTOCOL_SSLv3`
    - `ssl.PROTOCOL_TLS`

  - `ssl.PROTOCOL_TLSv1`
  - `ssl.PROTOCOL_TLSv1_1`
  - `ssl.PROTOCOL_TLSv1_2`
  - `ssl.TLSVersion.SSLv3`
  - `ssl.TLSVersion.TLSv1`
  - `ssl.TLSVersion.TLSv1_1`
- Métodos de `threading`:
  - **`threading.Condition.notifyAll()`**: use `notify_all()`.
  - **`threading.Event.isSet()`**: use `is_set()`.
  - **`threading.Thread.isDaemon()`**, `threading.Thread.setDaemon()`: use o atributo `threading.Thread.daemon`.
  - **`threading.Thread.getName()`**, `threading.Thread.setName()`: use o atributo `threading.Thread.name`.
  - **`threading.currentThread()`**: use `threading.current_thread()`.
  - **`threading.activeCount()`**: use `threading.active_count()`.
- `typing.Text` (gh-92332 [https://github.com/python/cpython/issues/92332]).
- A classe interna `typing._UnionGenericAlias` não é mais usada para implementar `typing.Union`. Para preservar a compatibilidade com usuários que utilizam esta classe privada, uma correção de compatibilidade será fornecida pelo menos até a versão 3.17 do Python. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)
- `unittest.IsolatedAsyncioTestCase`: foi descontinuado retornar um valor que não seja `None` de um caso de teste.
- Funções descontinuadas de `urllib.parse`: use `urlparse()`
  - `splitattr()`
  - `splithost()`
  - `splitnport()`
  - `splitpasswd()`
  - `splitport()`
  - `splitquery()`
  - `splittag()`
  - `splittype()`
  - `splituser()`
  - `splitvalue()`
  - `to_bytes()`
- `wsgiref`: `SimpleHandler.stdout.write()` não deve fazer gravações parciais.
- `xml.etree.ElementTree`: testar o valor verdade de um `Element` está descontinuado. Em um lançamento futuro isso sempre retornará `True`. Em vez disso, prefira os testes explícitos `len(elem)` ou `elem is not None`.
- `sys._clear_type_cache()` está descontinuada: use `sys._clear_internal_caches()`.

descontinuações suave

Não há planos para remover APIs suavemente descontinuadas

- `re.match()` e `re.Pattern.match()` estão agora suavemente descontinuados em favor das novas APIs `re.prefixmatch()` e `re.Pattern.prefixmatch()`, adicionadas como nomes alternativos e mais explícitos. Elas visam reduzir a confusão sobre o significado de *match*,

seguindo o princípio do Zen do Python: *"Explícito é melhor que implícito"*. A maioria das bibliotecas de expressões regulares de outras linguagens utiliza uma API chamada *match* para a funcionalidade que o Python sempre chamou de *search*.

Não planejamos remover o nome mais antigo **`match()`**, pois ele tem sido utilizado no código há mais de 30 anos. Códigos que oferecem suporte a versões mais antigas do Python devem continuar utilizando **`match()`**, enquanto novos códigos devem dar preferência a **`prefixmatch()`**. Veja prefixmatch() vs. match().

(Contribuição de Gregory P. Smith em gh-86519 [https://github.com/python/cpython/issues/86519] e Hugo van Kemenade em gh-148100 [https://github.com/python/cpython/issues/148100].)

- O uso dos códigos de tipo de formato `'F'` e `'D'` do módulo **`struct`** está agora suavemente descontinuado em favor das formas de duas letras `'Zf'` e `'Zd'`. (Contribuição de Sergey B Kirpichev em gh-121249 [https://github.com/python/cpython/issues/121249].)

Alterações na API C

Novas funcionalidades

- Adiciona as funções **`PyArg_ParseArray()`** e **`PyArg_ParseArrayAndKeywords()`** para analisar argumentos de funções usando a convenção de chamada **`METH_FASTCALL`**. (Contribuição de Victor Stinner em gh-144175 [https://github.com/python/cpython/issues/144175].)

- Adiciona as seguintes funções para o novo tipo **`frozendict`**:

  - **`PyAnyDict_Check()`**
  - **`PyAnyDict_CheckExact()`**
  - **`PyFrozenDict_Check()`**
  - **`PyFrozenDict_CheckExact()`**
  - **`PyFrozenDict_New()`**

  (Contribuição de Victor Stinner em gh-141510 [https://github.com/python/cpython/issues/141510].)

- Adiciona a função **`PyObject_CallFinalizerFromDealloc()`** à API C limitada. (Contribuição de Victor Stinner em gh-146063 [https://github.com/python/cpython/issues/146063].)

- Adiciona as funções **`PySys_GetAttr()`**, **`PySys_GetAttrString()`**, **`PySys_GetOptionalAttr()`** e **`PySys_GetOptionalAttrString()`** como substitutas para **`PySys_GetObject()`**. (Contribuição de Serhiy Storchaka em gh-108512 [https://github.com/python/cpython/issues/108512].)

- Adiciona **`PyUnstable_Unicode_GET_CACHED_HASH()`** para obter o hash em cache de uma string. Veja a documentação para ressalvas. (Contribuição de Petr Viktorin em gh-131510 [https://github.com/python/cpython/issues/131510].)

- Adiciona API para verificar a compatibilidade de ABI de um módulo de extensão: **`Py_mod_abi`**, **`PyABIInfo_Check()`** e **`PyABIInfo_VAR`**. (Contribuição de Petr Viktorin em gh-

137210 [https://github.com/python/cpython/issues/137210].)

- Implementa a **PEP 782** [https://peps.python.org/pep-0782/], a API PyBytesWriter. Adiciona as funções:

  - `PyBytesWriter_Create()`
  - `PyBytesWriter_Discard()`
  - `PyBytesWriter_Finish()`
  - `PyBytesWriter_FinishWithPointer()`
  - `PyBytesWriter_FinishWithSize()`
  - `PyBytesWriter_Format()`
  - `PyBytesWriter_GetData()`
  - `PyBytesWriter_GetSize()`
  - `PyBytesWriter_Grow()`
  - `PyBytesWriter_GrowAndUpdatePointer()`
  - `PyBytesWriter_Resize()`
  - `PyBytesWriter_WriteBytes()`

  (Contribuição de Victor Stinner em gh-129813 [https://github.com/python/cpython/issues/129813].)

- `PyCriticalSection` e funções relacionadas foram adicionadas à ABI Estável. (Contribuição de Petr Viktorin em gh-149225 [https://github.com/python/cpython/issues/149225].)

- Adiciona uma nova API C `PyImport_CreateModuleFromInitfunc()` para criar um módulo a partir de uma *spec* e *initfunc*. (Contribuição de Itamar Oren em gh-116146 [https://github.com/python/cpython/issues/116146].)

- Adiciona `PyTuple_FromArray()` para criar uma `tuple` a partir de um vetor. (Contribuição de Victor Stinner em gh-111489 [https://github.com/python/cpython/issues/111489].)

- Adiciona um novo gancho de exportação de módulo, `PyModExport_*`.

  (Contribuição de Petr Viktorin na **PEP 793** [https://peps.python.org/pep-0793/] e em gh-140550 [https://github.com/python/cpython/issues/140550].)

- Adiciona funções que têm garantia de serem seguras para uso em manipuladores `tp_traverse`: `PyObject_GetTypeData_DuringGC()`, `PyObject_GetItemData_DuringGC()`, `PyType_GetModuleState_DuringGC()`, `PyModule_GetState_DuringGC()`, `PyModule_GetToken_DuringGC()`, `PyType_GetBaseByToken_DuringGC()`, `PyType_GetModule_DuringGC()`, `PyType_GetModuleByToken_DuringGC()`. (Contribuição de Petr Viktorin em gh-145925 [https://github.com/python/cpython/issues/145925].)

- Adiciona `PyObject_Dump()` para descarregar um objeto para `stderr`. Ele só deve ser usado para depuração. (Contribuição de Victor Stinner em gh-141070 [https://github.com/python/cpython/issues/141070].)

- Implementa a **PEP 820** [https://peps.python.org/pep-0820/]: `PySlot` – Sistema unificado de slots para a API C. Veja Definition slots para documentação.

Isso adiciona:

- A estrutura `PySlot`;
- a função `PyType_FromSlots()`;
- novos IDs de slots: `Py_slot_end`, `Py_slot_invalid`; `Py_slot_subslots`, `Py_tp_slots`, `Py_mod_slots`; `Py_tp_name`, `Py_tp_basicsize`, `Py_tp_extra_basicsize`, `Py_tp_itemsize`, `Py_tp_flags`, `Py_tp_metaclass`, `Py_tp_module`;
- macros de conveniência: `PySlot_DATA`, `PySlot_FUNC`, `PySlot_SIZE`, `PySlot_INT64`, `PySlot_UINT64`, `PySlot_STATIC_DATA`, `PySlot_END`, `PySlot_PTR`, `PySlot_PTR_STATIC`.

A função `PyModule_FromSlotsAndSpec()` e o gancho de exportação de módulo `PyModExport` também usam a nova estrutura **`PySlot`**.

As seguintes funções estão suavemente descontinuadas:

- `PyType_FromSpec()`
- `PyType_FromSpecWithBases()`
- `PyType_FromModuleAndSpec()`
- `PyType_FromMetaclass()`
- `PyModule_FromDefAndSpec()`
- `PyModule_FromDefAndSpec2()`
- `PyModule_ExecDef()`

Os slots `Py_tp_bases` e `Py_tp_base` agora são equivalentes: eles podem ser definidos tanto para um único tipo quanto para uma tupla de tipos. O slot **`Py_tp_bases`** é preferido; o outro é ignorado se ambos forem especificados.

(Contribuição de Petr Viktorin em gh-149044 [https://github.com/python/cpython/issues/149044].)

- Adicionadas as funções `PyUnstable_ThreadState_SetStackProtection()` e `PyUnstable_ThreadState_ResetStackProtection()` para definir o endereço base e o tamanho da proteção de pilha de um estado de thread Python. (Contribuição de Victor Stinner em gh-139653 [https://github.com/python/cpython/issues/139653].)

- Adiciona a função da API C `PyUnstable_SetImmortal()` para marcar objetos como imortais. (Contribuição de Kumar Aditya em gh-143300 [https://github.com/python/cpython/issues/143300].)

- Restaura a função privada provisória `_Py_InitializeMain()` removida no Python 3.14. (Contribuição de Victor Stinner em gh-142417 [https://github.com/python/cpython/issues/142417].)

- Adiciona as funções `PyUnstable_DumpTraceback()` e `PyUnstable_DumpTracebackThreads()` para emitir tracebacks de pilha do Python. (Contribuição de Alex Malyshev em gh-145559 [https://github.com/python/cpython/issues/145559].)

Alterações na API C

- Se a flag `Py_TPFLAGS_MANAGED_DICT` ou `Py_TPFLAGS_MANAGED_WEAKREF` estiver definida, então `Py_TPFLAGS_HAVE_GC` também deve estar definida. (Contribuição de Sergey Miryanov

em gh-134786 [https://github.com/python/cpython/issues/134786].)

- **PyDateTime_IMPORT** agora é seguro para thread. O código que verifica diretamente `PyDateTimeAPI` para `NULL` deve ser atualizado para chamar **PyDateTime_IMPORT** em seu lugar. (Contribuição de Kumar Aditya em gh-141563 [https://github.com/python/cpython/issues/141563].)

APIs C removidas

- Removidas funções `PyUnicode` descontinuadas:

  - **PyUnicode_AsDecodedObject()**: use **PyCodec_Decode()**.
  - **PyUnicode_AsDecodedUnicode()**: Use **PyCodec_Decode()** em seu lugar; note que alguns codecs (por exemplo, "base64") podem retornar um tipo diferente de **str**, como **bytes**.
  - **PyUnicode_AsEncodedObject()**: use **PyCodec_Encode()**.
  - **PyUnicode_AsEncodedUnicode()**: Use **PyCodec_Encode()** em seu lugar; note que alguns codecs (por exemplo, "base64") podem retornar um tipo diferente de **bytes**, como **str**.

  (Contribuição de Stan Ulbrych em gh-133612 [https://github.com/python/cpython/issues/133612].)

- **PyImport_ImportModuleNoBlock()**: apelido descontinuado de **PyImport_ImportModule()**. (Contribuição de Bénédikt Tran em gh-133644 [https://github.com/python/cpython/issues/133644].)

- **PyWeakref_GetObject()** e **PyWeakref_GET_OBJECT**: use **PyWeakref_GetRef()** em seu lugar. O pythoncapi-compat project [https://github.com/python/pythoncapi-compat] pode ser usado para obter **PyWeakref_GetRef()** no Python 3.12 e anteriores. (Contribuição de Bénédikt Tran em gh-133644 [https://github.com/python/cpython/issues/133644].)

- Removido o **PySys_ResetWarnOptions()** descontinuado. Limpe **sys.warnoptions** e **warnings.filters** em seu lugar.

  (Contribuição de Nikita Sobolev em gh-138886 [https://github.com/python/cpython/issues/138886].)

As seguintes funções foram removidas em favor de **PyConfig_Get()**. O pythoncapi-compat project [https://github.com/python/pythoncapi-compat] pode ser usado para obter **PyConfig_Get()** no Python 3.13 e anteriores.

- Funções de inicialização do Python:

  - **Py_GetExecPrefix()**: use **PyConfig_Get("base_exec_prefix")** (**sys.base_exec_prefix**) em seu lugar. Use **PyConfig_Get("exec_prefix")** (**sys.exec_prefix**) se ambientes virtuais precisarem ser tratados.
  - **Py_GetPath()**: use **PyConfig_Get("module_search_paths")** (**sys.path**) em seu lugar.
  - **Py_GetPrefix()**: use **PyConfig_Get("base_prefix")** (**sys.base_prefix**) em seu lugar. Use **PyConfig_Get("prefix")** (**sys.prefix**) se ambientes virtuais precisarem ser tratados.
  - **Py_GetProgramFullPath()**: use **PyConfig_Get("executable")** (**sys.executable**) em seu lugar.

- **`Py_GetProgramName()`**: use **`PyConfig_Get("executable")`** (**`sys.executable`**) em seu lugar.
- **`Py_GetPythonHome()`**: use **`PyConfig_Get("home")`** ou a variável de ambiente **`PYTHONHOME`** em seu lugar.

(Contribuição de Bénédikt Tran em gh-133644 [https://github.com/python/cpython/issues/133644].)

APIs C descontinuadas

- Descontinua o suporte da **PEP 456** [https://peps.python.org/pep-0456/] para fornecer uma definição externa do esquema de hash de strings. A remoção está programada para o Python 3.19.

  Anteriormente, incorporadores podiam definir **`Py_HASH_ALGORITHM`** como `Py_HASH_EXTERNAL` para indicar que o esquema de hash era fornecido externamente, mas esse recurso não era documentado, não era testado e era muito provavelmente não utilizado.

  (Contribuição de Bénédikt Tran em gh-141226 [https://github.com/python/cpython/issues/141226].)

- Para formatos de inteiros não assinados em **`PyArg_ParseTuple()`**, aceitar inteiros Python com valor maior do que o valor máximo para o tipo C ou menor do que o valor mínimo para o tipo inteiro com sinal correspondente do mesmo tamanho agora está descontinuado. (Contribuição de Serhiy Storchaka em gh-132629 [https://github.com/python/cpython/issues/132629].)

- **`PyBytes_FromStringAndSize(NULL, len)`** e **`_PyBytes_Resize()`** estão suavemente descontinuadas, use a API **`PyBytesWriter`** em seu lugar. (Contribuição de Victor Stinner em gh-129813 [https://github.com/python/cpython/issues/129813].)

- **`_PyObject_CallMethodId()`**, **`_PyObject_GetAttrId()`** e **`_PyUnicode_FromId()`** foram descontinuadas desde o Python 3.15 e serão removidas no Python 3.20. Em vez delas, use **`PyUnicode_InternFromString()`** and armazene em cache o resultado no estado do módulo e chame **`PyObject_CallMethod()`** ou **`PyObject_GetAttr()`**. (Contribuição de Victor Stinner em gh-141049 [https://github.com/python/cpython/issues/141049].)

- Descontinua o campo **`cval`** do tipo **`PyComplexObject`**. Use **`PyComplex_AsCComplex()`** e **`PyComplex_FromCComplex()`** para converter um número complexo do Python para/da representação C **`Py_complex`**. (Contribuição de Sergey B Kirpichev em gh-128813 [https://github.com/python/cpython/issues/128813].)

- As funções **`_Py_c_sum()`**, **`_Py_c_diff()`**, **`_Py_c_neg()`**, **`_Py_c_prod()`**, **`_Py_c_quot()`**, **`_Py_c_pow()`** e **`_Py_c_abs()`** estão suavemente descontinuadas. (Contribuição de Sergey B Kirpichev em gh-128813 [https://github.com/python/cpython/issues/128813].)

- **`bytes_warning`** está descontinuado desde o Python 3.15 e será removido no Python 3.17. (Contribuição de Nikita Sobolev em gh-136355 [https://github.com/python/cpython/issues/136355].)

- A macro **`Py_INFINITY`** está suavemente descontinuada, use a macro **`INFINITY`** do padrão C11 de `<math.h>` em seu lugar. (Contribuição de Sergey B Kirpichev em gh-141004 [https://github.com/python/cpython/issues/141004].)

- As seguintes macros estão suavemente descontinuadas:

  - **`Py_ALIGNED`**: Prefira `alignas` em seu lugar.
  - **`PY_FORMAT_SIZE_T`**: Use `"z"` diretamente.
  - **`Py_LL`** e **`Py_ULL`**: Use sufixos padrão, `LL` e `ULL`.
  - **`PY_LONG_LONG`**, **`PY_LLONG_MIN`**, **`PY_LLONG_MAX`**, **`PY_ULLONG_MAX`**, **`PY_INT32_T`**, **`PY_UINT32_T`**, **`PY_INT64_T`**, **`PY_UINT64_T`**, **`PY_SIZE_MAX`**: Use tipos/limites do C99.
  - **`Py_UNICODE_SIZE`**: Use `sizeof(wchar_t)` diretamente.
  - **`Py_VA_COPY`**: Use `va_copy` diretamente.

  A macro **`Py_UNICODE_WIDE`**, que estava agendada para remoção, está agora suavemente descontinuada em seu lugar.

  (Contribuição de Petr Viktorin em gh-146175 [https://github.com/python/cpython/issues/146175].)

- **`Py_MATH_El`** e **`Py_MATH_PIl`** estão descontinuadas desde o Python 3.15 e serão removidas no Python 3.20. (Contribuição de Sergey B Kirpichev em gh-141004 [https://github.com/python/cpython/issues/141004].)

Alterações na construção

- Removido o recurso de fallback implícito para a cópia embutida da biblioteca `libmpdec`. Agora isso deve ser explicitamente habilitado com **`--with-system-libmpdec`** definido como `no` ou com **`--without-system-libmpdec`**. (Contribuição de Sergey B Kirpichev em gh-115119 [https://github.com/python/cpython/issues/115119].)

- A nova opção de configuração **`--with-missing-stdlib-config=FILE`** permite que distribuidores passem um arquivo de configuração JSON [https://www.json.org/json-pt.html] contendo mensagens de erro personalizadas para módulos da biblioteca padrão que estejam ausentes ou empacotados separadamente. (Contribuição de Stan Ulbrych e Petr Viktorin em gh-139707 [https://github.com/python/cpython/issues/139707].)

- A nova opção de configuração **`--with-pymalloc-hugepages`** habilita suporte a páginas gigantes (huge pages) para arenas do pymalloc. Quando habilitado, o tamanho da arena aumenta para 2 MiB e a alocação usa `MAP_HUGETLB` (Linux) ou `MEM_LARGE_PAGES` (Windows) com fallback automático para páginas regulares. No Windows, use `build.bat --pymalloc-hugepages`. Em tempo de execução, páginas gigantes devem ser explicitamente habilitadas definindo a variável de ambiente **`PYTHON_PYMALLOC_HUGEPAGES`** para `1`. (Contribuição de Pablo Galindo Salgado em gh-144319 [https://github.com/python/cpython/issues/144319].)

- A anotação do uso de mmap anônimo agora é suportada se o kernel do Linux suportar *PR_SET_VMA_ANON_NAME* *[https://manpages.debian.org/PR_SET_VMA(2const)]* (Linux 5.17 ou mais recente). As anotações são visíveis em `/proc/<pid>/maps` se o kernel der suporte ao recurso e **`-X dev`** for passado ao Python, ou se o Python for compilado no modo de depuração. (Contribuição de Donghee Na em gh-141770 [https://github.com/python/cpython/issues/141770].)

- O CPython agora é compilado com ponteiros de quadro (frame pointers) habilitados por padrão (**PEP 831** [https://peps.python.org/pep-0831/]). Passe **`--without-frame-pointers`** para optar

por não usar.

Autores de extensões C e bibliotecas nativas compiladas com sistemas de compilação personalizados devem garantir que a cadeia de desbobinamento permaneça intacta. Isso geralmente é feito adicionando `-fno-omit-frame-pointer` e flags semelhantes a `CFLAGS`. Veja a documentação de `--without-frame-pointers` para os sinalizadores específicos que o Python usa.

(Contribuição de Pablo Galindo Salgado e Savannah Ostrowski em gh-149201 [https://github.com/python/cpython/issues/149201].)

- Construções de 64 bits usando Visual Studio 2026 (MSVC 18) agora podem usar o novo interpretador com chamada de cauda. Resultados no Visual Studio 18.1.1 relatam entre 15-20% [https://github.com/faster-cpython/ideas/blob/main/results/5800X-msvc.pgo2-vs-msvc.pgo.tc.svg] de aceleração na média geométrica do pyperformance no Windows x86-64 em relação ao interpretador switch-case em um AMD Ryzen 7 5800X. Observamos acelerações variando de 14% para grandes bibliotecas em Python puro até 40% para scripts pequenos em Python puro de longa execução no Windows. Isso foi tornado possível por um novo recurso introduzido no MSVC 18, o qual os binários oficiais de 64 bits do Windows em python.org [https://www.python.org/downloads/windows/] agora usam. (Contribuição de Chris Eibl, Ken Jin e Brandt Bucher em gh-143068 [https://github.com/python/cpython/issues/143068]. Agradecimentos especiais a Steve Dower e à equipe do MSVC, incluindo Hulon Jenkins.)

Portando para o Python 3.15

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

- APIs do `sqlite3.Connection` foram limpas.

  - Todos os parâmetros do `sqlite3.connect()` exceto *database* agora são somente por somente-nomeados.
  - Os três primeiros parâmetros dos métodos `create_function()` e `create_aggregate()` agora são somente posicionais.
  - O primeiro parâmetro dos métodos `set_authorizer()`, `set_progress_handler()` e `set_trace_callback()` agora é somente-posicional.

  (Contribuição de Serhiy Storchaka em gh-133595 [https://github.com/python/cpython/issues/133595].)

- `resource.RLIM_INFINITY` agora é sempre positivo. Passar um valor inteiro negativo que correspondia ao seu valor antigo (como `-1` ou `-3`, dependendo da plataforma) para `resource.setrlimit()` e `resource.prlimit()` está agora descontinuado. (Contribuição de Serhiy Storchaka em gh-137044 [https://github.com/python/cpython/issues/137044].)

- `mmap.mmap.resize()` foi removido em plataformas que não oferecem suporte à chamada de sistema subjacente, em vez de levantar `SystemError`. (Contribuição de An Long em gh-138205 [https://github.com/python/cpython/issues/138205].)

- Um aviso de recurso (resource warning) é emitido agora para um iterador **`xml.etree.ElementTree.iterparse()`** não fechado se ele abriu um arquivo. Use seu método **`close()`** ou o gerenciador de contexto **`contextlib.closing()`** para fechá-lo. (Contribuição de Osama Abdelkader e Serhiy Storchaka em gh-140601 [https://github.com/python/cpython/issues/140601].)

- Se uma opção curta e uma opção longa de hífen único forem passadas para **`argparse.ArgumentParser.add_argument()`**, *dest* agora é inferido da opção longa de hífen único. Por exemplo, em `add_argument('-f', '-foo')`, *dest* é agora `'foo'` em vez de `'f'`. Passe um argumento *dest* explícito para preservar o comportamento antigo. (Contribuição de Serhiy Storchaka em gh-138697 [https://github.com/python/cpython/issues/138697].)

- O preenchimento (padding) de entrada não é mais necessário em **`base64.urlsafe_b64decode()`**. Passe um novo argumento `padded=True` ou use **`base64.b64decode()`** com o argumento `altchars=b'-_'` (isso funciona com versões anteriores do Python) para tornar o preenchimento obrigatório. (Contribuição de Serhiy Storchaka em gh-73613 [https://github.com/python/cpython/issues/73613].)

- Como **`unittest.TestCase.assertWarns()`** e **`unittest.TestCase.assertWarnsRegex()`** não descartam mais avisos que não correspondem à categoria ou expressão regular especificada, seus testes podem começar a deixar passar alguns avisos que antes eram mascarados. Use filtros de avisos para silenciá-los ou **`assertWarns*()`** adicionais para capturá-los e verificá-los. (Contribuição de Serhiy Storchaka em gh-143231 [https://github.com/python/cpython/issues/143231].)

O que há de novo no Python 3.14

Editores:: Adam Turner e Hugo van Kemenade

Este artigo explica os novos recursos do Python 3.14, em comparação com o 3.13. Python 3.14 foi lançado em 7 de outubro de 2025. Veja changelog para uma lista completa de mudanças.

Ver também

PEP 745 [https://peps.python.org/pep-0745/] – Cronograma de lançamento do Python 3.14

Resumo – Destaques da versão

Python 3.14 is a stable release of the Python programming language, with a mix of changes to the language, the implementation, and the standard library. The biggest changes include template string literals, deferred evaluation of annotations, and support for subinterpreters in the standard library.

As mudanças na biblioteca incluem recursos significativamente melhorados para introspecção no asyncio, suporte para Zstandard por meio de um novo módulo `compression.zstd`, destaque de sintaxe no REPL, bem como as descontinuações e remoções habituais e melhorias na facilidade de uso e correção.

Este artigo não tenta fornecer uma especificação completa de todos os novos recursos, mas fornece uma visão geral conveniente. Para detalhes completos, consulte a documentação, como Referência da Biblioteca e Referência da Linguagem. Para entender a implementação completa e a justificativa do design para uma mudança, consulte a PEP para um novo recurso específico; mas observe que as PEPs geralmente não são mantidas

atualizadas depois que um recurso é totalmente implementado. Veja Portando para o Python 3.14 para orientação sobre atualização a partir de versões anteriores do Python.

Melhorias no interpretador:

- **PEP 649** [https://peps.python.org/pep-0649/] e **PEP 749** [https://peps.python.org/pep-0749/]: avaliação adiada de anotações
- **PEP 734** [https://peps.python.org/pep-0734/]: múltiplos interpretadores na biblioteca padrão
- **PEP 750** [https://peps.python.org/pep-0750/]: strings template
- **PEP 758** [https://peps.python.org/pep-0758/]: permitir expressões except e except* sem parênteses
- **PEP 765** [https://peps.python.org/pep-0765/]: fluxo de controle em blocos finally
- **PEP 768** [https://peps.python.org/pep-0768/]: interface segura para depurador externo para CPython
- Um novo tipo de interpretador
- Melhorias no modo com threads livres
- Mensagens de erro melhoradas
- Coleta de lixo incremental

Melhorias significativas na biblioteca padrão:

- **PEP 784** [https://peps.python.org/pep-0784/]: suporte ao Zstandard na biblioteca padrão
- Capacidades de introspecção assíncrona
- Controle de avisos seguros simultâneos
- Realce de sintaxe no console interativo padrão e saída colorida em várias interfaces de linha de comando (CLIs) de módulos da biblioteca padrão

Melhorias na API C:

- **PEP 741** [https://peps.python.org/pep-0741/]: API C de configuração do Python

Suporte à plataforma:

- **PEP 776** [https://peps.python.org/pep-0776/]: Emscripten agora é uma plataforma oficialmente com suporte, no **nível 3** [https://peps.python.org/pep-0011/#tier-3].

Alterações na versão:

- **PEP 779** [https://peps.python.org/pep-0779/]: Python com threads livres é oficialmente suportado
- **PEP 761** [https://peps.python.org/pep-0761/]: Assinaturas PGP foram descontinuadas de lançamentos oficiais
- Versões binárias de Windows e macOS agora oferecem suporte ao compilador experimental just-in-time
- Versões binárias para Android são agora fornecidas

Novas funcionalidades

PEP 649 [https://peps.python.org/pep-0649/] e PEP 749 [https://peps.python.org/pep-0749/]: avaliação adiada de anotações

As anotações em funções, classes e módulos não são mais avaliadas de forma ansiosa. Em vez disso, as anotações são armazenadas em uma função de anotação com propósito especial e avaliadas somente quando necessário (exceto se `from __future__ import annotations` for usado).

Essa alteração foi projetada para melhorar o desempenho e usabilidade de anotações no Python na maioria das circunstâncias. O custo em tempo de execução para definir anotação é minimizado, mas continua sendo possível introspectar anotação em tempo de execução. Não é mais necessário incluir anotação em strings se eles contiverem referências futuras.

O novo módulo **`annotationlib`** fornece ferramentas para inspecionar anotação adiada. A anotação pode ser avaliada nos formatos **`VALUE`** (que avalia anotações para valores em tempo de execução, semelhante ao comportamento nas versões anteriores do Python), **`FORWARDREF`** (que substitui nomes indefinidos por marcadores especiais) e **`STRING`** (que retorna anotações como strings).

Este exemplo mostra como esses formatos se comportam:

```
>>> from annotationlib import get_annotations, Format
>>> def func(arg: Undefined):
...     pass
>>> get_annotations(func, format=Format.VALUE)
Traceback (most recent call last):
  ...
NameError: name 'Undefined' is not defined
>>> get_annotations(func, format=Format.FORWARDREF)
{'arg': ForwardRef('Undefined', owner=<function func at 0x...>)}
>>> get_annotations(func, format=Format.STRING)
{'arg': 'Undefined'}
```

A seção Portando contém orientações sobre alterações que podem ser necessárias devido a essas mudanças, embora na maioria dos casos, o código continuará funcionando como está.

(Contribuição de Jelle Zijlstra em **PEP 749** [https://peps.python.org/pep-0749/] e gh-119180 [https://github.com/python/cpython/issues/119180], **PEP 649** [https://peps.python.org/pep-0649/] foi escrita por Larry Hastings.)

Ver também

PEP 649 **[https://peps.python.org/pep-0649/]**
Avaliação adiada de anotações usando descritores

PEP 749 **[https://peps.python.org/pep-0749/]**
Implementando a PEP 649

PEP 734 [https://peps.python.org/pep-0734/]: múltiplos interpretadores na biblioteca padrão

O ambiente de execução do CPython permite a execução simultânea de várias cópias do Python no mesmo processo há mais de 20 anos. Cada uma dessas cópias separadas é chamada de “interpretador”. No entanto, o recurso estava disponível apenas por meio da API C.

Essa limitação foi removida no Python 3.14, com o novo módulo `concurrent.interpreters`.

Há pelo menos duas razões importantes pelas quais o uso de múltiplos interpretadores traz benefícios significativos:

- eles oferecem suporte a um novo modelo (para Python) de simultaneidade, amigável ao ser humano
- verdadeiro paralelismo multi-core

Em alguns casos de uso, a simultaneidade em software melhora a eficiência e pode simplificar o design em alto nível. Ao mesmo tempo, implementar e manter todos os tipos de simultaneidade, exceto as mais simples, costuma ser uma luta para o cérebro humano. Isso se aplica especialmente a threads simples (por exemplo, `threading`), em que toda a memória é compartilhada entre todas as threads.

Com múltiplos interpretadores isolados, você pode aproveitar vários modelos de simultaneidade, como o Communicating Sequential Processes (CSP) e o modelo de atores, que já obtiveram sucesso em outras linguagens de programação, como Smalltalk, Erlang, Haskell e Go. Pense em múltiplos interpretadores como threads, mas com compartilhamento opcional.

Sobre o paralelismo multinúcleo: a partir do Python 3.12, os interpretadores agora estão suficientemente isolados uns dos outros para serem usados em paralelo (veja **PEP 684** [https://peps.python.org/pep-0684/]). Isso destrava uma variedade de casos de uso intensivos de CPU em Python que eram limitados pela GIL.

O uso de múltiplos interpretadores é semelhante, em muitos aspectos, ao `multiprocessing`, pois ambos fornecem “processos” lógicos isolados que podem ser executados em paralelo, sem compartilhamento por padrão. No entanto, ao usar múltiplos interpretadores, uma aplicação consumirá menos recursos do sistema e operará com mais eficiência (já que permanece no mesmo processo). Pense em múltiplos interpretadores como se tivessem o isolamento de processos com a eficiência de threads.

Embora o recurso já exista há décadas, múltiplos interpretadores não têm sido amplamente utilizados devido à baixa conscientização e à ausência de um módulo da biblioteca padrão. Consequentemente, eles atualmente apresentam diversas limitações notáveis, que devem melhorar significativamente agora que o recurso finalmente está se tornando popular.

Limitações atuais:

- iniciar cada interpretador ainda não foi otimizado
- cada interpretador usa mais memória do que o necessário (o trabalho continua no compartilhamento interno extensivo entre interpretadores)
- não há muitas opções *ainda* para realmente compartilhar objetos ou outros dados entre interpretadores (além de `memoryview`)
- muitos módulos de extensão de terceiros no PyPI ainda não são compatíveis com múltiplos interpretadores (os módulos de extensão de biblioteca padrão *são* compatíveis)
- a abordagem para escrever aplicações que usam vários interpretadores isolados é, em grande parte, desconhecida para usuários do Python, por enquanto

O impacto dessas limitações dependerá de futuras melhorias no CPython, do uso dos interpretadores e do que a comunidade resolver por meio dos pacotes do PyPI. Dependendo do caso de uso, as limitações podem não ter muito impacto, então experimente!

Além disso, versões futuras do CPython reduzirão ou eliminarão a sobrecarga e fornecerão utilitários menos apropriados no PyPI. Enquanto isso, a maioria das limitações também pode ser solucionada por meio de módulos de extensão, o que significa que os pacotes do PyPI podem preencher qualquer lacuna da versão 3.14 e até mesmo da versão 3.12, onde os interpretadores foram finalmente isolados adequadamente e pararam de compartilhar a GIL. Da mesma forma, espera-se que bibliotecas no PyPI surjam para abstrações de alto nível sobre os interpretadores.

Em relação aos módulos de extensão, estamos trabalhando para atualizar alguns projetos no PyPI, bem como ferramentas como Cython, pybind11, nanobind e PyO3. As etapas para isolar um módulo de extensão podem ser encontradas em Isolando módulos de extensão. O isolamento de um módulo

tem muitas sobreposições com o que é necessário para oferecer suporte a threads livres, portanto, o trabalho contínuo da comunidade nessa área ajudará a acelerar o suporte a múltiplos interpretadores.

Também adicionado no 3.14: concurrent.futures.InterpreterPoolExecutor.

(Contribuição de Eric Snow em gh-134939 [https://github.com/python/cpython/issues/134939])

Ver também

PEP 734 [https://peps.python.org/pep-0734/]

PEP 750 [https://peps.python.org/pep-0750/]: literais de string template

Strings templates são um novo mecanismo para o processamento personalizado de strings. Elas compartilham a sintaxe familiar das f-strings, mas, diferentemente delas, retornam um objeto que representa as partes estáticas e interpoladas da string, em vez de um simples `str`.

Para escrever uma t-string, use um prefixo `'t'` em vez de `'f'`:

```
>>> variety = 'Stilton'
>>> template = t'Experimente um pouco de queijo {variety}!'
>>> type(template)
<class 'string.templatelib.Template'>
```

Objetos `Template` fornecem acesso às partes estáticas e interpoladas (entre chaves) de uma string *antes* de serem combinadas. Itere sobre instâncias de `Template` para acessar suas partes em ordem:

```
>>> list(template)
['Experimente um pouco de queijo ', Interpolation('Stilton', 'va
```

É fácil escrever (ou chamar) código para processar instâncias de `Template`. Por exemplo, aqui está uma função que renderiza partes estáticas em

minúsculas e instâncias de **`Interpolation`** em maiúsculas:

```
from string.templatelib import Interpolation

def lower_upper(template):
    """Render static parts lowercase and interpolations uppercas
    parts = []
    for part in template:
        if isinstance(part, Interpolation):
            parts.append(str(part.value).upper())
        else:
            parts.append(part.lower())
    return ''.join(parts)

name = 'Wenslydale'
template = t'Mister {name}'
assert lower_upper(template) == 'mister WENSLYDALE'
```

Como as instâncias de **`Template`** distinguem entre strings estáticas e interpolações em tempo de execução, elas podem ser úteis para sanitizar a entrada do usuário. Escrever uma função **`html()`** que escape a entrada do usuário em HTML é um exercício deixado para o leitor! O código de processamento de template pode proporcionar maior flexibilidade. Por exemplo, uma função **`html()`** mais avançada poderia aceitar um **`dict`** de atributos HTML diretamente no template:

```
attributes = {'src': 'limburger.jpg', 'alt': 'lovely cheese'}
template = t'<img {attributes}>'
assert html(template) == '<img src="limburger.jpg" alt="lovely c
```

É claro que o código de processamento de template não precisa retornar um resultado semelhante a uma string. Uma função **`html()`** ainda *mais* avançada poderia retornar um tipo personalizado que represente uma estrutura semelhante a um DOM.

Com t-strings, os desenvolvedores podem escrever sistemas que higienizam SQL, executar comandos de shell com segurança, melhorar o registro, explorar ideias modernas em desenvolvimento web (HTML, CSS e assim por diante) e implementar DSLs comerciais leves e personalizadas.

(Contribuição de Jim Baker, Guido van Rossum, Paul Everitt, Koudai Aono, Lysandros Nikolaou, Dave Peck, Adam Turner, Jelle Zijlstra, Bénédikt Tran e Pablo Galindo Salgado em gh-132661 [https://github.com/python/cpython/issues/132661].)

Ver também

PEP 750 [https://peps.python.org/pep-0750/].

PEP 768 [https://peps.python.org/pep-0768/]: interface segura para depurador externo

Python 3.14 introduz uma interface de depuração sem sobrecarga que permite que depuradores e perfiladores se conectem com segurança a processos Python em execução sem interrompê-los ou reiniciá-los. Esta é uma melhoria significativa nos recursos de depuração do Python, o que significa que alternativas inseguras não são mais necessárias.

A nova interface fornece pontos de execução seguros para anexar código do depurador sem modificar o caminho de execução normal do interpretador ou adicionar qualquer sobrecarga em tempo de execução. Graças a isso, as ferramentas agora podem inspecionar e interagir com aplicações Python em tempo real, um recurso crucial para sistemas de alta disponibilidade e ambientes de produção.

Por conveniência, essa interface é implementada na função `sys.remote_exec()`. Por exemplo:

```
import sys
from tempfile import NamedTemporaryFile

with NamedTemporaryFile(mode='w', suffix='.py', delete=False) as
    script_path = f.name
    f.write(f'import my_debugger; my_debugger.connect({os.getpic

# Executa no processo com PID 1234
```

```
print('Behold! An offering:')
sys.remote_exec(1234, script_path)
```

Esta função permite o envio de código Python para ser executado em um processo de destino no próximo ponto de execução seguro. No entanto, os autores da ferramenta também podem implementar o protocolo diretamente, conforme descrito na PEP, que detalha os mecanismos subjacentes usados para anexar com segurança a processos em execução.

A interface de depuração foi cuidadosamente projetada com a segurança em mente e inclui vários mecanismos para controlar o acesso:

- Uma variável de ambiente **`PYTHON_DISABLE_REMOTE_DEBUG`**.
- Uma opção de linha de comando **`-X disable-remote-debug`**.
- Um sinalizador de configuração **`--without-remote-debug`** para desabilitar completamente o recurso no momento da construção.

(Contribuição de Pablo Galindo Salgado, Matt Wozniski e Ivona Stojanovic em gh-131591 [https://github.com/python/cpython/issues/131591].)

Ver também

PEP 768 [https://peps.python.org/pep-0768/].

Um novo tipo de interpretador

Um novo tipo de interpretador foi adicionado ao CPython. Ele utiliza chamadas de cauda entre pequenas funções C que implementam opcodes Python individuais, em vez de uma única instrução `case` C. Em alguns compiladores mais recentes, este interpretador oferece desempenho significativamente melhor. Benchmarks preliminares sugerem uma média geométrica de 3 a 5% mais rápida no conjunto de benchmarks padrão `pyperformance`, dependendo da plataforma e da arquitetura. A referência é o Python 3.14, construído com o Clang 19, sem este novo interpretador.

Atualmente, este interpretador funciona apenas com o Clang 19 e versões mais recentes em arquiteturas x86-64 e AArch64. No entanto, espera-se que uma versão futura do GCC também ofereça suporte a ele.

Este recurso é opcional por enquanto. É altamente recomendável habilitar a otimização guiada por perfil ao usar o novo interpretador, pois é a única configuração testada e validada para melhorar o desempenho. Para mais informações, consulte `--with-tail-call-interp`.

Nota

Isso não deve ser confundido com a otimização de chamada de cauda [https://en.wikipedia.org/wiki/Tail_call] de funções Python, que atualmente não está implementada no CPython.

Este novo tipo de interpretador é um detalhe de implementação interna do interpretador CPython. Ele não altera em nada o comportamento visível dos programas Python. Pode melhorar o desempenho deles, mas não altera nada mais.

(Contribuição de Ken Jin em gh-128563 [https://github.com/python/cpython/issues/128563], com ideias sobre como implementar isso em CPython por Mark Shannon, Garrett Gu, Haoran Xu e Josh Haberman.)

Melhorias no modo com threads livres

O modo de threads livres do CPython (**PEP 703** [https://peps.python.org/pep-0703/]), adicionado inicialmente na versão 3.13, foi significativamente aprimorado no Python 3.14. A implementação descrita na PEP 703 foi concluída, incluindo alterações na API C, e soluções temporárias no interpretador foram substituídas por soluções mais permanentes. O interpretador adaptativo especializado (**PEP 659** [https://peps.python.org/pep-0659/]) agora está habilitado no modo de threads livres, o que, juntamente com muitas outras otimizações, melhora significativamente seu desempenho. A perda de desempenho em código de thread única no modo

de threads livres agora é de aproximadamente 5 a 10%, dependendo da plataforma e do compilador C utilizados.

A partir do Python 3.14, ao compilar módulos de extensão para a construção do CPython com threads livres no Windows, a variável de pré-processador `Py_GIL_DISABLED` agora precisa ser especificada pelo backend de construção, pois não será mais determinada automaticamente pelo compilador C. Para um interpretador em execução, a configuração usada em tempo de compilação pode ser encontrada com `sysconfig.get_config_var()`.

O novo sinalizador `-X context_aware_warnings` controla se o controle de avisos seguros concorrentes está habilitado. O sinalizador padrão é verdadeiro para a construção com threads livres e falso para a construção com a GIL habilitada.

Um novo sinalizador `thread_inherit_context` foi adicionado, o que, se habilitado, significa que threads criadas com `threading.Thread` iniciam com uma cópia de `Context()` do chamador de `start()`. Mais significativamente, isso faz com que o contexto de filtragem de avisos estabelecido por `catch_warnings` seja “herdado” por threads (ou tarefas assíncronas) iniciadas dentro desse contexto. Isso também afeta outros módulos que usam variáveis de contexto, como o gerenciador de contexto `decimal`. Este sinalizador assume como padrão verdadeiro para a construção com threads livres e falso para a construção com a GIL habilitada.

(Contribuição de Sam Gross, Matt Page, Neil Schemenauer, Thomas Wouters, Donghee Na, Kirill Podoprigora, Ken Jin, Itamar Oren, Brett Simmers, Dino Viehland, Nathan Goldbaum, Ralf Gommers, Lysandros Nikolaou, Kumar Aditya, Edgar Margffoy e muitos outros. Alguns desses colaboradores são funcionários da Meta, que continuou a fornecer recursos de engenharia significativos para dar suporte a este projeto.)

Mensagens de erro melhoradas

- O interpretador agora fornece sugestões úteis ao detectar erros de digitação em palavras reservadas do Python. Ao encontrar uma palavra que se assemelha muito a uma palavra reservada do Python, o interpretador sugere a palavra reservada correta na mensagem de erro. Esse recurso ajuda os programadores a identificar e corrigir rapidamente erros comuns de digitação. Por exemplo:

  ```
  >>> while True:
  ...     pass
  Traceback (most recent call last):
    File "<stdin>", line 1
      while True:
      ^^^^^^
  SyntaxError: invalid syntax. Did you mean 'while'?
  ```

  Embora o recurso se concentre nos casos mais comuns, algumas variações de erros ortográficos ainda podem resultar em erros de sintaxe comuns. (Contribuição de Pablo Galindo em gh-132449 [https://github.com/python/cpython/issues/132449].)

- Instruções **`elif`** que seguem um bloco **`else`** agora têm uma mensagem de erro específica. (Contribuição de Steele Farnsworth em gh-129902 [https://github.com/python/cpython/issues/129902].)

  ```
  >>> if who == "eu":
  ...     print("Sou eu!")
  ... else:
  ...     print("Não sou eu!")
  ... elif who is None:
  ...     print("Quem é?")
  File "<stdin>", line 5
    elif who is None:
    ^^^^
  SyntaxError: 'elif' block follows an 'else' block
  ```

- Se uma instrução for passada para Expressões condicionais após **`else`**, ou uma dentre **`pass`**, **`break`** ou **`continue`** for passada antes de **`if`**, a mensagem de erro destacará onde a **`expressão`** é necessária. (Contribuição de Sergey Miryanov em gh-129515 [https://github.com/python/cpython/issues/129515].)

```
>>> x = 1 if True else pass
Traceback (most recent call last):
  File "<string>", line 1
    x = 1 if True else pass
                       ^^^^
SyntaxError: expected expression after 'else', but statement

>>> x = continue if True else break
Traceback (most recent call last):
  File "<string>", line 1
    x = continue if True else break
        ^^^^^^^^
SyntaxError: expected expression before 'if', but statement
```

- Quando strings fechadas incorretamente são detectadas, a mensagem de erro sugere que pode ser que a intenção era a string ser parte da string. (Contribuição de Pablo Galindo em gh-88535 [https://github.com/python/cpython/issues/88535].)

```
>>> "O objeto interessante "O objeto interessante" é muito
Traceback (most recent call last):
SyntaxError: invalid syntax. Is this intended to be part of
```

- Quando strings têm prefixos incompatíveis, o erro agora mostra quais prefixos são incompatíveis. (Contribuição de Nikita Sobolev em gh-133197 [https://github.com/python/cpython/issues/133197].)

```
>>> ub'abc'
  File "<python-input-0>", line 1
    ub'abc'
    ^^
SyntaxError: 'u' and 'b' prefixes are incompatible
```

- Mensagens de erro aprimoradas ao usar `as` com alvos incompatíveis em:
  - Importações: `import ... as ...`
  - Importações “from”: `from ... import ... as ...`
  - Manipuladores “except”: `except ... as ...`

- Casos de correspondência de padrão: `case ... as ...`

(Contribuição de Nikita Sobolev in gh-123539 [https://github.com/python/cpython/issues/123539], gh-123562 [https://github.com/python/cpython/issues/123562] e gh-123440 [https://github.com/python/cpython/issues/123440].)

- Mensagem de erro aprimorada ao tentar adicionar uma instância de um tipo não-hasheável a um **`dict`** ou **`set`**. (Contribuição de CF Bolz-Tereick e Victor Stinner em gh-132828 [https://github.com/python/cpython/issues/132828].)

```
>>> s = set()
>>> s.add({'pages': 12, 'grade': 'A'})
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    s.add({'pages': 12, 'grade': 'A'})
    ~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: cannot use 'dict' as a set element (unhashable ty
>>> d = {}
>>> l = [1, 2, 3]
>>> d[l] = 12
Traceback (most recent call last):
  File "<python-input-4>", line 1, in <module>
    d[l] = 12
    ~^^^
TypeError: cannot use 'list' as a dict key (unhashable type
```

- Melhoria na mensagem de erro quando um objeto que oferece suporte ao protocolo do gerenciador de contexto síncrono é inserido usando **`async with`** em vez de **`with`**, e vice-versa para o protocolo do gerenciador de contexto assíncrono. (Contribuição de Bénédikt Tran em gh-128398 [https://github.com/python/cpython/issues/128398].)

PEP 784 [https://peps.python.org/pep-0784/]: suporte ao Zstandard na biblioteca padrão

O novo pacote **`compression`** contém os módulos **`compression.lzma`**, **`compression.bz2`**, **`compression.gzip`** e **`compression.zlib`**, que

reexportam os módulos **lzma**, **bz2**, **gzip** e **zlib**, respectivamente. Os novos nomes de importação em **compression** são os nomes preferidos para importar esses módulos de compactação no Python 3.14. No entanto, os nomes dos módulos existentes não foram descontinuados. Qualquer descontinuação ou remoção dos módulos de compressão existentes ocorrerá, no mínimo, 5 anos após o lançamento da versão 3.14.

O novo módulo **compression.zstd** fornece APIs de compactação e descompactação para o formato Zstandard por meio de ligações à biblioteca zstd da Meta [https://facebook.github.io/zstd/]. O Zstandard é um formato de compactação amplamente adotado, altamente eficiente e rápido. Além das APIs introduzidas em **compression.zstd**, o suporte para leitura e gravação de arquivos compactados do Zstandard foi adicionado aos módulos **tarfile**, **zipfile** e **shutil**.

Aqui está um exemplo de uso do novo módulo para compactar alguns dados:

```
from compression import zstd
import math

data = str(math.pi).encode() * 20
compressed = zstd.compress(data)
ratio = len(compressed) / len(data)
print(f"Achieved compression ratio of {ratio}")
```

Como pode ser visto, a API é semelhante às APIs dos módulos **lzma** e **bz2**.

(Contribuição de Emma Harper Smith, Adam Turner, Gregory P. Smith, Tomas Roun, Victor Stinner e Rogdham em gh-132983 [https://github.com/python/cpython/issues/132983].)

Ver também

PEP 784 [https://peps.python.org/pep-0784/].

Capacidades de introspecção assíncrona

Foi adicionada uma nova interface de linha de comando para inspecionar processos Python em execução que utilizam tarefas assíncronas, disponível via `python -m asyncio ps PID` ou `python -m asyncio pstree PID`.

O subcomando `ps` inspeciona o ID do processo (PID) fornecido e exibe informações sobre as tarefas assíncronas em execução. Ela gera uma tabela de tarefas: uma lista simples de todas as tarefas, seus nomes, suas pilhas de corrotinas e quais tarefas as aguardam.

O subcomando `pstree` busca as mesmas informações, mas em vez disso renderiza uma árvore de chamadas assíncrona visual, mostrando os relacionamentos das corrotinas em um formato hierárquico. Este comando é particularmente útil para depurar programas assíncronos de longa duração ou travados. Ele pode ajudar os desenvolvedores a identificar rapidamente onde um programa está bloqueado, quais tarefas estão pendentes e como as corrotinas estão encadeadas.

Por exemplo, dado este código:

```
import asyncio

async def play_track(track):
    await asyncio.sleep(5)
    print(f'□ Finished: {track}')

async def play_album(name, tracks):
    async with asyncio.TaskGroup() as tg:
        for track in tracks:
            tg.create_task(play_track(track), name=track)

async def main():
    async with asyncio.TaskGroup() as tg:
        tg.create_task(
          play_album('Sundowning', ['TNDNBTG', 'Levitate']),
          name='Sundowning')
        tg.create_task(
          play_album('TMBTE', ['DYWTYLM', 'Aqua Regia']),
          name='TMBTE')
```

```
if __name__ == '__main__':
    asyncio.run(main())
```

Executar a nova ferramenta no processo em execução produzirá uma tabela como esta:

```
python -m asyncio ps 12345

tid        task id              task name            coroutine s
-------------------------------------------------------------------
1935500    0x7fc930c18050       Task-1               TaskGroup._
1935500    0x7fc930c18230       Sundowning           TaskGroup._
1935500    0x7fc93173fa50       TMBTE                TaskGroup._
1935500    0x7fc93173fdf0       TNDNBTG              sleep -> pl
1935500    0x7fc930d32510       Levitate             sleep -> pl
1935500    0x7fc930d32890       DYWTYLM              sleep -> pl
1935500    0x7fc93161ec30       Aqua Regia           sleep -> pl
```

ou uma árvore como esta:

```
python -m asyncio pstree 12345

└── (T) Task-1
    └── main example.py:13
        └── TaskGroup.__aexit__ Lib/asyncio/taskgroups.py:72
            └── TaskGroup._aexit Lib/asyncio/taskgroups.py:121
                ├── (T) Sundowning
                │   └── album example.py:8
                │       └── TaskGroup.__aexit__ Lib/asyncio/tas
                │           └── TaskGroup._aexit Lib/asyncio/ta
                │               ├── (T) TNDNBTG
                │               │   └── play example.py:4
                │               │       └── sleep Lib/asyncio/t
                │               └── (T) Levitate
                │                   └── play example.py:4
                │                       └── sleep Lib/asyncio/t
                └── (T) TMBTE
                    └── album example.py:8
                        └── TaskGroup.__aexit__ Lib/asyncio/tas
                            └── TaskGroup._aexit Lib/asyncio/ta
                                ├── (T) DYWTYLM
                                │   └── play example.py:4
```

```
│        └──  sleep Lib/asyncio/t
└── (T) Aqua Regia
    └──  play example.py:4
         └──  sleep Lib/asyncio/t
```

Se um ciclo for detectado no grafo async await (o que pode indicar um problema de programação), a ferramenta levanta um erro e lista os caminhos de ciclo que impedem a construção da árvore:

```
python -m asyncio pstree 12345

ERROR: await-graph contains cycles - cannot print a tree!

cycle: Task-2 → Task-3 → Task-2
```

(Contribuição de Pablo Galindo, Łukasz Langa, Yury Selivanov e Marta Gomez Macias em gh-91048 [https://github.com/python/cpython/issues/91048].)

Controle de avisos seguros simultâneos

O gerenciador de contexto `warnings.catch_warnings` agora usará, opcionalmente, uma variável de contexto para filtros de aviso. Isso é habilitado definindo o sinalizador `context_aware_warnings`, seja com a opção de linha de comando `-X` ou com uma variável de ambiente. Isso fornece controle previsível de avisos ao usar **`catch_warnings`** em conjunto com múltiplas threads ou tarefas assíncronas. O sinalizador padrão é verdadeiro para a construção com threads livres e falso para a construção com GIL habilitada.

(Contribuição de Neil Schemenauer e Kumar Aditya em gh-130010 [https://github.com/python/cpython/issues/130010].)

Outras mudanças na linguagem

- Todas as páginas de código do Windows agora são suportadas como codecs ‘cpXXX’. (Contribuição de Serhiy Storchaka em gh-123803 [https://github.com/python/cpython/issues/123803].)

- Implementa regras aritméticas de modo misto combinando números reais e complexos, conforme especificado pelo C padrão desde C99. (Contribuição de Sergey B Kirpichev em gh-69639 [https://github.com/python/cpython/issues/69639].)
- Mais erros de sintaxe agora são detectados, independentemente da otimização e da opção de linha de comando `-O`. Isso inclui escritas no `__debug__`, uso incorreto de `await` e compreensões assíncronas fora de funções assíncronas. Por exemplo, `python -O -c 'assert (__debug__ := 1)'` ou `python -O -c 'assert await 1'` agora produzem `SyntaxError`s. (Contribuição de Irit Katriel e Jelle Zijlstra em gh-122245 [https://github.com/python/cpython/issues/122245] e gh-121637 [https://github.com/python/cpython/issues/121637].)
- Ao criar uma subclasse de um tipo C puro, os slots C do novo tipo não são mais substituídos por uma versão encapsulada na criação da classe, a menos que sejam explicitamente substituídos na subclasse. (Contribuição de Tomasz Pytel em gh-132284 [https://github.com/python/cpython/issues/132284].)

Embutidos

- Os métodos `bytes.fromhex()` e `bytearray.fromhex()` agora aceitam objetos ASCII `bytes` e similares. (Contribuição de Daniel Pope em gh-129349 [https://github.com/python/cpython/issues/129349].)
- Adiciona os métodos de classe `float.from_number()` e `complex.from_number()` para converter um número para o tipo `float` ou `complex`, respectivamente. Eles levantam um `TypeError` se o argumento não for um número real. (Contribuição de Serhiy Storchaka em gh-84978 [https://github.com/python/cpython/issues/84978].)
- Suporte para sublinhado e vírgula como separadores de milhares na parte fracionária, para tipos de apresentação de ponto flutuante, no novo estilo de formatação de string (com `format()` ou Literais de strings formatadas). (Contribuição de Sergey B Kirpichev em gh-87790 [https://github.com/python/cpython/issues/87790].)
- A função `int()` não delega mais para `__trunc__()`. Classes que desejam oferecer suporte à conversão para `int()` devem implementar `__int__()` ou `__index__()`. (Contribuição de Mark Dickinson em gh-119743 [https://github.com/python/cpython/issues/119743].)

- A função `map()` agora possui um sinalizador opcional *strict* somente-nomeado, como `zip()`, para verificar se todos os iteráveis têm o mesmo comprimento. (Contribuição de Wannes Boeykens em gh-119793 [https://github.com/python/cpython/issues/119793].)
- O tipo `memoryview` agora oferece suporte à subscrição, tornando-o um tipo genérico. (Contribuição de Brian Schubert em gh-126012 [https://github.com/python/cpython/issues/126012].)
- Usar `NotImplemented` em um contexto booleano agora levantará `TypeError`. Isso levantava `DeprecationWarning` desde Python 3.9. (Contribuição de Jelle Zijlstra em gh-118767 [https://github.com/python/cpython/issues/118767].)
- `pow()` com três argumentos agora tenta chamar `__rpow__()` se necessário. Anteriormente, ele só era chamado em `pow()` com dois argumentos e no operador binário de potência. (Contribuição de Serhiy Storchaka em gh-130104 [https://github.com/python/cpython/issues/130104].)
- Os objetos `super` agora são `copiáveis` e `serializáveis com pickle`. (Contribuição de Serhiy Storchaka em gh-125767 [https://github.com/python/cpython/issues/125767].)

Linha de comando e ambiente

- O sinalizador de tempo de importação agora pode rastrear módulos já carregados (‘em cache’), por meio da nova opção `-X importtime=2`. Quando um módulo desse tipo é importado, os tempos `self` e `cumulative` são substituídos pela string `cached`.

  Valores acima de `2` para `-X importtime` agora são reservados para uso futuro.

  (Contribuição de Noah Kim e Adam Turner em gh-118655 [https://github.com/python/cpython/issues/118655].)

- A opção de linha de comando `-c` agora recua automaticamente seu argumento de código antes da execução. (Contribuição de Jon Crall e Steven Sun em gh-103998 [https://github.com/python/cpython/issues/103998].)

- **`-J`** não é mais um sinalizador reservado para Jython [https://www.jython.org/] e agora não tem nenhum significado especial. (Contribuição de Adam Turner em gh-133336 [https://github.com/python/cpython/issues/133336].)

PEP 758: permite expressões `except` e `except*` sem parênteses

As expressões **`except`** e **`except*`** agora permitem a omissão de parênteses quando há vários tipos de exceção e a cláusula `as` não é usada. Por exemplo:

```
try:
    connect_to_server()
except TimeoutError, ConnectionRefusedError:
    print('A rede deixou de existir!')
```

(Contribuição de Pablo Galindo e Brett Cannon em **PEP 758** [https://peps.python.org/pep-0758/] e gh-131831 [https://github.com/python/cpython/issues/131831].)

PEP 765: fluxo de controle em blocos `finally`

O compilador agora emite uma **`SyntaxWarning`** quando uma instrução **`return`**, **`break`** ou **`continue`** resulta na saída de um bloco **`finally`**. Essa alteração é especificada na **PEP 765** [https://peps.python.org/pep-0765/].

Em situações em que essa alteração é inconveniente (como quando os avisos são redundantes devido à verificação de código), o filtro de avisos pode ser usado para desativar todos os avisos de sintaxe, adicionando `ignore::SyntaxWarning` como filtro. Isso pode ser especificado em conjunto com um filtro que converte outros avisos em erros (por exemplo, passando `-Werror -Wignore::SyntaxWarning` como opções da CLI ou definindo `PYTHONWARNINGS=error,ignore::SyntaxWarning`).

Observe que a aplicação desse filtro em tempo de execução usando o módulo **`warnings`** só suprimirá o aviso no código compilado *após* o ajuste do filtro. O código compilado antes do ajuste do filtro (por exemplo, quando um módulo é importado) ainda emitirá o aviso de sintaxe.

(Contribuição de Irit Katriel em gh-130080 [https://github.com/python/cpython/issues/130080].)

Coleta de lixo

A partir do Python 3.14.5:

O coletor de lixo (GC) mudou no Python 3.14.5.

As versões 3.14.0 a 3.14.4 do Python incluíam um novo coletor de lixo (GC) incremental. No entanto, devido a diversos relatos [https://github.com/python/cpython/issues/142516] de pressão significativa sobre a memória em ambientes de produção, houve um retorno ao GC geracional da versão 3.13. Esse é o GC utilizado atualmente no Python 3.14.5 e versões posteriores.

Anteriormente nas versões 3.14.0 a 3.14.4 do Python:

O coletor de lixo cíclico agora é incremental. Isso significa que os tempos máximos de pausa são reduzidos em uma ordem de magnitude ou mais para heaps maiores.

Agora existem apenas duas gerações: jovem e velha. Quando **`gc.collect()`** não é chamado diretamente, o coletor de lixo é invocado com menor frequência. Quando invocado, ele coleta a geração jovem e incrementa a geração antiga, em vez de coletar uma ou mais gerações.

O comportamento de **`gc.collect()`** muda ligeiramente:

- `gc.collect(1)`: executa um incremento da coleta de lixo, em vez de coletar a geração 1.
- Outras chamadas para **`gc.collect()`** não foram alteradas.

(Contribuição de Mark Shannon em gh-108362 [https://github.com/python/cpython/issues/108362].)

Console interativo padrão

- O console interativo padrão agora realça a sintaxe Python. O recurso é habilitado por padrão, exceto se **`PYTHON_BASIC_REPL`** ou qualquer outra variável de ambiente que desabilite cores estiver definida. Consulte Controlando cores para obter detalhes.

  O tema de cores padrão para realce de sintaxe busca um bom contraste e usa exclusivamente os códigos de cores ANSI padrão VGA de 4 bits para obter a máxima compatibilidade. O tema pode ser personalizado com o uso de uma API experimental **`_colorize.set_theme()`**. Isso pode ser chamado interativamente ou no script **`PYTHONSTARTUP`**. Observe que esta função não tem garantia de estabilidade e pode ser alterada ou removida.

  (Contribuição de Łukasz Langa em gh-131507 [https://github.com/python/cpython/issues/131507].)

- O console interativo padrão agora oferece suporte ao preenchimento automático de importações. Isso significa que digitar `import co` e pressionar `<Tab>` vai sugerir módulos que começam com `co`. Da mesma forma, digitar `from concurrent import i` vai sugerir submódulos de `concurrent` que começam com `i`. Observe que o preenchimento automático de atributos de módulo não é suportado atualmente. (Contribuição de Tomas Roun em gh-69605 [https://github.com/python/cpython/issues/69605].)

Novos módulos

- **`annotationlib`**: para introspecção de anotações. Veja PEP 749 para mais detalhes. (Contribuição de Jelle Zijlstra em gh-119180 [https://github.com/python/cpython/issues/119180].)
- **`compression`** (incluindo **`compression.zstd`**): um pacote para módulos relacionados à compactação, incluindo um novo módulo com suporte ao formato de compactação Zstandard. Consulte PEP 784 para mais detalhes. (Contribuição de Emma Harper Smith, Adam Turner, Gregory P. Smith, Tomas Roun, Victor Stinner e Rogdham em gh-132983 [https://github.com/python/cpython/issues/132983].)

- `concurrent.interpreters`: suporte a múltiplos interpretadores na biblioteca padrão. Consulte PEP 734 para mais detalhes. (Contribuição de Eric Snow em gh-134939 [https://github.com/python/cpython/issues/134939].)
- `string.templatelib`: suporte para literais de strings templates (t-strings). Veja PEP 750 para mais detalhes. (Contribuição de Jim Baker, Guido van Rossum, Paul Everitt, Koudai Aono, Lysandros Nikolaou, Dave Peck, Adam Turner, Jelle Zijlstra, Bénédikt Tran e Pablo Galindo Salgado em gh-132661 [https://github.com/python/cpython/issues/132661].)

Módulos melhorados

argparse

- O valor padrão do nome do programa para `argparse.ArgumentParser` agora reflete a maneira como o interpretador Python foi instruído a encontrar o código do módulo `__main__`. (Contribuição de Serhiy Storchaka e Alyssa Coghlan em gh-66436 [https://github.com/python/cpython/issues/66436].)
- Foi introduzido o parâmetro opcional *suggest_on_error* em `argparse.ArgumentParser`, permitindo sugestões para argumentos e nomes de subanalisadores quando digitados incorretamente pelo usuário. (Contribuição de Savannah Ostrowski em gh-124456 [https://github.com/python/cpython/issues/124456].)
- Habilita a cor do texto de ajuda, que pode ser desabilitada com o parâmetro opcional *color* em `argparse.ArgumentParser`. Isso também pode ser controlado por variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-130645 [https://github.com/python/cpython/issues/130645].)

ast

- Adiciona `compare()`, uma função para comparar dois ASTs. (Contribuição de Batuhan Taskaya e Jeremy Hylton em gh-60191 [https://github.com/python/cpython/issues/60191].)
- Adiciona suporte para `copy.replace()` em nós AST. (Contribuição de Bénédikt Tran em gh-121141 [https://github.com/python/cpython/issues/121141].)

- Docstrings agora são removidas de um AST otimizado no nível de otimização 2. (Contribuição de Irit Katriel em gh-123958 [https://github.com/python/cpython/issues/123958].)
- A saída `repr()` para nós AST agora inclui mais informações. (Contribuição de Tomas Roun em gh-116022 [https://github.com/python/cpython/issues/116022].)
- Quando chamada com um AST como entrada, a função `parse()` agora verifica sempre se o tipo de nó raiz é apropriado. (Contribuição de Irit Katriel em gh-130139 [https://github.com/python/cpython/issues/130139].)
- Adiciona novas opções à interface de linha de comando: `--feature-version`, `--optimize` e `--show-empty`. (Contribuição de Semyon Moroz em gh-133367 [https://github.com/python/cpython/issues/133367].)

asyncio

- A função e os métodos denominados `create_task()` agora aceitam uma lista arbitrária de argumentos nomeados. Todos os argumentos nomeados são passados para o construtor `Task` ou para a fábrica de tarefas personalizada. (Consulte `set_task_factory()` para obter detalhes.) Os argumentos nomeados `name` e `context` não são mais especiais; o nome agora deve ser definido pelo argumento nomeado `name` da fábrica, e `context` pode ser `None`.

  Isso afeta as seguintes funções e métodos: `asyncio.create_task()`, `asyncio.loop.create_task()`, `asyncio.TaskGroup.create_task()`.

  (Contribuição de Thomas Grainger em gh-128307 [https://github.com/python/cpython/issues/128307].)

- Há duas novas funções utilitárias para introspecção e exibição do gráfico de chamadas de um programa: `capture_call_graph()` e `print_call_graph()`. Veja Capacidades de introspecção do asyncio para mais detalhes. (Contribuição de Yury Selivanov, Pablo Galindo Salgado e Łukasz Langa em gh-91048 [https://github.com/python/cpython/issues/91048].)

calendar

- Por padrão, o dia de hoje é destacado em cor na saída de texto da linha de comando do módulo `calendar`. Isso pode ser controlado pelas variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-128317 [https://github.com/python/cpython/issues/128317].)

concurrent.futures

- Adiciona uma nova classe executora, `InterpreterPoolExecutor`, que expõe vários interpretadores Python no mesmo processo ('subinterpretadores') ao código Python. Isso usa um conjunto de interpretadores Python independentes para executar chamadas de forma assíncrona.

  Isto é separado do novo módulo `interpreters` introduzido pela PEP 734. (Contribuição de Eric Snow em gh-124548 [https://github.com/python/cpython/issues/124548].)

- Em plataformas Unix diferentes do macOS, 'forkserver' agora é o método de inicialização padrão para `ProcessPoolExecutor` (substituindo 'fork'). Essa alteração não afeta o Windows ou o macOS, onde 'spawn' continua sendo o método de inicialização padrão.

  Se o método *fork* incompatível com threading for necessário, você deverá solicitá-lo explicitamente fornecendo um contexto de multiprocessamento *mp_context* para `ProcessPoolExecutor`.

  Veja as restrições de forkserver para obter informações e diferenças com o método *fork* e como essa alteração pode afetar o código existente com variáveis globais compartilhadas mutáveis e/ou objetos compartilhados que não podem ser automaticamente `serializados com pickle`.

  (Contribuição de Gregory P. Smith em gh-84559 [https://github.com/python/cpython/issues/84559].)

- Adiciona dois novos métodos a `ProcessPoolExecutor`, `terminate_workers()` e `kill_workers()`, como formas de encerrar ou eliminar todos os processos de trabalho ativos no pool fornecido.

(Contribuição de Charles Machalow em gh-130849 [https://github.com/python/cpython/issues/130849].)

- Adiciona o parâmetro opcional *buffersize* a `Executor.map` para limitar o número de tarefas enviadas cujos resultados ainda não foram gerados. Se o buffer estiver cheio, a iteração sobre os *iteráveis* será pausada até que um resultado seja produzido pelo buffer. (Contribuição de Enzo Bonnal e Josh Rosenberg em gh-74028 [https://github.com/python/cpython/issues/74028].)

configparser

- `configparser` não gravará mais arquivos de configuração que não consegue ler, para melhorar a segurança. Tentar usar `write()` em chaves contendo delimitadores ou começando com o padrão de cabeçalho de seção levantará um `InvalidWriteError`. (Contribuição de Jacob Lincoln em gh-129270 [https://github.com/python/cpython/issues/129270].)

contextvars

- Adiciona suporte ao protocolo do gerenciador de contexto para objetos `Token`. (Contribuição de Andrew Svetlov em gh-129889 [https://github.com/python/cpython/issues/129889].)

ctypes

- O layout dos campos de bit em objetos `Structure` e `Union` é agora uma correspondência mais fiel aos padrões da plataforma (GCC/Clang ou MSVC). Em particular, os campos não se sobrepõem mais. (Contribuição de Matthias Görgens em gh-97702 [https://github.com/python/cpython/issues/97702].)
- O atributo de classe `Structure._layout_` agora pode ser definido para ajudar a corresponder a uma ABI não padrão. (Contribuição de Petr Viktorin em gh-97702 [https://github.com/python/cpython/issues/97702].)
- A classe de descritores de campo `Structure`/`Union` agora está disponível como `CField` e possui novos atributos para auxiliar na

depuração e na introspecção. (Contribuição de Petr Viktorin em gh-128715 [https://github.com/python/cpython/issues/128715].)

- No Windows, a exceção `COMError` agora é pública. (Contribuição de Jun Komoda em gh-126686 [https://github.com/python/cpython/issues/126686].)
- No Windows, a função `CopyComPointer()` agora é pública. (Contribuição de Jun Komoda em gh-127275 [https://github.com/python/cpython/issues/127275].)
- Adiciona `memoryview_at()`, uma função para criar um objeto `memoryview` que faz referência ao ponteiro e comprimento fornecidos. Isso funciona como `ctypes.string_at()`, exceto que evita uma cópia de buffer e é normalmente útil ao implementar funções de retorno de chamada Python puras que recebem buffers de tamanho dinâmico. (Contribuição de Rian Hunter em gh-112018 [https://github.com/python/cpython/issues/112018].)
- Tipos complexos, `c_float_complex`, `c_double_complex` e `c_longdouble_complex`, agora estão disponíveis se tanto o compilador quanto a biblioteca `libffi` oferecerem suporte a tipos C complexos. (Contribuição de Sergey B Kirpichev em gh-61103 [https://github.com/python/cpython/issues/61103].)
- Adiciona `ctypes.util.dllist()` para listar as bibliotecas compartilhadas carregadas pelo processo atual. (Contribuição de Brian Ward em gh-119349 [https://github.com/python/cpython/issues/119349].)
- Move o cache de tipos `ctypes.POINTER()` de um cache interno global (`_pointer_type_cache`) para o atributo `__CData__pointer_type__` dos tipos **`ctypes`** correspondentes. Isso impedirá que o cache cresça sem limites em algumas situações. (Contribuição de Sergey Miryanov em gh-100926 [https://github.com/python/cpython/issues/100926].)
- O tipo `py_object` agora oferece suporte à subscrição, tornando-o um tipo genérico. (Contribuição de Brian Schubert em gh-132168 [https://github.com/python/cpython/issues/132168].)
- **`ctypes`** agora oferece suporte a construções com threads livres. (Contribuição de Kumar Aditya e Peter Bierma em gh-127945 [https://github.com/python/cpython/issues/127945].)

curses

- Adiciona a função `assume_default_colors()`, um refinamento da função `use_default_colors()` que permite alterar o par de cores `0`. (Contribuição de Serhiy Storchaka em gh-133139 [https://github.com/python/cpython/issues/133139].)

datetime

- Adiciona o método `strptime()` às classes `datetime.date` e `datetime.time`. (Contribuição de Wannes Boeykens em gh-41431 [https://github.com/python/cpython/issues/41431].)

decimal

- Adiciona `Decimal.from_number()` como um construtor alternativo para `Decimal`. (Contribuição de Serhiy Storchaka em gh-121798 [https://github.com/python/cpython/issues/121798].)
- Expõe `IEEEContext()` para oferecer suporte à criação de contextos correspondentes aos formatos de intercâmbio decimal do IEEE 754 (2008). (Contribuição de Sergey B Kirpichev em gh-53032 [https://github.com/python/cpython/issues/53032].)

difflib

- Páginas de comparação com alterações destacadas geradas pela classe `HtmlDiff` agora oferecem suporte ao “modo escuro”. (Contribuição de Jiahao Li em gh-129939 [https://github.com/python/cpython/issues/129939].)

dis

- Adiciona suporte para renderizar informações completas sobre o local da fonte de `instruções`, em vez de apenas o número da linha. Este recurso é adicionado às seguintes interfaces por meio do argumento nomeado *show_positions*:
    - `dis.Bytecode`
    - `dis.dis()`
    - `dis.distb()`

- `dis.disassemble()`

  Esse recurso também é exposto via `dis --show-positions`. (Contribuição de Bénédikt Tran em gh-123165 [https://github.com/python/cpython/issues/123165].)

- Adiciona a opção de linha de comando `dis --specialized` para mostrar o bytecode especializado. (Contribuição de Bénédikt Tran em gh-127413 [https://github.com/python/cpython/issues/127413].)

errno

- Adiciona a constante de código de erro `EHWPOISON`. (Contribuição de James Roy em gh-126585 [https://github.com/python/cpython/issues/126585].)

faulthandler

- Adiciona suporte para exibir o stack trace (situação da pilha de execução) C em sistemas que oferecem suporte a isso via a função `dump_c_stack()` ou via o argumento *c_stack* em `faulthandler.enable()`. (Contribuição de Peter Bierma em gh-127604 [https://github.com/python/cpython/issues/127604].)

fnmatch

- Adiciona `filterfalse()`, uma função para rejeitar nomes que correspondem a um determinado padrão. (Contribuição de Bénédikt Tran em gh-74598 [https://github.com/python/cpython/issues/74598].)

fractions

- Um objeto `Fraction` agora pode ser construído a partir de qualquer objeto com o método `as_integer_ratio()`. (Contribuição de Serhiy Storchaka em gh-82017 [https://github.com/python/cpython/issues/82017].)
- Adiciona `Fraction.from_number()` como um construtor alternativo para `Fraction`. (Contribuição de Serhiy Storchaka em gh-121797 [https://github.com/python/cpython/issues/121797].)

functools

- Adiciona o sinalizador `Placeholder`. Ele pode ser usado com as funções `partial()` ou `partialmethod()` para reservar um lugar para argumentos posicionais no objeto parcial retornado. (Contribuição de Dominykas Grigonis em gh-119127 [https://github.com/python/cpython/issues/119127].)
- Permite que o parâmetro *initial* de `reduce()` seja passado como um argumento nomeado. (Contribuição de Sayandip Dutta em gh-125916 [https://github.com/python/cpython/issues/125916].)

getopt

- Adiciona suporte para opções com argumento opcional. (Contribuição de Serhiy Storchaka em gh-126374 [https://github.com/python/cpython/issues/126374].)
- Adiciona suporte para retornar opções misturadas e argumentos não-opcionais em ordem. (Contribuição de Serhiy Storchaka em gh-126390 [https://github.com/python/cpython/issues/126390].)

getpass

- Suporte a feedback do teclado na função `getpass()` por meio do argumento opcional somente-nomeado *echo_var*. Espaços reservados para caracteres são renderizados sempre que um caractere é inserido e removidos quando um caractere é excluído. (Contribuição de Semyon Moroz em gh-77065 [https://github.com/python/cpython/issues/77065].)

graphlib

- Permite que `TopologicalSorter.prepare()` seja chamado mais de uma vez, desde que a classificação não tenha sido iniciada. (Contribuição de por Daniel Pope em gh-130914 [https://github.com/python/cpython/issues/130914].)

heapq

- O módulo **`heapq`** melhorou o suporte para trabalhar com max-heaps, por meio das seguintes novas funções:
  - `heapify_max()`
  - `heappush_max()`
  - `heappop_max()`
  - `heapreplace_max()`
  - `heappushpop_max()`

hmac

- Adiciona uma implementação embutido para HMAC (**RFC 2104** [https://datatracker.ietf.org/doc/html/rfc2104.html]) usando código formalmente verificado do projeto HACL* [https://github.com/hacl-star/hacl-star/]. Esta implementação é usada como fallback quando a implementação OpenSSL do HMAC não está disponível. (Contribuição de Bénédikt Tran em gh-99108 [https://github.com/python/cpython/issues/99108].)

http

- As páginas de erro e listas de diretórios geradas pelo módulo `http.server` permitem que o navegador aplique seu modo escuro padrão. (Contribuição de Yorik Hansen em gh-123430 [https://github.com/python/cpython/issues/123430].)

- O módulo `http.server` agora provê suporte para HTTPS usando a classe `http.server.HTTPSServer`. Essa funcionalidade é exposta pela interface de linha de comando (`python -m http.server`) por meio das seguintes opções:

  - `--tls-cert <caminho>`: Caminho para o arquivo de certificado TLS.
  - `--tls-key <caminho>`: Caminho opcional para o arquivo chave privado.
  - `--tls-password-file <caminho>`: Caminho opcional para o arquivo de senha da chave privada.

(Contribuição de Semyon Moroz em gh-85162 [https://github.com/python/cpython/issues/85162].)

imaplib

- Adiciona `IMAP4.idle()`, que implementa o comando `IDLE` do IMAP4 conforme definido em **RFC 2177** [https://datatracker.ietf.org/doc/html/rfc2177.html]. (Contribuição de Forest em gh-55454 [https://github.com/python/cpython/issues/55454].)

inspect

- `signature()` recebe um novo argumento *annotation_format* para controlar o `annotationlib.Format` usado para representar anotações. (Contribuição de Jelle Zijlstra em gh-101552 [https://github.com/python/cpython/issues/101552].)
- `Signature.format()` recebe um novo argumento *unquote_annotations*. Se verdadeiro, string com anotações são exibidas sem as aspas ao redor. (Contribuição de Jelle Zijlstra em gh-101552 [https://github.com/python/cpython/issues/101552].)
- Adiciona a função `ispackage()` para determinar se um objeto é um pacote ou não. (Contribuição de Zhikang Yan em gh-125634 [https://github.com/python/cpython/issues/125634].)

io

- A leitura de texto de um fluxo sem bloqueio com `read` pode agora levantar um `BlockingIOError` se a operação não puder imediatamente retornar bytes. (Contribuição de Giovanni Siragusa em gh-109523 [https://github.com/python/cpython/issues/109523].)
- Adiciona os protocolos `Reader` e `Writer` como alternativas mais simples aos pseudoprotocolos `typing.IO`, `typing.TextIO` e `typing.BinaryIO`. (Contribuição de Sebastian Rittau em gh-127648 [https://github.com/python/cpython/issues/127648].)

json

- Adiciona notas de exceção para erros de serialização JSON que permitam identificar a origem do erro. (Contribuição de Serhiy Storchaka em gh-122163 [https://github.com/python/cpython/issues/122163].)
- Permite usar o módulo `json` como script usando a opção `-m`: **python -m json**. Isso é agora preferível a **python -m json.tool**, o que está suavemente descontinuado. Consulte a documentação da interface de linha de comando JSON. (Contribuição de Trey Hunner em gh-122873 [https://github.com/python/cpython/issues/122873].)
- Por padrão, a saída da interface de linha de comando JSON é destacada em cores. Isso pode ser controlado por variáveis de ambiente. (Contribuição de Tomas Roun em gh-131952 [https://github.com/python/cpython/issues/131952].)

linecache

- `getline()` pode agora recuperar código-fonte de módulos congelados. (Contribuição de Tian Gao em gh-131638 [https://github.com/python/cpython/issues/131638].)

logging.handlers

- Objetos `QueueListener` agora oferecem suporte ao protocolo de gerenciador de contexto. (Contribuição de Charles Machalow em gh-132106 [https://github.com/python/cpython/issues/132106].)
- `QueueListener.start` agora levanta `RuntimeError` se o ouvinte já tiver sido iniciado. (Contribuição de Charles Machalow em gh-132106 [https://github.com/python/cpython/issues/132106].)

math

- Adicionadas mensagens de erro mais detalhadas para erros de domínio no módulo. (Contribuição de Charlie Zhao e Sergey B Kirpichev em gh-101410 [https://github.com/python/cpython/issues/101410].)

mimetypes

- Adiciona uma linha de comando pública para o módulo, invocada via **python -m mimetypes**. (Contribuição de Oleg Iarygin e Hugo van Kemenade em gh-93096 [https://github.com/python/cpython/issues/93096].)

- Adiciona vários novos tipos MIME com base em RFCs e usos comuns:

  Tipos MIME da Microsoft e do RFC 8081 [https://datatracker.ietf.org/doc/html/rfc8081.html] para fontes

  - Embedded OpenType: `application/vnd.ms-fontobject`
  - OpenType Layout (OTF) `font/otf`
  - TrueType: `font/ttf`
  - WOFF 1.0 `font/woff`
  - WOFF 2.0 `font/woff2`

  Tipos MIME de RFC 9559 [https://datatracker.ietf.org/doc/html/rfc9559.html] para estruturas de contêiner de dados audiovisuais Matroska

  - áudio com nenhum vídeo: `audio/matroska` (`.mka`)
  - vídeo: `video/matroska` (`.mkv`)
  - vídeo estereoscópico: `video/matroska-3d` (`.mk3d`)

  Imagens com RFCs

  - **RFC 1494** [https://datatracker.ietf.org/doc/html/rfc1494.html]: CCITT Group 3 (`.g3`)
  - **RFC 3362** [https://datatracker.ietf.org/doc/html/rfc3362.html]: Real-time Facsimile, T.38 (`.t38`)
  - **RFC 3745** [https://datatracker.ietf.org/doc/html/rfc3745.html]: JPEG 2000 (`.jp2`), extensão (`.jpx`) e composto (`.jpm`)
  - **RFC 3950** [https://datatracker.ietf.org/doc/html/rfc3950.html]: Tag Image File Format Fax eXtended, TIFF-FX (`.tfx`)
  - **RFC 4047** [https://datatracker.ietf.org/doc/html/rfc4047.html]: Flexible Image Transport System (`.fits`)
  - **RFC 7903** [https://datatracker.ietf.org/doc/html/rfc7903.html]: Enhanced Metafile (`.emf`) and Windows Metafile (`.wmf`)

Outras adições e alterações de tipos MIME

- **RFC 2361** [https://datatracker.ietf.org/doc/html/rfc2361.html]: altera o tipo de `.avi` para `video/vnd.avi` e de `.wav` para `audio/vnd.wave`
- **RFC 4337** [https://datatracker.ietf.org/doc/html/rfc4337.html]: adiciona MPEG-4 `audio/mp4` (`.m4a`)
- **RFC 5334** [https://datatracker.ietf.org/doc/html/rfc5334.html]: adiciona mídia Ogg (`.oga`, `.ogg` e `.ogx`)
- **RFC 6713** [https://datatracker.ietf.org/doc/html/rfc6713.html]: adiciona gzip `aplication/gzip` (`.gz`)
- **RFC 9639** [https://datatracker.ietf.org/doc/html/rfc9639.html]: adiciona FLAC `audio/flac` (`.flac`)
- O tipo MIME `application/yaml` para arquivos YAML (`.yaml` e `.yml`) do **RFC 9512** [https://datatracker.ietf.org/doc/html/rfc9512.html]
- Adiciona 7z `aplication/x-7z-compressed` (`.7z`)
- Adiciona Android Package `application/vnd.android.package-archive` (`.apk`) quando não for rigoroso
- Adiciona deb `application/x-debian-package` (`.deb`)
- Adiciona glTF binário `model/gltf-binary` (`.glb`)
- Adiciona glTF JSON/ASCII `model/gltf+json` (`.gltf`)
- Adiciona M4V `video/x-m4v` (`.m4v`)
- Adiciona PHP `application/x-httpd-php` (`.php`)
- Adiciona RAR `application/vnd.rar` (`.rar`)
- Adiciona RPM `application/x-rpm` (`.rpm`)
- Adiciona STL `model/stl` (`.stl`)
- Adiciona Windows Media Video `video/x-ms-wmv` (`.wmv`)
- De facto: adiciona WebM `audio/webm` (`.weba`)
- ECMA-376 [https://ecma-international.org/publications-and-standards/standards/ecma-376/]: adiciona tipos `.docx`, `.pptx` e `.xlsx`
- OASIS [https://docs.oasis-open.org/office/v1.2/cs01/OpenDocument-v1.2-cs01-part1.html#Appendix_C]: adiciona tipos OpenDocument `.odg`, `.odp`, `.ods` e `.odt`
- W3C [https://www.w3.org/TR/epub-33/#app-media-type]: adiciona EPUB `application/epub+zip` (`.epub`)

(Contribuição de Sahil Prajapati e Hugo van Kemenade em gh-84852 [https://github.com/python/cpython/issues/84852], de Sasha “Nelie” Chernykh e

Hugo van Kemenade em gh-132056 [https://github.com/python/cpython/issues/132056], e de Hugo van Kemenade em gh-89416 [https://github.com/python/cpython/issues/89416], gh-85957 [https://github.com/python/cpython/issues/85957] e gh-129965 [https://github.com/python/cpython/issues/129965].)

multiprocessing

- Em plataformas Unix diferentes do macOS, 'forkserver' agora é o método de inicialização padrão (substituindo 'fork'). Essa alteração não afeta o Windows ou o macOS, onde 'spawn' continua sendo o método de inicialização padrão.

  Se o método *fork* incompatível com threads for necessário, você deverá solicitá-lo explicitamente com um contexto de **`get_context()`** (preferencial) ou mudar o método padrão com **`set_start_method()`**.

  Veja as restrições de forkserver para obter informações e diferenças com o método *fork* e como essa alteração pode afetar o código existente com variáveis globais compartilhadas mutáveis e/ou objetos compartilhados que não podem ser automaticamente **`serializados com pickle`**.

  (Contribuição de Gregory P. Smith em gh-84559 [https://github.com/python/cpython/issues/84559].)

- O método de inicialização `'forkserver'` do **`multiprocessing`** agora autentica seu soquete de controle para evitar depender exclusivamente das permissões do sistema de arquivos para restringir quais outros processos poderiam fazer com que o forkserver gerasse workers e executasse código. (Contribuição de Gregory P. Smith em gh-97514 [https://github.com/python/cpython/issues/97514].)

- Os objetos proxies de multiprocessing para os tipos *list* e *dict* ganhou métodos anteriormente ausentes:

  - **`clear()`** e **`copy()`** para proxies de **`list`**

    - `fromkeys()`, `reversed(d)`, `d | {}`, `{} | d`, `d |= {'b': 2}` para proxies de `dict`

  (Contribuição de Roy Hyunjin Han em gh-103134 [https://github.com/python/cpython/issues/103134].)

- Adiciona suporte para objetos `set` compartilhado via `SyncManager.set()`. O método `set()` em `Manager()` está agora disponível. (Contribuição de Mingyu Park em gh-129949 [https://github.com/python/cpython/issues/129949].)

- Adiciona `interrupt()` para objetos `multiprocessing.Process`, que encerra o processo filho enviando `SIGINT`. Isso permite a cláusula `finally` exibir uma stack trace (situação da pilha de execução) para o processo encerrado. (Contribuição de Artem Pulkin em gh-131913 [https://github.com/python/cpython/issues/131913].)

operador

- Adiciona `is_none()` e `is_not_none()` como um par de funções, de modo que `operator.is_none(obj)` equivale a `obj is None` e `operator.is_not_none(obj)` equivale a `obj is not None`. (Contribuição de Raymond Hettinger e Nico Mexis em gh-115808 [https://github.com/python/cpython/issues/115808].)

os

- Adiciona a função `reload_environ()` para atualizar `os.environ` e `os.environb` com alterações no ambiente feitas por `os.putenv()` e `os.unsetenv()`, ou feitas fora do Python no mesmo processo. (Contribuição de Victor Stinner em gh-120057 [https://github.com/python/cpython/issues/120057].)
- Adiciona as constantes `SCHED_DEADLINE` e `SCHED_NORMAL` ao módulo `os`. (Contribuição de James Roy em gh-127688 [https://github.com/python/cpython/issues/127688].)
- Adiciona a função `readinto()` para ler em um objeto buffer de um descritor de arquivo. (Contribuição de Cody Maloney em gh-129205

[https://github.com/python/cpython/issues/129205].)

os.path

- O parâmetro *strict* para `realpath()` aceita um novo valor, `ALLOW_MISSING`. Se usado, erros diferentes de `FileNotFoundError` serão levantados novamente; o caminho resultante pode estar ausente, mas estará livre de links simbólicos. (Contribuição de Petr Viktorin para **CVE 2025-4517** [https://www.cve.org/CVERecord?id=CVE-2025-4517].)

pathlib

- Adiciona métodos a `pathlib.Path` para copiar ou mover arquivos e diretórios recursivamente:

  - `copy()` copia um arquivo ou árvore de diretório para um destino.
  - `copy_into()` copia *para* um diretório de destino.
  - `move()` move um arquivo ou árvore de diretórios para um destino.
  - `move_into()` move *para* um diretório de destino.

  (Contribuição de Barney Gale em gh-73991 [https://github.com/python/cpython/issues/73991].)

- Adiciona o atributo `info`, que armazena um objeto que implementa o novo protocolo `pathlib.types.PathInfo`. O objeto oferece suporte à consulta do tipo de arquivo e armazenamento em cache interno dos resultados de `stat()`. Objetos de caminho gerados por `iterdir()` são inicializados com informações do tipo de arquivo obtidas a partir da varredura do diretório pai. (Contribuição de Barney Gale em gh-125413 [https://github.com/python/cpython/issues/125413].)

pdb

- O módulo `pdb` agora oferece suporte a anexação remota a um processo Python em execução usando uma nova opção de linha de comando `-p PID`:

```
python -m pdb -p 1234
```

Isso se conectará ao processo Python com o PID fornecido e permitirá que você o depure interativamente. Observe que, devido ao funcionamento do interpretador Python, a conexão a um processo remoto bloqueado em uma chamada de sistema ou aguardando E/S só funcionará quando a próxima instrução de bytecode for executada ou quando o processo receber um sinal.

Este recurso usa PEP 768 e a nova função `sys.remote_exec()` para anexar ao processo remoto e enviar os comandos PDB a ele.

(Contribuição de Matt Wozniski e Pablo Galindo em gh-131591 [https://github.com/python/cpython/issues/131591].)

- Pontos de interrupção codificados diretamente no código-fonte (`breakpoint()` e `set_trace()`) agora reutilizam a instância mais recente de `Pdb` que chama `set_trace()`, em vez de criar uma nova a cada vez. Como resultado, todos os dados específicos da instância, como `display` e `commands`, são preservados em todos os pontos de interrupção. (Contribuição de Tian Gao em gh-121450 [https://github.com/python/cpython/issues/121450].)

- Adiciona um novo argumento *mode* a `pdb.Pdb`. Desabilita o comando `restart` quando `pdb` estiver no modo `inline`. (Contribuição de Tian Gao em gh-123757 [https://github.com/python/cpython/issues/123757].)

- Um prompt de confirmação será exibido quando o usuário tentar sair do comando `pdb` no modo `inline`. `y`, `Y`, `<Enter>` ou `EOF` confirmarão o encerramento e chamarão `sys.exit()`, em vez de executar `bdb.BdbQuit`. (Contribuição de Tian Gao em gh-124704 [https://github.com/python/cpython/issues/124704].)

- Pontos de interrupção em linha como `breakpoint()` ou `pdb.set_trace()` sempre interromperão o programa no quadro da chamada, ignorando o padrão `skip` (se houver). (Contribuição de Tian Gao em gh-130493 [https://github.com/python/cpython/issues/130493].)

- `<tab>` no início da linha na entrada multilinha do **`pdb`** agora preencherá um recuo de 4 espaços, em vez de inserir um caractere `\t`. (Contribuição de Tian Gao em gh-130471 [https://github.com/python/cpython/issues/130471].)

- A indentação automática foi introduzida na entrada multilinha do **`pdb`**. Ela mantém a indentação da última linha ou insere uma indentação de 4 espaços ao detectar um novo bloco de código. (Contribuição de Tian Gao em gh-133350 [https://github.com/python/cpython/issues/133350].)

- `$_asynctask` foi adicionado para acessar a tarefa asyncio atual, se aplicável. (Contribuição de Tian Gao em gh-124367 [https://github.com/python/cpython/issues/124367].)

- **`pdb.set_trace_async()`** foi adicionado para oferecer suporte à depuração de corrotinas assíncronas. Instruções **`await`** são suportadas com esta função. (Contribuição de Tian Gao em gh-132576 [https://github.com/python/cpython/issues/132576].)

- O código-fonte exibido em **`pdb`** terá a sintaxe realçada. Este recurso pode ser controlado usando os mesmos métodos do console interativo padrão, além do argumento `colorize` recém-adicionado de **`pdb.Pdb`**. (Contribuição de Tian Gao e Łukasz Langa em gh-133355 [https://github.com/python/cpython/issues/133355].)

pickle

- Define a versão do protocolo padrão no módulo **`pickle`** como 5. Para mais detalhes, consulte protocolos pickle.
- Adiciona notas de exceção para erros de serialização com pickle que permitam identificar a origem do erro. (Contribuição de Serhiy Storchaka em gh-122213 [https://github.com/python/cpython/issues/122213].)

platform

- Adiciona **`invalidate_caches()`**, uma função que invalida resultados em cache no módulo **`platform`**. (Contribuição de Bénédikt Tran em

gh-122549 [https://github.com/python/cpython/issues/122549].)

pydoc

- Anotações na saída de ajuda agora são exibidas em um formato mais próximo ao da fonte original. (Contribuição de Jelle Zijlstra em gh-101552 [https://github.com/python/cpython/issues/101552].)

re

- Suporte a `\z` como sinônimo de `\Z` em **`expressões regulares`**. É interpretado sem ambiguidade em muitos outros mecanismos de expressão regular, ao contrário de `\Z`, que tem um comportamento ligeiramente diferente. (Contribuição de Serhiy Storchaka em gh-133306 [https://github.com/python/cpython/issues/133306].)
- `\B` em **`expressão regular`** agora corresponde a uma string de entrada vazia que é agora sempre o oposto de `\b`. (Contribuição de Serhiy Storchaka em gh-124130 [https://github.com/python/cpython/issues/124130].)

socket

- Melhora e corrige o suporte para soquetes Bluetooth.
  - Corrige o suporte a soquetes Bluetooth no NetBSD e no DragonFly BSD. (Contribuição de Serhiy Storchaka em gh-132429 [https://github.com/python/cpython/issues/132429].)
  - Corrige o suporte para **`BTPROTO_HCI`** no FreeBSD. (Contribuição de Victor Stinner em gh-111178 [https://github.com/python/cpython/issues/111178].)
  - Adiciona suporte para **`BTPROTO_SCO`** no FreeBSD. (Contribuição de Serhiy Storchaka em gh-85302 [https://github.com/python/cpython/issues/85302].)
  - Adiciona suporte para *cid* e *bdaddr_type* no endereço para **`BTPROTO_L2CAP`** no FreeBSD. (Contribuição de Serhiy Storchaka em gh-132429 [https://github.com/python/cpython/issues/132429].)
  - Adiciona suporte para *channel* no endereço para **`BTPROTO_HCI`** no Linux. (Contribuição de Serhiy Storchaka em gh-70145 [https://github.com/python/cpython/issues/70145].)

- Aceita um inteiro como endereço para `BTPROTO_HCI` no Linux. (Contribuição de Serhiy Storchaka em gh-132099 [https://github.com/python/cpython/issues/132099].)
- Retorna *cid* em `getsockname()` para `BTPROTO_L2CAP`. (Contribuição de Serhiy Storchaka em gh-132429 [https://github.com/python/cpython/issues/132429].)
- Adiciona muitas novas constantes. (Contribuição de Serhiy Storchaka em gh-132734 [https://github.com/python/cpython/issues/132734].)

ssl

- Indica através do booleano `HAS_PHA` se o módulo `ssl` oferece suporte a autenticação de cliente pós-handshake (PHA) TLSv1.3. (Contribuição de Will Childs-Klein em gh-128036 [https://github.com/python/cpython/issues/128036].)

struct

- Suporte aos tipos C float complex e double complex no módulo `struct` (formatando os caracteres `'F'` e `'D'`, respectivamente). (Contribuição de Sergey B Kirpichev em gh-121249 [https://github.com/python/cpython/issues/121249].)

symtable

- Expõe os seguintes métodos `Symbol`:

  - `is_comp_cell()`
  - `is_comp_iter()`
  - `is_free_class()`

  (Contribuição de Bénédikt Tran em gh-120029 [https://github.com/python/cpython/issues/120029].)

sys

- A função especial `sys.getobjects()`, anteriormente não documentada, que só existe em construções especializadas do Python, agora pode retornar objetos de outros interpretadores além daquele em que foi chamada. (Contribuição de Eric Snow em gh-125286 [https://github.com/python/cpython/issues/125286].)
- Adiciona `sys._is_immortal()` para determinar se um objeto é imortal. (Contribuição de Peter Bierma em gh-128509 [https://github.com/python/cpython/issues/128509].)
- No FreeBSD, `sys.platform` não contém mais o número da versão principal. Ele é sempre `'freebsd'`, em vez de `'freebsd13'` ou `'freebsd14'`. (Contribuição de Michael Osipov em gh-129393 [https://github.com/python/cpython/issues/129393].)
- Levanta `DeprecationWarning` para `sys._clear_type_cache()`. Esta função foi descontinuada no Python 3.13, mas não levantava um aviso em tempo de execução.
- Adiciona `sys.remote_exec()` para implementar a nova interface com depurador externo. Consulte a PEP 768 para obter detalhes. (Contribuição de Pablo Galindo Salgado, Matt Wozniski e Ivona Stojanovic em gh-131591 [https://github.com/python/cpython/issues/131591].)
- Adiciona o espaço de nomes de `sys._jit`, que contém utilitários para introspecção da compilação just-in-time. (Contribuição de Brandt Bucher em gh-133231 [https://github.com/python/cpython/issues/133231].)

sys.monitoring

- Adiciona dois novos eventos de monitoramento: `BRANCH_LEFT` e `BRANCH_RIGHT`. Estes substituem e descontinuam o evento `BRANCH`. (Contribuição de Mark Shannon em gh-122548 [https://github.com/python/cpython/issues/122548].)
- Adiciona `sys.monitoring.clear_tool_id()` para cancelar o registro de todos os eventos e funções de retorno de chamada associados a um identificador de ferramenta. A função `sys.monitoring.free_tool_id()` agora a chama antes de liberar o identificador da ferramenta, em vez de manter seus eventos e retornos de chamada registrados. (Contribuição de Tian Gao em gh-116750 [https://github.com/python/cpython/issues/116750].)

sysconfig

- Adiciona a chave `ABIFLAGS` a `get_config_vars()` no Windows. (Contribuição de Xuehai Pan em gh-131799 [https://github.com/python/cpython/issues/131799].)

tarfile

- `data_filter()` agora normaliza alvos de links simbólicos para evitar ataques de travessia de caminho. (Contribuição de Petr Viktorin em gh-127987 [https://github.com/python/cpython/issues/127987] e **CVE 2025-4138** [https://www.cve.org/CVERecord?id=CVE-2025-4138].)
- `extractall()` agora ignora a correção de atributos de diretório quando um diretório é removido ou substituído por outro tipo de arquivo. (Contribuição de Petr Viktorin em gh-127987 [https://github.com/python/cpython/issues/127987] e **CVE 2024-12718** [https://www.cve.org/CVERecord?id=CVE-2024-12718].)
- `extract()` e `extractall()` agora (re)aplicam o filtro de extração ao substituir um link (físico ou simbólico) por uma cópia de outro membro do arquivo e ao corrigir atributos de diretório. O primeiro levanta uma nova exceção, `LinkFallbackError`. (Contribuição de Petr Viktorin para **CVE 2025-4330** [https://www.cve.org/CVERecord?id=CVE-2025-4330] e **CVE 2024-12718** [https://www.cve.org/CVERecord?id=CVE-2024-12718].)
- `extract()` e `extractall()` não extraem mais membros rejeitados quando `errorlevel()` é zero. (Contribuição de Matt Prodani e Petr Viktorin em gh-112887 [https://github.com/python/cpython/issues/112887] e **CVE 2025-4435** [https://www.cve.org/CVERecord?id=CVE-2025-4435].)

threading

- `threading.Thread.start()` agora define o nome do thread do sistema operacional como `threading.Thread.name`. (Contribuição de Victor Stinner em gh-59705 [https://github.com/python/cpython/issues/59705].)

tkinter

- Faz com que os métodos **`after()`** e **`after_idle()`** de widgets do **`tkinter`** aceitem argumentos nomeados (Contribuição de Zhikang Yan em gh-126899 [https://github.com/python/cpython/issues/126899].)
- Adiciona a capacidade de especificar um nome para **`tkinter.OptionMenu`** e **`tkinter.ttk.OptionMenu`**. (Contribuição de Zhikang Yan em gh-130482 [https://github.com/python/cpython/issues/130482].)

turtle

- Adiciona gerenciadores de contexto para **`turtle.fill()`**, **`turtle.poly()`** e **`turtle.no_animation()`**. (Contribuição de Marie Roald e Yngve Mardal Moe em gh-126350 [https://github.com/python/cpython/issues/126350].)

types

- **`types.UnionType`** agora é um apelido para **`typing.Union`**. Veja abaixo para mais detalhes. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)

typing

- Os tipos **`types.UnionType`** e **`typing.Union`** agora são apelidos um do outro, o que significa que tanto as uniões no estilo antigo (criadas com `Union[int, str]`) quanto as uniões no estilo novo (`int | str`) agora criam instâncias do mesmo tipo em ambiente de execução. Isso unifica o comportamento entre as duas sintaxes, mas leva a algumas diferenças de comportamento que podem afetar os usuários que inspecionam tipos em ambiente de execução:
    - Ambas as sintaxes para criar uma união agora produzem a mesma representação de string em **`repr()`**. Por exemplo, `repr(Union[int, str])` agora é `"int | str"` em vez de `"typing.Union[int, str]"`.
    - Uniões criadas usando a sintaxe antiga não são mais armazenadas em cache. Anteriormente, executar `Union[int, str]` várias vezes retornava o mesmo objeto (`Union[int, str] is Union[int,`

`str]` seria `True`), mas agora retornará dois objetos diferentes. Use `==` para comparar uniões por igualdade, não `is`. Uniões de novo estilo nunca foram armazenadas em cache dessa forma. Essa alteração pode aumentar o uso de memória para alguns programas que usam um grande número de uniões criadas por meio do índice `typing.Union`. No entanto, vários fatores compensam esse custo: uniões usadas em anotações não são mais avaliadas por padrão no Python 3.14 devido a **PEP 649** [https://peps.python.org/pep-0649/]; uma instância de `types.UnionType` é muito menor do que o objeto retornado por `Union[]` em versões anteriores do Python; e remover o cache também economiza espaço. Portanto, é improvável que essa alteração cause um aumento significativo no uso de memória para a maioria dos usuários.
  - Anteriormente, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para compatibilidade com versões anteriores, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como `get_origin()` e `typing.get_args()`, em vez de depender de detalhes de implementação privada.
  - Agora é possível usar o próprio `typing.Union` em verificações de `isinstance()`. Por exemplo, `isinstance(int | str, typing.Union)` retornará `True`; anteriormente, isso levantava `TypeError`.
  - O atributo **__args__** dos objetos `typing.Union` não é mais gravável.
  - Não é mais possível definir atributos em objetos `Union`. Isso só funcionava para atributos dunder em versões anteriores, nunca foi documentado que funcionasse e apresentava falhas sutis em muitos casos.

(Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)

- `TypeAliasType` agora oferece suporte a desempacotamento em estrela.

unicodedata

- O banco de dados Unicode foi atualizado para Unicode 16.0.0.

unittest

- A saída do `unittest` agora é colorida por padrão. Isso pode ser controlado pelas variáveis de ambiente. (Contribuição de Hugo van Kemenade em gh-127221 [https://github.com/python/cpython/issues/127221].)

- O Unittest Discovery suporta novamente pacote de espaço de nomes como diretório inicial. Ele foi removido no Python 3.11. (Contribuição de Jacob Walls em gh-80958 [https://github.com/python/cpython/issues/80958].)

- Vários novos métodos foram adicionados à classe `TestCase` que fornecem testes mais especializados.

  - `assertHasAttr()` e `assertNotHasAttr()` verificam se o objeto tem um atributo específico.
  - `assertIsSubclass()` e `assertNotIsSubclass()` verificam se o objeto é uma subclasse de uma classe específica ou de uma tupla de classes.
  - `assertStartsWith()`, `assertNotStartsWith()`, `assertEndsWith()` e `assertNotEndsWith()` verificam se a sequência de caracteres Unicode ou de bytes começa ou termina com sequências de caracteres específicas.

  (Contribuição de Serhiy Storchaka em gh-71339 [https://github.com/python/cpython/issues/71339].)

urllib

- Atualiza o algoritmo de autenticação de resumo HTTP para `urllib.request`, oferecendo suporte à autenticação de resumo SHA-256, conforme especificado em **RFC 7616** [https://datatracker.ietf.org/doc/html/rfc7616.html]. (Contribuição de Calvin Bui em gh-128193 [https://github.com/python/cpython/issues/128193].)

- Melhora a ergonomia e a conformidade com os padrões ao analisar e emitir URLs `file:`.

  Em `url2pathname()`:

  - Aceita uma URL completa quando o novo argumento *require_scheme* for definido como verdadeiro.
  - Descarta a autoridade de URL se ela corresponder ao nome de host local.
  - Descarta a autoridade de URL se ela for resolvida para um endereço IP local quando o novo argumento *resolve_host* for definido como verdadeiro.
  - Descarta componentes de consulta e fragmento de URL.
  - Levanta `URLError` se uma autoridade de URL não for local, exceto no Windows, onde retornamos um caminho UNC como antes.

  Em `pathname2url()`:

  - Retorna uma URL completa quando o novo argumento *add_scheme* for definido como verdadeiro.
  - Inclui uma autoridade de URL vazia quando um caminho começar com uma barra. Por exemplo, o caminho `/etc/hosts` é convertido para a URL `///etc/hosts`.

  No Windows, as letras de unidade não são mais convertidas em maiúsculas, e os caracteres `:` que não seguem uma letra de unidade não causam mais o levantamento de uma exceção `OSError`.

  (Contribuição de Barney Gale em gh-125866 [https://github.com/python/cpython/issues/125866].)

uuid

- Adiciona suporte para as versões 6, 7 e 8 do UUID via `uuid6()`, `uuid7()` e `uuid8()`, respectivamente, conforme especificado em **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html]. (Contribuição de Bénédikt Tran em gh-89083 [https://github.com/python/cpython/issues/89083].)

- `uuid.NIL` e `uuid.MAX` agora estão disponíveis para representar os formatos UUID Nil e Max, conforme definido por **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html]. (Contribuição de Nick Pope em gh-128427 [https://github.com/python/cpython/issues/128427].)
- Permite gerar vários UUIDs simultaneamente na linha de comando via `python -m uuid --count`. (Contribuição de Simon Legner em gh-131236 [https://github.com/python/cpython/issues/131236].)

webbrowser

- Os nomes na variável de ambiente **`BROWSER`** agora podem se referir a navegadores já registrados para o módulo `webbrowser`, em vez de sempre gerar um novo comando de navegador.

  Isso torna possível definir **`BROWSER`** como o valor de um dos navegadores suportados no macOS.

zipfile

- Adicionado `ZipInfo._for_archive`, m método para resolver padrões adequados para um objeto `ZipInfo`, conforme usado por `ZipFile.writestr`. (Contribuição de Bénédikt Tran em gh-123424 [https://github.com/python/cpython/issues/123424].)
- `ZipFile.writestr()` agora respeita a variável de ambiente **`SOURCE_DATE_EPOCH`** para melhor oferece suporte a construções reproduzíveis. (Contribuição de Jiahao Li em gh-91279 [https://github.com/python/cpython/issues/91279].)

Otimizações

- O tempo de importação para vários módulos de biblioteca padrão foi melhorado, incluindo `annotationlib`, `ast`, `asyncio`, `base64`, `cmd`, `csv`, `gettext`, `importlib.util`, `locale`, `mimetypes`, `optparse`, `pickle`, `pprint`, `pstats`, `shlex`, `socket`, `string`, `subprocess`, `threading`, `tomllib`, `types` e `zipfile`.

(Contribuição de Adam Turner, Bénédikt Tran, Chris Markiewicz, Eli Schwartz, Hugo van Kemenade, Jelle Zijlstra e outros em gh-118761 [https://github.com/python/cpython/issues/118761].)

- O interpretador agora evita algumas modificações na contagem de referências internamente quando é seguro fazê-lo. Isso pode levar a valores diferentes sendo retornados de `sys.getrefcount()` e `Py_REFCNT()` em comparação com versões anteriores do Python. Consulte abaixo para obter detalhes.

asyncio

- Os resultados do benchmark padrão melhoraram em 10 a 20% após a implementação de uma nova lista duplamente encadeada por thread para `tarefas_nativas`, reduzindo também o uso de memória. Isso permite que ferramentas externas de introspecção, como python -m asyncio pstree, introspectem o grafo de chamadas de tarefas asyncio em execução em todas as threads. (Contribuição de Kumar Aditya em gh-107803 [https://github.com/python/cpython/issues/107803].)
- O módulo agora conta com suporte de primeira classe para construção com threads livres. Isso permite a execução paralela de múltiplos laços de eventos em diferentes threads, com escala linear com o número de threads. (Contribuição de Kumar Aditya em gh-128002 [https://github.com/python/cpython/issues/128002].)

base64

- `b16decode()` agora é até seis vezes mais rápido. (Contribuição de Bénédikt Tran, Chris Markiewicz e Adam Turner em gh-118761 [https://github.com/python/cpython/issues/118761].)

bdb

- O depurador básico agora conta com um backend baseado em `sys.monitoring`, que pode ser selecionado passando `'monitoring'` para o novo parâmetro *backend* da classe `Bdb`. (Contribuição de Tian Gao em gh-124533 [https://github.com/python/cpython/issues/124533].)

difflib

- A função `IS_LINE_JUNK()` agora é até duas vezes mais rápida. (Contribuição de Adam Turner e Semyon Moroz em gh-130167 [https://github.com/python/cpython/issues/130167].)

gc

- **A partir do Python 3.14.5:**

  As versões 3.14.0 a 3.14.4 do Python incluíam um novo coletor de lixo incremental. No entanto, devido a diversos relatos [https://github.com/python/cpython/issues/142516] de pressão significativa sobre a memória em ambientes de produção, houve um retorno ao GC geracional da versão 3.13. Esse é o GC utilizado atualmente no Python 3.14.5 e versões posteriores.

- **Anteriormente nas versões 3.14.0 a 3.14.4 do Python:**

  O novo coletor de lixo incremental significa que os tempos máximos de pausa são reduzidos em uma ordem de magnitude ou mais para heaps maiores.

  Devido a essa otimização, o significado dos resultados de `get_threshold()` e `set_threshold()` foi alterado, juntamente com `get_count()` e `get_stats()`.

  - Para retrocompatibilidade, `get_threshold()` continua retornando uma tupla de três itens. O primeiro valor é o limite para coleções novas, como antes; o segundo valor determina a taxa de varredura da coleção antiga (o padrão é 10, e valores mais altos significam que a coleção antiga é varrida mais lentamente). O terceiro valor agora não tem mais significado e é sempre zero.
  - `set_threshold()` agora ignora quaisquer itens após o segundo.
  - `get_count()` e `get_stats()` continuam retornando o mesmo formato de resultados. A única diferença é que, em vez dos resultados se referirem às gerações jovem, idosa e idosa, os

resultados se referem à geração jovem e aos espaços de envelhecimento e coleta da geração idosa.

Em resumo, o código que tenta manipular o comportamento do ciclo de coleta de lixo pode não funcionar exatamente como o esperado, mas é muito improvável que seja prejudicial. Todos os outros códigos funcionarão perfeitamente.

(Contribuição de Mark Shannon em gh-108362 [https://github.com/python/cpython/issues/108362].)

io

- Abrir e ler arquivos agora executa menos chamadas de sistema. Ler um pequeno arquivo em cache do sistema operacional na íntegra é até 15% mais rápido. (Contribuição de Cody Maloney e Victor Stinner em gh-120754 [https://github.com/python/cpython/issues/120754] e gh-90102 [https://github.com/python/cpython/issues/90102].)

pathlib

- `Path.read_bytes` agora usa o modo sem buffer para abrir arquivos, o que é entre 9% e 17% mais rápido para leitura completa. (Contribuição de Cody Maloney em gh-120754 [https://github.com/python/cpython/issues/120754].)

pdb

- `pdb` agora suporta dois backends, baseados em `sys.settrace()` ou `sys.monitoring`. Usar a CLI do pdb ou `breakpoint()` sempre usará o backend `sys.monitoring`. Instanciar explicitamente `pdb.Pdb` e suas classes derivadas usará o backend `sys.settrace()` por padrão, que é configurável. (Contribuição de Tian Gao em gh-124533 [https://github.com/python/cpython/issues/124533].)

textwrap

- Otimiza a função `dedent()`, melhorando o desempenho em média de 2,4x, com melhorias maiores para entradas mais longas, e corrige um bug de normalização incompleta de linhas em branco com caractere de espaço em branco que não seja espaço ou tabulação.

uuid

- `uuid3()` e `uuid5()` agora são aproximadamente 40% mais rápidos para nomes de 16 bytes e 20% mais rápidos para nomes de 1024 bytes. O desempenho para nomes mais longos permanece inalterado. (Contribuição de Bénédikt Tran em gh-128150 [https://github.com/python/cpython/issues/128150].)
- `uuid4()` agora é cerca de 30% mais rápido. (Contribuição de Bénédikt Tran em gh-128150 [https://github.com/python/cpython/issues/128150].)

zlib

- No Windows, zlib-ng [https://github.com/zlib-ng/zlib-ng] agora é usado como implementação do módulo `zlib` nos binários padrão. Não há incompatibilidades conhecidas entre `zlib-ng` e a implementação `zlib` usada anteriormente. Isso deve resultar em melhor desempenho em todos os níveis de compactação.

  Vale ressaltar que `zlib.Z_BEST_SPEED` (`1`) pode resultar em significativamente menos compressão do que a implementação anterior, além de reduzir significativamente o tempo necessário para compactar.

  (Contribuição de Steve Dower em gh-91349 [https://github.com/python/cpython/issues/91349].)

Removidos

argparse

- Remove os parâmetros *type*, *choices* e *metavar* de **`BooleanOptionalAction`**. Eles estão descontinuados desde o Python 3.12. (Contribuição de Nikita Sobolev em gh-118805 [https://github.com/python/cpython/issues/118805].)
- Chamar **`add_argument_group()`** em um grupo de argumentos agora gera um **`ValueError`**. Da mesma forma, **`add_argument_group()`** ou **`add_mutually_exclusive_group()`** em um grupo mutuamente exclusivo agora levantam **`ValueError`**s. Esse "aninhamento" nunca foi suportado, frequentemente não funcionava corretamente e era exposto involuntariamente por herança. Essa funcionalidade está descontinuada desde o Python 3.11. (Contribuição de Savannah Ostrowski em gh-127186 [https://github.com/python/cpython/issues/127186].)

ast

- Remove as seguintes classes, que são apelidos descontinuados de **`Constant`** desde o Python 3.8 e emitem avisos de descontinuação desde o Python 3.12:

  - **`Bytes`**
  - **`Ellipsis`**
  - **`NameConstant`**
  - **`Num`**
  - **`Str`**

  Como consequência dessas remoções, os métodos `visit_Num`, `visit_Str`, `visit_Bytes`, `visit_NameConstant` e `visit_Ellipsis` definidos pelo usuário nas subclasses personalizadas **`NodeVisitor`** não serão mais chamados quando a subclasse **`NodeVisitor`** estiver visitando uma AST. Em vez disso, defina um método `visit_Constant`.

  (Contribuição de Alex Waygood em gh-119562 [https://github.com/python/cpython/issues/119562].)

- Remove as seguintes propriedades descontinuadas em **`ast.Constant`**, que estavam presentes para compatibilidade com as classes AST agora removidas:

- `Constant.n`
- `Constant.s`

Em vez disso, use `Constant.value`. (Contribuição de Alex Waygood em gh-119562 [https://github.com/python/cpython/issues/119562].)

asyncio

- Remove as seguintes classes, métodos e funções, que foram descontinuados desde o Python 3.12:

  - `AbstractChildWatcher`
  - `FastChildWatcher`
  - `MultiLoopChildWatcher`
  - `PidfdChildWatcher`
  - `SafeChildWatcher`
  - `ThreadedChildWatcher`
  - `AbstractEventLoopPolicy.get_child_watcher()`
  - `AbstractEventLoopPolicy.set_child_watcher()`
  - `get_child_watcher()`
  - `set_child_watcher()`

  (Contribuição de Kumar Aditya em gh-120804 [https://github.com/python/cpython/issues/120804].)

- `asyncio.get_event_loop()` agora levanta uma `RuntimeError` se não houver um laço de eventos atual e não cria mais implicitamente um laço de eventos.

  (Contribuição de Kumar Aditya em gh-126353 [https://github.com/python/cpython/issues/126353].)

  Existem alguns padrões que usam `asyncio.get_event_loop()`, a maioria deles pode ser substituída por `asyncio.run()`.

  Se você estiver executando uma função assíncrona, basta usar `asyncio.run()`.

Antes:

```
async def main():
    ...


loop = asyncio.get_event_loop()
try:
    loop.run_until_complete(main())
finally:
    loop.close()
```

Depois:

```
async def main():
    ...

asyncio.run(main())
```

Se você precisar iniciar algo, por exemplo, um servidor escutando em um soquete e depois rodar para sempre, use **`asyncio.run()`** e um **`asyncio.Event`**.

Antes:

```
def start_server(loop): ...

loop = asyncio.get_event_loop()
try:
    start_server(loop)
    loop.run_forever()
finally:
    loop.close()
```

Depois:

```
def start_server(loop): ...

async def main():
    start_server(asyncio.get_running_loop())
    await asyncio.Event().wait()
```

```
asyncio.run(main())
```

Se você precisar executar algo em um laço de eventos, execute algum código de bloqueio em torno dele, use `asyncio.Runner`.

Antes:

```
async def operation_one(): ...
def blocking_code(): ...
async def operation_two(): ...

loop = asyncio.get_event_loop()
try:
    loop.run_until_complete(operation_one())
    blocking_code()
    loop.run_until_complete(operation_two())
finally:
    loop.close()
```

Depois:

```
async def operation_one(): ...
def blocking_code(): ...
async def operation_two(): ...

with asyncio.Runner() as runner:
    runner.run(operation_one())
    blocking_code()
    runner.run(operation_two())
```

email

- Remove o parâmetro *isdst* de `email.utils.localtime()`, que foi descontinuado e é ignorado desde o Python 3.12. (Contribuição de Hugo van Kemenade em gh-118798 [https://github.com/python/cpython/issues/118798].)

importlib.abc

- Remove classes descontinuadas `importlib.abc`:

  - `ResourceReader` (use `TraversableResources`)
  - `Traversable` (use `Traversable`)
  - `TraversableResources` (use `TraversableResources`)

  (Contribuição de Jason R. Coombs e Hugo van Kemenade em gh-93963 [https://github.com/python/cpython/issues/93963].)

itertools

- Remove o suporte para operações de cópia, cópia profunda e de serialização com pickle dos iteradores `itertools`. Eles emitem um `DeprecationWarning` desde o Python 3.12. (Contribuição de Raymond Hettinger em gh-101588 [https://github.com/python/cpython/issues/101588].)

pathlib

- Remove o suporte para passagem de argumentos nomeados adicionais para `Path`. Em versões anteriores, tais argumentos eram ignorados. (Contribuição de Barney Gale em gh-74033 [https://github.com/python/cpython/issues/74033].)
- Remove o suporte para passagem de argumentos posicionais adicionais para `PurePath.relative_to()` e `is_relative_to()`. Em versões anteriores, tais argumentos eram unidos a *other*. (Contribuição de Barney Gale em gh-78707 [https://github.com/python/cpython/issues/78707].)

pkgutil

- Remove as funções `get_loader()` e `find_loader()`, que foram descontinuadas desde o Python 3.12. (Contribuição de Bénédikt Tran em gh-97850 [https://github.com/python/cpython/issues/97850].)

pty

- Remove as funções `master_open()` e `slave_open()`, que foram descontinuadas desde o Python 3.12. Em vez disso, use

`pty.openpty()`. (Contribuição de Nikita Sobolev em gh-118824 [https://github.com/python/cpython/issues/118824].)

sqlite3

- Remove `version` e `version_info` do módulo `sqlite3`; use `sqlite_version` e `sqlite_version_info` para o número da versão real da biblioteca SQLite em tempo de execução. (Contribuição de Hugo van Kemenade em gh-118924 [https://github.com/python/cpython/issues/118924].)
- Usar uma sequência de parâmetros com espaços reservados nomeados agora levanta uma `ProgrammingError`, que estava descontinuado desde o Python 3.12. (Contribuição de Erlend E. Aasland em gh-118928 [https://github.com/python/cpython/issues/118928] e gh-101693 [https://github.com/python/cpython/issues/101693].)

urllib

- Remove a classe `Quoter` de `urllib.parse`, que está descontinuada desde o Python 3.11. (Contribuição de Nikita Sobolev em gh-118827 [https://github.com/python/cpython/issues/118827].)

- Remove as classes `URLopener` e `FancyURLopener` de `urllib.request`, que foram descontinuadas desde o Python 3.3.

  `myopener.open()` pode ser substituído por `urlopen()`. `myopener.retrieve()` pode ser substituído por `urlretrieve()`. Personalizações nas classes de abertura podem ser feitas passando manipuladores personalizados para `build_opener()`. (Contribuição de Barney Gale em gh-84850 [https://github.com/python/cpython/issues/84850].)

Descontinuados

Novas descontinuações

- Passar um número complexo como argumento *real* ou *imag* no construtor `complex()` agora está descontinuado; números complexos devem ser passados apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)

- `argparse`:

  - A passagem do argumento nomeado não documentado *prefix_chars* para o método `add_argument_group()` agora está descontinuado. (Contribuição de Savannah Ostrowski em gh-125563 [https://github.com/python/cpython/issues/125563].)
  - O conversor de tipo `argparse.FileType` foi descontinuado. Qualquer coisa relacionada a gerenciamento de recursos deve ser tratado posteriormente, após os argumentos terem sido analisados. (Contribuição de Serhiy Storchaka em gh-58032 [https://github.com/python/cpython/issues/58032].)

- `asyncio`:

  - A **`asyncio.iscoroutinefunction()`** está agora descontinuado e será removido no Python 3.16, use `inspect.iscoroutinefunction()` em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)

  - O sistema de políticas `asyncio` está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:

    - `asyncio.AbstractEventLoopPolicy`
    - `asyncio.DefaultEventLoopPolicy`
    - `asyncio.WindowsSelectorEventLoopPolicy`
    - `asyncio.WindowsProactorEventLoopPolicy`
    - `asyncio.get_event_loop_policy()`
    - `asyncio.set_event_loop_policy()`

Os usuários devem usar **`asyncio.run()`** ou **`asyncio.Runner`** com o argumento *loop_factory* para usar a implementação de laço de eventos desejada.

por exemplo, para usar **`asyncio.SelectorEventLoop`** no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventL
```

(Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- **`codecs`**: A função **`codecs.open()`** está descontinuada e será removida em uma versão futura do Python. Em vez disso, use **`open()`**. (Contribuição de Inada Naoki em gh-133036 [https://github.com/python/cpython/issues/133036].)

- **`ctypes`**:

  - Em plataformas não Windows, a configuração de **`Structure._pack_`** para usar um layout de memória padrão compatível com MSVC foi também descontinuada em favor da configuração de **`Structure._layout_`** para `'ms'`, e será removida no Python 3.19. (Contribuição de Petr Viktorin em gh-131747 [https://github.com/python/cpython/issues/131747].)
  - A chamada de **`ctypes.POINTER()`** em uma string está agora descontinuada. Use tipos incompletos para estruturas autorreferenciais. Além disso, o `ctypes._pointer_type_cache` interno está descontinuado. Consulte **`ctypes.POINTER()`** para obter detalhes de implementação atualizados. (Contribuição de Sergey Myrianov em gh-100926 [https://github.com/python/cpython/issues/100926].)

- **`functools`**: Chamar a implementação Python de **`functools.reduce()`** com *function* ou *sequence* como argumentos nomeados agora está descontinuado; os parâmetros serão tornados somente-posicionais no Python 3.16. (Contribuição de Kirill Podoprigora em gh-121676 [https://github.com/python/cpython/issues/121676].)

- **`logging`**: O suporte para manipuladores de registro personalizados com o argumento *strm* está agora descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)

- **`mimetypes`**: Extensões válidas estão vazias ou devem começar com '.' para **`mimetypes.MimeTypes.add_type()`**. Extensões sem ponto estão descontinuadas e levantarão uma exceção **`ValueError`** no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)

- **`nturl2path`**: Este módulo está descontinuado. Em vez disso, chame **`urllib.request.url2pathname()`** e **`pathname2url()`**. (Contribuição de Barney Gale em gh-125866 [https://github.com/python/cpython/issues/125866].)

- **`os`**: As funções **`os.popen()`** e **`os.spawn*`** estão agora suavemente descontinuadas. Elas não devem mais ser usadas para escrever código novo. O módulo **`subprocess`** é recomendado em seu lugar. (Contribuição de Victor Stinner em gh-120743 [https://github.com/python/cpython/issues/120743].)

- **`pathlib`**: **`pathlib.PurePath.as_uri()`** está agora descontinuado e está programado para remoção no Python 3.19. Em vez disso, use **`pathlib.Path.as_uri()`**. (Contribuição de Barney Gale em gh-123599 [https://github.com/python/cpython/issues/123599].)

- **`pdb`**: O atributo não documentado `pdb.Pdb.curframe_locals` agora é uma propriedade somente leitura descontinuada, que será removido em uma versão futura do Python. O acesso dinâmico de baixa sobrecarga aos locais de quadro, adicionado no Python 3.13 pela **PEP 667** [https://peps.python.org/pep-0667/], significa que a referência ao cache de

locais de quadro, armazenada anteriormente neste atributo, não é mais necessária. Depuradores derivados devem acessar `pdb.Pdb.curframe.f_locals` diretamente no Python 3.13 e versões posteriores. (Contribuição de Tian Gao em gh-124369 [https://github.com/python/cpython/issues/124369] e gh-125951 [https://github.com/python/cpython/issues/125951].)

- **`symtable`**: Descontinua **`symtable.Class.get_methods()`** por falta de interesse, programado para remoção no Python 3.16. (Contribuição de Bénédikt Tran em gh-119698 [https://github.com/python/cpython/issues/119698].)

- **`tkinter`**: Os métodos **`tkinter.Variable`** **`trace_variable()`**, **`trace_vdelete()`** e **`trace_vinfo()`** estão descontinuados. Em vez disso, use **`trace_add()`**, **`trace_remove()`** e **`trace_info()`**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

- **`urllib.parse`**: Aceitar objetos com valores falsos (como `0` e `[]`), exceto strings vazias, objetos bytes ou similares e `None` nas funções **`parse_qsl()`** e **`parse_qs()`**, agora está descontinuado. (Contribuição de Serhiy Storchaka em gh-116897 [https://github.com/python/cpython/issues/116897].)

Remoção pendente no Python 3.15

- O sistema de importação:
  - A definição de `__cached__` em um módulo enquanto falha na definição de **`__spec__.cached`** está descontinuado. No Python 3.15, `__cached__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
  - A definição de **`__package__`** em um módulo enquanto falha na definição de **`__spec__.parent`** está descontinuado. No Python 3.15, **`__package__`** deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
- **`ctypes`**:

    - A função não documentada **`ctypes.SetPointerType()`** foi descontinuada desde o Python 3.13.
- **`http.server`**:
    - A classe obsoleta e raramente usada **`CGIHTTPRequestHandler`** foi descontinuada desde o Python 3.13. Não existe substituição direta. *Qualquer coisa* é melhor que CGI para fazer a interface de um servidor web com um manipulador de requisição.
    - O sinalizador **`--cgi`** para a interface de linha de comando **python -m http.server** foi descontinuado desde o Python 3.13.
- **`importlib`**:
    - Método `load_module()`: use `exec_module()`.
- **`pathlib`**:
    - **`.PurePath.is_reserved()`** foi descontinuado desde o Python 3.13. Use **`os.path.isreserved()`** para detectar caminhos reservados no Windows.
- **`platform`**:
    - **`platform.java_ver()`** foi descontinuada desde o Python 3.13. Esta função é útil apenas para suporte Jython, tem uma API confusa e é amplamente não testada.
- **`sysconfig`**:
    - O argumento *check_home* de **`sysconfig.is_python_build()`** foi descontinuado desde o Python 3.12.
- **`threading`**:
    - **`RLock()`** não aceitará argumentos no Python 3.15. A passagem de quaisquer argumentos foi descontinuada a partir do Python 3.14, uma vez que a versão do Python não permite quaisquer argumentos, mas a versão C permite qualquer número de argumentos posicionais ou nomeados, ignorando todos os argumentos.
- **`types`**:
    - **`types.CodeType`**: o acesso a **`codeobject.co_lnotab`** foi descontinuado na **PEP 626** [https://peps.python.org/pep-0626/] desde 3.10 e foi planejado para ser removido em 3.12, mas só recebeu uma **`DeprecationWarning`** adequada em 3.12. Pode ser removido em 3.15. (Contribuição de Nikita Sobolev em gh-101866 [https://github.com/python/cpython/issues/101866].)
- **`typing`**:

    - A não-documentada sintaxe de argumento nomeado para criar classes `NamedTuple` (por exemplo, `Point = NamedTuple("Point", x=int, y=int)`) foi descontinuada desde o Python 3.13. Em vez disso, use as sintaxes baseada em classe ou funcional.
    - Ao usar a sintaxe funcional de `TypedDict`s, não passar um valor para o parâmetro *fields* (`TD = TypedDict("TD")`) ou passar `None` (`TD = TypedDict("TD", None)`) foi está descontinuado desde o Python 3.13. Use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})` para criar uma classe TypedDict com nenhum campo.
    - A função decoradora **`@typing.no_type_check_decorator`** foi descontinuada desde o Python 3.13. Após oito anos no módulo `typing`, ela ainda não foi suportada por nenhum verificador de tipo importante.
- Módulos **`sre_compile`**, **`sre_constants`** e **`sre_parse`**.
- `wave`:
    - Os métodos `getmark()`, `setmark()` e `getmarkers()` das classes `Wave_read` e `Wave_write` foram descontinuados desde o Python 3.13.
- `zipimport`:
    - **`zipimport.zipimporter.load_module()`** está descontinuado desde o Python 3.10. Em vez disso, use `exec_module()`. (gh-125746 [https://github.com/python/cpython/issues/125746].)

Remoção pendente no Python 3.16

- O sistema de importação:

    - A definição de `__loader__` em um módulo enquanto falha na definição de `__spec__.loader` está descontinuado. No Python 3.16, **`__loader__`** deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.

- `array`:

- O código de formato `'u'` (**`wchar_t`**) foi descontinuado na documentação desde o Python 3.3 e do ambiente de execução desde o Python 3.13. Em vez disso, use o código de formato `'w'` (**`Py_UCS4`**) para caracteres Unicode.

- **`asyncio`**:

  - **`asyncio.iscoroutinefunction()`** foi descontinuado e será removido no Python 3.16, use **`inspect.iscoroutinefunction()`** em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)

  - O sistema de políticas **`asyncio`** está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:

    - **`asyncio.AbstractEventLoopPolicy`**
    - **`asyncio.DefaultEventLoopPolicy`**
    - **`asyncio.WindowsSelectorEventLoopPolicy`**
    - **`asyncio.WindowsProactorEventLoopPolicy`**
    - **`asyncio.get_event_loop_policy()`**
    - **`asyncio.set_event_loop_policy()`**

    Os usuários devem usar **`asyncio.run()`** ou **`asyncio.Runner`** com *loop_factory* para usar a implementação de loop de eventos desejada.

    por exemplo, para usar **`asyncio.SelectorEventLoop`** no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventL
```

(Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- **builtins**:
  - A inversão bit a bit em tipos booleanos, `~True` ou `~False` foi descontinuada desde o Python 3.12, pois produz resultados surpreendentes e não intuitivos (`-2` e `-1`). Em vez disso, use `not x` para a negação lógica de um booleano. No caso raro de você precisar da inversão bit a bit do inteiro subjacente, converta para `int` explicitamente (`~int(x)`).

- **functools**:
  - A chamada da implementação Python de **`functools.reduce()`** com *function* ou *sequence* como argumentos nomeados foi descontinuada desde o Python 3.14.

- **logging**:
  - O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)

- **mimetypes**:
  - Extensões válidas começam com um '.' ou estão vazias para **`mimetypes.MimeTypes.add_type()`**. Extensões sem ponto estão descontinuadas e levantarão uma exceção **`ValueError`** no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)

- **shutil**:
  - A exceção **`ExecError`** foi descontinuada desde o Python 3.14. Ela não foi usada por nenhuma função em **`shutil`** desde o Python 3.4,

e agora é um alias de `RuntimeError`.

- `symtable`:
  - O método `Class.get_methods` foi descontinuado desde o Python 3.14.
- `sys`:
  - A função `_enablelegacywindowsfsencoding()` foi descontinuada desde o Python 3.13. Em vez disso, use a variável de ambiente `PYTHONLEGACYWINDOWSFSENCODING`.
- `sysconfig`:
  - A função `sysconfig.expand_makefile_vars()` está descontinuada desde o Python 3.14. Em vez disso, use o argumento `vars` de `sysconfig.get_paths()`.
- `tarfile`:
  - O atributo não documentado e não utilizado `TarFile.tarfile` foi descontinuado desde o Python 3.13.

Remoção pendente no Python 3.17

- `datetime`:
  - `strptime()` chama usando uma string de formato contendo `%e` (dia do mês) sem um ano. Isso está descontinuado desde Python 3.15. (Contribuição de Stan Ulbrych em gh-70647 [https://github.com/python/cpython/issues/70647].)
- `collections.abc`:
  - `collections.abc.ByteString` está agendado para remoção no Python 3.17.

Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

`ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`encodings`**:
  - Passar nomes de *codificação* não ASCII para **`encodings.normalize_encoding()`** está descontinuado e sua remoção está prevista para o Python 3.17. (Contribuição de Stan Ulbrych em gh-136702 [https://github.com/python/cpython/issues/136702].)
- **`profile`**:
  - O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)
- **`webbrowser`**:
  - **`webbrowser.MacOSXOSAScript`** está descontinuado em favor de **`webbrowser.MacOS`**. (gh-137586

[https://github.com/python/cpython/issues/137586])

- **`typing`**:

  - Antes do Python 3.14, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para compatibilidade com versões anteriores, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como **`typing.get_origin()`** e **`typing.get_args()`**, em vez de depender de detalhes de implementação privada.

  - **`typing.ByteString`**, obsoleto desde o Python 3.9, está programado para ser removido no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`tkinter`**:

- Os métodos **`trace_variable()`**, **`trace()`** (um apelido de **`trace_variable()`**), **`trace_vdelete()`** e **`trace_vinfo()`** da classe **`tkinter.Variable`**, descontinuados desde o Python 3.14, têm remoção prevista para o Python 3.17. Em vez deles, utilize **`trace_add()`**, **`trace_remove()`** e **`trace_info()`**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

Remoção pendente no Python 3.18

- Não aceita mais um valor booleano quando um descritor de arquivo é esperado. (Contribuição de Serhiy Storchaka em gh-82626 [https://github.com/python/cpython/issues/82626].)

- **`decimal`**:

  - O especificador de formato de **`Decimal`** não padrão e não documentado `'N'`, que só é suportado na implementação C do módulo **`decimal`**, foi descontinuada desde o Python 3.13. (Contribuição de Serhiy Storchaka em gh-89902 [https://github.com/python/cpython/issues/89902].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

  - Linhas de `import` em arquivos *`name`*`.pth` são silenciosamente ignoradas.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

Remoção pendente no Python 3.19

- **`ctypes`**:

  - Troca implícita para o layout de estrutura compatível com MSVC definindo **`_pack_`**, mas não **`_layout_`** em plataformas não Windows.

- **hashlib**:
  - Em construtores de funções de hash, como **new()**, ou nos construtores nomeados diretamente pelo algoritmo de hash, como **md5()** e **sha256()**, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado `data=` ou `string=` em várias implementações do **hashlib**.

    O suporte ao nome do argumento nomeado `string` está agora descontinuado e previsto para ser removido no Python 3.19.

    Antes do Python 3.13, o parâmetro nomeado `string` não era suportado corretamente, dependendo da implementação de funções de hash do backend. Dê preferência a passar os dados iniciais como um argumento posicional para garantir a máxima compatibilidade com versões anteriores.

- **http.cookies**:
  - **http.cookies.Morsel.js_output()** está descontinuado e será removido no Python 3.19.
  - **http.cookies.BaseCookie.js_output()** está descontinuado e será removido no Python 3.19.

- **imaplib**:
  - A alteração de **IMAP4.file** agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade agora não é utilizada e alterar seu valor não fecha automaticamente o arquivo atual.

    Antes do Python 3.14, essa propriedade era usada para implementar os métodos `read()` e `readline()` correspondentes para **IMAP4**, mas isso não ocorre mais desde então.

Remoção pendente no Python 3.20

- Chamar o método `__new__()` de `struct.Struct` sem o argumento *format* está descontinuado e será removido no Python 3.20. Chamar o método `__init__()` em objetos `Struct` já inicializados está descontinuado e será removido no Python 3.20.

  (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize `sys.version_info` em seu lugar.

  - `argparse`
  - `csv`
  - `ctypes`
  - `ctypes.macholib`
  - `decimal` (use `decimal.SPEC_VERSION`)
  - `http.server`
  - `imaplib`
  - `ipaddress`
  - `json`
  - `logging` (`__date__` também descontinuado)
  - `optparse`
  - `pickle`
  - `platform`
  - `re`
  - `socketserver`
  - `tabnanny`
  - `tarfile`
  - `tkinter.font`
  - `tkinter.ttk`
  - `wsgiref.simple_server`
  - `xml.etree.ElementTree`
  - `xml.sax.expatreader`
  - `xml.sax.handler`
  - `zlib`

(Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:
  - Avisos são gerados para linhas `import` encontradas em arquivos *`name`*`.pth`.
  - Arquivos *`name`*`.pth` não são mais decodificados usando a codificação da localidade (*locale encoding*) por padrão. Eles **DEVEM** estar codificados em `utf-8-sig`.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

- **`ast`**:
  - A criação de instâncias de nós abstratos da AST (como **`ast.AST`** ou **`ast.expr`**) está descontinuada e levantará um erro no Python 3.20.

- **`typing`**:
  - É desencorajado realizar verificações com **`isinstance()`** e **`issubclass()`** em classes de protocolo que não foram explicitamente decoradas com **`runtime_checkable()`**, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso resultará em um **`TypeError`** no Python 3.20.

    (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

Remoção pendente em versões futuras

As APIs a seguir serão removidas no futuro, embora atualmente não haja uma data agendada para sua remoção.

- **`argparse`**:

  - O aninhamento de grupos de argumentos e o aninhamento de grupos mutuamente exclusivos estão descontinuado.
  - A passagem do argumento nomeado não documentado *prefix_chars* para `add_argument_group()` agora está descontinuado.
  - O conversor de tipo `argparse.FileType` está descontinuado.
- `builtins`:
  - Geradores: a assinatura `throw(type, exc, tb)` e `athrow(type, exc, tb)` está descontinuada: use `throw(exc)` e `athrow(exc)`, a assinatura do argumento único.
  - Atualmente Python aceita literais numéricos imediatamente seguidos por palavras reservadas como, por exemplo, `0in x`, `1or x`, `0if 1else 2`. Ele permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). Um aviso de sintaxe é levantado se o literal numérico for imediatamente seguido por uma das palavras reservadas `and`, `else`, `for`, `if`, `in` , `is` e `or`. Em uma versão futura, será alterado para um erro de sintaxe. (gh-87999 [https://github.com/python/cpython/issues/87999])
  - Suporte para métodos `__index__()` e `__int__()` retornando tipo não-int: esses métodos serão necessários para retornar uma instância de uma subclasse estrita de `int`.
  - Suporte para o método `__float__()` retornando uma subclasse estrita de `float`: esses métodos serão necessários para retornar uma instância de **`float`**.
  - Suporte para o método `__complex__()` retornando uma subclasse estrita de `complex`: esses métodos serão necessários para retornar uma instância de **`complex`**.
  - Passar um número complexo como argumento *real* ou *imag* no construtor `complex()` agora está descontinuado; deve ser passado apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)
- `calendar`: as constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por `calendar.JANUARY` e `calendar.FEBRUARY`. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)

- **`codecs`**: use **`open()`** em vez de **`codecs.open()`**. (gh-133038 [https://github.com/python/cpython/issues/133038])
- **`codeobject.co_lnotab`**: use o método **`codeobject.co_lines()`**.
- **`datetime`**:
  - **`utcnow()`**: use `datetime.datetime.now(tz=datetime.UTC)`.
  - **`utcfromtimestamp()`**: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- **`gettext`**: o valor de plural deve ser um número inteiro.
- **`importlib`**:
  - O parâmetro *debug_override* de **`cache_from_source()`** foi descontinuado: use o parâmetro *optimization*.
- **`importlib.metadata`**:
  - Interface de tupla `EntryPoints`.
  - `None` implícito nos valores de retorno.
- **`logging`**: o método `warn()` foi descontinuado desde o Python 3.3, use **`warning()`**.
- **`mailbox`**: o uso da entrada StringIO e do modo de texto foi descontinuado; em vez disso, use BytesIO e o modo binário.
- **`os`**: chame **`os.register_at_fork()`** em um processo multithread.
- **`os.path`**: **`os.path.commonprefix()`** está descontinuada; use **`os.path.commonpath()`** para prefixos de caminho. A função **`os.path.commonprefix()`** está sendo descontinuada por possuir nome e módulo enganosos. O uso dessa função não é seguro para prefixos de caminho, apesar de ela estar incluída em um módulo voltado à manipulação de caminhos; isso significa que é fácil introduzir acidentalmente vulnerabilidades de *path traversal* (travessia de diretórios) em programas Python ao utilizá-la.
- **`pydoc.ErrorDuringImport`**: um valor de tupla para o parâmetro *exc_info* foi descontinuado, use uma instância de exceção.
- **`re`**: regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)

- `shutil`: o parâmetro *onerror* de `rmtree()` foi descontinuado no Python 3.12; use o parâmetro *onexc*.
- Protocolos e opções de `ssl`
  - `ssl.SSLContext` sem argumento de protocolo foi descontinuado.
  - `ssl.SSLContext`: `set_npn_protocols()` e **`selected_npn_protocol()`** foram descontinuados, use ALPN.
  - Opções de `ssl.OP_NO_SSL*`
  - Opções de `ssl.OP_NO_TLS*`
  - `ssl.PROTOCOL_SSLv3`
  - `ssl.PROTOCOL_TLS`
  - `ssl.PROTOCOL_TLSv1`
  - `ssl.PROTOCOL_TLSv1_1`
  - `ssl.PROTOCOL_TLSv1_2`
  - `ssl.TLSVersion.SSLv3`
  - `ssl.TLSVersion.TLSv1`
  - `ssl.TLSVersion.TLSv1_1`
- Métodos de `threading`:
  - **`threading.Condition.notifyAll()`**: use `notify_all()`.
  - **`threading.Event.isSet()`**: use `is_set()`.
  - **`threading.Thread.isDaemon()`**, `threading.Thread.setDaemon()`: use o atributo `threading.Thread.daemon`.
  - **`threading.Thread.getName()`**, `threading.Thread.setName()`: use o atributo `threading.Thread.name`.
  - **`threading.currentThread()`**: use `threading.current_thread()`.
  - **`threading.activeCount()`**: use `threading.active_count()`.
- `typing.Text` (gh-92332 [https://github.com/python/cpython/issues/92332]).
- A classe interna `typing._UnionGenericAlias` não é mais usada para implementar `typing.Union`. Para preservar a compatibilidade com usuários que utilizam esta classe privada, uma correção de compatibilidade será fornecida pelo menos até a versão 3.17 do Python. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)
- `unittest.IsolatedAsyncioTestCase`: foi descontinuado retornar um valor que não seja `None` de um caso de teste.
- Funções descontinuadas de `urllib.parse`: use `urlparse()`

  - `splitattr()`
  - `splithost()`
  - `splitnport()`
  - `splitpasswd()`
  - `splitport()`
  - `splitquery()`
  - `splittag()`
  - `splittype()`
  - `splituser()`
  - `splitvalue()`
  - `to_bytes()`
- **`wsgiref`**: `SimpleHandler.stdout.write()` não deve fazer gravações parciais.
- **`xml.etree.ElementTree`**: testar o valor verdade de um **`Element`** está descontinuado. Em um lançamento futuro isso sempre retornará `True`. Em vez disso, prefira os testes explícitos `len(elem)` ou `elem is not None`.
- **`sys._clear_type_cache()`** está descontinuada: use **`sys._clear_internal_caches()`**.

Alterações de bytecode do CPython

- Substituído o opcode **`BINARY_SUBSCR`** pelo opcode **`BINARY_OP`** com o oparg `NB_SUBSCR`. (Contribuição de Irit Katriel em gh-100239 [https://github.com/python/cpython/issues/100239].)
- Adiciona os opcodes **`BUILD_INTERPOLATION`** e **`BUILD_TEMPLATE`** para construir novas instâncias **`Interpolation`** e **`Template`**, respectivamente. (Contribuição de Lysandros Nikolaou e outros em gh-132661 [https://github.com/python/cpython/issues/132661]; veja também PEP 750: strings template).
- Remove o opcode **`BUILD_CONST_KEY_MAP`**. Em vez disso, use **`BUILD_MAP`**. (Contribuição de Mark Shannon em gh-122160 [https://github.com/python/cpython/issues/122160].)
- Substitui o opcode **`LOAD_ASSERTION_ERROR`** por **`LOAD_COMMON_CONSTANT`** e adiciona suporte para carregar **`NotImplementedError`**.

- Adiciona os opcodes **LOAD_FAST_BORROW** e **LOAD_FAST_BORROW_LOAD_FAST_BORROW** para reduzir a sobrecarga de contagem de referências quando o interpretador puder provar que a referência no quadro sobrevive à referência carregada na pilha. (Contribuição de Matt Page em gh-130704 [https://github.com/python/cpython/issues/130704].)
- Adiciona o opcode **LOAD_SMALL_INT**, que envia um inteiro pequeno igual a `oparg` para a pilha. O opcode **RETURN_CONST** foi removido, pois não é mais usado. (Contribuição de Mark Shannon em gh-125837 [https://github.com/python/cpython/issues/125837].)
- Adiciona a nova instrução **LOAD_SPECIAL**. Gere código para as instruções **with** e **async with** usando a nova instrução. Remove as instruções **BEFORE_WITH** e **BEFORE_ASYNC_WITH**. (Contribuição de Mark Shannon em gh-120507 [https://github.com/python/cpython/issues/120507].
- Adiciona o opcode **POP_ITER** para oferecer suporte a iteradores "virtuais". (Contribuição de Mark Shannon em gh-132554 [https://github.com/python/cpython/issues/132554].)

Pseudoinstruções

- Adiciona a pseudoinstrução **ANNOTATIONS_PLACEHOLDER** para oferecer suporte a anotações em nível de módulo parcialmente executadas com avaliação adiada de anotações. (Contribuição de Jelle Zijlstra em gh-130907 [https://github.com/python/cpython/issues/130907].)
- Adiciona a pseudoinstrução **BINARY_OP_EXTEND** que executa um par de funções (funções de guarda e especialização) acessadas do cache inline. (Contribuição de Irit Katriel em gh-100239 [https://github.com/python/cpython/issues/100239].)
- Adiciona três especializações para **CALL_KW**; **CALL_KW_PY** para chamadas de funções Python, **CALL_KW_BOUND_METHOD** para chamadas de métodos vinculados e **CALL_KW_NON_PY** para todas as outras chamadas. (Contribuição de Mark Shannon em gh-118093 [https://github.com/python/cpython/issues/118093].)
- Adiciona as pseudoinstruções **JUMP_IF_TRUE** e **JUMP_IF_FALSE**, saltos condicionais que não impactam a pilha. Substituídas pela sequência `COPY 1`, `TO_BOOL`, `POP_JUMP_IF_TRUE/FALSE`. (Contribuição de Irit Katriel em gh-124285 [https://github.com/python/cpython/issues/124285].)

- Adiciona a pseudoinstrução **`LOAD_CONST_MORTAL`**. (Contribuição de Mark Shannon em gh-128685 [https://github.com/python/cpython/issues/128685].)
- Adiciona a pseudoinstrução **`LOAD_CONST_IMMORTAL`**, que faz o mesmo que **`LOAD_CONST`**, mas é mais eficiente para objetos imortais. (Contribuição de Mark Shannon em gh-125837 [https://github.com/python/cpython/issues/125837].)
- Adiciona a pseudoinstrução **`NOT_TAKEN`**, usada por **`sys.monitoring`** para registrar eventos de ramificação (como **`BRANCH_LEFT`**). (Contribuição de Mark Shannon em gh-122548 [https://github.com/python/cpython/issues/122548].)

Alterações na API C

API C de configuração do Python

Adiciona uma PyInitConfig API C para configurar a inicialização do Python sem depender de estruturas C e a capacidade de fazer alterações compatíveis com ABI no futuro.

Conclui a API C PyConfig da **PEP 587** [https://peps.python.org/pep-0587/] adicionando **`PyInitConfig_AddModule()`** que pode ser usado para adicionar um módulo de extensão embutido; um recurso anteriormente chamado de "inittab".

Adiciona as funções **`PyConfig_Get()`** e **`PyConfig_Set()`** para obter e definir a configuração de tempo de execução atual.

A "Configuração de Inicialização do Python", da **PEP 587** [https://peps.python.org/pep-0587/], unificou todas as maneiras de configurar a inicialização do Python. Esta PEP também unifica a configuração da pré-inicialização do Python e a inicialização do Python em uma única API. Além disso, esta PEP oferece apenas uma opção para incorporar o Python, em vez de ter duas opções, "Python" e "Isolated" (PEP 587), simplificando ainda mais a API.

A API PyConfig da PEP 587 de nível inferior permanece disponível para casos de uso com um nível intencionalmente mais alto de acoplamento aos

detalhes de implementação do CPython (como emular a funcionalidade completa da CLI do CPython, incluindo seus mecanismos de configuração).

(Contribuição de Victor Stinner em gh-107954 [https://github.com/python/cpython/issues/107954].)

Ver também

PEP 741 [https://peps.python.org/pep-0741/] e **PEP 587** [https://peps.python.org/pep-0587/]

Novos recursos na API C

- Adiciona `Py_PACK_VERSION()` e `Py_PACK_FULL_VERSION()`, duas novas macros para empacotamento de bits de números de versão do Python. Isso é útil para comparações com `Py_Version` ou `PY_VERSION_HEX`. (Contribuição de Petr Viktorin em gh-128629 [https://github.com/python/cpython/issues/128629].)

- Adiciona a função `PyBytes_Join(sep, iterable)`, semelhante a `sep.join(iterable)` em Python. (Contribuição de Victor Stinner em gh-121645 [https://github.com/python/cpython/issues/121645].)

- Adiciona funções para manipular a configuração do interpretador Python de tempo de execução atual (PEP 741: API C de configuração do Python):

  - `PyConfig_Get()`
  - `PyConfig_GetInt()`
  - `PyConfig_Set()`
  - `PyConfig_Names()`

  (Contribuição de Victor Stinner em gh-107954 [https://github.com/python/cpython/issues/107954].)

- Adiciona funções para configurar a inicialização do Python (PEP 741: API C de configuração do Python):

- `Py_InitializeFromInitConfig()`
- `PyInitConfig_AddModule()`
- `PyInitConfig_Create()`
- `PyInitConfig_Free()`
- `PyInitConfig_FreeStrList()`
- `PyInitConfig_GetError()`
- `PyInitConfig_GetExitCode()`
- `PyInitConfig_GetInt()`
- `PyInitConfig_GetStr()`
- `PyInitConfig_GetStrList()`
- `PyInitConfig_HasOption()`
- `PyInitConfig_SetInt()`
- `PyInitConfig_SetStr()`
- `PyInitConfig_SetStrList()`

(Contribuição de Victor Stinner em gh-107954 [https://github.com/python/cpython/issues/107954].)

- Adiciona a função `Py_fopen()` para abrir um arquivo. Isso funciona de forma semelhante à função padrão do C `fopen()`, aceitando um objeto Python como parâmetro *path* e definindo uma exceção em caso de erro. A nova função correspondente `Py_fclose()` deve ser usada para fechar um arquivo. (Contribuição de Victor Stinner em gh-127350 [https://github.com/python/cpython/issues/127350].)

- Adiciona `Py_HashBuffer()` para calcular e retornar o valor de hash de um buffer. (Contribuição de Antoine Pitrou e Victor Stinner em gh-122854 [https://github.com/python/cpython/issues/122854].)

- Adiciona as funções auxiliares `PyImport_ImportModuleAttr()` e `PyImport_ImportModuleAttrString()` para importar um módulo e obter um atributo do módulo. (Contribuição de Victor Stinner em gh-128911 [https://github.com/python/cpython/issues/128911].)

- Adiciona `PyIter_NextItem()` para substituir `PyIter_Next()`, que tem um valor de retorno ambíguo. (Contribuição de Irit Katriel e Erlend Aasland em gh-105201 [https://github.com/python/cpython/issues/105201].)

- Adiciona a função `PyLong_GetSign()` para obter o sinal de objetos `int`. (Contribuição de Sergey B Kirpichev em gh-116560 [https://github.com/python/cpython/issues/116560].)

- Adiciona `PyLong_IsPositive()`, `PyLong_IsNegative()` e `PyLong_IsZero()` para verificar se `PyLongObject` é positivo, negativo ou zero, respectivamente. (Contribuição de James Roy e Sergey B Kirpichev em gh-126061 [https://github.com/python/cpython/issues/126061].)

- Adiciona novas funções para converter números C `<stdint.h>` para/de objetos `int` Python:

  - `PyLong_AsInt32()`
  - `PyLong_AsInt64()`
  - `PyLong_AsUInt32()`
  - `PyLong_AsUInt64()`
  - `PyLong_FromInt32()`
  - `PyLong_FromInt64()`
  - `PyLong_FromUInt32()`
  - `PyLong_FromUInt64()`

  (Contribuição de Victor Stinner em gh-120389 [https://github.com/python/cpython/issues/120389].)

- Adiciona uma nova API de importação e exportação para objetos Python `int` (**PEP 757** [https://peps.python.org/pep-0757/]):

  - `PyLong_GetNativeLayout()`
  - `PyLong_Export()`
  - `PyLong_FreeExport()`
  - `PyLongWriter_Create()`
  - `PyLongWriter_Finish()`
  - `PyLongWriter_Discard()`

  (Contribuição de Sergey B Kirpichev e Victor Stinner em gh-102471 [https://github.com/python/cpython/issues/102471].)

- Adiciona `PyMonitoring_FireBranchLeftEvent()` e `PyMonitoring_FireBranchRightEvent()` para gerar eventos `BRANCH_LEFT` e `BRANCH_RIGHT`, respectivamente. (Contribuição de Mark Shannon em gh-122548 [https://github.com/python/cpython/issues/122548].)

- Adiciona a função `PyType_Freeze()` para tornar um tipo imutável. (Contribuição de Victor Stinner em gh-121654 [https://github.com/python/cpython/issues/121654].)

- Adiciona `PyType_GetBaseByToken()` e o slot `Py_tp_token` para facilitar a identificação da superclasse, o que tenta resolver o problema de verificação de tipos mencionado na **PEP 630** [https://peps.python.org/pep-0630/#type-checking]. (Contribuição em gh-124153 [https://github.com/python/cpython/issues/124153].)

- Adiciona a nova função `PyUnicode_Equal()` para testar se duas strings são iguais. A função também foi adicionada à API C Limitada. (Contribuição de Victor Stinner em gh-124502 [https://github.com/python/cpython/issues/124502].)

- Adiciona uma nova API `PyUnicodeWriter` para criar um objeto Python `str` com as seguintes funções:

    - `PyUnicodeWriter_Create()`
    - `PyUnicodeWriter_DecodeUTF8Stateful()`
    - `PyUnicodeWriter_Discard()`
    - `PyUnicodeWriter_Finish()`
    - `PyUnicodeWriter_Format()`
    - `PyUnicodeWriter_WriteASCII()`
    - `PyUnicodeWriter_WriteChar()`
    - `PyUnicodeWriter_WriteRepr()`
    - `PyUnicodeWriter_WriteStr()`
    - `PyUnicodeWriter_WriteSubstring()`
    - `PyUnicodeWriter_WriteUCS4()`
    - `PyUnicodeWriter_WriteUTF8()`
    - `PyUnicodeWriter_WriteWideChar()`

(Contribuição de Victor Stinner em gh-119182 [https://github.com/python/cpython/issues/119182].)

- Os formatos `k` e `K` em `PyArg_ParseTuple()` e funções similares agora usam `__index__()` se disponível, assim como todos os outros formatos inteiros. (Contribuição de Serhiy Storchaka em gh-112068 [https://github.com/python/cpython/issues/112068].)

- Adiciona suporte para uma nova unidade de formato `p` em `Py_BuildValue()` que produz um objeto `bool` Python de um inteiro C. (Contribuição de Pablo Galindo em bpo-45325 [https://bugs.python.org/issue?@action=redirect&bpo=45325].)

- Adiciona `PyUnstable_IsImmortal()` para determinar se um objeto é imortal, para fins de depuração. (Contribuição de Peter Bierma em gh-128509 [https://github.com/python/cpython/issues/128509].)

- Adiciona `PyUnstable_Object_EnableDeferredRefcount()` para habilitar a contagem de referências adiada, conforme descrito em **PEP 703** [https://peps.python.org/pep-0703/].

- Adiciona `PyUnstable_Object_IsUniquelyReferenced()` como substituto para `Py_REFCNT(op) == 1` em construções com threads livres. (Contribuição de Peter Bierma em gh-133140 [https://github.com/python/cpython/issues/133140].)

- Adiciona `PyUnstable_Object_IsUniqueReferencedTemporary()` para determinar se um objeto é um objeto temporário exclusivo na pilha de operandos do interpretador. Isso pode ser usado em alguns casos como um substituto para verificar se `Py_REFCNT()` é `1` para objetos Python passados como argumentos para funções da API C. (Contribuição de Sam Gross em gh-133164 [https://github.com/python/cpython/issues/133164].)

Mudanças na API C limitada

- Na API C limitada versão 3.14 e versões mais novas, `Py_TYPE()` e `Py_REFCNT()` agora são implementadas como uma chamada de função opaca para ocultar detalhes de implementação. (Contribuição de Victor

Stinner em gh-120600 [https://github.com/python/cpython/issues/120600] e gh-124127 [https://github.com/python/cpython/issues/124127].)

- Remove as macros `PySequence_Fast_GET_SIZE`, `PySequence_Fast_GET_ITEM` e `PySequence_Fast_ITEMS` da API C limitada, já que elas estiveram quebradas na API C limitada. (Contribuição de Victor Stinner em gh-91417 [https://github.com/python/cpython/issues/91417].)

APIs C removidas

- A criação de `tipos_imutáveis` com bases mutáveis foi descontinuada no Python 3.12, e agora levanta `TypeError`. (Contribuição de Nikita Sobolev em gh-119775 [https://github.com/python/cpython/issues/119775].)
- Remove o membro `PyDictObject.ma_version_tag`, que foi descontinuado no Python 3.12. Em vez disso, use a API `PyDict_AddWatcher()`. (Contribuição de Sam Gross em gh-124296 [https://github.com/python/cpython/issues/124296].)
- Remova a função privada `_Py_InitializeMain()`. Era uma API provisória adicionada ao Python 3.8 pela **PEP 587** [https://peps.python.org/pep-0587/]. (Contribuição de Victor Stinner em gh-129033 [https://github.com/python/cpython/issues/129033].)
- Remove as APIs não documentadas **`Py_C_RECURSION_LIMIT`** e **`PyThreadState.c_recursion_remaining`**. Elas foram adicionadas no 3.13 e foram removidas sem descontinuação. Use `Py_EnterRecursiveCall()` para se proteger contra recursão descontrolada em código C. (Remoção por Petr Viktorin em gh-133079 [https://github.com/python/cpython/issues/133079], veja também gh-130396 [https://github.com/python/cpython/issues/130396].)

APIs C descontinuadas

- A macro **`Py_HUGE_VAL`** está agora suavemente descontinuada. Em vez disso, use **`INFINITY`**. (Contribuição de Sergey B Kirpichev em gh-120026 [https://github.com/python/cpython/issues/120026].)
- As macros **`Py_IS_NAN`**, **`Py_IS_INFINITY`** e **`Py_IS_FINITE`** estão agora suavemente descontinuadas. Em vez delas, use **`isnan`**, **`isinf`** e

isfinite, disponíveis em `math.h` desde C99. (Contribuição de Sergey B Kirpichev em gh-119613 [https://github.com/python/cpython/issues/119613].)

- Sequências não-tuplas estão agora descontinuadas como argumento para a unidade de formato `(items)` em **PyArg_ParseTuple()** e outras funções de análise de argumentos se *items* contiver unidades de formato que armazenam um buffer emprestado ou uma referência emprestada. (Contribuição de Serhiy Storchaka em gh-50333 [https://github.com/python/cpython/issues/50333].)
- A função `_PyMonitoring_FireBranchEvent` está agora descontinuada e deve ser substituída por chamadas para **PyMonitoring_FireBranchLeftEvent()** e **PyMonitoring_FireBranchRightEvent()**.
- A função **PySequence_In()**, anteriormente não documentada, está agora suavemente descontinuado. Em vez disso, use **PySequence_Contains()**. (Contribuição de Yuki Kobayashi em gh-127896 [https://github.com/python/cpython/issues/127896].)

Remoção pendente no Python 3.15

- A **PyImport_ImportModuleNoBlock()**: Use **PyImport_ImportModule()**

- **PyWeakref_GetObject()** e **PyWeakref_GET_OBJECT()**: Use **PyWeakref_GetRef()**. O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pod ser usado para obter **PyWeakref_GetRef()** no Python 3.12 e versões mais antigas.

- **PyUnicode_AsDecodedObject()**: use **PyCodec_Decode()**.

- **PyUnicode_AsDecodedUnicode()**: Use **PyCodec_Decode()**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **str**, tal como **bytes**.

- **PyUnicode_AsEncodedObject()**: use **PyCodec_Encode()**.

- **PyUnicode_AsEncodedUnicode()**: Use **PyCodec_Encode()**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo

diferente de `bytes`, tal com `str`.

- Funções de inicialização do Python, descontinuadas no Python 3.9:

  - **`Py_GetPath()`**: Use `PyConfig_Get("module_search_paths")` (`sys.path`).
  - **`Py_GetPrefix()`**: Use `PyConfig_Get("base_prefix")` (`sys.base_prefix`). Use **`PyConfig_Get("prefix")`** (`sys.prefix`) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetExecPrefix()`**: Use `PyConfig_Get("base_exec_prefix")` (`sys.base_exec_prefix`). Use **`PyConfig_Get("exec_prefix")`** (`sys.exec_prefix`) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetProgramFullPath()`**: Use `PyConfig_Get("executable")` (`sys.executable`).
  - **`Py_GetProgramName()`**: Use `PyConfig_Get("executable")` (`sys.executable`).
  - **`Py_GetPythonHome()`**: Use `PyConfig_Get("home")` ou a variável de ambiente `PYTHONHOME`.

  O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter `PyConfig_Get()` no Python 3.13 e versões anteriores.

- Funções para configurar a inicialização do Python, descontinuadas no Python 3.11:

  - **`PySys_SetArgvEx()`**: defina `PyConfig.argv`.
  - **`PySys_SetArgv()`**: defina `PyConfig.argv`.
  - **`Py_SetProgramName()`**: defina `PyConfig.program_name`.
  - **`Py_SetPythonHome()`**: defina `PyConfig.home`.
  - **`PySys_ResetWarnOptions()`**: Limpe `sys.warnoptions` e **`warnings.filters`**.

  Em vez disso, a API `Py_InitializeFromConfig()` deve ser usada com `PyConfig`.

- Variáveis de configuração globais

- `Py_DebugFlag`: Use `PyConfig.parser_debug` ou `PyConfig_Get("parser_debug")`.
- `Py_VerboseFlag`: Use `PyConfig.verbose` ou `PyConfig_Get("verbose")`.
- `Py_QuietFlag`: Use `PyConfig.quiet` ou `PyConfig_Get("quiet")`.
- `Py_InteractiveFlag`: Use `PyConfig.interactive` ou `PyConfig_Get("interactive")`.
- `Py_InspectFlag`: Use `PyConfig.inspect` ou `PyConfig_Get("inspect")`.
- `Py_OptimizeFlag`: Use `PyConfig.optimization_level` ou `PyConfig_Get("optimization_level")`.
- `Py_NoSiteFlag`: Use `PyConfig.site_import` ou `PyConfig_Get("site_import")`.
- `Py_BytesWarningFlag`: Use `PyConfig.bytes_warning` ou `PyConfig_Get("bytes_warning")`.
- `Py_FrozenFlag`: Use `PyConfig.pathconfig_warnings` ou `PyConfig_Get("pathconfig_warnings")`.
- `Py_IgnoreEnvironmentFlag`: Use `PyConfig.use_environment` ou `PyConfig_Get("use_environment")`.
- `Py_DontWriteBytecodeFlag`: Use `PyConfig.write_bytecode` ou `PyConfig_Get("write_bytecode")`.
- `Py_NoUserSiteDirectory`: Use `PyConfig.user_site_directory` ou `PyConfig_Get("user_site_directory")`.
- `Py_UnbufferedStdioFlag`: Use `PyConfig.buffered_stdio` ou `PyConfig_Get("buffered_stdio")`.
- `Py_HashRandomizationFlag`: Use `PyConfig.use_hash_seed` e `PyConfig.hash_seed` ou `PyConfig_Get("hash_seed")`.
- `Py_IsolatedFlag`: Use `PyConfig.isolated` ou `PyConfig_Get("isolated")`.
- `Py_LegacyWindowsFSEncodingFlag`: Use `PyPreConfig.legacy_windows_fs_encoding` ou `PyConfig_Get("legacy_windows_fs_encoding")`.
- `Py_LegacyWindowsStdioFlag`: Use `PyConfig.legacy_windows_stdio` ou `PyConfig_Get("legacy_windows_stdio")`.

- **`Py_FileSystemDefaultEncoding`**, **`Py_HasFileSystemDefaultEncoding`**: Use `PyConfig.filesystem_encoding` ou `PyConfig_Get("filesystem_encoding")`.
- **`Py_FileSystemDefaultEncodeErrors`**: Use `PyConfig.filesystem_errors` ou `PyConfig_Get("filesystem_errors")`.
- **`Py_UTF8Mode`**: Use `PyPreConfig.utf8_mode` ou `PyConfig_Get("utf8_mode")`. (veja `Py_PreInitialize()`)

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig` para definir essas opções. Ou `PyConfig_Get()` pode ser usado para obter essas opções em tempo de execução.

Remoção pendente no Python 3.16

- A cópia empacotada do `libmpdec`.

Remoção pendente no Python 3.18

- As seguintes funções privadas estão descontinuado e planejadas para remoção no Python 3.18:
  - **`_PyBytes_Join()`**: use `PyBytes_Join()`.
  - **`_PyDict_GetItemStringWithError()`**: use `PyDict_GetItemStringRef()`.
  - **`_PyDict_Pop()`**: use `PyDict_Pop()`.
  - **`_PyLong_Sign()`**: use `PyLong_GetSign()`.
  - **`_PyLong_FromDigits()`** e **`_PyLong_New()`**: use `PyLongWriter_Create()`.
  - **`_PyThreadState_UncheckedGet()`**: use `PyThreadState_GetUnchecked()`.
  - **`_PyUnicode_AsString()`**: use `PyUnicode_AsUTF8()`.
  - **`_PyUnicodeWriter_Init()`**: substitua `_PyUnicodeWriter_Init(&writer)` por `writer = PyUnicodeWriter_Create(0)`.

- **`_PyUnicodeWriter_Finish()`**: substitua `_PyUnicodeWriter_Finish(&writer)` por **`PyUnicodeWriter_Finish(writer)`**.
- **`_PyUnicodeWriter_Dealloc()`**: substitua `_PyUnicodeWriter_Dealloc(&writer)` por **`PyUnicodeWriter_Discard(writer)`**.
- **`_PyUnicodeWriter_WriteChar()`**: substituta `_PyUnicodeWriter_WriteChar(&writer, ch)` por **`PyUnicodeWriter_WriteChar(writer, ch)`**.
- **`_PyUnicodeWriter_WriteStr()`**: substitua `_PyUnicodeWriter_WriteStr(&writer, str)` por **`PyUnicodeWriter_WriteStr(writer, str)`**.
- **`_PyUnicodeWriter_WriteSubstring()`**: substitua `_PyUnicodeWriter_WriteSubstring(&writer, str, start, end)` por **`PyUnicodeWriter_WriteSubstring(writer, str, start, end)`**.
- **`_PyUnicodeWriter_WriteASCIIString()`**: substitua `_PyUnicodeWriter_WriteASCIIString(&writer, str)` por **`PyUnicodeWriter_WriteASCII(writer, str)`**.
- **`_PyUnicodeWriter_WriteLatin1String()`**: substitua `_PyUnicodeWriter_WriteLatin1String(&writer, str)` por **`PyUnicodeWriter_WriteUTF8(writer, str)`**.
- **`_PyUnicodeWriter_Prepare()`**: (sem substituto).
- **`_PyUnicodeWriter_PrepareKind()`**: (sem substituto).
- **`_Py_HashPointer()`**: use **`Py_HashPointer()`**.
- **`_Py_fopen_obj()`**: use **`Py_fopen()`**.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas novas funções públicas no Python 3.13 e versões anteriores. (Contribuição de Victor Stinner em gh-128863 [https://github.com/python/cpython/issues/128863].)

Remoção pendente em versões futuras

As APIs a seguir foram descontinuadas e serão removidas, embora atualmente não haja uma data agendada para sua remoção.

- `Py_TPFLAGS_HAVE_FINALIZE`: desnecessária desde o Python 3.8.
- `PyErr_Fetch()`: use `PyErr_GetRaisedException()`.
- `PyErr_NormalizeException()`: use `PyErr_GetRaisedException()`.
- `PyErr_Restore()`: use `PyErr_SetRaisedException()`.
- `PyModule_GetFilename()`: use `PyModule_GetFilenameObject()`.
- `PyOS_AfterFork()`: use `PyOS_AfterFork_Child()`.
- `PySlice_GetIndicesEx()`: use `PySlice_Unpack()` e `PySlice_AdjustIndices()`.
- `PyUnicode_READY()`: desnecessário desde o Python 3.12
- `PyErr_Display()`: use `PyErr_DisplayException()`.
- `_PyErr_ChainExceptions()`: use `_PyErr_ChainExceptions1()`.
- O membro `PyBytesObject.ob_shash`: chame `PyObject_Hash()`.
- API do Thread Local Storage (TLS):
  - `PyThread_create_key()`: use `PyThread_tss_alloc()`.
  - `PyThread_delete_key()`: use `PyThread_tss_free()`.
  - `PyThread_set_key_value()`: use `PyThread_tss_set()`.
  - `PyThread_get_key_value()`: use `PyThread_tss_get()`.
  - `PyThread_delete_key_value()`: use `PyThread_tss_delete()`.
  - `PyThread_ReInitTLS()`: desnecessário desde o Python 3.7.

Alterações na construção

- **PEP 776** [https://peps.python.org/pep-0776/]: O Emscripten agora é uma plataforma oficialmente suportada no **nível 3** [https://peps.python.org/pep-0011/#tier-3]. Como parte desse esforço, mais de 25 bugs na libc do Emscripten [https://emscripten.org/docs/porting/emscripten-runtime-environment.html] foram corrigidos. O Emscripten agora inclui suporte para `ctypes`, `termios` e `fcntl`, bem como suporte experimental para o novo console interativo padrão. (Contribuição de R. Hood Chatham em gh-127146 [https://github.com/python/cpython/issues/127146], gh-127683 [https://github.com/python/cpython/issues/127683] e gh-136931 [https://github.com/python/cpython/issues/136931].)
- Lançamentos oficiais de binários para Android são agora disponibilizados em python.org [https://www.python.org/downloads/android/].
- O GNU Autoconf 2.72 agora é necessário para gerar o `configure`. (Contribuição de Erlend Aasland em gh-115765 [https://github.com/python/cpython/issues/115765].)

- `wasm32-unknown-emscripten` agora é uma plataforma de nível 3 segundo a **PEP 11** [https://peps.python.org/pep-0011/]. (Contribuição de R. Hood Chatham em gh-127146 [https://github.com/python/cpython/issues/127146], gh-127683 [https://github.com/python/cpython/issues/127683] e gh-136931 [https://github.com/python/cpython/issues/136931].)
- A vinculação baseada em `#pragma` com `python3*.lib` agora pode ser desativada com Py_NO_LINK_LIB. (Contribuição de Jean-Christophe Fillion-Robin em gh-82909 [https://github.com/python/cpython/issues/82909].)
- O CPython agora habilita um conjunto de opções de compilador recomendadas por padrão para maior segurança. Use as opções **`--disable-safety`** e `configure` para desabilitá-las, ou a opção **`--enable-slower-safety`** para um conjunto maior de opções de compilador, embora com um custo de desempenho.
- A macro `WITH_FREELISTS` e a opção `--without-freelists` `configure` foram removidas.
- A nova opção `configure` **`--with-tail-call-interp`** pode ser usada para habilitar o interpretador experimental de chamada de cauda. Consulte Um novo tipo de interpretador para mais detalhes.
- Para desabilitar o novo suporte à depuração remota, use as opções **`--without-remote-debug`** e `configure`. Isso pode ser útil por motivos de segurança.
- Aplicações iOS e macOS agora podem ser configurados para redirecionar o conteúdo `stdout` e `stderr` para o log do sistema. (Contribuição de Russell Keith-Magee em gh-127592 [https://github.com/python/cpython/issues/127592].)
- O ambiente de teste do iOS agora pode transmitir a saída do teste enquanto ele está em execução. O ambiente de teste também pode ser usado para executar o conjunto de testes de outros projetos além do CPython. (Contribuição de Russell Keith-Magee em gh-127592 [https://github.com/python/cpython/issues/127592].)

`build-details.json`

As instalações do Python agora contêm um novo arquivo, `build-details.json`. Este é um documento JSON estático contendo detalhes da construção do CPython, permitindo introspecção sem a necessidade de

executar código. Isso é útil para casos de uso como inicializadores do Python, compilação cruzada e assim por diante.

`build-details.json` deve ser instalado no diretório da biblioteca padrão independente de plataforma. Isso corresponde ao caminho de instalação 'stdlib' **`sysconfig`**, que pode ser encontrado executando `sysconfig.get_path('stdlib')`.

Ver também

PEP 739 [https://peps.python.org/pep-0739/] – `build-details.json` 1.0 – um arquivo de descrição estática para detalhes de construção do Python

Descontinuação de assinaturas PGP

Assinaturas PGP (Pretty Good Privacy) não serão fornecidas para versões do Python 3.14 ou versões futuras. Para verificar artefatos do CPython, os usuários devem usar materiais de verificação do Sigstore [https://www.python.org/downloads/metadata/sigstore/]. As versões são assinadas usando o Sigstore [https://www.sigstore.dev/] desde o Python 3.11.

Essa mudança no processo de lançamento foi especificada em **PEP 761** [https://peps.python.org/pep-0761/].

Python com threads livres é oficialmente suportado

A construção do Python com threads livres agora é suportada e não mais experimental. Este é o início da fase II [https://discuss.python.org/t/37075], na qual o Python com threads livres é oficialmente suportado, mas ainda é opcional.

A equipe do projeto de threads livres confiantes de que ele está no caminho certo e agradece a dedicação contínua de todos que trabalham para deixar o modo threads livres pronto para uma adoção mais ampla na comunidade Python.

Com essas recomendações e a aceitação desta PEP, a comunidade de desenvolvedores do Python deve anunciar amplamente que threads livres é uma opção de construção do Python suportada agora e no futuro, e que ela não será removida sem um cronograma de descontinuação adequado.

Qualquer decisão de transição para a se III [https://discuss.python.org/t/37075], com o uso de threads livres como versão padrão ou única do Python, ainda está indefinida e depende de muitos fatores, tanto dentro do próprio CPython quanto da comunidade. Esta decisão é para o futuro.

Ver também

PEP 779 [https://peps.python.org/pep-0779/]

Aceitação da PEP 779 [https://discuss.python.org/t/84319/123]

Versões binárias para o compilador experimental just-in-time

Os binários oficiais de lançamento para macOS e Windows agora incluem um compilador JIT (just-in-time) *experimental*. Embora **não** seja recomendado para uso em produção, ele pode ser testado definindo `PYTHON_JIT=1` como uma variável de ambiente. Construções de código-fonte downstream e redistribuidores podem usar a opção de configuração `--enable-experimental-jit=yes-off` para obter um comportamento semelhante.

O JIT está em estágio inicial e ainda em desenvolvimento ativo. Portanto, o impacto típico de sua ativação no desempenho pode variar de 10% mais lento a 20% mais rápido, dependendo da carga de trabalho. Para auxiliar nos testes e na avaliação, um conjunto de funções de introspecção foi fornecido no espaço de nomes `sys._jit`. `sys._jit.is_available()` pode ser usado para determinar se o executável atual oferece suporte a compilação JIT, enquanto `sys._jit.is_enabled()` pode ser usado para informar se a compilação JIT foi ativada para o processo atual.

Atualmente, a funcionalidade ausente mais significativa é que depuradores e profilers nativos, como `gdb` e `perf`, não conseguem desenrolar quadros JIT (depuradores e profilers Python, como `pdb` ou `profile`, continuam funcionando sem modificações). Construções com threads livres não oferecem suporte a compilação JIT.

Por favor, relate quaisquer bugs ou grandes regressões de desempenho que você encontrar!

Ver também

PEP 744 [https://peps.python.org/pep-0744/]

Portando para o Python 3.14

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações na API Python

- Em plataformas Unix diferentes do macOS, *forkserver* é agora o método de início padrão para `multiprocessing` e `ProcessPoolExecutor`, em vez de *fork*.

  Consulte (1) e (2) para detalhes.

  Se você encontrar `NameError`s ou erros de serialização com pickle vindos de `multiprocessing` ou `concurrent.futures`, consulte as restrições do forkserver.

  Esta alteração não afeta o Windows ou macOS, onde 'spawn' continua a ser o método de início padrão.

- `functools.partial` agora é um descritor de método. Envolva-o em `staticmethod()` se quiser preservar o comportamento antigo.

(Contribuição de Serhiy Storchaka e Dominykas Grigonis em gh-121027 [https://github.com/python/cpython/issues/121027].)

- O coletor de lixo agora é incremental, o que significa que o comportamento de **`gc.collect()`** muda ligeiramente:
  - `gc.collect(1)`: executa um incremento da coleta de lixo, em vez de coletar a geração 1.
  - Outras chamadas para **`gc.collect()`** não foram alteradas.

- A função **`locale.nl_langinfo()`** agora define temporariamente a localidade `LC_CTYPE` em alguns casos. Esta alteração temporária afeta outras threads. (Contribuição de Serhiy Storchaka em gh-69998 [https://github.com/python/cpython/issues/69998].)

- **`pickle.dump()`** e **`pickle.dumps()`** agora levantam **`PicklingError`** para algumas falhas que anteriormente levantavam **`AttributeError`**, **`ImportError`**, **`ValueError`**, **`UnicodeEncodeError`** ou **`PicklingError`**, dependendo da implementação (por exemplo, ao serializar com pickle um objeto local ou um objeto cujo módulo não pode ser importado). A exceção original é encadeada à **`PicklingError`**. Códigos que capturavam essas exceções também devem capturar **`PicklingError`**. (Contribuição de Serhiy Storchaka em gh-122311 [https://github.com/python/cpython/issues/122311].)

- **`types.UnionType`** agora é um apelido para **`typing.Union`**, causando alterações em alguns comportamentos. Veja acima para mais detalhes. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)

- O comportamento das anotações em tempo de execução mudou de várias maneiras; veja acima para mais detalhes. Embora a maioria dos códigos que interagem com anotações deva continuar funcionando, alguns detalhes não documentados podem se comportar de forma diferente.

- Como parte da implementação pública da CLI do **`mimetypes`**, ela agora retorna `1` em caso de falha, em vez de `0`, e `2` em caso de parâmetros

incorretos na linha de comando, em vez de `1`. As mensagens de erro agora são impressas no stderr.

- O padrão `\B` em expressões regulares agora corresponde à string vazia quando fornecido como o padrão completo, o que pode causar alterações de comportamento.

- No FreeBSD, o arquivo `sys.platform` não contém mais o número da versão principal.

Mudanças em anotações (PEP 649 [https://peps.python.org/pep-0649/] e PEP 749 [https://peps.python.org/pep-0749/])

Esta seção contém orientações sobre as alterações que podem ser necessárias nas anotações ou no código Python que interage com ou introspecta anotações, devido às alterações relacionadas à avaliação adiada de anotações.

Na maioria dos casos, o código funcionando em uma versão mais antiga de Python não exigirá nenhuma alteração.

Implicações para o código anotado

Se você definir anotação em seu código (por exemplo, para uso com um verificador de tipo estático), essa alteração provavelmente não o afetará: você pode continuar escrevendo anotação da mesma forma que fazia com as versões anteriores de Python.

É provável que você consiga remover as strings entre aspas em anotações, frequentemente usadas para referências futuras. Da mesma forma, se você usar `from __future__ import annotations` para evitar ter que escrever strings em anotações, poderá remover essa importação se for manter seu código apenas para Python 3.14 e versões mais recentes. No entanto, se você depende de bibliotecas de terceiros que leem anotações, essas bibliotecas podem precisar de alterações no suporte a anotações sem aspas para funcionarem como esperado.

Implicações para leitores de `__annotations__`

Se o seu código lê o atributo `__annotations__` dos objetos, talvez você queira fazer alterações para oferecer suporte a código que depende da avaliação adiada de anotações. Por exemplo, talvez você queira usar `annotationlib.get_annotations()` no formato `FORWARDREF`, como o módulo `dataclasses` faz agora.

O pacote externo typing_extensions [https://pypi.org/project/typing_extensions/] fornece compatibilidade retroativa parcial de algumas das funcionalidades do módulo `annotationlib`, como a enumeração `Format` e a função `get_annotations()`. Eles podem ser usados para escrever código com versão-cruzada que aproveite as vantagens do novo comportamento em Python 3.14.

Mudanças relacionadas

As alterações em Python 3.14 foram projetadas para retrabalhar a forma como `__annotations__` funciona em tempo de execução e, ao mesmo tempo, minimizar a quebra do código que contém anotações em código-fonte e do código que lê `__annotations__`. No entanto, se você confiar em detalhes não documentados do comportamento de anotações ou em funções privadas na biblioteca padrão, há muitas maneiras pelas quais seu código pode não funcionar em Python 3.14. Para proteger seu código contra alterações futuras, use apenas as funcionalidades documentadas do módulo `annotationlib`.

Em particular, não leia anotação diretamente do atributo dicionário com o espaço de nomes dos objetos de tipo. Use `annotationlib.get_annotate_from_class_namespace()` durante a construção da classe e `annotationlib.get_annotations()` depois.

Em versões anteriores, às vezes era possível acessar as anotações da classe a partir de uma instância de uma classe anotada. Esse comportamento não estava documentado e era acidental e não será mais possível em Python 3.14.

```
from __future__ import annotations
```

Em Python 3.7, **PEP 563** [https://peps.python.org/pep-0563/] introduziu a instrução future `from __future__ import annotations`, que transforma todas as anotações em string.

No entanto, essa instrução está agora descontinuada e espera-se que seja removida em uma versão futura de Python. Essa remoção não ocorrerá até que o Python 3.13 atinja seu fim de vida útil em 2029, sendo a última versão de Python sem suporte para avaliação adiada de anotações.

Em Python 3.14, o comportamento do código que usa `from __future__ import annotations` não foi alterado.

Alterações na API C

- **`Py_Finalize()`** agora exclui todas as strings internalizadas. Isso é incompatível com versões anteriores de qualquer extensão C que retenha uma string internada após uma chamada para **`Py_Finalize()`** e seja reutilizada após uma chamada para **`Py_Initialize()`**. Quaisquer problemas decorrentes desse comportamento normalmente resultarão em travamentos durante a execução da chamada subsequente para **`Py_Initialize()`** ao acessar memória não inicializada. Para corrigir, use um sanitizador de endereços para identificar qualquer uso após liberação proveniente de uma string internalizada e desalocá-la durante o desligamento do módulo. (Contribuição de Eddie Elizondo em gh-113601 [https://github.com/python/cpython/issues/113601].)
- A API C objetos de exceção Unicode agora levanta **`TypeError`** se o argumento de exceção não for um objeto **`UnicodeError`**. (Contribuição de Bénédikt Tran em gh-127691 [https://github.com/python/cpython/issues/127691].)

- O interpretador evita internamente algumas modificações na contagem de referências ao carregar objetos na pilha de operandos, pegando emprestadas referências quando possível. Isso pode levar a valores menores de contagem de referências em comparação com versões anteriores do Python. Extensões da API C que verificavam

`Py_REFCNT()` de `1` para determinar se um argumento de função não é referenciado por nenhum outro código devem, em vez disso, usar `PyUnstable_Object_IsUniqueReferencedTemporary()` como uma substituição mais segura.

- Funções privadas promovidas para APIs C públicas:
  - `_PyBytes_Join()`: `PyBytes_Join()`
  - `_PyLong_IsNegative()`: `PyLong_IsNegative()`
  - `_PyLong_IsPositive()`: `PyLong_IsPositive()`
  - `_PyLong_IsZero()`: `PyLong_IsZero()`
  - `_PyLong_Sign()`: `PyLong_GetSign()`
  - `_PyUnicodeWriter_Dealloc()`: `PyUnicodeWriter_Discard()`
  - `_PyUnicodeWriter_Finish()`: `PyUnicodeWriter_Finish()`
  - `_PyUnicodeWriter_Init()`: use `PyUnicodeWriter_Create()`
  - `_PyUnicodeWriter_Prepare()`: (sem substituto)
  - `_PyUnicodeWriter_PrepareKind()`: (sem substituto)
  - `_PyUnicodeWriter_WriteChar()`: `PyUnicodeWriter_WriteChar()`
  - `_PyUnicodeWriter_WriteStr()`: `PyUnicodeWriter_WriteStr()`
  - `_PyUnicodeWriter_WriteSubstring()`: `PyUnicodeWriter_WriteSubstring()`
  - `_PyUnicode_EQ()`: `PyUnicode_Equal()`
  - `_PyUnicode_Equal()`: `PyUnicode_Equal()`
  - `_Py_GetConfig()`: `PyConfig_Get()` e `PyConfig_GetInt()`
  - `_Py_HashBytes()`: `Py_HashBuffer()`
  - `_Py_fopen_obj()`: `Py_fopen()`
  - `PyMutex_IsLocked()` : `PyMutex_IsLocked()`

  O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter a maioria dessas novas funções no Python 3.13 e versões anteriores.

Alterações notáveis no 3.14.5

gc

- O coletor de lixo incremental incluído nas versões Python 3.14.0 a 3.14.4 foi revertido para o coletor de lixo geracional da versão 3.13, devido a diversos relatos [https://github.com/python/cpython/issues/142516] de pressão significativa sobre a memória em ambientes de produção. Consulte Coleta de lixo para obter detalhes.

O que há de novo no Python 3.13

Editores:: Adam Turner e Thomas Wouters

Este artigo explica os novos recursos no Python 3.13, em comparação com 3.12. Python 3.13 foi lançado em 7 de outubro de 2024. Veja changelog para uma lista completa de mudanças.

Ver também

PEP 719 [https://peps.python.org/pep-0719/] – Cronograma de lançamento do Python 3.13

Resumo – Destaques da versão

O Python 3.13 é uma versão estável da linguagem de programação Python, com uma mistura de mudanças na linguagem, na implementação e na biblioteca padrão. As maiores mudanças incluem um novo interpretador interativo, suporte experimental para execução em um modo de threads livres (**PEP 703** [https://peps.python.org/pep-0703/]) e um compilador Just-In-Time (**PEP 744** [https://peps.python.org/pep-0744/]).

As mensagens de erro continuam a melhorar, com tracebacks (situação da pilha de execução) agora realçados em cores por padrão. A função embutida `locals()` agora tem semânticas definidas para alterar o mapeamento retornado, e os parâmetros de tipo agora oferecem suporte a valores padrão.

As alterações da biblioteca contêm a remoção de APIs e módulos descontinuados, bem como as melhorias usuais em facilidade de uso e correção. Vários módulos da biblioteca padrão legados agora foram removidos após sua descontinuação no Python 3.11 (**PEP 594** [https://peps.python.org/pep-0594/]).

Este artigo não tenta fornecer uma especificação completa de todos os novos recursos, mas fornece uma visão geral conveniente. Para detalhes completos, consulte a documentação, como Referência da Biblioteca e Referência da Linguagem. Para entender a implementação completa e a justificativa do design para uma mudança, consulte a PEP para um novo recurso específico; mas observe que as PEPs geralmente não são mantidas atualizadas depois que um recurso é totalmente implementado. Veja Portando para o Python 3.13 para orientação sobre atualização a partir de versões anteriores do Python.

Melhorias no interpretador:

- Um interpretador interativo bastante aprimorado e mensagens de erro aprimoradas.
- **PEP 667** [https://peps.python.org/pep-0667/]: a função embutida `locals()` agora tem semântica definida ao fazer a mutação do mapeamento retornado. Os depuradores do Python e ferramentas semelhantes agora podem atualizar variáveis locais de maneira mais confiável em escopos otimizados, mesmo durante a execução simultânea de código.
- **PEP 703** [https://peps.python.org/pep-0703/]: CPython 3.13 tem suporte experimental para ser executado com a trava global do interpretador, ou GIL, desabilitada. Veja CPython com threads livres para mais detalhes.
- **PEP 744** [https://peps.python.org/pep-0744/]: um compilador JIT básico foi adicionado. Atualmente está desativado por padrão (embora possamos ativá-lo mais tarde). As melhorias de desempenho são modestas – esperamos melhorar isso nas próximas versões.
- Suporte a cores no novo interpretador interativo, bem como na saída de tracebacks e de doctest. Isso pode ser desabilitado através das variáveis de ambiente `PYTHON_COLORS` e `NO_COLOR` [https://no-color.org/].

Melhorias no modelo de dados Python:

- `__static_attributes__` armazena os nomes dos atributos acessados por meio de `self.x` em qualquer função no corpo da classe.
- `__firstlineno__` registra o número da primeira linha da definição de classe.

Melhorias significativas na biblioteca padrão:

- Adiciona uma nova exceção `PythonFinalizationError`, levantada quando uma operação é bloqueada durante a finalização.
- O módulo `argparse` agora oferece suporte a descontinuar opções de linha de comando, argumentos posicionais e subcomandos.
- As novas funções `base64.z85encode()` e `base64.z85decode()` oferecem suporte à codificação e decodificação de dados Z85 [https://rfc.zeromq.org/spec/32/].
- O módulo `copy` agora tem uma função `copy.replace()`, com suporte para muitos tipos embutidos e qualquer classe que defina o método `__replace__()`.
- O novo módulo `dbm.sqlite3` agora é o backend `dbm` padrão.
- O módulo `os` tem um conjunto de novas funções para trabalhar com os descritores de arquivo de notificação de temporizador do Linux.
- O módulo `random` agora tem uma interface de linha de comando.

Melhorias de segurança:

- `ssl.create_default_context()` define `ssl.VERIFY_X509_PARTIAL_CHAIN` e `ssl.VERIFY_X509_STRICT` como sinalizadores padrão.

Melhorias na API C:

- O slot `Py_mod_gil` agora é usado para indicar que um módulo de extensão oferece suporte à execução com a GIL desabilitada.
- A API C PyTime foi adicionada, fornecendo acesso aos relógios do sistema.
- `PyMutex` é um novo mutex leve que ocupa um único byte.
- Há um novo conjunto de funções para gerar eventos de monitoramento da **PEP 669** [https://peps.python.org/pep-0669/] na API C.

Novos recursos de tipagem:

- **PEP 696** [https://peps.python.org/pep-0696/]: parâmetros de tipo (`typing.TypeVar`, `typing.ParamSpec` e `typing.TypeVarTuple`) agora oferecem suporte a valores padrão.

- **PEP 702** [https://peps.python.org/pep-0702/]: o novo decorador `warnings.deprecated()` adiciona suporte para marcar descontinuações no sistema de tipos e ambiente de execução.
- **PEP 705** [https://peps.python.org/pep-0705/]: `typing.ReadOnly` pode ser usado para marcar um item de `typing.TypedDict` como somente leitura para verificadores de tipo.
- **PEP 742** [https://peps.python.org/pep-0742/]: `typing.TypeIs` fornece um comportamento de estreitamento de tipo mais intuitivo, como uma alternativa ao `typing.TypeGuard`.

Suporte à plataforma:

- **PEP 730** [https://peps.python.org/pep-0730/]: iOS da Apple agora é uma plataforma oficialmente com suporte, no **tier 3** [https://peps.python.org/pep-0011/#tier-3].
- **PEP 738** [https://peps.python.org/pep-0738/]: Android é agora uma plataforma oficialmente com suporte, no **tier 3** [https://peps.python.org/pep-0011/#tier-3].
- `wasm32-wasi` agora é uma plataforma com suporte **tier 2** [https://peps.python.org/pep-0011/#tier-2].
- `wasm32-emscripten` não é mais uma plataforma oficialmente com suporte.

Remoções importantes:

- PEP 594: as 19 “baterias descarregadas” (módulos legados da biblioteca padrão) restantes foram removidas da biblioteca padrão: **`aifc`**, **`audioop`**, **`cgi`**, **`cgitb`**, **`chunk`**, **`crypt`**, **`imghdr`**, **`mailcap`**, **`msilib`**, **`nis`**, **`nntplib`**, **`ossaudiodev`**, **`pipes`**, **`sndhdr`**, **`spwd`**, **`sunau`**, **`telnetlib`**, **`uu`** e **`xdrlib`**.
- Remove a ferramenta **2to3** e o módulo **`lib2to3`** (descontinuados no Python 3.11).
- Remove o módulo **`tkinter.tix`** (descontinuado no Python 3.6).
- Remove a função **`locale.resetlocale()`**.
- Remove os espaços de nomes **`typing.io`** e **`typing.re`**.
- Remove descritores encadeados de **`classmethod`**.

Mudanças no cronograma de lançamento:

PEP 602 [https://peps.python.org/pep-0602/] ("Ciclo de Lançamento Anual para Python") foi atualizado para estender o período de suporte total ("bugfix") para novos lançamentos para dois anos. Esta política atualizada significa que:

- O Python 3.9–3.12 tem um ano e meio de suporte total, seguido por três anos e meio de correções de segurança.
- Python 3.13 e posteriores têm dois anos de suporte total, seguidos de três anos de correções de segurança.

Novas funcionalidades

Um melhor interpretador interativo

Python agora usa um novo console interativo por padrão, com base no código do projeto PyPy [https://pypy.org/]. Quando o usuário inicia o REPL de um terminal interativo, agora os seguintes novos recursos estão disponíveis:

- Edição multilinha com preservação do histórico.
- Suporte direto para comandos específicos do REPL, como `help`, `exit` e `quit`, sem a necessidade de chamá-los como funções.
- Prompts e tracebacks com cores habilitadas por padrão.
- Ajuda interativa para navegar usando `F1` com um histórico de comandos separado.
- Navegação no histórico usando `F2` que ignora a saída, bem como os prompts `>>>` e `...`.
- "Modo de colagem" com `F3` que facilita colar blocos de código maiores (pressione `F3` novamente para retornar ao prompt normal).

Para desabilitar o novo console interativo, defina a variável de ambiente **`PYTHON_BASIC_REPL`**. Para mais informações sobre o modo interativo, veja Modo interativo.

(Contribuição de Pablo Galindo Salgado, Łukasz Langa e Lysandros Nikolaou em gh-111201 [https://github.com/python/cpython/issues/111201] baseado no código do projeto PyPy. Suporte ao Windows foi uma contribuição de Dino Viehland e Anthony Shaw.)

Mensagens de erro melhoradas

- O interpretador agora usa cores por padrão ao exibir tracebacks no terminal. Este recurso pode ser controlado por meio da nova variável de ambiente **`PYTHON_COLORS`**, bem como das variáveis de ambiente canônicas **`NO_COLOR`** [https://no-color.org/] e **`FORCE_COLOR`** [https://force-color.org/]. (Contribuição de Pablo Galindo Salgado em gh-112730 [https://github.com/python/cpython/issues/112730].)

- Um erro comum é escrever um script com o mesmo nome de um módulo de biblioteca padrão. Quando isso resulta em erros, agora exibimos uma mensagem de erro mais útil:

```
$ python random.py
Traceback (most recent call last):
  File "/home/me/random.py", line 1, in <module>
    import random
  File "/home/me/random.py", line 3, in <module>
    print(random.randint(5))
          ^^^^^^^^^^^^^^^
AttributeError: module 'random' has no attribute 'randint' (
```

  Da mesma forma, se um script tiver o mesmo nome de um módulo de terceiros que ele tentar importar e isso resultar em erros, também exibiremos uma mensagem de erro mais útil:

```
$ python numpy.py
Traceback (most recent call last):
  File "/home/me/numpy.py", line 1, in <module>
    import numpy as np
  File "/home/me/numpy.py", line 3, in <module>
    np.array([1, 2, 3])
    ^^^^^^^^
AttributeError: module 'numpy' has no attribute 'array' (cor
```

  (Contribuição de Shantanu Jain em gh-95754 [https://github.com/python/cpython/issues/95754].)

- A mensagem de erro agora tenta sugerir o argumento nomeado correto quando um argumento nomeado incorreto é passado para uma função.

```
>>> "Better error messages!".split(max_split=1)
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    "Better error messages!".split(max_split=1)
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^
TypeError: split() got an unexpected keyword argument 'max_s
```

(Contribuição de Pablo Galindo Salgado e Shantanu Jain em gh-107944 [https://github.com/python/cpython/issues/107944].)

CPython com threads livres

O CPython agora tem suporte experimental para execução em modo de threads livres, com a trava global do interpretador (GIL) desabilitada. Este é um recurso experimental e, portanto, não é habilitado por padrão. O modo de threads livres requer um executável diferente, geralmente chamado de `python3.13t` ou `python3.13t.exe`. Binários pré-construídos marcados como *free-threaded* podem ser instalados como parte dos instaladores oficiais Windows e macOS, ou o CPython pode ser construído a partir da fonte com a opção `--disable-gil`.

A execução de threads livres permite a utilização total do poder de processamento disponível ao executar threads em paralelo nos núcleos de CPU disponíveis. Embora nem todos os softwares se beneficiem disso automaticamente, programas projetados com threading em mente serão executados mais rapidamente em hardware multi-core. **O modo de threads livres é experimental** e o trabalho está em andamento para melhorá-lo: espere alguns bugs e um impacto substancial no desempenho de thread único. As construções com threads livres do CPython oferecem suporte à execução opcional com a GIL habilitada no tempo de execução usando a variável de ambiente `PYTHON_GIL` ou a opção de linha de comando `-X gil=1`.

Para verificar se o interpretador atual oferece suporte a threads livres, `python -VV` e `sys.version` contêm “experimental free-threading build”. A

nova função **`sys._is_gil_enabled()`** pode ser usada para verificar se a GIL está realmente desabilitada no processo em execução.

Os módulos de extensão da API C precisam ser construídos especificamente para a construção com threads livres. Extensões que oferecem suporte à execução com GIL desabilitada devem usar o slot **`Py_mod_gil`**. Extensões que usam inicialização monofásica devem usar **`PyUnstable_Module_SetGIL()`** para indicar se oferecem suporte à execução com a GIL desabilitada. A importação de extensões C que não usam esses mecanismos fará com que a GIL seja habilitada, a menos que a GIL tenha sido explicitamente desabilitada com a variável de ambiente **`PYTHON_GIL`** ou a opção **`-X gil=0`**. pip 24.1 ou mais recentes é necessário para instalar pacotes com extensões C na construção com threads livres.

Este trabalho foi possível graças a muitos indivíduos e organizações, incluindo a grande comunidade de colaboradores do Python e projetos de terceiros para testar e habilitar o suporte para threads livres. Colaboradores notáveis incluem: Sam Gross, Ken Jin, Donghee Na, Itamar Oren, Matt Page, Brett Simmers, Dino Viehland, Carl Meyer, Nathan Goldbaum, Ralf Gommers, Lysandros Nikolaou e muitos outros. Muitos desses colaboradores são empregados pela Meta, que forneceu recursos de engenharia significativos para dar suporte a este projeto.

Ver também

PEP 703 [https://peps.python.org/pep-0703/] “Tornando a trava global do interpretador opcional no CPython” contém justificativa e informações sobre este trabalho.

Portando módulos de extensão para oferecer suporte a threads livres [https://py-free-threading.github.io/porting/]: um guia de portabilidade mantido pela comunidade para autores de extensões.

Um compilador just-in-time (JIT) experimental

Quando o CPython é configurado e construído usando a opção **`--enable-experimental-jit`**, um compilador just-in-time (JIT) é adicionado, o que pode acelerar alguns programas Python. No Windows, use `PCbuild/build.bat --experimental-jit` para habilitar o JIT ou `--experimental-jit-interpreter` para habilitar o interpretador Tier 2. Os requisitos de construção e outras informações de suporte estão contidos em [https://github.com/python/cpython/blob/main/Tools/jit/README.md] `Tools/jit/README.md`.

A opção **`--enable-experimental-jit`** aceita esses valores (opcionais), assumindo como padrão `yes` se **`--enable-experimental-jit`** estiver presente sem o valor opcional.

- `no`: desativa todo o pipeline de Tier 2 e JIT.
- `yes`: habilita o JIT. Para desabilitar o JIT em tempo de execução, passe a variável de ambiente `PYTHON_JIT=0`.
- `yes-off`: constrói o JIT, mas desabilita-o por padrão. Para habilitar o JIT em tempo de execução, passe a variável de ambiente `PYTHON_JIT=1`.
- `interpreter`: habilita o interpretador Tier 2, mas desabilita o JIT. O interpretador pode ser desabilitado executando com `PYTHON_JIT=0`.

A arquitetura interna é aproximadamente a seguinte:

- Começamos com especializado *bytecode de Tier 1*. Veja O que há de novo no 3.11 para detalhes.
- Quando o bytecode Tier 1 fica quente o suficiente, ele é traduzido para uma nova representação intermediária (IR) puramente interna, também chamado de *IR Tier 2*, e às vezes chamada de micro-ops (“uops”).
- O IR Tier 2 usa a mesma máquina virtual baseada em pilha que o Tier 1, mas o formato de instrução é mais adequado para tradução em código de máquina.
- Temos vários passes de otimização para IR Tier 2, que são aplicados antes de serem interpretados ou traduzidos em código de máquina.
- Existe um interpretador de Tier 2, mas ele se destina principalmente à depuração dos estágios iniciais do pipeline de otimização. O interpretador Tier 2 pode ser habilitado configurando o Python com `--enable-experimental-jit=interpreter`.

- Quando o JIT está habilitado, o IR Tier 2 otimizado é traduzido em código de máquina, que é então executado.
- O processo de tradução de código de máquina usa uma técnica chamada *copiar e corrigir*. Não possui dependências de tempo de execução, mas há uma nova dependência de tempo de construção no LLVM.

Ver também

PEP 744 [https://peps.python.org/pep-0744/]

(JIT de Brandt Bucher, inspirado em um artigo de Haoran Xu e Fredrik Kjolstad. Tier 2 IR de Mark Shannon e Guido van Rossum. Otimizador Tier 2 de Ken Jin.)

Definidas semânticas de mutação para `locals()`

Historicamente, o resultado esperado da mutação do valor de retorno de `locals()` foi deixada para as implementações individuais de Python definirem. A partir do Python 3.13, **PEP 667** [https://peps.python.org/pep-0667/] padroniza o comportamento histórico do CPython para a maioria dos escopos de execução de código, mas altera escopos otimizados (funções, geradores, corrotinas, compreensões e expressões geradoras) para retornar explicitamente instantâneos independentes das variáveis locais atualmente atribuídas, incluindo variáveis não locais referenciadas localmente capturadas em encerramentos.

Esta mudança na semântica de `locals()` em escopos otimizados também afeta o comportamento padrão das funções de execução de código que visam implicitamente `locals()` se nenhum espaço de nomes explícito for fornecido (como `exec()` e `eval()`). Nas versões anteriores, se as alterações podiam ou não ser acessadas chamando `locals()` após chamar a função de execução de código dependia da implementação. Especificamente no CPython, esse código normalmente parece funcionar conforme desejado, mas às vezes pode falhar em escopos otimizados com base em outro código (incluindo depuradores e ferramentas de rastreamento de execução de

código), redefinindo potencialmente o instantâneo compartilhado nesse escopo. Agora, o código sempre será executado em um instantâneo independente das variáveis locais em escopos otimizados e, portanto, as alterações nunca serão visíveis em chamadas subsequentes para **`locals()`**. Para acessar as alterações feitas nesses casos, uma referência explícita de espaço de nomes deve agora ser passada para a função relevante. Alternativamente, pode fazer sentido atualizar o código afetado para usar uma API de execução de código de nível mais alto que retorne o espaço de nomes de execução de código resultante (por exemplo, **`runpy.run_path()`** ao executar arquivos Python do disco).

Para garantir que depuradores e ferramentas semelhantes possam atualizar de forma confiável variáveis locais em escopos afetados por esta mudança, **`FrameType.f_locals`** agora retorna um proxy write-through para as variáveis locais e não locais referenciadas localmente do quadro nesses escopos, em vez de retornar uma instância `dict` compartilhada atualizada inconsistentemente com semântica de tempo de execução indefinida.

Veja **PEP 667** [https://peps.python.org/pep-0667/] para mais detalhes, incluindo alterações e descontinuações relacionadas à API C. Notas de portabilidade também são fornecidas abaixo para as APIs de Python e as APIs de C afetadas.

(PEP e implementação são contribuição de Mark Shannon e Tian Gao em gh-74929 [https://github.com/python/cpython/issues/74929]. Atualizações da documentação fornecidas por Guido van Rossum e Alyssa Coghlan.)

Suporte para plataformas móveis

PEP 730 [https://peps.python.org/pep-0730/]: iOS agora é uma plataforma com suporte conforme a **PEP 11** [https://peps.python.org/pep-0011/], com os alvos `arm64-apple-ios` e `arm64-apple-ios-simulator` no nível 3 (dispositivos iPhone e iPad lançados após 2013 e o simulador Xcode iOS sendo executado em hardware Apple Silicon, respectivamente). `x86_64-apple-ios-simulator` (o simulador Xcode iOS sendo executado em hardware `x86_64` mais antigo) não é uma plataforma com suporte nível 3, mas terá o melhor suporte possível. (PEP escrita e implementação como contribuição

de Russell Keith-Magee em gh-114099 [https://github.com/python/cpython/issues/114099].)

PEP 738 [https://peps.python.org/pep-0738/]: Android agora é uma plataforma com suporte conforme a **PEP 11** [https://peps.python.org/pep-0011/], com os alvos `aarch64-linux-android` e `x86_64-linux-android` no nível 3. Os alvos de 32 bits `arm-linux-androideabi` e `i686-linux-android` não são plataformas com suporte de nível 3, mas terão o melhor suporte possível. (PEP escrita e implementação como contribuição de Malcolm Smith em gh-116622 [https://github.com/python/cpython/issues/116622].)

Ver também

PEP 730 [https://peps.python.org/pep-0730/], **PEP 738** [https://peps.python.org/pep-0738/]

Outras mudanças na linguagem

- O compilador agora remove espaços em branco comuns de cada linha em uma docstring. Isso reduz o tamanho do cache de bytecode (como arquivos `.pyc`), com reduções no tamanho do arquivo de cerca de 5%, por exemplo, em **`sqlalchemy.orm.session`** do SQLAlchemy 2.0. Essa alteração afeta ferramentas que usam docstrings, como **`doctest`**.

```
>>> def spam():
...     """
...         Esta é uma docstring com
...           espaços em branco no começo.
...
...         Ela atém mesmo tem vários parágrafos!
...     """
...
>>> spam.__doc__
'\n\nEsta é uma docstring com\n\n espaços em branco no comec
```

(Contribuição de Inada Naoki em gh-81283 [https://github.com/python/cpython/issues/81283].)

- Escopos de anotação dentro dos escopos de classe agora pode conter lambdas e compreensões. As compreensões localizadas nos escopos de classe não são incorporadas ao escopo pai.

  ```
  class C[T]:
      type Alias = lambda: T
  ```

  (Contribuição de Jelle Zijlstra em gh-109118 [https://github.com/python/cpython/issues/109118] e gh-118160 [https://github.com/python/cpython/issues/118160].)

- Instruções future não são mais acionadas por importações relativas do módulo `__future__`, o que significa que instruções no formato `from .__future__ import ...` agora são simplesmente importações relativas padrão, sem recursos especiais ativados. (Contribuição de Jeremiah Gabriel Pascual em gh-118216 [https://github.com/python/cpython/issues/118216].)

- Declarações `global` agora são permitidas em blocos `except` quando esse global é usado no bloco `else`. Anteriormente, isso erroneamente levantava uma exceção `SyntaxError`. (Contribuição de Irit Katriel em gh-111123 [https://github.com/python/cpython/issues/111123].)

- Adiciona `PYTHON_FROZEN_MODULES`, uma nova variável de ambiente que determina se os módulos congelados são ignorados pelo maquinário de importação, equivalente à opção de linha de comando `-X frozen_modules`. (Contribuição de Yilei Yang em gh-111374 [https://github.com/python/cpython/issues/111374].)

- Adicionado suporte ao perfilador perf funcionando sem ponteiros de quadro [https://pt.wikipedia.org/wiki/Pilha_de_chamada] através da nova variável de ambiente `PYTHON_PERF_JIT_SUPPORT` e opção de linha de comando `-X perf_jit`. (Contribuição de Pablo Galindo em gh-118518 [https://github.com/python/cpython/issues/118518].)

- A localização de um arquivo `.python_history` pode ser alterada por meio da nova variável de ambiente `PYTHON_HISTORY`. (Contribuição de

Levi Sabah, Zackery Spytz e Hugo van Kemenade em gh-73965 [https://github.com/python/cpython/issues/73965].)

- As classes têm um novo atributo `__static_attributes__`. Isso é preenchido pelo compilador, com uma tupla de nomes de atributos da classe que são atribuídos através de `self.<name>` de qualquer função em seu corpo. (Contribuição de Irit Katriel em gh-115775 [https://github.com/python/cpython/issues/115775].)

- O compilador agora cria um atributo **`__firstlineno__`** em classes com o número da primeira linha da definição de classe. (Contribuição de Serhiy Storchaka em gh-118465 [https://github.com/python/cpython/issues/118465].)

- As funções embutidas **`exec()`** e **`eval()`** agora aceitam os argumentos *globals* e *locals* como nomeados. (Contribuição de Raphael Gaschignard em gh-105879 [https://github.com/python/cpython/issues/105879])

- A função embutida **`compile()`** agora aceita um novo sinalizador, `ast.PyCF_OPTIMIZED_AST`, que é semelhante a `ast.PyCF_ONLY_AST` exceto que a AST retornada é otimizada de acordo com o valor do argumento *optimize*. (Contribuição de Irit Katriel em gh-108113 [https://github.com/python/cpython/issues/108113]).

- Adiciona um atributo `__name__` em objetos **`property`**. (Contribuição de Eugene Toder em gh-101860 [https://github.com/python/cpython/issues/101860].)

- Adiciona **`PythonFinalizationError`**, uma nova exceção derivada de **`RuntimeError`** e usada para sinalizar quando as operações são bloqueadas durante a finalização. Os seguintes chamáveis agora levantam **`PythonFinalizationError`**, em vez de **`RuntimeError`**:

  - **`_thread.start_new_thread()`**
  - **`os.fork()`**
  - **`os.forkpty()`**
  - **`subprocess.Popen`**

(Contribuição de Victor Stinner em gh-114570 [https://github.com/python/cpython/issues/114570].)

- Permite que o argumento *count* de `str.replace()` seja um argumento nomeado. (Contribuição de Hugo van Kemenade em gh-106487 [https://github.com/python/cpython/issues/106487].)

- Muitas funções agora emitem um aviso se um valor booleano for passado como argumento do descritor de arquivo. Isso pode ajudar a detectar alguns erros mais cedo. (Contribuição de Serhiy Storchaka em gh-82626 [https://github.com/python/cpython/issues/82626].)

- Adicionados atributos `name` e `mode` para objetos arquivo ou similares compactados e arquivados nos módulos `bz2`, `lzma`, `tarfile` e `zipfile`. (Contribuição de Serhiy Storchaka em gh-115961 [https://github.com/python/cpython/issues/115961].)

Novos módulos

- `dbm.sqlite3`: um backend SQLite para `dbm`. (Contribuição de Raymond Hettinger e Erlend E. Aasland em gh-100414 [https://github.com/python/cpython/issues/100414].)

Módulos melhorados

argparse

- Adiciona o parâmetro *deprecated* aos métodos `add_argument()` e `add_parser()`, o qual permite descontinuar opções de linha de comando, argumentos posicionais e subcomandos. (Contribuição de Serhiy Storchaka em gh-83648 [https://github.com/python/cpython/issues/83648].)

array

- Adiciona o código do tipo `'w'` (`Py_UCS4`) para caracteres Unicode. Ele deve ser usado no lugar do código do tipo `'u'` descontinuado.

(Contribuição de Inada Naoki em gh-80480 [https://github.com/python/cpython/issues/80480].)

- Registra `array.array` como `MutableSequence` implementando o método `clear()`. (Contribuição de Mike Zimin em gh-114894 [https://github.com/python/cpython/issues/114894].)

ast

- Os construtores dos tipos de nós no módulo `ast` agora são mais rígidos nos argumentos que aceitam, com um comportamento mais intuitivo quando os argumentos são omitidos.

  Se um campo opcional em um nó da AST não for incluído como argumento ao construir uma instância, o campo agora será definido como `None`. Da mesma forma, se um campo de lista for omitido, esse campo será agora definido como uma lista vazia, e se um campo `expr_context` for omitido, o padrão será `Load()`. (Anteriormente, em todos os casos, o atributo estaria ausente na instância do nó da AST recém-construída.)

  Em todos os outros casos, nos quais um argumento é omitido, o construtor de nós vai emitir uma `DeprecationWarning`. Isso vai levantar uma exceção no Python 3.15. Da mesma forma, passar um argumento nomeado para o construtor que não mapeia para um campo no nó da AST agora está descontinuado e vai levantar uma exceção no Python 3.15.

  Estas mudanças não se aplicam às subclasses de `ast.AST` definidas pelo usuário, a menos que a classe opte pelo novo comportamento definindo o mapeamento `AST._field_types`.

  (Contribuição de Jelle Zijlstra em gh-105858 [https://github.com/python/cpython/issues/105858], gh-117486 [https://github.com/python/cpython/issues/117486] e gh-118851 [https://github.com/python/cpython/issues/118851].)

- `ast.parse()` agora aceita um argumento opcional *optimize* que é passado para `compile()`. Isto torna possível obter uma AST otimizada.

(Contribuição de Irit Katriel em gh-108113 [https://github.com/python/cpython/issues/108113].)

asyncio

- `asyncio.as_completed()` agora retorna um objeto que é ao mesmo tempo um iterador assíncrono e um iterator simples de aguardáveis. Os aguardáveis gerados pela iteração assíncrona incluem tarefas originais ou objetos futuros que foram passados, facilitando a associação dos resultados às tarefas que estão sendo concluídas. (Contribuição de Justin Arthur em gh-77714 [https://github.com/python/cpython/issues/77714].)

- `asyncio.loop.create_unix_server()` agora removerá automaticamente o soquete Unix quando o servidor for fechado. (Contribuição de Pierre Ossman em gh-111246 [https://github.com/python/cpython/issues/111246].)

- `DatagramTransport.sendto()` agora enviará datagramas de comprimento zero se for chamado com um objeto de bytes vazios. O controle de fluxo de transporte agora também leva em conta o cabeçalho do datagrama ao calcular o tamanho do buffer. (Contribuição de Jamie Phan em gh-115199 [https://github.com/python/cpython/issues/115199].)

- Adiciona `Queue.shutdown` e `QueueShutDown` para gerenciar a terminação da fila. (Contribuição de Laurie Opperman e Yves Duprat em gh-104228 [https://github.com/python/cpython/issues/104228].)

- Adiciona os métodos `Server.close_clients()` e `Server.abort_clients()`, que fecham de forma mais forçada um servidor asyncio. (Contribuição de Pierre Ossman em gh-113538 [https://github.com/python/cpython/issues/113538].)

- Aceita uma tupla de separadores em `StreamReader.readuntil()`, parando quando qualquer um deles for encontrado. (Contribuição de Bruce Merry em gh-81322 [https://github.com/python/cpython/issues/81322].)

- Melhora o comportamento de `TaskGroup` quando um cancelamento externo colide com um cancelamento interno. Por exemplo, quando dois grupos de tarefas estão aninhados e ambos experimentam uma exceção em uma tarefa filho simultaneamente, é possível que o grupo de tarefas externo seja interrompido, porque seu cancelamento interno foi engolido pelo grupo de tarefas interno.

  No caso em que um grupo de tarefas é cancelado externamente e também deve levantar `ExceptionGroup`, ele agora chamará o método `cancel()` da tarefa pai. Isso garante que uma `CancelledError` será levantada no próximo `await`, para que o cancelamento não seja perdido.

  Um benefício adicional dessas mudanças é que os grupos de tarefas agora preservam a contagem de cancelamentos (`cancelling()`).

  Para lidar com alguns casos extremos, `uncancel()` agora pode redefinir o sinalizador não documentado `_must_cancel` quando a contagem de cancelamentos chegar a zero.

  (Inspirado por um relatório de problema enviado por Arthur Tacca em gh-116720 [https://github.com/python/cpython/issues/116720].)

- Quando `TaskGroup.create_task()` é chamado em um `TaskGroup` inativo, a corrotina fornecida será fechada (o que evita um `RuntimeWarning` sobre uma corrotina que nunca foi aguardada). (Contribuição de Arthur Tacca e Jason Zhang em gh-115957 [https://github.com/python/cpython/issues/115957].)

- A função e os métodos `create_task` receberam um novo argumento `**kwargs` que é passado para o construtor da tarefa. Essa alteração foi adicionada acidentalmente na versão 3.13.3 e quebrou o contrato da API para fábricas de tarefas personalizadas. Várias fábricas de tarefas de terceiros implementaram soluções alternativas para isso. Na versão 3.13.4 e versões posteriores, o antigo contrato de fábrica é mantido novamente (até a versão 3.14). Para manter as soluções alternativas funcionando, o argumento extra `**kwargs` ainda permite a passagem

de argumentos nomeados adicionais para `Task` e para fábricas de tarefas personalizadas.

Isso afeta as seguintes funções e métodos: `asyncio.create_task()`, `asyncio.loop.create_task()`, `asyncio.TaskGroup.create_task()`. (Contribuição de Thomas Grainger em gh-128307 [https://github.com/python/cpython/issues/128307].)

base64

- Adiciona as funções `z85encode()` e `z85decode()` para codificar `bytes` como dados Z85 [https://rfc.zeromq.org/spec/32/] e decodificar dados codificados em Z85 para `bytes`. (Contribuição de Matan Perelman em gh-75299 [https://github.com/python/cpython/issues/75299].)

compileall

- O número padrão de threads e processos de trabalho é agora selecionado usando `os.process_cpu_count()` em vez de `os.cpu_count()`. (Contribuição de Victor Stinner em gh-109649 [https://github.com/python/cpython/issues/109649].)

concurrent.futures

- O número padrão de threads e processos de trabalho é agora selecionado usando `os.process_cpu_count()` em vez de `os.cpu_count()`. (Contribuição de Victor Stinner em gh-109649 [https://github.com/python/cpython/issues/109649].)

configparser

- `ConfigParser` agora tem suporte para seções sem nome, o que permite pares de chave-valor de nível superior. Isso pode ser habilitado com o novo parâmetro *allow_unnamed_section*. (Contribuição de Pedro Sousa Lacerda em gh-66449 [https://github.com/python/cpython/issues/66449].)

copy

- A nova função `replace()` e o protocolo `replace` tornam a criação de cópias modificadas de objetos muito mais simples. Isso é especialmente útil ao trabalhar com objetos imutáveis. Os seguintes tipos dão suporte à função **`replace()`** e implementam o protocolo replace:

  - `collections.namedtuple()`
  - `dataclasses.dataclass`
  - `datetime.datetime`, `datetime.date`, `datetime.time`
  - `inspect.Signature`, `inspect.Parameter`
  - `types.SimpleNamespace`
  - objetos código

  Qualquer classe definida pelo usuário também pode ter suporte a `copy.replace()` definindo o método `__replace__()`. (Contribuição de Serhiy Storchaka em gh-108751 [https://github.com/python/cpython/issues/108751].)

ctypes

- Como consequência da refatoração interna necessária, a inicialização de metaclasses internas agora acontece em `__init__` em vez de `__new__`. Isso afeta projetos que fazem subclasse dessas metaclasses internas para fornecer inicialização personalizada. De forma geral:

  - A lógica personalizada que foi feita em `__new__` após chamar `super().__new__` deve ser movida para `__init__`.
  - Para criar uma classe, chame a metaclasse, não apenas o método `__new__` da metaclasse.

  Veja gh-124520 [https://github.com/python/cpython/issues/124520] para discussão e links para mudanças em alguns projetos afetados.

- Os objetos `ctypes.Structure` têm um novo atributo `_align_` que permite que o alinhamento da estrutura que está sendo empacotada para/da memória seja especificado explicitamente. (Contribuição de Matt Sanderson em gh-112433 [https://github.com/python/cpython/issues/112433])

dbm

- Adiciona `dbm.sqlite3`, um novo módulo que implementa um backend SQLite, e torna-o o backend padrão `dbm`. (Contribuição de Raymond Hettinger e Erlend E. Aasland em gh-100414 [https://github.com/python/cpython/issues/100414].)
- Permite a remoção de todos os itens do banco de dados por meio dos novos métodos `clear()` dos objetos de banco de dados GDBM e NDBM. (Contribuição de Donghee Na em gh-107122 [https://github.com/python/cpython/issues/107122].)

dis

- Altera a saída das funções do módulo `dis` para mostrar rótulos lógicos para alvos de salto e manipuladores de exceção, em vez de deslocamentos. Os deslocamentos podem ser adicionados com a nova opção de linha de comando `-O` ou o argumento *show_offsets*. (Contribuição de Irit Katriel em gh-112137 [https://github.com/python/cpython/issues/112137].)
- `get_instructions()` não representa mais entradas de cache como instruções separadas. Em vez disso, ele os retorna como parte de `Instruction`, no novo campo *cache_info*. O argumento *show_caches* para `get_instructions()` foi descontinuado e não tem mais nenhum efeito. (Contribuição de Irit Katriel em gh-112962 [https://github.com/python/cpython/issues/112962].)

doctest

- A saída do `doctest` é agora colorida por padrão. Isso pode ser controlado através da nova variável de ambiente `PYTHON_COLORS`, bem como das variáveis de ambiente canônicas `NO_COLOR` [https://no-color.org/] e `FORCE_COLOR` [https://force-color.org/]. Veja também Controlando cores. (Contribuição de Hugo van Kemenade em gh-117225 [https://github.com/python/cpython/issues/117225].)
- O método `DocTestRunner.run()` agora conta o número de testes ignorados. Adiciona os atributos `DocTestRunner.skips` e

`TestResults.skipped`. (Contribuição de Victor Stinner em gh-108794 [https://github.com/python/cpython/issues/108794].)

email

- Cabeçalhos com novas linhas embutidas são agora colocadas entre aspas na saída. O `generator` agora se recusará a serializar (escrever) cabeçalhos que estão dobrados ou delimitados incorretamente, de modo que eles seriam analisados como vários cabeçalhos ou unidos a dados adjacentes. Se você precisar desativar esse recurso de segurança, defina `verify_generated_headers`. (Contribuição de Bas Bloemsaat e Petr Viktorin em gh-121650 [https://github.com/python/cpython/issues/121650].)
- `getaddresses()` e `parseaddr()` agora retornam pares `('', '')` em mais situações onde endereços de e-mail inválidos são encontrados em vez de valores potencialmente imprecisos. As duas funções têm agora um novo parâmetro opcional *strict* (padrão `True`). Para obter o comportamento antigo (aceitar entrada malformada), `strict=False`. `getattr(email.utils, 'supports_strict_parsing', False)` pode ser usado para verificar se o parâmetro *strict* está disponível. (Contribuição de Thomas Dwyer e Victor Stinner para gh-102988 [https://github.com/python/cpython/issues/102988] para melhorar a correção **CVE 2023-27043** [https://www.cve.org/CVERecord?id=CVE-2023-27043].)

enum

- `EnumDict` foi tornada pública para melhor suporte às subclasses de `EnumType`.

fractions

- Os objetos `Fraction` agora oferecem suporte às regras padrões da minilinguagem de especificação de formato para preenchimento, alinhamento, manipulação de sinais, largura mínima e agrupamento. (Contribuição de Mark Dickinson em gh-111320 [https://github.com/python/cpython/issues/111320].)

glob

- Adiciona `translate()`, uma função que converte uma especificação de caminho com curingas estilo shell em uma expressão regular. (Contribuição de Barney Gale em gh-72904 [https://github.com/python/cpython/issues/72904].)

importlib

- As seguintes funções em `importlib.resources` agora permitem acessar um diretório (ou árvore) de recursos, usando múltiplos argumentos posicionais (os argumentos *encoding* e *errors* nas funções de leitura de texto agora são somente-nomeados):

  - `is_resource()`
  - `open_binary()`
  - `open_text()`
  - `path()`
  - `read_binary()`
  - `read_text()`

  Essas funções não estão mais descontinuadas e não estão programadas para remoção. (Contribuição de Petr Viktorin em gh-116608 [https://github.com/python/cpython/issues/116608].)

- A função `contents()` permanece descontinuada em favor da API completa `Traversable`. No entanto, agora não há planos para removê-la. (Contribuição de Petr Viktorin em gh-116608 [https://github.com/python/cpython/issues/116608].)

io

- O finalizador `IOBase` agora registra quaisquer erros levantados pelo método `close()` com `sys.unraisablehook`. Anteriormente, os erros eram ignorados silenciosamente por padrão e registrados apenas em Modo de Desenvolvimento do Python ou ao usar uma construção de depuração do Python. (Contribuição de Victor Stinner em gh-62948 [https://github.com/python/cpython/issues/62948].)

ipaddress

- Adiciona a propriedade `IPv4Address.ipv6_mapped`, que retorna o endereço IPv6 mapeado para IPv4. (Contribuição de Charles Machalow em gh-109466 [https://github.com/python/cpython/issues/109466].)
- Corrige os comportamentos de `is_global` e `is_private` em `IPv4Address`, `IPv6Address`, `IPv4Network` e `IPv6Network`. (Contribuição de Jakub Stasiak em gh-113171 [https://github.com/python/cpython/issues/113171].)

itertools

- `batched()` tem um novo parâmetro *strict*, que levanta uma `ValueError` se o lote final for menor que o tamanho do lote especificado. (Contribuição de Raymond Hettinger em gh-113202 [https://github.com/python/cpython/issues/113202].)

marshal

- Adiciona o parâmetro *allow_code* nas funções do módulo. Passar `allow_code=False` evita a serialização e desserialização de objetos de código que são incompatíveis entre versões do Python. (Contribuição de Serhiy Storchaka em gh-113626 [https://github.com/python/cpython/issues/113626].)

math

- A nova função `fma()` realiza operações fundidas de multiplicação e adição. Ela calcula `x * y + z` com apenas uma única rodada e, portanto, evita qualquer perda intermediária de precisão. Ele envolve a função `fma()` fornecida pelo C99 e segue a especificação da operação IEEE 754 “fusedMultiplyAdd” para casos especiais. (Contribuição de Mark Dickinson e Victor Stinner em gh-73468 [https://github.com/python/cpython/issues/73468].)

mimetypes

- Adiciona a função `guess_file_type()` para adivinhar um tipo MIME de um caminho do sistema de arquivos. Usar caminhos com `guess_type()` agora está suavemente descontinuada. (Contribuição de Serhiy Storchaka em gh-66543 [https://github.com/python/cpython/issues/66543].)

mmap

- `mmap` agora está protegida contra travamentos no Windows quando a memória mapeada está inacessível devido a erros no sistema de arquivos ou violações de acesso. (Contribuição de Jannis Weigend em gh-118209 [https://github.com/python/cpython/issues/118209].)
- `mmap` tem um novo método `seekable()` que pode ser usado quando um objeto arquivo ou similar pesquisável é necessário. O método `seek()` agora retorna a nova posição absoluta. (Contribuição de Donghee Na e Sylvie Liberman em gh-111835 [https://github.com/python/cpython/issues/111835].)
- O novo parâmetro *trackfd* somente para UNIX para `mmap` controla a duplicação do descritor de arquivo; se falso, o descritor de arquivo especificado por *fileno* não será duplicado. (Contribuição de Zackery Spytz e Petr Viktorin em gh-78502 [https://github.com/python/cpython/issues/78502].)

multiprocessing

- O número padrão de threads e processos de trabalho é agora selecionado usando `os.process_cpu_count()` em vez de `os.cpu_count()`. (Contribuição de Victor Stinner em gh-109649 [https://github.com/python/cpython/issues/109649].)

os

- Adiciona a função `process_cpu_count()` para obter o número de núcleos lógicos de CPU utilizáveis pela thread de chamada do processo atual. (Contribuição de Victor Stinner em gh-109649 [https://github.com/python/cpython/issues/109649].)
- `cpu_count()` e `process_cpu_count()` podem ser substituídos através da nova variável de ambiente `PYTHON_CPU_COUNT` ou da nova opção de

linha de comando `-X cpu count`. Esta opção é útil para usuários que precisam limitar os recursos da CPU de um sistema contêiner sem precisar modificar código da aplicação ou o contêiner em si. (Contribuição de Donghee Na em gh-109595 [https://github.com/python/cpython/issues/109595].)

- Adiciona uma interface de baixo nível aos *descritores de arquivo de temporizador* *[https://manpages.debian.org/timerfd_create(2)]* do Linux via `timerfd_create()`, `timerfd_settime()`, `timerfd_settime_ns()`, `timerfd_gettime()`, `timerfd_gettime_ns()`, `TFD_NONBLOCK`, `TFD_CLOEXEC`, `TFD_TIMER_ABSTIME` e `TFD_TIMER_CANCEL_ON_SET` (Contribuição de Masaru Tsuchiyama em gh-108277 [https://github.com/python/cpython/issues/108277].)
- `lchmod()` e o argumento *follow_symlinks* em `chmod()` estão agora disponíveis no Windows. Observe que o valor padrão de *follow_symlinks* em **`lchmod()`** é `False` no Windows. (Contribuição de Serhiy Storchaka em gh-59616 [https://github.com/python/cpython/issues/59616].)
- `fchmod()` e suporte para descritores de arquivo em `chmod()` estão agora disponíveis no Windows. (Contribuição de Serhiy Storchaka em gh-113191 [https://github.com/python/cpython/issues/113191].)
- No Windows, `mkdir()` e `makedirs()` agora oferecem suporte a passar um valor *mode* de `0o700` para aplicar controle de acesso ao novo diretório. Isso afeta implicitamente `tempfile.mkdtemp()` e é uma mitigação para **CVE 2024-4030** [https://www.cve.org/CVERecord?id=CVE-2024-4030]. Outros valores para *mode* continuam a ser ignorados. (Contribuição de Steve Dower em gh-118486 [https://github.com/python/cpython/issues/118486].)
- `posix_spawn()` agora aceita `None` para o argumento *env*, o que faz com que o processo recém-gerado use o ambiente de processo atual. (Contribuição de Jakub Kulik em gh-113119 [https://github.com/python/cpython/issues/113119].)
- `posix_spawn()` agora pode usar o atributo `POSIX_SPAWN_CLOSEFROM` no parâmetro *file_actions* em plataformas que oferecem suporte a **`posix_spawn_file_actions_addclosefrom_np()`**. (Contribuição de Jakub Kulik em gh-113117 [https://github.com/python/cpython/issues/113117].)

os.path

- Adiciona `isreserved()` para verificar se um caminho está reservado no sistema atual. Esta função está disponível apenas no Windows. (Contribuição de Barney Gale em gh-88569 [https://github.com/python/cpython/issues/88569].)
- No Windows, `isabs()` não considera mais caminhos que começam com exatamente uma barra (`\` ou `/`) como absolutos. (Contribuição de Barney Gale e Jon Foster em gh-44626 [https://github.com/python/cpython/issues/44626].)
- `realpath()` agora resolve nomes de arquivos no estilo MS-DOS, mesmo que o arquivo não esteja acessível. (Contribuição de Moonsik Park em gh-82367 [https://github.com/python/cpython/issues/82367].)

pathlib

- Adiciona `UnsupportedOperation`, que é levantada em vez de `NotImplementedError` quando uma operação de caminho não é possível. (Contribuição de Barney Gale em gh-89812 [https://github.com/python/cpython/issues/89812].)
- Adiciona um novo construtor para criar objetos `Path` a partir de URIs 'file' (`file:///`), `Path.from_uri()`. (Contribuição de Barney Gale em gh-107465 [https://github.com/python/cpython/issues/107465].)
- Adiciona `PurePath.full_match()` para corresponder caminhos com curingas do tipo shell, incluindo o curinga recursivo "`**`". (Contribuição de Barney Gale em gh-73435 [https://github.com/python/cpython/issues/73435].)
- Adiciona o atributo de classe `PurePath.parser` para armazenar a implementação de `os.path` usada para análise e junção de caminhos de baixo nível. Isso vai ser **`posixpath`** ou **`ntpath`**.
- Adiciona o argumento somente-nomeado *recurse_symlinks* a `Path.glob()` e `rglob()`. (Contribuição de Barney Gale em gh-77609 [https://github.com/python/cpython/issues/77609].)
- `Path.glob()` e `rglob()` agora retornam arquivos e diretórios quando recebem um padrão que termina com "`**`". Anteriormente, apenas os diretórios eram retornados. (Contribuição de Barney Gale em gh-70303 [https://github.com/python/cpython/issues/70303].)
- Adiciona o argumento somente-nomeado *follow_symlinks* a `Path.is_file`, `Path.is_dir`, `Path.owner()` e `Path.group()`.

(Contribuição de Barney Gale em gh-105793 [https://github.com/python/cpython/issues/105793] e Kamil Turek em gh-107962 [https://github.com/python/cpython/issues/107962].)

pdb

- **`breakpoint()`** e **`set_trace()`** agora entram no depurador imediatamente em vez de na próxima linha de código a ser executada. Esta mudança evita que o depurador quebre fora do contexto quando **`breakpoint()`** estiver posicionado no final do contexto. (Contribuição de Tian Gao em gh-118579 [https://github.com/python/cpython/issues/118579].)
- `sys.path[0]` não mais é substituído pelo diretório do script que está sendo depurado quando **`sys.flags.safe_path`** está definido. (Contribuição de Tian Gao e Christian Walther em gh-111762 [https://github.com/python/cpython/issues/111762].)
- **`zipapp`** agora é aceito como alvo de depuração. (Contribuição de Tian Gao em gh-118501 [https://github.com/python/cpython/issues/118501].)
- Adiciona a capacidade de mover entre exceções encadeadas durante a depuração post-mortem em **`pm()`** usando o novo comando **`exceptions [número_exc]`** para Pdb. (Contribuição de Matthias Bussonnier em gh-106676 [https://github.com/python/cpython/issues/106676].)
- Expressões e instruções cujo prefixo é um comando pdb agora são identificadas e executadas corretamente. (Contribuição de Tian Gao em gh-108464 [https://github.com/python/cpython/issues/108464].)

queue

- Adiciona **`Queue.shutdown`** e **`ShutDown`** para gerenciar a finalização da fila. (Contribuição de Laurie Opperman e Yves Duprat em gh-104750 [https://github.com/python/cpython/issues/104750].)

random

- Adiciona uma interface de linha de comando. (Contribuição de Hugo van Kemenade em gh-118131 [https://github.com/python/cpython/issues/118131].)

re

- Renomeia `re.error` para `PatternError` para maior clareza. `re.error` é mantido para compatibilidade com versões anteriores.

shutil

- Suporte aos argumentos nomeados *dir_fd* e *follow_symlinks* em `chown()`. (Contribuição de Berker Peksag e Tahia K em gh-62308 [https://github.com/python/cpython/issues/62308])

site

- Os arquivos `.pth` agora são decodificados primeiro usando UTF-8 e depois usando codificação da localidade se a decodificação UTF-8 falhar. (Contribuição de Inada Naoki em gh-117802 [https://github.com/python/cpython/issues/117802].)

sqlite3

- Uma `ResourceWarning` agora é emitida se um objeto `Connection` não for explicitamente `fechado`. (Contribuição de Erlend E. Aasland em gh-105539 [https://github.com/python/cpython/issues/105539].)
- Adiciona o parâmetro somente-nomeado *filter* a `Connection.iterdump()` para filtrar objetos de banco de dados para despejo. (Contribuição de Mariusz Felisiak em gh-91602 [https://github.com/python/cpython/issues/91602].)

ssl

- A API `create_default_context()` agora inclui `VERIFY_X509_PARTIAL_CHAIN` e `VERIFY_X509_STRICT` em seus sinalizadores padrão.

  Nota

  `VERIFY_X509_STRICT` pode rejeitar certificados pré-**RFC 5280** [https://datatracker.ietf.org/doc/html/rfc5280.html] ou malformados que a

implementação OpenSSL subjacente poderia aceitar. Embora não seja recomendado desativar isso, você pode fazer isso usando:

```
import ssl

ctx = ssl.create_default_context()
ctx.verify_flags &= ~ssl.VERIFY_X509_STRICT
```

(Contribuição de William Woodruff em gh-112389 [https://github.com/python/cpython/issues/112389].)

statistics

- Adiciona **`kde()`** para estimativa de densidade do núcleo. Isto torna possível estimar uma função de densidade de probabilidade contínua a partir de um número fixo de amostras discretas. (Contribuição de Raymond Hettinger em gh-115863 [https://github.com/python/cpython/issues/115863].)
- Adiciona **`kde_random()`** para amostragem de uma função de densidade de probabilidade estimada criada por **`kde()`**. (Contribuição de Raymond Hettinger em gh-115863 [https://github.com/python/cpython/issues/115863].)

subprocess

- O módulo **`subprocess`** agora usa a função **`posix_spawn()`** em mais situações.

  Notavelmente, quando *close_fds* é `True` (o padrão), **`posix_spawn()`** será usado quando a biblioteca C fornecer **`posix_spawn_file_actions_addclosefrom_np()`**, que inclui versões recentes do Linux, FreeBSD e Solaris. No Linux, isso deve ter um desempenho semelhante ao código existente baseado em **`vfork()`** do Linux.

  Um botão de controle privado **`subprocess._USE_POSIX_SPAWN`** pode ser definido como `False` se você precisar forçar **`subprocess`** a nunca

usar `posix_spawn()`. Por favor, informe o motivo e os detalhes da plataforma no rastreador de problemas se você definir isso para que possamos melhorar nossa lógica de seleção de API para todos. (Contribuído por Jakub Kulik em gh-113117 [https://github.com/python/cpython/issues/113117].)

sys

- Adiciona a função `_is_interned()` para testar se uma string foi internada. Não há garantia de que esta função exista em todas as implementações do Python. (Contribuição de Serhiy Storchaka em gh-78573 [https://github.com/python/cpython/issues/78573].)

tempfile

- No Windows, o modo padrão `0o700` usado por `tempfile.mkdtemp()` agora limita o acesso ao novo diretório devido a alterações em `os.mkdir()`. Esta é uma mitigação para **CVE 2024-4030** [https://www.cve.org/CVERecord?id=CVE-2024-4030]. (Contribuição de Steve Dower em gh-118486 [https://github.com/python/cpython/issues/118486].)

time

- No Windows, `monotonic()` agora usa o relógio `QueryPerformanceCounter()` para uma resolução de 1 microssegundo, em vez do relógio `GetTickCount64()` que tem uma resolução de 15.6 milissegundos. (Contribuição de Victor Stinner em gh-88494 [https://github.com/python/cpython/issues/88494].)
- No Windows, `time()` agora usa o relógio `GetSystemTimePreciseAsFileTime()` para uma resolução de 1 microssegundo, em vez do relógio `GetSystemTimeAsFileTime()` que tem uma resolução de 15.6 milissegundos. (Contribuição de Victor Stinner em gh-63207 [https://github.com/python/cpython/issues/63207].)

tkinter

- Adiciona métodos de widget **`tkinter`**: **`tk_busy_hold()`**, **`tk_busy_configure()`**, **`tk_busy_cget()`**, **`tk_busy_forget()`**, **`tk_busy_current()`** e **`tk_busy_status()`**. (Contribuição de Miguel, klappnase e Serhiy Storchaka em gh-72684 [https://github.com/python/cpython/issues/72684].)
- O método **`wm_attributes()`** de widget **`tkinter`** agora aceita o nome do atributo sem o prefixo negativo para obter atributos de janela (por exemplo, `w.wm_attributes('alpha')`) e permite especificar atributos e valores para definir como argumentos nomeados (por exemplo, `w.wm_attributes(alpha=0.5)`). (Contribuição de Serhiy Storchaka em gh-43457 [https://github.com/python/cpython/issues/43457].)
- **`wm_attributes()`** agora pode retornar atributos como um **`dict`**, usando o novo parâmetro somente-nomeado opcional *return_python_dict*. (Contribuição de Serhiy Storchaka em gh-43457 [https://github.com/python/cpython/issues/43457].)
- **`Text.count()`** agora pode retornar um **`int`** simples quando o novo parâmetro somente-nomeado opcional *return_ints* é usado. Caso contrário, a contagem única é retornada como uma tupla de 1 elemento ou `None`. (Contribuição de Serhiy Storchaka em gh-97928 [https://github.com/python/cpython/issues/97928].)
- Suporte ao tipo de elemento “vsapi” no método **`element_create()`** de **`tkinter.ttk.Style`**. (Contribuição de Serhiy Storchaka em gh-68166 [https://github.com/python/cpython/issues/68166].)
- Adiciona o método **`after_info()`** para widgets do Tkinter. (Contribuição de Cheryl Sabella em gh-77020 [https://github.com/python/cpython/issues/77020].)
- Adiciona um novo método **`copy_replace()`** a **`PhotoImage`** para copiar uma região de uma imagem para outra, possivelmente com zoom de pixel, subamostragem ou ambos. (Contribuição de Serhiy Storchaka em gh-118225 [https://github.com/python/cpython/issues/118225].)
- Adiciona parâmetro *from_coords* para os métodos **`copy()`**, **`zoom()`** e **`subsample()`** da classe **`PhotoImage`**. Adiciona parâmetros *zoom* e *subsample* para o método **`copy()`** da classe **`PhotoImage`**. (Contribuição de Serhiy Storchaka em gh-118225 [https://github.com/python/cpython/issues/118225].)
- Adiciona os métodos **`PhotoImage read()`** para ler uma imagem de um arquivo e **`data()`** para obter os dados da imagem. Adiciona os

parâmetros *background* e *grayscale* ao método **`write()`**. (Contribuição de Serhiy Storchaka em gh-118271 [https://github.com/python/cpython/issues/118271].)

traceback

- Adiciona o atributo **`exc_type_str`** a **`TracebackException`**, que contém uma exibição de string do *exc_type*. Descontinua o atributo **`exc_type`**, que contém o próprio objeto de tipo. Adicione o parâmetro *save_exc_type* (padrão `True`) para indicar se `exc_type` deve ser salvo. (Contribuição de Irit Katriel em gh-112332 [https://github.com/python/cpython/issues/112332].)
- Adiciona um novo parâmetro somente-nomeado *show_group* a **`TracebackException.format_exception_only()`** para formatar (recursivamente) as exceções aninhadas de uma instância **`BaseExceptionGroup`**. (Contribuição de Irit Katriel em gh-105292 [https://github.com/python/cpython/issues/105292].)

types

- **`SimpleNamespace`** agora pode receber um único argumento posicional para inicializar os argumentos do espaço de nomes. Este argumento deve ser um mapeamento ou um iterável de pares chave-valor. (Contribuição de Serhiy Storchaka em gh-108191 [https://github.com/python/cpython/issues/108191].)

typing

- **PEP 705** [https://peps.python.org/pep-0705/]: adiciona **`ReadOnly`**, uma construção especial de tipagem para marcar um item **`TypedDict`** como somente leitura para verificadores de tipo.
- **PEP 742** [https://peps.python.org/pep-0742/]: adiciona **`TypeIs`**, uma construção de tipagem que pode ser usada para instruir um verificador de tipos sobre como restringir um tipo.
- Adiciona **`NoDefault`**, um objeto sinalizador usado para representar os padrões de alguns parâmetros no módulo **`typing`**. (Contribuição de Jelle Zijlstra em gh-116126 [https://github.com/python/cpython/issues/116126].)

- Adicione `get_protocol_members()` para retornar o conjunto de membros que definem uma `typing.Protocol`. (Contribuição de Jelle Zijlstra em gh-104873 [https://github.com/python/cpython/issues/104873].)
- Adiciona `is_protocol()` para verificar se uma classe é uma `Protocol`. (Contribuição de Jelle Zijlstra em gh-104873 [https://github.com/python/cpython/issues/104873].)
- `ClassVar` agora pode ser aninhado em `Final`, e vice-versa. (Contribuição de Mehdi Drissi em gh-89547 [https://github.com/python/cpython/issues/89547].)

unicodedata

- Atualiza o banco de dados Unicode para a versão 15.1.0 [https://www.unicode.org/versions/Unicode15.1.0/]. (Contribuição de James Gerity em gh-109559 [https://github.com/python/cpython/issues/109559].)

venv

- Adiciona suporte para criar arquivos ignorados pelo gerenciamento de controle de fontes (SCM) no diretório de um ambiente virtual. Por padrão, o Git é compatível. Isso está implementado como ativado por padrão por meio da API, que pode ser estendido para oferecer suporte a outros SCMs (`EnvBuilder` e `create()`) e desativado por padrão por meio da CLI, usando `--without-scm-ignore-files`. (Contribuição de Brett Cannon em gh-108125 [https://github.com/python/cpython/issues/108125].)

warnings

- **PEP 702** [https://peps.python.org/pep-0702/]: O novo decorador `warnings.deprecated()` fornece uma maneira de comunicar descontinuações para um verificador de tipo estático e avisar sobre o uso de classes e funções obsoletas. Um `DeprecationWarning` também pode ser emitido quando uma função ou classe decorada é usada em tempo de execução. (Contribuição de Jelle Zijlstra em gh-104003 [https://github.com/python/cpython/issues/104003].)

xml

- Permite controlar o adiamento da nova análise do Expat >=2.6.0 (**CVE 2023-52425** [https://www.cve.org/CVERecord?id=CVE-2023-52425]) adicionando cinco novos métodos:

  - `xml.etree.ElementTree.XMLParser.flush()`
  - `xml.etree.ElementTree.XMLPullParser.flush()`
  - `xml.parsers.expat.xmlparser.GetReparseDeferralEnabled()`
  - `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()`
  - `xml.sax.expatreader.ExpatParser.flush()`

  (Contribuição de Sebastian Pipping em gh-115623 [https://github.com/python/cpython/issues/115623].)

- Adiciona o método `close()` para o iterador retornado por `iterparse()` para limpeza explícita. (Contribuição de Serhiy Storchaka em gh-69893 [https://github.com/python/cpython/issues/69893].)

zipimport

- Adiciona suporte para arquivos no formato ZIP64 [https://en.wikipedia.org/wiki/Zip_(file_format)#ZIP64]. Todo mundo adora dados enormes, certo? (Contribuição de Tim Hatch em gh-94146 [https://github.com/python/cpython/issues/94146].)

Otimizações

- Vários módulos de biblioteca padrão tiveram seus tempos de importação significativamente melhorados. Por exemplo, o tempo de importação do módulo `typing` foi reduzido em cerca de um terço pela remoção de dependências em `re` e `contextlib`. Outros módulos para aproveitar a aceleração no tempo de importação incluem `email.utils`, `enum`, `functools`, `importlib.metadata` e `threading`. (Contribuição de Alex Waygood, Shantanu Jain, Adam Turner, Daniel Hollas e outros em gh-109653 [https://github.com/python/cpython/issues/109653].)
- `textwrap.indent()` agora é cerca de 30% mais rápido do que antes para entradas grandes. (Contribuição de Inada Naoki em gh-107369 [https://github.com/python/cpython/issues/107369].)

- O módulo **`subprocess`** agora a função **`posix_spawn()`** em mais situações, incluindo quando *close_fds* é `True` (o padrão) em muitas plataformas. Isto deve fornecer um aumento notável de desempenho ao iniciar processos no FreeBSD e Solaris. Veja a seção de subprocess acima para detalhes. (Contribuição de Jakub Kulik em gh-113117 [https://github.com/python/cpython/issues/113117].)

Módulos e APIs removidas

PEP 594: remove "baterias descarregadas" da biblioteca padrão

PEP 594 [https://peps.python.org/pep-0594/] propôs remover 19 módulos da biblioteca padrão, coloquialmente chamados de "baterias descarregadas" devido ao seu status histórico, obsoleto ou inseguro. Todos os módulos a seguir foram descontinuados no Python 3.11 e agora foram removidos:

- **`aifc`**
  - standard-aifc [https://pypi.org/project/standard-aifc/]: Use a redistribuição da biblioteca `aifc` do PyPI.
- **`audioop`**
  - audioop-lts [https://pypi.org/project/audioop-lts/]: Use a biblioteca `audioop-lts` do PyPI.
- **`chunk`**
  - standard-chunk [https://pypi.org/project/standard-chunk/]: Use a redistribuição da biblioteca `chunk` do PyPI.
- **`cgi`** e **`cgitb`**
  - **`cgi.FieldStorage`** normalmente pode ser substituído por **`urllib.parse.parse_qsl()`** para solicitações `GET` e `HEAD`, e o

módulo **email.message** ou a biblioteca multipart [https://pypi.org/project/multipart/] para solicitações `POST` e `PUT`.

- **cgi.parse()** pode ser substituído chamando **urllib.parse.parse_qs()** diretamente na string de consulta desejada, a menos que a entrada seja `multipart/form-data`, que deve ser substituída conforme descrito abaixo para **cgi.parse_multipart()**.

- **cgi.parse_header()** pode ser substituído pela funcionalidade do pacote **email**, que implementa os mesmos RFCs MIME. Por exemplo, com **email.message.EmailMessage**:

  ```
  from email.message import EmailMessage

  msg = EmailMessage()
  msg['content-type'] = 'application/json; charset="utf8"
  main, params = msg.get_content_type(), msg['content-typ
  ```

- **cgi.parse_multipart()** pode ser substituído pela funcionalidade no pacote **email**, que implementa os mesmos RFCs de MIME, ou pela biblioteca multipart [https://pypi.org/project/multipart/]. Por exemplo, as classes **email.message.EmailMessage** e **email.message.Message**.

- standard-cgi [https://pypi.org/project/standard-cgi/]: e standard-cgitb [https://pypi.org/project/standard-cgitb/]: Use a redistribuição das bibliotecas `cgi` e `cgitb` do PyPI.

- **crypt** e a extensão privada **_crypt**. O módulo **hashlib** pode ser uma substituição apropriada quando simplesmente fazer hash de um valor é necessário. Caso contrário, várias bibliotecas de terceiros no PyPI estão disponíveis:

  - bcrypt [https://pypi.org/project/bcrypt/]: hashing de senha moderno para seu software e servidores.
  - argon2-cffi [https://pypi.org/project/argon2-cffi/]: algoritmo seguro de hashing de senha Argon2.

    - legacycrypt [https://pypi.org/project/legacycrypt/]: envólucro ao **`ctypes`** para a chamada POSIX da biblioteca de criptografia e funcionalidade associada.
    - crypt_r [https://pypi.org/project/crypt_r/]: fork do módulo **`crypt`**, um envólucro para a chamada da biblioteca *crypt_r(3) [https://manpages.debian.org/crypt_r(3)]* e funcionalidade associada.
    - standard-crypt [https://pypi.org/project/standard-crypt/] e deprecated-crypt-alternative [https://pypi.org/project/deprecated-crypt-alternative/]: Use a redistribuição de `crypt` e a reimplementação das bibliotecas `_crypt` do PyPI.

- **`imghdr`**: as bibliotecas filetype [https://pypi.org/project/filetype/], puremagic [https://pypi.org/project/puremagic/] ou python-magic [https://pypi.org/project/python-magic/] devem ser usadas como substituições. Por exemplo, a função **`puremagic.what()`** pode ser usada para substituir a função **`imghdr.what()`** para todos os formatos de arquivo que tinham suporte de **`imghdr`**.

    - standard-imghdr [https://pypi.org/project/standard-imghdr/]: Use a redistribuição da biblioteca `imghdr` do PyPI.

- **`mailcap`**: use o módulo **`mimetypes`**.

    - standard-mailcap [https://pypi.org/project/standard-mailcap/]: Use a redistribuição da biblioteca `mailcap` do PyPI.

- **`msilib`**

- **`nis`**

- **`nntplib`**: use a biblioteca pynntp [https://pypi.org/project/pynntp/] do PyPI.

    - standard-nntplib [https://pypi.org/project/standard-nntplib/]: Use a redistribuição da biblioteca `nntplib` do PyPI.

- **`ossaudiodev`**: para reprodução de áudio, use a biblioteca pygame [https://pypi.org/project/pygame/] do PyPI.

- **`pipes`**: Use o módulo **`subprocess`** em vez disso. Use **`shlex.quote()`** para substituir a função não documentada `pipes.quote`.
    - standard-pipes [https://pypi.org/project/standard-pipes/]: Use a redistribuição da biblioteca `pipes` do PyPI.
- **`sndhdr`**: as bibliotecas filetype [https://pypi.org/project/filetype/], puremagic [https://pypi.org/project/puremagic/] ou python-magic [https://pypi.org/project/python-magic/] devem ser usadas como substitutas.
    - standard-sndhdr [https://pypi.org/project/standard-sndhdr/]: Use a redistribuição da biblioteca `sndhdr` do PyPI.
- **`spwd`**: use a biblioteca python-pam [https://pypi.org/project/python-pam/] do PyPI.
- **`sunau`**
    - standard-sunau [https://pypi.org/project/standard-sunau/]: Use a redistribuição da biblioteca `sunau` do PyPI.
- **`telnetlib`**: use as bibliotecas telnetlib3 [https://pypi.org/project/telnetlib3/] ou Exscript [https://pypi.org/project/Exscript/] do PyPI.
    - standard-telnetlib [https://pypi.org/project/standard-telnetlib/]: Use a redistribuição da biblioteca `telnetlib` do PyPI.
- **`uu`**: use o módulo **`base64`** como uma alternativa moderna.
    - standard-uu [https://pypi.org/project/standard-uu/]: Use a redistribuição da biblioteca `uu` do PyPI.
- **`xdrlib`**
    - standard-xdrlib [https://pypi.org/project/standard-xdrlib/]: Use a redistribuição da biblioteca `xdrlib` do PyPI.

(Contribuição de Victor Stinner e Zachary Ware em gh-104773 [https://github.com/python/cpython/issues/104773] e gh-104780

[https://github.com/python/cpython/issues/104780].)

2to3

- Remove o programa **2to3** e o módulo `lib2to3`, já descontinuados no Python 3.11. (Contribuição de Victor Stinner em gh-104780 [https://github.com/python/cpython/issues/104780].)

embutidos

- Remove suporte aos descritores `classmethod` encadeados (introduzidos em gh-63272 [https://github.com/python/cpython/issues/63272]). Eles não podem mais serem usados para agrupar outros descritores como `property`. O design central desse recurso apresentava falhas e levou a problemas. Para "passar" um `classmethod`, considere usar o atributo `__wrapped__` que foi adicionado no Python 3.10. (Contribuição de Raymond Hettinger em gh-89519 [https://github.com/python/cpython/issues/89519].)
- Chamar `frame.clear()` em um quadro suspenso levanta `RuntimeError` (como sempre foi o caso para um quadro em execução). (Contribuição de Irit Katriel em gh-79932 [https://github.com/python/cpython/issues/79932].)

configparser

- Remove a classe não documentada `LegacyInterpolation`, descontinuada na docstring desde Python 3.2 e em tempo de execução desde Python 3.11. (Contribuição de Hugo van Kemenade em gh-104886 [https://github.com/python/cpython/issues/104886].)

importlib.metadata

- Remove o acesso descontinuado a subscrito (`__getitem__()`) para objetos EntryPoint. (Contribuição de Jason R. Coombs em gh-113175 [https://github.com/python/cpython/issues/113175].)

locale

- Remove a função **`locale.resetlocale()`**, descontinuada no Python 3.11. Use `locale.setlocale(locale.LC_ALL, "")`. (Contribuição de Victor Stinner em gh-104783 [https://github.com/python/cpython/issues/104783].)

opcode

- Move **`opcode.ENABLE_SPECIALIZATION`** para **`_opcode.ENABLE_SPECIALIZATION`**. Este campo foi adicionado na versão 3.12, nunca foi documentado e não se destina ao uso externo. (Contribuição de Irit Katriel em gh-105481 [https://github.com/python/cpython/issues/105481].)
- Remove **`opcode.is_pseudo()`**, **`opcode.MIN_PSEUDO_OPCODE`** e **`opcode.MAX_PSEUDO_OPCODE`**, que foram adicionados no Python 3.12, mas nunca foram documentados nem expostos através de **`dis`**, e não foram planejados para serem usados externamente. (Contribuição de Irit Katriel em gh-105481 [https://github.com/python/cpython/issues/105481].)

optparse

- Este módulo não é mais considerado suavemente descontinuado. Enquanto o **`argparse`** continua sendo o preferido para novos projetos que não estão usando uma biblioteca de processamento de argumentos de linha de comando de terceiros, há aspectos da maneira como `argparse` funciona que significam que o módulo `optparse` de nível inferior pode fornecer uma base melhor para *escrever* bibliotecas de processamento de argumentos e para implementar aplicações de linha de comando que aderem mais estritamente do que `argparse` a várias convenções de processamento de linha de comando Unix que se originam no comportamento da função C **`getopt()`**. (Contribuição de Alyssa Coghlan e Serhiy Storchaka em gh-126180 [https://github.com/python/cpython/issues/126180].)

pathlib

- Remove a capacidade de usar objetos `Path` como gerenciadores de contexto. Essa funcionalidade foi descontinuada e não tinha efeito desde o Python 3.9. (Contribuição de Barney Gale em gh-83863 [https://github.com/python/cpython/issues/83863].)

re

- Remove a função `re.template()` não documentada, descontinuada e quebrada e o sinalizador `re.TEMPLATE` / `re.T`. (Contribuição de Serhiy Storchaka e Nikita Sobolev em gh-105687 [https://github.com/python/cpython/issues/105687].)

tkinter.tix

- Remove o módulo `tkinter.tix`, descontinuado no Python 3.6. A biblioteca Tix de terceiros que o módulo empacota não é mantida. (Contribuição de Zachary Ware em gh-75552 [https://github.com/python/cpython/issues/75552].)

turtle

- Remove o método `RawTurtle.settiltangle()`, descontinuado na documentação desde o Python 3.1 e em tempo de execução desde o Python 3.11. (Contribuição de Hugo van Kemenade em gh-104876 [https://github.com/python/cpython/issues/104876].)

typing

- Remove os espaços de nomes `typing.io` e `typing.re`, descontinuados desde o Python 3.8. Os itens nesses espaços de nomes podem ser importados diretamente do módulo `typing`. (Contribuição de Sebastian Rittau em gh-92871 [https://github.com/python/cpython/issues/92871].)
- Remove o método de argumento nomeado de criação de tipos `TypedDict`, descontinuados no Python 3.11. (Contribuição de Tomas Roun em gh-104786 [https://github.com/python/cpython/issues/104786].)

unittest

- Remove as seguintes funções `unittest`, descontinuadas no Python 3.11:

  - `unittest.findTestCases()`
  - `unittest.makeSuite()`
  - `unittest.getTestCaseNames()`

  Em vez delas, use os métodos de `TestLoader`:

  - `loadTestsFromModule()`
  - `loadTestsFromTestCase()`
  - `getTestCaseNames()`

  (Contribuição de Hugo van Kemenade em gh-104835 [https://github.com/python/cpython/issues/104835].)

- Remove o método `TestProgram.usageExit()` não testado e não documentado, descontinuado no Python 3.11. (Contribuição de Hugo van Kemenade em gh-104992 [https://github.com/python/cpython/issues/104992].)

urllib

- Remove os parâmetros *cafile*, *capath* e *cadefault* da função `urllib.request.urlopen()`, descontinuada no Python 3.6. Em vez disso, use o parâmetro *context* com uma instância de `SSLContext`. A função `ssl.SSLContext.load_cert_chain()` pode ser usada para carregar certificados específicos ou deixe `ssl.create_default_context()` selecionar os certificados de autoridade certificadora (AC) confiável do sistema operacional. (Contribuição de Victor Stinner em gh-105382 [https://github.com/python/cpython/issues/105382].)

webbrowser

- Remove a classe **`MacOSX`** não testada e não documentada, descontinuada no Python 3.11. Use a classe **`MacOSXOSAScript`** (introduzida no Python 3.2). (Contribuição de Hugo van Kemenade em gh-104804 [https://github.com/python/cpython/issues/104804].)
- Remove o atributo descontinuado **`MacOSXOSAScript._name`**. Use o atributo **`MacOSXOSAScript.name`**. (Contribuição de Nikita Sobolev em gh-105546 [https://github.com/python/cpython/issues/105546].)

Novas descontinuações

- Funções definidas pelo usuário:
  - A atribuição ao atributo **`__code__`** de uma função onde o tipo do novo objeto código não corresponde ao tipo da função está descontinuada. Os diferentes tipos são: função simples, gerador, gerador assíncrono e corrotina. (Contribuição de Irit Katriel em gh-81137 [https://github.com/python/cpython/issues/81137].)
- **`array`**:
  - Descontinua o código de formato `'u'` (**`wchar_t`**). Este código de formato foi descontinuado na documentação desde o Python 3.3 e será removido no Python 3.16. Use o código de formato `'w'` (**`Py_UCS4`**) para caracteres Unicode. (Contribuição de Hugo van Kemenade em gh-80480 [https://github.com/python/cpython/issues/80480].)
- **`ctypes`**:
  - Descontinua a função não documentada **`SetPointerType()`**, que será removida no Python 3.15. (Contribuição de Victor Stinner em gh-105733 [https://github.com/python/cpython/issues/105733].)
  - Descontinua suavemente a função **`ARRAY()`** em favor da multiplicação `type * length`. (Contribuição de Victor Stinner em gh-105733 [https://github.com/python/cpython/issues/105733].)
- **`decimal`**:
  - Descontinua o especificador de formato de **`Decimal`** não padrão e não documentado `'N'`, que só é suportado na implementação em C do módulo **`decimal`**. Programado para ser removido no Python 3.18. (Contribuição de Serhiy Storchaka em gh-89902 [https://github.com/python/cpython/issues/89902].)
- **`dis`**:

    - Descontinua o separador **`HAVE_ARGUMENT`**. Em vez disso, Verifique a associação em **`hasarg`**. (Contribuição de Irit Katriel em gh-109319 [https://github.com/python/cpython/issues/109319].)
- **`gettext`**:
    - Descontinua números não inteiros como argumentos para funções e métodos que consideram formas plurais no módulo **`gettext`**, ainda que nenhuma tradução tenha sido encontrada. (Contribuição de Serhiy Storchaka em gh-88434 [https://github.com/python/cpython/issues/88434].)
- **`glob`**:
    - Descontinua as funções não documentadas **`glob0()`** e **`glob1()`**. Em vez disso, use **`glob()`** e passe um objeto caminho ou similar especificando o diretório raiz para o parâmetro *root_dir*. (Contribuição de Barney Gale em gh-117337 [https://github.com/python/cpython/issues/117337].)
- **`http.server`**:
    - Deprecate **`CGIHTTPRequestHandler`**, to be removed in Python 3.15. Process-based CGI HTTP servers have been out of favor for a very long time. This code was outdated, unmaintained, and rarely used. It has a high potential for both security and functionality bugs. (Contributed by Gregory P. Smith in gh-109096 [https://github.com/python/cpython/issues/109096].)
    - Descontinua o sinalizador **`--cgi`** para a interface de linha de comando **python -m http.server**, a ser removido no Python 3.15. (Contribuição de Gregory P. Smith em gh-109096 [https://github.com/python/cpython/issues/109096].)
- **`mimetypes`**:
    - Descontinua suavemente argumentos caminho de arquivo para **`guess_type()`**, use **`guess_file_type()`** em vez disso. (Contribuição de Serhiy Storchaka em gh-66543 [https://github.com/python/cpython/issues/66543].)
- **`re`**:
    - Descontinua a passagem dos argumentos opcionais *maxsplit*, *count* ou *flags* como argumentos posicionais para as funções de nível de módulo **`split()`**, **`sub()`** e **`subn()`**. Esses parâmetros se tornarão somente-nomeados em uma versão futura do Python.

(Contribuição de Serhiy Storchaka em gh-56166 [https://github.com/python/cpython/issues/56166].)

- **`pathlib`**:
  - Deprecate **`.PurePath.is_reserved()`**, to be removed in Python 3.15. Use **`os.path.isreserved()`** to detect reserved paths on Windows. (Contributed by Barney Gale in gh-88569 [https://github.com/python/cpython/issues/88569].)
- **`platform`**:
  - Deprecate **`platform.java_ver()`**, to be removed in Python 3.15. This function is only useful for Jython support, has a confusing API, and is largely untested. (Contributed by Nikita Sobolev in gh-116349 [https://github.com/python/cpython/issues/116349].)
- **`pydoc`**:
  - Descontinua a função não documentada **`ispackage()`**. (Contribuição de Zackery Spytz em gh-64020 [https://github.com/python/cpython/issues/64020].)
- **`sqlite3`**:
  - Descontinua a passagem de mais de um argumento posicional para a função **`connect()`** e o construtor **`Connection`**. Os parâmetros restantes se tornarão somente-nomeados no Python 3.15. (Contribuição de Erlend E. Aasland em gh-107948 [https://github.com/python/cpython/issues/107948].)
  - Descontinua a passagem de nome, número de argumentos e o chamável como argumentos nomeados para **`Connection.create_function()`** e **`Connection.create_aggregate()`** Esses parâmetros se tornarão somente-posicionais no Python 3.15. (Contribuição de Erlend E. Aasland em gh-108278 [https://github.com/python/cpython/issues/108278].)
  - Descontinua a passagem do chamável de função de retorno como argumento nomeado para os métodos **`set_authorizer()`**, **`set_progress_handler()`** e **`set_trace_callback()`** da classe **`Connection`**. Os chamáveis de função de retorno se tornarão somente-posicionais no Python 3.15. (Contribuição de Erlend E. Aasland em gh-108278 [https://github.com/python/cpython/issues/108278].)
- **`sys`**:
  - Descontinua a função **`_enablelegacywindowsfsencoding()`**, para ser removida no Python 3.16. Em vez disso, use a variável de

ambiente **`PYTHONLEGACYWINDOWSFSENCODING`**. (Contribuição de Inada Naoki em gh-73427 [https://github.com/python/cpython/issues/73427].)

- **`tarfile`**:
  - Descontinua o atributo **`TarFile.tarfile`** não documentado e não utilizado, a ser removido no Python 3.16. (Contribuição em gh-115256 [https://github.com/python/cpython/issues/115256].)
- **`traceback`**:
  - Descontinua o atributo **`TracebackException.exc_type`**. Em vez disso, use **`TracebackException.exc_type_str`**. (Contribuição de Irit Katriel em gh-112332 [https://github.com/python/cpython/issues/112332].)
- **`typing`**:
  - Descontinua a não-documentada sintaxe de argumento nomeado para criação de classes **`NamedTuple`** (por exemplo, `Point = NamedTuple("Point", x=int, y=int)`), para ser removido no Python 3.15. Em vez disso, use as sintaxes baseada em classe ou a funcional. (Contribuição de Alex Waygood in gh-105566 [https://github.com/python/cpython/issues/105566].)
  - Descontinua a omissão do parâmetro *fields* ao criar uma classe **`NamedTuple`** ou **`typing.TypedDict`** e descontinua a passagem de `None` para o parâmetro *fields* de ambos os tipos. O Python 3.15 exigirá uma sequência válida para o parâmetro *fields*. Para criar uma classe NamedTuple com zero campos, use `class NT(NamedTuple): pass` ou `NT = NamedTuple("NT", ())`. Para criar uma classe TypedDict com zero campos, use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})`. (Contribuição de Alex Waygood em gh-105566 [https://github.com/python/cpython/issues/105566] e gh-105570 [https://github.com/python/cpython/issues/105570].)
  - Deprecate the **`@typing.no_type_check_decorator`** decorator function, to be removed in Python 3.15. After eight years in the **`typing`** module, it has yet to be supported by any major type checker. (Contributed by Alex Waygood in gh-106309 [https://github.com/python/cpython/issues/106309].)
  - Descontinua **`typing.AnyStr`**. No Python 3.16, isso será removido de `typing.__all__`, e uma **`DeprecationWarning`** será levantada em tempo de execução quando isso for importado ou acessado. Será removido completamente em Python 3.18. Em vez disso, use

a nova sintaxe de parâmetro de tipo. (Contribuição de Michael The em gh-107116 [https://github.com/python/cpython/issues/107116].)

- **wave**:
  - Deprecate the `getmark()`, `setmark()` and `getmarkers()` methods of the **Wave_read** and **Wave_write** classes, to be removed in Python 3.15. (Contributed by Victor Stinner in gh-105096 [https://github.com/python/cpython/issues/105096].)

Remoção pendente no Python 3.14

- **argparse**: Os parâmetros *type*, *choices* e *metavar* de **argparse.BooleanOptionalAction** foram descontinuados e serão removidos na versão 3.14. (Contribuição de Nikita Sobolev em gh-92248 [https://github.com/python/cpython/issues/92248].)

- **ast**: os seguintes recursos foram descontinuados na documentação desde Python 3.8, agora fazem com que um **DeprecationWarning** seja emitido em tempo de execução quando eles são acessados ou usados, e serão removidos no Python 3.14:

  - **ast.Num**
  - **ast.Str**
  - **ast.Bytes**
  - **ast.NameConstant**
  - **ast.Ellipsis**

  Em vez disso, use **ast.Constant**. (Contribuição de Serhiy Storchaka em gh-90953 [https://github.com/python/cpython/issues/90953].)

- **asyncio**:

  - As classes filhas dos observadores **asyncio.MultiLoopChildWatcher**, **asyncio.FastChildWatcher**, **asyncio.AbstractChildWatcher** e **asyncio.SafeChildWatcher** foram descontinuadas e serão removidas no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)

  - **`asyncio.set_child_watcher()`**, **`asyncio.get_child_watcher()`**, **`asyncio.AbstractEventLoopPolicy.set_child_watcher()`** e **`asyncio.AbstractEventLoopPolicy.get_child_watcher()`** foram descontinuados e serão removidos no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
  - O método **`get_event_loop()`** da política de laço de eventos padrão agora emite um **`DeprecationWarning`** se não houver nenhum laço de eventos atual definido e decidir criar um. (Contribuição de Serhiy Storchaka e Guido van Rossum em gh-100160 [https://github.com/python/cpython/issues/100160].)

- **`email`**: Descontinua o parâmetro *isdst* em **`email.utils.localtime()`**. (Contribuição de Alan Williams em gh-72346 [https://github.com/python/cpython/issues/72346].)

- **`importlib.abc`** descontinuou as classes:

  - **`importlib.abc.ResourceReader`**
  - **`importlib.abc.Traversable`**
  - **`importlib.abc.TraversableResources`**

  Em vez disso, use classes de **`importlib.resources.abc`**:

  - **`importlib.resources.abc.Traversable`**
  - **`importlib.resources.abc.TraversableResources`**

  (Contribuição de Jason R. Coombs e Hugo van Kemenade em gh-93963 [https://github.com/python/cpython/issues/93963].)

- **`itertools`** tinha suporte não documentado, ineficiente, historicamente cheio de bugs e inconsistente para operações de cópia, cópia profunda e serialização com pickle. Isso será removido na versão 3.14 para uma redução significativa no volume de código e na carga de manutenção. (Contribuição de Raymond Hettinger em gh-101588 [https://github.com/python/cpython/issues/101588].)

- **multiprocessing**: o método de inicialização padrão será alterado para um mais seguro no Linux, BSDs e outras plataformas POSIX não-macOS onde `'fork'` é atualmente o padrão (gh-84559 [https://github.com/python/cpython/issues/84559]). Adicionar um aviso de tempo de execução sobre isso foi considerado muito perturbador, pois não se espera que a maior parte do código se importe. Use as APIs **get_context()** ou **set_start_method()** para especificar explicitamente quando seu código *requer* `'fork'`. Veja Contextos e métodos de inicialização.

- **pathlib**: **is_relative_to()** e **relative_to()**: passar argumentos adicionais foi descontinuado.

- **pkgutil**: **pkgutil.find_loader()** e **pkgutil.get_loader()** agora levantam **DeprecationWarning**; use **importlib.util.find_spec()**. (Contribuição de Nikita Sobolev em gh-97850 [https://github.com/python/cpython/issues/97850].)

- **pty**:

  - `master_open()`: use **pty.openpty()**.
  - `slave_open()`: use **pty.openpty()**.

- **sqlite3**:

  - **version** e **version_info**.
  - **execute()** e **executemany()** se espaços reservados nomeados forem usados e *parameters* for uma sequência em vez de um **dict**.

- **urllib**: **urllib.parse.Quoter** está obsoleto: não foi planejado para ser uma API pública. (Contribuição de Gregory P. Smith em gh-88168 [https://github.com/python/cpython/issues/88168].)

Remoção pendente no Python 3.15

- O sistema de importação:

  - A definição de `__cached__` em um módulo enquanto falha na definição de `__spec__.cached` está descontinuado. No Python 3.15, `__cached__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
  - A definição de `__package__` em um módulo enquanto falha na definição de `__spec__.parent` está descontinuado. No Python 3.15, `__package__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
- `ctypes`:
  - A função não documentada `ctypes.SetPointerType()` foi descontinuada desde o Python 3.13.
- `http.server`:
  - A classe obsoleta e raramente usada `CGIHTTPRequestHandler` foi descontinuada desde o Python 3.13. Não existe substituição direta. *Qualquer coisa* é melhor que CGI para fazer a interface de um servidor web com um manipulador de requisição.
  - O sinalizador `--cgi` para a interface de linha de comando **python -m http.server** foi descontinuado desde o Python 3.13.
- `importlib`:
  - Método `load_module()`: use `exec_module()`.
- `pathlib`:
  - `.PurePath.is_reserved()` foi descontinuado desde o Python 3.13. Use `os.path.isreserved()` para detectar caminhos reservados no Windows.
- `platform`:
  - `platform.java_ver()` foi descontinuada desde o Python 3.13. Esta função é útil apenas para suporte Jython, tem uma API confusa e é amplamente não testada.
- `sysconfig`:
  - O argumento *check_home* de `sysconfig.is_python_build()` foi descontinuado desde o Python 3.12.
- `threading`:
  - `RLock()` não aceitará argumentos no Python 3.15. A passagem de quaisquer argumentos foi descontinuada a partir do Python 3.14, uma vez que a versão do Python não permite quaisquer

argumentos, mas a versão C permite qualquer número de argumentos posicionais ou nomeados, ignorando todos os argumentos.

- `types`:
  - `types.CodeType`: o acesso a **`codeobject.co_lnotab`** foi descontinuado na **PEP 626** [https://peps.python.org/pep-0626/] desde 3.10 e foi planejado para ser removido em 3.12, mas só recebeu uma `DeprecationWarning` adequada em 3.12. Pode ser removido em 3.15. (Contribuição de Nikita Sobolev em gh-101866 [https://github.com/python/cpython/issues/101866].)
- `typing`:
  - A não-documentada sintaxe de argumento nomeado para criar classes `NamedTuple` (por exemplo, `Point = NamedTuple("Point", x=int, y=int)`) foi descontinuada desde o Python 3.13. Em vez disso, use as sintaxes baseada em classe ou funcional.
  - Ao usar a sintaxe funcional de `TypedDict`s, não passar um valor para o parâmetro *fields* (`TD = TypedDict("TD")`) ou passar `None` (`TD = TypedDict("TD", None)`) foi está descontinuado desde o Python 3.13. Use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})` para criar uma classe TypedDict com nenhum campo.
  - A função decoradora **`@typing.no_type_check_decorator`** foi descontinuada desde o Python 3.13. Após oito anos no módulo `typing`, ela ainda não foi suportada por nenhum verificador de tipo importante.
- Módulos **`sre_compile`**, **`sre_constants`** e **`sre_parse`**.
- `wave`:
  - Os métodos `getmark()`, `setmark()` e `getmarkers()` das classes `Wave_read` e `Wave_write` foram descontinuados desde o Python 3.13.
- `zipimport`:
  - **`zipimport.zipimporter.load_module()`** está descontinuado desde o Python 3.10. Em vez disso, use `exec_module()`. (gh-125746 [https://github.com/python/cpython/issues/125746].)

Remoção pendente no Python 3.16

- O sistema de importação:
  - A definição de `__loader__` em um módulo enquanto falha na definição de `__spec__.loader` está descontinuado. No Python 3.16, `__loader__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.
- `array`:
  - O código de formato `'u'` (`wchar_t`) foi descontinuado na documentação desde o Python 3.3 e do ambiente de execução desde o Python 3.13. Em vez disso, use o código de formato `'w'` (`Py_UCS4`) para caracteres Unicode.
- `asyncio`:
  - `asyncio.iscoroutinefunction()` foi descontinuado e será removido no Python 3.16, use `inspect.iscoroutinefunction()` em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)
  - O sistema de políticas `asyncio` está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:
    - `asyncio.AbstractEventLoopPolicy`
    - `asyncio.DefaultEventLoopPolicy`
    - `asyncio.WindowsSelectorEventLoopPolicy`
    - `asyncio.WindowsProactorEventLoopPolicy`
    - `asyncio.get_event_loop_policy()`
    - `asyncio.set_event_loop_policy()`

    Os usuários devem usar `asyncio.run()` ou `asyncio.Runner` com *loop_factory* para usar a implementação de loop de eventos desejada.

por exemplo, para usar `asyncio.SelectorEventLoop` no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventL
```

(Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- `builtins`:
  - A inversão bit a bit em tipos booleanos, `~True` ou `~False` foi descontinuada desde o Python 3.12, pois produz resultados surpreendentes e não intuitivos (`-2` e `-1`). Em vez disso, use `not x` para a negação lógica de um booleano. No caso raro de você precisar da inversão bit a bit do inteiro subjacente, converta para `int` explicitamente (`~int(x)`).
- `functools`:
  - A chamada da implementação Python de `functools.reduce()` com *function* ou *sequence* como argumentos nomeados foi descontinuada desde o Python 3.14.
- `logging`:
  - O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)
- `mimetypes`:

    - Extensões válidas começam com um '.' ou estão vazias para `mimetypes.MimeTypes.add_type()`. Extensões sem ponto estão descontinuadas e levantarão uma exceção `ValueError` no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)

- `shutil`:
    - A exceção `ExecError` foi descontinuada desde o Python 3.14. Ela não foi usada por nenhuma função em `shutil` desde o Python 3.4, e agora é um alias de `RuntimeError`.

- `symtable`:
    - O método `Class.get_methods` foi descontinuado desde o Python 3.14.

- `sys`:
    - A função `_enablelegacywindowsfsencoding()` foi descontinuada desde o Python 3.13. Em vez disso, use a variável de ambiente `PYTHONLEGACYWINDOWSFSENCODING`.

- `sysconfig`:
    - A função `sysconfig.expand_makefile_vars()` está descontinuada desde o Python 3.14. Em vez disso, use o argumento `vars` de `sysconfig.get_paths()`.

- `tarfile`:
    - O atributo não documentado e não utilizado `TarFile.tarfile` foi descontinuado desde o Python 3.13.

Remoção pendente no Python 3.17

- `datetime`:

- **`strptime()`** chama usando uma string de formato contendo `%e` (dia do mês) sem um ano. Isso está descontinuado desde Python 3.15. (Contribuição de Stan Ulbrych em gh-70647 [https://github.com/python/cpython/issues/70647].)

- **`collections.abc`**:

  - **`collections.abc.ByteString`** está agendado para remoção no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`encodings`**:

  - Passar nomes de *codificação* não ASCII para **`encodings.normalize_encoding()`** está descontinuado e sua remoção está prevista para o Python 3.17. (Contribuição de Stan Ulbrych em gh-136702 [https://github.com/python/cpython/issues/136702].)

- **`profile`**:

- O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)

- **`webbrowser`**:

  - **`webbrowser.MacOSXOSAScript`** está descontinuado em favor de **`webbrowser.MacOS`**. (gh-137586 [https://github.com/python/cpython/issues/137586])

- **`typing`**:

  - Antes do Python 3.14, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para compatibilidade com versões anteriores, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como **`typing.get_origin()`** e **`typing.get_args()`**, em vez de depender de detalhes de implementação privada.

  - **`typing.ByteString`**, obsoleto desde o Python 3.9, está programado para ser removido no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de

`ByteString` (seja em tempo de execução ou por verificadores de tipo estáticos).

Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`tkinter`**:
  - Os métodos **`trace_variable()`**, **`trace()`** (um apelido de **`trace_variable()`**), **`trace_vdelete()`** e **`trace_vinfo()`** da classe **`tkinter.Variable`**, descontinuados desde o Python 3.14, têm remoção prevista para o Python 3.17. Em vez deles, utilize **`trace_add()`**, **`trace_remove()`** e **`trace_info()`**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

Remoção pendente no Python 3.18

- Não aceita mais um valor booleano quando um descritor de arquivo é esperado. (Contribuição de Serhiy Storchaka em gh-82626 [https://github.com/python/cpython/issues/82626].)

- **`decimal`**:
  - O especificador de formato de **`Decimal`** não padrão e não documentado `'N'`, que só é suportado na implementação C do módulo **`decimal`**, foi descontinuada desde o Python 3.13. (Contribuição de Serhiy Storchaka em gh-89902 [https://github.com/python/cpython/issues/89902].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:
  - Linhas de `import` em arquivos *`name`*`.pth` são silenciosamente ignoradas.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

Remoção pendente no Python 3.19

- `ctypes`:
  - Troca implícita para o layout de estrutura compatível com MSVC definindo `_pack_`, mas não `_layout_` em plataformas não Windows.
- `hashlib`:
  - Em construtores de funções de hash, como `new()`, ou nos construtores nomeados diretamente pelo algoritmo de hash, como `md5()` e `sha256()`, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado `data=` ou `string=` em várias implementações do **`hashlib`**.

    O suporte ao nome do argumento nomeado `string` está agora descontinuado e previsto para ser removido no Python 3.19.

    Antes do Python 3.13, o parâmetro nomeado `string` não era suportado corretamente, dependendo da implementação de funções de hash do backend. Dê preferência a passar os dados iniciais como um argumento posicional para garantir a máxima compatibilidade com versões anteriores.
- `http.cookies`:
  - `http.cookies.Morsel.js_output()` está descontinuado e será removido no Python 3.19.
  - `http.cookies.BaseCookie.js_output()` está descontinuado e será removido no Python 3.19.
- `imaplib`:
  - A alteração de `IMAP4.file` agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade agora não é utilizada e alterar seu valor não fecha automaticamente o arquivo atual.

Antes do Python 3.14, essa propriedade era usada para implementar os métodos `read()` e `readline()` correspondentes para **`IMAP4`**, mas isso não ocorre mais desde então.

Remoção pendente no Python 3.20

- Chamar o método `__new__()` de **`struct.Struct`** sem o argumento *format* está descontinuado e será removido no Python 3.20. Chamar o método **`__init__()`** em objetos **`Struct`** já inicializados está descontinuado e será removido no Python 3.20.

  (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize **`sys.version_info`** em seu lugar.

  - **`argparse`**
  - **`csv`**
  - **`ctypes`**
  - **`ctypes.macholib`**
  - **`decimal`** (use **`decimal.SPEC_VERSION`**)
  - **`http.server`**
  - **`imaplib`**
  - **`ipaddress`**
  - **`json`**
  - **`logging`** (`__date__` também descontinuado)
  - **`optparse`**
  - **`pickle`**
  - **`platform`**
  - **`re`**
  - **`socketserver`**
  - **`tabnanny`**
  - **`tarfile`**
  - **`tkinter.font`**
  - **`tkinter.ttk`**

    - `wsgiref.simple_server`
    - `xml.etree.ElementTree`
    - `xml.sax.expatreader`
    - `xml.sax.handler`
    - `zlib`

  (Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

    - Avisos são gerados para linhas `import` encontradas em arquivos *`name`*`.pth`.
    - Arquivos *`name`*`.pth` não são mais decodificados usando a codificação da localidade (*locale encoding*) por padrão. Eles **DEVEM** estar codificados em `utf-8-sig`.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

- `ast`:

    - A criação de instâncias de nós abstratos da AST (como `ast.AST` ou `ast.expr`) está descontinuada e levantará um erro no Python 3.20.

- `typing`:

    - É desencorajado realizar verificações com `isinstance()` e `issubclass()` em classes de protocolo que não foram explicitamente decoradas com `runtime_checkable()`, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso resultará em um `TypeError` no Python 3.20.

      (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

Remoção pendente em versões futuras

As APIs a seguir serão removidas no futuro, embora atualmente não haja uma data agendada para sua remoção.

- **`argparse`**:
  - O aninhamento de grupos de argumentos e o aninhamento de grupos mutuamente exclusivos estão descontinuado.
  - A passagem do argumento nomeado não documentado *prefix_chars* para **`add_argument_group()`** agora está descontinuado.
  - O conversor de tipo **`argparse.FileType`** está descontinuado.
- **`builtins`**:
  - Geradores: a assinatura `throw(type, exc, tb)` e `athrow(type, exc, tb)` está descontinuada: use `throw(exc)` e `athrow(exc)`, a assinatura com um único argumento.
  - Atualmente Python aceita literais numéricos imediatamente seguidos por palavras reservadas como, por exemplo, `0in x`, `1or x`, `0if 1else 2`. Ele permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). Um aviso de sintaxe é levantado se o literal numérico for imediatamente seguido por uma das palavras reservadas **`and`**, **`else`**, **`for`**, **`if`**, **`in`** , **`is`** e **`or`**. Em uma versão futura, será alterado para um erro de sintaxe. (gh-87999 [https://github.com/python/cpython/issues/87999])
  - Suporte para métodos `__index__()` e `__int__()` retornando tipo não-int: esses métodos serão necessários para retornar uma instância de uma subclasse estrita de **`int`**.
  - Suporte para o método `__float__()` retornando uma subclasse estrita de **`float`**: esses métodos serão necessários para retornar uma instância de **`float`**.
  - Suporte para o método `__complex__()` retornando uma subclasse estrita de **`complex`**: esses métodos serão necessários para retornar uma instância de **`complex`**.
  - Passar um número complexo como argumento *real* ou *imag* no construtor **`complex()`** agora está descontinuado; deve ser passado apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)

- **calendar**: as constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por **calendar.JANUARY** e **calendar.FEBRUARY**. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)
- **codecs**: use **open()** em vez de **codecs.open()**. (gh-133038 [https://github.com/python/cpython/issues/133038])
- **codeobject.co_lnotab**: use o método **codeobject.co_lines()**.
- **datetime**:
  - **utcnow()**: use `datetime.datetime.now(tz=datetime.UTC)`.
  - **utcfromtimestamp()**: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- **gettext**: o valor de plural deve ser um número inteiro.
- **importlib**:
  - O parâmetro *debug_override* de **cache_from_source()** foi descontinuado: use o parâmetro *optimization*.
- **importlib.metadata**:
  - Interface de tupla `EntryPoints`.
  - `None` implícito nos valores de retorno.
- **logging**: o método `warn()` foi descontinuado desde o Python 3.3, use **warning()**.
- **mailbox**: o uso da entrada StringIO e do modo de texto foi descontinuado, use BytesIO e o modo binário.
- **os**: chame **os.register_at_fork()** em um processo multithread.
- **os.path**: **os.path.commonprefix()** está descontinuada; use **os.path.commonpath()** para prefixos de caminho. A função **os.path.commonprefix()** está sendo descontinuada por possuir nome e módulo enganosos. O uso dessa função não é seguro para prefixos de caminho, apesar de ela estar incluída em um módulo voltado à manipulação de caminhos; isso significa que é fácil introduzir acidentalmente vulnerabilidades de *path traversal* (travessia de diretórios) em programas Python ao utilizá-la.
- **pydoc.ErrorDuringImport**: Um valor de tupla para o parâmetro *exc_info* foi descontinuado, use uma instância de exceção.
- **re**: regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência

numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)

- **shutil**: o parâmetro *onerror* de **rmtree()** foi descontinuado no Python 3.12; use o parâmetro *onexc*.
- Protocolos e opções de **ssl**
  - **ssl.SSLContext** sem argumento de protocolo foi descontinuado.
  - **ssl.SSLContext**: **set_npn_protocols()** e **selected_npn_protocol()** foram descontinuados, use ALPN.
  - Opções de `ssl.OP_NO_SSL*`
  - Opções de `ssl.OP_NO_TLS*`
  - `ssl.PROTOCOL_SSLv3`
  - `ssl.PROTOCOL_TLS`
  - `ssl.PROTOCOL_TLSv1`
  - `ssl.PROTOCOL_TLSv1_1`
  - `ssl.PROTOCOL_TLSv1_2`
  - `ssl.TLSVersion.SSLv3`
  - `ssl.TLSVersion.TLSv1`
  - `ssl.TLSVersion.TLSv1_1`
- Métodos de **threading**:
  - **threading.Condition.notifyAll()**: use **notify_all()**.
  - **threading.Event.isSet()**: use **is_set()**.
  - **threading.Thread.isDaemon()**, **threading.Thread.setDaemon()**: use o atributo **threading.Thread.daemon**.
  - **threading.Thread.getName()**, **threading.Thread.setName()**: use o atributo **threading.Thread.name**.
  - **threading.currentThread()**: use **threading.current_thread()**.
  - **threading.activeCount()**: use **threading.active_count()**.
- **typing.Text** (gh-92332 [https://github.com/python/cpython/issues/92332]).
- A classe interna `typing._UnionGenericAlias` não é mais usada para implementar **typing.Union**. Para preservar a compatibilidade com usuários que utilizam esta classe privada, uma correção de compatibilidade será fornecida pelo menos até a versão 3.17 do

Python. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)

- **`unittest.IsolatedAsyncioTestCase`**: foi descontinuado retornar um valor que não seja `None` de um caso de teste.
- Funções descontinuadas de **`urllib.parse`**: use **`urlparse()`**
  - `splitattr()`
  - `splithost()`
  - `splitnport()`
  - `splitpasswd()`
  - `splitport()`
  - `splitquery()`
  - `splittag()`
  - `splittype()`
  - `splituser()`
  - `splitvalue()`
  - `to_bytes()`
- **`wsgiref`**: `SimpleHandler.stdout.write()` não deve fazer gravações parciais.
- **`xml.etree.ElementTree`**: testar o valor verdade de um **`Element`** está descontinuado. Em um lançamento futuro isso sempre retornará `True`. Em vez disso, prefira os testes explícitos `len(elem)` ou `elem is not None`.
- **`sys._clear_type_cache()`** está descontinuada: use **`sys._clear_internal_caches()`**.

Alterações de bytecode do CPython

- O oparg de **`YIELD_VALUE`** agora é `1` se o yield fizer parte de um yield-from ou um await, e `0` caso contrário. O oparg de **`RESUME`** foi alterado para adicionar um bit indicando se a profundidade de exceção é 1, o que é necessário para otimizar o fechamento dos geradores. (Contribuição de Irit Katriel em gh-111354 [https://github.com/python/cpython/issues/111354].)

Alterações na API C

Novas funcionalidades

- Adiciona a API C PyMonitoring C para gerar eventos de monitoramento da **PEP 669** [https://peps.python.org/pep-0669/]:

  - `PyMonitoringState`
  - `PyMonitoring_FirePyStartEvent()`
  - `PyMonitoring_FirePyResumeEvent()`
  - `PyMonitoring_FirePyReturnEvent()`
  - `PyMonitoring_FirePyYieldEvent()`
  - `PyMonitoring_FireCallEvent()`
  - `PyMonitoring_FireLineEvent()`
  - `PyMonitoring_FireJumpEvent()`
  - `PyMonitoring_FireBranchEvent`
  - `PyMonitoring_FireCReturnEvent()`
  - `PyMonitoring_FirePyThrowEvent()`
  - `PyMonitoring_FireRaiseEvent()`
  - `PyMonitoring_FireCRaiseEvent()`
  - `PyMonitoring_FireReraiseEvent()`
  - `PyMonitoring_FireExceptionHandledEvent()`
  - `PyMonitoring_FirePyUnwindEvent()`
  - `PyMonitoring_FireStopIterationEvent()`
  - `PyMonitoring_EnterScope()`
  - `PyMonitoring_ExitScope()`

  (Contribuição de Irit Katriel em gh-111997 [https://github.com/python/cpython/issues/111997]).

- Adiciona `PyMutex`, um mutex leve que ocupa um único byte, e as novas funções `PyMutex_Lock()` e `PyMutex_Unlock()`. **`PyMutex_Lock()`** irá liberar a GIL (se atualmente mantida) se a operação precisar de bloqueio. (Contribuição de Sam Gross em gh-108724 [https://github.com/python/cpython/issues/108724].)

- Adiciona a API C PyTime, fornecendo acesso aos relógios do sistema.

  - `PyTime_t`.
  - `PyTime_MIN` e `PyTime_MAX`.

  - `PyTime_AsSecondsDouble()`.
  - `PyTime_Monotonic()`.
  - `PyTime_MonotonicRaw()`.
  - `PyTime_PerfCounter()`.
  - `PyTime_PerfCounterRaw()`.
  - `PyTime_Time()`.
  - `PyTime_TimeRaw()`.

  (Contribuição de Victor Stinner e Petr Viktorin em gh-110850 [https://github.com/python/cpython/issues/110850].)

- Adiciona a função `PyDict_ContainsString()` com o mesmo comportamento de `PyDict_Contains()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*. (Contribuição de Victor Stinner em gh-108314 [https://github.com/python/cpython/issues/108314].)

- Adiciona as funções `PyDict_GetItemRef()` e `PyDict_GetItemStringRef()` functions, que são semelhantes a `PyDict_GetItemWithError()`, mas retornam uma referência forte em vez de uma referência emprestada. Além disso, essas funções retornam `-1` em caso de erro, dispensando a necessidade de checar `PyErr_Occurred()`. (Contribuição de Victor Stinner em gh-106004 [https://github.com/python/cpython/issues/106004].)

- Adiciona a função `PyDict_SetDefaultRef()`, que é semelhante a `PyDict_SetDefault()`, mas retorna uma referência forte em vez de uma referência emprestada. Esta função retorna `-1` em caso de erro, `0` na inserção e `1` se a chave já estivesse presente no dicionário. (Contribuição de Sam Gross em gh-112066 [https://github.com/python/cpython/issues/112066].)

- Adiciona as funções `PyDict_Pop()` e `PyDict_PopString()` para remover uma chave de um dicionário e, opcionalmente, retornam o valor removido. Isto é semelhante a `dict.pop()`, mas sem o valor padrão e sem levantar `KeyError` se a chave estiver ausente. (Contribuição de Stefan Behnel e Victor Stinner em gh-111262 [https://github.com/python/cpython/issues/111262].)

- Adiciona as funções `PyMapping_GetOptionalItem()` e `PyMapping_GetOptionalItemString()` como alternativas a `PyObject_GetAttr()` e `PyObject_GetAttrString()`, respectivamente. As novas funções não levantam `KeyError` se o atributo não for encontrado no mapeamento. Essas variantes são mais convenientes e rápidas se o atributo ausente não for tratado como uma falha. (Contribuição de Serhiy Storchaka em gh-106307 [https://github.com/python/cpython/issues/106307].)

- Adiciona as funções `PyObject_GetOptionalAttr()` e `PyObject_GetOptionalAttrString()`, variantes de `PyObject_GetAttr()` e `PyObject_GetAttrString()`, respectivamente. Essas novas funções não levantam `AttributeError` se o atributo não for encontrado. Essas variantes são mais convenientes e rápidas se o atributo ausente não for tratado como uma falha. (Contribuição de Serhiy Storchaka em gh-106521 [https://github.com/python/cpython/issues/106521].)

- Adiciona a função `PyErr_FormatUnraisable()` como uma extensão para `PyErr_WriteUnraisable()`, que permite personalizar a mensagem de aviso. (Contribuição de Serhiy Storchaka em gh-108082 [https://github.com/python/cpython/issues/108082].)

- Adiciona novas funções que retornam uma referência forte em vez de uma referência emprestada para quadros locais, globais, e embutidos, como parte de PEP 667:

  - `PyEval_GetFrameBuiltins()` substitui `PyEval_GetBuiltins()`
  - `PyEval_GetFrameGlobals()` substitui `PyEval_GetGlobals()`
  - `PyEval_GetFrameLocals()` substitui `PyEval_GetLocals()`

  (Contribuição de Mark Shannon e Tian Gao em gh-74929 [https://github.com/python/cpython/issues/74929].)

- Adiciona as funções `Py_GetConstant()` e `Py_GetConstantBorrowed()` para referências fortes ou emprestadas para constantes. Por exemplo, `Py_GetConstant(Py_CONSTANT_ZERO)` retorna uma referência forte

para a constante zero. (Contribuição de Victor Stinner em gh-115754 [https://github.com/python/cpython/issues/115754].)

- Adiciona a função `PyImport_AddModuleRef()` como uma substituta para `PyImport_AddModule()` mas retorna uma referência forte em vez de uma referência emprestada. (Contribuição de Victor Stinner em gh-105922 [https://github.com/python/cpython/issues/105922].)

- Adiciona a função `Py_IsFinalizing()` para verificar se o interpretador Python principal está sendo desligado. (Contribuição de Victor Stinner em gh-108014 [https://github.com/python/cpython/issues/108014].)

- Adiciona a função `PyList_GetItemRef()` como uma substituta para `PyList_GetItem()` mas que retorna uma referência forte em vez de uma referência emprestada. (Contribuição de Sam Gross em gh-114329 [https://github.com/python/cpython/issues/114329].)

- Adiciona as funções `PyList_Extend()` e `PyList_Clear()`, espelhando os métodos `list.extend()` e `list.clear()` do Python. (Contribuição de Victor Stinner em gh-111138 [https://github.com/python/cpython/issues/111138].)

- Adiciona a função `PyLong_AsInt()`, semelhante à `PyLong_AsLong()`, mas armazena o resultado em um int em vez de um long em C. (Contribuição de Victor Stinner em gh-108014 [https://github.com/python/cpython/issues/108014].)

- Adiciona as funções `PyLong_AsNativeBytes()`, `PyLong_FromNativeBytes()` e `PyLong_FromUnsignedNativeBytes()` para simplificar a conversão entre tipos inteiros nativos e objetos `int` do Python. (Contribuição de Steve Dower em gh-111140 [https://github.com/python/cpython/issues/111140].)

- Adiciona a função `PyModule_Add()`, que é semelhante a `PyModule_AddObjectRef()` e `PyModule_AddObject()`, mas sempre rouba uma referência ao valor. (Contribuição de Serhiy Storchaka em gh-86493 [https://github.com/python/cpython/issues/86493].)

- Adiciona a função `PyObject_GenericHash()` que implementa a função hashing padrão de um objeto Python. (Contribuição de Serhiy Storchaka em gh-113024 [https://github.com/python/cpython/issues/113024].)

- Adiciona a função `Py_HashPointer()` para fazer hash de um ponteiro. (Contribuição de Victor Stinner em gh-111545 [https://github.com/python/cpython/issues/111545].)

- Adiciona as funções `PyObject_VisitManagedDict()` e `PyObject_ClearManagedDict()` que devem ser chamadas pelas funções traverse e clear de um tipo usando o sinalizador `Py_TPFLAGS_MANAGED_DICT`. O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas funções no Python 3.11 e 3.12. (Contribuição de Victor Stinner em gh-107073 [https://github.com/python/cpython/issues/107073].)

- Adiciona as funções `PyRefTracer_SetTracer()` e `PyRefTracer_GetTracer()`, que permitem rastrear a criação e destruição de objetos da mesma forma que o módulo `tracemalloc`. (Contribuição de Pablo Galindo em gh-93502 [https://github.com/python/cpython/issues/93502].)

- Adiciona a função `PySys_AuditTuple()` como uma alternativa à `PySys_Audit()`, que recebe argumentos de evento como um objeto `tuple` do Python. (Contribuição de Victor Stinner em gh-85283 [https://github.com/python/cpython/issues/85283].)

- Adiciona a função `PyThreadState_GetUnchecked()` como uma alternativa a `PyThreadState_Get()` que não mata o processo com um erro fatal se for `NULL`. O chamador é responsável por verificar se o resultado é `NULL`. (Contribuição de Victor Stinner em gh-108867 [https://github.com/python/cpython/issues/108867].)

- Adiciona a função `PyType_GetFullyQualifiedName()` para obter o nome totalmente qualificado do tipo. O nome do módulo é prefixado se `type.__module__` for uma string e não for igual a `'builtins'` ou `'__main__'`. (Contribuição de Victor Stinner em gh-111696 [https://github.com/python/cpython/issues/111696].)

- Adiciona a função `PyType_GetModuleName()` para obter o nome do módulo do tipo. Isto é equivalente a obter o atributo `type.__module__`. (Contribuição de Eric Snow e Victor Stinner em gh-111696 [https://github.com/python/cpython/issues/111696].)

- Adiciona as funções `PyUnicode_EqualToUTF8AndSize()` e `PyUnicode_EqualToUTF8()`: comparam o objeto Unicode com uma string const char* codificada em UTF-8 e retornam `1` se forem iguais ou `0` caso contrário. Essas funções não levantam exceções. (Contribuição de Serhiy Storchaka em gh-110289 [https://github.com/python/cpython/issues/110289].)

- Add the `PyWeakref_GetRef()` function as an alternative to `PyWeakref_GetObject()` that returns a strong reference or `NULL` if the referent is no longer live. (Contributed by Victor Stinner in gh-105927 [https://github.com/python/cpython/issues/105927].)

- Adiciona variantes fixas de funções que ignoram erros silenciosamente:

  - `PyObject_HasAttrWithError()` substitui `PyObject_HasAttr()`.
  - `PyObject_HasAttrStringWithError()` substitui `PyObject_HasAttrString()`.
  - `PyMapping_HasKeyWithError()` substitui `PyMapping_HasKey()`.
  - `PyMapping_HasKeyStringWithError()` substitui `PyMapping_HasKeyString()`.

  As novas funções retornam `-1` para erros e o padrão `1` para verdadeiro e `0` para falso.

  (Contribuição de Serhiy Storchaka em gh-108511 [https://github.com/python/cpython/issues/108511].)

Alterações na API C

- O parâmetro *keywords* das funções `PyArg_ParseTupleAndKeywords()` e `PyArg_VaParseTupleAndKeywords()` agora é do tipo char *const* em C e const char *const* em C++, em vez de char**. Em C++, isso

torna essas funções compatíveis com argumentos dos tipos const char *const*, const char**, ou char *const* sem uma conversão de tipo explícita. Em C, as funções aceitam apenas argumentos do tipo char *const*. Isso pode ser sobrescrito com a macro **`PY_CXX_CONST`**. (Contribuição de Serhiy Storchaka em gh-65210 [https://github.com/python/cpython/issues/65210].)

- **`PyArg_ParseTupleAndKeywords()`** agora oferece suporte a nomes de parâmetros nomeados não-ASCII. (Contribuição de Serhiy Storchaka em gh-110815 [https://github.com/python/cpython/issues/110815].)

- A função **`PyCode_GetFirstFree()`** é agora uma API instável e foi renomeada para **`PyUnstable_Code_GetFirstFree()`**. (Contribuição de Bogdan Romanyuk em gh-115781 [https://github.com/python/cpython/issues/115781].)

- As funções **`PyDict_GetItem()`**, **`PyDict_GetItemString()`**, **`PyMapping_HasKey()`**, **`PyMapping_HasKeyString()`**, **`PyObject_HasAttr()`**, **`PyObject_HasAttrString()`**, e **`PySys_GetObject()`**, que apagam todos os erros que ocorrem ao chamá-las, agora reportam tais erros usando **`sys.unraisablehook()`**. Você pode substituí-las por outras funções conforme recomendado na documentação. (Contribuição de Serhiy Storchaka em gh-106672 [https://github.com/python/cpython/issues/106672].)

- Adiciona suporte para os formatos `%T`, `%#T`, `%N` e `%#N` a **`PyUnicode_FromFormat()`**:

  - `%T`: Obtém o nome totalmente qualificado de um tipo de objeto
  - `%#T`: Como acima, mas usa dois pontos como separador
  - `%N`: Obtém o nome totalmente qualificado de um tipo
  - `%#N`: Como acima, mas usa dois pontos como separador

  Veja **PEP 737** [https://peps.python.org/pep-0737/] para uma descrição completa. (Contribuição de Victor Stinner em gh-111696 [https://github.com/python/cpython/issues/111696].)

- Você não precisa mais definir a macro `PY_SSIZE_T_CLEAN` antes de incluir `Python.h` ao usar formatos `#` em códigos de formato. APIs que aceitam os códigos de formato sempre usam `Py_ssize_t` para formatos `#`. (Contribuição de Inada Naoki em gh-104922 [https://github.com/python/cpython/issues/104922].)

- Se Python for construído em modo de depuração ou **`com asserções`**, **`PyTuple_SET_ITEM()`** e **`PyList_SET_ITEM()`** agora verificam o argumento do índice com uma asserção. (Contribuição de Victor Stinner em gh-106168 [https://github.com/python/cpython/issues/106168].)

Mudanças na API C Limitada

- As seguintes funções agora estão incluídas na API C limitada:

  - **`PyMem_RawMalloc()`**
  - **`PyMem_RawCalloc()`**
  - **`PyMem_RawRealloc()`**
  - **`PyMem_RawFree()`**
  - **`PySys_Audit()`**
  - **`PySys_AuditTuple()`**
  - **`PyType_GetModuleByDef()`**

  (Contribuição de Victor Stinner em gh-85283 [https://github.com/python/cpython/issues/85283] e gh-116936 [https://github.com/python/cpython/issues/116936].)

- Python construído com **`--with-trace-refs`** (referências de rastreamento) agora oferece suporte à API Limitada. (Contribuição de Victor Stinner em gh-108634 [https://github.com/python/cpython/issues/108634].)

APIs C removidas

- Remove diversas funções, macros, variáveis, etc com nomes prefixados por `_Py` ou `_PY` (consideradas como API privada). Se o seu projeto for afetado por uma dessas remoções e você acreditar que a API removida deve permanecer disponível, por favor abra um novo

relatório de problema para solicitar uma API C pública e adicione `cc:@vstinner` ao problema para notificar Victor Stinner. (Contribuição de Victor Stinner em gh-106320 [https://github.com/python/cpython/issues/106320].)

- Remove protocolos de buffer antigos descontinuados no Python 3.0: use Protocolo de buffer.

  - **`PyObject_CheckReadBuffer()`**: use **`PyObject_CheckBuffer()`** para testar se o objeto oferece suporte ao protocolo de buffer. Observe que **`PyObject_CheckBuffer()`** não garante que **`PyObject_GetBuffer()`** terá sucesso. Para testar se o objeto é realmente legível, veja o próximo exemplo de **`PyObject_GetBuffer()`**.

  - **`PyObject_AsCharBuffer()`**, **`PyObject_AsReadBuffer()`**: use **`PyObject_GetBuffer()`** e **`PyBuffer_Release()`**:

    ```
    Py_buffer view;
    if (PyObject_GetBuffer(obj, &view, PyBUF_SIMPLE) < 0) {
        return NULL;
    }
    // Usa `view.buf` e `view.len` para ler do buffer.
    // Você pode precisar converter buf como `(const char*)`
    PyBuffer_Release(&view);
    ```

  - **`PyObject_AsWriteBuffer()`**: use **`PyObject_GetBuffer()`** e **`PyBuffer_Release()`**:

    ```
    Py_buffer view;
    if (PyObject_GetBuffer(obj, &view, PyBUF_WRITABLE) < 0)
        return NULL;
    }
    // Usa `view.buf` e `view.len` para gravar no buffer.
    PyBuffer_Release(&view);
    ```

  (Contribuição de Inada Naoki in gh-85275 [https://github.com/python/cpython/issues/85275].)

- Remove diversas funções descontinuadas no Python 3.9:

- **`PyEval_CallObject()`**, **`PyEval_CallObjectWithKeywords()`**: use `PyObject_CallNoArgs()` ou `PyObject_Call()`.

  Aviso

  Os argumentos posicionais de `PyObject_Call()` devem ser uma `tuple` e não podem ser `NULL`, os argumentos nomeados devem ser `dict` ou `NULL`, enquanto funções removidas verificavam o tipo de argumentos e aceitavam argumentos posicionais e nomeados `NULL`. Para substituir `PyEval_CallObjectWithKeywords(func, NULL, kwargs)` por **`PyObject_Call()`**, passe uma tupla vazia como argumentos posicionais usando `PyTuple_New(0)`.

- **`PyEval_CallFunction()`**: use `PyObject_CallFunction()`.
- **`PyEval_CallMethod()`**: use `PyObject_CallMethod()`.
- **`PyCFunction_Call()`**: use `PyObject_Call()`.

(Contribuição de Victor Stinner em gh-105107 [https://github.com/python/cpython/issues/105107].)

- Remove as seguintes funções antigas da configuração de inicialização do Python, descontinuadas no Python 3.11:
  - **`PySys_AddWarnOptionUnicode()`**: use `PyConfig.warnoptions`.
  - **`PySys_AddWarnOption()`**: use `PyConfig.warnoptions`.
  - **`PySys_AddXOption()`**: use `PyConfig.xoptions`.
  - **`PySys_HasWarnOptions()`**: use `PyConfig.xoptions`.
  - **`PySys_SetPath()`**: defina `PyConfig.module_search_paths`.
  - **`Py_SetPath()`**: defina `PyConfig.module_search_paths`.
  - **`Py_SetStandardStreamEncoding()`**: defina `PyConfig.stdio_encoding`, e também talvez `PyConfig.legacy_windows_stdio` (no Windows).
  - **`_Py_SetProgramFullPath()`**: defina `PyConfig.executable`.

Em vez disso, use a nova API **`PyConfig`** da Configuração de Inicialização do Python (**PEP 587** [https://peps.python.org/pep-0587/]), adicionada ao Python 3.8. (Contribuição de Victor Stinner em gh-105145 [https://github.com/python/cpython/issues/105145].)

- Remove as funções **`PyEval_AcquireLock()`** e **`PyEval_ReleaseLock()`**, descontinuadas no Python 3.2. Eles não atualizavam o estado atual da thread. Elas podem ser substituídas por:

  - **`PyEval_SaveThread()`** e **`PyEval_RestoreThread()`**;
  - **`PyEval_AcquireThread()`** e **`PyEval_RestoreThread()`** de baixo nível;
  - ou **`PyGILState_Ensure()`** e **`PyGILState_Release()`**.

  (Contribuição de Victor Stinner em gh-105182 [https://github.com/python/cpython/issues/105182].)

- Remove a função **`PyEval_ThreadsInitialized()`**, descontinuada no Python 3.9. Desde Python 3.7, **`Py_Initialize()`** sempre cria a GIL: chamar **`PyEval_InitThreads()`** não faz nada e **`PyEval_ThreadsInitialized()`** sempre retorna diferente de zero. (Contribuição de Victor Stinner em gh-105182 [https://github.com/python/cpython/issues/105182].)

- Remove o apelido **`_PyInterpreterState_Get()`** de **`PyInterpreterState_Get()`** que era mantido para compatibilidade com versões anteriores do Python 3.8. O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter **`PyInterpreterState_Get()`** no Python 3.8 e versões anteriores. (Contribuição de Victor Stinner em gh-106320 [https://github.com/python/cpython/issues/106320].)

- Remove a função privada **`_PyObject_FastCall()`**: use **`PyObject_Vectorcall()`**, a qual está disponível desde Python 3.8 (**PEP 590** [https://peps.python.org/pep-0590/]). (Contribuição de Victor Stinner em gh-106023 [https://github.com/python/cpython/issues/106023].)

- Remove o arquivo de cabeçalho `cpython/pytime.h`, que continha apenas funções privadas. (Contribuição de Victor Stinner em gh-106316 [https://github.com/python/cpython/issues/106316].)

- Remove a constante não documentada `PY_TIMEOUT_MAX` da API C limitada. (Contribuição de Victor Stinner em gh-110014 [https://github.com/python/cpython/issues/110014].)

- Remove as antigas macros de trashcan `Py_TRASHCAN_SAFE_BEGIN` e `Py_TRASHCAN_SAFE_END`. Substitua ambas pelas novas macros `Py_TRASHCAN_BEGIN` e `Py_TRASHCAN_END`. (Contribuição de Irit Katriel em gh-105111 [https://github.com/python/cpython/issues/105111].)

APIs C descontinuadas

- Descontinua antigas funções de inicialização do Python:

  - **`PySys_ResetWarnOptions()`**: Limpe **`sys.warnoptions`** e **`warnings.filters`**.
  - **`Py_GetExecPrefix()`**: Get **`sys.exec_prefix`** instead.
  - **`Py_GetPath()`**: Get **`sys.path`** instead.
  - **`Py_GetPrefix()`**: Get **`sys.prefix`** instead.
  - **`Py_GetProgramFullPath()`**: Get **`sys.executable`** instead.
  - **`Py_GetProgramName()`**: Get **`sys.executable`** instead.
  - **`Py_GetPythonHome()`**: Get **`PyConfig.home`** or the **`PYTHONHOME`** environment variable instead.

  (Contribuição de Victor Stinner em gh-105145 [https://github.com/python/cpython/issues/105145].)

- Descontinua suavemente as funções **`PyEval_GetBuiltins()`**, **`PyEval_GetGlobals()`** e **`PyEval_GetLocals()`**, que retornam uma referência emprestada. (Suavemente descontinuadas como parte da **PEP 667** [https://peps.python.org/pep-0667/].)

- Deprecate the **`PyImport_ImportModuleNoBlock()`** function, which is just an alias to **`PyImport_ImportModule()`** since Python 3.3.

(Contributed by Victor Stinner in gh-105396 [https://github.com/python/cpython/issues/105396].)

- Descontinua suavemente a função **`PyModule_AddObject()`**. Ela deve ser substituída por **`PyModule_Add()`** ou **`PyModule_AddObjectRef()`**. (Contribuição de Serhiy Storchaka em gh-86493 [https://github.com/python/cpython/issues/86493].)

- Descontinua os tipos antigos `Py_UNICODE` e `PY_UNICODE_TYPE` e define **`Py_UNICODE_WIDE`**. Em vez disso, use o tipo **`wchar_t`** diretamente. Desde Python 3.3, `Py_UNICODE` e `PY_UNICODE_TYPE` são apenas apelidos para **`wchar_t`**. (Contribuição de Victor Stinner em gh-105156 [https://github.com/python/cpython/issues/105156].)

- Deprecate the **`PyWeakref_GetObject()`** and **`PyWeakref_GET_OBJECT()`** functions, which return a borrowed reference. Replace them with the new **`PyWeakref_GetRef()`** function, which returns a strong reference. The pythoncapi-compat project [https://github.com/python/pythoncapi-compat/] can be used to get **`PyWeakref_GetRef()`** on Python 3.12 and older. (Contributed by Victor Stinner in gh-105927 [https://github.com/python/cpython/issues/105927].)

Remoção pendente no Python 3.14

- O campo `ma_version_tag` em **`PyDictObject`** para módulos de extensão (**PEP 699** [https://peps.python.org/pep-0699/]; gh-101193 [https://github.com/python/cpython/issues/101193]).
- A criação de **`tipos imutáveis`** com bases mutáveis (gh-95388 [https://github.com/python/cpython/issues/95388]).

Remoção pendente no Python 3.15

- A **`PyImport_ImportModuleNoBlock()`**: Use **`PyImport_ImportModule()`**

- **`PyWeakref_GetObject()`** e **`PyWeakref_GET_OBJECT()`**: Use **`PyWeakref_GetRef()`**. O projeto pythoncapi-compat

[https://github.com/python/pythoncapi-compat/] pod ser usado para obter **`PyWeakref_GetRef()`** no Python 3.12 e versões mais antigas.

- **`PyUnicode_AsDecodedObject()`**: use **`PyCodec_Decode()`**.

- **`PyUnicode_AsDecodedUnicode()`**: Use **`PyCodec_Decode()`**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **`str`**, tal como **`bytes`**.

- **`PyUnicode_AsEncodedObject()`**: use **`PyCodec_Encode()`**.

- **`PyUnicode_AsEncodedUnicode()`**: Use **`PyCodec_Encode()`**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **`bytes`**, tal com **`str`**.

- Funções de inicialização do Python, descontinuadas no Python 3.9:

  - **`Py_GetPath()`**: Use **`PyConfig_Get("module_search_paths")`** (**`sys.path`**).
  - **`Py_GetPrefix()`**: Use **`PyConfig_Get("base_prefix")`** (**`sys.base_prefix`**). Use **`PyConfig_Get("prefix")`** (**`sys.prefix`**) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetExecPrefix()`**: Use **`PyConfig_Get("base_exec_prefix")`** (**`sys.base_exec_prefix`**). Use **`PyConfig_Get("exec_prefix")`** (**`sys.exec_prefix`**) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetProgramFullPath()`**: Use **`PyConfig_Get("executable")`** (**`sys.executable`**).
  - **`Py_GetProgramName()`**: Use **`PyConfig_Get("executable")`** (**`sys.executable`**).
  - **`Py_GetPythonHome()`**: Use **`PyConfig_Get("home")`** ou a variável de ambiente **`PYTHONHOME`**.

  O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter **`PyConfig_Get()`** no Python 3.13 e versões anteriores.

- Funções para configurar a inicialização do Python, descontinuadas no Python 3.11:

- **`PySys_SetArgvEx()`**: defina `PyConfig.argv`.
- **`PySys_SetArgv()`**: defina `PyConfig.argv`.
- **`Py_SetProgramName()`**: defina `PyConfig.program_name`.
- **`Py_SetPythonHome()`**: defina `PyConfig.home`.
- **`PySys_ResetWarnOptions()`**: Limpe `sys.warnoptions` e **`warnings.filters`**.

Em vez disso, a API `Py_InitializeFromConfig()` deve ser usada com `PyConfig`.

- Variáveis de configuração globais

  - `Py_DebugFlag`: Use `PyConfig.parser_debug` ou `PyConfig_Get("parser_debug")`.
  - `Py_VerboseFlag`: Use `PyConfig.verbose` ou `PyConfig_Get("verbose")`.
  - `Py_QuietFlag`: Use `PyConfig.quiet` ou `PyConfig_Get("quiet")`.
  - `Py_InteractiveFlag`: Use `PyConfig.interactive` ou `PyConfig_Get("interactive")`.
  - `Py_InspectFlag`: Use `PyConfig.inspect` ou `PyConfig_Get("inspect")`.
  - `Py_OptimizeFlag`: Use `PyConfig.optimization_level` ou `PyConfig_Get("optimization_level")`.
  - `Py_NoSiteFlag`: Use `PyConfig.site_import` ou `PyConfig_Get("site_import")`.
  - `Py_BytesWarningFlag`: Use `PyConfig.bytes_warning` ou `PyConfig_Get("bytes_warning")`.
  - `Py_FrozenFlag`: Use `PyConfig.pathconfig_warnings` ou `PyConfig_Get("pathconfig_warnings")`.
  - `Py_IgnoreEnvironmentFlag`: Use `PyConfig.use_environment` ou `PyConfig_Get("use_environment")`.
  - `Py_DontWriteBytecodeFlag`: Use `PyConfig.write_bytecode` ou `PyConfig_Get("write_bytecode")`.
  - `Py_NoUserSiteDirectory`: Use `PyConfig.user_site_directory` ou `PyConfig_Get("user_site_directory")`.

- `Py_UnbufferedStdioFlag`: Use `PyConfig.buffered_stdio` ou `PyConfig_Get("buffered_stdio")`.
- `Py_HashRandomizationFlag`: Use `PyConfig.use_hash_seed` e `PyConfig.hash_seed` ou `PyConfig_Get("hash_seed")`.
- `Py_IsolatedFlag`: Use `PyConfig.isolated` ou `PyConfig_Get("isolated")`.
- `Py_LegacyWindowsFSEncodingFlag`: Use `PyPreConfig.legacy_windows_fs_encoding` ou `PyConfig_Get("legacy_windows_fs_encoding")`.
- `Py_LegacyWindowsStdioFlag`: Use `PyConfig.legacy_windows_stdio` ou `PyConfig_Get("legacy_windows_stdio")`.
- **`Py_FileSystemDefaultEncoding`**, **`Py_HasFileSystemDefaultEncoding`**: Use `PyConfig.filesystem_encoding` ou `PyConfig_Get("filesystem_encoding")`.
- **`Py_FileSystemDefaultEncodeErrors`**: Use `PyConfig.filesystem_errors` ou `PyConfig_Get("filesystem_errors")`.
- **`Py_UTF8Mode`**: Use `PyPreConfig.utf8_mode` ou `PyConfig_Get("utf8_mode")`. (veja `Py_PreInitialize()`)

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig` para definir essas opções. Ou `PyConfig_Get()` pode ser usado para obter essas opções em tempo de execução.

Remoção pendente no Python 3.16

- A cópia empacotada do `libmpdec`.

Remoção pendente no Python 3.18

- As seguintes funções privadas estão descontinuado e planejadas para remoção no Python 3.18:
  - **`_PyBytes_Join()`**: use `PyBytes_Join()`.

- **`_PyDict_GetItemStringWithError()`**: use `PyDict_GetItemStringRef()`.
- **`_PyDict_Pop()`**: use `PyDict_Pop()`.
- **`_PyLong_Sign()`**: use `PyLong_GetSign()`.
- **`_PyLong_FromDigits()`** e **`_PyLong_New()`**: use `PyLongWriter_Create()`.
- **`_PyThreadState_UncheckedGet()`**: use `PyThreadState_GetUnchecked()`.
- **`_PyUnicode_AsString()`**: use `PyUnicode_AsUTF8()`.
- **`_PyUnicodeWriter_Init()`**: substitua `_PyUnicodeWriter_Init(&writer)` com `writer = PyUnicodeWriter_Create(0)`.
- **`_PyUnicodeWriter_Finish()`**: substitua `_PyUnicodeWriter_Finish(&writer)` por `PyUnicodeWriter_Finish(writer)`.
- **`_PyUnicodeWriter_Dealloc()`**: substitua `_PyUnicodeWriter_Dealloc(&writer)` por `PyUnicodeWriter_Discard(writer)`.
- **`_PyUnicodeWriter_WriteChar()`**: substituta `_PyUnicodeWriter_WriteChar(&writer, ch)` com `PyUnicodeWriter_WriteChar(writer, ch)`.
- **`_PyUnicodeWriter_WriteStr()`**: substitua `_PyUnicodeWriter_WriteStr(&writer, str)` por `PyUnicodeWriter_WriteStr(writer, str)`.
- **`_PyUnicodeWriter_WriteSubstring()`**: substitua `_PyUnicodeWriter_WriteSubstring(&writer, str, start, end)` por `PyUnicodeWriter_WriteSubstring(writer, str, start, end)`.
- **`_PyUnicodeWriter_WriteASCIIString()`**: substitua `_PyUnicodeWriter_WriteASCIIString(&writer, str)` por `PyUnicodeWriter_WriteASCII(writer, str)`.
- **`_PyUnicodeWriter_WriteLatin1String()`**: substitua `_PyUnicodeWriter_WriteLatin1String(&writer, str)` por `PyUnicodeWriter_WriteUTF8(writer, str)`.
- **`_PyUnicodeWriter_Prepare()`**: (sem substituto).
- **`_PyUnicodeWriter_PrepareKind()`**: (sem substituto).
- **`_Py_HashPointer()`**: use `Py_HashPointer()`.

- **`_Py_fopen_obj()`**: use **`Py_fopen()`**.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas novas funções públicas no Python 3.13 e versões anteriores. (Contribuição de Victor Stinner em gh-128863 [https://github.com/python/cpython/issues/128863].)

Remoção pendente em versões futuras

As APIs a seguir foram descontinuadas e serão removidas, embora atualmente não haja uma data agendada para sua remoção.

- **`Py_TPFLAGS_HAVE_FINALIZE`**: desnecessária desde o Python 3.8.
- **`PyErr_Fetch()`**: use **`PyErr_GetRaisedException()`**.
- **`PyErr_NormalizeException()`**: use **`PyErr_GetRaisedException()`**.
- **`PyErr_Restore()`**: use **`PyErr_SetRaisedException()`**.
- **`PyModule_GetFilename()`**: use **`PyModule_GetFilenameObject()`**.
- **`PyOS_AfterFork()`**: use **`PyOS_AfterFork_Child()`**.
- **`PySlice_GetIndicesEx()`**: use **`PySlice_Unpack()`** e **`PySlice_AdjustIndices()`**.
- **`PyUnicode_READY()`**: desnecessário desde o Python 3.12
- **`PyErr_Display()`**: use **`PyErr_DisplayException()`**.
- **`_PyErr_ChainExceptions()`**: use **`_PyErr_ChainExceptions1()`**.
- O membro **`PyBytesObject.ob_shash`**: chame **`PyObject_Hash()`**.
- API do Thread Local Storage (TLS):
  - **`PyThread_create_key()`**: use **`PyThread_tss_alloc()`**.
  - **`PyThread_delete_key()`**: use **`PyThread_tss_free()`**.
  - **`PyThread_set_key_value()`**: use **`PyThread_tss_set()`**.
  - **`PyThread_get_key_value()`**: use **`PyThread_tss_get()`**.
  - **`PyThread_delete_key_value()`**: use **`PyThread_tss_delete()`**.
  - **`PyThread_ReInitTLS()`**: desnecessário desde o Python 3.7.

Mudanças na construção

- `arm64-apple-ios` e `arm64-apple-ios-simulator` são agora plataformas de nível 3 da **PEP 11** [https://peps.python.org/pep-0011/]. (Escrita

e implementação da PEP 730 foi uma contribuição de Russell Keith-Magee em gh-114099 [https://github.com/python/cpython/issues/114099].)

- `aarch64-linux-android` e `x86_64-linux-android` são agora plataformas de nível 3 da **PEP 11** [https://peps.python.org/pep-0011/]. (Escrita e implementação da PEP 738 foi uma contribuição de Malcolm Smith em gh-116622 [https://github.com/python/cpython/issues/116622].)
- `wasm32-wasi` agora é uma plataforma de nível 2 segundo a **PEP 11** [https://peps.python.org/pep-0011/]. (Contribuição de Brett Cannon em gh-115192 [https://github.com/python/cpython/issues/115192].)
- `wasm32-emscripten` não é mais uma plataforma suportada pela **PEP 11** [https://peps.python.org/pep-0011/]. (Contribuição de Brett Cannon em gh-115192 [https://github.com/python/cpython/issues/115192].)
- Construir CPython agora requer um compilador com suporte para a biblioteca atômica C11, funções atômicas embutidas do GCC ou intrínsecos interligados MSVC.
- Autoconf 2.71 e aclocal 1.16.5 agora são necessários para regerar o script `configure`. (Contribuição de Christian Heimes em gh-89886 [https://github.com/python/cpython/issues/89886] e de Victor Stinner em gh-112090 [https://github.com/python/cpython/issues/112090].)
- SQLite 3.15.2 ou mais recente é necessário para construir o módulo de extensão **`sqlite3`**. (Contribuição de Erlend Aasland em gh-105875 [https://github.com/python/cpython/issues/105875].)
- CPython agora inclui a biblioteca mimalloc [https://github.com/microsoft/mimalloc/] por padrão. Está licenciada sob a licença do MIT; veja licença do mimalloc. O mimalloc incluído tem alterações personalizadas, veja gh-113141 [https://github.com/python/cpython/issues/113141] para detalhes. (Contribuição de Dino Viehland em gh-109914 [https://github.com/python/cpython/issues/109914].)
- A opção **`--with-system-libmpdec`** do `configure` agora tem como padrão `yes`. A cópia empacotada de `libmpdec` será removida no Python 3.16.
- Python construído com **`--with-trace-refs`** (referências de rastreamento) do `configure` tem compatibilidade de ABI com a construção de lançamento do Python e a construção de depuração. (Contribuição de Victor Stinner em gh-108634 [https://github.com/python/cpython/issues/108634].)

- Em sistemas POSIX, os nomes de arquivos pkg-config (`.pc`) agora incluem os sinalizadores de ABI. Por exemplo, a construção com threads livres gera `python-3.13t.pc` e a construção de depuração gera `python-3.13d.pc`.
- As extensões C `errno`, `fcntl`, `grp`, `md5`, `pwd`, `resource`, `termios`, `winsound`, `_ctypes_test`, `_multiprocessing.posixshmem`, `_scproxy`, `_stat`, `_statistics`, `_testconsole`, `_testimportmultiple` e `_uuid` são agora construídas com a API C limitada. (Contribuição de Victor Stinner em gh-85283 [https://github.com/python/cpython/issues/85283].)

Portando para o Python 3.13

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações na API Python

- PEP 667 introduz várias mudanças na semântica de **`locals()`** e **`f_locals`**:
    - Chamar **`locals()`** em um escopo otimizado agora produz uma captura independente em cada chamada e, portanto, não atualiza mais implicitamente as referências retornadas anteriormente. Obter o comportamento legado do CPython agora requer chamadas explícitas para atualizar o dicionário inicialmente retornado com os resultados de chamadas subsequentes para **`locals()`**. Funções de execução de código que visam implicitamente **`locals()`** (como `exec` e `eval`) devem receber um espaço de nomes explícito para acessar seus resultados em um escopo otimizado. (Alterado como parte da **PEP 667** [https://peps.python.org/pep-0667/].)
    - Chamar **`locals()`** de uma compreensão no módulo ou escopo de classe (inclusive via `exec` ou `eval`) mais uma vez se comporta como se a compreensão estivesse sendo executada como uma função aninhada independente (isto é, as variáveis locais de o escopo que contém não está incluído). No Python 3.12, isso mudou para incluir as variáveis locais do escopo que o contém ao

implementar a **PEP 709** [https://peps.python.org/pep-0709/]. (Alterado como parte da **PEP 667** [https://peps.python.org/pep-0667/].)
  - Acessar `FrameType.f_locals` em um escopo otimizado agora retorna um proxy de "write-through" em vez de uma captura que é atualizado em horários mal especificados. Se uma captura for desejada, ela deve ser criada explicitamente com `dict` ou com o método `.copy()` do proxy. (Alterado como parte da **PEP 667** [https://peps.python.org/pep-0667/].)

- `functools.partial` agora emite uma `FutureWarning` quando é usado como método. Seu comportamento será alterado em versões futuras do Python. Envolva-a em `staticmethod()` se quiser preservar o comportamento antigo. (Contribuição de Serhiy Storchaka em gh-121027 [https://github.com/python/cpython/issues/121027].)

- Uma exceção `OSError` agora é levantada por `getpass.getuser()` para qualquer falha na recuperação de um nome de usuário, em vez de `ImportError` em plataformas não-Unix ou `KeyError` em plataformas Unix onde o banco de dados de senhas está vazio.

- O valor do atributo `mode` de `gzip.GzipFile` agora é uma string (`'rb'` ou `'wb'`) e não mais um inteiro (`1` ou `2`). O valor do atributo `mode` do objeto arquivo ou similar legível retornado por `zipfile.ZipFile.open()` é agora `'rb'` e não mais `'r'`. (Contribuição de Serhiy Storchaka em gh-115961 [https://github.com/python/cpython/issues/115961].)

- `mailbox.Maildir` agora ignora arquivos com um ponto inicial (`.`). (Contribuição de Zackery Spytz em gh-65559 [https://github.com/python/cpython/issues/65559].)

- `pathlib.Path.glob()` e `rglob()` agora retornam arquivos e diretórios se um padrão que termina com "`**`" for fornecido, em vez de apenas diretórios. Os usuários podem adicionar uma barra final para manter o comportamento anterior e corresponder apenas aos diretórios.

- O módulo `threading` agora espera que o módulo `_thread` tenha uma função `_is_main_interpreter`. É uma função sem argumentos que

retorna `True` se o interpretador atual for o interpretador principal.

Qualquer biblioteca ou aplicação que forneça um módulo **`_thread`** personalizado deve obrigatoriamente fornecer **`_is_main_interpreter()`**, da mesma forma que outros atributos "privados" do módulo. (Veja gh-112826 [https://github.com/python/cpython/issues/112826].)

Alterações na API C

- `Python.h` não inclui mais o cabeçalho padrão `<ieeefp.h>`. Ele foi incluído para a função **`finite()`** que agora é fornecida pelo cabeçalho `<math.h>`. Agora deve ser incluído explicitamente, se necessário. Remove também a macro `HAVE_IEEEFP_H`. (Contribuição de Victor Stinner em gh-108765 [https://github.com/python/cpython/issues/108765].)

- `Python.h` não inclui mais estes arquivos de cabeçalho padrão: `<time.h>`, `<sys/select.h>` e `<sys/time.h>`. Se necessário, deverão agora ser incluídos explicitamente. Por exemplo, `<time.h>` fornece as funções **`clock()`** e **`gmtime()`**, `<sys/select.h>` fornece o **`select()`**, e `<sys/time.h>` fornece as funções **`futimes()`**, **`gettimeofday()`** e **`setitimer()`**. (Contribuição de Victor Stinner em gh-108765 [https://github.com/python/cpython/issues/108765].)

- No Windows, `Python.h` não inclui mais o arquivo de cabeçalho padrão `<stddef.h>`. Se necessário, deverá agora ser incluído explicitamente. Por exemplo, ele fornece a função **`offsetof()`** e os tipos `size_t` e `ptrdiff_t`. Incluindo `<stddef.h>` explicitamente já era necessário para todas as outras plataformas, a macro `HAVE_STDDEF_H` só é definida no Windows. (Contribuição de Victor Stinner em gh-108765 [https://github.com/python/cpython/issues/108765].)

- Se a macro **`Py_LIMITED_API`** estiver definida, as macros **`Py_BUILD_CORE`**, **`Py_BUILD_CORE_BUILTIN`** e **`Py_BUILD_CORE_MODULE`** agora são indefinidas por `<Python.h>`. (Contribuição de Victor Stinner em gh-85283 [https://github.com/python/cpython/issues/85283].)

- As macros antigas da lixeira `Py_TRASHCAN_SAFE_BEGIN` e `Py_TRASHCAN_SAFE_END` foram removidas. Elas devem ser substituídas pelas novas macros `Py_TRASHCAN_BEGIN` e `Py_TRASHCAN_END`.

  Uma função `tp_dealloc` que contém as macros antigas, como:

  ```
  static void
  dealocador_de_meutipo(meutipo *p)
  {
      PyObject_GC_UnTrack(p);
      Py_TRASHCAN_SAFE_BEGIN(p);
      ...
      Py_TRASHCAN_SAFE_END
  }
  ```

  deve migrar para as novas macros da seguinte forma:

  ```
  static void
  dealocador_de_meutipo(meutipo *p)
  {
      PyObject_GC_UnTrack(p);
      Py_TRASHCAN_BEGIN(p, dealocador_de_meutipo);
      ...
      Py_TRASHCAN_END
  }
  ```

  Observe que `Py_TRASHCAN_BEGIN` tem um segundo argumento que deve ser a função de desalocação em que está. As novas macros foram adicionadas no Python 3.8 e as macros antigas foram descontinuadas no Python 3.11. (Contribuição de Irit Katriel em gh-105111 [https://github.com/python/cpython/issues/105111].)

- PEP 667 introduz várias mudanças em funções relacionadas a quadro:

  - Os efeitos da mutação do dicionário retornado de **`PyEval_GetLocals()`** em um escopo otimizado foram alterados. Novas entradas de dict adicionadas desta forma agora *apenas* ficarão visíveis para chamadas **`PyEval_GetLocals()`** subsequentes nesse quadro, como **`PyFrame_GetLocals()`**, **`locals()`** e **`FrameType.f_locals`** não acessa mais o mesmo

dicionário armazenado em cache subjacente. As alterações feitas nas entradas para nomes de variáveis reais e nomes adicionados através das interfaces proxy de “write-through” serão substituídas em chamadas subsequentes para **`PyEval_GetLocals()`** nesse quadro. A atualização de código recomendada depende de como a função estava sendo usada, portanto, consulte o aviso de descontinuação na função para obter detalhes.
  - Chamar **`PyFrame_GetLocals()`** em um escopo otimizado agora retorna um proxy de “write-through” em vez de uma captura que é atualizada em horários mal especificados. Se uma captura for desejada, ela deve ser criada explicitamente (por exemplo, com **`PyDict_Copy()`**) ou chamando a nova API **`PyEval_GetFrameLocals()`**.
  - **`PyFrame_FastToLocals()`** e **`PyFrame_FastToLocalsWithError()`** não têm mais nenhum efeito. Chamar essas funções tem sido redundante desde o Python 3.11, quando **`PyFrame_GetLocals()`** foi introduzido pela primeira vez.
  - **`PyFrame_LocalsToFast()`** não tem mais efeito. Chamar esta função é redundante agora que **`PyFrame_GetLocals()`** retorna um proxy de “write-through” para escopos otimizados.

- O Python 3.13 removeu muitas funções privadas. Algumas delas podem ser substituídas usando estas alternativas:

  - `_PyDict_Pop()`: **`PyDict_Pop()`** ou **`PyDict_PopString()`**;
  - `_PyDict_GetItemWithError()`: **`PyDict_GetItemRef()`**;
  - `_PyErr_WriteUnraisableMsg()`: **`PyErr_FormatUnraisable()`**;
  - `_PyEval_SetTrace()`: **`PyEval_SetTrace()`** ou **`PyEval_SetTraceAllThreads()`**;
  - `_PyList_Extend()`: **`PyList_Extend()`**;
  - `_PyLong_AsInt()`: **`PyLong_AsInt()`**;
  - `_PyMem_RawStrdup()`: `strdup()`;
  - `_PyMem_Strdup()`: `strdup()`;
  - `_PyObject_ClearManagedDict()`: **`PyObject_ClearManagedDict()`**;
  - `_PyObject_VisitManagedDict()`: **`PyObject_VisitManagedDict()`**;

- `_PyThreadState_UncheckedGet()`: **`PyThreadState_GetUnchecked()`**;
- `_PyTime_AsSecondsDouble()`: **`PyTime_AsSecondsDouble()`**;
- `_PyTime_GetMonotonicClock()`: **`PyTime_Monotonic()`** ou **`PyTime_MonotonicRaw()`**;
- `_PyTime_GetPerfCounter()`: **`PyTime_PerfCounter()`** ou **`PyTime_PerfCounterRaw()`**;
- `_PyTime_GetSystemClock()`: **`PyTime_Time()`** ou **`PyTime_TimeRaw()`**;
- `_PyTime_MAX`: **`PyTime_MAX`**;
- `_PyTime_MIN`: **`PyTime_MIN`**;
- `_PyTime_t`: **`PyTime_t`**;
- `_Py_HashPointer()`: **`Py_HashPointer()`**;
- `_Py_IsFinalizing()`: **`Py_IsFinalizing()`**.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter a maioria dessas novas funções no Python 3.12 e versões anteriores.

Mudanças em teste de regressão

- Python construído com **`--with-pydebug`** do `configure` agora oferece suporte a uma opção de linha de comando **`-X presite=pacote.módulo`**. Se usado, especifica um módulo que deve ser importado no início do ciclo de vida do interpretador, antes que `site.py` seja executado. (Contribuição de Łukasz Langa em gh-110769 [https://github.com/python/cpython/issues/110769].)

O que há de novo no Python 3.12

Editor:: Adam Turner

Este artigo explica os novos recursos no Python 3.12, em comparação com 3.11. Python 3.12 foi lançado em 2 de outubro de 2023. Veja changelog para uma lista completa de mudanças.

Ver também

PEP 693 [https://peps.python.org/pep-0693/] – Agendamento de lançamento do Python 3.12

Resumo – Destaques da versão

Python 3.12 é uma versão estável da linguagem de programação Python, com uma combinação de alterações na linguagem e na biblioteca padrão. As alterações da biblioteca se concentram na limpeza de APIs descontinuadas, usabilidade e correção. É importante notar que o pacote `distutils` foi removido da biblioteca padrão. O suporte ao sistema de arquivos em `os` e `pathlib` teve uma série de melhorias e vários módulos têm melhor desempenho.

As mudanças de linguagem se concentram na usabilidade, já que f-strings tiveram muitas limitações removidas e as sugestões 'Did you mean …' continuam a melhorar. A nova sintaxe de parâmetro type e a instrução `type` melhoram a ergonomia para usar tipos genéricos e apelidos de tipos com verificadores de tipo estáticos.

Este artigo não tenta fornecer uma especificação completa de todos os novos recursos, mas fornece uma visão geral conveniente. Para detalhes completos, você deve consultar a documentação, como Referência da Biblioteca e Referência da Linguagem. Se você quiser entender a implementação completa e a justificativa do design para uma mudança, consulte a PEP para um novo recurso específico; mas observe que as PEPs geralmente não são mantidas atualizadas depois que um recurso é totalmente implementado.

Novos recursos de sintaxe:

- PEP 695, sintaxe de parâmetro de tipo e a instrução `type`

Novos recursos de gramática:

- PEP 701, f-strings na gramática

Melhorias no interpretador:

- PEP 684, uma única GIL por interpretador
- PEP 669, monitoramento de baixo impacto
- Aprimoradas as sugestões 'Did you mean …' para as exceções `NameError`, `ImportError` e `SyntaxError`

Melhorias no modelo de dados Python:

- PEP 688, usando o protocolo de buffer do Python

Melhorias significativas na biblioteca padrão:

- A classe `pathlib.Path` agora oferece suporte a fazer subclasses
- O módulo `os` recebeu vários aprimoramentos para suporte ao Windows
- Uma interface de linha de comando foi adicionada ao módulo `sqlite3`
- As verificações de `isinstance()` para `protocolos verificáveis em tempo de execução` desfruta de um aumento de velocidade entre duas e 20 vezes
- O pacote `asyncio` teve vários aprimoramentos desempenho, com alguns benchmarks mostrando um aumento de 75% na velocidade.
- Uma interface de linha de comando foi adicionada ao módulo `uuid`
- Devido às alterações na PEP 701, a produção de tokens por meio do módulo `tokenize` é até 64% mais rápida.

Melhorias de segurança:

- Substitui as implementações embutidas do `hashlib` de SHA1, SHA3, SHA2-384, SHA2-512 e MD5 pelo código formalmente verificado do projeto HACL* [https://github.com/hacl-star/hacl-star/]. Essas implementações embutidas permanecem como fallbacks que são usados somente quando o OpenSSL não as fornece.

Melhorias na API C:

- PEP 697, tier instável de API C
- PEP 683, objetos imortal

Melhorias na implementação do CPython:

- PEP 709, inlining de compreensão
- Suporte do CPython para o perfilador Linux `perf`
- Implementa proteção contra estouro de pilha em plataformas suportadas

Novos recursos de tipagem:

- PEP 692, usando **`TypedDict`** para anotar **kwargs
- PEP 698, decorador **`@typing.override`**

Descontinuações, remoções ou restrições importantes:

- **PEP 623** [https://peps.python.org/pep-0623/]: Remove `wstr` os objetos Unicode na API C do Python, reduzindo o tamanho de cada objeto **`str`** em pelo menos 8 bytes.
- **PEP 632** [https://peps.python.org/pep-0632/]: Remove o pacote **`distutils`**. Consulte **o guia de migração** [https://peps.python.org/pep-0632/#migration-advice] para obter conselhos sobre a substituição das APIs fornecidas por ele. O pacote de terceiros Setuptools [https://setuptools.pypa.io/en/latest/deprecated/distutils-legacy.html] continua fornecendo **`distutils`**, se você ainda precisar dele no Python 3.12 e posterior.
- gh-95299 [https://github.com/python/cpython/issues/95299]: Não pré-instala `setuptools` em ambientes virtuais criados com **`venv`**. Isso significa que `distutils`, `setuptools`, `pkg_resources` e `easy_install` não estarão mais disponíveis por padrão; para acessá-los, execute `pip install setuptools` no ambiente virtual ativado.
- Os módulos **`asynchat`**, **`asyncore`** e **`imp`** foram removidos, juntamente com vários apelidos de métodos de **`unittest.TestCase`**.

Novas funcionalidades

PEP 695: Sintaxe do parâmetro de tipo

As classes e funções genéricas sob a **PEP 484** [https://peps.python.org/pep-0484/] foram declaradas usando uma sintaxe detalhada que deixou o escopo dos parâmetros de tipo pouco claro e exigiu declarações explícitas de variação.

PEP 695 [https://peps.python.org/pep-0695/] apresenta uma maneira nova, mais compacta e explícita de criar classes genéricas e funções:

```
def max[T](args: Iterable[T]) -> T:
    ...

class list[T]:
    def __getitem__(self, index: int, /) -> T:
        ...

    def append(self, element: T) -> None:
        ...
```

Além disso, a PEP introduz uma nova maneira de declarar apelidos de tipos usando a instrução **`type`**, que cria uma instância de **`TypeAliasType`**:

```
type Point = tuple[float, float]
```

Os apelidos de tipo também podem ser genéricos:

```
type Point[T] = tuple[T, T]
```

A nova sintaxe permite declarar os parâmetros **`TypeVarTuple`** e **`ParamSpec`**, bem como os parâmetros **`TypeVar`** com limites ou restrições:

```
type IntFunc[**P] = Callable[P, int]  # ParamSpec
type LabeledTuple[*Ts] = tuple[str, *Ts]  # TypeVarTuple
type HashableSequence[T: Hashable] = Sequence[T]  # TypeVar com víncu
type IntOrStrSequence[T: (int, str)] = Sequence[T]  # TypeVar com rest
```

O valor dos apelidos de tipo e os limites e restrições das variáveis de tipo criadas por meio dessa sintaxe são avaliados somente sob demanda (consulte avaliação preguiçosa). Isso significa que os apelidos de tipo podem se referir a outros tipos definidos posteriormente no arquivo.

Os parâmetros de tipo declarados por meio de uma lista de parâmetros de tipo são visíveis no escopo da declaração e em quaisquer escopos aninhados, mas não no escopo externo. Por exemplo, eles podem ser usados nas anotações de tipo para os métodos de uma classe genérica ou no corpo da classe. Entretanto, não podem ser usadas no escopo do módulo depois que a classe é definida. Consulte Listas de parâmetros de tipo para obter uma descrição detalhada da semântica de tempo de execução dos parâmetros de tipo.

Para dar suporte a essa semântica de escopo, um novo tipo de escopo é introduzido, o escopo de anotação. Os escopos de anotação se comportam, em sua maior parte, como escopos de função, mas interagem de forma diferente com os escopos de classe. No Python 3.13, anotações também serão avaliadas em escopos de anotação.

Consulte **PEP 695** [https://peps.python.org/pep-0695/] para obter mais detalhes.

(PEP escrita por Eric Traut. Implementação por Jelle Zijlstra, Eric Traut e outros em gh-103764 [https://github.com/python/cpython/issues/103764]).

PEP 701: Formalização sintática de f-strings

PEP 701 [https://peps.python.org/pep-0701/] resolve algumas restrições no uso de f-strings. Componentes de expressão dentro de f-strings agora podem ser qualquer expressão válida do Python, incluindo strings reutilizando a mesma aspa que a f-string contida, expressões multi-linhas, comentários, barras invertidas e sequências de escape unicode. Vamos cobri-los em detalhes:

- Reuso de aspas: no Python 3.11, reusar as mesmas aspas que a f-string que contém levanta um `SyntaxError`, forçando o usuário a usar outras aspas disponíveis (como usar aspas duplas ou triplas se a f-string usa aspas simples). No Python 3.12, agora você pode fazer coisas como esta:

  ```
  >>> songs = ['Take me back to Eden', 'Alkaline', 'Ascensionism']
  >>> f"This is the playlist: {", ".join(songs)}"
  'This is the playlist: Take me back to Eden, Alkaline, Ascensioni
  ```

  Observe que, antes dessa alteração, não havia limite explícito de como f-strings podem ser aninhadas, mas o fato de as aspas de string não poderem ser reusadas dentro do componente de expressão de f-strings tornava impossível aninhar f-strings arbitrariamente. Na verdade, esta é a f-string mais aninhada que poderia ser escrita:

  ```
  >>> f"""{f'''{f'{f"{1+1}"}'}'''}"""
  '2'
  ```

  Como agora f-strings podem conter qualquer expressão Python válida dentro de componentes de expressão, agora é possível aninhar f-strings arbitrariamente:

```
>>> f"{f"{f"{f"{f"{f"{1+1}"}"}"}"}"}"
'2'
```

- Expressões e comentários multilinhas: no Python 3.11, as expressões f-string devem ser definidas em uma única linha, ainda que a expressão dentro de f-string externas possa normalmente abranger várias linhas (como listas literais sendo definidas em várias linhas), tornando-as mais difíceis de ler. No Python 3.12, agora você pode definir f-strings abrangendo várias linhas e adicionar comentários inline:

```
>>> f"This is the playlist: {", ".join([
...     'Take me back to Eden',  # My, my, those eyes like fire
...     'Alkaline',              # Not acid nor alkaline
...     'Ascensionism'           # Take to the broken skies at la
... ])}"
'This is the playlist: Take me back to Eden, Alkaline, Ascensioni
```

- Contrabarra e caracteres unicode: antes do Python 3.12, as expressões f-string não podiam conter nenhum caractere `\`. Isso também afetou as sequências de escape unicode (como `\N{snowman}`), pois elas contêm a parte `\N` que anteriormente não podia fazer parte dos componentes de expressão de f-strings. Agora, você pode definir expressões como esta:

```
>>> print(f"This is the playlist: {"\n".join(songs)}")
This is the playlist: Take me back to Eden
Alkaline
Ascensionism
>>> print(f"This is the playlist: {"\N{BLACK HEART SUIT}".join(so
This is the playlist: Take me back to Eden♥Alkaline♥Ascensionism
```

Veja **PEP 701** [https://peps.python.org/pep-0701/] para mais detalhes.

Como um efeito colateral positivo de como esse recurso foi implementado (analisando f-strings com o **analisador GASE ou PEG** [https://peps.python.org/pep-0617/]), agora as mensagens de erro para f-strings são mais precisas e incluem o local exato do erro. Por exemplo, no Python 3.11, a seguinte f-string levanta um `SyntaxError`:

```
>>> my_string = f"{x z y}" + f"{1 + 1}"
  File "<stdin>", line 1
    (x z y)
```

```
     ^^^
SyntaxError: f-string: invalid syntax. Perhaps you forgot a comma?
```

mas a mensagem de erro não inclui o local exato do erro dentro da linha e também tem a expressão artificialmente cercada por parênteses. No Python 3.12, como as f-strings são analisadas com o analisador GASE, as mensagens de erro podem ser mais precisas e mostrar a linha inteira:

```
>>> minha_string = f"{x z y}" + f"{1 + 1}"
  File "<stdin>", line 1
    minha_string = f"{x z y}" + f"{1 + 1}"
                   ^^^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

(Contribuição de Pablo Galindo, Batuhan Taskaya, Lysandros Nikolaou, Cristián Maureira-Fredes e Marta Gómez em gh-102856 [https://github.com/python/cpython/issues/102856]. PEP escrito por Pablo Galindo, Batuhan Taskaya, Lysandros Nikolaou e Marta Gómez).

PEP 684: Uma GIL por interpretador

A **PEP 684** [https://peps.python.org/pep-0684/] introduz uma GIL por interpretador, para que subinterpretadores possam agora ser criados com uma GIL único por interpretador. Isso permite que programas Python aproveitem ao máximo vários núcleos de CPU. Isso está atualmente disponível apenas por meio da API C, embora uma API Python seja **esperada para 3.13** [https://peps.python.org/pep-0554/].

Use a nova função `Py_NewInterpreterFromConfig()` para criar um interpretador com sua própria GIL:

```
PyInterpreterConfig config = {
    .check_multi_interp_extensions = 1,
    .gil = PyInterpreterConfig_OWN_GIL,
};
PyThreadState *tstate = NULL;
PyStatus status = Py_NewInterpreterFromConfig(&tstate, &config);
if (PyStatus_Exception(status)) {
    return -1;
}
/* O novo interpretador agora está ativo na atual thread. */
```

Para obter mais exemplos de como usar a API C para subinterpretadores com uma GIL por interpretador, consulte `Modules/_xxsubinterpretersmodule.c`.

(Contribuição de Eric Snow em gh-104210 [https://github.com/python/cpython/issues/104210], etc.)

PEP 669: Monitoramento de baixo impacto para CPython

A **PEP 669** [https://peps.python.org/pep-0669/] define uma nova **`API`** para perfis, depuradores e outras ferramentas monitorarem eventos no CPython. Ela abrange uma ampla gama de eventos, incluindo chamadas, retornos, linhas, exceções, saltos e mais. Isso significa que você só paga pelo que usa, fornecendo suporte para depuradores e ferramentas de cobertura com sobrecarga quase zero. Veja **`sys.monitoring`** para mais detalhes.

(Contribuição de Mark Shannon em gh-103082 [https://github.com/python/cpython/issues/103082].)

PEP 688: Tornando o protocolo de buffer acessível no Python

A **PEP 688** [https://peps.python.org/pep-0688/] apresenta uma maneira de usar o protocolo de buffer do código Python. As classes que implementam o método **`__buffer__()`** agora podem ser usadas como tipos de buffer.

O novo **`collections.abc.Buffer`** ABC fornece uma maneira padrão de representar objetos de buffer, por exemplo, em anotações de tipo. A nova enumeração **`inspect.BufferFlags`** representa os sinalizadores que podem ser usados para personalizar a criação do buffer. (Contribuição de Jelle Zijlstra em gh-102500 [https://github.com/python/cpython/issues/102500].)

PEP 709: Fazendo inline de compreensão

As compreensões de dicionário, lista e conjunto agora estão embutidas, em vez de criar um novo objeto função de uso único para cada execução da compreensão. Isso acelera a execução de uma compreensão em até duas vezes. Veja **PEP 709** [https://peps.python.org/pep-0709/] para mais detalhes.

As variáveis de iteração de compreensão permanecem isoladas e não sobrescrevem uma variável de mesmo nome no escopo externo, nem são visíveis após a compreensão. O uso de inlining resulta em algumas mudanças visíveis de comportamento:

- Não há mais um quadro separado para a compreensão em tracebacks (situação da pilha de execução), e o rastreamento/criação de perfil não mostra

mais a compreensão como uma chamada de função.

- O módulo **`symtable`** não produzirá mais tabelas de símbolos filhos para cada compreensão; em vez disso, os locais da compreensão serão incluídos na tabela de símbolos da função pai.
- Chamar **`locals()`** dentro de uma compreensão agora inclui variáveis de fora da compreensão e não inclui mais a variável sintética `.0` para o "argumento" de compreensão.
- Uma compreensão que itera diretamente sobre `locals()` (por exemplo, `[k for k in locals()]`) pode ver "RuntimeError: dictionary changed size during iteration" quando executada sob rastreamento (por exemplo, medição de cobertura de código). Esse é o mesmo comportamento já observado, por exemplo, em `for k in locals():`. Para evitar o erro, primeiro crie uma lista de chaves para iterar: `keys = list(locals()); [k for k in keys]`.

(Contribuição de Carl Meyer and Vladimir Matveev em **PEP 709** [https://peps.python.org/pep-0709/].)

Mensagens de erro melhoradas

- Os módulos da biblioteca padrão agora são potencialmente sugeridos como parte das mensagens de erro exibidas pelo interpretador quando uma exceção **`NameError`** é levantada ao nível superior. (Contribuição de Pablo Galindo em gh-98254 [https://github.com/python/cpython/issues/98254].)

```
>>> sys.version_info
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'sys' is not defined. Did you forget to import 's
```

- Melhora a sugestão de erro para exceções **`NameError`** para instâncias. Agora, se uma exceção **`NameError`** for levantada em um método e a instância tiver um atributo que é exatamente igual ao nome na exceção, a sugestão incluirá `self.<NOME>` em vez da correspondência mais próxima no escopo do método. (Contribuição de Pablo Galindo em gh-99139 [https://github.com/python/cpython/issues/99139].)

```
>>> class A:
...     def __init__(self):
...         self.blech = 1
...
...     def foo(self):
```

```
...         somethin = blech
...
>>> A().foo()
Traceback (most recent call last):
  File "<stdin>", line 1
    somethin = blech
               ^^^^^
NameError: name 'blech' is not defined. Did you mean: 'self.blech
```

- Melhora a mensagem de erro de **`SyntaxError`** quando o usuário digita `import x from y` ao invés de `from y import x`. (Contribuição de Pablo Galindo em gh-98931 [https://github.com/python/cpython/issues/98931].)

```
>>> import a.y.z from b.y.z
Traceback (most recent call last):
  File "<stdin>", line 1
    import a.y.z from b.y.z
    ^^^^^^^^^^^^^^^^^^^^^^^
SyntaxError: Did you mean to use 'from ... import ...' instead?
```

- Exceções **`ImportError`** levantadas a partir de instruções `from <módulo> import <nome>` com falha agora incluem sugestões para o valor de `<nome>` com base nos nomes disponíveis em `<módulo>`. (Contribuição de Pablo Galindo em gh-91058 [https://github.com/python/cpython/issues/91058].)

```
>>> from collections import chainmap
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ImportError: cannot import name 'chainmap' from 'collections'. Di
```

Novos recursos relacionados a dicas de tipo

Esta seção cobre as principais mudanças que afetam as dicas de tipo da **dicas de tipo** [https://peps.python.org/pep-0484/] e o módulo **`typing`**.

PEP 692: Usando `TypedDict` para tipagem mais precisa de `**kwargs`

A tipagem de `**kwargs` em uma assinatura de função conforme introduzido por **PEP 484** [https://peps.python.org/pep-0484/] permitia anotações válidas apenas nos casos em que todos os `**kwargs` eram do mesmo tipo.

A **PEP 692** [https://peps.python.org/pep-0692/] especifica uma maneira mais precisa de definir tipos de `**kwargs`, contando com dicionários tipados:

```
from typing import TypedDict, Unpack

class Filme(TypedDict):
  nome: str
  ano: int

def foo(**kwargs: Unpack[Filme]): ...
```

Veja **PEP 692** [https://peps.python.org/pep-0692/] para mais detalhes.

(Contribuição de Franek Magiera em gh-103629 [https://github.com/python/cpython/issues/103629].)

PEP 698: substitui decorador para tipagem estática

Um novo decorador **`@typing.override`** foi adicionado ao módulo **`typing`**. Ele indica aos verificadores de tipo que o método se destina a substituir um método em uma superclasse. Isso permite que os verificadores de tipo detectem erros onde um método destinado a substituir algo em uma classe base não o faz de fato.

Exemplo:

```
from typing import override

class Base:
  def obtém_cor(self) -> str:
    return "azul"

class FilhaBoa(Base):
  @override  # ok: substitui Base.obtém_cor
  def obtém_cor(self) -> str:
    return "amarelo"

class FilhaRuim(Base):
  @override  # erro de verificador de tipo: não substitui Base.obtém_c
  def obtém_cores(self) -> str:
    return "vermelho"
```

Consulte **PEP 698** [https://peps.python.org/pep-0698/] para mais detalhes.

(Contribuição de Steven Troxler em gh-101561 [https://github.com/python/cpython/issues/101561].)

Outras mudanças na linguagem

- O analisador sintático agora exibe `SyntaxError` ao analisar o código-fonte que contém bytes nulos. (Contribuição de Pablo Galindo em gh-96670 [https://github.com/python/cpython/issues/96670].)
- Um par de caracteres de barra invertida que não seja uma sequência de escape válida agora levanta uma exceção `SyntaxWarning`, em vez de `DeprecationWarning`. Por exemplo, `re.compile("\d+\.\d+")` agora emite um `SyntaxWarning` (`"\d"` é uma sequência de escape inválida, use strings brutas para expressão regular: `re.compile(r"\d+\.\d+")`). Em uma versão futura do Python, `SyntaxError` acabará sendo levantada, em vez de `SyntaxWarning`. (Contribuição de Victor Stinner em gh-98401 [https://github.com/python/cpython/issues/98401].)
- Escapes octais com valor maior do que `0o377` (ex: `"\477"`), descontinuados no Python 3.11, agora produzem um `SyntaxWarning`, em vez de `DeprecationWarning`. Em uma versão futura do Python, eles serão eventualmente um `SyntaxError`. (Contribuição de Victor Stinner em gh-98401 [https://github.com/python/cpython/issues/98401].)
- As variáveis usadas na parte de destino das compreensões que não são armazenadas agora podem ser usadas em expressões de atribuição (`:=`). Por exemplo, em `[(b := 1) for a, b.prop in some_iter]`, a atribuição a `b` agora é permitida. Observe que a atribuição a variáveis armazenadas em na parte de destino das compreensões (como `a`) ainda não é permitida, conforme **PEP 572** [https://peps.python.org/pep-0572/]. (Contribuição de Nikita Sobolev em gh-100581 [https://github.com/python/cpython/issues/100581].)
- As exceções levantadas em um método `__set_name__` de uma classe ou um tipo não são mais envolvidas por um `RuntimeError`. As informações de contexto são adicionadas à exceção como uma nota **PEP 678** [https://peps.python.org/pep-0678/]. (Contribuição de Irit Katriel em gh-77757 [https://github.com/python/cpython/issues/77757].)
- Quando uma construção `try-except*` lida com toda a `ExceptionGroup` e levanta uma outra exceção, essa exceção não é mais envolvida em uma `ExceptionGroup`. Também alterada na versão 3.11.4. (Contribuição de Irit Katriel em gh-103590 [https://github.com/python/cpython/issues/103590].)
- O coletor de lixo agora é executado somente no mecanismo de quebra de avaliação do laço de avaliação de bytecode do Python em vez de alocações de objetos. O coletor de lixo também pode ser executado quando

`PyErr_CheckSignals()` é chamado, de modo que as extensões C que precisam ser executadas por um longo período sem executar nenhum código Python também têm a chance de executar o coletor de lixo periodicamente. (Contribuição de Pablo Galindo em gh-97922 [https://github.com/python/cpython/issues/97922].)

- Todos os chamáveis internos e de extensão que esperam parâmetros booleanos agora aceitam argumentos de qualquer tipo, em vez de apenas `bool` e `int`. (Contribuição de Serhiy Storchaka em gh-60203 [https://github.com/python/cpython/issues/60203].)
- `memoryview` agora oferece suporte ao tipo meio ponto flutuante (o código de formato “e”). (Contribuição de Donghee Na e Antoine Pitrou em gh-90751 [https://github.com/python/cpython/issues/90751].)
- Objetos `slice` agora são hasheáveis, permitindo que sejam usados como chaves de dicionário e itens de conjunto. (Contribuição de Will Bradshaw, Furkan Onder e Raymond Hettinger em gh-101264 [https://github.com/python/cpython/issues/101264].)
- `sum()` agora usa a soma de Neumaier para melhorar a precisão e comutatividade ao somar valores flutuantes ou misturas de ints e floats. (Contribuição de Raymond Hettinger em gh-100425 [https://github.com/python/cpython/issues/100425].)
- `ast.parse()` agora exibe `SyntaxError` em vez de `ValueError` ao analisar o código-fonte que contém bytes nulos. (Contribuição de Pablo Galindo em gh-96670 [https://github.com/python/cpython/issues/96670].)
- Os métodos de extração em `tarfile` e `shutil.unpack_archive()` têm um novo argumento *filter* que permite limitar os recursos do tar que podem ser surpreendentes ou perigosos, como a criação de arquivos fora do diretório de destino. Consulte arquivos de extração de de arquivos tar para obter detalhes. No Python 3.14, o padrão mudará para `'data'`. (Contribuição de Petr Viktorin em **PEP 706** [https://peps.python.org/pep-0706/].)
- Instâncias de `types.MappingProxyType` agora são hasheáveis se o mapeamento subjacente for hasheável (Contribuição de Serhiy Storchaka em gh-87995 [https://github.com/python/cpython/issues/87995].)
- Adiciona suporte ao perfilador perf por meio da nova variável de ambiente `PYTHONPERFSUPPORT` e da opção de linha de comando `-X perf`, bem como das novas funções `sys.activate_stack_trampoline()`, `sys.deactivate_stack_trampoline()` e `sys.is_stack_trampoline_active()`. (Design de Pablo Galindo. Contribuição de Pablo Galindo e Christian Heimes com contribuições de Gregory P. Smith [Google] e Mark Shannon em gh-96123 [https://github.com/python/cpython/issues/96123]).

Novos módulos

- Nenhum.

Módulos melhorados

array

- A classe `array.array` agora suporta subscrição, tornando-a um tipo genérico. (Contribuição de Jelle Zijlstra em gh-98658 [https://github.com/python/cpython/issues/98658].)

asyncio

- O desempenho da gravação em soquetes no `asyncio` foi significativamente aprimorado. O `asyncio` agora evita cópias desnecessárias ao gravar em soquetes e usa o `sendmsg()` se a plataforma o suportar. (Contribuição de Kumar Aditya em gh-91166 [https://github.com/python/cpython/issues/91166].)
- Adiciona as funções `asyncio.eager_task_factory()` e `asyncio.create_eager_task_factory()` para permitir a opção de um laço de eventos na execução de tarefas ansiosas, tornando alguns casos de uso de 2x a 5x mais rápidos. (Contribuição de Jacob Bower e Itamar Oren em gh-102853 [https://github.com/python/cpython/issues/102853], gh-104140 [https://github.com/python/cpython/issues/104140] e gh-104138 [https://github.com/python/cpython/issues/104138])
- No Linux, `asyncio` usa `asyncio.PidfdChildWatcher` por padrão se `os.pidfd_open()` estiver disponível e funcional em vez de `asyncio.ThreadedChildWatcher`. (Contribuição de Kumar Aditya em gh-98024 [https://github.com/python/cpython/issues/98024].)
- O laço de eventos agora usa o melhor observador filho disponível para cada plataforma (`asyncio.PidfdChildWatcher` se suportado e `asyncio.ThreadedChildWatcher` caso contrário), portanto, configurar manualmente um observador filho não é recomendado. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
- Adiciona o parâmetro *loop_factory* a `asyncio.run()` para permitir a especificação de uma fábrica de laço de eventos personalizada. (Contribuição de Kumar Aditya em gh-99388 [https://github.com/python/cpython/issues/99388].)
- Adiciona implementação C de `asyncio.current_task()` para aceleração de 4x-6x. (Contribuição de Itamar Oren e Pranav Thulasiram Bhat em gh-

100344 [https://github.com/python/cpython/issues/100344].)

- `asyncio.iscoroutine()` agora retorna `False` para geradores, pois `asyncio` não tem suporte a corrotinas legadas baseadas em geradores. (Contribuição de Kumar Aditya em gh-102748 [https://github.com/python/cpython/issues/102748].)
- `asyncio.wait()` e `asyncio.as_completed()` agora aceita geradores que produzem tarefas. (Contribuição de Kumar Aditya em gh-78530 [https://github.com/python/cpython/issues/78530].)

calendar

- Adiciona enums `calendar.Month` e `calendar.Day` definindo meses do ano e dias da semana. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)

csv

- Adiciona os sinalizadores `csv.QUOTE_NOTNULL` e `csv.QUOTE_STRINGS` para fornecer um controle mais refinado de `None` e strings vazias por objetos `reader` e `writer`.

dis

- Opcodes de pseudo-instruções (que são usados pelo compilador, mas não aparecem no bytecode executável) agora são expostos no módulo `dis`. `HAVE_ARGUMENT` ainda é relevante para opcodes reais, mas não é útil para pseudo-instruções. Use a nova coleção `dis.hasarg`. (Contribuição de Irit Katriel em gh-94216 [https://github.com/python/cpython/issues/94216].)
- Adiciona a coleção `dis.hasexc` para indicar instruções que definem um manipulador de exceções. (Contribuição de Irit Katriel em gh-94216 [https://github.com/python/cpython/issues/94216].)

fractions

- Objetos do tipo `fractions.Fraction` agora têm suporte a formatação estilo float. (Contribuição de Mark Dickinson em gh-100161 [https://github.com/python/cpython/issues/100161].)

importlib.resources

- `importlib.resources.as_file()` agora tem suporta a diretórios de recursos. (Contribuição de Jason R. Coombs em gh-97930 [https://github.com/python/cpython/issues/97930].)
- Renomeia o primeiro parâmetro de `importlib.resources.files()` para *anchor*. (Contribuição de Jason R. Coombs em gh-100598 [https://github.com/python/cpython/issues/100598].)

inspect

- Adiciona `inspect.markcoroutinefunction()` para marcar funções de sincronização que retornam uma corrotina para uso com `inspect.iscoroutinefunction()`. (Contribuição de Carlton Gibson em gh-99247 [https://github.com/python/cpython/issues/99247].)
- Adiciona `inspect.getasyncgenstate()` e `inspect.getasyncgenlocals()` para determinar o estado atual dos geradores assíncronos. (Contribuição de Thomas Krennwallner em gh-79940 [https://github.com/python/cpython/issues/79940].)
- O desempenho de `inspect.getattr_static()` foi consideravelmente melhorado. A maioria das chamadas para a função deve ser pelo menos 2x mais rápida do que no Python 3.11. (Contribuição de Alex Waygood em gh-103193 [https://github.com/python/cpython/issues/103193].)

itertools

- Adiciona `itertools.batched()` para coletar tuplas de tamanhos iguais, onde o último lote pode ser menor que o resto. (Contribuição de Raymond Hettinger em gh-98363 [https://github.com/python/cpython/issues/98363].)

math

- Adiciona `math.sumprod()` para calcular uma soma de produtos. (Contribuição de Raymond Hettinger em gh-100485 [https://github.com/python/cpython/issues/100485].)
- Estende `math.nextafter()` para incluir um argumento *steps* para subir ou descer várias etapas ao mesmo tempo. (Contribuição de Matthias Goergens, Mark Dickinson e Raymond Hettinger em gh-94906 [https://github.com/python/cpython/issues/94906].)

os

- Adiciona `os.PIDFD_NONBLOCK` para abrir um descritor de arquivo para um processo com `os.pidfd_open()` em modo sem bloqueio. (Contribuição de Kumar Aditya em gh-93312 [https://github.com/python/cpython/issues/93312].)
- `os.DirEntry` agora inclui um método `os.DirEntry.is_junction()` para verificar se a entrada é uma junção. (Contribuição de Charles Machalow em gh-99547 [https://github.com/python/cpython/issues/99547].)
- Adiciona funções `os.listdrives()`, `os.listvolumes()` e `os.listmounts()` no Windows para enumerar unidades, volumes e pontos de montagem. (Contribuição de Steve Dower em gh-102519 [https://github.com/python/cpython/issues/102519].)
- `os.stat()` e `os.lstat()` agora são mais precisos no Windows. O campo `st_birthtime` agora será preenchido com a hora de criação do arquivo, e `st_ctime` foi descontinuado, mas ainda contém a hora de criação (mas no futuro vai retornar a última alteração de metadados, para consistência com outras plataformas). `st_dev` pode ter até 64 bits e `st_ino` até 128 bits dependendo do seu sistema de arquivos, e `st_rdev` é sempre definido como zero em vez de valores incorretos. Ambas as funções podem ser significativamente mais rápidas nas lançamentos mais recentes do Windows. (Contribuição de Steve Dower em gh-99726 [https://github.com/python/cpython/issues/99726].)

os.path

- Adiciona `os.path.isjunction()` para verificar se um determinado caminho é uma junção. (Contribuição de Charles Machalow em gh-99547 [https://github.com/python/cpython/issues/99547].)
- Adiciona `os.path.splitroot()` para dividir um caminho em uma tríade `(drive, root, tail)`. (Contribuição de Barney Gale em gh-101000 [https://github.com/python/cpython/issues/101000].)

pathlib

- Adiciona suporte para fazer subclasse de `pathlib.PurePath` e `pathlib.Path`, além de suas variantes específicas do Posix e do Windows. As subclasses podem substituir o método `pathlib.PurePath.with_segments()` para passar informações entre instâncias de caminho.
- Adiciona `pathlib.Path.walk()` para percorrer as árvores de diretórios e gerar todos os nomes de arquivos ou diretórios dentro delas, semelhante a `os.walk()`. (Contribuição de Stanislav Zmiev em gh-90385 [https://github.com/python/cpython/issues/90385].)

- Adiciona o parâmetro opcional *walk_up* a `pathlib.PurePath.relative_to()` para permitir a inserção de entradas `..` no resultado; este comportamento é mais consistente com `os.path.relpath()`. (Contribuição de Domenico Ragusa em gh-84538 [https://github.com/python/cpython/issues/84538].)
- Adiciona `pathlib.Path.is_junction()` como proxy para `os.path.isjunction()`. (Contribuição de Charles Machalow em gh-99547 [https://github.com/python/cpython/issues/99547].)
- Adiciona o parâmetro opcional *case_sensitive* a `pathlib.Path.glob()`, `pathlib.Path.rglob()` e `pathlib.PurePath.match()` para fazer correspondência com distinção entre maiúsculas e minúsculas do caminho, permitindo uma precisão mais precisa controle sobre o processo de correspondência.

platform

- Adiciona suporte para detectar versões do Windows 11 e do Windows Server posteriores a 2012. Anteriormente, pesquisas em plataformas Windows Server mais recentes que o Windows Server 2012 e no Windows 11 retornariam `Windows-10`. (Contribuição de Steve Dower em gh-89545 [https://github.com/python/cpython/issues/89545].)

pdb

- Adiciona variáveis de conveniência para armazenar valores temporariamente para a sessão de depuração e fornece acesso rápido a valores como o quadro atual ou o valor de retorno. (Contribuição de Tian Gao em gh-103693 [https://github.com/python/cpython/issues/103693].)

random

- Adiciona `random.binomialvariate()`. (Contribuição de Raymond Hettinger em gh-81620 [https://github.com/python/cpython/issues/81620].)
- Adiciona um padrão de `lambd=1.0` a `random.expovariate()`. (Contribuição de Raymond Hettinger em gh-100234 [https://github.com/python/cpython/issues/100234].)

shutil

- `shutil.make_archive()` agora passa o argumento *root_dir* para arquivadores personalizados que o suportam. Neste caso, ele não altera mais temporariamente o diretório de trabalho atual do processo para *root_dir* para realizar o arquivamento. (Contribuição de Serhiy Storchaka em gh-74696 [https://github.com/python/cpython/issues/74696].)

- `shutil.rmtree()` agora aceita um novo argumento *onexc* que é um tratador de erros como *onerror* mas que espera uma instância de exceção em vez de um trio *(typ, val, tb)*. *onerror* foi descontinuado. (Contribuição de Irit Katriel em gh-102828 [https://github.com/python/cpython/issues/102828].)

- **`shutil. which()`** agora consulta a variável de ambiente *PATHEXT* para encontrar correspondências dentro de *PATH* no Windows mesmo quando o *cmd* fornecido inclui um componente de diretório. (Contribuição de Charles Machalow em gh-103179 [https://github.com/python/cpython/issues/103179].)

  `shutil. which()` chamará `NeedCurrentDirectoryForExePathW` ao consultar executáveis no Windows para determinar se o diretório de trabalho atual deve ser anexado ao caminho de pesquisa. (Contribuição de Charles Machalow em gh-103179 [https://github.com/python/cpython/issues/103179].)

  `shutil.which()` vai retornar um caminho que corresponde ao *cmd* com um componente de `PATHEXT` antes de uma correspondência direta em outro lugar no caminho de pesquisa no Windows. (Contribuição de Charles Machalow em gh-103179 [https://github.com/python/cpython/issues/103179].)

sqlite3

- Adiciona uma interface de linha de comando. (Contribuição de Erlend E. Aasland em gh-77617 [https://github.com/python/cpython/issues/77617].)
- Adiciona o atributo `sqlite3.Connection.autocommit` a `sqlite3.Connection` e o parâmetro *autocommit* a `sqlite3.connect()` para controlar a **PEP 249** [https://peps.python.org/pep-0249/] compatível com tratamento de transação. (Contribuição de Erlend E. Aasland em gh-83638 [https://github.com/python/cpython/issues/83638].)
- Adiciona o parâmetro somente-nomeado *entrypoint* a `sqlite3.Connection.load_extension()`, para substituir o ponto de entrada da extensão SQLite. (Contribuição de Erlend E. Aasland em gh-103015 [https://github.com/python/cpython/issues/103015].)
- Adiciona `sqlite3.Connection.getconfig()` e `sqlite3.Connection.setconfig()` a `sqlite3.Connection` para fazer

alterações na configuração de uma conexão de banco de dados. (Contribuição de Erlend E. Aasland em gh-103489 [https://github.com/python/cpython/issues/103489].)

statistics

- Estende `statistics.correlation()` para incluir como um método `ranked` para calcular a correlação de Spearman de dados classificados. (Contribuição de Raymond Hettinger em gh-95861 [https://github.com/python/cpython/issues/95861].)

sys

- Adiciona o espaço de nomes `sys.monitoring` para expor a nova API de monitoramento PEP 669. (Contribuição de Mark Shannon em gh-103082 [https://github.com/python/cpython/issues/103082].)
- Adiciona `sys.activate_stack_trampoline()` e `sys.deactivate_stack_trampoline()` para ativar e desativar trampolins do perfilador de pilha e `sys.is_stack_trampoline_active()` para consultar se os trampolins do perfilador de pilha estão ativos. (Contribuição de Pablo Galindo e Christian Heimes com contribuições de Gregory P. Smith [Google] e Mark Shannon em gh-96123 [https://github.com/python/cpython/issues/96123].)
- Adiciona `sys.last_exc` que contém a última exceção não tratada que foi levantada (para casos de uso de depuração post-mortem). Descontinuar os três campos que possuem as mesmas informações em seu formato legado: `sys.last_type`, `sys.last_value` e `sys.last_traceback`. (Contribuição de Irit Katriel em gh-102778 [https://github.com/python/cpython/issues/102778].)
- `sys._current_exceptions()` agora retorna um mapeamento de thread-id para uma instância de exceção, ao invés de uma tupla `(typ, exc, tb)`. (Contribuição de Irit Katriel em gh-103176 [https://github.com/python/cpython/issues/103176].)
- `sys.setrecursionlimit()` e `sys.getrecursionlimit()`. O limite de recursão agora se aplica apenas ao código Python. As funções integradas não usam o limite de recursão, mas são protegidas por um mecanismo diferente que evita que a recursão cause uma falha na máquina virtual.

tempfile

- A função `tempfile.NamedTemporaryFile` tem um novo parâmetro opcional *delete_on_close* (Contribuição de Evgeny Zorin em gh-58451 [https://github.com/python/cpython/issues/58451].)

- `tempfile.mkdtemp()` agora sempre retorna um caminho absoluto, mesmo se o argumento fornecido para o parâmetro *dir* for um caminho relativo.

threading

- Adiciona `threading.settrace_all_threads()` e `threading.setprofile_all_threads()` que permitem definir funções de rastreamento e criação de perfil em todos os threads em execução além do que está chamando. (Contribuição de Pablo Galindo em gh-93503 [https://github.com/python/cpython/issues/93503].)

tkinter

- `tkinter.Canvas.coords()` agora nivela seus argumentos. Agora ele aceita não apenas coordenadas como argumentos separados (`x1, y1, x2, y2, ...`) e uma sequência de coordenadas (`[x1, y1, x2, y2, ...]`) , mas também coordenadas agrupadas em pares (`(x1, y1), (x2, y2), ...` e `[(x1, y1), (x2, y2), ...]`) , como métodos `create_*()`. (Contribuição de Serhiy Storchaka em gh-94473 [https://github.com/python/cpython/issues/94473].)

tokenize

- O módulo `tokenize` inclui as mudanças introduzidas em **PEP 701** [https://peps.python.org/pep-0701/]. (Contribuição de Marta Gómez Macías e Pablo Galindo em gh-102856 [https://github.com/python/cpython/issues/102856].) Veja Portando para Python 3.12 para mais informações sobre as mudanças no módulo `tokenize`.

types

- Adiciona `types.get_original_bases()` para permitir uma introspecção adicional de Tipos genéricos definidos pelo usuário quando subclassificado. (Contribuição de James Hilton-Balfe e Alex Waygood em gh-101827 [https://github.com/python/cpython/issues/101827].)

typing

- As verificações de `isinstance()` com `protocolos verificáveis em tempo de execução` agora usa `inspect.getattr_static()` em vez de `hasattr()` para verificar se existem atributos. Isso significa que descritores e métodos

`__getattr__()` não são mais avaliados inesperadamente durante verificações `isinstance()` em protocolos verificáveis em tempo de execução. No entanto, também pode significar que alguns objetos que costumavam ser considerados instâncias de um protocolo verificável em tempo de execução podem não ser mais considerados instâncias desse protocolo no Python 3.12+ e vice-versa. É improvável que a maioria dos usuários seja afetada por esta mudança. (Contribuição de Alex Waygood em gh-102433 [https://github.com/python/cpython/issues/102433].)

- Os membros de um protocolo verificável em tempo de execução agora são considerados "congelados" em tempo de execução assim que a classe é criada. Fazer alterações dinâmicas em atributos em tempo de execução em um protocolo verificável em tempo de execução ainda vai funcionar, mas não terá impacto nas verificações de `isinstance()` comparando objetos com o protocolo. Por exemplo:

```
>>> from typing import Protocol, runtime_checkable
>>> @runtime_checkable
... class TemX(Protocol):
...     x = 1
...
>>> class Foo: ...
...
>>> f = Foo()
>>> isinstance(f, TemX)
False
>>> f.x = 1
>>> isinstance(f, TemX)
True
>>> TemX.y = 2
>>> isinstance(f, TemX)  # inalterado, ainda que agora TemX també
True
```

  Esta mudança foi feita para acelerar as verificações de `isinstance()` em protocolos verificáveis em tempo de execução.

- O perfil de desempenho de verificações de `isinstance()` com `protocolos verificáveis em tempo de execução` mudou significativamente. A maioria das verificações `isinstance()` a esses protocolos com apenas alguns membros devem ser pelo menos 2x mais rápidas que na versão 3.11, e algumas podem ser 20x mais rápidas ou mais. No entanto, as verificações de `isinstance()` em protocolos com muito mais membros podem ser mais

lentas do que no Python 3.11. (Contribuição de Alex Waygood em gh-74690 [https://github.com/python/cpython/issues/74690] e gh-103193 [https://github.com/python/cpython/issues/103193].)

- Todas as classes `typing.TypedDict` e `typing.NamedTuple` agora possuem o atributo `__orig_bases__`. (Contribuição de Adrian Garcia Badaracco em gh-103699 [https://github.com/python/cpython/issues/103699].)

- Adiciona o parâmetro `frozen_default` a `@typing.dataclass_transform`. (Contribuição de Erik De Bonte em gh-99957 [https://github.com/python/cpython/issues/99957].)

unicodedata

- O banco de dados Unicode foi atualizado para a versão 15.0.0. (Contribuição de Benjamin Peterson em gh-96734 [https://github.com/python/cpython/issues/96734]).

unittest

Adiciona uma opção de linha de comando `--durations`, mostrando os N casos de teste mais lentos:

```
python3 -m unittest --durations=3 lib.tests.test_threading
.....
Slowest test durations
----------------------------------------------------------------------
1.210s     test_timeout (Lib.test.test_threading.BarrierTests)
1.003s     test_default_timeout (Lib.test.test_threading.BarrierTests)
0.518s     test_timeout (Lib.test.test_threading.EventTests)

(0.000 durations hidden.  Use -v to show these durations.)
----------------------------------------------------------------------
Ran 158 tests in 9.869s

OK (skipped=3)
```

(Contribuição de Giampaolo Rodola em gh-48330 [https://github.com/python/cpython/issues/48330])

uuid

- Adiciona uma interface de linha de comando. (Contribuição de Adam Chhina em gh-88597 [https://github.com/python/cpython/issues/88597].)

Otimizações

- Remove os membros `wstr` e `wstr_length` dos objetos Unicode. Reduz o tamanho do objeto em 8 ou 16 bytes na plataforma de 64 bits. (**PEP 623** [https://peps.python.org/pep-0623/]) (Contribuição de Inada Naoki em gh-92536 [https://github.com/python/cpython/issues/92536].)
- Adiciona suporte experimental para usar o otimizador binário BOLT no processo de construção, o que melhora o desempenho de 1 a 5%. (Contribuição de Kevin Modzelewski em gh-90536 [https://github.com/python/cpython/issues/90536] e ajuste de Donghee Na em gh-101525 [https://github.com/python/cpython/issues/101525])
- Acelera a substituição de expressões regulares (funções **`re.sub()`** e **`re.subn()`** e métodos **`re.Pattern`** correspondentes) para strings de substituição contendo referências de grupo em 2 ou 3 vezes . (Contribuição de Serhiy Storchaka em gh-91524 [https://github.com/python/cpython/issues/91524].)
- Acelera a criação de **`asyncio.Task`** adiando a dispendiosa formatação de strings. (Contribuição de Itamar Oren em gh-103793 [https://github.com/python/cpython/issues/103793].)
- As funções **`tokenize.tokenize()`** e **`tokenize.generate_tokens()`** são até 64% mais rápidas como efeito colateral das mudanças necessárias para cobrir **PEP 701** [https://peps.python.org/pep-0701/] no módulo **`tokenize`** . (Contribuição de Marta Gómez Macías e Pablo Galindo em gh-102856 [https://github.com/python/cpython/issues/102856].)
- Acelera chamadas de métodos **`super()`** e carregamentos de atributos através da nova instrução **`LOAD_SUPER_ATTR`**. (Contribuição de Carl Meyer e Vladimir Matveev em gh-103497 [https://github.com/python/cpython/issues/103497].)

Alterações de bytecode do CPython

- Remove a instrução **`LOAD_METHOD`**. Ela foi mesclada em **`LOAD_ATTR`**. **`LOAD_ATTR`** agora se comportará como a antiga instrução **`LOAD_METHOD`** se o bit inferior de seu oparg estiver definido. (Contribuição de Ken Jin em gh-93429 [https://github.com/python/cpython/issues/93429].)
- Remova as instruções **`JUMP_IF_FALSE_OR_POP`** e **`JUMP_IF_TRUE_OR_POP`**. (Contribuição de Irit Katriel em gh-102859 [https://github.com/python/cpython/issues/102859].)

- Remove a instrução **PRECALL**. (Contribuição de Mark Shannon em gh-92925 [https://github.com/python/cpython/issues/92925].)
- Adiciona as instruções **BINARY_SLICE** e **STORE_SLICE**. (Contribuição de Mark Shannon em gh-94163 [https://github.com/python/cpython/issues/94163].)
- Adiciona as instruções **CALL_INTRINSIC_1**. (Contribuição de Mark Shannon em gh-99005 [https://github.com/python/cpython/issues/99005].)
- Adiciona a instrução **CALL_INTRINSIC_2**. (Contribuição de Irit Katriel em gh-101799 [https://github.com/python/cpython/issues/101799].)
- Adiciona a instrução **CLEANUP_THROW**. (Contribuição de Brandt Bucher em gh-90997 [https://github.com/python/cpython/issues/90997].)
- Adiciona a instrução **END_SEND**. (Contribuição de Mark Shannon em gh-103082 [https://github.com/python/cpython/issues/103082].)
- Adiciona a instrução **LOAD_FAST_AND_CLEAR** como parte da implementação de **PEP 709** [https://peps.python.org/pep-0709/]. (Contribuição de Carl Meyer em gh-101441 [https://github.com/python/cpython/issues/101441].)
- Adiciona a instrução **LOAD_FAST_CHECK**. (Contribuição de Dennis Sweeney em gh-93143 [https://github.com/python/cpython/issues/93143].)
- Adiciona os opcodes **LOAD_FROM_DICT_OR_DEREF**, **LOAD_FROM_DICT_OR_GLOBALS** e **LOAD_LOCALS** como parte da implementação da **PEP 695** [https://peps.python.org/pep-0695/]. Remove o opcode **LOAD_CLASSDEREF**, que pode ser substituído por **LOAD_LOCALS** mais **LOAD_FROM_DICT_OR_DEREF**. (Contribuição de Jelle Zijlstra em gh-103764 [https://github.com/python/cpython/issues/103764].)
- Adiciona a instrução **LOAD_SUPER_ATTR**. (Contribuição de Carl Meyer e Vladimir Matveev em gh-103497 [https://github.com/python/cpython/issues/103497].)
- Adiciona a instrução `RETURN_CONST`. (Contribuição de Wenyang Wang em gh-101632 [https://github.com/python/cpython/issues/101632].)

Ferramentas e daemons

- Remove o diretório `Tools/demo/` que continha scripts antigos de demonstração. Uma cópia pode ser encontrada no projeto old-demos [https://github.com/gvanrossum/old-demos]. (Contribuição de Victor Stinner em gh-97681 [https://github.com/python/cpython/issues/97681].)
- Remove scripts de exemplo desatualizados do diretório `Tools/scripts/`. Uma cópia pode ser encontrada no projeto old-demos [https://github.com/gvanrossum/old-demos]. (Contribuição de Victor Stinner em gh-97669 [https://github.com/python/cpython/issues/97669].)

Descontinuados

- **argparse**: Os parâmetros *type*, *choices* e *metavar* de **argparse.BooleanOptionalAction** foram descontinuados e serão removidos na versão 3.14. (Contribuição de Nikita Sobolev em gh-92248 [https://github.com/python/cpython/issues/92248].)

- **ast**: Os seguintes recursos **ast** foram descontinuados na documentação desde Python 3.8, agora fazem com que um **DeprecationWarning** seja emitido em tempo de execução quando eles são acessados ou usados, e serão removidos no Python 3.14:

  - **ast.Num**
  - **ast.Str**
  - **ast.Bytes**
  - **ast.NameConstant**
  - **ast.Ellipsis**

  Usa **ast.Constant** em vez disso. (Contribuição de Serhiy Storchaka em gh-90953 [https://github.com/python/cpython/issues/90953].)

- **asyncio**:

  - As classes filhas dos observadores **asyncio.MultiLoopChildWatcher**, **asyncio.FastChildWatcher**, **asyncio.AbstractChildWatcher** e **asyncio.SafeChildWatcher** foram descontinuadas e serão removidas no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
  - **asyncio.set_child_watcher()**, **asyncio.get_child_watcher()**, **asyncio.AbstractEventLoopPolicy.set_child_watcher()** e **asyncio.AbstractEventLoopPolicy.get_child_watcher()** foram descontinuados e serão removidos no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
  - O método **get_event_loop()** da política de laço de eventos padrão agora emite um **DeprecationWarning** se não houver nenhum laço de eventos atual definido e decidir criar um. (Contribuição de Serhiy Storchaka e Guido van Rossum em gh-100160 [https://github.com/python/cpython/issues/100160].)

- **calendar**: As constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por **calendar.JANUARY** e **calendar.FEBRUARY**.

(Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)

- **`collections.abc`**: Descontinuou **`collections.abc.ByteString`**.

  Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

  `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

  Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`datetime`**: **`utcnow()`** e **`utcfromtimestamp()`** da classe **`datetime.datetime`** foram descontinuados e serão removidos em uma versão futura. Em vez disso, use objetos conscientes do fuso horário para representar datas e horas em UTC: respectivamente, chame **`now()`** e **`fromtimestamp()`** com o parâmetro *tz* definido como **`datetime.UTC`**. (Contribuição de Paul Ganssle em gh-103857 [https://github.com/python/cpython/issues/103857].)

- **`email`**: Descontinua o parâmetro *isdst* em **`email.utils.localtime()`**. (Contribuição de Alan Williams em gh-72346 [https://github.com/python/cpython/issues/72346].)

- **`importlib.abc`**: Foram descontinuadas as seguintes classes, programadas para remoção no Python 3.14:

  - **`importlib.abc.ResourceReader`**
  - **`importlib.abc.Traversable`**
  - **`importlib.abc.TraversableResources`**

  Em vez disso, use classes **`importlib.resources.abc`**:

  - **`importlib.resources.abc.Traversable`**
  - **`importlib.resources.abc.TraversableResources`**

  (Contribuição de Jason R. Coombs e Hugo van Kemenade em gh-93963 [https://github.com/python/cpython/issues/93963].)

- **`itertools`**: Descontinua o suporte para operações de cópia, deepcopy e pickle, que é não documentado, ineficiente, historicamente cheio de bugs e inconsistente. Isso será removido na versão 3.14 para uma redução significativa no volume de código e na carga de manutenção. (Contribuição de Raymond Hettinger em gh-101588 [https://github.com/python/cpython/issues/101588].)

- **`multiprocessing`**: No Python 3.14, o método de início padrão de **`multiprocessing`** será alterado para um mais seguro no Linux, BSDs e outras plataformas POSIX não-macOS onde `'fork'` é atualmente o padrão (gh-84559 [https://github.com/python/cpython/issues/84559]). Adicionar um aviso em tempo de execução sobre isso foi considerado muito perturbador, pois não se espera que a maior parte do código se importe. Use as APIs **`get_context()`** ou **`set_start_method()`** para especificar explicitamente quando seu código *requer* `'fork'`. Veja contextos e métodos de início.

- **`pkgutil`**: **`pkgutil.find_loader()`** e **`pkgutil.get_loader()`** foram descontinuados e serão removidos no Python 3.14; use **`importlib.util.find_spec()`** em vez disso. (Contribuição de Nikita Sobolev em gh-97850 [https://github.com/python/cpython/issues/97850].)

- **`pty`**: O módulo tem duas funções `master_open()` e `slave_open()` não documentadas que foram descontinuados desde Python 2, mas só ganharam um **`DeprecationWarning`** adequado em 3.12. Remova-os em 3.14. (Contribuição de Soumendra Ganguly e Gregory P. Smith em gh-85984 [https://github.com/python/cpython/issues/85984].)

- **`os`**:

  - Os campos `st_ctime` retornados por **`os.stat()`** e **`os.lstat()`** no Windows foram descontinuados. Em um lançamento futuro, eles conterão o horário da última alteração de metadados, consistente com outras plataformas. Por enquanto, eles ainda contêm o horário de criação, que também está disponível no novo campo `st_birthtime`.

(Contribuição de Steve Dower em gh-99726 [https://github.com/python/cpython/issues/99726].)

  - Nas plataformas POSIX, `os.fork()` agora pode levantar uma exceção `DeprecationWarning` quando puder detectar a chamada de um processo multithread. Sempre houve uma incompatibilidade fundamental com a plataforma POSIX ao fazer isso. Mesmo que esse código *pareça* funcionar. Adicionamos o aviso para aumentar a conscientização, pois os problemas encontrados pelo código ao fazer isso estão se tornando mais frequentes. Veja a documentação `os.fork()` para mais detalhes junto com esta discussão sobre fork ser incompatível com threads [https://discuss.python.org/t/concerns-regarding-deprecation-of-fork-with-alive-threads/33555] para *por que* estamos agora trazendo à tona esse antigo problema de compatibilidade de plataforma para os desenvolvedores.

  Quando este aviso aparece devido ao uso de `multiprocessing` ou `concurrent.futures`, a correção é usar um método de início de `multiprocessing` diferente, como `"spawn"` ou `"forkserver"`.

- `shutil`: O argumento *onerror* de `shutil.rmtree()` foi descontinuado; use *onexc* em vez disso. (Contribuição de Irit Katriel em gh-102828 [https://github.com/python/cpython/issues/102828].)

- `sqlite3`:

  - adaptadores e conversores padrão agora foram descontinuados. Em vez disso, use Adapter and converter recipes e adapte-os às suas necessidades. (Contribuição de Erlend E. Aasland em gh-90016 [https://github.com/python/cpython/issues/90016].)
  - Em `execute()`, `DeprecationWarning` agora é emitida quando espaços reservados nomeados são usados junto com parâmetros fornecidos como sequência em vez de como `dict`. A partir do Python 3.14, usar espaços reservados nomeados com parâmetros fornecidos como uma sequência vai levantar uma `ProgrammingError`. (Contribuição de Erlend E. Aasland em gh-101698 [https://github.com/python/cpython/issues/101698].)

- `sys`: Os campos `sys.last_type`, `sys.last_value` e `sys.last_traceback` foram descontinuados. Use `sys.last_exc` em vez disso. (Contribuição de Irit Katriel em gh-102778 [https://github.com/python/cpython/issues/102778].)

- `tarfile`: Extrair arquivos tar sem especificar *filter* foi descontinuado até Python 3.14, quando o filtro `'data'` se tornará o padrão. Veja Filtros de

extração para detalhes.

- **typing**:
  - **typing.Hashable** e **typing.Sized**, apelidos para **collections.abc.Hashable** e **collections.abc.Sized** respectivamente, foram descontinuados. (gh-94309 [https://github.com/python/cpython/issues/94309].)
  - **typing.ByteString**, descontinuado desde Python 3.9, agora faz com que um **DeprecationWarning** seja emitida quando é usado. (Contribuição de Alex Waygood em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **xml.etree.ElementTree**: O módulo agora emite **DeprecationWarning** ao testar o valor verdade de um **xml.etree.ElementTree.Element**. Antes, a implementação Python emitia **FutureWarning**, e a implementação C não emitia nada. (Contribuição de Jacob Walls em gh-83122 [https://github.com/python/cpython/issues/83122].)

- As assinaturas de 3 argumentos (tipo, valor, traceback) de **coroutine throw()**, **generator throw()** e **async generator throw()** foram descontinuados e podem ser removidos em uma versão futura do Python. Use as versões de argumento único dessas funções. (Contribuição de Ofey Chan em gh-89874 [https://github.com/python/cpython/issues/89874].)

- **DeprecationWarning** agora é levantada quando **__package__** em um módulo difere de **__spec__.parent** (anteriormente era **ImportWarning**). (Contribuição de Brett Cannon em gh-65961 [https://github.com/python/cpython/issues/65961].)

- Definir **__package__** ou `__cached__` em um módulo foi descontinuado e deixará de ser definido ou levado em consideração pelo sistema de importação no Python 3.14. (Contribuição de Brett Cannon em gh-65961 [https://github.com/python/cpython/issues/65961].)

- O operador de inversão bit a bit (~) em booleano foi descontinuado. Isso levantará um erro no Python 3.16. Use `not` para negação lógica de booleanos. No raro caso em que você realmente precisa da inversão bit a bit do `int` subjacente, converta para int explicitamente: `~int(x)`. (Contribuição de Tim Hoffmann em gh-103487 [https://github.com/python/cpython/issues/103487].)

- Acessar **`codeobject.co_lnotab`** em objetos de código foi descontinuado no Python 3.10 via **PEP 626** [https://peps.python.org/pep-0626/], mas só obteve um **`DeprecationWarning`** adequado em 3.12. Pode ser removido em 3.15. (Contribuição de Nikita Sobolev em gh-101866 [https://github.com/python/cpython/issues/101866].)

Remoção pendente no Python 3.13

Módulos (veja **PEP 594** [https://peps.python.org/pep-0594/]):

- **`aifc`**
- **`audioop`**
- **`cgi`**
- **`cgitb`**
- **`chunk`**
- **`crypt`**
- **`imghdr`**
- **`mailcap`**
- **`msilib`**
- **`nis`**
- **`nntplib`**
- **`ossaudiodev`**
- **`pipes`**
- **`sndhdr`**
- **`spwd`**
- **`sunau`**
- **`telnetlib`**
- **`uu`**
- **`xdrlib`**

Outros módulos:

- **`lib2to3`** e o programa **2to3** (gh-84540 [https://github.com/python/cpython/issues/84540])

APIs:

- **`configparser.LegacyInterpolation`** (gh-90765 [https://github.com/python/cpython/issues/90765])
- `locale.resetlocale()` (gh-90817 [https://github.com/python/cpython/issues/90817])
- **`turtle.RawTurtle.settiltangle()`** (gh-50096 [https://github.com/python/cpython/issues/50096])

- **`unittest.findTestCases()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.getTestCaseNames()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.makeSuite()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.TestProgram.usageExit()`** (gh-67048 [https://github.com/python/cpython/issues/67048])
- **`webbrowser.MacOSX`** (gh-86421 [https://github.com/python/cpython/issues/86421])
- Encadeamento do descritor de **`classmethod`** (gh-89519 [https://github.com/python/cpython/issues/89519])

Remoção pendente no Python 3.14

- **`argparse`**: Os parâmetros *type*, *choices* e *metavar* de **`argparse.BooleanOptionalAction`** foram descontinuados e serão removidos na versão 3.14. (Contribuição de Nikita Sobolev em gh-92248 [https://github.com/python/cpython/issues/92248].)

- **`ast`**: Os seguintes recursos foram descontinuados na documentação desde Python 3.8, agora fazem com que um **`DeprecationWarning`** seja emitido em tempo de execução quando eles são acessados ou usados, e serão removidos no Python 3.14:

  - **`ast.Num`**
  - **`ast.Str`**
  - **`ast.Bytes`**
  - **`ast.NameConstant`**
  - **`ast.Ellipsis`**

  Usa **`ast.Constant`** em vez disso. (Contribuição de Serhiy Storchaka em gh-90953 [https://github.com/python/cpython/issues/90953].)

- **`asyncio`**:

  - As classes filhas dos observadores **`asyncio.MultiLoopChildWatcher`**, **`asyncio.FastChildWatcher`**, **`asyncio.AbstractChildWatcher`** e **`asyncio.SafeChildWatcher`** foram descontinuadas e serão removidas no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
  - **`asyncio.set_child_watcher()`**, **`asyncio.get_child_watcher()`**, **`asyncio.AbstractEventLoopPolicy.set_child_watcher()`** e

`asyncio.AbstractEventLoopPolicy.get_child_watcher()` foram descontinuados e serão removidos no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
  - O método **`get_event_loop()`** da política de laço de eventos padrão agora emite um **`DeprecationWarning`** se não houver nenhum laço de eventos atual definido e decidir criar um. (Contribuição de Serhiy Storchaka e Guido van Rossum em gh-100160 [https://github.com/python/cpython/issues/100160].)

- **`email`**: Descontinua o parâmetro *isdst* em **`email.utils.localtime()`**. (Contribuição de Alan Williams em gh-72346 [https://github.com/python/cpython/issues/72346].)

- **`importlib.abc`** descontinuou as classes:

  - **`importlib.abc.ResourceReader`**
  - **`importlib.abc.Traversable`**
  - **`importlib.abc.TraversableResources`**

  Em vez disso, use classes **`importlib.resources.abc`**:

  - **`importlib.resources.abc.Traversable`**
  - **`importlib.resources.abc.TraversableResources`**

  (Contribuição de Jason R. Coombs e Hugo van Kemenade em gh-93963 [https://github.com/python/cpython/issues/93963].)

- **`itertools`** tinha suporte não documentado, ineficiente, historicamente cheio de bugs e inconsistente para operações de cópia, cópia profunda e serialização com pickle. Isso será removido na versão 3.14 para uma redução significativa no volume de código e na carga de manutenção. (Contribuição de Raymond Hettinger em gh-101588 [https://github.com/python/cpython/issues/101588].)

- **`multiprocessing`**: O método de inicialização padrão será alterado para um mais seguro no Linux, BSDs e outras plataformas POSIX não-macOS onde `'fork'` é atualmente o padrão (gh-84559 [https://github.com/python/cpython/issues/84559]). Adicionar um aviso de tempo de execução sobre isso foi considerado muito perturbador, pois não se espera que a maior parte do código se importe. Use as APIs **`get_context()`** ou **`set_start_method()`** para especificar explicitamente quando seu código *requer* `'fork'`. Veja Contextos e métodos de inicialização.

- **`pathlib`**: **`is_relative_to()`** e **`relative_to()`**: passar argumentos adicionais foi descontinuado.

- **`pkgutil`**: **`pkgutil.find_loader()`** e **`pkgutil.get_loader()`** agora levantam **`DeprecationWarning`**; use **`importlib.util.find_spec()`**. (Contribuição de Nikita Sobolev em gh-97850 [https://github.com/python/cpython/issues/97850].)

- **`pty`**:

  - `master_open()`: use **`pty.openpty()`**.
  - `slave_open()`: use **`pty.openpty()`**.

- **`sqlite3`**:

  - **`version`** e **`version_info`**.
  - **`execute()`** e **`executemany()`** se espaços reservados nomeados forem usados e *parameters* for uma sequência em vez de um **`dict`** .

- **`urllib`**: **`urllib.parse.Quoter`** está obsoleto: não foi planejado para ser uma API pública. (Contribuição de Gregory P. Smith em gh-88168 [https://github.com/python/cpython/issues/88168].)

Remoção pendente no Python 3.15

- O sistema de importação:
  - A definição de `__cached__` em um módulo enquanto falha na definição de **`__spec__.cached`** está descontinuado. No Python 3.15, `__cached__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
  - A definição de **`__package__`** em um módulo enquanto falha na definição de **`__spec__.parent`** está descontinuado. No Python 3.15, **`__package__`** deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
- **`ctypes`**:
  - A função não documentada **`ctypes.SetPointerType()`** foi descontinuada desde o Python 3.13.
- **`http.server`**:
  - A classe obsoleta e raramente usada **`CGIHTTPRequestHandler`** foi descontinuada desde o Python 3.13. Não existe substituição direta.

Qualquer coisa é melhor que CGI para fazer a interface de um servidor web com um manipulador de requisição.
    - O sinalizador **`--cgi`** para a interface de linha de comando **python -m http.server** foi descontinuado desde o Python 3.13.
- `importlib`:
    - Método `load_module()`: use `exec_module()` em vez disso.
- `pathlib`:
    - **`.PurePath.is_reserved()`** foi descontinuado desde o Python 3.13. Use `os.path.isreserved()` para detectar caminhos reservados no Windows.
- `platform`:
    - **`platform.java_ver()`** foi descontinuada desde o Python 3.13. Esta função é útil apenas para suporte Jython, tem uma API confusa e é amplamente não testada.
- `sysconfig`:
    - O argumento *check_home* de `sysconfig.is_python_build()` foi descontinuado desde o Python 3.12.
- `threading`:
    - `RLock()` não aceitará argumentos no Python 3.15. A passagem de quaisquer argumentos foi descontinuada a partir do Python 3.14, uma vez que a versão do Python não permite quaisquer argumentos, mas a versão C permite qualquer número de argumentos posicionais ou nomeados, ignorando todos os argumentos.
- `types`:
    - `types.CodeType`: o acesso a **`codeobject.co_lnotab`** foi descontinuado na **PEP 626** [https://peps.python.org/pep-0626/] desde 3.10 e foi planejado para ser removido em 3.12, mas só recebeu uma `DeprecationWarning` adequada em 3.12. Pode ser removido em 3.15. (Contribuição de Nikita Sobolev em gh-101866 [https://github.com/python/cpython/issues/101866].)
- `typing`:
    - A não-documentada sintaxe de argumento nomeado para criar classes `NamedTuple` (por exemplo, `Point = NamedTuple("Point", x=int, y=int)`) foi descontinuada desde o Python 3.13. Em vez disso, use as sintaxes baseada em classe ou funcional.
    - Ao usar a sintaxe funcional de `TypedDict`s, não passar um valor para o parâmetro *fields* (`TD = TypedDict("TD")`) ou passar `None` (`TD = TypedDict("TD", None)`) foi está descontinuado desde o Python 3.13. Use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})` para criar uma classe TypedDict com nenhum campo.
    - A função decoradora **`@typing.no_type_check_decorator`** foi descontinuada desde o Python 3.13. Após oito anos no módulo `typing`,

ela ainda não foi suportada por nenhum verificador de tipo importante.

- Módulos **`sre_compile`**, **`sre_constants`** e **`sre_parse`**.
- `wave`:
  - Os métodos `getmark()`, `setmark()` e `getmarkers()` das classes `Wave_read` e `Wave_write` foram descontinuados desde o Python 3.13.
- `zipimport`:
  - **`zipimport.zipimporter.load_module()`** está descontinuado desde o Python 3.10. Em vez disso, use `exec_module()`. (gh-125746 [https://github.com/python/cpython/issues/125746].)

Remoção pendente no Python 3.16

- O sistema de importação:

  - A definição de `__loader__` em um módulo enquanto falha na definição de `__spec__.loader` está descontinuado. No Python 3.16, **`__loader__`** deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.

- `array`:

  - O código de formato `'u'` (**`wchar_t`**) foi descontinuado na documentação desde o Python 3.3 e em tempo de execução desde o Python 3.13. Use o código de formato `'w'` (`Py_UCS4`) para caracteres Unicode.

- `asyncio`:

  - **`asyncio.iscoroutinefunction()`** foi descontinuado e será removido no Python 3.16, use `inspect.iscoroutinefunction()` em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)

  - O sistema de políticas `asyncio` está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:

    - `asyncio.AbstractEventLoopPolicy`
    - `asyncio.DefaultEventLoopPolicy`
    - `asyncio.WindowsSelectorEventLoopPolicy`
    - `asyncio.WindowsProactorEventLoopPolicy`
    - `asyncio.get_event_loop_policy()`

    - `asyncio.set_event_loop_policy()`

  Os usuários devem usar `asyncio.run()` ou `asyncio.Runner` com *loop_factory* para usar a implementação de loop de eventos desejada.

  por exemplo, para usar `asyncio.SelectorEventLoop` no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventLoop)
```

  (Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- `builtins`:
  - A inversão bit a bit em tipos booleanos, `~True` ou `~False` foi descontinuada desde o Python 3.12, pois produz resultados surpreendentes e não intuitivos (`-2` e `-1`). Use `not x` em vez disso para a negação lógica de um Booleano. No caso raro de você precisar da inversão bit a bit do inteiro subjacente, converta para `int` explicitamente (`~int(x)`).

- `functools`:
  - A chamada da implementação Python de `functools.reduce()` com *function* ou *sequence* como argumentos nomeados foi descontinuada desde o Python 3.14.

- `logging`:
  - O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)

- `mimetypes`:

  - Extensões válidas começam com um '.' ou estão vazias para `mimetypes.MimeTypes.add_type()`. Extensões sem ponto estão descontinuadas e levantarão uma exceção `ValueError` no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)

- `shutil`:
  - A exceção `ExecError` foi descontinuada desde o Python 3.14. Ela não foi usada por nenhuma função em `shutil` desde o Python 3.4, e agora é um alias de `RuntimeError`.

- `symtable`:
  - O método `Class.get_methods` foi descontinuado desde o Python 3.14.

- `sys`:
  - A função `_enablelegacywindowsfsencoding()` foi descontinuada desde o Python 3.13. Use a variável de ambiente `PYTHONLEGACYWINDOWSFSENCODING`.

- `sysconfig`:
  - A função `sysconfig.expand_makefile_vars()` está descontinuada desde o Python 3.14. Em vez disso, use o argumento `vars` de `sysconfig.get_paths()`.

- `tarfile`:
  - O atributo não documentado e não utilizado `TarFile.tarfile` foi descontinuado desde o Python 3.13.

Remoção pendente no Python 3.17

- `datetime`:
  - `strptime()` chama usando uma string de formato contendo `%e` (dia do mês) sem um ano. Isso está descontinuado desde Python 3.15. (Contribuição de Stan Ulbrych em gh-70647 [https://github.com/python/cpython/issues/70647].)

- **`collections.abc`**:
  - **`collections.abc.ByteString`** está agendado para remoção no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`encodings`**:
  - Passar nomes de *codificação* não ASCII para **`encodings.normalize_encoding()`** está descontinuado e sua remoção está prevista para o Python 3.17. (Contribuição de Stan Ulbrych em gh-136702 [https://github.com/python/cpython/issues/136702].)

- **`profile`**:
  - O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)

- **`webbrowser`**:
  - **`webbrowser.MacOSXOSAScript`** está descontinuado em favor de **`webbrowser.MacOS`**. (gh-137586 [https://github.com/python/cpython/issues/137586])

- **typing**:
  - Antes do Python 3.14, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para compatibilidade com versões anteriores, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como **typing.get_origin()** e **typing.get_args()**, em vez de depender de detalhes de implementação privada.
  - **typing.ByteString**, obsoleto desde o Python 3.9, está programado para ser removido no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **Buffer** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    ByteString foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **bytes** e **bytearray**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **ByteString** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **memoryview**, também nunca foram entendidos como subtipos de **ByteString** (seja em tempo de execução ou por verificadores de tipo estáticos).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **tkinter**:
  - Os métodos **trace_variable()**, **trace()** (um apelido de **trace_variable()**), **trace_vdelete()** e **trace_vinfo()** da classe **tkinter.Variable**, descontinuados desde o Python 3.14, têm remoção prevista para o Python 3.17. Em vez deles, utilize **trace_add()**, **trace_remove()** e **trace_info()**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

Remoção pendente no Python 3.19

- `ctypes`:
  - Troca implícita para o layout de estrutura compatível com MSVC definindo `_pack_`, mas não `_layout_` em plataformas não Windows.
- `hashlib`:
  - Em construtores de funções de hash, como `new()`, ou nos construtores nomeados diretamente pelo algoritmo de hash, como `md5()` e `sha256()`, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado `data=` ou `string=` em várias implementações do **`hashlib`**.

    O suporte ao nome do argumento nomeado `string` está agora descontinuado e previsto para ser removido no Python 3.19.

    Antes do Python 3.13, o parâmetro nomeado `string` não era suportado corretamente, dependendo da implementação de funções de hash do backend. Dê preferência a passar os dados iniciais como um argumento posicional para garantir a máxima compatibilidade com versões anteriores.
- `http.cookies`:
  - `http.cookies.Morsel.js_output()` está descontinuado e será removido no Python 3.19.
  - `http.cookies.BaseCookie.js_output()` está descontinuado e será removido no Python 3.19.
- `imaplib`:
  - A alteração de `IMAP4.file` agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade agora não é utilizada e alterar seu valor não fecha automaticamente o arquivo atual.

    Antes do Python 3.14, essa propriedade era usada para implementar os métodos `read()` e `readline()` correspondentes para `IMAP4`, mas isso não ocorre mais desde então.

Remoção pendente no Python 3.20

- Chamar o método `__new__()` de `struct.Struct` sem o argumento *format* está descontinuado e será removido no Python 3.20. Chamar o método `__init__()` em objetos `Struct` já inicializados está descontinuado e será removido no Python 3.20.

  (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize `sys.version_info` em seu lugar.

  - `argparse`
  - `csv`
  - `ctypes`
  - `ctypes.macholib`
  - `decimal` (use `decimal.SPEC_VERSION`)
  - `http.server`
  - `imaplib`
  - `ipaddress`
  - `json`
  - `logging` (`__date__` também descontinuado)
  - `optparse`
  - `pickle`
  - `platform`
  - `re`
  - `socketserver`
  - `tabnanny`
  - `tarfile`
  - `tkinter.font`
  - `tkinter.ttk`
  - `wsgiref.simple_server`
  - `xml.etree.ElementTree`
  - `xml.sax.expatreader`
  - `xml.sax.handler`
  - `zlib`

  (Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

    - Avisos são gerados para linhas `import` encontradas em arquivos *`name`*`.pth`.
    - Arquivos *`name`*`.pth` não são mais decodificados usando a codificação da localidade (*locale encoding*) por padrão. Eles **DEVEM** estar codificados em `utf-8-sig`.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

- **`ast`**:

    - A criação de instâncias de nós abstratos da AST (como **`ast.AST`** ou **`ast.expr`**) está descontinuada e levantará um erro no Python 3.20.

- **`typing`**:

    - É desencorajado realizar verificações com **`isinstance()`** e **`issubclass()`** em classes de protocolo que não foram explicitamente decoradas com **`runtime_checkable()`**, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso resultará em um **`TypeError`** no Python 3.20.

      (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

Remoção pendente em versões futuras

As APIs a seguir serão removidas no futuro, embora atualmente não haja uma data agendada para sua remoção.

- **`argparse`**:
    - O aninhamento de grupos de argumentos e o aninhamento de grupos mutuamente exclusivos estão descontinuado.
    - A passagem do argumento nomeado não documentado *prefix_chars* para **`add_argument_group()`** agora está descontinuado.
    - O conversor de tipo **`argparse.FileType`** está descontinuado.
- **`builtins`**:
    - Geradores: A assinatura `throw(type, exc, tb)` e `athrow(type, exc, tb)` está descontinuada: use `throw(exc)` e `athrow(exc)`, a assinatura do argumento único.

    - Atualmente Python aceita literais numéricos imediatamente seguidos por palavras reservadas como, por exemplo, `0in x`, `1or x`, `0if 1else 2`. Ele permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). Um aviso de sintaxe é levantado se o literal numérico for imediatamente seguido por uma das palavras reservadas **`and`**, **`else`**, **`for`**, **`if`**, **`in`** , **`is`** e **`or`**. Em uma versão futura, será alterado para um erro de sintaxe. (gh-87999 [https://github.com/python/cpython/issues/87999])
    - Suporte para métodos `__index__()` e `__int__()` retornando tipo não-int: esses métodos serão necessários para retornar uma instância de uma subclasse estrita de **`int`**.
    - Suporte para o método `__float__()` retornando uma subclasse estrita de **`float`**: esses métodos serão necessários para retornar uma instância de **`float`**.
    - Suporte para o método `__complex__()` retornando uma subclasse estrita de **`complex`**: esses métodos serão necessários para retornar uma instância de **`complex`**.
    - Passar um número complexo como argumento *real* ou *imag* no construtor **`complex()`** agora está descontinuado; deve ser passado apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)
- **`calendar`**: As constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por **`calendar.JANUARY`** e **`calendar.FEBRUARY`**. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)
- **`codecs`**: use **`open()`** em vez de **`codecs.open()`**. (gh-133038 [https://github.com/python/cpython/issues/133038])
- **`codeobject.co_lnotab`**: use o método **`codeobject.co_lines()`**.
- **`datetime`**:
    - **`utcnow()`**: use `datetime.datetime.now(tz=datetime.UTC)`.
    - **`utcfromtimestamp()`**: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- **`gettext`**: O valor de plural deve ser um número inteiro.
- **`importlib`**:
    - O parâmetro *debug_override* de **`cache_from_source()`** foi descontinuado: em vez disso, use o parâmetro *optimization*.
- **`importlib.metadata`**:
    - Interface de tupla `EntryPoints`.
    - `None` implícito nos valores de retorno.

- `logging`: o método `warn()` foi descontinuado desde o Python 3.3, use `warning()`.
- `mailbox`: O uso da entrada StringIO e do modo de texto foi descontinuado; em vez disso, use BytesIO e o modo binário.
- `os`: chame `os.register_at_fork()` em um processo multithread.
- `os.path`: `os.path.commonprefix()` está descontinuada; use `os.path.commonpath()` para prefixos de caminho. A função **`os.path.commonprefix()`** está sendo descontinuada por possuir nome e módulo enganosos. O uso dessa função não é seguro para prefixos de caminho, apesar de ela estar incluída em um módulo voltado à manipulação de caminhos; isso significa que é fácil introduzir acidentalmente vulnerabilidades de *path traversal* (travessia de diretórios) em programas Python ao utilizá-la.
- **`pydoc.ErrorDuringImport`**: Um valor de tupla para o parâmetro *exc_info* foi descontinuado, use uma instância de exceção.
- `re`: Regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)
- `shutil`: O parâmetro *onerror* de `rmtree()` foi descontinuado no Python 3.12; use o parâmetro *onexc*.
- Protocolos e opções de `ssl`
  - `ssl.SSLContext` sem argumento de protocolo foi descontinuado.
  - `ssl.SSLContext`: `set_npn_protocols()` e **`selected_npn_protocol()`** foram descontinuados: use ALPN.
  - Opções de `ssl.OP_NO_SSL*`
  - Opções de `ssl.OP_NO_TLS*`
  - `ssl.PROTOCOL_SSLv3`
  - `ssl.PROTOCOL_TLS`
  - `ssl.PROTOCOL_TLSv1`
  - `ssl.PROTOCOL_TLSv1_1`
  - `ssl.PROTOCOL_TLSv1_2`
  - `ssl.TLSVersion.SSLv3`
  - `ssl.TLSVersion.TLSv1`
  - `ssl.TLSVersion.TLSv1_1`
- Métodos de `threading`:
  - **`threading.Condition.notifyAll()`**: use `notify_all()`.
  - **`threading.Event.isSet()`**: use `is_set()`.

    - **threading.Thread.isDaemon()**, threading.Thread.setDaemon(): use o atributo threading.Thread.daemon.
    - **threading.Thread.getName()**, threading.Thread.setName(): use o atributo threading.Thread.name.
    - **threading.currentThread()**: use threading.current_thread().
    - **threading.activeCount()**: use threading.active_count().
- typing.Text (gh-92332 [https://github.com/python/cpython/issues/92332]).
- A classe interna `typing._UnionGenericAlias` não é mais usada para implementar typing.Union. Para preservar a compatibilidade com usuários que utilizam esta classe privada, uma correção de compatibilidade será fornecida pelo menos até a versão 3.17 do Python. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)
- unittest.IsolatedAsyncioTestCase: foi descontinuado retornar um valor que não seja `None` de um caso de teste.
- Funções descontinuadas de urllib.parse: use urlparse()
    - `splitattr()`
    - `splithost()`
    - `splitnport()`
    - `splitpasswd()`
    - `splitport()`
    - `splitquery()`
    - `splittag()`
    - `splittype()`
    - `splituser()`
    - `splitvalue()`
    - `to_bytes()`
- wsgiref: `SimpleHandler.stdout.write()` não deve fazer gravações parciais.
- xml.etree.ElementTree: testar o valor verdade de um Element está descontinuado. Em um lançamento futuro isso sempre retornará `True`. Em vez disso, prefira os testes explícitos `len(elem)` ou `elem is not None`.
- sys._clear_type_cache() está descontinuada: use sys._clear_internal_caches().

Removidos

asynchat e asyncore

- Esses dois módulos foram removidos de acordo com o cronograma em PEP 594 [https://peps.python.org/pep-0594/], tendo sido descontinuados no Python 3.6. Use

asyncio em vez disso. (Contribuição de Nikita Sobolev em gh-96580 [https://github.com/python/cpython/issues/96580].)

configparser

- Vários nomes descontinuados no **configparser** na versão 3.2 foram removidos por gh-89336 [https://github.com/python/cpython/issues/89336]:
  - **configparser.ParsingError** não possui mais um argumento ou atributo `filename`. Use o atributo e o argumento `source`.
  - **configparser** não possui mais uma classe `SafeConfigParser`. Use o nome **ConfigParser** mais curto.
  - **configparser.ConfigParser** não possui mais um método `readfp`. Use **read_file()** em vez disso.

distutils

- Remove o pacote **distutils**. Foi descontinuado no Python 3.10 por **PEP 632** [https://peps.python.org/pep-0632/] "Descontinuar o módulo distutils". Para projetos que ainda usam `distutils` e não podem ser atualizados para outra coisa, o projeto `setuptools` pode ser instalado: ele ainda fornece `distutils`. (Contribuição de Victor Stinner em gh-92584 [https://github.com/python/cpython/issues/92584].)

ensurepip

- Remove o wheel de setuptools incluído em **ensurepip** e para de instalar setuptools em ambientes criados por **venv**.

  `pip (>= 22.1)` não requer que setuptools sejam instalados no ambiente. Pacotes baseados em `setuptools` (e baseados em `distutils`) ainda podem ser usados com `pip install`, já que o pip fornecerá `setuptools` no ambiente de construção que ele usa para construir um pacote.

  `easy_install`, `pkg_resources`, `setuptools` e `distutils` não são mais fornecidos por padrão em ambientes criados com `venv` ou inicializados com `ensurepip`, já que eles são parte do pacote `setuptools`. Para projetos que dependem destes em tempo de execução, o projeto `setuptools` deve ser declarado como uma dependência e instalado separadamente (normalmente, usando pip).

(Contribuição de Pradyun Gedam em gh-95299 [https://github.com/python/cpython/issues/95299].)

enum

- Remove `EnumMeta.__getattr__` de **`enum`**, que não é mais necessário para acesso ao atributos do enum. (Contribuição de Ethan Furman em gh-95083 [https://github.com/python/cpython/issues/95083].)

ftplib

- Remove o atributo de classe `FTP_TLS.ssl_version` do **`ftplib`**: use o parâmetro *context* em seu lugar. (Contribuição de Victor Stinner em gh-94172 [https://github.com/python/cpython/issues/94172].)

gzip

- Remove o atributo `filename` do **`gzip.GzipFile`** de **`gzip`**, descontinuado desde Python 2.6, use o atributo **`name`** em seu lugar. No modo de gravação, o atributo `filename` adicionou a extensão de arquivo `'.gz'` se ela não estivesse presente. (Contribuição de Victor Stinner em gh-94196 [https://github.com/python/cpython/issues/94196].)

hashlib

- Remove a implementação Python pura de **`hashlib.pbkdf2_hmac()`** de **`hashlib`**, descontinuado em Python 3.10. Python 3.10 e mais recente requer OpenSSL 1.1.1 (**PEP 644** [https://peps.python.org/pep-0644/]): esta versão do OpenSSL fornece uma implementação C de **`pbkdf2_hmac()`** que é mais rápida. (Contribuição de Victor Stinner em gh-94199 [https://github.com/python/cpython/issues/94199].)

importlib

- Foi concluída a remoção de muitos alvos de **`importlib`** que haviam sido descontinuados anteriormente:
  - Referências e suporte a **`module_repr()`** foram removidos. (Contribuição de Barry Warsaw em gh-97850 [https://github.com/python/cpython/issues/97850].)
  - `importlib.util.set_package`, `importlib.util.set_loader` e `importlib.util.module_for_loader` foram todos removidos.

(Contribuição de Brett Cannon e Nikita Sobolev em gh-65961 [https://github.com/python/cpython/issues/65961] e gh-97850 [https://github.com/python/cpython/issues/97850].)

- O suporte para APIs `find_loader()` e `find_module()` foi removido. (Contribuição de Barry Warsaw em gh-98040 [https://github.com/python/cpython/issues/98040].)
- `importlib.abc.Finder`, `pkgutil.ImpImporter` e `pkgutil.ImpLoader` foram removidos. (Contribuição de Barry Warsaw em gh-98040 [https://github.com/python/cpython/issues/98040].)

imp

- O módulo **`imp`** foi removido. (Contribuição de Barry Warsaw em gh-98040 [https://github.com/python/cpython/issues/98040].)

  Para migrar consulte a seguinte tabela de correspondência:

imp	importlib
`imp.NullImporter`	Insere `None` em `sys.path_importer_cache`
`imp.cache_from_source()`	**`importlib.util.cache_from_source()`**
`imp.find_module()`	**`importlib.util.find_spec()`**
`imp.get_magic()`	**`importlib.util.MAGIC_NUMBER`**
`imp.get_suffixes()`	**`importlib.machinery.SOURCE_SUFFIXES`**, **`importlib.machinery.EXTENSION_SUFFIXES`** e **`importlib.machinery.BYTECODE_SUFFIXES`**
`imp.get_tag()`	**`sys.implementation.cache_tag`**

imp	importlib
`imp.load_module()`	`importlib.import_module()`
`imp.new_module(name)`	`types.ModuleType(name)`
`imp.reload()`	`importlib.reload()`
`imp.source_from_cache()`	`importlib.util.source_from_cache()`
`imp.load_source()`	*Veja abaixo*

Substitua `imp.load_source()` com:

```
import importlib.util
import importlib.machinery

def load_source(modname, filename):
    loader = importlib.machinery.SourceFileLoader(modname, filena
    spec = importlib.util.spec_from_file_location(modname, filena
    module = importlib.util.module_from_spec(spec)
    # O módulo é sempre executado e não colocado em cache em sys.
    # Descomente a linha a seguir para colocar o módulo em cache.
    # sys.modules[module.__name__] = module
    loader.exec_module(module)
    return module
```

- Remove funções e atributos do **`imp`** sem substituições:
  - Funções não documentadas:
    - `imp.init_builtin()`
    - `imp.load_compiled()`
    - `imp.load_dynamic()`
    - `imp.load_package()`
  - `imp.lock_held()`, `imp.acquire_lock()`, `imp.release_lock()`: o esquema de trava foi alterado no Python 3.3 para travas por módulo.

- Constantes de `imp.find_module()`: `SEARCH_ERROR`, `PY_SOURCE`, `PY_COMPILED`, `C_EXTENSION`, `PY_RESOURCE`, `PKG_DIRECTORY`, `C_BUILTIN`, `PY_FROZEN`, `PY_CODERESOURCE`, `IMP_HOOK`.

io

- Remove `io.OpenWrapper` e `_pyio.OpenWrapper` de **`io`**, descontinuados em Python 3.10: basta usar **`open()`** em seu lugar. A função **`open()`** (**`io.open()`**) é uma função embutida. Desde o Python 3.10, **`_pyio.open()`** também é um método estático. (Contribuição de Victor Stinner em gh-94169 [https://github.com/python/cpython/issues/94169].)

locale

- Remove a função **`locale.format()`** de **`locale`**, descontinuada no Python 3.7: use **`locale.format_string()`** em seu lugar. (Contribuição de Victor Stinner em gh-94226 [https://github.com/python/cpython/issues/94226].)

smtpd

- O módulo `smtpd` foi removido de acordo com o cronograma em **PEP 594** [https://peps.python.org/pep-0594/], tendo sido descontinuado em Python 3.4.7 e 3.5.4. Use o módulo PyPI aiosmtpd [https://pypi.org/project/aiosmtpd/] ou qualquer outro servidor baseado em **`asyncio`**. (Contribuição de Oleg Iarygin em gh-93243 [https://github.com/python/cpython/issues/93243].)

sqlite3

- Os seguintes recursos não documentados do **`sqlite3`**, descontinuados no Python 3.10, foram agora removidos:

  - `sqlite3.enable_shared_cache()`
  - `sqlite3.OptimizedUnicode`

  Se um cache compartilhado deve ser usado, abra o banco de dados no modo URI usando o parâmetro de consulta `cache=shared`.

  A fábrica de texto `sqlite3.OptimizedUnicode` tem sido um apelido para **`str`** desde Python 3.3. O código que anteriormente definiu a fábrica de texto como

`OptimizedUnicode` pode usar `str` explicitamente ou confiar no valor padrão que também é `str`.

(Contribuição de Erlend E. Aasland em gh-92548 [https://github.com/python/cpython/issues/92548].)

ssl

- Remove a função **`ssl.RAND_pseudo_bytes()`** do **`ssl`**, descontinuada no Python 3.6: use **`os.urandom()`** ou **`ssl.RAND_bytes()`** em seu lugar. (Contribuição de Victor Stinner em gh-94199 [https://github.com/python/cpython/issues/94199].)
- Remove a função **`ssl.match_hostname()`**. Foi descontinuada no Python 3.7. OpenSSL realiza correspondência de nome de host desde Python 3.7, Python não usa mais a função **`ssl.match_hostname()`**. (Contribuição de Victor Stinner em gh-94199 [https://github.com/python/cpython/issues/94199].)
- Remove a função **`ssl.wrap_socket()`**, descontinuada no Python 3.7: em vez desta, crie um objeto **`ssl.SSLContext`** e chame seu método **`ssl.SSLContext.wrap_socket`**. Qualquer pacote que ainda use **`ssl.wrap_socket()`** está quebrado e é inseguro. A função não envia uma extensão SNI TLS nem valida o nome do host do servidor. O código está sujeito a **CWE 295** [https://cwe.mitre.org/data/definitions/295.html] (validação inadequada de certificado). (Contribuição de Victor Stinner em gh-94199 [https://github.com/python/cpython/issues/94199].)

unittest

- Remove recursos do **`unittest`** há muito tempo descontinuados:
  - Vários apelidos de método de **`TestCase`**:

Apelido descontinuado	Método	Descontinuado em
`failUnless`	**`assertTrue()`**	3.1
`failIf`	**`assertFalse()`**	3.1

Apelido descontinuado	Método	Descontinuado em
`failUnlessEqual`	`assertEqual()`	3.1
`failIfEqual`	`assertNotEqual()`	3.1
`failUnlessAlmostEqual`	`assertAlmostEqual()`	3.1
`failIfAlmostEqual`	`assertNotAlmostEqual()`	3.1
`failUnlessRaises`	`assertRaises()`	3.1
`assert_`	`assertTrue()`	3.2
`assertEquals`	`assertEqual()`	3.2
`assertNotEquals`	`assertNotEqual()`	3.2
`assertAlmostEquals`	`assertAlmostEqual()`	3.2
`assertNotAlmostEquals`	`assertNotAlmostEqual()`	3.2
`assertRegexpMatches`	`assertRegex()`	3.2
`assertRaisesRegexp`	`assertRaisesRegex()`	3.2

Apelido descontinuado	Método	Descontinuado em
`assertNotRegexpMatches`	`assertNotRegex()`	3.5

Você pode usar https://github.com/isidentical/teyit para modernizar automaticamente seus testes de unidade.

- Método não documentado e quebrado `assertDictContainsSubset` de `TestCase` (descontinuado no Python 3.2).
- Parâmetro não documentado *use_load_tests* de `TestLoader.loadTestsFromModule` (descontinuado e ignorado desde Python 3.5).
- Um apelido da classe `TextTestResult`: `_TextTestResult` (descontinuado no Python 3.2).

(Contribuição de Serhiy Storchaka em gh-89325 [https://github.com/python/cpython/issues/89325].)

webbrowser

- Remove o suporte para navegadores descontinuados de `webbrowser`. Os navegadores removidos incluem: Grail, Mosaic, Netscape, Galeon, Skipstone, Iceape, Firebird e Firefox versões 35 e inferiores (gh-102871 [https://github.com/python/cpython/issues/102871]).

xml.etree.ElementTree

- Remove o método `ElementTree.Element.copy()` da implementação Python pura, descontinuado no Python 3.10; use a função `copy.copy()` em seu lugar. A implementação C de `xml.etree.ElementTree` não possui nenhum método `copy()`, apenas um método `__copy__()`. (Contribuição de Victor Stinner em gh-94383 [https://github.com/python/cpython/issues/94383].)

zipimport

- Remove os métodos `find_loader()` e `find_module()` do **`zipimport`**, descontinuados no Python 3.10: use o método `find_spec()` em seu lugar. Veja **PEP 451** [https://peps.python.org/pep-0451/] para o raciocínio. (Contribuição de Victor Stinner em gh-94379 [https://github.com/python/cpython/issues/94379].)

Outros

- Remove a regra `suspicious` do `Makefile` da documentação e do `Doc/tools/rstlint.py`, ambas em favor do sphinx-lint [https://github.com/sphinx-contrib/sphinx-lint]. (Contribuição de Julien Palard em gh-98179 [https://github.com/python/cpython/issues/98179].)
- Remove os parâmetros *keyfile* e *certfile* dos módulos **`ftplib`**, **`imaplib`**, **`poplib`** e **`smtplib`**, e dos módulos *key_file*, *cert_file* e *check_hostname* do módulo **`http.client`**, todos descontinuados desde Python 3.6. Use o parâmetro *context* (*ssl_context* em **`imaplib`**). (Contribuição de Victor Stinner em gh-94172 [https://github.com/python/cpython/issues/94172].)
- Remove hacks de compatibilidade `Jython` de vários módulos e testes da stdlib. (Contribuição de Nikita Sobolev em gh-99482 [https://github.com/python/cpython/issues/99482].)
- Remove o sinalizador `_use_broken_old_ctypes_structure_semantics_` do módulo **`ctypes`**. (Contribuição de Nikita Sobolev em gh-99285 [https://github.com/python/cpython/issues/99285].)

Portando para Python 3.12

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações na API Python

- Regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)

- Remove a funcionalidade `randrange()` descontinuada desde o Python 3.10. Anteriormente, `randrange(10.0)` era convertido sem perdas para `randrange(10)`. Agora, ele levanta um **`TypeError`**. Além disso, a exceção

levantada para valores não inteiros como `randrange(10.5)` ou `randrange('10')` foi alterada de **`ValueError`** para **`TypeError`**. Isso também evita bugs onde `randrange(1e25)` selecionaria silenciosamente um intervalo maior que `randrange(10**25)`. (Originalmente sugerido por Serhiy Storchaka gh-86388 [https://github.com/python/cpython/issues/86388].)

- **`argparse.ArgumentParser`** alterou a codificação e o tratador de erros para ler argumentos do arquivo (por exemplo, opção `fromfile_prefix_chars`) da codificação de texto padrão (por exemplo, **`locale.getpreferredencoding(False)`**) para tratador de erros e codificação do sistema de arquivos. Os arquivos de argumento devem ser codificados em UTF-8 em vez de ANSI Codepage no Windows.

- Remove o módulo `smtpd` baseado em `asyncore` descontinuado no Python 3.4.7 e 3.5.4. Um substituto recomendado é o módulo aiosmtpd [https://pypi.org/project/aiosmtpd/] do PyPI, baseado em **`asyncio`** .

- **`shlex.split()`**: passar `None` para o argumento *s* agora levanta uma exceção, ao invés de ler **`sys.stdin`**. O recurso foi descontinuado no Python 3.9. (Contribuição de Victor Stinner em gh-94352 [https://github.com/python/cpython/issues/94352].)

- O módulo **`os`** não aceita mais caminhos bytes ou similar, como os tipos **`bytearray`** e **`memoryview`**: apenas o tipo exato **`bytes`** é aceito para strings de bytes. (Contribuição de Victor Stinner em gh-98393 [https://github.com/python/cpython/issues/98393].)

- **`syslog.openlog()`** e **`syslog.closelog()`** agora falham se usadas em subinterpretadores. **`syslog.syslog()`** ainda pode ser usada em subinterpretadores, mas agora somente se **`syslog.openlog()`** já tiver sido chamada no interpretador principal. Estas novas restrições não se aplicam ao interpretador principal, então apenas um pequeno grupo de usuários poderá ser afetado. Essa mudança ajuda no isolamento do interpretador. Além disso, **`syslog`** é um wrapper em torno de recursos globais de processo, que são melhor gerenciados a partir do interpretador principal. (Contribuição de Donghee Na em gh-99127 [https://github.com/python/cpython/issues/99127].)

- O comportamento não documentado de trava de **`@~functools.cached_property`** foi removido, porque travava todas as instâncias da classe, levando a uma alta contenção de trava. Isso significa que uma função getter de propriedade em cache agora pode ser executada mais de

uma vez para uma única instância, se duas threads competirem. Para a maioria das propriedades simples em cache (por exemplo, aquelas que são idempotentes e simplesmente calculam um valor com base em outros atributos da instância), isso será suficiente. Se a sincronização for necessária, implemente a trava na função getter de propriedade em cache ou em torno de pontos de acesso multithread.

- `sys._current_exceptions()` agora retorna um mapeamento de thread-id para uma instância de exceção, ao invés de uma tupla `(typ, exc, tb)`. (Contribuição de Irit Katriel em gh-103176 [https://github.com/python/cpython/issues/103176].)

- Ao extrair arquivos tar usando `tarfile` ou `shutil.unpack_archive()`, passe o argumento *filter* para limitar recursos que podem surpreender ou ser perigosos. Veja Filtros de extração para detalhes.

- A saída das funções `tokenize.tokenize()` e `tokenize.generate_tokens()` agora foi alterada devido às mudanças introduzidas na **PEP 701** [https://peps.python.org/pep-0701/]. Isso significa que os tokens `STRING` não são mais emitidos para f-strings e os tokens descritos em **PEP 701** [https://peps.python.org/pep-0701/] agora são produzidos: `FSTRING_START`, `FSTRING_MIDDLE` e `FSTRING_END` agora são emitidos para partes "string" de f-strings, além dos tokens apropriados para a tokenização nos componentes da expressão. Por exemplo, para a f-string `f"start {1+1} end"` a versão antiga do tokenizer emitida:

```
1,0-1,18:           STRING         'f"start {1+1} end"'
```

enquanto a nova versão emite:

```
1,0-1,2:            FSTRING_START  'f"'
1,2-1,8:            FSTRING_MIDDLE 'start '
1,8-1,9:            OP             '{'
1,9-1,10:           NUMBER         '1'
1,10-1,11:          OP             '+'
1,11-1,12:          NUMBER         '1'
1,12-1,13:          OP             '}'
1,13-1,17:          FSTRING_MIDDLE ' end'
1,17-1,18:          FSTRING_END    '"'
```

Além disso, pode haver algumas pequenas alterações comportamentais como consequência das alterações necessárias para oferecer suporte à **PEP 701** [https://peps.python.org/pep-0701/]. Algumas dessas mudanças incluem:

  - O atributo `type` dos tokens emitidos ao tokenizar alguns caracteres Python inválidos, como `!`, mudou de `ERRORTOKEN` para `OP`.
  - Strings incompletas de linha única agora também levantam **`tokenize.TokenError`** como fazem strings multilinhas incompletas.
  - Algum código Python incompleto ou inválido agora levanta **`tokenize.TokenError`** em vez de retornar tokens `ERRORTOKEN` arbitrários ao tokenizá-lo.
  - Não é mais possível misturar tabulações e espaços como indentação no mesmo arquivo, isso vai levantar uma **`TabError`**.

- O módulo **`threading`** agora espera que o módulo **`_thread`** tenha um atributo `_is_main_interpreter`. É uma função sem argumentos que retorna `True` se o interpretador atual for o interpretador principal.

  Qualquer biblioteca ou aplicação que forneça um módulo `_thread` personalizado deve fornecer `_is_main_interpreter()`. (Veja gh-112826 [https://github.com/python/cpython/issues/112826].)

Mudanças na construção

- Python não usa mais `setup.py` para construir módulos de extensão C compartilhados. Parâmetros de construção como cabeçalhos e bibliotecas são detectados no script `configure`. As extensões são construídas por `Makefile`. A maioria das extensões usa `pkg-config` e recorre à detecção manual. (Contribuição de Christian Heimes em gh-93939 [https://github.com/python/cpython/issues/93939].)

- `va_start()` com dois parâmetros, como `va_start(args, format)`, agora é necessário para construir Python. `va_start()` não é mais chamado com um único parâmetro. (Contribuição de Kumar Aditya em gh-93207 [https://github.com/python/cpython/issues/93207].)

- CPython agora usa a opção ThinLTO como política de otimização de tempo de vincualção padrão se o compilador Clang aceitar o sinalizador. (Contribuição de Donghee Na em gh-89536 [https://github.com/python/cpython/issues/89536].)

- Adiciona a variável `COMPILEALL_OPTS` em `Makefile` para substituir as opções **`compileall`** (padrão: `-j0`) em `make install`. Também fundiu os 3 comandos `compileall` em um único comando para construir arquivos .pyc para todos os

níveis de otimização (0, 1, 2) de uma só vez. (Contribuição de Victor Stinner em gh-99289 [https://github.com/python/cpython/issues/99289].)

- Adiciona trios de plataforma para LoongArch de 64 bits:

  - loongarch64-linux-gnusf
  - loongarch64-linux-gnuf32
  - loongarch64-linux-gnu

  (Contribuição de Zhang Na em gh-90656 [https://github.com/python/cpython/issues/90656].)

- `PYTHON_FOR_REGEN` agora requer Python 3.10 ou mais novo.

- Autoconf 2.71 e aclocal 1.16.4 agora são necessários para regerar `configure`. (Contribuição de Christian Heimes em gh-89886 [https://github.com/python/cpython/issues/89886].)

- Construções do Windows e instaladores do macOS em python.org agora usam OpenSSL 3.0.

Alterações na API C

Novas funcionalidades

- **PEP 697** [https://peps.python.org/pep-0697/]: Introduz o tier de API C Instável, destinada a ferramentas de baixo nível como depuradores e compiladores JIT. Esta API pode mudar em cada versão secundária do CPython sem avisos de descontinuação. Seu conteúdo é marcado pelo prefixo `PyUnstable_` nos nomes.

  Construtores de objeto código:

  - `PyUnstable_Code_New()` (renomeado de `PyCode_New`)
  - `PyUnstable_Code_NewWithPosOnlyArgs()` (renomeado de `PyCode_NewWithPosOnlyArgs`)

  Armazenamento extra para objetos código (**PEP 523** [https://peps.python.org/pep-0523/]):

- `PyUnstable_Eval_RequestCodeExtraIndex()` (renomeado de `_PyEval_RequestCodeExtraIndex`)
- `PyUnstable_Code_GetExtra()` (renomeado de `_PyCode_GetExtra`)
- `PyUnstable_Code_SetExtra()` (renomeado de `_PyCode_SetExtra`)

Os nomes originais continuarão disponíveis até que a respectiva API seja alterada.

(Contribuição de Petr Viktorin em gh-101101 [https://github.com/python/cpython/issues/101101].)

- **PEP 697** [https://peps.python.org/pep-0697/]: Adiciona uma API para estender tipos cujo layout de memória de instância é opaco:

  - **`PyType_Spec.basicsize`** pode ser zero ou negativo para especificar a herança ou extensão do tamanho da classe base.
  - **`PyObject_GetTypeData()`** e **`PyType_GetTypeDataSize()`** adicionadas para permitir acesso a dados de instância específicos da subclasse.
  - **`Py_TPFLAGS_ITEMS_AT_END`** e **`PyObject_GetItemData()`** adicionadas para permitir a extensão segura de certos tipos de tamanho variável, incluindo **`PyType_Type`**.
  - **`Py_RELATIVE_OFFSET`** adicionada para permitir a definição de **`membros`** em termos de uma estrutura específica da subclasse.

  (Contribuição de Petr Viktorin em gh-103509 [https://github.com/python/cpython/issues/103509].)

- Adiciona à API C limitada a nova função **`PyType_FromMetaclass()`**, que generaliza a **`PyType_FromModuleAndSpec()`** existente usando um argumento de metaclasse adicional. (Contribuição de Wenzel Jakob em gh-93012 [https://github.com/python/cpython/issues/93012].)

- API para criação de objetos que podem ser chamados usando o protocolo vectorcall foi adicionada à API Limitada:

  - **`Py_TPFLAGS_HAVE_VECTORCALL`**
  - **`PyVectorcall_NARGS()`**
  - **`PyVectorcall_Call()`**
  - **`vectorcallfunc`**

  O sinalizador **`Py_TPFLAGS_HAVE_VECTORCALL`** agora é removido de uma classe quando o método **`__call__()`** da classe é reatribuído. Isso torna vectorcall

seguro para uso com tipos mutáveis (ou seja, tipos de heap sem o sinalizador imutável, `Py_TPFLAGS_IMMUTABLETYPE`). Tipos mutáveis que não substituem `tp_call` agora herdam o sinalizador `Py_TPFLAGS_HAVE_VECTORCALL`. (Contribuição de Petr Viktorin em gh-93274 [https://github.com/python/cpython/issues/93274].)

Os sinalizadores `Py_TPFLAGS_MANAGED_DICT` e `Py_TPFLAGS_MANAGED_WEAKREF` foram adicionados. Isso permite que as classes de extensões ofereçam suporte a objetos `__dict__` e weakrefs com menos trabalho, usando menos memória e com acesso mais rápido.

- API para realizar chamadas usando o protocolo vectorcall foi adicionada à API Limitada:

  - `PyObject_Vectorcall()`
  - `PyObject_VectorcallMethod()`
  - `PY_VECTORCALL_ARGUMENTS_OFFSET`

  Isto significa que ambas as extremidades de entrada e saída do protocolo de chamada vetorial estão agora disponíveis na API Limitada. (Contribuição de Wenzel Jakob em gh-98586 [https://github.com/python/cpython/issues/98586].)

- Adiciona duas novas funções públicas, `PyEval_SetProfileAllThreads()` e `PyEval_SetTraceAllThreads()`, que permitem definir funções de rastreamento e perfilação em todos os threads em execução, além do de chamada. (Contribuição de Pablo Galindo em gh-93503 [https://github.com/python/cpython/issues/93503].)

- Adiciona a nova função `PyFunction_SetVectorcall()` à API C que define o campo vectorcall de um determinado `PyFunctionObject`. (Contribuição de Andrew Frost em gh-92257 [https://github.com/python/cpython/issues/92257].)

- A API C agora permite registrar funções de retorno de chamada via `PyDict_AddWatcher()`, `PyDict_Watch()` e APIs relacionadas a serem chamadas sempre que um dicionário é modificado. Destina-se a ser usado para otimizar interpretadores, compiladores JIT ou depuradores. (Contribuição de Carl Meyer em gh-91052 [https://github.com/python/cpython/issues/91052].)

- Adiciona a API `PyType_AddWatcher()` e `PyType_Watch()` para registrar retornos de chamada para receber notificações sobre alterações em um tipo.

(Contribuição de Carl Meyer em gh-91051 [https://github.com/python/cpython/issues/91051].)

- Adiciona APIs `PyCode_AddWatcher()` e `PyCode_ClearWatcher()` para registrar retornos de chamada para receber notificação sobre criação e destruição de objetos de código. (Contribuição de Itamar Oren em gh-91054 [https://github.com/python/cpython/issues/91054].)

- Adiciona as funções `PyFrame_GetVar()` e `PyFrame_GetVarString()` para obter uma variável de quadro por seu nome. (Contribuição de Victor Stinner em gh-91248 [https://github.com/python/cpython/issues/91248].)

- Adiciona `PyErr_GetRaisedException()` e `PyErr_SetRaisedException()` para salvar e restaurar a exceção atual. Essas funções retornam e aceitam um único objeto de exceção, em vez dos argumentos triplos dos agora obsoletos `PyErr_Fetch()` e `PyErr_Restore()`. Isso é menos sujeito a erros e um pouco mais eficiente. (Contribuição de Mark Shannon em gh-101578 [https://github.com/python/cpython/issues/101578].)

- Adiciona `_PyErr_ChainExceptions1`, que usa uma instância de exceção, para substituir a API legada `_PyErr_ChainExceptions`, que agora foi descontinuada. (Contribuição de Mark Shannon em gh-101578 [https://github.com/python/cpython/issues/101578].)

- Adiciona `PyException_GetArgs()` e `PyException_SetArgs()` como funções de conveniência para recuperar e modificar o `args` passado para o construtor da exceção. (Contribuição de Mark Shannon em gh-101578 [https://github.com/python/cpython/issues/101578].)

- Adiciona `PyErr_DisplayException()`, que usa uma instância de exceção, para substituir a API legada `PyErr_Display()`. (Contribuição de Irit Katriel em gh-102755 [https://github.com/python/cpython/issues/102755]).

- **PEP 683** [https://peps.python.org/pep-0683/]: Introduz *Objetos Imortais*, que permite que objetos ignorem contagens de referências e alterações relacionadas à API C:

  - `_Py_IMMORTAL_REFCNT`: A contagem de referências que define um objeto como imortal.

  - `_Py_IsImmortal` Verifica se um objeto possui a contagem de referências imortal.

- `PyObject_HEAD_INIT` Isso agora inicializará a contagem de referências para
  `_Py_IMMORTAL_REFCNT` quando usado com `Py_BUILD_CORE`.
- `SSTATE_INTERNED_IMMORTAL` Um identificador para objetos unicode internalizados
  que são imortais.
- `SSTATE_INTERNED_IMMORTAL_STATIC` Um identificador para unicode internalizado
  objetos que são imortais e estáticos
- `sys.getunicodeinternedsize` Isso retorna o número total de unicode
  objetos que foram internalizados. Isso agora é necessário para `refleak.py` rastrear corretamente contagens de referências e blocos alocados

(Contribuição de Eddie Elizondo em gh-84436 [https://github.com/python/cpython/issues/84436].)

- **PEP 684** [https://peps.python.org/pep-0684/]: Adiciona a nova função **`Py_NewInterpreterFromConfig()`** e **`PyInterpreterConfig`**, que podem ser usadas para criar subinterpretadores com suas próprias GILs. (Veja PEP 684: Uma GIL por interpretador para mais informações.) (Contribuição de Eric Snow em gh-104110 [https://github.com/python/cpython/issues/104110].)

- Na API C limitada versão 3.12, as funções **`Py_INCREF()`** e **`Py_DECREF()`** agora são implementadas como chamadas de função opacas para ocultar detalhes de implementação. (Contribuição de Victor Stinner em gh-105387 [https://github.com/python/cpython/issues/105387].)

Portando para Python 3.12

- APIs Unicode legadas baseadas na representação de `Py_UNICODE*` foram removidas. Migre para APIs baseadas em UTF-8 ou `wchar_t*`.

- Funções de análise de argumentos como **`PyArg_ParseTuple()`** não mais oferecem suporte a formato baseado em `Py_UNICODE*` (por exemplo, `u`, `Z`). Migre para outros formatos para Unicode como `s`, `z`, `es` e `U`.

- `tp_weaklist` para todos os tipos embutidos estáticos é sempre `NULL`. Este é um campo somente interno em `PyTypeObject`, mas estamos apontando a

mudança caso alguém acesse o campo diretamente de qualquer maneira. Para evitar quebras, considere usar a API C pública existente ou, se necessário, a macro `_PyObject_GET_WEAKREFS_LISTPTR()` (somente interna).

- Este **`PyTypeObject.tp_subclasses`** somente interno pode agora não ser um ponteiro de objeto válido. Seu tipo foi alterado para void* para refletir isso. Mencionamos isso caso alguém esteja acessando diretamente o campo somente interno.

  Para obter uma lista de subclasses, chame o método Python **`__subclasses__()`** (usando **`PyObject_CallMethod()`**, por exemplo).

- Adicionado suporte para mais opções de formatação (alinhamento à esquerda, octais, hexadecimais maiúsculos, **`intmax_t`**, **`ptrdiff_t`**, **`wchar_t`** strings C, largura variável e precisão) em **`PyUnicode_FromFormat()`** e **`PyUnicode_FromFormatV()`**. (Contribuição de Serhiy Storchaka em gh-98836 [https://github.com/python/cpython/issues/98836].)

- Um caractere de formato não reconhecido em **`PyUnicode_FromFormat()`** e **`PyUnicode_FromFormatV()`** agora define um **`SystemError`**. Nas versões anteriores, fazia com que todo o restante da string de formato fosse copiado como estava para a string de resultado e quaisquer argumentos extras descartados. (Contribuição de Serhiy Storchaka em gh-95781 [https://github.com/python/cpython/issues/95781].)

- Corrige o posicionamento incorreto do sinal em **`PyUnicode_FromFormat()`** e **`PyUnicode_FromFormatV()`**. (Contribuição de Philip Georgi em gh-95504 [https://github.com/python/cpython/issues/95504].)

- Classes de extensão que desejam adicionar um **`__dict__`** ou slot de referência fraca devem usar **`Py_TPFLAGS_MANAGED_DICT`** e **`Py_TPFLAGS_MANAGED_WEAKREF`** em vez de `tp_dictoffset` e `tp_weaklistoffset`, respectivamente. O uso de `tp_dictoffset` e `tp_weaklistoffset` ainda é aceitado, mas não oferece suporte totalmente a herança múltipla (gh-95589 [https://github.com/python/cpython/issues/95589]), e o desempenho pode ser pior. Classes que declaram **`Py_TPFLAGS_MANAGED_DICT`** devem chamar **`_PyObject_VisitManagedDict()`** e **`_PyObject_ClearManagedDict()`** para percorrer e limpar os dicionários de sua instância. Para limpar weakrefs (referências fracas), chame **`PyObject_ClearWeakRefs()`**, como antes.

- A função **PyUnicode_FSDecoder()** não aceita mais caminhos bytes ou similar, como os tipos **bytearray** e **memoryview**: apenas o tipo exato **bytes** é aceito para strings de bytes. (Contribuição de Victor Stinner em gh-98393 [https://github.com/python/cpython/issues/98393].)

- As macros **Py_CLEAR**, **Py_SETREF** e **Py_XSETREF** agora avaliam seus argumentos apenas uma vez. Se um argumento tiver efeitos colaterais, esses efeitos colaterais não serão mais duplicados. (Contribuição de Victor Stinner em gh-98724 [https://github.com/python/cpython/issues/98724].)

- O indicador de erro do interpretador agora está sempre normalizado. Isso significa que **PyErr_SetObject()**, **PyErr_SetString()** e as outras funções que definem o indicador de erro agora normalizam a exceção antes de armazená-la. (Contribuição de Mark Shannon em gh-101578 [https://github.com/python/cpython/issues/101578].)

- `_Py_RefTotal` não é mais oficial e é mantido apenas para compatibilidade com ABI. Observe que é um global interno e está disponível apenas em compilações de depuração. Se acontecer de você usá-lo, você precisará começar a usar `_Py_GetGlobalRefTotal()`.

- As seguintes funções agora selecionam uma metaclasse apropriada para o tipo recém-criado:

  - **PyType_FromSpec()**
  - **PyType_FromSpecWithBases()**
  - **PyType_FromModuleAndSpec()**

  A criação de classes cujas substituições de metaclasse **tp_new** foi descontinuada e no Python 3.14+ não será permitida. Observe que essas funções ignoram `tp_new` da metaclasse, possivelmente permitindo uma inicialização incompleta.

  Observe que **PyType_FromMetaclass()** (adicionada em Python 3.12) já não permite a criação de classes cuja metaclasse substitui `tp_new` (**__new__()** em Python).

  Como `tp_new` substitui quase tudo que as funções `PyType_From*` fazem, os dois são incompatíveis entre si. O comportamento existente – ignorar a metaclasse em vários passos de criação do tipo – é inseguro em geral, uma

vez que as (meta)classes presumem que `tp_new` foi chamado. Não existe uma solução geral simples. Um dos seguintes pode funcionar para você:

- Se você controla a metaclasse, evite usar `tp_new` nela:
  - Se a inicialização puder ser ignorada, isso pode ser feito em **`tp_init`**.
  - Se a metaclasse não precisa ser instanciada do Python, defina seu `tp_new` como `NULL` usando o sinalizador **`Py_TPFLAGS_DISALLOW_INSTANTIATION`**. Isso o torna aceitável para funções `PyType_From*`.
- Evite funções `PyType_From*`: se você não precisa de recursos específicos do C (slots ou configuração do tamanho da instância), crie tipos chamando a metaclasse.
- Se você *sabe* que `tp_new` pode ser ignorado com segurança, filtre o aviso de descontinuação usando **`warnings.catch_warnings()`** do Python.

- **`PyOS_InputHook`** e **`PyOS_ReadlineFunctionPointer`** não são mais chamados em subinterpretadores. Isso ocorre porque os clientes geralmente dependem do estado global de todo o processo (uma vez que esses retornos de chamada não têm como recuperar o estado do módulo de extensão).

  Isso também evita situações em que as extensões podem ser executadas em um subinterpretador que não oferecem suporte (ou no qual ainda não foram carregadas). Veja gh-104668 [https://github.com/python/cpython/issues/104668] para mais informações.

- **`PyLongObject`** teve seus componentes internos alterados para melhor desempenho. Embora os internos de **`PyLongObject`** sejam privados, eles são usados por alguns módulos de extensão. Os campos internos não devem mais ser acessados diretamente, em vez disso, as funções da API que começam com `PyLong_...` devem ser usadas. Duas novas funções de API *instáveis* são fornecidas para acesso eficiente ao valor de **`PyLongObject`**s que cabe em uma única palavra de máquina:

  - **`PyUnstable_Long_IsCompact()`**
  - **`PyUnstable_Long_CompactValue()`**

- Alocadores personalizados, definidos via **`PyMem_SetAllocator()`**, agora precisam ser seguros para thread, independentemente do domínio de memória. Os alocadores que não possuem estado próprio, incluindo “ganchos”, não são afetados. Se o seu alocador personalizado ainda não for

seguro para thread e você precisar de orientação, crie um novo relatório de problema no GitHub e CC `@ericsnowcurrently`.

Descontinuados

- Em conformidade com a **PEP 699** [https://peps.python.org/pep-0699/], o campo `ma_version_tag` em `PyDictObject` foi descontinuado para módulos de extensão. Acessar este campo gerará um aviso do compilador em tempo de compilação. Este campo será removido no Python 3.14. (Contribuição de Ramvikrams e Kumar Aditya em gh-101193 [https://github.com/python/cpython/issues/101193]. PEP de Ken Jin.)

- Variáveis de configuração global descontinuadas:

  - `Py_DebugFlag`: use `PyConfig.parser_debug`
  - `Py_VerboseFlag`: use `PyConfig.verbose`
  - `Py_QuietFlag`: use `PyConfig.quiet`
  - `Py_InteractiveFlag`: use `PyConfig.interactive`
  - `Py_InspectFlag`: use `PyConfig.inspect`
  - `Py_OptimizeFlag`: use `PyConfig.optimization_level`
  - `Py_NoSiteFlag`: use `PyConfig.site_import`
  - `Py_BytesWarningFlag`: use `PyConfig.bytes_warning`
  - `Py_FrozenFlag`: use `PyConfig.pathconfig_warnings`
  - `Py_IgnoreEnvironmentFlag`: use `PyConfig.use_environment`
  - `Py_DontWriteBytecodeFlag`: use `PyConfig.write_bytecode`
  - `Py_NoUserSiteDirectory`: use `PyConfig.user_site_directory`
  - `Py_UnbufferedStdioFlag`: use `PyConfig.buffered_stdio`
  - `Py_HashRandomizationFlag`: use `PyConfig.use_hash_seed` and `PyConfig.hash_seed`
  - `Py_IsolatedFlag`: use `PyConfig.isolated`
  - `Py_LegacyWindowsFSEncodingFlag`: use `PyPreConfig.legacy_windows_fs_encoding`
  - `Py_LegacyWindowsStdioFlag`: use `PyConfig.legacy_windows_stdio`
  - **`Py_FileSystemDefaultEncoding`**: use `PyConfig.filesystem_encoding`
  - **`Py_HasFileSystemDefaultEncoding`**: use `PyConfig.filesystem_encoding`
  - **`Py_FileSystemDefaultEncodeErrors`**: use `PyConfig.filesystem_errors`
  - **`Py_UTF8Mode`**: use `PyPreConfig.utf8_mode` (see `Py_PreInitialize()`)

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig`. (Contribuição de Victor Stinner em gh-77782 [https://github.com/python/cpython/issues/77782].)

- A criação de `tipos imutáveis` com bases mutáveis foi descontinuada e será desativada no Python 3.14. (gh-95388 [https://github.com/python/cpython/issues/95388])

- O cabeçalho `structmember.h` foi descontinuado, embora continue disponível e não haja planos para removê-lo.

  Seu conteúdo agora está disponível apenas incluindo `Python.h`, com um prefixo `Py` adicionado se estiver faltando:

  - `PyMemberDef`, `PyMember_GetOne()` e `PyMember_SetOne()`
  - Macros de tipo como `Py_T_INT`, `Py_T_DOUBLE`, etc. (anteriormente `T_INT`, `T_DOUBLE`, etc.)
  - Os sinalizadores `Py_READONLY` (anteriormente `READONLY`) e `Py_AUDIT_READ` (anteriormente todas em maiúsculas)

  Vários itens não são expostos em `Python.h`:

  - `T_OBJECT` (use `Py_T_OBJECT_EX`)
  - `T_NONE` (anteriormente não documentada e bastante estranha)
  - A macro `WRITE_RESTRICTED` que não faz nada.
  - As macros `RESTRICTED` e `READ_RESTRICTED`, equivalentes a `Py_AUDIT_READ`.
  - Em algumas configurações, `<stddef.h>` não está incluído em `Python.h`. Deve ser incluído manualmente ao usar `offsetof()`.

  O cabeçalho descontinuado continua a fornecer seu conteúdo original com os nomes originais. Seu código antigo pode permanecer inalterado, a menos que as macros extras e sem namespace o incomodem muito.

  (Contribuição em gh-47146 [https://github.com/python/cpython/issues/47146] de Petr Viktorin, baseado em trabalhos anteriores de Alexander Belopolsky e Matthias Braun.)

- `PyErr_Fetch()` e `PyErr_Restore()` foram descontinuado. Use `PyErr_GetRaisedException()` e `PyErr_SetRaisedException()` em vez disso. (Contribuição de Mark Shannon em gh-101578 [https://github.com/python/cpython/issues/101578].)

- **`PyErr_Display()`** foi descontinuada. Use **`PyErr_DisplayException()`** em vez disso. (Contribuição de Irit Katriel em gh-102755 [https://github.com/python/cpython/issues/102755]).

- `_PyErr_ChainExceptions` foi descontinuada. Use `_PyErr_ChainExceptions1` em vez disso. (Contribuição de Irit Katriel em gh-102192 [https://github.com/python/cpython/issues/102192].)

- Usar **`PyType_FromSpec()`**, **`PyType_FromSpecWithBases()`** ou **`PyType_FromModuleAndSpec()`** para criar uma classe cujas substituições de metaclasse **`tp_new`** foi descontinuado. Chame a metaclasse.

Remoção pendente no Python 3.14

- O campo `ma_version_tag` em **`PyDictObject`** para módulos de extensão (**PEP 699** [https://peps.python.org/pep-0699/]; gh-101193 [https://github.com/python/cpython/issues/101193]).
- A criação de **`tipos imutáveis`** com bases mutáveis (gh-95388 [https://github.com/python/cpython/issues/95388]).

Remoção pendente no Python 3.15

- A **`PyImport_ImportModuleNoBlock()`**: Use **`PyImport_ImportModule()`**

- **`PyWeakref_GetObject()`** e **`PyWeakref_GET_OBJECT()`**: Use **`PyWeakref_GetRef()`**. O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pod ser usado para obter **`PyWeakref_GetRef()`** no Python 3.12 e versões mais antigas.

- **`PyUnicode_AsDecodedObject()`**: Use **`PyCodec_Decode()`**.

- **`PyUnicode_AsDecodedUnicode()`**: Use **`PyCodec_Decode()`**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **`str`**, tal como **`bytes`**.

- **`PyUnicode_AsEncodedObject()`**: Use **`PyCodec_Encode()`**.

- **`PyUnicode_AsEncodedUnicode()`**: Use **`PyCodec_Encode()`**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **`bytes`**, tal com **`str`**.

- Funções de inicialização do Python, descontinuadas no Python 3.9:
  - **`Py_GetPath()`**: Use `PyConfig_Get("module_search_paths")` (`sys.path`).
  - **`Py_GetPrefix()`**: Use `PyConfig_Get("base_prefix")` (`sys.base_prefix`). Use **`PyConfig_Get("prefix")`** (`sys.prefix`) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetExecPrefix()`**: Use `PyConfig_Get("base_exec_prefix")` (`sys.base_exec_prefix`). Use **`PyConfig_Get("exec_prefix")`** (`sys.exec_prefix`) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetProgramFullPath()`**: Use `PyConfig_Get("executable")` (`sys.executable`).
  - **`Py_GetProgramName()`**: Use `PyConfig_Get("executable")` (`sys.executable`).
  - **`Py_GetPythonHome()`**: Use `PyConfig_Get("home")` ou a variável de ambiente `PYTHONHOME`.

  O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter `PyConfig_Get()` no Python 3.13 e versões anteriores.

- Funções para configurar a inicialização do Python, descontinuadas no Python 3.11:
  - **`PySys_SetArgvEx()`**: Defina `PyConfig.argv`.
  - **`PySys_SetArgv()`**: Defina `PyConfig.argv`.
  - **`Py_SetProgramName()`**: Defina `PyConfig.program_name`.
  - **`Py_SetPythonHome()`**: Defina `PyConfig.home`.
  - **`PySys_ResetWarnOptions()`**: Limpe `sys.warnoptions` e **`warnings.filters`**.

  A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig`.

- Variáveis de configuração globais
  - `Py_DebugFlag`: Use `PyConfig.parser_debug` ou `PyConfig_Get("parser_debug")`.
  - `Py_VerboseFlag`: Use `PyConfig.verbose` ou `PyConfig_Get("verbose")`.
  - `Py_QuietFlag`: Use `PyConfig.quiet` ou `PyConfig_Get("quiet")`.
  - `Py_InteractiveFlag`: Use `PyConfig.interactive` ou `PyConfig_Get("interactive")`.

- `Py_InspectFlag`: Use `PyConfig.inspect` ou `PyConfig_Get("inspect")`.
- `Py_OptimizeFlag`: Use `PyConfig.optimization_level` ou `PyConfig_Get("optimization_level")`.
- `Py_NoSiteFlag`: Use `PyConfig.site_import` ou `PyConfig_Get("site_import")`.
- `Py_BytesWarningFlag`: Use `PyConfig.bytes_warning` ou `PyConfig_Get("bytes_warning")`.
- `Py_FrozenFlag`: Use `PyConfig.pathconfig_warnings` ou `PyConfig_Get("pathconfig_warnings")`.
- `Py_IgnoreEnvironmentFlag`: Use `PyConfig.use_environment` ou `PyConfig_Get("use_environment")`.
- `Py_DontWriteBytecodeFlag`: Use `PyConfig.write_bytecode` ou `PyConfig_Get("write_bytecode")`.
- `Py_NoUserSiteDirectory`: Use `PyConfig.user_site_directory` ou `PyConfig_Get("user_site_directory")`.
- `Py_UnbufferedStdioFlag`: Use `PyConfig.buffered_stdio` ou `PyConfig_Get("buffered_stdio")`.
- `Py_HashRandomizationFlag`: Use `PyConfig.use_hash_seed` e `PyConfig.hash_seed` ou `PyConfig_Get("hash_seed")`.
- `Py_IsolatedFlag`: Use `PyConfig.isolated` ou `PyConfig_Get("isolated")`.
- `Py_LegacyWindowsFSEncodingFlag`: Use `PyPreConfig.legacy_windows_fs_encoding` ou `PyConfig_Get("legacy_windows_fs_encoding")`.
- `Py_LegacyWindowsStdioFlag`: Use `PyConfig.legacy_windows_stdio` ou `PyConfig_Get("legacy_windows_stdio")`.
- **`Py_FileSystemDefaultEncoding`**, **`Py_HasFileSystemDefaultEncoding`**: Use `PyConfig.filesystem_encoding` ou `PyConfig_Get("filesystem_encoding")`.
- **`Py_FileSystemDefaultEncodeErrors`**: Use `PyConfig.filesystem_errors` ou `PyConfig_Get("filesystem_errors")`.
- **`Py_UTF8Mode`**: Use `PyPreConfig.utf8_mode` ou `PyConfig_Get("utf8_mode")`. (veja `Py_PreInitialize()`)

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig` para definir essas opções. Ou `PyConfig_Get()` pode ser usado para obter essas opções em tempo de execução.

Remoção pendente no Python 3.16

- A cópia empacotada do `libmpdec`.

Remoção pendente em versões futuras

As APIs a seguir foram descontinuado e serão removidas, embora atualmente não haja uma data agendada para sua remoção.

- `Py_TPFLAGS_HAVE_FINALIZE`: desnecessária desde o Python 3.8.
- `PyErr_Fetch()`: Use `PyErr_GetRaisedException()`.
- `PyErr_NormalizeException()`: Use `PyErr_GetRaisedException()`.
- `PyErr_Restore()`: Use `PyErr_SetRaisedException()`.
- `PyModule_GetFilename()`: Use `PyModule_GetFilenameObject()`.
- `PyOS_AfterFork()`: Use `PyOS_AfterFork_Child()`.
- `PySlice_GetIndicesEx()`: Use `PySlice_Unpack()` e `PySlice_AdjustIndices()`.
- `PyUnicode_READY()`: Desnecessário desde o Python 3.12
- `PyErr_Display()`: Use `PyErr_DisplayException()`.
- `_PyErr_ChainExceptions()`: Use `_PyErr_ChainExceptions1()`.
- O membro `PyBytesObject.ob_shash`: chame `PyObject_Hash()`.
- API do Thread Local Storage (TLS):
  - `PyThread_create_key()`: Use `PyThread_tss_alloc()`.
  - `PyThread_delete_key()`: Use `PyThread_tss_free()`.
  - `PyThread_set_key_value()`: Use `PyThread_tss_set()`.
  - `PyThread_get_key_value()`: Use `PyThread_tss_get()`.
  - `PyThread_delete_key_value()`: Use `PyThread_tss_delete()`.
  - `PyThread_ReInitTLS()`: Desnecessário desde o Python 3.7.

Removidos

- Remove o arquivo de cabeçalho `token.h`. Nunca houve nenhuma API C de tokenizador público. O arquivo de cabeçalho `token.h` foi projetado apenas para ser usado por internos do Python. (Contribuição de Victor Stinner em gh-92651 [https://github.com/python/cpython/issues/92651].)
- As APIs Unicode legadas foram removidas. Veja a **PEP 623** [https://peps.python.org/pep-0623/] para detalhes.
  - `PyUnicode_WCHAR_KIND`
  - `PyUnicode_AS_UNICODE()`
  - `PyUnicode_AsUnicode()`

  - **`PyUnicode_AsUnicodeAndSize()`**
  - **`PyUnicode_AS_DATA()`**
  - **`PyUnicode_FromUnicode()`**
  - **`PyUnicode_GET_SIZE()`**
  - **`PyUnicode_GetSize()`**
  - **`PyUnicode_GET_DATA_SIZE()`**
- Remove a macro de função `PyUnicode_InternImmortal()`. (Contribuição de Victor Stinner em gh-85858 [https://github.com/python/cpython/issues/85858].)

O que há de novo no Python 3.11

Editor:: Pablo Galindo Salgado

Este artigo explica os novos recursos no Python 3.11, em comparação com 3.10. Python 3.11 foi lançado em 24 de outubro de 2022. Veja changelog para uma lista completa de mudanças.

Resumo – Destaques da versão

- O Python 3.11 é entre 10-60% mais rápido que o Python 3.10. Em média, medimos uma aceleração de 1,25x no conjunto de benchmarks padrão. Veja CPython mais rápido para detalhes.

Novos recursos de sintaxe:

- PEP 654: Grupos de exceção e except*

Novos recursos embutidos:

- PEP 678: Exceções podem ser enriquecidas com notas

Novos módulos da biblioteca padrão:

- **PEP 680** [https://peps.python.org/pep-0680/]: `tomllib` — Suporte para analisar TOML [https://toml.io/] na Biblioteca Padrão

Melhorias no interpretador:

- PEP 657: Localizações de erros refinadas em tracebacks
- Nova opção de linha de comando `-P` e variável de ambiente `PYTHONSAFEPATH` para desabilitar automaticamente a anteposição automática de caminhos potencialmente inseguros para `sys.path`

Novos recursos de tipagem:

- PEP 646: Genéricos variádicos
- PEP 655: Marcando itens TypedDict individuais como obrigatórios ou não obrigatórios

- PEP 673: Tipo Self
- PEP 675: Tipo de string literal arbitrário
- PEP 681: Transformações de classe de dados

Descontinuações, remoções e restrições importantes:

- **PEP 594** [https://peps.python.org/pep-0594/]: Muitos módulos legados de biblioteca padrão foram descontinuados e serão removidos no Python 3.13
- **PEP 624** [https://peps.python.org/pep-0624/]: APIs do codificador Py_UNICODE foram removidas
- **PEP 670** [https://peps.python.org/pep-0670/]: Macros convertidos em funções estáticas em linha

Novas funcionalidades

PEP 657: Localizações de erros refinadas em tracebacks

Ao imprimir tracebacks (situação da pilha de execução), o interpretador agora apontará para a expressão exata que causou o erro, em vez de apenas a linha. Por exemplo:

```
Traceback (most recent call last):
  File "distância.py", line 11, in <module>
    print(distância_de_manhattan(p1, p2))
          ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "distância.py", line 6, in distância_de_manhattan
    return abs(ponto_1.x - ponto_2.x) + abs(ponto_1.y - ponto_2.y)
                           ^^^^^^^^^
AttributeError: 'NoneType' object has no attribute 'x'
```

Versões anteriores do interpretador apontavam apenas para a linha, tornando ambíguo qual objeto era `None`. Esses erros aprimorados também podem ser úteis ao lidar com objetos `dict` profundamente aninhados e várias chamadas de função:

```
Traceback (most recent call last):
  File "consultar.py", line 37, in <module>
    mágica_aritmética('foo')
  File "consultar.py", line 18, in mágica_aritmética
    return adicionar_contagens(x) / 25
           ^^^^^^^^^^^^^
  File "consultar.py", line 24, in adicionar_contagens
    return 25 + consultar_usuário(usuário1) + consultar_usuário(usuár
```

```
                ^^^^^^^^^^^^^^^^^^
  File "consultar.consultar", line 32, in consultar_usuário
    return 1 + consultar_contagem(db, resposta['a']['b']['c']['user']
                                      ~~~~~~~~~~~~~~~~~~~^^^^^
TypeError: 'NoneType' object is not subscriptable
```

Bem como expressões aritméticas complexas:

```
Traceback (most recent call last):
  File "cálculo.py", line 54, in <module>
    resultado = (x / y / z) * (a / b / c)
                 ~~~~~~^~~
ZeroDivisionError: division by zero
```

Além disso, as informações usadas pela funcionalidade de traceback aprimorado são disponibilizadas por meio de uma API geral, que pode ser usada para correlacionar instruções bytecode com a localização do código-fonte. Essas informações podem ser recuperadas usando:

- O método `codeobject.co_positions()` no Python.
- A função `PyCode_Addr2Location()` na API C.

Veja **PEP 657** [https://peps.python.org/pep-0657/] para mais detalhes. (Contribuição de Pablo Galindo, Batuhan Taskaya e Ammar Askar em bpo-43950 [https://bugs.python.org/issue?@action=redirect&bpo=43950].)

Nota

Esse recurso requer o armazenamento de posições de coluna em Objetos código, o que pode resultar em um pequeno aumento no uso de memória do interpretador e no uso de disco para arquivos Python compilados. Para evitar armazenar as informações extras e desativar a exibição das informações extras de rastreamento, use a opção de linha de comando `-X no_debug_ranges` ou a variável de ambiente `PYTHONNODEBUGRANGES`.

PEP 654: Grupos de exceção e `except*`

PEP 654 [https://peps.python.org/pep-0654/] introduz recursos de linguagem que permitem que um programa levante e manipule várias exceções não relacionadas simultaneamente. Os tipos internos `ExceptionGroup` e `BaseExceptionGroup`

tornam possível agrupar exceções e criá-las juntas, e a nova sintaxe **`except*`** generaliza **`except`** para corresponder a subgrupos de grupos de exceção.

Veja **PEP 654** [https://peps.python.org/pep-0654/] para mais detalhes.

(Contribuição de Irit Katriel em bpo-45292 [https://bugs.python.org/issue?@action=redirect&bpo=45292]. PEP escrita por Irit Katriel, Yury Selivanov e Guido van Rossum.)

PEP 678: Exceções podem ser enriquecidas com notas

O método **`add_note()`** é adicionado a **`BaseException`**. Ele pode ser usado para enriquecer exceções com informações de contexto que não estão disponíveis no momento em que a exceção é gerada. As notas adicionadas aparecem no traceback padrão.

Veja **PEP 678** [https://peps.python.org/pep-0678/] para mais detalhes.

(Contribuição de Irit Katriel em bpo-45607 [https://bugs.python.org/issue?@action=redirect&bpo=45607]. PEP escrita por Zac Hatfield-Dodds.)

Melhorias no inicializador `py.exe` do Windows

A cópia do Gerenciador de instalação do Python incluída no Python 3.11 foi significativamente atualizada. Ele agora oferece suporte à sintaxe de company/tag conforme definido em **PEP 514** [https://peps.python.org/pep-0514/] usando o argumento `-V:<company>/<tag>` em vez do limitado `-<major>.<minor>`. Isso permite iniciar outras distribuições além de `PythonCore`, aquela hospedada em python.org [https://www.python.org].

Ao usar seletores `-V:`, tanto company quanto tag podem ser omitidos, mas todas as instalações serão pesquisadas. Por exemplo, `-V:OtherPython/` selecionará a "melhor" tag registrada para `OtherPython`, enquanto `-V:3.11` ou `-V:/3.11` selecionará o "best" com a tag `3.11`.

Ao usar os argumentos legados `-<major>`, `-<major>.<minor>`, `-<major>-<bitness>` ou `-<major>.<minor>-<bitness>`, todo o comportamento existente deve ser preservado de versões anteriores e somente lançamentos de `PythonCore` será selecionado. No entanto, o sufixo `-64` agora implica "não 32 bits" (não necessariamente x86-64), pois existem várias plataformas de 64 bits suportadas. Tempos de execução de 32 bits são detectados verificando a tag do tempo de

execução para um sufixo `-32`. Todas as versões do Python desde 3.5 incluíram isso em suas compilações de 32 bits.

Novos recursos relacionados a dicas de tipo

Esta seção cobre as principais mudanças que afetam as dicas de tipo da **PEP 484** [https://peps.python.org/pep-0484/] e o módulo **`typing`**.

PEP 646: Genéricos variádicos

PEP 484 [https://peps.python.org/pep-0484/] introduzido anteriormente **`TypeVar`**, permitindo a criação de genéricos parametrizados com um único tipo. **PEP 646** [https://peps.python.org/pep-0646/] adiciona **`TypeVarTuple`**, permitindo a parametrização com um número *arbitrário* de tipos. Em outras palavras, uma **`TypeVarTuple`** é uma variável do tipo *variádico*, habilitando genéricos *variádicos*.

Isso permite uma ampla variedade de casos de uso. Em particular, ele permite que o tipo de estruturas do tipo array em bibliotecas de computação numérica, como NumPy e TensorFlow, sejam parametrizadas com o array *shape*. Os verificadores de tipo estáticos agora poderão capturar bugs relacionados a formas no código que usa essas bibliotecas.

Veja **PEP 646** [https://peps.python.org/pep-0646/] para mais detalhes.

(Contribuição de Matthew Rahtz em bpo-43224 [https://bugs.python.org/issue?@action=redirect&bpo=43224], com contribuições de Serhiy Storchaka e Jelle Zijlstra. PEP escrita por Mark Mendoza, Matthew Rahtz, Pradeep Kumar Srinivasan e Vincent Siles.)

PEP 655: Marcando itens `TypedDict` individuais como obrigatórios ou não obrigatórios

`Required` e **`NotRequired`** fornecem uma maneira direta de marcar se itens individuais em um **`TypedDict`** devem estar presentes. Anteriormente, isso só era possível usando herança.

Todos os campos ainda são obrigatórios por padrão, a menos que o parâmetro *total* seja definido como `False`, caso em que todos os campos ainda não são obrigatórios por padrão. Por exemplo, o seguinte especifica um **`TypedDict`** com uma chave obrigatória e uma chave não obrigatória:

```
class Filme(TypedDict):
   título: str
   ano: NotRequired[int]

m1: Filme = {"título": "Pantera Negra", "ano": 2018}  # OK
m2: Filme = {"título": "Star Wars"}  # OK (ano não é obrigatório)
m3: Filme = {"ano": 2022}  # ERRO (falta título, um campo obrigatório
```

A seguinte definição é equivalente:

```
class File(TypedDict, total=False):
   título: Required[str]
   ano: int
```

Veja **PEP 655** [https://peps.python.org/pep-0655/] para mais detalhes.

(Contribuição de David Foster e Jelle Zijlstra em bpo-47087 [https://bugs.python.org/issue?@action=redirect&bpo=47087]. PEP escrita por David Foster.)

PEP 673: Tipo `Self`

A nova anotação `Self` fornece uma maneira simples e intuitiva de anotar métodos que retornam uma instância de sua classe. Isso se comporta da mesma forma que a abordagem baseada em `TypeVar` **especificada na PEP 484** [https://peps.python.org/pep-0484/#annotating-instance-and-class-methods], mas é mais concisa e fácil de seguir.

Casos de uso comuns incluem construtores alternativos fornecidos como `classmethod`s e métodos `__enter__()` que retornam `self`:

```
class MinhaTrava:
    def __enter__(self) -> Self:
        self.trava()
        return self

    ...

class MeuInt:
    @classmethod
    def de_hex(cls, s: str) -> Self:
        return cls(int(s, 16))

    ...
```

`Self` também pode ser usado para anotar parâmetros de métodos ou atributos do mesmo tipo que sua classe de inclusão.

Veja **PEP 673** [https://peps.python.org/pep-0673/] para mais detalhes.

(Contribuição de James Hilton-Balfe em bpo-46534 [https://bugs.python.org/issue?@action=redirect&bpo=46534]. PEP escrita por Pradeep Kumar Srinivasan e James Hilton-Balfe.)

PEP 675: Tipo de string literal arbitrário

A nova anotação `LiteralString` pode ser usada para indicar que um parâmetro de função pode ser de qualquer tipo de string literal. Isso permite que uma função aceite tipos de string literais arbitrários, bem como strings criadas a partir de outras strings literais. Os verificadores de tipo podem então impor que funções confidenciais, como aquelas que executam instruções SQL ou comandos de console, sejam chamadas apenas com argumentos estáticos, fornecendo proteção contra ataques de injeção.

Por exemplo, uma função de consulta SQL pode ser anotada da seguinte forma:

```
def faz_consulta(sql: LiteralString) -> ...
    ...

def chamador(
    string_arbitrária: str,
    string_consulta: LiteralString,
    nome_tabela: LiteralString,
) -> None:
    faz_consulta("SELECT * FROM students")       # ok
    faz_consulta(string_consulta)                # ok
    faz_consulta("SELECT * FROM " + nome_tabela) # ok
    faz_consulta(string_arbitrária)              # erro no verificado
    faz_consulta(                                # erro no verificado
        f"SELECT * FROM students WHERE name = {string_arbitrária}"
    )
```

Veja **PEP 675** [https://peps.python.org/pep-0675/] para mais detalhes.

(Contribuição de Jelle Zijlstra em bpo-47088 [https://bugs.python.org/issue?@action=redirect&bpo=47088]. PEP escrita por Pradeep Kumar Srinivasan e Graham Bleaney.)

PEP 681: Transformações de classe de dados

`@~typing.dataclass_transform` pode ser usado para decorar uma classe, metaclasse ou uma função que é um decorador. A presença de `@dataclass_transform()` informa a um verificador de tipo estático que o objeto decorado executa uma “mágica” de tempo de execução que transforma uma classe, dando a ela comportamentos semelhantes a `dataclass`.

Por exemplo:

```
# O decorador cria_modelo está definido por uma biblioteca.
@typing.dataclass_transform()
def cria_modelo(cls: Type[T]) -> Type[T]:
    cls.__init__ = ...
    cls.__eq__ = ...
    cls.__ne__ = ...
    return cls

# O decorador cria_modelo agora pode ser usado para criar novas class
@cria_modelo
class ModeloCliente:
    id: int
    nome: str

c = ModeloCliente(id=327, nome="Eric Idle")
```

Veja **PEP 681** [https://peps.python.org/pep-0681/] para mais detalhes.

(Contribuição de Jelle Zijlstra em gh-91860 [https://github.com/python/cpython/issues/91860]. PEP escrita por Erik De Bonte e Eric Traut.)

PEP 563 pode não ser o futuro

PEP 563 [https://peps.python.org/pep-0563/] Avaliação adiada de anotações (a `from __future__ import annotations` instrução future) que foi originalmente planejada para lançamento no Python 3.10 foi colocada em espera indefinidamente. Veja esta mensagem do Conselho Diretor [https://mail.python.org/archives/list/python-dev@python.org/message/VIZEBX5EYMSYIJNDBF6DMUMZOCWHARSO/] para mais informações.

Outras mudanças na linguagem

- Expressões de desempacotamento com asterisco agora podem ser usadas em instruções `for`. (Veja bpo-46725 [https://bugs.python.org/issue?@action=redirect&bpo=46725] para mais detalhes.)
- Compreensões assíncronas agora são permitidas dentro de compreensões em funções assíncronas. As compreensões externas tornam-se implicitamente assíncronas neste caso. (Contribuição de Serhiy Storchaka em bpo-33346 [https://bugs.python.org/issue?@action=redirect&bpo=33346].)
- Uma exceção `TypeError` agora é levantada em vez de uma `AttributeError` em instruções `with` e `contextlib.ExitStack.enter_context()` para objetos que não oferecem suporte ao protocolo de gerenciador de contexto, e em instruções `async with` e `contextlib.AsyncExitStack.enter_async_context()` para objetos que não oferecem suporte ao protocolo de gerenciador de contexto assíncrono. (Contribuição de Serhiy Storchaka em bpo-12022 [https://bugs.python.org/issue?@action=redirect&bpo=12022] e bpo-44471 [https://bugs.python.org/issue?@action=redirect&bpo=44471].)
- Adicionado `object.__getstate__()`, que fornece a implementação padrão do método `__getstate__()`. Fazer `copy` e `pickle` de instâncias de subclasses de tipos embutidos `bytearray`, `set`, `frozenset`, `collections.OrderedDict`, `collections.deque`, `weakref.WeakSet` e `datetime.tzinfo` agora copia e serializa com pickle atributos de instância implementados como slots. Essa mudança tem um efeito colateral não intencional: ela atrapalha uma pequena minoria de projetos Python existentes que não esperavam que `object.__getstate__()` existisse. Consulte os comentários posteriores em gh-70766 [https://github.com/python/cpython/issues/70766] para discussões sobre quais soluções alternativas esse código pode precisar. (Contribuição de Serhiy Storchaka em bpo-26579 [https://bugs.python.org/issue?@action=redirect&bpo=26579].)

- Adicionada uma opção de linha de comando `-P` e uma variável de ambiente `PYTHONSAFEPATH`, que desativa o prefixo automático para `sys.path` do diretório do script ao executar um script ou o diretório atual ao usar `-c` e `-m`. Isso garante que apenas os módulos stdlib e instalados sejam escolhidos por `import`, e evita sombreamento involuntário ou mal-intencionado de módulos com aqueles em um diretório local (e normalmente gravável pelo usuário). (Contribuição de Victor Stinner em gh-57684 [https://github.com/python/cpython/issues/57684].)
- Uma opção `"z"` foi adicionada ao Minilinguagem de especificação de formato que força zero negativo para positivo após o arredondamento para a

precisão do formato. Veja **PEP 682** [https://peps.python.org/pep-0682/] para mais detalhes. (Contribuição de John Belmonte em gh-90153 [https://github.com/python/cpython/issues/90153].)

- Bytes não são mais aceitos em `sys.path`. O suporte quebrou em algum momento entre o Python 3.2 e o 3.6, sem que ninguém percebesse até o lançamento do Python 3.10.0. Além disso, trazer de volta o suporte seria problemático devido às interações entre `-b` e `sys.path_importer_cache` quando há uma mistura de chaves `str` e `bytes`. (Contribuição de Thomas Grainger em gh-91181 [https://github.com/python/cpython/issues/91181].)

Outras mudanças na implementação do CPython

- Os métodos especiais `__complex__()` para `complex` e `__bytes__()` para `bytes` são implementados para oferecer suporte aos protocolos `typing.SupportsComplex` e `typing.SupportsBytes`. (Contribuição de Mark Dickinson e Donghee Na em bpo-24234 [https://bugs.python.org/issue?@action=redirect&bpo=24234].)
- `siphash13` foi adicionado como um novo algoritmo de hash interno. Ele tem propriedades de segurança semelhantes ao `siphash24`, mas é um pouco mais rápido para entradas longas. `str`, `bytes` e alguns outros tipos agora o usam como o algoritmo padrão para `hash()`. Arquivos .pyc baseados em hash da **PEP 552** [https://peps.python.org/pep-0552/] agora usam `siphash13` também. (Contribuição de Inada Naoki em bpo-29410 [https://bugs.python.org/issue?@action=redirect&bpo=29410].)
- Quando uma exceção ativa é reativada por uma instrução `raise` sem parâmetros, o traceback anexado a essa exceção agora é sempre `sys.exc_info()[1].__traceback__`. Isso significa que as alterações feitas no traceback na cláusula `except` atual são refletidas na exceção levantada novamente. (Contribuição de Irit Katriel em bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711].)
- A representação do estado do interpretador de exceções tratadas (também conhecidas como `exc_info` ou `_PyErr_StackItem`) agora tem apenas o campo `exc_value`; `exc_type` e `exc_traceback` foram removidos, pois podem ser derivados de `exc_value`. (Contribuição de Irit Katriel em bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711].)
- Uma nova opção de linha de comando, `AppendPath`, foi adicionada para o instalador do Windows. Ela se comporta de maneira semelhante ao `PrependPath`, mas anexa os diretórios de instalação e scripts em vez de antecedê-los. (Contribuição de Bastian Neuburger em bpo-44934 [https://bugs.python.org/issue?@action=redirect&bpo=44934].)

- O campo `PyConfig.module_search_paths_set` agora deve ser definido como `1` para a inicialização usar `PyConfig.module_search_paths` para inicializar `sys.path`. Caso contrário, a inicialização irá recalcular o caminho e substituir quaisquer valores adicionados a `module_search_paths`.
- A saída da opção `--help` agora cabe em 50 linhas/80 colunas. Informações sobre as variáveis de ambiente do Python e a opção `-X` agora estão disponíveis usando os respectivos sinalizadores `--help-env` e `--help-xoptions`, e com o novo `--help-all`. (Contribuição de Éric Araujo em bpo-46142 [https://bugs.python.org/issue?@action=redirect&bpo=46142].)
- Converter entre `int` e `str` em bases diferentes de 2 (binário), 4, 8 (octal), 16 (hexadecimal) ou 32 como base 10 (decimal) agora levanta uma exceção `ValueError` se o número de dígitos em forma de string estiver acima de um limite para evitar possíveis ataques de negação de serviço devido à complexidade algorítmica. Esta é uma mitigação para **CVE 2020-10735** [https://www.cve.org/CVERecord?id=CVE-2020-10735]. Este limite pode ser configurado ou desabilitado por variável de ambiente, sinalizador de linha de comando ou APIs de `sys`. Veja a documentação de limitação de comprimento de conversão de string inteira. O limite padrão é de 4300 dígitos em forma de string.

Novos módulos

- `tomllib`: Para analisar TOML [https://toml.io/]. Veja **PEP 680** [https://peps.python.org/pep-0680/] para mais detalhes. (Contribuição de Taneli Hukkinen em bpo-40059 [https://bugs.python.org/issue?@action=redirect&bpo=40059].)
- `wsgiref.types`: Tipos específicos de **WSGI** [https://peps.python.org/pep-3333/] para verificação de tipo estático. (Contribuição de Sebastian Rittau em bpo-42012 [https://bugs.python.org/issue?@action=redirect&bpo=42012].)

Módulos melhorados

asyncio

- Adicionada a classe `TaskGroup`, um assíncrono gerenciador de contexto contendo um grupo de tarefas que irá esperar por todas elas ao sair. Para código novo, é recomendado usar `create_task()` e `gather()` diretamente. (Contribuição de Yury Selivanov e outros em gh-90908 [https://github.com/python/cpython/issues/90908].)

- Adicionado `timeout()`, um gerenciador de contexto assíncrono para definir um tempo limite em operações assíncronas. Para código novo, é recomendado usar `wait_for()` diretamente. (Contribuição de Andrew Svetlov em gh-90927 [https://github.com/python/cpython/issues/90927].)
- Adicionada a classe `Runner`, que expõe o maquinário usado por `run()`. (Contribuição de Andrew Svetlov em gh-91218 [https://github.com/python/cpython/issues/91218].)
- Adicionada a classe `Barrier` às primitivas de sincronização na biblioteca asyncio, e a exceção `BrokenBarrierError` relacionada. (Contribuição de Yves Duprat e Andrew Svetlov em gh-87518 [https://github.com/python/cpython/issues/87518].)
- Adicionado o argumento nomeado *all_errors* para `asyncio.loop.create_connection()` para que vários erros de conexão possam ser levantados como `ExceptionGroup`.
- Adicionado o método `asyncio.StreamWriter.start_tls()` para atualizar conexões baseadas em fluxo existentes para TLS. (Contribuição de Ian Good em bpo-34975 [https://bugs.python.org/issue?@action=redirect&bpo=34975].)
- Adicionadas funções de soquete de datagrama *raw* ao laço de eventos: `sock_sendto()`, `sock_recvfrom()` e `sock_recvfrom_into()`. Eles têm implementações em `SelectorEventLoop` e `ProactorEventLoop`. (Contribuição de Alex Grönholm em bpo-46805 [https://bugs.python.org/issue?@action=redirect&bpo=46805].)
- Adicionados os métodos `cancelling()` e `uncancel()` a `Task`. Estes são destinados principalmente para uso interno, notadamente por `TaskGroup`.

contextlib

- Adicionado gerenciador de contexto não paralelo seguro `chdir()` para alterar o diretório de trabalho atual e depois restaurá-lo na saída. Wrapper simples em torno de `chdir()`. (Contribuição de Filipe Laíns em bpo-25625 [https://bugs.python.org/issue?@action=redirect&bpo=25625])

dataclasses

- Alterar a verificação de mutabilidade padrão do campo, permitindo apenas padrões que são hasheáveis em vez de qualquer objeto que não seja uma instância de `dict`, `list` ou `set`. (Contribuição de Eric V. Smith em bpo-44674 [https://bugs.python.org/issue?@action=redirect&bpo=44674].)

datetime

- Adicione `datetime.UTC`, um apelido de conveniência para `datetime.timezone.utc`. (Contribuição de Kabir Kwatra em gh-91973 [https://github.com/python/cpython/issues/91973].)
- `datetime.date.fromisoformat()`, `datetime.time.fromisoformat()` e `datetime.datetime.fromisoformat()` agora podem ser usados para analisar a maioria dos formatos ISO 8601 (exceto aqueles que têm suporte a horas fracionárias e minutos). (Contribuição de Paul Ganssle em gh-80010 [https://github.com/python/cpython/issues/80010].)

enum

- Renomeado **`EnumMeta`** para `EnumType` (**`EnumMeta`** mantido como um apelido).
- Adicionada `StrEnum`, com membros que podem ser usados como (e devem ser) strings.
- Adicionada `ReprEnum`, que apenas modifica o `__repr__()` dos membros enquanto retorna seus valores literais (ao invés de nomes) para `__str__()` e `__format__()` (usado por `str()`, `format()` e f-strings).
- Alterado `Enum.__format__()` (o padrão para `format()`, `str.format()` e f-strings) para sempre produz o mesmo resultado que `Enum.__str__()`: para enums herdados de `ReprEnum` será o valor do membro; para todos os outros enums, será o enum e o nome do membro (por exemplo, `Color.RED`).
- Adicionado um novo parâmetro de classe *boundary* para enums `Flag` e `FlagBoundary` com suas opções, para controlar como lidar com valores de sinalizadores fora do intervalo.
- Adicionado o decorador de enum `verify()` e o enum `EnumCheck` com suas opções, para verificar classes de enum em relação a várias restrições específicas.
- Adicionados os decoradores `member()` e `nonmember()`, para garantir que o objeto decorado seja/não seja convertido em um membro enum.
- Adicionado o decorador `@~enum.property`, que funciona como `@property` exceto para enums. Use isso em vez de `types.DynamicClassAttribute()`.
- Adicionado o decorador enum `global_enum()`, que ajusta `__repr__()` e `__str__()` para mostrar valores como membros de seu módulo em vez da classe enum. Por exemplo, `'re.ASCII'` para o membro `ASCII` de `re.RegexFlag` em vez de `'RegexFlag.ASCII'`.
- Aprimorada `Flag` para ter suporte a `len()`, iteração e `in`/`not in` em seus membros. Por exemplo, o seguinte agora funciona: `len(AFlag(3)) == 2 and list(AFlag(3)) == (AFlag.ONE, AFlag.TWO)`
- Alterada `Enum` e `Flag` para que os membros sejam agora definidos antes de `__init_subclass__()` ser chamado; `dir()` agora inclui métodos, etc., de

tipos de dados mistos.

- Alterada `Flag` para considerar apenas valores primários (potência de dois) canônicos enquanto valores compostos (`3`, `6`, `10`, etc.) são considerados apelidos; bandeiras invertidas são coagidas ao seu equivalente positivo.

fcntl

- No FreeBSD, os sinalizadores **`F_DUP2FD`** e **`F_DUP2FD_CLOEXEC`** respectivamente são suportados, o primeiro é igual ao uso de `dup2` enquanto o último define o sinalizador `FD_CLOEXEC` adicionalmente.

fractions

- Suporte à inicialização no estilo **PEP 515** [https://peps.python.org/pep-0515/] de `Fraction` da string. (Contribuição de Sergey B Kirpichev em bpo-44258 [https://bugs.python.org/issue?@action=redirect&bpo=44258].)
- `Fraction` agora implementa um método `__int__`, para que uma verificação de `isinstance(some_fraction, typing.SupportsInt)` seja aprovada. (Contribuição de Mark Dickinson em bpo-44547 [https://bugs.python.org/issue?@action=redirect&bpo=44547].)

functools

- `@functools.singledispatch` agora oferece suporte a `types.UnionType` e `typing.Union` como anotações para o argumento de dispatch.:

```
>>> from functools import singledispatch
>>> @singledispatch
... def fun(arg, verboso=False):
...     if verboso:
...         print("Deixe-me apenas dizer,", end=" ")
...     print(arg)
...
>>> @fun.register
... def _(arg: int | float, verboso=False):
...     if verboso:
...         print("Força nos números, não é?", end=" ")
...     print(arg)
...
>>> from typing import Union
>>> @fun.register
... def _(arg: Union[list, set], verbose=False):
```

```
...     if verboso:
...         print("Enumere isso:")
...     for i, elem in enumerate(arg):
...         print(i, elem)
...
```

(Contribuição de Yurii Karabas na bpo-46014 [https://bugs.python.org/issue?@action=redirect&bpo=46014].)

gzip

- A função `gzip.compress()` agora é mais rápida quando usada com o argumento **mtime=0**, pois delega a compactação inteiramente para uma única operação `zlib.compress()`. Há um efeito colateral dessa mudança: o cabeçalho do arquivo gzip contém um byte de "OS" em seu cabeçalho. Tradicionalmente, isso sempre foi definido com um valor de 255 representando "desconhecido" pelo módulo `gzip`. Agora, ao usar `compress()` com **mtime=0**, ele pode ser definido com um valor diferente pela biblioteca zlib C subjacente à qual o Python estava vinculado. (Veja gh-112346 [https://github.com/python/cpython/issues/112346] para detalhes sobre o efeito colateral.)

hashlib

- `hashlib.blake2b()` e `hashlib.blake2s()` agora preferem libb2 [https://www.blake2.net/] em vez da cópia do Python. (Contribuição de Christian Heimes em bpo-47095 [https://bugs.python.org/issue?@action=redirect&bpo=47095].)
- O módulo interno `_sha3` com algoritmos SHA3 e SHAKE agora usa *tiny_sha3* em vez do *Keccak Code Package* para reduzir o código e o tamanho binário. O módulo `hashlib` prefere implementações otimizadas de SHA3 e SHAKE do OpenSSL. A alteração afeta apenas instalações sem suporte a OpenSSL. (Contribuição de Christian Heimes em bpo-47098 [https://bugs.python.org/issue?@action=redirect&bpo=47098].)
- Adiciona `hashlib.file_digest()`, uma função auxiliar para hash eficiente de objetos arquivo ou similar. (Contribuição de Christian Heimes em gh-89313 [https://github.com/python/cpython/issues/89313].)

IDLE e idlelib

- Aplica realce de sintaxe em arquivos `.pyi`. (Contribuição de Alex Waygood e Terry Jan Reedy em bpo-45447 [https://bugs.python.org/issue?@action=redirect&bpo=45447].)

- Inclui prompts ao salvar o console com entradas e saídas. (Contribuição de Terry Jan Reedy em gh-95191 [https://github.com/python/cpython/issues/95191].)

inspect

- Adiciona `getmembers_static()` para retornar todos os membros sem acionar a pesquisa dinâmica por meio do protocolo descritor. (Contribuição de Weipeng Hong em bpo-30533 [https://bugs.python.org/issue?@action=redirect&bpo=30533].)

- Adiciona `ismethodwrapper()` para verificar se o tipo de um objeto é um `MethodWrapperType`. (Contribuição de Hakan Çelik em bpo-29418 [https://bugs.python.org/issue?@action=redirect&bpo=29418].)

- Altera as funções relacionadas a quadros no módulo `inspect` para retornar novas instâncias de classe `FrameInfo` e `Traceback` (compatíveis com as anteriores interfaces do tipo tupla nomeada) que inclui as informações de posição estendidas **PEP 657** [https://peps.python.org/pep-0657/] (número da linha final, coluna e coluna final). As funções afetadas são:

  - `inspect.getframeinfo()`
  - `inspect.getouterframes()`
  - `inspect.getinnerframes()`,
  - `inspect.stack()`
  - `inspect.trace()`

  (Contribuição por Pablo Galindo em gh-88116 [https://github.com/python/cpython/issues/88116].)

locale

- Adiciona `locale.getencoding()` para obter a codificação local atual. É semelhante a `locale.getpreferredencoding(False)`, mas ignora o Modo UTF-8 do Python.

logging

- Adicionada `getLevelNamesMapping()` para retornar um mapeamento de nomes de nível de registro (por exemplo, `'CRITICAL'`) para os valores de seus Níveis de Logging correspondentes (por exemplo, `50`, por padrão).

(Contribuição de Andrei Kulakovin em gh-88024 [https://github.com/python/cpython/issues/88024].)

- Adicionado um método `createSocket()` a `SysLogHandler`, para corresponder a `SocketHandler.createSocket()` . Ele é chamado automaticamente durante a inicialização do manipulador e ao emitir um evento, se não houver nenhum soquete ativo. (Contribuição de Kirill Pinchuk em gh-88457 [https://github.com/python/cpython/issues/88457].)

math

- Adiciona `math.exp2()`: retorna 2 elevado à potência de x. (Contribuição de Gideon Mitchell em bpo-45917 [https://bugs.python.org/issue?@action=redirect&bpo=45917].)
- Adiciona `math.cbrt()`: retorna a raiz cúbica de x. (Contribuição de Ajith Ramachandran em bpo-44357 [https://bugs.python.org/issue?@action=redirect&bpo=44357].)
- O comportamento de dois casos excepcionais `math.pow()` foi alterado, para consistência com a especificação IEEE 754. As operações `math.pow(0.0, -math.inf)` e `math.pow(-0.0, -math.inf)` agora retornam `inf`. Anteriormente eles levantavam `ValueError`. (Contribuição de Mark Dickinson em bpo-44339 [https://bugs.python.org/issue?@action=redirect&bpo=44339].)
- O valor `math.nan` agora está sempre disponível. (Contribuição de Victor Stinner em bpo-46917 [https://bugs.python.org/issue?@action=redirect&bpo=46917].)

operator

- Uma nova função `operator.call` foi adicionada, tal que `operator.call(obj, *args, **kwargs) == obj(*args, **kwargs)` (Contribuição de Antony Lee em bpo-44019 [https://bugs.python.org/issue?@action=redirect&bpo=44019].)

os

- No Windows, `os.urandom()` agora usa `BCryptGenRandom()`, em vez de `CryptGenRandom()` que foi descontinuado. (Contribuição de Dong-hee Na em bpo-44611 [https://bugs.python.org/issue?@action=redirect&bpo=44611].)

pathlib

- `glob()` e `rglob()` retornam apenas diretórios se *pattern* terminar com um separador de componentes de nome de caminho: `sep` ou `altsep`. (Contribuição de Eisuke Kawasima em bpo-22276 [https://bugs.python.org/issue?@action=redirect&bpo=22276] e bpo-33392 [https://bugs.python.org/issue?@action=redirect&bpo=33392].)

re

- Agrupamento atômico (`(?>...)`) e quantificadores possessivos (`*+`, `++`, `?+`, `{m,n}+`) agora são suportados em expressões regulares. (Contribuição de Jeffrey C. Jacobs e Serhiy Storchaka em bpo-433030 [https://bugs.python.org/issue?@action=redirect&bpo=433030].)

shutil

- Adicione o parâmetro opcional *dir_fd* em `shutil.rmtree()`. (Contribuição de Serhiy Storchaka em bpo-46245 [https://bugs.python.org/issue?@action=redirect&bpo=46245].)

socket

- Adiciona suporte ao soquete CAN para NetBSD. (Contribuição de Thomas Klausner em bpo-30512 [https://bugs.python.org/issue?@action=redirect&bpo=30512].)
- `create_connection()` tem a opção de levantar, em caso de falha na conexão, um `ExceptionGroup` contendo todos os erros ao invés de somente levantar o último erro. (Contribuição de Irit Katriel em bpo-29980 [https://bugs.python.org/issue?@action=redirect&bpo=29980].)

sqlite3

- Agora você pode desabilitar o autorizador passando `None` para `set_authorizer()`. (Contribuição de Erlend E. Aasland em bpo-44491 [https://bugs.python.org/issue?@action=redirect&bpo=44491].)
- O nome do agrupamento `create_collation()` agora pode conter qualquer caractere Unicode. Nomes de agrupamento com caracteres inválidos agora levantar `UnicodeEncodeError` em vez de `sqlite3.ProgrammingError`. (Contribuição de Erlend E. Aasland em bpo-44688 [https://bugs.python.org/issue?@action=redirect&bpo=44688].)
- As exceções `sqlite3` agora incluem o código de erro estendido SQLite como `sqlite_errorcode` e o nome do erro SQLite como `sqlite_errorname`.

(Contribuição de Aviv Palivoda, Daniel Shahaf e Erlend E. Aasland em bpo-16379 [https://bugs.python.org/issue?@action=redirect&bpo=16379] e bpo-24139 [https://bugs.python.org/issue?@action=redirect&bpo=24139].)

- Adiciona `setlimit()` e `getlimit()` a `sqlite3.Connection` para definir e obter limites SQLite por base de conexão. (Contribuição de Erlend E. Aasland em bpo-45243 [https://bugs.python.org/issue?@action=redirect&bpo=45243].)
- `sqlite3` agora define `sqlite3.threadsafety` com base no modo de threading padrão com o qual a biblioteca SQLite subjacente foi compilada. (Contribuição de Erlend E. Aasland em bpo-45613 [https://bugs.python.org/issue?@action=redirect&bpo=45613].)
- O retorno de chamadas C `sqlite3` agora usam exceções que não podem ser levantadas se os rastreamentos de retorno de chamada estiverem habilitados. Os usuários agora podem registrar um `unraisable_hook_handler` para melhorar sua experiência de depuração. (Contribuição de Erlend E. Aasland em bpo-45828 [https://bugs.python.org/issue?@action=redirect&bpo=45828].)
- A busca por rollback não levanta mais `InterfaceError`. Em vez disso, deixamos para a biblioteca SQLite lidar com esses casos. (Contribuição de Erlend E. Aasland em bpo-44092 [https://bugs.python.org/issue?@action=redirect&bpo=44092].)
- Adiciona `serialize()` e `deserialize()` a `sqlite3.Connection` para serializar e desserializar bancos de dados. (Contribuição de Erlend E. Aasland em bpo-41930 [https://bugs.python.org/issue?@action=redirect&bpo=41930].)
- Adiciona `create_window_function()` a `sqlite3.Connection` para criar funções de janela agregadas. (Contribuição de Erlend E. Aasland em bpo-34916 [https://bugs.python.org/issue?@action=redirect&bpo=34916].)
- Adiciona `blobopen()` a `sqlite3.Connection`. `sqlite3.Blob` permite operações de E/S incrementais em blobs. (Contribuição de Aviv Palivoda e Erlend E. Aasland em bpo-24905 [https://bugs.python.org/issue?@action=redirect&bpo=24905].)

string

- Adiciona `get_identifiers()` e `is_valid()` a `string.Template`, que respectivamente retornam todos os marcadores de posição válidos e se quaisquer marcadores de posição inválidos estão presentes. (Contribuição de Ben Kehoe em gh-90465 [https://github.com/python/cpython/issues/90465].)

sys

- `sys.exc_info()` agora deriva os campos `type` e `traceback` do `value` (a instância de exceção) de forma que quanto uma exceção é modificada enquanto está sendo tratada, são refletidos nos resultados de chamadas subsequentes para **`exc_info()`**. (Contribuição de Irit Katriel em bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711].)
- Adiciona `sys.exception()` que retorna a instância de exceção ativa (equivalente a `sys.exc_info()[1]`). (Contribuição de Irit Katriel em bpo-46328 [https://bugs.python.org/issue?@action=redirect&bpo=46328].)
- Adiciona o sinalizador `sys.flags.safe_path`. (Contribuição de Victor Stinner em gh-57684 [https://github.com/python/cpython/issues/57684].)

sysconfig

- Três novos esquemas de instalação (*posix_venv*, *nt_venv* e *venv*) foram adicionados e são usados quando o Python cria novos ambientes virtuais ou quando está sendo executado a partir de um ambiente virtual. Os dois primeiros esquemas (*posix_venv* e *nt_venv*) são específicos do SO para não Windows e Windows, o *venv* é essencialmente um apelido para um deles de acordo com o SO em que o Python é executado. Isso é útil para distribuidores downstream que modificam `sysconfig.get_preferred_scheme()`. O código de terceiros que cria novos ambientes virtuais deve usar o novo esquema de instalação *venv* para determinar os caminhos, assim como `venv`. (Contribuição de Miro Hrončok em bpo-45413 [https://bugs.python.org/issue?@action=redirect&bpo=45413].)

tempfile

- Objetos `SpooledTemporaryFile` agora implementam totalmente os métodos de `io.BufferedIOBase` ou `io.TextIOBase` (dependendo do modo de arquivo). Isso permite que eles funcionem corretamente com APIs que esperam objetos arquivo ou similares, como módulos de compactação. (Contribuição de Carey Metcalfe em gh-70363 [https://github.com/python/cpython/issues/70363].)

threading

- No Unix, se a função `sem_clockwait()` estiver disponível na biblioteca C (glibc 2.30 e mais recente), o método `threading.Lock.acquire()` agora usa o relógio monotônico (`time.CLOCK_MONOTONIC`) para o tempo limite, em vez de usar o relógio do sistema (`time.CLOCK_REALTIME`), para não ser afetado

pelas alterações do relógio do sistema. (Contribuição de Victor Stinner em bpo-41710 [https://bugs.python.org/issue?@action=redirect&bpo=41710].)

time

- No Unix, `time.sleep()` agora usa a função `clock_nanosleep()` ou `nanosleep()`, se disponível, que tem uma resolução de 1 nanossegundo (10^{-9} segundos), em vez de usar `select()` que tem uma resolução de 1 microssegundo (10^{-6} segundos). (Contribuição de Benjamin Szőke e Victor Stinner em bpo-21302 [https://bugs.python.org/issue?@action=redirect&bpo=21302].)
- No Windows 8.1 e mais recente, `time.sleep()` agora usa um temporizador aguardável baseado em temporizadores de alta resolução [https://docs.microsoft.com/en-us/windows-hardware/drivers/kernel/high-resolution-timers] que tem uma resolução de 100 nanossegundos (10^{-7} segundos). Anteriormente, tinha uma resolução de 1 milissegundo (10^{-3} segundos). (Contribuição de Benjamin Szőke, Donghee Na, Eryk Sun e Victor Stinner em bpo-21302 [https://bugs.python.org/issue?@action=redirect&bpo=21302] e bpo-45429 [https://bugs.python.org/issue?@action=redirect&bpo=45429].)

tkinter

- Adicionado o método `info_patchlevel()` que retorna a versão exata da biblioteca Tcl como uma tupla nomeada semelhante a `sys.version_info`. (Contribuição de Serhiy Storchaka em gh-91827 [https://github.com/python/cpython/issues/91827].)

traceback

- Adiciona `traceback.StackSummary.format_frame_summary()` para permitir que os usuários substituam quais quadros aparecem no traceback e como eles são formatados. (Contribuição de Ammar Askar em bpo-44569 [https://bugs.python.org/issue?@action=redirect&bpo=44569].)
- Adiciona `traceback.TracebackException.print()`, que imprime a instância formatada `TracebackException` em um arquivo. (Contribuição de Irit Katriel em bpo-33809 [https://bugs.python.org/issue?@action=redirect&bpo=33809].)

typing

Para alterações principais, veja Novos recursos relacionados a dicas de tipo.

- Adiciona **`typing.assert_never()`** e **`typing.Never`**. **`typing.assert_never()`** é útil para pedir a um verificador de tipo para confirmar que uma linha de código não está acessível. Em tempo de execução, ele levanta uma exceção **`AssertionError`**. (Contribuição de Jelle Zijlstra em gh-90633 [https://github.com/python/cpython/issues/90633].)
- Adiciona **`typing.reveal_type()`**. Isso é útil para perguntar a um verificador de tipo qual tipo ele inferiu para uma determinada expressão. Em tempo de execução imprime o tipo do valor recebido. (Contribuição de Jelle Zijlstra em gh-90572 [https://github.com/python/cpython/issues/90572].)
- Adicione **`typing.assert_type()`**. Isso é útil para solicitar a um verificador de tipos que confirme se o tipo que ele inferiu para uma determinada expressão corresponde ao tipo fornecido. Em tempo de execução ele simplesmente retorna o valor recebido. (Contribuição de Jelle Zijlstra em gh-90638 [https://github.com/python/cpython/issues/90638].)
- Os tipos **`typing.TypedDict`** agora podem ser genéricos. (Contribuição de Samodya Abeysiriwardane em gh-89026 [https://github.com/python/cpython/issues/89026].)
- Os tipos **`NamedTuple`** agora podem ser genéricos. (Contribuição de Serhiy Storchaka em bpo-43923 [https://bugs.python.org/issue?@action=redirect&bpo=43923].)
- Permite criar subclasse de **`typing.Any`**. Isso é útil para evitar erros de verificador de tipo relacionados a classes altamente dinâmicas, como mocks. (Contribuição de Shantanu Jain em gh-91154 [https://github.com/python/cpython/issues/91154].)
- O decorador **`@typing.final`** agora define o `__final__` atribuído no objeto decorado. (Contribuição de Jelle Zijlstra em gh-90500 [https://github.com/python/cpython/issues/90500].)
- A função **`typing.get_overloads()`** pode ser usada para introspecção das sobrecargas de uma função. **`typing.clear_overloads()`** pode ser usado para limpar todas as sobrecargas registradas de uma função. (Contribuição de Jelle Zijlstra em gh-89263 [https://github.com/python/cpython/issues/89263].)
- O método **`__init__()`** das subclasses **`Protocol`** agora é preservado. (Contribuição de Adrian Garcia Badarasco em gh-88970 [https://github.com/python/cpython/issues/88970].)
- A representação de tipos de tupla vazias (`Tuple[()]`) está simplificada. Isso afeta a introspecção, por exemplo. `get_args(Tuple[()])` agora é avaliado como `()` em vez de `((),)`. (Contribuição de Serhiy Storchaka em gh-91137 [https://github.com/python/cpython/issues/91137].)
- Afrouxa os requisitos de tempo de execução para anotações de tipo removendo a verificação chamável na função privada `typing._type_check`.

(Contribuição de Gregory Beauregard em gh-90802 [https://github.com/python/cpython/issues/90802].)

- `typing.get_type_hints()` agora tem suporte à avaliação de strings como referências diretas em apelidos genéricos da PEP 585. (Contribuição de Niklas Rosenstein em gh-85542 [https://github.com/python/cpython/issues/85542].)
- `typing.get_type_hints()` não adiciona mais `Optional` aos parâmetros com `None` como padrão. (Contribuição de Nikita Sobolev em gh-90353 [https://github.com/python/cpython/issues/90353].)
- `typing.get_type_hints()` agora tem suporte à avaliação de anotações `ClassVar` de strings simples. (Contribuição de Gregory Beauregard em gh-90711 [https://github.com/python/cpython/issues/90711].)
- `@typing.no_type_check` não modifica mais classes e funções externas. Agora também marca corretamente os métodos de classe para não serem verificados quanto ao tipo. (Contribuição de Nikita Sobolev em gh-90729 [https://github.com/python/cpython/issues/90729].)

unicodedata

- O banco de dados Unicode foi atualizado para a versão 14.0.0. (Contribuição de Benjamin Peterson em bpo-45190 [https://bugs.python.org/issue?@action=redirect&bpo=45190]).

unittest

- Adicionados os métodos `enterContext()` e `enterClassContext()` da classe `TestCase`, método `enterAsyncContext()` da classe `IsolatedAsyncioTestCase` e a função `unittest.enterModuleContext()`. (Contribuição de Serhiy Storchaka em bpo-45046 [https://bugs.python.org/issue?@action=redirect&bpo=45046].)

venv

- Quando novos ambientes virtuais Python são criados, o esquema de instalação do sysconfig do *venv* é usado para determinar os caminhos dentro do ambiente. Quando o Python é executado em um ambiente virtual, o mesmo esquema de instalação é o padrão. Isso significa que os distribuidores downstream podem alterar o esquema de instalação padrão do sysconfig sem alterar o comportamento dos ambientes virtuais. O código de terceiros que também cria novos ambientes virtuais deve fazer o mesmo. (Contribuição de

Miro Hrončok em bpo-45413 [https://bugs.python.org/issue?@action=redirect&bpo=45413].)

warnings

- `warnings.catch_warnings()` agora aceita argumentos para `warnings.simplefilter()`, fornecendo uma maneira mais concisa de ignorar localmente avisos ou convertê-los em erros. (Contribuição de Zac Hatfield-Dodds em bpo-47074 [https://bugs.python.org/issue?@action=redirect&bpo=47074].)

zipfile

- Adicionado suporte para especificar a codificação de nome de membro para leitura de metadados em um diretório `ZipFile` e cabeçalhos de arquivo. (Contribuição de Stephen J. Turnbull e Serhiy Storchaka em bpo-28080 [https://bugs.python.org/issue?@action=redirect&bpo=28080].)
- Adicionado `ZipFile.mkdir()` para criar novos diretórios dentro de arquivos ZIP. (Contribuição de Sam Ezeh em gh-49083 [https://github.com/python/cpython/issues/49083].)
- Adicionado `stem`, `suffix` e `suffixes` a `zipfile.Path`. (Contribuição de Miguel Brito em gh-88261 [https://github.com/python/cpython/issues/88261].)

Otimizações

Esta seção cobre otimizações específicas independentes do projeto CPython mais rápido, que é abordado em sua própria seção.

- O compilador agora otimiza a formatação simples de % no estilo printf em literais de string contendo apenas os códigos de formato `%s`, `%r` e `%a` e a torna tão rápida quanto uma expressão f-string correspondente. (Contribuição de Serhiy Storchaka em bpo-28307 [https://bugs.python.org/issue?@action=redirect&bpo=28307].)
- A divisão inteira (`//`) é melhor ajustada para otimização por compiladores. Agora é cerca de 20% mais rápido em x86-64 ao dividir um `int` por um valor menor que `2**30`. (Contribuição de Gregory P. Smith e Tim Peters em gh-90564 [https://github.com/python/cpython/issues/90564].)
- `sum()` agora é quase 30% mais rápida para inteiros menores que `2**30`. (Contribuição de Stefan Behnel em gh-68264 [https://github.com/python/cpython/issues/68264].)

- O redimensionamento de listas é simplificado para o caso comum, acelerando `list.append()` em ≈15% e compreensões de lista em até 20-30% (Contribuição de Dennis Sweeney em gh-91165 [https://github.com/python/cpython/issues/91165].)
- Dicionários não armazenam valores de hash quando todas as chaves são objetos Unicode, diminuindo o tamanho de `dict`. Por exemplo, `sys.getsizeof(dict.fromkeys("abcdefg"))` foi reduzido de 352 bytes para 272 bytes (23% menor) em plataformas de 64 bits. (Contribuição de Inada Naoki em bpo-46845 [https://bugs.python.org/issue?@action=redirect&bpo=46845].)
- Usar `asyncio.DatagramProtocol` agora é muito mais rápido ao transferir arquivos grandes por UDP, com velocidades mais de 100 vezes maiores para um arquivo de ≈60 MiB. (Contribuição de msoxzw em gh-91487 [https://github.com/python/cpython/issues/91487].)
- As funções `comb()` e `perm()` de `math` são agora ≈10 vezes mais rápidas para argumentos grandes (com uma aceleração maior para *k* maiores). (Contribuição de Serhiy Storchaka em bpo-37295 [https://bugs.python.org/issue?@action=redirect&bpo=37295].)
- As funções `mean()`, `variance()` e `stdev()` do `statistics` agora consomem iteradores em uma passagem em vez de convertê-los em uma `list` primeiro. Isso é duas vezes mais rápido e pode economizar memória de forma substancial. (Contribuição de Raymond Hettinger em gh-90415 [https://github.com/python/cpython/issues/90415].)
- `unicodedata.normalize()` agora normaliza strings de puro ASCII em tempo constante. (Contribuição de Dong-hee Na em bpo-44987 [https://bugs.python.org/issue?@action=redirect&bpo=44987].)

CPython mais rápido

O CPython 3.11 é em média 25% mais rápido [https://github.com/faster-cpython/ideas#published-results] que o CPython 3.10 medido com o conjunto de ferramentas de benchmarks pyperformance [https://github.com/python/pyperformance], quando compilado com GCC no Ubuntu Linux. Dependendo da sua carga de trabalho, a aceleração geral pode ser de 10 a 60%.

Este projeto se concentra em duas áreas principais em Python: Inicialização mais rápida e Tempo de execução mais rápido. Otimizações que não são abordadas neste projeto estão listadas em Otimizações.

Inicialização mais rápida

Importações congeladas/objetos de código estático

O Python armazena o bytecode no diretório __pycache__ para acelerar o carregamento do módulo.

Anteriormente na versão 3.10, a execução do módulo Python era assim:

```
Read __pycache__ -> Unmarshal -> Heap allocated code object -> Evalua
```

No Python 3.11, os principais módulos essenciais para a inicialização do Python são “congelados”. Isso significa que seus Objetos código (e bytecode) são alocados estaticamente pelo interpretador. Isso reduz as etapas no processo de execução do módulo para isso:

```
Statically allocated code object -> Evaluate
```

A inicialização do interpretador agora é 10-15% mais rápida no Python 3.11. Isso tem um grande impacto para programas de execução curta usando Python.

(Contribuição de Eric Snow, Guido van Rossum e Kumar Aditya em vários issues.)

Tempo de execução mais rápido

Quadros Python menos custosos e mais preguiçosos

Os quadros do Python, contendo informações de execução, são criados sempre que o Python chama uma função do Python. A seguir estão as novas otimizações de quadro:

- Simplificado o processo de criação do quadro.
- Alocação de memória evitada reutilizando generosamente o espaço do quadro na pilha C.
- Simplificada a estrutura interna do quadro para conter apenas as informações essenciais. Os quadros anteriormente continham informações extras de depuração e gerenciamento de memória.

Os antigos objetos quadros agora são criados apenas quando solicitados por depuradores ou por funções de introspecção do Python, como `sys._getframe()` e `inspect.currentframe()`. Para a maioria dos códigos de usuário, nenhum objeto

quadro é criado. Como resultado, quase todas as chamadas de funções do Python aceleraram significativamente. Medimos uma aceleração de 3 a 7% no desempenho do py.

(Contribuição de Mark Shannon em bpo-44590 [https://bugs.python.org/issue?@action=redirect&bpo=44590].)

Chamadas de função Python em linha

Durante a chamada de uma função Python, o Python chamará uma função C de avaliação para interpretar o código dessa função. Isso efetivamente limita a recursão Python pura ao que é seguro para a pilha C.

No 3.11, quando o CPython detecta o código Python chamando outra função Python, ele configura um novo quadro e "pula" para o novo código dentro do novo quadro. Isso evita chamar a função de interpretação C completamente.

A maioria das chamadas de função do Python agora não consome espaço de pilha C, acelerando-as. Em funções recursivas simples como fibonacci ou fatorial, observamos um aumento de velocidade de 1,7x. Isso também significa que as funções recursivas podem recursar significativamente mais profundamente (se o usuário aumentar o limite de recursão com `sys.setrecursionlimit()`). Medimos uma melhoria de 1 a 3% no desempenho do py.

(Contribuição de Pablo Galindo e Mark Shannon em bpo-45256 [https://bugs.python.org/issue?@action=redirect&bpo=45256].)

PEP 659: Interpretador adaptativo especializado

PEP 659 [https://peps.python.org/pep-0659/] é uma das partes principais do projeto Faster CPython. A ideia geral é que, embora o Python seja uma linguagem dinâmica, a maior parte do código possui regiões onde objetos e tipos raramente mudam. Este conceito é conhecido como *estabilidade de tipo*.

Em tempo de execução, o Python tentará procurar padrões comuns e estabilidade de tipo no código em execução. O Python substituirá a operação atual por uma mais especializada. Essa operação especializada usa caminhos rápidos disponíveis apenas para esses casos/tipos de uso, que geralmente superam suas contrapartes genéricas. Isso também traz outro conceito chamado *cache inline*, onde o Python armazena em cache os resultados de operações caras diretamente no bytecode.

O especializador também combinará certos pares de instruções comuns em uma superinstrução, reduzindo a sobrecarga durante a execução.

O Python só se especializará quando vir um código “quente” (executado várias vezes). Isso evita que o Python perca tempo com código de execução única. O Python também pode se especializar quando o código é muito dinâmico ou quando o uso muda. A especialização é tentada periodicamente e as tentativas de especialização não são muito caras, permitindo que a especialização se adapte às novas circunstâncias.

(PEP escrita por Mark Shannon, com ideias inspiradas por Stefan Brunthaler. Veja **PEP 659** [https://peps.python.org/pep-0659/] para mais informações. Implementação por Mark Shannon e Brandt Bucher, com ajuda adicional de Irit Katriel e Dennis Sweeney.)

Operação	Forma	Especialização	Aceleração da operação	Contribuidor(es)

Operações binárias	`x + x`
	`x - x`
	`x * x`
		Adicionar, multiplicar e subtrair binários para tipos comuns como `int`, `float` e `str` usam caminhos rápidos personalizados para seus tipos subjacentes.	10%	Mark Shannon, Donghee Na, Brandt Bucher, Dennis Sweeney

Operação	Forma	Especialização	Aceleração da operação	Contribuidor(es)
Subscrição	`a[i]`	Subscrever tipos contêineres como `list`, `tuple` e `dict` indexam diretamente as estruturas de dados subjacentes. Subscrever `__getitem__()` personalizado também é inserido em linha de forma similar a Chamadas de função Python em linha.	10-25%	Irit Katriel, Mark Shannon
Armazenar com subscrição	`a[i] = z`	Similar à especialização de subscrição acima.	10-25%	Dennis Sweeney

Operação	Forma	Especialização	Aceleração da operação	Contribuidor(es)

Chamadas	`f(arg)`
	
	`C(arg)`
		Chamadas para funções e tipos (C) embutidos comuns como `len()` e `str` chamam diretamente sua versão C subjacente. Isso evita passar pela convenção de chamada interna.	20%	Mark Shannon, Ken Jin

Carregar variável global	`print`
	
	`len`
		O índice do objeto no espaço de nomes globais/embutidas é armazenado em cache. Carregar globais e embutidas requer zero pesquisas de espaço de nomes.	[[1]]	Mark Shannon

Operação	Forma	Especialização	Aceleração da operação	Contribuidor(es)
Carregar atributo	`o.attr`	Semelhante ao carregamento de variáveis globais. O índice do atributo dentro do espaço de nomes da classe/objeto é armazenado em cache. Na maioria dos casos, o carregamento de atributos exigirá zero pesquisas de espaço de nomes.	[[2]]	Mark Shannon
Carregar métodos para chamada	`o.meth()`	O endereço real do método é armazenado em cache. O carregamento de método agora não tem pesquisas de espaço de nomes – mesmo para classes com longas cadeias de herança.	10-20%	Ken Jin, Mark Shannon
Armazenar atributo	`o.attr = z`	Semelhante à otimização de carregamento de atributos.	2% no pyperformance	Mark Shannon

Operação	Forma	Especialização	Aceleração da operação	Contribuidor(es)
Desempacotar sequência	`*seq`	Especializado para contêineres comuns como `list` e `tuple`. Evita convenção de chamada interna.	8%	Brandt Bucher

[1]

Uma otimização semelhante já existia desde o Python 3.8. O 3.11 é especializado em mais formas e reduz algumas despesas gerais.

[2]

Uma otimização semelhante já existia desde o Python 3.10. 3.11 é especializado em mais formas. Além disso, todos os carregamentos de atributos devem ser acelerados por bpo-45947 [https://bugs.python.org/issue?@action=redirect&bpo=45947].

Diversos

- Os objetos agora requerem menos memória devido aos espaços de nomes de objetos criados lentamente. Seus dicionários de espaços de nomes agora também compartilham chaves mais livremente. (Contribuição de Mark Shannon em bpo-45340 [https://bugs.python.org/issue?@action=redirect&bpo=45340] e bpo-40116 [https://bugs.python.org/issue?@action=redirect&bpo=40116].)
- Exceções de “custo zero” estão implementadas, eliminando o custo das instruções `try` quando nenhuma exceção é gerada. (Contribuição de Mark Shannon em bpo-40222 [https://bugs.python.org/issue?@action=redirect&bpo=40222].)
- Uma representação mais concisa de exceções no interpretador reduziu o tempo necessário para capturar uma exceção em cerca de 10%. (Contribuição de Irit Katriel em bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711].)
- O mecanismo de correspondência de expressão regular do `re` foi parcialmente refatorado e agora usa *gotos* computados (ou “código encadeado”) em plataformas suportadas. Como resultado, o Python 3.11 executa os benchmarks de expressão regular do pyperformance

[https://pyperformance.readthedocs.io/benchmarks.html#regex-dna] até 10% mais rápido que o Python 3.10. (Contribuição de Brandt Bucher em gh-91404 [https://github.com/python/cpython/issues/91404].)

FAQ

Como devo escrever meu código para utilizar esses aceleradores?

Escrever código pythônico que segue as melhores práticas comuns; você não precisa alterar seu código. O projeto Faster CPython otimiza para padrões de código comuns que observamos.

O CPython 3.11 usará mais memória?

Talvez não; não esperamos que o uso de memória exceda 20% a mais que 3.10. Isso é compensado por otimizações de memória para objetos quadro e dicionários de objetos, conforme mencionado acima.

Não vejo nenhuma aceleração em minha carga de trabalho. Por que?

Certos códigos não terão benefícios perceptíveis. Se seu código passa a maior parte do tempo em operações de E/S ou já faz a maior parte de sua computação em uma biblioteca de extensão C como NumPy, não haverá acelerações significativas. Atualmente, esse projeto é o que mais beneficia as cargas de trabalho Python puro.

Além disso, os valores de desempenho py são uma média geométrica. Mesmo dentro dos benchmarks de pyperformance, alguns benchmarks desaceleraram um pouco, enquanto outros aceleraram quase 2x!

Existe um compilador JIT?

Não. Ainda estamos explorando outras otimizações.

Sobre

O Faster CPython explora otimizações para o CPython. A equipe principal é financiada pela Microsoft para trabalhar nisso em tempo integral. Pablo Galindo Salgado também é financiado pela Bloomberg LP para trabalhar no projeto em tempo parcial. Finalmente, muitos colaboradores são voluntários da comunidade.

Alterações de bytecode do CPython

O bytecode agora contém entradas de cache inline, que assumem a forma das instruções recém-adicionadas `CACHE`. Muitos códigos de operações, ou *opcodes*, esperam ser seguidos por um número exato de caches e instruem o interpretador a ignorá-los em tempo de execução. Os caches preenchidos podem parecer instruções arbitrárias, portanto, muito cuidado deve ser tomado ao ler ou modificar o bytecode adaptativo bruto que contém dados acelerados.

Novos opcodes

- `ASYNC_GEN_WRAP`, `RETURN_GENERATOR` e `SEND`, usados em geradores e corrotinas.
- `COPY_FREE_VARS`, que evita precisar de código especial no lado do chamado para fechamentos.
- `JUMP_BACKWARD_NO_INTERRUPT`, para usasr em certos loops nos quais o tratamento de interrupções é indesejável.
- `MAKE_CELL`, para criar Objeto célula.
- `CHECK_EG_MATCH` e `PREP_RERAISE_STAR`, para manipular novos grupos de exceção e except*, adicionados na **PEP 654** [https://peps.python.org/pep-0654/].
- `PUSH_EXC_INFO`, para uso em manipuladores de exceção.
- `RESUME`, um *no-op*, para rastreamento interno, depuração e verificações de otimização.

Opcodes substituídos

Opcode(s) substituído(s)	Novo(s) opcode(s)	Notas
`BINARY_*` `INPLACE_*`	`BINARY_OP`	Substituiu todos os opcodes numéricos binários/no local por um único opcode

Opcode(s) substituído(s)	Novo(s) opcode(s)	Notas

`CALL_FUNCTION`
`CALL_FUNCTION_KW`
`CALL_METHOD`
	`CALL`
	`KW_NAMES`
	`PRECALL`
	`PUSH_NULL`
		Separa a mudança de argumento para métodos da manipulação de argumentos nomeados; permite uma melhor especialização dos atendimentos

`DUP_TOP`
`DUP_TOP_TWO`
`ROT_TWO`
`ROT_THREE`
`ROT_FOUR`
`ROT_N`
	`COPY`
	`SWAP`
		Instruções de manipulação de pilha

`JUMP_IF_NOT_EXC_MATCH`	`CHECK_EXC_MATCH`	Agora efetua verificações, mas não pula

`JUMP_ABSOLUTE`
`POP_JUMP_IF_FALSE`
`POP_JUMP_IF_TRUE`
	`JUMP_BACKWARD`
	`POP_JUMP_BACKWARD_IF_*`
	`POP_JUMP_FORWARD_IF_*`
		Veja [[3]]. Variantes `TRUE`, `FALSE`, `NONE` e `NOT_NONE` para cada direção

`SETUP_WITH` `SETUP_ASYNC_WITH`	`BEFORE_WITH`	Configuração de bloco `with`

[3]

Todos os opcodes de salto agora são relativos, incluindo os existentes **`JUMP_IF_TRUE_OR_POP`** e **`JUMP_IF_FALSE_OR_POP`**. O argumento agora é um deslocamento da instrução atual em vez de um local absoluto.

Opcodes alterados/removidos

- Alterados `MATCH_CLASS` e `MATCH_KEYS` para não enviar mais um valor booleano adicional para indicar sucesso/falha. Em vez disso, `None` é enviado em caso de falha no lugar da tupla de valores extraídos.
- Alterados opcodes que funcionam com exceções para refleti-los agora sendo representados como um item na pilha em vez de três (consulte gh-89874 [https://github.com/python/cpython/issues/89874]).
- Removidos **`COPY_DICT_WITHOUT_KEYS`**, **`GEN_START`**, **`POP_BLOCK`**, **`SETUP_FINALLY`** e **`YIELD_FROM`**.

Descontinuados

Esta seção lista as APIs do Python que foram descontinuadas no Python 3.11.

As APIs C descontinuadas são listadas separadamente.

Linguagem/Embutidos

- O encadeamento de descritores `classmethod` (introduzido em bpo-19072 [https://bugs.python.org/issue?@action=redirect&bpo=19072]) agora foi descontinuado. Ele não pode mais ser usado para agrupar outros descritores como `property`. O design principal desse recurso era falho e causava vários problemas posteriores. Para "passar" um **`classmethod`**, considere usar o atributo **`__wrapped__`** que foi adicionado no Python 3.10. (Contribuição de Raymond Hettinger em gh-89519 [https://github.com/python/cpython/issues/89519].)
- Escapes octais em strings e bytes literais com valores maiores que `0o377` (255 em decimal) agora produzem um `DeprecationWarning`. Em uma versão futura do Python, eles levantarão um `SyntaxWarning` e eventualmente um `SyntaxError`. (Contribuição de Serhiy Storchaka em gh-81548 [https://github.com/python/cpython/issues/81548].)
- A delegação de `int()` para `__trunc__()` agora foi descontinuada. Chamar `int(a)` quando `type(a)` implementa **`__trunc__()`**, mas não `__int__()` ou `__index__()`, agora levanta uma exceção `DeprecationWarning`. (Contribuição de Zackery Spytz em bpo-44977 [https://bugs.python.org/issue?@action=redirect&bpo=44977].)

Módulos

- **PEP 594** [https://peps.python.org/pep-0594/] levou à descontinuação dos seguintes módulos programados para remoção no Python 3.13:

aifc	**chunk**	**msilib**	**pipes**	**telnetlib**
audioop	**crypt**	**nis**	**sndhdr**	**uu**
cgi	**imghdr**	**nntplib**	**spwd**	**xdrlib**
cgitb	**mailcap**	**ossaudiodev**	**sunau**	

  (Contribuição de Brett Cannon em bpo-47061 [https://bugs.python.org/issue?@action=redirect&bpo=47061] e Victor Stinner em gh-68966 [https://github.com/python/cpython/issues/68966].)

- Os módulos **`asynchat`**, **`asyncore`** e **`smtpd`** foram descontinuados desde pelo menos o Python 3.6. A documentação e os avisos de descontinuação foram atualizados para observar que serão removidos no Python 3.12. (Contribuição de Hugo van Kemenade em bpo-47022 [https://bugs.python.org/issue?@action=redirect&bpo=47022].)

- O pacote **`lib2to3`** e a ferramenta `2to3` agora foram descontinuados e podem não ser capazes de analisar o Python 3.10 ou mais recente. Veja **PEP 617** [https://peps.python.org/pep-0617/], apresentando o novo analisador GASE, para detalhes. (Contribuição de Victor Stinner em bpo-40360 [https://bugs.python.org/issue?@action=redirect&bpo=40360].)

- Módulos não documentados **`sre_compile`**, **`sre_constants`** e **`sre_parse`** agora fora descontinuados. (Contribuição de Serhiy Storchaka em bpo-47152 [https://bugs.python.org/issue?@action=redirect&bpo=47152].)

Biblioteca Padrão

- Os itens a seguir foram descontinuados em **`configparser`** desde o Python 3.2. Seus avisos de descontinuação agora foram atualizados para observar que serão removidos no Python 3.12:

  - a classe **`configparser.SafeConfigParser`**
  - a propriedade **`configparser.ParsingError.filename`**
  - o método **`configparser.RawConfigParser.readfp()`**

  (Contribuição de Hugo van Kemenade em bpo-45173 [https://bugs.python.org/issue?@action=redirect&bpo=45173].)

- **`configparser.LegacyInterpolation`** foi descontinuada na docstring desde o Python 3.2, e não está listada na documentação do **`configparser`**. Agora ela emite um **`DeprecationWarning`** e será removida no Python 3.13. Use **`configparser.BasicInterpolation`** ou **`configparser.ExtendedInterpolation`**. (Contribuição de Hugo van Kemenade em bpo-46607 [https://bugs.python.org/issue?@action=redirect&bpo=46607].)

- O antigo conjunto de funções **`importlib.resources`** foi descontinuado em favor das substituições adicionadas no Python 3.9 e será removido em uma versão futura do Python, devido ao não suporte de recursos localizados nos subdiretórios do pacote:

  - **`importlib.resources.contents()`**
  - **`importlib.resources.is_resource()`**
  - **`importlib.resources.open_binary()`**
  - **`importlib.resources.open_text()`**
  - **`importlib.resources.read_binary()`**
  - **`importlib.resources.read_text()`**
  - **`importlib.resources.path()`**

- A função **`locale.getdefaultlocale()`** foi descontinuada e será removida no Python 3.15. Use as funções **`locale.setlocale()`**, **`locale.getpreferredencoding(False)`** e **`locale.getlocale()`**. (Contribuição de Victor Stinner em gh-90817 [https://github.com/python/cpython/issues/90817].)

- A função **`locale.resetlocale()`** foi descontinuada e será removida no Python 3.13. Use `locale.setlocale(locale.LC_ALL, "")`. (Contribuição de Victor Stinner em gh-90817 [https://github.com/python/cpython/issues/90817].)

- Regras mais rígidas agora serão aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas sequências de dígitos ASCII serão aceitas como referência numérica, e o nome do grupo em padrões **`bytes`** e strings de substituição podem conter apenas letras ASCII,

dígitos e sublinhados. Por enquanto, um aviso de descontinuação é levantado para a sintaxe que viola essas regras. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)

- No módulo **re**, a função **re.template()** e os sinalizadores **re.TEMPLATE** e **re.T** correspondentes foram descontinuados, pois não foram documentados e careciam da um propósito óbvio. Eles serão removidos no Python 3.13. (Contribuição de Serhiy Storchaka e Miro Hrončok em gh-92728 [https://github.com/python/cpython/issues/92728].)

- **turtle.settiltangle()** foi descontinuada desde o Python 3.1; ela agora emite um aviso de descontinuação e será removida no Python 3.13. Em vez dela, use **turtle.tiltangle()** (foi marcada anteriormente incorretamente como descontinuada e sua docstring agora está corrigida). (Contribuição de Hugo van Kemenade em bpo-45837 [https://bugs.python.org/issue?@action=redirect&bpo=45837].)

- **typing.Text**, que existe apenas para fornecer suporte de compatibilidade entre o código Python 2 e Python 3, foi descontinuada nesta versão. Sua remoção não está planejada, mas os usuários são encorajados a usar **str** sempre que possível. (Contribuição de Alex Waygood em gh-92332 [https://github.com/python/cpython/issues/92332].)

- A sintaxe do argumento nomeado para a construção de tipos **typing.TypedDict** foi descontinuado neste versão. O suporte será removido no Python 3.13. (Contribuição de Jingchen Ye em gh-90224 [https://github.com/python/cpython/issues/90224].)

- **webbrowser.MacOSX** foi descontinuada e será removida no Python 3.13. Ela não foi testada, não foi documentada e não é usada pelo próprio **webbrowser**. (Contribuição de Dong-hee Na em bpo-42255 [https://bugs.python.org/issue?@action=redirect&bpo=42255].)

- O comportamento de retornar um valor de um dos métodos de teste de **TestCase** e **IsolatedAsyncioTestCase** (diferente do valor padrão `None`) foi descontinuado nesta versão.

- Foram descontinuadas as seguintes funções **unittest** não formalmente documentadas, agendadas para remoção no Python 3.13:

  - **unittest.findTestCases()**

  - `unittest.makeSuite()`
  - `unittest.getTestCaseNames()`

  Em vez delas, use os métodos de `TestLoader`:

  - `unittest.TestLoader.loadTestsFromModule()`
  - `unittest.TestLoader.loadTestsFromTestCase()`
  - `unittest.TestLoader.getTestCaseNames()`

  (Contribuição de Erlend E. Aasland em bpo-5846 [https://bugs.python.org/issue?@action=redirect&bpo=5846].)

- `unittest.TestProgram.usageExit()` foi marcado como descontinuado, para ser removido em 3.13. (Contribuição de Carlos Damázio em gh-67048 [https://github.com/python/cpython/issues/67048].)

Pendente remoção no Python 3.12

As seguintes APIs do Python foram descontinuadas em versões anteriores do Python e serão removidas no Python 3.12.

As APIs C com remoção pendente são listadas separadamente.

- O módulo `asynchat`
- O módulo `asyncore`
- Todo o pacote distutils
- O módulo `imp`
- O espaço de nomes `typing.io`
- O espaço de nomes `typing.re`
- `cgi.log()`
- `importlib.find_loader()`
- `importlib.abc.Loader.module_repr()`
- `importlib.abc.MetaPathFinder.find_module()`

- **`importlib.abc.PathEntryFinder.find_loader()`**
- **`importlib.abc.PathEntryFinder.find_module()`**
- **`importlib.machinery.BuiltinImporter.find_module()`**
- **`importlib.machinery.BuiltinLoader.module_repr()`**
- **`importlib.machinery.FileFinder.find_loader()`**
- **`importlib.machinery.FileFinder.find_module()`**
- **`importlib.machinery.FrozenImporter.find_module()`**
- **`importlib.machinery.FrozenLoader.module_repr()`**
- **`importlib.machinery.PathFinder.find_module()`**
- **`importlib.machinery.WindowsRegistryFinder.find_module()`**
- **`importlib.util.module_for_loader()`**
- **`importlib.util.set_loader_wrapper()`**
- **`importlib.util.set_package_wrapper()`**
- **`pkgutil.ImpImporter`**
- **`pkgutil.ImpLoader`**
- **`pathlib.Path.link_to()`**
- **`sqlite3.enable_shared_cache()`**
- **`sqlite3.OptimizedUnicode()`**
- Variável de ambiente **`PYTHONTHREADDEBUG`**
- Os seguintes aliases foram descontinuados no **`unittest`**:

Apelido descontinuado	Nome do método	Descontinuado em
`failUnless`	`assertTrue()`	3.1
`failIf`	`assertFalse()`	3.1
`failUnlessEqual`	`assertEqual()`	3.1
`failIfEqual`	`assertNotEqual()`	3.1
`failUnlessAlmostEqual`	`assertAlmostEqual()`	3.1
`failIfAlmostEqual`	`assertNotAlmostEqual()`	3.1
`failUnlessRaises`	`assertRaises()`	3.1
`assert_`	`assertTrue()`	3.2
`assertEquals`	`assertEqual()`	3.2
`assertNotEquals`	`assertNotEqual()`	3.2
`assertAlmostEquals`	`assertAlmostEqual()`	3.2
`assertNotAlmostEquals`	`assertNotAlmostEqual()`	3.2

Apelido descontinuado	Nome do método	Descontinuado em
`assertRegexpMatches`	`assertRegex()`	3.2
`assertRaisesRegexp`	`assertRaisesRegex()`	3.2
`assertNotRegexpMatches`	`assertNotRegex()`	3.5

Removidos

Esta seção lista as APIs do Python que foram removidas no Python 3.11.

As APIs C removidas são listadas separadamente.

- Removido o decorador **`@asyncio.coroutine()`** permitindo que corrotinas baseadas em gerador legado sejam compatíveis com o código **`async`** / **`await`**. A função foi descontinuada desde o Python 3.8 e a remoção foi inicialmente agendada para o Python 3.10. Use **`async def`** em vez disso. (Contribuição de Illia Volochii em bpo-43216 [https://bugs.python.org/issue?@action=redirect&bpo=43216].)

- Removida **`asyncio.coroutines.CoroWrapper`**, que era usada para encapsular objetos de corrotina baseados em gerador legado no modo de depuração. (Contribuição de Illia Volochii em bpo-43216 [https://bugs.python.org/issue?@action=redirect&bpo=43216].)

- Devido a preocupações significativas de segurança, o parâmetro *reuse_address* de **`asyncio.loop.create_datagram_endpoint()`**, desabilitado no Python 3.9, agora foi totalmente removido. Isto é devido ao comportamento da opção de socket `SO_REUSEDDR` no UDP. (Contribuição de Hugo van Kemenade em bpo-45129 [https://bugs.python.org/issue?@action=redirect&bpo=45129].)

- Removido o módulo **`binhex`**, descontinuado no Python 3.9. Também foram removidas as funções **`binascii`** relacionadas e igualmente descontinuadas:

- **binascii.a2b_hqx()**
- **binascii.b2a_hqx()**
- **binascii.rlecode_hqx()**
- **binascii.rldecode_hqx()**

A função **binascii.crc_hqx()** continua disponível.

(Contribuição de Victor Stinner em bpo-45085 [https://bugs.python.org/issue?@action=redirect&bpo=45085].)

- Removido o comando `bdist_msi` do **distutils** que foi descontinuado no Python 3.9. Em vez disso, use `bdist_wheel` (pacotes wheel). (Contribuição de Hugo van Kemenade em bpo-45124 [https://bugs.python.org/issue?@action=redirect&bpo=45124].)

- Removidos os métodos **__getitem__()** das classes **xml.dom.pulldom.DOMEventStream**, **wsgiref.util.FileWrapper** e **fileinput.FileInput**, descontinuados desde o Python 3.9 . (Contribuição de Hugo van Kemenade em bpo-45132 [https://bugs.python.org/issue?@action=redirect&bpo=45132].)

- Removidas as funções descontinuadas **lgettext()**, **ldgettext()**, **lngettext()** e **ldngettext()** de **gettext**. Também removidas a função **bind_textdomain_codeset()**, os métodos **NullTranslations.output_charset()** e **NullTranslations.set_output_charset()**, e o parâmetro *codeset* de **translation()** e **install()**, já que eles são usados apenas para as funções **l*gettext()**. (Contribuição de Dong-hee Na e Serhiy Storchaka em bpo-44235 [https://bugs.python.org/issue?@action=redirect&bpo=44235].)

- Removido do módulo **inspect**:

  - A função **getargspec()**, descontinuada desde o Python 3.0; use **inspect.signature()** ou **inspect.getfullargspec()**.
  - A função **formatargspec()**, descontinuada desde o Python 3.5; use a função **inspect.signature()** ou o objeto **inspect.Signature** diretamente.
  - Os métodos não documentados **Signature.from_builtin()** e **Signature.from_function()**, descontinuados desde o Python 3.5; use o método **Signature.from_callable()**.

(Contribuição de Hugo van Kemenade em bpo-45320 [https://bugs.python.org/issue?@action=redirect&bpo=45320].)

- Removido o método **`__class_getitem__()`** de **`pathlib.PurePath`**, pois não era usado e foi adicionado por engano em versões anteriores. (Contribuição de Nikita Sobolev em bpo-46483 [https://bugs.python.org/issue?@action=redirect&bpo=46483].)

- Removida a classe **`MailmanProxy`** do módulo **`smtpd`**, pois ela não pode ser usada sem o pacote **`mailman`** externo. (Contribuição de Dong-hee Na em bpo-35800 [https://bugs.python.org/issue?@action=redirect&bpo=35800].)

- Removido o método descontinuado **`split()`** de **`_tkinter.TkappType`**. (Contribuição de Erlend E. Aasland em bpo-38371 [https://bugs.python.org/issue?@action=redirect&bpo=38371].)

- Removido o suporte a pacote de espaço de nomes da descoberta de **`unittest`**. Foi introduzido no Python 3.4, mas está quebrado desde o Python 3.7. (Contribuição de Inada Naoki em bpo-23882 [https://bugs.python.org/issue?@action=redirect&bpo=23882].)

- Removido o método privado não documentado **`float.__set_format__()`**, anteriormente conhecido como **`float.__setformat__()`** no Python 3.7. Sua docstring dizia: "Você provavelmente não quer usar esta função. Ela existe principalmente para ser usada na conjunto de testes do Python." (Contribuição de Victor Stinner em bpo-46852 [https://bugs.python.org/issue?@action=redirect&bpo=46852].)

- O sinalizador de configuração **`--experimental-isolated-subinterpreters`** e a macro **`EXPERIMENTAL_ISOLATED_SUBINTERPRETERS`** correspondente foram removidos.

- Pynche [https://pypi.org/project/Pynche/] – O Pythonically Natural Color and Hue Editor – foi removido de `Tools/scripts` e está sendo desenvolvido independentemente [https://gitlab.com/warsaw/pynche/-/tree/main] da árvore fonte do Python.

Portando para Python 3.11

Esta seção lista as alterações descritas anteriormente e outras correções de bugs na API do Python que podem exigir alterações em seu código Python.

As notas de portabilidade para a API C são listadas separadamente.

- `open()`, `io.open()`, `codecs.open()` e `fileinput.FileInput` não aceitam mais `'U'` (“nova linha universal”) em o modo de arquivo. No Python 3, o modo “nova linha universal” é usado por padrão sempre que um arquivo é aberto no modo de texto, e o sinalizador `'U'` foi descontinuado desde o Python 3.3. O parâmetro de nova linha para essas funções controla como as novas linhas universais funcionam. (Contribuição de Victor Stinner em bpo-37330 [https://bugs.python.org/issue?@action=redirect&bpo=37330].)
- As posições do nó da `ast.AST` agora são validadas quando fornecidas para `compile()` e outras funções relacionadas. Se posições inválidas forem detectadas, uma exceção `ValueError` será levantado. (Contribuição de Pablo Galindo em gh-93351 [https://github.com/python/cpython/issues/93351])
- Proibido passar executores que não sejam `concurrent.futures.ThreadPoolExecutor` para `asyncio.loop.set_default_executor()` após uma descontinuação no Python 3.8. (Contribuição de Illia Volochii em bpo-43234 [https://bugs.python.org/issue?@action=redirect&bpo=43234].)
- `calendar`: As classes `calendar.LocaleTextCalendar` e `calendar.LocaleHTMLCalendar` agora usam `locale.getlocale()`, em vez de usar `locale.getdefaultlocale()`, se nenhuma localidade é especificada. (Contribuição de Victor Stinner em bpo-46659 [https://bugs.python.org/issue?@action=redirect&bpo=46659].)
- O módulo `pdb` agora lê o arquivo de configuração `.pdbrc` com a codificação `'UTF-8'`. (Contribuição de Srinivas Reddy Thatiparty (□□□□□□□□□□ □□□□□□ □□□□□□□□□□) em bpo-41137 [https://bugs.python.org/issue?@action=redirect&bpo=41137].)
- O parâmetro *population* de `random.sample()` deve ser uma sequência, e a conversão automática de `set`s para `list`s não é mais suportada. Além disso, se o tamanho da amostra for maior que o tamanho da população, uma exceção `ValueError` é levantada. (Contribuição de Raymond Hettinger em bpo-40465 [https://bugs.python.org/issue?@action=redirect&bpo=40465].)
- O parâmetro opcional *random* de `random.shuffle()` foi removido. Anteriormente, era uma função aleatória arbitrária a ser usada para o shuffle; agora, `random.random()` (seu padrão anterior) sempre será usado.
- Em Sintaxe de expressão regular do `re`, sinalizadores globais inline (por exemplo, `(?i)`) agora só podem ser usados no início de expressões regulares. Se uso em outro lugar foi descontinuado no Python 3.6. (Contribuição de Serhiy Storchaka em bpo-47066 [https://bugs.python.org/issue?@action=redirect&bpo=47066].)

- No módulo `re`, vários bugs de longa data foram corrigidos que, em casos raros, poderiam fazer com que os grupos de captura obtivessem o resultado errado. Portanto, isso pode alterar a saída capturada nesses casos. (Contribuição de Ma Lin em bpo-35859 [https://bugs.python.org/issue?@action=redirect&bpo=35859].)

Mudanças na construção

- CPython agora tem **suporte a Tier 3** [https://peps.python.org/pep-0011/#tier-3] da **PEP 11** [https://peps.python.org/pep-0011/] para compilação cruzada para as plataformas WebAssembly [https://webassembly.org/] Emscripten [https://emscripten.org/] (`wasm32-unknown-emscripten`, ou seja, Python no navegador) e WebAssembly System Interface (WASI) [https://wasi.dev/] (`wasm32-unknown-wasi`). O esforço é inspirado em trabalhos anteriores como Pyodide [https://pyodide.org/]. Essas plataformas fornecem um subconjunto limitado de APIs POSIX; Os recursos e módulos das bibliotecas padrão do Python relacionados a redes, processos, encadeamento, sinais, mmap e usuários/grupos não estão disponíveis ou não funcionam. (Emscripten contribuído por Christian Heimes e Ethan Smith em gh-84461 [https://github.com/python/cpython/issues/84461] e WASI contribuído por Christian Heimes em gh-90473 [https://github.com/python/cpython/issues/90473]; plataformas promovidas em gh-95085 [https://github.com/python/cpython/issues/95085])

- A construção do CPython agora requer:

  - Um compilador C11 [https://en.cppreference.com/w/c/11]. Recursos C11 opcionais [https://en.wikipedia.org/wiki/C11_(C_standard_revision)#Optional_features] não são necessários. (Contribuição de Victor Stinner em bpo-46656 [https://bugs.python.org/issue?@action=redirect&bpo=46656], bpo-45440 [https://bugs.python.org/issue?@action=redirect&bpo=45440] e bpo-46640 [https://bugs.python.org/issue?@action=redirect&bpo=46640].)
  - Suporte para números de ponto flutuante IEEE 754 [https://en.wikipedia.org/wiki/IEEE_754]. (Contribuição de Victor Stinner em bpo-46917 [https://bugs.python.org/issue?@action=redirect&bpo=46917].)

- A macro `Py_NO_NAN` foi removida. Como o CPython agora requer pontos flutuantes de IEEE 754, os valores NaN estão sempre disponíveis. (Contribuição de Victor Stinner em bpo-46656 [https://bugs.python.org/issue?@action=redirect&bpo=46656].)

- O pacote **`tkinter`** agora requer Tcl/Tk [https://www.tcl.tk] versão 8.5.12 ou mais recente. (Contribuição de Serhiy Storchaka em bpo-46996 [https://bugs.python.org/issue?@action=redirect&bpo=46996].)

- Dependências de construção, sinalizadores de compilador e sinalizadores de vinculador para a maioria dos módulos de extensão stdlib agora são detectados pelo **configure**. Os sinalizadores, ou *flags*, de libffi, libnsl, libsqlite3, zlib, bzip2, liblzma, libcrypt, Tcl/Tk e uuid são detectados por pkg-config [https://www.freedesktop.org/wiki/Software/pkg-config/] (Quando disponível). **`tkinter`** agora requer um comando pkg-config para detectar configurações de desenvolvimento para cabeçalhos e bibliotecas Tcl/Tk [https://www.tcl.tk]. (Contribuição de Christian Heimes e Erlend Egeberg Aasland em bpo-45847 [https://bugs.python.org/issue?@action=redirect&bpo=45847], bpo-45747 [https://bugs.python.org/issue?@action=redirect&bpo=45747] e bpo-45763 [https://bugs.python.org/issue?@action=redirect&bpo=45763].)

- libpython não está mais vinculado a libcrypt. (Contribuição de Mike Gilbert em bpo-45433 [https://bugs.python.org/issue?@action=redirect&bpo=45433].)

- O CPython agora pode ser construído com a opção ThinLTO [https://clang.llvm.org/docs/ThinLTO.html] passando `thin` para **`--with-lto`**, ou seja, `--with-lto=thin`. (Contribuição de Donghee Na e Brett Holman em bpo-44340 [https://bugs.python.org/issue?@action=redirect&bpo=44340].)

- Freelists para structs de objeto agora podem ser desabilitadas. Uma nova opção `--without-freelists` do **configure** pode ser usada para desabilitar todas as freelists, exceto a tupla singleton vazia. (Contribuição de Christian Heimes em bpo-45522 [https://bugs.python.org/issue?@action=redirect&bpo=45522].)

- `Modules/Setup` e `Modules/makesetup` foram melhorados e amarrados. Módulos de extensão agora podem ser construídos através do `makesetup`. Todos, exceto alguns módulos de teste, podem ser vinculados estaticamente em um binário principal ou biblioteca. (Contribuição de Brett Cannon e Christian Heimes em bpo-45548 [https://bugs.python.org/issue?@action=redirect&bpo=45548], bpo-45570 [https://bugs.python.org/issue?@action=redirect&bpo=45570], bpo-45571 [https://bugs.python.org/issue?@action=redirect&bpo=45571] e bpo-43974 [https://bugs.python.org/issue?@action=redirect&bpo=43974].)

Nota

Use as variáveis de ambiente **`TCLTK_CFLAGS`** e **`TCLTK_LIBS`** para especificar manualmente a localização dos cabeçalhos e bibliotecas Tcl/Tk. As opções **configure** **`--with-tcltk-includes`** e **`--with-tcltk-libs`** foram removidas.

No RHEL 7 e CentOS 7 os pacotes de desenvolvimento não fornecem `tcl.pc` e `tk.pc`; use `TCLTK_LIBS="-ltk8.5 -ltkstub8.5 -ltcl8.5"`. O diretório `Misc/rhel7` contém arquivos `.pc` e instruções sobre como construir Python com RHEL 7's e CentOS 7's Tcl/Tk e OpenSSL.

- O CPython agora usará dígitos de 30 bits por padrão para a implementação de **`int`** do Python. Anteriormente, o padrão era usar dígitos de 30 bits em plataformas com `SIZEOF_VOID_P >= 8` e, caso contrário, dígitos de 15 bits. Ainda é possível solicitar explicitamente o uso de dígitos de 15 bits através da opção **`--enable-big-digits`** para o script de configuração ou (para Windows) a variável `PYLONG_BITS_IN_DIGIT` em `PC/pyconfig.h`, mas esta opção pode ser removida em algum momento no futuro. (Contribuição de Mark Dickinson em bpo-45569 [https://bugs.python.org/issue?@action=redirect&bpo=45569].)

Alterações na API C

Novas funcionalidades

- Adiciona uma nova função **`PyType_GetName()`** para obter o nome curto do tipo. (Contribuição de Hai Shi em bpo-42035 [https://bugs.python.org/issue?@action=redirect&bpo=42035].)

- Adiciona uma nova função **`PyType_GetQualName()`** para obter o nome qualificado do tipo. (Contribuição de Hai Shi em bpo-42035 [https://bugs.python.org/issue?@action=redirect&bpo=42035].)

- Adiciona novas funções **`PyThreadState_EnterTracing()`** e **`PyThreadState_LeaveTracing()`** à API C limitada para suspender e retomar rastreamento e criação de perfil. (Contribuição de Victor Stinner em bpo-43760 [https://bugs.python.org/issue?@action=redirect&bpo=43760].)

- Adicionada a constante **`Py_Version`** que possui o mesmo valor que **`PY_VERSION_HEX`**. (Contribuição de Gabriele N. Tornetta em bpo-43931

[https://bugs.python.org/issue?@action=redirect&bpo=43931].)

- `Py_buffer` e APIs agora fazem parte da API limitada e da ABI estável:
  - `PyObject_CheckBuffer()`
  - `PyObject_GetBuffer()`
  - `PyBuffer_GetPointer()`
  - `PyBuffer_SizeFromFormat()`
  - `PyBuffer_ToContiguous()`
  - `PyBuffer_FromContiguous()`
  - `PyObject_CopyData()`
  - `PyBuffer_IsContiguous()`
  - `PyBuffer_FillContiguousStrides()`
  - `PyBuffer_FillInfo()`
  - `PyBuffer_Release()`
  - `PyMemoryView_FromBuffer()`
  - Slotes de tipo `bf_getbuffer` e `bf_releasebuffer`

  (Contribuição de Christian Heimes em bpo-45459 [https://bugs.python.org/issue?@action=redirect&bpo=45459].)

- Adicionada a função `PyType_GetModuleByDef()`, utilizada para obter o módulo no qual um método foi definido, nos casos em que esta informação não está disponível diretamente (via `PyCMethod`). (Contribuição de Petr Viktorin em bpo-46613 [https://bugs.python.org/issue?@action=redirect&bpo=46613].)

- Adiciona novas funções para compactar e desempacotar double C (serializar e desserializar): `PyFloat_Pack2()`, `PyFloat_Pack4()`, `PyFloat_Pack8()`, `PyFloat_Unpack2()` , `PyFloat_Unpack4()` e `PyFloat_Unpack8()`. (Contribuição de Victor Stinner em bpo-46906 [https://bugs.python.org/issue?@action=redirect&bpo=46906].)

- Adiciona novas funções para obter atributos do objeto quadro: `PyFrame_GetBuiltins()`, `PyFrame_GetGenerator()`, `PyFrame_GetGlobals()`, `PyFrame_GetLasti()`.

- Adicionadas duas novas funções para obter e definir a instância de exceção ativa: `PyErr_GetHandledException()` e `PyErr_SetHandledException()`. Estas são alternativas para `PyErr_SetExcInfo()` e `PyErr_GetExcInfo()` que funcionam com a representação de tuplas de 3 elementos legadas de

exceções. (Contribuição de Irit Katriel em bpo-46343 [https://bugs.python.org/issue?@action=redirect&bpo=46343].)

- Adicionada o membro `PyConfig.safe_path`. (Contribuição de Victor Stinner em gh-57684 [https://github.com/python/cpython/issues/57684].)

Portando para Python 3.11

- Algumas macros foram convertidas em funções inline estáticas para evitar armadilhas de macro [https://gcc.gnu.org/onlinedocs/cpp/Macro-Pitfalls.html]. A alteração deve ser transparente para os usuários, pois as funções de substituição converterão seus argumentos para os tipos esperados para evitar avisos do compilador devido a verificações de tipo estático. No entanto, quando a API C limitada é definida como >=3.11, essas conversões não são feitas e os chamadores precisarão converter argumentos para seus tipos esperados. Veja **PEP 670** [https://peps.python.org/pep-0670/] para mais detalhes. (Contribuição de Victor Stinner e Erlend E. Aasland em gh-89653 [https://github.com/python/cpython/issues/89653].)

- `PyErr_SetExcInfo()` não usa mais os argumentos `type` e `traceback`, o interpretador agora deriva esses valores da instância de exceção (o argumento `value`). A função ainda rouba referências de todos os três argumentos. (Contribuição de Irit Katriel em bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711].)

- `PyErr_GetExcInfo()` agora deriva os campos `type` e `traceback` do resultado da instância de exceção (o campo `value`). (Contribuição de Irit Katriel em bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711].)

- `_frozen` tem um novo campo `is_package` para indicar se o módulo frozen é ou não um pacote. Anteriormente, um valor negativo no campo `size` era o indicador. Agora apenas valores não negativos serão usados para `size`. (Contribuição de Kumar Aditya em bpo-46608 [https://bugs.python.org/issue?@action=redirect&bpo=46608].)

- `_PyFrameEvalFunction()` agora usa `_PyInterpreterFrame*` como seu segundo parâmetro, em vez de `PyFrameObject*`. Veja **PEP 523** [https://peps.python.org/pep-0523/] para mais detalhes de como usar este tipo de ponteiro de função.

- **`PyCode_New()`** e **`PyCode_NewWithPosOnlyArgs()`** agora recebem um argumento adicional `exception_table`. O uso dessas funções deve ser evitado, se possível. Para obter um objeto código personalizado: crie um objeto código usando o compilador e obtenha uma versão modificada com o método `replace`.

- **`PyCodeObject`** não possui mais os campos `co_code`, `co_varnames`, `co_cellvars` e `co_freevars`. Em vez disso, use **`PyCode_GetCode()`**, **`PyCode_GetVarnames()`**, **`PyCode_GetCellvars()`** e **`PyCode_GetFreevars()`** respectivamente para acessá-los por meio da API C. (Contribuição de Brandt Bucher em bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841] e Ken Jin em gh-92154 [https://github.com/python/cpython/issues/92154] e gh-94936 [https://github.com/python/cpython/issues/94936].)

- As antigas macros trashcan (`Py_TRASHCAN_SAFE_BEGIN`/`Py_TRASHCAN_SAFE_END`) foram descontinuadas. Elas devem ser substituídas pelas novas macros `Py_TRASHCAN_BEGIN` e `Py_TRASHCAN_END`.

  Uma função tp_dealloc que possui as macros antigas, como:

  ```
  static void
  dealocador_de_meutipo(meutipo *p)
  {
      PyObject_GC_UnTrack(p);
      Py_TRASHCAN_SAFE_BEGIN(p);
      ...
      Py_TRASHCAN_SAFE_END
  }
  ```

  deve migrar para as novas macros da seguinte forma:

  ```
  static void
  dealocador_de_meutipo(meutipo *p)
  {
      PyObject_GC_UnTrack(p);
      Py_TRASHCAN_BEGIN(p, dealocador_de_meutipo);
      ...
      Py_TRASHCAN_END
  }
  ```

  Observe que `Py_TRASHCAN_BEGIN` tem um segundo argumento que deve ser a função de desalocação em que está.

Para oferecer suporte a versões mais antigas do Python na mesma base de código, você pode definir as seguintes macros e usá-las em todo o código (crédito: elas foram copiadas da base de código do `mypy`):

```
#if PY_VERSION_HEX >= 0x03080000
#  define CPy_TRASHCAN_BEGIN(op, dealloc) Py_TRASHCAN_BEGIN(op, d
#  define CPy_TRASHCAN_END(op) Py_TRASHCAN_END
#else
#  define CPy_TRASHCAN_BEGIN(op, dealloc) Py_TRASHCAN_SAFE_BEGIN(
#  define CPy_TRASHCAN_END(op) Py_TRASHCAN_SAFE_END(op)
#endif
```

- A função **`PyType_Ready()`** agora levanta um erro se um tipo é definido com o sinalizador **`Py_TPFLAGS_HAVE_GC`** definido, mas não tem função transversal (**`PyTypeObject.tp_traverse`**). (Contribuição de Victor Stinner em bpo-44263 [https://bugs.python.org/issue?@action=redirect&bpo=44263].)

- Tipos de heap com o sinalizador **`Py_TPFLAGS_IMMUTABLETYPE`** agora podem herdar o protocolo vectorcall da **PEP 590** [https://peps.python.org/pep-0590/]. Anteriormente, isso só era possível para tipos estáticos. (Contribuição de Erlend E. Aasland em bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908])

- Uma vez que **`Py_TYPE()`** é alterada para uma função estática inline, `Py_TYPE(obj) = new_type` deve ser substituído por `Py_SET_TYPE(obj, new_type)`: consulte a função **`Py_SET_TYPE()`** (disponível desde Python 3.9). Para compatibilidade com versões anteriores, esta macro pode ser usada:

  ```
  #if PY_VERSION_HEX < 0x030900A4 && !defined(Py_SET_TYPE)
  static inline void _Py_SET_TYPE(PyObject *ob, PyTypeObject *type)
  { ob->ob_type = type; }
  #define Py_SET_TYPE(ob, type) _Py_SET_TYPE((PyObject*)(ob), type)
  #endif
  ```

  (Contribuição de Victor Stinner em bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573].)

- Uma vez que **`Py_SIZE()`** é alterada para uma função estática inline, `Py_SIZE(obj) = new_size` deve ser substituído por `Py_SET_SIZE(obj, new_size)`: veja a função **`Py_SET_SIZE()`** (disponível desde Python 3.9). Para compatibilidade com versões anteriores, esta macro pode ser usada:

```
#if PY_VERSION_HEX < 0x030900A4 && !defined(Py_SET_SIZE)
static inline void _Py_SET_SIZE(PyVarObject *ob, Py_ssize_t size)
{ ob->ob_size = size; }
#define Py_SET_SIZE(ob, size) _Py_SET_SIZE((PyVarObject*)(ob), si
#endif
```

(Contribuição de Victor Stinner em bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573].)

- `<Python.h>` não inclui mais os arquivos de cabeçalho `<stdlib.h>`, `<stdio.h>`, `<errno.h>` e `<string.h>` quando a macro `Py_LIMITED_API` é definida como `0x030b0000` (Python 3.11) ou superior. As extensões C devem incluir explicitamente os arquivos de cabeçalho após `#include <Python.h>`. (Contribuição de Victor Stinner em bpo-45434 [https://bugs.python.org/issue?@action=redirect&bpo=45434].)

- Os arquivos API não limitados `cellobject.h`, `classobject.h`, `code.h`, `context.h`, `funcobject.h`, `genobject.h` e `longintrepr.h` foram movidos para o diretório `Include/cpython`. Além disso, o arquivo de cabeçalho `eval.h` foi removido. Esses arquivos não devem ser incluídos diretamente, pois já estão incluídos aos arquivos de inclusão do `Python.h`. Se eles foram incluídos diretamente, considere incluir `Python.h` em seu lugar. (Contribuição de Victor Stinner em bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134].)

- A macro **`PyUnicode_CHECK_INTERNED()`** foi excluída da API C limitada. Nunca foi utilizável lá, porque usava estruturas internas que não estão disponíveis na API C limitada. (Contribuição de Victor Stinner em bpo-46007 [https://bugs.python.org/issue?@action=redirect&bpo=46007].)

- As seguintes funções e tipo de quadro agora estão disponíveis diretamente com `#include <Python.h>`, não é mais necessário adicionar `#include <frameobject.h>`:

  - **`PyFrame_Check()`**
  - **`PyFrame_GetBack()`**
  - **`PyFrame_GetBuiltins()`**
  - **`PyFrame_GetGenerator()`**
  - **`PyFrame_GetGlobals()`**
  - **`PyFrame_GetLasti()`**
  - **`PyFrame_GetLocals()`**

- **PyFrame_Type**

(Contribuição de Victor Stinner em gh-93937 [https://github.com/python/cpython/issues/93937].)

- Os membros da estrutura **PyFrameObject** foram removidos da API C pública.

  Embora a documentação observe que os campos **PyFrameObject** estão sujeitos a alterações a qualquer momento, eles permaneceram estáveis por um longo tempo e foram usados em várias extensões populares.

  No Python 3.11, a estrutura do quadro foi reorganizada para permitir otimizações de desempenho. Alguns campos foram totalmente removidos, pois eram detalhes da antiga implementação.

  Campos do **PyFrameObject**:

  - `f_back`: use **PyFrame_GetBack()**.
  - `f_blockstack`: removido.
  - `f_builtins`: use **PyFrame_GetBuiltins()**.
  - `f_code`: use **PyFrame_GetCode()**.
  - `f_gen`: use **PyFrame_GetGenerator()**.
  - `f_globals`: use **PyFrame_GetGlobals()**.
  - `f_iblock`: removido.
  - `f_lasti`: use **PyFrame_GetLasti()**. Código usando `f_lasti` com `PyCode_Addr2Line()` deve usar **PyFrame_GetLineNumber()**, pois deve ser mais rápido.
  - `f_lineno`: use **PyFrame_GetLineNumber()**
  - `f_locals`: use **PyFrame_GetLocals()**.
  - `f_stackdepth`: removido.
  - `f_state`: nenhuma API pública (renomeado para `f_frame.f_state`).
  - `f_trace`: nenhuma API pública.
  - `f_trace_lines`: use `PyObject_GetAttrString((PyObject*)frame, "f_trace_lines")`.
  - `f_trace_opcodes`: use `PyObject_GetAttrString((PyObject*)frame, "f_trace_opcodes")`.
  - `f_localsplus`: nenhuma API pública (renomeado para `f_frame.localsplus`).
  - `f_valuestack`: removido.

O objeto quadro do Python agora é criado de forma “preguiçosa” (*lazy*). Um efeito colateral é que o membro **`f_back`** não deve ser acessado diretamente, já que seu valor agora também é calculado de forma “preguiçosa”. A função **`PyFrame_GetBack()`** deve ser chamada em seu lugar.

Depuradores que acessam **`f_locals`** diretamente *devem* chamar **`PyFrame_GetLocals()`**. Eles não precisam mais chamar **`PyFrame_FastToLocalsWithError()`** ou **`PyFrame_LocalsToFast()`**. Na verdade, eles não devem chamar essas funções. A atualização necessária do quadro agora é gerenciada pela máquina virtual.

Código definindo `PyFrame_GetCode()` no Python 3.8 e anteriores:

```
#if PY_VERSION_HEX < 0x030900B1
static inline PyCodeObject* PyFrame_GetCode(PyFrameObject *frame)
{
    Py_INCREF(frame->f_code);
    return frame->f_code;
}
#endif
```

Código definindo `PyFrame_GetBack()` no Python 3.8 e anteriores:

```
#if PY_VERSION_HEX < 0x030900B1
static inline PyFrameObject* PyFrame_GetBack(PyFrameObject *frame
{
    Py_XINCREF(frame->f_back);
    return frame->f_back;
}
#endif
```

Ou use o projeto pythoncapi_compat [https://github.com/python/pythoncapi-compat] para obter essas duas funções em versões mais antigas do Python.

- Mudanças nos membros da estrutura **`PyThreadState`**:
    - `frame`: removido, use **`PyThreadState_GetFrame()`** (função adicionada ao Python 3.9 por bpo-40429 [https://bugs.python.org/issue?@action=redirect&bpo=40429]). Aviso: a função retorna um referência forte, precisa chamar **`Py_XDECREF()`**.
    - `tracing`: alterado, use **`PyThreadState_EnterTracing()`** e **`PyThreadState_LeaveTracing()`** (funções adicionadas ao Python 3.11

por bpo-43760 [https://bugs.python.org/issue?@action=redirect&bpo=43760]).
  - `recursion_depth`: removido, use `(tstate->recursion_limit - tstate->recursion_remaining)`.
  - `stackcheck_counter`: removido.

Código definindo `PyThreadState_GetFrame()` no Python 3.8 e anteriores:

```
#if PY_VERSION_HEX < 0x030900B1
static inline PyFrameObject* PyThreadState_GetFrame(PyThreadState
{
    Py_XINCREF(tstate->frame);
    return tstate->frame;
}
#endif
```

Código definindo `PyThreadState_EnterTracing()` e `PyThreadState_LeaveTracing()` no Python 3.10 e anteriores:

```
#if PY_VERSION_HEX < 0x030B00A2
static inline void PyThreadState_EnterTracing(PyThreadState *tsta
{
    tstate->tracing++;
#if PY_VERSION_HEX >= 0x030A00A1
    tstate->cframe->use_tracing = 0;
#else
    tstate->use_tracing = 0;
#endif
}

static inline void PyThreadState_LeaveTracing(PyThreadState *tsta
{
    int use_tracing = (tstate->c_tracefunc != NULL || tstate->c_p
    tstate->tracing--;
#if PY_VERSION_HEX >= 0x030A00A1
    tstate->cframe->use_tracing = use_tracing;
#else
    tstate->use_tracing = use_tracing;
#endif
}
#endif
```

Ou use o projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat] para obter essas funções em funções antigas do Python.

- Os distribuidores são encorajados a compilar o Python com a biblioteca Blake2 otimizada libb2 [https://www.blake2.net/].

- O campo **`PyConfig.module_search_paths_set`** agora deve ser definido como 1 para inicialização para usar **`PyConfig.module_search_paths`** para inicializar **`sys.path`**. Caso contrário, a inicialização recalculará o caminho e substituirá quaisquer valores adicionados a `module_search_paths`.

- **`PyConfig_Read()`** não calcula mais o caminho de pesquisa inicial e não preencherá nenhum valor em **`PyConfig.module_search_paths`**. Para calcular caminhos padrão e modificá-los, termine a inicialização e use **`PySys_GetObject()`** para recuperar **`sys.path`** como um objeto lista do Python e modifique-o diretamente.

Descontinuados

- Descontinua as seguintes funções para configurar a inicialização do Python:

  - **`PySys_AddWarnOptionUnicode()`**
  - **`PySys_AddWarnOption()`**
  - **`PySys_AddXOption()`**
  - **`PySys_HasWarnOptions()`**
  - **`PySys_SetArgvEx()`**
  - **`PySys_SetArgv()`**
  - **`PySys_SetPath()`**
  - **`Py_SetPath()`**
  - **`Py_SetProgramName()`**
  - **`Py_SetPythonHome()`**
  - **`Py_SetStandardStreamEncoding()`**
  - **`_Py_SetProgramFullPath()`**

  Em vez disso, use a nova API **`PyConfig`** da Configuração de Inicialização do Python (**PEP 587** [https://peps.python.org/pep-0587/]). (Contribuição de Victor Stinner em gh-88279 [https://github.com/python/cpython/issues/88279].)

- Descontinua o membro `ob_shash` do **`PyBytesObject`**. Use **`PyObject_Hash()`** em vez disso. (Contribuição de Inada Naoki em bpo-46864 [https://bugs.python.org/issue?@action=redirect&bpo=46864].)

Pendente remoção no Python 3.12

As seguintes APIs C foram descontinuadas em versões anteriores do Python e serão removidas no Python 3.12.

- **`PyUnicode_AS_DATA()`**
- **`PyUnicode_AS_UNICODE()`**
- **`PyUnicode_AsUnicodeAndSize()`**
- **`PyUnicode_AsUnicode()`**
- **`PyUnicode_FromUnicode()`**
- **`PyUnicode_GET_DATA_SIZE()`**
- **`PyUnicode_GET_SIZE()`**
- **`PyUnicode_GetSize()`**
- **`PyUnicode_IS_COMPACT()`**
- **`PyUnicode_IS_READY()`**
- **`PyUnicode_READY()`**
- **`PyUnicode_WSTR_LENGTH()`**
- **`_PyUnicode_AsUnicode()`**
- **`PyUnicode_WCHAR_KIND`**
- **`PyUnicodeObject`**
- **`PyUnicode_InternImmortal()`**

Removidos

- **`PyFrame_BlockSetup()`** e **`PyFrame_BlockPop()`** foram removidas. (Contribuição de Mark Shannon em bpo-40222 [https://bugs.python.org/issue?@action=redirect&bpo=40222].)

- Remove as seguintes macros matemáticas usando a variável `errno`:

  - `Py_ADJUST_ERANGE1()`
  - `Py_ADJUST_ERANGE2()`
  - `Py_OVERFLOWED()`
  - `Py_SET_ERANGE_IF_OVERFLOW()`
  - `Py_SET_ERRNO_ON_MATH_ERROR()`

  (Contribuição de Victor Stinner em bpo-45412 [https://bugs.python.org/issue?@action=redirect&bpo=45412].)

- Remove as macros `Py_UNICODE_COPY()` e `Py_UNICODE_FILL()`, descontinuadas desde o Python 3.3. Use `PyUnicode_CopyCharacters()` ou `memcpy()` (string `wchar_t*`) e as funções `PyUnicode_Fill()`. (Contribuição

de Victor Stinner em bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123].)

- Remove o arquivo de cabeçalho `pystrhex.h`. Ele contém apenas funções privadas. As extensões C devem incluir apenas o arquivo principal `<Python.h>`. (Contribuição de Victor Stinner em bpo-45434 [https://bugs.python.org/issue?@action=redirect&bpo=45434].)

- Remove a macro `Py_FORCE_DOUBLE()`. Era usada pela macro `Py_IS_INFINITY()`. (Contribuição de Victor Stinner em bpo-45440 [https://bugs.python.org/issue?@action=redirect&bpo=45440].)

- Os seguintes itens não estão mais disponíveis quando **`Py_LIMITED_API`** é definida:

  - **`PyMarshal_WriteLongToFile()`**
  - **`PyMarshal_WriteObjectToFile()`**
  - **`PyMarshal_ReadObjectFromString()`**
  - **`PyMarshal_WriteObjectToString()`**
  - a macro `Py_MARSHAL_VERSION`

  Eles não fazem parte da API limitada.

  (Contribuição de Victor Stinner em bpo-45474 [https://bugs.python.org/issue?@action=redirect&bpo=45474].)

- Exclui **`PyWeakref_GET_OBJECT()`** da API C limitada. Nunca funcionou porque a estrutura PyWeakReference é opaca na API C limitada. (Contribuição de Victor Stinner em bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134].)

- Remove a macro `PyHeapType_GET_MEMBERS()`. Estava exposta na API C pública por engano, deve ser usada apenas pelo Python internamente. Em vez disso, use o membro `PyTypeObject.tp_members`. (Contribuição de Victor Stinner em bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170].)

- Remove a macro `HAVE_PY_SET_53BIT_PRECISION` (movida para a API C interna). (Contribuição de Victor Stinner em bpo-45412 [https://bugs.python.org/issue?@action=redirect&bpo=45412].)

- Remove as APIs de codificação **`Py_UNICODE`**, pois foram descontinuadas desde o Python 3.3, são pouco usadas e ineficientes em relação às alternativas

recomendadas.

As funções removidas são:

- **`PyUnicode_Encode()`**
- **`PyUnicode_EncodeASCII()`**
- **`PyUnicode_EncodeLatin1()`**
- **`PyUnicode_EncodeUTF7()`**
- **`PyUnicode_EncodeUTF8()`**
- **`PyUnicode_EncodeUTF16()`**
- **`PyUnicode_EncodeUTF32()`**
- **`PyUnicode_EncodeUnicodeEscape()`**
- **`PyUnicode_EncodeRawUnicodeEscape()`**
- **`PyUnicode_EncodeCharmap()`**
- **`PyUnicode_TranslateCharmap()`**
- **`PyUnicode_EncodeDecimal()`**
- **`PyUnicode_TransformDecimalToASCII()`**

Veja a **PEP 624** [https://peps.python.org/pep-0624/] para detalhes e **orientação sobre migração** [https://peps.python.org/pep-0624/#alternative-apis]. (Contribuição de Inada Naoki em bpo-44029 [https://bugs.python.org/issue?@action=redirect&bpo=44029].)

Alterações notáveis no 3.11.4

tarfile

- Os métodos de extração em **`tarfile`** e **`shutil.unpack_archive()`**, têm um novo argumento *filter* que permite limitar recursos do tar que podem ser surpreendentes ou perigosos, como criar arquivos fora do diretório de destino. Veja Filtros de extração para detalhes. No Python 3.12, usar sem o argumento *filter* mostrará um **`DeprecationWarning`**. No Python 3.14, o padrão mudará para `'data'`. (Contribuição de Petr Viktorin em **PEP 706** [https://peps.python.org/pep-0706/].)

Alterações notáveis no 3.11.5

OpenSSL

- Construções do Windows e instaladores do macOS em python.org agora usam OpenSSL 3.0.

O que há de novo no Python 3.10

Editor:: Pablo Galindo Salgado

Este artigo explica os novos recursos no Python 3.10, em comparação com 3.9. Python 3.10 foi lançado em 4 de outubro de 2021. Veja changelog para uma lista completa de mudanças.

Resumo – Destaques da versão

Novos recursos de sintaxe:

- **PEP 634** [https://peps.python.org/pep-0634/], Correspondência de padrão estrutural: especificação
- **PEP 635** [https://peps.python.org/pep-0635/], Correspondência de padrão estrutural: motivação e justificativa
- **PEP 636** [https://peps.python.org/pep-0636/], Correspondência de padrão estrutural: Tutorial
- bpo-12782 [https://bugs.python.org/issue?@action=redirect&bpo=12782], Gerenciadores de contexto entre parênteses agora são permitidos oficialmente.

Novos recursos na biblioteca padrão:

- **PEP 618** [https://peps.python.org/pep-0618/], Adiciona verificação de comprimento opcional ao zip.

Melhorias no interpretador:

- **PEP 626** [https://peps.python.org/pep-0626/], Números de linha precisos para depuração e outras ferramentas.

Novos recursos de tipagem:

- **PEP 604** [https://peps.python.org/pep-0604/], Permite escrever tipos de união como X | Y
- **PEP 612** [https://peps.python.org/pep-0612/], Variáveis de especificação de parâmetro
- **PEP 613** [https://peps.python.org/pep-0613/], Apelidos de tipo explícitos
- **PEP 647** [https://peps.python.org/pep-0647/], Guardas de Tipo Definidas Pelo Usuário

Descontinuações, remoções ou restrições importantes:

- **PEP 644** [https://peps.python.org/pep-0644/], Exige OpenSSL 1.1.1 ou mais novo
- **PEP 632** [https://peps.python.org/pep-0632/], Descontinua o módulo distutils.
- **PEP 623** [https://peps.python.org/pep-0623/], Descontinua e prepara para a remoção do membro wstr em PyUnicodeObject.
- **PEP 624** [https://peps.python.org/pep-0624/], Remove APIs codificadoras de Py_UNICODE
- **PEP 597** [https://peps.python.org/pep-0597/], Adiciona EncodingWarning opcional

Novas funcionalidades

Gerenciadores de contexto entre parênteses

O uso de parênteses para continuação em várias linhas em gerenciadores de contexto agora é suportado. Isso permite a formatação de uma longa coleção de gerenciadores de contexto em várias linhas de maneira semelhante à que era possível anteriormente com instruções de importação. Por exemplo, todos esses exemplos agora são válidos:

```
with (CtxManager() as exemplo):
    ...

with (
    CtxManager1(),
    CtxManager2()
):
    ...
```

```
with (CtxManager1() as exemplo,
      CtxManager2()):
    ...

with (CtxManager1(),
      CtxManager2() as exemplo):
    ...

with (
    CtxManager1() as exemplo1,
    CtxManager2() as exemplo2
):
    ...
```

também é possível usar uma vírgula no final do grupo fechado:

```
with (
    CtxManager1() as exemplo1,
    CtxManager2() as exemplo2,
    CtxManager3() as exemplo3,
):
    ...
```

Esta nova sintaxe usa as capacidades não LL(1) do novo analisador sintático. Confira **PEP 617** [https://peps.python.org/pep-0617/] para mais detalhes.

(Contribuição de Guido van Rossum, Pablo Galindo e Lysandros Nikolaou em bpo-12782 [https://bugs.python.org/issue?@action=redirect&bpo=12782] e bpo-40334 [https://bugs.python.org/issue?@action=redirect&bpo=40334].)

Melhores mensagens de erro

SyntaxErrors

Ao analisar o código que contém parênteses ou chaves não fechados, o interpretador agora inclui o local da chave não fechado de parênteses em vez de exibir *SyntaxError: unexpected EOF while parsing* ou apontando para algum local incorreto. Por exemplo, considere o seguinte código (observe o “{” não fechado):

```
esperado = {9: 1, 18: 2, 19: 2, 27: 3, 28: 3, 29: 3, 36: 4, 37:
            38: 4, 39: 4, 45: 5, 46: 5, 47: 5, 48: 5, 49: 5, 54:
algum_outro_codigo = foo()
```

Versões anteriores do interpretador relatavam lugares confusos como local do erro de sintaxe:

```
File "exemplo.py", line 3
    algum_outro_codigo = foo()
                       ^
SyntaxError: invalid syntax
```

mas no Python 3.10, um erro mais informativo é emitido:

```
File "exemplo.py", line 1
    esperado = {9: 1, 18: 2, 19: 2, 27: 3, 28: 3, 29: 3, 36: 4,
               ^
SyntaxError: '{' was never closed
```

De maneira semelhante, erros envolvendo literais de string não fechadas (entre aspas simples e triplas) agora apontam para o início da string em vez de relatar EOF/EOL.

Essas melhorias são inspiradas em trabalhos anteriores no interpretador PyPy.

(Contribuição de Pablo Galindo em bpo-42864 [https://bugs.python.org/issue?@action=redirect&bpo=42864] e Batuhan Taskaya em bpo-40176 [https://bugs.python.org/issue?@action=redirect&bpo=40176].)

As exceções `SyntaxError` levantadas pelo interpretador agora destacam o intervalo total de erros da expressão que constitui o próprio erro de sintaxe, em vez de apenas onde o problema foi detectado. Desta forma, em vez de exibir (antes do Python 3.10):

```
>>> foo(x, z for z in range(10), t, w)
  File "<stdin>", line 1
    foo(x, z for z in range(10), t, w)
```

```
           ^
SyntaxError: Generator expression must be parenthesized
```

agora o Python 3.10 vai exibir a exceção como:

```
>>> foo(x, z for z in range(10), t, w)
  File "<stdin>", line 1
    foo(x, z for z in range(10), t, w)
           ^^^^^^^^^^^^^^^^^^^^
SyntaxError: Generator expression must be parenthesized
```

Esta melhoria foi contribuída por Pablo Galindo em bpo-43914 [https://bugs.python.org/issue?@action=redirect&bpo=43914].

Uma quantidade considerável de novas mensagens especializadas para exceções `SyntaxError` foram incorporadas. Alguns dos mais notáveis são os seguintes:

- Faltando `:` antes de blocos:

  ```
  >>> if foguete.posicao > horizonte_evento
    File "<stdin>", line 1
      if foguete.posicao > horizonte_evento
                                           ^
  SyntaxError: expected ':'
  ```

  (Contribuição de Pablo Galindo em bpo-42997 [https://bugs.python.org/issue?@action=redirect&bpo=42997].)

- Tuplas sem parênteses em alvos de compreensão:

  ```
  >>> {x,y for x,y in zip('abcd', '1234')}
    File "<stdin>", line 1
      {x,y for x,y in zip('abcd', '1234')}
       ^
  SyntaxError: did you forget parentheses around the compreher
  ```

  (Contribuição de Pablo Galindo em bpo-43017 [https://bugs.python.org/issue?@action=redirect&bpo=43017].)

- Faltando vírgulas em literais de coleção e entre expressões:

```
>>> items = {
... x: 1,
... y: 2
... z: 3,
  File "<stdin>", line 3
    y: 2
       ^
SyntaxError: invalid syntax. Perhaps you forgot a comma?
```

(Contribuição de Pablo Galindo em bpo-43822 [https://bugs.python.org/issue?@action=redirect&bpo=43822].)

- Vários tipos de exceção sem parênteses:

```
>>> try:
...     construir_esfera_de_dyson()
... except NotEnoughScienceError, NotEnoughResourcesError:
  File "<stdin>", line 3
    except NotEnoughScienceError, NotEnoughResourcesError:
           ^
SyntaxError: multiple exception types must be parenthesized
```

(Contribuição de Pablo Galindo em bpo-43149 [https://bugs.python.org/issue?@action=redirect&bpo=43149].)

- Faltando `:` em valores em literais de dicionário:

```
>>> values = {
... x: 1,
... y: 2,
... z:
... }
  File "<stdin>", line 4
    z:
     ^
SyntaxError: expression expected after dictionary key and '

>>> values = {x:1, y:2, z w:3}
  File "<stdin>", line 1
    values = {x:1, y:2, z w:3}
```

```
                          ^
SyntaxError: ':' expected after dictionary key
```

(Contribuição de Pablo Galindo em bpo-43823 [https://bugs.python.org/issue?@action=redirect&bpo=43823].)

- Blocos `try` sem blocos `except` ou `finally`:

```
>>> try:
...     x = 2
... algumacoisa = 3
  File "<stdin>", line 3
    algumacoisa = 3
    ^^^^^^^^^^^
SyntaxError: expected 'except' or 'finally' block
```

(Contribuição de Pablo Galindo em bpo-44305 [https://bugs.python.org/issue?@action=redirect&bpo=44305].)

- Uso de = em vez de == nas comparações:

```
>>> if foguete.posicao = horizonte_evento:
  File "<stdin>", line 1
    if foguete.posicao = horizonte_evento:
                       ^
SyntaxError: cannot assign to attribute here. Maybe you mean
```

(Contribuição de Pablo Galindo em bpo-43797 [https://bugs.python.org/issue?@action=redirect&bpo=43797].)

- Uso de `*` em f-strings:

```
>>> f"Buracos negros {*all_black_holes} e revelações"
  File "<stdin>", line 1
    (*all_black_holes)
     ^
SyntaxError: f-string: cannot use starred expression here
```

(Contribuição de Pablo Galindo em bpo-41064 [https://bugs.python.org/issue?@action=redirect&bpo=41064].)

IndentationErrors

Muitas exceções `IndentationError` agora têm mais contexto sobre que tipo de bloco estava esperando um indentação, incluindo a localização da instrução:

```
>>> def foo():
...     if lel:
...     x = 2
  File "<stdin>", line 3
    x = 2
    ^
IndentationError: expected an indented block after 'if' statemer
```

AttributeErrors

Ao exibir `AttributeError`, `PyErr_Display()` oferecerá sugestões de nomes de atributos semelhantes no objeto a partir do qual a exceção foi levantada:

```
>>> collections.namedtoplo
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'collections' has no attribute 'namedtopl
```

(Contribuição de Pablo Galindo em bpo-38530 [https://bugs.python.org/issue?@action=redirect&bpo=38530].)

> **Aviso**
>
> Observe que isso não funcionará se `PyErr_Display()` não for chamado para exibir o erro que pode ocorrer se alguma outra função personalizada de exibição de erro for usada. Este é um cenário comum em alguns REPLs, laços de leitura-avaliação-impressão, como o IPython.

NameErrors

Ao exibir `NameError` levantada pelo interpretador, `PyErr_Display()` irá oferecer sugestões de nomes de variáveis semelhantes na função de onde a exceção foi levantada:

```
>>> buraco_negro_de_schwarzschild = None
>>> buraco_negro_de_schwarschild
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'buraco_negro_de_schwarschild' is not defined. D
```

(Contribuição de Pablo Galindo em bpo-38530 [https://bugs.python.org/issue?@action=redirect&bpo=38530].)

Aviso

Observe que isso não funcionará se `PyErr_Display()` não for chamado para exibir o erro que pode ocorrer se alguma outra função personalizada de exibição de erro for usada. Este é um cenário comum em alguns REPLs, laços de leitura-avaliação-impressão, como o IPython.

PEP 626: Números de linha precisos para depuração e outras ferramentas

O PEP 626 traz números de linha mais precisos e confiáveis para ferramentas de depuração, criação de perfil e cobertura. Eventos de rastreamento, com o número de linha correto, são gerados para todas as linhas de código executadas e apenas para linhas de código que são executadas.

O atributo `f_lineno` de objetos de quadro sempre conterá o número de linha esperado.

O atributo `codeobject.co_lnotab` de objetos código foi descontinuado e será removido no 3.12. O código que precisa ser convertido do deslocamento para o número da linha deve usar o novo método `co_lines()`.

PEP 634: Correspondência de padrão estrutural

A correspondência de padrão estrutural foi adicionada na forma de uma *instrução de correspondência* e *instruções de caso* de padrões com ações associadas. Os padrões consistem em sequências, mapeamentos, tipos de dados primitivos, bem como instâncias de classe. A correspondência de padrão permite que os programas extraiam informações de tipos de dados complexos, ramifiquem na estrutura de dados e apliquem ações específicas com base em diferentes formas de dados.

Sintaxe e operações

A sintaxe genérica da correspondência de padrão é:

```
match sujeito:
    case <padrão_1>:
        <ação_1>
    case <padrão_2>:
        <ação_2>
    case <padrão_3>:
        <ação_3>
    case _:
        <ação_curinga>
```

Uma instrução de correspondência pega uma expressão e compara seu valor com padrões sucessivos fornecidos como um ou mais blocos de caso. Especificamente, a correspondência de padrões opera:

1. usando dados com tipo e forma (o `sujeito`)
2. avaliando o `sujeito` na instrução `match`
3. comparando o assunto com cada padrão em uma instrução `case` de cima para baixo até que uma correspondência seja confirmada.
4. executando a ação associada ao padrão da correspondência confirmada
5. Se uma correspondência exata não for confirmada, no último caso, um curinga _, se fornecido, será usado como o caso de correspondência. Se uma correspondência exata não for confirmada e não houver um caractere curinga, todo o bloco de correspondência será autônomo.

Abordagem declarativa

Os leitores podem estar cientes da correspondência de padrão por meio do exemplo simples de correspondência de um assunto (objeto de dados) a um literal (padrão) com a instrução switch encontrada em C, Java ou JavaScript (e muitas outras linguagens). Frequentemente, a instrução switch é usada para comparação de um objeto/expressão com instruções case contendo literais.

Exemplos mais poderosos de correspondência de padrão podem ser encontrados em linguagens como Scala e Elixir. Com a correspondência de padrão estrutural, a abordagem é “declarativa” e declara explicitamente as condições (os padrões) para que os dados correspondam.

Embora uma série “imperativa” de instruções usando instruções “if” aninhadas possa ser usada para realizar algo semelhante à correspondência de padrão estrutural, é menos clara do que a abordagem “declarativa”. Em vez disso, a abordagem “declarativa” estabelece as condições a serem atendidas para uma correspondência e é mais legível por meio de seus padrões explícitos. Embora a correspondência de padrão estrutural possa ser usada em sua forma mais simples comparando uma variável a um literal em uma instrução case, seu verdadeiro valor para Python reside em seu tratamento do tipo e forma do sujeito.

Padrão simples: corresponder a um literal

Vejamos este exemplo como correspondência de padrão em sua forma mais simples: um valor, o assunto, sendo correspondido a vários literais, os padrões. No exemplo abaixo, `status` é o assunto da instrução de correspondência. Os padrões são cada uma das instruções de caso, onde literais representam códigos de status de solicitação. A ação associada ao caso é executada após uma partida:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
```

```
            return "Not found"
        case 418:
            return "I'm a teapot"
        case _:
            return "Something's wrong with the internet"
```

Se a função acima receber um `status` de 418, “I’m a teapot” será retornado. Se a função acima receber um `status` de 500, a instrução case com _ irá corresponder a um curinga, e “Something’s wrong with the Internet” é retornado. Observe o último bloco: o nome da variável, _, atua como um *curinga* e garante que o assunto sempre corresponderá. O uso de _ é opcional.

Você pode combinar vários literais em um único padrão usando `|` (“ou”):

```
case 401 | 403 | 404:
    return "Não permitido"
```

Comportamento sem o curinga

Se modificarmos o exemplo acima removendo o último bloco case, o exemplo se tornará:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 418:
            return "I'm a teapot"
```

Sem o uso de _ em uma instrução case, uma correspondência pode não existir. Se não houver correspondência, o comportamento é autônomo. Por exemplo, se o `status` de 500 for passado, ocorre um no-op.

Padrões com uma literal e variável

Os padrões podem parecer atribuições de desempacotamento e um padrão pode ser usado para vincular variáveis. Neste exemplo, um ponto de dados pode ser desempacotado em sua coordenada x e coordenada y:

```
# ponto é uma tupla (x, y)
match ponto:
    case (0, 0):
        print("Origem")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y):
        print(f"X={x}, Y={y}")
    case _:
        raise ValueError("Não é um ponto")
```

O primeiro padrão tem dois literais, `(0, 0)`, e pode ser considerado uma extensão do padrão literal mostrado acima. Os próximos dois padrões combinam um literal e uma variável, e a variável *vincula* um valor do assunto (`ponto`). O quarto padrão captura dois valores, o que o torna conceitualmente semelhante à atribuição de desempacotamento `(x, y) = ponto`.

Padrões e classes

Se estiver usando classes para estruturar seus dados, você pode usar como padrão o nome da classe seguido por uma lista de argumentos semelhante a um construtor. Este padrão tem a capacidade de capturar atributos da instância em variáveis:

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def location(point):
    match point:
        case Point(x=0, y=0):
           print("O ponto está na origem.")
```

```
        case Point(x=0, y=y):
           print(f "Y={y} e o ponto está no eixo y.")
        case Point(x=x, y=0):
           print(f "X={x} e o ponto está no eixo x.")
        case Point():
           print("O ponto está localizado em outro lugar no plar
        case _:
           print("Não é um ponto")
```

Padrões com parâmetros posicionais

Você pode usar parâmetros posicionais com algumas classes embutidas que fornecem uma ordem para seus atributos (por exemplo, classes de dados). Você também pode definir uma posição específica para atributos em padrões configurando o atributo especial `__match_args__` em suas classes. Se for definido como (“x”, “y”), os seguintes padrões são todos equivalentes (e todos ligam o atributo `y` à variável `var`):

```
Ponto(1, var)
Ponto(1, y=var)
Ponto(x=1, y=var)
Ponto(y=var, x=1)
```

Padrões aninhados

Os padrões podem ser aninhados arbitrariamente. Por exemplo, se nossos dados forem uma pequena lista de pontos, eles podem ser correspondidos assim:

```
match points:
    case []:
        print("Nenhum ponto na lista.")
    case [Point(0, 0)]:
        print("A origem é o único ponto na lista.")
    case [Point(x, y)]:
        print(f"Um único ponto {x}, {y} está na lista.")
    case [Point(0, y1), Point(0, y2)]:
        print(f"Dois pontos no eixo Y em {y1}, {y2} estão na lis
```

```
    case _:
        print("Algo diferente foi encontrado na lista.")
```

Padrões complexos e o curinga

Até este ponto, os exemplos usaram _ sozinho na última instrução case. Um curinga pode ser usado em padrões mais complexos, como `('error', code, _)`. Por exemplo:

```
match test_variable:
    case ('warning', code, 40):
        print("Foi recebido um alerta.")
    case ('error', code, _):
        print(f"Ocorreu um erro {code}.")
```

No caso acima, `test_variable` irá corresponder a (‘erro’, código, 100) e (‘erro’, código, 800).

Guarda

Podemos adicionar uma cláusula `if` a um padrão, conhecido como “guarda”. Se a guarda for falsa, `match` tenta o próximo bloco de caso. Observe que a captura de valor ocorre antes que a guarda seja avaliada:

```
match point:
    case Point(x, y) if x == y:
        print(f"O ponto está localizado em uma diagonal Y=X at {
    case Point(x, y):
        print(f"Ponto não está na diagonal.")
```

Outros recursos-chave

Vários outros recursos-chave:

- Assim como desempacotar atribuições, os padrões de tupla e lista têm exatamente o mesmo significado e realmente correspondem a sequências arbitrárias. Tecnicamente, o sujeito deve ser uma

sequência. Portanto, uma exceção importante é que padrões não correspondem a iteradores. Também evita um erro comum, sequência de padrões não correspondem a strings.

- Os padrões de sequência têm suporte a curingas: `[x, y, *rest]` e `(x, y, *rest)` funcionam de forma semelhante a curingas em desempacotamentos de atribuições. O nome depois de `*` também pode ser `_`, então `(x, y, *_)` corresponde a uma sequência de pelo menos dois itens sem ligar os itens restantes.

- Padrões de mapeamento: `{"bandwidth": b, "latency": l}` captura os valores `"bandwidth"` e `"latency"` de um dicionário. Diferente dos padrões de sequência, chaves extra são ignoradas. Um curinga `**rest` também é permitido. (Mas `**_` seria redundante, então não é permitido.)

- Subpadrões podem ser capturados usando a palavra reservada `as`

  ```
  case (Ponto(x1, y1), Ponto(x2, y2) as p2): ...
  ```

  Isso liga x1, y1, x2, y2 como você esperaria sem a cláusula `as` e p2 a todo o segundo item do sujeito.

- A maioria dos literais são comparados por igualdade. No entanto, os singletons `True`, `False` e `None` são comparados por identidade.

- Constantes nomeadas podem ser usadas em padrões. Essas constantes nomeadas devem ser nomes pontilhados para evitar que a constante seja interpretada como uma variável de captura:

  ```
  from enum import Enum
  class Color(Enum):
      RED = 0
      GREEN = 1
      BLUE = 2

  color = Color.GREEN
  match color:
      case Color.RED:
          print("Eu vejo vermelho!")
  ```

```
        case Color.GREEN:
            print("A grama é verde")
        case Color.BLUE:
            print("Eu azul de fome :(")
```

Para obter as especificações completas, consulte a **PEP 634** [https://peps.python.org/pep-0634/]. A motivação e o raciocínio estão na **PEP 635** [https://peps.python.org/pep-0635/], e um tutorial mais longo está na **PEP 636** [https://peps.python.org/pep-0636/].

`EncodingWarning` opcional e opção `encoding="locale"`

A codificação padrão de `TextIOWrapper` e `open()` depende da plataforma e da localidade. Como o UTF-8 é usado na maioria das plataformas Unix, omitir a opção `encoding` ao abrir arquivos UTF-8 (por exemplo, JSON, YAML, TOML, Markdown) é um bug muito comum. Por exemplo:

```
# BUG: mode "rb" ou encoding="utf-8" deve ser usado.
with open("data.json") as f:
    data = json.load(f)
```

Para encontrar este tipo de bug, uma `EncodingWarning` opcional é adicionada. É emitido quando `sys.flags.warn_default_encoding` é verdadeiro e a codificação padrão específica da localidade é usada.

A opção `-X warn_default_encoding` e `PYTHONWARNDEFAULTENCODING` são adicionadas para ativar o aviso.

Veja Codificação do texto para mais informações.

Novos recursos relacionados a dicas de tipo

Esta seção cobre as principais mudanças que afetam as dicas de tipo da **PEP 484** [https://peps.python.org/pep-0484/] e o módulo `typing`.

PEP 604: novo operador de união de tipo

Um novo operador de união de tipo foi introduzido, o que permite a sintaxe `X | Y`. Isso fornece uma maneira mais limpa de expressar "tanto o tipo X quanto o tipo Y" ao invés de usar **`typing.Union`**, especialmente em dicas de tipo.

Nas versões anteriores do Python, para aplicar uma dica de tipo para funções que aceitam argumentos de vários tipos, era usado **`typing.Union`**:

```
def square(number: Union[int, float]) -> Union[int, float]:
    return number ** 2
```

As dicas de tipo agora podem ser escritas de uma maneira mais sucinta:

```
def square(number: int | float) -> int | float:
    return number ** 2
```

Esta nova sintaxe também é aceita como o segundo argumento para **`isinstance()`** e **`issubclass()`**:

```
>>> isinstance(1, int | str)
True
```

Veja Tipo União e **PEP 604** [https://peps.python.org/pep-0604/] para mais detalhes.

(Contribuição de Maggie Moss e Philippe Prados em bpo-41428 [https://bugs.python.org/issue?@action=redirect&bpo=41428], com acréscimos por Yurii Karabas e Serhiy Storchaka em bpo-44490 [https://bugs.python.org/issue?@action=redirect&bpo=44490].)

PEP 612: variáveis de especificação de parâmetros

Duas novas opções para melhorar as informações fornecidas para verificadores de tipo estático para `Callable` da **PEP 484** [https://peps.python.org/pep-0484/] foram adicionadas ao módulo **`typing`**.

A primeira é a variável de especificação de parâmetro. Eles são usados para encaminhar os tipos de parâmetro de um chamável para outro chamável – um padrão comumente encontrado em funções de ordem superior e decoradores. Exemplos de uso podem ser encontrados em

`typing.ParamSpec`. Anteriormente, não havia uma maneira fácil de digitar a dependência de anotação de tipos de parâmetro de maneira tão precisa.

A segunda opção é o novo operador `Concatenate`. É usado em conjunto com variáveis de especificação de parâmetro para digitar anotar um chamável de ordem superior que adiciona ou remove parâmetros de outro chamável. Exemplos de uso podem ser encontrados em `typing.Concatenate`.

Veja `typing.Callable`, `typing.ParamSpec`, `typing.Concatenate`, `typing.ParamSpecArgs`, `typing.ParamSpecKwargs` e **PEP 612** [https://peps.python.org/pep-0612/] para mais detalhes.

(Contribuição de Ken Jin em bpo-41559 [https://bugs.python.org/issue?@action=redirect&bpo=41559], com pequenas melhorias por Jelle Zijlstra em bpo-43783 [https://bugs.python.org/issue?@action=redirect&bpo=43783]. PEP escrita por Mark Mendoza.)

PEP 613: TypeAlias

PEP 484 [https://peps.python.org/pep-0484/] introduziu o conceito de apelidos de tipo, exigindo apenas que fossem atribuições não anotadas de nível superior. Essa simplicidade às vezes tornava difícil para os verificadores de tipo distinguirem entre apelidos de tipo e atribuições comuns, especialmente quando referências diretas ou tipos inválidos estavam envolvidos. Compare:

```
StrCache = 'Cache[str]'  # um apelido de tipo
LOG_PREFIX = 'LOG[DEBUG]'  # uma constante do módulo
```

Agora o módulo `typing` tem um valor especial `TypeAlias` que permite declarar apelidos de tipo, mais explicitamente:

```
StrCache: TypeAlias = 'Cache[str]'  # um apelido de tipo
LOG_PREFIX = 'LOG[DEBUG]'  # uma constante do módulo
```

Veja **PEP 613** [https://peps.python.org/pep-0613/] para mais detalhes.

(Contribuição de Mikhail Golubev em bpo-41923 [https://bugs.python.org/issue?@action=redirect&bpo=41923].)

PEP 647: guardas de tipo definidas pelo usuário

`TypeGuard` foi adicionado ao módulo `typing` para anotar funções de guarda de tipo e melhorar informações fornecidas a verificadores de tipo estático durante um estreitamento de tipo. Para mais informações, veja a documentação do `TypeGuard` e a **PEP 647** [https://peps.python.org/pep-0647/].

(Contribuição de Ken Jin e Guido van Rossum em bpo-43766 [https://bugs.python.org/issue?@action=redirect&bpo=43766]. PEP escrita por Eric Traut.)

Outras mudanças na linguagem

- O tipo `int` tem um novo método `int.bit_count()`, retornando o número de unidades na expansão binária de um dado inteiro, também conhecido como contagem da população. (Contribuição de Niklas Fiekas em bpo-29882 [https://bugs.python.org/issue?@action=redirect&bpo=29882].)
- As visualizações retornadas por `dict.keys()`, `dict.values()` e `dict.items()` agora têm um atributo `mapping` que fornece um objeto `types.MappingProxyType` que envolve o dicionário original. (Contribuição de Dennis Sweeney em bpo-40890 [https://bugs.python.org/issue?@action=redirect&bpo=40890].)
- **PEP 618** [https://peps.python.org/pep-0618/]: A função `zip()` agora tem um sinalizador opcional `strict`, usado para exigir que todos os iteráveis tenham um comprimento igual.
- Funções embutidas e de extensão que recebem argumentos inteiros não aceitam mais `Decimal`, `Fraction` e outros objetos que podem ser convertidos em inteiros apenas com uma perda (por exemplo, tem o método `__int__()`, mas não tem o método `__index__()`). (Contribuição de Serhiy Storchaka em bpo-37999 [https://bugs.python.org/issue?@action=redirect&bpo=37999].)
- Se `object.__ipow__()` retorna `NotImplemented`, o operador vai corretamente recorrer ao `object.__pow__()` e `object.__rpow__()` como esperado. (Contribuição de Alex Shkop em bpo-38302 [https://bugs.python.org/issue?@action=redirect&bpo=38302].)

- Expressões de atribuição agora podem ser usadas sem parênteses dentro de literais de conjuntos e compreensões de conjuntos, bem como em índices de sequência (mas não em fatias).
- As funções têm um novo atributo `__builtins__` que é usado para procurar por símbolos embutidos quando uma função é executada, em vez de procurar em `__globals__['__builtins__']`. O atributo é inicializado a partir de `__globals__["__builtins__"]` se existir; do contrário, a partir dos embutidos atuais. (Contribuição de Mark Shannon em bpo-42990 [https://bugs.python.org/issue?@action=redirect&bpo=42990].)
- Duas novas funções embutidas – **`aiter()`** e **`anext()`** foram adicionadas para fornecer contrapartes assíncronas para **`iter()`** e **`next()`**, respectivamente. (Contribuição de Joshua Bronson, Daniel Pope e Justin Wang em bpo-31861 [https://bugs.python.org/issue?@action=redirect&bpo=31861].)
- Métodos estáticos (**`@staticmethod`**) e métodos de classe (**`@classmethod`**) agora herdam os atributos de método (`__module__`, `__name__`, `__qualname__`, `__doc__`, `__annotations__`) e têm um novo atributo `__wrapped__` . Além disso, métodos estáticos são agora chamáveis como funções regulares. (Contribuição de Victor Stinner em bpo-43682 [https://bugs.python.org/issue?@action=redirect&bpo=43682].)
- Anotações para alvos complexos (tudo além de alvos de nome simples, ou `simple name`, definidos pela **PEP 526** [https://peps.python.org/pep-0526/]) não causam mais nenhum efeito de tempo de execução com `from __future__ import annotations`. (Contribuição de Batuhan Taskaya em bpo-42737 [https://bugs.python.org/issue?@action=redirect&bpo=42737].)
- Objetos classe e módulo agora criam preguiçosamente dados vazios de anotações sob demanda. Os dicionários de anotações são armazenados em `__dict__` do objeto para compatibilidade com versões anteriores. Isso melhora as melhores práticas para trabalhar com `__annotations__`; para mais informações, veja Boas práticas para anotações. (Contribuição de Larry Hastings em bpo-43901 [https://bugs.python.org/issue?@action=redirect&bpo=43901].)
- Anotações consistindo em `yield`, `yield from`, `await` ou expressões nomeadas agora são proibidas em `from __future__ import annotations` por causa de seus efeitos colaterais. (Contribuição de

Batuhan Taskaya em bpo-42725 [https://bugs.python.org/issue?@action=redirect&bpo=42725].)

- O uso de variáveis não ligadas, `super()` e outras expressões que podem alterar o processamento da tabela de símbolos como anotações são agora processadas sem efeito sob `from __future__ import annotations`. (Contribuição de Batuhan Taskaya em bpo-42725 [https://bugs.python.org/issue?@action=redirect&bpo=42725].)
- Hashes de valores NaN de ambos os tipos **`float`** e **`decimal.Decimal`** agora dependem da identidade do objeto. Anteriormente, eles sempre hash para `0` mesmo que os valores NaN não sejam iguais uns aos outros. Isso causou um comportamento de tempo de execução potencialmente quadrático devido a colisões de hash excessivas ao criar dicionários e conjuntos contendo vários NaNs. (Contribuição de Raymond Hettinger em bpo-43475 [https://bugs.python.org/issue?@action=redirect&bpo=43475].)
- Uma **`SyntaxError`** (ao invés de uma **`NameError`**) será levantada ao excluir a constante **`__debug__`**. (Contribuição de Donghee Na em bpo-45000 [https://bugs.python.org/issue?@action=redirect&bpo=45000].)
- Exceções **`SyntaxError`** agora possuem atributos `end_lineno` e `end_offset`. Eles serão `None` se não forem determinados. (Contribuição de Pablo Galindo em bpo-43914 [https://bugs.python.org/issue?@action=redirect&bpo=43914].)

Novos módulos

- Nenhum.

Módulos melhorados

asyncio

Adiciona o método **`connect_accepted_socket()`** até então em falta. (Contribuição de Alex Grönholm em bpo-41332 [https://bugs.python.org/issue?@action=redirect&bpo=41332].)

argparse

A frase enganosa “argumentos opcionais” foi substituída por “opções” na ajuda do argparse. Alguns testes podem exigir adaptação se eles dependerem da correspondência de saída exata. (Contribuição de Raymond Hettinger em bpo-9694 [https://bugs.python.org/issue?@action=redirect&bpo=9694].)

array

O método `index()` de `array.array` agora possui os parâmetros *start* e *stop*. (Contribuição de Anders Lorentsen e Zackery Spytz em bpo-31956 [https://bugs.python.org/issue?@action=redirect&bpo=31956].)

asynchat, asyncore, smtpd

Esses módulos foram marcados como descontinuados em sua documentação de módulo desde o Python 3.6. Uma exceção `DeprecationWarning` em tempo de importação agora foi adicionada a todos esses três módulos.

base64

Adiciona `base64.b32hexencode()` e `base64.b32hexdecode()` para dar suporte a Codificação Base32 com alfabeto hexa estendido.

bdb

Adiciona `clearBreakpoints()` para redefinir todos os pontos de interrupção definidos. (Contribuição de Irit Katriel em bpo-24160 [https://bugs.python.org/issue?@action=redirect&bpo=24160].)

bisect

Adicionada a possibilidade de fornecer uma função *key* para as APIs no módulo `bisect`. (Contribuição de Raymond Hettinger em bpo-4356 [https://bugs.python.org/issue?@action=redirect&bpo=4356].)

codecs

Adiciona uma função **`codecs.unregister()`** para cancelar um registro de uma função de pesquisa de codecs. (Contribuição de Hai Shi em bpo-41842 [https://bugs.python.org/issue?@action=redirect&bpo=41842].)

collections.abc

Os `__args__` do genérico parametrizado para **`collections.abc.Callable`** agora são consistentes com **`typing.Callable`**. A classe genérica **`collections.abc.Callable`** agora achata parâmetros de tipo, de forma semelhante ao que **`typing.Callable`** atualmente faz. Isso significa que `collections.abc.Callable[[int, str], str]` terá `__args__` de `(int, str, str)`; anteriormente, isso era `([int, str], str)`. Para permitir esta alteração, agora é possível criar subclasse de **`types.GenericAlias`** e uma subclasse será retornada ao fazer um subscript do tipo **`collections.abc.Callable`**. Observe que uma **`TypeError`** pode ser levantada para formas inválidas de parametrizar **`collections.abc.Callable`** que podem ter passado silenciosamente no Python 3.9. (Contribuição de Ken Jin em bpo-42195 [https://bugs.python.org/issue?@action=redirect&bpo=42195].)

contextlib

Adiciona um gerenciador de contexto **`contextlib.aclosing()`** para fechar com segurança geradores assíncronos e objetos que representam recursos liberados de forma assíncrona. (Contribuição de Joongi Kim e John Belmonte em bpo-41229 [https://bugs.python.org/issue?@action=redirect&bpo=41229].)

Adiciona suporte a gerenciador de contexto assíncrono a **`contextlib.nullcontext()`**. (Contribuição de Tom Gringauz em bpo-41543 [https://bugs.python.org/issue?@action=redirect&bpo=41543].)

Adiciona **`AsyncContextDecorator`**, para dar suporte ao uso de gerenciadores de contexto assíncronos como decoradores.

curses

As funções de cores estendidas adicionadas no ncurses 6.1 serão usadas transparentemente por `curses.color_content()`, `curses.init_color()`, `curses.init_pair()` e `curses.pair_content()`. Uma nova função, `curses.has_extended_color_support()`, indica se o suporte a cores estendidas é fornecido pela biblioteca ncurses subjacente. (Contribuição de Jeffrey Kintscher e Hans Petter Jansson em bpo-36982 [https://bugs.python.org/issue?@action=redirect&bpo=36982].)

As constantes `BUTTON5_*` agora são expostas no módulo `curses` se forem fornecidas pela biblioteca curses subjacente. (Contribuição de Zackery Spytz em bpo-39273 [https://bugs.python.org/issue?@action=redirect&bpo=39273].)

dataclasses

__slots__

Adicionado o parâmetro `slots` no decorador `@dataclasses.dataclass`. (Contribuição de Yurii Karabas em bpo-42269 [https://bugs.python.org/issue?@action=redirect&bpo=42269])

Campos somente-nomeados

dataclasses agora oferece suporte a campos que são somente-nomeados no método __init__ gerado. Há várias maneiras de especificar campos somente-nomeados.

Você pode dizer que todos os campos são somente-nomeados:

```
from dataclasses import dataclass

@dataclass(kw_only=True)
class Birthday:
    name: str
    birthday: datetime.date
```

Ambos `name` e `birthday` são parâmetros somente-nomeados para o método __init__ gerado.

Você pode especificar somente-nomeado por campo:

```
from dataclasses import dataclass, field

@dataclass
class Birthday:
    name: str
    birthday: datetime.date = field(kw_only=True)
```

Aqui apenas `birthday` é somente-nomeado. Se você definir `kw_only` em campos individuais, esteja ciente de que existem regras sobre a reordenação de campos devido a campos somente-nomeados que precisam seguir campos não somente-nomeados. Consulte a documentação completa de dataclasses para obter detalhes.

Você também pode especificar que todos os campos a partir de um marcador KW_ONLY sejam somente-nomeados. Este provavelmente será o uso mais comum:

```
from dataclasses import dataclass, KW_ONLY

@dataclass
class Point:
    x: float
    y: float
    _: KW_ONLY
    z: float = 0.0
    t: float = 0.0
```

Aqui, `z` e `t` são parâmetros somente-nomeados, enquanto `x` e `y` não são. (Contribuição de Eric V. Smith em bpo-43532 [https://bugs.python.org/issue?@action=redirect&bpo=43532].)

distutils

Todo o pacote `distutils` foi descontinuado, para ser removido no Python 3.12. Sua funcionalidade para especificar compilações de pacote já foi completamente substituída por pacotes de terceiros `setuptools` e `packaging`, e a maioria das outras APIs comumente usadas estão disponíveis em outro lugar na biblioteca padrão (como **`platform`**, **`shutil`**,

`subprocess` ou `sysconfig`). Não há planos para migrar qualquer outra funcionalidade de `distutils`, e aplicativos que estão usando outras funções devem planejar fazer cópias privadas do código. Consulte **PEP 632** [https://peps.python.org/pep-0632/] para discussão.

O comando descontinuado `bdist_wininst` no Python 3.8 foi removido. O comando `bdist_wheel` agora é recomendado para distribuir pacotes binários no Windows. (Contribuição de Victor Stinner em bpo-42802 [https://bugs.python.org/issue?@action=redirect&bpo=42802].)

doctest

Quando um módulo não define `__loader__`, recorre a `__spec__.loader`. (Contribuição de Brett Cannon em bpo-42133 [https://bugs.python.org/issue?@action=redirect&bpo=42133].)

encodings

`encodings.normalize_encoding()` agora ignora caracteres não-ASCII. (Contribuição de Hai Shi em bpo-39337 [https://bugs.python.org/issue?@action=redirect&bpo=39337].)

enum

`__repr__()` do `Enum` agora retorna `enum_name.member_name` e `__str__()` agora retorna `member_name`. Os enums da stdlib disponíveis como constantes de módulo têm `repr()` de `module_name.member_name`. (Contribuição de Ethan Furman em bpo-40066 [https://bugs.python.org/issue?@action=redirect&bpo=40066].)

Adiciona `enum.StrEnum` para enums onde todos os membros são strings. (Contribuição de Ethan Furman em bpo-41816 [https://bugs.python.org/issue?@action=redirect&bpo=41816].)

fileinput

Adiciona os parâmetros *encoding* e *errors* a `fileinput.input()` e `fileinput.FileInput`. (Contribuição de Inada Naoki em bpo-43712 [https://bugs.python.org/issue?@action=redirect&bpo=43712].)

`fileinput.hook_compressed()` agora retorna um objeto `TextIOWrapper` quando *mode* é “r” e o arquivo está compactado, como arquivos descompactados. (Contribuição de Inada Naoki em bpo-5758 [https://bugs.python.org/issue?@action=redirect&bpo=5758].)

faulthandler

O módulo `faulthandler` agora detecta se um erro fatal ocorre durante a coleta do coletor de lixo. (Contribuição de Victor Stinner em bpo-44466 [https://bugs.python.org/issue?@action=redirect&bpo=44466].)

gc

Adiciona ganchos de auditoria para `gc.get_objects()`, `gc.get_referrers()` e `gc.get_referents()`. (Contribuição de Pablo Galindo em bpo-43439 [https://bugs.python.org/issue?@action=redirect&bpo=43439].)

glob

Adiciona os parâmetros *root_dir* e *dir_fd* em `glob()` e `iglob()`, o que permite especificar o diretório raiz para a pesquisa. (Contribuição de Serhiy Storchaka em bpo-38144 [https://bugs.python.org/issue?@action=redirect&bpo=38144].)

hashlib

O módulo hashlib requer OpenSSL 1.1.1 ou mais recente. (Contribuição de Christian Heimes em **PEP 644** [https://peps.python.org/pep-0644/] e bpo-43669 [https://bugs.python.org/issue?@action=redirect&bpo=43669].)

O módulo hashlib tem suporte preliminar a OpenSSL 3.0.0. (Contribuição de Christian Heimes em bpo-38820 [https://bugs.python.org/issue?@action=redirect&bpo=38820] e outras issues.)

A alternativa puramente Python de `pbkdf2_hmac()` foi descontinuada. No futuro, o PBKDF2-HMAC só estará disponível quando o Python for desenvolvido com suporte a OpenSSL. (Contribuição de Christian Heimes em bpo-43880 [https://bugs.python.org/issue?@action=redirect&bpo=43880].)

hmac

O módulo hmac agora usa a implementação HMAC do OpenSSL internamente. (Contribuição de Christian Heimes em bpo-40645 [https://bugs.python.org/issue?@action=redirect&bpo=40645].)

IDLE e idlelib

Faz o IDLE invocar `sys.excepthook()` (quando iniciado sem '-n'). Ganchos de usuário eram ignorados anteriormente. (Contribuição de Ken Hilton em bpo-43008 [https://bugs.python.org/issue?@action=redirect&bpo=43008].)

Reorganiza a caixa de diálogo de configurações. Divide a aba General nas abas Windows e Shell/Ed. Move as fontes de ajuda, que estendem o menu Help, para a aba Extensions. Abre espaço para novas opções e encurta a caixa de diálogo. O último faz com que o diálogo se ajuste melhor a telas pequenas. (Contribuição de Terry Jan Reedy em bpo-40468 [https://bugs.python.org/issue?@action=redirect&bpo=40468].) Move a configuração do espaço de recuo da aba Font para a nova aba Windows. (Contribuição de Mark Roseman e Terry Jan Reedy em bpo-33962 [https://bugs.python.org/issue?@action=redirect&bpo=33962].)

As alterações acima foram portadas para uma versão de manutenção 3.9.

Adiciona uma barra lateral ao console. Move o prompt principal ('>>>') para a barra lateral. Adiciona prompts secundários ('…') à barra lateral. Clicar com o botão esquerdo e opcionalmente arrastar seleciona uma ou mais linhas de texto, como na barra lateral do número da linha do editor. Clicar com o botão direito após selecionar as linhas de texto exibe um menu de contexto com "copy with prompts". Isso compacta os prompts da barra lateral com linhas do texto selecionado. Esta opção também aparece no

menu de contexto para o texto. (Contribuição de Tal Einat em bpo-37903 [https://bugs.python.org/issue?@action=redirect&bpo=37903].)

Use espaços em vez de tabulações para indentar o código interativo. Isso faz com que as entradas de código interativo “pareçam corretas”. Tornar isso viável foi a principal motivação para adicionar a barra lateral do console. Contribuição de Terry Jan Reedy em bpo-37892 [https://bugs.python.org/issue?@action=redirect&bpo=37892].)

Realça as novas palavra reservada suave **`match`**, **`case`** e `_` em instruções de correspondência de padrões. No entanto, este realce não é perfeito e estará incorreto em alguns casos raros, incluindo alguns `_` em padrões de `case`. (Contribuição de Tal Einat em bpo-44010 [https://bugs.python.org/issue?@action=redirect&bpo=44010].)

Novo nas versões de manutenção 3.10.

Aplica realce de sintaxe em arquivos `.pyi`. (Contribuição de Alex Waygood e Terry Jan Reedy em bpo-45447 [https://bugs.python.org/issue?@action=redirect&bpo=45447].)

Inclui prompts ao salvar o console com entradas e saídas. (Contribuição de Terry Jan Reedy em gh-95191 [https://github.com/python/cpython/issues/95191].)

importlib.metadata

Paridade de recursos com `importlib_metadata` 4.6 (histórico [https://importlib-metadata.readthedocs.io/en/latest/history.html]).

Pontos de entrada importlib.metadata agora oferecem uma experiência melhor para selecionar pontos de entrada por grupo e nome através do novo importlib.metadata.EntryPoints. Consulte a Nota de Compatibilidade nos documentos para obter mais informações sobre a descontinuação e uso.

Adicionada importlib.metadata.packages_distributions() para resolver módulos e pacotes Python de alto nível com suas importlib.metadata.Distribution.

inspect

Quando um módulo não define `__loader__`, recorre a `__spec__.loader`. (Contribuição de Brett Cannon em bpo-42133 [https://bugs.python.org/issue?@action=redirect&bpo=42133].)

Adiciona **`inspect.get_annotations()`**, que calcula com segurança as anotações definidas em um objeto. Ele contorna as peculiaridades de acessar as anotações em vários tipos de objetos e faz poucas suposições sobre o objeto que examina. **`inspect.get_annotations()`** também pode desfazer a string corretamente de anotações em string. **`inspect.get_annotations()`** agora é considerada a melhor prática para acessar o dict de anotações definido em qualquer objeto Python; para mais informações sobre as melhores práticas para trabalhar com anotações, consulte Boas práticas para anotações. Da mesma forma, **`inspect.signature()`**, **`inspect.Signature.from_callable()`** e **`inspect.Signature.from_function()`** agora chamam **`inspect.get_annotations()`** para recuperar anotações. Isso significa que **`inspect.signature()`** e **`inspect.Signature.from_callable()`** agora também podem remover a string de anotações em string. (Contribuição de Larry Hastings em bpo-43817 [https://bugs.python.org/issue?@action=redirect&bpo=43817].)

itertools

Adiciona **`itertools.pairwise()`**. (Contribuição de Raymond Hettinger em bpo-38200 [https://bugs.python.org/issue?@action=redirect&bpo=38200].)

linecache

Quando um módulo não define `__loader__`, recorre a `__spec__.loader`. (Contribuição de Brett Cannon em bpo-42133 [https://bugs.python.org/issue?@action=redirect&bpo=42133].)

os

Adiciona suporte a `os.cpu_count()` para RTOS de VxWorks. (Contribuição de Peixing Xin em bpo-41440 [https://bugs.python.org/issue?@action=redirect&bpo=41440].)

Adiciona uma nova função `os.eventfd()` e auxiliares relacionados para envolver a chamada de sistema `eventfd2` no Linux. (Contribuição de Christian Heimes em bpo-41001 [https://bugs.python.org/issue?@action=redirect&bpo=41001].)

Adiciona `os.splice()` que permite mover dados entre dois descritores de arquivo sem copiar entre o espaço de endereço do kernel e o espaço de endereço do usuário, onde um dos descritores de arquivo deve se referir a um encadeamento (pipe). (Contribuição de Pablo Galindo em bpo-41625 [https://bugs.python.org/issue?@action=redirect&bpo=41625].)

Adiciona `O_EVTONLY`, `O_FSYNC`, `O_SYMLINK` e `O_NOFOLLOW_ANY` para macOS. (Contribuição de Donghee Na em bpo-43106 [https://bugs.python.org/issue?@action=redirect&bpo=43106].)

os.path

`os.path.realpath()` agora aceita um argumento somente-nomeado *strict*. Quando definido como `True`, a exceção `OSError` é levantada se um caminho não existe ou um loop de link simbólico é encontrado. (Contribuição de Barney Gale em bpo-43757 [https://bugs.python.org/issue?@action=redirect&bpo=43757].)

pathlib

Adiciona suporte a fatiamento a `PurePath.parents`. (Contribuição de Joshua Cannon em bpo-35498 [https://bugs.python.org/issue?@action=redirect&bpo=35498].)

Adiciona suporte a indexação negativa a `PurePath.parents`. (Contribuição de Yaroslav Pankovych em bpo-21041 [https://bugs.python.org/issue?@action=redirect&bpo=21041].)

Adiciona o método `Path.hardlink_to` que substitui `link_to()`. O novo método tem a mesma ordem de argumentos que `symlink_to()`. (Contribuição de Barney Gale em bpo-39950 [https://bugs.python.org/issue?@action=redirect&bpo=39950].)

`pathlib.Path.stat()` e `chmod()` agora aceita um argumento somente-nomeado *follow_symlinks* para consistência com as funções correspondentes no módulo `os`. (Contribuição de Barney Gale em bpo-39906 [https://bugs.python.org/issue?@action=redirect&bpo=39906].)

platform

Adiciona `platform.freedesktop_os_release()` para obter a identificação do sistema operacional a partir do arquivo padrão os-release do freedesktop.org [https://www.freedesktop.org/software/systemd/man/os-release.html]. (Contribuição de Christian Heimes em bpo-28468 [https://bugs.python.org/issue?@action=redirect&bpo=28468].)

pprint

`pprint.pprint()` agora aceita um novo argumento nomeado `underscore_numbers`. (Contribuição de sblondon em bpo-42914 [https://bugs.python.org/issue?@action=redirect&bpo=42914].)

`pprint` agora pode fazer impressão bonita de instâncias de `dataclasses.dataclass`. (Contribuição de Lewis Gaul em bpo-43080 [https://bugs.python.org/issue?@action=redirect&bpo=43080].)

py_compile

Adiciona a opção `--quiet` à interface de linha de comando de `py_compile`. (Contribuição de Gregory Schevchenko em bpo-38731 [https://bugs.python.org/issue?@action=redirect&bpo=38731].)

pyclbr

Adiciona um atributo `end_lineno` aos objetos `Function` e `Class` na árvore retornada por **`pyclbr.readmodule()`** e **`pyclbr.readmodule_ex()`**. Isso corresponde ao `lineno` (início) existente. (Contribuição de Aviral Srivastava em bpo-38307 [https://bugs.python.org/issue?@action=redirect&bpo=38307].)

shelve

O módulo **`shelve`** agora usa **`pickle.DEFAULT_PROTOCOL`** por padrão em vez do protocolo `3` do **`pickle`** ao criar "shelves". (Contribuição de Zackery Spytz em bpo-34204 [https://bugs.python.org/issue?@action=redirect&bpo=34204].)

statistics

Adiciona **`covariance()`**, **`correlation()`** do Pearson, e funções simples **`linear_regression()`**. (Contribuição de Tymoteusz Wołodźko em bpo-38490 [https://bugs.python.org/issue?@action=redirect&bpo=38490].)

site

Quando um módulo não define `__loader__`, recorre a `__spec__.loader`. (Contribuição de Brett Cannon em bpo-42133 [https://bugs.python.org/issue?@action=redirect&bpo=42133].)

socket

A exceção **`socket.timeout`** é agora um apelido de **`TimeoutError`**. (Contribuição de Christian Heimes em bpo-42413 [https://bugs.python.org/issue?@action=redirect&bpo=42413].)

Adiciona a opção de criar soquetes MPTCP com `IPPROTO_MPTCP` (Contribuição de Rui Cunha em bpo-43571 [https://bugs.python.org/issue?@action=redirect&bpo=43571].)

Adiciona a opção `IP_RECVTOS` para receber o tipo do serviço (ToS) ou campos DSCP/ECN (Contribuição de Georg Sauthoff em bpo-44077 [https://bugs.python.org/issue?@action=redirect&bpo=44077].)

ssl

O módulo ssl requer OpenSSL 1.1.1 ou mais recente. (Contribuição de Christian Heimes em **PEP 644** [https://peps.python.org/pep-0644/] e bpo-43669 [https://bugs.python.org/issue?@action=redirect&bpo=43669].)

O módulo ssl tem um suporte preliminar para OpenSSL 3.0.0 e a nova opção `OP_IGNORE_UNEXPECTED_EOF`. (Contribuição de Christian Heimes em bpo-38820 [https://bugs.python.org/issue?@action=redirect&bpo=38820], bpo-43794 [https://bugs.python.org/issue?@action=redirect&bpo=43794], bpo-43788 [https://bugs.python.org/issue?@action=redirect&bpo=43788], bpo-43791 [https://bugs.python.org/issue?@action=redirect&bpo=43791], bpo-43799 [https://bugs.python.org/issue?@action=redirect&bpo=43799], bpo-43920 [https://bugs.python.org/issue?@action=redirect&bpo=43920], bpo-43789 [https://bugs.python.org/issue?@action=redirect&bpo=43789] e bpo-43811 [https://bugs.python.org/issue?@action=redirect&bpo=43811].)

Função descontinuada e o uso de constantes descontinuadas agora resultam em uma `DeprecationWarning`. `ssl.SSLContext.options` tem `OP_NO_SSLv2` e `OP_NO_SSLv3` definidos por padrão e, portanto, não consegue avisar sobre a definição do sinalizador novamente. A seção de descontinuidade tem uma lista de recursos descontinuados. (Contribuição de Christian Heimes em bpo-43880 [https://bugs.python.org/issue?@action=redirect&bpo=43880].)

O módulo ssl agora tem configurações padrão mais seguras. Cifras sem *forward secrecy* ou SHA-1 MAC são desabilitadas por padrão. O nível de segurança 2 proíbe chaves fracas RSA, DH e ECC com menos de 112 bits de segurança. `SSLContext` assume como padrão a versão mínima do protocolo TLS 1.2. As configurações são baseadas na pesquisa de Hynek Schlawack. (Contribuição de Christian Heimes em bpo-43998 [https://bugs.python.org/issue?@action=redirect&bpo=43998].)

Os protocolos descontinuados SSL 3.0, TLS 1.0 e TLS 1.1 não são mais oficialmente suportados. Python não os bloqueia ativamente. No entanto, as opções de compilação do OpenSSL, configurações de distro, patches de fornecedores e suítes de criptografia podem impedir um handshake bem-sucedido.

Adiciona um parâmetro *timeout* à função `ssl.get_server_certificate()`. (Contribuição de Zackery Spytz em bpo-31870 [https://bugs.python.org/issue?@action=redirect&bpo=31870].)

O módulo ssl usa tipos de heap e inicialização multifásica. (Contribuição de Christian Heimes em bpo-42333 [https://bugs.python.org/issue?@action=redirect&bpo=42333].)

Uma nova sinalização de verificação `VERIFY_X509_PARTIAL_CHAIN` foi adicionada. (Contribuição de l0x em bpo-40849 [https://bugs.python.org/issue?@action=redirect&bpo=40849].)

sqlite3

Adiciona eventos de auditoria para `connect()`, `enable_load_extension()` e `load_extension()`. (Contribuição de Erlend E. Aasland em bpo-43762 [https://bugs.python.org/issue?@action=redirect&bpo=43762].)

sys

Adiciona o atributo `sys.orig_argv`: a lista de argumentos de linha de comando originais passada para o executável Python. (Contribuição de Victor Stinner em bpo-23427 [https://bugs.python.org/issue?@action=redirect&bpo=23427].)

Adiciona `sys.stdlib_module_names`, contendo a lista de nomes de módulos da biblioteca padrão. (Contribuição de Victor Stinner em bpo-42955 [https://bugs.python.org/issue?@action=redirect&bpo=42955].)

_thread

`_thread.interrupt_main()` agora aceita um número de sinal opcional para simular (o padrão ainda é `signal.SIGINT`). (Contribuição de Antoine Pitrou em bpo-43356 [https://bugs.python.org/issue?@action=redirect&bpo=43356].)

threading

Adiciona `threading.gettrace()` e `threading.getprofile()` para obter as funções definidas por `threading.settrace()` e `threading.setprofile()`, respectivamente. (Contribuição de Mario Corchero em bpo-42251 [https://bugs.python.org/issue?@action=redirect&bpo=42251].)

Adiciona `threading.__excepthook__` para permitir obtenção do valor original de `threading.excepthook()` no caso dele estar definido com um valor quebrado ou diferente. (Contribuição de Mario Corchero em bpo-42308 [https://bugs.python.org/issue?@action=redirect&bpo=42308].)

traceback

As funções `format_exception()`, `format_exception_only()` e `print_exception()` podem agora receber um objeto exceção como um argumento somente-posicional. (Contribuição de Zackery Spytz e Matthias Bussonnier em bpo-26389 [https://bugs.python.org/issue?@action=redirect&bpo=26389].)

types

Reintroduz as classes `types.EllipsisType`, `types.NoneType` e `types.NotImplementedType`, fornecendo um novo conjunto de tipos prontamente interpretáveis pelos verificadores de tipo. (Contribuição de Bas van Beek em bpo-41810 [https://bugs.python.org/issue?@action=redirect&bpo=41810].)

typing

Para alterações principais, veja Novos recursos relacionados a dicas de tipo.

O comportamento de `typing.Literal` foi alterado para ficar em conformidade com a **PEP 586** [https://peps.python.org/pep-0586/] e para corresponder ao comportamento de verificadores de tipo estático especificados na PEP.

1. `Literal` agora elimina a duplicação de parâmetros.

2. Comparações de igualdade entre objetos `Literal` agora são independentes da ordem.

3. Comparações de `Literal` agora respeitam os tipos. Por exemplo, `Literal[0] == Literal[False]` avaliava anteriormente como `True`. Agora é `False`. Para oferecer suporte a essa mudança, o cache de tipo usado internamente agora oferece suporte a tipos de diferenciação.

4. Objetos `Literal` agora irão levantar uma exceção **`TypeError`** durante as comparações de igualdade se algum de seus parâmetros não for hasheáveis. Observe que declarar `Literal` com parâmetros inalteráveis não acusará um erro:

```
>>> from typing import Literal
>>> Literal[{0}]
>>> Literal[{0}] == Literal[{False}]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'set'
```

(Contribuição de Yurii Karabas em bpo-42345 [https://bugs.python.org/issue?@action=redirect&bpo=42345].)

Adiciona uma nova função **`typing.is_typeddict()`** para fazer introspecção se uma anotação for uma **`typing.TypedDict`**. (Contribuição de Patrick Reader em bpo-41792 [https://bugs.python.org/issue?@action=redirect&bpo=41792].)

Subclasses de `typing.Protocol` que apenas têm variáveis de dados declaradas agora irão levantar um `TypeError` quando verificadas com `isinstance` a menos que sejam decoradas com **`@~typing.runtime_checkable`**. Anteriormente, essas verificações eram aprovadas silenciosamente. Os usuários devem decorar suas subclasses com o decorador **`@runtime_checkable`** se quiserem protocolos de tempo de execução. (Contribuição de Yurii Karabas em bpo-38908 [https://bugs.python.org/issue?@action=redirect&bpo=38908].)

A importação dos submódulos `typing.io` e `typing.re` agora emitirá **`DeprecationWarning`**. Esses submódulos fora descontinuados desde o Python 3.8 e serão removidos em uma versão futura do Python. Qualquer coisa pertencente a esses submódulos deve ser importada diretamente de

`typing`. (Contribuição de Sebastian Rittau em bpo-38291 [https://bugs.python.org/issue?@action=redirect&bpo=38291].)

unittest

Adiciona novo método new `assertNoLogs()` para complementar o existente `assertLogs()`. (Contribuição de Kit Yan Choi em bpo-39385 [https://bugs.python.org/issue?@action=redirect&bpo=39385].)

urllib.parse

Versões do Python anteriores ao Python 3.10 permitiam o uso de `;` e `&` como separadores de parâmetros de consulta em `urllib.parse.parse_qs()` e `urllib.parse.parse_qsl()`. Devido a questões de segurança e em conformidade com as recomendações mais recentes do W3C, isso foi alterado para permitir apenas uma única chave separadora, com `&` como padrão. Esta mudança também afeta **`cgi.parse()`** e **`cgi.parse_multipart()`** já que elas usam as funções afetadas internamente. Para obter mais detalhes, consulte a respectiva documentação. (Contribuição de Adam Goldschmidt, Senthil Kumaran e Ken Jin em bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967].)

A presença de caracteres de nova linha ou tab em partes de um URL permite algumas formas de ataques. Seguindo a especificação WHATWG que atualiza **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], nova linha ASCII `\n`, `\r` e os caracteres de tabulação `\t` são retirados da URL pelo analisador sintático em `urllib.parse` impedindo tais ataques. Os caracteres de remoção são controlados por uma nova variável de nível de módulo `urllib.parse._UNSAFE_URL_BYTES_TO_REMOVE`. (Veja gh-88048 [https://github.com/python/cpython/issues/88048])

xml

Adiciona uma classe `LexicalHandler` ao módulo `xml.sax.handler`. (Contribuição de Jonathan Gossage e Zackery Spytz em bpo-35018 [https://bugs.python.org/issue?@action=redirect&bpo=35018].)

zipimport

Adiciona métodos relacionados à **PEP 451** [https://peps.python.org/pep-0451/]: `find_spec()`, `zipimport.zipimporter.create_module()` e `zipimport.zipimporter.exec_module()`. (Contribuição de Brett Cannon em bpo-42131 [https://bugs.python.org/issue?@action=redirect&bpo=42131].)

Adiciona o método `invalidate_caches()`. (Contribuição de Desmond Cheong em bpo-14678 [https://bugs.python.org/issue?@action=redirect&bpo=14678].)

Otimizações

- Os construtores `str()`, `bytes()` e `bytearray()` estão agora mais rápido (cerca de 30–40% para objetos pequenos). (Contribuição de Serhiy Storchaka em bpo-41334 [https://bugs.python.org/issue?@action=redirect&bpo=41334].)
- O módulo `runpy` agora importa menos módulos. O tempo de inicialização do comando `python3 -m module-name` é 1,4 vezes mais rápido em média. No Linux, `python3 -I -m module-name` importa 69 módulos no Python 3.9, sendo que importa apenas 51 módulos (-18) no Python 3.10. (Contribuição de Victor Stinner em bpo-41006 [https://bugs.python.org/issue?@action=redirect&bpo=41006] e bpo-41718 [https://bugs.python.org/issue?@action=redirect&bpo=41718].)
- A instrução `LOAD_ATTR` agora usa o novo mecanismo “cache por opcode”. É cerca de 36% mais rápido agora para atributos regulares e 44% mais rápido para slots. (Contribuição de Pablo Galindo e Yury Selivanov em bpo-42093 [https://bugs.python.org/issue?@action=redirect&bpo=42093] e Guido van Rossum em bpo-42927 [https://bugs.python.org/issue?@action=redirect&bpo=42927], com base em ideias implementadas originalmente em PyPy e MicroPython.)
- Ao construir o Python com `--enable-optimizations` agora, `-fno-semantic-interposition` é adicionado à linha de compilação e vinculação. Isso acelera as compilações do interpretador Python criado com `--enable-shared` com `gcc` em até 30%. Consulte este artigo [https://developers.redhat.com/blog/2020/06/25/red-hat-enterprise-linux-8-2-brings-faster-python-3-8-run-speeds/] para mais detalhes. (Contribuição de Victor Stinner

e Pablo Galindo em bpo-38980 [https://bugs.python.org/issue?@action=redirect&bpo=38980].)

- Usa um novo código de gerenciamento de buffer de saída para os módulos **`bz2`** / **`lzma`** / **`zlib`** e adiciona a função `.readall()` à classe `_compression.DecompressReader`. A descompressão bz2 agora é 1,09x ~ 1.17x mais rápida, a descompressão lzma 1.20x ~ 1.32x mais rápida, `GzipFile.read(-1)` 1,11x ~ 1.18x mais rápida. (Contribuição de Ma Lin, revisada por Gregory P. Smith, em bpo-41486 [https://bugs.python.org/issue?@action=redirect&bpo=41486])
- Ao usar anotações em strings, dicts de anotações para funções não são mais criados quando a função é criada. Em vez disso, eles são armazenados como uma tupla de strings, e o objeto função converte lentamente isso no dict de anotações sob demanda. Essa otimização reduz pela metade o tempo de CPU necessário para definir uma função anotada. (Contribuição de Yurii Karabas e Inada Naoki em bpo-42202 [https://bugs.python.org/issue?@action=redirect&bpo=42202].)
- Funções de pesquisa de substring como `str1 in str2` e `str2.find(str1)` agora às vezes usam o algoritmo de pesquisa de string "Two-Way" de Crochemore & Perrin para evitar comportamento quadrático em strings longas. (Contribuição de Dennis Sweeney em bpo-41972 [https://bugs.python.org/issue?@action=redirect&bpo=41972])
- Adiciona micro-otimizações a `_PyType_Lookup()` para melhorar o desempenho de pesquisa de cache de atributo de tipo no caso comum de acessos de cache. Isso torna o interpretador 1,04 vezes mais rápido, em média. (Contribuição de Dino Viehland em bpo-43452 [https://bugs.python.org/issue?@action=redirect&bpo=43452].)
- As seguintes funções embutidas agora oferecem suporte a uma convenção de chamada de vectorcalls mais rápidos da **PEP 590** [https://peps.python.org/pep-0590/]: **`map()`**, **`filter()`**, **`reversed()`**, **`bool()`** e **`float()`**. (Contribuição de Donghee Na e Jeroen Demeyer em bpo-43575 [https://bugs.python.org/issue?@action=redirect&bpo=43575], bpo-43287 [https://bugs.python.org/issue?@action=redirect&bpo=43287], bpo-41922 [https://bugs.python.org/issue?@action=redirect&bpo=41922], bpo-41873 [https://bugs.python.org/issue?@action=redirect&bpo=41873] e bpo-41870 [https://bugs.python.org/issue?@action=redirect&bpo=41870].)
- O desempenho de **`BZ2File`** foi melhorado removendo o `RLock` interno. Isso torna **`BZ2File`** inseguro para threads em face a vários leitores ou

gravadores simultâneos, da mesma forma que suas classes equivalentes em **gzip** e **lzma** têm sido. (Contribuição de Inada Naoki em bpo-43785 [https://bugs.python.org/issue?@action=redirect&bpo=43785].)

Descontinuados

- Atualmente Python aceita literais numéricos imediatamente seguidos por palavras-chave, por exemplo `0in x`, `1or x`, `0if 1else 2`. Permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). A partir desta versão, um aviso de descontinuidade é levantado se o literal numérico for seguido imediatamente por uma das palavras-chave **and**, **else**, **for**, **if**, **in**, **is** e **or**. Em versões futuras, ele será alterado para aviso de sintaxe e, finalmente, para erro de sintaxe. (Contribuição de Serhiy Storchaka em bpo-43833 [https://bugs.python.org/issue?@action=redirect&bpo=43833].)

- A partir desta versão, haverá um esforço concentrado para começar a limpar a semântica de importação antiga que foi mantida para compatibilidade com Python 2.7. Especificamente, **find_loader()**/**find_module()** (substituídos por **find_spec()**), `importlib.abc.Loader.load_module` (substituído por **exec_module()**), **module_repr()** (que o sistema de importação cuida para você), o atributo `__package__` (substituído por `__spec__.parent`), o atributo `__loader__` (substituído por `__spec__.loader`), e o atributo `__cached__` (substituído por `__spec__.cached`) será removido lentamente (assim como outras classes e métodos em **importlib**). **ImportWarning** e/ou **DeprecationWarning** será levantada conforme apropriado para ajudar a identificar o código que precisa ser atualizado durante esta transição.

- Todo o espaço de nomes de `distutils` foi descontinuado, para ser removido no Python 3.12. Consulte a seção de alterações do módulo para mais informações.

- Argumentos não inteiros para **random.randrange()** foram descontinuados. A exceção **ValueError** foi descontinuada em favor de

uma exceção `TypeError`. (Contribuição de Serhiy Storchaka e Raymond Hettinger em bpo-37319 [https://bugs.python.org/issue?@action=redirect&bpo=37319].)

- Os vários métodos de `load_module()` de `importlib` foram documentados como descontinuados desde Python 3.6, mas agora também irão disparar um `DeprecationWarning`. Use `exec_module()` em vez disso. (Contribuição de Brett Cannon em bpo-26131 [https://bugs.python.org/issue?@action=redirect&bpo=26131].)

- **`zimport.zipimporter.load_module()`** foi descontinuado em preferência a `exec_module()`. (Contribuição de Brett Cannon em bpo-26131 [https://bugs.python.org/issue?@action=redirect&bpo=26131].)

- O uso de `importlib.abc.Loader.load_module` pelo sistema de importação agora dispara uma exceção `ImportWarning`, pois `exec_module()` é preferível. (Contribuição de Brett Cannon em bpo-26131 [https://bugs.python.org/issue?@action=redirect&bpo=26131].)

- O uso de **`importlib.abc.MetaPathFinder.find_module()`** e **`importlib.abc.PathEntryFinder.find_module()`** pelo sistema de importação agora dispara uma exceção `ImportWarning`, pois `importlib.abc.MetaPathFinder.find_spec()` e `importlib.abc.PathEntryFinder.find_spec()` são preferidos, respectivamente. Você pode usar `importlib.util.spec_from_loader()` para ajudar no port. (Contribuição de Brett Cannon em bpo-42134 [https://bugs.python.org/issue?@action=redirect&bpo=42134].)

- O uso de **`importlib.abc.PathEntryFinder.find_loader()`** pelo sistema de importação agora dispara uma exceção `ImportWarning`, pois `importlib.abc.PathEntryFinder.find_spec()` é preferido. Você pode usar `importlib.util.spec_from_loader()` para ajudar no port. (Contribuição de Brett Cannon em bpo-43672 [https://bugs.python.org/issue?@action=redirect&bpo=43672].)

- As várias implementações de **`importlib.abc.MetaPathFinder.find_module()`** (

importlib.machinery.BuiltinImporter.find_module(), **importlib.machinery.FrozenImporter.find_module()**, **importlib.machinery.WindowsRegistryFinder.find_module()**, **importlib.machinery.PathFinder.find_module()**, **importlib.abc.MetaPathFinder.find_module()**), **importlib.abc.PathEntryFinder.find_module()** (**importlib.machinery.FileFinder.find_module()**) e **importlib.abc.PathEntryFinder.find_loader()** (**importlib.machinery.FileFinder.find_loader()**) agora levantam **DeprecationWarning** e estão programadas para remoção no Python 3.12 (anteriormente, eles foram documentados como descontinuados no Python 3.4). (Contribuição de Brett Cannon em bpo-42135 [https://bugs.python.org/issue?@action=redirect&bpo=42135].)

- **importlib.abc.Finder** foi descontinuada (incluindo seu único método, **find_module()**). **importlib.abc.MetaPathFinder** e **importlib.abc.PathEntryFinder** não mais herdam da classe. Usuários devem herdar de uma dessas duas classes conforme apropriado. (Contribuição de Brett Cannon em bpo-42135 [https://bugs.python.org/issue?@action=redirect&bpo=42135].)

- As descontinuações de **imp**, **importlib.find_loader()**, **importlib.util.set_package_wrapper()**, **importlib.util.set_loader_wrapper()**, **importlib.util.module_for_loader()**, **pkgutil.ImpImporter** e **pkgutil.ImpLoader** foram todas atualizadas para listar Python 3.12 como a versão programada para remoção (elas começaram a levantar **DeprecationWarning** em versões anteriores do Python). (Contribuição de Brett Cannon em bpo-43720 [https://bugs.python.org/issue?@action=redirect&bpo=43720].)

- O sistema de importação agora usa o atributo `__spec__` em módulos antes de recorrer a **module_repr()** para um método `__repr__()` do módulo. A remoção do uso de `module_repr()` está programado para Python 3.12. (Contribuição de Brett Cannon em bpo-42137 [https://bugs.python.org/issue?@action=redirect&bpo=42137].)

- **`importlib.abc.Loader.module_repr()`**, **`importlib.machinery.FrozenLoader.module_repr()`** e **`importlib.machinery.BuiltinLoader.module_repr()`** foram descontinuadas e programadas para remoção no Python 3.12. (Contribuição de Brett Cannon em bpo-42136 [https://bugs.python.org/issue?@action=redirect&bpo=42136].)

- `sqlite3.OptimizedUnicode` teve a documentação removida e foi descontinuado desde o Python 3.3, quando foi tornado um apelido para **`str`**. Foi agora descontinuado, programado para remoção no Python 3.12. (Contribuição de Erlend E. Aasland em bpo-42264 [https://bugs.python.org/issue?@action=redirect&bpo=42264].)

- A função embutida não documentada `sqlite3.enable_shared_cache` foi agora descontinuada, agendada para remoção no Python 3.12. Seu uso é fortemente desencorajado pela documentação do SQLite3. Veja a documentação do SQLite3 [https://sqlite.org/c3ref/enable_shared_cache.html] para mais detalhes. Se um cache compartilhado deve ser usado, abra o banco de dados no modo URI usando o parâmetro de consulta `cache=shared`. (Contribuição de Erlend E. Aasland em bpo-24464 [https://bugs.python.org/issue?@action=redirect&bpo=24464].)

- Os seguintes métodos de `threading` foram agora descontinuados:

  - `threading.currentThread` => **`threading.current_thread()`**
  - `threading.activeCount` => **`threading.active_count()`**
  - `threading.Condition.notifyAll` => **`threading.Condition.notify_all()`**
  - `threading.Event.isSet` => **`threading.Event.is_set()`**
  - `threading.Thread.setName` => **`threading.Thread.name`**
  - `threading.thread.getName` => **`threading.Thread.name`**
  - `threading.Thread.isDaemon` => **`threading.Thread.daemon`**
  - `threading.Thread.setDaemon` => **`threading.Thread.daemon`**

  (Contribuição de Jelle Zijlstra em gh-87889 [https://github.com/python/cpython/issues/87889].)

- **`pathlib.Path.link_to()`** foi descontinuado e programado para remoção no Python 3.12. Use **`pathlib.Path.hardlink_to()`** em vez disso. (Contribuição de Barney Gale em bpo-39950 [https://bugs.python.org/issue?@action=redirect&bpo=39950].)

- `cgi.log()` foi descontinuado e programado para remoção no Python 3.12. (Contribuição de Inada Naoki em bpo-41139 [https://bugs.python.org/issue?@action=redirect&bpo=41139].)

- Os recurso a seguir do **`ssl`** foram descontinuados desde o Python 3.6, Python 3.7 ou OpenSSL 1.1.0, e serão removidos no 3.11:

  - **`OP_NO_SSLv2`**, **`OP_NO_SSLv3`**, **`OP_NO_TLSv1`**, **`OP_NO_TLSv1_1`**, **`OP_NO_TLSv1_2`** e **`OP_NO_TLSv1_3`** foram substituídos por **`minimum_version`** e **`maximum_version`**.
  - **`PROTOCOL_SSLv2`**, **`PROTOCOL_SSLv3`**, **`PROTOCOL_SSLv23`**, **`PROTOCOL_TLSv1`**, **`PROTOCOL_TLSv1_1`**, **`PROTOCOL_TLSv1_2`** e **`PROTOCOL_TLS`** foram descontinuados em favor de **`PROTOCOL_TLS_CLIENT`** e **`PROTOCOL_TLS_SERVER`**
  - **`wrap_socket()`** foi substituída por **`ssl.SSLContext.wrap_socket()`**
  - **`match_hostname()`**
  - **`RAND_pseudo_bytes()`**, **`RAND_egd()`**
  - Recurso de NPN como **`ssl.SSLSocket.selected_npn_protocol()`** e **`ssl.SSLContext.set_npn_protocols()`** foram substituídos por ALPN.

- A depuração de threads (variável de ambiente **`PYTHONTHREADDEBUG`**) foi descontinuada no Python 3.10 e será removida no Python 3.12. Este recurso exige uma construção de depuração de Python. (Contribuição de Victor Stinner em bpo-44584 [https://bugs.python.org/issue?@action=redirect&bpo=44584].)

- A importação dos submódulos `typing.io` e `typing.re` agora emitirá **`DeprecationWarning`**. Esses submódulos serão removidos em uma versão futura do Python. Qualquer coisa pertencente a esses submódulos deve ser importada diretamente de **`typing`**. (Contribuição

de Sebastian Rittau em bpo-38291 [https://bugs.python.org/issue?@action=redirect&bpo=38291].)

Removidos

- Removidos os métodos especiais `__int__`, `__float__`, `__floordiv__`, `__mod__`, `__divmod__`, `__rfloordiv__`, `__rmod__` e `__rdivmod__` da classe **`complex`**. Eles sempre levantavam uma exceção **`TypeError`**. (Contribuição de Serhiy Storchaka em bpo-41974 [https://bugs.python.org/issue?@action=redirect&bpo=41974].)

- O método `ParserBase.error()` do módulo privado e não documentado `_markupbase` foi removido. **`html.parser.HTMLParser`** é a única subclasse de `ParserBase` e sua implementação de `error()` já tinha sido removida no Python 3.5. (Contribuição de Berker Peksag em bpo-31844 [https://bugs.python.org/issue?@action=redirect&bpo=31844].)

- Removido o atributo `unicodedata.ucnhash_CAPI` que foi um objeto interno PyCapsule. A estrutura privada relacionada `_PyUnicode_Name_CAPI` foi movida para uma API C interna. (Contribuição de Victor Stinner em bpo-42157 [https://bugs.python.org/issue?@action=redirect&bpo=42157].)

- Removido o módulo `parser`, que foi descontinuado em 3.9 em razão da mudança para o novo analisador sintático GASE, bem como todo os arquivos código-fonte e cabeçalhos C que estavam sendo usados pelo analisador sintático antigo, incluindo `node.h`, `parser.h`, `graminit.h` e `grammar.h`.

- Removidas as funções de API C pública `PyParser_SimpleParseStringFlags`, `PyParser_SimpleParseStringFlagsFilename`, `PyParser_SimpleParseFileFlags` e `PyNode_Compile` que foram descontinuadas no 3.9 em razão da mudança para o novo analisador sintático GASE.

- Removido o módulo `formatter`, que foi descontinuado no Python 3.4. É um tanto obsoleto, pouco usado e não testado. Ele foi originalmente programado para ser removido no Python 3.6, mas tais remoções foram adiadas até depois do fim de vida do Python 2.7. Os usuários existentes devem copiar quaisquer classes que usam em seu código. (Contribuição de Dong-hee Na e Terry J. Reedy em bpo-42299 [https://bugs.python.org/issue?@action=redirect&bpo=42299].)

- Removida a função **`PyModule_GetWarningsModule()`** que era inútil agora porque o módulo **`_warnings`** foi convertido em um módulo embutido no 2.6. (Contribuição de Hai Shi em bpo-42599 [https://bugs.python.org/issue?@action=redirect&bpo=42599].)

- Remove apelidos descontinuados para Classes Base Abstratas de Coleções do módulo **`collections`**. (Contribuição de Victor Stinner em bpo-37324 [https://bugs.python.org/issue?@action=redirect&bpo=37324].)

- O parâmetro `loop` foi removido da maioria da API de alto nível do **`asyncio`** seguindo a descontinuidade no Python 3.8. A motivação por trás desta alteração é multifacetada:

  1. Isso simplifica a API de alto nível.
  2. As funções na API de alto nível obtêm implicitamente o laço de eventos em execução do thread atual desde o Python 3.7. Não há necessidade de passar o laço de eventos para a API na maioria dos casos de uso normais.
  3. A passagem de laço de eventos está sujeita a erros, especialmente ao lidar com laços em execução em diferentes threads.

  Observe que a API de baixo nível ainda aceitará `loop`. Veja Alterações na API Python para exemplos de como substituir o código existente.

  (Contribuição de Yurii Karabas, Andrew Svetlov, Yury Selivanov e Kyle Stanley em bpo-42392 [https://bugs.python.org/issue?@action=redirect&bpo=42392].)

Portando para Python 3.10

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações na sintaxe Python

- O aviso de descontinuação agora é emitido ao compilar a sintaxe anteriormente válida se o literal numérico for seguido imediatamente por uma palavra reservada (como em `0in x`). Em versões futuras, ele será alterado para aviso de sintaxe e, finalmente, para um erro de sintaxe. Para se livrar do aviso e tornar o código compatível com versões futuras, basta adicionar um espaço entre o literal numérico e a palavra reservada seguinte. (Contribuição de Serhiy Storchaka em bpo-43833 [https://bugs.python.org/issue?@action=redirect&bpo=43833].)

Alterações na API Python

- Os parâmetros *etype* das funções `format_exception()`, `format_exception_only()` e `print_exception()` no módulo `traceback` foram renomeadas para *exc*. (Contribuição de Zackery Spytz e Matthias Bussonnier em bpo-26389 [https://bugs.python.org/issue?@action=redirect&bpo=26389].)

- `atexit`: ao sair do Python, se uma função de retorno registrada com `atexit.register()` falhar, sua exceção agora é registrada nos logs. Anteriormente, apenas algumas exceções eram registradas nos logs e a última exceção era sempre ignorada silenciosamente. (Contribuição de Victor Stinner em bpo-42639 [https://bugs.python.org/issue?@action=redirect&bpo=42639].)

- A classe genérica **`Collections.abc.Callable`** agora nivela os parâmetros de tipo, similar ao que `typing.Callable` faz atualmente. Isso significa que `collections.abc.Callable[[int, str], str]` terá `__args__` de `(int, str, str)`; anteriormente era `([int, str], str)`. O código que acessa os argumentos via `typing.get_args()` ou `__args__` precisa levar em conta esta mudança. Além disso, `TypeError` pode ser levantada para formas inválidas de parametrização **`Collections.abc.Callable`** que pode ter passada silenciosamente no

Python 3.9. (Contribuição de Ken Jin em bpo-42195 [https://bugs.python.org/issue?@action=redirect&bpo=42195].)

- `socket.htons()` e `socket.ntohs()` agora levantam `OverflowError` em vez de `DeprecationWarning` se o parâmetro dado não couber em um inteiro sem sinal de 16 bits. (Contribuição de Erlend E. Aasland em bpo-42393 [https://bugs.python.org/issue?@action=redirect&bpo=42393].)

- O parâmetro `loop` foi removido da maioria da API de alto nível do `asyncio` seguindo a descontinuidade no Python 3.8.

  A corrotina que atualmente se parece com isso:

  ```
  async def foo(loop):
      await asyncio.sleep(1, loop=loop)
  ```

  Deve ser substituída por isso:

  ```
  async def foo():
      await asyncio.sleep(1)
  ```

  Se `foo()` for especificamente projetado para *não* executar no laço de eventos em execução da thread atual (por exemplo, executar no laço de eventos de outra thread), considere usar `asyncio.run_coroutine_threadsafe()`.

  (Contribuição de Yurii Karabas, Andrew Svetlov, Yury Selivanov e Kyle Stanley em bpo-42392 [https://bugs.python.org/issue?@action=redirect&bpo=42392].)

- O construtor `types.FunctionType` agora herda os embutidos atuais se o dicionário *globals* tiver nenhuma chave `"__builtins__"`, em vez de usar `{"None": None}` como embutidos: o mesmo comportamento que as funções `eval()` e `exec()`. Definir uma função com `def function(...): ...` no Python não é afetado, globais não podem ser substituídos com esta sintaxe: também herda os embutidos atuais. (Contribuição de Victor Stinner em bpo-42990 [https://bugs.python.org/issue?@action=redirect&bpo=42990].)

Alterações na API C

- As funções de API C `PyParser_SimpleParseStringFlags`, `PyParser_SimpleParseStringFlagsFilename`, `PyParser_SimpleParseFileFlags`, `PyNode_Compile` e o tipo usado por essas funções, `struct _node`, foram removidos em razão da mudança para o novo analisador sintático GASE.

  O código-fonte deve ser agora compilado diretamente para um objeto código usando, por exemplo, **`Py_CompileString()`**. O objeto código resultante pode ser então avaliado usando, por exemplo, **`PyEval_EvalCode()`**.

  Especificamente:

  - Uma chamada para `PyParser_SimpleParseStringFlags` seguida por `PyNode_Compile` pode ser substituída por **`Py_CompileString()`**.

  - Não há substituição direta para `PyParser_SimpleParseFileFlags`. Para compilar código de um argumento `FILE *`, você vai precisar ler o arquivo em C e passar o buffer resultante para **`Py_CompileString()`**.

  - Para compilar um arquivo de um nome de arquivo `char *` dado, abra explicitamente o arquivo, leia-o e compile o resultado. Uma forma de fazer isso é usando o módulo **`io`** com **`PyImport_ImportModule()`**, **`PyObject_CallMethod()`**, **`PyBytes_AsString()`** e **`Py_CompileString()`**, como esboçado abaixo. (Declarações e tratamento de erro foram omitidos.)

    ```
    io_module = Import_ImportModule("io");
    fileobject = PyObject_CallMethod(io_module, "open", "ss
    source_bytes_object = PyObject_CallMethod(fileobject, "
    result = PyObject_CallMethod(fileobject, "close", "");
    source_buf = PyBytes_AsString(source_bytes_object);
    code = Py_CompileString(source_buf, filename, Py_file_i
    ```

- Para objetos `FrameObject`, o membro `f_lasti` agora representa um deslocamento de código de palavra em vez de um deslocamento simples na string de bytecode. Isso significa que esse número precisa ser multiplicado por 2 para ser usado com APIs que esperam um deslocamento de byte (como `PyCode_Addr2Line()`, por exemplo). Observe também que o membro `f_lasti` de objetos `FrameObject` não é considerado estável: por favor, use `PyFrame_GetLineNumber()` em vez disso.

Alterações de bytecode do CPython

- A instrução `MAKE_FUNCTION` agora aceita um dicionário ou uma tupla de strings como as anotações da função. (Contribuição de Yurii Karabas e Inada Naoki em bpo-42202 [https://bugs.python.org/issue?@action=redirect&bpo=42202].)

Mudanças na construção

- **PEP 644** [https://peps.python.org/pep-0644/]: Python agora exige OpenSSL 1.1.1 ou mais novo. OpenSSL 1.0.2 não é mais suportado. (Contribuição de Christian Heimes em bpo-43669 [https://bugs.python.org/issue?@action=redirect&bpo=43669].)

- As funções C99 `snprintf()` e `vsnprintf()` agora são exigidas para compilar o Python. (Contribuição de Victor Stinner em bpo-36020 [https://bugs.python.org/issue?@action=redirect&bpo=36020].)

- `sqlite3` exige SQLite 3.7.15 ou superior. (Contribuição de Sergey Fedoseev e Erlend E. Aasland em bpo-40744 [https://bugs.python.org/issue?@action=redirect&bpo=40744] e bpo-40810 [https://bugs.python.org/issue?@action=redirect&bpo=40810].)

- O módulo `atexit` deve agora sempre ser construído como um módulo embutido. (Contribuição de Victor Stinner em bpo-42639 [https://bugs.python.org/issue?@action=redirect&bpo=42639].)

- Adiciona a opção **`--disable-test-modules`** ao script `configure`: não constrói nem instala módulos de teste. (Contribuição de Xavier de Gaye, Thomas Petazzoni e Peixing Xin em bpo-27640 [https://bugs.python.org/issue?@action=redirect&bpo=27640].)

- Adiciona a **`option --with-wheel-pkg-dir=CAMINHO`** ao script `./configure`. Se especificado, o módulo **`ensurepip`** procura por pacotes wheel de `setuptools` e `pip` neste diretório: se ambos estiverem presentes, esses pacotes wheel são usados em vez de pacotes wheel empacotados por ensurepip.

  Algumas distribuições Linux possuem políticas de empacotamento recomendando evitar o empacotamento de dependências. Por exemplo, Fedora instala pacotes wheel no diretório `/usr/share/python-wheels/` e não instala o pacote `ensurepip._bundled`.

  (Contribuição de Victor Stinner em bpo-42856 [https://bugs.python.org/issue?@action=redirect&bpo=42856].)

- Adiciona uma nova **`opção --without-static-libpython de configure`** para não construir biblioteca estática `libpythonMAJOR.MINOR.a` e não instala o arquivo objeto `python.o`.

  (Contribuição de Victor Stinner em bpo-43103 [https://bugs.python.org/issue?@action=redirect&bpo=43103].)

- O script `configure` agora usa o utilitário `pkg-config`, se disponível, para detectar a localização dos cabeçalhos Tcl/Tk e bibliotecas. Como antes, esses locais podem ser explicitamente especificados com as opções de configuração `--with-tcltk-includes` e `--with-tcltk-libs` . (Contribuição de Manolis Stamatogiannakis em bpo-42603 [https://bugs.python.org/issue?@action=redirect&bpo=42603].)

- Adiciona a opção **`--with-openssl-rpath`** ao script `configure`. A opção simplifica a construção do Python com uma instalação OpenSSL personalizada, por exemplo, `./configure --with-openssl=/path/to/openssl --with-openssl-rpath=auto`.

(Contribuição de Christian Heimes em bpo-43466 [https://bugs.python.org/issue?@action=redirect&bpo=43466].)

Alterações na API C

PEP 652: Mantendo a ABI estável

A ABI (interface binária de aplicação) Estável para módulos de extensão ou Python embutido agora está explicitamente definido. C API and ABI Stability descreve as garantias de estabilidade da API C e ABI junto com as melhores práticas para usar a ABI estável.

(Contribuição de Petr Viktorin em **PEP 652** [https://peps.python.org/pep-0652/] e bpo-43795 [https://bugs.python.org/issue?@action=redirect&bpo=43795].)

Novas funcionalidades

- O resultado de `PyNumber_Index()` agora sempre tem o tipo exato `int`. Anteriormente, o resultado poderia ser uma instância de uma subclasse de `int`. (Contribuição de Serhiy Storchaka em bpo-40792 [https://bugs.python.org/issue?@action=redirect&bpo=40792].)

- Adiciona um novo membro `orig_argv` à estrutura `PyConfig`: a lista dos argumentos originais da linha de comando passados para o executável Python. (Contribuição de Victor Stinner em bpo-23427 [https://bugs.python.org/issue?@action=redirect&bpo=23427].)

- As macros `PyDateTime_DATE_GET_TZINFO()` e `PyDateTime_TIME_GET_TZINFO()` foram adicionadas para acessar os atributos `tzinfo` dos objetos `datetime.datetime` e `datetime.time`. (Contribuição de Zackery Spytz em bpo-30155 [https://bugs.python.org/issue?@action=redirect&bpo=30155].)

- Adiciona uma função `PyCodec_Unregister()` para cancelar um registro de uma função de pesquisa de codecs. (Contribuição de Hai Shi em bpo-41842 [https://bugs.python.org/issue?@action=redirect&bpo=41842].)

- A função `PyIter_Send()` foi adicionada para permitir o envio de valor para o iterador sem levantar a exceção `StopIteration`. (Contribuição de Vladimir Matveev em bpo-41756 [https://bugs.python.org/issue?@action=redirect&bpo=41756].)

- Adiciona `PyUnicode_AsUTF8AndSize()` à API C limitada. (Contribuição de Alex Gaynor em bpo-41784 [https://bugs.python.org/issue?@action=redirect&bpo=41784].)

- Adiciona a função `PyModule_AddObjectRef()`: semelhante a `PyModule_AddObject()`, mas não roube uma referência ao valor em caso de sucesso. (Contribuição de Victor Stinner em bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741].)

- Adiciona as funções `Py_NewRef()` e `Py_XNewRef()` para incrementar a contagem de referências de um objeto e retornar o objeto. (Contribuição de Victor Stinner em bpo-42262 [https://bugs.python.org/issue?@action=redirect&bpo=42262].)

- As funções `PyType_FromSpecWithBases()` e `PyType_FromModuleAndSpec()` agora aceitam uma única classe como o argumento *bases*. (Contribuição de Serhiy Storchaka em bpo-42423 [https://bugs.python.org/issue?@action=redirect&bpo=42423].)

- A função `PyType_FromModuleAndSpec()` agora aceita o slot NULL `tp_doc`. (Contribuição de Hai Shi em bpo-41832 [https://bugs.python.org/issue?@action=redirect&bpo=41832].)

- A função `PyType_GetSlot()` pode aceitar tipos estáticos. (Contribuição de Hai Shi e Petr Viktorin em bpo-41073 [https://bugs.python.org/issue?@action=redirect&bpo=41073].)

- Adiciona uma nova função `PySet_CheckExact()` à C-API para verificar se um objeto é uma instância de `set`, mas não uma instância de um subtipo. (Contribuição de Pablo Galindo em bpo-43277 [https://bugs.python.org/issue?@action=redirect&bpo=43277].)

- Adiciona `PyErr_SetInterruptEx()` que permite passar um número de sinal para simular. (Contribuição de Antoine Pitrou em bpo-43356 [https://bugs.python.org/issue?@action=redirect&bpo=43356].)

- A API C limitada agora é suportada se o Python for compilado no modo de depuração (se a macro `Py_DEBUG` estiver definida). Na API C limitada, as funções `Py_INCREF()` e `Py_DECREF()` agora são implementadas como chamadas de função opacas, em vez de acessar diretamente o membro `PyObject.ob_refcnt`, se o Python for compilado no modo de depuração e a macro `Py_LIMITED_API` tiver como alvo o Python 3.10 ou mais recente. Tornou-se possível suportar a API C limitada no modo de depuração porque a estrutura `PyObject` é a mesma no modo de lançamento e depuração desde o Python 3.8 (consulte bpo-36465 [https://bugs.python.org/issue?@action=redirect&bpo=36465]).

  A API C limitada ainda não é suportada na compilação especial `--with-trace-refs` (macro `Py_TRACE_REFS`). (Contribuição de Victor Stinner em bpo-43688 [https://bugs.python.org/issue?@action=redirect&bpo=43688].)

- Adiciona a função `Py_Is(x, y)` para testar se o objeto *x* é o objeto *y*, o mesmo que `x is y` em Python. Adiciona também as funções `Py_IsNone()`, `Py_IsTrue()`, `Py_IsFalse()` para testar se um objeto é, respectivamente, o singleton `None`, o singleton `True` ou o singleton `False`. (Contribuição de Victor Stinner em bpo-43753 [https://bugs.python.org/issue?@action=redirect&bpo=43753].)

- Adiciona novas funções para controlar o coletor de lixo do código C: `PyGC_Enable()`, `PyGC_Disable()`, `PyGC_IsEnabled()`. Estas funções permitem ativar, desativar e consultar o estado do coletor de lixo do código C sem precisar importar o módulo `gc`.

- Adiciona um novo sinalizador de tipo `Py_TPFLAGS_DISALLOW_INSTANTIATION` para impedir a criação de instâncias de tipo. (Contribuição de Victor Stinner em bpo-43916 [https://bugs.python.org/issue?@action=redirect&bpo=43916].)

- Adiciona um novo sinalizador de tipo `Py_TPFLAGS_IMMUTABLETYPE` para criar objetos de tipo imutável: atributos de tipo não podem ser

definidos nem excluídos. (Contribuição de Victor Stinner e Erlend E. Aasland em bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908].)

Portando para Python 3.10

- A macro `PY_SSIZE_T_CLEAN` deve agora ser definida para usar os formatos **`PyArg_ParseTuple()`** e **`Py_BuildValue()`** que usam `#`: `es#`, `et#`, `s#`, `u#`, `y#`, `z#`, `U#` e `Z#`. Veja Análise de argumentos e construção de valores e **PEP 353** [https://peps.python.org/pep-0353/]. (Contribuição de Victor Stinner em bpo-40943 [https://bugs.python.org/issue?@action=redirect&bpo=40943].)

- Como **`Py_REFCNT()`** é alterado para a função estática em linha, `Py_REFCNT(obj) = new_refcnt` deve ser substituído por `Py_SET_REFCNT(obj, new_refcnt)`: veja **`Py_SET_REFCNT()`** (disponível desde o Python 3.9). Para compatibilidade com versões anteriores, esta macro pode ser usada:

```
#if PY_VERSION_HEX < 0x030900A4
#  define Py_SET_REFCNT(obj, refcnt) ((Py_REFCNT(obj) = (ref
#endif
```

  (Contribuição de Victor Stinner em bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573].)

- A chamada de **`PyDict_GetItem()`** sem a GIL retida era permitido por razões históricas. Isso não é mais permitido. (Contribuição de Victor Stinner em bpo-40839 [https://bugs.python.org/issue?@action=redirect&bpo=40839].)

- `PyUnicode_FromUnicode(NULL, size)` e `PyUnicode_FromStringAndSize(NULL, size)` levantam `DeprecationWarning` agora. Use **`PyUnicode_New()`** para alocar objeto Unicode sem dados iniciais. (Contribuição de Inada Naoki em bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346].)

- A estrutura privada `_PyUnicode_Name_CAPI` da API PyCapsule `unicodedata.ucnhash_CAPI` foi movida para a API C interna. (Contribuição de Victor Stinner em bpo-42157 [https://bugs.python.org/issue?@action=redirect&bpo=42157].)

- As funções **`Py_GetPath()`**, **`Py_GetPrefix()`**, **`Py_GetExecPrefix()`**, **`Py_GetProgramFullPath()`** , **`Py_GetPythonHome()`** e **`Py_GetProgramName()`** agora retornam `NULL` se chamadas antes de **`Py_Initialize()`** (antes do Python ser inicializado). Use a nova API de Configuração de inicialização do Python para obter a Python Path Configuration. (Contribuição de Victor Stinner em bpo-42260 [https://bugs.python.org/issue?@action=redirect&bpo=42260].)

- As macros **`PyList_SET_ITEM()`**, **`PyTuple_SET_ITEM()`** e **`PyCell_SET()`** não podem mais ser usadas como valor-l ou valor-r. Por exemplo, `x = PyList_SET_ITEM(a, b, c)` e `PyList_SET_ITEM(a, b, c) = x` agora falham com um erro do compilador. Isso previne bugs como o teste `if (PyList_SET_ITEM (a, b, c) < 0) ...`. (Contribuição de Zackery Spytz e Victor Stinner em bpo-30459 [https://bugs.python.org/issue?@action=redirect&bpo=30459].)

- Os arquivos de API não limitada `odictobject.h`, `parser_interface.h`, `piclebufobject.h`, `pyarena.h`, `pyctype.h`, `pydebug. h`, `pyfpe.h` e `pytime.h` foram movidos para o diretório `Include/cpython`. Esses arquivos não devem ser incluídos diretamente, pois já estão incluídos em `Python.h`; veja Arquivos de inclusão. Se eles foram incluídos diretamente, considere incluir `Python.h` em vez disso. (Contribuição de Nicholas Sim em bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134].)

- Use o sinalizador de tipo **`Py_TPFLAGS_IMMUTABLETYPE`** para criar objetos de tipo imutável. Não confie em **`Py_TPFLAGS_HEAPTYPE`** para decidir se um objeto de tipo é mutável ou não; verifique se **`Py_TPFLAGS_IMMUTABLETYPE`** está definido. (Contribuição de Victor Stinner e Erlend E. Aasland em bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908].)

- A função não documentada `Py_FrozenMain` foi removida da API limitada. A função é útil principalmente para compilações personalizadas do Python. (Contribuição de Petr Viktorin em bpo-26241 [https://bugs.python.org/issue?@action=redirect&bpo=26241].)

Descontinuados

- A função `PyUnicode_InternImmortal()` agora foi descontinuada e será removida no Python 3.12: use **`PyUnicode_InternInPlace()`** em vez disso. (Contribuição de Victor Stinner em bpo-41692 [https://bugs.python.org/issue?@action=redirect&bpo=41692].)

Removidos

- Removidas as funções `Py_UNICODE_str*` que manipulam strings `Py_UNICODE*`. (Contribuição de Inada Naoki em bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123].)

  - `Py_UNICODE_strlen`: use **`PyUnicode_GetLength()`** ou **`PyUnicode_GET_LENGTH`**
  - `Py_UNICODE_strcat`: use **`PyUnicode_CopyCharacters()`** ou **`PyUnicode_FromFormat()`**
  - `Py_UNICODE_strcpy`, `Py_UNICODE_strncpy`: use **`PyUnicode_CopyCharacters()`** ou **`PyUnicode_Substring()`**
  - `Py_UNICODE_strcmp`: use **`PyUnicode_Compare()`**
  - `Py_UNICODE_strncmp`: use **`PyUnicode_Tailmatch()`**
  - `Py_UNICODE_strchr`, `Py_UNICODE_strrchr`: use **`PyUnicode_FindChar()`**

- Removida `PyUnicode_GetMax()`. Migre para as novas APIs (**PEP 393** [https://peps.python.org/pep-0393/]). (Contribuição de Inada Naoki em bpo-41103 [https://bugs.python.org/issue?@action=redirect&bpo=41103].)

- Removida `PyLong_FromUnicode()`. Migre para **`PyLong_FromUnicodeObject()`**. (Contribuição de Inada Naoki em bpo-41103 [https://bugs.python.org/issue?@action=redirect&bpo=41103].)

- Removida `PyUnicode_AsUnicodeCopy()`. Use **`PyUnicode_AsUCS4Copy()`** ou **`PyUnicode_AsWideCharString()`** (Contribuição de Inada Naoki em bpo-41103 [https://bugs.python.org/issue?@action=redirect&bpo=41103].)

- Removida a variável `_Py_CheckRecursionLimit`: foi substituída por `ceval.recursion_limit` da estrutura **`PyInterpreterState`**. (Contribuição de Victor Stinner em bpo-41834 [https://bugs.python.org/issue?@action=redirect&bpo=41834].)

- Removidas as macros não documentadas `Py_ALLOW_RECURSION` e `Py_END_ALLOW_RECURSION` e o campo `recursion_critical` da estrutura **`PyInterpreterState`**. (Contribuição de Serhiy Storchaka em bpo-41936 [https://bugs.python.org/issue?@action=redirect&bpo=41936].)

- Removida a função `PyOS_InitInterrupts()` não documentada. Inicializar o Python já instala implicitamente manipuladores de sinal: veja **`PyConfig.install_signal_handlers`**. (Contribuição de Victor Stinner em bpo-41713 [https://bugs.python.org/issue?@action=redirect&bpo=41713].)

- Remova a função `PyAST_Validate()`. Não é mais possível construir um objeto AST (tipo `mod_ty`) com a API C pública. A função já foi excluída da API C limitada (**PEP 384** [https://peps.python.org/pep-0384/]). (Contribuição de Victor Stinner em bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244].)

- Remove o arquivo de cabeçalho `symtable.h` e as funções não documentadas:

  - `PyST_GetScope()`
  - `PySymtable_Build()`
  - `PySymtable_BuildObject()`
  - `PySymtable_Free()`
  - `Py_SymtableString()`
  - `Py_SymtableStringObject()`

  A função `Py_SymtableString()` fazia parte da ABI estável por engano, mas não pôde ser usada, porque o arquivo de cabeçalho

`symtable.h` foi excluído da API C limitada.

Use o módulo Python **`symtable`** em vez disso. (Contribuição de Victor Stinner em bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244].)

- Remove **`PyOS_ReadlineFunctionPointer()`** dos cabeçalhos limitados da API C e de `python3.dll`, a biblioteca que fornece a ABI estável no Windows. Como a função recebe um argumento `FILE*`, sua estabilidade ABI não pode ser garantida. (Contribuição de Petr Viktorin em bpo-43868 [https://bugs.python.org/issue?@action=redirect&bpo=43868].)

- Remove os arquivos de cabeçalho `ast.h`, `asdl.h` e `Python-ast.h`. Essas funções não estavam documentadas e excluídas da API C limitada. A maioria dos nomes definidos por esses arquivos de cabeçalho não foram prefixados por `Py` e, portanto, podem criar conflitos de nomes. Por exemplo, `Python-ast.h` definiu uma macro `Yield` que estava em conflito com o nome `Yield` usado pelo cabeçalho `<winbase.h>` do Windows. Use o módulo Python **`ast`** em vez disso. (Contribuição de Victor Stinner em bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244].)

- Remove as funções do compilador e do analisador usando o tipo `struct _mod`, porque a API AST C pública foi removida:

  - `PyAST_Compile()`
  - `PyAST_CompileEx()`
  - `PyAST_CompileObject()`
  - `PyFuture_FromAST()`
  - `PyFuture_FromASTObject()`
  - `PyParser_ASTFromFile()`
  - `PyParser_ASTFromFileObject()`
  - `PyParser_ASTFromFilename()`
  - `PyParser_ASTFromString()`
  - `PyParser_ASTFromStringObject()`

  Essas funções não estavam documentadas e excluídas da API C limitada. (Contribuição de Victor Stinner em bpo-43244

[https://bugs.python.org/issue?@action=redirect&bpo=43244].)

- Remove o arquivo de cabeçalho `pyarena.h` com funções:

  - `PyArena_New()`
  - `PyArena_Free()`
  - `PyArena_Malloc()`
  - `PyArena_AddPyObject()`

  Essas funções não estavam documentadas, excluídas da API C limitada e eram usadas apenas internamente pelo compilador. (Contribuição de Victor Stinner em bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244].)

- O membro `PyThreadState.use_tracing` foi removido para otimizar o Python. (Contribuição de Mark Shannon em bpo-43760 [https://bugs.python.org/issue?@action=redirect&bpo=43760].)

Recursos de segurança notáveis no 3.10.7

Converter entre **`int`** e **`str`** em bases diferentes de 2 (binário), 4, 8 (octal), 16 (hexadecimal) ou 32 como base 10 (decimal) agora levanta uma exceção **`ValueError`** se o número de dígitos em forma de string estiver acima de um limite para evitar possíveis ataques de negação de serviço devido à complexidade algorítmica. Esta é uma mitigação para **CVE 2020-10735** [https://www.cve.org/CVERecord?id=CVE-2020-10735]. Este limite pode ser configurado ou desabilitado por variável de ambiente, sinalizador de linha de comando ou APIs de **`sys`**. Veja a documentação de limitação de comprimento de conversão de string inteira. O limite padrão é de 4300 dígitos em forma de string.

Recursos de segurança notáveis no 3.10.8

O módulo descontinuado **`mailcap`** agora se recusa a injetar texto não seguro (nomes de arquivos, tipos MIME, parâmetros) em comandos shell. Em vez de usar esse texto, ele avisará e agirá como se uma correspondência não fosse encontrada (ou para comandos de teste, como se o teste tivesse

falhado). (Contribuição de Petr Viktorin em gh-98966 [https://github.com/python/cpython/issues/98966].)

Alterações notáveis no 3.10.12

tarfile

- Os métodos de extração em `tarfile` e `shutil.unpack_archive()`, têm um novo argumento *filter* que permite limitar recursos do tar que podem ser surpreendentes ou perigosos, como criar arquivos fora do diretório de destino. Veja Filtros de extração para detalhes. No Python 3.12, usar sem o argumento *filter* mostrará um `DeprecationWarning`. No Python 3.14, o padrão mudará para `'data'`. (Contribuição de Petr Viktorin em **PEP 706** [https://peps.python.org/pep-0706/].)

O que há de novo no Python 3.9

Editor:: Łukasz Langa

Este artigo explica os novos recursos do Python 3.9, em comparação com o 3.8. Python 3.9 foi lançado em 15 de outubro de 2020. Veja o changelog para uma lista completa de mudanças.

Ver também

PEP 596 [https://peps.python.org/pep-0596/] - Cronograma de lançamento do Python 3.9

Resumo – Destaques da versão

Novos recursos de sintaxe:

- **PEP 584** [https://peps.python.org/pep-0584/], operadores de união adicionados ao `dict`;
- **PEP 585** [https://peps.python.org/pep-0585/], type hinting genéricos em coleções comuns;
- **PEP 614** [https://peps.python.org/pep-0614/], restrições de gramática relaxadas em decoradores.

Novos recursos embutidos:

- **PEP 616** [https://peps.python.org/pep-0616/], métodos de strings para remover prefixos e sufixos.

Novos recursos na biblioteca padrão:

- **PEP 593** [https://peps.python.org/pep-0593/], anotações flexíveis de função e variável;

- `os.pidfd_open()` adicionada para permitidos gerenciamento de processo sem corridas e sinais.

Melhorias no interpretador:

- **PEP 573** [https://peps.python.org/pep-0573/], acesso rápido ao estado do módulo a partir de métodos de tipos de extensão C;
- **PEP 617** [https://peps.python.org/pep-0617/], CPython agora usa um novo analisador sintático com base em GASE;
- vários tipos embutidos do Python (range, tuple, set, frozenset, list, dict) estão agora mais rápidos usando vectorcall da **PEP 590** [https://peps.python.org/pep-0590/];
- coleta de lixo não causa bloqueio em objetos ressuscitados;
- vários módulos do Python (`_abc`, `audioop`, `_bz2`, `_codecs`, `_contextvars`, `_crypt`, `_functools`, `_json`, `_locale`, `math`, `operator`, `resource`, `time`, `_weakref`) agora usam inicialização multifásica conforme definido pela PEP 489;
- vários módulos da biblioteca padrão (`audioop`, `ast`, `grp`, `_hashlib`, `pwd`, `_posixsubprocess`, `random`, `select`, `struct`, `termios`, `zlib`) estão agora usando a ABI estável definida pela PEP 384.

Novos módulos de biblioteca:

- **PEP 615** [https://peps.python.org/pep-0615/], o banco de dados de fusos horários da IANA é agora apresentado na biblioteca padrão no módulo `zoneinfo`;
- uma implementação de um tipo topológico de um grafo agora é fornecida no novo módulo `graphlib`.

Alterações no processo de lançamento:

- **PEP 602** [https://peps.python.org/pep-0602/], CPython adota um ciclo de lançamento anual.

Você deve verificar DeprecationWarning no seu código

Quando o Python 2.7 ainda tinha suporte, muitas funcionalidades do Python 3 foram mantidas para compatibilidade com versões anteriores do Python 2.7. Com o fim do suporte ao Python 2, essas camadas de compatibilidade com versões anteriores foram removidas ou serão removidas em breve. A maioria deles emitia um aviso de **`DeprecationWarning`** por vários anos. Por exemplo, usar `collections.Mapping` em vez de `collections.abc.Mapping` emite uma **`DeprecationWarning`** desde Python 3.3, lançado em 2012.

Teste sua aplicação com a opção de linha de comando **`-W`** `default` para ver **`DeprecationWarning`** e **`PendingDeprecationWarning`**, ou mesmo com **`-W`** `error` para tratá-los como erros. Filtro de avisos pode ser usado para ignorar avisos de código de terceiros.

Python 3.9 é a última versão que fornece essas camadas de compatibilidade com versões anteriores do Python 2, para dar mais tempo aos mantenedores de projetos Python para organizar a remoção do suporte Python 2 e adicionar suporte para Python 3.9.

Aliases para classes bases abstratas no módulo **`collections`**, como o alias `collections.Mapping` para **`collections.abc.Mapping`**, são mantidos para uma última versão para compatibilidade reversa. Eles serão removidos do Python 3.10.

De maneira mais geral, experimente executar seus testes no Modo de Desenvolvimento do Python, que ajuda a preparar seu código para torná-lo compatível com a próxima versão do Python.

Observação: vários itens descontinuados pré-existentes também foram removidos nesta versão do Python. Consulte a seção Removidos.

Novas funcionalidades

Operadores de mesclagem & atualização de dicionário

Operadores de mesclagem (`|`) e atualização (`|=`) foram adicionados à classe embutida **`dict`**. Eles complementam o métodos `dict.update` e `{**d1,`

`**d2}` existentes de mesclagem de dicionários.

Exemplo:

```
>>> x = {"chave1": "valor1 from x", "chave2": "valor2 from x"}
>>> y = {"chave2": "valor2 from y", "chave3": "valor3 from y"}
>>> x | y
{'chave1': 'valor1 from x', 'chave2': 'valor2 from y', 'chave3':
>>> y | x
{'chave2': 'valor2 from x', 'chave3': 'valor3 from y', 'chave1':
```

Veja **PEP 584** [https://peps.python.org/pep-0584/] para uma descrição completa. (Contribuição de Brandt Bucher em bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144].)

Novos métodos de strings para remover prefixos e sufixos

`str.removeprefix(prefix)` e **`str.removesuffix(suffix)`** foram adicionados para remover facilmente um prefixo ou sufixo desnecessário de uma string. Os métodos correspondentes `bytes`, `bytearray` e `collections.UserString` também foram adicionados. Veja **PEP 616** [https://peps.python.org/pep-0616/] para uma descrição completa. (Contribuição de Dennis Sweeney em bpo-39939 [https://bugs.python.org/issue?@action=redirect&bpo=39939].)

Type hinting genéricos nas coleções padrão

Agora, nas anotações de tipo, você pode usar tipos de coleção embutidos como `list` e `dict` como tipos genéricos, em vez de importar os tipos de letras maiúsculas correspondentes (por exemplo, `List` ou `Dict`) de `typing`. Agora, outros tipos na biblioteca padrão também são genéricos, por exemplo, `queue.Queue`.

Exemplo:

```
def cumprimentar_todos(nomes: list[str]) -> None:
    for nome in nomes:
        print("Olá", nome)
```

Consulte **PEP 585** [https://peps.python.org/pep-0585/] para mais detalhes. (Contribuição de Guido van Rossum, Ethan Smith e Batuhan Taşkaya em bpo-39481 [https://bugs.python.org/issue?@action=redirect&bpo=39481].)

Novo analisador sintático

O Python 3.9 usa um novo analisador sintático, baseado em GASE [https://pt.wikipedia.org/wiki/Gramática_de_análise_sintática_de_expressão] em vez de LL(1) [https://pt.wikipedia.org/wiki/Analisador_sintático_LL]. O desempenho do novo analisador sintático é aproximadamente comparável ao do analisador sintático antigo, mas o formalismo do GASE é mais flexível que do LL(1) quando se trata de projetar novos recursos de linguagem. Começaremos a usar essa flexibilidade no Python 3.10 e posterior.

O módulo `ast` usa o novo analisador e produz o mesmo AST que o analisador antigo.

No Python 3.10, o analisador antigo será excluído e todas as funcionalidades que dependem dele (principalmente o módulo `parser`, que foi descontinuado a muito tempo). No Python 3.9 *apenas*, você pode voltar ao analisador LL(1) usando uma opção de linha de comando (`-X oldparser`) ou uma variável de ambiente (`PYTHONOLDPARSER=1`).

Consulte **PEP 617** [https://peps.python.org/pep-0617/] para mais detalhes. (Contribuição de Guido van Rossum, Pablo Galindo e Lysandros Nikolaou em bpo-40334 [https://bugs.python.org/issue?@action=redirect&bpo=40334].)

Outras mudanças na linguagem

- `__import__()` agora levanta `ImportError` em vez de `ValueError`, o que é usado para ocorrer quando uma importação relativa passou de seu um pacote de nível superior. (Contribuição de Ngalim Siregar em bpo-37444 [https://bugs.python.org/issue?@action=redirect&bpo=37444].)

- Agora, o Python obtém o caminho absoluto do nome do arquivo do script especificado na linha de comando (p.ex., `python3 script.py`): o atributo `__file__` do módulo `__main__` tornou-se um caminho

absoluto, não um caminho relativo. Esses caminhos agora permanecem válidos depois que o diretório atual é alterado por **os.chdir()**. Como efeito colateral, o traceback também exibe o caminho absoluto para quadros de módulos de **__main__** neste caso. (Contribuição de Victor Stinner em bpo-20443 [https://bugs.python.org/issue?@action=redirect&bpo=20443].)

- No Modo de Desenvolvimento do Python e na compilação de depuração , os argumentos *encoding* e *errors* agora são verificados quanto às operações de codificação e decodificação de strings. Exemplos: **open()**, **str.encode()** e **bytes.decode()**.

  Por padrão, para obter o melhor desempenho, o argumento *errors* é verificado apenas no primeiro erro de codificação/decodificação, e o argumento *encoding* às vezes é ignorado para strings vazias. (Contribuição de Victor Stinner em bpo-37388 [https://bugs.python.org/issue?@action=redirect&bpo=37388].)

- `"".replace("", s, n)` agora retorna `s` em vez de uma string vazia para todos `n` não zero. Isso é agora consistente com `"".replace("", s)`. Há alterações similares para objetos **bytes** e **bytearray**. (Contribuição de Serhiy Storchaka em bpo-28029 [https://bugs.python.org/issue?@action=redirect&bpo=28029].)

- Qualquer expressão válida agora pode ser usada como decorador. Anteriormente, a gramática era muito mais restritiva. Veja **PEP 614** [https://peps.python.org/pep-0614/] para detalhes. (Contribuição de Brandt Bucher em bpo-39702 [https://bugs.python.org/issue?@action=redirect&bpo=39702].)

- Aprimorada a ajuda para o módulo **digitando**. Agora, as linhas de documento são mostradas para todos os formulários especiais e aliases genéricos especiais (como `Union` e `List`). Usando **help()** com alias genérico como `List[int]` mostrará a ajuda para o tipo concreto correspondente (neste caso, `list`). (Contribuição de Serhiy Storchaka em bpo-40257 [https://bugs.python.org/issue?@action=redirect&bpo=40257].)

- A execução paralela de **aclose()** / **asend()** / **athrow()** é agora proibida e `ag_running` agora reflete o status atual de execução do

gerador async. (Contribuição de Yury Selivanov em bpo-30773 [https://bugs.python.org/issue?@action=redirect&bpo=30773].)

- Erros inesperados ao chamar o método `__iter__` não são mais mascarados por `TypeError` no operador `in` e nas funções `contains()`, `indexOf()` e `countOf()` do módulo `operator`. (Contribuição de Serhiy Storchaka em bpo-40824 [https://bugs.python.org/issue?@action=redirect&bpo=40824].)

- Expressões lambda sem parênteses não podem mais ser a parte da expressão em uma cláusula `if` em compreensões e expressões geradoras. Veja bpo-41848 [https://bugs.python.org/issue?@action=redirect&bpo=41848] e bpo-43755 [https://bugs.python.org/issue?@action=redirect&bpo=43755] para detalhes.

Novos módulos

zoneinfo

O módulo `zoneinfo` oferece suporte ao banco de dados de fuso horário da IANA para a biblioteca padrão. Ele adiciona `zoneinfo.ZoneInfo`, uma implementação concreta de `datetime.tzinfo`, baseada nos dados do fuso horário do sistema.

Exemplo:

```
>>> from zoneinfo import ZoneInfo
>>> import datetime as dt

>>> # Horário de verão
>>> when = dt.datetime(2020, 10, 31, 12, tzinfo=ZoneInfo("Americ
>>> print(when)
2020-10-31 12:00:00-07:00
>>> when.tzname()
'PDT'

>>> # Horário padrão
>>> when += dt.timedelta(days=7)
>>> print(when)
```

```
2020-11-07 12:00:00-08:00
>>> print(when.tzname())
PST
```

Como fonte alternativa de dados para plataformas que não entregam o banco de dados da IANA, o módulo tzdata [https://pypi.org/project/tzdata/] foi lançado como um pacote oficial – distribuído via PyPI e mantido pela equipe principal do CPython.

Ver também

PEP 615 [https://peps.python.org/pep-0615/] – Suporte ao banco de dados de fuso horário da IANA na biblioteca padrão
PEP escrita e implementada por Paul Ganssle

graphlib

Um novo módulo, `graphlib`, foi adicionado contendo a classe `graphlib.TopologicalSorter` para oferecer funcionalidade de realizar classificação topológica de grafos. (Contribuição de Pablo Galindo, Tim Peters e Larry Hastings em bpo-17005 [https://bugs.python.org/issue?@action=redirect&bpo=17005].)

Módulos melhorados

ast

Adicionada a opção *indent* a `dump()`, que permite produzir uma saída indentada com várias linhas. (Contribuição de Serhiy Storchaka em bpo-37995 [https://bugs.python.org/issue?@action=redirect&bpo=37995].)

Adicionada `ast.unparse()` como uma função no módulo `ast` que pode ser usado para remover um objeto `ast.AST` e produzir uma string com código que produziria um objeto `ast.AST` equivalente quando analisado.

(Contribuição de Pablo Galindo e Batuhan Taskaya em bpo-38870 [https://bugs.python.org/issue?@action=redirect&bpo=38870].)

Adicionadas docstrings aos nós AST que contêm a assinatura ASDL usada para construir esse nó. (Contribuição de Batuhan Taskaya em bpo-39638 [https://bugs.python.org/issue?@action=redirect&bpo=39638].)

asyncio

Devido a significativas preocupações de segurança, o parâmetro *reuse_address* de **`asyncio.loop.create_datagram_endpoint()`** não é mais suportado. Isso ocorre devido ao comportamento da opção de soquete `SO_REUSEADDR` no UDP. Para mais detalhes, consulte a documentação de `loop.create_datagram_endpoint()`. (Contribuição de Kyle Stanley, Antoine Pitrou e Yury Selivanov em bpo-37228 [https://bugs.python.org/issue?@action=redirect&bpo=37228].)

Adicionada uma nova corrotina **`shutdown_default_executor()`** que agenda um desligamento para o executor padrão que espera na **`ThreadPoolExecutor`** para concluir o fechamento. Além disso, **`asyncio.run()`** foi atualizada para usar o novo corrotina. (Contribuição de Kyle Stanley em bpo-34037 [https://bugs.python.org/issue?@action=redirect&bpo=34037].)

Adicionada **`asyncio.PidfdChildWatcher`**, uma implementação de monitorador de filhos específica do Linux que controla descritores de arquivo de processo. (bpo-38692 [https://bugs.python.org/issue?@action=redirect&bpo=38692])

Adicionada uma nova corrotina **`asyncio.to_thread()`**. É usada principalmente para executar funções vinculadas à E/S em uma thread separada para evitar o bloqueio do loop de eventos e funciona essencialmente como uma versão de alto nível de **`run_in_executor()`** que pode receber argumentos nomeados diretamente. (Contribuição de Kyle Stanley e Yury Selivanov em bpo-32309 [https://bugs.python.org/issue?@action=redirect&bpo=32309].)

Ao cancelar a tarefa devido a um tempo limite, `asyncio.wait_for()` irá agora esperar até que o cancelamento seja concluído também no caso quando *timeout* é <= 0, como acontece com tempos limite positivos. (Contribuição de Elvis Pranskevichus em bpo-32751 [https://bugs.python.org/issue?@action=redirect&bpo=32751].)

`asyncio` agora levanta `TypeError` ao chamar métodos incompatíveis com um soquete `ssl.SSLSocket`. (Contribuição de Ido Michael em bpo-37404 [https://bugs.python.org/issue?@action=redirect&bpo=37404].)

compileall

Adicionada nova possibilidade de usar hardlinks para arquivos `.pyc` duplicados: parâmetro *hardlink_dupes* e opção de linha de comando –hardlink-dupes. (Contribuição de Lumír 'Frenzy' Balhar em bpo-40495 [https://bugs.python.org/issue?@action=redirect&bpo=40495].)

Adicionadas novas opções para manipulação de caminho nos arquivos `.pyc` resultantes: parâmetros *stripdir*, *prependdir*, *limit_sl_dest* e opções de linha de comando -s, -p, -e. Foi adicionada a possibilidade de especificar a opção para um nível de otimização várias vezes. (Contribuição de Lumír 'Frenzy' Balhar em bpo-38112 [https://bugs.python.org/issue?@action=redirect&bpo=38112].)

concurrent.futures

Adicionado um novo parâmetro *cancel_futures* em `concurrent.futures.Executor.shutdown()` que cancela todos os futuros pendentes que ainda não começaram a ser executados, em vez de esperar que eles sejam concluídos antes de desligar o executor. (Contribuição de Kyle Stanley em bpo-39349 [https://bugs.python.org/issue?@action=redirect&bpo=39349].)

Removidos as threads do daemon de `ThreadPoolExecutor` e `ProcessPoolExecutor`. Isso melhora a compatibilidade com subinterpretadores e a previsibilidade em seus processos de desligamento. (Contribuição de Kyle Stanley em bpo-39812 [https://bugs.python.org/issue?@action=redirect&bpo=39812].)

Workers em `ProcessPoolExecutor` agora são gerados sob demanda, apenas quando não há workers ociosos disponíveis para reutilização. Isso otimiza a sobrecarga da inicialização e reduz a quantidade de tempo perdido da CPU para os workers ociosos. (Contribuição de Kyle Stanley em bpo-39207 [https://bugs.python.org/issue?@action=redirect&bpo=39207].)

curses

Adicionadas as funções `curses.get_escdelay()`, `curses.set_escdelay()`, `curses.get_tabsize()` e `curses.set_tabsize()`. (Contribuição de Anthony Sottile em bpo-38312 [https://bugs.python.org/issue?@action=redirect&bpo=38312].)

datetime

Os métodos `isocalendar()` de `datetime.date` e `isocalendar()` de `datetime.datetime` agora retornam uma `namedtuple()` em vez de uma `tuple`. (Contribuição de Donghee Na em bpo-24416 [https://bugs.python.org/issue?@action=redirect&bpo=24416].)

distutils

O comando **upload** agora cria digests de hash SHA2-256 e Blake2b-256. Ele pula o MD5 em plataformas que bloqueiam digest do MD5. (Contribuição de Christian Heimes em bpo-40698 [https://bugs.python.org/issue?@action=redirect&bpo=40698].)

fcntl

Adicionadas as constantes `fcntl.F_OFD_GETLK`, `fcntl.F_OFD_SETLK` e `fcntl.F_OFD_SETLKW`. (Contribuição de Donghee Na em bpo-38602 [https://bugs.python.org/issue?@action=redirect&bpo=38602].)

ftplib

`FTP` e `FTP_TLS` agora levantam a `ValueError` se o tempo limite especificado para o construtor for zero para impedir a criação de um soquete não-bloqueante. (Contribuição de Donghee Na em bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259].)

gc

Quando o coletor de lixo faz uma coleção na qual alguns objetos ressuscitam (são alcançáveis fora dos ciclos isolados após a execução dos finalizadores), não bloqueia a coleção de todos os objetos que ainda estão inacessíveis. (Contribuição de Pablo Galindo e Tim Peters em bpo-38379 [https://bugs.python.org/issue?@action=redirect&bpo=38379].)

Adicionada uma nova função `gc.is_finalized()` para verificar se um objeto foi finalizado pelo coletor de lixo. (Contribuição de Pablo Galindo em bpo-39322 [https://bugs.python.org/issue?@action=redirect&bpo=39322].)

hashlib

O módulo `hashlib` pode agora usar hashes SHA3 e SHAKE XOF de OpenSSL quando disponível. (Contribuição de Christian Heimes em bpo-37630 [https://bugs.python.org/issue?@action=redirect&bpo=37630].)

Os módulos hash integrados agora podem ser desativados com `./configure --without-builtin-hashlib-hashes` ou ativados seletivamente com por exemplo `./configure --with-builtin-hashlib-hashes=sha3,blake2` para forçar o uso da implementação baseada em OpenSSL. (Contribuição de Christian Heimes em bpo-40479 [https://bugs.python.org/issue?@action=redirect&bpo=40479])

http

Códigos de status HTTP `103 EARLY_HINTS`, `418 IM_A_TEAPOT` e `425 TOO_EARLY` foram adicionados a `http.HTTPStatus`. (Contribuição de Donghee Na em bpo-39509 [https://bugs.python.org/issue?@action=redirect&bpo=39509] e Ross Rhodes em bpo-39507 [https://bugs.python.org/issue?@action=redirect&bpo=39507].)

IDLE e idlelib

Adicionada a opção para ativar e desativar o piscar do cursor. (Contribuição de Zackery Spytz na bpo-4603 [https://bugs.python.org/issue?@action=redirect&bpo=4603].)

A tecla Escape agora fecha as janelas de conclusão IDLE. (Contribuição de Johnny Najera em bpo-38944 [https://bugs.python.org/issue?@action=redirect&bpo=38944].)

Adicionada argumentos nomeados à lista de completamento de nome de módulo. (Contribuição de Terry J. Reedy em bpo-37765 [https://bugs.python.org/issue?@action=redirect&bpo=37765].)

Novo nas versões de manutenção 3.9.

Faz o IDLE invocar `sys.excepthook()` (quando iniciado sem '-n'). Ganchos de usuário eram ignorados anteriormente. (Contribuição de Ken Hilton em bpo-43008 [https://bugs.python.org/issue?@action=redirect&bpo=43008].)

As alterações acima foram portadas para versões de manutenção do 3.8.

Reorganiza a caixa de diálogo de configurações. Divide a aba General nas abas Windows e Shell/Ed. Move as fontes de ajuda, que estendem o menu Help, para a aba Extensions. Abre espaço para novas opções e encurta a caixa de diálogo. O último faz com que o diálogo se ajuste melhor a telas pequenas. (Contribuição de Terry Jan Reedy em bpo-40468 [https://bugs.python.org/issue?@action=redirect&bpo=40468].) Move a configuração do espaço de recuo da aba Font para a nova aba Windows. (Contribuição de Mark Roseman e Terry Jan Reedy em bpo-33962 [https://bugs.python.org/issue?@action=redirect&bpo=33962].)

Aplica realce de sintaxe em arquivos `.pyi`. (Contribuição de Alex Waygood e Terry Jan Reedy em bpo-45447 [https://bugs.python.org/issue?@action=redirect&bpo=45447].)

imaplib

`IMAP4` e **`IMAP4_SSL`** agora possuem o parâmetro opcional *timeout* para seus construtores. Também, o método **`open()`** agora possui um parâmetro opcional *timeout* com esta alteração. Os métodos substituídos de **`IMAP4_SSL`** e **`IMAP4_stream`** foram aplicados nesta alteração. (Contribuição de Dong-hee Na em bpo-38615 [https://bugs.python.org/issue?@action=redirect&bpo=38615].)

`imaplib.IMAP4.unselect()` foi adicionado. **`imaplib.IMAP4.unselect()`** libera os recursos do servidor associados à caixa de correio selecionada e retorna o servidor ao estado autenticado. Este comando executa as mesmas ações que **`imaplib.IMAP4.close()`**, exceto que nenhuma mensagem é removida permanentemente da caixa de correio selecionada no momento. (Contribuição de Dong-hee Na em bpo-40375 [https://bugs.python.org/issue?@action=redirect&bpo=40375].)

importlib

Para melhorar a consistência com as instruções de importação, **`importlib.util.resolve_name()`** agora levanta **`ImportError`** em vez de **`ValueError`** para tentativas de importação relativas inválidas. (Contribuição de Ngalim Siregar em bpo-37444 [https://bugs.python.org/issue?@action=redirect&bpo=37444].)

Carregadores de importação que publicam objetos de módulo imutáveis agora podem publicar pacotes imutáveis além de módulos individuais. (Contribuição de Dino Viehland em bpo-39336 [https://bugs.python.org/issue?@action=redirect&bpo=39336].)

Adicionada a função **`importlib.resources.files()`** com suporte a subdiretórios nos dados do pacote, correspondendo ao backport na versão 1.5 do `importlib_resources`. (Contribuição de Jason R. Coombs em bpo-39791 [https://bugs.python.org/issue?@action=redirect&bpo=39791].)

Renovado `importlib.metadata` de `importlib_metadata` versão 1.6.1.

inspect

`inspect.BoundArguments.arguments` foi alterado de `OrderedDict` para dict regular. (Contribuição de Inada Naoki em bpo-36350 [https://bugs.python.org/issue?@action=redirect&bpo=36350] e bpo-39775 [https://bugs.python.org/issue?@action=redirect&bpo=39775].)

ipaddress

`ipaddress` agora suporta endereços IPv6 com escopo definido (endereço IPv6 com sufixo `%<scope_id>`).

Os endereços IPv6 com escopo definido podem ser analisados usando `ipaddress.IPv6Address`. Se presente, o ID da zona de escopo está disponível através do atributo `scope_id`. (Contribuição de Oleksandr Pavliuk em bpo-34788 [https://bugs.python.org/issue?@action=redirect&bpo=34788].)

A partir do Python 3.9.5, o módulo `ipaddress` não aceita mais zeros à esquerda em strings de endereço IPv4. (Contribuição de Christian Heimes em bpo-36384 [https://bugs.python.org/issue?@action=redirect&bpo=36384]).

math

Expandida a função `math.gcd()` para manipular vários argumentos. Anteriormente, tinha suporte apenas a dois argumentos. (Contribuição de Serhiy Storchaka em bpo-39648 [https://bugs.python.org/issue?@action=redirect&bpo=39648].)

Adicionada `math.lcm()`: retorna o menor múltiplo comum dos argumentos especificados. (Contribuição de Mark Dickinson, Ananthakrishnan e Serhiy Storchaka em bpo-39479 [https://bugs.python.org/issue?@action=redirect&bpo=39479] e :issue:` 39648`.)

Adicionada `math.nextafter()`: retorna o próximo valor de ponto flutuante após *x* em direção a *y*. (Contribuição de Victor Stinner em bpo-39288 [https://bugs.python.org/issue?@action=redirect&bpo=39288].)

Adicionada `math.ulp()`: retorna o valor do bit menos significativo de um ponto flutuante. (Contribuição de Victor Stinner em bpo-39310

[https://bugs.python.org/issue?@action=redirect&bpo=39310].)

multiprocessing

A classe `multiprocessing.SimpleQueue` tem um novo método `close()` para fechar explicitamente a fila. (Contribuição de Victor Stinner em bpo-30966 [https://bugs.python.org/issue?@action=redirect&bpo=30966].)

nntplib

`NNTP` e `NNTP_SSL` agora levantam uma exceção `ValueError` se o tempo limite fornecido ao construtor é zero, para prevenir a criação de um socket não-bloqueante. (Contribuição de Donghee Na em bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259].)

os

Adicionadas as constantes `CLD_KILLED` e `CLD_STOPPED` para `si_code`. (Contribuição de Donghee Na em bpo-38493 [https://bugs.python.org/issue?@action=redirect&bpo=38493].)

Exposta `os.pidfd_open()` específica do Linux (bpo-38692 [https://bugs.python.org/issue?@action=redirect&bpo=38692]) e `os.P_PIDFD` (bpo-38713 [https://bugs.python.org/issue?@action=redirect&bpo=38713]) para gerenciamento de processos com descritores de arquivo.

A função `os.unsetenv()` agora também está disponível no Windows. (Contribuição de Victor Stinner em bpo-39413 [https://bugs.python.org/issue?@action=redirect&bpo=39413].)

As funções `os.putenv()` e `os.unsetenv()` estão agora sempre disponíveis. (Contribuição de Victor Stinner em bpo-39395 [https://bugs.python.org/issue?@action=redirect&bpo=39395].)

Adicionada a função `os.waitstatus_to_exitcode()`: converte um status de espera em um código de saída. (Contribuição de Victor Stinner em bpo-40094 [https://bugs.python.org/issue?@action=redirect&bpo=40094].)

pathlib

Adicionado `pathlib.Path.readlink()`, que age de maneira semelhante a `os.readlink()`. (Contribuição de Girts Folkmanis em bpo-30618 [https://bugs.python.org/issue?@action=redirect&bpo=30618])

pdb

No Windows, agora `Pdb` tem suporte a `~/.pdbrc`. (Contribuição de Tim Hopper e Dan Lidral-Porter em bpo-20523 [https://bugs.python.org/issue?@action=redirect&bpo=20523].)

poplib

`POP3` e `POP3_SSL` agora levantam a `ValueError` se o tempo limite especificado para o construtor for zero para impedir a criação de um soquete sem bloqueio. (Contribuição de Donghee Na em bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259].)

pprint

`pprint` agora pode imprimir `types.SimpleNamespace`. (Contribuição de Carl Bordum Hansen em bpo-37376 [https://bugs.python.org/issue?@action=redirect&bpo=37376].)

pydoc

A string de documentação agora é mostrada não apenas para classe, função, método etc, mas para qualquer objeto que tenha seu próprio atributo `__doc__`. (Contribuição de Serhiy Storchaka em bpo-40257 [https://bugs.python.org/issue?@action=redirect&bpo=40257].)

random

Adicionado um novo método `random.Random.randbytes()`: gera bytes aleatórios. (Contribuição de Victor Stinner em bpo-40286

[https://bugs.python.org/issue?@action=redirect&bpo=40286].)

signal

Exposta a `signal.pidfd_send_signal()` específica do Linux para enviar sinais a um processo usando um descritor de arquivo em vez de um pid. (bpo-38712 [https://bugs.python.org/issue?@action=redirect&bpo=38712])

smtplib

`SMTP` e `SMTP_SSL` agora levantam uma `ValueError` se o tempo limite especificado para seu construtor for zero para impedir a criação de um soquete sem bloqueio. (Contribuição de Dong-hee Na em bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259].)

O construtor de `LMTP` agora tem um parâmetro opcional *timeout*. (Contribuição de Dong-hee Na em bpo-39329 [https://bugs.python.org/issue?@action=redirect&bpo=39329].)

socket

O módulo `socket` agora exporta a constante `CAN_RAW_JOIN_FILTERS` no Linux 4.1 e superior. (Contribuição de Stefan Tatschner e Zackery Spytz em bpo-25780 [https://bugs.python.org/issue?@action=redirect&bpo=25780].)

O módulo socket agora suporta o protocolo `CAN_J1939` nas plataformas que o suportam. (Contribuição de Karl Ding em bpo-40291 [https://bugs.python.org/issue?@action=redirect&bpo=40291].)

O módulo socket agora tem as funções `socket.send_fds()` e `socket.recv_fds()`. (Contribuição de Joannah Nanjekye, Shinya Okano e Victor Stinner em bpo-28724 [https://bugs.python.org/issue?@action=redirect&bpo=28724].)

time

No AIX, `thread_time()` agora é implementado com `thread_cputime()` que possui resolução de nanossegundos, em vez de `clock_gettime(CLOCK_THREAD_CPUTIME_ID)`, que tem uma resolução de 10 milissegundos. (Contribuição de Batuhan Taskaya em bpo-40192 [https://bugs.python.org/issue?@action=redirect&bpo=40192])

sys

Adicionado um novo atributo `sys.platlibdir`: nome do diretório da biblioteca específica da plataforma. É usado para construir o caminho da biblioteca padrão e os caminhos dos módulos de extensão instalados. É igual a `"lib"` na maioria das plataformas. No Fedora e SuSE, é igual a `"lib64"` nas plataformas de 64 bits. (Contribuição de Jan Matějek, Matěj Cepl, Charalampos Stratakis e Victor Stinner em bpo-1294959 [https://bugs.python.org/issue?@action=redirect&bpo=1294959].)

Anteriormente, `sys.stderr` era armazenado em buffer quando não interativo. Agora, `stderr` assume como padrão sempre o buffer de linha. (Contribuição de Jendrik Seipp em bpo-13601 [https://bugs.python.org/issue?@action=redirect&bpo=13601].)

tracemalloc

Adicionada `tracemalloc.reset_peak()` para definir o tamanho do pico dos blocos de memória rastreados para o tamanho atual, para medir o pico de partes específicas do código. (Contribuição de Huon Wilson em bpo-40630 [https://bugs.python.org/issue?@action=redirect&bpo=40630].)

typing

PEP 593 [https://peps.python.org/pep-0593/] introduziu um tipo `typing.Annotated` para decorar tipos existentes com metadados específicos de contexto e o novo parâmetro `include_extras` para `typing.get_type_hints()` para acessar os metadados em tempo de execução. (Contribuição de Till Varoquaux e Konstantin Kashin.)

unicodedata

O banco de dados Unicode foi atualizado para a versão 13.0.0. (bpo-39926 [https://bugs.python.org/issue?@action=redirect&bpo=39926]).

venv

Os scripts de ativação fornecidos por `venv` agora especificam sua personalização de prompt de forma consistente, sempre usando o valor especificado por `__VENV_PROMPT__`. Anteriormente, alguns scripts usavam incondicionalmente `__VENV_PROMPT__`, outros apenas se fosse definido (que era o caso padrão) e um usava `__VENV_NAME__`. (Contribuição de Brett Cannon em bpo-37663 [https://bugs.python.org/issue?@action=redirect&bpo=37663].)

xml

Os caracteres de espaço em branco nos atributos agora são preservados ao serializar `xml.etree.ElementTree` para o arquivo XML. EOLNs não são mais normalizados para “n”. Este é o resultado da discussão sobre como interpretar a seção 2.11 da especificação XML. (Contribuição de Mefistotelis em bpo-39011 [https://bugs.python.org/issue?@action=redirect&bpo=39011].)

Otimizações

- Otimizado o idioma para atribuição de uma variável temporária nas compreensões. Agora `for y in [expr]` nas compreensões é tão rápido quanto uma atribuição simples `y = expr`. Por exemplo:

  sums = [s for s in [0] for x in data for s in [s + x]]

  Ao contrário do operador `:=`, este idioma não vaza uma variável para o escopo externo.

  (Contribuição de Serhiy Storchaka em bpo-32856 [https://bugs.python.org/issue?@action=redirect&bpo=32856].)

- Otimizada a manipulação de sinais em aplicações multithread. Se uma thread diferente da thread principal obtiver um sinal, o loop de avaliação do bytecode não será mais interrompido a cada instrução de

bytecode para verificar se há sinais pendentes que não podem ser manipulados. Somente o segmento principal do interpretador principal pode manipular sinais.

Anteriormente, o loop de avaliação do bytecode era interrompido em cada instrução até que a thread principal lidasse com os sinais. (Contribuição de Victor Stinner em bpo-40010 [https://bugs.python.org/issue?@action=redirect&bpo=40010].)

- Otimizado o módulo `subprocess` no FreeBSD usando `closefrom()`. (Contribuição de Ed Maste, Conrad Meyer, Kyle Evans, Kubilay Kocak e Victor Stinner em bpo-38061 [https://bugs.python.org/issue?@action=redirect&bpo=38061].)

- `PyLong_FromDouble()` está agora até 1.87x mais rápido para valores que cabem em um long. (Contribuição de Sergey Fedoseev em bpo-37986 [https://bugs.python.org/issue?@action=redirect&bpo=37986].)

- Vários componentes embutidos do Python (`range`, `tuple`, `set`, `frozenset`, `list`, `dict`) agora são acelerados usando o protocolo vectorcall da **PEP 590** [https://peps.python.org/pep-0590/]. (Contribuição de Donghee Na, Mark Shannon, Jeroen Demeyer e Petr Viktorin em bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207].)

- Otimizada a `set.difference_update()` para o caso em que o outro conjunto é muito maior do que o conjunto base. (Sugestão de Evgeny Kapun com contribuição de código de Michele Orrù em bpo-8425 [https://bugs.python.org/issue?@action=redirect&bpo=8425].)

- O alocador de pequenos objetos do Python (`obmalloc.c`) agora permite (não mais que) uma arena vazia para permanecer disponível para reutilização imediata, sem retorná-la ao sistema operacional. Isso evita o desperdiçar loops simples, onde uma arena pode ser criada e destruída novamente em cada iteração. (Contribuição de Tim Peters em bpo-37257 [https://bugs.python.org/issue?@action=redirect&bpo=37257].)

- divisão pelo piso de operação flutuante agora tem um melhor desempenho. Além disso, a mensagem de `ZeroDivisionError` para

esta operação foi atualizada. (Contribuição de Donghee Na em bpo-39434 [https://bugs.python.org/issue?@action=redirect&bpo=39434].)

- A decodificação de strings curtas ASCII com codecs UTF-8 e ascii é agora cerca de 15% mais rápida. (Contribuição de Inada Naoki em bpo-37348 [https://bugs.python.org/issue?@action=redirect&bpo=37348].)

Aqui está um resumo das melhorias de desempenho do Python 3.4 ao Python 3.9:

Versão do Python	3.4	3.5	3.6	3.7
Acesso de leitura a variável e atributo:				
read_local	7.1	7.1	5.4	5.1
read_nonlocal	7.1	8.1	5.8	5.4
read_global	15.5	19.0	14.3	13.6
read_builtin	21.1	21.6	18.5	19.0
read_classvar_from_class	25.6	26.5	20.7	19.5
read_classvar_from_instance	22.8	23.5	18.8	17.1
read_instancevar	32.4	33.1	28.0	26.3
read_instancevar_slots	27.8	31.3	20.8	20.8
read_namedtuple	73.8	57.5	45.0	46.8
read_boundmethod	37.6	37.9	29.6	26.9
Acesso de escrita a variável e atributo:				
write_local	8.7	9.3	5.5	5.3
write_nonlocal	10.5	11.1	5.6	5.5
write_global	19.7	21.2	18.0	18.0
write_classvar	92.9	96.0	104.6	102.1
write_instancevar	44.6	45.8	40.0	38.9
write_instancevar_slots	35.6	36.1	27.3	26.6
Acesso de leitura a estrutura de dados:				
read_list	24.2	24.5	20.8	20.8
read_deque	24.7	25.5	20.2	20.6
read_dict	24.3	25.7	22.3	23.0
read_strdict	22.6	24.3	19.5	21.2
Acesso de escrita a estrutura de dados:				
write_list	27.1	28.5	22.5	21.6
write_deque	28.7	30.1	22.7	21.8

```
    write_dict                        31.4    33.3    29.3    29.2
    write_strdict                     28.4    29.9    27.5    25.2

Operações em pilha (ou fila):
    list_append_pop                   93.4   112.7    75.4    74.2
    deque_append_pop                  43.5    57.0    49.4    49.2
    deque_append_popleft              43.7    57.3    49.7    49.7

Tempo de laços:
    loop_overhead                      0.5     0.6     0.4     0.3
```

Estes resultados foram gerados a partir do script de benchmark de acesso variável em: `Tools/scripts/var_access_benchmark.py`. O script de benchmark exibe tempos em nanossegundos. Os benchmarks foram medidos em um processador Intel® Core ™ i7-4960HQ [https://ark.intel.com/content/www/us/en/ark/products/76088/intel-core-i7-4960hq-processor-6m-cache-up-to-3-80-ghz.html] executando as compilações do macOS de 64 bits encontradas em python.org [https://www.python.org/downloads/macos/].

Descontinuados

- O comando `bdist_msi` do distutils está agora descontinuado, use `bdist_wheel` (pacotes wheel). (Contribuição de Hugo van Kemenade em bpo-39586 [https://bugs.python.org/issue?@action=redirect&bpo=39586].)

- Atualmente **`math.factorial()`** aceita instâncias da **`float`** com valores inteiros não negativos (como `5.0`). Ela levanta uma **`ValueError`** para flutuadores não integrais e negativos. Agora está descontinuado. Nas versões futuras do Python, será levantada uma **`TypeError`** para todos os flutuadores. (Contribuição de Serhiy Storchaka em bpo-37315 [https://bugs.python.org/issue?@action=redirect&bpo=37315].)

- Os módulos **`parser`** e **`symbol`** foram descontinuados e serão removidos em versões futuras do Python. Para a maioria dos casos de uso, os usuários podem aproveitar o estágio de geração e compilação da Árvore de Sintaxe Abstrata (AST), usando o módulo **`ast`**.

- As funções públicas de API C **`PyParser_SimpleParseStringFlags()`**, **`PyParser_SimpleParseStringFlagsFilename()`**, **`PyParser_SimpleParseFileFlags()`** e **`PyNode_Compile()`** foram descontinuados e serão removidos no Python 3.10 junto com o analisador sintático antigo.

- O uso de **`NotImplemented`** em um contexto booleano foi descontinuado, pois é quase exclusivamente o resultado de implementações incorretas de comparadores avançados. Isso será tornado em uma **`TypeError`** em uma versão futura do Python. (Contribuição de Josh Rosenberg em bpo-35712 [https://bugs.python.org/issue?@action=redirect&bpo=35712].)

- O módulo **`random`** atualmente aceita qualquer tipo hasheável como um possível valor inicial. Infelizmente, alguns desses tipos não têm garantia de ter um valor de hash determinístico. Após o Python 3.9, o módulo restringirá suas sementes a **`None`**, **`int`**, **`float`**, **`str`**, **`bytes`** e **`bytearray`**.

- A abertura do arquivo **`GzipFile`** para escrita sem especificar o argumento *mode* foi descontinuado. Nas versões futuras do Python, ele sempre será aberto para leitura por padrão. Especifique o argumento *mode* para abri-lo para escrever e silenciar um aviso. (Contribuição de Serhiy Storchaka em bpo-28286 [https://bugs.python.org/issue?@action=redirect&bpo=28286].)

- Descontinuado o método `split()` de **`_tkinter.TkappType`** a favor do método `splitlist()` que possui um comportamento mais consistente e previsível. (Contribuição de Serhiy Storchaka em bpo-38371 [https://bugs.python.org/issue?@action=redirect&bpo=38371].)

- A passagem explícita dos objetos da corrotina para **`asyncio.wait()`** foi descontinuada e será removida na versão 3.11. (Contribuição de Yury Selivanov e Kyle Stanley em bpo-34790 [https://bugs.python.org/issue?@action=redirect&bpo=34790].)

- Os padrões binhex4 e hexbin4 agora foram descontinuados. O módulo **`binhex`** e as seguintes funções **`binascii`** foram descontinuadas:

  - **`b2a_hqx()`**, **`a2b_hqx()`**
  - **`rlecode_hqx()`**, **`rledecode_hqx()`**

  (Contribuição de Victor Stinner em bpo-39353 [https://bugs.python.org/issue?@action=redirect&bpo=39353].)

- As classes `slice`, `Index` e `ExtSlice` de **`ast`** são consideradas descontinuadas e serão removidas nas versões futuras do Python. O próprio `value` deve ser usado em vez de `Index(value)`. `Tuple(slices, Load())` deve ser usada em vez de `ExtSlice(slices)`. (Contribuição de Serhiy Storchaka em bpo-34822 [https://bugs.python.org/issue?@action=redirect&bpo=34822].)

- As classes `Suite`, `Param`, `AugLoad` e `AugStore` de **`ast`** são consideradas descontinuadas e serão removidas nas versões futuras do Python. Elas não foram geradas pelo analisador sintático e não foram aceitos pelo gerador de código no Python 3. (Contribuição de Batuhan Taskaya em bpo-39639 [https://bugs.python.org/issue?@action=redirect&bpo=39639] e bpo-39969 [https://bugs.python.org/issue?@action=redirect&bpo=39969] e Serhiy Storchaka em bpo-39988 [https://bugs.python.org/issue?@action=redirect&bpo=39988].)

- As funções **`PyEval_InitThreads()`** e **`PyEval_ThreadsInitialized()`** agora estão descontinuadas e serão removidas no Python 3.11. Chamar **`PyEval_InitThreads()`** agora não faz nada. A GIL é inicializada por **`Py_Initialize()`** desde o Python 3.7. (Contribuição de Victor Stinner em bpo-39877 [https://bugs.python.org/issue?@action=redirect&bpo=39877].)

- Passar `None` como o primeiro argumento para a função **`shlex.split()`** foi descontinuada. (Contribuição de Zackery Spytz em bpo-33262 [https://bugs.python.org/issue?@action=redirect&bpo=33262].)

- **`smtpd.MailmanProxy()`** está agora descontinuado, pois está inutilizável sem um módulo externo, `mailman`. (Contribuição de Samuel Colvin em bpo-35800 [https://bugs.python.org/issue?@action=redirect&bpo=35800].)

- O módulo **`lib2to3`** agora emite um **`PendingDeprecationWarning`**. Python 3.9 mudou para um analisador sintático GASE (veja **PEP 617**

[https://peps.python.org/pep-0617/]) e Python 3.10 pode incluir uma nova sintaxe de linguagem que não é analisável pelo analisador sintático LL(1) de lib2to3. O módulo **`lib2to3`** pode ser removido da biblioteca padrão em uma versão futura do Python. Considere alternativas de terceiros, como LibCST [https://libcst.readthedocs.io/] ou parso [https://parso.readthedocs.io/]. (Contribuição de Carl Meyer em bpo-40360 [https://bugs.python.org/issue?@action=redirect&bpo=40360].)

- Adicionado o parâmetro *random* da **`random.shuffle()`** foi descontinuado. (Contribuição de Raymond Hettinger em bpo-40465 [https://bugs.python.org/issue?@action=redirect&bpo=40465])

Removidos

- A versão errônea em **`unittest.mock.__version__`** foi removida.
- **`nntplib.NNTP`**: os métodos `xpath()` e `xgtitle()` foram removidos. Esses métodos foram descontinuados desde Python 3.3. Geralmente, essas extensões não são suportadas ou não são habilitadas pelos administradores do servidor NNTP. Para `xgtitle()`, use **`nntplib.NNTP.descriptions()`** ou **`nntplib.NNTP.description()`**. (Contribuição de Donghee Na em bpo-39366 [https://bugs.python.org/issue?@action=redirect&bpo=39366].)
- **`array.array`**: os métodos `tostring()` e `fromstring()` foram removidos. Eles eram aliases para `tobytes()` e `frombytes()`, descontinuado desde Python 3.2. (Contribuição de Victor Stinner em bpo-38916 [https://bugs.python.org/issue?@action=redirect&bpo=38916].)
- A função `sys.callstats()` não documentada foi removida. Desde o Python 3.7, ela foi descontinuada e sempre retornou **`None`**. É necessária uma opção de compilação especial `CALL_PROFILE` que já foi removida no Python 3.7. (Contribuição de Victor Stinner em bpo-37414 [https://bugs.python.org/issue?@action=redirect&bpo=37414].)
- As funções `sys.getcheckinterval()` e `sys.setcheckinterval()` foram removidas. Elas foram descontinuadas desde o Python 3.2. Use **`sys.getswitchinterval()`** e **`sys.setswitchinterval()`** em seu lugar. (Contribuição de Victor Stinner em bpo-37392 [https://bugs.python.org/issue?@action=redirect&bpo=37392].)

- A função C `PyImport_Cleanup()` foi removida. Foi documentada como: “Esvazia a tabela do módulo. Apenas para uso interno.” (Contribuição de Victor Stinner em bpo-36710 [https://bugs.python.org/issue?@action=redirect&bpo=36710].)
- Os módulos `_dummy_thread` e `dummy_threading` foram removidos. Esses módulos foram descontinuados desde o Python 3.7, que requer suporte a threading. (Contribuição de Victor Stinner em bpo-37312 [https://bugs.python.org/issue?@action=redirect&bpo=37312].)
- O apelido `aifc.openfp()` para `aifc.open()`, o apelido `sunau.openfp()` para `sunau.open()` e o apelido `wave.openfp()` para **`wave.open()`** foram removidos. Eles estavam descontinuados desde o Python 3.7. (Contribuição de Victor Stinner em bpo-37320 [https://bugs.python.org/issue?@action=redirect&bpo=37320].)
- O método **`isAlive()`** de **`threading.Thread`** foi removido. Ele foi descontinuado desde o Python 3.8. Use **`is_alive()`** ao invés. (Contribuição de Donghee Na em bpo-37804 [https://bugs.python.org/issue?@action=redirect&bpo=37804].)
- Os métodos `getchildren()` e `getiterator()` das classes **`ElementTree`** e **`Element`** no módulo **`ElementTree`** foram removidos. Eles foram descontinuados no Python 3.2. Use `iter(x)` ou `list(x)` em vez de `x.getchildren()` e `x.iter()` ou `list(x.iter())` em vez de `x.getiterator()`. (Contribuição Serhiy Storchaka no bpo-36543 [https://bugs.python.org/issue?@action=redirect&bpo=36543].)
- A API antiga **`plistlib`** foi removida. Ela tinha sido descontinuada desde o Python 3.4. Use as funções **`load()`**, **`loads()`**, **`dump()`** e **`dumps()`**. Além disso, o parâmetro *use_builtin_types* foi removido, os objetos padrão **`bytes`** são sempre usados em seu lugar. (Contribuição de Jon Janzen em bpo-36409 [https://bugs.python.org/issue?@action=redirect&bpo=36409].)
- A função C `PyGen_NeedsFinalizing` foi removida. Ela não foi documentada, testada ou usada em qualquer lugar dentro do CPython após a implementação de **PEP 442** [https://peps.python.org/pep-0442/]. Patch de Joannah Nanjekye. (Contribuição de Joannah Nanjekye em bpo-15088 [https://bugs.python.org/issue?@action=redirect&bpo=15088])
- `base64.encodestring()` e `base64.decodestring()`, aliases descontinuados desde Python 3.1, foram removidos: use **`base64.encodebytes()`** e **`base64.decodebytes()`**. (Contribuição de

Victor Stinner em bpo-39351 [https://bugs.python.org/issue?@action=redirect&bpo=39351].)

- A função `fractions.gcd()` foi removida, foi descontinuado desde Python 3.5 (bpo-22486 [https://bugs.python.org/issue?@action=redirect&bpo=22486]): use **`math.gcd()`** em seu lugar. (Contribuição de Victor Stinner em bpo-39350 [https://bugs.python.org/issue?@action=redirect&bpo=39350].)
- O parâmetro *buffering* de **`bz2.BZ2File`** foi removido. Desde Python 3.0, ele era ignorado e ao usá-lo emitia uma **`DeprecationWarning`**. Passe um objeto arquivo aberto para controlar como o arquivo é aberto. (Contribuição de Victor Stinner em bpo-39357 [https://bugs.python.org/issue?@action=redirect&bpo=39357].)
- O parâmetro *encoding* de **`json.loads()`** foi removido. A partir do Python 3.1, ele foi descontinuado e ignorado; usá-lo emitia uma **`DeprecationWarning`** desde Python 3.8. (Contribuição de Inada Naoki em bpo-39377 [https://bugs.python.org/issue?@action=redirect&bpo=39377])
- As instruções `with (await asyncio.lock):` e `with (yield from asyncio.lock):` não são mais suportadas, use `async with lock`. O mesmo é se aplica para `asyncio.Condition` e `asyncio.Semaphore`. (Contribuição de Andrew Svetlov em bpo-34793 [https://bugs.python.org/issue?@action=redirect&bpo=34793].)
- A função **`sys.getcounts()`**, a opção de linha de comando `-X showalloccount` e o campo `show_alloc_count` da estrutura C **`PyConfig`** foram removidos. Eles exigiam uma construção especial do Python definindo a macro `COUNT_ALLOCS`. (Contribuição de Victor Stinner em bpo-39489 [https://bugs.python.org/issue?@action=redirect&bpo=39489].)
- O atributo `_field_types` da classe **`typing.NamedTuple`** foi removido. Ele foi descontinuado desde o Python 3.8. Use o atributo `__annotations__`. (Contribuição de Serhiy Storchaka em bpo-40182 [https://bugs.python.org/issue?@action=redirect&bpo=40182].)
- O método **`symtable.SymbolTable.has_exec()`** foi removido. Ele foi descontinuado desde 2006, e retornando apenas `False` quando é chamado. (Contribuição de Batuhan Taskaya em bpo-40208 [https://bugs.python.org/issue?@action=redirect&bpo=40208])
- Os métodos **`asyncio.Task.current_task()`** e **`asyncio.Task.all_tasks()`** foram removidos. Eles foram descontinuados desde o Python 3.7 e você pode usar

`asyncio.current_task()` e `asyncio.all_tasks()` em seu lugar. (Contribuição de Rémi Lapeyre em bpo-40967 [https://bugs.python.org/issue?@action=redirect&bpo=40967])
- O método `unescape()` na classe `html.parser.HTMLParser` foi removido (foi descontinuado desde o Python 3.4). `html.unescape()` deve ser usada para converter referências de caracteres para os caracteres Unicode correspondentes.

Portando para Python 3.9

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações na API Python

- `__import__()` e `importlib.util.resolve_name()` agora levanta `ImportError` onde anteriormente levantava `ValueError`. Os chamadores capturando o tipo de exceção específico e suportando Python 3.9 e versões anteriores precisarão capturar ambas usando `except (ImportError, ValueError):`.
- Os scripts de ativação `venv` não são mais casos especiais quando `__VENV_PROMPT__` está definido como `""`.
- O método `select.epoll.unregister()` não ignora mais o erro `EBADF`. (Contribuição de Victor Stinner em bpo-39239 [https://bugs.python.org/issue?@action=redirect&bpo=39239].)
- O parâmetro *compresslevel* de `bz2.BZ2File` tornou-se uma parâmetro somente-nomeado, uma vez que o parâmetro *buffering* foi removido. (Contribuição de Victor Stinner em bpo-39357 [https://bugs.python.org/issue?@action=redirect&bpo=39357].)
- AST simplificada para assinatura. Índices simples serão representados por seus valores, fatias estendidas serão representadas como tuplas. `Index(value)` retornará um `value` em si, `ExtSlice(slices)` retornará `Tuple(slices, Load())`. (Contribuição de Serhiy Storchaka em bpo-34822 [https://bugs.python.org/issue?@action=redirect&bpo=34822].)
- O módulo `importlib` agora ignora a variável de ambiente `PYTHONCASEOK` quando as opções de linha de comando `-E` ou `-I` estão

sendo usadas.

- O parâmetro *encoding* foi adicionado às classes `ftplib.FTP` e `ftplib.FTP_TLS` como um parâmetro somente-nomeado, e a codificação padrão foi alterada de Latin-1 para UTF-8 para siga **RFC 2640** [https://datatracker.ietf.org/doc/html/rfc2640.html].
- `asyncio.loop.shutdown_default_executor()` foi adicionado a `AbstractEventLoop`, significando que laços de eventos alternativos que herdam dele devem ter este método definido. (Contribuição de Kyle Stanley em bpo-34037 [https://bugs.python.org/issue?@action=redirect&bpo=34037].)
- Os valores constantes de futuras sinalizações no módulo `__future__` são atualizados para evitar colisão com sinalizações do compilador. Anteriormente, `PyCF_ALLOW_TOP_LEVEL_AWAIT` estava em conflito com `CO_FUTURE_DIVISION`. (Contribuição de Batuhan Taskaya em bpo-39562 [https://bugs.python.org/issue?@action=redirect&bpo=39562])
- `array('u')` agora usa `wchar_t` como tipo C em vez de `Py_UNICODE`. Esta mudança não afeta seu comportamento porque `Py_UNICODE` é um apelido de `wchar_t` desde Python 3.3. (Contribuição de Inada Naoki em bpo-34538 [https://bugs.python.org/issue?@action=redirect&bpo=34538].)
- A API de `logging.getLogger()` agora retorna o logger raiz quando passado o nome `'root'`, enquanto anteriormente retornava um logger não-raiz chamado `'root'`. Isso pode afetar casos onde o código do usuário deseja explicitamente um logger não-root chamado `'root'`, ou instancia um logger usando `logging.getLogger(__name__)` em algum módulo de nível superior chamado `'root.py'`. (Contribuição de Vinay Sajip em bpo-37742 [https://bugs.python.org/issue?@action=redirect&bpo=37742].)
- O tratamento de divisão de `PurePath` agora retorna `NotImplemented` ao invés de levantar um `TypeError` quando passado algo diferente de uma instância de `str` ou `PurePath`. Isso permite a criação de classes compatíveis que não herdam desses tipos mencionados. (Contribuição de Roger Aiudi em bpo-34775 [https://bugs.python.org/issue?@action=redirect&bpo=34775]).
- A partir do Python 3.9.5, o módulo `ipaddress` não aceita mais zeros à esquerda em strings de endereço IPv4. Os zeros à esquerda são ambíguos e interpretados como notação octal por algumas bibliotecas. Por exemplo, a função legada `socket.inet_aton()` trata os zeros à esquerda como notação octal. A implementação glibc de `inet_pton()`

moderna não aceita nenhum zeros à esquerda. (Contribuição de Christian Heimes em bpo-36384 [https://bugs.python.org/issue?@action=redirect&bpo=36384]).

- **`codecs.lookup()`** agora normaliza o nome da codificação da mesma forma que **`encodings.normalize_encoding()`**, exceto que **`codecs.lookup()`** também converte o nome para minúsculas. Por exemplo, `"latex+latin1"` nome de codificação agora está normalizado para `"latex_latin1"`. (Contribuição de Jordon Xu em bpo-37751 [https://bugs.python.org/issue?@action=redirect&bpo=37751].)

Alterações na API C

- Instâncias de tipos alocados por heap (como aquelas criadas com **`PyType_FromSpec()`** e APIs semelhantes) mantêm uma referência para seu objeto de tipo desde o Python 3.8. Conforme indicado nas "Mudanças na API C" do Python 3.8, para a grande maioria dos casos, não deve haver nenhum efeito colateral, mas para os tipos que têm uma função personalizada **`tp_traverse`**, certifique-se de que todas as funções `tp_traverse` personalizadas de tipos alocados por heap visitam o tipo do objeto.

  Exemplo:

  ```
  int
  travessia_foo(PyObject *self, visitproc visit, void *ar
  {
  // Resto da função de travessia
  #if PY_VERSION_HEX >= 0x03090000
      // Isso não era necessário antes do Python 3.9 (prol
      Py_VISIT(Py_TYPE(self));
  #endif
  }
  ```

  Se sua função transversal delega para `tp_traverse` de sua classe base (ou outro tipo), certifique-se de que `Py_TYPE(self)` seja visitado apenas uma vez. Observe que apenas os tipos de heap devem visitar o tipo em `tp_traverse`.

Por exemplo, se sua função `tp_traverse` inclui:

```
base->tp_traverse(self, visit, arg)
```

adicione:

```
#if PY_VERSION_HEX >= 0x03090000
    // Isso não era necessário antes do Python 3.9 (bpo
    if (base->tp_flags & Py_TPFLAGS_HEAPTYPE) {
        // tp_traverse de um tipo de heap já visitou Py
    } else {
        Py_VISIT(Py_TYPE(self));
    }
#else
```

(Veja bpo-35810 [https://bugs.python.org/issue?@action=redirect&bpo=35810] e bpo-40217 [https://bugs.python.org/issue?@action=redirect&bpo=40217] para mais informações.)

- As funções `PyEval_CallObject`, `PyEval_CallFunction`, `PyEval_CallMethod` e `PyEval_CallObjectWithKeywords` foram descontinuadas. Use **`PyObject_Call()`** e suas variantes. (Veja mais detalhes em bpo-29548 [https://bugs.python.org/issue?@action=redirect&bpo=29548].)

Alterações de bytecode do CPython

- O opcode **`LOAD_ASSERTION_ERROR`** foi adicionado para tratar a instrução **`assert`**. Anteriormente, a instrução assert não funcionava corretamente se a exceção **`AssertionError`** estivesse sendo ocultada. (Contribuição de Zackery Spytz em bpo-34880 [https://bugs.python.org/issue?@action=redirect&bpo=34880].)

- O código de operação **`COMPARE_OP`** foi dividido em quatro instruções distintas:

  - `COMPARE_OP` para comparações ricas
  - `IS_OP` para testes “is” e “is not”

- `CONTAINS_OP` para testes “in” e “not in”
- `JUMP_IF_NOT_EXC_MATCH` para verificar exceções em instruções “try-except”.

(Contribuição de Mark Shannon em bpo-39156 [https://bugs.python.org/issue?@action=redirect&bpo=39156].)

Mudanças na construção

- Adicionada a opção `--with-platlibdir` ao script `configure`: nome do diretório de biblioteca específico da plataforma, armazenado no novo atributo **`sys.platlibdir`**. Veja o atributo **`sys.platlibdir`** para mais informações. (Contribuição de Jan Matějek, Matěj Cepl, Charalampos Stratakis e Victor Stinner em bpo-1294959 [https://bugs.python.org/issue?@action=redirect&bpo=1294959].)
- A macro especial de construção `COUNT_ALLOCS` foi removida. (Contribuição de Victor Stinner em bpo-39489 [https://bugs.python.org/issue?@action=redirect&bpo=39489].)
- Em plataformas não Windows, as funções **`setenv()`** e **`unsetenv()`** agora são necessárias para construir o Python. (Contribuição de Victor Stinner em bpo-39395 [https://bugs.python.org/issue?@action=redirect&bpo=39395].)
- Em plataformas não-Windows, a criação de instaladores `bdist_wininst` não possui mais suporte oficial. (Veja bpo-10945 [https://bugs.python.org/issue?@action=redirect&bpo=10945] para mais detalhes.)
- Ao compilar Python no macOS a partir do código-fonte, `_tkinter` agora se vincula a frameworks Tcl e Tk não pertencentes ao sistema se eles estiverem instalados em `/Library/Frameworks`, como era o caso em versões mais antigas do macOS. Se um macOS SDK for explicitamente configurado, usando **`--enable-universalsdk`** ou `-isysroot`, somente o SDK em si será procurado. Esse comportamento padrão pode ser sobreposto com `--with-tcltk-include` e `--with-tcltk-libs`. (Contribuição de Ned Deily em bpo-34956 [https://bugs.python.org/issue?@action=redirect&bpo=34956].)
- Python pode agora ser construído para Windows 10 ARM64. (Contribuição de Steve Dower em bpo-33125 [https://bugs.python.org/issue?@action=redirect&bpo=33125].)

- Alguns testes individuais agora são ignorados quando `--pgo` é usada. Os testes em questão aumentaram significativamente o tempo da tarefa de PGO e provavelmente não ajudaram a melhorar a otimização do executável final. Isso acelera a tarefa por um fator de cerca de 15x. A execução de todo o conjunto de testes de unidade é lenta. Esta mudança pode resultar em uma construção um pouco menos otimizada, uma vez que nem tantos ramos de código serão executados. Se você estiver disposto a esperar por uma construção muito mais lenta, o comportamento antigo pode ser restaurado usando `./configure [..] PROFILE_TASK="-m test --pgo-extended"`. Não oferecemos garantia de qual conjunto de tarefas de PGO produz uma construção mais rápida. Os usuários que se importam devem executar seus próprios benchmarks relevantes, pois os resultados podem depender do ambiente, da carga de trabalho e da cadeia de ferramentas do compilador. (Veja bpo-36044 [https://bugs.python.org/issue?@action=redirect&bpo=36044] e bpo-37707 [https://bugs.python.org/issue?@action=redirect&bpo=37707] para mais detalhes.)

Alterações na API C

Novas funcionalidades

- **PEP 573** [https://peps.python.org/pep-0573/]: Adicionada `PyType_FromModuleAndSpec()` para associar um módulo a uma classe; `PyType_GetModule()` e `PyType_GetModuleState()` para obter o módulo e seu estado; e `PyCMethod` e `METH_METHOD` para permitir que um método acesse a classe na qual ela foi definida. (Contribuição de Marcel Plch e Petr Viktorin em bpo-38787 [https://bugs.python.org/issue?@action=redirect&bpo=38787].)

- Adicionada a função `PyFrame_GetCode()`: obtém um código de quadro. Adiciona a função `PyFrame_GetBack()`: obtém o próximo quadro externo. (Contribuição de Victor Stinner em bpo-40421 [https://bugs.python.org/issue?@action=redirect&bpo=40421].)

- Adicionada `PyFrame_GetLineNumber()` à API C limitada. (Contribuição de Victor Stinner em bpo-40421 [https://bugs.python.org/issue?

@action=redirect&bpo=40421].)

- Adicionada as funções `PyThreadState_GetInterpreter()` e `PyInterpreterState_Get()` para obter o interpretador. Adiciona a função `PyThreadState_GetFrame()` para obter o quadro atual de um estado de thread do Python. Adiciona a função `PyThreadState_GetID()`: obtém o identificador único de um estado de thread do Python. (Contribuição de Victor Stinner em bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947].)

- Adicionada uma nova função pública `PyObject_CallNoArgs()` à API C, que chama um objeto chamável Python sem nenhum argumento. É a maneira mais eficiente de chamar um objeto chamável Python sem nenhum argumento. (Contribuição de Victor Stinner em bpo-37194 [https://bugs.python.org/issue?@action=redirect&bpo=37194].)

- Alterações na API C limitada (se a macro `Py_LIMITED_API` estiver definida):

  - Fornece `Py_EnterRecursiveCall()` e `Py_LeaveRecursiveCall()` como funções regulares para a API limitada. Anteriormente, eram definidas como macros, mas essas macros não compilavam com a API C limitada, que não pode acessar o campo `PyThreadState.recursion_depth` (a estrutura não é visível na API C limitada).
  - `PyObject_INIT()` e `PyObject_INIT_VAR()` se uma função regular não visível para ocultar os detalhes da implementação.

  (Contribuição de Victor Stinner em bpo-38644 [https://bugs.python.org/issue?@action=redirect&bpo=38644] e bpo-39542 [https://bugs.python.org/issue?@action=redirect&bpo=39542].)

- A função `PyModule_AddType()` é adicionado para ajudar a adicionar um tipo a um módulo. (Contribuição de Donghee Na em bpo-40024 [https://bugs.python.org/issue?@action=redirect&bpo=40024].)

- Adicionadas as funções `PyObject_GC_IsTracked()` e `PyObject_GC_IsFinalized()` para a API pública para permitir a

consulta se os objetos Python estão sendo rastreados ou já foram finalizados pelo coletor de lixo, respectivamente. (Contribuição de Pablo Galindo em bpo-40241 [https://bugs.python.org/issue?@action=redirect&bpo=40241].)

- Adicionada a **`_PyObject_FunctionStr()`** para obter uma representação amigável para o usuário de strings de um objeto função ou similar. (Patch por Jeroen Demeyer em bpo-37645 [https://bugs.python.org/issue?@action=redirect&bpo=37645].)

- Adicionada **`PyObject_CallOneArg()`** para chamar um objeto com um argumento posicional (Patch de Jeroen Demeyer em bpo-37483 [https://bugs.python.org/issue?@action=redirect&bpo=37483].)

Portando para Python 3.9

- `PyInterpreterState.eval_frame` (**PEP 523** [https://peps.python.org/pep-0523/]) agora exige um novo parâmetro *tstate* obrigatório (`PyThreadState*`). (Contribuição de Victor Stinner em bpo-38500 [https://bugs.python.org/issue?@action=redirect&bpo=38500].)

- Módulos de extensão: As funções **`m_traverse`**, **`m_clear`** e **`m_free`** de **`PyModuleDef`** não são mais chamados se o estado do módulo foi solicitado, mas ainda não está alocado. Este é o caso imediatamente após o módulo ser criado e antes de o módulo ser executado (função **`Py_mod_exec`**). Mais precisamente, estas funções são são chamados se **`m_size`** for maior que 0 e o estado do módulo (como retornado por **`PyModule_GetState()`**) for `NULL`.

  Módulos de extensão sem um estado de módulo (`m_size <= 0`) não são afetados.

- Se **`Py_AddPendingCall()`** for chamada em um subinterpretador, a função agora está programada para ser chamada a partir do subinterpretador, em vez de ser chamada a partir do interpretador principal. Cada subinterpretador agora tem sua própria lista de chamadas programadas. (Contribuição de Victor Stinner em bpo-39984 [https://bugs.python.org/issue?@action=redirect&bpo=39984].)

- O registro do Windows não é mais usado para inicializar **`sys.path`** quando a opção `-E` é usada (se **`PyConfig.use_environment`** estiver definido como `0`). Isso é significativo ao incorporar Python no Windows. (Contribuição de Zackery Spytz na bpo-8901 [https://bugs.python.org/issue?@action=redirect&bpo=8901].)

- A variável global **`PyStructSequence_UnnamedField`** agora é uma constante e se refere a uma string constante. (Contribuição de Serhiy Storchaka em bpo-38650 [https://bugs.python.org/issue?@action=redirect&bpo=38650].)

- A estrutura **`PyGC_Head`** agora é opaca. Ela só é definida na API C interna (`pycore_gc.h`). (Contribuição de Victor Stinner em bpo-40241 [https://bugs.python.org/issue?@action=redirect&bpo=40241].)

- `Py_UNICODE_COPY`, `Py_UNICODE_FILL`, , `PyUnicode_WSTR_LENGTH`, **`PyUnicode_FromUnicode()`**, **`PyUnicode_AsUnicode()`**, `_PyUnicode_AsUnicode` e **`PyUnicode_AsUnicodeAndSize()`** estão marcadas como descontinuadas no C. Elas foram descontinuadas pela **PEP 393** [https://peps.python.org/pep-0393/] desde Python 3.3. (Contribuição de Inada Naoki em bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346].)

- A função **`Py_FatalError()`** é substituída por uma macro que registra automaticamente o nome da função atual, a menos que a macro `Py_LIMITED_API` seja definida. (Contribuição de Victor Stinner em bpo-39882 [https://bugs.python.org/issue?@action=redirect&bpo=39882].)

- O protocolo vectorcall agora exige que o chamador passe apenas strings como nomes de argumentos. (Veja bpo-37540 [https://bugs.python.org/issue?@action=redirect&bpo=37540] para mais informações.)

- Os detalhes de implementação de uma série de macros e funções agora estão ocultos:

    - A macro **`PyObject_IS_GC()`** foi convertida para uma função.

- A macro **`PyObject_NEW()`** se torna um apelido para a macro **`PyObject_New`** e a macro **`PyObject_NEW_VAR()`** se torna um apelido para a macro **`PyObject_NewVar`**. Elas não acessam mais diretamente o membro **`PyTypeObject.tp_basicsize`**.
- A macro **`PyObject_GET_WEAKREFS_LISTPTR()`** foi convertida para uma função: a macro acessava diretamente o membro **`PyTypeObject.tp_weaklistoffset`**.
- A macro **`PyObject_CheckBuffer()`** foi convertida para uma função: a macro acessava diretamente o membro **`PyTypeObject.tp_as_buffer`**.
- **`PyIndex_Check()`** está agora declarada como uma função "opaca" para ocultar detalhes da implementação: removida a macro `PyIndex_Check()`. A macro acessava diretamente o membro **`PyTypeObject.tp_as_number`**.

(Veja bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170] para mais detalhes.)

Removidos

- Excluídas as macros `PyFPE_START_PROTECT()` e `PyFPE_END_PROTECT()` de `pyfpe.h` da API C limitada. (Contribuição por Victor Stinner em bpo-38835 [https://bugs.python.org/issue?@action=redirect&bpo=38835].)

- O slot `tp_print` de PyTypeObject foi removido. Ele era usado para imprimir objetos em arquivos no Python 2.7 e anteriores. Desde Python 3.0, ele era ignorado e não utilizado. (Contribuição de Jeroen Demeyer em bpo-36974 [https://bugs.python.org/issue?@action=redirect&bpo=36974].)

- Alterações na API C limitada (se a macro `Py_LIMITED_API` estiver definida):

  - Excluídas as seguintes funções da API C limitada:
    - `PyThreadState_DeleteCurrent()` (Contribuição de Joannah Nanjekye em bpo-37878 [https://bugs.python.org/issue?

@action=redirect&bpo=37878].)
    - `_Py_CheckRecursionLimit`
    - `_Py_NewReference()`
    - `_Py_ForgetReference()`
    - `_PyTraceMalloc_NewReference()`
    - `_Py_GetRefTotal()`
    - O mecanismo trashcan que nunca funcionou na API C limitada.
    - `PyTrash_UNWIND_LEVEL`
    - `Py_TRASHCAN_BEGIN_CONDITION`
    - `Py_TRASHCAN_BEGIN`
    - `Py_TRASHCAN_END`
    - `Py_TRASHCAN_SAFE_BEGIN`
    - `Py_TRASHCAN_SAFE_END`
  - Movidas as seguintes funções e definições para a API C interna:
    - `_PyDebug_PrintTotalRefs()`
    - `_Py_PrintReferences()`
    - `_Py_PrintReferenceAddresses()`
    - `_Py_tracemalloc_config`
    - `_Py_AddToAllObjects()` (específico para construção de `Py_TRACE_REFS`)

  (Contribuição de Victor Stinner em bpo-38644 [https://bugs.python.org/issue?@action=redirect&bpo=38644] e bpo-39542 [https://bugs.python.org/issue?@action=redirect&bpo=39542].)

- Removido o gancho `_PyRuntime.getframe` e removida a macro `_PyThreadState_GetFrame` que era um alias para `_PyRuntime.getframe`. Eles eram os únicos expostos pela API C interna. Removido também o tipo `PyThreadFrameGetter`. (Contribuição de Victor Stinner em bpo-39946 [https://bugs.python.org/issue?@action=redirect&bpo=39946].)

- Removidas as funções a seguir da API C. Chame **`PyGC_Collect()`** explicitamente para limpar todas as listas livres. (Contribuição de Inada Naoki e Victor Stinner em bpo-37340 [https://bugs.python.org/issue?@action=redirect&bpo=37340], bpo-38896 [https://bugs.python.org/issue?

@action=redirect&bpo=38896] e bpo-40428 [https://bugs.python.org/issue?@action=redirect&bpo=40428].)

  - `PyAsyncGen_ClearFreeLists()`
  - `PyContext_ClearFreeList()`
  - `PyDict_ClearFreeList()`
  - `PyFloat_ClearFreeList()`
  - `PyFrame_ClearFreeList()`
  - `PyList_ClearFreeList()`
  - `PyMethod_ClearFreeList()` e `PyCFunction_ClearFreeList()`: as listas livres de objetos de método vinculado foram removidas.
  - `PySet_ClearFreeList()`: a definição de lista livres foi removida no Python 3.4.
  - `PyTuple_ClearFreeList()`
  - `PyUnicode_ClearFreeList()`: a lista livre de Unicode foi removida no Python 3.3.

- Removida a função `_PyUnicode_ClearStaticStrings()`. (Contribuição de Victor Stinner em bpo-39465 [https://bugs.python.org/issue?@action=redirect&bpo=39465].)

- Removido `Py_UNICODE_MATCH`. Ele foi descontinuado por **PEP 393** [https://peps.python.org/pep-0393/] e quebrado desde o Python 3.3. A função **`PyUnicode_Tailmatch()`** pode ser usada em seu lugar. (Contribuição de Inada Naoki em bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346].)

- Limpados os arquivos de cabeçalho de interfaces definidas, mas sem implementação. Os símbolos públicos da API sendo removidos são: `_PyBytes_InsertThousandsGroupingLocale`, `_PyBytes_InsertThousandsGrouping`, `_Py_InitializeFromArgs`, `_Py_InitializeFromWideArgs`, `_PyFloat_Repr`, `_PyFloat_Digits`, `_PyFloat_DigitsInit`, `PyFrame_ExtendStack`, `_PyAIterWrapper_Type`, `PyNullImporter_Type`, `PyCmpWrapper_Type`, `PySortWrapper_Type`, `PyNoArgsFunction`. (Contribuição de Pablo Galindo Salgado em bpo-39372 [https://bugs.python.org/issue?@action=redirect&bpo=39372].)

Alterações notáveis no Python 3.9.1

typing

O comportamento de `typing.Literal` foi alterado para ficar em conformidade com a **PEP 586** [https://peps.python.org/pep-0586/] e para corresponder ao comportamento de verificadores de tipo estático especificados na PEP.

1. `Literal` agora elimina a duplicação de parâmetros.

2. Comparações de igualdade entre objetos `Literal` agora são independentes da ordem.

3. Comparações de `Literal` agora respeitam os tipos. Por exemplo, `Literal[0] == Literal[False]` avaliava anteriormente como `True`. Agora é `False`. Para oferecer suporte a essa mudança, o cache de tipo usado internamente agora oferece suporte a tipos de diferenciação.

4. Objetos `Literal` agora irão levantar uma exceção `TypeError` durante as comparações de igualdade se algum de seus parâmetros não for hasheável. Observe que declarar `Literal` com parâmetros mutáveis não gerará um erro:

```
>>> from typing import Literal
>>> Literal[{0}]
>>> Literal[{0}] == Literal[{False}]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: unhashable type: 'set'
```

(Contribuição de Yurii Karabas em bpo-42345 [https://bugs.python.org/issue?@action=redirect&bpo=42345].)

Suporte a macOS 11.0 (Big Sur) e Apple Silicon Mac

A partir do 3.9.1, o Python agora oferece suporte total à construção e execução no macOS 11.0 (Big Sur) e no Apple Silicon Macs (baseado na

arquitetura `ARM64`). Uma nova variante de compilação universal, `universal2`, está agora disponível para suportar nativamente ambos `ARM64` e `Intel 64` em um conjunto de executáveis. Os binários também podem ser construídos em versões atuais do macOS para serem implantados em uma variedade de versões mais antigas do macOS (testado para 10.9), enquanto algumas funções e opções do sistema operacional mais recentes estão disponíveis condicionalmente com base na versão do sistema operacional em uso no tempo de execução ("ligação fraca")

(Contribuição de Ronald Oussoren e Lawrence D'Anna em bpo-41100 [https://bugs.python.org/issue?@action=redirect&bpo=41100].)

Alterações notáveis no Python 3.9.2

collections.abc

`collections.abc.Callable` genérica agora nivela os parâmetros de tipo, semelhante ao que **`typing.Callable`** faz atualmente. Isso significa que `collections.abc.Callable[[int, str], str]` terá `__args__` de `(int, str, str)`; anteriormente era `([int, str], str)`. Para permitir esta mudança, **`types.GenericAlias`** agora pode ter uma subclasse, e uma subclasse será retornada ao subscrever o tipo **`Collections.abc.Callable`**. O código que acessa os argumentos via **`typing.get_args()`** ou `__args__` precisa levar em conta esta mudança. Um **`DeprecationWarning`** pode ser emitido para formas inválidas de parametrização **`Collections.abc.Callable`** que pode ter passado silenciosamente no Python 3.9.1. Esta **`DeprecationWarning`** se tornará uma **`TypeError`** no Python 3.10. (Contribuição de Ken Jin em bpo-42195 [https://bugs.python.org/issue?@action=redirect&bpo=42195].)

urllib.parse

Versões anteriores do Python permitiam o uso de `;` e `&` como separadores de parâmetros de consulta nas funções **`urllib.parse.parse_qs()`** e **`urllib.parse.parse_qsl()`**. Devido a questões de segurança, e em conformidade com as recomendações mais recentes do W3C, isso foi

alterado para permitir apenas uma única chave separadora, com `&` como padrão. Esta mudança também afeta as funções **`cgi.parse()`** e **`cgi.parse_multipart()`** já que elas usam as funções afetadas internamente. Para obter mais detalhes, consulte sua respectiva documentação. (Contribuição de Adam Goldschmidt, Senthil Kumaran e Ken Jin em bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967].)

Alterações notáveis no Python 3.9.3

Uma correção de segurança altera o comportamento **`ftplib.FTP`** para não confiar no endereço IPv4 enviado do servidor remoto ao configurar um canal de dados passivo. Em vez disso, reutilizamos o endereço IP do servidor ftp. Para códigos incomuns que requerem o comportamento antigo, defina um atributo `trust_server_pasv_ipv4_address` em sua instância de FTP para `True`. (Veja gh-87451 [https://github.com/python/cpython/issues/87451])

Alterações notáveis no Python 3.9.5

urllib.parse

A presença de caracteres de nova linha ou tab em partes de um URL permite algumas formas de ataques. Seguindo a especificação WHATWG que atualiza **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], nova linha ASCII `\n`, `\r` e os caracteres de tabulação `\t` são retirados da URL pelo analisador sintático em **`urllib.parse`** impedindo tais ataques. Os caracteres de remoção são controlados por uma nova variável de nível de módulo `urllib.parse._UNSAFE_URL_BYTES_TO_REMOVE`. (Veja gh-88048 [https://github.com/python/cpython/issues/88048])

Recursos de segurança notáveis no 3.9.14

Converter entre **`int`** e **`str`** em bases diferentes de 2 (binário), 4, 8 (octal), 16 (hexadecimal) ou 32 como base 10 (decimal) agora levanta uma exceção **`ValueError`** se o número de dígitos em forma de string estiver acima de um limite para evitar possíveis ataques de negação de serviço devido à

complexidade algorítmica. Esta é uma mitigação para **CVE 2020-10735** [https://www.cve.org/CVERecord?id=CVE-2020-10735]. Este limite pode ser configurado ou desabilitado por variável de ambiente, sinalizador de linha de comando ou APIs de `sys`. Veja a documentação de limitação de comprimento de conversão de string inteira. O limite padrão é de 4300 dígitos em forma de string.

Alterações notáveis no 3.9.17

tarfile

- Os métodos de extração em `tarfile` e `shutil.unpack_archive()`, têm um novo argumento *filter* que permite limitar recursos do tar que podem ser surpreendentes ou perigosos, como criar arquivos fora do diretório de destino. Veja Filtros de extração para detalhes. No Python 3.12, usar sem o argumento *filter* mostrará um `DeprecationWarning`. No Python 3.14, o padrão mudará para `'data'`. (Contribuição de Petr Viktorin em **PEP 706** [https://peps.python.org/pep-0706/].)

O que há de novo no Python 3.8

Editor:: Raymond Hettinger

Este artigo explica os novos recursos no Python 3.8, em comparação com 3.7. Python 3.8 foi lançado em 14 de outubro de 2019. Veja changelog para uma lista completa de mudanças.

Resumo – Destaques da versão

Novas funcionalidades

Expressões de atribuição

Existe uma nova sintaxe `:=` que atribui valores a variáveis como parte de uma expressão maior. É carinhosamente conhecido como “o operador morsa” (“the walrus operator”, em inglês) devido à sua semelhança com os olhos e presas de uma morsa
[https://en.wikipedia.org/wiki/Walrus#/media/File:Pacific_Walrus_-_Bull_(8247646168).jpg]

Neste exemplo, a expressão de atribuição ajuda a evitar a chamada de **`len()`** duas vezes:

```
if (n := len(a)) > 10:
    print(f"List is too long ({n} elements, expected <= 10)")
```

Um benefício semelhante surge durante a correspondência de expressões regulares, onde os objetos de correspondência são necessários duas vezes, uma vez para testar se ocorreu uma correspondência e outra para extrair um subgrupo:

```
discount = 0.0
if (mo := re.search(r'(\d+)% discount', advertisement)):
    discount = float(mo.group(1)) / 100.0
```

O operador também é útil com loops while que calculam um valor para testar a finalização do loop e, em seguida, precisam desse mesmo valor novamente no corpo do loop:

```
# Loop over fixed length blocks
while (block := f.read(256)) != '':
    process(block)
```

Outro caso de uso motivador surge em compreensões de lista em que um valor calculado em uma condição de filtragem também é necessário no corpo da expressão:

```
[clean_name.title() for name in names
 if (clean_name := normalize('NFC', name)) in allowed_names]
```

Tente limitar o uso do operador morsa para limpar os casos que reduzem a complexidade e melhoram a legibilidade.

Consulte **PEP 572** [https://peps.python.org/pep-0572/] para uma descrição completa.

(Contribuição de Emily Morehouse em bpo-35224 [https://bugs.python.org/issue?@action=redirect&bpo=35224].)

Parâmetros somente-posicionais

Existe uma nova sintaxe de parâmetro de função `/` para indicar que alguns parâmetros de função devem ser especificados posicionalmente e não podem ser usados como argumentos nomeados. Esta é a mesma notação mostrada por `help()` para funções C anotadas com a ferramenta Argument Clinic [https://devguide.python.org/development-tools/clinic/] de Larry Hastings.

No exemplo a seguir, os parâmetros *a* e *b* são apenas posicionais, enquanto *c* ou *d* podem ser posicionais ou nomeados e *e* ou *f* são obrigatoriamente nomeados:

```
def f(a, b, /, c, d, *, e, f):
    print(a, b, c, d, e, f)
```

A seguir, uma chamada válida:

```
f(10, 20, 30, d=40, e=50, f=60)
```

Porém, essas chamadas são inválidas:

```
f(10, b=20, c=30, d=40, e=50, f=60)   # b não pode ser um argume
f(10, 20, 30, 40, 50, f=60)           # e deve ser um argumento
```

Um caso de uso para essa notação é que ele permite que funções puras do Python emulem completamente os comportamentos das funções codificadas em C existentes. Por exemplo, a função embutida `divmod()` não aceita argumentos nomeados:

```
def divmod(a, b, /):
    "Emula a função embutida divmod()"
    return (a // b, a % b)
```

Outro caso de uso é impedir argumentos nomeados quando o nome do parâmetro não for útil. Por exemplo, a função embutida `len()` possui a assinatura `len(obj, /)`. Isso exclui chamadas estranhas, como:

```
len(obj='hello')  # O argumento nomeado "obj" prejudica a legibi
```

Um benefício adicional de marcar um parâmetro como somente posicional é que ele permite que o nome do parâmetro seja alterado no futuro sem risco de quebrar o código do cliente. Por exemplo, no módulo `statistics`, o nome do parâmetro *dist* pode ser alterado no futuro. Isso foi possível com a seguinte especificação de função:

```
def quantiles(dist, /, *, n=4, method='exclusive')
    ...
```

Como os parâmetros à esquerda de `/` não são expostos como possíveis palavras-chave, os nomes dos parâmetros permanecem disponíveis para uso em `**kwargs`:

```
>>> def f(a, b, /, **kwargs):
...     print(a, b, kwargs)
...
```

```
>>> f(10, 20, a=1, b=2, c=3)         # a e b são usados de duas
10 20 {'a': 1, 'b': 2, 'c': 3}
```

Isso simplifica bastante a implementação de funções e métodos que precisam aceitar argumentos nomeados arbitrários. Por exemplo, aqui está um trecho do código no módulo **`collections`**:

```
class Counter(dict):

    def __init__(self, iterable=None, /, **kwds):
        # Observe que "iterable" é um possível argumento nomeado
```

Consulte a **PEP 570** [https://peps.python.org/pep-0570/] para uma descrição completa.

(Contribuição de Pablo Galindo em bpo-36540 [https://bugs.python.org/issue?@action=redirect&bpo=36540].)

Cache do sistema de arquivos paralelo para arquivos de bytecode compilados

A nova configuração **`PYTHONPYCACHEPREFIX`** (também disponível como **`-X`** `pycache_prefix`) configura o cache implícito de bytecode para usar uma árvore paralela separada do sistema de arquivos, em vez dos subdiretórios padrão `__pycache__` dentro cada diretório fonte.

O local do cache é relatado em **`sys.pycache_prefix`** (**`None`** indica o local padrão nos subdiretórios `__pycache__`).

(Contribuição de Carl Meyer em bpo-33499 [https://bugs.python.org/issue?@action=redirect&bpo=33499].)

A compilação de depuração usa a mesma ABI da compilação de lançamento

A ABI do Python compilado para depuração agora é compatível com compilações para lançamento. No Unix, quando o Python é compilado no

modo de depuração, agora é possível carregar extensões C compiladas no modo de lançamento e extensões C compiladas usando a ABI estável. O inverso não é verdadeiro, uma vez que construções de depuração exponhem símbolos adicionais não disponíveis em construções de lançamento.

Definir a macro `Py_DEBUG` não implica mais na definição da macro `Py_TRACE_REFS`, que introduz a única incompatibilidade da ABI. A macro `Py_TRACE_REFS`, que adiciona a função **`sys.getobjects()`** e a variável de ambiente **`PYTHONDUMPREFS`**, pode ser configurada com a nova opção de compilação **`./configure --with-trace-refs`**. (Contribuição de Victor Stinner em bpo-36465 [https://bugs.python.org/issue?@action=redirect&bpo=36465].)

No Unix, as extensões C não estão mais vinculadas ao libpython, exceto no Android e no Cygwin. Agora é possível que um Python vinculado estaticamente carregue uma extensão C criada com uma biblioteca Python compartilhada. (Contribuição de Victor Stinner em bpo-21536 [https://bugs.python.org/issue?@action=redirect&bpo=21536].)

No Unix, quando o Python é compilado no modo de depuração, a importação agora também procura extensões C compiladas no modo de lançamento e extensões C compiladas com a ABI estável. (Contribuição de Victor Stinner em bpo-36722 [https://bugs.python.org/issue?@action=redirect&bpo=36722].)

Para incorporar o Python em uma aplicação, uma nova opção `--embed` deve ser passada para `python3-config --libs --embed` para obter o `-lpython3.8` (vincula a aplicação ao libpython). Para dar suporte a 3.8 e a versões mais antigas, tente `python3-config --libs --embed` primeiro e, como fallback, use `python3-config --libs` (sem `--embed`) se o comando anterior falhar.

Adicione um módulo pkg-config `python-3.8-embed` para incorporar o Python a uma aplicação: `pkg-config python-3.8-embed --libs` inclui `-lpython3.8`. Para suportar tanto o 3.8 quanto o anterior, tente `pkg-config python-X.Y-embed --libs` primeiro e use como fallback `pkg-config python-X.Y --libs` (sem `--embed`) se o comando anterior falhar (substitua `X.Y` pela versão do Python).

Por outro lado, `pkg-config python3.8 --libs` não contém mais `-lpython3.8`. As extensões C não devem estar vinculadas ao libpython (exceto no Android e no Cygwin, cujos casos são tratados pelo script); essa alteração é retroativamente incompatível de propósito. (Contribuição de Victor Stinner em bpo-36721 [https://bugs.python.org/issue?@action=redirect&bpo=36721].)

As *f-strings* aceitam = para expressões auto-documentadas e depuração

Adicionado um especificador = para f-strings. Uma f-string como `f'{expr=}'` será expandida para o texto da expressão, um sinal de igual e, em seguida, a representação da expressão avaliada. Por exemplo:

```
>>> import datetime as dt
>>> user = 'eric_idle'
>>> member_since = dt.date(1975, 7, 31)
>>> f'{user=} {member_since=}'
"user='eric_idle' member_since=datetime.date(1975, 7, 31)"
```

Os especificadores de formato de f-string habituais permitem maior controle sobre como o resultado da expressão é apresentado:

```
>>> delta = dt.date.today() - member_since
>>> f'{user=!s}  {delta.days=:,d}'
'user=eric_idle  delta.days=16,075'
```

O especificador = apresentará a expressão completa, de modo que os cálculos possam ser apresentados:

```
>>> print(f'{theta=}  {cos(radians(theta))=:.3f}')
theta=30  cos(radians(theta))=0.866
```

(Contribuição de Eric V. Smith e Larry Hastings em bpo-36817 [https://bugs.python.org/issue?@action=redirect&bpo=36817].)

PEP 578: Ganchos de auditoria em tempo de execução Python

O PEP adiciona um Gancho de Auditoria e um Gancho Aberto Verificado. Ambos estão disponíveis no Python e no código nativo, permitindo que aplicações e estruturas escritas em código Python puro aproveitem as notificações extras, além de permitir que incorporadores ou administradores de sistema implementem versões do Python onde a auditoria está sempre ativada.

Veja **PEP 578** [https://peps.python.org/pep-0578/] para mais detalhes.

PEP 587: Configuração de Inicialização do Python

O **PEP 587** [https://peps.python.org/pep-0587/] adiciona uma nova API C para configurar a Inicialização do Python, fornecendo um controle mais preciso de toda a configuração e melhores relatórios de erros.

Novas estruturas:

- `PyConfig`
- `PyPreConfig`
- `PyStatus`
- `PyWideStringList`

Novas Funções:

- `PyConfig_Clear()`
- `PyConfig_InitIsolatedConfig()`
- `PyConfig_InitPythonConfig()`
- `PyConfig_Read()`
- `PyConfig_SetArgv()`
- `PyConfig_SetBytesArgv()`
- `PyConfig_SetBytesString()`
- `PyConfig_SetString()`
- `PyPreConfig_InitIsolatedConfig()`
- `PyPreConfig_InitPythonConfig()`
- `PyStatus_Error()`
- `PyStatus_Exception()`
- `PyStatus_Exit()`
- `PyStatus_IsError()`

- `PyStatus_IsExit()`
- `PyStatus_NoMemory()`
- `PyStatus_Ok()`
- `PyWideStringList_Append()`
- `PyWideStringList_Insert()`
- `Py_BytesMain()`
- `Py_ExitStatusException()`
- `Py_InitializeFromConfig()`
- `Py_PreInitialize()`
- `Py_PreInitializeFromArgs()`
- `Py_PreInitializeFromBytesArgs()`
- `Py_RunMain()`

Esse PEP também adiciona os campos `_PyRuntimeState.preconfig` (tipo `PyPreConfig`) e `PyInterpreterState.config` (tipo `PyConfig`) campos a essas estruturas internas. `PyInterpreterState.config` se torna a nova configuração de referência, substituindo variáveis de configuração global e outras variáveis privadas.

Veja Configuração de Inicialização do Python para a documentação.

Consulte a **PEP 587** [https://peps.python.org/pep-0587/] para uma descrição completa.

(Contribuição de Victor Stinner em bpo-36763 [https://bugs.python.org/issue?@action=redirect&bpo=36763].)

PEP 590: Vectorcall: um protocolo de chamada rápida para CPython

O protocolo vectorcall é adicionado à API Python/C. Ele visa formalizar as otimizações existentes que já foram feitas para várias classes. Qualquer tipo estático que implemente um chamável pode usar esse protocolo.

Atualmente, isso é provisório. O objetivo é torná-lo totalmente público no Python 3.9.

Consulte a **PEP 590** [https://peps.python.org/pep-0590/] para uma descrição completa.

(Contribuição de Jeroen Demeyer, Mark Shannon e Petr Viktorin em bpo-36974 [https://bugs.python.org/issue?@action=redirect&bpo=36974].)

Protocolo de Pickle 5 com buffers de dados fora da banda

Quando `pickle` é usado para transferir grande quantidade de dados entre processos Python, a fim de tirar proveito do processamento com vários núcleos ou com várias máquinas, é importante otimizar a transferência reduzindo cópias de memória e, possivelmente, aplicando técnicas personalizadas, como como compactação dependente de dados.

O protocolo `pickle` 5 introduz suporte para buffers fora de banda, onde dados compatíveis com a **PEP 3118** [https://peps.python.org/pep-3118/] podem ser transmitidos separadamente do fluxo de pickle principal, a critério da camada de comunicação.

Consulte a **PEP 574** [https://peps.python.org/pep-0574/] para uma descrição completa.

(Contribuição de Antoine Pitrou em bpo-36785 [https://bugs.python.org/issue?@action=redirect&bpo=36785].)

Outras mudanças na linguagem

- A instrução `continue` era ilegal na cláusula `finally` devido a um problema com a implementação. No Python 3.8, essa restrição foi levantada. (Contribuição de Serhiy Storchaka em bpo-32489 [https://bugs.python.org/issue?@action=redirect&bpo=32489].)

- Os tipos `bool`, `int` e `fractions.Fraction` agora têm o método `as_integer_ratio()` como o encontrado em `float` e `decimal.Decimal`. Esta pequena extensão da API torna possível escrever `numerator, denominator = x.as_integer_ratio()` e fazer com que funcione em vários tipos numéricos. (Contribuição de Lisa

Roach em bpo-33073 [https://bugs.python.org/issue?@action=redirect&bpo=33073] e Raymond Hettinger em bpo-37819 [https://bugs.python.org/issue?@action=redirect&bpo=37819].)

- Os construtores de **`int`**, **`float`** e **`complex`** agora usarão o método especial **`__index__()`**, se disponível, e o método correspondente **`__int__()`**, **`__float__()`** ou **`__complex__()`** não está disponível. (Contribuição de Serhiy Storchaka em bpo-20092 [https://bugs.python.org/issue?@action=redirect&bpo=20092].)

- Adicionado suporte a escapes `\N{name}` em **`expressões regulares`**:

```
>>> notice = 'Copyright © 2019'
>>> copyright_year_pattern = re.compile(r'\N{copyright sign
>>> int(copyright_year_pattern.search(notice).group(1))
2019
```

  (Contribuição de Jonathan Eunice e Serhiy Storchaka em bpo-30688 [https://bugs.python.org/issue?@action=redirect&bpo=30688].)

- Dict e dictviews agora são iteráveis em ordem de inserção reversa usando **`reversed()`**. (Contribuição de Rémi Lapeyre em bpo-33462 [https://bugs.python.org/issue?@action=redirect&bpo=33462].)

- A sintaxe permitida para nomes de palavras-chave em chamadas de função foi ainda mais restrita. Em particular, `f((keyword)=arg)` não é mais permitido. Ele nunca teve a intenção de permitir mais do que um nome simples no lado esquerdo de um termo de atribuição de argumento nomeado. (Contribuição de Benjamin Peterson em bpo-34641 [https://bugs.python.org/issue?@action=redirect&bpo=34641].)

- O desempacotamento de iterável generalizado das instruções **`yield`** e **`return`** não requer mais parênteses. Isso traz a sintaxe *yield* e *return* uma melhor concordância com a sintaxe de atribuição normal:

```
>>> def parse(family):
...     lastname, *members = family.split()
...     return lastname.upper(), *members
...
```

```
>>> parse('simpsons homer marge bart lisa maggie')
('SIMPSONS', 'homer', 'marge', 'bart', 'lisa', 'maggie')
```

(Contribuição de David Cuthbert e Jordan Chapman em bpo-32117 [https://bugs.python.org/issue?@action=redirect&bpo=32117].)

- Quando uma vírgula está faltando em código como `[(10, 20) (30, 40)]`, o compilador exibe um **`SyntaxWarning`** com uma sugestão útil. Isso melhora apenas tendo um **`TypeError`** indicando que a primeira tupla não era chamável. (Contribuição de Serhiy Storchaka em bpo-15248 [https://bugs.python.org/issue?@action=redirect&bpo=15248].)

- As operações aritméticas entre as subclasses de **`datetime.date`** ou **`datetime.datetime`** e **`datetime.timedelta`** agora retornam uma instância da subclasse, em vez da classe base. Isso também afeta o tipo de retorno de operações cuja implementação (direta ou indiretamente) utiliza aritmética de **`datetime.timedelta`**, como **`astimezone()`**. (Contribuição de Paul Ganssle em bpo-32417 [https://bugs.python.org/issue?@action=redirect&bpo=32417].)

- Quando o interpretador Python é interrompido por Ctrl-C (SIGINT) e a exceção resultante **`KeyboardInterrupt`** não é capturada, o processo Python agora sai por meio de um sinal SIGINT ou com o código de saída correto, para que o processo de chamada possa detectar que foi encerrado devido a um Ctrl-C. Os shells no POSIX e Windows usam isso para finalizar corretamente os scripts em sessões interativas. (Contribuição do Google via Gregory P. Smith em bpo-1054041 [https://bugs.python.org/issue?@action=redirect&bpo=1054041].)

- Alguns estilos avançados de programação requerem a atualização do objeto **`types.CodeType`** para uma função existente. Como os objetos código são imutáveis, um novo objeto código precisa ser criado, modelado no objeto código existente. Com 19 parâmetros, isso foi um pouco tedioso. Agora, o novo método `replace()` torna possível criar um clone com alguns parâmetros alterados.

  Aqui está um exemplo que altera a função **`Statistics.mean()`** para impedir que o parâmetro *data* seja usado como argumento nomeado:

```
>>> from statistics import mean
>>> mean(data=[10, 20, 90])
40
>>> mean.__code__ = mean.__code__.replace(co_posonlyargcount
>>> mean(data=[10, 20, 90])
Traceback (most recent call last):
  ...
TypeError: mean() got some positional-only arguments passed
```

(Contribuição de Victor Stinner em bpo-37032 [https://bugs.python.org/issue?@action=redirect&bpo=37032].)

- Para números inteiros, a forma de três argumentos da função `pow()` agora permite que o expoente seja negativo no caso em que a base é relativamente primária ao módulo. Em seguida, calcula um inverso modular para a base quando o expoente é `-1` e uma potência adequada desse inverso para outros expoentes negativos. Por exemplo, para calcular o inverso multiplicativo modular [https://en.wikipedia.org/wiki/Modular_multiplicative_inverse] de 38 módulo 137, escreva:

```
>>> pow(38, -1, 137)
119
>>> 119 * 38 % 137
1
```

Inversos modulares surgem na solução de equações diofantinas lineares [https://en.wikipedia.org/wiki/Diophantine_equation]. Por exemplo, para encontrar soluções inteiras para $4258x + 147y = 369$, primeiro reescreva como $4258x \equiv 369 \pmod{147}$ e depois resolva:

```
>>> x = 369 * pow(4258, -1, 147) % 147
>>> y = (4258 * x - 369) // -147
>>> 4258 * x + 147 * y
369
```

(Contribuição de Mark Dickinson em bpo-36027 [https://bugs.python.org/issue?@action=redirect&bpo=36027].)

- As compreensões de dict foram sincronizadas com literais de dict, para que a chave seja computada primeiro e o valor em segundo:

```
>>> # Compreensão de dicionário
>>> cast = {input('role? '): input('actor? ') for i in range
role? King Arthur
actor? Chapman
role? Black Knight
actor? Cleese

>>> # Literal de dicionário
>>> cast = {input('role? '): input('actor? ')}
role? Sir Robin
actor? Eric Idle
```

  A ordem de execução garantida é útil nas expressões de atribuição, porque as variáveis atribuídas na expressão de chave estarão disponíveis na expressão de valor:

```
>>> names = ['Martin von Löwis', 'Łukasz Langa', 'Walter Dö
>>> {(n := normalize('NFC', name)).casefold() : n for name
{'martin von löwis': 'Martin von Löwis',
 'łukasz langa': 'Łukasz Langa',
 'walter dörwald': 'Walter Dörwald'}
```

  (Contribuição de Jörn Heissler em bpo-35224 [https://bugs.python.org/issue?@action=redirect&bpo=35224].)

- O método `object.__reduce__()` agora pode retornar uma tupla de dois a seis elementos. Anteriormente, o limite era cinco. O novo sexto elemento opcional é um chamável com uma assinatura `(obj, state)`. Isso permite o controle direto sobre o comportamento de atualização de estado de um objeto específico. Se não for *None*, este chamável terá prioridade sobre o método `__setstate__()` do objeto. (Contribuição de Pierre Glaser e Olivier Grisel em bpo-35900 [https://bugs.python.org/issue?@action=redirect&bpo=35900].)

Novos módulos

- O novo módulo **`importlib.metadata`** fornece suporte (provisório) para a leitura de metadados de pacotes de terceiros. Por exemplo, ele pode extrair o número da versão de um pacote instalado, a lista de pontos de entrada e muito mais:

  ```
  >>> # Note o exemplo a seguir requer que o pacote popular
  >>> # "requests" tenha sido instalado.
  >>>
  >>> from importlib.metadata import version, requires, files
  >>> version('requests')
  '2.22.0'
  >>> list(requires('requests'))
  ['chardet (<3.1.0,>=3.0.2)']
  >>> list(files('requests'))[:5]
  [PackagePath('requests-2.22.0.dist-info/INSTALLER'),
   PackagePath('requests-2.22.0.dist-info/LICENSE'),
   PackagePath('requests-2.22.0.dist-info/METADATA'),
   PackagePath('requests-2.22.0.dist-info/RECORD'),
   PackagePath('requests-2.22.0.dist-info/WHEEL')]
  ```

  (Contribuição de Barry Warsaw e Jason R. Coombs em bpo-34632 [https://bugs.python.org/issue?@action=redirect&bpo=34632].)

Módulos melhorados

ast

Os nós de AST agora têm os atributos `end_lineno` e `end_col_offset`, que fornecem a localização precisa do final do nó. (Isso se aplica apenas aos nós que possuem os atributos `lineno` e `col_offset`.)

Nova função **`ast.get_source_segment()`** retorna o código-fonte para um nó AST específico.

(Contribuição de Ivan Levkivskyi em bpo-33416 [https://bugs.python.org/issue?@action=redirect&bpo=33416].)

A função **`ast.parse()`** possui alguns novos sinalizadores:

- `type_comments=True` faz com que retorne o texto dos comentários de tipos de **PEP 484** [https://peps.python.org/pep-0484/] e **PEP 526** [https://peps.python.org/pep-0526/] associados a determinados nós AST;
- `mode='func_type'` pode ser usado para analisar "comentários do tipo de assinatura" do **PEP 484** [https://peps.python.org/pep-0484/] (retornados para nós AST de definição de função);
- `feature_version=(3, N)` permite especificar uma versão anterior do Python 3. Por exemplo, `feature_version=(3, 4)` tratará **`async`** e **`await`** como palavras não reservadas.

(Contribuição de Guido van Rossum em bpo-35766 [https://bugs.python.org/issue?@action=redirect&bpo=35766].)

asyncio

`asyncio.run()` passou da API provisória para a estável. Esta função pode ser usada para executar uma corrotina e retornar o resultado enquanto gerencia automaticamente o loop de eventos. Por exemplo:

```
import asyncio

async def main():
    await asyncio.sleep(0)
    return 42

asyncio.run(main())
```

Isso é *aproximadamente* equivalente a:

```
import asyncio

async def main():
    await asyncio.sleep(0)
    return 42

loop = asyncio.new_event_loop()
asyncio.set_event_loop(loop)
try:
    loop.run_until_complete(main())
finally:
```

```
    asyncio.set_event_loop(None)
    loop.close()
```

A implementação real é significativamente mais complexa. Assim, **`asyncio.run()`** deve ser a maneira preferida de executar programas assíncronos.

(Contribuição de Yury Selivanov em bpo-32314 [https://bugs.python.org/issue?@action=redirect&bpo=32314].)

A execução de `python -m asyncio` lança um REPL nativo assíncrono. Isso permite a experimentação rápida de código que possui um nível superior **`await`**. Não é mais necessário chamar diretamente `asyncio.run()`, o que geraria um novo loop de evento a cada chamada:

```
$ python -m asyncio
asyncio REPL 3.8.0
Use "await" directly instead of "asyncio.run()".
Type "help", "copyright", "credits" or "license" for more inform
>>> import asyncio
>>> await asyncio.sleep(10, result='hello')
hello
```

(Contribuição de Yury Selivanov em bpo-37028 [https://bugs.python.org/issue?@action=redirect&bpo=37028].)

A exceção **`asyncio.CancelledError`** agora herda de **`BaseException`** em vez de **`Exception`** e não mais herda de **`concurrent.futures.CancelledError`**. (Contribuição de Yury Selivanov em bpo-32528 [https://bugs.python.org/issue?@action=redirect&bpo=32528].)

No Windows, o loop de eventos padrão é agora **`ProactorEventLoop`**. (Contribuição de Victor Stinner em bpo-34687 [https://bugs.python.org/issue?@action=redirect&bpo=34687].)

`ProactorEventLoop` agora também tem suporte a UDP. (Contribuição de Adam Meily e Andrew Svetlov em bpo-29883 [https://bugs.python.org/issue?@action=redirect&bpo=29883].)

ProactorEventLoop pode agora ser interrompido por **KeyboardInterrupt** (“CTRL+C”). (Contribuição de Vladimir Matveev em bpo-23057 [https://bugs.python.org/issue?@action=redirect&bpo=23057].)

Adicionado **asyncio.Task.get_coro()** para obter a corrotina de dentro de um **asyncio.Task**. (Contribuição de Alex Grönholm em bpo-36999 [https://bugs.python.org/issue?@action=redirect&bpo=36999].)

As tarefas de asyncio agora podem ser nomeadas, passando o argumento nomeado `name` para **asyncio.create_task()** ou o método de loop de evento **create_task()** ou chamando o método **set_name()** no objeto de tarefa. O nome da tarefa é visível na saída de `repr()` de **asyncio.Task** e também pode ser recuperado usando o método **get_name()**. (Contribuição de Alex Grönholm em bpo-34270 [https://bugs.python.org/issue?@action=redirect&bpo=34270].)

Adicionado suporte a Happy Eyeballs [https://en.wikipedia.org/wiki/Happy_Eyeballs] para **asyncio.loop.create_connection()**. Para especificar o comportamento, dois novos parâmetros foram adicionados: *happy_eyeballs_delay* e *interleave*. O algoritmo Happy Eyeballs melhora a capacidade de resposta em aplicativos compatíveis com IPv4 e IPv6, tentando conectar-se simultaneamente usando os dois. (Contribuição de twisteroid ambassador em bpo-33530 [https://bugs.python.org/issue?@action=redirect&bpo=33530].)

builtins

A função embutida **compile()** foi aprimorada para aceitar o sinalizador `ast.PyCF_ALLOW_TOP_LEVEL_AWAIT`. Com esse novo sinalizador passado, **`compile()`** permitirá construções de nível superior `await`, `async for` e `async with` que geralmente são consideradas sintaxe inválida. O objeto de código assíncrono marcado com o sinalizador `CO_COROUTINE` pode ser retornado. (Contribuição de Matthias Bussonnier em bpo-34616 [https://bugs.python.org/issue?@action=redirect&bpo=34616])

collections

O método `_asdict()` para `collections.namedtuple()` agora retorna a `dict` em vez de um `collections.OrderedDict`. Isso funciona porque os dict regulares garantem pedidos desde o Python 3.7. Se os recursos extras de `OrderedDict` forem necessários, a solução sugerida é converter o resultado no tipo desejado: `OrderedDict(nt._asdict())`. (Contribuição de Raymond Hettinger em bpo-35864 [https://bugs.python.org/issue?@action=redirect&bpo=35864].)

cProfile

A classe `cProfile.Profile` agora pode ser usada como um gerenciador de contexto. Perfile um bloco de código executando:

```
import cProfile

with cProfile.Profile() as profiler:
      # código as ser analisada o desempenho
      ...
```

(Contribuição de Scott Sanderson em bpo-29235 [https://bugs.python.org/issue?@action=redirect&bpo=29235].)

csv

O `csv.DictReader` agora retorna instâncias de `dict` em vez de um `collections.OrderedDict`. Agora, a ferramenta é mais rápida e utiliza menos memória enquanto preserva a ordem dos campos. (Contribuição de Michael Selik em bpo-34003 [https://bugs.python.org/issue?@action=redirect&bpo=34003].)

curses

Adicionada uma nova variável contendo informações de versão estruturada para a biblioteca ncurses subjacente: `ncurses_version`. (Contribuição de Serhiy Storchaka em bpo-31680 [https://bugs.python.org/issue?@action=redirect&bpo=31680].)

ctypes

No Windows, `CDLL` e subclasses agora aceitam um parâmetro *winmode* para especificar sinalizadores para a chamada subjacente `LoadLibraryEx`. Os sinalizadores padrão são definidos para carregar apenas dependências da DLL em locais confiáveis, incluindo o caminho em que a DLL está armazenada (se um caminho completo ou parcial for usado para carregar a DLL inicial) e os caminhos adicionados por `add_dll_directory()` . (Contribuição de Steve Dower em bpo-36085 [https://bugs.python.org/issue?@action=redirect&bpo=36085].)

datetime

Adicionados novos construtores alternativos `datetime.date.fromisocalendar()` e `datetime.datetime.fromisocalendar()`, que constroem objetos de `date` e `datetime` respectivamente do ano ISO, número da semana e dia da semana; estes são o inverso do método `isocalendar` de cada classe. (Contribuição de Paul Ganssle em bpo-36004 [https://bugs.python.org/issue?@action=redirect&bpo=36004].)

functools

`@functools.lru_cache` can now be used as a straight decorator rather than as a function returning a decorator. So both of these are now supported:

```
@lru_cache
def f(x):
    ...

@lru_cache(maxsize=256)
def f(x):
    ...
```

(Contribuição de Raymond Hettinger em bpo-36772 [https://bugs.python.org/issue?@action=redirect&bpo=36772].)

Added a new `@functools.cached_property` decorator, for computed properties cached for the life of the instance.

```
import functools
import statistics

class Dataset:
   def __init__(self, sequence_of_numbers):
      self.data = sequence_of_numbers

   @functools.cached_property
   def variance(self):
      return statistics.variance(self.data)
```

(Contribuição de Carl Meyer em bpo-21145 [https://bugs.python.org/issue?@action=redirect&bpo=21145])

Adicionado um novo decorador `functools.singledispatchmethod()` que converte métodos em funções genéricas usando despacho único:

```
from functools import singledispatchmethod
from contextlib import suppress

class TaskManager:

    def __init__(self, tasks):
        self.tasks = list(tasks)

    @singledispatchmethod
    def discard(self, value):
        with suppress(ValueError):
            self.tasks.remove(value)

    @discard.register(list)
    def _(self, tasks):
        targets = set(tasks)
        self.tasks = [x for x in self.tasks if x not in targets]
```

(Contribuição de Ethan Smith em bpo-32380 [https://bugs.python.org/issue?@action=redirect&bpo=32380])

gc

`get_objects()` agora pode receber um parâmetro opcional *generation* indicando uma geração da qual obter objetos. (Contribuição de Pablo Galindo em bpo-36016 [https://bugs.python.org/issue?@action=redirect&bpo=36016].)

gettext

Adicionado `pgettext()` e suas variantes. (Contribuição de Franz Glasner, Éric Araujo e Cheryl Sabella em bpo-2504 [https://bugs.python.org/issue?@action=redirect&bpo=2504].)

gzip

Adicionado o parâmetro *mtime* em `gzip.compress()` para saída reproduzível. (Contribuição de Guo Ci Teo em bpo-34898 [https://bugs.python.org/issue?@action=redirect&bpo=34898].)

A exceção `BadGzipFile` agora é levantada em vez de `OSError` para certos tipos de arquivos gzip inválidos ou corrompidos. (Contribuição de Filip Gruszczyński, Michele Orrù e Zackery Spytz em bpo-6584 [https://bugs.python.org/issue?@action=redirect&bpo=6584].)

IDLE e idlelib

A saída em N linhas (50 por padrão) é compactada até um limite inferior. N pode ser alterado na seção PyShell da página General da caixa de diálogo Settings. Menos linhas, mas possivelmente extra longas, podem ser espremidas clicando com o botão direito do mouse na saída. A saída compactada pode ser expandida no local clicando duas vezes no botão ou na área de transferência ou em uma janela separada clicando com o botão direito do mouse. (Contribuição de Tal Einat em bpo-1529353 [https://bugs.python.org/issue?@action=redirect&bpo=1529353].)

Adiciona “Run Customized” ao menu Executar para executar um módulo com configurações personalizadas. Quaisquer argumentos de linha de comando inseridos são adicionados ao sys.argv. Eles também reaparecem

na caixa para a próxima execução personalizada. Pode-se também suprimir a reinicialização normal do módulo principal do console. (Contribuição de Cheryl Sabella, Terry Jan Reedy e outros em bpo-5680 [https://bugs.python.org/issue?@action=redirect&bpo=5680] e bpo-37627 [https://bugs.python.org/issue?@action=redirect&bpo=37627].)

Adicionados números de linha opcionais para janelas do editor IDLE. O Windows é aberto sem números de linha, a menos que definido de outra forma na guia General da caixa de diálogo de configuração. Os números de linha de uma janela existente são mostrados e ocultos no menu Opções. (Contribuição de Tal Einat e Saimadhav Heblikar em bpo-17535 [https://bugs.python.org/issue?@action=redirect&bpo=17535].)

A codificação nativa do sistema operacional agora é usada para converter entre strings Python e objetos Tcl. Isso permite que o IDLE trabalhe com emoji e outros caracteres não BMP. Esses caracteres podem ser exibidos ou copiados e colados na ou da área de transferência. A conversão de strings de Tcl para Python e de volta agora nunca falha. (Muitas pessoas trabalharam nisso por oito anos, mas o problema foi finalmente resolvido por Serhiy Storchaka em bpo-13153 [https://bugs.python.org/issue?@action=redirect&bpo=13153].)

Novo em 3.8.1:

Adiciona a opção para ativar e desativar o piscar do cursor. (Contribuição de Zackery Spytz na bpo-4603 [https://bugs.python.org/issue?@action=redirect&bpo=4603].)

A tecla Escape agora fecha as janelas de conclusão IDLE. (Contribuição de Johnny Najera em bpo-38944 [https://bugs.python.org/issue?@action=redirect&bpo=38944].)

As alterações acima foram suportadas para versões de manutenção 3.7.

Adiciona argumentos nomeados à lista de completamento de nome de módulo. (Contribuição de Terry J. Reedy em bpo-37765 [https://bugs.python.org/issue?@action=redirect&bpo=37765].)

inspect

The `inspect.getdoc()` function can now find docstrings for `__slots__` if that attribute is a `dict` where the values are docstrings. This provides documentation options similar to what we already have for `@property`, `@classmethod`, and `@staticmethod`:

```
class AudioClip:
    __slots__ = {'bit_rate': 'expressed in kilohertz to one deci
                 'duration': 'in seconds, rounded up to an integ
    def __init__(self, bit_rate, duration):
        self.bit_rate = round(bit_rate / 1000.0, 1)
        self.duration = ceil(duration)
```

(Contribuição de Raymond Hettinger em bpo-36326 [https://bugs.python.org/issue?@action=redirect&bpo=36326].)

io

No modo de desenvolvimento (`-X` `env`) e na compilação de depuração, o finalizador `io.IOBase` agora registra a exceção se o método `close()` falhar. A exceção é ignorada silenciosamente por padrão na compilação de lançamento. (Contribuição de Victor Stinner em bpo-18748 [https://bugs.python.org/issue?@action=redirect&bpo=18748].)

itertools

A função `itertools.accumulate()` adicionou uma opção de argumento nomeado *initial* para especificar um valor inicial:

```
>>> from itertools import accumulate
>>> list(accumulate([10, 5, 30, 15], initial=1000))
[1000, 1010, 1015, 1045, 1060]
```

(Contribuído por Lisa Roach em bpo-34659 [https://bugs.python.org/issue?@action=redirect&bpo=34659].)

json.tool

Adiciona a opção `--json-lines` para analisar cada linha de entrada como um objeto JSON separado. (Contribuição de Weipeng Hong em bpo-31553 [https://bugs.python.org/issue?@action=redirect&bpo=31553].)

logging

Adicionado um argumento nomeado *force* a **`logging.basicConfig()`**. Quando definido como true, qualquer manipulador existente anexado ao logger raiz é removido e fechado antes de executar a configuração especificada pelos outros argumentos.

Isso resolve um problema de longa data. Depois que um logger ou *basicConfig()* foi chamado, as chamadas subsequentes para *basicConfig()* foram ignoradas silenciosamente. Isso dificultava a atualização, a experimentação ou o ensino das várias opções de configuração de log usando o prompt interativo ou um bloco de notas de Jupyter.

(Sugestão de Raymond Hettinger, implementação de Dong-hee Na e revisão de Vinay Sajip em bpo-33897 [https://bugs.python.org/issue?@action=redirect&bpo=33897].)

math

Adicionada nova função **`math.dist()`** para calcular a distância euclidiana entre dois pontos. (Contribuição de Raymond Hettinger em bpo-33089 [https://bugs.python.org/issue?@action=redirect&bpo=33089].)

Expandida a função **`math.hypot()`** para manipular várias dimensões. Anteriormente, suportava apenas o caso 2-D. (Contribuição de Raymond Hettinger em bpo-33089 [https://bugs.python.org/issue?@action=redirect&bpo=33089].)

Adicionada nova função, **`math.prod()`**, como função análoga a **`sum()`** que retorna o produto de um valor ‘start’ (padrão: 1) vezes uma iterável de números:

```
>>> prior = 0.8
>>> likelihoods = [0.625, 0.84, 0.30]
```

```
>>> math.prod(likelihoods, start=prior)
0.126
```

(Contribuição de Pablo Galindo em bpo-35606 [https://bugs.python.org/issue?@action=redirect&bpo=35606].)

Adicionadas duas novas funções combinatórias **`math.perm()`** e **`math.comb()`**:

```
>>> math.perm(10, 3)    # Permutações de 10 coisas, 3 por vez
720
>>> math.comb(10, 3)    # Combinações de 10 coisas, 3 por vez
120
```

(Contribuição de Yash Aggarwal, Keller Fuchs, Serhiy Storchaka e Raymond Hettinger em bpo-37128 [https://bugs.python.org/issue?@action=redirect&bpo=37128], bpo-37178 [https://bugs.python.org/issue?@action=redirect&bpo=37178] e bpo-35431 [https://bugs.python.org/issue?@action=redirect&bpo=35431].)

Adicionada uma nova função **`math.isqrt()`** para calcular raízes quadradas inteiras precisas sem conversão para ponto flutuante. A nova função suporta números inteiros arbitrariamente grandes. É mais rápido que `floor(sqrt(n))`, mas mais lento que **`math.sqrt()`**:

```
>>> r = 650320427
>>> s = r ** 2
>>> isqrt(s - 1)         # correto
650320426
>>> floor(sqrt(s - 1))   # incorreto
650320427
```

(Contribuição de Mark Dickinson em bpo-36887 [https://bugs.python.org/issue?@action=redirect&bpo=36887].)

A função **`math.factorial()`** não aceita mais argumentos que não são parecidos com int. (Contribuição de Pablo Galindo em bpo-33083 [https://bugs.python.org/issue?@action=redirect&bpo=33083].)

mmap

A classe `mmap.mmap` agora possui o método `madvise()` para acessar a chamada do sistema `madvise()`. (Contribuição de Zackery Spytz em bpo-32941 [https://bugs.python.org/issue?@action=redirect&bpo=32941].)

multiprocessing

Adicionado novo módulo `multiprocessing.shared_memory`. (Contribuição de Davin Potts em bpo-35813 [https://bugs.python.org/issue?@action=redirect&bpo=35813].)

No macOS, o método inicial *spawn* agora é usado por padrão. (Contribuição de Victor Stinner em bpo-33725 [https://bugs.python.org/issue?@action=redirect&bpo=33725].)

os

Adicionada nova função `add_dll_directory()` no Windows para fornecer caminhos de pesquisa adicionais para dependências nativas ao importar módulos de extensão ou carregar DLLs usando `ctypes`. (Contribuição de Steve Dower em bpo-36085 [https://bugs.python.org/issue?@action=redirect&bpo=36085].)

Uma nova função `os.memfd_create()` foi adicionada para envolver a chamada de sistema `memfd_create()`. (Contribuição de Zackery Spytz e Christian Heimes em bpo-26836 [https://bugs.python.org/issue?@action=redirect&bpo=26836].)

No Windows, grande parte da lógica manual para lidar com pontos de nova análise (incluindo links simbólicos e junções de diretórios) foi delegada ao sistema operacional. Especificamente, `os.stat()` agora percorrerá qualquer objeto suportado pelo sistema operacional, enquanto `os.lstat()` abrirá apenas pontos de nova análise que se identifiquem como “substitutos de nome”, enquanto outros serão abertos como para **`os.stat()`**. Em todos os casos, `os.stat_result.st_mode` terá `S_IFLNK` definido apenas para links simbólicos e não para outros tipos de pontos de nova análise. Para identificar outros tipos de pontos de nova análise, verifique o novo atributo `os.stat_result.st_reparse_tag`.

No Windows, `os.readlink()` agora pode ler junções de diretório. Observe que `islink()` retornará `False` para junções de diretório e, portanto, o código que verifica primeiro `islink` continuará tratando junções como diretórios, enquanto o código que lida com erros de **`os.readlink()`** agora pode tratar junções como links.

(Contribuição de Steve Dower em bpo-37834 [https://bugs.python.org/issue?@action=redirect&bpo=37834].)

os.path

Funções de `os.path` que retornam um resultado booleano como `exists()`, `lexists()`, `isdir()`, `isfile()`, `islink()` e `ismount()` agora retornam `False` em vez de levantar `ValueError` ou suas subclasses `UnicodeEncodeError` e `UnicodeDecodeError` para caminhos que contêm caracteres ou bytes não representáveis no nível do SO. (Contribuição de Serhiy Storchaka em bpo-33721 [https://bugs.python.org/issue?@action=redirect&bpo=33721].)

`expanduser()` no Windows agora prefere a variável de ambiente **`USERPROFILE`** e não usa **`HOME`**, que normalmente não é definido para contas de usuário comuns. (Contribuição de Anthony Sottile em bpo-36264 [https://bugs.python.org/issue?@action=redirect&bpo=36264].)

`isdir()` no Windows não retorna mais `True` para um link para um diretório inexistente.

`realpath()` no Windows agora resolve pontos de nova análise, incluindo links simbólicos e junções de diretório.

(Contribuição de Steve Dower em bpo-37834 [https://bugs.python.org/issue?@action=redirect&bpo=37834].)

pathlib

Os métodos `pathlib.Path` que retornam um resultado booleano como `exists()`, `is_dir()`, `is_file()`, `is_mount()`, `is_symlink()`,

`is_block_device()`, `is_char_device()`, `is_fifo()`, `is_socket()` agora retornam `False` em vez de levantar `ValueError` ou sua subclasse `UnicodeEncodeError` para caminhos que contêm caracteres não representáveis no nível do sistema operacional. (Contribuição de Serhiy Storchaka em bpo-33721 [https://bugs.python.org/issue?@action=redirect&bpo=33721].)

Adicionado **`pathlib.Path.link_to()`** que cria um link físico apontando para um caminho. (Contribuição de Joannah Nanjekye em bpo-26978 [https://bugs.python.org/issue?@action=redirect&bpo=26978]) Observe que `link_to` foi descontinuado na versão 3.10 e removido na versão 3.12 em favor de um método `hardlink_to` adicionado na versão 3.10 que corresponde à semântica do método `symlink_to` existente.

pickle

Extensões de `pickle`, que atuem como subclasse da classe otimizada para C `Pickler`, agora podem substituir a lógica de "pickling" de funções e classes definindo o método especial `reducer_override()`. (Contribuição de Pierre Glaser e Olivier Grisel em bpo-35900 [https://bugs.python.org/issue?@action=redirect&bpo=35900].)

plistlib

Adicionado novo `plistlib.UID` e habilitado suporte para leitura e escrita de plists binárias codificadas em NSKeyedArchiver. (Contribuição de Jon Janzen em bpo-26707 [https://bugs.python.org/issue?@action=redirect&bpo=26707].)

pprint

O módulo `pprint` adicionou um parâmetro *sort_dicts* a várias funções. Por padrão, essas funções continuam a classificar dicionários antes da renderização ou impressão. No entanto, se *sort_dicts* estiver definido como false, os dicionários manterão a ordem em que as chaves foram inseridas. Isso pode ser útil para comparação com entradas JSON durante a depuração.

Além disso, existe uma nova função de conveniência, **pprint.pp()**, que é como **pprint.pprint()**, mas com *sort_dicts* padronizando como `False`:

```
>>> from pprint import pprint, pp
>>> d = dict(source='input.txt', operation='filter', destinatior
>>> pp(d, width=40)                  # Ordem original
{'source': 'input.txt',
 'operation': 'filter',
 'destination': 'output.txt'}
>>> pprint(d, width=40)              # Chaves ordenadas alfabeti
{'destination': 'output.txt',
 'operation': 'filter',
 'source': 'input.txt'}
```

(Contribuição de Rémi Lapeyre em bpo-30670 [https://bugs.python.org/issue?@action=redirect&bpo=30670].)

py_compile

py_compile.compile() agora possui suporte a um modo silencioso. (Contribuição de Joannah Nanjekye em bpo-22640 [https://bugs.python.org/issue?@action=redirect&bpo=22640].)

shlex

A nova função **shlex.join()** atua como o inverso da **shlex.split()**. (Contribuição de Bo Bayles em bpo-32102 [https://bugs.python.org/issue?@action=redirect&bpo=32102].)

shutil

shutil.copytree() agora aceita um novo argumento nomeado `dirs_exist_ok`. (Contribuição de Josh Bronson em bpo-20849 [https://bugs.python.org/issue?@action=redirect&bpo=20849].)

shutil.make_archive() agora tem como padrão o formato pax moderno (POSIX.1-2001) de novos arquivos para melhorar a portabilidade e a conformidade com os padrões, herdados da alteração correspondente no

módulo `tarfile`. (Contribuição de C.A.M. Gerlach em bpo-30661 [https://bugs.python.org/issue?@action=redirect&bpo=30661].)

`shutil.rmtree()` no Windows agora remove junções de diretórios sem remover recursivamente o conteúdo primeiro. (Contribuição de Steve Dower em bpo-37834 [https://bugs.python.org/issue?@action=redirect&bpo=37834].)

socket

Adicionadas as funções de conveniência `create_server()` e `has_dualstack_ipv6()` para automatizar as tarefas necessárias geralmente envolvidas na criação de um soquete de servidor, incluindo a aceitação de conexões IPv4 e IPv6 no mesmo soquete. (Contribuição de Giampaolo Rodolà em bpo-17561 [https://bugs.python.org/issue?@action=redirect&bpo=17561].)

As funções `socket.if_nameindex()`, `socket.if_nametoindex()` e `socket.if_indextoname()` foram agora implementadas no Windows. (Contribuição de Zackery Spytz em bpo-37007 [https://bugs.python.org/issue?@action=redirect&bpo=37007].)

ssl

Adicionado `post_handshake_auth` para ativar e `verify_client_post_handshake()` para iniciar a autenticação pós-handshake do TLS 1.3. (Contribuição de Christian Heimes em bpo-34670 [https://bugs.python.org/issue?@action=redirect&bpo=34670].)

statistics

Adicionada `statistics.fmean()` como uma variante de ponto flutuante mais rápida de `statistics.mean()`. (Contribuição de Raymond Hettinger e Steven D'Aprano em bpo-35904 [https://bugs.python.org/issue?@action=redirect&bpo=35904].)

Adicionada `statistics.geometric_mean()` (Contribuição de Raymond Hettinger em bpo-27181 [https://bugs.python.org/issue?@action=redirect&bpo=27181].

Adicionada **`Statistics.multimode()`** que retorna uma lista dos valores mais comuns. (Contribuição de Raymond Hettinger em bpo-35892 [https://bugs.python.org/issue?@action=redirect&bpo=35892].)

Adicionada **`statistics.quantiles()`** que divide dados ou uma distribuição em intervalos equivalentes (por exemplo, quartis, decis ou percentis). (Contribuição de Raymond Hettinger em bpo-36546 [https://bugs.python.org/issue?@action=redirect&bpo=36546].)

Adicionada **`statistics.NormalDist`**, uma ferramenta para criar e manipular distribuições normais de uma variável aleatória. (Contribuição de Raymond Hettinger em bpo-36018 [https://bugs.python.org/issue?@action=redirect&bpo=36018].)

```
>>> temperature_feb = NormalDist.from_samples([4, 12, -3, 2, 7,
>>> temperature_feb.mean
6.0
>>> temperature_feb.stdev
6.356099432828281

>>> temperature_feb.cdf(3)            # Chance de ser manos de 3
0.3184678262814532
>>> # Chance relativa de ser 7 graus contra 10 graus
>>> temperature_feb.pdf(7) / temperature_feb.pdf(10)
1.2039930378537762

>>> el_niño = NormalDist(4, 2.5)
>>> temperature_feb += el_niño        # Adiciona em um efeito do
>>> temperature_feb
NormalDist(mu=10.0, sigma=6.830080526611674)

>>> temperature_feb * (9/5) + 32      # Converte para Fahrenheit
NormalDist(mu=50.0, sigma=12.294144947901014)
>>> temperature_feb.samples(3)        # Gera amostras aleatórias
[7.672102882379219, 12.000027119750287, 4.647488369766392]
```

sys

Adiciona uma nova função `sys.unraisablehook()`, que pode ser substituída para controlar como as “exceções não passíveis de levantamento” são tratadas. É chamada quando uma exceção ocorre, mas não há como o Python manipulá-la. Por exemplo, quando um destruidor gera uma exceção ou durante a coleta de lixo (`gc.collect()`). (Contribuição de Victor Stinner em bpo-36829 [https://bugs.python.org/issue?@action=redirect&bpo=36829].)

tarfile

O módulo `tarfile` agora tem como padrão o formato pax moderno (POSIX.1-2001) para novos arquivos, em vez do anterior específico do GNU. Isso melhora a portabilidade de plataforma cruzada com uma codificação consistente (UTF-8) em um formato padronizado e extensível e oferece vários outros benefícios. (Contribuição de C.A.M. Gerlach em bpo-36268 [https://bugs.python.org/issue?@action=redirect&bpo=36268].)

threading

Adiciona uma nova função `threading.excepthook()` que manipula a exceção não capturada `threading.Thread.run()`. Pode ser substituída para controlar como exceções não capturadas `threading.Thread.run()` são manipuladas. (Contribuição de Victor Stinner em bpo-1230540 [https://bugs.python.org/issue?@action=redirect&bpo=1230540].)

Adiciona uma nova função `threading.get_native_id()` e um atributo `native_id` à classe `threading.Thread`. Eles retornam o Thread ID integral nativo do thread atual atribuído pelo kernel. Este recurso está disponível apenas em determinadas plataformas, consulte `get_native_id` para mais informações. (Contribuição de Jake Tesler em bpo-36084 [https://bugs.python.org/issue?@action=redirect&bpo=36084].)

tokenize

O módulo `tokenize` agora emite implicitamente um token `NEWLINE` quando fornecido com entrada que não possui uma nova linha à direita. Esse comportamento agora corresponde ao que o tokenizador do C faz

internamente. (Contribuição de Ammar Askar em bpo-33899 [https://bugs.python.org/issue?@action=redirect&bpo=33899].)

tkinter

Adicionados os métodos **`selection_from()`**, **`selection_present()`**, **`selection_range()`** e **`selection_to()`** na classe **`tkinter.Spinbox`**. (Contribuição de Juliette Monsel em bpo-34829 [https://bugs.python.org/issue?@action=redirect&bpo=34829].)

Adicionado o método **`moveto()`** na classe **`tkinter.Canvas`**. (Contribuição de Juliette Monsel em bpo-23831 [https://bugs.python.org/issue?@action=redirect&bpo=23831].)

A classe **`tkinter.PhotoImage`** agora possui os métodos **`transparency_get()`** e **`transparency_set()`**. (Contribuição de Zackery Spytz em bpo-25451 [https://bugs.python.org/issue?@action=redirect&bpo=25451].)

time

Adiciona o novo relógio **`CLOCK_UPTIME_RAW`** ao macOS 10.12. (Contribuição de Joannah Nanjekye em bpo-35702 [https://bugs.python.org/issue?@action=redirect&bpo=35702].)

typing

O módulo **`typing`** incorpora vários novos recursos:

- Um tipo de dicionário com tipos por chave. Veja **PEP 589** [https://peps.python.org/pep-0589/] e **`typing.TypedDict`**. TypedDict usa apenas chaves de strings. Por padrão, é necessário que todas as chaves estejam presentes. Especifique “total=False” para permitir que as chaves sejam opcionais:

  ```
  class Location(TypedDict, total=False):
      lat_long: tuple
      grid_square: str
      xy_coordinate: tuple
  ```

- Tipos literais. Veja **PEP 586** [https://peps.python.org/pep-0586/] e `typing.Literal`. Tipos literais indicam que um parâmetro ou valor de retorno está restrito a um ou mais valores literais específicos:

  ```
  def get_status(port: int) -> Literal['connected', 'disconnec
      ...
  ```

- "Final" variables, functions, methods and classes. See **PEP 591** [https://peps.python.org/pep-0591/], `typing.Final` and `@typing.final`. The final qualifier instructs a static type checker to restrict subclassing, overriding, or reassignment:

  ```
  pi: Final[float] = 3.1415926536
  ```

- Protocol definitions. See **PEP 544** [https://peps.python.org/pep-0544/], `typing.Protocol` and `@typing.runtime_checkable`. Simple ABCs like `typing.SupportsInt` are now `Protocol` subclasses.

- Nova classe de protocolo `typing.SupportsIndex`.

- Novas funções `typing.get_origin()` e `typing.get_args()`.

unicodedata

O módulo `unicodedata` agora foi atualizado para usar o Unicode 12.1.0 [https://blog.unicode.org/2019/05/unicode-12-1-en.html].

A nova função `is_normalized()` pode ser usada para verificar se uma string está em uma forma normal específica, geralmente muito mais rápida do que normalizar a string. (Contribuição de Max Belanger, David Euresti e Greg Price em bpo-32285 [https://bugs.python.org/issue?@action=redirect&bpo=32285] e bpo-37966 [https://bugs.python.org/issue?@action=redirect&bpo=37966]).

unittest

Adicionada `AsyncMock` para dar suporte a uma versão assíncrona de `Mock`. Também foram adicionadas novas funções apropriadas para teste.

(Contribuição de Lisa Roach em bpo-26467 [https://bugs.python.org/issue?@action=redirect&bpo=26467]).

Adicionados `addModuleCleanup()` e `addClassCleanup()` a unittest para dar suporte a limpezas para `setUpModule()` e `setUpClass()`. (Contribuição de Lisa Roach em bpo-24412 [https://bugs.python.org/issue?@action=redirect&bpo=24412].)

Várias funções de declaração de mock agora também exibem uma lista de chamadas reais em caso de falha. (Contribuição de Petter Strandmark em bpo-35047 [https://bugs.python.org/issue?@action=redirect&bpo=35047].)

O módulo `unittest` ganhou suporte a corrotinas a serem usadas como casos de teste com `unittest.IsolatedAsyncioTestCase`. (Contribuição de Andrew Svetlov em bpo-32972 [https://bugs.python.org/issue?@action=redirect&bpo=32972].)

Exemplo:

```
import unittest


class TestRequest(unittest.IsolatedAsyncioTestCase):

    async def asyncSetUp(self):
        self.connection = await AsyncConnection()

    async def test_get(self):
        response = await self.connection.get("https://example.co
        self.assertEqual(response.status_code, 200)

    async def asyncTearDown(self):
        await self.connection.close()


if __name__ == "__main__":
    unittest.main()
```

venv

venv agora inclui um script `Activate.ps1` em todas as plataformas para ativar ambientes virtuais no PowerShell Core 6.1. (Contribuição de Brett Cannon em bpo-32718 [https://bugs.python.org/issue?@action=redirect&bpo=32718].)

weakref

Os objetos proxy retornados por **weakref.proxy()** agora possuem suporte aos operadores de multiplicação de matrizes `@` e `@=` além dos outros operadores numéricos. (Contribuição de Mark Dickinson em bpo-36669 [https://bugs.python.org/issue?@action=redirect&bpo=36669].)

xml

Como atenuação a DTD e recuperação de entidade externa, os módulos **xml.dom.minidom** e **xml.sax** não processam mais entidades externas por padrão. (Contribuição de Christian Heimes em bpo-17239 [https://bugs.python.org/issue?@action=redirect&bpo=17239].)

Os métodos `.find*()` no módulo **xml.etree.ElementTree** possuem suporte a pesquisas curinga como `{*}tag`, que ignora o espaço para nome e `{namespace}*`, que retorna todas as tags no espaço para nome especificado. (Contribuição de Stefan Behnel em bpo-28238 [https://bugs.python.org/issue?@action=redirect&bpo=28238].)

O módulo **xml.etree.ElementTree** fornece uma nova função **canonicalize()** que implementa C14N 2.0. (Contribuição de Stefan Behnel em bpo-13611 [https://bugs.python.org/issue?@action=redirect&bpo=13611].)

O objeto de destino de **xml.etree.ElementTree.XMLParser** pode receber eventos de declaração de espaço de nomes através dos novos métodos de retorno de chamada `start_ns()` e `end_ns ()`. Além disso, o destino **xml.etree.ElementTree.TreeBuilder** pode ser configurado para processar eventos sobre comentários e instruções de processamento para incluí-los na árvore gerada. (Contribuição de Stefan Behnel em bpo-36676 [https://bugs.python.org/issue?@action=redirect&bpo=36676] e bpo-36673 [https://bugs.python.org/issue?@action=redirect&bpo=36673].)

xmlrpc

`xmlrpc.client.ServerProxy` agora possui suporte a um argumento nomeado *headers* opcional para que uma sequência de cabeçalhos HTTP seja enviada a cada solicitação. Entre outras coisas, isso possibilita a atualização da autenticação básica padrão para uma autenticação de sessão mais rápida. (Contribuição de Cédric Krier em bpo-35153 [https://bugs.python.org/issue?@action=redirect&bpo=35153].)

Otimizações

- O módulo `subprocess` agora pode usar a função `os.posix_spawn()` em alguns casos para obter melhor desempenho. Atualmente, ele é usado apenas no macOS e Linux (usando glibc 2.24 ou mais recente) se todas essas condições forem atendidas:

  - *close_fds* ser false;
  - os parâmetros *preexec_fn*, *pass_fds*, *cwd* and *start_new_session* não estarem definidos;
  - o caminho *executable* conter um diretório.

  (Contribuição de Joannah Nanjekye e Victor Stinner em bpo-35537 [https://bugs.python.org/issue?@action=redirect&bpo=35537].)

- `shutil.copyfile()`, `shutil.copy()`, `shutil.copy2()`, `shutil.copytree()` e **`shutil.copytree()`** usam chamadas de sistema "fast-copy" específicas da plataforma no Linux e no macOS para copiar o arquivo com mais eficiência. "fast-copy" significa que a operação de cópia ocorre dentro do kernel, evitando o uso de buffers de espaço do usuário no Python como em "`outfd.write(infd.read())`". No Windows, **`shutil.copyfile()`** usa um buffer padrão com tamanho maior (1 MiB em vez de 16 KiB) e é usada uma variante de `shutil.copyfileobj()` baseada em `memoryview()`. A velocidade de cópia de um arquivo 512 MiB na mesma partição é de aproximadamente +26% no Linux, +50% no macOS e +40% no Windows. Além disso, muito menos ciclos de CPU são consumidos. Veja a seção Operações de cópia eficientes

dependentes da plataforma. (Contribuição de Giampaolo Rodolà em bpo-33671 [https://bugs.python.org/issue?@action=redirect&bpo=33671].)

- **`shutil.copytree()`** usa a função **`os.scandir()`** e todas as funções de cópia que dependem dela usam valores em cache de **`os.stat()`**. A velocidade para copiar um diretório com 8000 arquivos é de cerca de +9% no Linux, +20% no Windows e +30% em um compartilhamento SMB do Windows. Além disso, o número de chamadas de sistema **`os.stat()`** é reduzido em 38%, tornando **`shutil.copytree()`** especialmente mais rápida em sistemas de arquivos em rede. (Contribuição de Giampaolo Rodolà em bpo-33695 [https://bugs.python.org/issue?@action=redirect&bpo=33695].)

- O protocolo padrão no módulo **`pickle`** agora é o Protocolo 4, introduzido pela primeira vez no Python 3.4. Ele oferece melhor desempenho e tamanho menor em comparação com o Protocolo 3 disponível desde o Python 3.0.

- Removido um membro **`Py_ssize_t`** de `PyGC_Head`. O tamanho de todos os objetos rastreados pelo coletor de lixo (por exemplo, tupla, lista, ditado) é reduzido em 4 ou 8 bytes. (Contribuição de Inada Naoki em bpo-33597 [https://bugs.python.org/issue?@action=redirect&bpo=33597].)

- **`uuid.UUID`** agora usa `__slots__` para reduzir seu uso de espaço de memória. (Contribuição de Wouter Bolsterlee e Tal Einat em bpo-30977 [https://bugs.python.org/issue?@action=redirect&bpo=30977])

- Melhor desempenho de **`operator.itemgetter()`** em 33%. Manipulação de argumento otimizada e incluído um caminho rápido para o caso comum de um único índice inteiro não negativo em uma tupla (que é o caso de uso típico na biblioteca padrão). (Contribuição de Raymond Hettinger em bpo-35664 [https://bugs.python.org/issue?@action=redirect&bpo=35664].)

- Pesquisas de campo aceleradas em **`collections.namedtuple()`**. Agora elas são duas vezes mais rápidas, tornando-as a forma mais rápida de pesquisa de variáveis de instância no Python. (Contribuição

de Raymond Hettinger, Pablo Galindo e Joe Jevnik, Serhiy Storchaka em bpo-32492 [https://bugs.python.org/issue?@action=redirect&bpo=32492].)

- O construtor `list` não atribui globalmente o buffer interno do item se a entrada iterável tiver um comprimento conhecido (a entrada implementa `__len__`). Isso torna a lista criada 12% menor em média. (Contribuição de Raymond Hettinger e Pablo Galindo em bpo-33234 [https://bugs.python.org/issue?@action=redirect&bpo=33234].)

- Duplicada a velocidade de gravação de variável de classe. Quando um atributo não-dunder era atualizado, havia uma chamada desnecessária para atualizar os slots. (Contribuição de Stefan Behnel, Pablo Galindo Salgado, Raymond Hettinger, Neil Schemenauer e Serhiy Storchaka em bpo-36012 [https://bugs.python.org/issue?@action=redirect&bpo=36012].)

- Reduzida a sobrecarga dos argumentos de conversão passados para muitas funções e métodos internos. Isso acelerou chamando algumas funções e métodos internos simples de 20–50%. (Contribuição de Serhiy Storchaka em bpo-23867 [https://bugs.python.org/issue?@action=redirect&bpo=23867], bpo-35582 [https://bugs.python.org/issue?@action=redirect&bpo=35582] e bpo-36127 [https://bugs.python.org/issue?@action=redirect&bpo=36127].)

- A instrução `LOAD_GLOBAL` agora usa o novo mecanismo “por cache de código de operação”. É cerca de 40% mais rápido agora. (Contribuição de Yury Selivanov e Inada Naoki em :issue: *26219*.)

Mudanças a construções e API C

- O `sys.abiflags` padrão se tornou uma string vazia: o sinalizador `m` para pymalloc se tornou inútil (compilações com e sem pymalloc são compatíveis com ABI) e, portanto, foi removido. (Contribuição de Victor Stinner em bpo-36707 [https://bugs.python.org/issue?@action=redirect&bpo=36707].)

  Exemplos de alterações:

  - Somente o programa `python3.8` é instalado, o programa `python3.8m` se foi.
  - Somente o script `python3.8-config` é instalado, o programa `python3.8m-config` se foi.
  - O sinalizador `m` foi removido do sufixo dos nomes de arquivos da biblioteca dinâmica: módulos de extensão na biblioteca padrão, bem como daqueles produzidos e instalados por pacotes de terceiros, como os baixados do PyPI. No Linux, por exemplo, o sufixo de `.cpython-37m-x86_64-linux-gnu.so` do Python 3.7 se tornou `.cpython-38-x86_64-linux-gnu.so` no Python 3.8.

- Os arquivos de cabeçalho foram reorganizados para separar melhor os diferentes tipos de APIs:

  - `Include/*.h` deve ser a API C pública estável e portátil.
  - `Include/cpython/*.h` deve ser a API C instável específica do CPython; API pública, com alguma API privada prefixada por `_Py` ou `_PY`.
  - `Include/internal/*.h` é a API C interna privada muito específica do CPython. Essa API é fornecida sem garantia de compatibilidade com versões anteriores e não deve ser usada fora do CPython. Ele é exposto apenas a necessidades muito específicas, como depuradores e perfis, que precisam acessar os componentes internos do CPython sem chamar funções. Esta API agora está instalada por `make install`.

  (Contribuição de Victor Stinner em bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134] e bpo-35081 [https://bugs.python.org/issue?@action=redirect&bpo=35081], trabalho iniciado por Eric Snow em Python 3.7.)

- Algumas macros foram convertidas em funções in-line estáticas: os tipos de parâmetros e o tipo de retorno são bem definidos, eles não têm problemas específicos para as macros, as variáveis têm escopos locais. Exemplos:

  - `Py_INCREF()`, `Py_DECREF()`
  - `Py_XINCREF()`, `Py_XDECREF()`

  - **`PyObject_INIT`**, **`PyObject_INIT_VAR`**
  - Funções privadas: **`_PyObject_GC_TRACK()`**, **`_PyObject_GC_UNTRACK()`**, **`_Py_Dealloc()`**

  (Contribuição de Victor Stinner em bpo-35059 [https://bugs.python.org/issue?@action=redirect&bpo=35059].)

- As funções **`PyByteArray_Init()`** e **`PyByteArray_Fini()`** foram removidas. Elas não faziam nada desde o Python 2.7.4 e o Python 3.2.0, foram excluídas da API limitada (ABI estável) e não foram documentados. (Contribuição de Victor Stinner em bpo-35713 [https://bugs.python.org/issue?@action=redirect&bpo=35713].)

- O resultado de **`PyExceptionClass_Name()`** agora é do tipo `const char *` em vez de `char *`. (Contribuição de Serhiy Storchaka em bpo-33818 [https://bugs.python.org/issue?@action=redirect&bpo=33818].)

- A dualidade de `Modules/Setup.dist` e `Modules/Setup` foi removida. Anteriormente, ao atualizar a árvore de fontes do CPython, era necessário copiar manualmente `Modules/Setup.dist` (dentro da árvore de fontes) para `Modules/Setup` (dentro da árvore de compilação), a fim de refletir todas as alterações upstream. Isso foi um pequeno benefício para os empacotadores, à custa de um aborrecimento frequente para os desenvolvedores após o desenvolvimento do CPython, pois esquecer de copiar o arquivo pode causar falhas na compilação.

  Agora, o sistema de compilação sempre lê `Modules/Setup` dentro da árvore de fontes. As pessoas que desejam personalizar esse arquivo são incentivadas a manter suas alterações em um fork do git do CPython ou como arquivos de patch, como faria em qualquer outra alteração na árvore de fontes.

  (Contribuição de Antoine Pitrou em bpo-32430 [https://bugs.python.org/issue?@action=redirect&bpo=32430].)

- Funções que convertem um número Python em número inteiro C, como **`PyLong_AsLong()`**, e funções de análise de argumentos, como

PyArg_ParseTuple(), com unidades de formato de conversão de números inteiros, como `'i'`, agora usarão o método especial **__index__()** em vez de **__int__()**, se disponível. O aviso de descontinuação será emitido para objetos com o método `__int__()`, mas sem o método `__index__()` (como **Decimal** e **Fraction**). **PyNumber_Check()** agora retornará `1` para objetos implementando `__index__()`. **PyNumber_Long()**, **PyNumber_Float()** e **PyFloat_AsDouble()** agora também usam o método `__index__()`, se disponível. (Contribuição de Serhiy Storchaka em bpo-36048 [https://bugs.python.org/issue?@action=redirect&bpo=36048] e bpo-20092 [https://bugs.python.org/issue?@action=redirect&bpo=20092].)

- Objetos de tipo alocado a heap agora aumentarão sua contagem de referências em **PyObject_Init()** (e sua macro paralela `PyObject_INIT`) em vez de em **PyType_GenericAlloc()**. Os tipos que modificam a alocação ou desalocação de instância podem precisar ser ajustados. (Contribuição de Eddie Elizondo em bpo-35810 [https://bugs.python.org/issue?@action=redirect&bpo=35810].)

- A nova função **PyCode_NewWithPosOnlyArgs()** permite criar objetos de código como **PyCode_New()**, mas com um parâmetro extra *posonlyargcount* para indicar o número de argumentos somente posicionais. (Contribuição de Pablo Galindo em bpo-37221 [https://bugs.python.org/issue?@action=redirect&bpo=37221].)

- **Py_SetPath()** agora define **sys.executable** para o caminho completo do programa (**Py_GetProgramFullPath()**) em vez do nome do programa (**Py_GetProgramName()**). (Contribuição de Victor Stinner em bpo-38234 [https://bugs.python.org/issue?@action=redirect&bpo=38234].)

Descontinuados

- O comando distutils `bdist_wininst` agora está descontinuado; use `bdist_wheel` (pacotes wheel). (Contribuição de Victor Stinner em bpo-37481 [https://bugs.python.org/issue?@action=redirect&bpo=37481].)

- Os métodos descontinuados `getchildren()` e `getiterator()` no módulo **`ElementTree`** agora emitem um **`DeprecationWarning`** em vez de **`PendingDeprecationWarning`**. Eles serão removidos no Python 3.9. (Contribuição de Serhiy Storchaka em bpo-29209 [https://bugs.python.org/issue?@action=redirect&bpo=29209].)

- Passar um objeto que não é uma instância de **`concurrent.futures.ThreadPoolExecutor`** para **`loop.set_default_executor()`** foi descontinuado e será proibido no Python 3.9. (Contribuição de Elvis Pranskevichus em bpo-34075 [https://bugs.python.org/issue?@action=redirect&bpo=34075].)

- Os métodos **`__getitem__()`** de **`xml.dom.pulldom.DOMEventStream`**, **`wsgiref.util.FileWrapper`** e **`fileinput.FileInput`** foram descontinuados.

  As implementações desses métodos têm ignorado seu parâmetro *index* e retornado o próximo item. (Contribuição de Berker Peksag em bpo-9372 [https://bugs.python.org/issue?@action=redirect&bpo=9372].)

- A classe **`typing.NamedTuple`** descontinuou o atributo `_field_types` em favor do atributo `__annotations__` que possui as mesmas informações. (Contribuição de Raymond Hettinger em bpo-36320 [https://bugs.python.org/issue?@action=redirect&bpo=36320].)

- As classes `Num`, `Str`, `Bytes`, `NameConstant` e `Ellipsis` do módulo **`ast`** são consideradas descontinuadas e serão removidos em versões futuras do Python. A **`Constant`** deve ser usado. (Contribuição de Serhiy Storchaka em bpo-32892 [https://bugs.python.org/issue?@action=redirect&bpo=32892].)

- Os métodos `visit_Num()`, `visit_Str()`, `visit_Bytes()`, `visit_NameConstant()` e `visit_Ellipsis()` da classe **`ast.NodeVisitor`** estão agora descontinuados e não serão chamados nas versões futuras do Python. Adicione o método **`visit_Constant()`** para manipular todos os nós constantes. (Contribuição de Serhiy Storchaka em bpo-36917 [https://bugs.python.org/issue?@action=redirect&bpo=36917].)

- O decorador **`@asyncio.coroutine`** está descontinuado e será removido na versão 3.10. Em vez de `@asyncio.coroutine`, use **`async def`**. (Contribuição de Andrew Svetlov em bpo-36921 [https://bugs.python.org/issue?@action=redirect&bpo=36921].)

- Em **`asyncio`**, a passagem explícita de um argumento *loop* foi descontinuado e será removido na versão 3.10 para os seguintes: **`asyncio.sleep()`**, **`asyncio.gather()`**, **`asyncio.shield()`**, **`asyncio.wait_for()`**, **`asyncio.wait()`**, **`asyncio.as_completed()`**, **`asyncio.Task`**, **`asyncio.Lock`**, **`asyncio.Event`**, **`asyncio.Condition`**, **`asyncio.Semaphore`**, **`asyncio.BoundedSemaphore`**, **`asyncio.Queue`**, **`asyncio.create_subprocess_exec()`** e **`asyncio.create_subprocess_shell()`**.

- A passagem explícita dos objetos da corrotina para **`asyncio.wait()`** foi descontinuada e será removida na versão 3.11. (Contribuição de Yury Selivanov em bpo-34790 [https://bugs.python.org/issue?@action=redirect&bpo=34790].)

- As seguintes funções e métodos estão descontinuados no módulo **`gettext`**: **`lgettext()`**, **`ldgettext()`**, **`lngettext()`** e **`ldngettext()`**. Eles retornam bytes codificados, e é possível que você receba exceções inesperadas relacionadas ao Unicode se houver problemas de codificação com as strings traduzidas. É muito melhor usar alternativas que retornam strings Unicode no Python 3. Essas funções estão desativadas há muito tempo.

  A função **`bind_textdomain_codeset()`**, os métodos **`NullTranslations.output_charset()`** e **`NullTranslations.set_output_charset()`** e o parâmetro *codeset* das funções **`translation()`** e **`install()`** também estão descontinuados, pois são usados apenas para as funções `l*gettext()`. (Contribuição de Serhiy Storchaka em bpo-33710 [https://bugs.python.org/issue?@action=redirect&bpo=33710].)

- O método **`isAlive()`** de **`threading.Thread`** foi descontinuado. (Contribuição de Donghee Na em bpo-35283 [https://bugs.python.org/issue?@action=redirect&bpo=35283].)

- Muitas funções internas e de extensão que recebem argumentos inteiros agora emitem um aviso de descontinuidade para objetos de `Decimal`, de `Fraction` e quaisquer outros objetos que possam ser convertidos apenas em números inteiros com perda (por exemplo, que possuem o método `__int__()`, mas não têm o método `__index__()`). Em uma versão futura, haverá erros. (Contribuição de Serhiy Storchaka em bpo-36048 [https://bugs.python.org/issue?@action=redirect&bpo=36048].)

- Descontinuada a passagem dos seguintes argumentos como argumentos nomeados:

  - *func* em `functools.partialmethod()`, `weakref.finalize()`, `profile.Profile.runcall()`, `cProfile.Profile.runcall()`, `bdb.Bdb.runcall()`, `trace.Trace.runfunc()` e `curses.wrapper()`.
  - *function* em `unittest.TestCase.addCleanup()`.
  - *fn* no método `submit()` de `concurrent.futures.ThreadPoolExecutor` e `concurrent.futures.ProcessPoolExecutor`.
  - *callback* em `contextlib.ExitStack.callback()`, `contextlib.AsyncExitStack.callback()` e `contextlib.AsyncExitStack.push_async_callback()`.
  - *c* e *typeid* no metódo `create()` de `multiprocessing.managers.Server` e `multiprocessing.managers.SharedMemoryServer`.
  - *obj* em `weakref.finalize()`.

  Em versões futuras do Python, elas serão somente-posicionais. (Contribuição de Serhiy Storchaka em bpo-36492 [https://bugs.python.org/issue?@action=redirect&bpo=36492].)

Remoção de APIs e recursos

Os seguintes recursos e APIs foram removidos do Python 3.8:

- A partir do Python 3.3, a importação de ABCs de **`collections`** foi descontinuada e a importação deve ser feita de **`collections.abc`**. A possibilidade de importar de coleções foi marcada para remoção em 3.8, mas foi adiada para 3.9. (Veja gh-81134 [https://github.com/python/cpython/issues/81134].)
- O módulo **`macpath`**, descontinuado no Python 3.7, foi removido. (Contribuição de Victor Stinner em bpo-35471 [https://bugs.python.org/issue?@action=redirect&bpo=35471].)
- A função **`platform.popen()`** foi removida, após ter sido descontinuada desde o Python 3.3: em vez disso, use **`os.popen()`**. (Contribuição de Victor Stinner em bpo-35345 [https://bugs.python.org/issue?@action=redirect&bpo=35345].)
- A função **`time.clock()`** foi removida, após ter sido descontinuada desde o Python 3.3: use **`time.perf_counter()`** ou **`time.process_time()`**, dependendo de suas necessidades, para ter um comportamento bem definido. (Contribuição de Matthias Bussonnier em bpo-36895 [https://bugs.python.org/issue?@action=redirect&bpo=36895].)
- O script `pyvenv` foi removido em favor do `python3.8 -m venv` para ajudar a eliminar a confusão sobre a qual interpretador Python o script `pyvenv` está associado. (Contribuição de Brett Cannon em bpo-25427 [https://bugs.python.org/issue?@action=redirect&bpo=25427].)
- `parse_qs`, `parse_qsl` e `escape` foram removidos do módulo **`cgi`**. Eles foram descontinuados no Python 3.2 ou mais antigo. Eles devem ser importados dos módulos `urllib.parse` e `html`.
- A função `filemode` foi removida do módulo **`tarfile`**. Não é documentada e foi descontinuada desde o Python 3.3.
- O construtor **`XMLParser`** não aceita mais o argumento *html*. Isso nunca teve efeito e foi descontinuado no Python 3.4. Todos os outros parâmetros agora são somente-nomeados. (Contribuição de Serhiy Storchaka em bpo-29209 [https://bugs.python.org/issue?@action=redirect&bpo=29209].)
- Removido o método `doctype()` de **`XMLParser`**. (Contribuição de Serhiy Storchaka em bpo-29209 [https://bugs.python.org/issue?@action=redirect&bpo=29209].)
- O codec “unicode_internal” foi removido. (Contribuição de Inada Naoki em bpo-36297 [https://bugs.python.org/issue?@action=redirect&bpo=36297].)

- Os objetos `Cache` e `Statement` do módulo **`sqlite3`** não expostos para o usuário. (Contribuição de Aviv Palivoda em bpo-30262 [https://bugs.python.org/issue?@action=redirect&bpo=30262].)
- O argumento nomeado `bufsize` de **`fileinput.input()`** e **`fileinput.FileInput()`**, que era ignorado e estava descontinuado desde o Python 3.6, foi removido. bpo-36952 [https://bugs.python.org/issue?@action=redirect&bpo=36952] (Contribuição de Matthias Bussonnier.)
- As funções **`sys.set_coroutine_wrapper()`** e **`sys.get_coroutine_wrapper()`** descontinuadas no Python 3.7 foram removidas; bpo-36933 [https://bugs.python.org/issue?@action=redirect&bpo=36933] (Contribuição de Matthias Bussonnier.)

Portando para o Python 3.8

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações no comportamento do Python

- Expressões *yield* (ambas as cláusulas `yield` e `yield from`) agora não são permitidas nas compreensões e expressões geradoras (além da expressão iterável na cláusula mais à esquerda **`for`**). (Contribuição de Serhiy Storchaka em bpo-10544 [https://bugs.python.org/issue?@action=redirect&bpo=10544].)
- Agora, o compilador produz um **`SyntaxWarning`** quando verificações de identidade (`is` e `is not`) são usadas com certos tipos de literais (por exemplo, strings, números). Geralmente, eles podem funcionar acidentalmente no CPython, mas não são garantidos pelas especificações da linguagem. O aviso aconselha os usuários a usar testes de igualdade (`==` e `!=`). (Contribuição de Serhiy Storchaka em bpo-34850 [https://bugs.python.org/issue?@action=redirect&bpo=34850].)
- O interpretador do CPython pode engolir exceções em algumas circunstâncias. No Python 3.8, isso acontece em menos casos. Em particular, as exceções levantadas ao obter o atributo do dicionário de tipos não são mais ignoradas. (Contribuição de Serhiy Storchaka em bpo-35459 [https://bugs.python.org/issue?@action=redirect&bpo=35459].)

- Removidas as implementações `__str__` dos tipos embutidos **`bool`**, **`int`**, **`float`**, **`complex`** e poucas classes da biblioteca padrão. Agora eles herdam `__str__()` de **`object`**. Como resultado, definir o método `__repr__()` na subclasse dessas classes afetará sua representação de string. (Contribuição de Serhiy Storchaka em bpo-36793 [https://bugs.python.org/issue?@action=redirect&bpo=36793].)
- No AIX, **`sys.platform`** não contém mais a versão principal. É sempre `'aix'`, em vez de `'aix3'` .. `'aix7'`. Como as versões mais antigas do Python incluem o número da versão, é recomendável usar sempre `sys.platform.startswith('aix')`. (Contribuição de M. Felt em bpo-36588 [https://bugs.python.org/issue?@action=redirect&bpo=36588].)
- **`PyEval_AcquireLock()`** e **`PyEval_AcquireThread()`** agora encerram a thread atual se chamado enquanto o interpretador está finalizando, tornando-os consistentes com **`PyEval_RestoreThread()`**, **`Py_END_ALLOW_THREADS()`** e **`PyGILState_Ensure()`**. Se esse comportamento não for desejado, proteja a chamada verificando **`_Py_IsFinalizing()`** ou **`sys.is_finalizing()`**. (Contribuição de Joannah Nanjekye em bpo-36475 [https://bugs.python.org/issue?@action=redirect&bpo=36475].)

Alterações na API Python

- A função **`os.getcwdb()`** agora usa a codificação UTF-8 no Windows, em vez da página de códigos ANSI: veja **PEP 529** [https://peps.python.org/pep-0529/] para a justificativa. A função não é mais descontinuada no Windows. (Contribuição de Victor Stinner em bpo-37412 [https://bugs.python.org/issue?@action=redirect&bpo=37412].)
- **`subprocess.Popen`** agora pode usar **`os.posix_spawn()`** em alguns casos para melhor desempenho. No Subsistema Windows para Linux e na Emulação de Usuário QEMU, o construtor **`Popen`** usando **`os.posix_spawn()`** não levanta mais uma exceção em erros como “missing program” (programa ausente). Em vez disso, o processo filho falha com um **`returncode`** diferente de zero. (Contribuição de Joannah Nanjekye e Victor Stinner em bpo-35537 [https://bugs.python.org/issue?@action=redirect&bpo=35537].)
- O argumento *preexec_fn* de * **`subprocess.Popen`** não é mais compatível com subinterpretadores. O uso do parâmetro em um

subinterpretador agora levanta **`RuntimeError`**. (Contribuição de Eric Snow em bpo-34651 [https://bugs.python.org/issue?@action=redirect&bpo=34651], modificado por Christian Heimes em bpo-37951 [https://bugs.python.org/issue?@action=redirect&bpo=37951].)

- O método **`imaplib.IMAP4.logout()`** não ignora mais silenciosamente exceções arbitrárias. (Contribuição de Victor Stinner em bpo-36348 [https://bugs.python.org/issue?@action=redirect&bpo=36348].)
- A função **`platform.popen()`** foi removida, após ter sido descontinuada desde o Python 3.3: em vez disso, use **`os.popen()`**. (Contribuição de Victor Stinner em bpo-35345 [https://bugs.python.org/issue?@action=redirect&bpo=35345].)
- A função **`Statistics.mode()`** não levanta mais uma exceção quando dados multimodais são fornecidos. Em vez disso, ela retorna o primeiro modo encontrado nos dados de entrada. (Contribuição de Raymond Hettinger em bpo-35892 [https://bugs.python.org/issue?@action=redirect&bpo=35892].)
- O método **`selection()`** da classe **`tkinter.ttk.Treeview`** não recebe mais argumentos. Seu uso com argumentos para alterar a seleção foi descontinuado no Python 3.6. Use métodos especializados como **`selection_set()`** para alterar a seleção. (Contribuição de Serhiy Storchaka em bpo-31508 [https://bugs.python.org/issue?@action=redirect&bpo=31508].)
- Os métodos **`writexml()`**, **`toxml()`** e **`toprettyxml()`** de **`xml.dom.minidom`** e o método **`write()`** de **`xml.etree.ElementTree`** agora preservam a ordem dos atributos especificada pelo usuário. (Contribuição de Diego Rojas e Raymond Hettinger em bpo-34160 [https://bugs.python.org/issue?@action=redirect&bpo=34160].)
- Um banco de dados **`dbm.dumb`** aberto com sinalizadores `'r'` agora é somente leitura. **`dbm.dumb.open()`** com os sinalizadores `'r'` e `'w'` não cria mais um banco de dados se ele não existir. (Contribuição de Serhiy Storchaka em bpo-32749 [https://bugs.python.org/issue?@action=redirect&bpo=32749].)
- O método `doctype()` definido em uma subclasse de **`XMLParser`** não será mais chamado e emitirá um **`RuntimeWarning`** em vez de um **`DeprecationWarning`**. Defina o método **`doctype()`** em um alvo para manipular uma declaração doctype XML. (Contribuição de Serhiy

Storchaka em bpo-29209 [https://bugs.python.org/issue?@action=redirect&bpo=29209].)

- Uma `RuntimeError` agora é levantada quando a metaclasse personalizada não fornece a entrada `__classcell__` no espaço de nomes passado para `type.__new__`. Uma `DeprecationWarning` era emitida no Python 3.6-3.7. (Contribuição de Serhiy Storchaka em bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722].)
- A classe `cProfile.Profile` agora pode ser usada como um gerenciador de contexto. (Contribuição de Scott Sanderson em bpo-29235 [https://bugs.python.org/issue?@action=redirect&bpo=29235].)
- `shutil.copyfile()`, `shutil.copy()`, `shutil.copy2()`, `shutil.copytree()` e `shutil.copytree()` usam chamadas de sistema de "cópia rápido" específica da plataforma (veja a seção Operações de cópia eficientes dependentes da plataforma).
- O tamanho padrão do buffer de `shutil.copyfile()` no Windows foi alterado de 16 KiB para 1 MiB.
- A estrutura de `PyGC_Head` mudou completamente. Todo o código que tocava no membro struct deve ser reescrito. (Veja bpo-33597 [https://bugs.python.org/issue?@action=redirect&bpo=33597].)
- A estrutura de `PyInterpreterState` foi movida para os arquivos de cabeçalho "internal" (especificamente Include/internal/pycore_pystate.h). Um `PyInterpreterState` opaco ainda está disponível como parte da API pública (e ABI estável). Os documentos indicam que nenhum dos campos da estrutura é público, por isso esperamos que ninguém os tenha usado. No entanto, se você confia em um ou mais desses campos em particular e não tiver alternativa, abra um relatório de erro no BPO. Vamos trabalhar para ajudá-lo a ajustar (possivelmente incluindo a adição de funções de acessador à API pública). (Veja bpo-35886 [https://bugs.python.org/issue?@action=redirect&bpo=35886].)
- O método `mmap.flush()` agora retorna `None` com êxito e levanta uma exceção no erro em todas as plataformas. Anteriormente, seu comportamento dependia da plataforma: um valor diferente de zero era retornado com êxito; zero era retornado com erro no Windows. Um valor zero era retornado com sucesso; uma exceção era levanta no caso de erro no Unix. (Contribuição de Berker Peksag em bpo-2122 [https://bugs.python.org/issue?@action=redirect&bpo=2122].)

- Os módulos `xml.dom.minidom` e `xml.sax` não mais processam entidades externas por padrão. (Contribuição de Christian Heimes em bpo-17239 [https://bugs.python.org/issue?@action=redirect&bpo=17239].)
- Excluir uma chave de um banco de dados `dbm` somente leitura (`dbm.dumb`, `dbm.gnu` ou `dbm.ndbm`) levanta `error` (`dbm.dumb.error`, `dbm.gnu.error` ou `dbm.ndbm.error`) em vez de `KeyError`. (Contribuição de Xiang Zhang em bpo-33106 [https://bugs.python.org/issue?@action=redirect&bpo=33106].)
- AST foi simplificado para literais. Todas as constantes serão representadas como instâncias de `ast.Constant`. Instanciar classes antigas `Num`, `Str`, `Bytes`, `NameConstant` e `Ellipsis` retornará uma instância de `Constant`. (Contribuição de Serhiy Storchaka em bpo-32892 [https://bugs.python.org/issue?@action=redirect&bpo=32892].)
- `expanduser()` no Windows agora prefere a variável de ambiente `USERPROFILE` e não usa `HOME`, que normalmente não é definido para contas de usuário comuns. (Contribuição de Anthony Sottile em bpo-36264 [https://bugs.python.org/issue?@action=redirect&bpo=36264].)
- A exceção `asyncio.CancelledError` agora herda de `BaseException` em vez de `Exception` e não mais herda de `concurrent.futures.CancelledError`. (Contribuição de Yury Selivanov em bpo-32528 [https://bugs.python.org/issue?@action=redirect&bpo=32528].)
- A função `asyncio.wait_for()` agora aguarda corretamente o cancelamento ao usar uma instância de `asyncio.Task`. Anteriormente, ao atingir tempo limite, ele era cancelado e devolvido imediatamente. (Contribuição de Elvis Pranskevichus em bpo-32751 [https://bugs.python.org/issue?@action=redirect&bpo=32751].)
- A função `asyncio.BaseTransport.get_extra_info()` agora retorna um objeto soquete seguro de usar quando ‘socket’ é passado para o parâmetro *name*. (Contribuição de Yury Selivanov em bpo-37027 [https://bugs.python.org/issue?@action=redirect&bpo=37027].)
- `asyncio.BufferedProtocol` foi promovida a API estável.

- As dependências de DLL para módulos de extensão e DLLs carregadas com `ctypes` no Windows agora são resolvidas com mais segurança. Somente os caminhos do sistema, o diretório que contém o arquivo DLL ou PYD e os diretórios adicionados com

add_dll_directory() são pesquisados por dependências de tempo de carregamento. Especificamente, **PATH** e o diretório de trabalho atual não são mais usados, e as modificações nelas não terão mais efeito na resolução normal da DLL. Se seu aplicativo conta com esses mecanismos, você deve verificar **add_dll_directory()** e, se existir, use-o para adicionar seu diretório DLLs ao carregar sua biblioteca. Observe que os usuários do Windows 7 precisarão garantir que o Windows Update KB2533623 esteja instalado (isso também é verificado pelo instalador). (Contribuição de Steve Dower em bpo-36085 [https://bugs.python.org/issue?@action=redirect&bpo=36085].)

- Os arquivos de cabeçalho e as funções relacionadas ao pgen foram removidos após sua substituição por uma implementação pura do Python. (Contribuição de Pablo Galindo em bpo-36623 [https://bugs.python.org/issue?@action=redirect&bpo=36623].)
- **types.CodeType** possui um novo parâmetro na segunda posição do construtor (*posonlyargcount *) para suportar argumentos apenas de posição definidos em :pep:`570`. O primeiro argumento (*argcount*) agora representa o número total de argumentos posicionais (incluindo argumentos somente posicionais). O novo método `replace()` de **types.CodeType** pode ser usado para tornar o código à prova de futuro.
- O parâmetro `digestmod` para **hmac.new()** não usa mais o resumo MD5 por padrão.

Alterações na API C

- A estrutura **PyCompilerFlags** recebeu um novo campo *cf_feature_version*. Ele deve ser inicializado para `PY_MINOR_VERSION`. O campo é ignorado por padrão e é usado se e somente se o sinalizador `PyCF_ONLY_AST` estiver definido em *cf_flags*. (Contribuição de Guido van Rossum em bpo-35766 [https://bugs.python.org/issue?@action=redirect&bpo=35766].)

- A função **PyEval_ReInitThreads()** foi removida da API C. Não deve ser chamada explicitamente: use **PyOS_AfterFork_Child()**. (Contribuição de Victor Stinner em bpo-36728 [https://bugs.python.org/issue?@action=redirect&bpo=36728].)

- No Unix, as extensões C não estão mais vinculadas ao libpython, exceto no Android e Cygwin. Quando o Python é incorporado, `libpython` não deve ser carregado com `RTLD_LOCAL`, mas `RTLD_GLOBAL`. Anteriormente, usando `RTLD_LOCAL`, já não era possível carregar extensões C que não estavam vinculadas a `libpython`, como extensões C da biblioteca padrão criada pela seção `*shared*` de `Modules/Setup`. (Contribuição de Victor Stinner em bpo-21536 [https://bugs.python.org/issue?@action=redirect&bpo=21536].)

- O uso de variantes `#` de formatos na análise ou construção de valor (por exemplo, **`PyArg_ParseTuple()`**, **`Py_BuildValue()`**, **`PyObject_CallFunction()`**, etc.) sem `PY_SSIZE_T_CLEAN` definido agora levanta `DeprecationWarning`. Será removido em 3.10 ou 4.0. Leia Análise de argumentos e construção de valores para detalhes. (Contribuição de Inada Naoki em bpo-36381 [https://bugs.python.org/issue?@action=redirect&bpo=36381].)

- Instâncias de tipos alocados em heap (como aquelas criadas com **`PyType_FromSpec()`**) mantêm uma referência ao seu objeto de tipo. O aumento da contagem de referências desses objetos de tipo foi movido de **`PyType_GenericAlloc()`** para as funções de nível mais baixo, **`PyObject_Init()`** e **`PyObject_INIT`**. Isso faz com que os tipos criados por meio de **`PyType_FromSpec()`** se comportem como outras classes em código gerenciado.

  Tipos alocados estaticamente não são afetados.

  Para a grande maioria dos casos, não deve haver efeito colateral. No entanto, tipos que aumentam manualmente a contagem de referência após alocar uma instância (talvez para solucionar o bug) agora podem se tornar imortais. Para evitar isso, essas classes precisam chamar Py_DECREF no objeto de tipo durante a desalocação da instância.

  Para portar corretamente esses tipos para o 3.8, aplique as seguintes alterações:

  - Remove **`Py_INCREF`** do objeto de tipo após alocar uma instância, se houver. Isso pode ocorrer após chamar **`PyObject_New`**,

`PyObject_NewVar`, `PyObject_GC_New()`, `PyObject_GC_NewVar()` ou qualquer outro alocador personalizado que use `PyObject_Init()` ou **`PyObject_INIT`**.

Exemplo:

```
static foo_struct *
foo_new(PyObject *type) {
    foo_struct *foo = PyObject_GC_New(foo_struct, (PyTy
    if (foo == NULL)
        return NULL;
#if PY_VERSION_HEX < 0x03080000
    // Solução de contorno para o issue 35810 do Python
    PY_INCREF(type)
#endif
    return foo;
}
```

- Certifica-se de que todas as funções `tp_dealloc` personalizadas de tipos alocados em heap diminuam a contagem de referências do tipo.

  Exemplo:

```
static void
foo_dealloc(foo_struct *instance) {
    PyObject *type = Py_TYPE(instance);
    PyObject_GC_Del(instance);
#if PY_VERSION_HEX >= 0x03080000
    // Isso não era necessário antes do Python 3.8 (issu
    Py_DECREF(type);
#endif
}
```

(Contribuição de Eddie Elizondo em bpo-35810 [https://bugs.python.org/issue?@action=redirect&bpo=35810].)

- A macro `Py_DEPRECATED()` foi implementada para MSVC. A macro agora deve ser colocada antes do nome do símbolo.

Exemplo:

```
Py_DEPRECATED(3.8) PyAPI_FUNC(int) Py_OldFunction(void);
```

(Contribuição de Zackery Spytz em bpo-33407 [https://bugs.python.org/issue?@action=redirect&bpo=33407].)

- O interpretador não pretende mais dar suporte à compatibilidade binária de tipos de extensão nas liberações de recursos. O **`PyTypeObject`** exportado por um módulo de extensão de terceiros deve ter todos os slots esperados na versão atual do Python, incluindo **`tp_finalize`** (**`Py_TPFLAGS_HAVE_FINALIZE`** não é mais verificado antes de ler **`tp_finalize`**).

  (Contribuição de Antoine Pitrou em bpo-32388 [https://bugs.python.org/issue?@action=redirect&bpo=32388].)

- As funções **`PyNode_AddChild()`** e **`PyParser_AddToken()`** agora aceitam dois argumentos adicionais `int` *end_lineno* e *end_col_offset*.

- O arquivo `libpython38.a` para permitir que as ferramentas do MinGW se vinculem diretamente a `python38.dll` não está mais incluído na distribuição regular do Windows. Se você precisar deste arquivo, ele poderá ser gerado com as ferramentas `gendef` e `dlltool`, que fazem parte do pacote binutils do MinGW:

  ```
  gendef - python38.dll > tmp.def
  dlltool --dllname python38.dll --def tmp.def --output-lib l
  ```

  A localização de um arquivo `pythonXY.dll` dependerá das opções de instalação e da versão e idioma do Windows. Consulte Utilizando Python no Windows para mais informações. A biblioteca resultante deve ser colocada no mesmo diretório que `pythonXY.lib`, que geralmente é o diretório `libs` na sua instalação do Python.

  (Contribuição de Steve Dower em bpo-37351 [https://bugs.python.org/issue?@action=redirect&bpo=37351].)

Alterações de bytecode do CPython

- O loop do interpretador foi simplificado movendo a lógica de desenrolar a pilha de blocos no compilador. O compilador agora emite instruções explícitas para ajustar a pilha de valores e chamar o código de limpeza para **`break`**, **`continue`** e **`return`**.

  Removidos os códigos de operação **`BREAK_LOOP`**, **`CONTINUE_LOOP`**, **`SETUP_LOOP`** e **`SETUP_EXCEPT`**. Adicionados novos códigos de opção **`ROT_FOUR`**, **`BEGIN_FINALLY`**, **`CALL_FINALLY`** e **`POP_FINALLY`**. Alterado o comportamento de **`END_FINALLY`** e **`WITH_CLEANUP_START`**.

  (Contribuição de Mark Shannon, Antoine Pitrou e Serhiy Storchaka em bpo-17611 [https://bugs.python.org/issue?@action=redirect&bpo=17611].)

- Adicionado novo código de operação **`END_ASYNC_FOR`** para lidar com exceções geradas ao aguardar um próximo item em um loop **`async for`**. (Contribuição de Serhiy Storchaka em bpo-33041 [https://bugs.python.org/issue?@action=redirect&bpo=33041].)

- O **`MAP_ADD`** agora espera o valor como o primeiro elemento na pilha e a chave como o segundo elemento. Essa alteração foi feita para que a chave seja sempre avaliada antes do valor na compreensão do dicionário, conforme proposto em **PEP 572** [https://peps.python.org/pep-0572/]. (Contribuição de Jörn Heissler em bpo-35224 [https://bugs.python.org/issue?@action=redirect&bpo=35224].)

Ferramentas e daemons

Adicionado um script de benchmark para cronometrar várias maneiras de acessar variáveis: `Tools/scripts/var_access_benchmark.py`. (Contribuição de Raymond Hettinger em bpo-35884 [https://bugs.python.org/issue?@action=redirect&bpo=35884].)

Aqui está um resumo das melhorias de desempenho desde Python 3.3:

```
Python version                      3.3     3.4     3.5     3.6
--------------                      ---     ---     ---     ---
```

```
Variable and attribute read access:
    read_local                        4.0     7.1     7.1     5.4
    read_nonlocal                     5.3     7.1     8.1     5.8
    read_global                      13.3    15.5    19.0    14.3
    read_builtin                     20.0    21.1    21.6    18.5
    read_classvar_from_class         20.5    25.6    26.5    20.7
    read_classvar_from_instance      18.5    22.8    23.5    18.8
    read_instancevar                 26.8    32.4    33.1    28.0
    read_instancevar_slots           23.7    27.8    31.3    20.8
    read_namedtuple                  68.5    73.8    57.5    45.0
    read_boundmethod                 29.8    37.6    37.9    29.6

Variable and attribute write access:
    write_local                       4.6     8.7     9.3     5.5
    write_nonlocal                    7.3    10.5    11.1     5.6
    write_global                     15.9    19.7    21.2    18.0
    write_classvar                   81.9    92.9    96.0   104.6
    write_instancevar                36.4    44.6    45.8    40.0
    write_instancevar_slots          28.7    35.6    36.1    27.3

Data structure read access:
    read_list                        19.2    24.2    24.5    20.8
    read_deque                       19.9    24.7    25.5    20.2
    read_dict                        19.7    24.3    25.7    22.3
    read_strdict                     17.9    22.6    24.3    19.5

Data structure write access:
    write_list                       21.2    27.1    28.5    22.5
    write_deque                      23.8    28.7    30.1    22.7
    write_dict                       25.9    31.4    33.3    29.3
    write_strdict                    22.9    28.4    29.9    27.5

Stack (or queue) operations:
    list_append_pop                 144.2    93.4   112.7    75.4
    deque_append_pop                 30.4    43.5    57.0    49.4
    deque_append_popleft             30.8    43.7    57.3    49.7

Timing loop:
    loop_overhead                     0.3     0.5     0.6     0.4
```

Os benchmarks foram medidos em um processador Intel® Core™ i7-4960HQ [https://ark.intel.com/content/www/us/en/ark/products/76088/intel-core-i7-4960hq-processor-6m-cache-up-to-3-80-ghz.html] nos sistemas macOS 64-bit encontrados em python.org [https://www.python.org/downloads/macos/]. O script de benchmark exibe tempos em nanossegundos.

Alterações notáveis no Python 3.8.1

Devido a significativas preocupações de segurança, o parâmetro *reuse_address* de `asyncio.loop.create_datagram_endpoint()` não é mais suportado. Isso ocorre devido ao comportamento da opção de soquete `SO_REUSEADDR` no UDP. Para mais detalhes, consulte a documentação de `loop.create_datagram_endpoint()`. (Contribuição de Kyle Stanley, Antoine Pitrou e Yury Selivanov em bpo-37228 [https://bugs.python.org/issue?@action=redirect&bpo=37228].)

Alterações notáveis no Python 3.8.2

Corrigida uma regressão com o retorno de chamada `ignore` de `shutil.copytree()`. Os tipos de argumento agora são str e List[str] novamente. (Contribuição de Manuel Barkhau e Giampaolo Rodola em gh-83571 [https://github.com/python/cpython/issues/83571].)

Alterações notáveis no Python 3.8.3

Os valores constantes dos futuros sinalizadores no módulo `__future__` fpram atualizados para evitar colisão com os sinalizadores do compilador. Anteriormente, `PyCF_ALLOW_TOP_LEVEL_AWAIT` estava colidindo com `CO_FUTURE_DIVISION`. (Contribuição de Batuhan Taskaya em gh-83743 [https://github.com/python/cpython/issues/83743])

Alterações notáveis no Python 3.8.8

Versões anteriores do Python permitiam o uso de `;` e `&` como separadores de parâmetros de consulta nas funções `urllib.parse.parse_qs()` e

`urllib.parse.parse_qsl()`. Devido a questões de segurança, e em conformidade com as recomendações mais recentes do W3C, isso foi alterado para permitir apenas uma única chave separadora, com `&` como padrão. Esta mudança também afeta as funções **`cgi.parse()`** e **`cgi.parse_multipart()`** já que elas usam as funções afetadas internamente. Para obter mais detalhes, consulte sua respectiva documentação. (Contribuição de Adam Goldschmidt, Senthil Kumaran e Ken Jin em bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967].)

Alterações notáveis no Python 3.8.9

Uma correção de segurança altera o comportamento **`ftplib.FTP`** para não confiar no endereço IPv4 enviado do servidor remoto ao configurar um canal de dados passivo. Em vez disso, reutilizamos o endereço IP do servidor ftp. Para códigos incomuns que requerem o comportamento antigo, defina um atributo `trust_server_pasv_ipv4_address` em sua instância de FTP para `True`. (Veja gh-87451 [https://github.com/python/cpython/issues/87451])

Alterações notáveis no Python 3.8.10

Suporte a macOS 11.0 (Big Sur) e Apple Silicon Mac

A partir do 3.8.10, Python oferece suporte total à construção e execução no macOS 11 (Big Sur) e no Apple Silicon Macs (baseado na arquitetura `ARM64`). Uma nova variante de compilação universal, `universal2`, está agora disponível com suporte nativo para `ARM64` e `Intel 64` em um conjunto de executáveis. Observe que suporte para “ligação fraca”, criar binários direcionados para versões mais recentes do macOS que também serão executados corretamente em versões mais antigas através da verificação em tempo de execução por recursos ausentes, não está mais incluída de forma retroativa do Python 3.9; para oferece suporte a uma gama de versões do macOS, continue a visar e a desenvolver para a versão mais antiga do intervalo.

(Contribuição original de Ronald Oussoren e Lawrence D’Anna em gh-85272 [https://github.com/python/cpython/issues/85272], com correções de FX Coudert

e Eli Rykoff, e portado para a versão 3.8 por Maxime Bélanger e Ned Deily)

Alterações notáveis no Python 3.8.10

urllib.parse

A presença de caracteres de nova linha ou tab em partes de um URL permite algumas formas de ataques. Seguindo a especificação WHATWG que atualiza **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], nova linha ASCII `\n`, `\r` e os caracteres de tabulação `\t` são retirados da URL pelo analisador sintático em **`urllib.parse`** impedindo tais ataques. Os caracteres de remoção são controlados por uma nova variável de nível de módulo `urllib.parse._UNSAFE_URL_BYTES_TO_REMOVE`. (Veja bpo-43882 [https://bugs.python.org/issue?@action=redirect&bpo=43882])

Alterações notáveis no Python 3.8.12

Alterações na API Python

A partir do Python 3.8.12, o módulo **`ipaddress`** não aceita mais zeros à esquerda em strings de endereço IPv4. Os zeros à esquerda são ambíguos e interpretados como notação octal por algumas bibliotecas. Por exemplo, a função legada **`socket.inet_aton()`** trata os zeros à esquerda como notação octal. A implementação glibc de **`inet_pton()`** moderna não aceita nenhum zeros à esquerda.

(Originalmente contribuição de Christian Heimes em bpo-36384 [https://bugs.python.org/issue?@action=redirect&bpo=36384] e backport para 3.8 por Achraf Merzouki.)

Recursos de segurança notáveis no 3.8.14

Converter entre **`int`** e **`str`** em bases diferentes de 2 (binário), 4, 8 (octal), 16 (hexadecimal) ou 32 como base 10 (decimal) agora levanta uma exceção

`ValueError` se o número de dígitos em forma de string estiver acima de um limite para evitar possíveis ataques de negação de serviço devido à complexidade algorítmica. Esta é uma mitigação para **CVE 2020-10735** [https://www.cve.org/CVERecord?id=CVE-2020-10735]. Este limite pode ser configurado ou desabilitado por variável de ambiente, sinalizador de linha de comando ou APIs de `sys`. Veja a documentação de limitação de comprimento de conversão de string inteira. O limite padrão é de 4300 dígitos em forma de string.

Alterações notáveis no 3.8.17

tarfile

- Os métodos de extração em `tarfile` e `shutil.unpack_archive()`, têm um novo argumento *filter* que permite limitar recursos do tar que podem ser surpreendentes ou perigosos, como criar arquivos fora do diretório de destino. Veja Filtros de extração para detalhes. No Python 3.12, usar sem o argumento *filter* mostrará um `DeprecationWarning`. No Python 3.14, o padrão mudará para `'data'`. (Contribuição de Petr Viktorin em **PEP 706** [https://peps.python.org/pep-0706/].)

O que há de novo no Python 3.7

Editor:: Elvis Pranskevichus <elvis@magic.io>

Este artigo explica os novos recursos no Python 3.7, em comparação com 3.6. Python 3.7 foi lançado em 27 de Junho de 2018. Veja changelog para uma lista completa de mudanças.

Resumo – Destaques da versão

Novos recursos de sintaxe:

- PEP 563, avaliação postergada de anotações de tipos.

Alterações de sintaxe incompatíveis com versões anteriores:

- `async` e `await` são agora palavras chave reservadas.

Novos módulos de biblioteca:

- `contextvars`: PEP 567 – Variáveis de Contexto
- `dataclasses`: PEP 557 – Classes de Dados
- importlib.resources

Novos recursos embutidos:

- PEP 553, a nova função `breakpoint()`.;

Melhorias no modelo de dados Python:

- PEP 562, personalização de acesso aos atributos do módulo.
- PEP 560, suporte principal para o módulo typing e tipos genéricos.
- a natureza de preservação da ordem de inserção de objetos dict foi declarada [https://mail.python.org/pipermail/python-dev/2017-December/151283.html] como uma parte oficial da especificação da linguagem Python.

Melhorias significativas na biblioteca padrão:

- O módulo `asyncio` recebeu novos recursos, significativo à melhorias de usabilidade e desempenho.
- O módulo `time` ganhou suporte para funções com resolução de nanossegundos.

Melhorias na implementação do CPython:

- Evitando o uso de ASCII como a codificação de texto padrão:
  - PEP 538, coerção de localidade C legada
  - PEP 540, modo de tempo de execução UTF-8 forçado.
- PEP 552, .pycs determinísticos
- Novo Modo de Desenvolvimento do Python
- PEP 565, aprimorada o tratamento de `DeprecationWarning`

Melhorias na API C:

- PEP 539, nova API C para armazenamento local de thread

Melhorias na documentação:

- PEP 545, Traduções da documentação do Python
- Novas traduções da documentação: japonês [https://docs.python.org/ja/], francês [https://docs.python.org/fr/] e coreano [https://docs.python.org/ko/].

Esta versão apresenta melhorias de desempenho notáveis em muitas áreas. A seção Otimizações os lista em detalhes.

Para uma lista de mudanças que podem afetar a compatibilidade com versões anteriores do Python, consulte a seção Porting to Python 3.7.

Novas funcionalidades

PEP 563: Avaliação Postergada de Anotações

O advento das dicas de tipo no Python revelou dois problemas de usabilidade gritantes com a funcionalidade das anotações adicionadas em

PEP 3107 [https://peps.python.org/pep-3107/] e refinadas ainda mais em **PEP 526** [https://peps.python.org/pep-0526/]:

- as anotações só podiam usar nomes que já estavam disponíveis no escopo atual, ou seja, não suportavam referências de encaminhamento de nenhum tipo; e
- anotar o código-fonte teve efeitos adversos no tempo de inicialização dos programas Python.

Ambos os problemas são corrigidos com o adiamento da avaliação das anotações. Em vez de compilar o código que executa expressões em anotações em seu tempo de definição, o compilador armazena a anotação em um formato de string equivalente ao AST da expressão em questão. Se necessário, as anotações podem ser resolvidas em tempo de execução usando **`digitando.get_type_hints()`**. No caso comum em que isso não é necessário, as anotações são mais baratas de armazenar (uma vez que strings curtas são internadas pelo interpretador) e tornam o tempo de inicialização mais rápido.

Em termos de usabilidade, as anotações agora suportam referências futuras, tornando a seguinte sintaxe válida:

```
class C:
    @classmethod
    def from_string(cls, source: str) -> C:
        ...

    def validate_b(self, obj: B) -> bool:
        ...

class B:
    ...
```

Como essa mudança quebra a compatibilidade, o novo comportamento precisa ser habilitado por módulo no Python 3.7 usando uma importação de **`__future__`**

```
from __future__ import annotations
```

Ele se tornará o padrão no Python 3.10.

Ver também

PEP 563 [https://peps.python.org/pep-0563/] – avaliação postergada de anotações de tipos

PEP escrita e implementada por Łukasz Langa.

PEP 538: Coerção de localidade C legada

Um desafio contínuo dentro da série Python 3 tem sido determinar uma estratégia padrão sensata para lidar com a suposição de codificação de texto "ASCII de 7 bits" atualmente implícita pelo uso da localidade C ou POSIX padrão em plataformas não Windows.

A **PEP 538** [https://peps.python.org/pep-0538/] atualiza a interface de linha de comando do interpretador padrão para forçar automaticamente essa localidade para uma localidade baseada em UTF-8 disponível conforme descrito na documentação da nova variável de ambiente `PYTHONCOERCECLOCALE`. Configurar automaticamente `LC_CTYPE` desta forma significa que tanto o interpretador central quanto as extensões C com reconhecimento da localidade (como `readline`) irão presumir o uso de UTF-8 como a codificação de texto padrão, ao invés de ASCII.

A definição de suporte de plataforma na **PEP 11** [https://peps.python.org/pep-0011/] também foi atualizada para limitar o suporte de manipulação de texto completo para localidades não baseadas em ASCII adequadamente configuradas.

Como parte dessa mudança, o tratador de erros padrão para `stdin` e `stdout` agora é `surrogateescape` (em vez de `strict`) ao usar qualquer uma das localidades de destino de coerção definidas (atualmente `C.UTF-8`, `C.utf8` e `UTF-8`). O tratador de erros padrão para `stderr` continua a ser `backslashreplace`, independente da localidade.

A coerção de localidade é silenciosa por padrão, mas para ajudar na depuração de problemas de integração potencialmente relacionados à localidade, avisos explícitos (emitidos diretamente em `stderr`) podem ser solicitados configurando `PYTHONCOERCECLOCALE=warn`. Essa configuração também fará com que o tempo de execução do Python emita um aviso se a localidade C legada permanecer ativa quando o interpretador principal for inicializado.

Embora a coerção de localidade **PEP 538** [https://peps.python.org/pep-0538/] tenha o benefício de também afetar os módulos de extensão (como GNU `readline`), bem como processos filhos (incluindo aqueles que executam aplicativos não Python e versões mais antigas do Python), tem a desvantagem de exigir que uma localidade de destino adequado esteja presente no sistema em execução. Para tratador melhor o caso em que nenhuma localidade de destino adequado está disponível (como ocorre no RHEL/CentOS 7, por exemplo), o Python 3.7 também implementa PEP 540: Modo de tempo de execução UTF-8 forçado.

Ver também

PEP 538 [https://peps.python.org/pep-0538/] – Forçando a localidade C legada para uma localidade baseada em UTF-8
PEP escrita e implementada por Nick Coghlan.

PEP 540: Modo de tempo de execução UTF-8 forçado

A nova opção de linha de comando `-X utf8` e a variável de ambiente `PYTHONUTF8` podem ser usadas para habilitar o modo UTF-8 do Python.

Quando no modo UTF-8, o CPython ignora as configurações de localidade e usa a codificação UTF-8 por padrão. Os tratador de erros para fluxos `sys.stdin` e `sys.stdout` são configurados para `surrogateescape`.

O modo UTF-8 forçado pode ser usado para alterar o comportamento de manipulação de texto em um interpretador Python embutido sem alterar as configurações de localidade de uma aplicação embutida.

Embora o modo UTF-8 da **PEP 540** [https://peps.python.org/pep-0540/] tenha a vantagem de funcionar independentemente de quais localidades estão disponíveis no sistema em execução, ele tem a desvantagem de não ter efeito sobre os módulos de extensão (como GNU `readline`) , processos filho executando aplicativos não-Python e processos filho executando versões anteriores do Python. Para reduzir o risco de corromper dados de texto ao se comunicar com tais componentes, o Python 3.7 também implementa PEP 540: Modo de tempo de execução UTF-8 forçado).

O modo UTF-8 é habilitado por padrão quando a localidade é `C` ou `POSIX`, e o recurso de coerção de localidade **PEP 538** [https://peps.python.org/pep-0538/] falha em mudá-la para uma alternativa baseada em UTF-8 (seja falha é devido a `PYTHONCOERCECLOCALE=0` sendo definido, `LC_ALL` sendo definido, ou a falta de uma localidade de destino adequada).

Ver também

PEP 540 [https://peps.python.org/pep-0540/] – Adiciona um novo modo UTF-8
PEP escrito e implementado por Victor Stinner

PEP 553: `breakpoint()` embutida

Python 3.7 inclui a nova função embutida **`breakpoint()`** como uma maneira fácil e consistente de entrar no depurador Python.

`breakpoint()` embutida chama **`sys.breakpointhook()`**. Por padrão, o último importa **`pdb`** e então chama `pdb.set_trace()`, mas ao ligar `sys.breakpointhook()` à função de sua escolha, `breakpoint()` pode entrar em qualquer depurador. Além disso, a variável de ambiente **`PYTHONBREAKPOINT`** pode ser configurada para o chamável de seu depurador de escolha. Defina `PYTHONBREAKPOINT=0` para desabilitar completamente o `breakpoint()` embutida.

Ver também

PEP 553 [https://peps.python.org/pep-0553/] – breakpoint() embutida

PEP escrita e implementada por Barry Warsaw

PEP 539: Nova API C para armazenamento local de thread

Enquanto Python fornece uma API C para suporte de armazenamento local de thread; a existente API da armazenamento local de thread (TLS) usou int para representar as chaves TLS em todas as plataformas. Isso geralmente não tem sido um problema para plataformas oficialmente suportadas, mas isso não é compatível com POSIX, nem portátil em qualquer sentido prático.

A **PEP 539** [https://peps.python.org/pep-0539/] muda isso fornecendo uma nova API de armazenamento específico de thread (TSS) para CPython que substitui o uso da API de TLS existente dentro do interpretador CPython, enquanto descontinua a API existente. A API de TSS usa um novo tipo `Py_tss_t` em vez de int para representar as chaves de TSS – um tipo não visível cuja definição pode depender da implementação TLS subjacente. Portanto, isso permitirá construir CPython em plataformas onde a chave nativa de TLS é definida de uma forma que não pode ser convertida com segurança para int.

Observe que em plataformas onde a chave TLS nativa é definida de uma forma que não pode ser convertida com segurança para int, todas as funções da API TLS existente serão no-op e retornarão falha imediatamente. Isso indica claramente que a API antiga não é suportada em plataformas onde não pode ser usada de forma confiável e que nenhum esforço será feito para adicionar tal suporte.

Ver também

PEP 539 [https://peps.python.org/pep-0539/] – Uma nova API C para armazenamento local de thread no CPython

PEP escrita por Erik M. Bray; implementação por Masayuki Yamamoto.

PEP 562: Personalização de acesso aos atributos do módulo

Python 3.7 allows defining `__getattr__()` on modules and will call it whenever a module attribute is otherwise not found. Defining `__dir__()` on modules is now also allowed.

Um exemplo típico de onde isso pode ser útil é a descontinuação do atributo do módulo e o carregamento preguiçoso ou *lazy*.

Ver também

PEP 562 **[https://peps.python.org/pep-0562/] – Módulo `__getattr__` e `__dir__`**
PEP escrita e implementada por Ivan Levkivskyi

PEP 564: Novas funções de tempo com resolução de nanossegundos

A resolução dos relógios em sistemas modernos pode exceder a precisão limitada de um número de ponto flutuante retornado pela função `time.time()` e suas variantes. Para evitar perda de precisão, **PEP 564** [https://peps.python.org/pep-0564/] adiciona seis novas variantes em "nanossegundos" das funções de temporizador existentes ao módulo `time`:

- `time.clock_gettime_ns()`
- `time.clock_settime_ns()`
- `time.monotonic_ns()`
- `time.perf_counter_ns()`
- `time.process_time_ns()`
- `time.time_ns()`

As novas funções retornam o número de nanossegundos como um valor inteiro.

Medições [https://peps.python.org/pep-0564/#annex-clocks-resolution-in-python] mostram que no Linux e no Windows a resolução de `time.time_ns()` é aproximadamente 3 vezes melhor do que `time.time()`.

Ver também

PEP 564 [https://peps.python.org/pep-0564/] – Adiciona novas funções de tempo com resolução de nanossegundos

PEP escrito e implementado por Victor Stinner

PEP 565: Mostra DeprecationWarning em `__main__`

O tratamento padrão de `DeprecationWarning` foi alterado de forma que esses avisos sejam mais uma vez mostrados por padrão, mas apenas quando o código que os aciona está sendo executado diretamente no módulo `__main__`. Como resultado, os desenvolvedores de scripts de arquivo único e aqueles que usam Python interativamente devem mais uma vez começar a ver avisos de descontinuação para as APIs que usam, mas os avisos de descontinuação acionados por aplicações importadas, biblioteca e módulos de estrutura continuarão ocultos por padrão.

Como resultado dessa mudança, a biblioteca padrão agora permite que os desenvolvedores escolham entre três comportamentos diferentes de aviso de descontinuação:

- `FutureWarning`: sempre exibida por padrão, recomendado para avisos que devem ser vistos pelos usuários finais da aplicação (por exemplo, para definições de configuração de aplicações descontinuadas).
- `DeprecationWarning`: exibida por padrão apenas em `__main__` e ao executar testes, recomendado para avisos que devem ser vistos por outros desenvolvedores Python onde uma atualização de versão pode resultar em mudança de comportamento ou um erro.
- `PendingDeprecationWarning`: exibida por padrão apenas ao executar testes, destinados a casos em que uma atualização de versão futura mudará a categoria de aviso para `DeprecationWarning` ou `FutureWarning`.

Anteriormente, ambas `DeprecationWarning` e `PendingDeprecationWarning` eram visíveis apenas durante a execução de testes, o que significava que os desenvolvedores principalmente escrevendo scripts de arquivo único ou usando Python interativamente poderiam ser surpreendidos por alterações nas APIs que usavam.

Ver também

PEP 565 [https://peps.python.org/pep-0565/] – Show DeprecationWarning in `__main__`

PEP escrita e implementada por Nick Coghlan

PEP 560: Suporte básico para módulo `typing` e tipos genéricos

Initially **PEP 484** [https://peps.python.org/pep-0484/] was designed in such way that it would not introduce *any* changes to the core CPython interpreter. Now type hints and the `typing` module are extensively used by the community, so this restriction is removed. The PEP introduces two special methods `__class_getitem__()` and `__mro_entries__()`, these methods are now used by most classes and special constructs in `typing`. As a result, the speed of various operations with types increased up to 7 times, the generic types can be used without metaclass conflicts, and several long standing bugs in `typing` module are fixed.

Ver também

PEP 560 [https://peps.python.org/pep-0560/] – Suporte básico para módulo typing e tipos genéricos

PEP escrita e implementada por Ivan Levkivskyi

PEP 552: Arquivos .pyc baseados em hash

O Python tradicionalmente verifica a atualização dos arquivos de cache em bytecode (ou seja, arquivos `.pyc`) comparando os metadados da fonte (registro de data e hora e tamanho da última modificação) com os metadados da fonte salvos no cabeçalho do arquivo de cache quando foi gerado. Embora eficaz, esse método de invalidação tem suas desvantagens. Quando os registros de data e hora do sistema de arquivos são muito grosseiros, o Python pode perder as atualizações de origem, levando à confusão do usuário. Além disso, ter um registro de data e hora no arquivo

de cache é problemático para reprodutibilidade de compilação [https://reproducible-builds.org/] e sistemas de compilação baseados em conteúdo.

PEP 552 [https://peps.python.org/pep-0552/] estende o formato pyc para permitir que o hash do arquivo de origem seja usado para invalidação em vez do registro de data e hora da fonte. Esses arquivos `.pyc` são chamados de "baseados em hash". Por padrão, Python ainda usa invalidação baseada em registro de data e hora e não gera arquivos `.pyc` baseados em hash em tempo de execução. Arquivos `.pyc` baseados em hash podem ser gerados com **`py_compile`** ou **`compileall`**.

Arquivos `.pyc` baseados em hash vêm em duas variantes: verificados e não verificados. Python valida arquivos `.pyc` baseados em hash verificados em relação aos arquivos fonte correspondentes em tempo de execução, mas não o faz para pycs baseados em hash não verificados. Arquivos `.pyc` baseados em hash não verificados são uma otimização de desempenho útil para ambientes onde um sistema externo ao Python (por exemplo, o sistema de construção) é responsável por manter os arquivos `.pyc` atualizados.

Veja a função Invalidação de bytecode em cache para mais informações.

Ver também

PEP 552 [https://peps.python.org/pep-0552/] – pycs determinísticos
PEP escrita e implementada por Benjamin Peterson

PEP 545: Traduções da documentação do Python

PEP 545 [https://peps.python.org/pep-0545/] descreve o processo de criação a manutenção das traduções da documentação do Python.

Três novas traduções foram adicionadas:

- Japonês: https://docs.python.org/ja/
- Francês: https://docs.python.org/fr/
- Coreano: https://docs.python.org/ko/

Ver também

PEP 545 [https://peps.python.org/pep-0545/] – Traduções da documentação do Python

PEP escrita e implementada por Julien Palard, Inada Naoki e Victor Stinner.

Modo de Desenvolvimento do Python (-X dev)

A nova opção de linha de comando `-X dev` ou a nova variável de ambiente `PYTHONDEVMODE` pode ser usada para habilitar Modo de Desenvolvimento do Python. Quando está no modo de desenvolvimento, o Python executa verificações adicionais em tempo de execução que são muito caras para serem habilitadas por padrão. Veja a documentação Modo de Desenvolvimento do Python para a descrição completa.

Outras mudanças na linguagem

- Uma expressão `await` e compreensões contendo uma cláusula `async for` eram ilegais nas expressões em literais de strings formatadas devido a um problema com a implementação. No Python 3.7, esta restrição foi retirada.
- Mais de 255 argumentos agora podem ser passados para uma função, e uma função agora pode ter mais de 255 parâmetros. (Contribuição de Serhiy Storchaka em bpo-12844 [https://bugs.python.org/issue?@action=redirect&bpo=12844] e bpo-18896 [https://bugs.python.org/issue?@action=redirect&bpo=18896].)
- `bytes.fromhex()` e `bytearray.fromhex()` agora ignoram todos os espaços em branco ASCII, não apenas espaços. (Contribuição de Robert Xiao em bpo-28927 [https://bugs.python.org/issue?@action=redirect&bpo=28927].)
- `str`, `bytes` e `bytearray` ganharam suporte para o novo método `isascii()`, que pode ser usado para testar se uma string ou bytes contêm apenas os caracteres ASCII. (Contribuição de INADA Naoki em bpo-32677 [https://bugs.python.org/issue?@action=redirect&bpo=32677].)

- `ImportError` agora exibe o nome do módulo e o caminho do módulo `__file__` quando `from ... import ...` falha. (Contribuição de Matthias Bussonnier em bpo-29546 [https://bugs.python.org/issue?@action=redirect&bpo=29546].)
- Importações circulares envolvendo importações absolutas com ligação de um submódulo a um nome agora são suportadas. (Contribuição de Serhiy Storchaka em bpo-30024 [https://bugs.python.org/issue?@action=redirect&bpo=30024].)
- `object.__format__(x, '')` agora é equivalente a `str(x)` em vez de `format(str(self), '')`. (Contribuição de Serhiy Storchaka em bpo-28974 [https://bugs.python.org/issue?@action=redirect&bpo=28974].)
- Para melhor prover suporte à criação dinâmica de rastreamentos de pilha, `types.TracebackType` agora pode ser instanciado a partir do código Python, e o atributo `tb_next` em tracebacks agora é gravável. (Contribuição de Nathaniel J. Smith em bpo-30579 [https://bugs.python.org/issue?@action=redirect&bpo=30579].)
- Ao usar a opção `-m`, `sys.path[0]` agora é expandido para o caminho completo do diretório inicial, em vez de ser deixado como o diretório vazio (o que permite importações do *atual* diretório de trabalho no momento em que ocorre uma importação) (Contribuição de Nick Coghlan em bpo-33053 [https://bugs.python.org/issue?@action=redirect&bpo=33053].)
- A nova opção `-X importtime` ou a variável de ambiente `PYTHONPROFILEIMPORTTIME` pode ser usada para mostrar o tempo de cada importação de módulo. (Contribuição de Inada Naoki em bpo-31415 [https://bugs.python.org/issue?@action=redirect&bpo=31415].)

Novos módulos

contextvars

O novo módulo `contextvars` e um conjunto de novas APIs C introduzem suporte para *variáveis de contexto*. Variáveis de contexto são conceitualmente semelhantes às variáveis locais de thread. Ao contrário do TLS, as variáveis de contexto oferecem suporte a código assíncrono corretamente.

Os módulos `asyncio` e `decimal` foram atualizados para usar e prover suporte a variáveis de contexto prontas para uso. Particularmente o contexto decimal ativo agora é armazenado em uma variável de contexto, o que permite que operações decimais funcionem com o contexto correto em código assíncrono.

Ver também

PEP 567 **[https://peps.python.org/pep-0567/]** **– Variáveis de contexto**
PEP escrita e implementada por Yury Selivanov

dataclasses

The new `@~dataclasses.dataclass` decorator provides a way to declare *data classes*. A data class describes its attributes using class variable annotations. Its constructor and other magic methods, such as `__repr__()`, `__eq__()`, and `__hash__()` are generated automatically.

Exemplo:

```
@dataclass
class Point:
    x: float
    y: float
    z: float = 0.0

p = Point(1.5, 2.5)
print(p)   # produces "Point(x=1.5, y=2.5, z=0.0)"
```

Ver também

PEP 557 **[https://peps.python.org/pep-0557/]** **– Classes de dados**
PEP escrita e implementada por Eric V. Smith

importlib.resources

The new `importlib.resources` module provides several new APIs and one new ABC for access to, opening, and reading *resources* inside packages. Resources are roughly similar to files inside packages, but they needn't be actual files on the physical file system. Module loaders can provide a `get_resource_reader()` function which returns a `importlib.abc.ResourceReader` instance to support this new API. Built-in file path loaders and zip file loaders both support this.

(Contribuição de Barry Warsaw e Brett Cannon em bpo-32248 [https://bugs.python.org/issue?@action=redirect&bpo=32248].)

Ver também

importlib_resources [https://importlib-resources.readthedocs.io/en/latest/] – um backport do PyPI para versões anteriores do Python.

Módulos melhorados

argparse

O novo método `ArgumentParser.parse_intermixed_args()` permite misturar opções e argumentos posicionais. (Contribuição de paul.j3 em bpo-14191 [https://bugs.python.org/issue?@action=redirect&bpo=14191].)

asyncio

O módulo `asyncio` recebeu muitos novos recursos, usabilidade e melhorias de desempenho. Mudanças notáveis incluem:

- A nova função provisória `asyncio.run()` pode ser usada para executar uma corrotina a partir de código síncrono criando e destruindo automaticamente o laço de eventos. (Contribuição de Yury Selivanov em bpo-32314 [https://bugs.python.org/issue?@action=redirect&bpo=32314].)

- asyncio ganhou suporte para `contextvars`. `loop.call_soon()`, `loop.call_soon_threadsafe()`, `loop.call_later()`, `loop.call_at()` e `Future.add_done_callback()` têm um novo parâmetro somente-nomeado opcional *context*. `Tasks` agora rastreia seu contexto automaticamente. Veja **PEP 567** [https://peps.python.org/pep-0567/] para mais detalhes. (Contribuição de Yury Selivanov em bpo-32436 [https://bugs.python.org/issue?@action=redirect&bpo=32436].)

- A nova função `asyncio.create_task()` foi adicionada como um atalho para `asyncio.get_event_loop().create_task()`. (Contribuição de Andrew Svetlov em bpo-32311 [https://bugs.python.org/issue?@action=redirect&bpo=32311].)

- O novo método `loop.start_tls()` pode ser usado para atualizar uma conexão existente para TLS. (Contribuição de Yury Selivanov em bpo-23749 [https://bugs.python.org/issue?@action=redirect&bpo=23749].)

- O novo método `loop.sock_recv_into()` permite a leitura de dados de um soquete diretamente em um buffer fornecido, tornando possível reduzir as cópias de dados. (Contribuição de Antoine Pitrou em bpo-31819 [https://bugs.python.org/issue?@action=redirect&bpo=31819].)

- A nova função `asyncio.current_task()` retorna a instância `Task` atualmente em execução, e a nova função `asyncio.all_tasks()` retorna um conjunto de todas as instâncias `Task` existentes em um determinado laço. Os métodos `Task.current_task()` e `Task.all_tasks()` foram descontinuados. (Contribuição de Andrew Svetlov em bpo-32250 [https://bugs.python.org/issue?@action=redirect&bpo=32250].)

- A nova classe *provisória* `BufferedProtocol` permite implementar protocolos de streaming com controle manual sobre o buffer de recebimento. (Contribuição de Yury Selivanov em bpo-32251 [https://bugs.python.org/issue?@action=redirect&bpo=32251].)

- A nova função `asyncio.get_running_loop()` retorna o laço atualmente em execução e levanta uma exceção `RuntimeError` se nenhum laço estiver em execução. Isto contrasta com

asyncio.get_event_loop(), que irá *criar* um novo laço de eventos se nenhum estiver em execução. (Contribuição de Yury Selivanov em bpo-32269 [https://bugs.python.org/issue?@action=redirect&bpo=32269].)

- O novo método de corrotina **StreamWriter.wait_closed()** permite esperar até que o gravador de fluxo seja fechado. O novo método **StreamWriter.is_closing()** pode ser usado para determinar se o gravador está fechando. (Contribuição de Andrew Svetlov em bpo-32391 [https://bugs.python.org/issue?@action=redirect&bpo=32391].)

- O novo método de corrotina **loop.sock_sendfile()** permite enviar arquivos usando **os.sendfile** quando possível. (Contribuição de Andrew Svetlov em bpo-32410 [https://bugs.python.org/issue?@action=redirect&bpo=32410].)

- Os novos métodos **Future.get_loop()** e `Task.get_loop()` retornam a instância do laço no qual uma tarefa ou um future foi criado. **Server.get_loop()** permite fazer o mesmo para objetos **asyncio.Server**. (Contribuição de Yury Selivanov em bpo-32415 [https://bugs.python.org/issue?@action=redirect&bpo=32415] e Srinivas Reddy Thatiparthy em bpo-32418 [https://bugs.python.org/issue?@action=redirect&bpo=32418].)

- Agora é possível controlar como as instâncias de **asyncio.Server** começam a servir. Anteriormente, o servidor começaria a servir imediatamente após ser criado. O novo argumento nomeado *start_serving* para **loop.create_server()** e **loop.create_unix_server()**, bem como **Server.start_serving()** e **Server.serve_forever()** podem ser usados para dissociar a instanciação e o serviço do servidor. O novo método **Server.is_serving()** retorna `True` se o servidor estiver servindo. Os objetos **`Server`** agora são gerenciadores de contexto assíncronos:

```
srv = await loop.create_server(...)

async with srv:
    # algum código

# Neste ponto, srv é fechado e não mais aceita novas conexõe
```

(Contribuição de Yury Selivanov em bpo-32662 [https://bugs.python.org/issue?@action=redirect&bpo=32662].)

- Objetos de retorno de chamada retornados por `loop.call_later()` ganharam o novo método `when()` que retorna um registro de data e hora de retorno de chamada agendado absoluto. (Contribuição de Andrew Svetlov em bpo-32741 [https://bugs.python.org/issue?@action=redirect&bpo=32741].)

- O método `loop.create_datagram_endpoint()` ganhou suporte para soquetes Unix. (Contribuição de Quentin Dawans em bpo-31245 [https://bugs.python.org/issue?@action=redirect&bpo=31245].)

- As funções `asyncio.open_connection()`, `asyncio.start_server()`, os métodos `loop.create_connection()`, `loop.create_server()`, `loop.create_accepted_socket()` e suas variantes de soquete UNIX correspondentes agora aceitam o argumento nomeado *ssl_handshake_timeout*. (Contribuição de Neil Aspinall em bpo-29970 [https://bugs.python.org/issue?@action=redirect&bpo=29970].)

- O novo método `Handle.cancelled()` retorna `True` se o retorno de chamada foi cancelado. (Contribuição de Marat Sharafutdinov em bpo-31943 [https://bugs.python.org/issue?@action=redirect&bpo=31943].)

- A fonte de asyncio foi convertida para usar a sintaxe `async`/`await`. (Contribuição de Andrew Svetlov em bpo-32193 [https://bugs.python.org/issue?@action=redirect&bpo=32193].)

- O novo método `ReadTransport.is_reading()` pode ser usado para determinar o estado de leitura do transporte. Além disso, chamadas para `ReadTransport.resume_reading()` e `ReadTransport.pause_reading()` agora são idempotentes. (Contribuição de Yury Selivanov em bpo-32356 [https://bugs.python.org/issue?@action=redirect&bpo=32356].)

- Loop methods which accept socket paths now support passing path-like objects. (Contributed by Yury Selivanov in bpo-32066 [https://bugs.python.org/issue?@action=redirect&bpo=32066].)

- In **`asyncio`** TCP sockets on Linux are now created with `TCP_NODELAY` flag set by default. (Contributed by Yury Selivanov and Victor Stinner in bpo-27456 [https://bugs.python.org/issue?@action=redirect&bpo=27456].)

- Exceptions occurring in cancelled tasks are no longer logged. (Contributed by Yury Selivanov in bpo-30508 [https://bugs.python.org/issue?@action=redirect&bpo=30508].)

- New `WindowsSelectorEventLoopPolicy` and `WindowsProactorEventLoopPolicy` classes. (Contributed by Yury Selivanov in bpo-33792 [https://bugs.python.org/issue?@action=redirect&bpo=33792].)

Several `asyncio` APIs have been deprecated.

binascii

The **`b2a_uu()`** function now accepts an optional *backtick* keyword argument. When it's true, zeros are represented by `'`'` instead of spaces. (Contributed by Xiang Zhang in bpo-30103 [https://bugs.python.org/issue?@action=redirect&bpo=30103].)

calendar

The **`HTMLCalendar`** class has new class attributes which ease the customization of CSS classes in the produced HTML calendar. (Contributed by Oz Tiram in bpo-30095 [https://bugs.python.org/issue?@action=redirect&bpo=30095].)

collections

`collections.namedtuple()` now supports default values. (Contributed by Raymond Hettinger in bpo-32320 [https://bugs.python.org/issue?@action=redirect&bpo=32320].)

compileall

`compileall.compile_dir()` learned the new *invalidation_mode* parameter, which can be used to enable hash-based .pyc invalidation. The invalidation mode can also be specified on the command line using the new `--invalidation-mode` argument. (Contributed by Benjamin Peterson in bpo-31650 [https://bugs.python.org/issue?@action=redirect&bpo=31650].)

concurrent.futures

`ProcessPoolExecutor` and `ThreadPoolExecutor` now support the new *initializer* and *initargs* constructor arguments. (Contributed by Antoine Pitrou in bpo-21423 [https://bugs.python.org/issue?@action=redirect&bpo=21423].)

The `ProcessPoolExecutor` can now take the multiprocessing context via the new *mp_context* argument. (Contributed by Thomas Moreau in bpo-31540 [https://bugs.python.org/issue?@action=redirect&bpo=31540].)

contextlib

The new `nullcontext()` is a simpler and faster no-op context manager than `ExitStack`. (Contributed by Jesse-Bakker in bpo-10049 [https://bugs.python.org/issue?@action=redirect&bpo=10049].)

The new `@~contextlib.asynccontextmanager`, `AbstractAsyncContextManager`, and `AsyncExitStack` have been added to complement their synchronous counterparts. (Contributed by Jelle Zijlstra in bpo-29679 [https://bugs.python.org/issue?@action=redirect&bpo=29679] and bpo-30241 [https://bugs.python.org/issue?@action=redirect&bpo=30241], and by Alexander Mohr and Ilya Kulakov in bpo-29302 [https://bugs.python.org/issue?@action=redirect&bpo=29302].)

cProfile

The `cProfile` command line now accepts `-m module_name` as an alternative to script path. (Contributed by Sanyam Khurana in bpo-21862 [https://bugs.python.org/issue?@action=redirect&bpo=21862].)

crypt

The `crypt` module now supports the Blowfish hashing method. (Contributed by Serhiy Storchaka in bpo-31664 [https://bugs.python.org/issue?@action=redirect&bpo=31664].)

The `mksalt()` function now allows specifying the number of rounds for hashing. (Contributed by Serhiy Storchaka in bpo-31702 [https://bugs.python.org/issue?@action=redirect&bpo=31702].)

datetime

The new `datetime.fromisoformat()` method constructs a `datetime` object from a string in one of the formats output by `datetime.isoformat()`. (Contributed by Paul Ganssle in bpo-15873 [https://bugs.python.org/issue?@action=redirect&bpo=15873].)

The `tzinfo` class now supports sub-minute offsets. (Contributed by Alexander Belopolsky in bpo-5288 [https://bugs.python.org/issue?@action=redirect&bpo=5288].)

dbm

`dbm.dumb` now supports reading read-only files and no longer writes the index file when it is not changed.

decimal

The `decimal` module now uses context variables to store the decimal context. (Contributed by Yury Selivanov in bpo-32630 [https://bugs.python.org/issue?@action=redirect&bpo=32630].)

dis

The `dis()` function is now able to disassemble nested code objects (the code of comprehensions, generator expressions and nested functions, and the code used for building nested classes). The maximum depth of

disassembly recursion is controlled by the new *depth* parameter. (Contributed by Serhiy Storchaka in bpo-11822 [https://bugs.python.org/issue?@action=redirect&bpo=11822].)

distutils

`README.rst` is now included in the list of distutils standard READMEs and therefore included in source distributions. (Contributed by Ryan Gonzalez in bpo-11913 [https://bugs.python.org/issue?@action=redirect&bpo=11913].)

enum

The **`Enum`** learned the new `_ignore_` class property, which allows listing the names of properties which should not become enum members. (Contributed by Ethan Furman in bpo-31801 [https://bugs.python.org/issue?@action=redirect&bpo=31801].)

In Python 3.8, attempting to check for non-Enum objects in **`Enum`** classes will raise a **`TypeError`** (e.g. `1 in Color`); similarly, attempting to check for non-Flag objects in a **`Flag`** member will raise **`TypeError`** (e.g. `1 in Perm.RW`); currently, both operations return **`False`** instead and are deprecated. (Contributed by Ethan Furman in bpo-33217 [https://bugs.python.org/issue?@action=redirect&bpo=33217].)

functools

`@functools.singledispatch` now supports registering implementations using type annotations. (Contributed by Łukasz Langa in bpo-32227 [https://bugs.python.org/issue?@action=redirect&bpo=32227].)

gc

The new **`gc.freeze()`** function allows freezing all objects tracked by the garbage collector and excluding them from future collections. This can be used before a POSIX `fork()` call to make the GC copy-on-write friendly or to speed up collection. The new **`gc.unfreeze()`** functions reverses this operation. Additionally, **`gc.get_freeze_count()`** can be used to obtain the

number of frozen objects. (Contributed by Li Zekun in bpo-31558 [https://bugs.python.org/issue?@action=redirect&bpo=31558].)

hmac

The **`hmac`** module now has an optimized one-shot **`digest()`** function, which is up to three times faster than **`HMAC()`**. (Contributed by Christian Heimes in bpo-32433 [https://bugs.python.org/issue?@action=redirect&bpo=32433].)

http.client

`HTTPConnection` and **`HTTPSConnection`** now support the new *blocksize* argument for improved upload throughput. (Contributed by Nir Soffer in bpo-31945 [https://bugs.python.org/issue?@action=redirect&bpo=31945].)

http.server

`SimpleHTTPRequestHandler` now supports the HTTP `If-Modified-Since` header. The server returns the 304 response status if the target file was not modified after the time specified in the header. (Contributed by Pierre Quentel in bpo-29654 [https://bugs.python.org/issue?@action=redirect&bpo=29654].)

`SimpleHTTPRequestHandler` accepts the new *directory* argument, in addition to the new `--directory` command line argument. With this parameter, the server serves the specified directory, by default it uses the current working directory. (Contributed by Stéphane Wirtel and Julien Palard in bpo-28707 [https://bugs.python.org/issue?@action=redirect&bpo=28707].)

The new **`ThreadingHTTPServer`** class uses threads to handle requests using **`ThreadingMixIn`**. It is used when `http.server` is run with `-m`. (Contributed by Julien Palard in bpo-31639 [https://bugs.python.org/issue?@action=redirect&bpo=31639].)

idlelib e IDLE

Multiple fixes for autocompletion. (Contributed by Louie Lu in bpo-15786 [https://bugs.python.org/issue?@action=redirect&bpo=15786].)

Module Browser (on the File menu, formerly called Class Browser), now displays nested functions and classes in addition to top-level functions and classes. (Contributed by Guilherme Polo, Cheryl Sabella, and Terry Jan Reedy in bpo-1612262 [https://bugs.python.org/issue?@action=redirect&bpo=1612262].)

The Settings dialog (Options, Configure IDLE) has been partly rewritten to improve both appearance and function. (Contributed by Cheryl Sabella and Terry Jan Reedy in multiple issues.)

The font sample now includes a selection of non-Latin characters so that users can better see the effect of selecting a particular font. (Contributed by Terry Jan Reedy in bpo-13802 [https://bugs.python.org/issue?@action=redirect&bpo=13802].) The sample can be edited to include other characters. (Contributed by Serhiy Storchaka in bpo-31860 [https://bugs.python.org/issue?@action=redirect&bpo=31860].)

The IDLE features formerly implemented as extensions have been reimplemented as normal features. Their settings have been moved from the Extensions tab to other dialog tabs. (Contributed by Charles Wohlganger and Terry Jan Reedy in bpo-27099 [https://bugs.python.org/issue?@action=redirect&bpo=27099].)

Editor code context option revised. Box displays all context lines up to maxlines. Clicking on a context line jumps the editor to that line. Context colors for custom themes is added to Highlights tab of Settings dialog. (Contributed by Cheryl Sabella and Terry Jan Reedy in bpo-33642 [https://bugs.python.org/issue?@action=redirect&bpo=33642], bpo-33768 [https://bugs.python.org/issue?@action=redirect&bpo=33768], and bpo-33679 [https://bugs.python.org/issue?@action=redirect&bpo=33679].)

On Windows, a new API call tells Windows that tk scales for DPI. On Windows 8.1+ or 10, with DPI compatibility properties of the Python binary unchanged, and a monitor resolution greater than 96 DPI, this should make text and lines sharper. It should otherwise have no effect. (Contributed by Terry Jan Reedy in bpo-33656 [https://bugs.python.org/issue?@action=redirect&bpo=33656].)

New in 3.7.1:

A saída em N linhas (50 por padrão) é compactada até um limite inferior. N pode ser alterado na seção PyShell da página General da caixa de diálogo Settings. Menos linhas, mas possivelmente extra longas, podem ser espremidas clicando com o botão direito do mouse na saída. A saída compactada pode ser expandida no local clicando duas vezes no botão ou na área de transferência ou em uma janela separada clicando com o botão direito do mouse. (Contribuição de Tal Einat em bpo-1529353 [https://bugs.python.org/issue?@action=redirect&bpo=1529353].)

The changes above have been backported to 3.6 maintenance releases.

NEW in 3.7.4:

Add “Run Customized” to the Run menu to run a module with customized settings. Any command line arguments entered are added to sys.argv. They re-appear in the box for the next customized run. One can also suppress the normal Shell main module restart. (Contributed by Cheryl Sabella, Terry Jan Reedy, and others in bpo-5680 [https://bugs.python.org/issue?@action=redirect&bpo=5680] and bpo-37627 [https://bugs.python.org/issue?@action=redirect&bpo=37627].)

New in 3.7.5:

Add optional line numbers for IDLE editor windows. Windows open without line numbers unless set otherwise in the General tab of the configuration dialog. Line numbers for an existing window are shown and hidden in the Options menu. (Contributed by Tal Einat and Saimadhav Heblikar in bpo-17535 [https://bugs.python.org/issue?@action=redirect&bpo=17535].)

importlib

The **`importlib.abc.ResourceReader`** ABC was introduced to support the loading of resources from packages. See also importlib.resources. (Contributed by Barry Warsaw, Brett Cannon in bpo-32248 [https://bugs.python.org/issue?@action=redirect&bpo=32248].)

`importlib.reload()` now raises **`ModuleNotFoundError`** if the module lacks a spec. (Contributed by Garvit Khatri in bpo-29851 [https://bugs.python.org/issue?

@action=redirect&bpo=29851].)

`importlib.util.find_spec()` now raises `ModuleNotFoundError` instead of `AttributeError` if the specified parent module is not a package (i.e. lacks a `__path__` attribute). (Contributed by Milan Oberkirch in bpo-30436 [https://bugs.python.org/issue?@action=redirect&bpo=30436].)

The new `importlib.util.source_hash()` can be used to compute the hash of the passed source. A hash-based .pyc file embeds the value returned by this function.

io

The new `TextIOWrapper.reconfigure()` method can be used to reconfigure the text stream with the new settings. (Contributed by Antoine Pitrou in bpo-30526 [https://bugs.python.org/issue?@action=redirect&bpo=30526] and INADA Naoki in bpo-15216 [https://bugs.python.org/issue?@action=redirect&bpo=15216].)

ipaddress

The new `subnet_of()` and `supernet_of()` methods of `ipaddress.IPv6Network` and `ipaddress.IPv4Network` can be used for network containment tests. (Contributed by Michel Albert and Cheryl Sabella in bpo-20825 [https://bugs.python.org/issue?@action=redirect&bpo=20825].)

itertools

`itertools.islice()` now accepts `integer-like objects` as start, stop, and slice arguments. (Contributed by Will Roberts in bpo-30537 [https://bugs.python.org/issue?@action=redirect&bpo=30537].)

locale

The new *monetary* argument to `locale.format_string()` can be used to make the conversion use monetary thousands separators and grouping

strings. (Contributed by Garvit in bpo-10379 [https://bugs.python.org/issue?@action=redirect&bpo=10379].)

The `locale.getpreferredencoding()` function now always returns `'UTF-8'` on Android or when in the forced UTF-8 mode.

logging

`Logger` instances can now be pickled. (Contributed by Vinay Sajip in bpo-30520 [https://bugs.python.org/issue?@action=redirect&bpo=30520].)

The new `StreamHandler.setStream()` method can be used to replace the logger stream after handler creation. (Contributed by Vinay Sajip in bpo-30522 [https://bugs.python.org/issue?@action=redirect&bpo=30522].)

It is now possible to specify keyword arguments to handler constructors in configuration passed to `logging.config.fileConfig()`. (Contributed by Preston Landers in bpo-31080 [https://bugs.python.org/issue?@action=redirect&bpo=31080].)

math

The new `math.remainder()` function implements the IEEE 754-style remainder operation. (Contributed by Mark Dickinson in bpo-29962 [https://bugs.python.org/issue?@action=redirect&bpo=29962].)

mimetypes

The MIME type of .bmp has been changed from `'image/x-ms-bmp'` to `'image/bmp'`. (Contributed by Nitish Chandra in bpo-22589 [https://bugs.python.org/issue?@action=redirect&bpo=22589].)

msilib

The new `Database.Close()` method can be used to close the MSI database. (Contributed by Berker Peksag in bpo-20486 [https://bugs.python.org/issue?@action=redirect&bpo=20486].)

multiprocessing

The new `Process.close()` method explicitly closes the process object and releases all resources associated with it. `ValueError` is raised if the underlying process is still running. (Contributed by Antoine Pitrou in bpo-30596 [https://bugs.python.org/issue?@action=redirect&bpo=30596].)

The new `Process.kill()` method can be used to terminate the process using the `SIGKILL` signal on Unix. (Contributed by Vitor Pereira in bpo-30794 [https://bugs.python.org/issue?@action=redirect&bpo=30794].)

Non-daemonic threads created by `Process` are now joined on process exit. (Contributed by Antoine Pitrou in bpo-18966 [https://bugs.python.org/issue?@action=redirect&bpo=18966].)

os

`os.fwalk()` now accepts the *path* argument as `bytes`. (Contributed by Serhiy Storchaka in bpo-28682 [https://bugs.python.org/issue?@action=redirect&bpo=28682].)

`os.scandir()` gained support for file descriptors. (Contributed by Serhiy Storchaka in bpo-25996 [https://bugs.python.org/issue?@action=redirect&bpo=25996].)

The new `register_at_fork()` function allows registering Python callbacks to be executed at process fork. (Contributed by Antoine Pitrou in bpo-16500 [https://bugs.python.org/issue?@action=redirect&bpo=16500].)

Added `os.preadv()` (combine the functionality of `os.readv()` and `os.pread()`) and `os.pwritev()` functions (combine the functionality of `os.writev()` and `os.pwrite()`). (Contributed by Pablo Galindo in bpo-31368 [https://bugs.python.org/issue?@action=redirect&bpo=31368].)

The mode argument of `os.makedirs()` no longer affects the file permission bits of newly created intermediate-level directories. (Contributed by Serhiy Storchaka in bpo-19930 [https://bugs.python.org/issue?@action=redirect&bpo=19930].)

os.dup2() now returns the new file descriptor. Previously, `None` was always returned. (Contributed by Benjamin Peterson in bpo-32441 [https://bugs.python.org/issue?@action=redirect&bpo=32441].)

The structure returned by **os.stat()** now contains the **st_fstype** attribute on Solaris and its derivatives. (Contributed by Jesús Cea Avión in bpo-32659 [https://bugs.python.org/issue?@action=redirect&bpo=32659].)

pathlib

The new **Path.is_mount()** method is now available on POSIX systems and can be used to determine whether a path is a mount point. (Contributed by Cooper Ry Lees in bpo-30897 [https://bugs.python.org/issue?@action=redirect&bpo=30897].)

pdb

pdb.set_trace() now takes an optional *header* keyword-only argument. If given, it is printed to the console just before debugging begins. (Contributed by Barry Warsaw in bpo-31389 [https://bugs.python.org/issue?@action=redirect&bpo=31389].)

pdb command line now accepts `-m module_name` as an alternative to script file. (Contributed by Mario Corchero in bpo-32206 [https://bugs.python.org/issue?@action=redirect&bpo=32206].)

py_compile

py_compile.compile() – and by extension, **compileall** – now respects the **SOURCE_DATE_EPOCH** environment variable by unconditionally creating `.pyc` files for hash-based validation. This allows for guaranteeing reproducible builds [https://reproducible-builds.org/] of `.pyc` files when they are created eagerly. (Contributed by Bernhard M. Wiedemann in bpo-29708 [https://bugs.python.org/issue?@action=redirect&bpo=29708].)

pydoc

The pydoc server can now bind to an arbitrary hostname specified by the new `-n` command-line argument. (Contributed by Feanil Patel in bpo-31128 [https://bugs.python.org/issue?@action=redirect&bpo=31128].)

queue

The new `SimpleQueue` class is an unbounded FIFO queue. (Contributed by Antoine Pitrou in bpo-14976 [https://bugs.python.org/issue?@action=redirect&bpo=14976].)

re

The flags `re.ASCII`, `re.LOCALE` and `re.UNICODE` can be set within the scope of a group. (Contributed by Serhiy Storchaka in bpo-31690 [https://bugs.python.org/issue?@action=redirect&bpo=31690].)

`re.split()` now supports splitting on a pattern like `r'\b'`, `'^$'` or `(?=-)` that matches an empty string. (Contributed by Serhiy Storchaka in bpo-25054 [https://bugs.python.org/issue?@action=redirect&bpo=25054].)

Regular expressions compiled with the `re.LOCALE` flag no longer depend on the locale at compile time. Locale settings are applied only when the compiled regular expression is used. (Contributed by Serhiy Storchaka in bpo-30215 [https://bugs.python.org/issue?@action=redirect&bpo=30215].)

`FutureWarning` is now emitted if a regular expression contains character set constructs that will change semantically in the future, such as nested sets and set operations. (Contributed by Serhiy Storchaka in bpo-30349 [https://bugs.python.org/issue?@action=redirect&bpo=30349].)

Compiled regular expression and match objects can now be copied using `copy.copy()` and `copy.deepcopy()`. (Contributed by Serhiy Storchaka in bpo-10076 [https://bugs.python.org/issue?@action=redirect&bpo=10076].)

signal

The new *warn_on_full_buffer* argument to the **signal.set_wakeup_fd()** function makes it possible to specify whether Python prints a warning on stderr when the wakeup buffer overflows. (Contributed by Nathaniel J. Smith in bpo-30050 [https://bugs.python.org/issue?@action=redirect&bpo=30050].)

socket

The new **socket.getblocking()** method returns `True` if the socket is in blocking mode and `False` otherwise. (Contributed by Yury Selivanov in bpo-32373 [https://bugs.python.org/issue?@action=redirect&bpo=32373].)

The new **socket.close()** function closes the passed socket file descriptor. This function should be used instead of **os.close()** for better compatibility across platforms. (Contributed by Christian Heimes in bpo-32454 [https://bugs.python.org/issue?@action=redirect&bpo=32454].)

The **socket** module now exposes the socket.TCP_CONGESTION (Linux 2.6.13), socket.TCP_USER_TIMEOUT (Linux 2.6.37), and socket.TCP_NOTSENT_LOWAT (Linux 3.12) constants. (Contributed by Omar Sandoval in bpo-26273 [https://bugs.python.org/issue?@action=redirect&bpo=26273] and Nathaniel J. Smith in bpo-29728 [https://bugs.python.org/issue?@action=redirect&bpo=29728].)

Support for **socket.AF_VSOCK** sockets has been added to allow communication between virtual machines and their hosts. (Contributed by Cathy Avery in bpo-27584 [https://bugs.python.org/issue?@action=redirect&bpo=27584].)

Sockets now auto-detect family, type and protocol from file descriptor by default. (Contributed by Christian Heimes in bpo-28134 [https://bugs.python.org/issue?@action=redirect&bpo=28134].)

socketserver

socketserver.ThreadingMixIn.server_close now waits until all non-daemon threads complete. **socketserver.ForkingMixIn.server_close** now waits until all child processes complete.

Add a new `socketserver.ForkingMixIn.block_on_close` class attribute to `socketserver.ForkingMixIn` and `socketserver.ThreadingMixIn` classes. Set the class attribute to `False` to get the pre-3.7 behaviour.

sqlite3

`sqlite3.Connection` now exposes the `backup()` method when the underlying SQLite library is at version 3.6.11 or higher. (Contributed by Lele Gaifax in bpo-27645 [https://bugs.python.org/issue?@action=redirect&bpo=27645].)

The *database* argument of `sqlite3.connect()` now accepts any path-like object, instead of just a string. (Contributed by Anders Lorentsen in bpo-31843 [https://bugs.python.org/issue?@action=redirect&bpo=31843].)

ssl

The `ssl` module now uses OpenSSL's builtin API instead of `match_hostname()` to check a host name or an IP address. Values are validated during TLS handshake. Any certificate validation error including failing the host name check now raises `SSLCertVerificationError` and aborts the handshake with a proper TLS Alert message. The new exception contains additional information. Host name validation can be customized with `SSLContext.hostname_checks_common_name`. (Contributed by Christian Heimes in bpo-31399 [https://bugs.python.org/issue?@action=redirect&bpo=31399].)

Nota

The improved host name check requires a *libssl* implementation compatible with OpenSSL 1.0.2 or 1.1. Consequently, OpenSSL 0.9.8 and 1.0.1 are no longer supported (see Platform Support Removals for more details). The ssl module is mostly compatible with LibreSSL 2.7.2 and newer.

The `ssl` module no longer sends IP addresses in SNI TLS extension. (Contributed by Christian Heimes in bpo-32185 [https://bugs.python.org/issue?

@action=redirect&bpo=32185].)

`match_hostname()` no longer supports partial wildcards like `www*.example.org`. (Contributed by Mandeep Singh in bpo-23033 [https://bugs.python.org/issue?@action=redirect&bpo=23033] and Christian Heimes in bpo-31399 [https://bugs.python.org/issue?@action=redirect&bpo=31399].)

The default cipher suite selection of the `ssl` module now uses a blacklist approach rather than a hard-coded whitelist. Python no longer re-enables ciphers that have been blocked by OpenSSL security updates. Default cipher suite selection can be configured at compile time. (Contributed by Christian Heimes in bpo-31429 [https://bugs.python.org/issue?@action=redirect&bpo=31429].)

Validation of server certificates containing internationalized domain names (IDNs) is now supported. As part of this change, the **`SSLSocket.server_hostname`** attribute now stores the expected hostname in A-label form (`"xn--pythn-mua.org"`), rather than the U-label form (`"pythön.org"`). (Contributed by Nathaniel J. Smith and Christian Heimes in bpo-28414 [https://bugs.python.org/issue?@action=redirect&bpo=28414].)

The `ssl` module has preliminary and experimental support for TLS 1.3 and OpenSSL 1.1.1. At the time of Python 3.7.0 release, OpenSSL 1.1.1 is still under development and TLS 1.3 hasn't been finalized yet. The TLS 1.3 handshake and protocol behaves slightly differently than TLS 1.2 and earlier, see TLS 1.3. (Contributed by Christian Heimes in bpo-32947 [https://bugs.python.org/issue?@action=redirect&bpo=32947], bpo-20995 [https://bugs.python.org/issue?@action=redirect&bpo=20995], bpo-29136 [https://bugs.python.org/issue?@action=redirect&bpo=29136], bpo-30622 [https://bugs.python.org/issue?@action=redirect&bpo=30622] and bpo-33618 [https://bugs.python.org/issue?@action=redirect&bpo=33618])

`SSLSocket` and **`SSLObject`** no longer have a public constructor. Direct instantiation was never a documented and supported feature. Instances must be created with **`SSLContext`** methods **`wrap_socket()`** and **`wrap_bio()`**. (Contributed by Christian Heimes in bpo-32951 [https://bugs.python.org/issue?@action=redirect&bpo=32951])

OpenSSL 1.1 APIs for setting the minimum and maximum TLS protocol version are available as `SSLContext.minimum_version` and `SSLContext.maximum_version`. Supported protocols are indicated by several new flags, such as `HAS_TLSv1_1`. (Contributed by Christian Heimes in bpo-32609 [https://bugs.python.org/issue?@action=redirect&bpo=32609].)

Added `ssl.SSLContext.post_handshake_auth` to enable and `ssl.SSLSocket.verify_client_post_handshake()` to initiate TLS 1.3 post-handshake authentication. (Contributed by Christian Heimes in gh-78851 [https://github.com/python/cpython/issues/78851].)

string

`string.Template` now lets you to optionally modify the regular expression pattern for braced placeholders and non-braced placeholders separately. (Contributed by Barry Warsaw in bpo-1198569 [https://bugs.python.org/issue?@action=redirect&bpo=1198569].)

subprocess

The `subprocess.run()` function accepts the new *capture_output* keyword argument. When true, stdout and stderr will be captured. This is equivalent to passing `subprocess.PIPE` as *stdout* and *stderr* arguments. (Contributed by Bo Bayles in bpo-32102 [https://bugs.python.org/issue?@action=redirect&bpo=32102].)

The `subprocess.run` function and the `subprocess.Popen` constructor now accept the *text* keyword argument as an alias to *universal_newlines*. (Contributed by Andrew Clegg in bpo-31756 [https://bugs.python.org/issue?@action=redirect&bpo=31756].)

On Windows the default for *close_fds* was changed from `False` to `True` when redirecting the standard handles. It's now possible to set *close_fds* to true when redirecting the standard handles. See `subprocess.Popen`. This means that *close_fds* now defaults to `True` on all supported platforms. (Contributed by Segev Finer in bpo-19764 [https://bugs.python.org/issue?@action=redirect&bpo=19764].)

The subprocess module is now more graceful when handling `KeyboardInterrupt` during `subprocess.call()`, `subprocess.run()`, or in a `Popen` context manager. It now waits a short amount of time for the child to exit, before continuing the handling of the `KeyboardInterrupt` exception. (Contributed by Gregory P. Smith in bpo-25942 [https://bugs.python.org/issue?@action=redirect&bpo=25942].)

sys

The new `sys.breakpointhook()` hook function is called by the built-in `breakpoint()`. (Contributed by Barry Warsaw in bpo-31353 [https://bugs.python.org/issue?@action=redirect&bpo=31353].)

On Android, the new `sys.getandroidapilevel()` returns the build-time Android API version. (Contributed by Victor Stinner in bpo-28740 [https://bugs.python.org/issue?@action=redirect&bpo=28740].)

The new `sys.get_coroutine_origin_tracking_depth()` function returns the current coroutine origin tracking depth, as set by the new `sys.set_coroutine_origin_tracking_depth()`. `asyncio` has been converted to use this new API instead of the deprecated `sys.set_coroutine_wrapper()`. (Contributed by Nathaniel J. Smith in bpo-32591 [https://bugs.python.org/issue?@action=redirect&bpo=32591].)

time

PEP 564 [https://peps.python.org/pep-0564/] adds six new functions with nanosecond resolution to the `time` module:

- `time.clock_gettime_ns()`
- `time.clock_settime_ns()`
- `time.monotonic_ns()`
- `time.perf_counter_ns()`
- `time.process_time_ns()`
- `time.time_ns()`

New clock identifiers have been added:

- `time.CLOCK_BOOTTIME` (Linux): Identical to `time.CLOCK_MONOTONIC`, except it also includes any time that the system is suspended.
- `time.CLOCK_PROF` (FreeBSD, NetBSD and OpenBSD): High-resolution per-process CPU timer.
- `time.CLOCK_UPTIME` (FreeBSD, OpenBSD): Time whose absolute value is the time the system has been running and not suspended, providing accurate uptime measurement.

The new `time.thread_time()` and `time.thread_time_ns()` functions can be used to get per-thread CPU time measurements. (Contributed by Antoine Pitrou in bpo-32025 [https://bugs.python.org/issue?@action=redirect&bpo=32025].)

The new `time.pthread_getcpuclockid()` function returns the clock ID of the thread-specific CPU-time clock.

tkinter

The new `tkinter.ttk.Spinbox` class is now available. (Contributed by Alan Moore in bpo-32585 [https://bugs.python.org/issue?@action=redirect&bpo=32585].)

tracemalloc

`tracemalloc.Traceback` behaves more like regular tracebacks, sorting the frames from oldest to most recent. `Traceback.format()` now accepts negative *limit*, truncating the result to the `abs(limit)` oldest frames. To get the old behaviour, use the new *most_recent_first* argument to `Traceback.format()`. (Contributed by Jesse Bakker in bpo-32121 [https://bugs.python.org/issue?@action=redirect&bpo=32121].)

types

The new `WrapperDescriptorType`, `MethodWrapperType`, `MethodDescriptorType`, and `ClassMethodDescriptorType` classes are now available. (Contributed by Manuel Krebber and Guido van Rossum in bpo-29377 [https://bugs.python.org/issue?@action=redirect&bpo=29377], and Serhiy Storchaka in bpo-32265 [https://bugs.python.org/issue?@action=redirect&bpo=32265].)

The new `types.resolve_bases()` function resolves MRO entries dynamically as specified by **PEP 560** [https://peps.python.org/pep-0560/]. (Contributed by Ivan Levkivskyi in bpo-32717 [https://bugs.python.org/issue?@action=redirect&bpo=32717].)

unicodedata

The internal `unicodedata` database has been upgraded to use Unicode 11 [https://www.unicode.org/versions/Unicode11.0.0/]. (Contributed by Benjamin Peterson.)

unittest

The new `-k` command-line option allows filtering tests by a name substring or a Unix shell-like pattern. For example, `python -m unittest -k foo` runs `foo_tests.SomeTest.test_something`, `bar_tests.SomeTest.test_foo`, but not `bar_tests.FooTest.test_something`. (Contributed by Jonas Haag in bpo-32071 [https://bugs.python.org/issue?@action=redirect&bpo=32071].)

unittest.mock

The `sentinel` attributes now preserve their identity when they are `copied` or `pickled`. (Contributed by Serhiy Storchaka in bpo-20804 [https://bugs.python.org/issue?@action=redirect&bpo=20804].)

The new `seal()` function allows sealing `Mock` instances, which will disallow further creation of attribute mocks. The seal is applied recursively to all attributes that are themselves mocks. (Contributed by Mario Corchero in bpo-30541 [https://bugs.python.org/issue?@action=redirect&bpo=30541].)

urllib.parse

`urllib.parse.quote()` has been updated from **RFC 2396** [https://datatracker.ietf.org/doc/html/rfc2396.html] to **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], adding ~ to the set of characters that are never quoted by default. (Contributed by Christian Theune and

Ratnadeep Debnath in bpo-16285 [https://bugs.python.org/issue?@action=redirect&bpo=16285].)

uu

The **`uu.encode()`** function now accepts an optional *backtick* keyword argument. When it's true, zeros are represented by `'``'` instead of spaces. (Contributed by Xiang Zhang in bpo-30103 [https://bugs.python.org/issue?@action=redirect&bpo=30103].)

uuid

The new **`UUID.is_safe`** attribute relays information from the platform about whether generated UUIDs are generated with a multiprocessing-safe method. (Contributed by Barry Warsaw in bpo-22807 [https://bugs.python.org/issue?@action=redirect&bpo=22807].)

`uuid.getnode()` now prefers universally administered MAC addresses over locally administered MAC addresses. This makes a better guarantee for global uniqueness of UUIDs returned from **`uuid.uuid1()`**. If only locally administered MAC addresses are available, the first such one found is returned. (Contributed by Barry Warsaw in bpo-32107 [https://bugs.python.org/issue?@action=redirect&bpo=32107].)

avisos

The initialization of the default warnings filters has changed as follows:

- warnings enabled via command line options (including those for **`-b`** and the new CPython-specific **`-X`** `dev` option) are always passed to the warnings machinery via the **`sys.warnoptions`** attribute.
- warnings filters enabled via the command line or the environment now have the following order of precedence:
  - the `BytesWarning` filter for **`-b`** (or `-bb`)
  - any filters specified with the **`-W`** option
  - any filters specified with the **`PYTHONWARNINGS`** environment variable

    - any other CPython specific filters (e.g. the `default` filter added for the new `-X dev` mode)
    - any implicit filters defined directly by the warnings machinery
- in CPython debug builds, all warnings are now displayed by default (the implicit filter list is empty)

(Contributed by Nick Coghlan and Victor Stinner in bpo-20361 [https://bugs.python.org/issue?@action=redirect&bpo=20361], bpo-32043 [https://bugs.python.org/issue?@action=redirect&bpo=32043], and bpo-32230 [https://bugs.python.org/issue?@action=redirect&bpo=32230].)

Deprecation warnings are once again shown by default in single-file scripts and at the interactive prompt. See PEP 565: Mostra DeprecationWarning em __main__ for details. (Contributed by Nick Coghlan in bpo-31975 [https://bugs.python.org/issue?@action=redirect&bpo=31975].)

xml

As mitigation against DTD and external entity retrieval, the **`xml.dom.minidom`** and **`xml.sax`** modules no longer process external entities by default. (Contributed by Christian Heimes in gh-61441 [https://github.com/python/cpython/issues/61441].)

xml.etree

ElementPath predicates in the **`find()`** methods can now compare text of the current node with `[. = "text"]`, not only text in children. Predicates also allow adding spaces for better readability. (Contributed by Stefan Behnel in bpo-31648 [https://bugs.python.org/issue?@action=redirect&bpo=31648].)

xmlrpc.server

`SimpleXMLRPCDispatcher.register_function()` can now be used as a decorator. (Contributed by Xiang Zhang in bpo-7769 [https://bugs.python.org/issue?@action=redirect&bpo=7769].)

zipapp

Function `create_archive()` now accepts an optional *filter* argument to allow the user to select which files should be included in the archive. (Contributed by Irmen de Jong in bpo-31072 [https://bugs.python.org/issue?@action=redirect&bpo=31072].)

Function `create_archive()` now accepts an optional *compressed* argument to generate a compressed archive. A command line option `--compress` has also been added to support compression. (Contributed by Zhiming Wang in bpo-31638 [https://bugs.python.org/issue?@action=redirect&bpo=31638].)

zipfile

`ZipFile` now accepts the new *compresslevel* parameter to control the compression level. (Contributed by Bo Bayles in bpo-21417 [https://bugs.python.org/issue?@action=redirect&bpo=21417].)

Subdirectories in archives created by `ZipFile` are now stored in alphabetical order. (Contributed by Bernhard M. Wiedemann in bpo-30693 [https://bugs.python.org/issue?@action=redirect&bpo=30693].)

Alterações na API C

A new API for thread-local storage has been implemented. See PEP 539: Nova API C para armazenamento local de thread for an overview and Thread-specific storage API for a complete reference. (Contributed by Masayuki Yamamoto in bpo-25658 [https://bugs.python.org/issue?@action=redirect&bpo=25658].)

The new context variables functionality exposes a number of new C APIs.

The new `PyImport_GetModule()` function returns the previously imported module with the given name. (Contributed by Eric Snow in bpo-28411 [https://bugs.python.org/issue?@action=redirect&bpo=28411].)

The new `Py_RETURN_RICHCOMPARE` macro eases writing rich comparison functions. (Contributed by Petr Victorin in bpo-23699 [https://bugs.python.org/issue?@action=redirect&bpo=23699].)

The new `Py_UNREACHABLE` macro can be used to mark unreachable code paths. (Contributed by Barry Warsaw in bpo-31338 [https://bugs.python.org/issue?@action=redirect&bpo=31338].)

The `tracemalloc` now exposes a C API through the new `PyTraceMalloc_Track()` and `PyTraceMalloc_Untrack()` functions. (Contributed by Victor Stinner in bpo-30054 [https://bugs.python.org/issue?@action=redirect&bpo=30054].)

The new import__find_load__start and import__find_load__done static markers can be used to trace module imports. (Contributed by Christian Heimes in bpo-31574 [https://bugs.python.org/issue?@action=redirect&bpo=31574].)

The fields **`name`** and **`doc`** of structures `PyMemberDef`, `PyGetSetDef`, `PyStructSequence_Field`, `PyStructSequence_Desc`, and **`wrapperbase`** are now of type `const char *` rather of `char *`. (Contributed by Serhiy Storchaka in bpo-28761 [https://bugs.python.org/issue?@action=redirect&bpo=28761].)

The result of `PyUnicode_AsUTF8AndSize()` and `PyUnicode_AsUTF8()` is now of type `const char *` rather of `char *`. (Contributed by Serhiy Storchaka in bpo-28769 [https://bugs.python.org/issue?@action=redirect&bpo=28769].)

The result of `PyMapping_Keys()`, `PyMapping_Values()` and `PyMapping_Items()` is now always a list, rather than a list or a tuple. (Contributed by Oren Milman in bpo-28280 [https://bugs.python.org/issue?@action=redirect&bpo=28280].)

Added functions `PySlice_Unpack()` and `PySlice_AdjustIndices()`. (Contributed by Serhiy Storchaka in bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867].)

`PyOS_AfterFork()` is deprecated in favour of the new functions `PyOS_BeforeFork()`, `PyOS_AfterFork_Parent()` and `PyOS_AfterFork_Child()`. (Contributed by Antoine Pitrou in bpo-16500 [https://bugs.python.org/issue?@action=redirect&bpo=16500].)

The `PyExc_RecursionErrorInst` singleton that was part of the public API has been removed as its members being never cleared may cause a segfault

during finalization of the interpreter. Contributed by Xavier de Gaye in bpo-22898 [https://bugs.python.org/issue?@action=redirect&bpo=22898] and bpo-30697 [https://bugs.python.org/issue?@action=redirect&bpo=30697].

Added C API support for timezones with timezone constructors `PyTimeZone_FromOffset()` and `PyTimeZone_FromOffsetAndName()`, and access to the UTC singleton with `PyDateTime_TimeZone_UTC`. Contributed by Paul Ganssle in bpo-10381 [https://bugs.python.org/issue?@action=redirect&bpo=10381].

The type of results of `PyThread_start_new_thread()` and `PyThread_get_thread_ident()`, and the *id* parameter of `PyThreadState_SetAsyncExc()` changed from long to unsigned long. (Contributed by Serhiy Storchaka in bpo-6532 [https://bugs.python.org/issue?@action=redirect&bpo=6532].)

`PyUnicode_AsWideCharString()` now raises a `ValueError` if the second argument is `NULL` and the wchar_t* string contains null characters. (Contributed by Serhiy Storchaka in bpo-30708 [https://bugs.python.org/issue?@action=redirect&bpo=30708].)

Changes to the startup sequence and the management of dynamic memory allocators mean that the long documented requirement to call `Py_Initialize()` before calling most C API functions is now relied on more heavily, and failing to abide by it may lead to segfaults in embedding applications. See the Porting to Python 3.7 section in this document and the Before Python initialization section in the C API documentation for more details.

The new `PyInterpreterState_GetID()` returns the unique ID for a given interpreter. (Contributed by Eric Snow in bpo-29102 [https://bugs.python.org/issue?@action=redirect&bpo=29102].)

`Py_DecodeLocale()`, `Py_EncodeLocale()` now use the UTF-8 encoding when the UTF-8 mode is enabled. (Contributed by Victor Stinner in bpo-29240 [https://bugs.python.org/issue?@action=redirect&bpo=29240].)

`PyUnicode_DecodeLocaleAndSize()` and `PyUnicode_EncodeLocale()` now use the current locale encoding for `surrogateescape` error handler. (Contributed by Victor Stinner in bpo-29240 [https://bugs.python.org/issue?@action=redirect&bpo=29240].)

The *start* and *end* parameters of `PyUnicode_FindChar()` are now adjusted to behave like string slices. (Contributed by Xiang Zhang in bpo-28822 [https://bugs.python.org/issue?@action=redirect&bpo=28822].)

Mudanças na construção

Support for building `--without-threads` has been removed. The `threading` module is now always available. (Contributed by Antoine Pitrou in bpo-31370 [https://bugs.python.org/issue?@action=redirect&bpo=31370].).

A full copy of libffi is no longer bundled for use when building the `_ctypes` module on non-OSX UNIX platforms. An installed copy of libffi is now required when building `_ctypes` on such platforms. (Contributed by Zachary Ware in bpo-27979 [https://bugs.python.org/issue?@action=redirect&bpo=27979].)

The Windows build process no longer depends on Subversion to pull in external sources, a Python script is used to download zipfiles from GitHub instead. If Python 3.6 is not found on the system (via `py -3.6`), NuGet is used to download a copy of 32-bit Python for this purpose. (Contributed by Zachary Ware in bpo-30450 [https://bugs.python.org/issue?@action=redirect&bpo=30450].)

The `ssl` module requires OpenSSL 1.0.2 or 1.1 compatible libssl. OpenSSL 1.0.1 has reached end of lifetime on 2016-12-31 and is no longer supported. LibreSSL is temporarily not supported as well. LibreSSL releases up to version 2.6.4 are missing required OpenSSL 1.0.2 APIs.

Otimizações

The overhead of calling many methods of various standard library classes implemented in C has been significantly reduced by porting more code to

use the `METH_FASTCALL` convention. (Contributed by Victor Stinner in bpo-29300 [https://bugs.python.org/issue?@action=redirect&bpo=29300], bpo-29507 [https://bugs.python.org/issue?@action=redirect&bpo=29507], bpo-29452 [https://bugs.python.org/issue?@action=redirect&bpo=29452], and bpo-29286 [https://bugs.python.org/issue?@action=redirect&bpo=29286].)

Various optimizations have reduced Python startup time by 10% on Linux and up to 30% on macOS. (Contributed by Victor Stinner, INADA Naoki in bpo-29585 [https://bugs.python.org/issue?@action=redirect&bpo=29585], and Ivan Levkivskyi in bpo-31333 [https://bugs.python.org/issue?@action=redirect&bpo=31333].)

Method calls are now up to 20% faster due to the bytecode changes which avoid creating bound method instances. (Contributed by Yury Selivanov and INADA Naoki in bpo-26110 [https://bugs.python.org/issue?@action=redirect&bpo=26110].)

The **`asyncio`** module received a number of notable optimizations for commonly used functions:

- The **`asyncio.get_event_loop()`** function has been reimplemented in C to make it up to 15 times faster. (Contributed by Yury Selivanov in bpo-32296 [https://bugs.python.org/issue?@action=redirect&bpo=32296].)
- **`asyncio.Future`** callback management has been optimized. (Contributed by Yury Selivanov in bpo-32348 [https://bugs.python.org/issue?@action=redirect&bpo=32348].)
- **`asyncio.gather()`** is now up to 15% faster. (Contributed by Yury Selivanov in bpo-32355 [https://bugs.python.org/issue?@action=redirect&bpo=32355].)
- **`asyncio.sleep()`** is now up to 2 times faster when the *delay* argument is zero or negative. (Contributed by Andrew Svetlov in bpo-32351 [https://bugs.python.org/issue?@action=redirect&bpo=32351].)
- The performance overhead of asyncio debug mode has been reduced. (Contributed by Antoine Pitrou in bpo-31970 [https://bugs.python.org/issue?@action=redirect&bpo=31970].)

As a result of PEP 560 work, the import time of **`typing`** has been reduced by a factor of 7, and many typing operations are now faster. (Contributed by

Ivan Levkivskyi in bpo-32226 [https://bugs.python.org/issue?@action=redirect&bpo=32226].)

`sorted()` and `list.sort()` have been optimized for common cases to be up to 40-75% faster. (Contributed by Elliot Gorokhovsky in bpo-28685 [https://bugs.python.org/issue?@action=redirect&bpo=28685].)

`dict.copy()` is now up to 5.5 times faster. (Contributed by Yury Selivanov in bpo-31179 [https://bugs.python.org/issue?@action=redirect&bpo=31179].)

`hasattr()` and `getattr()` are now about 4 times faster when *name* is not found and *obj* does not override `object.__getattr__()` or `object.__getattribute__()`. (Contributed by INADA Naoki in bpo-32544 [https://bugs.python.org/issue?@action=redirect&bpo=32544].)

Searching for certain Unicode characters (like Ukrainian capital "Є") in a string was up to 25 times slower than searching for other characters. It is now only 3 times slower in the worst case. (Contributed by Serhiy Storchaka in bpo-24821 [https://bugs.python.org/issue?@action=redirect&bpo=24821].)

The `collections.namedtuple()` factory has been reimplemented to make the creation of named tuples 4 to 6 times faster. (Contributed by Jelle Zijlstra with further improvements by INADA Naoki, Serhiy Storchaka, and Raymond Hettinger in bpo-28638 [https://bugs.python.org/issue?@action=redirect&bpo=28638].)

`datetime.date.fromordinal()` and `datetime.date.fromtimestamp()` are now up to 30% faster in the common case. (Contributed by Paul Ganssle in bpo-32403 [https://bugs.python.org/issue?@action=redirect&bpo=32403].)

The `os.fwalk()` function is now up to 2 times faster thanks to the use of `os.scandir()`. (Contributed by Serhiy Storchaka in bpo-25996 [https://bugs.python.org/issue?@action=redirect&bpo=25996].)

The speed of the `shutil.rmtree()` function has been improved by 20–40% thanks to the use of the `os.scandir()` function. (Contributed by Serhiy Storchaka in bpo-28564 [https://bugs.python.org/issue?@action=redirect&bpo=28564].)

Optimized case-insensitive matching and searching of **regular expressions**. Searching some patterns can now be up to 20 times faster. (Contributed by Serhiy Storchaka in bpo-30285 [https://bugs.python.org/issue?@action=redirect&bpo=30285].)

re.compile() now converts `flags` parameter to int object if it is `RegexFlag`. It is now as fast as Python 3.5, and faster than Python 3.6 by about 10% depending on the pattern. (Contributed by INADA Naoki in bpo-31671 [https://bugs.python.org/issue?@action=redirect&bpo=31671].)

The **modify()** methods of classes **selectors.EpollSelector**, **selectors.PollSelector** and **selectors.DevpollSelector** may be around 10% faster under heavy loads. (Contributed by Giampaolo Rodola' in bpo-30014 [https://bugs.python.org/issue?@action=redirect&bpo=30014])

Constant folding has been moved from the peephole optimizer to the new AST optimizer, which is able perform optimizations more consistently. (Contributed by Eugene Toder and INADA Naoki in bpo-29469 [https://bugs.python.org/issue?@action=redirect&bpo=29469] and bpo-11549 [https://bugs.python.org/issue?@action=redirect&bpo=11549].)

Most functions and methods in **abc** have been rewritten in C. This makes creation of abstract base classes, and calling **isinstance()** and **issubclass()** on them 1.5x faster. This also reduces Python start-up time by up to 10%. (Contributed by Ivan Levkivskyi and INADA Naoki in bpo-31333 [https://bugs.python.org/issue?@action=redirect&bpo=31333])

Significant speed improvements to alternate constructors for **datetime.date** and **datetime.datetime** by using fast-path constructors when not constructing subclasses. (Contributed by Paul Ganssle in bpo-32403 [https://bugs.python.org/issue?@action=redirect&bpo=32403])

The speed of comparison of **array.array** instances has been improved considerably in certain cases. It is now from 10x to 70x faster when comparing arrays holding values of the same integer type. (Contributed by Adrian Wielgosik in bpo-24700 [https://bugs.python.org/issue?@action=redirect&bpo=24700].)

The **math.erf()** and **math.erfc()** functions now use the (faster) C library implementation on most platforms. (Contributed by Serhiy Storchaka in bpo-26121 [https://bugs.python.org/issue?@action=redirect&bpo=26121].)

Outras mudanças na implementação do CPython

- Trace hooks may now opt out of receiving the `line` and opt into receiving the `opcode` events from the interpreter by setting the corresponding new **f_trace_lines** and **f_trace_opcodes** attributes on the frame being traced. (Contributed by Nick Coghlan in bpo-31344 [https://bugs.python.org/issue?@action=redirect&bpo=31344].)
- Fixed some consistency problems with namespace package module attributes. Namespace module objects now have an `__file__` that is set to `None` (previously unset), and their `__spec__.origin` is also set to `None` (previously the string `"namespace"`). See bpo-32305 [https://bugs.python.org/issue?@action=redirect&bpo=32305]. Also, the namespace module object's `__spec__.loader` is set to the same value as `__loader__` (previously, the former was set to `None`). See bpo-32303 [https://bugs.python.org/issue?@action=redirect&bpo=32303].
- The **locals()** dictionary now displays in the lexical order that variables were defined. Previously, the order was undefined. (Contributed by Raymond Hettinger in bpo-32690 [https://bugs.python.org/issue?@action=redirect&bpo=32690].)
- The `distutils upload` command no longer tries to change CR end-of-line characters to CRLF. This fixes a corruption issue with sdists that ended with a byte equivalent to CR. (Contributed by Bo Bayles in bpo-32304 [https://bugs.python.org/issue?@action=redirect&bpo=32304].)

Comportamento do Python descontinuado

Yield expressions (both `yield` and `yield from` clauses) are now deprecated in comprehensions and generator expressions (aside from the iterable expression in the leftmost **for** clause). This ensures that comprehensions always immediately return a container of the appropriate type (rather than potentially returning a generator iterator object), while generator expressions won't attempt to interleave their implicit output with the output

from any explicit yield expressions. In Python 3.7, such expressions emit **`DeprecationWarning`** when compiled, in Python 3.8 this will be a **`SyntaxError`**. (Contributed by Serhiy Storchaka in bpo-10544 [https://bugs.python.org/issue?@action=redirect&bpo=10544].)

Returning a subclass of **`complex`** from **`object.__complex__()`** is deprecated and will be an error in future Python versions. This makes `__complex__()` consistent with **`object.__int__()`** and **`object.__float__()`**. (Contributed by Serhiy Storchaka in bpo-28894 [https://bugs.python.org/issue?@action=redirect&bpo=28894].)

Deprecated Python modules, functions and methods

aifc

`aifc.openfp()` has been deprecated and will be removed in Python 3.9. Use **`aifc.open()`** instead. (Contributed by Brian Curtin in bpo-31985 [https://bugs.python.org/issue?@action=redirect&bpo=31985].)

asyncio

Support for directly `await`-ing instances of **`asyncio.Lock`** and other asyncio synchronization primitives has been deprecated. An asynchronous context manager must be used in order to acquire and release the synchronization resource. (Contributed by Andrew Svetlov in bpo-32253 [https://bugs.python.org/issue?@action=redirect&bpo=32253].)

The **`asyncio.Task.current_task()`** and **`asyncio.Task.all_tasks()`** methods have been deprecated. (Contributed by Andrew Svetlov in bpo-32250 [https://bugs.python.org/issue?@action=redirect&bpo=32250].)

collections

In Python 3.8, the abstract base classes in **`collections.abc`** will no longer be exposed in the regular **`collections`** module. This will help create a

clearer distinction between the concrete classes and the abstract base classes. (Contributed by Serhiy Storchaka in bpo-25988 [https://bugs.python.org/issue?@action=redirect&bpo=25988].)

dbm

`dbm.dumb` now supports reading read-only files and no longer writes the index file when it is not changed. A deprecation warning is now emitted if the index file is missing and recreated in the `'r'` and `'w'` modes (this will be an error in future Python releases). (Contributed by Serhiy Storchaka in bpo-28847 [https://bugs.python.org/issue?@action=redirect&bpo=28847].)

enum

In Python 3.8, attempting to check for non-Enum objects in **`Enum`** classes will raise a **`TypeError`** (e.g. `1 in Color`); similarly, attempting to check for non-Flag objects in a **`Flag`** member will raise **`TypeError`** (e.g. `1 in Perm.RW`); currently, both operations return **`False`** instead. (Contributed by Ethan Furman in bpo-33217 [https://bugs.python.org/issue?@action=redirect&bpo=33217].)

gettext

Using non-integer value for selecting a plural form in **`gettext`** is now deprecated. It never correctly worked. (Contributed by Serhiy Storchaka in bpo-28692 [https://bugs.python.org/issue?@action=redirect&bpo=28692].)

importlib

Methods **`MetaPathFinder.find_module()`** (replaced by **`MetaPathFinder.find_spec()`**) and **`PathEntryFinder.find_loader()`** (replaced by **`PathEntryFinder.find_spec()`**) both deprecated in Python 3.4 now emit **`DeprecationWarning`**. (Contributed by Matthias Bussonnier in bpo-29576 [https://bugs.python.org/issue?@action=redirect&bpo=29576].)

The **`importlib.abc.ResourceLoader`** ABC has been deprecated in favour of **`importlib.abc.ResourceReader`**.

locale

`locale.format()` has been deprecated, use **`locale.format_string()`** instead. (Contributed by Garvit in bpo-10379 [https://bugs.python.org/issue?@action=redirect&bpo=10379].)

macpath

The **`macpath`** is now deprecated and will be removed in Python 3.8. (Contributed by Chi Hsuan Yen in bpo-9850 [https://bugs.python.org/issue?@action=redirect&bpo=9850].)

threading

`dummy_threading` and **`_dummy_thread`** have been deprecated. It is no longer possible to build Python with threading disabled. Use **`threading`** instead. (Contributed by Antoine Pitrou in bpo-31370 [https://bugs.python.org/issue?@action=redirect&bpo=31370].)

socket

The silent argument value truncation in **`socket.htons()`** and **`socket.ntohs()`** has been deprecated. In future versions of Python, if the passed argument is larger than 16 bits, an exception will be raised. (Contributed by Oren Milman in bpo-28332 [https://bugs.python.org/issue?@action=redirect&bpo=28332].)

ssl

`ssl.wrap_socket()` is deprecated. Use **`ssl.SSLContext.wrap_socket()`** instead. (Contributed by Christian Heimes in bpo-28124 [https://bugs.python.org/issue?@action=redirect&bpo=28124].)

sunau

`sunau.openfp()` has been deprecated and will be removed in Python 3.9. Use **`sunau.open()`** instead. (Contributed by Brian Curtin in bpo-31985 [https://bugs.python.org/issue?@action=redirect&bpo=31985].)

sys

Deprecated **`sys.set_coroutine_wrapper()`** and **`sys.get_coroutine_wrapper()`**.

The undocumented `sys.callstats()` function has been deprecated and will be removed in a future Python version. (Contributed by Victor Stinner in bpo-28799 [https://bugs.python.org/issue?@action=redirect&bpo=28799].)

wave

`wave.openfp()` has been deprecated and will be removed in Python 3.9. Use **`wave.open()`** instead. (Contributed by Brian Curtin in bpo-31985 [https://bugs.python.org/issue?@action=redirect&bpo=31985].)

Deprecated functions and types of the C API

Function **`PySlice_GetIndicesEx()`** is deprecated and replaced with a macro if `Py_LIMITED_API` is not set or set to a value in the range between `0x03050400` and `0x03060000` (not inclusive), or is `0x03060100` or higher. (Contributed by Serhiy Storchaka in bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867].)

`PyOS_AfterFork()` has been deprecated. Use **`PyOS_BeforeFork()`**, **`PyOS_AfterFork_Parent()`** or **`PyOS_AfterFork_Child()`** instead. (Contributed by Antoine Pitrou in bpo-16500 [https://bugs.python.org/issue?@action=redirect&bpo=16500].)

Platform Support Removals

- FreeBSD 9 and older are no longer officially supported.

- For full Unicode support, including within extension modules, *nix platforms are now expected to provide at least one of `C.UTF-8` (full locale), `C.utf8` (full locale) or `UTF-8` (`LC_CTYPE`-only locale) as an alternative to the legacy `ASCII`-based `C` locale.

- OpenSSL 0.9.8 and 1.0.1 are no longer supported, which means building CPython 3.7 with SSL/TLS support on older platforms still using these versions requires custom build options that link to a more recent version of OpenSSL.

  Notably, this issue affects the Debian 8 (aka "jessie") and Ubuntu 14.04 (aka "Trusty") LTS Linux distributions, as they still use OpenSSL 1.0.1 by default.

  Debian 9 ("stretch") and Ubuntu 16.04 ("xenial"), as well as recent releases of other LTS Linux releases (e.g. RHEL/CentOS 7.5, SLES 12-SP3), use OpenSSL 1.0.2 or later, and remain supported in the default build configuration.

  CPython's own CI configuration file [https://github.com/python/cpython/blob/v3.7.13/.travis.yml] provides an example of using the SSL compatibility testing infrastructure [https://github.com/python/cpython/tree/3.15/Tools/ssl/multissltests.py] in CPython's test suite to build and link against OpenSSL 1.1.0 rather than an outdated system provided OpenSSL.

Remoção de APIs e recursos

The following features and APIs have been removed from Python 3.7:

- The `os.stat_float_times()` function has been removed. It was introduced in Python 2.3 for backward compatibility with Python 2.2, and was deprecated since Python 3.1.
- Unknown escapes consisting of `'\'` and an ASCII letter in replacement templates for `re.sub()` were deprecated in Python 3.5, and will now cause an error.

- Removed support of the *exclude* argument in **tarfile.TarFile.add()**. It was deprecated in Python 2.7 and 3.2. Use the *filter* argument instead.
- The **ntpath.splitunc()** function was deprecated in Python 3.1, and has now been removed. Use **splitdrive()** instead.
- **collections.namedtuple()** no longer supports the *verbose* parameter or `_source` attribute which showed the generated source code for the named tuple class. This was part of an optimization designed to speed-up class creation. (Contributed by Jelle Zijlstra with further improvements by INADA Naoki, Serhiy Storchaka, and Raymond Hettinger in bpo-28638 [https://bugs.python.org/issue?@action=redirect&bpo=28638].)
- Functions **bool()**, **float()**, **list()** and **tuple()** no longer take keyword arguments. The first argument of **int()** can now be passed only as positional argument.
- Removed previously deprecated in Python 2.4 classes `Plist`, `Dict` and `_InternalDict` in the **plistlib** module. Dict values in the result of functions **readPlist()** and **readPlistFromBytes()** are now normal dicts. You no longer can use attribute access to access items of these dictionaries.
- The `asyncio.windows_utils.socketpair()` function has been removed. Use the **socket.socketpair()** function instead, it is available on all platforms since Python 3.5. `asyncio.windows_utils.socketpair` was just an alias to `socket.socketpair` on Python 3.5 and newer.
- **asyncio** no longer exports the **selectors** and **_overlapped** modules as `asyncio.selectors` and `asyncio._overlapped`. Replace `from asyncio import selectors` with `import selectors`.
- Direct instantiation of **ssl.SSLSocket** and **ssl.SSLObject** objects is now prohibited. The constructors were never documented, tested, or designed as public constructors. Users were supposed to use **ssl.wrap_socket()** or **ssl.SSLContext**. (Contributed by Christian Heimes in bpo-32951 [https://bugs.python.org/issue?@action=redirect&bpo=32951].)
- The unused `distutils install_misc` command has been removed. (Contributed by Eric N. Vander Weele in bpo-29218 [https://bugs.python.org/issue?@action=redirect&bpo=29218].)

Module Removals

The `fpectl` module has been removed. It was never enabled by default, never worked correctly on x86-64, and it changed the Python ABI in ways that caused unexpected breakage of C extensions. (Contributed by Nathaniel J. Smith in bpo-29137 [https://bugs.python.org/issue?@action=redirect&bpo=29137].)

Windows-only Changes

The python launcher, (py.exe), can accept 32 & 64 bit specifiers **without** having to specify a minor version as well. So `py -3-32` and `py -3-64` become valid as well as `py -3.7-32`, also the *-m*-64 and *-m.n*-64 forms are now accepted to force 64 bit python even if 32 bit would have otherwise been used. If the specified version is not available py.exe will error exit. (Contributed by Steve Barnes in bpo-30291 [https://bugs.python.org/issue?@action=redirect&bpo=30291].)

The launcher can be run as `py -0` to produce a list of the installed pythons, *with default marked with an asterisk*. Running `py -0p` will include the paths. If py is run with a version specifier that cannot be matched it will also print the *short form* list of available specifiers. (Contributed by Steve Barnes in bpo-30362 [https://bugs.python.org/issue?@action=redirect&bpo=30362].)

Porting to Python 3.7

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Changes in Python Behavior

- **`async`** and **`await`** names are now reserved keywords. Code using these names as identifiers will now raise a **`SyntaxError`**. (Contributed by Jelle Zijlstra in bpo-30406 [https://bugs.python.org/issue?@action=redirect&bpo=30406].)

- **PEP 479** [https://peps.python.org/pep-0479/] is enabled for all code in Python 3.7, meaning that `StopIteration` exceptions raised directly or indirectly in coroutines and generators are transformed into `RuntimeError` exceptions. (Contributed by Yury Selivanov in bpo-32670 [https://bugs.python.org/issue?@action=redirect&bpo=32670].)

- `object.__aiter__()` methods can no longer be declared as asynchronous. (Contributed by Yury Selivanov in bpo-31709 [https://bugs.python.org/issue?@action=redirect&bpo=31709].)

- Due to an oversight, earlier Python versions erroneously accepted the following syntax:

  ```
  f(1 for x in [1],)

  class C(1 for x in [1]):
      pass
  ```

  Python 3.7 now correctly raises a `SyntaxError`, as a generator expression always needs to be directly inside a set of parentheses and cannot have a comma on either side, and the duplication of the parentheses can be omitted only on calls. (Contributed by Serhiy Storchaka in bpo-32012 [https://bugs.python.org/issue?@action=redirect&bpo=32012] and bpo-32023 [https://bugs.python.org/issue?@action=redirect&bpo=32023].)

- When using the `-m` switch, the initial working directory is now added to `sys.path`, rather than an empty string (which dynamically denoted the current working directory at the time of each import). Any programs that are checking for the empty string, or otherwise relying on the previous behaviour, will need to be updated accordingly (e.g. by also checking for `os.getcwd()` or `os.path.dirname(__main__.__file__)`, depending on why the code was checking for the empty string in the first place).

Alterações na API Python

- `socketserver.ThreadingMixIn.server_close` now waits until all non-daemon threads complete. Set the new

`socketserver.ThreadingMixIn.block_on_close` class attribute to `False` to get the pre-3.7 behaviour. (Contributed by Victor Stinner in bpo-31233 [https://bugs.python.org/issue?@action=redirect&bpo=31233] and bpo-33540 [https://bugs.python.org/issue?@action=redirect&bpo=33540].)

- `socketserver.ForkingMixIn.server_close` now waits until all child processes complete. Set the new `socketserver.ForkingMixIn.block_on_close` class attribute to `False` to get the pre-3.7 behaviour. (Contributed by Victor Stinner in bpo-31151 [https://bugs.python.org/issue?@action=redirect&bpo=31151] and bpo-33540 [https://bugs.python.org/issue?@action=redirect&bpo=33540].)

- The `locale.localeconv()` function now temporarily sets the `LC_CTYPE` locale to the value of `LC_NUMERIC` in some cases. (Contributed by Victor Stinner in bpo-31900 [https://bugs.python.org/issue?@action=redirect&bpo=31900].)

- `pkgutil.walk_packages()` now raises a `ValueError` if *path* is a string. Previously an empty list was returned. (Contributed by Sanyam Khurana in bpo-24744 [https://bugs.python.org/issue?@action=redirect&bpo=24744].)

- A format string argument for `string.Formatter.format()` is now positional-only. Passing it as a keyword argument was deprecated in Python 3.5. (Contributed by Serhiy Storchaka in bpo-29193 [https://bugs.python.org/issue?@action=redirect&bpo=29193].)

- Attributes `key`, `value` and `coded_value` of class `http.cookies.Morsel` are now read-only. Assigning to them was deprecated in Python 3.5. Use the `set()` method for setting them. (Contributed by Serhiy Storchaka in bpo-29192 [https://bugs.python.org/issue?@action=redirect&bpo=29192].)

- The *mode* argument of `os.makedirs()` no longer affects the file permission bits of newly created intermediate-level directories. To set their file permission bits you can set the umask before invoking `makedirs()`. (Contributed by Serhiy Storchaka in bpo-19930 [https://bugs.python.org/issue?@action=redirect&bpo=19930].)

- The **struct.Struct.format** type is now **str** instead of **bytes**. (Contributed by Victor Stinner in bpo-21071 [https://bugs.python.org/issue?@action=redirect&bpo=21071].)

- **cgi.parse_multipart()** now accepts the *encoding* and *errors* arguments and returns the same results as **FieldStorage**: for non-file fields, the value associated to a key is a list of strings, not bytes. (Contributed by Pierre Quentel in bpo-29979 [https://bugs.python.org/issue?@action=redirect&bpo=29979].)

- Due to internal changes in **socket**, calling **socket.fromshare()** on a socket created by **socket.share** in older Python versions is not supported.

- `repr` for **BaseException** has changed to not include the trailing comma. Most exceptions are affected by this change. (Contributed by Serhiy Storchaka in bpo-30399 [https://bugs.python.org/issue?@action=redirect&bpo=30399].)

- `repr` for **datetime.timedelta** has changed to include the keyword arguments in the output. (Contributed by Utkarsh Upadhyay in bpo-30302 [https://bugs.python.org/issue?@action=redirect&bpo=30302].)

- Because **shutil.rmtree()** is now implemented using the **os.scandir()** function, the user specified handler *onerror* is now called with the first argument `os.scandir` instead of `os.listdir` when listing the directory is failed.

- Support for nested sets and set operations in regular expressions as in Unicode Technical Standard #18 [https://unicode.org/reports/tr18/] might be added in the future. This would change the syntax. To facilitate this future change a **FutureWarning** will be raised in ambiguous cases for the time being. That include sets starting with a literal `'['` or containing literal character sequences `'--'`, `'&&'`, `'~~'`, and `'||'`. To avoid a warning, escape them with a backslash. (Contributed by Serhiy Storchaka in bpo-30349 [https://bugs.python.org/issue?@action=redirect&bpo=30349].)

- The result of splitting a string on a **regular expression** that could match an empty string has been changed. For example splitting on `r'\s*'` will now split not only on whitespaces as it did previously, but also on empty strings before all non-whitespace characters and just before the end of the string. The previous behavior can be restored by changing the pattern to `r'\s+'`. A **FutureWarning** was emitted for such patterns since Python 3.5.

  For patterns that match both empty and non-empty strings, the result of searching for all matches may also be changed in other cases. For example in the string `'a\n\n'`, the pattern `r'(?m)^\s*?$'` will not only match empty strings at positions 2 and 3, but also the string `'\n'` at positions 2–3. To match only blank lines, the pattern should be rewritten as `r'(?m)^[^\S\n]*$'`.

  re.sub() now replaces empty matches adjacent to a previous non-empty match. For example `re.sub('x*', '-', 'abxd')` returns now `'-a-b--d-'` instead of `'-a-b-d-'` (the first minus between 'b' and 'd' replaces 'x', and the second minus replaces an empty string between 'x' and 'd').

  (Contributed by Serhiy Storchaka in bpo-25054 [https://bugs.python.org/issue?@action=redirect&bpo=25054] and bpo-32308 [https://bugs.python.org/issue?@action=redirect&bpo=32308].)

- Change **re.escape()** to only escape regex special characters instead of escaping all characters other than ASCII letters, numbers, and `'_'`. (Contributed by Serhiy Storchaka in bpo-29995 [https://bugs.python.org/issue?@action=redirect&bpo=29995].)

- **tracemalloc.Traceback** frames are now sorted from oldest to most recent to be more consistent with **traceback**. (Contributed by Jesse Bakker in bpo-32121 [https://bugs.python.org/issue?@action=redirect&bpo=32121].)

- On OSes that support **socket.SOCK_NONBLOCK** or **socket.SOCK_CLOEXEC** bit flags, the **socket.type** no longer has them applied. Therefore, checks like `if sock.type == socket.SOCK_STREAM` work as expected on all platforms. (Contributed

by Yury Selivanov in bpo-32331 [https://bugs.python.org/issue?@action=redirect&bpo=32331].)

- On Windows the default for the *close_fds* argument of `subprocess.Popen` was changed from `False` to `True` when redirecting the standard handles. If you previously depended on handles being inherited when using `subprocess.Popen` with standard io redirection, you will have to pass `close_fds=False` to preserve the previous behaviour, or use `STARTUPINFO.lpAttributeList`.

- `importlib.machinery.PathFinder.invalidate_caches()` – which implicitly affects `importlib.invalidate_caches()` – now deletes entries in `sys.path_importer_cache` which are set to `None`. (Contributed by Brett Cannon in bpo-33169 [https://bugs.python.org/issue?@action=redirect&bpo=33169].)

- In `asyncio`, `loop.sock_recv()`, `loop.sock_sendall()`, `loop.sock_accept()`, `loop.getaddrinfo()`, `loop.getnameinfo()` have been changed to be proper coroutine methods to match their documentation. Previously, these methods returned `asyncio.Future` instances. (Contributed by Yury Selivanov in bpo-32327 [https://bugs.python.org/issue?@action=redirect&bpo=32327].)

- `asyncio.Server.sockets` now returns a copy of the internal list of server sockets, instead of returning it directly. (Contributed by Yury Selivanov in bpo-32662 [https://bugs.python.org/issue?@action=redirect&bpo=32662].)

- `Struct.format` is now a `str` instance instead of a `bytes` instance. (Contributed by Victor Stinner in bpo-21071 [https://bugs.python.org/issue?@action=redirect&bpo=21071].)

- `argparse` subparsers can now be made mandatory by passing `required=True` to `ArgumentParser.add_subparsers()`. (Contributed by Anthony Sottile in bpo-26510 [https://bugs.python.org/issue?@action=redirect&bpo=26510].)

- `ast.literal_eval()` is now stricter. Addition and subtraction of arbitrary numbers are no longer allowed. (Contributed by Serhiy Storchaka in bpo-31778 [https://bugs.python.org/issue?@action=redirect&bpo=31778].)

- `Calendar.itermonthdates` will now consistently raise an exception when a date falls outside of the `0001-01-01` through `9999-12-31` range. To support applications that cannot tolerate such exceptions, the new `Calendar.itermonthdays3` and `Calendar.itermonthdays4` can be used. The new methods return tuples and are not restricted by the range supported by `datetime.date`. (Contributed by Alexander Belopolsky in bpo-28292 [https://bugs.python.org/issue?@action=redirect&bpo=28292].)

- `collections.ChainMap` now preserves the order of the underlying mappings. (Contributed by Raymond Hettinger in bpo-32792 [https://bugs.python.org/issue?@action=redirect&bpo=32792].)

- The `submit()` method of `concurrent.futures.ThreadPoolExecutor` and `concurrent.futures.ProcessPoolExecutor` now raises a `RuntimeError` if called during interpreter shutdown. (Contributed by Mark Nemec in bpo-33097 [https://bugs.python.org/issue?@action=redirect&bpo=33097].)

- The `configparser.ConfigParser` constructor now uses `read_dict()` to process the default values, making its behavior consistent with the rest of the parser. Non-string keys and values in the defaults dictionary are now being implicitly converted to strings. (Contributed by James Tocknell in bpo-23835 [https://bugs.python.org/issue?@action=redirect&bpo=23835].)

- Several undocumented internal imports were removed. One example is that `os.errno` is no longer available; use `import errno` directly instead. Note that such undocumented internal imports may be removed any time without notice, even in micro version releases.

Alterações na API C

The function **PySlice_GetIndicesEx()** is considered unsafe for resizable sequences. If the slice indices are not instances of **int**, but objects that implement the **__index__()** method, the sequence can be resized after passing its length to **PySlice_GetIndicesEx()**. This can lead to returning indices out of the length of the sequence. For avoiding possible problems use new functions **PySlice_Unpack()** and **PySlice_AdjustIndices()**. (Contributed by Serhiy Storchaka in bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867].)

Alterações de bytecode do CPython

There are two new opcodes: **LOAD_METHOD** and **CALL_METHOD**. (Contributed by Yury Selivanov and INADA Naoki in bpo-26110 [https://bugs.python.org/issue?@action=redirect&bpo=26110].)

The **STORE_ANNOTATION** opcode has been removed. (Contributed by Mark Shannon in bpo-32550 [https://bugs.python.org/issue?@action=redirect&bpo=32550].)

Windows-only Changes

The file used to override **sys.path** is now called `<python-executable>._pth` instead of `'sys.path'`. See Encontrando módulos for more information. (Contributed by Steve Dower in bpo-28137 [https://bugs.python.org/issue?@action=redirect&bpo=28137].)

Other CPython implementation changes

In preparation for potential future changes to the public CPython runtime initialization API (see **PEP 432** [https://peps.python.org/pep-0432/] for an initial, but somewhat outdated, draft), CPython's internal startup and configuration management logic has been significantly refactored. While these updates are intended to be entirely transparent to both embedding applications and users of the regular CPython CLI, they're being mentioned here as the refactoring changes the internal order of various operations during interpreter startup, and hence may uncover previously latent defects, either in embedding applications, or in CPython itself. (Initially contributed by Nick Coghlan and Eric Snow as part of bpo-22257 [https://bugs.python.org/issue?

@action=redirect&bpo=22257], and further updated by Nick, Eric, and Victor Stinner in a number of other issues). Some known details affected:

- **`PySys_AddWarnOptionUnicode()`** is not currently usable by embedding applications due to the requirement to create a Unicode object prior to calling `Py_Initialize`. Use **`PySys_AddWarnOption()`** instead.
- warnings filters added by an embedding application with **`PySys_AddWarnOption()`** should now more consistently take precedence over the default filters set by the interpreter

Due to changes in the way the default warnings filters are configured, setting **`Py_BytesWarningFlag`** to a value greater than one is no longer sufficient to both emit **`BytesWarning`** messages and have them converted to exceptions. Instead, the flag must be set (to cause the warnings to be emitted in the first place), and an explicit `error::BytesWarning` warnings filter added to convert them to exceptions.

Due to a change in the way docstrings are handled by the compiler, the implicit `return None` in a function body consisting solely of a docstring is now marked as occurring on the same line as the docstring, not on the function's header line.

The current exception state has been moved from the frame object to the co-routine. This simplified the interpreter and fixed a couple of obscure bugs caused by having swap exception state when entering or exiting a generator. (Contributed by Mark Shannon in bpo-25612 [https://bugs.python.org/issue?@action=redirect&bpo=25612].)

Notable changes in Python 3.7.1

Starting in 3.7.1, **`Py_Initialize()`** now consistently reads and respects all of the same environment settings as **`Py_Main()`** (in earlier Python versions, it respected an ill-defined subset of those environment variables, while in Python 3.7.0 it didn't read any of them due to bpo-34247 [https://bugs.python.org/issue?@action=redirect&bpo=34247]). If this behavior is unwanted, set **`Py_IgnoreEnvironmentFlag`** to 1 before calling **`Py_Initialize()`**.

In 3.7.1 the C API for Context Variables was updated to use **PyObject** pointers. See also bpo-34762 [https://bugs.python.org/issue?@action=redirect&bpo=34762].

In 3.7.1 the **tokenize** module now implicitly emits a NEWLINE token when provided with input that does not have a trailing new line. This behavior now matches what the C tokenizer does internally. (Contributed by Ammar Askar in bpo-33899 [https://bugs.python.org/issue?@action=redirect&bpo=33899].)

Notable changes in Python 3.7.2

In 3.7.2, **venv** on Windows no longer copies the original binaries, but creates redirector scripts named python.exe and pythonw.exe instead. This resolves a long standing issue where all virtual environments would have to be upgraded or recreated with each Python update. However, note that this release will still require recreation of virtual environments in order to get the new scripts.

Notable changes in Python 3.7.6

Devido a significativas preocupações de segurança, o parâmetro *reuse_address* de **asyncio.loop.create_datagram_endpoint()** não é mais suportado. Isso ocorre devido ao comportamento da opção de soquete SO_REUSEADDR no UDP. Para mais detalhes, consulte a documentação de loop.create_datagram_endpoint(). (Contribuição de Kyle Stanley, Antoine Pitrou e Yury Selivanov em bpo-37228 [https://bugs.python.org/issue?@action=redirect&bpo=37228].)

Notable changes in Python 3.7.10

Versões anteriores do Python permitiam o uso de ; e & como separadores de parâmetros de consulta nas funções **urllib.parse.parse_qs()** e **urllib.parse.parse_qsl()**. Devido a questões de segurança, e em conformidade com as recomendações mais recentes do W3C, isso foi alterado para permitir apenas uma única chave separadora, com & como padrão. Esta mudança também afeta as funções **cgi.parse()** e

`cgi.parse_multipart()` já que elas usam as funções afetadas internamente. Para obter mais detalhes, consulte sua respectiva documentação. (Contribuição de Adam Goldschmidt, Senthil Kumaran e Ken Jin em bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967].)

Notable changes in Python 3.7.11

Uma correção de segurança altera o comportamento **`ftplib.FTP`** para não confiar no endereço IPv4 enviado do servidor remoto ao configurar um canal de dados passivo. Em vez disso, reutilizamos o endereço IP do servidor ftp. Para códigos incomuns que requerem o comportamento antigo, defina um atributo `trust_server_pasv_ipv4_address` em sua instância de FTP para `True`. (Veja gh-87451 [https://github.com/python/cpython/issues/87451])

The presence of newline or tab characters in parts of a URL allows for some forms of attacks. Following the WHATWG specification that updates RFC 3986, ASCII newline `\n`, `\r` and tab `\t` characters are stripped from the URL by the parser **`urllib.parse()`** preventing such attacks. The removal characters are controlled by a new module level variable `urllib.parse._UNSAFE_URL_BYTES_TO_REMOVE`. (See gh-88048 [https://github.com/python/cpython/issues/88048])

Notable security feature in 3.7.14

Converter entre **`int`** e **`str`** em bases diferentes de 2 (binário), 4, 8 (octal), 16 (hexadecimal) ou 32 como base 10 (decimal) agora levanta uma exceção **`ValueError`** se o número de dígitos em forma de string estiver acima de um limite para evitar possíveis ataques de negação de serviço devido à complexidade algorítmica. Esta é uma mitigação para **CVE 2020-10735** [https://www.cve.org/CVERecord?id=CVE-2020-10735]. Este limite pode ser configurado ou desabilitado por variável de ambiente, sinalizador de linha de comando ou APIs de **`sys`**. Veja a documentação de limitação de comprimento de conversão de string inteira. O limite padrão é de 4300 dígitos em forma de string.

What's New In Python 3.6

Editores:: Elvis Pranskevichus <elvis@magic.io>, Yury Selivanov <yury@magic.io>

This article explains the new features in Python 3.6, compared to 3.5. Python 3.6 was released on December 23, 2016. See the changelog [https://docs.python.org/3.6/whatsnew/changelog.html] for a full list of changes.

Ver também

PEP 494 [https://peps.python.org/pep-0494/] - Python 3.6 Release Schedule

Resumo – Destaques da versão

Novos recursos de sintaxe:

- PEP 498, formatted string literals.
- PEP 515, underscores in numeric literals.
- PEP 526, syntax for variable annotations.
- PEP 525, asynchronous generators.
- PEP 530: asynchronous comprehensions.

Novos módulos de biblioteca:

- `secrets`: PEP 506 – Adding A Secrets Module To The Standard Library.

Melhorias na implementação do CPython:

- The dict type has been reimplemented to use a more compact representation based on a proposal by Raymond Hettinger [https://mail.python.org/pipermail/python-dev/2012-December/123028.html] and similar to the PyPy dict implementation [https://morepypy.blogspot.com/2015/01/faster-

more-memory-efficient-and-more.html]. This resulted in dictionaries using 20% to 25% less memory when compared to Python 3.5.
- Customization of class creation has been simplified with the new protocol.
- The class attribute definition order is now preserved.
- The order of elements in `**kwargs` now corresponds to the order in which keyword arguments were passed to the function.
- DTrace and SystemTap probing support has been added.
- The new PYTHONMALLOC environment variable can now be used to debug the interpreter memory allocation and access errors.

Melhorias significativas na biblioteca padrão:

- The **`asyncio`** module has received new features, significant usability and performance improvements, and a fair amount of bug fixes. Starting with Python 3.6 the `asyncio` module is no longer provisional and its API is considered stable.
- A new file system path protocol has been implemented to support path-like objects. All standard library functions operating on paths have been updated to work with the new protocol.
- The **`datetime`** module has gained support for Local Time Disambiguation.
- The **`typing`** module received a number of improvements.
- The **`tracemalloc`** module has been significantly reworked and is now used to provide better output for **`ResourceWarning`** as well as provide better diagnostics for memory allocation errors. See the PYTHONMALLOC section for more information.

Melhorias de Segurança:

- The new **`secrets`** module has been added to simplify the generation of cryptographically strong pseudo-random numbers suitable for managing secrets such as account authentication, tokens, and similar.
- On Linux, **`os.urandom()`** now blocks until the system urandom entropy pool is initialized to increase the security. See the **PEP 524** [https://peps.python.org/pep-0524/] for the rationale.
- The **`hashlib`** and **`ssl`** modules now support OpenSSL 1.1.0.

- The default settings and feature set of the **ssl** module have been improved.
- The **hashlib** module received support for the BLAKE2, SHA-3 and SHAKE hash algorithms and the **scrypt()** key derivation function.

Melhorias do Windows:

- PEP 528 and PEP 529, Windows filesystem and console encoding changed to UTF-8.
- The `py.exe` launcher, when used interactively, no longer prefers Python 2 over Python 3 when the user doesn't specify a version (via command line arguments or a config file). Handling of shebang lines remains unchanged - "python" refers to Python 2 in that case.
- `python.exe` and `pythonw.exe` have been marked as long-path aware, which means that the 260 character path limit may no longer apply. See removing the MAX_PATH limitation for details.
- A `._pth` file can be added to force isolated mode and fully specify all search paths to avoid registry and environment lookup. See the documentation for more information.
- A `python36.zip` file now works as a landmark to infer **PYTHONHOME**. See the documentation for more information.

Novas funcionalidades

PEP 498: Formatted string literals

PEP 498 [https://peps.python.org/pep-0498/] introduces a new kind of string literals: *f-strings*, or formatted string literals.

Formatted string literals are prefixed with `'f'` and are similar to the format strings accepted by **str.format()**. They contain replacement fields surrounded by curly braces. The replacement fields are expressions, which are evaluated at run time, and then formatted using the **format()** protocol:

```
>>> name = "Fred"
>>> f"He said his name is {name}."
'He said his name is Fred.'
```

```
>>> width = 10
>>> precision = 4
>>> value = decimal.Decimal("12.34567")
>>> f"result: {value:{width}.{precision}}"  # nested fields
'result:      12.35'
```

Ver também

PEP 498 **[https://peps.python.org/pep-0498/] – Literal String Interpolation.**
PEP written and implemented by Eric V. Smith.

Feature documentation.

PEP 526: Syntax for variable annotations

PEP 484 [https://peps.python.org/pep-0484/] introduced the standard for type annotations of function parameters, a.k.a. type hints. This PEP adds syntax to Python for annotating the types of variables including class variables and instance variables:

```
primes: List[int] = []

captain: str  # Note: no initial value!

class Starship:
    stats: Dict[str, int] = {}
```

Just as for function annotations, the Python interpreter does not attach any particular meaning to variable annotations and only stores them in the `__annotations__` attribute of a class or module.

In contrast to variable declarations in statically typed languages, the goal of annotation syntax is to provide an easy way to specify structured type metadata for third party tools and libraries via the abstract syntax tree and the `__annotations__` attribute.

Ver também

PEP 526 [https://peps.python.org/pep-0526/] – Syntax for variable annotations.

PEP written by Ryan Gonzalez, Philip House, Ivan Levkivskyi, Lisa Roach, and Guido van Rossum. Implemented by Ivan Levkivskyi.

Tools that use or will use the new syntax: mypy [https://www.mypy-lang.org/], pytype [https://github.com/google/pytype], PyCharm, etc.

PEP 515: Underscores in Numeric Literals

PEP 515 [https://peps.python.org/pep-0515/] adds the ability to use underscores in numeric literals for improved readability. For example:

```
>>> 1_000_000_000_000_000
1000000000000000
>>> 0x_FF_FF_FF_FF
4294967295
```

Single underscores are allowed between digits and after any base specifier. Leading, trailing, or multiple underscores in a row are not allowed.

The string formatting language also now has support for the `'_'` option to signal the use of an underscore for a thousands separator for floating-point presentation types and for integer presentation type `'d'`. For integer presentation types `'b'`, `'o'`, `'x'`, and `'X'`, underscores will be inserted every 4 digits:

```
>>> '{:_}'.format(1000000)
'1_000_000'
>>> '{:_x}'.format(0xFFFFFFFF)
'ffff_ffff'
```

Ver também

PEP 515 [https://peps.python.org/pep-0515/] – Underscores in Numeric Literals

PEP written by Georg Brandl and Serhiy Storchaka.

PEP 525: Asynchronous Generators

PEP 492 [https://peps.python.org/pep-0492/] introduced support for native coroutines and `async` / `await` syntax to Python 3.5. A notable limitation of the Python 3.5 implementation is that it was not possible to use `await` and `yield` in the same function body. In Python 3.6 this restriction has been lifted, making it possible to define *asynchronous generators*:

```
async def ticker(delay, to):
    """Yield numbers from 0 to *to* every *delay* seconds."""
    for i in range(to):
        yield i
        await asyncio.sleep(delay)
```

The new syntax allows for faster and more concise code.

Ver também

PEP 525 **[https://peps.python.org/pep-0525/] – Asynchronous Generators**
PEP written and implemented by Yury Selivanov.

PEP 530: Asynchronous Comprehensions

PEP 530 [https://peps.python.org/pep-0530/] adds support for using `async for` in list, set, dict comprehensions and generator expressions:

```
result = [i async for i in aiter() if i % 2]
```

Additionally, `await` expressions are supported in all kinds of comprehensions:

```
result = [await fun() for fun in funcs if await condition()]
```

Ver também

PEP 530 **[https://peps.python.org/pep-0530/] – Asynchronous Comprehensions**

PEP written and implemented by Yury Selivanov.

PEP 487: Simpler customization of class creation

It is now possible to customize subclass creation without using a metaclass. The new `__init_subclass__` classmethod will be called on the base class whenever a new subclass is created:

```
class PluginBase:
    subclasses = []

    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        cls.subclasses.append(cls)

class Plugin1(PluginBase):
    pass

class Plugin2(PluginBase):
    pass
```

In order to allow zero-argument **`super()`** calls to work correctly from **`__init_subclass__()`** implementations, custom metaclasses must ensure that the new `__classcell__` namespace entry is propagated to `type.__new__` (as described in Criando o objeto classe).

Ver também

PEP 487 [https://peps.python.org/pep-0487/] – Simpler customization of class creation

PEP written and implemented by Martin Teichmann.

Feature documentation

PEP 487: Descriptor Protocol Enhancements

PEP 487 [https://peps.python.org/pep-0487/] extends the descriptor protocol to include the new optional `__set_name__()` method. Whenever a new class is defined, the new method will be called on all descriptors included in the definition, providing them with a reference to the class being defined and the name given to the descriptor within the class namespace. In other words, instances of descriptors can now know the attribute name of the descriptor in the owner class:

```
class IntField:
    def __get__(self, instance, owner):
        return instance.__dict__[self.name]

    def __set__(self, instance, value):
        if not isinstance(value, int):
            raise ValueError(f'expecting integer in {self.name}'
        instance.__dict__[self.name] = value

    # this is the new initializer:
    def __set_name__(self, owner, name):
        self.name = name

class Model:
    int_field = IntField()
```

Ver também

PEP 487 [https://peps.python.org/pep-0487/] – Simpler customization of class creation

PEP written and implemented by Martin Teichmann.

Feature documentation

PEP 519: Adding a file system path protocol

File system paths have historically been represented as `str` or `bytes` objects. This has led to people who write code which operate on file system paths to assume that such objects are only one of those two types (an `int`

representing a file descriptor does not count as that is not a file path). Unfortunately that assumption prevents alternative object representations of file system paths like `pathlib` from working with pre-existing code, including Python's standard library.

To fix this situation, a new interface represented by `os.PathLike` has been defined. By implementing the `__fspath__()` method, an object signals that it represents a path. An object can then provide a low-level representation of a file system path as a `str` or `bytes` object. This means an object is considered path-like if it implements `os.PathLike` or is a `str` or `bytes` object which represents a file system path. Code can use `os.fspath()`, `os.fsdecode()`, or `os.fsencode()` to explicitly get a `str` and/or `bytes` representation of a path-like object.

The built-in `open()` function has been updated to accept `os.PathLike` objects, as have all relevant functions in the `os` and `os.path` modules, and most other functions and classes in the standard library. The `os.DirEntry` class and relevant classes in `pathlib` have also been updated to implement `os.PathLike`.

The hope is that updating the fundamental functions for operating on file system paths will lead to third-party code to implicitly support all path-like objects without any code changes, or at least very minimal ones (e.g. calling `os.fspath()` at the beginning of code before operating on a path-like object).

Here are some examples of how the new interface allows for `pathlib.Path` to be used more easily and transparently with pre-existing code:

```
>>> import pathlib
>>> with open(pathlib.Path("README")) as f:
...     contents = f.read()
...
>>> import os.path
>>> os.path.splitext(pathlib.Path("some_file.txt"))
('some_file', '.txt')
>>> os.path.join("/a/b", pathlib.Path("c"))
'/a/b/c'
>>> import os
```

```
>>> os.fspath(pathlib.Path("some_file.txt"))
'some_file.txt'
```

(Implemented by Brett Cannon, Ethan Furman, Dusty Phillips, and Jelle Zijlstra.)

Ver também

PEP 519 [https://peps.python.org/pep-0519/] – Adding a file system path protocol

PEP written by Brett Cannon and Koos Zevenhoven.

PEP 495: Local Time Disambiguation

In most world locations, there have been and will be times when local clocks are moved back. In those times, intervals are introduced in which local clocks show the same time twice in the same day. In these situations, the information displayed on a local clock (or stored in a Python datetime instance) is insufficient to identify a particular moment in time.

PEP 495 [https://peps.python.org/pep-0495/] adds the new *fold* attribute to instances of `datetime.datetime` and `datetime.time` classes to differentiate between two moments in time for which local times are the same:

```
>>> u0 = datetime(2016, 11, 6, 4, tzinfo=timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
...     print(u.time(), 'UTC =', t.time(), t.tzname(), t.fold)
...
04:00:00 UTC = 00:00:00 EDT 0
05:00:00 UTC = 01:00:00 EDT 0
06:00:00 UTC = 01:00:00 EST 1
07:00:00 UTC = 02:00:00 EST 0
```

The values of the `fold` attribute have the value `0` for all instances except those that represent the second (chronologically) moment in time in an ambiguous case.

Ver também

PEP 495 **[https://peps.python.org/pep-0495/] – Local Time Disambiguation**
PEP written by Alexander Belopolsky and Tim Peters, implementation by Alexander Belopolsky.

PEP 529: Change Windows filesystem encoding to UTF-8

Representing filesystem paths is best performed with str (Unicode) rather than bytes. However, there are some situations where using bytes is sufficient and correct.

Prior to Python 3.6, data loss could result when using bytes paths on Windows. With this change, using bytes to represent paths is now supported on Windows, provided those bytes are encoded with the encoding returned by `sys.getfilesystemencoding()`, which now defaults to `'utf-8'`.

Applications that do not use str to represent paths should use `os.fsencode()` and `os.fsdecode()` to ensure their bytes are correctly encoded. To revert to the previous behaviour, set `PYTHONLEGACYWINDOWSFSENCODING` or call `sys. enablelegacywindowsfsencoding()`.

See **PEP 529** [https://peps.python.org/pep-0529/] for more information and discussion of code modifications that may be required.

PEP 528: Change Windows console encoding to UTF-8

The default console on Windows will now accept all Unicode characters and provide correctly read str objects to Python code. `sys.stdin`, `sys.stdout` and `sys.stderr` now default to utf-8 encoding.

This change only applies when using an interactive console, and not when redirecting files or pipes. To revert to the previous behaviour for interactive console use, set `PYTHONLEGACYWINDOWSSTDIO`.

Ver também

PEP 528 [https://peps.python.org/pep-0528/] – Change Windows console encoding to UTF-8
PEP written and implemented by Steve Dower.

PEP 520: Preserving Class Attribute Definition Order

Attributes in a class definition body have a natural ordering: the same order in which the names appear in the source. This order is now preserved in the new class's `__dict__` attribute.

Also, the effective default class *execution* namespace (returned from type.__prepare__()) is now an insertion-order-preserving mapping.

Ver também

PEP 520 [https://peps.python.org/pep-0520/] – Preserving Class Attribute Definition Order
PEP written and implemented by Eric Snow.

PEP 468: Preserving Keyword Argument Order

`**kwargs` in a function signature is now guaranteed to be an insertion-order-preserving mapping.

Ver também

PEP 468 [https://peps.python.org/pep-0468/] – Preserving Keyword Argument Order
PEP written and implemented by Eric Snow.

New dict implementation

The dict type now uses a “compact” representation based on a proposal by Raymond Hettinger [https://mail.python.org/pipermail/python-dev/2012-December/123028.html] which was first implemented by PyPy [https://morepypy.blogspot.com/2015/01/faster-more-memory-efficient-and-more.html]. The memory usage of the new `dict()` is between 20% and 25% smaller compared to Python 3.5.

The order-preserving aspect of this new implementation is considered an implementation detail and should not be relied upon (this may change in the future, but it is desired to have this new dict implementation in the language for a few releases before changing the language spec to mandate order-preserving semantics for all current and future Python implementations; this also helps preserve backwards-compatibility with older versions of the language where random iteration order is still in effect, e.g. Python 3.5).

(Contributed by INADA Naoki in bpo-27350 [https://bugs.python.org/issue?@action=redirect&bpo=27350]. Idea originally suggested by Raymond Hettinger [https://mail.python.org/pipermail/python-dev/2012-December/123028.html].)

PEP 523: Adding a frame evaluation API to CPython

While Python provides extensive support to customize how code executes, one place it has not done so is in the evaluation of frame objects. If you wanted some way to intercept frame evaluation in Python there really wasn’t any way without directly manipulating function pointers for defined functions.

PEP 523 [https://peps.python.org/pep-0523/] changes this by providing an API to make frame evaluation pluggable at the C level. This will allow for tools such as debuggers and JITs to intercept frame evaluation before the execution of Python code begins. This enables the use of alternative evaluation implementations for Python code, tracking frame evaluation, etc.

This API is not part of the limited C API and is marked as private to signal that usage of this API is expected to be limited and only applicable to very select, low-level use-cases. Semantics of the API will change with Python as necessary.

Ver também

PEP 523 [https://peps.python.org/pep-0523/] – Adding a frame evaluation API to CPython

PEP written by Brett Cannon and Dino Viehland.

PYTHONMALLOC environment variable

The new `PYTHONMALLOC` environment variable allows setting the Python memory allocators and installing debug hooks.

It is now possible to install debug hooks on Python memory allocators on Python compiled in release mode using `PYTHONMALLOC=debug`. Effects of debug hooks:

- Newly allocated memory is filled with the byte `0xCB`
- Freed memory is filled with the byte `0xDB`
- Detect violations of the Python memory allocator API. For example, `PyObject_Free()` called on a memory block allocated by `PyMem_Malloc()`.
- Detect writes before the start of a buffer (buffer underflows)
- Detect writes after the end of a buffer (buffer overflows)
- Check that the GIL is held when allocator functions of `PYMEM_DOMAIN_OBJ` (ex: `PyObject_Malloc()`) and `PYMEM_DOMAIN_MEM` (ex: `PyMem_Malloc()`) domains are called.

Checking if the GIL is held is also a new feature of Python 3.6.

See the `PyMem_SetupDebugHooks()` function for debug hooks on Python memory allocators.

It is now also possible to force the usage of the **`malloc()`** allocator of the C library for all Python memory allocations using `PYTHONMALLOC=malloc`. This is helpful when using external memory debuggers like Valgrind on a Python compiled in release mode.

On error, the debug hooks on Python memory allocators now use the **tracemalloc** module to get the traceback where a memory block was allocated.

Example of fatal error on buffer overflow using `python3.6 -X tracemalloc=5` (store 5 frames in traces):

```
Debug memory block at address p=0x7fbcd41666f8: API 'o'
    4 bytes originally requested
    The 7 pad bytes at p-7 are FORBIDDENBYTE, as expected.
    The 8 pad bytes at tail=0x7fbcd41666fc are not all FORBIDDEN
        at tail+0: 0x02 *** OUCH
        at tail+1: 0xfb
        at tail+2: 0xfb
        at tail+3: 0xfb
        at tail+4: 0xfb
        at tail+5: 0xfb
        at tail+6: 0xfb
        at tail+7: 0xfb
    The block was made by call #1233329 to debug malloc/realloc.
    Data at p: 1a 2b 30 00

Memory block allocated at (most recent call first):
  File "test/test_bytes.py", line 323
  File "unittest/case.py", line 600
  File "unittest/case.py", line 648
  File "unittest/suite.py", line 122
  File "unittest/suite.py", line 84

Fatal Python error: bad trailing pad byte

Current thread 0x00007fbcdbd32700 (most recent call first):
  File "test/test_bytes.py", line 323 in test_hex
  File "unittest/case.py", line 600 in run
  File "unittest/case.py", line 648 in __call__
  File "unittest/suite.py", line 122 in run
  File "unittest/suite.py", line 84 in __call__
  File "unittest/suite.py", line 122 in run
  File "unittest/suite.py", line 84 in __call__
  ...
```

(Contributed by Victor Stinner in bpo-26516 [https://bugs.python.org/issue?@action=redirect&bpo=26516] and bpo-26564 [https://bugs.python.org/issue?@action=redirect&bpo=26564].)

DTrace and SystemTap probing support

Python can now be built `--with-dtrace` which enables static markers for the following events in the interpreter:

- function call/return
- garbage collection started/finished
- line of code executed.

This can be used to instrument running interpreters in production, without the need to recompile specific debug builds or providing application-specific profiling/debugging code.

More details in Instrumentando o CPython com DTrace e SystemTap.

The current implementation is tested on Linux and macOS. Additional markers may be added in the future.

(Contributed by Łukasz Langa in bpo-21590 [https://bugs.python.org/issue?@action=redirect&bpo=21590], based on patches by Jesús Cea Avión, David Malcolm, and Nikhil Benesch.)

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- A `global` or `nonlocal` statement must now textually appear before the first use of the affected name in the same scope. Previously this was a **`SyntaxWarning`**.
- It is now possible to set a special method to `None` to indicate that the corresponding operation is not available. For example, if a class sets **`__iter__()`** to `None`, the class is not iterable. (Contributed by Andrew

Barnert and Ivan Levkivskyi in bpo-25958 [https://bugs.python.org/issue?@action=redirect&bpo=25958].)

- Long sequences of repeated traceback lines are now abbreviated as `"[Previous line repeated {count} more times]"` (see traceback for an example). (Contributed by Emanuel Barry in bpo-26823 [https://bugs.python.org/issue?@action=redirect&bpo=26823].)
- Import now raises the new exception `ModuleNotFoundError` (subclass of `ImportError`) when it cannot find a module. Code that currently checks for ImportError (in try-except) will still work. (Contributed by Eric Snow in bpo-15767 [https://bugs.python.org/issue?@action=redirect&bpo=15767].)
- Class methods relying on zero-argument `super()` will now work correctly when called from metaclass methods during class creation. (Contributed by Martin Teichmann in bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722].)

Novos módulos

segredos

The main purpose of the new `secrets` module is to provide an obvious way to reliably generate cryptographically strong pseudo-random values suitable for managing secrets, such as account authentication, tokens, and similar.

Aviso

Note that the pseudo-random generators in the `random` module should *NOT* be used for security purposes. Use `secrets` on Python 3.6+ and `os.urandom()` on Python 3.5 and earlier.

Ver também

PEP 506 [https://peps.python.org/pep-0506/] – Adding A Secrets Module To The Standard Library

PEP written and implemented by Steven D'Aprano.

Módulos melhorados

array

Exhausted iterators of `array.array` will now stay exhausted even if the iterated array is extended. This is consistent with the behavior of other mutable sequences.

Contributed by Serhiy Storchaka in bpo-26492 [https://bugs.python.org/issue?@action=redirect&bpo=26492].

ast

The new `ast.Constant` AST node has been added. It can be used by external AST optimizers for the purposes of constant folding.

Contributed by Victor Stinner in bpo-26146 [https://bugs.python.org/issue?@action=redirect&bpo=26146].

asyncio

Starting with Python 3.6 the `asyncio` module is no longer provisional and its API is considered stable.

Notable changes in the `asyncio` module since Python 3.5.0 (all backported to 3.5.x due to the provisional status):

- The `get_event_loop()` function has been changed to always return the currently running loop when called from coroutines and callbacks. (Contributed by Yury Selivanov in bpo-28613 [https://bugs.python.org/issue?@action=redirect&bpo=28613].)
- The `ensure_future()` function and all functions that use it, such as `loop.run_until_complete()`, now accept all kinds of awaitable objects. (Contributed by Yury Selivanov.)
- New `run_coroutine_threadsafe()` function to submit coroutines to event loops from other threads. (Contributed by Vincent Michel.)

- New `Transport.is_closing()` method to check if the transport is closing or closed. (Contributed by Yury Selivanov.)
- The `loop.create_server()` method can now accept a list of hosts. (Contributed by Yann Sionneau.)
- New `loop.create_future()` method to create Future objects. This allows alternative event loop implementations, such as uvloop [https://github.com/MagicStack/uvloop], to provide a faster `asyncio.Future` implementation. (Contributed by Yury Selivanov in bpo-27041 [https://bugs.python.org/issue?@action=redirect&bpo=27041].)
- New `loop.get_exception_handler()` method to get the current exception handler. (Contributed by Yury Selivanov in bpo-27040 [https://bugs.python.org/issue?@action=redirect&bpo=27040].)
- New `StreamReader.readuntil()` method to read data from the stream until a separator bytes sequence appears. (Contributed by Mark Korenberg.)
- The performance of `StreamReader.readexactly()` has been improved. (Contributed by Mark Korenberg in bpo-28370 [https://bugs.python.org/issue?@action=redirect&bpo=28370].)
- The `loop.getaddrinfo()` method is optimized to avoid calling the system `getaddrinfo` function if the address is already resolved. (Contributed by A. Jesse Jiryu Davis.)
- The `loop.stop()` method has been changed to stop the loop immediately after the current iteration. Any new callbacks scheduled as a result of the last iteration will be discarded. (Contributed by Guido van Rossum in bpo-25593 [https://bugs.python.org/issue?@action=redirect&bpo=25593].)
- `Future.set_exception` will now raise `TypeError` when passed an instance of the `StopIteration` exception. (Contributed by Chris Angelico in bpo-26221 [https://bugs.python.org/issue?@action=redirect&bpo=26221].)
- New `loop.connect_accepted_socket()` method to be used by servers that accept connections outside of asyncio, but that use asyncio to handle them. (Contributed by Jim Fulton in bpo-27392 [https://bugs.python.org/issue?@action=redirect&bpo=27392].)
- `TCP_NODELAY` flag is now set for all TCP transports by default. (Contributed by Yury Selivanov in bpo-27456 [https://bugs.python.org/issue?@action=redirect&bpo=27456].)

- New `loop.shutdown_asyncgens()` to properly close pending asynchronous generators before closing the loop. (Contributed by Yury Selivanov in bpo-28003 [https://bugs.python.org/issue?@action=redirect&bpo=28003].)
- `Future` and `Task` classes now have an optimized C implementation which makes asyncio code up to 30% faster. (Contributed by Yury Selivanov and INADA Naoki in bpo-26081 [https://bugs.python.org/issue?@action=redirect&bpo=26081] and bpo-28544 [https://bugs.python.org/issue?@action=redirect&bpo=28544].)

binascii

The `b2a_base64()` function now accepts an optional *newline* keyword argument to control whether the newline character is appended to the return value. (Contributed by Victor Stinner in bpo-25357 [https://bugs.python.org/issue?@action=redirect&bpo=25357].)

cmath

The new `cmath.tau` (τ) constant has been added. (Contributed by Lisa Roach in bpo-12345 [https://bugs.python.org/issue?@action=redirect&bpo=12345], see **PEP 628** [https://peps.python.org/pep-0628/] for details.)

New constants: `cmath.inf` and `cmath.nan` to match `math.inf` and `math.nan`, and also `cmath.infj` and `cmath.nanj` to match the format used by complex repr. (Contributed by Mark Dickinson in bpo-23229 [https://bugs.python.org/issue?@action=redirect&bpo=23229].)

collections

The new `Collection` abstract base class has been added to represent sized iterable container classes. (Contributed by Ivan Levkivskyi, docs by Neil Girdhar in bpo-27598 [https://bugs.python.org/issue?@action=redirect&bpo=27598].)

The new `Reversible` abstract base class represents iterable classes that also provide the `__reversed__()` method. (Contributed by Ivan Levkivskyi in bpo-25987 [https://bugs.python.org/issue?@action=redirect&bpo=25987].)

The new `AsyncGenerator` abstract base class represents asynchronous generators. (Contributed by Yury Selivanov in bpo-28720 [https://bugs.python.org/issue?@action=redirect&bpo=28720].)

The `namedtuple()` function now accepts an optional keyword argument *module*, which, when specified, is used for the `__module__` attribute of the returned named tuple class. (Contributed by Raymond Hettinger in bpo-17941 [https://bugs.python.org/issue?@action=redirect&bpo=17941].)

The *verbose* and *rename* arguments for `namedtuple()` are now keyword-only. (Contributed by Raymond Hettinger in bpo-25628 [https://bugs.python.org/issue?@action=redirect&bpo=25628].)

Recursive `collections.deque` instances can now be pickled. (Contributed by Serhiy Storchaka in bpo-26482 [https://bugs.python.org/issue?@action=redirect&bpo=26482].)

concurrent.futures

The `ThreadPoolExecutor` class constructor now accepts an optional *thread_name_prefix* argument to make it possible to customize the names of the threads created by the pool. (Contributed by Gregory P. Smith in bpo-27664 [https://bugs.python.org/issue?@action=redirect&bpo=27664].)

contextlib

The `contextlib.AbstractContextManager` class has been added to provide an abstract base class for context managers. It provides a sensible default implementation for `__enter__()` which returns `self` and leaves `__exit__()` an abstract method. A matching class has been added to the `typing` module as `typing.ContextManager`. (Contributed by Brett Cannon in bpo-25609 [https://bugs.python.org/issue?@action=redirect&bpo=25609].)

datetime

The `datetime` and `time` classes have the new `fold` attribute used to disambiguate local time when necessary. Many functions in the `datetime`

have been updated to support local time disambiguation. See Local Time Disambiguation section for more information. (Contributed by Alexander Belopolsky in bpo-24773 [https://bugs.python.org/issue?@action=redirect&bpo=24773].)

The **`datetime.strftime()`** and **`date.strftime()`** methods now support ISO 8601 date directives `%G`, `%u` and `%V`. (Contributed by Ashley Anderson in bpo-12006 [https://bugs.python.org/issue?@action=redirect&bpo=12006].)

The **`datetime.isoformat()`** function now accepts an optional *timespec* argument that specifies the number of additional components of the time value to include. (Contributed by Alessandro Cucci and Alexander Belopolsky in bpo-19475 [https://bugs.python.org/issue?@action=redirect&bpo=19475].)

The **`datetime.combine()`** now accepts an optional *tzinfo* argument. (Contributed by Alexander Belopolsky in bpo-27661 [https://bugs.python.org/issue?@action=redirect&bpo=27661].)

decimal

New **`Decimal.as_integer_ratio()`** method that returns a pair `(n, d)` of integers that represent the given **`Decimal`** instance as a fraction, in lowest terms and with a positive denominator:

```
>>> Decimal('-3.14').as_integer_ratio()
(-157, 50)
```

(Contributed by Stefan Krah amd Mark Dickinson in bpo-25928 [https://bugs.python.org/issue?@action=redirect&bpo=25928].)

distutils

The `default_format` attribute has been removed from `distutils.command.sdist.sdist` and the `formats` attribute defaults to `['gztar']`. Although not anticipated, any code relying on the presence of `default_format` may need to be adapted. See bpo-27819 [https://bugs.python.org/issue?@action=redirect&bpo=27819] for more details.

email

The new email API, enabled via the *policy* keyword to various constructors, is no longer provisional. The `email` documentation has been reorganized and rewritten to focus on the new API, while retaining the old documentation for the legacy API. (Contributed by R. David Murray in bpo-24277 [https://bugs.python.org/issue?@action=redirect&bpo=24277].)

The `email.mime` classes now all accept an optional *policy* keyword. (Contributed by Berker Peksag in bpo-27331 [https://bugs.python.org/issue?@action=redirect&bpo=27331].)

The `DecodedGenerator` now supports the *policy* keyword.

There is a new `policy` attribute, `message_factory`, that controls what class is used by default when the parser creates new message objects. For the `email.policy.compat32` policy this is `Message`, for the new policies it is `EmailMessage`. (Contributed by R. David Murray in bpo-20476 [https://bugs.python.org/issue?@action=redirect&bpo=20476].)

encodings

On Windows, added the `'oem'` encoding to use `CP_OEMCP`, and the `'ansi'` alias for the existing `'mbcs'` encoding, which uses the `CP_ACP` code page. (Contributed by Steve Dower in bpo-27959 [https://bugs.python.org/issue?@action=redirect&bpo=27959].)

enum

Two new enumeration base classes have been added to the `enum` module: `Flag` and `IntFlag`. Both are used to define constants that can be combined using the bitwise operators. (Contributed by Ethan Furman in bpo-23591 [https://bugs.python.org/issue?@action=redirect&bpo=23591].)

Many standard library modules have been updated to use the `IntFlag` class for their constants.

The new `enum.auto` value can be used to assign values to enum members automatically:

```
>>> from enum import Enum, auto
>>> class Color(Enum):
...     red = auto()
...     blue = auto()
...     green = auto()
...
>>> list(Color)
[<Color.red: 1>, <Color.blue: 2>, <Color.green: 3>]
```

faulthandler

On Windows, the `faulthandler` module now installs a handler for Windows exceptions: see `faulthandler.enable()`. (Contributed by Victor Stinner in bpo-23848 [https://bugs.python.org/issue?@action=redirect&bpo=23848].)

fileinput

`hook_encoded()` now supports the *errors* argument. (Contributed by Joseph Hackman in bpo-25788 [https://bugs.python.org/issue?@action=redirect&bpo=25788].)

hashlib

`hashlib` supports OpenSSL 1.1.0. The minimum recommend version is 1.0.2. (Contributed by Christian Heimes in bpo-26470 [https://bugs.python.org/issue?@action=redirect&bpo=26470].)

BLAKE2 hash functions were added to the module. `blake2b()` and `blake2s()` are always available and support the full feature set of BLAKE2. (Contributed by Christian Heimes in bpo-26798 [https://bugs.python.org/issue?@action=redirect&bpo=26798] based on code by Dmitry Chestnykh and Samuel Neves. Documentation written by Dmitry Chestnykh.)

The SHA-3 hash functions `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, and SHAKE hash functions `shake_128()` and `shake_256()` were added. (Contributed by Christian Heimes in bpo-16113 [https://bugs.python.org/issue?@action=redirect&bpo=16113]. Keccak Code Package by Guido Bertoni, Joan Daemen, Michaël Peeters, Gilles Van Assche, and Ronny Van Keer.)

The password-based key derivation function `scrypt()` is now available with OpenSSL 1.1.0 and newer. (Contributed by Christian Heimes in bpo-27928 [https://bugs.python.org/issue?@action=redirect&bpo=27928].)

http.client

`HTTPConnection.request()` and `endheaders()` both now support chunked encoding request bodies. (Contributed by Demian Brecht and Rolf Krahl in bpo-12319 [https://bugs.python.org/issue?@action=redirect&bpo=12319].)

idlelib e IDLE

The idlelib package is being modernized and refactored to make IDLE look and work better and to make the code easier to understand, test, and improve. Part of making IDLE look better, especially on Linux and Mac, is using ttk widgets, mostly in the dialogs. As a result, IDLE no longer runs with tcl/tk 8.4. It now requires tcl/tk 8.5 or 8.6. We recommend running the latest release of either.

'Modernizing' includes renaming and consolidation of idlelib modules. The renaming of files with partial uppercase names is similar to the renaming of, for instance, Tkinter and TkFont to tkinter and tkinter.font in 3.0. As a result, imports of idlelib files that worked in 3.5 will usually not work in 3.6. At least a module name change will be needed (see idlelib/README.txt), sometimes more. (Name changes contributed by Al Swiegart and Terry Reedy in bpo-24225 [https://bugs.python.org/issue?@action=redirect&bpo=24225]. Most idlelib patches since have been and will be part of the process.)

In compensation, the eventual result with be that some idlelib classes will be easier to use, with better APIs and docstrings explaining them. Additional useful information will be added to idlelib when available.

New in 3.6.2:

Multiple fixes for autocompletion. (Contributed by Louie Lu in bpo-15786 [https://bugs.python.org/issue?@action=redirect&bpo=15786].)

New in 3.6.3:

Module Browser (on the File menu, formerly called Class Browser), now displays nested functions and classes in addition to top-level functions and classes. (Contributed by Guilherme Polo, Cheryl Sabella, and Terry Jan Reedy in bpo-1612262 [https://bugs.python.org/issue?@action=redirect&bpo=1612262].)

The IDLE features formerly implemented as extensions have been reimplemented as normal features. Their settings have been moved from the Extensions tab to other dialog tabs. (Contributed by Charles Wohlganger and Terry Jan Reedy in bpo-27099 [https://bugs.python.org/issue?@action=redirect&bpo=27099].)

The Settings dialog (Options, Configure IDLE) has been partly rewritten to improve both appearance and function. (Contributed by Cheryl Sabella and Terry Jan Reedy in multiple issues.)

New in 3.6.4:

The font sample now includes a selection of non-Latin characters so that users can better see the effect of selecting a particular font. (Contributed by Terry Jan Reedy in bpo-13802 [https://bugs.python.org/issue?@action=redirect&bpo=13802].) The sample can be edited to include other characters. (Contributed by Serhiy Storchaka in bpo-31860 [https://bugs.python.org/issue?@action=redirect&bpo=31860].)

New in 3.6.6:

Editor code context option revised. Box displays all context lines up to maxlines. Clicking on a context line jumps the editor to that line. Context colors for custom themes is added to Highlights tab of Settings dialog. (Contributed by Cheryl Sabella and Terry Jan Reedy in bpo-33642 [https://bugs.python.org/issue?@action=redirect&bpo=33642], bpo-33768 [https://bugs.python.org/issue?@action=redirect&bpo=33768], and bpo-33679 [https://bugs.python.org/issue?@action=redirect&bpo=33679].)

On Windows, a new API call tells Windows that tk scales for DPI. On Windows 8.1+ or 10, with DPI compatibility properties of the Python

binary unchanged, and a monitor resolution greater than 96 DPI, this should make text and lines sharper. It should otherwise have no effect. (Contributed by Terry Jan Reedy in bpo-33656 [https://bugs.python.org/issue?@action=redirect&bpo=33656].)

New in 3.6.7:

A saída em N linhas (50 por padrão) é compactada até um limite inferior. N pode ser alterado na seção PyShell da página General da caixa de diálogo Settings. Menos linhas, mas possivelmente extra longas, podem ser espremidas clicando com o botão direito do mouse na saída. A saída compactada pode ser expandida no local clicando duas vezes no botão ou na área de transferência ou em uma janela separada clicando com o botão direito do mouse. (Contribuição de Tal Einat em bpo-1529353 [https://bugs.python.org/issue?@action=redirect&bpo=1529353].)

importlib

Import now raises the new exception `ModuleNotFoundError` (subclass of `ImportError`) when it cannot find a module. Code that current checks for `ImportError` (in try-except) will still work. (Contributed by Eric Snow in bpo-15767 [https://bugs.python.org/issue?@action=redirect&bpo=15767].)

`importlib.util.LazyLoader` now calls `create_module()` on the wrapped loader, removing the restriction that `importlib.machinery.BuiltinImporter` and `importlib.machinery.ExtensionFileLoader` couldn't be used with `importlib.util.LazyLoader`.

`importlib.util.cache_from_source()`, `importlib.util.source_from_cache()`, and `importlib.util.spec_from_file_location()` now accept a path-like object.

inspect

The `inspect.signature()` function now reports the implicit `.0` parameters generated by the compiler for comprehension and generator expression scopes as if they were positional-only parameters called `implicit0`. (Contributed by Jelle Zijlstra in bpo-19611 [https://bugs.python.org/issue?@action=redirect&bpo=19611].)

To reduce code churn when upgrading from Python 2.7 and the legacy `inspect.getargspec()` API, the previously documented deprecation of `inspect.getfullargspec()` has been reversed. While this function is convenient for single/source Python 2/3 code bases, the richer `inspect.signature()` interface remains the recommended approach for new code. (Contributed by Nick Coghlan in bpo-27172 [https://bugs.python.org/issue?@action=redirect&bpo=27172])

json

`json.load()` and `json.loads()` now support binary input. Encoded JSON should be represented using either UTF-8, UTF-16, or UTF-32. (Contributed by Serhiy Storchaka in bpo-17909 [https://bugs.python.org/issue?@action=redirect&bpo=17909].)

logging

The new `WatchedFileHandler.reopenIfNeeded()` method has been added to add the ability to check if the log file needs to be reopened. (Contributed by Marian Horban in bpo-24884 [https://bugs.python.org/issue?@action=redirect&bpo=24884].)

math

The tau (τ) constant has been added to the `math` and `cmath` modules. (Contributed by Lisa Roach in bpo-12345 [https://bugs.python.org/issue?@action=redirect&bpo=12345], see **PEP 628** [https://peps.python.org/pep-0628/] for details.)

multiprocessing

Proxy Objects returned by `multiprocessing.Manager()` can now be nested. (Contributed by Davin Potts in bpo-6766 [https://bugs.python.org/issue?@action=redirect&bpo=6766].)

os

See the summary of PEP 519 for details on how the `os` and `os.path` modules now support path-like objects.

`scandir()` now supports `bytes` paths on Windows.

A new `close()` method allows explicitly closing a `scandir()` iterator. The `scandir()` iterator now supports the context manager protocol. If a `scandir()` iterator is neither exhausted nor explicitly closed a `ResourceWarning` will be emitted in its destructor. (Contributed by Serhiy Storchaka in bpo-25994 [https://bugs.python.org/issue?@action=redirect&bpo=25994].)

On Linux, `os.urandom()` now blocks until the system urandom entropy pool is initialized to increase the security. See the **PEP 524** [https://peps.python.org/pep-0524/] for the rationale.

The Linux `getrandom()` syscall (get random bytes) is now exposed as the new `os.getrandom()` function. (Contributed by Victor Stinner, part of the **PEP 524** [https://peps.python.org/pep-0524/])

pathlib

`pathlib` now supports path-like objects. (Contributed by Brett Cannon in bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186].)

See the summary of PEP 519 for details.

pdb

The `Pdb` class constructor has a new optional *readrc* argument to control whether `.pdbrc` files should be read.

pickle

Objects that need `__new__` called with keyword arguments can now be pickled using pickle protocols older than protocol version 4. Protocol version 4 already supports this case. (Contributed by Serhiy Storchaka in bpo-24164 [https://bugs.python.org/issue?@action=redirect&bpo=24164].)

pickletools

`pickletools.dis()` now outputs the implicit memo index for the `MEMOIZE` opcode. (Contributed by Serhiy Storchaka in bpo-25382 [https://bugs.python.org/issue?@action=redirect&bpo=25382].)

pydoc

The **`pydoc`** module has learned to respect the `MANPAGER` environment variable. (Contributed by Matthias Klose in bpo-8637 [https://bugs.python.org/issue?@action=redirect&bpo=8637].)

`help()` and **`pydoc`** can now list named tuple fields in the order they were defined rather than alphabetically. (Contributed by Raymond Hettinger in bpo-24879 [https://bugs.python.org/issue?@action=redirect&bpo=24879].)

random

The new **`choices()`** function returns a list of elements of specified size from the given population with optional weights. (Contributed by Raymond Hettinger in bpo-18844 [https://bugs.python.org/issue?@action=redirect&bpo=18844].)

re

Added support of modifier spans in regular expressions. Examples: `'(?i:p)ython'` matches `'python'` and `'Python'`, but not `'PYTHON'`; `'(?i)g(?-i:v)r'` matches `'GvR'` and `'gvr'`, but not `'GVR'`. (Contributed by Serhiy Storchaka in bpo-433028 [https://bugs.python.org/issue?@action=redirect&bpo=433028].)

Match object groups can be accessed by `__getitem__`, which is equivalent to `group()`. So `mo['name']` is now equivalent to `mo.group('name')`. (Contributed by Eric Smith in bpo-24454 [https://bugs.python.org/issue?@action=redirect&bpo=24454].)

`Match` objects now support `index-like objects` as group indices. (Contributed by Jeroen Demeyer and Xiang Zhang in bpo-27177 [https://bugs.python.org/issue?@action=redirect&bpo=27177].)

readline

Added `set_auto_history()` to enable or disable automatic addition of input to the history list. (Contributed by Tyler Crompton in bpo-26870 [https://bugs.python.org/issue?@action=redirect&bpo=26870].)

rlcompleter

Private and special attribute names now are omitted unless the prefix starts with underscores. A space or a colon is added after some completed keywords. (Contributed by Serhiy Storchaka in bpo-25011 [https://bugs.python.org/issue?@action=redirect&bpo=25011] and bpo-25209 [https://bugs.python.org/issue?@action=redirect&bpo=25209].)

shlex

The `shlex` has much improved shell compatibility through the new *punctuation_chars* argument to control which characters are treated as punctuation. (Contributed by Vinay Sajip in bpo-1521950 [https://bugs.python.org/issue?@action=redirect&bpo=1521950].)

site

When specifying paths to add to `sys.path` in a `.pth` file, you may now specify file paths on top of directories (e.g. zip files). (Contributed by Wolfgang Langner in bpo-26587 [https://bugs.python.org/issue?@action=redirect&bpo=26587]).

sqlite3

sqlite3.Cursor.lastrowid now supports the `REPLACE` statement. (Contributed by Alex LordThorsen in bpo-16864 [https://bugs.python.org/issue?@action=redirect&bpo=16864].)

socket

The **ioctl()** function now supports the **SIO_LOOPBACK_FAST_PATH** control code. (Contributed by Daniel Stokes in bpo-26536 [https://bugs.python.org/issue?@action=redirect&bpo=26536].)

The **getsockopt()** constants `SO_DOMAIN`, `SO_PROTOCOL`, `SO_PEERSEC`, and `SO_PASSSEC` are now supported. (Contributed by Christian Heimes in bpo-26907 [https://bugs.python.org/issue?@action=redirect&bpo=26907].)

The **setsockopt()** now supports the `setsockopt(level, optname, None, optlen: int)` form. (Contributed by Christian Heimes in bpo-27744 [https://bugs.python.org/issue?@action=redirect&bpo=27744].)

The socket module now supports the address family **AF_ALG** to interface with Linux Kernel crypto API. `ALG_*`, `SOL_ALG` and **sendmsg_afalg()** were added. (Contributed by Christian Heimes in bpo-27744 [https://bugs.python.org/issue?@action=redirect&bpo=27744] with support from Victor Stinner.)

New Linux constants `TCP_USER_TIMEOUT` and `TCP_CONGESTION` were added. (Contributed by Omar Sandoval, bpo-26273 [https://bugs.python.org/issue?@action=redirect&bpo=26273]).

socketserver

Servers based on the **socketserver** module, including those defined in **http.server**, **xmlrpc.server** and **wsgiref.simple_server**, now support the context manager protocol. (Contributed by Aviv Palivoda in bpo-26404 [https://bugs.python.org/issue?@action=redirect&bpo=26404].)

The `wfile` attribute of `StreamRequestHandler` classes now implements the `io.BufferedIOBase` writable interface. In particular, calling `write()` is now guaranteed to send the data in full. (Contributed by Martin Panter in bpo-26721 [https://bugs.python.org/issue?@action=redirect&bpo=26721].)

ssl

`ssl` supports OpenSSL 1.1.0. The minimum recommend version is 1.0.2. (Contributed by Christian Heimes in bpo-26470 [https://bugs.python.org/issue?@action=redirect&bpo=26470].)

3DES has been removed from the default cipher suites and ChaCha20 Poly1305 cipher suites have been added. (Contributed by Christian Heimes in bpo-27850 [https://bugs.python.org/issue?@action=redirect&bpo=27850] and bpo-27766 [https://bugs.python.org/issue?@action=redirect&bpo=27766].)

`SSLContext` has better default configuration for options and ciphers. (Contributed by Christian Heimes in bpo-28043 [https://bugs.python.org/issue?@action=redirect&bpo=28043].)

SSL session can be copied from one client-side connection to another with the new `SSLSession` class. TLS session resumption can speed up the initial handshake, reduce latency and improve performance (Contributed by Christian Heimes in bpo-19500 [https://bugs.python.org/issue?@action=redirect&bpo=19500] based on a draft by Alex Warhawk.)

The new `get_ciphers()` method can be used to get a list of enabled ciphers in order of cipher priority.

All constants and flags have been converted to `IntEnum` and `IntFlag`. (Contributed by Christian Heimes in bpo-28025 [https://bugs.python.org/issue?@action=redirect&bpo=28025].)

Server and client-side specific TLS protocols for `SSLContext` were added. (Contributed by Christian Heimes in bpo-28085 [https://bugs.python.org/issue?@action=redirect&bpo=28085].)

Added `ssl.SSLContext.post_handshake_auth` to enable and `ssl.SSLSocket.verify_client_post_handshake()` to initiate TLS 1.3 post-handshake authentication. (Contributed by Christian Heimes in gh-78851 [https://github.com/python/cpython/issues/78851].)

statistics

A new `harmonic_mean()` function has been added. (Contributed by Steven D'Aprano in bpo-27181 [https://bugs.python.org/issue?@action=redirect&bpo=27181].)

struct

`struct` now supports IEEE 754 half-precision floats via the `'e'` format specifier. (Contributed by Eli Stevens, Mark Dickinson in bpo-11734 [https://bugs.python.org/issue?@action=redirect&bpo=11734].)

subprocess

`subprocess.Popen` destructor now emits a `ResourceWarning` warning if the child process is still running. Use the context manager protocol (`with proc: ...`) or explicitly call the `wait()` method to read the exit status of the child process. (Contributed by Victor Stinner in bpo-26741 [https://bugs.python.org/issue?@action=redirect&bpo=26741].)

The `subprocess.Popen` constructor and all functions that pass arguments through to it now accept *encoding* and *errors* arguments. Specifying either of these will enable text mode for the *stdin*, *stdout* and *stderr* streams. (Contributed by Steve Dower in bpo-6135 [https://bugs.python.org/issue?@action=redirect&bpo=6135].)

sys

The new `getfilesystemencodeerrors()` function returns the name of the error mode used to convert between Unicode filenames and bytes filenames. (Contributed by Steve Dower in bpo-27781 [https://bugs.python.org/issue?@action=redirect&bpo=27781].)

On Windows the return value of the `getwindowsversion()` function now includes the *platform_version* field which contains the accurate major version, minor version and build number of the current operating system, rather than the version that is being emulated for the process (Contributed by Steve Dower in bpo-27932 [https://bugs.python.org/issue?@action=redirect&bpo=27932].)

telnetlib

`telnetlib.Telnet` is now a context manager (contributed by Stéphane Wirtel in bpo-25485 [https://bugs.python.org/issue?@action=redirect&bpo=25485]).

time

The `struct time` attributes **`tm_gmtoff`** and **`tm_zone`** are now available on all platforms.

timeit

The new `Timer.autorange()` convenience method has been added to call `Timer.timeit()` repeatedly so that the total run time is greater or equal to 200 milliseconds. (Contributed by Steven D'Aprano in bpo-6422 [https://bugs.python.org/issue?@action=redirect&bpo=6422].)

`timeit` now warns when there is substantial (4x) variance between best and worst times. (Contributed by Serhiy Storchaka in bpo-23552 [https://bugs.python.org/issue?@action=redirect&bpo=23552].)

tkinter

Added methods **`Variable.trace_add()`**, **`Variable.trace_remove()`** and **`trace_info()`** in the **`tkinter.Variable`** class. They replace old methods **`trace_variable()`**, **`trace()`**, **`trace_vdelete()`** and **`trace_vinfo()`** that use obsolete Tcl commands and might not work in future versions of Tcl. (Contributed by Serhiy Storchaka in bpo-22115 [https://bugs.python.org/issue?@action=redirect&bpo=22115]).

traceback

Both the traceback module and the interpreter's builtin exception display now abbreviate long sequences of repeated lines in tracebacks as shown in the following example:

```
>>> def f(): f()
...
>>> f()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 1, in f
  File "<stdin>", line 1, in f
  File "<stdin>", line 1, in f
  [Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded
```

(Contributed by Emanuel Barry in bpo-26823 [https://bugs.python.org/issue?@action=redirect&bpo=26823].)

tracemalloc

The **`tracemalloc`** module now supports tracing memory allocations in multiple different address spaces.

The new **`DomainFilter`** filter class has been added to filter block traces by their address space (domain).

(Contributed by Victor Stinner in bpo-26588 [https://bugs.python.org/issue?@action=redirect&bpo=26588].)

typing

Since the **`typing`** module is provisional, all changes introduced in Python 3.6 have also been backported to Python 3.5.x.

The **`typing`** module has a much improved support for generic type aliases. For example `Dict[str, Tuple[S, T]]` is now a valid type annotation.

(Contributed by Guido van Rossum in Github #195 [https://github.com/python/typing/pull/195].)

The `typing.ContextManager` class has been added for representing `contextlib.AbstractContextManager`. (Contributed by Brett Cannon in bpo-25609 [https://bugs.python.org/issue?@action=redirect&bpo=25609].)

The `typing.Collection` class has been added for representing `collections.abc.Collection`. (Contributed by Ivan Levkivskyi in bpo-27598 [https://bugs.python.org/issue?@action=redirect&bpo=27598].)

The `typing.ClassVar` type construct has been added to mark class variables. As introduced in **PEP 526** [https://peps.python.org/pep-0526/], a variable annotation wrapped in ClassVar indicates that a given attribute is intended to be used as a class variable and should not be set on instances of that class. (Contributed by Ivan Levkivskyi in Github #280 [https://github.com/python/typing/pull/280].)

A new `TYPE_CHECKING` constant that is assumed to be `True` by the static type checkers, but is `False` at runtime. (Contributed by Guido van Rossum in Github #230 [https://github.com/python/typing/issues/230].)

A new `NewType()` helper function has been added to create lightweight distinct types for annotations:

```
from typing import NewType

UserId = NewType('UserId', int)
some_id = UserId(524313)
```

The static type checker will treat the new type as if it were a subclass of the original type. (Contributed by Ivan Levkivskyi in Github #189 [https://github.com/python/typing/issues/189].)

unicodedata

The `unicodedata` module now uses data from Unicode 9.0.0 [https://unicode.org/versions/Unicode9.0.0/]. (Contributed by Benjamin Peterson.)

unittest.mock

The `Mock` class has the following improvements:

- Two new methods, `Mock.assert_called()` and `Mock.assert_called_once()` to check if the mock object was called. (Contributed by Amit Saha in bpo-26323 [https://bugs.python.org/issue?@action=redirect&bpo=26323].)
- The `Mock.reset_mock()` method now has two optional keyword only arguments: *return_value* and *side_effect*. (Contributed by Kushal Das in bpo-21271 [https://bugs.python.org/issue?@action=redirect&bpo=21271].)

urllib.request

If a HTTP request has a file or iterable body (other than a bytes object) but no `Content-Length` header, rather than throwing an error, `AbstractHTTPHandler` now falls back to use chunked transfer encoding. (Contributed by Demian Brecht and Rolf Krahl in bpo-12319 [https://bugs.python.org/issue?@action=redirect&bpo=12319].)

urllib.robotparser

`RobotFileParser` now supports the `Crawl-delay` and `Request-rate` extensions. (Contributed by Nikolay Bogoychev in bpo-16099 [https://bugs.python.org/issue?@action=redirect&bpo=16099].)

venv

`venv` accepts a new parameter `--prompt`. This parameter provides an alternative prefix for the virtual environment. (Proposed by Łukasz Balcerzak and ported to 3.6 by Stéphane Wirtel in bpo-22829 [https://bugs.python.org/issue?@action=redirect&bpo=22829].)

avisos

A new optional *source* parameter has been added to the `warnings.warn_explicit()` function: the destroyed object which emitted a

ResourceWarning. A *source* attribute has also been added to **warnings.WarningMessage** (contributed by Victor Stinner in bpo-26568 [https://bugs.python.org/issue?@action=redirect&bpo=26568] and bpo-26567 [https://bugs.python.org/issue?@action=redirect&bpo=26567]).

When a **ResourceWarning** warning is logged, the **tracemalloc** module is now used to try to retrieve the traceback where the destroyed object was allocated.

Example with the script `example.py`:

```
import warnings

def func():
    return open(__file__)

f = func()
f = None
```

Output of the command `python3.6 -Wd -X tracemalloc=5 example.py`:

```
example.py:7: ResourceWarning: unclosed file <_io.TextIOWrapper
  f = None
Object allocated at (most recent call first):
  File "example.py", lineno 4
    return open(__file__)
  File "example.py", lineno 6
    f = func()
```

The “Object allocated at” traceback is new and is only displayed if **tracemalloc** is tracing Python memory allocations and if the **warnings** module was already imported.

winreg

Added the 64-bit integer type **REG_QWORD**. (Contributed by Clement Rouault in bpo-23026 [https://bugs.python.org/issue?@action=redirect&bpo=23026].)

winsound

Allowed keyword arguments to be passed to `Beep`, `MessageBeep`, and `PlaySound` (bpo-27982 [https://bugs.python.org/issue?@action=redirect&bpo=27982]).

xmlrpc.client

The `xmlrpc.client` module now supports unmarshalling additional data types used by the Apache XML-RPC implementation for numerics and `None`. (Contributed by Serhiy Storchaka in bpo-26885 [https://bugs.python.org/issue?@action=redirect&bpo=26885].)

zipfile

A new `ZipInfo.from_file()` class method allows making a `ZipInfo` instance from a filesystem file. A new `ZipInfo.is_dir()` method can be used to check if the `ZipInfo` instance represents a directory. (Contributed by Thomas Kluyver in bpo-26039 [https://bugs.python.org/issue?@action=redirect&bpo=26039].)

The `ZipFile.open()` method can now be used to write data into a ZIP file, as well as for extracting data. (Contributed by Thomas Kluyver in bpo-26039 [https://bugs.python.org/issue?@action=redirect&bpo=26039].)

zlib

The `compress()` and `decompress()` functions now accept keyword arguments. (Contributed by Aviv Palivoda in bpo-26243 [https://bugs.python.org/issue?@action=redirect&bpo=26243] and Xiang Zhang in bpo-16764 [https://bugs.python.org/issue?@action=redirect&bpo=16764] respectively.)

Otimizações

- The Python interpreter now uses a 16-bit wordcode instead of bytecode which made a number of opcode optimizations possible. (Contributed by Demur Rumed with input and reviews from Serhiy Storchaka and Victor Stinner in bpo-26647 [https://bugs.python.org/issue?

@action=redirect&bpo=26647] and bpo-28050 [https://bugs.python.org/issue?@action=redirect&bpo=28050].)

- The **`asyncio.Future`** class now has an optimized C implementation. (Contributed by Yury Selivanov and INADA Naoki in bpo-26081 [https://bugs.python.org/issue?@action=redirect&bpo=26081].)
- The **`asyncio.Task`** class now has an optimized C implementation. (Contributed by Yury Selivanov in bpo-28544 [https://bugs.python.org/issue?@action=redirect&bpo=28544].)
- Various implementation improvements in the **`typing`** module (such as caching of generic types) allow up to 30 times performance improvements and reduced memory footprint.
- The ASCII decoder is now up to 60 times as fast for error handlers `surrogateescape`, `ignore` and `replace` (Contributed by Victor Stinner in bpo-24870 [https://bugs.python.org/issue?@action=redirect&bpo=24870]).
- The ASCII and the Latin1 encoders are now up to 3 times as fast for the error handler `surrogateescape` (Contributed by Victor Stinner in bpo-25227 [https://bugs.python.org/issue?@action=redirect&bpo=25227]).
- The UTF-8 encoder is now up to 75 times as fast for error handlers `ignore`, `replace`, `surrogateescape`, `surrogatepass` (Contributed by Victor Stinner in bpo-25267 [https://bugs.python.org/issue?@action=redirect&bpo=25267]).
- The UTF-8 decoder is now up to 15 times as fast for error handlers `ignore`, `replace` and `surrogateescape` (Contributed by Victor Stinner in bpo-25301 [https://bugs.python.org/issue?@action=redirect&bpo=25301]).
- `bytes % args` is now up to 2 times faster. (Contributed by Victor Stinner in bpo-25349 [https://bugs.python.org/issue?@action=redirect&bpo=25349]).
- `bytearray % args` is now between 2.5 and 5 times faster. (Contributed by Victor Stinner in bpo-25399 [https://bugs.python.org/issue?@action=redirect&bpo=25399]).
- Optimize **`bytes.fromhex()`** and **`bytearray.fromhex()`**: they are now between 2x and 3.5x faster. (Contributed by Victor Stinner in bpo-25401 [https://bugs.python.org/issue?@action=redirect&bpo=25401]).
- Optimize `bytes.replace(b'', b'.')` and `bytearray.replace(b'', b'.')`: up to 80% faster. (Contributed by Josh Snider in bpo-26574 [https://bugs.python.org/issue?@action=redirect&bpo=26574]).
- Allocator functions of the **`PyMem_Malloc()`** domain (**`PYMEM_DOMAIN_MEM`**) now use the pymalloc memory allocator instead

of **`malloc()`** function of the C library. The pymalloc allocator is optimized for objects smaller or equal to 512 bytes with a short lifetime, and use **`malloc()`** for larger memory blocks. (Contributed by Victor Stinner in bpo-26249 [https://bugs.python.org/issue?@action=redirect&bpo=26249]).

- **`pickle.load()`** and **`pickle.loads()`** are now up to 10% faster when deserializing many small objects (Contributed by Victor Stinner in bpo-27056 [https://bugs.python.org/issue?@action=redirect&bpo=27056]).
- Passing keyword arguments to a function has an overhead in comparison with passing positional arguments. Now in extension functions implemented with using Argument Clinic this overhead is significantly decreased. (Contributed by Serhiy Storchaka in bpo-27574 [https://bugs.python.org/issue?@action=redirect&bpo=27574]).
- Optimized **`glob()`** and **`iglob()`** functions in the **`glob`** module; they are now about 3–6 times faster. (Contributed by Serhiy Storchaka in bpo-25596 [https://bugs.python.org/issue?@action=redirect&bpo=25596]).
- Optimized globbing in **`pathlib`** by using **`os.scandir()`**; it is now about 1.5–4 times faster. (Contributed by Serhiy Storchaka in bpo-26032 [https://bugs.python.org/issue?@action=redirect&bpo=26032]).
- **`xml.etree.ElementTree`** parsing, iteration and deepcopy performance has been significantly improved. (Contributed by Serhiy Storchaka in bpo-25638 [https://bugs.python.org/issue?@action=redirect&bpo=25638], bpo-25873 [https://bugs.python.org/issue?@action=redirect&bpo=25873], and bpo-25869 [https://bugs.python.org/issue?@action=redirect&bpo=25869].)
- Creation of **`fractions.Fraction`** instances from floats and decimals is now 2 to 3 times faster. (Contributed by Serhiy Storchaka in bpo-25971 [https://bugs.python.org/issue?@action=redirect&bpo=25971].)

Mudanças a construções e API C

- Python now requires some C99 support in the toolchain to build. Most notably, Python now uses standard integer types and macros in place of custom macros like `PY_LONG_LONG`. For more information, see **PEP 7** [https://peps.python.org/pep-0007/] and bpo-17884 [https://bugs.python.org/issue?@action=redirect&bpo=17884].
- Cross-compiling CPython with the Android NDK and the Android API level set to 21 (Android 5.0 Lollipop) or greater runs successfully.

While Android is not yet a supported platform, the Python test suite runs on the Android emulator with only about 16 tests failures. See the Android meta-issue bpo-26865 [https://bugs.python.org/issue?@action=redirect&bpo=26865].

- The `--enable-optimizations` configure flag has been added. Turning it on will activate expensive optimizations like PGO. (Original patch by Alecsandru Patrascu of Intel in bpo-26359 [https://bugs.python.org/issue?@action=redirect&bpo=26359].)
- The GIL must now be held when allocator functions of `PYMEM_DOMAIN_OBJ` (ex: `PyObject_Malloc()`) and `PYMEM_DOMAIN_MEM` (ex: `PyMem_Malloc()`) domains are called.
- New `Py_FinalizeEx()` API which indicates if flushing buffered data failed. (Contributed by Martin Panter in bpo-5319 [https://bugs.python.org/issue?@action=redirect&bpo=5319].)
- `PyArg_ParseTupleAndKeywords()` now supports positional-only parameters. Positional-only parameters are defined by empty names. (Contributed by Serhiy Storchaka in bpo-26282 [https://bugs.python.org/issue?@action=redirect&bpo=26282]).
- `PyTraceback_Print` method now abbreviates long sequences of repeated lines as `"[Previous line repeated {count} more times]"`. (Contributed by Emanuel Barry in bpo-26823 [https://bugs.python.org/issue?@action=redirect&bpo=26823].)
- The new `PyErr_SetImportErrorSubclass()` function allows for specifying a subclass of `ImportError` to raise. (Contributed by Eric Snow in bpo-15767 [https://bugs.python.org/issue?@action=redirect&bpo=15767].)
- The new `PyErr_ResourceWarning()` function can be used to generate a `ResourceWarning` providing the source of the resource allocation. (Contributed by Victor Stinner in bpo-26567 [https://bugs.python.org/issue?@action=redirect&bpo=26567].)
- The new `PyOS_FSPath()` function returns the file system representation of a path-like object. (Contributed by Brett Cannon in bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186].)
- The `PyUnicode_FSConverter()` and `PyUnicode_FSDecoder()` functions will now accept path-like objects.

Other Improvements

- When `--version` (short form: `-V`) is supplied twice, Python prints `sys.version` for detailed information.

```
$ ./python -VV
Python 3.6.0b4+ (3.6:223967b49e49+, Nov 21 2016, 20:55:04)
[GCC 4.2.1 Compatible Apple LLVM 8.0.0 (clang-800.0.42.1)]
```

Descontinuados

New Keywords

`async` and `await` are not recommended to be used as variable, class, function or module names. Introduced by **PEP 492** [https://peps.python.org/pep-0492/] in Python 3.5, they will become proper keywords in Python 3.7. Starting in Python 3.6, the use of `async` or `await` as names will generate a `DeprecationWarning`.

Deprecated Python behavior

Raising the `StopIteration` exception inside a generator will now generate a `DeprecationWarning`, and will trigger a `RuntimeError` in Python 3.7. See PEP 479: Change StopIteration handling inside generators for details.

The `__aiter__()` method is now expected to return an asynchronous iterator directly instead of returning an awaitable as previously. Doing the former will trigger a `DeprecationWarning`. Backward compatibility will be removed in Python 3.7. (Contributed by Yury Selivanov in bpo-27243 [https://bugs.python.org/issue?@action=redirect&bpo=27243].)

A backslash-character pair that is not a valid escape sequence now generates a `DeprecationWarning`. Although this will eventually become a `SyntaxError`, that will not be for several Python releases. (Contributed by Emanuel Barry in bpo-27364 [https://bugs.python.org/issue?@action=redirect&bpo=27364].)

When performing a relative import, falling back on `__name__` and `__path__` from the calling module when `__spec__` or `__package__` are not defined

now raises an **`ImportWarning`**. (Contributed by Rose Ames in bpo-25791 [https://bugs.python.org/issue?@action=redirect&bpo=25791].)

Deprecated Python modules, functions and methods

asynchat

The **`asynchat`** has been deprecated in favor of **`asyncio`**. (Contributed by Mariatta in bpo-25002 [https://bugs.python.org/issue?@action=redirect&bpo=25002].)

asyncore

The **`asyncore`** has been deprecated in favor of **`asyncio`**. (Contributed by Mariatta in bpo-25002 [https://bugs.python.org/issue?@action=redirect&bpo=25002].)

dbm

Unlike other **`dbm`** implementations, the **`dbm.dumb`** module creates databases with the `'rw'` mode and allows modifying the database opened with the `'r'` mode. This behavior is now deprecated and will be removed in 3.8. (Contributed by Serhiy Storchaka in bpo-21708 [https://bugs.python.org/issue?@action=redirect&bpo=21708].)

distutils

The undocumented `extra_path` argument to the `distutils.Distribution` constructor is now considered deprecated and will raise a warning if set. Support for this parameter will be removed in a future Python release. See bpo-27919 [https://bugs.python.org/issue?@action=redirect&bpo=27919] for details.

grp

The support of non-integer arguments in **`getgrgid()`** has been deprecated. (Contributed by Serhiy Storchaka in bpo-26129 [https://bugs.python.org/issue?@action=redirect&bpo=26129].)

importlib

The `importlib.machinery.SourceFileLoader.load_module` and `importlib.machinery.SourcelessFileLoader.load_module` methods are now deprecated. They were the only remaining implementations of `importlib.abc.Loader.load_module` in **`importlib`** that had not been deprecated in previous versions of Python in favour of **`importlib.abc.Loader.exec_module()`**.

The **`importlib.machinery.WindowsRegistryFinder`** class is now deprecated. As of 3.6.0, it is still added to **`sys.meta_path`** by default (on Windows), but this may change in future releases.

os

Undocumented support of general bytes-like objects as paths in **`os`** functions, **`compile()`** and similar functions is now deprecated. (Contributed by Serhiy Storchaka in bpo-25791 [https://bugs.python.org/issue?@action=redirect&bpo=25791] and bpo-26754 [https://bugs.python.org/issue?@action=redirect&bpo=26754].)

re

Support for inline flags `(?letters)` in the middle of the regular expression has been deprecated and will be removed in a future Python version. Flags at the start of a regular expression are still allowed. (Contributed by Serhiy Storchaka in bpo-22493 [https://bugs.python.org/issue?@action=redirect&bpo=22493].)

ssl

OpenSSL 0.9.8, 1.0.0 and 1.0.1 are deprecated and no longer supported. In the future the **`ssl`** module will require at least OpenSSL 1.0.2 or 1.1.0.

SSL-related arguments like `certfile`, `keyfile` and `check_hostname` in **`ftplib`**, **`http.client`**, **`imaplib`**, **`poplib`**, and **`smtplib`** have been deprecated

in favor of `context`. (Contributed by Christian Heimes in bpo-28022 [https://bugs.python.org/issue?@action=redirect&bpo=28022].)

A couple of protocols and functions of the **`ssl`** module are now deprecated. Some features will no longer be available in future versions of OpenSSL. Other features are deprecated in favor of a different API. (Contributed by Christian Heimes in bpo-28022 [https://bugs.python.org/issue?@action=redirect&bpo=28022] and bpo-26470 [https://bugs.python.org/issue?@action=redirect&bpo=26470].)

tkinter

The **`tkinter.tix`** module is now deprecated. **`tkinter`** users should use **`tkinter.ttk`** instead.

venv

The `pyvenv` script has been deprecated in favour of `python3 -m venv`. This prevents confusion as to what Python interpreter `pyvenv` is connected to and thus what Python interpreter will be used by the virtual environment. (Contributed by Brett Cannon in bpo-25154 [https://bugs.python.org/issue?@action=redirect&bpo=25154].)

xml

- As mitigation against DTD and external entity retrieval, the **`xml.dom.minidom`** and **`xml.sax`** modules no longer process external entities by default. (Contributed by Christian Heimes in gh-61441 [https://github.com/python/cpython/issues/61441].)

Deprecated functions and types of the C API

Undocumented functions **`PyUnicode_AsEncodedObject()`**, **`PyUnicode_AsDecodedObject()`**, **`PyUnicode_AsEncodedUnicode()`** and **`PyUnicode_AsDecodedUnicode()`** are deprecated now. Use the generic codec based API instead.

Deprecated Build Options

The `--with-system-ffi` configure flag is now on by default on non-macOS UNIX platforms. It may be disabled by using `--without-system-ffi`, but using the flag is deprecated and will not be accepted in Python 3.7. macOS is unaffected by this change. Note that many OS distributors already use the `--with-system-ffi` flag when building their system Python.

Removidos

Remoção de APIs e recursos

- Unknown escapes consisting of `'\'` and an ASCII letter in regular expressions will now cause an error. In replacement templates for `re.sub()` they are still allowed, but deprecated. The `re.LOCALE` flag can now only be used with binary patterns.
- `inspect.getmoduleinfo()` was removed (was deprecated since CPython 3.3). `inspect.getmodulename()` should be used for obtaining the module name for a given path. (Contributed by Yury Selivanov in bpo-13248 [https://bugs.python.org/issue?@action=redirect&bpo=13248].)
- `traceback.Ignore` class and `traceback.usage`, `traceback.modname`, `traceback.fullmodname`, `traceback.find_lines_from_code`, `traceback.find_lines`, `traceback.find_strings`, `traceback.find_executable_lines` methods were removed from the `traceback` module. They were undocumented methods deprecated since Python 3.2 and equivalent functionality is available from private methods.
- The `tk_menuBar()` and `tk_bindForTraversal()` dummy methods in `tkinter` widget classes were removed (corresponding Tk commands were obsolete since Tk 4.0).
- The `open()` method of the `zipfile.ZipFile` class no longer supports the `'U'` mode (was deprecated since Python 3.4). Use `io.TextIOWrapper` for reading compressed text files in universal newlines mode.
- The undocumented `IN`, `CDROM`, `DLFCN`, `TYPES`, `CDIO`, and `STROPTS` modules have been removed. They had been available in the platform

specific `Lib/plat-*/` directories, but were chronically out of date, inconsistently available across platforms, and unmaintained. The script that created these modules is still available in the source distribution at Tools/scripts/h2py.py [https://github.com/python/cpython/blob/v3.6.15/Tools/scripts/h2py.py].

- The deprecated `asynchat.fifo` class has been removed.

Porting to Python 3.6

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Changes in 'python' Command Behavior

- The output of a special Python build with defined `COUNT_ALLOCS`, `SHOW_ALLOC_COUNT` or `SHOW_TRACK_COUNT` macros is now off by default. It can be re-enabled using the `-X showalloccount` option. It now outputs to `stderr` instead of `stdout`. (Contributed by Serhiy Storchaka in bpo-23034 [https://bugs.python.org/issue?@action=redirect&bpo=23034].)

Alterações na API Python

- **`open()`** will no longer allow combining the `'U'` mode flag with `'+'`. (Contributed by Jeff Balogh and John O'Connor in bpo-2091 [https://bugs.python.org/issue?@action=redirect&bpo=2091].)

- **`sqlite3`** no longer implicitly commits an open transaction before DDL statements.

- On Linux, **`os.urandom()`** now blocks until the system urandom entropy pool is initialized to increase the security.

- When **`importlib.abc.Loader.exec_module()`** is defined, **`importlib.abc.Loader.create_module()`** must also be defined.

- `PyErr_SetImportError()` now sets `TypeError` when its **msg** argument is not set. Previously only `NULL` was returned.

- The format of the **`codeobject.co_lnotab`** attribute of code objects changed to support a negative line number delta. By default, Python does not emit bytecode with a negative line number delta. Functions using `frame.f_lineno`, `PyFrame_GetLineNumber()` or `PyCode_Addr2Line()` are not affected. Functions directly decoding **`co_lnotab`** should be updated to use a signed 8-bit integer type for the line number delta, but this is only required to support applications using a negative line number delta. See `Objects/lnotab_notes.txt` for the **`co_lnotab`** format and how to decode it, and see the **PEP 511** [https://peps.python.org/pep-0511/] for the rationale.

- The functions in the `compileall` module now return booleans instead of `1` or `0` to represent success or failure, respectively. Thanks to booleans being a subclass of integers, this should only be an issue if you were doing identity checks for `1` or `0`. See bpo-25768 [https://bugs.python.org/issue?@action=redirect&bpo=25768].

- Reading the **`port`** attribute of `urllib.parse.urlsplit()` and `urlparse()` results now raises `ValueError` for out-of-range values, rather than returning `None`. See bpo-20059 [https://bugs.python.org/issue?@action=redirect&bpo=20059].

- The **`imp`** module now raises a `DeprecationWarning` instead of `PendingDeprecationWarning`.

- The following modules have had missing APIs added to their `__all__` attributes to match the documented APIs: `calendar`, **`cgi`**, `csv`, `ElementTree`, `enum`, `fileinput`, `ftplib`, `logging`, `mailbox`, `mimetypes`, `optparse`, `plistlib`, **`smtpd`**, `subprocess`, `tarfile`, `threading` and `wave`. This means they will export new symbols when `import *` is used. (Contributed by Joel Taddei and Jacek Kołodziej in bpo-23883 [https://bugs.python.org/issue?@action=redirect&bpo=23883].)

- When performing a relative import, if `__package__` does not compare equal to `__spec__.parent` then `ImportWarning` is raised. (Contributed

by Brett Cannon in bpo-25791 [https://bugs.python.org/issue?@action=redirect&bpo=25791].)

- When a relative import is performed and no parent package is known, then `ImportError` will be raised. Previously, `SystemError` could be raised. (Contributed by Brett Cannon in bpo-18018 [https://bugs.python.org/issue?@action=redirect&bpo=18018].)

- Servers based on the `socketserver` module, including those defined in `http.server`, `xmlrpc.server` and `wsgiref.simple_server`, now only catch exceptions derived from `Exception`. Therefore if a request handler raises an exception like `SystemExit` or `KeyboardInterrupt`, `handle_error()` is no longer called, and the exception will stop a single-threaded server. (Contributed by Martin Panter in bpo-23430 [https://bugs.python.org/issue?@action=redirect&bpo=23430].)

- **`spwd.getspnam()`** now raises a `PermissionError` instead of `KeyError` if the user doesn't have privileges.

- The `socket.socket.close()` method now raises an exception if an error (e.g. `EBADF`) was reported by the underlying system call. (Contributed by Martin Panter in bpo-26685 [https://bugs.python.org/issue?@action=redirect&bpo=26685].)

- The *decode_data* argument for the **`smtpd.SMTPChannel`** and **`smtpd.SMTPServer`** constructors is now `False` by default. This means that the argument passed to **`process_message()`** is now a bytes object by default, and **`process_message()`** will be passed keyword arguments. Code that has already been updated in accordance with the deprecation warning generated by 3.5 will not be affected.

- All optional arguments of the `dump()`, `dumps()`, `load()` and `loads()` functions and `JSONEncoder` and `JSONDecoder` class constructors in the `json` module are now keyword-only. (Contributed by Serhiy Storchaka in bpo-18726 [https://bugs.python.org/issue?@action=redirect&bpo=18726].)

- Subclasses de `type` que não substituem `type.__new__` não podem mais usar a forma com apenas um argumento para obter o tipo de um objeto.

- As part of **PEP 487** [https://peps.python.org/pep-0487/], the handling of keyword arguments passed to `type` (other than the metaclass hint, `metaclass`) is now consistently delegated to `object.__init_subclass__()`. This means that `type.__new__` and `type.__init__` both now accept arbitrary keyword arguments, but **`object.__init_subclass__()`** (which is called from **`type.__new__`**) will reject them by default. Custom metaclasses accepting additional keyword arguments will need to adjust their calls to **`type.__new__`** (whether direct or via `super`) accordingly.

- In `distutils.command.sdist.sdist`, the `default_format` attribute has been removed and is no longer honored. Instead, the gzipped tarfile format is the default on all platforms and no platform-specific selection is made. In environments where distributions are built on Windows and zip distributions are required, configure the project with a `setup.cfg` file containing the following:

  ```
  [sdist]
  formats=zip
  ```

  This behavior has also been backported to earlier Python versions by Setuptools 26.0.0.

- In the `urllib.request` module and the `http.client.HTTPConnection.request()` method, if no Content-Length header field has been specified and the request body is a file object, it is now sent with HTTP 1.1 chunked encoding. If a file object has to be sent to a HTTP 1.0 server, the Content-Length value now has to be specified by the caller. (Contributed by Demian Brecht and Rolf Krahl with tweaks from Martin Panter in bpo-12319 [https://bugs.python.org/issue?@action=redirect&bpo=12319].)

- The `DictReader` now returns rows of type `OrderedDict`. (Contributed by Steve Holden in bpo-27842 [https://bugs.python.org/issue?@action=redirect&bpo=27842].)

- The **`crypt.METHOD_CRYPT`** will no longer be added to `crypt.methods` if unsupported by the platform. (Contributed by Victor Stinner in bpo-

25287 [https://bugs.python.org/issue?@action=redirect&bpo=25287].)

- The *verbose* and *rename* arguments for `namedtuple()` are now keyword-only. (Contributed by Raymond Hettinger in bpo-25628 [https://bugs.python.org/issue?@action=redirect&bpo=25628].)

- On Linux, `ctypes.util.find_library()` now looks in `LD_LIBRARY_PATH` for shared libraries. (Contributed by Vinay Sajip in bpo-9998 [https://bugs.python.org/issue?@action=redirect&bpo=9998].)

- The `imaplib.IMAP4` class now handles flags containing the `']'` character in messages sent from the server to improve real-world compatibility. (Contributed by Lita Cho in bpo-21815 [https://bugs.python.org/issue?@action=redirect&bpo=21815].)

- The `mmap.mmap.write()` function now returns the number of bytes written like other write methods. (Contributed by Jakub Stasiak in bpo-26335 [https://bugs.python.org/issue?@action=redirect&bpo=26335].)

- The `pkgutil.iter_modules()` and `pkgutil.walk_packages()` functions now return `ModuleInfo` named tuples. (Contributed by Ramchandra Apte in bpo-17211 [https://bugs.python.org/issue?@action=redirect&bpo=17211].)

- `re.sub()` now raises an error for invalid numerical group references in replacement templates even if the pattern is not found in the string. The error message for invalid group references now includes the group index and the position of the reference. (Contributed by SilentGhost, Serhiy Storchaka in bpo-25953 [https://bugs.python.org/issue?@action=redirect&bpo=25953].)

- `zipfile.ZipFile` will now raise `NotImplementedError` for unrecognized compression values. Previously a plain `RuntimeError` was raised. Additionally, calling `ZipFile` methods on a closed ZipFile or calling the `write()` method on a ZipFile created with mode `'r'` will raise a `ValueError`. Previously, a `RuntimeError` was raised in those scenarios.

- when custom metaclasses are combined with zero-argument `super()` or direct references from methods to the implicit `__class__` closure variable, the implicit `__classcell__` namespace entry must now be passed up to `type.__new__` for initialisation. Failing to do so will result in a `DeprecationWarning` in Python 3.6 and a `RuntimeError` in Python 3.8.

- With the introduction of `ModuleNotFoundError`, import system consumers may start expecting import system replacements to raise that more specific exception when appropriate, rather than the less-specific `ImportError`. To provide future compatibility with such consumers, implementers of alternative import systems that completely replace `__import__()` will need to update their implementations to raise the new subclass when a module can't be found at all. Implementers of compliant plugins to the default import system shouldn't need to make any changes, as the default import system will raise the new subclass when appropriate.

Alterações na API C

- The `PyMem_Malloc()` allocator family now uses the pymalloc allocator rather than the system **`malloc()`**. Applications calling **`PyMem_Malloc()`** without holding the GIL can now crash. Set the `PYTHONMALLOC` environment variable to `debug` to validate the usage of memory allocators in your application. See bpo-26249 [https://bugs.python.org/issue?@action=redirect&bpo=26249].
- `Py_Exit()` (and the main interpreter) now override the exit status with 120 if flushing buffered data failed. See bpo-5319 [https://bugs.python.org/issue?@action=redirect&bpo=5319].

Alterações de bytecode do CPython

There have been several major changes to the bytecode in Python 3.6.

- The Python interpreter now uses a 16-bit wordcode instead of bytecode. (Contributed by Demur Rumed with input and reviews from Serhiy Storchaka and Victor Stinner in bpo-26647

[https://bugs.python.org/issue?@action=redirect&bpo=26647] and bpo-28050 [https://bugs.python.org/issue?@action=redirect&bpo=28050].)

- The new **FORMAT_VALUE** and **BUILD_STRING** opcodes as part of the formatted string literal implementation. (Contributed by Eric Smith in bpo-25483 [https://bugs.python.org/issue?@action=redirect&bpo=25483] and Serhiy Storchaka in bpo-27078 [https://bugs.python.org/issue?@action=redirect&bpo=27078].)
- The new **BUILD_CONST_KEY_MAP** opcode to optimize the creation of dictionaries with constant keys. (Contributed by Serhiy Storchaka in bpo-27140 [https://bugs.python.org/issue?@action=redirect&bpo=27140].)
- The function call opcodes have been heavily reworked for better performance and simpler implementation. The **MAKE_FUNCTION**, **CALL_FUNCTION**, **CALL_FUNCTION_KW** and **BUILD_MAP_UNPACK_WITH_CALL** opcodes have been modified, the new **CALL_FUNCTION_EX** and **BUILD_TUPLE_UNPACK_WITH_CALL** have been added, and `CALL_FUNCTION_VAR`, `CALL_FUNCTION_VAR_KW` and `MAKE_CLOSURE` opcodes have been removed. (Contributed by Demur Rumed in bpo-27095 [https://bugs.python.org/issue?@action=redirect&bpo=27095], and Serhiy Storchaka in bpo-27213 [https://bugs.python.org/issue?@action=redirect&bpo=27213], bpo-28257 [https://bugs.python.org/issue?@action=redirect&bpo=28257].)
- The new **SETUP_ANNOTATIONS** and **STORE_ANNOTATION** opcodes have been added to support the new variable annotation syntax. (Contributed by Ivan Levkivskyi in bpo-27985 [https://bugs.python.org/issue?@action=redirect&bpo=27985].)

Notable changes in Python 3.6.2

Novo alvo de construção `make regen-all`

Para simplificar a compilação cruzada e garantir que o CPython possa ser compilado de forma confiável sem exigir que uma versão existente do Python já esteja disponível, o sistema de construção baseado em autotools não tenta mais recompilar implicitamente os arquivos gerados com base nos tempos de modificação dos arquivos.

Em vez disso, um novo comando `make regen-all` foi adicionado para forçar a regeneração desses arquivos quando desejado (por exemplo, após uma versão inicial do Python já ter sido construída com base nas versões pré-geradas).

Alvos de regeneração mais seletivos também são definidos – veja Makefile.pre.in [https://github.com/python/cpython/tree/3.15/Makefile.pre.in] para detalhes.

(Contribuição de Victor Stinner em bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404].)

Adicionado na versão 3.6.2.

Remoção do alvo de construção `make touch`

O alvo de construção `make touch` usado anteriormente para solicitar regeneração implícita de arquivos gerados atualizando seus tempos de modificação foi removido.

Ele foi substituído pelo novo alvo `make regen-all`.

(Contribuição de Victor Stinner em bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404].)

Alterado na versão 3.6.2.

Notable changes in Python 3.6.4

The `PyExc_RecursionErrorInst` singleton that was part of the public API has been removed as its members being never cleared may cause a segfault during finalization of the interpreter. (Contributed by Xavier de Gaye in bpo-22898 [https://bugs.python.org/issue?@action=redirect&bpo=22898] and bpo-30697 [https://bugs.python.org/issue?@action=redirect&bpo=30697].)

Notable changes in Python 3.6.5

The **locale.localeconv()** function now sets temporarily the `LC_CTYPE` locale to the `LC_NUMERIC` locale in some cases. (Contributed by Victor Stinner in bpo-31900 [https://bugs.python.org/issue?@action=redirect&bpo=31900].)

Notable changes in Python 3.6.7

xml.dom.minidom and **xml.sax** modules no longer process external entities by default. See also gh-61441 [https://github.com/python/cpython/issues/61441].

In 3.6.7 the **tokenize** module now implicitly emits a `NEWLINE` token when provided with input that does not have a trailing new line. This behavior now matches what the C tokenizer does internally. (Contributed by Ammar Askar in bpo-33899 [https://bugs.python.org/issue?@action=redirect&bpo=33899].)

Notable changes in Python 3.6.10

Devido a significativas preocupações de segurança, o parâmetro *reuse_address* de **asyncio.loop.create_datagram_endpoint()** não é mais suportado. Isso ocorre devido ao comportamento da opção de soquete `SO_REUSEADDR` no UDP. Para mais detalhes, consulte a documentação de `loop.create_datagram_endpoint()`. (Contribuição de Kyle Stanley, Antoine Pitrou e Yury Selivanov em bpo-37228 [https://bugs.python.org/issue?@action=redirect&bpo=37228].)

Notable changes in Python 3.6.13

Versões anteriores do Python permitiam o uso de `;` e `&` como separadores de parâmetros de consulta nas funções **urllib.parse.parse_qs()** e **urllib.parse.parse_qsl()**. Devido a questões de segurança, e em conformidade com as recomendações mais recentes do W3C, isso foi alterado para permitir apenas uma única chave separadora, com `&` como padrão. Esta mudança também afeta as funções **cgi.parse()** e **cgi.parse_multipart()** já que elas usam as funções afetadas internamente. Para obter mais detalhes, consulte sua respectiva documentação. (Contribuição de Adam Goldschmidt, Senthil Kumaran e Ken Jin em bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967].)

Notable changes in Python 3.6.14

Uma correção de segurança altera o comportamento **`ftplib.FTP`** para não confiar no endereço IPv4 enviado do servidor remoto ao configurar um canal de dados passivo. Em vez disso, reutilizamos o endereço IP do servidor ftp. Para códigos incomuns que requerem o comportamento antigo, defina um atributo `trust_server_pasv_ipv4_address` em sua instância de FTP para `True`. (Veja gh-87451 [https://github.com/python/cpython/issues/87451])

The presence of newline or tab characters in parts of a URL allows for some forms of attacks. Following the WHATWG specification that updates RFC 3986, ASCII newline `\n`, `\r` and tab `\t` characters are stripped from the URL by the parser **`urllib.parse()`** preventing such attacks. The removal characters are controlled by a new module level variable `urllib.parse._UNSAFE_URL_BYTES_TO_REMOVE`. (See gh-88048 [https://github.com/python/cpython/issues/88048])

O que há de novo no Python 3.5

Editores:: Elvis Pranskevichus <elvis@magic.io>, Yury Selivanov <yury@magic.io>

This article explains the new features in Python 3.5, compared to 3.4. Python 3.5 was released on September 13, 2015. See the changelog [https://docs.python.org/3.5/whatsnew/changelog.html] for a full list of changes.

Ver também

PEP 478 [https://peps.python.org/pep-0478/] - Python 3.5 Release Schedule

Resumo – Destaques da versão

Novos recursos de sintaxe:

- PEP 492, coroutines with async and await syntax.
- PEP 465, a new matrix multiplication operator: `a @ b`.
- PEP 448, additional unpacking generalizations.

Novos módulos de biblioteca:

- **`typing`**: PEP 484 – Type Hints.
- **`zipapp`**: PEP 441 Improving Python ZIP Application Support.

Novos recursos embutidos:

- `bytes % args`, `bytearray % args`: PEP 461 – Adding `%` formatting to bytes and bytearray.
- New **`bytes.hex()`**, **`bytearray.hex()`** and **`memoryview.hex()`** methods. (Contributed by Arnon Yaari in bpo-9951 [https://bugs.python.org/issue?@action=redirect&bpo=9951].)

- **memoryview** now supports tuple indexing (including multi-dimensional). (Contributed by Antoine Pitrou in bpo-23632 [https://bugs.python.org/issue?@action=redirect&bpo=23632].)
- Generators have a new `gi_yieldfrom` attribute, which returns the object being iterated by `yield from` expressions. (Contributed by Benno Leslie and Yury Selivanov in bpo-24450 [https://bugs.python.org/issue?@action=redirect&bpo=24450].)
- A new **RecursionError** exception is now raised when maximum recursion depth is reached. (Contributed by Georg Brandl in bpo-19235 [https://bugs.python.org/issue?@action=redirect&bpo=19235].)

Melhorias na implementação do CPython:

- When the `LC_TYPE` locale is the POSIX locale (`C` locale), **sys.stdin** and **sys.stdout** now use the `surrogateescape` error handler, instead of the `strict` error handler. (Contributed by Victor Stinner in bpo-19977 [https://bugs.python.org/issue?@action=redirect&bpo=19977].)
- `.pyo` files are no longer used and have been replaced by a more flexible scheme that includes the optimization level explicitly in `.pyc` name. (See PEP 488 overview.)
- Builtin and extension modules are now initialized in a multi-phase process, which is similar to how Python modules are loaded. (See PEP 489 overview.)

Melhorias significativas na biblioteca padrão:

- **collections.OrderedDict** is now implemented in C, which makes it 4 to 100 times faster.
- The **ssl** module gained support for Memory BIO, which decouples SSL protocol handling from network IO.
- The new **os.scandir()** function provides a better and significantly faster way of directory traversal.
- **@functools.lru_cache** has been mostly reimplemented in C, yielding much better performance.
- The new **subprocess.run()** function provides a streamlined way to run subprocesses.
- The **traceback** module has been significantly enhanced for improved performance and developer convenience.

Melhorias de Segurança:

- SSLv3 is now disabled throughout the standard library. It can still be enabled by instantiating a `ssl.SSLContext` manually. (See bpo-22638 [https://bugs.python.org/issue?@action=redirect&bpo=22638] for more details; this change was backported to CPython 3.4 and 2.7.)
- HTTP cookie parsing is now stricter, in order to protect against potential injection attacks. (Contributed by Antoine Pitrou in bpo-22796 [https://bugs.python.org/issue?@action=redirect&bpo=22796].)

Melhorias do Windows:

- A new installer for Windows has replaced the old MSI. See Utilizando Python no Windows for more information.
- Windows builds now use Microsoft Visual C++ 14.0, and extension modules should use the same.

Please read on for a comprehensive list of user-facing changes, including many other smaller improvements, CPython optimizations, deprecations, and potential porting issues.

Novas funcionalidades

PEP 492 - Coroutines with async and await syntax

PEP 492 [https://peps.python.org/pep-0492/] greatly improves support for asynchronous programming in Python by adding awaitable objects, coroutine functions, asynchronous iteration, and asynchronous context managers.

Coroutine functions are declared using the new `async def` syntax:

```
>>> async def coro():
...     return 'spam'
```

Inside a coroutine function, the new `await` expression can be used to suspend coroutine execution until the result is available. Any object can be

awaited, as long as it implements the awaitable protocol by defining the `__await__()` method.

PEP 492 also adds `async for` statement for convenient iteration over asynchronous iterables.

An example of a rudimentary HTTP client written using the new syntax:

```
import asyncio

async def http_get(domain):
    reader, writer = await asyncio.open_connection(domain, 80)

    writer.write(b'\r\n'.join([
        b'GET / HTTP/1.1',
        b'Host: %b' % domain.encode('latin-1'),
        b'Connection: close',
        b'', b''
    ]))

    async for line in reader:
        print('>>>', line)

    writer.close()

loop = asyncio.get_event_loop()
try:
    loop.run_until_complete(http_get('example.com'))
finally:
    loop.close()
```

Similarly to asynchronous iteration, there is a new syntax for asynchronous context managers. The following script:

```
import asyncio

async def coro(name, lock):
    print('coro {}: waiting for lock'.format(name))
    async with lock:
        print('coro {}: holding the lock'.format(name))
        await asyncio.sleep(1)
        print('coro {}: releasing the lock'.format(name))
```

```
loop = asyncio.get_event_loop()
lock = asyncio.Lock()
coros = asyncio.gather(coro(1, lock), coro(2, lock))
try:
    loop.run_until_complete(coros)
finally:
    loop.close()
```

will output:

```
coro 2: waiting for lock
coro 2: holding the lock
coro 1: waiting for lock
coro 2: releasing the lock
coro 1: holding the lock
coro 1: releasing the lock
```

Note that both `async for` and `async with` can only be used inside a coroutine function declared with `async def`.

Coroutine functions are intended to be run inside a compatible event loop, such as the asyncio loop.

Nota

Alterado na versão 3.5.2: Starting with CPython 3.5.2, `__aiter__` can directly return asynchronous iterators. Returning an awaitable object will result in a `PendingDeprecationWarning`.

See more details in the Iteradores assíncronos documentation section.

Ver também

PEP 492 [https://peps.python.org/pep-0492/] – Coroutines with async and await syntax

PEP written and implemented by Yury Selivanov.

PEP 465 - A dedicated infix operator for matrix multiplication

PEP 465 [https://peps.python.org/pep-0465/] adds the `@` infix operator for matrix multiplication. Currently, no builtin Python types implement the new operator, however, it can be implemented by defining `__matmul__()`, `__rmatmul__()`, and `__imatmul__()` for regular, reflected, and in-place matrix multiplication. The semantics of these methods is similar to that of methods defining other infix arithmetic operators.

Matrix multiplication is a notably common operation in many fields of mathematics, science, engineering, and the addition of `@` allows writing cleaner code:

```
S = (H @ beta - r).T @ inv(H @ V @ H.T) @ (H @ beta - r)
```

instead of:

```
S = dot((dot(H, beta) - r).T,
        dot(inv(dot(dot(H, V), H.T)), dot(H, beta) - r))
```

NumPy 1.10 has support for the new operator:

```
>>> import numpy

>>> x = numpy.ones(3)
>>> x
array([ 1., 1., 1.])

>>> m = numpy.eye(3)
>>> m
array([[ 1., 0., 0.],
       [ 0., 1., 0.],
       [ 0., 0., 1.]])

>>> x @ m
array([ 1., 1., 1.])
```

Ver também

PEP 465 [https://peps.python.org/pep-0465/] – A dedicated infix operator for matrix multiplication

PEP written by Nathaniel J. Smith; implemented by Benjamin Peterson.

PEP 448 - Additional Unpacking Generalizations

PEP 448 [https://peps.python.org/pep-0448/] extends the allowed uses of the `*` iterable unpacking operator and `**` dictionary unpacking operator. It is now possible to use an arbitrary number of unpackings in function calls:

```
>>> print(*[1], *[2], 3, *[4, 5])
1 2 3 4 5

>>> def fn(a, b, c, d):
...     print(a, b, c, d)
...

>>> fn(**{'a': 1, 'c': 3}, **{'b': 2, 'd': 4})
1 2 3 4
```

Similarly, tuple, list, set, and dictionary displays allow multiple unpackings (see Listas de expressões and Sintaxes de criação de dicionário):

```
>>> *range(4), 4
(0, 1, 2, 3, 4)

>>> [*range(4), 4]
[0, 1, 2, 3, 4]

>>> {*range(4), 4, *(5, 6, 7)}
{0, 1, 2, 3, 4, 5, 6, 7}

>>> {'x': 1, **{'y': 2}}
{'x': 1, 'y': 2}
```

Ver também

PEP 448 **[https://peps.python.org/pep-0448/] – Additional Unpacking Generalizations**

PEP written by Joshua Landau; implemented by Neil Girdhar, Thomas Wouters, and Joshua Landau.

PEP 461 - percent formatting support for bytes and bytearray

PEP 461 [https://peps.python.org/pep-0461/] adds support for the `%` interpolation operator to **`bytes`** and **`bytearray`**.

While interpolation is usually thought of as a string operation, there are cases where interpolation on `bytes` or `bytearrays` makes sense, and the work needed to make up for this missing functionality detracts from the overall readability of the code. This issue is particularly important when dealing with wire format protocols, which are often a mixture of binary and ASCII compatible text.

Exemplos:

```
>>> b'Hello %b!' % b'World'
b'Hello World!'

>>> b'x=%i y=%f' % (1, 2.5)
b'x=1 y=2.500000'
```

Unicode is not allowed for `%b`, but it is accepted by `%a` (equivalent of `repr(obj).encode('ascii', 'backslashreplace')`):

```
>>> b'Hello %b!' % 'World'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: %b requires bytes, or an object that implements __byt

>>> b'price: %a' % '10€'
b"price: '10\\u20ac'"
```

Note that `%s` and `%r` conversion types, although supported, should only be used in codebases that need compatibility with Python 2.

Ver também

PEP 461 [https://peps.python.org/pep-0461/] – Adding % formatting to bytes and bytearray

PEP written by Ethan Furman; implemented by Neil Schemenauer and Ethan Furman.

PEP 484 - Type Hints

Function annotation syntax has been a Python feature since version 3.0 (**PEP 3107** [https://peps.python.org/pep-3107/]), however the semantics of annotations has been left undefined.

Experience has shown that the majority of function annotation uses were to provide type hints to function parameters and return values. It became evident that it would be beneficial for Python users, if the standard library included the base definitions and tools for type annotations.

PEP 484 [https://peps.python.org/pep-0484/] introduces a provisional module to provide these standard definitions and tools, along with some conventions for situations where annotations are not available.

For example, here is a simple function whose argument and return type are declared in the annotations:

```
def greeting(name: str) -> str:
    return 'Hello ' + name
```

While these annotations are available at runtime through the usual `__annotations__` attribute, *no automatic type checking happens at runtime*. Instead, it is assumed that a separate off-line type checker (e.g. mypy [https://mypy-lang.org]) will be used for on-demand source code analysis.

The type system supports unions, generic types, and a special type named `Any` which is consistent with (i.e. assignable to and from) all types.

Ver também

- `typing` documentação do módulo
- **PEP 484** **[https://peps.python.org/pep-0484/]** **– Type Hints**
  PEP written by Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa; implemented by Guido van Rossum.
- **PEP 483** **[https://peps.python.org/pep-0483/]** **– The Theory of Type Hints**
  PEP escrito por Guido van Rossum

PEP 471 - os.scandir() function – a better and faster directory iterator

PEP 471 [https://peps.python.org/pep-0471/] adds a new directory iteration function, `os.scandir()`, to the standard library. Additionally, `os.walk()` is now implemented using `scandir`, which makes it 3 to 5 times faster on POSIX systems and 7 to 20 times faster on Windows systems. This is largely achieved by greatly reducing the number of calls to `os.stat()` required to walk a directory tree.

Additionally, `scandir` returns an iterator, as opposed to returning a list of file names, which improves memory efficiency when iterating over very large directories.

The following example shows a simple use of `os.scandir()` to display all the files (excluding directories) in the given *path* that don't start with `'.'`. The `entry.is_file()` call will generally not make an additional system call:

```
for entry in os.scandir(path):
    if not entry.name.startswith('.') and entry.is_file():
        print(entry.name)
```

Ver também

PEP 471 [https://peps.python.org/pep-0471/] – os.scandir() function – a better and faster directory iterator

PEP written and implemented by Ben Hoyt with the help of Victor Stinner.

PEP 475: Retry system calls failing with EINTR

An `errno.EINTR` error code is returned whenever a system call, that is waiting for I/O, is interrupted by a signal. Previously, Python would raise `InterruptedError` in such cases. This meant that, when writing a Python application, the developer had two choices:

1. Ignore the `InterruptedError`.
2. Handle the `InterruptedError` and attempt to restart the interrupted system call at every call site.

The first option makes an application fail intermittently. The second option adds a large amount of boilerplate that makes the code nearly unreadable. Compare:

```
print("Hello World")
```

e:

```
while True:
    try:
        print("Hello World")
        break
    except InterruptedError:
        continue
```

PEP 475 [https://peps.python.org/pep-0475/] implements automatic retry of system calls on `EINTR`. This removes the burden of dealing with `EINTR` or `InterruptedError` in user code in most situations and makes Python programs, including the standard library, more robust. Note that the system call is only retried if the signal handler does not raise an exception.

Below is a list of functions which are now retried when interrupted by a signal:

- `open()` and `io.open()`;
- functions of the `faulthandler` module;
- `os` functions: `fchdir()`, `fchmod()`, `fchown()`, `fdatasync()`, `fstat()`, `fstatvfs()`, `fsync()`, `ftruncate()`, `mkfifo()`, `mknod()`, `open()`, `posix_fadvise()`, `posix_fallocate()`, `pread()`, `pwrite()`, `read()`, `readv()`, `sendfile()`, `wait3()`, `wait4()`, `wait()`, `waitid()`, `waitpid()`, `write()`, `writev()`;
- special cases: `os.close()` and `os.dup2()` now ignore `EINTR` errors; the syscall is not retried (see the PEP for the rationale);
- `select` functions: `devpoll.poll()`, `epoll.poll()`, `kqueue.control()`, `poll.poll()`, `select()`;
- methods of the `socket` class: `accept()`, `connect()` (except for non-blocking sockets), `recv()`, `recvfrom()`, `recvmsg()`, `send()`, `sendall()`, `sendmsg()`, `sendto()`;
- `signal.sigtimedwait()` and `signal.sigwaitinfo()`;
- `time.sleep()`.

Ver também

PEP 475 [https://peps.python.org/pep-0475/] – Retry system calls failing with EINTR

PEP and implementation written by Charles-François Natali and Victor Stinner, with the help of Antoine Pitrou (the French connection).

PEP 479: Change StopIteration handling inside generators

The interaction of generators and `StopIteration` in Python 3.4 and earlier was sometimes surprising, and could conceal obscure bugs. Previously, `StopIteration` raised accidentally inside a generator function was interpreted as the end of the iteration by the loop construct driving the generator.

PEP 479 [https://peps.python.org/pep-0479/] changes the behavior of generators: when a `StopIteration` exception is raised inside a generator, it is replaced with a **`RuntimeError`** before it exits the generator frame. The main goal of this change is to ease debugging in the situation where an unguarded **`next()`** call raises `StopIteration` and causes the iteration controlled by the generator to terminate silently. This is particularly pernicious in combination with the `yield from` construct.

This is a backwards incompatible change, so to enable the new behavior, a `__future__` import is necessary:

```
>>> from __future__ import generator_stop

>>> def gen():
...     next(iter([]))
...     yield
...
>>> next(gen())
Traceback (most recent call last):
  File "<stdin>", line 2, in gen
StopIteration

The above exception was the direct cause of the following except

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
RuntimeError: generator raised StopIteration
```

Without a `__future__` import, a **`PendingDeprecationWarning`** will be raised whenever a **`StopIteration`** exception is raised inside a generator.

Ver também

PEP 479 [https://peps.python.org/pep-0479/] – Change StopIteration handling inside generators

PEP written by Chris Angelico and Guido van Rossum. Implemented by Chris Angelico, Yury Selivanov and Nick Coghlan.

PEP 485: A function for testing approximate equality

PEP 485 [https://peps.python.org/pep-0485/] adds the `math.isclose()` and `cmath.isclose()` functions which tell whether two values are approximately equal or "close" to each other. Whether or not two values are considered close is determined according to given absolute and relative tolerances. Relative tolerance is the maximum allowed difference between `isclose` arguments, relative to the larger absolute value:

```
>>> import math
>>> a = 5.0
>>> b = 4.99998
>>> math.isclose(a, b, rel_tol=1e-5)
True
>>> math.isclose(a, b, rel_tol=1e-6)
False
```

It is also possible to compare two values using absolute tolerance, which must be a non-negative value:

```
>>> import math
>>> a = 5.0
>>> b = 4.99998
>>> math.isclose(a, b, abs_tol=0.00003)
True
>>> math.isclose(a, b, abs_tol=0.00001)
False
```

Ver também

PEP 485 [https://peps.python.org/pep-0485/] – Uma função para testar igualdade aproximada

PEP written by Christopher Barker; implemented by Chris Barker and Tal Einat.

PEP 486: Make the Python Launcher aware of virtual environments

PEP 486 [https://peps.python.org/pep-0486/] makes the Windows launcher (see **PEP 397** [https://peps.python.org/pep-0397/]) aware of an active virtual environment. When the default interpreter would be used and the `VIRTUAL_ENV` environment variable is set, the interpreter in the virtual environment will be used.

Ver também

PEP 486 [https://peps.python.org/pep-0486/] – Make the Python Launcher aware of virtual environments
PEP written and implemented by Paul Moore.

PEP 488: Elimination of PYO files

PEP 488 [https://peps.python.org/pep-0488/] does away with the concept of `.pyo` files. This means that `.pyc` files represent both unoptimized and optimized bytecode. To prevent the need to constantly regenerate bytecode files, `.pyc` files now have an optional `opt-` tag in their name when the bytecode is optimized. This has the side-effect of no more bytecode file name clashes when running under either `-O` or `-OO`. Consequently, bytecode files generated from `-O`, and `-OO` may now exist simultaneously. `importlib.util.cache_from_source()` has an updated API to help with this change.

Ver também

PEP 488 [https://peps.python.org/pep-0488/] – Elimination of PYO files
PEP written and implemented by Brett Cannon.

PEP 489: Multi-phase extension module initialization

PEP 489 [https://peps.python.org/pep-0489/] updates extension module initialization to take advantage of the two step module loading mechanism introduced by **PEP 451** [https://peps.python.org/pep-0451/] in Python 3.4.

This change brings the import semantics of extension modules that opt-in to using the new mechanism much closer to those of Python source and bytecode modules, including the ability to use any valid identifier as a module name, rather than being restricted to ASCII.

Ver também

PEP 489 [https://peps.python.org/pep-0489/] – Inicialização de módulo extensão multifase

PEP written by Petr Viktorin, Stefan Behnel, and Nick Coghlan; implemented by Petr Viktorin.

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- Added the `"namereplace"` error handlers. The `"backslashreplace"` error handlers now work with decoding and translating. (Contributed by Serhiy Storchaka in bpo-19676 [https://bugs.python.org/issue?@action=redirect&bpo=19676] and bpo-22286 [https://bugs.python.org/issue?@action=redirect&bpo=22286].)
- The `-b` option now affects comparisons of `bytes` with `int`. (Contributed by Serhiy Storchaka in bpo-23681 [https://bugs.python.org/issue?@action=redirect&bpo=23681].)
- New Kazakh `kz1048` and Tajik `koi8_t` codecs. (Contributed by Serhiy Storchaka in bpo-22682 [https://bugs.python.org/issue?@action=redirect&bpo=22682] and bpo-22681 [https://bugs.python.org/issue?@action=redirect&bpo=22681].)
- Property docstrings are now writable. This is especially useful for `collections.namedtuple()` docstrings. (Contributed by Berker Peksag in bpo-24064 [https://bugs.python.org/issue?@action=redirect&bpo=24064].)
- Circular imports involving relative imports are now supported. (Contributed by Brett Cannon and Antoine Pitrou in bpo-17636 [https://bugs.python.org/issue?@action=redirect&bpo=17636].)

Novos módulos

typing

The new `typing` provisional module provides standard definitions and tools for function type annotations. See Type Hints for more information.

zipapp

The new `zipapp` module (specified in **PEP 441** [https://peps.python.org/pep-0441/]) provides an API and command line tool for creating executable Python Zip Applications, which were introduced in Python 2.6 in bpo-1739468 [https://bugs.python.org/issue?@action=redirect&bpo=1739468], but which were not well publicized, either at the time or since.

With the new module, bundling your application is as simple as putting all the files, including a `__main__.py` file, into a directory `myapp` and running:

```
$ python -m zipapp myapp
$ python myapp.pyz
```

The module implementation has been contributed by Paul Moore in bpo-23491 [https://bugs.python.org/issue?@action=redirect&bpo=23491].

Ver também

PEP 441 [https://peps.python.org/pep-0441/] – Improving Python ZIP Application Support

Módulos melhorados

argparse

The `ArgumentParser` class now allows disabling abbreviated usage of long options by setting allow_abbrev to `False`. (Contributed by Jonathan Paugh, Steven Bethard, paul j3 and Daniel Eriksson in bpo-14910 [https://bugs.python.org/issue?@action=redirect&bpo=14910].)

asyncio

Since the `asyncio` module is provisional, all changes introduced in Python 3.5 have also been backported to Python 3.4.x.

Notable changes in the `asyncio` module since Python 3.4.0:

- New debugging APIs: `loop.set_debug()` and `loop.get_debug()` methods. (Contributed by Victor Stinner.)
- The proactor event loop now supports SSL. (Contributed by Antoine Pitrou and Victor Stinner in bpo-22560 [https://bugs.python.org/issue?@action=redirect&bpo=22560].)
- A new `loop.is_closed()` method to check if the event loop is closed. (Contributed by Victor Stinner in bpo-21326 [https://bugs.python.org/issue?@action=redirect&bpo=21326].)
- A new `loop.create_task()` to conveniently create and schedule a new `Task` for a coroutine. The `create_task` method is also used by all asyncio functions that wrap coroutines into tasks, such as `asyncio.wait()`, `asyncio.gather()`, etc. (Contributed by Victor Stinner.)
- A new `transport.get_write_buffer_limits()` method to inquire for *high*- and *low*- water limits of the flow control. (Contributed by Victor Stinner.)
- The `async()` function is deprecated in favor of `ensure_future()`. (Contributed by Yury Selivanov.)
- New `loop.set_task_factory()` and `loop.get_task_factory()` methods to customize the task factory that `loop.create_task()` method uses. (Contributed by Yury Selivanov.)
- New `Queue.join()` and `Queue.task_done()` queue methods. (Contributed by Victor Stinner.)
- The `JoinableQueue` class was removed, in favor of the `asyncio.Queue` class. (Contributed by Victor Stinner.)

Updates in 3.5.1:

- The `ensure_future()` function and all functions that use it, such as `loop.run_until_complete()`, now accept all kinds of awaitable

objects. (Contributed by Yury Selivanov.)

- New `run_coroutine_threadsafe()` function to submit coroutines to event loops from other threads. (Contributed by Vincent Michel.)
- New `Transport.is_closing()` method to check if the transport is closing or closed. (Contributed by Yury Selivanov.)
- The `loop.create_server()` method can now accept a list of hosts. (Contributed by Yann Sionneau.)

Updates in 3.5.2:

- New `loop.create_future()` method to create Future objects. This allows alternative event loop implementations, such as uvloop [https://github.com/MagicStack/uvloop], to provide a faster `asyncio.Future` implementation. (Contributed by Yury Selivanov.)
- New `loop.get_exception_handler()` method to get the current exception handler. (Contributed by Yury Selivanov.)
- New `StreamReader.readuntil()` method to read data from the stream until a separator bytes sequence appears. (Contributed by Mark Korenberg.)
- The `loop.create_connection()` and `loop.create_server()` methods are optimized to avoid calling the system `getaddrinfo` function if the address is already resolved. (Contributed by A. Jesse Jiryu Davis.)
- The `loop.sock_connect(sock, address)` no longer requires the *address* to be resolved prior to the call. (Contributed by A. Jesse Jiryu Davis.)

bz2

The `BZ2Decompressor.decompress` method now accepts an optional *max_length* argument to limit the maximum size of decompressed data. (Contributed by Nikolaus Rath in bpo-15955 [https://bugs.python.org/issue?@action=redirect&bpo=15955].)

cgi

The `FieldStorage` class now supports the context manager protocol. (Contributed by Berker Peksag in bpo-20289 [https://bugs.python.org/issue?

@action=redirect&bpo=20289].)

cmath

A new function `isclose()` provides a way to test for approximate equality. (Contributed by Chris Barker and Tal Einat in bpo-24270 [https://bugs.python.org/issue?@action=redirect&bpo=24270].)

código

The `InteractiveInterpreter.showtraceback()` method now prints the full chained traceback, just like the interactive interpreter. (Contributed by Claudiu Popa in bpo-17442 [https://bugs.python.org/issue?@action=redirect&bpo=17442].)

collections

The `OrderedDict` class is now implemented in C, which makes it 4 to 100 times faster. (Contributed by Eric Snow in bpo-16991 [https://bugs.python.org/issue?@action=redirect&bpo=16991].)

`OrderedDict.items()`, `OrderedDict.keys()`, and `OrderedDict.values()` views now support `reversed()` iteration. (Contributed by Serhiy Storchaka in bpo-19505 [https://bugs.python.org/issue?@action=redirect&bpo=19505].)

The `deque` class now defines `index()`, `insert()`, and `copy()`, and supports the `+` and `*` operators. This allows deques to be recognized as a `MutableSequence` and improves their substitutability for lists. (Contributed by Raymond Hettinger in bpo-23704 [https://bugs.python.org/issue?@action=redirect&bpo=23704].)

Docstrings produced by `namedtuple()` can now be updated:

```
Point = namedtuple('Point', ['x', 'y'])
Point.__doc__ += ': Cartesian coordinate'
Point.x.__doc__ = 'abscissa'
Point.y.__doc__ = 'ordinate'
```

(Contributed by Berker Peksag in bpo-24064 [https://bugs.python.org/issue?@action=redirect&bpo=24064].)

The `UserString` class now implements the `__getnewargs__()`, `__rmod__()`, `casefold()`, `format_map()`, `isprintable()`, and `maketrans()` methods to match the corresponding methods of `str`. (Contributed by Joe Jevnik in bpo-22189 [https://bugs.python.org/issue?@action=redirect&bpo=22189].)

collections.abc

The `Sequence.index()` method now accepts *start* and *stop* arguments to match the corresponding methods of `tuple`, `list`, etc. (Contributed by Devin Jeanpierre in bpo-23086 [https://bugs.python.org/issue?@action=redirect&bpo=23086].)

A new `Generator` abstract base class. (Contributed by Stefan Behnel in bpo-24018 [https://bugs.python.org/issue?@action=redirect&bpo=24018].)

New `Awaitable`, `Coroutine`, `AsyncIterator`, and `AsyncIterable` abstract base classes. (Contributed by Yury Selivanov in bpo-24184 [https://bugs.python.org/issue?@action=redirect&bpo=24184].)

For earlier Python versions, a backport of the new ABCs is available in an external PyPI package [https://pypi.org/project/backports_abc/].

compileall

A new `compileall` option, `-j N`, allows running *N* workers simultaneously to perform parallel bytecode compilation. The `compile_dir()` function has a corresponding `workers` parameter. (Contributed by Claudiu Popa in bpo-16104 [https://bugs.python.org/issue?@action=redirect&bpo=16104].)

Another new option, `-r`, allows controlling the maximum recursion level for subdirectories. (Contributed by Claudiu Popa in bpo-19628 [https://bugs.python.org/issue?@action=redirect&bpo=19628].)

The `-q` command line option can now be specified more than once, in which case all output, including errors, will be suppressed. The corresponding `quiet` parameter in `compile_dir()`, `compile_file()`, and `compile_path()` can now accept an integer value indicating the level of output suppression. (Contributed by Thomas Kluyver in bpo-21338 [https://bugs.python.org/issue?@action=redirect&bpo=21338].)

concurrent.futures

The `Executor.map()` method now accepts a *chunksize* argument to allow batching of tasks to improve performance when `ProcessPoolExecutor()` is used. (Contributed by Dan O'Reilly in bpo-11271 [https://bugs.python.org/issue?@action=redirect&bpo=11271].)

The number of workers in the `ThreadPoolExecutor` constructor is optional now. The default value is 5 times the number of CPUs. (Contributed by Claudiu Popa in bpo-21527 [https://bugs.python.org/issue?@action=redirect&bpo=21527].)

configparser

`configparser` now provides a way to customize the conversion of values by specifying a dictionary of converters in the `ConfigParser` constructor, or by defining them as methods in `ConfigParser` subclasses. Converters defined in a parser instance are inherited by its section proxies.

Exemplo:

```
>>> import configparser
>>> conv = {}
>>> conv['list'] = lambda v: [e.strip() for e in v.split() if e.
>>> cfg = configparser.ConfigParser(converters=conv)
>>> cfg.read_string("""
... [s]
... list = a b c d e f g
... """)
>>> cfg.get('s', 'list')
'a b c d e f g'
>>> cfg.getlist('s', 'list')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

```
>>> section = cfg['s']
>>> section.getlist('list')
['a', 'b', 'c', 'd', 'e', 'f', 'g']
```

(Contributed by Łukasz Langa in bpo-18159 [https://bugs.python.org/issue?@action=redirect&bpo=18159].)

contextlib

The new `redirect_stderr()` context manager (similar to `redirect_stdout()`) makes it easier for utility scripts to handle inflexible APIs that write their output to `sys.stderr` and don't provide any options to redirect it:

```
>>> import contextlib, io, logging
>>> f = io.StringIO()
>>> with contextlib.redirect_stderr(f):
...     logging.warning('warning')
...
>>> f.getvalue()
'WARNING:root:warning\n'
```

(Contributed by Berker Peksag in bpo-22389 [https://bugs.python.org/issue?@action=redirect&bpo=22389].)

csv

The `writerow()` method now supports arbitrary iterables, not just sequences. (Contributed by Serhiy Storchaka in bpo-23171 [https://bugs.python.org/issue?@action=redirect&bpo=23171].)

curses

The new `update_lines_cols()` function updates the `LINES` and `COLS` module variables. This is useful for detecting manual screen resizing. (Contributed by Arnon Yaari in bpo-4254 [https://bugs.python.org/issue?@action=redirect&bpo=4254].)

dbm

`dumb.open` always creates a new database when the flag has the value `"n"`. (Contributed by Claudiu Popa in bpo-18039 [https://bugs.python.org/issue?@action=redirect&bpo=18039].)

difflib

The charset of HTML documents generated by **`HtmlDiff.make_file()`** can now be customized by using a new *charset* keyword-only argument. The default charset of HTML document changed from `"ISO-8859-1"` to `"utf-8"`. (Contributed by Berker Peksag in bpo-2052 [https://bugs.python.org/issue?@action=redirect&bpo=2052].)

The **`diff_bytes()`** function can now compare lists of byte strings. This fixes a regression from Python 2. (Contributed by Terry J. Reedy and Greg Ward in bpo-17445 [https://bugs.python.org/issue?@action=redirect&bpo=17445].)

distutils

Both the `build` and `build_ext` commands now accept a `-j` option to enable parallel building of extension modules. (Contributed by Antoine Pitrou in bpo-5309 [https://bugs.python.org/issue?@action=redirect&bpo=5309].)

The `distutils` module now supports `xz` compression, and can be enabled by passing `xztar` as an argument to `bdist --format`. (Contributed by Serhiy Storchaka in bpo-16314 [https://bugs.python.org/issue?@action=redirect&bpo=16314].)

doctest

The **`DocTestSuite()`** function returns an empty **`unittest.TestSuite`** if *module* contains no docstrings, instead of raising **`ValueError`**. (Contributed by Glenn Jones in bpo-15916 [https://bugs.python.org/issue?@action=redirect&bpo=15916].)

email

A new policy option `Policy.mangle_from_` controls whether or not lines that start with `"From "` in email bodies are prefixed with a `">"` character by generators. The default is `True` for `compat32` and `False` for all other policies. (Contributed by Milan Oberkirch in bpo-20098 [https://bugs.python.org/issue?@action=redirect&bpo=20098].)

A new `Message.get_content_disposition()` method provides easy access to a canonical value for the *Content-Disposition* header. (Contributed by Abhilash Raj in bpo-21083 [https://bugs.python.org/issue?@action=redirect&bpo=21083].)

A new policy option `EmailPolicy.utf8` can be set to `True` to encode email headers using the UTF-8 charset instead of using encoded words. This allows `Messages` to be formatted according to **RFC 6532** [https://datatracker.ietf.org/doc/html/rfc6532.html] and used with an SMTP server that supports the **RFC 6531** [https://datatracker.ietf.org/doc/html/rfc6531.html] `SMTPUTF8` extension. (Contributed by R. David Murray in bpo-24211 [https://bugs.python.org/issue?@action=redirect&bpo=24211].)

The `mime.text.MIMEText` constructor now accepts a `charset.Charset` instance. (Contributed by Claude Paroz and Berker Peksag in bpo-16324 [https://bugs.python.org/issue?@action=redirect&bpo=16324].)

enum

The `Enum` callable has a new parameter *start* to specify the initial number of enum values if only *names* are provided:

```
>>> Animal = enum.Enum('Animal', 'cat dog', start=10)
>>> Animal.cat
<Animal.cat: 10>
>>> Animal.dog
<Animal.dog: 11>
```

(Contributed by Ethan Furman in bpo-21706 [https://bugs.python.org/issue?@action=redirect&bpo=21706].)

faulthandler

The `enable()`, `register()`, `dump_traceback()` and `dump_traceback_later()` functions now accept file descriptors in addition to file-like objects. (Contributed by Wei Wu in bpo-23566 [https://bugs.python.org/issue?@action=redirect&bpo=23566].)

functools

Most of the `@~functools.lru_cache` machinery is now implemented in C, making it significantly faster. (Contributed by Matt Joiner, Alexey Kachayev, and Serhiy Storchaka in bpo-14373 [https://bugs.python.org/issue?@action=redirect&bpo=14373].)

glob

The `iglob()` and `glob()` functions now support recursive search in subdirectories, using the `"**"` pattern. (Contributed by Serhiy Storchaka in bpo-13968 [https://bugs.python.org/issue?@action=redirect&bpo=13968].)

gzip

The *mode* argument of the `GzipFile` constructor now accepts `"x"` to request exclusive creation. (Contributed by Tim Heaney in bpo-19222 [https://bugs.python.org/issue?@action=redirect&bpo=19222].)

heapq

Element comparison in `merge()` can now be customized by passing a key function in a new optional *key* keyword argument, and a new optional *reverse* keyword argument can be used to reverse element comparison:

```
>>> import heapq
>>> a = ['9', '777', '55555']
>>> b = ['88', '6666']
>>> list(heapq.merge(a, b, key=len))
['9', '88', '777', '6666', '55555']
>>> list(heapq.merge(reversed(a), reversed(b), key=len, reverse=
['55555', '6666', '777', '88', '9']
```

(Contributed by Raymond Hettinger in bpo-13742 [https://bugs.python.org/issue?@action=redirect&bpo=13742].)

http

A new **`HTTPStatus`** enum that defines a set of HTTP status codes, reason phrases and long descriptions written in English. (Contributed by Demian Brecht in bpo-21793 [https://bugs.python.org/issue?@action=redirect&bpo=21793].)

http.client

`HTTPConnection.getresponse()` now raises a **`RemoteDisconnected`** exception when a remote server connection is closed unexpectedly. Additionally, if a **`ConnectionError`** (of which `RemoteDisconnected` is a subclass) is raised, the client socket is now closed automatically, and will reconnect on the next request:

```
import http.client
conn = http.client.HTTPConnection('www.python.org')
for retries in range(3):
    try:
        conn.request('GET', '/')
        resp = conn.getresponse()
    except http.client.RemoteDisconnected:
        pass
```

(Contributed by Martin Panter in bpo-3566 [https://bugs.python.org/issue?@action=redirect&bpo=3566].)

idlelib e IDLE

Since idlelib implements the IDLE shell and editor and is not intended for import by other programs, it gets improvements with every release. See `Lib/idlelib/NEWS.txt` for a cumulative list of changes since 3.4.0, as well as changes made in future 3.5.x releases. This file is also available from the IDLE *Help* ▸ *About IDLE* dialog.

imaplib

The **`IMAP4`** class now supports the context manager protocol. When used in a **`with`** statement, the IMAP4 `LOGOUT` command will be called automatically at the end of the block. (Contributed by Tarek Ziadé and Serhiy Storchaka in bpo-4972 [https://bugs.python.org/issue?@action=redirect&bpo=4972].)

The **`imaplib`** module now supports **RFC 5161** [https://datatracker.ietf.org/doc/html/rfc5161.html] (ENABLE Extension) and **RFC 6855** [https://datatracker.ietf.org/doc/html/rfc6855.html] (UTF-8 Support) via the **`IMAP4.enable()`** method. A new **`IMAP4.utf8_enabled`** attribute tracks whether or not **RFC 6855** [https://datatracker.ietf.org/doc/html/rfc6855.html] support is enabled. (Contributed by Milan Oberkirch, R. David Murray, and Maciej Szulik in bpo-21800 [https://bugs.python.org/issue?@action=redirect&bpo=21800].)

The **`imaplib`** module now automatically encodes non-ASCII string usernames and passwords using UTF-8, as recommended by the RFCs. (Contributed by Milan Oberkirch in bpo-21800 [https://bugs.python.org/issue?@action=redirect&bpo=21800].)

imghdr

The **`what()`** function now recognizes the OpenEXR [https://www.openexr.com] format (contributed by Martin Vignali and Claudiu Popa in bpo-20295 [https://bugs.python.org/issue?@action=redirect&bpo=20295]), and the WebP [https://en.wikipedia.org/wiki/WebP] format (contributed by Fabrice Aneche and Claudiu Popa in bpo-20197 [https://bugs.python.org/issue?@action=redirect&bpo=20197].)

importlib

The **`util.LazyLoader`** class allows for lazy loading of modules in applications where startup time is important. (Contributed by Brett Cannon in bpo-17621 [https://bugs.python.org/issue?@action=redirect&bpo=17621].)

The **`abc.InspectLoader.source_to_code()`** method is now a static method. This makes it easier to initialize a module object with code compiled from a string by running `exec(code, module.__dict__)`. (Contributed by Brett Cannon in bpo-21156 [https://bugs.python.org/issue?@action=redirect&bpo=21156].)

The new `util.module_from_spec()` function is now the preferred way to create a new module. As opposed to creating a `types.ModuleType` instance directly, this new function will set the various import-controlled attributes based on the passed-in spec object. (Contributed by Brett Cannon in bpo-20383 [https://bugs.python.org/issue?@action=redirect&bpo=20383].)

inspect

Both the `Signature` and `Parameter` classes are now picklable and hashable. (Contributed by Yury Selivanov in bpo-20726 [https://bugs.python.org/issue?@action=redirect&bpo=20726] and bpo-20334 [https://bugs.python.org/issue?@action=redirect&bpo=20334].)

A new `BoundArguments.apply_defaults()` method provides a way to set default values for missing arguments:

```
>>> def foo(a, b='ham', *args): pass
>>> ba = inspect.signature(foo).bind('spam')
>>> ba.apply_defaults()
>>> ba.arguments
OrderedDict([('a', 'spam'), ('b', 'ham'), ('args', ())])
```

(Contributed by Yury Selivanov in bpo-24190 [https://bugs.python.org/issue?@action=redirect&bpo=24190].)

A new class method `Signature.from_callable()` makes subclassing of `Signature` easier. (Contributed by Yury Selivanov and Eric Snow in bpo-17373 [https://bugs.python.org/issue?@action=redirect&bpo=17373].)

The `signature()` function now accepts a *follow_wrapped* optional keyword argument, which, when set to `False`, disables automatic following of `__wrapped__` links. (Contributed by Yury Selivanov in bpo-20691 [https://bugs.python.org/issue?@action=redirect&bpo=20691].)

A set of new functions to inspect coroutine functions and coroutine objects has been added: `iscoroutine()`, `iscoroutinefunction()`, `isawaitable()`, `getcoroutinelocals()`, and `getcoroutinestate()`. (Contributed by Yury

Selivanov in bpo-24017 [https://bugs.python.org/issue?@action=redirect&bpo=24017] and bpo-24400 [https://bugs.python.org/issue?@action=redirect&bpo=24400].)

The **stack()**, **trace()**, **getouterframes()**, and **getinnerframes()** functions now return a list of named tuples. (Contributed by Daniel Shahaf in bpo-16808 [https://bugs.python.org/issue?@action=redirect&bpo=16808].)

io

A new **BufferedIOBase.readinto1()** method, that uses at most one call to the underlying raw stream's **RawIOBase.read()** or **RawIOBase.readinto()** methods. (Contributed by Nikolaus Rath in bpo-20578 [https://bugs.python.org/issue?@action=redirect&bpo=20578].)

ipaddress

Both the **IPv4Network** and **IPv6Network** classes now accept an `(address, netmask)` tuple argument, so as to easily construct network objects from existing addresses:

```
>>> import ipaddress
>>> ipaddress.IPv4Network(('127.0.0.0', 8))
IPv4Network('127.0.0.0/8')
>>> ipaddress.IPv4Network(('127.0.0.0', '255.0.0.0'))
IPv4Network('127.0.0.0/8')
```

(Contributed by Peter Moody and Antoine Pitrou in bpo-16531 [https://bugs.python.org/issue?@action=redirect&bpo=16531].)

A new **reverse_pointer** attribute for the **IPv4Address** and **IPv6Address** classes returns the name of the reverse DNS PTR record:

```
>>> import ipaddress
>>> addr = ipaddress.IPv4Address('127.0.0.1')
>>> addr.reverse_pointer
'1.0.0.127.in-addr.arpa'
>>> addr6 = ipaddress.IPv6Address('::1')
>>> addr6.reverse_pointer
'1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.
```

(Contributed by Leon Weber in bpo-20480 [https://bugs.python.org/issue?@action=redirect&bpo=20480].)

json

The `json.tool` command line interface now preserves the order of keys in JSON objects passed in input. The new `--sort-keys` option can be used to sort the keys alphabetically. (Contributed by Berker Peksag in bpo-21650 [https://bugs.python.org/issue?@action=redirect&bpo=21650].)

JSON decoder now raises `JSONDecodeError` instead of `ValueError` to provide better context information about the error. (Contributed by Serhiy Storchaka in bpo-19361 [https://bugs.python.org/issue?@action=redirect&bpo=19361].)

linecache

A new `lazycache()` function can be used to capture information about a non-file-based module to permit getting its lines later via `getline()`. This avoids doing I/O until a line is actually needed, without having to carry the module globals around indefinitely. (Contributed by Robert Collins in bpo-17911 [https://bugs.python.org/issue?@action=redirect&bpo=17911].)

locale

A new `delocalize()` function can be used to convert a string into a normalized number string, taking the `LC_NUMERIC` settings into account:

```
>>> import locale
>>> locale.setlocale(locale.LC_NUMERIC, 'de_DE.UTF-8')
'de_DE.UTF-8'
>>> locale.delocalize('1.234,56')
'1234.56'
>>> locale.setlocale(locale.LC_NUMERIC, 'en_US.UTF-8')
'en_US.UTF-8'
>>> locale.delocalize('1,234.56')
'1234.56'
```

(Contributed by Cédric Krier in bpo-13918 [https://bugs.python.org/issue?@action=redirect&bpo=13918].)

logging

All logging methods (`Logger` `log()`, `exception()`, `critical()`, `debug()`, etc.), now accept exception instances as an *exc_info* argument, in addition to boolean values and exception tuples:

```
>>> import logging
>>> try:
...     1/0
... except ZeroDivisionError as ex:
...     logging.error('exception', exc_info=ex)
ERROR:root:exception
```

(Contributed by Yury Selivanov in bpo-20537 [https://bugs.python.org/issue?@action=redirect&bpo=20537].)

The `handlers.HTTPHandler` class now accepts an optional `ssl.SSLContext` instance to configure SSL settings used in an HTTP connection. (Contributed by Alex Gaynor in bpo-22788 [https://bugs.python.org/issue?@action=redirect&bpo=22788].)

The `handlers.QueueListener` class now takes a *respect_handler_level* keyword argument which, if set to `True`, will pass messages to handlers taking handler levels into account. (Contributed by Vinay Sajip.)

lzma

The `LZMADecompressor.decompress()` method now accepts an optional *max_length* argument to limit the maximum size of decompressed data. (Contributed by Martin Panter in bpo-15955 [https://bugs.python.org/issue?@action=redirect&bpo=15955].)

math

Two new constants have been added to the `math` module: `inf` and `nan`. (Contributed by Mark Dickinson in bpo-23185 [https://bugs.python.org/issue?@action=redirect&bpo=23185].)

A new function `isclose()` provides a way to test for approximate equality. (Contributed by Chris Barker and Tal Einat in bpo-24270 [https://bugs.python.org/issue?@action=redirect&bpo=24270].)

A new `gcd()` function has been added. The `fractions.gcd()` function is now deprecated. (Contributed by Mark Dickinson and Serhiy Storchaka in bpo-22486 [https://bugs.python.org/issue?@action=redirect&bpo=22486].)

multiprocessing

`sharedctypes.synchronized()` objects now support the context manager protocol. (Contributed by Charles-François Natali in bpo-21565 [https://bugs.python.org/issue?@action=redirect&bpo=21565].)

operador

`attrgetter()`, `itemgetter()`, and `methodcaller()` objects now support pickling. (Contributed by Josh Rosenberg and Serhiy Storchaka in bpo-22955 [https://bugs.python.org/issue?@action=redirect&bpo=22955].)

New `matmul()` and `imatmul()` functions to perform matrix multiplication. (Contributed by Benjamin Peterson in bpo-21176 [https://bugs.python.org/issue?@action=redirect&bpo=21176].)

os

The new `scandir()` function returning an iterator of `DirEntry` objects has been added. If possible, `scandir()` extracts file attributes while scanning a directory, removing the need to perform subsequent system calls to determine file type or attributes, which may significantly improve performance. (Contributed by Ben Hoyt with the help of Victor Stinner in bpo-22524 [https://bugs.python.org/issue?@action=redirect&bpo=22524].)

On Windows, a new **`stat_result.st_file_attributes`** attribute is now available. It corresponds to the `dwFileAttributes` member of the `BY_HANDLE_FILE_INFORMATION` structure returned by `GetFileInformationByHandle()`. (Contributed by Ben Hoyt in bpo-21719 [https://bugs.python.org/issue?@action=redirect&bpo=21719].)

The **`urandom()`** function now uses the `getrandom()` syscall on Linux 3.17 or newer, and `getentropy()` on OpenBSD 5.6 and newer, removing the need to use `/dev/urandom` and avoiding failures due to potential file descriptor exhaustion. (Contributed by Victor Stinner in bpo-22181 [https://bugs.python.org/issue?@action=redirect&bpo=22181].)

New **`get_blocking()`** and **`set_blocking()`** functions allow getting and setting a file descriptor's blocking mode (**`O_NONBLOCK`**.) (Contributed by Victor Stinner in bpo-22054 [https://bugs.python.org/issue?@action=redirect&bpo=22054].)

The **`truncate()`** and **`ftruncate()`** functions are now supported on Windows. (Contributed by Steve Dower in bpo-23668 [https://bugs.python.org/issue?@action=redirect&bpo=23668].)

There is a new **`os.path.commonpath()`** function returning the longest common sub-path of each passed pathname. Unlike the **`os.path.commonprefix()`** function, it always returns a valid path:

```
>>> os.path.commonprefix(['/usr/lib', '/usr/local/lib'])
'/usr/l'

>>> os.path.commonpath(['/usr/lib', '/usr/local/lib'])
'/usr'
```

(Contributed by Rafik Draoui and Serhiy Storchaka in bpo-10395 [https://bugs.python.org/issue?@action=redirect&bpo=10395].)

pathlib

The new **`Path.samefile()`** method can be used to check whether the path points to the same file as another path, which can be either another **`Path`**

object, or a string:

```
>>> import pathlib
>>> p1 = pathlib.Path('/etc/hosts')
>>> p2 = pathlib.Path('/etc/../etc/hosts')
>>> p1.samefile(p2)
True
```

(Contributed by Vajrasky Kok and Antoine Pitrou in bpo-19775 [https://bugs.python.org/issue?@action=redirect&bpo=19775].)

The `Path.mkdir()` method now accepts a new optional *exist_ok* argument to match `mkdir -p` and `os.makedirs()` functionality. (Contributed by Berker Peksag in bpo-21539 [https://bugs.python.org/issue?@action=redirect&bpo=21539].)

There is a new `Path.expanduser()` method to expand ~ and `~user` prefixes. (Contributed by Serhiy Storchaka and Claudiu Popa in bpo-19776 [https://bugs.python.org/issue?@action=redirect&bpo=19776].)

A new `Path.home()` class method can be used to get a `Path` instance representing the user's home directory. (Contributed by Victor Salgado and Mayank Tripathi in bpo-19777 [https://bugs.python.org/issue?@action=redirect&bpo=19777].)

New `Path.write_text()`, `Path.read_text()`, `Path.write_bytes()`, `Path.read_bytes()` methods to simplify read/write operations on files.

The following code snippet will create or rewrite existing file `~/spam42`:

```
>>> import pathlib
>>> p = pathlib.Path('~/spam42')
>>> p.expanduser().write_text('ham')
3
```

(Contributed by Christopher Welborn in bpo-20218 [https://bugs.python.org/issue?@action=redirect&bpo=20218].)

pickle

Nested objects, such as unbound methods or nested classes, can now be pickled using pickle protocols older than protocol version 4. Protocol version 4 already supports these cases. (Contributed by Serhiy Storchaka in bpo-23611 [https://bugs.python.org/issue?@action=redirect&bpo=23611].)

poplib

A new **POP3.utf8()** command enables **RFC 6856** [https://datatracker.ietf.org/doc/html/rfc6856.html] (Internationalized Email) support, if a POP server supports it. (Contributed by Milan OberKirch in bpo-21804 [https://bugs.python.org/issue?@action=redirect&bpo=21804].)

re

References and conditional references to groups with fixed length are now allowed in lookbehind assertions:

```
>>> import re
>>> pat = re.compile(r'(a|b).(?<=\1)c')
>>> pat.match('aac')
<_sre.SRE_Match object; span=(0, 3), match='aac'>
>>> pat.match('bbc')
<_sre.SRE_Match object; span=(0, 3), match='bbc'>
```

(Contributed by Serhiy Storchaka in bpo-9179 [https://bugs.python.org/issue?@action=redirect&bpo=9179].)

The number of capturing groups in regular expressions is no longer limited to 100. (Contributed by Serhiy Storchaka in bpo-22437 [https://bugs.python.org/issue?@action=redirect&bpo=22437].)

The **sub()** and **subn()** functions now replace unmatched groups with empty strings instead of raising an exception. (Contributed by Serhiy Storchaka in bpo-1519638 [https://bugs.python.org/issue?@action=redirect&bpo=1519638].)

The **re.error** exceptions have new attributes, **msg**, **pattern**, **pos**, **lineno**, and **colno**, that provide better context information about the error:

```
>>> re.compile("""
...     (?x)
...     .++
... """)
Traceback (most recent call last):
   ...
sre_constants.error: multiple repeat at position 16 (line 3, col
```

(Contributed by Serhiy Storchaka in bpo-22578 [https://bugs.python.org/issue?@action=redirect&bpo=22578].)

readline

A new `append_history_file()` function can be used to append the specified number of trailing elements in history to the given file. (Contributed by Bruno Cauet in bpo-22940 [https://bugs.python.org/issue?@action=redirect&bpo=22940].)

selectors

The new `DevpollSelector` supports efficient `/dev/poll` polling on Solaris. (Contributed by Giampaolo Rodola' in bpo-18931 [https://bugs.python.org/issue?@action=redirect&bpo=18931].)

shutil

The `move()` function now accepts a *copy_function* argument, allowing, for example, the `copy()` function to be used instead of the default `copy2()` if there is a need to ignore file metadata when moving. (Contributed by Claudiu Popa in bpo-19840 [https://bugs.python.org/issue?@action=redirect&bpo=19840].)

The `make_archive()` function now supports the *xztar* format. (Contributed by Serhiy Storchaka in bpo-5411 [https://bugs.python.org/issue?@action=redirect&bpo=5411].)

signal

On Windows, the **`set_wakeup_fd()`** function now also supports socket handles. (Contributed by Victor Stinner in bpo-22018 [https://bugs.python.org/issue?@action=redirect&bpo=22018].)

Various `SIG*` constants in the **`signal`** module have been converted into **`Enums`**. This allows meaningful names to be printed during debugging, instead of integer "magic numbers". (Contributed by Giampaolo Rodola' in bpo-21076 [https://bugs.python.org/issue?@action=redirect&bpo=21076].)

smtpd

Both the **`SMTPServer`** and **`SMTPChannel`** classes now accept a *decode_data* keyword argument to determine if the `DATA` portion of the SMTP transaction is decoded using the `"utf-8"` codec or is instead provided to the **`SMTPServer.process_message()`** method as a byte string. The default is `True` for backward compatibility reasons, but will change to `False` in Python 3.6. If *decode_data* is set to `False`, the `process_message` method must be prepared to accept keyword arguments. (Contributed by Maciej Szulik in bpo-19662 [https://bugs.python.org/issue?@action=redirect&bpo=19662].)

The **`SMTPServer`** class now advertises the `8BITMIME` extension (**RFC 6152** [https://datatracker.ietf.org/doc/html/rfc6152.html]) if *decode_data* has been set `True`. If the client specifies `BODY=8BITMIME` on the `MAIL` command, it is passed to **`SMTPServer.process_message()`** via the *mail_options* keyword. (Contributed by Milan Oberkirch and R. David Murray in bpo-21795 [https://bugs.python.org/issue?@action=redirect&bpo=21795].)

The **`SMTPServer`** class now also supports the `SMTPUTF8` extension (**RFC 6531** [https://datatracker.ietf.org/doc/html/rfc6531.html]: Internationalized Email). If the client specified `SMTPUTF8 BODY=8BITMIME` on the `MAIL` command, they are passed to **`SMTPServer.process_message()`** via the *mail_options* keyword. It is the responsibility of the `process_message` method to correctly handle the `SMTPUTF8` data. (Contributed by Milan Oberkirch in bpo-21725 [https://bugs.python.org/issue?@action=redirect&bpo=21725].)

It is now possible to provide, directly or via name resolution, IPv6 addresses in the **`SMTPServer`** constructor, and have it successfully connect.

(Contributed by Milan Oberkirch in bpo-14758 [https://bugs.python.org/issue?@action=redirect&bpo=14758].)

smtplib

A new `SMTP.auth()` method provides a convenient way to implement custom authentication mechanisms. (Contributed by Milan Oberkirch in bpo-15014 [https://bugs.python.org/issue?@action=redirect&bpo=15014].)

The `SMTP.set_debuglevel()` method now accepts an additional debuglevel (2), which enables timestamps in debug messages. (Contributed by Gavin Chappell and Maciej Szulik in bpo-16914 [https://bugs.python.org/issue?@action=redirect&bpo=16914].)

Both the `SMTP.sendmail()` and `SMTP.send_message()` methods now support **RFC 6531** [https://datatracker.ietf.org/doc/html/rfc6531.html] (SMTPUTF8). (Contributed by Milan Oberkirch and R. David Murray in bpo-22027 [https://bugs.python.org/issue?@action=redirect&bpo=22027].)

sndhdr

The `what()` and `whathdr()` functions now return a `namedtuple()`. (Contributed by Claudiu Popa in bpo-18615 [https://bugs.python.org/issue?@action=redirect&bpo=18615].)

socket

Functions with timeouts now use a monotonic clock, instead of a system clock. (Contributed by Victor Stinner in bpo-22043 [https://bugs.python.org/issue?@action=redirect&bpo=22043].)

A new `socket.sendfile()` method allows sending a file over a socket by using the high-performance `os.sendfile()` function on UNIX, resulting in uploads being from 2 to 3 times faster than when using plain `socket.send()`. (Contributed by Giampaolo Rodola' in bpo-17552 [https://bugs.python.org/issue?@action=redirect&bpo=17552].)

The `socket.sendall()` method no longer resets the socket timeout every time bytes are received or sent. The socket timeout is now the maximum total duration to send all data. (Contributed by Victor Stinner in bpo-23853 [https://bugs.python.org/issue?@action=redirect&bpo=23853].)

The *backlog* argument of the `socket.listen()` method is now optional. By default it is set to `SOMAXCONN` or to `128`, whichever is less. (Contributed by Charles-François Natali in bpo-21455 [https://bugs.python.org/issue?@action=redirect&bpo=21455].)

ssl

Підтримка BIO пам'яті

(Contributed by Geert Jansen in bpo-21965 [https://bugs.python.org/issue?@action=redirect&bpo=21965].)

The new `SSLObject` class has been added to provide SSL protocol support for cases when the network I/O capabilities of `SSLSocket` are not necessary or are suboptimal. `SSLObject` represents an SSL protocol instance, but does not implement any network I/O methods, and instead provides a memory buffer interface. The new `MemoryBIO` class can be used to pass data between Python and an SSL protocol instance.

The memory BIO SSL support is primarily intended to be used in frameworks implementing asynchronous I/O for which `SSLSocket`'s readiness model ("select/poll") is inefficient.

A new `SSLContext.wrap_bio()` method can be used to create a new `SSLObject` instance.

Application-Layer Protocol Negotiation Support

(Contributed by Benjamin Peterson in bpo-20188 [https://bugs.python.org/issue?@action=redirect&bpo=20188].)

Where OpenSSL support is present, the `ssl` module now implements the *Application-Layer Protocol Negotiation* TLS extension as described in **RFC 7301** [https://datatracker.ietf.org/doc/html/rfc7301.html].

The new `SSLContext.set_alpn_protocols()` can be used to specify which protocols a socket should advertise during the TLS handshake.

The new `SSLSocket.selected_alpn_protocol()` returns the protocol that was selected during the TLS handshake. The `HAS_ALPN` flag indicates whether ALPN support is present.

Other Changes

There is a new `SSLSocket.version()` method to query the actual protocol version in use. (Contributed by Antoine Pitrou in bpo-20421 [https://bugs.python.org/issue?@action=redirect&bpo=20421].)

The `SSLSocket` class now implements a `SSLSocket.sendfile()` method. (Contributed by Giampaolo Rodola' in bpo-17552 [https://bugs.python.org/issue?@action=redirect&bpo=17552].)

The `SSLSocket.send()` method now raises either the `ssl.SSLWantReadError` or `ssl.SSLWantWriteError` exception on a non-blocking socket if the operation would block. Previously, it would return `0`. (Contributed by Nikolaus Rath in bpo-20951 [https://bugs.python.org/issue?@action=redirect&bpo=20951].)

The `cert_time_to_seconds()` function now interprets the input time as UTC and not as local time, per **RFC 5280** [https://datatracker.ietf.org/doc/html/rfc5280.html]. Additionally, the return value is always an `int`. (Contributed by Akira Li in bpo-19940 [https://bugs.python.org/issue?@action=redirect&bpo=19940].)

New `SSLObject.shared_ciphers()` and `SSLSocket.shared_ciphers()` methods return the list of ciphers sent by the client during the handshake. (Contributed by Benjamin Peterson in bpo-23186 [https://bugs.python.org/issue?@action=redirect&bpo=23186].)

The `SSLSocket.do_handshake()`, `SSLSocket.read()`, `SSLSocket.shutdown()`, and `SSLSocket.write()` methods of the `SSLSocket` class no longer reset the socket timeout every time bytes are received or sent. The socket timeout is now the maximum total duration of the method. (Contributed by Victor Stinner in bpo-23853 [https://bugs.python.org/issue?@action=redirect&bpo=23853].)

The `match_hostname()` function now supports matching of IP addresses. (Contributed by Antoine Pitrou in bpo-23239 [https://bugs.python.org/issue?@action=redirect&bpo=23239].)

sqlite3

The `Row` class now fully supports the sequence protocol, in particular `reversed()` iteration and slice indexing. (Contributed by Claudiu Popa in bpo-10203 [https://bugs.python.org/issue?@action=redirect&bpo=10203]; by Lucas Sinclair, Jessica McKellar, and Serhiy Storchaka in bpo-13583 [https://bugs.python.org/issue?@action=redirect&bpo=13583].)

subprocess

The new `run()` function has been added. It runs the specified command and returns a `CompletedProcess` object, which describes a finished process. The new API is more consistent and is the recommended approach to invoking subprocesses in Python code that does not need to maintain compatibility with earlier Python versions. (Contributed by Thomas Kluyver in bpo-23342 [https://bugs.python.org/issue?@action=redirect&bpo=23342].)

Exemplos:

```
>>> subprocess.run(["ls", "-l"])  # doesn't capture output
CompletedProcess(args=['ls', '-l'], returncode=0)

>>> subprocess.run("exit 1", shell=True, check=True)
Traceback (most recent call last):
  ...
subprocess.CalledProcessError: Command 'exit 1' returned non-zer
```

```
>>> subprocess.run(["ls", "-l", "/dev/null"], stdout=subprocess.
CompletedProcess(args=['ls', '-l', '/dev/null'], returncode=0,
stdout=b'crw-rw-rw- 1 root root 1, 3 Jan 23 16:23 /dev/null\n')
```

sys

A new **`set_coroutine_wrapper()`** function allows setting a global hook that will be called whenever a coroutine object is created by an **`async def`** function. A corresponding **`get_coroutine_wrapper()`** can be used to obtain a currently set wrapper. Both functions are provisional, and are intended for debugging purposes only. (Contributed by Yury Selivanov in bpo-24017 [https://bugs.python.org/issue?@action=redirect&bpo=24017].)

A new **`is_finalizing()`** function can be used to check if the Python interpreter is shutting down. (Contributed by Antoine Pitrou in bpo-22696 [https://bugs.python.org/issue?@action=redirect&bpo=22696].)

sysconfig

The name of the user scripts directory on Windows now includes the first two components of the Python version. (Contributed by Paul Moore in bpo-23437 [https://bugs.python.org/issue?@action=redirect&bpo=23437].)

tarfile

The *mode* argument of the **`open()`** function now accepts `"x"` to request exclusive creation. (Contributed by Berker Peksag in bpo-21717 [https://bugs.python.org/issue?@action=redirect&bpo=21717].)

The **`TarFile.extractall()`** and **`TarFile.extract()`** methods now take a keyword argument *numeric_owner*. If set to `True`, the extracted files and directories will be owned by the numeric `uid` and `gid` from the tarfile. If set to `False` (the default, and the behavior in versions prior to 3.5), they will be owned by the named user and group in the tarfile. (Contributed by Michael Vogt and Eric Smith in bpo-23193 [https://bugs.python.org/issue?@action=redirect&bpo=23193].)

The **`TarFile.list()`** now accepts an optional *members* keyword argument that can be set to a subset of the list returned by **`TarFile.getmembers()`**. (Contributed by Serhiy Storchaka in bpo-21549 [https://bugs.python.org/issue?@action=redirect&bpo=21549].)

threading

Both the **`Lock.acquire()`** and **`RLock.acquire()`** methods now use a monotonic clock for timeout management. (Contributed by Victor Stinner in bpo-22043 [https://bugs.python.org/issue?@action=redirect&bpo=22043].)

time

The **`monotonic()`** function is now always available. (Contributed by Victor Stinner in bpo-22043 [https://bugs.python.org/issue?@action=redirect&bpo=22043].)

timeit

A new command line option `-u` or `--unit=`*`U`* can be used to specify the time unit for the timer output. Supported options are `usec`, `msec`, or `sec`. (Contributed by Julian Gindi in bpo-18983 [https://bugs.python.org/issue?@action=redirect&bpo=18983].)

The **`timeit()`** function has a new *globals* parameter for specifying the namespace in which the code will be running. (Contributed by Ben Roberts in bpo-2527 [https://bugs.python.org/issue?@action=redirect&bpo=2527].)

tkinter

The **`tkinter._fix`** module used for setting up the Tcl/Tk environment on Windows has been replaced by a private function in the **`_tkinter`** module which makes no permanent changes to environment variables. (Contributed by Zachary Ware in bpo-20035 [https://bugs.python.org/issue?@action=redirect&bpo=20035].)

traceback

New `walk_stack()` and `walk_tb()` functions to conveniently traverse frame and traceback objects. (Contributed by Robert Collins in bpo-17911 [https://bugs.python.org/issue?@action=redirect&bpo=17911].)

New lightweight classes: `TracebackException`, `StackSummary`, and `FrameSummary`. (Contributed by Robert Collins in bpo-17911 [https://bugs.python.org/issue?@action=redirect&bpo=17911].)

Both the `print_tb()` and `print_stack()` functions now support negative values for the *limit* argument. (Contributed by Dmitry Kazakov in bpo-22619 [https://bugs.python.org/issue?@action=redirect&bpo=22619].)

types

A new `coroutine()` function to transform generator and `generator-like` objects into awaitables. (Contributed by Yury Selivanov in bpo-24017 [https://bugs.python.org/issue?@action=redirect&bpo=24017].)

A new type called `CoroutineType`, which is used for coroutine objects created by `async def` functions. (Contributed by Yury Selivanov in bpo-24400 [https://bugs.python.org/issue?@action=redirect&bpo=24400].)

unicodedata

The `unicodedata` module now uses data from Unicode 8.0.0 [https://unicode.org/versions/Unicode8.0.0/].

unittest

The `TestLoader.loadTestsFromModule()` method now accepts a keyword-only argument *pattern* which is passed to `load_tests` as the third argument. Found packages are now checked for `load_tests` regardless of whether their path matches *pattern*, because it is impossible for a package name to match the default pattern. (Contributed by Robert Collins and Barry A. Warsaw in bpo-16662 [https://bugs.python.org/issue?@action=redirect&bpo=16662].)

Unittest discovery errors now are exposed in the `TestLoader.errors` attribute of the `TestLoader` instance. (Contributed by Robert Collins in bpo-19746 [https://bugs.python.org/issue?@action=redirect&bpo=19746].)

A new command line option `--locals` to show local variables in tracebacks. (Contributed by Robert Collins in bpo-22936 [https://bugs.python.org/issue?@action=redirect&bpo=22936].)

unittest.mock

The `Mock` class has the following improvements:

- The class constructor has a new *unsafe* parameter, which causes mock objects to raise `AttributeError` on attribute names starting with `"assert"`. (Contributed by Kushal Das in bpo-21238 [https://bugs.python.org/issue?@action=redirect&bpo=21238].)
- A new `Mock.assert_not_called()` method to check if the mock object was called. (Contributed by Kushal Das in bpo-21262 [https://bugs.python.org/issue?@action=redirect&bpo=21262].)

The `MagicMock` class now supports `__truediv__()`, `__divmod__()` and `__matmul__()` operators. (Contributed by Johannes Baiter in bpo-20968 [https://bugs.python.org/issue?@action=redirect&bpo=20968], and Håkan Lövdahl in bpo-23581 [https://bugs.python.org/issue?@action=redirect&bpo=23581] and bpo-23568 [https://bugs.python.org/issue?@action=redirect&bpo=23568].)

It is no longer necessary to explicitly pass `create=True` to the `patch()` function when patching builtin names. (Contributed by Kushal Das in bpo-17660 [https://bugs.python.org/issue?@action=redirect&bpo=17660].)

urllib

A new `request.HTTPPasswordMgrWithPriorAuth` class allows HTTP Basic Authentication credentials to be managed so as to eliminate unnecessary `401` response handling, or to unconditionally send credentials on the first request in order to communicate with servers that return a `404` response instead of a `401` if the `Authorization` header is not sent. (Contributed by

Matej Cepl in bpo-19494 [https://bugs.python.org/issue?@action=redirect&bpo=19494] and Akshit Khurana in bpo-7159 [https://bugs.python.org/issue?@action=redirect&bpo=7159].)

A new *quote_via* argument for the `parse.urlencode()` function provides a way to control the encoding of query parts if needed. (Contributed by Samwyse and Arnon Yaari in bpo-13866 [https://bugs.python.org/issue?@action=redirect&bpo=13866].)

The `request.urlopen()` function accepts an `ssl.SSLContext` object as a *context* argument, which will be used for the HTTPS connection. (Contributed by Alex Gaynor in bpo-22366 [https://bugs.python.org/issue?@action=redirect&bpo=22366].)

The `parse.urljoin()` was updated to use the **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html] semantics for the resolution of relative URLs, rather than **RFC 1808** [https://datatracker.ietf.org/doc/html/rfc1808.html] and **RFC 2396** [https://datatracker.ietf.org/doc/html/rfc2396.html]. (Contributed by Demian Brecht and Senthil Kumaran in bpo-22118 [https://bugs.python.org/issue?@action=redirect&bpo=22118].)

wsgiref

The *headers* argument of the `headers.Headers` class constructor is now optional. (Contributed by Pablo Torres Navarrete and SilentGhost in bpo-5800 [https://bugs.python.org/issue?@action=redirect&bpo=5800].)

xmlrpc

The `client.ServerProxy` class now supports the context manager protocol. (Contributed by Claudiu Popa in bpo-20627 [https://bugs.python.org/issue?@action=redirect&bpo=20627].)

The `client.ServerProxy` constructor now accepts an optional `ssl.SSLContext` instance. (Contributed by Alex Gaynor in bpo-22960 [https://bugs.python.org/issue?@action=redirect&bpo=22960].)

xml.sax

SAX parsers now support a character stream of the `xmlreader.InputSource` object. (Contributed by Serhiy Storchaka in bpo-2175 [https://bugs.python.org/issue?@action=redirect&bpo=2175].)

`parseString()` now accepts a `str` instance. (Contributed by Serhiy Storchaka in bpo-10590 [https://bugs.python.org/issue?@action=redirect&bpo=10590].)

zipfile

ZIP output can now be written to unseekable streams. (Contributed by Serhiy Storchaka in bpo-23252 [https://bugs.python.org/issue?@action=redirect&bpo=23252].)

The *mode* argument of `ZipFile.open()` method now accepts `"x"` to request exclusive creation. (Contributed by Serhiy Storchaka in bpo-21717 [https://bugs.python.org/issue?@action=redirect&bpo=21717].)

Other module-level changes

Many functions in the `mmap`, `ossaudiodev`, `socket`, `ssl`, and `codecs` modules now accept writable bytes-like objects. (Contributed by Serhiy Storchaka in bpo-23001 [https://bugs.python.org/issue?@action=redirect&bpo=23001].)

Otimizações

The `os.walk()` function has been sped up by 3 to 5 times on POSIX systems, and by 7 to 20 times on Windows. This was done using the new `os.scandir()` function, which exposes file information from the underlying `readdir` or `FindFirstFile`/`FindNextFile` system calls. (Contributed by Ben Hoyt with help from Victor Stinner in bpo-23605 [https://bugs.python.org/issue?@action=redirect&bpo=23605].)

Construction of `bytes(int)` (filled by zero bytes) is faster and uses less memory for large objects. `calloc()` is used instead of `malloc()` to allocate

memory for these objects. (Contributed by Victor Stinner in bpo-21233 [https://bugs.python.org/issue?@action=redirect&bpo=21233].)

Some operations on `ipaddress` `IPv4Network` and `IPv6Network` have been massively sped up, such as `subnets()`, `supernet()`, `summarize_address_range()`, `collapse_addresses()`. The speed up can range from 3 to 15 times. (Contributed by Antoine Pitrou, Michel Albert, and Markus in bpo-21486 [https://bugs.python.org/issue?@action=redirect&bpo=21486], bpo-21487 [https://bugs.python.org/issue?@action=redirect&bpo=21487], bpo-20826 [https://bugs.python.org/issue?@action=redirect&bpo=20826], bpo-23266 [https://bugs.python.org/issue?@action=redirect&bpo=23266].)

Pickling of `ipaddress` objects was optimized to produce significantly smaller output. (Contributed by Serhiy Storchaka in bpo-23133 [https://bugs.python.org/issue?@action=redirect&bpo=23133].)

Many operations on `io.BytesIO` are now 50% to 100% faster. (Contributed by Serhiy Storchaka in bpo-15381 [https://bugs.python.org/issue?@action=redirect&bpo=15381] and David Wilson in bpo-22003 [https://bugs.python.org/issue?@action=redirect&bpo=22003].)

The `marshal.dumps()` function is now faster: 65–85% with versions 3 and 4, 20–25% with versions 0 to 2 on typical data, and up to 5 times in best cases. (Contributed by Serhiy Storchaka in bpo-20416 [https://bugs.python.org/issue?@action=redirect&bpo=20416] and bpo-23344 [https://bugs.python.org/issue?@action=redirect&bpo=23344].)

The UTF-32 encoder is now 3 to 7 times faster. (Contributed by Serhiy Storchaka in bpo-15027 [https://bugs.python.org/issue?@action=redirect&bpo=15027].)

Regular expressions are now parsed up to 10% faster. (Contributed by Serhiy Storchaka in bpo-19380 [https://bugs.python.org/issue?@action=redirect&bpo=19380].)

The `json.dumps()` function was optimized to run with `ensure_ascii=False` as fast as with `ensure_ascii=True`. (Contributed by Naoki Inada in bpo-23206 [https://bugs.python.org/issue?@action=redirect&bpo=23206].)

The `PyObject_IsInstance()` and `PyObject_IsSubclass()` functions have been sped up in the common case that the second argument has `type` as its metaclass. (Contributed Georg Brandl by in bpo-22540 [https://bugs.python.org/issue?@action=redirect&bpo=22540].)

Method caching was slightly improved, yielding up to 5% performance improvement in some benchmarks. (Contributed by Antoine Pitrou in bpo-22847 [https://bugs.python.org/issue?@action=redirect&bpo=22847].)

Objects from the `random` module now use 50% less memory on 64-bit builds. (Contributed by Serhiy Storchaka in bpo-23488 [https://bugs.python.org/issue?@action=redirect&bpo=23488].)

The `@property` getter calls are up to 25% faster. (Contributed by Joe Jevnik in bpo-23910 [https://bugs.python.org/issue?@action=redirect&bpo=23910].)

Instantiation of `fractions.Fraction` is now up to 30% faster. (Contributed by Stefan Behnel in bpo-22464 [https://bugs.python.org/issue?@action=redirect&bpo=22464].)

String methods `find()`, `rfind()`, `split()`, `partition()` and the `in` string operator are now significantly faster for searching 1-character substrings. (Contributed by Serhiy Storchaka in bpo-23573 [https://bugs.python.org/issue?@action=redirect&bpo=23573].)

Mudanças a construções e API C

New `calloc` functions were added:

- `PyMem_RawCalloc()`,
- `PyMem_Calloc()`,
- `PyObject_Calloc()`.

(Contribuição de Victor Stinner em bpo-21233 [https://bugs.python.org/issue?@action=redirect&bpo=21233].)

New encoding/decoding helper functions:

- `Py_DecodeLocale()` (replaced `_Py_char2wchar()`),
- `Py_EncodeLocale()` (replaced `_Py_wchar2char()`).

(Contribuição de Victor Stinner em bpo-18395 [https://bugs.python.org/issue?@action=redirect&bpo=18395].)

A new `PyCodec_NameReplaceErrors()` function to replace the unicode encode error with `\N{...}` escapes. (Contributed by Serhiy Storchaka in bpo-19676 [https://bugs.python.org/issue?@action=redirect&bpo=19676].)

A new `PyErr_FormatV()` function similar to `PyErr_Format()`, but accepts a **`va_list`** argument. (Contributed by Antoine Pitrou in bpo-18711 [https://bugs.python.org/issue?@action=redirect&bpo=18711].)

A new `PyExc_RecursionError` exception. (Contributed by Georg Brandl in bpo-19235 [https://bugs.python.org/issue?@action=redirect&bpo=19235].)

New `PyModule_FromDefAndSpec()`, `PyModule_FromDefAndSpec2()`, and `PyModule_ExecDef()` functions introduced by **PEP 489** [https://peps.python.org/pep-0489/] – multi-phase extension module initialization. (Contributed by Petr Viktorin in bpo-24268 [https://bugs.python.org/issue?@action=redirect&bpo=24268].)

New `PyNumber_MatrixMultiply()` and `PyNumber_InPlaceMatrixMultiply()` functions to perform matrix multiplication. (Contributed by Benjamin Peterson in bpo-21176 [https://bugs.python.org/issue?@action=redirect&bpo=21176]. See also **PEP 465** [https://peps.python.org/pep-0465/] for details.)

The `PyTypeObject.tp_finalize` slot is now part of the stable ABI.

Windows builds now require Microsoft Visual C++ 14.0, which is available as part of Visual Studio 2015 [https://visualstudio.microsoft.com/en/vs/older-downloads/#visual-studio-2015-and-other-products].

Extension modules now include a platform information tag in their filename on some platforms (the tag is optional, and CPython will import extensions

without it, although if the tag is present and mismatched, the extension won't be loaded):

- On Linux, extension module filenames end with `.cpython-<major><minor>m-<architecture>-<os>.pyd`:
  - `<major>` is the major number of the Python version; for Python 3.5 this is `3`.
  - `<minor>` is the minor number of the Python version; for Python 3.5 this is `5`.
  - `<architecture>` is the hardware architecture the extension module was built to run on. It's most commonly either `i386` for 32-bit Intel platforms or `x86_64` for 64-bit Intel (and AMD) platforms.
  - `<os>` is always `linux-gnu`, except for extensions built to talk to the 32-bit ABI on 64-bit platforms, in which case it is `linux-gnu32` (and `<architecture>` will be `x86_64`).
- On Windows, extension module filenames end with `<debug>.cp<major><minor>-<platform>.pyd`:
  - `<major>` is the major number of the Python version; for Python 3.5 this is `3`.
  - `<minor>` is the minor number of the Python version; for Python 3.5 this is `5`.
  - `<platform>` is the platform the extension module was built for, either `win32` for Win32, `win_amd64` for Win64, `win_ia64` for Windows Itanium 64, and `win_arm` for Windows on ARM.
  - If built in debug mode, `<debug>` will be `_d`, otherwise it will be blank.
- On OS X platforms, extension module filenames now end with `-darwin.so`.
- On all other platforms, extension module filenames are the same as they were with Python 3.4.

Descontinuados

New Keywords

`async` and `await` are not recommended to be used as variable, class, function or module names. Introduced by **PEP 492** [https://peps.python.org/pep-0492/] in Python 3.5, they will become proper keywords in Python 3.7.

Comportamento do Python descontinuado

Raising the **`StopIteration`** exception inside a generator will now generate a silent **`PendingDeprecationWarning`**, which will become a non-silent deprecation warning in Python 3.6 and will trigger a **`RuntimeError`** in Python 3.7. See PEP 479: Change StopIteration handling inside generators for details.

Unsupported Operating Systems

Windows XP is no longer supported by Microsoft, thus, per **PEP 11** [https://peps.python.org/pep-0011/], CPython 3.5 is no longer officially supported on this OS.

Deprecated Python modules, functions and methods

The **`formatter`** module has now graduated to full deprecation and is still slated for removal in Python 3.6.

The **`asyncio.async()`** function is deprecated in favor of **`ensure_future()`**.

The **`smtpd`** module has in the past always decoded the DATA portion of email messages using the `utf-8` codec. This can now be controlled by the new *decode_data* keyword to **`SMTPServer`**. The default value is `True`, but this default is deprecated. Specify the *decode_data* keyword with an appropriate value to avoid the deprecation warning.

Directly assigning values to the **`key`**, **`value`** and **`coded_value`** of **`http.cookies.Morsel`** objects is deprecated. Use the **`set()`** method instead. In addition, the undocumented *LegalChars* parameter of **`set()`** is deprecated, and is now ignored.

Passing a format string as keyword argument *format_string* to the **`format()`** method of the **`string.Formatter`** class has been deprecated. (Contributed by Serhiy Storchaka in bpo-23671 [https://bugs.python.org/issue?@action=redirect&bpo=23671].)

The **`platform.dist()`** and **`platform.linux_distribution()`** functions are now deprecated. Linux distributions use too many different ways of describing themselves, so the functionality is left to a package. (Contributed by Vajrasky Kok and Berker Peksag in bpo-1322 [https://bugs.python.org/issue?@action=redirect&bpo=1322].)

The previously undocumented `from_function` and `from_builtin` methods of **`inspect.Signature`** are deprecated. Use the new **`Signature.from_callable()`** method instead. (Contributed by Yury Selivanov in bpo-24248 [https://bugs.python.org/issue?@action=redirect&bpo=24248].)

The **`inspect.getargspec()`** function is deprecated and scheduled to be removed in Python 3.6. (See bpo-20438 [https://bugs.python.org/issue?@action=redirect&bpo=20438] for details.)

The **`inspect`** **`getfullargspec()`**, **`getcallargs()`**, and **`formatargspec()`** functions are deprecated in favor of the **`inspect.signature()`** API. (Contributed by Yury Selivanov in bpo-20438 [https://bugs.python.org/issue?@action=redirect&bpo=20438].)

`getargvalues()` and **`formatargvalues()`** functions were inadvertently marked as deprecated with the release of Python 3.5.0.

Use of **`re.LOCALE`** flag with str patterns or **`re.ASCII`** is now deprecated. (Contributed by Serhiy Storchaka in bpo-22407 [https://bugs.python.org/issue?@action=redirect&bpo=22407].)

Use of unrecognized special sequences consisting of `'\'` and an ASCII letter in regular expression patterns and replacement patterns now raises a deprecation warning and will be forbidden in Python 3.6. (Contributed by Serhiy Storchaka in bpo-23622 [https://bugs.python.org/issue?@action=redirect&bpo=23622].)

The undocumented and unofficial *use_load_tests* default argument of the `unittest.TestLoader.loadTestsFromModule()` method now is deprecated and ignored. (Contributed by Robert Collins and Barry A. Warsaw in bpo-16662 [https://bugs.python.org/issue?@action=redirect&bpo=16662].)

Removidos

Remoção de APIs e recursos

The following obsolete and previously deprecated APIs and features have been removed:

- The `__version__` attribute has been dropped from the email package. The email code hasn't been shipped separately from the stdlib for a long time, and the `__version__` string was not updated in the last few releases.
- The internal `Netrc` class in the `ftplib` module was deprecated in 3.4, and has now been removed. (Contributed by Matt Chaput in bpo-6623 [https://bugs.python.org/issue?@action=redirect&bpo=6623].)
- The concept of `.pyo` files has been removed.
- The JoinableQueue class in the provisional `asyncio` module was deprecated in 3.4.4 and is now removed. (Contributed by A. Jesse Jiryu Davis in bpo-23464 [https://bugs.python.org/issue?@action=redirect&bpo=23464].)

Porting to Python 3.5

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Alterações no comportamento do Python

- Due to an oversight, earlier Python versions erroneously accepted the following syntax:

```
f(1 for x in [1], *args)
f(1 for x in [1], **kwargs)
```

Python 3.5 now correctly raises a **SyntaxError**, as generator expressions must be put in parentheses if not a sole argument to a function.

Alterações na API Python

- **PEP 475** [https://peps.python.org/pep-0475/]: System calls are now retried when interrupted by a signal instead of raising **InterruptedError** if the Python signal handler does not raise an exception.
- Before Python 3.5, a **datetime.time** object was considered to be false if it represented midnight in UTC. This behavior was considered obscure and error-prone and has been removed in Python 3.5. See bpo-13936 [https://bugs.python.org/issue?@action=redirect&bpo=13936] for full details.
- The **ssl.SSLSocket.send()** method now raises either **ssl.SSLWantReadError** or **ssl.SSLWantWriteError** on a non-blocking socket if the operation would block. Previously, it would return `0`. (Contributed by Nikolaus Rath in bpo-20951 [https://bugs.python.org/issue?@action=redirect&bpo=20951].)
- The `__name__` attribute of generators is now set from the function name, instead of being set from the code name. Use `gen.gi_code.co_name` to retrieve the code name. Generators also have a new `__qualname__` attribute, the qualified name, which is now used for the representation of a generator (`repr(gen)`). (Contributed by Victor Stinner in bpo-21205 [https://bugs.python.org/issue?@action=redirect&bpo=21205].)
- The deprecated "strict" mode and argument of **HTMLParser**, **HTMLParser.error()**, and the **HTMLParserError** exception have been removed. (Contributed by Ezio Melotti in bpo-15114 [https://bugs.python.org/issue?@action=redirect&bpo=15114].) The *convert_charrefs* argument of **HTMLParser** is now `True` by default. (Contributed by Berker Peksag in bpo-21047 [https://bugs.python.org/issue?@action=redirect&bpo=21047].)
- Although it is not formally part of the API, it is worth noting for porting purposes (ie: fixing tests) that error messages that were

previously of the form "'sometype' does not support the buffer protocol" are now of the form "a bytes-like object is required, not 'sometype'". (Contributed by Ezio Melotti in bpo-16518 [https://bugs.python.org/issue?@action=redirect&bpo=16518].)

- If the current directory is set to a directory that no longer exists then `FileNotFoundError` will no longer be raised and instead `find_spec()` will return `None` **without** caching `None` in `sys.path_importer_cache`, which is different than the typical case (bpo-22834 [https://bugs.python.org/issue?@action=redirect&bpo=22834]).
- HTTP status code and messages from `http.client` and `http.server` were refactored into a common `HTTPStatus` enum. The values in `http.client` and `http.server` remain available for backwards compatibility. (Contributed by Demian Brecht in bpo-21793 [https://bugs.python.org/issue?@action=redirect&bpo=21793].)
- When an import loader defines `exec_module()` it is now expected to also define `create_module()` (raises a `DeprecationWarning` now, will be an error in Python 3.6). If the loader inherits from `importlib.abc.Loader` then there is nothing to do, else simply define `create_module()` to return `None`. (Contributed by Brett Cannon in bpo-23014 [https://bugs.python.org/issue?@action=redirect&bpo=23014].)
- The `re.split()` function always ignored empty pattern matches, so the `"x*"` pattern worked the same as `"x+"`, and the `"\b"` pattern never worked. Now `re.split()` raises a warning if the pattern could match an empty string. For compatibility, use patterns that never match an empty string (e.g. `"x+"` instead of `"x*"`). Patterns that could only match an empty string (such as `"\b"`) now raise an error. (Contributed by Serhiy Storchaka in bpo-22818 [https://bugs.python.org/issue?@action=redirect&bpo=22818].)
- The `http.cookies.Morsel` dict-like interface has been made self consistent: morsel comparison now takes the `key` and `value` into account, `copy()` now results in a `Morsel` instance rather than a `dict`, and `update()` will now raise an exception if any of the keys in the update dictionary are invalid. In addition, the undocumented *LegalChars* parameter of `set()` is deprecated and is now ignored. (Contributed by Demian Brecht in bpo-2211 [https://bugs.python.org/issue?@action=redirect&bpo=2211].)

- **PEP 488** [https://peps.python.org/pep-0488/] has removed `.pyo` files from Python and introduced the optional `opt-` tag in `.pyc` file names. The `importlib.util.cache_from_source()` has gained an *optimization* parameter to help control the `opt-` tag. Because of this, the *debug_override* parameter of the function is now deprecated. `.pyo` files are also no longer supported as a file argument to the Python interpreter and thus serve no purpose when distributed on their own (i.e. sourceless code distribution). Due to the fact that the magic number for bytecode has changed in Python 3.5, all old `.pyo` files from previous versions of Python are invalid regardless of this PEP.
- The `socket` module now exports the `CAN_RAW_FD_FRAMES` constant on linux 3.6 and greater.
- The `ssl.cert_time_to_seconds()` function now interprets the input time as UTC and not as local time, per **RFC 5280** [https://datatracker.ietf.org/doc/html/rfc5280.html]. Additionally, the return value is always an `int`. (Contributed by Akira Li in bpo-19940 [https://bugs.python.org/issue?@action=redirect&bpo=19940].)
- The `pygettext.py` Tool now uses the standard +NNNN format for timezones in the POT-Creation-Date header.
- The `smtplib` module now uses `sys.stderr` instead of the previous module-level **`stderr`** variable for debug output. If your (test) program depends on patching the module-level variable to capture the debug output, you will need to update it to capture sys.stderr instead.
- The `str.startswith()` and `str.endswith()` methods no longer return `True` when finding the empty string and the indexes are completely out of range. (Contributed by Serhiy Storchaka in bpo-24284 [https://bugs.python.org/issue?@action=redirect&bpo=24284].)
- The `inspect.getdoc()` function now returns documentation strings inherited from base classes. Documentation strings no longer need to be duplicated if the inherited documentation is appropriate. To suppress an inherited string, an empty string must be specified (or the documentation may be filled in). This change affects the output of the `pydoc` module and the `help()` function. (Contributed by Serhiy Storchaka in bpo-15582 [https://bugs.python.org/issue?@action=redirect&bpo=15582].)
- Nested `functools.partial()` calls are now flattened. If you were relying on the previous behavior, you can now either add an attribute

to a **`functools.partial()`** object or you can create a subclass of **`functools.partial()`**. (Contributed by Alexander Belopolsky in bpo-7830 [https://bugs.python.org/issue?@action=redirect&bpo=7830].)

Alterações na API C

- The undocumented **`format`** member of the (non-public) **`PyMemoryViewObject`** structure has been removed. All extensions relying on the relevant parts in `memoryobject.h` must be rebuilt.
- The **`PyMemAllocator`** structure was renamed to **`PyMemAllocatorEx`** and a new `calloc` field was added.
- Removed non-documented macro **`PyObject_REPR()`** which leaked references. Use format character `%R` in **`PyUnicode_FromFormat()`**-like functions to format the **`repr()`** of the object. (Contributed by Serhiy Storchaka in bpo-22453 [https://bugs.python.org/issue?@action=redirect&bpo=22453].)
- Because the lack of the **`__module__`** attribute breaks pickling and introspection, a deprecation warning is now raised for builtin types without the **`__module__`** attribute. This will be an **`AttributeError`** in the future. (Contributed by Serhiy Storchaka in bpo-20204 [https://bugs.python.org/issue?@action=redirect&bpo=20204].)
- As part of the **PEP 492** [https://peps.python.org/pep-0492/] implementation, the `tp_reserved` slot of **`PyTypeObject`** was replaced with a **`tp_as_async`** slot. Refer to Objetos corrotina for new types, structures and functions.

Notable changes in Python 3.5.4

Novo alvo de construção `make regen-all`

Para simplificar a compilação cruzada e garantir que o CPython possa ser compilado de forma confiável sem exigir que uma versão existente do Python já esteja disponível, o sistema de construção baseado em autotools não tenta mais recompilar implicitamente os arquivos gerados com base nos tempos de modificação dos arquivos.

Em vez disso, um novo comando `make regen-all` foi adicionado para forçar a regeneração desses arquivos quando desejado (por exemplo, após uma versão inicial do Python já ter sido construída com base nas versões pré-geradas).

Alvos de regeneração mais seletivos também são definidos – veja Makefile.pre.in [https://github.com/python/cpython/tree/3.15/Makefile.pre.in] para detalhes.

(Contribuição de Victor Stinner em bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404].)

Adicionado na versão 3.5.4.

Remoção do alvo de construção `make touch`

O alvo de construção `make touch` usado anteriormente para solicitar regeneração implícita de arquivos gerados atualizando seus tempos de modificação foi removido.

Ele foi substituído pelo novo alvo `make regen-all`.

(Contribuição de Victor Stinner em bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404].)

Alterado na versão 3.5.4.

O que há de novo no Python 3.4

Autor:: R. David Murray <rdmurray@bitdance.com> (Editor)

This article explains the new features in Python 3.4, compared to 3.3. Python 3.4 was released on March 16, 2014. For full details, see the changelog [https://docs.python.org/3.4/whatsnew/changelog.html].

Ver também

PEP 429 [https://peps.python.org/pep-0429/] – Python 3.4 Release Schedule

Resumo – Destaques da versão

Novos recursos de sintaxe:

- Não foram adicionados novos recursos de sintaxe no Python 3.4.

Outras novas funcionalidades:

- pip should always be available (**PEP 453** [https://peps.python.org/pep-0453/]).
- Newly created file descriptors are non-inheritable (**PEP 446** [https://peps.python.org/pep-0446/]).
- command line option for isolated mode (bpo-16499 [https://bugs.python.org/issue?@action=redirect&bpo=16499]).
- improvements in the handling of codecs that are not text encodings (multiple issues).
- A ModuleSpec Type for the Import System (**PEP 451** [https://peps.python.org/pep-0451/]). (Affects importer authors.)
- The `marshal` format has been made more compact and efficient (bpo-16475 [https://bugs.python.org/issue?@action=redirect&bpo=16475]).

Novos módulos de biblioteca:

- `asyncio`: New provisional API for asynchronous IO (**PEP 3156** [https://peps.python.org/pep-3156/]).
- `ensurepip`: Bootstrapping the pip installer (**PEP 453** [https://peps.python.org/pep-0453/]).
- `enum`: Support for enumeration types (**PEP 435** [https://peps.python.org/pep-0435/]).
- `pathlib`: Object-oriented filesystem paths (**PEP 428** [https://peps.python.org/pep-0428/]).
- `selectors`: High-level and efficient I/O multiplexing, built upon the `select` module primitives (part of **PEP 3156** [https://peps.python.org/pep-3156/]).
- `statistics`: A basic numerically stable statistics library (**PEP 450** [https://peps.python.org/pep-0450/]).
- `tracemalloc`: Trace Python memory allocations (**PEP 454** [https://peps.python.org/pep-0454/]).

Módulos de biblioteca significativamente melhorados:

- Single-dispatch generic functions in `functools` (**PEP 443** [https://peps.python.org/pep-0443/]).
- New `pickle` protocol 4 (**PEP 3154** [https://peps.python.org/pep-3154/]).
- `multiprocessing` now has an option to avoid using os.fork on Unix (bpo-8713 [https://bugs.python.org/issue?@action=redirect&bpo=8713]).
- `email` has a new submodule, `contentmanager`, and a new `Message` subclass (`EmailMessage`) that simplify MIME handling (bpo-18891 [https://bugs.python.org/issue?@action=redirect&bpo=18891]).
- The `inspect` and `pydoc` modules are now capable of correct introspection of a much wider variety of callable objects, which improves the output of the Python `help()` system.
- The `ipaddress` module API has been declared stable

Melhorias de segurança:

- Secure and interchangeable hash algorithm (**PEP 456** [https://peps.python.org/pep-0456/]).
- Make newly created file descriptors non-inheritable (**PEP 446** [https://peps.python.org/pep-0446/]) to avoid leaking file descriptors to child processes.

- New command line option for isolated mode, (bpo-16499 [https://bugs.python.org/issue?@action=redirect&bpo=16499]).
- `multiprocessing` now has an option to avoid using os.fork on Unix. *spawn* and *forkserver* are more secure because they avoid sharing data with child processes.
- `multiprocessing` child processes on Windows no longer inherit all of the parent's inheritable handles, only the necessary ones.
- A new `hashlib.pbkdf2_hmac()` function provides the PKCS#5 password-based key derivation function 2 [https://en.wikipedia.org/wiki/PBKDF2].
- TLSv1.1 and TLSv1.2 support for `ssl`.
- Retrieving certificates from the Windows system cert store support for `ssl`.
- Server-side SNI (Server Name Indication) support for `ssl`.
- The `ssl.SSLContext` class has a lot of improvements.
- All modules in the standard library that support SSL now support server certificate verification, including hostname matching (`ssl.match_hostname()`) and CRLs (Certificate Revocation lists, see `ssl.SSLContext.load_verify_locations()`).

Melhorias na implementação do CPython:

- Safe object finalization (**PEP 442** [https://peps.python.org/pep-0442/]).
- Leveraging **PEP 442** [https://peps.python.org/pep-0442/], in most cases module globals are no longer set to None during finalization (bpo-18214 [https://bugs.python.org/issue?@action=redirect&bpo=18214]).
- Configurable memory allocators (**PEP 445** [https://peps.python.org/pep-0445/]).
- Argument Clinic (**PEP 436** [https://peps.python.org/pep-0436/]).

Please read on for a comprehensive list of user-facing changes, including many other smaller improvements, CPython optimizations, deprecations, and potential porting issues.

Novas funcionalidades

PEP 453: Explicit Bootstrapping of PIP in Python Installations

Bootstrapping pip By Default

The new **ensurepip** module (defined in **PEP 453** [https://peps.python.org/pep-0453/]) provides a standard cross-platform mechanism to bootstrap the pip installer into Python installations and virtual environments. The version of `pip` included with Python 3.4.0 is `pip` 1.5.4, and future 3.4.x maintenance releases will update the bundled version to the latest version of `pip` that is available at the time of creating the release candidate.

By default, the commands `pipX` and `pipX.Y` will be installed on all platforms (where X.Y stands for the version of the Python installation), along with the `pip` Python package and its dependencies. On Windows and in virtual environments on all platforms, the unversioned `pip` command will also be installed. On other platforms, the system wide unversioned `pip` command typically refers to the separately installed Python 2 version.

The `pyvenv` command line utility and the **venv** module make use of the **ensurepip** module to make `pip` readily available in virtual environments. When using the command line utility, `pip` is installed by default, while when using the **venv** module API installation of `pip` must be requested explicitly.

For CPython source builds on POSIX systems, the `make install` and `make altinstall` commands bootstrap `pip` by default. This behaviour can be controlled through configure options, and overridden through Makefile options.

On Windows and Mac OS X, the CPython installers now default to installing `pip` along with CPython itself (users may opt out of installing it during the installation process). Window users will need to opt in to the automatic `PATH` modifications to have `pip` available from the command line by default, otherwise it can still be accessed through the Python launcher for Windows as `py -m pip`.

As **discussed in the PEP** [https://peps.python.org/pep-0453/#recommendations-for-downstream-distributors] platform packagers may choose not to install these commands by default, as long as, when invoked, they provide clear and

simple directions on how to install them on that platform (usually using the system package manager).

Nota

To avoid conflicts between parallel Python 2 and Python 3 installations, only the versioned `pip3` and `pip3.4` commands are bootstrapped by default when `ensurepip` is invoked directly - the `--default-pip` option is needed to also request the unversioned `pip` command. `pyvenv` and the Windows installer ensure that the unqualified `pip` command is made available in those environments, and `pip` can always be invoked via the `-m` switch rather than directly to avoid ambiguity on systems with multiple Python installations.

Mudanças na documentação

As part of this change, the Instalando módulos Python and Distribuindo módulos Python sections of the documentation have been completely redesigned as short getting started and FAQ documents. Most packaging documentation has now been moved out to the Python Packaging Authority maintained Python Packaging User Guide [https://packaging.python.org] and the documentation of the individual projects.

No entanto, como esta migração ainda está incompleta, as versões legadas desses guias permanecem disponíveis como Construindo extensões C e C ++ com setuptools e Construindo extensões C e C ++ com setuptools.

Ver também

PEP 453 [https://peps.python.org/pep-0453/] – Explicit bootstrapping of pip in Python installations

PEP written by Donald Stufft and Nick Coghlan, implemented by Donald Stufft, Nick Coghlan, Martin von Löwis and Ned Deily.

PEP 446: Newly Created File Descriptors Are Non-Inheritable

PEP 446 [https://peps.python.org/pep-0446/] makes newly created file descriptors non-inheritable. In general, this is the behavior an application will want: when launching a new process, having currently open files also open in the new process can lead to all sorts of hard to find bugs, and potentially to security issues.

However, there are occasions when inheritance is desired. To support these cases, the following new functions and methods are available:

- `os.get_inheritable()`, `os.set_inheritable()`
- `os.get_handle_inheritable()`, `os.set_handle_inheritable()`
- `socket.socket.get_inheritable()`, `socket.socket.set_inheritable()`

Ver também

PEP 446 [https://peps.python.org/pep-0446/] – Make newly created file descriptors non-inheritable

PEP escrito e implementado por Victor Stinner.

Melhorias no manuseio de Codecs

Since it was first introduced, the `codecs` module has always been intended to operate as a type-neutral dynamic encoding and decoding system. However, its close coupling with the Python text model, especially the type restricted convenience methods on the builtin `str`, `bytes` and `bytearray` types, has historically obscured that fact.

As a key step in clarifying the situation, the `codecs.encode()` and `codecs.decode()` convenience functions are now properly documented in Python 2.7, 3.3 and 3.4. These functions have existed in the `codecs` module (and have been covered by the regression test suite) since Python 2.4, but were previously only discoverable through runtime introspection.

Unlike the convenience methods on **str**, **bytes** and **bytearray**, the **codecs** convenience functions support arbitrary codecs in both Python 2 and Python 3, rather than being limited to Unicode text encodings (in Python 3) or `basestring` <-> `basestring` conversions (in Python 2).

In Python 3.4, the interpreter is able to identify the known non-text encodings provided in the standard library and direct users towards these general purpose convenience functions when appropriate:

```
>>> b"abcdef".decode("hex")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
LookupError: 'hex' is not a text encoding; use codecs.decode() t

>>> "hello".encode("rot13")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
LookupError: 'rot13' is not a text encoding; use codecs.encode()

>>> open("foo.txt", encoding="hex")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
LookupError: 'hex' is not a text encoding; use codecs.open() to
```

In a related change, whenever it is feasible without breaking backwards compatibility, exceptions raised during encoding and decoding operations are wrapped in a chained exception of the same type that mentions the name of the codec responsible for producing the error:

```
>>> import codecs

>>> codecs.decode(b"abcdefgh", "hex")
Traceback (most recent call last):
  File "/usr/lib/python3.4/encodings/hex_codec.py", line 20, in
    return (binascii.a2b_hex(input), len(input))
binascii.Error: Non-hexadecimal digit found

The above exception was the direct cause of the following except

Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
binascii.Error: decoding with 'hex' codec failed (Error: Non-hex

>>> codecs.encode("hello", "bz2")
Traceback (most recent call last):
  File "/usr/lib/python3.4/encodings/bz2_codec.py", line 17, in
    return (bz2.compress(input), len(input))
  File "/usr/lib/python3.4/bz2.py", line 498, in compress
    return comp.compress(data) + comp.flush()
TypeError: 'str' does not support the buffer interface

The above exception was the direct cause of the following except

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: encoding with 'bz2' codec failed (TypeError: 'str' do
```

Finally, as the examples above show, these improvements have permitted the restoration of the convenience aliases for the non-Unicode codecs that were themselves restored in Python 3.2. This means that encoding binary data to and from its hexadecimal representation (for example) can now be written as:

```
>>> from codecs import encode, decode
>>> encode(b"hello", "hex")
b'68656c6c6f'
>>> decode(b"68656c6c6f", "hex")
b'hello'
```

The binary and text transforms provided in the standard library are detailed in Binary Transforms and Text Transforms.

(Contributed by Nick Coghlan in bpo-7475 [https://bugs.python.org/issue?@action=redirect&bpo=7475], bpo-17827 [https://bugs.python.org/issue?@action=redirect&bpo=17827], bpo-17828 [https://bugs.python.org/issue?@action=redirect&bpo=17828] and bpo-19619 [https://bugs.python.org/issue?@action=redirect&bpo=19619].)

PEP 451: A ModuleSpec Type for the Import System

PEP 451 [https://peps.python.org/pep-0451/] provides an encapsulation of the information about a module that the import machinery will use to load it (that is, a module specification). This helps simplify both the import implementation and several import-related APIs. The change is also a stepping stone for several future import-related improvements [https://mail.python.org/pipermail/python-dev/2013-November/130111.html].

The public-facing changes from the PEP are entirely backward-compatible. Furthermore, they should be transparent to everyone but importer authors. Key finder and loader methods have been deprecated, but they will continue working. New importers should use the new methods described in the PEP. Existing importers should be updated to implement the new methods. See the Descontinuados section for a list of methods that should be replaced and their replacements.

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- Unicode database updated to UCD version 6.3.
- `min()` and `max()` now accept a *default* keyword-only argument that can be used to specify the value they return if the iterable they are evaluating has no elements. (Contributed by Julian Berman in bpo-18111 [https://bugs.python.org/issue?@action=redirect&bpo=18111].)
- Objetos módulo são agora fracamente referenciáveis.
- Module `__file__` attributes (and related values) should now always contain absolute paths by default, with the sole exception of `__main__.__file__` when a script has been executed directly using a relative path. (Contributed by Brett Cannon in bpo-18416 [https://bugs.python.org/issue?@action=redirect&bpo=18416].)
- All the UTF-* codecs (except UTF-7) now reject surrogates during both encoding and decoding unless the `surrogatepass` error handler is used, with the exception of the UTF-16 decoder (which accepts valid surrogate pairs) and the UTF-16 encoder (which produces them while encoding non-BMP characters). (Contributed by Victor Stinner, Kang-Hao (Kenny) Lu and Serhiy Storchaka in bpo-12892 [https://bugs.python.org/issue?@action=redirect&bpo=12892].)

- New German EBCDIC codec `cp273`. (Contributed by Michael Bierenfeld and Andrew Kuchling in bpo-1097797 [https://bugs.python.org/issue?@action=redirect&bpo=1097797].)
- New Ukrainian codec `cp1125`. (Contributed by Serhiy Storchaka in bpo-19668 [https://bugs.python.org/issue?@action=redirect&bpo=19668].)
- `bytes`.join() and `bytearray`.join() now accept arbitrary buffer objects as arguments. (Contributed by Antoine Pitrou in bpo-15958 [https://bugs.python.org/issue?@action=redirect&bpo=15958].)
- The `int` constructor now accepts any object that has an `__index__` method for its *base* argument. (Contributed by Mark Dickinson in bpo-16772 [https://bugs.python.org/issue?@action=redirect&bpo=16772].)
- Frame objects now have a `clear()` method that clears all references to local variables from the frame. (Contributed by Antoine Pitrou in bpo-17934 [https://bugs.python.org/issue?@action=redirect&bpo=17934].)
- `memoryview` is now registered as a `Sequence`, and supports the `reversed()` builtin. (Contributed by Nick Coghlan and Claudiu Popa in bpo-18690 [https://bugs.python.org/issue?@action=redirect&bpo=18690] and bpo-19078 [https://bugs.python.org/issue?@action=redirect&bpo=19078].)
- Signatures reported by `help()` have been modified and improved in several cases as a result of the introduction of Argument Clinic and other changes to the `inspect` and `pydoc` modules.
- `__length_hint__()` is now part of the formal language specification (see **PEP 424** [https://peps.python.org/pep-0424/]). (Contributed by Armin Ronacher in bpo-16148 [https://bugs.python.org/issue?@action=redirect&bpo=16148].)

Novos módulos

asyncio

The new `asyncio` module (defined in **PEP 3156** [https://peps.python.org/pep-3156/]) provides a standard pluggable event loop model for Python, providing solid asynchronous IO support in the standard library, and making it easier for other event loop implementations to interoperate with the standard library and each other.

For Python 3.4, this module is considered a provisional API.

Ver também

PEP 3156 [https://peps.python.org/pep-3156/] – Asynchronous IO Support Rebooted: the "asyncio" Module

PEP written and implementation led by Guido van Rossum.

ensurepip

The new `ensurepip` module is the primary infrastructure for the **PEP 453** [https://peps.python.org/pep-0453/] implementation. In the normal course of events end users will not need to interact with this module, but it can be used to manually bootstrap `pip` if the automated bootstrapping into an installation or virtual environment was declined.

`ensurepip` includes a bundled copy of `pip`, up-to-date as of the first release candidate of the release of CPython with which it ships (this applies to both maintenance releases and feature releases). `ensurepip` does not access the internet. If the installation has Internet access, after `ensurepip` is run the bundled `pip` can be used to upgrade `pip` to a more recent release than the bundled one. (Note that such an upgraded version of `pip` is considered to be a separately installed package and will not be removed if Python is uninstalled.)

The module is named *ensure*pip because if called when `pip` is already installed, it does nothing. It also has an `--upgrade` option that will cause it to install the bundled copy of `pip` if the existing installed version of `pip` is older than the bundled copy.

enum

The new `enum` module (defined in **PEP 435** [https://peps.python.org/pep-0435/]) provides a standard implementation of enumeration types, allowing other modules (such as `socket`) to provide more informative error messages and better debugging support by replacing opaque integer constants with backwards compatible enumeration values.

Ver também

PEP 435 **[https://peps.python.org/pep-0435/]** **– Adding an Enum type to the Python standard library**

PEP written by Barry Warsaw, Eli Bendersky and Ethan Furman, implemented by Ethan Furman.

pathlib

The new `pathlib` module offers classes representing filesystem paths with semantics appropriate for different operating systems. Path classes are divided between *pure paths*, which provide purely computational operations without I/O, and *concrete paths*, which inherit from pure paths but also provide I/O operations.

For Python 3.4, this module is considered a provisional API.

Ver também

PEP 428 **[https://peps.python.org/pep-0428/]** **– The pathlib module – object-oriented filesystem paths**

PEP escrita e implementada por Antoine Pitrou.

selectors

The new `selectors` module (created as part of implementing **PEP 3156** [https://peps.python.org/pep-3156/]) allows high-level and efficient I/O multiplexing, built upon the `select` module primitives.

statistics

The new `statistics` module (defined in **PEP 450** [https://peps.python.org/pep-0450/]) offers some core statistics functionality directly in the standard

library. This module supports calculation of the mean, median, mode, variance and standard deviation of a data series.

Ver também

PEP 450 [https://peps.python.org/pep-0450/] – Adding A Statistics Module To The Standard Library
PEP written and implemented by Steven D'Aprano

tracemalloc

The new `tracemalloc` module (defined in **PEP 454** [https://peps.python.org/pep-0454/]) is a debug tool to trace memory blocks allocated by Python. It provides the following information:

- Trace where an object was allocated
- Statistics on allocated memory blocks per filename and per line number: total size, number and average size of allocated memory blocks
- Compute the differences between two snapshots to detect memory leaks

Ver também

PEP 454 [https://peps.python.org/pep-0454/] – Add a new tracemalloc module to trace Python memory allocations
PEP escrita e implementada por Victor Stinner

Módulos melhorados

abc

New function `abc.get_cache_token()` can be used to know when to invalidate caches that are affected by changes in the object graph.

(Contributed by Łukasz Langa in bpo-16832 [https://bugs.python.org/issue?@action=redirect&bpo=16832].)

New class **ABC** has **ABCMeta** as its meta class. Using `ABC` as a base class has essentially the same effect as specifying `metaclass=abc.ABCMeta`, but is simpler to type and easier to read. (Contributed by Bruno Dupuis in bpo-16049 [https://bugs.python.org/issue?@action=redirect&bpo=16049].)

aifc

The **getparams()** method now returns a namedtuple rather than a plain tuple. (Contributed by Claudiu Popa in bpo-17818 [https://bugs.python.org/issue?@action=redirect&bpo=17818].)

aifc.open() now supports the context management protocol: when used in a **with** block, the **close()** method of the returned object will be called automatically at the end of the block. (Contributed by Serhiy Storchacha in bpo-16486 [https://bugs.python.org/issue?@action=redirect&bpo=16486].)

The **writeframesraw()** and **writeframes()** methods now accept any bytes-like object. (Contributed by Serhiy Storchaka in bpo-8311 [https://bugs.python.org/issue?@action=redirect&bpo=8311].)

argparse

The **FileType** class now accepts *encoding* and *errors* arguments, which are passed through to **open()**. (Contributed by Lucas Maystre in bpo-11175 [https://bugs.python.org/issue?@action=redirect&bpo=11175].)

audioop

audioop now supports 24-bit samples. (Contributed by Serhiy Storchaka in bpo-12866 [https://bugs.python.org/issue?@action=redirect&bpo=12866].)

New **byteswap()** function converts big-endian samples to little-endian and vice versa. (Contributed by Serhiy Storchaka in bpo-19641 [https://bugs.python.org/issue?@action=redirect&bpo=19641].)

All **audioop** functions now accept any bytes-like object. Strings are not accepted: they didn't work before, now they raise an error right away. (Contributed by Serhiy Storchaka in bpo-16685 [https://bugs.python.org/issue?@action=redirect&bpo=16685].)

base64

The encoding and decoding functions in **base64** now accept any bytes-like object in cases where it previously required a **bytes** or **bytearray** instance. (Contributed by Nick Coghlan in bpo-17839 [https://bugs.python.org/issue?@action=redirect&bpo=17839].)

New functions **a85encode()**, **a85decode()**, **b85encode()**, and **b85decode()** provide the ability to encode and decode binary data from and to `Ascii85` and the git/mercurial `Base85` formats, respectively. The `a85` functions have options that can be used to make them compatible with the variants of the `Ascii85` encoding, including the Adobe variant. (Contributed by Martin Morrison, the Mercurial project, Serhiy Storchaka, and Antoine Pitrou in bpo-17618 [https://bugs.python.org/issue?@action=redirect&bpo=17618].)

collections

The **ChainMap.new_child()** method now accepts an *m* argument specifying the child map to add to the chain. This allows an existing mapping and/or a custom mapping type to be used for the child. (Contributed by Vinay Sajip in bpo-16613 [https://bugs.python.org/issue?@action=redirect&bpo=16613].)

colorsys

The number of digits in the coefficients for the RGB — YIQ conversions have been expanded so that they match the FCC NTSC versions. The change in results should be less than 1% and may better match results found elsewhere. (Contributed by Brian Landers and Serhiy Storchaka in bpo-14323 [https://bugs.python.org/issue?@action=redirect&bpo=14323].)

contextlib

The new **`contextlib.suppress`** context manager helps to clarify the intent of code that deliberately suppresses exceptions from a single statement. (Contributed by Raymond Hettinger in bpo-15806 [https://bugs.python.org/issue?@action=redirect&bpo=15806] and Zero Piraeus in bpo-19266 [https://bugs.python.org/issue?@action=redirect&bpo=19266].)

The new **`contextlib.redirect_stdout()`** context manager makes it easier for utility scripts to handle inflexible APIs that write their output to **`sys.stdout`** and don't provide any options to redirect it. Using the context manager, the **`sys.stdout`** output can be redirected to any other stream or, in conjunction with **`io.StringIO`**, to a string. The latter can be especially useful, for example, to capture output from a function that was written to implement a command line interface. It is recommended only for utility scripts because it affects the global state of **`sys.stdout`**. (Contributed by Raymond Hettinger in bpo-15805 [https://bugs.python.org/issue?@action=redirect&bpo=15805].)

The **`contextlib`** documentation has also been updated to include a discussion of the differences between single use, reusable and reentrant context managers.

dbm

`dbm.open()` objects now support the context management protocol. When used in a **`with`** statement, the `close` method of the database object will be called automatically at the end of the block. (Contributed by Claudiu Popa and Nick Coghlan in bpo-19282 [https://bugs.python.org/issue?@action=redirect&bpo=19282].)

dis

Functions **`show_code()`**, **`dis()`**, **`distb()`**, and **`disassemble()`** now accept a keyword-only *file* argument that controls where they write their output.

The **`dis`** module is now built around an **`Instruction`** class that provides object oriented access to the details of each individual bytecode operation.

A new method, **get_instructions()**, provides an iterator that emits the Instruction stream for a given piece of Python code. Thus it is now possible to write a program that inspects and manipulates a bytecode object in ways different from those provided by the **dis** module itself. For example:

```
>>> import dis
>>> for instr in dis.get_instructions(lambda x: x + 1):
...     print(instr.opname)
LOAD_FAST
LOAD_CONST
BINARY_ADD
RETURN_VALUE
```

The various display tools in the **dis** module have been rewritten to use these new components.

In addition, a new application-friendly class **Bytecode** provides an object-oriented API for inspecting bytecode in both in human-readable form and for iterating over instructions. The **Bytecode** constructor takes the same arguments that **get_instructions()** does (plus an optional *current_offset*), and the resulting object can be iterated to produce **Instruction** objects. But it also has a **dis** method, equivalent to calling **dis** on the constructor argument, but returned as a multi-line string:

```
>>> bytecode = dis.Bytecode(lambda x: x + 1, current_offset=3)
>>> for instr in bytecode:
...     print('{} ({})'.format(instr.opname, instr.opcode))
LOAD_FAST (124)
LOAD_CONST (100)
BINARY_ADD (23)
RETURN_VALUE (83)
>>> bytecode.dis().splitlines()
['  1           0 LOAD_FAST                0 (x)',
 '      -->     3 LOAD_CONST               1 (1)',
 '              6 BINARY_ADD',
 '              7 RETURN_VALUE']
```

Bytecode also has a class method, **from_traceback()**, that provides the ability to manipulate a traceback (that is,

`print(Bytecode.from_traceback(tb).dis())` is equivalent to `distb(tb)`).

(Contributed by Nick Coghlan, Ryan Kelly and Thomas Kluyver in bpo-11816 [https://bugs.python.org/issue?@action=redirect&bpo=11816] and Claudiu Popa in bpo-17916 [https://bugs.python.org/issue?@action=redirect&bpo=17916].)

New function **`stack_effect()`** computes the effect on the Python stack of a given opcode and argument, information that is not otherwise available. (Contributed by Larry Hastings in bpo-19722 [https://bugs.python.org/issue?@action=redirect&bpo=19722].)

doctest

A new option flag, **`FAIL_FAST`**, halts test running as soon as the first failure is detected. (Contributed by R. David Murray and Daniel Urban in bpo-16522 [https://bugs.python.org/issue?@action=redirect&bpo=16522].)

The **`doctest`** command line interface now uses **`argparse`**, and has two new options, `-o` and `-f`. `-o` allows doctest options to be specified on the command line, and `-f` is a shorthand for `-o FAIL_FAST` (to parallel the similar option supported by the **`unittest`** CLI). (Contributed by R. David Murray in bpo-11390 [https://bugs.python.org/issue?@action=redirect&bpo=11390].)

`doctest` will now find doctests in extension module `__doc__` strings. (Contributed by Zachary Ware in bpo-3158 [https://bugs.python.org/issue?@action=redirect&bpo=3158].)

email

`as_string()` now accepts a *policy* argument to override the default policy of the message when generating a string representation of it. This means that `as_string` can now be used in more circumstances, instead of having to create and use a **`generator`** in order to pass formatting parameters to its `flatten` method. (Contributed by R. David Murray in bpo-18600 [https://bugs.python.org/issue?@action=redirect&bpo=18600].)

New method **`as_bytes()`** added to produce a bytes representation of the message in a fashion similar to how `as_string` produces a string representation. It does not accept the *maxheaderlen* argument, but does accept the *unixfrom* and *policy* arguments. The **`Message.__bytes__()`** method calls it, meaning that `bytes(mymsg)` will now produce the intuitive result: a bytes object containing the fully formatted message. (Contributed by R. David Murray in bpo-18600 [https://bugs.python.org/issue?@action=redirect&bpo=18600].)

The **`Message.set_param()`** message now accepts a *replace* keyword argument. When specified, the associated header will be updated without changing its location in the list of headers. For backward compatibility, the default is `False`. (Contributed by R. David Murray in bpo-18891 [https://bugs.python.org/issue?@action=redirect&bpo=18891].)

A pair of new subclasses of **`Message`** have been added (**`EmailMessage`** and **`MIMEPart`**), along with a new sub-module, **`contentmanager`** and a new **`policy`** attribute **`content_manager`**. All documentation is currently in the new module, which is being added as part of email's new provisional API. These classes provide a number of new methods that make extracting content from and inserting content into email messages much easier. For details, see the **`contentmanager`** documentation and the email: Exemplos. These API additions complete the bulk of the work that was planned as part of the email6 project. The currently provisional API is scheduled to become final in Python 3.5 (possibly with a few minor additions in the area of error handling). (Contributed by R. David Murray in bpo-18891 [https://bugs.python.org/issue?@action=redirect&bpo=18891].)

filecmp

A new **`clear_cache()`** function provides the ability to clear the **`filecmp`** comparison cache, which uses **`os.stat()`** information to determine if the file has changed since the last compare. This can be used, for example, if the file might have been changed and re-checked in less time than the resolution of a particular filesystem's file modification time field. (Contributed by Mark Levitt in bpo-18149 [https://bugs.python.org/issue?@action=redirect&bpo=18149].)

New module attribute `DEFAULT_IGNORES` provides the list of directories that are used as the default value for the *ignore* parameter of the `dircmp()` function. (Contributed by Eli Bendersky in bpo-15442 [https://bugs.python.org/issue?@action=redirect&bpo=15442].)

functools

The new `partialmethod()` descriptor brings partial argument application to descriptors, just as `partial()` provides for normal callables. The new descriptor also makes it easier to get arbitrary callables (including `partial()` instances) to behave like normal instance methods when included in a class definition. (Contributed by Alon Horev and Nick Coghlan in bpo-4331 [https://bugs.python.org/issue?@action=redirect&bpo=4331].)

The new `@~functools.singledispatch` decorator brings support for single-dispatch generic functions to the Python standard library. Where object oriented programming focuses on grouping multiple operations on a common set of data into a class, a generic function focuses on grouping multiple implementations of an operation that allows it to work with *different* kinds of data.

Ver também

PEP 443 [https://peps.python.org/pep-0443/] – Single-dispatch generic functions

PEP escrita e implementada por Łukasz Langa.

`@~functools.total_ordering` now supports a return value of `NotImplemented` from the underlying comparison function. (Contributed by Katie Miller in bpo-10042 [https://bugs.python.org/issue?@action=redirect&bpo=10042].)

A pure-python version of the `partial()` function is now in the stdlib; in CPython it is overridden by the C accelerated version, but it is available for other implementations to use. (Contributed by Brian Thorne in bpo-12428 [https://bugs.python.org/issue?@action=redirect&bpo=12428].)

gc

New function **`get_stats()`** returns a list of three per-generation dictionaries containing the collections statistics since interpreter startup. (Contributed by Antoine Pitrou in bpo-16351 [https://bugs.python.org/issue?@action=redirect&bpo=16351].)

glob

A new function **`escape()`** provides a way to escape special characters in a filename so that they do not become part of the globbing expansion but are instead matched literally. (Contributed by Serhiy Storchaka in bpo-8402 [https://bugs.python.org/issue?@action=redirect&bpo=8402].)

hashlib

A new **`hashlib.pbkdf2_hmac()`** function provides the PKCS#5 password-based key derivation function 2 [https://en.wikipedia.org/wiki/PBKDF2]. (Contributed by Christian Heimes in bpo-18582 [https://bugs.python.org/issue?@action=redirect&bpo=18582].)

The **`name`** attribute of **`hashlib`** hash objects is now a formally supported interface. It has always existed in CPython's **`hashlib`** (although it did not return lower case names for all supported hashes), but it was not a public interface and so some other Python implementations have not previously supported it. (Contributed by Jason R. Coombs in bpo-18532 [https://bugs.python.org/issue?@action=redirect&bpo=18532].)

hmac

`hmac` now accepts `bytearray` as well as `bytes` for the *key* argument to the **`new()`** function, and the *msg* parameter to both the **`new()`** function and the **`update()`** method now accepts any type supported by the **`hashlib`** module. (Contributed by Jonas Borgström in bpo-18240 [https://bugs.python.org/issue?@action=redirect&bpo=18240].)

The *digestmod* argument to the `hmac.new()` function may now be any hash digest name recognized by `hashlib`. In addition, the current behavior in which the value of *digestmod* defaults to `MD5` is deprecated: in a future version of Python there will be no default value. (Contributed by Christian Heimes in bpo-17276 [https://bugs.python.org/issue?@action=redirect&bpo=17276].)

With the addition of `block_size` and `name` attributes (and the formal documentation of the `digest_size` attribute), the `hmac` module now conforms fully to the **PEP 247** [https://peps.python.org/pep-0247/] API. (Contributed by Christian Heimes in bpo-18775 [https://bugs.python.org/issue?@action=redirect&bpo=18775].)

html

New function `unescape()` function converts HTML5 character references to the corresponding Unicode characters. (Contributed by Ezio Melotti in bpo-2927 [https://bugs.python.org/issue?@action=redirect&bpo=2927].)

`HTMLParser` accepts a new keyword argument *convert_charrefs* that, when `True`, automatically converts all character references. For backward-compatibility, its value defaults to `False`, but it will change to `True` in a future version of Python, so you are invited to set it explicitly and update your code to use this new feature. (Contributed by Ezio Melotti in bpo-13633 [https://bugs.python.org/issue?@action=redirect&bpo=13633].)

The *strict* argument of `HTMLParser` is now deprecated. (Contributed by Ezio Melotti in bpo-15114 [https://bugs.python.org/issue?@action=redirect&bpo=15114].)

http

`send_error()` now accepts an optional additional *explain* parameter which can be used to provide an extended error description, overriding the hardcoded default if there is one. This extended error description will be formatted using the `error_message_format` attribute and sent as the body of the error response. (Contributed by Karl Cow in bpo-12921 [https://bugs.python.org/issue?@action=redirect&bpo=12921].)

The **`http.server`** command line interface now has a `-b/--bind` option that causes the server to listen on a specific address. (Contributed by Malte Swart in bpo-17764 [https://bugs.python.org/issue?@action=redirect&bpo=17764].)

idlelib e IDLE

Since idlelib implements the IDLE shell and editor and is not intended for import by other programs, it gets improvements with every release. See `Lib/idlelib/NEWS.txt` for a cumulative list of changes since 3.3.0, as well as changes made in future 3.4.x releases. This file is also available from the IDLE *Help* ▸ *About IDLE* dialog.

importlib

The **`InspectLoader`** ABC defines a new method, **`source_to_code()`** that accepts source data and a path and returns a code object. The default implementation is equivalent to `compile(data, path, 'exec', dont_inherit=True)`. (Contributed by Eric Snow and Brett Cannon in bpo-15627 [https://bugs.python.org/issue?@action=redirect&bpo=15627].)

`InspectLoader` also now has a default implementation for the **`get_code()`** method. However, it will normally be desirable to override the default implementation for performance reasons. (Contributed by Brett Cannon in bpo-18072 [https://bugs.python.org/issue?@action=redirect&bpo=18072].)

The **`reload()`** function has been moved from **`imp`** to **`importlib`** as part of the **`imp`** module deprecation. (Contributed by Berker Peksag in bpo-18193 [https://bugs.python.org/issue?@action=redirect&bpo=18193].)

`importlib.util` now has a **`MAGIC_NUMBER`** attribute providing access to the bytecode version number. This replaces the **`get_magic()`** function in the deprecated **`imp`** module. (Contributed by Brett Cannon in bpo-18192 [https://bugs.python.org/issue?@action=redirect&bpo=18192].)

New **`importlib.util`** functions **`cache_from_source()`** and **`source_from_cache()`** replace the same-named functions in the deprecated

imp module. (Contributed by Brett Cannon in bpo-18194 [https://bugs.python.org/issue?@action=redirect&bpo=18194].)

The **importlib** bootstrap **NamespaceLoader** now conforms to the **InspectLoader** ABC, which means that `runpy` and `python -m` can now be used with namespace packages. (Contributed by Brett Cannon in bpo-18058 [https://bugs.python.org/issue?@action=redirect&bpo=18058].)

importlib.util has a new function **decode_source()** that decodes source from bytes using universal newline processing. This is useful for implementing **InspectLoader.get_source()** methods.

importlib.machinery.ExtensionFileLoader now has a **get_filename()** method. This was inadvertently omitted in the original implementation. (Contributed by Eric Snow in bpo-19152 [https://bugs.python.org/issue?@action=redirect&bpo=19152].)

inspect

The **inspect** module now offers a basic command line interface to quickly display source code and other information for modules, classes and functions. (Contributed by Claudiu Popa and Nick Coghlan in bpo-18626 [https://bugs.python.org/issue?@action=redirect&bpo=18626].)

unwrap() makes it easy to unravel wrapper function chains created by **@functools.wraps** (and any other API that sets the `__wrapped__` attribute on a wrapper function). (Contributed by Daniel Urban, Aaron Iles and Nick Coghlan in bpo-13266 [https://bugs.python.org/issue?@action=redirect&bpo=13266].)

As part of the implementation of the new **enum** module, the **inspect** module now has substantially better support for custom `__dir__` methods and dynamic class attributes provided through metaclasses. (Contributed by Ethan Furman in bpo-18929 [https://bugs.python.org/issue?@action=redirect&bpo=18929] and bpo-19030 [https://bugs.python.org/issue?@action=redirect&bpo=19030].)

getfullargspec() and **getargspec()** now use the **signature()** API. This allows them to support a much broader range of callables, including those with `__signature__` attributes, those with metadata provided by argument

clinic, **functools.partial()** objects and more. Note that, unlike **signature()**, these functions still ignore __wrapped__ attributes, and report the already bound first argument for bound methods, so it is still necessary to update your code to use **signature()** directly if those features are desired. (Contributed by Yury Selivanov in bpo-17481 [https://bugs.python.org/issue?@action=redirect&bpo=17481].)

signature() now supports duck types of CPython functions, which adds support for functions compiled with Cython. (Contributed by Stefan Behnel and Yury Selivanov in bpo-17159 [https://bugs.python.org/issue?@action=redirect&bpo=17159].)

ipaddress

ipaddress was added to the standard library in Python 3.3 as a provisional API. With the release of Python 3.4, this qualification has been removed: **ipaddress** is now considered a stable API, covered by the normal standard library requirements to maintain backwards compatibility.

A new **is_global** property is True if an address is globally routeable. (Contributed by Peter Moody in bpo-17400 [https://bugs.python.org/issue?@action=redirect&bpo=17400].)

logging

The **TimedRotatingFileHandler** has a new *atTime* parameter that can be used to specify the time of day when rollover should happen. (Contributed by Ronald Oussoren in bpo-9556 [https://bugs.python.org/issue?@action=redirect&bpo=9556].)

SocketHandler and **DatagramHandler** now support Unix domain sockets (by setting *port* to None). (Contributed by Vinay Sajip in commit ce46195b56a9.)

fileConfig() now accepts a **configparser.RawConfigParser** subclass instance for the *fname* parameter. This facilitates using a configuration file when logging configuration is just a part of the overall application

configuration, or where the application modifies the configuration before passing it to **`fileConfig()`**. (Contributed by Vinay Sajip in bpo-16110 [https://bugs.python.org/issue?@action=redirect&bpo=16110].)

Logging configuration data received from a socket via the **`logging.config.listen()`** function can now be validated before being processed by supplying a verification function as the argument to the new *verify* keyword argument. (Contributed by Vinay Sajip in bpo-15452 [https://bugs.python.org/issue?@action=redirect&bpo=15452].)

marshal

The default **`marshal`** version has been bumped to 3. The code implementing the new version restores the Python2 behavior of recording only one copy of interned strings and preserving the interning on deserialization, and extends this "one copy" ability to any object type (including handling recursive references). This reduces both the size of `.pyc` files and the amount of memory a module occupies in memory when it is loaded from a `.pyc` (or `.pyo`) file. (Contributed by Kristján Valur Jónsson in bpo-16475 [https://bugs.python.org/issue?@action=redirect&bpo=16475], with additional speedups by Antoine Pitrou in bpo-19219 [https://bugs.python.org/issue?@action=redirect&bpo=19219].)

mmap

mmap objects are now weakly referenceable. (Contributed by Valerie Lambert in bpo-4885 [https://bugs.python.org/issue?@action=redirect&bpo=4885].)

multiprocessing

On Unix two new start methods, `spawn` and `forkserver`, have been added for starting processes using **`multiprocessing`**. These make the mixing of processes with threads more robust, and the `spawn` method matches the semantics that multiprocessing has always used on Windows. New function **`get_all_start_methods()`** reports all start methods available on the platform, **`get_start_method()`** reports the current start method, and

`set_start_method()` sets the start method. (Contributed by Richard Oudkerk in bpo-8713 [https://bugs.python.org/issue?@action=redirect&bpo=8713].)

`multiprocessing` also now has the concept of a `context`, which determines how child processes are created. New function `get_context()` returns a context that uses a specified start method. It has the same API as the `multiprocessing` module itself, so you can use it to create `Pool`s and other objects that will operate within that context. This allows a framework and an application or different parts of the same application to use multiprocessing without interfering with each other. (Contributed by Richard Oudkerk in bpo-18999 [https://bugs.python.org/issue?@action=redirect&bpo=18999].)

Except when using the old *fork* start method, child processes no longer inherit unneeded handles/file descriptors from their parents (part of bpo-8713 [https://bugs.python.org/issue?@action=redirect&bpo=8713]).

`multiprocessing` now relies on `runpy` (which implements the `-m` switch) to initialise `__main__` appropriately in child processes when using the `spawn` or `forkserver` start methods. This resolves some edge cases where combining multiprocessing, the `-m` command line switch, and explicit relative imports could cause obscure failures in child processes. (Contributed by Nick Coghlan in bpo-19946 [https://bugs.python.org/issue?@action=redirect&bpo=19946].)

operador

New function `length_hint()` provides an implementation of the specification for how the `__length_hint__()` special method should be used, as part of the **PEP 424** [https://peps.python.org/pep-0424/] formal specification of this language feature. (Contributed by Armin Ronacher in bpo-16148 [https://bugs.python.org/issue?@action=redirect&bpo=16148].)

There is now a pure-python version of the `operator` module available for reference and for use by alternate implementations of Python. (Contributed by Zachary Ware in bpo-16694 [https://bugs.python.org/issue?@action=redirect&bpo=16694].)

os

There are new functions to get and set the inheritable flag of a file descriptor (`os.get_inheritable()`, `os.set_inheritable()`) or a Windows handle (`os.get_handle_inheritable()`, `os.set_handle_inheritable()`).

New function `cpu_count()` reports the number of CPUs available on the platform on which Python is running (or `None` if the count can't be determined). The `multiprocessing.cpu_count()` function is now implemented in terms of this function). (Contributed by Trent Nelson, Yogesh Chaudhari, Victor Stinner, and Charles-François Natali in bpo-17914 [https://bugs.python.org/issue?@action=redirect&bpo=17914].)

`os.path.samestat()` is now available on the Windows platform (and the `os.path.samefile()` implementation is now shared between Unix and Windows). (Contributed by Brian Curtin in bpo-11939 [https://bugs.python.org/issue?@action=redirect&bpo=11939].)

`os.path.ismount()` now recognizes volumes mounted below a drive root on Windows. (Contributed by Tim Golden in bpo-9035 [https://bugs.python.org/issue?@action=redirect&bpo=9035].)

`os.open()` supports two new flags on platforms that provide them, `O_PATH` (un-opened file descriptor), and `O_TMPFILE` (unnamed temporary file; as of 3.4.0 release available only on Linux systems with a kernel version of 3.11 or newer that have uapi headers). (Contributed by Christian Heimes in bpo-18673 [https://bugs.python.org/issue?@action=redirect&bpo=18673] and Benjamin Peterson, respectively.)

pdb

`pdb` has been enhanced to handle generators, `yield`, and `yield from` in a more useful fashion. This is especially helpful when debugging `asyncio` based programs. (Contributed by Andrew Svetlov and Xavier de Gaye in bpo-16596 [https://bugs.python.org/issue?@action=redirect&bpo=16596].)

The `print` command has been removed from `pdb`, restoring access to the Python `print()` function from the pdb command line. Python2's `pdb` did not have a `print` command; instead, entering `print` executed the `print` statement. In Python3 `print` was mistakenly made an alias for the pdb `p` command. `p`, however, prints the `repr` of its argument, not the `str` like the Python2 `print` command did. Worse, the Python3 `pdb print` command shadowed the Python3 `print` function, making it inaccessible at the `pdb` prompt. (Contributed by Connor Osborn in bpo-18764 [https://bugs.python.org/issue?@action=redirect&bpo=18764].)

pickle

`pickle` now supports (but does not use by default) a new pickle protocol, protocol 4. This new protocol addresses a number of issues that were present in previous protocols, such as the serialization of nested classes, very large strings and containers, and classes whose `__new__()` method takes keyword-only arguments. It also provides some efficiency improvements.

Ver também

PEP 3154 [https://peps.python.org/pep-3154/] – Pickle protocol 4
PEP written by Antoine Pitrou and implemented by Alexandre Vassalotti.

plistlib

`plistlib` now has an API that is similar to the standard pattern for stdlib serialization protocols, with new `load()`, `dump()`, `loads()`, and `dumps()` functions. (The older API is now deprecated.) In addition to the already supported XML plist format (`FMT_XML`), it also now supports the binary plist format (`FMT_BINARY`). (Contributed by Ronald Oussoren and others in bpo-14455 [https://bugs.python.org/issue?@action=redirect&bpo=14455].)

poplib

Two new methods have been added to `poplib`: `capa()`, which returns the list of capabilities advertised by the POP server, and `stls()`, which switches a clear-text POP3 session into an encrypted POP3 session if the POP server supports it. (Contributed by Lorenzo Catucci in bpo-4473 [https://bugs.python.org/issue?@action=redirect&bpo=4473].)

pprint

The `pprint` module's `PrettyPrinter` class and its `pformat()`, and `pprint()` functions have a new option, *compact*, that controls how the output is formatted. Currently setting *compact* to `True` means that sequences will be printed with as many sequence elements as will fit within *width* on each (indented) line. (Contributed by Serhiy Storchaka in bpo-19132 [https://bugs.python.org/issue?@action=redirect&bpo=19132].)

Long strings are now wrapped using Python's normal line continuation syntax. (Contributed by Antoine Pitrou in bpo-17150 [https://bugs.python.org/issue?@action=redirect&bpo=17150].)

pty

`pty.spawn()` now returns the status value from `os.waitpid()` on the child process, instead of `None`. (Contributed by Gregory P. Smith.)

pydoc

The `pydoc` module is now based directly on the `inspect.signature()` introspection API, allowing it to provide signature information for a wider variety of callable objects. This change also means that `__wrapped__` attributes are now taken into account when displaying help information. (Contributed by Larry Hastings in bpo-19674 [https://bugs.python.org/issue?@action=redirect&bpo=19674].)

The `pydoc` module no longer displays the `self` parameter for already bound methods. Instead, it aims to always display the exact current signature of the supplied callable. (Contributed by Larry Hastings in bpo-20710 [https://bugs.python.org/issue?@action=redirect&bpo=20710].)

In addition to the changes that have been made to **pydoc** directly, its handling of custom `__dir__` methods and various descriptor behaviours has also been improved substantially by the underlying changes in the **inspect** module.

As the **help()** builtin is based on **pydoc**, the above changes also affect the behaviour of **help()**.

re

New **fullmatch()** function and **Pattern.fullmatch()** method anchor the pattern at both ends of the string to match. This provides a way to be explicit about the goal of the match, which avoids a class of subtle bugs where `$` characters get lost during code changes or the addition of alternatives to an existing regular expression. (Contributed by Matthew Barnett in bpo-16203 [https://bugs.python.org/issue?@action=redirect&bpo=16203].)

The repr of regex objects now includes the pattern and the flags; the repr of match objects now includes the start, end, and the part of the string that matched. (Contributed by Hugo Lopes Tavares and Serhiy Storchaka in bpo-13592 [https://bugs.python.org/issue?@action=redirect&bpo=13592] and bpo-17087 [https://bugs.python.org/issue?@action=redirect&bpo=17087].)

resource

New **prlimit()** function, available on Linux platforms with a kernel version of 2.6.36 or later and glibc of 2.13 or later, provides the ability to query or set the resource limits for processes other than the one making the call. (Contributed by Christian Heimes in bpo-16595 [https://bugs.python.org/issue?@action=redirect&bpo=16595].)

On Linux kernel version 2.6.36 or later, there are also some new Linux specific constants: **RLIMIT_MSGQUEUE**, **RLIMIT_NICE**, **RLIMIT_RTPRIO**, **RLIMIT_RTTIME**, and **RLIMIT_SIGPENDING**. (Contributed by Christian Heimes in bpo-19324 [https://bugs.python.org/issue?@action=redirect&bpo=19324].)

On FreeBSD version 9 and later, there some new FreeBSD specific constants: `RLIMIT_SBSIZE`, `RLIMIT_SWAP`, and `RLIMIT_NPTS`. (Contributed by Claudiu Popa in bpo-19343 [https://bugs.python.org/issue?@action=redirect&bpo=19343].)

select

`epoll` objects now support the context management protocol. When used in a `with` statement, the `close()` method will be called automatically at the end of the block. (Contributed by Serhiy Storchaka in bpo-16488 [https://bugs.python.org/issue?@action=redirect&bpo=16488].)

`devpoll` objects now have `fileno()` and `close()` methods, as well as a new attribute `closed`. (Contributed by Victor Stinner in bpo-18794 [https://bugs.python.org/issue?@action=redirect&bpo=18794].)

shelve

`Shelf` instances may now be used in `with` statements, and will be automatically closed at the end of the `with` block. (Contributed by Filip Gruszczyński in bpo-13896 [https://bugs.python.org/issue?@action=redirect&bpo=13896].)

shutil

`copyfile()` now raises a specific `Error` subclass, `SameFileError`, when the source and destination are the same file, which allows an application to take appropriate action on this specific error. (Contributed by Atsuo Ishimoto and Hynek Schlawack in bpo-1492704 [https://bugs.python.org/issue?@action=redirect&bpo=1492704].)

smtpd

The `SMTPServer` and `SMTPChannel` classes now accept a *map* keyword argument which, if specified, is passed in to `asynchat.async_chat` as its *map* argument. This allows an application to avoid affecting the global

socket map. (Contributed by Vinay Sajip in bpo-11959 [https://bugs.python.org/issue?@action=redirect&bpo=11959].)

smtplib

`SMTPException` is now a subclass of `OSError`, which allows both socket level errors and SMTP protocol level errors to be caught in one try/except statement by code that only cares whether or not an error occurred. (Contributed by Ned Jackson Lovely in bpo-2118 [https://bugs.python.org/issue?@action=redirect&bpo=2118].)

socket

The socket module now supports the `CAN_BCM` protocol on platforms that support it. (Contributed by Brian Thorne in bpo-15359 [https://bugs.python.org/issue?@action=redirect&bpo=15359].)

Socket objects have new methods to get or set their inheritable flag, `get_inheritable()` and `set_inheritable()`.

The `socket.AF_*` and `socket.SOCK_*` constants are now enumeration values using the new `enum` module. This allows meaningful names to be printed during debugging, instead of integer “magic numbers”.

The `AF_LINK` constant is now available on BSD and OSX.

`inet_pton()` and `inet_ntop()` are now supported on Windows. (Contributed by Atsuo Ishimoto in bpo-7171 [https://bugs.python.org/issue?@action=redirect&bpo=7171].)

sqlite3

A new boolean parameter to the `connect()` function, *uri*, can be used to indicate that the *database* parameter is a `uri` (see the SQLite URI documentation [https://www.sqlite.org/uri.html]). (Contributed by poq in bpo-13773 [https://bugs.python.org/issue?@action=redirect&bpo=13773].)

ssl

PROTOCOL_TLSv1_1 and **PROTOCOL_TLSv1_2** (TLSv1.1 and TLSv1.2 support) have been added; support for these protocols is only available if Python is linked with OpenSSL 1.0.1 or later. (Contributed by Michele Orrù and Antoine Pitrou in bpo-16692 [https://bugs.python.org/issue?@action=redirect&bpo=16692].)

New function **create_default_context()** provides a standard way to obtain an **SSLContext** whose settings are intended to be a reasonable balance between compatibility and security. These settings are more stringent than the defaults provided by the **SSLContext** constructor, and may be adjusted in the future, without prior deprecation, if best-practice security requirements change. The new recommended best practice for using stdlib libraries that support SSL is to use **create_default_context()** to obtain an **SSLContext** object, modify it if needed, and then pass it as the *context* argument of the appropriate stdlib API. (Contributed by Christian Heimes in bpo-19689 [https://bugs.python.org/issue?@action=redirect&bpo=19689].)

SSLContext method **load_verify_locations()** accepts a new optional argument *cadata*, which can be used to provide PEM or DER encoded certificates directly via strings or bytes, respectively. (Contributed by Christian Heimes in bpo-18138 [https://bugs.python.org/issue?@action=redirect&bpo=18138].)

New function **get_default_verify_paths()** returns a named tuple of the paths and environment variables that the **set_default_verify_paths()** method uses to set OpenSSL's default `cafile` and `capath`. This can be an aid in debugging default verification issues. (Contributed by Christian Heimes in bpo-18143 [https://bugs.python.org/issue?@action=redirect&bpo=18143].)

SSLContext has a new method, **cert_store_stats()**, that reports the number of loaded `X.509` certs, `X.509 CA` certs, and certificate revocation lists (`crls`), as well as a **get_ca_certs()** method that returns a list of the loaded `CA` certificates. (Contributed by Christian Heimes in bpo-18147 [https://bugs.python.org/issue?@action=redirect&bpo=18147].)

If OpenSSL 0.9.8 or later is available, `SSLContext` has a new attribute `verify_flags` that can be used to control the certificate verification process by setting it to some combination of the new constants `VERIFY_DEFAULT`, `VERIFY_CRL_CHECK_LEAF`, `VERIFY_CRL_CHECK_CHAIN`, or `VERIFY_X509_STRICT`. OpenSSL does not do any CRL verification by default. (Contributed by Christien Heimes in bpo-8813 [https://bugs.python.org/issue?@action=redirect&bpo=8813].)

New `SSLContext` method `load_default_certs()` loads a set of default "certificate authority" (CA) certificates from default locations, which vary according to the platform. It can be used to load both TLS web server authentication certificates (`purpose=SERVER_AUTH`) for a client to use to verify a server, and certificates for a server to use in verifying client certificates (`purpose=CLIENT_AUTH`). (Contributed by Christian Heimes in bpo-19292 [https://bugs.python.org/issue?@action=redirect&bpo=19292].)

Two new windows-only functions, `enum_certificates()` and `enum_crls()` provide the ability to retrieve certificates, certificate information, and CRLs from the Windows cert store. (Contributed by Christian Heimes in bpo-17134 [https://bugs.python.org/issue?@action=redirect&bpo=17134].)

Support for server-side SNI (Server Name Indication) using the new `ssl.SSLContext.set_servername_callback()` method. (Contributed by Daniel Black in bpo-8109 [https://bugs.python.org/issue?@action=redirect&bpo=8109].)

The dictionary returned by `SSLSocket.getpeercert()` contains additional `X509v3` extension items: `crlDistributionPoints`, `calIssuers`, and `OCSP` URIs. (Contributed by Christian Heimes in bpo-18379 [https://bugs.python.org/issue?@action=redirect&bpo=18379].)

stat

The `stat` module is now backed by a C implementation in `_stat`. A C implementation is required as most of the values aren't standardized and are platform-dependent. (Contributed by Christian Heimes in bpo-11016 [https://bugs.python.org/issue?@action=redirect&bpo=11016].)

The module supports new **`ST_MODE`** flags, **`S_IFDOOR`**, **`S_IFPORT`**, and **`S_IFWHT`**. (Contributed by Christian Hiemes in bpo-11016 [https://bugs.python.org/issue?@action=redirect&bpo=11016].)

struct

New function **`iter_unpack`** and a new **`struct.Struct.iter_unpack()`** method on compiled formats provide streamed unpacking of a buffer containing repeated instances of a given format of data. (Contributed by Antoine Pitrou in bpo-17804 [https://bugs.python.org/issue?@action=redirect&bpo=17804].)

subprocess

`check_output()` now accepts an *input* argument that can be used to provide the contents of `stdin` for the command that is run. (Contributed by Zack Weinberg in bpo-16624 [https://bugs.python.org/issue?@action=redirect&bpo=16624].)

`getoutput()` and **`getstatusoutput()`** now work on Windows. This change was actually inadvertently made in 3.3.4. (Contributed by Tim Golden in bpo-10197 [https://bugs.python.org/issue?@action=redirect&bpo=10197].)

sunau

The **`getparams()`** method now returns a namedtuple rather than a plain tuple. (Contributed by Claudiu Popa in bpo-18901 [https://bugs.python.org/issue?@action=redirect&bpo=18901].)

`sunau.open()` now supports the context management protocol: when used in a **`with`** block, the `close` method of the returned object will be called automatically at the end of the block. (Contributed by Serhiy Storchaka in bpo-18878 [https://bugs.python.org/issue?@action=redirect&bpo=18878].)

`AU_write.setsampwidth()` now supports 24 bit samples, thus adding support for writing 24 sample using the module. (Contributed by Serhiy Storchaka in bpo-19261 [https://bugs.python.org/issue?@action=redirect&bpo=19261].)

The **writeframesraw()** and **writeframes()** methods now accept any bytes-like object. (Contributed by Serhiy Storchaka in bpo-8311 [https://bugs.python.org/issue?@action=redirect&bpo=8311].)

sys

New function **sys.getallocatedblocks()** returns the current number of blocks allocated by the interpreter. (In CPython with the default `--with-pymalloc` setting, this is allocations made through the **PyObject_Malloc()** API.) This can be useful for tracking memory leaks, especially if automated via a test suite. (Contributed by Antoine Pitrou in bpo-13390 [https://bugs.python.org/issue?@action=redirect&bpo=13390].)

When the Python interpreter starts in interactive mode, it checks for an **__interactivehook__** attribute on the **sys** module. If the attribute exists, its value is called with no arguments just before interactive mode is started. The check is made after the **PYTHONSTARTUP** file is read, so it can be set there. The **site** module sets it to a function that enables tab completion and history saving (in `~/.python-history`) if the platform supports **readline**. If you do not want this (new) behavior, you can override it in **PYTHONSTARTUP**, **sitecustomize**, or **usercustomize** by deleting this attribute from **sys** (or setting it to some other callable). (Contributed by Éric Araujo and Antoine Pitrou in bpo-5845 [https://bugs.python.org/issue?@action=redirect&bpo=5845].)

tarfile

The **tarfile** module now supports a simple Interface de Linha de Comando when called as a script directly or via `-m`. This can be used to create and extract tarfile archives. (Contributed by Berker Peksag in bpo-13477 [https://bugs.python.org/issue?@action=redirect&bpo=13477].)

textwrap

The **TextWrapper** class has two new attributes/constructor arguments: **max_lines**, which limits the number of lines in the output, and **placeholder**, which is a string that will appear at the end of the output if it

has been truncated because of *max_lines*. Building on these capabilities, a new convenience function `shorten()` collapses all of the whitespace in the input to single spaces and produces a single line of a given *width* that ends with the *placeholder* (by default, `[...]`). (Contributed by Antoine Pitrou and Serhiy Storchaka in bpo-18585 [https://bugs.python.org/issue?@action=redirect&bpo=18585] and bpo-18725 [https://bugs.python.org/issue?@action=redirect&bpo=18725].)

threading

The `Thread` object representing the main thread can be obtained from the new `main_thread()` function. In normal conditions this will be the thread from which the Python interpreter was started. (Contributed by Andrew Svetlov in bpo-18882 [https://bugs.python.org/issue?@action=redirect&bpo=18882].)

traceback

A new `traceback.clear_frames()` function takes a traceback object and clears the local variables in all of the frames it references, reducing the amount of memory consumed. (Contributed by Andrew Kuchling in bpo-1565525 [https://bugs.python.org/issue?@action=redirect&bpo=1565525].)

types

A new `DynamicClassAttribute()` descriptor provides a way to define an attribute that acts normally when looked up through an instance object, but which is routed to the *class* `__getattr__` when looked up through the class. This allows one to have properties active on a class, and have virtual attributes on the class with the same name (see `enum` for an example). (Contributed by Ethan Furman in bpo-19030 [https://bugs.python.org/issue?@action=redirect&bpo=19030].)

urllib

`urllib.request` now supports `data:` URLs via the `DataHandler` class. (Contributed by Mathias Panzenböck in bpo-16423 [https://bugs.python.org/issue?@action=redirect&bpo=16423].)

The http method that will be used by a **`Request`** class can now be specified by setting a **`method`** class attribute on the subclass. (Contributed by Jason R Coombs in bpo-18978 [https://bugs.python.org/issue?@action=redirect&bpo=18978].)

`Request` objects are now reusable: if the **`full_url`** or **`data`** attributes are modified, all relevant internal properties are updated. This means, for example, that it is now possible to use the same **`Request`** object in more than one **`OpenerDirector.open()`** call with different *data* arguments, or to modify a **`Request`**'s `url` rather than recomputing it from scratch. There is also a new **`remove_header()`** method that can be used to remove headers from a **`Request`**. (Contributed by Alexey Kachayev in bpo-16464 [https://bugs.python.org/issue?@action=redirect&bpo=16464], Daniel Wozniak in bpo-17485 [https://bugs.python.org/issue?@action=redirect&bpo=17485], and Damien Brecht and Senthil Kumaran in bpo-17272 [https://bugs.python.org/issue?@action=redirect&bpo=17272].)

`HTTPError` objects now have a **`headers`** attribute that provides access to the HTTP response headers associated with the error. (Contributed by Berker Peksag in bpo-15701 [https://bugs.python.org/issue?@action=redirect&bpo=15701].)

unittest

The **`TestCase`** class has a new method, **`subTest()`**, that produces a context manager whose **`with`** block becomes a "sub-test". This context manager allows a test method to dynamically generate subtests by, say, calling the `subTest` context manager inside a loop. A single test method can thereby produce an indefinite number of separately identified and separately counted tests, all of which will run even if one or more of them fail. For example:

```
class NumbersTest(unittest.TestCase):
    def test_even(self):
        for i in range(6):
            with self.subTest(i=i):
                self.assertEqual(i % 2, 0)
```

will result in six subtests, each identified in the unittest verbose output with a label consisting of the variable name `i` and a particular value for that

variable (`i=0`, `i=1`, etc). See Distinguindo iterações de teste utilizando subtestes for the full version of this example. (Contributed by Antoine Pitrou in bpo-16997 [https://bugs.python.org/issue?@action=redirect&bpo=16997].)

`unittest.main()` now accepts an iterable of test names for *defaultTest*, where previously it only accepted a single test name as a string. (Contributed by Jyrki Pulliainen in bpo-15132 [https://bugs.python.org/issue?@action=redirect&bpo=15132].)

If **`SkipTest`** is raised during test discovery (that is, at the module level in the test file), it is now reported as a skip instead of an error. (Contributed by Zach Ware in bpo-16935 [https://bugs.python.org/issue?@action=redirect&bpo=16935].)

`discover()` now sorts the discovered files to provide consistent test ordering. (Contributed by Martin Melin and Jeff Ramnani in bpo-16709 [https://bugs.python.org/issue?@action=redirect&bpo=16709].)

`TestSuite` now drops references to tests as soon as the test has been run, if the test is successful. On Python interpreters that do garbage collection, this allows the tests to be garbage collected if nothing else is holding a reference to the test. It is possible to override this behavior by creating a **`TestSuite`** subclass that defines a custom `_removeTestAtIndex` method. (Contributed by Tom Wardill, Matt McClure, and Andrew Svetlov in bpo-11798 [https://bugs.python.org/issue?@action=redirect&bpo=11798].)

A new test assertion context-manager, **`assertLogs()`**, will ensure that a given block of code emits a log message using the **`logging`** module. By default the message can come from any logger and have a priority of `INFO` or higher, but both the logger name and an alternative minimum logging level may be specified. The object returned by the context manager can be queried for the **`LogRecord`**s and/or formatted messages that were logged. (Contributed by Antoine Pitrou in bpo-18937 [https://bugs.python.org/issue?@action=redirect&bpo=18937].)

Test discovery now works with namespace packages (Contributed by Claudiu Popa in bpo-17457 [https://bugs.python.org/issue?@action=redirect&bpo=17457].)

unittest.mock objects now inspect their specification signatures when matching calls, which means an argument can now be matched by either position or name, instead of only by position. (Contributed by Antoine Pitrou in bpo-17015 [https://bugs.python.org/issue?@action=redirect&bpo=17015].)

mock open() objects now have `readline` and `readlines` methods. (Contributed by Toshio Kuratomi in bpo-17467 [https://bugs.python.org/issue?@action=redirect&bpo=17467].)

venv

venv now includes activation scripts for the `csh` and `fish` shells. (Contributed by Andrew Svetlov in bpo-15417 [https://bugs.python.org/issue?@action=redirect&bpo=15417].)

EnvBuilder and the **create()** convenience function take a new keyword argument *with_pip*, which defaults to `False`, that controls whether or not **EnvBuilder** ensures that `pip` is installed in the virtual environment. (Contributed by Nick Coghlan in bpo-19552 [https://bugs.python.org/issue?@action=redirect&bpo=19552] as part of the **PEP 453** [https://peps.python.org/pep-0453/] implementation.)

wave

The **getparams()** method now returns a namedtuple rather than a plain tuple. (Contributed by Claudiu Popa in bpo-17487 [https://bugs.python.org/issue?@action=redirect&bpo=17487].)

wave.open() now supports the context management protocol. (Contributed by Claudiu Popa in bpo-17616 [https://bugs.python.org/issue?@action=redirect&bpo=17616].)

wave can now write output to unseekable files. (Contributed by David Jones, Guilherme Polo, and Serhiy Storchaka in bpo-5202 [https://bugs.python.org/issue?@action=redirect&bpo=5202].)

The `writeframesraw()` and `writeframes()` methods now accept any bytes-like object. (Contributed by Serhiy Storchaka in bpo-8311 [https://bugs.python.org/issue?@action=redirect&bpo=8311].)

weakref

New `WeakMethod` class simulates weak references to bound methods. (Contributed by Antoine Pitrou in bpo-14631 [https://bugs.python.org/issue?@action=redirect&bpo=14631].)

New `finalize` class makes it possible to register a callback to be invoked when an object is garbage collected, without needing to carefully manage the lifecycle of the weak reference itself. (Contributed by Richard Oudkerk in bpo-15528 [https://bugs.python.org/issue?@action=redirect&bpo=15528].)

The callback, if any, associated with a `ref` is now exposed via the `__callback__` attribute. (Contributed by Mark Dickinson in bpo-17643 [https://bugs.python.org/issue?@action=redirect&bpo=17643].)

xml.etree

A new parser, `XMLPullParser`, allows a non-blocking applications to parse XML documents. An example can be seen at A API de pull para análise sem bloqueio. (Contributed by Antoine Pitrou in bpo-17741 [https://bugs.python.org/issue?@action=redirect&bpo=17741].)

The `xml.etree.ElementTree` `tostring()` and `tostringlist()` functions, and the `ElementTree` `write()` method, now have a *short_empty_elements* keyword-only parameter providing control over whether elements with no content are written in abbreviated (`<tag />`) or expanded (`<tag></tag>`) form. (Contributed by Ariel Poliak and Serhiy Storchaka in bpo-14377 [https://bugs.python.org/issue?@action=redirect&bpo=14377].)

zipfile

The `writepy()` method of the `PyZipFile` class has a new *filterfunc* option that can be used to control which directories and files are added to the

archive. For example, this could be used to exclude test files from the archive. (Contributed by Christian Tismer in bpo-19274 [https://bugs.python.org/issue?@action=redirect&bpo=19274].)

The *allowZip64* parameter to `ZipFile` and `PyZipFile` is now `True` by default. (Contributed by William Mallard in bpo-17201 [https://bugs.python.org/issue?@action=redirect&bpo=17201].)

CPython Implementation Changes

PEP 445: Customization of CPython Memory Allocators

PEP 445 [https://peps.python.org/pep-0445/] adds new C level interfaces to customize memory allocation in the CPython interpreter.

Ver também

PEP 445 [https://peps.python.org/pep-0445/] – Add new APIs to customize Python memory allocators
PEP escrito e implementado por Victor Stinner.

PEP 442: Safe Object Finalization

PEP 442 [https://peps.python.org/pep-0442/] removes the current limitations and quirks of object finalization in CPython. With it, objects with `__del__()` methods, as well as generators with `finally` clauses, can be finalized when they are part of a reference cycle.

As part of this change, module globals are no longer forcibly set to `None` during interpreter shutdown in most cases, instead relying on the normal operation of the cyclic garbage collector. This avoids a whole class of interpreter-shutdown-time errors, usually involving `__del__` methods, that have plagued Python since the cyclic GC was first introduced.

Ver também

PEP 442 [https://peps.python.org/pep-0442/] – Safe object finalization

PEP escrita e implementada por Antoine Pitrou.

PEP 456: Secure and Interchangeable Hash Algorithm

PEP 456 [https://peps.python.org/pep-0456/] follows up on earlier security fix work done on Python's hash algorithm to address certain DOS attacks to which public facing APIs backed by dictionary lookups may be subject. (See bpo-14621 [https://bugs.python.org/issue?@action=redirect&bpo=14621] for the start of the current round of improvements.) The PEP unifies CPython's hash code to make it easier for a packager to substitute a different hash algorithm, and switches Python's default implementation to a SipHash implementation on platforms that have a 64 bit data type. Any performance differences in comparison with the older FNV algorithm are trivial.

The PEP adds additional fields to the `sys.hash_info` named tuple to describe the hash algorithm in use by the currently executing binary. Otherwise, the PEP does not alter any existing CPython APIs.

PEP 436: Argument Clinic

"Argument Clinic" (**PEP 436** [https://peps.python.org/pep-0436/]) is now part of the CPython build process and can be used to simplify the process of defining and maintaining accurate signatures for builtins and standard library extension modules implemented in C.

Some standard library extension modules have been converted to use Argument Clinic in Python 3.4, and `pydoc` and `inspect` have been updated accordingly.

It is expected that signature metadata for programmatic introspection will be added to additional callables implemented in C as part of Python 3.4 maintenance releases.

Nota

The Argument Clinic PEP is not fully up to date with the state of the implementation. This has been deemed acceptable by the release manager and core development team in this case, as Argument Clinic will not be made available as a public API for third party use in Python 3.4.

Ver também

PEP 436 [https://peps.python.org/pep-0436/] – The Argument Clinic DSL
PEP written and implemented by Larry Hastings.

Other Alterações a compilações e API C

- The new `PyType_GetSlot()` function has been added to the stable ABI, allowing retrieval of function pointers from named type slots when using the limited API. (Contributed by Martin von Löwis in bpo-17162 [https://bugs.python.org/issue?@action=redirect&bpo=17162].)
- The new `Py_SetStandardStreamEncoding()` pre-initialization API allows applications embedding the CPython interpreter to reliably force a particular encoding and error handler for the standard streams. (Contributed by Bastien Montagne and Nick Coghlan in bpo-16129 [https://bugs.python.org/issue?@action=redirect&bpo=16129].)
- Most Python C APIs that don't mutate string arguments are now correctly marked as accepting `const char *` rather than `char *`. (Contributed by Serhiy Storchaka in bpo-1772673 [https://bugs.python.org/issue?@action=redirect&bpo=1772673].)
- A new shell version of `python-config` can be used even when a python interpreter is not available (for example, in cross compilation scenarios).
- `PyUnicode_FromFormat()` now supports width and precision specifications for `%s`, `%A`, `%U`, `%V`, `%S`, and `%R`. (Contributed by Ysj Ray and Victor Stinner in bpo-7330 [https://bugs.python.org/issue?@action=redirect&bpo=7330].)
- New function `PyStructSequence_InitType2()` supplements the existing `PyStructSequence_InitType()` function. The difference is that it returns `0` on success and `-1` on failure.

- The CPython source can now be compiled using the address sanity checking features of recent versions of GCC and clang: the false alarms in the small object allocator have been silenced. (Contributed by Dhiru Kholia in bpo-18596 [https://bugs.python.org/issue?@action=redirect&bpo=18596].)
- The Windows build now uses Address Space Layout Randomization [https://en.wikipedia.org/wiki/Address_space_layout_randomization] and Data Execution Prevention [https://en.wikipedia.org/wiki/Data_Execution_Prevention]. (Contributed by Christian Heimes in bpo-16632 [https://bugs.python.org/issue?@action=redirect&bpo=16632].)
- New function **`PyObject_LengthHint()`** is the C API equivalent of **`operator.length_hint()`**. (Contributed by Armin Ronacher in bpo-16148 [https://bugs.python.org/issue?@action=redirect&bpo=16148].)

Other Improvements

- The python command has a new option, `-I`, which causes it to run in "isolated mode", which means that **`sys.path`** contains neither the script's directory nor the user's `site-packages` directory, and all **`PYTHON*`** environment variables are ignored (it implies both `-s` and `-E`). Other restrictions may also be applied in the future, with the goal being to isolate the execution of a script from the user's environment. This is appropriate, for example, when Python is used to run a system script. On most POSIX systems it can and should be used in the `#!` line of system scripts. (Contributed by Christian Heimes in bpo-16499 [https://bugs.python.org/issue?@action=redirect&bpo=16499].)
- Tab-completion is now enabled by default in the interactive interpreter on systems that support **`readline`**. History is also enabled by default, and is written to (and read from) the file `~/.python-history`. (Contributed by Antoine Pitrou and Éric Araujo in bpo-5845 [https://bugs.python.org/issue?@action=redirect&bpo=5845].)
- Invoking the Python interpreter with `--version` now outputs the version to standard output instead of standard error (bpo-18338 [https://bugs.python.org/issue?@action=redirect&bpo=18338]). Similar changes were made to **`argparse`** (bpo-18920 [https://bugs.python.org/issue?@action=redirect&bpo=18920]) and other modules that have script-like

invocation capabilities (bpo-18922 [https://bugs.python.org/issue?@action=redirect&bpo=18922]).

- The CPython Windows installer now adds `.py` to the **`PATHEXT`** variable when extensions are registered, allowing users to run a python script at the windows command prompt by just typing its name without the `.py` extension. (Contributed by Paul Moore in bpo-18569 [https://bugs.python.org/issue?@action=redirect&bpo=18569].)
- A new `make` target coverage-report [https://devguide.python.org/coverage/#measuring-coverage-of-c-code-with-gcov-and-lcov] will build python, run the test suite, and generate an HTML coverage report for the C codebase using `gcov` and lcov [https://github.com/linux-test-project/lcov].
- The `-R` option to the python regression test suite now also checks for memory allocation leaks, using **`sys.getallocatedblocks()`**. (Contributed by Antoine Pitrou in bpo-13390 [https://bugs.python.org/issue?@action=redirect&bpo=13390].)
- `python -m` now works with namespace packages.
- The **`stat`** module is now implemented in C, which means it gets the values for its constants from the C header files, instead of having the values hard-coded in the python module as was previously the case.
- Loading multiple python modules from a single OS module (`.so`, `.dll`) now works correctly (previously it silently returned the first python module in the file). (Contributed by Václav Šmilauer in bpo-16421 [https://bugs.python.org/issue?@action=redirect&bpo=16421].)
- A new opcode, **`LOAD_CLASSDEREF`**, has been added to fix a bug in the loading of free variables in class bodies that could be triggered by certain uses of `__prepare__`. (Contributed by Benjamin Peterson in bpo-17853 [https://bugs.python.org/issue?@action=redirect&bpo=17853].)
- A number of MemoryError-related crashes were identified and fixed by Victor Stinner using his **PEP 445** [https://peps.python.org/pep-0445/]-based `pyfailmalloc` tool (bpo-18408 [https://bugs.python.org/issue?@action=redirect&bpo=18408], bpo-18520 [https://bugs.python.org/issue?@action=redirect&bpo=18520]).
- The `pyvenv` command now accepts a `--copies` option to use copies rather than symlinks even on systems where symlinks are the default. (Contributed by Vinay Sajip in bpo-18807 [https://bugs.python.org/issue?@action=redirect&bpo=18807].)

- The `pyvenv` command also accepts a `--without-pip` option to suppress the otherwise-automatic bootstrapping of pip into the virtual environment. (Contributed by Nick Coghlan in bpo-19552 [https://bugs.python.org/issue?@action=redirect&bpo=19552] as part of the **PEP 453** [https://peps.python.org/pep-0453/] implementation.)
- The encoding name is now optional in the value set for the **`PYTHONIOENCODING`** environment variable. This makes it possible to set just the error handler, without changing the default encoding. (Contributed by Serhiy Storchaka in bpo-18818 [https://bugs.python.org/issue?@action=redirect&bpo=18818].)
- The **`bz2`**, **`lzma`**, and **`gzip`** module `open` functions now support `x` (exclusive creation) mode. (Contributed by Tim Heaney and Vajrasky Kok in bpo-19201 [https://bugs.python.org/issue?@action=redirect&bpo=19201], bpo-19222 [https://bugs.python.org/issue?@action=redirect&bpo=19222], and bpo-19223 [https://bugs.python.org/issue?@action=redirect&bpo=19223].)

Otimizações Relevantes

- The UTF-32 decoder is now 3x to 4x faster. (Contributed by Serhiy Storchaka in bpo-14625 [https://bugs.python.org/issue?@action=redirect&bpo=14625].)
- The cost of hash collisions for sets is now reduced. Each hash table probe now checks a series of consecutive, adjacent key/hash pairs before continuing to make random probes through the hash table. This exploits cache locality to make collision resolution less expensive. The collision resolution scheme can be described as a hybrid of linear probing and open addressing. The number of additional linear probes defaults to nine. This can be changed at compile-time by defining LINEAR_PROBES to be any value. Set LINEAR_PROBES=0 to turn-off linear probing entirely. (Contributed by Raymond Hettinger in bpo-18771 [https://bugs.python.org/issue?@action=redirect&bpo=18771].)
- The interpreter starts about 30% faster. A couple of measures lead to the speedup. The interpreter loads fewer modules on startup, e.g. the **`re`**, **`collections`** and **`locale`** modules and their dependencies are no longer imported by default. The marshal module has been improved to load compiled Python code faster. (Contributed by Antoine Pitrou, Christian Heimes and Victor Stinner in bpo-19219

[https://bugs.python.org/issue?@action=redirect&bpo=19219], bpo-19218 [https://bugs.python.org/issue?@action=redirect&bpo=19218], bpo-19209 [https://bugs.python.org/issue?@action=redirect&bpo=19209], bpo-19205 [https://bugs.python.org/issue?@action=redirect&bpo=19205] and bpo-9548 [https://bugs.python.org/issue?@action=redirect&bpo=9548].)

- **`bz2.BZ2File`** is now as fast or faster than the Python2 version for most cases. **`lzma.LZMAFile`** has also been optimized. (Contributed by Serhiy Storchaka and Nadeem Vawda in bpo-16034 [https://bugs.python.org/issue?@action=redirect&bpo=16034].)
- **`random.getrandbits()`** is 20%-40% faster for small integers (the most common use case). (Contributed by Serhiy Storchaka in bpo-16674 [https://bugs.python.org/issue?@action=redirect&bpo=16674].)
- By taking advantage of the new storage format for strings, pickling of strings is now significantly faster. (Contributed by Victor Stinner and Antoine Pitrou in bpo-15596 [https://bugs.python.org/issue?@action=redirect&bpo=15596].)
- A performance issue in **`io.FileIO.readall()`** has been solved. This particularly affects Windows, and significantly speeds up the case of piping significant amounts of data through **`subprocess`**. (Contributed by Richard Oudkerk in bpo-15758 [https://bugs.python.org/issue?@action=redirect&bpo=15758].)
- **`html.escape()`** is now 10x faster. (Contributed by Matt Bryant in bpo-18020 [https://bugs.python.org/issue?@action=redirect&bpo=18020].)
- On Windows, the native `VirtualAlloc` is now used instead of the CRT `malloc` in `obmalloc`. Artificial benchmarks show about a 3% memory savings.
- **`os.urandom()`** now uses a lazily opened persistent file descriptor so as to avoid using many file descriptors when run in parallel from multiple threads. (Contributed by Antoine Pitrou in bpo-18756 [https://bugs.python.org/issue?@action=redirect&bpo=18756].)

Descontinuados

This section covers various APIs and other features that have been deprecated in Python 3.4, and will be removed in Python 3.5 or later. In most (but not all) cases, using the deprecated APIs will produce a

`DeprecationWarning` when the interpreter is run with deprecation warnings enabled (for example, by using `-Wd`).

Deprecations in the Python API

- As mentioned in PEP 451: A ModuleSpec Type for the Import System, a number of `importlib` methods and functions are deprecated: **`importlib.find_loader()`** is replaced by `importlib.util.find_spec()`; **`importlib.machinery.PathFinder.find_module()`** is replaced by `importlib.machinery.PathFinder.find_spec()`; **`importlib.abc.MetaPathFinder.find_module()`** is replaced by `importlib.abc.MetaPathFinder.find_spec()`; **`importlib.abc.PathEntryFinder.find_loader()`** and **`find_module()`** are replaced by `importlib.abc.PathEntryFinder.find_spec()`; all of the *`xxx`*`Loader` ABC `load_module` methods (`importlib.abc.Loader.load_module`, `importlib.abc.InspectLoader.load_module`, `importlib.abc.FileLoader.load_module`, `importlib.abc.SourceLoader.load_module`) should no longer be implemented, instead loaders should implement an `exec_module` method (`importlib.abc.Loader.exec_module()`, `importlib.abc.InspectLoader.exec_module()` `importlib.abc.SourceLoader.exec_module()`) and let the import system take care of the rest; and **`importlib.abc.Loader.module_repr()`**, **`importlib.util.module_for_loader()`**, **`importlib.util.set_loader()`**, and **`importlib.util.set_package()`** are no longer needed because their functions are now handled automatically by the import system.
- The **`imp`** module is pending deprecation. To keep compatibility with Python 2/3 code bases, the module's removal is currently not scheduled.
- The **`formatter`** module is pending deprecation and is slated for removal in Python 3.6.
- `MD5` as the default *digestmod* for the `hmac.new()` function is deprecated. Python 3.6 will require an explicit digest name or

constructor as *digestmod* argument.

- The internal `Netrc` class in the **`ftplib`** module has been documented as deprecated in its docstring for quite some time. It now emits a **`DeprecationWarning`** and will be removed completely in Python 3.5.
- The undocumented *endtime* argument to **`subprocess.Popen.wait()`** should not have been exposed and is hopefully not in use; it is deprecated and will mostly likely be removed in Python 3.5.
- The *strict* argument of **`HTMLParser`** is deprecated.
- The **`plistlib`** **`readPlist()`**, **`writePlist()`**, **`readPlistFromBytes()`**, and **`writePlistToBytes()`** functions are deprecated in favor of the corresponding new functions **`load()`**, **`dump()`**, **`loads()`**, and **`dumps()`**. **`Data()`** is deprecated in favor of just using the **`bytes`** constructor.
- The **`sysconfig`** key `SO` is deprecated, it has been replaced by `EXT_SUFFIX`.
- The `U` mode accepted by various `open` functions is deprecated. In Python3 it does not do anything useful, and should be replaced by appropriate uses of **`io.TextIOWrapper`** (if needed) and its *newline* argument.
- The *parser* argument of **`xml.etree.ElementTree.iterparse()`** has been deprecated, as has the *html* argument of **`XMLParser()`**. To prepare for the removal of the latter, all arguments to `XMLParser` should be passed by keyword.

Deprecated Features

- Running IDLE — Editor e console Python with the `-n` flag (no subprocess) is deprecated. However, the feature will not be removed until bpo-18823 [https://bugs.python.org/issue?@action=redirect&bpo=18823] is resolved.
- The site module adding a “site-python” directory to sys.path, if it exists, is deprecated (bpo-19375 [https://bugs.python.org/issue?@action=redirect&bpo=19375]).

Removidos

Operating Systems No Longer Supported

Support for the following operating systems has been removed from the source and build tools:

- OS/2 (bpo-16135 [https://bugs.python.org/issue?@action=redirect&bpo=16135]).
- Windows 2000 (changeset e52df05b496a).
- Windows systems where `COMSPEC` points to `command.com` (bpo-14470 [https://bugs.python.org/issue?@action=redirect&bpo=14470]).
- VMS (bpo-16136 [https://bugs.python.org/issue?@action=redirect&bpo=16136]).

Remoção de APIs e recursos

The following obsolete and previously deprecated APIs and features have been removed:

- The unmaintained `Misc/TextMate` and `Misc/vim` directories have been removed (see the devguide [https://devguide.python.org] for suggestions on what to use instead).
- The `SO` makefile macro is removed (it was replaced by the `SHLIB_SUFFIX` and `EXT_SUFFIX` macros) (bpo-16754 [https://bugs.python.org/issue?@action=redirect&bpo=16754]).
- The `PyThreadState.tick_counter` field has been removed; its value has been meaningless since Python 3.2, when the "new GIL" was introduced (bpo-19199 [https://bugs.python.org/issue?@action=redirect&bpo=19199]).
- `PyLoader` and `PyPycLoader` have been removed from **`importlib`**. (Contributed by Taras Lyapun in bpo-15641 [https://bugs.python.org/issue?@action=redirect&bpo=15641].)
- The *strict* argument to **`HTTPConnection`** and **`HTTPSConnection`** has been removed. HTTP 0.9-style "Simple Responses" are no longer supported.
- The deprecated **`urllib.request.Request`** getter and setter methods `add_data`, `has_data`, `get_data`, `get_type`, `get_host`, `get_selector`, `set_proxy`, `get_origin_req_host`, and `is_unverifiable` have been removed (use direct attribute access instead).
- Support for loading the deprecated `TYPE_INT64` has been removed from **`marshal`**. (Contributed by Dan Riti in bpo-15480 [https://bugs.python.org/issue?@action=redirect&bpo=15480].)

- **inspect.Signature**: positional-only parameters are now required to have a valid name.
- **object.__format__()** no longer accepts non-empty format strings, it now raises a **TypeError** instead. Using a non-empty string has been deprecated since Python 3.2. This change has been made to prevent a situation where previously working (but incorrect) code would start failing if an object gained a __format__ method, which means that your code may now raise a **TypeError** if you are using an `'s'` format code with objects that do not have a __format__ method that handles it. See bpo-7994 [https://bugs.python.org/issue?@action=redirect&bpo=7994] for background.
- **difflib.SequenceMatcher.isbjunk()** and **difflib.SequenceMatcher.isbpopular()** were deprecated in 3.2, and have now been removed: use `x in sm.bjunk` and `x in sm.bpopular`, where *sm* is a **SequenceMatcher** object (bpo-13248 [https://bugs.python.org/issue?@action=redirect&bpo=13248]).

Code Cleanups

- The unused and undocumented internal `Scanner` class has been removed from the **pydoc** module.
- The private and effectively unused `_gestalt` module has been removed, along with the private **platform** functions `_mac_ver_lookup`, `_mac_ver_gstalt`, and `_bcd2str`, which would only have ever been called on badly broken OSX systems (see bpo-18393 [https://bugs.python.org/issue?@action=redirect&bpo=18393]).
- The hardcoded copies of certain **stat** constants that were included in the **tarfile** module namespace have been removed.

Porting to Python 3.4

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Changes in ‘python’ Command Behavior

- In a posix shell, setting the **PATH** environment variable to an empty value is equivalent to not setting it at all. However, setting **PYTHONPATH** to an empty value was *not* equivalent to not setting it at all: setting **PYTHONPATH** to an empty value was equivalent to setting it to `.`, which leads to confusion when reasoning by analogy to how **PATH** works. The behavior now conforms to the posix convention for **PATH**.
- The [X refs, Y blocks] output of a debug (`--with-pydebug`) build of the CPython interpreter is now off by default. It can be re-enabled using the `-X showrefcount` option. (Contributed by Ezio Melotti in bpo-17323 [https://bugs.python.org/issue?@action=redirect&bpo=17323].)
- The python command and most stdlib scripts (as well as **argparse**) now output `--version` information to `stdout` instead of `stderr` (for issue list see Other Improvements above).

Alterações na API Python

- The ABCs defined in **importlib.abc** now either raise the appropriate exception or return a default value instead of raising **NotImplementedError** blindly. This will only affect code calling **super()** and falling through all the way to the ABCs. For compatibility, catch both **NotImplementedError** or the appropriate exception as needed.
- The module type now initializes the **__package__** and **__loader__** attributes to `None` by default. To determine if these attributes were set in a backwards-compatible fashion, use e.g. `getattr(module, '__loader__', None) is not None`. (bpo-17115 [https://bugs.python.org/issue?@action=redirect&bpo=17115].)
- **importlib.util.module_for_loader()** now sets `__loader__` and `__package__` unconditionally to properly support reloading. If this is not desired then you will need to set these attributes manually. You can use **importlib.util.module_to_load()** for module management.
- Import now resets relevant attributes (e.g. `__name__`, `__loader__`, `__package__`, `__file__`, `__cached__`) unconditionally when reloading. Note that this restores a pre-3.3 behavior in that it means a module is re-found when re-loaded (bpo-19413 [https://bugs.python.org/issue?@action=redirect&bpo=19413]).

- Frozen packages no longer set `__path__` to a list containing the package name, they now set it to an empty list. The previous behavior could cause the import system to do the wrong thing on submodule imports if there was also a directory with the same name as the frozen package. The correct way to determine if a module is a package or not is to use `hasattr(module, '__path__')` (bpo-18065 [https://bugs.python.org/issue?@action=redirect&bpo=18065]).
- Frozen modules no longer define a `__file__` attribute. It's semantically incorrect for frozen modules to set the attribute as they are not loaded from any explicit location. If you must know that a module comes from frozen code then you can see if the module's `__spec__.location` is set to `'frozen'`, check if the loader is a subclass of **`importlib.machinery.FrozenImporter`**, or if Python 2 compatibility is necessary you can use **`imp.is_frozen()`**.
- **`py_compile.compile()`** now raises **`FileExistsError`** if the file path it would write to is a symlink or a non-regular file. This is to act as a warning that import will overwrite those files with a regular file regardless of what type of file path they were originally.
- **`importlib.abc.SourceLoader.get_source()`** no longer raises **`ImportError`** when the source code being loaded triggers a **`SyntaxError`** or **`UnicodeDecodeError`**. As **`ImportError`** is meant to be raised only when source code cannot be found but it should, it was felt to be over-reaching/overloading of that meaning when the source code is found but improperly structured. If you were catching ImportError before and wish to continue to ignore syntax or decoding issues, catch all three exceptions now.
- **`functools.update_wrapper()`** and **`@functools.wraps`** now correctly set the `__wrapped__` attribute to the function being wrapped, even if that function also had its `__wrapped__` attribute set. This means `__wrapped__` attributes now correctly link a stack of decorated functions rather than every `__wrapped__` attribute in the chain referring to the innermost function. Introspection libraries that assumed the previous behaviour was intentional can use **`inspect.unwrap()`** to access the first function in the chain that has no `__wrapped__` attribute.
- **`inspect.getfullargspec()`** foi reimplementada em cima de **`inspect.signature()`** e, portanto, lida com uma variedade muito maior de objetos que podem ser chamados do que no passado. Espera-

se que chamáveis adicionais de módulos embutidos e de extensão ganhem metadados de assinatura ao longo da série Python 3.4. O código que presume que **`inspect.getfullargspec()`** falhará em chamáveis não-Python pode precisar ser ajustado de acordo.

- **`importlib.machinery.PathFinder`** now passes on the current working directory to objects in **`sys.path_hooks`** for the empty string. This results in **`sys.path_importer_cache`** never containing `''`, thus iterating through **`sys.path_importer_cache`** based on **`sys.path`** will not find all keys. A module's `__file__` when imported in the current working directory will also now have an absolute path, including when using `-m` with the interpreter (except for `__main__.__file__` when a script has been executed directly using a relative path) (Contributed by Brett Cannon in bpo-18416 [https://bugs.python.org/issue?@action=redirect&bpo=18416]). is specified on the command-line) (bpo-18416 [https://bugs.python.org/issue?@action=redirect&bpo=18416]).
- The removal of the *strict* argument to **`HTTPConnection`** and **`HTTPSConnection`** changes the meaning of the remaining arguments if you are specifying them positionally rather than by keyword. If you've been paying attention to deprecation warnings your code should already be specifying any additional arguments via keywords.
- Strings between `from __future__ import ...` statements now *always* raise a **`SyntaxError`**. Previously if there was no leading docstring, an interstitial string would sometimes be ignored. This brings CPython into compliance with the language spec; Jython and PyPy already were. (bpo-17434 [https://bugs.python.org/issue?@action=redirect&bpo=17434]).
- **`ssl.SSLSocket.getpeercert()`** and **`ssl.SSLSocket.do_handshake()`** now raise an **`OSError`** with `ENOTCONN` when the `SSLSocket` is not connected, instead of the previous behavior of raising an **`AttributeError`**. In addition, **`getpeercert()`** will raise a **`ValueError`** if the handshake has not yet been done.
- **`base64.b32decode()`** now raises a **`binascii.Error`** when the input string contains non-b32-alphabet characters, instead of a **`TypeError`**. This particular **`TypeError`** was missed when the other **`TypeError`**s were converted. (Contributed by Serhiy Storchaka in bpo-18011 [https://bugs.python.org/issue?@action=redirect&bpo=18011].) Note: this change was also inadvertently applied in Python 3.3.3.

- The **`file`** attribute is now automatically closed when the creating **`cgi.FieldStorage`** instance is garbage collected. If you were pulling the file object out separately from the **`cgi.FieldStorage`** instance and not keeping the instance alive, then you should either store the entire **`cgi.FieldStorage`** instance or read the contents of the file before the **`cgi.FieldStorage`** instance is garbage collected.
- Calling `read` or `write` on a closed SSL socket now raises an informative **`ValueError`** rather than the previous more mysterious **`AttributeError`** (bpo-9177 [https://bugs.python.org/issue?@action=redirect&bpo=9177]).
- **`slice.indices()`** no longer produces an **`OverflowError`** for huge values. As a consequence of this fix, **`slice.indices()`** now raises a **`ValueError`** if given a negative length; previously it returned nonsense values (bpo-14794 [https://bugs.python.org/issue?@action=redirect&bpo=14794]).
- The **`complex`** constructor, unlike the **`cmath`** functions, was incorrectly accepting **`float`** values if an object's `__complex__` special method returned one. This now raises a **`TypeError`**. (bpo-16290 [https://bugs.python.org/issue?@action=redirect&bpo=16290].)
- The **`int`** constructor in 3.2 and 3.3 erroneously accepts **`float`** values for the *base* parameter. It is unlikely anyone was doing this, but if so, it will now raise a **`TypeError`** (bpo-16772 [https://bugs.python.org/issue?@action=redirect&bpo=16772]).
- Defaults for keyword-only arguments are now evaluated *after* defaults for regular keyword arguments, instead of before. Hopefully no one wrote any code that depends on the previous buggy behavior (bpo-16967 [https://bugs.python.org/issue?@action=redirect&bpo=16967]).
- Stale thread states are now cleared after **`fork()`**. This may cause some system resources to be released that previously were incorrectly kept perpetually alive (for example, database connections kept in thread-local storage). (bpo-17094 [https://bugs.python.org/issue?@action=redirect&bpo=17094].)
- Parameter names in `__annotations__` dicts are now mangled properly, similarly to **`__kwdefaults__`**. (Contributed by Yury Selivanov in bpo-20625 [https://bugs.python.org/issue?@action=redirect&bpo=20625].)
- **`hashlib.hash.name`** now always returns the identifier in lower case. Previously some builtin hashes had uppercase names, but now that it is

a formal public interface the naming has been made consistent (bpo-18532 [https://bugs.python.org/issue?@action=redirect&bpo=18532]).

- Because **`unittest.TestSuite`** now drops references to tests after they are run, test harnesses that reuse a **`TestSuite`** to re-run a set of tests may fail. Test suites should not be re-used in this fashion since it means state is retained between test runs, breaking the test isolation that **`unittest`** is designed to provide. However, if the lack of isolation is considered acceptable, the old behavior can be restored by creating a **`TestSuite`** subclass that defines a `_removeTestAtIndex` method that does nothing (see **`TestSuite.__iter__()`**) (bpo-11798 [https://bugs.python.org/issue?@action=redirect&bpo=11798]).
- **`unittest`** now uses **`argparse`** for command line parsing. There are certain invalid command forms that used to work that are no longer allowed; in theory this should not cause backward compatibility issues since the disallowed command forms didn't make any sense and are unlikely to be in use.
- The **`re.split()`**, **`re.findall()`**, and **`re.sub()`** functions, and the **`group()`** and **`groups()`** methods of `match` objects now always return a *bytes* object when the string to be matched is a bytes-like object. Previously the return type matched the input type, so if your code was depending on the return value being, say, a `bytearray`, you will need to change your code.
- **`audioop`** functions now raise an error immediately if passed string input, instead of failing randomly later on (bpo-16685 [https://bugs.python.org/issue?@action=redirect&bpo=16685]).
- The new *convert_charrefs* argument to **`HTMLParser`** currently defaults to `False` for backward compatibility, but will eventually be changed to default to `True`. It is recommended that you add this keyword, with the appropriate value, to any **`HTMLParser`** calls in your code (bpo-13633 [https://bugs.python.org/issue?@action=redirect&bpo=13633]).
- Since the *digestmod* argument to the **`hmac.new()`** function will in the future have no default, all calls to **`hmac.new()`** should be changed to explicitly specify a *digestmod* (bpo-17276 [https://bugs.python.org/issue?@action=redirect&bpo=17276]).
- Calling **`sysconfig.get_config_var()`** with the `SO` key, or looking `SO` up in the results of a call to **`sysconfig.get_config_vars()`** is deprecated. This key should be replaced by `EXT_SUFFIX` or

`SHLIB_SUFFIX`, depending on the context (bpo-19555 [https://bugs.python.org/issue?@action=redirect&bpo=19555]).

- Any calls to `open` functions that specify `U` should be modified. `U` is ineffective in Python3 and will eventually raise an error if used. Depending on the function, the equivalent of its old Python2 behavior can be achieved using either a *newline* argument, or if necessary by wrapping the stream in **`TextIOWrapper`** to use its *newline* argument (bpo-15204 [https://bugs.python.org/issue?@action=redirect&bpo=15204]).
- If you use `pyvenv` in a script and desire that pip *not* be installed, you must add `--without-pip` to your command invocation.
- The default behavior of **`json.dump()`** and **`json.dumps()`** when an indent is specified has changed: it no longer produces trailing spaces after the item separating commas at the ends of lines. This will matter only if you have tests that are doing white-space-sensitive comparisons of such output (bpo-16333 [https://bugs.python.org/issue?@action=redirect&bpo=16333]).
- **`doctest`** now looks for doctests in extension module `__doc__` strings, so if your doctest test discovery includes extension modules that have things that look like doctests in them you may see test failures you've never seen before when running your tests (bpo-3158 [https://bugs.python.org/issue?@action=redirect&bpo=3158]).
- The **`collections.abc`** module has been slightly refactored as part of the Python startup improvements. As a consequence of this, it is no longer the case that importing **`collections`** automatically imports **`collections.abc`**. If your program depended on the (undocumented) implicit import, you will need to add an explicit `import collections.abc` (bpo-20784 [https://bugs.python.org/issue?@action=redirect&bpo=20784]).

Alterações na API C

- **`PyEval_EvalFrameEx()`**, **`PyObject_Repr()`**, and **`PyObject_Str()`**, along with some other internal C APIs, now include a debugging assertion that ensures they are not used in situations where they may silently discard a currently active exception. In cases where discarding the active exception is expected and desired (for example, because it has already been saved locally with **`PyErr_Fetch()`** or is being

deliberately replaced with a different exception), an explicit `PyErr_Clear()` call will be needed to avoid triggering the assertion when invoking these operations (directly or indirectly) and running against a version of Python that is compiled with assertions enabled.

- `PyErr_SetImportError()` now sets `TypeError` when its **msg** argument is not set. Previously only `NULL` was returned with no exception set.
- The result of the `PyOS_ReadlineFunctionPointer` callback must now be a string allocated by `PyMem_RawMalloc()` or `PyMem_RawRealloc()`, or `NULL` if an error occurred, instead of a string allocated by `PyMem_Malloc()` or `PyMem_Realloc()` (bpo-16742 [https://bugs.python.org/issue?@action=redirect&bpo=16742])
- `PyThread_set_key_value()` now always set the value. In Python 3.3, the function did nothing if the key already exists (if the current value is a non-`NULL` pointer).
- The `f_tstate` (thread state) field of the `PyFrameObject` structure has been removed to fix a bug: see bpo-14432 [https://bugs.python.org/issue?@action=redirect&bpo=14432] for the rationale.

Alterado em 3.4.3

PEP 476: Habilitando verificação de certificado por padrão para clientes http stdlib

`http.client` and modules which use it, such as `urllib.request` and `xmlrpc.client`, will now verify that the server presents a certificate which is signed by a CA in the platform trust store and whose hostname matches the hostname being requested by default, significantly improving security for many applications.

Para aplicações que exigem o antigo comportamento anterior, elas podem passar um contexto alternativo:

```
import urllib.request
import ssl

# This disables all verification
context = ssl._create_unverified_context()
```

```
# This allows using a specific certificate for the host, which c
# to be in the trust store
context = ssl.create_default_context(cafile="/path/to/file.crt")

urllib.request.urlopen("https://invalid-cert", context=context)
```

O que há de novo no Python 3.3

This article explains the new features in Python 3.3, compared to 3.2. Python 3.3 was released on September 29, 2012. For full details, see the changelog [https://docs.python.org/3.3/whatsnew/changelog.html].

Ver também

PEP 398 [https://peps.python.org/pep-0398/] - Python 3.3 Release Schedule

Resumo – Destaques da versão

Novos recursos de sintaxe:

- New `yield from` expression for generator delegation.
- A sintaxe `u'unicode'` é aceita novamente para objetos tipo `str`.

Novos módulos de biblioteca:

- `faulthandler` (helps debugging low-level crashes)
- `ipaddress` (high-level objects representing IP addresses and masks)
- `lzma` (compressão de dados usando o algoritmo XZ / LZMA)
- `unittest.mock` (replace parts of your system under test with mock objects)
- `venv` (Python virtual environments, as in the popular `virtualenv` package)

Novos recursos embutidos:

- Reworked I/O exception hierarchy.

Melhorias na implementação:

- Rewritten import machinery based on `importlib`.

- More compact unicode strings.
- More compact attribute dictionaries.

Significantly Improved Library Modules:

- C Accelerator for the decimal module.
- Better unicode handling in the email module (provisional).

Melhorias de Segurança:

- Hash randomization is switched on by default.

Please read on for a comprehensive list of user-facing changes.

PEP 405: Ambientes Virtuais

Virtual environments help create separate Python setups while sharing a system-wide base install, for ease of maintenance. Virtual environments have their own set of private site packages (i.e. locally installed libraries), and are optionally segregated from the system-wide site packages. Their concept and implementation are inspired by the popular `virtualenv` third-party package, but benefit from tighter integration with the interpreter core.

This PEP adds the **`venv`** module for programmatic access, and the `pyvenv` script for command-line access and administration. The Python interpreter checks for a `pyvenv.cfg`, file whose existence signals the base of a virtual environment's directory tree.

Ver também

PEP 405 [https://peps.python.org/pep-0405/] - Ambientes Virtuais Python
PEP written by Carl Meyer; implementation by Carl Meyer and Vinay Sajip

PEP 420: Implicit Namespace Packages

Native support for package directories that don't require `__init__.py` marker files and can automatically span multiple path segments (inspired by various third party approaches to namespace packages, as described in **PEP 420** [https://peps.python.org/pep-0420/])

Ver também

PEP 420 **[https://peps.python.org/pep-0420/]** **- Implicit Namespace Packages**
PEP written by Eric V. Smith; implementation by Eric V. Smith and Barry Warsaw

PEP 3118: New memoryview implementation and buffer protocol documentation

A implementação de **PEP 3118** [https://peps.python.org/pep-3118/] foi significativamente melhorada.

The new memoryview implementation comprehensively fixes all ownership and lifetime issues of dynamically allocated fields in the Py_buffer struct that led to multiple crash reports. Additionally, several functions that crashed or returned incorrect results for non-contiguous or multi-dimensional input have been fixed.

The memoryview object now has a PEP-3118 compliant getbufferproc() that checks the consumer's request type. Many new features have been added, most of them work in full generality for non-contiguous arrays and arrays with suboffsets.

The documentation has been updated, clearly spelling out responsibilities for both exporters and consumers. Buffer request flags are grouped into basic and compound flags. The memory layout of non-contiguous and multi-dimensional NumPy-style arrays is explained.

Recursos

- All native single character format specifiers in struct module syntax (optionally prefixed with '@') are now supported.
- With some restrictions, the cast() method allows changing of format and shape of C-contiguous arrays.
- Multi-dimensional list representations are supported for any array type.
- Multi-dimensional comparisons are supported for any array type.
- One-dimensional memoryviews of hashable (read-only) types with formats B, b or c are now hashable. (Contributed by Antoine Pitrou in bpo-13411 [https://bugs.python.org/issue?@action=redirect&bpo=13411].)
- Arbitrary slicing of any 1-D arrays type is supported. For example, it is now possible to reverse a memoryview in *O*(1) by using a negative step.

Mudanças na API

- The maximum number of dimensions is officially limited to 64.
- The representation of empty shape, strides and suboffsets is now an empty tuple instead of `None`.
- Accessing a memoryview element with format 'B' (unsigned bytes) now returns an integer (in accordance with the struct module syntax). For returning a bytes object the view must be cast to 'c' first.
- memoryview comparisons now use the logical structure of the operands and compare all array elements by value. All format strings in struct module syntax are supported. Views with unrecognised format strings are still permitted, but will always compare as unequal, regardless of view contents.
- For further changes see Build and C API Changes and Porting C code.

(Contributed by Stefan Krah in bpo-10181 [https://bugs.python.org/issue?@action=redirect&bpo=10181].)

Ver também

PEP 3118 [https://peps.python.org/pep-3118/] - Revising the Buffer Protocol

PEP 393: Flexible String Representation

The Unicode string type is changed to support multiple internal representations, depending on the character with the largest Unicode ordinal (1, 2, or 4 bytes) in the represented string. This allows a space-efficient representation in common cases, but gives access to full UCS-4 on all systems. For compatibility with existing APIs, several representations may exist in parallel; over time, this compatibility should be phased out.

On the Python side, there should be no downside to this change.

On the C API side, **PEP 393** [https://peps.python.org/pep-0393/] is fully backward compatible. The legacy API should remain available at least five years. Applications using the legacy API will not fully benefit of the memory reduction, or - worse - may use a bit more memory, because Python may have to maintain two versions of each string (in the legacy format and in the new efficient storage).

Funcionalidade

Changes introduced by **PEP 393** [https://peps.python.org/pep-0393/] are the following:

- Python now always supports the full range of Unicode code points, including non-BMP ones (i.e. from `U+0000` to `U+10FFFF`). The distinction between narrow and wide builds no longer exists and Python now behaves like a wide build, even under Windows.
- With the death of narrow builds, the problems specific to narrow builds have also been fixed, for example:
  - **`len()`** now always returns 1 for non-BMP characters, so `len('\U0010FFFF') == 1`;
  - surrogate pairs are not recombined in string literals, so `'\uDBFF\uDFFF' != '\U0010FFFF'`;
  - indexing or slicing non-BMP characters returns the expected value, so `'\U0010FFFF'[0]` now returns `'\U0010FFFF'` and not `'\uDBFF'`;

    - all other functions in the standard library now correctly handle non-BMP code points.
- The value of `sys.maxunicode` is now always `1114111` (`0x10FFFF` in hexadecimal). The `PyUnicode_GetMax()` function still returns either `0xFFFF` or `0x10FFFF` for backward compatibility, and it should not be used with the new Unicode API (see bpo-13054 [https://bugs.python.org/issue?@action=redirect&bpo=13054]).
- The `./configure` flag `--with-wide-unicode` has been removed.

Desempenho e utilização de recursos

The storage of Unicode strings now depends on the highest code point in the string:

- pure ASCII and Latin1 strings (`U+0000-U+00FF`) use 1 byte per code point;
- BMP strings (`U+0000-U+FFFF`) use 2 bytes per code point;
- non-BMP strings (`U+10000-U+10FFFF`) use 4 bytes per code point.

The net effect is that for most applications, memory usage of string storage should decrease significantly - especially compared to former wide unicode builds - as, in many cases, strings will be pure ASCII even in international contexts (because many strings store non-human language data, such as XML fragments, HTTP headers, JSON-encoded data, etc.). We also hope that it will, for the same reasons, increase CPU cache efficiency on non-trivial applications. The memory usage of Python 3.3 is two to three times smaller than Python 3.2, and a little bit better than Python 2.7, on a Django benchmark (see the PEP for details).

Ver também

PEP 393 [https://peps.python.org/pep-0393/] - Flexible String Representation
PEP written by Martin von Löwis; implementation by Torsten Becker and Martin von Löwis.

PEP 397: Python Launcher for Windows

The Python 3.3 Windows installer now includes a `py` launcher application that can be used to launch Python applications in a version independent fashion.

This launcher is invoked implicitly when double-clicking `*.py` files. If only a single Python version is installed on the system, that version will be used to run the file. If multiple versions are installed, the most recent version is used by default, but this can be overridden by including a Unix-style "shebang line" in the Python script.

The launcher can also be used explicitly from the command line as the `py` application. Running `py` follows the same version selection rules as implicitly launching scripts, but a more specific version can be selected by passing appropriate arguments (such as `-3` to request Python 3 when Python 2 is also installed, or `-2.6` to specifically request an earlier Python version when a more recent version is installed).

In addition to the launcher, the Windows installer now includes an option to add the newly installed Python to the system PATH. (Contributed by Brian Curtin in bpo-3561 [https://bugs.python.org/issue?@action=redirect&bpo=3561].)

Ver também

PEP 397 [https://peps.python.org/pep-0397/] - Python Launcher for Windows
PEP written by Mark Hammond and Martin v. Löwis; implementation by Vinay Sajip.

Launcher documentation: Gerenciador de instalação do Python

Installer PATH modification: Gerenciador de instalação do Python

PEP 3151: Reworking the OS and IO exception hierarchy

The hierarchy of exceptions raised by operating system errors is now both simplified and finer-grained.

You don't have to worry anymore about choosing the appropriate exception type between **OSError**, **IOError**, **EnvironmentError**, **WindowsError**, **mmap.error**, **socket.error** or **select.error**. All these exception types are now only one: **OSError**. The other names are kept as aliases for compatibility reasons.

Also, it is now easier to catch a specific error condition. Instead of inspecting the `errno` attribute (or `args[0]`) for a particular constant from the **errno** module, you can catch the adequate **OSError** subclass. The available subclasses are the following:

- **BlockingIOError**
- **ChildProcessError**
- **ConnectionError**
- **FileExistsError**
- **FileNotFoundError**
- **InterruptedError**
- **IsADirectoryError**
- **NotADirectoryError**
- **PermissionError**
- **ProcessLookupError**
- **TimeoutError**

And the **ConnectionError** itself has finer-grained subclasses:

- **BrokenPipeError**
- **ConnectionAbortedError**
- **ConnectionRefusedError**
- **ConnectionResetError**

Thanks to the new exceptions, common usages of the **errno** can now be avoided. For example, the following code written for Python 3.2:

```
from errno import ENOENT, EACCES, EPERM

try:
    with open("document.txt") as f:
        content = f.read()
except IOError as err:
```

```
    if err.errno == ENOENT:
        print("document.txt file is missing")
    elif err.errno in (EACCES, EPERM):
        print("You are not allowed to read document.txt")
    else:
        raise
```

can now be written without the `errno` import and without manual inspection of exception attributes:

```
try:
    with open("document.txt") as f:
        content = f.read()
except FileNotFoundError:
    print("document.txt file is missing")
except PermissionError:
    print("You are not allowed to read document.txt")
```

Ver também

PEP 3151 [https://peps.python.org/pep-3151/] - Reworking the OS and IO Exception Hierarchy
PEP escrita e implmentada por Antoine Pitrou

PEP 380: Syntax for Delegating to a Subgenerator

PEP 380 adds the `yield from` expression, allowing a generator to delegate part of its operations to another generator. This allows a section of code containing `yield` to be factored out and placed in another generator. Additionally, the subgenerator is allowed to return with a value, and the value is made available to the delegating generator.

While designed primarily for use in delegating to a subgenerator, the `yield from` expression actually allows delegation to arbitrary subiterators.

For simple iterators, `yield from iterable` is essentially just a shortened form of `for item in iterable: yield item`:

```
>>> def g(x):
...     yield from range(x, 0, -1)
...     yield from range(x)
...
>>> list(g(5))
[5, 4, 3, 2, 1, 0, 1, 2, 3, 4]
```

However, unlike an ordinary loop, `yield from` allows subgenerators to receive sent and thrown values directly from the calling scope, and return a final value to the outer generator:

```
>>> def accumulate():
...     tally = 0
...     while 1:
...         next = yield
...         if next is None:
...             return tally
...         tally += next
...
>>> def gather_tallies(tallies):
...     while 1:
...         tally = yield from accumulate()
...         tallies.append(tally)
...
>>> tallies = []
>>> acc = gather_tallies(tallies)
>>> next(acc)  # Ensure the accumulator is ready to accept value
>>> for i in range(4):
...     acc.send(i)
...
>>> acc.send(None)  # Finish the first tally
>>> for i in range(5):
...     acc.send(i)
...
>>> acc.send(None)  # Finish the second tally
>>> tallies
[6, 10]
```

The main principle driving this change is to allow even generators that are designed to be used with the `send` and `throw` methods to be split into

multiple subgenerators as easily as a single large function can be split into multiple subfunctions.

Ver também

PEP 380 [https://peps.python.org/pep-0380/] - Sintaxe para Delegar a um Subgerador

PEP written by Greg Ewing; implementation by Greg Ewing, integrated into 3.3 by Renaud Blanch, Ryan Kelly and Nick Coghlan; documentation by Zbigniew Jędrzejewski-Szmek and Nick Coghlan

PEP 409: Suppressing exception context

PEP 409 introduces new syntax that allows the display of the chained exception context to be disabled. This allows cleaner error messages in applications that convert between exception types:

```
>>> class D:
...     def __init__(self, extra):
...         self._extra_attributes = extra
...     def __getattr__(self, attr):
...         try:
...             return self._extra_attributes[attr]
...         except KeyError:
...             raise AttributeError(attr) from None
...
>>> D({}).x
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 8, in __getattr__
AttributeError: x
```

Without the `from None` suffix to suppress the cause, the original exception would be displayed by default:

```
>>> class C:
...     def __init__(self, extra):
...         self._extra_attributes = extra
```

```
...     def __getattr__(self, attr):
...         try:
...             return self._extra_attributes[attr]
...         except KeyError:
...             raise AttributeError(attr)
...
>>> C({}).x
Traceback (most recent call last):
  File "<stdin>", line 6, in __getattr__
KeyError: 'x'

During handling of the above exception, another exception occurr

Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 8, in __getattr__
AttributeError: x
```

No debugging capability is lost, as the original exception context remains available if needed (for example, if an intervening library has incorrectly suppressed valuable underlying details):

```
>>> try:
...     D({}).x
... except AttributeError as exc:
...     print(repr(exc.__context__))
...
KeyError('x',)
```

Ver também

PEP 409 **[https://peps.python.org/pep-0409/]** **- Suppressing exception context**
PEP written by Ethan Furman; implemented by Ethan Furman and Nick Coghlan.

PEP 414: Explicit Unicode literals

To ease the transition from Python 2 for Unicode aware Python applications that make heavy use of Unicode literals, Python 3.3 once again supports the "u" prefix for string literals. This prefix has no semantic significance in Python 3, it is provided solely to reduce the number of purely mechanical changes in migrating to Python 3, making it easier for developers to focus on the more significant semantic changes (such as the stricter default separation of binary and text data).

Ver também

PEP 414 [https://peps.python.org/pep-0414/] - Explicit Unicode literals
PEP written by Armin Ronacher.

PEP 3155: Qualified name for classes and functions

Functions and class objects have a new `__qualname__` attribute representing the "path" from the module top-level to their definition. For global functions and classes, this is the same as `__name__`. For other functions and classes, it provides better information about where they were actually defined, and how they might be accessible from the global scope.

Example with (non-bound) methods:

```
>>> class C:
...     def meth(self):
...         pass
...
>>> C.meth.__name__
'meth'
>>> C.meth.__qualname__
'C.meth'
```

Example with nested classes:

```
>>> class C:
...     class D:
```

```
...         def meth(self):
...             pass
...
>>> C.D.__name__
'D'
>>> C.D.__qualname__
'C.D'
>>> C.D.meth.__name__
'meth'
>>> C.D.meth.__qualname__
'C.D.meth'
```

Example with nested functions:

```
>>> def outer():
...     def inner():
...         pass
...     return inner
...
>>> outer().__name__
'inner'
>>> outer().__qualname__
'outer.<locals>.inner'
```

The string representation of those objects is also changed to include the new, more precise information:

```
>>> str(C.D)
"<class '__main__.C.D'>"
>>> str(C.D.meth)
'<function C.D.meth at 0x7f46b9fe31e0>'
```

Ver também

PEP 3155 [https://peps.python.org/pep-3155/] - Qualified name for classes and functions

PEP escrita e implementada por Antoine Pitrou.

PEP 412: Key-Sharing Dictionary

Dictionaries used for the storage of objects' attributes are now able to share part of their internal storage between each other (namely, the part which stores the keys and their respective hashes). This reduces the memory consumption of programs creating many instances of non-builtin types.

Ver também

PEP 412 [https://peps.python.org/pep-0412/] - Key-Sharing Dictionary
PEP written and implemented by Mark Shannon.

PEP 362: Function Signature Object

A new function `inspect.signature()` makes introspection of python callables easy and straightforward. A broad range of callables is supported: python functions, decorated or not, classes, and `functools.partial()` objects. New classes `inspect.Signature`, `inspect.Parameter` and `inspect.BoundArguments` hold information about the call signatures, such as, annotations, default values, parameters kinds, and bound arguments, which considerably simplifies writing decorators and any code that validates or amends calling signatures or arguments.

Ver também

PEP 362 [https://peps.python.org/pep-0362/]: - Function Signature Object
PEP written by Brett Cannon, Yury Selivanov, Larry Hastings, Jiwon Seo; implemented by Yury Selivanov.

PEP 421: Adding sys.implementation

A new attribute on the `sys` module exposes details specific to the implementation of the currently running interpreter. The initial set of attributes on `sys.implementation` are `name`, `version`, `hexversion`, and `cache_tag`.

The intention of `sys.implementation` is to consolidate into one namespace the implementation-specific data used by the standard library. This allows different Python implementations to share a single standard library code base much more easily. In its initial state, `sys.implementation` holds only a small portion of the implementation-specific data. Over time that ratio will shift in order to make the standard library more portable.

One example of improved standard library portability is `cache_tag`. As of Python 3.3, `sys.implementation.cache_tag` is used by **`importlib`** to support **PEP 3147** [https://peps.python.org/pep-3147/] compliance. Any Python implementation that uses `importlib` for its built-in import system may use `cache_tag` to control the caching behavior for modules.

SimpleNamespace

The implementation of `sys.implementation` also introduces a new type to Python: **`types.SimpleNamespace`**. In contrast to a mapping-based namespace, like **`dict`**, `SimpleNamespace` is attribute-based, like **`object`**. However, unlike `object`, `SimpleNamespace` instances are writable. This means that you can add, remove, and modify the namespace through normal attribute access.

Ver também

PEP 421 [https://peps.python.org/pep-0421/] - Adding sys.implementation
PEP written and implemented by Eric Snow.

Using importlib as the Implementation of Import

bpo-2377 [https://bugs.python.org/issue?@action=redirect&bpo=2377] - Replace __import__ w/ importlib.__import__ bpo-13959 [https://bugs.python.org/issue?@action=redirect&bpo=13959] - Re-implement parts of **`imp`** in pure Python bpo-14605 [https://bugs.python.org/issue?@action=redirect&bpo=14605] - Make import machinery explicit bpo-14646 [https://bugs.python.org/issue?@action=redirect&bpo=14646] - Require loaders set __loader__ and __package__

The `__import__()` function is now powered by `importlib.__import__()`. This work leads to the completion of "phase 2" of **PEP 302** [https://peps.python.org/pep-0302/]. There are multiple benefits to this change. First, it has allowed for more of the machinery powering import to be exposed instead of being implicit and hidden within the C code. It also provides a single implementation for all Python VMs supporting Python 3.3 to use, helping to end any VM-specific deviations in import semantics. And finally it eases the maintenance of import, allowing for future growth to occur.

For the common user, there should be no visible change in semantics. For those whose code currently manipulates import or calls import programmatically, the code changes that might possibly be required are covered in the Porting Python code section of this document.

New APIs

One of the large benefits of this work is the exposure of what goes into making the import statement work. That means the various importers that were once implicit are now fully exposed as part of the `importlib` package.

The abstract base classes defined in `importlib.abc` have been expanded to properly delineate between meta path finders and path entry finders by introducing `importlib.abc.MetaPathFinder` and `importlib.abc.PathEntryFinder`, respectively. The old ABC of `importlib.abc.Finder` is now only provided for backwards-compatibility and does not enforce any method requirements.

In terms of finders, `importlib.machinery.FileFinder` exposes the mechanism used to search for source and bytecode files of a module. Previously this class was an implicit member of `sys.path_hooks`.

For loaders, the new abstract base class `importlib.abc.FileLoader` helps write a loader that uses the file system as the storage mechanism for a module's code. The loader for source files (`importlib.machinery.SourceFileLoader`), sourceless bytecode files (`importlib.machinery.SourcelessFileLoader`), and extension modules

(`importlib.machinery.ExtensionFileLoader`) are now available for direct use.

`ImportError` now has `name` and `path` attributes which are set when there is relevant data to provide. The message for failed imports will also provide the full name of the module now instead of just the tail end of the module's name.

The `importlib.invalidate_caches()` function will now call the method with the same name on all finders cached in `sys.path_importer_cache` to help clean up any stored state as necessary.

Visible Changes

For potential required changes to code, see the Porting Python code section.

Beyond the expanse of what `importlib` now exposes, there are other visible changes to import. The biggest is that `sys.meta_path` and `sys.path_hooks` now store all of the meta path finders and path entry hooks used by import. Previously the finders were implicit and hidden within the C code of import instead of being directly exposed. This means that one can now easily remove or change the order of the various finders to fit one's needs.

Another change is that all modules have a `__loader__` attribute, storing the loader used to create the module. **PEP 302** [https://peps.python.org/pep-0302/] has been updated to make this attribute mandatory for loaders to implement, so in the future once 3rd-party loaders have been updated people will be able to rely on the existence of the attribute. Until such time, though, import is setting the module post-load.

Loaders are also now expected to set the `__package__` attribute from **PEP 366** [https://peps.python.org/pep-0366/]. Once again, import itself is already setting this on all loaders from `importlib` and import itself is setting the attribute post-load.

`None` is now inserted into `sys.path_importer_cache` when no finder can be found on `sys.path_hooks`. Since `imp.NullImporter` is not directly exposed

on **`sys.path_hooks`** it could no longer be relied upon to always be available to use as a value representing no finder found.

All other changes relate to semantic changes which should be taken into consideration when updating code for Python 3.3, and thus should be read about in the Porting Python code section of this document.

(Implementation by Brett Cannon)

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- Added support for Unicode name aliases and named sequences. Both **`unicodedata.lookup()`** and `'\N{...}'` now resolve name aliases, and **`unicodedata.lookup()`** resolves named sequences too.

  (Contributed by Ezio Melotti in bpo-12753 [https://bugs.python.org/issue?@action=redirect&bpo=12753].)

- Unicode database updated to UCD version 6.1.0

- Equality comparisons on **`range()`** objects now return a result reflecting the equality of the underlying sequences generated by those range objects. (bpo-13201 [https://bugs.python.org/issue?@action=redirect&bpo=13201])

- The `count()`, `find()`, `rfind()`, `index()` and `rindex()` methods of **`bytes`** and **`bytearray`** objects now accept an integer between 0 and 255 as their first argument.

  (Contributed by Petri Lehtinen in bpo-12170 [https://bugs.python.org/issue?@action=redirect&bpo=12170].)

- The `rjust()`, `ljust()`, and `center()` methods of **`bytes`** and **`bytearray`** now accept a **`bytearray`** for the `fill` argument. (Contributed by Petri Lehtinen in bpo-12380 [https://bugs.python.org/issue?@action=redirect&bpo=12380].)

- New methods have been added to **`list`** and **`bytearray`**: `copy()` and `clear()` (bpo-10516 [https://bugs.python.org/issue?@action=redirect&bpo=10516]). Consequently, **`MutableSequence`** now also defines a **`clear()`** method (bpo-11388 [https://bugs.python.org/issue?@action=redirect&bpo=11388]).

- Raw bytes literals can now be written `rb"..."` as well as `br"..."`.

  (Contributed by Antoine Pitrou in bpo-13748 [https://bugs.python.org/issue?@action=redirect&bpo=13748].)

- **`dict.setdefault()`** now does only one lookup for the given key, making it atomic when used with built-in types.

  (Contributed by Filip Gruszczyński in bpo-13521 [https://bugs.python.org/issue?@action=redirect&bpo=13521].)

- The error messages produced when a function call does not match the function signature have been significantly improved.

  (Contributed by Benjamin Peterson.)

A Finer-Grained Import Lock

Previous versions of CPython have always relied on a global import lock. This led to unexpected annoyances, such as deadlocks when importing a module would trigger code execution in a different thread as a side-effect. Clumsy workarounds were sometimes employed, such as the **`PyImport_ImportModuleNoBlock()`** C API function.

In Python 3.3, importing a module takes a per-module lock. This correctly serializes importation of a given module from multiple threads (preventing the exposure of incompletely initialized modules), while eliminating the aforementioned annoyances.

(Contributed by Antoine Pitrou in bpo-9260 [https://bugs.python.org/issue?@action=redirect&bpo=9260].)

Builtin functions and types

- `open()` gets a new *opener* parameter: the underlying file descriptor for the file object is then obtained by calling *opener* with (*file*, *flags*). It can be used to use custom flags like `os.O_CLOEXEC` for example. The `'x'` mode was added: open for exclusive creation, failing if the file already exists.
- `print()`: added the *flush* keyword argument. If the *flush* keyword argument is true, the stream is forcibly flushed.
- `hash()`: hash randomization is enabled by default, see `object.__hash__()` and `PYTHONHASHSEED`.
- The `str` type gets a new `casefold()` method: return a casefolded copy of the string, casefolded strings may be used for caseless matching. For example, `'ß'.casefold()` returns `'ss'`.
- The sequence documentation has been substantially rewritten to better explain the binary/text sequence distinction and to provide specific documentation sections for the individual builtin sequence types (bpo-4966 [https://bugs.python.org/issue?@action=redirect&bpo=4966]).

Novos módulos

faulthandler

This new debug module `faulthandler` contains functions to dump Python tracebacks explicitly, on a fault (a crash like a segmentation fault), after a timeout, or on a user signal. Call `faulthandler.enable()` to install fault handlers for the `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS`, and `SIGILL` signals. You can also enable them at startup by setting the `PYTHONFAULTHANDLER` environment variable or by using `-X` `faulthandler` command line option.

Example of a segmentation fault on Linux:

```
$ python -q -X faulthandler
>>> import ctypes
>>> ctypes.string_at(0)
Fatal Python error: Segmentation fault
```

```
Current thread 0x00007fb899f39700:
  File "/home/python/cpython/Lib/ctypes/__init__.py", line 486 i
  File "<stdin>", line 1 in <module>
Segmentation fault
```

ipaddress

The new **`ipaddress`** module provides tools for creating and manipulating objects representing IPv4 and IPv6 addresses, networks and interfaces (i.e. an IP address associated with a specific IP subnet).

(Contributed by Google and Peter Moody in **PEP 3144** [https://peps.python.org/pep-3144/].)

lzma

The newly added **`lzma`** module provides data compression and decompression using the LZMA algorithm, including support for the `.xz` and `.lzma` file formats.

(Contributed by Nadeem Vawda and Per Øyvind Karlsen in bpo-6715 [https://bugs.python.org/issue?@action=redirect&bpo=6715].)

Módulos melhorados

abc

Improved support for abstract base classes containing descriptors composed with abstract methods. The recommended approach to declaring abstract descriptors is now to provide **`__isabstractmethod__`** as a dynamically updated property. The built-in descriptors have been updated accordingly.

- **`@abc.abstractproperty`** has been deprecated, use **`@property`** with **`@abc.abstractmethod`** instead.
- **`@abc.abstractclassmethod`** has been deprecated, use **`@classmethod`** with **`@abc.abstractmethod`** instead.

- `@abc.abstractstaticmethod` has been deprecated, use `@staticmethod` with `@abc.abstractmethod` instead.

(Contributed by Darren Dale in bpo-11610 [https://bugs.python.org/issue?@action=redirect&bpo=11610].)

`abc.ABCMeta.register()` now returns the registered subclass, which means it can now be used as a class decorator (bpo-10868 [https://bugs.python.org/issue?@action=redirect&bpo=10868]).

array

The `array` module supports the long long type using `q` and `Q` type codes.

(Contributed by Oren Tirosh and Hirokazu Yamamoto in bpo-1172711 [https://bugs.python.org/issue?@action=redirect&bpo=1172711].)

base64

ASCII-only Unicode strings are now accepted by the decoding functions of the `base64` modern interface. For example, `base64.b64decode('YWJj')` returns `b'abc'`. (Contributed by Catalin Iacob in bpo-13641 [https://bugs.python.org/issue?@action=redirect&bpo=13641].)

binascii

In addition to the binary objects they normally accept, the `a2b_` functions now all also accept ASCII-only strings as input. (Contributed by Antoine Pitrou in bpo-13637 [https://bugs.python.org/issue?@action=redirect&bpo=13637].)

bz2

The `bz2` module has been rewritten from scratch. In the process, several new features have been added:

- New `bz2.open()` function: open a bzip2-compressed file in binary or text mode.

- **bz2.BZ2File** can now read from and write to arbitrary file-like objects, by means of its constructor's *fileobj* argument.

  (Contributed by Nadeem Vawda in bpo-5863 [https://bugs.python.org/issue?@action=redirect&bpo=5863].)

- **bz2.BZ2File** and **bz2.decompress()** can now decompress multi-stream inputs (such as those produced by the **pbzip2** tool). **bz2.BZ2File** can now also be used to create this type of file, using the `'a'` (append) mode.

  (Contributed by Nir Aides in bpo-1625 [https://bugs.python.org/issue?@action=redirect&bpo=1625].)

- **bz2.BZ2File** now implements all of the **io.BufferedIOBase** API, except for the **detach()** and **truncate()** methods.

codecs

The **mbcs** codec has been rewritten to handle correctly `replace` and `ignore` error handlers on all Windows versions. The **mbcs** codec now supports all error handlers, instead of only `replace` to encode and `ignore` to decode.

A new Windows-only codec has been added: `cp65001` (bpo-13216 [https://bugs.python.org/issue?@action=redirect&bpo=13216]). It is the Windows code page 65001 (Windows UTF-8, `CP_UTF8`). For example, it is used by `sys.stdout` if the console output code page is set to cp65001 (e.g., using `chcp 65001` command).

Multibyte CJK decoders now resynchronize faster. They only ignore the first byte of an invalid byte sequence. For example, `b'\xff\n'.decode('gb2312', 'replace')` now returns a `\n` after the replacement character.

(bpo-12016 [https://bugs.python.org/issue?@action=redirect&bpo=12016])

Incremental CJK codec encoders are no longer reset at each call to their encode() methods. For example:

```
>>> import codecs
>>> encoder = codecs.getincrementalencoder('hz')('strict')
>>> b''.join(encoder.encode(x) for x in '\u52ff\u65bd\u65bc\u4eb
b'~{NpJ)l6HK!#~} Bye.'
```

This example gives `b'~{Np~}~{J)~}~{l6~}~{HK~}~{!#~} Bye.'` with older Python versions.

(bpo-12100 [https://bugs.python.org/issue?@action=redirect&bpo=12100])

The `unicode_internal` codec has been deprecated.

collections

Addition of a new **`ChainMap`** class to allow treating a number of mappings as a single unit. (Written by Raymond Hettinger for bpo-11089 [https://bugs.python.org/issue?@action=redirect&bpo=11089], made public in bpo-11297 [https://bugs.python.org/issue?@action=redirect&bpo=11297].)

The abstract base classes have been moved in a new **`collections.abc`** module, to better differentiate between the abstract and the concrete collections classes. Aliases for ABCs are still present in the **`collections`** module to preserve existing imports. (bpo-11085 [https://bugs.python.org/issue?@action=redirect&bpo=11085])

The **`Counter`** class now supports the unary `+` and `-` operators, as well as the in-place operators `+=`, `-=`, `|=`, and `&=`. (Contributed by Raymond Hettinger in bpo-13121 [https://bugs.python.org/issue?@action=redirect&bpo=13121].)

contextlib

`ExitStack` now provides a solid foundation for programmatic manipulation of context managers and similar cleanup functionality. Unlike the previous `contextlib.nested` API (which was deprecated and removed), the new API is designed to work correctly regardless of whether context managers acquire their resources in their `__init__` method (for example, file objects)

or in their `__enter__` method (for example, synchronisation objects from the `threading` module).

(bpo-13585 [https://bugs.python.org/issue?@action=redirect&bpo=13585])

crypt

Addition of salt and modular crypt format (hashing method) and the `mksalt()` function to the `crypt` module.

(bpo-10924 [https://bugs.python.org/issue?@action=redirect&bpo=10924])

curses

- If the `curses` module is linked to the ncursesw library, use Unicode functions when Unicode strings or characters are passed (e.g. `waddwstr()`), and bytes functions otherwise (e.g. `waddstr()`).
- Use the locale encoding instead of `utf-8` to encode Unicode strings.
- `curses.window` has a new `curses.window.encoding` attribute.
- The `curses.window` class has a new `get_wch()` method to get a wide character
- The `curses` module has a new `unget_wch()` function to push a wide character so the next `get_wch()` will return it

(Contributed by Iñigo Serna in bpo-6755 [https://bugs.python.org/issue?@action=redirect&bpo=6755].)

datetime

- Equality comparisons between naive and aware `datetime` instances now return `False` instead of raising `TypeError` (bpo-15006 [https://bugs.python.org/issue?@action=redirect&bpo=15006]).
- New `datetime.datetime.timestamp()` method: Return POSIX timestamp corresponding to the `datetime` instance.
- The `datetime.datetime.strftime()` method supports formatting years older than 1000.

- The `datetime.datetime.astimezone()` method can now be called without arguments to convert datetime instance to the system timezone.

decimal

bpo-7652 [https://bugs.python.org/issue?@action=redirect&bpo=7652] - integrate fast native decimal arithmetic.
C-module and libmpdec written by Stefan Krah.

The new C version of the decimal module integrates the high speed libmpdec library for arbitrary precision correctly rounded decimal floating-point arithmetic. libmpdec conforms to IBM's General Decimal Arithmetic Specification.

Performance gains range from 10x for database applications to 100x for numerically intensive applications. These numbers are expected gains for standard precisions used in decimal floating-point arithmetic. Since the precision is user configurable, the exact figures may vary. For example, in integer bignum arithmetic the differences can be significantly higher.

The following table is meant as an illustration. Benchmarks are available at https://www.bytereef.org/mpdecimal/quickstart.html.

	decimal.py	**_decimal**	**speedup**
pi	42.02s	0.345s	120x
telco	172.19s	5.68s	30x
psycopg	3.57s	0.29s	12x

Recursos

- The `FloatOperation` signal optionally enables stricter semantics for mixing floats and Decimals.
- If Python is compiled without threads, the C version automatically disables the expensive thread local context machinery. In this case, the variable `HAVE_THREADS` is set to `False`.

Mudanças na API

- The C module has the following context limits, depending on the machine architecture:

	32 bits	64 bits
`MAX_PREC`	`425000000`	`999999999999999999`
`MAX_EMAX`	`425000000`	`999999999999999999`
`MIN_EMIN`	`-425000000`	`-999999999999999999`

- In the context templates (`DefaultContext`, `BasicContext` and `ExtendedContext`) the magnitude of `Emax` and `Emin` has changed to `999999`.

- The `Decimal` constructor in decimal.py does not observe the context limits and converts values with arbitrary exponents or precision exactly. Since the C version has internal limits, the following scheme is used: If possible, values are converted exactly, otherwise `InvalidOperation` is raised and the result is NaN. In the latter case it is always possible to use `create_decimal()` in order to obtain a rounded or inexact value.

- The power function in decimal.py is always correctly rounded. In the C version, it is defined in terms of the correctly rounded `exp()` and

`ln()` functions, but the final result is only “almost always correctly rounded”.

- In the C version, the context dictionary containing the signals is a `MutableMapping`. For speed reasons, `flags` and `traps` always refer to the same `MutableMapping` that the context was initialized with. If a new signal dictionary is assigned, `flags` and `traps` are updated with the new values, but they do not reference the RHS dictionary.

- Pickling a `Context` produces a different output in order to have a common interchange format for the Python and C versions.

- The order of arguments in the `Context` constructor has been changed to match the order displayed by `repr()`.

- The `watchexp` parameter in the `quantize()` method is deprecated.

email

Policy Framework

The email package now has a `policy` framework. A `Policy` is an object with several methods and properties that control how the email package behaves. The primary policy for Python 3.3 is the `Compat32` policy, which provides backward compatibility with the email package in Python 3.2. A `policy` can be specified when an email message is parsed by a `parser`, or when a `Message` object is created, or when an email is serialized using a `generator`. Unless overridden, a policy passed to a `parser` is inherited by all the `Message` object and sub-objects created by the `parser`. By default a `generator` will use the policy of the `Message` object it is serializing. The default policy is `compat32`.

The minimum set of controls implemented by all `policy` objects are:

max_line_length	The maximum length, excluding the linesep character(s), individual lines may have when a `Message`

	is serialized. Defaults to 78.
linesep	The character used to separate individual lines when a `Message` is serialized. Defaults to `\n`.
cte_type	`7bit` or `8bit`. `8bit` applies only to a `Bytes` `generator`, and means that non-ASCII may be used where allowed by the protocol (or where it exists in the original input).
raise_on_defect	Causes a `parser` to raise error when defects are encountered instead of adding them to the `Message` object's `defects` list.

A new policy instance, with new settings, is created using the **`clone()`** method of policy objects. `clone` takes any of the above controls as keyword arguments. Any control not specified in the call retains its default value. Thus you can create a policy that uses `\r\n` linesep characters like this:

```
mypolicy = compat32.clone(linesep='\r\n')
```

Policies can be used to make the generation of messages in the format needed by your application simpler. Instead of having to remember to specify `linesep='\r\n'` in all the places you call a `generator`, you can specify it once, when you set the policy used by the `parser` or the `Message`, whichever your program uses to create `Message` objects. On the other hand, if you need to generate messages in multiple forms, you can still specify the parameters in the appropriate `generator` call. Or you can have custom policy instances for your different cases, and pass those in when you create the `generator`.

Provisional Policy with New Header API

While the policy framework is worthwhile all by itself, the main motivation for introducing it is to allow the creation of new policies that implement

new features for the email package in a way that maintains backward compatibility for those who do not use the new policies. Because the new policies introduce a new API, we are releasing them in Python 3.3 as a provisional policy. Backwards incompatible changes (up to and including removal of the code) may occur if deemed necessary by the core developers.

The new policies are instances of **`EmailPolicy`**, and add the following additional controls:

refold_source	Controls whether or not headers parsed by a **`parser`** are refolded by the **`generator`**. It can be `none`, `long`, or `all`. The default is `long`, which means that source headers with a line longer than `max_line_length` get refolded. `none` means no line get refolded, and `all` means that all lines get refolded.
header_factory	A callable that take a `name` and `value` and produces a custom header object.

The `header_factory` is the key to the new features provided by the new policies. When one of the new policies is used, any header retrieved from a `Message` object is an object produced by the `header_factory`, and any time you set a header on a `Message` it becomes an object produced by `header_factory`. All such header objects have a `name` attribute equal to the header name. Address and Date headers have additional attributes that give you access to the parsed data of the header. This means you can now do things like this:

```
>>> m = Message(policy=SMTP)
>>> m['To'] = 'Éric <foo@example.com>'
>>> m['to']
'Éric <foo@example.com>'
>>> m['to'].addresses
(Address(display_name='Éric', username='foo', domain='example.cc
>>> m['to'].addresses[0].username
```

```
'foo'
>>> m['to'].addresses[0].display_name
'Éric'
>>> m['Date'] = email.utils.localtime()
>>> m['Date'].datetime
datetime.datetime(2012, 5, 25, 21, 39, 24, 465484, tzinfo=dateti
>>> m['Date']
'Fri, 25 May 2012 21:44:27 -0400'
>>> print(m)
To: =?utf-8?q?=C3=89ric?= <foo@example.com>
Date: Fri, 25 May 2012 21:44:27 -0400
```

You will note that the unicode display name is automatically encoded as `utf-8` when the message is serialized, but that when the header is accessed directly, you get the unicode version. This eliminates any need to deal with the **`email.header`** **`decode_header()`** or **`make_header()`** functions.

You can also create addresses from parts:

```
>>> m['cc'] = [Group('pals', [Address('Bob', 'bob', 'example.com
...                           Address('Sally', 'sally', 'example
...            Address('Bonzo', addr_spec='bonz@laugh.com')]
>>> print(m)
To: =?utf-8?q?=C3=89ric?= <foo@example.com>
Date: Fri, 25 May 2012 21:44:27 -0400
cc: pals: Bob <bob@example.com>, Sally <sally@example.com>;, Bor
```

Decoding to unicode is done automatically:

```
>>> m2 = message_from_string(str(m))
>>> m2['to']
'Éric <foo@example.com>'
```

When you parse a message, you can use the `addresses` and `groups` attributes of the header objects to access the groups and individual addresses:

```
>>> m2['cc'].addresses
(Address(display_name='Bob', username='bob', domain='example.com
```

```
>>> m2['cc'].groups
(Group(display_name='pals', addresses=(Address(display_name='Bob
```

In summary, if you use one of the new policies, header manipulation works the way it ought to: your application works with unicode strings, and the email package transparently encodes and decodes the unicode to and from the RFC standard Content Transfer Encodings.

Other API Changes

New **`BytesHeaderParser`**, added to the **`parser`** module to complement **`HeaderParser`** and complete the Bytes API.

New utility functions:

- **`format_datetime()`**: given a **`datetime`**, produce a string formatted for use in an email header.
- **`parsedate_to_datetime()`**: given a date string from an email header, convert it into an aware **`datetime`**, or a naive **`datetime`** if the offset is `-0000`.
- **`localtime()`**: With no argument, returns the current local time as an aware **`datetime`** using the local **`timezone`**. Given an aware **`datetime`**, converts it into an aware **`datetime`** using the local **`timezone`**.

ftplib

- **`ftplib.FTP`** now accepts a `source_address` keyword argument to specify the `(host, port)` to use as the source address in the bind call when creating the outgoing socket. (Contributed by Giampaolo Rodolà in bpo-8594 [https://bugs.python.org/issue?@action=redirect&bpo=8594].)
- The **`FTP_TLS`** class now provides a new **`ccc()`** function to revert control channel back to plaintext. This can be useful to take advantage of firewalls that know how to handle NAT with non-secure FTP without opening fixed ports. (Contributed by Giampaolo Rodolà in bpo-12139 [https://bugs.python.org/issue?@action=redirect&bpo=12139].)
- Added **`ftplib.FTP.mlsd()`** method which provides a parsable directory listing format and deprecates **`ftplib.FTP.nlst()`** and

`ftplib.FTP.dir()`. (Contributed by Giampaolo Rodolà in bpo-11072 [https://bugs.python.org/issue?@action=redirect&bpo=11072].)

functools

The `@functools.lru_cache` decorator now accepts a `typed` keyword argument (that defaults to `False` to ensure that it caches values of different types that compare equal in separate cache slots. (Contributed by Raymond Hettinger in bpo-13227 [https://bugs.python.org/issue?@action=redirect&bpo=13227].)

gc

It is now possible to register callbacks invoked by the garbage collector before and after collection using the new `callbacks` list.

hmac

A new `compare_digest()` function has been added to prevent side channel attacks on digests through timing analysis. (Contributed by Nick Coghlan and Christian Heimes in bpo-15061 [https://bugs.python.org/issue?@action=redirect&bpo=15061].)

http

`http.server.BaseHTTPRequestHandler` now buffers the headers and writes them all at once when `end_headers()` is called. A new method `flush_headers()` can be used to directly manage when the accumulated headers are sent. (Contributed by Andrew Schaaf in bpo-3709 [https://bugs.python.org/issue?@action=redirect&bpo=3709].)

`http.server` now produces valid `HTML 4.01 strict` output. (Contributed by Ezio Melotti in bpo-13295 [https://bugs.python.org/issue?@action=redirect&bpo=13295].)

`http.client.HTTPResponse` now has a `readinto()` method, which means it can be used as an `io.RawIOBase` class. (Contributed by John Kuhn in bpo-13464 [https://bugs.python.org/issue?@action=redirect&bpo=13464].)

html

`html.parser.HTMLParser` is now able to parse broken markup without raising errors, therefore the *strict* argument of the constructor and the `HTMLParseError` exception are now deprecated. The ability to parse broken markup is the result of a number of bug fixes that are also available on the latest bug fix releases of Python 2.7/3.2. (Contributed by Ezio Melotti in bpo-15114 [https://bugs.python.org/issue?@action=redirect&bpo=15114], and bpo-14538 [https://bugs.python.org/issue?@action=redirect&bpo=14538], bpo-13993 [https://bugs.python.org/issue?@action=redirect&bpo=13993], bpo-13960 [https://bugs.python.org/issue?@action=redirect&bpo=13960], bpo-13358 [https://bugs.python.org/issue?@action=redirect&bpo=13358], bpo-1745761 [https://bugs.python.org/issue?@action=redirect&bpo=1745761], bpo-755670 [https://bugs.python.org/issue?@action=redirect&bpo=755670], bpo-13357 [https://bugs.python.org/issue?@action=redirect&bpo=13357], bpo-12629 [https://bugs.python.org/issue?@action=redirect&bpo=12629], bpo-1200313 [https://bugs.python.org/issue?@action=redirect&bpo=1200313], bpo-670664 [https://bugs.python.org/issue?@action=redirect&bpo=670664], bpo-13273 [https://bugs.python.org/issue?@action=redirect&bpo=13273], bpo-12888 [https://bugs.python.org/issue?@action=redirect&bpo=12888], bpo-7311 [https://bugs.python.org/issue?@action=redirect&bpo=7311].)

A new `html5` dictionary that maps HTML5 named character references to the equivalent Unicode character(s) (e.g. `html5['gt;'] == '>'`) has been added to the `html.entities` module. The dictionary is now also used by `HTMLParser`. (Contributed by Ezio Melotti in bpo-11113 [https://bugs.python.org/issue?@action=redirect&bpo=11113] and bpo-15156 [https://bugs.python.org/issue?@action=redirect&bpo=15156].)

imaplib

The `IMAP4_SSL` constructor now accepts an SSLContext parameter to control parameters of the secure channel.

(Contributed by Sijin Joseph in bpo-8808 [https://bugs.python.org/issue?@action=redirect&bpo=8808].)

inspect

A new `getclosurevars()` function has been added. This function reports the current binding of all names referenced from the function body and where those names were resolved, making it easier to verify correct internal state when testing code that relies on stateful closures.

(Contributed by Meador Inge and Nick Coghlan in bpo-13062 [https://bugs.python.org/issue?@action=redirect&bpo=13062].)

A new `getgeneratorlocals()` function has been added. This function reports the current binding of local variables in the generator's stack frame, making it easier to verify correct internal state when testing generators.

(Contributed by Meador Inge in bpo-15153 [https://bugs.python.org/issue?@action=redirect&bpo=15153].)

io

The `open()` function has a new `'x'` mode that can be used to exclusively create a new file, and raise a `FileExistsError` if the file already exists. It is based on the C11 'x' mode to fopen().

(Contributed by David Townshend in bpo-12760 [https://bugs.python.org/issue?@action=redirect&bpo=12760].)

The constructor of the `TextIOWrapper` class has a new *write_through* optional argument. If *write_through* is `True`, calls to `write()` are guaranteed not to be buffered: any data written on the `TextIOWrapper` object is immediately handled to its underlying binary buffer.

itertools

`accumulate()` now takes an optional `func` argument for providing a user-supplied binary function.

logging

The **basicConfig()** function now supports an optional `handlers` argument taking an iterable of handlers to be added to the root logger.

A class level attribute **append_nul** has been added to **SysLogHandler** to allow control of the appending of the `NUL` (`\000`) byte to syslog records, since for some daemons it is required while for others it is passed through to the log.

math

The **math** module has a new function, **log2()**, which returns the base-2 logarithm of *x*.

(Written by Mark Dickinson in bpo-11888 [https://bugs.python.org/issue?@action=redirect&bpo=11888].)

mmap

The **read()** method is now more compatible with other file-like objects: if the argument is omitted or specified as `None`, it returns the bytes from the current file position to the end of the mapping. (Contributed by Petri Lehtinen in bpo-12021 [https://bugs.python.org/issue?@action=redirect&bpo=12021].)

multiprocessing

The new **multiprocessing.connection.wait()** function allows polling multiple objects (such as connections, sockets and pipes) with a timeout. (Contributed by Richard Oudkerk in bpo-12328 [https://bugs.python.org/issue?@action=redirect&bpo=12328].)

multiprocessing.connection.Connection objects can now be transferred over multiprocessing connections. (Contributed by Richard Oudkerk in bpo-4892 [https://bugs.python.org/issue?@action=redirect&bpo=4892].)

multiprocessing.Process now accepts a `daemon` keyword argument to override the default behavior of inheriting the `daemon` flag from the parent process (bpo-6064 [https://bugs.python.org/issue?@action=redirect&bpo=6064]).

New attribute `multiprocessing.Process.sentinel` allows a program to wait on multiple `Process` objects at one time using the appropriate OS primitives (for example, `select` on posix systems).

New methods `multiprocessing.pool.Pool.starmap()` and `starmap_async()` provide `itertools.starmap()` equivalents to the existing `multiprocessing.pool.Pool.map()` and `map_async()` functions. (Contributed by Hynek Schlawack in bpo-12708 [https://bugs.python.org/issue?@action=redirect&bpo=12708].)

nntplib

The `nntplib.NNTP` class now supports the context management protocol to unconditionally consume `socket.error` exceptions and to close the NNTP connection when done:

```
>>> from nntplib import NNTP
>>> with NNTP('news.gmane.org') as n:
...     n.group('gmane.comp.python.committers')
...
('211 1755 1 1755 gmane.comp.python.committers', 1755, 1, 1755,
>>>
```

(Contributed by Giampaolo Rodolà in bpo-9795 [https://bugs.python.org/issue?@action=redirect&bpo=9795].)

os

- The `os` module has a new `pipe2()` function that makes it possible to create a pipe with `O_CLOEXEC` or `O_NONBLOCK` flags set atomically. This is especially useful to avoid race conditions in multi-threaded programs.

- The `os` module has a new `sendfile()` function which provides an efficient “zero-copy” way for copying data from one file (or socket) descriptor to another. The phrase “zero-copy” refers to the fact that all of the copying of data between the two descriptors is done entirely by

the kernel, with no copying of data into userspace buffers. **`sendfile()`** can be used to efficiently copy data from a file on disk to a network socket, e.g. for downloading a file.

(Patch submitted by Ross Lagerwall and Giampaolo Rodolà in bpo-10882 [https://bugs.python.org/issue?@action=redirect&bpo=10882].)

- To avoid race conditions like symlink attacks and issues with temporary files and directories, it is more reliable (and also faster) to manipulate file descriptors instead of file names. Python 3.3 enhances existing functions and introduces new functions to work on file descriptors (bpo-4761 [https://bugs.python.org/issue?@action=redirect&bpo=4761], bpo-10755 [https://bugs.python.org/issue?@action=redirect&bpo=10755] and bpo-14626 [https://bugs.python.org/issue?@action=redirect&bpo=14626]).
    - The **`os`** module has a new **`fwalk()`** function similar to **`walk()`** except that it also yields file descriptors referring to the directories visited. This is especially useful to avoid symlink races.
    - The following functions get new optional *dir_fd* (paths relative to directory descriptors) and/or *follow_symlinks* (not following symlinks): **`access()`**, **`chflags()`**, **`chmod()`**, **`chown()`**, **`link()`**, **`lstat()`**, **`mkdir()`**, **`mkfifo()`**, **`mknod()`**, **`open()`**, **`readlink()`**, **`remove()`**, **`rename()`**, **`replace()`**, **`rmdir()`**, **`stat()`**, **`symlink()`**, **`unlink()`**, **`utime()`**. Platform support for using these parameters can be checked via the sets **`os.supports_dir_fd`** and **`os.supports_follow_symlinks`**.
    - The following functions now support a file descriptor for their path argument: **`chdir()`**, **`chmod()`**, **`chown()`**, **`execve()`**, **`listdir()`**, **`pathconf()`**, **`exists()`**, **`stat()`**, **`statvfs()`**, **`utime()`**. Platform support for this can be checked via the **`os.supports_fd`** set.

- **`access()`** accepts an `effective_ids` keyword argument to turn on using the effective uid/gid rather than the real uid/gid in the access check. Platform support for this can be checked via the **`supports_effective_ids`** set.

- The `os` module has two new functions: `getpriority()` and `setpriority()`. They can be used to get or set process niceness/priority in a fashion similar to `os.nice()` but extended to all processes instead of just the current one.

  (Patch submitted by Giampaolo Rodolà in bpo-10784 [https://bugs.python.org/issue?@action=redirect&bpo=10784].)

- The new `os.replace()` function allows cross-platform renaming of a file with overwriting the destination. With `os.rename()`, an existing destination file is overwritten under POSIX, but raises an error under Windows. (Contributed by Antoine Pitrou in bpo-8828 [https://bugs.python.org/issue?@action=redirect&bpo=8828].)

- The stat family of functions (`stat()`, `fstat()`, and `lstat()`) now support reading a file's timestamps with nanosecond precision. Symmetrically, `utime()` can now write file timestamps with nanosecond precision. (Contributed by Larry Hastings in bpo-14127 [https://bugs.python.org/issue?@action=redirect&bpo=14127].)

- The new `os.get_terminal_size()` function queries the size of the terminal attached to a file descriptor. See also `shutil.get_terminal_size()`. (Contributed by Zbigniew Jędrzejewski-Szmek in bpo-13609 [https://bugs.python.org/issue?@action=redirect&bpo=13609].)

- New functions to support Linux extended attributes (bpo-12720 [https://bugs.python.org/issue?@action=redirect&bpo=12720]): `getxattr()`, `listxattr()`, `removexattr()`, `setxattr()`.
- New interface to the scheduler. These functions control how a process is allocated CPU time by the operating system. New functions: `sched_get_priority_max()`, `sched_get_priority_min()`, `sched_getaffinity()`, `sched_getparam()`, `sched_getscheduler()`, `sched_rr_get_interval()`, `sched_setaffinity()`, `sched_setparam()`, `sched_setscheduler()`, `sched_yield()`,
- New functions to control the file system:
  - `posix_fadvise()`: Announces an intention to access data in a specific pattern thus allowing the kernel to make optimizations.

    - `posix_fallocate()`: Ensures that enough disk space is allocated for a file.
    - `sync()`: Force write of everything to disk.
- Additional new posix functions:
    - `lockf()`: Apply, test or remove a POSIX lock on an open file descriptor.
    - `pread()`: Read from a file descriptor at an offset, the file offset remains unchanged.
    - `pwrite()`: Write to a file descriptor from an offset, leaving the file offset unchanged.
    - `readv()`: Read from a file descriptor into a number of writable buffers.
    - `truncate()`: Truncate the file corresponding to *path*, so that it is at most *length* bytes in size.
    - `waitid()`: Wait for the completion of one or more child processes.
    - `writev()`: Write the contents of *buffers* to a file descriptor, where *buffers* is an arbitrary sequence of buffers.
    - `getgrouplist()` (bpo-9344 [https://bugs.python.org/issue?@action=redirect&bpo=9344]): Return list of group ids that specified user belongs to.
- `times()` and `uname()`: Return type changed from a tuple to a tuple-like object with named attributes.
- Some platforms now support additional constants for the `lseek()` function, such as `os.SEEK_HOLE` and `os.SEEK_DATA`.
- New constants `RTLD_LAZY`, `RTLD_NOW`, `RTLD_GLOBAL`, `RTLD_LOCAL`, `RTLD_NODELETE`, `RTLD_NOLOAD`, and `RTLD_DEEPBIND` are available on platforms that support them. These are for use with the `sys.setdlopenflags()` function, and supersede the similar constants defined in `ctypes` and `DLFCN`. (Contributed by Victor Stinner in bpo-13226 [https://bugs.python.org/issue?@action=redirect&bpo=13226].)
- `os.symlink()` now accepts (and ignores) the `target_is_directory` keyword argument on non-Windows platforms, to ease cross-platform support.

pdb

Tab-completion is now available not only for command names, but also their arguments. For example, for the `break` command, function and file names are completed.

(Contributed by Georg Brandl in bpo-14210 [https://bugs.python.org/issue?@action=redirect&bpo=14210])

pickle

`pickle.Pickler` objects now have an optional **`dispatch_table`** attribute allowing per-pickler reduction functions to be set.

(Contributed by Richard Oudkerk in bpo-14166 [https://bugs.python.org/issue?@action=redirect&bpo=14166].)

pydoc

The Tk GUI and the **`serve()`** function have been removed from the **`pydoc`** module: `pydoc -g` and **`serve()`** have been deprecated in Python 3.2.

re

`str` regular expressions now support `\u` and `\U` escapes.

(Contributed by Serhiy Storchaka in bpo-3665 [https://bugs.python.org/issue?@action=redirect&bpo=3665].)

sched

- **`run()`** now accepts a *blocking* parameter which when set to false makes the method execute the scheduled events due to expire soonest (if any) and then return immediately. This is useful in case you want to use the **`scheduler`** in non-blocking applications. (Contributed by Giampaolo Rodolà in bpo-13449 [https://bugs.python.org/issue?@action=redirect&bpo=13449].)
- **`scheduler`** class can now be safely used in multi-threaded environments. (Contributed by Josiah Carlson and Giampaolo Rodolà

in bpo-8684 [https://bugs.python.org/issue?@action=redirect&bpo=8684].)
- *timefunc* and *delayfunct* parameters of `scheduler` class constructor are now optional and defaults to `time.time()` and `time.sleep()` respectively. (Contributed by Chris Clark in bpo-13245 [https://bugs.python.org/issue?@action=redirect&bpo=13245].)
- `enter()` and `enterabs()` *argument* parameter is now optional. (Contributed by Chris Clark in bpo-13245 [https://bugs.python.org/issue?@action=redirect&bpo=13245].)
- `enter()` and `enterabs()` now accept a *kwargs* parameter. (Contributed by Chris Clark in bpo-13245 [https://bugs.python.org/issue?@action=redirect&bpo=13245].)

select

Solaris and derivative platforms have a new class `select.devpoll` for high performance asynchronous sockets via `/dev/poll`. (Contributed by Jesús Cea Avión in bpo-6397 [https://bugs.python.org/issue?@action=redirect&bpo=6397].)

shlex

The previously undocumented helper function `quote` from the `pipes` modules has been moved to the `shlex` module and documented. `quote()` properly escapes all characters in a string that might be otherwise given special meaning by the shell.

shutil

- Novas Funções:
  - `disk_usage()`: provides total, used and free disk space statistics. (Contributed by Giampaolo Rodolà in bpo-12442 [https://bugs.python.org/issue?@action=redirect&bpo=12442].)
  - `chown()`: allows one to change user and/or group of the given path also specifying the user/group names and not only their numeric ids. (Contributed by Sandro Tosi in bpo-12191 [https://bugs.python.org/issue?@action=redirect&bpo=12191].)
  - `shutil.get_terminal_size()`: returns the size of the terminal window to which the interpreter is attached. (Contributed by

Zbigniew Jędrzejewski-Szmek in bpo-13609 [https://bugs.python.org/issue?@action=redirect&bpo=13609].)

- **copy2()** and **copystat()** now preserve file timestamps with nanosecond precision on platforms that support it. They also preserve file "extended attributes" on Linux. (Contributed by Larry Hastings in bpo-14127 [https://bugs.python.org/issue?@action=redirect&bpo=14127] and bpo-15238 [https://bugs.python.org/issue?@action=redirect&bpo=15238].)
- Several functions now take an optional `symlinks` argument: when that parameter is true, symlinks aren't dereferenced and the operation instead acts on the symlink itself (or creates one, if relevant). (Contributed by Hynek Schlawack in bpo-12715 [https://bugs.python.org/issue?@action=redirect&bpo=12715].)
- When copying files to a different file system, **move()** now handles symlinks the way the posix `mv` command does, recreating the symlink rather than copying the target file contents. (Contributed by Jonathan Niehof in bpo-9993 [https://bugs.python.org/issue?@action=redirect&bpo=9993].) **move()** now also returns the `dst` argument as its result.
- **rmtree()** is now resistant to symlink attacks on platforms which support the new `dir_fd` parameter in **os.open()** and **os.unlink()**. (Contributed by Martin von Löwis and Hynek Schlawack in bpo-4489 [https://bugs.python.org/issue?@action=redirect&bpo=4489].)

signal

- The **signal** module has new functions:
  - **pthread_sigmask()**: fetch and/or change the signal mask of the calling thread (Contributed by Jean-Paul Calderone in bpo-8407 [https://bugs.python.org/issue?@action=redirect&bpo=8407]);
  - **pthread_kill()**: send a signal to a thread;
  - **sigpending()**: examine pending functions;
  - **sigwait()**: wait a signal;
  - **sigwaitinfo()**: wait for a signal, returning detailed information about it;
  - **sigtimedwait()**: like **sigwaitinfo()** but with a timeout.
- The signal handler writes the signal number as a single byte instead of a nul byte into the wakeup file descriptor. So it is possible to wait more than one signal and know which signals were raised.

- `signal.signal()` and `signal.siginterrupt()` raise an OSError, instead of a RuntimeError: OSError has an errno attribute.

smtpd

The `smtpd` module now supports **RFC 5321** [https://datatracker.ietf.org/doc/html/rfc5321.html] (extended SMTP) and **RFC 1870** [https://datatracker.ietf.org/doc/html/rfc1870.html] (size extension). Per the standard, these extensions are enabled if and only if the client initiates the session with an `EHLO` command.

(Initial `ELHO` support by Alberto Trevino. Size extension by Juhana Jauhiainen. Substantial additional work on the patch contributed by Michele Orrù and Dan Boswell. bpo-8739 [https://bugs.python.org/issue?@action=redirect&bpo=8739])

smtplib

The `SMTP`, `SMTP_SSL`, and `LMTP` classes now accept a `source_address` keyword argument to specify the `(host, port)` to use as the source address in the bind call when creating the outgoing socket. (Contributed by Paulo Scardine in bpo-11281 [https://bugs.python.org/issue?@action=redirect&bpo=11281].)

`SMTP` now supports the context management protocol, allowing an `SMTP` instance to be used in a `with` statement. (Contributed by Giampaolo Rodolà in bpo-11289 [https://bugs.python.org/issue?@action=redirect&bpo=11289].)

The `SMTP_SSL` constructor and the `starttls()` method now accept an SSLContext parameter to control parameters of the secure channel. (Contributed by Kasun Herath in bpo-8809 [https://bugs.python.org/issue?@action=redirect&bpo=8809].)

socket

- The `socket` class now exposes additional methods to process ancillary data when supported by the underlying platform:

- `sendmsg()`
- `recvmsg()`
- `recvmsg_into()`

(Contributed by David Watson in bpo-6560 [https://bugs.python.org/issue?@action=redirect&bpo=6560], based on an earlier patch by Heiko Wundram)

- The `socket` class now supports the PF_CAN protocol family (https://en.wikipedia.org/wiki/Socketcan), on Linux (https://lwn.net/Articles/253425).

  (Contributed by Matthias Fuchs, updated by Tiago Gonçalves in bpo-10141 [https://bugs.python.org/issue?@action=redirect&bpo=10141].)

- The `socket` class now supports the PF_RDS protocol family (https://en.wikipedia.org/wiki/Reliable_Datagram_Sockets and https://oss.oracle.com/projects/rds [https://web.archive.org/web/20130115155505/https://oss.oracle.com/projects/rds/]).

- The `socket` class now supports the `PF_SYSTEM` protocol family on OS X. (Contributed by Michael Goderbauer in bpo-13777 [https://bugs.python.org/issue?@action=redirect&bpo=13777].)

- New function `sethostname()` allows the hostname to be set on Unix systems if the calling process has sufficient privileges. (Contributed by Ross Lagerwall in bpo-10866 [https://bugs.python.org/issue?@action=redirect&bpo=10866].)

socketserver

`BaseServer` now has an overridable method `service_actions()` that is called by the `serve_forever()` method in the service loop. `ForkingMixIn` now uses this to clean up zombie child processes. (Contributed by Justin Warkentin in bpo-11109 [https://bugs.python.org/issue?@action=redirect&bpo=11109].)

sqlite3

New `sqlite3.Connection` method `set_trace_callback()` can be used to capture a trace of all sql commands processed by sqlite. (Contributed by Torsten Landschoff in bpo-11688 [https://bugs.python.org/issue?@action=redirect&bpo=11688].)

ssl

- The `ssl` module has two new random generation functions:

  - `RAND_bytes()`: generate cryptographically strong pseudo-random bytes.
  - `RAND_pseudo_bytes()`: generate pseudo-random bytes.

  (Contributed by Victor Stinner in bpo-12049 [https://bugs.python.org/issue?@action=redirect&bpo=12049].)

- The `ssl` module now exposes a finer-grained exception hierarchy in order to make it easier to inspect the various kinds of errors. (Contributed by Antoine Pitrou in bpo-11183 [https://bugs.python.org/issue?@action=redirect&bpo=11183].)

- `load_cert_chain()` now accepts a *password* argument to be used if the private key is encrypted. (Contributed by Adam Simpkins in bpo-12803 [https://bugs.python.org/issue?@action=redirect&bpo=12803].)

- Diffie-Hellman key exchange, both regular and Elliptic Curve-based, is now supported through the `load_dh_params()` and `set_ecdh_curve()` methods. (Contributed by Antoine Pitrou in bpo-13626 [https://bugs.python.org/issue?@action=redirect&bpo=13626] and bpo-13627 [https://bugs.python.org/issue?@action=redirect&bpo=13627].)

- SSL sockets have a new `get_channel_binding()` method allowing the implementation of certain authentication mechanisms such as SCRAM-SHA-1-PLUS. (Contributed by Jacek Konieczny in bpo-12551 [https://bugs.python.org/issue?@action=redirect&bpo=12551].)

- You can query the SSL compression algorithm used by an SSL socket, thanks to its new `compression()` method. The new attribute

`OP_NO_COMPRESSION` can be used to disable compression. (Contributed by Antoine Pitrou in bpo-13634 [https://bugs.python.org/issue?@action=redirect&bpo=13634].)

- Support has been added for the Next Protocol Negotiation extension using the `ssl.SSLContext.set_npn_protocols()` method. (Contributed by Colin Marc in bpo-14204 [https://bugs.python.org/issue?@action=redirect&bpo=14204].)

- SSL errors can now be introspected more easily thanks to `library` and `reason` attributes. (Contributed by Antoine Pitrou in bpo-14837 [https://bugs.python.org/issue?@action=redirect&bpo=14837].)

- The `get_server_certificate()` function now supports IPv6. (Contributed by Charles-François Natali in bpo-11811 [https://bugs.python.org/issue?@action=redirect&bpo=11811].)

- New attribute `OP_CIPHER_SERVER_PREFERENCE` allows setting SSLv3 server sockets to use the server's cipher ordering preference rather than the client's (bpo-13635 [https://bugs.python.org/issue?@action=redirect&bpo=13635]).

stat

The undocumented tarfile.filemode function has been moved to `stat.filemode()`. It can be used to convert a file's mode to a string of the form '-rwxrwxrwx'.

(Contributed by Giampaolo Rodolà in bpo-14807 [https://bugs.python.org/issue?@action=redirect&bpo=14807].)

struct

The `struct` module now supports `ssize_t` and `size_t` via the new codes `n` and `N`, respectively. (Contributed by Antoine Pitrou in bpo-3163 [https://bugs.python.org/issue?@action=redirect&bpo=3163].)

subprocess

Command strings can now be bytes objects on posix platforms. (Contributed by Victor Stinner in bpo-8513 [https://bugs.python.org/issue?@action=redirect&bpo=8513].)

A new constant `DEVNULL` allows suppressing output in a platform-independent fashion. (Contributed by Ross Lagerwall in bpo-5870 [https://bugs.python.org/issue?@action=redirect&bpo=5870].)

sys

The `sys` module has a new `thread_info` named tuple holding information about the thread implementation (bpo-11223 [https://bugs.python.org/issue?@action=redirect&bpo=11223]).

tarfile

`tarfile` now supports `lzma` encoding via the `lzma` module. (Contributed by Lars Gustäbel in bpo-5689 [https://bugs.python.org/issue?@action=redirect&bpo=5689].)

tempfile

`tempfile.SpooledTemporaryFile`'s `truncate()` method now accepts a `size` parameter. (Contributed by Ryan Kelly in bpo-9957 [https://bugs.python.org/issue?@action=redirect&bpo=9957].)

textwrap

The `textwrap` module has a new `indent()` that makes it straightforward to add a common prefix to selected lines in a block of text (bpo-13857 [https://bugs.python.org/issue?@action=redirect&bpo=13857]).

threading

`threading.Condition`, `threading.Semaphore`, `threading.BoundedSemaphore`, `threading.Event`, and `threading.Timer`, all of which used to be factory functions returning a class instance, are now classes and may be subclassed. (Contributed by Éric Araujo in bpo-10968 [https://bugs.python.org/issue?@action=redirect&bpo=10968].)

The `threading.Thread` constructor now accepts a `daemon` keyword argument to override the default behavior of inheriting the `daemon` flag value from the parent thread (bpo-6064 [https://bugs.python.org/issue?@action=redirect&bpo=6064]).

The formerly private function `_thread.get_ident` is now available as the public function `threading.get_ident()`. This eliminates several cases of direct access to the `_thread` module in the stdlib. Third party code that used `_thread.get_ident` should likewise be changed to use the new public interface.

time

The **PEP 418** [https://peps.python.org/pep-0418/] added new functions to the `time` module:

- `get_clock_info()`: Get information on a clock.
- `monotonic()`: Monotonic clock (cannot go backward), not affected by system clock updates.
- `perf_counter()`: Performance counter with the highest available resolution to measure a short duration.
- `process_time()`: Sum of the system and user CPU time of the current process.

Other new functions:

- `clock_getres()`, `clock_gettime()` and `clock_settime()` functions with `CLOCK_`*`xxx`* constants. (Contributed by Victor Stinner in bpo-10278 [https://bugs.python.org/issue?@action=redirect&bpo=10278].)

To improve cross platform consistency, `sleep()` now raises a `ValueError` when passed a negative sleep value. Previously this was an error on posix,

but produced an infinite sleep on Windows.

types

Add a new `types.MappingProxyType` class: Read-only proxy of a mapping. (bpo-14386 [https://bugs.python.org/issue?@action=redirect&bpo=14386])

The new functions `types.new_class()` and `types.prepare_class()` provide support for **PEP 3115** [https://peps.python.org/pep-3115/] compliant dynamic type creation. (bpo-14588 [https://bugs.python.org/issue?@action=redirect&bpo=14588])

unittest

`assertRaises()`, `assertRaisesRegex()`, `assertWarns()`, and `assertWarnsRegex()` now accept a keyword argument *msg* when used as context managers. (Contributed by Ezio Melotti and Winston Ewert in bpo-10775 [https://bugs.python.org/issue?@action=redirect&bpo=10775].)

`unittest.TestCase.run()` now returns the `TestResult` object.

urllib

The `Request` class, now accepts a *method* argument used by `get_method()` to determine what HTTP method should be used. For example, this will send a `'HEAD'` request:

```
>>> urlopen(Request('https://www.python.org', method='HEAD'))
```

(bpo-1673007 [https://bugs.python.org/issue?@action=redirect&bpo=1673007])

webbrowser

The `webbrowser` module supports more “browsers”: Google Chrome (named **chrome**, **chromium**, **chrome-browser** or **chromium-browser** depending on the version and operating system), and the generic launchers **xdg-open**, from the FreeDesktop.org project, and **gvfs-open**, which is the

default URI handler for GNOME 3. (The former contributed by Arnaud Calmettes in bpo-13620 [https://bugs.python.org/issue?@action=redirect&bpo=13620], the latter by Matthias Klose in bpo-14493 [https://bugs.python.org/issue?@action=redirect&bpo=14493].)

xml.etree.ElementTree

The `xml.etree.ElementTree` module now imports its C accelerator by default; there is no longer a need to explicitly import `xml.etree.cElementTree` (this module stays for backwards compatibility, but is now deprecated). In addition, the `iter` family of methods of `Element` has been optimized (rewritten in C). The module's documentation has also been greatly improved with added examples and a more detailed reference.

zlib

New attribute `zlib.Decompress.eof` makes it possible to distinguish between a properly formed compressed stream and an incomplete or truncated one. (Contributed by Nadeem Vawda in bpo-12646 [https://bugs.python.org/issue?@action=redirect&bpo=12646].)

New attribute `zlib.ZLIB_RUNTIME_VERSION` reports the version string of the underlying `zlib` library that is loaded at runtime. (Contributed by Torsten Landschoff in bpo-12306 [https://bugs.python.org/issue?@action=redirect&bpo=12306].)

Otimizações

Major performance enhancements have been added:

- Thanks to **PEP 393** [https://peps.python.org/pep-0393/], some operations on Unicode strings have been optimized:
    - the memory footprint is divided by 2 to 4 depending on the text
    - encode an ASCII string to UTF-8 doesn't need to encode characters anymore, the UTF-8 representation is shared with the ASCII representation
    - the UTF-8 encoder has been optimized

    - repeating a single ASCII letter and getting a substring of an ASCII string is 4 times faster

- UTF-8 is now 2x to 4x faster. UTF-16 encoding is now up to 10x faster.

  (Contributed by Serhiy Storchaka, bpo-14624 [https://bugs.python.org/issue?@action=redirect&bpo=14624], bpo-14738 [https://bugs.python.org/issue?@action=redirect&bpo=14738] and bpo-15026 [https://bugs.python.org/issue?@action=redirect&bpo=15026].)

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- New **PEP 3118** [https://peps.python.org/pep-3118/] related function:
  - `PyMemoryView_FromMemory()`
- **PEP 393** [https://peps.python.org/pep-0393/] added new Unicode types, macros and functions:
  - High-level API:
    - `PyUnicode_CopyCharacters()`
    - `PyUnicode_FindChar()`
    - `PyUnicode_GetLength()`, `PyUnicode_GET_LENGTH`
    - `PyUnicode_New()`
    - `PyUnicode_Substring()`
    - `PyUnicode_ReadChar()`, `PyUnicode_WriteChar()`
  - Low-level API:
    - `Py_UCS1`, `Py_UCS2`, `Py_UCS4` types
    - `PyASCIIObject` and `PyCompactUnicodeObject` structures
    - `PyUnicode_READY`
    - `PyUnicode_FromKindAndData()`
    - `PyUnicode_AsUCS4()`, `PyUnicode_AsUCS4Copy()`
    - `PyUnicode_DATA`, `PyUnicode_1BYTE_DATA`, `PyUnicode_2BYTE_DATA`, `PyUnicode_4BYTE_DATA`
    - `PyUnicode_KIND` with `PyUnicode_Kind` enum: `PyUnicode_WCHAR_KIND`, `PyUnicode_1BYTE_KIND`, `PyUnicode_2BYTE_KIND`, `PyUnicode_4BYTE_KIND`

    - `PyUnicode_READ`, `PyUnicode_READ_CHAR`, `PyUnicode_WRITE`
    - `PyUnicode_MAX_CHAR_VALUE`
- `PyArg_ParseTuple` now accepts a `bytearray` for the `c` format (bpo-12380 [https://bugs.python.org/issue?@action=redirect&bpo=12380]).

Descontinuados

Unsupported Operating Systems

OS/2 and VMS are no longer supported due to the lack of a maintainer.

Windows 2000 and Windows platforms which set `COMSPEC` to `command.com` are no longer supported due to maintenance burden.

OSF support, which was deprecated in 3.2, has been completely removed.

Deprecated Python modules, functions and methods

- Passing a non-empty string to `object.__format__()` is deprecated, and will produce a `TypeError` in Python 3.4 (bpo-9856 [https://bugs.python.org/issue?@action=redirect&bpo=9856]).
- The `unicode_internal` codec has been deprecated because of the **PEP 393** [https://peps.python.org/pep-0393/], use UTF-8, UTF-16 (`utf-16-le` or `utf-16-be`), or UTF-32 (`utf-32-le` or `utf-32-be`)
- `ftplib.FTP.nlst()` and `ftplib.FTP.dir()`: use `ftplib.FTP.mlsd()`
- **`platform.popen()`**: use the `subprocess` module. Check especially the Replacing Older Functions with the subprocess Module section (bpo-11377 [https://bugs.python.org/issue?@action=redirect&bpo=11377]).
- bpo-13374 [https://bugs.python.org/issue?@action=redirect&bpo=13374]: The Windows bytes API has been deprecated in the `os` module. Use Unicode filenames, instead of bytes filenames, to not depend on the ANSI code page anymore and to support any filename.
- bpo-13988 [https://bugs.python.org/issue?@action=redirect&bpo=13988]: The **`xml.etree.cElementTree`** module is deprecated. The accelerator is used automatically whenever available.
- The behaviour of **`time.clock()`** depends on the platform: use the new `time.perf_counter()` or `time.process_time()` function instead,

depending on your requirements, to have a well defined behaviour.
- The **os.stat_float_times()** function is deprecated.
- abc module:
  - @abc.abstractproperty has been deprecated, use @property with @abc.abstractmethod instead.
  - @abc.abstractclassmethod has been deprecated, use @classmethod with @abc.abstractmethod instead.
  - @abc.abstractstaticmethod has been deprecated, use @staticmethod with @abc.abstractmethod instead.
- importlib package:
  - importlib.abc.SourceLoader.path_mtime() is now deprecated in favour of importlib.abc.SourceLoader.path_stats() as bytecode files now store both the modification time and size of the source file the bytecode file was compiled from.

Deprecated functions and types of the C API

The Py_UNICODE has been deprecated by **PEP 393** [https://peps.python.org/pep-0393/] and will be removed in Python 4. All functions using this type are deprecated:

Unicode functions and methods using Py_UNICODE and Py_UNICODE* types:

- **PyUnicode_FromUnicode**: use PyUnicode_FromWideChar() or PyUnicode_FromKindAndData()
- **PyUnicode_AS_UNICODE**, **PyUnicode_AsUnicode()**, **PyUnicode_AsUnicodeAndSize()**: use PyUnicode_AsWideCharString()
- **PyUnicode_AS_DATA**: use PyUnicode_DATA with PyUnicode_READ and PyUnicode_WRITE
- **PyUnicode_GET_SIZE**, **PyUnicode_GetSize()**: use PyUnicode_GET_LENGTH or PyUnicode_GetLength()
- **PyUnicode_GET_DATA_SIZE**: use `PyUnicode_GET_LENGTH(str) * PyUnicode_KIND(str)` (only work on ready strings)
- **PyUnicode_AsUnicodeCopy()**: use PyUnicode_AsUCS4Copy() or PyUnicode_AsWideCharString()

- **PyUnicode_GetMax()**

Functions and macros manipulating Py_UNICODE* strings:

- **Py_UNICODE_strlen()**: use **PyUnicode_GetLength()** or **PyUnicode_GET_LENGTH**
- **Py_UNICODE_strcat()**: use **PyUnicode_CopyCharacters()** or **PyUnicode_FromFormat()**
- **Py_UNICODE_strcpy()**, **Py_UNICODE_strncpy()**, **Py_UNICODE_COPY()**: use **PyUnicode_CopyCharacters()** or **PyUnicode_Substring()**
- **Py_UNICODE_strcmp()**: use **PyUnicode_Compare()**
- **Py_UNICODE_strncmp()**: use **PyUnicode_Tailmatch()**
- **Py_UNICODE_strchr()**, **Py_UNICODE_strrchr()**: use **PyUnicode_FindChar()**
- **Py_UNICODE_FILL()**: use **PyUnicode_Fill()**
- **Py_UNICODE_MATCH**

Encoders:

- **PyUnicode_Encode()**: use **PyUnicode_AsEncodedObject()**
- **PyUnicode_EncodeUTF7()**
- **PyUnicode_EncodeUTF8()**: use **PyUnicode_AsUTF8()** or **PyUnicode_AsUTF8String()**
- **PyUnicode_EncodeUTF32()**
- **PyUnicode_EncodeUTF16()**
- **PyUnicode_EncodeUnicodeEscape()** use **PyUnicode_AsUnicodeEscapeString()**
- **PyUnicode_EncodeRawUnicodeEscape()** use **PyUnicode_AsRawUnicodeEscapeString()**
- **PyUnicode_EncodeLatin1()**: use **PyUnicode_AsLatin1String()**
- **PyUnicode_EncodeASCII()**: use **PyUnicode_AsASCIIString()**
- **PyUnicode_EncodeCharmap()**
- **PyUnicode_TranslateCharmap()**
- **PyUnicode_EncodeMBCS()**: use **PyUnicode_AsMBCSString()** or **PyUnicode_EncodeCodePage()** (with `CP_ACP` code_page)
- **PyUnicode_EncodeDecimal()**, **PyUnicode_TransformDecimalToASCII()**

Deprecated features

The `array` module's `'u'` format code is now deprecated and will be removed in Python 4 together with the rest of the (`Py_UNICODE`) API.

Porting to Python 3.3

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

Porting Python code

- Hash randomization is enabled by default. Set the `PYTHONHASHSEED` environment variable to `0` to disable hash randomization. See also the `object.__hash__()` method.
- bpo-12326 [https://bugs.python.org/issue?@action=redirect&bpo=12326]: On Linux, sys.platform doesn't contain the major version anymore. It is now always 'linux', instead of 'linux2' or 'linux3' depending on the Linux version used to build Python. Replace sys.platform == 'linux2' with sys.platform.startswith('linux'), or directly sys.platform == 'linux' if you don't need to support older Python versions.
- bpo-13847 [https://bugs.python.org/issue?@action=redirect&bpo=13847], bpo-14180 [https://bugs.python.org/issue?@action=redirect&bpo=14180]: `time` and `datetime`: `OverflowError` is now raised instead of `ValueError` if a timestamp is out of range. `OSError` is now raised if C functions **`gmtime()`** or **`localtime()`** failed.
- The default finders used by import now utilize a cache of what is contained within a specific directory. If you create a Python source file or sourceless bytecode file, make sure to call `importlib.invalidate_caches()` to clear out the cache for the finders to notice the new file.
- `ImportError` now uses the full name of the module that was attempted to be imported. Doctests that check ImportErrors' message will need to be updated to use the full name of the module instead of just the tail of the name.

- The *index* argument to `__import__()` now defaults to 0 instead of -1 and no longer support negative values. It was an oversight when **PEP 328** [https://peps.python.org/pep-0328/] was implemented that the default value remained -1. If you need to continue to perform a relative import followed by an absolute import, then perform the relative import using an index of 1, followed by another import using an index of 0. It is preferred, though, that you use `importlib.import_module()` rather than call **`__import__()`** directly.
- `__import__()` no longer allows one to use an index value other than 0 for top-level modules. E.g. `__import__('sys', level=1)` is now an error.
- Because `sys.meta_path` and `sys.path_hooks` now have finders on them by default, you will most likely want to use `list.insert()` instead of `list.append()` to add to those lists.
- Because `None` is now inserted into `sys.path_importer_cache`, if you are clearing out entries in the dictionary of paths that do not have a finder, you will need to remove keys paired with values of `None` **and** **`imp.NullImporter`** to be backwards-compatible. This will lead to extra overhead on older versions of Python that re-insert `None` into **`sys.path_importer_cache`** where it represents the use of implicit finders, but semantically it should not change anything.
- **`importlib.abc.Finder`** no longer specifies a `find_module()` abstract method that must be implemented. If you were relying on subclasses to implement that method, make sure to check for the method's existence first. You will probably want to check for `find_loader()` first, though, in the case of working with path entry finders.
- `pkgutil` has been converted to use `importlib` internally. This eliminates many edge cases where the old behaviour of the **PEP 302** [https://peps.python.org/pep-0302/] import emulation failed to match the behaviour of the real import system. The import emulation itself is still present, but is now deprecated. The `pkgutil.iter_importers()` and `pkgutil.walk_packages()` functions special case the standard import hooks so they are still supported even though they do not provide the non-standard `iter_modules()` method.
- A longstanding RFC-compliance bug (bpo-1079 [https://bugs.python.org/issue?@action=redirect&bpo=1079]) in the parsing done by `email.header.decode_header()` has been fixed. Code that uses the

standard idiom to convert encoded headers into unicode (`str(make_header(decode_header(h)))`) will see no change, but code that looks at the individual tuples returned by decode_header will see that whitespace that precedes or follows `ASCII` sections is now included in the `ASCII` section. Code that builds headers using `make_header` should also continue to work without change, since `make_header` continues to add whitespace between `ASCII` and non-`ASCII` sections if it is not already present in the input strings.

- **`email.utils.formataddr()`** now does the correct content transfer encoding when passed non-`ASCII` display names. Any code that depended on the previous buggy behavior that preserved the non-`ASCII` unicode in the formatted output string will need to be changed (bpo-1690608 [https://bugs.python.org/issue?@action=redirect&bpo=1690608]).
- **`poplib.POP3.quit()`** may now raise protocol errors like all other `poplib` methods. Code that assumes `quit` does not raise **`poplib.error_proto`** errors may need to be changed if errors on `quit` are encountered by a particular application (bpo-11291 [https://bugs.python.org/issue?@action=redirect&bpo=11291]).
- The `strict` argument to **`email.parser.Parser`**, deprecated since Python 2.4, has finally been removed.
- The deprecated method `unittest.TestCase.assertSameElements` has been removed.
- The deprecated variable `time.accept2dyear` has been removed.
- The deprecated `Context._clamp` attribute has been removed from the **`decimal`** module. It was previously replaced by the public attribute **`clamp`**. (See bpo-8540 [https://bugs.python.org/issue?@action=redirect&bpo=8540].)
- The undocumented internal helper class `SSLFakeFile` has been removed from **`smtplib`**, since its functionality has long been provided directly by **`socket.socket.makefile()`**.
- Passing a negative value to **`time.sleep()`** on Windows now raises an error instead of sleeping forever. It has always raised an error on posix.
- The `ast.__version__` constant has been removed. If you need to make decisions affected by the AST version, use **`sys.version_info`** to make the decision.
- Code that used to work around the fact that the **`threading`** module used factory functions by subclassing the private classes will need to change to subclass the now-public classes.

- The undocumented debugging machinery in the threading module has been removed, simplifying the code. This should have no effect on production code, but is mentioned here in case any application debug frameworks were interacting with it (bpo-13550 [https://bugs.python.org/issue?@action=redirect&bpo=13550]).

Porting C code

- In the course of changes to the buffer API the undocumented **`smalltable`** member of the **`Py_buffer`** structure has been removed and the layout of the **`PyMemoryViewObject`** has changed.

  All extensions relying on the relevant parts in `memoryobject.h` or `object.h` must be rebuilt.

- Due to PEP 393, the **`Py_UNICODE`** type and all functions using this type are deprecated (but will stay available for at least five years). If you were using low-level Unicode APIs to construct and access unicode objects and you want to benefit of the memory footprint reduction provided by **PEP 393** [https://peps.python.org/pep-0393/], you have to convert your code to the new Unicode API.

  However, if you only have been using high-level functions such as **`PyUnicode_Concat()`**, **`PyUnicode_Join()`** or **`PyUnicode_FromFormat()`**, your code will automatically take advantage of the new unicode representations.

- **`PyImport_GetMagicNumber()`** now returns `-1` upon failure.

- As a negative value for the *level* argument to **`__import__()`** is no longer valid, the same now holds for **`PyImport_ImportModuleLevel()`**. This also means that the value of *level* used by **`PyImport_ImportModuleEx()`** is now `0` instead of `-1`.

Building C extensions

- The range of possible file names for C extensions has been narrowed. Very rarely used spellings have been suppressed: under POSIX, files named `xxxmodule.so`, `xxxmodule.abi3.so` and `xxxmodule.cpython-*.so` are no longer recognized as implementing the `xxx` module. If you had been generating such files, you have to switch to the other spellings (i.e., remove the `module` string from the file names).

  (implemented in bpo-14040 [https://bugs.python.org/issue?@action=redirect&bpo=14040].)

Command Line Switch Changes

- The -Q command-line flag and related artifacts have been removed. Code checking sys.flags.division_warning will need updating.

  (bpo-10998 [https://bugs.python.org/issue?@action=redirect&bpo=10998], contributed by Éric Araujo.)

- When **python** is started with `-S`, `import site` will no longer add site-specific paths to the module search paths. In previous versions, it did.

  (bpo-11591 [https://bugs.python.org/issue?@action=redirect&bpo=11591], contributed by Carl Meyer with editions by Éric Araujo.)

O que há de novo no Python 3.2

Autor:: Raymond Hettinger

This article explains the new features in Python 3.2 as compared to 3.1. Python 3.2 was released on February 20, 2011. It focuses on a few highlights and gives a few examples. For full details, see the Misc/NEWS [https://github.com/python/cpython/blob/076ca6c3c8df3030307e548d9be792ce3c1c6eea/Misc/NEWS] file.

Ver também

PEP 392 [https://peps.python.org/pep-0392/] - Python 3.2 Release Schedule

PEP 384: Defining a Stable ABI

In the past, extension modules built for one Python version were often not usable with other Python versions. Particularly on Windows, every feature release of Python required rebuilding all extension modules that one wanted to use. This requirement was the result of the free access to Python interpreter internals that extension modules could use.

With Python 3.2, an alternative approach becomes available: extension modules which restrict themselves to a limited API (by defining Py_LIMITED_API) cannot use many of the internals, but are constrained to a set of API functions that are promised to be stable for several releases. As a consequence, extension modules built for 3.2 in that mode will also work with 3.3, 3.4, and so on. Extension modules that make use of details of memory structures can still be built, but will need to be recompiled for every feature release.

Ver também

PEP 384 **[https://peps.python.org/pep-0384/]** **- Defining a Stable ABI**
PEP written by Martin von Löwis.

PEP 389: Argparse Command Line Parsing Module

A new module for command line parsing, **`argparse`**, was introduced to overcome the limitations of **`optparse`** which did not provide support for positional arguments (not just options), subcommands, required options and other common patterns of specifying and validating options.

This module has already had widespread success in the community as a third-party module. Being more fully featured than its predecessor, the **`argparse`** module is now the preferred module for command-line processing. The older module is still being kept available because of the substantial amount of legacy code that depends on it.

Here's an annotated example parser showing features like limiting results to a set of choices, specifying a *metavar* in the help screen, validating that one or more positional arguments is present, and making a required option:

```
import argparse
parser = argparse.ArgumentParser(
            description = 'Manage servers',         # main descr
            epilog = 'Tested on Solaris and Linux') # displayed
parser.add_argument('action',                       # argument r
            choices = ['deploy', 'start', 'stop'],  # three allc
            help = 'action on each target')         # help msg
parser.add_argument('targets',
            metavar = 'HOSTNAME',                   # var name u
            nargs = '+',                            # require or
            help = 'url for target machines')       # help msg e
parser.add_argument('-u', '--user',                 # -u or --us
            required = True,                        # make it a
            help = 'login as user')
```

Example of calling the parser on a command string:

```
>>> cmd = 'deploy sneezy.example.com sleepy.example.com -u skyca
>>> result = parser.parse_args(cmd.split())
>>> result.action
'deploy'
>>> result.targets
['sneezy.example.com', 'sleepy.example.com']
>>> result.user
'skycaptain'
```

Example of the parser's automatically generated help:

```
>>> parser.parse_args('-h'.split())

usage: manage_cloud.py [-h] -u USER
                       {deploy,start,stop} HOSTNAME [HOSTNAME ..

Manage servers

positional arguments:
  {deploy,start,stop}   action on each target
  HOSTNAME              url for target machines

optional arguments:
  -h, --help            show this help message and exit
  -u USER, --user USER  login as user

Tested on Solaris and Linux
```

An especially nice **argparse** feature is the ability to define subparsers, each with their own argument patterns and help displays:

```
import argparse
parser = argparse.ArgumentParser(prog='HELM')
subparsers = parser.add_subparsers()

parser_l = subparsers.add_parser('launch', help='Launch Control'
parser_l.add_argument('-m', '--missiles', action='store_true')
parser_l.add_argument('-t', '--torpedos', action='store_true')

parser_m = subparsers.add_parser('move', help='Move Vessel',
                                 aliases=('steer', 'turn'))
```

```
parser_m.add_argument('-c', '--course', type=int, required=True)
parser_m.add_argument('-s', '--speed', type=int, default=0)
```

```
$ ./helm.py --help                          # top level help (lau
$ ./helm.py launch --help                   # help for launch opt
$ ./helm.py launch --missiles               # set missiles=True a
$ ./helm.py steer --course 180 --speed 5    # set movement parame
```

Ver também

PEP 389 [https://peps.python.org/pep-0389/] - New Command Line Parsing Module

PEP written by Steven Bethard.

Migrando código optparse para argparse for details on the differences from `optparse`.

PEP 391: Dictionary Based Configuration for Logging

The `logging` module provided two kinds of configuration, one style with function calls for each option or another style driven by an external file saved in a `configparser` format. Those options did not provide the flexibility to create configurations from JSON or YAML files, nor did they support incremental configuration, which is needed for specifying logger options from a command line.

To support a more flexible style, the module now offers `logging.config.dictConfig()` for specifying logging configuration with plain Python dictionaries. The configuration options include formatters, handlers, filters, and loggers. Here's a working example of a configuration dictionary:

```
{"version": 1,
 "formatters": {"brief": {"format": "%(levelname)-8s: %(name)-15
                "full": {"format": "%(asctime)s %(name)-15s %(le
```

```
                },
 "handlers": {"console": {
                    "class": "logging.StreamHandler",
                    "formatter": "brief",
                    "level": "INFO",
                    "stream": "ext://sys.stdout"},
               "console_priority": {
                    "class": "logging.StreamHandler",
                    "formatter": "full",
                    "level": "ERROR",
                    "stream": "ext://sys.stderr"}
               },
 "root": {"level": "DEBUG", "handlers": ["console", "console_pri
```

If that dictionary is stored in a file called `conf.json`, it can be loaded and called with code like this:

```
>>> import json, logging.config
>>> with open('conf.json') as f:
...     conf = json.load(f)
...
>>> logging.config.dictConfig(conf)
>>> logging.info("Transaction completed normally")
INFO    : root           : Transaction completed normally
>>> logging.critical("Abnormal termination")
2011-02-17 11:14:36,694 root            CRITICAL Abnormal termir
```

Ver também

PEP 391 [https://peps.python.org/pep-0391/] - Dictionary Based Configuration for Logging

PEP written by Vinay Sajip.

PEP 3148: The `concurrent.futures` module

Code for creating and managing concurrency is being collected in a new top-level namespace, *concurrent*. Its first member is a *futures* package

which provides a uniform high-level interface for managing threads and processes.

The design for `concurrent.futures` was inspired by the *java.util.concurrent* package. In that model, a running call and its result are represented by a `Future` object that abstracts features common to threads, processes, and remote procedure calls. That object supports status checks (running or done), timeouts, cancellations, adding callbacks, and access to results or exceptions.

The primary offering of the new module is a pair of executor classes for launching and managing calls. The goal of the executors is to make it easier to use existing tools for making parallel calls. They save the effort needed to setup a pool of resources, launch the calls, create a results queue, add time-out handling, and limit the total number of threads, processes, or remote procedure calls.

Ideally, each application should share a single executor across multiple components so that process and thread limits can be centrally managed. This solves the design challenge that arises when each component has its own competing strategy for resource management.

Both classes share a common interface with three methods: `submit()` for scheduling a callable and returning a `Future` object; `map()` for scheduling many asynchronous calls at a time, and `shutdown()` for freeing resources. The class is a context manager and can be used in a `with` statement to assure that resources are automatically released when currently pending futures are done executing.

A simple of example of `ThreadPoolExecutor` is a launch of four parallel threads for copying files:

```
import concurrent.futures, shutil
with concurrent.futures.ThreadPoolExecutor(max_workers=4) as e:
    e.submit(shutil.copy, 'src1.txt', 'dest1.txt')
    e.submit(shutil.copy, 'src2.txt', 'dest2.txt')
    e.submit(shutil.copy, 'src3.txt', 'dest3.txt')
    e.submit(shutil.copy, 'src3.txt', 'dest4.txt')
```

Ver também

PEP 3148 [https://peps.python.org/pep-3148/] - Futures – Execute Computations Asynchronously
PEP written by Brian Quinlan.

Code for Threaded Parallel URL reads, an example using threads to fetch multiple web pages in parallel.

Code for computing prime numbers in parallel, an example demonstrating `ProcessPoolExecutor`.

PEP 3147: PYC Repository Directories

Python's scheme for caching bytecode in *.pyc* files did not work well in environments with multiple Python interpreters. If one interpreter encountered a cached file created by another interpreter, it would recompile the source and overwrite the cached file, thus losing the benefits of caching.

The issue of "pyc fights" has become more pronounced as it has become commonplace for Linux distributions to ship with multiple versions of Python. These conflicts also arise with CPython alternatives such as Unladen Swallow.

To solve this problem, Python's import machinery has been extended to use distinct filenames for each interpreter. Instead of Python 3.2 and Python 3.3 and Unladen Swallow each competing for a file called "mymodule.pyc", they will now look for "mymodule.cpython-32.pyc", "mymodule.cpython-33.pyc", and "mymodule.unladen10.pyc". And to prevent all of these new files from cluttering source directories, the *pyc* files are now collected in a "__pycache__" directory stored under the package directory.

Aside from the filenames and target directories, the new scheme has a few aspects that are visible to the programmer:

- Imported modules now have a `__cached__` attribute which stores the name of the actual file that was imported:

```
>>> import collections
>>> collections.__cached__
'c:/py32/lib/__pycache__/collections.cpython-32.pyc'
```

- The tag that is unique to each interpreter is accessible from the **`imp`** module:

```
>>> import imp
>>> imp.get_tag()
'cpython-32'
```

- Scripts that try to deduce source filename from the imported file now need to be smarter. It is no longer sufficient to simply strip the "c" from a ".pyc" filename. Instead, use the new functions in the **`imp`** module:

```
>>> imp.source_from_cache('c:/py32/lib/__pycache__/collectio
'c:/py32/lib/collections.py'
>>> imp.cache_from_source('c:/py32/lib/collections.py')
'c:/py32/lib/__pycache__/collections.cpython-32.pyc'
```

- The **`py_compile`** and **`compileall`** modules have been updated to reflect the new naming convention and target directory. The command-line invocation of *compileall* has new options: `-i` for specifying a list of files and directories to compile and `-b` which causes bytecode files to be written to their legacy location rather than *__pycache__*.

- The **`importlib.abc`** module has been updated with new abstract base classes for loading bytecode files. The obsolete ABCs, **`PyLoader`** and **`PyPycLoader`**, have been deprecated (instructions on how to stay Python 3.1 compatible are included with the documentation).

Ver também

PEP 3147 **[https://peps.python.org/pep-3147/]** **- PYC Repository Directories**
PEP written by Barry Warsaw.

PEP 3149: ABI Version Tagged .so Files

The PYC repository directory allows multiple bytecode cache files to be co-located. This PEP implements a similar mechanism for shared object files by giving them a common directory and distinct names for each version.

The common directory is “pyshared” and the file names are made distinct by identifying the Python implementation (such as CPython, PyPy, Jython, etc.), the major and minor version numbers, and optional build flags (such as “d” for debug, “m” for pymalloc, “u” for wide-unicode). For an arbitrary package “foo”, you may see these files when the distribution package is installed:

```
/usr/share/pyshared/foo.cpython-32m.so
/usr/share/pyshared/foo.cpython-33md.so
```

In Python itself, the tags are accessible from functions in the `sysconfig` module:

```
>>> import sysconfig
>>> sysconfig.get_config_var('SOABI')       # find the version t
'cpython-32mu'
>>> sysconfig.get_config_var('EXT_SUFFIX')  # find the full file
'.cpython-32mu.so'
```

Ver também

PEP 3149 **[https://peps.python.org/pep-3149/]** **- ABI Version Tagged .so Files**
PEP written by Barry Warsaw.

PEP 3333: Python Web Server Gateway Interface v1.0.1

This informational PEP clarifies how bytes/text issues are to be handled by the WSGI protocol. The challenge is that string handling in Python 3 is most conveniently handled with the `str` type even though the HTTP protocol is itself bytes oriented.

The PEP differentiates so-called *native strings* that are used for request/response headers and metadata versus *byte strings* which are used for the bodies of requests and responses.

The *native strings* are always of type `str` but are restricted to code points between *U+0000* through *U+00FF* which are translatable to bytes using *Latin-1* encoding. These strings are used for the keys and values in the environment dictionary and for response headers and statuses in the **`start_response()`** function. They must follow **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html] with respect to encoding. That is, they must either be *ISO-8859-1* characters or use **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] MIME encoding.

For developers porting WSGI applications from Python 2, here are the salient points:

- If the app already used strings for headers in Python 2, no change is needed.
- If instead, the app encoded output headers or decoded input headers, then the headers will need to be re-encoded to Latin-1. For example, an output header encoded in utf-8 was using `h.encode('utf-8')` now needs to convert from bytes to native strings using `h.encode('utf-8').decode('latin-1')`.
- Values yielded by an application or sent using the **`write()`** method must be byte strings. The **`start_response()`** function and environ must use native strings. The two cannot be mixed.

For server implementers writing CGI-to-WSGI pathways or other CGI-style protocols, the users must to be able access the environment using native strings even though the underlying platform may have a different convention. To bridge this gap, the `wsgiref` module has a new function, `wsgiref.handlers.read_environ()` for transcoding CGI variables from `os.environ` into native strings and returning a new dictionary.

Ver também

PEP 3333 [https://peps.python.org/pep-3333/] - Python Web Server Gateway Interface v1.0.1

PEP written by Phillip Eby.

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- String formatting for `format()` and `str.format()` gained new capabilities for the format character #. Previously, for integers in binary, octal, or hexadecimal, it caused the output to be prefixed with '0b', '0o', or '0x' respectively. Now it can also handle floats, complex, and Decimal, causing the output to always have a decimal point even when no digits follow it.

  ```
  >>> format(20, '#o')
  '0o24'
  >>> format(12.34, '#5.0f')
  '  12.'
  ```

  (Suggested by Mark Dickinson and implemented by Eric Smith in bpo-7094 [https://bugs.python.org/issue?@action=redirect&bpo=7094].)

- There is also a new `str.format_map()` method that extends the capabilities of the existing `str.format()` method by accepting arbitrary mapping objects. This new method makes it possible to use string formatting with any of Python's many dictionary-like objects such as `defaultdict`, `Shelf`, `ConfigParser`, or `dbm`. It is also useful with custom `dict` subclasses that normalize keys before look-up or that supply a `__missing__()` method for unknown keys:

  ```
  >>> import shelve
  >>> d = shelve.open('tmp.shl')
  >>> 'The {project_name} status is {status} as of {date}'.for
  'The testing project status is green as of February 15, 201
```

```
>>> class LowerCasedDict(dict):
...     def __getitem__(self, key):
...         return dict.__getitem__(self, key.lower())
...
>>> lcd = LowerCasedDict(part='widgets', quantity=10)
>>> 'There are {QUANTITY} {Part} in stock'.format_map(lcd)
'There are 10 widgets in stock'

>>> class PlaceholderDict(dict):
...     def __missing__(self, key):
...         return '<{}>'.format(key)
...
>>> 'Hello {name}, welcome to {location}'.format_map(Placeho
'Hello <name>, welcome to <location>'
```

(Suggested by Raymond Hettinger and implemented by Eric Smith in bpo-6081 [https://bugs.python.org/issue?@action=redirect&bpo=6081].)

- The interpreter can now be started with a quiet option, `-q`, to prevent the copyright and version information from being displayed in the interactive mode. The option can be introspected using the **`sys.flags`** attribute:

  ```
  $ python -q
  >>> sys.flags
  sys.flags(debug=0, division_warning=0, inspect=0, interactiv
  optimize=0, dont_write_bytecode=0, no_user_site=0, no_site=(
  ignore_environment=0, verbose=0, bytes_warning=0, quiet=1)
  ```

  (Contributed by Marcin Wojdyr in bpo-1772833 [https://bugs.python.org/issue?@action=redirect&bpo=1772833]).

- The **`hasattr()`** function works by calling **`getattr()`** and detecting whether an exception is raised. This technique allows it to detect methods created dynamically by **`__getattr__()`** or **`__getattribute__()`** which would otherwise be absent from the class dictionary. Formerly, *hasattr* would catch any exception, possibly

masking genuine errors. Now, *hasattr* has been tightened to only catch **`AttributeError`** and let other exceptions pass through:

```
>>> class A:
...     @property
...     def f(self):
...         return 1 // 0
...
>>> a = A()
>>> hasattr(a, 'f')
Traceback (most recent call last):
  ...
ZeroDivisionError: integer division or modulo by zero
```

(Discovered by Yury Selivanov and fixed by Benjamin Peterson; bpo-9666 [https://bugs.python.org/issue?@action=redirect&bpo=9666].)

- The **`str()`** of a float or complex number is now the same as its **`repr()`**. Previously, the **`str()`** form was shorter but that just caused confusion and is no longer needed now that the shortest possible **`repr()`** is displayed by default:

```
>>> import math
>>> repr(math.pi)
'3.141592653589793'
>>> str(math.pi)
'3.141592653589793'
```

(Proposed and implemented by Mark Dickinson; bpo-9337 [https://bugs.python.org/issue?@action=redirect&bpo=9337].)

- **`memoryview`** objects now have a **`release()`** method and they also now support the context management protocol. This allows timely release of any resources that were acquired when requesting a buffer from the original object.

```
>>> with memoryview(b'abcdefgh') as v:
...     print(v.tolist())
[97, 98, 99, 100, 101, 102, 103, 104]
```

(Added by Antoine Pitrou; bpo-9757 [https://bugs.python.org/issue?@action=redirect&bpo=9757].)

- Previously it was illegal to delete a name from the local namespace if it occurs as a free variable in a nested block:

```
def outer(x):
    def inner():
        return x
    inner()
    del x
```

This is now allowed. Remember that the target of an **`except`** clause is cleared, so this code which used to work with Python 2.6, raised a **`SyntaxError`** with Python 3.1 and now works again:

```
def f():
    def print_error():
        print(e)
    try:
        something
    except Exception as e:
        print_error()
        # implicit "del e" here
```

(See bpo-4617 [https://bugs.python.org/issue?@action=redirect&bpo=4617].)

- Struct sequence types are now subclasses of tuple. This means that C structures like those returned by **`os.stat()`**, **`time.gmtime()`**, and **`sys.version_info`** now work like a named tuple and now work with functions and methods that expect a tuple as an argument. This is a big step forward in making the C structures as flexible as their pure Python counterparts:

```
>>> import sys
>>> isinstance(sys.version_info, tuple)
True
>>> 'Version %d.%d.%d %s(%d)' % sys.version_info
'Version 3.2.0 final(0)'
```

(Suggested by Arfrever Frehtes Taifersar Arahesis and implemented by Benjamin Peterson in bpo-8413 [https://bugs.python.org/issue?@action=redirect&bpo=8413].)

- Warnings are now easier to control using the `PYTHONWARNINGS` environment variable as an alternative to using `-W` at the command line:

```
$ export PYTHONWARNINGS='ignore::RuntimeWarning::,once::Unic
```

(Suggested by Barry Warsaw and implemented by Philip Jenvey in bpo-7301 [https://bugs.python.org/issue?@action=redirect&bpo=7301].)

- A new warning category, `ResourceWarning`, has been added. It is emitted when potential issues with resource consumption or cleanup are detected. It is silenced by default in normal release builds but can be enabled through the means provided by the `warnings` module, or on the command line.

  A `ResourceWarning` is issued at interpreter shutdown if the `gc.garbage` list isn't empty, and if `gc.DEBUG_UNCOLLECTABLE` is set, all uncollectable objects are printed. This is meant to make the programmer aware that their code contains object finalization issues.

  A `ResourceWarning` is also issued when a file object is destroyed without having been explicitly closed. While the deallocator for such object ensures it closes the underlying operating system resource (usually, a file descriptor), the delay in deallocating the object could produce various issues, especially under Windows. Here is an example of enabling the warning from the command line:

```
$ python -q -Wdefault
>>> f = open("foo", "wb")
>>> del f
__main__:1: ResourceWarning: unclosed file <_io.BufferedWrit
```

(Added by Antoine Pitrou and Georg Brandl in bpo-10093 [https://bugs.python.org/issue?@action=redirect&bpo=10093] and bpo-477863 [https://bugs.python.org/issue?@action=redirect&bpo=477863].)

- **`range`** objects now support *index* and *count* methods. This is part of an effort to make more objects fully implement the **`collections.Sequence`** abstract base class. As a result, the language will have a more uniform API. In addition, **`range`** objects now support slicing and negative indices, even with values larger than **`sys.maxsize`**. This makes *range* more interoperable with lists:

```
>>> range(0, 100, 2).count(10)
1
>>> range(0, 100, 2).index(10)
5
>>> range(0, 100, 2)[5]
10
>>> range(0, 100, 2)[0:5]
range(0, 10, 2)
```

(Contributed by Daniel Stutzbach in bpo-9213 [https://bugs.python.org/issue?@action=redirect&bpo=9213], by Alexander Belopolsky in bpo-2690 [https://bugs.python.org/issue?@action=redirect&bpo=2690], and by Nick Coghlan in bpo-10889 [https://bugs.python.org/issue?@action=redirect&bpo=10889].)

- The **`callable()`** builtin function from Py2.x was resurrected. It provides a concise, readable alternative to using an abstract base class in an expression like `isinstance(x, collections.Callable)`:

```
>>> callable(max)
True
>>> callable(20)
False
```

(See bpo-10518 [https://bugs.python.org/issue?@action=redirect&bpo=10518].)

- Python's import mechanism can now load modules installed in directories with non-ASCII characters in the path name. This solved an aggravating problem with home directories for users with non-ASCII characters in their usernames.

(Required extensive work by Victor Stinner in bpo-9425 [https://bugs.python.org/issue?@action=redirect&bpo=9425].)

New, Improved, and Deprecated Modules

Python's standard library has undergone significant maintenance efforts and quality improvements.

The biggest news for Python 3.2 is that the `email` package, `mailbox` module, and `nntplib` modules now work correctly with the bytes/text model in Python 3. For the first time, there is correct handling of messages with mixed encodings.

Throughout the standard library, there has been more careful attention to encodings and text versus bytes issues. In particular, interactions with the operating system are now better able to exchange non-ASCII data using the Windows MBCS encoding, locale-aware encodings, or UTF-8.

Another significant win is the addition of substantially better support for *SSL* connections and security certificates.

In addition, more classes now implement a context manager to support convenient and reliable resource clean-up using a `with` statement.

email

The usability of the `email` package in Python 3 has been mostly fixed by the extensive efforts of R. David Murray. The problem was that emails are typically read and stored in the form of `bytes` rather than `str` text, and they may contain multiple encodings within a single email. So, the email package had to be extended to parse and generate email messages in bytes format.

- New functions `message_from_bytes()` and `message_from_binary_file()`, and new classes `BytesFeedParser` and `BytesParser` allow binary message data to be parsed into model objects.

- Given bytes input to the model, `get_payload()` will by default decode a message body that has a *Content-Transfer-Encoding* of *8bit* using the charset specified in the MIME headers and return the resulting string.

- Given bytes input to the model, `Generator` will convert message bodies that have a *Content-Transfer-Encoding* of *8bit* to instead have a *7bit Content-Transfer-Encoding*.

  Headers with unencoded non-ASCII bytes are deemed to be **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html]-encoded using the *unknown-8bit* character set.

- A new class `BytesGenerator` produces bytes as output, preserving any unchanged non-ASCII data that was present in the input used to build the model, including message bodies with a *Content-Transfer-Encoding* of *8bit*.

- The `smtplib` `SMTP` class now accepts a byte string for the *msg* argument to the `sendmail()` method, and a new method, `send_message()` accepts a `Message` object and can optionally obtain the *from_addr* and *to_addrs* addresses directly from the object.

(Proposed and implemented by R. David Murray, bpo-4661 [https://bugs.python.org/issue?@action=redirect&bpo=4661] and bpo-10321 [https://bugs.python.org/issue?@action=redirect&bpo=10321].)

elementtree

The `xml.etree.ElementTree` package and its `xml.etree.cElementTree` counterpart have been updated to version 1.3.

Several new and useful functions and methods have been added:

- `xml.etree.ElementTree.fromstringlist()` which builds an XML document from a sequence of fragments
- `xml.etree.ElementTree.register_namespace()` for registering a global namespace prefix

- `xml.etree.ElementTree.tostringlist()` for string representation including all sublists
- `xml.etree.ElementTree.Element.extend()` for appending a sequence of zero or more elements
- `xml.etree.ElementTree.Element.iterfind()` searches an element and subelements
- `xml.etree.ElementTree.Element.itertext()` creates a text iterator over an element and its subelements
- `xml.etree.ElementTree.TreeBuilder.end()` closes the current element
- `xml.etree.ElementTree.TreeBuilder.doctype()` handles a doctype declaration

Two methods have been deprecated:

- **`xml.etree.ElementTree.getchildren()`** use `list(elem)` instead.
- **`xml.etree.ElementTree.getiterator()`** use `Element.iter` instead.

For details of the update, see Introducing ElementTree [https://web.archive.org/web/20200703234532/http://effbot.org/zone/elementtree-13-intro.htm] on Fredrik Lundh's website.

(Contributed by Florent Xicluna and Fredrik Lundh, bpo-6472 [https://bugs.python.org/issue?@action=redirect&bpo=6472].)

functools

- The `functools` module includes a new decorator for caching function calls. `@functools.lru_cache` can save repeated queries to an external resource whenever the results are expected to be the same.

  For example, adding a caching decorator to a database query function can save database accesses for popular searches:

  ```
  >>> import functools
  >>> @functools.lru_cache(maxsize=300)
  ... def get_phone_number(name):
  ...     c = conn.cursor()
  ```

```
...     c.execute('SELECT phonenumber FROM phonelist WHERE r
...     return c.fetchone()[0]
```

```
>>> for name in user_requests:
...     get_phone_number(name)        # cached lookup
```

To help with choosing an effective cache size, the wrapped function is instrumented for tracking cache statistics:

```
>>> get_phone_number.cache_info()
CacheInfo(hits=4805, misses=980, maxsize=300, currsize=300)
```

If the phonelist table gets updated, the outdated contents of the cache can be cleared with:

```
>>> get_phone_number.cache_clear()
```

(Contributed by Raymond Hettinger and incorporating design ideas from Jim Baker, Miki Tebeka, and Nick Coghlan; see recipe 498245 [https://code.activestate.com/recipes/498245-lru-and-lfu-cache-decorators/], recipe 577479 [https://code.activestate.com/recipes/577479-simple-caching-decorator/], bpo-10586 [https://bugs.python.org/issue?@action=redirect&bpo=10586], and bpo-10593 [https://bugs.python.org/issue?@action=redirect&bpo=10593].)

- The **`@functools.wraps`** decorator now adds a **`__wrapped__`** attribute pointing to the original callable function. This allows wrapped functions to be introspected. It also copies **`__annotations__`** if defined. And now it also gracefully skips over missing attributes such as **`__doc__`** which might not be defined for the wrapped callable.

  In the above example, the cache can be removed by recovering the original function:

```
>>> get_phone_number = get_phone_number.__wrapped__     # unc
```

  (By Nick Coghlan and Terrence Cole; bpo-9567 [https://bugs.python.org/issue?@action=redirect&bpo=9567], bpo-3445

[https://bugs.python.org/issue?@action=redirect&bpo=3445], and bpo-8814 [https://bugs.python.org/issue?@action=redirect&bpo=8814].)

- To help write classes with rich comparison methods, a new decorator `@functools.total_ordering` will use existing equality and inequality methods to fill in the remaining methods.

  For example, supplying *__eq__* and *__lt__* will enable `@~functools.total_ordering` to fill-in *__le__*, *__gt__* and *__ge__*:

  ```
  @total_ordering
  class Student:
      def __eq__(self, other):
          return ((self.lastname.lower(), self.firstname.lower
                  (other.lastname.lower(), other.firstname.low

      def __lt__(self, other):
          return ((self.lastname.lower(), self.firstname.lower
                  (other.lastname.lower(), other.firstname.low
  ```

  With the *total_ordering* decorator, the remaining comparison methods are filled in automatically.

  (Contributed by Raymond Hettinger.)

- To aid in porting programs from Python 2, the `functools.cmp_to_key()` function converts an old-style comparison function to modern key function:

  ```
  >>> # locale-aware sort order
  >>> sorted(iterable, key=cmp_to_key(locale.strcoll))
  ```

  For sorting examples and a brief sorting tutorial, see the Sorting HowTo [https://wiki.python.org/moin/HowTo/Sorting/] tutorial.

  (Contributed by Raymond Hettinger.)

itertools

- The `itertools` module has a new `accumulate()` function modeled on APL's *scan* operator and Numpy's *accumulate* function:

```
>>> from itertools import accumulate
>>> list(accumulate([8, 2, 50]))
[8, 10, 60]
```

```
>>> prob_dist = [0.1, 0.4, 0.2, 0.3]
>>> list(accumulate(prob_dist))      # cumulative probabilit
[0.1, 0.5, 0.7, 1.0]
```

  For an example using `accumulate()`, see the examples for the random module.

  (Contributed by Raymond Hettinger and incorporating design suggestions from Mark Dickinson.)

collections

- The `collections.Counter` class now has two forms of in-place subtraction, the existing `-=` operator for saturating subtraction [https://en.wikipedia.org/wiki/Saturation_arithmetic] and the new `subtract()` method for regular subtraction. The former is suitable for multisets [https://en.wikipedia.org/wiki/Multiset] which only have positive counts, and the latter is more suitable for use cases that allow negative counts:

```
>>> from collections import Counter
>>> tally = Counter(dogs=5, cats=3)
>>> tally -= Counter(dogs=2, cats=8)    # saturating subtrac
>>> tally
Counter({'dogs': 3})
```

```
>>> tally = Counter(dogs=5, cats=3)
>>> tally.subtract(dogs=2, cats=8)      # regular subtractio
>>> tally
Counter({'dogs': 3, 'cats': -5})
```

  (Contributed by Raymond Hettinger.)

- The `collections.OrderedDict` class has a new method `move_to_end()` which takes an existing key and moves it to either the first or last position in the ordered sequence.

  The default is to move an item to the last position. This is equivalent of renewing an entry with `od[k] = od.pop(k)`.

  A fast move-to-end operation is useful for resequencing entries. For example, an ordered dictionary can be used to track order of access by aging entries from the oldest to the most recently accessed.

  ```
  >>> from collections import OrderedDict
  >>> d = OrderedDict.fromkeys(['a', 'b', 'X', 'd', 'e'])
  >>> list(d)
  ['a', 'b', 'X', 'd', 'e']
  >>> d.move_to_end('X')
  >>> list(d)
  ['a', 'b', 'd', 'e', 'X']
  ```

  (Contributed by Raymond Hettinger.)

- The `collections.deque` class grew two new methods `count()` and `reverse()` that make them more substitutable for `list` objects:

  ```
  >>> from collections import deque
  >>> d = deque('simsalabim')
  >>> d.count('s')
  2
  >>> d.reverse()
  >>> d
  deque(['m', 'i', 'b', 'a', 'l', 'a', 's', 'm', 'i', 's'])
  ```

  (Contributed by Raymond Hettinger.)

threading

The `threading` module has a new `Barrier` synchronization class for making multiple threads wait until all of them have reached a common barrier point. Barriers are useful for making sure that a task with multiple preconditions does not run until all of the predecessor tasks are complete.

Barriers can work with an arbitrary number of threads. This is a generalization of a Rendezvous [https://en.wikipedia.org/wiki/Synchronous_rendezvous] which is defined for only two threads.

Implemented as a two-phase cyclic barrier, `Barrier` objects are suitable for use in loops. The separate *filling* and *draining* phases assure that all threads get released (drained) before any one of them can loop back and re-enter the barrier. The barrier fully resets after each cycle.

Example of using barriers:

```
from threading import Barrier, Thread

def get_votes(site):
    ballots = conduct_election(site)
    all_polls_closed.wait()        # do not count until all poll
    totals = summarize(ballots)
    publish(site, totals)

all_polls_closed = Barrier(len(sites))
for site in sites:
    Thread(target=get_votes, args=(site,)).start()
```

In this example, the barrier enforces a rule that votes cannot be counted at any polling site until all polls are closed. Notice how a solution with a barrier is similar to one with `threading.Thread.join()`, but the threads stay alive and continue to do work (summarizing ballots) after the barrier point is crossed.

If any of the predecessor tasks can hang or be delayed, a barrier can be created with an optional *timeout* parameter. Then if the timeout period elapses before all the predecessor tasks reach the barrier point, all waiting threads are released and a `BrokenBarrierError` exception is raised:

```
def get_votes(site):
    ballots = conduct_election(site)
    try:
        all_polls_closed.wait(timeout=midnight - time.now())
    except BrokenBarrierError:
```

```
        lockbox = seal_ballots(ballots)
        queue.put(lockbox)
    else:
        totals = summarize(ballots)
        publish(site, totals)
```

In this example, the barrier enforces a more robust rule. If some election sites do not finish before midnight, the barrier times-out and the ballots are sealed and deposited in a queue for later handling.

See Barrier Synchronization Patterns [https://osl.cs.illinois.edu/media/papers/karmani-2009-barrier_synchronization_pattern.pdf] for more examples of how barriers can be used in parallel computing. Also, there is a simple but thorough explanation of barriers in The Little Book of Semaphores [https://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf], *section 3.6*.

(Contributed by Kristján Valur Jónsson with an API review by Jeffrey Yasskin in bpo-8777 [https://bugs.python.org/issue?@action=redirect&bpo=8777].)

datetime and time

- The **`datetime`** module has a new type **`timezone`** that implements the **`tzinfo`** interface by returning a fixed UTC offset and timezone name. This makes it easier to create timezone-aware datetime objects:

  ```
  >>> import datetime as dt

  >>> dt.datetime.now(dt.timezone.utc)
  datetime.datetime(2010, 12, 8, 21, 4, 2, 923754, tzinfo=date

  >>> dt.datetime.strptime("01/01/2000 12:00 +0000", "%m/%d/%
  datetime.datetime(2000, 1, 1, 12, 0, tzinfo=datetime.timezor
  ```

- Also, **`timedelta`** objects can now be multiplied by **`float`** and divided by **`float`** and **`int`** objects. And **`timedelta`** objects can now divide one another.

- The **`datetime.date.strftime()`** method is no longer restricted to years after 1900. The new supported year range is from 1000 to 9999

inclusive.

- Whenever a two-digit year is used in a time tuple, the interpretation has been governed by **`time.accept2dyear`**. The default is `True` which means that for a two-digit year, the century is guessed according to the POSIX rules governing the `%y` strptime format.

  Starting with Py3.2, use of the century guessing heuristic will emit a **`DeprecationWarning`**. Instead, it is recommended that **`time.accept2dyear`** be set to `False` so that large date ranges can be used without guesswork:

  ```
  >>> import time, warnings
  >>> warnings.resetwarnings()      # remove the default warn

  >>> time.accept2dyear = True      # guess whether 11 means 
  >>> time.asctime((11, 1, 1, 12, 34, 56, 4, 1, 0))
  Warning (from warnings module):
    ...
  DeprecationWarning: Century info guessed for a 2-digit year
  'Fri Jan  1 12:34:56 2011'

  >>> time.accept2dyear = False     # use the full range of a
  >>> time.asctime((11, 1, 1, 12, 34, 56, 4, 1, 0))
  'Fri Jan  1 12:34:56 11'
  ```

  Several functions now have significantly expanded date ranges. When **`time.accept2dyear`** is false, the **`time.asctime()`** function will accept any year that fits in a C int, while the **`time.mktime()`** and **`time.strftime()`** functions will accept the full range supported by the corresponding operating system functions.

(Contributed by Alexander Belopolsky and Victor Stinner in bpo-1289118 [https://bugs.python.org/issue?@action=redirect&bpo=1289118], bpo-5094 [https://bugs.python.org/issue?@action=redirect&bpo=5094], bpo-6641 [https://bugs.python.org/issue?@action=redirect&bpo=6641], bpo-2706 [https://bugs.python.org/issue?@action=redirect&bpo=2706], bpo-1777412 [https://bugs.python.org/issue?@action=redirect&bpo=1777412], bpo-8013

[https://bugs.python.org/issue?@action=redirect&bpo=8013], and bpo-10827 [https://bugs.python.org/issue?@action=redirect&bpo=10827].)

math

The `math` module has been updated with six new functions inspired by the C99 standard.

The `isfinite()` function provides a reliable and fast way to detect special values. It returns `True` for regular numbers and `False` for *Nan* or *Infinity*:

```
>>> from math import isfinite
>>> [isfinite(x) for x in (123, 4.56, float('Nan'), float('Inf')
[True, True, False, False]
```

The `expm1()` function computes `e**x-1` for small values of *x* without incurring the loss of precision that usually accompanies the subtraction of nearly equal quantities:

```
>>> from math import expm1
>>> expm1(0.013671875)   # more accurate way to compute e**x-1 f
0.013765762467652909
```

The `erf()` function computes a probability integral or Gaussian error function [https://en.wikipedia.org/wiki/Error_function]. The complementary error function, `erfc()`, is `1 - erf(x)`:

```
>>> from math import erf, erfc, sqrt
>>> erf(1.0/sqrt(2.0))   # portion of normal distribution withir
0.682689492137086
>>> erfc(1.0/sqrt(2.0))  # portion of normal distribution outsic
0.31731050786291404
>>> erf(1.0/sqrt(2.0)) + erfc(1.0/sqrt(2.0))
1.0
```

The `gamma()` function is a continuous extension of the factorial function. See https://en.wikipedia.org/wiki/Gamma_function for details. Because the function is related to factorials, it grows large even for small values of *x*, so

there is also a `lgamma()` function for computing the natural logarithm of the gamma function:

```
>>> from math import gamma, lgamma
>>> gamma(7.0)           # six factorial
720.0
>>> lgamma(801.0)        # log(800 factorial)
4551.950730698041
```

(Contributed by Mark Dickinson.)

abc

The `abc` module now supports `@~abc.abstractclassmethod` and `@~abc.abstractstaticmethod`.

These tools make it possible to define an abstract base class that requires a particular `@classmethod` or `@staticmethod` to be implemented:

```
class Temperature(metaclass=abc.ABCMeta):
    @abc.abstractclassmethod
    def from_fahrenheit(cls, t):
        ...
    @abc.abstractclassmethod
    def from_celsius(cls, t):
        ...
```

(Patch submitted by Daniel Urban; bpo-5867 [https://bugs.python.org/issue?@action=redirect&bpo=5867].)

io

The `io.BytesIO` has a new method, `getbuffer()`, which provides functionality similar to `memoryview()`. It creates an editable view of the data without making a copy. The buffer's random access and support for slice notation are well-suited to in-place editing:

```
>>> REC_LEN, LOC_START, LOC_LEN = 34, 7, 11

>>> def change_location(buffer, record_number, location):
```

```
...     start = record_number * REC_LEN + LOC_START
...     buffer[start: start+LOC_LEN] = location

>>> import io

>>> byte_stream = io.BytesIO(
...     b'G3805  storeroom  Main chassis    '
...     b'X7899  shipping   Reserve cog     '
...     b'L6988  receiving  Primary sprocket'
... )
>>> buffer = byte_stream.getbuffer()
>>> change_location(buffer, 1, b'warehouse  ')
>>> change_location(buffer, 0, b'showroom   ')
>>> print(byte_stream.getvalue())
b'G3805  showroom   Main chassis    '
b'X7899  warehouse  Reserve cog     '
b'L6988  receiving  Primary sprocket'
```

(Contributed by Antoine Pitrou in bpo-5506 [https://bugs.python.org/issue?@action=redirect&bpo=5506].)

reprlib

When writing a `__repr__()` method for a custom container, it is easy to forget to handle the case where a member refers back to the container itself. Python's builtin objects such as `list` and `set` handle self-reference by displaying "…" in the recursive part of the representation string.

To help write such `__repr__()` methods, the `reprlib` module has a new decorator, `@~reprlib.recursive_repr`, for detecting recursive calls to `__repr__()` and substituting a placeholder string instead:

```
>>> class MyList(list):
...     @recursive_repr()
...     def __repr__(self):
...         return '<' + '|'.join(map(repr, self)) + '>'
...
>>> m = MyList('abc')
>>> m.append(m)
>>> m.append('x')
```

```
>>> print(m)
<'a'|'b'|'c'|...|'x'>
```

(Contributed by Raymond Hettinger in bpo-9826 [https://bugs.python.org/issue?@action=redirect&bpo=9826] and bpo-9840 [https://bugs.python.org/issue?@action=redirect&bpo=9840].)

logging

In addition to dictionary-based configuration described above, the **`logging`** package has many other improvements.

The logging documentation has been augmented by a basic tutorial, an advanced tutorial, and a cookbook of logging recipes. These documents are the fastest way to learn about logging.

The **`logging.basicConfig()`** set-up function gained a *style* argument to support three different types of string formatting. It defaults to "%" for traditional %-formatting, can be set to "{" for the new **`str.format()`** style, or can be set to "$" for the shell-style formatting provided by **`string.Template`**. The following three configurations are equivalent:

```
>>> from logging import basicConfig
>>> basicConfig(style='%', format="%(name)s -> %(levelname)s: %(
>>> basicConfig(style='{', format="{name} -> {levelname} {messag
>>> basicConfig(style='$', format="$name -> $levelname: $message
```

If no configuration is set-up before a logging event occurs, there is now a default configuration using a **`StreamHandler`** directed to **`sys.stderr`** for events of `WARNING` level or higher. Formerly, an event occurring before a configuration was set-up would either raise an exception or silently drop the event depending on the value of **`logging.raiseExceptions`**. The new default handler is stored in **`logging.lastResort`**.

The use of filters has been simplified. Instead of creating a **`Filter`** object, the predicate can be any Python callable that returns `True` or `False`.

There were a number of other improvements that add flexibility and simplify configuration. See the module documentation for a full listing of changes in Python 3.2.

csv

The `csv` module now supports a new dialect, `unix_dialect`, which applies quoting for all fields and a traditional Unix style with `'\n'` as the line terminator. The registered dialect name is `unix`.

The `csv.DictWriter` has a new method, `writeheader()` for writing-out an initial row to document the field names:

```
>>> import csv, sys
>>> w = csv.DictWriter(sys.stdout, ['name', 'dept'], dialect='ur
>>> w.writeheader()
"name","dept"
>>> w.writerows([
...     {'name': 'tom', 'dept': 'accounting'},
...     {'name': 'susan', 'dept': 'Salesl'}])
"tom","accounting"
"susan","sales"
```

(New dialect suggested by Jay Talbot in bpo-5975 [https://bugs.python.org/issue?@action=redirect&bpo=5975], and the new method suggested by Ed Abraham in bpo-1537721 [https://bugs.python.org/issue?@action=redirect&bpo=1537721].)

contextlib

There is a new and slightly mind-blowing tool `ContextDecorator` that is helpful for creating a context manager that does double duty as a function decorator.

As a convenience, this new functionality is used by `@~contextlib.contextmanager` so that no extra effort is needed to support both roles.

The basic idea is that both context managers and function decorators can be used for pre-action and post-action wrappers. Context managers wrap a group of statements using a `with` statement, and function decorators wrap a group of statements enclosed in a function. So, occasionally there is a need to write a pre-action or post-action wrapper that can be used in either role.

For example, it is sometimes useful to wrap functions or groups of statements with a logger that can track the time of entry and time of exit. Rather than writing both a function decorator and a context manager for the task, the `@~contextlib.contextmanager` provides both capabilities in a single definition:

```
from contextlib import contextmanager
import logging

logging.basicConfig(level=logging.INFO)

@contextmanager
def track_entry_and_exit(name):
    logging.info('Entering: %s', name)
    yield
    logging.info('Exiting: %s', name)
```

Formerly, this would have only been usable as a context manager:

```
with track_entry_and_exit('widget loader'):
    print('Some time consuming activity goes here')
    load_widget()
```

Now, it can be used as a decorator as well:

```
@track_entry_and_exit('widget loader')
def activity():
    print('Some time consuming activity goes here')
    load_widget()
```

Trying to fulfill two roles at once places some limitations on the technique. Context managers normally have the flexibility to return an argument usable by a `with` statement, but there is no parallel for function decorators.

In the above example, there is not a clean way for the *track_entry_and_exit* context manager to return a logging instance for use in the body of enclosed statements.

(Contributed by Michael Foord in bpo-9110 [https://bugs.python.org/issue?@action=redirect&bpo=9110].)

decimal and fractions

Mark Dickinson crafted an elegant and efficient scheme for assuring that different numeric datatypes will have the same hash value whenever their actual values are equal (bpo-8188 [https://bugs.python.org/issue?@action=redirect&bpo=8188]):

```
assert hash(Fraction(3, 2)) == hash(1.5) == \
       hash(Decimal("1.5")) == hash(complex(1.5, 0))
```

Some of the hashing details are exposed through a new attribute, `sys.hash_info`, which describes the bit width of the hash value, the prime modulus, the hash values for *infinity* and *nan*, and the multiplier used for the imaginary part of a number:

```
>>> sys.hash_info
sys.hash_info(width=64, modulus=2305843009213693951, inf=314159,
```

An early decision to limit the interoperability of various numeric types has been relaxed. It is still unsupported (and ill-advised) to have implicit mixing in arithmetic expressions such as `Decimal('1.1') + float('1.1')` because the latter loses information in the process of constructing the binary float. However, since existing floating-point value can be converted losslessly to either a decimal or rational representation, it makes sense to add them to the constructor and to support mixed-type comparisons.

- The `decimal.Decimal` constructor now accepts `float` objects directly so there in no longer a need to use the `from_float()` method (bpo-8257 [https://bugs.python.org/issue?@action=redirect&bpo=8257]).
- Mixed type comparisons are now fully supported so that `Decimal` objects can be directly compared with `float` and `fractions.Fraction`

(bpo-2531 [https://bugs.python.org/issue?@action=redirect&bpo=2531] and bpo-8188 [https://bugs.python.org/issue?@action=redirect&bpo=8188]).

Similar changes were made to `fractions.Fraction` so that the `from_float()` and `from_decimal()` methods are no longer needed (bpo-8294 [https://bugs.python.org/issue?@action=redirect&bpo=8294]):

```
>>> from decimal import Decimal
>>> from fractions import Fraction
>>> Decimal(1.1)
Decimal('1.1000000000000000888178419700125232338905334472656 25')
>>> Fraction(1.1)
Fraction(2476979795053773, 2251799813685248)
```

Another useful change for the `decimal` module is that the `Context.clamp` attribute is now public. This is useful in creating contexts that correspond to the decimal interchange formats specified in IEEE 754 (see bpo-8540 [https://bugs.python.org/issue?@action=redirect&bpo=8540]).

(Contributed by Mark Dickinson and Raymond Hettinger.)

ftp

The `ftplib.FTP` class now supports the context management protocol to unconditionally consume `socket.error` exceptions and to close the FTP connection when done:

```
>>> from ftplib import FTP
>>> with FTP("ftp1.at.proftpd.org") as ftp:
        ftp.login()
        ftp.dir()

'230 Anonymous login ok, restrictions apply.'
dr-xr-xr-x   9 ftp      ftp           154 May  6 10:43 .
dr-xr-xr-x   9 ftp      ftp           154 May  6 10:43 ..
dr-xr-xr-x   5 ftp      ftp          4096 May  6 10:43 CentOS
dr-xr-xr-x   3 ftp      ftp            18 Jul 10  2008 Fedora
```

Other file-like objects such as `mmap.mmap` and `fileinput.input()` also grew auto-closing context managers:

```
with fileinput.input(files=('log1.txt', 'log2.txt')) as f:
    for line in f:
        process(line)
```

(Contributed by Tarek Ziadé and Giampaolo Rodolà in bpo-4972 [https://bugs.python.org/issue?@action=redirect&bpo=4972], and by Georg Brandl in bpo-8046 [https://bugs.python.org/issue?@action=redirect&bpo=8046] and bpo-1286 [https://bugs.python.org/issue?@action=redirect&bpo=1286].)

The `FTP_TLS` class now accepts a *context* parameter, which is a `ssl.SSLContext` object allowing bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure.

(Contributed by Giampaolo Rodolà; bpo-8806 [https://bugs.python.org/issue?@action=redirect&bpo=8806].)

popen

The `os.popen()` and `subprocess.Popen()` functions now support `with` statements for auto-closing of the file descriptors.

(Contributed by Antoine Pitrou and Brian Curtin in bpo-7461 [https://bugs.python.org/issue?@action=redirect&bpo=7461] and bpo-10554 [https://bugs.python.org/issue?@action=redirect&bpo=10554].)

select

The `select` module now exposes a new, constant attribute, `PIPE_BUF`, which gives the minimum number of bytes which are guaranteed not to block when `select.select()` says a pipe is ready for writing.

```
>>> import select
>>> select.PIPE_BUF
512
```

(Available on Unix systems. Patch by Sébastien Sablé in bpo-9862 [https://bugs.python.org/issue?@action=redirect&bpo=9862])

gzip and zipfile

gzip.GzipFile now implements the **io.BufferedIOBase** abstract base class (except for `truncate()`). It also has a **peek()** method and supports unseekable as well as zero-padded file objects.

The **gzip** module also gains the **compress()** and **decompress()** functions for easier in-memory compression and decompression. Keep in mind that text needs to be encoded as **bytes** before compressing and decompressing:

```
>>> import gzip
>>> s = 'Three shall be the number thou shalt count, '
>>> s += 'and the number of the counting shall be three'
>>> b = s.encode()                        # convert to utf-8
>>> len(b)
89
>>> c = gzip.compress(b)
>>> len(c)
77
>>> gzip.decompress(c).decode()[:42]      # decompress and conve
'Three shall be the number thou shalt count'
```

(Contributed by Anand B. Pillai in bpo-3488 [https://bugs.python.org/issue?@action=redirect&bpo=3488]; and by Antoine Pitrou, Nir Aides and Brian Curtin in bpo-9962 [https://bugs.python.org/issue?@action=redirect&bpo=9962], bpo-1675951 [https://bugs.python.org/issue?@action=redirect&bpo=1675951], bpo-7471 [https://bugs.python.org/issue?@action=redirect&bpo=7471] and bpo-2846 [https://bugs.python.org/issue?@action=redirect&bpo=2846].)

Also, the **zipfile.ZipExtFile** class was reworked internally to represent files stored inside an archive. The new implementation is significantly faster and can be wrapped in an **io.BufferedReader** object for more speedups. It also solves an issue where interleaved calls to *read* and *readline* gave the wrong results.

(Patch submitted by Nir Aides in bpo-7610 [https://bugs.python.org/issue?@action=redirect&bpo=7610].)

tarfile

The **`TarFile`** class can now be used as a context manager. In addition, its **`add()`** method has a new option, *filter*, that controls which files are added to the archive and allows the file metadata to be edited.

The new *filter* option replaces the older, less flexible *exclude* parameter which is now deprecated. If specified, the optional *filter* parameter needs to be a keyword argument. The user-supplied filter function accepts a **`TarInfo`** object and returns an updated **`TarInfo`** object, or if it wants the file to be excluded, the function can return `None`:

```
>>> import tarfile, glob

>>> def myfilter(tarinfo):
...     if tarinfo.isfile():             # only save real files
...         tarinfo.uname = 'monty'      # redact the user name
...         return tarinfo

>>> with tarfile.open(name='myarchive.tar.gz', mode='w:gz') as t
...     for filename in glob.glob('*.txt'):
...         tf.add(filename, filter=myfilter)
...     tf.list()
-rw-r--r-- monty/501        902 2011-01-26 17:59:11 annotations.
-rw-r--r-- monty/501        123 2011-01-26 17:59:11 general_ques
-rw-r--r-- monty/501       3514 2011-01-26 17:59:11 prion.txt
-rw-r--r-- monty/501        124 2011-01-26 17:59:11 py_todo.txt
-rw-r--r-- monty/501       1399 2011-01-26 17:59:11 semaphore_nc
```

(Proposed by Tarek Ziadé and implemented by Lars Gustäbel in bpo-6856 [https://bugs.python.org/issue?@action=redirect&bpo=6856].)

hashlib

The **`hashlib`** module has two new constant attributes listing the hashing algorithms guaranteed to be present in all implementations and those

available on the current implementation:

```
>>> import hashlib

>>> hashlib.algorithms_guaranteed
{'sha1', 'sha224', 'sha384', 'sha256', 'sha512', 'md5'}

>>> hashlib.algorithms_available
{'md2', 'SHA256', 'SHA512', 'dsaWithSHA', 'mdc2', 'SHA224', 'MD4
'sha512', 'ripemd160', 'SHA1', 'MDC2', 'SHA', 'SHA384', 'MD2',
'ecdsa-with-SHA1','md4', 'md5', 'sha1', 'DSA-SHA', 'sha224',
'dsaEncryption', 'DSA', 'RIPEMD160', 'sha', 'MD5', 'sha384'}
```

(Suggested by Carl Chenet in bpo-7418 [https://bugs.python.org/issue?@action=redirect&bpo=7418].)

ast

The **ast** module has a wonderful a general-purpose tool for safely evaluating expression strings using the Python literal syntax. The **ast.literal_eval()** function serves as a secure alternative to the builtin **eval()** function which is easily abused. Python 3.2 adds **bytes** and **set** literals to the list of supported types: strings, bytes, numbers, tuples, lists, dicts, sets, booleans, and `None`.

```
>>> from ast import literal_eval

>>> request = "{'req': 3, 'func': 'pow', 'args': (2, 0.5)}"
>>> literal_eval(request)
{'args': (2, 0.5), 'req': 3, 'func': 'pow'}

>>> request = "os.system('do something harmful')"
>>> literal_eval(request)
Traceback (most recent call last):
  ...
ValueError: malformed node or string: <_ast.Call object at 0x101
```

(Implemented by Benjamin Peterson and Georg Brandl.)

os

Different operating systems use various encodings for filenames and environment variables. The `os` module provides two new functions, `fsencode()` and `fsdecode()`, for encoding and decoding filenames:

```
>>> import os
>>> filename = 'Sehenswürdigkeiten'
>>> os.fsencode(filename)
b'Sehensw\xc3\xbcrdigkeiten'
```

Some operating systems allow direct access to encoded bytes in the environment. If so, the `os.supports_bytes_environ` constant will be true.

For direct access to encoded environment variables (if available), use the new `os.getenvb()` function or use `os.environb` which is a bytes version of `os.environ`.

(Contributed by Victor Stinner.)

shutil

The `shutil.copytree()` function has two new options:

- *ignore_dangling_symlinks*: when `symlinks=False` so that the function copies a file pointed to by a symlink, not the symlink itself. This option will silence the error raised if the file doesn't exist.
- *copy_function*: is a callable that will be used to copy files. `shutil.copy2()` is used by default.

(Contributed by Tarek Ziadé.)

In addition, the `shutil` module now supports archiving operations for zipfiles, uncompressed tarfiles, gzipped tarfiles, and bzipped tarfiles. And there are functions for registering additional archiving file formats (such as xz compressed tarfiles or custom formats).

The principal functions are `make_archive()` and `unpack_archive()`. By default, both operate on the current directory (which can be set by `os.chdir()`) and on any sub-directories. The archive filename needs to be specified with a full pathname. The archiving step is non-destructive (the original files are left unchanged).

```
>>> import shutil, pprint

>>> os.chdir('mydata')  # change to the source directory
>>> f = shutil.make_archive('/var/backup/mydata',
...                         'zip')      # archive the current di
>>> f                                   # show the name of archi
'/var/backup/mydata.zip'
>>> os.chdir('tmp')                     # change to an unpacking
>>> shutil.unpack_archive('/var/backup/mydata.zip')  # recover t

>>> pprint.pprint(shutil.get_archive_formats())  # display knowr
[('bztar', "bzip2'ed tar-file"),
 ('gztar', "gzip'ed tar-file"),
 ('tar', 'uncompressed tar file'),
 ('zip', 'ZIP file')]

>>> shutil.register_archive_format(     # register a new archive
...     name='xz',
...     function=xz.compress,           # callable archiving fur
...     extra_args=[('level', 8)],      # arguments to the funct
...     description='xz compression'
... )
```

(Contributed by Tarek Ziadé.)

sqlite3

The `sqlite3` module was updated to pysqlite version 2.6.0. It has two new capabilities.

- The **`sqlite3.Connection.in_transit`** attribute is true if there is an active transaction for uncommitted changes.
- The `sqlite3.Connection.enable_load_extension()` and `sqlite3.Connection.load_extension()` methods allows you to load

SQLite extensions from ".so" files. One well-known extension is the fulltext-search extension distributed with SQLite.

(Contributed by R. David Murray and Shashwat Anand; bpo-8845 [https://bugs.python.org/issue?@action=redirect&bpo=8845].)

html

A new `html` module was introduced with only a single function, `escape()`, which is used for escaping reserved characters from HTML markup:

```
>>> import html
>>> html.escape('x > 2 && x < 7')
'x &gt; 2 &amp;&amp; x &lt; 7'
```

socket

The `socket` module has two new improvements.

- Socket objects now have a `detach()` method which puts the socket into closed state without actually closing the underlying file descriptor. The latter can then be reused for other purposes. (Added by Antoine Pitrou; bpo-8524 [https://bugs.python.org/issue?@action=redirect&bpo=8524].)
- `socket.create_connection()` now supports the context management protocol to unconditionally consume `socket.error` exceptions and to close the socket when done. (Contributed by Giampaolo Rodolà; bpo-9794 [https://bugs.python.org/issue?@action=redirect&bpo=9794].)

ssl

The `ssl` module added a number of features to satisfy common requirements for secure (encrypted, authenticated) internet connections:

- A new class, `SSLContext`, serves as a container for persistent SSL data, such as protocol settings, certificates, private keys, and various other options. It includes a `wrap_socket()` for creating an SSL socket from an SSL context.

- A new function, **`ssl.match_hostname()`**, supports server identity verification for higher-level protocols by implementing the rules of HTTPS (from **RFC 2818** [https://datatracker.ietf.org/doc/html/rfc2818.html]) which are also suitable for other protocols.
- The **`ssl.wrap_socket()`** constructor function now takes a *ciphers* argument. The *ciphers* string lists the allowed encryption algorithms using the format described in the OpenSSL documentation [https://docs.openssl.org/1.0.2/man1/ciphers/#cipher-list-format].
- When linked against recent versions of OpenSSL, the **`ssl`** module now supports the Server Name Indication extension to the TLS protocol, allowing multiple "virtual hosts" using different certificates on a single IP port. This extension is only supported in client mode, and is activated by passing the *server_hostname* argument to **`ssl.SSLContext.wrap_socket()`**.
- Various options have been added to the **`ssl`** module, such as **`OP_NO_SSLv2`** which disables the insecure and obsolete SSLv2 protocol.
- The extension now loads all the OpenSSL ciphers and digest algorithms. If some SSL certificates cannot be verified, they are reported as an "unknown algorithm" error.
- The version of OpenSSL being used is now accessible using the module attributes **`ssl.OPENSSL_VERSION`** (a string), **`ssl.OPENSSL_VERSION_INFO`** (a 5-tuple), and **`ssl.OPENSSL_VERSION_NUMBER`** (an integer).

(Contributed by Antoine Pitrou in bpo-8850 [https://bugs.python.org/issue?@action=redirect&bpo=8850], bpo-1589 [https://bugs.python.org/issue?@action=redirect&bpo=1589], bpo-8322 [https://bugs.python.org/issue?@action=redirect&bpo=8322], bpo-5639 [https://bugs.python.org/issue?@action=redirect&bpo=5639], bpo-4870 [https://bugs.python.org/issue?@action=redirect&bpo=4870], bpo-8484 [https://bugs.python.org/issue?@action=redirect&bpo=8484], and bpo-8321 [https://bugs.python.org/issue?@action=redirect&bpo=8321].)

nntp

The **`nntplib`** module has a revamped implementation with better bytes and text semantics as well as more practical APIs. These improvements break compatibility with the nntplib version in Python 3.1, which was partly dysfunctional in itself.

Support for secure connections through both implicit (using **`nntplib.NNTP_SSL`**) and explicit (using **`nntplib.NNTP.starttls()`**) TLS has also been added.

(Contributed by Antoine Pitrou in bpo-9360 [https://bugs.python.org/issue?@action=redirect&bpo=9360] and Andrew Vant in bpo-1926 [https://bugs.python.org/issue?@action=redirect&bpo=1926].)

сертифікати

`http.client.HTTPSConnection`, **`urllib.request.HTTPSHandler`** and **`urllib.request.urlopen()`** now take optional arguments to allow for server certificate checking against a set of Certificate Authorities, as recommended in public uses of HTTPS.

(Added by Antoine Pitrou, bpo-9003 [https://bugs.python.org/issue?@action=redirect&bpo=9003].)

imaplib

Support for explicit TLS on standard IMAP4 connections has been added through the new **`imaplib.IMAP4.starttls`** method.

(Contributed by Lorenzo M. Catucci and Antoine Pitrou, bpo-4471 [https://bugs.python.org/issue?@action=redirect&bpo=4471].)

http.client

There were a number of small API improvements in the **`http.client`** module. The old-style HTTP 0.9 simple responses are no longer supported and the *strict* parameter is deprecated in all classes.

The `HTTPConnection` and `HTTPSConnection` classes now have a *source_address* parameter for a (host, port) tuple indicating where the HTTP connection is made from.

Support for certificate checking and HTTPS virtual hosts were added to `HTTPSConnection`.

The `request()` method on connection objects allowed an optional *body* argument so that a file object could be used to supply the content of the request. Conveniently, the *body* argument now also accepts an iterable object so long as it includes an explicit `Content-Length` header. This extended interface is much more flexible than before.

To establish an HTTPS connection through a proxy server, there is a new `set_tunnel()` method that sets the host and port for HTTP Connect tunneling.

To match the behavior of `http.server`, the HTTP client library now also encodes headers with ISO-8859-1 (Latin-1) encoding. It was already doing that for incoming headers, so now the behavior is consistent for both incoming and outgoing traffic. (See work by Armin Ronacher in bpo-10980 [https://bugs.python.org/issue?@action=redirect&bpo=10980].)

unittest

The unittest module has a number of improvements supporting test discovery for packages, easier experimentation at the interactive prompt, new testcase methods, improved diagnostic messages for test failures, and better method names.

- The command-line call `python -m unittest` can now accept file paths instead of module names for running specific tests (bpo-10620 [https://bugs.python.org/issue?@action=redirect&bpo=10620]). The new test discovery can find tests within packages, locating any test importable from the top-level directory. The top-level directory can be specified with the `-t` option, a pattern for matching files with `-p`, and a directory to start discovery with `-s`:

```
$ python -m unittest discover -s my_proj_dir -p _test.py
```

(Contributed by Michael Foord.)

- Experimentation at the interactive prompt is now easier because the `unittest.TestCase` class can now be instantiated without arguments:

```
>>> from unittest import TestCase
>>> TestCase().assertEqual(pow(2, 3), 8)
```

(Contributed by Michael Foord.)

- The `unittest` module has two new methods, `assertWarns()` and `assertWarnsRegex()` to verify that a given warning type is triggered by the code under test:

```
with self.assertWarns(DeprecationWarning):
    legacy_function('XYZ')
```

(Contributed by Antoine Pitrou, bpo-9754 [https://bugs.python.org/issue?@action=redirect&bpo=9754].)

Another new method, `assertCountEqual()` is used to compare two iterables to determine if their element counts are equal (whether the same elements are present with the same number of occurrences regardless of order):

```
def test_anagram(self):
    self.assertCountEqual('algorithm', 'logarithm')
```

(Contributed by Raymond Hettinger.)

- A principal feature of the unittest module is an effort to produce meaningful diagnostics when a test fails. When possible, the failure is recorded along with a diff of the output. This is especially helpful for analyzing log files of failed test runs. However, since diffs can sometime be voluminous, there is a new `maxDiff` attribute that sets maximum length of diffs displayed.

- In addition, the method names in the module have undergone a number of clean-ups.

  For example, `assertRegex()` is the new name for `assertRegexpMatches()` which was misnamed because the test uses `re.search()`, not `re.match()`. Other methods using regular expressions are now named using short form “Regex” in preference to “Regexp” – this matches the names used in other unittest implementations, matches Python’s old name for the `re` module, and it has unambiguous camel-casing.

  (Contributed by Raymond Hettinger and implemented by Ezio Melotti.)

- To improve consistency, some long-standing method aliases are being deprecated in favor of the preferred names:

Old Name	Preferred Name
`assert_()`	`assertTrue()`
`assertEquals()`	`assertEqual()`
`assertNotEquals()`	`assertNotEqual()`
`assertAlmostEquals()`	`assertAlmostEqual()`
`assertNotAlmostEquals()`	`assertNotAlmostEqual()`

  Likewise, the `TestCase.fail*` methods deprecated in Python 3.1 are expected to be removed in Python 3.3.

(Contributed by Ezio Melotti; bpo-9424 [https://bugs.python.org/issue?@action=redirect&bpo=9424].)

- The **`assertDictContainsSubset()`** method was deprecated because it was misimplemented with the arguments in the wrong order. This created hard-to-debug optical illusions where tests like `TestCase().assertDictContainsSubset({'a':1, 'b':2}, {'a':1})` would fail.

  (Contributed by Raymond Hettinger.)

random

The integer methods in the **`random`** module now do a better job of producing uniform distributions. Previously, they computed selections with `int(n*random())` which had a slight bias whenever *n* was not a power of two. Now, multiple selections are made from a range up to the next power of two and a selection is kept only when it falls within the range `0 <= x < n`. The functions and methods affected are **`randrange()`**, **`randint()`**, **`choice()`**, **`shuffle()`** and **`sample()`**.

(Contributed by Raymond Hettinger; bpo-9025 [https://bugs.python.org/issue?@action=redirect&bpo=9025].)

poplib

`POP3_SSL` class now accepts a *context* parameter, which is a **`ssl.SSLContext`** object allowing bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure.

(Contributed by Giampaolo Rodolà; bpo-8807 [https://bugs.python.org/issue?@action=redirect&bpo=8807].)

asyncore

`asyncore.dispatcher` now provides a **`handle_accepted()`** method returning a `(sock, addr)` pair which is called when a connection has

actually been established with a new remote endpoint. This is supposed to be used as a replacement for old **`handle_accept()`** and avoids the user to call **`accept()`** directly.

(Contributed by Giampaolo Rodolà; bpo-6706 [https://bugs.python.org/issue?@action=redirect&bpo=6706].)

tempfile

The **`tempfile`** module has a new context manager, **`TemporaryDirectory`** which provides easy deterministic cleanup of temporary directories:

```
with tempfile.TemporaryDirectory() as tmpdirname:
    print('created temporary dir:', tmpdirname)
```

(Contributed by Neil Schemenauer and Nick Coghlan; bpo-5178 [https://bugs.python.org/issue?@action=redirect&bpo=5178].)

inspect

- The **`inspect`** module has a new function **`getgeneratorstate()`** to easily identify the current state of a generator-iterator:

  ```
  >>> from inspect import getgeneratorstate
  >>> def gen():
  ...     yield 'demo'
  ...
  >>> g = gen()
  >>> getgeneratorstate(g)
  'GEN_CREATED'
  >>> next(g)
  'demo'
  >>> getgeneratorstate(g)
  'GEN_SUSPENDED'
  >>> next(g, None)
  >>> getgeneratorstate(g)
  'GEN_CLOSED'
  ```

  (Contributed by Rodolpho Eckhardt and Nick Coghlan, bpo-10220 [https://bugs.python.org/issue?@action=redirect&bpo=10220].)

- To support lookups without the possibility of activating a dynamic attribute, the **inspect** module has a new function, **getattr_static()**. Unlike **hasattr()**, this is a true read-only search, guaranteed not to change state while it is searching:

```
>>> class A:
...     @property
...     def f(self):
...         print('Running')
...         return 10
...
>>> a = A()
>>> getattr(a, 'f')
Running
10
>>> inspect.getattr_static(a, 'f')
<property object at 0x1022bd788>
```

(Contributed by Michael Foord.)

pydoc

The **pydoc** module now provides a much-improved web server interface, as well as a new command-line option `-b` to automatically open a browser window to display that server:

```
$ pydoc3.2 -b
```

(Contributed by Ron Adam; bpo-2001 [https://bugs.python.org/issue?@action=redirect&bpo=2001].)

dis

The **dis** module gained two new functions for inspecting code, **code_info()** and **show_code()**. Both provide detailed code object information for the supplied function, method, source code string or code object. The former returns a string and the latter prints it:

```
>>> import dis, random
>>> dis.show_code(random.choice)
Name:              choice
Filename:          /Library/Frameworks/Python.framework/Versions
Argument count:    2
Kw-only arguments: 0
Number of locals:  3
Stack size:        11
Flags:             OPTIMIZED, NEWLOCALS, NOFREE
Constants:
   0: 'Choose a random element from a non-empty sequence.'
   1: 'Cannot choose from an empty sequence'
Names:
   0: _randbelow
   1: len
   2: ValueError
   3: IndexError
Variable names:
   0: self
   1: seq
   2: i
```

In addition, the **`dis()`** function now accepts string arguments so that the common idiom `dis(compile(s, '', 'eval'))` can be shortened to `dis(s)`:

```
>>> dis('3*x+1 if x%2==1 else x//2')
  1           0 LOAD_NAME                0 (x)
              3 LOAD_CONST               0 (2)
              6 BINARY_MODULO
              7 LOAD_CONST               1 (1)
             10 COMPARE_OP               2 (==)
             13 POP_JUMP_IF_FALSE       28
             16 LOAD_CONST               2 (3)
             19 LOAD_NAME                0 (x)
             22 BINARY_MULTIPLY
             23 LOAD_CONST               1 (1)
             26 BINARY_ADD
             27 RETURN_VALUE
        >>   28 LOAD_NAME                0 (x)
             31 LOAD_CONST               0 (2)
```

```
              34 BINARY_FLOOR_DIVIDE
              35 RETURN_VALUE
```

Taken together, these improvements make it easier to explore how CPython is implemented and to see for yourself what the language syntax does under-the-hood.

(Contributed by Nick Coghlan in bpo-9147 [https://bugs.python.org/issue?@action=redirect&bpo=9147].)

dbm

All database modules now support the **`get()`** and **`setdefault()`** methods.

(Suggested by Ray Allen in bpo-9523 [https://bugs.python.org/issue?@action=redirect&bpo=9523].)

ctypes

A new type, **`ctypes.c_ssize_t`** represents the C **`ssize_t`** datatype.

site

The **`site`** module has three new functions useful for reporting on the details of a given Python installation.

- **`getsitepackages()`** lists all global site-packages directories.
- **`getuserbase()`** reports on the user's base directory where data can be stored.
- **`getusersitepackages()`** reveals the user-specific site-packages directory path.

```
>>> import site
>>> site.getsitepackages()
['/Library/Frameworks/Python.framework/Versions/3.2/lib/python3.
 '/Library/Frameworks/Python.framework/Versions/3.2/lib/site-pyt
 '/Library/Python/3.2/site-packages']
>>> site.getuserbase()
'/Users/raymondhettinger/Library/Python/3.2'
```

```
>>> site.getusersitepackages()
'/Users/raymondhettinger/Library/Python/3.2/lib/python/site-pack
```

Conveniently, some of site's functionality is accessible directly from the command-line:

```
$ python -m site --user-base
/Users/raymondhettinger/.local
$ python -m site --user-site
/Users/raymondhettinger/.local/lib/python3.2/site-packages
```

(Contributed by Tarek Ziadé in bpo-6693 [https://bugs.python.org/issue?@action=redirect&bpo=6693].)

sysconfig

The new `sysconfig` module makes it straightforward to discover installation paths and configuration variables that vary across platforms and installations.

The module offers access simple access functions for platform and version information:

- `get_platform()` returning values like *linux-i586* or *macosx-10.6-ppc*.
- `get_python_version()` returns a Python version string such as "3.2".

It also provides access to the paths and variables corresponding to one of seven named schemes used by `distutils`. Those include *posix_prefix*, *posix_home*, *posix_user*, *nt*, *nt_user*, *os2*, *os2_home*:

- `get_paths()` makes a dictionary containing installation paths for the current installation scheme.
- `get_config_vars()` returns a dictionary of platform specific variables.

There is also a convenient command-line interface:

```
C:\Python32>python -m sysconfig
Platform: "win32"
Python version: "3.2"
```

```
Current installation scheme: "nt"

Paths:
        data = "C:\Python32"
        include = "C:\Python32\Include"
        platinclude = "C:\Python32\Include"
        platlib = "C:\Python32\Lib\site-packages"
        platstdlib = "C:\Python32\Lib"
        purelib = "C:\Python32\Lib\site-packages"
        scripts = "C:\Python32\Scripts"
        stdlib = "C:\Python32\Lib"

Variables:
        BINDIR = "C:\Python32"
        BINLIBDEST = "C:\Python32\Lib"
        EXE = ".exe"
        INCLUDEPY = "C:\Python32\Include"
        LIBDEST = "C:\Python32\Lib"
        SO = ".pyd"
        VERSION = "32"
        abiflags = ""
        base = "C:\Python32"
        exec_prefix = "C:\Python32"
        platbase = "C:\Python32"
        prefix = "C:\Python32"
        projectbase = "C:\Python32"
        py_version = "3.2"
        py_version_nodot = "32"
        py_version_short = "3.2"
        srcdir = "C:\Python32"
        userbase = "C:\Documents and Settings\Raymond\Applicatic
```

(Moved out of Distutils by Tarek Ziadé.)

pdb

The **pdb** debugger module gained a number of usability improvements:

- `pdb.py` now has a `-c` option that executes commands as given in a `.pdbrc` script file.

- A `.pdbrc` script file can contain `continue` and `next` commands that continue debugging.
- The **`Pdb`** class constructor now accepts a *nosigint* argument.
- New commands: `l(list)`, `ll(long list)` and `source` for listing source code.
- New commands: `display` and `undisplay` for showing or hiding the value of an expression if it has changed.
- New command: `interact` for starting an interactive interpreter containing the global and local names found in the current scope.
- Breakpoints can be cleared by breakpoint number.

(Contributed by Georg Brandl, Antonio Cuni and Ilya Sandler.)

configparser

The **`configparser`** module was modified to improve usability and predictability of the default parser and its supported INI syntax. The old **`ConfigParser`** class was removed in favor of **`SafeConfigParser`** which has in turn been renamed to **`ConfigParser`**. Support for inline comments is now turned off by default and section or option duplicates are not allowed in a single configuration source.

Config parsers gained a new API based on the mapping protocol:

```
>>> parser = ConfigParser()
>>> parser.read_string("""
... [DEFAULT]
... location = upper left
... visible = yes
... editable = no
... color = blue
...
... [main]
... title = Main Menu
... color = green
...
... [options]
... title = Options
... """)
```

```
>>> parser['main']['color']
'green'
>>> parser['main']['editable']
'no'
>>> section = parser['options']
>>> section['title']
'Options'
>>> section['title'] = 'Options (editable: %(editable)s)'
>>> section['title']
'Options (editable: no)'
```

The new API is implemented on top of the classical API, so custom parser subclasses should be able to use it without modifications.

The INI file structure accepted by config parsers can now be customized. Users can specify alternative option/value delimiters and comment prefixes, change the name of the *DEFAULT* section or switch the interpolation syntax.

There is support for pluggable interpolation including an additional interpolation handler **`ExtendedInterpolation`**:

```
>>> parser = ConfigParser(interpolation=ExtendedInterpolation())
>>> parser.read_dict({'buildout': {'directory': '/home/ambv/zope
...                   'custom': {'prefix': '/usr/local'}})
>>> parser.read_string("""
... [buildout]
... parts =
...   zope9
...   instance
... find-links =
...   ${buildout:directory}/downloads/dist
...
... [zope9]
... recipe = plone.recipe.zope9install
... location = /opt/zope
...
... [instance]
... recipe = plone.recipe.zope9instance
... zope9-location = ${zope9:location}
... zope-conf = ${custom:prefix}/etc/zope.conf
... """)
```

```
>>> parser['buildout']['find-links']
'\n/home/ambv/zope9/downloads/dist'
>>> parser['instance']['zope-conf']
'/usr/local/etc/zope.conf'
>>> instance = parser['instance']
>>> instance['zope-conf']
'/usr/local/etc/zope.conf'
>>> instance['zope9-location']
'/opt/zope'
```

A number of smaller features were also introduced, like support for specifying encoding in read operations, specifying fallback values for get-functions, or reading directly from dictionaries and strings.

(All changes contributed by Łukasz Langa.)

urllib.parse

A number of usability improvements were made for the **`urllib.parse`** module.

The **`urlparse()`** function now supports IPv6 [https://en.wikipedia.org/wiki/IPv6] addresses as described in **RFC 2732** [https://datatracker.ietf.org/doc/html/rfc2732.html]:

```
>>> import urllib.parse
>>> urllib.parse.urlparse('http://[dead:beef:cafe:5417:affe:8FA3
ParseResult(scheme='http',
            netloc='[dead:beef:cafe:5417:affe:8FA3:deaf:feed]',
            path='/foo/',
            params='',
            query='',
            fragment='')
```

The **`urldefrag()`** function now returns a named tuple:

```
>>> r = urllib.parse.urldefrag('http://python.org/about/#target'
>>> r
DefragResult(url='http://python.org/about/', fragment='target')
>>> r[0]
```

```
'http://python.org/about/'
>>> r.fragment
'target'
```

And, the **urlencode()** function is now much more flexible, accepting either a string or bytes type for the *query* argument. If it is a string, then the *safe*, *encoding*, and *error* parameters are sent to **quote_plus()** for encoding:

```
>>> urllib.parse.urlencode([
...      ('type', 'telenovela'),
...      ('name', '¿Dónde Está Elisa?')],
...      encoding='latin-1')
'type=telenovela&name=%BFD%F3nde+Est%E1+Elisa%3F'
```

As detailed in Analisando bytes codificados em ASCII, all the **urllib.parse** functions now accept ASCII-encoded byte strings as input, so long as they are not mixed with regular strings. If ASCII-encoded byte strings are given as parameters, the return types will also be an ASCII-encoded byte strings:

```
>>> urllib.parse.urlparse(b'http://www.python.org:80/about/')
ParseResultBytes(scheme=b'http', netloc=b'www.python.org:80',
                 path=b'/about/', params=b'', query=b'', fragmer
```

(Work by Nick Coghlan, Dan Mahn, and Senthil Kumaran in bpo-2987 [https://bugs.python.org/issue?@action=redirect&bpo=2987], bpo-5468 [https://bugs.python.org/issue?@action=redirect&bpo=5468], and bpo-9873 [https://bugs.python.org/issue?@action=redirect&bpo=9873].)

mailbox

Thanks to a concerted effort by R. David Murray, the **mailbox** module has been fixed for Python 3.2. The challenge was that mailbox had been originally designed with a text interface, but email messages are best represented with **bytes** because various parts of a message may have different encodings.

The solution harnessed the `email` package's binary support for parsing arbitrary email messages. In addition, the solution required a number of API changes.

As expected, the `add()` method for `mailbox.Mailbox` objects now accepts binary input.

`StringIO` and text file input are deprecated. Also, string input will fail early if non-ASCII characters are used. Previously it would fail when the email was processed in a later step.

There is also support for binary output. The `get_file()` method now returns a file in the binary mode (where it used to incorrectly set the file to text-mode). There is also a new `get_bytes()` method that returns a `bytes` representation of a message corresponding to a given *key*.

It is still possible to get non-binary output using the old API's `get_string()` method, but that approach is not very useful. Instead, it is best to extract messages from a `Message` object or to load them from binary input.

(Contributed by R. David Murray, with efforts from Steffen Daode Nurpmeso and an initial patch by Victor Stinner in bpo-9124 [https://bugs.python.org/issue?@action=redirect&bpo=9124].)

turtledemo

The demonstration code for the `turtle` module was moved from the *Demo* directory to main library. It includes over a dozen sample scripts with lively displays. Being on `sys.path`, it can now be run directly from the command-line:

```
$ python -m turtledemo
```

(Moved from the Demo directory by Alexander Belopolsky in bpo-10199 [https://bugs.python.org/issue?@action=redirect&bpo=10199].)

Multi-threading

- The mechanism for serializing execution of concurrently running Python threads (generally known as the GIL or Global Interpreter Lock) has been rewritten. Among the objectives were more predictable switching intervals and reduced overhead due to lock contention and the number of ensuing system calls. The notion of a "check interval" to allow thread switches has been abandoned and replaced by an absolute duration expressed in seconds. This parameter is tunable through `sys.setswitchinterval()`. It currently defaults to 5 milliseconds.

  Additional details about the implementation can be read from a python-dev mailing-list message [https://mail.python.org/pipermail/python-dev/2009-October/093321.html] (however, "priority requests" as exposed in this message have not been kept for inclusion).

  (Contribuição de Antoine Pitrou.)

- Regular and recursive locks now accept an optional *timeout* argument to their `acquire()` method. (Contributed by Antoine Pitrou; bpo-7316 [https://bugs.python.org/issue?@action=redirect&bpo=7316].)

- Similarly, `threading.Semaphore.acquire()` also gained a *timeout* argument. (Contributed by Torsten Landschoff; bpo-850728 [https://bugs.python.org/issue?@action=redirect&bpo=850728].)

- Regular and recursive lock acquisitions can now be interrupted by signals on platforms using Pthreads. This means that Python programs that deadlock while acquiring locks can be successfully killed by repeatedly sending SIGINT to the process (by pressing `Ctrl+C` in most shells). (Contributed by Reid Kleckner; bpo-8844 [https://bugs.python.org/issue?@action=redirect&bpo=8844].)

Otimizações

A number of small performance enhancements have been added:

- Python's peephole optimizer now recognizes patterns such `x in {1, 2, 3}` as being a test for membership in a set of constants. The

optimizer recasts the **set** as a **frozenset** and stores the pre-built constant.

Now that the speed penalty is gone, it is practical to start writing membership tests using set-notation. This style is both semantically clear and operationally fast:

```
extension = name.rpartition('.')[2]
if extension in {'xml', 'html', 'xhtml', 'css'}:
    handle(name)
```

(Patch and additional tests contributed by Dave Malcolm; bpo-6690 [https://bugs.python.org/issue?@action=redirect&bpo=6690]).

- Serializing and unserializing data using the **pickle** module is now several times faster.

  (Contributed by Alexandre Vassalotti, Antoine Pitrou and the Unladen Swallow team in bpo-9410 [https://bugs.python.org/issue?@action=redirect&bpo=9410] and bpo-3873 [https://bugs.python.org/issue?@action=redirect&bpo=3873].)

- The Timsort algorithm [https://en.wikipedia.org/wiki/Timsort] used in **list.sort()** and **sorted()** now runs faster and uses less memory when called with a key function. Previously, every element of a list was wrapped with a temporary object that remembered the key value associated with each element. Now, two arrays of keys and values are sorted in parallel. This saves the memory consumed by the sort wrappers, and it saves time lost to delegating comparisons.

  (Patch by Daniel Stutzbach in bpo-9915 [https://bugs.python.org/issue?@action=redirect&bpo=9915].)

- JSON decoding performance is improved and memory consumption is reduced whenever the same string is repeated for multiple keys. Also, JSON encoding now uses the C speedups when the `sort_keys` argument is true.

(Contributed by Antoine Pitrou in bpo-7451 [https://bugs.python.org/issue?@action=redirect&bpo=7451] and by Raymond Hettinger and Antoine Pitrou in bpo-10314 [https://bugs.python.org/issue?@action=redirect&bpo=10314].)

- Recursive locks (created with the `threading.RLock()` API) now benefit from a C implementation which makes them as fast as regular locks, and between 10x and 15x faster than their previous pure Python implementation.

  (Contributed by Antoine Pitrou; bpo-3001 [https://bugs.python.org/issue?@action=redirect&bpo=3001].)

- The fast-search algorithm in stringlib is now used by the `split()`, `rsplit()`, `splitlines()` and `replace()` methods on `bytes`, `bytearray` and `str` objects. Likewise, the algorithm is also used by `rfind()`, `rindex()`, `rsplit()` and `rpartition()`.

  (Patch by Florent Xicluna in bpo-7622 [https://bugs.python.org/issue?@action=redirect&bpo=7622] and bpo-7462 [https://bugs.python.org/issue?@action=redirect&bpo=7462].)

- Integer to string conversions now work two “digits” at a time, reducing the number of division and modulo operations.

  (bpo-6713 [https://bugs.python.org/issue?@action=redirect&bpo=6713] by Gawain Bolton, Mark Dickinson, and Victor Stinner.)

There were several other minor optimizations. Set differencing now runs faster when one operand is much larger than the other (patch by Andress Bennetts in bpo-8685 [https://bugs.python.org/issue?@action=redirect&bpo=8685]). The `array.repeat()` method has a faster implementation (bpo-1569291 [https://bugs.python.org/issue?@action=redirect&bpo=1569291] by Alexander Belopolsky). The `BaseHTTPRequestHandler` has more efficient buffering (bpo-3709 [https://bugs.python.org/issue?@action=redirect&bpo=3709] by Andrew Schaaf). The `operator.attrgetter()` function has been sped-up (bpo-10160 [https://bugs.python.org/issue?@action=redirect&bpo=10160] by Christos Georgiou). And `ConfigParser` loads multi-line arguments a bit faster (bpo-7113 [https://bugs.python.org/issue?@action=redirect&bpo=7113] by Łukasz Langa).

Unicode

Python has been updated to Unicode 6.0.0 [https://unicode.org/versions/Unicode6.0.0/]. The update to the standard adds over 2,000 new characters including emoji [https://en.wikipedia.org/wiki/Emoji] symbols which are important for mobile phones.

In addition, the updated standard has altered the character properties for two Kannada characters (U+0CF1, U+0CF2) and one New Tai Lue numeric character (U+19DA), making the former eligible for use in identifiers while disqualifying the latter. For more information, see Unicode Character Database Changes [https://www.unicode.org/versions/Unicode6.0.0/#Database_Changes].

Codecs

Support was added for *cp720* Arabic DOS encoding (bpo-1616979 [https://bugs.python.org/issue?@action=redirect&bpo=1616979]).

MBCS encoding no longer ignores the error handler argument. In the default strict mode, it raises an **`UnicodeDecodeError`** when it encounters an undecodable byte sequence and an **`UnicodeEncodeError`** for an unencodable character.

The MBCS codec supports `'strict'` and `'ignore'` error handlers for decoding, and `'strict'` and `'replace'` for encoding.

To emulate Python3.1 MBCS encoding, select the `'ignore'` handler for decoding and the `'replace'` handler for encoding.

On Mac OS X, Python decodes command line arguments with `'utf-8'` rather than the locale encoding.

By default, **`tarfile`** uses `'utf-8'` encoding on Windows (instead of `'mbcs'`) and the `'surrogateescape'` error handler on all operating systems.

Documentação

The documentation continues to be improved.

- A table of quick links has been added to the top of lengthy sections such as Funções embutidas. In the case of **`itertools`**, the links are accompanied by tables of cheatsheet-style summaries to provide an overview and memory jog without having to read all of the docs.

- In some cases, the pure Python source code can be a helpful adjunct to the documentation, so now many modules now feature quick links to the latest version of the source code. For example, the **`functools`** module documentation has a quick link at the top labeled:

  > **Source code** Lib/functools.py [https://github.com/python/cpython/tree/3.15/Lib/functools.py].

  (Contributed by Raymond Hettinger; see rationale [https://rhettinger.wordpress.com/2011/01/28/open-your-source-more/].)

- The docs now contain more examples and recipes. In particular, **`re`** module has an extensive section, Regular expression examples. Likewise, the **`itertools`** module continues to be updated with new Receitas com itertools.

- The **`datetime`** module now has an auxiliary implementation in pure Python. No functionality was changed. This just provides an easier-to-read alternate implementation.

  (Contributed by Alexander Belopolsky in bpo-9528 [https://bugs.python.org/issue?@action=redirect&bpo=9528].)

- The unmaintained `Demo` directory has been removed. Some demos were integrated into the documentation, some were moved to the `Tools/demo` directory, and others were removed altogether.

  (Contributed by Georg Brandl in bpo-7962 [https://bugs.python.org/issue?@action=redirect&bpo=7962].)

IDLE

- The format menu now has an option to clean source files by stripping trailing whitespace.

  (Contributed by Raymond Hettinger; bpo-5150 [https://bugs.python.org/issue?@action=redirect&bpo=5150].)

- IDLE on Mac OS X now works with both Carbon AquaTk and Cocoa AquaTk.

  (Contributed by Kevin Walzer, Ned Deily, and Ronald Oussoren; bpo-6075 [https://bugs.python.org/issue?@action=redirect&bpo=6075].)

Code Repository

In addition to the existing Subversion code repository at https://svn.python.org there is now a Mercurial [https://www.mercurial-scm.org/] repository at https://hg.python.org/.

After the 3.2 release, there are plans to switch to Mercurial as the primary repository. This distributed version control system should make it easier for members of the community to create and share external changesets. See **PEP 385** [https://peps.python.org/pep-0385/] for details.

To learn to use the new version control system, see the Quick Start [https://www.mercurial-scm.org/wiki/QuickStart] or the Guide to Mercurial Workflows [https://www.mercurial-scm.org/guide].

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- The *idle*, *pydoc* and *2to3* scripts are now installed with a version-specific suffix on `make altinstall` (bpo-10679 [https://bugs.python.org/issue?@action=redirect&bpo=10679]).

- The C functions that access the Unicode Database now accept and return characters from the full Unicode range, even on narrow unicode

builds (Py_UNICODE_TOLOWER, Py_UNICODE_ISDECIMAL, and others). A visible difference in Python is that **`unicodedata.numeric()`** now returns the correct value for large code points, and **`repr()`** may consider more characters as printable.

(Reported by Bupjoe Lee and fixed by Amaury Forgeot D'Arc; bpo-5127 [https://bugs.python.org/issue?@action=redirect&bpo=5127].)

- Computed gotos are now enabled by default on supported compilers (which are detected by the configure script). They can still be disabled selectively by specifying `--without-computed-gotos`.

  (Contributed by Antoine Pitrou; bpo-9203 [https://bugs.python.org/issue?@action=redirect&bpo=9203].)

- The option `--with-wctype-functions` was removed. The built-in unicode database is now used for all functions.

  (Contributed by Amaury Forgeot D'Arc; bpo-9210 [https://bugs.python.org/issue?@action=redirect&bpo=9210].)

- Hash values are now values of a new type, **`Py_hash_t`**, which is defined to be the same size as a pointer. Previously they were of type long, which on some 64-bit operating systems is still only 32 bits long. As a result of this fix, **`set`** and **`dict`** can now hold more than `2**32` entries on builds with 64-bit pointers (previously, they could grow to that size but their performance degraded catastrophically).

  (Suggested by Raymond Hettinger and implemented by Benjamin Peterson; bpo-9778 [https://bugs.python.org/issue?@action=redirect&bpo=9778].)

- A new macro **`Py_VA_COPY`** copies the state of the variable argument list. It is equivalent to C99 *va_copy* but available on all Python platforms (bpo-2443 [https://bugs.python.org/issue?@action=redirect&bpo=2443]).

- A new C API function **`PySys_SetArgvEx()`** allows an embedded interpreter to set **`sys.argv`** without also modifying **`sys.path`** (bpo-5753 [https://bugs.python.org/issue?@action=redirect&bpo=5753]).

- **`PyEval_CallObject()`** is now only available in macro form. The function declaration, which was kept for backwards compatibility reasons, is now removed – the macro was introduced in 1997 (bpo-8276 [https://bugs.python.org/issue?@action=redirect&bpo=8276]).

- There is a new function **`PyLong_AsLongLongAndOverflow()`** which is analogous to **`PyLong_AsLongAndOverflow()`**. They both serve to convert Python **`int`** into a native fixed-width type while providing detection of cases where the conversion won't fit (bpo-7767 [https://bugs.python.org/issue?@action=redirect&bpo=7767]).

- The **`PyUnicode_CompareWithASCIIString()`** function now returns *not equal* if the Python string is *NUL* terminated.

- There is a new function **`PyErr_NewExceptionWithDoc()`** that is like **`PyErr_NewException()`** but allows a docstring to be specified. This lets C exceptions have the same self-documenting capabilities as their pure Python counterparts (bpo-7033 [https://bugs.python.org/issue?@action=redirect&bpo=7033]).

- When compiled with the `--with-valgrind` option, the pymalloc allocator will be automatically disabled when running under Valgrind. This gives improved memory leak detection when running under Valgrind, while taking advantage of pymalloc at other times (bpo-2422 [https://bugs.python.org/issue?@action=redirect&bpo=2422]).

- Removed the `O?` format from the *PyArg_Parse* functions. The format is no longer used and it had never been documented (bpo-8837 [https://bugs.python.org/issue?@action=redirect&bpo=8837]).

There were a number of other small changes to the C-API. See the Misc/NEWS [https://github.com/python/cpython/blob/v3.2.6/Misc/NEWS] file for a complete list.

Also, there were a number of updates to the Mac OS X build, see Mac/BuildScript/README.txt [https://github.com/python/cpython/blob/v3.2.6/Mac/BuildScript/README.txt] for details. For users running a 32/64-bit build, there is a known problem with the default

Tcl/Tk on Mac OS X 10.6. Accordingly, we recommend installing an updated alternative such as ActiveState Tcl/Tk 8.5.9 [https://web.archive.org/web/20101208191259/https://www.activestate.com/activetcl/downloads]. See https://www.python.org/download/mac/tcltk/ for additional details.

Porting to Python 3.2

Esta seção lista as alterações descritas anteriormente e outras correções que podem exigir alterações no seu código.

- The `configparser` module has a number of clean-ups. The major change is to replace the old `ConfigParser` class with long-standing preferred alternative `SafeConfigParser`. In addition there are a number of smaller incompatibilities:
    - The interpolation syntax is now validated on `get()` and `set()` operations. In the default interpolation scheme, only two tokens with percent signs are valid: `%(name)s` and `%%`, the latter being an escaped percent sign.
    - The `set()` and `add_section()` methods now verify that values are actual strings. Formerly, unsupported types could be introduced unintentionally.
    - Duplicate sections or options from a single source now raise either `DuplicateSectionError` or `DuplicateOptionError`. Formerly, duplicates would silently overwrite a previous entry.
    - Inline comments are now disabled by default so now the `;` character can be safely used in values.
    - Comments now can be indented. Consequently, for `;` or `#` to appear at the start of a line in multiline values, it has to be interpolated. This keeps comment prefix characters in values from being mistaken as comments.
    - `""` is now a valid value and is no longer automatically converted to an empty string. For empty strings, use `"option =" in a line.`

- The `nntplib` module was reworked extensively, meaning that its APIs are often incompatible with the 3.1 APIs.

- **bytearray** objects can no longer be used as filenames; instead, they should be converted to **bytes**.

- The **array.tostring()** and **array.fromstring()** have been renamed to **array.tobytes()** and **array.frombytes()** for clarity. The old names have been deprecated. (See bpo-8990 [https://bugs.python.org/issue?@action=redirect&bpo=8990].)

- `PyArg_Parse*()` functions:

  - "t#" format has been removed: use "s#" or "s*" instead
  - "w" and "w#" formats has been removed: use "w*" instead

- The **PyCObject** type, deprecated in 3.1, has been removed. To wrap opaque C pointers in Python objects, the **PyCapsule** API should be used instead; the new type has a well-defined interface for passing typing safety information and a less complicated signature for calling a destructor.

- The **sys.setfilesystemencoding()** function was removed because it had a flawed design.

- The **random.seed()** function and method now salt string seeds with an sha512 hash function. To access the previous version of *seed* in order to reproduce Python 3.1 sequences, set the *version* argument to *1*, `random.seed(s, version=1)`.

- The previously deprecated **string.maketrans()** function has been removed in favor of the static methods **bytes.maketrans()** and **bytearray.maketrans()**. This change solves the confusion around which types were supported by the **string** module. Now, **str**, **bytes**, and **bytearray** each have their own **maketrans** and **translate** methods with intermediate translation tables of the appropriate type.

  (Contributed by Georg Brandl; bpo-5675 [https://bugs.python.org/issue?@action=redirect&bpo=5675].)

- The previously deprecated **`contextlib.nested()`** function has been removed in favor of a plain **`with`** statement which can accept multiple context managers. The latter technique is faster (because it is built-in), and it does a better job finalizing multiple context managers when one of them raises an exception:

  ```
  with open('mylog.txt') as infile, open('a.out', 'w') as outf
      for line in infile:
          if '<critical>' in line:
              outfile.write(line)
  ```

  (Contributed by Georg Brandl and Mattias Brändström; appspot issue 53094 [https://codereview.appspot.com/53094].)

- **`struct.pack()`** now only allows bytes for the `s` string pack code. Formerly, it would accept text arguments and implicitly encode them to bytes using UTF-8. This was problematic because it made assumptions about the correct encoding and because a variable-length encoding can fail when writing to fixed length segment of a structure.

  Code such as `struct.pack('<6sHHBBB', 'GIF87a', x, y)` should be rewritten with to use bytes instead of text, `struct.pack('<6sHHBBB', b'GIF87a', x, y)`.

  (Discovered by David Beazley and fixed by Victor Stinner; bpo-10783 [https://bugs.python.org/issue?@action=redirect&bpo=10783].)

- The **`xml.etree.ElementTree`** class now raises an **`xml.etree.ElementTree.ParseError`** when a parse fails. Previously it raised an **`xml.parsers.expat.ExpatError`**.

- The new, longer **`str()`** value on floats may break doctests which rely on the old output format.

- In **`subprocess.Popen`**, the default value for *close_fds* is now `True` under Unix; under Windows, it is `True` if the three standard streams are set to `None`, `False` otherwise. Previously, *close_fds* was always `False`

by default, which produced difficult to solve bugs or race conditions when open file descriptors would leak into the child process.

- Support for legacy HTTP 0.9 has been removed from `urllib.request` and `http.client`. Such support is still present on the server side (in `http.server`).

  (Contributed by Antoine Pitrou, bpo-10711 [https://bugs.python.org/issue?@action=redirect&bpo=10711].)

- SSL sockets in timeout mode now raise `socket.timeout` when a timeout occurs, rather than a generic `SSLError`.

  (Contributed by Antoine Pitrou, bpo-10272 [https://bugs.python.org/issue?@action=redirect&bpo=10272].)

- The misleading functions `PyEval_AcquireLock()` and `PyEval_ReleaseLock()` have been officially deprecated. The thread-state aware APIs (such as `PyEval_SaveThread()` and `PyEval_RestoreThread()`) should be used instead.

- Due to security risks, `asyncore.handle_accept()` has been deprecated, and a new function, `asyncore.handle_accepted()`, was added to replace it.

  (Contributed by Giampaolo Rodola in bpo-6706 [https://bugs.python.org/issue?@action=redirect&bpo=6706].)

- Due to the new GIL implementation, `PyEval_InitThreads()` cannot be called before `Py_Initialize()` anymore.

O que há de novo no Python 3.1

Autor:: Raymond Hettinger

Esse artigo explica os novos recursos em Python 3.1, comparado ao 3.0. Python 3.1 foi lançado em 27 de junho de 2009.

PEP 372: Dicionários ordenados

Regular Python dictionaries iterate over key/value pairs in arbitrary order. Over the years, a number of authors have written alternative implementations that remember the order that the keys were originally inserted. Based on the experiences from those implementations, a new `collections.OrderedDict` class has been introduced.

The OrderedDict API is substantially the same as regular dictionaries but will iterate over keys and values in a guaranteed order depending on when a key was first inserted. If a new entry overwrites an existing entry, the original insertion position is left unchanged. Deleting an entry and reinserting it will move it to the end.

A biblioteca padrão agora possui suporte para dicionários ordenados em vários módulos. O módulo `configparser` usa-os por padrão. Isso permite que os arquivos de configuração sejam lidos, modificados e depois gravados de volta na ordem original. O método *_asdict()* para `collections.namedtuple()` agora retorna um dicionário ordenado com os valores aparecendo na mesma ordem que os índices de tupla subjacentes. O módulo `json` está sendo construído com um *object_pairs_hook* para permitir que OrderedDicts sejam construídos pelo decodificador. Também foi adicionado suporte para ferramentas de terceiros como PyYAML [https://pyyaml.org/].

Ver também

PEP 372 [https://peps.python.org/pep-0372/] - Dicionários ordenados

PEP written by Armin Ronacher and Raymond Hettinger.
Implementation written by Raymond Hettinger.

Uma vez que dicionários ordenados lembram sua ordem de inserção, isso pode ser usada em conjunto com classificação para criar um dicionário classificado:

```
>>> # regular unsorted dictionary
>>> d = {'banana': 3, 'apple':4, 'pear': 1, 'orange': 2}

>>> # dictionary sorted by key
>>> OrderedDict(sorted(d.items(), key=lambda t: t[0]))
OrderedDict([('apple', 4), ('banana', 3), ('orange', 2), ('pear'

>>> # dictionary sorted by value
>>> OrderedDict(sorted(d.items(), key=lambda t: t[1]))
OrderedDict([('pear', 1), ('orange', 2), ('banana', 3), ('apple'

>>> # dictionary sorted by length of the key string
>>> OrderedDict(sorted(d.items(), key=lambda t: len(t[0])))
OrderedDict([('pear', 1), ('apple', 4), ('orange', 2), ('banana'
```

Os novos dicionários classificados mantem suas ordem de classificação quando entradas são removidas. Mas quando novas chaves são adicionadas, as chaves são adicionados ao fim e a classificação não é mantida.

PEP 378: Especificador de formato para separador de milhares

The built-in `format()` function and the `str.format()` method use a mini-language that now includes a simple, non-locale aware way to format a number with a thousands separator. That provides a way to humanize a program's output, improving its professional appearance and readability:

```
>>> format(1234567, ',d')
'1,234,567'
>>> format(1234567.89, ',.2f')
```

```
'1,234,567.89'
>>> format(12345.6 + 8901234.12j, ',f')
'12,345.600000+8,901,234.120000j'
>>> format(Decimal('1234567.89'), ',f')
'1,234,567.89'
```

The supported types are `int`, `float`, `complex` and `decimal.Decimal`.

Discussions are underway about how to specify alternative separators like dots, spaces, apostrophes, or underscores. Locale-aware applications should use the existing *n* format specifier which already has some support for thousands separators.

Ver também

PEP 378 [https://peps.python.org/pep-0378/] - Especificador de formato para separador de milhares

PEP escrita por Raymond Hettinger e implementada por Eric Smith e Mark Dickinson.

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- Directories and zip archives containing a `__main__.py` file can now be executed directly by passing their name to the interpreter. The directory/zipfile is automatically inserted as the first entry in sys.path. (Suggestion and initial patch by Andy Chu; revised patch by Phillip J. Eby and Nick Coghlan; bpo-1739468 [https://bugs.python.org/issue?@action=redirect&bpo=1739468].)

- The `int()` type gained a `bit_length` method that returns the number of bits necessary to represent its argument in binary:

  ```
  >>> n = 37
  >>> bin(37)
  '0b100101'
  ```

```
>>> n.bit_length()
6
>>> n = 2**123-1
>>> n.bit_length()
123
>>> (n+1).bit_length()
124
```

(Contributed by Fredrik Johansson, Victor Stinner, Raymond Hettinger, and Mark Dickinson; bpo-3439 [https://bugs.python.org/issue?@action=redirect&bpo=3439].)

- The fields in **format()** strings can now be automatically numbered:

```
>>> 'Sir {} of {}'.format('Gallahad', 'Camelot')
'Sir Gallahad of Camelot'
```

Formerly, the string would have required numbered fields such as: `'Sir {0} of {1}'`.

(Contributed by Eric Smith; bpo-5237 [https://bugs.python.org/issue?@action=redirect&bpo=5237].)

- The **string.maketrans()** function is deprecated and is replaced by new static methods, **bytes.maketrans()** and **bytearray.maketrans()**. This change solves the confusion around which types were supported by the **string** module. Now, **str**, **bytes**, and **bytearray** each have their own **maketrans** and **translate** methods with intermediate translation tables of the appropriate type.

(Contributed by Georg Brandl; bpo-5675 [https://bugs.python.org/issue?@action=redirect&bpo=5675].)

- The syntax of the **with** statement now allows multiple context managers in a single statement:

```
>>> with open('mylog.txt') as infile, open('a.out', 'w') as
...     for line in infile:
...         if '<critical>' in line:
...             outfile.write(line)
```

With the new syntax, the **`contextlib.nested()`** function is no longer needed and is now deprecated.

(Contributed by Georg Brandl and Mattias Brändström; appspot issue 53094 [https://codereview.appspot.com/53094].)

- `round(x, n)` now returns an integer if *x* is an integer. Previously it returned a float:

  ```
  >>> round(1123, -2)
  1100
  ```

  (Contributed by Mark Dickinson; bpo-4707 [https://bugs.python.org/issue?@action=redirect&bpo=4707].)

- Python now uses David Gay's algorithm for finding the shortest floating-point representation that doesn't change its value. This should help mitigate some of the confusion surrounding binary floating-point numbers.

  The significance is easily seen with a number like `1.1` which does not have an exact equivalent in binary floating point. Since there is no exact equivalent, an expression like `float('1.1')` evaluates to the nearest representable value which is `0x1.199999999999ap+0` in hex or `1.1000000000000000881784197001252323389053344726562 5` in decimal. That nearest value was and still is used in subsequent floating-point calculations.

  What is new is how the number gets displayed. Formerly, Python used a simple approach. The value of `repr(1.1)` was computed as `format(1.1, '.17g')` which evaluated to `'1.1000000000000001'`. The advantage of using 17 digits was that it relied on IEEE-754 guarantees to assure that `eval(repr(1.1))` would round-trip exactly to its original value. The disadvantage is that many people found the output to be confusing (mistaking intrinsic limitations of binary floating-point representation as being a problem with Python itself).

The new algorithm for `repr(1.1)` is smarter and returns `'1.1'`. Effectively, it searches all equivalent string representations (ones that get stored with the same underlying float value) and returns the shortest representation.

The new algorithm tends to emit cleaner representations when possible, but it does not change the underlying values. So, it is still the case that `1.1 + 2.2 != 3.3` even though the representations may suggest otherwise.

The new algorithm depends on certain features in the underlying floating-point implementation. If the required features are not found, the old algorithm will continue to be used. Also, the text pickle protocols assure cross-platform portability by using the old algorithm.

(Contributed by Eric Smith and Mark Dickinson; bpo-1580 [https://bugs.python.org/issue?@action=redirect&bpo=1580])

New, Improved, and Deprecated Modules

- Added a **`collections.Counter`** class to support convenient counting of unique items in a sequence or iterable:

```
>>> Counter(['red', 'blue', 'red', 'green', 'blue', 'blue'])
Counter({'blue': 3, 'red': 2, 'green': 1})
```

  (Contributed by Raymond Hettinger; bpo-1696199 [https://bugs.python.org/issue?@action=redirect&bpo=1696199].)

- Added a new module, **`tkinter.ttk`** for access to the Tk themed widget set. The basic idea of ttk is to separate, to the extent possible, the code implementing a widget's behavior from the code implementing its appearance.

  (Contributed by Guilherme Polo; bpo-2983 [https://bugs.python.org/issue?@action=redirect&bpo=2983].)

- The `gzip.GzipFile` and `bz2.BZ2File` classes now support the context management protocol:

  ```
  >>> # Automatically close file after writing
  >>> with gzip.GzipFile(filename, "wb") as f:
  ...     f.write(b"xxx")
  ```

  (Contribuição de Antoine Pitrou.)

- The `decimal` module now supports methods for creating a decimal object from a binary `float`. The conversion is exact but can sometimes be surprising:

  ```
  >>> Decimal.from_float(1.1)
  Decimal('1.100000000000000088817841970012523233890533447265
  ```

  The long decimal result shows the actual binary fraction being stored for *1.1*. The fraction has many digits because *1.1* cannot be exactly represented in binary.

  (Contributed by Raymond Hettinger and Mark Dickinson.)

- The `itertools` module grew two new functions. The `itertools.combinations_with_replacement()` function is one of four for generating combinatorics including permutations and Cartesian products. The `itertools.compress()` function mimics its namesake from APL. Also, the existing `itertools.count()` function now has an optional *step* argument and can accept any type of counting sequence including `fractions.Fraction` and `decimal.Decimal`:

  ```
  >>> [p+q for p,q in combinations_with_replacement('LOVE', 2
  ['LL', 'LO', 'LV', 'LE', 'OO', 'OV', 'OE', 'VV', 'VE', 'EE'

  >>> list(compress(data=range(10), selectors=[0,0,1,1,0,1,0,
  [2, 3, 5, 7]

  >>> c = count(start=Fraction(1,2), step=Fraction(1,6))
  >>> [next(c), next(c), next(c), next(c)]
  [Fraction(1, 2), Fraction(2, 3), Fraction(5, 6), Fraction(1,
  ```

(Contributed by Raymond Hettinger.)

- `collections.namedtuple()` now supports a keyword argument *rename* which lets invalid fieldnames be automatically converted to positional names in the form _0, _1, etc. This is useful when the field names are being created by an external source such as a CSV header, SQL field list, or user input:

```
>>> query = input()
SELECT region, dept, count(*) FROM main GROUPBY region, dept

>>> cursor.execute(query)
>>> query_fields = [desc[0] for desc in cursor.description]
>>> UserQuery = namedtuple('UserQuery', query_fields, rename
>>> pprint.pprint([UserQuery(*row) for row in cursor])
[UserQuery(region='South', dept='Shipping', _2=185),
 UserQuery(region='North', dept='Accounting', _2=37),
 UserQuery(region='West', dept='Sales', _2=419)]
```

(Contributed by Raymond Hettinger; bpo-1818 [https://bugs.python.org/issue?@action=redirect&bpo=1818].)

- The `re.sub()`, `re.subn()` and `re.split()` functions now accept a flags parameter.

(Contributed by Gregory Smith.)

- The `logging` module now implements a simple `logging.NullHandler` class for applications that are not using logging but are calling library code that does. Setting-up a null handler will suppress spurious warnings such as “No handlers could be found for logger foo”:

```
>>> h = logging.NullHandler()
>>> logging.getLogger("foo").addHandler(h)
```

(Contributed by Vinay Sajip; bpo-4384 [https://bugs.python.org/issue?@action=redirect&bpo=4384]).

- The **runpy** module which supports the `-m` command line switch now supports the execution of packages by looking for and executing a `__main__` submodule when a package name is supplied.

  (Contributed by Andi Vajda; bpo-4195 [https://bugs.python.org/issue?@action=redirect&bpo=4195].)

- The **pdb** module can now access and display source code loaded via **zipimport** (or any other conformant **PEP 302** [https://peps.python.org/pep-0302/] loader).

  (Contributed by Alexander Belopolsky; bpo-4201 [https://bugs.python.org/issue?@action=redirect&bpo=4201].)

- **functools.partial** objects can now be pickled.

(Suggested by Antoine Pitrou and Jesse Noller. Implemented by Jack Diederich; bpo-5228 [https://bugs.python.org/issue?@action=redirect&bpo=5228].)

- Add **pydoc** help topics for symbols so that `help('@')` works as expected in the interactive environment.

  (Contributed by David Laban; bpo-4739 [https://bugs.python.org/issue?@action=redirect&bpo=4739].)

- The **unittest** module now supports skipping individual tests or classes of tests. And it supports marking a test as an expected failure, a test that is known to be broken, but shouldn't be counted as a failure on a TestResult:

```
class TestGizmo(unittest.TestCase):

    @unittest.skipUnless(sys.platform.startswith("win"), "re
    def test_gizmo_on_windows(self):
        ...

    @unittest.expectedFailure
    def test_gimzo_without_required_library(self):
        ...
```

Also, tests for exceptions have been builtout to work with context managers using the `with` statement:

```
def test_division_by_zero(self):
    with self.assertRaises(ZeroDivisionError):
        x / 0
```

In addition, several new assertion methods were added including `assertSetEqual()`, `assertDictEqual()`, **`assertDictContainsSubset()`**, `assertListEqual()`, `assertTupleEqual()`, `assertSequenceEqual()`, `assertRaisesRegexp()`, `assertIsNone()`, and `assertIsNotNone()`.

(Contributed by Benjamin Peterson and Antoine Pitrou.)

- The `io` module has three new constants for the `seek()` method: `SEEK_SET`, `SEEK_CUR`, and `SEEK_END`.

- The `sys.version_info` tuple is now a named tuple:

```
>>> sys.version_info
sys.version_info(major=3, minor=1, micro=0, releaselevel='a
```

(Contributed by Ross Light; bpo-4285 [https://bugs.python.org/issue?@action=redirect&bpo=4285].)

- The **`nntplib`** and `imaplib` modules now support IPv6.

(Contributed by Derek Morr; bpo-1655 [https://bugs.python.org/issue?@action=redirect&bpo=1655] and bpo-1664 [https://bugs.python.org/issue?@action=redirect&bpo=1664].)

- The `pickle` module has been adapted for better interoperability with Python 2.x when used with protocol 2 or lower. The reorganization of the standard library changed the formal reference for many objects. For example, `__builtin__.set` in Python 2 is called `builtins.set` in Python 3. This change confounded efforts to share data between different versions of Python. But now when protocol 2 or lower is

selected, the pickler will automatically use the old Python 2 names for both loading and dumping. This remapping is turned-on by default but can be disabled with the *fix_imports* option:

```
>>> s = {1, 2, 3}
>>> pickle.dumps(s, protocol=0)
b'c__builtin__\nset\np0\n((lp1\nL1L\naL2L\naL3L\natp2\nRp3\r
>>> pickle.dumps(s, protocol=0, fix_imports=False)
b'cbuiltins\nset\np0\n((lp1\nL1L\naL2L\naL3L\natp2\nRp3\n.'
```

An unfortunate but unavoidable side-effect of this change is that protocol 2 pickles produced by Python 3.1 won't be readable with Python 3.0. The latest pickle protocol, protocol 3, should be used when migrating data between Python 3.x implementations, as it doesn't attempt to remain compatible with Python 2.x.

(Contributed by Alexandre Vassalotti and Antoine Pitrou, bpo-6137 [https://bugs.python.org/issue?@action=redirect&bpo=6137].)

- A new module, **`importlib`** was added. It provides a complete, portable, pure Python reference implementation of the **`import`** statement and its counterpart, the **`__import__()`** function. It represents a substantial step forward in documenting and defining the actions that take place during imports.

  (Contributed by Brett Cannon.)

Otimizações

Major performance enhancements have been added:

- The new I/O library (as defined in **PEP 3116** [https://peps.python.org/pep-3116/]) was mostly written in Python and quickly proved to be a problematic bottleneck in Python 3.0. In Python 3.1, the I/O library has been entirely rewritten in C and is 2 to 20 times faster depending on the task at hand. The pure Python version is still available for experimentation purposes through the `_pyio` module.

(Contributed by Amaury Forgeot d'Arc and Antoine Pitrou.)

- Added a heuristic so that tuples and dicts containing only untrackable objects are not tracked by the garbage collector. This can reduce the size of collections and therefore the garbage collection overhead on long-running programs, depending on their particular use of datatypes.

  (Contributed by Antoine Pitrou, bpo-4688 [https://bugs.python.org/issue?@action=redirect&bpo=4688].)

- Enabling a configure option named `--with-computed-gotos` on compilers that support it (notably: gcc, SunPro, icc), the bytecode evaluation loop is compiled with a new dispatch mechanism which gives speedups of up to 20%, depending on the system, the compiler, and the benchmark.

  (Contributed by Antoine Pitrou along with a number of other participants, bpo-4753 [https://bugs.python.org/issue?@action=redirect&bpo=4753]).

- The decoding of UTF-8, UTF-16 and LATIN-1 is now two to four times faster.

  (Contributed by Antoine Pitrou and Amaury Forgeot d'Arc, bpo-4868 [https://bugs.python.org/issue?@action=redirect&bpo=4868].)

- The **`json`** module now has a C extension to substantially improve its performance. In addition, the API was modified so that json works only with **`str`**, not with **`bytes`**. That change makes the module closely match the JSON specification [https://json.org/] which is defined in terms of Unicode.

  (Contributed by Bob Ippolito and converted to Py3.1 by Antoine Pitrou and Benjamin Peterson; bpo-4136 [https://bugs.python.org/issue?@action=redirect&bpo=4136].)

- Unpickling now interns the attribute names of pickled objects. This saves memory and allows pickles to be smaller.

(Contributed by Jake McGuire and Antoine Pitrou; bpo-5084 [https://bugs.python.org/issue?@action=redirect&bpo=5084].)

IDLE

- IDLE's format menu now provides an option to strip trailing whitespace from a source file.

  (Contributed by Roger D. Serwy; bpo-5150 [https://bugs.python.org/issue?@action=redirect&bpo=5150].)

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- Os inteiros agora são armazenados internamente na base `2**15` ou na base `2**30`, the base being determined at build time. Previously, they were sempre armazenado na base `2**15`. Usando a base `2**30` dá significativa performance de desempenho em máquinas de 64 bits, mas resultados de referência em 32 bits máquinas foram misturadas. Portanto, o padrão é usar a base `2**30` em máquinas de 64 bits e base `2**15` em máquinas de 32 bits; no Unix, há uma nova opção de configuração `--enable-big-digits` que pode ser usada para substituir este padrão.

  Além das melhorias de desempenho, essa alteração deve ser invisível para os usuários finais, com uma exceção: para fins de teste e depuração, há um novo structseq **`sys.int_info`** que fornece informações sobre o formato interno, fornecendo o número de bits por dígito e o tamanho em bytes do tipo C usado para armazenar cada dígito:

  ```
  >>> import sys
  >>> sys.int_info
  sys.int_info(bits_per_digit=30, sizeof_digit=4)
  ```

(Contributed by Mark Dickinson; bpo-4258 [https://bugs.python.org/issue?@action=redirect&bpo=4258].)

- The `PyLong_AsUnsignedLongLong()` function now handles a negative *pylong* by raising `OverflowError` instead of `TypeError`.

  (Contributed by Mark Dickinson and Lisandro Dalcrin; bpo-5175 [https://bugs.python.org/issue?@action=redirect&bpo=5175].)

- Deprecated `PyNumber_Int()`. Use `PyNumber_Long()` instead.

  (Contributed by Mark Dickinson; bpo-4910 [https://bugs.python.org/issue?@action=redirect&bpo=4910].)

- Added a new `PyOS_string_to_double()` function to replace the deprecated functions `PyOS_ascii_strtod()` and `PyOS_ascii_atof()`.

  (Contributed by Mark Dickinson; bpo-5914 [https://bugs.python.org/issue?@action=redirect&bpo=5914].)

- Added `PyCapsule` as a replacement for the `PyCObject` API. The principal difference is that the new type has a well defined interface for passing typing safety information and a less complicated signature for calling a destructor. The old type had a problematic API and is now deprecated.

  (Contributed by Larry Hastings; bpo-5630 [https://bugs.python.org/issue?@action=redirect&bpo=5630].)

Porting to Python 3.1

Esta seção lista as alterações descritas anteriormente e outras correções de bugs que podem exigir alterações em seu código:

- The new floating-point string representations can break existing doctests. For example:

```
def e():
    '''Compute the base of natural logarithms.

    >>> e()
    2.7182818284590451

    '''
    return sum(1/math.factorial(x) for x in reversed(range(3

doctest.testmod()

**********************************************************
Failed example:
    e()
Expected:
    2.7182818284590451
Got:
    2.718281828459045
**********************************************************
```

- The automatic name remapping in the pickle module for protocol 2 or lower can make Python 3.1 pickles unreadable in Python 3.0. One solution is to use protocol 3. Another solution is to set the *fix_imports* option to `False`. See the discussion above for more details.

O que há de novo no Python 3.0

Autor:: Guido van Rossum

Este artigo explica os novos recursos do Python 3.0, em comparação com o 2.6. O Python 3.0, também conhecido como “Python 3000” ou “Py3K”, é o primeiro lançamento do Python *intencionalmente incompatível com versões anteriores*. O Python 3.0 foi lançado em 3 de dezembro de 2008. Há mais mudanças do que em um lançamento típico, e mais que são importantes para todos os usuários do Python. No entanto, depois de digerir as mudanças, você verá que o Python realmente não mudou tanto assim – em geral, estamos principalmente corrigindo aborrecimentos bem conhecidos, e removendo muita sujeira antiga.

Este artigo não tenta fornecer uma especificação completa de todos os novos recursos, mas tenta dar uma visão geral conveniente. Para detalhes completos, você deve consultar a documentação do Python 3.0 e/ou as muitas PEPs referenciadas no texto. Se você quiser entender a implementação completa e a lógica de design para um recurso específico, as PEPs geralmente têm mais detalhes do que a documentação regular; mas observe que as PEPs geralmente não são mantidas atualizadas depois que um recurso foi totalmente implementado.

Devido a restrições de tempo, este documento não está tão completo quanto deveria ser. Como sempre para um novo lançamento, o arquivo `Misc/NEWS` na distribuição fonte contém uma riqueza de informações detalhadas sobre cada pequena coisa que foi alterada.

Obstáculos comuns

Esta seção lista as poucas mudanças que provavelmente irão lhe atrapalhar se você estiver acostumado com o Python 2.5.

print é uma função

A instrução `print` foi substituída por uma função **`print()`**, com argumentos nomeados para substituir a maior parte da sintaxe especial da antiga instrução `print` (**PEP 3105** [https://peps.python.org/pep-3105/]). Exemplos:

```
Antigo: print "The answer is", 2*2
Novo: print("The answer is", 2*2)

Antigo: print x,           # Vírgula no final suprime nova linha
Novo: print(x, end=" ")  # Anexa um espaço em vez de uma nova li

Antigo: print              # Exibe uma nova linha
Novo: print()            # Você precisa chamar a função!

Antigo: print >>sys.stderr, "fatal error"
Novo: print("fatal error", file=sys.stderr)

Antigo: print (x, y)       # Exibe repr((x, y))
Novo: print((x, y))      # Não é o mesmo que print(x, y)!
```

Você também pode personalizar o separador entre itens, por exemplo:

```
print("Há <", 2**32, "> possibilidades!", sep="")
```

que produz:

```
Há <4294967296> possibilidades!
```

Nota:

- A função **`print()`** não tem suporte ao recurso “softspace” da antiga instrução `print`. Por exemplo, no Python 2.x, `print "A\n", "B"` escreveria `"A\nB\n"`; mas no Python 3.0, `print("A\n", "B")` escreve `"A\n B\n"`.
- Inicialmente, você vai se pegar digitando o antigo `print x` bastante no modo interativo. É hora de treinar seus dedos para digitar `print(x)` em vez disso!
- Ao usar a ferramenta de conversão de fonte para fonte `2to3`, todas as instruções `print` são automaticamente convertidas em chamadas de

função `print()`, então isso geralmente não é um problema para projetos maiores.

Visualizações e iteradores em vez de listas

Algumas APIs conhecidas não retornam mais listas:

- Os métodos `dict.keys()`, `dict.items()` e `dict.values()` de `dict` retornam visualizações, ou "views", em vez de listas. Por exemplo, isso não funciona mais: `k = d.keys(); k.sort()`. Use `k = sorted(d)` em vez disso (isso funciona no Python 2.5 também e é igualmente eficiente).

- Além disso, os métodos **`dict.iterkeys()`**, **`dict.iteritems()`** e **`dict.itervalues()`** não são mais suportados.

- `map()` e `filter()` retornam iteradores. Se você realmente precisa de uma lista e as sequências de entrada são todas de comprimento igual, uma solução rápida é envolver **`map()`** em `list()`, por exemplo, `list(map(...))`, mas uma solução melhor é frequentemente usar uma compreensão de lista (especialmente quando o código original usa `lambda`), ou reescrever o código para que ele não precise de uma lista. Particularmente complicado é **`map()`** invocado para os efeitos colaterais da função; a transformação correta é usar um laço `for` regular (já que criar uma lista seria apenas desperdício).

  Se as sequências de entrada não tiverem o mesmo comprimento, `map()` irá parar no término da mais curta das sequências. Para compatibilidade total com **`map()`** do Python 2.x, também envolva as sequências em `itertools.zip_longest()`, por exemplo, `map(func, *sequences)` se torna `list(map(func, itertools.zip_longest(*sequences)))`.

- `range()` agora se comporta como **`xrange()`** costumava se comportar, exceto que funciona com valores de tamanho arbitrário. Este último não existe mais.

- `zip()` agora retorna um iterador.

Comparações de ordenação

O Python 3.0 simplificou as regras para ordenar comparações:

- Os operadores de comparação de ordenação (<, <=, >=, >) levantam uma exceção TypeError quando os operandos não têm uma ordenação natural significativa. Assim, expressões como `1 < ''`, `0 > None` ou `len <= len` não são mais válidas e, por exemplo, `None < None` levanta `TypeError` em vez de retornar `False`. Um corolário é que classificar uma lista heterogênea não faz mais sentido – todos os elementos devem ser comparáveis entre si. Observe que isso não se aplica aos operadores `==` e `!=`: objetos de diferentes tipos incomparáveis sempre são comparados de forma diferente entre si.
- `sorted()` e `list.sort()` não aceitam mais o argumento *cmp* fornecendo uma função de comparação. Use o argumento *key* em vez disso. N.B. os argumentos *key* e *reverse* agora são "somente-nomeados".
- A função **`cmp()`** deve ser tratada como extinta, e o método especial **`__cmp__()`** não é mais suportado. Use `__lt__()` para classificação, `__eq__()` com `__hash__()` e outras comparações avançadas conforme necessário. (Se você realmente precisa da funcionalidade **`cmp()`**, pode usar a expressão `(a > b) - (a < b)` como o equivalente para `cmp(a, b)`.)

Inteiros

- **PEP 237** [https://peps.python.org/pep-0237/]: Essencialmente, **`long`** foi renomeado para `int`. Ou seja, há apenas um tipo integral embutido, chamado **`int`**; mas ele se comporta principalmente como o antigo tipo **`long`**.
- **PEP 238** [https://peps.python.org/pep-0238/]: Uma expressão como `1/2` retorna um float. Use `1//2` para obter o comportamento de truncamento. (A última sintaxe existe há anos, pelo menos desde o Python 2.2.)
- A constante **`sys.maxint`** foi removida, pois não há mais um limite para o valor de inteiros. No entanto, `sys.maxsize` pode ser usado como um inteiro maior do que qualquer lista prática ou índice de string. Ele está em conformidade com o tamanho inteiro "natural" da implementação e

é tipicamente o mesmo que **`sys.maxint`** em versões anteriores na mesma plataforma (assumindo as mesmas opções de construção).
- O `repr()` de um inteiro longo não inclui mais o `L` final, então o código que remove esse caractere incondicionalmente cortará o último dígito. (Use `str()` em vez disso.)
- Literais octais não estão mais no formato `0720`; em vez disso, use `0o720`.

Texto vs. Dados em vez de Unicode vs. 8 bits

Tudo o que você achava que sabia sobre dados binários e Unicode mudou.

- O Python 3.0 usa os conceitos de *texto* e *dados* (binários) em vez de strings Unicode e strings de 8 bits. Todo texto é Unicode; no entanto, o Unicode *codificado* é representado como dados binários. O tipo usado para armazenar texto é `str`, o tipo usado para armazenar dados é `bytes`. A maior diferença com a situação 2.x é que qualquer tentativa de misturar texto e dados no Python 3.0 levanta `TypeError`, enquanto que se você misturasse strings Unicode e de 8 bits no Python 2.x, funcionaria se a string de 8 bits contivesse apenas bytes de 7 bits (ASCII), mas você obteria `UnicodeDecodeError` se contivesse valores não ASCII. Esse comportamento específico de valor causou inúmeras caras tristes ao longo dos anos.
- Como consequência dessa mudança na filosofia, praticamente todo código que usa Unicode, codificações ou dados binários provavelmente terá que mudar. A mudança é para melhor, pois no mundo 2.x havia vários bugs relacionados à mistura de texto codificado e não codificado. Para estar preparado no Python 2.x, comece a usar **`unicode`** para todo texto não codificado e `str` apenas para dados binários ou codificados. Então a ferramenta `2to3` fará a maior parte do trabalho para você.
- Você não pode mais usar literais `u"..."` para texto Unicode. No entanto, você deve usar literais `b"..."` para dados binários.
- Como os tipos `str` e `bytes` não podem ser misturados, você deve sempre converter explicitamente entre eles. Use `str.encode()` para ir de **`str`** para **`bytes`**, e `bytes.decode()` para ir de **`bytes`** para **`str`**. Você

também pode usar `bytes(s, encoding=...)` e `str(b, encoding=...)`, respectivamente.

- Assim como **str**, o tipo **bytes** é imutável. Há um tipo *mutável* separado para armazenar dados binários em buffer, **bytearray**. Quase todas as APIs que aceitam **bytes** também aceitam **bytearray**. A API mutável é baseada em **collections.MutableSequence**.
- Todas as contrabarras em literais de strings brutas são interpretadas literalmente. Isso significa que os escapes `'\U'` e `'\u'` em strings brutas não são tratados de forma especial. Por exemplo, `r'\u20ac'` é uma string de 6 caracteres no Python 3.0, enquanto no 2.6, `ur'\u20ac'` era o único caractere "euro". (É claro que essa mudança afeta apenas literais de strings brutas; o caractere euro é `'\u20ac'` no Python 3.0.)
- O tipo abstrato embutido **basestring** foi removido. Use **str** em vez disso. Os tipos **str** e **bytes** não têm funcionalidade suficiente em comum para garantir uma classe base compartilhada. A ferramenta `2to3` (veja abaixo) substitui cada ocorrência de **basestring** por **str**.
- Arquivos abertos como arquivos texto (ainda o modo padrão para **open()**) sempre usam uma codificação para mapear entre strings (na memória) e bytes (no disco). Arquivos binários (abertos com um `b` no argumento mode) sempre usam bytes na memória. Isso significa que se um arquivo for aberto usando um modo ou codificação incorretos, a E/S provavelmente falhará ruidosamente, em vez de produzir dados incorretos silenciosamente. Isso também significa que até mesmo usuários Unix terão que especificar o modo correto (texto ou binário) ao abrir um arquivo. Há uma codificação padrão dependente da plataforma, que em plataformas Unixy pode ser definida com a variável de ambiente `LANG` (e às vezes também com algumas outras variáveis de ambiente relacionadas à localidade específicas da plataforma). Em muitos casos, mas não em todos, o padrão do sistema é UTF-8; você nunca deve contar com esse padrão. Qualquer aplicação que leia ou escreva mais do que texto ASCII puro provavelmente deve ter uma maneira de substituir a codificação. Não há mais necessidade de usar os fluxos com reconhecimento de codificação no módulo **codecs**.
- Os valores iniciais de **sys.stdin**, **sys.stdout** e **sys.stderr** agora são arquivos texto somente unicode (ou seja, são instâncias de

io.TextIOBase). Para ler e gravar dados de bytes com esses fluxos, você precisa usar o atributo io.TextIOBase.buffer deles.

- Nomes de arquivo são passados e retornados de APIs como strings (Unicode). Isso pode apresentar problemas específicos de plataforma porque em algumas plataformas nomes de arquivo são strings de bytes arbitrárias. (Por outro lado, no Windows nomes de arquivo são armazenados nativamente como Unicode.) Como solução alternativa, a maioria das APIs (por exemplo, open() e muitas funções no módulo os) que aceitam nomes de arquivo aceitam objetos bytes assim como strings, e algumas APIs têm uma maneira de pedir um valor de retorno **bytes**. Assim, os.listdir() retorna uma lista de instâncias **bytes** se o argumento for uma instância **bytes**, e os.getcwdb() retorna o diretório de trabalho atual como uma instância **bytes**. Observe que quando **os.listdir()** retorna uma lista de strings, os nomes de arquivos que não podem ser decodificados corretamente são omitidos em vez de levantar UnicodeError.
- Algumas APIs de sistema como os.environ e sys.argv também podem apresentar problemas quando os bytes disponibilizados pelo sistema não são interpretáveis usando a codificação padrão. Definir a variável LANG e executar o programa novamente é provavelmente a melhor abordagem.
- **PEP 3138** [https://peps.python.org/pep-3138/]: A repr() de uma string não escapa mais caracteres não-ASCII. No entanto, ele ainda escapa caracteres de controle e pontos de código com status não imprimível no padrão Unicode.
- **PEP 3120** [https://peps.python.org/pep-3120/]: A codificação de fontes padrão agora é UTF-8.
- **PEP 3131** [https://peps.python.org/pep-3131/]: Letras não ASCII agora são permitidas em identificadores. (No entanto, a biblioteca padrão permanece somente ASCII, com exceção dos nomes dos colaboradores nos comentários.)
- Os módulos **StringIO** e **cStringIO** se foram. Em vez disso, importe o módulo io e use io.StringIO ou io.BytesIO para texto e dados, respectivamente.
- Veja também o guia Unicode, que foi atualizado para o Python 3.0.

Visão geral das alterações da sintaxe

Esta seção fornece uma breve visão geral de cada mudança *sintática* no Python 3.0.

Nova sintaxe

- **PEP 3107** [https://peps.python.org/pep-3107/]: Anotações de argumento de função e valor de retorno. Isso fornece uma maneira padronizada de anotar os parâmetros e o valor de retorno de uma função. Não há semântica anexada a essas anotações, exceto que elas podem ser introspectadas em tempo de execução usando o atributo `__annotations__`. A intenção é encorajar a experimentação por meio de metaclasses, decoradores ou frameworks.

- **PEP 3102** [https://peps.python.org/pep-3102/]: Argumentos somente-nomeados. Parâmetros nomeados que ocorrem após `*args` na lista de parâmetros *devem* ser especificados usando a sintaxe dos nomes na chamada. Você também pode usar um `*` simples na lista de parâmetros para indicar que não aceita uma lista de argumentos de comprimento variável, mas tem argumentos somente-nomeados.

- Argumentos nomeados são permitidos após a lista de classes base em uma definição de classe. Isso é usado pela nova convenção para especificar uma metaclasse (veja a próxima seção), mas pode ser usado para outros propósitos também, desde que a metaclasse dê suporte.

- **PEP 3104** [https://peps.python.org/pep-3104/]: instrução `nonlocal`. Usando `nonlocal x`, agora você pode atribuir diretamente a uma variável em um escopo externo (mas não global). **`nonlocal`** é uma nova palavra reservada.

- **PEP 3132** [https://peps.python.org/pep-3132/]: Desempacotamento iterável estendido. Agora você pode escrever coisas como `a, b, *rest = some_sequence`. E até mesmo `*rest, a = stuff`. O objeto `rest` é sempre uma lista (possivelmente vazia); o lado direito pode ser qualquer iterável. Exemplo:

  ```
  (a, *rest, b) = range(5)
  ```

Isso define *a* como `0`, *b* como `4` e *rest* como `[1, 2, 3]`.

- Compreensões de dicionário: `{k: v for k, v in stuff}` significa a mesma coisa que `dict(stuff)`, mas é mais flexível. (Isto é **PEP 274** [https://peps.python.org/pep-0274/] justificada. :-)

- Definir literais, por exemplo, `{1, 2}`. Note que `{}` é um dicionário vazio; use `set()` para um conjunto vazio. Compreensões de conjuntos também são suportadas; por exemplo, `{x for x in stuff}` significa a mesma coisa que `set(stuff)`, mas é mais flexível.

- Novos literais octais, por exemplo, `0o720` (já em 2.6). Os antigos literais octais (`0720`) se foram.

- Novos literais binários, por exemplo, `0b1010` (já em 2.6), e há uma nova função embutida correspondente, **`bin()`**.

- Literais de bytes são introduzidos com um `b` ou `B` no início, e há uma nova função embutida correspondente, **`bytes()`**.

Sintaxe alterada

- **PEP 3109** [https://peps.python.org/pep-3109/] e **PEP 3134** [https://peps.python.org/pep-3134/]: nova sintaxe da instrução **`raise`**: `raise [`*`expr`*` [from `*`expr`*`]]`. Veja abaixo.

- **`as`** e **`with`** agora são palavras reservadas. (Desde a versão 2.6, na verdade.)

- `True`, `False` e `None` são palavras reservadas. (A versão 2.6 já aplicava parcialmente as restrições em `None`.)

- Alteração de **`except`** *exc*, *var* para **`except`** *exc* **`as`** *var*. Veja a **PEP 3110** [https://peps.python.org/pep-3110/].

- **PEP 3115** [https://peps.python.org/pep-3115/]: Nova sintaxe de metaclasse. Em vez de:

```
class C:
    __metaclass__ = M
    ...
```

agora você deve usar:

```
class C(metaclass=M):
    ...
```

A variável global do módulo **`__metaclass__`** não é mais suportada. (Era uma ajuda para facilitar a mudança do padrão para classes de novo estilo sem derivar cada classe de **`object`**.)

- As compreensões de lista não têm mais suporte a mais a forma sintática `[... for var in item1, item2, ...]`. Use `[... for var in (item1, item2, ...)]` em vez disso. Observe também que as compreensões de lista têm semântica diferente: elas estão mais próximas do açúcar sintático para uma expressão geradora dentro de um construtor **`list()`** e, em particular, as variáveis de controle de laço não são mais vazadas para o escopo circundante.

- As *reticências* (`...`) podem ser usadas como uma expressão atômica em qualquer lugar. (Anteriormente, eram permitidas apenas em fatias.) Além disso, agora *devem* ser escritas como `....` (Anteriormente, também poderiam ser escritas como `. . .`, por um mero acidente da gramática.)

Sintaxe removida

- **PEP 3113** [https://peps.python.org/pep-3113/]: Desempacotamento de parâmetro de tupla removido. Você não pode mais escrever `def foo(a, (b, c)): ...`. Em vez disso, use `def foo(a, b_c): b, c = b_c`.
- Removidos backticks (use **`repr()`**).
- Removido `<>` (use `!=`).
- Removida palavra reservada: **`exec()`** não é mais uma palavra reservada; ela permanece como uma função. (Felizmente a sintaxe da função também era aceita nas versões 2.x.) Observe também que

`exec()` não aceita mais um argumento de fluxo; em vez de `exec(f)` você pode usar `exec(f.read())`.
- Literais de inteiro não mais têm suporte a um `l` ou `L` final.
- Literais de string não mais têm suporte a um `u` ou `U` no início.
- A sintaxe **`from`** *módulo* **`import`** `*` é permitida somente no nível do módulo, não mais dentro de funções.
- A única sintaxe aceitável para importações relativas é `from .[module] import name`. Todos as formas de **`import`** que não começam com `.` são interpretadas como importações absolutas. (**PEP 328** [https://peps.python.org/pep-0328/])
- As classes clássicas se foram.

Mudanças já presentes no Python 2.6

Como muitos usuários provavelmente fazem o salto direto do Python 2.5 para o Python 3.0, esta seção lembra o leitor dos novos recursos que foram originalmente projetados para o Python 3.0, mas que foram portados de volta para o Python 2.6. As seções correspondentes em O que há de novo no Python 2.6 devem ser consultadas para descrições mais longas.

- PEP 343: The 'with' statement. A instrução **`with`** agora é um recurso padrão e não precisa mais ser importada de **`__future__`**. Confira também Writing Context Managers e The contextlib module.
- PEP 366: Explicit Relative Imports From a Main Module. Isso aumenta a utilidade da opção **`-m`** quando o módulo referenciado reside em um pacote.
- PEP 370: Per-user site-packages Directory.
- PEP 371: The multiprocessing Package.
- PEP 3101: Advanced String Formatting. Nota: a descrição da versão 2.6 menciona o método **`format()`** para strings de 8 bits e Unicode. No 3.0, apenas o tipo **`str`** (strings de texto com suporte a Unicode) tem suporte a esse método; o tipo **`bytes`** não. O plano é eventualmente tornar esta a única API para formatação de strings e começar a descontinuar o operador `%` no Python 3.1.
- PEP 3105: print As a Function. Este é agora um recurso padrão e não precisa mais ser importado de **`__future__`**. Mais detalhes foram fornecidos acima.

- PEP 3110: Exception-Handling Changes. A sintaxe **`except`** *exc* **`as`** *var* agora é padrão e **`except`** *exc*, *var* não é mais suportado. (Claro, a parte **`as`** *var* ainda é opcional.)
- PEP 3112: Byte Literals. A notação de literal de string `b"..."` (e suas variantes como `b'...'`, `b"""..."""` e `br"..."`) agora produz um literal do tipo **`bytes`**.
- PEP 3116: New I/O Library. O módulo **`io`** agora é a maneira padrão de fazer E/S de arquivo. A função embutida **`open()`** agora é um apelido para **`io.open()`** e tem argumentos nomeados adicionais *encoding*, *errors*, *newline* e *closefd*. Observe também que um argumento *mode* inválido agora levanta **`ValueError`**, não **`IOError`**. O objeto arquivo binário subjacente a um objeto arquivo texto pode ser acessado como **`f.buffer`** (mas tome cuidado, pois o objeto texto mantém um buffer de si mesmo para acelerar as operações de codificação e decodificação).
- PEP 3118: Revised Buffer Protocol. A antiga função embutida **`buffer()`** agora realmente se foi; a nova função embutida **`memoryview()`** fornece funcionalidade (principalmente) semelhante.
- PEP 3119: Abstract Base Classes. O módulo **`abc`** e as ABCs definidas no módulo **`collections`** desempenham um papel um pouco mais proeminente na linguagem agora, e tipos embutidos de coleção como **`dict`** e **`list`** estão em conformidade com as ABCs **`collections.MutableMapping`** e **`collections.MutableSequence`**, respectivamente.
- PEP 3127: Integer Literal Support and Syntax. Conforme mencionado acima, a nova notação de literal octal é a única suportada, e literais binários foram adicionados.
- PEP 3129: Class Decorators.
- PEP 3141: A Type Hierarchy for Numbers. O módulo **`numbers`** é outro novo uso de ABCs, definindo a “torre numérica” do Python. Observe também o novo módulo **`fractions`** que implementa **`numbers.Rational`**.

Mudanças na biblioteca

Devido a restrições de tempo, este documento não cobre exaustivamente as mudanças muito extensas na biblioteca padrão. **PEP 3108**

[https://peps.python.org/pep-3108/] é a referência para as principais mudanças na biblioteca. Aqui está uma revisão resumida:

- Muitos módulos antigos foram removidos. Alguns, como **`gopherlib`** (não mais usado) e **`md5`** (substituído por **`hashlib`**), já tinham sido descontinuados pela **PEP 4** [https://peps.python.org/pep-0004/]. Outros foram removidos como resultado da remoção do suporte para várias plataformas, como Irix, BeOS e Mac OS 9 (veja **PEP 11** [https://peps.python.org/pep-0011/]). Alguns módulos também foram selecionados para remoção no Python 3.0 devido à falta de uso ou porque existe uma substituição melhor. Veja **PEP 3108** [https://peps.python.org/pep-3108/] para uma lista exaustiva.

- O pacote **`bsddb3`** foi removido porque sua presença na biblioteca padrão principal provou ao longo do tempo ser um fardo particular para os desenvolvedores principais devido à instabilidade dos testes e ao cronograma de lançamento do Berkeley DB. No entanto, o pacote está vivo e bem, mantido externamente em https://www.jcea.es/programacion/pybsddb.htm.

- Alguns módulos foram renomeados porque seu nome antigo desobedecia a **PEP 8** [https://peps.python.org/pep-0008/], ou por vários outros motivos. Aqui está a lista:

Antigo nome	Novo nome
_winreg	winreg
ConfigParser	configparser
copy_reg	copyreg

Antigo nome	Novo nome
Queue	queue
SocketServer	socketserver
markupbase	_markupbase
repr	reprlib
test.test_support	test.support

- Um padrão comum no Python 2.x é ter uma versão de um módulo implementada em Python puro, com uma versão acelerada opcional implementada como uma extensão C; por exemplo, `pickle` e `cPickle`. Isso coloca o fardo de importar a versão acelerada e recorrer à versão Python pura em cada usuário desses módulos. No Python 3.0, as versões aceleradas são consideradas detalhes de implementação das versões Python puras. Os usuários devem sempre importar a versão padrão, que tenta importar a versão acelerada e recorrer à versão Python pura. O par `pickle` / `cPickle` recebeu esse tratamento. O módulo `profile` está na lista para 3.1. O módulo `StringIO` foi transformado em uma classe no módulo `io`.

- Alguns módulos relacionados foram agrupados em pacotes e, geralmente, os nomes dos submódulos foram simplificados. Os novos pacotes resultantes são:

  - `dbm` (`anydbm`, `dbhash`, `dbm`, `dumbdbm`, `gdbm`, `whichdb`).
  - `html` (`HTMLParser`, `htmlentitydefs`).

- http (**`httplib`**, **`BaseHTTPServer`**, **`CGIHTTPServer`**, **`SimpleHTTPServer`**, **`Cookie`**, **`cookielib`**).
- tkinter (todos os módulos relacionados a `Tkinter`, exceto turtle). O público-alvo de **`turtle`** não se importa muito com **`tkinter`**. Observe também que, a partir do Python 2.6, a funcionalidade de **`turtle`** foi bastante aprimorada.
- urllib (**`urllib`**, **`urllib2`**, **`urlparse`**, **`robotparse`**).
- xmlrpc (**`xmlrpclib`**, **`DocXMLRPCServer`**, **`SimpleXMLRPCServer`**).

Algumas outras mudanças nos módulos da biblioteca padrão, não cobertas pela **PEP 3108** [https://peps.python.org/pep-3108/]:

- **`sets`** eliminado. Use a classe embutida **`set()`**.
- Limpeza do módulo **`sys`**: removidos **`sys.exitfunc()`**, **`sys.exc_clear()`**, **`sys.exc_type`**, **`sys.exc_value`**, **`sys.exc_traceback`**. (Observe que **`sys.last_type`** etc. permanecem.)
- Limpeza do tipo **`array.array`**: os métodos **`read()`** e **`write()`** foram removidos; use **`fromfile()`** e **`tofile()`** em vez disso. Além disso, o typecode `'c'` para vetor foi removido – use `'b'` para bytes ou `'u'` para caracteres Unicode.
- Cleanup of the **`operator`** module: removed **`sequenceIncludes()`** and **`isCallable()`**.
- Limpeza do módulo **`thread`**: **`acquire_lock()`** e **`release_lock()`** foram removidos; use **`acquire()`** e **`release()`** em vez disso.
- Cleanup of the **`random`** module: removed the **`jumpahead()`** API.
- O módulo **`new`** foi removido.
- The functions **`os.tmpnam()`**, **`os.tempnam()`** and **`os.tmpfile()`** have been removed in favor of the **`tempfile`** module.
- O módulo **`tokenize`** foi alterado para funcionar com bytes. O ponto de entrada principal agora é **`tokenize.tokenize()`**, em vez de generate_tokens.
- **`string.letters`** and its friends (**`string.lowercase`** and **`string.uppercase`**) are gone. Use **`string.ascii_letters`** etc. instead. (The reason for the removal is that **`string.letters`** and friends had locale-specific behavior, which is a bad idea for such attractively named global “constants”.)

- Renamed module `__builtin__` to `builtins` (removing the underscores, adding an ‘s’). The `__builtins__` variable found in most global namespaces is unchanged. To modify a builtin, you should use `builtins`, not `__builtins__`!

PEP 3101 [https://peps.python.org/pep-3101/]: Uma nova abordagem para formatação de strings

- Um novo sistema para operações de formatação de strings embutidas substitui o operador de formatação de strings `%`. (No entanto, o operador `%` ainda é suportado; ele será descontinuado no Python 3.1 e removido da linguagem em algum momento posterior.) Leia a **PEP 3101** [https://peps.python.org/pep-3101/] para obter informações completas.

Mudanças para exceções

As APIs para levantar e capturar exceções foram limpas e novos recursos poderosos foram adicionados:

- **PEP 352** [https://peps.python.org/pep-0352/]: Todas as exceções devem ser derivadas (direta ou indiretamente) de `BaseException`. Esta é a raiz da hierarquia de exceções. Isso não é novo como recomendação, mas o *requisito* para herdar de `BaseException` é novo. (O Python 2.6 ainda permitia que classes clássicas fossem levantadas e não colocava nenhuma restrição sobre o que você pode capturar.) Como consequência, exceções de string estão finalmente verdadeira e completamente mortas.

- Quase todas as exceções devem derivar de `Exception`; `BaseException` deve ser usado apenas como uma classe base para exceções que devem ser manipuladas apenas no nível superior, como `SystemExit` ou `KeyboardInterrupt`. O idioma recomendado para manipular todas as exceções, exceto para esta última categoria, é usar `except Exception`.

- `StandardError` was removed.

- Exceptions no longer behave as sequences. Use the `args` attribute instead.

- **PEP 3109** [https://peps.python.org/pep-3109/]: levantando exceções. Agora você deve usar `raise Exception(args)` em vez de `raise Exception, args`. Além disso, você não pode mais especificar explicitamente um traceback (situação da pilha de execução); em vez disso, se você *precisar* fazer isso, você pode atribuir diretamente ao atributo `__traceback__` (veja abaixo).

- **PEP 3110** [https://peps.python.org/pep-3110/]: Capturando exceções. Agora você deve usar `except SomeException as variable` em vez de `except SomeException, variable`. Além disso, a variável *variable* é explicitamente excluída quando o bloco `except` é deixado.

- **PEP 3134** [https://peps.python.org/pep-3134/]: Encadeamento de exceções. Existem dois casos: encadeamento implícito e encadeamento explícito. O encadeamento implícito acontece quando uma exceção é levantada em um bloco manipulador `except` ou `finally`. Isso geralmente acontece devido a um bug no bloco do manipulador; chamamos isso de exceção *secundária*. Nesse caso, a exceção original (que estava sendo manipulada) é salva como o atributo `__context__` da exceção secundária. O encadeamento explícito é invocado com esta sintaxe:

  ```
  raise SecondaryException() from primary_exception
  ```

  (onde *primary_exception* é qualquer expressão que produz um objeto de exceção, provavelmente uma exceção que foi capturada anteriormente). Neste caso, a exceção primária é armazenada no atributo `__cause__` da exceção secundária. O traceback impresso quando uma exceção não tratada ocorre percorre a cadeia de atributos `__cause__` e `__context__` e imprime um traceback separado para cada componente da cadeia, com a exceção primária no topo. (Usuários Java podem reconhecer este comportamento.)

- **PEP 3134** [https://peps.python.org/pep-3134/]: Objetos exceção agora armazenam seu traceback como o atributo `__traceback__`. Isso significa que um objeto de exceção agora contém todas as informações

pertencentes a uma exceção, e há menos motivos para usar `sys.exc_info()` (embora o último não seja removido).

- Algumas mensagens de exceção estão melhores quando o Windows falha ao carregar um módulo de extensão. Por exemplo, `error code 193` agora é `%1 is not a valid Win32 application`. As strings agora lidam com localidades que não sejam em inglês.

Outras mudanças diversas

Operadores e métodos especiais

- `!=` agora retorna o oposto de `==`, a menos que `==` retorne `NotImplemented`.
- O conceito de "métodos não vinculados" foi removido da linguagem. Ao referenciar um método como um atributo de classe, agora você obtém um objeto função simples.
- **`__getslice__()`**, **`__setslice__()`** and **`__delslice__()`** were killed. The syntax `a[i:j]` now translates to `a.__getitem__(slice(i, j))` (or `__setitem__()` or `__delitem__()`, when used as an assignment or deletion target, respectively).
- **PEP 3114** [https://peps.python.org/pep-3114/]: o método padrão `next()` foi renomeado para `__next__()`.
- The **`__oct__()`** and **`__hex__()`** special methods are removed – `oct()` and `hex()` use `__index__()` now to convert the argument to an integer.
- Removed support for **`__members__`** and **`__methods__`**.
- Os atributos de função chamados **`func_X`** foram renomeados para usar o formato **`__X__`**, liberando esses nomes no espaço de nomes de atributos de função para atributos definidos pelo usuário. A saber, **`func_closure`**, **`func_code`**, **`func_defaults`**, **`func_dict`**, **`func_doc`**, **`func_globals`**, **`func_name`** foram renomeados para `__closure__`, `__code__`, `__defaults__`, `__dict__`, `__doc__`, `__globals__`, `__name__`, respectivamente.
- **`__nonzero__()`** agora é `__bool__()`.

Funções embutidas

- **PEP 3135** [https://peps.python.org/pep-3135/]: Nova função `super()`. Agora você pode invocar `super()` sem argumentos e (presumindo que isso esteja em um método de instância regular definido dentro de uma instrução `class`) a classe e instância corretas serão automaticamente escolhidas. Com argumentos, o comportamento de `super()` não é alterado.
- **PEP 3111** [https://peps.python.org/pep-3111/]: `raw_input()` was renamed to `input()`. That is, the new `input()` function reads a line from `sys.stdin` and returns it with the trailing newline stripped. It raises `EOFError` if the input is terminated prematurely. To get the old behavior of `input()`, use `eval(input())`.
- Uma nova função embutido `next()` foi adicionada para chamar o método `__next__()` em um objeto.
- A estratégia de arredondamento da função `round()` e o tipo de retorno foram alterados. Casos exatos de meio caminho agora são arredondados para o resultado par mais próximo em vez de para longe de zero. (Por exemplo, `round(2.5)` agora retorna `2` em vez de `3`.) `round(x[, n])` agora delega para `x.__round__([n])` em vez de sempre retornar um float. Geralmente retorna um inteiro quando chamado com um único argumento e um valor do mesmo tipo que `x` quando chamado com dois argumentos.
- Moved `intern()` to `sys.intern()`.
- Removed: `apply()`. Instead of `apply(f, args)` use `f(*args)`.
- Removed `callable()`. Instead of `callable(f)` you can use `isinstance(f, collections.Callable)`. The `operator.isCallable()` function is also gone.
- Removed `coerce()`. This function no longer serves a purpose now that classic classes are gone.
- Removed `execfile()`. Instead of `execfile(fn)` use `exec(open(fn).read())`.
- Removed the `file` type. Use `open()`. There are now several different kinds of streams that open can return in the `io` module.
- Removed `reduce()`. Use `functools.reduce()` if you really need it; however, 99 percent of the time an explicit `for` loop is more readable.
- Removed `reload()`. Use `imp.reload()`.
- Removed. `dict.has_key()` – use the `in` operator instead.

Mudanças a construções e API C

Devido a restrições de tempo, aqui está uma lista *muito* incompleta de alterações na API C.

- O suporte para várias plataformas foi descartado, incluindo, mas não se limitando a Mac OS 9, BeOS, RISCOS, Irix e Tru64.
- **PEP 3118** [https://peps.python.org/pep-3118/]: Nova API de Buffer.
- **PEP 3121** [https://peps.python.org/pep-3121/]: Inicialização e finalização do módulo de extensão.
- **PEP 3123** [https://peps.python.org/pep-3123/]: Tornando **`PyObject_HEAD`** conforme o padrão C.
- Não há mais suporte à API C para execução restrita.
- As APIs C **`PyNumber_Coerce()`**, **`PyNumber_CoerceEx()`**, **`PyMember_Get()`** e **`PyMember_Set()`** foram removidas.
- New C API **`PyImport_ImportModuleNoBlock()`**, works like **`PyImport_ImportModule()`** but won't block on the import lock (returning an error instead).
- Renomeado o slot e o método de nível C de conversão booleana: `nb_nonzero` agora é `nb_bool`.
- Removidas **`METH_OLDARGS`** e **`WITH_CYCLE_GC`** da API C.

Desempenho

O resultado líquido das generalizações do 3.0 é que o Python 3.0 executa o benchmark pystone cerca de 10% mais lento que o Python 2.5. Provavelmente a maior causa é a remoção do uso de maiúsculas e minúsculas especiais para números inteiros pequenos. Há espaço para melhorias, mas isso acontecerá após o lançamento do 3.0!

Portando para o Python 3.0

Para portar o código-fonte existente do Python 2.5 ou 2.6 para o Python 3.0, a melhor estratégia é a seguinte:

0. (Pré-requisito:) Comece com uma excelente cobertura de teste.

1. Portar para o Python 2.6. Isso não deve dar mais trabalho do que o port médio do Python 2.x para o Python 2.(x+1). Certifique-se de que todos os seus testes passem.
2. (Ainda usando 2.6:) Ative a opção de linha de comando `-3`. Isso habilita avisos sobre recursos que serão removidos (ou alterados) no 3.0. Execute seu conjunto de testes novamente e corrija o código sobre o qual você recebe avisos até que não haja mais avisos e todos os seus testes ainda passem.
3. Execute o tradutor de fonte para fonte `2to3` em sua árvore de código-fonte. Execute o resultado da tradução no Python 3.0. Corrija manualmente quaisquer problemas restantes, corrigindo problemas até que todos os testes passem novamente.

Não é recomendado tentar escrever código-fonte que seja executado inalterado no Python 2.6 e 3.0; você teria que usar um estilo de codificação muito distorcido, por exemplo, evitando instruções `print`, metaclasses e muito mais. Se você estiver mantendo uma biblioteca que precisa ter suporte ao Python 2.6 e Python 3.0, a melhor abordagem é modificar a etapa 3 acima editando a versão 2.6 do código-fonte e executando o tradutor `2to3` novamente, em vez de editar a versão 3.0 do código-fonte.

Para portar extensões C para Python 3.0, consulte Portando módulos de extensão para o Python 3.

O que há de novo no Python 2.7

Autor:: A.M. Kuchling (amk at amk.ca)

Este artigo explica os novos recursos do Python 2.7. O Python 2.7 foi lançado em 3 de julho de 2010.

A manipulação numérica foi aprimorada de várias maneiras, tanto para números de ponto flutuante quanto para a classe `Decimal`. Existem algumas adições úteis à biblioteca padrão, como um módulo `unittest` bastante aprimorado, o módulo `argparse` para analisar as opções da linha de comando, classes convenientes `OrderedDict` e `Counter` no módulo `collections` e muitas outras melhorias.

O Python 2.7 está planejado para ser o último dos lançamentos 2.x, por isso trabalhamos para torná-lo um bom lançamento a longo prazo. Para ajudar na migração para o Python 3, vários novos recursos da série Python 3.x foram incluídos no 2.7.

Este artigo não tenta fornecer uma especificação completa dos novos recursos, mas fornece uma visão geral conveniente. Para detalhes completos, você deve consultar a documentação do Python 2.7 em https://docs.python.org. Se você deseja entender a lógica do design e implementação, consulte a PEP para obter um novo recurso específico ou o problema em https://bugs.python.org no qual uma alteração foi discutida. Sempre que possível, o "O que há de novo no Python" é vinculado ao item de bug/patch de cada alteração.

O futuro para o Python 2.x

Python 2.7 é o último grande lançamento da série 2.x, já que os mantenedores do Python mudaram o foco de seus esforços de desenvolvimento de novos recursos para a série Python 3.x. Isso significa que, embora o Python 2 continue recebendo correções de bugs e seja atualizado para construir corretamente em novo hardware e versões de

sistemas operados com suporte, não haverá novos lançamentos de recursos completos para a linguagem ou biblioteca padrão.

No entanto, embora haja um grande subconjunto comum entre Python 2.7 e Python 3, e muitas das mudanças envolvidas na migração para esse subconjunto comum, ou diretamente para Python 3, podem ser automatizadas com segurança, algumas outras mudanças (principalmente aquelas associadas ao tratamento Unicode) pode exigir uma consideração cuidadosa e, de preferência, conjuntos de testes de regressão automatizados robustos, para migrar com eficácia.

Isso significa que o Python 2.7 permanecerá em vigor por um longo tempo, fornecendo uma plataforma base estável e com suporte para sistemas de produção que ainda não foram portados para Python 3. O ciclo de vida completo esperado da série Python 2.7 é detalhado na **PEP 373** [https://peps.python.org/pep-0373/].

Algumas consequências principais da significância de longo prazo de 2.7 são:

- Conforme observado acima, a versão 2.7 tem um período de manutenção muito mais longo em comparação com as versões 2.x anteriores. Atualmente, espera-se que o Python 2.7 continue com o suporte da equipe de desenvolvimento central (recebendo atualizações de segurança e outras correções de bugs) até pelo menos 2020 (10 anos após seu lançamento inicial, em comparação com o período de suporte mais típico de 18-24 meses).
- Conforme a biblioteca padrão do Python 2.7 envelhece, fazer uso efetivo do Python Package Index (diretamente ou por meio de um redistribuidor) torna-se mais importante para os usuários do Python 2. Além de uma grande variedade de pacotes de terceiros para várias tarefas, os pacotes disponíveis incluem backports de novos módulos e recursos da biblioteca padrão do Python 3 que são compatíveis com o Python 2, bem como várias ferramentas e bibliotecas que podem tornar mais fácil migre para Python 3. O Guia de Usuário para Empacotamento de Python [https://packaging.python.org] fornece orientação sobre como baixar e instalar software do Python Package Index.

- Embora a abordagem preferida para aprimorar o Python 2 agora seja a publicação de novos pacotes no Python Package Index, essa abordagem não funciona necessariamente em todos os casos, especialmente aqueles relacionados à segurança de rede. Em casos excepcionais que não podem ser tratados adequadamente com a publicação de pacotes novos ou atualizados no PyPI, o processo de Proposta de Melhoria do Python, ou PEP, pode ser usado para justificar a adição de novos recursos diretamente à biblioteca padrão do Python 2. Quaisquer adições, e as versões de manutenção onde foram adicionadas, serão observadas na seção Novos recursos adicionados às versões de manutenção do Python 2.7 abaixo.

Para projetos que desejam migrar de Python 2 para Python 3, ou para desenvolvedores de bibliotecas e frameworks que desejam oferecer suporte a usuários em Python 2 e Python 3, há uma variedade de ferramentas e guias disponíveis para ajudar a decidir sobre uma abordagem adequada e gerenciar alguns dos os detalhes técnicos envolvidos. O ponto de partida recomendado é o guia Como portar códigos do Python 2 para o Python 3.

Mudanças no tratamento de avisos de descontinuação

Para Python 2.7, uma decisão política foi feita para silenciar avisos apenas de interesse para desenvolvedores por padrão. `DeprecationWarning` e seus descendentes agora são ignorados a menos que solicitado de outra forma, evitando que os usuários vejam avisos acionados por uma aplicação. Essa mudança também foi feita no branch que se tornou o Python 3.2. (Discutido em stdlib-sig e realizado em bpo-7319 [https://bugs.python.org/issue?@action=redirect&bpo=7319].)

Em versões anteriores, as mensagens `DeprecationWarning` eram habilitadas por padrão, fornecendo aos desenvolvedores de Python uma indicação clara de onde seu código pode quebrar em uma versão principal futura do Python.

No entanto, há cada vez mais usuários de aplicações baseadas em Python que não estão diretamente envolvidos no desenvolvimento dessas aplicações. Mensagens de `DeprecationWarning` são irrelevantes para tais

usuários, fazendo-os se preocupar com uma aplicação que está realmente funcionando corretamente e sobrecarregando os desenvolvedores de aplicações com a resposta a essas preocupações.

Você pode reativar a exibição de mensagens `DeprecationWarning` executando Python com a opção `-Wdefault` (forma curta: `-Wd`) ou definindo a variável de ambiente `PYTHONWARNINGS` para `"default"` (ou `"d"`) antes de executar o Python. O código Python também pode reativá-los chamando `warnings.simplefilter('default')`.

O módulo `unittest` também reativa automaticamente os avisos de depreciação ao executar testes.

Recursos do Python 3.1

Assim como o Python 2.6 incorporou recursos do Python 3.0, a versão 2.7 incorpora alguns dos novos recursos do Python 3.1. A série 2.x continua a fornecer ferramentas para migração para a série 3.x.

Uma lista parcial de recursos do 3.1 que foram transferidos para o 2.7:

- A sintaxe para conjuntos de literais (`{1,2,3}` é um conjunto mutável).
- Compreensões de dicionário e conjunto (`{i: i*2 for i in range(3)}`).
- Vários gerenciadores de contexto em uma única instrução `with`.
- Uma nova versão da biblioteca `io`, reescrita em C para melhor desempenho.
- O tipo de dicionário ordenado descrito na PEP 372: Adicionando um dicionário ordenado a coleções.
- O novo especificador de formato `","` descrito na PEP 378: Especificador de formato para separador de milhares.
- O objeto `memoryview`.
- Um pequeno subconjunto do módulo `importlib`, descrito sob.
- O `repr()` de um ponto flutuante `x` é mais curto em muitos casos: agora é baseado na string decimal mais curta que é garantida para arredondar de volta para `x`. Como nas versões anteriores do Python, é garantido que `float(repr(x))` recupera `x`.

- As conversões de ponto flutuante para string e string para ponto flutuante são arredondadas corretamente. A função `round()` também está agora corretamente arredondada.
- O tipo `PyCapsule`, usado para fornecer uma API C para módulos de extensão.
- A função de API C `PyLong_AsLongAndOverflow()`.

Outros novos avisos do modo Python3 incluem:

- **`operator.isCallable()`** e **`operator.sequenceIncludes()`**, que não são suportados no 3.x, agora disparam avisos.
- A opção **`-3`** agora habilita automaticamente a opção **`-Qwarn`** que causa avisos sobre o uso de divisão clássica com inteiros e inteiros longos.

PEP 372: Adicionando um dicionário ordenado a coleções

Dicionários regulares no Python iteram sobre pares de chave/valor em ordem arbitrária. Ao longo dos anos, vários autores escreveram implementações alternativas que lembram a ordem em que as chaves foram inseridas originalmente. Com base nas experiências dessas implementações, 2.7 introduz uma nova classe `OrderedDict` no módulo `collections`.

A API `OrderedDict` fornece a mesma interface que os dicionários regulares, mas itera sobre chaves e valores em uma ordem garantida dependendo de quando uma chave foi inserida pela primeira vez:

```
>>> from collections import OrderedDict
>>> d = OrderedDict([('first', 1),
...                  ('second', 2),
...                  ('third', 3)])
>>> d.items()
[('first', 1), ('second', 2), ('third', 3)]
```

Se uma nova entrada substituir uma entrada existente, a posição de inserção original permanece inalterada:

```
>>> d['second'] = 4
>>> d.items()
[('first', 1), ('second', 4), ('third', 3)]
```

Excluir uma entrada e reinseri-la irá movê-la para o final:

```
>>> del d['second']
>>> d['second'] = 5
>>> d.items()
[('first', 1), ('third', 3), ('second', 5)]
```

O método **`popitem()`** tem um argumento opcional *last* cujo valor padrão é `True`. Se *last* for true, a chave adicionada mais recentemente é retornada e removida; se for falso, a chave mais antiga é selecionada:

```
>>> od = OrderedDict([(x,0) for x in range(20)])
>>> od.popitem()
(19, 0)
>>> od.popitem()
(18, 0)
>>> od.popitem(last=False)
(0, 0)
>>> od.popitem(last=False)
(1, 0)
```

A comparação de dois dicionários ordenados verifica as chaves e os valores e exige que o pedido de inserção seja o mesmo:

```
>>> od1 = OrderedDict([('first', 1),
...                    ('second', 2),
...                    ('third', 3)])
>>> od2 = OrderedDict([('third', 3),
...                    ('first', 1),
...                    ('second', 2)])
>>> od1 == od2
False
>>> # Move a chave 'third' para o fim
>>> del od2['third']; od2['third'] = 3
>>> od1 == od2
True
```

Comparar um `OrderedDict` com um dicionário regular ignora a ordem de inserção e apenas compara as chaves e valores.

Como funciona a `OrderedDict`? Ela mantém uma lista duplamente vinculada de chaves, acrescentando novas chaves à lista à medida que são inseridas. Um dicionário secundário mapeia as chaves para seu nó de lista correspondente, portanto, a exclusão não precisa percorrer toda a lista vinculada e, portanto, permanece $O(1)$.

A biblioteca padrão agora oferece suporte ao uso de dicionários ordenados em vários módulos.

- O módulo `ConfigParser` os usa por padrão, o que significa que os arquivos de configuração agora podem ser lidos, modificados e então escritos de volta em sua ordem original.
- O método `_asdict()` para `collections.namedtuple()` agora retorna um dicionário ordenado com os valores aparecendo na mesma ordem que os índices de tupla subjacentes.
- O construtor de classe `JSONDecoder` do módulo `json` foi estendido com um parâmetro *object_pairs_hook* para permitir que instâncias `OrderedDict` sejam construídas pelo decodificador. O suporte também foi adicionado para ferramentas de terceiros como PyYAML [https://pyyaml.org/].

Ver também

PEP 372 [https://peps.python.org/pep-0372/] - Adicionando um dicionário ordenado às coleções

PEP escrita por Armin Ronacher e Raymond Hettinger; implementada por Raymond Hettinger.

PEP 378: Especificador de formato para separador de milhares

Para tornar a saída do programa mais legível, pode ser útil adicionar separadores a números grandes, renderizando-os como 18,446,744.073,709,551,616 em vez de 18446744073709551616.

A solução totalmente geral para fazer isso é o módulo `locale`, que pode usar diferentes separadores (“,” na América do Norte, “.” na Europa) e diferentes tamanhos de agrupamento, mas `locale` é complicado para usar e inadequado para aplicativos multithread onde diferentes threads estão produzindo saída para diferentes localidades.

Portanto, um mecanismo simples de agrupamento de vírgulas foi adicionado à minilinguagem usada pelo método `str.format()`. Ao formatar um número de ponto flutuante, basta incluir uma vírgula entre a largura e a precisão:

```
>>> '{:20,.2f}'.format(18446744073709551616.0)
'18,446,744,073,709,551,616.00'
```

Ao formatar um número inteiro, inclua a vírgula após a largura:

```
>>> '{:20,d}'.format(18446744073709551616)
'18,446,744,073,709,551,616'
```

Este mecanismo não é adaptável de forma alguma; vírgulas são sempre usadas como separadores e o agrupamento é sempre em grupos de três dígitos. O mecanismo de formatação de vírgula não é tão geral quanto o módulo `locale`, mas é mais fácil de usar.

Ver também

PEP 378 [https://peps.python.org/pep-0378/] - Especificador de formato para separador de milhares
PEP escrita por Raymond Hettinger; implementada por Eric Smith.

PEP 389: O módulo argparse para analisar linhas de comando

O módulo **argparse** para analisar argumentos de linha de comando foi adicionado como um substituto mais poderoso para o módulo **optparse**.

Isso significa que Python agora oferece suporte a três módulos diferentes para analisar argumentos de linha de comando: **getopt**, **optparse** e **argparse**. O módulo **getopt** se assemelha muito à função **getopt()** da biblioteca C, portanto continua útil se você estiver escrevendo um protótipo Python que eventualmente será reescrito em C. **optparse** torna-se redundante, mas não há planos para removê-lo porque muitos scripts ainda o utilizam e não há uma maneira automatizada de atualizar esses scripts. (Tornar a API **argparse** consistente com a interface de **optparse** foi discutido, mas rejeitado por ser muito confuso e difícil.)

Resumindo, se você está escrevendo um novo script e não precisa se preocupar com a compatibilidade com versões anteriores do Python, use **argparse** ao invés de **optparse**.

Aqui está um exemplo:

```
import argparse

parser = argparse.ArgumentParser(description='Command-line examp

# Adiciona alterações opcionais
parser.add_argument('-v', action='store_true', dest='is_verbose'
                    help='produce verbose output')
parser.add_argument('-o', action='store', dest='output',
                    metavar='FILE',
                    help='direct output to FILE instead of stdou
parser.add_argument('-C', action='store', type=int, dest='contex
                    metavar='NUM', default=0,
                    help='display NUM lines of added context')

# Permite qualquer número de argumentos adicionais.
parser.add_argument(nargs='*', action='store', dest='inputs',
                    help='input filenames (default is stdin)')

args = parser.parse_args()
print args.__dict__
```

A menos que você o substitua, as opções **`-h`** e **`--help`** são adicionadas automaticamente e produzem uma saída formatada de maneira organizada:

```
-> ./python.exe argparse-example.py --help
usage: argparse-example.py [-h] [-v] [-o FILE] [-C NUM] [inputs

Command-line example.

positional arguments:
  inputs      input filenames (default is stdin)

optional arguments:
  -h, --help  show this help message and exit
  -v          produce verbose output
  -o FILE     direct output to FILE instead of stdout
  -C NUM      display NUM lines of added context
```

Como em **`optparse`**, as opções e argumentos da linha de comando são retornados como um objeto com atributos nomeados pelos parâmetros *dest*:

```
-> ./python.exe argparse-example.py -v
{'output': None,
 'is_verbose': True,
 'context': 0,
 'inputs': []}

-> ./python.exe argparse-example.py -v -o /tmp/output -C 4 file1
{'output': '/tmp/output',
 'is_verbose': True,
 'context': 4,
 'inputs': ['file1', 'file2']}
```

`argparse` tem uma validação muito mais sofisticada que **`optparse`**; você pode especificar um número exato de argumentos como um inteiro, 0 ou mais argumentos passando `'*'`, 1 ou mais passando `'+'`, ou um argumento opcional com `'?'`. Um analisador de nível superior pode conter sub-analisadores para definir subcomandos que possuem diferentes conjuntos de opções, como em `svn commit`, `svn checkout`, etc. Você pode especificar o tipo de um argumento como **`FileType`**, que abrirá arquivos

automaticamente para você e entende que `'-'` significa entrada ou saída padrão.

Ver também

Documentação do `argparse`
A página de documentação do módulo argparse.

Migrando código optparse para argparse
Parte da documentação do Python, descrevendo como converter o código que usa `optparse`.

PEP 389 [https://peps.python.org/pep-0389/] - argparse - Novo módulo de análise sintática de linha de comando
PEP escrita e implementada por Steven Bethard.

PEP 391: Configuração baseada em dicionário para logging

O módulo `logging` é muito flexível; os aplicativos podem definir uma árvore de subsistemas de log e cada logger nessa árvore pode filtrar determinadas mensagens, formatá-las de maneira diferente e direcionar mensagens para um número variável de manipuladores.

Toda essa flexibilidade pode exigir muita configuração. Você pode escrever instruções Python para criar objetos e definir suas propriedades, mas uma configuração complexa requer um código detalhado, mas enfadonho. `logging` também suporta uma função `fileConfig()` que analisa um arquivo, mas o formato do arquivo não oferece suporte à configuração de filtros, e é mais confuso gerar programaticamente.

O Python 2.7 adiciona uma função `dictConfig()` que usa um dicionário para configurar o log. Existem muitas maneiras de produzir um dicionário a partir de diferentes fontes: construir um com código; analisar um arquivo

contendo JSON; ou use uma biblioteca de análise de YAML se houver uma instalada. Para mais informações veja Funções de configuração.

O exemplo a seguir configura dois loggers, o logger raiz e um logger denominado “network”. As mensagens enviadas para o registrador raiz serão enviadas para o log do sistema usando o protocolo syslog, e as mensagens para o logger “network” serão gravadas em um arquivo `network.log` que será rotacionado assim que o log atingir 1 MB.

```
import logging
import logging.config

configdict = {
 'version': 1,    # Configuration schema in use; must be 1 for r
 'formatters': {
     'standard': {
         'format': ('%(asctime)s %(name)-15s '
                    '%(levelname)-8s %(message)s')}},

 'handlers': {'netlog': {'backupCount': 10,
                      'class': 'logging.handlers.RotatingFileHanc
                      'filename': '/logs/network.log',
                      'formatter': 'standard',
                      'level': 'INFO',
                      'maxBytes': 1000000},
              'syslog': {'class': 'logging.handlers.SysLogHandle
                         'formatter': 'standard',
                         'level': 'ERROR'}},

 # Especifica todos os loggers subordinados
 'loggers': {
            'network': {
                         'handlers': ['netlog']
            }
 },
 # Especifica propriedades do logger raiz
 'root': {
          'handlers': ['syslog']
 },
}

# Prepara as configurações
```

```
logging.config.dictConfig(configdict)

# Como um exemplo, registra duas mensagens de erro
logger = logging.getLogger('/')
logger.error('Database not found')

netlogger = logging.getLogger('network')
netlogger.error('Connection failed')
```

Três melhorias menores no módulo `logging`, todas implementadas por Vinay Sajip, são:

- A classe `SysLogHandler` agora tem suporte a syslogging sobre TCP. O construtor tem um parâmetro *socktype* que fornece o tipo de soquete a ser usado, `socket.SOCK_DGRAM` para UDP ou `socket.SOCK_STREAM` para TCP. O protocolo padrão permanece UDP.
- As instâncias `Logger` ganharam um método `getChild()` que recupera um logger descendente usando um caminho relativo. Por exemplo, depois de recuperar um logger fazendo `log = getLogger('app')`, chamar `log.getChild('network.listen')` é equivalente a `getLogger('app.network.listen')`.
- A classe `LoggerAdapter` ganhou um método `isEnabledFor()` que pega um *level* e retorna se o logger subjacente processaria uma mensagem desse nível de importância.

Ver também

PEP 391 [https://peps.python.org/pep-0391/] - Configuração baseada em dicionário para logging
PEP escrita e implementada por Vinay Sajip.

PEP 3106: Views de dicionário

Os métodos de dicionário `keys()`, `values()` e `items()` são diferentes no Python 3.x. Eles retornam um objeto chamado *view* em vez de uma lista totalmente materializada.

Não é possível alterar os valores de retorno de **keys()**, **values()** e **items()** no Python 2.7 porque muito código seria interrompido. Em vez disso, as versões 3.x foram adicionadas com os novos nomes **viewkeys()**, **viewvalues()** e **viewitems()**.

```
>>> d = dict((i*10, chr(65+i)) for i in range(26))
>>> d
{0: 'A', 130: 'N', 10: 'B', 140: 'O', 20: ..., 250: 'Z'}
>>> d.viewkeys()
dict_keys([0, 130, 10, 140, 20, 150, 30, ..., 250])
```

As views podem ser iteradas, mas as exibições de chave e item também se comportam como conjuntos. O operador `&` realiza a interseção, e o `|` realiza uma união:

```
>>> d1 = dict((i*10, chr(65+i)) for i in range(26))
>>> d2 = dict((i**.5, i) for i in range(1000))
>>> d1.viewkeys() & d2.viewkeys()
set([0.0, 10.0, 20.0, 30.0])
>>> d1.viewkeys() | range(0, 30)
set([0, 1, 130, 3, 4, 5, 6, ..., 120, 250])
```

A view acompanha o dicionário e seu conteúdo muda à medida que o dicionário é modificado:

```
>>> vk = d.viewkeys()
>>> vk
dict_keys([0, 130, 10, ..., 250])
>>> d[260] = '&'
>>> vk
dict_keys([0, 130, 260, 10, ..., 250])
```

No entanto, observe que você não pode adicionar ou remover chaves enquanto estiver iterando na view:

```
>>> for k in vk:
...     d[k*2] = k
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
RuntimeError: dictionary changed size during iteration
```

Você pode usar os métodos de exibição no código Python 2.x, e o conversor 2to3 irá alterá-los para os métodos padrão `keys()`, `values()` e `items()`.

Ver também

PEP 3106 [https://peps.python.org/pep-3106/] - Repaginação de dict.keys(), .values() e .items()

PEP escrita por Guido van Rossum. Backported para 2.7 por Alexandre Vassalotti; bpo-1967 [https://bugs.python.org/issue?@action=redirect&bpo=1967].

PEP 3137: O objeto memoryview

O objeto `memoryview` fornece uma visão do conteúdo da memória de outro objeto que corresponde à interface do tipo `bytes`.

```
>>> import string
>>> m = memoryview(string.letters)
>>> m
<memory at 0x37f850>
>>> len(m)           # Retorna o comprimento de objeto subjacent
52
>>> m[0], m[25], m[26]   # Indexação retorna um byte
('a', 'z', 'A')
>>> m2 = m[0:26]         # Fatiamento retorna outra memoryview
>>> m2
<memory at 0x37f080>
```

O conteúdo da viewq pode ser convertido em uma string de bytes ou uma lista de inteiros:

```
>>> m2.tobytes()
'abcdefghijklmnopqrstuvwxyz'
>>> m2.tolist()
[97, 98, 99, 100, 101, 102, 103, ... 121, 122]
>>>
```

Objetos `memoryview` permitem modificar o objeto subjacente se for um objeto mutável.

```
>>> m2[0] = 75
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot modify read-only memory
>>> b = bytearray(string.letters)  # Criando um objeto mutável
>>> b
bytearray(b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ
>>> mb = memoryview(b)
>>> mb[0] = '*'          # Atribui para visão, alterando o bytear
>>> b[0:5]               # O bytearray foi alterado.
bytearray(b'*bcde')
>>>
```

Ver também

PEP 3137 [https://peps.python.org/pep-3137/] - Bytes imutáveis e buffer mutável

PEP escrita por Guido van Rossum. Implementada por Travis Oliphant, Antoine Pitrou e outros. Portado para 2.7 por Antoine Pitrou; bpo-2396 [https://bugs.python.org/issue?@action=redirect&bpo=2396].

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- A sintaxe para set literais foi portada do Python 3.x. Os colchetes são usados para cercar o conteúdo do conjunto mutável resultante; os literais definidos são diferenciados dos dicionários por não conterem caracteres de dois pontos e valores. `{}` continua a representar um dicionário vazio; use `set()` para um conjunto vazio.

  ```
  >>> {1, 2, 3, 4, 5}
  set([1, 2, 3, 4, 5])
  >>> set() # conjunto vazio
  ```

```
set([])
>>> {}    # dicionário vazio
{}
```

Portado por Alexandre Vassalotti; bpo-2335 [https://bugs.python.org/issue?@action=redirect&bpo=2335].

- As compreensões de dicionário e conjunto são outro recurso importado do 3.x, generalizando as compreensões de lista/gerador para usar a sintaxe literal para conjuntos e dicionários.

```
>>> {x: x*x for x in range(6)}
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
>>> {('a'*x) for x in range(6)}
set(['', 'a', 'aa', 'aaa', 'aaaa', 'aaaaa'])
```

Portado por Alexandre Vassalotti; bpo-2333 [https://bugs.python.org/issue?@action=redirect&bpo=2333].

- A instrução **`with`** agora pode usar vários gerenciadores de contexto em uma instrução. Os gerenciadores de contexto são processados da esquerda para a direita e cada um é tratado como o início de uma nova instrução **`with`**. Isso significa que:

```
with A() as a, B() as b:
    ... conjunto de instruções ...
```

é equivalente a:

```
with A() as a:
    with B() as b:
        ... conjunto de instruções ...
```

A função **`contextlib.nested()`** fornece uma função muito similar, então não é mais necessária e foi descontinuada.

(Proposta em https://codereview.appspot.com/53094; implementada por Georg Brandl.)

- As conversões entre números de ponto flutuante, ou float. e strings agora são arredondadas corretamente na maioria das plataformas. Estas conversões ocorrem em muitos lugares diferentes: `str()` em floats e números complexos; os construtores `float` e `complex`; formatação numérica; serializar e desserializar floats e números complexos usando os módulos `marshal`, `pickle` e `json`; análise de float e literais imaginários no código Python; e a conversão `Decimal`-para-float.

  Relacionado a isso, o `repr()` de um número de ponto flutuante *x* agora retorna um resultado baseado na string decimal mais curta que é garantida para arredondar de volta para *x* sob o arredondamento correto (com o modo de arredondamento round-half-to-even). Anteriormente, fornecia uma string baseada no arredondamento x para 17 dígitos decimais.

  A biblioteca de arredondamento responsável por essa melhoria funciona no Windows e em plataformas Unix usando os compiladores gcc, icc ou suncc. Pode haver um pequeno número de plataformas onde a operação correta deste código não pode ser garantida, então o código não é usado em tais sistemas. Você pode descobrir qual código está sendo usado verificando `sys.float_repr_style`, que será `short` se o novo código estiver em uso e `legacy` se não estiver.

  Implementada por Eric Smith and Mark Dickinson, usando a biblioteca `dtoa.c` de David Gay; bpo-7117 [https://bugs.python.org/issue?@action=redirect&bpo=7117].

- As conversões de inteiros longos e inteiros regulares para ponto flutuante agora arredondam de maneira diferente, retornando o número de ponto flutuante mais próximo ao número. Isso não importa para pequenos inteiros que podem ser convertidos exatamente, mas para grandes números que inevitavelmente perderão a precisão, o Python 2.7 agora se aproxima mais. Por exemplo, o Python 2.6 computou o seguinte:

```
>>> n = 295147905179352891391
>>> float(n)
2.9514790517935283e+20
```

```
>>> n - long(float(n))
65535L
```

O resultado de ponto flutuante do Python 2.7 é maior, mas muito mais próximo do valor verdadeiro:

```
>>> n = 295147905179352891391
>>> float(n)
2.9514790517935289e+20
>>> n - long(float(n))
-1L
```

(Implementada por Mark Dickinson; bpo-3166 [https://bugs.python.org/issue?@action=redirect&bpo=3166].)

A divisão inteira também é mais precisa em seus comportamentos de arredondamento. (Também implementada por Mark Dickinson; bpo-1811 [https://bugs.python.org/issue?@action=redirect&bpo=1811].)

- A coerção implícita para números complexos foi removida; o interpretador nunca mais tentará chamar um método **`__coerce__()`** em objetos complexos. (Removido por Meador Inge e Mark Dickinson; bpo-5211 [https://bugs.python.org/issue?@action=redirect&bpo=5211].)

- O método **`str.format()`** agora tem suporte à numeração automática dos campos de substituição. Isso torna o uso de **`str.format()`** mais parecido com o uso da formatação `%s`:

```
>>> '{}:{}:{}'.format(2009, 04, 'Sunday')
'2009:4:Sunday'
>>> '{}:{}:{day}'.format(2009, 4, day='Sunday')
'2009:4:Sunday'
```

A numeração automática leva os campos da esquerda para a direita, então o primeiro especificador `{...}` usará o primeiro argumento para **`str.format()`**, o próximo especificador usará o próximo argumento e breve. Você não pode misturar a numeração automática e a numeração explícita – numerar todos os seus campos especificadores ou nenhum deles – mas você pode misturar a numeração automática e os campos

nomeados, como no segundo exemplo acima. (Contribuição de Eric Smith; bpo-5237 [https://bugs.python.org/issue?@action=redirect&bpo=5237].)

Números complexos agora suportam corretamente o uso com `format()`, e o padrão é alinhamento à direita. A especificação de uma precisão ou separação por vírgula aplica-se às partes reais e imaginárias do número, mas uma largura de campo e alinhamento especificados são aplicados a toda a saída `1.5+3j` resultante. (Contribuição de Eric Smith; bpo-1588 [https://bugs.python.org/issue?@action=redirect&bpo=1588] e bpo-7988 [https://bugs.python.org/issue?@action=redirect&bpo=7988].)

O código de formato 'F' agora sempre formata sua saída usando caracteres maiúsculos, então agora produzirá 'INF' e 'NAN'. (Contribuição de Eric Smith; bpo-3382 [https://bugs.python.org/issue?@action=redirect&bpo=3382].)

Uma alteração de baixo nível: o método `object.__format__()` agora aciona um `PendingDeprecationWarning` se for passado uma string de formato, porque o método `__format__()` para `object` converte o objeto para uma representação de string e formata isso. Anteriormente, o método aplicava silenciosamente a string de formato à representação da string, mas isso poderia ocultar erros no código Python. Se você estiver fornecendo informações de formatação, como alinhamento ou precisão, presumivelmente espera que a formatação seja aplicada de alguma maneira específica do objeto. (Correção de Eric Smith; bpo-7994 [https://bugs.python.org/issue?@action=redirect&bpo=7994].)

- Os tipos `int()` e `long()` ganharam um método `bit_length` que retorna o número de bits necessários para representar seu argumento em binário:

```
>>> n = 37
>>> bin(n)
'0b100101'
>>> n.bit_length()
6
>>> n = 2**123-1
>>> n.bit_length()
```

```
123
>>> (n+1).bit_length()
124
```

(Contribuição de Fredrik Johansson e Victor Stinner; bpo-3439 [https://bugs.python.org/issue?@action=redirect&bpo=3439].)

- A instrução **`import`** não tentará mais uma importação absoluta se uma importação relativa (por exemplo, `from .os import sep`) falhar. Isso corrige um bug, mas pode possivelmente quebrar certas instruções **`import`** que estavam funcionando apenas por acidente. (Correção de Meador Inge; bpo-7902 [https://bugs.python.org/issue?@action=redirect&bpo=7902].)

- Agora é possível para uma subclasse do tipo embutido **`unicode`** substituir o método **`__unicode__()`**. (Implementado por Victor Stinner; bpo-1583863 [https://bugs.python.org/issue?@action=redirect&bpo=1583863].)

- O método **`translate()`** do tipo **`bytearray`** agora aceita `None` como seu primeiro argumento. (Correção de Georg Brandl; bpo-4759 [https://bugs.python.org/issue?@action=redirect&bpo=4759].)

- Ao usar **`@classmethod`** e **`@staticmethod`** para agrupar métodos como classe ou métodos estáticos, o objeto envólucro agora expõe a função agrupada como seu atributo **`__func__`**. (Contribuição de Amaury Forgeot d'Arc, após sugestão de George Sakkis; bpo-5982 [https://bugs.python.org/issue?@action=redirect&bpo=5982].)

- Quando um conjunto restrito de atributos foi definido usando `__slots__`, excluir um atributo não definido não levantaria **`AttributeError`** como seria de esperar. Correção de Benjamin Peterson; bpo-7604 [https://bugs.python.org/issue?@action=redirect&bpo=7604].)

- Duas novas codificações agora são suportadas: "cp720", usado principalmente para texto em árabe; e "cp858", uma variante do CP 850 que adiciona o símbolo do euro. (CP720 foi contribuição de Alexander Belchenko e Amaury Forgeot d'Arc em bpo-1616979

[https://bugs.python.org/issue?@action=redirect&bpo=1616979]; CP858 foi contribuição de Tim Hatch em bpo-8016 [https://bugs.python.org/issue?@action=redirect&bpo=8016].)

- O objeto **`file`** agora definirá o atributo **`filename`** na exceção **`IOError`** ao tentar abrir um diretório em plataformas POSIX (observado por Jan Kaliszewski; bpo-4764 [https://bugs.python.org/issue?@action=redirect&bpo=4764]) , e agora verifica explicitamente e proíbe a gravação em objetos arquivo somente leitura em vez de confiar na biblioteca C para detectar e relatar o erro (correção de Stefan Krah; bpo-5677 [https://bugs.python.org/issue?@action=redirect&bpo=5677]).

- O tokenizador do Python agora traduz as próprias terminações de linha, então a função embutida **`compile()`** agora aceita código usando qualquer convenção de terminação de linha. Além disso, não requer mais que o código termine em uma nova linha.

- Parênteses extras em definições de função são ilegais no Python 3.x, o que significa que você obtém um erro de sintaxe de `def f((x)): pass`. No modo de aviso do Python3, o Python 2.7 agora avisará sobre esse uso estranho. (Notado por James Lingard; bpo-7362 [https://bugs.python.org/issue?@action=redirect&bpo=7362].)

- Agora é possível criar referências fracas para objetos de classe de estilo antigo. As classes de novo estilo sempre tiveram referências fracas. (Correção de Antoine Pitrou; bpo-8268 [https://bugs.python.org/issue?@action=redirect&bpo=8268].)

- Quando um objeto de módulo é coletado como lixo, o dicionário do módulo agora só é limpo se ninguém mais estiver mantendo uma referência ao dicionário (bpo-7140 [https://bugs.python.org/issue?@action=redirect&bpo=7140]).

Alterações no interpretador

Uma nova variável de ambiente, **`PYTHONWARNINGS`**, permite controlar os avisos. Deve ser definido como uma string contendo configurações de aviso, equivalentes àquelas usadas com a opção **`-W`**, separadas por vírgulas.

(Contribuição de Brian Curtin; bpo-7301 [https://bugs.python.org/issue?@action=redirect&bpo=7301].)

Por exemplo, a configuração a seguir imprimirá avisos toda vez que ocorrerem, mas transformará os avisos do módulo `Cookie` em um erro. (A sintaxe exata para definir uma variável de ambiente varia entre sistemas operacionais e shells.)

```
export PYTHONWARNINGS=all,error:::Cookie:0
```

Otimizações

Vários aprimoramentos de desempenho foram adicionados:

- Um novo opcode foi adicionado para executar a configuração inicial das instruções `with`, procurando os métodos `__enter__()` e `__exit__()`. (Contribuição de Benjamin Peterson.)

- O coletor de lixo agora funciona melhor para um padrão de uso comum: quando muitos objetos estão sendo alocados sem desalocar nenhum deles. Anteriormente, isso levaria um tempo quadrático para coleta de lixo, mas agora o número de coletas de lixo completas é reduzido à medida que o número de objetos no heap aumenta. A nova lógica só executa uma passagem completa de coleta de lixo quando a geração intermediária tiver sido coletada 10 vezes e quando o número de objetos sobreviventes da geração intermediária exceder 10% do número de objetos da geração mais antiga. (Sugerido por Martin von Löwis e implementado por Antoine Pitrou; bpo-4074 [https://bugs.python.org/issue?@action=redirect&bpo=4074].)

- O coletor de lixo tenta evitar o rastreamento de contêineres simples que não podem fazer parte de um ciclo. No Python 2.7, isso agora é verdade para tuplas e dicts contendo tipos atômicos (como ints, strings, etc.). Transitivamente, um dict contendo tuplas de tipos atômicos também não será rastreado. Isso ajuda a reduzir o custo de cada coleta de lixo diminuindo o número de objetos a serem considerados e percorridos pelo coletor. (Contribuição de Antoine Pitrou; bpo-4688 [https://bugs.python.org/issue?@action=redirect&bpo=4688].)

- Inteiros longos agora são armazenados internamente na base `2**15` ou na base `2**30`, sendo a base determinada no momento da compilação. Anteriormente, eles eram sempre armazenados na base `2**15`. O uso de base `2**30` oferece melhorias significativas de desempenho em máquinas de 64 bits, mas os resultados de benchmark em máquinas de 32 bits foram mistos. Portanto, o padrão é usar base `2**30` em máquinas de 64 bits e base `2**15` em máquinas de 32 bits; no Unix, há uma nova opção de configuração **`--enable-big-digits`** que pode ser usada para substituir esse padrão.

  Além das melhorias de desempenho, essa alteração deve ser invisível para os usuários finais, com uma exceção: para fins de teste e depuração, há um novo structseq **`sys.long_info`** que fornece informações sobre o formato interno, fornecendo o número de bits por dígito e o tamanho em bytes do tipo C usado para armazenar cada dígito:

  ```
  >>> import sys
  >>> sys.long_info
  sys.long_info(bits_per_digit=30, sizeof_digit=4)
  ```

  (Contribuição de Mark Dickinson; bpo-4258 [https://bugs.python.org/issue?@action=redirect&bpo=4258].)

  Outro conjunto de alterações tornou os objetos longos alguns bytes menores: 2 bytes menores em sistemas de 32 bits e 6 bytes em sistemas de 64 bits. (Contribuição de Mark Dickinson; bpo-5260 [https://bugs.python.org/issue?@action=redirect&bpo=5260].)

- O algoritmo de divisão para números inteiros longos ficou mais rápido apertando o laço interno, fazendo deslocamentos em vez de multiplicações e corrigindo uma iteração extra desnecessária. Vários benchmarks mostram acelerações entre 50% e 150% para divisões inteiras longas e operações de módulo. (Contribuição de Mark Dickinson; bpo-5512 [https://bugs.python.org/issue?@action=redirect&bpo=5512].) Operações bit a bit também são significativamente mais rápidas (patch inicial de Gregory Smith; bpo-1087418 [https://bugs.python.org/issue?@action=redirect&bpo=1087418]).

- A implementação de `%` verifica se o operando do lado esquerdo é uma string Python e usa casos especiais; isso resulta em um aumento de desempenho de 1 a 3% para aplicativos que usam frequentemente `%` com strings, como templates em bibliotecas. (Implementação de Collin Winter; bpo-5176 [https://bugs.python.org/issue?@action=redirect&bpo=5176].)

- Compreensões de lista com uma condição `if` são compiladas em bytecode mais rápido. (Patch de Antoine Pitrou, retroportado para 2.7 por Jeffrey Yasskin; bpo-4715 [https://bugs.python.org/issue?@action=redirect&bpo=4715].)

- A conversão de um inteiro ou inteiro longo em uma string decimal ficou mais rápida com o uso de maiúsculas e minúsculas especiais na base 10, em vez de usar uma função de conversão generalizada que oferece suporte a bases arbitrárias. (Patch por Gawain Bolton; bpo-6713 [https://bugs.python.org/issue?@action=redirect&bpo=6713].)

- Os métodos **`split()`**, **`replace()`**, **`rindex()`**, **`rpartition()`** e **`rsplit()`** de tipos semelhantes a strings (strings, Strings Unicode e objetos **`bytearray`**) agora usam um algoritmo de pesquisa reversa rápida em vez de uma varredura caractere por caractere. Às vezes, isso é mais rápido por um fator de 10. (Adicionado por Florent Xicluna; bpo-7462 [https://bugs.python.org/issue?@action=redirect&bpo=7462] e bpo-7622 [https://bugs.python.org/issue?@action=redirect&bpo=7622].)

- Os módulos **`pickle`** e **`cPickle`** agora internalizam automaticamente as strings usadas para nomes de atributos, reduzindo o uso de memória dos objetos resultantes da desserialização com pickle. (Contribuição de Jake McGuire; bpo-5084 [https://bugs.python.org/issue?@action=redirect&bpo=5084].)

- O módulo **`cPickle`** agora usa dicionários de casos especiais, reduzindo quase pela metade o tempo necessário para serializá-los com pickle. (Contribuição de Collin Winter; bpo-5670 [https://bugs.python.org/issue?@action=redirect&bpo=5670].)

Módulos Novos ou Aprimorados

Como em todas os lançamentos, a biblioteca padrão do Python recebeu diversas melhorias e correções de bugs. Aqui está uma lista parcial das mudanças mais notáveis, classificadas em ordem alfabética por nome do módulo. Consulte o arquivo `Misc/NEWS` na árvore de código-fonte para uma lista mais completa de alterações, ou procure nos logs do Subversion para todos os detalhes.

- A classe de depuração base do módulo **`bdb`** **`Bdb`** ganhou um recurso para pular módulos. O construtor agora usa um iterável contendo padrões estilo glob como `django.*`; o depurador não entrará nos quadros de pilha de um módulo que corresponda a um desses padrões. (Contribuição de Maru Newby após sugestão de Senthil Kumaran; bpo-5142 [https://bugs.python.org/issue?@action=redirect&bpo=5142].)

- O módulo **`binascii`** agora oferece suporte à API de buffer, então ele pode ser usado com instâncias **`memoryview`** e outros objetos buffer similares. (Backport de 3.x por Florent Xicluna; bpo-7703 [https://bugs.python.org/issue?@action=redirect&bpo=7703].)

- Módulo atualizado: o módulo **`bsddb`** foi atualizado de 4.7.2devel9 para a versão 4.8.4 do pacote pybsddb [https://www.jcea.es/programacion/pybsddb.htm]. A nova versão apresenta melhor compatibilidade com Python 3.x, várias correções de bugs e adiciona vários novos sinalizadores e métodos BerkeleyDB. (Atualizado por Jesús Cea Avión; bpo-8156 [https://bugs.python.org/issue?@action=redirect&bpo=8156]. O changelog do pybsddb pode ser lido em https://hg.jcea.es/pybsddb/file/tip/ChangeLog.)

- O **`BZ2File`** do módulo **`bz2`** agora oferece suporte ao protocolo de gerenciamento de contexto, então você pode escrever `with bz2.BZ2File(...) as f:`. (Contribuição de Hagen Fürstenau; bpo-3860 [https://bugs.python.org/issue?@action=redirect&bpo=3860].)

- Nova classe: a classe **`Counter`** no módulo **`collections`** é útil para contabilizar dados. As instâncias **`Counter`** se comportam principalmente como dicionários, mas retornam zero para chaves ausentes em vez de levantar uma exceção **`KeyError`**:

```
>>> from collections import Counter
>>> c = Counter()
>>> for letter in 'here is a sample of english text':
...   c[letter] += 1
...
>>> c
Counter({' ': 6, 'e': 5, 's': 3, 'a': 2, 'i': 2, 'h': 2,
'l': 2, 't': 2, 'g': 1, 'f': 1, 'm': 1, 'o': 1, 'n': 1,
'p': 1, 'r': 1, 'x': 1})
>>> c['e']
5
>>> c['z']
0
```

Existem três métodos **`Counter`** adicionais. **`most_common()`** retorna os N elementos mais comuns e suas contagens. **`elements()`** retorna um iterador sobre os elementos contidos, repetindo cada elemento tantas vezes quanto sua contagem. **`subtract()`** pega um iterável e subtrai um para cada elemento em vez de adicionar; se o argumento for um dicionário ou outro **`Counter`**, as contagens são subtraídas.

```
>>> c.most_common(5)
[(' ', 6), ('e', 5), ('s', 3), ('a', 2), ('i', 2)]
>>> c.elements() ->
   'a', 'a', ' ', ' ', ' ', ' ', ' ', ' ',
   'e', 'e', 'e', 'e', 'e', 'g', 'f', 'i', 'i',
   'h', 'h', 'm', 'l', 'l', 'o', 'n', 'p', 's',
   's', 's', 'r', 't', 't', 'x'
>>> c['e']
5
>>> c.subtract('very heavy on the letter e')
>>> c['e']    # Count is now lower
-1
```

Contribuição de Raymond Hettinger; bpo-1696199 [https://bugs.python.org/issue?@action=redirect&bpo=1696199].

Nova classe: **`OrderedDict`** é descrita na seção anterior PEP 372: Adicionando um dicionário ordenado a coleções.

Novo método: O tipo de dados **deque** agora tem um método **count()** que retorna o número de elementos contidos igual ao argumento fornecido *x*, e um **reverse()** método que inverte os elementos do deque no local. **deque** também expõe seu comprimento máximo como o atributo somente leitura **maxlen**. (Ambos os recursos adicionados por Raymond Hettinger.)

A classe **namedtuple** agora possui um parâmetro opcional *rename*. Se *rename* for verdadeiro, os nomes de campos inválidos porque foram repetidos ou não são identificadores Python legais serão renomeados para nomes legais derivados da posição do campo na lista de campos:

```
>>> from collections import namedtuple
>>> T = namedtuple('T', ['field1', '$illegal', 'for', 'fiel
>>> T._fields
('field1', '_1', '_2', 'field2')
```

(Adição de Raymond Hettinger; bpo-1818 [https://bugs.python.org/issue?@action=redirect&bpo=1818].)

Finalmente, a classe base abstrata **Mapping** agora retorna **NotImplemented** se um mapeamento for comparado a outro tipo que não seja um **Mapping**. (Correção de Daniel Stutzbach; bpo-8729 [https://bugs.python.org/issue?@action=redirect&bpo=8729].)

- Construtores para as classes de análise no módulo **ConfigParser** agora recebem um parâmetro *allow_no_value*, cujo padrão é false; se verdadeiro, opções sem valores serão permitidas. Por exemplo:

```
>>> import ConfigParser, StringIO
>>> sample_config = """
... [mysqld]
... user = mysql
... pid-file = /var/run/mysqld/mysqld.pid
... skip-bdb
... """
>>> config = ConfigParser.RawConfigParser(allow_no_value=Tru
>>> config.readfp(StringIO.StringIO(sample_config))
>>> config.get('mysqld', 'user')
```

```
'mysql'
>>> print config.get('mysqld', 'skip-bdb')
None
>>> print config.get('mysqld', 'unknown')
Traceback (most recent call last):
  ...
NoOptionError: No option 'unknown' in section: 'mysqld'
```

(Contribuição de Mats Kindahl; bpo-7005 [https://bugs.python.org/issue?@action=redirect&bpo=7005].)

- Função descontinuada: **`contextlib.nested()`**, que permite lidar com mais de um gerenciador de contexto com uma única instrução **`with`**, foi descontinuada porque a instrução **`with`** agora oferece suporte a múltiplos contextos gerentes.

- O módulo **`cookielib`** agora ignora cookies que possuem um campo de versão inválido, que não contém um valor inteiro. (Correção de John J. Lee; bpo-3924 [https://bugs.python.org/issue?@action=redirect&bpo=3924].)

- A função **`deepcopy()`** do módulo **`copy`** agora copiará corretamente os métodos de instância vinculados. (Implementado por Robert Collins; bpo-1515 [https://bugs.python.org/issue?@action=redirect&bpo=1515].)

- O módulo **`ctypes`** agora sempre converte `None` em um ponteiro C `NULL` para argumentos declarados como ponteiros. (Alterado por Thomas Heller; bpo-4606 [https://bugs.python.org/issue?@action=redirect&bpo=4606].) A biblioteca libffi [https://sourceware.org/libffi/] subjacente foi atualizada para a versão 3.0.9, contendo várias correções para diferentes plataformas. (Atualizado por Matthias Klose; bpo-8142 [https://bugs.python.org/issue?@action=redirect&bpo=8142].)

- Novo método: a classe **`timedelta`** do módulo **`datetime`** ganhou um método **`total_seconds()`** que retorna o número de segundos na duração. (Contribuição de Brian Quinlan; bpo-5788 [https://bugs.python.org/issue?@action=redirect&bpo=5788].)

- Novo método: a classe **`Decimal`** ganhou um método de classe **`from_float()`** que realiza uma conversão exata de um número de ponto flutuante para um **`Decimal`**. Essa conversão exata busca a maior aproximação decimal do valor da representação de ponto flutuante; o valor decimal resultante ainda incluirá a imprecisão, se houver. Por exemplo, `Decimal.from_float(0.1)` retorna `Decimal('0.1000000000000000055511151231257827021181583404541015625')`. (Implementado por Raymond Hettinger; bpo-4796 [https://bugs.python.org/issue?@action=redirect&bpo=4796].)

  Comparar instâncias de **`Decimal`** com números de ponto flutuante agora produz resultados sensatos com base nos valores numéricos dos operandos. Anteriormente, essas comparações recorreriam às regras padrão do Python para comparar objetos, que produziam resultados arbitrários com base em seu tipo. Observe que você ainda não pode combinar **`Decimal`** e ponto flutuante em outras operações, como adição, já que você deve escolher explicitamente como converter entre float e **`Decimal`**. (Corrigido por Mark Dickinson; bpo-2531 [https://bugs.python.org/issue?@action=redirect&bpo=2531].)

  O construtor para **`Decimal`** agora aceita números de ponto flutuante (adicionados por Raymond Hettinger; bpo-8257 [https://bugs.python.org/issue?@action=redirect&bpo=8257]) e caracteres Unicode não europeus, como dígitos árabe-índicos (contribuição de Mark Dickinson; bpo-6595 [https://bugs.python.org/issue?@action=redirect&bpo=6595]).

  A maioria dos métodos da classe **`Context`** agora aceitam números inteiros, bem como instâncias **`Decimal`**; as únicas exceções são os métodos **`canonical()`** e **`is_canonical()`**. (Patch de Juan José Conti; bpo-7633 [https://bugs.python.org/issue?@action=redirect&bpo=7633].)

  Ao usar instâncias **`Decimal`** com o método **`format()`** de uma string, o alinhamento padrão era anteriormente o alinhamento à esquerda. Isso foi alterado para alinhamento à direita, que é mais sensato para tipos numéricos. (Alteração de Mark Dickinson; bpo-6857 [https://bugs.python.org/issue?@action=redirect&bpo=6857].)

Comparações envolvendo um valor NaN de sinalização (ou `sNAN`) agora sinalizam **`InvalidOperation`** em vez de retornar silenciosamente um valor verdadeiro ou falso dependendo do operador de comparação. Valores silenciosos de NaN (ou `NaN`) agora são hasheáveis. (Corrigido por Mark Dickinson; bpo-7279 [https://bugs.python.org/issue?@action=redirect&bpo=7279].)

- O módulo **`difflib`** agora produz uma saída que é mais compatível com as ferramentas **diff**/**patch** modernas através de uma pequena mudança, usando um caractere de tabulação em vez de espaços como separador no cabeçalho, fornecendo o nome do arquivo. (Correção de Anatoly Techtonik; bpo-7585 [https://bugs.python.org/issue?@action=redirect&bpo=7585].)

- O comando Distutils `sdist` agora sempre regenera o arquivo `MANIFEST`, pois mesmo que os arquivos `MANIFEST.in` ou `setup.py` não tenham sido modificados, o usuário pode ter criado alguns novos arquivos que deveriam ser incluídos. (Correção de Tarek Ziadé; bpo-8688 [https://bugs.python.org/issue?@action=redirect&bpo=8688].)

- O sinalizador **`IGNORE_EXCEPTION_DETAIL`** do módulo **`doctest`** irá agora ignorar o nome do módulo que contém a exceção que está sendo testada. (Patch de Lennart Regebro; bpo-7490 [https://bugs.python.org/issue?@action=redirect&bpo=7490].)

- A classe **`Message`** do módulo **`email`** agora aceitará uma carga útil com valor Unicode, convertendo automaticamente a carga útil para a codificação especificada por **`output_charset`**. (Adição de R. David Murray; bpo-1368247 [https://bugs.python.org/issue?@action=redirect&bpo=1368247].)

- A classe **`Fraction`** agora aceita uma única instância float ou **`Decimal`**, ou dois números racionais, como argumentos para seu construtor. (Implementação de Mark Dickinson; racionais adicionados em bpo-5812 [https://bugs.python.org/issue?@action=redirect&bpo=5812] e float/decimal em bpo-8294 [https://bugs.python.org/issue?@action=redirect&bpo=8294].)

Ordenar comparações (<, <=, >, >=) entre frações e números complexos agora levanta um `TypeError`. Isso corrige um descuido, fazendo com que `Fraction` corresponda aos outros tipos numéricos.

- Nova classe: `FTP_TLS` no módulo `ftplib` fornece conexões FTP seguras usando encapsulamento TLS de autenticação, bem como controle subsequente e transferências de dados. (Contribuição de Giampaolo Rodola; bpo-2054 [https://bugs.python.org/issue?@action=redirect&bpo=2054].)

  O método `storbinary()` para envios de binários agora pode reiniciar envios graças a um parâmetro *rest* adicionado (patch de Pablo Mouzo; bpo-6845 [https://bugs.python.org/issue?@action=redirect&bpo=6845].)

- Novo decorador de classe: `@~functools.total_ordering` no módulo `functools` pega uma classe que define um método `__eq__()` e um de `__lt__()`, `__le__()`, `__gt__()` ou `__ge__()` e gera os métodos de comparação ausentes. Como o método `__cmp__()` está sendo descontinuado no Python 3.x, esse decorador facilita a definição de classes ordenadas. (Adicionado por Raymond Hettinger; bpo-5479 [https://bugs.python.org/issue?@action=redirect&bpo=5479].)

  Nova função: `cmp_to_key()` usará uma função de comparação de estilo antigo que espera dois argumentos e retornará um novo chamável que pode ser usado como parâmetro *key* para funções como `sorted()`, `min()` e `max()`, etc. O principal uso pretendido é ajudar a tornar o código compatível com Python 3.x. (Adição de Raymond Hettinger.)

- Nova função: o `is_tracked()` do módulo `gc` retorna verdadeiro se uma determinada instância for rastreada pelo coletor de lixo, falso caso contrário. (Contribuição de Antoine Pitrou; bpo-4688 [https://bugs.python.org/issue?@action=redirect&bpo=4688])

- O `GzipFile` do módulo `gzip` agora oferece suporte ao protocolo de gerenciamento de contexto, então você pode escrever `with gzip.GzipFile(...) as f:` (contribuição de Hagen Fürstenau; bpo-3860 [https://bugs.python.org/issue?@action=redirect&bpo=3860]), e agora

implementa o `io.BufferedIOBase` ABC, então você pode envolvê-lo com `io.BufferedReader` para processamento mais rápido (contribuição de Nir Aides; bpo-7471 [https://bugs.python.org/issue?@action=redirect&bpo=7471]). Agora também é possível substituir o horário de modificação registrado em um arquivo compactado, fornecendo um registro de data e hora opcional ao construtor. (Contribuição de Jacques Frechet; bpo-4272 [https://bugs.python.org/issue?@action=redirect&bpo=4272].)

Arquivos no formato gzip podem ser preenchidos com zero bytes à direita; o módulo `gzip` agora consumirá esses bytes finais. (Correção de Tadek Pietraszek e Brian Curtin; bpo-2846 [https://bugs.python.org/issue?@action=redirect&bpo=2846].)

- Novo atributo: o módulo `hashlib` agora possui um atributo `algorithms` contendo uma tupla nomeando os algoritmos suportados. No Python 2.7, `hashlib.algorithms` contém `('md5', 'sha1', 'sha224', 'sha256', 'sha384', 'sha512')`. (Contribuição de Carl Chenet; bpo-7418 [https://bugs.python.org/issue?@action=redirect&bpo=7418].)

- A classe padrão `HTTPResponse` usada pelo módulo `httplib` agora oferece suporte a buffering, resultando em uma leitura muito mais rápida de respostas HTTP. (Contribuição de Kristján Valur Jónsson; bpo-4879 [https://bugs.python.org/issue?@action=redirect&bpo=4879].)

  As classes `HTTPConnection` e `HTTPSConnection` agora oferecem suporte a um parâmetro *source_address*, uma tupla de 2 elementos `(host, port)` fornecendo o endereço de origem que será usado para a conexão. (Contribuição de Eldon Ziegler; bpo-3972 [https://bugs.python.org/issue?@action=redirect&bpo=3972].)

- O módulo `ihooks` agora oferece suporte a importações relativas. Observe que `ihooks` é um módulo mais antigo para personalizar importações, substituído pelo módulo `imputil` adicionado no Python 2.0. (Suporte relativo à importação adicionado por Neil Schemenauer.)

- O módulo `imaplib` agora oferece suporte a endereços IPv6. (Contribuição de Derek Morr; bpo-1655 [https://bugs.python.org/issue?

@action=redirect&bpo=1655].)

- Nova função: **getcallargs()** do módulo **inspect** pega um chamável e seus argumentos nomeados e posicionais, e descobre quais dos parâmetros do chamável receberão cada argumento, retornando um dicionário mapeando nomes de argumentos para seus valores. Por exemplo:

```
>>> from inspect import getcallargs
>>> def f(a, b=1, *pos, **named):
...     pass
...
>>> getcallargs(f, 1, 2, 3)
{'a': 1, 'b': 2, 'pos': (3,), 'named': {}}
>>> getcallargs(f, a=2, x=4)
{'a': 2, 'b': 1, 'pos': (), 'named': {'x': 4}}
>>> getcallargs(f)
Traceback (most recent call last):
...
TypeError: f() takes at least 1 argument (0 given)
```

  Contribuição de George Sakkis; bpo-3135 [https://bugs.python.org/issue?@action=redirect&bpo=3135].

- Módulo atualizado: A biblioteca **io** foi atualizada para a versão fornecida com Python 3.1. Para 3.1, a biblioteca de E/S foi totalmente reescrita em C e é de 2 a 20 vezes mais rápida dependendo da tarefa que está sendo executada. A versão original do Python foi renomeada para o módulo **_pyio**.

  Uma pequena mudança resultante: a classe **io.TextIOBase** agora tem um atributo **errors** fornecendo a configuração de erro usada para erros de codificação e decodificação (um dos `'strict'`, `'replace'`, `'ignore'`).

  A classe **io.FileIO** agora levanta uma exceção **OSError** quando passa um descritor de arquivo inválido. (Implementado por Benjamin Peterson; bpo-4991 [https://bugs.python.org/issue?@action=redirect&bpo=4991].) O método **truncate()** agora preserva a posição do arquivo;

anteriormente, mudaria a posição do arquivo para o final do novo arquivo. (Correção de Pascal Chambon; bpo-6939 [https://bugs.python.org/issue?@action=redirect&bpo=6939].)

- Nova função: `itertools.compress(data, selectors)` aceita dois iteradores. Elementos de *data* serão retornados se o valor correspondente em *selectors* for verdadeiro:

  ```
  itertools.compress('ABCDEF', [1,0,1,0,1,1]) =>
    A, C, E, F
  ```

  Nova função: `itertools.combinations_with_replacement(iter, r)` retorna todas as combinações possíveis de elementos de comprimento *r* do *iter* iterável. Ao contrário de **`combinations()`**, elementos individuais podem ser repetidos nas combinações geradas:

  ```
  itertools.combinations_with_replacement('abc', 2) =>
    ('a', 'a'), ('a', 'b'), ('a', 'c'),
    ('b', 'b'), ('b', 'c'), ('c', 'c')
  ```

  Observe que os elementos são tratados como únicos dependendo de sua posição na entrada, não de seus valores reais.

  A função **`itertools.count()`** agora tem um argumento *step* que permite incrementar por valores diferentes de 1. **`count()`** agora também permite argumentos nomeados e o uso de valores não inteiros, como floats ou **`Decimal`** instâncias. (Implementação de Raymond Hettinger; bpo-5032 [https://bugs.python.org/issue?@action=redirect&bpo=5032].)

  `itertools.combinations()` e **`itertools.product()`** anteriormente levantavam **`ValueError`** para valores de *r* maiores que o iterável de entrada. Isto foi considerado um erro de especificação, então agora elas retornam um iterador vazio. (Corrigido por Raymond Hettinger; bpo-4816 [https://bugs.python.org/issue?@action=redirect&bpo=4816].)

- Módulo atualizado: O módulo **`json`** foi atualizado para a versão 2.0.9 do pacote simplejson, que inclui uma extensão C que torna a codificação e decodificação mais rápidas. (Contribuição de Bob Ippolito; bpo-4136 [https://bugs.python.org/issue?@action=redirect&bpo=4136].)

Para oferecer suporte ao novo tipo `collections.OrderedDict`, `json.load()` agora tem um parâmetro opcional *object_pairs_hook* que será chamado com qualquer literal de objeto que decodifica para uma lista de pares. (Contribuição de Raymond Hettinger; bpo-5381 [https://bugs.python.org/issue?@action=redirect&bpo=5381].)

- A classe `Maildir` do módulo `mailbox` agora registra o registro de data e hora nos diretórios que lê, e apenas os relê se o horário de modificação for alterado posteriormente. Isso melhora o desempenho, evitando varreduras de diretório desnecessárias. (Corrigido por AM Kuchling e Antoine Pitrou; bpo-1607951 [https://bugs.python.org/issue?@action=redirect&bpo=1607951], bpo-6896 [https://bugs.python.org/issue?@action=redirect&bpo=6896].)

- Novas funções: o módulo `math` ganhou `erf()` e `erfc()` para a função de erro e a função de erro complementar, `expm1()` que calcula `e**x - 1` com mais precisão do que usar `exp()` e subtrair 1, `gamma()` para a função Gamma, e `lgamma()` para o log natural da função Gamma. (Contribuição de Mark Dickinson e nirinA raseliarison; bpo-3366 [https://bugs.python.org/issue?@action=redirect&bpo=3366].)

- As classes `Manager*` do módulo `multiprocessing` agora podem receber um chamável que será chamado sempre que um subprocesso for iniciado, junto com um conjunto de argumentos que serão passados para o chamável. (Contribuição de lekma; bpo-5585 [https://bugs.python.org/issue?@action=redirect&bpo=5585].)

  A classe `Pool`, que controla um conjunto de processos de trabalho, agora possui um parâmetro opcional *maxtasksperchild*. Os processos de trabalho executarão o número especificado de tarefas e então sairão, fazendo com que `Pool` inicie um novo trabalhador. Isso é útil se as tarefas puderem vazar memória ou outros recursos, ou se algumas tarefas fizerem com que o trabalho fique muito grande. (Contribuição de Charles Cazabon; bpo-6963 [https://bugs.python.org/issue?@action=redirect&bpo=6963].)

- O módulo `nntplib` agora oferece suporte para endereços IPv6. (Contribuição de Derek Morr; bpo-1664 [https://bugs.python.org/issue?

@action=redirect&bpo=1664].)

- Novas funções: o módulo **os** envolve as seguintes chamadas de sistema POSIX: **getresgid()** e **getresuid()**, que retornam os GIDs e UIDs reais, efetivos e salvos ; **setresgid()** e **setresuid()**, que definem GIDs e UIDs reais, efetivos e salvos para novos valores; **initgroups()**, que inicializa a lista de acesso do grupo para o processo atual. (Funções GID/UID foram contribuição de Travis H.; bpo-6508 [https://bugs.python.org/issue?@action=redirect&bpo=6508]. Suporte para initgroups adicionado por Jean-Paul Calderone; bpo-7333 [https://bugs.python.org/issue?@action=redirect&bpo=7333].)

  A função **os.fork()** agora reinicializa a trava de importação no processo filho; isso corrige problemas no Solaris quando **fork()** é chamado de um thread. (Correção de Zsolt Cserna; bpo-7242 [https://bugs.python.org/issue?@action=redirect&bpo=7242].)

- No módulo **os.path**, as funções **normpath()** e **abspath()** agora preservam o Unicode; se o caminho de entrada for uma string Unicode, o valor de retorno também será uma string Unicode. (**normpath()** corrigido por Matt Giuca em bpo-5827 [https://bugs.python.org/issue?@action=redirect&bpo=5827]; **abspath()** corrigido por Ezio Melotti em bpo-3426 [https://bugs.python.org/issue?@action=redirect&bpo=3426].)

- O módulo **pydoc** agora tem ajuda para os vários símbolos que o Python usa. Agora você pode fazer `help('<<')` ou `help('@')`, por exemplo. (Contribuição de David Laban; bpo-4739 [https://bugs.python.org/issue?@action=redirect&bpo=4739].)

- Os módulos **re** **split()**, **sub()** e **subn()** agora aceitam um argumento opcional *flags*, para consistência com as outras funções do módulo. (Adição de Gregory P. Smith.)

- Nova função: **run_path()** no módulo **runpy** executará o código em um argumento *path* fornecido. *path* pode ser o caminho de um arquivo fonte Python (`example.py`), um arquivo de bytecode compilado (`example.pyc`), um diretório (`./package/`) ou um arquivo zip

(`example.zip`). Se um diretório ou zip path for fornecido, ele será adicionado na frente de `sys.path` e o módulo `__main__` será importado. É esperado que o diretório ou zip contenha um `__main__.py`; caso contrário, algum outro `__main__.py` poderá ser importado de um local posteriormente em `sys.path`. Isso torna mais o maquinário de **`runpy`** disponível para scripts que desejam imitar a maneira como a linha de comando do Python processa um nome de caminho explícito. (Adição de Nick Coghlan; bpo-6816 [https://bugs.python.org/issue?@action=redirect&bpo=6816].)

- Nova função: no módulo **`shutil`**, **`make_archive()`** pega um nome de arquivo, tipo de arquivo (formato zip ou tar) e um caminho de diretório, e cria um arquivo contendo o conteúdo do diretório. (Adição de Tarek Ziadé.)

  As funções **`copyfile()`** e **`copytree()`** de **`shutil`** agora levantam uma exceção **`SpecialFileError`** quando solicitado a copiar um encadeamento nomeado. Anteriormente, o código trataria pipes nomeados como um arquivo normal, abrindo-os para leitura, e isso seria bloqueado indefinidamente. (Correção de Antoine Pitrou; bpo-3002 [https://bugs.python.org/issue?@action=redirect&bpo=3002].)

- O módulo **`signal`** não reinstala mais o manipulador de sinal a menos que isso seja realmente necessário, o que corrige um bug que poderia tornar impossível capturar o sinal EINTR de forma robusta. (Correção de Charles-François Natali; bpo-8354 [https://bugs.python.org/issue?@action=redirect&bpo=8354].)

- Novas funções: no módulo **`site`**, três novas funções retornam vários caminhos específicos do site e do usuário. **`getsitepackages()`** retorna uma lista contendo todos os diretórios globais de pacotes de sites, **`getusersitepackages()`** retorna o caminho do diretório de pacotes de sites do usuário e **`getuserbase()`** retorna o valor da variável de ambiente **`USER_BASE`**, fornecendo o caminho para um diretório que pode ser usado para armazenar dados. (Contribuição de Tarek Ziadé; bpo-6693 [https://bugs.python.org/issue?@action=redirect&bpo=6693].)

O módulo `site` agora reporta exceções que ocorrem quando o módulo `sitecustomize` é importado, e não irá mais capturar e engolir a exceção `KeyboardInterrupt`. (Correção de Victor Stinner; bpo-3137 [https://bugs.python.org/issue?@action=redirect&bpo=3137].)

- A função `create_connection()` ganhou um parâmetro *source_address*, uma tupla de 2 elementos `(host, port)` fornecendo o endereço de origem que será usado para a conexão. (Contribuição de Eldon Ziegler; bpo-3972 [https://bugs.python.org/issue?@action=redirect&bpo=3972].)

  Os métodos `recv_into()` e `recvfrom_into()` agora escreverão em objetos que oferecem suporte à API de buffer, mais utilmente objetos `bytearray` e `memoryview`. (Implementação de Antoine Pitrou; bpo-8104 [https://bugs.python.org/issue?@action=redirect&bpo=8104].)

- A classe `TCPServer` do módulo `SocketServer` agora oferece suporte a limite de tempo de soquete e desativação do algoritmo Nagle. O atributo de classe `disable_nagle_algorithm` é padronizado como `False`; se substituído como verdadeiro, novas conexões de solicitação terão a opção TCP_NODELAY definida para evitar o armazenamento em buffer de muitos envios pequenos em um único pacote TCP. O atributo de classe `timeout` pode conter um tempo limite em segundos que será aplicado ao soquete de solicitação; se nenhuma solicitação for recebida dentro desse tempo, `handle_timeout()` será chamado e `handle_request()` retornará. (Contribuição de Kristján Valur Jónsson; bpo-6192 [https://bugs.python.org/issue?@action=redirect&bpo=6192] e bpo-6267 [https://bugs.python.org/issue?@action=redirect&bpo=6267].)

- Módulo atualizado: o módulo `sqlite3` foi atualizado para a versão 2.6.0 do pacote pysqlite [https://github.com/ghaering/pysqlite]. A versão 2.6.0 inclui uma série de correções de bugs e adiciona a capacidade de carregar extensões SQLite de bibliotecas compartilhadas. Chame o método `enable_load_extension(True)` para habilitar extensões, e então chame `load_extension()` para carregar uma biblioteca compartilhada específica. (Atualização de Gerhard Häring.)

- Os objetos **`SSLSocket`** do módulo **`ssl`** agora oferecem suporte à API de buffer, que corrigiu uma falha no conjunto de testes (correção por Antoine Pitrou; bpo-7133 [https://bugs.python.org/issue?@action=redirect&bpo=7133]) e definiu automaticamente o **`SSL_MODE_AUTO_RETRY`** do OpenSSL, o que impedirá que um código de erro seja retornado de operações **`recv()`** que acionam uma renegociação SSL (correção por Antoine Pitrou; bpo-8222 [https://bugs.python.org/issue?@action=redirect&bpo=8222]).

  A função construtora **`wrap_socket()`** agora recebe um argumento *ciphers* que é uma string listando os algoritmos de criptografia a serem permitidos; o formato da string é descrito na documentação do OpenSSL [https://docs.openssl.org/1.0.2/man1/ciphers/]. (Adição de Antoine Pitrou; bpo-8322 [https://bugs.python.org/issue?@action=redirect&bpo=8322].)

  Outra mudança faz com que a extensão carregue todas as cifras e algoritmos de resumo do OpenSSL para que estejam todos disponíveis. Não foi possível verificar alguns certificados SSL, reportando um erro de "algoritmo desconhecido". (Relato de Beda Kosata e correção de Antoine Pitrou; bpo-8484 [https://bugs.python.org/issue?@action=redirect&bpo=8484].)

  A versão do OpenSSL usada agora está disponível como os atributos do módulo **`ssl.OPENSSL_VERSION`** (uma string), **`ssl.OPENSSL_VERSION_INFO`** (uma tupla de 5) e **`ssl.OPENSSL_VERSION_NUMBER`** (um número inteiro). (Adição de Antoine Pitrou; bpo-8321 [https://bugs.python.org/issue?@action=redirect&bpo=8321].)

- O módulo **`struct`** não irá mais ignorar silenciosamente erros de overflow quando um valor for muito grande para um código de formato inteiro específico (um de `bBhHiIlLqQ`); agora sempre levanta uma exceção **`struct.error`**. (Alteração de Mark Dickinson; bpo-1523 [https://bugs.python.org/issue?@action=redirect&bpo=1523].) A função **`pack()`** também tentará usar **`__index__()`** para converter e empacotar números não inteiros antes de tentar o método **`__int__()`** ou relatar um erro. (Alteração de Mark Dickinson; bpo-8300 [https://bugs.python.org/issue?@action=redirect&bpo=8300].)

- Nova função: o `check_output()` do módulo `subprocess` executa um comando com um conjunto especificado de argumentos e retorna a saída do comando como uma string quando o comando é executado sem erros, ou levanta uma exceção `CalledProcessError` caso contrário.

  ```
  >>> subprocess.check_output(['df', '-h', '.'])
  'Filesystem     Size   Used  Avail Capacity  Mounted on\n
  /dev/disk0s2    52G    49G   3.0G    94%    /\n'

  >>> subprocess.check_output(['df', '-h', '/bogus'])
    ...
  subprocess.CalledProcessError: Command '['df', '-h', '/bogus
  ```

  (Contribuição de Gregory P. Smith.)

  O módulo `subprocess` agora tentará novamente suas chamadas internas do sistema ao receber um sinal `EINTR`. (Relato por várias pessoas; patch final por Gregory P. Smith em bpo-1068268 [https://bugs.python.org/issue?@action=redirect&bpo=1068268].)

- Nova função: `is_declared_global()` no módulo `symtable` retorna verdadeiro para variáveis que são explicitamente declaradas como globais, falso para aquelas que são implicitamente globais. (Contribuição de Jeremy Hylton.)

- O módulo `syslog` agora usará o valor de `sys.argv[0]` como identificador em vez do valor padrão anterior de `'python'`. (Alteração de Sean Reifschneider; bpo-8451 [https://bugs.python.org/issue?@action=redirect&bpo=8451].)

- O valor `sys.version_info` agora é uma tupla nomeada, com atributos chamados **`major`**, **`minor`**, **`micro`**, **`releaselevel`** e **`serial`**. (Contribuição de Ross Light; bpo-4285 [https://bugs.python.org/issue?@action=redirect&bpo=4285].)

  `sys.getwindowsversion()` também retorna uma tupla nomeada, com atributos chamados **`major`**, **`minor`**, **`build`**, **`platform`**, **`service_pack`**,

`service_pack_major`, **`service_pack_minor`**, **`suite_mask`** e **`product_type`**. (Contribuição de Brian Curtin; bpo-7766 [https://bugs.python.org/issue?@action=redirect&bpo=7766].)

- O tratamento de erros padrão do módulo **`tarfile`** foi alterado, para não mais suprimir erros fatais. O nível de erro padrão era anteriormente 0, o que significava que os erros resultariam apenas na gravação de uma mensagem no log de depuração, mas como o log de depuração não está ativado por padrão, esses erros passam despercebidos. O nível de erro padrão agora é 1, o que levanta uma exceção se houver um erro. (Alteração de Lars Gustäbel; bpo-7357 [https://bugs.python.org/issue?@action=redirect&bpo=7357].)

  `tarfile` agora oferece suporte à filtragem de objetos **`TarInfo`** sendo adicionados a um arquivo tar. Quando você chama **`add()`**, você pode fornecer um argumento *filter* opcional que é chamável. O chamável *filter* receberá o **`TarInfo`** para cada arquivo sendo adicionado, e pode modificá-lo e retorná-lo. Se o chamável retornar `None`, o arquivo será excluído do arquivo resultante. Isto é mais poderoso do que o argumento *exclude* existente, que foi, portanto, descontinuado. (Adição de Lars Gustäbel; bpo-6856 [https://bugs.python.org/issue?@action=redirect&bpo=6856].) A classe **`TarFile`** agora também oferece suporte ao protocolo de gerenciamento de contexto. (Adição de Lars Gustäbel; bpo-7232 [https://bugs.python.org/issue?@action=redirect&bpo=7232].)

- O método **`wait()`** da classe **`threading.Event`** agora retorna o sinalizador interno na saída. Isso significa que o método geralmente retornará verdadeiro porque **`wait()`** deve ser bloqueado até que o sinalizador interno se torne verdadeiro. O valor de retorno só será falso se um tempo limite tiver sido fornecido e a operação tiver expirado. (Contribuição de Tim Lesher; bpo-1674032 [https://bugs.python.org/issue?@action=redirect&bpo=1674032].)

- O banco de dados Unicode fornecido pelo módulo **`unicodedata`** agora é usado internamente para determinar quais caracteres são numéricos, espaços em branco ou representam quebras de linha. O banco de dados também inclui informações do arquivo de dados `Unihan.txt` (patch de Anders Chrigström e Amaury Forgeot d'Arc; bpo-1571184

[https://bugs.python.org/issue?@action=redirect&bpo=1571184]) e foi atualizado para a versão 5.2.0 (atualização de Florent Xicluna; bpo-8024 [https://bugs.python.org/issue?@action=redirect&bpo=8024]).

- A função **`urlsplit()`** do módulo **`urlparse`** agora lida com esquemas de URL desconhecidos de uma forma compatível com **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html]: se a URL estiver no formato "`<alguma_coisa>://...`", o texto antes de `://` é tratado como o esquema, mesmo que seja um esquema inventado que o módulo não conhece. Essa alteração pode quebrar o código que contornava o comportamento antigo. Por exemplo, Python 2.6.4 ou 2.5 retornará o seguinte:

```
>>> import urlparse
>>> urlparse.urlsplit('invented://host/filename?query')
('invented', '', '//host/filename?query', '', '')
```

Python 2.7 (e Python 2.6.5) vai retornar:

```
>>> import urlparse
>>> urlparse.urlsplit('invented://host/filename?query')
('invented', 'host', '/filename?query', '', '')
```

(Na verdade, o Python 2.7 produz uma saída ligeiramente diferente, pois retorna uma tupla nomeada em vez de uma tupla padrão.)

O módulo **`urlparse`** também oferece suporte a endereços IPv6 literais conforme definido pelo **RFC 2732** [https://datatracker.ietf.org/doc/html/rfc2732.html] (contribuição de Senthil Kumaran; bpo-2987 [https://bugs.python.org/issue?@action=redirect&bpo=2987]).

```
>>> urlparse.urlparse('http://[1080::8:800:200C:417A]/foo')
ParseResult(scheme='http', netloc='[1080::8:800:200C:417A]',
            path='/foo', params='', query='', fragment='')
```

- Nova classe: a classe **`WeakSet`** no módulo **`weakref`** é um conjunto que contém apenas referências fracas aos seus elementos; os elementos serão removidos quando não houver referências apontando para eles.

(Originalmente implementado em Python 3.x por Raymond Hettinger e portado para 2.7 por Michael Foord.)

- A biblioteca **`xml.etree.ElementTree`** não escapa mais caracteres de E comercial e de sinal de maior e de menor ao gerar uma instrução de processamento XML (que se parece com `<?xml-stylesheet href="#style1"?>`) ou comentário (que se parece com `<!--comentário -->`). (Patch de Neil Muller; bpo-2746 [https://bugs.python.org/issue?@action=redirect&bpo=2746].)

- O cliente e servidor de XML-RPC, fornecidos pelos módulos **`xmlrpclib`** e **`SimpleXMLRPCServer`**, melhoraram o desempenho ao prover suporte a kep-alive do HTTP/1.1 e, opcionalmente, usando a codificação gzip para compactar o XML que está sendo trocado. A compactação gzip é controlada pelo atributo **`encode_threshold`** de **`SimpleXMLRPCRequestHandler`**, que contém um tamanho em bytes; respostas maiores que isso serão compactadas. (Contribuição de Kristján Valur Jónsson; bpo-6267 [https://bugs.python.org/issue?@action=redirect&bpo=6267].)

- A classe **`ZipFile`** do módulo **`zipfile`** agora oferece suporte ao protocolo de gerenciamento de contexto, então você pode escrever `with zipfile.ZipFile(...) as f:`. (Contribuição de Brian Curtin; bpo-5511 [https://bugs.python.org/issue?@action=redirect&bpo=5511].)

  `zipfile` agora também oferece suporte ao arquivamento de diretórios vazios e os extrai corretamente. (Correção de Kuba Wieczorek; bpo-4710 [https://bugs.python.org/issue?@action=redirect&bpo=4710].) Ler arquivos de um arquivo é mais rápido e intercalar **`read()`** e **`readline()`** agora funciona corretamente. (Contribuição de Nir Aides; bpo-7610 [https://bugs.python.org/issue?@action=redirect&bpo=7610].)

  A função **`is_zipfile()`** agora aceita um objeto arquivo, além dos nomes de caminho aceitos em versões anteriores. (Contribuição de Gabriel Genellina; bpo-4756 [https://bugs.python.org/issue?@action=redirect&bpo=4756].)

O método **writestr()** agora tem um parâmetro *compress_type* opcional que permite substituir o método de compactação padrão especificado no construtor de **ZipFile**. (Contribuição de Ronald Oussoren; bpo-6003 [https://bugs.python.org/issue?@action=redirect&bpo=6003].)

Novo módulo: importlib

Python 3.1 inclui o pacote **importlib**, uma reimplementação da lógica subjacente à instrução **import** do Python. **importlib** é útil para implementadores de interpretadores Python e para usuários que desejam escrever novos importadores que possam participar do processo de importação. Python 2.7 não contém o pacote **importlib** completo, mas em vez disso possui um pequeno subconjunto que contém uma única função, **import_module()**.

`import_module(name, package=None)` importa um módulo. *name* é uma string contendo o nome do módulo ou pacote. É possível fazer importações relativas fornecendo uma string que começa com um caractere `.`, como `..utils.errors`. Para importações relativas, o argumento *package* deve ser fornecido e é o nome do pacote que será usado como âncora para a importação relativa. **import_module()** insere o módulo importado em `sys.modules` e retorna o objeto do módulo.

Veja alguns exemplos:

```
>>> from importlib import import_module
>>> anydbm = import_module('anydbm')  # Importação absoluta padr
>>> anydbm
<module 'anydbm' from '/p/python/Lib/anydbm.py'>
>>> # Importação relativa
>>> file_util = import_module('..file_util', 'distutils.command'
>>> file_util
<module 'distutils.file_util' from '/python/Lib/distutils/file_u
```

importlib foi implementado por Brett Cannon e introduzido no Python 3.1.

Novo módulo: sysconfig

O módulo `sysconfig` foi retirado do pacote Distutils, tornando-se um novo módulo de nível independente. `sysconfig` fornece funções para obter informações sobre o processo de construção do Python: opções do compilador, caminhos de instalação, o nome da plataforma e se o Python está sendo executado a partir de seu diretório de código-fonte.

Algumas das funções do módulo são:

- `get_config_var()` retorna variáveis do Makefile do Python e do arquivo `pyconfig.h`.
- `get_config_vars()` retorna um dicionário contendo todas as variáveis de configuração.
- `get_path()` retorna o caminho configurado para um tipo específico de módulo: a biblioteca padrão, módulos específicos do site, módulos específicos da plataforma, etc.
- `is_python_build()` retorna verdadeiro se você estiver executando um binário de uma árvore de código-fonte em Python, e falso caso contrário.

Consulte a documentação do `sysconfig` para mais detalhes e para uma lista completa de funções.

O pacote Distutils e `sysconfig` agora são mantidos por Tarek Ziadé, que também iniciou um pacote Distutils2 (repositório fonte em https://hg.python.org/distutils2/) para desenvolver uma versão de próxima geração do Distutils.

ttk: Widgets temáticos para Tk

O Tcl/Tk 8.5 inclui um conjunto de widgets temáticos que reimplementam widgets Tk básicos, mas têm uma aparência mais personalizável e podem, portanto, se assemelhar mais aos widgets da plataforma nativa. Este conjunto de widgets foi originalmente chamado de Tile, mas foi renomeado para Ttk (para “Tk temático”) ao ser adicionado ao Tcl/Tck versão 8.5.

Para saber mais, leia a documentação do módulo `ttk`. Você também pode ler a página de manual do Tcl/Tk que descreve o mecanismo do tema Ttk, disponível em https://www.tcl.tk/man/tcl8.5/TkCmd/ttk_intro.html.

Algumas capturas de tela do código Python/Ttk em uso estão em https://code.google.com/archive/p/python-ttk/wikis/Screenshots.wiki.

O módulo **`tkinter.ttk`** foi escrito por Guilherme Polo e adicionado em bpo-2983 [https://bugs.python.org/issue?@action=redirect&bpo=2983]. Uma versão alternativa chamada `Tile.py`, escrita por Martin Franklin e mantida por Kevin Walzer, foi proposta para inclusão em bpo-2618 [https://bugs.python.org/issue?@action=redirect&bpo=2618], mas os autores argumentaram que o trabalho de Guilherme Polo era mais abrangente.

Módulo atualizado: unittest

O módulo **`unittest`** foi bastante aprimorado; muitos novos recursos foram adicionados. A maioria desses recursos foi implementada por Michael Foord, salvo indicação em contrário. A versão aprimorada do módulo pode ser baixada separadamente para uso com as versões 2.4 a 2.6 do Python, empacotadas como o pacote **`unittest2`**, de unittest2 [https://pypi.org/project/unittest2/].

Quando usado na linha de comando, o módulo pode descobrir testes automaticamente. Não é tão sofisticado quanto py.test [https://pytest.org] ou nose [https://nose.readthedocs.io/], mas fornece uma maneira simples de executar testes mantidos dentro de um conjunto de diretórios de pacotes. Por exemplo, o comando a seguir irá pesquisar no subdiretório `test/` por quaisquer arquivos de teste importáveis chamados `test*.py`:

```
python -m unittest discover -s test
```

Consulte a documentação do módulo **`unittest`** para mais detalhes. (Desenvolvido em bpo-6001 [https://bugs.python.org/issue?@action=redirect&bpo=6001].)

A função **`main()`** oferece suporte a algumas outras novas opções:

- **`-b`** ou **`--buffer`** armazenará em buffer a saída padrão e os fluxos de erro padrão durante cada teste. Se o teste for aprovado, qualquer saída resultante será descartada; em caso de falha, a saída em buffer será exibida.

- `-c` ou `--catch` fará com que a interrupção control-C seja tratada com mais elegância. Em vez de interromper imediatamente o processo de teste, o teste em execução será concluído e então os resultados parciais até a interrupção serão relatados. Se você estiver impaciente, um segundo toque no controle C causará uma interrupção imediata.

  Este manipulador de control-C tenta evitar causar problemas quando o código que está sendo testado ou os testes em execução definiram um manipulador de sinal próprio, percebendo que um manipulador de sinal já estava definido e chamando-o. Se isso não funcionar para você, existe um decorador `removeHandler()` que pode ser usado para marcar testes que devem ter o manuseio de control-C desabilitado.

- `-f` ou `--failfast` faz com que a execução do teste pare imediatamente quando um teste falha, em vez de continuar a executar testes adicionais. (Sugerido por Cliff Dyer e implementado por Michael Foord; bpo-8074 [https://bugs.python.org/issue?@action=redirect&bpo=8074].)

As mensagens de progresso agora mostram ‘x’ para falhas esperadas e ‘u’ para sucessos inesperados quando executadas em modo detalhado. (Contribuição de Benjamin Peterson.)

Os casos de teste podem levantar a exceção `SkipTest` para pular um teste (bpo-1034053 [https://bugs.python.org/issue?@action=redirect&bpo=1034053]).

As mensagens de erro para falhas `assertEqual()`, `assertTrue()` e `assertFalse()` agora fornecem mais informações. Se você definir o atributo `longMessage` de suas classes `TestCase` como true, tanto a mensagem de erro padrão quanto qualquer mensagem adicional que você fornecer serão impressas para falhas. (Adição de Michael Foord; bpo-5663 [https://bugs.python.org/issue?@action=redirect&bpo=5663].)

O método `assertRaises()` agora retorna um manipulador de contexto quando chamado sem fornecer um objeto chamável para ser executado. Por exemplo, você pode escrever isto:

```
with self.assertRaises(KeyError):
    {}['foo']
```

(implementação de Antoine Pitrou; bpo-4444 [https://bugs.python.org/issue?@action=redirect&bpo=4444].)

Correções de desmontagem e configuração em nível de módulo e classe agora são suportados. Os módulos podem conter funções `setUpModule()` e `tearDownModule()`. As classes podem ter métodos `setUpClass()` e `tearDownClass()` que devem ser definidos como métodos de classe (usando `@classmethod` ou equivalente). Essas funções e métodos são invocados quando o executor de teste alterna para um caso de teste em um módulo ou classe diferente.

Os métodos `addCleanup()` e `doCleanups()` foram adicionados. **`addCleanup()`** permite adicionar funções de limpeza que serão chamadas incondicionalmente (após `setUp()` se **`setUp()`** falhar, caso contrário, após `tearDown()`). Isso permite uma alocação e desalocação de recursos muito mais simples durante os testes (bpo-5679 [https://bugs.python.org/issue?@action=redirect&bpo=5679]).

Foram adicionados vários novos métodos que fornecem testes mais especializados. Muitos desses métodos foram escritos por engenheiros do Google para uso em seus conjuntos de testes; Gregory P. Smith, Michael Foord e GvR trabalharam na fusão deles na versão Python de `unittest`.

- `assertIsNone()` e `assertIsNotNone()` pegam uma expressão e verificam se o resultado é ou não `None`.
- `assertIs()` e `assertIsNot()` pegam dois valores e verificam se os dois valores são avaliados como o mesmo objeto ou não. (Adicionado por Michael Foord; bpo-2578 [https://bugs.python.org/issue?@action=redirect&bpo=2578].)
- `assertIsInstance()` e `assertNotIsInstance()` verificam se o objeto resultante é uma instância de uma classe específica ou de uma tupla de classes. (Adicionado por Georg Brandl; bpo-7031 [https://bugs.python.org/issue?@action=redirect&bpo=7031].)
- `assertGreater()`, `assertGreaterEqual()`, `assertLess()`, e `assertLessEqual()` comparam duas quantidades.
- `assertMultiLineEqual()` compara duas strings e, se não forem iguais, exibe uma comparação útil que destaca as diferenças nas duas strings.

Esta comparação agora é usada por padrão quando strings Unicode são comparadas com `assertEqual()`.

- `assertRegexpMatches()` e `assertNotRegexpMatches()` verificam se o primeiro argumento é uma string que corresponde ou não à expressão regular fornecida como o segundo argumento (bpo-8038 [https://bugs.python.org/issue?@action=redirect&bpo=8038]).
- `assertRaisesRegexp()` verifica se uma exceção específica foi levantada e também verifica se a representação da string da exceção corresponde à expressão regular fornecida.
- `assertIn()` e `assertNotIn()` testa se *primeiro* está ou não em *segundo*.
- `assertItemsEqual()` testa se duas sequências fornecidas contêm os mesmos elementos.
- `assertSetEqual()` compara se dois conjuntos são iguais e apenas relata as diferenças entre os conjuntos em caso de erro.
- Da mesma forma, `assertListEqual()` e `assertTupleEqual()` comparam os tipos especificados e explicam quaisquer diferenças sem necessariamente imprimir seus valores completos; esses métodos agora são usados por padrão ao comparar listas e tuplas usando `assertEqual()`. De forma mais geral, `assertSequenceEqual()` compara duas sequências e pode opcionalmente verificar se ambas as sequências são de um tipo específico.
- `assertDictEqual()` compara dois dicionários e relata as diferenças; agora é usado por padrão quando você compara dois dicionários usando `assertEqual()`. `assertDictContainsSubset()` verifica se todos os pares chave/valor em *first* são encontrados em *second*.
- `assertAlmostEqual()` e `assertNotAlmostEqual()` testam se *first* e *second* são aproximadamente iguais. Este método pode arredondar a diferença para um número opcionalmente especificado de *places* (o padrão é 7) e compará-lo com zero, ou exigir que a diferença seja menor que um valor *delta* fornecido.
- `loadTestsFromName()` respeita corretamente o atributo `suiteClass` do `TestLoader`. (Correção de Mark Roddy; bpo-6866 [https://bugs.python.org/issue?@action=redirect&bpo=6866].)
- Um novo gancho permite estender o método `assertEqual()` para lidar com novos tipos de dados. O método `addTypeEqualityFunc()` leva um objeto de tipo e uma função. A função será usada quando ambos os

objetos comparados forem do tipo especificado. Esta função deve comparar os dois objetos e gerar uma exceção se eles não corresponderem; é uma boa ideia que a função forneça informações adicionais sobre por que os dois objetos não correspondem, assim como fazem os novos métodos de comparação de sequências.

`unittest.main()` agora recebe um argumento `exit` opcional. Se for falso, `main()` não chama `sys.exit()`, permitindo que `main()` seja usado a partir do interpretador interativo. (Contribuição de J. Pablo Fernández; bpo-3379 [https://bugs.python.org/issue?@action=redirect&bpo=3379].)

`TestResult` tem novos métodos `startTestRun()` e `stopTestRun()` que são chamados imediatamente antes e depois de uma execução de teste. (Contribuição de Robert Collins; bpo-5728 [https://bugs.python.org/issue?@action=redirect&bpo=5728].)

Com todas essas mudanças, o `unittest.py` estava ficando estranhamente grande, então o módulo foi transformado em um pacote e o código dividido em vários arquivos (por Benjamin Peterson). Isso não afeta como o módulo é importado ou usado.

Ver também

https://web.archive.org/web/20210619163128/http://www.voidspace.org.uk/python/articles/unittest2.shtml
Descreve os novos recursos, como usá-los e a justificativa para diversas decisões de design. (Por Michael Foord.)

Módulo atualizado: ElementTree 1.3

A versão da biblioteca ElementTree incluída no Python foi atualizada para a versão 1.3. Alguns dos novos recursos são:

- As várias funções de análise agora recebem um argumento nomeado *parser* fornecendo uma instância `XMLParser` que será usada. Isso torna possível substituir a codificação interna do arquivo:

```
p = ET.XMLParser(encoding='utf-8')
t = ET.XML("""<root/>""", parser=p)
```

Erros na análise de XML agora levantam uma exceção **ParseError**, cujas instâncias possuem um atributo **position** contendo uma tupla (*linha*, *coluna*) fornecendo a localização do problema.

- O código do ElementTree para converter árvores em strings foi significativamente reformulado, tornando-o quase duas vezes mais rápido em muitos casos. Os métodos **ElementTree.write()** e **Element.write()** agora têm um parâmetro *method* que pode ser "xml" (o padrão), "html" ou "text". O modo HTML exibirá elementos vazios como `<empty></empty>` em vez de `<empty/>`, e o modo texto ignorará os elementos e exibirá apenas os blocos de texto. Se você definir o atributo **tag** de um elemento como `None`, mas deixar seus filhos no lugar, o elemento será omitido quando a árvore for gravada, portanto, não será necessário fazer uma reorganização mais extensa para remover um único elemento.

  O tratamento de espaços de nomes também foi aprimorado. Todas as declarações `xmlns:<alguma-coisa>` agora são geradas no elemento raiz, e não espalhadas pelo XML resultante. Você pode definir o espaço de nomes padrão para uma árvore definindo o atributo **default_namespace** e registrar novos prefixos com **register_namespace()**. No modo XML, você pode usar o parâmetro true/false *xml_declaration* para suprimir a declaração XML.

- Novo método **Element**: **extend()** anexa os itens de uma sequência aos filhos do elemento. Os próprios elementos se comportam como sequências, por isso é fácil mover os filhos de um elemento para outro:

```
from xml.etree import ElementTree as ET

t = ET.XML("""<list>
  <item>1</item> <item>2</item>  <item>3</item>
</list>""")
new = ET.XML('<root/>')
new.extend(t)
```

```
# Outputs <root><item>1</item>...</root>
print ET.tostring(new)
```

- Novo método **`Element`**: **`iter()`** produz os filhos do elemento como um gerador. Também é possível escrever `for child in elem:` para percorrer os filhos de um elemento. O método existente **`getiterator()`** agora está descontinuado, assim como **`getchildren()`** que constrói e retorna uma lista de filhos.

- Novo método **`Element`**: **`itertext()`** produz todos os pedaços de texto que são descendentes do elemento. Por exemplo:

```
t = ET.XML("""<list>
  <item>1</item> <item>2</item>  <item>3</item>
</list>""")

# Outputs ['\n  ', '1', ' ', '2', '  ', '3', '\n']
print list(t.itertext())
```

- Descontinuado: usar um elemento como booleano (ou seja, `if elem:`) retornaria verdadeiro se o elemento tivesse algum filho, ou falso se não houvesse nenhum filho. Este comportamento é confuso – `None` é falso, mas um elemento sem filhos também o é? – então agora irá acionar um **`FutureWarning`**. Em seu código, você deve ser explícito: escreva `len(elem) != 0` se estiver interessado no número de filhos, ou `elem is not None`.

Fredrik Lundh desenvolve ElementTree e produziu a versão 1.3; você pode ler o artigo dele descrevendo 1.3 em https://web.archive.org/web/20200703234532/http://effbot.org/zone/elementtree-13-intro.htm. Florent Xicluna atualizou a versão incluída no Python, após discussões em python-dev e em bpo-6472 [https://bugs.python.org/issue?@action=redirect&bpo=6472].)

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- O uso da versão mais recente do GNU Debugger, GDB 7, pode ser programado usando Python [https://web.archive.org/web/20110715084810/http://sourceware.org/gdb/current/onlinedocs/gdb/Python.html]. Quando você começa a depurar um programa executável P, o GDB irá procurar por um arquivo chamado `P-gdb.py` e lê-lo automaticamente. Dave Malcolm contribuiu com um `python-gdb.py` que adiciona vários comandos úteis ao depurar o próprio Python. Por exemplo, `py-up` e `py-down` sobem ou descem um quadro de pilha Python, que geralmente corresponde a vários quadros de pilha C. `py-print` imprime o valor de uma variável Python e `py-bt` imprime o stack trace do Python. (Adicionado como resultado de bpo-8032 [https://bugs.python.org/issue?@action=redirect&bpo=8032].)

- Se você usar o arquivo `.gdbinit` fornecido com Python, a macro "pyo" na versão 2.7 agora funciona corretamente quando o thread que está sendo depurado não contém a GIL; a macro agora a adquire antes de imprimir. (Contribuição de Victor Stinner; bpo-3632 [https://bugs.python.org/issue?@action=redirect&bpo=3632].)

- **`Py_AddPendingCall()`** agora é segura para thread, permitindo que qualquer thread de trabalho envie notificações para o thread principal do Python. Isto é particularmente útil para operações de E/S assíncronas. (Contribuição de Kristján Valur Jónsson; bpo-4293 [https://bugs.python.org/issue?@action=redirect&bpo=4293].)

- Nova função: **`PyCode_NewEmpty()`** cria um objeto código vazio; apenas o nome do arquivo, o nome da função e o número da primeira linha são necessários. Isto é útil para módulos de extensão que estão tentando construir uma pilha de traceback mais útil. Anteriormente, essas extensões precisavam chamar **`PyCode_New()`**, que tinha muito mais argumentos. (Adição de Jeffrey Yasskin.)

- Nova função: **`PyErr_NewExceptionWithDoc()`** cria uma nova classe de exceção, assim como a **`PyErr_NewException()`** existente faz, mas recebe um argumento `char *` extra contendo a docstring para a nova classe de exceção . (Adição de 'lekma' no rastreador de bugs do Python; bpo-7033 [https://bugs.python.org/issue?@action=redirect&bpo=7033].)

- Nova função: **PyFrame_GetLineNumber()** pega um objeto quadro e retorna o número da linha que o frame está executando atualmente. Anteriormente, o código precisaria obter o índice da instrução de bytecode atualmente em execução e, em seguida, procurar o número da linha correspondente a esse endereço. (Adição de Jeffrey Yasskin.)

- Novas funções: **PyLong_AsLongAndOverflow()** e **PyLong_AsLongLongAndOverflow()** aproxima um inteiro longo do Python como um long ou long long do C. Se o número for muito grande para caber no tipo de saída, um sinalizador *overflow* será definido e retornado ao chamador. (Contribuição de Case Van Horsen; bpo-7528 [https://bugs.python.org/issue?@action=redirect&bpo=7528] e bpo-7767 [https://bugs.python.org/issue?@action=redirect&bpo=7767].)

- Nova função: decorrente da reescrita da conversão string-para-float, uma nova função **PyOS_string_to_double()** foi adicionada. As antigas funções **PyOS_ascii_strtod()** e **PyOS_ascii_atof()** agora foram descontinuadas.

- Nova função: **PySys_SetArgvEx()** define o valor de `sys.argv` e pode opcionalmente atualizar `sys.path` para incluir o diretório que contém o script nomeado por `sys.argv[0]` dependendo do valor de um parâmetro *updatepath*.

  Esta função foi adicionada para fechar uma falha de segurança em aplicativos que incorporam Python. A função antiga, **PySys_SetArgv()**, sempre atualizava `sys.path`, e às vezes adicionava o diretório atual. Isso significava que, se você executasse uma aplicação incorporando Python em um diretório controlado por outra pessoa, os invasores poderiam colocar um módulo cavalo de Troia no diretório (digamos, um arquivo chamado `os.py`) que sua aplicação então usaria. importar e executar.

  Se você mantém uma aplicação C/C++ que incorpora Python, verifique se está chamando **PySys_SetArgv()** e considere cuidadosamente se a aplicação deve usar **PySys_SetArgvEx()** com *updatepath* definido como false.

Problema de segurança relatado como **CVE 2008-5983** [https://www.cve.org/CVERecord?id=CVE-2008-5983]; discutido em bpo-5753 [https://bugs.python.org/issue?@action=redirect&bpo=5753], e corrigido por Antoine Pitrou.

- Novas macros: os arquivos de cabeçalho do Python agora definem as seguintes macros: `Py_ISALNUM`, `Py_ISALPHA`, `Py_ISDIGIT`, `Py_ISLOWER`, `Py_ISSPACE`, `Py_ISUPPER`, `Py_ISXDIGIT`, `Py_TOLOWER` e `Py_TOUPPER`. Todas essas funções são análogas às macros padrão C para classificação de caracteres, mas ignoram a configuração de localidade atual, porque em vários lugares o Python precisa analisar caracteres de maneira independente de localidade. (Adição de Eric Smith; bpo-5793 [https://bugs.python.org/issue?@action=redirect&bpo=5793].)

- Função removida: **`PyEval_CallObject()`** agora está disponível apenas como uma macro. Uma versão de função estava sendo mantida para preservar a compatibilidade de links ABI, mas isso foi em 1997; certamente pode ser excluído agora. (Remoção de Antoine Pitrou; bpo-8276 [https://bugs.python.org/issue?@action=redirect&bpo=8276].)

- Novos códigos de formato: as funções **`PyString_FromFormat()`**, **`PyString_FromFormatV()`** e `PyErr_Format()` agora aceitam `%lld` e `%llu` códigos de formato para exibir os tipos long long de C. (Contribuição de Mark Dickinson; bpo-7228 [https://bugs.python.org/issue?@action=redirect&bpo=7228].)

- A complicada interação entre threads e forks de processos foi alterada. Anteriormente, o processo filho criado por `os.fork()` pode falhar porque o filho é criado com apenas um único thread em execução, a thread executando **`os.fork()`**. Se outros threads estivessem mantendo uma trava, como a trava de importação do Python, quando o fork fosse executado, a trava ainda seria marcada como “retida” no novo processo. Mas no processo filho nada liberaria a trava, pois os outros threads não foram replicados e o processo filho não seria mais capaz de realizar importações.

  Python 2.7 adquire a trava de importação antes de executar um `os.fork()`, e também limpará quaisquer travas criadas usando o

módulo **threading**. Módulos de extensão C que possuem travas internas, ou que chamam **fork()** eles próprios, não se beneficiarão desta limpeza.

(Correção de Thomas Wouters; bpo-1590864 [https://bugs.python.org/issue?@action=redirect&bpo=1590864].)

- A função **Py_Finalize()** agora chama a função interna **threading._shutdown()**; isso evita que algumas exceções sejam levantadas quando um interpretador é encerrado. (Patch de Adam Olsen; bpo-1722344 [https://bugs.python.org/issue?@action=redirect&bpo=1722344].)

- Ao usar a estrutura **PyMemberDef** para definir atributos de um tipo, o Python não permitirá mais que você tente excluir ou definir um atributo **T_STRING_INPLACE**.

- Símbolos globais definidos pelo módulo **ctypes** agora são prefixados com `Py`, ou com `_ctypes`. (Implementado por Thomas Heller; bpo-3102 [https://bugs.python.org/issue?@action=redirect&bpo=3102].)

- Nova opção de configuração: a opção **--with-system-expat** permite construir o módulo **pyexpat** para usar a biblioteca Expat do sistema. (Contribuição de Arfrever Frehtes Taifersar Arahesis; bpo-7609 [https://bugs.python.org/issue?@action=redirect&bpo=7609].)

- Nova opção de configuração: a opção **--with-valgrind** agora desativará o alocador pymalloc, o que é difícil para o detector de erros de memória Valgrind analisar corretamente. Valgrind será, portanto, melhor na detecção de vazamentos e sobrecargas de memória. (Contribuição de James Henstridge; bpo-2422 [https://bugs.python.org/issue?@action=redirect&bpo=2422].)

- Nova opção de configuração: agora você pode fornecer uma string vazia para **--with-dbmliborder=** para desabilitar todos os vários módulos DBM. (Adição de Arfrever Frehtes Taifersar Arahesis; bpo-6491 [https://bugs.python.org/issue?@action=redirect&bpo=6491].)

- O script **configure** agora verifica bugs de arredondamento de ponto flutuante em determinados chips Intel de 32 bits e define uma definição de pré-processador **`X87_DOUBLE_ROUNDING`**. Nenhum código usa essa definição no momento, mas ela está disponível para quem quiser. (Adicionado por Mark Dickinson; bpo-2937 [https://bugs.python.org/issue?@action=redirect&bpo=2937].)

  configure agora também define uma variável Makefile **`LDCXXSHARED`** para oferecer suporte à ligação C++. (Contribuição de Arfrever Frehtes Taifersar Arahesis; bpo-1222585 [https://bugs.python.org/issue?@action=redirect&bpo=1222585].)

- O processo de construção agora cria os arquivos necessários para suporte ao pkg-config. (Contribuição de Clinton Roy; bpo-3585 [https://bugs.python.org/issue?@action=redirect&bpo=3585].)

- O processo de construção agora prover suporte ao Subversion 1.7. (Contribuição de Arfrever Frehtes Taifersar Arahesis; bpo-6094 [https://bugs.python.org/issue?@action=redirect&bpo=6094].)

Capsules

Python 3.1 adiciona um novo tipo de dados C, **`PyCapsule`**, para fornecer uma API C para um módulo de extensão. Uma cápsula é essencialmente o detentor de um ponteiro C `void *` e é disponibilizada como um atributo de módulo; por exemplo, a API do módulo **`socket`** é exposta como `socket.CAPI`, e **`unicodedata`** expõe `ucnhash_CAPI`. Outras extensões podem importar o módulo, acessar seu dicionário para obter o objeto cápsula e então obter o ponteiro `void *`, que normalmente apontará para uma matriz de ponteiros para as diversas funções da API do módulo.

Já existe um tipo de dados usado para isso, **`PyCObject`**, mas ele não fornece segurança de tipo. Código maligno escrito em Python puro pode causar uma falha de segmentação pegando um **`PyCObject`** do módulo A e de alguma forma substituindo-o por **`PyCObject`** no módulo B. As cápsulas sabem seu próprio nome , e para obter o ponteiro é necessário fornecer o nome:

```
void *vtable;

if (!PyCapsule_IsValid(capsule, "mymodule.CAPI") {
        PyErr_SetString(PyExc_ValueError, "argument type invalic
        return NULL;
}

vtable = PyCapsule_GetPointer(capsule, "mymodule.CAPI");
```

Você tem certeza de que `vtable` aponta para o que você espera. Se uma cápsula diferente fosse passada, **`PyCapsule_IsValid()`** detectaria o nome incompatível e retornaria falso. Consulte Fornecendo uma API C para um módulo de extensão para mais informações sobre como usar esses objetos.

Python 2.7 agora usa cápsulas internamente para fornecer várias APIs de módulo de extensão, mas o **`PyCObject_AsVoidPtr()`** foi modificado para lidar com cápsulas, preservando a compatibilidade em tempo de compilação com a interface **`PyCObject`**. O uso de **`PyCObject_AsVoidPtr()`** sinalizará uma exceção **`PendingDeprecationWarning`**, que é silencioso por padrão.

Implementado em Python 3.1 e portado para 2.7 por Larry Hastings; discutido em bpo-5630 [https://bugs.python.org/issue?@action=redirect&bpo=5630].

Alterações específicas da porta: Windows

- O módulo **`msvcrt`** agora contém algumas constantes do arquivo de cabeçalho `crtassem.h`: **`CRT_ASSEMBLY_VERSION`**, **`VC_ASSEMBLY_PUBLICKEYTOKEN`** e **`LIBRARIES_ASSEMBLY_NAME_PREFIX`**. (Contribuição de David Cournapeau; bpo-4365 [https://bugs.python.org/issue?@action=redirect&bpo=4365].)
- O módulo **`_winreg`** para acessar o registro agora implementa as funções **`CreateKeyEx()`** e **`DeleteKeyEx()`**, versões estendidas de funções suportadas anteriormente que recebem vários argumentos extras. Os **`DisableReflectionKey()`**, **`EnableReflectionKey()`** e **`QueryReflectionKey()`** também foram testados e documentados. (Implementação de Brian Curtin: bpo-7347 [https://bugs.python.org/issue?@action=redirect&bpo=7347].)

- A nova API **`_beginthreadex()`** é usada para iniciar threads, e as funções nativas de armazenamento local de thread agora são usadas. (Contribuição de Kristján Valur Jónsson; bpo-3582 [https://bugs.python.org/issue?@action=redirect&bpo=3582].)
- A função **`os.kill()`** agora funciona no Windows. O valor do sinal pode ser as constantes **`CTRL_C_EVENT`**, **`CTRL_BREAK_EVENT`** ou qualquer número inteiro. As duas primeiras constantes enviarão eventos de pressionamento de tecla `Control-C` e `Control-Break` para subprocessos; qualquer outro valor usará a API **`TerminateProcess()`**. (Contribuição de Miki Tebeka; bpo-1220212 [https://bugs.python.org/issue?@action=redirect&bpo=1220212].)
- A função **`os.listdir()`** agora falha corretamente para um caminho vazio. (Correção de Hirokazu Yamamoto; bpo-5913 [https://bugs.python.org/issue?@action=redirect&bpo=5913].)
- O módulo **`mimetypes`** agora irá ler o banco de dados MIME do registro do Windows ao inicializar. (Patch de Gabriel Genellina; bpo-4969 [https://bugs.python.org/issue?@action=redirect&bpo=4969].)

Alterações específicas da porta: Mac OS X

- O caminho `/Library/Python/2.7/site-packages` agora é anexado a `sys.path`, para compartilhar pacotes adicionados entre a instalação do sistema e uma cópia da mesma versão instalada pelo usuário. (Alteração de Ronald Oussoren; bpo-4865 [https://bugs.python.org/issue?@action=redirect&bpo=4865].)

  Alterado na versão 2.7.13: A partir da versão 2.7.13, esta alteração foi removida. `/Library/Python/2.7/site-packages`, o diretório site-packages usado pelo sistema Python 2.7 fornecido pela Apple não é mais anexado a `sys.path` para Pythons instalados pelo usuário, como do Instaladores do python.org. A partir do macOS 10.12, a Apple mudou a forma como o diretório site-packages do sistema é configurado, o que pode causar falha na instalação de componentes pip, como setuptools. Os pacotes instalados para o sistema Python não serão mais compartilhados com Pythons instalados pelo usuário. (bpo-28440 [https://bugs.python.org/issue?@action=redirect&bpo=28440])

Alterações específicas da porta: FreeBSD

- Constante **`SO_SETFIB`** do FreeBSD 7.1, usada com os métodos **`getsockopt()`** e **`setsockopt()`** do **`socket()`** para selecionar uma tabela de roteamento alternativa, agora está disponível no módulo **`socket`**. (Adição de Kyle VanderBeek; bpo-8235 [https://bugs.python.org/issue?@action=redirect&bpo=8235].)

Outras alterações e correções

- Dois scripts de benchmark, `iobench` e `ccbench`, foram adicionados ao diretório `Tools`. `iobench` mede a velocidade dos objetos embutidos de E/S de arquivo retornados por **`open()`** durante a execução de várias operações, e `ccbench` é um benchmark de simultaneidade que tenta medir o rendimento da computação, latência de comutação de threads e largura de banda de processamento de E/S ao executar diversas tarefas usando um número variável de threads.
- O script `Tools/i18n/msgfmt.py` agora entende formas plurais em arquivos `.po`. (Correção de Martin von Löwis; bpo-5464 [https://bugs.python.org/issue?@action=redirect&bpo=5464].)
- Ao importar um módulo de um arquivo `.pyc` ou `.pyo` com uma contraparte `.py` existente, os atributos **`co_filename`** dos objetos código resultantes são substituídos quando o nome do arquivo original fica obsoleto. Isso pode acontecer se o arquivo tiver sido renomeado, movido ou acessado por caminhos diferentes. (Patch de Ziga Seilnacht e Jean-Paul Calderone; bpo-1180193 [https://bugs.python.org/issue?@action=redirect&bpo=1180193].)
- O script `regrtest.py` agora usa uma opção **`--randseed=`** que recebe um número inteiro que será usado como semente aleatória para a opção **`-r`** que executa testes em ordem aleatória. A opção **`-r`** também informa a semente que foi usada (Adição de Collin Winter.)
- Outra opção `regrtest.py` é **`-j`**, que recebe um número inteiro especificando quantos testes são executados em paralelo. Isto permite reduzir o tempo total de execução em máquinas com vários núcleos. Esta opção é compatível com várias outras opções, incluindo a opção **`-R`** que é conhecida por produzir longos tempos de execução.

(Adicionado por Antoine Pitrou, bpo-6152 [https://bugs.python.org/issue?@action=redirect&bpo=6152].) Isso também pode ser usado com uma nova opção `-F` que executa testes selecionados em um loop até que eles falhem. (Adição de Antoine Pitrou; bpo-7312 [https://bugs.python.org/issue?@action=redirect&bpo=7312].)

- Quando executado como um script, o módulo `py_compile.py` agora aceita `'-'` como argumento, que lerá a entrada padrão para a lista de nomes de arquivos a serem compilados. (Contribuição de Piotr Ożarowski; bpo-8233 [https://bugs.python.org/issue?@action=redirect&bpo=8233].)

Portando para Python 2.7

Esta seção lista as alterações descritas anteriormente e outras correções de bugs que podem exigir alterações em seu código:

- A função `range()` processa seus argumentos de forma mais consistente; agora ele chamará `__int__()` em argumentos que não são ponto flutuantes nem inteiros que são fornecidos a ele. (Correção de Alexander Belopolsky; bpo-1533 [https://bugs.python.org/issue?@action=redirect&bpo=1533].)
- O método string `format()` alterou a precisão padrão usada para números de ponto flutuante e complexos de 6 casas decimais para 12, o que corresponde à precisão usada por `str()`. (Alteração de Eric Smith; bpo-5920 [https://bugs.python.org/issue?@action=redirect&bpo=5920].)
- Por causa de uma otimização para a instrução `with`, os métodos especiais `__enter__()` e `__exit__()` devem pertencer ao tipo do objeto e não podem ser diretamente anexados ao instância do objeto. Isto afeta classes de novo estilo (derivadas de `object`) e tipos de extensão C. (bpo-6101 [https://bugs.python.org/issue?@action=redirect&bpo=6101].)
- Devido a um bug no Python 2.6, o parâmetro *exc_value* para os métodos `__exit__()` era frequentemente a representação de string da exceção, não uma instância. Isso foi corrigido na versão 2.7, então *exc_value* será uma instância conforme esperado. (Correção de Florent Xicluna; bpo-7853 [https://bugs.python.org/issue?@action=redirect&bpo=7853].)
- Quando um conjunto restrito de atributos foi definido usando `__slots__`, excluir um atributo não definido não levantaria

`AttributeError` como seria de esperar. Correção de Benjamin Peterson; bpo-7604 [https://bugs.python.org/issue?@action=redirect&bpo=7604].)

Na biblioteca padrão:

- Operações com instâncias `datetime` que resultaram em um ano fora do intervalo suportado nem sempre levantam `OverflowError`. Esses erros agora são verificados com mais cuidado e irão levantar a exceção. (Relato de Mark Leander, patch de Anand B. Pillai e Alexander Belopolsky; bpo-7150 [https://bugs.python.org/issue?@action=redirect&bpo=7150].)

- Ao usar instâncias `Decimal` com o método `format()` de uma string, o alinhamento padrão era anteriormente o alinhamento à esquerda. Isso foi alterado para alinhamento à direita, o que pode alterar a saída dos seus programas. (Alteração de Mark Dickinson; bpo-6857 [https://bugs.python.org/issue?@action=redirect&bpo=6857].)

  Comparações envolvendo um valor NaN de sinalização (ou `sNAN`) agora sinalizam `InvalidOperation` em vez de retornar silenciosamente um valor verdadeiro ou falso dependendo do operador de comparação. Valores silenciosos de NaN (ou `NaN`) agora são hasheáveis. (Corrigido por Mark Dickinson; bpo-7279 [https://bugs.python.org/issue?@action=redirect&bpo=7279].)

- A biblioteca `xml.etree.ElementTree` não escapa mais os caracteres de e-comercial e colchetes angulares ao gerar uma instrução de processamento XML (que se parece com `<?xml-stylesheet href="#style1"?>`) ou comentário (que se parece com `<!-- comment -->`). (Patch de Neil Muller; bpo-2746 [https://bugs.python.org/issue?@action=redirect&bpo=2746].)

- O método `readline()` dos objetos `StringIO` agora não faz nada quando um comprimento negativo é solicitado, como fazem outros objetos arquivo ou similares. (bpo-7348 [https://bugs.python.org/issue?@action=redirect&bpo=7348]).

- O módulo `syslog` agora usará o valor de `sys.argv[0]` como identificador em vez do valor padrão anterior de `'python'`. (Alteração

de Sean Reifschneider; bpo-8451 [https://bugs.python.org/issue?@action=redirect&bpo=8451].)

- O tratamento de erros padrão do módulo **`tarfile`** foi alterado, para não mais suprimir erros fatais. O nível de erro padrão era anteriormente 0, o que significava que os erros resultariam apenas na gravação de uma mensagem no log de depuração, mas como o log de depuração não está ativado por padrão, esses erros passam despercebidos. O nível de erro padrão agora é 1, o que levanta uma exceção se houver um erro. (Alteração de Lars Gustäbel; bpo-7357 [https://bugs.python.org/issue?@action=redirect&bpo=7357].)

- A função **`urlsplit()`** do módulo **`urlparse`** agora lida com esquemas de URL desconhecidos de uma forma compatível com **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html]: se a URL estiver no formato "`<alguma_coisa>://...`", o texto antes de `://` é tratado como o esquema, mesmo que seja um esquema inventado que o módulo não conhece. Essa alteração pode quebrar o código que contornava o comportamento antigo. Por exemplo, Python 2.6.4 ou 2.5 retornará o seguinte:

  ```
  >>> import urlparse
  >>> urlparse.urlsplit('invented://host/filename?query')
  ('invented', '', '//host/filename?query', '', '')
  ```

  Python 2.7 (e Python 2.6.5) vai retornar:

  ```
  >>> import urlparse
  >>> urlparse.urlsplit('invented://host/filename?query')
  ('invented', 'host', '/filename?query', '', '')
  ```

  (Na verdade, o Python 2.7 produz uma saída ligeiramente diferente, pois retorna uma tupla nomeada em vez de uma tupla padrão.)

Para extensões C:

- Extensões C que usam códigos de formato inteiro com a família de funções `PyArg_Parse*` agora levantarão uma exceção **`TypeError`** em

vez de disparar uma **`DeprecationWarning`** (bpo-5080 [https://bugs.python.org/issue?@action=redirect&bpo=5080]).
- Use a nova função **`PyOS_string_to_double()`** em vez das antigas funções **`PyOS_ascii_strtod()`** e **`PyOS_ascii_atof()`**, que agora estão descontinuadas.

Para aplicações que incorporam Python:

- A função **`PySys_SetArgvEx()`** foi adicionada, permitindo que aplicativos fechem uma falha de segurança quando a função **`PySys_SetArgv()`** existente for usada. Verifique se você está chamando **`PySys_SetArgv()`** e considere cuidadosamente se a aplicação deve usar **`PySys_SetArgvEx()`** com *updatepath* definido como falso.

Novos recursos adicionados às versões de manutenção do Python 2.7

Novos recursos podem ser adicionados às versões de manutenção do Python 2.7 quando a situação realmente exigir. Quaisquer adições devem passar pelo processo de proposta de melhoria do Python, ou em inglês Python Enhancement Proposal – PEP, e apresentar um argumento convincente sobre por que não podem ser abordadas adequadamente adicionando o novo recurso apenas ao Python 3 ou publicando-o no Python Package Index.

Além das propostas específicas listadas abaixo, há uma isenção geral que permite que novos avisos `-3` sejam adicionados em qualquer versão de manutenção do Python 2.7.

Duas novas variáveis de ambiente para modo de depuração

No modo de depuração, a estatística `[xxx refs]` não é escrita por padrão, a variável de ambiente **`PYTHONSHOWREFCOUNT`** agora também deve ser definida. (Contribuição de Victor Stinner; bpo-31733 [https://bugs.python.org/issue?@action=redirect&bpo=31733].)

Quando Python é compilado com `COUNT_ALLOC` definido, as contagens de alocação não são mais despejadas por padrão: a variável de ambiente **`PYTHONSHOWALLOCCOUNT`** agora também deve ser definida. Além disso, as contagens de alocação agora são despejadas no stderr, em vez do stdout. (Contribuição de Victor Stinner; bpo-31692 [https://bugs.python.org/issue?@action=redirect&bpo=31692].)

Adicionado na versão 2.7.15.

PEP 434: Exceção de aprimoramento do IDLE para todos os branches

PEP 434 [https://peps.python.org/pep-0434/] descreve uma isenção geral para alterações feitas no ambiente de desenvolvimento IDLE enviado junto com o Python. Esta isenção permite que os desenvolvedores de IDLE forneçam uma experiência de usuário mais consistente em todas as versões suportadas do Python 2 e 3.

Para obter detalhes sobre quaisquer alterações do IDLE, consulte o arquivo NEWS da versão específica.

PEP 466: Aprimoramentos de segurança de rede do Python 2.7

PEP 466 [https://peps.python.org/pep-0466/] descreve uma série de propostas de melhoria de segurança de rede que foram aprovadas para inclusão nas versões de manutenção do Python 2.7, com a primeira dessas mudanças aparecendo na versão Python 2.7.7.

Recursos relacionados a **PEP 466** [https://peps.python.org/pep-0466/] adicionados no Python 2.7.7:

- **`hmac.compare_digest()`** foi portado do Python 3 para disponibilizar uma operação de comparação resistente a ataques de tempo para aplicações em Python 2. (Contribuição de Alex Gaynor; bpo-21306 [https://bugs.python.org/issue?@action=redirect&bpo=21306].)
- OpenSSL 1.0.1g foi atualizado nos instaladores oficiais do Windows publicados em python.org. (Contribuição de Zachary Ware; bpo-21462

[https://bugs.python.org/issue?@action=redirect&bpo=21462].)

Recursos relacionados a **PEP 466** [https://peps.python.org/pep-0466/] adicionados no Python 2.7.8:

- `hashlib.pbkdf2_hmac()` foi portado do Python 3 para tornar um algoritmo de hash adequado para armazenamento seguro de senhas amplamente disponível para aplicações Python 2. (Contribuição de Alex Gaynor; bpo-21304 [https://bugs.python.org/issue?@action=redirect&bpo=21304].)
- OpenSSL 1.0.1h foi atualizado para os instaladores oficiais do Windows publicados em python.org. (Contribuição de Zachary Ware em bpo-21671 [https://bugs.python.org/issue?@action=redirect&bpo=21671] para **CVE 2014-0224** [https://www.cve.org/CVERecord?id=CVE-2014-0224].)

Recursos relacionados a **PEP 466** [https://peps.python.org/pep-0466/] adicionados no Python 2.7.9:

- A maior parte do módulo `ssl` do Python 3.4 foi portada. Isso significa que `ssl` agora provê suporte a indicar de nome de servidor, configurações TLS1.x, acesso ao armazenamento de certificados da plataforma, a classe `SSLContext` e outros recursos. (Contribuição de Alex Gaynor e David Reid; bpo-21308 [https://bugs.python.org/issue?@action=redirect&bpo=21308].)

  Consulte as notas “Versão adicionada: 2.7.9” na documentação do módulo para obter detalhes específicos.

- `os.urandom()` foi alterado para armazenar em cache um descritor de arquivo para `/dev/urandom` em vez de reabrir `/dev/urandom` em cada chamada. (Contribuição de Alex Gaynor; bpo-21305 [https://bugs.python.org/issue?@action=redirect&bpo=21305].)

- `hashlib.algorithms_guaranteed` e `hashlib.algorithms_available` foram portados do Python 3 para tornar mais fácil para os aplicações Python 2 selecionarem o algoritmo de hash mais forte disponível. (Contribuição de Alex Gaynor em bpo-21307 [https://bugs.python.org/issue?@action=redirect&bpo=21307])

PEP 477: Backport do ensurepip (PEP 453) para o Python 2.7

A **PEP 477** [https://peps.python.org/pep-0477/] aprova a inclusão do módulo ensurepip do **PEP 453** [https://peps.python.org/pep-0453/] e da documentação aprimorada que foi habilitada por ele nas versões de manutenção do Python 2.7, aparecendo primeiro na versão Python 2.7.9.

Inicializando pip por padrão

O novo módulo **`ensurepip`** (definido em **PEP 453** [https://peps.python.org/pep-0453/]) fornece um mecanismo padrão de plataforma cruzada para inicializar o instalador pip em instalações Python. A versão do `pip` incluída no Python 2.7.9 é `pip` 1.5.6, e futuras versões de manutenção 2.7.x atualizarão a versão empacotada para a versão mais recente do `pip` que está disponível em o momento da criação do candidato a lançamento.

Por padrão, os comandos `pip`, `pipX` e `pipX.Y` serão instalados em todas as plataformas (onde X.Y representa a versão da instalação do Python), junto com o pacote Python via `pip` e suas dependências.

Para construções de fontes do CPython em sistemas POSIX, os comandos `make install` e `make altinstall` não inicializam `pip` por padrão. Este comportamento pode ser controlado através de opções de configuração e substituído através de opções do Makefile.

No Windows e Mac OS X, os instaladores do CPython agora instalam o `pip` junto com o próprio CPython (os usuários podem optar por não instalá-lo durante o processo de instalação). Os usuários do Windows precisarão optar pelas modificações automáticas do `PATH` para ter o `pip` disponível na linha de comando por padrão, caso contrário ele ainda poderá ser acessado através do iniciador Python para Windows como `py -m pip`.

Como **discutido na PEP** [https://peps.python.org/pep-0477/#disabling-ensurepip-by-downstream-distributors], os empacotadores de plataforma podem optar por não instalar estes comandos por padrão, desde que, quando invocados, forneçam instruções claras e simples sobre como instalá-los naquela plataforma (geralmente usando o gerenciador de pacotes do sistema). .

Mudanças na documentação

Como parte desta mudança, as seções Instalando módulos Python e Distribuindo módulos Python da documentação foram completamente redesenhadas como breves documentos de introdução e FAQ. A maior parte da documentação de empacotamento agora foi transferida para o Python Packaging Authority, mantido no Guia de Usuário para Empacotamento de Python [https://packaging.python.org] e na documentação dos projetos individuais.

No entanto, como esta migração ainda está incompleta, as versões legadas desses guias permanecem disponíveis como Construindo extensões C e C ++ com setuptools e Construindo extensões C e C ++ com setuptools.

Ver também

PEP 453 [https://peps.python.org/pep-0453/] – Inicialização explícita do pip em instalações Python

PEP escrita por Donald Stufft e Nick Coghlan, implementada por Donald Stufft, Nick Coghlan, Martin von Löwis e Ned Deily.

PEP 476: Habilitando verificação de certificado por padrão para clientes http stdlib

PEP 476 [https://peps.python.org/pep-0476/] atualizou `httplib` e módulos que o utilizam, como `urllib2` e `xmlrpclib`, para agora verificar se o servidor apresenta um certificado que é assinado por uma Autoridade Certificadora no armazenamento confiável da plataforma e cujo nome de host corresponde ao nome de host solicitado por padrão, melhorando significativamente a segurança para muitas aplicações. Essa alteração foi feita na versão 2.7.9 do Python.

Para aplicações que exigem o antigo comportamento anterior, elas podem passar um contexto alternativo:

```
import urllib2
import ssl
```

```
# Isso desabilita toda verificação
context = ssl._create_unverified_context()

# Isso permite usar um certificado específico para o host,
# o que não precisa ser no armazenamento de confiança
context = ssl.create_default_context(cafile="/path/to/file.crt")

urllib2.urlopen("https://invalid-cert", context=context)
```

PEP 493: Ferramentas de migração de verificação HTTPS para Python 2.7

PEP 493 [https://peps.python.org/pep-0493/] fornece ferramentas de migração adicionais para provê um processo de atualização de infraestrutura mais incremental para ambientes contendo aplicações e serviços que dependem do processamento historicamente permissivo de certificados de servidor ao estabelecer conexões HTTPS de clientes. Essas adições foram feitas na versão 2.7.12 do Python.

Estas ferramentas destinam-se a ser utilizadas em casos em que as aplicações e serviços afetados não podem ser modificados para passar explicitamente um contexto SSL mais permissivo ao estabelecer a ligação.

Para aplicações e serviços que não podem ser modificados, a nova variável de ambiente `PYTHONHTTPSVERIFY` pode ser definida como `0` para reverter todo um processo Python de volta ao comportamento permissivo padrão do Python 2.7.8 e anteriores.

Para casos em que o código de estabelecimento da conexão não pode ser modificado, mas a aplicação geral pode, a nova função **`ssl._https_verify_certificates()`** pode ser usada para ajustar o comportamento padrão em tempo de execução.

Novo alvo de construção `make regen-all`

Para simplificar a compilação cruzada e garantir que o CPython possa ser compilado de forma confiável sem exigir que uma versão existente do

Python já esteja disponível, o sistema de construção baseado em autotools não tenta mais recompilar implicitamente os arquivos gerados com base nos tempos de modificação dos arquivos.

Em vez disso, um novo comando `make regen-all` foi adicionado para forçar a regeneração desses arquivos quando desejado (por exemplo, após uma versão inicial do Python já ter sido construída com base nas versões pré-geradas).

Alvos de regeneração mais seletivos também são definidos – veja Makefile.pre.in [https://github.com/python/cpython/tree/3.15/Makefile.pre.in] para detalhes.

(Contribuição de Victor Stinner em bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404].)

Adicionado na versão 2.7.14.

Remoção do alvo de construção `make touch`

O alvo de construção `make touch` usado anteriormente para solicitar regeneração implícita de arquivos gerados atualizando seus tempos de modificação foi removido.

Ele foi substituído pelo novo alvo `make regen-all`.

(Contribuição de Victor Stinner em bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404].)

Alterado na versão 2.7.14.

Reconhecimentos

O autor gostaria de agradecer às seguintes pessoas por oferecerem sugestões, correções e assistência com vários rascunhos deste artigo: Nick Coghlan, Philip Jenvey, Ryan Lovett, R. David Murray, Hugh Secker-Walker.

O que há de novo no Python 2.6

Autor:: A.M. Kuchling (amk at amk.ca)

This article explains the new features in Python 2.6, released on October 1, 2008. The release schedule is described in **PEP 361** [https://peps.python.org/pep-0361/].

The major theme of Python 2.6 is preparing the migration path to Python 3.0, a major redesign of the language. Whenever possible, Python 2.6 incorporates new features and syntax from 3.0 while remaining compatible with existing code by not removing older features or syntax. When it's not possible to do that, Python 2.6 tries to do what it can, adding compatibility functions in a `future_builtins` module and a `-3` switch to warn about usages that will become unsupported in 3.0.

Some significant new packages have been added to the standard library, such as the `multiprocessing` and `json` modules, but there aren't many new features that aren't related to Python 3.0 in some way.

Python 2.6 also sees a number of improvements and bugfixes throughout the source. A search through the change logs finds there were 259 patches applied and 612 bugs fixed between Python 2.5 and 2.6. Both figures are likely to be underestimates.

This article doesn't attempt to provide a complete specification of the new features, but instead provides a convenient overview. For full details, you should refer to the documentation for Python 2.6. If you want to understand the rationale for the design and implementation, refer to the PEP for a particular new feature. Whenever possible, "What's New in Python" links to the bug/patch item for each change.

Python 3.0

The development cycle for Python versions 2.6 and 3.0 was synchronized, with the alpha and beta releases for both versions being made on the same days. The development of 3.0 has influenced many features in 2.6.

Python 3.0 is a far-ranging redesign of Python that breaks compatibility with the 2.x series. This means that existing Python code will need some conversion in order to run on Python 3.0. However, not all the changes in 3.0 necessarily break compatibility. In cases where new features won't cause existing code to break, they've been backported to 2.6 and are described in this document in the appropriate place. Some of the 3.0-derived features are:

- A `__complex__()` method for converting objects to a complex number.
- Alternate syntax for catching exceptions: `except TypeError as exc`.
- The addition of `functools.reduce()` as a synonym for the built-in `reduce()` function.

Python 3.0 adds several new built-in functions and changes the semantics of some existing builtins. Functions that are new in 3.0 such as `bin()` have simply been added to Python 2.6, but existing builtins haven't been changed; instead, the `future_builtins` module has versions with the new 3.0 semantics. Code written to be compatible with 3.0 can do `from future_builtins import hex, map` as necessary.

A new command-line switch, `-3`, enables warnings about features that will be removed in Python 3.0. You can run code with this switch to see how much work will be necessary to port code to 3.0. The value of this switch is available to Python code as the boolean variable `sys.py3kwarning`, and to C extension code as `Py_Py3kWarningFlag`.

Ver também

The 3*xxx* series of PEPs, which contains proposals for Python 3.0. **PEP 3000** [https://peps.python.org/pep-3000/] describes the development process for Python 3.0. Start with **PEP 3100** [https://peps.python.org/pep-3100/] that describes the general goals for Python 3.0, and then explore the higher-numbered PEPs that propose specific features.

Changes to the Development Process

While 2.6 was being developed, the Python development process underwent two significant changes: we switched from SourceForge's issue tracker to a customized Roundup installation, and the documentation was converted from LaTeX to reStructuredText.

New Issue Tracker: Roundup

For a long time, the Python developers had been growing increasingly annoyed by SourceForge's bug tracker. SourceForge's hosted solution doesn't permit much customization; for example, it wasn't possible to customize the life cycle of issues.

The infrastructure committee of the Python Software Foundation therefore posted a call for issue trackers, asking volunteers to set up different products and import some of the bugs and patches from SourceForge. Four different trackers were examined: Jira [https://www.atlassian.com/software/jira/], Launchpad [https://launchpad.net/], Roundup [https://roundup.sourceforge.io/], and Trac [https://trac.edgewall.org/]. The committee eventually settled on Jira and Roundup as the two candidates. Jira is a commercial product that offers no-cost hosted instances to free-software projects; Roundup is an open-source project that requires volunteers to administer it and a server to host it.

After posting a call for volunteers, a new Roundup installation was set up at https://bugs.python.org. One installation of Roundup can host multiple trackers, and this server now also hosts issue trackers for Jython and for the Python web site. It will surely find other uses in the future. Where possible, this edition of "What's New in Python" links to the bug/patch item for each change.

Hosting of the Python bug tracker is kindly provided by Upfront Systems [https://upfrontsoftware.co.za] of Stellenbosch, South Africa. Martin von Löwis put a lot of effort into importing existing bugs and patches from SourceForge; his scripts for this import operation are at

`https://svn.python.org/view/tracker/importer/` and may be useful to other projects wishing to move from SourceForge to Roundup.

Ver também

https://bugs.python.org
The Python bug tracker.

https://bugs.jython.org:
The Jython bug tracker.

https://roundup.sourceforge.io/
Roundup downloads and documentation.

https://svn.python.org/view/tracker/importer/
Martin von Löwis's conversion scripts.

New Documentation Format: reStructuredText Using Sphinx

The Python documentation was written using LaTeX since the project started around 1989. In the 1980s and early 1990s, most documentation was printed out for later study, not viewed online. LaTeX was widely used because it provided attractive printed output while remaining straightforward to write once the basic rules of the markup were learned.

Today LaTeX is still used for writing publications destined for printing, but the landscape for programming tools has shifted. We no longer print out reams of documentation; instead, we browse through it online and HTML has become the most important format to support. Unfortunately, converting LaTeX to HTML is fairly complicated and Fred L. Drake Jr., the long-time Python documentation editor, spent a lot of time maintaining the conversion process. Occasionally people would suggest converting the documentation into SGML and later XML, but performing a good conversion is a major task and no one ever committed the time required to finish the job.

During the 2.6 development cycle, Georg Brandl put a lot of effort into building a new toolchain for processing the documentation. The resulting package is called Sphinx, and is available from https://www.sphinx-doc.org/.

Sphinx concentrates on HTML output, producing attractively styled and modern HTML; printed output is still supported through conversion to LaTeX. The input format is reStructuredText, a markup syntax supporting custom extensions and directives that is commonly used in the Python community.

Sphinx is a standalone package that can be used for writing, and almost two dozen other projects (listed on the Sphinx web site [https://www.sphinx-doc.org/en/master/examples.html]) have adopted Sphinx as their documentation tool.

Ver também

Documenting Python **[https://devguide.python.org/documenting/]**
Describes how to write for Python's documentation.

Sphinx **[https://www.sphinx-doc.org/]**
Documentation and code for the Sphinx toolchain.

Docutils **[https://docutils.sourceforge.io]**
The underlying reStructuredText parser and toolset.

PEP 343: The 'with' statement

The previous version, Python 2.5, added the '`with`' statement as an optional feature, to be enabled by a `from __future__ import with_statement` directive. In 2.6 the statement no longer needs to be specially enabled; this means that `with` is now always a keyword. The rest of this section is a copy of the corresponding section from the "What's New in Python 2.5" document; if you're familiar with the '`with`' statement from Python 2.5, you can skip this section.

The ‘`with`’ statement clarifies code that previously would use `try...finally` blocks to ensure that clean-up code is executed. In this section, I’ll discuss the statement as it will commonly be used. In the next section, I’ll examine the implementation details and show how to write objects for use with this statement.

The ‘`with`’ statement is a control-flow structure whose basic structure is:

```
with expression [as variable]:
    with-block
```

The expression is evaluated, and it should result in an object that supports the context management protocol (that is, has `__enter__()` and `__exit__()` methods).

The object’s `__enter__()` is called before *with-block* is executed and therefore can run set-up code. It also may return a value that is bound to the name *variable*, if given. (Note carefully that *variable* is *not* assigned the result of *expression*.)

After execution of the *with-block* is finished, the object’s `__exit__()` method is called, even if the block raised an exception, and can therefore run clean-up code.

Some standard Python objects now support the context management protocol and can be used with the ‘`with`’ statement. File objects are one example:

```
with open('/etc/passwd', 'r') as f:
    for line in f:
        print line
        ... more processing code ...
```

After this statement has executed, the file object in *f* will have been automatically closed, even if the `for` loop raised an exception part-way through the block.

Nota

In this case, *f* is the same object created by `open()`, because `__enter__()` returns *self*.

The `threading` module's locks and condition variables also support the '`with`' statement:

```
lock = threading.Lock()
with lock:
    # Critical section of code
    ...
```

The lock is acquired before the block is executed and always released once the block is complete.

The `localcontext()` function in the `decimal` module makes it easy to save and restore the current decimal context, which encapsulates the desired precision and rounding characteristics for computations:

```
from decimal import Decimal, Context, localcontext

# Displays with default precision of 28 digits
v = Decimal('578')
print v.sqrt()

with localcontext(Context(prec=16)):
    # All code in this block uses a precision of 16 digits.
    # The original context is restored on exiting the block.
    print v.sqrt()
```

Writing Context Managers

Under the hood, the '`with`' statement is fairly complicated. Most people will only use '**`with`**' in company with existing objects and don't need to know these details, so you can skip the rest of this section if you like. Authors of new objects will need to understand the details of the underlying implementation and should keep reading.

A high-level explanation of the context management protocol is:

- The expression is evaluated and should result in an object called a "context manager". The context manager must have `__enter__()` and `__exit__()` methods.
- The context manager's `__enter__()` method is called. The value returned is assigned to *VAR*. If no `as VAR` clause is present, the value is simply discarded.
- The code in *BLOCK* is executed.
- If *BLOCK* raises an exception, the context manager's `__exit__()` method is called with three arguments, the exception details (`type, value, traceback`, the same values returned by `sys.exc_info()`, which can also be `None` if no exception occurred). The method's return value controls whether an exception is re-raised: any false value re-raises the exception, and `True` will result in suppressing it. You'll only rarely want to suppress the exception, because if you do the author of the code containing the '`with`' statement will never realize anything went wrong.
- If *BLOCK* didn't raise an exception, the `__exit__()` method is still called, but *type*, *value*, and *traceback* are all `None`.

Let's think through an example. I won't present detailed code but will only sketch the methods necessary for a database that supports transactions.

(For people unfamiliar with database terminology: a set of changes to the database are grouped into a transaction. Transactions can be either committed, meaning that all the changes are written into the database, or rolled back, meaning that the changes are all discarded and the database is unchanged. See any database textbook for more information.)

Vamos supor que exista um objeto representando uma conexão com o banco de dados. Nosso objetivo será permitir que o usuário escreva código como este:

```
db_connection = DatabaseConnection()
with db_connection as cursor:
    cursor.execute('insert into ...')
    cursor.execute('delete from ...')
    # ... more operations ...
```

The transaction should be committed if the code in the block runs flawlessly or rolled back if there's an exception. Here's the basic interface for **`DatabaseConnection`** that I'll assume:

```
class DatabaseConnection:
    # Database interface
    def cursor(self):
        "Returns a cursor object and starts a new transaction"
    def commit(self):
        "Commits current transaction"
    def rollback(self):
        "Rolls back current transaction"
```

The **`__enter__()`** method is pretty easy, having only to start a new transaction. For this application the resulting cursor object would be a useful result, so the method will return it. The user can then add `as cursor` to their '**`with`**' statement to bind the cursor to a variable name.

```
class DatabaseConnection:
    ...
    def __enter__(self):
        # Code to start a new transaction
        cursor = self.cursor()
        return cursor
```

The **`__exit__()`** method is the most complicated because it's where most of the work has to be done. The method has to check if an exception occurred. If there was no exception, the transaction is committed. The transaction is rolled back if there was an exception.

In the code below, execution will just fall off the end of the function, returning the default value of `None`. `None` is false, so the exception will be re-raised automatically. If you wished, you could be more explicit and add a **`return`** statement at the marked location.

```
class DatabaseConnection:
    ...
    def __exit__(self, type, value, tb):
        if tb is None:
            # No exception, so commit
```

```
            self.commit()
        else:
            # Exception occurred, so rollback.
            self.rollback()
            # return False
```

The contextlib module

The `contextlib` module provides some functions and a decorator that are useful when writing objects for use with the '`with`' statement.

The decorator is called `@~contextlib.contextmanager`, and lets you write a single generator function instead of defining a new class. The generator should yield exactly one value. The code up to the `yield` will be executed as the `__enter__()` method, and the value yielded will be the method's return value that will get bound to the variable in the '`with`' statement's **`as`** clause, if any. The code after the **`yield`** will be executed in the `__exit__()` method. Any exception raised in the block will be raised by the **`yield`** statement.

Using this decorator, our database example from the previous section could be written as:

```
from contextlib import contextmanager

@contextmanager
def db_transaction(connection):
    cursor = connection.cursor()
    try:
        yield cursor
    except:
        connection.rollback()
        raise
    else:
        connection.commit()

db = DatabaseConnection()
with db_transaction(db) as cursor:
    ...
```

The **contextlib** module also has a `nested(mgr1, mgr2, ...)` function that combines a number of context managers so you don't need to write nested '**with**' statements. In this example, the single '**with**' statement both starts a database transaction and acquires a thread lock:

```
lock = threading.Lock()
with nested (db_transaction(db), lock) as (cursor, locked):
    ...
```

Finally, the **closing()** function returns its argument so that it can be bound to a variable, and calls the argument's `.close()` method at the end of the block.

```
import urllib, sys
from contextlib import closing

with closing(urllib.urlopen('http://www.yahoo.com')) as f:
    for line in f:
        sys.stdout.write(line)
```

Ver também

PEP 343 **[https://peps.python.org/pep-0343/]** **- A instrução "with"**
PEP written by Guido van Rossum and Nick Coghlan; implemented by Mike Bland, Guido van Rossum, and Neal Norwitz. The PEP shows the code generated for a '**with**' statement, which can be helpful in learning how the statement works.

The documentation for the **contextlib** module.

PEP 366: Explicit Relative Imports From a Main Module

Python's **-m** switch allows running a module as a script. When you ran a module that was located inside a package, relative imports didn't work correctly.

The fix for Python 2.6 adds a **`module.__package__`** attribute. When this attribute is present, relative imports will be relative to the value of this attribute instead of the **`__name__`** attribute.

PEP 302-style importers can then set **`__package__`** as necessary. The **`runpy`** module that implements the **`-m`** switch now does this, so relative imports will now work correctly in scripts running from inside a package.

PEP 370: Per-user `site-packages` Directory

When you run Python, the module search path `sys.path` usually includes a directory whose path ends in `"site-packages"`. This directory is intended to hold locally installed packages available to all users using a machine or a particular site installation.

Python 2.6 introduces a convention for user-specific site directories. The directory varies depending on the platform:

- Unix and Mac OS X: `~/.local/`
- Windows: `%APPDATA%/Python`

Within this directory, there will be version-specific subdirectories, such as `lib/python2.6/site-packages` on Unix/Mac OS and `Python26/site-packages` on Windows.

If you don't like the default directory, it can be overridden by an environment variable. **`PYTHONUSERBASE`** sets the root directory used for all Python versions supporting this feature. On Windows, the directory for application-specific data can be changed by setting the **`APPDATA`** environment variable. You can also modify the `site.py` file for your Python installation.

The feature can be disabled entirely by running Python with the **`-s`** option or setting the **`PYTHONNOUSERSITE`** environment variable.

Ver também

PEP 370 **[https://peps.python.org/pep-0370/]** **- Per-user `site-packages` Directory**

PEP written and implemented by Christian Heimes.

PEP 371: The `multiprocessing` Package

The new `multiprocessing` package lets Python programs create new processes that will perform a computation and return a result to the parent. The parent and child processes can communicate using queues and pipes, synchronize their operations using locks and semaphores, and can share simple arrays of data.

The `multiprocessing` module started out as an exact emulation of the `threading` module using processes instead of threads. That goal was discarded along the path to Python 2.6, but the general approach of the module is still similar. The fundamental class is the `Process`, which is passed a callable object and a collection of arguments. The `start()` method sets the callable running in a subprocess, after which you can call the `is_alive()` method to check whether the subprocess is still running and the `join()` method to wait for the process to exit.

Here's a simple example where the subprocess will calculate a factorial. The function doing the calculation is written strangely so that it takes significantly longer when the input argument is a multiple of 4.

```
import time
from multiprocessing import Process, Queue


def factorial(queue, N):
    "Compute a factorial."
    # If N is a multiple of 4, this function will take much long
    if (N % 4) == 0:
        time.sleep(.05 * N/4)

    # Calculate the result
    fact = 1L
```

```
    for i in range(1, N+1):
        fact = fact * i

    # Put the result on the queue
    queue.put(fact)

if __name__ == '__main__':
    queue = Queue()

    N = 5

    p = Process(target=factorial, args=(queue, N))
    p.start()
    p.join()

    result = queue.get()
    print 'Factorial', N, '=', result
```

A **Queue** is used to communicate the result of the factorial. The **Queue** object is stored in a global variable. The child process will use the value of the variable when the child was created; because it's a **Queue**, parent and child can use the object to communicate. (If the parent were to change the value of the global variable, the child's value would be unaffected, and vice versa.)

Two other classes, **Pool** and **Manager**, provide higher-level interfaces. **Pool** will create a fixed number of worker processes, and requests can then be distributed to the workers by calling **apply()** or **apply_async()** to add a single request, and **map()** or **map_async()** to add a number of requests. The following code uses a **Pool** to spread requests across 5 worker processes and retrieve a list of results:

```
from multiprocessing import Pool

def factorial(N, dictionary):
    "Compute a factorial."
    ...
p = Pool(5)
result = p.map(factorial, range(1, 1000, 10))
```

```
for v in result:
    print v
```

This produces the following output:

```
1
39916800
51090942171709440000
8222838654177922817725562880000000
334525266131638071081700620534407516651520000000000
...
```

The other high-level interface, the **Manager** class, creates a separate server process that can hold master copies of Python data structures. Other processes can then access and modify these data structures using proxy objects. The following example creates a shared dictionary by calling the **dict()** method; the worker processes then insert values into the dictionary. (Locking is not done for you automatically, which doesn't matter in this example. **Manager**'s methods also include **Lock()**, **RLock()**, and **Semaphore()** to create shared locks.)

```
import time
from multiprocessing import Pool, Manager

def factorial(N, dictionary):
    "Compute a factorial."
    # Calculate the result
    fact = 1L
    for i in range(1, N+1):
        fact = fact * i

    # Store result in dictionary
    dictionary[N] = fact

if __name__ == '__main__':
    p = Pool(5)
    mgr = Manager()
    d = mgr.dict()         # Create shared dictionary

    # Run tasks using the pool
    for N in range(1, 1000, 10):
```

```
        p.apply_async(factorial, (N, d))

    # Mark pool as closed -- no more tasks can be added.
    p.close()

    # Wait for tasks to exit
    p.join()

    # Output results
    for k, v in sorted(d.items()):
        print k, v
```

This will produce the output:

```
1 1
11 39916800
21 51090942171709440000
31 8222838654177922817725562880000000
41 334525266131638071081700620534407516651520000000000
51 15511187532873822802242430164693032110632597200169861120000..
```

Ver também

The documentation for the `multiprocessing` module.

PEP 371 [https://peps.python.org/pep-0371/] - Addition of the multiprocessing package

PEP written by Jesse Noller and Richard Oudkerk; implemented by Richard Oudkerk and Jesse Noller.

PEP 3101: Advanced String Formatting

In Python 3.0, the `%` operator is supplemented by a more powerful string formatting method, `format()`. Support for the `str.format()` method has been backported to Python 2.6.

In 2.6, both 8-bit and Unicode strings have a `.format()` method that treats the string as a template and takes the arguments to be formatted. The

formatting template uses curly brackets (`{`, `}`) as special characters:

```
>>> # Substitute positional argument 0 into the string.
>>> "User ID: {0}".format("root")
'User ID: root'
>>> # Use the named keyword arguments
>>> "User ID: {uid}   Last seen: {last_login}".format(
...    uid="root",
...    last_login = "5 Mar 2008 07:20")
'User ID: root   Last seen: 5 Mar 2008 07:20'
```

Curly brackets can be escaped by doubling them:

```
>>> "Empty dict: {{}}".format()
"Empty dict: {}"
```

Field names can be integers indicating positional arguments, such as `{0}`, `{1}`, etc. or names of keyword arguments. You can also supply compound field names that read attributes or access dictionary keys:

```
>>> import sys
>>> print 'Platform: {0.platform}\nPython version: {0.version}'.
Platform: darwin
Python version: 2.6a1+ (trunk:61261M, Mar  5 2008, 20:29:41)
[GCC 4.0.1 (Apple Computer, Inc. build 5367)]'

>>> import mimetypes
>>> 'Content-type: {0[.mp4]}'.format(mimetypes.types_map)
'Content-type: video/mp4'
```

Note that when using dictionary-style notation such as `[.mp4]`, you don't need to put any quotation marks around the string; it will look up the value using `.mp4` as the key. Strings beginning with a number will be converted to an integer. You can't write more complicated expressions inside a format string.

So far we've shown how to specify which field to substitute into the resulting string. The precise formatting used is also controllable by adding a colon followed by a format specifier. For example:

```
>>> # Field 0: left justify, pad to 15 characters
>>> # Field 1: right justify, pad to 6 characters
>>> fmt = '{0:15} ${1:>6}'
>>> fmt.format('Registration', 35)
'Registration    $    35'
>>> fmt.format('Tutorial', 50)
'Tutorial        $    50'
>>> fmt.format('Banquet', 125)
'Banquet         $   125'
```

Format specifiers can reference other fields through nesting:

```
>>> fmt = '{0:{1}}'
>>> width = 15
>>> fmt.format('Invoice #1234', width)
'Invoice #1234  '
>>> width = 35
>>> fmt.format('Invoice #1234', width)
'Invoice #1234                      '
```

The alignment of a field within the desired width can be specified:

Caractere	Effect
< (default)	Left-align
>	Right-align
^	Center
=	(For numeric types only) Pad after the sign.

Format specifiers can also include a presentation type, which controls how the value is formatted. For example, floating-point numbers can be

formatted as a general number or in exponential notation:

```
>>> '{0:g}'.format(3.75)
'3.75'
>>> '{0:e}'.format(3.75)
'3.750000e+00'
```

A variety of presentation types are available. Consult the 2.6 documentation for a complete list; here's a sample:

b	Binary. Outputs the number in base 2.
c	Character. Converts the integer to the corresponding Unicode character before printing.
d	Inteiro decimal. Exibe o número na base 10.
o	Formato octal. Exibe o número na base 8.
x	Formato hexadecimal. Produz o número na base 16, usando letras minúsculas para os dígitos acima de 9.
e	Exponent notation. Prints the number in scientific notation using the letter 'e' to indicate the exponent.
g	General format. This prints the number as a fixed-point number, unless the number is too large, in which case it switches to 'e' exponent notation.
n	Number. This is the same as 'g' (for floats) or 'd' (for integers), except that it uses the current locale setting to insert the appropriate number

separator characters.

`%` Percentage. Multiplies the number by 100 and displays in fixed ('f') format, followed by a percent sign.

Classes and types can define a `__format__()` method to control how they're formatted. It receives a single argument, the format specifier:

```
def __format__(self, format_spec):
    if isinstance(format_spec, unicode):
        return unicode(str(self))
    else:
        return str(self)
```

There's also a `format()` builtin that will format a single value. It calls the type's `__format__()` method with the provided specifier:

```
>>> format(75.6564, '.2f')
'75.66'
```

Ver também

Sintaxe das strings de formato
The reference documentation for format fields.

PEP 3101 **[https://peps.python.org/pep-3101/]** **- Advanced String Formatting**
PEP written by Talin. Implemented by Eric Smith.

PEP 3105: `print` As a Function

The `print` statement becomes the `print()` function in Python 3.0. Making **`print()`** a function makes it possible to replace the function by doing `def print(...)` or importing a new function from somewhere else.

Python 2.6 has a `__future__` import that removes `print` as language syntax, letting you use the functional form instead. For example:

```
>>> from __future__ import print_function
>>> print('# of entries', len(dictionary), file=sys.stderr)
```

The signature of the new function is:

```
def print(*args, sep=' ', end='\n', file=None)
```

The parameters are:

- *args*: positional arguments whose values will be printed out.
- *sep*: the separator, which will be printed between arguments.
- *end*: the ending text, which will be printed after all of the arguments have been output.
- *file*: the file object to which the output will be sent.

Ver também

PEP 3105 [https://peps.python.org/pep-3105/] - Make print a function
PEP written by Georg Brandl.

PEP 3110: Exception-Handling Changes

One error that Python programmers occasionally make is writing the following code:

```
try:
    ...
except TypeError, ValueError:  # Wrong!
    ...
```

The author is probably trying to catch both `TypeError` and `ValueError` exceptions, but this code actually does something different: it will catch `TypeError` and bind the resulting exception object to the local name

`"ValueError"`. The **`ValueError`** exception will not be caught at all. The correct code specifies a tuple of exceptions:

```
try:
    ...
except (TypeError, ValueError):
    ...
```

This error happens because the use of the comma here is ambiguous: does it indicate two different nodes in the parse tree, or a single node that's a tuple?

Python 3.0 makes this unambiguous by replacing the comma with the word "as". To catch an exception and store the exception object in the variable `exc`, you must write:

```
try:
    ...
except TypeError as exc:
    ...
```

Python 3.0 will only support the use of "as", and therefore interprets the first example as catching two different exceptions. Python 2.6 supports both the comma and "as", so existing code will continue to work. We therefore suggest using "as" when writing new Python code that will only be executed with 2.6.

Ver também

PEP 3110 [https://peps.python.org/pep-3110/] - Catching Exceptions in Python 3000

PEP written and implemented by Collin Winter.

PEP 3112: Byte Literals

Python 3.0 adopts Unicode as the language's fundamental string type and denotes 8-bit literals differently, either as `b'string'` or using a **`bytes`**

constructor. For future compatibility, Python 2.6 adds **bytes** as a synonym for the **str** type, and it also supports the `b''` notation.

The 2.6 **str** differs from 3.0's **bytes** type in various ways; most notably, the constructor is completely different. In 3.0, `bytes([65, 66, 67])` is 3 elements long, containing the bytes representing `ABC`; in 2.6, `bytes([65, 66, 67])` returns the 12-byte string representing the **str()** of the list.

The primary use of **bytes** in 2.6 will be to write tests of object type such as `isinstance(x, bytes)`. This will help the 2to3 converter, which can't tell whether 2.x code intends strings to contain either characters or 8-bit bytes; you can now use either **bytes** or **str** to represent your intention exactly, and the resulting code will also be correct in Python 3.0.

There's also a `__future__` import that causes all string literals to become Unicode strings. This means that `\u` escape sequences can be used to include Unicode characters:

```
from __future__ import unicode_literals

s = ('\u751f\u3080\u304e\u3000\u751f\u3054'
     '\u3081\u3000\u751f\u305f\u307e\u3054')

print len(s)               # 12 Unicode characters
```

At the C level, Python 3.0 will rename the existing 8-bit string type, called **PyStringObject** in Python 2.x, to **PyBytesObject**. Python 2.6 uses `#define` to support using the names **PyBytesObject()**, **PyBytes_Check()**, **PyBytes_FromStringAndSize()**, and all the other functions and macros used with strings.

Instances of the **bytes** type are immutable just as strings are. A new **bytearray** type stores a mutable sequence of bytes:

```
>>> bytearray([65, 66, 67])
bytearray(b'ABC')
>>> b = bytearray(u'\u21ef\u3244', 'utf-8')
>>> b
bytearray(b'\xe2\x87\xaf\xe3\x89\x84')
>>> b[0] = '\xe3'
```

```
>>> b
bytearray(b'\xe3\x87\xaf\xe3\x89\x84')
>>> unicode(str(b), 'utf-8')
u'\u31ef \u3244'
```

Byte arrays support most of the methods of string types, such as `startswith()`/`endswith()`, `find()`/`rfind()`, and some of the methods of lists, such as `append()`, `pop()`, and `reverse()`.

```
>>> b = bytearray('ABC')
>>> b.append('d')
>>> b.append(ord('e'))
>>> b
bytearray(b'ABCde')
```

There's also a corresponding C API, with `PyByteArray_FromObject()`, `PyByteArray_FromStringAndSize()`, and various other functions.

Ver também

PEP 3112 [https://peps.python.org/pep-3112/] - Bytes literals in Python 3000
PEP written by Jason Orendorff; backported to 2.6 by Christian Heimes.

PEP 3116: New I/O Library

Python's built-in file objects support a number of methods, but file-like objects don't necessarily support all of them. Objects that imitate files usually support **`read()`** and **`write()`**, but they may not support **`readline()`**, for example. Python 3.0 introduces a layered I/O library in the `io` module that separates buffering and text-handling features from the fundamental read and write operations.

There are three levels of abstract base classes provided by the `io` module:

- `RawIOBase` defines raw I/O operations: `read()`, `readinto()`, `write()`, `seek()`, `tell()`, `truncate()`, and `close()`. Most of the methods of

this class will often map to a single system call. There are also `readable()`, `writable()`, and `seekable()` methods for determining what operations a given object will allow.

Python 3.0 has concrete implementations of this class for files and sockets, but Python 2.6 hasn't restructured its file and socket objects in this way.

- `BufferedIOBase` is an abstract base class that buffers data in memory to reduce the number of system calls used, making I/O processing more efficient. It supports all of the methods of `RawIOBase`, and adds a `raw` attribute holding the underlying raw object.

  There are five concrete classes implementing this ABC. `BufferedWriter` and `BufferedReader` are for objects that support write-only or read-only usage that have a `seek()` method for random access. `BufferedRandom` objects support read and write access upon the same underlying stream, and `BufferedRWPair` is for objects such as TTYs that have both read and write operations acting upon unconnected streams of data. The `BytesIO` class supports reading, writing, and seeking over an in-memory buffer.

- `TextIOBase`: Provides functions for reading and writing strings (remember, strings will be Unicode in Python 3.0), and supporting universal newlines. **`TextIOBase`** defines the `readline()` method and supports iteration upon objects.

  There are two concrete implementations. `TextIOWrapper` wraps a buffered I/O object, supporting all of the methods for text I/O and adding a `buffer` attribute for access to the underlying object. `StringIO` simply buffers everything in memory without ever writing anything to disk.

  (In Python 2.6, `io.StringIO` is implemented in pure Python, so it's pretty slow. You should therefore stick with the existing **`StringIO`** module or **`cStringIO`** for now. At some point Python 3.0's `io` module will be rewritten into C for speed, and perhaps the C implementation will be backported to the 2.x releases.)

In Python 2.6, the underlying implementations haven't been restructured to build on top of the `io` module's classes. The module is being provided to make it easier to write code that's forward-compatible with 3.0, and to save developers the effort of writing their own implementations of buffering and text I/O.

Ver também

PEP 3116 **[https://peps.python.org/pep-3116/]** **- New I/O**
PEP written by Daniel Stutzbach, Mike Verdone, and Guido van Rossum. Code by Guido van Rossum, Georg Brandl, Walter Doerwald, Jeremy Hylton, Martin von Löwis, Tony Lownds, and others.

PEP 3118: Revised Buffer Protocol

The buffer protocol is a C-level API that lets Python types exchange pointers into their internal representations. A memory-mapped file can be viewed as a buffer of characters, for example, and this lets another module such as `re` treat memory-mapped files as a string of characters to be searched.

The primary users of the buffer protocol are numeric-processing packages such as NumPy, which expose the internal representation of arrays so that callers can write data directly into an array instead of going through a slower API. This PEP updates the buffer protocol in light of experience from NumPy development, adding a number of new features such as indicating the shape of an array or locking a memory region.

The most important new C API function is `PyObject_GetBuffer(PyObject *obj, Py_buffer *view, int flags)`, which takes an object and a set of flags, and fills in the `Py_buffer` structure with information about the object's memory representation. Objects can use this operation to lock memory in place while an external caller could be modifying the contents, so there's a corresponding `PyBuffer_Release(Py_buffer *view)` to indicate that the external caller is done.

The *flags* argument to `PyObject_GetBuffer()` specifies constraints upon the memory returned. Some examples are:

- `PyBUF_WRITABLE` indicates that the memory must be writable.
- `PyBUF_LOCK` requests a read-only or exclusive lock on the memory.
- `PyBUF_C_CONTIGUOUS` and `PyBUF_F_CONTIGUOUS` requests a C-contiguous (last dimension varies the fastest) or Fortran-contiguous (first dimension varies the fastest) array layout.

Two new argument codes for `PyArg_ParseTuple()`, `s*` and `z*`, return locked buffer objects for a parameter.

Ver também

PEP 3118 **[https://peps.python.org/pep-3118/]** **- Revising the buffer protocol**
PEP written by Travis Oliphant and Carl Banks; implemented by Travis Oliphant.

PEP 3119: Abstract Base Classes

Some object-oriented languages such as Java support interfaces, declaring that a class has a given set of methods or supports a given access protocol. Abstract Base Classes (or ABCs) are an equivalent feature for Python. The ABC support consists of an `abc` module containing a metaclass called `ABCMeta`, special handling of this metaclass by the `isinstance()` and `issubclass()` builtins, and a collection of basic ABCs that the Python developers think will be widely useful. Future versions of Python will probably add more ABCs.

Let's say you have a particular class and wish to know whether it supports dictionary-style access. The phrase "dictionary-style" is vague, however. It probably means that accessing items with `obj[1]` works. Does it imply that setting items with `obj[2] = value` works? Or that the object will have **`keys()`**, **`values()`**, and **`items()`** methods? What about the iterative variants such as **`iterkeys()`**? **`copy`and :meth:()`**!update`? Iterating over the object with **`iter()`**?

The Python 2.6 **collections** module includes a number of different ABCs that represent these distinctions. **Iterable** indicates that a class defines **__iter__()**, and **Container** means the class defines a **__contains__()** method and therefore supports `x in y` expressions. The basic dictionary interface of getting items, setting items, and **keys()**, **values()**, and **items()**, is defined by the **MutableMapping** ABC.

You can derive your own classes from a particular ABC to indicate they support that ABC's interface:

```
import collections

class Storage(collections.MutableMapping):
    ...
```

Alternatively, you could write the class without deriving from the desired ABC and instead register the class by calling the ABC's **register()** method:

```
import collections

class Storage:
    ...

collections.MutableMapping.register(Storage)
```

For classes that you write, deriving from the ABC is probably clearer. The **register()** method is useful when you've written a new ABC that can describe an existing type or class, or if you want to declare that some third-party class implements an ABC. For example, if you defined a **PrintableType** ABC, it's legal to do:

```
# Register Python's types
PrintableType.register(int)
PrintableType.register(float)
PrintableType.register(str)
```

Classes should obey the semantics specified by an ABC, but Python can't check this; it's up to the class author to understand the ABC's requirements and to implement the code accordingly.

To check whether an object supports a particular interface, you can now write:

```
def func(d):
    if not isinstance(d, collections.MutableMapping):
        raise ValueError("Mapping object expected, not %r" % d)
```

Don’t feel that you must now begin writing lots of checks as in the above example. Python has a strong tradition of duck-typing, where explicit type-checking is never done and code simply calls methods on an object, trusting that those methods will be there and raising an exception if they aren’t. Be judicious in checking for ABCs and only do it where it’s absolutely necessary.

You can write your own ABCs by using `abc.ABCMeta` as the metaclass in a class definition:

```
from abc import ABCMeta, abstractmethod

class Drawable():
    __metaclass__ = ABCMeta

    @abstractmethod
    def draw(self, x, y, scale=1.0):
        pass

    def draw_doubled(self, x, y):
        self.draw(x, y, scale=2.0)


class Square(Drawable):
    def draw(self, x, y, scale):
        ...
```

In the **`Drawable`** ABC above, the **`draw_doubled()`** method renders the object at twice its size and can be implemented in terms of other methods described in **`Drawable`**. Classes implementing this ABC therefore don’t need to provide their own implementation of **`draw_doubled()`**, though they can do so. An implementation of **`draw()`** is necessary, though; the ABC can’t provide a useful generic implementation.

You can apply the `@~abc.abstractmethod` decorator to methods such as **`draw()`** that must be implemented; Python will then raise an exception for classes that don't define the method. Note that the exception is only raised when you actually try to create an instance of a subclass lacking the method:

```
>>> class Circle(Drawable):
...     pass
...
>>> c = Circle()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: Can't instantiate abstract class Circle with abstract
>>>
```

Abstract data attributes can be declared using the `@abstractproperty` decorator:

```
from abc import abstractproperty
...

@abstractproperty
def readonly(self):
   return self._x
```

Subclasses must then define a `readonly` property.

Ver também

PEP 3119 [https://peps.python.org/pep-3119/] - Introduzindo classes base abstratas

PEP written by Guido van Rossum and Talin. Implemented by Guido van Rossum. Backported to 2.6 by Benjamin Aranguren, with Alex Martelli.

PEP 3127: Integer Literal Support and Syntax

Python 3.0 changes the syntax for octal (base-8) integer literals, prefixing them with “0o” or “0O” instead of a leading zero, and adds support for binary (base-2) integer literals, signalled by a “0b” or “0B” prefix.

Python 2.6 doesn’t drop support for a leading 0 signalling an octal number, but it does add support for “0o” and “0b”:

```
>>> 0o21, 2*8 + 1
(17, 17)
>>> 0b101111
47
```

The **oct()** builtin still returns numbers prefixed with a leading zero, and a new **bin()** builtin returns the binary representation for a number:

```
>>> oct(42)
'052'
>>> future_builtins.oct(42)
'0o52'
>>> bin(173)
'0b10101101'
```

The **int()** and **long()** builtins will now accept the “0o” and “0b” prefixes when base-8 or base-2 are requested, or when the *base* argument is zero (signalling that the base used should be determined from the string):

```
>>> int ('0o52', 0)
42
>>> int('1101', 2)
13
>>> int('0b1101', 2)
13
>>> int('0b1101', 0)
13
```

Ver também

PEP 3127 [https://peps.python.org/pep-3127/] - Integer Literal Support and Syntax

PEP written by Patrick Maupin; backported to 2.6 by Eric Smith.

PEP 3129: Class Decorators

Decorators have been extended from functions to classes. It's now legal to write:

```
@foo
@bar
class A:
  pass
```

Isso equivale a:

```
class A:
  pass

A = foo(bar(A))
```

Ver também

PEP 3129 **[https://peps.python.org/pep-3129/]** **- Class Decorators**
PEP written by Collin Winter.

PEP 3141: A Type Hierarchy for Numbers

Python 3.0 adds several abstract base classes for numeric types inspired by Scheme's numeric tower. These classes were backported to 2.6 as the `numbers` module.

The most general ABC is `Number`. It defines no operations at all, and only exists to allow checking if an object is a number by doing `isinstance(obj, Number)`.

`Complex` is a subclass of `Number`. Complex numbers can undergo the basic operations of addition, subtraction, multiplication, division, and exponentiation, and you can retrieve the real and imaginary parts and obtain

a number's conjugate. Python's built-in complex type is an implementation of **`Complex`**.

`Real` further derives from **`Complex`**, and adds operations that only work on real numbers: **`floor()`**, **`trunc()`**, rounding, taking the remainder mod N, floor division, and comparisons.

`Rational` numbers derive from **`Real`**, have **`numerator`** and **`denominator`** properties, and can be converted to floats. Python 2.6 adds a simple rational-number class, **`Fraction`**, in the **`fractions`** module. (It's called **`Fraction`** instead of `Rational` to avoid a name clash with **`numbers.Rational`**.)

`Integral` numbers derive from **`Rational`**, and can be shifted left and right with `<<` and `>>`, combined using bitwise operations such as `&` and `|`, and can be used as array indexes and slice boundaries.

In Python 3.0, the PEP slightly redefines the existing builtins **`round()`**, **`math.floor()`**, **`math.ceil()`**, and adds a new one, **`math.trunc()`**, that's been backported to Python 2.6. **`math.trunc()`** rounds toward zero, returning the closest **`Integral`** that's between the function's argument and zero.

Ver também

PEP 3141 **[https://peps.python.org/pep-3141/]** **- A Type Hierarchy for Numbers**

PEP written by Jeffrey Yasskin.

Scheme's numerical tower [https://www.gnu.org/software/guile/manual/html_node/Numerical-Tower.html#Numerical-Tower], from the Guile manual.

Scheme's number datatypes [https://conservatory.scheme.org/schemers/Documents/Standards/R5RS/HTML/r5rs-Z-H-9.html#%_sec_6.2] from the R5RS Scheme specification.

The `fractions` Module

To fill out the hierarchy of numeric types, the `fractions` module provides a rational-number class. Rational numbers store their values as a numerator and denominator forming a fraction, and can exactly represent numbers such as `2/3` that floating-point numbers can only approximate.

The `Fraction` constructor takes two `Integral` values that will be the numerator and denominator of the resulting fraction.

```
>>> from fractions import Fraction
>>> a = Fraction(2, 3)
>>> b = Fraction(2, 5)
>>> float(a), float(b)
(0.66666666666666663, 0.40000000000000002)
>>> a+b
Fraction(16, 15)
>>> a/b
Fraction(5, 3)
```

For converting floating-point numbers to rationals, the float type now has an `as_integer_ratio()` method that returns the numerator and denominator for a fraction that evaluates to the same floating-point value:

```
>>> (2.5) .as_integer_ratio()
(5, 2)
>>> (3.1415) .as_integer_ratio()
(7074029114692207L, 2251799813685248L)
>>> (1./3) .as_integer_ratio()
(6004799503160661L, 18014398509481984L)
```

Note that values that can only be approximated by floating-point numbers, such as 1./3, are not simplified to the number being approximated; the fraction attempts to match the floating-point value **exactly**.

The `fractions` module is based upon an implementation by Sjoerd Mullender that was in Python's `Demo/classes/` directory for a long time. This implementation was significantly updated by Jeffrey Yasskin.

Outras mudanças na linguagem

Algumas das mudanças menores feitas no núcleo da linguagem Python são:

- Directories and zip archives containing a `__main__.py` file can now be executed directly by passing their name to the interpreter. The directory or zip archive is automatically inserted as the first entry in sys.path. (Suggestion and initial patch by Andy Chu, subsequently revised by Phillip J. Eby and Nick Coghlan; bpo-1739468 [https://bugs.python.org/issue?@action=redirect&bpo=1739468].)

- The **`hasattr()`** function was catching and ignoring all errors, under the assumption that they meant a **`__getattr__()`** method was failing somehow and the return value of **`hasattr()`** would therefore be `False`. This logic shouldn't be applied to **`KeyboardInterrupt`** and **`SystemExit`**, however; Python 2.6 will no longer discard such exceptions when **`hasattr()`** encounters them. (Fixed by Benjamin Peterson; bpo-2196 [https://bugs.python.org/issue?@action=redirect&bpo=2196].)

- When calling a function using the `**` syntax to provide keyword arguments, you are no longer required to use a Python dictionary; any mapping will now work:

  ```
  >>> def f(**kw):
  ...     print sorted(kw)
  ...
  >>> ud=UserDict.UserDict()
  >>> ud['a'] = 1
  >>> ud['b'] = 'string'
  >>> f(**ud)
  ['a', 'b']
  ```

  (Contributed by Alexander Belopolsky; bpo-1686487 [https://bugs.python.org/issue?@action=redirect&bpo=1686487].)

  It's also become legal to provide keyword arguments after a `*args` argument to a function call.

```
>>> def f(*args, **kw):
...     print args, kw
...
>>> f(1,2,3, *(4,5,6), keyword=13)
(1, 2, 3, 4, 5, 6) {'keyword': 13}
```

Previously this would have been a syntax error. (Contributed by Amaury Forgeot d'Arc; bpo-3473 [https://bugs.python.org/issue?@action=redirect&bpo=3473].)

- A new builtin, `next(iterator, [default])` returns the next item from the specified iterator. If the *default* argument is supplied, it will be returned if *iterator* has been exhausted; otherwise, the **`StopIteration`** exception will be raised. (Backported in bpo-2719 [https://bugs.python.org/issue?@action=redirect&bpo=2719].)

- Tuples now have **`index()`** and **`count()`** methods matching the list type's **`index()`** and **`count()`** methods:

```
>>> t = (0,1,2,3,4,0,1,2)
>>> t.index(3)
3
>>> t.count(0)
2
```

(Contributed by Raymond Hettinger)

- The built-in types now have improved support for extended slicing syntax, accepting various combinations of `(start, stop, step)`. Previously, the support was partial and certain corner cases wouldn't work. (Implemented by Thomas Wouters.)

- Properties now have three decorators, **`@~property.getter`**, **`@~property.setter`** and **`@~property.deleter`**, that are decorators providing useful shortcuts for adding a getter, setter or deleter function to an existing property. You would use them like this:

```
class C(object):
    @property
    def x(self):
```

```
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
    def x(self):
        del self._x

class D(C):
    @C.x.getter
    def x(self):
        return self._x * 2

    @x.setter
    def x(self, value):
        self._x = value / 2
```

- Several methods of the built-in set types now accept multiple iterables: `intersection()`, `intersection_update()`, `union()`, `update()`, `difference()` and `difference_update()`.

```
>>> s=set('1234567890')
>>> s.intersection('abc123', 'cdf246')  # Intersection betwe
set(['2'])
>>> s.difference('246', '789')
set(['1', '0', '3', '5'])
```

  (Contributed by Raymond Hettinger.)

- Many floating-point features were added. The `float()` function will now turn the string `nan` into an IEEE 754 Not A Number value, and `+inf` and `-inf` into positive or negative infinity. This works on any platform with IEEE 754 semantics. (Contributed by Christian Heimes; bpo-1635 [https://bugs.python.org/issue?@action=redirect&bpo=1635].)

  Other functions in the `math` module, `isinf()` and `isnan()`, return true if their floating-point argument is infinite or Not A Number. (bpo-1640 [https://bugs.python.org/issue?@action=redirect&bpo=1640])

Conversion functions were added to convert floating-point numbers into hexadecimal strings (bpo-3008 [https://bugs.python.org/issue?@action=redirect&bpo=3008]). These functions convert floats to and from a string representation without introducing rounding errors from the conversion between decimal and binary. Floats have a **hex()** method that returns a string representation, and the `float.fromhex()` method converts a string back into a number:

```
>>> a = 3.75
>>> a.hex()
'0x1.e000000000000p+1'
>>> float.fromhex('0x1.e000000000000p+1')
3.75
>>> b=1./3
>>> b.hex()
'0x1.5555555555555p-2'
```

- A numerical nicety: when creating a complex number from two floats on systems that support signed zeros (-0 and +0), the **complex()** constructor will now preserve the sign of the zero. (Fixed by Mark T. Dickinson; bpo-1507 [https://bugs.python.org/issue?@action=redirect&bpo=1507].)

- Classes that inherit a **__hash__()** method from a parent class can set `__hash__ = None` to indicate that the class isn't hashable. This will make `hash(obj)` raise a **TypeError** and the class will not be indicated as implementing the **Hashable** ABC.

  You should do this when you've defined a **__cmp__()** or **__eq__()** method that compares objects by their value rather than by identity. All objects have a default hash method that uses `id(obj)` as the hash value. There's no tidy way to remove the **__hash__()** method inherited from a parent class, so assigning `None` was implemented as an override. At the C level, extensions can set `tp_hash` to **PyObject_HashNotImplemented()**. (Fixed by Nick Coghlan and Amaury Forgeot d'Arc; bpo-2235 [https://bugs.python.org/issue?@action=redirect&bpo=2235].)

- The **GeneratorExit** exception now subclasses **BaseException** instead of **Exception**. This means that an exception handler that does `except`

`Exception:` will not inadvertently catch **`GeneratorExit`**. (Contributed by Chad Austin; bpo-1537 [https://bugs.python.org/issue?@action=redirect&bpo=1537].)

- Generator objects now have a **`gi_code`** attribute that refers to the original code object backing the generator. (Contributed by Collin Winter; bpo-1473257 [https://bugs.python.org/issue?@action=redirect&bpo=1473257].)

- The **`compile()`** built-in function now accepts keyword arguments as well as positional parameters. (Contributed by Thomas Wouters; bpo-1444529 [https://bugs.python.org/issue?@action=redirect&bpo=1444529].)

- The **`complex()`** constructor now accepts strings containing parenthesized complex numbers, meaning that `complex(repr(cplx))` will now round-trip values. For example, `complex('(3+4j)')` now returns the value (3+4j). (bpo-1491866 [https://bugs.python.org/issue?@action=redirect&bpo=1491866])

- The string **`translate()`** method now accepts `None` as the translation table parameter, which is treated as the identity transformation. This makes it easier to carry out operations that only delete characters. (Contributed by Bengt Richter and implemented by Raymond Hettinger; bpo-1193128 [https://bugs.python.org/issue?@action=redirect&bpo=1193128].)

- The built-in **`dir()`** function now checks for a **`__dir__()`** method on the objects it receives. This method must return a list of strings containing the names of valid attributes for the object, and lets the object control the value that **`dir()`** produces. Objects that have **`__getattr__()`** or **`__getattribute__()`** methods can use this to advertise pseudo-attributes they will honor. (bpo-1591665 [https://bugs.python.org/issue?@action=redirect&bpo=1591665])

- Instance method objects have new attributes for the object and function comprising the method; the new synonym for **`im_self`** is **`__self__`**, and **`im_func`** is also available as **`__func__`**. The old names are still supported in Python 2.6, but are gone in 3.0.

- An obscure change: when you use the `locals()` function inside a `class` statement, the resulting dictionary no longer returns free variables. (Free variables, in this case, are variables referenced in the `class` statement that aren't attributes of the class.)

Otimizações

- The `warnings` module has been rewritten in C. This makes it possible to invoke warnings from the parser, and may also make the interpreter's startup faster. (Contributed by Neal Norwitz and Brett Cannon; bpo-1631171 [https://bugs.python.org/issue?@action=redirect&bpo=1631171].)

- Type objects now have a cache of methods that can reduce the work required to find the correct method implementation for a particular class; once cached, the interpreter doesn't need to traverse base classes to figure out the right method to call. The cache is cleared if a base class or the class itself is modified, so the cache should remain correct even in the face of Python's dynamic nature. (Original optimization implemented by Armin Rigo, updated for Python 2.6 by Kevin Jacobs; bpo-1700288 [https://bugs.python.org/issue?@action=redirect&bpo=1700288].)

  By default, this change is only applied to types that are included with the Python core. Extension modules may not necessarily be compatible with this cache, so they must explicitly add `Py_TPFLAGS_HAVE_VERSION_TAG` to the module's `tp_flags` field to enable the method cache. (To be compatible with the method cache, the extension module's code must not directly access and modify the `tp_dict` member of any of the types it implements. Most modules don't do this, but it's impossible for the Python interpreter to determine that. See bpo-1878 [https://bugs.python.org/issue?@action=redirect&bpo=1878] for some discussion.)

- Function calls that use keyword arguments are significantly faster by doing a quick pointer comparison, usually saving the time of a full string comparison. (Contributed by Raymond Hettinger, after an initial

implementation by Antoine Pitrou; bpo-1819 [https://bugs.python.org/issue?@action=redirect&bpo=1819].)

- All of the functions in the `struct` module have been rewritten in C, thanks to work at the Need For Speed sprint. (Contributed by Raymond Hettinger.)

- Some of the standard built-in types now set a bit in their type objects. This speeds up checking whether an object is a subclass of one of these types. (Contributed by Neal Norwitz.)

- Unicode strings now use faster code for detecting whitespace and line breaks; this speeds up the `split()` method by about 25% and `splitlines()` by 35%. (Contributed by Antoine Pitrou.) Memory usage is reduced by using pymalloc for the Unicode string's data.

- The `with` statement now stores the `__exit__()` method on the stack, producing a small speedup. (Implemented by Jeffrey Yasskin.)

- To reduce memory usage, the garbage collector will now clear internal free lists when garbage-collecting the highest generation of objects. This may return memory to the operating system sooner.

Alterações no interpretador

Two command-line options have been reserved for use by other Python implementations. The `-J` switch has been reserved for use by Jython for Jython-specific options, such as switches that are passed to the underlying JVM. `-X` has been reserved for options specific to a particular implementation of Python such as CPython, Jython, or IronPython. If either option is used with Python 2.6, the interpreter will report that the option isn't currently used.

Python can now be prevented from writing `.pyc` or `.pyo` files by supplying the `-B` switch to the Python interpreter, or by setting the `PYTHONDONTWRITEBYTECODE` environment variable before running the interpreter. This setting is available to Python programs as the `sys.dont_write_bytecode` variable, and Python code can change the value

to modify the interpreter's behaviour. (Contributed by Neal Norwitz and Georg Brandl.)

The encoding used for standard input, output, and standard error can be specified by setting the **PYTHONIOENCODING** environment variable before running the interpreter. The value should be a string in the form `<encoding>` or `<encoding>:<errorhandler>`. The *encoding* part specifies the encoding's name, e.g. `utf-8` or `latin-1`; the optional *errorhandler* part specifies what to do with characters that can't be handled by the encoding, and should be one of "error", "ignore", or "replace". (Contributed by Martin von Löwis.)

Módulos novos ou aprimorados

Como em todas os lançamentos, a biblioteca padrão do Python recebeu diversas melhorias e correções de bugs. Aqui está uma lista parcial das mudanças mais notáveis, classificadas em ordem alfabética por nome do módulo. Consulte o arquivo `Misc/NEWS` na árvore de código-fonte para uma lista mais completa de alterações, ou procure nos logs do Subversion para todos os detalhes.

- Os módulos **`asyncore`** e **`asynchat`** estão sendo mantidos ativamente novamente e várias correções e correções foram aplicadas. (Mantido por Josiah Carlson; veja bpo-1736190 [https://bugs.python.org/issue?@action=redirect&bpo=1736190] para um patch.)

- The **`bsddb`** module also has a new maintainer, Jesús Cea Avión, and the package is now available as a standalone package. The web page for the package is www.jcea.es/programacion/pybsddb.htm [https://www.jcea.es/programacion/pybsddb.htm]. The plan is to remove the package from the standard library in Python 3.0, because its pace of releases is much more frequent than Python's.

  The **`bsddb.dbshelve`** module now uses the highest pickling protocol available, instead of restricting itself to protocol 1. (Contributed by W. Barnes.)

- The **cgi** module will now read variables from the query string of an HTTP POST request. This makes it possible to use form actions with URLs that include query strings such as "/cgi-bin/add.py?category=1". (Contributed by Alexandre Fiori and Nubis; bpo-1817 [https://bugs.python.org/issue?@action=redirect&bpo=1817].)

  The `parse_qs()` and `parse_qsl()` functions have been relocated from the **cgi** module to the `urlparse` module. The versions still available in the **cgi** module will trigger `PendingDeprecationWarning` messages in 2.6 (bpo-600362 [https://bugs.python.org/issue?@action=redirect&bpo=600362]).

- The `cmath` module underwent extensive revision, contributed by Mark Dickinson and Christian Heimes. Five new functions were added:

  - `polar()` converts a complex number to polar form, returning the modulus and argument of the complex number.
  - `rect()` does the opposite, turning a modulus, argument pair back into the corresponding complex number.
  - `phase()` returns the argument (also called the angle) of a complex number.
  - `isnan()` returns True if either the real or imaginary part of its argument is a NaN.
  - `isinf()` returns True if either the real or imaginary part of its argument is infinite.

  The revisions also improved the numerical soundness of the `cmath` module. For all functions, the real and imaginary parts of the results are accurate to within a few units of least precision (ulps) whenever possible. See bpo-1381 [https://bugs.python.org/issue?@action=redirect&bpo=1381] for the details. The branch cuts for `asinh()`, `atanh()`, and `atan()` have also been corrected.

  The tests for the module have been greatly expanded; nearly 2000 new test cases exercise the algebraic functions.

  On IEEE 754 platforms, the `cmath` module now handles IEEE 754 special values and floating-point exceptions in a manner consistent with Annex 'G' of the C99 standard.

- A new data type in the **collections** module: `namedtuple(typename, fieldnames)` is a factory function that creates subclasses of the standard tuple whose fields are accessible by name as well as index. For example:

```
>>> var_type = collections.namedtuple('variable',
...                   'id name type size')
>>> # Names are separated by spaces or commas.
>>> # 'id, name, type, size' would also work.
>>> var_type._fields
('id', 'name', 'type', 'size')

>>> var = var_type(1, 'frequency', 'int', 4)
>>> print var[0], var.id    # Equivalent
1 1
>>> print var[2], var.type  # Equivalent
int int
>>> var._asdict()
{'size': 4, 'type': 'int', 'id': 1, 'name': 'frequency'}
>>> v2 = var._replace(name='amplitude')
>>> v2
variable(id=1, name='amplitude', type='int', size=4)
```

  Several places in the standard library that returned tuples have been modified to return **namedtuple()** instances. For example, the **decimal.Decimal.as_tuple()** method now returns a named tuple with **sign**, **digits**, and **exponent** fields.

  (Contributed by Raymond Hettinger.)

- Another change to the **collections** module is that the **deque** type now supports an optional *maxlen* parameter; if supplied, the deque's size will be restricted to no more than *maxlen* items. Adding more items to a full deque causes old items to be discarded.

```
>>> from collections import deque
>>> dq=deque(maxlen=3)
>>> dq
deque([], maxlen=3)
>>> dq.append(1); dq.append(2); dq.append(3)
>>> dq
```

```
deque([1, 2, 3], maxlen=3)
>>> dq.append(4)
>>> dq
deque([2, 3, 4], maxlen=3)
```

(Contributed by Raymond Hettinger.)

- The **Cookie** module's **Morsel** objects now support an **httponly** attribute. In some browsers. cookies with this attribute set cannot be accessed or manipulated by JavaScript code. (Contributed by Arvin Schnell; bpo-1638033 [https://bugs.python.org/issue?@action=redirect&bpo=1638033].)

- A new window method in the **curses** module, **chgat()**, changes the display attributes for a certain number of characters on a single line. (Contributed by Fabian Kreutz.)

```
# Boldface text starting at y=0,x=21
# and affecting the rest of the line.
stdscr.chgat(0, 21, curses.A_BOLD)
```

  The **Textbox** class in the **curses.textpad** module now supports editing in insert mode as well as overwrite mode. Insert mode is enabled by supplying a true value for the *insert_mode* parameter when creating the **Textbox** instance.

- The **datetime** module's **strftime()** methods now support a `%f` format code that expands to the number of microseconds in the object, zero-padded on the left to six places. (Contributed by Skip Montanaro; bpo-1158 [https://bugs.python.org/issue?@action=redirect&bpo=1158].)

- The **decimal** module was updated to version 1.66 of the General Decimal Specification [https://speleotrove.com/decimal/decarith.html]. New features include some methods for some basic mathematical functions such as **exp()** and **log10()**:

```
>>> Decimal(1).exp()
Decimal("2.718281828459045235360287471")
>>> Decimal("2.7182818").ln()
Decimal("0.9999999895305022877376682436")
```

```
>>> Decimal(1000).log10()
Decimal("3")
```

The **as_tuple()** method of **Decimal** objects now returns a named tuple with **sign**, **digits**, and **exponent** fields.

(Implemented by Facundo Batista and Mark Dickinson. Named tuple support added by Raymond Hettinger.)

- The **difflib** module's **SequenceMatcher** class now returns named tuples representing matches, with **a**, **b**, and **size** attributes. (Contributed by Raymond Hettinger.)

- An optional `timeout` parameter, specifying a timeout measured in seconds, was added to the **ftplib.FTP** class constructor as well as the **connect()** method. (Added by Facundo Batista.) Also, the **FTP** class's **storbinary()** and **storlines()** now take an optional *callback* parameter that will be called with each block of data after the data has been sent. (Contributed by Phil Schwartz; bpo-1221598 [https://bugs.python.org/issue?@action=redirect&bpo=1221598].)

- The **reduce()** built-in function is also available in the **functools** module. In Python 3.0, the builtin has been dropped and **reduce()** is only available from **functools**; currently there are no plans to drop the builtin in the 2.x series. (Patched by Christian Heimes; bpo-1739906 [https://bugs.python.org/issue?@action=redirect&bpo=1739906].)

- When possible, the **getpass** module will now use `/dev/tty` to print a prompt message and read the password, falling back to standard error and standard input. If the password may be echoed to the terminal, a warning is printed before the prompt is displayed. (Contributed by Gregory P. Smith.)

- The **glob.glob()** function can now return Unicode filenames if a Unicode path was used and Unicode filenames are matched within the directory. (bpo-1001604 [https://bugs.python.org/issue?@action=redirect&bpo=1001604])

- A new function in the **heapq** module, `merge(iter1, iter2, ...)`, takes any number of iterables returning data in sorted order, and returns a new generator that returns the contents of all the iterators, also in sorted order. For example:

  ```
  >>> list(heapq.merge([1, 3, 5, 9], [2, 8, 16]))
  [1, 2, 3, 5, 8, 9, 16]
  ```

  Another new function, `heappushpop(heap, item)`, pushes *item* onto *heap*, then pops off and returns the smallest item. This is more efficient than making a call to **heappush()** and then **heappop()**.

  heapq is now implemented to only use less-than comparison, instead of the less-than-or-equal comparison it previously used. This makes **heapq**'s usage of a type match the **list.sort()** method. (Contributed by Raymond Hettinger.)

- An optional `timeout` parameter, specifying a timeout measured in seconds, was added to the **httplib.HTTPConnection** and **HTTPSConnection** class constructors. (Added by Facundo Batista.)

- Most of the **inspect** module's functions, such as **getmoduleinfo()** and **getargs()**, now return named tuples. In addition to behaving like tuples, the elements of the return value can also be accessed as attributes. (Contributed by Raymond Hettinger.)

  Some new functions in the module include **isgenerator()**, **isgeneratorfunction()**, and **isabstract()**.

- The **itertools** module gained several new functions.

  `izip_longest(iter1, iter2, ...[, fillvalue])` makes tuples from each of the elements; if some of the iterables are shorter than others, the missing values are set to *fillvalue*. For example:

  ```
  >>> tuple(itertools.izip_longest([1,2,3], [1,2,3,4,5]))
  ((1, 1), (2, 2), (3, 3), (None, 4), (None, 5))
  ```

`product(iter1, iter2, ..., [repeat=N])` returns the Cartesian product of the supplied iterables, a set of tuples containing every possible combination of the elements returned from each iterable.

```
>>> list(itertools.product([1,2,3], [4,5,6]))
[(1, 4), (1, 5), (1, 6),
 (2, 4), (2, 5), (2, 6),
 (3, 4), (3, 5), (3, 6)]
```

The optional *repeat* keyword argument is used for taking the product of an iterable or a set of iterables with themselves, repeated *N* times. With a single iterable argument, *N*-tuples are returned:

```
>>> list(itertools.product([1,2], repeat=3))
[(1, 1, 1), (1, 1, 2), (1, 2, 1), (1, 2, 2),
 (2, 1, 1), (2, 1, 2), (2, 2, 1), (2, 2, 2)]
```

With two iterables, *2N*-tuples are returned.

```
>>> list(itertools.product([1,2], [3,4], repeat=2))
[(1, 3, 1, 3), (1, 3, 1, 4), (1, 3, 2, 3), (1, 3, 2, 4),
 (1, 4, 1, 3), (1, 4, 1, 4), (1, 4, 2, 3), (1, 4, 2, 4),
 (2, 3, 1, 3), (2, 3, 1, 4), (2, 3, 2, 3), (2, 3, 2, 4),
 (2, 4, 1, 3), (2, 4, 1, 4), (2, 4, 2, 3), (2, 4, 2, 4)]
```

`combinations(iterable, r)` returns sub-sequences of length *r* from the elements of *iterable*.

```
>>> list(itertools.combinations('123', 2))
[('1', '2'), ('1', '3'), ('2', '3')]
>>> list(itertools.combinations('123', 3))
[('1', '2', '3')]
>>> list(itertools.combinations('1234', 3))
[('1', '2', '3'), ('1', '2', '4'),
 ('1', '3', '4'), ('2', '3', '4')]
```

`permutations(iter[, r])` returns all the permutations of length *r* of the iterable's elements. If *r* is not specified, it will default to the number of elements produced by the iterable.

```
>>> list(itertools.permutations([1,2,3,4], 2))
[(1, 2), (1, 3), (1, 4),
 (2, 1), (2, 3), (2, 4),
 (3, 1), (3, 2), (3, 4),
 (4, 1), (4, 2), (4, 3)]
```

`itertools.chain(*iterables)` is an existing function in **`itertools`** that gained a new constructor in Python 2.6. `itertools.chain.from_iterable(iterable)` takes a single iterable that should return other iterables. **`chain()`** will then return all the elements of the first iterable, then all the elements of the second, and so on.

```
>>> list(itertools.chain.from_iterable([[1,2,3], [4,5,6]]))
[1, 2, 3, 4, 5, 6]
```

(All contributed by Raymond Hettinger.)

- The **`logging`** module's **`FileHandler`** class and its subclasses **`WatchedFileHandler`**, **`RotatingFileHandler`**, and **`TimedRotatingFileHandler`** now have an optional *delay* parameter to their constructors. If *delay* is true, opening of the log file is deferred until the first **`emit()`** call is made. (Contributed by Vinay Sajip.)

  `TimedRotatingFileHandler` also has a *utc* constructor parameter. If the argument is true, UTC time will be used in determining when midnight occurs and in generating filenames; otherwise local time will be used.

- Several new functions were added to the **`math`** module:

  - **`isinf()`** and **`isnan()`** determine whether a given float is a (positive or negative) infinity or a NaN (Not a Number), respectively.
  - **`copysign()`** copies the sign bit of an IEEE 754 number, returning the absolute value of *x* combined with the sign bit of *y*. For example, `math.copysign(1, -0.0)` returns -1.0. (Contributed by Christian Heimes.)

- `factorial()` computes the factorial of a number. (Contributed by Raymond Hettinger; bpo-2138 [https://bugs.python.org/issue?@action=redirect&bpo=2138].)
- `fsum()` adds up the stream of numbers from an iterable, and is careful to avoid loss of precision through using partial sums. (Contributed by Jean Brouwers, Raymond Hettinger, and Mark Dickinson; bpo-2819 [https://bugs.python.org/issue?@action=redirect&bpo=2819].)
- `acosh()`, `asinh()` and `atanh()` compute the inverse hyperbolic functions.
- `log1p()` returns the natural logarithm of *1+x* (base *e*).
- `trunc()` rounds a number toward zero, returning the closest `Integral` that's between the function's argument and zero. Added as part of the backport of PEP 3141's type hierarchy for numbers.

- The `math` module has been improved to give more consistent behaviour across platforms, especially with respect to handling of floating-point exceptions and IEEE 754 special values.

  Whenever possible, the module follows the recommendations of the C99 standard about 754's special values. For example, `sqrt(-1.)` should now give a `ValueError` across almost all platforms, while `sqrt(float('NaN'))` should return a NaN on all IEEE 754 platforms. Where Annex 'F' of the C99 standard recommends signaling 'divide-by-zero' or 'invalid', Python will raise `ValueError`. Where Annex 'F' of the C99 standard recommends signaling 'overflow', Python will raise `OverflowError`. (See bpo-711019 [https://bugs.python.org/issue?@action=redirect&bpo=711019] and bpo-1640 [https://bugs.python.org/issue?@action=redirect&bpo=1640].)

  (Contributed by Christian Heimes and Mark Dickinson.)

- `mmap` objects now have a `rfind()` method that searches for a substring beginning at the end of the string and searching backwards. The `find()` method also gained an *end* parameter giving an index at which to stop searching. (Contributed by John Lenton.)

- The **operator** module gained a **methodcaller()** function that takes a name and an optional set of arguments, returning a callable that will call the named function on any arguments passed to it. For example:

```
>>> # Equivalent to lambda s: s.replace('old', 'new')
>>> replacer = operator.methodcaller('replace', 'old', 'new
>>> replacer('old wine in old bottles')
'new wine in new bottles'
```

  (Contributed by Georg Brandl, after a suggestion by Gregory Petrosyan.)

  The **attrgetter()** function now accepts dotted names and performs the corresponding attribute lookups:

```
>>> inst_name = operator.attrgetter(
...         '__class__.__name__')
>>> inst_name('')
'str'
>>> inst_name(help)
'_Helper'
```

  (Contributed by Georg Brandl, after a suggestion by Barry Warsaw.)

- The **os** module now wraps several new system calls. `fchmod(fd, mode)` and `fchown(fd, uid, gid)` change the mode and ownership of an opened file, and `lchmod(path, mode)` changes the mode of a symlink. (Contributed by Georg Brandl and Christian Heimes.)

  chflags() and **lchflags()** are wrappers for the corresponding system calls (where they're available), changing the flags set on a file. Constants for the flag values are defined in the **stat** module; some possible values include **UF_IMMUTABLE** to signal the file may not be changed and **UF_APPEND** to indicate that data can only be appended to the file. (Contributed by M. Levinson.)

  `os.closerange(low, high)` efficiently closes all file descriptors from *low* to *high*, ignoring any errors and not including *high* itself. This function is now used by the **subprocess** module to make starting

processes faster. (Contributed by Georg Brandl; bpo-1663329 [https://bugs.python.org/issue?@action=redirect&bpo=1663329].)

- The `os.environ` object's **`clear()`** method will now unset the environment variables using **`os.unsetenv()`** in addition to clearing the object's keys. (Contributed by Martin Horcicka; bpo-1181 [https://bugs.python.org/issue?@action=redirect&bpo=1181].)

- The **`os.walk()`** function now has a `followlinks` parameter. If set to True, it will follow symlinks pointing to directories and visit the directory's contents. For backward compatibility, the parameter's default value is false. Note that the function can fall into an infinite recursion if there's a symlink that points to a parent directory. (bpo-1273829 [https://bugs.python.org/issue?@action=redirect&bpo=1273829])

- In the **`os.path`** module, the **`splitext()`** function has been changed to not split on leading period characters. This produces better results when operating on Unix's dot-files. For example, `os.path.splitext('.ipython')` now returns `('.ipython', '')` instead of `('', '.ipython')`. (bpo-1115886 [https://bugs.python.org/issue?@action=redirect&bpo=1115886])

  A new function, `os.path.relpath(path, start='.')`, returns a relative path from the `start` path, if it's supplied, or from the current working directory to the destination `path`. (Contributed by Richard Barran; bpo-1339796 [https://bugs.python.org/issue?@action=redirect&bpo=1339796].)

  On Windows, **`os.path.expandvars()`** will now expand environment variables given in the form "%var%", and "~user" will be expanded into the user's home directory path. (Contributed by Josiah Carlson; bpo-957650 [https://bugs.python.org/issue?@action=redirect&bpo=957650].)

- The Python debugger provided by the **`pdb`** module gained a new command: "run" restarts the Python program being debugged and can optionally take new command-line arguments for the program. (Contributed by Rocky Bernstein; bpo-1393667 [https://bugs.python.org/issue?@action=redirect&bpo=1393667].)

- The **pdb.post_mortem()** function, used to begin debugging a traceback, will now use the traceback returned by **sys.exc_info()** if no traceback is supplied. (Contributed by Facundo Batista; bpo-1106316 [https://bugs.python.org/issue?@action=redirect&bpo=1106316].)

- The **pickletools** module now has an **optimize()** function that takes a string containing a pickle and removes some unused opcodes, returning a shorter pickle that contains the same data structure. (Contributed by Raymond Hettinger.)

- A **get_data()** function was added to the **pkgutil** module that returns the contents of resource files included with an installed Python package. For example:

```
>>> import pkgutil
>>> print pkgutil.get_data('test', 'exception_hierarchy.txt
BaseException
 +-- SystemExit
 +-- KeyboardInterrupt
 +-- GeneratorExit
 +-- Exception
      +-- StopIteration
      +-- StandardError
 ...
```

  (Contributed by Paul Moore; bpo-2439 [https://bugs.python.org/issue?@action=redirect&bpo=2439].)

- The **pyexpat** module's **Parser** objects now allow setting their **buffer_size** attribute to change the size of the buffer used to hold character data. (Contributed by Achim Gaedke; bpo-1137 [https://bugs.python.org/issue?@action=redirect&bpo=1137].)

- The **queue** module now provides queue variants that retrieve entries in different orders. The **PriorityQueue** class stores queued items in a heap and retrieves them in priority order, and **LifoQueue** retrieves the most recently added entries first, meaning that it behaves like a stack. (Contributed by Raymond Hettinger.)

- The `random` module's `Random` objects can now be pickled on a 32-bit system and unpickled on a 64-bit system, and vice versa. Unfortunately, this change also means that Python 2.6's `Random` objects can't be unpickled correctly on earlier versions of Python. (Contributed by Shawn Ligocki; bpo-1727780 [https://bugs.python.org/issue?@action=redirect&bpo=1727780].)

  The new `triangular(low, high, mode)` function returns random numbers following a triangular distribution. The returned values are between *low* and *high*, not including *high* itself, and with *mode* as the most frequently occurring value in the distribution. (Contributed by Wladmir van der Laan and Raymond Hettinger; bpo-1681432 [https://bugs.python.org/issue?@action=redirect&bpo=1681432].)

- Long regular expression searches carried out by the `re` module will check for signals being delivered, so time-consuming searches can now be interrupted. (Contributed by Josh Hoyt and Ralf Schmitt; bpo-846388 [https://bugs.python.org/issue?@action=redirect&bpo=846388].)

  The regular expression module is implemented by compiling bytecodes for a tiny regex-specific virtual machine. Untrusted code could create malicious strings of bytecode directly and cause crashes, so Python 2.6 includes a verifier for the regex bytecode. (Contributed by Guido van Rossum from work for Google App Engine; bpo-3487 [https://bugs.python.org/issue?@action=redirect&bpo=3487].)

- The `rlcompleter` module's `complete()` method will now ignore exceptions triggered while evaluating a name. (Fixed by Lorenz Quack; bpo-2250 [https://bugs.python.org/issue?@action=redirect&bpo=2250].)

- The `sched` module's `scheduler` instances now have a read-only `queue` attribute that returns the contents of the scheduler's queue, represented as a list of named tuples with the fields `(time, priority, action, argument)`. (Contributed by Raymond Hettinger; bpo-1861 [https://bugs.python.org/issue?@action=redirect&bpo=1861].)

- The `select` module now has wrapper functions for the Linux `epoll()` and BSD `kqueue()` system calls. `modify()` method was added to the

existing `poll` objects; `pollobj.modify(fd, eventmask)` takes a file descriptor or file object and an event mask, modifying the recorded event mask for that file. (Contributed by Christian Heimes; bpo-1657 [https://bugs.python.org/issue?@action=redirect&bpo=1657].)

- The **`shutil.copytree()`** function now has an optional *ignore* argument that takes a callable object. This callable will receive each directory path and a list of the directory's contents, and returns a list of names that will be ignored, not copied.

  The **`shutil`** module also provides an **`ignore_patterns()`** function for use with this new parameter. **`ignore_patterns()`** takes an arbitrary number of glob-style patterns and returns a callable that will ignore any files and directories that match any of these patterns. The following example copies a directory tree, but skips both `.svn` directories and Emacs backup files, which have names ending with '~':

  ```
  shutil.copytree('Doc/library', '/tmp/library',
                  ignore=shutil.ignore_patterns('*~', '.svn'))
  ```

  (Contributed by Tarek Ziadé; bpo-2663 [https://bugs.python.org/issue?@action=redirect&bpo=2663].)

- Integrating signal handling with GUI handling event loops like those used by Tkinter or GTk+ has long been a problem; most software ends up polling, waking up every fraction of a second to check if any GUI events have occurred. The **`signal`** module can now make this more efficient. Calling `signal.set_wakeup_fd(fd)` sets a file descriptor to be used; when a signal is received, a byte is written to that file descriptor. There's also a C-level function, **`PySignal_SetWakeupFd()`**, for setting the descriptor.

  Event loops will use this by opening a pipe to create two descriptors, one for reading and one for writing. The writable descriptor will be passed to **`set_wakeup_fd()`**, and the readable descriptor will be added to the list of descriptors monitored by the event loop via **`select()`** or

`poll()`. On receiving a signal, a byte will be written and the main event loop will be woken up, avoiding the need to poll.

(Contributed by Adam Olsen; bpo-1583 [https://bugs.python.org/issue?@action=redirect&bpo=1583].)

The **`siginterrupt()`** function is now available from Python code, and allows changing whether signals can interrupt system calls or not. (Contributed by Ralf Schmitt.)

The **`setitimer()`** and **`getitimer()`** functions have also been added (where they're available). **`setitimer()`** allows setting interval timers that will cause a signal to be delivered to the process after a specified time, measured in wall-clock time, consumed process time, or combined process+system time. (Contributed by Guilherme Polo; bpo-2240 [https://bugs.python.org/issue?@action=redirect&bpo=2240].)

- The **`smtplib`** module now supports SMTP over SSL thanks to the addition of the **`SMTP_SSL`** class. This class supports an interface identical to the existing **`SMTP`** class. (Contributed by Monty Taylor.) Both class constructors also have an optional `timeout` parameter that specifies a timeout for the initial connection attempt, measured in seconds. (Contributed by Facundo Batista.)

  An implementation of the LMTP protocol (**RFC 2033** [https://datatracker.ietf.org/doc/html/rfc2033.html]) was also added to the module. LMTP is used in place of SMTP when transferring e-mail between agents that don't manage a mail queue. (LMTP implemented by Leif Hedstrom; bpo-957003 [https://bugs.python.org/issue?@action=redirect&bpo=957003].)

  `smtplib.SMTP.starttls()` now complies with **RFC 3207** [https://datatracker.ietf.org/doc/html/rfc3207.html] and forgets any knowledge obtained from the server not obtained from the TLS negotiation itself. (Patch contributed by Bill Fenner; bpo-829951 [https://bugs.python.org/issue?@action=redirect&bpo=829951].)

- The **socket** module now supports TIPC (https://tipc.sourceforge.net/), a high-performance non-IP-based protocol designed for use in clustered environments. TIPC addresses are 4- or 5-tuples. (Contributed by Alberto Bertogli; bpo-1646 [https://bugs.python.org/issue?@action=redirect&bpo=1646].)

  A new function, **create_connection()**, takes an address and connects to it using an optional timeout value, returning the connected socket object. This function also looks up the address's type and connects to it using IPv4 or IPv6 as appropriate. Changing your code to use **create_connection()** instead of `socket(socket.AF_INET, ...)` may be all that's required to make your code work with IPv6.

- The base classes in the **SocketServer** module now support calling a **handle_timeout()** method after a span of inactivity specified by the server's **timeout** attribute. (Contributed by Michael Pomraning.) The **serve_forever()** method now takes an optional poll interval measured in seconds, controlling how often the server will check for a shutdown request. (Contributed by Pedro Werneck and Jeffrey Yasskin; bpo-742598 [https://bugs.python.org/issue?@action=redirect&bpo=742598], bpo-1193577 [https://bugs.python.org/issue?@action=redirect&bpo=1193577].)

- The **sqlite3** module, maintained by Gerhard Häring, has been updated from version 2.3.2 in Python 2.5 to version 2.4.1.

- The **struct** module now supports the C99 _Bool type, using the format character `'?'`. (Contributed by David Remahl.)

- The **Popen** objects provided by the **subprocess** module now have **terminate()**, **kill()**, and **send_signal()** methods. On Windows, **send_signal()** only supports the **SIGTERM** signal, and all these methods are aliases for the Win32 API function **TerminateProcess()**. (Contributed by Christian Heimes.)

- A new variable in the **sys** module, **float_info**, is an object containing information derived from the `float.h` file about the platform's floating-point support. Attributes of this object include **mant_dig** (number of digits in the mantissa), **epsilon** (smallest difference

between 1.0 and the next largest value representable), and several others. (Contributed by Christian Heimes; bpo-1534 [https://bugs.python.org/issue?@action=redirect&bpo=1534].)

Another new variable, **`dont_write_bytecode`**, controls whether Python writes any `.pyc` or `.pyo` files on importing a module. If this variable is true, the compiled files are not written. The variable is initially set on start-up by supplying the **`-B`** switch to the Python interpreter, or by setting the **`PYTHONDONTWRITEBYTECODE`** environment variable before running the interpreter. Python code can subsequently change the value of this variable to control whether bytecode files are written or not. (Contributed by Neal Norwitz and Georg Brandl.)

Information about the command-line arguments supplied to the Python interpreter is available by reading attributes of a named tuple available as `sys.flags`. For example, the **`verbose`** attribute is true if Python was executed in verbose mode, **`debug`** is true in debugging mode, etc. These attributes are all read-only. (Contributed by Christian Heimes.)

A new function, **`getsizeof()`**, takes a Python object and returns the amount of memory used by the object, measured in bytes. Built-in objects return correct results; third-party extensions may not, but can define a **`__sizeof__()`** method to return the object's size. (Contributed by Robert Schuppenies; bpo-2898 [https://bugs.python.org/issue?@action=redirect&bpo=2898].)

It's now possible to determine the current profiler and tracer functions by calling **`sys.getprofile()`** and **`sys.gettrace()`**. (Contributed by Georg Brandl; bpo-1648 [https://bugs.python.org/issue?@action=redirect&bpo=1648].)

- The **`tarfile`** module now supports POSIX.1-2001 (pax) tarfiles in addition to the POSIX.1-1988 (ustar) and GNU tar formats that were already supported. The default format is GNU tar; specify the `format` parameter to open a file using a different format:

```
tar = tarfile.open("output.tar", "w",
                   format=tarfile.PAX_FORMAT)
```

The new `encoding` and `errors` parameters specify an encoding and an error handling scheme for character conversions. `'strict'`, `'ignore'`, and `'replace'` are the three standard ways Python can handle errors,; `'utf-8'` is a special value that replaces bad characters with their UTF-8 representation. (Character conversions occur because the PAX format supports Unicode filenames, defaulting to UTF-8 encoding.)

The **`tarfile.TarFile.add()`** method now accepts an `exclude` argument that's a function that can be used to exclude certain filenames from an archive. The function must take a filename and return true if the file should be excluded or false if it should be archived. The function is applied to both the name initially passed to **`add()`** and to the names of files in recursively added directories.

(All changes contributed by Lars Gustäbel).

- An optional `timeout` parameter was added to the **`telnetlib.Telnet`** class constructor, specifying a timeout measured in seconds. (Added by Facundo Batista.)

- The **`tempfile.NamedTemporaryFile`** class usually deletes the temporary file it created when the file is closed. This behaviour can now be changed by passing `delete=False` to the constructor. (Contributed by Damien Miller; bpo-1537850 [https://bugs.python.org/issue?@action=redirect&bpo=1537850].)

  A new class, **`SpooledTemporaryFile`**, behaves like a temporary file but stores its data in memory until a maximum size is exceeded. On reaching that limit, the contents will be written to an on-disk temporary file. (Contributed by Dustin J. Mitchell.)

  The **`NamedTemporaryFile`** and **`SpooledTemporaryFile`** classes both work as context managers, so you can write `with tempfile.NamedTemporaryFile() as tmp: ...`. (Contributed by Alexander Belopolsky; bpo-2021 [https://bugs.python.org/issue?@action=redirect&bpo=2021].)

- The **test.test_support** module gained a number of context managers useful for writing tests. **EnvironmentVarGuard()** is a context manager that temporarily changes environment variables and automatically restores them to their old values.

  Another context manager, **TransientResource**, can surround calls to resources that may or may not be available; it will catch and ignore a specified list of exceptions. For example, a network test may ignore certain failures when connecting to an external web site:

  ```
  with test_support.TransientResource(IOError,
                                      errno=errno.ETIMEDOUT):
      f = urllib.urlopen('https://sf.net')
      ...
  ```

  Finally, **check_warnings()** resets the **warnings** module's warning filters and returns an object that will record all warning messages triggered (bpo-3781 [https://bugs.python.org/issue?@action=redirect&bpo=3781]):

  ```
  with test_support.check_warnings() as wrec:
      warnings.simplefilter("always")
      # ... code that triggers a warning ...
      assert str(wrec.message) == "function is outdated"
      assert len(wrec.warnings) == 1, "Multiple warnings raise
  ```

  (Contributed by Brett Cannon.)

- The **textwrap** module can now preserve existing whitespace at the beginnings and ends of the newly created lines by specifying `drop_whitespace=False` as an argument:

  ```
  >>> S = """This  sentence  has a bunch   of
  ...   extra   whitespace."""
  >>> print textwrap.fill(S, width=15)
  This  sentence
  has a bunch
  of    extra
  whitespace.
  >>> print textwrap.fill(S, drop_whitespace=False, width=15)
  This  sentence
  ```

```
  has a bunch
   of    extra
   whitespace.
>>>
```

(Contributed by Dwayne Bailey; bpo-1581073 [https://bugs.python.org/issue?@action=redirect&bpo=1581073].)

- The **threading** module API is being changed to use properties such as **daemon** instead of **setDaemon()** and **isDaemon()** methods, and some methods have been renamed to use underscores instead of camel-case; for example, the **activeCount()** method is renamed to **active_count()**. Both the 2.6 and 3.0 versions of the module support the same properties and renamed methods, but don't remove the old methods. No date has been set for the deprecation of the old APIs in Python 3.x; the old APIs won't be removed in any 2.x version. (Carried out by several people, most notably Benjamin Peterson.)

  The **threading** module's **Thread** objects gained an **ident** property that returns the thread's identifier, a nonzero integer. (Contributed by Gregory P. Smith; bpo-2871 [https://bugs.python.org/issue?@action=redirect&bpo=2871].)

- The **timeit** module now accepts callables as well as strings for the statement being timed and for the setup code. Two convenience functions were added for creating **Timer** instances: `repeat(stmt, setup, time, repeat, number)` and `timeit(stmt, setup, time, number)` create an instance and call the corresponding method. (Contributed by Erik Demaine; bpo-1533909 [https://bugs.python.org/issue?@action=redirect&bpo=1533909].)

- The **tkinter** module now accepts lists and tuples for options, separating the elements by spaces before passing the resulting value to Tcl/Tk. (Contributed by Guilherme Polo; bpo-2906 [https://bugs.python.org/issue?@action=redirect&bpo=2906].)

- The **turtle** module for turtle graphics was greatly enhanced by Gregor Lingl. New features in the module include:

- Better animation of turtle movement and rotation.
- Control over turtle movement using the new **`delay()`**, **`tracer()`**, and **`speed()`** methods.
- The ability to set new shapes for the turtle, and to define a new coordinate system.
- Turtles now have an **`undo()`** method that can roll back actions.
- Simple support for reacting to input events such as mouse and keyboard activity, making it possible to write simple games.
- A `turtle.cfg` file can be used to customize the starting appearance of the turtle's screen.
- The module's docstrings can be replaced by new docstrings that have been translated into another language.

(bpo-1513695 [https://bugs.python.org/issue?@action=redirect&bpo=1513695])

- An optional `timeout` parameter was added to the **`urllib.urlopen`** function and the **`urllib.ftpwrapper`** class constructor, as well as the **`urllib2.urlopen`** function. The parameter specifies a timeout measured in seconds. For example:

```
>>> u = urllib2.urlopen("http://slow.example.com",
                        timeout=3)
Traceback (most recent call last):
  ...
urllib2.URLError: <urlopen error timed out>
>>>
```

(Added by Facundo Batista.)

- The Unicode database provided by the **`unicodedata`** module has been updated to version 5.1.0. (Updated by Martin von Löwis; bpo-3811 [https://bugs.python.org/issue?@action=redirect&bpo=3811].)

- The **`warnings`** module's **`formatwarning()`** and **`showwarning()`** gained an optional *line* argument that can be used to supply the line of source code. (Added as part of bpo-1631171 [https://bugs.python.org/issue?@action=redirect&bpo=1631171], which re-implemented part of the **`warnings`** module in C code.)

A new function, `catch_warnings()`, is a context manager intended for testing purposes that lets you temporarily modify the warning filters and then restore their original values (bpo-3781 [https://bugs.python.org/issue?@action=redirect&bpo=3781]).

- The XML-RPC `SimpleXMLRPCServer` and `DocXMLRPCServer` classes can now be prevented from immediately opening and binding to their socket by passing `False` as the *bind_and_activate* constructor parameter. This can be used to modify the instance's `allow_reuse_address` attribute before calling the `server_bind()` and `server_activate()` methods to open the socket and begin listening for connections. (Contributed by Peter Parente; bpo-1599845 [https://bugs.python.org/issue?@action=redirect&bpo=1599845].)

  `SimpleXMLRPCServer` also has a `_send_traceback_header` attribute; if true, the exception and formatted traceback are returned as HTTP headers "X-Exception" and "X-Traceback". This feature is for debugging purposes only and should not be used on production servers because the tracebacks might reveal passwords or other sensitive information. (Contributed by Alan McIntyre as part of his project for Google's Summer of Code 2007.)

- The `xmlrpclib` module no longer automatically converts `datetime.date` and `datetime.time` to the `xmlrpclib.DateTime` type; the conversion semantics were not necessarily correct for all applications. Code using `xmlrpclib` should convert `date` and `time` instances. (bpo-1330538 [https://bugs.python.org/issue?@action=redirect&bpo=1330538]) The code can also handle dates before 1900 (contributed by Ralf Schmitt; bpo-2014 [https://bugs.python.org/issue?@action=redirect&bpo=2014]) and 64-bit integers represented by using `<i8>` in XML-RPC responses (contributed by Riku Lindblad; bpo-2985 [https://bugs.python.org/issue?@action=redirect&bpo=2985]).

- The `zipfile` module's `ZipFile` class now has `extract()` and `extractall()` methods that will unpack a single file or all the files in the archive to the current directory, or to a specified directory:

```
z = zipfile.ZipFile('python-251.zip')

# Unpack a single file, writing it relative
# to the /tmp directory.
z.extract('Python/sysmodule.c', '/tmp')

# Unpack all the files in the archive.
z.extractall()
```

(Contributed by Alan McIntyre; bpo-467924 [https://bugs.python.org/issue?@action=redirect&bpo=467924].)

The `open()`, `read()` and `extract()` methods can now take either a filename or a `ZipInfo` object. This is useful when an archive accidentally contains a duplicated filename. (Contributed by Graham Horler; bpo-1775025 [https://bugs.python.org/issue?@action=redirect&bpo=1775025].)

Finally, `zipfile` now supports using Unicode filenames for archived files. (Contributed by Alexey Borzenkov; bpo-1734346 [https://bugs.python.org/issue?@action=redirect&bpo=1734346].)

The `ast` module

The `ast` module provides an Abstract Syntax Tree representation of Python code, and Armin Ronacher contributed a set of helper functions that perform a variety of common tasks. These will be useful for HTML templating packages, code analyzers, and similar tools that process Python code.

The `parse()` function takes an expression and returns an AST. The `dump()` function outputs a representation of a tree, suitable for debugging:

```
import ast

t = ast.parse("""
d = {}
for i in 'abcdefghijklm':
    d[i + i] = ord(i) - ord('a') + 1
print d
```

```
""")
print ast.dump(t)
```

This outputs a deeply nested tree:

```
Module(body=[
  Assign(targets=[
    Name(id='d', ctx=Store())
   ], value=Dict(keys=[], values=[]))
  For(target=Name(id='i', ctx=Store()),
      iter=Str(s='abcdefghijklm'), body=[
    Assign(targets=[
      Subscript(value=
        Name(id='d', ctx=Load()),
          slice=
          Index(value=
            BinOp(left=Name(id='i', ctx=Load()), op=Add(),
             right=Name(id='i', ctx=Load()))), ctx=Store())
     ], value=
     BinOp(left=
      BinOp(left=
       Call(func=
        Name(id='ord', ctx=Load()), args=[
          Name(id='i', ctx=Load())
         ], keywords=[], starargs=None, kwargs=None),
       op=Sub(), right=Call(func=
        Name(id='ord', ctx=Load()), args=[
          Str(s='a')
         ], keywords=[], starargs=None, kwargs=None)),
       op=Add(), right=Num(n=1)))
    ], orelse=[])
   Print(dest=None, values=[
     Name(id='d', ctx=Load())
   ], nl=True)
 ])
```

The `literal_eval()` method takes a string or an AST representing a literal expression, parses and evaluates it, and returns the resulting value. A literal expression is a Python expression containing only strings, numbers, dictionaries, etc. but no statements or function calls. If you need to evaluate an expression but cannot accept the security risk of using an `eval()` call, **`literal_eval()`** will handle it safely:

```
>>> literal = '("a", "b", {2:4, 3:8, 1:2})'
>>> print ast.literal_eval(literal)
('a', 'b', {1: 2, 2: 4, 3: 8})
>>> print ast.literal_eval('"a" + "b"')
Traceback (most recent call last):
  ...
ValueError: malformed string
```

The module also includes **`NodeVisitor`** and **`NodeTransformer`** classes for traversing and modifying an AST, and functions for common transformations such as changing line numbers.

The `future_builtins` module

Python 3.0 makes many changes to the repertoire of built-in functions, and most of the changes can't be introduced in the Python 2.x series because they would break compatibility. The **`future_builtins`** module provides versions of these built-in functions that can be imported when writing 3.0-compatible code.

The functions in this module currently include:

- `ascii(obj)`: equivalent to **`repr()`**. In Python 3.0, **`repr()`** will return a Unicode string, while **`ascii()`** will return a pure ASCII bytestring.
- `filter(predicate, iterable)`, `map(func, iterable1, ...)`: the 3.0 versions return iterators, unlike the 2.x builtins which return lists.
- `hex(value)`, `oct(value)`: instead of calling the **`__hex__()`** or **`__oct__()`** methods, these versions will call the **`__index__()`** method and convert the result to hexadecimal or octal. **`oct()`** will use the new `0o` notation for its result.

The `json` module: JavaScript Object Notation

The new **`json`** module supports the encoding and decoding of Python types in JSON (Javascript Object Notation). JSON is a lightweight interchange format often used in web applications. For more information about JSON, see http://www.json.org.

`json` comes with support for decoding and encoding most built-in Python types. The following example encodes and decodes a dictionary:

```
>>> import json
>>> data = {"spam": "foo", "parrot": 42}
>>> in_json = json.dumps(data) # Encode the data
>>> in_json
'{"parrot": 42, "spam": "foo"}'
>>> json.loads(in_json) # Decode into a Python object
{"spam": "foo", "parrot": 42}
```

It's also possible to write your own decoders and encoders to support more types. Pretty-printing of the JSON strings is also supported.

`json` (originally called simplejson) was written by Bob Ippolito.

The `plistlib` module: A Property-List Parser

The `.plist` format is commonly used on Mac OS X to store basic data types (numbers, strings, lists, and dictionaries) by serializing them into an XML-based format. It resembles the XML-RPC serialization of data types.

Despite being primarily used on Mac OS X, the format has nothing Mac-specific about it and the Python implementation works on any platform that Python supports, so the `plistlib` module has been promoted to the standard library.

Using the module is simple:

```
import sys
import plistlib
import datetime as dt

# Create data structure
data_struct = dict(lastAccessed=dt.datetime.now(),
                   version=1,
                   categories=('Personal','Shared','Private'))

# Create string containing XML.
plist_str = plistlib.writePlistToString(data_struct)
```

```
new_struct = plistlib.readPlistFromString(plist_str)
print data_struct
print new_struct

# Write data structure to a file and read it back.
plistlib.writePlist(data_struct, '/tmp/customizations.plist')
new_struct = plistlib.readPlist('/tmp/customizations.plist')

# read/writePlist accepts file-like objects as well as paths.
plistlib.writePlist(data_struct, sys.stdout)
```

ctypes Enhancements

Thomas Heller continued to maintain and enhance the **`ctypes`** module.

`ctypes` now supports a **`c_bool`** datatype that represents the C99 `bool` type. (Contributed by David Remahl; bpo-1649190 [https://bugs.python.org/issue?@action=redirect&bpo=1649190].)

The **`ctypes`** string, buffer and array types have improved support for extended slicing syntax, where various combinations of `(start, stop, step)` are supplied. (Implemented by Thomas Wouters.)

All **`ctypes`** data types now support **`from_buffer()`** and **`from_buffer_copy()`** methods that create a ctypes instance based on a provided buffer object. **`from_buffer_copy()`** copies the contents of the object, while **`from_buffer()`** will share the same memory area.

A new calling convention tells **`ctypes`** to clear the `errno` or Win32 LastError variables at the outset of each wrapped call. (Implemented by Thomas Heller; bpo-1798 [https://bugs.python.org/issue?@action=redirect&bpo=1798].)

You can now retrieve the Unix `errno` variable after a function call. When creating a wrapped function, you can supply `use_errno=True` as a keyword parameter to the `DLL` function and then call the module-level methods **`set_errno()`** and **`get_errno()`** to set and retrieve the error value.

The Win32 LastError variable is similarly supported by the `DLL`, **`OleDLL()`**, and **`WinDLL()`** functions. You supply `use_last_error=True` as a keyword

parameter and then call the module-level methods `set_last_error()` and `get_last_error()`.

The `byref()` function, used to retrieve a pointer to a ctypes instance, now has an optional *offset* parameter that is a byte count that will be added to the returned pointer.

Improved SSL Support

Bill Janssen made extensive improvements to Python 2.6's support for the Secure Sockets Layer by adding a new module, `ssl`, that's built atop the OpenSSL [https://www.openssl.org/] library. This new module provides more control over the protocol negotiated, the X.509 certificates used, and has better support for writing SSL servers (as opposed to clients) in Python. The existing SSL support in the `socket` module hasn't been removed and continues to work, though it will be removed in Python 3.0.

To use the new module, you must first create a TCP connection in the usual way and then pass it to the `ssl.wrap_socket()` function. It's possible to specify whether a certificate is required, and to obtain certificate info by calling the `getpeercert()` method.

Ver também

The documentation for the `ssl` module.

Deprecations and Removals

- String exceptions have been removed. Attempting to use them raises a `TypeError`.
- Changes to the `Exception` interface as dictated by **PEP 352** [https://peps.python.org/pep-0352/] continue to be made. For 2.6, the `message` attribute is being deprecated in favor of the `args` attribute.

- (3.0-warning mode) Python 3.0 will feature a reorganized standard library that will drop many outdated modules and rename others. Python 2.6 running in 3.0-warning mode will warn about these modules when they are imported.

  The list of deprecated modules is: **`audiodev`**, **`bgenlocations`**, **`buildtools`**, **`bundlebuilder`**, **`Canvas`**, **`compiler`**, **`dircache`**, **`dl`**, **`fpformat`**, **`gensuitemodule`**, **`ihooks`**, **`imageop`**, **`imgfile`**, **`linuxaudiodev`**, **`mhlib`**, **`mimetools`**, **`multifile`**, **`new`**, **`pure`**, **`statvfs`**, **`sunaudiodev`**, **`test.testall`**, and **`toaiff`**.

- The **`gopherlib`** module has been removed.

- The **`MimeWriter`** module and **`mimify`** module have been deprecated; use the **`email`** package instead.

- The **`md5`** module has been deprecated; use the **`hashlib`** module instead.

- The **`posixfile`** module has been deprecated; **`fcntl.lockf()`** provides better locking.

- The **`popen2`** module has been deprecated; use the **`subprocess`** module.

- The **`rgbimg`** module has been removed.

- The **`sets`** module has been deprecated; it's better to use the built-in **`set`** and **`frozenset`** types.

- The **`sha`** module has been deprecated; use the **`hashlib`** module instead.

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- Python now must be compiled with C89 compilers (after 19 years!). This means that the Python source tree has dropped its own implementations of **`memmove()`** and **`strerror()`**, which are in the C89 standard library.

- Python 2.6 can be built with Microsoft Visual Studio 2008 (version 9.0), and this is the new default compiler. See the `PCbuild` directory for the build files. (Implemented by Christian Heimes.)

- On Mac OS X, Python 2.6 can be compiled as a 4-way universal build. The **configure** script can take a **`--with-universal-archs=[32-bit|64-bit|all]`** switch, controlling whether the binaries are built for 32-bit architectures (x86, PowerPC), 64-bit (x86-64 and PPC-64), or both. (Contributed by Ronald Oussoren.)

- A new function added in Python 2.6.6, **`PySys_SetArgvEx()`**, sets the value of `sys.argv` and can optionally update `sys.path` to include the directory containing the script named by `sys.argv[0]` depending on the value of an *updatepath* parameter.

  Esta função foi adicionada para fechar uma falha de segurança em aplicativos que incorporam Python. A função antiga, **`PySys_SetArgv()`**, sempre atualizava `sys.path`, e às vezes adicionava o diretório atual. Isso significava que, se você executasse uma aplicação incorporando Python em um diretório controlado por outra pessoa, os invasores poderiam colocar um módulo cavalo de Troia no diretório (digamos, um arquivo chamado `os.py`) que sua aplicação então usaria. importar e executar.

  If you maintain a C/C++ application that embeds Python, check whether you're calling **`PySys_SetArgv()`** and carefully consider whether the application should be using **`PySys_SetArgvEx()`** with *updatepath* set to false. Note that using this function will break compatibility with Python versions 2.6.5 and earlier; if you have to continue working with earlier versions, you can leave the call to **`PySys_SetArgv()`** alone and call `PyRun_SimpleString("sys.path.pop(0)\n")` afterwards to discard the first `sys.path` component.

  Security issue reported as **CVE 2008-5983** [https://www.cve.org/CVERecord?id=CVE-2008-5983]; discussed in gh-50003 [https://github.com/python/cpython/issues/50003], and fixed by Antoine Pitrou.

- The BerkeleyDB module now has a C API object, available as `bsddb.db.api`. This object can be used by other C extensions that wish to use the **`bsddb`** module for their own purposes. (Contributed by Duncan Grisby.)

- The new buffer interface, previously described in the PEP 3118 section, adds **`PyObject_GetBuffer()`** and **`PyBuffer_Release()`**, as well as a few other functions.

- Python's use of the C stdio library is now thread-safe, or at least as thread-safe as the underlying library is. A long-standing potential bug occurred if one thread closed a file object while another thread was reading from or writing to the object. In 2.6 file objects have a reference count, manipulated by the **`PyFile_IncUseCount()`** and **`PyFile_DecUseCount()`** functions. File objects can't be closed unless the reference count is zero. **`PyFile_IncUseCount()`** should be called while the GIL is still held, before carrying out an I/O operation using the `FILE *` pointer, and **`PyFile_DecUseCount()`** should be called immediately after the GIL is re-acquired. (Contributed by Antoine Pitrou and Gregory P. Smith.)

- Importing modules simultaneously in two different threads no longer deadlocks; it will now raise an **`ImportError`**. A new API function, **`PyImport_ImportModuleNoBlock()`**, will look for a module in `sys.modules` first, then try to import it after acquiring an import lock. If the import lock is held by another thread, an **`ImportError`** is raised. (Contributed by Christian Heimes.)

- Several functions return information about the platform's floating-point support. **`PyFloat_GetMax()`** returns the maximum representable floating-point value, and **`PyFloat_GetMin()`** returns the minimum positive value. **`PyFloat_GetInfo()`** returns an object containing more information from the `float.h` file, such as `"mant_dig"` (number of digits in the mantissa), `"epsilon"` (smallest difference between 1.0 and the next largest value representable), and several others. (Contributed by Christian Heimes; bpo-1534 [https://bugs.python.org/issue?@action=redirect&bpo=1534].)

- C functions and methods that use **PyComplex_AsCComplex()** will now accept arguments that have a **__complex__()** method. In particular, the functions in the **cmath** module will now accept objects with this method. This is a backport of a Python 3.0 change. (Contributed by Mark Dickinson; bpo-1675423 [https://bugs.python.org/issue?@action=redirect&bpo=1675423].)

- Python's C API now includes two functions for case-insensitive string comparisons, `PyOS_stricmp(char*, char*)` and `PyOS_strnicmp(char*, char*, Py_ssize_t)`. (Contributed by Christian Heimes; bpo-1635 [https://bugs.python.org/issue?@action=redirect&bpo=1635].)

- Many C extensions define their own little macro for adding integers and strings to the module's dictionary in the `init*` function. Python 2.6 finally defines standard macros for adding values to a module, **PyModule_AddStringMacro** and **PyModule_AddIntMacro()**. (Contributed by Christian Heimes.)

- Some macros were renamed in both 3.0 and 2.6 to make it clearer that they are macros, not functions. **Py_Size()** became **Py_SIZE()**, **Py_Type()** became **Py_TYPE()**, and **Py_Refcnt()** became **Py_REFCNT()**. The mixed-case macros are still available in Python 2.6 for backward compatibility. (bpo-1629 [https://bugs.python.org/issue?@action=redirect&bpo=1629])

- Distutils now places C extensions it builds in a different directory when running on a debug version of Python. (Contributed by Collin Winter; bpo-1530959 [https://bugs.python.org/issue?@action=redirect&bpo=1530959].)

- Several basic data types, such as integers and strings, maintain internal free lists of objects that can be re-used. The data structures for these free lists now follow a naming convention: the variable is always named `free_list`, the counter is always named `numfree`, and a macro `Py<typename>_MAXFREELIST` is always defined.

- A new Makefile target, “make patchcheck”, prepares the Python source tree for making a patch: it fixes trailing whitespace in all modified `.py` files, checks whether the documentation has been changed, and reports whether the `Misc/ACKS` and `Misc/NEWS` files have been updated. (Contributed by Brett Cannon.)

  Another new target, “make profile-opt”, compiles a Python binary using GCC’s profile-guided optimization. It compiles Python with profiling enabled, runs the test suite to obtain a set of profiling results, and then compiles using these results for optimization. (Contributed by Gregory P. Smith.)

Port-Specific Changes: Windows

- The support for Windows 95, 98, ME and NT4 has been dropped. Python 2.6 requires at least Windows 2000 SP4.

- The new default compiler on Windows is Visual Studio 2008 (version 9.0). The build directories for Visual Studio 2003 (version 7.1) and 2005 (version 8.0) were moved into the PC/ directory. The new `PCbuild` directory supports cross compilation for X64, debug builds and Profile Guided Optimization (PGO). PGO builds are roughly 10% faster than normal builds. (Contributed by Christian Heimes with help from Amaury Forgeot d’Arc and Martin von Löwis.)

- The **`msvcrt`** module now supports both the normal and wide char variants of the console I/O API. The **`getwch()`** function reads a keypress and returns a Unicode value, as does the **`getwche()`** function. The **`putwch()`** function takes a Unicode character and writes it to the console. (Contributed by Christian Heimes.)

- **`os.path.expandvars()`** will now expand environment variables in the form “%var%”, and “~user” will be expanded into the user’s home directory path. (Contributed by Josiah Carlson; bpo-957650 [https://bugs.python.org/issue?@action=redirect&bpo=957650].)

- The **`socket`** module’s socket objects now have an **`ioctl()`** method that provides a limited interface to the **`WSAIoctl()`** system interface.

- The `_winreg` module now has a function, `ExpandEnvironmentStrings()`, that expands environment variable references such as `%NAME%` in an input string. The handle objects provided by this module now support the context protocol, so they can be used in `with` statements. (Contributed by Christian Heimes.)

  `_winreg` also has better support for x64 systems, exposing the `DisableReflectionKey()`, `EnableReflectionKey()`, and `QueryReflectionKey()` functions, which enable and disable registry reflection for 32-bit processes running on 64-bit systems. (bpo-1753245 [https://bugs.python.org/issue?@action=redirect&bpo=1753245])

- The **msilib** module's **Record** object gained **GetInteger()** and **GetString()** methods that return field values as an integer or a string. (Contributed by Floris Bruynooghe; bpo-2125 [https://bugs.python.org/issue?@action=redirect&bpo=2125].)

Port-Specific Changes: Mac OS X

- When compiling a framework build of Python, you can now specify the framework name to be used by providing the **--with-framework-name=** option to the **configure** script.
- The **macfs** module has been removed. This in turn required the **macostools.touched()** function to be removed because it depended on the **macfs** module. (bpo-1490190 [https://bugs.python.org/issue?@action=redirect&bpo=1490190])
- Many other Mac OS modules have been deprecated and will be removed in Python 3.0: **_builtinSuites**, **aepack**, **aetools**, **aetypes**, **applesingle**, **appletrawmain**, **appletrunner**, **argvemulator**, **Audio_mac**, **autoGIL**, **Carbon**, **cfmfile**, **CodeWarrior**, **ColorPicker**, **EasyDialogs**, **Explorer**, **Finder**, **FrameWork**, **findertools**, **ic**, **icglue**, **icopen**, **macerrors**, **MacOS**, **macfs**, **macostools**, **macresource**, **MiniAEFrame**, **Nav**, **Netscape**, **OSATerminology**, **pimp**, **PixMapWrapper**, **StdSuites**, **SystemEvents**, **Terminal**, and **terminalcommand**.

Port-Specific Changes: IRIX

A number of old IRIX-specific modules were deprecated and will be removed in Python 3.0: **`al`** and **`AL`**, **`cd`**, **`cddb`**, **`cdplayer`**, **`CL`** and **`cl`**, **`DEVICE`**, **`ERRNO`**, **`FILE`**, **`FL`** and **`fl`**, **`flp`**, **`fm`**, **`GET`**, **`GLWS`**, **`GL`** and **`gl`**, **`IN`**, **`IOCTL`**, **`jpeg`**, **`panelparser`**, **`readcd`**, **`SV`** and **`sv`**, **`torgb`**, **`videoreader`**, and **`WAIT`**.

Porting to Python 2.6

Esta seção lista as alterações descritas anteriormente e outras correções de bugs que podem exigir alterações em seu código:

- Classes that aren't supposed to be hashable should set `__hash__ = None` in their definitions to indicate the fact.

- String exceptions have been removed. Attempting to use them raises a **`TypeError`**.

- The **`__init__()`** method of **`collections.deque`** now clears any existing contents of the deque before adding elements from the iterable. This change makes the behavior match `list.__init__()`.

- **`object.__init__()`** previously accepted arbitrary arguments and keyword arguments, ignoring them. In Python 2.6, this is no longer allowed and will result in a **`TypeError`**. This will affect **`__init__()`** methods that end up calling the corresponding method on **`object`** (perhaps through using **`super()`**). See bpo-1683368 [https://bugs.python.org/issue?@action=redirect&bpo=1683368] for discussion.

- The **`Decimal`** constructor now accepts leading and trailing whitespace when passed a string. Previously it would raise an **`InvalidOperation`** exception. On the other hand, the **`create_decimal()`** method of **`Context`** objects now explicitly disallows extra whitespace, raising a **`ConversionSyntax`** exception.

- Due to an implementation accident, if you passed a file path to the built-in **`__import__()`** function, it would actually import the specified file. This was never intended to work, however, and the

implementation now explicitly checks for this case and raises an `ImportError`.

- C API: the `PyImport_Import()` and `PyImport_ImportModule()` functions now default to absolute imports, not relative imports. This will affect C extensions that import other modules.

- C API: extension data types that shouldn't be hashable should define their `tp_hash` slot to `PyObject_HashNotImplemented()`.

- The `socket` module exception `socket.error` now inherits from `IOError`. Previously it wasn't a subclass of `StandardError` but now it is, through `IOError`. (Implemented by Gregory P. Smith; bpo-1706815 [https://bugs.python.org/issue?@action=redirect&bpo=1706815].)

- The `xmlrpclib` module no longer automatically converts `datetime.date` and `datetime.time` to the `xmlrpclib.DateTime` type; the conversion semantics were not necessarily correct for all applications. Code using `xmlrpclib` should convert `date` and `time` instances. (bpo-1330538 [https://bugs.python.org/issue?@action=redirect&bpo=1330538])

- (3.0-warning mode) The `Exception` class now warns when accessed using slicing or index access; having `Exception` behave like a tuple is being phased out.

- (3.0-warning mode) inequality comparisons between two dictionaries or two objects that don't implement comparison methods are reported as warnings. `dict1 == dict2` still works, but `dict1 < dict2` is being phased out.

  Comparisons between cells, which are an implementation detail of Python's scoping rules, also cause warnings because such comparisons are forbidden entirely in 3.0.

For applications that embed Python:

- The **`PySys_SetArgvEx()`** function was added in Python 2.6.6, letting applications close a security hole when the existing **`PySys_SetArgv()`** function was used. Check whether you're calling **`PySys_SetArgv()`** and carefully consider whether the application should be using **`PySys_SetArgvEx()`** with *updatepath* set to false.

Reconhecimentos

The author would like to thank the following people for offering suggestions, corrections and assistance with various drafts of this article: Georg Brandl, Steve Brown, Nick Coghlan, Ralph Corderoy, Jim Jewett, Kent Johnson, Chris Lambacher, Martin Michlmayr, Antoine Pitrou, Brian Warner.

What’s New in Python 2.5

Autor:: A.M. Kuchling

This article explains the new features in Python 2.5. The final release of Python 2.5 is scheduled for August 2006; **PEP 356** [https://peps.python.org/pep-0356/] describes the planned release schedule. Python 2.5 was released on September 19, 2006.

The changes in Python 2.5 are an interesting mix of language and library improvements. The library enhancements will be more important to Python’s user community, I think, because several widely useful packages were added. New modules include ElementTree for XML processing (`xml.etree.ElementTree`), the SQLite database module (`sqlite3`), and the `ctypes` module for calling C functions.

The language changes are of middling significance. Some pleasant new features were added, but most of them aren’t features that you’ll use every day. Conditional expressions were finally added to the language using a novel syntax; see section PEP 308: Conditional Expressions. The new ‘`with`’ statement will make writing cleanup code easier (section PEP 343: The ‘with’ statement). Values can now be passed into generators (section PEP 342: New Generator Features). Imports are now visible as either absolute or relative (section PEP 328: Absolute and Relative Imports). Some corner cases of exception handling are handled better (section PEP 341: Unified try/except/finally). All these improvements are worthwhile, but they’re improvements to one specific language feature or another; none of them are broad modifications to Python’s semantics.

As well as the language and library additions, other improvements and bugfixes were made throughout the source tree. A search through the SVN change logs finds there were 353 patches applied and 458 bugs fixed between Python 2.4 and 2.5. (Both figures are likely to be underestimates.)

This article doesn't try to be a complete specification of the new features; instead changes are briefly introduced using helpful examples. For full details, you should always refer to the documentation for Python 2.5 at https://docs.python.org. If you want to understand the complete implementation and design rationale, refer to the PEP for a particular new feature.

Comments, suggestions, and error reports for this document are welcome; please e-mail them to the author or open a bug in the Python bug tracker.

PEP 308: Conditional Expressions

For a long time, people have been requesting a way to write conditional expressions, which are expressions that return value A or value B depending on whether a Boolean value is true or false. A conditional expression lets you write a single assignment statement that has the same effect as the following:

```
if condition:
    x = true_value
else:
    x = false_value
```

There have been endless tedious discussions of syntax on both python-dev and comp.lang.python. A vote was even held that found the majority of voters wanted conditional expressions in some form, but there was no syntax that was preferred by a clear majority. Candidates included C's `cond ? true_v : false_v`, `if cond then true_v else false_v`, and 16 other variations.

Guido van Rossum eventually chose a surprising syntax:

```
x = true_value if condition else false_value
```

Evaluation is still lazy as in existing Boolean expressions, so the order of evaluation jumps around a bit. The *condition* expression in the middle is evaluated first, and the *true_value* expression is evaluated only if the

condition was true. Similarly, the *false_value* expression is only evaluated when the condition is false.

This syntax may seem strange and backwards; why does the condition go in the *middle* of the expression, and not in the front as in C's `c ? x : y`? The decision was checked by applying the new syntax to the modules in the standard library and seeing how the resulting code read. In many cases where a conditional expression is used, one value seems to be the 'common case' and one value is an 'exceptional case', used only on rarer occasions when the condition isn't met. The conditional syntax makes this pattern a bit more obvious:

```
contents = ((doc + '\n') if doc else '')
```

I read the above statement as meaning "here *contents* is usually assigned a value of `doc+'\n'`; sometimes *doc* is empty, in which special case an empty string is returned." I doubt I will use conditional expressions very often where there isn't a clear common and uncommon case.

There was some discussion of whether the language should require surrounding conditional expressions with parentheses. The decision was made to *not* require parentheses in the Python language's grammar, but as a matter of style I think you should always use them. Consider these two statements:

```
# First version -- no parens
level = 1 if logging else 0

# Second version -- with parens
level = (1 if logging else 0)
```

In the first version, I think a reader's eye might group the statement into 'level = 1', 'if logging', 'else 0', and think that the condition decides whether the assignment to *level* is performed. The second version reads better, in my opinion, because it makes it clear that the assignment is always performed and the choice is being made between two values.

Another reason for including the brackets: a few odd combinations of list comprehensions and lambdas could look like incorrect conditional

expressions. See **PEP 308** [https://peps.python.org/pep-0308/] for some examples. If you put parentheses around your conditional expressions, you won't run into this case.

Ver também

PEP 308 **[https://peps.python.org/pep-0308/]** **- Conditional Expressions**
PEP written by Guido van Rossum and Raymond D. Hettinger; implemented by Thomas Wouters.

PEP 309: Partial Function Application

The `functools` module is intended to contain tools for functional-style programming.

One useful tool in this module is the `partial()` function. For programs written in a functional style, you'll sometimes want to construct variants of existing functions that have some of the parameters filled in. Consider a Python function `f(a, b, c)`; you could create a new function `g(b, c)` that was equivalent to `f(1, b, c)`. This is called "partial function application".

`partial()` takes the arguments `(function, arg1, arg2, ... kwarg1=value1, kwarg2=value2)`. The resulting object is callable, so you can just call it to invoke *function* with the filled-in arguments.

Aqui está um pequeno mas bem realístico exemplo:

```
import functools

def log (message, subsystem):
    "Write the contents of 'message' to the specified subsystem.
    print '%s: %s' % (subsystem, message)
    ...

server_log = functools.partial(log, subsystem='server')
server_log('Unable to open socket')
```

Here's another example, from a program that uses PyGTK. Here a context-sensitive pop-up menu is being constructed dynamically. The callback provided for the menu option is a partially applied version of the `open_item()` method, where the first argument has been provided.

```
...
class Application:
    def open_item(self, path):
       ...
    def init (self):
        open_func = functools.partial(self.open_item, item_path)
        popup_menu.append( ("Open", open_func, 1) )
```

Another function in the **`functools`** module is the `update_wrapper(wrapper, wrapped)` function that helps you write well-behaved decorators. **`update_wrapper()`** copies the name, module, and docstring attribute to a wrapper function so that tracebacks inside the wrapped function are easier to understand. For example, you might write:

```
def my_decorator(f):
    def wrapper(*args, **kwds):
        print 'Calling decorated function'
        return f(*args, **kwds)
    functools.update_wrapper(wrapper, f)
    return wrapper
```

`wraps()` is a decorator that can be used inside your own decorators to copy the wrapped function's information. An alternate version of the previous example would be:

```
def my_decorator(f):
    @functools.wraps(f)
    def wrapper(*args, **kwds):
        print 'Calling decorated function'
        return f(*args, **kwds)
    return wrapper
```

Ver também

PEP 309 [https://peps.python.org/pep-0309/] - Partial Function Application

PEP proposed and written by Peter Harris; implemented by Hye-Shik Chang and Nick Coghlan, with adaptations by Raymond Hettinger.

PEP 314: Metadata for Python Software Packages v1.1

Some simple dependency support was added to Distutils. The **`setup()`** function now has `requires`, `provides`, and `obsoletes` keyword parameters. When you build a source distribution using the `sdist` command, the dependency information will be recorded in the `PKG-INFO` file.

Another new keyword parameter is `download_url`, which should be set to a URL for the package's source code. This means it's now possible to look up an entry in the package index, determine the dependencies for a package, and download the required packages.

```
VERSION = '1.0'
setup(name='PyPackage',
      version=VERSION,
      requires=['numarray', 'zlib (>=1.1.4)'],
      obsoletes=['OldPackage']
      download_url=('http://www.example.com/pypackage/dist/pkg-%
                    % VERSION),
     )
```

Another new enhancement to the Python package index at https://pypi.org is storing source and binary archives for a package. The new **upload** Distutils command will upload a package to the repository.

Before a package can be uploaded, you must be able to build a distribution using the **sdist** Distutils command. Once that works, you can run `python setup.py upload` to add your package to the PyPI archive. Optionally you can GPG-sign the package by supplying the **`--sign`** and **`--identity`** options.

Package uploading was implemented by Martin von Löwis and Richard Jones.

Ver também

PEP 314 [https://peps.python.org/pep-0314/] - Metadata for Python Software Packages v1.1

PEP proposed and written by A.M. Kuchling, Richard Jones, and Fred Drake; implemented by Richard Jones and Fred Drake.

PEP 328: Absolute and Relative Imports

The simpler part of **PEP 328** [https://peps.python.org/pep-0328/] was implemented in Python 2.4: parentheses could now be used to enclose the names imported from a module using the `from ... import ...` statement, making it easier to import many different names.

The more complicated part has been implemented in Python 2.5: importing a module can be specified to use absolute or package-relative imports. The plan is to move toward making absolute imports the default in future versions of Python.

Let's say you have a package directory like this:

```
pkg/
pkg/__init__.py
pkg/main.py
pkg/string.py
```

This defines a package named `pkg` containing the `pkg.main` and `pkg.string` submodules.

Consider the code in the `main.py` module. What happens if it executes the statement `import string`? In Python 2.4 and earlier, it will first look in the package's directory to perform a relative import, finds `pkg/string.py`, imports the contents of that file as the `pkg.string` module, and that module is bound to the name `string` in the `pkg.main` module's namespace.

That's fine if `pkg.string` was what you wanted. But what if you wanted Python's standard **`string`** module? There's no clean way to ignore `pkg.string` and look for the standard module; generally you had to look at the contents of `sys.modules`, which is slightly unclean. Holger Krekel's **`py.std`** package provides a tidier way to perform imports from the standard library, `import py; py.std.string.join()`, but that package isn't available on all Python installations.

Reading code which relies on relative imports is also less clear, because a reader may be confused about which module, **`string`** or `pkg.string`, is intended to be used. Python users soon learned not to duplicate the names of standard library modules in the names of their packages' submodules, but you can't protect against having your submodule's name being used for a new module added in a future version of Python.

In Python 2.5, you can switch **`import`**'s behaviour to absolute imports using a `from __future__ import absolute_import` directive. This absolute-import behaviour will become the default in a future version (probably Python 2.7). Once absolute imports are the default, `import string` will always find the standard library's version. It's suggested that users should begin using absolute imports as much as possible, so it's preferable to begin writing `from pkg import string` in your code.

Relative imports are still possible by adding a leading period to the module name when using the `from ... import` form:

```
# Import names from pkg.string
from .string import name1, name2
# Import pkg.string
from . import string
```

This imports the **`string`** module relative to the current package, so in `pkg.main` this will import *name1* and *name2* from `pkg.string`. Additional leading periods perform the relative import starting from the parent of the current package. For example, code in the `A.B.C` module can do:

```
from . import D                 # Imports A.B.D
from .. import E                # Imports A.E
from ..F import G               # Imports A.F.G
```

Leading periods cannot be used with the `import modname` form of the import statement, only the `from ... import` form.

Ver também

PEP 328 [https://peps.python.org/pep-0328/] - Importações: Multilinha e absoluta/relativa
PEP written by Aahz; implemented by Thomas Wouters.

https://pylib.readthedocs.io/
The py library by Holger Krekel, which contains the **`py.std`** package.

PEP 338: Executing Modules as Scripts

The `-m` switch added in Python 2.4 to execute a module as a script gained a few more abilities. Instead of being implemented in C code inside the Python interpreter, the switch now uses an implementation in a new module, **`runpy`**.

The **`runpy`** module implements a more sophisticated import mechanism so that it's now possible to run modules in a package such as **`pychecker.checker`**. The module also supports alternative import mechanisms such as the **`zipimport`** module. This means you can add a .zip archive's path to `sys.path` and then use the `-m` switch to execute code from the archive.

Ver também

PEP 338 [https://peps.python.org/pep-0338/] - Executing modules as scripts
PEP written and implemented by Nick Coghlan.

PEP 341: Unified try/except/finally

Until Python 2.5, the `try` statement came in two flavours. You could use a `finally` block to ensure that code is always executed, or one or more `except` blocks to catch specific exceptions. You couldn't combine both **`except`** blocks and a **`finally`** block, because generating the right bytecode for the combined version was complicated and it wasn't clear what the semantics of the combined statement should be.

Guido van Rossum spent some time working with Java, which does support the equivalent of combining `except` blocks and a `finally` block, and this clarified what the statement should mean. In Python 2.5, you can now write:

```
try:
    block-1 ...
except Exception1:
    handler-1 ...
except Exception2:
    handler-2 ...
else:
    else-block
finally:
    final-block
```

The code in *block-1* is executed. If the code raises an exception, the various `except` blocks are tested: if the exception is of class `Exception1`, *handler-1* is executed; otherwise if it's of class `Exception2`, *handler-2* is executed, and so forth. If no exception is raised, the *else-block* is executed.

No matter what happened previously, the *final-block* is executed once the code block is complete and any raised exceptions handled. Even if there's an error in an exception handler or the *else-block* and a new exception is raised, the code in the *final-block* is still run.

Ver também

PEP 341 [https://peps.python.org/pep-0341/] - Unifying try-except and try-finally

PEP written by Georg Brandl; implementation by Thomas Lee.

PEP 342: New Generator Features

Python 2.5 adds a simple way to pass values *into* a generator. As introduced in Python 2.3, generators only produce output; once a generator's code was invoked to create an iterator, there was no way to pass any new information into the function when its execution is resumed. Sometimes the ability to pass in some information would be useful. Hackish solutions to this include making the generator's code look at a global variable and then changing the global variable's value, or passing in some mutable object that callers then modify.

To refresh your memory of basic generators, here's a simple example:

```
def counter (maximum):
    i = 0
    while i < maximum:
        yield i
        i += 1
```

When you call `counter(10)`, the result is an iterator that returns the values from 0 up to 9. On encountering the **`yield`** statement, the iterator returns the provided value and suspends the function's execution, preserving the local variables. Execution resumes on the following call to the iterator's **`next()`** method, picking up after the **`yield`** statement.

In Python 2.3, **`yield`** was a statement; it didn't return any value. In 2.5, **`yield`** is now an expression, returning a value that can be assigned to a variable or otherwise operated on:

```
val = (yield i)
```

I recommend that you always put parentheses around a **`yield`** expression when you're doing something with the returned value, as in the above example. The parentheses aren't always necessary, but it's easier to always add them instead of having to remember when they're needed.

(**PEP 342** [https://peps.python.org/pep-0342/] explains the exact rules, which are that a **`yield`**-expression must always be parenthesized except when it

occurs at the top-level expression on the right-hand side of an assignment. This means you can write `val = yield i` but have to use parentheses when there's an operation, as in `val = (yield i) + 12`.)

Values are sent into a generator by calling its `send(value)` method. The generator's code is then resumed and the **`yield`** expression returns the specified *value*. If the regular **`next()`** method is called, the **`yield`** returns **`None`**.

Here's the previous example, modified to allow changing the value of the internal counter.

```
def counter (maximum):
    i = 0
    while i < maximum:
        val = (yield i)
        # If value provided, change counter
        if val is not None:
            i = val
        else:
            i += 1
```

And here's an example of changing the counter:

```
>>> it = counter(10)
>>> print it.next()
0
>>> print it.next()
1
>>> print it.send(8)
8
>>> print it.next()
9
>>> print it.next()
Traceback (most recent call last):
  File "t.py", line 15, in ?
    print it.next()
StopIteration
```

`yield` will usually return **`None`**, so you should always check for this case. Don't just use its value in expressions unless you're sure that the **`send()`**

method will be the only method used to resume your generator function.

In addition to `send()`, there are two other new methods on generators:

- `throw(type, value=None, traceback=None)` is used to raise an exception inside the generator; the exception is raised by the `yield` expression where the generator's execution is paused.

- `close()` raises a new `GeneratorExit` exception inside the generator to terminate the iteration. On receiving this exception, the generator's code must either raise `GeneratorExit` or `StopIteration`. Catching the `GeneratorExit` exception and returning a value is illegal and will trigger a `RuntimeError`; if the function raises some other exception, that exception is propagated to the caller. `close()` will also be called by Python's garbage collector when the generator is garbage-collected.

  If you need to run cleanup code when a `GeneratorExit` occurs, I suggest using a `try: ... finally:` suite instead of catching `GeneratorExit`.

The cumulative effect of these changes is to turn generators from one-way producers of information into both producers and consumers.

Generators also become *coroutines*, a more generalized form of subroutines. Subroutines are entered at one point and exited at another point (the top of the function, and a `return` statement), but coroutines can be entered, exited, and resumed at many different points (the `yield` statements). We'll have to figure out patterns for using coroutines effectively in Python.

The addition of the `close()` method has one side effect that isn't obvious. `close()` is called when a generator is garbage-collected, so this means the generator's code gets one last chance to run before the generator is destroyed. This last chance means that `try...finally` statements in generators can now be guaranteed to work; the `finally` clause will now always get a chance to run. The syntactic restriction that you couldn't mix `yield` statements with a `try...finally` suite has therefore been removed. This seems like a minor bit of language trivia, but using generators and

`try...finally` is actually necessary in order to implement the `with` statement described by **PEP 343** [https://peps.python.org/pep-0343/]. I'll look at this new statement in the following section.

Another even more esoteric effect of this change: previously, the **`gi_frame`** attribute of a generator was always a frame object. It's now possible for **`gi_frame`** to be `None` once the generator has been exhausted.

Ver também

PEP 342 [https://peps.python.org/pep-0342/] - Corrotinas via Geradores Aprimorados

PEP written by Guido van Rossum and Phillip J. Eby; implemented by Phillip J. Eby. Includes examples of some fancier uses of generators as coroutines.

Earlier versions of these features were proposed in **PEP 288** [https://peps.python.org/pep-0288/] by Raymond Hettinger and **PEP 325** [https://peps.python.org/pep-0325/] by Samuele Pedroni.

https://en.wikipedia.org/wiki/Coroutine

The Wikipedia entry for coroutines.

https://web.archive.org/web/20160321211320/http://www.sidhe.org/~dan/blog/archives/000178.html

An explanation of coroutines from a Perl point of view, written by Dan Sugalski.

PEP 343: The 'with' statement

The '`with`' statement clarifies code that previously would use `try...finally` blocks to ensure that clean-up code is executed. In this section, I'll discuss the statement as it will commonly be used. In the next section, I'll examine the implementation details and show how to write objects for use with this statement.

The ‘`with`’ statement is a new control-flow structure whose basic structure is:

```
with expression [as variable]:
    with-block
```

The expression is evaluated, and it should result in an object that supports the context management protocol (that is, has `__enter__()` and `__exit__()` methods.

The object’s `__enter__()` is called before *with-block* is executed and therefore can run set-up code. It also may return a value that is bound to the name *variable*, if given. (Note carefully that *variable* is *not* assigned the result of *expression*.)

After execution of the *with-block* is finished, the object’s `__exit__()` method is called, even if the block raised an exception, and can therefore run clean-up code.

To enable the statement in Python 2.5, you need to add the following directive to your module:

```
from __future__ import with_statement
```

The statement will always be enabled in Python 2.6.

Some standard Python objects now support the context management protocol and can be used with the ‘`with`’ statement. File objects are one example:

```
with open('/etc/passwd', 'r') as f:
    for line in f:
        print line
        ... more processing code ...
```

After this statement has executed, the file object in *f* will have been automatically closed, even if the `for` loop raised an exception part-way through the block.

Nota

In this case, *f* is the same object created by `open()`, because `__enter__()` returns *self*.

As travas e variáveis de condição do módulo `threading` também oferecem suporte à instrução '`with`':

```
lock = threading.Lock()
with lock:
    # Critical section of code
    ...
```

The lock is acquired before the block is executed and always released once the block is complete.

The new `localcontext()` function in the `decimal` module makes it easy to save and restore the current decimal context, which encapsulates the desired precision and rounding characteristics for computations:

```
from decimal import Decimal, Context, localcontext

# Displays with default precision of 28 digits
v = Decimal('578')
print v.sqrt()

with localcontext(Context(prec=16)):
    # All code in this block uses a precision of 16 digits.
    # The original context is restored on exiting the block.
    print v.sqrt()
```

Writing Context Managers

Under the hood, the '`with`' statement is fairly complicated. Most people will only use '**`with`**' in company with existing objects and don't need to know these details, so you can skip the rest of this section if you like. Authors of new objects will need to understand the details of the underlying implementation and should keep reading.

A high-level explanation of the context management protocol is:

- The expression is evaluated and should result in an object called a "context manager". The context manager must have `__enter__()` and `__exit__()` methods.
- The context manager's `__enter__()` method is called. The value returned is assigned to *VAR*. If no `'as VAR'` clause is present, the value is simply discarded.
- The code in *BLOCK* is executed.
- If *BLOCK* raises an exception, the `__exit__(type, value, traceback)` is called with the exception details, the same values returned by `sys.exc_info()`. The method's return value controls whether the exception is re-raised: any false value re-raises the exception, and `True` will result in suppressing it. You'll only rarely want to suppress the exception, because if you do the author of the code containing the '`with`' statement will never realize anything went wrong.
- If *BLOCK* didn't raise an exception, the `__exit__()` method is still called, but *type*, *value*, and *traceback* are all `None`.

Let's think through an example. I won't present detailed code but will only sketch the methods necessary for a database that supports transactions.

(For people unfamiliar with database terminology: a set of changes to the database are grouped into a transaction. Transactions can be either committed, meaning that all the changes are written into the database, or rolled back, meaning that the changes are all discarded and the database is unchanged. See any database textbook for more information.)

Vamos supor que exista um objeto representando uma conexão com o banco de dados. Nosso objetivo será permitir que o usuário escreva código como este:

```
db_connection = DatabaseConnection()
with db_connection as cursor:
    cursor.execute('insert into ...')
    cursor.execute('delete from ...')
    # ... more operations ...
```

The transaction should be committed if the code in the block runs flawlessly or rolled back if there's an exception. Here's the basic interface for `DatabaseConnection` that I'll assume:

```
class DatabaseConnection:
    # Database interface
    def cursor (self):
        "Returns a cursor object and starts a new transaction"
    def commit (self):
        "Commits current transaction"
    def rollback (self):
        "Rolls back current transaction"
```

The **`__enter__()`** method is pretty easy, having only to start a new transaction. For this application the resulting cursor object would be a useful result, so the method will return it. The user can then add `as cursor` to their '**`with`**' statement to bind the cursor to a variable name.

```
class DatabaseConnection:
    ...
    def __enter__ (self):
        # Code to start a new transaction
        cursor = self.cursor()
        return cursor
```

The **`__exit__()`** method is the most complicated because it's where most of the work has to be done. The method has to check if an exception occurred. If there was no exception, the transaction is committed. The transaction is rolled back if there was an exception.

In the code below, execution will just fall off the end of the function, returning the default value of `None`. `None` is false, so the exception will be re-raised automatically. If you wished, you could be more explicit and add a **`return`** statement at the marked location.

```
class DatabaseConnection:
    ...
    def __exit__ (self, type, value, tb):
        if tb is None:
            # No exception, so commit
```

```
            self.commit()
        else:
            # Exception occurred, so rollback.
            self.rollback()
            # return False
```

The contextlib module

The new **contextlib** module provides some functions and a decorator that are useful for writing objects for use with the '**with**' statement.

The decorator is called **@~contextlib.contextmanager**, and lets you write a single generator function instead of defining a new class. The generator should yield exactly one value. The code up to the **yield** will be executed as the **__enter__()** method, and the value yielded will be the method's return value that will get bound to the variable in the '**with**' statement's **as** clause, if any. The code after the **yield** will be executed in the **__exit__()** method. Any exception raised in the block will be raised by the **yield** statement.

Our database example from the previous section could be written using this decorator as:

```
from contextlib import contextmanager

@contextmanager
def db_transaction (connection):
    cursor = connection.cursor()
    try:
        yield cursor
    except:
        connection.rollback()
        raise
    else:
        connection.commit()

db = DatabaseConnection()
with db_transaction(db) as cursor:
    ...
```

The **contextlib** module also has a `nested(mgr1, mgr2, ...)` function that combines a number of context managers so you don't need to write nested '**with**' statements. In this example, the single '**with**' statement both starts a database transaction and acquires a thread lock:

```
lock = threading.Lock()
with nested (db_transaction(db), lock) as (cursor, locked):
    ...
```

Finally, the `closing(object)` function returns *object* so that it can be bound to a variable, and calls `object.close` at the end of the block.

```
import urllib, sys
from contextlib import closing

with closing(urllib.urlopen('http://www.yahoo.com')) as f:
    for line in f:
        sys.stdout.write(line)
```

Ver também

PEP 343 **[https://peps.python.org/pep-0343/]** **- A instrução "with"**
PEP written by Guido van Rossum and Nick Coghlan; implemented by Mike Bland, Guido van Rossum, and Neal Norwitz. The PEP shows the code generated for a '**with**' statement, which can be helpful in learning how the statement works.

The documentation for the **contextlib** module.

PEP 352: Exceptions as New-Style Classes

Exception classes can now be new-style classes, not just classic classes, and the built-in **Exception** class and all the standard built-in exceptions (**NameError**, **ValueError**, etc.) are now new-style classes.

The inheritance hierarchy for exceptions has been rearranged a bit. In 2.5, the inheritance relationships are:

```
BaseException       # New in Python 2.5
|- KeyboardInterrupt
|- SystemExit
|- Exception
   |- (all other current built-in exceptions)
```

This rearrangement was done because people often want to catch all exceptions that indicate program errors. **`KeyboardInterrupt`** and **`SystemExit`** aren't errors, though, and usually represent an explicit action such as the user hitting `Control-C` or code calling **`sys.exit()`**. A bare `except:` will catch all exceptions, so you commonly need to list **`KeyboardInterrupt`** and **`SystemExit`** in order to re-raise them. The usual pattern is:

```
try:
    ...
except (KeyboardInterrupt, SystemExit):
    raise
except:
    # Log error...
    # Continue running program...
```

In Python 2.5, you can now write `except Exception` to achieve the same result, catching all the exceptions that usually indicate errors but leaving **`KeyboardInterrupt`** and **`SystemExit`** alone. As in previous versions, a bare `except:` still catches all exceptions.

The goal for Python 3.0 is to require any class raised as an exception to derive from **`BaseException`** or some descendant of **`BaseException`**, and future releases in the Python 2.x series may begin to enforce this constraint. Therefore, I suggest you begin making all your exception classes derive from **`Exception`** now. It's been suggested that the bare `except:` form should be removed in Python 3.0, but Guido van Rossum hasn't decided whether to do this or not.

Raising of strings as exceptions, as in the statement `raise "Error occurred"`, is deprecated in Python 2.5 and will trigger a warning. The aim is to be able to remove the string-exception feature in a few releases.

Ver também

PEP 352 [https://peps.python.org/pep-0352/] - Required Superclass for Exceptions

PEP written by Brett Cannon and Guido van Rossum; implemented by Brett Cannon.

PEP 353: Using ssize_t as the index type

A wide-ranging change to Python's C API, using a new `Py_ssize_t` type definition instead of int, will permit the interpreter to handle more data on 64-bit platforms. This change doesn't affect Python's capacity on 32-bit platforms.

Various pieces of the Python interpreter used C's int type to store sizes or counts; for example, the number of items in a list or tuple were stored in an int. The C compilers for most 64-bit platforms still define int as a 32-bit type, so that meant that lists could only hold up to `2**31 - 1` = 2147483647 items. (There are actually a few different programming models that 64-bit C compilers can use – see https://unix.org/version2/whatsnew/lp64_wp.html for a discussion – but the most commonly available model leaves int as 32 bits.)

A limit of 2147483647 items doesn't really matter on a 32-bit platform because you'll run out of memory before hitting the length limit. Each list item requires space for a pointer, which is 4 bytes, plus space for a `PyObject` representing the item. 2147483647*4 is already more bytes than a 32-bit address space can contain.

It's possible to address that much memory on a 64-bit platform, however. The pointers for a list that size would only require 16 GiB of space, so it's not unreasonable that Python programmers might construct lists that large. Therefore, the Python interpreter had to be changed to use some type other than int, and this will be a 64-bit type on 64-bit platforms. The change will cause incompatibilities on 64-bit machines, so it was deemed worth making the transition now, while the number of 64-bit users is still relatively small.

(In 5 or 10 years, we may *all* be on 64-bit machines, and the transition would be more painful then.)

This change most strongly affects authors of C extension modules. Python strings and container types such as lists and tuples now use `Py_ssize_t` to store their size. Functions such as `PyList_Size()` now return `Py_ssize_t`. Code in extension modules may therefore need to have some variables changed to `Py_ssize_t`.

The `PyArg_ParseTuple()` and `Py_BuildValue()` functions have a new conversion code, `n`, for `Py_ssize_t`. `PyArg_ParseTuple()`'s `s#` and `t#` still output int by default, but you can define the macro `PY_SSIZE_T_CLEAN` before including `Python.h` to make them return `Py_ssize_t`.

PEP 353 [https://peps.python.org/pep-0353/] has a section on conversion guidelines that extension authors should read to learn about supporting 64-bit platforms.

Ver também

PEP 353 [https://peps.python.org/pep-0353/] - Using ssize_t as the index type
PEP written and implemented by Martin von Löwis.

PEP 357: The '__index__' method

The NumPy developers had a problem that could only be solved by adding a new special method, `__index__()`. When using slice notation, as in `[start:stop:step]`, the values of the *start*, *stop*, and *step* indexes must all be either integers or long integers. NumPy defines a variety of specialized integer types corresponding to unsigned and signed integers of 8, 16, 32, and 64 bits, but there was no way to signal that these types could be used as slice indexes.

Slicing can't just use the existing `__int__()` method because that method is also used to implement coercion to integers. If slicing used `__int__()`,

floating-point numbers would also become legal slice indexes and that's clearly an undesirable behaviour.

Instead, a new special method called `__index__()` was added. It takes no arguments and returns an integer giving the slice index to use. For example:

```
class C:
    def __index__ (self):
        return self.value
```

The return value must be either a Python integer or long integer. The interpreter will check that the type returned is correct, and raises a `TypeError` if this requirement isn't met.

A corresponding `nb_index` slot was added to the C-level `PyNumberMethods` structure to let C extensions implement this protocol. `PyNumber_Index(obj)` can be used in extension code to call the `__index__()` function and retrieve its result.

Ver também

PEP 357 [https://peps.python.org/pep-0357/] - Allowing Any Object to be Used for Slicing
PEP written and implemented by Travis Oliphant.

Outras mudanças na linguagem

Here are all of the changes that Python 2.5 makes to the core Python language.

- The `dict` type has a new hook for letting subclasses provide a default value when a key isn't contained in the dictionary. When a key isn't found, the dictionary's `__missing__(key)` method will be called. This hook is used to implement the new `defaultdict` class in the `collections` module. The following example defines a dictionary that returns zero for any missing key:

```
class zerodict (dict):
    def __missing__ (self, key):
        return 0

d = zerodict({1:1, 2:2})
print d[1], d[2]   # Prints 1, 2
print d[3], d[4]   # Prints 0, 0
```

- Both 8-bit and Unicode strings have new `partition(sep)` and `rpartition(sep)` methods that simplify a common use case.

  The `find(S)` method is often used to get an index which is then used to slice the string and obtain the pieces that are before and after the separator. `partition(sep)` condenses this pattern into a single method call that returns a 3-tuple containing the substring before the separator, the separator itself, and the substring after the separator. If the separator isn't found, the first element of the tuple is the entire string and the other two elements are empty. `rpartition(sep)` also returns a 3-tuple but starts searching from the end of the string; the `r` stands for 'reverse'.

  Alguns exemplos:

  ```
  >>> ('http://www.python.org').partition('://')
  ('http', '://', 'www.python.org')
  >>> ('file:/usr/share/doc/index.html').partition('://')
  ('file:/usr/share/doc/index.html', '', '')
  >>> (u'Subject: a quick question').partition(':')
  (u'Subject', u':', u' a quick question')
  >>> 'www.python.org'.rpartition('.')
  ('www.python', '.', 'org')
  >>> 'www.python.org'.rpartition(':')
  ('', '', 'www.python.org')
  ```

  (Implemented by Fredrik Lundh following a suggestion by Raymond Hettinger.)

- The **`startswith()`** and **`endswith()`** methods of string types now accept tuples of strings to check for.

```
def is_image_file (filename):
    return filename.endswith(('.gif', '.jpg', '.tiff'))
```

(Implemented by Georg Brandl following a suggestion by Tom Lynn.)

- The `min()` and `max()` built-in functions gained a `key` keyword parameter analogous to the `key` argument for `sort()`. This parameter supplies a function that takes a single argument and is called for every value in the list; **`min()`**/**`max()`** will return the element with the smallest/largest return value from this function. For example, to find the longest string in a list, you can do:

```
L = ['medium', 'longest', 'short']
# Prints 'longest'
print max(L, key=len)
# Prints 'short', because lexicographically 'short' has the
print max(L)
```

(Contributed by Steven Bethard and Raymond Hettinger.)

- Two new built-in functions, `any()` and `all()`, evaluate whether an iterator contains any true or false values. **`any()`** returns `True` if any value returned by the iterator is true; otherwise it will return `False`. **`all()`** returns **`True`** only if all of the values returned by the iterator evaluate as true. (Suggested by Guido van Rossum, and implemented by Raymond Hettinger.)

- The result of a class's `__hash__()` method can now be either a long integer or a regular integer. If a long integer is returned, the hash of that value is taken. In earlier versions the hash value was required to be a regular integer, but in 2.5 the `id()` built-in was changed to always return non-negative numbers, and users often seem to use `id(self)` in **`__hash__()`** methods (though this is discouraged).

- ASCII is now the default encoding for modules. It's now a syntax error if a module contains string literals with 8-bit characters but doesn't have an encoding declaration. In Python 2.4 this triggered a warning, not a syntax error. See **PEP 263** [https://peps.python.org/pep-0263/] for how to

declare a module's encoding; for example, you might add a line like this near the top of the source file:

```
# -*- coding: latin1 -*-
```

- A new warning, **UnicodeWarning**, is triggered when you attempt to compare a Unicode string and an 8-bit string that can't be converted to Unicode using the default ASCII encoding. The result of the comparison is false:

```
>>> chr(128) == unichr(128)   # Can't convert chr(128) to Ur
__main__:1: UnicodeWarning: Unicode equal comparison failed
  to convert both arguments to Unicode - interpreting them
  as being unequal
False
>>> chr(127) == unichr(127)   # chr(127) can be converted
True
```

  Previously this would raise a **UnicodeDecodeError** exception, but in 2.5 this could result in puzzling problems when accessing a dictionary. If you looked up `unichr(128)` and `chr(128)` was being used as a key, you'd get a **`UnicodeDecodeError`** exception. Other changes in 2.5 resulted in this exception being raised instead of suppressed by the code in `dictobject.c` that implements dictionaries.

  Raising an exception for such a comparison is strictly correct, but the change might have broken code, so instead **UnicodeWarning** was introduced.

  (Implemented by Marc-André Lemburg.)

- One error that Python programmers sometimes make is forgetting to include an `__init__.py` module in a package directory. Debugging this mistake can be confusing, and usually requires running Python with the **-v** switch to log all the paths searched. In Python 2.5, a new **ImportWarning** warning is triggered when an import would have picked up a directory as a package but no `__init__.py` was found. This warning is silently ignored by default; provide the **-Wd** option

when running the Python executable to display the warning message. (Implemented by Thomas Wouters.)

- The list of base classes in a class definition can now be empty. As an example, this is now legal:

  ```
  class C():
      pass
  ```

  (Implemented by Brett Cannon.)

Interactive Interpreter Changes

In the interactive interpreter, `quit` and `exit` have long been strings so that new users get a somewhat helpful message when they try to quit:

```
>>> quit
'Use Ctrl-D (i.e. EOF) to exit.'
```

In Python 2.5, `quit` and `exit` are now objects that still produce string representations of themselves, but are also callable. Newbies who try `quit()` or `exit()` will now exit the interpreter as they expect. (Implemented by Georg Brandl.)

The Python executable now accepts the standard long options **`--help`** and **`--version`**; on Windows, it also accepts the **`/?`** option for displaying a help message. (Implemented by Georg Brandl.)

Otimizações

Several of the optimizations were developed at the NeedForSpeed sprint, an event held in Reykjavik, Iceland, from May 21–28 2006. The sprint focused on speed enhancements to the CPython implementation and was funded by EWT LLC with local support from CCP Games. Those optimizations added at this sprint are specially marked in the following list.

- When they were introduced in Python 2.4, the built-in **`set`** and **`frozenset`** types were built on top of Python's dictionary type. In 2.5

the internal data structure has been customized for implementing sets, and as a result sets will use a third less memory and are somewhat faster. (Implemented by Raymond Hettinger.)

- The speed of some Unicode operations, such as finding substrings, string splitting, and character map encoding and decoding, has been improved. (Substring search and splitting improvements were added by Fredrik Lundh and Andrew Dalke at the NeedForSpeed sprint. Character maps were improved by Walter Dörwald and Martin von Löwis.)
- The `long(str, base)` function is now faster on long digit strings because fewer intermediate results are calculated. The peak is for strings of around 800–1000 digits where the function is 6 times faster. (Contributed by Alan McIntyre and committed at the NeedForSpeed sprint.)
- It's now illegal to mix iterating over a file with `for line in file` and calling the file object's **`read()`**/**`readline()`**/**`readlines()`** methods. Iteration uses an internal buffer and the **`read*()`** methods don't use that buffer. Instead they would return the data following the buffer, causing the data to appear out of order. Mixing iteration and these methods will now trigger a **`ValueError`** from the **`read*()`** method. (Implemented by Thomas Wouters.)
- The **`struct`** module now compiles structure format strings into an internal representation and caches this representation, yielding a 20% speedup. (Contributed by Bob Ippolito at the NeedForSpeed sprint.)
- The **`re`** module got a 1 or 2% speedup by switching to Python's allocator functions instead of the system's **`malloc()`** and **`free()`**. (Contributed by Jack Diederich at the NeedForSpeed sprint.)
- The code generator's peephole optimizer now performs simple constant folding in expressions. If you write something like `a = 2+3`, the code generator will do the arithmetic and produce code corresponding to `a = 5`. (Proposed and implemented by Raymond Hettinger.)
- Function calls are now faster because code objects now keep the most recently finished frame (a "zombie frame") in an internal field of the code object, reusing it the next time the code object is invoked. (Original patch by Michael Hudson, modified by Armin Rigo and Richard Jones; committed at the NeedForSpeed sprint.) Frame objects

are also slightly smaller, which may improve cache locality and reduce memory usage a bit. (Contributed by Neal Norwitz.)
- Python's built-in exceptions are now new-style classes, a change that speeds up instantiation considerably. Exception handling in Python 2.5 is therefore about 30% faster than in 2.4. (Contributed by Richard Jones, Georg Brandl and Sean Reifschneider at the NeedForSpeed sprint.)
- Importing now caches the paths tried, recording whether they exist or not so that the interpreter makes fewer **`open()`** and **`stat()`** calls on startup. (Contributed by Martin von Löwis and Georg Brandl.)

New, Improved, and Removed Modules

The standard library received many enhancements and bug fixes in Python 2.5. Here's a partial list of the most notable changes, sorted alphabetically by module name. Consult the `Misc/NEWS` file in the source tree for a more complete list of changes, or look through the SVN logs for all the details.

- The **`audioop`** module now supports the a-LAW encoding, and the code for u-LAW encoding has been improved. (Contributed by Lars Immisch.)

- The **`codecs`** module gained support for incremental codecs. The **`codecs.lookup()`** function now returns a **`CodecInfo`** instance instead of a tuple. **`CodecInfo`** instances behave like a 4-tuple to preserve backward compatibility but also have the attributes **`encode`**, **`decode`**, **`incrementalencoder`**, **`incrementaldecoder`**, **`streamwriter`**, and **`streamreader`**. Incremental codecs can receive input and produce output in multiple chunks; the output is the same as if the entire input was fed to the non-incremental codec. See the **`codecs`** module documentation for details. (Designed and implemented by Walter Dörwald.)

- The **`collections`** module gained a new type, **`defaultdict`**, that subclasses the standard **`dict`** type. The new type mostly behaves like a dictionary but constructs a default value when a key isn't present, automatically adding it to the dictionary for the requested key value.

The first argument to **defaultdict**'s constructor is a factory function that gets called whenever a key is requested but not found. This factory function receives no arguments, so you can use built-in type constructors such as **list()** or **int()**. For example, you can make an index of words based on their initial letter like this:

```
words = """Nel mezzo del cammin di nostra vita
mi ritrovai per una selva oscura
che la diritta via era smarrita""".lower().split()

index = defaultdict(list)

for w in words:
    init_letter = w[0]
    index[init_letter].append(w)
```

Printing `index` results in the following output:

```
defaultdict(<type 'list'>, {'c': ['cammin', 'che'], 'e': ['e
        'd': ['del', 'di', 'diritta'], 'm': ['mezzo', 'mi'],
        'l': ['la'], 'o': ['oscura'], 'n': ['nel', 'nostra']
        'p': ['per'], 's': ['selva', 'smarrita'],
        'r': ['ritrovai'], 'u': ['una'], 'v': ['vita', 'via
```

(Contributed by Guido van Rossum.)

- The **deque** double-ended queue type supplied by the **collections** module now has a `remove(value)` method that removes the first occurrence of *value* in the queue, raising **ValueError** if the value isn't found. (Contributed by Raymond Hettinger.)

- New module: The **contextlib** module contains helper functions for use with the new '**with**' statement. See section The contextlib module for more about this module.

- New module: The **cProfile** module is a C implementation of the existing **profile** module that has much lower overhead. The module's interface is the same as **profile**: you run `cProfile.run('main()')` to profile a function, can save profile data to a file, etc. It's not yet known

if the Hotshot profiler, which is also written in C but doesn't match the **`profile`** module's interface, will continue to be maintained in future versions of Python. (Contributed by Armin Rigo.)

Also, the **`pstats`** module for analyzing the data measured by the profiler now supports directing the output to any file object by supplying a *stream* argument to the **`Stats`** constructor. (Contributed by Skip Montanaro.)

- The **`csv`** module, which parses files in comma-separated value format, received several enhancements and a number of bugfixes. You can now set the maximum size in bytes of a field by calling the `csv.field_size_limit(new_limit)` function; omitting the *new_limit* argument will return the currently set limit. The **`reader`** class now has a **`line_num`** attribute that counts the number of physical lines read from the source; records can span multiple physical lines, so **`line_num`** is not the same as the number of records read.

  The CSV parser is now stricter about multi-line quoted fields. Previously, if a line ended within a quoted field without a terminating newline character, a newline would be inserted into the returned field. This behavior caused problems when reading files that contained carriage return characters within fields, so the code was changed to return the field without inserting newlines. As a consequence, if newlines embedded within fields are important, the input should be split into lines in a manner that preserves the newline characters.

  (Contributed by Skip Montanaro and Andrew McNamara.)

- The **`datetime`** class in the **`datetime`** module now has a `strptime(string, format)` method for parsing date strings, contributed by Josh Spoerri. It uses the same format characters as **`time.strptime()`** and **`time.strftime()`**:

```
import datetime as dt

ts = dt.datetime.strptime('10:13:15 2006-03-07',
                          '%H:%M:%S %Y-%m-%d')
```

- The `difflib.SequenceMatcher.get_matching_blocks()` method in the `difflib` module now guarantees to return a minimal list of blocks describing matching subsequences. Previously, the algorithm would occasionally break a block of matching elements into two list entries. (Enhancement by Tim Peters.)

- The `doctest` module gained a `SKIP` option that keeps an example from being executed at all. This is intended for code snippets that are usage examples intended for the reader and aren't actually test cases.

  An *encoding* parameter was added to the `testfile()` function and the `DocFileSuite` class to specify the file's encoding. This makes it easier to use non-ASCII characters in tests contained within a docstring. (Contributed by Bjorn Tillenius.)

- The `email` package has been updated to version 4.0. (Contributed by Barry Warsaw.)

- The `fileinput` module was made more flexible. Unicode filenames are now supported, and a *mode* parameter that defaults to `"r"` was added to the `input()` function to allow opening files in binary or universal newlines mode. Another new parameter, *openhook*, lets you use a function other than `open()` to open the input files. Once you're iterating over the set of files, the `FileInput` object's new `fileno()` returns the file descriptor for the currently opened file. (Contributed by Georg Brandl.)

- In the `gc` module, the new `get_count()` function returns a 3-tuple containing the current collection counts for the three GC generations. This is accounting information for the garbage collector; when these counts reach a specified threshold, a garbage collection sweep will be made. The existing `gc.collect()` function now takes an optional *generation* argument of 0, 1, or 2 to specify which generation to collect. (Contributed by Barry Warsaw.)

- The `nsmallest()` and `nlargest()` functions in the `heapq` module now support a `key` keyword parameter similar to the one provided by the `min()`/`max()` functions and the `sort()` methods. For example:

```
>>> import heapq
>>> L = ["short", 'medium', 'longest', 'longer still']
>>> heapq.nsmallest(2, L)  # Return two lowest elements, le
['longer still', 'longest']
>>> heapq.nsmallest(2, L, key=len)   # Return two shortest e
['short', 'medium']
```

(Contributed by Raymond Hettinger.)

- The `itertools.islice()` function now accepts `None` for the start and step arguments. This makes it more compatible with the attributes of slice objects, so that you can now write the following:

  ```
  s = slice(5)     # Create slice object
  itertools.islice(iterable, s.start, s.stop, s.step)
  ```

  (Contributed by Raymond Hettinger.)

- The `format()` function in the `locale` module has been modified and two new functions were added, `format_string()` and `currency()`.

  The `format()` function's *val* parameter could previously be a string as long as no more than one %char specifier appeared; now the parameter must be exactly one %char specifier with no surrounding text. An optional *monetary* parameter was also added which, if `True`, will use the locale's rules for formatting currency in placing a separator between groups of three digits.

  To format strings with multiple %char specifiers, use the new `format_string()` function that works like `format()` but also supports mixing %char specifiers with arbitrary text.

  A new `currency()` function was also added that formats a number according to the current locale's settings.

  (Contributed by Georg Brandl.)

- The `mailbox` module underwent a massive rewrite to add the capability to modify mailboxes in addition to reading them. A new set of classes

that include **mbox**, **MH**, and **Maildir** are used to read mailboxes, and have an `add(message)` method to add messages, `remove(key)` to remove messages, and **lock()**/**unlock()** to lock/unlock the mailbox. The following example converts a maildir-format mailbox into an mbox-format one:

```
import mailbox

# 'factory=None' uses email.Message.Message as the class rep
# individual messages.
src = mailbox.Maildir('maildir', factory=None)
dest = mailbox.mbox('/tmp/mbox')

for msg in src:
    dest.add(msg)
```

(Contributed by Gregory K. Johnson. Funding was provided by Google's 2005 Summer of Code.)

- New module: the **msilib** module allows creating Microsoft Installer `.msi` files and CAB files. Some support for reading the `.msi` database is also included. (Contributed by Martin von Löwis.)

- The **nis** module now supports accessing domains other than the system default domain by supplying a *domain* argument to the **nis.match()** and **nis.maps()** functions. (Contributed by Ben Bell.)

- The **operator** module's **itemgetter()** and **attrgetter()** functions now support multiple fields. A call such as `operator.attrgetter('a', 'b')` will return a function that retrieves the `a` and `b` attributes. Combining this new feature with the **sort()** method's `key` parameter lets you easily sort lists using multiple fields. (Contributed by Raymond Hettinger.)

- The **optparse** module was updated to version 1.5.1 of the Optik library. The **OptionParser** class gained an **epilog** attribute, a string that will be printed after the help message, and a **destroy()** method to

break reference cycles created by the object. (Contributed by Greg Ward.)

- The os module underwent several changes. The **stat_float_times** variable now defaults to true, meaning that os.stat() will now return time values as floats. (This doesn't necessarily mean that **os.stat()** will return times that are precise to fractions of a second; not all systems support such precision.)

  Constants named os.SEEK_SET, os.SEEK_CUR, and os.SEEK_END have been added; these are the parameters to the os.lseek() function. Two new constants for locking are os.O_SHLOCK and os.O_EXLOCK.

  Two new functions, wait3() and wait4(), were added. They're similar the waitpid() function which waits for a child process to exit and returns a tuple of the process ID and its exit status, but **wait3()** and **wait4()** return additional information. **wait3()** doesn't take a process ID as input, so it waits for any child process to exit and returns a 3-tuple of *process-id*, *exit-status*, *resource-usage* as returned from the resource.getrusage() function. `wait4(pid)` does take a process ID. (Contributed by Chad J. Schroeder.)

  On FreeBSD, the os.stat() function now returns times with nanosecond resolution, and the returned object now has st_gen and st_birthtime. The st_flags attribute is also available, if the platform supports it. (Contributed by Antti Louko and Diego Pettenò.)

- The Python debugger provided by the pdb module can now store lists of commands to execute when a breakpoint is reached and execution stops. Once breakpoint #1 has been created, enter `commands 1` and enter a series of commands to be executed, finishing the list with `end`. The command list can include commands that resume execution, such as `continue` or `next`. (Contributed by Grégoire Dooms.)

- The pickle and **cPickle** modules no longer accept a return value of `None` from the __reduce__() method; the method must return a tuple of arguments instead. The ability to return `None` was deprecated in Python 2.4, so this completes the removal of the feature.

- The **pkgutil** module, containing various utility functions for finding packages, was enhanced to support **PEP 302** [https://peps.python.org/pep-0302/]'s import hooks and now also works for packages stored in ZIP-format archives. (Contributed by Phillip J. Eby.)

- The pybench benchmark suite by Marc-André Lemburg is now included in the `Tools/pybench` directory. The pybench suite is an improvement on the commonly used `pystone.py` program because pybench provides a more detailed measurement of the interpreter's speed. It times particular operations such as function calls, tuple slicing, method lookups, and numeric operations, instead of performing many different operations and reducing the result to a single number as `pystone.py` does.

- The **pyexpat** module now uses version 2.0 of the Expat parser. (Contributed by Trent Mick.)

- The **Queue** class provided by the **queue** module gained two new methods. **join()** blocks until all items in the queue have been retrieved and all processing work on the items have been completed. Worker threads call the other new method, **task_done()**, to signal that processing for an item has been completed. (Contributed by Raymond Hettinger.)

- The old **regex** and **regsub** modules, which have been deprecated ever since Python 2.0, have finally been deleted. Other deleted modules: **statcache**, **tzparse**, **whrandom**.

- Also deleted: the `lib-old` directory, which includes ancient modules such as **dircmp** and **ni**, was removed. `lib-old` wasn't on the default `sys.path`, so unless your programs explicitly added the directory to `sys.path`, this removal shouldn't affect your code.

- The **rlcompleter** module is no longer dependent on importing the **readline** module and therefore now works on non-Unix platforms. (Patch from Robert Kiendl.)

- The **SimpleXMLRPCServer** and **DocXMLRPCServer** classes now have a **rpc_paths** attribute that constrains XML-RPC operations to a limited set of URL paths; the default is to allow only `'/'` and `'/RPC2'`. Setting **rpc_paths** to `None` or an empty tuple disables this path checking.

- The **socket** module now supports **AF_NETLINK** sockets on Linux, thanks to a patch from Philippe Biondi. Netlink sockets are a Linux-specific mechanism for communications between a user-space process and kernel code; an introductory article about them is at https://www.linuxjournal.com/article/7356. In Python code, netlink addresses are represented as a tuple of 2 integers, `(pid, group_mask)`.

  Two new methods on socket objects, `recv_into(buffer)` and `recvfrom_into(buffer)`, store the received data in an object that supports the buffer protocol instead of returning the data as a string. This means you can put the data directly into an array or a memory-mapped file.

  Socket objects also gained **getfamily()**, **gettype()**, and **getproto()** accessor methods to retrieve the family, type, and protocol values for the socket.

- New module: the **spwd** module provides functions for accessing the shadow password database on systems that support shadow passwords.

- The **struct** is now faster because it compiles format strings into **Struct** objects with **pack()** and **unpack()** methods. This is similar to how the **re** module lets you create compiled regular expression objects. You can still use the module-level **pack()** and **unpack()** functions; they'll create **Struct** objects and cache them. Or you can use **Struct** instances directly:

  ```
  s = struct.Struct('ih3s')

  data = s.pack(1972, 187, 'abc')
  year, number, name = s.unpack(data)
  ```

  You can also pack and unpack data to and from buffer objects directly using the `pack_into(buffer, offset, v1, v2, ...)` and

`unpack_from(buffer, offset)` methods. This lets you store data directly into an array or a memory-mapped file.

(**`Struct`** objects were implemented by Bob Ippolito at the NeedForSpeed sprint. Support for buffer objects was added by Martin Blais, also at the NeedForSpeed sprint.)

- The Python developers switched from CVS to Subversion during the 2.5 development process. Information about the exact build version is available as the `sys.subversion` variable, a 3-tuple of `(interpreter-name, branch-name, revision-range)`. For example, at the time of writing my copy of 2.5 was reporting `('CPython', 'trunk', '45313:45315')`.

  This information is also available to C extensions via the **`Py_GetBuildInfo()`** function that returns a string of build information like this: `"trunk:45355:45356M, Apr 13 2006, 07:42:19"`. (Contributed by Barry Warsaw.)

- Another new function, **`sys._current_frames()`**, returns the current stack frames for all running threads as a dictionary mapping thread identifiers to the topmost stack frame currently active in that thread at the time the function is called. (Contributed by Tim Peters.)

- The **`TarFile`** class in the **`tarfile`** module now has an **`extractall()`** method that extracts all members from the archive into the current working directory. It's also possible to set a different directory as the extraction target, and to unpack only a subset of the archive's members.

  The compression used for a tarfile opened in stream mode can now be autodetected using the mode `'r|*'`. (Contributed by Lars Gustäbel.)

- The **`threading`** module now lets you set the stack size used when new threads are created. The `stack_size([*size*])` function returns the currently configured stack size, and supplying the optional *size* parameter sets a new value. Not all platforms support changing the

stack size, but Windows, POSIX threading, and OS/2 all do. (Contributed by Andrew MacIntyre.)

- The `unicodedata` module has been updated to use version 4.1.0 of the Unicode character database. Version 3.2.0 is required by some specifications, so it's still available as `unicodedata.ucd_3_2_0`.

- New module: the `uuid` module generates universally unique identifiers (UUIDs) according to **RFC 4122** [https://datatracker.ietf.org/doc/html/rfc4122.html]. The RFC defines several different UUID versions that are generated from a starting string, from system properties, or purely randomly. This module contains a `UUID` class and functions named `uuid1()`, `uuid3()`, `uuid4()`, and `uuid5()` to generate different versions of UUID. (Version 2 UUIDs are not specified in **RFC 4122** [https://datatracker.ietf.org/doc/html/rfc4122.html] and are not supported by this module.)

```
>>> import uuid
>>> # make a UUID based on the host ID and current time
>>> uuid.uuid1()
UUID('a8098c1a-f86e-11da-bd1a-00112444be1e')

>>> # make a UUID using an MD5 hash of a namespace UUID and
>>> uuid.uuid3(uuid.NAMESPACE_DNS, 'python.org')
UUID('6fa459ea-ee8a-3ca4-894e-db77e160355e')

>>> # make a random UUID
>>> uuid.uuid4()
UUID('16fd2706-8baf-433b-82eb-8c7fada847da')

>>> # make a UUID using a SHA-1 hash of a namespace UUID anc
>>> uuid.uuid5(uuid.NAMESPACE_DNS, 'python.org')
UUID('886313e1-3b8a-5372-9b90-0c9aee199e5d')
```

(Contributed by Ka-Ping Yee.)

- The `weakref` module's `WeakKeyDictionary` and `WeakValueDictionary` types gained new methods for iterating over the weak references contained in the dictionary. `iterkeyrefs()` and

keyrefs() methods were added to **WeakKeyDictionary**, and **itervaluerefs()** and valuerefs() were added to **WeakValueDictionary**. (Contributed by Fred L. Drake, Jr.)

- The webbrowser module received a number of enhancements. It's now usable as a script with `python -m webbrowser`, taking a URL as the argument; there are a number of switches to control the behaviour (**-n** for a new browser window, **-t** for a new tab). New module-level functions, open_new() and open_new_tab(), were added to support this. The module's open() function supports an additional feature, an *autoraise* parameter that signals whether to raise the open window when possible. A number of additional browsers were added to the supported list such as Firefox, Opera, Konqueror, and elinks. (Contributed by Oleg Broytmann and Georg Brandl.)

- The xmlrpclib module now supports returning datetime objects for the XML-RPC date type. Supply `use_datetime=True` to the loads() function or the **Unmarshaller** class to enable this feature. (Contributed by Skip Montanaro.)

- The zipfile module now supports the ZIP64 version of the format, meaning that a .zip archive can now be larger than 4 GiB and can contain individual files larger than 4 GiB. (Contributed by Ronald Oussoren.)

- The zlib module's **Compress** and **Decompress** objects now support a copy() method that makes a copy of the object's internal state and returns a new **Compress** or **Decompress** object. (Contributed by Chris AtLee.)

The ctypes package

The ctypes package, written by Thomas Heller, has been added to the standard library. **ctypes** lets you call arbitrary functions in shared libraries or DLLs. Long-time users may remember the **dl** module, which provides functions for loading shared libraries and calling functions in them. The **ctypes** package is much fancier.

To load a shared library or DLL, you must create an instance of the **CDLL** class and provide the name or path of the shared library or DLL. Once that's done, you can call arbitrary functions by accessing them as attributes of the **CDLL** object.

```
import ctypes

libc = ctypes.CDLL('libc.so.6')
result = libc.printf("Line of output\n")
```

Type constructors for the various C types are provided: **c_int()**, **c_float()**, **c_double()**, **c_char_p()** (equivalent to char*), and so forth. Unlike Python's types, the C versions are all mutable; you can assign to their **value** attribute to change the wrapped value. Python integers and strings will be automatically converted to the corresponding C types, but for other types you must call the correct type constructor. (And I mean *must*; getting it wrong will often result in the interpreter crashing with a segmentation fault.)

You shouldn't use **c_char_p()** with a Python string when the C function will be modifying the memory area, because Python strings are supposed to be immutable; breaking this rule will cause puzzling bugs. When you need a modifiable memory area, use **create_string_buffer()**:

```
s = "this is a string"
buf = ctypes.create_string_buffer(s)
libc.strfry(buf)
```

C functions are assumed to return integers, but you can set the **restype** attribute of the function object to change this:

```
>>> libc.atof('2.71828')
-1783957616
>>> libc.atof.restype = ctypes.c_double
>>> libc.atof('2.71828')
2.71828
```

ctypes also provides a wrapper for Python's C API as the `ctypes.pythonapi` object. This object does *not* release the global interpreter lock before calling a function, because the lock must be held

when calling into the interpreter's code. There's a **py_object** type constructor that will create a PyObject* pointer. A simple usage:

```
import ctypes

d = {}
ctypes.pythonapi.PyObject_SetItem(ctypes.py_object(d),
          ctypes.py_object("abc"),  ctypes.py_object(1))
# d is now {'abc', 1}.
```

Don't forget to use **py_object()**; if it's omitted you end up with a segmentation fault.

ctypes has been around for a while, but people still write and distribution hand-coded extension modules because you can't rely on **ctypes** being present. Perhaps developers will begin to write Python wrappers atop a library accessed through **ctypes** instead of extension modules, now that **ctypes** is included with core Python.

Ver também

https://web.archive.org/web/20180410025338/http://starship.python.net/crew/theller/ctypes/
The pre-stdlib ctypes web page, with a tutorial, reference, and FAQ.

The documentation for the **ctypes** module.

The ElementTree package

A subset of Fredrik Lundh's ElementTree library for processing XML has been added to the standard library as **xml.etree**. The available modules are **ElementTree**, **ElementPath**, and **ElementInclude** from ElementTree 1.2.6. The **cElementTree** accelerator module is also included.

The rest of this section will provide a brief overview of using ElementTree. Full documentation for ElementTree is available at

https://web.archive.org/web/20201124024954/http://effbot.org/zone/element-index.htm.

ElementTree represents an XML document as a tree of element nodes. The text content of the document is stored as the **`text`** and **`tail`** attributes of (This is one of the major differences between ElementTree and the Document Object Model; in the DOM there are many different types of node, including **`TextNode`**.)

The most commonly used parsing function is **`parse()`**, that takes either a string (assumed to contain a filename) or a file-like object and returns an **`ElementTree`** instance:

```
from xml.etree import ElementTree as ET

tree = ET.parse('ex-1.xml')

feed = urllib.urlopen(
           'http://planet.python.org/rss10.xml')
tree = ET.parse(feed)
```

Once you have an **`ElementTree`** instance, you can call its **`getroot()`** method to get the root **`Element`** node.

There's also an **`XML()`** function that takes a string literal and returns an **`Element`** node (not an **`ElementTree`**). This function provides a tidy way to incorporate XML fragments, approaching the convenience of an XML literal:

```
svg = ET.XML("""<svg width="10px" version="1.0">
             </svg>""")
svg.set('height', '320px')
svg.append(elem1)
```

Each XML element supports some dictionary-like and some list-like access methods. Dictionary-like operations are used to access attribute values, and list-like operations are used to access child nodes.

Operação	Resultado
`elem[n]`	Returns n’th child element.
`elem[m:n]`	Returns list of m’th through n’th child elements.
`len(elem)`	Returns number of child elements.
`list(elem)`	Returns list of child elements.
`elem.append(elem2)`	Adds *elem2* as a child.
`elem.insert(index, elem2)`	Inserts *elem2* at the specified location.
`del elem[n]`	Deletes n’th child element.
`elem.keys()`	Returns list of attribute names.
`elem.get(name)`	Returns value of attribute *name*.
`elem.set(name, value)`	Sets new value for attribute *name*.
`elem.attrib`	Retrieves the dictionary containing attributes.

Operação	Resultado
`del elem.attrib[name]`	Deletes attribute *name*.

Comments and processing instructions are also represented as `Element` nodes. To check if a node is a comment or processing instructions:

```
if elem.tag is ET.Comment:
    ...
elif elem.tag is ET.ProcessingInstruction:
    ...
```

To generate XML output, you should call the `xml.etree.ElementTree.ElementTree.write()` method. Like `parse()`, it can take either a string or a file-like object:

```
# Encoding is US-ASCII
tree.write('output.xml')

# Encoding is UTF-8
f = open('output.xml', 'w')
tree.write(f, encoding='utf-8')
```

(Caution: the default encoding used for output is ASCII. For general XML work, where an element's name may contain arbitrary Unicode characters, ASCII isn't a very useful encoding because it will raise an exception if an element's name contains any characters with values greater than 127. Therefore, it's best to specify a different encoding such as UTF-8 that can handle any Unicode character.)

This section is only a partial description of the ElementTree interfaces. Please read the package's official documentation for more details.

Ver também

https://web.archive.org/web/20201124024954/http://effbot.org/zone/element-index.htm
Official documentation for ElementTree.

The hashlib package

A new **hashlib** module, written by Gregory P. Smith, has been added to replace the **md5** and **sha** modules. **hashlib** adds support for additional secure hashes (SHA-224, SHA-256, SHA-384, and SHA-512). When available, the module uses OpenSSL for fast platform optimized implementations of algorithms.

The old **md5** and **sha** modules still exist as wrappers around hashlib to preserve backwards compatibility. The new module's interface is very close to that of the old modules, but not identical. The most significant difference is that the constructor functions for creating new hashing objects are named differently.

```
# Old versions
h = md5.md5()
h = md5.new()

# New version
h = hashlib.md5()

# Old versions
h = sha.sha()
h = sha.new()

# New version
h = hashlib.sha1()

# Hash that weren't previously available
h = hashlib.sha224()
h = hashlib.sha256()
h = hashlib.sha384()
h = hashlib.sha512()
```

```
# Alternative form
h = hashlib.new('md5')          # Provide algorithm as a string
```

Once a hash object has been created, its methods are the same as before: `update(string)` hashes the specified string into the current digest state, **`digest()`** and **`hexdigest()`** return the digest value as a binary string or a string of hex digits, and **`copy()`** returns a new hashing object with the same digest state.

Ver também

The documentation for the **`hashlib`** module.

The sqlite3 package

The pysqlite module (https://www.pysqlite.org), a wrapper for the SQLite embedded database, has been added to the standard library under the package name **`sqlite3`**.

SQLite é uma biblioteca C que fornece um banco de dados leve baseado em disco que não requer um processo de servidor separado e permite acessar o banco de dados usando uma variante não padrão da linguagem de consulta SQL. Algumas aplicações podem usar SQLite para armazenamento interno de dados. Também é possível prototipar um aplicativo usando SQLite e, em seguida, portar o código para um banco de dados maior, como PostgreSQL ou Oracle.

pysqlite was written by Gerhard Häring and provides a SQL interface compliant with the DB-API 2.0 specification described by **PEP 249** [https://peps.python.org/pep-0249/].

If you're compiling the Python source yourself, note that the source tree doesn't include the SQLite code, only the wrapper module. You'll need to have the SQLite libraries and headers installed before compiling Python, and the build process will compile the module when the necessary headers are available.

To use the module, you must first create a **Connection** object that represents the database. Here the data will be stored in the `/tmp/example` file:

```
conn = sqlite3.connect('/tmp/example')
```

You can also supply the special name `:memory:` to create a database in RAM.

Once you have a **Connection**, you can create a **Cursor** object and call its **execute()** method to perform SQL commands:

```
c = conn.cursor()

# Create table
c.execute('''create table stocks
(date text, trans text, symbol text,
 qty real, price real)''')

# Insert a row of data
c.execute("""insert into stocks
          values ('2006-01-05','BUY','RHAT',100,35.14)""")
```

Usually your SQL operations will need to use values from Python variables. You shouldn't assemble your query using Python's string operations because doing so is insecure; it makes your program vulnerable to an SQL injection attack.

Instead, use the DB-API's parameter substitution. Put `?` as a placeholder wherever you want to use a value, and then provide a tuple of values as the second argument to the cursor's **execute()** method. (Other database modules may use a different placeholder, such as `%s` or `:1`.) For example:

```
# Never do this -- insecure!
symbol = 'IBM'
c.execute("... where symbol = '%s'" % symbol)

# Do this instead
t = (symbol,)
c.execute('select * from stocks where symbol=?', t)

# Larger example
```

```
for t in (('2006-03-28', 'BUY', 'IBM', 1000, 45.00),
          ('2006-04-05', 'BUY', 'MSOFT', 1000, 72.00),
          ('2006-04-06', 'SELL', 'IBM', 500, 53.00),
         ):
    c.execute('insert into stocks values (?,?,?,?,?)', t)
```

To retrieve data after executing a SELECT statement, you can either treat the cursor as an iterator, call the cursor's `fetchone()` method to retrieve a single matching row, or call `fetchall()` to get a list of the matching rows.

This example uses the iterator form:

```
>>> c = conn.cursor()
>>> c.execute('select * from stocks order by price')
>>> for row in c:
...     print row
...
(u'2006-01-05', u'BUY', u'RHAT', 100, 35.140000000000001)
(u'2006-03-28', u'BUY', u'IBM', 1000, 45.0)
(u'2006-04-06', u'SELL', u'IBM', 500, 53.0)
(u'2006-04-05', u'BUY', u'MSOFT', 1000, 72.0)
>>>
```

For more information about the SQL dialect supported by SQLite, see https://www.sqlite.org.

Ver também

https://www.pysqlite.org
The pysqlite web page.

https://www.sqlite.org
A página web do SQLite; a documentação descreve a sintaxe e os tipos de dados disponíveis para o dialeto SQL suportado.

The documentation for the `sqlite3` module.

PEP 249 [https://peps.python.org/pep-0249/] - Especificação 2.0 da API de banco de dados
PEP escrita por Marc-André Lemburg.

The wsgiref package

The Web Server Gateway Interface (WSGI) v1.0 defines a standard interface between web servers and Python web applications and is described in **PEP 333** [https://peps.python.org/pep-0333/]. The `wsgiref` package is a reference implementation of the WSGI specification.

The package includes a basic HTTP server that will run a WSGI application; this server is useful for debugging but isn't intended for production use. Setting up a server takes only a few lines of code:

```
from wsgiref import simple_server

wsgi_app = ...

host = ''
port = 8000
httpd = simple_server.make_server(host, port, wsgi_app)
httpd.serve_forever()
```

Ver também

https://web.archive.org/web/20160331090247/http://wsgi.readthedocs.org/en/latest/
A central web site for WSGI-related resources.

PEP 333 [https://peps.python.org/pep-0333/] - Python Web Server Gateway Interface v1.0
PEP written by Phillip J. Eby.

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- The Python source tree was converted from CVS to Subversion, in a complex migration procedure that was supervised and flawlessly carried out by Martin von Löwis. The procedure was developed as **PEP 347** [https://peps.python.org/pep-0347/].

- Coverity, a company that markets a source code analysis tool called Prevent, provided the results of their examination of the Python source code. The analysis found about 60 bugs that were quickly fixed. Many of the bugs were refcounting problems, often occurring in error-handling code. See https://scan.coverity.com for the statistics.

- The largest change to the C API came from **PEP 353** [https://peps.python.org/pep-0353/], which modifies the interpreter to use a `Py_ssize_t` type definition instead of int. See the earlier section PEP 353: Using ssize_t as the index type for a discussion of this change.

- The design of the bytecode compiler has changed a great deal, no longer generating bytecode by traversing the parse tree. Instead the parse tree is converted to an abstract syntax tree (or AST), and it is the abstract syntax tree that's traversed to produce the bytecode.

  It's possible for Python code to obtain AST objects by using the `compile()` built-in and specifying `_ast.PyCF_ONLY_AST` as the value of the *flags* parameter:

  ```
  from _ast import PyCF_ONLY_AST
  ast = compile("""a=0
  for i in range(10):
      a += i
  """, "<string>", 'exec', PyCF_ONLY_AST)

  assignment = ast.body[0]
  for_loop = ast.body[1]
  ```

  No official documentation has been written for the AST code yet, but **PEP 339** [https://peps.python.org/pep-0339/] discusses the design. To start learning about the code, read the definition of the various AST nodes in `Parser/Python.asdl`. A Python script reads this file and generates a set of C structure definitions in `Include/Python-ast.h`. The

PyParser_ASTFromString() and **PyParser_ASTFromFile()**, defined in `Include/pythonrun.h`, take Python source as input and return the root of an AST representing the contents. This AST can then be turned into a code object by **PyAST_Compile()**. For more information, read the source code, and then ask questions on python-dev.

The AST code was developed under Jeremy Hylton's management, and implemented by (in alphabetical order) Brett Cannon, Nick Coghlan, Grant Edwards, John Ehresman, Kurt Kaiser, Neal Norwitz, Tim Peters, Armin Rigo, and Neil Schemenauer, plus the participants in a number of AST sprints at conferences such as PyCon.

- Evan Jones's patch to obmalloc, first described in a talk at PyCon DC 2005, was applied. Python 2.4 allocated small objects in 256K-sized arenas, but never freed arenas. With this patch, Python will free arenas when they're empty. The net effect is that on some platforms, when you allocate many objects, Python's memory usage may actually drop when you delete them and the memory may be returned to the operating system. (Implemented by Evan Jones, and reworked by Tim Peters.)

  Note that this change means extension modules must be more careful when allocating memory. Python's API has many different functions for allocating memory that are grouped into families. For example, **PyMem_Malloc()**, **PyMem_Realloc()**, and **PyMem_Free()** are one family that allocates raw memory, while **PyObject_Malloc()**, **PyObject_Realloc()**, and **PyObject_Free()** are another family that's supposed to be used for creating Python objects.

  Previously these different families all reduced to the platform's **malloc()** and **free()** functions. This meant it didn't matter if you got things wrong and allocated memory with the `PyMem` function but freed it with the `PyObject` function. With 2.5's changes to obmalloc, these families now do different things and mismatches will probably result in a segfault. You should carefully test your C extension modules with Python 2.5.

- The built-in set types now have an official C API. Call **PySet_New()** and **PyFrozenSet_New()** to create a new set, **PySet_Add()** and **PySet_Discard()** to add and remove elements, and **PySet_Contains()** and **PySet_Size()** to examine the set's state. (Contributed by Raymond Hettinger.)

- C code can now obtain information about the exact revision of the Python interpreter by calling the **Py_GetBuildInfo()** function that returns a string of build information like this: `"trunk:45355:45356M, Apr 13 2006, 07:42:19"`. (Contributed by Barry Warsaw.)

- Two new macros can be used to indicate C functions that are local to the current file so that a faster calling convention can be used. **Py_LOCAL** declares the function as returning a value of the specified *type* and uses a fast-calling qualifier. **Py_LOCAL_INLINE** does the same thing and also requests the function be inlined. If macro **PY_LOCAL_AGGRESSIVE** is defined before `python.h` is included, a set of more aggressive optimizations are enabled for the module; you should benchmark the results to find out if these optimizations actually make the code faster. (Contributed by Fredrik Lundh at the NeedForSpeed sprint.)

- `PyErr_NewException(name, base, dict)` can now accept a tuple of base classes as its *base* argument. (Contributed by Georg Brandl.)

- The **PyErr_Warn()** function for issuing warnings is now deprecated in favour of `PyErr_WarnEx(category, message, stacklevel)` which lets you specify the number of stack frames separating this function and the caller. A *stacklevel* of 1 is the function calling **PyErr_WarnEx()**, 2 is the function above that, and so forth. (Added by Neal Norwitz.)

- The CPython interpreter is still written in C, but the code can now be compiled with a C++ compiler without errors. (Implemented by Anthony Baxter, Martin von Löwis, Skip Montanaro.)

- The **PyRange_New()** function was removed. It was never documented, never used in the core code, and had dangerously lax error checking.

In the unlikely case that your extensions were using it, you can replace it by something like the following:

```
range = PyObject_CallFunction((PyObject*) &PyRange_Type, "l
                              start, stop, step);
```

Port-Specific Changes

- MacOS X (10.3 and higher): dynamic loading of modules now uses the **`dlopen()`** function instead of MacOS-specific functions.
- MacOS X: an **`--enable-universalsdk`** switch was added to the **configure** script that compiles the interpreter as a universal binary able to run on both PowerPC and Intel processors. (Contributed by Ronald Oussoren; bpo-2573 [https://bugs.python.org/issue?@action=redirect&bpo=2573].)
- Windows: `.dll` is no longer supported as a filename extension for extension modules. `.pyd` is now the only filename extension that will be searched for.

Porting to Python 2.5

This section lists previously described changes that may require changes to your code:

- ASCII is now the default encoding for modules. It's now a syntax error if a module contains string literals with 8-bit characters but doesn't have an encoding declaration. In Python 2.4 this triggered a warning, not a syntax error.
- Previously, the **`gi_frame`** attribute of a generator was always a frame object. Because of the **PEP 342** [https://peps.python.org/pep-0342/] changes described in section PEP 342: New Generator Features, it's now possible for **`gi_frame`** to be `None`.
- A new warning, **`UnicodeWarning`**, is triggered when you attempt to compare a Unicode string and an 8-bit string that can't be converted to Unicode using the default ASCII encoding. Previously such comparisons would raise a **`UnicodeDecodeError`** exception.

- Library: the `csv` module is now stricter about multi-line quoted fields. If your files contain newlines embedded within fields, the input should be split into lines in a manner which preserves the newline characters.
- Library: the `locale` module's `format()` function's would previously accept any string as long as no more than one %char specifier appeared. In Python 2.5, the argument must be exactly one %char specifier with no surrounding text.
- Library: The `pickle` and `cPickle` modules no longer accept a return value of `None` from the `__reduce__()` method; the method must return a tuple of arguments instead. The modules also no longer accept the deprecated *bin* keyword parameter.
- Library: The `SimpleXMLRPCServer` and `DocXMLRPCServer` classes now have a `rpc_paths` attribute that constrains XML-RPC operations to a limited set of URL paths; the default is to allow only `'/'` and `'/RPC2'`. Setting `rpc_paths` to `None` or an empty tuple disables this path checking.
- C API: Many functions now use `Py_ssize_t` instead of int to allow processing more data on 64-bit machines. Extension code may need to make the same change to avoid warnings and to support 64-bit machines. See the earlier section PEP 353: Using ssize_t as the index type for a discussion of this change.
- C API: The obmalloc changes mean that you must be careful to not mix usage of the `PyMem_*` and `PyObject_*` families of functions. Memory allocated with one family's `*_Malloc` must be freed with the corresponding family's `*_Free` function.

Reconhecimentos

The author would like to thank the following people for offering suggestions, corrections and assistance with various drafts of this article: Georg Brandl, Nick Coghlan, Phillip J. Eby, Lars Gustäbel, Raymond Hettinger, Ralf W. Grosse-Kunstleve, Kent Johnson, Iain Lowe, Martin von Löwis, Fredrik Lundh, Andrew McNamara, Skip Montanaro, Gustavo Niemeyer, Paul Prescod, James Pryor, Mike Rovner, Scott Weikart, Barry Warsaw, Thomas Wouters.

What’s New in Python 2.4

Autor:: A.M. Kuchling

This article explains the new features in Python 2.4.1, released on March 30, 2005.

Python 2.4 is a medium-sized release. It doesn’t introduce as many changes as the radical Python 2.2, but introduces more features than the conservative 2.3 release. The most significant new language features are function decorators and generator expressions; most other changes are to the standard library.

According to the CVS change logs, there were 481 patches applied and 502 bugs fixed between Python 2.3 and 2.4. Both figures are likely to be underestimates.

This article doesn’t attempt to provide a complete specification of every single new feature, but instead provides a brief introduction to each feature. For full details, you should refer to the documentation for Python 2.4, such as the Python Library Reference and the Python Reference Manual. Often you will be referred to the PEP for a particular new feature for explanations of the implementation and design rationale.

PEP 218: Built-In Set Objects

Python 2.3 introduced the **`sets`** module. C implementations of set data types have now been added to the Python core as two new built-in types, `set(iterable)` and `frozenset(iterable)`. They provide high speed operations for membership testing, for eliminating duplicates from sequences, and for mathematical operations like unions, intersections, differences, and symmetric differences.

```
>>> a = set('abracadabra')              # form a set from a stri
>>> 'z' in a                            # fast membership testir
```

```
False
>>> a                                   # unique letters in a
set(['a', 'r', 'b', 'c', 'd'])
>>> ''.join(a)                          # convert back into a st
'arbcd'

>>> b = set('alacazam')                 # form a second set
>>> a - b                               # letters in a but not i
set(['r', 'd', 'b'])
>>> a | b                               # letters in either a or
set(['a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'])
>>> a & b                               # letters in both a and
set(['a', 'c'])
>>> a ^ b                               # letters in a or b but
set(['r', 'd', 'b', 'm', 'z', 'l'])

>>> a.add('z')                          # add a new element
>>> a.update('wxy')                     # add multiple new eleme
>>> a
set(['a', 'c', 'b', 'd', 'r', 'w', 'y', 'x', 'z'])
>>> a.remove('x')                       # take one element out
>>> a
set(['a', 'c', 'b', 'd', 'r', 'w', 'y', 'z'])
```

The `frozenset()` type is an immutable version of `set()`. Since it is immutable and hashable, it may be used as a dictionary key or as a member of another set.

The `sets` module remains in the standard library, and may be useful if you wish to subclass the `Set` or `ImmutableSet` classes. There are currently no plans to deprecate the module.

Ver também

PEP 218 [https://peps.python.org/pep-0218/] - Adding a Built-In Set Object Type

Originally proposed by Greg Wilson and ultimately implemented by Raymond Hettinger.

PEP 237: Unifying Long Integers and Integers

The lengthy transition process for this PEP, begun in Python 2.2, takes another step forward in Python 2.4. In 2.3, certain integer operations that would behave differently after int/long unification triggered `FutureWarning` warnings and returned values limited to 32 or 64 bits (depending on your platform). In 2.4, these expressions no longer produce a warning and instead produce a different result that's usually a long integer.

The problematic expressions are primarily left shifts and lengthy hexadecimal and octal constants. For example, `2 << 32` results in a warning in 2.3, evaluating to 0 on 32-bit platforms. In Python 2.4, this expression now returns the correct answer, 8589934592.

Ver também

PEP 237 [https://peps.python.org/pep-0237/] - Unifying Long Integers and Integers

Original PEP written by Moshe Zadka and GvR. The changes for 2.4 were implemented by Kalle Svensson.

PEP 289: Generator Expressions

The iterator feature introduced in Python 2.2 and the `itertools` module make it easier to write programs that loop through large data sets without having the entire data set in memory at one time. List comprehensions don't fit into this picture very well because they produce a Python list object containing all of the items. This unavoidably pulls all of the objects into memory, which can be a problem if your data set is very large. When trying to write a functionally styled program, it would be natural to write something like:

```
links = [link for link in get_all_links() if not link.followed]
for link in links:
    ...
```

instead of

```
for link in get_all_links():
    if link.followed:
        continue
    ...
```

The first form is more concise and perhaps more readable, but if you're dealing with a large number of link objects you'd have to write the second form to avoid having all link objects in memory at the same time.

Generator expressions work similarly to list comprehensions but don't materialize the entire list; instead they create a generator that will return elements one by one. The above example could be written as:

```
links = (link for link in get_all_links() if not link.followed)
for link in links:
    ...
```

Generator expressions always have to be written inside parentheses, as in the above example. The parentheses signalling a function call also count, so if you want to create an iterator that will be immediately passed to a function you could write:

```
print sum(obj.count for obj in list_all_objects())
```

Generator expressions differ from list comprehensions in various small ways. Most notably, the loop variable (*obj* in the above example) is not accessible outside of the generator expression. List comprehensions leave the variable assigned to its last value; future versions of Python will change this, making list comprehensions match generator expressions in this respect.

Ver também

PEP 289 **[https://peps.python.org/pep-0289/]** **- Generator Expressions**
Proposed by Raymond Hettinger and implemented by Jiwon Seo with early efforts steered by Hye-Shik Chang.

PEP 292: Simpler String Substitutions

Some new classes in the standard library provide an alternative mechanism for substituting variables into strings; this style of substitution may be better for applications where untrained users need to edit templates.

The usual way of substituting variables by name is the `%` operator:

```
>>> '%(page)i: %(title)s' % {'page':2, 'title': 'The Best of Tim
'2: The Best of Times'
```

When writing the template string, it can be easy to forget the `i` or `s` after the closing parenthesis. This isn't a big problem if the template is in a Python module, because you run the code, get an "Unsupported format character" `ValueError`, and fix the problem. However, consider an application such as Mailman where template strings or translations are being edited by users who aren't aware of the Python language. The format string's syntax is complicated to explain to such users, and if they make a mistake, it's difficult to provide helpful feedback to them.

PEP 292 adds a `Template` class to the `string` module that uses `$` to indicate a substitution:

```
>>> import string
>>> t = string.Template('$page: $title')
>>> t.substitute({'page':2, 'title': 'The Best of Times'})
'2: The Best of Times'
```

If a key is missing from the dictionary, the `substitute()` method will raise a `KeyError`. There's also a `safe_substitute()` method that ignores missing keys:

```
>>> t = string.Template('$page: $title')
>>> t.safe_substitute({'page':3})
'3: $title'
```

Ver também

PEP 292 **[https://peps.python.org/pep-0292/]** **- Simpler String Substitutions**
Written and implemented by Barry Warsaw.

PEP 318: Decorators for Functions and Methods

Python 2.2 extended Python's object model by adding static methods and class methods, but it didn't extend Python's syntax to provide any new way of defining static or class methods. Instead, you had to write a `def` statement in the usual way, and pass the resulting method to a `staticmethod()` or `classmethod()` function that would wrap up the function as a method of the new type. Your code would look like this:

```
class C:
   def meth (cls):
       ...

   meth = classmethod(meth)   # Rebind name to wrapped-up class
```

If the method was very long, it would be easy to miss or forget the `classmethod()` invocation after the function body.

The intention was always to add some syntax to make such definitions more readable, but at the time of 2.2's release a good syntax was not obvious. Today a good syntax *still* isn't obvious but users are asking for easier access to the feature; a new syntactic feature has been added to meet this need.

The new feature is called "function decorators". The name comes from the idea that `classmethod()`, `staticmethod()`, and friends are storing additional information on a function object; they're *decorating* functions with more details.

The notation borrows from Java and uses the `'@'` character as an indicator. Using the new syntax, the example above would be written:

```
class C:

   @classmethod
```

```
   def meth (cls):
       ...
```

The `@classmethod` is shorthand for the `meth=classmethod(meth)` assignment. More generally, if you have the following:

```
@A
@B
@C
def f ():
    ...
```

It's equivalent to the following pre-decorator code:

```
def f(): ...
f = A(B(C(f)))
```

Decorators must come on the line before a function definition, one decorator per line, and can't be on the same line as the def statement, meaning that `@A def f(): ...` is illegal. You can only decorate function definitions, either at the module level or inside a class; you can't decorate class definitions.

A decorator is just a function that takes the function to be decorated as an argument and returns either the same function or some new object. The return value of the decorator need not be callable (though it typically is), unless further decorators will be applied to the result. It's easy to write your own decorators. The following simple example just sets an attribute on the function object:

```
>>> def deco(func):
...     func.attr = 'decorated'
...     return func
...
>>> @deco
... def f(): pass
...
>>> f
<function f at 0x402ef0d4>
>>> f.attr
```

```
'decorated'
>>>
```

As a slightly more realistic example, the following decorator checks that the supplied argument is an integer:

```
def require_int (func):
    def wrapper (arg):
        assert isinstance(arg, int)
        return func(arg)

    return wrapper

@require_int
def p1 (arg):
    print arg

@require_int
def p2(arg):
    print arg*2
```

An example in **PEP 318** [https://peps.python.org/pep-0318/] contains a fancier version of this idea that lets you both specify the required type and check the returned type.

Decorator functions can take arguments. If arguments are supplied, your decorator function is called with only those arguments and must return a new decorator function; this function must take a single function and return a function, as previously described. In other words, `@A @B @C(args)` becomes:

```
def f(): ...
_deco = C(args)
f = A(B(_deco(f)))
```

Getting this right can be slightly brain-bending, but it's not too difficult.

A small related change makes the **`func_name`** attribute of functions writable. This attribute is used to display function names in tracebacks, so decorators

should change the name of any new function that's constructed and returned.

Ver também

PEP 318 [https://peps.python.org/pep-0318/] - Decorators for Functions, Methods and Classes

Written by Kevin D. Smith, Jim Jewett, and Skip Montanaro. Several people wrote patches implementing function decorators, but the one that was actually checked in was patch #979728, written by Mark Russell.

https://wiki.python.org/moin/PythonDecoratorLibrary

This Wiki page contains several examples of decorators.

PEP 322: Reverse Iteration

A new built-in function, `reversed(seq)`, takes a sequence and returns an iterator that loops over the elements of the sequence in reverse order.

```
>>> for i in reversed(xrange(1,4)):
...     print i
...
3
2
1
```

Compared to extended slicing, such as `range(1,4)[::-1]`, `reversed()` is easier to read, runs faster, and uses substantially less memory.

Note that `reversed()` only accepts sequences, not arbitrary iterators. If you want to reverse an iterator, first convert it to a list with `list()`.

```
>>> input = open('/etc/passwd', 'r')
>>> for line in reversed(list(input)):
...   print line
...
```

```
root:*:0:0:System Administrator:/var/root:/bin/tcsh
  ...
```

Ver também

PEP 322 **[https://peps.python.org/pep-0322/]** **- Reverse Iteration**
Written and implemented by Raymond Hettinger.

PEP 324: New subprocess Module

The standard library provides a number of ways to execute a subprocess, offering different features and different levels of complexity. `os.system(command)` is easy to use, but slow (it runs a shell process which executes the command) and dangerous (you have to be careful about escaping the shell's metacharacters). The **`popen2`** module offers classes that can capture standard output and standard error from the subprocess, but the naming is confusing. The **`subprocess`** module cleans this up, providing a unified interface that offers all the features you might need.

Instead of **`popen2`**'s collection of classes, **`subprocess`** contains a single class called **`subprocess.Popen`** whose constructor supports a number of different keyword arguments.

```
class Popen(args, bufsize=0, executable=None,
            stdin=None, stdout=None, stderr=None,
            preexec_fn=None, close_fds=False, shell=False,
            cwd=None, env=None, universal_newlines=False,
            startupinfo=None, creationflags=0):
```

args is commonly a sequence of strings that will be the arguments to the program executed as the subprocess. (If the *shell* argument is true, *args* can be a string which will then be passed on to the shell for interpretation, just as **`os.system()`** does.)

stdin, *stdout*, and *stderr* specify what the subprocess's input, output, and error streams will be. You can provide a file object or a file descriptor, or

you can use the constant `subprocess.PIPE` to create a pipe between the subprocess and the parent.

The constructor has a number of handy options:

- *close_fds* requests that all file descriptors be closed before running the subprocess.
- *cwd* specifies the working directory in which the subprocess will be executed (defaulting to whatever the parent's working directory is).
- *env* is a dictionary specifying environment variables.
- *preexec_fn* is a function that gets called before the child is started.
- *universal_newlines* opens the child's input and output using Python's universal newlines feature.

Once you've created the **`Popen`** instance, you can call its **`wait()`** method to pause until the subprocess has exited, **`poll()`** to check if it's exited without pausing, or `communicate(data)` to send the string *data* to the subprocess's standard input. `communicate(data)` then reads any data that the subprocess has sent to its standard output or standard error, returning a tuple `(stdout_data, stderr_data)`.

`call()` is a shortcut that passes its arguments along to the **`Popen`** constructor, waits for the command to complete, and returns the status code of the subprocess. It can serve as a safer analog to **`os.system()`**:

```
sts = subprocess.call(['dpkg', '-i', '/tmp/new-package.deb'])
if sts == 0:
    # Success
    ...
else:
    # dpkg returned an error
    ...
```

The command is invoked without use of the shell. If you really do want to use the shell, you can add `shell=True` as a keyword argument and provide a string instead of a sequence:

```
sts = subprocess.call('dpkg -i /tmp/new-package.deb', shell=True
```

The PEP takes various examples of shell and Python code and shows how they'd be translated into Python code that uses `subprocess`. Reading this section of the PEP is highly recommended.

Ver também

PEP 324 [https://peps.python.org/pep-0324/] - subprocess - New process module

Written and implemented by Peter Åstrand, with assistance from Fredrik Lundh and others.

PEP 327: Decimal Data Type

Python has always supported floating-point (FP) numbers, based on the underlying C double type, as a data type. However, while most programming languages provide a floating-point type, many people (even programmers) are unaware that floating-point numbers don't represent certain decimal fractions accurately. The new `Decimal` type can represent these fractions accurately, up to a user-specified precision limit.

Why is Decimal needed?

The limitations arise from the representation used for floating-point numbers. FP numbers are made up of three components:

- The sign, which is positive or negative.
- The mantissa, which is a single-digit binary number followed by a fractional part. For example, `1.01` in base-2 notation is `1 + 0/2 + 1/4`, or 1.25 in decimal notation.
- The exponent, which tells where the decimal point is located in the number represented.

For example, the number 1.25 has positive sign, a mantissa value of 1.01 (in binary), and an exponent of 0 (the decimal point doesn't need to be shifted). The number 5 has the same sign and mantissa, but the exponent is 2

because the mantissa is multiplied by 4 (2 to the power of the exponent 2); 1.25 * 4 equals 5.

Modern systems usually provide floating-point support that conforms to a standard called IEEE 754. C's double type is usually implemented as a 64-bit IEEE 754 number, which uses 52 bits of space for the mantissa. This means that numbers can only be specified to 52 bits of precision. If you're trying to represent numbers whose expansion repeats endlessly, the expansion is cut off after 52 bits. Unfortunately, most software needs to produce output in base 10, and common fractions in base 10 are often repeating decimals in binary. For example, 1.1 decimal is binary `1.0001100110011 ...`; .1 = 1/16 + 1/32 + 1/256 plus an infinite number of additional terms. IEEE 754 has to chop off that infinitely repeated decimal after 52 digits, so the representation is slightly inaccurate.

Sometimes you can see this inaccuracy when the number is printed:

```
>>> 1.1
1.1000000000000001
```

The inaccuracy isn't always visible when you print the number because the FP-to-decimal-string conversion is provided by the C library, and most C libraries try to produce sensible output. Even if it's not displayed, however, the inaccuracy is still there and subsequent operations can magnify the error.

For many applications this doesn't matter. If I'm plotting points and displaying them on my monitor, the difference between 1.1 and 1.1000000000000001 is too small to be visible. Reports often limit output to a certain number of decimal places, and if you round the number to two or three or even eight decimal places, the error is never apparent. However, for applications where it does matter, it's a lot of work to implement your own custom arithmetic routines.

Hence, the `Decimal` type was created.

The `Decimal` type

A new module, **decimal**, was added to Python's standard library. It contains two classes, **Decimal** and **Context**. **Decimal** instances represent numbers, and **Context** instances are used to wrap up various settings such as the precision and default rounding mode.

Decimal instances are immutable, like regular Python integers and FP numbers; once it's been created, you can't change the value an instance represents. **Decimal** instances can be created from integers or strings:

```
>>> import decimal
>>> decimal.Decimal(1972)
Decimal("1972")
>>> decimal.Decimal("1.1")
Decimal("1.1")
```

You can also provide tuples containing the sign, the mantissa represented as a tuple of decimal digits, and the exponent:

```
>>> decimal.Decimal((1, (1, 4, 7, 5), -2))
Decimal("-14.75")
```

Cautionary note: the sign bit is a Boolean value, so 0 is positive and 1 is negative.

Converting from floating-point numbers poses a bit of a problem: should the FP number representing 1.1 turn into the decimal number for exactly 1.1, or for 1.1 plus whatever inaccuracies are introduced? The decision was to dodge the issue and leave such a conversion out of the API. Instead, you should convert the floating-point number into a string using the desired precision and pass the string to the **Decimal** constructor:

```
>>> f = 1.1
>>> decimal.Decimal(str(f))
Decimal("1.1")
>>> decimal.Decimal('%.12f' % f)
Decimal("1.100000000000")
```

Once you have **Decimal** instances, you can perform the usual mathematical operations on them. One limitation: exponentiation requires an integer exponent:

```
>>> a = decimal.Decimal('35.72')
>>> b = decimal.Decimal('1.73')
>>> a+b
Decimal("37.45")
>>> a-b
Decimal("33.99")
>>> a*b
Decimal("61.7956")
>>> a/b
Decimal("20.64739884393063583815028902")
>>> a ** 2
Decimal("1275.9184")
>>> a**b
Traceback (most recent call last):
  ...
decimal.InvalidOperation: x ** (non-integer)
```

You can combine **Decimal** instances with integers, but not with floating-point numbers:

```
>>> a + 4
Decimal("39.72")
>>> a + 4.5
Traceback (most recent call last):
  ...
TypeError: You can interact Decimal only with int, long or Decim
>>>
```

Decimal numbers can be used with the **math** and **cmath** modules, but note that they'll be immediately converted to floating-point numbers before the operation is performed, resulting in a possible loss of precision and accuracy. You'll also get back a regular floating-point number and not a **Decimal**.

```
>>> import math, cmath
>>> d = decimal.Decimal('123456789012.345')
>>> math.sqrt(d)
351364.18288201344
>>> cmath.sqrt(-d)
351364.18288201344j
```

`Decimal` instances have a `sqrt()` method that returns a **`Decimal`**, but if you need other things such as trigonometric functions you'll have to implement them.

```
>>> d.sqrt()
Decimal("351364.18288201345921772450 01")
```

The `Context` type

Instances of the `Context` class encapsulate several settings for decimal operations:

- `prec` is the precision, the number of decimal places.
- `rounding` specifies the rounding mode. The `decimal` module has constants for the various possibilities: `ROUND_DOWN`, `ROUND_CEILING`, `ROUND_HALF_EVEN`, and various others.
- `traps` is a dictionary specifying what happens on encountering certain error conditions: either an exception is raised or a value is returned. Some examples of error conditions are division by zero, loss of precision, and overflow.

There's a thread-local default context available by calling `getcontext()`; you can change the properties of this context to alter the default precision, rounding, or trap handling. The following example shows the effect of changing the precision of the default context:

```
>>> decimal.getcontext().prec
28
>>> decimal.Decimal(1) / decimal.Decimal(7)
Decimal("0.1428571428571428571428571429")
>>> decimal.getcontext().prec = 9
>>> decimal.Decimal(1) / decimal.Decimal(7)
Decimal("0.142857143")
```

The default action for error conditions is selectable; the module can either return a special value such as infinity or not-a-number, or exceptions can be raised:

```
>>> decimal.Decimal(1) / decimal.Decimal(0)
Traceback (most recent call last):
  ...
decimal.DivisionByZero: x / 0
>>> decimal.getcontext().traps[decimal.DivisionByZero] = False
>>> decimal.Decimal(1) / decimal.Decimal(0)
Decimal("Infinity")
>>>
```

The `Context` instance also has various methods for formatting numbers such as `to_eng_string()` and `to_sci_string()`.

For more information, see the documentation for the `decimal` module, which includes a quick-start tutorial and a reference.

Ver também

PEP 327 **[https://peps.python.org/pep-0327/]** **- Decimal Data Type**
Written by Facundo Batista and implemented by Facundo Batista, Eric Price, Raymond Hettinger, Aahz, and Tim Peters.

http://www.lahey.com/float.htm
[https://web.archive.org/web/20230604072523/http://www.lahey.com/float.htm]
The article uses Fortran code to illustrate many of the problems that floating-point inaccuracy can cause.

https://speleotrove.com/decimal/
A description of a decimal-based representation. This representation is being proposed as a standard, and underlies the new Python decimal type. Much of this material was written by Mike Cowlishaw, designer of the Rexx language.

PEP 328: Multi-line Imports

One language change is a small syntactic tweak aimed at making it easier to import many names from a module. In a `from module import names` statement, *names* is a sequence of names separated by commas. If the

sequence is very long, you can either write multiple imports from the same module, or you can use backslashes to escape the line endings like this:

```
from SimpleXMLRPCServer import SimpleXMLRPCServer,\
            SimpleXMLRPCRequestHandler,\
            CGIXMLRPCRequestHandler,\
            resolve_dotted_attribute
```

The syntactic change in Python 2.4 simply allows putting the names within parentheses. Python ignores newlines within a parenthesized expression, so the backslashes are no longer needed:

```
from SimpleXMLRPCServer import (SimpleXMLRPCServer,
                                SimpleXMLRPCRequestHandler,
                                CGIXMLRPCRequestHandler,
                                resolve_dotted_attribute)
```

The PEP also proposes that all `import` statements be absolute imports, with a leading `.` character to indicate a relative import. This part of the PEP was not implemented for Python 2.4, but was completed for Python 2.5.

Ver também

PEP 328 [https://peps.python.org/pep-0328/] - Imports: Multi-Line and Absolute/Relative

Written by Aahz. Multi-line imports were implemented by Dima Dorfman.

PEP 331: Locale-Independent Float/String Conversions

The `locale` modules lets Python software select various conversions and display conventions that are localized to a particular country or language. However, the module was careful to not change the numeric locale because various functions in Python's implementation required that the numeric

locale remain set to the `'C'` locale. Often this was because the code was using the C library's **`atof()`** function.

Not setting the numeric locale caused trouble for extensions that used third-party C libraries, however, because they wouldn't have the correct locale set. The motivating example was GTK+, whose user interface widgets weren't displaying numbers in the current locale.

The solution described in the PEP is to add three new functions to the Python API that perform ASCII-only conversions, ignoring the locale setting:

- `PyOS_ascii_strtod(str, ptr)` and `PyOS_ascii_atof(str, ptr)` both convert a string to a C double.
- `PyOS_ascii_formatd(buffer, buf_len, format, d)` converts a double to an ASCII string.

The code for these functions came from the GLib library (https://developer-old.gnome.org/glib/2.26/ [http://web.archive.org/web/20210306104320/https://developer.gnome.org/glib/2.26/]), whose developers kindly relicensed the relevant functions and donated them to the Python Software Foundation. The **`locale`** module can now change the numeric locale, letting extensions such as GTK+ produce the correct results.

Ver também

PEP 331 [https://peps.python.org/pep-0331/] - Locale-Independent Float/String Conversions
Written by Christian R. Reis, and implemented by Gustavo Carneiro.

Outras mudanças na linguagem

Here are all of the changes that Python 2.4 makes to the core Python language.

- Decorators for functions and methods were added (**PEP 318** [https://peps.python.org/pep-0318/]).

- Built-in `set()` and `frozenset()` types were added (**PEP 218** [https://peps.python.org/pep-0218/]). Other new built-ins include the `reversed(seq)` function (**PEP 322** [https://peps.python.org/pep-0322/]).

- Generator expressions were added (**PEP 289** [https://peps.python.org/pep-0289/]).

- Certain numeric expressions no longer return values restricted to 32 or 64 bits (**PEP 237** [https://peps.python.org/pep-0237/]).

- You can now put parentheses around the list of names in a `from module import names` statement (**PEP 328** [https://peps.python.org/pep-0328/]).

- The `dict.update()` method now accepts the same argument forms as the `dict` constructor. This includes any mapping, any iterable of key/value pairs, and keyword arguments. (Contributed by Raymond Hettinger.)

- The string methods `ljust()`, `rjust()`, and `center()` now take an optional argument for specifying a fill character other than a space. (Contributed by Raymond Hettinger.)

- Strings also gained an `rsplit()` method that works like the `split()` method but splits from the end of the string. (Contributed by Sean Reifschneider.)

  ```
  >>> 'www.python.org'.split('.', 1)
  ['www', 'python.org']
  'www.python.org'.rsplit('.', 1)
  ['www.python', 'org']
  ```

- Three keyword parameters, *cmp*, *key*, and *reverse*, were added to the `sort()` method of lists. These parameters make some common usages of **`sort()`** simpler. All of these parameters are optional.

For the *cmp* parameter, the value should be a comparison function that takes two parameters and returns -1, 0, or +1 depending on how the parameters compare. This function will then be used to sort the list. Previously this was the only parameter that could be provided to **`sort()`**.

key should be a single-parameter function that takes a list element and returns a comparison key for the element. The list is then sorted using the comparison keys. The following example sorts a list case-insensitively:

```
>>> L = ['A', 'b', 'c', 'D']
>>> L.sort()                 # Case-sensitive sort
>>> L
['A', 'D', 'b', 'c']
>>> # Using 'key' parameter to sort list
>>> L.sort(key=lambda x: x.lower())
>>> L
['A', 'b', 'c', 'D']
>>> # Old-fashioned way
>>> L.sort(cmp=lambda x,y: cmp(x.lower(), y.lower()))
>>> L
['A', 'b', 'c', 'D']
```

The last example, which uses the *cmp* parameter, is the old way to perform a case-insensitive sort. It works but is slower than using a *key* parameter. Using *key* calls **`lower()`** method once for each element in the list while using *cmp* will call it twice for each comparison, so using *key* saves on invocations of the **`lower()`** method.

For simple key functions and comparison functions, it is often possible to avoid a **`lambda`** expression by using an unbound method instead. For example, the above case-insensitive sort is best written as:

```
>>> L.sort(key=str.lower)
>>> L
['A', 'b', 'c', 'D']
```

Finally, the *reverse* parameter takes a Boolean value. If the value is true, the list will be sorted into reverse order. Instead of `L.sort();`

`L.reverse()`, you can now write `L.sort(reverse=True)`.

The results of sorting are now guaranteed to be stable. This means that two entries with equal keys will be returned in the same order as they were input. For example, you can sort a list of people by name, and then sort the list by age, resulting in a list sorted by age where people with the same age are in name-sorted order.

(All changes to **`sort()`** contributed by Raymond Hettinger.)

- There is a new built-in function `sorted(iterable)` that works like the in-place **`list.sort()`** method but can be used in expressions. The differences are:

- the input may be any iterable;

- a newly formed copy is sorted, leaving the original intact; and

- the expression returns the new sorted copy

```
>>> L = [9,7,8,3,2,4,1,6,5]
>>> [10+i for i in sorted(L)]       # usable in a list compr
[11, 12, 13, 14, 15, 16, 17, 18, 19]
>>> L                               # original is left uncha
[9,7,8,3,2,4,1,6,5]
>>> sorted('Monty Python')          # any iterable may be ar
[' ', 'M', 'P', 'h', 'n', 'n', 'o', 'o', 't', 't', 'y', 'y']

>>> # List the contents of a dict sorted by key values
>>> colormap = dict(red=1, blue=2, green=3, black=4, yellow=
>>> for k, v in sorted(colormap.iteritems()):
...     print k, v
...
black 4
blue 2
green 3
red 1
yellow 5
```

(Contributed by Raymond Hettinger.)

- Integer operations will no longer trigger an **`OverflowWarning`**. The **`OverflowWarning`** warning will disappear in Python 2.5.

- The interpreter gained a new switch, `-m`, that takes a name, searches for the corresponding module on `sys.path`, and runs the module as a script. For example, you can now run the Python profiler with `python -m profile`. (Contributed by Nick Coghlan.)

- The `eval(expr, globals, locals)` and `execfile(filename, globals, locals)` functions and the `exec` statement now accept any mapping type for the *locals* parameter. Previously this had to be a regular Python dictionary. (Contributed by Raymond Hettinger.)

- The **`zip()`** built-in function and **`itertools.izip()`** now return an empty list if called with no arguments. Previously they raised a **`TypeError`** exception. This makes them more suitable for use with variable length argument lists:

  ```
  >>> def transpose(array):
  ...     return zip(*array)
  ...
  >>> transpose([(1,2,3), (4,5,6)])
  [(1, 4), (2, 5), (3, 6)]
  >>> transpose([])
  []
  ```

  (Contributed by Raymond Hettinger.)

- Encountering a failure while importing a module no longer leaves a partially initialized module object in `sys.modules`. The incomplete module object left behind would fool further imports of the same module into succeeding, leading to confusing errors. (Fixed by Tim Peters.)

- **`None`** is now a constant; code that binds a new value to the name `None` is now a syntax error. (Contributed by Raymond Hettinger.)

Otimizações

- The inner loops for list and tuple slicing were optimized and now run about one-third faster. The inner loops for dictionaries were also optimized, resulting in performance boosts for **keys()**, **values()**, **items()**, **iterkeys()**, **itervalues()**, and **iteritems()**. (Contributed by Raymond Hettinger.)
- The machinery for growing and shrinking lists was optimized for speed and for space efficiency. Appending and popping from lists now runs faster due to more efficient code paths and less frequent use of the underlying system **realloc()**. List comprehensions also benefit. **list.extend()** was also optimized and no longer converts its argument into a temporary list before extending the base list. (Contributed by Raymond Hettinger.)
- **list()**, **tuple()**, **map()**, **filter()**, and **zip()** now run several times faster with non-sequence arguments that supply a **__len__()** method. (Contributed by Raymond Hettinger.)
- The methods **list.__getitem__()**, **dict.__getitem__()**, and **dict.__contains__()** are now implemented as **method_descriptor** objects rather than **wrapper_descriptor** objects. This form of access doubles their performance and makes them more suitable for use as arguments to functionals: `map(mydict.__getitem__, keylist)`. (Contributed by Raymond Hettinger.)
- Added a new opcode, `LIST_APPEND`, that simplifies the generated bytecode for list comprehensions and speeds them up by about a third. (Contributed by Raymond Hettinger.)
- The peephole bytecode optimizer has been improved to produce shorter, faster bytecode; remarkably, the resulting bytecode is more readable. (Enhanced by Raymond Hettinger.)
- String concatenations in statements of the form `s = s + "abc"` and `s += "abc"` are now performed more efficiently in certain circumstances. This optimization won't be present in other Python implementations such as Jython, so you shouldn't rely on it; using the **join()** method of strings is still recommended when you want to efficiently glue a large number of strings together. (Contributed by Armin Rigo.)

The net result of the 2.4 optimizations is that Python 2.4 runs the pystone benchmark around 5% faster than Python 2.3 and 35% faster than Python 2.2. (pystone is not a particularly good benchmark, but it's the most

commonly used measurement of Python's performance. Your own applications may show greater or smaller benefits from Python 2.4.)

New, Improved, and Deprecated Modules

As usual, Python's standard library received a number of enhancements and bug fixes. Here's a partial list of the most notable changes, sorted alphabetically by module name. Consult the `Misc/NEWS` file in the source tree for a more complete list of changes, or look through the CVS logs for all the details.

- The **`asyncore`** module's **`loop()`** function now has a *count* parameter that lets you perform a limited number of passes through the polling loop. The default is still to loop forever.

- The **`base64`** module now has more complete **RFC 3548** [https://datatracker.ietf.org/doc/html/rfc3548.html] support for Base64, Base32, and Base16 encoding and decoding, including optional case folding and optional alternative alphabets. (Contributed by Barry Warsaw.)

- The **`bisect`** module now has an underlying C implementation for improved performance. (Contributed by Dmitry Vasiliev.)

- The CJKCodecs collections of East Asian codecs, maintained by Hye-Shik Chang, was integrated into 2.4. The new encodings are:

- Chinese (PRC): gb2312, gbk, gb18030, big5hkscs, hz

- Chinese (ROC): big5, cp950

- Japanese: cp932, euc-jis-2004, euc-jp, euc-jisx0213, iso-2022-jp, iso-2022-jp-1, iso-2022-jp-2, iso-2022-jp-3, iso-2022-jp-ext, iso-2022-jp-2004, shift-jis, shift-jisx0213, shift-jis-2004

- Korean: cp949, euc-kr, johab, iso-2022-kr

- Some other new encodings were added: HP Roman8, ISO_8859-11, ISO_8859-16, PCTP-154, and TIS-620.

- The UTF-8 and UTF-16 codecs now cope better with receiving partial input. Previously the `StreamReader` class would try to read more data, making it impossible to resume decoding from the stream. The `read()` method will now return as much data as it can and future calls will resume decoding where previous ones left off. (Implemented by Walter Dörwald.)

- There is a new `collections` module for various specialized collection datatypes. Currently it contains just one type, `deque`, a double-ended queue that supports efficiently adding and removing elements from either end:

```
>>> from collections import deque
>>> d = deque('ghi')        # make a new deque with three it
>>> d.append('j')           # add a new entry to the right s
>>> d.appendleft('f')       # add a new entry to the left si
>>> d                       # show the representation of the
deque(['f', 'g', 'h', 'i', 'j'])
>>> d.pop()                 # return and remove the rightmos
'j'
>>> d.popleft()             # return and remove the leftmost
'f'
>>> list(d)                 # list the contents of the deque
['g', 'h', 'i']
>>> 'h' in d                # search the deque
True
```

  Several modules, such as the `queue` and `threading` modules, now take advantage of `collections.deque` for improved performance. (Contributed by Raymond Hettinger.)

- The `ConfigParser` classes have been enhanced slightly. The `read()` method now returns a list of the files that were successfully parsed, and the `set()` method raises `TypeError` if passed a *value* argument that isn't a string. (Contributed by John Belmonte and David Goodger.)

- The `curses` module now supports the ncurses extension `use_default_colors()`. On platforms where the terminal supports

transparency, this makes it possible to use a transparent background. (Contributed by Jörg Lehmann.)

- The **difflib** module now includes an **HtmlDiff** class that creates an HTML table showing a side by side comparison of two versions of a text. (Contributed by Dan Gass.)

- The **email** package was updated to version 3.0, which dropped various deprecated APIs and removes support for Python versions earlier than 2.3. The 3.0 version of the package uses a new incremental parser for MIME messages, available in the **email.FeedParser** module. The new parser doesn't require reading the entire message into memory, and doesn't raise exceptions if a message is malformed; instead it records any problems in the **defect** attribute of the message. (Developed by Anthony Baxter, Barry Warsaw, Thomas Wouters, and others.)

- The **heapq** module has been converted to C. The resulting tenfold improvement in speed makes the module suitable for handling high volumes of data. In addition, the module has two new functions **nlargest()** and **nsmallest()** that use heaps to find the N largest or smallest values in a dataset without the expense of a full sort. (Contributed by Raymond Hettinger.)

- The **httplib** module now contains constants for HTTP status codes defined in various HTTP-related RFC documents. Constants have names such as **OK**, **CREATED**, **CONTINUE**, and **MOVED_PERMANENTLY**; use pydoc to get a full list. (Contributed by Andrew Eland.)

- The **imaplib** module now supports IMAP's THREAD command (contributed by Yves Dionne) and new **deleteacl()** and **myrights()** methods (contributed by Arnaud Mazin).

- The **itertools** module gained a `groupby(iterable[, *func*])` function. *iterable* is something that can be iterated over to return a stream of elements, and the optional *func* parameter is a function that takes an element and returns a key value; if omitted, the key is simply the element itself. **groupby()** then groups the elements into subsequences which have matching values of the key, and returns a

series of 2-tuples containing the key value and an iterator over the subsequence.

Here's an example to make this clearer. The *key* function simply returns whether a number is even or odd, so the result of **groupby()** is to return consecutive runs of odd or even numbers.

```
>>> import itertools
>>> L = [2, 4, 6, 7, 8, 9, 11, 12, 14]
>>> for key_val, it in itertools.groupby(L, lambda x: x % 2)
...     print key_val, list(it)
...
0 [2, 4, 6]
1 [7]
0 [8]
1 [9, 11]
0 [12, 14]
>>>
```

groupby() is typically used with sorted input. The logic for **groupby()** is similar to the Unix `uniq` filter which makes it handy for eliminating, counting, or identifying duplicate elements:

```
>>> word = 'abracadabra'
>>> letters = sorted(word)   # Turn string into a sorted lis
>>> letters
['a', 'a', 'a', 'a', 'a', 'b', 'b', 'c', 'd', 'r', 'r']
>>> for k, g in itertools.groupby(letters):
...     print k, list(g)
...
a ['a', 'a', 'a', 'a', 'a']
b ['b', 'b']
c ['c']
d ['d']
r ['r', 'r']
>>> # List unique letters
>>> [k for k, g in groupby(letters)]
['a', 'b', 'c', 'd', 'r']
>>> # Count letter occurrences
>>> [(k, len(list(g))) for k, g in groupby(letters)]
[('a', 5), ('b', 2), ('c', 1), ('d', 1), ('r', 2)]
```

(Contributed by Hye-Shik Chang.)

- **itertools** also gained a function named `tee(iterator, N)` that returns *N* independent iterators that replicate *iterator*. If *N* is omitted, the default is 2.

```
>>> L = [1,2,3]
>>> i1, i2 = itertools.tee(L)
>>> i1,i2
(<itertools.tee object at 0x402c2080>, <itertools.tee object
>>> list(i1)                # Run the first iterator to exhau
[1, 2, 3]
>>> list(i2)                # Run the second iterator to exha
[1, 2, 3]
```

  Note that **tee()** has to keep copies of the values returned by the iterator; in the worst case, it may need to keep all of them. This should therefore be used carefully if the leading iterator can run far ahead of the trailing iterator in a long stream of inputs. If the separation is large, then you might as well use **list()** instead. When the iterators track closely with one another, **tee()** is ideal. Possible applications include bookmarking, windowing, or lookahead iterators. (Contributed by Raymond Hettinger.)

- A number of functions were added to the **locale** module, such as **bind_textdomain_codeset()** to specify a particular encoding and a family of **l*gettext()** functions that return messages in the chosen encoding. (Contributed by Gustavo Niemeyer.)

- Some keyword arguments were added to the **logging** package's **basicConfig()** function to simplify log configuration. The default behavior is to log messages to standard error, but various keyword arguments can be specified to log to a particular file, change the logging format, or set the logging level. For example:

```
import logging
logging.basicConfig(filename='/var/log/application.log',
```

```
level=0,  # Log all messages
format='%(levelname):%(process):%(thread):%(message)')
```

Other additions to the **logging** package include a `log(level, msg)` convenience method, as well as a **TimedRotatingFileHandler** class that rotates its log files at a timed interval. The module already had **RotatingFileHandler**, which rotated logs once the file exceeded a certain size. Both classes derive from a new **BaseRotatingHandler** class that can be used to implement other rotating handlers.

(Changes implemented by Vinay Sajip.)

- The **marshal** module now shares interned strings on unpacking a data structure. This may shrink the size of certain pickle strings, but the primary effect is to make `.pyc` files significantly smaller. (Contributed by Martin von Löwis.)

- The **nntplib** module's **NNTP** class gained **description()** and **descriptions()** methods to retrieve newsgroup descriptions for a single group or for a range of groups. (Contributed by Jürgen A. Erhard.)

- Two new functions were added to the **operator** module, `attrgetter(attr)` and `itemgetter(index)`. Both functions return callables that take a single argument and return the corresponding attribute or item; these callables make excellent data extractors when used with **map()** or **sorted()**. For example:

```
>>> L = [('c', 2), ('d', 1), ('a', 4), ('b', 3)]
>>> map(operator.itemgetter(0), L)
['c', 'd', 'a', 'b']
>>> map(operator.itemgetter(1), L)
[2, 1, 4, 3]
>>> sorted(L, key=operator.itemgetter(1)) # Sort list by sec
[('d', 1), ('c', 2), ('b', 3), ('a', 4)]
```

(Contributed by Raymond Hettinger.)

- The **optparse** module was updated in various ways. The module now passes its messages through **gettext.gettext()**, making it possible to internationalize Optik's help and error messages. Help messages for options can now include the string `'%default'`, which will be replaced by the option's default value. (Contributed by Greg Ward.)

- The long-term plan is to deprecate the **rfc822** module in some future Python release in favor of the **email** package. To this end, the **email.Utils.formatdate** function has been changed to make it usable as a replacement for **rfc822.formatdate()**. You may want to write new e-mail processing code with this in mind. (Change implemented by Anthony Baxter.)

- A new `urandom(n)` function was added to the **os** module, returning a string containing *n* bytes of random data. This function provides access to platform-specific sources of randomness such as `/dev/urandom` on Linux or the Windows CryptoAPI. (Contributed by Trevor Perrin.)

- Another new function: `os.path.lexists(path)` returns true if the file specified by *path* exists, whether or not it's a symbolic link. This differs from the existing `os.path.exists(path)` function, which returns false if *path* is a symlink that points to a destination that doesn't exist. (Contributed by Beni Cherniavsky.)

- A new **getsid()** function was added to the **posix** module that underlies the **os** module. (Contributed by J. Raynor.)

- The **poplib** module now supports POP over SSL. (Contributed by Hector Urtubia.)

- The **profile** module can now profile C extension functions. (Contributed by Nick Bastin.)

- The **random** module has a new method called `getrandbits(N)` that returns a long integer *N* bits in length. The existing **randrange()** method now uses **getrandbits()** where appropriate, making generation of arbitrarily large random numbers more efficient. (Contributed by Raymond Hettinger.)

- The regular expression language accepted by the `re` module was extended with simple conditional expressions, written as `(?(group)A|B)`. *group* is either a numeric group ID or a group name defined with `(?P<group>...)` earlier in the expression. If the specified group matched, the regular expression pattern *A* will be tested against the string; if the group didn't match, the pattern *B* will be used instead. (Contributed by Gustavo Niemeyer.)

- The `re` module is also no longer recursive, thanks to a massive amount of work by Gustavo Niemeyer. In a recursive regular expression engine, certain patterns result in a large amount of C stack space being consumed, and it was possible to overflow the stack. For example, if you matched a 30000-byte string of `a` characters against the expression `(a|b)+`, one stack frame was consumed per character. Python 2.3 tried to check for stack overflow and raise a `RuntimeError` exception, but certain patterns could sidestep the checking and if you were unlucky Python could segfault. Python 2.4's regular expression engine can match this pattern without problems.

- The `signal` module now performs tighter error-checking on the parameters to the `signal.signal()` function. For example, you can't set a handler on the `SIGKILL` signal; previous versions of Python would quietly accept this, but 2.4 will raise a `RuntimeError` exception.

- Two new functions were added to the `socket` module. `socketpair()` returns a pair of connected sockets and `getservbyport(port)` looks up the service name for a given port number. (Contributed by Dave Cole and Barry Warsaw.)

- The **`sys.exitfunc()`** function has been deprecated. Code should be using the existing `atexit` module, which correctly handles calling multiple exit functions. Eventually **`sys.exitfunc()`** will become a purely internal interface, accessed only by **`atexit`**.

- The `tarfile` module now generates GNU-format tar files by default. (Contributed by Lars Gustäbel.)

- The `threading` module now has an elegantly simple way to support thread-local data. The module contains a `local` class whose attribute values are local to different threads.

  ```
  import threading

  data = threading.local()
  data.number = 42
  data.url = ('www.python.org', 80)
  ```

  Other threads can assign and retrieve their own values for the `number` and `url` attributes. You can subclass `local` to initialize attributes or to add methods. (Contributed by Jim Fulton.)

- The `timeit` module now automatically disables periodic garbage collection during the timing loop. This change makes consecutive timings more comparable. (Contributed by Raymond Hettinger.)

- The `weakref` module now supports a wider variety of objects including Python functions, class instances, sets, frozensets, deques, arrays, files, sockets, and regular expression pattern objects. (Contributed by Raymond Hettinger.)

- The `xmlrpclib` module now supports a multi-call extension for transmitting multiple XML-RPC calls in a single HTTP operation. (Contributed by Brian Quinlan.)

- The `mpz`, `rotor`, and `xreadlines` modules have been removed.

cookielib

The `cookielib` library supports client-side handling for HTTP cookies, mirroring the `Cookie` module's server-side cookie support. Cookies are stored in cookie jars; the library transparently stores cookies offered by the web server in the cookie jar, and fetches the cookie from the jar when connecting to the server. As in web browsers, policy objects control whether cookies are accepted or not.

In order to store cookies across sessions, two implementations of cookie jars are provided: one that stores cookies in the Netscape format so applications can use the Mozilla or Lynx cookie files, and one that stores cookies in the same format as the Perl libwww library.

`urllib2` has been changed to interact with `cookielib`: `HTTPCookieProcessor` manages a cookie jar that is used when accessing URLs.

This module was contributed by John J. Lee.

doctest

The `doctest` module underwent considerable refactoring thanks to Edward Loper and Tim Peters. Testing can still be as simple as running `doctest.testmod()`, but the refactorings allow customizing the module's operation in various ways

The new `DocTestFinder` class extracts the tests from a given object's docstrings:

```
def f (x, y):
    """>>> f(2,2)
4
>>> f(3,2)
6
    """
    return x*y

finder = doctest.DocTestFinder()

# Get list of DocTest instances
tests = finder.find(f)
```

The new `DocTestRunner` class then runs individual tests and can produce a summary of the results:

```
runner = doctest.DocTestRunner()
for t in tests:
    tried, failed = runner.run(t)
```

```
runner.summarize(verbose=1)
```

The above example produces the following output:

```
1 items passed all tests:
   2 tests in f
2 tests in 1 items.
2 passed and 0 failed.
Test passed.
```

DocTestRunner uses an instance of the **OutputChecker** class to compare the expected output with the actual output. This class takes a number of different flags that customize its behaviour; ambitious users can also write a completely new subclass of **OutputChecker**.

The default output checker provides a number of handy features. For example, with the **doctest.ELLIPSIS** option flag, an ellipsis (. . .) in the expected output matches any substring, making it easier to accommodate outputs that vary in minor ways:

```
def o (n):
    """>>> o(1)
<__main__.C instance at 0x...>
>>>
"""
```

Another special string, `<BLANKLINE>`, matches a blank line:

```
def p (n):
    """>>> p(1)
<BLANKLINE>
>>>
"""
```

Another new capability is producing a diff-style display of the output by specifying the **doctest.REPORT_UDIFF** (unified diffs), **doctest.REPORT_CDIFF** (context diffs), or **doctest.REPORT_NDIFF** (delta-style) option flags. For example:

```
def g (n):
    """>>> g(4)
here
is
a
lengthy
>>>"""
    L = 'here is a rather lengthy list of words'.split()
    for word in L[:n]:
        print word
```

Running the above function's tests with `doctest.REPORT_UDIFF` specified, you get the following output:

```
**********************************************************************
File "t.py", line 15, in g
Failed example:
    g(4)
Differences (unified diff with -expected +actual):
    @@ -2,3 +2,3 @@
     is
     a
    -lengthy
    +rather
**********************************************************************
```

Mudanças a construções e API C

Some of the changes to Python's build process and to the C API are:

- Three new convenience macros were added for common return values from extension functions: `Py_RETURN_NONE`, `Py_RETURN_TRUE`, and `Py_RETURN_FALSE`. (Contributed by Brett Cannon.)
- Another new macro, `Py_CLEAR`, decreases the reference count of *obj* and sets *obj* to the null pointer. (Contributed by Jim Fulton.)
- A new function, `PyTuple_Pack(N, obj1, obj2, ..., objN)`, constructs tuples from a variable length argument list of Python objects. (Contributed by Raymond Hettinger.)

- A new function, `PyDict_Contains(d, k)`, implements fast dictionary lookups without masking exceptions raised during the look-up process. (Contributed by Raymond Hettinger.)
- The `Py_IS_NAN` macro returns 1 if its float or double argument *X* is a NaN. (Contributed by Tim Peters.)
- C code can avoid unnecessary locking by using the new **`PyEval_ThreadsInitialized()`** function to tell if any thread operations have been performed. If this function returns false, no lock operations are needed. (Contributed by Nick Coghlan.)
- A new function, `PyArg_VaParseTupleAndKeywords()`, is the same as `PyArg_ParseTupleAndKeywords()` but takes a **`va_list`** instead of a number of arguments. (Contributed by Greg Chapman.)
- A new method flag, `METH_COEXIST`, allows a function defined in slots to co-exist with a `PyCFunction` having the same name. This can halve the access time for a method such as **`set.__contains__()`**. (Contributed by Raymond Hettinger.)
- Python can now be built with additional profiling for the interpreter itself, intended as an aid to people developing the Python core. Providing **`--enable-profiling`** to the **configure** script will let you profile the interpreter with **gprof**, and providing the **`--with-tsc`** switch enables profiling using the Pentium's Time-Stamp-Counter register. Note that the **`--with-tsc`** switch is slightly misnamed, because the profiling feature also works on the PowerPC platform, though that processor architecture doesn't call that register "the TSC register". (Contributed by Jeremy Hylton.)
- The **`tracebackobject`** type has been renamed to **`PyTracebackObject`**.

Port-Specific Changes

- The Windows port now builds under MSVC++ 7.1 as well as version 6. (Contributed by Martin von Löwis.)

Porting to Python 2.4

This section lists previously described changes that may require changes to your code:

- Left shifts and hexadecimal/octal constants that are too large no longer trigger a `FutureWarning` and return a value limited to 32 or 64 bits; instead they return a long integer.
- Integer operations will no longer trigger an `OverflowWarning`. The `OverflowWarning` warning will disappear in Python 2.5.
- The `zip()` built-in function and `itertools.izip()` now return an empty list instead of raising a `TypeError` exception if called with no arguments.
- You can no longer compare the `date` and `datetime` instances provided by the `datetime` module. Two instances of different classes will now always be unequal, and relative comparisons (<, >) will raise a `TypeError`.
- `dircache.listdir()` now passes exceptions to the caller instead of returning empty lists.
- `LexicalHandler.startDTD` used to receive the public and system IDs in the wrong order. This has been corrected; applications relying on the wrong order need to be fixed.
- `fcntl.ioctl()` now warns if the *mutate* argument is omitted and relevant.
- The `tarfile` module now generates GNU-format tar files by default.
- Encountering a failure while importing a module no longer leaves a partially initialized module object in `sys.modules`.
- `None` is now a constant; code that binds a new value to the name `None` is now a syntax error.
- The `signal.signal()` function now raises a `RuntimeError` exception for certain illegal values; previously these errors would pass silently. For example, you can no longer set a handler on the `SIGKILL` signal.

Reconhecimentos

The author would like to thank the following people for offering suggestions, corrections and assistance with various drafts of this article: Koray Can, Hye-Shik Chang, Michael Dyck, Raymond Hettinger, Brian Hurt, Hamish Lawson, Fredrik Lundh, Sean Reifschneider, Sadruddin Rejeb.

What’s New in Python 2.3

Autor:: A.M. Kuchling

This article explains the new features in Python 2.3. Python 2.3 was released on July 29, 2003.

The main themes for Python 2.3 are polishing some of the features added in 2.2, adding various small but useful enhancements to the core language, and expanding the standard library. The new object model introduced in the previous version has benefited from 18 months of bugfixes and from optimization efforts that have improved the performance of new-style classes. A few new built-in functions have been added such as `sum()` and `enumerate()`. The `in` operator can now be used for substring searches (e.g. `"ab" in "abc"` returns `True`).

Some of the many new library features include Boolean, set, heap, and date/time data types, the ability to import modules from ZIP-format archives, metadata support for the long-awaited Python catalog, an updated version of IDLE, and modules for logging messages, wrapping text, parsing CSV files, processing command-line options, using BerkeleyDB databases… the list of new and enhanced modules is lengthy.

This article doesn’t attempt to provide a complete specification of the new features, but instead provides a convenient overview. For full details, you should refer to the documentation for Python 2.3, such as the Python Library Reference and the Python Reference Manual. If you want to understand the complete implementation and design rationale, refer to the PEP for a particular new feature.

PEP 218: A Standard Set Datatype

The new `sets` module contains an implementation of a set datatype. The `Set` class is for mutable sets, sets that can have members added and removed. The `ImmutableSet` class is for sets that can’t be modified, and

instances of **`ImmutableSet`** can therefore be used as dictionary keys. Sets are built on top of dictionaries, so the elements within a set must be hashable.

Here's a simple example:

```
>>> import sets
>>> S = sets.Set([1,2,3])
>>> S
Set([1, 2, 3])
>>> 1 in S
True
>>> 0 in S
False
>>> S.add(5)
>>> S.remove(3)
>>> S
Set([1, 2, 5])
>>>
```

The union and intersection of sets can be computed with the **`union()`** and **`intersection()`** methods; an alternative notation uses the bitwise operators `&` and `|`. Mutable sets also have in-place versions of these methods, **`union_update()`** and **`intersection_update()`**.

```
>>> S1 = sets.Set([1,2,3])
>>> S2 = sets.Set([4,5,6])
>>> S1.union(S2)
Set([1, 2, 3, 4, 5, 6])
>>> S1 | S2                  # Alternative notation
Set([1, 2, 3, 4, 5, 6])
>>> S1.intersection(S2)
Set([])
>>> S1 & S2                  # Alternative notation
Set([])
>>> S1.union_update(S2)
>>> S1
Set([1, 2, 3, 4, 5, 6])
>>>
```

It's also possible to take the symmetric difference of two sets. This is the set of all elements in the union that aren't in the intersection. Another way of putting it is that the symmetric difference contains all elements that are in exactly one set. Again, there's an alternative notation (^), and an in-place version with the ungainly name **`symmetric_difference_update()`**.

```
>>> S1 = sets.Set([1,2,3,4])
>>> S2 = sets.Set([3,4,5,6])
>>> S1.symmetric_difference(S2)
Set([1, 2, 5, 6])
>>> S1 ^ S2
Set([1, 2, 5, 6])
>>>
```

There are also **`issubset()`** and **`issuperset()`** methods for checking whether one set is a subset or superset of another:

```
>>> S1 = sets.Set([1,2,3])
>>> S2 = sets.Set([2,3])
>>> S2.issubset(S1)
True
>>> S1.issubset(S2)
False
>>> S1.issuperset(S2)
True
>>>
```

Ver também

PEP 218 [https://peps.python.org/pep-0218/] - Adding a Built-In Set Object Type

PEP written by Greg V. Wilson. Implemented by Greg V. Wilson, Alex Martelli, and GvR.

PEP 255: Simple Generators

In Python 2.2, generators were added as an optional feature, to be enabled by a `from __future__ import generators` directive. In 2.3 generators no

longer need to be specially enabled, and are now always present; this means that **yield** is now always a keyword. The rest of this section is a copy of the description of generators from the "What's New in Python 2.2" document; if you read it back when Python 2.2 came out, you can skip the rest of this section.

You're doubtless familiar with how function calls work in Python or C. When you call a function, it gets a private namespace where its local variables are created. When the function reaches a **return** statement, the local variables are destroyed and the resulting value is returned to the caller. A later call to the same function will get a fresh new set of local variables. But, what if the local variables weren't thrown away on exiting a function? What if you could later resume the function where it left off? This is what generators provide; they can be thought of as resumable functions.

Here's the simplest example of a generator function:

```
def generate_ints(N):
    for i in range(N):
        yield i
```

A new keyword, **yield**, was introduced for generators. Any function containing a **yield** statement is a generator function; this is detected by Python's bytecode compiler which compiles the function specially as a result.

When you call a generator function, it doesn't return a single value; instead it returns a generator object that supports the iterator protocol. On executing the **yield** statement, the generator outputs the value of `i`, similar to a **return** statement. The big difference between **yield** and a **return** statement is that on reaching a **yield** the generator's state of execution is suspended and local variables are preserved. On the next call to the generator's `.next()` method, the function will resume executing immediately after the **yield** statement. (For complicated reasons, the **yield** statement isn't allowed inside the **try** block of a **try**…**finally** statement; read **PEP 255** [https://peps.python.org/pep-0255/] for a full explanation of the interaction between **yield** and exceptions.)

Here's a sample usage of the **`generate_ints()`** generator:

```
>>> gen = generate_ints(3)
>>> gen
<generator object at 0x8117f90>
>>> gen.next()
0
>>> gen.next()
1
>>> gen.next()
2
>>> gen.next()
Traceback (most recent call last):
  File "stdin", line 1, in ?
  File "stdin", line 2, in generate_ints
StopIteration
```

You could equally write `for i in generate_ints(5)`, or `a,b,c = generate_ints(3)`.

Inside a generator function, the **`return`** statement can only be used without a value, and signals the end of the procession of values; afterwards the generator cannot return any further values. **`return`** with a value, such as `return 5`, is a syntax error inside a generator function. The end of the generator's results can also be indicated by raising **`StopIteration`** manually, or by just letting the flow of execution fall off the bottom of the function.

You could achieve the effect of generators manually by writing your own class and storing all the local variables of the generator as instance variables. For example, returning a list of integers could be done by setting `self.count` to 0, and having the **`next()`** method increment `self.count` and return it. However, for a moderately complicated generator, writing a corresponding class would be much messier. `Lib/test/test_generators.py` contains a number of more interesting examples. The simplest one implements an in-order traversal of a tree using generators recursively.

```
# A recursive generator that generates Tree leaves in in-order.
def inorder(t):
```

```
    if t:
        for x in inorder(t.left):
            yield x
        yield t.label
        for x in inorder(t.right):
            yield x
```

Two other examples in `Lib/test/test_generators.py` produce solutions for the N-Queens problem (placing N queens on an NxN chess board so that no queen threatens another) and the Knight's Tour (a route that takes a knight to every square of an NxN chessboard without visiting any square twice).

The idea of generators comes from other programming languages, especially Icon (https://www2.cs.arizona.edu/icon/), where the idea of generators is central. In Icon, every expression and function call behaves like a generator. One example from "An Overview of the Icon Programming Language" at https://www2.cs.arizona.edu/icon/docs/ipd266.htm gives an idea of what this looks like:

```
sentence := "Store it in the neighboring harbor"
if (i := find("or", sentence)) > 5 then write(i)
```

In Icon the **`find()`** function returns the indexes at which the substring "or" is found: 3, 23, 33. In the **`if`** statement, `i` is first assigned a value of 3, but 3 is less than 5, so the comparison fails, and Icon retries it with the second value of 23. 23 is greater than 5, so the comparison now succeeds, and the code prints the value 23 to the screen.

Python doesn't go nearly as far as Icon in adopting generators as a central concept. Generators are considered part of the core Python language, but learning or using them isn't compulsory; if they don't solve any problems that you have, feel free to ignore them. One novel feature of Python's interface as compared to Icon's is that a generator's state is represented as a concrete object (the iterator) that can be passed around to other functions or stored in a data structure.

Ver também

PEP 255 **[https://peps.python.org/pep-0255/]** **- Simple Generators**

Written by Neil Schemenauer, Tim Peters, Magnus Lie Hetland. Implemented mostly by Neil Schemenauer and Tim Peters, with other fixes from the Python Labs crew.

PEP 263: Source Code Encodings

Python source files can now be declared as being in different character set encodings. Encodings are declared by including a specially formatted comment in the first or second line of the source file. For example, a UTF-8 file can be declared with:

```
#!/usr/bin/env python
# -*- coding: UTF-8 -*-
```

Without such an encoding declaration, the default encoding used is 7-bit ASCII. Executing or importing modules that contain string literals with 8-bit characters and have no encoding declaration will result in a `DeprecationWarning` being signalled by Python 2.3; in 2.4 this will be a syntax error.

The encoding declaration only affects Unicode string literals, which will be converted to Unicode using the specified encoding. Note that Python identifiers are still restricted to ASCII characters, so you can’t have variable names that use characters outside of the usual alphanumerics.

Ver também

PEP 263 **[https://peps.python.org/pep-0263/]** **- Defining Python Source Code Encodings**

Written by Marc-André Lemburg and Martin von Löwis; implemented by Suzuki Hisao and Martin von Löwis.

PEP 273: Importing Modules from ZIP Archives

The new **zipimport** module adds support for importing modules from a ZIP-format archive. You don't need to import the module explicitly; it will be automatically imported if a ZIP archive's filename is added to `sys.path`. For example:

```
amk@nyman:~/src/python$ unzip -l /tmp/example.zip
Archive:  /tmp/example.zip
  Length     Date   Time    Name
 --------    ----   ----    ----
     8467  11-26-02 22:30   jwzthreading.py
 --------                   -------
     8467                   1 file
amk@nyman:~/src/python$ ./python
Python 2.3 (#1, Aug 1 2003, 19:54:32)
>>> import sys
>>> sys.path.insert(0, '/tmp/example.zip')  # Add .zip file to f
>>> import jwzthreading
>>> jwzthreading.__file__
'/tmp/example.zip/jwzthreading.py'
>>>
```

An entry in `sys.path` can now be the filename of a ZIP archive. The ZIP archive can contain any kind of files, but only files named `*.py`, `*.pyc`, or `*.pyo` can be imported. If an archive only contains `*.py` files, Python will not attempt to modify the archive by adding the corresponding `*.pyc` file, meaning that if a ZIP archive doesn't contain `*.pyc` files, importing may be rather slow.

A path within the archive can also be specified to only import from a subdirectory; for example, the path `/tmp/example.zip/lib/` would only import from the `lib/` subdirectory within the archive.

Ver também

PEP 273 [https://peps.python.org/pep-0273/] - Import Modules from Zip Archives

Written by James C. Ahlstrom, who also provided an implementation. Python 2.3 follows the specification in **PEP 273** [https://peps.python.org/pep-0273/], but uses an implementation written by Just van Rossum that

uses the import hooks described in **PEP 302** [https://peps.python.org/pep-0302/]. See section PEP 302: New Import Hooks for a description of the new import hooks.

PEP 277: Unicode file name support for Windows NT

On Windows NT, 2000, and XP, the system stores file names as Unicode strings. Traditionally, Python has represented file names as byte strings, which is inadequate because it renders some file names inaccessible.

Python now allows using arbitrary Unicode strings (within the limitations of the file system) for all functions that expect file names, most notably the `open()` built-in function. If a Unicode string is passed to `os.listdir()`, Python now returns a list of Unicode strings. A new function, `os.getcwdu()`, returns the current directory as a Unicode string.

Byte strings still work as file names, and on Windows Python will transparently convert them to Unicode using the `mbcs` encoding.

Other systems also allow Unicode strings as file names but convert them to byte strings before passing them to the system, which can cause a `UnicodeError` to be raised. Applications can test whether arbitrary Unicode strings are supported as file names by checking `os.path.supports_unicode_filenames`, a Boolean value.

Under MacOS, `os.listdir()` may now return Unicode filenames.

Ver também

PEP 277 [https://peps.python.org/pep-0277/] - Unicode file name support for Windows NT

Written by Neil Hodgson; implemented by Neil Hodgson, Martin von Löwis, and Mark Hammond.

PEP 278: Universal Newline Support

The three major operating systems used today are Microsoft Windows, Apple's Macintosh OS, and the various Unix derivatives. A minor irritation of cross-platform work is that these three platforms all use different characters to mark the ends of lines in text files. Unix uses the linefeed (ASCII character 10), MacOS uses the carriage return (ASCII character 13), and Windows uses a two-character sequence of a carriage return plus a newline.

Python's file objects can now support end of line conventions other than the one followed by the platform on which Python is running. Opening a file with the mode `'U'` or `'rU'` will open a file for reading in universal newlines mode. All three line ending conventions will be translated to a `'\n'` in the strings returned by the various file methods such as **`read()`** and **`readline()`**.

Universal newline support is also used when importing modules and when executing a file with the **`execfile()`** function. This means that Python modules can be shared between all three operating systems without needing to convert the line-endings.

This feature can be disabled when compiling Python by specifying the **`--without-universal-newlines`** switch when running Python's **configure** script.

Ver também

PEP 278 **[https://peps.python.org/pep-0278/]** - **Universal Newline Support**
Written and implemented by Jack Jansen.

PEP 279: enumerate()

A new built-in function, **`enumerate()`**, will make certain loops a bit clearer. `enumerate(thing)`, where *thing* is either an iterator or a sequence, returns

an iterator that will return `(0, thing[0])`, `(1, thing[1])`, `(2, thing[2])`, and so forth.

A common idiom to change every element of a list looks like this:

```
for i in range(len(L)):
    item = L[i]
    # ... compute some result based on item ...
    L[i] = result
```

This can be rewritten using `enumerate()` as:

```
for i, item in enumerate(L):
    # ... compute some result based on item ...
    L[i] = result
```

Ver também

PEP 279 [https://peps.python.org/pep-0279/] - The enumerate() built-in function

Written and implemented by Raymond D. Hettinger.

PEP 282: The logging Package

A standard package for writing logs, `logging`, has been added to Python 2.3. It provides a powerful and flexible mechanism for generating logging output which can then be filtered and processed in various ways. A configuration file written in a standard format can be used to control the logging behavior of a program. Python includes handlers that will write log records to standard error or to a file or socket, send them to the system log, or even e-mail them to a particular address; of course, it's also possible to write your own handler classes.

The `Logger` class is the primary class. Most application code will deal with one or more `Logger` objects, each one used by a particular subsystem of the application. Each `Logger` is identified by a name, and names are organized into a hierarchy using `.` as the component separator. For example, you

might have **`Logger`** instances named `server`, `server.auth` and `server.network`. The latter two instances are below `server` in the hierarchy. This means that if you turn up the verbosity for `server` or direct `server` messages to a different handler, the changes will also apply to records logged to `server.auth` and `server.network`. There's also a root **`Logger`** that's the parent of all other loggers.

For simple uses, the **`logging`** package contains some convenience functions that always use the root log:

```
import logging

logging.debug('Debugging information')
logging.info('Informational message')
logging.warning('Warning:config file %s not found', 'server.conf
logging.error('Error occurred')
logging.critical('Critical error -- shutting down')
```

This produces the following output:

```
WARNING:root:Alerta:arquivo de configuração server.conf não encc
ERROR:root:Ocorreu um erro
CRITICAL:root:Erro crítico -- shutting down
```

In the default configuration, informational and debugging messages are suppressed and the output is sent to standard error. You can enable the display of informational and debugging messages by calling the **`setLevel()`** method on the root logger.

Notice the **`warning()`** call's use of string formatting operators; all of the functions for logging messages take the arguments `(msg, arg1, arg2, ...)` and log the string resulting from `msg % (arg1, arg2, ...)`.

There's also an **`exception()`** function that records the most recent traceback. Any of the other functions will also record the traceback if you specify a true value for the keyword argument *exc_info*.

```
def f():
    try:    1/0
    except: logging.exception('Problem recorded')

f()
```

This produces the following output:

```
ERROR:root:Problem recorded
Traceback (most recent call last):
  File "t.py", line 6, in f
    1/0
ZeroDivisionError: integer division or modulo by zero
```

Slightly more advanced programs will use a logger other than the root logger. The `getLogger(name)` function is used to get a particular log, creating it if it doesn't exist yet. `getLogger(None)` returns the root logger.

```
log = logging.getLogger('server')
 ...
log.info('Listening on port %i', port)
 ...
log.critical('Disk full')
 ...
```

Log records are usually propagated up the hierarchy, so a message logged to `server.auth` is also seen by `server` and `root`, but a **`Logger`** can prevent this by setting its **`propagate`** attribute to **`False`**.

There are more classes provided by the **`logging`** package that can be customized. When a **`Logger`** instance is told to log a message, it creates a **`LogRecord`** instance that is sent to any number of different **`Handler`** instances. Loggers and handlers can also have an attached list of filters, and each filter can cause the **`LogRecord`** to be ignored or can modify the record before passing it along. When they're finally output, **`LogRecord`** instances are converted to text by a **`Formatter`** class. All of these classes can be replaced by your own specially written classes.

With all of these features the **`logging`** package should provide enough flexibility for even the most complicated applications. This is only an

incomplete overview of its features, so please see the package's reference documentation for all of the details. Reading **PEP 282** [https://peps.python.org/pep-0282/] will also be helpful.

Ver também

PEP 282 [https://peps.python.org/pep-0282/] - A Logging System
Written by Vinay Sajip and Trent Mick; implemented by Vinay Sajip.

PEP 285: A Boolean Type

A Boolean type was added to Python 2.3. Two new constants were added to the `__builtin__` module, `True` and `False`. (`True` and `False` constants were added to the built-ins in Python 2.2.1, but the 2.2.1 versions are simply set to integer values of 1 and 0 and aren't a different type.)

The type object for this new type is named `bool`; the constructor for it takes any Python value and converts it to `True` or `False`.

```
>>> bool(1)
True
>>> bool(0)
False
>>> bool([])
False
>>> bool( (1,) )
True
```

Most of the standard library modules and built-in functions have been changed to return Booleans.

```
>>> obj = []
>>> hasattr(obj, 'append')
True
>>> isinstance(obj, list)
True
>>> isinstance(obj, tuple)
False
```

Python's Booleans were added with the primary goal of making code clearer. For example, if you're reading a function and encounter the statement `return 1`, you might wonder whether the `1` represents a Boolean truth value, an index, or a coefficient that multiplies some other quantity. If the statement is `return True`, however, the meaning of the return value is quite clear.

Python's Booleans were *not* added for the sake of strict type-checking. A very strict language such as Pascal would also prevent you performing arithmetic with Booleans, and would require that the expression in an `if` statement always evaluate to a Boolean result. Python is not this strict and never will be, as **PEP 285** [https://peps.python.org/pep-0285/] explicitly says. This means you can still use any expression in an `if` statement, even ones that evaluate to a list or tuple or some random object. The Boolean type is a subclass of the `int` class so that arithmetic using a Boolean still works.

```
>>> True + 1
2
>>> False + 1
1
>>> False * 75
0
>>> True * 75
75
```

To sum up `True` and `False` in a sentence: they're alternative ways to spell the integer values 1 and 0, with the single difference that `str()` and `repr()` return the strings `'True'` and `'False'` instead of `'1'` and `'0'`.

Ver também

PEP 285 **[https://peps.python.org/pep-0285/]** **- Adding a bool type**
Written and implemented by GvR.

PEP 293: Codec Error Handling Callbacks

When encoding a Unicode string into a byte string, unencodable characters may be encountered. So far, Python has allowed specifying the error processing as either “strict” (raising `UnicodeError`), “ignore” (skipping the character), or “replace” (using a question mark in the output string), with “strict” being the default behavior. It may be desirable to specify alternative processing of such errors, such as inserting an XML character reference or HTML entity reference into the converted string.

Python now has a flexible framework to add different processing strategies. New error handlers can be added with `codecs.register_error()`, and codecs then can access the error handler with `codecs.lookup_error()`. An equivalent C API has been added for codecs written in C. The error handler gets the necessary state information such as the string being converted, the position in the string where the error was detected, and the target encoding. The handler can then either raise an exception or return a replacement string.

Two additional error handlers have been implemented using this framework: “backslashreplace” uses Python backslash quoting to represent unencodable characters and “xmlcharrefreplace” emits XML character references.

Ver também

PEP 293 [https://peps.python.org/pep-0293/] - Codec Error Handling Callbacks

Written and implemented by Walter Dörwald.

PEP 301: Package Index and Metadata for Distutils

Support for the long-requested Python catalog makes its first appearance in 2.3.

The heart of the catalog is the new Distutils **register** command. Running `python setup.py register` will collect the metadata describing a package, such as its name, version, maintainer, description, &c., and send it to a central catalog server. The resulting catalog is available from https://pypi.org.

To make the catalog a bit more useful, a new optional *classifiers* keyword argument has been added to the Distutils **`setup()`** function. A list of Trove [http://catb.org/~esr/trove/]-style strings can be supplied to help classify the software.

Here's an example `setup.py` with classifiers, written to be compatible with older versions of the Distutils:

```
from distutils import core
kw = {'name': "Quixote",
      'version': "0.5.1",
      'description': "A highly Pythonic Web application framewor
      # ...
      }

if (hasattr(core, 'setup_keywords') and
    'classifiers' in core.setup_keywords):
    kw['classifiers'] = \
        ['Topic :: Internet :: WWW/HTTP :: Dynamic Content',
         'Environment :: No Input/Output (Daemon)',
         'Intended Audience :: Developers'],

core.setup(**kw)
```

The full list of classifiers can be obtained by running `python setup.py register --list-classifiers`.

Ver também

PEP 301 [https://peps.python.org/pep-0301/] - Package Index and Metadata for Distutils

Written and implemented by Richard Jones.

PEP 302: New Import Hooks

While it's been possible to write custom import hooks ever since the **`ihooks`** module was introduced in Python 1.3, no one has ever been really happy with it because writing new import hooks is difficult and messy. There have been various proposed alternatives such as the **`imputil`** and **`iu`** modules, but none of them has ever gained much acceptance, and none of them were easily usable from C code.

PEP 302 [https://peps.python.org/pep-0302/] borrows ideas from its predecessors, especially from Gordon McMillan's **`iu`** module. Three new items are added to the **`sys`** module:

- `sys.path_hooks` is a list of callable objects; most often they'll be classes. Each callable takes a string containing a path and either returns an importer object that will handle imports from this path or raises an **`ImportError`** exception if it can't handle this path.
- `sys.path_importer_cache` caches importer objects for each path, so `sys.path_hooks` will only need to be traversed once for each path.
- `sys.meta_path` is a list of importer objects that will be traversed before `sys.path` is checked. This list is initially empty, but user code can add objects to it. Additional built-in and frozen modules can be imported by an object added to this list.

Importer objects must have a single method, `find_module(fullname, path=None)`. *fullname* will be a module or package name, e.g. `string` or `distutils.core`. **`find_module()`** must return a loader object that has a single method, `load_module(fullname)`, that creates and returns the corresponding module object.

Pseudo-code for Python's new import logic, therefore, looks something like this (simplified a bit; see **PEP 302** [https://peps.python.org/pep-0302/] for the full details):

```
for mp in sys.meta_path:
    loader = mp(fullname)
    if loader is not None:
        <module> = loader.load_module(fullname)
```

```
for path in sys.path:
    for hook in sys.path_hooks:
        try:
            importer = hook(path)
        except ImportError:
            # ImportError, so try the other path hooks
            pass
        else:
            loader = importer.find_module(fullname)
            <module> = loader.load_module(fullname)

# Not found!
raise ImportError
```

Ver também

PEP 302 **[https://peps.python.org/pep-0302/]** **- New Import Hooks**
Written by Just van Rossum and Paul Moore. Implemented by Just van Rossum.

PEP 305: Comma-separated Files

Comma-separated files are a format frequently used for exporting data from databases and spreadsheets. Python 2.3 adds a parser for comma-separated files.

Comma-separated format is deceptively simple at first glance:

```
Costs,150,200,3.95
```

Read a line and call `line.split(',')`: what could be simpler? But toss in string data that can contain commas, and things get more complicated:

```
"Costs",150,200,3.95,"Includes taxes, shipping, and sundry items
```

A big ugly regular expression can parse this, but using the new **`csv`** package is much simpler:

```
import csv

input = open('datafile', 'rb')
reader = csv.reader(input)
for line in reader:
    print line
```

The `reader()` function takes a number of different options. The field separator isn't limited to the comma and can be changed to any character, and so can the quoting and line-ending characters.

Different dialects of comma-separated files can be defined and registered; currently there are two dialects, both used by Microsoft Excel. A separate `csv.writer` class will generate comma-separated files from a succession of tuples or lists, quoting strings that contain the delimiter.

Ver também

PEP 305 **[https://peps.python.org/pep-0305/]** **- CSV File API**
Written and implemented by Kevin Altis, Dave Cole, Andrew McNamara, Skip Montanaro, Cliff Wells.

PEP 307: Pickle Enhancements

The `pickle` and `cPickle` modules received some attention during the 2.3 development cycle. In 2.2, new-style classes could be pickled without difficulty, but they weren't pickled very compactly; **PEP 307** [https://peps.python.org/pep-0307/] quotes a trivial example where a new-style class results in a pickled string three times longer than that for a classic class.

The solution was to invent a new pickle protocol. The `pickle.dumps()` function has supported a text-or-binary flag for a long time. In 2.3, this flag is redefined from a Boolean to an integer: 0 is the old text-mode pickle format, 1 is the old binary format, and now 2 is a new 2.3-specific format. A new constant, `pickle.HIGHEST_PROTOCOL`, can be used to select the fanciest protocol available.

Unpickling is no longer considered a safe operation. 2.2's `pickle` provided hooks for trying to prevent unsafe classes from being unpickled (specifically, a `__safe_for_unpickling__` attribute), but none of this code was ever audited and therefore it's all been ripped out in 2.3. You should not unpickle untrusted data in any version of Python.

To reduce the pickling overhead for new-style classes, a new interface for customizing pickling was added using three special methods: `__getstate__()`, `__setstate__()`, and `__getnewargs__()`. Consult **PEP 307** [https://peps.python.org/pep-0307/] for the full semantics of these methods.

As a way to compress pickles yet further, it's now possible to use integer codes instead of long strings to identify pickled classes. The Python Software Foundation will maintain a list of standardized codes; there's also a range of codes for private use. Currently no codes have been specified.

Ver também

PEP 307 [https://peps.python.org/pep-0307/] - Extensions to the pickle protocol
Written and implemented by Guido van Rossum and Tim Peters.

Extended Slices

Ever since Python 1.4, the slicing syntax has supported an optional third "step" or "stride" argument. For example, these are all legal Python syntax: `L[1:10:2]`, `L[:-1:1]`, `L[::-1]`. This was added to Python at the request of the developers of Numerical Python, which uses the third argument extensively. However, Python's built-in list, tuple, and string sequence types have never supported this feature, raising a `TypeError` if you tried it. Michael Hudson contributed a patch to fix this shortcoming.

For example, you can now easily extract the elements of a list that have even indexes:

```
>>> L = range(10)
>>> L[::2]
```

```
[0, 2, 4, 6, 8]
```

Negative values also work to make a copy of the same list in reverse order:

```
>>> L[::-1]
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

This also works for tuples, arrays, and strings:

```
>>> s='abcd'
>>> s[::2]
'ac'
>>> s[::-1]
'dcba'
```

If you have a mutable sequence such as a list or an array you can assign to or delete an extended slice, but there are some differences between assignment to extended and regular slices. Assignment to a regular slice can be used to change the length of the sequence:

```
>>> a = range(3)
>>> a
[0, 1, 2]
>>> a[1:3] = [4, 5, 6]
>>> a
[0, 4, 5, 6]
```

Extended slices aren’t this flexible. When assigning to an extended slice, the list on the right hand side of the statement must contain the same number of items as the slice it is replacing:

```
>>> a = range(4)
>>> a
[0, 1, 2, 3]
>>> a[::2]
[0, 2]
>>> a[::2] = [0, -1]
>>> a
[0, 1, -1, 3]
>>> a[::2] = [0,1,2]
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in ?
ValueError: attempt to assign sequence of size 3 to extended sli
```

Deletion is more straightforward:

```
>>> a = range(4)
>>> a
[0, 1, 2, 3]
>>> a[::2]
[0, 2]
>>> del a[::2]
>>> a
[1, 3]
```

One can also now pass slice objects to the **`__getitem__()`** methods of the built-in sequences:

```
>>> range(10).__getitem__(slice(0, 5, 2))
[0, 2, 4]
```

Or use slice objects directly in subscripts:

```
>>> range(10)[slice(0, 5, 2)]
[0, 2, 4]
```

To simplify implementing sequences that support extended slicing, slice objects now have a method `indices(length)` which, given the length of a sequence, returns a `(start, stop, step)` tuple that can be passed directly to **`range()`**. **`indices()`** handles omitted and out-of-bounds indices in a manner consistent with regular slices (and this innocuous phrase hides a welter of confusing details!). The method is intended to be used like this:

```
class FakeSeq:
    ...
    def calc_item(self, i):
        ...
    def __getitem__(self, item):
        if isinstance(item, slice):
            indices = item.indices(len(self))
            return FakeSeq([self.calc_item(i) for i in range(*ir
```

```
        else:
            return self.calc_item(i)
```

From this example you can also see that the built-in `slice` object is now the type object for the slice type, and is no longer a function. This is consistent with Python 2.2, where `int`, `str`, etc., underwent the same change.

Outras mudanças na linguagem

Here are all of the changes that Python 2.3 makes to the core Python language.

- The `yield` statement is now always a keyword, as described in section PEP 255: Simple Generators of this document.
- A new built-in function `enumerate()` was added, as described in section PEP 279: enumerate() of this document.
- Two new constants, `True` and `False` were added along with the built-in `bool` type, as described in section PEP 285: A Boolean Type of this document.
- The `int()` type constructor will now return a long integer instead of raising an `OverflowError` when a string or floating-point number is too large to fit into an integer. This can lead to the paradoxical result that `isinstance(int(expression), int)` is false, but that seems unlikely to cause problems in practice.
- Built-in types now support the extended slicing syntax, as described in section Extended Slices of this document.
- A new built-in function, `sum(iterable, start=0)`, adds up the numeric items in the iterable object and returns their sum. `sum()` only accepts numbers, meaning that you can’t use it to concatenate a bunch of strings. (Contributed by Alex Martelli.)

- `list.insert(pos, value)` used to insert *value* at the front of the list when *pos* was negative. The behaviour has now been changed to be consistent with slice indexing, so when *pos* is -1 the value will be inserted before the last element, and so forth.

- `list.index(value)`, which searches for *value* within the list and returns its index, now takes optional *start* and *stop* arguments to limit the search to only part of the list.

- Dictionaries have a new method, `pop(key[, *default*])`, that returns the value corresponding to *key* and removes that key/value pair from the dictionary. If the requested key isn't present in the dictionary, *default* is returned if it's specified and **`KeyError`** raised if it isn't.

  ```
  >>> d = {1:2}
  >>> d
  {1: 2}
  >>> d.pop(4)
  Traceback (most recent call last):
    File "stdin", line 1, in ?
  KeyError: 4
  >>> d.pop(1)
  2
  >>> d.pop(1)
  Traceback (most recent call last):
    File "stdin", line 1, in ?
  KeyError: 'pop(): dictionary is empty'
  >>> d
  {}
  >>>
  ```

  There's also a new class method, `dict.fromkeys(iterable, value)`, that creates a dictionary with keys taken from the supplied iterator *iterable* and all values set to *value*, defaulting to `None`.

  (Patches contributed by Raymond Hettinger.)

  Also, the **`dict()`** constructor now accepts keyword arguments to simplify creating small dictionaries:

```
>>> dict(red=1, blue=2, green=3, black=4)
{'blue': 2, 'black': 4, 'green': 3, 'red': 1}
```

(Contributed by Just van Rossum.)

- The `assert` statement no longer checks the `__debug__` flag, so you can no longer disable assertions by assigning to `__debug__`. Running Python with the `-O` switch will still generate code that doesn't execute any assertions.

- Most type objects are now callable, so you can use them to create new objects such as functions, classes, and modules. (This means that the **`new`** module can be deprecated in a future Python version, because you can now use the type objects available in the `types` module.) For example, you can create a new module object with the following code:

```
>>> import types
>>> m = types.ModuleType('abc','docstring')
>>> m
<module 'abc' (built-in)>
>>> m.__doc__
'docstring'
```

- A new warning, `PendingDeprecationWarning` was added to indicate features which are in the process of being deprecated. The warning will *not* be printed by default. To check for use of features that will be deprecated in the future, supply `-Walways::PendingDeprecationWarning::` on the command line or use `warnings.filterwarnings()`.

- The process of deprecating string-based exceptions, as in `raise "Error occurred"`, has begun. Raising a string will now trigger `PendingDeprecationWarning`.

- Using `None` as a variable name will now result in a `SyntaxWarning` warning. In a future version of Python, `None` may finally become a keyword.

- The **`xreadlines()`** method of file objects, introduced in Python 2.1, is no longer necessary because files now behave as their own iterator. **`xreadlines()`** was originally introduced as a faster way to loop over all the lines in a file, but now you can simply write `for line in file_obj`. File objects also have a new read-only **`encoding`** attribute that gives the encoding used by the file; Unicode strings written to the file will be automatically converted to bytes using the given encoding.

- The method resolution order used by new-style classes has changed, though you'll only notice the difference if you have a really complicated inheritance hierarchy. Classic classes are unaffected by this change. Python 2.2 originally used a topological sort of a class's ancestors, but 2.3 now uses the C3 algorithm as described in the paper "A Monotonic Superclass Linearization for Dylan" [https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.19.3910]. To understand the motivation for this change, read Michele Simionato's article A Ordem de Resolução de Métodos do Python 2.3, or read the thread on python-dev starting with the message at https://mail.python.org/pipermail/python-dev/2002-October/029035.html. Samuele Pedroni first pointed out the problem and also implemented the fix by coding the C3 algorithm.

- Python runs multithreaded programs by switching between threads after executing N bytecodes. The default value for N has been increased from 10 to 100 bytecodes, speeding up single-threaded applications by reducing the switching overhead. Some multithreaded applications may suffer slower response time, but that's easily fixed by setting the limit back to a lower number using `sys.setcheckinterval(N)`. The limit can be retrieved with the new **`sys.getcheckinterval()`** function.

- One minor but far-reaching change is that the names of extension types defined by the modules included with Python now contain the module and a `'.'` in front of the type name. For example, in Python 2.2, if you created a socket and printed its **`__class__`**, you'd get this output:

```
>>> s = socket.socket()
>>> s.__class__
```

```
<type 'socket'>
```

In 2.3, you get this:

```
>>> s.__class__
<type '_socket.socket'>
```

- One of the noted incompatibilities between old- and new-style classes has been removed: you can now assign to the `__name__` and `__bases__` attributes of new-style classes. There are some restrictions on what can be assigned to **__bases__** along the lines of those relating to assigning to an instance's `__class__` attribute.

String Changes

- The `in` operator now works differently for strings. Previously, when evaluating `X in Y` where *X* and *Y* are strings, *X* could only be a single character. That's now changed; *X* can be a string of any length, and `X in Y` will return `True` if *X* is a substring of *Y*. If *X* is the empty string, the result is always **True**.

  ```
  >>> 'ab' in 'abcd'
  True
  >>> 'ad' in 'abcd'
  False
  >>> '' in 'abcd'
  True
  ```

  Note that this doesn't tell you where the substring starts; if you need that information, use the `find()` string method.

- The `strip()`, `lstrip()`, and `rstrip()` string methods now have an optional argument for specifying the characters to strip. The default is still to remove all whitespace characters:

  ```
  >>> '   abc '.strip()
  'abc'
  >>> '><><abc><><><>'.strip('<>')
  'abc'
  >>> '><><abc><><><>\n'.strip('<>')
  ```

```
'abc<><><>\n'
>>> u'\u4000\u4001abc\u4000'.strip(u'\u4000')
u'\u4001abc'
>>>
```

(Suggested by Simon Brunning and implemented by Walter Dörwald.)

- The `startswith()` and `endswith()` string methods now accept negative numbers for the *start* and *end* parameters.

- Another new string method is `zfill()`, originally a function in the `string` module. **`zfill()`** pads a numeric string with zeros on the left until it's the specified width. Note that the `%` operator is still more flexible and powerful than **`zfill()`**.

```
>>> '45'.zfill(4)
'0045'
>>> '12345'.zfill(4)
'12345'
>>> 'goofy'.zfill(6)
'0goofy'
```

(Contributed by Walter Dörwald.)

- A new type object, **`basestring`**, has been added. Both 8-bit strings and Unicode strings inherit from this type, so `isinstance(obj, basestring)` will return `True` for either kind of string. It's a completely abstract type, so you can't create **`basestring`** instances.

- Interned strings are no longer immortal and will now be garbage-collected in the usual way when the only reference to them is from the internal dictionary of interned strings. (Implemented by Oren Tirosh.)

Otimizações

- The creation of new-style class instances has been made much faster; they're now faster than classic classes!
- The `sort()` method of list objects has been extensively rewritten by Tim Peters, and the implementation is significantly faster.

- Multiplication of large long integers is now much faster thanks to an implementation of Karatsuba multiplication, an algorithm that scales better than the $O(n^2)$ required for the grade-school multiplication algorithm. (Original patch by Christopher A. Craig, and significantly reworked by Tim Peters.)
- The `SET_LINENO` opcode is now gone. This may provide a small speed increase, depending on your compiler's idiosyncrasies. See section Outras alterações e correções for a longer explanation. (Removed by Michael Hudson.)
- **`xrange()`** objects now have their own iterator, making `for i in xrange(n)` slightly faster than `for i in range(n)`. (Patch by Raymond Hettinger.)
- A number of small rearrangements have been made in various hotspots to improve performance, such as inlining a function or removing some code. (Implemented mostly by GvR, but lots of people have contributed single changes.)

The net result of the 2.3 optimizations is that Python 2.3 runs the pystone benchmark around 25% faster than Python 2.2.

New, Improved, and Deprecated Modules

As usual, Python's standard library received a number of enhancements and bug fixes. Here's a partial list of the most notable changes, sorted alphabetically by module name. Consult the `Misc/NEWS` file in the source tree for a more complete list of changes, or look through the CVS logs for all the details.

- The **`array`** module now supports arrays of Unicode characters using the `'u'` format character. Arrays also now support using the `+=` assignment operator to add another array's contents, and the `*=` assignment operator to repeat an array. (Contributed by Jason Orendorff.)

- The **`bsddb`** module has been replaced by version 4.1.6 of the PyBSDDB [https://pybsddb.sourceforge.net] package, providing a more

complete interface to the transactional features of the BerkeleyDB library.

The old version of the module has been renamed to **bsddb185** and is no longer built automatically; you'll have to edit `Modules/Setup` to enable it. Note that the new **bsddb** package is intended to be compatible with the old module, so be sure to file bugs if you discover any incompatibilities. When upgrading to Python 2.3, if the new interpreter is compiled with a new version of the underlying BerkeleyDB library, you will almost certainly have to convert your database files to the new version. You can do this fairly easily with the new scripts `db2pickle.py` and `pickle2db.py` which you will find in the distribution's `Tools/scripts` directory. If you've already been using the PyBSDDB package and importing it as **bsddb3**, you will have to change your `import` statements to import it as **bsddb**.

- The new **bz2** module is an interface to the bz2 data compression library. bz2-compressed data is usually smaller than corresponding **zlib**-compressed data. (Contributed by Gustavo Niemeyer.)

- A set of standard date/time types has been added in the new **datetime** module. See the following section for more details.

- The Distutils **Extension** class now supports an extra constructor argument named *depends* for listing additional source files that an extension depends on. This lets Distutils recompile the module if any of the dependency files are modified. For example, if `sampmodule.c` includes the header file `sample.h`, you would create the **Extension** object like this:

  ```
  ext = Extension("samp",
                  sources=["sampmodule.c"],
                  depends=["sample.h"])
  ```

  Modifying `sample.h` would then cause the module to be recompiled. (Contributed by Jeremy Hylton.)

- Other minor changes to Distutils: it now checks for the **CC**, **CFLAGS**, **CPP**, **LDFLAGS**, and **CPPFLAGS** environment variables, using them to override the settings in Python's configuration (contributed by Robert Weber).

- Previously the **doctest** module would only search the docstrings of public methods and functions for test cases, but it now also examines private ones as well. The **DocTestSuite()** function creates a **unittest.TestSuite** object from a set of **doctest** tests.

- The new `gc.get_referents(object)` function returns a list of all the objects referenced by *object*.

- The **getopt** module gained a new function, **gnu_getopt()**, that supports the same arguments as the existing **getopt()** function but uses GNU-style scanning mode. The existing **getopt()** stops processing options as soon as a non-option argument is encountered, but in GNU-style mode processing continues, meaning that options and arguments can be mixed. For example:

```
>>> getopt.getopt(['-f', 'filename', 'output', '-v'], 'f:v'
([('-f', 'filename')], ['output', '-v'])
>>> getopt.gnu_getopt(['-f', 'filename', 'output', '-v'], 'f
([('-f', 'filename'), ('-v', '')], ['output'])
```

  (Contributed by Peter Åstrand.)

- The **grp**, **pwd**, and **resource** modules now return enhanced tuples:

```
>>> import grp
>>> g = grp.getgrnam('amk')
>>> g.gr_name, g.gr_gid
('amk', 500)
```

- The **gzip** module can now handle files exceeding 2 GiB.

- The new **heapq** module contains an implementation of a heap queue algorithm. A heap is an array-like data structure that keeps items in a partially sorted order such that, for every index *k*, `heap[k] <=`

`heap[2*k+1]` and `heap[k] <= heap[2*k+2]`. This makes it quick to remove the smallest item, and inserting a new item while maintaining the heap property is *O*(log *n*). (See https://xlinux.nist.gov/dads//HTML/priorityque.html for more information about the priority queue data structure.)

The **`heapq`** module provides **`heappush()`** and **`heappop()`** functions for adding and removing items while maintaining the heap property on top of some other mutable Python sequence type. Here's an example that uses a Python list:

```
>>> import heapq
>>> heap = []
>>> for item in [3, 7, 5, 11, 1]:
...     heapq.heappush(heap, item)
...
>>> heap
[1, 3, 5, 11, 7]
>>> heapq.heappop(heap)
1
>>> heapq.heappop(heap)
3
>>> heap
[5, 7, 11]
```

(Contributed by Kevin O'Connor.)

- The IDLE integrated development environment has been updated using the code from the IDLEfork project (https://idlefork.sourceforge.net). The most notable feature is that the code being developed is now executed in a subprocess, meaning that there's no longer any need for manual `reload()` operations. IDLE's core code has been incorporated into the standard library as the **`idlelib`** package.

- The **`imaplib`** module now supports IMAP over SSL. (Contributed by Piers Lauder and Tino Lange.)

- The **`itertools`** contains a number of useful functions for use with iterators, inspired by various functions provided by the ML and

Haskell languages. For example, `itertools.ifilter(predicate, iterator)` returns all elements in the iterator for which the function **`predicate()`** returns **`True`**, and `itertools.repeat(obj, N)` returns `obj` *N* times. There are a number of other functions in the module; see the package's reference documentation for details. (Contributed by Raymond Hettinger.)

- Two new functions in the **`math`** module, `degrees(rads)` and `radians(degs)`, convert between radians and degrees. Other functions in the **`math`** module such as **`math.sin()`** and **`math.cos()`** have always required input values measured in radians. Also, an optional *base* argument was added to **`math.log()`** to make it easier to compute logarithms for bases other than `e` and `10`. (Contributed by Raymond Hettinger.)

- Several new POSIX functions (**`getpgid()`**, **`killpg()`**, **`lchown()`**, **`loadavg()`**, **`major()`**, **`makedev()`**, **`minor()`**, and **`mknod()`**) were added to the **`posix`** module that underlies the **`os`** module. (Contributed by Gustavo Niemeyer, Geert Jansen, and Denis S. Otkidach.)

- In the **`os`** module, the **`*stat()`** family of functions can now report fractions of a second in a timestamp. Such time stamps are represented as floats, similar to the value returned by **`time.time()`**.

  During testing, it was found that some applications will break if time stamps are floats. For compatibility, when using the tuple interface of the **`stat_result`** time stamps will be represented as integers. When using named fields (a feature first introduced in Python 2.2), time stamps are still represented as integers, unless **`os.stat_float_times()`** is invoked to enable float return values:

  ```
  >>> os.stat("/tmp").st_mtime
  1034791200
  >>> os.stat_float_times(True)
  >>> os.stat("/tmp").st_mtime
  1034791200.6335014
  ```

  In Python 2.4, the default will change to always returning floats.

Application developers should enable this feature only if all their libraries work properly when confronted with floating-point time stamps, or if they use the tuple API. If used, the feature should be activated on an application level instead of trying to enable it on a per-use basis.

- The **optparse** module contains a new parser for command-line arguments that can convert option values to a particular Python type and will automatically generate a usage message. See the following section for more details.

- The old and never-documented **linuxaudiodev** module has been deprecated, and a new version named **ossaudiodev** has been added. The module was renamed because the OSS sound drivers can be used on platforms other than Linux, and the interface has also been tidied and brought up to date in various ways. (Contributed by Greg Ward and Nicholas FitzRoy-Dale.)

- The new **platform** module contains a number of functions that try to determine various properties of the platform you're running on. There are functions for getting the architecture, CPU type, the Windows OS version, and even the Linux distribution version. (Contributed by Marc-André Lemburg.)

- The parser objects provided by the **pyexpat** module can now optionally buffer character data, resulting in fewer calls to your character data handler and therefore faster performance. Setting the parser object's **buffer_text** attribute to **True** will enable buffering.

- The `sample(population, k)` function was added to the **random** module. *population* is a sequence or **xrange** object containing the elements of a population, and **sample()** chooses *k* elements from the population without replacing chosen elements. *k* can be any value up to `len(population)`. For example:

```
>>> days = ['Mo', 'Tu', 'We', 'Th', 'Fr', 'St', 'Sn']
>>> random.sample(days, 3)      # Choose 3 elements
['St', 'Sn', 'Th']
```

```
>>> random.sample(days, 7)      # Choose 7 elements
['Tu', 'Th', 'Mo', 'We', 'St', 'Fr', 'Sn']
>>> random.sample(days, 7)      # Choose 7 again
['We', 'Mo', 'Sn', 'Fr', 'Tu', 'St', 'Th']
>>> random.sample(days, 8)      # Can't choose eight
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "random.py", line 414, in sample
      raise ValueError, "sample larger than population"
ValueError: sample larger than population
>>> random.sample(xrange(1,10000,2), 10)   # Choose ten odd
[3407, 3805, 1505, 7023, 2401, 2267, 9733, 3151, 8083, 9195]
```

The `random` module now uses a new algorithm, the Mersenne Twister, implemented in C. It's faster and more extensively studied than the previous algorithm.

(All changes contributed by Raymond Hettinger.)

- The `readline` module also gained a number of new functions: `get_history_item()`, `get_current_history_length()`, and `redisplay()`.

- The **`rexec`** and **`Bastion`** modules have been declared dead, and attempts to import them will fail with a `RuntimeError`. New-style classes provide new ways to break out of the restricted execution environment provided by **`rexec`**, and no one has interest in fixing them or time to do so. If you have applications using **`rexec`**, rewrite them to use something else.

  (Sticking with Python 2.2 or 2.1 will not make your applications any safer because there are known bugs in the **`rexec`** module in those versions. To repeat: if you're using **`rexec`**, stop using it immediately.)

- The **`rotor`** module has been deprecated because the algorithm it uses for encryption is not believed to be secure. If you need encryption, use one of the several AES Python modules that are available separately.

- The **shutil** module gained a `move(src, dest)` function that recursively moves a file or directory to a new location.

- Support for more advanced POSIX signal handling was added to the **signal** but then removed again as it proved impossible to make it work reliably across platforms.

- The **socket** module now supports timeouts. You can call the `settimeout(t)` method on a socket object to set a timeout of *t* seconds. Subsequent socket operations that take longer than *t* seconds to complete will abort and raise a **socket.timeout** exception.

  The original timeout implementation was by Tim O'Malley. Michael Gilfix integrated it into the Python **socket** module and shepherded it through a lengthy review. After the code was checked in, Guido van Rossum rewrote parts of it. (This is a good example of a collaborative development process in action.)

- On Windows, the **socket** module now ships with Secure Sockets Layer (SSL) support.

- The value of the C **PYTHON_API_VERSION** macro is now exposed at the Python level as `sys.api_version`. The current exception can be cleared by calling the new **sys.exc_clear()** function.

- The new **tarfile** module allows reading from and writing to **tar**-format archive files. (Contributed by Lars Gustäbel.)

- The new **textwrap** module contains functions for wrapping strings containing paragraphs of text. The `wrap(text, width)` function takes a string and returns a list containing the text split into lines of no more than the chosen width. The `fill(text, width)` function returns a single string, reformatted to fit into lines no longer than the chosen width. (As you can guess, **fill()** is built on top of **wrap()**. For example:

```
>>> import textwrap
>>> paragraph = "Not a whit, we defy augury: ... more text
>>> textwrap.wrap(paragraph, 60)
```

```
["Not a whit, we defy augury: there's a special providence i
 "the fall of a sparrow. If it be now, 'tis not to come; if
 ...]
>>> print textwrap.fill(paragraph, 35)
Not a whit, we defy augury: there's
a special providence in the fall of
a sparrow. If it be now, 'tis not
to come; if it be not to come, it
will be now; if it be not now, yet
it will come: the readiness is all.
>>>
```

The module also contains a **`TextWrapper`** class that actually implements the text wrapping strategy. Both the **`TextWrapper`** class and the **`wrap()`** and **`fill()`** functions support a number of additional keyword arguments for fine-tuning the formatting; consult the module's documentation for details. (Contributed by Greg Ward.)

- The **`thread`** and **`threading`** modules now have companion modules, **`dummy_thread`** and **`dummy_threading`**, that provide a do-nothing implementation of the **`thread`** module's interface for platforms where threads are not supported. The intention is to simplify thread-aware modules (ones that *don't* rely on threads to run) by putting the following code at the top:

  ```
  try:
      import threading as _threading
  except ImportError:
      import dummy_threading as _threading
  ```

  In this example, **`_threading`** is used as the module name to make it clear that the module being used is not necessarily the actual **`threading`** module. Code can call functions and use classes in **`_threading`** whether or not threads are supported, avoiding an **`if`** statement and making the code slightly clearer. This module will not magically make multithreaded code run without threads; code that waits for another thread to return or to do something will simply hang forever.

- The **time** module's **strptime()** function has long been an annoyance because it uses the platform C library's **strptime()** implementation, and different platforms sometimes have odd bugs. Brett Cannon contributed a portable implementation that's written in pure Python and should behave identically on all platforms.

- The new **timeit** module helps measure how long snippets of Python code take to execute. The `timeit.py` file can be run directly from the command line, or the module's **Timer** class can be imported and used directly. Here's a short example that figures out whether it's faster to convert an 8-bit string to Unicode by appending an empty Unicode string to it or by using the **unicode()** function:

```
import timeit

timer1 = timeit.Timer('unicode("abc")')
timer2 = timeit.Timer('"abc" + u""')

# Run three trials
print timer1.repeat(repeat=3, number=100000)
print timer2.repeat(repeat=3, number=100000)

# On my laptop this outputs:
# [0.36831796169281006, 0.37441694736480713, 0.353048920631
# [0.17574405670166016, 0.18193507194519043, 0.175657987594
```

- The **Tix** module has received various bug fixes and updates for the current version of the Tix package.

- The **Tkinter** module now works with a thread-enabled version of Tcl. Tcl's threading model requires that widgets only be accessed from the thread in which they're created; accesses from another thread can cause Tcl to panic. For certain Tcl interfaces, **Tkinter** will now automatically avoid this when a widget is accessed from a different thread by marshalling a command, passing it to the correct thread, and waiting for the results. Other interfaces can't be handled automatically but **Tkinter** will now raise an exception on such an access so that you can at least find out about the problem. See https://mail.python.org/pipermail/python-dev/2002-

December/031107.html for a more detailed explanation of this change. (Implemented by Martin von Löwis.)

- Calling Tcl methods through **_tkinter** no longer returns only strings. Instead, if Tcl returns other objects those objects are converted to their Python equivalent, if one exists, or wrapped with a **_tkinter.Tcl_Obj** object if no Python equivalent exists. This behavior can be controlled through the **wantobjects()** method of **tkapp** objects.

  When using **_tkinter** through the **Tkinter** module (as most Tkinter applications will), this feature is always activated. It should not cause compatibility problems, since Tkinter would always convert string results to Python types where possible.

  If any incompatibilities are found, the old behavior can be restored by setting the **wantobjects** variable in the **Tkinter** module to false before creating the first **tkapp** object.

  ```
  import Tkinter
  Tkinter.wantobjects = 0
  ```

  Any breakage caused by this change should be reported as a bug.

- The **UserDict** module has a new **DictMixin** class which defines all dictionary methods for classes that already have a minimum mapping interface. This greatly simplifies writing classes that need to be substitutable for dictionaries, such as the classes in the **shelve** module.

  Adding the mix-in as a superclass provides the full dictionary interface whenever the class defines **__getitem__()**, **__setitem__()**, **__delitem__()**, and **keys()**. For example:

  ```
  >>> import UserDict
  >>> class SeqDict(UserDict.DictMixin):
  ...     """Dictionary lookalike implemented with lists."""
  ...     def __init__(self):
  ...         self.keylist = []
  ...         self.valuelist = []
  ...     def __getitem__(self, key):
  ```

```
...         try:
...             i = self.keylist.index(key)
...         except ValueError:
...             raise KeyError
...         return self.valuelist[i]
...     def __setitem__(self, key, value):
...         try:
...             i = self.keylist.index(key)
...             self.valuelist[i] = value
...         except ValueError:
...             self.keylist.append(key)
...             self.valuelist.append(value)
...     def __delitem__(self, key):
...         try:
...             i = self.keylist.index(key)
...         except ValueError:
...             raise KeyError
...         self.keylist.pop(i)
...         self.valuelist.pop(i)
...     def keys(self):
...         return list(self.keylist)
...
>>> s = SeqDict()
>>> dir(s)      # See that other dictionary methods are impl
['__cmp__', '__contains__', '__delitem__', '__doc__', '__get
 '__init__', '__iter__', '__len__', '__module__', '__repr__
 '__setitem__', 'clear', 'get', 'has_key', 'items', 'iterite
 'iterkeys', 'itervalues', 'keylist', 'keys', 'pop', 'popite
 'setdefault', 'update', 'valuelist', 'values']
```

(Contributed by Raymond Hettinger.)

- The DOM implementation in `xml.dom.minidom` can now generate XML output in a particular encoding by providing an optional encoding argument to the `toxml()` and `toprettyxml()` methods of DOM nodes.

- The `xmlrpclib` module now supports an XML-RPC extension for handling nil data values such as Python's `None`. Nil values are always supported on unmarshalling an XML-RPC response. To generate

requests containing `None`, you must supply a true value for the *allow_none* parameter when creating a **`Marshaller`** instance.

- The new **`DocXMLRPCServer`** module allows writing self-documenting XML-RPC servers. Run it in demo mode (as a program) to see it in action. Pointing the web browser to the RPC server produces pydoc-style documentation; pointing xmlrpclib to the server allows invoking the actual methods. (Contributed by Brian Quinlan.)

- Support for internationalized domain names (RFCs 3454, 3490, 3491, and 3492) has been added. The "idna" encoding can be used to convert between a Unicode domain name and the ASCII-compatible encoding (ACE) of that name.

  ```
  >{}>{}> u"www.Alliancefrançaise.nu".encode("idna")
  'www.xn--alliancefranaise-npb.nu'
  ```

  The **`socket`** module has also been extended to transparently convert Unicode hostnames to the ACE version before passing them to the C library. Modules that deal with hostnames such as **`httplib`** and **`ftplib`**) also support Unicode host names; **`httplib`** also sends HTTP `Host` headers using the ACE version of the domain name. **`urllib`** supports Unicode URLs with non-ASCII host names as long as the `path` part of the URL is ASCII only.

  To implement this change, the **`stringprep`** module, the `mkstringprep` tool and the `punycode` encoding have been added.

Date/Time Type

Date and time types suitable for expressing timestamps were added as the **`datetime`** module. The types don't support different calendars or many fancy features, and just stick to the basics of representing time.

The three primary types are: **`date`**, representing a day, month, and year; **`time`**, consisting of hour, minute, and second; and **`datetime`**, which contains all the attributes of both **`date`** and **`time`**. There's also a **`timedelta`** class

representing differences between two points in time, and time zone logic is implemented by classes inheriting from the abstract **tzinfo** class.

You can create instances of **date** and **time** by either supplying keyword arguments to the appropriate constructor, e.g. `datetime.date(year=1972, month=10, day=15)`, or by using one of a number of class methods. For example, the **today()** class method returns the current local date.

Once created, instances of the date/time classes are all immutable. There are a number of methods for producing formatted strings from objects:

```
>>> import datetime as dt
>>> now = dt.datetime.now()
>>> now.isoformat()
'2002-12-30T21:27:03.994956'
>>> now.ctime()  # Only available on date, datetime
'Mon Dec 30 21:27:03 2002'
>>> now.strftime('%Y %d %b')
'2002 30 Dec'
```

The **replace()** method allows modifying one or more fields of a **date** or **datetime** instance, returning a new instance:

```
>>> d = dt.datetime.now()
>>> d
datetime.datetime(2002, 12, 30, 22, 15, 38, 827738)
>>> d.replace(year=2001, hour=12)
datetime.datetime(2001, 12, 30, 12, 15, 38, 827738)
>>>
```

Instances can be compared, hashed, and converted to strings (the result is the same as that of **isoformat()**). **date** and **datetime** instances can be subtracted from each other, and added to **timedelta** instances. The largest missing feature is that there's no standard library support for parsing strings and getting back a **date** or **datetime**.

For more information, refer to the module's reference documentation. (Contributed by Tim Peters.)

The optparse Module

The **getopt** module provides simple parsing of command-line arguments. The new **optparse** module (originally named Optik) provides more elaborate command-line parsing that follows the Unix conventions, automatically creates the output for **--help**, and can perform different actions for different options.

You start by creating an instance of **OptionParser** and telling it what your program's options are.

```
import sys
from optparse import OptionParser

op = OptionParser()
op.add_option('-i', '--input',
              action='store', type='string', dest='input',
              help='set input filename')
op.add_option('-l', '--length',
              action='store', type='int', dest='length',
              help='set maximum length of output')
```

Parsing a command line is then done by calling the **parse_args()** method.

```
options, args = op.parse_args(sys.argv[1:])
print options
print args
```

This returns an object containing all of the option values, and a list of strings containing the remaining arguments.

Invoking the script with the various arguments now works as you'd expect it to. Note that the length argument is automatically converted to an integer.

```
$ ./python opt.py -i data arg1
<Values at 0x400cad4c: {'input': 'data', 'length': None}>
['arg1']
$ ./python opt.py --input=data --length=4
<Values at 0x400cad2c: {'input': 'data', 'length': 4}>
[]
$
```

The help message is automatically generated for you:

```
$ ./python opt.py --help
usage: opt.py [options]

options:
  -h, --help            show this help message and exit
  -iINPUT, --input=INPUT
                        set input filename
  -lLENGTH, --length=LENGTH
                        set maximum length of output
$
```

See the module's documentation for more details.

Optik was written by Greg Ward, with suggestions from the readers of the Getopt SIG.

Pymalloc: A Specialized Object Allocator

Pymalloc, a specialized object allocator written by Vladimir Marangozov, was a feature added to Python 2.1. Pymalloc is intended to be faster than the system **`malloc()`** and to have less memory overhead for allocation patterns typical of Python programs. The allocator uses C's **`malloc()`** function to get large pools of memory and then fulfills smaller memory requests from these pools.

In 2.1 and 2.2, pymalloc was an experimental feature and wasn't enabled by default; you had to explicitly enable it when compiling Python by providing the **`--with-pymalloc`** option to the **configure** script. In 2.3, pymalloc has had further enhancements and is now enabled by default; you'll have to supply **`--without-pymalloc`** to disable it.

This change is transparent to code written in Python; however, pymalloc may expose bugs in C extensions. Authors of C extension modules should test their code with pymalloc enabled, because some incorrect code may cause core dumps at runtime.

There's one particularly common error that causes problems. There are a number of memory allocation functions in Python's C API that have previously just been aliases for the C library's **`malloc()`** and **`free()`**,

meaning that if you accidentally called mismatched functions the error wouldn't be noticeable. When the object allocator is enabled, these functions aren't aliases of **`malloc()`** and **`free()`** any more, and calling the wrong function to free memory may get you a core dump. For example, if memory was allocated using **`PyObject_Malloc()`**, it has to be freed using **`PyObject_Free()`**, not **`free()`**. A few modules included with Python fell afoul of this and had to be fixed; doubtless there are more third-party modules that will have the same problem.

As part of this change, the confusing multiple interfaces for allocating memory have been consolidated down into two API families. Memory allocated with one family must not be manipulated with functions from the other family. There is one family for allocating chunks of memory and another family of functions specifically for allocating Python objects.

- To allocate and free an undistinguished chunk of memory use the "raw memory" family: **`PyMem_Malloc()`**, **`PyMem_Realloc()`**, and **`PyMem_Free()`**.
- The "object memory" family is the interface to the pymalloc facility described above and is biased towards a large number of "small" allocations: **`PyObject_Malloc()`**, **`PyObject_Realloc()`**, and **`PyObject_Free()`**.
- To allocate and free Python objects, use the "object" family **`PyObject_New`**, **`PyObject_NewVar`**, and **`PyObject_Del()`**.

Thanks to lots of work by Tim Peters, pymalloc in 2.3 also provides debugging features to catch memory overwrites and doubled frees in both extension modules and in the interpreter itself. To enable this support, compile a debugging version of the Python interpreter by running **configure** with **`--with-pydebug`**.

To aid extension writers, a header file `Misc/pymemcompat.h` is distributed with the source to Python 2.3 that allows Python extensions to use the 2.3 interfaces to memory allocation while compiling against any version of Python since 1.5.2. You would copy the file from Python's source distribution and bundle it with the source of your extension.

Ver também

https://hg.python.org/cpython/file/default/Objects/obmalloc.c
For the full details of the pymalloc implementation, see the comments at the top of the file `Objects/obmalloc.c` in the Python source code. The above link points to the file within the python.org SVN browser.

Mudanças a construções e API C

As alterações no processo de construção do Python e na API C incluem:

- The cycle detection implementation used by the garbage collection has proven to be stable, so it's now been made mandatory. You can no longer compile Python without it, and the **`--with-cycle-gc`** switch to **configure** has been removed.
- Python can now optionally be built as a shared library (`libpython2.3.so`) by supplying **`--enable-shared`** when running Python's **configure** script. (Contributed by Ondrej Palkovsky.)
- The **`DL_EXPORT`** and **`DL_IMPORT`** macros are now deprecated. Initialization functions for Python extension modules should now be declared using the new macro **`PyMODINIT_FUNC`**, while the Python core will generally use the **`PyAPI_FUNC`** and **`PyAPI_DATA`** macros.
- The interpreter can be compiled without any docstrings for the built-in functions and modules by supplying **`--without-doc-strings`** to the **configure** script. This makes the Python executable about 10% smaller, but will also mean that you can't get help for Python's built-ins. (Contributed by Gustavo Niemeyer.)
- The **`PyArg_NoArgs()`** macro is now deprecated, and code that uses it should be changed. For Python 2.2 and later, the method definition table can specify the **`METH_NOARGS`** flag, signalling that there are no arguments, and the argument checking can then be removed. If compatibility with pre-2.2 versions of Python is important, the code could use `PyArg_ParseTuple(args, "")` instead, but this will be slower than using **`METH_NOARGS`**.

- `PyArg_ParseTuple()` accepts new format characters for various sizes of unsigned integers: `B` for unsigned char, `H` for unsigned short int, `I` for unsigned int, and `K` for unsigned long long.
- A new function, `PyObject_DelItemString(mapping, char *key)` was added as shorthand for `PyObject_DelItem(mapping, PyString_New(key))`.
- File objects now manage their internal string buffer differently, increasing it exponentially when needed. This results in the benchmark tests in `Lib/test/test_bufio.py` speeding up considerably (from 57 seconds to 1.7 seconds, according to one measurement).
- It's now possible to define class and static methods for a C extension type by setting either the `METH_CLASS` or `METH_STATIC` flags in a method's `PyMethodDef` structure.
- Python now includes a copy of the Expat XML parser's source code, removing any dependence on a system version or local installation of Expat.
- If you dynamically allocate type objects in your extension, you should be aware of a change in the rules relating to the `__module__` and `__name__` attributes. In summary, you will want to ensure the type's dictionary contains a `'__module__'` key; making the module name the part of the type name leading up to the final period will no longer have the desired effect. For more detail, read the API reference documentation or the source.

Port-Specific Changes

Support for a port to IBM's OS/2 using the EMX runtime environment was merged into the main Python source tree. EMX is a POSIX emulation layer over the OS/2 system APIs. The Python port for EMX tries to support all the POSIX-like capability exposed by the EMX runtime, and mostly succeeds; **`fork()`** and `fcntl()` are restricted by the limitations of the underlying emulation layer. The standard OS/2 port, which uses IBM's Visual Age compiler, also gained support for case-sensitive import semantics as part of the integration of the EMX port into CVS. (Contributed by Andrew MacIntyre.)

On MacOS, most toolbox modules have been weaklinked to improve backward compatibility. This means that modules will no longer fail to load if a single routine is missing on the current OS version. Instead calling the missing routine will raise an exception. (Contributed by Jack Jansen.)

The RPM spec files, found in the `Misc/RPM/` directory in the Python source distribution, were updated for 2.3. (Contributed by Sean Reifschneider.)

Other new platforms now supported by Python include AtheOS (http://www.atheos.cx/), GNU/Hurd, and OpenVMS.

Outras alterações e correções

As usual, there were a bunch of other improvements and bugfixes scattered throughout the source tree. A search through the CVS change logs finds there were 523 patches applied and 514 bugs fixed between Python 2.2 and 2.3. Both figures are likely to be underestimates.

Some of the more notable changes are:

- If the **`PYTHONINSPECT`** environment variable is set, the Python interpreter will enter the interactive prompt after running a Python program, as if Python had been invoked with the **`-i`** option. The environment variable can be set before running the Python interpreter, or it can be set by the Python program as part of its execution.

- The `regrtest.py` script now provides a way to allow "all resources except *foo*." A resource name passed to the **`-u`** option can now be prefixed with a hyphen (`'-'`) to mean "remove this resource." For example, the option '`-uall,-bsddb`' could be used to enable the use of all resources except `bsddb`.

- The tools used to build the documentation now work under Cygwin as well as Unix.

- The `SET_LINENO` opcode has been removed. Back in the mists of time, this opcode was needed to produce line numbers in tracebacks and

support trace functions (for, e.g., **pdb**). Since Python 1.5, the line numbers in tracebacks have been computed using a different mechanism that works with "python -O". For Python 2.3 Michael Hudson implemented a similar scheme to determine when to call the trace function, removing the need for `SET_LINENO` entirely.

It would be difficult to detect any resulting difference from Python code, apart from a slight speed up when Python is run without **-O**.

C extensions that access the **f_lineno** field of frame objects should instead call `PyCode_Addr2Line(f->f_code, f->f_lasti)`. This will have the added effect of making the code work as desired under "python -O" in earlier versions of Python.

A nifty new feature is that trace functions can now assign to the **f_lineno** attribute of frame objects, changing the line that will be executed next. A `jump` command has been added to the **pdb** debugger taking advantage of this new feature. (Implemented by Richie Hindle.)

Porting to Python 2.3

This section lists previously described changes that may require changes to your code:

- **yield** is now always a keyword; if it's used as a variable name in your code, a different name must be chosen.
- For strings *X* and *Y*, `X in Y` now works if *X* is more than one character long.
- The **int()** type constructor will now return a long integer instead of raising an **OverflowError** when a string or floating-point number is too large to fit into an integer.
- If you have Unicode strings that contain 8-bit characters, you must declare the file's encoding (UTF-8, Latin-1, or whatever) by adding a

comment to the top of the file. See section PEP 263: Source Code Encodings for more information.

- Calling Tcl methods through **`_tkinter`** no longer returns only strings. Instead, if Tcl returns other objects those objects are converted to their Python equivalent, if one exists, or wrapped with a **`_tkinter.Tcl_Obj`** object if no Python equivalent exists.

- Large octal and hex literals such as `0xffffffff` now trigger a **`FutureWarning`**. Currently they're stored as 32-bit numbers and result in a negative value, but in Python 2.4 they'll become positive long integers.

  There are a few ways to fix this warning. If you really need a positive number, just add an `L` to the end of the literal. If you're trying to get a 32-bit integer with low bits set and have previously used an expression such as `~(1 << 31)`, it's probably clearest to start with all bits set and clear the desired upper bits. For example, to clear just the top bit (bit 31), you could write `0xffffffffL &~(1L<<31)`.

- You can no longer disable assertions by assigning to `__debug__`.

- The Distutils **`setup()`** function has gained various new keyword arguments such as *depends*. Old versions of the Distutils will abort if passed unknown keywords. A solution is to check for the presence of the new **`get_distutil_options()`** function in your `setup.py` and only uses the new keywords with a version of the Distutils that supports them:

  ```
  from distutils import core

  kw = {'sources': 'foo.c', ...}
  if hasattr(core, 'get_distutil_options'):
      kw['depends'] = ['foo.h']
  ext = Extension(**kw)
  ```

- Using `None` as a variable name will now result in a **`SyntaxWarning`** warning.

- Names of extension types defined by the modules included with Python now contain the module and a `'.'` in front of the type name.

Reconhecimentos

The author would like to thank the following people for offering suggestions, corrections and assistance with various drafts of this article: Jeff Bauer, Simon Brunning, Brett Cannon, Michael Chermside, Andrew Dalke, Scott David Daniels, Fred L. Drake, Jr., David Fraser, Kelly Gerber, Raymond Hettinger, Michael Hudson, Chris Lambert, Detlef Lannert, Martin von Löwis, Andrew MacIntyre, Lalo Martins, Chad Netzer, Gustavo Niemeyer, Neal Norwitz, Hans Nowak, Chris Reedy, Francesco Ricciardi, Vinay Sajip, Neil Schemenauer, Roman Suzi, Jason Tishler, Just van Rossum.

What’s New in Python 2.2

Autor:: A.M. Kuchling

Introdução

This article explains the new features in Python 2.2.2, released on October 14, 2002. Python 2.2.2 is a bugfix release of Python 2.2, originally released on December 21, 2001.

Python 2.2 can be thought of as the “cleanup release”. There are some features such as generators and iterators that are completely new, but most of the changes, significant and far-reaching though they may be, are aimed at cleaning up irregularities and dark corners of the language design.

This article doesn’t attempt to provide a complete specification of the new features, but instead provides a convenient overview. For full details, you should refer to the documentation for Python 2.2, such as the Python Library Reference [https://docs.python.org/2.2/lib/lib.html] and the Python Reference Manual [https://docs.python.org/2.2/ref/ref.html]. If you want to understand the complete implementation and design rationale for a change, refer to the PEP for a particular new feature.

PEPs 252 and 253: Type and Class Changes

The largest and most far-reaching changes in Python 2.2 are to Python’s model of objects and classes. The changes should be backward compatible, so it’s likely that your code will continue to run unchanged, but the changes provide some amazing new capabilities. Before beginning this, the longest and most complicated section of this article, I’ll provide an overview of the changes and offer some comments.

A long time ago I wrote a web page listing flaws in Python’s design. One of the most significant flaws was that it’s impossible to subclass Python types

implemented in C. In particular, it's not possible to subclass built-in types, so you can't just subclass, say, lists in order to add a single useful method to them. The `UserList` module provides a class that supports all of the methods of lists and that can be subclassed further, but there's lots of C code that expects a regular Python list and won't accept a `UserList` instance.

Python 2.2 fixes this, and in the process adds some exciting new capabilities. A brief summary:

- You can subclass built-in types such as lists and even integers, and your subclasses should work in every place that requires the original type.
- It's now possible to define static and class methods, in addition to the instance methods available in previous versions of Python.
- It's also possible to automatically call methods on accessing or setting an instance attribute by using a new mechanism called *properties*. Many uses of `__getattr__()` can be rewritten to use properties instead, making the resulting code simpler and faster. As a small side benefit, attributes can now have docstrings, too.
- The list of legal attributes for an instance can be limited to a particular set using *slots*, making it possible to safeguard against typos and perhaps make more optimizations possible in future versions of Python.

Some users have voiced concern about all these changes. Sure, they say, the new features are neat and lend themselves to all sorts of tricks that weren't possible in previous versions of Python, but they also make the language more complicated. Some people have said that they've always recommended Python for its simplicity, and feel that its simplicity is being lost.

Personally, I think there's no need to worry. Many of the new features are quite esoteric, and you can write a lot of Python code without ever needed to be aware of them. Writing a simple class is no more difficult than it ever was, so you don't need to bother learning or teaching them unless they're actually needed. Some very complicated tasks that were previously only

possible from C will now be possible in pure Python, and to my mind that's all for the better.

I'm not going to attempt to cover every single corner case and small change that were required to make the new features work. Instead this section will paint only the broad strokes. See section Related Links, "Related Links", for further sources of information about Python 2.2's new object model.

Old and New Classes

First, you should know that Python 2.2 really has two kinds of classes: classic or old-style classes, and new-style classes. The old-style class model is exactly the same as the class model in earlier versions of Python. All the new features described in this section apply only to new-style classes. This divergence isn't intended to last forever; eventually old-style classes will be dropped, possibly in Python 3.0.

So how do you define a new-style class? You do it by subclassing an existing new-style class. Most of Python's built-in types, such as integers, lists, dictionaries, and even files, are new-style classes now. A new-style class named `object`, the base class for all built-in types, has also been added so if no built-in type is suitable, you can just subclass `object`:

```
class C(object):
    def __init__ (self):
        ...
    ...
```

This means that `class` statements that don't have any base classes are always classic classes in Python 2.2. (Actually you can also change this by setting a module-level variable named `__metaclass__` — see **PEP 253** [https://peps.python.org/pep-0253/] for the details — but it's easier to just subclass `object`.)

The type objects for the built-in types are available as built-ins, named using a clever trick. Python has always had built-in functions named `int()`, `float()`, and `str()`. In 2.2, they aren't functions any more, but type objects that behave as factories when called.

```
>>> int
<type 'int'>
>>> int('123')
123
```

To make the set of types complete, new type objects such as **dict()** and **file()** have been added. Here's a more interesting example, adding a **lock()** method to file objects:

```
class LockableFile(file):
    def lock (self, operation, length=0, start=0, whence=0):
        import fcntl
        return fcntl.lockf(self.fileno(), operation,
                           length, start, whence)
```

The now-obsolete **posixfile** module contained a class that emulated all of a file object's methods and also added a **lock()** method, but this class couldn't be passed to internal functions that expected a built-in file, something which is possible with our new **LockableFile**.

Descriptors

In previous versions of Python, there was no consistent way to discover what attributes and methods were supported by an object. There were some informal conventions, such as defining **__members__** and **__methods__** attributes that were lists of names, but often the author of an extension type or a class wouldn't bother to define them. You could fall back on inspecting the **__dict__** of an object, but when class inheritance or an arbitrary **__getattr__()** hook were in use this could still be inaccurate.

The one big idea underlying the new class model is that an API for describing the attributes of an object using *descriptors* has been formalized. Descriptors specify the value of an attribute, stating whether it's a method or a field. With the descriptor API, static methods and class methods become possible, as well as more exotic constructs.

Attribute descriptors are objects that live inside class objects, and have a few attributes of their own:

- `__name__` is the attribute's name.
- `__doc__` is the attribute's docstring.
- `__get__(object)` is a method that retrieves the attribute value from *object*.
- `__set__(object, value)` sets the attribute on *object* to *value*.
- `__delete__(object, value)` deletes the *value* attribute of *object*.

For example, when you write `obj.x`, the steps that Python actually performs are:

```
descriptor = obj.__class__.x
descriptor.__get__(obj)
```

For methods, `descriptor.__get__` returns a temporary object that's callable, and wraps up the instance and the method to be called on it. This is also why static methods and class methods are now possible; they have descriptors that wrap up just the method, or the method and the class. As a brief explanation of these new kinds of methods, static methods aren't passed the instance, and therefore resemble regular functions. Class methods are passed the class of the object, but not the object itself. Static and class methods are defined like this:

```
class C(object):
    def f(arg1, arg2):
        ...
    f = staticmethod(f)

    def g(cls, arg1, arg2):
        ...
    g = classmethod(g)
```

The `staticmethod()` function takes the function **`f()`**, and returns it wrapped up in a descriptor so it can be stored in the class object. You might expect there to be special syntax for creating such methods (`def static f`, `defstatic f()`, or something like that) but no such syntax has been defined yet; that's been left for future versions of Python.

More new features, such as slots and properties, are also implemented as new kinds of descriptors, and it's not difficult to write a descriptor class that

does something novel. For example, it would be possible to write a descriptor class that made it possible to write Eiffel-style preconditions and postconditions for a method. A class that used this feature might be defined like this:

```
from eiffel import eiffelmethod

class C(object):
    def f(self, arg1, arg2):
        # The actual function
        ...
    def pre_f(self):
        # Check preconditions
        ...
    def post_f(self):
        # Check postconditions
        ...

    f = eiffelmethod(f, pre_f, post_f)
```

Note that a person using the new **`eiffelmethod()`** doesn't have to understand anything about descriptors. This is why I think the new features don't increase the basic complexity of the language. There will be a few wizards who need to know about it in order to write **`eiffelmethod()`** or the ZODB or whatever, but most users will just write code on top of the resulting libraries and ignore the implementation details.

Multiple Inheritance: The Diamond Rule

Multiple inheritance has also been made more useful through changing the rules under which names are resolved. Consider this set of classes (diagram taken from **PEP 253** [https://peps.python.org/pep-0253/] by Guido van Rossum):

```
      class A:
        ^ ^  def save(self): ...
       /   \
      /     \
     /       \
    /         \
class B     class C:
```

```
^         ^  def save(self): ...
 \       /
  \     /
   \   /
    \ /
  class D
```

The lookup rule for classic classes is simple but not very smart; the base classes are searched depth-first, going from left to right. A reference to **`D.save()`** will search the classes **`D`**, **`B`**, and then **`A`**, where **`save()`** would be found and returned. **`C.save()`** would never be found at all. This is bad, because if **`C`**'s **`save()`** method is saving some internal state specific to **`C`**, not calling it will result in that state never getting saved.

New-style classes follow a different algorithm that's a bit more complicated to explain, but does the right thing in this situation. (Note that Python 2.3 changes this algorithm to one that produces the same results in most cases, but produces more useful results for really complicated inheritance graphs.)

1. List all the base classes, following the classic lookup rule and include a class multiple times if it's visited repeatedly. In the above example, the list of visited classes is [**`D`**, **`B`**, **`A`**, **`C`**, **`A`**].
2. Scan the list for duplicated classes. If any are found, remove all but one occurrence, leaving the *last* one in the list. In the above example, the list becomes [**`D`**, **`B`**, **`C`**, **`A`**] after dropping duplicates.

Following this rule, referring to **`D.save()`** will return **`C.save()`**, which is the behaviour we're after. This lookup rule is the same as the one followed by Common Lisp. A new built-in function, **`super()`**, provides a way to get at a class's superclasses without having to reimplement Python's algorithm. The most commonly used form will be `super(class, obj)`, which returns a bound superclass object (not the actual class object). This form will be used in methods to call a method in the superclass; for example, **`D`**'s **`save()`** method would look like this:

```
class D (B,C):
    def save (self):
        # Call superclass .save()
        super(D, self).save()
```

```
        # Save D's private information here
        ...
```

super() can also return unbound superclass objects when called as `super(class)` or `super(class1, class2)`, but this probably won't often be useful.

Attribute Access

A fair number of sophisticated Python classes define hooks for attribute access using **__getattr__()**; most commonly this is done for convenience, to make code more readable by automatically mapping an attribute access such as `obj.parent` into a method call such as `obj.get_parent`. Python 2.2 adds some new ways of controlling attribute access.

First, `__getattr__(attr_name)` is still supported by new-style classes, and nothing about it has changed. As before, it will be called when an attempt is made to access `obj.foo` and no attribute named `foo` is found in the instance's dictionary.

New-style classes also support a new method, `__getattribute__(attr_name)`. The difference between the two methods is that **__getattribute__()** is *always* called whenever any attribute is accessed, while the old **__getattr__()** is only called if `foo` isn't found in the instance's dictionary.

However, Python 2.2's support for *properties* will often be a simpler way to trap attribute references. Writing a **__getattr__()** method is complicated because to avoid recursion you can't use regular attribute accesses inside them, and instead have to mess around with the contents of **__dict__**. **__getattr__()** methods also end up being called by Python when it checks for other methods such as **__repr__()** or **__coerce__()**, and so have to be written with this in mind. Finally, calling a function on every attribute access results in a sizable performance loss.

property is a new built-in type that packages up three functions that get, set, or delete an attribute, and a docstring. For example, if you want to define a **size** attribute that's computed, but also settable, you could write:

```
class C(object):
    def get_size (self):
        result = ... computation ...
        return result
    def set_size (self, size):
        ... compute something based on the size
        and set internal state appropriately ...

    # Define a property.  The 'delete this attribute'
    # method is defined as None, so the attribute
    # can't be deleted.
    size = property(get_size, set_size,
                    None,
                    "Storage size of this instance")
```

That is certainly clearer and easier to write than a pair of **__getattr__()**/**__setattr__()** methods that check for the **size** attribute and handle it specially while retrieving all other attributes from the instance's **__dict__**. Accesses to **size** are also the only ones which have to perform the work of calling a function, so references to other attributes run at their usual speed.

Finally, it's possible to constrain the list of attributes that can be referenced on an object using the new **__slots__** class attribute. Python objects are usually very dynamic; at any time it's possible to define a new attribute on an instance by just doing `obj.new_attr=1`. A new-style class can define a class attribute named **__slots__** to limit the legal attributes to a particular set of names. An example will make this clear:

```
>>> class C(object):
...     __slots__ = ('template', 'name')
...
>>> obj = C()
>>> print obj.template
None
>>> obj.template = 'Test'
>>> print obj.template
Test
>>> obj.newattr = None
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in ?
AttributeError: 'C' object has no attribute 'newattr'
```

Note how you get an `AttributeError` on the attempt to assign to an attribute not listed in `__slots__`.

Related Links

This section has just been a quick overview of the new features, giving enough of an explanation to start you programming, but many details have been simplified or ignored. Where should you go to get a more complete picture?

The Guia de descritores is a lengthy tutorial introduction to the descriptor features, written by Guido van Rossum. If my description has whetted your appetite, go read this tutorial next, because it goes into much more detail about the new features while still remaining quite easy to read.

Next, there are two relevant PEPs, **PEP 252** [https://peps.python.org/pep-0252/] and **PEP 253** [https://peps.python.org/pep-0253/]. **PEP 252** [https://peps.python.org/pep-0252/] is titled “Making Types Look More Like Classes”, and covers the descriptor API. **PEP 253** [https://peps.python.org/pep-0253/] is titled “Subtyping Built-in Types”, and describes the changes to type objects that make it possible to subtype built-in objects. **PEP 253** [https://peps.python.org/pep-0253/] is the more complicated PEP of the two, and at a few points the necessary explanations of types and meta-types may cause your head to explode. Both PEPs were written and implemented by Guido van Rossum, with substantial assistance from the rest of the Zope Corp. team.

Finally, there’s the ultimate authority: the source code. Most of the machinery for the type handling is in `Objects/typeobject.c`, but you should only resort to it after all other avenues have been exhausted, including posting a question to python-list or python-dev.

PEP 234: Iterators

Another significant addition to 2.2 is an iteration interface at both the C and Python levels. Objects can define how they can be looped over by callers.

In Python versions up to 2.1, the usual way to make `for item in obj` work is to define a **`__getitem__()`** method that looks something like this:

```
def __getitem__(self, index):
    return <next item>
```

`__getitem__()` is more properly used to define an indexing operation on an object so that you can write `obj[5]` to retrieve the sixth element. It's a bit misleading when you're using this only to support **`for`** loops. Consider some file-like object that wants to be looped over; the *index* parameter is essentially meaningless, as the class probably assumes that a series of **`__getitem__()`** calls will be made with *index* incrementing by one each time. In other words, the presence of the **`__getitem__()`** method doesn't mean that using `file[5]` to randomly access the sixth element will work, though it really should.

In Python 2.2, iteration can be implemented separately, and **`__getitem__()`** methods can be limited to classes that really do support random access. The basic idea of iterators is simple. A new built-in function, `iter(obj)` or `iter(C, sentinel)`, is used to get an iterator. `iter(obj)` returns an iterator for the object *obj*, while `iter(C, sentinel)` returns an iterator that will invoke the callable object *C* until it returns *sentinel* to signal that the iterator is done.

Python classes can define an **`__iter__()`** method, which should create and return a new iterator for the object; if the object is its own iterator, this method can just return `self`. In particular, iterators will usually be their own iterators. Extension types implemented in C can implement a **`tp_iter`** function in order to return an iterator, and extension types that want to behave as iterators can define a **`tp_iternext`** function.

So, after all this, what do iterators actually do? They have one required method, **`next()`**, which takes no arguments and returns the next value. When there are no more values to be returned, calling **`next()`** should raise the **`StopIteration`** exception.

```
>>> L = [1,2,3]
>>> i = iter(L)
>>> print i
<iterator object at 0x8116870>
>>> i.next()
1
>>> i.next()
2
>>> i.next()
3
>>> i.next()
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
StopIteration
>>>
```

In 2.2, Python's **for** statement no longer expects a sequence; it expects something for which **iter()** will return an iterator. For backward compatibility and convenience, an iterator is automatically constructed for sequences that don't implement **__iter__()** or a **tp_iter** slot, so `for i in [1,2,3]` will still work. Wherever the Python interpreter loops over a sequence, it's been changed to use the iterator protocol. This means you can do things like this:

```
>>> L = [1,2,3]
>>> i = iter(L)
>>> a,b,c = i
>>> a,b,c
(1, 2, 3)
```

Iterator support has been added to some of Python's basic types. Calling **iter()** on a dictionary will return an iterator which loops over its keys:

```
>>> m = {'Jan': 1, 'Feb': 2, 'Mar': 3, 'Apr': 4, 'May': 5, 'Jun'
...      'Jul': 7, 'Aug': 8, 'Sep': 9, 'Oct': 10, 'Nov': 11, 'De
>>> for key in m: print key, m[key]
...
Mar 3
Feb 2
Aug 8
Sep 9
```

```
May 5
Jun 6
Jul 7
Jan 1
Apr 4
Nov 11
Dec 12
Oct 10
```

That's just the default behaviour. If you want to iterate over keys, values, or key/value pairs, you can explicitly call the **`iterkeys()`**, **`itervalues()`**, or **`iteritems()`** methods to get an appropriate iterator. In a minor related change, the **`in`** operator now works on dictionaries, so `key in dict` is now equivalent to `dict.has_key(key)`.

Files also provide an iterator, which calls the **`readline()`** method until there are no more lines in the file. This means you can now read each line of a file using code like this:

```
for line in file:
    # do something for each line
    ...
```

Note that you can only go forward in an iterator; there's no way to get the previous element, reset the iterator, or make a copy of it. An iterator object could provide such additional capabilities, but the iterator protocol only requires a **`next()`** method.

Ver também

PEP 234 **[https://peps.python.org/pep-0234/]** **- Iterators**
Written by Ka-Ping Yee and GvR; implemented by the Python Labs crew, mostly by GvR and Tim Peters.

PEP 255: Simple Generators

Generators are another new feature, one that interacts with the introduction of iterators.

You're doubtless familiar with how function calls work in Python or C. When you call a function, it gets a private namespace where its local variables are created. When the function reaches a **`return`** statement, the local variables are destroyed and the resulting value is returned to the caller. A later call to the same function will get a fresh new set of local variables. But, what if the local variables weren't thrown away on exiting a function? What if you could later resume the function where it left off? This is what generators provide; they can be thought of as resumable functions.

Here's the simplest example of a generator function:

```
def generate_ints(N):
    for i in range(N):
        yield i
```

A new keyword, **`yield`**, was introduced for generators. Any function containing a **`yield`** statement is a generator function; this is detected by Python's bytecode compiler which compiles the function specially as a result. Because a new keyword was introduced, generators must be explicitly enabled in a module by including a `from __future__ import generators` statement near the top of the module's source code. In Python 2.3 this statement will become unnecessary.

When you call a generator function, it doesn't return a single value; instead it returns a generator object that supports the iterator protocol. On executing the **`yield`** statement, the generator outputs the value of `i`, similar to a **`return`** statement. The big difference between **`yield`** and a **`return`** statement is that on reaching a **`yield`** the generator's state of execution is suspended and local variables are preserved. On the next call to the generator's `next()` method, the function will resume executing immediately after the **`yield`** statement. (For complicated reasons, the **`yield`** statement isn't allowed inside the **`try`** block of a **`try`**…**`finally`** statement; read **PEP 255** [https://peps.python.org/pep-0255/] for a full explanation of the interaction between **`yield`** and exceptions.)

Here's a sample usage of the **`generate_ints()`** generator:

```
>>> gen = generate_ints(3)
>>> gen
<generator object at 0x8117f90>
>>> gen.next()
0
>>> gen.next()
1
>>> gen.next()
2
>>> gen.next()
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
  File "<stdin>", line 2, in generate_ints
StopIteration
```

You could equally write `for i in generate_ints(5)`, or `a,b,c = generate_ints(3)`.

Inside a generator function, the **`return`** statement can only be used without a value, and signals the end of the procession of values; afterwards the generator cannot return any further values. **`return`** with a value, such as `return 5`, is a syntax error inside a generator function. The end of the generator's results can also be indicated by raising **`StopIteration`** manually, or by just letting the flow of execution fall off the bottom of the function.

You could achieve the effect of generators manually by writing your own class and storing all the local variables of the generator as instance variables. For example, returning a list of integers could be done by setting `self.count` to 0, and having the **`next()`** method increment `self.count` and return it. However, for a moderately complicated generator, writing a corresponding class would be much messier. `Lib/test/test_generators.py` contains a number of more interesting examples. The simplest one implements an in-order traversal of a tree using generators recursively.

```
# A recursive generator that generates Tree leaves in in-order.
def inorder(t):
```

```
    if t:
        for x in inorder(t.left):
            yield x
        yield t.label
        for x in inorder(t.right):
            yield x
```

Two other examples in `Lib/test/test_generators.py` produce solutions for the N-Queens problem (placing N queens on an NxN chess board so that no queen threatens another) and the Knight's Tour (a route that takes a knight to every square of an NxN chessboard without visiting any square twice).

The idea of generators comes from other programming languages, especially Icon (https://www2.cs.arizona.edu/icon/), where the idea of generators is central. In Icon, every expression and function call behaves like a generator. One example from "An Overview of the Icon Programming Language" at https://www2.cs.arizona.edu/icon/docs/ipd266.htm gives an idea of what this looks like:

```
sentence := "Store it in the neighboring harbor"
if (i := find("or", sentence)) > 5 then write(i)
```

In Icon the **`find()`** function returns the indexes at which the substring "or" is found: 3, 23, 33. In the **`if`** statement, `i` is first assigned a value of 3, but 3 is less than 5, so the comparison fails, and Icon retries it with the second value of 23. 23 is greater than 5, so the comparison now succeeds, and the code prints the value 23 to the screen.

Python doesn't go nearly as far as Icon in adopting generators as a central concept. Generators are considered a new part of the core Python language, but learning or using them isn't compulsory; if they don't solve any problems that you have, feel free to ignore them. One novel feature of Python's interface as compared to Icon's is that a generator's state is represented as a concrete object (the iterator) that can be passed around to other functions or stored in a data structure.

Ver também

PEP 255 **[https://peps.python.org/pep-0255/]** **- Simple Generators**

Written by Neil Schemenauer, Tim Peters, Magnus Lie Hetland. Implemented mostly by Neil Schemenauer and Tim Peters, with other fixes from the Python Labs crew.

PEP 237: Unifying Long Integers and Integers

In recent versions, the distinction between regular integers, which are 32-bit values on most machines, and long integers, which can be of arbitrary size, was becoming an annoyance. For example, on platforms that support files larger than `2**32` bytes, the **`tell()`** method of file objects has to return a long integer. However, there were various bits of Python that expected plain integers and would raise an error if a long integer was provided instead. For example, in Python 1.5, only regular integers could be used as a slice index, and `'abc'[1L:]` would raise a **`TypeError`** exception with the message 'slice index must be int'.

Python 2.2 will shift values from short to long integers as required. The 'L' suffix is no longer needed to indicate a long integer literal, as now the compiler will choose the appropriate type. (Using the 'L' suffix will be discouraged in future 2.x versions of Python, triggering a warning in Python 2.4, and probably dropped in Python 3.0.) Many operations that used to raise an **`OverflowError`** will now return a long integer as their result. For example:

```
>>> 1234567890123
1234567890123L
>>> 2 ** 64
18446744073709551616L
```

In most cases, integers and long integers will now be treated identically. You can still distinguish them with the **`type()`** built-in function, but that's rarely needed.

Ver também

PEP 237 [https://peps.python.org/pep-0237/] - Unifying Long Integers and Integers

Written by Moshe Zadka and Guido van Rossum. Implemented mostly by Guido van Rossum.

PEP 238: Changing the Division Operator

The most controversial change in Python 2.2 heralds the start of an effort to fix an old design flaw that's been in Python from the beginning. Currently Python's division operator, `/`, behaves like C's division operator when presented with two integer arguments: it returns an integer result that's truncated down when there would be a fractional part. For example, `3/2` is 1, not 1.5, and `(-1)/2` is -1, not -0.5. This means that the results of division can vary unexpectedly depending on the type of the two operands and because Python is dynamically typed, it can be difficult to determine the possible types of the operands.

(The controversy is over whether this is *really* a design flaw, and whether it's worth breaking existing code to fix this. It's caused endless discussions on python-dev, and in July 2001 erupted into a storm of acidly sarcastic postings on *comp.lang.python*. I won't argue for either side here and will stick to describing what's implemented in 2.2. Read **PEP 238** [https://peps.python.org/pep-0238/] for a summary of arguments and counter-arguments.)

Because this change might break code, it's being introduced very gradually. Python 2.2 begins the transition, but the switch won't be complete until Python 3.0.

First, I'll borrow some terminology from **PEP 238** [https://peps.python.org/pep-0238/]. "True division" is the division that most non-programmers are familiar with: 3/2 is 1.5, 1/4 is 0.25, and so forth. "Floor division" is what Python's `/` operator currently does when given integer operands; the result is the floor of the value returned by true division. "Classic division" is the current mixed behaviour of `/`; it returns the result of floor division when the

operands are integers, and returns the result of true division when one of the operands is a floating-point number.

Here are the changes 2.2 introduces:

- A new operator, `//`, is the floor division operator. (Yes, we know it looks like C++'s comment symbol.) `//` *always* performs floor division no matter what the types of its operands are, so `1 // 2` is 0 and `1.0 // 2.0` is also 0.0.

  `//` is always available in Python 2.2; you don't need to enable it using a `__future__` statement.

- By including a `from __future__ import division` in a module, the `/` operator will be changed to return the result of true division, so `1/2` is 0.5. Without the `__future__` statement, `/` still means classic division. The default meaning of `/` will not change until Python 3.0.

- Classes can define methods called `__truediv__()` and `__floordiv__()` to overload the two division operators. At the C level, there are also slots in the `PyNumberMethods` structure so extension types can define the two operators.

- Python 2.2 supports some command-line arguments for testing whether code will work with the changed division semantics. Running python with `-Q warn` will cause a warning to be issued whenever division is applied to two integers. You can use this to find code that's affected by the change and fix it. By default, Python 2.2 will simply perform classic division without a warning; the warning will be turned on by default in Python 2.3.

Ver também

PEP 238 [https://peps.python.org/pep-0238/] - Changing the Division Operator
Written by Moshe Zadka and Guido van Rossum. Implemented by Guido van Rossum..

Unicode Changes

Python's Unicode support has been enhanced a bit in 2.2. Unicode strings are usually stored as UCS-2, as 16-bit unsigned integers. Python 2.2 can also be compiled to use UCS-4, 32-bit unsigned integers, as its internal encoding by supplying **`--enable-unicode=ucs4`** to the configure script. (It's also possible to specify **`--disable-unicode`** to completely disable Unicode support.)

When built to use UCS-4 (a "wide Python"), the interpreter can natively handle Unicode characters from U+000000 to U+110000, so the range of legal values for the **`unichr()`** function is expanded accordingly. Using an interpreter compiled to use UCS-2 (a "narrow Python"), values greater than 65535 will still cause **`unichr()`** to raise a **`ValueError`** exception. This is all described in **PEP 261** [https://peps.python.org/pep-0261/], "Support for 'wide' Unicode characters"; consult it for further details.

Another change is simpler to explain. Since their introduction, Unicode strings have supported an **`encode()`** method to convert the string to a selected encoding such as UTF-8 or Latin-1. A symmetric `decode([*encoding*])` method has been added to 8-bit strings (though not to Unicode strings) in 2.2. **`decode()`** assumes that the string is in the specified encoding and decodes it, returning whatever is returned by the codec.

Using this new feature, codecs have been added for tasks not directly related to Unicode. For example, codecs have been added for uu-encoding, MIME's base64 encoding, and compression with the **`zlib`** module:

```
>>> s = """Here is a lengthy piece of redundant, overly verbose,
... and repetitive text.
... """
>>> data = s.encode('zlib')
>>> data
'x\x9c\r\xc9\xc1\r\x80 \x10\x04\xc0?Ul...'
>>> data.decode('zlib')
'Here is a lengthy piece of redundant, overly verbose,\nand repe
>>> print s.encode('uu')
```

```
begin 666 <data>
M2&5R92!I<R!A(&QE;F=T:'D@<&EE8V4@;V8@<F5D=6YD86YT+"!O=F5R;'D@
>=F5R8F]S92P*86YD(')E<&5T:71I=F4@=&5X="X*

end
>>> "sheesh".encode('rot-13')
'furrfu'
```

To convert a class instance to Unicode, a **`__unicode__()`** method can be defined by a class, analogous to **`__str__()`**.

`encode()`, **`decode()`**, and **`__unicode__()`** were implemented by Marc-André Lemburg. The changes to support using UCS-4 internally were implemented by Fredrik Lundh and Martin von Löwis.

Ver também

PEP 261 [https://peps.python.org/pep-0261/] - Support for 'wide' Unicode characters
Written by Paul Prescod.

PEP 227: Nested Scopes

In Python 2.1, statically nested scopes were added as an optional feature, to be enabled by a `from __future__ import nested_scopes` directive. In 2.2 nested scopes no longer need to be specially enabled, and are now always present. The rest of this section is a copy of the description of nested scopes from my "What's New in Python 2.1" document; if you read it when 2.1 came out, you can skip the rest of this section.

The largest change introduced in Python 2.1, and made complete in 2.2, is to Python's scoping rules. In Python 2.0, at any given time there are at most three namespaces used to look up variable names: local, module-level, and the built-in namespace. This often surprised people because it didn't match their intuitive expectations. For example, a nested recursive function definition doesn't work:

```
def f():
    ...
    def g(value):
        ...
        return g(value-1) + 1
    ...
```

The function **`g()`** will always raise a **`NameError`** exception, because the binding of the name `g` isn't in either its local namespace or in the module-level namespace. This isn't much of a problem in practice (how often do you recursively define interior functions like this?), but this also made using the **`lambda`** expression clumsier, and this was a problem in practice. In code which uses **`lambda`** you can often find local variables being copied by passing them as the default values of arguments.

```
def find(self, name):
    "Return list of any entries equal to 'name'"
    L = filter(lambda x, name=name: x == name,
               self.list_attribute)
    return L
```

The readability of Python code written in a strongly functional style suffers greatly as a result.

The most significant change to Python 2.2 is that static scoping has been added to the language to fix this problem. As a first effect, the `name=name` default argument is now unnecessary in the above example. Put simply, when a given variable name is not assigned a value within a function (by an assignment, or the **`def`**, **`class`**, or **`import`** statements), references to the variable will be looked up in the local namespace of the enclosing scope. A more detailed explanation of the rules, and a dissection of the implementation, can be found in the PEP.

This change may cause some compatibility problems for code where the same variable name is used both at the module level and as a local variable within a function that contains further function definitions. This seems rather unlikely though, since such code would have been pretty confusing to read in the first place.

One side effect of the change is that the `from module import *` and `exec` statements have been made illegal inside a function scope under certain conditions. The Python reference manual has said all along that `from module import *` is only legal at the top level of a module, but the CPython interpreter has never enforced this before. As part of the implementation of nested scopes, the compiler which turns Python source into bytecodes has to generate different code to access variables in a containing scope. `from module import *` and `exec` make it impossible for the compiler to figure this out, because they add names to the local namespace that are unknowable at compile time. Therefore, if a function contains function definitions or **`lambda`** expressions with free variables, the compiler will flag this by raising a **`SyntaxError`** exception.

To make the preceding explanation a bit clearer, here's an example:

```
x = 1
def f():
    # The next line is a syntax error
    exec 'x=2'
    def g():
        return x
```

Line 4 containing the `exec` statement is a syntax error, since `exec` would define a new local variable named `x` whose value should be accessed by **`g()`**.

This shouldn't be much of a limitation, since `exec` is rarely used in most Python code (and when it is used, it's often a sign of a poor design anyway).

Ver também

PEP 227 **[https://peps.python.org/pep-0227/]** **- Statically Nested Scopes**
Written and implemented by Jeremy Hylton.

Módulos novos ou aprimorados

- The **xmlrpclib** module was contributed to the standard library by Fredrik Lundh, providing support for writing XML-RPC clients. XML-RPC is a simple remote procedure call protocol built on top of HTTP and XML. For example, the following snippet retrieves a list of RSS channels from the O'Reilly Network, and then lists the recent headlines for one channel:

```
import xmlrpclib
s = xmlrpclib.Server(
      'http://www.oreillynet.com/meerkat/xml-rpc/server.php
channels = s.meerkat.getChannels()
# channels is a list of dictionaries, like this:
# [{'id': 4, 'title': 'Freshmeat Daily News'}
#  {'id': 190, 'title': '32Bits Online'},
#  {'id': 4549, 'title': '3DGamers'}, ... ]

# Get the items for one channel
items = s.meerkat.getItems( {'channel': 4} )

# 'items' is another list of dictionaries, like this:
# [{'link': 'http://freshmeat.net/releases/52719/',
#   'description': 'A utility which converts HTML to XSL FO
#   'title': 'html2fo 0.3 (Default)'}, ... ]
```

  The **SimpleXMLRPCServer** module makes it easy to create straightforward XML-RPC servers. See http://xmlrpc.scripting.com/ for more information about XML-RPC.

- The new **hmac** module implements the HMAC algorithm described by **RFC 2104** [https://datatracker.ietf.org/doc/html/rfc2104.html]. (Contributed by Gerhard Häring.)

- Several functions that originally returned lengthy tuples now return pseudo-sequences that still behave like tuples but also have mnemonic attributes such as **memberst_mtime** or **tm_year**. The enhanced functions include **stat()**, **fstat()**, **statvfs()**, and **fstatvfs()** in the **os** module, and **localtime()**, **gmtime()**, and **strptime()** in the **time** module.

For example, to obtain a file's size using the old tuples, you'd end up writing something like `file_size = os.stat(filename)[stat.ST_SIZE]`, but now this can be written more clearly as `file_size = os.stat(filename).st_size`.

The original patch for this feature was contributed by Nick Mathewson.

- The Python profiler has been extensively reworked and various errors in its output have been corrected. (Contributed by Fred L. Drake, Jr. and Tim Peters.)

- The **`socket`** module can be compiled to support IPv6; specify the **`--enable-ipv6`** option to Python's configure script. (Contributed by Jun-ichiro "itojun" Hagino.)

- Two new format characters were added to the **`struct`** module for 64-bit integers on platforms that support the C long long type. `q` is for a signed 64-bit integer, and `Q` is for an unsigned one. The value is returned in Python's long integer type. (Contributed by Tim Peters.)

- In the interpreter's interactive mode, there's a new built-in function **`help()`** that uses the **`pydoc`** module introduced in Python 2.1 to provide interactive help. `help(object)` displays any available help text about *object*. **`help()`** with no argument puts you in an online help utility, where you can enter the names of functions, classes, or modules to read their help text. (Contributed by Guido van Rossum, using Ka-Ping Yee's **`pydoc`** module.)

- Various bugfixes and performance improvements have been made to the SRE engine underlying the **`re`** module. For example, the **`re.sub()`** and **`re.split()`** functions have been rewritten in C. Another contributed patch speeds up certain Unicode character ranges by a factor of two, and a new **`finditer()`** method that returns an iterator over all the non-overlapping matches in a given string. (SRE is maintained by Fredrik Lundh. The BIGCHARSET patch was contributed by Martin von Löwis.)

- The **smtplib** module now supports **RFC 2487** [https://datatracker.ietf.org/doc/html/rfc2487.html], "Secure SMTP over TLS", so it's now possible to encrypt the SMTP traffic between a Python program and the mail transport agent being handed a message. **smtplib** also supports SMTP authentication. (Contributed by Gerhard Häring.)

- The **imaplib** module, maintained by Piers Lauder, has support for several new extensions: the NAMESPACE extension defined in **RFC 2342** [https://datatracker.ietf.org/doc/html/rfc2342.html], SORT, GETACL and SETACL. (Contributed by Anthony Baxter and Michel Pelletier.)

- The **rfc822** module's parsing of email addresses is now compliant with **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html], an update to **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html]. (The module's name is *not* going to be changed to `rfc2822`.) A new package, **email**, has also been added for parsing and generating e-mail messages. (Contributed by Barry Warsaw, and arising out of his work on Mailman.)

- The **difflib** module now contains a new **Differ** class for producing human-readable lists of changes (a "delta") between two sequences of lines of text. There are also two generator functions, **ndiff()** and **restore()**, which respectively return a delta from two sequences, or one of the original sequences from a delta. (Grunt work contributed by David Goodger, from ndiff.py code by Tim Peters who then did the generatorization.)

- New constants **ascii_letters**, **ascii_lowercase**, and **ascii_uppercase** were added to the **string** module. There were several modules in the standard library that used **string.letters** to mean the ranges A-Za-z, but that assumption is incorrect when locales are in use, because **string.letters** varies depending on the set of legal characters defined by the current locale. The buggy modules have all been fixed to use **ascii_letters** instead. (Reported by an unknown person; fixed by Fred L. Drake, Jr.)

- The **mimetypes** module now makes it easier to use alternative MIME-type databases by the addition of a **MimeTypes** class, which takes a list

of filenames to be parsed. (Contributed by Fred L. Drake, Jr.)

- A `Timer` class was added to the `threading` module that allows scheduling an activity to happen at some future time. (Contributed by Itamar Shtull-Trauring.)

Interpreter Changes and Fixes

Some of the changes only affect people who deal with the Python interpreter at the C level because they're writing Python extension modules, embedding the interpreter, or just hacking on the interpreter itself. If you only write Python code, none of the changes described here will affect you very much.

- Profiling and tracing functions can now be implemented in C, which can operate at much higher speeds than Python-based functions and should reduce the overhead of profiling and tracing. This will be of interest to authors of development environments for Python. Two new C functions were added to Python's API, `PyEval_SetProfile()` and `PyEval_SetTrace()`. The existing `sys.setprofile()` and `sys.settrace()` functions still exist, and have simply been changed to use the new C-level interface. (Contributed by Fred L. Drake, Jr.)

- Another low-level API, primarily of interest to implementers of Python debuggers and development tools, was added. `PyInterpreterState_Head()` and `PyInterpreterState_Next()` let a caller walk through all the existing interpreter objects; `PyInterpreterState_ThreadHead()` and `PyThreadState_Next()` allow looping over all the thread states for a given interpreter. (Contributed by David Beazley.)

- The C-level interface to the garbage collector has been changed to make it easier to write extension types that support garbage collection and to debug misuses of the functions. Various functions have slightly different semantics, so a bunch of functions had to be renamed. Extensions that use the old API will still compile but will *not*

participate in garbage collection, so updating them for 2.2 should be considered fairly high priority.

To upgrade an extension module to the new API, perform the following steps:

- Rename **Py_TPFLAGS_GC** to **Py_TPFLAGS_HAVE_GC**.
- Use **PyObject_GC_New()** or **PyObject_GC_NewVar()** to allocate objects, and **PyObject_GC_Del()** to deallocate them.
- Rename **PyObject_GC_Init()** to **PyObject_GC_Track()** and **PyObject_GC_Fini()** to **PyObject_GC_UnTrack()**.
- Remove **PyGC_HEAD_SIZE** from object size calculations.
- Remove calls to **PyObject_AS_GC()** and **PyObject_FROM_GC()**.
- A new `et` format sequence was added to **PyArg_ParseTuple()**; `et` takes both a parameter and an encoding name, and converts the parameter to the given encoding if the parameter turns out to be a Unicode string, or leaves it alone if it's an 8-bit string, assuming it to already be in the desired encoding. This differs from the `es` format character, which assumes that 8-bit strings are in Python's default ASCII encoding and converts them to the specified new encoding. (Contributed by M.-A. Lemburg, and used for the MBCS support on Windows described in the following section.)
- A different argument parsing function, **PyArg_UnpackTuple()**, has been added that's simpler and presumably faster. Instead of specifying a format string, the caller simply gives the minimum and maximum number of arguments expected, and a set of pointers to PyObject* variables that will be filled in with argument values.
- Two new flags **METH_NOARGS** and **METH_O** are available in method definition tables to simplify implementation of methods with no arguments or a single untyped argument. Calling such methods is more efficient than calling a corresponding method that uses **METH_VARARGS**.

Also, the old **METH_OLDARGS** style of writing C methods is now officially deprecated.

- Two new wrapper functions, **PyOS_snprintf()** and **PyOS_vsnprintf()** were added to provide cross-platform implementations for the relatively new **snprintf()** and **vsnprintf()** C lib APIs. In contrast to the standard **sprintf()** and **vsprintf()** functions, the Python versions check the bounds of the buffer used to protect against buffer overruns. (Contributed by M.-A. Lemburg.)

- The **_PyTuple_Resize()** function has lost an unused parameter, so now it takes 2 parameters instead of 3. The third argument was never used, and can simply be discarded when porting code from earlier versions to Python 2.2.

Outras alterações e correções

As usual there were a bunch of other improvements and bugfixes scattered throughout the source tree. A search through the CVS change logs finds there were 527 patches applied and 683 bugs fixed between Python 2.1 and 2.2; 2.2.1 applied 139 patches and fixed 143 bugs; 2.2.2 applied 106 patches and fixed 82 bugs. These figures are likely to be underestimates.

Some of the more notable changes are:

- The code for the MacOS port for Python, maintained by Jack Jansen, is now kept in the main Python CVS tree, and many changes have been made to support MacOS X.

  The most significant change is the ability to build Python as a framework, enabled by supplying the **--enable-framework** option to the configure script when compiling Python. According to Jack Jansen, “This installs a self-contained Python installation plus the OS X framework “glue” into `/Library/Frameworks/Python.framework` (or another location of choice). For now there is little immediate added benefit to this (actually, there is the disadvantage that you have to change your PATH to be able to find Python), but it is the basis for

creating a full-blown Python application, porting the MacPython IDE, possibly using Python as a standard OSA scripting language and much more."

Most of the MacPython toolbox modules, which interface to MacOS APIs such as windowing, QuickTime, scripting, etc. have been ported to OS X, but they've been left commented out in `setup.py`. People who want to experiment with these modules can uncomment them manually.

- Keyword arguments passed to built-in functions that don't take them now cause a `TypeError` exception to be raised, with the message "*function* takes no keyword arguments".

- Weak references, added in Python 2.1 as an extension module, are now part of the core because they're used in the implementation of new-style classes. The `ReferenceError` exception has therefore moved from the `weakref` module to become a built-in exception.

- A new script, `Tools/scripts/cleanfuture.py` by Tim Peters, automatically removes obsolete `__future__` statements from Python source code.

- An additional *flags* argument has been added to the built-in function `compile()`, so the behaviour of `__future__` statements can now be correctly observed in simulated shells, such as those presented by IDLE and other development environments. This is described in **PEP 264** [https://peps.python.org/pep-0264/]. (Contributed by Michael Hudson.)

- The new license introduced with Python 1.6 wasn't GPL-compatible. This is fixed by some minor textual changes to the 2.2 license, so it's now legal to embed Python inside a GPLed program again. Note that Python itself is not GPLed, but instead is under a license that's essentially equivalent to the BSD license, same as it always was. The license changes were also applied to the Python 2.0.1 and 2.1.1 releases.

- When presented with a Unicode filename on Windows, Python will now convert it to an MBCS encoded string, as used by the Microsoft file APIs. As MBCS is explicitly used by the file APIs, Python's choice of ASCII as the default encoding turns out to be an annoyance. On Unix, the locale's character set is used if `locale.nl_langinfo(CODESET)` is available. (Windows support was contributed by Mark Hammond with assistance from Marc-André Lemburg. Unix support was added by Martin von Löwis.)

- Large file support is now enabled on Windows. (Contributed by Tim Peters.)

- The `Tools/scripts/ftpmirror.py` script now parses a `.netrc` file, if you have one. (Contributed by Mike Romberg.)

- Some features of the object returned by the **`xrange()`** function are now deprecated, and trigger warnings when they're accessed; they'll disappear in Python 2.3. **`xrange`** objects tried to pretend they were full sequence types by supporting slicing, sequence multiplication, and the **`in`** operator, but these features were rarely used and therefore buggy. The **`tolist()`** method and the **`start`**, **`stop`**, and **`step`** attributes are also being deprecated. At the C level, the fourth argument to the **`PyRange_New()`** function, `repeat`, has also been deprecated.

- There were a bunch of patches to the dictionary implementation, mostly to fix potential core dumps if a dictionary contains objects that sneakily changed their hash value, or mutated the dictionary they were contained in. For a while python-dev fell into a gentle rhythm of Michael Hudson finding a case that dumped core, Tim Peters fixing the bug, Michael finding another case, and round and round it went.

- On Windows, Python can now be compiled with Borland C thanks to a number of patches contributed by Stephen Hansen, though the result isn't fully functional yet. (But this *is* progress…)

- Another Windows enhancement: Wise Solutions generously offered PythonLabs use of their InstallerMaster 8.1 system. Earlier

PythonLabs Windows installers used Wise 5.0a, which was beginning to show its age. (Packaged up by Tim Peters.)

- Files ending in `.pyw` can now be imported on Windows. `.pyw` is a Windows-only thing, used to indicate that a script needs to be run using PYTHONW.EXE instead of PYTHON.EXE in order to prevent a DOS console from popping up to display the output. This patch makes it possible to import such scripts, in case they're also usable as modules. (Implemented by David Bolen.)

- On platforms where Python uses the C **`dlopen()`** function to load extension modules, it's now possible to set the flags used by **`dlopen()`** using the **`sys.getdlopenflags()`** and **`sys.setdlopenflags()`** functions. (Contributed by Bram Stolk.)

- The **`pow()`** built-in function no longer supports 3 arguments when floating-point numbers are supplied. `pow(x, y, z)` returns `(x**y) % z`, but this is never useful for floating-point numbers, and the final result varies unpredictably depending on the platform. A call such as `pow(2.0, 8.0, 7.0)` will now raise a **`TypeError`** exception.

Reconhecimentos

The author would like to thank the following people for offering suggestions, corrections and assistance with various drafts of this article: Fred Bremmer, Keith Briggs, Andrew Dalke, Fred L. Drake, Jr., Carel Fellinger, David Goodger, Mark Hammond, Stephen Hansen, Michael Hudson, Jack Jansen, Marc-André Lemburg, Martin von Löwis, Fredrik Lundh, Michael McLay, Nick Mathewson, Paul Moore, Gustavo Niemeyer, Don O'Donnell, Joonas Paalasma, Tim Peters, Jens Quade, Tom Reinhardt, Neil Schemenauer, Guido van Rossum, Greg Ward, Edward Welbourne.

What’s New in Python 2.1

Autor:: A.M. Kuchling

Introdução

This article explains the new features in Python 2.1. While there aren’t as many changes in 2.1 as there were in Python 2.0, there are still some pleasant surprises in store. 2.1 is the first release to be steered through the use of Python Enhancement Proposals, or PEPs, so most of the sizable changes have accompanying PEPs that provide more complete documentation and a design rationale for the change. This article doesn’t attempt to document the new features completely, but simply provides an overview of the new features for Python programmers. Refer to the Python 2.1 documentation, or to the specific PEP, for more details about any new feature that particularly interests you.

One recent goal of the Python development team has been to accelerate the pace of new releases, with a new release coming every 6 to 9 months. 2.1 is the first release to come out at this faster pace, with the first alpha appearing in January, 3 months after the final version of 2.0 was released.

The final release of Python 2.1 was made on April 17, 2001.

PEP 227: Nested Scopes

The largest change in Python 2.1 is to Python’s scoping rules. In Python 2.0, at any given time there are at most three namespaces used to look up variable names: local, module-level, and the built-in namespace. This often surprised people because it didn’t match their intuitive expectations. For example, a nested recursive function definition doesn’t work:

```
def f():
    ...
```

```
    def g(value):
        ...
        return g(value-1) + 1
    ...
```

The function **g()** will always raise a **NameError** exception, because the binding of the name `g` isn't in either its local namespace or in the module-level namespace. This isn't much of a problem in practice (how often do you recursively define interior functions like this?), but this also made using the **lambda** expression clumsier, and this was a problem in practice. In code which uses **lambda** you can often find local variables being copied by passing them as the default values of arguments.

```
def find(self, name):
    "Return list of any entries equal to 'name'"
    L = filter(lambda x, name=name: x == name,
               self.list_attribute)
    return L
```

The readability of Python code written in a strongly functional style suffers greatly as a result.

The most significant change to Python 2.1 is that static scoping has been added to the language to fix this problem. As a first effect, the `name=name` default argument is now unnecessary in the above example. Put simply, when a given variable name is not assigned a value within a function (by an assignment, or the **def**, **class**, or **import** statements), references to the variable will be looked up in the local namespace of the enclosing scope. A more detailed explanation of the rules, and a dissection of the implementation, can be found in the PEP.

This change may cause some compatibility problems for code where the same variable name is used both at the module level and as a local variable within a function that contains further function definitions. This seems rather unlikely though, since such code would have been pretty confusing to read in the first place.

One side effect of the change is that the `from module import *` and `exec` statements have been made illegal inside a function scope under certain

conditions. The Python reference manual has said all along that `from module import *` is only legal at the top level of a module, but the CPython interpreter has never enforced this before. As part of the implementation of nested scopes, the compiler which turns Python source into bytecodes has to generate different code to access variables in a containing scope. `from module import *` and `exec` make it impossible for the compiler to figure this out, because they add names to the local namespace that are unknowable at compile time. Therefore, if a function contains function definitions or **`lambda`** expressions with free variables, the compiler will flag this by raising a **`SyntaxError`** exception.

To make the preceding explanation a bit clearer, here's an example:

```
x = 1
def f():
    # The next line is a syntax error
    exec 'x=2'
    def g():
        return x
```

Line 4 containing the `exec` statement is a syntax error, since `exec` would define a new local variable named `x` whose value should be accessed by **`g()`**.

This shouldn't be much of a limitation, since `exec` is rarely used in most Python code (and when it is used, it's often a sign of a poor design anyway).

Compatibility concerns have led to nested scopes being introduced gradually; in Python 2.1, they aren't enabled by default, but can be turned on within a module by using a future statement as described in **PEP 236** [https://peps.python.org/pep-0236/]. (See the following section for further discussion of **PEP 236** [https://peps.python.org/pep-0236/].) In Python 2.2, nested scopes will become the default and there will be no way to turn them off, but users will have had all of 2.1's lifetime to fix any breakage resulting from their introduction.

Ver também

PEP 227 **[https://peps.python.org/pep-0227/]** **- Statically Nested Scopes**
Written and implemented by Jeremy Hylton.

PEP 236: __future__ Directives

The reaction to nested scopes was widespread concern about the dangers of breaking code with the 2.1 release, and it was strong enough to make the Pythoneers take a more conservative approach. This approach consists of introducing a convention for enabling optional functionality in release N that will become compulsory in release N+1.

The syntax uses a `from...import` statement using the reserved module name `__future__`. Nested scopes can be enabled by the following statement:

```
from __future__ import nested_scopes
```

While it looks like a normal `import` statement, it's not; there are strict rules on where such a future statement can be put. They can only be at the top of a module, and must precede any Python code or regular `import` statements. This is because such statements can affect how the Python bytecode compiler parses code and generates bytecode, so they must precede any statement that will result in bytecodes being produced.

Ver também

PEP 236 **[https://peps.python.org/pep-0236/]** **- Back to the `__future__`**
Written by Tim Peters, and primarily implemented by Jeremy Hylton.

PEP 207: Rich Comparisons

In earlier versions, Python's support for implementing comparisons on user-defined classes and extension types was quite simple. Classes could implement a `__cmp__()` method that was given two instances of a class, and

could only return 0 if they were equal or +1 or -1 if they weren't; the method couldn't raise an exception or return anything other than a Boolean value. Users of Numeric Python often found this model too weak and restrictive, because in the number-crunching programs that numeric Python is used for, it would be more useful to be able to perform elementwise comparisons of two matrices, returning a matrix containing the results of a given comparison for each element. If the two matrices are of different sizes, then the compare has to be able to raise an exception to signal the error.

In Python 2.1, rich comparisons were added in order to support this need. Python classes can now individually overload each of the <, <=, >, >=, ==, and `!=` operations. The new magic method names are:

Operação	Nome do método
<	`__lt__()`
<=	`__le__()`
>	`__gt__()`
>=	`__ge__()`
==	`__eq__()`
!=	`__ne__()`

(The magic methods are named after the corresponding Fortran operators `.LT.`. `.LE.`, &c. Numeric programmers are almost certainly quite familiar

with these names and will find them easy to remember.)

Each of these magic methods is of the form `method(self, other)`, where `self` will be the object on the left-hand side of the operator, while `other` will be the object on the right-hand side. For example, the expression `A < B` will cause `A.__lt__(B)` to be called.

Each of these magic methods can return anything at all: a Boolean, a matrix, a list, or any other Python object. Alternatively they can raise an exception if the comparison is impossible, inconsistent, or otherwise meaningless.

The built-in `cmp(A,B)` function can use the rich comparison machinery, and now accepts an optional argument specifying which comparison operation to use; this is given as one of the strings `"<"`, `"<="`, `">"`, `">="`, `"=="`, or `"!="`. If called without the optional third argument, **`cmp()`** will only return -1, 0, or +1 as in previous versions of Python; otherwise it will call the appropriate method and can return any Python object.

There are also corresponding changes of interest to C programmers; there's a new slot `tp_richcmp` in type objects and an API for performing a given rich comparison. I won't cover the C API here, but will refer you to **PEP 207** [https://peps.python.org/pep-0207/], or to 2.1's C API documentation, for the full list of related functions.

Ver também

PEP 207 [https://peps.python.org/pep-0207/] - Rich Comparisons
Written by Guido van Rossum, heavily based on earlier work by David Ascher, and implemented by Guido van Rossum.

PEP 230: Warning Framework

Over its 10 years of existence, Python has accumulated a certain number of obsolete modules and features along the way. It's difficult to know when a feature is safe to remove, since there's no way of knowing how much code

uses it — perhaps no programs depend on the feature, or perhaps many do. To enable removing old features in a more structured way, a warning framework was added. When the Python developers want to get rid of a feature, it will first trigger a warning in the next version of Python. The following Python version can then drop the feature, and users will have had a full release cycle to remove uses of the old feature.

Python 2.1 adds the warning framework to be used in this scheme. It adds a **warnings** module that provide functions to issue warnings, and to filter out warnings that you don't want to be displayed. Third-party modules can also use this framework to deprecate old features that they no longer wish to support.

For example, in Python 2.1 the **regex** module is deprecated, so importing it causes a warning to be printed:

```
>>> import regex
__main__:1: DeprecationWarning: the regex module
         is deprecated; please use the re module
>>>
```

Warnings can be issued by calling the **warnings.warn()** function:

```
warnings.warn("feature X no longer supported")
```

The first parameter is the warning message; an additional optional parameters can be used to specify a particular warning category.

Filters can be added to disable certain warnings; a regular expression pattern can be applied to the message or to the module name in order to suppress a warning. For example, you may have a program that uses the **regex** module and not want to spare the time to convert it to use the **re** module right now. The warning can be suppressed by calling

```
import warnings
warnings.filterwarnings(action = 'ignore',
                        message='.*regex module is deprecated',
                        category=DeprecationWarning,
                        module = '__main__')
```

This adds a filter that will apply only to warnings of the class `DeprecationWarning` triggered in the `__main__` module, and applies a regular expression to only match the message about the `regex` module being deprecated, and will cause such warnings to be ignored. Warnings can also be printed only once, printed every time the offending code is executed, or turned into exceptions that will cause the program to stop (unless the exceptions are caught in the usual way, of course).

Functions were also added to Python's C API for issuing warnings; refer to PEP 230 or to Python's API documentation for the details.

Ver também

PEP 5 [https://peps.python.org/pep-0005/] - Guidelines for Language Evolution

Written by Paul Prescod, to specify procedures to be followed when removing old features from Python. The policy described in this PEP hasn't been officially adopted, but the eventual policy probably won't be too different from Prescod's proposal.

PEP 230 [https://peps.python.org/pep-0230/] - Warning Framework

Written and implemented by Guido van Rossum.

PEP 229: New Build System

When compiling Python, the user had to go in and edit the `Modules/Setup` file in order to enable various additional modules; the default set is relatively small and limited to modules that compile on most Unix platforms. This means that on Unix platforms with many more features, most notably Linux, Python installations often don't contain all useful modules they could.

Python 2.0 added the Distutils, a set of modules for distributing and installing extensions. In Python 2.1, the Distutils are used to compile much of the standard library of extension modules, autodetecting which ones are

supported on the current machine. It's hoped that this will make Python installations easier and more featureful.

Instead of having to edit the `Modules/Setup` file in order to enable modules, a `setup.py` script in the top directory of the Python source distribution is run at build time, and attempts to discover which modules can be enabled by examining the modules and header files on the system. If a module is configured in `Modules/Setup`, the `setup.py` script won't attempt to compile that module and will defer to the `Modules/Setup` file's contents. This provides a way to specific any strange command-line flags or libraries that are required for a specific platform.

In another far-reaching change to the build mechanism, Neil Schemenauer restructured things so Python now uses a single makefile that isn't recursive, instead of makefiles in the top directory and in each of the `Python/`, `Parser/`, `Objects/`, and `Modules/` subdirectories. This makes building Python faster and also makes hacking the Makefiles clearer and simpler.

Ver também

PEP 229 **[https://peps.python.org/pep-0229/]** **- Using Distutils to Build Python**
Written and implemented by A.M. Kuchling.

PEP 205: Weak References

Weak references, available through the `weakref` module, are a minor but useful new data type in the Python programmer's toolbox.

Storing a reference to an object (say, in a dictionary or a list) has the side effect of keeping that object alive forever. There are a few specific cases where this behaviour is undesirable, object caches being the most common one, and another being circular references in data structures such as trees.

For example, consider a memoizing function that caches the results of another function `f(x)` by storing the function's argument and its result in a

dictionary:

```
_cache = {}
def memoize(x):
    if _cache.has_key(x):
        return _cache[x]

    retval = f(x)

    # Cache the returned object
    _cache[x] = retval

    return retval
```

This version works for simple things such as integers, but it has a side effect; the `_cache` dictionary holds a reference to the return values, so they'll never be deallocated until the Python process exits and cleans up. This isn't very noticeable for integers, but if **`f()`** returns an object, or a data structure that takes up a lot of memory, this can be a problem.

Weak references provide a way to implement a cache that won't keep objects alive beyond their time. If an object is only accessible through weak references, the object will be deallocated and the weak references will now indicate that the object it referred to no longer exists. A weak reference to an object *obj* is created by calling `wr = weakref.ref(obj)`. The object being referred to is returned by calling the weak reference as if it were a function: `wr()`. It will return the referenced object, or `None` if the object no longer exists.

This makes it possible to write a **`memoize()`** function whose cache doesn't keep objects alive, by storing weak references in the cache.

```
_cache = {}
def memoize(x):
    if _cache.has_key(x):
        obj = _cache[x]()
        # If weak reference object still exists,
        # return it
        if obj is not None: return obj
```

```
    retval = f(x)

    # Cache a weak reference
    _cache[x] = weakref.ref(retval)

    return retval
```

The **weakref** module also allows creating proxy objects which behave like weak references — an object referenced only by proxy objects is deallocated – but instead of requiring an explicit call to retrieve the object, the proxy transparently forwards all operations to the object as long as the object still exists. If the object is deallocated, attempting to use a proxy will cause a **weakref.ReferenceError** exception to be raised.

```
proxy = weakref.proxy(obj)
proxy.attr   # Equivalent to obj.attr
proxy.meth() # Equivalent to obj.meth()
del obj
proxy.attr   # raises weakref.ReferenceError
```

Ver também

PEP 205 **[https://peps.python.org/pep-0205/]** - **Weak References**
Written and implemented by Fred L. Drake, Jr.

PEP 232: Function Attributes

In Python 2.1, functions can now have arbitrary information attached to them. People were often using docstrings to hold information about functions and methods, because the **__doc__** attribute was the only way of attaching any information to a function. For example, in the Zope web application server, functions are marked as safe for public access by having a docstring, and in John Aycock's SPARK parsing framework, docstrings hold parts of the BNF grammar to be parsed. This overloading is unfortunate, since docstrings are really intended to hold a function's documentation; for example, it means you can't properly document functions intended for private use in Zope.

Arbitrary attributes can now be set and retrieved on functions using the regular Python syntax:

```
def f(): pass

f.publish = 1
f.secure = 1
f.grammar = "A ::= B (C D)*"
```

The dictionary containing attributes can be accessed as the function's `__dict__`. Unlike the `__dict__` attribute of class instances, in functions you can actually assign a new dictionary to `__dict__`, though the new value is restricted to a regular Python dictionary; you *can't* be tricky and set it to a `UserDict` instance, or any other random object that behaves like a mapping.

Ver também

PEP 232 **[https://peps.python.org/pep-0232/]** **- Function Attributes**
Written and implemented by Barry Warsaw.

PEP 235: Importing Modules on Case-Insensitive Platforms

Some operating systems have filesystems that are case-insensitive, MacOS and Windows being the primary examples; on these systems, it's impossible to distinguish the filenames `FILE.PY` and `file.py`, even though they do store the file's name in its original case (they're case-preserving, too).

In Python 2.1, the `import` statement will work to simulate case-sensitivity on case-insensitive platforms. Python will now search for the first case-sensitive match by default, raising an `ImportError` if no such file is found, so `import file` will not import a module named `FILE.PY`. Case-insensitive matching can be requested by setting the `PYTHONCASEOK` environment variable before starting the Python interpreter.

PEP 217: Interactive Display Hook

When using the Python interpreter interactively, the output of commands is displayed using the built-in `repr()` function. In Python 2.1, the variable `sys.displayhook()` can be set to a callable object which will be called instead of **`repr()`**. For example, you can set it to a special pretty-printing function:

```
>>> # Create a recursive data structure
... L = [1,2,3]
>>> L.append(L)
>>> L # Show Python's default output
[1, 2, 3, [...]]
>>> # Use pprint.pprint() as the display function
... import sys, pprint
>>> sys.displayhook = pprint.pprint
>>> L
[1, 2, 3,  <Recursion on list with id=135143996>]
>>>
```

Ver também

PEP 217 [https://peps.python.org/pep-0217/] - Display Hook for Interactive Use

Written and implemented by Moshe Zadka.

PEP 208: New Coercion Model

How numeric coercion is done at the C level was significantly modified. This will only affect the authors of C extensions to Python, allowing them more flexibility in writing extension types that support numeric operations.

Extension types can now set the type flag `Py_TPFLAGS_CHECKTYPES` in their `PyTypeObject` structure to indicate that they support the new coercion model. In such extension types, the numeric slot functions can no longer assume that they'll be passed two arguments of the same type; instead they may be passed two arguments of differing types, and can then perform their own internal coercion. If the slot function is passed a type it can't handle, it can indicate the failure by returning a reference to the `Py_NotImplemented`

singleton value. The numeric functions of the other type will then be tried, and perhaps they can handle the operation; if the other type also returns `Py_NotImplemented`, then a **`TypeError`** will be raised. Numeric methods written in Python can also return `Py_NotImplemented`, causing the interpreter to act as if the method did not exist (perhaps raising a **`TypeError`**, perhaps trying another object's numeric methods).

Ver também

PEP 208 **[https://peps.python.org/pep-0208/]** **- Reworking the Coercion Model**
Written and implemented by Neil Schemenauer, heavily based upon earlier work by Marc-André Lemburg. Read this to understand the fine points of how numeric operations will now be processed at the C level.

PEP 241: Metadata in Python Packages

A common complaint from Python users is that there's no single catalog of all the Python modules in existence. T. Middleton's Vaults of Parnassus at `www.vex.net/parnassus/` (retired in February 2009, available in the Internet Archive Wayback Machine [https://web.archive.org/web/20090130140102/http://www.vex.net/parnassus/]) was the largest catalog of Python modules, but registering software at the Vaults is optional, and many people did not bother.

As a first small step toward fixing the problem, Python software packaged using the Distutils **sdist** command will include a file named `PKG-INFO` containing information about the package such as its name, version, and author (metadata, in cataloguing terminology). **PEP 241** [https://peps.python.org/pep-0241/] contains the full list of fields that can be present in the `PKG-INFO` file. As people began to package their software using Python 2.1, more and more packages will include metadata, making it possible to build automated cataloguing systems and experiment with them. With the result experience, perhaps it'll be possible to design a really good catalog and then build support for it into Python 2.2. For example, the

Distutils **sdist** and **bdist_*** commands could support an `upload` option that would automatically upload your package to a catalog server.

You can start creating packages containing `PKG-INFO` even if you're not using Python 2.1, since a new release of the Distutils will be made for users of earlier Python versions. Version 1.0.2 of the Distutils includes the changes described in **PEP 241** [https://peps.python.org/pep-0241/], as well as various bugfixes and enhancements. It will be available from the Distutils SIG at https://www.python.org/community/sigs/current/distutils-sig/.

Ver também

PEP 241 [https://peps.python.org/pep-0241/] - Metadata for Python Software Packages

Written and implemented by A.M. Kuchling.

PEP 243 [https://peps.python.org/pep-0243/] - Module Repository Upload Mechanism

Written by Sean Reifschneider, this draft PEP describes a proposed mechanism for uploading Python packages to a central server.

Módulos novos ou aprimorados

- Ka-Ping Yee contributed two new modules: **`inspect.py`**, a module for getting information about live Python code, and **`pydoc.py`**, a module for interactively converting docstrings to HTML or text. As a bonus, `Tools/scripts/pydoc`, which is now automatically installed, uses **`pydoc.py`** to display documentation given a Python module, package, or class name. For example, `pydoc xml.dom` displays the following:

```
Python Library Documentation: package xml.dom in xml

NAME
    xml.dom - W3C Document Object Model implementation for P

FILE
    /usr/local/lib/python2.1/xml/dom/__init__.pyc
```

```
DESCRIPTION
    The Python mapping of the Document Object Model is docur
    Python Library Reference in the section on the xml.dom p

    This package contains the following modules:
      ...
```

`pydoc` also includes a Tk-based interactive help browser. `pydoc` quickly becomes addictive; try it out!

- Two different modules for unit testing were added to the standard library. The **`doctest`** module, contributed by Tim Peters, provides a testing framework based on running embedded examples in docstrings and comparing the results against the expected output. PyUnit, contributed by Steve Purcell, is a unit testing framework inspired by JUnit, which was in turn an adaptation of Kent Beck's Smalltalk testing framework. See https://pyunit.sourceforge.net/ for more information about PyUnit.

- The **`difflib`** module contains a class, **`SequenceMatcher`**, which compares two sequences and computes the changes required to transform one sequence into the other. For example, this module can be used to write a tool similar to the Unix **diff** program, and in fact the sample program `Tools/scripts/ndiff.py` demonstrates how to write such a script.

- **`curses.panel`**, a wrapper for the panel library, part of ncurses and of SYSV curses, was contributed by Thomas Gellekum. The panel library provides windows with the additional feature of depth. Windows can be moved higher or lower in the depth ordering, and the panel library figures out where panels overlap and which sections are visible.

- The PyXML package has gone through a few releases since Python 2.0, and Python 2.1 includes an updated version of the **`xml`** package. Some of the noteworthy changes include support for Expat 1.2 and later versions, the ability for Expat parsers to handle files in any

encoding supported by Python, and various bugfixes for SAX, DOM, and the **minidom** module.

- Ping also contributed another hook for handling uncaught exceptions. **sys.excepthook()** can be set to a callable object. When an exception isn't caught by any **try**…**except** blocks, the exception will be passed to **sys.excepthook()**, which can then do whatever it likes. At the Ninth Python Conference, Ping demonstrated an application for this hook: printing an extended traceback that not only lists the stack frames, but also lists the function arguments and the local variables for each frame.

- Various functions in the **time** module, such as **asctime()** and **localtime()**, require a floating-point argument containing the time in seconds since the epoch. The most common use of these functions is to work with the current time, so the floating-point argument has been made optional; when a value isn't provided, the current time will be used. For example, log file entries usually need a string containing the current time; in Python 2.1, `time.asctime()` can be used, instead of the lengthier `time.asctime(time.localtime(time.time()))` that was previously required.

  This change was proposed and implemented by Thomas Wouters.

- The **ftplib** module now defaults to retrieving files in passive mode, because passive mode is more likely to work from behind a firewall. This request came from the Debian bug tracking system, since other Debian packages use **ftplib** to retrieve files and then don't work from behind a firewall. It's deemed unlikely that this will cause problems for anyone, because Netscape defaults to passive mode and few people complain, but if passive mode is unsuitable for your application or network setup, call `set_pasv(0)` on FTP objects to disable passive mode.

- Support for raw socket access has been added to the **socket** module, contributed by Grant Edwards.

- The **pstats** module now contains a simple interactive statistics browser for displaying timing profiles for Python programs, invoked when the module is run as a script. Contributed by Eric S. Raymond.

- A new implementation-dependent function, `sys._getframe([depth])`, has been added to return a given frame object from the current call stack. **sys._getframe()** returns the frame at the top of the call stack; if the optional integer argument *depth* is supplied, the function returns the frame that is *depth* calls below the top of the stack. For example, `sys._getframe(1)` returns the caller's frame object.

  This function is only present in CPython, not in Jython or the .NET implementation. Use it for debugging, and resist the temptation to put it into production code.

Outras alterações e correções

There were relatively few smaller changes made in Python 2.1 due to the shorter release cycle. A search through the CVS change logs turns up 117 patches applied, and 136 bugs fixed; both figures are likely to be underestimates. Some of the more notable changes are:

- A specialized object allocator is now optionally available, that should be faster than the system **malloc()** and have less memory overhead. The allocator uses C's **malloc()** function to get large pools of memory, and then fulfills smaller memory requests from these pools. It can be enabled by providing the **--with-pymalloc** option to the **configure** script; see `Objects/obmalloc.c` for the implementation details.

  Authors of C extension modules should test their code with the object allocator enabled, because some incorrect code may break, causing core dumps at runtime. There are a bunch of memory allocation functions in Python's C API that have previously been just aliases for the C library's **malloc()** and **free()**, meaning that if you accidentally called mismatched functions, the error wouldn't be noticeable. When the object allocator is enabled, these functions aren't aliases of **malloc()** and **free()** any more, and calling the wrong function to free

memory will get you a core dump. For example, if memory was allocated using **PyMem_New**, it has to be freed using **PyMem_Del()**, not **free()**. A few modules included with Python fell afoul of this and had to be fixed; doubtless there are more third-party modules that will have the same problem.

The object allocator was contributed by Vladimir Marangozov.

- The speed of line-oriented file I/O has been improved because people often complain about its lack of speed, and because it's often been used as a naïve benchmark. The **readline()** method of file objects has therefore been rewritten to be much faster. The exact amount of the speedup will vary from platform to platform depending on how slow the C library's **getc()** was, but is around 66%, and potentially much faster on some particular operating systems. Tim Peters did much of the benchmarking and coding for this change, motivated by a discussion in comp.lang.python.

  A new module and method for file objects was also added, contributed by Jeff Epler. The new method, **xreadlines()**, is similar to the existing **xrange()** built-in. **xreadlines()** returns an opaque sequence object that only supports being iterated over, reading a line on every iteration but not reading the entire file into memory as the existing **readlines()** method does. You'd use it like this:

  ```
  for line in sys.stdin.xreadlines():
      # ... do something for each line ...
      ...
  ```

  For a fuller discussion of the line I/O changes, see the python-dev summary for January 1–15, 2001 at https://mail.python.org/pipermail/python-dev/2001-January/.

- A new method, **popitem()**, was added to dictionaries to enable destructively iterating through the contents of a dictionary; this can be faster for large dictionaries because there's no need to construct a list containing all the keys or values. `D.popitem()` removes a random `(key, value)` pair from the dictionary `D` and returns it as a 2-tuple.

This was implemented mostly by Tim Peters and Guido van Rossum, after a suggestion and preliminary patch by Moshe Zadka.

- Modules can now control which names are imported when `from module import *` is used, by defining an `__all__` attribute containing a list of names that will be imported. One common complaint is that if the module imports other modules such as `sys` or `string`, `from module import *` will add them to the importing module's namespace. To fix this, simply list the public names in `__all__`:

  ```
  # List public names
  __all__ = ['Database', 'open']
  ```

  A stricter version of this patch was first suggested and implemented by Ben Wolfson, but after some python-dev discussion, a weaker final version was checked in.

- Applying `repr()` to strings previously used octal escapes for non-printable characters; for example, a newline was `'\012'`. This was a vestigial trace of Python's C ancestry, but today octal is of very little practical use. Ka-Ping Yee suggested using hex escapes instead of octal ones, and using the `\n`, `\t`, `\r` escapes for the appropriate characters, and implemented this new formatting.

- Syntax errors detected at compile-time can now raise exceptions containing the filename and line number of the error, a pleasant side effect of the compiler reorganization done by Jeremy Hylton.

- C extensions which import other modules have been changed to use `PyImport_ImportModule()`, which means that they will use any import hooks that have been installed. This is also encouraged for third-party extensions that need to import some other module from C code.

- The size of the Unicode character database was shrunk by another 340K thanks to Fredrik Lundh.

- Some new ports were contributed: MacOS X (by Steven Majewski), Cygwin (by Jason Tishler); RISCOS (by Dietmar Schwertberger);

Unixware 7 (by Billy G. Allie).

And there’s the usual list of minor bugfixes, minor memory leaks, docstring edits, and other tweaks, too lengthy to be worth itemizing; see the CVS logs for the full details if you want them.

Reconhecimentos

The author would like to thank the following people for offering suggestions on various drafts of this article: Graeme Cross, David Goodger, Jay Graves, Michael Hudson, Marc-André Lemburg, Fredrik Lundh, Neil Schemenauer, Thomas Wouters.

What’s New in Python 2.0

Autor:: A.M. Kuchling and Moshe Zadka

Introdução

A new release of Python, version 2.0, was released on October 16, 2000. This article covers the exciting new features in 2.0, highlights some other useful changes, and points out a few incompatible changes that may require rewriting code.

Python’s development never completely stops between releases, and a steady flow of bug fixes and improvements are always being submitted. A host of minor fixes, a few optimizations, additional docstrings, and better error messages went into 2.0; to list them all would be impossible, but they’re certainly significant. Consult the publicly available CVS logs if you want to see the full list. This progress is due to the five developers working for PythonLabs are now getting paid to spend their days fixing bugs, and also due to the improved communication resulting from moving to SourceForge.

What About Python 1.6?

Python 1.6 can be thought of as the Contractual Obligations Python release. After the core development team left CNRI in May 2000, CNRI requested that a 1.6 release be created, containing all the work on Python that had been performed at CNRI. Python 1.6 therefore represents the state of the CVS tree as of May 2000, with the most significant new feature being Unicode support. Development continued after May, of course, so the 1.6 tree received a few fixes to ensure that it’s forward-compatible with Python 2.0. 1.6 is therefore part of Python’s evolution, and not a side branch.

So, should you take much interest in Python 1.6? Probably not. The 1.6final and 2.0beta1 releases were made on the same day (September 5, 2000), the

plan being to finalize Python 2.0 within a month or so. If you have applications to maintain, there seems little point in breaking things by moving to 1.6, fixing them, and then having another round of breakage within a month by moving to 2.0; you're better off just going straight to 2.0. Most of the really interesting features described in this document are only in 2.0, because a lot of work was done between May and September.

New Development Process

The most important change in Python 2.0 may not be to the code at all, but to how Python is developed: in May 2000 the Python developers began using the tools made available by SourceForge for storing source code, tracking bug reports, and managing the queue of patch submissions. To report bugs or submit patches for Python 2.0, use the bug tracking and patch manager tools available from Python's project page, located at https://sourceforge.net/projects/python/.

The most important of the services now hosted at SourceForge is the Python CVS tree, the version-controlled repository containing the source code for Python. Previously, there were roughly 7 or so people who had write access to the CVS tree, and all patches had to be inspected and checked in by one of the people on this short list. Obviously, this wasn't very scalable. By moving the CVS tree to SourceForge, it became possible to grant write access to more people; as of September 2000 there were 27 people able to check in changes, a fourfold increase. This makes possible large-scale changes that wouldn't be attempted if they'd have to be filtered through the small group of core developers. For example, one day Peter Schneider-Kamp took it into his head to drop K&R C compatibility and convert the C source for Python to ANSI C. After getting approval on the python-dev mailing list, he launched into a flurry of checkins that lasted about a week, other developers joined in to help, and the job was done. If there were only 5 people with write access, probably that task would have been viewed as "nice, but not worth the time and effort needed" and it would never have gotten done.

The shift to using SourceForge's services has resulted in a remarkable increase in the speed of development. Patches now get submitted,

commented on, revised by people other than the original submitter, and bounced back and forth between people until the patch is deemed worth checking in. Bugs are tracked in one central location and can be assigned to a specific person for fixing, and we can count the number of open bugs to measure progress. This didn't come without a cost: developers now have more e-mail to deal with, more mailing lists to follow, and special tools had to be written for the new environment. For example, SourceForge sends default patch and bug notification e-mail messages that are completely unhelpful, so Ka-Ping Yee wrote an HTML screen-scraper that sends more useful messages.

The ease of adding code caused a few initial growing pains, such as code was checked in before it was ready or without getting clear agreement from the developer group. The approval process that has emerged is somewhat similar to that used by the Apache group. Developers can vote +1, +0, -0, or -1 on a patch; +1 and -1 denote acceptance or rejection, while +0 and -0 mean the developer is mostly indifferent to the change, though with a slight positive or negative slant. The most significant change from the Apache model is that the voting is essentially advisory, letting Guido van Rossum, who has Benevolent Dictator For Life status, know what the general opinion is. He can still ignore the result of a vote, and approve or reject a change even if the community disagrees with him.

Producing an actual patch is the last step in adding a new feature, and is usually easy compared to the earlier task of coming up with a good design. Discussions of new features can often explode into lengthy mailing list threads, making the discussion hard to follow, and no one can read every posting to python-dev. Therefore, a relatively formal process has been set up to write Python Enhancement Proposals (PEPs), modelled on the internet RFC process. PEPs are draft documents that describe a proposed new feature, and are continually revised until the community reaches a consensus, either accepting or rejecting the proposal. Quoting from the introduction to **PEP 1** [https://peps.python.org/pep-0001/], "PEP Purpose and Guidelines":

> PEP stands for Python Enhancement Proposal. A PEP is a design document providing information to the Python community, or

> describing a new feature for Python. The PEP should provide a concise technical specification of the feature and a rationale for the feature.
>
> We intend PEPs to be the primary mechanisms for proposing new features, for collecting community input on an issue, and for documenting the design decisions that have gone into Python. The PEP author is responsible for building consensus within the community and documenting dissenting opinions.

Read the rest of **PEP 1** [https://peps.python.org/pep-0001/] for the details of the PEP editorial process, style, and format. PEPs are kept in the Python CVS tree on SourceForge, though they're not part of the Python 2.0 distribution, and are also available in HTML form from https://peps.python.org/. As of September 2000, there are 25 PEPs, ranging from **PEP 201** [https://peps.python.org/pep-0201/], "Lockstep Iteration", to PEP 225, "Elementwise/Objectwise Operators".

Unicode

The largest new feature in Python 2.0 is a new fundamental data type: Unicode strings. Unicode uses 16-bit numbers to represent characters instead of the 8-bit number used by ASCII, meaning that 65,536 distinct characters can be supported.

The final interface for Unicode support was arrived at through countless often-stormy discussions on the python-dev mailing list, and mostly implemented by Marc-André Lemburg, based on a Unicode string type implementation by Fredrik Lundh. A detailed explanation of the interface was written up as **PEP 100** [https://peps.python.org/pep-0100/], "Python Unicode Integration". This article will simply cover the most significant points about the Unicode interfaces.

In Python source code, Unicode strings are written as `u"string"`. Arbitrary Unicode characters can be written using a new escape sequence, `\u`*`HHHH`*, where *HHHH* is a 4-digit hexadecimal number from 0000 to FFFF. The existing `\x`*`HH`* escape sequence can also be used, and octal escapes can be used for characters up to U+01FF, which is represented by `\777`.

Unicode strings, just like regular strings, are an immutable sequence type. They can be indexed and sliced, but not modified in place. Unicode strings have an `encode( [encoding] )` method that returns an 8-bit string in the desired encoding. Encodings are named by strings, such as `'ascii'`, `'utf-8'`, `'iso-8859-1'`, or whatever. A codec API is defined for implementing and registering new encodings that are then available throughout a Python program. If an encoding isn't specified, the default encoding is usually 7-bit ASCII, though it can be changed for your Python installation by calling the `sys.setdefaultencoding(encoding)` function in a customized version of `site.py`.

Combining 8-bit and Unicode strings always coerces to Unicode, using the default ASCII encoding; the result of `'a' + u'bc'` is `u'abc'`.

New built-in functions have been added, and existing built-ins modified to support Unicode:

- `unichr(ch)` returns a Unicode string 1 character long, containing the character *ch*.
- `ord(u)`, where *u* is a 1-character regular or Unicode string, returns the number of the character as an integer.
- `unicode(string [, encoding]  [, errors] )` creates a Unicode string from an 8-bit string. `encoding` is a string naming the encoding to use. The `errors` parameter specifies the treatment of characters that are invalid for the current encoding; passing `'strict'` as the value causes an exception to be raised on any encoding error, while `'ignore'` causes errors to be silently ignored and `'replace'` uses U+FFFD, the official replacement character, in case of any problems.
- The `exec` statement, and various built-ins such as `eval()`, `getattr()`, and `setattr()` will also accept Unicode strings as well as regular strings. (It's possible that the process of fixing this missed some built-ins; if you find a built-in function that accepts strings but doesn't accept Unicode strings at all, please report it as a bug.)

A new module, **`unicodedata`**, provides an interface to Unicode character properties. For example, `unicodedata.category(u'A')` returns the 2-character string 'Lu', the 'L' denoting it's a letter, and 'u' meaning that it's

uppercase. `unicodedata.bidirectional(u'\u0660')` returns ‘AN’, meaning that U+0660 is an Arabic number.

The **`codecs`** module contains functions to look up existing encodings and register new ones. Unless you want to implement a new encoding, you’ll most often use the `codecs.lookup(encoding)` function, which returns a 4-element tuple: `(encode_func, decode_func, stream_reader, stream_writer)`.

- *encode_func* is a function that takes a Unicode string, and returns a 2-tuple `(string, length)`. *string* is an 8-bit string containing a portion (perhaps all) of the Unicode string converted into the given encoding, and *length* tells you how much of the Unicode string was converted.
- *decode_func* is the opposite of *encode_func*, taking an 8-bit string and returning a 2-tuple `(ustring, length)`, consisting of the resulting Unicode string *ustring* and the integer *length* telling how much of the 8-bit string was consumed.
- *stream_reader* is a class that supports decoding input from a stream. *stream_reader(file_obj)* returns an object that supports the **`read()`**, **`readline()`**, and **`readlines()`** methods. These methods will all translate from the given encoding and return Unicode strings.
- *stream_writer*, similarly, is a class that supports encoding output to a stream. *stream_writer(file_obj)* returns an object that supports the **`write()`** and **`writelines()`** methods. These methods expect Unicode strings, translating them to the given encoding on output.

For example, the following code writes a Unicode string into a file, encoding it as UTF-8:

```
import codecs

unistr = u'\u0660\u2000ab ...'

(UTF8_encode, UTF8_decode,
 UTF8_streamreader, UTF8_streamwriter) = codecs.lookup('UTF-8')

output = UTF8_streamwriter( open( '/tmp/output', 'wb') )
output.write( unistr )
output.close()
```

The following code would then read UTF-8 input from the file:

```
input = UTF8_streamreader( open( '/tmp/output', 'rb') )
print repr(input.read())
input.close()
```

Unicode-aware regular expressions are available through the `re` module, which has a new underlying implementation called SRE written by Fredrik Lundh of Secret Labs AB.

A `-U` command line option was added which causes the Python compiler to interpret all string literals as Unicode string literals. This is intended to be used in testing and future-proofing your Python code, since some future version of Python may drop support for 8-bit strings and provide only Unicode strings.

Compreensões de lista

Lists are a workhorse data type in Python, and many programs manipulate a list at some point. Two common operations on lists are to loop over them, and either pick out the elements that meet a certain criterion, or apply some function to each element. For example, given a list of strings, you might want to pull out all the strings containing a given substring, or strip off trailing whitespace from each line.

The existing `map()` and `filter()` functions can be used for this purpose, but they require a function as one of their arguments. This is fine if there's an existing built-in function that can be passed directly, but if there isn't, you have to create a little function to do the required work, and Python's scoping rules make the result ugly if the little function needs additional information. Take the first example in the previous paragraph, finding all the strings in the list containing a given substring. You could write the following to do it:

```
# Given the list L, make a list of all strings
# containing the substring S.
sublist = filter( lambda s, substring=S:
```

```
                    string.find(s, substring) != -1,
                L)
```

Because of Python's scoping rules, a default argument is used so that the anonymous function created by the **lambda** expression knows what substring is being searched for. List comprehensions make this cleaner:

```
sublist = [ s for s in L if string.find(s, S) != -1 ]
```

List comprehensions have the form:

```
[ expression for expr in sequence1
             for expr2 in sequence2 ...
             for exprN in sequenceN
             if condition ]
```

The **for**...**in** clauses contain the sequences to be iterated over. The sequences do not have to be the same length, because they are *not* iterated over in parallel, but from left to right; this is explained more clearly in the following paragraphs. The elements of the generated list will be the successive values of *expression*. The final **if** clause is optional; if present, *expression* is only evaluated and added to the result if *condition* is true.

To make the semantics very clear, a list comprehension is equivalent to the following Python code:

```
for expr1 in sequence1:
    for expr2 in sequence2:
    ...
        for exprN in sequenceN:
             if (condition):
                  # Append the value of
                  # the expression to the
                  # resulting list.
```

This means that when there are multiple **for**...**in** clauses, the resulting list will be equal to the product of the lengths of all the sequences. If you have two lists of length 3, the output list is 9 elements long:

```
seq1 = 'abc'
seq2 = (1,2,3)
>>> [ (x,y) for x in seq1 for y in seq2]
[('a', 1), ('a', 2), ('a', 3), ('b', 1), ('b', 2), ('b', 3), ('c
('c', 2), ('c', 3)]
```

To avoid introducing an ambiguity into Python's grammar, if *expression* is creating a tuple, it must be surrounded with parentheses. The first list comprehension below is a syntax error, while the second one is correct:

```
# Syntax error
[ x,y for x in seq1 for y in seq2]
# Correct
[ (x,y) for x in seq1 for y in seq2]
```

The idea of list comprehensions originally comes from the functional programming language Haskell (https://www.haskell.org). Greg Ewing argued most effectively for adding them to Python and wrote the initial list comprehension patch, which was then discussed for a seemingly endless time on the python-dev mailing list and kept up-to-date by Skip Montanaro.

Augmented Assignment

Augmented assignment operators, another long-requested feature, have been added to Python 2.0. Augmented assignment operators include `+=`, `-=`, `*=`, and so forth. For example, the statement `a += 2` increments the value of the variable `a` by 2, equivalent to the slightly lengthier `a = a + 2`.

The full list of supported assignment operators is `+=`, `-=`, `*=`, `/=`, `%=`, `**=`, `&=`, `|=`, `^=`, `>>=`, and `<<=`. Python classes can override the augmented assignment operators by defining methods named **`__iadd__()`**, **`__isub__()`**, etc. For example, the following **`Number`** class stores a number and supports using `+=` to create a new instance with an incremented value.

```
class Number:
    def __init__(self, value):
        self.value = value
    def __iadd__(self, increment):
```

```
        return Number( self.value + increment)

n = Number(5)
n += 3
print n.value
```

The **__iadd__()** special method is called with the value of the increment, and should return a new instance with an appropriately modified value; this return value is bound as the new value of the variable on the left-hand side.

Augmented assignment operators were first introduced in the C programming language, and most C-derived languages, such as **awk**, C++, Java, Perl, and PHP also support them. The augmented assignment patch was implemented by Thomas Wouters.

String Methods

Until now string-manipulation functionality was in the **string** module, which was usually a front-end for the **strop** module written in C. The addition of Unicode posed a difficulty for the **strop** module, because the functions would all need to be rewritten in order to accept either 8-bit or Unicode strings. For functions such as **string.replace()**, which takes 3 string arguments, that means eight possible permutations, and correspondingly complicated code.

Instead, Python 2.0 pushes the problem onto the string type, making string manipulation functionality available through methods on both 8-bit strings and Unicode strings.

```
>>> 'andrew'.capitalize()
'Andrew'
>>> 'hostname'.replace('os', 'linux')
'hlinuxtname'
>>> 'moshe'.find('sh')
2
```

One thing that hasn’t changed, a noteworthy April Fools’ joke notwithstanding, is that Python strings are immutable. Thus, the string

methods return new strings, and do not modify the string on which they operate.

The old **`string`** module is still around for backwards compatibility, but it mostly acts as a front-end to the new string methods.

Two methods which have no parallel in pre-2.0 versions, although they did exist in JPython for quite some time, are **`startswith()`** and **`endswith()`**. `s.startswith(t)` is equivalent to `s[:len(t)] == t`, while `s.endswith(t)` is equivalent to `s[-len(t):] == t`.

One other method which deserves special mention is **`join()`**. The **`join()`** method of a string receives one parameter, a sequence of strings, and is equivalent to the **`string.join()`** function from the old **`string`** module, with the arguments reversed. In other words, `s.join(seq)` is equivalent to the old `string.join(seq, s)`.

Garbage Collection of Cycles

The C implementation of Python uses reference counting to implement garbage collection. Every Python object maintains a count of the number of references pointing to itself, and adjusts the count as references are created or destroyed. Once the reference count reaches zero, the object is no longer accessible, since you need to have a reference to an object to access it, and if the count is zero, no references exist any longer.

Reference counting has some pleasant properties: it’s easy to understand and implement, and the resulting implementation is portable, fairly fast, and reacts well with other libraries that implement their own memory handling schemes. The major problem with reference counting is that it sometimes doesn’t realise that objects are no longer accessible, resulting in a memory leak. This happens when there are cycles of references.

Consider the simplest possible cycle, a class instance which has a reference to itself:

```
instance = SomeClass()
instance.myself = instance
```

After the above two lines of code have been executed, the reference count of `instance` is 2; one reference is from the variable named `'instance'`, and the other is from the `myself` attribute of the instance.

If the next line of code is `del instance`, what happens? The reference count of `instance` is decreased by 1, so it has a reference count of 1; the reference in the `myself` attribute still exists. Yet the instance is no longer accessible through Python code, and it could be deleted. Several objects can participate in a cycle if they have references to each other, causing all of the objects to be leaked.

Python 2.0 fixes this problem by periodically executing a cycle detection algorithm which looks for inaccessible cycles and deletes the objects involved. A new **`gc`** module provides functions to perform a garbage collection, obtain debugging statistics, and tuning the collector's parameters.

Running the cycle detection algorithm takes some time, and therefore will result in some additional overhead. It is hoped that after we've gotten experience with the cycle collection from using 2.0, Python 2.1 will be able to minimize the overhead with careful tuning. It's not yet obvious how much performance is lost, because benchmarking this is tricky and depends crucially on how often the program creates and destroys objects. The detection of cycles can be disabled when Python is compiled, if you can't afford even a tiny speed penalty or suspect that the cycle collection is buggy, by specifying the **`--without-cycle-gc`** switch when running the **configure** script.

Several people tackled this problem and contributed to a solution. An early implementation of the cycle detection approach was written by Toby Kelsey. The current algorithm was suggested by Eric Tiedemann during a visit to CNRI, and Guido van Rossum and Neil Schemenauer wrote two different implementations, which were later integrated by Neil. Lots of other people offered suggestions along the way; the March 2000 archives of the python-dev mailing list contain most of the relevant discussion, especially in the threads titled "Reference cycle collection for Python" and "Finalization again".

Other Core Changes

Various minor changes have been made to Python's syntax and built-in functions. None of the changes are very far-reaching, but they're handy conveniences.

Minor Language Changes

A new syntax makes it more convenient to call a given function with a tuple of arguments and/or a dictionary of keyword arguments. In Python 1.5 and earlier, you'd use the **`apply()`** built-in function: `apply(f, args, kw)` calls the function **`f()`** with the argument tuple *args* and the keyword arguments in the dictionary *kw*. **`apply()`** is the same in 2.0, but thanks to a patch from Greg Ewing, `f(*args, **kw)` is a shorter and clearer way to achieve the same effect. This syntax is symmetrical with the syntax for defining functions:

```
def f(*args, **kw):
    # args is a tuple of positional args,
    # kw is a dictionary of keyword args
    ...
```

The `print` statement can now have its output directed to a file-like object by following the `print` with `>> file`, similar to the redirection operator in Unix shells. Previously you'd either have to use the **`write()`** method of the file-like object, which lacks the convenience and simplicity of `print`, or you could assign a new value to `sys.stdout` and then restore the old value. For sending output to standard error, it's much easier to write this:

```
print >> sys.stderr, "Warning: action field not supplied"
```

Modules can now be renamed on importing them, using the syntax `import module as name` or `from module import name as othername`. The patch was submitted by Thomas Wouters.

A new format style is available when using the `%` operator; '%r' will insert the **`repr()`** of its argument. This was also added from symmetry considerations, this time for symmetry with the existing '%s' format style,

which inserts the **str()** of its argument. For example, `'%r %s' % ('abc', 'abc')` returns a string containing `'abc' abc`.

Previously there was no way to implement a class that overrode Python's built-in **in** operator and implemented a custom version. `obj in seq` returns true if *obj* is present in the sequence *seq*; Python computes this by simply trying every index of the sequence until either *obj* is found or an **IndexError** is encountered. Moshe Zadka contributed a patch which adds a **__contains__()** magic method for providing a custom implementation for **in**. Additionally, new built-in objects written in C can define what **in** means for them via a new slot in the sequence protocol.

Earlier versions of Python used a recursive algorithm for deleting objects. Deeply nested data structures could cause the interpreter to fill up the C stack and crash; Christian Tismer rewrote the deletion logic to fix this problem. On a related note, comparing recursive objects recursed infinitely and crashed; Jeremy Hylton rewrote the code to no longer crash, producing a useful result instead. For example, after this code:

```
a = []
b = []
a.append(a)
b.append(b)
```

The comparison `a==b` returns true, because the two recursive data structures are isomorphic. See the thread "trashcan and PR#7" in the April 2000 archives of the python-dev mailing list for the discussion leading up to this implementation, and some useful relevant links. Note that comparisons can now also raise exceptions. In earlier versions of Python, a comparison operation such as `cmp(a,b)` would always produce an answer, even if a user-defined **__cmp__()** method encountered an error, since the resulting exception would simply be silently swallowed.

Work has been done on porting Python to 64-bit Windows on the Itanium processor, mostly by Trent Mick of ActiveState. (Confusingly, `sys.platform` is still `'win32'` on Win64 because it seems that for ease of porting, MS Visual C++ treats code as 32 bit on Itanium.) PythonWin also

supports Windows CE; see the Python CE page at https://pythonce.sourceforge.net/ for more information.

Another new platform is Darwin/MacOS X; initial support for it is in Python 2.0. Dynamic loading works, if you specify "configure –with-dyld –with-suffix=.x". Consult the README in the Python source distribution for more instructions.

An attempt has been made to alleviate one of Python's warts, the often-confusing **`NameError`** exception when code refers to a local variable before the variable has been assigned a value. For example, the following code raises an exception on the `print` statement in both 1.5.2 and 2.0; in 1.5.2 a **`NameError`** exception is raised, while 2.0 raises a new **`UnboundLocalError`** exception. **`UnboundLocalError`** is a subclass of **`NameError`**, so any existing code that expects **`NameError`** to be raised should still work.

```
def f():
    print "i=",i
    i = i + 1
f()
```

Two new exceptions, **`TabError`** and **`IndentationError`**, have been introduced. They're both subclasses of **`SyntaxError`**, and are raised when Python code is found to be improperly indented.

Changes to Built-in Functions

A new built-in, `zip(seq1, seq2, ...)`, has been added. **`zip()`** returns a list of tuples where each tuple contains the i-th element from each of the argument sequences. The difference between **`zip()`** and `map(None, seq1, seq2)` is that **`map()`** pads the sequences with `None` if the sequences aren't all of the same length, while **`zip()`** truncates the returned list to the length of the shortest argument sequence.

The **`int()`** and **`long()`** functions now accept an optional "base" parameter when the first argument is a string. `int('123', 10)` returns 123, while `int('123', 16)` returns 291. `int(123, 16)` raises a **`TypeError`** exception with the message "can't convert non-string with explicit base".

A new variable holding more detailed version information has been added to the **sys** module. `sys.version_info` is a tuple `(major, minor, micro, level, serial)` For example, in a hypothetical 2.0.1beta1, `sys.version_info` would be `(2, 0, 1, 'beta', 1)`. *level* is a string such as `"alpha"`, `"beta"`, or `"final"` for a final release.

Dictionaries have an odd new method, `setdefault(key, default)`, which behaves similarly to the existing **`get()`** method. However, if the key is missing, **`setdefault()`** both returns the value of *default* as **`get()`** would do, and also inserts it into the dictionary as the value for *key*. Thus, the following lines of code:

```
if dict.has_key( key ): return dict[key]
else:
    dict[key] = []
    return dict[key]
```

can be reduced to a single `return dict.setdefault(key, [])` statement.

The interpreter sets a maximum recursion depth in order to catch runaway recursion before filling the C stack and causing a core dump or GPF.. Previously this limit was fixed when you compiled Python, but in 2.0 the maximum recursion depth can be read and modified using **`sys.getrecursionlimit()`** and **`sys.setrecursionlimit()`**. The default value is 1000, and a rough maximum value for a given platform can be found by running a new script, `Misc/find_recursionlimit.py`.

Porting to 2.0

New Python releases try hard to be compatible with previous releases, and the record has been pretty good. However, some changes are considered useful enough, usually because they fix initial design decisions that turned out to be actively mistaken, that breaking backward compatibility can’t always be avoided. This section lists the changes in Python 2.0 that may cause old Python code to break.

The change which will probably break the most code is tightening up the arguments accepted by some methods. Some methods would take multiple

arguments and treat them as a tuple, particularly various list methods such as **append()** and **insert()**. In earlier versions of Python, if `L` is a list, `L.append( 1,2 )` appends the tuple `(1,2)` to the list. In Python 2.0 this causes a **TypeError** exception to be raised, with the message: 'append requires exactly 1 argument; 2 given'. The fix is to simply add an extra set of parentheses to pass both values as a tuple: `L.append( (1,2) )`.

The earlier versions of these methods were more forgiving because they used an old function in Python's C interface to parse their arguments; 2.0 modernizes them to use **PyArg_ParseTuple()**, the current argument parsing function, which provides more helpful error messages and treats multi-argument calls as errors. If you absolutely must use 2.0 but can't fix your code, you can edit `Objects/listobject.c` and define the preprocessor symbol `NO_STRICT_LIST_APPEND` to preserve the old behaviour; this isn't recommended.

Some of the functions in the **socket** module are still forgiving in this way. For example, `socket.connect( ('hostname', 25) )` is the correct form, passing a tuple representing an IP address, but `socket.connect('hostname', 25)` also works. **socket.connect_ex** and **socket.bind** are similarly easy-going. 2.0alpha1 tightened these functions up, but because the documentation actually used the erroneous multiple argument form, many people wrote code which would break with the stricter checking. GvR backed out the changes in the face of public reaction, so for the **socket** module, the documentation was fixed and the multiple argument form is simply marked as deprecated; it *will* be tightened up again in a future Python version.

The `\x` escape in string literals now takes exactly 2 hex digits. Previously it would consume all the hex digits following the 'x' and take the lowest 8 bits of the result, so `\x123456` was equivalent to `\x56`.

The **AttributeError** and **NameError** exceptions have a more friendly error message, whose text will be something like `'Spam' instance has no attribute 'eggs'` or `name 'eggs' is not defined`. Previously the error message was just the missing attribute name `eggs`, and code written to take advantage of this fact will break in 2.0.

Some work has been done to make integers and long integers a bit more interchangeable. In 1.5.2, large-file support was added for Solaris, to allow reading files larger than 2 GiB; this made the **`tell()`** method of file objects return a long integer instead of a regular integer. Some code would subtract two file offsets and attempt to use the result to multiply a sequence or slice a string, but this raised a **`TypeError`**. In 2.0, long integers can be used to multiply or slice a sequence, and it'll behave as you'd intuitively expect it to; `3L * 'abc'` produces 'abcabcabc', and `(0,1,2,3)[2L:4L]` produces (2,3). Long integers can also be used in various contexts where previously only integers were accepted, such as in the **`seek()`** method of file objects, and in the formats supported by the `%` operator (`%d`, `%i`, `%x`, etc.). For example, `"%d" % 2L**64` will produce the string `18446744073709551616`.

The subtlest long integer change of all is that the **`str()`** of a long integer no longer has a trailing 'L' character, though **`repr()`** still includes it. The 'L' annoyed many people who wanted to print long integers that looked just like regular integers, since they had to go out of their way to chop off the character. This is no longer a problem in 2.0, but code which does `str(longval)[:-1]` and assumes the 'L' is there, will now lose the final digit.

Taking the **`repr()`** of a float now uses a different formatting precision than **`str()`**. **`repr()`** uses `%.17g` format string for C's **`sprintf()`**, while **`str()`** uses `%.12g` as before. The effect is that **`repr()`** may occasionally show more decimal places than **`str()`**, for certain numbers. For example, the number 8.1 can't be represented exactly in binary, so `repr(8.1)` is `'8.0999999999999996'`, while str(8.1) is `'8.1'`.

The `-X` command-line option, which turned all standard exceptions into strings instead of classes, has been removed; the standard exceptions will now always be classes. The **`exceptions`** module containing the standard exceptions was translated from Python to a built-in C module, written by Barry Warsaw and Fredrik Lundh.

Extending/Embedding Changes

Some of the changes are under the covers, and will only be apparent to people writing C extension modules or embedding a Python interpreter in a larger application. If you aren't dealing with Python's C API, you can safely skip this section.

The version number of the Python C API was incremented, so C extensions compiled for 1.5.2 must be recompiled in order to work with 2.0. On Windows, it's not possible for Python 2.0 to import a third party extension built for Python 1.5.x due to how Windows DLLs work, so Python will raise an exception and the import will fail.

Users of Jim Fulton's ExtensionClass module will be pleased to find out that hooks have been added so that ExtensionClasses are now supported by **`isinstance()`** and **`issubclass()`**. This means you no longer have to remember to write code such as `if type(obj) == myExtensionClass`, but can use the more natural `if isinstance(obj, myExtensionClass)`.

The `Python/importdl.c` file, which was a mass of #ifdefs to support dynamic loading on many different platforms, was cleaned up and reorganised by Greg Stein. `importdl.c` is now quite small, and platform-specific code has been moved into a bunch of `Python/dynload_*.c` files. Another cleanup: there were also a number of `my*.h` files in the Include/ directory that held various portability hacks; they've been merged into a single file, `Include/pyport.h`.

Vladimir Marangozov's long-awaited malloc restructuring was completed, to make it easy to have the Python interpreter use a custom allocator instead of C's standard **`malloc()`**. For documentation, read the comments in `Include/pymem.h` and `Include/objimpl.h`. For the lengthy discussions during which the interface was hammered out, see the web archives of the 'patches' and 'python-dev' lists at python.org.

Recent versions of the GUSI development environment for MacOS support POSIX threads. Therefore, Python's POSIX threading support now works on the Macintosh. Threading support using the user-space GNU `pth` library was also contributed.

Threading support on Windows was enhanced, too. Windows supports thread locks that use kernel objects only in case of contention; in the common case when there's no contention, they use simpler functions which are an order of magnitude faster. A threaded version of Python 1.5.2 on NT is twice as slow as an unthreaded version; with the 2.0 changes, the difference is only 10%. These improvements were contributed by Yakov Markovitch.

Python 2.0's source now uses only ANSI C prototypes, so compiling Python now requires an ANSI C compiler, and can no longer be done using a compiler that only supports K&R C.

Previously the Python virtual machine used 16-bit numbers in its bytecode, limiting the size of source files. In particular, this affected the maximum size of literal lists and dictionaries in Python source; occasionally people who are generating Python code would run into this limit. A patch by Charles G. Waldman raises the limit from `2**16` to `2**32`.

Three new convenience functions intended for adding constants to a module's dictionary at module initialization time were added: `PyModule_AddObject()`, `PyModule_AddIntConstant()`, and `PyModule_AddStringConstant()`. Each of these functions takes a module object, a null-terminated C string containing the name to be added, and a third argument for the value to be assigned to the name. This third argument is, respectively, a Python object, a C long, or a C string.

A wrapper API was added for Unix-style signal handlers. `PyOS_getsig()` gets a signal handler and `PyOS_setsig()` will set a new handler.

Distutils: Making Modules Easy to Install

Before Python 2.0, installing modules was a tedious affair – there was no way to figure out automatically where Python is installed, or what compiler options to use for extension modules. Software authors had to go through an arduous ritual of editing Makefiles and configuration files, which only really work on Unix and leave Windows and MacOS unsupported. Python users faced wildly differing installation instructions which varied between

different extension packages, which made administering a Python installation something of a chore.

The SIG for distribution utilities, shepherded by Greg Ward, has created the Distutils, a system to make package installation much easier. They form the `distutils` package, a new part of Python's standard library. In the best case, installing a Python module from source will require the same steps: first you simply mean unpack the tarball or zip archive, and the run "`python setup.py install`". The platform will be automatically detected, the compiler will be recognized, C extension modules will be compiled, and the distribution installed into the proper directory. Optional command-line arguments provide more control over the installation process, the distutils package offers many places to override defaults – separating the build from the install, building or installing in non-default directories, and more.

In order to use the Distutils, you need to write a `setup.py` script. For the simple case, when the software contains only .py files, a minimal `setup.py` can be just a few lines long:

```
from distutils.core import setup
setup (name = "foo", version = "1.0",
       py_modules = ["module1", "module2"])
```

The `setup.py` file isn't much more complicated if the software consists of a few packages:

```
from distutils.core import setup
setup (name = "foo", version = "1.0",
       packages = ["package", "package.subpackage"])
```

A C extension can be the most complicated case; here's an example taken from the PyXML package:

```
from distutils.core import setup, Extension

expat_extension = Extension('xml.parsers.pyexpat',
     define_macros = [('XML_NS', None)],
     include_dirs = [ 'extensions/expat/xmltok',
                      'extensions/expat/xmlparse' ],
     sources = [ 'extensions/pyexpat.c',
```

```
                  'extensions/expat/xmltok/xmltok.c',
                  'extensions/expat/xmltok/xmlrole.c', ]
       )
setup (name = "PyXML", version = "0.5.4",
       ext_modules =[ expat_extension ] )
```

The Distutils can also take care of creating source and binary distributions. The “sdist” command, run by “`python setup.py sdist`’, builds a source distribution such as `foo-1.0.tar.gz`. Adding new commands isn’t difficult, “bdist_rpm” and “bdist_wininst” commands have already been contributed to create an RPM distribution and a Windows installer for the software, respectively. Commands to create other distribution formats such as Debian packages and Solaris `.pkg` files are in various stages of development.

All this is documented in a new manual, *Distributing Python Modules*, that joins the basic set of Python documentation.

XML Modules

Python 1.5.2 included a simple XML parser in the form of the **`xmllib`** module, contributed by Sjoerd Mullender. Since 1.5.2’s release, two different interfaces for processing XML have become common: SAX2 (version 2 of the Simple API for XML) provides an event-driven interface with some similarities to **`xmllib`**, and the DOM (Document Object Model) provides a tree-based interface, transforming an XML document into a tree of nodes that can be traversed and modified. Python 2.0 includes a SAX2 interface and a stripped-down DOM interface as part of the **`xml`** package. Here we will give a brief overview of these new interfaces; consult the Python documentation or the source code for complete details. The Python XML SIG is also working on improved documentation.

SAX2 Support

SAX defines an event-driven interface for parsing XML. To use SAX, you must write a SAX handler class. Handler classes inherit from various classes provided by SAX, and override various methods that will then be called by the XML parser. For example, the **`startElement()`** and

endElement() methods are called for every starting and end tag encountered by the parser, the **characters()** method is called for every chunk of character data, and so forth.

The advantage of the event-driven approach is that the whole document doesn't have to be resident in memory at any one time, which matters if you are processing really huge documents. However, writing the SAX handler class can get very complicated if you're trying to modify the document structure in some elaborate way.

For example, this little example program defines a handler that prints a message for every starting and ending tag, and then parses the file `hamlet.xml` using it:

```
from xml import sax

class SimpleHandler(sax.ContentHandler):
    def startElement(self, name, attrs):
        print 'Start of element:', name, attrs.keys()

    def endElement(self, name):
        print 'End of element:', name

# Create a parser object
parser = sax.make_parser()

# Tell it what handler to use
handler = SimpleHandler()
parser.setContentHandler( handler )

# Parse a file!
parser.parse( 'hamlet.xml' )
```

For more information, consult the Python documentation, or the XML HOWTO at https://pyxml.sourceforge.net/topics/howto/xml-howto.html.

DOM Support

The Document Object Model is a tree-based representation for an XML document. A top-level **Document** instance is the root of the tree, and has a

single child which is the top-level **`Element`** instance. This **`Element`** has children nodes representing character data and any sub-elements, which may have further children of their own, and so forth. Using the DOM you can traverse the resulting tree any way you like, access element and attribute values, insert and delete nodes, and convert the tree back into XML.

The DOM is useful for modifying XML documents, because you can create a DOM tree, modify it by adding new nodes or rearranging subtrees, and then produce a new XML document as output. You can also construct a DOM tree manually and convert it to XML, which can be a more flexible way of producing XML output than simply writing `<tag1>...</tag1>` to a file.

The DOM implementation included with Python lives in the **`xml.dom.minidom`** module. It's a lightweight implementation of the Level 1 DOM with support for XML namespaces. The **`parse()`** and **`parseString()`** convenience functions are provided for generating a DOM tree:

```
from xml.dom import minidom
doc = minidom.parse('hamlet.xml')
```

`doc` is a **`Document`** instance. **`Document`**, like all the other DOM classes such as **`Element`** and **`Text`**, is a subclass of the **`Node`** base class. All the nodes in a DOM tree therefore support certain common methods, such as **`toxml()`** which returns a string containing the XML representation of the node and its children. Each class also has special methods of its own; for example, **`Element`** and **`Document`** instances have a method to find all child elements with a given tag name. Continuing from the previous 2-line example:

```
perslist = doc.getElementsByTagName( 'PERSONA' )
print perslist[0].toxml()
print perslist[1].toxml()
```

For the *Hamlet* XML file, the above few lines output:

```
<PERSONA>CLAUDIUS, king of Denmark. </PERSONA>
<PERSONA>HAMLET, son to the late, and nephew to the present king
```

The root element of the document is available as `doc.documentElement`, and its children can be easily modified by deleting, adding, or removing nodes:

```
root = doc.documentElement

# Remove the first child
root.removeChild( root.childNodes[0] )

# Move the new first child to the end
root.appendChild( root.childNodes[0] )

# Insert the new first child (originally,
# the third child) before the 20th child.
root.insertBefore( root.childNodes[0], root.childNodes[20] )
```

Again, I will refer you to the Python documentation for a complete listing of the different **`Node`** classes and their various methods.

Relationship to PyXML

The XML Special Interest Group has been working on XML-related Python code for a while. Its code distribution, called PyXML, is available from the SIG's web pages at https://www.python.org/community/sigs/current/xml-sig. The PyXML distribution also used the package name `xml`. If you've written programs that used PyXML, you're probably wondering about its compatibility with the 2.0 **`xml`** package.

The answer is that Python 2.0's **`xml`** package isn't compatible with PyXML, but can be made compatible by installing a recent version PyXML. Many applications can get by with the XML support that is included with Python 2.0, but more complicated applications will require that the full PyXML package will be installed. When installed, PyXML versions 0.6.0 or greater will replace the **`xml`** package shipped with Python, and will be a strict superset of the standard package, adding a bunch of additional features. Some of the additional features in PyXML include:

- 4DOM, a full DOM implementation from FourThought, Inc.
- The xmlproc validating parser, written by Lars Marius Garshol.

- The **sgmlop** parser accelerator module, written by Fredrik Lundh.

Module changes

Lots of improvements and bugfixes were made to Python's extensive standard library; some of the affected modules include **readline**, **ConfigParser**, **cgi**, **calendar**, **posix**, **readline**, **xmllib**, **aifc**, **chunk**, **wave**, **random**, **shelve**, and **nntplib**. Consult the CVS logs for the exact patch-by-patch details.

Brian Gallew contributed OpenSSL support for the **socket** module. OpenSSL is an implementation of the Secure Socket Layer, which encrypts the data being sent over a socket. When compiling Python, you can edit `Modules/Setup` to include SSL support, which adds an additional function to the **socket** module: `socket.ssl(socket, keyfile, certfile)`, which takes a socket object and returns an SSL socket. The **httplib** and **urllib** modules were also changed to support `https://` URLs, though no one has implemented FTP or SMTP over SSL.

The **httplib** module has been rewritten by Greg Stein to support HTTP/1.1.

Backward compatibility with the 1.5 version of **httplib** is provided, though using HTTP/1.1 features such as pipelining will require rewriting code to use a different set of interfaces.

The **Tkinter** module now supports Tcl/Tk version 8.1, 8.2, or 8.3, and support for the older 7.x versions has been dropped. The Tkinter module now supports displaying Unicode strings in Tk widgets. Also, Fredrik Lundh contributed an optimization which makes operations like `create_line` and `create_polygon` much faster, especially when using lots of coordinates.

The **curses** module has been greatly extended, starting from Oliver Andrich's enhanced version, to provide many additional functions from ncurses and SYSV curses, such as colour, alternative character set support, pads, and mouse support. This means the module is no longer compatible

with operating systems that only have BSD curses, but there don't seem to be any currently maintained OSes that fall into this category.

As mentioned in the earlier discussion of 2.0's Unicode support, the underlying implementation of the regular expressions provided by the `re` module has been changed. SRE, a new regular expression engine written by Fredrik Lundh and partially funded by Hewlett Packard, supports matching against both 8-bit strings and Unicode strings.

Novos módulos

A number of new modules were added. We'll simply list them with brief descriptions; consult the 2.0 documentation for the details of a particular module.

- `atexit`: For registering functions to be called before the Python interpreter exits. Code that currently sets `sys.exitfunc` directly should be changed to use the **`atexit`** module instead, importing **`atexit`** and calling `atexit.register()` with the function to be called on exit. (Contributed by Skip Montanaro.)
- `codecs`, **`encodings`**, `unicodedata`: Added as part of the new Unicode support.
- `filecmp`: Supersedes the old **`cmp`**, **`cmpcache`** and **`dircmp`** modules, which have now become deprecated. (Contributed by Gordon MacMillan and Moshe Zadka.)
- `gettext`: This module provides internationalization (I18N) and localization (L10N) support for Python programs by providing an interface to the GNU gettext message catalog library. (Integrated by Barry Warsaw, from separate contributions by Martin von Löwis, Peter Funk, and James Henstridge.)
- **`linuxaudiodev`**: Support for the `/dev/audio` device on Linux, a twin to the existing **`sunaudiodev`** module. (Contributed by Peter Bosch, with fixes by Jeremy Hylton.)
- `mmap`: An interface to memory-mapped files on both Windows and Unix. A file's contents can be mapped directly into memory, at which point it behaves like a mutable string, so its contents can be read and modified. They can even be passed to functions that expect ordinary

strings, such as the `re` module. (Contributed by Sam Rushing, with some extensions by A.M. Kuchling.)

- **`pyexpat`**: An interface to the Expat XML parser. (Contributed by Paul Prescod.)
- `robotparser`: Parse a `robots.txt` file, which is used for writing web spiders that politely avoid certain areas of a web site. The parser accepts the contents of a `robots.txt` file, builds a set of rules from it, and can then answer questions about the fetchability of a given URL. (Contributed by Skip Montanaro.)
- `tabnanny`: A module/script to check Python source code for ambiguous indentation. (Contributed by Tim Peters.)
- **`UserString`**: A base class useful for deriving objects that behave like strings.
- `webbrowser`: A module that provides a platform independent way to launch a web browser on a specific URL. For each platform, various browsers are tried in a specific order. The user can alter which browser is launched by setting the *BROWSER* environment variable. (Originally inspired by Eric S. Raymond's patch to `urllib` which added similar functionality, but the final module comes from code originally implemented by Fred Drake as `Tools/idle/BrowserControl.py`, and adapted for the standard library by Fred.)
- `_winreg`: An interface to the Windows registry. **`_winreg`** is an adaptation of functions that have been part of PythonWin since 1995, but has now been added to the core distribution, and enhanced to support Unicode. **`_winreg`** was written by Bill Tutt and Mark Hammond.
- `zipfile`: A module for reading and writing ZIP-format archives. These are archives produced by **PKZIP** on DOS/Windows or **zip** on Unix, not to be confused with **gzip**-format files (which are supported by the `gzip` module) (Contributed by James C. Ahlstrom.)
- **`imputil`**: A module that provides a simpler way for writing customized import hooks, in comparison to the existing **`ihooks`** module. (Implemented by Greg Stein, with much discussion on python-dev along the way.)

IDLE Improvements

IDLE is the official Python cross-platform IDE, written using Tkinter. Python 2.0 includes IDLE 0.6, which adds a number of new features and improvements. A partial list:

- UI improvements and optimizations, especially in the area of syntax highlighting and auto-indentation.
- The class browser now shows more information, such as the top level functions in a module.
- Tab width is now a user settable option. When opening an existing Python file, IDLE automatically detects the indentation conventions, and adapts.
- There is now support for calling browsers on various platforms, used to open the Python documentation in a browser.
- IDLE now has a command line, which is largely similar to the vanilla Python interpreter.
- Call tips were added in many places.
- IDLE can now be installed as a package.
- In the editor window, there is now a line/column bar at the bottom.
- Three new keystroke commands: Check module (`Alt-F5`), Import module (`F5`) and Run script (`Ctrl-F5`).

Deleted and Deprecated Modules

A few modules have been dropped because they're obsolete, or because there are now better ways to do the same thing. The **`stdwin`** module is gone; it was for a platform-independent windowing toolkit that's no longer developed.

A number of modules have been moved to the `lib-old` subdirectory: **`cmp`**, **`cmpcache`**, **`dircmp`**, **`dump`**, **`find`**, **`grep`**, **`packmail`**, **`poly`**, **`util`**, **`whatsound`**, **`zmod`**. If you have code which relies on a module that's been moved to `lib-old`, you can simply add that directory to `sys.path` to get them back, but you're encouraged to update any code that uses these modules.

Reconhecimentos

The authors would like to thank the following people for offering suggestions on various drafts of this article: David Bolen, Mark Hammond, Gregg Hauser, Jeremy Hylton, Fredrik Lundh, Detlef Lannert, Aahz Maruch, Skip Montanaro, Vladimir Marangozov, Tobias Polzin, Guido van Rossum, Neil Schemenauer, and Russ Schmidt.

Changelog

Python seguinte

Data de lançamento: XXXX-XX-XX

Núcleo e embutidos

- gh-158803 [https://github.com/python/cpython/issues/158803]: Fix a crash in **`bytes.join()`** and **`bytearray.join()`** in the free-threaded build when another thread concurrently mutates the list being joined. Patch by Christian Aurich Zanettini Martins.
- gh-158820 [https://github.com/python/cpython/issues/158820]: Fix a crash when a lazy import is declared or resolved by a finalizer running during interpreter shutdown.
- gh-158592 [https://github.com/python/cpython/issues/158592]: Skip the array reallocation when shrinking a small **`list`**. Patch by Donghee Na.
- gh-158592 [https://github.com/python/cpython/issues/158592]: `del list[index]` now triggers shrinking the list's internal array.
- gh-158486 [https://github.com/python/cpython/issues/158486]: Fix the PowerSort mask calculation in **`list.sort()`** for very large lists.
- gh-156091 [https://github.com/python/cpython/issues/156091]: Fix a crash when compiling deeply nested inlined list, set, or dict comprehensions. A **`SyntaxError`** is now raised when the nesting exceeds the compiler's static block limit.
- gh-156459 [https://github.com/python/cpython/issues/156459]: Fix cleanup on error in `compiler_set_qualname`. Previously it was called in `_PyCompile_EnterScope`, after the scope had been entered, and this was not reversed in case of an error.
- gh-156371 [https://github.com/python/cpython/issues/156371]: Fix missing `PyRefTracer_DESTROY` events for trashcan deferred objects. Patch by Donghee Na.
- gh-155504 [https://github.com/python/cpython/issues/155504]: Fix a crash when thread state creation fails before the thread state is fully initialized.

- gh-155315 [https://github.com/python/cpython/issues/155315]: Fix `marshal` so that a `frozendict` referenced more than once in the serialized data round-trips correctly, instead of failing to load with `ValueError`. Patch by tonghuaroot.
- gh-154937 [https://github.com/python/cpython/issues/154937]: Fix a data race on thread handle identifiers when `_thread._shutdown()` runs concurrently with the startup of non-daemon threads in the free-threaded build.
- gh-152075 [https://github.com/python/cpython/issues/152075]: Reduced lock contention during LOAD_GLOBAL bytecode specialization under free threading.
- gh-153578 [https://github.com/python/cpython/issues/153578]: Fix an out-of-bounds write in `bytearray.extend()` when the bytearray is resized while the argument's `__buffer__()` is being acquired, for example by another thread. Patch by tonghuaroot.
- gh-151895 [https://github.com/python/cpython/issues/151895]: Fixed a crash in `marshal.loads()` when an allocation failed while loading a reference-tracked dictionary; it now raises `MemoryError`.

Biblioteca

- gh-87720 [https://github.com/python/cpython/issues/87720]: Fixed error in `email` new policies where header folding failed to detect the need to re-quote text obtained from encoded words in certain cases.
- gh-158936 [https://github.com/python/cpython/issues/158936]: `dataclass()` no longer fails when a string annotation names a lazy import that cannot be resolved, matching the behavior for non-string annotations.
- gh-156321 [https://github.com/python/cpython/issues/156321]: Fix `asyncio.shield()` reporting an exception from the inner future through the loop exception handler as soon as it completes after the shield was cancelled, even when the exception is retrieved afterwards. The exception is now reported only if it is never retrieved, as with any other future.
- gh-158563 [https://github.com/python/cpython/issues/158563]: Fix `turtle.undo()` after `turtle.teleport()` and after an exception in `turtle.circle()`, `turtle.dot()` or `turtle.write()`. Fix `turtle.stamp()`,

`turtle.clearstamp()`, `turtle.clear()` and `turtle.reset()` when the undo buffer is disabled.

- gh-158169 [https://github.com/python/cpython/issues/158169]: Fix `zlib.Decompress.unconsumed_tail` duplicating `unused_data` after end of stream when *max_length* was used.
- gh-153888 [https://github.com/python/cpython/issues/153888]: Fix `unittest.mock.patch()` with `autospec`, `spec`, or `spec_set` when used with lazy-imported objects.
- gh-157672 [https://github.com/python/cpython/issues/157672]: Corrige o `tkinter` no macOS: a criação de uma instância de `Tk` não substitui mais os manipuladores Python para SIGINT, SIGHUP e SIGTERM pelo manipulador do Tk que encerra o processo; assim, Ctrl-C volta a levantar `KeyboardInterrupt`.
- gh-157621 [https://github.com/python/cpython/issues/157621]: Fix a crash when a weak-reference callback is attached to a `typing.ParamSpecArgs` or `typing.ParamSpecKwargs` instance.
- gh-69919 [https://github.com/python/cpython/issues/69919]: `code.InteractiveInterpreter.runsource()` and the REPL now report any exception raised by compiling the source, such as `MemoryError` or `RecursionError` for too deeply nested source, instead of propagating it or printing a traceback of the REPL internals.
- gh-157451 [https://github.com/python/cpython/issues/157451]: Fix `gettext` plural form selection when the `Plural-Forms` expression combines the unary `!` operator with a binary operator, for example `!n + 1` or `2 * !n`. The latter raised `SyntaxError`.
- gh-157406 [https://github.com/python/cpython/issues/157406]: Corrige a implementação em Python de `xml.etree.ElementTree.XMLParser`: o método `doctype()` do destino agora é chamado para uma declaração de tipo de documento sem identificador externo, como `<!DOCTYPE html>`, assim como na implementação em C.
- gh-157325 [https://github.com/python/cpython/issues/157325]: Fix an out-of-bounds read in the `hz` incremental decoder when the input ends with `~`.
- gh-157048 [https://github.com/python/cpython/issues/157048]: Preserve the position and contents of `io.BytesIO` when **`BytesIO.__init__()`** fails because a buffer is exported.
- gh-156961 [https://github.com/python/cpython/issues/156961]: Corrige `tkinter.font.nametofont()` e o construtor `tkinter.font.Font` para

um nome de fonte retornado pelo Tk como um objeto Tcl — por exemplo, por `ttk.Style.lookup()`.

- gh-156946 [https://github.com/python/cpython/issues/156946]: Fix a crash in `curses.panel` when the finalizer of a panel's user pointer runs while the panel is being deallocated. The panel is now taken off the panel stack before its user pointer is dropped.
- gh-156797 [https://github.com/python/cpython/issues/156797]: Fix `xml.sax.saxutils.XMLGenerator`: the namespace URI is now escaped in the namespace declaration. Previously a URI containing `&`, `<` or a quote character produced a document which cannot be parsed.
- gh-83895 [https://github.com/python/cpython/issues/83895]: o `xml.etree.ElementTree` agora aceita entradas maiores que 2 GiB na implementação em C. Os dados são fornecidos ao Expat em blocos — como o `xml.parsers.expat` já fazia — em vez de levantar uma `OverflowError`.
- gh-99064 [https://github.com/python/cpython/issues/99064]: Corrige `xml.etree.ElementTree.parse()` ao lidar com um arquivo de texto ou outra fonte de dados do tipo `str` na implementação em C. A codificação declarada no documento era aplicada ao texto já decodificado, resultando em *mojibake*; agora, ela é ignorada, assim como ocorre ao analisar com `XMLParser.feed()` ou `fromstring()`.
- gh-156713 [https://github.com/python/cpython/issues/156713]: Fix `nturl2path.pathname2url()` and `nturl2path.url2pathname()`: the filesystem encoding and error handler are now used for percent-encoding and decoding, as in `urllib.request`. Previously paths containing surrogate characters raised `UnicodeEncodeError`.
- gh-156658 [https://github.com/python/cpython/issues/156658]: `xml.dom` and `xml.etree.ElementTree` no longer treat characters which are not white space in XML (such as U+00A0) as white space. Previously they could be lost in `indent()`, `canonicalize()` with `strip_text=True`, and when parsing with the `whitespace-in-element-content` feature turned off.
- gh-156476 [https://github.com/python/cpython/issues/156476]: The pure Python implementation of `queue.SimpleQueue.get()` now ignores *timeout* when *block* is false, matching the documented behavior and the C implementation. It previously raised `ValueError`.

- gh-156444 [https://github.com/python/cpython/issues/156444]: Fix matching of a negated character set in a bytes pattern compiled with both **re.IGNORECASE** and **re.LOCALE**. The case closure is now applied to the members of the set, so that `[^bc]` no longer matches `b'B'`.
- gh-156180 [https://github.com/python/cpython/issues/156180]: **zlib.adler32_combine()** e **zlib.crc32_combine()** agora levantam **ValueError** se o argumento *len2* for negativo, em vez de retornarem valores inválidos ou travarem indefinidamente, respectivamente.
- gh-156166 [https://github.com/python/cpython/issues/156166]: **ssl**: A failed assignment or deletion of the `_msg_callback` attribute of **ssl.SSLContext** no longer removes the current callback. Deleting it now raises **AttributeError** instead of **TypeError**.
- gh-156173 [https://github.com/python/cpython/issues/156173]: Calling **zlib.Decompress.flush()** on invalid compressed data now raises **zlib.error** instead of being silently ignored.
- gh-152817 [https://github.com/python/cpython/issues/152817]: **sqlite3**: Impede a remoção do atributo `row_factory` de um cursor para evitar uma falha durante uma consulta.
- gh-156124 [https://github.com/python/cpython/issues/156124]: Fix a crash in the free-threaded build when deleting the **contents** attribute of a **ctypes** pointer.
- gh-156101 [https://github.com/python/cpython/issues/156101]: Fix **sqlite3.Cursor.arraysize** being set to 0 if the assigned value is too large. The attribute is now left unchanged if the assignment fails.
- gh-156100 [https://github.com/python/cpython/issues/156100]: Fix crashes in **sqlite3.Connection** when deleting the **autocommit** attribute or setting it to an integer which does not fit in C long. Both now raise an exception.
- gh-156099 [https://github.com/python/cpython/issues/156099]: Corrige uma falha ao excluir o atributo `keylog_filename` de **ssl.SSLContext**. Agora, isso levanta uma exceção **AttributeError**.
- gh-156067 [https://github.com/python/cpython/issues/156067]: Fix error handling in the **zoneinfo** accelerator module when a transition index is `-1` or a TZ string's `__bool__` raises.
- gh-155997 [https://github.com/python/cpython/issues/155997]: Fix **concurrent.interpreters.list_all()**. It failed if an interpreter was

destroyed during the call, in particular by a garbage collection which finalized the object owning the last reference to it.

- gh-155941 [https://github.com/python/cpython/issues/155941]: Fix **asyncio.start_server()** when a plain-function *client_connected_cb* raises: the error is now reported like the coroutine case and the transport is closed, instead of leaving the connection open forever (which also made **asyncio.Server.wait_closed()** hang).
- gh-155717 [https://github.com/python/cpython/issues/155717]: **multiprocessing**'s default start method on systems with non-writeable tempfile filesystem is now "spawn" instead of `"forkserver"`. Patch by Bénédikt Tran.
- gh-155775 [https://github.com/python/cpython/issues/155775]: Fix a regression in **imaplib** introduced in the fix for gh-63121 [https://github.com/python/cpython/issues/63121]: refreshing the capabilities after a successful **login()** or **authenticate()** consumed the `CAPABILITY` response, so it could no longer be read with **response()**.
- gh-113318 [https://github.com/python/cpython/issues/113318]: Corrige falhas ao excluir um atributo cujo método *setter* é gerado pela Clínica de Argumentos e não está preparado para a exclusão, incluindo **frame.f_trace_opcodes** e os atributos `context`, `owner` e `session` de `_ssl._SSLSocket`. A exclusão de tal atributo agora levanta uma **AttributeError**.
- gh-150449 [https://github.com/python/cpython/issues/150449]: Corrige fatias de **sqlite3.Blob** com passo negativo. A leitura ou escrita de tal fatia levantava **SystemError**. Agora, levanta **ValueError**.
- gh-155336 [https://github.com/python/cpython/issues/155336]: Fix error handling in **socket.gethostbyaddr()** and **socket.gethostbyname_ex()** when hostname resolution fails.
- gh-102475 [https://github.com/python/cpython/issues/102475]: Fix **os.path.realpath()** on Windows: the unresolved part of the path is now appended, not joined, so a file name which looks like a drive (e.g. `spam:eggs`) no longer discards the resolved part. A path relative to another drive is now resolved against the root directory of that drive.
- gh-89760 [https://github.com/python/cpython/issues/89760]: Corrige **os.path.realpath()** no Windows: o prefixo `\\?\` não é mais removido de caminhos com GUID de volume, o que tornava o resultado inválido.

- gh-84419 [https://github.com/python/cpython/issues/84419]: Corrige `os.stat()` no Windows: pontos e espaços ao final do nome, que são ignorados pelo sistema operacional, não são mais levados em consideração ao inferir as permissões de execução a partir da extensão do arquivo.
- gh-155319 [https://github.com/python/cpython/issues/155319]: `warnings.warn_explicit()` now displays the source line taken from the loader of the module whose globals are passed as *module_globals*. It also no longer raises `IndexError` if *lineno* is out of the range of the module source.
- gh-123011 [https://github.com/python/cpython/issues/123011]: `warnings.warn_explicit()` no longer emits a spurious `DeprecationWarning` or raises `ImportError` when it is called with the globals of the `__main__` module.
- gh-69370 [https://github.com/python/cpython/issues/69370]: **python -m inspect --details** no longer fails with `UnicodeEncodeError` if the path of the module contains characters unencodable in the encoding of `sys.stdout`. Such characters are now escaped with backslashes.
- gh-69371 [https://github.com/python/cpython/issues/69371]: Fix `pydoc` for modules whose path contains undecodable bytes. `pydoc.writedoc()` and the pydoc HTTP server no longer fail with `UnicodeEncodeError`: the file URL is now percent-encoded using the filesystem encoding, and characters unencodable in the generated HTML page are escaped with backslashes.
- gh-154855 [https://github.com/python/cpython/issues/154855]: Corrige `curses.window.instr()` retornava um caractere a menos quando o módulo `curses` era compilado com uma biblioteca curses que conta o caractere terminador no comprimento solicitado, como a do NetBSD.
- gh-135683 [https://github.com/python/cpython/issues/135683]: `logging.FileHandler` and `logging.handlers.WatchedFileHandler` now honor `logging.raiseExceptions` for errors that occur while opening the file, like for other errors during logging.
- gh-82535 [https://github.com/python/cpython/issues/82535]: `logging.handlers.SysLogHandler` no longer fails if the address cannot be resolved when the handler is created. The address is resolved again when a record is emitted.
- gh-153970 [https://github.com/python/cpython/issues/153970]: Calling `str()` on a `subprocess.CalledProcessError` no longer raises `TypeError` when

its **returncode** is not an integer, such as None.

- gh-153005 [https://github.com/python/cpython/issues/153005]: **concurrent.interpreters.Queue.get()** and **concurrent.interpreters.Queue.put()** now compute their timeout deadline from **time.monotonic()** instead of the wall clock, so adjusting the system clock during the call no longer makes them over- or under-wait.
- gh-81241 [https://github.com/python/cpython/issues/81241]: Fix **OSError** when importing **ctypes** from a statically linked interpreter.
- gh-153603 [https://github.com/python/cpython/issues/153603]: Fix a crash in the ISO-2022 decoders when decoding a byte after an unknown charset designation is set via the decoder's setstate method. Patch by tonghuaroot.
- gh-153539 [https://github.com/python/cpython/issues/153539]: Corrige uma falha na implementação do C de **io.TextIOWrapper.tell()** quando o método getstate do decodificador aciona uma busca reentrante, ou quando outro thread busca o mesmo fluxo simultaneamente. Patch por tonghuaroot.
- gh-151126 [https://github.com/python/cpython/issues/151126]: Corrige uma falha causada pela não definição de **MemoryError** em caso de falha de alocação ao passar instâncias de **ctypes.Structure** ou **ctypes.Union** por valor para funções externas do ctypes.
- gh-152204 [https://github.com/python/cpython/issues/152204]: Fix the pure-Python implementations of **datetime.date.fromisoformat()**, **datetime.time.fromisoformat()** and **datetime.datetime.fromisoformat()** silently accepting some malformed ISO 8601 strings, such as non-ASCII digits or a sign in a fixed-width field (for example '2020+12' or '20201212T0102٣٤'). Each field is now required to be exactly *N* ASCII digits, matching the C implementation.
- gh-117807 [https://github.com/python/cpython/issues/117807]: Fix **mimetypes** initialization from MIME map files containing invalid UTF-8 bytes.
- gh-145856 [https://github.com/python/cpython/issues/145856]: Fix **plistlib.dumps()** and **plistlib.dump()** so that skipkeys=True together with sort_keys=True correctly drops non-string keys when the dictionary contains a mix of string and non-string keys. Previously the sort ran before the filter and raised **TypeError**.

- gh-102967 [https://github.com/python/cpython/issues/102967]: A bug in **`doctest._SpoofOut.truncate()`** was causing None to be passed to **`StringIO.seek()`** when no size was given. A simple fix skips the seek call when no size is given so the buffer can be truncated from the current position.
- gh-82039 [https://github.com/python/cpython/issues/82039]: **`http.cookiejar.FileCookieJar.load()`** now checks the first, format signature line in a case-insensitive manner. Patch by Ashley Harvey.
- gh-70758 [https://github.com/python/cpython/issues/70758]: Creating a new **`turtle`** screen after the previous one was closed no longer raises `turtle.Terminator` on the first turtle command.
- bpo-44012 [https://bugs.python.org/issue?@action=redirect&bpo=44012]: The `exploded` attribute of **`ipaddress.IPv6Address`** and **`ipaddress.IPv6Interface`** now supports addresses with a scope ID (link-local addresses). Previously the former raised **`AddressValueError`** and the latter omitted the scope ID.
- gh-83869 [https://github.com/python/cpython/issues/83869]: Fix **`tarfile`** reading an archive with a GNU sparse 1.0 member whose size is set in the pax extended header. The offset of the next header was computed from the offset of the data, which is already past the sparse map, and from the size of the member, which can be the apparent size of the sparse file. All following members were unreachable.
- gh-61366 [https://github.com/python/cpython/issues/61366]: **`http.cookiejar.MozillaCookieJar`** now reads session cookies written by curl and Wget, which use `0` in the expiration time field. Contributed by Jérémie Detrey.

Testes

- gh-157628 [https://github.com/python/cpython/issues/157628]: In regrtest, only consider that a test leaks if all test runs leak at least one file descriptor. For example, ignore “[0, 1, -1] file descriptors, sum=0” deltas, instead of reporting a leak. Patch by Victor Stinner.
- gh-140550 [https://github.com/python/cpython/issues/140550]: Enable limited C API tests on Free Threading in test_cext and test_cppext: test the **`Py_TARGET_ABI3T`** macro. Patch by Victor Stinner.

- gh-75876 [https://github.com/python/cpython/issues/75876]: A test decorated with **`bigmemtest()`** now runs in a subprocess if it really allocates the memory it asks for (that is, if the `-M` option is used), so that the memory it uses and the address space it fragments are released when it ends. A dummy run stays in the process. The separate memory watchdog process is no longer needed: the parent process watches the memory usage of the subprocess.
- gh-155411 [https://github.com/python/cpython/issues/155411]: Fix **`test.support.subTests()`** for asynchronous test methods. They were wrapped in a synchronous function, which discarded the coroutine without awaiting it, so the test silently did not run at all.
- gh-132581 [https://github.com/python/cpython/issues/132581]: The list of tests which altered the execution environment now includes the reasons why the environment was considered altered, for example an unraisable exception or a modified **`sys.path`**. The final result no longer repeats the same state twice (like `ENV CHANGED then ENV CHANGED`).
- gh-155109 [https://github.com/python/cpython/issues/155109]: Add `test.support.run_with_limited_c_stack()` and use it in tests that exhaust the C stack with a fixed number of recursive calls, so that their outcome no longer depends on the C stack size.

Construção

- gh-157142 [https://github.com/python/cpython/issues/157142]: Fix the link of `Programs/_freeze_module` when extension modules are built in (`MODULE_BUILDTYPE=static`) and the test modules are enabled: `Modules/getpath_noop.c` now defines `_Py_Get_Getpath_CodeObject()`, which `_testinternalcapi` uses.
- gh-155911 [https://github.com/python/cpython/issues/155911]: Corrige a detecção do curses quando uma biblioteca curses já está em `LIBS`. Patch por Shamil Abdulaev.

Windows

- gh-157789 [https://github.com/python/cpython/issues/157789]: Use compiler option `-Og` in case of clang-cl debug builds on Windows to reduce

stack usage. Patch by Chris Eibl.

- gh-72353 [https://github.com/python/cpython/issues/72353]: Reading from the console on Windows now continues if the read was cancelled by Ctrl+C, but the **SIGINT** handler did not raise an exception. Previously the read ended as if at end of file.
- gh-86427 [https://github.com/python/cpython/issues/86427]: Fix the encoding of the standard streams in the legacy Windows stdio mode (**PYTHONLEGACYWINDOWSSTDIO**). It is now the code page of the console, as in Python 3.7, not the ANSI code page.
- gh-87587 [https://github.com/python/cpython/issues/87587]: **os.device_encoding()** on Windows now returns the code page of any console file descriptor, not only 0, 1 and 2, and returns `None` for other character devices like `NUL`. The UTF-8 code page is now reported as `"utf-8"` instead of `"cp65001"`.

IDLE

- gh-56596 [https://github.com/python/cpython/issues/56596]: Faz com que os atalhos de teclado do IDLE que utilizam letras ASCII funcionem com o Caps Lock ativado, mesmo quando apenas uma das versões (maiúscula ou minúscula) estiver definida.
- gh-64007 [https://github.com/python/cpython/issues/64007]: O IDLE não aplica mais coloração de sintaxe ao texto inserido para **input()** no Console.
- gh-85560 [https://github.com/python/cpython/issues/85560]: Corrige a falha do IDLE em encontrar o parêntese de abertura quando uma linha de continuação dentro dos parênteses começa com `else` de uma expressão condicional ou com `yield`.
- gh-75487 [https://github.com/python/cpython/issues/75487]: Corrige o salvamento de um número truncado quando um campo de número inteiro na caixa de diálogo de configurações do IDLE é esvaziado usando a tecla Backspace.
- gh-74112 [https://github.com/python/cpython/issues/74112]: Ctrl-C in the IDLE Shell now interrupts blocking calls such as **time.sleep()** and **socket.recv**.
- gh-151471 [https://github.com/python/cpython/issues/151471]: Colorize the soft keyword `type` in IDLE.

- gh-103089 [https://github.com/python/cpython/issues/103089]: IDLE no longer hangs when opening a file with very long lines. Only the first 2,000 characters of a line are colorized.
- gh-70331 [https://github.com/python/cpython/issues/70331]: IDLE no longer fails to start with `python -m idlelib`, and its user process no longer fails to start, when the current directory contains user files with the same names as standard library modules that IDLE imports, such as `random.py` or `tkinter.py`.
- gh-93016 [https://github.com/python/cpython/issues/93016]: Fix parsing of command line arguments in the IDLE "Run… Customized" dialog on Windows: backslashes are no longer treated as escape characters. Fix also display of the previous arguments in the dialog.
- gh-69365 [https://github.com/python/cpython/issues/69365]: As caixas de diálogo de pesquisa do IDLE agora exibem um erro de expressão regular abaixo do campo de entrada, em vez de em uma caixa de mensagem.
- gh-105689 [https://github.com/python/cpython/issues/105689]: Corrige as dicas de chamada (calltips), a correspondência de parênteses e a indentação automática no Console IDLE após a exibição de uma saída contendo aspas ou parênteses desbalanceados.
- gh-60402 [https://github.com/python/cpython/issues/60402]: No IDLE, pressionar Tab em uma string que não esteja no início de um nome de arquivo insere uma tabulação em vez de abrir a lista de preenchimento automático.
- gh-68453 [https://github.com/python/cpython/issues/68453]: Corrigida a execução do IDLE com a opção -s em conjunto com -c, -r ou -: o comando ou script agora é executado após o arquivo de inicialização, sem exibir uma caixa de diálogo de erro e sem reiniciar o console.
- gh-69919 [https://github.com/python/cpython/issues/69919]: IDLE now reports any exception raised by compiling the source, such as **`MemoryError`** for too deeply nested source, instead of hanging the Shell.
- gh-91398 [https://github.com/python/cpython/issues/91398]: Remove the border around the IDLE Shell sidebar, which was drawn in the window background color and stood out with dark themes.
- gh-71136 [https://github.com/python/cpython/issues/71136]: Fix a hang of the IDLE Shell when code run under the debugger raises **`KeyboardInterrupt`**.

- gh-154511 [https://github.com/python/cpython/issues/154511]: Consolida o tratamento da roda do mouse do IDLE em `idlelib.util`. A função `wheel_event` é transferida de `idlelib.tree` para lá e junta-se a `x11_buttons` — que indica se o Tk reporta rotações da roda a um widget como eventos `<Button-4>`/`<Button-5>` — e a `bind_wheel`, que associa os eventos enviados pelo Tk.
- gh-156896 [https://github.com/python/cpython/issues/156896]: Para de excluir submódulos do tkinter quando idlelib.run for importado.
- gh-153480 [https://github.com/python/cpython/issues/153480]: Fix IDLE failing to start when opening a file which does not exist yet. Fix a traceback when a file open in the IDLE editor is deleted by another program; IDLE now asks whether to close the window, save the file elsewhere, or ignore it.
- gh-66172 [https://github.com/python/cpython/issues/66172]: O IDLE não falha mais ao iniciar quando um arquivo de configuração do usuário está corrompido. O arquivo ilegível é renomeado com o sufixo ".bad", as configurações padrão são utilizadas em seu lugar e um aviso lista os arquivos afetados.
- gh-90304 [https://github.com/python/cpython/issues/90304]: Impede que o IDLE falhe ao iniciar quando uma fonte relata largura zero para o caractere `'0'`. Esse tipo de fonte defeituosa causava um **`ZeroDivisionError`**; agora, o editor utiliza a largura configurada como alternativa.
- gh-135511 [https://github.com/python/cpython/issues/135511]: Fix display of **`NameError`** and **`AttributeError`** with multi-line message.

Ferramentas/Demos

- gh-158451 [https://github.com/python/cpython/issues/158451]: `python-gdb.py` is now able to format an invalid Unicode string: render invalid characters as `\Uhhhhhhhh` instead of raising an exception. Patch by Victor Stinner.
- gh-156219 [https://github.com/python/cpython/issues/156219]: Fix Argument Clinic generating code which reads an optional positional argument which was not passed, if the function has a `**kwds` parameter.
- gh-113318 [https://github.com/python/cpython/issues/113318]: Correção na Clínica de Argumentos para `@getter` e `@setter` dentro de um bloco condicional de pré-processador; anteriormente, ocorria um erro interno. A Clínica de Argumentos agora também rejeita acessores de

um mesmo atributo que possuam nomes base em C diferentes, bem como o mesmo acessor definido duas vezes — situações que, silenciosamente, geravam entradas inválidas ou duplicadas de `PyGetSetDef`.

- gh-155218 [https://github.com/python/cpython/issues/155218]: Fix Argument Clinic generating the flags of the optional groups in different order on 32-bit and 64-bit platforms.
- gh-155207 [https://github.com/python/cpython/issues/155207]: Argument Clinic now supports the `--dry-run` and `--diff` options. They list the files which would be changed, or write a unified diff of the changes to the standard output, without modifying any file.
- gh-64502 [https://github.com/python/cpython/issues/64502]: Fix Argument Clinic support of parameters with a default value used together with optional groups. Such parameters were always required in the generated parsing code.

API C

- gh-149044 [https://github.com/python/cpython/issues/149044]: Fix the `PySlot_END` macro for C++: initialize all members to avoid compiler warnings. Patch by Victor Stinner.
- gh-156101 [https://github.com/python/cpython/issues/156101]: Don't set the result in error in `PyLong_AsInt32()`, `PyLong_AsUInt32()`, `PyLong_AsInt64()` and `PyLong_AsUInt64()`. Leave the result unchanged in this case. Patch by Victor Stinner.

Python 3.15.0 final

Data de lançamento: 2026-10-09

Não houve novas alterações na versão 3.15.0.

Python 3.15.0 candidato a lançamento 3

Data de lançamento: 2026-10-02

Segurança

- gh-158446 [https://github.com/python/cpython/issues/158446]: Corrige uma falha ou saída incorreta que poderia ocorrer ao formatar um `float` ou `complex` com uma precisão próxima ao `INT_MAX` da plataforma. A função `PyOS_double_to_string()` agora levanta uma `ValueError` para qualquer precisão dessa magnitude, independentemente do tipo de apresentação ou do valor, assim como os analisadores de string de formato já faziam para precisões acima de `INT_MAX`.

- gh-156793 [https://github.com/python/cpython/issues/156793]: `asyncio`: `loop.start_tls()` e `loop.create_connection()` agora valida o argumento *server_hostname* se um `ssl.SSLContext` for passado com *check_hostname* definido com `True`.

- gh-156793 [https://github.com/python/cpython/issues/156793]: `ssl.SSLContext.wrap_bio()` agora valida seus argumentos *server_side*, *server_hostname* e *session* similares a `ssl.SSLContext.wrap_socket()`.

  Em particular, um contexto com `check_hostname` habilitado e sem que um *server_hostname* seja passado para `wrap_bio()` agora levanta uma `ValueError` em vez de concluir um handshake que verificava a cadeia de certificados sem verificar a identidade da outra parte, sem qualquer indicação de que a verificação havia sido ignorada.

- gh-157265 [https://github.com/python/cpython/issues/157265]: Em `tarfile`, quando a extração de um link recorre à extração de um membro do arquivo, o membro deve ser ignorado se a função de filtro retornar None ao ser chamada com o nome do membro extraído substituído pelo nome do link.

- gh-157953 [https://github.com/python/cpython/issues/157953]: Atualiza o libexpat [https://libexpat.github.io/] empacotado para a versão 2.8.5.

- gh-157579 [https://github.com/python/cpython/issues/157579]: Corrige uma condição de corrida na limpeza de `tempfile.TemporaryDirectory`. Ao contornar erros de permissão do sistema de arquivos, os arquivos

agora são removidos com base em descritores de arquivo de diretórios abertos, em vez de resolver novamente seus caminhos; assim, a substituição de um diretório da árvore por um link simbólico não pode mais fazer com que a limpeza exclua arquivos fora do diretório temporário. Isso soluciona a **CVE 2026-12345** [https://www.cve.org/CVERecord?id=CVE-2026-12345].

- gh-156293 [https://github.com/python/cpython/issues/156293]: Corrige uma falha no `ssl` que ocorria quando um `sni_callback` alternava uma conexão para outro `SSLContext` e o contexto que continha a função de retorno deixava de ser referenciado pela aplicação. Servidores que mantêm ativo o contexto do seu `sni_callback` (o caso habitual, quando ele encapsula o *socket* de escuta ou está armazenado no objeto do servidor) não foram afetados. Isso soluciona a **CVE 2026-19445** [https://www.cve.org/CVERecord?id=CVE-2026-19445].

Núcleo e embutidos

- gh-158140 [https://github.com/python/cpython/issues/158140]: Resolve submódulos irmãos importados de forma preguiçosa (lazy) e de maneira independente, de modo que acessar um não importe outro apenas porque sua declaração apareceu por último.
- gh-158364 [https://github.com/python/cpython/issues/158364]: Corrige uma falha que ocorria quando `sys.current_frames()` ou `sys.current_exceptions()` eram chamados enquanto outro interpretador estava em execução.
- gh-158254 [https://github.com/python/cpython/issues/158254]: Iteradores reversos sobre um `dict` e suas visualizações agora levantam `RuntimeError` se as chaves do dicionário forem alteradas durante a iteração — assim como os iteradores diretos —, em vez de fornecerem entradas para as novas chaves.
- gh-157914 [https://github.com/python/cpython/issues/157914]: Reverte gh-146064 [https://github.com/python/cpython/issues/146064]. A alteração relacionada ao PyMutex estava causando uma regressão significativa de desempenho em alguns casos.
- gh-157838 [https://github.com/python/cpython/issues/157838]: Mescle contagens de referências enviesadas em nome de threads desvinculadas

(detached) em vez de aguardar que elas se vinculem novamente, na construção com threads livres.

- gh-157778 [https://github.com/python/cpython/issues/157778]: Inclui o número da linha da chave de abertura na `SyntaxError` para um campo de substituição de f-string ou t-string não fechado, quando uma aspa correspondente em uma linha posterior for interpretada como o início de uma string não terminada.
- gh-157757 [https://github.com/python/cpython/issues/157757]: Corrige um `import a.b as c` preguiçoso que lia `b` a partir de `a` em vez de importar o módulo `a.b`. Agora, ele associa o submódulo e levanta uma exceção `ModuleNotFoundError` quando não há um módulo correspondente ao nome.
- gh-155695 [https://github.com/python/cpython/issues/155695]: Nomes resolvidos agora são removidos de `sys.lazy_modules` de forma mais consistente: uma importação preguiçosa (*lazy import*) de um módulo já carregado não é mais registrada, e a materialização de `lazy from pkg import attr` agora descarta a entrada `pkg.attr`.
- gh-157378 [https://github.com/python/cpython/issues/157378]: Corrige `SyntaxError.offset` e `SyntaxError.end_offset` para o erro “Non-UTF-8 code starting with …” quando um caractere não ASCII precede o byte inválido na mesma linha. Patch por Shamil Abdulaev.
- gh-156762 [https://github.com/python/cpython/issues/156762]: Correção de comportamento indefinido em `operator.methodcaller`: a função do slot `tp_clear` retornava `void` em vez de `int`, fazendo com que o coletor de lixo a chamasse por meio de um tipo de função incompatível. Patch por Shamil Abdulaev.
- gh-157377 [https://github.com/python/cpython/issues/157377]: Corrige `gc.get_count()` na construção com threads livres: a função estava reiniciando o contador de alocação local da thread que agenda a coleta de lixo automática. Uma thread que a chamasse durante uma alocação poderia impedir que lixo cíclico fosse coletado.
- gh-157247 [https://github.com/python/cpython/issues/157247]: Corrige `sys.activate_stack_trampoline()` permitindo a ativação a partir de um subinterpretador enquanto o JIT está habilitado no interpretador principal.
- gh-156995 [https://github.com/python/cpython/issues/156995]: Corrigido o problema em que um `bytearray` compartilhava seu buffer com um

objeto **bytes** de um único byte e de mesmo valor, fazendo com que a escrita no bytearray modificasse aquele objeto **bytes**. Isso ocorria com **bytearray.take_bytes()** quando restava exatamente um byte e, na construção com threads livres, quando um bytearray era reduzido para um único byte a partir de outra thread.

- gh-156910 [https://github.com/python/cpython/issues/156910]: Corrige um impasse na construção com threads livres ao definir `__abstractmethods__` em um tipo enquanto outra thread detém a trava do tipo.
- gh-156894 [https://github.com/python/cpython/issues/156894]: Corrige a posição de erros de sintaxe que abrangem um intervalo, caso a linha contenha caracteres não ASCII antes do erro.
- gh-155525 [https://github.com/python/cpython/issues/155525]: Corrige a tokenização de tempo quadrático de módulos que contêm muitas f-strings ou t-strings.
- gh-156689 [https://github.com/python/cpython/issues/156689]: Corrige uma leitura fora dos limites em **compile()** e **ast.parse()** quando um objeto AST é passado com `mode='func_type'`.
- gh-156114 [https://github.com/python/cpython/issues/156114]: Corrige uma falha no *trampoline* do perf quando um objeto código possui um nome ou nome de arquivo que não pode ser codificado em UTF-8. Correção feita por Shamil Abdulaev.
- gh-156126 [https://github.com/python/cpython/issues/156126]: Corrige uma falha ao importar um módulo cujo nome contém caracteres que não podem ser codificados em UTF-8 (como *surrogates* isolados) enquanto **-X importtime** está habilitado.
- gh-155526 [https://github.com/python/cpython/issues/155526]: Corrige um **OverflowError** espúrio para `abs(nanj)` caso **errno** tenha sido definido anteriormente como **ERANGE** por alguma chamada de biblioteca. Patch por Sergey B Kirpichev.
- gh-151464 [https://github.com/python/cpython/issues/151464]: Exclui o token inválido `<>` da saída de **tokenize**. **SyntaxError** para `<>` agora sugere `!=`.
- gh-154719 [https://github.com/python/cpython/issues/154719]: Espaços em branco ao final de uma expressão de interpolação em *t-string* agora são preservados em **string.templatelib.Interpolation.expression**, até o `}` de fechamento ou até o delimitador de conversão (`!`), formatação (`:`) ou depuração (`=`). Continuações de linha explícitas após

um = de depuração permanecem como parte do texto de depuração e são excluídas de **`expression`**.

- gh-152708 [https://github.com/python/cpython/issues/152708]: Corrige a assimetria na geração da AST em que o operador unário positivo `+` era implicitamente descartado em expressões de correspondência de padrões, restaurando a paridade de análise sintática com o operador unário negativo `-`.

Biblioteca

- gh-157788 [https://github.com/python/cpython/issues/157788]: Corrige a falha do comando `python -m asyncio ps` ao lidar com um processo que possui uma tarefa com nome superior a 255 caracteres.
- gh-157660 [https://github.com/python/cpython/issues/157660]: Corrige **`profiling.sampling`** relatando erros ou números de linha incorretos em construções com threads livres quando um array de bytecode local à thread aumenta de tamanho ou recebe novas entradas após ter sido armazenado em cache.
- gh-157639 [https://github.com/python/cpython/issues/157639]: Corrige uma falha no **`cProfile`** quando um temporizador externo reentra no perfilador. Eventos de perfilamento levantados enquanto o temporizador externo está em execução agora são ignorados.
- gh-157335 [https://github.com/python/cpython/issues/157335]: Corrige uma escrita fora dos limites em `mmap.mmap.__setitem__` que poderia ocorrer quando a conversão do índice ou do valor atribuído (via **`__index__()`** para um único item, ou via protocolo de buffer para uma fatia) redimensionasse ou fechasse o objeto mmap durante a atribuição.
- gh-128509 [https://github.com/python/cpython/issues/128509]: **`marshal.load()`** e **`marshal.loads()`** agora podem obter singletons de strings de 1 byte. Patch por Victor Stinner.
- gh-157301 [https://github.com/python/cpython/issues/157301]: Corrige **`asyncio.Task`** travando o laço de eventos quando uma inicialização imediata (*eager start*) falha ao entrar no contexto da tarefa.
- gh-157213 [https://github.com/python/cpython/issues/157213]: Corrige o problema em que **`asyncio.gather()`** deixava arestas obsoletas no grafo de *await* em tarefas filhas que persistiam após a conclusão da função.

- gh-156002 [https://github.com/python/cpython/issues/156002]: O módulo **`zipfile`** volta a ler membros por meio de um descompressor de terceiros instalado via alteração dinâmica (monkey-patching) do método privado `_get_decompressor()` para retornar um objeto que implementa apenas a antiga API do BZ2Decompressor do Python 3.3. Note que descompressores que não possuem o atributo `needs_input` e um método `decompress()` que aceite dois argumentos são vulneráveis à **CVE 2026-15310** [https://www.cve.org/CVERecord?id=CVE-2026-15310].
- gh-157159 [https://github.com/python/cpython/issues/157159]: A mensagem de uso exibida por `python -m tabnanny` quando nenhum argumento é fornecido agora sugere `python -m` em vez do arquivo-fonte.
- gh-157058 [https://github.com/python/cpython/issues/157058]: **`asyncio.wait_for()`** com um tempo limite não positivo agora registra o *waiter* no grafo de chamadas.
- gh-156933 [https://github.com/python/cpython/issues/156933]: Corrige valores de retorno inteiros incorretos de funções de retorno do **`ctypes`** em algumas plataformas, como s390x.
- gh-156988 [https://github.com/python/cpython/issues/156988]: **`asyncio.print_call_graph()`** não mais trunca a pilha de chamadas no gerador síncrono.
- gh-156970 [https://github.com/python/cpython/issues/156970]: Define a docstring de **`unittest.mock.Mock.return_value`**.
- gh-156924 [https://github.com/python/cpython/issues/156924]: **`annotationlib.ForwardRef.evaluate()`** agora resolve proxies de importação preguiçosa (*lazy import*) encontrados no espaço de nomes. Com **`FORWARDREF`**, uma importação preguiçosa que falha ao ser resolvida agora resulta em um **`ForwardRef`** em vez do **`proxy de importação preguiçosa`**, e com **`VALUE`**, a exceção subjacente é propagada.
- gh-127057 [https://github.com/python/cpython/issues/127057]: Corrige transportes UDP de **`asyncio.ProactorEventLoop`** de forma que uma **`ConnectionResetError`** levantada por `WSARecvFrom` deixe de interromper o laço de leitura.
- gh-156512 [https://github.com/python/cpython/issues/156512]: Corrige **`asyncio`** perdendo uma conexão duas vezes quando `resume_writing()` fecha o transporte.

- gh-155596 [https://github.com/python/cpython/issues/155596]: Correção para **`pprint.pprint()`** e **`pprint.pformat()`** com `expand=True`, que não respeitavam o parâmetro *width* para valores aninhados. A largura do prefixo `'key':` não era contabilizada ao determinar se um valor cabia na linha atual; assim, valores associados a chaves longas podiam exceder o limite de *width* em vez de serem expandidos.
- gh-155433 [https://github.com/python/cpython/issues/155433]: Corrige a perda de cancelamento externo por **`asyncio.TaskGroup`** após `cancel()`.
- gh-155418 [https://github.com/python/cpython/issues/155418]: Corrige o travamento de **`asyncio.TaskGroup`** quando uma tarefa criada por **`asyncio.eager_task_factory()`** cancela o grupo antes de ser suspendido.
- gh-155143 [https://github.com/python/cpython/issues/155143]: Corrige o vazamento da tarefa chamadora em **`asyncio.shield()`** — via grafo de *await* e funções de retorno — quando a função é chamada em um *future* que nunca é resolvido.
- gh-154470 [https://github.com/python/cpython/issues/154470]: Corrigido um `^J` espúrio impresso ao final de cada linha de código-fonte pelo comando `list` do **`pdb`** quando a saída está colorida (por exemplo, ao anexar a um processo em execução). A linha de código-fonte agora é limpa antes de ser colorida, assim como o comando `where` já fazia.
- gh-154335 [https://github.com/python/cpython/issues/154335]: O perfilador **`profiling.sampling`** agora respeita `NO_COLOR`, `FORCE_COLOR` e `PYTHON_COLORS`, e evita saídas coloridas quando não está escrevendo em um terminal.
- gh-154090 [https://github.com/python/cpython/issues/154090]: Registra o modo de perfilamento nos perfis binários do Tachyon e rejeite comparações diferenciais cujos modos conhecidos não correspondem.
- gh-152907 [https://github.com/python/cpython/issues/152907]: Restaura os sinalizadores de saída do terminal em modo “cooked” ao redor das funções de chamada de **`PyOS_InputHook`** no novo REPL (**`_pyrepl`**), de modo que a saída escrita por um gancho de entrada (por exemplo, um laço de eventos de um toolkit de GUI) não seja mais emitida com `OPOST` desabilitado e preserve seus caracteres de retorno de carro (*carriage returns*).
- gh-151179 [https://github.com/python/cpython/issues/151179]: Corrige um vazamento de *pidfd* em `_PidfdChildWatcher` no Linux: o observador

não vaza mais o descritor de arquivo do processo quando `waitpid()` falha com um erro diferente de **`ChildProcessError`**.

Documentação

- gh-57491 [https://github.com/python/cpython/issues/57491]: Reduz a profundidade do sumário na documentação EPUB de três para dois níveis, tornando-o muito mais curto e fácil de navegar em leitores de e-books.

Testes

- gh-158051 [https://github.com/python/cpython/issues/158051]: O ambiente de testes para iOS foi atualizado para garantir compatibilidade com o Xcode 27.

Construção

- gh-158010 [https://github.com/python/cpython/issues/158010]: Atualiza os scripts de build para Android e iOS para usar o OpenSSL 3.5.9.
- gh-157549 [https://github.com/python/cpython/issues/157549]: Usa `sysconf(_SC_PAGESIZE)` em vez de `getpagesize()` em `remote_debug.h` para que `_remote_debugging` compile no Darwin com `-Werror=implicit-function-declaration`.
- gh-152023 [https://github.com/python/cpython/issues/152023]: Atualiza as construções para Android para o SQLite 3.53.4. Além disso, habilita as funções `median()` e `percentile()`.

Windows

- gh-158010 [https://github.com/python/cpython/issues/158010]: Construções do Windows atualizadas para usar o OpenSSL 3.5.9.
- gh-157368 [https://github.com/python/cpython/issues/157368]: Corrige o truncamento de **`sys.version`** para versões do clang-cl 22 ou mais recentes no Windows. Também corrige o caso do Windows em Arm, onde o compilador da Microsoft era relatado em vez do clang. Patch por Chris Eibl.

- gh-157212 [https://github.com/python/cpython/issues/157212]: Corrige a compilação de `_Py_ThreadId()` no Windows ARM64 com toolchains MinGW, onde `__getReg()` não está disponível. O endereço do TEB agora é lido diretamente de `x18`. Patch por Christian Aurich.
- gh-152023 [https://github.com/python/cpython/issues/152023]: Atualiza as construções para Windows para o SQLite 3.53.4. Além disso, habilita as funções `median()` e `percentile()`.

macOS

- gh-158010 [https://github.com/python/cpython/issues/158010]: Atualiza o instalador do macOS para usar o OpenSSL 3.5.9.
- gh-152023 [https://github.com/python/cpython/issues/152023]: Atualiza o construções do instalador do macOS para usar o SQLite 3.53.4.

IDLE

- gh-158067 [https://github.com/python/cpython/issues/158067]: Corrige a caixa de diálogo File Open ("Abrir Arquivo") do IDLE para que liste arquivos de texto antes de arquivos Python ao ser aberta a partir do console ou de uma janela de saída.:
- gh-75512 [https://github.com/python/cpython/issues/75512]: A saída de uma thread, exibida no console IDLE enquanto uma instrução está sendo digitada, não mais se sobrepõe ao prompt da instrução.
- gh-89544 [https://github.com/python/cpython/issues/89544]: Corrige o travamento do IDLE após reiniciar o console enquanto recebe uma saída grande do processo do usuário.
- gh-59568 [https://github.com/python/cpython/issues/59568]: Corrige as caixas de diálogo "Save As" ("Salvar como") e "Open" ("Abrir") no IDLE para arquivos cujos nomes começam com ~ no Windows e no X11, ao utilizar versões do Tcl/Tk anteriores à 9.
- gh-69004 [https://github.com/python/cpython/issues/69004]: O item "Debugger" do menu "Debug" do IDLE agora fica desabilitado enquanto o código do usuário é executado, assim como o "Stack Viewer".
- gh-93966 [https://github.com/python/cpython/issues/93966]: O destaque de um erro de sintaxe no Console do IDLE agora é removido quando a

próxima instrução é inserida, e não apenas quando ocorre o próximo erro de sintaxe.

- gh-75234 [https://github.com/python/cpython/issues/75234]: Corrige as fontes de ajuda de edição e as associações de tecla na caixa de diálogo de configurações do IDLE após selecioná-los com o teclado.
- gh-55646 [https://github.com/python/cpython/issues/55646]: Corrige a falha do IDLE na inicialização quando a configuração do usuário contém uma associação de teclas inválida, como `<Alt-Key-up>` em vez de `<Alt-Key-Up>`. A associação inválida agora é ignorada, com a emissão de um aviso.

API C

- gh-156939 [https://github.com/python/cpython/issues/156939]: Corrige **`PyBytesWriter_FinishWithSize()`** para retornar um bytearray vazio quando **`_PyBytesWriter_CreateByteArray()`** é utilizado, em vez de retornar uma string de bytes vazia. Patch por Victor Stinner.
- gh-156939 [https://github.com/python/cpython/issues/156939]: Corrige o tratamento de erros de **`PyBytesWriter_Format()`**: restaura o escritor ao seu estado anterior em caso de erro. Patch por Victor Stinner.
- gh-140550 [https://github.com/python/cpython/issues/140550]: Correção da macro **`PyABIInfo_VAR`**: remoção de um `;` redundante. Anteriormente, o código `PyABIInfo_VAR(abi_info);` adicionava dois `;;`, o que é inválido em C++03. Patch por Victor Stinner.

Python 3.15.0 candidato a lançamento 2

Data de lançamento: 2026-09-01

Segurança

- gh-156723 [https://github.com/python/cpython/issues/156723]: Atualiza o libexpat [https://libexpat.github.io/] empacotado para a versão 2.8.4.
- gh-156002 [https://github.com/python/cpython/issues/156002]: Limita a quantidade de dados que o **`zipfile`** descompacta por leitura para membros compactados com bzip2, LZMA ou Zstandard, correspondendo ao

limite existente para deflate. Um pequeno membro de arquivo podia anteriormente se expandir em uma alocação ilimitada, mesmo quando lido em pequenos pedaços.

- gh-155999 [https://github.com/python/cpython/issues/155999]: Corrige os filtros de extração `tar` e `data` do **`tarfile`** criando diretórios fora do destino para membros cujo nome sai do destino e retorna a ele, como `../evil/../dest/sub/file`. A verificação de contenção usava o caminho resolvido, mas diretórios intermediários eram criados a partir do nome como fornecido.
- gh-155292 [https://github.com/python/cpython/issues/155292]: Altera o módulo **`stringprep`** e o codec **`encodings.idna`** para não considerar atributos de pontos de código Unicode além daqueles definidos na **RFC 3454** [https://datatracker.ietf.org/doc/html/rfc3454.html].
- gh-155694 [https://github.com/python/cpython/issues/155694]: Corrige a **CVE 2026-15806** [https://www.cve.org/CVERecord?id=CVE-2026-15806] restringindo as credenciais do **`HTTPPasswordMgr`** ao esquema da URL, evitando que credenciais armazenadas para uma URL HTTPS sejam usadas para uma URL HTTP correspondente, enquanto URIs sem esquema continuam a corresponder a qualquer esquema.

Núcleo e embutidos

- gh-155725 [https://github.com/python/cpython/issues/155725]: **`tracemalloc`** não adquire mais a GIL nem cria um estado de thread temporário para rastrear alocações de memória: quadros de traceback agora armazenam strings UTF-8 simples em vez de objetos str do Python, e tracebacks são capturados usando o estado de thread Python já associado à thread chamadora, mesmo se ela não estiver anexada. Threads sem um estado de thread Python registram o traceback como `<unknown>`.
- gh-148750 [https://github.com/python/cpython/issues/148750]: **`encodings`** não é mais congelado, devido a problemas ao importar submódulos ao usar uma biblioteca padrão compactada em zip.
- gh-155254 [https://github.com/python/cpython/issues/155254]: Evita aviso sobre biblioteca padrão ausente ao usar Arquivos _pth.
- gh-155978 [https://github.com/python/cpython/issues/155978]: Corrige um vazamento de memória na construção com threads livres ao definir ou

excluir um método especial (como `__repr__`) em uma classe que possui muitas subclasses.

- gh-155752 [https://github.com/python/cpython/issues/155752]: Corrige um travamento quando um argumento de **`types.GenericAlias`** ganha um gancho `__typing_subst__` após os parâmetros do apelido terem sido armazenados em cache.
- gh-129752 [https://github.com/python/cpython/issues/129752]: Não atualiza contadores adaptativos na construção com threads livres quando o bytecode local de thread está desativado (`-X tlbc=0`). Patch por Donghee Na.
- gh-155515 [https://github.com/python/cpython/issues/155515]: Rastreia os iteradores internos do HAMT, que dão suporte à iteração sobre um **`contextvars.Context`**, com o coletor de lixo. Um ciclo de referência passando por tal iterador nunca era coletado, vazando todo o contexto sobre o qual iterava.
- gh-155400 [https://github.com/python/cpython/issues/155400]: Corrige um impasse (*deadlock*) na construção com threads livres entre duas threads atribuindo a um método especial da mesma classe. Ao aplicar as atualizações de slot de tipo com o mundo parado (stop-the-world), apenas a trava de tipo era impedida de ser liberada; o mutex do dicionário do tipo ainda podia ser liberado e readquirido, na ordem incorreta, se a thread bloqueasse no mutex do stop-the-world.
- gh-155363 [https://github.com/python/cpython/issues/155363]: Corrige um vazamento na construção com threads livres quando a criação de um estado de thread falha após um slot QSBR interno ter sido reservado para ele. O slot nunca podia ser recuperado, fazendo com que a matriz QSBR crescesse sem limites em falhas repetidas.
- gh-154701 [https://github.com/python/cpython/issues/154701]: Corrige um laço infinito no JIT quando uma saída lateral de `FOR_ITER` vincula um executor de volta a si mesmo.
- gh-154196 [https://github.com/python/cpython/issues/154196]: Melhora as mensagens de **`AttributeError`** de importações preguiçosas (lazy imports) não resolvidas. Patch por Bartosz Sławecki.
- gh-151377 [https://github.com/python/cpython/issues/151377]: Corrige condições de corrida em construções com threads livres ao atualizar slots de tipo para classes recém-criadas e ao remover entradas do dicionário de subclasses de um tipo base.

- gh-148817 [https://github.com/python/cpython/issues/148817]: Dobra literais constantes grandes de listas e conjuntos usados como o iterável de um laço `for` ou teste `in`/`not in` em uma `tuple` ou `frozenset` constante, restaurando uma otimização feita anteriormente pelo otimizador AST que foi perdida quando a dobragem de constantes mudou para o CFG.
- gh-85260 [https://github.com/python/cpython/issues/85260]: `compile()` agora levanta `ValueError` em vez de travar em uma construção de depuração se um campo identificador de um nó AST (como o nome de uma função, uma classe, um módulo importado ou uma exceção capturada) for `"None"`, `"True"` ou `"False"`.

Biblioteca

- gh-156523 [https://github.com/python/cpython/issues/156523]: Corrige `asyncio.as_completed()` não registrando a tarefa aguardada no grafo de chamadas.
- gh-156408 [https://github.com/python/cpython/issues/156408]: Corrige `asyncio.print_call_graph()` levantando `AttributeError` quando chamado em uma tarefa que já foi concluída.
- gh-156404 [https://github.com/python/cpython/issues/156404]: Restaura a compatibilidade com arquivos `.pth` gerados antes do Python 3.15 no módulo `site`. Injeta a variável `fullname` no quadro que executa o código pth.
- gh-156353 [https://github.com/python/cpython/issues/156353]: Corrige a análise do `configparser` ao usar espaço em branco em *delimiters*.
- gh-156112 [https://github.com/python/cpython/issues/156112]: `_ios_support` não falha mais ao importar quando `ctypes.util.find_library()` não consegue resolver o ambiente de execução do ObjC através do cache compartilhado do dyld, como é o caso no iOS 13. O ambiente de execução agora é carregado pelo nome, o qual o dyld resolve diretamente contra o cache.
- gh-155974 [https://github.com/python/cpython/issues/155974]: Corrige uma regressão no Python 3.15: `addstr()`, `addnstr()`, `insstr()` e `insnstr()` voltam a restaurar os atributos da janela quando a escrita falha, em vez de deixar o *attr* temporário aplicado.
- gh-155952 [https://github.com/python/cpython/issues/155952]: Corrige as assinaturas de `sqlite3.Connection.execute()`, `warnings.warn()` e

locale.setlocale() que renderizavam `<unrepresentable>` em vez dos padrões corretos para os parâmetros.

- gh-155888 [https://github.com/python/cpython/issues/155888]: Corrige **asyncio.WriteTransport.writelines()** travando o transporte quando o último pedaço de dados estiver vazio.
- gh-155869 [https://github.com/python/cpython/issues/155869]: Corrige **reorganize()** em **dbm.dumb** falhando ao persistir deslocamentos de valores atualizados, o que podia causar perda de dados após reabrir o banco de dados.
- gh-155781 [https://github.com/python/cpython/issues/155781]: Corrige um possível travamento em construções com threads livres quando o registro de conversores do **sqlite3** for modificado concorrentemente enquanto um cursor constrói seu mapa de conversão de linhas.
- gh-155702 [https://github.com/python/cpython/issues/155702]: Corrige atribuição de fatia em **sqlite3.Blob** com passo (step). Ele alterava o objeto de bytes lido do blob, que para um único byte é um singleton imortal, de modo que o valor daquele byte era alterado em todo o processo.
- gh-155468 [https://github.com/python/cpython/issues/155468]: Corrige **netrc** para distinguir tokens entre aspas vazios do fim do arquivo, para que arquivos malformados não façam mais com que o conteúdo restante seja silenciosamente ignorado.
- gh-155519 [https://github.com/python/cpython/issues/155519]: Evita uma condição de corrida de dados em construções com threads livres ao ler e escrever variáveis de contexto a partir de threads diferentes.
- gh-155009 [https://github.com/python/cpython/issues/155009]: Corrige **argparse.ArgumentParser** para preservar o nome do programa a partir de `sys.argv[0]` quando um módulo nomeado for executado como o programa principal sem substituir `sys.argv[0]`.
- gh-155063 [https://github.com/python/cpython/issues/155063]: Atualiza a versão do pip empacotado no ensurepip para a versão 26.2.1
- gh-153711 [https://github.com/python/cpython/issues/153711]: No macOS, adiciona verificações em tempo de execução ao redor das chamadas de sistema *pipe2 (2)* *[https://manpages.debian.org/pipe2 (2)]* e *dup3 (2)* *[https://manpages.debian.org/dup3 (2)]*, além das verificações existentes em tempo de construção. Isso significa que o Python construído no macOS 27 (onde essas chamadas estão disponíveis) pode ser executado no macOS 26 (onde não estão).

- gh-154871 [https://github.com/python/cpython/issues/154871]: Corrigido um travamento em `asyncio.Task.get_context()` quando chamado em uma tarefa não inicializada.
- gh-154568 [https://github.com/python/cpython/issues/154568]: Corrige a desserialização com pickle no `array` de float16 em little-endian.
- gh-154566 [https://github.com/python/cpython/issues/154566]: Corrige `array.array.byteswap()` corrompendo dados para arrays `'Zd'` (complex double) com mais de um elemento: o laço de item de 16 bytes avançava o ponteiro do buffer em apenas 8 bytes por iteração, fazendo com que os itens após o primeiro fossem embaralhados.
- gh-154467 [https://github.com/python/cpython/issues/154467]: Corrigido o anexo remoto do `pdb` (`python -m pdb -p PID`) enviando um prompt vazio para o cliente em vez de `(Pdb)` quando o processo alvo possui um terminal interativo.
- gh-153772 [https://github.com/python/cpython/issues/153772]: Verificações `isinstance()` em relação a classes do `collections.abc` tais como `Mapping` não levantam mais `AttributeError` quando a instância não possui `__class__`. O mecanismo de classe base abstrata agora recorre ao tipo do objeto nesse caso, correspondendo ao comportamento do `isinstance()` embutido.
- gh-154086 [https://github.com/python/cpython/issues/154086]: Armazena contagens de amostras por thread nos flamegraphs Tachyon para que a filtragem de uma thread atualize as larguras e totais dos quadros.
- gh-154060 [https://github.com/python/cpython/issues/154060]: Armazena a duração medida e a taxa de amostragem em perfis binários do Tachyon para que permaneçam disponíveis em flamegraphs reproduzidos.
- gh-154085 [https://github.com/python/cpython/issues/154085]: Evita que flamegraphs diferenciais dupliquem o tempo próprio (self time) entre nós de linha para a mesma função.
- gh-154088 [https://github.com/python/cpython/issues/154088]: Faz com que o Tachyon conte e renderize pilhas omitidas de forma consistente em flamegraphs diferenciais.
- gh-154089 [https://github.com/python/cpython/issues/154089]: Não exibe estatísticas de eficiência de amostragem indisponíveis em flamegraphs do Tachyon reproduzidos.
- gh-153158 [https://github.com/python/cpython/issues/153158]: Remove o argumento errôneo *width* de

`calendar.HTMLCalendar.formatmonthpage()`, o qual podia descartar o ano do cabeçalho.
- gh-113329 [https://github.com/python/cpython/issues/113329]: Corrige **`OSError`** sendo levantada ao tentar executar doctests em objetos de classe no REPL. Patch por Sten Wessel.

Construção

- gh-156369 [https://github.com/python/cpython/issues/156369]: Atualiza os scripts de construção para Android e iOS para usar OpenSSL 3.5.8.
- gh-156115 [https://github.com/python/cpython/issues/156115]: Construções do simulador iOS agora são configuradas com `-mios-simulator-version-min` em vez de `-mios-version-min` do dispositivo. O sinalizador do dispositivo fazia com que o ligador (link) rejeitasse bibliotecas resolvidas do SDK do simulador, falhando silenciosamente a detecção da biblioteca.

Windows

- gh-156369 [https://github.com/python/cpython/issues/156369]: Construções do Windows atualizadas para usar o OpenSSL 3.5.8.

macOS

- gh-156369 [https://github.com/python/cpython/issues/156369]: Atualiza o instalador do macOS para usar OpenSSL 3.5.8.

Ferramentas/Demos

- gh-154059 [https://github.com/python/cpython/issues/154059]: Corrige as unidades de tempo nas dicas de ferramentas do gráfico de chama (flamegraph) do Tachyon, levando em consideração o intervalo de amostragem ao converter amostras para milissegundos.

API C

- gh-131740 [https://github.com/python/cpython/issues/131740]: Na construção com threads livres, `PyUnstable_GC_VisitObjects()` agora também visita objetos congelados (objetos movidos para a geração permanente por `gc.freeze()`), correspondendo ao comportamento da construção padrão.

Python 3.15.0 candidato a lançamento 1

Data de lançamento: 2026-08-04

Segurança

- gh-152674 [https://github.com/python/cpython/issues/152674]: Os métodos de `xml.etree.ElementTree.Element`: `findall()`, `iterfind()` e `find()` evitam comportamento quadrático ao usar predicados de índice XPath (`[1]`, `[last()]`, `[last()-N]`) em documentos XML com muitos irmãos de mesma tag.

Núcleo e embutidos

- gh-154902 [https://github.com/python/cpython/issues/154902]: Corrige uma falha quando `__conditional_annotations__` é reatribuído a um objeto que não é um conjunto.
- gh-133931 [https://github.com/python/cpython/issues/133931]: Corrige condições de corrida de dados ao definir atributos de objetos de função na construção com threads livres.
- gh-154775 [https://github.com/python/cpython/issues/154775]: Ao corresponder um literal complexo em instruções `case`, um sinal `+` estranho (por exemplo, `1++1j` ou `1-+1j`) não é mais permitido.
- gh-154709 [https://github.com/python/cpython/issues/154709]: Corrige um acesso fora dos limites em iteradores reversos de dicionário quando o dicionário subjacente é limpo e modificado após a criação do iterador.
- gh-154695 [https://github.com/python/cpython/issues/154695]: Corrige `asyncio.Task` levantando `AttributeError` quando criada com `eager_start=True` e sem argumento *loop* explícito.

- gh-153809 [https://github.com/python/cpython/issues/153809]: Corrige falha do interpretador ao desalocar objetos de `asyncio.Task` em construções com threads livres. Contribuição de Sergey Miryanov.
- gh-154275 [https://github.com/python/cpython/issues/154275]: Corrige uma falha ao obter `__parameters__` profundamente aninhados de objetos `types.GenericAlias`.
- gh-154014 [https://github.com/python/cpython/issues/154014]: Corrige uma asserção do JIT durante o encerramento do interpretador inicializando campos `vm_data` para executores frios que ignoram `_Py_ExecutorInit()`.
- gh-153932 [https://github.com/python/cpython/issues/153932]: Corrige problema de segurança de thread no método `__reduce__` do `enumerate`.
- gh-153881 [https://github.com/python/cpython/issues/153881]: Corrige potencial corrida de dados ao chamar `__getstate__()` sob a construção com threads livres.
- gh-153570 [https://github.com/python/cpython/issues/153570]: Corrige um uso-após-liberação (use-after-free) em `bytearray.take_bytes()` quando o método `__index__()` do argumento redimensiona o bytearray. Patch por tonghuaroot.
- gh-153419 [https://github.com/python/cpython/issues/153419]: Corrige múltiplas falhas e vazamentos de referência em `bytearray` causados por ignorar o `__init__()` e código de configuração de estado quebrado.
- gh-153236 [https://github.com/python/cpython/issues/153236]: Propaga exceções levantadas ao importar submódulos preguiçosos em vez de relatá-las como atributos ausentes.
- gh-148874 [https://github.com/python/cpython/issues/148874]: Ignores interrupções imediatamente após chamar o método `__enter__` de um gerenciador de contexto em uma instrução `with`. Isso garante que o método `__exit__` seja sempre chamado em uma instrução `with`.
- gh-150208 [https://github.com/python/cpython/issues/150208]: Evite usar aspas duplas em valores de string provenientes de `pyconfig.h` nas variáveis de `sysconfigdata`.

Biblioteca

- gh-155063 [https://github.com/python/cpython/issues/155063]: Atualiza a versão do pip empacotado no ensurepip para a versão 26.2

- gh-154936 [https://github.com/python/cpython/issues/154936]: Corrige o decodificador `json` em Python puro para relatar a posição correta para caracteres de controle literais inválidos em strings JSON.
- gh-154892 [https://github.com/python/cpython/issues/154892]: Corrige um erro no acelerador C para `zoneinfo` onde subclasses de `datetime.datetime` retornando `-1` para `hour`, `minute` ou `second` podiam incorretamente levantar um `SystemError`.
- gh-154848 [https://github.com/python/cpython/issues/154848]: O acelerador C do `pickle` agora impõe limites de quadro ao desserializar com pickle, assim como a implementação em Python puro já fazia. Um argumento que se estende além do limite de um quadro, ou um quadro que começa antes do anterior ter terminado, agora levanta `pickle.UnpicklingError` em vez de ser lido silenciosamente através do limite. Isso evita que os dados carregados divirjam da desmontagem do `pickletools` do mesmo pickle.
- gh-154885 [https://github.com/python/cpython/issues/154885]: `os.get_terminal_size()` agora verifica `isatty` antes de chamar `ioctl`, o que reduz o ruído de log no Android.
- gh-109638 [https://github.com/python/cpython/issues/109638]: Corrige tempo exponencial em `csv.Sniffer.sniff()` para uma amostra que contém muitos caracteres de aspas. Um caractere de aspa duplicado agora também é detectado em um campo que contém o delimitador ou uma quebra de linha.
- gh-98820 [https://github.com/python/cpython/issues/98820]: Corrige tempo quadrático em `csv.Sniffer.sniff()` para uma amostra que contém campos entre aspas, em particular para uma única coluna de campos entre aspas.
- gh-154738 [https://github.com/python/cpython/issues/154738]: Corrige **`ExternalEntityParserCreate()`** não propagando a configuração de adiamento de reanálise para o subanalisador, o que deixava **`GetReparseDeferralEnabled()`** retornando um valor não inicializado. Patch por tonghuaroot.
- gh-93251 [https://github.com/python/cpython/issues/93251]: Corrige `UnicodeDecodeError` em funções do `socket` (como `getaddrinfo()` e `gethostbyaddr()`) quando a mensagem de erro localizadas da biblioteca C não é UTF-8: decodifica-a a partir da codificação da localidade (locale).

- gh-148468 [https://github.com/python/cpython/issues/148468]: Corrige uma regressão no `argparse` onde a ajuda de argumento colorida contendo especificadores de formato não aceitava objetos proxy semelhantes a strings.
- gh-142035 [https://github.com/python/cpython/issues/142035]: Corrige a quebra de linha incorreta do texto de ajuda do `argparse` quando a cor está ativada.
- gh-135736 [https://github.com/python/cpython/issues/135736]: Corrige `asyncio.TaskGroup` para não envolver um `GeneratorExit` em um `BaseExceptionGroup` se ele foi levantado pelo corpo do grupo de tarefas e nenhuma das tarefas no grupo levantou exceções.
- gh-154551 [https://github.com/python/cpython/issues/154551]: Corrige `ctypes.util.find_library()` retornando `None` em localidades não-UTF-8.
- gh-79366 [https://github.com/python/cpython/issues/79366]: Corrigida uma condição de corrida no `logging`: se um manipulador era removido enquanto um registro estava sendo emitido, os manipuladores seguintes do mesmo logger podiam ser ignorados.
- gh-154189 [https://github.com/python/cpython/issues/154189]: Corrigido um potencial uso-após-liberação ao chamar `functools.partial()`. Agora, ao invocar um objeto `partial()`, a função armazenada, argumentos posicionais e argumentos nomeados são preservados durante a chamada em caso de reentrada.
- gh-73458 [https://github.com/python/cpython/issues/73458]: Corrige `logging.config.listen()`: deixava o chamador esperando pelo evento `ready` para sempre se o servidor não pudesse ser iniciado, por exemplo, se a porta fosse inválida ou já estivesse em uso. Agora também se vincula a um endereço IPv6 se o host não tiver endereço IPv4, por exemplo, se `localhost` for apenas um alias para `::1`.
- gh-154460 [https://github.com/python/cpython/issues/154460]: Corrige `time.strftime()` e `datetime.datetime.strftime()` retornando um número de semana ISO 8601 errado (`%V`) no OpenBSD.
- gh-154435 [https://github.com/python/cpython/issues/154435]: Corrige `os.posix_fadvise()` e `os.posix_fallocate()` no DragonFly BSD: levantavam `OSError` com um código de erro sem sentido, porque essas funções retornam -1 e definem `errno` lá.

- gh-154399 [https://github.com/python/cpython/issues/154399]: Corrige a ativação do `venv` em um csh não interativo: `activate.csh` não falha mais quando a variável `prompt` não está definida.
- gh-154389 [https://github.com/python/cpython/issues/154389]: Corrige `uuid.uuid1()` no OpenBSD: retornava um UUID versão 4, porque `uuid_create()` gera UUIDs aleatórios nesta plataforma.
- gh-154324 [https://github.com/python/cpython/issues/154324]: Corrige `os.sendfile()` no illumos: não relata mais uma transferência bem-sucedida quando a chamada de sistema subjacente falhou sem escrever nenhum dado.
- gh-154307 [https://github.com/python/cpython/issues/154307]: Corrige `tempfile.TemporaryDirectory.cleanup()` no DragonFly BSD, onde a remoção de um arquivo com o sinalizador `UF_NOUNLINK` falhava com `EISDIR` em vez de `EPERM`.
- gh-154291 [https://github.com/python/cpython/issues/154291]: Corrige `socket.has_dualstack_ipv6()` para retornar `False` em plataformas como DragonFly BSD, onde definir `IPV6_V6ONLY` para 0 silenciosamente não tem efeito.
- gh-154283 [https://github.com/python/cpython/issues/154283]: No DragonFly BSD, `threading.get_native_id()` agora retorna um valor que é único entre processos, correspondendo a outras plataformas.
- gh-154258 [https://github.com/python/cpython/issues/154258]: Corrige uma falha em `mmap.mmap.resize()` no NetBSD ao aumentar um mapeamento anônimo compartilhado. O **`resize()`** agora levanta `ValueError` neste caso, como já fazia no Linux.
- gh-131565 [https://github.com/python/cpython/issues/131565]: `ctypes.util.dllist()` agora funciona no NetBSD. Ele é implementado no módulo de extensão **`_ctypes`** para que `dl_iterate_phdr()` reporte todas as bibliotecas compartilhadas carregadas: no NetBSD ele reporta apenas o grupo de mapa de links do objeto chamador, o que os excluía quando chamado via ctypes.
- gh-154225 [https://github.com/python/cpython/issues/154225]: Corrige `os.openpty()` no Solaris e illumos: não deixa mais o pseudoterminal como o terminal de controle do processo chamador.
- gh-154227 [https://github.com/python/cpython/issues/154227]: Corrige `os.posix_openpt()` no OpenBSD, onde rejeitava o sinalizador `O_CLOEXEC`.

- gh-145030 [https://github.com/python/cpython/issues/145030]: Corrige transportes de encadeamento (pipe) de escrita do **asyncio** para FIFOs nomeados no macOS e Solaris. Dados não lidos no FIFO faziam o transporte reinterpretar um evento de poll como desconexão do leitor, fechando incorretamente o transporte.
- gh-154176 [https://github.com/python/cpython/issues/154176]: Corrige uma falha em **locale.strxfrm()** no DragonFly BSD, cujo `wcsxfrm()` não oferece suporte a tamanho zero.
- gh-154836 [https://github.com/python/cpython/issues/154836]: Corrige **subprocess.Popen.wait()** levantando **TypeError** (macOS e outros BSDs) ou **OverflowError** (Linux) para valores muito grandes de *timeout* como `float('inf')`, uma regressão do 3.15 no novo wait orientado a eventos. Também corrige **select.kqueue.control()** mascarando **OverflowError** para timeouts fora do intervalo como **TypeError**.
- gh-154053 [https://github.com/python/cpython/issues/154053]: Corrige a compilação do módulo **ssl** contra o LibreSSL, que não fornece `SSL_CTX_set1_sigalgs_list()` e `SSL_CTX_set1_client_sigalgs_list()`. Os métodos **set_server_sigalgs()** e **set_client_sigalgs()** de **ssl.SSLContext** agora levantam **NotImplementedError** no LibreSSL.
- gh-154001 [https://github.com/python/cpython/issues/154001]: Corrige **random.binomialvariate()** levantando **ZeroDivisionError** quando **random.random()** retorna zero.
- gh-153908 [https://github.com/python/cpython/issues/153908]: Corrige corrida de dados ao chamar **repr()** em **itertools.count** sob a construção com threads livres.
- gh-153896 [https://github.com/python/cpython/issues/153896]: Desduplica argumentos não-hasheáveis em **typing.Literal**.
- gh-153864 [https://github.com/python/cpython/issues/153864]: Em uma construção de **curses** larga (wide), **curses.window.insch()** agora insere um byte não-ASCII como o caractere que ele codifica na codificação da janela, consistentemente com **addch()**, em vez de seu ponto de código.
- gh-153862 [https://github.com/python/cpython/issues/153862]: Em uma construção de **curses** larga, **curses.window.inch()** agora retorna o byte codificado da localidade de um caractere não-ASCII, correspondendo a **instr()**, em vez do byte baixo de seu ponto de código.

- gh-146011 [https://github.com/python/cpython/issues/146011]: Corrige um uso-após-liberação na heap na implementação C do `decimal` ao chamar `repr()` após excluir o `Context`.
- gh-153761 [https://github.com/python/cpython/issues/153761]: Corrige o cancelamento de `asyncio.loop.sock_accept()` descartando uma conexão pendente.
- gh-127049 [https://github.com/python/cpython/issues/127049]: Corrige uma condição de corrida no `asyncio` no Unix onde `asyncio.subprocess.Process.send_signal()`, `terminate()` ou `kill()` podiam sinalizar um processo não relacionado que foi reciclado no PID do filho já recolhido quando ThreadedChildWatcher é usado. Patch por Kumar Aditya.
- gh-153404 [https://github.com/python/cpython/issues/153404]: `urllib.robotparser.RobotFileParser` agora ignora silenciosamente um valor de `Crawl-delay` ou `Request-rate` escrito com dígitos não-decimais (como `U+00B2 SUPERSCRIPT TWO`) em vez de levantar `ValueError` e abortar a análise de todo o arquivo `robots.txt`.
- gh-153291 [https://github.com/python/cpython/issues/153291]: Corrige uma corrida de dados em `readline.get_completer()` e `readline.get_pre_input_hook()` na construção com threads livres: os getters liam o gancho armazenado sem a seção crítica que os setters correspondentes mantêm.
- gh-153133 [https://github.com/python/cpython/issues/153133]: Corrige um vazamento de socket em `asyncio.loop.create_connection()` quando o transporte não pode ser criado.
- gh-152431 [https://github.com/python/cpython/issues/152431]: Corrige `asyncio.StreamWriter.start_tls()` para manter o transporte vinculado do `StreamReader` em sincronia com o transporte atualizado.
- gh-151955 [https://github.com/python/cpython/issues/151955]: Permite que mais tipos sejam usados no argumento `bound` para `typing.ParamSpec` e `typing.TypeVarTuple`.
- gh-119710 [https://github.com/python/cpython/issues/119710]: Corrige o subprocesso `wait()` de `asyncio` travando quando o processo saiu, mas um de seus pipes é mantido aberto por um processo filho herdado (então o pipe nunca atinge o EOF). O `wait()` agora retorna assim que o processo sai, independentemente do estado dos pipes.

- gh-151022 [https://github.com/python/cpython/issues/151022]: Corrige rastreamentos de pilha do `_remote_debugging` para objetos de código com tabelas de linhas grandes.
- gh-149319 [https://github.com/python/cpython/issues/149319]: O REPL do **`asyncio`** agora ignora **`PYTHONSTARTUP`** e **`PYTHON_BASIC_REPL`** quando **`-E`** ou **`-I`** é usado. Patch por Jonathan Dung.
- gh-148665 [https://github.com/python/cpython/issues/148665]: A função socket.shutdown agora está ativada para compilações do Emscripten. No entanto, se o runtime não implementar a chamada de sistema shutdown, exibirá o erro “Function not implemented”.
- gh-140326 [https://github.com/python/cpython/issues/140326]: Corrige o espaço de nomes do REPL do **`asyncio`** para que importações relativas não sejam mais resolvidas contra o pacote **`asyncio`** e `__file__` não seja mais definido.
- gh-103925 [https://github.com/python/cpython/issues/103925]: Corrige **`csv.Sniffer.sniff()`** para uma amostra com terminações de linha `\r\n` em que um campo entre aspas termina uma linha: uma letra podia ser detectada como o delimitador.

Documentação

- gh-118150 [https://github.com/python/cpython/issues/118150]: Esclarece na documentação do **`difflib`** o que *junk* realmente faz, suas desvantagens e como controlá-lo.

Testes

- gh-76595 [https://github.com/python/cpython/issues/76595]: Adiciona testes de C API para **`PyCapsule_Import()`**.
- gh-154211 [https://github.com/python/cpython/issues/154211]: Adiciona `test.support.skip_if_huge_c_stack()` e usa-o para pular testes que esgotam a pilha C se o limite da pilha for muito grande (por exemplo, no DragonFly BSD ou com `ulimit -s unlimited`).
- gh-154167 [https://github.com/python/cpython/issues/154167]: O executor de testes (regrtest) agora restaura o manipulador de SIGINT padrão se ele foi herdado como ignorado, para que a suíte de testes não trave mais quando executada como um trabalho em segundo plano do shell.

- gh-154144 [https://github.com/python/cpython/issues/154144]: Corrige a construção do módulo `_testcapi` no NetBSD.
- gh-152548 [https://github.com/python/cpython/issues/152548]: Adiciona o decorador `test.support.isolation.runInSubprocess()` para executar um método de teste ou subclasse de `TestCase` em um subprocesso do interpretador limpo, isolado do restante da execução do teste.

Construção

- gh-154070 [https://github.com/python/cpython/issues/154070]: Construção do módulo `curses` contra um ncurses capaz de caracteres largos mesmo quando não for chamado `ncursesw` – por exemplo, o ncurses pkgsrc no NetBSD e illumos, ou o ncurses do sistema no macOS. Essa biblioteca anteriormente produzia uma compilação estreita.
- gh-138800 [https://github.com/python/cpython/issues/138800]: Corrige o nome da biblioteca no python3.pc no Android.

Windows

- gh-124111 [https://github.com/python/cpython/issues/124111]: Atualiza as compilações do Windows para usar Tcl/Tk 9.0.4.

Ferramentas/Demos

- gh-154580 [https://github.com/python/cpython/issues/154580]: Corrige `python-gdb.py` levantando `UnicodeEncodeError` ao exibir formatadamente (pretty-printing) uma `str` não-ASCII em uma localidade cujo conjunto de caracteres do host não pode codificá-la, como qualquer string não-ASCII na localidade C.
- gh-152384 [https://github.com/python/cpython/issues/152384]: Os pacotes CPython Pixi agora estão todos acessíveis no mesmo subdiretório `Tools/pixi-packages`, em vez de `Tools/pixi-packages/{variant}` como antes. As variantes agora são selecionadas não via subdiretório, mas via `flags`; veja https://pixi.prefix.dev/latest/concepts/package_specifications/#extras-and-flags para instruções de uso. A variante `tsan-freethreading` foi

renomeada para `tsan_freethreading`, enquanto as variantes `default`, `asan` e `freethreading` mantêm seus nomes anteriores.

API C

- gh-122931 [https://github.com/python/cpython/issues/122931]: Permite importar extensões C de ABI estável que incluem uma tupla multiarch em seu nome de arquivo, por exemplo, `foo.abi3-x86-64-linux-gnu.so`.

Python 3.15.0 beta 4

Data de lançamento: 2026-07-18

Segurança

- gh-153030 [https://github.com/python/cpython/issues/153030]: Corrigida a complexidade quadrática na análise incremental de construções longas não terminadas (como tags ou comentários) no **`html.parser.HTMLParser`**, que poderia ser explorada para uma negação de serviço.
- gh-152216 [https://github.com/python/cpython/issues/152216]: Atualiza o libexpat [https://libexpat.github.io/] empacotado para a versão 2.8.2.
- gh-151987 [https://github.com/python/cpython/issues/151987]: O método **`tarfile.TarFile.extract()`** agora aplica o filtro fornecido quando extrai um alvo de link do arquivo como reserva (fallback).
- gh-151981 [https://github.com/python/cpython/issues/151981]: No **`tarfile`**, a busca (seeking) em um fluxo agora para quando o fim do fluxo é atingido.
- gh-151558 [https://github.com/python/cpython/issues/151558]: Corrigida uma vulnerabilidade nos filtros de extração `data` e `tar` do **`tarfile`** onde arquivos manipulados podiam criar um link simbólico apontando para fora do diretório de destino. Esta foi uma desvio da **CVE 2025-4330** [https://www.cve.org/CVERecord?id=CVE-2025-4330].
- gh-150743 [https://github.com/python/cpython/issues/150743]: O **`http.client`** agora limita o número de linhas de cabeçalho final de resposta em blocos (chunked-response trailer lines) que ele lerá para

`max_response_headers` (100 por padrão), e o número de respostas intermediárias (1xx) que ele pulará para 100. Um servidor malicioso ou quebrado podia anteriormente transmitir linhas de cabeçalho final ou respostas `100 Continue` para sempre, travando o cliente mesmo quando um limite de tempo de socket estava em uso. Relatado por `@YLChen-007` via GHSA-w4q2-g22w-6fr4.

- gh-143927 [https://github.com/python/cpython/issues/143927]: Normaliza todas as terminações de linha (CR, CRLF e LF) para LF+TAB ao escrever valores multilinha do configparser.

Núcleo e embutidos

- gh-153298 [https://github.com/python/cpython/issues/153298]: Corrige uma corrida de dados na inicialização dos `__parameters__` de `types.GenericAlias` em construções com threads livres.
- gh-153205 [https://github.com/python/cpython/issues/153205]: Corrige um potencial `SystemError` durante chamadas vetoriais quando a alocação de memória falha. Um `MemoryError` agora é levantado em seu lugar.
- gh-152682 [https://github.com/python/cpython/issues/152682]: Corrige desreferência de ponteiro NULL em `compile()` quando um nome reservado (por exemplo, `__classdict__`) é usado como nome de parâmetro de tipo e a alocação de memória falha ao formatar a mensagem de erro.
- gh-152635 [https://github.com/python/cpython/issues/152635]: Corrige uma falha causada ao ficar sem memória ao criar um canal `_interpchannels`. Agora uma exceção `MemoryError` é levantada corretamente.
- gh-152405 [https://github.com/python/cpython/issues/152405]: Não expõe o mapeamento interno de `types.MappingProxyType` ao realizar operações de comparação rica com tipos fora da biblioteca padrão. Anteriormente, era possível mutar o mapeamento interno de um proxy lá. Agora passamos não a instância original de `dict`, mas sua cópia, ao lidar com tipos personalizados.
- gh-152492 [https://github.com/python/cpython/issues/152492]: O método `update` de `collections.OrderedDict` agora pode aceitar `frozendict` como argumento.
- gh-152375 [https://github.com/python/cpython/issues/152375]: Corrige comportamento indefinido quando uma função de retorno do

`sys.monitoring` levantava uma exceção enquanto o programa estava seguindo um desvio ou laço.

- gh-152235 [https://github.com/python/cpython/issues/152235]: Adia o rastreamento pelo GC de `set.intersection()`, `set.difference()`, `set.symmetric_difference()`, `set.union()` e `set.__sub__`. Patch por Donghee Na.
- gh-152235 [https://github.com/python/cpython/issues/152235]: Adia o rastreamento pelo GC de um `set` ou `frozenset` para o final de sua construção a partir de um iterável. Patch por Donghee Na.
- gh-152228 [https://github.com/python/cpython/issues/152228]: Corrige uma falha de asserção quando o python é construído em modo de depuração que acontecia em `str.replace()` sob uma situação de memória limitada.
- gh-151722 [https://github.com/python/cpython/issues/151722]: **`frozendict.fromkeys()`** agora só rastreia o `frozendict` no coletor de lixo depois que o dicionário está totalmente inicializado. Patch por Donghee Na e Victor Stinner.
- gh-151763 [https://github.com/python/cpython/issues/151763]: Corrige possível falha na desalocação de `types.CodeType`.
- gh-152020 [https://github.com/python/cpython/issues/152020]: Na construção com threads livres, `asyncio.all_tasks()` não perde mais tarefas iniciadas de forma antecipada (eager) quando chamado a partir de uma thread diferente daquela que executa o laço de eventos.
- gh-151763 [https://github.com/python/cpython/issues/151763]: Corrige uma potencial falha em `compile()`, `exec()`, `eval()` e `ast.parse()` quando uma alocação falha: o analisador ou compilador podia retornar sem definir uma exceção.
- gh-151912 [https://github.com/python/cpython/issues/151912]: Corrigida uma falha em `type()` ao selecionar uma metaclasse cujo slot `tp_new` é `NULL`. Tais metaclasses agora são rejeitadas com `TypeError` em vez de causar uma desreferência de ponteiro NULL.
- gh-151773 [https://github.com/python/cpython/issues/151773]: Corrige uma falha em `contextvars.ContextVar.set()` quando a alocação de memória falha.
- gh-151672 [https://github.com/python/cpython/issues/151672]: Corrige uma inconsistência onde chamar `__lazy_import__` com uma string `fromlist` retornava um `types.LazyImportType` que se resolve para o membro nomeado, em vez do módulo sendo importado.

- gh-151619 [https://github.com/python/cpython/issues/151619]: Corrige um problema onde o uso de espaços de nomes globais ou embutidos não de módulo (como dicionários passados para `exec()`) podia fazer com que cargas globais em cache produzissem importações preguiçosas não resolvidas.
- gh-151126 [https://github.com/python/cpython/issues/151126]: Corrige uma falha ao compartilhar objetos `memoryview` entre interpretadores que falha devido à falta de memória. Agora levanta um `MemoryError` adequado.
- gh-151644 [https://github.com/python/cpython/issues/151644]: Corrige uma corrida de dados em `sys.setdlopenflags()` e `sys.getdlopenflags()` quando chamados concorrentemente na construção com threads livres. As funções subjacentes `_PyImport_GetDLOpenFlags` e `_PyImport_SetDLOpenFlags` agora usam operações atômicas de carga/armazenamento.
- gh-151126 [https://github.com/python/cpython/issues/151126]: Evita possível falha no `_winapi.c` onde um dispositivo não tem mais memória. Agora ele levanta adequadamente um `MemoryError`. Patch por Ivy Xu.
- gh-151029 [https://github.com/python/cpython/issues/151029]: No Linux, corrige `sys.remote_exec()` incapaz de encontrar memória gravável remota quando `libpython` foi substituída no disco.
- gh-150459 [https://github.com/python/cpython/issues/150459]: Corrige mensagem de erro de `SyntaxError` para `from x lazy import y`. Levanta `SyntaxWarning` em `from . lazy import x` (com espaço em branco entre os pontos e um módulo chamado `lazy`).
- gh-150858 [https://github.com/python/cpython/issues/150858]: Corrige uma corrida de dados ao alterar o `__qualname__` de um tipo concorrentemente em construções com threads livres.
- gh-144774 [https://github.com/python/cpython/issues/144774]: Corrige corrida de dados em `BaseException` quando uma exceção é copiada enquanto está sendo mutada.
- gh-150411 [https://github.com/python/cpython/issues/150411]: Corrige uma corrida de dados na construção com threads livres quando `gc.get_count()` lê a contagem de alocação da geração jovem enquanto outra thread a atualiza.
- gh-149689 [https://github.com/python/cpython/issues/149689]: Corrige a propagação de erro ausente em auxiliares de ação do analisador quando a alocação de memória falha. Patch por Thomas Kowalski.

- gh-149162 [https://github.com/python/cpython/issues/149162]: Corrige um potencial impasse (deadlock) em `PyUnicode_InternFromString()` e outras funções de estagioamento (interning) na construção com threads livres quando chamadas a partir de inicializadores locais estáticos do C++.
- gh-148825 [https://github.com/python/cpython/issues/148825]: Corrige erro de construção se a especialização estiver desativada.

Biblioteca

- gh-153695 [https://github.com/python/cpython/issues/153695]: Aplica hash a um `sqlite3.Row` que contém um valor não-hasheável agora levanta `TypeError` em vez de `SystemError`. Patch por tonghuaroot.
- gh-153658 [https://github.com/python/cpython/issues/153658]: Corrige `sqlite3.Connection.iterdump()` levantando `sqlite3.OperationalError` quando o nome de uma tabela contém uma aspa simples. Patch por tonghuaroot.
- gh-85943 [https://github.com/python/cpython/issues/85943]: Corrige funções do `struct` levantando `BytesWarning` sob a opção de linha de comando `-bb` quando um formato `str` é usado após um formato `bytes` igual (ou vice-versa). O cache de formato interno não mistura mais chaves de `str` e `bytes`.
- gh-151292 [https://github.com/python/cpython/issues/151292]: Armazena a contagem de amostras no formato binário de `profiling.sampling` como um inteiro de 64 bits, ampliando o inteiro de 32 bits usado anteriormente. Isso quebra as gravações existentes. Sessões de perfilamento longas ou com muitas threads não falharão mais por causa de `OverflowError`. Patch por Maurycy Pawłowski-Wieroński.
- gh-153417 [https://github.com/python/cpython/issues/153417]: Mensagens de erro de `imaplib.IMAP4.select()` e `imaplib.IMAP4.uid()` não levantam mais `BytesWarning` sob `-bb` quando o argumento de caixa de correio ou comando é `bytes`.
- gh-153406 [https://github.com/python/cpython/issues/153406]: `email.utils.parsedate_to_datetime()` agora levanta `ValueError` em vez de `OverflowError` quando o ano analisado ou deslocamento de fuso horário está fora do intervalo, correspondendo ao seu comportamento documentado.

- gh-153333 [https://github.com/python/cpython/issues/153333]: O método `readprofile` de `tkinter.Tk` agora lê os scripts de perfil do usuário usando a codificação declarada no arquivo, em vez da codificação da localidade.
- gh-153083 [https://github.com/python/cpython/issues/153083]: Adia o rastreamento pelo GC de um `array.array` para o final de sua construção. Patch por Donghee Na.
- gh-153292 [https://github.com/python/cpython/issues/153292]: Corrige corrida de dados no repr de `threading.RLock` na construção com threads livres.
- gh-153293 [https://github.com/python/cpython/issues/153293]: Corrige a TUI do perfilador de amostragem ao vivo mantendo estatísticas agregadas de opcodes obsoletas após uma redefinição de estatísticas.
- gh-143990 [https://github.com/python/cpython/issues/143990]: Um `tkinter.font.Font` criado a partir de uma fonte nomeada, inclusive por `copy()`, agora copia suas opções configuradas em vez das opções resolvidas pelo `font actual` do Tcl, preservando um tamanho especificado em pixels (um tamanho negativo).
- gh-148286 [https://github.com/python/cpython/issues/148286]: Corrige comportamento indefinido em `compression.zstd.ZstdDecompressor.unused_data` quando um quadro completo foi descompactado em uma única chamada.
- gh-153210 [https://github.com/python/cpython/issues/153210]: Corrige falha na importação de `array` sob pressão de memória.
- gh-153200 [https://github.com/python/cpython/issues/153200]: Corrige `math.isqrt()` retornando um resultado incorreto para argumentos não menores que 2**64 que são instâncias de uma subclasse de `int` com um operador de comparação substituído.
- gh-153068 [https://github.com/python/cpython/issues/153068]: Corrige `cProfile.Profile.enable()` para não mais sobrescrever erros do `sys.monitoring`.
- gh-153062 [https://github.com/python/cpython/issues/153062]: Corrige uma falha ao iterar concorrentemente um iterador `itertools.tee()` na construção com threads livres.
- gh-153056 [https://github.com/python/cpython/issues/153056]: Corrige `string.Template` levantando um `ValueError` espúrio quando o atributo *pattern* é um objeto de expressão regular compilado, o que a

documentação permite. Na construção com threads livres, isso também ocorria como uma corrida de dados no primeiro uso concorrente.

- gh-143921 [https://github.com/python/cpython/issues/143921]: Restringe a verificação de caracteres de controle nos comandos do `imaplib`: apenas NUL, CR e LF são rejeitados agora. Outros caracteres de controle são válidos em strings entre aspas e podem ocorrer em nomes de caixas de correio retornados pelo servidor, portanto, agora são aceitos e enviados entre aspas.
- gh-153037 [https://github.com/python/cpython/issues/153037]: Corrige `ZstdFile` levantando `AttributeError` em vez de `io.UnsupportedOperation` ao iterar sobre um arquivo que não está aberto para leitura.
- gh-135661 [https://github.com/python/cpython/issues/135661]: Corrige `html.parser.HTMLParser`: um comentário vazio fechado abruptamente (`<!-->` ou `<!--->`) não se estende mais até um `-->` posterior na mesma chamada `feed()`.
- gh-152851 [https://github.com/python/cpython/issues/152851]: Evita uma falha quando a alocação falha ao copiar um objeto `BLAKE-2s/2b`. Patch por Bénédikt Tran.
- gh-54930 [https://github.com/python/cpython/issues/54930]: Respostas de erro de `http.server.BaseHTTPRequestHandler` para linhas de requisição malformadas agora incluem uma linha de status e cabeçalhos em vez de serem enviadas no estilo HTTP/0.9 puro. Apenas uma requisição HTTP/0.9 válida (uma linha de requisição `GET` de duas palavras) agora recebe uma resposta no estilo HTTP/0.9.
- gh-119592 [https://github.com/python/cpython/issues/119592]: Corrige `concurrent.futures.ProcessPoolExecutor` abandonando o trabalho enviado para sempre quando um processo trabalhador saía ao atingir seu limite *max_tasks_per_child* após `shutdown()` ser chamado com `wait=False`: um trabalhador de substituição é agora gerado e o trabalho restante executado conforme documentado. Se o executor tiver sido coletado pelo GC sem `shutdown()` (gh-152967 [https://github.com/python/cpython/issues/152967]), ou um trabalhador de substituição não puder ser iniciado, os futures restantes agora falham com `BrokenProcessPool` em vez de nunca resolver. A saída de um trabalhador em corrida com `shutdown(wait=False)` também não pode mais fazer o thread de gerenciamento do executor falhar.

- gh-150579 [https://github.com/python/cpython/issues/150579]: O **`concurrent.futures`** agora usa importações preguiçosas para seus submódulos executores em vez de um hook `__getattr__` do módulo.
- gh-152951 [https://github.com/python/cpython/issues/152951]: **`collections.deque`** evita falha rara ao chamar `extend` sob condições de alta pressão de memória.
- gh-150880 [https://github.com/python/cpython/issues/150880]: Normaliza caminhos do Windows não estendidos antes de anexar o curinga usado por `os.listdir()` e `os.scandir()`, fazendo com que caminhos com espaços finais se comportem de forma consistente com outras APIs de sistema de arquivos.
- gh-152068 [https://github.com/python/cpython/issues/152068]: Corrige um erro quando uma linha era dividida (particularmente no Terminal.app do macOS) no meio de uma palavra reservada colorida, fazendo com que a sequência ANSI Color Reset (ESC0m) não fosse impressa corretamente, fazendo com que a saída ficasse colorida quando não deveria
- gh-152849 [https://github.com/python/cpython/issues/152849]: Marcas de data/hora do tipo float e inteiro fora do intervalo agora levantam **`OverflowError`** com a mesma mensagem. Patch por tonghuaroot.
- gh-152847 [https://github.com/python/cpython/issues/152847]: Rejeita uma regra de transição POSIX TZ com caracteres não-dígitos no campo dia do ano no analisador **`zoneinfo`** em Python puro. Patch por tonghuaroot.
- gh-152718 [https://github.com/python/cpython/issues/152718]: Corrige alocação de memória ilimitada no leitor de perfil binário do **`profiling.sampling`** quando um arquivo declara mais entradas de string ou quadro do que contém.
- gh-108280 [https://github.com/python/cpython/issues/108280]: Conectar o **`imaplib`** a um servidor que não envia uma saudação IMAP4 válida (por exemplo, um servidor POP3 respondendo na porta IMAP) agora levanta um erro relatando a resposta do servidor em vez de `imaplib.IMAP4.error: None`.
- gh-63121 [https://github.com/python/cpython/issues/63121]: O **`imaplib`** agora atualiza a lista de recursos em cache após um **`login()`** ou **`authenticate()`** bem-sucedido, usando a resposta `CAPABILITY` enviada pelo servidor ou, se nenhuma foi enviada, consultando-o, para que os recursos que ficam disponíveis apenas após a autenticação

(como `ENABLE` no Gmail) sejam reconhecidos. Recursos anunciados na saudação do servidor agora também são usados, evitando um comando `CAPABILITY` redundante.

- gh-88574 [https://github.com/python/cpython/issues/88574]: O **`imaplib`** não falha mais quando um servidor envia uma linha em branco espúria após os dados contados de um literal, inclusive após um literal que termina uma resposta (como um nome de caixa de correio retornado por `LIST`). Tais linhas em branco agora são puladas sem engolir a linha seguinte.
- gh-152502 [https://github.com/python/cpython/issues/152502]: Detecta a interface de mouse do **`curses`** (**`getmouse()`**, as constantes `BUTTON*` e outras) com uma sondagem de capacidade do configure ou macros de biblioteca em vez de restringi-la a macros específicas do ncurses. Agora também está disponível com outras implementações de curses que a fornecem, como o curses do NetBSD e PDCurses (este último oferece suporte ao `windows-curses`).
- gh-151842 [https://github.com/python/cpython/issues/151842]: Corrige uma falha em **`_interpreters.capture_exception()`** quando ocorre **`MemoryError`**. Patch por Amrutha Modela.
- gh-40038 [https://github.com/python/cpython/issues/40038]: O **`imaplib`** agora volta a colocar argumentos de comando entre aspas quando necessário, por exemplo, nomes de caixa de correio contendo um espaço. Tais aspas foram desativadas inadvertidamente quando o módulo foi portado para o Python 3, e os argumentos agora são colocados entre aspas de acordo com a gramática da **RFC 3501** [https://datatracker.ietf.org/doc/html/rfc3501.html]. Para retrocompatibilidade, um argumento já envolto em aspas duplas é deixado inalterado, para que o código que coloca argumentos entre aspas por conta própria continue funcionando.
- gh-50966 [https://github.com/python/cpython/issues/50966]: Corrige recursão ilimitada no **`turtle`** quando um manipulador de eventos de mouse que move a tartaruga é reentrado enquanto a tela está sendo redesenhada, por exemplo com `screen.ondrag(turtle.goto)`. Isso podia anteriormente fazer o interpretador falhar.
- gh-152569 [https://github.com/python/cpython/issues/152569]: Corrige o **`asyncio.wait()`** vazando tarefas em espera através do grafo de await (await-graph) ao disputar com um future que nunca se resolve. A

tarefa em espera agora é descartada do conjunto `awaited_by` de cada future uma vez que **`wait()`** retorna, mesmo para futures pendentes.

- gh-110357 [https://github.com/python/cpython/issues/110357]: Importar **`hashlib`** não registra mais um erro no stderr quando um algoritmo de hash normalmente garantido não está disponível no ambiente de execução atual (por exemplo, sob uma configuração OpenSSL FIPS ou uma compilação usando **`--without-builtin-hashlib-hashes`**). O código que realmente usa o algoritmo ausente ainda obtém um **`ValueError`** claro.
- gh-152356 [https://github.com/python/cpython/issues/152356]: Corrige um travamento em `profiling.sampling run --blocking` no Windows quando o processo de destino sai. O perfilador agora finaliza perfis binários em vez de continuar a amostrar o processo finalizado.
- gh-78335 [https://github.com/python/cpython/issues/78335]: Atualiza as docstrings das classes de widgets de **`tkinter`** e **`tkinter.ttk`** para listar todas as opções de widgets suportadas, incluindo opções adicionadas no Tk 9.0 e 9.1. `tkinter.Menubutton` e `tkinter.Message` anteriormente não tinham lista de opções.
- gh-151126 [https://github.com/python/cpython/issues/151126]: Corrige duas falhas na inicialização dos módulos **`tkinter`** e **`socket`** sob pressão de memória. Define o **`MemoryError`** ausente.
- gh-133031 [https://github.com/python/cpython/issues/133031]: O **`curses.textpad.Textbox`** agora insere e lê de volta os caracteres não-ASCII de uma codificação da localidade de 8 bits, em vez de corrompê-los com uma máscara de 7 bits.
- gh-71880 [https://github.com/python/cpython/issues/71880]: A classe **`curses.textpad.Textbox`** agora permite editar a célula do canto inferior direito da janela. Escrever nela usando **`addch()`** fazia o cursor ultrapassar o final da janela, gerando um erro e provocando a rolagem de uma janela rolável; por isso, a operação agora é realizada com **`insch()`**, que mantém o cursor na mesma posição.
- gh-83274 [https://github.com/python/cpython/issues/83274]: Desalocar uma aplicação **`tkinter`** de uma thread diferente daquela em que foi criada não faz mais o interpretador falhar. Em vez disso, o interpretador Tcl subjacente vaza e um **`RuntimeWarning`** é relatado.
- gh-152305 [https://github.com/python/cpython/issues/152305]: Corrige a implementação de **`datetime.time.strftime()`** em Python puro

levantando **`AttributeError`** para as diretivas de ano. Patch por tonghuaroot.

- gh-88758 [https://github.com/python/cpython/issues/88758]: **`tkinter.Misc.focus_get()`**, **`focus_displayof()`**, **`focus_lastfor()`** e **`winfo_containing()`** agora retornam `None` em vez de levantar **`KeyError`** quando o widget não foi criado pelo **`tkinter`** (por exemplo, um menu destacado).
- gh-38464 [https://github.com/python/cpython/issues/38464]: **`tkinter.Misc.nametowidget()`** agora resolve os nomes gerados automaticamente de menus clonados (um menu usado como barra de menu ou cascata) de volta ao widget original.
- gh-152248 [https://github.com/python/cpython/issues/152248]: Faz com que os analisadores C e em Python puro do **`zoneinfo`** validem abreviações POSIX TZ de forma consistente, rejeitando abreviações sem aspas com caracteres não alfabéticos e abreviações entre aspas vazias. Patch por tonghuaroot.
- gh-80937 [https://github.com/python/cpython/issues/80937]: Corrige um vazamento de memória no **`tkinter`** quando um comando Tcl criado com `createcommand` não era explicitamente removido antes que o interpretador fosse excluído. O comando não mantém mais o interpretador vivo através de um ciclo de referência.
- gh-152246 [https://github.com/python/cpython/issues/152246]: Corrige o analisador do **`zoneinfo`** em Python puro aceitando uma regra de transição POSIX TZ inválida com um separador que não é ponto. Patch por tonghuaroot.
- gh-151496 [https://github.com/python/cpython/issues/151496]: Corrigido `profiling.sampling --gecko` com `--async-aware` achatando as pilhas de tarefas assíncronas antes de gerar amostras do Gecko. `--binary` agora rejeita `--async-aware` até que o formato binário suporte dados de tarefas assíncronas.
- gh-139816 [https://github.com/python/cpython/issues/139816]: Corrige um travamento no **`tkinter`** no Python interativo compilado sem **`readline`**. Uma exceção levantada em um callback não faz mais o laço de eventos parar e esperar que o usuário pressione Enter; funções de retorno pendentes agora continuam rodando até que a entrada esteja realmente disponível no stdin.

- gh-139145 [https://github.com/python/cpython/issues/139145]: Corrige um laço ocupado no `tkinter` no Python interativo. Quando um comando Tcl executando seu próprio laço de eventos (como `vwait` ou `wait_variable()`) estava ativo e a entrada chegava no stdin, o laço de eventos continuava girando a 100% de CPU. O manipulador de arquivo stdin é agora removido assim que a entrada está disponível. Baseado em um patch por Michiel de Hoon.
- gh-152212 [https://github.com/python/cpython/issues/152212]: Corrige o analisador em Python puro do `zoneinfo` aceitando uma string POSIX TZ com uma abreviação `std`, mas sem deslocamento. Isso é inválido de acordo com o POSIX e agora levanta `ValueError`, correspondendo ao acelerador C. Patch por tonghuaroot.
- gh-152156 [https://github.com/python/cpython/issues/152156]: Corrige uma possível falha em `concurrent.interpreters.create()` sob condições de memória limitada.
- gh-152157 [https://github.com/python/cpython/issues/152157]: As implementações C de `fromisoformat()` e `fromisoformat()` agora rejeitam um separador decimal que não seja seguido por nenhum dígito fracionário antes de um designador de fuso horário.
- gh-151763 [https://github.com/python/cpython/issues/151763]: Corrige falha em `_interpqueues.create()` quando `MemoryError` acontece na criação da fila.
- gh-151669 [https://github.com/python/cpython/issues/151669]: No Windows, ao popular arquivos tar a partir do conteúdo do sistema de arquivos, para se adequar ao padrão do formato tar, contrabarras em alvos de links simbólicos são substituídas por barras simples.
- gh-105895 [https://github.com/python/cpython/issues/105895]: Adiciona `match` e `case` à lista de tópicos suportados por `help()`.
- gh-151378 [https://github.com/python/cpython/issues/151378]: Corrige um erro no coletor binário do Tachyon que causava crescimento ilimitado de memória ao perfilar uma thread que permanecia adormecida. Patch por Maurycy Pawłowski-Wieroński.
- gh-152079 [https://github.com/python/cpython/issues/152079]: Corrige `datetime.datetime.fromisoformat()` na implementação C descartando a parte de sub-segundos de um deslocamento UTC cuja parte inteira de segundos é zero, correspondendo à implementação em Python puro.

- gh-152052 [https://github.com/python/cpython/issues/152052]: O acelerador C do **`json`** agora relata corretamente uma string não terminada para um escape `\uXXXX` no final da entrada.
- gh-152060 [https://github.com/python/cpython/issues/152060]: Corrige **`datetime.datetime.fromisoformat()`** levantando **`AssertionError`** em vez de **`ValueError`** para algumas strings malformadas na implementação em Python puro, correspondendo à implementação C.
- gh-127802 [https://github.com/python/cpython/issues/127802]: Os métodos descontinuados **`tkinter.Variable`** **`trace_variable()`**, **`trace()`**, **`trace_vdelete()`** e **`trace_vinfo()`** estão agora programados para remoção no Python 3.17.
- gh-126219 [https://github.com/python/cpython/issues/126219]: Corrigida uma falha em **`tkinter.Tk`** quando *className* contém um caractere não-BMP e o tkinter é construído com Tcl/Tk 8.x. Tal nome é agora rejeitado com um **`ValueError`**.
- gh-86165 [https://github.com/python/cpython/issues/86165]: Corrige **`imaplib.Time2Internaldate()`** para usar o deslocamento do fuso horário local para valores de `time.struct_time` com `tm_gmtoff` definido como `None`, conforme retornado por `datetime.datetime.timetuple()`. Contribuição de Xiao Yuan.
- gh-151814 [https://github.com/python/cpython/issues/151814]: Corrige crescimento ilimitado de memória em **`io.TextIOWrapper`** ao escrever repetidamente uma string vazia.
- gh-151596 [https://github.com/python/cpython/issues/151596]: Adiciona o argumento posicional `size` ausente à implementação em Python puro de **`io.TextIOBase.readline()`**.
- gh-151640 [https://github.com/python/cpython/issues/151640]: Corrige uma corrida de dados em **`io.BytesIO`** em construções com threads livres quando leituras de buffer completo ou espiadas (peeks), ou **`getvalue()`**, compartilham o buffer interno com escritas concorrentes.
- gh-151613 [https://github.com/python/cpython/issues/151613]: Corrige outra maneira pela qual o cache de quadros do perfilador Tachyon podia produzir rastreamentos de pilha mistos impossíveis quando endereços de `_PyInterpreterFrame` são reutilizados, validando âncoras de quadro armazenadas em cache com um contador de sequência.
- gh-148660 [https://github.com/python/cpython/issues/148660]: Corrige uma falha em **`collections.OrderedDict.copy()`** quando o `__eq__` de uma

chave ou o método de uma subclasse altera o dict durante a cópia. Agora levanta **`RuntimeError`** em seu lugar, como faz a iteração.

- gh-151497 [https://github.com/python/cpython/issues/151497]: Abrir um arquivo **`tarfile`** não tenta mais pré-alocar um buffer enorme quando um membro manipulado ou truncado reinivindica um cabeçalho estendido sobredimensionado (um nome longo/link GNU ou um cabeçalho pax). O cabeçalho estendido é agora lido em blocos delimitados, de modo que seu campo de tamanho não pode mais disparar exaustão de memória.
- gh-151416 [https://github.com/python/cpython/issues/151416]: Corrige uma falha em **`os.spawnv()`** e **`os.spawnve()`** quando o método **`__fspath__()`** de um item de *argv* altera a lista *argv* durante a conversão de argumentos. Erros de conversão de argumento de **`os.spawnv()`** diferentes de **`TypeError`**, como o **`ValueError`** para um nulo embutido, não são mais substituídos por um **`TypeError`** genérico.
- gh-150994 [https://github.com/python/cpython/issues/150994]: Torna as anotações de tipo no módulo privado `_colorize` resolvíveis.
- gh-150583 [https://github.com/python/cpython/issues/150583]: Define corretamente o nível de compressão padrão em **`compression.zstd`** ao passar um dicionário digerido durante a compressão.
- gh-150641 [https://github.com/python/cpython/issues/150641]: Corrige erro onde **`typing.evaluate_forward_ref()`** com o formato `STRING` podia vazar nomes internos usados pelo mecanismo de anotações.
- gh-149816 [https://github.com/python/cpython/issues/149816]: Corrige uma potencial condição de uso-após-liberação em **`pickle.dumps()`** no modo threads livres ao serializar listas.
- gh-47005 [https://github.com/python/cpython/issues/47005]: Corrige **`urllib.request.AbstractHTTPHandler.do_open()`** para dar prioridade aos cabeçalhos regulares definidos via **`add_header()`** sobre cabeçalhos não redirecionados, consistente com **`get_header()`** e **`header_items()`**.
- gh-140729 [https://github.com/python/cpython/issues/140729]: Corrige um erro de serialização com pickle no módulo `cProfile` ao perfilar um script que usa **`multiprocessing.Process`** com os métodos de inicialização `spawn` e `forkserver`.
- gh-115634 [https://github.com/python/cpython/issues/115634]: Corrige um impasse (deadlock) em **`concurrent.futures.ProcessPoolExecutor`** ao usar

`max_tasks_per_child`, presente desde que o recurso foi introduzido no Python 3.11. O executor parava de agendar tarefas na fila depois que um processo trabalhador saía ao atingir seu limite de tarefas. Baseado em uma correção proposta por Tabrez Mohammed.

- gh-79638 [https://github.com/python/cpython/issues/79638]: Proíbe todo o acesso em **`urllib.robotparser`** se o arquivo `robots.txt` for inalcançável devido a erros de servidor ou rede.
- gh-105708 [https://github.com/python/cpython/issues/105708]: Aceita um prefixo V maiúsculo em endereços IPvFuture em **`urllib.parse.urlsplit()`**.

Testes

- gh-151626 [https://github.com/python/cpython/issues/151626]: Corrige vários testes em `test.test_inspect`, `test.test_import`, `test.test_importlib`, `test.test_py_compile` e `test.test_compileall` que falhavam quando a suíte de testes era executada com **`PYTHONPYCACHEPREFIX`** definido. Esses testes agora neutralizam o prefixo pycache onde eles assumem o layout de bytecode padrão de `__pycache__`.
- gh-151096 [https://github.com/python/cpython/issues/151096]: Corrige falha no `test_embed` quando o CPython é configurado com um prefixo de execução dividido (`--exec-prefix` diferente de `--prefix`).

Construção

- gh-126877 [https://github.com/python/cpython/issues/126877]: Corrige a verificação do **configure** para Tcl/Tk, que podia ser bem-sucedida incorretamente com compiladores otimizadores quando as bibliotecas estavam ausentes.
- gh-153438 [https://github.com/python/cpython/issues/153438]: Atualiza ferramentas e documentação de construção e instalador do Windows para usar a URL de download atual do `nuget.exe`.
- gh-152870 [https://github.com/python/cpython/issues/152870]: Corrige um erro de compilação na extensão C do **`decimal`** (`_decimal`) quando compilada com `EXTRA_FUNCTIONALITY`. `Context.apply()` chamava o auxiliar interno `_apply` usando sua assinatura pré-Argument-Clinic; a chamada agora é feita através da função `_impl` gerada.

- gh-152769 [https://github.com/python/cpython/issues/152769]: Ativa o trampolim do perfilador perf no Alpine Linux com a biblioteca C musl em `x86_64` e `aarch64`. O trampolim é específico da arquitetura e não depende da biblioteca C, portanto o mesmo trampolim em assembly usado para a glibc é reutilizado para a musl.
- gh-152502 [https://github.com/python/cpython/issues/152502]: O módulo **`curses`** agora detecta `set_escdelay()`, `set_tabsize()` e as variáveis `ESCDELAY` e `TABSIZE` com sondagens de capacidade do **configure** em vez da macro `NCURSES_EXT_FUNCS` específica do ncurses, portanto elas são expostas ao compilar contra outras implementações de curses que as fornecem, como o curses do NetBSD.

Windows

- gh-140146 [https://github.com/python/cpython/issues/140146]: Impede que o **`tkinter`** trave no Windows se o stdin for redirecionado para um encadeamento (pipe) em uma sessão interativa. Isso é útil para testar o uso interativo do tkinter a partir de um script, por exemplo como parte da suíte de testes do cpython.

macOS

- gh-124111 [https://github.com/python/cpython/issues/124111]: Atualiza o instalador do macOS para usar o Tcl/Tk 9.0.4.
- gh-152023 [https://github.com/python/cpython/issues/152023]: Atualiza as construções do instalador do macOS para SQLite 3.53.3. Ativa as funções median() e percentile().

IDLE

- gh-82183 [https://github.com/python/cpython/issues/82183]: Quando o shell está ocupado executando código, usar “Run… Customized” com “Restart shell” desmarcado agora relata que o shell está executando em vez de reiniciá-lo de qualquer maneira.
- gh-83653 [https://github.com/python/cpython/issues/83653]: Deixar em branco uma entrada inteira na caixa de diálogo Configurações do IDLE, como

“Auto squeeze min lines”, não salva mais uma string vazia como um valor de configuração inválido.

- gh-65339 [https://github.com/python/cpython/issues/65339]: Salvar o Shell do IDLE ou uma janela de saída agora usa como padrão a extensão `.txt` e lista arquivos de texto antes de arquivos Python, já que seu conteúdo não é código-fonte Python.
- gh-80504 [https://github.com/python/cpython/issues/80504]: O campo “In files:” da caixa de diálogo Find in Files do IDLE agora sempre contém um caminho de diretório completo, mesmo para um editor não salvo ou para o Shell. Isso mostra na saída do grep qual diretório foi pesquisado.
- gh-134300 [https://github.com/python/cpython/issues/134300]: Não adiciona o diretório `idlelib` ao caminho do processo de usuário do IDLE. O código do usuário executado no IDLE não pode mais importar submódulos do `idlelib` como módulos de nível superior, como `import help`.
- gh-89360 [https://github.com/python/cpython/issues/89360]: Corrige uma falha rara no editor IDLE quando a janela de autocompletar é fechada: excluir uma vinculação de chave para uma sequência que não está vinculada ao evento virtual é agora ignorado em vez de levantar um `ValueError`.
- gh-71956 [https://github.com/python/cpython/issues/71956]: Corrige o Replace All na caixa de diálogo Substituir do editor IDLE quando a direção de pesquisa é “Up” e “Wrap around” está desligado: agora substitui todas as correspondências acima da posição atual em vez de apenas a primeira.
- gh-152728 [https://github.com/python/cpython/issues/152728]: Move as funções run.fix_scaling, editor.fixwordbreaks (como fix_word_breaks) e pyshell.fix_x11_paste para idlelib.util.
- gh-66331 [https://github.com/python/cpython/issues/66331]: Define a propriedade de janela `WM_CLASS` das janelas do IDLE como `Idle` no X11, para que os gerenciadores de janelas as agrupem e e listem corretamente em vez de usar o `Toplevel` padrão.
- gh-85320 [https://github.com/python/cpython/issues/85320]: O IDLE agora lê e escreve seus arquivos de configuração e o arquivo de pontos de interrupção (breakpoints) usando UTF-8 em vez da codificação da localidade. Isso evita que dados não-ASCII (como caminhos não-

ASCII) sejam corrompidos e torna os arquivos portáveis entre ambientes.

API C

- gh-152132 [https://github.com/python/cpython/issues/152132]: Corrige `Py_RunMain()` para retornar um código de saída, em vez de chamar `Py_Exit()`, ao executar um script, um comando ou o REPL. Patch por Victor Stinner.
- gh-145633 [https://github.com/python/cpython/issues/145633]: As funções `PyFloat_Pack8()` e `PyFloat_Unpack*` não têm mais garantia de sempre ter sucesso no CPython.
- gh-153300 [https://github.com/python/cpython/issues/153300]: `PyConfig_Set()` agora também define variáveis de configuração globais. Por exemplo, `PyConfig_Set("inspect", value)` agora também define `Py_InspectFlag`. Patch por Victor Stinner.

Python 3.15.0 beta 3

Data de lançamento: 2026-06-23

Segurança

- gh-151544 [https://github.com/python/cpython/issues/151544]: `Modules/Setup.local` não é mais usado como um ponto de referência para descobrir se o Python está rodando em uma árvore de código-fonte, pois poderia afetar potencialmente instalações reais. O arquivo `pybuilddir.txt` é agora o único indicador de execução em uma árvore de código-fonte.
- gh-151159 [https://github.com/python/cpython/issues/151159]: Atualiza o instalador do macOS para usar OpenSSL 3.5.7.
- gh-151159 [https://github.com/python/cpython/issues/151159]: Atualiza os instaladores do Android e do iOS para usar OpenSSL 3.5.7.
- gh-150599 [https://github.com/python/cpython/issues/150599]: Corrige um possível estouro de buffer de pilha (stack buffer overflow) no `bz2` quando um `bz2.BZ2Decompressor` é reutilizado após um erro de descompressão. O

descompressor agora se torna inutilizável depois que a libbz2 relata um erro.

- gh-149835 [https://github.com/python/cpython/issues/149835]: **`shutil.move()`** agora resolve links simbólicos via **`os.path.realpath()`** ao verificar se o destino está dentro do diretório de origem, impedindo um desvio dessa proteção baseado em link simbólico.

Núcleo e embutidos

- gh-151905 [https://github.com/python/cpython/issues/151905]: Corrige o tratamento de erro de falta de memória (OOM) em **`PyFrame_GetBack()`** para propagar exceções em vez de mascará-las como None.

- gh-151722 [https://github.com/python/cpython/issues/151722]: Adia o rastreamento pelo GC de **`frozendict`** para o final da construção. Patch por Donghee Na.

- gh-151546 [https://github.com/python/cpython/issues/151546]: Corrige a verificação do limite da pilha se o Python estiver vinculado à musl (ex: Alpine Linux). Usa o tamanho da pilha definido pelo ligador (linker) para calcular os limites da pilha. Patch por Victor Stinner.

- gh-151510 [https://github.com/python/cpython/issues/151510]: Corrige uma falha em **`__lazy_import__()`** quando chamado sem um argumento `globals` explícito e sem um quadro Python atual.

- gh-151461 [https://github.com/python/cpython/issues/151461]: Corrige a execução direta de arquivos com codificações de origem inválidas para relatar a busca do codec subjacente ou erro de decodificação em vez da mensagem genérica `SyntaxError: encoding problem`. Patch por Bartosz Sławecki.

- gh-151218 [https://github.com/python/cpython/issues/151218]: **`PyConfig_Set()`** e **`sys.set_int_max_str_digits()`** agora substituem **`sys.flags`** (criam um novo objeto), em vez de modificar **`sys.flags`** no local (in-place). Patch por Victor Stinner.

- gh-151297 [https://github.com/python/cpython/issues/151297]: Corrige uma desreferência de ponteiro inválida que podia ocorrer ao chamar **`PyObject_Realloc()`** com um ponteiro NULL em construções com threads livres ou com **`PYTHONMALLOC`** definido como `mimalloc`.

- gh-151253 [https://github.com/python/cpython/issues/151253]: Se `import encodings` (primeira importação) falhar na inicialização do Python, descarrega a configuração do caminho do Python para ajudar os usuários a depurarem sua configuração. Patch por Victor Stinner.

- gh-151238 [https://github.com/python/cpython/issues/151238]: Corrige uma falha ao compilar uma f-string ou t-string concatenada se ocorrer um erro ao processar uma de suas partes.

- gh-151112 [https://github.com/python/cpython/issues/151112]: Corrige uma falha no compilador que podia ocorrer ao ficar sem memória.

- gh-151126 [https://github.com/python/cpython/issues/151126]: Corrige uma falha quando não há memória restante em um dispositivo, que acontecia em: compilação de código, módulo **`_interpchannels`**, função **`_winapi.CreateProcess()`**.

  Agora esses locais levantam erros **`MemoryError`** adequados.

- gh-150902 [https://github.com/python/cpython/issues/150902]: Aplica uma otimização existente de PyCriticalSection (mutex único) para PyCriticalSection2: evita adquirir as mesmas travas que a CS atual já adquiriu.

- gh-151065 [https://github.com/python/cpython/issues/151065]: Corrige vazamento de memória ao usar o alocador de memória mimalloc.

- gh-150988 [https://github.com/python/cpython/issues/150988]: Corrige um vazamento de referência em **`OSError`** quando atributos são definidos antes de `super().__init__()`.

- gh-150723 [https://github.com/python/cpython/issues/150723]: Corrige marcas de data/hora do perf jitdump no macOS. Os eventos eram marcados

usando `CLOCK_MONOTONIC`, mas os perfiladores do macOS marcam suas amostras com `mach_absolute_time()`. A incompatibilidade impedia que os mapeamentos de código JIT se alinhassem com as amostras, então nenhum quadro Python podia ser resolvido.

- gh-150723 [https://github.com/python/cpython/issues/150723]: Corrige IDs de thread malformados do perf jitdump no macOS. O campo `thread_id` do registro `JR_CODE_LOAD` era gravado como um valor de 64 bits em vez do valor de 32 bits exigido pelo formato jitdump, o que deslocava cada campo seguinte e impedia que os perfiladores resolvessem quadros Python.

- gh-150700 [https://github.com/python/cpython/issues/150700]: Corrige um **`SystemError`** ao compilar uma compreensão de escopo de classe contendo um `lambda` que referencia `__class__`, `__classdict__` ou `__conditional_annotations__`. Patch por Bartosz Sławecki.

- gh-150633 [https://github.com/python/cpython/issues/150633]: Corrige o importador congelado aceitando nomes de módulos com bytes nulos embutidos, o que fazia com que ele ignorasse o cache do **`sys.modules`** e criasse objetos de módulo duplicados.

- gh-148613 [https://github.com/python/cpython/issues/148613]: Corrige uma corrida de dados na compilação free-threaded entre **`gc.set_threshold()`** e o agendamento da coleta de lixo durante a alocação de objetos.

- gh-150207 [https://github.com/python/cpython/issues/150207]: Corrige uma falha quando a alocação de memória falha durante a inicialização do tokenizador. Uma exceção **`MemoryError`** adequado é agora levantada em seu lugar.

- gh-149805 [https://github.com/python/cpython/issues/149805]: Corrige uma exceção **`SystemError`** ao compilar uma anotação de classe `__classdict__`. Encontrado por OSS-Fuzz no #512907042 [https://issues.oss-fuzz.com/issues/512907042].

- gh-149321 [https://github.com/python/cpython/issues/149321]: Não oferece suporte a `none` como modo de importação preguiçosa.

Biblioteca

- gh-75666 [https://github.com/python/cpython/issues/75666]: Corrige um vazamento de referência no `tkinter`: os comandos Tcl criados para funções de retorno de eventos são agora excluídos quando uma ligação é substituída ou desvinculada.
- gh-151770 [https://github.com/python/cpython/issues/151770]: Corrige `datetime.datetime.fromisoformat()` levantando `AssertionError` em vez de `ValueError` para um mês fora do intervalo combinado com um horário `24:00`.
- gh-151665 [https://github.com/python/cpython/issues/151665]: `inspect.signature()` agora funciona em avaliadores preguiçosos de aliases de tipo e parâmetros de tipo em vez de levantar `ValueError`.
- gh-151695 [https://github.com/python/cpython/issues/151695]: Corrige um uso-após-liberação no módulo `curses`. A codificação da tela inicial, usada por `curses.unctrl()` e `curses.ungetch()` para codificar caracteres não-ASCII, é agora mantida como uma cópia privada em vez de um ponteiro emprestado para um objeto de janela que pode ser desalocado.
- gh-151436 [https://github.com/python/cpython/issues/151436]: Corrige rastreamentos de pilha distorcidos no perfilador Tachyon quando o armazenamento em cache está ativado e quando geradores e corrotinas são perfilados, atualizando `tstate->last_profiled_frame` em cada local de remoção de quadro. O problema resultava no apagamento total de alguns chamadores. Patch por Maurycy Pawłowski-Wieroński.
- gh-151426 [https://github.com/python/cpython/issues/151426]: Corrige rastreamentos de pilha impossíveis (chamadores e chamados cruzados, órfãos e linhas incorretas) no perfilador Tachyon ao armazenar quadros em cache, tirando um instantâneo (snapshot) dos blocos de pilha antes de percorrer a cadeia de quadros em uma falha de cache. Patch por Maurycy Pawłowski-Wieroński.
- gh-151403 [https://github.com/python/cpython/issues/151403]: Corrigida uma falha em `subprocess.Popen` (e `_posixsubprocess.fork_exec`) quando o `__fspath__()` de um item de `argv` altera concorrentemente a sequência `args` sendo convertida.
- gh-151390 [https://github.com/python/cpython/issues/151390]: Coloriza `match` no REPL quando seguido por um operador unário `+` ou `-`. Patch por Bartosz Sławecki.

- gh-151126 [https://github.com/python/cpython/issues/151126]: Evita falha por `MemoryError` não definido na falha de alocação em `ctypes.get_errno()`.
- gh-151337 [https://github.com/python/cpython/issues/151337]: Evita possível vazamento de memória em `tkinter.c` no Windows.
- gh-151126 [https://github.com/python/cpython/issues/151126]: Corrige uma falha quando o `MemoryError` em `os._path_splitroot()` não foi definido adequadamente.
- gh-149671 [https://github.com/python/cpython/issues/149671]: Restaura a compatibilidade com arquivos setuptools `-nspkg.pth` no módulo `site`. Injeta a variável `sitedir` no quadro que executa o código pth. Patch por Victor Stinner.
- gh-151295 [https://github.com/python/cpython/issues/151295]: Corrigida uma falha (uso-após-liberação) em `bytes.join()` e `bytearray.join()` que podia ocorrer se o `__buffer__()` de um item alterasse concorrentemente a sequência sendo unida. A alteração é agora relatada como um `RuntimeError`.
- gh-109940 [https://github.com/python/cpython/issues/109940]: Corrige a ativação do `venv` no Windows no `cmd.exe` para respeitar `VIRTUAL_ENV_DISABLE_PROMPT`.
- gh-150771 [https://github.com/python/cpython/issues/150771]: Corrige mensagens do `email` criadas com conjuntos de caracteres `shift_jis` ou `euc-jp`. `set_content()` agora armazena a carga útil usando o conjunto de caracteres de saída (`iso-2022-jp`) para que a impressão da mensagem não levante mais `UnicodeEncodeError`.
- gh-151039 [https://github.com/python/cpython/issues/151039]: Corrige uma falha quando tipos estáticos de `datetime` sobrevivem ao módulo `_datetime`.
- gh-151021 [https://github.com/python/cpython/issues/151021]: Corrige `mmap.mmap.find()` e `rfind()` para retornar `-1` ao pesquisar por uma subsequência vazia com uma posição inicial além do final do mapeamento.
- gh-62825 [https://github.com/python/cpython/issues/62825]: Codificações “KS_C_5601-1987”, “KS X 1001”, etc agora são aliases de “CP949” em vez de “EUC-KR”.
- gh-150913 [https://github.com/python/cpython/issues/150913]: Corrige a atribuição de fatia em `sqlite3.Blob` para levantar `TypeError` e `IndexError` para

inconsistências de tipo e tamanho, respectivamente, mesmo quando a fatia de destino está vazia.

- gh-143008 [https://github.com/python/cpython/issues/143008]: Corrige condições de corrida ao re-inicializar um objeto `io.TextIOWrapper`.
- gh-150662 [https://github.com/python/cpython/issues/150662]: Corrige o coletor `--gecko` em `profiling.sampling` que mantinha todas as amostras na memória. Agora ele escreve dados de amostra e marcador em arquivos temporários e os lê de volta, construindo o arquivo de saída no final. Patch por Pablo Galindo e Maurycy Pawłowski-Wieroński.
- gh-150750 [https://github.com/python/cpython/issues/150750]: Corrige uma condição de corrida em `collections.deque.index()` com threads livres.
- gh-148932 [https://github.com/python/cpython/issues/148932]: Corrige `profiling.sampling` em ambientes virtuais Windows para resolver o PID real do Python a partir de um shim de ambiente virtual.
- gh-149816 [https://github.com/python/cpython/issues/149816]: Corrige condição de corrida em `ssl.SSLContext.sni_callback`
- gh-53144 [https://github.com/python/cpython/issues/53144]: O pacote `email` agora oferece suporte a todos os apelidos de codecs do Python e usa nomes MIME/IANA para todos os conjuntos de caracteres registrados na IANA.
- gh-149891 [https://github.com/python/cpython/issues/149891]: Adiciona suporte para mais apelidos de codificação registrados oficialmente na IANA [https://www.iana.org/assignments/character-sets/character-sets.xhtml].
- gh-149473 [https://github.com/python/cpython/issues/149473]: Chamar `os.environ.clear()` agora emite o evento de auditoria `os._clearenv`. Patch por Victor Stinner.
- gh-148954 [https://github.com/python/cpython/issues/148954]: Corrige vulnerabilidade de injeção XML em `xmlrpc.client.dumps()` onde o `methodname` não estava sendo escapado antes da interpolação no corpo do XML.
- gh-143988 [https://github.com/python/cpython/issues/143988]: Corrigidas falhas em `socket.socket.sendmsg()` e `socket.socket.recvmsg_into()` que podiam ocorrer se sequências de buffer fossem alteradas concorrentemente.
- gh-120665 [https://github.com/python/cpython/issues/120665]: Corrigido um problema onde os carregadores do `unittest` carregavam e

instanciavam subclasses derivadas de `unittest.TestCase` que também são classes base abstratas, as quais não podem ser instanciadas.

- gh-91099 [https://github.com/python/cpython/issues/91099]: `imaplib.IMAP4.login()` agora levanta exceções com `str` em vez de `bytes`. Patch por Florian Best.
- gh-101267 [https://github.com/python/cpython/issues/101267]: Quando um processo trabalhador termina inesperadamente, `concurrent.futures.ProcessPoolExecutor` agora define uma exceção `BrokenProcessPool` separada em cada future pendente em vez de compartilhar uma única instância entre todos eles. Compartilhar uma exceção produzia rastreamentos de pilha malformados: cada chamada `Future.result()` re-levantava o mesmo objeto, anexando outra cópia do rastreamento a ele.

Documentação

- gh-86726 [https://github.com/python/cpython/issues/86726]: Expande grandemente a documentação do `tkinter` para cobrir toda a API pública do pacote e seus submódulos. As descrições são orientadas ao Python em vez do Tcl/Tk, com tipos de retorno corrigidos e informações de `versionadded`/`versionchanged`.
- gh-150319 [https://github.com/python/cpython/issues/150319]: Tipos genéricos embutidos e da biblioteca padrão agora documentam o significado de seus parâmetros de tipo.
- gh-109503 [https://github.com/python/cpython/issues/109503]: Corrige a documentação para `shutil.move()` no uso do `os.rename()`, já que a movimentação não-atômica pode ser usada mesmo se os arquivos estiverem no mesmo sistema de arquivos. Patch por Fang Li

Testes

- gh-151130 [https://github.com/python/cpython/issues/151130]: Adiciona mais testes para a API C `PyWeakref_*`.
- gh-150966 [https://github.com/python/cpython/issues/150966]: Evita terminar prematuramente alvos de teste do perfilador de amostragem ao vivo

com falha, o que tornava instáveis as asserções no stderr em buildbots ASAN.
- gh-148853 [https://github.com/python/cpython/issues/148853]: Corrige testes falhando no FreeBSD no in_systemd_nspawn_sync_suppressed() do test.support devido ao diretório /run ilegível.

Construção

- gh-151163 [https://github.com/python/cpython/issues/151163]: Atualiza construção do Android para incluir SQLite versão 3.53.2.

Windows

- gh-151163 [https://github.com/python/cpython/issues/151163]: Atualiza construções do Windows para incluir SQLite versão 3.53.2.
- gh-151159 [https://github.com/python/cpython/issues/151159]: Atualiza versão embutida do OpenSSL para 3.5.7.
- gh-150836 [https://github.com/python/cpython/issues/150836]: Faz o tkinter instalado funcionar com construções Tcl/Tk 9 que embutem a biblioteca de scripts Tk na DLL do Tk no Windows.

macOS

- gh-151163 [https://github.com/python/cpython/issues/151163]: Atualiza o instalador do macOS para incluir SQLite versão 3.53.2.

IDLE

- bpo-6699 [https://bugs.python.org/issue?@action=redirect&bpo=6699]: Avisa o usuário se um arquivo for sobrescrito ao salvar.

API C

- gh-141510 [https://github.com/python/cpython/issues/141510]: Adiciona `frozendict` aos caminhos rápidos de `PyMapping_GetOptionalItem()`, `PyMapping_Keys()`, `PyMapping_Values()` e `PyMapping_Items()`.

- gh-123619 [https://github.com/python/cpython/issues/123619]: **`PyUnstable_Object_EnableDeferredRefcount()`** agora retorna `0` se o objeto não for rastreado pelo coletor de lixo: se **`gc.is_tracked()`** for falso. Patch por Victor Stinner.
- gh-149044 [https://github.com/python/cpython/issues/149044]: Mensagem de erro melhorada ao especificar classes base que não são tipos em **`Py_tp_bases`**, **`Py_tp_base`** e no argumento *bases* para **`PyType_FromMetaclass()`** e outras funções `PyType_From*`.
- gh-150907 [https://github.com/python/cpython/issues/150907]: Corrige o arquivo de cabeçalho `dynamic_annotations.h` quando construído com C++ e Valgrind: adiciona `extern "C++" scope` para o template C++. Patch por Victor Stinner.

Python 3.15.0 beta 2

Data de lançamento: 2026-06-02

Segurança

- gh-149698 [https://github.com/python/cpython/issues/149698]: Atualiza o libexpat [https://libexpat.github.io/] empacotado para a versão 2.8.1 para a correção da **CVE 2026-45186** [https://www.cve.org/CVERecord?id=CVE-2026-45186].
- gh-87451 [https://github.com/python/cpython/issues/87451]: A função não documentada `ftpcp` do módulo **`ftplib`** não confia mais no valor do endereço IPv4 retornado do servidor de origem em resposta ao comando `PASV` por padrão, concluindo a correção para CVE-2021-4189. Como acontece com **`ftplib.FTP`**, o comportamento anterior pode ser reativado definindo o atributo `trust_server_pasv_ipv4_address` na instância **`ftplib.FTP`** de origem como `True`. Agradecimentos a Qi Deng na Aurascape AI pelo relato.
- gh-149474 [https://github.com/python/cpython/issues/149474]: Corrige o escritor binário em **`profiling.sampling`** não disparando a auditoria (**PEP 578** [https://peps.python.org/pep-0578/]) ao criar o arquivo de saída. O escritor e o leitor agora aceitam qualquer objeto caminho ou similar. Patch por Maurycy Pawłowski-Wieroński.

- gh-149486 [https://github.com/python/cpython/issues/149486]: `tarfile.data_filter()` agora valida alvos de link usando o mesmo valor normalizado que é gravado no disco, remove separadores finais do nome do membro ao resolver o diretório de um link simbólico e rejeita membros de link que substituiriam o próprio diretório de destino. Isso fecha vários desvios de travessia de caminho do filtro de extração `data`.
- gh-149079 [https://github.com/python/cpython/issues/149079]: Corrige uma potencial negação de serviço em `unicodedata.normalize()`. A etapa de ordenação canônica da normalização Unicode usava uma ordenação por inserção de tempo quadrático para reordenar caracteres combinantes, que poderia ser explorada com uma entrada manipulada contendo muitos caracteres combinantes em ordem não-canônica. Substituída por uma ordenação por contagem de tempo linear para sequências longas.
- gh-149018 [https://github.com/python/cpython/issues/149018]: Proteção melhorada contra ataques de inundação de hash XML (XML hash-flooding) em `xml.parsers.expat` e `xml.etree.ElementTree` quando o Python é compilado com libExpat 2.8.0 ou posterior.

Núcleo e embutidos

- gh-150374 [https://github.com/python/cpython/issues/150374]: Corrige a liberação dupla da trava de importação em erros de reificação de importação preguiçosa.

- gh-149156 [https://github.com/python/cpython/issues/149156]: Corrige uma falha intermitente após `os.fork()` quando o perfilamento de trampolim do perf está ativado e o filho retorna através de quadros de trampolim herdados do processo pai.

- gh-149449 [https://github.com/python/cpython/issues/149449]: Corrige uma falha de uso-após-liberação quando o módulo `unicodedata` era removido do `sys.modules` e coletado pelo GC entre chamadas que decodificam escapes `\N{...}` ou usam o tratador de erros do codec `namereplace`.

- gh-150107 [https://github.com/python/cpython/issues/150107]: **`asyncio`**: métodos de laço de eventos `sendfile()` e `sock_sendfile()` agora chamam `file.seek(offset)` se *file* tiver um método `seek()`, mesmo se *offset* for `0` (valor padrão).

- gh-150146 [https://github.com/python/cpython/issues/150146]: Corrige uma falha em uma substituição de variável de tipo complexa.

  `from typing import TypeVar; memoryview[TypeVar("")]` `[*typing.Mapping[..., ...]]` costumava falhar devido à verificação `NULL` ausente na chamada de função C `_unpack_args`.

- gh-148587 [https://github.com/python/cpython/issues/148587]: **`sys.lazy_modules`** é agora um conjunto em vez de um dicionário, como originalmente especificado no PEP 810.

- gh-150042 [https://github.com/python/cpython/issues/150042]: Corrige vazamento de referência em queue.SimpleQueue.put se a alocação de memória falhar.

- gh-149590 [https://github.com/python/cpython/issues/149590]: Corrige falha quando o faulthandler é importado mais de uma vez.

- gh-149816 [https://github.com/python/cpython/issues/149816]: Corrige uma condição de corrida em `_PyBytes_FromList` no modo threads livres.

- gh-149816 [https://github.com/python/cpython/issues/149816]: Corrige uma condição de corrida no **`memoryview`** com free-threading.

- gh-149807 [https://github.com/python/cpython/issues/149807]: Corrige `hash(frozendict)`: calcula o hash de cada par `(chave, valor)` corretamente. Patch por Victor Stinner.

- gh-149738 [https://github.com/python/cpython/issues/149738]: **`sqlite3`**: Proíbe a remoção dos atributos `row_factory` e `text_factory` de uma conexão para evitar uma falha em uma consulta.

- gh-139808 [https://github.com/python/cpython/issues/139808]: Adiciona proteções de ramificação para AArch64 (BTI/PAC) em código assembly usado por **`-X perf_jit`** (integração com o perfilador perf do Linux).

- gh-149676 [https://github.com/python/cpython/issues/149676]: Corrige o hash de `frozendict | frozendict`.

- gh-148829 [https://github.com/python/cpython/issues/148829]: Objetos **`sentinel`** agora oferecem suporte a um argumento `repr=` e seu atributo **`__module__`** é gravável.

- gh-149642 [https://github.com/python/cpython/issues/149642]: Permite importações dentro de chamadas `exec()` dentro de funções sob `PYTHON_LAZY_IMPORTS=all`.

- gh-144957 [https://github.com/python/cpython/issues/144957]: Corrige importações preguiçosas no formato `from` de atributos do módulo fornecidos por `__getattr__` em nível de módulo.

- gh-149459 [https://github.com/python/cpython/issues/149459]: Corrige uma falha no otimizador JIT quando uma proteção `LOAD_SPECIAL` especializada desotimizou após inserir a entrada sintética de pilha `NULL`.

- gh-148450 [https://github.com/python/cpython/issues/148450]: Corrige `abc.register()` para que ele invalide tags de versão de tipo para classes registradas.

Biblioteca

- gh-150685 [https://github.com/python/cpython/issues/150685]: Atualiza pip empacotado para 26.1.2
- gh-150228 [https://github.com/python/cpython/issues/150228]: A nova classe **`site.StartupState`** permite que os chamadores processem em lote arquivos de configuração de inicialização da **PEP 829** [https://peps.python.org/pep-0829/] em múltiplos diretórios de site antes que qualquer código de inicialização execute, com os métodos públicos **`addsitedir()`**, **`addusersitepackages()`**, **`addsitepackages()`** e **`process()`**. A assinatura de **`site.addsitedir()`** permanece inalterada

a partir do Python 3.14. O argumento **`defer_processing_start_files`** e a função `process_startup_files()` adicionados anteriormente no ciclo 3.15 foram removidos; use **`site.StartupState`** em seu lugar.

- gh-150406 [https://github.com/python/cpython/issues/150406]: Corrige uma possível falha ocorrendo durante a inicialização do módulo **`socket`** quando o sistema está sem memória em plataformas sem um `gethostbyname` reentrante.
- gh-150372 [https://github.com/python/cpython/issues/150372]: **`readline`**: Corrige uma potencial falha durante o autocompletar com Tab causada por um erro de memória esgotada durante a inicialização do módulo.
- gh-150157 [https://github.com/python/cpython/issues/150157]: Corrige uma falha em construções com threads livres que ocorre ao serializar por nome objetos sem um atributo `__module__` enquanto **`sys.modules`** está sendo modificado concorrentemente.
- gh-150175 [https://github.com/python/cpython/issues/150175]: Corrige condição de corrida em **`unittest.mock.ThreadingMock`** onde chamadas concorrentes podiam perder incrementos em `call_count` e outros atributos devido à falta de uma trava em `_increment_mock_call`.
- gh-84353 [https://github.com/python/cpython/issues/84353]: Preserva nomes de arquivos codificados em não-UTF-8 ao anexar a um **`zipfile.ZipFile`**. Anteriormente, nomes não-ASCII armazenados em uma codificação legada (sem o bit de flag UTF-8 definido) podiam ser corrompidos quando o diretório central era reescrito: eles eram decodificados como cp437 e então re-armazenados como UTF-8.
- gh-149189 [https://github.com/python/cpython/issues/149189]: Reverte as alterações nos padrões do **`pprint`**. Patch por Hugo van Kemenade.
- gh-149995 [https://github.com/python/cpython/issues/149995]: Atualiza várias docstrings no **`typing`**.
- gh-88726 [https://github.com/python/cpython/issues/88726]: O pacote **`email`** agora usa nomes de conjuntos de caracteres MIME padrão “gb2312” e “big5” em vez de nomes não padrão “eucgb2312_cn” e “big5_tw”.
- gh-149571 [https://github.com/python/cpython/issues/149571]: Corrige a implementação C de **`xml.etree.ElementTree.Element.itertext()`**: ela não emite mais texto para comentários e instruções de processamento.

- gh-149921 [https://github.com/python/cpython/issues/149921]: Corrige vazamentos de referência em caminhos de erro dos módulos de extensão **`_interpchannels`** e **`_interpqueues`**.
- gh-142349 [https://github.com/python/cpython/issues/142349]: Adiciona **`lazy`** à lista de tópicos suportados por **`help()`**.
- gh-149819 [https://github.com/python/cpython/issues/149819]: Corrige regressão em **`site.addsitedir()`** onde arquivos `.pth` não eram mais processados em subprocessos Python. Isso aconteceu porque **`site.main()`** semeou `known_paths` com entradas herdadas do processo pai, fazendo com que o `addsitedir` pulasse o processamento do `.pth`.
- gh-149816 [https://github.com/python/cpython/issues/149816]: Corrige uma condição de corrida no método `_random.Random.__init__` no modo threads livres.
- gh-149801 [https://github.com/python/cpython/issues/149801]: Adiciona nomes registrados na IANA e aliases com zeros à esquerda antes do número (como IBM00858, CP00858, IBM01140, CP01140) para os codecs correspondentes.
- gh-149718 [https://github.com/python/cpython/issues/149718]: Agrupa quadros de pilha idênticos consecutivos no Tachyon, de modo que os coletor de agregação (pstats, collapsed, flamegraph, gecko) recebam uma única coleta. Melhora a taxa de amostragem em 3x, a taxa de erros e a taxa de falhas caem 70%. Patch por Maurycy Pawłowski-Wieroński.
- gh-149701 [https://github.com/python/cpython/issues/149701]: Corrige código de retorno ruim do Lib/venv/bin/activate se o hashing estiver desativado
- gh-149504 [https://github.com/python/cpython/issues/149504]: Corrige **`site.addsitedir()`** para permitir chamadas reentrantes de dentro de arquivos de inicialização. Anteriormente, um arquivo `.pth` contendo uma linha `import` que chamava **`site.addsitedir()`** (ou um ponto de entrada `.start` fazendo o mesmo) podia falhar com `RuntimeError: dictionary changed size during iteration` durante a inicialização do site, quebrando ferramentas como `uv run --with`.
- gh-149584 [https://github.com/python/cpython/issues/149584]: Corrige sobretaxa excessiva no perfilador Tachyon ao inspecionar um processo remoto, evitando varreduras repetidas do cache de página remoto, agrupando leituras remotas previstas e reutilizando objetos de resultado do perfilador armazenados em cache. Patch por Pablo Galindo e Maurycy Pawłowski-Wieroński.

- gh-139489 [https://github.com/python/cpython/issues/139489]: Adiciona `xml.is_valid_text()` ao `xml.__all__`.
- gh-149614 [https://github.com/python/cpython/issues/149614]: Corrige uma regressão que quebrou a capacidade de realizar cópia profunda (deepcopy) em instâncias de `argparse.ArgumentParser`.
- gh-112821 [https://github.com/python/cpython/issues/112821]: No REPL, o autocompletar podia executar código arbitrário no getter de um descritor. Se esse getter levantasse uma exceção, o autocompletar falharia em apresentar qualquer opção para todo o objeto. O autocompletar agora funciona como esperado para esses objetos.
- gh-149534 [https://github.com/python/cpython/issues/149534]: Corrige a fusão de `collections.defaultdict` e `frozendict`.
- gh-149388 [https://github.com/python/cpython/issues/149388]: Torna idempotente o fechamento de **`asyncio.windows_utils.PipeHandle`**.
- gh-149489 [https://github.com/python/cpython/issues/149489]: Corrige a serialização do `ElementTree` para HTML. O conteúdo de comentários, instruções de processamento e elementos “xmp”, “iframe”, “noembed”, “noframes” e “plaintext” não é mais escapado. O elemento “plaintext” não tem mais a tag de fechamento. Adiciona suporte a atributos vazios (com valor `None`).
- gh-149056 [https://github.com/python/cpython/issues/149056]: Corrige `json.load()` não repassando o argumento *array_hook* para `json.loads()`. Patch por Thomas Kowalski.
- gh-149046 [https://github.com/python/cpython/issues/149046]: `io`: Corrige a serialização de `io.StringIO`: não chama mais `str(obj)` em subclasses de `str`. Patch por Thomas Kowalski.
- gh-148441 [https://github.com/python/cpython/issues/148441]: `xml.parsers.expat`: evita uma falha em `CharacterDataHandler()` quando o tamanho dos dados de caracteres excede o `tamanho_do_buffer` do analisador.
- gh-146452 [https://github.com/python/cpython/issues/146452]: Corrige falha de segmentação (segfault) no `pickle` ao serializar um dicionário alterado concorrentemente por outra thread na construção com threads livres.
- gh-86533 [https://github.com/python/cpython/issues/86533]: A função `os.makedirs()` e o método `pathlib.Path.mkdir()` agora têm um parâmetro *parent_mode* para especificar o modo dos diretórios

intermediários ao criar diretórios pais. Isso permite corresponder ao comportamento do Python 3.6 e anterior para **`os.makedirs()`**.

- gh-134261 [https://github.com/python/cpython/issues/134261]: zip: Em construções reproduzíveis, o ZipFile usa UTC em vez da hora local ao escrever marcas de data/hora nos arquivos para evitar estouro negativo (underflow).
- gh-133998 [https://github.com/python/cpython/issues/133998]: Corrige a exceção **`struct.error`** ao criar um arquivo com **`gzip.GzipFile`** ou comprimir dados com **`gzip.compress()`** se o horário do sistema estiver fora do intervalo de 00:00:00 UTC de 1º de janeiro de 1970 a 06:28:15 UTC de 7 de fevereiro de 2106, ou se o argumento *mtime* passado explicitamente estiver fora do intervalo de `0` a `2**32-1`.
- gh-128110 [https://github.com/python/cpython/issues/128110]: Corrige erro na análise de cabeçalhos de endereço do **`email`** que podia resultar em espaços irrelevantes no texto decodificado ao usar uma política de e-mail moderna. Espaços entre pares de palavras codificadas adjacentes da **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] agora são ignorados, conforme a seção 6.2 (e consistente com a análise existente de cabeçalhos não estruturados como *Subject*).
- gh-107398 [https://github.com/python/cpython/issues/107398]: Corrige exceção no modo de fluxo do **`tarfile`** ao processar o arquivo com o campo extra do gzip.
- gh-121109 [https://github.com/python/cpython/issues/121109]: Corrige problema de desempenho do **`tarfile`** ao ler arquivos no modo de fluxo (ex: `r|*`).
- bpo-45509 [https://bugs.python.org/issue?@action=redirect&bpo=45509]: Os cabeçalhos do Gzip agora são verificados para campos NAME, COMMENT e HCRC corrompidos.

Testes

- gh-150387 [https://github.com/python/cpython/issues/150387]: Corrige travamento em `test.test_profiling.test_sampling_profiler.test_live_collector_ui.TestLiveModeErrors.test_run_failed_script_live` em buildbots lentos. O teste agora sempre enfileira uma tecla `q` final para

que o loop da TUI ao vivo saia mesmo quando o perfilador coleta amostras suficientes para entrar no laço de entrada pós-finalizado.
- gh-149776 [https://github.com/python/cpython/issues/149776]: Corrige test_socket no kernel do Linux 7.1 e mais recente: pula testes de UDP Lite se não for suportado. Patch por Victor Stinner.

Construção

- gh-148294 [https://github.com/python/cpython/issues/148294]: Corrigido o uso de `AC_PATH_TOOL` em `configure.ac` para permitir que um compilador C++ seja encontrado no **`PATH`**.
- gh-148260 [https://github.com/python/cpython/issues/148260]: No Linux, quando o Python está vinculado à biblioteca C musl, usa um tamanho de pilha de thread de pelo menos 1 MiB em vez do padrão da musl, que é 128 kiB. Patch por Victor Stinner.

Windows

- gh-149786 [https://github.com/python/cpython/issues/149786]: Corrige inicializadores de ambiente virtual em construções com threads livres do Windows.
- gh-124111 [https://github.com/python/cpython/issues/124111]: Atualiza construções do Windows para usar Tcl/Tk 9.0.3.
- gh-138489 [https://github.com/python/cpython/issues/138489]: As distribuições do Windows agora incluem um arquivo `build-details.json` (veja **PEP 739** [https://peps.python.org/pep-0739/]). O instalador legado não o instala, mas todas as outras distribuições do python.org e todas as configurações predefinidas no script `PC\layout` incluirão um.
- gh-149029 [https://github.com/python/cpython/issues/149029]: Atualiza o instalador do Windows para ser fornecido com o SQLite 3.53.1.

macOS

- gh-150644 [https://github.com/python/cpython/issues/150644]: Quando o registro do sistema (system logging) está ativado (com `config.use_system_logger`), as mensagens agora são marcadas como

públicas. Isso permite que o registrador do sistema do macOS 26 veja as mensagens sem configuração especial.
- gh-149029 [https://github.com/python/cpython/issues/149029]: Atualização do instalador do macOS para ser fornecido com o SQLite versão 3.53.1.

Ferramentas/Demos

- gh-150258 [https://github.com/python/cpython/issues/150258]: Atualiza a dica de ferramenta no gráfico de chama do Tachyon para mostrar porcentagens absolutas e relativas.

API C

- gh-149725 [https://github.com/python/cpython/issues/149725]: Adiciona `PySentinel_CheckExact()` para testes exatos de tipo de `sentinel` para acompanhar o `PySentinel_Check()` existente.
- gh-145235 [https://github.com/python/cpython/issues/145235]: Tornou `PyDict_AddWatcher()`, `PyDict_ClearWatcher()`, `PyDict_Watch()` e `PyDict_Unwatch()` seguros para threads na compilação threads livres.

Python 3.15.0 beta 1

Release date: 2026-05-07

Segurança

- gh-149254 [https://github.com/python/cpython/issues/149254]: Update Android and iOS installer to use OpenSSL 3.5.6.
- gh-149017 [https://github.com/python/cpython/issues/149017]: Update bundled libexpat [https://libexpat.github.io/] to version 2.8.0.
- gh-148252 [https://github.com/python/cpython/issues/148252]: Fixed string table and sample record bounds checks in `_remote_debugging` when decoding certain `.pyb` inputs on 32-bit builds. Patch by Maurycy Pawłowski-Wieroński.
- gh-90309 [https://github.com/python/cpython/issues/90309]: Base64-encode values when embedding cookies to JavaScript using the

`http.cookies.BaseCookie.js_output()` method to avoid injection and escaping.

- gh-148808 [https://github.com/python/cpython/issues/148808]: Added buffer boundary check when using `nbytes` parameter with **`asyncio.AbstractEventLoop.sock_recvfrom_into()`**. Only relevant for Windows and the `asyncio.ProactorEventLoop`.
- gh-148395 [https://github.com/python/cpython/issues/148395]: Fix a dangling input pointer in `lzma.LZMADecompressor`, `bz2.BZ2Decompressor`, and internal **`zlib._ZlibDecompressor`** when memory allocation fails with `MemoryError`, which could let a subsequent **`decompress()`** call read or write through a stale pointer to the already-released caller buffer.
- gh-148252 [https://github.com/python/cpython/issues/148252]: Fixed stack depth calculation in **`_remote_debugging`** when decoding certain `.pyb` inputs on 32-bit builds. Issue originally identified and diagnosed by Tristan Madani (@TristanInSec on GitHub).
- gh-148178 [https://github.com/python/cpython/issues/148178]: Hardened **`_remote_debugging`** by validating remote debug offset tables before using them to size memory reads or interpret remote layouts.
- gh-148169 [https://github.com/python/cpython/issues/148169]: A bypass in `webbrowser` allowed URLs prefixed with `%action` to pass the dash-prefix safety check.
- gh-146581 [https://github.com/python/cpython/issues/146581]: Fix vulnerability in `shutil.unpack_archive()` for ZIP files on Windows which allowed to write files outside of the destination tree if the patch in the archive contains a Windows drive prefix. Now such invalid paths will be skipped. Files containing “..” in the name (like “foo..bar”) are no longer skipped.
- gh-137586 [https://github.com/python/cpython/issues/137586]: Fix a PATH-injection vulnerability in `webbrowser` on macOS where `osascript` was invoked without an absolute path. The new **`MacOS`** class uses `/usr/bin/open` directly, eliminating the dependency on `osascript` entirely.
- gh-146333 [https://github.com/python/cpython/issues/146333]: Fix quadratic backtracking in `configparser.RawConfigParser` option parsing regexes (`OPTCRE` and `OPTCRE_NV`). A crafted configuration line with many whitespace characters could cause excessive CPU usage.

- gh-146211 [https://github.com/python/cpython/issues/146211]: Reject CR/LF characters in tunnel request headers for the HTTPConnection.set_tunnel() method.

Núcleo e embutidos

- gh-148940 [https://github.com/python/cpython/issues/148940]: Revert the process size based deferral of garbage collection (gh-133464 [https://github.com/python/cpython/issues/133464]). The performance issue this change resolves is also fixed by gh-142562 [https://github.com/python/cpython/issues/142562]. This approach has the problem that process size as seen by the OS (e.g. the resident size or RSS) does not immediately decrease after cyclic garbage is freed since mimalloc defers returning memory of the OS. This change applies to the free-threaded GC only.

- gh-149243 [https://github.com/python/cpython/issues/149243]: Check for recursion limits in `CALL_ALLOC_AND_ENTER_INIT` opcode.

- gh-126910 [https://github.com/python/cpython/issues/126910]: Add support for unwinding JIT frames using GNU backtrace. Patch by Diego Russo and Pablo Galindo

- gh-149171 [https://github.com/python/cpython/issues/149171]: Allow assignment to the `__module__` attribute of **`typing.TypeAliasType`** instances.

- gh-149122 [https://github.com/python/cpython/issues/149122]: Fix a crash in optimized calls to **`all()`**, **`any()`**, **`tuple()`**, **`list()`**, and **`set()`** with an async generator expression argument (for example, `tuple(await x for x in y)`). These calls now correctly raise `TypeError` instead of crashing.

- gh-149049 [https://github.com/python/cpython/issues/149049]: Fix stack underflow for `BINARY_OP` in tier 2.

- gh-83065 [https://github.com/python/cpython/issues/83065]: Fix a deadlock that could occur when one thread is importing a submodule (for example `import pkg.sub.mod`) while another thread is importing one of its

parent packages (for example `import pkg.sub`) and that parent's `__init__.py` itself imports the submodule. The import system now acquires module locks in hierarchical (parent-before-child) order so the two threads serialise instead of raising `_DeadlockError`.

- gh-113956 [https://github.com/python/cpython/issues/113956]: Fix a data race in **`sys.intern()`** in the free-threaded build when interning a string owned by another thread. An interned copy owned by the current thread is used instead when it is not safe to immortalize the original.

- gh-148850 [https://github.com/python/cpython/issues/148850]: Fix the memory sanitizer false positive in **`os.getrandom()`**.

- gh-148820 [https://github.com/python/cpython/issues/148820]: Fix a race in **`_PyRawMutex`** on the free-threaded build where a `Py_PARK_INTR` return from `_PySemaphore_Wait` could let the waiter destroy its semaphore before the unlocking thread's `_PySemaphore_Wakeup` completed, causing a fatal `ReleaseSemaphore` error.

- gh-148829 [https://github.com/python/cpython/issues/148829]: Add **`sentinel`**, implementing **PEP 661** [https://peps.python.org/pep-0661/]. PEP by Tal Einat; patch by Jelle Zijlstra.

- gh-146270 [https://github.com/python/cpython/issues/146270]: Fix a sequential consistency bug in `structmember.c`.

- gh-148766 [https://github.com/python/cpython/issues/148766]: The interpreter help (such as `python --help`) is now in color. Patch by Hugo van Kemenade.

- gh-148571 [https://github.com/python/cpython/issues/148571]: Fix a crash in the JIT optimizer when specialized opcode families inherited incompatible recorded operand layouts.

- gh-148653 [https://github.com/python/cpython/issues/148653]: Forbid **`marshalling`** recursive code objects, **`slice`** and **`frozendict`** objects which cannot be correctly unmarshalled.

- gh-142516 [https://github.com/python/cpython/issues/142516]: Forward-port the generational cycle garbage collector to the default 3.15 build, replacing the incremental collector while leaving the free-threaded collector unchanged.

- gh-146462 [https://github.com/python/cpython/issues/146462]: Added `PyTypeObject.tp_basicsize`, `PyTypeObject.tp_dictoffset`, and `PyHeapTypeObject.ht_cached_keys` offsets to **`_Py_DebugOffsets`** to support version-independent read-only dict introspection tools.

- gh-145239 [https://github.com/python/cpython/issues/145239]: Unary plus is now accepted in **`match`** literal patterns, mirroring the existing support for unary minus. Patch by Bartosz Sławecki.

- gh-148515 [https://github.com/python/cpython/issues/148515]: Fix a bug in the JIT optimizer reading operands for uops with multiple caches.

- gh-148390 [https://github.com/python/cpython/issues/148390]: Fix an undefined behavior in **`memoryview`** when using the native boolean format (`?`) in **`cast()`**. Previously, on some common platforms, calling `memoryview(b).cast("?").tolist()` incorrectly returned `[False]` instead of `[True]` for any even byte *b*. Patch by Bénédikt Tran.

- gh-148418 [https://github.com/python/cpython/issues/148418]: Fix a possible reference leak in a corrupted `TYPE_CODE` marshal stream.

- gh-148393 [https://github.com/python/cpython/issues/148393]: Fix data races between **`PyDict_Watch()`** / **`PyDict_Unwatch()`** and concurrent dict mutation in the free-threaded build.

- gh-148398 [https://github.com/python/cpython/issues/148398]: Fix a bug in the JIT optimizer where class attribute loads were not invalidated after type mutation.

- gh-146527 [https://github.com/python/cpython/issues/146527]: Add a `GCMonitor` class with a `get_gc_stats` method to the **`_remote_debugging`** module to allow reading GC statistics from an external Python process without

requiring the full `RemoteUnwinder` functionality. Patch by Sergey Miryanov and Pablo Galindo.

- gh-148284 [https://github.com/python/cpython/issues/148284]: Fix high stack consumption in Python's interpreter loop on Clang 22 by setting function limits for inlining when building with computed gotos.

- gh-148037 [https://github.com/python/cpython/issues/148037]: Remove critical section from **`PyCode_Addr2Line()`** in free-threading.

- gh-115802 [https://github.com/python/cpython/issues/115802]: Improve JIT code generation on Linux AArch64 by reducing the indirect call to external symbols. Patch by Diego Russo.

- gh-148189 [https://github.com/python/cpython/issues/148189]: Repaired undercount of bytes in type-specific free lists reported by sys._debugmallocstats(). For types that participate in cyclic garbage collection, it was missing two pointers used by GC.

- gh-148222 [https://github.com/python/cpython/issues/148222]: Fix vectorcall support in **`types.GenericAlias`** when the underlying type does not support the vectorcall protocol. Fix possible leaks in **`types.GenericAlias`** and **`types.UnionType`** in case of memory error.

- gh-148208 [https://github.com/python/cpython/issues/148208]: Fix recursion depth leak in **`PyObject_Print()`**

- gh-95004 [https://github.com/python/cpython/issues/95004]: The specializing interpreter now specializes for **`enum.Enum`** improving performance and scaling in free-threading. Patch by Kumar Aditya.

- gh-149202 [https://github.com/python/cpython/issues/149202]: Enable frame pointers by default for GCC-compatible CPython builds, including `-mno-omit-leaf-frame-pointer`, `-marm` on 32-bit ARM, and/or `-mbackchain` on s390x platforms when the compiler supports them, so profilers and debuggers can unwind native interpreter frames more reliably. Users can pass **`--without-frame-pointers`** to `./configure` to opt out.

- gh-148014 [https://github.com/python/cpython/issues/148014]: Accept a function name in **`-X presite`** command line option and **`PYTHON_PRESITE`** environment variable. Patch by Victor Stinner.

- gh-147998 [https://github.com/python/cpython/issues/147998]: Fixed a memory leak in interpreter helper calls so cleanup works when an operation falls across interpreter boundaries. Patch by Maurycy Pawłowski-Wieroński.

- gh-146455 [https://github.com/python/cpython/issues/146455]: Fix $O(N^2)$ compile-time regression in constant folding after it was moved from AST to CFG optimizer.

- gh-146306 [https://github.com/python/cpython/issues/146306]: Specialize float true division in the tier 2 optimizer with inplace mutation for uniquely-referenced operands.

- gh-142186 [https://github.com/python/cpython/issues/142186]: Global **`sys.monitoring`** events can now be turned on and disabled on a per code object basis. Returning `DISABLE` from a callback disables the event for the entire code object (for the current tool).

- gh-126910 [https://github.com/python/cpython/issues/126910]: Add support for unwinding JIT frames using GDB. Patch by Diego Russo and Pablo Galindo.

- gh-146031 [https://github.com/python/cpython/issues/146031]: The unstable API _PyInterpreterState_SetEvalFrameFunc has a companion function _PyInterpreterState_SetEvalFrameAllowSpecialization to specify if specialization should be allowed. When this option is set to 1 the specializer will turn Python -> Python calls into specialized opcodes which the replacement interpreter loop can choose to respect and perform inlined dispatch.

- gh-145278 [https://github.com/python/cpython/issues/145278]: The **`encodings`** is now partially frozen, including the `aliases` and `utf_8` submodules.

  The **`linecache`** is now frozen.

- gh-134584 [https://github.com/python/cpython/issues/134584]: Optimize and eliminate redundant ref-counting for `MAKE_FUNCTION` in the JIT.

- gh-143886 [https://github.com/python/cpython/issues/143886]: Reorder function annotations so positional-only arguments are returned before other arguments. This fixes how **`functools.singledispatch()`** registers functions with positional-only arguments.

- gh-98894 [https://github.com/python/cpython/issues/98894]: Restore `function__entry` and `function__return` DTrace/SystemTap probes that were broken since Python 3.11.

- gh-116021 [https://github.com/python/cpython/issues/116021]: Support for creating instances of abstract AST nodes from the **`ast`** module is deprecated and scheduled for removal in Python 3.20. Patch by Brian Schubert.

- gh-137814 [https://github.com/python/cpython/issues/137814]: Fix the `__qualname__` attribute of `__annotate__` functions on functions.

- gh-137600 [https://github.com/python/cpython/issues/137600]: **`ast`**: The constructors of AST nodes now raise a **`TypeError`** when a required argument is omitted or when a keyword argument that does not map to a field on the AST node is passed. These cases had previously raised a **`DeprecationWarning`** since Python 3.13. Patch by Brian Schubert.

- gh-137293 [https://github.com/python/cpython/issues/137293]: Fix **`SystemError`** when searching ELF Files in **`sys.remote_exec()`**.

- gh-135357 [https://github.com/python/cpython/issues/135357]: Add support for **`socket.SO_PASSRIGHTS`** on Linux.

- gh-134690 [https://github.com/python/cpython/issues/134690]: Removed deprecated in **PEP 626** [https://peps.python.org/pep-0626/] since Python 3.12 **`codeobject.co_lnotab`** from **`types.CodeType`**.

- gh-100239 [https://github.com/python/cpython/issues/100239]: Specialize `BINARY_OP` for concatenation of lists and tuples, and propagate the

result type through `_BINARY_OP_EXTEND` in the tier 2 optimizer so that follow-up type guards can be eliminated.

Biblioteca

- gh-148823 [https://github.com/python/cpython/issues/148823]: Defer the import of `_colorize` in `argparse` until needed for coloring output.

- gh-141560 [https://github.com/python/cpython/issues/141560]: Add an *annotation_format* parameter to **`inspect.getfullargspec()`**.

- gh-139489 [https://github.com/python/cpython/issues/139489]: Add the **`xml.is_valid_text()`** function, which allows to check whether a string can be used in the XML document.

- gh-142389 [https://github.com/python/cpython/issues/142389]: Add backticks to stdlib argparse help to display in colour. Patch by Hugo van Kemenade.

- gh-149377 [https://github.com/python/cpython/issues/149377]: Update bundled pip to 26.1.1

- gh-142389 [https://github.com/python/cpython/issues/142389]: Add backtick markup support in **`argparse`** option help text to highlight inline code when color output is enabled. Patch by Hugo van Kemenade.

- gh-148675 [https://github.com/python/cpython/issues/148675]: Remove `F` and `D` formats from **`array`** and **`memoryview`**. Patch by Victor Stinner.

- gh-149342 [https://github.com/python/cpython/issues/149342]: Fix **`_remote_debugging`** binary writing so that sampling a thread whose Python frame stack is empty (for example while it is in a C call or mid-syscall) no longer raises `RuntimeError("Invalid stack encoding type")`, and so that `BinaryWriter.total_samples` after **`finalize()`** or context-manager exit includes samples flushed from the RLE buffer. Patch by Maurycy Pawłowski-Wieroński.

- gh-149010 [https://github.com/python/cpython/issues/149010]: The `inspect` module CLI now reports as much information as it has available for non-source modules when `--details` is specified, and provides an error message rather than a traceback when `--details` is omitted. It also reports improved information when the given target location is not the target's defining location and when the given target is a data value rather than a class or function definition.

- gh-146609 [https://github.com/python/cpython/issues/146609]: Use **`argparse`** for colour help **`timeit`** CLI. Patch by Hugo van Kemenade.

- gh-142389 [https://github.com/python/cpython/issues/142389]: Add backticks for colour to regrtest and pdb's help description. Patch by Hugo van Kemenade.

- gh-144384 [https://github.com/python/cpython/issues/144384]: Lazily import **`_colorize`**. Patch by Hugo van Kemenade.

- gh-149321 [https://github.com/python/cpython/issues/149321]: Fix import cycles exposed by running standard library modules with `-X lazy_imports=none`.

- gh-145378 [https://github.com/python/cpython/issues/145378]: Generate consistent colors for **`pdb`** commands in **`pdb`** REPL.

- gh-149296 [https://github.com/python/cpython/issues/149296]: Add a `dump` subcommand to **`profiling.sampling`** that prints a single traceback-style snapshot of a running process's Python stack, including per-thread status, source line highlighting, optional bytecode opcode names, and async-aware task reconstruction. Patch by Pablo Galindo.

- gh-143231 [https://github.com/python/cpython/issues/143231]: A *module* attribute has been added to **`warnings.WarningMessage`**.

- gh-148675 [https://github.com/python/cpython/issues/148675]: **`ctypes`**: Change the **`_type_`** of **`c_float_complex`**, **`c_double_complex`** and **`c_longdouble_complex`** from `F`, `D` and `G` to `Zf`, `Zd` and `Zg` for compatibility with numpy. Patch by Victor Stinner.

- gh-148675 [https://github.com/python/cpython/issues/148675]: The `array.typecodes` type changed from `str` to `tuple` to support type codes longer than 1 character (`Zf` and `Zd`). Patch by Victor Stinner.

- gh-149221 [https://github.com/python/cpython/issues/149221]: Catch rare math domain error for `random.binomialvariate()`.

- gh-149231 [https://github.com/python/cpython/issues/149231]: In `tomllib`, the number of parts in TOML keys is now limited.

- gh-143231 [https://github.com/python/cpython/issues/143231]: `unittest.TestCase.assertWarns()` and `unittest.TestCase.assertWarnsRegex()` no longer swallow warnings that do not match the specified category or regex. Nested context managers are now supported.

- gh-149214 [https://github.com/python/cpython/issues/149214]: Fix `_remote_debugging` misreading non-ASCII Unicode strings (Latin-1, BMP and non-BMP) from a remote process. Filenames and function names that contain non-ASCII characters are now reported correctly in stack traces, the sampling profiler, and `asyncio` task introspection.

- gh-149189 [https://github.com/python/cpython/issues/149189]: `pprint` now uses modern defaults: `indent=4` and `width=88`, and the default `compact=False` output is now formatted similar to pretty-printed `json.dumps()`, with opening parentheses and brackets followed by a newline and the contents indented by one level. The *expand* parameter, added in 3.15.0a8, has been removed; `compact=False` (the default) now produces the former `expand=True` layout. Patch by Hugo van Kemenade.

- gh-149173 [https://github.com/python/cpython/issues/149173]: Fix inverted `PYTHON_BASIC_REPL` environment check in `pdb._pyrepl_available`.

- gh-149117 [https://github.com/python/cpython/issues/149117]: Fix `runpy.run_module()` and `runpy.run_path()` to set the `name` attribute on the `ImportError` they raise.

- gh-149148 [https://github.com/python/cpython/issues/149148]: **`ensurepip`**: Upgrade bundled pip to 26.1. This version fixes the **CVE 2026-3219** [https://www.cve.org/CVERecord?id=CVE-2026-3219] vulnerability. Patch by Victor Stinner.

- gh-149009 [https://github.com/python/cpython/issues/149009]: Validate that **`profiling.sampling`** binary profiles do not contain more unique (thread, interpreter) pairs than declared in the header. Patch by Maurycy Pawłowski-Wieroński.

- gh-148292 [https://github.com/python/cpython/issues/148292]: **`ssl`**: Update **`ssl.SSLSocket`** and **`ssl.SSLObject`** for OpenSSL 4. The classes now remember if they get a **`ssl.SSLEOFError`**. In this case, following **`read()`**, **`sendfile()`**, **`write()`**, and **`do_handshake()`** calls raise **`ssl.SSLEOFError`** without calling the underlying OpenSSL function. Thanks to that, **`ssl.SSLSocket`** behaves the same on all OpenSSL versions on EOF. Patch by Victor Stinner.

- gh-149085 [https://github.com/python/cpython/issues/149085]: Add a *max_threads* keyword argument to **`faulthandler.dump_traceback()`**, **`faulthandler.dump_traceback_later()`**, **`faulthandler.enable()`**, and **`faulthandler.register()`**.

- gh-148641 [https://github.com/python/cpython/issues/148641]: **`pkgutil.resolve_name()`** gets a new optional, keyword-only argument called `strict`. The default is `False` for backward compatibility.

- gh-148093 [https://github.com/python/cpython/issues/148093]: Fix an out-of-bounds read of one byte in **`binascii.a2b_uu()`**. Raise **`binascii.Error`**, instead of reading past the buffer end.

- gh-149083 [https://github.com/python/cpython/issues/149083]: **`dataclasses.MISSING`** and **`dataclasses.KW_ONLY`** are now instances of **`sentinel`**.

- gh-148914 [https://github.com/python/cpython/issues/148914]: Fix memoization of in-band **`PickleBuffer`** in the Python implementation of **`pickle`**.

Previously, identical **`PickleBuffer`**s did not preserve identity, and empty writable **`PickleBuffer`** memoized an empty bytearray object in place of `b''`, so the following references to `b''` were unpickled as an empty bytearray object.

- gh-149026 [https://github.com/python/cpython/issues/149026]: Add colour to **`pickletools`** CLI output. Patch by Hugo van Kemenade.

- gh-148991 [https://github.com/python/cpython/issues/148991]: Add colour to **`tokenize`** CLI output. Patch by Hugo van Kemenade.

- gh-138907 [https://github.com/python/cpython/issues/138907]: Support **RFC 9309** [https://datatracker.ietf.org/doc/html/rfc9309.html] in **`urllib.robotparser`**.

- gh-148981 [https://github.com/python/cpython/issues/148981]: Add *color* parameter to **`ast.dump()`**.

- gh-148849 [https://github.com/python/cpython/issues/148849]: Deprecate **`http.cookies.Morsel.js_output()`** and **`http.cookies.BaseCookie.js_output()`**, which will be removed in Python 3.19. Use **`http.cookies.Morsel.output()`** or **`http.cookies.BaseCookie.output()`** instead.

- gh-146311 [https://github.com/python/cpython/issues/146311]: Add a *canonical* keyword-only parameter to the base32, base64, base85, ascii85, and Z85 decoders in **`base64`** and **`binascii`**. When true, encodings with non-zero padding bits (base32/64) or non-canonical encodings (base85/ascii85) are rejected. Single-character final groups in **`binascii.a2b_ascii85()`** and **`binascii.a2b_base85()`** are now always rejected as encoding violations, regardless of *canonical*; previously they were silently ignored and produced no output bytes.

- gh-148947 [https://github.com/python/cpython/issues/148947]: Fix crash in **`@dataclasses.dataclass`** with `slots=True` that occurred when a function found within the class had an empty `__class__` cell.

- gh-148680 [https://github.com/python/cpython/issues/148680]: `ForwardRef` objects that contain internal names to represent known objects now show the

`type_repr` of the known object rather than the internal `__annotationlib_name_x__` name when evaluated as strings.

- gh-124397 [https://github.com/python/cpython/issues/124397]: The threading module added tooling to support concurrent iterator access: **`threading.serialize_iterator`**, **`threading.synchronized_iterator()`**, and **`threading.concurrent_tee()`**.

- gh-148801 [https://github.com/python/cpython/issues/148801]: **`xml.etree.ElementTree`**: Fix a crash in **`Element.__deepcopy__`** on deeply nested trees.

- gh-148735 [https://github.com/python/cpython/issues/148735]: **`xml.etree.ElementTree`**: Fix a use-after-free in **`Element.findtext`** when the element tree is mutated concurrently during the search.

- gh-148740 [https://github.com/python/cpython/issues/148740]: Fix usage for **`uuid`** command-line interface to support a custom namespace be provided for uuid3 and uuid5.

- gh-148688 [https://github.com/python/cpython/issues/148688]: **`bz2`**, **`compression.zstd`**, **`lzma`**, **`zlib`**: Fix a double free on memory allocation failure. Patch by Victor Stinner.

- gh-148675 [https://github.com/python/cpython/issues/148675]: **`array`**, **`struct`**: Add support for `Zd` and `Zf` formats for double complex and float complex. Patch by Victor Stinner.

- gh-148651 [https://github.com/python/cpython/issues/148651]: Fix reference leak in **`compression.zstd.ZstdDecompressor`** when an invalid option key is passed.

- gh-148641 [https://github.com/python/cpython/issues/148641]: **PEP 829** [https://peps.python.org/pep-0829/] (package startup configuration files) implements a new format `<name>.start` parallel to `<name>.pth` files, to replace `import` lines in the latter.

- gh-148639 [https://github.com/python/cpython/issues/148639]: Implement **PEP 800** [https://peps.python.org/pep-0800/], adding the `@typing.disjoint_base` decorator. Patch by Jelle Zijlstra.

- gh-148615 [https://github.com/python/cpython/issues/148615]: Fix `pdb` to accept standard – end of options separator. Reported by haampie. Patched by Shrey Naithani.

- gh-146553 [https://github.com/python/cpython/issues/146553]: Fix infinite loop in `typing.get_type_hints()` when `__wrapped__` forms a cycle. Patch by Shamil Abdulaev.

- gh-148599 [https://github.com/python/cpython/issues/148599]: Update the `socket` module's WSA error messages to match official documentation.

- gh-148508 [https://github.com/python/cpython/issues/148508]: An intermittent timing error when running SSL tests on iOS has been resolved.

- gh-144881 [https://github.com/python/cpython/issues/144881]: `asyncio` debugging tools (`python -m asyncio ps` and `pstree`) now retry automatically on transient errors that can occur when attaching to a process under active thread delegation. The number of retries can be controlled with the `--retries` flag. Patch by Bartosz Sławecki.

- gh-148518 [https://github.com/python/cpython/issues/148518]: If an email containing an address header that ended in an open double quote was parsed with a non-`compat32` policy, accessing the `username` attribute of the mailbox accessed through that header object would result in an `IndexError`. It now correctly returns an empty string as the result.

- gh-148464 [https://github.com/python/cpython/issues/148464]: Add missing `__ctype_le/be__` attributes for `c_float_complex` and `c_double_complex`. Patch by Sergey B Kirpichev.

- gh-148370 [https://github.com/python/cpython/issues/148370]: `configparser`: prevent quadratic behavior when a `ParsingError` is raised after a parser fails to parse multiple lines. Patch by Bénédikt Tran.

- gh-121190 [https://github.com/python/cpython/issues/121190]: `importlib.resources.files()` now emits a more meaningful error message when module spec is None (as found in some `__main__` modules).

- gh-127012 [https://github.com/python/cpython/issues/127012]: `importlib.abc.Traversable.read_text` now allows/solicits an `errors` parameter.

- gh-137855 [https://github.com/python/cpython/issues/137855]: Improve import time of **`dataclasses`** module by lazy importing **`re`** and **`copy`** modules.

- gh-148352 [https://github.com/python/cpython/issues/148352]: Add more color to **`calendar`**'s CLI output. Patch by Hugo van Kemenade.

- gh-148254 [https://github.com/python/cpython/issues/148254]: Use singular "sec" instead of "secs" in **`timeit`** verbose output for consistency with other time units.

- gh-130472 [https://github.com/python/cpython/issues/130472]: Integrate fancycompleter with import completions.

- gh-148241 [https://github.com/python/cpython/issues/148241]: **`json`**: Fix serialization: no longer call `str(obj)` on **`str`** subclasses. Patch by Victor Stinner.

- gh-148225 [https://github.com/python/cpython/issues/148225]: The **`profiling.sampling`** `replay` command now rejects non-binary profile files with a clear error explaining that replay only accepts files created with `--binary`.

- gh-148192 [https://github.com/python/cpython/issues/148192]: `email.generator.Generator._make_boundary` could fail to detect a duplicate boundary string if linesep was not n. It now correctly detects boundary strings when linesep is rn as well.

- gh-148207 [https://github.com/python/cpython/issues/148207]: **`typing.TypeVarTuple`** now accepts `bound`, `covariant`,

contravariant, and infer_variance parameters, matching the interface of **typing.TypeVar** and **typing.ParamSpec**.

- gh-148100 [https://github.com/python/cpython/issues/148100]: Soft deprecate **re.match()** and **re.Pattern.match()** in favour of **re.prefixmatch()** and **re.Pattern.prefixmatch()**. Patch by Hugo van Kemenade.

- gh-147991 [https://github.com/python/cpython/issues/147991]: Improve **tomllib** import time (up to 10x faster). Patch by Victor Stinner.

- gh-147957 [https://github.com/python/cpython/issues/147957]: Guarantees that **collections.UserDict.popitem()** will pop in the same order as the wrapped dictionary rather than an arbitrary order.

- gh-146256 [https://github.com/python/cpython/issues/146256]: The profiling.sampling module now supports JSONL output format via --jsonl. Each run emits a newline-delimited JSON file that is sequentially parseable by external tools, scripts, and programmatic consumers. Patch by Maurycy Pawłowski-Wieroński.

- gh-146609 [https://github.com/python/cpython/issues/146609]: Add colour to **timeit** CLI output. Patch by Hugo van Kemenade.

- gh-146563 [https://github.com/python/cpython/issues/146563]: **xml.parsers.expat**: add an exception note when a custom Expat handler return value cannot be properly interpreted. Patch by Bénédikt Tran.

- gh-137586 [https://github.com/python/cpython/issues/137586]: Add **MacOS** to **webbrowser** for macOS, which opens URLs via /usr/bin/open instead of piping AppleScript to osascript. Deprecate **MacOSXOSAScript** in favour of **MacOS**.

- gh-146406 [https://github.com/python/cpython/issues/146406]: Cross-language method suggestions are now shown for **AttributeError** on builtin types and their subclasses. For example, [].push() suggests append, (1,2).append(3) suggests using a list, None.keys() suggests expecting a dict, and 1.0.__or__ suggests using an int.

- gh-146313 [https://github.com/python/cpython/issues/146313]: Fix a deadlock in **`multiprocessing`**'s resource tracker where the parent process could hang indefinitely in **`os.waitpid()`** during interpreter shutdown if a child created via **`os.fork()`** still held the resource tracker's pipe open.

- gh-146292 [https://github.com/python/cpython/issues/146292]: Add colour to **`BaseHTTPRequestHandler`** logs, as used by the **`http.server`** CLI. Patch by Hugo van Kemenade.

- gh-145917 [https://github.com/python/cpython/issues/145917]: Add MIME types for TTC and Haptics formats to **`mimetypes`**. (Contributed by Charlie Lin in gh-145918 [https://github.com/python/cpython/issues/145918].)

- gh-145846 [https://github.com/python/cpython/issues/145846]: Fix memory leak in `_lsprof` when `clear()` is called during active profiling with nested calls. `clearEntries()` now walks the entire `currentProfilerContext` linked list instead of only freeing the top context.

- gh-145831 [https://github.com/python/cpython/issues/145831]: Fix **`email.quoprimime.decode()`** leaving a stray `\r` when `eol='\r\n'` by stripping the full *eol* string instead of one character.

- gh-145378 [https://github.com/python/cpython/issues/145378]: Use `PyREPL` as the default input console for **`pdb`**

- gh-145244 [https://github.com/python/cpython/issues/145244]: Fixed a use-after-free in **`json`** encoder when a `default` callback mutates the dictionary being serialized.

- gh-117716 [https://github.com/python/cpython/issues/117716]: Fix **`wave`** writing of odd-sized `data` chunks by appending the required RIFF pad byte and correcting the RIFF chunk size field accordingly.

- gh-145200 [https://github.com/python/cpython/issues/145200]: **`hashlib`**: fix a memory leak when allocating or initializing an OpenSSL HMAC context fails.

- gh-145056 [https://github.com/python/cpython/issues/145056]: Add support for `frozendict` in `dataclasses.asdict()` and `dataclasses.astuple()`.

- gh-145105 [https://github.com/python/cpython/issues/145105]: Fix crash in `csv` reader when iterating with a re-entrant iterator that calls `next()` on the same reader from within `__next__`.

- gh-130750 [https://github.com/python/cpython/issues/130750]: Restore quoting of choices in `argparse` error messages for improved clarity and consistency with documentation.

- gh-137855 [https://github.com/python/cpython/issues/137855]: Reduce the import time of `dataclasses` module by ~20%.

- gh-70647 [https://github.com/python/cpython/issues/70647]: `strptime()` now raises `ValueError` when the format string contains `%d` without a year directive. Using `%e` without a year now emits a `DeprecationWarning`.

- gh-105936 [https://github.com/python/cpython/issues/105936]: Attempting to mutate non-field attributes of `dataclasses` with both *frozen* and *slots* being `True` now raises `FrozenInstanceError` instead of `TypeError`. Their non-dataclass subclasses can now freely mutate non-field attributes, and the original non-slotted class can be garbage collected.

- gh-142831 [https://github.com/python/cpython/issues/142831]: Fix a crash in the `json` module where a use-after-free could occur if the object being encoded is modified during serialization.

- gh-108411 [https://github.com/python/cpython/issues/108411]: `typing.IO` and `typing.BinaryIO` method arguments are now positional-only.

- gh-130273 [https://github.com/python/cpython/issues/130273]: Fix traceback color output with Unicode characters.

- gh-142307 [https://github.com/python/cpython/issues/142307]: `imaplib`: deprecate support for `IMAP4.file`. This attribute was never meant to be part of the public interface and altering its value may result in unclosed files

or other synchronization issues with the underlying socket. Patch by Bénédikt Tran.

- gh-141449 [https://github.com/python/cpython/issues/141449]: Improve tests and documentation for non-function callables as annotate functions.

- gh-140287 [https://github.com/python/cpython/issues/140287]: The `asyncio` REPL now handles exceptions when executing `PYTHONSTARTUP` scripts. Patch by Bartosz Sławecki.

- gh-139489 [https://github.com/python/cpython/issues/139489]: Add the `xml.is_valid_name()` function, which allows to check whether a string can be used as an element or attribute name in XML.

- gh-75707 [https://github.com/python/cpython/issues/75707]: Add optional `mtime` argument to `tarfile.open()`, for setting the `mtime` header field in `.tar.gz` archives.

- gh-125862 [https://github.com/python/cpython/issues/125862]: The `contextlib.contextmanager()` and `contextlib.asynccontextmanager()` decorators now work correctly with generators, coroutine functions, and async generators when the wrapped callables are used as decorators.

- gh-135528 [https://github.com/python/cpython/issues/135528]: `http.cookiejar`: add “tv”, “or”, “nom”, “sch”, and “web” to the default list of supported country code second-level domains.

- gh-135056 [https://github.com/python/cpython/issues/135056]: Add a `-H` or `--header` CLI option to **python -m http.server**. Contributed by Anton I. Sipos.

- gh-134551 [https://github.com/python/cpython/issues/134551]: Add t-strings support to pprint functions

- gh-133956 [https://github.com/python/cpython/issues/133956]: Fix bug where `@dataclass` wouldn’t detect `ClassVar` fields if `ClassVar` was re-exported from a module other than `typing`.

- gh-132631 [https://github.com/python/cpython/issues/132631]: Fix “I/O operation on closed file” when parsing JSON Lines file with `JSON CLI`.

- gh-108951 [https://github.com/python/cpython/issues/108951]: `asyncio`: Add `TaskGroup.cancel` which cancels unfinished tasks and exits the group without raising `asyncio.CancelledError`.

- gh-123853 [https://github.com/python/cpython/issues/123853]: Update the table of Windows language code identifiers (LCIDs) used by `locale.getdefaultlocale()` on Windows to protocol version 16.0 (2024-04-23).

- gh-122476 [https://github.com/python/cpython/issues/122476]: The `email` module no longer incorrectly uses **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] encoding for a mailbox with non-ASCII characters in its local-part. Under a policy with `utf8` set `False`, attempting to serialize such a message will now raise an `HeaderWriteError`. There is no valid 7-bit encoding for an internationalized local-part. Use `email.policy.SMTPUTF8` (or another policy with `utf8=True`) to correctly pass through the local-part as Unicode characters.

- gh-83938 [https://github.com/python/cpython/issues/83938]: The `email` module no longer incorrectly uses **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] encoding for a mailbox with non-ASCII characters in its domain. Under a policy with `utf8` set `False`, attempting to serialize such a message will now raise an `HeaderWriteError`. Either apply an appropriate IDNA encoding to convert the domain to ASCII before serialization, or use `email.policy.SMTPUTF8` (or another policy with `utf8=True`) to correctly pass through the internationalized domain name as Unicode characters.

- gh-81074 [https://github.com/python/cpython/issues/81074]: The `email` module no longer treats email addresses with non-ASCII characters as defects when parsing a Unicode string or in the `addr_spec` parameter to `email.headerregistry.Address`. **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] permits such addresses, and

they were already supported when parsing bytes and in the Address `username` parameter.

The (undocumented) **`email.errors.NonASCIILocalPartDefect`** is no longer used and should be considered deprecated.

- gh-70039 [https://github.com/python/cpython/issues/70039]: Fixed bug where **`smtplib.SMTP.starttls()`** could fail if **`smtplib.SMTP.connect()`** is called explicitly rather than implicitly.

- gh-113471 [https://github.com/python/cpython/issues/113471]: Allow **`http.server`** to set a default content-type when serving files with an unknown or missing extension.

- gh-83281 [https://github.com/python/cpython/issues/83281]: **`email`**: improve handling trailing garbage in address lists to avoid throwing AttributeError in certain edge cases

- gh-96894 [https://github.com/python/cpython/issues/96894]: Do not turn echo off for subsequent commands in batch activators (`activate.bat` and `deactivate.bat`) of **`venv`**.

Documentação

- gh-148663 [https://github.com/python/cpython/issues/148663]: Document that **`calendar.IllegalMonthError`** is a subclass of both **`ValueError`** and **`IndexError`** since Python 3.12.
- gh-146646 [https://github.com/python/cpython/issues/146646]: Document that **`glob.glob()`**, **`glob.iglob()`**, **`pathlib.Path.glob()`**, and **`pathlib.Path.rglob()`** silently suppress **`OSError`** exceptions raised from scanning the filesystem.

Testes

- gh-149425 [https://github.com/python/cpython/issues/149425]: Increase time delta in

`test.test_zipfile.test_core.OtherTests.test_write_without_source_date_epoch`
- gh-148600 [https://github.com/python/cpython/issues/148600]: Add OpenSSL 4.0.0 support to test configurations.

Construção

- gh-149353 [https://github.com/python/cpython/issues/149353]: Avoid unnecessary JIT-related rebuilds during `make install` after `--enable-optimizations` builds.
- gh-149351 [https://github.com/python/cpython/issues/149351]: Avoid possible broken macOS framework install names when DESTDIR is specified during builds.
- gh-149252 [https://github.com/python/cpython/issues/149252]: Update to WASI SDK 33.
- gh-148690 [https://github.com/python/cpython/issues/148690]: Windows free-threaded builds now output to a different default path with default filenames, for example, `PCbuild/amd64t/python.exe` rather than `PCbuild/amd64/python3.15t.exe`. The `PC/layout` script has been updated to ensure compatibility of generated layouts.
- gh-146475 [https://github.com/python/cpython/issues/146475]: Block Apple Clang from being used to build the JIT as it ships without required LLVM tools.
- gh-148644 [https://github.com/python/cpython/issues/148644]: Errors during the PGO training job on Windows are no longer ignored, and a non-zero return code will cause the build to fail.
- gh-148535 [https://github.com/python/cpython/issues/148535]: No longer use the `gcc -fprofile-update=atomic` flag on i686. The flag has been added to fix a random GCC internal error on PGO build (gh-145801 [https://github.com/python/cpython/issues/145801]) caused by corruption of profile data (.gcda files). The problem is that it makes the PGO build way slower (up to 47x slower) on i686. Since the GCC internal error was not seen on i686 so far, don't use `-fprofile-update=atomic` on i686 anymore. Patch by Victor Stinner.
- gh-148483 [https://github.com/python/cpython/issues/148483]: Use `Py_GCC_ATTRIBUTE(unused)` for stop_tracing label.

- gh-148474 [https://github.com/python/cpython/issues/148474]: Fixed compilation of `Python/pystrhex.c` with older clang versions.
- gh-146445 [https://github.com/python/cpython/issues/146445]: The Android build tools have been moved to the Platforms folder.
- gh-146264 [https://github.com/python/cpython/issues/146264]: Fix static module builds on non-WASI targets by linking HACL dependencies as static libraries when `MODULE_BUILDTYPE=static`, preventing duplicate `_Py_LibHacl_*` symbol errors at link time.
- gh-138451 [https://github.com/python/cpython/issues/138451]: Allow for custom LLVM path using `LLVM_TOOLS_INSTALL_DIR` during JIT build.
- gh-133312 [https://github.com/python/cpython/issues/133312]: Add a new `./configure` option **`--enable-static-libpython-for-interpreter`** which, when used with **`--enable-shared`**, continues to build the shared library but does not use it for the interpreter. Instead, libpython is statically linked into the interpreter, as if **`--enable-shared`** had not been used. This allows you to do a single build and get a Python interpreter binary that does not use a shared library but also get a shared library for use by other programs.

Windows

- gh-149254 [https://github.com/python/cpython/issues/149254]: Updated bundled version of OpenSSL to 3.5.6.
- gh-148690 [https://github.com/python/cpython/issues/148690]: Non-freethreaded builds on Windows now support extensions linked to `python3t.dll`, and will include a copy of that library in normal installs that references the non-freethreaded runtime.
- gh-146458 [https://github.com/python/cpython/issues/146458]: Fix incorrect REPL height and width tracking on console window resize on Windows.

macOS

- gh-142295 [https://github.com/python/cpython/issues/142295]: For Python macOS framework builds, update Info.plist files to be more compliant with current Apple guidelines. Original patch contributed by Martinus Verburg.

- gh-149254 [https://github.com/python/cpython/issues/149254]: Atualiza o instalador do macOS para usar OpenSSL 3.5.6.

IDLE

- gh-94523 [https://github.com/python/cpython/issues/94523]: Detect file if modified at local disk and prompt to ask refresh. Patch by Shixian Li.
- gh-139551 [https://github.com/python/cpython/issues/139551]: Support rendering `BaseExceptionGroup` in IDLE.
- gh-89520 [https://github.com/python/cpython/issues/89520]: Make IDLE extension configuration look at user config files, allowing user-installed extensions to have settings and key bindings defined in ~/.idlerc.

API C

- gh-149225 [https://github.com/python/cpython/issues/149225]: `PyCriticalSection` and related functions are added to the Stable ABI.
- gh-149216 [https://github.com/python/cpython/issues/149216]: `PyType_WatchCallback` callbacks registered via `PyType_AddWatcher()` are now also invoked when a watched heap type is deallocated. Previously, type watchers were only notified of modifications, which could cause stale references when a type was freed and its address was reused.
- gh-149101 [https://github.com/python/cpython/issues/149101]: Implement **PEP 788** [https://peps.python.org/pep-0788/].
- gh-149044 [https://github.com/python/cpython/issues/149044]: Implement **PEP 820** [https://peps.python.org/pep-0820/]: Unified slot system for the C API.
- gh-148267 [https://github.com/python/cpython/issues/148267]: Using `Py_LIMITED_API` on a non-Windows free-threaded build no longer needs an extra `Py_GIL_DISABLED`.
- gh-145559 [https://github.com/python/cpython/issues/145559]: Rename `_Py_DumpTraceback` and `_Py_DumpTracebackThreads` to `PyUnstable_DumpTraceback()` and `PyUnstable_DumpTracebackThreads()`.
- gh-146636 [https://github.com/python/cpython/issues/146636]: Implement **PEP 803** [https://peps.python.org/pep-0803/] – `abi3t`: Stable ABI for Free-Threaded Builds.

- gh-146302 [https://github.com/python/cpython/issues/146302]: `Py_IsInitialized()` no longer returns true until initialization has fully completed, including import of the `site` module. The underlying runtime flags now use atomic operations.
- gh-146063 [https://github.com/python/cpython/issues/146063]: Add `PyObject_CallFinalizerFromDealloc()` function to the limited C API. Patch by Victor Stinner.
- gh-145921 [https://github.com/python/cpython/issues/145921]: Add functions that are guaranteed to be safe for use in `tp_traverse` handlers: `PyObject_GetTypeData_DuringGC()`, `PyObject_GetItemData_DuringGC()`, `PyType_GetModuleState_DuringGC()`, `PyModule_GetState_DuringGC()`, `PyModule_GetToken_DuringGC()`, `PyType_GetBaseByToken_DuringGC()`, `PyType_GetModule_DuringGC()`, `PyType_GetModuleByToken_DuringGC()`.

Python 3.15.0 alpha 8

Release date: 2026-04-07

Segurança

- gh-145986 [https://github.com/python/cpython/issues/145986]: `xml.parsers.expat`: Fixed a crash caused by unbounded C recursion when converting deeply nested XML content models with `ElementDeclHandler()`. This addresses **CVE 2026-4224** [https://www.cve.org/CVERecord?id=CVE-2026-4224].
- gh-145599 [https://github.com/python/cpython/issues/145599]: Reject control characters in `http.cookies.Morsel` `update()` and `js_output()`. This addresses **CVE 2026-3644** [https://www.cve.org/CVERecord?id=CVE-2026-3644].
- gh-143930 [https://github.com/python/cpython/issues/143930]: Reject leading dashes in URLs passed to `webbrowser.open()`.

Núcleo e embutidos

- gh-148157 [https://github.com/python/cpython/issues/148157]: Fix an unlikely crash when parsing an invalid type comments for function parameters. Found by OSS Fuzz in #492782951 [https://issues.oss-fuzz.com/issues/492782951].
- gh-100239 [https://github.com/python/cpython/issues/100239]: Propagate result type and uniqueness information through `_BINARY_OP_EXTEND` in the tier 2 optimizer, enabling elimination of downstream type guards and selection of inplace float operations.
- gh-148144 [https://github.com/python/cpython/issues/148144]: Initialize `_PyInterpreterFrame.visited` when copying interpreter frames so incremental GC does not read an uninitialized byte from generator and frame-object copies.
- gh-148072 [https://github.com/python/cpython/issues/148072]: Cache `pickle.dumps` and `pickle.loads` per interpreter in the XIData framework, avoiding repeated module lookups on every cross-interpreter data transfer. This speeds up **`InterpreterPoolExecutor`** for mutable types (`list`, `dict`) by 1.7x–3.3x.
- gh-148110 [https://github.com/python/cpython/issues/148110]: Fix **`sys.set_lazy_imports_filter()`** so relative lazy imports pass the resolved imported module name to the filter callback. Patch by Pablo Galindo.
- gh-148083 [https://github.com/python/cpython/issues/148083]: Constant-fold `_CONTAINS_OP_SET` for **`frozenset`**. Patch by Donghee Na.
- gh-144319 [https://github.com/python/cpython/issues/144319]: Fix a bug that could cause applications with specific allocation patterns to leak memory via Huge Pages if compiled with Huge Page support. Patch by Pablo Galindo
- gh-147985 [https://github.com/python/cpython/issues/147985]: Make **`PySet_Contains()`** attempt a lock-free lookup, similar to **`set.__contains__()`**. This avoids acquiring the set object mutex in the normal case.
- gh-147856 [https://github.com/python/cpython/issues/147856]: Allow the *count* argument of **`bytes.replace()`** to be a keyword.
- gh-146615 [https://github.com/python/cpython/issues/146615]: Fix a crash in **`__get__()`** for METH_METHOD descriptors when an invalid (non-type) object is passed as the second argument. Patch by Steven Sun.

- gh-146306 [https://github.com/python/cpython/issues/146306]: Optimize compact integer arithmetic in the JIT by mutating uniquely-referenced operands in place, avoiding allocation of a new int object. Speeds up the pyperformance `spectral_norm` benchmark by ~10%.
- gh-146587 [https://github.com/python/cpython/issues/146587]: Fix type slot assignment incase of multiple slots for same name in type object implementation. Patch by Kumar Aditya.
- gh-126910 [https://github.com/python/cpython/issues/126910]: Set frame pointers in `aarch64-unknown-linux-gnu` JIT code, allowing most native profilers and debuggers to unwind through them. Patch by Diego Russo
- gh-146388 [https://github.com/python/cpython/issues/146388]: Adds a null check to handle when the JIT optimizer runs out of space when dealing with contradictions in `make_bottom`.
- gh-146308 [https://github.com/python/cpython/issues/146308]: Fixed multiple error handling issues in the **`_remote_debugging`** module including a double-free in code object caching, memory leaks on allocation failure, missing exception checks in binary format varint decoding, reference leaks on error paths in frame chain processing, and inconsistent thread status error reporting across platforms. Patch by Pablo Galindo.
- gh-146306 [https://github.com/python/cpython/issues/146306]: Optimize float arithmetic in the JIT by mutating uniquely-referenced operands in place, avoiding allocation of a new float object. Speeds up the pyperformance `nbody` benchmark by ~19%.
- gh-146128 [https://github.com/python/cpython/issues/146128]: Fix a bug which could cause constant values to be partially corrupted in AArch64 JIT code. This issue is theoretical, and hasn't actually been observed in unmodified Python interpreters.
- gh-146250 [https://github.com/python/cpython/issues/146250]: Fixed a memory leak in **`SyntaxError`** when re-initializing it.
- gh-146245 [https://github.com/python/cpython/issues/146245]: Fixed reference leaks in **`socket`** when audit hooks raise exceptions in **`socket.getaddrinfo()`** and **`socket.sendto()`**.
- gh-146151 [https://github.com/python/cpython/issues/146151]: **`memoryview`** now supports the float complex and double complex C types: formatting characters `'F'` and `'D'` respectively. Patch by Sergey B Kirpichev.

- gh-146196 [https://github.com/python/cpython/issues/146196]: Fix potential Undefined Behavior in `PyUnicodeWriter_WriteASCII()` by adding a zero-length check. Patch by Shamil Abdulaev.
- gh-146227 [https://github.com/python/cpython/issues/146227]: Fix wrong type in `_Py_atomic_load_uint16` in the C11 atomics backend (`pyatomic_std.h`), which used a 32-bit atomic load instead of 16-bit. Found by Mohammed Zuhaib.
- gh-146205 [https://github.com/python/cpython/issues/146205]: Fixed a bug where `select.epoll.close()`, `select.kqueue.close()`, and `select.devpoll.close()` silently ignored errors.
- gh-146199 [https://github.com/python/cpython/issues/146199]: Comparison of code objects now handles errors correctly.
- gh-145667 [https://github.com/python/cpython/issues/145667]: Remove the `GET_ITER_YIELD_FROM` instruction, modifying `SEND` to pair with `GET_ITER` when compiling `yield from` expressions.
- gh-146192 [https://github.com/python/cpython/issues/146192]: Add Base32 support to `binascii` and improve the performance of the Base32 converters in `base64`. Patch by James Seo.
- gh-135871 [https://github.com/python/cpython/issues/135871]: Improve multithreaded scaling of PyMutex in low-contention scenarios by reloading the lock's internal state, without slowing down high-contention scenarios.
- gh-146096 [https://github.com/python/cpython/issues/146096]: Fixed segmentation fault when called repr for BaseExceptionGroup with empty or 1-size tuple args.
- gh-146056 [https://github.com/python/cpython/issues/146056]: Fix `repr()` for lists and tuples containing `NULL`s.
- gh-145059 [https://github.com/python/cpython/issues/145059]: Fixed `sys.lazy_modules` to include lazy modules without submodules. Patch by Bartosz Sławecki.
- gh-146041 [https://github.com/python/cpython/issues/146041]: Fix free-threading scaling bottleneck in `sys.intern()` and `PyObject_SetAttr()` by avoiding the interpreter-wide lock when the string is already interned and immortalized.
- gh-145990 [https://github.com/python/cpython/issues/145990]: `python --help-env` sections are now sorted by environment variable name.

- gh-145990 [https://github.com/python/cpython/issues/145990]: `python --help-xoptions` is now sorted by `-X` option name.
- gh-145876 [https://github.com/python/cpython/issues/145876]: **`AttributeError`**s and **`KeyError`**s raised in **`keys()`** or **`__getitem__()`** during dictionary unpacking (`{**mymapping}` or `func(**mymapping)`) are no longer masked by **`TypeError`**.
- gh-127958 [https://github.com/python/cpython/issues/127958]: Support tracing from function entrypoints in the JIT. Patch by Ken Jin.
- gh-145376 [https://github.com/python/cpython/issues/145376]: Fix GC tracking in `structseq.__replace__()`.
- gh-145792 [https://github.com/python/cpython/issues/145792]: Fix out-of-bounds access when invoking faulthandler on a CPython build compiled without support for VLAs.
- gh-142183 [https://github.com/python/cpython/issues/142183]: Avoid a pathological case where repeated calls at a specific stack depth could be significantly slower.
- gh-145779 [https://github.com/python/cpython/issues/145779]: Improve scaling of **`classmethod()`** and **`staticmethod()`** calls in the free-threaded build by avoiding the descriptor `__get__` call.
- gh-145783 [https://github.com/python/cpython/issues/145783]: Fix an unlikely crash in the parser when certain errors were erroneously not propagated. Found by OSS Fuzz in #491369109 [https://issues.oss-fuzz.com/issues/491369109].
- gh-145685 [https://github.com/python/cpython/issues/145685]: Improve scaling of type attribute lookups in the free-threaded build by avoiding contention on the internal type lock.
- gh-145713 [https://github.com/python/cpython/issues/145713]: Make **`bytearray.resize()`** thread-safe in the free-threaded build by using a critical section and calling the lock-held variant of the resize function.
- gh-145036 [https://github.com/python/cpython/issues/145036]: In free-threaded build, fix race condition when calling **`__sizeof__()`** on a **`list`**
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting for `MATCH_CLASS` in the JIT.
- gh-69605 [https://github.com/python/cpython/issues/69605]: Add **`math.integer`** to REPL auto-completion of imports.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize `_ITER_CHECK_RANGE` and `_ITER_CHECK_LIST` in the JIT

- gh-143414 [https://github.com/python/cpython/issues/143414]: Add tracking to the JIT optimizer to determine whether a reference is uniquely owned or shared
- gh-143636 [https://github.com/python/cpython/issues/143636]: Fix a crash when calling `SimpleNamespace.__replace__()` on non-namespace instances. Patch by Bénédikt Tran.
- gh-126910 [https://github.com/python/cpython/issues/126910]: Set frame pointers in `x86_64-unknown-linux-gnu` JIT code, allowing most native profilers and debuggers to unwind through them.
- gh-140594 [https://github.com/python/cpython/issues/140594]: Fix an out of bounds read when a single NUL character is read from the standard input. Patch by Shamil Abdulaev.
- gh-140870 [https://github.com/python/cpython/issues/140870]: Add support for module attributes in the REPL auto-completion of imports.

Biblioteca

- gh-144503 [https://github.com/python/cpython/issues/144503]: Fix a regression introduced in 3.14.3 and 3.13.12 where the `multiprocessing` `forkserver` start method would fail with `BrokenPipeError` when the parent process had a very large `sys.argv`. The argv is now passed to the forkserver as separate command-line arguments rather than being embedded in the `-c` command string, avoiding the operating system's per-argument length limit.
- gh-148153 [https://github.com/python/cpython/issues/148153]: `base64.b32encode()` now always raises `ValueError` instead of `AssertionError` for the value of *map01* with invalid length.
- gh-73613 [https://github.com/python/cpython/issues/73613]: Add the *padded* parameter in functions related to Base32 and Base64 codecs in the `binascii` and `base64` modules. In the encoding functions it controls whether the pad character can be added in the output, in the decoding functions it controls whether padding is required in input. Padding of input no longer required in `base64.urlsafe_b64decode()` by default.
- gh-146613 [https://github.com/python/cpython/issues/146613]: `itertools`: Fix a crash in `itertools.groupby()` when the grouper iterator is concurrently mutated.

- gh-147944 [https://github.com/python/cpython/issues/147944]: Accepted range for the *bytes_per_sep* argument of `bytes.hex()`, `bytearray.hex()`, `memoryview.hex()`, and `binascii.b2a_hex()` is now increased, so passing `sys.maxsize` and `-sys.maxsize` is now valid.
- gh-146080 [https://github.com/python/cpython/issues/146080]: `ssl`: fix a crash when an SNI callback tries to use an SSL object that has already been garbage-collected. Patch by Bénédikt Tran.
- gh-146556 [https://github.com/python/cpython/issues/146556]: Fix `annotationlib.get_annotations()` hanging indefinitely when called with `eval_str=True` on a callable that has a circular `__wrapped__` chain (e.g. `f.__wrapped__ = f`). Cycle detection using an id-based visited set now stops the traversal and falls back to the globals found so far, mirroring the approach of `inspect.unwrap()`.
- gh-146090 [https://github.com/python/cpython/issues/146090]: `sqlite3`: fix a crash when `sqlite3.Connection.create_collation()` fails with SQLITE_BUSY [https://sqlite.org/rescode.html#busy]. Patch by Bénédikt Tran.
- gh-146090 [https://github.com/python/cpython/issues/146090]: `sqlite3`: properly raise `MemoryError` instead of `SystemError` when a context callback fails to be allocated. Patch by Bénédikt Tran.
- gh-146507 [https://github.com/python/cpython/issues/146507]: Make `asyncio.SelectorEventLoop()` stream transport's `get_write_buffer_size()` O(1) by maintaining a running byte counter instead of iterating the buffer on every call.
- gh-145056 [https://github.com/python/cpython/issues/145056]: Fix merging of `collections.OrderedDict` and `frozendict`.
- gh-145056 [https://github.com/python/cpython/issues/145056]: Add support for merging `collections.UserDict` and `frozendict`.
- gh-145633 [https://github.com/python/cpython/issues/145633]: Fix `struct.pack('f', float)`: use `PyFloat_Pack4()` to raise `OverflowError`. Patch by Sergey B Kirpichev and Victor Stinner.
- gh-146440 [https://github.com/python/cpython/issues/146440]: `json`: Add the *array_hook* parameter to `load()` and `loads()` functions: allow a callback for JSON literal array types to customize Python lists in the resulting decoded object. Passing combined `frozendict` to *object_pairs_hook* param and `tuple` to `array_hook` will yield a deeply nested immutable Python structure representing the JSON data.

- gh-146431 [https://github.com/python/cpython/issues/146431]: Add the *wrapcol* parameter to **base64** functions **b16encode()**, **b32encode()**, **b32hexencode()**, **b85encode()** and **z85encode()**, and **binascii** functions **b2a_base32()** and **b2a_base85()**. Add the *ignorechars* parameter to **base64** functions **b16decode()**, **b32decode()**, **b32hexdecode()**, **b85decode()** and **z85decode()**, and **binascii** functions **a2b_hex()**, **unhexlify()**, **a2b_base32()** and **a2b_base85()**.
- gh-146310 [https://github.com/python/cpython/issues/146310]: The **ensurepip** module no longer looks for `pip-*.whl` wheel packages in the current directory.
- gh-141510 [https://github.com/python/cpython/issues/141510]: Support **frozendict** in **plistlib**, for serialization only. Patch by Hugo van Kemenade.
- gh-146238 [https://github.com/python/cpython/issues/146238]: Support half-floats (type code `'e'` of the **struct** module) in the **array** module. Patch by Sergey B Kirpichev.
- gh-140947 [https://github.com/python/cpython/issues/140947]: Fix incorrect contextvars handling in server tasks created by **asyncio**. Patch by Kumar Aditya.
- gh-146151 [https://github.com/python/cpython/issues/146151]: Support the float complex and double complex C types in the **array** module: formatting characters `'F'` and `'D'` respectively. Patch by Sergey B Kirpichev.
- gh-143387 [https://github.com/python/cpython/issues/143387]: In importlib.metadata, when a distribution file is corrupt and there is no metadata file, calls to `Distribution.metadata()` (including implicit calls from other properties like `.name` and `.requires`) will now raise a `MetadataNotFound` Exception. This allows callers to distinguish between missing metadata and a degenerate (empty) metadata. Previously, if the file was missing, an empty `PackageMetadata` would be returned and would be indistinguishable from the presence of an empty file.
- gh-146228 [https://github.com/python/cpython/issues/146228]: Cached FastPath objects in importlib.metadata are now cleared on fork, avoiding broken references to zip files during fork.
- gh-146171 [https://github.com/python/cpython/issues/146171]: Nested **AttributeError** suggestions now include property-backed attributes on nested objects without executing the property getter.

- gh-145410 [https://github.com/python/cpython/issues/145410]: On Windows, `sysconfig.get_platform()` now gets the platform from the `_sysconfig` module instead of parsing `sys.version` string. Patch by Victor Stinner.
- gh-146091 [https://github.com/python/cpython/issues/146091]: Fix a bug in `termios.tcsetwinsize()` where passing a sequence that raises an exception in `__getitem__` would cause a `SystemError` instead of propagating the original exception.
- gh-146083 [https://github.com/python/cpython/issues/146083]: Update bundled libexpat [https://libexpat.github.io/] to version 2.7.5.
- gh-146076 [https://github.com/python/cpython/issues/146076]: `zoneinfo`: fix crashes when deleting `_weak_cache` from a `zoneinfo.ZoneInfo` subclass.
- gh-123471 [https://github.com/python/cpython/issues/123471]: Make concurrent iteration over `itertools.zip_longest` safe under free-threading.
- gh-146075 [https://github.com/python/cpython/issues/146075]: Errors when calling `functools.partial()` with a malformed keyword will no longer crash the interpreter.
- gh-146054 [https://github.com/python/cpython/issues/146054]: Limit the size of `encodings.search_function()` cache. Found by OSS Fuzz in #493449985 [https://issues.oss-fuzz.com/issues/493449985].
- gh-146004 [https://github.com/python/cpython/issues/146004]: All `-X` options from the Python command line are now propagated to child processes spawned by `multiprocessing`, not just a hard-coded subset. This makes the behavior consistent between default “spawn” and “forkserver” start methods and the old “fork” start method. The options that were previously not propagated are: `context_aware_warnings`, `cpu_count`, `disable-remote-debug`, `int_max_str_digits`, `lazy_imports`, `no_debug_ranges`, `pathconfig_warnings`, `perf`, `perf_jit`, `presite`, `pycache_prefix`, `thread_inherit_context`, and `warn_default_encoding`.
- gh-145980 [https://github.com/python/cpython/issues/145980]: Added the *alphabet* parameter in `b2a_base64()`, `a2b_base64()`, `b2a_base85()` and `a2b_base85()` and a number of `*_ALPHABET` constants in the `binascii` module. Removed **`b2a_z85()`** and **`a2b_z85()`**.
- gh-145968 [https://github.com/python/cpython/issues/145968]: Fix translation in `base64.b64decode()` when altchars overlaps with the standard ones.

- gh-145966 [https://github.com/python/cpython/issues/145966]: Non-`AttributeError` exceptions raised during dialect attribute lookup in `csv` are no longer silently suppressed.
- gh-145883 [https://github.com/python/cpython/issues/145883]: `zoneinfo`: Fix heap buffer overflow reads from malformed TZif data. Found by OSS Fuzz, issues #492245058 [https://issues.oss-fuzz.com/issues/492245058] and #492230068 [https://issues.oss-fuzz.com/issues/492230068].
- gh-145850 [https://github.com/python/cpython/issues/145850]: Changed some implementation details in `struct.Struct`: calling it with non-ASCII string format will now raise a `ValueError` instead of `UnicodeEncodeError`, calling it with non-ASCII bytes format will now raise a `ValueError` instead of `struct.error`, getting the `format` attribute of uninitialized object will now raise an `AttributeError` instead of `RuntimeError`.
- gh-123720 [https://github.com/python/cpython/issues/123720]: asyncio: Fix `asyncio.Server.serve_forever()` shutdown regression. Since 3.12, cancelling `serve_forever()` could hang waiting for a handler blocked on a read from a client that never closed (effectively requiring two interrupts to stop); the shutdown sequence now ensures client streams are closed so `serve_forever()` exits promptly and handlers observe EOF.
- gh-138122 [https://github.com/python/cpython/issues/138122]: The `profiling.sampling` module now supports differential flamegraph visualization via `--diff-flamegraph` to compare two profiling runs. Functions are colored red (regressions), blue (improvements), gray (neutral), or purple (new). Elided stacks show code paths that disappeared between runs.
- gh-145754 [https://github.com/python/cpython/issues/145754]: Request signature during mock autospec with `FORWARDREF` annotation format. This prevents runtime errors when an annotation uses a name that is not defined at runtime.
- gh-145750 [https://github.com/python/cpython/issues/145750]: Avoid undefined behaviour from signed integer overflow when parsing format strings in the `struct` module. Found by OSS Fuzz in #488466741 [https://issues.oss-fuzz.com/issues/488466741].
- gh-145719 [https://github.com/python/cpython/issues/145719]: Add `application/efi` MIME type to `mimetypes`.

- gh-145717 [https://github.com/python/cpython/issues/145717]: Add a few Microsoft-specific MIME types.
- gh-145703 [https://github.com/python/cpython/issues/145703]: **`asyncio`**: Make sure that **`loop.call_at`** and **`loop.call_later`** trigger scheduled events on time when the clock resolution becomes too small.
- gh-145697 [https://github.com/python/cpython/issues/145697]: Add `application/sql` and `application/vnd.sqlite3` into `mimetypes`.
- gh-145492 [https://github.com/python/cpython/issues/145492]: Fix infinite recursion in **`collections.defaultdict`** `__repr__` when a `defaultdict` contains itself. Based on analysis by KowalskiThomas in gh-145492 [https://github.com/python/cpython/issues/145492].
- gh-145650 [https://github.com/python/cpython/issues/145650]: Add **`__repr__()`** support to **`logging.Formatter`** and **`logging.Filter`**, showing the format string and filter name respectively.
- gh-145587 [https://github.com/python/cpython/issues/145587]: Resolved a performance regression in `multiprocessing.connection.wait` on Windows that caused infinite busy loops when called with no objects. The function now properly yields control to the OS to conserve CPU resources. Patch By Shrey Naithani
- gh-145616 [https://github.com/python/cpython/issues/145616]: Detect Android sysconfig ABI correctly on 32-bit ARM Android on 64-bit ARM kernel
- gh-145546 [https://github.com/python/cpython/issues/145546]: Fix `unittest.util.sorted_list_difference()` to deduplicate remaining elements when one input list is exhausted before the other.
- gh-145446 [https://github.com/python/cpython/issues/145446]: Now **`functools`** is safer in free-threaded build when using keywords in **`functools.partial()`**
- gh-145264 [https://github.com/python/cpython/issues/145264]: Base64 decoder (see **`binascii.a2b_base64()`**, **`base64.b64decode()`**, etc) no longer ignores excess data after the first padded quad in non-strict (default) mode. Instead, in conformance with **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html], section 3.3, it now ignores the pad character, “=”, if it is present before the end of the encoded data.
- gh-145035 [https://github.com/python/cpython/issues/145035]: Allows omitting the internal library `_pyrepl` with limited loss of functionality. This allows

complete removal of the modern REPL, which is an unsupported configuration, but still desirable for some distributions.

- gh-144270 [https://github.com/python/cpython/issues/144270]: Made the *tag* parameter of `xml.etree.ElementTree.Element` and the *parent* and *tag* parameters of `xml.etree.ElementTree.SubElement()` positional-only, matching the behavior of the C accelerator.
- gh-144984 [https://github.com/python/cpython/issues/144984]: Fix crash in `xml.parsers.expat.xmlparser.ExternalEntityParserCreate()` when an allocation fails. The error paths could dereference NULL `handlers` and double-decrement the parent parser's reference count.
- gh-144975 [https://github.com/python/cpython/issues/144975]: `wave.Wave_write.setframerate()` now validates the frame rate after rounding to an integer, preventing values like `0.5` from being accepted and causing confusing errors later. Patch by Michiel Beijen.
- gh-140715 [https://github.com/python/cpython/issues/140715]: Add `%n` and `%t` support to `strptime()`.
- gh-144259 [https://github.com/python/cpython/issues/144259]: Fix inconsistent display of long multiline pasted content in the REPL.
- gh-140814 [https://github.com/python/cpython/issues/140814]: `multiprocessing.freeze_support()` no longer sets the default start method as a side effect, which previously caused a subsequent `multiprocessing.set_start_method()` call to raise `RuntimeError`.
- gh-123471 [https://github.com/python/cpython/issues/123471]: Make concurrent iteration over `itertools.accumulate` safe under free-threading.
- gh-143715 [https://github.com/python/cpython/issues/143715]: Calling the `Struct.__new__()` without required argument now is deprecated. Calling `__init__()` method on initialized `Struct` objects is deprecated.
- gh-142763 [https://github.com/python/cpython/issues/142763]: Fix a race condition between `zoneinfo.ZoneInfo` creation and `zoneinfo.ZoneInfo.clear_cache()` that could raise `KeyError`.
- gh-141707 [https://github.com/python/cpython/issues/141707]: Don't change `tarfile.TarInfo` type from `AREGTYPE` to `DIRTYPE` when parsing GNU long name or link headers.
- gh-138577 [https://github.com/python/cpython/issues/138577]: `getpass.getpass()` with non-empty `echo_char` now handles keyboard shortcuts including Ctrl+A/E (cursor movement), Ctrl+K/U

(kill line), Ctrl+W (erase word), and Ctrl+V (literal next) by reading the terminal's control character settings and processing them appropriately in non-canonical mode. Patch by Sanyam Khurana.

- gh-140049 [https://github.com/python/cpython/issues/140049]: **`traceback.format_exception_only()`** now colorizes exception notes.
- gh-139933 [https://github.com/python/cpython/issues/139933]: Improve **`AttributeError`** suggestions for classes with a custom **`__dir__()`** method returning a list of unsortable values. Patch by Bénédikt Tran.
- gh-139633 [https://github.com/python/cpython/issues/139633]: The **`netrc`** security check is now run once per parse rather than once per entry.
- gh-130472 [https://github.com/python/cpython/issues/130472]: Add fancycompleter and enable it by default when using pyrepl. This gives colored tab completion.
- gh-112632 [https://github.com/python/cpython/issues/112632]: Add an *expand* keyword argument for **`pprint.pprint()`**, **`pprint.pformat()`**, **`pprint.pp()`** by passing on all *kwargs* and **`pprint.PrettyPrinter`**. Contributed by Stefan Todoran and Semyon Moroz.
- gh-66419 [https://github.com/python/cpython/issues/66419]: Optional argument with nargs equals to `argparse.REMAINDER` now consumes all remaining arguments including `'--'`.
- gh-60729 [https://github.com/python/cpython/issues/60729]: Add support for floating point audio wave files in **`wave`**.
- bpo-36461 [https://bugs.python.org/issue?@action=redirect&bpo=36461]: Make the target time of **`timeit.Timer.autorange()`** configurable and add `--target-time` option to the command-line interface of **`timeit`**.

Documentação

- gh-126676 [https://github.com/python/cpython/issues/126676]: Expand **`argparse`** documentation for `type=bool` with a demonstration of the surprising behavior and pointers to common alternatives.
- gh-145649 [https://github.com/python/cpython/issues/145649]: Fix text wrapping and formatting of `-X` option descriptions in the *python(1) [https://manpages.debian.org/python(1)]* man page by using proper roff markup.

Testes

- gh-144418 [https://github.com/python/cpython/issues/144418]: The Android testbed's emulator RAM has been increased from 2 GB to 4 GB.
- gh-146202 [https://github.com/python/cpython/issues/146202]: Fix a race condition in regrtest: make sure that the temporary directory is created in the worker process. Previously, temp_cwd() could fail on Windows if the "build" directory was not created. Patch by Victor Stinner.

Construção

- gh-146541 [https://github.com/python/cpython/issues/146541]: The Android testbed can now be built for 32-bit ARM and x86 targets.
- gh-146498 [https://github.com/python/cpython/issues/146498]: The iOS XCframework build script now ensures libpython isn't included in installed app content, and is more robust in identifying standard library binary content that requires processing.
- gh-146450 [https://github.com/python/cpython/issues/146450]: The Android build script was modified to improve parity with other platform build scripts.
- gh-146446 [https://github.com/python/cpython/issues/146446]: The clean target for the Apple/iOS XCframework build script is now more selective when targeting a single architecture.
- gh-146444 [https://github.com/python/cpython/issues/146444]: The Apple/iOS build script has been moved to the Platforms directory.
- gh-146210 [https://github.com/python/cpython/issues/146210]: Fix building the jit stencils on Windows when the interpreter is built with a different clang version. Patch by Chris Eibl.
- gh-145844 [https://github.com/python/cpython/issues/145844]: Update to WASI SDK 32.
- gh-145801 [https://github.com/python/cpython/issues/145801]: When Python build is optimized with GCC using PGO, use `-fprofile-update=atomic` option to use atomic operations when updating profile information. This option reduces the risk of gcov Data Files (.gcda) corruption which can cause random GCC crashes. Patch by Victor Stinner.
- gh-138850 [https://github.com/python/cpython/issues/138850]: Add **`--disable-epoll`** to `configure`
- gh-145633 [https://github.com/python/cpython/issues/145633]: Remove support for ancient ARM platforms (ARMv4L and ARMv5L OABI boards), using mixed-endian representation for doubles. Patch by Sergey B Kirpichev.

- gh-85277 [https://github.com/python/cpython/issues/85277]: Fix building without `stropts.h` or empty `stropts.h`

Windows

- gh-140131 [https://github.com/python/cpython/issues/140131]: Fix REPL cursor position on Windows when module completion suggestion line hits console width.

macOS

- gh-137586 [https://github.com/python/cpython/issues/137586]: Invoke **osascript** with absolute path in **`webbrowser`** and **`turtledemo`**.

Ferramentas/Demos

- gh-135953 [https://github.com/python/cpython/issues/135953]: Properly identify the main thread in the Gecko profiler collector by using a status flag from the interpreter state instead of relying on **`threading.main_thread()`** in the collector process.
- gh-145976 [https://github.com/python/cpython/issues/145976]: Remove `Misc/indent.pro`, a configuration file for GNU *indent(1)* *[https://manpages.debian.org/indent(1)]*.
- gh-145976 [https://github.com/python/cpython/issues/145976]: Remove `Misc/vgrindefs` and `Misc/Porting`.

API C

- gh-146636 [https://github.com/python/cpython/issues/146636]: The **`Py_mod_abi`** slot is now mandatory for modules created from a slots array (using **`PyModule_FromSlotsAndSpec()`** or the **`PyModExport_*`** export hook).

- gh-146175 [https://github.com/python/cpython/issues/146175]: The following macros are soft deprecated: **`Py_ALIGNED`**, **`PY_FORMAT_SIZE_T`**, **`Py_LL`**, **`Py_ULL`**, **`PY_LONG_LONG`**, **`PY_LLONG_MIN`**, **`PY_LLONG_MAX`**, **`PY_ULLONG_MAX`**, **`PY_INT32_T`**, **`PY_UINT32_T`**, **`PY_INT64_T`**, **`PY_UINT64_T`**, **`PY_SIZE_MAX`**, **`Py_UNICODE_SIZE`**, **`Py_VA_COPY`**.

A macro **`Py_UNICODE_WIDE`**, que estava agendada para remoção, está agora suavemente descontinuada em seu lugar.

- gh-146143 [https://github.com/python/cpython/issues/146143]: **`PyUnicodeWriter_WriteUCS4()`** now accepts a pointer to a constant buffer of `Py_UCS4`.

- gh-146056 [https://github.com/python/cpython/issues/146056]: **`PyUnicodeWriter_WriteRepr()`** now supports `NULL` argument.

- gh-145010 [https://github.com/python/cpython/issues/145010]: Use GCC dialect alternatives for inline assembly in `object.h` so that the Python headers compile correctly with `-masm=intel`.

Python 3.15.0 alpha 7

Release date: 2026-03-10

Windows

- gh-145731 [https://github.com/python/cpython/issues/145731]: Fix negative timestamp during DST on Windows. Patch by Hugo van Kemenade.
- gh-145307 [https://github.com/python/cpython/issues/145307]: Defers loading of the `psapi.dll` module until it is used by **`ctypes.util.dllist()`**.
- gh-144551 [https://github.com/python/cpython/issues/144551]: Updated bundled version of OpenSSL to 3.5.5.

Testes

- gh-144741 [https://github.com/python/cpython/issues/144741]: Fix `test_frame_pointer_unwind` when Python is built with **`--enable-shared`**. Classify also libpython frames as `"python"`. Patch by Victor Stinner.
- gh-144739 [https://github.com/python/cpython/issues/144739]: When Python was compiled with system expat older then 2.7.2 but tests run with newer expat, still skip **`test.test_pyexpat.MemoryProtectionTest`**.

Segurança

- gh-145506 [https://github.com/python/cpython/issues/145506]: Fixes **CVE 2026-2297** [https://www.cve.org/CVERecord?id=CVE-2026-2297] by ensuring that `SourcelessFileLoader` uses **`io.open_code()`** when opening `.pyc` files.
- gh-144370 [https://github.com/python/cpython/issues/144370]: Disallow usage of control characters in status in **`wsgiref.handlers`** to prevent HTTP header injections. Patch by Benedikt Johannes.

Biblioteca

- gh-145623 [https://github.com/python/cpython/issues/145623]: Fix crash in **`struct`** when calling **`repr()`** or `__sizeof__()` on an uninitialized **`struct.Struct`** object created via `Struct.__new__()` without calling `__init__()`.

- gh-145551 [https://github.com/python/cpython/issues/145551]: Fix InvalidStateError when cancelling process created by **`asyncio.create_subprocess_exec()`** or **`asyncio.create_subprocess_shell()`**. Patch by Daan De Meyer.

- gh-141510 [https://github.com/python/cpython/issues/141510]: **`marshal`** now supports **`frozendict`** objects. The marshal format version was increased to 6. Patch by Victor Stinner.

- gh-145417 [https://github.com/python/cpython/issues/145417]: **`venv`**: Prevent incorrect preservation of SELinux context when copying the `Activate.ps1` script. The script inherited the SELinux security context of the system template directory, rather than the destination project directory.

- gh-145335 [https://github.com/python/cpython/issues/145335]: `os.listdir(-1)` and `os.scandir(-1)` now fail with `OSError(errno.EBADF)` rather than listing the current directory. `os.listxattr(-1)` now fails with `OSError(errno.EBADF)` rather than listing extended attributes of the current directory. Patch by Victor Stinner.

- gh-145376 [https://github.com/python/cpython/issues/145376]: Fix double free and null pointer dereference in unusual error scenarios in **`hashlib`** and **`hmac`** modules.

- gh-145301 [https://github.com/python/cpython/issues/145301]: **`hmac`**: fix a crash when the initialization of the underlying C extension module fails.

- gh-145301 [https://github.com/python/cpython/issues/145301]: **`hashlib`**: fix a crash when the initialization of the underlying C extension module fails.

- gh-76007 [https://github.com/python/cpython/issues/76007]: The `version` attribute of the **`tarfile`** module is deprecated and slated for removal in Python 3.20.

- gh-145158 [https://github.com/python/cpython/issues/145158]: Avoid undefined behaviour from signed integer overflow when parsing format strings in the **`struct`** module.

- gh-123853 [https://github.com/python/cpython/issues/123853]: Removed Windows 95 compatibility for **`locale.getdefaultlocale()`**.

- gh-66802 [https://github.com/python/cpython/issues/66802]: Add **`unicodedata.block()`** function to return the Unicode block [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G64189] of a character.

- gh-145033 [https://github.com/python/cpython/issues/145033]: Add **`typing.TypeForm`**, implementing **PEP 747** [https://peps.python.org/pep-0747/]. Patch by Jelle Zijlstra.

- gh-141510 [https://github.com/python/cpython/issues/141510]: **`dataclasses.field()`**: if *metadata* is `None`, use an empty **`frozendict`**, instead of a **`MappingProxyType()`** of an empty **`dict`**. Patch by Victor Stinner.

- gh-145006 [https://github.com/python/cpython/issues/145006]: Add **`ModuleNotFoundError`** hints when a module for a different ABI exists.

- gh-141510 [https://github.com/python/cpython/issues/141510]: `ParameterizedMIMEHeader.params` of **`email.headerregistry`** is now a **`frozendict`** instead of a **`types.MappingProxyType`**. Patch by Victor Stinner.

- gh-134872 [https://github.com/python/cpython/issues/134872]: Add valid import name suggestions on **`ModuleNotFoundError`**.

- gh-88091 [https://github.com/python/cpython/issues/88091]: Fix **`unicodedata.decomposition()`** for Hangul characters.

- gh-144986 [https://github.com/python/cpython/issues/144986]: Fix a memory leak in **`atexit.register()`**. Patch by Shamil Abdulaev.

- gh-144777 [https://github.com/python/cpython/issues/144777]: Fix data races in **`io.IncrementalNewlineDecoder`** in the free-threaded build.

- gh-144809 [https://github.com/python/cpython/issues/144809]: Make **`collections.deque`** copy atomic in the free-threaded build.

- gh-141510 [https://github.com/python/cpython/issues/141510]: The **`copy`** module now supports the **`frozendict`** type. Patch by Pieter Eendebak based on work by Victor Stinner.

- gh-141510 [https://github.com/python/cpython/issues/141510]: The **`json`** module now supports the **`frozendict`** type. Patch by Victor Stinner.

- gh-144835 [https://github.com/python/cpython/issues/144835]: Added missing explanations for some parameters in **`glob.glob()`** and **`glob.iglob()`**.

- gh-144833 [https://github.com/python/cpython/issues/144833]: Fixed a use-after-free in **`ssl`** when `SSL_new()` returns NULL in `newPySSLSocket()`. The error was reported via a dangling pointer after the object had already been freed.

- gh-140715 [https://github.com/python/cpython/issues/140715]: Add `'%D'` support to **`strptime()`**.

- gh-144782 [https://github.com/python/cpython/issues/144782]: Fix `argparse.ArgumentParser` to be `pickleable`.

- gh-144763 [https://github.com/python/cpython/issues/144763]: Fix a race condition in `tracemalloc`: it no longer detaches the attached thread state to acquire its internal lock. Patch by Victor Stinner.

- gh-142224 [https://github.com/python/cpython/issues/142224]: `unicodedata.bidirectional()` now return the correct default bidi class for unassigned code points.

- gh-117865 [https://github.com/python/cpython/issues/117865]: Reduce the import time of `inspect` module by ~20%.

- gh-144156 [https://github.com/python/cpython/issues/144156]: Fix the folding of headers by the `email` library when **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] encoded words are used. Now whitespace is correctly preserved and also correctly added between adjacent encoded words. The latter property was broken by the fix for gh-92081 [https://github.com/python/cpython/issues/92081], which mostly fixed previous failures to preserve whitespace.

- gh-66305 [https://github.com/python/cpython/issues/66305]: Fixed a hang on Windows in the `tempfile` module when trying to create a temporary file or subdirectory in a non-writable directory.

- gh-144615 [https://github.com/python/cpython/issues/144615]: Methods directly decorated with `@functools.singledispatchmethod` now dispatch on the second argument when called after being accessed as class attributes. Patch by Bartosz Sławecki.

- gh-144321 [https://github.com/python/cpython/issues/144321]: The functional syntax for creating `typing.NamedTuple` classes now supports passing any iterable of fields and types. Previously, only sequences were supported.

- gh-144475 [https://github.com/python/cpython/issues/144475]: Calling `repr()` on `functools.partial()` is now safer when the partial object's internal

attributes are replaced while the string representation is being generated.

- gh-144285 [https://github.com/python/cpython/issues/144285]: Attribute suggestions in `AttributeError` tracebacks are now formatted differently to make them easier to understand, for example: `Did you mean '.datetime.now' instead of '.now'`. Contributed by Bartosz Sławecki.

- gh-144316 [https://github.com/python/cpython/issues/144316]: Fix crash in `_remote_debugging` that caused `test_external_inspection` to intermittently fail. Patch by Taegyun Kim.

- gh-143637 [https://github.com/python/cpython/issues/143637]: Fixed a crash in socket.sendmsg() that could occur if ancillary data is mutated re-entrantly during argument parsing.

- gh-140652 [https://github.com/python/cpython/issues/140652]: Fix a crash in **`_interpchannels.list_all()`** after closing a channel.

- gh-143698 [https://github.com/python/cpython/issues/143698]: Allow *scheduler* and *setpgroup* arguments to be explicitly `None` when calling `os.posix_spawn()` or `os.posix_spawnp()`. Patch by Bénédikt Tran.

- gh-143698 [https://github.com/python/cpython/issues/143698]: Raise `TypeError` instead of `SystemError` when the *scheduler* in `os.posix_spawn()` or `os.posix_spawnp()` is not a tuple. Patch by Bénédikt Tran.

- gh-142516 [https://github.com/python/cpython/issues/142516]: `ssl`: fix reference leaks in `ssl.SSLContext` objects. Patch by Bénédikt Tran.

- gh-85809 [https://github.com/python/cpython/issues/85809]: Added path-like object support for `shutil.make_archive()`.

- gh-143304 [https://github.com/python/cpython/issues/143304]: Fix `ctypes.CDLL` to honor the `handle` parameter on POSIX systems.

- gh-142781 [https://github.com/python/cpython/issues/142781]: `zoneinfo`: fix a crash when instantiating `ZoneInfo` objects for which the internal class-level cache is inconsistent.

- gh-142787 [https://github.com/python/cpython/issues/142787]: Fix assertion failure in `sqlite3` blob subscript when slicing with indices that result in an empty slice.

- gh-142352 [https://github.com/python/cpython/issues/142352]: Fix `asyncio.StreamWriter.start_tls()` to transfer buffered data from `StreamReader` to the SSL layer, preventing data loss when upgrading a connection to TLS mid-stream (e.g., when implementing PROXY protocol support).

- gh-139899 [https://github.com/python/cpython/issues/139899]: Introduced `importlib.abc.MetaPathFinder.discover()` and `importlib.abc.PathEntryFinder.discover()` to allow module and submodule name discovery without assuming the use of traditional filesystem based imports.

- gh-137335 [https://github.com/python/cpython/issues/137335]: Get rid of any possibility of a name conflict for named pipes in `multiprocessing` and `asyncio` on Windows, no matter how small.

- gh-135883 [https://github.com/python/cpython/issues/135883]: Fix `sqlite3`'s interactive shell keeping part of previous commands when scrolling history.

- gh-124748 [https://github.com/python/cpython/issues/124748]: Improve `TypeError` error message when `weakref.WeakKeyDictionary.update()` is used with keyword-only parameters.

- gh-80667 [https://github.com/python/cpython/issues/80667]: Add support for Tangut Ideographs names in `unicodedata`.

- bpo-42353 [https://bugs.python.org/issue?@action=redirect&bpo=42353]: The `re` module gains a new `re.prefixmatch()` function as an explicit spelling

of what has to date always been known as `re.match()`. `re.Pattern` similary gains a `re.Pattern.prefixmatch()` method.

Why? Explicit is better than implicit. Other widely used languages all use the term “match” to mean what Python uses the term “search” for. The unadorened “match” name in Python has been a frequent case of confusion and coding bugs due to the inconsistency with the rest if the software industry.

We do not plan to deprecate and remove the older `match` name.

- bpo-40243 [https://bugs.python.org/issue?@action=redirect&bpo=40243]: Fix **`unicodedata.ucd_3_2_0.numeric()`** for non-decimal values.

- bpo-40212 [https://bugs.python.org/issue?@action=redirect&bpo=40212]: Re-enable `os.posix_fallocate()` and `os.posix_fadvise()` on AIX.

- bpo-3405 [https://bugs.python.org/issue?@action=redirect&bpo=3405]: Add support for user data of Tk virtual events and detail for `Enter`, `Leave`, `FocusIn`, `FocusOut`, and `ConfigureRequest` events to `tkinter`.

- bpo-32234 [https://bugs.python.org/issue?@action=redirect&bpo=32234]: `mailbox.Mailbox` instances can now be used as a context manager. The Mailbox is locked on context entry and unlocked and closed at context exit.

Documentação

- gh-145450 [https://github.com/python/cpython/issues/145450]: Document missing public `wave.Wave_write` getter methods.
- gh-110937 [https://github.com/python/cpython/issues/110937]: Document rest of full public `importlib.metadata.Distribution` API. Also add the (already documented) `PackagePath` to `__all__`.
- gh-136246 [https://github.com/python/cpython/issues/136246]: A new “Improve this page” link is available in the left-hand sidebar of the docs, offering links to create GitHub issues, discussion forum posts, or pull requests.

Núcleo e embutidos

- gh-145701 [https://github.com/python/cpython/issues/145701]: Fix **`SystemError`** when `__classdict__` or `__conditional_annotations__` is in a class-scope inlined comprehension. Found by OSS Fuzz in #491105000 [https://issues.oss-fuzz.com/issues/491105000].

- gh-145615 [https://github.com/python/cpython/issues/145615]: Fixed a memory leak in the free-threaded build where mimalloc pages could become permanently unreclaimable until the owning thread exited.

- gh-116738 [https://github.com/python/cpython/issues/116738]: Make **`mmap.mmap.set_name()`** thread-safe on the free threaded build.

- gh-145566 [https://github.com/python/cpython/issues/145566]: In the free threading build, skip the stop-the-world pause when reassigning `__class__` on a newly created object.

- gh-143055 [https://github.com/python/cpython/issues/143055]: Fix crash in AST unparser when unparsing dict comprehension unpacking. Found by OSS Fuzz in #489790200 [https://issues.oss-fuzz.com/issues/489790200].

- gh-145335 [https://github.com/python/cpython/issues/145335]: Fix a crash in **`os.pathconf()`** when called with `-1` as the path argument.

- gh-145376 [https://github.com/python/cpython/issues/145376]: Fix reference leaks in various unusual error scenarios.

- gh-145234 [https://github.com/python/cpython/issues/145234]: Fixed a `SystemError` in the parser when an encoding cookie (for example, UTF-7) decodes to carriage returns (`\r`). Newlines are now normalized after decoding in the string tokenizer.

  Patch by Pablo Galindo.

- gh-145275 [https://github.com/python/cpython/issues/145275]: Added the **`-X pathconfig_warnings`** and **`PYTHON_PATHCONFIG_WARNINGS`** options, allowing to disable warnings from A inicialização do caminho de pesquisa de módulos sys.path.

- gh-145273 [https://github.com/python/cpython/issues/145273]: A warning is now shown during A inicialização do caminho de pesquisa de módulos sys.path if it can't find a valid standard library.

- gh-145241 [https://github.com/python/cpython/issues/145241]: Specialized the parser error for when `with` items are followed by a trailing comma (for example, `with item,:`), raising a clearer **`SyntaxError`** message. Patch by Pablo Galindo and Bartosz Sławecki.

- gh-130555 [https://github.com/python/cpython/issues/130555]: Fix use-after-free in **`dict.clear()`** when the dictionary values are embedded in an object and a destructor causes re-entrant mutation of the dictionary.

- gh-145197 [https://github.com/python/cpython/issues/145197]: Fix JIT trace crash when recording function from cleared generator frame.

- gh-145187 [https://github.com/python/cpython/issues/145187]: Fix compiler assertion fail when a type parameter bound contains an invalid expression in a conditional block.

- gh-145142 [https://github.com/python/cpython/issues/145142]: Fix a crash in the free-threaded build when the dictionary argument to **`str.maketrans()`** is concurrently modified.

- gh-145118 [https://github.com/python/cpython/issues/145118]: **`str.maketrans()`** now accepts **`frozendict`**.

- gh-144015 [https://github.com/python/cpython/issues/144015]: Speed up **`bytes.hex()`**, **`bytearray.hex()`**, **`binascii.hexlify()`**, and **`hashlib`** `.hexdigest()` operations with SIMD on x86-64, ARM64, and ARM32 with NEON when built with gcc (version 12 or higher) or clang (version 3 or higher) compilers. Around 1.1-3x faster for common 16-64 byte inputs such as hashlib hex digests, and up to 8x faster for larger data.

- gh-145118 [https://github.com/python/cpython/issues/145118]: **`type()`** now accepts **`frozendict`** as an argument.

- gh-145064 [https://github.com/python/cpython/issues/145064]: Fix JIT optimizer assertion failure during `CALL_ALLOC_AND_ENTER_INIT` side exit.

- gh-145055 [https://github.com/python/cpython/issues/145055]: **`exec()`** and **`eval()`** now accept **`frozendict`** for *globals*. Patch by Victor Stinner.

- gh-145058 [https://github.com/python/cpython/issues/145058]: Fix a crash when **`__lazy_import__()`** is passed a non-string argument, by raising an **`TypeError`** instead.

- gh-144995 [https://github.com/python/cpython/issues/144995]: Optimize **`memoryview`** comparison: a **`memoryview`** is equal to itself, there is no need to compare values. Patch by Victor Stinner.

- gh-141510 [https://github.com/python/cpython/issues/141510]: Update specializer to support frozendict. Patch by Donghee Na.

- gh-141510 [https://github.com/python/cpython/issues/141510]: Optimize **`frozendict.fromkeys()`** to avoid unnecessary thread-safety operations in frozendict cases. Patch by Donghee Na.

- gh-100239 [https://github.com/python/cpython/issues/100239]: Speedup `BINARY_OP_EXTEND` for exact floats and medium-size integers by up to 15%. Patch by Chris Eibl.

- gh-144914 [https://github.com/python/cpython/issues/144914]: Use `mimalloc` for raw memory allocations such as via **`PyMem_RawMalloc()`** for better performance on free-threaded builds. Patch by Kumar Aditya.

- gh-144872 [https://github.com/python/cpython/issues/144872]: Fix heap buffer overflow in the parser found by OSS-Fuzz.

- gh-144766 [https://github.com/python/cpython/issues/144766]: Fix a crash in fork child process when perf support is enabled.

- gh-144759 [https://github.com/python/cpython/issues/144759]: Fix undefined behavior in the lexer when `start` and `multi_line_start` pointers are `NULL` in `_PyLexer_remember_fstring_buffers()` and

`_PyLexer_restore_fstring_buffers()`. The `NULL` pointer arithmetic (`NULL - valid_pointer`) is now guarded with explicit `NULL` checks.

- gh-141510 [https://github.com/python/cpython/issues/141510]: Add built-in **`frozendict`** type. Patch by Victor Stinner.

- gh-144681 [https://github.com/python/cpython/issues/144681]: Fix a JIT assertion failure when a conditional branch jumps to the same target as the fallthrough path.

- gh-143300 [https://github.com/python/cpython/issues/143300]: Add **`PyUnstable_SetImmortal()`** C-API function to mark objects as immortal.

- gh-144702 [https://github.com/python/cpython/issues/144702]: Clarify the error message raised when a class pattern is used to match on a non-class object.

- gh-144569 [https://github.com/python/cpython/issues/144569]: Optimize `BINARY_SLICE` for list, tuple, and unicode by avoiding temporary `slice` object creation.

- gh-144438 [https://github.com/python/cpython/issues/144438]: Align the QSBR thread state array to a 64-byte cache line boundary to avoid false sharing in the free-threaded build.

- gh-142349 [https://github.com/python/cpython/issues/142349]: Implement **PEP 810** [https://peps.python.org/pep-0810/]. Patch by Pablo Galindo and Dino Viehland.

- gh-141226 [https://github.com/python/cpython/issues/141226]: Deprecate **PEP 456** [https://peps.python.org/pep-0456/] support for providing an external definition of the string hashing scheme. Removal is scheduled for Python 3.19. Patch by Bénédikt Tran.

- gh-138912 [https://github.com/python/cpython/issues/138912]: Improve **`MATCH_CLASS`** performance by up to 52% in certain cases. Patch by Marc Mueller.

- gh-130327 [https://github.com/python/cpython/issues/130327]: Fix erroneous clearing of an object's `__dict__` if overwritten at runtime.

- gh-80667 [https://github.com/python/cpython/issues/80667]: Literals using the `\N{name}` escape syntax can now construct CJK ideographs and Hangul syllables using case-insensitive names.

API C

- gh-142417 [https://github.com/python/cpython/issues/142417]: Restore private provisional `_Py_InitializeMain()` function removed in Python 3.14. Patch by Victor Stinner.

- gh-144748 [https://github.com/python/cpython/issues/144748]: `PyErr_CheckSignals()` now raises the exception scheduled by `PyThreadState_SetAsyncExc()`, if any.

- gh-144981 [https://github.com/python/cpython/issues/144981]: Made `PyUnstable_Code_SetExtra()`, `PyUnstable_Code_GetExtra()`, and `PyUnstable_Eval_RequestCodeExtraIndex()` thread-safe on the free threaded build.

- gh-141510 [https://github.com/python/cpython/issues/141510]: Add the following functions for the new `frozendict` type:

  - `PyAnyDict_Check()`
  - `PyAnyDict_CheckExact()`
  - `PyFrozenDict_Check()`
  - `PyFrozenDict_CheckExact()`
  - `PyFrozenDict_New()`

  Patch por Victor Stinner.

- gh-121617 [https://github.com/python/cpython/issues/121617]: `Python.h` now also includes `<string.h>` in the limited C API version 3.11 and newer to fix the `Py_CLEAR` macro which uses `memcpy()`. Patch by Victor Stinner.

- gh-144175 [https://github.com/python/cpython/issues/144175]: Add `PyArg_ParseArray()` and `PyArg_ParseArrayAndKeywords()` functions to parse arguments of functions using the `METH_FASTCALL` calling convention. Patch by Victor Stinner.

Construção

- gh-144533 [https://github.com/python/cpython/issues/144533]: Use wasmtime's `--argv0` to auto-discover sysconfig in WASI builds
- gh-145110 [https://github.com/python/cpython/issues/145110]: Fix targets "Clean" and "CLeanAll" in case of PGO builds on Windows. Patch by Chris Eibl.
- gh-144679 [https://github.com/python/cpython/issues/144679]: When building with Visual Studio 2026 (Version 18), use PlatformToolSet v145 by default. Patch by Chris Eibl.
- gh-144675 [https://github.com/python/cpython/issues/144675]: Update to WASI SDK 30.
- gh-136677 [https://github.com/python/cpython/issues/136677]: Introduce executable specific linker flags to `./configure`.

Python 3.15.0 alpha 6

Release date: 2026-02-11

macOS

- gh-144648 [https://github.com/python/cpython/issues/144648]: Allowed _remote_debugging to build on more OS versions by using proc_listpids() rather than proc_listallpids().
- gh-124111 [https://github.com/python/cpython/issues/124111]: Update macOS installer to use Tcl/Tk 9.0.3.
- gh-144551 [https://github.com/python/cpython/issues/144551]: Atualiza o instalador do macOS para usar OpenSSL 3.5.5.

Windows

- gh-80620 [https://github.com/python/cpython/issues/80620]: Support negative timestamps in **`time.gmtime()`**, **`time.localtime()`**, and various **`datetime`** functions.

Testes

- gh-144415 [https://github.com/python/cpython/issues/144415]: The Android testbed now distinguishes between stdout/stderr messages which were triggered by a newline, and those triggered by a manual call to `flush`. This fixes logging of progress indicators and similar content.
- gh-65784 [https://github.com/python/cpython/issues/65784]: Add support for parametrized resource `wantobjects` in regrtests, which allows to run Tkinter tests with the specified value of **`tkinter.wantobjects`**, for example `-u wantobjects=0`.

Segurança

- gh-144125 [https://github.com/python/cpython/issues/144125]: **`BytesGenerator`** will now refuse to serialize (write) headers that are unsafely folded or delimited; see **`verify_generated_headers`**. (Contributed by Bas Bloemsaat and Petr Viktorin in gh-121650 [https://github.com/python/cpython/issues/121650]).
- gh-143935 [https://github.com/python/cpython/issues/143935]: Fixed a bug in the folding of comments when flattening an email message using a modern email policy. Comments consisting of a very long sequence of non-foldable characters could trigger a forced line wrap that omitted the required leading space on the continuation line, causing the remainder of the comment to be interpreted as a new header field. This enabled header injection with carefully crafted inputs.
- gh-143925 [https://github.com/python/cpython/issues/143925]: Reject control characters in `data:` URL media types.
- gh-143923 [https://github.com/python/cpython/issues/143923]: Reject control characters in POP3 commands.
- gh-143921 [https://github.com/python/cpython/issues/143921]: Reject control characters in IMAP commands.
- gh-143919 [https://github.com/python/cpython/issues/143919]: Reject control characters in **`http.cookies.Morsel`** fields and values.

- gh-143916 [https://github.com/python/cpython/issues/143916]: Reject C0 control characters within wsgiref.headers.Headers fields, values, and parameters.

Biblioteca

- gh-144538 [https://github.com/python/cpython/issues/144538]: Bump the version of pip bundled in ensurepip to version 26.0.1

- gh-144493 [https://github.com/python/cpython/issues/144493]: Improve an exception error message in `_overlapped.BindLocal()` that is raised when `asyncio.loop.sock_connect()` is called on a `asyncio.ProactorEventLoop` with a socket that has an invalid address family.

- gh-144386 [https://github.com/python/cpython/issues/144386]: Add support for arbitrary descriptors **`__enter__()`**, **`__exit__()`**, **`__aenter__()`**, and **`__aexit__()`** in `contextlib.ExitStack` and `contextlib.AsyncExitStack`, for consistency with the `with` and `async with` statements.

- gh-123471 [https://github.com/python/cpython/issues/123471]: Make concurrent iteration over `itertools.combinations_with_replacement` and `itertools.permutations` safe under free-threading.

- gh-74453 [https://github.com/python/cpython/issues/74453]: Deprecate `os.path.commonprefix()` in favor of `os.path.commonpath()` for path segment prefixes.

  The `os.path.commonprefix()` function is being deprecated due to having a misleading name and module. The function is not safe to use for path prefixes despite being included in a module about path manipulation, meaning it is easy to accidentally introduce path traversal vulnerabilities into Python programs by using this function.

- gh-144380 [https://github.com/python/cpython/issues/144380]: Improve performance of `io.BufferedReader` line iteration by ~49%.

- gh-144363 [https://github.com/python/cpython/issues/144363]: Update bundled libexpat [https://libexpat.github.io/] to 2.7.4

- gh-140824 [https://github.com/python/cpython/issues/140824]: When **`faulthandler`** dumps the list of third-party extension modules, ignore sub-modules of stdlib packages. Patch by Victor Stinner.

- gh-144206 [https://github.com/python/cpython/issues/144206]: Improve error messages for buffer overflow in **`fcntl.fcntl()`** and **`fcntl.ioctl()`**.

- gh-144264 [https://github.com/python/cpython/issues/144264]: Speed up Base64 decoding of data containing ignored characters (both in non-strict mode and with an explicit *ignorechars* argument). It is now up to 2 times faster for multiline Base64 data.

- gh-144249 [https://github.com/python/cpython/issues/144249]: Add filename context to **`OSError`** exceptions raised by **`ssl.SSLContext.load_cert_chain()`**, allowing users to have more context.

- gh-132888 [https://github.com/python/cpython/issues/132888]: Fix incorrect use of **`ctypes.GetLastError()`** and add missing error checks for Windows API calls in **`_pyrepl.windows_console`**.

- gh-142956 [https://github.com/python/cpython/issues/142956]: Updated **`tomllib`** to parse TOML 1.1.0.

- gh-144217 [https://github.com/python/cpython/issues/144217]: **`mimetypes`**: Add support for DICOM files (for medical imaging) with the official MIME type `application/dicom`. Patch by Benedikt Johannes.

- gh-144212 [https://github.com/python/cpython/issues/144212]: Mime type `image/jxl` is now supported by **`mimetypes`**.

- gh-143594 [https://github.com/python/cpython/issues/143594]: Add **`symtable.Function.get_cells()`** and **`symtable.Symbol.is_cell()`** methods.

- gh-144169 [https://github.com/python/cpython/issues/144169]: Fix three crashes when non-string keyword arguments are supplied to objects in the `ast` module.

- gh-144128 [https://github.com/python/cpython/issues/144128]: Fix a crash in `array.array.fromlist()` when an element's `__index__()` method mutates the input list during conversion.

- gh-144100 [https://github.com/python/cpython/issues/144100]: Fixed a crash in ctypes when using a deprecated `POINTER(str)` type in `argtypes`. Instead of aborting, ctypes now raises a proper Python exception when the pointer target type is unresolved.

- gh-143658 [https://github.com/python/cpython/issues/143658]: `importlib.metadata`: Use `str.lower()` and `str.replace()` to further improve performance of **`importlib.metadata.Prepared.normalize()`**. Patch by Hugo van Kemenade and Henry Schreiner.

- gh-144050 [https://github.com/python/cpython/issues/144050]: Fix `stat.filemode()` in the pure-Python implementation to avoid misclassifying invalid mode values as block devices.

- gh-83069 [https://github.com/python/cpython/issues/83069]: `subprocess.Popen.wait()`: when `timeout` is not `None`, an efficient event-driven mechanism now waits for process termination, if available. Linux >= 5.3 uses `os.pidfd_open()` + `select.poll()`. macOS and other BSD variants use `select.kqueue()` + `KQ_FILTER_PROC` + `KQ_NOTE_EXIT`. Windows keeps using `WaitForSingleObject` (unchanged). If none of these mechanisms are available, the function falls back to the traditional busy loop (non-blocking call and short sleeps). Patch by Giampaolo Rodola.

- gh-144030 [https://github.com/python/cpython/issues/144030]: The Python implementation of `functools.lru_cache()` differed from the default C implementation in that it did not check that its argument is callable. This discrepancy is now fixed and both raise a `TypeError`.

- gh-144001 [https://github.com/python/cpython/issues/144001]: Added the *ignorechars* parameter in `binascii.a2b_base64()` and `base64.b64decode()`.

- gh-144023 [https://github.com/python/cpython/issues/144023]: Fixed validation of file descriptor 0 in posix functions when used with follow_symlinks parameter.

- gh-143999 [https://github.com/python/cpython/issues/143999]: Fix an issue where `inspect.getgeneratorstate()` and `inspect.getcoroutinestate()` could fail for generators wrapped by `types.coroutine()` in the suspended state.

- gh-143952 [https://github.com/python/cpython/issues/143952]: Fixed `asyncio` debugging tools to work with new remote debugging API. Patch by Bartosz Sławecki.

- gh-143904 [https://github.com/python/cpython/issues/143904]: `struct.pack_into()` now raises OverflowError instead of IndexError for too large *offset* argument.

- gh-143897 [https://github.com/python/cpython/issues/143897]: Remove the `isxidstart()` and `isxidcontinue()` methods of `unicodedata.ucd_3_2_0`. They are now only exposed as `unicodedata.isxidstart()` and `unicodedata.isxidcontinue()`.

- gh-143831 [https://github.com/python/cpython/issues/143831]: `annotationlib.ForwardRef` objects are now hashable when created from annotation scopes with closures. Previously, hashing such objects would throw an exception. Patch by Bartosz Sławecki.

- gh-143874 [https://github.com/python/cpython/issues/143874]: Fixed a bug in `pdb` where expression results were not sent back to remote client.

- gh-143754 [https://github.com/python/cpython/issues/143754]: Add new `tkinter` widget methods `pack_content()`, `place_content()` and `grid_content()` which are alternative spelling of old `*_slaves()` methods.

- gh-143756 [https://github.com/python/cpython/issues/143756]: Fix potential thread safety issues in `ssl` module.

- gh-132604 [https://github.com/python/cpython/issues/132604]: Previously, `Protocol` classes that were not decorated with `@~typing.runtime_checkable`, but that inherited from another `Protocol` class that did have this decorator, could be used in `isinstance()` and `issubclass()` checks. This behavior is now deprecated and such checks will throw a `TypeError` in Python 3.20. Patch by Bartosz Sławecki.

- gh-143543 [https://github.com/python/cpython/issues/143543]: Fix a crash in itertools.groupby that could occur when a user-defined `__eq__()` method re-enters the iterator during key comparison.

- gh-143689 [https://github.com/python/cpython/issues/143689]: Fix `io.BufferedReader.read1()` state cleanup on buffer allocation failure.

- gh-143602 [https://github.com/python/cpython/issues/143602]: Fix a inconsistency issue in `write()` that leads to unexpected buffer overwrite by deduplicating the buffer exports.

- gh-142434 [https://github.com/python/cpython/issues/142434]: Use `ppoll()` if available in `select.poll()` to have a timeout resolution of 1 nanosecond, instead of a resolution of 1 ms. Patch by Victor Stinner.

- gh-140557 [https://github.com/python/cpython/issues/140557]: `array.array` buffers now have the same alignment when empty as when allocated. Unaligned buffers can still be created by slicing.

- gh-143423 [https://github.com/python/cpython/issues/143423]: Fix free-threaded build detection in the sampling profiler when Py_GIL_DISABLED is set to 0.

- gh-101178 [https://github.com/python/cpython/issues/101178]: Add Ascii85, Base85, and Z85 support to `binascii` and improve the performance of the base-85 converters in `base64`.

- gh-142966 [https://github.com/python/cpython/issues/142966]: Fix **`ctypes.POINTER.set_type()`** not updating the format string to match the type.

- gh-142555 [https://github.com/python/cpython/issues/142555]: **`array`**: fix a crash in `a[i] = v` when converting *i* to an index via **`i.__index__`** or **`i.__float__`** mutates the array.

- gh-142438 [https://github.com/python/cpython/issues/142438]: Fix _decimal builds configured with EXTRA_FUNCTIONALITY by correcting the Context.apply wrapper to pass the right argument.

- gh-141860 [https://github.com/python/cpython/issues/141860]: Add an `on_error` keyword-only parameter to **`multiprocessing.set_forkserver_preload()`** to control how import failures during module preloading are handled. Accepts `'ignore'` (default, silent), `'warn'` (emit **`ImportWarning`**), or `'fail'` (raise exception). Contributed by Nick Neumann and Gregory P. Smith.

- gh-125346 [https://github.com/python/cpython/issues/125346]: Accepting `+` and `/` characters with an alternative alphabet in **`base64.b64decode()`** and **`base64.urlsafe_b64decode()`** is now deprecated. In future Python versions they will be errors in the strict mode and discarded in the non-strict mode.

- gh-140715 [https://github.com/python/cpython/issues/140715]: Add `'%F'` support to **`strptime()`**.

- gh-67041 [https://github.com/python/cpython/issues/67041]: Add the *missing_as_none* parameter to **`urlparse()`**, **`urlsplit()`** and **`urldefrag()`** functions. Add the *keep_empty* parameter to **`urlunparse()`** and **`urlunsplit()`** functions. This allows to distinguish between empty and not defined URI components and preserve empty components.

- gh-77188 [https://github.com/python/cpython/issues/77188]: The **`pickle`** module now properly handles name-mangled private methods.

IDLE

- gh-143774 [https://github.com/python/cpython/issues/143774]: Better explain the operation of Format / Format Paragraph.

Núcleo e embutidos

- gh-134584 [https://github.com/python/cpython/issues/134584]: Optimize and eliminate ref-counting in `_BINARY_OP_SUBSCR_LIST_SLICE`
- gh-144563 [https://github.com/python/cpython/issues/144563]: Fix interaction of the Tachyon profiler and **`ctypes`** and other modules that load the Python shared library (if present) in an independent map as this was causing the mechanism that loads the binary information to be confused. Patch by Pablo Galindo
- gh-144601 [https://github.com/python/cpython/issues/144601]: Fix crash when importing a module whose `PyInit` function raises an exception from a subinterpreter.
- gh-144549 [https://github.com/python/cpython/issues/144549]: Fix building the tail calling interpreter on Visual Studio 2026 with free-threading.
- gh-144513 [https://github.com/python/cpython/issues/144513]: Fix potential deadlock when using critical sections during stop-the-world pauses in the free-threaded build.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimise `_GUARD_TOS_SLICE` in the JIT.
- gh-144330 [https://github.com/python/cpython/issues/144330]: Move `classmethod` and `staticmethod` initialization from `__init__()` to `__new__()`. Patch by Victor Stinner.
- gh-144446 [https://github.com/python/cpython/issues/144446]: Fix data races in the free-threaded build when reading frame object attributes while another thread is executing the frame.
- gh-120321 [https://github.com/python/cpython/issues/120321]: Add `gi_state`, `cr_state`, and `ag_state` attributes to generators, coroutines, and async generators that return the current state as a string (e.g., `GEN_RUNNING`). The **`inspect`** module functions **`getgeneratorstate()`**, **`getcoroutinestate()`**, and **`getasyncgenstate()`** now return these attributes directly.

- gh-141563 [https://github.com/python/cpython/issues/141563]: Fix thread safety of **`PyDateTime_IMPORT`**.
- gh-144280 [https://github.com/python/cpython/issues/144280]: Fix a bug in JIT where the predicate symbol had no truthiness
- gh-140550 [https://github.com/python/cpython/issues/140550]: In **`PyModuleDef.m_slots`**, allow slots that repeat information present in **`PyModuleDef`**.
- gh-139103 [https://github.com/python/cpython/issues/139103]: Improve scaling of **`namedtuple()`** instantiation in the free-threaded build.
- gh-144307 [https://github.com/python/cpython/issues/144307]: Prevent a reference leak in module teardown at interpreter finalization.
- gh-144319 [https://github.com/python/cpython/issues/144319]: Add huge pages support for the pymalloc allocator. Patch by Pablo Galindo
- gh-120321 [https://github.com/python/cpython/issues/120321]: Made `gi_yieldfrom` thread-safe in the free-threading build by using a lightweight lock on the frame state.
- gh-144194 [https://github.com/python/cpython/issues/144194]: Fix error handling in perf jitdump initialization on memory allocation failure.
- gh-143962 [https://github.com/python/cpython/issues/143962]: Name suggestion for not normalized name suggests now the normalized name or the closest name to the normalized name. If the suggested name is not ASCII, include also its ASCII representation.
- gh-144157 [https://github.com/python/cpython/issues/144157]: **`bytes.translate()`** now allows the compiler to unroll its loop more usefully for a 2x speedup in the common no-deletions specified case.
- gh-144068 [https://github.com/python/cpython/issues/144068]: Fix JIT tracer memory leak, ensure the JIT tracer state is freed when daemon threads are cleaned up during interpreter shutdown.
- gh-144012 [https://github.com/python/cpython/issues/144012]: Check if the result is `NULL` in `BINARY_OP_EXTENT` opcode.
- gh-144007 [https://github.com/python/cpython/issues/144007]: Eliminate redundant refcounting in the JIT for `BINARY_OP`.
- gh-144005 [https://github.com/python/cpython/issues/144005]: Eliminate redundant refcounting from `BINARY_OP_EXTEND`.
- gh-143939 [https://github.com/python/cpython/issues/143939]: Fix erroneous “cannot reuse already awaited coroutine” error that could occur when a generator was run during the process of clearing a coroutine’s frame.

- gh-141805 [https://github.com/python/cpython/issues/141805]: Fix crash in `set` when objects with the same hash are concurrently added to the set after removing an element with the same hash while the set still contains elements with the same hash.
- gh-143670 [https://github.com/python/cpython/issues/143670]: Fixes a crash in `ga_repr_items_list` function.
- gh-143650 [https://github.com/python/cpython/issues/143650]: Fix race condition in `importlib` where a thread could receive a stale module reference when another thread's import fails.
- gh-143569 [https://github.com/python/cpython/issues/143569]: Generator expressions in 3.15 now conform to the documented behavior when the iterable does not support iteration. This matches the behavior in 3.14 and earlier
- gh-143192 [https://github.com/python/cpython/issues/143192]: Improve performance of bitwise operations on multi-digit ints.
- gh-132657 [https://github.com/python/cpython/issues/132657]: If we are specializing to `LOAD_GLOBAL_MODULE` or `LOAD_ATTR_MODULE`, try to enable deferred reference counting for the value, if the object is owned by a different thread. This applies to the free-threaded build only and should improve scaling of multi-threaded programs. Note that when deferred reference counting is enabled, the object will be deallocated by the GC, rather than by `Py_DECREF()`.
- gh-143055 [https://github.com/python/cpython/issues/143055]: Implement **PEP 798** [https://peps.python.org/pep-0798/] (Unpacking in Comprehensions). Patch by Adam Hartz.
- gh-142037 [https://github.com/python/cpython/issues/142037]: Improve error messages for printf-style formatting. For errors in the format string, always include the position of the start of the format unit. For errors related to the formatted arguments, always include the number or the name of the argument. Raise more specific errors and include more information (type and number of arguments, most probable causes of error).
- gh-140557 [https://github.com/python/cpython/issues/140557]: `bytearray` buffers now have the same alignment when empty as when allocated. Unaligned buffers can still be created by slicing.
- gh-140232 [https://github.com/python/cpython/issues/140232]: Frozenset objects with immutable elements are no longer tracked by the garbage

collector.
- gh-115231 [https://github.com/python/cpython/issues/115231]: Setup `__module__` attribute for built-in static methods. Patch by Sergey B Kirpichev.

API C

- gh-143869 [https://github.com/python/cpython/issues/143869]: Added **`PyLong_GetNativeLayout()`**, **`PyLongLayout`**, **`PyLongExport`**, **`PyLong_Export()`**, **`PyLong_FreeExport()`**, **`PyLongWriter`**, **`PyLongWriter_Create()`**, **`PyLongWriter_Finish()`** and **`PyLongWriter_Discard()`** to the limited API.
- gh-141070 [https://github.com/python/cpython/issues/141070]: Renamed **`PyUnstable_Object_Dump()`** to **`PyObject_Dump()`**.

Construção

- gh-140421 [https://github.com/python/cpython/issues/140421]: Disable the perf trampoline on older macOS versions where it cannot be built.
- gh-144309 [https://github.com/python/cpython/issues/144309]: Build Python with POSIX 2024, instead of POSIX 2008. Patch by Victor Stinner.
- gh-144278 [https://github.com/python/cpython/issues/144278]: Enables defining the `_PY_IMPL_NAME` and `_PY_IMPL_CACHE_TAG` preprocessor definitions to override **`sys.implementation`** at build time. Definitions need to include quotes when setting to a string literal. Setting the cache tag to `NULL` has the effect of completely disabling automatic creation and use of `.pyc` files.
- gh-143960 [https://github.com/python/cpython/issues/143960]: Add support for OpenSSL 3.6, drop EOL 3.2. Patch by Hugo van Kemenade.
- gh-143941 [https://github.com/python/cpython/issues/143941]: Move WASI-related files to `Platforms/WASI`. Along the way, leave a deprecated `Tools/wasm/wasi/__main__.py` behind for backwards-compatibility.
- gh-143842 [https://github.com/python/cpython/issues/143842]: Prevent static builds from clashing with curses by making the optimizer COLORS table static.

Python 3.15.0 alpha 5

Release date: 2026-01-14

Windows

- gh-143082 [https://github.com/python/cpython/issues/143082]: Fix **`pdb`** arrow key history not working when `stdin` is `sys.stdin`.
- gh-128067 [https://github.com/python/cpython/issues/128067]: Fix a bug in PyREPL on Windows where output without a trailing newline was overwritten by the next prompt.

Ferramentas/Demos

- gh-142095 [https://github.com/python/cpython/issues/142095]: Make gdb ‘py-bt’ command use frame from thread local state when available. Patch by Sam Gross and Victor Stinner.

Testes

- gh-143460 [https://github.com/python/cpython/issues/143460]: Skip tests relying on infinite recusion if stack size is unlimited.
- gh-143553 [https://github.com/python/cpython/issues/143553]: Add support for parametrized resources, such as `-u xpickle=2.7`.
- bpo-31391 [https://bugs.python.org/issue?@action=redirect&bpo=31391]: Forward-port test_xpickle from Python 2 to Python 3 and add the resource back to test’s command line.

Biblioteca

- gh-143706 [https://github.com/python/cpython/issues/143706]: Fix **`multiprocessing`** forkserver so that **`sys.argv`** is correctly set before `__main__` is preloaded. Previously, **`sys.argv`** was empty during main module import in forkserver child processes. This fixes a regression introduced in 3.13.8 and 3.14.1. Root caused by Aaron Wieczorek, test provided by Thomas Watson, thanks!
- gh-143638 [https://github.com/python/cpython/issues/143638]: Forbid reentrant calls of the **`pickle.Pickler`** and **`pickle.Unpickler`** methods for the C implementation. Previously, this could cause crash or data corruption,

now concurrent calls of methods of the same object raise **`RuntimeError`**.

- gh-143658 [https://github.com/python/cpython/issues/143658]: **`importlib.metadata`**: Use **`str.translate()`** to improve performance of **`importlib.metadata.Prepared.normalize()`**. Patch by Hugo van Kemenade and Henry Schreiner.
- gh-78724 [https://github.com/python/cpython/issues/78724]: Raise **`RuntimeError`**'s when user attempts to call methods on half-initialized **`Struct`** objects, For example, created by `Struct.__new__(Struct)`. Patch by Sergey B Kirpichev.
- gh-143196 [https://github.com/python/cpython/issues/143196]: Fix crash when the internal encoder object returned by undocumented function **`json.encoder.c_make_encoder()`** was called with non-zero second (*_current_indent_level*) argument.
- gh-143191 [https://github.com/python/cpython/issues/143191]: **`_thread.stack_size()`** now raises **`ValueError`** if the stack size is too small. Patch by Victor Stinner.
- gh-143547 [https://github.com/python/cpython/issues/143547]: Fix **`sys.unraisablehook()`** when the hook raises an exception and changes **`sys.unraisablehook()`**: hold a strong reference to the old hook. Patch by Victor Stinner.
- gh-139686 [https://github.com/python/cpython/issues/139686]: Revert 0a97941245f1dda6d838f9aaf0512104e5253929 and 57db12514ac686f0a752ec8fe1c08b6daa0c6219 which made importlib.reload a no-op for lazy modules; caused Buildbot failures.
- gh-143517 [https://github.com/python/cpython/issues/143517]: **`annotationlib.get_annotations()`** no longer raises a **`SyntaxError`** when evaluating a stringified starred annotation that starts with one or more whitespace characters followed by a `*`. Patch by Bartosz Sławecki.
- gh-143474 [https://github.com/python/cpython/issues/143474]: Add **`os.RWF_ATOMIC`** constant for Linux 6.11+.
- gh-143445 [https://github.com/python/cpython/issues/143445]: Speed up **`copy.deepcopy()`** by 1.04x.
- gh-143378 [https://github.com/python/cpython/issues/143378]: Fix use-after-free crashes when a **`BytesIO`** object is concurrently mutated during **`write()`** or **`writelines()`**.

- gh-143368 [https://github.com/python/cpython/issues/143368]: Fix endless retry loop in `profiling.sampling` blocking mode when threads cannot be seized due to `EPERM`. Such threads are now skipped instead of causing repeated error messages. Patch by Pablo Galindo.
- gh-143346 [https://github.com/python/cpython/issues/143346]: Fix incorrect wrapping of the Base64 data in `plistlib._PlistWriter` when the indent contains a mix of tabs and spaces.
- gh-140025 [https://github.com/python/cpython/issues/140025]: `queue`: Fix `SimpleQueue.__sizeof__()` computation.
- gh-143310 [https://github.com/python/cpython/issues/143310]: `tkinter`: fix a crash when a Python `list` is mutated during the conversion to a Tcl object (e.g., when setting a Tcl variable). Patch by Bénédikt Tran.
- gh-143309 [https://github.com/python/cpython/issues/143309]: Fix a crash in `os.execve()` on non-Windows platforms when given a custom environment mapping which is then mutated during parsing. Patch by Bénédikt Tran.
- gh-143308 [https://github.com/python/cpython/issues/143308]: `pickle`: fix use-after-free crashes when a `PickleBuffer` is concurrently mutated by a custom buffer callback during pickling. Patch by Bénédikt Tran and Aaron Wieczorek.
- gh-142939 [https://github.com/python/cpython/issues/142939]: Performance optimisations for `difflib.get_close_matches()`
- gh-124951 [https://github.com/python/cpython/issues/124951]: The base64 implementation behind the `binascii`, `base64`, and related codec has been optimized for modern pipelined CPU architectures and now performs 2-3x faster across all platforms.
- gh-143237 [https://github.com/python/cpython/issues/143237]: Fix support of named pipes in the rotating `logging` handlers.
- gh-143249 [https://github.com/python/cpython/issues/143249]: Fix possible buffer leaks in Windows overlapped I/O on error handling.
- gh-143241 [https://github.com/python/cpython/issues/143241]: `zoneinfo`: fix infinite loop in `ZoneInfo.from_file` when parsing a malformed TZif file. Patch by Fatih Celik.
- gh-142830 [https://github.com/python/cpython/issues/142830]: `sqlite3`: fix use-after-free crashes when the connection's callbacks are mutated during a callback execution. Patch by Bénédikt Tran.

- gh-143200 [https://github.com/python/cpython/issues/143200]: `xml.etree.ElementTree`: fix use-after-free crashes in `__getitem__()` and `__setitem__()` methods of `Element` when the element is concurrently mutated. Patch by Bénédikt Tran.
- gh-143214 [https://github.com/python/cpython/issues/143214]: Add the *wrapcol* parameter in `binascii.b2a_base64()` and `base64.b64encode()`.
- gh-142195 [https://github.com/python/cpython/issues/142195]: Updated timeout evaluation logic in `subprocess` to be compatible with deterministic environments like Shadow where time moves exactly as requested.
- gh-140739 [https://github.com/python/cpython/issues/140739]: Fix several crashes due to reading invalid memory in the new Tachyon sampling profiler. Patch by Pablo Galindo.
- gh-142164 [https://github.com/python/cpython/issues/142164]: Fix the ctypes bitfield overflow error message to report the correct offset and size calculation.
- gh-143145 [https://github.com/python/cpython/issues/143145]: Fixed a possible reference leak in ctypes when constructing results with multiple output parameters on error.
- gh-143103 [https://github.com/python/cpython/issues/143103]: Add padding support to `base64.z85encode()` via the `pad` parameter.
- gh-130796 [https://github.com/python/cpython/issues/130796]: Undeprecate the `locale.getdefaultlocale()` function. Patch by Victor Stinner.
- gh-74902 [https://github.com/python/cpython/issues/74902]: Add the `iter_graphemes()` function in the `unicodedata` module to iterate over grapheme clusters according to rules defined in Unicode Standard Annex #29, "Unicode Text Segmentation" [https://www.unicode.org/reports/tr29/]. Add `grapheme_cluster_break()`, `indic_conjunct_break()` and `extended_pictographic()` functions to get the properties of the character which are related to the above algorithm.
- gh-143004 [https://github.com/python/cpython/issues/143004]: Fix a potential use-after-free in `collections.Counter.update()` when user code mutates the Counter during an update.
- gh-140648 [https://github.com/python/cpython/issues/140648]: The `asyncio` REPL now respects the `-I` flag (isolated mode). Previously, it would load and execute `PYTHONSTARTUP` even if the flag was set. Contributed by Bartosz Sławecki.

- gh-142991 [https://github.com/python/cpython/issues/142991]: Fixed socket operations such as recvfrom() and sendto() for FreeBSD divert(4) socket.
- gh-116738 [https://github.com/python/cpython/issues/116738]: Make the attributes in `lzma` thread-safe on the free threaded build.
- gh-142950 [https://github.com/python/cpython/issues/142950]: Fix regression in `argparse` where format specifiers in help strings raised `ValueError`.
- gh-142881 [https://github.com/python/cpython/issues/142881]: Fix concurrent and reentrant call of `atexit.unregister()`.
- gh-142615 [https://github.com/python/cpython/issues/142615]: Fix possible crashes when initializing `asyncio.Task` or `asyncio.Future` multiple times. These classes can now be initialized only once and any subsequent initialization attempt will raise a RuntimeError. Patch by Kumar Aditya.
- gh-142517 [https://github.com/python/cpython/issues/142517]: The non-`compat32` `email` policies now correctly handle refolding encoded words that contain bytes that can not be decoded in their specified character set. Previously this resulted in an encoding exception during folding.
- gh-138122 [https://github.com/python/cpython/issues/138122]: The Tachyon profiler's live TUI now integrates with the experimental `_colorize` theming system. Users can customize colors via `_colorize.set_theme()` (experimental API, subject to change). A `LiveProfilerLight` theme is provided for light terminal backgrounds. Patch by Pablo Galindo.
- gh-142306 [https://github.com/python/cpython/issues/142306]: Improve errors for `Element.remove`.
- gh-63016 [https://github.com/python/cpython/issues/63016]: Add a `flags` parameter to `mmap.mmap.flush()` to control synchronization behavior.
- gh-139262 [https://github.com/python/cpython/issues/139262]: Some keystrokes can be swallowed in the new `PyREPL` on Windows, especially when used together with the ALT key. Fix by Chris Eibl.
- gh-138897 [https://github.com/python/cpython/issues/138897]: Improved `license`/`copyright`/`credits` display in the REPL: now uses a pager.
- gh-135852 [https://github.com/python/cpython/issues/135852]: Add `_winapi.RegisterEventSource()`, `_winapi.DeregisterEventSource()` and `_winapi.ReportEvent()`. Using these functions in `NTEventLogHandler` to replace `pywin32`.

- gh-109263 [https://github.com/python/cpython/issues/109263]: Starting a process from spawn context in **`multiprocessing`** no longer sets the start method globally.
- gh-132715 [https://github.com/python/cpython/issues/132715]: Skip writing objects during marshalling once a failure has occurred.

Documentação

- gh-140806 [https://github.com/python/cpython/issues/140806]: Add documentation for **`enum.bin()`**.

Núcleo e embutidos

- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_CONTAINS_OP`, `_CONTAINS_OP_SET` and `_CONTAINS_OP_DICT`.
- gh-143604 [https://github.com/python/cpython/issues/143604]: Fix a reference counting issue in the JIT tracer where the current executor could be prematurely freed during tracing.
- gh-143469 [https://github.com/python/cpython/issues/143469]: Enable **`LOAD_ATTR_MODULE`** specialization even if **`__getattr__()`** is defined in module.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `TO_BOOL_STR`.
- gh-143377 [https://github.com/python/cpython/issues/143377]: Fix a crash in **`_interpreters.capture_exception()`** when the exception is incorrectly formatted. Patch by Bénédikt Tran.
- gh-139757 [https://github.com/python/cpython/issues/139757]: Add `BINARY_OP_SUBSCR_USTR_INT` to specialize reading an ASCII character from any string. Patch by Chris Eibl.
- gh-141504 [https://github.com/python/cpython/issues/141504]: Factor out tracing and optimization heuristics into a single object. Patch by Donghee Na.
- gh-142982 [https://github.com/python/cpython/issues/142982]: Specialize **`CALL_FUNCTION_EX`** for Python and non-Python callables.
- gh-136924 [https://github.com/python/cpython/issues/136924]: The interactive help mode in the REPL no longer incorrectly syntax highlights text input as Python code. Contributed by Olga Matoula.

- gh-139757 [https://github.com/python/cpython/issues/139757]: Fix unintended bytecode specialization for non-ascii string. Patch by Donghee Na, Ken Jin and Chris Eibl.
- gh-143361 [https://github.com/python/cpython/issues/143361]: Add `PY_VECTORCALL_ARGUMENTS_OFFSET` to `_Py_CallBuiltinClass_StackRefSteal` to avoid redundant allocations
- gh-131798 [https://github.com/python/cpython/issues/131798]: The JIT optimizer now understands more generator instructions.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_LOAD_ATTR_SLOT`.
- gh-143189 [https://github.com/python/cpython/issues/143189]: Fix crash when inserting a non-**`str`** key into a split table dictionary when the key matches an existing key in the split table but has no corresponding value in the dict.
- gh-143228 [https://github.com/python/cpython/issues/143228]: Fix use-after-free in perf trampoline when toggling profiling while threads are running or during interpreter finalization with daemon threads active. The fix uses reference counting to ensure trampolines are not freed while any code object could still reference them. Pach by Pablo Galindo
- gh-142664 [https://github.com/python/cpython/issues/142664]: Fix a use-after-free crash in **`memoryview.__hash__`** when the `__hash__` method of the referenced object mutates that object or the view. Patch by Bénédikt Tran.
- gh-142557 [https://github.com/python/cpython/issues/142557]: Fix a use-after-free crash in bytearray.__mod__ when the **`bytearray`** is mutated while formatting the `%`-style arguments. Patch by Bénédikt Tran.
- gh-143195 [https://github.com/python/cpython/issues/143195]: Fix use-after-free crashes in **`bytearray.hex()`** and **`memoryview.hex()`** when the separator's **`__len__()`** mutates the original object. Patch by Bénédikt Tran.
- gh-143183 [https://github.com/python/cpython/issues/143183]: Fix a bug in the JIT when dealing with unsupported control-flow or operations.
- gh-142975 [https://github.com/python/cpython/issues/142975]: Fix crash after unfreezing all objects tracked by the garbage collector on the free threaded build.

- gh-143135 [https://github.com/python/cpython/issues/143135]: Set `sys.flags.inspect` to `1` when `PYTHONINSPECT` is `0`. Previously, it was set to `0` in this case.
- gh-143123 [https://github.com/python/cpython/issues/143123]: Protect the JIT against recursive tracing.
- gh-143092 [https://github.com/python/cpython/issues/143092]: Fix a crash in the JIT when dealing with `list.append(x)` style code.
- gh-143003 [https://github.com/python/cpython/issues/143003]: Fix an overflow of the shared empty buffer in `bytearray.extend()` when `__length_hint__()` returns 0 for non-empty iterator.
- gh-143006 [https://github.com/python/cpython/issues/143006]: Fix a possible assertion error when comparing negative non-integer `float` and `int` with the same number of bits in the integer part.
- gh-116738 [https://github.com/python/cpython/issues/116738]: Fix thread safety of `contextvars.Context.run()`.
- gh-142829 [https://github.com/python/cpython/issues/142829]: Fix a use-after-free crash in `contextvars.Context` comparison when a custom `__eq__` method modifies the context via `set()`.
- gh-142863 [https://github.com/python/cpython/issues/142863]: Generate optimized bytecode when calling `list` or `set` with generator expression.
- gh-41779 [https://github.com/python/cpython/issues/41779]: Allowed defining any `__slots__` for a class derived from `tuple` (including classes created by `collections.namedtuple()`).
- gh-69605 [https://github.com/python/cpython/issues/69605]: Fix edge-cases around already imported modules in the REPL auto-completion of imports.
- gh-138568 [https://github.com/python/cpython/issues/138568]: Adjusted the built-in `help()` function so that empty inputs are ignored in interactive mode.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Remove bounds check when indexing into tuples with a constant index.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_CALL_TYPE_1`. Patch by Tomas Roun
- gh-132108 [https://github.com/python/cpython/issues/132108]: Speed up `int.from_bytes()` when passed object supports buffer protocol, like `bytearray` by ~1.2x.
- gh-128334 [https://github.com/python/cpython/issues/128334]: Make the `slice` class subscriptable at runtime to be consistent with typing

implementation.

API C

- gh-141671 [https://github.com/python/cpython/issues/141671]: **`PyMODINIT_FUNC`** (and the new **`PyMODEXPORT_FUNC`**) now adds a linkage declaration (`__declspec(dllexport)`) on Windows.

Python 3.15.0 alpha 4

Release date: 2026-01-13

Testes

- gh-142836 [https://github.com/python/cpython/issues/142836]: Accommodated Solaris in `test_pdb.test_script_target_anonymous_pipe`.

Biblioteca

- gh-122431 [https://github.com/python/cpython/issues/122431]: Corrected the error message in **`readline.append_history_file()`** to state that `nelements` must be non-negative instead of positive.
- gh-143046 [https://github.com/python/cpython/issues/143046]: The **`asyncio`** REPL no longer prints copyright and version messages in the quiet mode (**`-q`**). Patch by Bartosz Sławecki.
- gh-80744 [https://github.com/python/cpython/issues/80744]: Fix issue where `pdb` would read a `.pdbrc` twice if launched from the home directory
- gh-138122 [https://github.com/python/cpython/issues/138122]: Add blocking mode to Tachyon for accurate stack traces in applications with many generators or fast-changing call stacks. Patch by Pablo Galindo.
- gh-143010 [https://github.com/python/cpython/issues/143010]: Fixed a bug in **`mailbox`** where the precise timing of an external event could result in the library opening an existing file instead of a file it expected to create.
- gh-112127 [https://github.com/python/cpython/issues/112127]: Fix possible use-after-free in **`atexit.unregister()`** when the callback is unregistered

during comparison.

- gh-138122 [https://github.com/python/cpython/issues/138122]: Fix incomplete stack traces in the Tachyon profiler's frame cache when profiling code with deeply nested generators. The frame cache now validates that stack traces reach the base frame before caching, preventing broken flamegraphs. Patch by Pablo Galindo.
- gh-142834 [https://github.com/python/cpython/issues/142834]: Change the **pdb** `commands` command to use the last available breakpoint instead of failing when the most recently created breakpoint was deleted.
- gh-142783 [https://github.com/python/cpython/issues/142783]: Fix zoneinfo use-after-free with descriptor _weak_cache. a descriptor as _weak_cache could cause crashes during object creation. The fix ensures proper reference counting for descriptor-provided objects.
- gh-76007 [https://github.com/python/cpython/issues/76007]: Deprecate `VERSION` from **`xml.etree.ElementTree`** and `version` from **`xml.sax.expatreader`** and **`xml.sax.handler`**. Patch by Hugo van Kemenade.
- gh-142784 [https://github.com/python/cpython/issues/142784]: The **`asyncio`** REPL now properly closes the loop upon the end of interactive session. Previously, it could cause surprising warnings. Contributed by Bartosz Sławecki.
- gh-138122 [https://github.com/python/cpython/issues/138122]: Add binary output format to **`profiling.sampling`** for compact storage of profiling data. The new `--binary` option captures samples to a file that can be converted to other formats using the `replay` command. Patch by Pablo Galindo
- gh-142495 [https://github.com/python/cpython/issues/142495]: **`collections.defaultdict`** now prioritizes **`__setitem__()`** when inserting default values from `default_factory`. This prevents race conditions where a default value would overwrite a value set before `default_factory` returns.
- gh-142654 [https://github.com/python/cpython/issues/142654]: Show the clearer error message when using `profiling.sampling` on an unknown PID.
- gh-142560 [https://github.com/python/cpython/issues/142560]: Fix use-after-free in **`bytearray`** search-like methods (**`find()`**, **`count()`**, **`index()`**, **`rindex()`**, and **`rfind()`**) by marking the storage as exported which causes

reallocation attempts to raise **`BufferError`**. For **`contains()`**, **`split()`**, and **`rsplit()`** the buffer protocol is used for this.

- gh-142419 [https://github.com/python/cpython/issues/142419]: **`mmap.mmap.set_name()`** method added to annotate an anonymous memory map if Linux kernel supports `PR_SET_VMA_ANON_NAME` (Linux 5.17 or newer). Patch by Donghee Na.
- gh-139971 [https://github.com/python/cpython/issues/139971]: **`pydoc`**: Ensure that the link to the online documentation of a stdlib module is correct.
- gh-124098 [https://github.com/python/cpython/issues/124098]: Fix issue where methods in handlers that lacked the protocol name but matched a valid base handler method (e.g., `_open()` or `error()`) were incorrectly added to **`urllib.request.OpenerDirector`**'s handlers. Contributed by Andrea Mattei.
- gh-136282 [https://github.com/python/cpython/issues/136282]: Add support for **`UNNAMED_SECTION`** when creating a section via the mapping protocol access

Núcleo e embutidos

- gh-143057 [https://github.com/python/cpython/issues/143057]: Avoid locking in **`PyTraceMalloc_Track()`** and **`PyTraceMalloc_Untrack()`** when **`tracemalloc`** is not enabled.
- gh-139109 [https://github.com/python/cpython/issues/139109]: Add missing terminator in certain cases when tracing in the new JIT compiler.
- gh-142961 [https://github.com/python/cpython/issues/142961]: Fix a segfault in the JIT when constant folding `len(tuple)`.
- gh-142776 [https://github.com/python/cpython/issues/142776]: Fix a file descriptor leak in import.c
- gh-139757 [https://github.com/python/cpython/issues/139757]: Fix building JIT stencils on free-threaded builds.
- gh-129068 [https://github.com/python/cpython/issues/129068]: Make concurrent iteration over the same range iterator thread-safe in the free threading build.
- gh-142543 [https://github.com/python/cpython/issues/142543]: Fix a stack overflow on Clang JIT build configurations with full LTO.
- gh-142448 [https://github.com/python/cpython/issues/142448]: Fix a bug when using monitoring with the JIT.

- gh-142766 [https://github.com/python/cpython/issues/142766]: Clear the frame of a generator when `generator.close()` is called.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_LOAD_ATTR_INSTANCE_VALUE`.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_STORE_ATTR_WITH_HINT`.
- gh-142476 [https://github.com/python/cpython/issues/142476]: Fix a memory leak in the experimental Tier 2 optimizer when creating executors. Patched by Shamil Abdulaev.
- gh-100964 [https://github.com/python/cpython/issues/100964]: Fix reference cycle in exhausted generator frames. Patch by Savannah Ostrowski.
- gh-139922 [https://github.com/python/cpython/issues/139922]: Allow building CPython with the tail calling interpreter on Visual Studio 2026 MSVC. This provides a performance gain over the prior interpreter for MSVC. Patch by Ken Jin, Brandt Bucher, and Chris Eibl. With help from the MSVC team including Hulon Jenkins.

Python 3.15.0 alpha 3

Release date: 2025-12-16

Ferramentas/Demos

- gh-141692 [https://github.com/python/cpython/issues/141692]: Cada componente de um XCframework para iOS agora contém uma pasta `lib` que contém um link simbólico para a biblioteca dinâmica libpython. Isso permite que módulos binários sejam compilados para iOS usando vinculação dinâmica de bibliotecas, em vez de vinculação de framework.

Testes

- gh-140381 [https://github.com/python/cpython/issues/140381]: Fix flaky test_profiling tests on i686 and s390x architectures by increasing slow_fibonacci call frequency from every 5th iteration to every 2nd iteration.

- gh-140210 [https://github.com/python/cpython/issues/140210]: Make `test_sysconfig.test_parse_makefile_renamed_vars` less fragile by clearing the environment variables before parsing the Makefile.

Segurança

- gh-142145 [https://github.com/python/cpython/issues/142145]: Remove quadratic behavior in `xml.minidom` node ID cache clearing.
- gh-42400 [https://github.com/python/cpython/issues/42400]: Fix buffer overflow in `_Py_wrealpath()` for paths exceeding `MAXPATHLEN` bytes by using dynamic memory allocation instead of fixed-size buffer. Patch by Shamil Abdulaev.
- gh-119451 [https://github.com/python/cpython/issues/119451]: Fix a potential memory denial of service in the **`http.client`** module. When connecting to a malicious server, it could cause an arbitrary amount of memory to be allocated. This could have led to symptoms including a **`MemoryError`**, swapping, out of memory (OOM) killed processes or containers, or even system crashes.
- gh-119342 [https://github.com/python/cpython/issues/119342]: Fix a potential memory denial of service in the **`plistlib`** module. When reading a Plist file received from untrusted source, it could cause an arbitrary amount of memory to be allocated. This could have led to symptoms including a **`MemoryError`**, swapping, out of memory (OOM) killed processes or containers, or even system crashes.

Biblioteca

- gh-142754 [https://github.com/python/cpython/issues/142754]: Add the *ownerDocument* attribute to **`xml.dom.minidom`** elements and attributes created by directly instantiating the `Element` or `Attr` class. Note that this way of creating nodes is not supported; creator functions like **`xml.dom.Document.documentElement()`** should be used instead.

- gh-142594 [https://github.com/python/cpython/issues/142594]: Fix crash in `TextIOWrapper.close()` when the underlying buffer's `closed` property calls **`detach()`**.

- gh-76007 [https://github.com/python/cpython/issues/76007]: Deprecate `__version__` from `ctypes`. Patch by Hugo van Kemenade.

- gh-76007 [https://github.com/python/cpython/issues/76007]: Deprecate `__version__` from `wsgiref.simple_server`. Patch by Hugo van Kemenade.

- gh-142651 [https://github.com/python/cpython/issues/142651]: `unittest.mock`: fix a thread safety issue where `Mock.call_count` may return inaccurate values when the mock is called concurrently from multiple threads.

- gh-76007 [https://github.com/python/cpython/issues/76007]: Deprecate `__version__` from `http.server`. Patch by Hugo van Kemenade.

- gh-138122 [https://github.com/python/cpython/issues/138122]: Add `--subprocesses` flag to `profiling.sampling` CLI to automatically profile subprocesses spawned by the target. When enabled, the profiler monitors for new Python subprocesses and profiles each one separately, writing results to individual output files. This is useful for profiling applications that use `multiprocessing`, `ProcessPoolExecutor`, or other subprocess-based parallelism. Patch by Pablo Galindo.

- gh-142595 [https://github.com/python/cpython/issues/142595]: Added type check during initialization of the `decimal` module to prevent a crash in case of broken stdlib. Patch by Sergey B Kirpichev.

- gh-142556 [https://github.com/python/cpython/issues/142556]: Fix crash when a task gets re-registered during finalization in `asyncio`. Patch by Kumar Aditya.

- gh-138122 [https://github.com/python/cpython/issues/138122]: Add `--mode=exception` to the sampling profiler to capture samples only from threads with an active exception, useful for analyzing exception handling overhead. Patch by Pablo Galindo.

- gh-142539 [https://github.com/python/cpython/issues/142539]: `traceback`: Fix location of carets in `SyntaxError`s when the source contains wide

characters.

- gh-123241 [https://github.com/python/cpython/issues/123241]: Avoid reference count operations in garbage collection of `ctypes` objects.

- gh-142451 [https://github.com/python/cpython/issues/142451]: `hmac`: correctly copy `HMAC` attributes for objects copied through `HMAC.copy()`. Patch by Bénédikt Tran.

- gh-138122 [https://github.com/python/cpython/issues/138122]: The `profiling.sampling` flamegraph profiler now supports inverted flamegraph view that aggregates all leaf nodes. In a standard flamegraph, if a hot function is called from multiple locations, it appears multiple times as separate leaf nodes. In the inverted flamegraph, all occurrences of the same leaf function are merged into a single aggregated node at the root, showing the total hotness of that function in one place. The children of each aggregated node represent its callers, making it easier to identify which functions consume the most CPU time and where they are called from.

- gh-112527 [https://github.com/python/cpython/issues/112527]: The help text for required options in `argparse` no longer extended with “ (default: None)”.

- gh-142438 [https://github.com/python/cpython/issues/142438]: Fixed a possible leaked GIL in _PySSL_keylog_callback.

- gh-138122 [https://github.com/python/cpython/issues/138122]: Add bytecode-level instruction profiling to the sampling profiler via the new `--opcodes` flag. When enabled, the profiler captures which bytecode opcode is executing at each sample, including Python 3.11+ adaptive specializations, and visualizes this data in the heatmap, flamegraph, gecko, and live output formats. Patch by Pablo Galindo

- gh-142389 [https://github.com/python/cpython/issues/142389]: Add backtick markup support in `argparse` description and epilog text to highlight inline code when color output is enabled.

- gh-142346 [https://github.com/python/cpython/issues/142346]: Corrige a formatação de uso para grupos mutuamente exclusivos em `argparse` quando eles são precedidos por argumentos posicionais ou seguidos ou misturados com outros argumentos opcionais.

- gh-142374 [https://github.com/python/cpython/issues/142374]: Fix cumulative percentage calculation for recursive functions in the new sampling profiler. When profiling recursive functions, cumulative statistics (cumul%, cumtime) could exceed 100% because each recursive frame in a stack was counted separately. For example, a function recursing 500 times in every sample would show 50000% cumulative presence. The fix deduplicates locations within each sample so cumulative stats correctly represent “percentage of samples where this function was on the stack”. Patch by Pablo Galindo.

- gh-142315 [https://github.com/python/cpython/issues/142315]: Pdb can now run scripts from anonymous pipes used in process substitution. Patch by Bartosz Sławecki.

- gh-64532 [https://github.com/python/cpython/issues/64532]: Subparser help now includes required optional arguments from the parent parser in the usage, making it clearer what arguments are needed to run a subcommand. Patch by Savannah Ostrowski.

- gh-142207 [https://github.com/python/cpython/issues/142207]: Fix: profiling.sampling may cause assertion `!(has_gil && gil_requested)`

- gh-142332 [https://github.com/python/cpython/issues/142332]: Corrige formatação de uso para argumentos posicionais em grupos mutuamente exclusivos em `argparse`. em **`argparse`**.

- gh-142282 [https://github.com/python/cpython/issues/142282]: Fix `winreg.QueryValueEx()` to not accidentally read garbage buffer under race condition.

- gh-142318 [https://github.com/python/cpython/issues/142318]: Fix typing `'q'` at the help of the interactive tachyon profiler exiting the profiler.

- gh-75949 [https://github.com/python/cpython/issues/75949]: Corrige o **`argparse`** para preservar os separadores `|` em grupos mutuamente exclusivos quando a linha de uso quebra devido ao comprimento.

- gh-142267 [https://github.com/python/cpython/issues/142267]: Melhora o desempenho do **`argparse`** armazenando em cache o formatador usado para validação de argumentos.

- gh-139862 [https://github.com/python/cpython/issues/139862]: Remove `color` parameter from **`argparse.HelpFormatter`** constructor. Color is controlled by **`ArgumentParser`**.

- gh-68552 [https://github.com/python/cpython/issues/68552]: Os defeitos `MisplacedEnvelopeHeaderDefect` e `Missing header name` agora são passados corretamente para o método `handle_defect` de `policy` em **`FeedParser`**.

- gh-142206 [https://github.com/python/cpython/issues/142206]: The resource tracker in the **`multiprocessing`** module can now understand messages from older versions of itself. This avoids issues with upgrading Python while it is running. (Note that such 'in-place' upgrades are not tested.)

- gh-142214 [https://github.com/python/cpython/issues/142214]: Fix two regressions in **`dataclasses`** in Python 3.14.1 related to annotations.

  - An exception is no longer raised if `slots=True` is used and the `__init__` method does not have an `__annotate__` attribute (likely because `init=False` was used).
  - An exception is no longer raised if annotations are requested on the `__init__` method and one of the fields is not present in the class annotations. This can occur in certain dynamic scenarios.

  Patch by Jelle Zijlstra.

- gh-142203 [https://github.com/python/cpython/issues/142203]: Remove the *debug_override* parameter from **`importlib.util.cache_from_source()`** which has been deprecated since Python 3.5.

- gh-138122 [https://github.com/python/cpython/issues/138122]: The `_remote_debugging` module now implements frame caching in the `RemoteUnwinder` class to reduce memory reads when profiling remote processes. When `cache_frames=True`, unchanged portions of the call stack are reused from previous samples, significantly improving profiling performance for deep call stacks.

- gh-116738 [https://github.com/python/cpython/issues/116738]: Fix **`cmath`** data race when initializing trigonometric tables with subinterpreters.

- gh-141982 [https://github.com/python/cpython/issues/141982]: Allow **`pdb`** to set breakpoints on async functions with function names.

- gh-74389 [https://github.com/python/cpython/issues/74389]: Quando a entrada padrão (stdin) usada por uma instância de **`subprocess.Popen`** é fechada, isso agora é ignorado em **`subprocess.Popen.communicate()`**, em vez de deixar a classe em um estado inconsistente.

- gh-87512 [https://github.com/python/cpython/issues/87512]: Corrige o tratamento do limite de tempo de **`subprocess.Popen.communicate()`** no Windows ao escrever entradas grandes. Anteriormente, o timeout era ignorado durante a escrita na entrada padrão (stdin), fazendo com que o método bloqueasse indefinidamente se o processo filho não consumisse a entrada rapidamente. Agora, a escrita na entrada padrão é realizada em uma thread em segundo plano, permitindo que o tempo limite seja aplicado corretamente.

- gh-141939 [https://github.com/python/cpython/issues/141939]: Add color to all interpolated values in **`argparse`** help, like `%(default)s` or `%(choices)s`. Patch by Alex Prengère.

- gh-141473 [https://github.com/python/cpython/issues/141473]: Quando o método **`subprocess.Popen.communicate()`** é chamado com um valor de entrada e um tempo limite definido, e é chamado uma segunda vez após uma exceção **`TimeoutExpired`** antes que o processo seja encerrado, ele não deve mais travar.

- gh-141999 [https://github.com/python/cpython/issues/141999]: Correctly allow `KeyboardInterrupt` to stop the process when using `profiling.sampling`.

- gh-142006 [https://github.com/python/cpython/issues/142006]: Corrige um bug no algoritmo de recolhimento do mod `email.policy.default` que resultava incorretamente em uma quebra de linha duplicada quando uma linha que terminava exatamente em max_line_length era seguida por um token que podia ser desdobrado.

- gh-141968 [https://github.com/python/cpython/issues/141968]: Remove data copy from `re` compilation of regexes with large charsets by using `bytearray.take_bytes()`.

- gh-141968 [https://github.com/python/cpython/issues/141968]: Remove data copy from `encodings.idna` `encode()` and `encode()` by using `bytearray.take_bytes()`.

- gh-141968 [https://github.com/python/cpython/issues/141968]: Remove data copy from `codecs` punycode encoding by using `bytearray.take_bytes()`.

- gh-141968 [https://github.com/python/cpython/issues/141968]: Remove data copy from `wave.Wave_read.readframes()` and `wave.Wave_write.writeframes()` by using `bytearray.take_bytes()`.

- gh-141968 [https://github.com/python/cpython/issues/141968]: Remove a data copy from `base64.b32decode()` and `base64.b32encode()` by using `bytearray.take_bytes()`.

- gh-59000 [https://github.com/python/cpython/issues/59000]: Fix `pdb` breakpoint resolution for class methods when the module defining the class is not imported.

- gh-116738 [https://github.com/python/cpython/issues/116738]: Corrige problema de segurança de threads com objetos scanners de `re` em construções com threads livres.

- gh-138122 [https://github.com/python/cpython/issues/138122]: The `profiling.sampling` flamegraph profiler now displays thread status statistics showing the percentage of time threads spend holding the GIL, running without the GIL, waiting for the GIL, and performing garbage collection. These statistics help identify GIL contention and thread behavior patterns. When filtering by thread, the display shows per-thread metrics.

- gh-141781 [https://github.com/python/cpython/issues/141781]: Fixed an issue where pdb.line_prefix assignment was ignored if assigned after the module was imported.

- gh-141863 [https://github.com/python/cpython/issues/141863]: Update Streams to use **`bytearray.take_bytes()`** for a over 10% performance improvement on pyperformance asyncio_tcp benchmark.

- gh-141817 [https://github.com/python/cpython/issues/141817]: Add **`socket.IPV6_HDRINCL`** constant.

- gh-105836 [https://github.com/python/cpython/issues/105836]: Fix **`asyncio.run_coroutine_threadsafe()`** leaving underlying cancelled asyncio task running.

- gh-141570 [https://github.com/python/cpython/issues/141570]: Suporte para objeto arquivo ou similar que levanta **`OSError`** de **`fileno()`** na detecção de cores (`_colorize.can_colorize()`). Isso pode ocorrer quando `sys.stdout` é redirecionado.

- gh-141679 [https://github.com/python/cpython/issues/141679]: Add colour to defaults in **`argparse`** help. Patch by Hugo van Kemenade.

- gh-141686 [https://github.com/python/cpython/issues/141686]: Break reference cycles created by each call to **`json.dump()`** or **`json.JSONEncoder.iterencode()`**.

- gh-141659 [https://github.com/python/cpython/issues/141659]: Corrige erros de descritor de arquivo inválido provenientes do processo `_posixsubprocess` no AIX.

- gh-141645 [https://github.com/python/cpython/issues/141645]: Add a new `--live` mode to the tachyon profiler in **`profiling.sampling`** module. This mode consist of a live TUI that displays real-time profiling statistics as the target application runs, similar to `top`. Patch by Pablo Galindo

- gh-141615 [https://github.com/python/cpython/issues/141615]: Check `stdin` instead of `stdout` for `use_rawinput` in **`pdb`**.

- gh-69113 [https://github.com/python/cpython/issues/69113]: Fix **`doctest`** to correctly report line numbers for doctests in `__test__` dictionary when formatted as triple-quoted strings by finding unique lines in the string and matching them in the source file.

- gh-141600 [https://github.com/python/cpython/issues/141600]: Corrigeoa detecção da versão musl no Void Linux.

- gh-48752 [https://github.com/python/cpython/issues/48752]: Add **`readline.get_pre_input_hook()`** function to retrieve the current pre-input hook. This allows applications to save and restore the hook without overwriting user settings. Patch by Sanyam Khurana.

- gh-141565 [https://github.com/python/cpython/issues/141565]: Add async-aware profiling to the Tachyon sampling profiler. The profiler now reconstructs and displays async task hierarchies in flamegraphs, making the output more actionable for users. Patch by Savannah Ostrowski and Pablo Galindo Salgado.

- gh-60107 [https://github.com/python/cpython/issues/60107]: Remove a copy from **`io.RawIOBase.read()`**. If the underlying I/O class keeps a reference to the mutable memory, raise a **`BufferError`**.

- gh-116738 [https://github.com/python/cpython/issues/116738]: Torna o módulo csv seguro para thread na construção com threads livres.

- gh-140911 [https://github.com/python/cpython/issues/140911]: **`collections`**: Garante que os métodos `UserString.rindex()` e `UserString.index()` aceitem instâncias de **`collections.UserString`** como sub-argumento.

- gh-140875 [https://github.com/python/cpython/issues/140875]: Corrigido o tratamento de referências de caracteres não fechadas (nomeadas e numéricas) seguidas pelo fim do arquivo em **`html.parser.HTMLParser`** com `convert_charrefs=False`.

- gh-140677 [https://github.com/python/cpython/issues/140677]: Add heatmap visualization mode to the Tachyon sampling profiler. The new `--heatmap` output format provides a line-by-line view showing execution intensity with color-coded samples, inline statistics, and interactive call graph navigation between callers and callees.

- gh-139946 [https://github.com/python/cpython/issues/139946]: Distinguish stdout and stderr when colorizing output in argparse module.

- gh-76007 [https://github.com/python/cpython/issues/76007]: **`pydoc`**: Fix **`DeprecationWarning`** being raised when generating doc for stdlib modules.

- gh-138697 [https://github.com/python/cpython/issues/138697]: Fix inferring *dest* from a single-dash long option in **`argparse`**. If a short option and a single-dash long option are passed to **`add_argument()`**, *dest* is now inferred from the single-dash long option.

- gh-138525 [https://github.com/python/cpython/issues/138525]: Add support for single-dash long options and alternate prefix characters in **`argparse.BooleanOptionalAction`**.

- gh-79986 [https://github.com/python/cpython/issues/79986]: Add parsing for `References` and `In-Reply-To` headers to the **`email`** library that parses the header content as lists of message id tokens. This prevents them from being folded incorrectly.

- gh-135559 [https://github.com/python/cpython/issues/135559]: Flag: a `dir()` on a `Flag` enumeration now shows non-canonical members. (i.e. aliases).

- gh-134453 [https://github.com/python/cpython/issues/134453]: Corrigido o tratamento de `input=` da função **`subprocess.Popen.communicate()`** por instâncias de **`memoryview`** que não tinham formato de bytes em

plataformas POSIX. Agora, essas instâncias são convertidas corretamente para uma visualização em formato de byte, em vez de truncar a entrada. Plataformas Windows não apresentavam esse bug.

- gh-127930 [https://github.com/python/cpython/issues/127930]: Add `__all__` to `tkinter.simpledialog`.

- gh-115952 [https://github.com/python/cpython/issues/115952]: Fix a potential memory denial of service in the `pickle` module. When reading a pickled data received from untrusted source, it could cause an arbitrary amount of memory to be allocated, even if the code that is allowed to execute is restricted by overriding the `find_class()` method. This could have led to symptoms including a `MemoryError`, swapping, out of memory (OOM) killed processes or containers, or even system crashes.

- bpo-40350 [https://bugs.python.org/issue?@action=redirect&bpo=40350]: Fix support for namespace packages in `modulefinder`.

Documentação

- gh-141994 [https://github.com/python/cpython/issues/141994]: `xml.sax.handler`: Faz a documentação de `xml.sax.handler.feature_external_ges` alertar sobre a vulnerabilidade a ataques de entidades externas [https://en.wikipedia.org/wiki/XML_external_entity_attack]. Patch por Sebastian Pipping.

Núcleo e embutidos

- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_STORE_ATTR_INSTANCE_VALUE`.
- gh-142718 [https://github.com/python/cpython/issues/142718]: JIT: Fix segfault caused by not flushing the stack to memory at side exits.
- gh-142737 [https://github.com/python/cpython/issues/142737]: Tracebacks will be displayed in fallback mode even if `io.open()` is lost. Previously, this would crash the interpreter. Patch by Bartosz Sławecki.

- gh-116738 [https://github.com/python/cpython/issues/116738]: Make the attributes in `bz2` thread-safe on the free threaded build.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_CALL_LIST_APPEND`.
- gh-142554 [https://github.com/python/cpython/issues/142554]: Fix a crash in `divmod()` when `_pylong.int_divmod()` does not return a tuple of length two exactly. Patch by Bénédikt Tran.
- gh-142531 [https://github.com/python/cpython/issues/142531]: Fix a free-threaded GC performance regression. If there are many untracked tuples, the GC will run too often, resulting in poor performance. The fix is to include untracked tuples in the "long lived" object count. The number of frozen objects is also now included since the free-threaded GC must scan those too.
- gh-142402 [https://github.com/python/cpython/issues/142402]: Fix reference counting when adjacent literal parts are merged while constructing `string.templatelib.Template`, preventing the displaced string object from leaking.
- gh-116738 [https://github.com/python/cpython/issues/116738]: Make the attributes in `zlib` thread-safe on the free threaded build.
- gh-142343 [https://github.com/python/cpython/issues/142343]: Fix SIGILL crash on m68k due to incorrect assembly constraint.
- gh-142236 [https://github.com/python/cpython/issues/142236]: Improve the "Perhaps you forgot a comma?" syntax error for multi-line string concatenations to point to the last string instead of the first, making it easier to locate where the comma is missing. Patch by Pablo Galindo.
- gh-142236 [https://github.com/python/cpython/issues/142236]: Fix incorrect keyword suggestions for syntax errors in `traceback`. The keyword typo suggestion mechanism would incorrectly suggest replacements when the extracted source code was incomplete rather than containing an actual typo. Patch by Pablo Galindo.
- gh-142305 [https://github.com/python/cpython/issues/142305]: Decrease the size of the generated stencils and the runtime JIT code. Patch by Diego Russo.
- gh-135379 [https://github.com/python/cpython/issues/135379]: Implement a limited form of register allocation known as "top of stack caching" in the JIT. It works by keeping 0-3 of the top items in the stack in registers. The code generator generates multiple versions of those uops that do not escape and are relatively small. During JIT compilation, the copy that

produces the least memory traffic is selected, spilling or reloading values when needed.

- gh-142276 [https://github.com/python/cpython/issues/142276]: Fix missing type watcher when promoting attribute loads to constants in the JIT. Patch by Ken Jin. Reproducer by Yuancheng Jiang.
- gh-142218 [https://github.com/python/cpython/issues/142218]: Fix crash when inserting into a split table dictionary with a non `str` key that matches an existing key.
- gh-141976 [https://github.com/python/cpython/issues/141976]: Check against abstract stack overflow in the JIT optimizer.
- gh-97850 [https://github.com/python/cpython/issues/97850]: Remove all `*.load_module()` usage and definitions from the import system and importlib. The method has been deprecated in favor of `importlib.abc.Loader.exec_module()` since Python 3.4.
- gh-142048 [https://github.com/python/cpython/issues/142048]: Fix quadratically increasing garbage collection delays in free-threaded build.
- gh-65961 [https://github.com/python/cpython/issues/65961]: Stop setting `__cached__` on modules.
- gh-141770 [https://github.com/python/cpython/issues/141770]: Annotate anonymous mmap usage only when supported by the Linux kernel and if `-X dev` is used or Python is built in debug mode. Patch by Donghee Na.
- gh-142029 [https://github.com/python/cpython/issues/142029]: Raise `ModuleNotFoundError` instead of crashing when a nonexistent module is used as a name in `_imp.create_builtin()`.
- gh-142029 [https://github.com/python/cpython/issues/142029]: Raise `ValueError` instead of crashing when empty string is used as a name in `_imp.create_builtin()`.
- gh-141976 [https://github.com/python/cpython/issues/141976]: Protect against specialization failures in the tracing JIT compiler for performance reasons.
- gh-141861 [https://github.com/python/cpython/issues/141861]: Fix invalid memory read in the `ENTER_EXECUTOR` instruction.
- gh-141930 [https://github.com/python/cpython/issues/141930]: When importing a module, use Python's regular file object to ensure that writes to `.pyc` files are complete or an appropriate error is raised.

- gh-138122 [https://github.com/python/cpython/issues/138122]: Add incomplete sample detection to prevent corrupted profiling data. Each thread state now contains an embedded base frame (sentinel at the bottom of the frame stack) with owner type `FRAME_OWNED_BY_INTERPRETER`. The profiler validates that stack unwinding terminates at this sentinel frame. Samples that fail to reach the base frame (due to race conditions, memory corruption, or other errors) are now rejected rather than being included as spurious data.
- gh-120158 [https://github.com/python/cpython/issues/120158]: Corrige estado inconsistente ao habilitar e desabilitar eventos de monitoramento muitas vezes.
- gh-140638 [https://github.com/python/cpython/issues/140638]: Expose a `"candidates"` stat in **`gc.get_stats()`** and **`gc.callbacks`**.
- gh-141780 [https://github.com/python/cpython/issues/141780]: Fix **`Py_mod_gil`** with API added in **PEP 793** [https://peps.python.org/pep-0793/]: **`PyModule_FromSlotsAndSpec()`** and `PyModExport` hooks
- gh-141732 [https://github.com/python/cpython/issues/141732]: Ensure the **`__repr__()`** for **`ExceptionGroup`** and **`BaseExceptionGroup`** does not change when the exception sequence that was original passed in to its constructor is subsequently mutated.
- gh-140638 [https://github.com/python/cpython/issues/140638]: Expose a `"duration"` stat in **`gc.get_stats()`** and **`gc.callbacks`**.
- gh-139653 [https://github.com/python/cpython/issues/139653]: Somente levanta um erro `RecursionError` ou aciona um erro fatal se o ponteiro da pilha estiver abaixo do ponteiro limite *e* acima da base da pilha. Se estiver fora desses limites, presume que está tudo bem. Isso evita falsos positivos quando threads do espaço do usuário trocam pilhas.
- gh-41779 [https://github.com/python/cpython/issues/41779]: Allowed defining the *__dict__* and *__weakref__* `__slots__` for any class.
- gh-139103 [https://github.com/python/cpython/issues/139103]: Melhora a escalabilidade multithread das classes de dados na construção com threads livres.
- gh-141589 [https://github.com/python/cpython/issues/141589]: Change `backoff counter` to use prime numbers instead of powers of 2. Use only 3 bits for `counter` and 13 bits for `value`. This allows to support values up to 8191. Patch by Mikhail Efimov.

- gh-137007 [https://github.com/python/cpython/issues/137007]: Fix a bug during JIT compilation failure which caused garbage collection debug assertions to fail.
- gh-132657 [https://github.com/python/cpython/issues/132657]: For the free-threaded build, avoid locking the `set` object for the `__contains__` method.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_CALL_STR_1`.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_CALL_BUILTIN_O`.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Eliminate redundant refcounting from `_CALL_TUPLE_1`. Patch by Noam Cohen

API C

- gh-142589 [https://github.com/python/cpython/issues/142589]: Fix `PyUnstable_Object_IsUniqueReferencedTemporary()` handling of tagged ints on the interpreter stack.
- gh-142571 [https://github.com/python/cpython/issues/142571]: **`PyUnstable_CopyPerfMapFile()`** now checks that opening the file succeeded before flushing.
- gh-142225 [https://github.com/python/cpython/issues/142225]: Fixed the `PyABIInfo_VAR` macro.
- gh-141049 [https://github.com/python/cpython/issues/141049]: **`_PyObject_CallMethodId()`**, **`_PyObject_GetAttrId()`** and **`_PyUnicode_FromId()`** are deprecated since 3.15 and will be removed in 3.20. Instead, use `PyUnicode_InternFromString()` and cache the result in the module state, then call `PyObject_CallMethod()` or `PyObject_GetAttr()`. Patch by Victor Stinner.
- gh-142163 [https://github.com/python/cpython/issues/142163]: Fix the `HAVE_THREAD_LOCAL` macro being defined without the `Py_BUILD_CORE` macro set after including `Python.h`.
- gh-137422 [https://github.com/python/cpython/issues/137422]: Corrige o problema de condição de corrida com threads livres em `PyImport_AddModuleRef()`. Anteriormente, era possível que duas chamadas à função retornassem dois objetos diferentes, dos quais apenas um era armazenado em `sys.modules`.

- gh-141726 [https://github.com/python/cpython/issues/141726]: Add **`PyDict_SetDefaultRef()`** to the Stable ABI.
- gh-140042 [https://github.com/python/cpython/issues/140042]: Removed the sqlite3_shutdown call that could cause closing connections for sqlite when used with multiple sub interpreters.
- gh-141070 [https://github.com/python/cpython/issues/141070]: Add **`PyUnstable_Object_Dump()`** to dump an object to `stderr`. It should only be used for debugging. Patch by Victor Stinner.
- gh-139165 [https://github.com/python/cpython/issues/139165]: Expose the functions **`Py_SIZE()`**, **`Py_IS_TYPE()`** and **`Py_SET_SIZE()`** in the Stable ABI.

Construção

- gh-131372 [https://github.com/python/cpython/issues/131372]: Add `LDVERSION` and `EXE` to the `base_interpreter` value of `build-details.json`.
- gh-142454 [https://github.com/python/cpython/issues/142454]: When calculating the digest of the JIT stencils input, sort the hashed files by filenames before adding their content to the hasher. This ensures deterministic hash input and hence deterministic hash, independent on filesystem order.
- gh-131372 [https://github.com/python/cpython/issues/131372]: `build-details.py` will only be installed as part of the main install (`make install`). `make altinstall` will no longer include it.
- gh-142234 [https://github.com/python/cpython/issues/142234]: Allow `--enable-wasm-dynamic-linking` for WASI. While CPython doesn't directly support it so external/downstream users do not have to patch in support for the flag.
- gh-142050 [https://github.com/python/cpython/issues/142050]: Fixed a bug where JIT stencils produced on Windows contained debug data. Patch by Chris Eibl.
- gh-141808 [https://github.com/python/cpython/issues/141808]: Do not generate the jit stencils twice in case of PGO builds on Windows.
- gh-141926 [https://github.com/python/cpython/issues/141926]: `RUNSHARED` is no longer cleared when cross-compiling. Previously, `RUNSHARED` was cleared when cross-compiling, which breaks PGO when using `--`

`enabled-shared` on systems where the cross-compiled CPython is otherwise executable (e.g., via transparent emulation).

- gh-141808 [https://github.com/python/cpython/issues/141808]: When running `make clean-retain-profile`, keep the generated JIT stencils. That way, the stencils are not generated twice when Profile-guided optimization (PGO) is used. It also allows distributors to supply their own pre-built JIT stencils.
- gh-141784 [https://github.com/python/cpython/issues/141784]: Correção na construção de `_remote_debugging_module.c` em Linux de 32 bits. Inclusão de Python.h antes dos cabeçalhos do sistema para garantir que `_remote_debugging_module.c` utilize os mesmos tipos (ABI) que o Python. Patch por Victor Stinner.
- gh-141172 [https://github.com/python/cpython/issues/141172]: Update to WASI SDK 29.
- gh-139707 [https://github.com/python/cpython/issues/139707]: Add configure option **`--with-missing-stdlib-config=FILE`** allows which distributors to pass a JSON [https://www.json.org/json-en.html] configuration file containing custom error messages for missing standard library modules.
- gh-108819 [https://github.com/python/cpython/issues/108819]: Honor **`--with-platlibdir`** in the pure-Python standard library installation path, if `PLATLIBDIR` doesn't match the value used in `LIBDIR`.

Python 3.15.0 alpha 2

Release date: 2025-11-18

Windows

- gh-140849 [https://github.com/python/cpython/issues/140849]: Update bundled liblzma to version 5.8.1.

Ferramentas/Demos

- gh-141442 [https://github.com/python/cpython/issues/141442]: O ambiente de testes do iOS agora lida corretamente com argumentos de teste que

contêm espaços.

- gh-140702 [https://github.com/python/cpython/issues/140702]: A aplicação de teste para iOS agora exporá a variável de ambiente `GITHUB_ACTIONS` para os aplicativos iOS que estão sendo testados.
- gh-139198 [https://github.com/python/cpython/issues/139198]: Remove `Tools/scripts/checkpip.py` script.
- gh-139188 [https://github.com/python/cpython/issues/139188]: Remove `Tools/tz/zdump.py` script.

Testes

- gh-140482 [https://github.com/python/cpython/issues/140482]: Preserva e restaura o estado de `stty echo` como parte do ambiente de teste.
- gh-140082 [https://github.com/python/cpython/issues/140082]: Atualiza o comando `python -m test` para definir `FORCE_COLOR=1` quando executado com cores ativadas, para que o **`unittest`**, executado por ele com saída redirecionada, seja exibido em cores.
- gh-136442 [https://github.com/python/cpython/issues/136442]: Usa o código de saída `1` em vez de `5` se a **`unittest.TestCase.setUpClass()`** levantar uma exceção.

Segurança

- gh-137836 [https://github.com/python/cpython/issues/137836]: Adiciona suporte ao elemento “plaintext”, elementos RAWTEXT “xmp”, “iframe”, “noembed e “noframes”, e elementos RAWTEXT “noscript” opicionais em **`html.parser.HTMLParser`**.
- gh-136063 [https://github.com/python/cpython/issues/136063]: **`email.message`**: ensure linear complexity for legacy HTTP parameters parsing. Patch by Bénédikt Tran.
- gh-136065 [https://github.com/python/cpython/issues/136065]: Corrige complexidade quadrática em **`os.path.expandvars()`**.

Biblioteca

- gh-141497 [https://github.com/python/cpython/issues/141497]: **`ipaddress`**: garante que os métodos **`IPv4Network.hosts()`** e **`IPv6Network.hosts()`**

sempre retornem um iterador.

- gh-140938 [https://github.com/python/cpython/issues/140938]: As funções `statistics.stdev()` e `statistics.pstdev()` agora levantam `ValueError` quando a entrada contém um infinito ou um NaN.

- gh-124111 [https://github.com/python/cpython/issues/124111]: Atualizada a configuração de threads do Tcl em `_tkinter` para presumir que threads estão sempre disponíveis no Tcl 9 e versões posteriores.

- gh-137109 [https://github.com/python/cpython/issues/137109]: O `os.fork` e as APIs de fork relacionadas não emitirão mais avisos no caso comum em que as APIs da plataforma Linux ou macOS retornam o número de threads em um processo e encontram a resposta como 1, mesmo quando um função de retorno `os.register_at_fork()` `after_in_parent=` (re)inicia uma thread.

- gh-141314 [https://github.com/python/cpython/issues/141314]: Corrigido o erro de asserção em `io.TextIOWrapper.tell()` ao ler arquivos com quebras de linha de retorno de carro isoladas (`\r`).

- gh-141311 [https://github.com/python/cpython/issues/141311]: Corrigido o erro de asserção em **`io.BytesIO.readinto()`** e o comportamento indefinido que ocorria quando a posição de leitura estava acima da capacidade em `io.BytesIO`.

- gh-87710 [https://github.com/python/cpython/issues/87710]: `mimetypes`: Update mime type for `.ai` files to `application/pdf`.

- gh-85524 [https://github.com/python/cpython/issues/85524]: Update `io.FileIO.readall`, an implementation of `io.RawIOBase.readall()`, to follow `io.IOBase` guidelines and raise `io.UnsupportedOperation` when a file is in “w” mode rather than `OSError`

- gh-141141 [https://github.com/python/cpython/issues/141141]: Corrige um problema de segurança de threads com `base64.b85decode()`. Contribuição de Benel Tayar.

- gh-141018 [https://github.com/python/cpython/issues/141018]: **`mimetypes`**: Update `.exe`, `.dll`, `.rtf` and (when `strict=False`) `.jpg` to their correct IANA mime type.

- gh-137969 [https://github.com/python/cpython/issues/137969]: Corrigido o método **`annotationlib.ForwardRef.evaluate()`** que retorna objetos **`ForwardRef`** que não são atualizados com novas variáveis globais.

- gh-75593 [https://github.com/python/cpython/issues/75593]: Add support of path-like objects and bytes-like objects in **`wave.open()`**.

- gh-140797 [https://github.com/python/cpython/issues/140797]: A classe não documentada **`re.Scanner`** agora proíbe expressões regulares que contenham grupos de captura em seus padrões de léxico. Padrões que utilizavam grupos de captura podiam anteriormente causar falhas de segmentação. Em vez disso, utilize grupos sem captura (?:…).

- gh-125115 [https://github.com/python/cpython/issues/125115]: Refatora a correção do problema de análise sintática do **`pdb`** para que os argumentos posicionais possam ser passados de forma intuitiva.

- gh-140815 [https://github.com/python/cpython/issues/140815]: **`faulthandler`** agora detecta se um quadro ou um objeto código é inválido ou foi liberado. Patch por Victor Stinner.

- gh-100218 [https://github.com/python/cpython/issues/100218]: Define corretamente **`errno`** quando as **`socket.if_nametoindex()`** ou **`socket.if_indextoname()`** levantam **`OSError`**. Patch por Bénédikt Tran.

- gh-140734 [https://github.com/python/cpython/issues/140734]: **`multiprocessing`**: Corrige erro de diferença de um bit ao verificar o comprimento do caminho de um arquivo de socket temporário. Patch por Bénédikt Tran.

- gh-140873 [https://github.com/python/cpython/issues/140873]: Add support of non-descriptor callables in **`functools.singledispatchmethod()`**.

- gh-140874 [https://github.com/python/cpython/issues/140874]: Atualiza a versão do pip empacotado no ensurepip para a versão 25.3.

- gh-140826 [https://github.com/python/cpython/issues/140826]: Now **`winreg.HKEYType`** objects are compared by their underlying Windows registry handle value instead of their object identity.

- gh-140808 [https://github.com/python/cpython/issues/140808]: The internal class `mailbox._ProxyFile` is no longer a parameterized generic.

- gh-140691 [https://github.com/python/cpython/issues/140691]: Em **`urllib.request`**, quando a abertura de um URL FTP falha porque não é possível estabelecer uma conexão de dados, o socket da conexão de controle é agora fechado para evitar um **`ResourceWarning`**.

- gh-103847 [https://github.com/python/cpython/issues/103847]: Corrigido o travamento que ocorria ao cancelar um processo criado por **`asyncio.create_subprocess_exec()`** ou **`asyncio.create_subprocess_shell()`**. Patch por Kumar Aditya.

- gh-137821 [https://github.com/python/cpython/issues/137821]: Convert `_json` module to use Argument Clinic. Patched by Yoonho Hann.

- gh-140790 [https://github.com/python/cpython/issues/140790]: Initialize all Pdb's instance variables in `__init__`, remove some hasattr/getattr

- gh-140766 [https://github.com/python/cpython/issues/140766]: Add **`enum.show_flag_values()`** and `enum.bin` to `enum.__all__`.

- gh-120057 [https://github.com/python/cpython/issues/120057]: Adiciona **`os.reload_environ()`** para `os.__all__`.

- gh-140741 [https://github.com/python/cpython/issues/140741]: Fix `profiling.sampling.sample()` incorrectly handling a **`FileNotFoundError`** or **`PermissionError`**.

- gh-140228 [https://github.com/python/cpython/issues/140228]: Evita fazer chamadas desnecessárias ao sistema de arquivos para módulos

congelados em **`linecache`** quando o cache de módulos global não estiver presente.

- gh-139946 [https://github.com/python/cpython/issues/139946]: Error and warning keywords in `argparse.ArgumentParser` messages are now colorized when color output is enabled, fixing a visual inconsistency in which they remained plain text while other output was colorized.

- gh-140590 [https://github.com/python/cpython/issues/140590]: Correção na verificação de argumentos para o método **`functools.partial.__setstate__()`**, que pode levar à corrupção do estado interno e ao travamento do sistema. Patch por Sergey Miryanov.

- gh-125434 [https://github.com/python/cpython/issues/125434]: Exibe o nome da thread em **`faulthandler`** no Windows. Patch por Victor Stinner.

- gh-140634 [https://github.com/python/cpython/issues/140634]: Corrige um bug de contagem de referências em **`os.sched_param.__reduce__()`**.

- gh-140650 [https://github.com/python/cpython/issues/140650]: Fix an issue where closing **`io.BufferedWriter`** could crash if the closed attribute raised an exception on access or could not be converted to a boolean.

- gh-140633 [https://github.com/python/cpython/issues/140633]: Ignora **`AttributeError`** ao definir o atributo `__file__` de um módulo ao carregar um módulo de extensão empacotado como Apple Framework.

- gh-140601 [https://github.com/python/cpython/issues/140601]: **`xml.etree.ElementTree.iterparse()`** now emits a **`ResourceWarning`** when the iterator is not explicitly closed and was opened with a filename. This helps developers identify and fix resource leaks. Patch by Osama Abdelkader.

- gh-140593 [https://github.com/python/cpython/issues/140593]: **`xml.parsers.expat`**: Corrige um vazamento de memória que poderia afetar usuários com **`ElementDeclHandler()`** definido como um manipulador de declaração de elemento personalizado. Patch por Sebastian Pipping.

- gh-140607 [https://github.com/python/cpython/issues/140607]: Dentro de `io.RawIOBase.read()`, verifica se a contagem de bytes retornada por `io.RawIOBase.readinto()` é válida (dentro do buffer fornecido).

- gh-138162 [https://github.com/python/cpython/issues/138162]: Corrige `logging.LoggerAdapter` com `merge_extra=True` e sem o argumento *extra*.

- gh-140481 [https://github.com/python/cpython/issues/140481]: Improve error message when trying to iterate a Tk widget, image or font.

- gh-138774 [https://github.com/python/cpython/issues/138774]: `ast.unparse()` agora gera código-fonte completo ao manipular nós `ast.Interpolation` que não têm uma fonte especificada.

- gh-140474 [https://github.com/python/cpython/issues/140474]: Corrige vazamento de memória em `array.array` ao criar vetores a partir de um objeto `str` vazio e do código do tipo `u`.

- gh-140448 [https://github.com/python/cpython/issues/140448]: Change the default of `suggest_on_error` to `True` in `argparse.ArgumentParser`.

- gh-137530 [https://github.com/python/cpython/issues/137530]: `dataclasses`: Corrige as anotações dos métodos `__init__` gerados, substituindo as anotações que estavam embutidas no código-fonte gerado por funções `__annotate__` anexadas aos métodos.

- gh-140348 [https://github.com/python/cpython/issues/140348]: Corrige uma regressão no Python 3.14.0 em que o uso do operador `|` em um objeto `typing.Union` combinado com um objeto que não é um tipo levantava um erro.

- gh-76007 [https://github.com/python/cpython/issues/76007]: `decimal`: Deprecate `__version__` and replace with `decimal.SPEC_VERSION`.

- gh-76007 [https://github.com/python/cpython/issues/76007]: Deprecate `__version__` from `imaplib`. Patch by Hugo van Kemenade.

- gh-140272 [https://github.com/python/cpython/issues/140272]: Corrige vazamento de memória no método **`clear()`** do banco de dados **`dbm.gnu`**.

- gh-129117 [https://github.com/python/cpython/issues/129117]: **`unicodedata`**: Add **`isxidstart()`** and **`isxidcontinue()`** functions to check whether a character can start or continue a Unicode Standard Annex #31 [https://www.unicode.org/reports/tr31/] identifier.

- gh-140251 [https://github.com/python/cpython/issues/140251]: Colorize the default import statement `import asyncio` in asyncio REPL.

- gh-140212 [https://github.com/python/cpython/issues/140212]: Calendar's HTML formatting now accepts year and month as options. Previously, running `python -m calendar -t html 2025 10` would result in an error message. It now generates an HTML document displaying the calendar for the specified month. Contributed by Pål Grønås Drange.

- gh-135801 [https://github.com/python/cpython/issues/135801]: Improve filtering by module in **`warnings.warn_explicit()`** if no *module* argument is passed. It now tests the module regular expression in the warnings filter not only against the filename with `.py` stripped, but also against module names constructed starting from different parent directories of the filename (with `/__init__.py`, `.py` and, on Windows, `.pyw` stripped).

- gh-139707 [https://github.com/python/cpython/issues/139707]: Improve **`ModuleNotFoundError`** error message when a standard library module is missing.

- gh-140041 [https://github.com/python/cpython/issues/140041]: Corrige importação de **`ctypes`** no Android e Cygwin quando sinalizadores ABI estiverem presentes.

- gh-140120 [https://github.com/python/cpython/issues/140120]: Corrigido um vazamento de memória em **`hmac`** ao usar o backend hacl-star. Descoberto por `@ashm-dev` usando AddressSanitizer.

- gh-140141 [https://github.com/python/cpython/issues/140141]: The **`importlib.metadata.PackageNotFoundError`** traceback raised when `importlib.metadata.Distribution.from_name` cannot discover a distribution no longer includes a transient **`StopIteration`** exception trace.

  Contributed by Bartosz Sławecki in gh-140142 [https://github.com/python/cpython/issues/140142].

- gh-140166 [https://github.com/python/cpython/issues/140166]: **`mimetypes`**: Per the IANA assignment [https://www.iana.org/assignments/media-types/application/texinfo], update the MIME type for the `.texi` and `.texinfo` file formats to `application/texinfo`, instead of `application/x-texinfo`.

- gh-140135 [https://github.com/python/cpython/issues/140135]: Speed up **`io.RawIOBase.readall()`** by using PyBytesWriter API (about 4x faster)

- gh-76007 [https://github.com/python/cpython/issues/76007]: **`zlib`**: Deprecate `__version__` and schedule for removal in Python 3.20.

- gh-136702 [https://github.com/python/cpython/issues/136702]: **`encodings`**: Deprecate passing a non-ascii *encoding* name to **`encodings.normalize_encoding()`** and schedule removal of support for Python 3.17.

- gh-139940 [https://github.com/python/cpython/issues/139940]: Print clearer error message when using `pdb` to attach to a non-existing process.

- gh-139462 [https://github.com/python/cpython/issues/139462]: When a child process in a **`concurrent.futures.ProcessPoolExecutor`** terminates abruptly, the resulting traceback will now tell you the PID and exit code of the terminated process. Contributed by Jonathan Berg.

- gh-63161 [https://github.com/python/cpython/issues/63161]: Corrige **`tokenize.detect_encoding()`**. Suporte a shebang e comentários não UTF-8 se a codificação não UTF-8 for especificada. Detecta erro de

decodificação para codificação não UTF-8. Detecta bytes nulos no código-fonte.

- gh-101828 [https://github.com/python/cpython/issues/101828]: Corrige os codecs `'shift_jisx0213'`, `'shift_jis_2004'`, `'euc_jisx0213'` e `'euc_jis_2004'` que truncavam caracteres nulos, pois eram tratados como parte de sequências de múltiplos caracteres.

- gh-139246 [https://github.com/python/cpython/issues/139246]: correção: colar texto com largura zero na largura padrão do REPL está incorreto.

- gh-83714 [https://github.com/python/cpython/issues/83714]: Implement **`os.statx()`** on Linux kernel versions 4.11 and later with glibc versions 2.28 and later. Contributed by Jeffrey Bosboom and Victor Stinner.

- gh-138891 [https://github.com/python/cpython/issues/138891]: Corrige `SyntaxError` quando `inspect.get_annotations(f, eval_str=True)` for chamado em uma função anotada com `star_expression` da **PEP 646** [https://peps.python.org/pep-0646/]

- gh-138859 [https://github.com/python/cpython/issues/138859]: Corrige a parametrização de tipo genérico levantando **`TypeError`** ao omitir uma **`ParamSpec`** que tem um padrão que não é uma lista de tipos.

- gh-138764 [https://github.com/python/cpython/issues/138764]: Impede que **`annotationlib.call_annotate_function()`** chame funções `__annotate__` que não ofereçam suporte a `VALUE_WITH_FAKE_GLOBALS` em um espaço de nomes globals falso com globais vazios.

  Faz com que as anotações `FORWARDREF` e `STRING` voltem a usar anotações `VALUE` no caso de nem seu próprio formato nem `VALUE_WITH_FAKE_GLOBALS` serem suportados.

- gh-138775 [https://github.com/python/cpython/issues/138775]: O uso de `python -m` com **`base64`** foi corrigido para detectar a entrada de um terminal, de forma que reconheça corretamente o EOF (fim de arquivo).

- gh-98896 [https://github.com/python/cpython/issues/98896]: Corrige um erro em resource_tracker do módulo multiprocessing quando os nomes de SharedMemory contêm dois pontos. Patch por Rani Pinchuk.

- gh-138425 [https://github.com/python/cpython/issues/138425]: Corrige avaliação parcial de objetos `annotationlib.ForwardRef` que dependem de nomes definidos como globais.

- gh-138151 [https://github.com/python/cpython/issues/138151]: Em `annotationlib`, aprimora-se a avaliação de referências futuras a variáveis não locais que ainda não estão definidas quando as anotações são avaliadas inicialmente.

- gh-69528 [https://github.com/python/cpython/issues/69528]: The `mode` attribute of files opened in the `'wb+'` mode is now `'wb+'` instead of `'rb+'`.

- gh-137627 [https://github.com/python/cpython/issues/137627]: Speed up `csv.Sniffer.sniff()` delimiter detection by up to 1.6x.

- gh-55531 [https://github.com/python/cpython/issues/55531]: `encodings`: Improve `normalize_encoding()` performance by implementing the function in C using the private `_Py_normalize_encoding` which has been modified to make lowercase conversion optional.

- gh-136057 [https://github.com/python/cpython/issues/136057]: Corrigido o bug em `pdb` e `bdb` onde `next` e `step` não conseguiam ultrapassar a linha se existisse um laço na linha.

- gh-133390 [https://github.com/python/cpython/issues/133390]: Support table, index, trigger, view, column, function, and schema completion for `sqlite3`'s command-line interface.

- gh-135307 [https://github.com/python/cpython/issues/135307]: `email`: Corrige a exceção em `set_content()` ao codificar texto quando max_line_length está definido como `0` ou `None` (ilimitado).

- gh-133789 [https://github.com/python/cpython/issues/133789]: Corrige a desserialização com pickle de objetos `pathlib` que foram serializados

com pickle no Python 3.13.

- gh-133601 [https://github.com/python/cpython/issues/133601]: Remove deprecated **`typing.no_type_check_decorator()`**.

- gh-132686 [https://github.com/python/cpython/issues/132686]: Add parameters *inherit_class_doc* and *fallback_to_class_doc* for **`inspect.getdoc()`**.

- gh-131116 [https://github.com/python/cpython/issues/131116]: **`inspect.getdoc()`** now correctly returns an inherited docstring on **`cached_property`** objects if none is given in a subclass.

- gh-130693 [https://github.com/python/cpython/issues/130693]: Add support for `-nolinestop`, and `-strictlimits` options to **`tkinter.Text.search()`**. Also add the **`tkinter.Text.search_all()`** method for `-all` and `-overlap` options.

- gh-122255 [https://github.com/python/cpython/issues/122255]: In the **`linecache`** module and in the Python implementation of the **`warnings`** module, a `DeprecationWarning` is issued when `mod.__loader__` differs from `mod.__spec__.loader` (like in the C implementation of the **`warnings`** module).

- gh-121011 [https://github.com/python/cpython/issues/121011]: **`math.log()`** now supports arbitrary large integer-like arguments in the same way as arbitrary large integer arguments.

- gh-119668 [https://github.com/python/cpython/issues/119668]: Publicly expose and document **`importlib.machinery.NamespacePath`**.

- gh-102431 [https://github.com/python/cpython/issues/102431]: Esclarece restrições para argumentos “lógicos” em métodos de **`decimal.Context`**.

- gh-81313 [https://github.com/python/cpython/issues/81313]: Add the **`math.integer`** module (**PEP 791** [https://peps.python.org/pep-0791/]).

Núcleo e embutidos

- gh-141579 [https://github.com/python/cpython/issues/141579]: Corrigido o problema na função `sys.activate_stack_trampoline()` para que ela ofereça suporte correto ao backend `perf_jit`. Patch por Pablo Galindo.
- gh-114203 [https://github.com/python/cpython/issues/114203]: Ignora a trava se o objeto já estiver travado por uma seção crítica de dois mutexes.
- gh-141528 [https://github.com/python/cpython/issues/141528]: Sugere o uso de `concurrent.interpreters.Interpreter.close()` em vez da função privada `_interpreters.destroy` ao emitir avisos sobre subinterpretadores restantes. Patch por Sergey Miryanov.
- gh-141367 [https://github.com/python/cpython/issues/141367]: Specialize `CALL_LIST_APPEND` instruction only for lists, not for list subclasses, to avoid unnecessary deopt. Patch by Mikhail Efimov.
- gh-141312 [https://github.com/python/cpython/issues/141312]: Corrige a falha de asserção no método `__setstate__` do iterador de intervalo quando um argumento não inteiro é passado. Patch por Sergey Miryanov.
- gh-140643 [https://github.com/python/cpython/issues/140643]: Add support for `<GC>` and `<native>` frames to **`profiling.sampling`** output to denote active garbage collection and calls to native code.

Biblioteca

- gh-140942 [https://github.com/python/cpython/issues/140942]: Add `.cjs` to `mimetypes` to give CommonJS modules a MIME type of `application/node`.

Núcleo e embutidos

- gh-140479 [https://github.com/python/cpython/issues/140479]: Update JIT compilation to use LLVM 21 at build time.
- gh-140939 [https://github.com/python/cpython/issues/140939]: Corrige vazamento de memória quando `bytearray` ou `bytes` é formatado com o formato `%*b` com uma largura grande que resulta em uma `MemoryError`.

Biblioteca

- gh-140260 [https://github.com/python/cpython/issues/140260]: Correção de condição de corrida de dados em `struct` na inicialização de tabelas endian com subinterpretadores. Patch por Shamil Abdulaev.

Núcleo e embutidos

- gh-140530 [https://github.com/python/cpython/issues/140530]: Corrige um vazamento de referência que ocorria quando `raise exc from cause` falhava. Patch por Bénédikt Tran.

Biblioteca

- gh-90344 [https://github.com/python/cpython/issues/90344]: Replace `io.IncrementalNewlineDecoder` with non incremental newline decoders in codebase where `io.IncrementalNewlineDecoder.decode()` was being called once.

Núcleo e embutidos

- gh-140373 [https://github.com/python/cpython/issues/140373]: Correctly emit `PY_UNWIND` event when generator object is closed. Patch by Mikhail Efimov.
- gh-140729 [https://github.com/python/cpython/issues/140729]: Fix pickling error in the sampling profiler when using `concurrent.futures.ProcessPoolExecutor` script can not be properly pickled and executed in worker processes.
- gh-131527 [https://github.com/python/cpython/issues/131527]: Dynamic borrow checking for stackrefs is added to `Py_STACKREF_DEBUG` mode. Patch by Mikhail Efimov.
- gh-140576 [https://github.com/python/cpython/issues/140576]: Corrigido o erro que causava o fechamento inesperado do código em `tokenize.generate_tokens()` em caso de entrada incorreta específica. Patch por Mikhail Efimov.
- gh-140544 [https://github.com/python/cpython/issues/140544]: Speed up accessing interpreter state by caching it in a thread local variable. Patch by Kumar Aditya.

- gh-140551 [https://github.com/python/cpython/issues/140551]: Corrigido o problema que causava o fechamento inesperado do arquivo `dict` se o método `dict.clear()` fosse chamado na fase de busca. Patch por Mikhail Efimov e Inada Naoki.
- gh-140517 [https://github.com/python/cpython/issues/140517]: Corrigido um vazamento de referência ao iterar sobre o resultado de `map()` com `strict=True` quando os iteráveis de entrada tinham comprimentos diferentes. Patch por Mikhail Efimov.
- gh-133467 [https://github.com/python/cpython/issues/133467]: Fix race when updating `type.__bases__` that could allow a read of `type.__base__` to observe an inconsistent value on the free threaded build.
- gh-140471 [https://github.com/python/cpython/issues/140471]: Corrige possível estouro de buffer na inicialização do nó `ast.AST` ao encontrar `_fields` malformados contendo não-`str`.

Biblioteca

- gh-140443 [https://github.com/python/cpython/issues/140443]: The logarithm functions (such as `math.log10()` and `math.log()`) may now produce slightly different results for extremely large integers that cannot be converted to floats without overflow. These results are generally more accurate, with reduced worst-case error and a tighter overall error distribution.

Núcleo e embutidos

- gh-140431 [https://github.com/python/cpython/issues/140431]: Corrige uma falha no coletor de lixo do Python devido a objetos corrotina parcialmente inicializados quando a profundidade do rastreamento da origem da corrotina está habilitada (`sys.set_coroutine_origin_tracking_depth()`).
- gh-140476 [https://github.com/python/cpython/issues/140476]: Optimize `PySet_Add()` for `frozenset` in free threaded build.
- gh-135904 [https://github.com/python/cpython/issues/135904]: Add special labels to the assembly created during stencil creation to support relocations that the native object file format does not support. Specifically, 19 bit branches for AArch64 in Mach-O object files.

Biblioteca

- gh-140398 [https://github.com/python/cpython/issues/140398]: Corrige vazamento de memória nas funções `read_init_file()`, `read_history_file()`, `write_history_file()` e `append_history_file()` do módulo `readline` quando `PySys_Audit()` falha.

Núcleo e embutidos

- gh-140406 [https://github.com/python/cpython/issues/140406]: Corrige vazamento de memória quando o método `__hash__()` de um objeto retorna um objeto que não é um `int`.
- gh-140358 [https://github.com/python/cpython/issues/140358]: Restaura o tempo decorrido e a contagem de objetos inacessíveis na saída de depuração do GC. Esses dados foram removidos inadvertidamente durante uma refatoração de `gc.c`. O log de depuração agora relata novamente o tempo de coleta decorrido e o número de objetos inacessíveis. Contribuição de Pål Grønås Drange.
- gh-136895 [https://github.com/python/cpython/issues/136895]: Update JIT compilation to use LLVM 20 at build time.
- gh-139109 [https://github.com/python/cpython/issues/139109]: A new tracing frontend for the JIT compiler has been implemented. Patch by Ken Jin. Design for CPython by Ken Jin, Mark Shannon and Brandt Bucher.
- gh-140306 [https://github.com/python/cpython/issues/140306]: Corrige vazamentos de memória em operações de canal entre interpretadores e manipulação de espaço de nomes compartilhado.
- gh-116738 [https://github.com/python/cpython/issues/116738]: Make _suggestions module thread-safe on the free threaded build.
- gh-140301 [https://github.com/python/cpython/issues/140301]: Corrige vazamento de memória de `PyConfig` em subinterpretadores.
- gh-140257 [https://github.com/python/cpython/issues/140257]: Corrige corrida de dados entre interpreter_clear() e take_gil() em eval_breaker durante a finalização com threads daemon.
- gh-139951 [https://github.com/python/cpython/issues/139951]: Corrige uma regressão no desempenho do GC para um heap crescente composto principalmente de pequenas tuplas.

    - Conta o número de objetos realmente rastreados, em vez de objetos rastreáveis. Isso garante que o cancelamento do rastreamento de tuplas tenha o efeito desejado de reduzir a sobrecarga do coletor de lixo.
    - Não rastreia a maioria das tuplas não rastreáveis durante a criação. Isso evita que um grande número de tuplas pequenas causem GCs excessivos.
- gh-140253 [https://github.com/python/cpython/issues/140253]: Wrong placement of a double-star pattern inside a mapping pattern now throws a specialized syntax error. Contributed by Bartosz Sławecki in gh-140253 [https://github.com/python/cpython/issues/140253].
- gh-140104 [https://github.com/python/cpython/issues/140104]: Correção de um bug no tratamento de exceções no JIT. Patch por Ken Jin. Bug relatado por Daniel Diniz.
- gh-140149 [https://github.com/python/cpython/issues/140149]: Speed up parsing bytes literals concatenation by using PyBytesWriter API and a single memory allocation (about 3x faster).
- gh-140061 [https://github.com/python/cpython/issues/140061]: Correção da verificação de se um objeto é referenciado exclusivamente para garantir compatibilidade com threads livres. Patch por Sergey Miryanov.
- gh-140080 [https://github.com/python/cpython/issues/140080]: Fix hang during finalization when attempting to call `atexit` handlers under no memory.
- gh-139871 [https://github.com/python/cpython/issues/139871]: Update `bytearray` to use a `bytes` under the hood as its buffer and add `bytearray.take_bytes()` to take it out.
- gh-140067 [https://github.com/python/cpython/issues/140067]: Corrige vazamento de memória na criação de subinterpretadores.
- gh-139914 [https://github.com/python/cpython/issues/139914]: Restaura o suporte para HP PA-RISC, que possui uma pilha crescente.
- gh-139817 [https://github.com/python/cpython/issues/139817]: Attribute `__qualname__` is added to `typing.TypeAliasType`. Patch by Mikhail Efimov.
- gh-135801 [https://github.com/python/cpython/issues/135801]: Many functions related to compiling or parsing Python code, such as `compile()`, `ast.parse()`, `symtable.symtable()`, and `importlib.abc.InspectLoader.source_to_code()` now allow to

specify the module name. It is needed to unambiguous filter syntax warnings by module name.

- gh-139640 [https://github.com/python/cpython/issues/139640]: **`ast.parse()`** não emite mais avisos de sintaxe para `return`/`break`/`continue` em `finally` (veja **PEP 765** [https://peps.python.org/pep-0765/]) – eles são emitidos apenas durante a compilação.
- gh-139640 [https://github.com/python/cpython/issues/139640]: Corrige a ingestão de alguns avisos de sintaxe em módulos diferentes caso eles acidentalmente tenham a mesma mensagem e sejam emitidos na mesma linha. Corrige avisos duplicados no bloco `finally`.
- gh-139475 [https://github.com/python/cpython/issues/139475]: Changes in stackref debugging mode when `Py_STACKREF_DEBUG` is set. We use the same pattern of refcounting for stackrefs as in production build.
- gh-139269 [https://github.com/python/cpython/issues/139269]: Fix undefined behavior when using unaligned store in JIT's `patch_*` functions.
- gh-138944 [https://github.com/python/cpython/issues/138944]: Corrige a mensagem **`SyntaxError`** que ocorre quando uma sintaxe inválida aparece na mesma linha que uma instrução `import ... as ...` ou `from ... import ... as ...` válida. Patch por Brian Schubert.
- gh-138857 [https://github.com/python/cpython/issues/138857]: Improve **`SyntaxError`** message for `case` keyword placed outside **`match`** body.
- gh-131253 [https://github.com/python/cpython/issues/131253]: Support the `--enable-pystats` build option for the free-threaded build.
- gh-136327 [https://github.com/python/cpython/issues/136327]: Errors when calling functions with invalid values after `*` and `**` now do not include the function name. Patch by Ilia Solin.
- gh-134786 [https://github.com/python/cpython/issues/134786]: If **`Py_TPFLAGS_MANAGED_DICT`** and **`Py_TPFLAGS_MANAGED_WEAKREF`** are used, then **`Py_TPFLAGS_HAVE_GC`** must be used as well.

API C

- gh-141341 [https://github.com/python/cpython/issues/141341]: On Windows, rename the `COMPILER` macro to `_Py_COMPILER` to avoid name conflicts. Patch by Victor Stinner.
- gh-116146 [https://github.com/python/cpython/issues/116146]: Add a new **`PyImport_CreateModuleFromInitfunc()`** C-API for creating a module

from a *spec* and *initfunc*. Patch by Itamar Oren.

- gh-141042 [https://github.com/python/cpython/issues/141042]: Faz qNaN em **`PyFloat_Pack2()`** e **`PyFloat_Pack4()`**, se durante a conversão para um formato de ponto flutuante de menor precisão — o valor restante após o truncamento for zero. Patch por Sergey B Kirpichev.
- gh-141004 [https://github.com/python/cpython/issues/141004]: **`Py_MATH_El`** and **`Py_MATH_PIl`** are deprecated.
- gh-141004 [https://github.com/python/cpython/issues/141004]: The **`Py_INFINITY`** macro is soft deprecated.
- gh-140556 [https://github.com/python/cpython/issues/140556]: **PEP 793** [https://peps.python.org/pep-0793/]: Add a new entry point for C extension modules, `PyModExport_<modulename>`.
- gh-140487 [https://github.com/python/cpython/issues/140487]: Corrige **`Py_RETURN_NOTIMPLEMENTED`** na API C limitada 3.11 e anteriores: não trata `Py_NotImplemented` como imortal. Patch por Victor Stinner.
- gh-140153 [https://github.com/python/cpython/issues/140153]: Correção da definição **`Py_REFCNT()`** na API C limitada 3.11-3.13. Patch por Victor Stinner.
- gh-139653 [https://github.com/python/cpython/issues/139653]: Adicionadas as funções **`PyUnstable_ThreadState_SetStackProtection()`** e **`PyUnstable_ThreadState_ResetStackProtection()`** para definir o endereço base e o tamanho da proteção de pilha de um estado de thread Python. Patch por Victor Stinner.

Construção

- gh-140454 [https://github.com/python/cpython/issues/140454]: When building the JIT, match the jit_stencils filename expectations in Makefile with the generator script. This avoid needless JIT recompilation during `make install`.
- gh-140768 [https://github.com/python/cpython/issues/140768]: Avisa quando a versão do SDK do WASI não corresponder à versão suportada.
- gh-140513 [https://github.com/python/cpython/issues/140513]: Gera um erro de compilação claro quando `_Py_TAIL_CALL_INTERP` estiver habilitado, mas `preserve_none` ou `musttail` não forem suportados.
- gh-140475 [https://github.com/python/cpython/issues/140475]: Support WASI SDK 25.

- gh-140239 [https://github.com/python/cpython/issues/140239]: Check `statx` availability only on Linux (including Android).
- gh-140189 [https://github.com/python/cpython/issues/140189]: As versões para iOS foram adicionadas à integração contínua.
- gh-137618 [https://github.com/python/cpython/issues/137618]: `PYTHON_FOR_REGEN` agora requer Python 3.10 a Python 3.15. Patch por Adam Turner.

Python 3.15.0 alpha 1

Release date: 2025-10-14

macOS

- gh-115119 [https://github.com/python/cpython/issues/115119]: Update macOS installer to use libmpdecimal 4.0.1.
- gh-124111 [https://github.com/python/cpython/issues/124111]: Update macOS installer to use Tcl/Tk 9.0.2.
- gh-132339 [https://github.com/python/cpython/issues/132339]: Update macOS installer version of OpenSSL to 3.5.4.
- gh-137450 [https://github.com/python/cpython/issues/137450]: macOS instalador console melhorias no gerenciamento de caminhos: separe o script pós-instalação do instalador `console profile updater` do `Update console Profile.comando` para permitir um manuseio mais robusto do erro.
- gh-137134 [https://github.com/python/cpython/issues/137134]: Atualização do instalador do macOS para ser fornecido com o SQLite versão 3.50.4.

Windows

- gh-139810 [https://github.com/python/cpython/issues/139810]: A instalação com `py install 3[.x]-dev` agora selecionará versões finais e também pré-lançamentos.
- gh-139573 [https://github.com/python/cpython/issues/139573]: Atualizada a versão empacotada do OpenSSL para 3.0.18.
- gh-138896 [https://github.com/python/cpython/issues/138896]: Corrige erro ao instalar o runtime do C em máquinas Windows não atualizadas

- gh-138314 [https://github.com/python/cpython/issues/138314]: Add **`winreg.DeleteTree()`**.
- gh-137136 [https://github.com/python/cpython/issues/137136]: Suppress build warnings when build on Windows with `--experimental-jit-interpreter`.
- gh-137134 [https://github.com/python/cpython/issues/137134]: Atualizar o instalador do Windows para que seja fornecido com o SQLite 3.50.4.
- gh-135099 [https://github.com/python/cpython/issues/135099]: Corrige uma falha que poderia ocorrer no Windows quando uma thread em segundo plano aguarda um **`PyMutex`** enquanto o thread principal está desligando o interpretador.
- gh-130727 [https://github.com/python/cpython/issues/130727]: Corrige uma corrida em chamadas internas para o WMI que pode resultar em uma exceção de “identificador inválido” sob carga alta. Patch por Chris Eibl.
- gh-76023 [https://github.com/python/cpython/issues/76023]: Faz com que **`os.path.realpath()`** ignore o erro 1005 do Windows quando estiver no modo não estrito.
- gh-133779 [https://github.com/python/cpython/issues/133779]: Reverte a alteração para gerar arquivos `pyconfig.h` diferentes com base nas configurações do compilador, pois isso frequentemente causava falhas nas construções de extensões. Em particular, a variável de pré-processador `Py_GIL_DISABLED` agora deve ser sempre definida explicitamente ao compilar para o tempo de execução experimental de threads livres. A função **`sysconfig.get_config_var()`** pode ser usada para determinar se o tempo de execução atual foi compilado com esse sinalizador ou não.
- gh-133626 [https://github.com/python/cpython/issues/133626]: Garante que os pacotes não sejam acidentalmente agrupados no instalador tradicional.
- gh-133580 [https://github.com/python/cpython/issues/133580]: Fix **`sys.getwindowsversion()`** failing without setting an exception when called on some WinAPI partitions.
- gh-133572 [https://github.com/python/cpython/issues/133572]: Avoid LsaNtStatus to WinError conversion on unsupported WinAPI partitions.
- gh-133568 [https://github.com/python/cpython/issues/133568]: Fix compile error when using a WinAPI partition that doesn’t support the RPC runtime library.

- gh-133562 [https://github.com/python/cpython/issues/133562]: Disable handling of security descriptors by `os.mkdir()` with mode `0o700` on WinAPI partitions that do not support it. This only affects custom builds for specialized targets.
- gh-133537 [https://github.com/python/cpython/issues/133537]: Avoid using console I/O in WinAPI partitions that don't support it
- gh-131942 [https://github.com/python/cpython/issues/131942]: Use the Python-specific `Py_DEBUG` macro rather than `_DEBUG` in Windows-related C code. Patch by Xuehai Pan.

Ferramentas/Demos

- gh-139330 [https://github.com/python/cpython/issues/139330]: A ferramenta de geração de SBOM não verificava os valores de versão e checksum em relação ao script `Modules/expat/refresh.sh`, o que levava a que os valores ficassem desatualizados durante atualizações de rotina.
- gh-132006 [https://github.com/python/cpython/issues/132006]: XCframeworks agora incluem manifestos de privacidade para atender aos requisitos de envio da Apple App Store.
- gh-138171 [https://github.com/python/cpython/issues/138171]: Foi adicionado um script para a construção de um framework iOS XC. Como parte dessa alteração, a pasta de nível superior `iOS` foi movida para um subdiretório da pasta `Apple`.
- gh-137873 [https://github.com/python/cpython/issues/137873]: O executor de testes iOS foi simplificado, resolvendo alguns problemas que foram observados ao usar o executor nos ambientes de teste do GitHub Actions e do Azure Pipelines.
- gh-137484 [https://github.com/python/cpython/issues/137484]: Coloca o `Tools/wasm/wasi` no Python de construção em um diretório com o nome do triplo de construção em vez de "build".
- gh-137025 [https://github.com/python/cpython/issues/137025]: The `wasm_build.py` script has been removed. `Tools/wasm/emscripten` and `Tools/wasm/wasi` should be used instead, as described in the Dev Guide [https://devguide.python.org/contrib/workflows/compile/].
- gh-137248 [https://github.com/python/cpython/issues/137248]: Adiciona uma opção `--logdir` em `Tools/wasm/wasi` para especificar onde gravar os arquivos de log.

- gh-137243 [https://github.com/python/cpython/issues/137243]: Faz com que o Tools/wasm/wasi detecte uma instalação do WASI SDK em /opt quando ele foi extraído diretamente de um tarball de lançamento.
- gh-136251 [https://github.com/python/cpython/issues/136251]: Correções e melhorias de usabilidade para `Tools/wasm/emscripten/web_example`
- gh-135968 [https://github.com/python/cpython/issues/135968]: Os stubs para `strip` agora são fornecidos como parte de uma instalação do iOS.
- gh-135379 [https://github.com/python/cpython/issues/135379]: The cases generator no longer accepts type annotations on stack items. Conversions to non-default types are now done explicitly in bytecodes.c and optimizer_bytecodes.c. This will simplify code generation for top-of-stack caching and other future features.
- gh-134215 [https://github.com/python/cpython/issues/134215]: Autopreenchimento de importação do REPL sugere apenas módulos privados quando explicitamente especificado.

Testes

- gh-139208 [https://github.com/python/cpython/issues/139208]: Corrige o regrtest `--fast-ci --verbose`: não ignora mais a opção `--verbose`. Patch por Victor Stinner.
- gh-138313 [https://github.com/python/cpython/issues/138313]: Restore skipped test and add janky workaround to prevent select buildbots from failing with a ResourceWarning.
- gh-135966 [https://github.com/python/cpython/issues/135966]: O ambiente de teste do iOS agora manipula a pasta `app_packages` como um diretório de site.
- gh-135494 [https://github.com/python/cpython/issues/135494]: Corrige o regrtest para suportar a exclusão de testes de `--pgo`. Patch por Victor Stinner.
- gh-132815 [https://github.com/python/cpython/issues/132815]: Corrige test__opcode: adiciona `JUMP_BACKWARD` às estatísticas de especialização.
- gh-135489 [https://github.com/python/cpython/issues/135489]: Exibe saída detalhada para testes com falha durante a etapa de criação de perfil do PGO com –enable-optimizations.
- gh-135401 [https://github.com/python/cpython/issues/135401]: Add a new GitHub CI job to test the **`ssl`** module with AWS-LC [https://github.com/aws/aws-lc] as

the backing cryptography and TLS library.

- gh-135120 [https://github.com/python/cpython/issues/135120]: Adiciona **`test.support.subTests()`**.
- gh-134567 [https://github.com/python/cpython/issues/134567]: Expose log formatter to users in TestCase.assertLogs. **`unittest.TestCase.assertLogs()`** will now optionally accept a formatter that will be used to format the strings in output if provided.
- gh-133744 [https://github.com/python/cpython/issues/133744]: Corrigido o teste de interrupção de multiprocessamento. Adicionado um evento para sincronizar o processo pai com o processo filho: aguardar até que o processo filho comece a hibernar. Patch por Victor Stinner.
- gh-133682 [https://github.com/python/cpython/issues/133682]: Corrigido caso de teste `test.test_annotationlib.TestStringFormat.test_displays` que garante o tratamento adequado de estruturas de dados complexas (listas, conjuntos, dicionários e tuplas) em anotações de string.
- gh-133639 [https://github.com/python/cpython/issues/133639]: Corrige `TestPyReplAutoindent.test_auto_indent_default()` não executar `input_code`.

Segurança

- gh-139700 [https://github.com/python/cpython/issues/139700]: Verifica a consistência do registro zip64 no final do diretório central. Suporte a registros com “dados extensíveis zip64” se não houver bytes anexados ao arquivo ZIP.

- gh-139400 [https://github.com/python/cpython/issues/139400]: **`xml.parsers.expat`**: Garante que os analisadores Expat pais sejam coletados como lixo somente quando não forem mais referenciados por subanalisadores criados por **`ExternalEntityParserCreate()`**. Patch por Sebastian Pipping.

- gh-139283 [https://github.com/python/cpython/issues/139283]: **`sqlite3`**: manipula corretamente o número máximo de linhas a serem buscadas em **`Cursor.fetchmany`** e rejeita valores negativos para **`Cursor.arraysize`**. Patch por Bénédikt Tran.

- gh-136053 [https://github.com/python/cpython/issues/136053]: **`marshal`**: corrige uma possível falha ao desserializar objetos **`slice`**.

- gh-135661 [https://github.com/python/cpython/issues/135661]: Corrige a análise das tags de início e fim em **`html.parser.HTMLParser`** de acordo com o padrão HTML5.

    - Espaços em branco não são mais aceitos entre `</` e o nome da tag. Por exemplo, `</ script>` não encerra a seção de script.
    - Tabulação vertical (`\v`) e espaços em branco não ASCII não são mais reconhecidos como espaços em branco. Os únicos espaços em branco são `\t\n\r\f` e espaço.
    - O caractere nulo (U+0000) não termina mais o nome da tag.
    - Atributos e barras após o nome da tag em tags finais agora são ignorados, em vez de terminar após o primeiro `>` no valor do atributo entre aspas. Por exemplo, `</script/foo=">"/>`.
    - Barras múltiplas e espaços em branco entre o último atributo e o `>` de fechamento agora são ignorados nas tags de início e fim. Por exemplo, `<a foo=bar/ //>`.
    - Múltiplos `=` entre o nome do atributo e o valor não são mais recolhidos. Por exemplo, `<a foo==bar>` produz o atributo “foo” com o valor “=bar”.

- gh-135661 [https://github.com/python/cpython/issues/135661]: Corrige a análise da seção CDATA em **`html.parser.HTMLParser`** de acordo com o padrão HTML5: `] ]>` e `]] >` não encerram mais a seção CDATA. Adiciona o método privado `_set_support_cdata()`, que pode ser usado para especificar como analisar `<[CDATA[` — como uma seção CDATA em conteúdo externo (SVG ou MathML) ou como um comentário falso no espaço de nomes HTML.

- gh-102555 [https://github.com/python/cpython/issues/102555]: Corrige a análise de comentários em **`html.parser.HTMLParser`** de acordo com o padrão HTML5. `--!>` agora encerra o comentário. `-- >` não encerra mais o comentário. Suporte a comentários vazios `<-->` e `<--->` que encerravam anormalmente.

- gh-135462 [https://github.com/python/cpython/issues/135462]: Corrigida a complexidade quadrática no processamento de entradas especialmente criadas em `html.parser.HTMLParser`. Erros de fim de arquivo agora são tratados de acordo com as especificações do HTML5 – comentários e declarações são fechados automaticamente, tags são ignoradas.

- gh-118350 [https://github.com/python/cpython/issues/118350]: Corrige suporte ao modo de texto bruto escapável (elementos “textarea” e “title”) em `html.parser.HTMLParser`.

- gh-135034 [https://github.com/python/cpython/issues/135034]: Corrige vários problemas que permitiam que os filtros de extração `tarfile` (`filter="data"` e `filter="tar"`) fossem ignorados usando links simbólicos e links físicos criados.

  Endereços **CVE 2024-12718** [https://www.cve.org/CVERecord?id=CVE-2024-12718], **CVE 2025-4138** [https://www.cve.org/CVERecord?id=CVE-2025-4138], **CVE 2025-4330** [https://www.cve.org/CVERecord?id=CVE-2025-4330] e **CVE 2025-4517** [https://www.cve.org/CVERecord?id=CVE-2025-4517].

- gh-133767 [https://github.com/python/cpython/issues/133767]: Corrige uso após liberação no decodificador “unicode-escape” com um tratador de erros não “estrito”.

- gh-133623 [https://github.com/python/cpython/issues/133623]: Indicate through `ssl.HAS_PSK_TLS13` whether the `ssl` module supports “External PSKs” in TLSv1.3, as described in RFC 9258. Patch by Will Childs-Klein.

- gh-128840 [https://github.com/python/cpython/issues/128840]: Faz um curto-circuito no processamento de endereços IPv6 longos no início de `ipaddress` para evitar consumo excessivo de memória e uma pequena negação de serviço.

Biblioteca

- gh-139482 [https://github.com/python/cpython/issues/139482]: Optimize `os.environ.clear()` by calling *clearenv(3)* *[https://manpages.debian.org/clearenv(3)]* when this function is available. Patch by Victor Stinner.

- gh-139958 [https://github.com/python/cpython/issues/139958]: The `application/toml` mime type is now supported by `mimetypes`. Patch by Gil Forcada.

- gh-139823 [https://github.com/python/cpython/issues/139823]: `ensurepip` now fails with a nicer error message when the `zlib` module is not available.

- gh-139905 [https://github.com/python/cpython/issues/139905]: Adiciona sugestão à mensagem de erro para subclasses `typing.Generic` quando `cls.__parameters__` estiver ausente devido a uma classe pai falhar ao chamar `super().__init_subclass__()` em sua `__init_subclass__`.

- gh-139894 [https://github.com/python/cpython/issues/139894]: Corrige compartilhamento incorreto da tarefa atual com o processo filho durante o fork em `asyncio`. Patch por Kumar Aditya.

- gh-139845 [https://github.com/python/cpython/issues/139845]: Correção para não exibir KeyboardInterrupt duas vezes no REPL padrão de asyncio.

- gh-139783 [https://github.com/python/cpython/issues/139783]: Corrige `inspect.getsourcelines()` para o caso em que um decorador é seguido por um comentário ou uma linha vazia.

- gh-139809 [https://github.com/python/cpython/issues/139809]: Evita a colorização prematura do subanalisador `prog` em `argparse.ArgumentParser.add_subparsers()` para respeitar as alterações da variável de ambiente de cor após a criação do analisador.

- gh-139736 [https://github.com/python/cpython/issues/139736]: Corrige recuo excessivo no `argparse` padrão `HelpFormatter`. Patch por Alexander Edland.

- gh-70765 [https://github.com/python/cpython/issues/70765]: **`http.server`**: corrige o tratamento padrão de requisições HTTP/0.9 em **`BaseHTTPRequestHandler`**. Anteriormente, **`BaseHTTPRequestHandler.parse_request()`** aguardava incorretamente por cabeçalhos na requisição, embora estes não sejam suportados em HTTP/0.9. Patch por Bénédikt Tran.

- gh-139322 [https://github.com/python/cpython/issues/139322]: Fix **`os.getlogin()`** error handling: fix the error number. Patch by Victor Stinner.

- gh-135953 [https://github.com/python/cpython/issues/135953]: Add a Gecko format output to the tachyon profiler via `--gecko`.

- gh-139184 [https://github.com/python/cpython/issues/139184]: **`os.forkpty()`** does now make the returned file descriptor non-inheritable.

- gh-139391 [https://github.com/python/cpython/issues/139391]: Corrige um problema em que, em plataformas não Windows, não era possível sair normalmente de um processo `python -m asyncio` suspenso por Ctrl+Z e posteriormente retomado por *fg* *[https://manpages.debian.org/fg]*, exceto com *kill* *[https://manpages.debian.org/kill]*.

- gh-139374 [https://github.com/python/cpython/issues/139374]: **`timeit`**: Add color to error tracebacks.

- gh-90949 [https://github.com/python/cpython/issues/90949]: Add **`SetBillionLaughsAttackProtectionActivationThreshold()`** and **`SetBillionLaughsAttackProtectionMaximumAmplification()`** to xmlparser objects to tune protections against billion laughs [https://en.wikipedia.org/wiki/Billion_laughs_attack] attacks. Patch by Bénédikt Tran.

- gh-139312 [https://github.com/python/cpython/issues/139312]: Atualiza libexpat empacotado para 2.7.3

- gh-139289 [https://github.com/python/cpython/issues/139289]: Faz uma importação preguiçosa real em **`rlcompleter`** em **`pdb`** e restaura o completer existente após importar **`rlcompleter`**.

- gh-90949 [https://github.com/python/cpython/issues/90949]: Add `SetAllocTrackerActivationThreshold()` and `SetAllocTrackerMaximumAmplification()` to xmlparser objects to tune protections against disproportional amounts of dynamic memory usage from within an Expat parser. Patch by Bénédikt Tran.

- gh-67795 [https://github.com/python/cpython/issues/67795]: Functions that take timestamp or timeout arguments now accept any real numbers (such as `Decimal` and `Fraction`), not only integers or floats, although this does not improve precision.

- gh-95953 [https://github.com/python/cpython/issues/95953]: A CSS class, `diff_changed`, was added to the changed lines in the `make_table` output of `difflib.HtmlDiff`. Patch by Katie Gardner.

- gh-139210 [https://github.com/python/cpython/issues/139210]: Corrige o problema de uso após liberação ao relatar um evento desconhecido em `xml.etree.ElementTree.iterparse()`. Patch por Ken Jin.

- gh-138860 [https://github.com/python/cpython/issues/138860]: Importação preguiçosa de `rlcompleter` em `pdb` para evitar impasse no subprocesso.

- gh-112729 [https://github.com/python/cpython/issues/112729]: Corrige falha ao chamar `concurrent.interpreters.create()` quando o processo está sem memória.

- gh-126016 [https://github.com/python/cpython/issues/126016]: Fix an assertion failure when sending `KeyboardInterrupt` to a Python process running a subinterpreter in a separate thread.

- gh-118803 [https://github.com/python/cpython/issues/118803]: `collections.abc.ByteString` has been removed from `collections.abc.__all__`, and `typing.ByteString` has been removed from `typing.__all__`. The former has been deprecated since Python 3.12, and the latter has been deprecated since Python 3.9. Both classes are scheduled for removal in Python 3.17.

Additionally, the following statements now cause `DeprecationWarnings` to be emitted at runtime: `from collections.abc import ByteString`, `from typing import ByteString`, `import collections.abc; collections.abc.ByteString` and `import typing; typing.ByteString`. Both classes already caused `DeprecationWarnings` to be emitted if they were subclassed or used as the second argument to `isinstance()` or `issubclass()`, but they did not previously lead to `DeprecationWarnings` if they were merely imported or accessed from their respective modules.

- gh-135729 [https://github.com/python/cpython/issues/135729]: Corrige exceção não levantável durante a finalização ao usar **`concurrent.interpreters`** no REPL.

- gh-139076 [https://github.com/python/cpython/issues/139076]: Corrige um bug no módulo **`pydoc`** que estava escondendo funções em um módulo Python se elas fossem implementadas em um módulo de extensão e o módulo não tivesse `__all__`.

- gh-139090 [https://github.com/python/cpython/issues/139090]: Add **`os.RWF_DONTCACHE`** constant for Linux 6.14+.

- gh-139065 [https://github.com/python/cpython/issues/139065]: Corrige espaço final antes de uma palavra longa encapsulada se o comprimento da linha for exatamente *width* em **`textwrap`**.

- gh-139001 [https://github.com/python/cpython/issues/139001]: Corrige condição de corrida em **`pathlib.Path`** no campo interno `_raw_paths`.

- gh-138813 [https://github.com/python/cpython/issues/138813]: **`multiprocessing.BaseProcess`** define `kwargs` como `None` em vez de um dicionário compartilhado.

- gh-138998 [https://github.com/python/cpython/issues/138998]: Atualiza libexpat empacotado para 2.7.2

- gh-138993 [https://github.com/python/cpython/issues/138993]: Reduz indentação do texto de `credits`.

- gh-118803 [https://github.com/python/cpython/issues/118803]: Adiciona novamente `collections.abc.ByteString` e `typing.ByteString`. Ambas foram removidas nas versões alfa, beta e candidatos a lançamento anteriores do Python 3.14, mas sua remoção foi adiada para o Python 3.17.

- gh-130567 [https://github.com/python/cpython/issues/130567]: Corrige possível falha em `locale.strxfrm()` devido a um bug de plataforma no macOS.

- gh-137226 [https://github.com/python/cpython/issues/137226]: Corrige chamadas `typing.get_type_hints()` em classes genéricas `typing.TypedDict` definidas com anotações de string.

- gh-138899 [https://github.com/python/cpython/issues/138899]: Executing `quit` command in `pdb` will raise `bdb.BdbQuit` when **`pdb`** is started from an asyncio console using `breakpoint()` or `pdb.set_trace()`.

- gh-138804 [https://github.com/python/cpython/issues/138804]: Levanta `TypeError` em vez de `AttributeError` quando um argumento de tipo incorreto for passado para `shlex.quote()`. Isso restaura o comportamento da função anterior à versão 3.14.

- gh-138779 [https://github.com/python/cpython/issues/138779]: Suporte a números de dispositivos maiores que `2**63-1` para o campo `st_rdev` da estrutura `os.stat_result`.

- gh-138682 [https://github.com/python/cpython/issues/138682]: Added symmetric difference support to `collections.Counter` objects.

- gh-138712 [https://github.com/python/cpython/issues/138712]: Add `os.NODEV`.

- gh-128636 [https://github.com/python/cpython/issues/128636]: Corrige falha no PyREPL quando os.environ é substituído por um valor inválido para Mac

- gh-138720 [https://github.com/python/cpython/issues/138720]: Fix an issue where `io.BufferedWriter` and `io.BufferedRandom` had different definitions of “closed” for `close()` and `flush()` which resulted in an exception when close called flush but flush thought the file was already closed.

- gh-138706 [https://github.com/python/cpython/issues/138706]: Update `unicodedata` database to Unicode 17.0.0.

- gh-76007 [https://github.com/python/cpython/issues/76007]: Deprecate `__version__` from a number of standard library modules. Patch by Hugo van Kemenade.

- gh-138535 [https://github.com/python/cpython/issues/138535]: Speed up `os.stat()` for files with reasonable timestamps. Contributed by Jeffrey Bosboom.

- gh-116946 [https://github.com/python/cpython/issues/116946]: `curses.panel`: the type of `curses.panel.new_panel()` is now immutable. Patch by Bénédikt Tran.

- gh-116946 [https://github.com/python/cpython/issues/116946]: `zlib`: the types of `zlib.compressobj()` and `zlib.decompressobj()` are now immutable. Patch by Bénédikt Tran.

- gh-116946 [https://github.com/python/cpython/issues/116946]: `os`: the `os.DirEntry` type and the type of `os.scandir()` are now immutable. Patch by Bénédikt Tran.

- gh-116946 [https://github.com/python/cpython/issues/116946]: `tkinter`: the types `_tkinter.Tcl_Obj` (wrapper for Tcl objects), `_tkinter.tktimertoken` (obtained by calling `createtimerhandler()` on a `Tk` application) and `_tkinter.tkapp` (the runtime type of Tk applications) are now immutable. Patch by Bénédikt Tran.

- gh-138514 [https://github.com/python/cpython/issues/138514]: Levanta `ValueError` quando uma string com vários caracteres é passada para o parâmetro *echo_char* de `getpass.getpass()`. Patch de Benjamin Johnson.

- gh-137706 [https://github.com/python/cpython/issues/137706]: Corrige a avaliação parcial de anotações que usam `typing.Annotated[T, x]` onde `T` é uma referência direta.

- gh-88375 [https://github.com/python/cpython/issues/88375]: Corrige a normalização das regras e URLs `robots.txt` no módulo **`urllib.robotparser`**. Não ignora mais o `?` final. Distingue os caracteres especiais brutos `?`, `=` e `&` dos codificados em porcentagem.

- gh-138515 [https://github.com/python/cpython/issues/138515]: **`email`** é adicionado à construção Emscripten.

- gh-99948 [https://github.com/python/cpython/issues/99948]: **`ctypes.util.find_library()`** agora funciona em construções Emscripten.

- gh-111788 [https://github.com/python/cpython/issues/111788]: Corrige erros de análise no módulo **`urllib.robotparser`**. Não falhe ao tentar analisar caminhos estranhos. Não falha ao tentar decodificar arquivos `robots.txt` não UTF-8.

- gh-138432 [https://github.com/python/cpython/issues/138432]: **`zoneinfo.reset_tzpath()`** agora converterá quaisquer objetos **`os.PathLike`** recebidos em strings antes de adicioná-los a `TZPATH`. Ele levantará `TypeError` se algo diferente de uma string for encontrado após essa conversão. Se for fornecido um objeto **`os.PathLike`** que represente um caminho relativo, ele levantará `ValueError` em vez de `TypeError` e apresentará uma mensagem de erro mais informativa.

- gh-132657 [https://github.com/python/cpython/issues/132657]: Improve the scaling of **`copy.copy()`** and **`copy.deepcopy()`** in the free-threading build.

- gh-116946 [https://github.com/python/cpython/issues/116946]: The types of **`select.poll()`** and **`select.epoll()`** objects are now immutable. Patch by Bénédikt Tran.

- gh-116946 [https://github.com/python/cpython/issues/116946]: The **`_random.Random`** C type is now immutable. Patch by Bénédikt Tran.

- gh-57911 [https://github.com/python/cpython/issues/57911]: When extracting tar files on Windows, slashes in symlink targets will be replaced by backslashes to prevent corrupted links.

- gh-138205 [https://github.com/python/cpython/issues/138205]: Removed the `resize()` method on platforms that don't support the underlying syscall, instead of raising a `SystemError`.

- gh-138008 [https://github.com/python/cpython/issues/138008]: Corrige falhas de segmentação no módulo `ctypes` devido a `argtypes` inválido. Patch por Dung Nguyen.

- gh-138252 [https://github.com/python/cpython/issues/138252]: `ssl`: `SSLContext` objects can now set client and server TLS signature algorithms. If Python has been built with OpenSSL 3.5 or later, `SSLSocket` objects can return the signature algorithms selected on a connection.

- gh-138253 [https://github.com/python/cpython/issues/138253]: Adiciona o parâmetro *block* nos métodos `put()` e `get()` das filas `concurrent.interpreters` para compatibilidade com a interface `queue.Queue`.

- gh-60462 [https://github.com/python/cpython/issues/60462]: Corrige `locale.strxfrm()` no Solaris (e possivelmente em outras plataformas).

- gh-138239 [https://github.com/python/cpython/issues/138239]: O REPL agora realça `type` como uma palavra reservada contextual em instruções de tipo.

- gh-78502 [https://github.com/python/cpython/issues/78502]: `mmap.mmap` now has a *trackfd* parameter on Windows; if it is `False`, the file handle corresponding to *fileno* will not be duplicated.

- gh-138204 [https://github.com/python/cpython/issues/138204]: Proibe expansão de `mapas de memória` anônimos compartilhados no Linux, o que causou um erro de barramento.

- gh-138010 [https://github.com/python/cpython/issues/138010]: Fix an issue where defining a class with a `@warnings.deprecated`-decorated base class may not invoke the correct `__init_subclass__()` method in cases involving multiple inheritance. Patch by Brian Schubert.

- gh-134716 [https://github.com/python/cpython/issues/134716]: Add support of regular expressions in the `-W` option and the `PYTHONWARNINGS` environment variable.

- gh-138133 [https://github.com/python/cpython/issues/138133]: Evita laço infinito de traceback ao enviar CTRL^C para Python através de `strace`.

- gh-138122 [https://github.com/python/cpython/issues/138122]: Implement **PEP 799** [https://peps.python.org/pep-0799/] – A dedicated profiling package for organizing Python profiling tools. Patch by Pablo Galindo.

- gh-138092 [https://github.com/python/cpython/issues/138092]: Fixed a bug in `mmap.mmap.flush()` where calling with only an offset parameter would fail.

- gh-138044 [https://github.com/python/cpython/issues/138044]: Remove compatibility shim for deprecated parameter *package* in `importlib.resources.files()`. Patch by Semyon Moroz.

- gh-86819 [https://github.com/python/cpython/issues/86819]: `socket`: Add missing constants for ISO-TP sockets.

- gh-137884 [https://github.com/python/cpython/issues/137884]: Add `threading.get_native_id()` support for Illumos/Solaris. Patch by Yüce Tekol.

- gh-134869 [https://github.com/python/cpython/issues/134869]: Correção de um problema em que pressionar Ctrl+C durante o preenchimento de guias no site REPL deixava o menu de preenchimento automático em um estado corrompido.

- gh-137840 [https://github.com/python/cpython/issues/137840]: `typing.TypedDict` now supports the `closed` and `extra_items` keyword arguments (as

described in **PEP 728** [https://peps.python.org/pep-0728/]) to control whether additional non-required keys are allowed and to specify their value type.

- gh-132947 [https://github.com/python/cpython/issues/132947]: Applied changes to `importlib.metadata` from importlib_metadata 8.7 [https://importlib-metadata.readthedocs.io/en/latest/history.html#v8-7-0], including `dist` now disallowed for `EntryPoints.select`; deferred imports for faster import times; added support for metadata with newlines (python/cpython#119650); and `metadata()` function now returns `None` when a metadata directory is present but no metadata is present.

- gh-90548 [https://github.com/python/cpython/issues/90548]: Corrige a detecção de `musl` para **`platform.libc_ver()`** no Alpine Linux se compilado com –strip-all.

- gh-137317 [https://github.com/python/cpython/issues/137317]: **`inspect.signature()`** agora trata corretamente classes que usam um descritor em um método encapsulado **`__init__()`** ou **`__new__()`**. Contribuição de Yongyu Yan.

- gh-137754 [https://github.com/python/cpython/issues/137754]: Corrige a importação do módulo **`zoneinfo`** se a implementação C do módulo **`datetime`** não estiver disponível.

- gh-125854 [https://github.com/python/cpython/issues/125854]: Improve error messages for invalid category in **`warnings.warn()`**.

- gh-137729 [https://github.com/python/cpython/issues/137729]: **`locale.setlocale()`** now supports language codes with `@`-modifiers. `@`-modifier are no longer silently removed in **`locale.getlocale()`**, but included in the language code.

- gh-73487 [https://github.com/python/cpython/issues/73487]: Speedup processing arguments (up to 1.5x) in the **`decimal`** module methods, that now using **`METH_FASTCALL`** calling convention. Patch by Sergey B Kirpichev.

- gh-137634 [https://github.com/python/cpython/issues/137634]: Calendar pages generated by the `calendar.HTMLCalendar` class now support dark mode and have been migrated to the HTML5 standard for improved accessibility.

- gh-137630 [https://github.com/python/cpython/issues/137630]: The `_interpreters` module now uses Argument Clinic to parse arguments. Patch by Adam Turner.

- gh-137583 [https://github.com/python/cpython/issues/137583]: Fix a deadlock introduced in 3.13.6 when a call to `ssl.SSLSocket.recv` was blocked in one thread, and then another method on the object (such as `ssl.SSLSocket.send`) was subsequently called in another thread.

- gh-92936 [https://github.com/python/cpython/issues/92936]: Update regex used by `http.cookies.SimpleCookie` to handle values containing double quotes.

- gh-137426 [https://github.com/python/cpython/issues/137426]: Remove a descontinuação de código de `importlib.abc.ResourceLoader`. Ele está documentado como descontinuado, mas foi mantido pela retrocompatibilidade com outras classes em `importlib.abc`.

- gh-137490 [https://github.com/python/cpython/issues/137490]: Manipula `ECANCELED` da mesma forma que `EINTR` em `signal.sigwaitinfo()` no NetBSD.

- gh-137512 [https://github.com/python/cpython/issues/137512]: Add new constants in the `resource` module: `RLIMIT_NTHR`, `RLIMIT_UMTXP`, `RLIMIT_PIPEBUF`, `RLIMIT_THREADS`, `RLIM_SAVED_CUR`, and `RLIM_SAVED_MAX`.

- gh-137044 [https://github.com/python/cpython/issues/137044]: `resource.RLIM_INFINITY` is now always a positive integer. On all supported platforms, it is larger than any limited resource value, which simplifies comparison of the resource values. Previously, it could be negative, such as -1 or -3, depending on platform.

- gh-137477 [https://github.com/python/cpython/issues/137477]: Corrige **`inspect.getblock()`**, **`inspect.getsourcelines()`** e **`inspect.getsource()`** para expressões geradoras.

- gh-137481 [https://github.com/python/cpython/issues/137481]: Calendar uses the lengths of the locale's weekdays to decide if the width requires abbreviation.

- gh-137466 [https://github.com/python/cpython/issues/137466]: Remove undocumented **`glob.glob0()`** and **`glob.glob1()`** functions, which have been deprecated since Python 3.13. Use **`glob.glob()`** and pass a directory to its *root_dir* argument instead.

- gh-137044 [https://github.com/python/cpython/issues/137044]: Retorna valores de limite grandes como inteiros positivos em vez de inteiros negativos em **`resource.getrlimit()`**. Aceita valores grandes e rejeita valores negativos (exceto **`RLIM_INFINITY`**) para limites em **`resource.setrlimit()`**.

- gh-115766 [https://github.com/python/cpython/issues/115766]: Fix **`ipaddress.IPv4Interface.is_unspecified`**.

- gh-75989 [https://github.com/python/cpython/issues/75989]: **`tarfile.TarFile.extractall()`** e **`tarfile.TarFile.extract()`** agora substituem links simbólicos ao extrair links físicos. (Contribuição de Alexander Enrique Urieles Nieto em gh-75989 [https://github.com/python/cpython/issues/75989].)

- gh-137017 [https://github.com/python/cpython/issues/137017]: Corrige **`threading.Thread.is_alive`** para permanecer `True` até que a thread do SO subjacente seja completamente limpa. Isso evita falsos negativos em casos extremos envolvendo monitoramento de threads ou chamadas prematuras de **`threading.Thread.is_alive`**.

- gh-137273 [https://github.com/python/cpython/issues/137273]: Corrige falha de asserção de depuração em **`locale.setlocale()`** no Windows.

- gh-137191 [https://github.com/python/cpython/issues/137191]: Fix how type parameters are collected, when `typing.Protocol` are specified with explicit parameters. Now, `typing.Generic` and `typing.Protocol` always dictate the parameter number and parameter ordering of types. Previous behavior was a bug.

- gh-137282 [https://github.com/python/cpython/issues/137282]: Corrige o preenchimento por tabulação e `dir()` em `concurrent.futures`.

- gh-137257 [https://github.com/python/cpython/issues/137257]: Atualiza a versão do pip empacotado no ensurepip para a versão 25.2.

- gh-137239 [https://github.com/python/cpython/issues/137239]: `heapq`: Atualiza `heapq.__all__` com funções `*_max`.

- gh-124503 [https://github.com/python/cpython/issues/124503]: `ast.literal_eval()` is 10-20% faster for small inputs.

- gh-137226 [https://github.com/python/cpython/issues/137226]: Corrige o comportamento de `annotationlib.ForwardRef.evaluate()` quando o parâmetro *type_params* é passado e o nome de um parâmetro de tipo também está presente em um escopo envolvente.

- gh-137197 [https://github.com/python/cpython/issues/137197]: `SSLContext` objects can now set TLS 1.3 cipher suites via `set_ciphersuites()`.

- gh-81325 [https://github.com/python/cpython/issues/81325]: `tarfile.TarFile` agora aceita um caminho ou similar ao trabalhar em um arquivo tar. (Contribuição de Alexander Enrique Urieles Nieto em gh-81325 [https://github.com/python/cpython/issues/81325].)

- gh-137185 [https://github.com/python/cpython/issues/137185]: Corrige um possível problema de segurança de sinal assíncrono em `faulthandler` ao exibir stack traces (situação da pilha de execução).

- gh-133951 [https://github.com/python/cpython/issues/133951]: Remove lib64-lib symlink creation when creating new virtual environments in `venv` module

- gh-130522 [https://github.com/python/cpython/issues/130522]: Corrige **`TypeError`** não levantável levantada durante desligamento do interpretador no módulo **`threading`**.

- gh-137059 [https://github.com/python/cpython/issues/137059]: Corrige o tratamento de URLs de arquivo com uma letra de unidade do Windows na autoridade de URL por **`urllib.request.url2pathname()`**. Isso corrige uma regressão em versões anteriores do Python 3.14.

- gh-136980 [https://github.com/python/cpython/issues/136980]: Remove unused C tracing code in bdb for event type `c_call`, `c_return` and `c_exception`

- gh-130577 [https://github.com/python/cpython/issues/130577]: **`tarfile`** agora valida arquivos para garantir que os deslocamentos dos membros não sejam negativos. (Contribuição de Alexander Enrique Urieles Nieto em gh-130577 [https://github.com/python/cpython/issues/130577].)

- gh-136170 [https://github.com/python/cpython/issues/136170]: Foi removida a propriedade não lançada `zipfile.ZipFile.data_offset` adicionada na versão 3.14.0a7, pois não estava totalmente claro qual comportamento ela deveria ter em algumas situações, então o resultado nem sempre era o que o usuário esperava.

- gh-121237 [https://github.com/python/cpython/issues/121237]: Support `%:z` directive for **`datetime.datetime.strptime()`**, **`datetime.time.strptime()`** and **`time.strptime()`**. Patch by Lucas Esposito and Semyon Moroz.

- gh-136929 [https://github.com/python/cpython/issues/136929]: Ensure that hash functions guaranteed to be always *available* exist as attributes of **`hashlib`** even if they will not work at runtime due to missing backend implementations. For instance, `hashlib.md5` will no longer raise **`AttributeError`** if OpenSSL is not available and Python has been built without MD5 support. Patch by Bénédikt Tran.

- gh-124621 [https://github.com/python/cpython/issues/124621]: pyrepl agora funciona em Emscripten.

- gh-136914 [https://github.com/python/cpython/issues/136914]: Corrige recuperação de `doctest.DocTest.lineno` para objetos decorados com `functools.cache()` ou `functools.cached_property`.

- gh-136912 [https://github.com/python/cpython/issues/136912]: `hmac.digest()` agora lida corretamente com chaves e mensagens grandes, recorrendo à implementação pura do Python quando necessário. Patch por Bénédikt Tran.

- gh-83424 [https://github.com/python/cpython/issues/83424]: Permite criar um `ctypes.CDLL` sem nome ao passar um identificador como argumento.

- gh-135228 [https://github.com/python/cpython/issues/135228]: Quando `dataclasses` substitui uma classe por uma dataclass com slot, a classe original agora pode ser coletada como lixo novamente. Alterações anteriores no Python 3.14 fizeram com que essa classe permanecesse sempre em existência, juntamente com a classe substituta sintetizada por `dataclasses`.

- gh-136874 [https://github.com/python/cpython/issues/136874]: Descarta consulta e fragmento de URL em `urllib.request.url2pathname()`.

- gh-136787 [https://github.com/python/cpython/issues/136787]: `hashlib`: improve exception messages when a hash algorithm is not recognized, blocked by the current security policy or incompatible with the desired operation (for instance, using HMAC with SHAKE). Patch by Bénédikt Tran.

- gh-130645 [https://github.com/python/cpython/issues/130645]: Habilita ajuda de cores por padrão em `argparse`.

- gh-131724 [https://github.com/python/cpython/issues/131724]: In `http.client`, a new *max_response_headers* keyword-only parameter has been added to `HTTPConnection` and `HTTPSConnection` constructors. This parameter sets the maximum number of allowed response headers, helping to prevent denial-of-service attacks.

- gh-135427 [https://github.com/python/cpython/issues/135427]: With `-Werror`, the DeprecationWarning emitted by `os.fork()` and `os.forkpty()` in mutli-threaded processes is now raised as an exception. Previously it was silently ignored. Patch by Rani Pinchuk.

- gh-136234 [https://github.com/python/cpython/issues/136234]: Corrige `asyncio.WriteTransport.writelines()` para ser robusto a falhas de conexão, usando o mesmo comportamento de `write()`.

- gh-53144 [https://github.com/python/cpython/issues/53144]: **`encodings.aliases`**: Add `latin_N` aliases

- gh-136669 [https://github.com/python/cpython/issues/136669]: **`_asyncio`** is now statically linked for improved performance.

- gh-136134 [https://github.com/python/cpython/issues/136134]: **`SMTP.auth_cram_md5()`** agora levanta uma `SMTPException` em vez de uma `ValueError` se o Python tiver sido construído sem suporte a MD5. Em particular, clientes `SMTP` não tentarão usar este método, mesmo que se presuma que o servidor remoto o suporte. Patch por Bénédikt Tran.

- gh-136134 [https://github.com/python/cpython/issues/136134]: `IMAP4.login_cram_md5` agora levanta `IMAP4.error` se a autenticação CRAM-MD5 não for suportada. Patch por Bénédikt Tran.

- gh-136591 [https://github.com/python/cpython/issues/136591]: **`_hashlib`**: avoid using deprecated functions *ERR_func_error_string* *[https://manpages.debian.org/ERR_func_error_string]* and *EVP_MD_CTX_md* *[https://manpages.debian.org/EVP_MD_CTX_md]* when using OpenSSL 3.0 and later. Patch by Bénédikt Tran.

- gh-136571 [https://github.com/python/cpython/issues/136571]: `datetime.date.fromisocalendar()` can now raise OverflowError for out of range arguments.

- gh-136549 [https://github.com/python/cpython/issues/136549]: Corrige assinatura de `threading.excepthook()`.

- gh-136492 [https://github.com/python/cpython/issues/136492]: Expose **PEP 667** [https://peps.python.org/pep-0667/]'s **`FrameLocalsProxyType`** in the **`types`** module.

- gh-83336 [https://github.com/python/cpython/issues/83336]: `utf8_sig` is now aliased to **`encodings.utf_8_sig`**

- gh-136523 [https://github.com/python/cpython/issues/136523]: Corrige **`wave.Wave_write`** emitindo uma exceção não levantável quando open a levanta.

- gh-136507 [https://github.com/python/cpython/issues/136507]: Corrige o CLI dos tipos MIME para manipular vários parâmetros de arquivo.

- gh-52876 [https://github.com/python/cpython/issues/52876]: Adiciona o parâmetro `keepends` ausente (padrão `True`) para **`codecs.StreamReaderWriter.readline()`** e **`codecs.StreamReaderWriter.readlines()`**.

- gh-136470 [https://github.com/python/cpython/issues/136470]: Corrige o nome da thread padrão de **`concurrent.futures.InterpreterPoolExecutor`**.

- gh-136476 [https://github.com/python/cpython/issues/136476]: Corrige um bug que estava fazendo com que a função `get_async_stack_trace` perdesse alguns quadros no stack trace (situação da pilha de execução).

- gh-136434 [https://github.com/python/cpython/issues/136434]: Corrige geração de documentos de `UnboundItem` em **`concurrent.interpreters`** ao executar com **`-OO`**.

- gh-136380 [https://github.com/python/cpython/issues/136380]: LEvanta **`AttributeError`** ao acessar **`concurrent.futures.InterpreterPoolExecutor`** e subinterpretadores não estão disponíveis.

- gh-72327 [https://github.com/python/cpython/issues/72327]: Suggest using the system command prompt when `pip install` is typed into the REPL. Patch by Tom Viner, Richard Si, and Brian Schubert.

- gh-135953 [https://github.com/python/cpython/issues/135953]: Implement a new high-frequency runtime profiler that leverages the existing remote debugging functionality to collect detailed execution statistics from running Python processes. This tool is exposed in the `profile.sample` module and enables non-intrusive observation of production applications by attaching to already-running processes without requiring any code modifications, restarts, or special startup flags. The observer can perform extremely high-frequency sampling of stack traces and interpreter state, providing detailed runtime execution analysis of live applications.

- gh-136021 [https://github.com/python/cpython/issues/136021]: Make `type_params` parameter required in **`typing._eval_type()`** after a deprecation period for not providing this parameter. Also remove the **`DeprecationWarning`** for the old behavior.

- gh-136286 [https://github.com/python/cpython/issues/136286]: Fix pickling failures for protocols 0 and 1 for many objects related to subinterpreters.

- gh-136047 [https://github.com/python/cpython/issues/136047]: Fix issues with **`typing`** when the C implementation of **`abc`** is not available.

- gh-136316 [https://github.com/python/cpython/issues/136316]: Melhora o suporte para avaliar referências aninhadas em **`typing.evaluate_forward_ref()`**.

- gh-136306 [https://github.com/python/cpython/issues/136306]: **`ssl`** can now get and set groups used for key agreement.

- gh-136156 [https://github.com/python/cpython/issues/136156]: **`tempfile.TemporaryFile()`** no longer uses **`os.O_EXCL`** with **`os.O_TMPFILE`**, so it's possible to use `linkat()` on the file descriptor. Patch by Victor Stinner.

- gh-133982 [https://github.com/python/cpython/issues/133982]: Update Python implementation of **`io.BytesIO`** to be thread safe.

- gh-136193 [https://github.com/python/cpython/issues/136193]: Improve `TypeError` error message, when richcomparing two `types.SimpleNamespace` objects.

- gh-136097 [https://github.com/python/cpython/issues/136097]: Fix potential infinite recursion and KeyError in `sysconfig --generate-posix-vars`.

- gh-85702 [https://github.com/python/cpython/issues/85702]: Se `zoneinfo._common.load_tzdata` receber um pacote sem um recurso, um `zoneinfo.ZoneInfoNotFoundError` será levantada em vez de uma `PermissionError`. Patch por Victor Stinner.

- gh-90733 [https://github.com/python/cpython/issues/90733]: Improve error messages when reporting invalid parameters in `hashlib.scrypt()`. Patch by Bénédikt Tran.

- gh-134759 [https://github.com/python/cpython/issues/134759]: Corrige `UnboundLocalError` em `email.message.Message.get_payload()` quando o payload a ser decodificado for um objeto `bytes`. Patch por Kliment Lamonov.

- gh-136028 [https://github.com/python/cpython/issues/136028]: Corrige a análise de nomes de meses contendo “İ” (U+0130, LETRA I MAIÚSCULA COM PONTO EM CIMA) em `time.strptime()`. Isso afeta as localidades az_AZ, ber_DZ, ber_MA e crh_UA.

- gh-87135 [https://github.com/python/cpython/issues/87135]: Acquiring a `threading.Lock` or `threading.RLock` at interpreter shutdown will raise `PythonFinalizationError` if Python can determine that it would otherwise deadlock.

- gh-135995 [https://github.com/python/cpython/issues/135995]: Na codificação palmos, faz o byte `0x9b` ser decodificado para › (U+203A - ASPAS ÚNICAS DE ÂNGULO À DIREITA).

- gh-105456 [https://github.com/python/cpython/issues/105456]: Removed **`sre_compile`**, **`sre_constants`** and **`sre_parse`** modules.

- gh-53203 [https://github.com/python/cpython/issues/53203]: Corrige **`time.strptime()`** para os formatos `%c` e `%x` nos locais byn_ER, wal_ET e lzh_TW, e para o formato `%X` nos locais ar_SA, bg_BG e lzh_TW.

- gh-135878 [https://github.com/python/cpython/issues/135878]: Corrige uma falha de **`types.SimpleNamespace`** em construções com threads livres, quando várias threads estavam chamando seu método **`__repr__()`** ao mesmo tempo.

- gh-135853 [https://github.com/python/cpython/issues/135853]: Add **`math.fmax()`** and **`math.fmin()`** to get the larger and smaller of two floating-point values. Patch by Bénédikt Tran.

- gh-135836 [https://github.com/python/cpython/issues/135836]: Corrige **`IndexError`** em **`asyncio.loop.create_connection()`** que poderia ocorrer quando uma exceção não-**`OSError`** fosse levantada durante a conexão e o `close()` do soquete levantasse **`OSError`**.

- gh-135853 [https://github.com/python/cpython/issues/135853]: **`math`**: expose C99 **`signbit()`** function to determine whether the sign bit of a floating-point value is set. Patch by Bénédikt Tran.

- gh-134531 [https://github.com/python/cpython/issues/134531]: **`hmac`**: use the *EVP_MAC(3ssl)* *[https://manpages.debian.org/EVP_MAC(3ssl)]* interface for HMAC when Python is built with OpenSSL 3.0 and later instead of the deprecated *HMAC_CTX(3ssl)* *[https://manpages.debian.org/hmac(3)]* interface. Patch by Bénédikt Tran.

- gh-135836 [https://github.com/python/cpython/issues/135836]: Corrige **`IndexError`** em **`asyncio.loop.create_connection()`** que poderia ocorrer quando o algoritmo Happy Eyeballs resultava em uma lista de exceções vazia durante tentativas de conexão.

- gh-135855 [https://github.com/python/cpython/issues/135855]: Levanta **`TypeError`** em vez de **`SystemError`** quando **`_interpreters.set___main___attrs()`** recebe um objeto não dict. Patch por Brian Schubert.

- gh-135823 [https://github.com/python/cpython/issues/135823]: **netrc**: improve the error message when the security check for the ownership of the default configuration file `~/.netrc` fails. Patch by Bénédikt Tran.

- gh-135815 [https://github.com/python/cpython/issues/135815]: **netrc**: Ignora as verificações de segurança se **os.getuid()** estiver ausente. Patch por Bénédikt Tran.

- gh-135640 [https://github.com/python/cpython/issues/135640]: Corrigido um bug que permitia chamar **xml.etree.ElementTree.ElementTree.write()** em um objeto ElementTree com um elemento raiz inválido. Esse comportamento apagava o arquivo passado para `write` caso ele já existisse.

- gh-90117 [https://github.com/python/cpython/issues/90117]: Speed up **pprint** for **list** and **tuple**.

- gh-135759 [https://github.com/python/cpython/issues/135759]: **hashlib**: reject negative digest lengths in OpenSSL-based SHAKE objects by raising a **ValueError**. Previously, negative lengths were implicitly rejected by raising a **MemoryError** or a **SystemError**. Patch by Bénédikt Tran.

- gh-123471 [https://github.com/python/cpython/issues/123471]: Make concurrent iterations over **itertools.chain** safe under free threading.

- gh-135645 [https://github.com/python/cpython/issues/135645]: Adicionado campo `supports_isolated_interpreters` a **sys.implementation**.

- gh-135646 [https://github.com/python/cpython/issues/135646]: Levanta exceções consistentes **NameError** em **annotationlib.ForwardRef.evaluate()**

- gh-135557 [https://github.com/python/cpython/issues/135557]: Corrige corridas em atualizações de **heapq** e leituras de **list** na construção com threads livres.

- gh-119180 [https://github.com/python/cpython/issues/119180]: Busca globais e locais somente se necessário em **annotationlib.get_annotations()**

- gh-135561 [https://github.com/python/cpython/issues/135561]: Corrige uma falha em construções DEBUG quando uma rotina HACL* HMAC falha. Patch por Bénédikt Tran.

- gh-135386 [https://github.com/python/cpython/issues/135386]: Corrige a abertura de um banco de dados `dbm.sqlite3` para leitura de um arquivo ou diretório somente leitura.

- gh-135444 [https://github.com/python/cpython/issues/135444]: Corrige `asyncio.DatagramTransport.sendto()` para levar em conta o tamanho do cabeçalho do datagrama quando os dados não podem ser enviados.

- gh-65697 [https://github.com/python/cpython/issues/65697]: A mensagem de erro do `configparser` ao tentar escrever uma chave inválida agora é mais útil.

- gh-135497 [https://github.com/python/cpython/issues/135497]: Corrige falha em `os.getlogin()` para nomes de usuários mais longos em plataformas baseadas em BSD.

- gh-135487 [https://github.com/python/cpython/issues/135487]: Corrige `reprlib.Repr.repr_int()` quando são fornecidos inteiros com mais de `sys.get_int_max_str_digits()` dígitos. Patch por Bénédikt Tran.

- gh-135429 [https://github.com/python/cpython/issues/135429]: Corrige a incompatibilidade de argumentos em `_lsprof` para o evento `PY_THROW`.

- gh-135368 [https://github.com/python/cpython/issues/135368]: Corrige a geração de `unittest.mock.Mock` em objetos `dataclasses.dataclass()`. Agora, todos os atributos especiais são definidos como antes de gh-124429 [https://github.com/python/cpython/issues/124429].

- gh-135336 [https://github.com/python/cpython/issues/135336]: `json` now encodes strings up to 2.2x faster if they consist solely of characters that don’t require escaping.

- gh-135335 [https://github.com/python/cpython/issues/135335]: **multiprocessing**: Limpa `stdout` e `stderr` após pré-carregar módulos no `forkserver`.

- gh-126631 [https://github.com/python/cpython/issues/126631]: Corrige bug no `forkserver` do **multiprocessing** que impedia `__main__` de ser pré-carregado.

- gh-133967 [https://github.com/python/cpython/issues/133967]: Não normaliza o nome **locale** 'C.UTF-8' para 'en_US.UTF-8'.

- gh-130870 [https://github.com/python/cpython/issues/130870]: Preserve **types.GenericAlias** subclasses in **typing.get_type_hints()**

- gh-135321 [https://github.com/python/cpython/issues/135321]: Levanta uma exceção correta para valores maiores que 0x7fffffff para o opcode `BINSTRING` na implementação C de **pickle**.

- gh-121914 [https://github.com/python/cpython/issues/121914]: Changed the names of the symbol tables for lambda expressions and generator expressions to "<lambda>" and "<genexpr>" respectively to avoid conflicts with user-defined names.

- gh-135276 [https://github.com/python/cpython/issues/135276]: Synchronized zipfile.Path with zipp 3.23, including improved performance of **zipfile.Path.open()** for non-reading modes, rely on **functools.cached_property()** to cache values on the instance. Rely on `save_method_args` to save the initialization method arguments. Fixed `.name`, `.stem` and other basename-based properties on Windows when working with a zipfile on disk.

- gh-135234 [https://github.com/python/cpython/issues/135234]: **hashlib**: improve exception messages when an OpenSSL function failed. When memory allocation fails on OpenSSL's side, a **MemoryError** is raised instead of a **ValueError**. Patch by Bénédikt Tran.

- gh-135244 [https://github.com/python/cpython/issues/135244]: **uuid**: quando o endereço MAC não pode ser determinado, o ID do nó de 48 bits agora é gerado com um gerador de números pseudoaleatórios

criptograficamente seguro (CSPRNG), conforme **RFC 9562, §6.10.3** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-6.10-3]. Isso afeta **`uuid1()`** e **`uuid6()`**.

- gh-135241 [https://github.com/python/cpython/issues/135241]: The `INT` opcode of the C accelerator **`_pickle`** module was updated to look only for "00" and "01" to push booleans onto the stack, aligning with the Python **`pickle`** module.

- gh-133934 [https://github.com/python/cpython/issues/133934]: Improve **`sqlite3`** CLI's `.help` message.

- gh-135069 [https://github.com/python/cpython/issues/135069]: Corrige a exceção "Invalid error handling" em **`encodings.idna.IncrementalDecoder`** para substituir corretamente o parâmetro 'errors'.

- gh-130662 [https://github.com/python/cpython/issues/130662]: Aceita zeros à esquerda em campos de precisão e largura para formatação **`Decimal`**, por exemplo `format(Decimal(1.25), '.016f')`.

- gh-130662 [https://github.com/python/cpython/issues/130662]: Aceita zeros à esquerda em campos de precisão e largura para formatação **`Fraction`**, por exemplo `format(Fraction(1, 3), '.016f')`.

- gh-135004 [https://github.com/python/cpython/issues/135004]: Rewrite and cleanup the internal **`_blake2`** module. Some exception messages were changed but their types were left untouched. Patch by Bénédikt Tran.

- gh-134953 [https://github.com/python/cpython/issues/134953]: Expande o tema `_colorize` com `keyword_constant` e implementa no repl.

- gh-134978 [https://github.com/python/cpython/issues/134978]: **`hashlib`**: Supporting the `string` keyword parameter in hash function constructors such as **`new()`** or the direct hash-named constructors such as **`md5()`** and **`sha256()`** is now deprecated and slated for removal in Python 3.19. Prefer passing the initial data as a positional argument for maximum backwards compatibility. Patch by Bénédikt Tran.

- gh-134970 [https://github.com/python/cpython/issues/134970]: Corrige a exceção “ação desconhecida” em `argparse.ArgumentParser.add_argument_group()` para substituir corretamente a classe de ação.

- gh-134718 [https://github.com/python/cpython/issues/134718]: By default, omit optional `Load()` values in `ast.dump()`.

- gh-134718 [https://github.com/python/cpython/issues/134718]: `ast.dump()` agora só omite os valores `None` e `[]` se forem valores padrão.

- gh-134939 [https://github.com/python/cpython/issues/134939]: Adiciona o módulo `concurrent.interpreters`. Veja **PEP 734** [https://peps.python.org/pep-0734/].

- gh-108885 [https://github.com/python/cpython/issues/108885]: Run each example as a subtest in unit tests synthesized by `doctest.DocFileSuite()` and `doctest.DocTestSuite()`. Add the `doctest.DocTestRunner.report_skip()` method.

- gh-134885 [https://github.com/python/cpython/issues/134885]: Corrige possível falha no módulo `compression.zstd` relacionada à configuração de tipos de parâmetros. Patch por Jelle Zijlstra.

- gh-134857 [https://github.com/python/cpython/issues/134857]: Melhoria no relatório de erros para `doctest` executado com `unittest`. Remoção de quadros do módulo `doctest` de tracebacks e caracteres de quebra de linha redundantes de mensagens de falha.

- gh-128840 [https://github.com/python/cpython/issues/128840]: Corrige a análise de endereços IPv6 longos com endereço IPv4 incorporado.

- gh-133579 [https://github.com/python/cpython/issues/133579]: `curses`: Consistently report failures of curses C API calls in module-level methods by raising a `curses.error`. This affects `assume_default_colors()`, `baudrate()`, `cbreak()`, `echo()`, `longname()`, `initscr()`, `nl()`, `raw()`, `termattrs()`, `termname()` and `unctrl()`. Patch by Bénédikt Tran.

- gh-133579 [https://github.com/python/cpython/issues/133579]: **`curses.window.refresh()`** and **`curses.window.noutrefresh()`** now raise a **`TypeError`** instead of **`curses.error`** when called with an incorrect number of arguments for pads. Patch by Bénédikt Tran.

- gh-133579 [https://github.com/python/cpython/issues/133579]: curses.window: Consistently report failures of curses C API calls in Window methods by raising a **`curses.error`**. This affects **`addch()`**, **`addnstr()`**, **`addstr()`**, **`border()`**, **`box()`**, **`chgat()`**, **`getbkgd()`**, **`inch()`**, **`insstr()`** and **`insnstr()`**. Patch by Bénédikt Tran.

- gh-134771 [https://github.com/python/cpython/issues/134771]: The `time_clockid_converter()` function now selects correct type for `clockid_t` on Cygwin which fixes a build error.

- gh-134637 [https://github.com/python/cpython/issues/134637]: Corrige regressão de desempenho ao chamar um ponteiro de função **`ctypes`** em threads livres.

- gh-134696 [https://github.com/python/cpython/issues/134696]: Implementações HACL* e OpenSSL embutidas de construtores de funções hash agora aceitam corretamente os mesmos argumentos nomeados *documentados*. Por exemplo, **`md5()`** podia ser invocado anteriormente como `md5(data=data)` ou `md5(string=string)` dependendo da implementação subjacente, mas essas chamadas não eram compatíveis. Patch por Bénédikt Tran.

- gh-132710 [https://github.com/python/cpython/issues/132710]: Se possível, garante que **`uuid.getnode()`** retorne o mesmo resultado mesmo em processos diferentes. Anteriormente, o resultado era constante apenas dentro do mesmo processo. Patch por Bénédikt Tran.

- gh-134531 [https://github.com/python/cpython/issues/134531]: **`_hashlib`**: Rename internal C functions for **`_hashlib.HASH`** and **`_hashlib.HASHXOF`** objects. Patch by Bénédikt Tran.

- gh-134698 [https://github.com/python/cpython/issues/134698]: Corrige uma falha ao chamar métodos de **`ssl.SSLContext`** ou **`ssl.SSLSocket`** em várias

threads.

- gh-134151 [https://github.com/python/cpython/issues/134151]: `email`: Corrige `TypeError` em `email.utils.decode_params()` ao classificar continuações **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] que contêm uma seção não numerada.

- gh-134635 [https://github.com/python/cpython/issues/134635]: `zlib`: Allow to combine Adler-32 and CRC-32 checksums via `adler32_combine()` and `crc32_combine()`. Patch by Callum Attryde and Bénédikt Tran.

- gh-134657 [https://github.com/python/cpython/issues/134657]: `asyncio`: Remove alguns nomes privados de `asyncio.__all__`.

- gh-134210 [https://github.com/python/cpython/issues/134210]: `curses.window.getch()` agora manipula sinais corretamente. Patch por Bénédikt Tran.

- gh-80334 [https://github.com/python/cpython/issues/80334]: `multiprocessing.freeze_support()` agora verifica o trabalho em qualquer plataforma de método de inicialização “spawn”, em vez de apenas no Windows.

- gh-134582 [https://github.com/python/cpython/issues/134582]: Corrige erros de ida e volta de tokenize.untokenize() relacionados a chaves de escape de t-strings

- gh-134580 [https://github.com/python/cpython/issues/134580]: Improved the styling of HTML diff pages generated by the `difflib.HtmlDiff` class, and migrated the output to the HTML5 standard.

- gh-134565 [https://github.com/python/cpython/issues/134565]: `unittest.doModuleCleanups()` no longer swallows all but first exception raised in the cleanup code, but raises a `ExceptionGroup` if multiple errors occurred.

- gh-134546 [https://github.com/python/cpython/issues/134546]: Garante que o script de depuração remota `pdb` seja legível pelo processo Python remoto.

- gh-134451 [https://github.com/python/cpython/issues/134451]: Convertido `asyncio.tools.CycleFoundException` de dataclass para um tipo de exceção regular.

- gh-114177 [https://github.com/python/cpython/issues/114177]: Corrige **`asyncio`** para não fechar encadeamentos de subprocesso, o que de outra forma geraria erro quando o laço de eventos já estivesse fechado.

- gh-90871 [https://github.com/python/cpython/issues/90871]: Corrigido um erro de um erro referente ao parâmetro backlog em **`create_unix_server()`**. Contribuição de Christian Harries.

- gh-134323 [https://github.com/python/cpython/issues/134323]: Corrige o método **`threading.RLock.locked()`**.

- gh-86802 [https://github.com/python/cpython/issues/86802]: Corrigido vazamento de memória assíncrona em tarefas de escudo canceladas. Para tarefas protegidas em que o escudo foi cancelado, registre possíveis exceções por meio do tratador de exceções. Contribuição de Christian Harries.

- gh-71189 [https://github.com/python/cpython/issues/71189]: Add support of the all-but-last mode in **`os.path.realpath()`**.

- gh-72902 [https://github.com/python/cpython/issues/72902]: Improve speed (x1.1-1.8) of the **`Fraction`** constructor for typical inputs (**`float`**'s, **`Decimal`**'s or strings).

- gh-134209 [https://github.com/python/cpython/issues/134209]: **`curses`**: Os métodos **`curses.window.instr()`** e **`curses.window.getstr()`** agora alocam seu buffer interno no heap em vez da heap; além disso, o tamanho máximo do buffer foi aumentado de 1023 para 2047.

- gh-88994 [https://github.com/python/cpython/issues/88994]: Change **`datetime.datetime.now()`** to half-even rounding for consistency with **`datetime.datetime.fromtimestamp()`**. Patch by John Keith Hohm.

- gh-80184 [https://github.com/python/cpython/issues/80184]: The default queue size is now `socket.SOMAXCONN` for **`socketserver.TCPServer`**.

- gh-132983 [https://github.com/python/cpython/issues/132983]: Add **`compression.zstd`** version information to `test.pythoninfo`.

- gh-134235 [https://github.com/python/cpython/issues/134235]: Atualizada a complementação de tabulação no REPL para incluir módulos embutidos. Contribuição de Tom Wang e Hunter Young.

- gh-134152 [https://github.com/python/cpython/issues/134152]: Corrigida **`UnboundLocalError`** que poderia ocorrer durante a análise do cabeçalho **`email`** se um delimitador final esperado estivesse faltando em alguns contextos.

- gh-134152 [https://github.com/python/cpython/issues/134152]: **`email`**: Corrige análise de ID de mensagem de e-mail com domínio inválido.

- gh-134168 [https://github.com/python/cpython/issues/134168]: **`http.server`**: Corrige a ligação de endereço IPv6 e tratamento de **`--directory`** ao usar HTTPS.

- gh-62184 [https://github.com/python/cpython/issues/62184]: Remove a importação da implementação C de **`io.FileIO`** da implementação Python que tem sua própria implementação

- gh-134087 [https://github.com/python/cpython/issues/134087]: Remove support for arbitrary positional or keyword arguments in the C implementation of **`threading.RLock`** objects. This was deprecated since Python 3.14. Patch by Bénédikt Tran.

- gh-134173 [https://github.com/python/cpython/issues/134173]: Speed up **`asyncio`** performance of transferring state from thread pool **`concurrent.futures.Future`** by up to 4.4x. Patch by J. Nick Koston.

- gh-133982 [https://github.com/python/cpython/issues/133982]: Emite **`RuntimeWarning`** na implementação Python de **`io`** quando o objeto arquivo ou similar não for fechado explicitamente na presença de múltiplas camadas de E/S.

- gh-133890 [https://github.com/python/cpython/issues/133890]: O módulo `tarfile` agora manipula `UnicodeEncodeError` da mesma forma que `OSError` quando não é possível extrair um membro.

- gh-134097 [https://github.com/python/cpython/issues/134097]: Corrige interação da REPL e a nova opção de linha de comando `-X showrefcount`.

- gh-133889 [https://github.com/python/cpython/issues/133889]: A página de listagem de diretório gerada em `http.server.SimpleHTTPRequestHandler` agora mostra apenas o componente do caminho decodificado da URL solicitada, e não a consulta e o fragmento.

- gh-134098 [https://github.com/python/cpython/issues/134098]: Corrige caminhos de manipulação que terminam com uma barra codificada em porcentagem (`%2f` ou `%2F`) em `http.server.SimpleHTTPRequestHandler`.

- gh-132124 [https://github.com/python/cpython/issues/132124]: Em sistemas compatíveis com POSIX, **`multiprocessing.util.get_temp_dir()`** agora ignora **`TMPDIR`** (e variáveis de ambiente semelhantes) se o comprimento do caminho dos arquivos de soquete `AF_UNIX` exceder o comprimento máximo específico da plataforma ao usar o método de inicialização forkserver. Patch por Bénédikt Tran.

- gh-134062 [https://github.com/python/cpython/issues/134062]: `ipaddress`: corrige colisões em `__hash__()` para objetos `IPv4Network` e `IPv6Network`.

- gh-134004 [https://github.com/python/cpython/issues/134004]: `shelve` as well as underlying **`dbm.dumb`** and **`dbm.sqlite`** now have **`reorganize()`** methods to recover unused free space previously occupied by deleted entries.

- gh-133970 [https://github.com/python/cpython/issues/133970]: Torna **`string.templatelib.Template`** e **`string.templatelib.Interpolation`** genéricos.

- gh-71253 [https://github.com/python/cpython/issues/71253]: Levanta **`ValueError`** em **`open()`** se *opener* retornar um descritor de arquivo negativo na implementação Python de **`io`** para corresponder à implementação C.

- gh-133960 [https://github.com/python/cpython/issues/133960]: Simplifica e aprimora **`typing.evaluate_forward_ref()`**. Agora, ele não levanta mais erros em determinados tipos inválidos. Em diversas situações, agora é possível avaliar referências futuras que antes não eram suportadas.

- gh-133925 [https://github.com/python/cpython/issues/133925]: Torna a classe privada `typing._UnionGenericAlias` hasheável.

- gh-133604 [https://github.com/python/cpython/issues/133604]: Remove **`platform.java_ver()`** which was deprecated since Python 3.13.

- gh-133875 [https://github.com/python/cpython/issues/133875]: Removed deprecated **`pathlib.PurePath.is_reserved()`**. Use **`os.path.isreserved()`** to detect reserved paths on Windows.

- gh-133873 [https://github.com/python/cpython/issues/133873]: Remove the deprecated `getmark()`, `setmark()` and `getmarkers()` methods of the **`Wave_read`** and **`Wave_write`** classes, which were deprecated since Python 3.13. Patch by Bénédikt Tran.

- gh-133866 [https://github.com/python/cpython/issues/133866]: Remove the undocumented function **`ctypes.SetPointerType()`**, which has been deprecated since Python 3.13. Patch by Bénédikt Tran.

- gh-133823 [https://github.com/python/cpython/issues/133823]: Remove support for `TD = TypedDict("TD")` and `TD = TypedDict("TD", None)` calls for constructing **`typing.TypedDict`** objects with zero field. Patch by Bénédikt Tran.

- gh-125996 [https://github.com/python/cpython/issues/125996]: Correção da segurança de thread de **`collections.OrderedDict`**. Patch por Kumar Aditya.

- gh-133817 [https://github.com/python/cpython/issues/133817]: Remove support for creating **`NamedTuple`** classes via the undocumented keyword argument syntax. Patch by Bénédikt Tran.

- gh-133653 [https://github.com/python/cpython/issues/133653]: Corrige **`argparse.ArgumentParser`** com o argumento *formatter_class*. Corrige TypeError quando *formatter_class* for uma subclasse personalizada de **`HelpFormatter`**. Corrige TypeError quando *formatter_class* não for uma subclasse de **`HelpFormatter`** e *prefix_char* não padrão for usado. Corrige suporte à colorização quando *formatter_class* não for uma subclasse de **`HelpFormatter`**.

- gh-133810 [https://github.com/python/cpython/issues/133810]: Remove **`http.server.CGIHTTPRequestHandler`** and `--cgi` flag from the **python -m http.server** command-line interface. They were deprecated in Python 3.13. Patch by Bénédikt Tran.

- gh-132641 [https://github.com/python/cpython/issues/132641]: Corrigida uma corrida em **`functools.lru_cache()`** em threads livres.

- gh-133783 [https://github.com/python/cpython/issues/133783]: Corrige bug ao aplicar **`copy.replace()`** a objetos **`ast`**. Atributos cujo padrão é `None` eram tratados incorretamente como obrigatórios para nós de AST criados manualmente.

- gh-133684 [https://github.com/python/cpython/issues/133684]: Corrige bug em que **`annotationlib.get_annotations()`** retornaria o resultado errado para determinadas classes que fazem parte de uma hierarquia de classes onde `from __future__ import annotations` é usado.

- gh-77057 [https://github.com/python/cpython/issues/77057]: Corrige o tratamento de declarações de marcação inválidas em **`html.parser.HTMLParser`**.

- gh-130328 [https://github.com/python/cpython/issues/130328]: Aceleração da colagem em `PyREPL` no Windows em um console antigo. Patch por Chris Eibl.

- gh-133701 [https://github.com/python/cpython/issues/133701]: Corrige bug onde classes `typing.TypedDict` definidas em `from __future__ import annotations` e herdadas de outro `TypedDict` tinham um atributo `__annotations__` incorreto.

- gh-133656 [https://github.com/python/cpython/issues/133656]: Remove deprecated **`zipimport.zipimporter.load_module()`**. Use `zipimport.zipimporter.exec_module()` instead.

- gh-133722 [https://github.com/python/cpython/issues/133722]: Added a *color* option to `difflib.unified_diff()` that colors output similar to **git diff**.

- gh-133489 [https://github.com/python/cpython/issues/133489]: `random.getrandbits()` agora pode gerar mais de 2^{31} bits. `random.randbytes()` agora pode gerar mais de 256 MiB.

- gh-133595 [https://github.com/python/cpython/issues/133595]: Clean up `sqlite3.Connection` APIs. All parameters of `sqlite3.connect()` except *database* are now keyword-only. The first three parameters of methods `create_function()` and `create_aggregate()` are now positional-only. The first parameter of methods `set_authorizer()`, `set_progress_handler()` and `set_trace_callback()` is now positional-only.

- gh-133581 [https://github.com/python/cpython/issues/133581]: Melhoria na desanálise de t-strings em `ast.unparse()` e `from __future__ import annotations`. T-strings vazias agora realizam a ida e volta corretamente e a formatação em interpolações é preservada. Patch por Jelle Zijlstra.

- gh-133577 [https://github.com/python/cpython/issues/133577]: Add parameter `formatter` to `logging.basicConfig()`.

- gh-92897 [https://github.com/python/cpython/issues/92897]: Removed the `check_home` parameter from `sysconfig.is_python_build()`, deprecated since Python 3.12.

- gh-133551 [https://github.com/python/cpython/issues/133551]: Suporte a t-strings (**PEP 750** [https://peps.python.org/pep-0750/]) em `annotationlib`. Patch por Jelle Zijlstra.

- gh-133517 [https://github.com/python/cpython/issues/133517]: Remove `os.listdrives()`, `os.listvolumes()` and `os.listmounts()` in non Windows desktop builds since the underlying functionality is missing.

- gh-133439 [https://github.com/python/cpython/issues/133439]: Corrige comandos ponto com espaços à direita são confundidos com instruções SQL multilinha na interface de linha de comando sqlite3.

- gh-133447 [https://github.com/python/cpython/issues/133447]: Add basic color to `sqlite3` CLI interface.

- gh-133253 [https://github.com/python/cpython/issues/133253]: Fix thread-safety issues in `linecache`.

- gh-133390 [https://github.com/python/cpython/issues/133390]: Support keyword completion in the `sqlite3` command-line interface and add `sqlite3.SQLITE_KEYWORDS` constant.

- gh-132493 [https://github.com/python/cpython/issues/132493]: Evita acessar `__annotations__` desnecessariamente em `inspect.signature()`.

- gh-133017 [https://github.com/python/cpython/issues/133017]: Improve the error message of `multiprocessing.sharedctypes.Array()`, `multiprocessing.sharedctypes.RawArray()`, `multiprocessing.sharedctypes.Value()` and `multiprocessing.sharedctypes.RawValue()` when an invalid typecode is passed. Patch by Tomas Roun

- gh-132813 [https://github.com/python/cpython/issues/132813]: Melhora as mensagens de erro para tipos e valores incorretos de atributos `csv.Dialect`.

- gh-132969 [https://github.com/python/cpython/issues/132969]: Impede que a thread executora `ProcessPoolExecutor`, que permanece em execução

quando `shutdown(wait=False)`, tente ajustar os processos de trabalho do pool após o estado do objeto já ter sido redefinido durante o desligamento. Uma combinação de condições, incluindo um processo de trabalho encerrado de forma anormal, resultou em uma exceção e um possível travamento quando a thread executora ainda em execução tentou substituir os processos de trabalho inativos no pool.

- gh-132876 [https://github.com/python/cpython/issues/132876]: `ldexp()` no Windows não arredonda resultados abaixo do normal antes do Windows 11, mas deveria. O invólucro de `math.ldexp()` do Python agora os arredonda, então os resultados podem mudar um pouco, em casos raros de resultados muito pequenos, em versões do Windows anteriores à 11.

- gh-133009 [https://github.com/python/cpython/issues/133009]: `xml.etree.ElementTree`: Corrige uma falha em `Element.__deepcopy__` quando o elemento sofre mutação simultânea. Patch por Bénédikt Tran.

- gh-132908 [https://github.com/python/cpython/issues/132908]: Add `math.isnormal()` and `math.issubnormal()` functions. Patch by Sergey B Kirpichev.

- gh-95380 [https://github.com/python/cpython/issues/95380]: `fcntl.fcntl()` and `fcntl.ioctl()`: Remove the 1024 bytes limit on the size of not mutated bytes-like argument.

- gh-122781 [https://github.com/python/cpython/issues/122781]: Fix `%z` directive in `datetime.datetime.strptime()` to allow for no provided offset as was documented.

- gh-123471 [https://github.com/python/cpython/issues/123471]: Make concurrent iterations over `itertools.combinations` and `itertools.product` safe under free-threading.

- gh-127081 [https://github.com/python/cpython/issues/127081]: Corrige problemas de segurança de thread da libc com `dbm` executando operações com estado em seções críticas.

- gh-127081 [https://github.com/python/cpython/issues/127081]: Corrige problemas de segurança do thread libc com **os** substituindo `getlogin` pela versão reentrante `getlogin_r`.

- gh-127081 [https://github.com/python/cpython/issues/127081]: Fix libc thread safety issues with **pwd** by locking access to `getpwall`.

- gh-132551 [https://github.com/python/cpython/issues/132551]: Torna **io.BytesIO** segura na construção com threads livres.

- gh-107583 [https://github.com/python/cpython/issues/107583]: Fix **Flag** inversion when flag set has missing values (**IntFlag** still flips all bits); fix negative assigned values during flag creation (both **Flag** and **IntFlag** ignore missing values).

- gh-87790 [https://github.com/python/cpython/issues/87790]: Adicionado suporte para sublinhado e vírgula como separadores de milhares na parte fracionária para a formatação de **Fraction**. Patch por Sergey B Kirpichev.

- gh-87790 [https://github.com/python/cpython/issues/87790]: Suporte para sublinhado e vírgula como separadores de milhares na parte fracionária para a formatação de **Decimal**. Patch por Sergey B Kirpichev.

- gh-131884 [https://github.com/python/cpython/issues/131884]: Corrige problemas de formatação em **json.dump()** quando *indent* e *skipkeys* são usados.

- gh-131788 [https://github.com/python/cpython/issues/131788]: Torna `ResourceTracker.send` de **multiprocessing** reentrante seguro

- gh-91349 [https://github.com/python/cpython/issues/91349]: Adjust default `compressionlevel=` to 6 (down from 9) in **gzip** and **tarfile**. It is the default level used by most compression tools and a better tradeoff between speed and performance.

- gh-131146 [https://github.com/python/cpython/issues/131146]: Fix **calendar.TextCalendar**, **calendar.HTMLCalendar**, and the **calendar** CLI to display month names in the nominative case by adding

`calendar.standalone_month_name` and `calendar.standalone_month_abbr`, which provide month names and abbreviations in the grammatical form used when a month name stands by itself, if the locale supports it.

- gh-123471 [https://github.com/python/cpython/issues/123471]: Make concurrent iterations over `itertools.cycle` safe under free-threading.

- gh-130664 [https://github.com/python/cpython/issues/130664]: Lida com maiúsculas e minúsculas para a formatação de `Fraction`: tratar o preenchimento com zeros (precedendo o campo de largura por um caractere zero (`'0'`)) como um equivalente a um caractere de preenchimento de `'0'` com um tipo de alinhamento de `'='`, assim como no caso de `float`.

- gh-130999 [https://github.com/python/cpython/issues/130999]: Evita sair do novo REPL e oferece sugestões mesmo se houver candidatos que não sejam strings quando ocorrerem erros.

- gh-88473 [https://github.com/python/cpython/issues/88473]: Implement a fast path for `datetime.date` objects in `datetime.date.today()` which results in a 5x performance gain while proper subclasses retain their previous performance.

- gh-126883 [https://github.com/python/cpython/issues/126883]: Add check that timezone fields are in range for `datetime.datetime.fromisoformat()` and `datetime.time.fromisoformat()`. Patch by Semyon Moroz.

- gh-125028 [https://github.com/python/cpython/issues/125028]: `functools.Placeholder` não pode ser passado para `functools.partial()` como um argumento nomeado.

- gh-125843 [https://github.com/python/cpython/issues/125843]: If possible, indicate which `curses` C function or macro is responsible for raising a `curses.error` exception. Patch by Bénédikt Tran.

- gh-119109 [https://github.com/python/cpython/issues/119109]: **functools.partial()** calls are now faster when keyword arguments are used.

- gh-124033 [https://github.com/python/cpython/issues/124033]: `SimplePath` is now presented in `importlib.metadata.__all__`.

- gh-91216 [https://github.com/python/cpython/issues/91216]: `importlib.metadata` now raises a `KeyError` instead of returning `None` when a key is missing from the metadata.

- gh-120492 [https://github.com/python/cpython/issues/120492]: `importlib.metadata` now prioritizes valid dists to invalid dists when retrieving by name.

- gh-99631 [https://github.com/python/cpython/issues/99631]: The **shelve** module now accepts custom serialization and deserialization functions.

- gh-119186 [https://github.com/python/cpython/issues/119186]: Slightly speed up **os.walk()** by calling **os.path.join()** less often.

- gh-120170 [https://github.com/python/cpython/issues/120170]: Fix an issue in the **_pickle** extension module in which importing **multiprocessing** could change how pickle identifies which module an object belongs to, potentially breaking the unpickling of those objects.

- gh-118981 [https://github.com/python/cpython/issues/118981]: Corrige possível travamento em `multiprocessing.popen_spawn_posix` que pode ocorrer quando o processo filho morre prematuramente, fechando os descritores de arquivo filhos imediatamente.

- gh-105497 [https://github.com/python/cpython/issues/105497]: Corrige a inversão da máscara de sinalizador quando existem flags sem nome.

- gh-99813 [https://github.com/python/cpython/issues/99813]: **ssl** now uses `SSL_sendfile` internally when it is possible (see **OP_ENABLE_KTLS**). The function sends a file more efficiently because it performs TLS

encryption in the kernel to avoid additional context switches. Patch by Illia Volochii.

- gh-62824 [https://github.com/python/cpython/issues/62824]: Correção de apelidos para codificação `iso8859_8`. Patch por Dave Goncalves.

- gh-86155 [https://github.com/python/cpython/issues/86155]: **`html.parser.HTMLParser.close()`** não perde mais dados quando a tag `<script>` não está fechada. Patch por Waylan Limberg.

- gh-78319 [https://github.com/python/cpython/issues/78319]: O suporte UTF8 para o comando IMAP APPEND foi tornado compatível com RFC.

- gh-93334 [https://github.com/python/cpython/issues/93334]: Reraise **`KeyError`** as **`ModuleNotFoundError`** when **`importlib.machinery.PathFinder.find_spec()`** is called on a submodule without importing the parent (and without a `path` argument).

- gh-69426 [https://github.com/python/cpython/issues/69426]: Corrige **`html.parser.HTMLParser`** para não desfazer o escape de entidades de caracteres em valores de atributos se eles forem seguidos por um alfanumérico ASCII ou um sinal de igual.

- bpo-38735 [https://bugs.python.org/issue?@action=redirect&bpo=38735]: Corrige falha ao importar um módulo do diretório raiz em plataformas do tipo Unix com sys.pycache_prefix definido.

- bpo-45959 [https://bugs.python.org/issue?@action=redirect&bpo=45959]: **`pprint`** can now pretty-print dict views.

- gh-84683 [https://github.com/python/cpython/issues/84683]: **`zoneinfo`**: Check in `<prefix>/share/zoneinfo` for data files on Windows

- bpo-43429 [https://bugs.python.org/issue?@action=redirect&bpo=43429]: The **`size()`** method of the **`mmap.mmap`** class now returns the size of an anonymous mapping on both Unix and Windows. Previously, the size would be

returned on Windows and an `OSError` would be raised on Unix. `ValueError` is now raised instead of `OSError` when `trackfd=False`.

- bpo-41839 [https://bugs.python.org/issue?@action=redirect&bpo=41839]: Permite valores de prioridade negativos das funções `os.sched_get_priority_min()` e `os.sched_get_priority_max()`.

- bpo-28494 [https://bugs.python.org/issue?@action=redirect&bpo=28494]: Melhora a taxa de falsos positivos na validação de arquivo Zip em `zipfile.is_zipfile()`.

IDLE

- gh-96491 [https://github.com/python/cpython/issues/96491]: Desduplica o número da versão na barra de título do shell do IDLE após salvar em um arquivo.
- gh-139742 [https://github.com/python/cpython/issues/139742]: Colore prefixos de t-string para strings template no IDLE, como feito para prefixos de f-string.

Documentação

- gh-136155 [https://github.com/python/cpython/issues/136155]: Agora estamos verificando erros fatais em construções de EPUB no CI.
- gh-135171 [https://github.com/python/cpython/issues/135171]: Documenta que o iterador para a cláusula `for` mais à esquerda na expressão geradora é criado imediatamente.
- bpo-45210 [https://bugs.python.org/issue?@action=redirect&bpo=45210]: Documenta que o indicador de erro pode ser definido em tp_dealloc e como evitar sobrecarregá-lo.

Núcleo e embutidos

- gh-140000 [https://github.com/python/cpython/issues/140000]: Corrige possível vazamento de memória quando há um ciclo de referência entre uma instância de `typing.TypeAliasType`, `typing.TypeVar`,

`typing.ParamSpec` ou `typing.TypeVarTuple` e seu atributo `__name__`. Patch por Mikhail Efimov.

- gh-140009 [https://github.com/python/cpython/issues/140009]: Improve performance of list extension by dictionary items.
- gh-139988 [https://github.com/python/cpython/issues/139988]: Corrige um vazamento de memória causado pela falha na criação do tipo `Union`. Patch por Bénédikt Tran.
- gh-139748 [https://github.com/python/cpython/issues/139748]: Corrige vazamentos de referência em ramificações de erro de funções que aceitam strings de caminho ou bytes, como `compile()` e `os.system()`. Patch por Bénédikt Tran.
- gh-139516 [https://github.com/python/cpython/issues/139516]: Corrigir erro no formato de início de especificação de dois pontos lambda na f-string no tokenizador.
- gh-63161 [https://github.com/python/cpython/issues/63161]: Suporte a shebang e comentários não UTF-8 em arquivos-fonte Python se a codificação não UTF-8 for especificada. Detecte erros de decodificação em comentários para a codificação padrão (UTF-8). Exibe a linha e a posição do erro de decodificação para a codificação padrão em um traceback. Exiba a linha que contém o cookie de codificação quando ele entrar em conflito com o BOM em um traceback.
- gh-139116 [https://github.com/python/cpython/issues/139116]: Prevent a deadlock when multiple threads start, stop and use `tracemalloc` simultaneously.
- gh-139275 [https://github.com/python/cpython/issues/139275]: Fix compilation problems in `_remote_debugging_module.c` when the system doesn't have `process_vm_readv`. Patch by Pablo Galindo
- gh-133059 [https://github.com/python/cpython/issues/133059]: Increased the number of cached small positive integers from 256 to 1024.
- gh-74857 [https://github.com/python/cpython/issues/74857]: **PEP 538** [https://peps.python.org/pep-0538/]: Coerce the POSIX locale to a UTF-8 based locale. Patch by Victor Stinner.

Biblioteca

- gh-116738 [https://github.com/python/cpython/issues/116738]: Torna o `mmap` seguro para thread na construção com threads livres.

Núcleo e embutidos

- gh-138558 [https://github.com/python/cpython/issues/138558]: Corrige o tratamento de anotações incomuns de t-string na annotationlib. Patch por Dave Peck.
- gh-134466 [https://github.com/python/cpython/issues/134466]: Não executa o PyREPL em um ambiente degradado onde a definição de atributos termios não é permitida.
- gh-138794 [https://github.com/python/cpython/issues/138794]: When a new tracing function is registered with `PyRefTracer_SetTracer()`, replacing the current a call to the trace function will be made with the object set to **NULL** and **event** set to `PyRefTracer_TRACKER_REMOVED`. This will happen just before the new function is registered. Patch by Pablo Galindo
- gh-71810 [https://github.com/python/cpython/issues/71810]: Levanta `OverflowError` para `(-1).to_bytes()` para conversões com sinal quando a contagem de bytes for zero. Patch por Sergey B Kirpichev.
- gh-138716 [https://github.com/python/cpython/issues/138716]: Improve `SyntaxError` message for `assert` in cases like `assert a := b`.
- gh-105487 [https://github.com/python/cpython/issues/105487]: Remove `__copy__()`, `__deepcopy__()` e `__bases__` inexistentes das entradas `__dir__()` de `types.GenericAlias`.
- gh-138192 [https://github.com/python/cpython/issues/138192]: Corrige a inicialização de `contextvars` para que todos os subinterpretadores recebam o valor `MISSING`.
- gh-138479 [https://github.com/python/cpython/issues/138479]: Corrige uma falha quando `__typing_subst__` de um objeto genérico retorna um objeto que não é uma `tuple`.
- gh-138431 [https://github.com/python/cpython/issues/138431]: Fix a bug in the JIT optimizer when round-tripping strings and tuples.
- gh-138378 [https://github.com/python/cpython/issues/138378]: Move the globals-to-const JIT optimizer pass into to the main JIT optimizer pass

Biblioteca

- gh-138401 [https://github.com/python/cpython/issues/138401]: Add missing validation of argument `count` in `os.sendfile()` to be non-negative.

Núcleo e embutidos

- gh-138372 [https://github.com/python/cpython/issues/138372]: Correção de `SyntaxWarning` emitido para subscrito expressão errôneo envolvendo literal de string template. Patch por Brian Schubert.
- gh-138302 [https://github.com/python/cpython/issues/138302]: `BINARY_OP` now specializes to `BINARY_OP_ADD_INT`, `BINARY_OP_SUBTRACT_INT` or `BINARY_OP_MULTIPLY_INT` if operands are compact ints.
- gh-138318 [https://github.com/python/cpython/issues/138318]: O REPL padrão agora evita destacar nomes embutidos (por exemplo, `set` ou `format()`) quando eles são usados como nomes de atributos (por exemplo, em `value.set` ou `text.format`).
- gh-138349 [https://github.com/python/cpython/issues/138349]: Corrige falha em certos casos em que um módulo contém uma anotação de nível de módulo e uma compreensão.
- gh-69605 [https://github.com/python/cpython/issues/69605]: Corrige alguns submódulos da biblioteca padrão ausentes no preenchimento automático de importações do REPL.
- gh-61206 [https://github.com/python/cpython/issues/61206]: `zipimport` now supports zstandard compressed zip file entries.

Biblioteca

- gh-116738 [https://github.com/python/cpython/issues/116738]: Make `cProfile` thread-safe on the free threaded build.

Núcleo e embutidos

- gh-137838 [https://github.com/python/cpython/issues/137838]: Fix JIT trace buffer overrun by increasing possible exit stubs. Patch by Donghee Na.
- gh-71679 [https://github.com/python/cpython/issues/71679]: Use the same quoting algorithm for the repr of bytearrays as for bytes objects and strings – use double quotes for quoting if the bytearray contains single quotes and does not contain double quotes.
- gh-137384 [https://github.com/python/cpython/issues/137384]: Correção de uma falha ao usar o `warnings` módulo em um finalizador no desligamento. Patch por Kumar Aditya .

Biblioteca

- gh-138004 [https://github.com/python/cpython/issues/138004]: Nas plataformas Solaris/Illumos, os nomes de threads agora são codificados como ASCII para evitar erros em sistemas (por exemplo, OpenIndiana) que não oferecem suporte a nomes não ASCII.
- gh-137976 [https://github.com/python/cpython/issues/137976]: Removed `localtime` from the list of reported system timezones.

Núcleo e embutidos

- gh-137992 [https://github.com/python/cpython/issues/137992]: Ensure that **`PyRefTracer_SetTracer()`** sync with all existing threads when called to avoid races in the free threaded build. Patch by Pablo Galindo
- gh-137967 [https://github.com/python/cpython/issues/137967]: Show error suggestions on nested attribute access. Patch by Pablo Galindo
- gh-137959 [https://github.com/python/cpython/issues/137959]: Replace the shim code added to every piece of jitted code with a single trampoline function.
- gh-137883 [https://github.com/python/cpython/issues/137883]: Corrige recursão descontrolada ao chamar uma função com argumentos nomeados.
- gh-137079 [https://github.com/python/cpython/issues/137079]: Correção do reconhecimento de erros de digitação em palavra reservada ao analisar arquivos. Patch por Pablo Galindo.
- gh-137728 [https://github.com/python/cpython/issues/137728]: Corrige o tratamento de muitas variáveis locais pelo JIT. Isso anteriormente causava uma falha de segmentação.
- gh-137716 [https://github.com/python/cpython/issues/137716]: Fix double period in **`AttributeError`** message for invalid mock assertions
- gh-137433 [https://github.com/python/cpython/issues/137433]: Corrige um possível impasse na construção com threads livres quando threads do daemon habilitam ou desabilitam a criação de perfil ou rastreamento enquanto a thread principal está desligando o interpretador.
- gh-137576 [https://github.com/python/cpython/issues/137576]: Correção para código-fonte incorreto exibido em tracebacks do REPL básico quando **`PYTHONSTARTUP`** é fornecido. Patch por Adam Hartz.

- gh-37817 [https://github.com/python/cpython/issues/37817]: Allow assignment to `__bases__` of direct subclasses of builtin classes.
- gh-132732 [https://github.com/python/cpython/issues/132732]: Optimize `_COMPARE_OP`, `_CONTAINS_OP`, `_UNARY_NEGATIVE`, `_UNARY_NOT`, and `_UNARY_INVERT` in JIT builds with constant-loading uops (`_POP_TWO_LOAD_CONST_INLINE_BORROW` and `_POP_TOP_LOAD_CONST_INLINE_BORROW`), and then remove both to reduce instruction count.
- gh-137400 [https://github.com/python/cpython/issues/137400]: Corrige uma falha na construção com threads livres ao desabilitar a criação de perfil ou o rastreamento em todos os threads com `PyEval_SetProfileAllThreads()` ou `PyEval_SetTraceAllThreads()` ou seus equivalentes em Python `threading.settrace_all_threads()` e `threading.setprofile_all_threads()`.
- gh-133143 [https://github.com/python/cpython/issues/133143]: Add `sys.abi_info` object to make ABI information more easily accessible.
- gh-137400 [https://github.com/python/cpython/issues/137400]: Corrige uma falha na construção com threads livres ao desabilitar a criação de perfil ou o rastreamento em todos os threads com `PyEval_SetProfileAllThreads()` ou `PyEval_SetTraceAllThreads()` ou seus equivalentes em Python `threading.settrace_all_threads()` e `threading.setprofile_all_threads()`.
- gh-120037 [https://github.com/python/cpython/issues/120037]: Disable user site packages directory when a `._pth` file is used, even if it contains `import site`.
- gh-58124 [https://github.com/python/cpython/issues/58124]: Corrige erros de codificação Python em Unicode do codec da página de código: usar “cp65000” e “cp65001” em vez de “CP_UTF7” e “CP_UTF8”, que não são nomes de código Python válidos. Patch por Victor Stinner.
- gh-136966 [https://github.com/python/cpython/issues/136966]: The `object.__dict__` and `__weakref__` descriptors now use a single descriptor instance per interpreter, shared across all types that need them. This speeds up class creation, and helps avoid reference cycles.
- gh-137314 [https://github.com/python/cpython/issues/137314]: Corrigida uma regressão em que f-strings brutas interpretavam incorretamente sequências de escape em especificações de formato. As f-strings brutas agora preservam corretamente as contrabarras literais nas

especificações de formato, correspondendo ao comportamento do Python 3.11. Por exemplo, `rf"{obj:\xFF}"` agora produz corretamente `'\\xFF'` em vez de `'ÿ'`. Patch por Pablo Galindo.

- gh-137308 [https://github.com/python/cpython/issues/137308]: Uma docstring independente no corpo de um nó é otimizada como uma instrução **`pass`** para garantir que o corpo do nó nunca fique vazio. Caso contrário, havia um **`ValueError`** em **`compile()`**.
- gh-137288 [https://github.com/python/cpython/issues/137288]: Corrige bug em que algumas instruções de bytecode de uma expressão booleana não estão associadas ao manipulador de exceção correto.
- gh-137291 [https://github.com/python/cpython/issues/137291]: The perf profiler can now be used if a previous frame evaluation API has been provided.
- gh-134291 [https://github.com/python/cpython/issues/134291]: Remove alguns usos mais recentes da API do macOS do compilador JIT para restaurar a compatibilidade com destinos de implantação mais antigos do OSX 10.15.
- gh-88886 [https://github.com/python/cpython/issues/88886]: The codecs lookup function now again performs only minimal normalization of the encoding name before passing it to the search functions: all ASCII letters are converted to lower case, spaces are replaced with hyphens. This restores the pre-Python 3.9 behavior.
- gh-131338 [https://github.com/python/cpython/issues/131338]: Desabilita verificações de limite de pilha computada em plataformas Linux não-glibc para corrigir travamentos em recursão profunda.
- gh-136870 [https://github.com/python/cpython/issues/136870]: Corrige disputas de dados ao desinstrumentar o bytecode de objetos código executados simultaneamente em threads.
- gh-132732 [https://github.com/python/cpython/issues/132732]: Optimize constant comparison for `_COMPARE_OP_INT`, `_COMPARE_OP_FLOAT` and `_COMPARE_OP_STR` in JIT builds
- gh-127598 [https://github.com/python/cpython/issues/127598]: Improve **`ModuleNotFoundError`** by adding flavour text to the exception when the **`-S`** option is passed. Patch by Andrea Mattei.
- gh-136801 [https://github.com/python/cpython/issues/136801]: Corrige o destaque de sintaxe do PyREPL em casos de correspondência após casos multilinhas. Contribuição de Olga Matoula.

- gh-74185 [https://github.com/python/cpython/issues/74185]: The `__repr__()` of `ImportError` and `ModuleNotFoundError` now shows “name” and “path” as `name=<name>` and `path=<path>` if they were given as keyword arguments at construction time. Patch by Serhiy Storchaka, Oleg Iarygin, and Yoav Nir

Biblioteca

- gh-116738 [https://github.com/python/cpython/issues/116738]: Make functions in `syslog` thread-safe on the free threaded build.
- gh-116738 [https://github.com/python/cpython/issues/116738]: Make functions in `pwd` thread-safe on the free threaded build.

Núcleo e embutidos

- gh-136616 [https://github.com/python/cpython/issues/136616]: Improve `SyntaxError` error messages for invalid `assert` usages.
- gh-136599 [https://github.com/python/cpython/issues/136599]: Improve performance of `int` hash calculations.

Biblioteca

- gh-136421 [https://github.com/python/cpython/issues/136421]: Corrige falha ao inicializar `datetime` simultaneamente.

Núcleo e embutidos

- gh-136541 [https://github.com/python/cpython/issues/136541]: Corrigimos alguns problemas com os trampolins do Perf em x86-64 e aarch64. Os trampolins não estavam sendo gerados corretamente em alguns casos, o que poderia fazer com que a integração do Perf não funcionasse corretamente. Patch por Pablo Galindo.

Biblioteca

- gh-107545 [https://github.com/python/cpython/issues/107545]: Improve the error messages that may be raised by `setsockopt()`.

Núcleo e embutidos

- gh-136517 [https://github.com/python/cpython/issues/136517]: Corrigido um erro de digitação que impedia a exibição de objetos não colecionáveis quando o modo `gc.DEBUG_UNCOLLECTABLE` estava definido.

- gh-136525 [https://github.com/python/cpython/issues/136525]: Corrige problema em que o bytecode por thread não era instrumentado para threads recém-criadas.

- gh-132657 [https://github.com/python/cpython/issues/132657]: Melhora o desempenho de `frozenset` removendo travas na construção com threads livres.

- gh-136459 [https://github.com/python/cpython/issues/136459]: Add support for perf trampoline on macOS, to allow profilers wit JIT map support to read Python calls. While profiling, `PYTHONPERFSUPPORT=1` can be appended to enable the trampoline.

- gh-132661 [https://github.com/python/cpython/issues/132661]: `Interpolation.expression` agora tem um padrão, a string vazia.

- gh-132661 [https://github.com/python/cpython/issues/132661]: Reflete a recente mudança **PEP 750** [https://peps.python.org/pep-0750/].

  Não permite a concatenação de `string.templatelib.Template` e `str`. Também não permite a concatenação implícita de literais de t-string com strings e literais de f-strings.

- gh-91636 [https://github.com/python/cpython/issues/91636]: Ao realizar a coleta de lixo, limpa as referências fracas (weakrefs) a objetos inacessíveis criados durante a execução dos finalizadores. Se essas referências fracas não forem limpas, elas podem revelar objetos inacessíveis.

- gh-136355 [https://github.com/python/cpython/issues/136355]: Deprecate **-b** and **-bb** command line options and schedule them to become no-op in Python 3.17.

- gh-109700 [https://github.com/python/cpython/issues/109700]: Corrige tratamento de erro de memória em **PyDict_SetDefault()**.

- gh-135552 [https://github.com/python/cpython/issues/135552]: Fix a bug caused by the garbage collector clearing weakrefs too early. The weakrefs in the `tp_subclasses` dictionary are needed in order to correctly invalidate type caches (for example, by calling `PyType_Modified()`). Clearing weakrefs before calling finalizers causes the caches to not be correctly invalidated. That can cause crashes since the caches can refer to invalid objects. Defer the clearing of weakrefs without callbacks until after finalizers are executed.

- gh-136203 [https://github.com/python/cpython/issues/136203]: Improve **TypeError** error message, when richcomparing two **types.MappingProxyType** objects.

- gh-136003 [https://github.com/python/cpython/issues/136003]: Fix **threading.Thread** objects becoming incorrectly daemon when created from an **atexit** callback or a pending call (**Py_AddPendingCall()**).

- gh-78465 [https://github.com/python/cpython/issues/78465]: Corrige mensagem de erro para `cls.__new__(cls, ...)` onde `cls` não é um tipo de extensão ou embutido instanciável (com `tp_new` definido como `NULL`).

- gh-135904 [https://github.com/python/cpython/issues/135904]: Perform more aggressive control-flow optimizations on the machine code templates emitted by the experimental JIT compiler.

- gh-129958 [https://github.com/python/cpython/issues/129958]: Diferencia entre t-strings e f-strings em erros de sintaxe para novas linhas em especificadores de formato de strings interpoladas entre aspas simples.

- gh-135871 [https://github.com/python/cpython/issues/135871]: Tentativas de trava de mutex não bloqueantes agora retornam imediatamente quando a

trava está ocupado, em vez de girar brevemente na construção com threads livres.

- gh-134584 [https://github.com/python/cpython/issues/134584]: Specialize **`POP_TOP`** in the JIT compiler by specializing for reference lifetime and type. This will also enable easier top of stack caching in the JIT compiler.

- gh-135106 [https://github.com/python/cpython/issues/135106]: Restringe o mecanismo da lixeira aos objetos coletados pelo GC e cancele o rastreamento deles enquanto estiverem na lixeira para evitar conflitos entre os mecanismos do GC e da lixeira.

- gh-135379 [https://github.com/python/cpython/issues/135379]: Changes specialization of `BINARY_OP` for ints to only specialize for “compact” ints. This streamlines the fast path at the cost of fewer specializations when very large integers are used.

- gh-135607 [https://github.com/python/cpython/issues/135607]: Corrige possíveis corridas **`weakref`** no destrutor de um objeto na construção com threads livres.

- gh-135608 [https://github.com/python/cpython/issues/135608]: Corrige uma falha no JIT envolvendo atributos de módulos.

- gh-82088 [https://github.com/python/cpython/issues/82088]: Improve performance of `PyLongObject` conversion functions `PyLong_AsLongAndOverflow()`, `PyLong_AsSsize_t()`, `PyLong_AsUnsignedLong()`, `PyLong_AsSize_t()`, `PyLong_AsUnsignedLongMask()`, `PyLong_AsUnsignedLongLongMask()`, `PyLong_AsLongLongAndOverflow()` for integers larger than 2**30 up to 30%.

- gh-135551 [https://github.com/python/cpython/issues/135551]: Sorting randomly ordered lists will often run a bit faster, thanks to a new scheme for picking minimum run lengths from Stefan Pochmann, which arranges for the merge tree to be as evenly balanced as is possible.

- gh-135543 [https://github.com/python/cpython/issues/135543]: Emite o evento de auditoria `sys.remote_exec` quando **`sys.remote_exec()`** for chamado e migra `remote_debugger_script` para `cpython.remote_debugger_script`.

- gh-135496 [https://github.com/python/cpython/issues/135496]: Corrige erro de digitação no tipo de conversão de f-strings (“exclamanation” -> “exclamation”).

- gh-135474 [https://github.com/python/cpython/issues/135474]: Specialize integer operations only on compact integers. This is a CPython internal change.

- gh-135371 [https://github.com/python/cpython/issues/135371]: Corrige as ferramentas de depuração **`asyncio`** para exibir corretamente as pilhas de chamadas de corrotinas internas juntamente com as dependências de tarefas externas. Os comandos `python -m asyncio ps` e `python -m asyncio pstree` agora mostram o contexto de execução completo. Patch por Pablo Galindo.

- gh-135422 [https://github.com/python/cpython/issues/135422]: Fix regression in **`SyntaxError`** messages after gh-134036 [https://github.com/python/cpython/issues/134036].

Biblioteca

- gh-116738 [https://github.com/python/cpython/issues/116738]: Torna as funções em **`grp`** seguros para thread na construção com threads livres.
- gh-127319 [https://github.com/python/cpython/issues/127319]: Define a variável de classe `allow_reuse_port` como `False` nos servidores XMLRPC, de registro e HTTP. Isso corresponde ao comportamento das versões anteriores do Python, que era não permitir a reutilização de portas.

Núcleo e embutidos

- gh-130077 [https://github.com/python/cpython/issues/130077]: Gera corretamente erros de sintaxe personalizados quando for encontrada uma sintaxe

incorreta contendo nomes que são prefixos de palavras reservadas contextuais. Patch por Pablo Galindo.

- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize _CALL_LEN in the JIT when the length is known. Patch by Tomas Roun
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize `_UNARY_NEGATIVE` in JIT-compiled code.
- gh-135148 [https://github.com/python/cpython/issues/135148]: Corrigido um bug em que expressões de depuração de strings f (usando =) removiam incorretamente partes de strings contendo aspas de escape e caracteres #. Patch por Pablo Galindo.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize `_UNARY_INVERT` in JIT-compiled code.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize away `_CALL_TYPE_1` in the JIT when the return type is known. Patch by Tomas Roun
- gh-133136 [https://github.com/python/cpython/issues/133136]: Limita o uso excessivo de memória na construção com threads livres quando um dicionário ou lista grande é redimensionado e acessado por várias threads.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize `_CHECK_METHOD_VERSION` into `_CHECK_FUNCTION_VERSION_INLINE` in JIT-compiled code.

Biblioteca

- gh-116738 [https://github.com/python/cpython/issues/116738]: Torna os métodos em **`heapq`** seguros para thread na construção com threads livres.

Núcleo e embutidos

- gh-134280 [https://github.com/python/cpython/issues/134280]: Desabilita a dobragem constante para ~ com um argumento booleano. Isso move o aviso de descontinuação do tempo de compilação para o tempo de execução.
- gh-134876 [https://github.com/python/cpython/issues/134876]: Adiciona suporte à depuração remota **PEP 768** [https://peps.python.org/pep-0768/] para kernels

Linux que não tenham CONFIG_CROSS_MEMORY_ATTACH configurado.
- gh-134889 [https://github.com/python/cpython/issues/134889]: Corrige o tratamento de alguns opcodes que deixam operandos na pilha ao otimizar `LOAD_FAST`.

Biblioteca

- gh-134908 [https://github.com/python/cpython/issues/134908]: Corrige falha ao iterar sobre linhas em um arquivo texto na construção com threads livres.

Núcleo e embutidos

- gh-117852 [https://github.com/python/cpython/issues/117852]: Fix argument checking of **`athrow()`**.
- gh-132617 [https://github.com/python/cpython/issues/132617]: Corrige verificação de modificação **`dict.update()`** que poderia levantar incorretamente um erro “dict mutated during update” quando um dicionário diferente era modificado e compartilhava o mesmo objeto de chaves subjacente.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Allow the JIT to remove unnecessary `_ITER_CHECK_TUPLE` ops.
- gh-134679 [https://github.com/python/cpython/issues/134679]: Corrige falha no código QSBR da construção com threads livres que poderia ocorrer ao alterar o atributo `__dict__` de um objeto.
- gh-133912 [https://github.com/python/cpython/issues/133912]: Corrige a função da API C `PyObject_GenericSetDict` para manipular classes de extensão com valores embutidos.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Make the JIT optimizer understand that slicing a string/list/tuple returns the same type.
- gh-134584 [https://github.com/python/cpython/issues/134584]: Add a reference count elimination pass to the JIT compiler. Patch by Ken Jin.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Optimize `_POP_CALL_TWO_LOAD_CONST_INLINE_BORROW`.

Biblioteca

- gh-134381 [https://github.com/python/cpython/issues/134381]: Corrige `RuntimeError` ao usar um `threading.Thread` não iniciado após chamar `os.fork()`

Núcleo e embutidos

- gh-127960 [https://github.com/python/cpython/issues/127960]: O shell interativo do PyREPL não inicia mais com os nomes globais `__package__` e `__file__` definidos como internos do pacote `_pyrepl`. Contribuição de Yuichiro Tachibana.
- gh-130397 [https://github.com/python/cpython/issues/130397]: Remove o uso de maiúsculas e minúsculas especiais para os limites de profundidade da pilha C para WASI. Devido à proteção de pilha embutida do WebAssembly, isso não representa um problema de segurança.
- gh-131798 [https://github.com/python/cpython/issues/131798]: JIT: replace `_LOAD_SMALL_INT` with `_LOAD_CONST_INLINE_BORROW`
- gh-131798 [https://github.com/python/cpython/issues/131798]: Improve the JIT's ability to optimize away cached class attribute and method loads.
- gh-128066 [https://github.com/python/cpython/issues/128066]: Corrige um caso extremo em que o PyREPL levantava um erro incorretamente quando o Python era invocado em um sistema de arquivos somente leitura ao tentar gravar entradas no arquivo de histórico.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Improve the JIT's ability to narrow unknown classes to constant values.
- gh-134268 [https://github.com/python/cpython/issues/134268]: Add `_POP_CALL_TWO_LOAD_CONST_INLINE_BORROW` and use it to further optimize `CALL_ISINSTANCE`.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Split `CALL_LIST_APPEND` into several uops. Patch by Diego Russo.
- gh-69605 [https://github.com/python/cpython/issues/69605]: Ao completar automaticamente uma importação no REPL, não encontrar candidatos agora não emite nenhuma sugestão, em vez de sugestões do espaço de nomes atual.
- gh-134170 [https://github.com/python/cpython/issues/134170]: Add colorization to `sys.unraisablehook()` by default.
- gh-91153 [https://github.com/python/cpython/issues/91153]: Corrige uma falha quando um `bytearray` é mutado simultaneamente durante a atribuição

de item.

- gh-134158 [https://github.com/python/cpython/issues/134158]: Corrige a coloração de chaves duplas em f-strings e t-strings no REPL.
- gh-134119 [https://github.com/python/cpython/issues/134119]: Corrige falha ao chamar `next()` em um iterador de string template esgotado. Patch por Jelle Zijlstra.
- gh-134100 [https://github.com/python/cpython/issues/134100]: Corrige um bug de uso após liberação que ocorre quando um módulo importado não está em `sys.modules` após sua importação inicial. Patch por Nico-Posada.
- gh-134036 [https://github.com/python/cpython/issues/134036]: Improve `SyntaxError` message when using invalid `raise` statements.
- gh-133999 [https://github.com/python/cpython/issues/133999]: Corrige regressão `SyntaxError` na análise de `except` após gh-123440 [https://github.com/python/cpython/issues/123440].
- gh-133886 [https://github.com/python/cpython/issues/133886]: Corrige `sys.remote_exec()` para caminhos não-ASCII em localidades não-UTF-8 e caminhos não-UTF-8 em localidades UTF-8.
- gh-133400 [https://github.com/python/cpython/issues/133400]: Comportamento corrigido do Ctrl+D (^D) no módulo _pyrepl para corresponder ao comportamento antigo do REPL pré-3.13.
- gh-133703 [https://github.com/python/cpython/issues/133703]: Corrige que a tabela de hash no dicionário pode ser maior do que o pretendido em algumas situações.
- gh-133778 [https://github.com/python/cpython/issues/133778]: Corrige bug onde atribuir a atributos `__annotations__` de classes definidas em `from __future__ import annotations` não tinham efeito.
- gh-133711 [https://github.com/python/cpython/issues/133711]: Implement **PEP 686** [https://peps.python.org/pep-0686/]: Enable Python UTF-8 Mode by default. Patch by Adam Turner.
- gh-132762 [https://github.com/python/cpython/issues/132762]: `fromkeys()` não entra mais em laço infinito ao adicionar um pequeno conjunto de chaves a um dicionário base grande. Patch por Angela Liss.
- gh-133541 [https://github.com/python/cpython/issues/133541]: Indentação inconsistente na entrada do usuário travava o novo REPL quando o realce de sintaxe estava ativo. Isso já foi corrigido.
- gh-133516 [https://github.com/python/cpython/issues/133516]: Levanta `ValueError` quando as constantes `True`, `False` ou `None` forem usadas

como um identificador após a normalização NFKC.

- gh-131798 [https://github.com/python/cpython/issues/131798]: Allow the JIT to remove int guards after `_GET_LEN` by setting the return type to int.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Split `CALL_ISINSTANCE` into several uops, allowing the JIT to remove some of them.
- gh-132554 [https://github.com/python/cpython/issues/132554]: Change iteration to use “virtual iterators” for sequences. Instead of creating an iterator, a tagged integer representing the next index is pushed to the stack above the iterable. For non-sequence iterators, `NULL` is pushed.
- gh-130821 [https://github.com/python/cpython/issues/130821]: Enhance wrong type error messages and make them more consistent. Patch by Semyon Moroz.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Narrow the return type and constant-evaluate `CALL_ISINSTANCE` for a subset of known values in the JIT. Patch by Tomas Roun
- gh-132542 [https://github.com/python/cpython/issues/132542]: Atualiza **`Thread.native_id`** após *fork(2)* *[https://manpages.debian.org/fork(2)]* para garantir a precisão. Patch por Noam Cohen.

Biblioteca

- gh-132732 [https://github.com/python/cpython/issues/132732]: Automatically constant evaluate bytecode operations marked as pure in the JIT optimizer.

Núcleo e embutidos

- gh-127971 [https://github.com/python/cpython/issues/127971]: Corrige leituras fora de um ponto além do final de uma string na pesquisa de string.
- gh-132042 [https://github.com/python/cpython/issues/132042]: Improve class creation times by up to 12% by pre-computing type slots just once. Patch by Sergey Miryanov.
- gh-133379 [https://github.com/python/cpython/issues/133379]: Correct usage of *arguments* in error messages.
- gh-127266 [https://github.com/python/cpython/issues/127266]: In the free-threaded build, avoid data races caused by updating type slots or type flags after

the type was initially created. For those (typically rare) cases, use the stop-the-world mechanism. Remove the use of atomics when reading or writing type flags. The use of atomics is not sufficient to avoid races (since flags are sometimes read without a lock and without atomics) and are no longer required.

- gh-130425 [https://github.com/python/cpython/issues/130425]: Add `"Did you mean: 'attr'?"` suggestion when using `del obj.attr` if `attr` does not exist.
- gh-128640 [https://github.com/python/cpython/issues/128640]: Corrige uma falha ao usar threads dentro de um subinterpretador.

Biblioteca

- gh-116738 [https://github.com/python/cpython/issues/116738]: Make the module **`json`** safe to use under the free-threading build.

Núcleo e embutidos

- gh-119494 [https://github.com/python/cpython/issues/119494]: Exception text when trying to delete attributes of types was clarified.

API C

- gh-139924 [https://github.com/python/cpython/issues/139924]: Function watchers can now receive a PyFunction_PYFUNC_EVENT_MODIFY_QUALNAME event when a watched functions qualname is changed.

- gh-111489 [https://github.com/python/cpython/issues/111489]: Add **`PyTuple_FromArray()`** to create a **`tuple`** from an array. Patch by Victor Stinner.

- gh-136355 [https://github.com/python/cpython/issues/136355]: Deprecate **`PyConfig.bytes_warning`** field and schedule its removal in 3.17.

- gh-138886 [https://github.com/python/cpython/issues/138886]: Remove deprecated **`PySys_ResetWarnOptions()`** C-API function.

- gh-129813 [https://github.com/python/cpython/issues/129813]: Implement **PEP 782** [https://peps.python.org/pep-0782/], the `PyBytesWriter` API. Add functions:

  - `PyBytesWriter_Create()`
  - `PyBytesWriter_Discard()`
  - `PyBytesWriter_FinishWithPointer()`
  - `PyBytesWriter_FinishWithSize()`
  - `PyBytesWriter_Finish()`
  - `PyBytesWriter_Format()`
  - `PyBytesWriter_GetData()`
  - `PyBytesWriter_GetSize()`
  - `PyBytesWriter_GrowAndUpdatePointer()`
  - `PyBytesWriter_Grow()`
  - `PyBytesWriter_Resize()`
  - `PyBytesWriter_WriteBytes()`

  Patch por Victor Stinner.

- gh-137956 [https://github.com/python/cpython/issues/137956]: Display and raise an exception if an extension compiled for non-free-threaded Python is loaded in a free-threaded interpreter.

- gh-137573 [https://github.com/python/cpython/issues/137573]: Marca `_PyOptimizer_Optimize` como `Py_NO_INLINE` para evitar travamentos de estouro de pilha no macOS.

- gh-128813 [https://github.com/python/cpython/issues/128813]: Functions `_Py_c_sum()`, `_Py_c_diff()`, `_Py_c_neg()`, `_Py_c_prod()`, `_Py_c_quot()`, `_Py_c_pow()` and previously undocumented `_Py_c_abs()` are soft deprecated. Deprecate also `cval` field of the `PyComplexObject` type. Patch by Sergey B Kirpichev.

- gh-137210 [https://github.com/python/cpython/issues/137210]: Add API for checking an extension module's ABI compatibility: `Py_mod_abi`, `PyABIInfo_Check()`, `PyABIInfo_VAR` and `Py_mod_abi`.

- gh-136759 [https://github.com/python/cpython/issues/136759]: Rename `lock.h` to `pylock.h` to avoid potential include conflicts.

- gh-112068 [https://github.com/python/cpython/issues/112068]: Reverte suporte de argumentos anuláveis em **`PyArg_Parse()`**.

- gh-136006 [https://github.com/python/cpython/issues/136006]: On Solaris, the **`Py_NAN`** macro now expands to a **`double`** instead of a function address. Patch by Bénédikt Tran.

- gh-135075 [https://github.com/python/cpython/issues/135075]: Make **`PyObject_SetAttr()`** and **`PyObject_SetAttrString()`** fail if called with `NULL` value and an exception set. Patch by Victor Stinner.

- gh-135906 [https://github.com/python/cpython/issues/135906]: Corrige erros de construção ao compilar os cabeçalhos internos com um compilador C++.

- gh-133296 [https://github.com/python/cpython/issues/133296]: Novas variantes para a API da seção crítica que aceitam um ou dois ponteiros **`PyMutex`** em vez de instâncias **`PyObject`** agora são públicas na API C não limitada.

- gh-133157 [https://github.com/python/cpython/issues/133157]: Remove the private, undocumented macro **`_Py_NO_SANITIZE_UNDEFINED`**.

- gh-134989 [https://github.com/python/cpython/issues/134989]: Corrige as macros `Py_RETURN_NONE`, `Py_RETURN_TRUE` e `Py_RETURN_FALSE` na API C limitada 3.11 e anteriores: não trata `Py_None`, `Py_True` e `Py_False` como imortais. Patch por Victor Stinner.

- gh-134989 [https://github.com/python/cpython/issues/134989]: Implementa **`PyObject_DelAttr()`** e **`PyObject_DelAttrString()`** como macros na API C limitada 3.12 e anteriores. Patch por Victor Stinner.

- gh-134745 [https://github.com/python/cpython/issues/134745]: Change **`PyThread_allocate_lock()`** implementation to `PyMutex`. On

Windows, **`PyThread_acquire_lock_timed()`** now supports the *intr_flag* parameter: it can be interrupted. Patch by Victor Stinner.

- gh-134891 [https://github.com/python/cpython/issues/134891]: Add **`PyUnstable_Unicode_GET_CACHED_HASH`** to get the cached hash of a string.

- gh-134009 [https://github.com/python/cpython/issues/134009]: Expõe **`PyMutex_IsLocked()`** como parte da API C pública.

- gh-134144 [https://github.com/python/cpython/issues/134144]: Corrige falha ao chamar **`Py_EndInterpreter()`** com um estado de thread que não é a thread inicial do interpretador.

- gh-133968 [https://github.com/python/cpython/issues/133968]: Adiciona a função **`PyUnicodeWriter_WriteASCII()`** para escrever uma string ASCII em um **`PyUnicodeWriter`**. A função é mais rápida que **`PyUnicodeWriter_WriteUTF8()`**, mas tem um comportamento indefinido se a string de entrada contiver caracteres não ASCII. Patch por Victor Stinner.

- gh-133644 [https://github.com/python/cpython/issues/133644]: Remove deprecated Python initialization getter functions `Py_Get*`. Patch by Bénédikt Tran.

- gh-133644 [https://github.com/python/cpython/issues/133644]: Remove deprecated function **`PyWeakref_GetObject()`** and macro **`PyWeakref_GET_OBJECT`**. Use **`PyWeakref_GetRef()`** instead. Patch by Bénédikt Tran.

- gh-133644 [https://github.com/python/cpython/issues/133644]: Remove deprecated alias **`PyImport_ImportModuleNoBlock()`** of **`PyImport_ImportModule()`**. Patch by Bénédikt Tran.

- gh-133610 [https://github.com/python/cpython/issues/133610]: Remove deprecated functions **`PyUnicode_AsDecodedObject()`**, **`PyUnicode_AsDecodedUnicode()`**, **`PyUnicode_AsEncodedObject()`**, and **`PyUnicode_AsEncodedUnicode()`**.

- gh-132629 [https://github.com/python/cpython/issues/132629]: For unsigned integer formats in `PyArg_ParseTuple()`, accepting Python integers with value that is larger than the maximal value for the C type or less than the minimal value for the corresponding signed integer type of the same size is now deprecated.

- gh-131185 [https://github.com/python/cpython/issues/131185]: `PyGILState_Ensure()` no longer crashes when called after interpreter finalization.

- gh-108512 [https://github.com/python/cpython/issues/108512]: Add functions `PySys_GetAttr()`, `PySys_GetAttrString()`, `PySys_GetOptionalAttr()` and `PySys_GetOptionalAttrString()`.

Construção

- gh-138489 [https://github.com/python/cpython/issues/138489]: Ao fazer compilação cruzada para WASI por `build_wasm` ou `build_emscripten`, a etapa `build-details.json` agora é incluída no processo de construção, assim como nas construções nativas.

  Isso corrige a tarefa `libinstall` que requer o arquivo `build-details.json` durante o processo.

- gh-138497 [https://github.com/python/cpython/issues/138497]: The LLVM version used by the JIT at build time can now be modified using the `LLVM_VERSION` environment variable. Use this at your own risk, as there is only one officially supported LLVM version. For more information, please check `Tools/jit/README.md`.

- gh-95952 [https://github.com/python/cpython/issues/95952]: When cross-compiling for WASI, require that the HOSTRUNNER environment variable be explicitly set.

  This was needed as macOS lacks the appropriate CLI tools to set a reasonable default.

- gh-138061 [https://github.com/python/cpython/issues/138061]: Ensure reproducible builds by making JIT stencil header generation deterministic.

- gh-128042 [https://github.com/python/cpython/issues/128042]: `./configure` now warns when `--enable-optimizations` and `CFLAGS=-O0` are both set, suggesting removing `-O0` from `CFLAGS` for optimal performance. Patch by Taegyun Kim.

- gh-132339 [https://github.com/python/cpython/issues/132339]: Adiciona suporte para OpenSSL 3.5.

- gh-135621 [https://github.com/python/cpython/issues/135621]: O PyREPL não depende mais da biblioteca padrão **`curses`**. Contribuição de Łukasz Langa.

- gh-135927 [https://github.com/python/cpython/issues/135927]: Fix building with MSVC when passing option `/std:clatest`.

- gh-119132 [https://github.com/python/cpython/issues/119132]: Remove a tag “experimental” da construção com threads livres do CPython.

- gh-135497 [https://github.com/python/cpython/issues/135497]: Corrige a detecção de `MAXLOGNAME` no script `configure.ac`.

- gh-134923 [https://github.com/python/cpython/issues/134923]: As construções do Windows com otimização guiada por perfil habilitada agora usam `/GENPROFILE` e `/USEPROFILE` em vez das opções descontinuadas `/LTCG:`.

- gh-134632 [https://github.com/python/cpython/issues/134632]: Corrigida a geração de `build-details.json` para usar `INCLUDEPY`, a fim de referenciar o subdiretório `pythonX.Y` do diretório include, conforme exigido em **PEP 739** [https://peps.python.org/pep-0739/], em vez do diretório include de nível superior.

- gh-134486 [https://github.com/python/cpython/issues/134486]: The **`ctypes`** module now performs a more portable test for the definition of *alloca(3)*

[https://manpages.debian.org/alloca(3)], fixing a compilation failure on NetBSD.

- gh-134455 [https://github.com/python/cpython/issues/134455]: Corrigida a geração de `build-details.json` para usar o `c_api.headers` correto, conforme definido em **PEP 739** [https://peps.python.org/pep-0739/], em vez de `c_api.include`.

- gh-134273 [https://github.com/python/cpython/issues/134273]: Adiciona suporte para configurar sinalizadores do compilador para o JIT com `CFLAGS_JIT`

- gh-115119 [https://github.com/python/cpython/issues/115119]: Removed implicit fallback to the bundled copy of the `libmpdec` library. Now this should be explicitly enabled via **`--with-system-libmpdec`** set to `no` or **`--without-system-libmpdec`**. Patch by Sergey B Kirpichev.

- gh-131769 [https://github.com/python/cpython/issues/131769]: Corrige a detecção de quando a construção Python em uma construção cruzada é uma construção de pydebug.

- gh-117088 [https://github.com/python/cpython/issues/117088]: O ligador AIX não oferece suporte à opção -h, portanto, evita-a por meio da verificação da plataforma

- gh-123681 [https://github.com/python/cpython/issues/123681]: Verifica o comportamento de `strftime()` em tempo de execução, em vez de em tempo de compilação, para oferecer suporte à compilação cruzada. Remove a macro interna `_Py_NORMALIZE_CENTURY`.

- gh-127545 [https://github.com/python/cpython/issues/127545]: Fix crash when building on Linux/m68k.

Python 3.14.0 beta 1

Data de lançamento: 2025-05-06

Windows

- gh-132930 [https://github.com/python/cpython/issues/132930]: Marca o instalador para Windows como descontinuado e atualiza a documentação para abranger o novo gerenciador de instalação do Python.
- gh-127405 [https://github.com/python/cpython/issues/127405]: Adiciona `ABIFLAGS` a **`sysconfig.get_config_vars()`** no Windows. Patch por Xuehai Pan.

Ferramentas/Demos

- gh-130453 [https://github.com/python/cpython/issues/130453]: Permite a passagem de múltiplos argumentos nomeados com o mesmo nome de função em **pygettext**.
- gh-130195 [https://github.com/python/cpython/issues/130195]: Adiciona mensagens de aviso quando a opção não implementada `-a/--extract-all` do **pygettext** for chamada.

Testes

- gh-133131 [https://github.com/python/cpython/issues/133131]: O ambiente de testes do iOS agora selecionará o dispositivo “classe SE” lançado mais recentemente para testes, caso um dispositivo não seja especificado explicitamente.
- gh-91048 [https://github.com/python/cpython/issues/91048]: Adiciona a capacidade de inspecionar externamente todas as tarefas asyncio pendentes, mesmo que nenhuma tarefa esteja inserida no laço de eventos.
- gh-109981 [https://github.com/python/cpython/issues/109981]: O auxiliar de teste que conta a lista de descritores de arquivo abertos agora usa a abordagem otimizada `/dev/fd` em todas as plataformas Apple, não apenas no macOS. Isso evita travamentos causados por descritores de arquivo protegidos.
- gh-132678 [https://github.com/python/cpython/issues/132678]: Adiciona `--prioritize` a `-m test`. Esta opção permite ao usuário especificar quais testes selecionados devem ser executados primeiro, mesmo que a ordem seja aleatória. Isso é particularmente útil para testes que demoram mais tempo para serem executados.

- gh-131290 [https://github.com/python/cpython/issues/131290]: Os testes em `Lib/test` agora podem ser executados corretamente como scripts independentes.

Segurança

- gh-115322 [https://github.com/python/cpython/issues/115322]: Os módulos de extensão subjacentes a **`readline`**:, **`subprocess`** e **`ctypes`** agora levantam eventos de auditoria em caminhos de código previamente descobertos, que podem levar ao acesso ao sistema de arquivos relacionado à chamada de função C e à execução de binários externos. O gancho de auditoria `ctypes.call_function` também foi corrigido para usar um valor sem sinal para seu `function pointer`.

Biblioteca

- gh-133490 [https://github.com/python/cpython/issues/133490]: Adiciona suporte de cores ao PDB no modo remoto.

- gh-132493 [https://github.com/python/cpython/issues/132493]: Evita avaliar ansiosamente anotações em funções decoradas com **`reprlib.recursive_repr()`**.

- gh-130645 [https://github.com/python/cpython/issues/130645]: Adiciona cor às CLIs de argparse da stdlib. Patch por Hugo van Kemenade.

- gh-119180 [https://github.com/python/cpython/issues/119180]: Faz com que **`annotationlib.get_annotations()`** tenha sucesso com o formato `FORWARDREF` se a avaliação das anotações gerar uma exceção diferente de **`NameError`** ou **`AttributeError`**.

- gh-133351 [https://github.com/python/cpython/issues/133351]: Corrige o PDB remoto para solicitar corretamente a conclusão de tabulações para expressões Python do servidor ao concluir uma linha de continuação de um bloco Python de várias linhas.

- gh-133367 [https://github.com/python/cpython/issues/133367]: Adiciona as opções `--feature-version`, `--optimize` e `--show-empty` à interface de linha de comando **`ast`**. Patch por Semyon Moroz.

- gh-133363 [https://github.com/python/cpython/issues/133363]: A classe **`cmd.Cmd`** foi corrigida para chamar de forma confiável o método `completedefault` sempre que o método `do_shell` não estiver definido e a conclusão da tabulação for solicitada para uma linha que começa com `!`.

- gh-113081 [https://github.com/python/cpython/issues/113081]: Realça a sintaxe no código-fonte em **`pdb`**.

- gh-133349 [https://github.com/python/cpython/issues/133349]: Introduzida a indentação automática na entrada multilinha **`pdb`**.

- gh-133306 [https://github.com/python/cpython/issues/133306]: Usa `\z` em vez de `\Z` em **`fnmatch.translate()`** e **`glob.translate()`**.

- gh-133306 [https://github.com/python/cpython/issues/133306]: Suporte a `\z` como sinônimo de `\Z` em **`expressões regulares`**.

- gh-133300 [https://github.com/python/cpython/issues/133300]: Torna `suggest_on_error` de **`argparse.ArgumentParser`** um parâmetro somente-nomeado. Patch por Hugo van Kemenade.

- gh-133290 [https://github.com/python/cpython/issues/133290]: Corrige problema de armazenamento em cache de atributo ao definir **`ctypes._Pointer._type_`** na função não documentada e descontinuada **`ctypes.SetPointerType()`** e no método não documentado **`set_type()`**.

- gh-133223 [https://github.com/python/cpython/issues/133223]: Quando o PDB estiver anexado a um processo remoto, faz um trabalho melhor de interceptar Ctrl+C e encaminhá-lo ao processo remoto.

- gh-133153 [https://github.com/python/cpython/issues/133153]: Não conclui os comandos **`pdb`** no modo `interact` do **`pdb`**.

- gh-133139 [https://github.com/python/cpython/issues/133139]: Adiciona a função `curses.assume_default_colors()`, um refinamento da função `curses.use_default_colors()` que permite alterar o par de cores `0`.

- gh-133089 [https://github.com/python/cpython/issues/133089]: Usa o valor de tempo limite original para `subprocess.TimeoutExpired` quando a função `subprocess.run()` for chamada com um tempo limite em vez de, às vezes, um valor de tempo limite parcial restante confuso usado internamente no `wait()` final.

- gh-133036 [https://github.com/python/cpython/issues/133036]: `codecs.open()` está descontinuada. Em vez disso, use `open()`. Contribuição de Inada Naoki.

- gh-132987 [https://github.com/python/cpython/issues/132987]: Muitas funções embutidas e de extensão que aceitam um argumento inteiro sem sinal agora usam `__index__()` se disponível.

- gh-124703 [https://github.com/python/cpython/issues/124703]: Define o código de retorno como `1` ao abortar o processo de `pdb`.

- gh-133005 [https://github.com/python/cpython/issues/133005]: Suporte à passagem da opção `preset` para `tarfile.open()` ao usar o modo `'w|xz'`.

- gh-115032 [https://github.com/python/cpython/issues/115032]: O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores com o argumento *stream*. Patch por Mariusz Felisiak.

- gh-132991 [https://github.com/python/cpython/issues/132991]: Adiciona a constante **`socket.IP_FREEBIND`** no Linux 2.4 e posteriores.

- gh-132995 [https://github.com/python/cpython/issues/132995]: Atualiza a versão do pip empacotado no ensurepip para a versão 25.1.1

- gh-132933 [https://github.com/python/cpython/issues/132933]: O módulo zipapp agora aplica o filtro ao criar a lista de arquivos a serem adicionados,

em vez de esperar até que o arquivo seja adicionado ao arquivo.

- gh-121249 [https://github.com/python/cpython/issues/121249]: Sempre suporte os tipos C float complex e double complex no módulo `struct`. Patch por Sergey B Kirpichev.

- gh-132915 [https://github.com/python/cpython/issues/132915]: `fcntl.fcntl()` e `fcntl.ioctl()` agora podem detectar um estouro de buffer e levantar `SystemError`. A pilha e a memória podem ser corrompidas nesse caso, portanto, trata esse erro como fatal.

- gh-132017 [https://github.com/python/cpython/issues/132017]: Corrige erro quando `pyrepl` é suspenso e depois retomado e encerrado.

- gh-132893 [https://github.com/python/cpython/issues/132893]: Precisão aprimorada de `statistics.NormalDist.cdf()` para entradas menores que a média.

- gh-130328 [https://github.com/python/cpython/issues/130328]: Aceleração da colagem em `PyREPL` no Windows. Correção por Chris Eibl.

- gh-132882 [https://github.com/python/cpython/issues/132882]: Corrige a cópia de objetos `typing.Union` contendo objetos que não oferecem o suporte ao operador `|`.

- gh-93696 [https://github.com/python/cpython/issues/93696]: Corrigido o erro de exibição do ponto de interrupção para módulos congelados em `pdb`.

- gh-129965 [https://github.com/python/cpython/issues/129965]: Adiciona tipos MIME para `.7z`, `.apk`, `.deb`, `.glb`, `.gltf`, `.gz`, `.m4v`, `.php`, `.rar`, `.rpm`, `.stl` e `.wmv`. Patch por Hugo van Kemenade.

- gh-132742 [https://github.com/python/cpython/issues/132742]: `fcntl.fcntl()` agora oferece suporte a objetos bytes ou similares arbitrários, não apenas `bytes`. `fcntl.ioctl()` agora tenta novamente chamadas de sistema que falham com EINTR e libera a GIL durante uma chamada de sistema, mesmo para grandes objetos bytes ou similares.

- gh-132451 [https://github.com/python/cpython/issues/132451]: A CLI do depurador PDB agora aceita o argumento `-p PID` para permitir a anexação a um processo em execução. O processo deve estar executando a mesma versão do Python que o PDB.

- gh-125618 [https://github.com/python/cpython/issues/125618]: Adiciona um parâmetro *format* a **`annotationlib.ForwardRef.evaluate()`**. A avaliação de anotações no formato `FORWARDREF` agora é bem-sucedida em mais casos que antes levantavam uma exceção.

- gh-132805 [https://github.com/python/cpython/issues/132805]: Corrige tratamento incorreto de valores não constantes aninhados no formato FORWARDREF em **`annotationlib`**.

- gh-132734 [https://github.com/python/cpython/issues/132734]: Adiciona novas constantes para Bluetooth **`sockets`**.

- gh-132673 [https://github.com/python/cpython/issues/132673]: Corrige **`AssertionError`** levantada em **`ctypes.Structure`** com `_align_ = 0` e `_fields_ = []`.

- gh-132578 [https://github.com/python/cpython/issues/132578]: Renomeia o campo `threading.Thread._handle` para evitar sombrear métodos definidos em subclasses de `threading.Thread`.

- gh-132561 [https://github.com/python/cpython/issues/132561]: Corrige o método público `locked` da classe `multiprocessing.SemLock`. Adiciona também 2 testes para as classes derivadas **`multiprocessing.Lock`** e **`multiprocessing.RLock`**.

- gh-121468 [https://github.com/python/cpython/issues/121468]: Adiciona a função **`pdb.set_trace_async()`** para oferece suporte a instruções **`await`** em **`pdb`**.

- gh-132493 [https://github.com/python/cpython/issues/132493]: **`typing.Protocol`** agora usa **`annotationlib.get_annotations()`** ao verificar se uma instância implementa ou não o protocolo com **`isinstance()`**. Isso

habilita o suporte para verificações `isinstance` em classes com anotações adiadas.

- gh-132536 [https://github.com/python/cpython/issues/132536]: Não desabilitar o evento **`PY_THROW`** em **`bdb`** porque ele não pode ser desabilitado.

- gh-132527 [https://github.com/python/cpython/issues/132527]: Inclui o typecode válido ‘w’ na mensagem de erro quando um typecode inválido for passado para **`array.array`**.

- gh-132099 [https://github.com/python/cpython/issues/132099]: O soquete Bluetooth com o protocolo **`BTPROTO_HCI`** no Linux agora aceita um endereço no formato de um inteiro `device_id`, não apenas uma tupla `(device_id,)`.

- gh-81793 [https://github.com/python/cpython/issues/81793]: Corrige **`os.link()`** em plataformas (como Linux) onde a função de sistema **`link()`** não segue links simbólicos. No Linux, agora ela segue links simbólicos por padrão ou se `follow_symlinks=True` for especificado. No Windows, agora levanta um erro se `follow_symlinks=True` for passado. No macOS, agora levanta um erro se `follow_symlinks=False` for passado e a função de sistema **`linkat()`** não estiver disponível em tempo de execução.

- gh-132493 [https://github.com/python/cpython/issues/132493]: Suporte à criação de classes **`typing.Protocol`** com anotações que não podem ser resolvidas no momento da criação da classe.

- gh-132491 [https://github.com/python/cpython/issues/132491]: Renomeia `annotationlib.value_to_string` para **`annotationlib.type_repr()`** e forneça melhor tratamento para objetos função.

- gh-132426 [https://github.com/python/cpython/issues/132426]: Adiciona **`annotationlib.get_annotate_from_class_namespace()`** como um auxiliar para acessar anotações em metaclasses e remova `annotationlib.get_annotate_function`.

- gh-70145 [https://github.com/python/cpython/issues/70145]: Adiciona suporte para canais no protocolo Bluetooth HCI (`BTPROTO_HCI`).

- gh-131913 [https://github.com/python/cpython/issues/131913]: Adiciona uma função de atalho `multiprocessing.Process.interrupt()` junto com as existentes `multiprocessing.Process.terminate()` e `multiprocessing.Process.kill()` para um controle melhorado sobre o término do processo filho.

- gh-132439 [https://github.com/python/cpython/issues/132439]: Corrigido `PyREPL` no Windows: caracteres inseridos via AltGr são engolidos. Patch por Chris Eibl.

- gh-132429 [https://github.com/python/cpython/issues/132429]: Corrige o suporte a soquetes Bluetooth no NetBSD e no DragonFly BSD. Adiciona suporte para *cid* e *bdaddr_type* no endereço BTPROTO_L2CAP no FreeBSD. Retorna *cid* em `getsockname()` para BTPROTO_L2CAP se for diferente de zero.

- gh-132106 [https://github.com/python/cpython/issues/132106]: `QueueListener.start` agora levanta uma `RuntimeError` se o listener já estiver iniciado.

- gh-132417 [https://github.com/python/cpython/issues/132417]: Corrige uma desreferência de ponteiro `NULL` quando uma função C chamada usando `ctypes` com `restype` `py_object` retorna `NULL`.

- gh-132385 [https://github.com/python/cpython/issues/132385]: Corrige sugestões de erro de instância que acionam possíveis exceções em `object.__getattr__()` em `traceback`.

- gh-125866 [https://github.com/python/cpython/issues/125866]: Adiciona o argumento opcional *add_scheme* a `urllib.request.pathname2url()`; quando definido como verdadeiro, uma URL completa é retornada. Da mesma forma, adicione o argumento opcional *require_scheme* a `url2pathname()`; quando definido como verdadeiro, uma URL completa é aceita.

- gh-132308 [https://github.com/python/cpython/issues/132308]: Uma exceção `traceback.TracebackException` agora renderiza corretamente os atributos `__context__` e `__cause__` da `Exception` falsa, e o atributo `exceptions` de falsey `ExceptionGroup`.

- gh-130645 [https://github.com/python/cpython/issues/130645]: Adiciona cor à saída de ajuda do `argparse`. Patch por Hugo van Kemenade.

- gh-127495 [https://github.com/python/cpython/issues/127495]: No PyREPL, adicione uma nova entrada ao arquivo `PYTHON_HISTORY` *após* cada instrução. Isso deve preservar o histórico da linha de comando após o encerramento do interpretador. Patch por Sergey B Kirpichev.

- gh-129463 [https://github.com/python/cpython/issues/129463]: A comparação de objetos `annotationlib.ForwardRef` não usa mais os atributos internos `__code__` e `__ast_node__`, que são usados como caches.

- gh-132250 [https://github.com/python/cpython/issues/132250]: Fixed the `SystemError` in `cProfile` when locating the actual C function of a method raises an exception.

- gh-132064 [https://github.com/python/cpython/issues/132064]: `annotationlib.get_annotations()` agora usa o atributo `__annotate__` se estiver presente, mesmo que `__annotations__` não esteja presente. Além disso, a função agora levanta uma exceção `TypeError` se for passado um objeto que não possui nenhuma anotação.

- gh-130664 [https://github.com/python/cpython/issues/130664]: Suporte ao separador de dígitos `'_'` na formatação da parte integral de `Decimal`'s. Patch por Sergey B Kirpichev.

- gh-131952 [https://github.com/python/cpython/issues/131952]: Adiciona a saída de cor à CLI **json**. Patch por Tomas Roun.

- gh-132063 [https://github.com/python/cpython/issues/132063]: Evita que exceções avaliadas como falsas (ou seja, quando o método `__bool__` retorna `False` ou o método `__len__` retorna 0) sejam ignoradas por

`concurrent.futures.ProcessPoolExecutor` e `concurrent.futures.ThreadPoolExecutor`.

- gh-132106 [https://github.com/python/cpython/issues/132106]: `logging.handlers.QueueListener` agora implementa o protocolo do gerenciador de contexto, permitindo que ele seja usado em uma instrução `with`.

- gh-132054 [https://github.com/python/cpython/issues/132054]: O tipo mime `application/yaml` (**RFC 9512** [https://datatracker.ietf.org/doc/html/rfc9512.html]) agora é suportado por `mimetypes`. Patch por Sasha “Nelie” Chernykh e Hugo van Kemenade.

- gh-119605 [https://github.com/python/cpython/issues/119605]: Respeita `follow_wrapped` para os métodos `__init__()` e `__new__()` ao obter a assinatura de classe para uma classe com `inspect.signature()`. Preserva a assinatura de classe após o encapsulamento com `warnings.deprecated()`. Patch por Xuehai Pan.

- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação do módulo `shlex` em até 33x e o desempenho de `shlex.quote()` em até 12x. Patch por Adam Turner.

- gh-85302 [https://github.com/python/cpython/issues/85302]: Adiciona suporte para `BTPROTO_SCO` em soquetes no FreeBSD.

- gh-131757 [https://github.com/python/cpython/issues/131757]: Faz com que `functools.lru_cache()` chame a função destravada em cache para permitir a simultaneidade.

- gh-131423 [https://github.com/python/cpython/issues/131423]: `ssl` pode exibir descrições de erros adicionados no OpenSSL 3.4.1. Patch por Bénédikt Tran.

- gh-131434 [https://github.com/python/cpython/issues/131434]: Melhora o relatório de erros para formato incorreto em `time.strptime()`.

- gh-131524 [https://github.com/python/cpython/issues/131524]: Adicionada mensagem de ajuda à interface de linha de comando `platform`. Contribuição de Harry Lees.

- gh-100926 [https://github.com/python/cpython/issues/100926]: Move o cache dos tipos `ctypes.POINTER()` de um cache interno global (`_pointer_type_cache`) para o atributo `ctypes._CData.__pointer_type__` dos tipos `ctypes` correspondentes. Isso impedirá que o cache cresça sem limites em algumas situações.

- gh-85702 [https://github.com/python/cpython/issues/85702]: Se `zoneinfo._common.load_tzdata` receber um pacote sem um recurso, uma `ZoneInfoNotFoundError` será levantada em vez de uma `IsADirectoryError`.

- gh-123471 [https://github.com/python/cpython/issues/123471]: Torna as iterações simultâneas sobre `itertools.repeat` seguras sob threads livres

- gh-131127 [https://github.com/python/cpython/issues/131127]: Sistemas que usam LibreSSL agora são construídos com sucesso.

- gh-89157 [https://github.com/python/cpython/issues/89157]: Cria a implementação Python pura de `datetime.date.fromisoformat()`, aceitando apenas strings ASCII para consistência com a implementação C.

- gh-130941 [https://github.com/python/cpython/issues/130941]: Corrige `configparser.ConfigParser` analisando interpolação vazia com `allow_no_value` definido como `True`.

- gh-110067 [https://github.com/python/cpython/issues/110067]: Torna públicas as funções max-heap `heapqheapq.heapify_max()`, `heapq.heappush_max()`, `heapq.heappop_max()` e `heapq.heapreplace_max()`. A nomenclatura sublinhada anterior foi mantida para retrocompatibilidade. Além disso, a função ausente `heapq.heappushpop_max()` foi adicionada às implementações em C e Python.

- gh-129098 [https://github.com/python/cpython/issues/129098]: Corrige o relatório de traceback do REPL ao usar `compile()` com um arquivo inexistente. Patch por Bénédikt Tran.

- gh-130631 [https://github.com/python/cpython/issues/130631]: **`http.cookiejar.join_header_words()`** agora é mais semelhante à versão original do Perl. Agora, ele coloca aspas o mesmo conjunto de caracteres e sempre coloca aspas em valores que terminam com `"\n"`.

- gh-130482 [https://github.com/python/cpython/issues/130482]: Adiciona a capacidade de especificar o nome para **`tkinter.OptionMenu`** e **`tkinter.ttk.OptionMenu`**.

- gh-77065 [https://github.com/python/cpython/issues/77065]: Adiciona o argumento somente-nomeado opcional *echo_char* para `getpass.getpass()` para suporte opcional ao feedback visual do teclado. Patch por Semyon Moroz.

- gh-130317 [https://github.com/python/cpython/issues/130317]: Corrige `PyFloat_Pack2()` e `PyFloat_Unpack2()` para NaNs com payload. Isso corrige a idea e volta para `struct.unpack()` e `struct.pack()` no caso do tipo IEEE 754 binary16 de “meia precisão”. Patch por Sergey B Kirpichev.

- gh-130402 [https://github.com/python/cpython/issues/130402]: A união de threads de daemon em execução durante o desligamento do interpretador agora levanta `PythonFinalizationError`.

- gh-130167 [https://github.com/python/cpython/issues/130167]: Melhoria na velocidade de `difflib.IS_LINE_JUNK()`. Patch por Semyon Moroz.

- gh-101410 [https://github.com/python/cpython/issues/101410]: Adicionadas mensagens mais detalhadas para erros de domínio no módulo `math`.

- gh-128384 [https://github.com/python/cpython/issues/128384]: Make `warnings.catch_warnings` use a context variable for holding the warning filtering state if the `sys.flags.context_aware_warnings` flag is set to true. This makes using the context manager thread-safe in

multi-threaded programs. The flag is true by default in free-threaded builds and is otherwise false. The value of the flag can be overridden by the `-X context_aware_warnings` command-line option or by the `PYTHON_CONTEXT_AWARE_WARNINGS` environment variable.

- gh-129719 [https://github.com/python/cpython/issues/129719]: Corrige a constante **`socket.CAN_RAW_ERR_FILTER`** ausente no módulo socket em sistemas Linux. Ela estava ausente desde o Python 3.11.

- gh-129027 [https://github.com/python/cpython/issues/129027]: Levanta `DeprecationWarning` para `sys._clear_type_cache()`. Esta função foi descontinuada no Python 3.13, mas não levanta um aviso em tempo de execução.

- gh-128307 [https://github.com/python/cpython/issues/128307]: Adiciona o argumento nomeado `eager_start` a `asyncio.loop.create_task()`

- gh-127604 [https://github.com/python/cpython/issues/127604]: Adiciona suporte para exibir o stack trace (situação da pilha de execução) C em sistemas que oferece suporte a ele via `faulthandler.dump_c_stack()` ou via argumento *c_stack* em `faulthandler.enable()`.

- gh-127385 [https://github.com/python/cpython/issues/127385]: Adiciona a constante `F_DUPFD_QUERY` ao módulo `fcntl`.

- gh-126838 [https://github.com/python/cpython/issues/126838]: Adiciona o parâmetro somente-nomeado *resolve_host* a `urllib.request.url2pathname()` e corrige o tratamento de URLs de arquivo com autoridades.

- gh-82129 [https://github.com/python/cpython/issues/82129]: Corrige `NameError` ao chamar `typing.get_type_hints()` em uma `dataclasses.dataclass()` criado por `dataclasses.make_dataclass()` com campos não anotados.

- gh-122559 [https://github.com/python/cpython/issues/122559]: Remove os métodos **`__reduce__()`** e **`__reduce_ex__()`** que sempre levantam `TypeError` na implementação em C de `io.FileIO`, `io.BufferedReader`,

`io.BufferedWriter` e `io.BufferedRandom` e substitua-os pelos métodos padrão `__getstate__()` que levantam `TypeError`. Isso restaura detalhes precisos do comportamento do Python 3.11 e versões anteriores.

- gh-122179 [https://github.com/python/cpython/issues/122179]: `hashlib.file_digest()` agora levanta `BlockingIOError` quando não há dados disponíveis durante E/S não bloqueantes. Antes, ele adicionava bytes nulos espúrios ao resumo.

- gh-53032 [https://github.com/python/cpython/issues/53032]: Expõe `decimal.IEEEContext()` para oferecer suporte à criação de contextos correspondentes aos formatos de intercâmbio decimal do IEEE 754 (2008). Patch por Sergey B Kirpichev.

- gh-120220 [https://github.com/python/cpython/issues/120220]: Descontinua os métodos `trace_variable()`, `trace_vdelete()` e `trace_vinfo()` de `tkinter.Variable`. Os métodos `trace_add()`, `trace_remove()` e `trace_info()` podem ser usados em seu lugar.

- gh-113539 [https://github.com/python/cpython/issues/113539]: `webbrowser`: Os nomes na variável de ambiente `BROWSER` agora podem se referir a navegadores web já registrados, em vez de sempre gerar um novo comando do navegador.

  Isso torna possível definir `BROWSER` como o valor de um dos navegadores suportados no macOS.

- bpo-44172 [https://bugs.python.org/issue?@action=redirect&bpo=44172]: Mantém uma referência às janelas originais `curses` nas subjanelas para que a janela original não seja excluída antes das subjanelas.

- gh-75223 [https://github.com/python/cpython/issues/75223]: Descontinua extensões sem pontos em `mimetypes.MimeTypes.add_type()`. Patch por Hugo van Kemenade.

IDLE

- gh-112936 [https://github.com/python/cpython/issues/112936]: corrige IDLE: nenhum item de menu Shell no modo de processo único.

Documentação

- gh-107006 [https://github.com/python/cpython/issues/107006]: Move a documentação e o código de exemplo para `threading.local` da docstring para a documentação oficial.
- gh-125142 [https://github.com/python/cpython/issues/125142]: Como parte do texto de introdução da ajuda embutida, mostre os atalhos de teclado para o novo REPL não básico (F1, F2 e F3).

Núcleo e embutidos

- gh-133336 [https://github.com/python/cpython/issues/133336]: `-J` não está mais reservado para uso do Jython. Patch por Adam Turner.
- gh-133261 [https://github.com/python/cpython/issues/133261]: Corrige um bug em que o ciclo GC não conseguia rastrear objetos na lixeira porque eles pareciam imortais. Quando objetos são adicionados à lixeira, tomamos cuidado para garantir que eles mantenham uma contagem de referências mortal.
- gh-133346 [https://github.com/python/cpython/issues/133346]: Adicionado suporte experimental para temas de cores ao módulo `_colorize`.
- gh-132917 [https://github.com/python/cpython/issues/132917]: Para a construção com threads livres, verifica o aumento no uso de memória do processo antes de acionar uma coleta de lixo automática completa. Se a memória usada não tiver aumentado 10% desde a última coleta, adia-a.
- gh-91048 [https://github.com/python/cpython/issues/91048]: Adiciona uma nova interface de linha de comando `python -m asyncio ps PID` para inspecionar tarefas assíncronas em um processo Python em execução. Exibe uma tabela simples de relacionamentos de espera. Uma variante que mostra uma visualização em árvore também está disponível como `python -m asyncio pstree PID`. Ambas são úteis para depuração de código assíncrono. Patch por Pablo Galindo, Łukasz Langa, Yury Selivanov e Marta Gomez Macias.
- gh-133304 [https://github.com/python/cpython/issues/133304]: Solução alternativa para a “canonização” do NaN em `PyFloat_Pack4()` e

`PyFloat_Unpack4()` no RISC-V.

- gh-133197 [https://github.com/python/cpython/issues/133197]: Melhora as mensagens de erro `SyntaxError` para prefixos de strings / bytes incompatíveis.
- gh-133231 [https://github.com/python/cpython/issues/133231]: Adiciona novos utilitários de observação da compilação JIT: `sys._jit.is_available()`, `sys._jit.is_enabled()` e `sys._jit.is_active()`.

Biblioteca

- gh-133194 [https://github.com/python/cpython/issues/133194]: `ast.parse()` não analisará mais a nova sintaxe **PEP 758** [https://peps.python.org/pep-0758/] com a *feature_version* mais antiga passada.

Núcleo e embutidos

- gh-131798 [https://github.com/python/cpython/issues/131798]: Divide `CALL_LEN` em vários UOPs, permitindo que o JIT os remova durante a otimização. Patch por Diego Russo.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Usa `sym_new_type` em vez de `sym_new_not_null` para _BUILD_STRING, _BUILD_SET
- gh-132942 [https://github.com/python/cpython/issues/132942]: Corrige duas corridas no cache de pesquisa de tipos. Isso afetava a construção com threads livres e podia causar travamentos (aparentemente bastante difíceis de acionar).
- gh-131798 [https://github.com/python/cpython/issues/131798]: Propaga o tipo de retorno de `_BINARY_OP_SUBSCR_TUPLE_INT` em JIT. Patch por Tomas Roun
- gh-132952 [https://github.com/python/cpython/issues/132952]: Acelera a inicialização com o argumento `-S` importando o módulo privado `_io` em vez de `io`. Isso corrige uma regressão de desempenho introduzida anteriormente no desenvolvimento do Python 3.14 e restaura o desempenho ao nível do Python 3.13.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Permite que o JIT remova as proteções int após `_CALL_LEN` definindo o tipo de retorno como int. Patch por Diego Russo

- gh-131798 [https://github.com/python/cpython/issues/131798]: Divide `CALL_TUPLE_1` em vários UOPs, permitindo que o JIT remova alguns deles. Patch por Tomas Roun
- gh-131798 [https://github.com/python/cpython/issues/131798]: Divide `CALL_STR_1` em vários UOPs, permitindo que o JIT remova alguns deles. Patch por Tomas Roun
- gh-132825 [https://github.com/python/cpython/issues/132825]: Melhorias nas mensagens de erro de chaves/elementos não fragmentáveis para **`dict`** e **`set`**. Patch por Victor Stinner.
- gh-131591 [https://github.com/python/cpython/issues/131591]: Redefine qualquer chamada pendente de depuração remota **PEP 768** [https://peps.python.org/pep-0768/] em filhos após chamadas **`os.fork()`**.
- gh-132713 [https://github.com/python/cpython/issues/132713]: Corrige a condição de corrida `repr(list)`: mantém uma referência forte ao item ao chamar `repr(item)`. Patch por Victor Stinner.
- gh-132661 [https://github.com/python/cpython/issues/132661]: Implementa **PEP 750** [https://peps.python.org/pep-0750/] (Template Strings). Adiciona nova sintaxe para t-strings e implementa novos tipos internos **`string.templatelib.Template`** e **`string.templatelib.Interpolation`**.
- gh-132479 [https://github.com/python/cpython/issues/132479]: Corrige travamento do compilador em determinadas circunstâncias em que várias anotações em nível de módulo incluem compreensões e outros escopos aninhados.
- gh-132747 [https://github.com/python/cpython/issues/132747]: Corrige uma falha ao chamar **`__get__()`** de um method com um segundo argumento **`None`**.
- gh-132744 [https://github.com/python/cpython/issues/132744]: Certas chamadas agora verificam a recursão descontrolada e respeitam o limite de recursão do sistema.
- gh-132449 [https://github.com/python/cpython/issues/132449]: Erros de sintaxe que parecem erros de ortografia de palavra reservada do Python agora oferecem uma sugestão útil de correção. Contribuição de Pablo Galindo Salgado.

Biblioteca

- gh-132737 [https://github.com/python/cpython/issues/132737]: Suporte a código de criação de perfil que requer `__main__`, como `pickle`.

Núcleo e embutidos

- gh-132639 [https://github.com/python/cpython/issues/132639]: Adicionadas `PyLong_AsNativeBytes()`, `PyLong_FromNativeBytes()` e `PyLong_FromUnsignedNativeBytes()` à API C limitada.
- gh-100239 [https://github.com/python/cpython/issues/100239]: Adiciona especialização para `BINARY_OP/SUBSCR` na lista e no fatiamento.
- gh-132508 [https://github.com/python/cpython/issues/132508]: Utiliza inteiros marcados na pilha de avaliação para representar os deslocamentos de instrução ao levantar novamente uma exceção. Isso evita a necessidade de encapsular o inteiro, o que poderia falhar em condições de pouca memória.
- gh-124476 [https://github.com/python/cpython/issues/124476]: Corrige decodificação da codificação local no local C.UTF-8.
- gh-131927 [https://github.com/python/cpython/issues/131927]: Avisos do compilador originados do mesmo módulo e número de linha agora são emitidos apenas uma vez, correspondendo ao comportamento dos avisos emitidos pelo código do usuário. Isso também pode ser configurado com filtros `warnings`.
- gh-132457 [https://github.com/python/cpython/issues/132457]: Torna `staticmethod()` e `classmethod()` genéricos.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Usa `sym_new_type` em vez de `sym_new_not_null` para _BUILD_LIST, _BUILD_SET, _BUILD_MAP
- gh-131798 [https://github.com/python/cpython/issues/131798]: Divide `CALL_TYPE_1` em vários uops, permitindo que o JIT remova alguns deles.
- gh-132386 [https://github.com/python/cpython/issues/132386]: Corrige falha ao passar uma subclasse dict como parâmetro `globals` para `exec()`.
- gh-127682 [https://github.com/python/cpython/issues/127682]: Não é mais necessário chamar `__iter__` duas vezes ao criar e executar uma expressão geradora. A criação de uma expressão geradora a partir de uma expressão não iterável só será levantada quando a expressão

geradora for executada. Isso alinha o comportamento das expressões geradoras com outros geradores.

- gh-132261 [https://github.com/python/cpython/issues/132261]: O armazenamento interno para anotações e funções de anotação em classes agora usa chaves diferentes no dicionário de classes. Isso elimina vários casos extremos em que o acesso aos atributos `__annotate__` e `__annotations__` se comportaria de forma imprevisível.
- gh-132284 [https://github.com/python/cpython/issues/132284]: Não envolve slots base `PyCFunction` na criação da classe se não forem substituídos.
- gh-130415 [https://github.com/python/cpython/issues/130415]: Melhora a capacidade do JIT de remover cargas constantes e variáveis locais não utilizadas e corrige um problema em que a desalocação de valores não utilizados poderia fazer com que o código JIT travasse ou se comportasse incorretamente.
- gh-126703 [https://github.com/python/cpython/issues/126703]: Corrige possível uso após free em casos em que a definição de um método tem o mesmo tempo de vida que seu `self`.
- gh-132286 [https://github.com/python/cpython/issues/132286]: Corrige que **`type.__annotate__`** não foi excluído quando **`type.__annotations__`** foi excluído.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Permite que o JIT remova uma instrução extra `_TO_BOOL_BOOL` após `_CONTAINS_OP_DICT` definindo o tipo de retorno como bool.
- gh-124715 [https://github.com/python/cpython/issues/124715]: Evita estouros de pilha ao chamar **`Py_DECREF()`**. Objetos de extensão de terceiros não precisam mais usar o mecanismo de “lixeira”, pois a proteção agora está incorporada à macro **`Py_DECREF()`**.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Permite que o compilador JIT remova algumas verificações de tipo para operações em listas, tuplas, dicionários e conjuntos.
- gh-128398 [https://github.com/python/cpython/issues/128398]: Melhoria na mensagem de erro quando um objeto que ofereça suporte ao protocolo de gerenciador de contexto síncrono (ou assíncrono) é inserido usando **`async with`** (ou seja, **`with`**) em vez de **`with`** (ou seja, **`async with`**). Patch por Bénédikt Tran.
- gh-131798 [https://github.com/python/cpython/issues/131798]: Permite que o JIT remova as proteções Unicode após `_BINARY_OP_SUBSCR_STR_INT`

definindo o tipo de retorno como string.

- gh-131878 [https://github.com/python/cpython/issues/131878]: Trata exceções não capturadas no laço de entrada principal para o novo REPL.
- gh-131878 [https://github.com/python/cpython/issues/131878]: Corrige suporte a caracteres Unicode com dois ou mais pontos de código no Windows no novo REPL.
- gh-126835 [https://github.com/python/cpython/issues/126835]: Move a dobragem de constante para o otimizador de olho mágico. Renomeia os arquivos relacionados à otimização AST (Python/ast_opt.c -> Python/ast_preprocess.c), structs (_PyASTOptimizeState -> _PyASTPreprocessState) e funções (_PyAST_Optimize -> _PyAST_Preprocess, _PyCompile_AstOptimize -> _PyCompile_AstPreprocess).
- gh-114809 [https://github.com/python/cpython/issues/114809]: Adiciona suporte para construções multiarquitetura do macOS com o JIT habilitado
- gh-131507 [https://github.com/python/cpython/issues/131507]: O PyREPL agora oferece suporte a realce de sintaxe. Contribuição de Łukasz Langa.
- gh-130907 [https://github.com/python/cpython/issues/130907]: Se as `__annotations__` de um objeto de módulo forem acessadas enquanto o módulo estiver em execução, retorne as anotações que foram definidas até agora, sem armazená-las em cache.
- gh-130104 [https://github.com/python/cpython/issues/130104]: **`pow()`** de três argumentos agora tenta chamar **`__rpow__()`** se necessário. Anteriormente, ele só era chamado em **`pow()`** de dois argumentos e no operador binário de potência.
- gh-130070 [https://github.com/python/cpython/issues/130070]: Corrigido um erro de asserção para **`exec()`** passando uma string `source` e uma `closure` não `None`. Patch por Bartosz Sławecki.
- gh-129958 [https://github.com/python/cpython/issues/129958]: Corrige um bug que permitia quebras de linha inconsistentes em especificadores de formato para f-strings entre aspas simples. Patch por Pablo Galindo.
- gh-129858 [https://github.com/python/cpython/issues/129858]: instruções `elif` que seguem um bloco `else` agora têm uma mensagem de erro específica.
- gh-69605 [https://github.com/python/cpython/issues/69605]: Adiciona o módulo autocomplete ao PyREPL.
- gh-128555 [https://github.com/python/cpython/issues/128555]: Adiciona o sinalizador **`sys.flags.thread_inherit_context`**.

    - Este sinalizador é definido como verdadeiro por padrão na construção com threads livres e como falso caso contrário. Se o sinalizador for verdadeiro, iniciar uma nova thread usando `threading.Thread` usará, por padrão, uma cópia de `contextvars.Context` do chamador de `threading.Thread.start()` em vez de usar um contexto vazio.
    - Adiciona a opção de linha de comando `-X thread_inherit_context` e a variável de ambiente `PYTHON_THREAD_INHERIT_CONTEXT`, que definem o sinalizador `thread_inherit_context`.
    - Adiciona o parâmetro nomeado `context` a `Thread`. Ele pode ser usado para passar explicitamente um valor de contexto a ser usado por uma nova thread.
    - Torna o módulo `_contextvars` embutido.
- gh-123539 [https://github.com/python/cpython/issues/123539]: Melhora a mensagem `SyntaxError` para usar `import ... as` e `from ... import ... as` sem um nome.
- gh-102567 [https://github.com/python/cpython/issues/102567]: `-X importtime` agora aceita o valor `2`, que indica que uma entrada `importtime` também deve ser impressa se um módulo importado já tiver sido carregado. Patch por Noah Kim e Adam Turner.
- gh-116436 [https://github.com/python/cpython/issues/116436]: Melhora a mensagem de erro quando `TypeError` ocorre durante `dict.update()`
- gh-103997 [https://github.com/python/cpython/issues/103997]: String arguments passed to “-c” are now automatically dedented. This allows “python -c” invocations to be indented in shell scripts without causing indentation errors. (Patch by Jon Crall and Steven Sun)

Biblioteca

- gh-89562 [https://github.com/python/cpython/issues/89562]: Remove o membro `hostflags` da struct `PySSLContext`.

API C

- gh-133166 [https://github.com/python/cpython/issues/133166]: Corrige regressão onde `PyType_GetModuleByDef()` retorna NULL sem definir

`TypeError` quando um tipo estático é passado.

- gh-133164 [https://github.com/python/cpython/issues/133164]: Adicionada a função `PyUnstable_Object_IsUniqueReferencedTemporary()` para determinar se um objeto existe como uma variável temporária única na pilha do interpretador. Esta função substitui alguns casos em que verificar se `Py_REFCNT()` é uma variável temporária não é mais suficiente para determinar se é seguro modificar um objeto Python no local sem efeitos colaterais visíveis.
- gh-133140 [https://github.com/python/cpython/issues/133140]: Adiciona `PyUnstable_Object_IsUniquelyReferenced()` como um substituto para `Py_REFNCT(op) == 1` em construções com threads livres do Python.
- gh-131747 [https://github.com/python/cpython/issues/131747]: Em plataformas não Windows, descontinua o uso de `ctypes.Structure._pack_` para usar um layout compatível com o Windows. O layout deve ser especificado explicitamente definindo `ctypes.Structure._layout_` como `'ms'`.
- gh-128972 [https://github.com/python/cpython/issues/128972]: Para construções sem threads livres, o layout de memória de `PyASCIIObject` é revertido para corresponder ao Python 3.13. (Observe que a estrutura não faz parte da ABI estável e, portanto, seu layout de memória tem *garantia* de estabilidade.)
- gh-133079 [https://github.com/python/cpython/issues/133079]: As APIs não documentadas **`Py_C_RECURSION_LIMIT`** e **`PyThreadState.c_recursion_remaining`**, adicionadas na versão 3.13, foram removidas sem um período de descontinuação.
- gh-132987 [https://github.com/python/cpython/issues/132987]: Os formatos `k` e `K` em `PyArg_Parse()` agora oferecem suporte ao método especial `__index__()`, como todos os outros formatos inteiros.
- gh-132909 [https://github.com/python/cpython/issues/132909]: Corrige um estouro ao manipular o formato K em `Py_BuildValue()`. Patch por Bénédikt Tran.
- gh-132798 [https://github.com/python/cpython/issues/132798]: As funções descontinuadas e não documentadas **`PyUnicode_AsEncodedObject()`**, **`PyUnicode_AsDecodedObject()`**, **`PyUnicode_AsEncodedUnicode()`** e **`PyUnicode_AsDecodedUnicode()`** estão programadas para remoção na versão 3.15.

- gh-132470 [https://github.com/python/cpython/issues/132470]: Cria um **`ctypes.CField`** com um *byte_size* que não corresponde ao tamanho do tipo real agora levanta um **`ValueError`** em vez de travar o interpretador.
- gh-112068 [https://github.com/python/cpython/issues/112068]: [Revertido em gh-136991 [https://github.com/python/cpython/issues/136991]] Adicionado suporte a argumentos anuláveis em **`PyArg_Parse()`** e funções similares. Adicionar `?` após qualquer unidade de formato faz com que `None` seja aceito como valor.
- gh-50333 [https://github.com/python/cpython/issues/50333]: Sequências não-tuplas estão descontinuadas como argumento para a unidade de formato `(items)` em **`PyArg_ParseTuple()`** e outras funções de análise de argumentos se *items* contiver unidades de formato que armazenam um buffer emprestado ou uma referência emprestada.

Construção

- gh-113464 [https://github.com/python/cpython/issues/113464]: Use o repositório “externals” cpython-bin-deps para o gerenciamento de dependências do Windows LLVM. A instalação manual do LLVM não é mais necessária para construções JIT do Windows.
- gh-133183 [https://github.com/python/cpython/issues/133183]: Os shims do compilador iOS agora incluem `IPHONEOS_DEPLOYMENT_TARGET` em triplos de destino, garantindo que os mínimos da versão do SDK sejam respeitados.
- gh-133167 [https://github.com/python/cpython/issues/133167]: Corrige o processo de compilação com `--enable-optimizations` e `--without-docstrings`.
- gh-133171 [https://github.com/python/cpython/issues/133171]: Como as construções com threads livres não oferecem suporte ao compilador JIT experimental, evita que essas configurações sejam combinadas.
- gh-132758 [https://github.com/python/cpython/issues/132758]: Corrige construção com interpretador de chamada de cauda e pystats.
- gh-132649 [https://github.com/python/cpython/issues/132649]: O script `PClayout` agora permite passar `--include-tcltk` no Windows ARM64.
- gh-132257 [https://github.com/python/cpython/issues/132257]: Altera os sinalizadores LTO padrão no GCC para não passar `-flto-`

`partition=none` e permitir a paralelização de LTO. Para versões GNU mais recentes e GCC, isso representa um aumento de velocidade múltiplo nos tempos de construção com LTO, sem perda perceptível de desempenho.
- gh-132026 [https://github.com/python/cpython/issues/132026]: Corrige o uso de identificadores indefinidos na detecção de tripletos de plataforma em plataformas MIPS Linux.

Python 3.14.0 alpha 7

Data de lançamento: 2025-04-08

macOS

- gh-124111 [https://github.com/python/cpython/issues/124111]: Atualiza o instalador do macOS para usar o Tcl/Tk 8.6.16.
- gh-131423 [https://github.com/python/cpython/issues/131423]: Atualiza o instalador do macOS para usar o OpenSSL 3.0.16. Patch por Bénédikt Tran.
- gh-131025 [https://github.com/python/cpython/issues/131025]: Atualiza o instalador do macOS para ser fornecido com o SQLite 3.49.1.

Windows

- gh-131423 [https://github.com/python/cpython/issues/131423]: Atualiza a versão empacotada do OpenSSL para 3.0.16. A nova construção também desabilita o suporte a uplink, o que pode ser relevante para incorporadores, mas não tem impacto no uso normal.
- gh-131453 [https://github.com/python/cpython/issues/131453]: Algumas constantes **`SND_*`** e **`MB_*`** foram adicionadas ao **`winsound`**.
- gh-91349 [https://github.com/python/cpython/issues/91349]: Substitui nossa cópia de `zlib` por `zlib-ng`, para melhorias de desempenho em **`zlib`**.
- gh-131025 [https://github.com/python/cpython/issues/131025]: Atualiza o instalador do Windows para ser fornecido com o SQLite 3.49.1.

Ferramentas/Demos

- gh-132121 [https://github.com/python/cpython/issues/132121]: Sempre escapa caracteres Unicode não imprimíveis em **pygettext**.
- gh-131852 [https://github.com/python/cpython/issues/131852]: **msgfmt** não adiciona mais `POT-Creation-Date` aos arquivos `.mo` gerados para consistência com GNU `msgfmt`.

Testes

- gh-131277 [https://github.com/python/cpython/issues/131277]: Permite remover a configuração de uma ou mais variáveis de ambiente simultaneamente via **`EnvironmentVarGuard.unset()`**. Patch por Bénédikt Tran.
- gh-131050 [https://github.com/python/cpython/issues/131050]: `test_ssl.test_dh_params` é ignorado se a biblioteca TLS subjacente não oferecer suporte a Diffie-Hellman efêmero de campo finito.

Segurança

- gh-131809 [https://github.com/python/cpython/issues/131809]: Atualiza libexpat empacotado para 2.7.1
- gh-131261 [https://github.com/python/cpython/issues/131261]: Atualiza para libexpat 2.7.0
- gh-121284 [https://github.com/python/cpython/issues/121284]: Corrige bug na dobragem (*folding*) de palavras codificadas do rfc2047 ao achatar uma mensagem de e-mail usando uma política de e-mail moderna. Anteriormente, quando uma palavra codificada era muito longa para uma linha, ela era decodificada, dividida em linhas e recodificada. Mas vírgulas e outros caracteres especiais no texto original podiam ser deixados sem codificação e sem aspas. Isso poderia teoricamente ser usado para falsificar linhas de cabeçalho usando uma palavra codificada cuidadosamente construída se o e-mail renderizado resultante fosse transmitido ou reanalisado.

Biblioteca

- gh-132174 [https://github.com/python/cpython/issues/132174]: Corrige o nome da função na mensagem de erro `_interpreters.run_string`.

- gh-132171 [https://github.com/python/cpython/issues/132171]: Corrige falha de `_interpreters.run_string` em subclasses de string.
- gh-129204 [https://github.com/python/cpython/issues/129204]: Introduz o novo botão de variável de ambiente `_PYTHON_SUBPROCESS_USE_POSIX_SPAWN` em **`subprocess`** para controlar o uso de **`os.posix_spawn()`**.
- gh-132159 [https://github.com/python/cpython/issues/132159]: Não oculta argumentos de usuário em **`__new__()`** gerado pelo decorador **`warnings.deprecated`**. Patch por Xuehai Pan.
- gh-132168 [https://github.com/python/cpython/issues/132168]: O tipo **`ctypes.py_object`** agora oferece suporte a subscrição, tornando-o um tipo genérico.
- gh-84481 [https://github.com/python/cpython/issues/84481]: Adiciona o atributo **`zipfile.ZipFile.data_offset`**, que armazena o deslocamento para o início dos dados ZIP em um arquivo, quando disponível. Quando o atributo **`zipfile.ZipFile`** for aberto no modo `'w'` ou `'x'` e o arquivo subjacente não oferecer suporte a `tell()`, o valor será `None`.
- gh-132075 [https://github.com/python/cpython/issues/132075]: Corrige possível uso de estruturas de endereço **`socket`** com membros não inicializados. Agora, todos os membros da estrutura são inicializados com zeros por padrão.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação em até 27x para o módulo **`string`**. Patch por Adam Turner.
- gh-125434 [https://github.com/python/cpython/issues/125434]: Exibe o nome da thread em **`faulthandler`**. Patch por Victor Stinner.
- gh-132002 [https://github.com/python/cpython/issues/132002]: Corrige falha ao desalocar **`contextvars.ContextVar`** com nomes de strings estranhos e não hasheáveis.
- gh-131938 [https://github.com/python/cpython/issues/131938]: **`xml.etree.ElementTree`**: Atualiza a mensagem de erro quando um elemento a ser removido via **`Element.remove`** não é encontrado. Patch por Bénédikt Tran.
- gh-115942 [https://github.com/python/cpython/issues/115942]: Adiciona **`threading.RLock.locked()`**, **`multiprocessing.Lock.locked()`**, **`multiprocessing.RLock.locked()`** e permite **`multiprocessing.managers.SyncManager.Lock()`** e

`multiprocessing.managers.SyncManager.RLock()` intermediar chamada de `locked()`.

- gh-131974 [https://github.com/python/cpython/issues/131974]: Corrige vários problemas de segurança de thread em `ctypes` na construção com threads livres.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação do módulo `ast` extraindo a função `unparse()` para um módulo auxiliar.
- gh-107369 [https://github.com/python/cpython/issues/107369]: Melhorado o desempenho de `textwrap.indent()` em uma média de ~1,3x. Patch por Adam Turner.
- gh-131792 [https://github.com/python/cpython/issues/131792]: Melhoramos o desempenho de `textwrap.dedent()` em uma média de ~2,4x (com melhorias de até 4x para entradas grandes) e corrigimos um bug em que linhas em branco com caracteres de espaço em branco diferentes de espaço ou de tabulação horizontal não eram normalizadas para quebra de linha. Patch por Adam Turner, Marius Juston e Pieter Eendebak.
- gh-131668 [https://github.com/python/cpython/issues/131668]: `socket`: Corrige código que analisa endereços de soquete AF_BLUETOOTH.
- gh-60115 [https://github.com/python/cpython/issues/60115]: suporte a módulos congelados para `linecache.getline()`.
- gh-131492 [https://github.com/python/cpython/issues/131492]: Corrige um vazamento de recursos quando a construção de um `gzip.GzipFile` com um nome de arquivo falha, por exemplo, ao passar um `compresslevel` inválido.
- gh-131435 [https://github.com/python/cpython/issues/131435]: Melhoria de desempenho de 10-20% de `random.randint()`.
- gh-131461 [https://github.com/python/cpython/issues/131461]: Corrige `ResourceWarning` ao construir um `gzip.GzipFile` no modo de gravação com um objeto de arquivo quebrado.
- gh-125866 [https://github.com/python/cpython/issues/125866]: Descontinua o módulo **`nturl2path`**. Em vez disso, chame `urllib.request.url2pathname()` e `pathname2url()`.
- gh-126367 [https://github.com/python/cpython/issues/126367]: Corrige problema em que `urllib.request.url2pathname()` levantava `OSError` ao receber um URI do Windows contendo um caractere de dois pontos

que não seguia uma letra de unidade, como antes de um fluxo de dados alternativo NTFS.

- gh-120144 [https://github.com/python/cpython/issues/120144]: Desabilita o evento `CALL` em **`bdb`** no backend `monitoring` quando não precisarmos de novos eventos no objeto código para obter um melhor desempenho.
- gh-131358 [https://github.com/python/cpython/issues/131358]: Registra `cseuckr` como um apelido de codificação para `euc_kr`.
- gh-131325 [https://github.com/python/cpython/issues/131325]: Corrige implementação de fallback do sendfile para drenar dados após gravar no transporte em **`asyncio`**.
- gh-90548 [https://github.com/python/cpython/issues/90548]: **`platform.libc_ver()`** agora pode detectar e relatar a versão de `musl` no Alpine Linux.
- gh-129843 [https://github.com/python/cpython/issues/129843]: Corrige passagem de argumento incorreta em **`warnings.warn_explicit()`**.
- gh-70647 [https://github.com/python/cpython/issues/70647]: Ao criar um objeto **`datetime`** com uma data fora do intervalo, um erro mais informativo é levantado.
- gh-130914 [https://github.com/python/cpython/issues/130914]: Permite que **`graphlib.TopologicalSorter.prepare()`** seja chamado mais de uma vez, desde que a classificação não tenha sido iniciada. Patch por Daniel Pope.
- gh-131236 [https://github.com/python/cpython/issues/131236]: Permite gerar vários UUIDs de uma só vez via **`python -m uuid --count`**.
- gh-126895 [https://github.com/python/cpython/issues/126895]: Corrige **`readline`** em construção com threads livres.
- gh-121468 [https://github.com/python/cpython/issues/121468]: `$_asynctask` é adicionado como uma variável de conveniência **`pdb`** para acessar a tarefa asyncio atual, se aplicável.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhoria no tempo de importação de **`locale`** usando a importação preguiçosa `re`. Patch por Semyon Moroz.
- gh-129598 [https://github.com/python/cpython/issues/129598]: Corrige **`ast.unparse()`** quando **`ast.Interactive`** contém múltiplas instruções.
- gh-85162 [https://github.com/python/cpython/issues/85162]: O módulo **`http.server`** agora inclui suporte embutido para servidores HTTPS

expostos por **`http.server.HTTPSServer`**. Essa funcionalidade é exposta pela interface de linha de comando (`python -m http.server`) por meio das opções `--tls-cert`, `--tls-key` e `--tls-password-file`. Patch por Semyon Moroz.

- gh-129463 [https://github.com/python/cpython/issues/129463]: As implementações de igualdade e hash para **`annotationlib.ForwardRef`** agora usam todos os atributos do objeto. Dois objetos **`ForwardRef`** são iguais somente se todos os atributos forem iguais.
- gh-128593 [https://github.com/python/cpython/issues/128593]: Objetos **`annotationlib.ForwardRef`** não armazenam mais seus valores em cache quando são avaliados com sucesso. Chamadas sucessivas para **`annotationlib.ForwardRef.evaluate()`** podem retornar valores diferentes.
- gh-117779 [https://github.com/python/cpython/issues/117779]: Corrigida a leitura de entradas duplicadas em **`zipfile`** por nome. A leitura de entradas duplicadas (exceto a última) por `ZipInfo` agora emite um aviso em vez de levantar uma exceção.
- gh-128715 [https://github.com/python/cpython/issues/128715]: A classe de descritores de campo **`Structure`**/**`Union`** agora está disponível como **`CField`** e tem novos atributos para auxiliar na depuração e introspecção.
- gh-128055 [https://github.com/python/cpython/issues/128055]: Corrige `test.test_sysconfig.test_sysconfigdata_json` ao executar fora do diretório de construção (por exemplo, após a instalação).
- gh-126037 [https://github.com/python/cpython/issues/126037]: **`xml.etree.ElementTree`**: Corrige uma falha em **`Element.find`**, **`Element.findtext`** e **`Element.findall`** quando a tag a ser encontrada implementa um método **`__eq__()`** que altera o elemento que está sendo consultado. Patch por Bénédikt Tran.
- gh-127794 [https://github.com/python/cpython/issues/127794]: Quando cabeçalhos são adicionados aos objetos **`email.message.Message`**, por meio de **`email.message.Message.__setitem__()`** ou **`email.message.Message.add_header()`**, o nome do campo agora é validado de acordo com **RFC 5322, Seção 2.2** [https://datatracker.ietf.org/doc/html/rfc5322.html#section-2.2] e uma **`ValueError`** é levantada se o nome do campo contiver caracteres inválidos.

- gh-123599 [https://github.com/python/cpython/issues/123599]: Descontinua **`pathlib.PurePath.as_uri()`**; em vez disso, use **`pathlib.Path.as_uri()`**.
- gh-126033 [https://github.com/python/cpython/issues/126033]: **`xml.etree.ElementTree`**: Corrige uma falha em **`Element.remove`** quando o elemento é mutado simultaneamente. Patch por Bénédikt Tran.
- gh-120144 [https://github.com/python/cpython/issues/120144]: Adiciona o backend opcional de `sys.monitoring` a **`bdb`** e usa-o para **`pdb`**.
- gh-74598 [https://github.com/python/cpython/issues/74598]: Adiciona **`fnmatch.filterfalse()`** para excluir nomes que correspondam a um padrão. Patch por Bénédikt Tran.
- gh-114917 [https://github.com/python/cpython/issues/114917]: Adiciona suporte para AI_NUMERICSERV na emulação getaddrinfo
- bpo-17254 [https://bugs.python.org/issue?@action=redirect&bpo=17254]: Adicionados apelidos para o idioma tailandês usando o Microsoft Code Pages.

Documentação

- gh-131417 [https://github.com/python/cpython/issues/131417]: Menciona **`asyncio.Future`** e **`asyncio.Task`** na lista de classes genéricas.

Núcleo e embutidos

- gh-131798 [https://github.com/python/cpython/issues/131798]: Permite que o JIT remova uma instrução extra `_TO_BOOL_BOOL` após `_CONTAINS_OP_SET` definindo o tipo de retorno como bool.
- gh-132011 [https://github.com/python/cpython/issues/132011]: Corrige falha ao chamar **`list.append()`** como um método não vinculado.
- gh-131998 [https://github.com/python/cpython/issues/131998]: Corrige uma falha ao usar um objeto de método não vinculado descritor em uma função onde um descritor de método vinculado foi usado.
- gh-131591 [https://github.com/python/cpython/issues/131591]: Implementa a **PEP 768** [https://peps.python.org/pep-0768/] (interface de depuração externa segura para CPython). Adiciona uma nova função **`sys.remote_exec()`** ao módulo **`sys`**. Esta função agenda a execução de um arquivo Python em

um processo separado. Patch por Pablo Galindo, Matt Wozniski e Ivona Stojanovic.

- gh-131798 [https://github.com/python/cpython/issues/131798]: Permite que o JIT omita a proteção str no teste de veracidade quando o tipo str for conhecido.
- gh-131833 [https://github.com/python/cpython/issues/131833]: Adiciona suporte para a remoção opcional de parênteses de agrupamento ao usar vários tipos de exceção, conforme a **PEP 758** [https://peps.python.org/pep-0758/]. Patch por Pablo Galindo.
- gh-130924 [https://github.com/python/cpython/issues/130924]: O uso de um nome em uma anotação de escopo de função não aciona mais a criação de uma célula para essa variável. Isso corrige uma regressão em versões alfa anteriores do Python 3.14.
- gh-131800 [https://github.com/python/cpython/issues/131800]: Melhora a capacidade do JIT experimental de remover verificações de tipo para determinadas operações de subscrição.
- gh-131738 [https://github.com/python/cpython/issues/131738]: O compilador emite código otimizado para chamadas any/all/tuple embutidas sobre uma expressão geradora.
- gh-131719 [https://github.com/python/cpython/issues/131719]: Corrige a falta de verificação de NULL em `_PyMem_FreeDelayed` na construção com threads livres.
- gh-131670 [https://github.com/python/cpython/issues/131670]: Corrige falha em **`anext()`** na sincronização **`__anext__()`** levantando uma exceção.
- gh-131666 [https://github.com/python/cpython/issues/131666]: Correção da assinatura dos objetos `anext_awaitable.close`. Patch por Bénédikt Tran.
- gh-130415 [https://github.com/python/cpython/issues/130415]: Otimiza a comparação de duas constantes em construções JIT
- gh-129149 [https://github.com/python/cpython/issues/129149]: Adiciona um atalho para inteiros pequenos e médios em **`PyLong_FromInt32()`**, **`PyLong_FromUInt32()`**, **`PyLong_FromInt64()`** e **`PyLong_FromUInt64()`**. Patch por Chris Eibl.
- gh-130887 [https://github.com/python/cpython/issues/130887]: Otimiza a geração de código AArch64 para o JIT. Patch por Diego Russo
- gh-130956 [https://github.com/python/cpython/issues/130956]: Otimiza a geração de código AArch64 para o JIT. Patch por Diego Russo

- gh-130928 [https://github.com/python/cpython/issues/130928]: Corrige mensagem de erro ao formatar bytes usando o sinalizador `'i'`. Patch por Maxim Ageev.
- gh-130935 [https://github.com/python/cpython/issues/130935]: Anotações em nível de classe e módulo definidas condicionalmente agora só são refletidas em `__annotations__` se o bloco em que se encontram for executado. Patch por Jelle Zijlstra.
- gh-130775 [https://github.com/python/cpython/issues/130775]: Não falha em `column` negativo e `end_column` em locais **`ast`**.
- gh-130704 [https://github.com/python/cpython/issues/130704]: Otimize `LOAD_FAST` e sua forma de superinstrução para reduzir a sobrecarga na contagem de referências. Essas instruções são substituídas por variantes mais rápidas que carregam referências emprestadas na pilha de operandos quando podemos provar que a referência no quadro sobrevive à referência carregada na pilha.
- gh-88887 [https://github.com/python/cpython/issues/88887]: Corrigindo vazamento do processo de multiprocessamento Resource Tracker, geralmente observado ao executar Python como PID 1.
- gh-130115 [https://github.com/python/cpython/issues/130115]: Corrige um problema com identificadores de thread sendo estendidos por sinal em algumas plataformas.

Biblioteca

- gh-99108 [https://github.com/python/cpython/issues/99108]: Adiciona suporte para implementação embutida de HMAC (**RFC 2104** [https://datatracker.ietf.org/doc/html/rfc2104.html]) com base em HACL*. Patch por Bénédikt Tran.

Núcleo e embutidos

- gh-130080 [https://github.com/python/cpython/issues/130080]: Implementa a PEP 765: Não permitir return/break/continue que saiam de um bloco finally.
- gh-129900 [https://github.com/python/cpython/issues/129900]: Corrige códigos de retorno dentro de **`SystemExit`** que não estão sendo retornados pelo REPL.

- gh-128632 [https://github.com/python/cpython/issues/128632]: Não mais permite `__classdict__` como nome de um parâmetro de tipo. Usar esse nome anteriormente travava o interpretador em algumas circunstâncias.
- gh-126703 [https://github.com/python/cpython/issues/126703]: Melhora o desempenho dos métodos embutidos usando uma freelist.
- gh-126703 [https://github.com/python/cpython/issues/126703]: Melhora o desempenho de `range` usando uma freelist.

API C

- gh-131740 [https://github.com/python/cpython/issues/131740]: Atualiza PyUnstable_GC_VisitObjects para percorrer a geração permanente.
- gh-131525 [https://github.com/python/cpython/issues/131525]: O PyTupleObject agora armazena em cache o valor de hash calculado no novo campo ob_hash.

Construção

- gh-131865 [https://github.com/python/cpython/issues/131865]: A construção do DTrace agora passa corretamente as variáveis `CC` e `CFLAGS` para o comando `dtrace` ao utilizar o SystemTap no Linux.
- gh-131675 [https://github.com/python/cpython/issues/131675]: Corrige construções da biblioteca mimalloc para alvos ARM de 32 bits.
- gh-131691 [https://github.com/python/cpython/issues/131691]: O clang-cl no Windows precisa da opção `/EHa` para oferecer suporte a SEH (gerenciamento de exceções estruturadas) corretamente. Correção por Chris Eibl.
- gh-131278 [https://github.com/python/cpython/issues/131278]: Adiciona o sinalizador de otimização `WITH_COMPUTED_GOTOS` às construções do Windows para quando usar um compilador compatível (atualmente clang-cl). Patch por Chris Eibl.
- gh-130213 [https://github.com/python/cpython/issues/130213]: Atualiza a biblioteca HACL* fornecida para corrigir problemas de construção com compiladores clang mais antigos.
- gh-130673 [https://github.com/python/cpython/issues/130673]: Corrige possível erro `KeyError` ao manipular seções de objetos durante o processo de construção JIT.

Python 3.14.0 alpha 6

Data de lançamento: 2025-03-14

macOS

- gh-128540 [https://github.com/python/cpython/issues/128540]: Garante que o navegador da web seja iniciado por `webbrowser.open()` no macOS, mesmo para URLs `file://`.

Windows

- gh-131020 [https://github.com/python/cpython/issues/131020]: pylauncher [https://github.com/python/cpython/tree/3.15/PC/launcher2.c] detecta corretamente um BOM ao procurar pelo shebang. Correção por Chris Eibl.

Ferramentas/Demos

- gh-130453 [https://github.com/python/cpython/issues/130453]: Torna possível substituir palavra reservada padrão em **pygettext**.
- gh-85012 [https://github.com/python/cpython/issues/85012]: Redefine corretamente `msgctxt` ao compilar mensagens em **msgfmt**.
- gh-130453 [https://github.com/python/cpython/issues/130453]: Estende o suporte para especificar palavras reservadas personalizadas em **pygettext**.
- gh-130195 [https://github.com/python/cpython/issues/130195]: Adiciona mensagens de aviso quando a opção `-a/--extract-all` não implementada `pygettext` for chamada.
- gh-130057 [https://github.com/python/cpython/issues/130057]: Adiciona suporte para comentários do tradutor em **pygettext.py**.
- gh-130025 [https://github.com/python/cpython/issues/130025]: O ambiente de teste do iOS agora manipula corretamente links simbólicos usados como referências de framework Python.
- gh-129911 [https://github.com/python/cpython/issues/129911]: Corrige a entrada de palavra reservada na saída de ajuda de **pygettext**.

Testes

- gh-129200 [https://github.com/python/cpython/issues/129200]: Agora, vários executores do ambiente de testes do iOS podem ser iniciados ao mesmo tempo sem gerar ambiguidade sobre a propriedade do simulador.
- gh-130292 [https://github.com/python/cpython/issues/130292]: O banco de testes do iOS agora será executado com sucesso em uma máquina que não executou testes do Xcode anteriormente (como configurações de CI).
- gh-130293 [https://github.com/python/cpython/issues/130293]: Os testes de colorização terminal não são mais sensíveis ao valor da variável `TERM` no ambiente de teste.
- gh-129401 [https://github.com/python/cpython/issues/129401]: Corrige um teste instável em `test_repr_rlock` que verifica a representação de **`multiprocessing.RLock`**.
- gh-126332 [https://github.com/python/cpython/issues/126332]: Adiciona testes de unidade para pyrepl.

Segurança

- gh-127371 [https://github.com/python/cpython/issues/127371]: Evita buffering ilimitado para **`tempfile.SpooledTemporaryFile.writelines()`**. Anteriormente, o spillover de disco era verificado somente após o iterador de linhas ter sido esgotado. Agora, isso é feito após cada linha ser escrita.

Biblioteca

- gh-131204 [https://github.com/python/cpython/issues/131204]: Usa a fonte monoespaçada do System Font Stack para suporte multiplataforma em **`difflib.HtmlDiff`**.
- gh-131196 [https://github.com/python/cpython/issues/131196]: Melhora o desempenho de **`uuid.UUID.hex`** e **`uuid.UUID.__str__`**.
- gh-130940 [https://github.com/python/cpython/issues/130940]: O comportamento de `PyConfig.use_system_logger` foi modificado para ser habilitado por padrão no iOS. Ele permanece desabilitado por padrão no macOS.
- gh-131123 [https://github.com/python/cpython/issues/131123]: Complementações suportadas para atributos de variáveis de conveniência em **`pdb`**.

- gh-93096 [https://github.com/python/cpython/issues/93096]: Removida CLI não documentado `python -m difflib`. Em vez disso, use `python -m doctest Lib/difflib.py -v`. Patch por Semyon Moroz.
- gh-93096 [https://github.com/python/cpython/issues/93096]: Removidos os argumentos não documentados `-t` e `-v` de `python -m pickle`. Use `python -m doctest Lib/pickle.py -v` em seu lugar. Patch por Semyon Moroz.
- gh-81267 [https://github.com/python/cpython/issues/81267]: Corrige a mensagem de erro **`time.sleep()`** quando um objeto que não pode ser interpretado como um inteiro ou float é fornecido.
- gh-93096 [https://github.com/python/cpython/issues/93096]: Removidos os argumentos não documentados `-t` e `-v` de `python -m pickletools`. Em vez disso, use `python -m doctest Lib/pickletools.py -v`. Patch por Semyon Moroz.
- gh-131045 [https://github.com/python/cpython/issues/131045]: Corrige problema com `__contains__`, valores e pseudomembros para **`enum.Flag`**.
- gh-130959 [https://github.com/python/cpython/issues/130959]: Corrige implementação puro Python de **`datetime.time.fromisoformat()`** para rejeitar horários com espaços na parte fracionária (por exemplo, `12:34:56.400 +02:00`), correspondendo à implementação C. Patch por Michał Gorny.
- gh-130806 [https://github.com/python/cpython/issues/130806]: Exclui **`gzip.GzipFile`** antes de ser fechado agora emite uma **`ResourceWarning`**.
- gh-130637 [https://github.com/python/cpython/issues/130637]: Adiciona validação para dados de resposta numérica no método poplib.POP3.stat()
- gh-130665 [https://github.com/python/cpython/issues/130665]: Aplica somente a localidade à CLI calendar quando definido via `--locale` e não via variável de ambiente `LANG`.
- gh-130660 [https://github.com/python/cpython/issues/130660]: `sys.ps1` e `sys.ps2` agora são restaurados após a chamada **`code.interact()`**.
- gh-130608 [https://github.com/python/cpython/issues/130608]: Remove o argumento *dirs_exist_ok* de **`pathlib.Path.copy()`** e **`copy_into()`**. Esses métodos são novos no Python 3.14.
- gh-130461 [https://github.com/python/cpython/issues/130461]: Remove as diretivas `.. index::` da documentação do módulo **`uuid`**. Essas diretivas

criavam entradas no índice geral para `getnode()`, bem como para as funções construtoras `uuid1()`, `uuid3()`, `uuid4()`, `uuid5()` e `uuid8()`.

- gh-130379 [https://github.com/python/cpython/issues/130379]: O módulo zipapp agora calcula a lista de arquivos a serem adicionados ao arquivo antes de criá-lo. Isso evita incluir acidentalmente o alvo quando ele está sendo criado no diretório de origem.
- gh-82987 [https://github.com/python/cpython/issues/82987]: Pontos de interrupção em linha como `breakpoint()` ou `pdb.set_trace()` sempre interromperão o programa ao chamar o quadro, ignorando o padrão `skip` (se houver).
- gh-125377 [https://github.com/python/cpython/issues/125377]: `<tab>` no início da linha em `pdb` a entrada multilinha preencherá uma indentação de 4 espaços agora, em vez de inserir um caractere `\t`.
- gh-125413 [https://github.com/python/cpython/issues/125413]: Certifica-se de que o caminho retornado de `pathlib.Path.copy()` ou `move()` tenha `info` atualizado.
- gh-65697 [https://github.com/python/cpython/issues/65697]: O configparser do stdlib agora tentará validar se as chaves que ele grava não resultarão em corrupção de arquivo (criando um arquivo que não poderá ser analisado com precisão por uma futura chamada read() do mesmo analisador sintático). Tentar corromper o comando write() levantará uma InvalidWriteError.
- gh-125413 [https://github.com/python/cpython/issues/125413]: Acelera `Path.copy` fazendo melhor uso de `info` internamente.
- gh-130285 [https://github.com/python/cpython/issues/130285]: Corrige casos extremos para `random.sample()` permitindo que o parâmetro *counts* especifique uma população vazia. Então agora, `sample([], 0, counts=[])` e `sample('abc', k=0, counts=[0, 0, 0])` ambos dão o mesmo resultado que `sample([], 0)`.
- gh-124703 [https://github.com/python/cpython/issues/124703]: Executa o comando `quit` em `pdb` levantará `bdb.BdbQuit` quando **`pdb`** for iniciado em um console interativo usando `breakpoint()` ou `pdb.set_trace()`.
- gh-107773 [https://github.com/python/cpython/issues/107773]: Torna a subclasse `datetime.__repr__()` consistente em ambas as implementações. Patch por Semyon Moroz.
- gh-130250 [https://github.com/python/cpython/issues/130250]: Corrige uma regressão em `traceback.print_last()`.

- gh-123471 [https://github.com/python/cpython/issues/123471]: Torna as iterações simultâneas sobre `itertools.batched` seguras sob threads livres.
- gh-57537 [https://github.com/python/cpython/issues/57537]: Suporte a pontos de interrupção para módulos `zipimport` em `pdb`
- gh-130230 [https://github.com/python/cpython/issues/130230]: Corrige falha em `pow()` com apenas `Decimal` terceiro argumento.
- gh-126944 [https://github.com/python/cpython/issues/126944]: Mostra erros explícitos quando os argumentos obrigatórios dos comandos `pdb` estiverem faltando
- gh-127750 [https://github.com/python/cpython/issues/127750]: Melhora a representação dos métodos e descritores `functools.singledispatchmethod`.
- gh-128520 [https://github.com/python/cpython/issues/128520]: Aplica conversão de tipo consistentemente nos métodos `pathlib.PurePath` e `Path` pode aceitar um objeto de caminho como argumento, como `match()` e `rename()`. O argumento agora é convertido em objeto de caminho se não possuir o atributo `with_segments()`, e não em caso contrário.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Reverte uma alteração na versão anterior, tentando tornar preguiçosas algumas importações stdlib usadas no módulo `subprocess`, pois isso estava causando erros durante os finalizadores `__del__` que chamavam métodos como `terminate`, `kill` ou `send_signal`.
- gh-130164 [https://github.com/python/cpython/issues/130164]: Corrigia uma falha ao levantar `TypeError` em `inspect.Signature.bind()` para argumentos somente posicionais fornecidos como nomeados quando um argumento nomeado variável (por exemplo, `**kwargs`) está presente.
- gh-130151 [https://github.com/python/cpython/issues/130151]: Corrigir vazamentos de referência em **`_hashlib.hmac_new()`** e **`_hashlib.hmac_digest()`**. Patch por Bénédikt Tran.
- gh-130145 [https://github.com/python/cpython/issues/130145]: Corrige **`asyncio.AbstractEventloop.run_forever()`** quando outro laço já estiver em execução.
- gh-130139 [https://github.com/python/cpython/issues/130139]: Corrige bug onde `ast.parse()` não gerava erro na entrada AST que não era do tipo correto, quando chamado com optimize=False.

- gh-127260 [https://github.com/python/cpython/issues/127260]: Proibição do uso de dois pontos (“:”) como separador de componentes fracionários e outras melhorias na consistência de levantar erros entre as implementações em C e Python de **`datetime.time.fromisoformat()`** e **`datetime.datetime.fromisoformat()`**. Patch por Semyon Moroz.
- gh-85795 [https://github.com/python/cpython/issues/85795]: O uso de **`super()`** e `__class__` variável de clausura em métodos definidos pelo usuário das subclasses **`typing.NamedTuple`** agora é explicitamente proibido em tempo de execução. Contribuição de Bartosz Sławecki em gh-130082 [https://github.com/python/cpython/issues/130082].
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação de **`cmd`** importando lentamente **`inspect`** e removendo **`string`**. Patch por Semyon Moroz.
- gh-129726 [https://github.com/python/cpython/issues/129726]: Corrige **`gzip.GzipFile`** levantando uma exceção não levantável durante a coleta de lixo ao fazer referência a um objeto temporário, interrompendo o laço de referências com **`weakref`**.
- gh-127750 [https://github.com/python/cpython/issues/127750]: Remove o armazenamento em cache quebrado **`functools.singledispatchmethod()`** introduzido em gh-85160 [https://github.com/python/cpython/issues/85160]. Obtém o mesmo desempenho usando otimizações diferentes.
- gh-129948 [https://github.com/python/cpython/issues/129948]: Adiciona suporte para **`set`** compartilhado para **`multiprocessing.managers.SyncManager`** via **`SyncManager.set()`**.
- gh-129965 [https://github.com/python/cpython/issues/129965]: Atualização dos tipos MIME para `.avi` e `.wav`. Adição dos tipos MIME para `.docx`, `.pptx`, `.xlsx`, `.epub`, `.flac`, `.m4a`, `.odg`, `.odp`, `.ods`, `.odt`, `.oga`, `.ogg`, `.ogx` e `.weba`. Patch por Hugo van Kemenade.
- gh-129889 [https://github.com/python/cpython/issues/129889]: Suporte ao protocolo do gerenciador de contexto por **`contextvars.Token`**. Patch por Andrew Svetlov.
- gh-97850 [https://github.com/python/cpython/issues/97850]: Update the deprecation warning of `importlib.abc.Loader.load_module`.
- gh-129678 [https://github.com/python/cpython/issues/129678]: **`configparser.ConfigParser`**: não escreve uma seção vazia sem nome

- gh-128641 [https://github.com/python/cpython/issues/128641]: Restaura o desempenho de `configparser.ConfigParser.read()`.
- gh-129569 [https://github.com/python/cpython/issues/129569]: Corrige `unicodedata.normalize()` para sempre retornar um objeto embutido `str` quando receber uma entrada de uma subclasse `str`, independentemente de a string já estar normalizada.
- gh-128231 [https://github.com/python/cpython/issues/128231]: A execução de múltiplas instruções no novo REPL agora para imediatamente após a primeira exceção encontrada. Patch por Bartosz Sławecki.
- gh-96092 [https://github.com/python/cpython/issues/96092]: Corrige um bug em `traceback.walk_stack()` chamado com None, que estava pulando mais quadros do que nas versões anteriores. Esta correção de bug também altera o walk_stack para percorrer a pilha no quadro em que foi chamado, em vez de onde foi usado pela primeira vez.
- gh-129288 [https://github.com/python/cpython/issues/129288]: Adiciona os campos opcionais `l2_cid` e `l2_bdaddr_type` à tupla sockaddr `socket` `BTPROTO_L2CAP`.
- gh-128703 [https://github.com/python/cpython/issues/128703]: Corrige `mimetypes.guess_type()` para usar o mapeamento padrão para `Content-Type` vazio no registro.
- gh-128647 [https://github.com/python/cpython/issues/128647]: Escreve rapidamente nos buffers passados para as implementações `readinto()` e `readinto1()` de `gzip.GzipFile`, evitando alocações desnecessárias. Patch por Chris Markiewicz.
- gh-128184 [https://github.com/python/cpython/issues/128184]: Melhoria na exibição do objeto `annotationlib.ForwardRef` dentro das representações `inspect.Signature`. Isso também corrige uma `NameError` levantada ao usar `dataclasses.dataclass()` em classes com referências futuras não resolvíveis.
- gh-128041 [https://github.com/python/cpython/issues/128041]: Adiciona `concurrent.futures.ProcessPoolExecutor.terminate_workers()` e `concurrent.futures.ProcessPoolExecutor.kill_workers()` como formas de encerrar ou eliminar todos os processos de trabalho ativos no pool fornecido. (Contribuição de Charles Machalow em gh-130849 [https://github.com/python/cpython/issues/130849].)
- gh-127647 [https://github.com/python/cpython/issues/127647]: Adiciona os protocolos `io.Reader` e `io.Writer` como alternativas para `typing.IO`,

`typing.TextIO` e `typing.BinaryIO`.

- gh-109798 [https://github.com/python/cpython/issues/109798]: Adicionadas informações adicionais às mensagens de erro em `datetime` e tornando as mensagens mais consistentes entre as implementações em C e Python. Patch por Semyon Moroz.
- gh-125746 [https://github.com/python/cpython/issues/125746]: Delay deprecated **`zipimport.zipimporter.load_module()`** removal time to 3.15. Use `zipimport.zipimporter.exec_module()` instead.
- gh-74028 [https://github.com/python/cpython/issues/74028]: Adiciona o parâmetro opcional `buffersize` a `concurrent.futures.Executor.map()` para limitar o número de tarefas enviadas cujos resultados ainda não foram produzidos. Se o buffer estiver cheio, a iteração sobre os *iteráveis* será pausada até que um resultado seja produzido pelo buffer.
- gh-124927 [https://github.com/python/cpython/issues/124927]: Caracteres não imprimíveis agora são manipulados corretamente no novo REPL.
- gh-124096 [https://github.com/python/cpython/issues/124096]: Ativa o modo de terminal virtual e habilita a colagem entre colchetes no REPL no console do Windows. (Se o terminal não oferecer suporte à colagem entre colchetes, habilitá-la não fará nada.)
- gh-89083 [https://github.com/python/cpython/issues/89083]: Adiciona `uuid.uuid7()` para gerar objetos UUIDv7, conforme especificado em **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html]. Patch por Bénédikt Tran.
- gh-89083 [https://github.com/python/cpython/issues/89083]: Adiciona `uuid.uuid6()` para gerar objetos UUIDv6, conforme especificado em **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html]. Patch por Bénédikt Tran.
- gh-117151 [https://github.com/python/cpython/issues/117151]: Aumenta `io.DEFAULT_BUFFER_SIZE` de 8k para 128k e ajusta `open()` em plataformas onde `os.fstat()` fornece um campo `st_blksize` (como Linux) para usar `max(min(blocksize, 8 MiB), io.DEFAULT_BUFFER_SIZE)` em vez de sempre usar o tamanho do bloco do dispositivo. Isso deve melhorar o desempenho de E/S. Patch por Romain Morotti.
- gh-105499 [https://github.com/python/cpython/issues/105499]: Transforma `types.UnionType` em um apelido para `typing.Union`. Tanto `int |`

`str` quanto `Union[int, str]` agora criam instâncias do mesmo tipo. Patch por Jelle Zijlstra.
- gh-93096 [https://github.com/python/cpython/issues/93096]: Documenta a linha de comando para `mimetypes`. Agora, ele sai com `1` em caso de falha, em vez de `0`, e `2` em caso de parâmetros de linha de comando incorretos, em vez de `1`. Além disso, os erros são exibidos em stderr em vez de stdout, e seu texto ficou mais conciso. Patch por Oleg Iarygin e Hugo van Kemenade.

Documentação

- gh-125722 [https://github.com/python/cpython/issues/125722]: Requer Sphinx 8.2.0 ou posterior para construir a documentação do Python. Patch por Adam Turner.
- gh-129712 [https://github.com/python/cpython/issues/129712]: As tags de wheels suportadas por cada opção do macOS Universal SDK agora estão documentadas.
- gh-46236 [https://github.com/python/cpython/issues/46236]: API C: Documenta `PyUnicode_RSplit()`, `PyUnicode_Partition()` e `PyUnicode_RPartition()`.

Núcleo e embutidos

- gh-131141 [https://github.com/python/cpython/issues/131141]: Corrige disputa de dados na instrumentação `sys.monitoring` ao registrar retorno de chamada.
- gh-130804 [https://github.com/python/cpython/issues/130804]: Corrige suporte a caracteres Unicode no Windows no novo REPL.
- gh-130932 [https://github.com/python/cpython/issues/130932]: Corrige tratamento de exceção incorreto em `_PyModule_IsPossiblyShadowing`
- gh-122029 [https://github.com/python/cpython/issues/122029]: `sys.setprofile()` e `sys.settrace()` não gerarão um evento `c_call` para `INSTRUMENTED_CALL_FUNCTION_EX` se o método chamável for um método com uma função C encapsulada, porque não geramos um evento `c_return` nesse caso.
- gh-129964 [https://github.com/python/cpython/issues/129964]: Corrige travamento do JIT no Windows no Arm. Patch por Diego Russo e Brandt Bucher.

- gh-130851 [https://github.com/python/cpython/issues/130851]: Corrige uma falha na construção com threads livres ao construir um objeto `code` com `co_consts` que contém instâncias de tipos que não são gerados pelo compilador de bytecode.
- gh-128534 [https://github.com/python/cpython/issues/128534]: Certifica-se de que os ramos esquerdo e direito tenham a mesma origem para laços `async for`. Adicione esses ramos ao iterador `co_branches()`.
- gh-130794 [https://github.com/python/cpython/issues/130794]: Corrige vazamento de memória na construção com threads livres ao redimensionar uma lista ou dicionário compartilhado de vários threads de curta duração.
- gh-130415 [https://github.com/python/cpython/issues/130415]: Melhora a compreensão do JIT de números inteiros em contexto booleano.
- gh-130382 [https://github.com/python/cpython/issues/130382]: Corrige `PyRefTracer_DESTROY` não sendo enviado de `Py_DECREF()` de `Python/ceval.c`
- gh-130574 [https://github.com/python/cpython/issues/130574]: Renumera `RESUME` de 149 para 128.
- gh-124878 [https://github.com/python/cpython/issues/124878]: Corrige condições de corrida durante a finalização do tempo de execução que poderiam levar ao acesso à memória liberada.
- gh-130415 [https://github.com/python/cpython/issues/130415]: Melhora a capacidade do JIT experimental de restringir valores booleanos com base nos resultados de testes de veracidade.
- gh-130618 [https://github.com/python/cpython/issues/130618]: Corrige um bug que estava causando o levantamento de `UnicodeDecodeError` ou `SystemError` ao usar f-strings com expressões `lambda` com caracteres não-ASCII. Patch por Pablo Galindo
- gh-123044 [https://github.com/python/cpython/issues/123044]: Certifica-se de que a localização dos alvos de ramificação nos casos `match` esteja no corpo, não no padrão.
- gh-128534 [https://github.com/python/cpython/issues/128534]: Adiciona monitoramento de ramificação (eventos `BRANCH_LEFT` e `BRANCH_RIGHT`) para laços `async for`.
- gh-130163 [https://github.com/python/cpython/issues/130163]: Corrige possíveis travamentos relacionados à alteração simultânea e ao uso dos atributos do módulo `sys`.

- gh-122029 [https://github.com/python/cpython/issues/122029]: `INSTRUMENTED_CALL_KW` expandirá o método antes do monitoramento para refletir o comportamento real com mais precisão.
- gh-130415 [https://github.com/python/cpython/issues/130415]: Melhora a capacidade do JIT de otimizar strings em contextos booleanos.
- gh-130396 [https://github.com/python/cpython/issues/130396]: Usa limites de pilha reais (de *pthread_getattr_np(3)* *[https://manpages.debian.org/pthread_getattr_np(3)]*) para Linux e outros sistemas com `_GNU_SOURCE` definido ao determinar limites para proteção de pilha C.
- gh-128396 [https://github.com/python/cpython/issues/128396]: Corrige uma falha que ocorre ao chamar **`locals()`** dentro de uma compreensão em linha que usa a mesma variável local que o escopo do quadro externo, onde a variável é uma variável livre ou de célula.
- gh-129107 [https://github.com/python/cpython/issues/129107]: Corrige mais duas funções **`bytearray`** para threads livres.
- gh-127705 [https://github.com/python/cpython/issues/127705]: Usa referências marcadas (`_PyStackRef`) para a construção padrão, bem como para a construção com threads livres. Isso tem um pequeno impacto negativo no desempenho a curto prazo, mas permitirá acelerações maiores no futuro e reduzirá significativamente os custos de manutenção, permitindo uma única implementação de referências marcadas no futuro.
- gh-130094 [https://github.com/python/cpython/issues/130094]: Corrige duas condições de corrida envolvendo importações simultâneas que poderiam levar a falhas espúrias com **`ModuleNotFoundError`**.
- gh-129107 [https://github.com/python/cpython/issues/129107]: Torna o iterador **`bytearray`** seguro sob threads livres.
- gh-115802 [https://github.com/python/cpython/issues/115802]: Use o modelo de código “médio” mais eficiente para código compilado JIT em plataformas suportadas.
- gh-107956 [https://github.com/python/cpython/issues/107956]: Um arquivo `build-details.json` agora está instalado no diretório da biblioteca padrão independente de plataforma (implementação da **PEP 739** [https://peps.python.org/pep-0739/]).
- gh-116042 [https://github.com/python/cpython/issues/116042]: Corrige local para SyntaxErrors de escapes inválidos no tokenizador. Patch por Pablo

Galindo

- gh-91079 [https://github.com/python/cpython/issues/91079]: Altera a proteção contra estouro de pilha em C para considerar a quantidade de pilha consumida, em vez de um contador. Isso permite recursão mais profunda em muitos casos, mas permanece seguro.
- gh-129715 [https://github.com/python/cpython/issues/129715]: Melhora o tratamento do JIT experimental de retornos para chamadores desconhecidos.

Biblioteca

- gh-129983 [https://github.com/python/cpython/issues/129983]: Corrige corrida de dados em compile_template em `sre.c`.

Núcleo e embutidos

- gh-129967 [https://github.com/python/cpython/issues/129967]: Corrige uma condição de corrida na construção com threads livres quando `repr(set)` é chamado simultaneamente com `set.clear()`.
- gh-129953 [https://github.com/python/cpython/issues/129953]: A pilha interna (de avaliação) agora é transferida para a memória sempre que a execução escapa do interpretador ou do código compilado por JIT. Isso não deve ter efeito observável em Python ou em extensões embutidas, mas permitirá várias otimizações importantes no futuro.
- gh-129515 [https://github.com/python/cpython/issues/129515]: Esclarece mensagens de erro de sintaxe para expressões condicionais quando uma instrução é especificada antes de uma palavra reservada **`if`** ou depois de uma palavra reservada **`else`**.
- gh-129349 [https://github.com/python/cpython/issues/129349]: **`bytes.fromhex()`** e **`bytearray.fromhex()`** agora aceita ASCII **`bytes`** e objetos bytes ou similares.
- gh-129149 [https://github.com/python/cpython/issues/129149]: Adiciona um atalho para inteiros de tamanho médio em **`PyLong_FromSsize_t()`**. Patch por Chris Eibl.
- gh-129107 [https://github.com/python/cpython/issues/129107]: Torne o **`bytearray`** seguro em threads livres.

- gh-128974 [https://github.com/python/cpython/issues/128974]: Corrige uma falha em **`UnicodeError.__str__`** quando atributos personalizados implementam **`__str__()`** com efeitos colaterais. Patch por Bénédikt Tran.
- gh-126085 [https://github.com/python/cpython/issues/126085]: **`typing.TypeAliasType`** agora oferece suporte a desempacotamento em estrela.
- gh-125331 [https://github.com/python/cpython/issues/125331]: `from __future__ import barry_as_FLUFL` agora funciona em mais contextos, incluindo quando é usado em arquivos, com o sinalizador `-c` e no REPL quando há várias instruções na mesma linha. Anteriormente, funcionava apenas em linhas subsequentes no REPL e quando os sinalizadores apropriados eram passados diretamente para **`compile()`**. Patch por Pablo Galindo.
- gh-121464 [https://github.com/python/cpython/issues/121464]: Torna iterações simultâneas sobre o mesmo iterador **`enumerate()`** seguras em threads livres. Consulte Estratégia para Iteradores em Threads Livres [https://github.com/python/cpython/issues/124397].
- gh-87790 [https://github.com/python/cpython/issues/87790]: Suporte para sublinhado e vírgula como separadores de milhares na parte fracionária para tipos de apresentação de ponto flutuante do novo estilo de formatação de string (com **`format()`** ou Literais de strings formatadas). Patch por Sergey B Kirpichev.
- gh-124445 [https://github.com/python/cpython/issues/124445]: Corrige a especialização de apelidos genéricos que são genéricos em uma **`typing.ParamSpec`** e foram especializados com um tipo variável aninhado.
- gh-120608 [https://github.com/python/cpython/issues/120608]: Adapta **`reversed()`** para uso na construção com threads livres. **`reversed()`** ainda não é seguro para thread, no sentido de que iterações simultâneas podem ver o mesmo objeto, mas não corromperão o estado do interpretador.

Biblioteca

- gh-100388 [https://github.com/python/cpython/issues/100388]: Corrige o método `platform._sys_version()` quando `__DATE__` não estiver definido no

momento da compilação, alterando a string de data e hora padrão do momento da construção para a época UNIX.

Núcleo e embutidos

- bpo-44369 [https://bugs.python.org/issue?@action=redirect&bpo=44369]: Melhoria nos erros de sintaxe para strings fechadas incorretamente. Patch por Pablo Galindo.

API C

- gh-111178 [https://github.com/python/cpython/issues/111178]: Correção da API **`PyCMethod`**: substituição de `size_t nargs` por `Py_ssize_t nargs` em **`PyCMethod`**. Patch por Victor Stinner.

- gh-130947 [https://github.com/python/cpython/issues/130947]: Adiciona novamente **`PySequence_Fast()`** à API C limitada. Patch por Victor Stinner.

- gh-128863 [https://github.com/python/cpython/issues/128863]: As seguintes funções privadas estão descontinuado e planejadas para remoção no Python 3.18:

  - **`_PyUnicodeWriter_Init()`**: substitua `_PyUnicodeWriter_Init(&writer)` com **`writer = PyUnicodeWriter_Create(0)`**.
  - **`_PyUnicodeWriter_Finish()`**: substitua `_PyUnicodeWriter_Finish(&writer)` por **`PyUnicodeWriter_Finish(writer)`**.
  - **`_PyUnicodeWriter_Dealloc()`**: substitua `_PyUnicodeWriter_Dealloc(&writer)` por **`PyUnicodeWriter_Discard(writer)`**.
  - **`_PyUnicodeWriter_WriteChar()`**: substituta `_PyUnicodeWriter_WriteChar(&writer, ch)` com **`PyUnicodeWriter_WriteChar(writer, ch)`**.
  - **`_PyUnicodeWriter_WriteStr()`**: substitua `_PyUnicodeWriter_WriteStr(&writer, str)` por

`PyUnicodeWriter_WriteStr(writer, str)`.
  - **`_PyUnicodeWriter_WriteSubstring()`**: substitua `_PyUnicodeWriter_WriteSubstring(&writer, str, start, end)` por `PyUnicodeWriter_WriteSubstring(writer, str, start, end)`.
  - **`_PyUnicodeWriter_WriteASCIIString()`**: substitua `_PyUnicodeWriter_WriteASCIIString(&writer, str)` por `PyUnicodeWriter_WriteUTF8(writer, str)`.
  - **`_PyUnicodeWriter_WriteLatin1String()`**: substitua `_PyUnicodeWriter_WriteLatin1String(&writer, str)` por `PyUnicodeWriter_WriteUTF8(writer, str)`.
  - **`_PyUnicodeWriter_Prepare()`**: (sem substituto).
  - **`_PyUnicodeWriter_PrepareKind()`**: (sem substituto).

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas novas funções públicas no Python 3.13 e versões anteriores.

Patch por Victor Stinner.

- gh-45325 [https://github.com/python/cpython/issues/45325]: Adiciona um novo parâmetro de formato `p` a `Py_BuildValue()` que permite pegar um inteiro em C e produzir um objeto Python `bool`. Patch por Pablo Galindo.

Construção

- gh-131035 [https://github.com/python/cpython/issues/131035]: Usa `-flto=thin` para tempos de construção mais rápidos usando clang-cl no Windows. Patch por Chris Eibl.
- gh-130740 [https://github.com/python/cpython/issues/130740]: Certifica-se de que `Python.h` esteja incluído antes de `stdbool.h`, a menos que `pyconfig.h` esteja incluído antes ou em alguns contextos específicos da plataforma.
- gh-130090 [https://github.com/python/cpython/issues/130090]: Construir com `PlatformToolset=ClangCL` no Windows agora oferece suporte a PGO (otimização guiada por perfil). Patch por Chris Eibl com apoio inestimável de Steve Dover.

- gh-129819 [https://github.com/python/cpython/issues/129819]: Permite a construção do JIT com o interpretador de chamada de cauda.
- gh-129989 [https://github.com/python/cpython/issues/129989]: Corrige um bug em que o interpretador da chamada de cauda era habilitado quando `--without-tail-call-interp` era fornecido ao script configure.
- gh-129838 [https://github.com/python/cpython/issues/129838]: Não redefine `_Py_NO_SANITIZE_UNDEFINED` ao compilar com uma versão recente do GCC e um sanitizador indefinido habilitado.
- gh-82909 [https://github.com/python/cpython/issues/82909]: `#pragma`-based linking with `python3*.lib` agora pode ser desabilitado com Py_NO_LINK_LIB. Patch por Jean-Christophe Fillion-Robin.

Python 3.14.0 alpha 5

Data de lançamento: 2025-02-11

macOS

- gh-91132 [https://github.com/python/cpython/issues/91132]: Atualiza o instalador do macOS para usar o ncurses 6.5.

Ferramentas/Demos

- gh-129248 [https://github.com/python/cpython/issues/129248]: O executor de testes do iOS agora remove o prefixo de log de cada linha de saída do conjunto de testes.
- gh-104400 [https://github.com/python/cpython/issues/104400]: Corrige vários bugs na extração mudando para um analisador sintático de AST em **pygettext**.

Testes

- gh-129386 [https://github.com/python/cpython/issues/129386]: Adiciona `test.support.reset_code`, que pode ser usado para redefinir várias otimizações em nível de bytecode e instrumentação local para uma função.

- gh-128474 [https://github.com/python/cpython/issues/128474]: Desabilita os casos de teste `test_embed` que apresentam falhas de segmentação em binários de instrumentos BOLT. Os testes são desabilitados somente quando o BOLT está habilitado.
- gh-128003 [https://github.com/python/cpython/issues/128003]: Adiciona a opção `--parallel-threads=N` ao executor de testes de regressão, que executa testes individuais em múltiplas threads em paralelo para encontrar bugs de simultaneidade. Observe que a maior parte do conjunto de testes ainda não foi revisada quanto à segurança de threads nem anotada com `@thread_unsafe` quando necessário.

Segurança

- gh-105704 [https://github.com/python/cpython/issues/105704]: Ao usar **`urllib.parse.urlsplit()`** e **`urllib.parse.urlparse()`**, a análise de host não rejeitaria nomes de domínio contendo colchetes (`[` e `]`). Os colchetes são válidos somente para hosts IPv6 e IPvFuture de acordo com RFC 3986 Seção 3.2.2 [https://www.rfc-editor.org/rfc/rfc3986#section-3.2.2].
- gh-126108 [https://github.com/python/cpython/issues/126108]: Corrige uma possível desreferência de ponteiro `NULL` em **`PySys_AddWarnOptionUnicode()`**.
- gh-80222 [https://github.com/python/cpython/issues/80222]: Corrige bug na dobragem *(folding)* de strings entre aspas ao achatar uma mensagem de e-mail usando uma política de e-mail moderna. Anteriormente, quando uma string entre aspas era dobrada para abranger mais de uma linha, as aspas circundantes e escapes internos eram omitidos. Isso poderia, teoricamente, ser usado para fazer *spoof* das linhas de cabeçalho usando uma string entre aspas cuidadosamente construída se o e-mail renderizado resultante fosse transmitido ou reanalisado.
- gh-119511 [https://github.com/python/cpython/issues/119511]: Corrige uma potencial negação de serviço no módulo **`imaplib`**. Ao conectar-se a um servidor malicioso, isso poderia fazer com que uma quantidade arbitrária de memória fosse alocada. Em muitos sistemas, isso é inofensivo, pois a memória virtual não utilizada é apenas um mapeamento, mas se isso atingir um limite de tamanho de endereço virtual, pode levar a um **`MemoryError`** ou outra falha de processo. Em sistemas ou construções incomuns, onde toda a memória alocada é

tocada e apoiada por RAM ou armazenamento real, isso pode ter consumido recursos ao fazer isso até travar de forma semelhante.

Biblioteca

- gh-129939 [https://github.com/python/cpython/issues/129939]: Páginas de comparação com alterações destacadas geradas pela classe **`difflib.HtmlDiff`** agora oferecem suporte ao modo escuro.
- gh-129928 [https://github.com/python/cpython/issues/129928]: Levanta **`sqlite3.ProgrammingError`** se uma função SQL definida pelo usuário com um número inválido de parâmetros for criada. Patch por Erlend Aasland.
- gh-129583 [https://github.com/python/cpython/issues/129583]: Atualiza pip empacotado para 25.0.1
- gh-129766 [https://github.com/python/cpython/issues/129766]: Corrige falha em **`warnings`**, ao chamar `_release_lock()` sem trava existente.
- gh-129005 [https://github.com/python/cpython/issues/129005]: `_pyio.FileIO.readall()` agora aloca, redimensiona e preenche um buffer de dados usando o mesmo algoritmo que `_io.FileIO.readall()` usa.
- gh-129646 [https://github.com/python/cpython/issues/129646]: Atualiza o mapeamento de apelido de localidade no módulo **`locale`** para corresponder ao mapeamento de apelido de localidade mais recente do X Org e oferecer suporte a novas localidades no Glibc 2.41.
- gh-128317 [https://github.com/python/cpython/issues/128317]: Coloca o destaque do calendário CLI na classe privada, removendo `highlight_day` da API pública **`calendar.TextCalendar`**. Patch por Hugo van Kemenade.
- gh-129603 [https://github.com/python/cpython/issues/129603]: Corrige bugs onde objetos **`sqlite3.Row`** poderiam sofrer segfault se seu **`description`** herdado fosse definido como `None`. Patch por Erlend Aasland.
- gh-129559 [https://github.com/python/cpython/issues/129559]: Adiciona o método **`bytearray.resize()`** para que **`bytearray`** possa ser redimensionado eficientemente internamente.
- gh-129502 [https://github.com/python/cpython/issues/129502]: Erros improváveis na preparação de argumentos para o retorno de chamada **`ctypes`** agora são tratados da mesma forma que erros levantados no retorno de chamada ou na conversão do resultado do retorno de chamada —

usando `sys.unraisablehook()` em vez de `sys.excepthook()` e não definindo `sys.last_exc` e outras variáveis.

- gh-129403 [https://github.com/python/cpython/issues/129403]: Corrigida a mensagem de `ValueError` para `asyncio.Barrier` e `threading.Barrier`.
- gh-129409 [https://github.com/python/cpython/issues/129409]: Corrige um estouro de inteiro no módulo `csv` ao gravar um campo de dados maior que 2 GB.
- gh-126400 [https://github.com/python/cpython/issues/126400]: Adicione um argumento nomeado de soquete *timeout* a `logging.handlers.SysLogHandler`.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Sempre faz importação preguiçosa de `warnings` em `threading`. Patch por Taneli Hukkinen.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação de `subprocess` importando `locale` e `signal` de forma preguiçosa. Patch por Taneli Hukkinen.
- gh-129346 [https://github.com/python/cpython/issues/129346]: Em `sqlite3`, manipula a falta de memória ao criar funções SQL definidas pelo usuário.
- gh-129005 [https://github.com/python/cpython/issues/129005]: Otimiza `_pyio.FileIO.readinto` evitando objetos e cópias desnecessários usando `os.readinto()`.
- gh-129195 [https://github.com/python/cpython/issues/129195]: Suporte para geração de relatórios de informações do grafo de chamadas de **`asyncio.staggered.staggered_race()`**.
- gh-129205 [https://github.com/python/cpython/issues/129205]: Adiciona `os.readinto()` para ler um objeto buffer de um descritor de arquivo.
- gh-128772 [https://github.com/python/cpython/issues/128772]: Corrige `pydoc` para métodos com o atributo `__module__` igual a `None`.
- gh-129061 [https://github.com/python/cpython/issues/129061]: Corrige FORCE_COLOR e NO_COLOR quando strings vazias. Patch por Hugo van Kemenade.
- gh-92897 [https://github.com/python/cpython/issues/92897]: Foi agendada a descontinuação do argumento `check_home` de `sysconfig.is_python_build()` para o Python 3.15.

- gh-129064 [https://github.com/python/cpython/issues/129064]: Descontinua **`sysconfig.expand_makefile_vars()`**, em favor do uso de **`sysconfig.get_paths()`** com o argumento `vars`.
- gh-128550 [https://github.com/python/cpython/issues/128550]: Foi removida uma otimização incorreta relacionada a tarefas ansiosas em **`asyncio.TaskGroup`** que resultava em cancelamentos perdidos.
- gh-128991 [https://github.com/python/cpython/issues/128991]: Libera a referência do quadro de entrada dentro do retorno de chamada **`bdb`**
- gh-118761 [https://github.com/python/cpython/issues/118761]: Reduz o tempo de importação de **`pstats`** e **`zipfile`** em até 20%, removendo importações desnecessárias para **`typing`**. Patch por Bénédikt Tran.
- gh-128978 [https://github.com/python/cpython/issues/128978]: Corrige uma exceção **`NameError`** em **`sysconfig.expand_makefile_vars()`**. Patch por Bénédikt Tran.
- gh-128961 [https://github.com/python/cpython/issues/128961]: Corrige uma falha ao definir o estado em um iterador **`array.array`** esgotado.
- gh-128894 [https://github.com/python/cpython/issues/128894]: Corrige `traceback.TracebackException._format_syntax_error` para não falhar em exceções com metadados personalizados.
- gh-128916 [https://github.com/python/cpython/issues/128916]: Não tenta definir `SO_REUSEPORT` em soquetes de famílias de endereços diferentes de `AF_INET` e `AF_INET6`, pois isso não faz sentido com essas famílias de endereços, e a chamada falhará com o kernel Linux 6.12.9 e mais recentes.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhorado o tempo de importação de **`tomllib`** removendo as importações `typing`, `string` e `tomllib._types`. Patch por Taneli Hukkinen.
- gh-128679 [https://github.com/python/cpython/issues/128679]: **`tracemalloc`**: Corrige condições de corrida quando **`tracemalloc.stop()`** é chamado por uma thread, enquanto outras threads estão rastreando alocações de memória. Patch por Victor Stinner.
- gh-128891 [https://github.com/python/cpython/issues/128891]: Adiciona opcodes especializados a `opcode.opname`.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Reduz o tempo de importação de **`gettext`** em até dez vezes, importando **`re`** sob demanda. Em particular, `re` não é mais exposto implicitamente como `gettext.re`. Patch por Eli Schwartz.

- gh-118761 [https://github.com/python/cpython/issues/118761]: Reduz o tempo de importação de **`optparse`** quando nenhum texto de ajuda for impresso. Patch por Eli Schwartz.
- gh-128657 [https://github.com/python/cpython/issues/128657]: Corrige possível referência extra ao usar objetos retornados por **`hashlib.sha256()`** em threads livres.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Reduz o tempo de importação de **`csv`** em até cinco vezes, importando **`re`** sob demanda. Em particular, `re` não é mais exposto implicitamente como `csv.re`. Patch por Bénédikt Tran.
- gh-128308 [https://github.com/python/cpython/issues/128308]: Adiciona suporte ao argumento nomeado *name* para tarefas ansiosas em **`asyncio.loop.create_task()`**, **`asyncio.create_task()`** e **`asyncio.TaskGroup.create_task()`**, passando todos os *kwargs* para a fábrica de tarefas definida por **`asyncio.loop.set_task_factory()`**.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o desempenho de **`base64.b16decode()`** em até dez vezes, verificando com mais eficiência a sequência de bytes em busca de dígitos hexadecimais. Reduz o tempo de importação de **`base64`** em até seis vezes, não importando mais **`re`**. Patch por Bénédikt Tran, Chris Markiewicz e Adam Turner.
- gh-128156 [https://github.com/python/cpython/issues/128156]: Ao usar o sistema macOS `libffi`, o suporte para tipos complexos em **`ctypes`** agora é verificado em tempo de execução (macOS 10.15 ou mais recente). Os tipos também devem estar disponíveis em tempo de construção.
- gh-128636 [https://github.com/python/cpython/issues/128636]: Corrige falha do PyREPL quando **`os.environ`** é substituído por um valor inválido.
- gh-128498 [https://github.com/python/cpython/issues/128498]: Padrão para stdout isatty para detecção de cor em vez de stderr. Patch por Hugo van Kemenade.
- gh-128384 [https://github.com/python/cpython/issues/128384]: Adiciona trava para **`warnings`** para evitar algumas disputas de dados quando uma construção com threads livres for usado. Altera `_warnings_runtime_state.mutex` para um mutex recursivo e expõe-o a **`warnings`**, por meio das funções **`_acquire_lock()`** e **`_release_lock()`**. A trava é mantido quando `filters` e `_filters_version` são atualizados.

- gh-128509 [https://github.com/python/cpython/issues/128509]: Adiciona `sys._is_immortal()` para identificar objetos imortais em tempo de execução.
- gh-128479 [https://github.com/python/cpython/issues/128479]: Corrige **`asyncio.staggered.staggered_race()`** vazando tarefas e emitindo uma exceção não tratada.
- gh-128427 [https://github.com/python/cpython/issues/128427]: `uuid.NIL` e `uuid.MAX` agora estão disponíveis para representar os formatos UUID Nil e Max, conforme definido por **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html].
- gh-91279 [https://github.com/python/cpython/issues/91279]: `zipfile.ZipFile.writestr()` agora respeita `SOURCE_DATE_EPOCH` que as distribuições podem definir centralmente e fazer com que as ferramentas de construção consumam isso para produzir uma saída reproduzível.
- gh-112064 [https://github.com/python/cpython/issues/112064]: Corrige tratamento incorreto de tamanhos de leitura negativos em `HTTPResponse.read`. Patch por Yury Manushkin.
- gh-58956 [https://github.com/python/cpython/issues/58956]: Corrigido um vazamento de referência de quadro em `bdb`.
- gh-128131 [https://github.com/python/cpython/issues/128131]: Suporte completo ao acesso aleatório de arquivos zip somente leitura, não criptografados e descompactados obtidos por `ZipFile.open`.
- gh-127975 [https://github.com/python/cpython/issues/127975]: Evita reutilizar tipos de aspas em `ast.unparse()` se não for necessário.
- gh-115514 [https://github.com/python/cpython/issues/115514]: Corrige exceções e escritas incompletas após **`asyncio._SelectorTransport`** ser fechado antes das gravações serem concluídas.
- gh-121604 [https://github.com/python/cpython/issues/121604]: Adiciona avisos de Deprecation em falta para `importlib.machinery.DEBUG_BYTECODE_SUFFIXES`, `importlib.machinery.OPTIMIZED_BYTECODE_SUFFIXES`, `importlib.machinery.WindowsRegistryFinder`, `importlib.abc.ResourceLoader`, `importlib.abc.SourceLoader.path_mtime()`.
- gh-127873 [https://github.com/python/cpython/issues/127873]: Quando `-E` estiver definido, ignora apenas `PYTHON_COLORS` e não

FORCE_COLOR/NO_COLOR/TERM ao colorir a saída. Patch por Hugo van Kemenade.

- gh-125413 [https://github.com/python/cpython/issues/125413]: Adiciona o atributo **pathlib.Path.info**, que armazena um objeto que implementa o protocolo **pathlib.types.PathInfo** (também novo). O objeto oferece suporte à consulta do tipo de arquivo e o armazenamento em cache interno dos resultados de **stat()**. Objetos de caminho gerados por **iterdir()** são inicializados com informações do tipo de arquivo obtidas a partir da varredura do diretório pai.
- gh-127712 [https://github.com/python/cpython/issues/127712]: Corrige o tratamento do argumento secure de **logging.handlers.SMTPHandler**.
- gh-127096 [https://github.com/python/cpython/issues/127096]: Não recria a seção sem nome em cada leitura em **configparser.ConfigParser**. Patch por Andrey Efremov.
- gh-124369 [https://github.com/python/cpython/issues/124369]: Descontinua pdb.Pdb.curframe_locals
- gh-126332 [https://github.com/python/cpython/issues/126332]: Corrige falha do _pyrepl ao dar dois CTRL-Z em uma linha que transborda.
- gh-125553 [https://github.com/python/cpython/issues/125553]: Corrige invariância de ida e volta para continuações de contrabarra em **tokenize.untokenize()**.
- gh-91048 [https://github.com/python/cpython/issues/91048]: Adiciona funções **asyncio.capture_call_graph()** e **asyncio.print_call_graph()**.
- gh-124703 [https://github.com/python/cpython/issues/124703]: Sair do **pdb** no modo inline emitirá um prompt de confirmação e sairá normalmente, em vez de exibir um traceback de exceção.
- gh-123987 [https://github.com/python/cpython/issues/123987]: Corrigido problema no NamespaceReader em que um item não-caminho em um caminho de espaço de nomes, como um sinalizador adicionado por um instalador editável, interrompia o carregamento de recursos.
- gh-119349 [https://github.com/python/cpython/issues/119349]: Adiciona a função **ctypes.util.dllist()** para listar as bibliotecas compartilhadas carregadas para o processo atual.
- gh-55454 [https://github.com/python/cpython/issues/55454]: Adiciona suporte a IMAP4 IDLE ao módulo **imaplib**. Patch por Forest.
- gh-119257 [https://github.com/python/cpython/issues/119257]: Exibe o menu de autopreenchimento com tecla tab abaixo da linha atual, o que resulta

em um comportamento menos instável e corrige um bug de movimento do cursor. Patch por Daniel Hollas

- gh-101410 [https://github.com/python/cpython/issues/101410]: Suporte a mensagens personalizadas para erros de domínio no módulo `math` (`math.sqrt()`, `math.log()` e `math.atanh()` foram modificados como exemplos). Patch por Charlie Zhao e Sergey B Kirpichev.
- gh-81340 [https://github.com/python/cpython/issues/81340]: Usa `os.copy_file_range()` nas funções `shutil.copy()`, `shutil.copy2()` e `shutil.copyfile()` por padrão. Uma chamada de sistema Linux subjacente oferece aos sistemas de arquivos a oportunidade de implementar o uso de cópia na gravação (no caso de btrfs e XFS) ou cópia do lado do servidor (no caso de NFS). Patch por Illia Volochii.
- bpo-27307 [https://bugs.python.org/issue?@action=redirect&bpo=27307]: Adiciona suporte a atributos e acesso a itens para `string.Formatter` no modo de numeração automática, o que permite strings de formato como '{.name}' e '{[1]}'.

IDLE

- gh-129873 [https://github.com/python/cpython/issues/129873]: Simplifica a exibição da documentação do IDLE copiando apenas a seção de texto de idle.html para idlelib/help.html. Patch por Stan Ulbrych.

Documentação

- gh-125722 [https://github.com/python/cpython/issues/125722]: Requer Sphinx 8.1.3 ou posterior para construir a documentação do Python. Patch por Adam Turner.
- gh-67206 [https://github.com/python/cpython/issues/67206]: Documenta que `string.printable` não é imprimível no sentido POSIX. Em particular, `string.printable.isprintable()` retorna `False`. Patch por Bénédikt Tran.

Núcleo e embutidos

- gh-100239 [https://github.com/python/cpython/issues/100239]: Substitui o opcode BINARY_SUBSCR e sua família por BINARY_OP com oparg

NB_SUBSCR.

- gh-129732 [https://github.com/python/cpython/issues/129732]: Corrigida uma corrida em `_Py_qsbr_reserve` na construção com threads livres.
- gh-129763 [https://github.com/python/cpython/issues/129763]: Remove a macro interna `LLTRACE` (use **`Py_DEBUG`** em vez disso).
- gh-129715 [https://github.com/python/cpython/issues/129715]: Melhora o desempenho JIT dos geradores.
- gh-129643 [https://github.com/python/cpython/issues/129643]: Corrige a segurança de thread de **`PyList_Insert()`** em construções com threads livres.
- gh-129668 [https://github.com/python/cpython/issues/129668]: Corrige condição de corrida ao levantar **`MemoryError`** na construção com threads livres.
- gh-129643 [https://github.com/python/cpython/issues/129643]: Corrige a segurança de thread de **`PyList_SetItem()`** em construções com threads livres. Patch por Kumar Aditya.
- gh-128563 [https://github.com/python/cpython/issues/128563]: Corrige um problema em que o recurso de depuração “lltrace” poderia ter sido habilitado incorretamente para alguns quadros.
- gh-129393 [https://github.com/python/cpython/issues/129393]: No FreeBSD, **`sys.platform`** não contém mais a versão principal. É sempre `'freebsd'`, em vez de `'freebsd13'` ou `'freebsd14'`.

Biblioteca

- gh-129345 [https://github.com/python/cpython/issues/129345]: Corrige desreferência de ponteiro nulo em **`syslog.openlog()`** quando um gancho de auditoria levanta uma exceção.

Núcleo e embutidos

- gh-129231 [https://github.com/python/cpython/issues/129231]: Melhora o layout de memória dos rastreamentos JIT. Patch por Diego Russo
- gh-129149 [https://github.com/python/cpython/issues/129149]: Adiciona um caminho rápido para inteiros de tamanho médio em **`PyLong_FromUnsignedLong()`**, **`PyLong_FromUnsignedLongLong()`** e **`PyLong_FromSize_t()`**.
- gh-129201 [https://github.com/python/cpython/issues/129201]: A versão com threads livres do coletor de lixo cíclico foi otimizada para usar

condicionalmente instruções de pré-busca da CPU durante a coleta. Isso pode reduzir o tempo de coleta, aumentando a probabilidade de os dados estarem no cache da CPU quando necessário. As instruções de pré-busca são habilitadas se o número de objetos de longa duração (objetos que sobrevivem a uma coleta completa) exceder um limite.

- gh-129093 [https://github.com/python/cpython/issues/129093]: Corrige f-strings como `f'{expr=}'` que às vezes não exibem a expressão completa quando a expressão contém `!=`.
- gh-124363 [https://github.com/python/cpython/issues/124363]: Trata expressões de depuração em f-string como strings brutas. Patch por Pablo Galindo
- gh-128714 [https://github.com/python/cpython/issues/128714]: Corrige as possíveis corridas nos métodos especiais get/set `__annotations__`, `__annotate__` e `__type_params__` para o objeto função e adiciona testes relacionados.
- gh-128799 [https://github.com/python/cpython/issues/128799]: Adiciona o quadro `except*` ao traceback quando ele encapsula uma exceção simples.
- gh-128842 [https://github.com/python/cpython/issues/128842]: Coleta estatísticas de memória JIT usando pystats. Patch por Diego Russo.
- gh-100239 [https://github.com/python/cpython/issues/100239]: Especializa `BINARY_OP` para operações lógicas bit a bit em inteiros compactos.
- gh-128910 [https://github.com/python/cpython/issues/128910]: As funções privadas, não documentadas e não utilizadas `_PyTrash_begin` e `_PyTrash_end` da API C foram removidas.
- gh-128807 [https://github.com/python/cpython/issues/128807]: Adiciona uma fase de marcação ao coletor de lixo com threads livres. Isso é semelhante ao que foi feito em gh-126491 [https://github.com/python/cpython/issues/126491]. Como o coletor de lixo com threads livres não possui gerações e não é incremental, a fase de marcação procura todos os objetos acessíveis a partir de raízes conhecidas. As raízes são objetos que se sabe não serem lixo, como o dicionário de módulos para **`sys`**. Para a maioria dos programas, essa fase de marcação deve tornar o coletor de lixo um pouco mais rápido, já que normalmente menos trabalho é feito por objeto.
- gh-100239 [https://github.com/python/cpython/issues/100239]: Adiciona o opcode `BINARY_OP_EXTEND` que executa um par de funções (funções de guarda e especialização) acessadas do cache inline.

- gh-128563 [https://github.com/python/cpython/issues/128563]: Um novo tipo de interpretador foi adicionado ao CPython. Este interpretador utiliza chamadas de cauda para seus manipuladores de instruções. Resultados preliminares de benchmark sugerem uma velocidade média geométrica de 7 a 11% no pyperformance (dependendo da plataforma) e até 30% mais rápida em cargas de trabalho com uso intensivo de Python. Atualmente, este interpretador funciona apenas em compiladores mais recentes, como o `clang-19`. Outros compiladores continuarão usando o interpretador antigo. Patch por Ken Jin, com ideias de como implementar isso no CPython por Mark Shannon, Garret Gu, Haoran Xu e Josh Haberman.
- gh-126703 [https://github.com/python/cpython/issues/126703]: Melhora o desempenho da iteração em listas e tuplas usando uma lista livre para os objetos iterador.
- gh-127953 [https://github.com/python/cpython/issues/127953]: O tempo para manipular um evento `LINE` em sys.monitoring (e sys.settrace) agora é independente do número de linhas no objeto de código.
- gh-128330 [https://github.com/python/cpython/issues/128330]: Restaura caracteres de controle do terminal na saída do REPL.
- gh-128016 [https://github.com/python/cpython/issues/128016]: A mensagem `SyntaxWarning` para sequências de escape inválidas foi aprimorada para esclarecer que tais sequências levantarão um `SyntaxError` em versões futuras do Python. A nova mensagem também sugere uma possível correção, ou seja, `Did you mean "\\e"?`.
- gh-126004 [https://github.com/python/cpython/issues/126004]: Corrige o tratamento dos valores **`UnicodeError.start`** e **`UnicodeError.end`** no tratador de erros **`codecs.replace_errors()`**. Patch por Bénédikt Tran.
- gh-126004 [https://github.com/python/cpython/issues/126004]: Corrige o tratamento dos valores **`UnicodeError.start`** e **`UnicodeError.end`** no tratador de erros **`codecs.backslashreplace_errors()`**. Patch por Bénédikt Tran.
- gh-126004 [https://github.com/python/cpython/issues/126004]: Corrige o tratamento dos valores **`UnicodeError.start`** e **`UnicodeError.end`** no tratador de erros **`codecs.xmlcharrefreplace_errors()`**. Patch por Bénédikt Tran.
- gh-127119 [https://github.com/python/cpython/issues/127119]: Otimiza ligeiramente o desalocador **`int`**.

- gh-127349 [https://github.com/python/cpython/issues/127349]: Corrigido o erro ao redimensionar o terminal no Python REPL. Patch por Semyon Moroz.
- gh-125723 [https://github.com/python/cpython/issues/125723]: Corrige falha com `gi_frame.f_locals` quando os frames do gerador duram mais que o gerador. Patch por Mikhail Efimov.

Biblioteca

- gh-126349 [https://github.com/python/cpython/issues/126349]: Adiciona os gerenciadores de contexto `turtle.fill()`, `turtle.poly()` e `turtle.no_animation()`. Patch por Marie Roald e Yngve Mardal Moe.

Núcleo e embutidos

- gh-115911 [https://github.com/python/cpython/issues/115911]: Se o diretório de trabalho atual não puder ser determinado devido às permissões, a importação não levantará mais `PermissionError`. Patch por Alex Willmer.

Biblioteca

- gh-112713 [https://github.com/python/cpython/issues/112713]: Adicionado suporte para o sinalizador de cookie `Partitioned` em `http.cookies`.

API C

- gh-129533 [https://github.com/python/cpython/issues/129533]: Atualiza `PyGC_Enable()`, `PyGC_Disable()`, `PyGC_IsEnabled()` para usar operação atômica para segurança de thread em construção com threads livres. Patch por Donghee Na.

- gh-89188 [https://github.com/python/cpython/issues/89188]: Implementa `PyUnicode_KIND()` e `PyUnicode_DATA()` como funções, além das macros com os mesmos nomes. As macros dependem de campos de bits C com layout específico do compilador. Patch por Victor Stinner.

- gh-91417 [https://github.com/python/cpython/issues/91417]: Remove `PySequence_Fast()` da API C limitada, pois esta função precisa ser usada com `PySequence_Fast_GET_ITEM`, que nunca funcionou na API C limitada. Patch por Victor Stinner.

- gh-128509 [https://github.com/python/cpython/issues/128509]: Adiciona `PyUnstable_IsImmortal()` para determinar se um objeto é imortal.

- gh-129033 [https://github.com/python/cpython/issues/129033]: Remove as funções privadas `_PyInterpreterState_GetConfigCopy()` e `_PyInterpreterState_SetConfig()`. Em vez disso, use `PyConfig_Get()` e `PyConfig_Set()`, API C pública adicionada por **PEP 741** [https://peps.python.org/pep-0741/] "API C de Configuração do Python". Patch por Victor Stinner.

- gh-129033 [https://github.com/python/cpython/issues/129033]: Remova a função privada `_Py_InitializeMain()`. Era uma API provisória adicionada ao Python 3.8 pela **PEP 587** [https://peps.python.org/pep-0587/]. Patch por Victor Stinner.

- gh-128844 [https://github.com/python/cpython/issues/128844]: Adiciona as APIs instáveis `PyUnstable_TryIncRef()` e `PyUnstable_EnableTryIncRef()`. Elas são auxiliares para lidar com referências sem dono de forma segura para thread, principalmente na versão com threads livres.

- gh-128911 [https://github.com/python/cpython/issues/128911]: Adiciona as funções auxiliares `PyImport_ImportModuleAttr()` e `PyImport_ImportModuleAttrString()` para importar um módulo e obter um atributo do módulo. Patch por Victor Stinner.

- gh-128863 [https://github.com/python/cpython/issues/128863]: As seguintes funções privadas estão descontinuado e planejadas para remoção no Python 3.18:

  - **`_PyBytes_Join()`**: use `PyBytes_Join()`.
  - **`_PyDict_GetItemStringWithError()`**: use `PyDict_GetItemStringRef()`.

- **`_PyDict_Pop()`**: use **`PyDict_Pop()`**.
- **`_PyLong_Sign()`**: use **`PyLong_GetSign()`**.
- **`_PyLong_FromDigits()`** e **`_PyLong_New()`**: use **`PyLongWriter_Create()`**.
- **`_PyThreadState_UncheckedGet()`**: use **`PyThreadState_GetUnchecked()`**.
- **`_PyUnicode_AsString()`**: use **`PyUnicode_AsUTF8()`**.
- **`_Py_HashPointer()`**: use **`Py_HashPointer()`**.
- **`_Py_fopen_obj()`**: use **`Py_fopen()`**.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas novas funções públicas no Python 3.13 e versões anteriores.

Patch por Victor Stinner.

- gh-126599 [https://github.com/python/cpython/issues/126599]: Remove algumas APIs de teste internas para o compilador JIT experimental.

- gh-127925 [https://github.com/python/cpython/issues/127925]: Converte o módulo **`decimal`** para usar a API C **PEP 757** [https://peps.python.org/pep-0757/] (exportação e importação de inteiros), oferecendo alguma aceleração se a parte inteira da instância **`Decimal`** for pequena. Patch por Sergey B Kirpichev.

Construção

- gh-129660 [https://github.com/python/cpython/issues/129660]: Remove `test_embed` do treinamento de PGO, cuja contribuição em versões recentes é considerada ignorável.
- gh-128902 [https://github.com/python/cpython/issues/128902]: Corrige erros de compilação com Clang 9 e versões anteriores devido à falta de suporte a `__attribute__((fallthrough))`.

Python 3.14.0 alpha 4

Data de lançamento: 2025-01-14

macOS

- gh-127592 [https://github.com/python/cpython/issues/127592]: O uso das APIs unificadas do Apple System Log foi desabilitado quando a versão mínima do macOS era anterior à 10.12.

Ferramentas/Demos

- gh-128152 [https://github.com/python/cpython/issues/128152]: Corrige um bug em que o analisador de pré-processador C da Argument Clinic tentava analisar diretivas de pré-processador dentro de comentários C. Patch por Erlend Aasland.

Testes

- gh-128690 [https://github.com/python/cpython/issues/128690]: Temporariamente, não use test_embed em construções de perfil PGO até que o problema com test_init_pyvenv_cfg falhando em algumas configurações seja resolvido.

Biblioteca

- gh-128731 [https://github.com/python/cpython/issues/128731]: Corrige `ResourceWarning` em `urllib.robotparser.RobotFileParser.read()`.
- gh-71339 [https://github.com/python/cpython/issues/71339]: Adiciona novos métodos de asserção para `unittest`: `assertHasAttr()`, `assertNotHasAttr()`, `assertIsSubclass()`, `assertNotIsSubclass()` `assertStartsWith()`, `assertNotStartsWith()`, `assertEndsWith()` e `assertNotEndsWith()`.
- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação de `pickle` em 25% removendo uma expressão regular desnecessária. Dessa forma, `re` não está mais implicitamente disponível como `pickle.re`. Patch por Bénédikt Tran.
- gh-128661 [https://github.com/python/cpython/issues/128661]: Corrige `typing.evaluate_forward_ref()` não mostrando descontinuação quando o argumento `type_params` não é passado.

- gh-128562 [https://github.com/python/cpython/issues/128562]: Corrige possíveis conflitos nos nomes dos widgets **`tkinter`** gerados se o nome da classe do widget terminar com um dígito.
- gh-128559 [https://github.com/python/cpython/issues/128559]: Tempo de importação melhorado de **`asyncio`**.
- gh-128552 [https://github.com/python/cpython/issues/128552]: Corrige lixo cíclico introduzido por **`asyncio.loop.create_task()`** e **`asyncio.TaskGroup.create_task()`** mantendo uma referência à tarefa criada se ela estiver ansiosa.
- gh-128340 [https://github.com/python/cpython/issues/128340]: Adiciona um identificador interno seguro para thread para ser usado em **`asyncio.loop.call_soon_threadsafe()`** para cancelamento seguro para thread.
- gh-128182 [https://github.com/python/cpython/issues/128182]: Corrige falha ao usar ponteiros **`ctypes`** simultaneamente na construção com threads livres.
- gh-128400 [https://github.com/python/cpython/issues/128400]: Exibe somente a thread atual em **`faulthandler`** na construção com threads livres para evitar corridas.
- gh-128400 [https://github.com/python/cpython/issues/128400]: Corrige falha ao usar **`faulthandler.dump_traceback()`** enquanto outras threads estiverem ativas na construção com threads livres.
- gh-128388 [https://github.com/python/cpython/issues/128388]: Corrige `PyREPL` no Windows para oferecer suporte a mais atalhos de teclado, como os atalhos de teclado para pular palavras `Control-←` e `Control-→` e aqueles com meta (isto é, `Alt`), por exemplo, `Alt-d` para `kill-word` ou `Alt-Backspace` para `backward-kill-word`.
- gh-88834 [https://github.com/python/cpython/issues/88834]: Unifica a verificação de instância para **`typing.Union`** e **`types.UnionType`**: **`Union`** agora usa as verificações de instância em relação aos seus parâmetros em vez das verificações de subclasse.
- gh-128302 [https://github.com/python/cpython/issues/128302]: Corrige **`xml.dom.xmlbuilder.DOMEntityResolver.resolveEntity()`**, que foi quebrado pela transição do Python 3.0.
- gh-128317 [https://github.com/python/cpython/issues/128317]: Realça today em cores na saída da CLI do **`calendar`**. Patch por Hugo van Kemenade.

- gh-128302 [https://github.com/python/cpython/issues/128302]: Permite que **`xml.dom.xmlbuilder.DOMParser.parse()`** manipule corretamente instâncias **`xml.dom.xmlbuilder.DOMInputSource`** que tenham apenas um atributo **`systemId`** definido.
- gh-128151 [https://github.com/python/cpython/issues/128151]: Melhoria de 30% na geração de objetos **`UUID`** nas versões 3, 4, 5 e 8 por meio de suas funções dedicadas. Patch por Bénédikt Tran.
- gh-128118 [https://github.com/python/cpython/issues/128118]: Melhora o desempenho de **`copy.copy()`** em 30% por meio de um caminho rápido para tipos atômicos e tipos de contêiner.
- gh-127946 [https://github.com/python/cpython/issues/127946]: Corrige falha ao modificar objetos **`ctypes. CFuncPtr`** simultaneamente na construção com threads livres.
- gh-128062 [https://github.com/python/cpython/issues/128062]: Reverte a fonte da barra de menu do **`turtledemo`** para seu valor padrão e exibe as teclas de atalho na posição correta.
- gh-128014 [https://github.com/python/cpython/issues/128014]: Corrige a redefinição do ícone da janela padrão passando `default=''` para o método **`wm_iconbitmap()`** do **`tkinter`**.
- gh-41872 [https://github.com/python/cpython/issues/41872]: Corrige extração rápida de docstrings de módulo de um arquivo em **`pydoc`**. Agora ele oferece suporte a docstrings com aspas simples, sequências de escape, literais de strings brutas e outras sintaxes Python.
- gh-127060 [https://github.com/python/cpython/issues/127060]: Define a variável de ambiente TERM como “dumb” para desabilitar cores de traceback no IDLE, já que o IDLE não entende sequências de escape ANSI. Patch por Victor Stinner.
- gh-126742 [https://github.com/python/cpython/issues/126742]: Corrige o suporte de mensagens de erro localizadas relatadas por *dlerror(3)* *[https://manpages.debian.org/dlerror(3)]* e *gdbm_strerror* *[https://manpages.debian.org/gdbm(3)]* nas funções **`ctypes`** e **`dbm.gnu`** respectivamente. Patch por Bénédikt Tran.
- gh-122548 [https://github.com/python/cpython/issues/122548]: Adiciona dois novos eventos locais ao sys.monitoring, `BRANCH_LEFT` e `BRANCH_RIGHT`. Isso permite que os dois braços de ramificação sejam desabilitados independentemente, o que deve melhorar significativamente o

desempenho das ferramentas de cobertura em nível de ramificação. O antigo evento de ramificação, `BRANCH`, agora está descontinuado.

- gh-127847 [https://github.com/python/cpython/issues/127847]: Corrige a posição ao fazer buscas e leituras intercaladas em arquivos zip não compactados e não criptografados retornados por **`zipfile.ZipFile.open()`**.
- gh-127688 [https://github.com/python/cpython/issues/127688]: Adiciona as constantes **`SCHED_DEADLINE`** e **`SCHED_NORMAL`** ao módulo **`os`**.
- gh-83662 [https://github.com/python/cpython/issues/83662]: Adiciona o método `__class_getitem__` ausente à implementação Python de **`functools.partial()`**, para torná-lo compatível com a versão C. Isso é relevante principalmente para implementações Python alternativas como PyPy e GraalPy, porque o CPython geralmente usará a implementação C dessa função.
- gh-127586 [https://github.com/python/cpython/issues/127586]: **`multiprocessing.pool.Pool`** agora restaura corretamente os manipuladores de sinais bloqueados da thread mãe ao criar processos via *spawn* ou *forkserver*.
- gh-98188 [https://github.com/python/cpython/issues/98188]: Corrige um problema em **`email.message.Message.get_payload()`** onde os dados não podem ser decodificados se o mecanismo de Codificação de Transferência de Conteúdo contiver espaços em branco à direita ou texto lixo adicional. Patch por Hui Liu.
- gh-127529 [https://github.com/python/cpython/issues/127529]: Corrige o comportamento de **`asyncio.selector_events.BaseSelectorEventLoop._accept_connection()`** ao lidar com **`ConnectionAbortedError`** em um laço. Isso melhora o desempenho no OpenBSD.
- gh-127360 [https://github.com/python/cpython/issues/127360]: Quando uma mensagem de erro descritiva não pode ser fornecida para um **`ssl.SSLError`**, a mensagem “erro desconhecido” agora mostra o código de erro interno (conforme recuperado por `ERR_get_error` e funções OpenSSL semelhantes).
- gh-127196 [https://github.com/python/cpython/issues/127196]: Corrige falha quando um dicionário com chaves em codificação inválida era passado para diversas funções no módulo `_interpreters`.

- gh-124130 [https://github.com/python/cpython/issues/124130]: Corrige um bug na correspondência da expressão regular `\B` em uma string de entrada vazia. Agora é sempre o oposto de `\b`. Para obter um comportamento antigo, use `(?!\A\Z)\B`. Para obter um novo comportamento em versões antigas do Python, use `(?!\b)`.
- gh-126639 [https://github.com/python/cpython/issues/126639]: **`tempfile.NamedTemporaryFile`** agora emitirá um **`ResourceWarning`** quando for finalizado pelo coletor de lixo sem ser explicitamente fechado.
- gh-126624 [https://github.com/python/cpython/issues/126624]: Expõe o código de erro **`XML_ERROR_NOT_STARTED`** do Expat >=2.6.4 em **`xml.parsers.expat.errors`**.
- gh-126225 [https://github.com/python/cpython/issues/126225]: **`getopt`** e **`optparse`** não estão mais marcados como descontinuados. Há razões legítimas para usar um desses módulos em preferência a **`argparse`**, e nenhum desses módulos corre o risco de ser removido da biblioteca padrão. Dos três, `argparse` continua sendo a escolha padrão recomendada, *a menos que* uma das preocupações observadas no topo da documentação do módulo `optparse` se aplique.
- gh-124761 [https://github.com/python/cpython/issues/124761]: Adiciona a constante **`SO_REUSEPORT_LB`** ao **`socket`** para FreeBSD.
- gh-121720 [https://github.com/python/cpython/issues/121720]: **`enum.EnumDict`** agora pode ser usada sem recorrer à API privada.
- gh-123424 [https://github.com/python/cpython/issues/123424]: Adicionada **`zipfile.ZipInfo._for_archive()`** definindo propriedades padrão em objetos **`ZipInfo`**. Patch por Bénédikt Tran e Jason R. Coombs.
- gh-121676 [https://github.com/python/cpython/issues/121676]: Descontinua a chamada da implementação Python de **`functools.reduce()`** com uma `function` ou `sequence` como um argumento nomeado. Isso será proibido no Python 3.16 para corresponder à implementação em C.
- gh-112015 [https://github.com/python/cpython/issues/112015]: **`ctypes.memoryview_at()`** agora existe para criar um objeto **`memoryview`** que se refere ao ponteiro e comprimento fornecidos. Isso funciona como **`ctypes.string_at()`**, exceto que evita uma cópia de buffer e é normalmente útil ao implementar funções de retorno de chamada Python puras que recebem buffers de tamanho dinâmico.

- gh-95371 [https://github.com/python/cpython/issues/95371]: Adicionado suporte para outros formatos de imagem (PNG, PGM e PPM) ao módulo turtle. Patch por Shin-myoung-serp.

Núcleo e embutidos

- gh-128078 [https://github.com/python/cpython/issues/128078]: Corrige um `SystemError` ao usar `anext()` com um valor de tupla padrão. Patch por Bénédikt Tran.
- gh-128717 [https://github.com/python/cpython/issues/128717]: Corrige uma falha ao definir o limite de recursão enquanto outros threads estão ativos na construção com threads livres.
- gh-124483 [https://github.com/python/cpython/issues/124483]: Trata `Py_DECREF` e variantes como escape ao gerar opcode e metadados UOP. Isso evita a possibilidade de um método `__del__` causar comportamento incorreto do JIT.
- gh-126703 [https://github.com/python/cpython/issues/126703]: Melhora o desempenho dos métodos de classe usando uma freelist.
- gh-128137 [https://github.com/python/cpython/issues/128137]: Atualização do layout `PyASCIIObject` para manipular campos internos com a operação atômica. Patch por Donghee Na.

Biblioteca

- gh-128192 [https://github.com/python/cpython/issues/128192]: Atualiza o algoritmo de autenticação de resumo HTTP para `urllib.request` dando suporte à autenticação de resumo SHA-256 conforme especificado em **RFC 7616** [https://datatracker.ietf.org/doc/html/rfc7616.html].

Núcleo e embutidos

- gh-126868 [https://github.com/python/cpython/issues/126868]: Aumenta o uso de freelist para alocação de `int`.

- gh-114203 [https://github.com/python/cpython/issues/114203]: Otimiza `Py_BEGIN_CRITICAL_SECTION` para chamadas recursivas simples.

- gh-127705 [https://github.com/python/cpython/issues/127705]: Adiciona depuração de stackref quando `Py_STACKREF_DEBUG` está definido. Encontra todos os fechamentos duplos e vazamentos, registrando a origem e o último empréstimo.

  Inspirado no modo de depuração do HPy. https://docs.hpyproject.org/en/latest/debug-mode.html

- gh-128079 [https://github.com/python/cpython/issues/128079]: Corrige um bug em que **`except*`** não verifica corretamente o valor de retorno de uma função **`split()`** de **`ExceptionGroup`**, levando a uma falha em alguns casos. Agora, quando **`split()`** retorna um objeto inválido, **`except*`** levanta **`TypeError`** com o objeto **`ExceptionGroup`** levantado originalmente encadeado a ele.

- gh-128030 [https://github.com/python/cpython/issues/128030]: Evita erro ao chamar `PyModule_GetFilenameObject` em um objeto que não é módulo ao importar um símbolo inexistente de um objeto que não é módulo.

Biblioteca

- gh-128035 [https://github.com/python/cpython/issues/128035]: Indica através de **`ssl.HAS_PHA`** se o módulo **`ssl`** oferece suporte a autenticação de cliente pós-handshake (PHA) TLSv1.3. Patch por Will Childs-Klein.

Núcleo e embutidos

- gh-127274 [https://github.com/python/cpython/issues/127274]: Adiciona um novo sinalizador, `CO_METHOD`, a **`co_flags`** que indica se o objeto código pertence a uma função definida no escopo da classe.
- gh-66409 [https://github.com/python/cpython/issues/66409]: Durante a inicialização do caminho, agora verificamos se `base_exec_prefix` é o mesmo que `base_prefix` antes de retornar à pesquisa no diretório do interpretador Python.
- gh-127970 [https://github.com/python/cpython/issues/127970]: Agora usamos o local da biblioteca de tempo de execução `libpython` usada no processo

atual para determinar `sys.base_prefix` em todas as plataformas que implementam a função dladdr [https://pubs.opengroup.org/onlinepubs/9799919799/functions/dladdr.html] definida pelo padrão UNIX — isso inclui Linux, Android, macOS, iOS, FreeBSD, etc. Isso já acontecia nas construções do Windows e do macOS Framework.

- gh-127773 [https://github.com/python/cpython/issues/127773]: Não usa o cache de atributos de tipo para tipos com uma MRO incompatível.
- gh-127903 [https://github.com/python/cpython/issues/127903]: `Objects/unicodeobject.c`: corrige uma falha em compilações de depuração em `_copy_characters` quando não há nada para copiar.
- gh-127809 [https://github.com/python/cpython/issues/127809]: Corrige um problema em que o JIT experimental pode inferir um tipo de resultado incorreto para exponenciação (`**` e `**=`), levando a bugs ou travamentos.
- gh-126862 [https://github.com/python/cpython/issues/126862]: Corrige um possível estouro quando uma classe herda de um número absurdo de superclasses. Reportado por Valery Fedorenko. Patch por Bénédikt Tran.

API C

- gh-128400 [https://github.com/python/cpython/issues/128400]: `Py_FatalError()` não mostra mais todos os threads na construção com threads livres para evitar travamentos.
- gh-128629 [https://github.com/python/cpython/issues/128629]: Adiciona macros `Py_PACK_VERSION()` e `Py_PACK_FULL_VERSION()` para empacotamento de bits de números de versão do Python.
- gh-128008 [https://github.com/python/cpython/issues/128008]: Adiciona a função `PyWeakref_IsDead()`, que testa se uma referência fraca está morta.
- gh-127350 [https://github.com/python/cpython/issues/127350]: Adiciona a função `Py_fopen()` para abrir um arquivo. Semelhante à função `fopen()`, mas o parâmetro *path* é um objeto Python e uma exceção é definida em caso de erro. Adiciona também a função `Py_fclose()` para fechar um arquivo, função necessária para suporte ao Windows. Patch por Victor Stinner.

Construção

- gh-128627 [https://github.com/python/cpython/issues/128627]: Para construções do Emscripten, a chamada de conversão do ponteiro de função trampoline agora usa a instrução wasm-gc ref.test se estiver disponível em vez da reflexão do tipo Wasm JS.
- gh-128472 [https://github.com/python/cpython/issues/128472]: Ignora a otimização BOLT de funções usando gotos computados, corrigindo erros na compilação com LLVM 19.
- gh-115765 [https://github.com/python/cpython/issues/115765]: O GNU Autoconf 2.72 agora é necessário para gerar o `configure`. Patch de Erlend Aasland.
- gh-123925 [https://github.com/python/cpython/issues/123925]: Corrige a construção do módulo **`curses`** em plataformas com libncurses, mas sem libncursesw.
- gh-90905 [https://github.com/python/cpython/issues/90905]: Adiciona suporte para compilação cruzada para x86_64 no macOS aarch64/arm64.
- gh-128321 [https://github.com/python/cpython/issues/128321]: Define `LIBS` em vez de `LDFLAGS` ao verificar se as funções de biblioteca **`sqlite3`** estão disponíveis. Isso corrige a ordenação de bibliotecas vinculadas durante as verificações, que era incorreta ao usar uma `libsqlite3` vinculada estaticamente.
- gh-100384 [https://github.com/python/cpython/issues/100384]: Erro em `unguarded-availability` em construções do macOS, impedindo o uso inválido de símbolos que não estão disponíveis em versões mais antigas do sistema operacional.
- gh-128104 [https://github.com/python/cpython/issues/128104]: Remoção das condições `Py_STRFTIME_C99_SUPPORT` em favor da exigência de suporte ao especificador C99 *strftime(3)* *[https://manpages.debian.org/strftime(3)]* em tempo de construção. Na compilação cruzada, não há verificação em tempo de construção e o suporte é presumido.
- gh-127951 [https://github.com/python/cpython/issues/127951]: Adiciona a opção `--pystats` à construção do Windows para habilitar a coleta de estatísticas de desempenho.

Python 3.14.0 alpha 3

Data de lançamento: 2024-12-17

Windows

- gh-127353 [https://github.com/python/cpython/issues/127353]: Permite forçar a saída de cores no Windows usando variáveis de ambiente. Patch por Andrey Efremov.
- gh-125729 [https://github.com/python/cpython/issues/125729]: Torna a presença do módulo `turtle` dependente da opção do instalador Tcl/Tk. Anteriormente, o módulo era sempre instalado, mas seria inutilizável sem o Tcl/Tk.

Ferramentas/Demos

- gh-126700 [https://github.com/python/cpython/issues/126700]: Adiciona suporte para funções multi-argumentos `gettext` em **pygettext.py**.

Testes

- gh-127906 [https://github.com/python/cpython/issues/127906]: Testa a API C limitada em test_cppext. Patch por Victor Stinner.
- gh-127637 [https://github.com/python/cpython/issues/127637]: Adiciona testes para a interface de linha de comando `dis`. Patch por Bénédikt Tran.
- gh-126925 [https://github.com/python/cpython/issues/126925]: Os resultados dos testes do iOS agora são transmitidos durante a execução do teste, e o xcresulttool descontinuado não é mais usado.
- gh-127076 [https://github.com/python/cpython/issues/127076]: Desabilita testes de chamada de sistema baseados em strace quando LD_PRELOAD estiver definido.
- gh-127076 [https://github.com/python/cpython/issues/127076]: Filtra chamadas `mmap`, `munmap` e `mprotect` relacionadas à memória daquelas relacionadas ao arquivo ao testar o comportamento de `io` usando strace.

Segurança

- gh-127655 [https://github.com/python/cpython/issues/127655]: Corrigido o transporte de **`asyncio.selector_events._SelectorSocketTransport`** não pausando escritas para o protocolo quando o buffer atinge a marca d'água alta ao usar **`asyncio.WriteTransport.writelines()`**.

Biblioteca

- gh-126907 [https://github.com/python/cpython/issues/126907]: Corrige falha ao usar **`atexit`** simultaneamente na construção com threads livres.

- gh-127870 [https://github.com/python/cpython/issues/127870]: Detecta chamadas recursivas no tratamento de ctypes `_as_parameter_`. Patch por Victor Stinner.

- gh-127732 [https://github.com/python/cpython/issues/127732]: O módulo **`platform`** agora detecta corretamente o Windows Server 2025.

- gh-126789 [https://github.com/python/cpython/issues/126789]: Corrigidas **`sysconfig.get_config_vars()`**, **`sysconfig.get_paths()`** e similares, que retornavam dados em cache desatualizados caso o valor de **`sys.prefix`** ou **`sys.exec_prefix`** fosse alterado. A substituição de **`sys.prefix`** ou **`sys.exec_prefix`** ainda é desencorajada, pois isso poderia danificar outras partes do código.

- gh-127718 [https://github.com/python/cpython/issues/127718]: Adiciona cor à saída de ajuda do **`test.regrtest`**. Patch por Hugo van Kemenade.

- gh-127610 [https://github.com/python/cpython/issues/127610]: Adicionada validação para mais de um parâmetro var-posicional ou var-keyword em **`inspect.Signature`**. Patch por Maxim Ageev.

- gh-127627 [https://github.com/python/cpython/issues/127627]: Adicionado `posix._emscripten_debugger()` para ajudar na depuração do conjunto de testes no alvo Emscripten.

- gh-126821 [https://github.com/python/cpython/issues/126821]: As aplicações de macOS e iOS agora podem escolher redirecionar stdout e stderr para o log do sistema durante a configuração do interpretador.

- gh-93312 [https://github.com/python/cpython/issues/93312]: Inclui `<sys/pidfd.h>` para obter a constante `os.PIDFD_NONBLOCK`. Patch por Victor Stinner.

- gh-127481 [https://github.com/python/cpython/issues/127481]: Adiciona a constante `EPOLLWAKEUP` ao módulo **`select`**.

- gh-127065 [https://github.com/python/cpython/issues/127065]: Torna **`operator.methodcaller()`** seguro para thread e seguro para reentrância.

- gh-127321 [https://github.com/python/cpython/issues/127321]: **`pdb.set_trace()`** não parará em um opcode que não tenha mais um número de linha associado.

- gh-127429 [https://github.com/python/cpython/issues/127429]: Corrigido um bug em que, em construções cruzadas, os dados POSIX **`sysconfig`** estavam sendo gerados com o `Makefile` do Python host. Os dados agora são gerados a partir do `Makefile` da construção atual.

- gh-127413 [https://github.com/python/cpython/issues/127413]: Adiciona a opção de linha de comando **`dis --specialized`** para mostrar o bytecode especializado. Patch por Bénédikt Tran.

- gh-125413 [https://github.com/python/cpython/issues/125413]: Reverte a adição de **`pathlib.Path.scandir()`**. Este método foi adicionado na versão 3.14.0a2. As otimizações permanecem para caminhos do sistema de arquivos, mas outras subclasses devem implementar apenas **`pathlib.Path.iterdir()`**.

- gh-127257 [https://github.com/python/cpython/issues/127257]: Em **`ssl`**, falhas de chamada de sistema que o OpenSSL relata usando `ERR_LIB_SYS` agora são levantadas como **`OSError`**.

- gh-59705 [https://github.com/python/cpython/issues/59705]: No Linux, **`threading.Thread`** agora define o nome da thread para o sistema operacional. Patch por Victor Stinner.

- gh-127303 [https://github.com/python/cpython/issues/127303]: Expõe publicamente **`EXACT_TOKEN_TYPES`** em **`token.__all__`**.

- gh-127331 [https://github.com/python/cpython/issues/127331]: **`ssl`** pode mostrar descrições de erros adicionados no OpenSSL 3.4.

- gh-123967 [https://github.com/python/cpython/issues/123967]: Corrige o manipulador de falhas para quadros de trampolim. Se o quadro mais alto for um quadro de trampolim, pule-o. Patch por Victor Stinner.

- gh-127178 [https://github.com/python/cpython/issues/127178]: Um arquivo `_sysconfig_vars_(...).json` agora está disponível no diretório da biblioteca padrão. Ele contém a saída de **`sysconfig.get_config_vars()`** no ambiente padrão, codificada como dados JSON. Este é um detalhe de implementação e pode mudar a qualquer momento.

- gh-127072 [https://github.com/python/cpython/issues/127072]: Remove constantes `socket.NETLINK_*` desatualizadas que não estão presentes nos kernels Linux posteriores à versão 2.6.17.

- gh-127255 [https://github.com/python/cpython/issues/127255]: A função **`CopyComPointer()`** agora é pública. Anteriormente, era privada e estava disponível apenas em `_ctypes`.

- gh-127182 [https://github.com/python/cpython/issues/127182]: Corrige o travamento de **`io.StringIO.__setstate__()`**, quando **`None`** foi passado como o primeiro valor.

- gh-127217 [https://github.com/python/cpython/issues/127217]: Corrige **`urllib.request.pathname2url()`** para caminhos começando com várias barras no Posix.

- gh-125866 [https://github.com/python/cpython/issues/125866]: `urllib.request.pathname2url()` agora adiciona uma autoridade vazia ao gerar uma URL para um caminho que começa com exatamente uma barra. Por exemplo, o caminho `/etc/hosts` é convertido para a URL sem esquema `///etc/hosts`. Como resultado dessa alteração, URLs sem autoridades são geradas apenas para caminhos relativos.

- gh-127221 [https://github.com/python/cpython/issues/127221]: Adiciona cor à saída do `unittest`. Patch por Hugo van Kemenade.

- gh-127035 [https://github.com/python/cpython/issues/127035]: Corrige `shutil.which` no Windows. Agora ele analisa a correspondência direta se e somente se o comando termina com uma extensão PATHEXT ou X_OK não está no modo. Suporta arquivos sem extensão se "." estiver em PATHEXT. Suporta extensões PATHEXT que terminam com um ponto.

- gh-122273 [https://github.com/python/cpython/issues/122273]: Suporte ao histórico de PyREPL no Windows. Patch por devdanzin e Victor Stinner.

- gh-125866 [https://github.com/python/cpython/issues/125866]: `urllib.request.pathname2url()` e `url2pathname()` não convertem mais letras de unidade do Windows para maiúsculas.

- gh-127078 [https://github.com/python/cpython/issues/127078]: Corrige o problema em que `urllib.request.url2pathname()` falhava ao descartar uma barra extra antes de uma unidade UNC no caminho da URL no Windows.

- gh-126766 [https://github.com/python/cpython/issues/126766]: Corrige o problema em que `urllib.request.url2pathname()` falhava ao descartar qualquer autoridade 'localhost' presente na URL.

- gh-127065 [https://github.com/python/cpython/issues/127065]: Corrige o travamento ao chamar uma instância `operator.methodcaller()` de vários threads na construção com threads livres.

- gh-127090 [https://github.com/python/cpython/issues/127090]: Corrigimos o valor de `urllib.response.addinfourl.url` para URLs `file:` que expressam caminhos relativos e absolutos do Windows. A URL canônica gerada por `urllib.request.pathname2url()` agora é usada.

- gh-126992 [https://github.com/python/cpython/issues/126992]: Corrige os opcodes LONG e INT para usar apenas a base 10 na conversão de string para inteiro em `pickle`.

- gh-126997 [https://github.com/python/cpython/issues/126997]: Corrige o suporte de opcodes STRING e GLOBAL com argumentos não ASCII em `pickletools`. `pickletools.dis()` agora emite bytes não ASCII em argumentos STRING, BINSTRING e SHORT_BINSTRING como escape (`\\xXX`).

- gh-126316 [https://github.com/python/cpython/issues/126316]: `grp`: Torna `grp.getgrall()` seguro para thread adicionando um mutex. Patch por Victor Stinner.

- gh-126618 [https://github.com/python/cpython/issues/126618]: Corrige a representação de objetos `itertools.count` quando o valor da contagem é `sys.maxsize`.

- gh-126615 [https://github.com/python/cpython/issues/126615]: A exceção `COMError` agora é pública. Anteriormente, ela era privada e estava disponível apenas em `_ctypes`.

- gh-126985 [https://github.com/python/cpython/issues/126985]: Ao executar em um ambiente virtual com o `site` desativado (consulte `-S`), `sys.prefix` e `sys.base_prefix` agora apontarão para o ambiente virtual, em vez da instalação base.

- gh-112192 [https://github.com/python/cpython/issues/112192]: No módulo `trace`, aumenta a precisão da cobertura (`cov%`) para uma casa decimal.

- gh-118761 [https://github.com/python/cpython/issues/118761]: Melhora o tempo de importação do `mimetypes` em cerca de 11 a 16 vezes. Patch por Hugo van Kemenade.

- gh-126947 [https://github.com/python/cpython/issues/126947]: Levanta um erro **`TypeError`** em **`_pydatetime.timedelta.__new__()`** se os argumentos passados não forem **`int`** ou **`float`**, para que a implementação em Python esteja em conformidade com a implementação em C.

- gh-126946 [https://github.com/python/cpython/issues/126946]: Melhora a mensagem de erro **`GetoptError`** quando um prefixo de opção longo corresponder a várias opções aceitas em **`getopt.getopt()`** e **`getopt.gnu_getopt()`**.

- gh-126899 [https://github.com/python/cpython/issues/126899]: Faz com que os métodos do widget tkinter **`after()`** e **`after_idle()`** aceitem argumentos passados como nomeados.

- gh-85168 [https://github.com/python/cpython/issues/85168]: Corrige problema em que **`urllib.request.url2pathname()`** e **`pathname2url()`** sempre usavam UTF-8 ao citar e descitar URIs de arquivo. Agora eles usam tratador de erros e codificação do sistema de arquivos.

- gh-126780 [https://github.com/python/cpython/issues/126780]: Corrige **`os.path.normpath()`** para caminhos relativos à unidade no Windows.

- gh-126775 [https://github.com/python/cpython/issues/126775]: Torna **`linecache.checkcache()`** seguro para thread e seguro para reentrada do GC.

- Corrige problema em que **`urllib.request.pathname2url()`** levantava **`OSError`** ao receber um caminho do Windows contendo um caractere de dois pontos que não seguia uma letra de unidade, como antes de um fluxo de dados alternativo NTFS.

- gh-126727 [https://github.com/python/cpython/issues/126727]: `locale.nl_langinfo(locale.ERA)` agora retorna vários segmentos de descrição de era separados por ponto e vírgula. Anteriormente, ele retornava apenas o primeiro segmento em plataformas com Glibc.

- gh-85046 [https://github.com/python/cpython/issues/85046]: Adiciona o código de erro **`EHWPOISON`** para **`errno`**.

- gh-118201 [https://github.com/python/cpython/issues/118201]: Falhas intermitentes corrigidas de `os.confstr`, `os.pathconf` e `os.sysconf` no iOS e no Android.

- gh-86463 [https://github.com/python/cpython/issues/86463]: O parâmetro `usage` de `argparse.ArgumentParser` não afeta mais o valor padrão do parâmetro `prog` em subanalisadores.

- gh-124008 [https://github.com/python/cpython/issues/124008]: Corrige possível travamento (na construção de depuração), saída incorreta ou retorno de valor incorreto do binário bruto `write()` ao gravar no console no Windows.

- gh-123401 [https://github.com/python/cpython/issues/123401]: O módulo `http.cookies` agora suporta análise de formatos de data obsoletos de **RFC 850** [https://datatracker.ietf.org/doc/html/rfc850.html], de acordo com os requisitos do **RFC 9110** [https://datatracker.ietf.org/doc/html/rfc9110.html]. Patch por Nano Zheng.

- gh-122431 [https://github.com/python/cpython/issues/122431]: `readline.append_history_file()` agora levanta `ValueError` quando recebe um valor negativo.

- gh-122356 [https://github.com/python/cpython/issues/122356]: Garante que a posição de um objeto arquivo ou similar passado para `zipfile.is_zipfile()` permaneça inalterada após a chamada. Patch por Bénédikt Tran.

- gh-122288 [https://github.com/python/cpython/issues/122288]: Melhora o desempenho de `fnmatch.translate()` em 1,7 vezes. Patch por Bénédikt Tran.

- gh-88110 [https://github.com/python/cpython/issues/88110]: Corrigido `multiprocessing.Process` relatando um `.exitcode` de 1 mesmo em caso de sucesso ao usar o método de início `"fork"` ao usar um `concurrent.futures.ThreadPoolExecutor`.

- gh-97514 [https://github.com/python/cpython/issues/97514]: Foi adicionada autenticação ao socket de controle do método de inicialização do forkserver do mod **`multiprocessing`**, de forma que apenas processos com a chave de autenticação gerada pelo processo que criou o forkserver possam controlá-lo. Isso representa uma melhoria em relação às outras correções do gh-97514 [https://github.com/python/cpython/issues/97514], de modo que o acesso não seja mais limitado apenas pelas permissões do sistema de arquivos.

  A troca de descritores de arquivo dos encadeamento de controle com o processo de trabalho bifurcado agora exige o envio explícito de um byte de confirmação pelo socket após a troca em todas as plataformas que suportam o forkserver. Isso facilita bastante o teste do procedimento descrito acima.

Documentação

- gh-127347 [https://github.com/python/cpython/issues/127347]: Expõe publicamente **`traceback.print_list()`** em **`traceback.__all__`**.

Núcleo e embutidos

- gh-127740 [https://github.com/python/cpython/issues/127740]: Corrige a mensagem de erro em **`bytes.fromhex()`** quando é dado um número ímpar de dígitos para indicar corretamente que é necessário um número par de dígitos hexadecimais.
- gh-127058 [https://github.com/python/cpython/issues/127058]: `PySequence_Tuple` agora cria a tupla resultante atomicamente, evitando que tuplas parcialmente criados sejam visíveis para o coletor de lixo ou através de `gc.get_referrers()`.
- gh-127599 [https://github.com/python/cpython/issues/127599]: Corrige estatísticas para incrementos de contagens de referências de objetos (em particular, quando uma contagem de referências foi aumentada em mais de 1 em uma única operação).
- gh-127651 [https://github.com/python/cpython/issues/127651]: Ao levantar **`ImportError`** para símbolos ausentes em importações `from`, use `__file__` na mensagem de erro se `__spec__.origin` não for um local

- gh-127582 [https://github.com/python/cpython/issues/127582]: Corrige ressurreição de objeto não seguro para thread ao chamar finalizadores e retornos de chamada do observador na construção com threads livres.
- gh-127434 [https://github.com/python/cpython/issues/127434]: Os shims do compilador iOS agora podem aceitar argumentos com espaços.
- gh-127536 [https://github.com/python/cpython/issues/127536]: Adiciona travas faltando em torno de algumas operações de atribuição de lista na construção com threads livres.
- gh-127085 [https://github.com/python/cpython/issues/127085]: Corrigido problema de condição de corrida ao exportar um buffer de um objeto `memoryview` na construção com threads livres.
- gh-127238 [https://github.com/python/cpython/issues/127238]: Corrige mensagem de erro de `sys.set_int_max_str_digits()`.
- gh-113841 [https://github.com/python/cpython/issues/113841]: Corrige possível comportamento indefinido de divisão por zero em `complex`'s `_Py_c_pow()`.

Biblioteca

- gh-127133 [https://github.com/python/cpython/issues/127133]: Chamar `argparse.ArgumentParser.add_argument_group()` em um grupo de argumentos, e chamar os métodos `argparse.ArgumentParser.add_argument_group()` ou `argparse.ArgumentParser.add_mutually_exclusive_group()` em um grupo mutuamente exclusivo, agora levanta exceções. Esse aninhamento nunca foi suportado, frequentemente falhava e era exposto involuntariamente por meio de herança. Essa funcionalidade foi descontinuada desde o Python 3.11.

Núcleo e embutidos

- gh-126491 [https://github.com/python/cpython/issues/126491]: Adiciona uma fase de marcação ao coletor de lixo (GC). Todos os objetos que podem ser alcançados transitivamente a partir de módulos embutidos ou da pilha são marcados como alcançáveis antes da detecção de ciclos. Isso reduz a quantidade de trabalho realizada pelo GC em aproximadamente metade.

- gh-127020 [https://github.com/python/cpython/issues/127020]: Corrige uma falha na construção com threads livres quando `PyCode_GetCode()`, `PyCode_GetVarnames()`, `PyCode_GetCellvars()` ou `PyCode_GetFreevars()` eram chamados de várias threads ao mesmo tempo.
- gh-127010 [https://github.com/python/cpython/issues/127010]: Simplifica o rastreamento de dicionários pelo coletor de lixo. Todos os dicionários são rastreados no momento da criação, em vez de serem rastreados de forma preguiçosa quando um objeto rastreável é adicionado a eles. Isso simplifica consideravelmente o código e resulta em um pequeno ganho de velocidade.
- gh-126980 [https://github.com/python/cpython/issues/126980]: Corrige `__buffer__()` de `bytearray` travando quando `READ` ou `WRITE` são passados como sinalizadores.
- gh-126937 [https://github.com/python/cpython/issues/126937]: Corrigida `TypeError` que ocorria quando uma `ctypes.Structure` tinha um tamanho de campo que não cabia em um inteiro sem sinal de 16 bits. Em vez disso, o número máximo de bits era definido como `sys.maxsize`.
- gh-126868 [https://github.com/python/cpython/issues/126868]: Aumenta o desempenho de `int` adicionando uma lista livre para inteiros compactos.
- gh-126881 [https://github.com/python/cpython/issues/126881]: Corrige falha na finalização do estado dtoa. Patch por Kumar Aditya.
- gh-126892 [https://github.com/python/cpython/issues/126892]: Exige que o código frio ou inválido “aqueça” antes de ser compilado JIT novamente.
- gh-126091 [https://github.com/python/cpython/issues/126091]: Garante que os stacks trace estejam completos ao lançar em uma cadeia de geradores que termina em um gerador personalizado.
- gh-126024 [https://github.com/python/cpython/issues/126024]: Otimiza a decodificação de sequências curtas de UTF-8 contendo caracteres não ASCII em aproximadamente 15%.
- gh-125420 [https://github.com/python/cpython/issues/125420]: Adiciona o `memoryview.index()` aos objetos `memoryview`. Patch por Bénédikt Tran.
- gh-125420 [https://github.com/python/cpython/issues/125420]: Adiciona o `memoryview.count()` aos objetos `memoryview`. Patch por Bénédikt

Tran.

- gh-124470 [https://github.com/python/cpython/issues/124470]: Corrige problema de travamento em construções com threads livres ao substituir o dicionário de objetos enquanto se lê um atributo em outra thread.
- gh-69639 [https://github.com/python/cpython/issues/69639]: Implementa regras aritméticas de modo misto, combinando números reais e números complexos, conforme especificado pelos padrões C desde o C99. Patch por Sergey B Kirpichev.
- gh-120010 [https://github.com/python/cpython/issues/120010]: Corrige casos extremos inválidos que resultavam em saídas como `(nan+nanj)` na multiplicação de números complexos, por exemplo, `(1e300+1j)*(nan+infj)`. Patch por Sergey B Kirpichev.
- gh-109746 [https://github.com/python/cpython/issues/109746]: Se **`_thread.start_new_thread()`** falhar ao iniciar uma nova thread, ele exclui seu estado do interpretador e, portanto, evita sua limpeza repetida na finalização.

API C

- gh-127896 [https://github.com/python/cpython/issues/127896]: A função anteriormente não documentada **`PySequence_In()`** está suavemente descontinuada. Use **`PySequence_Contains()`** em vez disso.

- gh-127791 [https://github.com/python/cpython/issues/127791]: Corrige perda de retornos de chamada após mais de uma chamada para **`PyUnstable_AtExit()`**.

- gh-127691 [https://github.com/python/cpython/issues/127691]: The Unicode Exception Objects C API now raises a **`TypeError`** if its exception argument is not a **`UnicodeError`** object. Patch by Bénédikt Tran.

- gh-123378 [https://github.com/python/cpython/issues/123378]: Ensure that the value of **`UnicodeEncodeError.end`** retrieved by **`PyUnicodeEncodeError_GetEnd()`** lies in `[min(1, objlen), max(min(1, objlen), objlen)]` where *objlen* is the length of **`UnicodeEncodeError.object`**. Similar arguments apply to

`UnicodeDecodeError` and `UnicodeTranslateError` and their corresponding C interface. Patch by Bénédikt Tran.

- gh-127314 [https://github.com/python/cpython/issues/127314]: Improve error message when calling the C API without an active thread state on the free-threaded build.

- gh-123378 [https://github.com/python/cpython/issues/123378]: Ensure that the value of `UnicodeEncodeError.start` retrieved by `PyUnicodeEncodeError_GetStart()` lies in `[0, max(0, objlen - 1)]` where *objlen* is the length of `UnicodeEncodeError.object`. Similar arguments apply to `UnicodeDecodeError` and `UnicodeTranslateError` and their corresponding C interface. Patch by Bénédikt Tran.

- gh-109523 [https://github.com/python/cpython/issues/109523]: Reading text from a non-blocking stream with `read` may now raise a `BlockingIOError` if the operation cannot immediately return bytes.

- gh-102471 [https://github.com/python/cpython/issues/102471]: Add a new import and export API for Python `int` objects (**PEP 757** [https://peps.python.org/pep-0757/]):

  - `PyLong_GetNativeLayout()`;
  - `PyLong_Export()`;
  - `PyLong_FreeExport()`;
  - `PyLongWriter_Create()`;
  - `PyLongWriter_Finish()`;
  - `PyLongWriter_Discard()`.

  Patch por Victor Stinner.

- gh-121058 [https://github.com/python/cpython/issues/121058]: `PyThreadState_Clear()` now warns (and calls `sys.excepthook`) if the thread state still has an active exception.

Construção

- gh-127865 [https://github.com/python/cpython/issues/127865]: Corrige falha de construção em sistemas sem suporte a thread-locals.
- gh-127629 [https://github.com/python/cpython/issues/127629]: Emscripten builds now include ctypes support.
- gh-127111 [https://github.com/python/cpython/issues/127111]: Updated the Emscripten web example to use ES6 modules and be built into a distinct `web_example` subfolder.
- gh-115869 [https://github.com/python/cpython/issues/115869]: Make `jit_stencils.h` (which is produced during JIT builds) reproducible.
- gh-126898 [https://github.com/python/cpython/issues/126898]: The Emscripten build of Python is now based on ES6 modules.

Python 3.14.0 alpha 2

Data de lançamento: 2024-11-19

Windows

- gh-126911 [https://github.com/python/cpython/issues/126911]: Atualiza a saída do comando credits.
- gh-118973 [https://github.com/python/cpython/issues/118973]: Garante que a instalação experimental de threads livres inclua o módulo `_tkinter`. O componente Tcl/Tk opcional também deve ser instalado para que o módulo funcione.
- gh-126497 [https://github.com/python/cpython/issues/126497]: Corrige a falha do venv devido à ausência de executáveis do redirecionador em instalações experimentais de threads livres.
- gh-126074 [https://github.com/python/cpython/issues/126074]: Remoção de DLLs desnecessárias do pacote incorporável do Windows
- gh-125315 [https://github.com/python/cpython/issues/125315]: Evita travamentos em **`platform`** devido a chamadas WMI lentas em algumas máquinas Windows.
- gh-126084 [https://github.com/python/cpython/issues/126084]: Corrige o venvwlauncher para iniciar o pythonw em vez do python para que nenhuma janela de console extra seja criada.

- gh-125842 [https://github.com/python/cpython/issues/125842]: Corrige uma `SystemError` quando `sys.exit()` é chamado com `0xffffffff` no Windows.
- gh-125550 [https://github.com/python/cpython/issues/125550]: Habilita o Gerenciador de instalação do Python para detectar instalações do Python 3.14 da Windows Store.
- gh-123803 [https://github.com/python/cpython/issues/123803]: All Windows code pages are now supported as “cpXXX” codecs on Windows.

Ferramentas/Demos

- gh-126807 [https://github.com/python/cpython/issues/126807]: Corrige avisos de extração em **pygettext.py** causados por definições de função equivocadas para chamadas de função.
- gh-126167 [https://github.com/python/cpython/issues/126167]: O conjunto de textos do iOS foi modificado para que possa ser usado por projetos de terceiros para fins de teste.

Testes

- gh-126909 [https://github.com/python/cpython/issues/126909]: Corrige testes de atributo estendido test_os para funcionar em sistemas de arquivos com limite de tamanho xattr de 1 KiB.
- gh-125730 [https://github.com/python/cpython/issues/125730]: Change `make test` to not run GUI tests by default. Use `make ci` to run tests with GUI tests instead.
- gh-124295 [https://github.com/python/cpython/issues/124295]: Adiciona testes de tradução ao módulo `argparse`.

Segurança

- gh-126623 [https://github.com/python/cpython/issues/126623]: Atualiza libexpat para 2.6.4

Biblioteca

- gh-85957 [https://github.com/python/cpython/issues/85957]: Add missing MIME types for images with RFCs: emf, fits, g3fax, jp2, jpm, jpx, t38, tiff-fx and wmf. Patch by Hugo van Kemenade.

- gh-126920 [https://github.com/python/cpython/issues/126920]: Fix the `prefix` and `exec_prefix` keys from **`sysconfig.get_config_vars()`** incorrectly having the same value as **`sys.base_prefix`** and **`sys.base_exec_prefix`**, respectively, inside virtual environments. They now accurately reflect **`sys.prefix`** and **`sys.exec_prefix`**.

- gh-67877 [https://github.com/python/cpython/issues/67877]: Corrige vazamentos de memória quando a correspondência de **`expressão regular`** termina abruptamente, seja por causa de um sinal ou porque a alocação de memória falha.

- gh-125063 [https://github.com/python/cpython/issues/125063]: **`marshal`** now supports **`slice`** objects. The marshal format version was increased to 5.

- gh-126789 [https://github.com/python/cpython/issues/126789]: Corrigidos os valores de **`sysconfig.get_config_vars()`**, **`sysconfig.get_paths()`** e seus irmãos quando a inicialização de **`site`** acontece após **`sysconfig`** ter construído um cache para **`sysconfig.get_config_vars()`**.

- gh-126188 [https://github.com/python/cpython/issues/126188]: Atualiza pip empacotado para 24.3.1

- gh-126766 [https://github.com/python/cpython/issues/126766]: Corrige problema em que **`urllib.request.url2pathname()`** falhava ao descartar duas barras iniciais introduzindo uma seção de autoridade vazia.

- gh-126705 [https://github.com/python/cpython/issues/126705]: Allow **`os.PathLike`** to be a base for Protocols.

- gh-126699 [https://github.com/python/cpython/issues/126699]: Permite que **`collections.abc.AsyncIterator`** seja uma base para Protocols.

- gh-126654 [https://github.com/python/cpython/issues/126654]: Corrige falha quando um objeto não dicionário era passado para várias funções no módulo `_interpreters`.

- gh-104745 [https://github.com/python/cpython/issues/104745]: Limita o início de um patcher (de **`unittest.mock.patch()`** ou **`unittest.mock.patch.object()`**) mais de uma vez sem pará-lo

- gh-126595 [https://github.com/python/cpython/issues/126595]: Corrige falha ao instanciar **`itertools.count`** com uma contagem inicial de **`sys.maxsize`** em construções de depuração. Patch por Bénédikt Tran.

- gh-120423 [https://github.com/python/cpython/issues/120423]: Corrige problema em que **`urllib.request.pathname2url()`** manipulava incorretamente caminhos do Windows com barras inseridas.

- gh-126565 [https://github.com/python/cpython/issues/126565]: Melhora o desempenho de **`zipfile.Path.open()`** para modos que não sejam de leitura.

- gh-126505 [https://github.com/python/cpython/issues/126505]: Corrige bugs na compilação de **`expressões regulares`** sem distinção entre maiúsculas e minúsculas com classes de caracteres que contenham caracteres não BMP: o caractere maiúsculo não BMP foi ignorado e o sinalizador ASCII foi ignorado ao corresponder a um intervalo de caracteres cujo limite superior está além da região BMP.

- gh-117378 [https://github.com/python/cpython/issues/117378]: Corrigido o processo de forkserver do método de início **`multiprocessing`** `"forkserver"` para herdar corretamente o **`sys.path`** do pai durante a importação de módulos **`multiprocessing.set_forkserver_preload()`** da mesma maneira que **`sys.path`** é configurado em workers antes de executar itens de trabalho.

  This bug caused some forkserver module preloading to silently fail to preload. This manifested as a performance degradation in child

processes when the `sys.path` was required due to additional repeated work in every worker.

Também poderia ter um efeito colateral de `""` permanecer em **`sys.path`** durante importações de pré-carregamento do forkserver em vez do caminho absoluto de **`os.getcwd()`** no momento da importação de multiprocessamento usado no worker `sys.path`.

As diferenças de `sys.path` entre as fases no processo filho poderiam ter causado o pré-carregamento para importar coisas incorretas do local errado. Não temos conhecimento de que isso realmente tenha acontecido na prática.

- gh-125679 [https://github.com/python/cpython/issues/125679]: Os valores de **`multiprocessing.Lock`** e **`multiprocessing.RLock`** `repr` não dizem mais "unknown" no macOS.

- gh-126476 [https://github.com/python/cpython/issues/126476]: Levanta **`calendar.IllegalMonthError`** (agora uma subclasse de **`IndexError`**) para **`calendar.month()`** quando o mês de entrada não está correto.

- gh-126489 [https://github.com/python/cpython/issues/126489]: A implementação Python de **`pickle`** não chama mais **`pickle.Pickler.persistent_id()`** para o resultado de **`persistent_id()`**.

- gh-126451 [https://github.com/python/cpython/issues/126451]: Register the **`contextvars.Context`** type to **`collections.abc.Mapping`**.

- gh-126175 [https://github.com/python/cpython/issues/126175]: Add `msg`, `doc`, `pos`, `lineno` and `colno` attributes to **`tomllib.TOMLDecodeError`**. Deprecate instantiating with free-form arguments.

- gh-89416 [https://github.com/python/cpython/issues/89416]: Add **RFC 9559** [https://datatracker.ietf.org/doc/html/rfc9559.html] MIME types for Matroska audiovisual container formats. Patch by Hugo van Kemenade.

- gh-126417 [https://github.com/python/cpython/issues/126417]: Register the **`multiprocessing.managers.DictProxy`** and

multiprocessing.managers.ListProxy types in **multiprocessing.managers** to **collections.abc.MutableMapping** and **collections.abc.MutableSequence**, respectively.

- gh-126390 [https://github.com/python/cpython/issues/126390]: Add support for returning intermixed options and non-option arguments in order in **getopt.gnu_getopt()**.

- gh-126374 [https://github.com/python/cpython/issues/126374]: Add support for options with optional arguments in the **getopt** module.

- gh-126363 [https://github.com/python/cpython/issues/126363]: Speed up pattern parsing in **pathlib.Path.glob()** by skipping creation of a **pathlib.Path** object for the pattern.

- gh-126353 [https://github.com/python/cpython/issues/126353]: **asyncio.get_event_loop()** now does not implicitly creates an event loop. It now raises a **RuntimeError** if there is no set event loop. Patch by Kumar Aditya.

- gh-126313 [https://github.com/python/cpython/issues/126313]: Corrige um problema em **curses.napms()** quando **curses.initscr()** ainda não foi chamado. Patch por Bénédikt Tran.

- gh-126303 [https://github.com/python/cpython/issues/126303]: Corrige serializar com pickle e cópia de objetos **os.sched_param**.

- gh-126138 [https://github.com/python/cpython/issues/126138]: Corrige uma falha de uso após liberação em objetos **asyncio.Task** cuja corrotina subjacente produz um objeto que implementa um **__getattribute__()** maligno. Patch por Nico Posada.

- gh-120057 [https://github.com/python/cpython/issues/120057]: Replace the `os.environ.refresh()` method with a new **os.reload_environ()** function. Patch by Victor Stinner.

- gh-126220 [https://github.com/python/cpython/issues/126220]: Corrige falha em **cProfile.Profile** e **_lsprof.Profiler** quando seus retornos de

chamada eram chamados diretamente com 0 argumentos.

- gh-126212 [https://github.com/python/cpython/issues/126212]: Corrige problema em que `urllib.request.pathname2url()` e `url2pathname()` removiam barras de caminhos de unidade DOS do Windows e URLs.

- gh-126223 [https://github.com/python/cpython/issues/126223]: Levanta `UnicodeEncodeError` em vez de um `SystemError` ao chamar `_interpreters.create()` com um caractere Unicode inválido.

- gh-126205 [https://github.com/python/cpython/issues/126205]: Corrige o problema em que `urllib.request.pathname2url()` gerava URLs começando com quatro barras (em vez de duas) quando recebia um caminho UNC do Windows.

- gh-126156 [https://github.com/python/cpython/issues/126156]: Improved performances of creating `Morsel` objects by a factor of 3.8x.

- gh-126105 [https://github.com/python/cpython/issues/126105]: Corrige uma falha em `ast` quando o atributo `ast.AST._fields` é excluído.

- gh-126106 [https://github.com/python/cpython/issues/126106]: Corrige uma possível desreferência de ponteiro `NULL` em `ssl`.

- gh-126080 [https://github.com/python/cpython/issues/126080]: Corrige uma falha de uso após liberação em objetos `asyncio.Task` para os quais o loop de eventos subjacente implementa um `__getattribute__()` maligno. Relatado por Nico-Posada. Patch por Bénédikt Tran.

- gh-125322 [https://github.com/python/cpython/issues/125322]: Correct detection of complex numbers support in libffi.

- gh-126083 [https://github.com/python/cpython/issues/126083]: Corrige um vazamento de referência em objetos `asyncio.Task` ao reinicializar o mesmo objeto com um contexto não `None`. Patch por Nico Posada.

- gh-126068 [https://github.com/python/cpython/issues/126068]: Fix exceptions in the `argparse` module so that only error messages for ArgumentError

and ArgumentTypeError are now translated. ArgumentError is now only used for command line errors, not for logical errors in the program. TypeError is now raised instead of ValueError for some logical errors.

- gh-125413 [https://github.com/python/cpython/issues/125413]: Add **`pathlib.Path.scandir()`** method to efficiently fetch directory children and their file attributes. This is a trivial wrapper of **`os.scandir()`**.

- gh-125984 [https://github.com/python/cpython/issues/125984]: Corrige falhas de uso após liberação em objetos **`asyncio.Future`** para os quais o laço de eventos subjacente implementa um **`__getattribute__()`** maligno. Relatado por Nico-Posada. Patch por Bénédikt Tran.

- gh-125926 [https://github.com/python/cpython/issues/125926]: Fix **`urllib.parse.urljoin()`** for base URI with undefined authority. Although **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html] only specify reference resolution for absolute base URI, **`urljoin()`** should continue to return sensible result for relative base URI.

- gh-125969 [https://github.com/python/cpython/issues/125969]: Corrige uma falha fora dos limites quando um **`asyncio.loop.call_soon()`** maligno altera o comprimento da lista de retornos de chamada internos. Patch por Bénédikt Tran.

- gh-125966 [https://github.com/python/cpython/issues/125966]: Corrige uma falha de uso após liberação em **`asyncio.Future.remove_done_callback()`**. Patch por Bénédikt Tran.

- gh-125789 [https://github.com/python/cpython/issues/125789]: Corrige possível falha ao alterar a lista de retornos de chamada retornados por **`asyncio.Future._callbacks`**. Agora ele sempre retorna uma nova cópia na implementação C **`_asyncio`**. Patch por Kumar Aditya.

- gh-126916 [https://github.com/python/cpython/issues/126916]: Allow the *initial* parameter of **`functools.reduce()`** to be passed as a keyword argument. Patch by Sayandip Dutta.

- gh-124452 [https://github.com/python/cpython/issues/124452]: Corrige um problema em `email.policy.EmailPolicy.header_source_parse()` e `email.policy.Compat32.header_source_parse()` que introduziam espaços em branco iniciais espúrios em valores de cabeçalho quando o cabeçalho incluía um caractere de nova linha após o delimitador de nome do cabeçalho (`:`) e antes do valor.

- gh-117941 [https://github.com/python/cpython/issues/117941]: **`argparse.BooleanOptionalAction`** now rejects option names starting with `--no-`.

- gh-125884 [https://github.com/python/cpython/issues/125884]: Corrige o bug para `pdb` onde ele não pode definir pontos de interrupção em funções com certas anotações.

- gh-125355 [https://github.com/python/cpython/issues/125355]: Corrige vários bugs em `argparse.ArgumentParser.parse_intermixed_args()`.

    - O analisador não muda mais temporariamente durante a análise.
    - Os valores padrão não são processados duas vezes.
    - Os grupos mutuamente exclusivos obrigatórios contendo argumentos posicionais agora são suportados.
    - O relatório de argumentos ausentes agora inclui os nomes de todos os argumentos posicionais e opcionais necessários.
    - Opções desconhecidas podem ser misturadas com argumentos posicionais em parse_known_intermixed_args().

- gh-125767 [https://github.com/python/cpython/issues/125767]: `super` objects are now `pickleable` and `copyable`.

- gh-124969 [https://github.com/python/cpython/issues/124969]: `locale.nl_langinfo(locale.ALT_DIGITS)` now returns a string again. The returned value consists of up to 100 semicolon-separated symbols.

- gh-84850 [https://github.com/python/cpython/issues/84850]: Remove **`URLopener`** and **`FancyURLopener`** classes from `urllib.request`. They had previously raised `DeprecationWarning` since Python 3.3.

- gh-125666 [https://github.com/python/cpython/issues/125666]: Evita a saída do interpretador se um byte nulo for fornecido como entrada no novo REPL.

- gh-125710 [https://github.com/python/cpython/issues/125710]: [Enum] corrige comparações hasheável<->não-hasheável para valores de membros

- gh-125631 [https://github.com/python/cpython/issues/125631]: Restaura a capacidade de definir atributos `persistent_id` e `persistent_load` de instâncias das classes `Pickler` e `Unpickler` no módulo `pickle`.

- gh-125378 [https://github.com/python/cpython/issues/125378]: Corrige o bug em `pdb` onde após um comando multilinha, uma linha vazia repete a primeira linha do comando multilinha, em vez do comando completo.

- gh-125682 [https://github.com/python/cpython/issues/125682]: Rejeita dígitos não ASCII na implementação Python de `json.loads()` em conformidade com a especificação JSON.

- gh-125660 [https://github.com/python/cpython/issues/125660]: Rejeita escapes unicode inválidos para implementação Python de `json.loads()`.

- gh-52551 [https://github.com/python/cpython/issues/52551]: Use `wcsftime()` to implement `time.strftime()` on Windows.

- gh-125259 [https://github.com/python/cpython/issues/125259]: Corrige a lógica de remoção de notas para erros lançados na inicialização de enumeração.

- gh-125633 [https://github.com/python/cpython/issues/125633]: Add function `inspect.ispackage()` to determine whether an object is a package or not.

- gh-125614 [https://github.com/python/cpython/issues/125614]: In the `FORWARDREF` format of `annotationlib`, fix bug where nested expressions were not returned as `annotationlib.ForwardRef` format.

- gh-125590 [https://github.com/python/cpython/issues/125590]: Permite que `FrameLocalsProxy` exclua e remova se a chave não for uma variável

rápida.

- gh-125600 [https://github.com/python/cpython/issues/125600]: Only show stale code warning in `pdb` when we display source code.

- gh-125542 [https://github.com/python/cpython/issues/125542]: Deprecate passing keyword-only *prefix_chars* argument to `argparse.ArgumentParser.add_argument_group()`.

- gh-125541 [https://github.com/python/cpython/issues/125541]: Pressing `Ctrl-C` while blocked in `threading.Lock.acquire()`, `threading.RLock.acquire()`, and `threading.Thread.join()` now interrupts the function call and raises a `KeyboardInterrupt` exception on Windows, similar to how those functions behave on macOS and Linux.

- gh-125519 [https://github.com/python/cpython/issues/125519]: Melhora o traceback se `importlib.reload()` for chamado com um objeto que não seja um módulo. Patch por Alex Waygood.

- gh-125451 [https://github.com/python/cpython/issues/125451]: Corrige um impasse quando `concurrent.futures.ProcessPoolExecutor` desliga simultaneamente com um erro ao alimentar um trabalho para um processo de trabalho.

- gh-125115 [https://github.com/python/cpython/issues/125115]: Corrige um bug em `pdb` onde argumentos começando com `-` não podem ser passados para o script depurado.

- gh-125398 [https://github.com/python/cpython/issues/125398]: Corrige a conversão do caminho `VIRTUAL_ENV` no script activate em `venv` ao executar no Git Bash para Windows.

- gh-125245 [https://github.com/python/cpython/issues/125245]: Corrige condição de corrida ao importar `collections.abc`, que poderia retornar incorretamente um módulo vazio.

- gh-52551 [https://github.com/python/cpython/issues/52551]: Corrige problemas de codificação em `time.strftime()`, o método `strftime()` das classes `datetime` `datetime`, `date` e `time` e a formatação dessas classes. Caracteres não codificáveis no localidade atual agora são aceitáveis na string de formato. Pares substitutos e sequência de bytes codificados por surrogatescape não são mais recombinados. O caractere nulo incorporado não termina mais a string de formato.

- gh-124984 [https://github.com/python/cpython/issues/124984]: Corrige segurança de thread em `ssl` na construção com threads livres. As operações OpenSSL agora são protegidas por uma trava por objeto.

- gh-124651 [https://github.com/python/cpython/issues/124651]: Coloca corretamente entre aspas as strings template em scripts de ativação `venv`.

- gh-124694 [https://github.com/python/cpython/issues/124694]: We've added `concurrent.futures.InterpreterPoolExecutor`, which allows you to run code in multiple isolated interpreters. This allows you to circumvent the limitations of CPU-bound threads (due to the GIL). Patch by Eric Snow.

  This addition is unrelated to **PEP 734** [https://peps.python.org/pep-0734/].

- gh-58032 [https://github.com/python/cpython/issues/58032]: Deprecate the `argparse.FileType` type converter.

- gh-99749 [https://github.com/python/cpython/issues/99749]: Adds a feature to optionally enable suggestions for argument choices and subparser names if mistyped by the user.

- gh-58956 [https://github.com/python/cpython/issues/58956]: Corrigido um bug em `pdb` onde às vezes o ponto de interrupção não disparava se ele fosse definido em uma função que já está na pilha de chamadas.

- gh-124111 [https://github.com/python/cpython/issues/124111]: The tkinter module can now be built to use either the new version 9.0.0 of Tcl/Tk or the latest release 8.6.15 of Tcl/Tk 8. Tcl/Tk 9 includes many

improvements, both to the Tcl language and to the appearance and utility of the graphical user interface provided by Tk.

- gh-80958 [https://github.com/python/cpython/issues/80958]: unittest discovery supports PEP 420 namespace packages as start directory again.

- gh-123370 [https://github.com/python/cpython/issues/123370]: Corrige a tela que não limpa após executar o relógio turtledemo.

- gh-89083 [https://github.com/python/cpython/issues/89083]: Add `uuid.uuid8()` for generating UUIDv8 objects as specified in **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html]. Patch by Bénédikt Tran

- gh-122549 [https://github.com/python/cpython/issues/122549]: Add `platform.invalidate_caches()` to invalidate cached results.

- gh-120754 [https://github.com/python/cpython/issues/120754]: Atualiza chamadas `read` ilimitadas em `zipfile` para especificar um `size` explícito, colocando um limite na quantidade de dados que eles podem ler. Isso também atualiza o tratamento em torno do tamanho máximo de comentário do ZIP para corresponder ao padrão em vez de ler comentários que são um byte muito longos.

- gh-121267 [https://github.com/python/cpython/issues/121267]: Improve the performance of `tarfile` when writing files, by caching user names and group names.

- gh-70764 [https://github.com/python/cpython/issues/70764]: Corrigido um problema em que `inspect.getclosurevars()` classificaria incorretamente um nome de atributo como uma variável global quando o nome existe tanto como um nome de atributo quanto como uma variável global.

- gh-118289 [https://github.com/python/cpython/issues/118289]: **`posixpath.realpath()`** agora levanta `NotADirectoryError` quando o modo *strict* está habilitado e um caminho sem diretório com uma barra final é fornecido.

- gh-119826 [https://github.com/python/cpython/issues/119826]: Sempre retorna um caminho absoluto para `os.path.abspath()` no Windows.

- gh-97850 [https://github.com/python/cpython/issues/97850]: Remove deprecated `pkgutil.get_loader()` and `pkgutil.find_loader()`.

- gh-118986 [https://github.com/python/cpython/issues/118986]: Add `socket.IPV6_RECVERR` constant (available since Linux 2.2).

- gh-116897 [https://github.com/python/cpython/issues/116897]: Accepting objects with false values (like `0` and `[]`) except empty strings, byte-like objects and `None` in `urllib.parse` functions `parse_qsl()` and `parse_qs()` is now deprecated.

- gh-101955 [https://github.com/python/cpython/issues/101955]: Corrige SystemError ao corresponder ao padrão de expressão regular contendo alguma combinação de quantificador possessivo, alternativo e grupo de captura.

- gh-71936 [https://github.com/python/cpython/issues/71936]: Corrigir uma condição de corrida em `multiprocessing.pool.Pool`.

- bpo-46128 [https://bugs.python.org/issue?@action=redirect&bpo=46128]: Remove quadros de pilha `unittest.IsolatedAsyncioTestCase` de stacktraces relatados.

- gh-84852 [https://github.com/python/cpython/issues/84852]: Add MIME types for MS Embedded OpenType, OpenType Layout, TrueType, WOFF 1.0 and 2.0 fonts. Patch by Sahil Prajapati and Hugo van Kemenade.

Documentação

- gh-126622 [https://github.com/python/cpython/issues/126622]: Adicionadas páginas de rascunho para módulos removidos explicando sua remoção, onde encontrar substituições e links para a última versão do Python que os suportava. Contribuição de Ned Batchelder.
- gh-125277 [https://github.com/python/cpython/issues/125277]: Exige Sphinx 7.2.6 ou posterior para construir a documentação do Python. Patch por

Adam Turner.

- gh-60712 [https://github.com/python/cpython/issues/60712]: Inclui o tipo `object` nas listas de tipos documentados. Alteração por Furkan Onder e Martin Panter.

Núcleo e embutidos

- gh-126795 [https://github.com/python/cpython/issues/126795]: Increase the threshold for JIT code warmup. Depending on platform and workload, this can result in performance gains of 1-9% and memory savings of 3-5%.
- gh-126341 [https://github.com/python/cpython/issues/126341]: Agora `ValueError` é levantada em vez de `SystemError` ao tentar iterar sobre um objeto `memoryview` liberado.
- gh-126688 [https://github.com/python/cpython/issues/126688]: Corrige uma falha ao chamar `os.fork()` em alguns sistemas operacionais, incluindo SerenityOS.

Biblioteca

- gh-126066 [https://github.com/python/cpython/issues/126066]: Corrige `importlib` para não gravar arquivos .pyc incompletos quando um ulimit ou algum outro mecanismo do sistema operacional estiver impedindo que a gravação seja feita completamente.

Núcleo e embutidos

- gh-126222 [https://github.com/python/cpython/issues/126222]: Do not include count of “peek” items in `_PyUop_num_popped`. This ensures that the correct number of items are popped from the stack when a micro-op exits with an error.
- gh-126366 [https://github.com/python/cpython/issues/126366]: Fix crash when using `yield from` on an object that raises an exception in its `__iter__`.

Biblioteca

- gh-126209 [https://github.com/python/cpython/issues/126209]: Fix an issue with `skip_file_prefixes` parameter which resulted in an inconsistent behaviour between the C and Python implementations of **`warnings.warn()`**. Patch by Daehee Kim.

Núcleo e embutidos

- gh-126312 [https://github.com/python/cpython/issues/126312]: Corrige travamento durante a coleta de lixo em um objeto congelado por **`gc.freeze()`** na construção com threads livres.
- gh-103951 [https://github.com/python/cpython/issues/103951]: Relax optimization requirements to allow fast attribute access to module subclasses.
- gh-126072 [https://github.com/python/cpython/issues/126072]: Following gh-126101 [https://github.com/python/cpython/issues/126101], for Objetos código like lambda, annotation and type alias, we no longer add `None` to its **`co_consts`**.
- gh-126195 [https://github.com/python/cpython/issues/126195]: Improve JIT performance by 1.4% on macOS Apple Silicon by using platform-specific memory protection APIs. Patch by Diego Russo.
- gh-126139 [https://github.com/python/cpython/issues/126139]: Fornece melhor localização de erro ao tentar usar uma instrução future com um recurso futuro desconhecido.
- gh-126072 [https://github.com/python/cpython/issues/126072]: Add a new attribute in **`co_flags`** to indicate whether the first item in **`co_consts`** is the docstring. If a code object has no docstring, `None` will **NOT** be inserted.
- gh-126076 [https://github.com/python/cpython/issues/126076]: Objetos realocados, como objetos `tuple`, `bytes` e `str` são rastreados corretamente por **`tracemalloc`** e seus ganchos associados. Patch por Pablo Galindo.
- gh-90370 [https://github.com/python/cpython/issues/90370]: Avoid temporary tuple creation for vararg in argument passing with Argument Clinic generated code (if arguments either vararg or positional-only).
- gh-126018 [https://github.com/python/cpython/issues/126018]: Corrige travamento em **`sys.audit()`** ao passar uma não string como primeiro argumento e o Python foi compilado no modo de depuração.
- gh-126012 [https://github.com/python/cpython/issues/126012]: The **`memoryview`** type now supports subscription, making it a generic type.

- gh-125837 [https://github.com/python/cpython/issues/125837]: Adds **`LOAD_SMALL_INT`** and **`LOAD_CONST_IMMORTAL`** instructions. `LOAD_SMALL_INT` pushes a small integer equal to the `oparg` to the stack. `LOAD_CONST_IMMORTAL` does the same as `LOAD_CONST` but is more efficient for immortal objects. Removes `RETURN_CONST` instruction.
- gh-125942 [https://github.com/python/cpython/issues/125942]: No Android, a configuração `errors` de **`sys.stdout`** foi alterada de `surrogateescape` para `backslashreplace`.
- gh-125859 [https://github.com/python/cpython/issues/125859]: Corrige uma falha na construção com threads livres quando **`gc.get_objects()`** ou **`gc.get_referrers()`** é chamado durante uma coleta de lixo em andamento.
- gh-125868 [https://github.com/python/cpython/issues/125868]: It was possible in 3.14.0a1 only for attribute lookup to give the wrong value. This was due to an incorrect specialization in very specific circumstances. This is fixed in 3.14.0a2.
- gh-125498 [https://github.com/python/cpython/issues/125498]: The JIT has been updated to leverage Clang 19's new `preserve_none` attribute, which supports more platforms and is more useful than LLVM's existing `ghccc` calling convention. This also removes the need to manually patch the calling convention in LLVM IR, simplifying the JIT compilation process.
- gh-125703 [https://github.com/python/cpython/issues/125703]: Honra corretamente os ganchos **`tracemalloc`** em caminhos `Py_DECREF` especializados. Patch por Pablo Galindo
- gh-125593 [https://github.com/python/cpython/issues/125593]: Usa cores para destacar locais de erro no traceback do grupo de exceções
- gh-125017 [https://github.com/python/cpython/issues/125017]: Fix crash on certain accesses to the `__annotations__` of **`staticmethod`** and **`classmethod`** objects.
- gh-125588 [https://github.com/python/cpython/issues/125588]: The Python PEG generator can now use f-strings in the grammar actions. Patch by Pablo Galindo
- gh-125444 [https://github.com/python/cpython/issues/125444]: Corrige instruções ilegais para arquiteturas Arm mais antigas. Patch por Diego Russo, testes de Ross Burton.

- gh-118423 [https://github.com/python/cpython/issues/118423]: Add a new `INSTRUCTION_SIZE` macro to the cases generator which returns the current instruction size.
- gh-125038 [https://github.com/python/cpython/issues/125038]: Corrige falha ao iterar sobre uma expressão geradora após alterações diretas em `gi_frame.f_locals`. Patch por Mikhail Efimov.
- gh-124855 [https://github.com/python/cpython/issues/124855]: Don't allow the JIT and perf support to be active at the same time. Patch by Pablo Galindo
- gh-123714 [https://github.com/python/cpython/issues/123714]: Update JIT compilation to use LLVM 19
- gh-123930 [https://github.com/python/cpython/issues/123930]: Melhora a mensagem de erro quando um script que faz "shadowing" de um módulo da biblioteca padrão faz com que `ImportError` seja levantada durante uma importação "from". Da mesma forma, melhore a mensagem de erro quando um script que faz shadowing de um módulo de terceiros tenta importar um atributo desse módulo de terceiros enquanto ainda está inicializando.
- gh-119793 [https://github.com/python/cpython/issues/119793]: The `map()` built-in now has an optional keyword-only *strict* flag like `zip()` to check that all the iterables are of equal length. Patch by Wannes Boeykens.

Biblioteca

- gh-118950 [https://github.com/python/cpython/issues/118950]: Corrige o bug em que SSLProtocol.connection_lost não estava sendo chamado quando OSError era lançada na gravação no soquete.
- gh-113570 [https://github.com/python/cpython/issues/113570]: Corrige um bug em `reprlib.repr` em que ele chamava incorretamente o método repr em tipos embutidos Python com shadowing.

API C

- gh-126554 [https://github.com/python/cpython/issues/126554]: Corrige o tratamento de erros em objetos `ctypes.CDLL` que podem resultar em uma falha em situações raras.
- gh-126061 [https://github.com/python/cpython/issues/126061]: Add `PyLong_IsPositive()`, `PyLong_IsNegative()` and `PyLong_IsZero()`

for checking if a **PyLongObject** is positive, negative, or zero, respectively.

- gh-125608 [https://github.com/python/cpython/issues/125608]: Corrige um bug em que os observadores de dicionário (por exemplo, **PyDict_Watch()**) no dicionário de atributos de um objeto (**__dict__**) não eram acionados quando os atributos do objeto eram modificados.
- gh-123619 [https://github.com/python/cpython/issues/123619]: Added the **PyUnstable_Object_EnableDeferredRefcount()** function for enabling **PEP 703** [https://peps.python.org/pep-0703/] deferred reference counting.
- gh-121654 [https://github.com/python/cpython/issues/121654]: Add **PyType_Freeze()** function to make a type immutable. Patch by Victor Stinner.
- gh-120026 [https://github.com/python/cpython/issues/120026]: The **Py_HUGE_VAL** macro is soft deprecated.

Construção

- gh-126691 [https://github.com/python/cpython/issues/126691]: Removed the `--with-emscripten-target` configure flag. We unified the `node` and `browser` options and the same build can now be used, independent of target runtime.
- gh-123877 [https://github.com/python/cpython/issues/123877]: Use `wasm32-wasip1` as the target triple for WASI instead of `wasm32-wasi`. The latter will eventually be reclaimed for WASI 1.0 while CPython currently only supports WASI preview1.
- gh-126458 [https://github.com/python/cpython/issues/126458]: Disable SIMD support for HACL under WASI.
- gh-89640 [https://github.com/python/cpython/issues/89640]: Codifica a ordem de palavras float como little endian no WASM.
- gh-126206 [https://github.com/python/cpython/issues/126206]: `make clinic` now runs Argument Clinic using the `--force` option, thus forcefully regenerating generated code.
- gh-126187 [https://github.com/python/cpython/issues/126187]: Introduced `Tools/wasm/emscripten.py` to simplify doing Emscripten builds.
- gh-124932 [https://github.com/python/cpython/issues/124932]: For cross builds, there is now support for having a different install `prefix` than the

`host_prefix` used by `getpath.py`. This is set to `/` by default for Emscripten, on other platforms the default behavior is the same as before.
- gh-125946 [https://github.com/python/cpython/issues/125946]: The minimum supported Android version is now 7.0 (API level 24).
- gh-125940 [https://github.com/python/cpython/issues/125940]: A construção no Android agora oferece suporte a tamanhos de página de 16 KB [https://developer.android.com/guide/practices/page-sizes].
- gh-89640 [https://github.com/python/cpython/issues/89640]: Melhora a detecção da ordem de palavras flutuantes no Linux quando as otimizações de tempo de link estiverem habilitadas.
- gh-124928 [https://github.com/python/cpython/issues/124928]: Emscripten builds now require node >= 18.
- gh-115382 [https://github.com/python/cpython/issues/115382]: Corrige falhas de compilação cruzada quando o host e o SOABIs de destino correspondem.

Python 3.14.0 alpha 1

Data de lançamento: 2024-10-15

macOS

- gh-124448 [https://github.com/python/cpython/issues/124448]: Atualiza o Tcl/Tk empacotado no instalador do macOS para 8.6.15.
- gh-123797 [https://github.com/python/cpython/issues/123797]: Verifica a disponibilidade do tempo de execução da função `ptsname_r` no macOS.
- gh-123418 [https://github.com/python/cpython/issues/123418]: Construção do instalador do macOS atualizada para usar o OpenSSL 3.0.15.

Windows

- gh-124487 [https://github.com/python/cpython/issues/124487]: Increases Windows required OS and API level to Windows 10.

- gh-124609 [https://github.com/python/cpython/issues/124609]: Corrige `_Py_ThreadId` para construções do Windows usando MinGW. Patch por Tony Roberts.
- gh-124448 [https://github.com/python/cpython/issues/124448]: Tcl/Tk empacotado atualizado para 8.6.15.
- gh-124254 [https://github.com/python/cpython/issues/124254]: Garante que binários experimentais de threads livres permaneçam instalados durante a atualização.
- gh-123915 [https://github.com/python/cpython/issues/123915]: Garante que `Tools\\msi\\buildrelease.bat` use diretórios diferentes para compilações AMD64 e ARM64.
- gh-123418 [https://github.com/python/cpython/issues/123418]: Construção do Windows atualizada para usar o OpenSSL 3.0.15.
- gh-123476 [https://github.com/python/cpython/issues/123476]: Add support for `socket.TCP_QUICKACK` on Windows platforms.
- gh-122573 [https://github.com/python/cpython/issues/122573]: A construção do Windows do CPython agora requer 3.10 ou mais recente.
- gh-100256 [https://github.com/python/cpython/issues/100256]: **`mimetypes`** não falha mais quando encontra uma chave de registro inacessível.
- gh-119679 [https://github.com/python/cpython/issues/119679]: Garante que bibliotecas de importação corretas sejam incluídas nas instalações do Windows.
- gh-119690 [https://github.com/python/cpython/issues/119690]: Adiciona suporte a Unicode e corrige eventos de auditoria para `_winapi.CreateNamedPipe`.
- gh-111201 [https://github.com/python/cpython/issues/111201]: Adiciona suporte para o novo pyrepl no Windows
- gh-119070 [https://github.com/python/cpython/issues/119070]: Corrige o tratamento de `py.exe` de shebangs como `/usr/bin/env python3.12`, que eram interpretados anteriormente como `python3.exe` em vez de `python3.12.exe`.
- gh-117505 [https://github.com/python/cpython/issues/117505]: Corrige um problema com o instalador do Windows não executando ensurepip em um ambiente totalmente isolado. Isso pode causar interações inesperadas com os pacotes do site do usuário.
- gh-118209 [https://github.com/python/cpython/issues/118209]: Evita travamentos em **`mmap`** no Windows quando a memória mapeada estiver inacessível

devido a erros do sistema de arquivos ou violações de acesso.
- gh-79846 [https://github.com/python/cpython/issues/79846]: Faz com que `ssl.create_default_context()` ignore certificados inválidos no armazenamento de certificados do Windows

Ferramentas/Demos

- gh-123418 [https://github.com/python/cpython/issues/123418]: Atualiza os fluxos de trabalho do GitHub CI para usar o OpenSSL 3.0.15 e multissltests para usar 3.0.15, 3.1.7 e 3.2.3.

Testes

- gh-125041 [https://github.com/python/cpython/issues/125041]: Reativa testes ignorados para **`zlib`** na arquitetura s390x: ignora apenas verificações dos bytes compactados, que podem ser diferentes entre a implementação de software do zlib e a implementação acelerada por hardware.
- gh-124378 [https://github.com/python/cpython/issues/124378]: Atualizado `test_ttk` para passar com Tcl/Tk 8.6.15.
- gh-124213 [https://github.com/python/cpython/issues/124213]: Detect whether the test suite is running inside a systemd-nspawn container with `--suppress-sync=true` option, and skip the `test_os` and `test_mmap` tests that are failing in this scenario.
- gh-124190 [https://github.com/python/cpython/issues/124190]: Add capability to ignore entire files or directories in check warning CI tool
- gh-121921 [https://github.com/python/cpython/issues/121921]: Update `Lib/test/crashers/bogus_code_obj.py` so that it crashes properly again.
- gh-112301 [https://github.com/python/cpython/issues/112301]: Add tooling to check for changes in compiler warnings. Patch by Nate Ohlson.
- gh-59022 [https://github.com/python/cpython/issues/59022]: Adiciona testes para **`pkgutil.extend_path()`**. Patch por Andreas Stocker.
- gh-99242 [https://github.com/python/cpython/issues/99242]: **`os.getloadavg()`** pode levantar **`OSError`** ao executar testes de regressão sob certas condições (por exemplo, chroot). Este erro agora é capturado e ignorado, pois o relatório da média de carga é opcional.

- gh-121084 [https://github.com/python/cpython/issues/121084]: Corrige vazamentos aleatórios de test_typing. Limpa caches ABC de tipagem ao executar testes para refleaks (opção `-R`): chama `_abc_caches_clear()` em classes abstratas de tipagem e suas subclasses. Patch por Victor Stinner.
- gh-121160 [https://github.com/python/cpython/issues/121160]: Adiciona um teste para **`readline.set_history_length()`**. Observe que este teste pode falhar em bibliotecas readline.
- gh-121200 [https://github.com/python/cpython/issues/121200]: Corrige `test_expanduser_pwd2()` de `test_posixpath`. Chama `getpwnam()` para obter `pw_dir`, pois pode ser diferente de `getpwall()` `pw_dir`. Patch por Victor Stinner.
- gh-121188 [https://github.com/python/cpython/issues/121188]: Ao criar o arquivo XML JUnit, o regrtest agora escapa caracteres inválidos em XML, como o caractere de controle chr(27) usado em sequências de escape ANSI. Patch por Victor Stinner.
- gh-120801 [https://github.com/python/cpython/issues/120801]: Cleaned up fixtures for importlib.metadata tests and consolidated behavior with 'test.support.os_helper'.
- gh-119727 [https://github.com/python/cpython/issues/119727]: Adiciona a opção de linha de comando `--single-process` ao executor de testes Python (regrtest). Patch por Victor Stinner.
- gh-119273 [https://github.com/python/cpython/issues/119273]: Python test runner no longer runs tests using TTY (ex: test_ioctl) in a process group (using `setsid()`). Previously, tests using TTY were skipped. Patch by Victor Stinner.
- gh-119050 [https://github.com/python/cpython/issues/119050]: Executor de teste regrtest: Adiciona suporte XML ao verificador de reflexos (opção -R). Patch de Victor Stinner.
- gh-101525 [https://github.com/python/cpython/issues/101525]: Ignora `test_gdb` se o binário for realocado pelo BOLT. Patch por Donghee Na.
- gh-107562 [https://github.com/python/cpython/issues/107562]: Test certificates have been updated to expire far in the future. This allows testing Y2038 with system time set to after that, so that actual Y2038 issues can be exposed, and not masked by expired certificate errors.

Segurança

- gh-125140 [https://github.com/python/cpython/issues/125140]: Remove o diretório atual de `sys.path` ao usar PyREPL.

- gh-123678 [https://github.com/python/cpython/issues/123678]: Atualiza o libexpat para 2.6.3

- gh-112301 [https://github.com/python/cpython/issues/112301]: Enable compiler options that warn of potential security vulnerabilities.

- gh-122792 [https://github.com/python/cpython/issues/122792]: Alterado `ipaddress.IPv6Address` mapeado para IPv4 para usar consistentemente o valor do endereço IPv4 mapeado para decidir propriedades. Propriedades que têm seu comportamento corrigido são `is_multicast`, `is_reserved`, `is_link_local`, `is_global` e `is_unspecified`.

- gh-112301 [https://github.com/python/cpython/issues/112301]: Add ability to ignore warnings per file with warning count in warning checking tooling. Patch by Nate Ohlson.

- gh-112301 [https://github.com/python/cpython/issues/112301]: Add macOS warning tracking to warning check tooling. Patch by Nate Ohlson.

- gh-122133 [https://github.com/python/cpython/issues/122133]: Autentica a conexão de soquete para o fallback `socket.socketpair()` em plataformas onde `AF_UNIX` não está disponível, como o Windows.

  Patch por Gregory P. Smith <greg@krypto.org> e Seth Larson <seth@python.org>. Reportado por Ellie <el@horse64.org>

- gh-121957 [https://github.com/python/cpython/issues/121957]: Corrigidos eventos de auditoria ausentes no uso interativo do Python, agora também sendo acionados corretamente para `python -i`, bem como para `python -m asyncio`. Os eventos em questão são `cpython.run_stdin` e `cpython.run_startup`.

- gh-112301 [https://github.com/python/cpython/issues/112301]: Enable runtime protections for glibc to abort execution when unsafe behavior is

encountered, for all platforms except Windows.

- gh-121285 [https://github.com/python/cpython/issues/121285]: Remove o backtracking da análise de cabeçalho do arquivo tar para `hdrcharset`, PAX e cabeçalhos esparsos GNU.

- gh-112301 [https://github.com/python/cpython/issues/112301]: Add default compiler options to improve security. Enable -Wimplicit-fallthrough, -fstack-protector-strong, -Wtrampolines.

- gh-118773 [https://github.com/python/cpython/issues/118773]: Corrige a criação de ACLs em **`os.mkdir()`** no Windows para funcionar corretamente em máquinas que não falam inglês.

- gh-118486 [https://github.com/python/cpython/issues/118486]: **`os.mkdir()`** no Windows agora aceita o *modo* de `0o700` para restringir o novo diretório ao usuário atual. Isso corrige **CVE 2024-4030** [https://www.cve.org/CVERecord?id=CVE-2024-4030] afetando **`tempfile.mkdtemp()`** em cenários onde o diretório temporário base é mais permissivo do que o padrão.

Biblioteca

- gh-125422 [https://github.com/python/cpython/issues/125422]: Corrige o bug onde **`pdb`** e **`bdb`** podem entrar no quadro do chamador inferior.

- gh-100141 [https://github.com/python/cpython/issues/100141]: Corrige o bug onde **`pdb`** ficará preso em um laço infinito ao depurar um arquivo vazio.

- gh-53203 [https://github.com/python/cpython/issues/53203]: Corrige **`time.strptime()`** para os formatos `%c`, `%x` e `%X` em muitos locais que usam dígitos não ASCII, como persa, birmanês, odia e shan.

- gh-125206 [https://github.com/python/cpython/issues/125206]: Workaround for old libffi versions is added. Module ctypes supports double complex only with libffi >= 3.3.0. Patch by Mikhail Efimov.

- gh-89967 [https://github.com/python/cpython/issues/89967]: Make `WeakKeyDictionary` and `WeakValueDictionary` safe against concurrent mutations from other threads. Patch by Kumar Aditya.

- gh-125260 [https://github.com/python/cpython/issues/125260]: The `gzip.compress()` *mtime* parameter now defaults to 0 for reproducible output. Patch by Bernhard M. Wiedemann and Adam Turner.

- gh-125243 [https://github.com/python/cpython/issues/125243]: Corrige corrida de dados ao criar objetos `zoneinfo.ZoneInfo` na construção com threads livres.

- gh-125254 [https://github.com/python/cpython/issues/125254]: Corrige um bug em que ArgumentError inclui a opção ambígua incorreta em `argparse`.

- gh-125235 [https://github.com/python/cpython/issues/125235]: Mantém os caminhos TCL `tkinter` em venv apontando para a instalação base no Windows.

- gh-61011 [https://github.com/python/cpython/issues/61011]: Corrige herança de grupos mutuamente exclusivos aninhados do analisador pai em `argparse.ArgumentParser`. Anteriormente, todos os grupos mutuamente exclusivos aninhados perdiam sua conexão com o grupo que os continha e eram exibidos como pertencentes diretamente ao analisador.

- gh-125118 [https://github.com/python/cpython/issues/125118]: Não copia valores arbitrários para _Bool no módulo `struct`.

- gh-125069 [https://github.com/python/cpython/issues/125069]: Corrige um problema em que fornecer um objeto `pathlib.PurePath` como um argumento inicializador para um segundo objeto `PurePath` com um `parser` diferente resultava em argumentos para o inicializador do objeto anterior sendo unidos pelo analisador do objeto posterior.

- gh-125096 [https://github.com/python/cpython/issues/125096]: Se a variável de ambiente `PYTHON_BASIC_REPL` estiver definida, o módulo `site` não importa mais o módulo `_pyrepl`. Além disso, o módulo `site` agora

respeita as opções de linha de comando **`-E`** e **`-I`**: ignore **`PYTHON_BASIC_REPL`** neste caso. Patch por Victor Stinner.

- gh-124969 [https://github.com/python/cpython/issues/124969]: Fix `locale.nl_langinfo(locale.ALT_DIGITS)`. Now it returns a tuple of up to 100 strings (an empty tuple on most locales). Previously it returned the first item of that tuple or an empty string.

- gh-124960 [https://github.com/python/cpython/issues/124960]: Corrige suporte para o sinalizador future `barry_as_FLUFL` no novo REPL.

- gh-69998 [https://github.com/python/cpython/issues/69998]: Fix **`locale.nl_langinfo()`** in case when different categories have different locales. The function now sets temporarily the `LC_CTYPE` locale in some cases. This temporary change affects other threads.

- gh-124958 [https://github.com/python/cpython/issues/124958]: Corrige refcycles em exceções levantadas de **`asyncio.TaskGroup`** e a implementação python de **`asyncio.Future`**

- gh-53203 [https://github.com/python/cpython/issues/53203]: Corrige **`time.strptime()`** para os formatos `%c` e `%x` em muitos idiomas: árabe, bislama, bretão, bodo, cassubiano, chuvache, estoniano, francês, irlandês, ge'ez, gurajati, gaélico manx, hebraico, hindi, chhattisgarhi, crioulo haitiano, japonês, canarês, coreano, marata, malaio, norueguês, nynorsk, punjabi, rajasthani, tok pisin, iorubá, chinês yue, yau nungon e chinês.

- gh-123961 [https://github.com/python/cpython/issues/123961]: Convert **`curses`** to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]), thereby fixing reference leaks at interpreter shutdown. Patch by Bénédikt Tran.

- gh-124944 [https://github.com/python/cpython/issues/124944]: Add `SO_ORIGINAL_DST` to the **`socket`** module.

- gh-117151 [https://github.com/python/cpython/issues/117151]: The default buffer size used by **`shutil.copyfileobj()`** has been increased from 64k to

256k on non-Windows platforms. It was already larger on Windows.

- gh-90102 [https://github.com/python/cpython/issues/90102]: Skip the `isatty` system call during open() when the file is known to not be a character device. This provides a slight performance improvement when reading whole files.

- gh-124917 [https://github.com/python/cpython/issues/124917]: Permite chamar **`os.path.exists()`** e **`os.path.lexists()`** com argumentos nomeados no Windows. Corrige uma regressão em 3.13.0.

- gh-65865 [https://github.com/python/cpython/issues/65865]: **`argparse`** now raises early error for invalid `help` arguments to **`add_argument()`**, **`add_subparsers()`** and **`add_parser()`**.

- gh-124653 [https://github.com/python/cpython/issues/124653]: Corrige a detecção da API de fila mínima necessária pelo módulo **`logging`**. Patch por Bénédikt Tran.

- gh-91818 [https://github.com/python/cpython/issues/91818]: The CLI of many modules (**`ast`**, **`ensurepip`**, **`json`**, **`pdb`**, **`sqlite3`**, **`tokenize`**, **`venv`**) now uses the actual executable name instead of simply “python” to display in the usage message.

- gh-124858 [https://github.com/python/cpython/issues/124858]: Corrige ciclos de referência deixados em tracebacks em **`asyncio.open_connection()`** quando usado com `happy_eyeballs_delay`

- gh-124390 [https://github.com/python/cpython/issues/124390]: Fixed **`AssertionError`** when using **`asyncio.staggered.staggered_race()`** with **`asyncio.eager_task_factory`**.

- gh-85935 [https://github.com/python/cpython/issues/85935]: **`argparse.ArgumentParser.add_argument()`** now raises an exception if an action that does not consume arguments (like ‘store_const’ or ‘store_true’) or explicit `nargs=0` are specified for positional arguments.

- gh-124835 [https://github.com/python/cpython/issues/124835]: Make `tomllib.loads()` raise `TypeError` not `AttributeError` on bad input types that do not have the `replace` attribute. Improve error message when `bytes` is received.

- gh-124693 [https://github.com/python/cpython/issues/124693]: Fix a bug where `argparse` doesn't recognize negative complex numbers or negative numbers using scientific notation.

- gh-124787 [https://github.com/python/cpython/issues/124787]: Fix `typing.TypeAliasType` with incorrect `type_params` argument. Now it raises a `TypeError` when a type parameter without a default follows one with a default, and when an entry in the `type_params` tuple is not a type parameter object.

- gh-66436 [https://github.com/python/cpython/issues/66436]: Improved prog default value for `argparse.ArgumentParser`. It will now include the name of the Python executable along with the module or package name, or the path to a directory, ZIP file, or directory within a ZIP file if the code was run that way.

- gh-116850 [https://github.com/python/cpython/issues/116850]: Corrige `argparse` para espaços de nomes com dict não diretamente gravável (por exemplo, classes).

- gh-101552 [https://github.com/python/cpython/issues/101552]: Add an *annoation_format* parameter to `inspect.signature()`. Add an *quote_annotation_strings* parameter to `inspect.Signature.format()`. Use the new functionality to improve the display of annotations in signatures in `pydoc`. Patch by Jelle Zijlstra.

- gh-58573 [https://github.com/python/cpython/issues/58573]: Corrige conflitos entre opções longas abreviadas no analisador pai e subanalisadores em `argparse`.

- gh-124594 [https://github.com/python/cpython/issues/124594]: Todos os prompts REPL `asyncio` são executados no mesmo `context`. Contribuição de

Bartosz Sławecki.

- gh-61181 [https://github.com/python/cpython/issues/61181]: Corrige suporte de choices com valor de string em **`argparse`**. Substrings da string especificada não são mais considerados valores válidos.

- gh-116750 [https://github.com/python/cpython/issues/116750]: Provide **`sys.monitoring.clear_tool_id()`** to unregister all events and callbacks set by the tool.

- gh-124552 [https://github.com/python/cpython/issues/124552]: Improve the accuracy of **`bdb`**'s check for the possibility of breakpoint in a frame. This makes it possible to disable unnecessary events in functions.

- gh-124538 [https://github.com/python/cpython/issues/124538]: Corrige falha ao usar **`gc.get_referents()`** em um objeto cápsula.

- gh-80259 [https://github.com/python/cpython/issues/80259]: Corrige suporte de **`argparse`** de argumentos posicionais com `nargs='?'`, `default=argparse.SUPPRESS` e `type` especificado.

- gh-120378 [https://github.com/python/cpython/issues/120378]: Corrige uma falha relacionada a um estouro de inteiro em **`curses.resizeterm()`** e **`curses.resize_term()`**.

- gh-124498 [https://github.com/python/cpython/issues/124498]: Corrige **`typing.TypeAliasType`** para não ser genérico, quando `type_params` é uma tupla vazia.

- gh-53834 [https://github.com/python/cpython/issues/53834]: Fix support of arguments with choices in **`argparse`**. Positional arguments with nargs equal to `'?'` or `'*'` no longer check default against `choices`. Optional arguments with `nargs` equal to `'?'` no longer check const against `choices`.

- gh-123884 [https://github.com/python/cpython/issues/123884]: Corrige bug no tratamento de itertools.tee() de outras entradas tee (um tee em um tee). A saída agora tem os prometidos *n* novos iteradores independentes.

Anteriormente, o primeiro iterador era idêntico (não independente) ao iterador de entrada. Isso às vezes dava resultados surpreendentes.

- gh-123017 [https://github.com/python/cpython/issues/123017]: Devido a resultados não confiáveis em alguns dispositivos, `time.strftime()` não aceita mais anos negativos no Android.

- gh-123014 [https://github.com/python/cpython/issues/123014]: `os.pidfd_open()` e `signal.pidfd_send_signal()` agora estão indisponíveis ao construir em níveis de API do Android mais antigos que 31, pois as chamadas de sistema subjacentes podem causar uma falha.

- gh-124176 [https://github.com/python/cpython/issues/124176]: Add support for `dataclasses.dataclass()` in `unittest.mock.create_autospec()`. Now `create_autospec` will check for potential dataclasses and use `dataclasses.fields()` function to retrieve the spec information.

- gh-124345 [https://github.com/python/cpython/issues/124345]: `argparse` supports abbreviated single-dash long options separated by `=` from its value.

- gh-124400 [https://github.com/python/cpython/issues/124400]: Fixed a `pdb` bug where `until` has no effect when it appears in a `commands` sequence. Also avoid printing the frame information at a breakpoint that has a command list containing a command that resumes execution.

- gh-90562 [https://github.com/python/cpython/issues/90562]: Modify dataclasses to support zero-argument super() when `slots=True` is specified. This works by modifying all references to `__class__` to point to the newly created class.

- gh-104860 [https://github.com/python/cpython/issues/104860]: Corrigido a falta de permissão de abreviação de opções longas de traço simples em `argparse` com `allow_abbrev=False`.

- gh-63143 [https://github.com/python/cpython/issues/63143]: Corrigida a análise de argumentos mutuamente exclusivos em `argparse`. Argumentos com o valor idêntico ao valor padrão (por exemplo, booleanos, inteiros

pequenos, strings vazias ou de 1 caractere) não são mais considerados “não presentes”.

- gh-72795 [https://github.com/python/cpython/issues/72795]: Argumentos posicionais com nargs igual a `'*'` ou **`argparse.REMAINDER`** não são mais necessários. Isso permite usar argumento posicional com `nargs='*'` e sem `default` em grupo mutuamente exclusivo e melhora a mensagem de erro sobre argumentos necessários.

- gh-59317 [https://github.com/python/cpython/issues/59317]: Corrige análise de argumento posicional com nargs igual a `'?'` ou `'*'` se for precedido por uma opção e outro argumento posicional.

- gh-100980 [https://github.com/python/cpython/issues/100980]: The **`_fields_`** attribute of **`ctypes.Structure`** and **`Union`** is no longer set if the setattr operation raises an error.

- gh-53780 [https://github.com/python/cpython/issues/53780]: **`argparse`** agora ignora o primeiro `"--"` (traço duplo) entre uma opção e um comando.

- gh-124217 [https://github.com/python/cpython/issues/124217]: Adicionar bloco IPv6 reservado do RFC 9637 `3fff::/20` no módulo **`ipaddress`**.

- gh-111513 [https://github.com/python/cpython/issues/111513]: Improve the error message that may be raised by **`datetime.date.fromtimestamp()`**.

- gh-124248 [https://github.com/python/cpython/issues/124248]: Corrige possível falha ao usar **`struct`** para processar campos ‘Pascal string’ de largura zero (`0p`).

- gh-81691 [https://github.com/python/cpython/issues/81691]: Corrige manipulação de múltiplos `"--"` (traços duplos) em **`argparse`**. Apenas o primeiro foi removido, todos os subsequentes agora são tomados literalmente.

- gh-87041 [https://github.com/python/cpython/issues/87041]: Corrige um bug em **`argparse`** onde a ajuda de argumento de sub-analisador longa é indentada incorretamente.

- gh-84559 [https://github.com/python/cpython/issues/84559]: The default **`multiprocessing`** start method on Linux and other POSIX systems has been changed away from often unsafe `"fork"` to `"forkserver"` (when the platform supports sending file handles over pipes as most do) or `"spawn"`. Mac and Windows are unchanged as they already default to `"spawn"`.

- gh-124212 [https://github.com/python/cpython/issues/124212]: Corrige variável inválida em **`venv`** manipulação de link simbólico com falha no Windows

- gh-124171 [https://github.com/python/cpython/issues/124171]: Adiciona solução alternativa para implementações quebradas de **`fmod()`** no Windows, que perdem o sinal de zero (por exemplo, `fmod(-10, 1)` retorna `0.0`). Patch por Sergey B Kirpichev.

- gh-123978 [https://github.com/python/cpython/issues/123978]: Remove **`time.thread_time()`** e **`time.thread_time_ns()`** quebrados no NetBSD.

- gh-123934 [https://github.com/python/cpython/issues/123934]: Fix **`unittest.mock.MagicMock`** resetting magic methods return values after `.reset_mock(return_value=True)` was called.

- gh-124016 [https://github.com/python/cpython/issues/124016]: Update **`unicodedata`** database to Unicode 16.0.0.

- gh-123968 [https://github.com/python/cpython/issues/123968]: Corrige a interface de linha de comando para o módulo **`random`** para selecionar floats entre 0 e N, não 1 e N.

- gh-123945 [https://github.com/python/cpython/issues/123945]: Fix a bug where **`argparse`** doesn’t recognize negative numbers with underscores

- gh-123935 [https://github.com/python/cpython/issues/123935]: Corrige detecção de slots pai para dataclasses que herdam de classes com `__dictoffset__`.

- gh-123892 [https://github.com/python/cpython/issues/123892]: Adicionar `"_wmi"` a `sys.stdlib_module_names`. Patch por Victor Stinner.

- gh-84808 [https://github.com/python/cpython/issues/84808]: Fix error handling in `socket` method `connect_ex()` on platforms where `errno` can be negative.

- gh-123756 [https://github.com/python/cpython/issues/123756]: Added a new argument `mode` to `pdb.Pdb`. Only allow `pdb` from command line to use `restart` command.

- gh-122765 [https://github.com/python/cpython/issues/122765]: Corrige erros de aspas desbalanceadas que ocorrem quando activate.csh em `venv` era carregado com um prompt personalizado contendo aspas desemparelhadas ou novas linhas.

- gh-123657 [https://github.com/python/cpython/issues/123657]: Corrige o travamento e o vazamento de memória em `decimal.getcontext()`. Ele travava ao usar um contexto local de thread por `--with-decimal-contextvar=no`.

- gh-123339 [https://github.com/python/cpython/issues/123339]: Corrige `inspect.getsource()` para classes nos módulos `collections.abc` e `decimal` (para implementação Python pura). `inspect.getcomments()` agora levanta OSError em vez de IndexError se o valor `__firstlineno__` para uma classe estiver fora dos limites.

- gh-123374 [https://github.com/python/cpython/issues/123374]: Remove check for redefined memo entry in `pickletools.dis()`.

- gh-123504 [https://github.com/python/cpython/issues/123504]: Fixed reference leak in the finalization of `tkinter`.

- gh-123430 [https://github.com/python/cpython/issues/123430]: Pages generated by the `http.server` module allow the browser to apply its default dark mode.

- gh-123446 [https://github.com/python/cpython/issues/123446]: Fix empty function name in `TypeError` when `csv.reader()`, `csv.writer()`, or `csv.register_dialect()` are used without the required args.

- gh-123448 [https://github.com/python/cpython/issues/123448]: Corrigido vazamento de memória de `typing.NoDefault` movendo-o para o vetor de tipos estáticos.

- gh-123409 [https://github.com/python/cpython/issues/123409]: Corrige a saída de `ipaddress.IPv6Address.reverse_pointer` de acordo com **RFC 3596, §2.5** [https://datatracker.ietf.org/doc/html/rfc3596.html#section-2.5]. Patch por Bénédikt Tran.

- gh-123089 [https://github.com/python/cpython/issues/123089]: Make `weakref.WeakSet` safe against concurrent mutations while it is being iterated. Patch by Kumar Aditya.

- gh-123363 [https://github.com/python/cpython/issues/123363]: Show string value of `CONTAINS_OP` oparg in `dis` output. Patch by Alexandr153.

- gh-123341 [https://github.com/python/cpython/issues/123341]: Add `__class_getitem__()` to `tkinter.Event` for type subscript support at runtime. Patch by Adonis Rakateli.

- gh-123340 [https://github.com/python/cpython/issues/123340]: Show string value of `IS_OP` oparg in `dis` output.

- gh-119518 [https://github.com/python/cpython/issues/119518]: Speed up normalization of `pathlib.PurePath` and `Path` objects by not interning string parts.

- gh-123270 [https://github.com/python/cpython/issues/123270]: Aplicada uma correção mais cirúrgica para payloads malformados em `zipfile.Path` causando loops infinitos (gh-122905 [https://github.com/python/cpython/issues/122905]) sem quebrar o conteúdo usando caracteres legítimos.

- gh-73991 [https://github.com/python/cpython/issues/73991]: Add `pathlib.Path.copy_into()` and `move_into()`, which copy and move files and directories into *existing* directories.

- gh-123228 [https://github.com/python/cpython/issues/123228]: Corrige tipo de retorno para `_pyrepl.readline._ReadlineWrapper.get_line_buffer()` para ser `str()`. Patch por Sergey B Kirpichev.

- gh-123240 [https://github.com/python/cpython/issues/123240]: Levanta eventos de auditoria para `input()` no novo REPL.

- gh-76960 [https://github.com/python/cpython/issues/76960]: Fix `urllib.parse.urljoin()` and `urllib.parse.urldefrag()` for URIs containing empty components. For example, `urljoin()` with relative reference "?" now sets empty query and removes fragment. Preserve empty components (authority, params, query, fragment) in `urljoin()`. Preserve empty components (authority, params, query) in `urldefrag()`.

- gh-116810 [https://github.com/python/cpython/issues/116810]: Resolve um vazamento de memória introduzido no `ssl` do CPython 3.10 quando a propriedade `ssl.SSLSocket.session` era acessada. Acelera o acesso de leitura e gravação à referida propriedade ao não clonar mais objetos de sessão desnecessariamente por meio de serialização.

- gh-123243 [https://github.com/python/cpython/issues/123243]: Corrige vazamento de memória em `_decimal`.

- gh-122546 [https://github.com/python/cpython/issues/122546]: Usar consistentemente o mesmo nome de arquivo para diferentes exceções no novo repl. Patch por Sergey B Kirpichev.

- gh-123213 [https://github.com/python/cpython/issues/123213]: `xml.etree.ElementTree.Element.extend()` and `Element` assignment no longer hide the internal exception if an erroneous generator is passed. Patch by Bar Harel.

- gh-85110 [https://github.com/python/cpython/issues/85110]: Preserva caminho relativo em URL sem netloc em `urllib.parse.urlunsplit()` e `urllib.parse.urlunparse()`.

- gh-123165 [https://github.com/python/cpython/issues/123165]: Add support for rendering `Positions` in `dis`.

- gh-122909 [https://github.com/python/cpython/issues/122909]: In urllib.request when URLError is raised opening an ftp URL, the exception argument is now consistently a string. Earlier versions passed either a string or an ftplib exception instance as the argument to URLError.

- gh-123084 [https://github.com/python/cpython/issues/123084]: Deprecate `shutil.ExecError`, which hasn't been raised by any `shutil` function since Python 3.4. It's now an alias for `RuntimeError`.

- gh-123085 [https://github.com/python/cpython/issues/123085]: In a bare call to `importlib.resources.files()`, ensure the caller's frame is properly detected when `importlib.resources` is itself available as a compiled module only (no source).

- gh-123067 [https://github.com/python/cpython/issues/123067]: Corrige complexidade quadrática na análise de valores de cookies entre aspas `"` com contrabarras por `http.cookies`.

- gh-123049 [https://github.com/python/cpython/issues/123049]: Add support for `UNNAMED_SECTION` in `configparser.ConfigParser.add_section()`.

- gh-121735 [https://github.com/python/cpython/issues/121735]: Ao trabalhar com arquivos zip, importlib.resources agora honra corretamente referências adjacentes ao módulo (por exemplo, `files(pkg.mod)` e não apenas `files(pkg)`).

- gh-122981 [https://github.com/python/cpython/issues/122981]: Corrige `inspect.getsource()` para classes geradas com classes base Python (por exemplo, enums).

- gh-122903 [https://github.com/python/cpython/issues/122903]: `zipfile.Path.glob` agora corresponde corretamente aos diretórios em vez de omiti-los silenciosamente.

- gh-122905 [https://github.com/python/cpython/issues/122905]: Objetos **`zipfile.Path`** agora sanitizam nomes do arquivo zip.

- gh-122873 [https://github.com/python/cpython/issues/122873]: Enable **`json`** module to work as a script using the **`-m`** switch: `python -m json`. See the JSON command-line interface documentation. Patch by Trey Hunner.

- gh-122858 [https://github.com/python/cpython/issues/122858]: Deprecate **`asyncio.iscoroutinefunction()`** in favor of **`inspect.iscoroutinefunction()`**.

- gh-116263 [https://github.com/python/cpython/issues/116263]: **`logging.handlers.RotatingFileHandler`** não substitui mais arquivos de log vazios.

- gh-105376 [https://github.com/python/cpython/issues/105376]: Restaura o método descontinuado `warn()` do **`logging`**. Ele foi removido no Python 3.13 alpha 1. Mantém o método descontinuado `warn()` no Python 3.13. Patch por Victor Stinner.

- gh-122311 [https://github.com/python/cpython/issues/122311]: Improve errors in the **`pickle`** module. **`PicklingError`** is now raised more often instead of **`UnicodeEncodeError`**, **`ValueError`** and **`AttributeError`**, and the original exception is chained to it. Improve and unify error messages in Python and C implementations.

- gh-122744 [https://github.com/python/cpython/issues/122744]: Atualiza a versão do pip empacotado no ensurepip para a versão 24.2.

- gh-118761 [https://github.com/python/cpython/issues/118761]: Improve import time of **`pprint`** by around seven times. Patch by Hugo van Kemenade.

- gh-118974 [https://github.com/python/cpython/issues/118974]: Add `decorator` parameter to `dataclasses.make_dataclass()` to customize the functional creation of dataclasses.

- gh-118814 [https://github.com/python/cpython/issues/118814]: Corrige o construtor `typing.TypeVar` quando o nome for passado como argumento nomeado.

- gh-122637 [https://github.com/python/cpython/issues/122637]: Adjust `cmath.tanh(nanj)` and `cmath.tanh(infj)` for recent C standards.

- gh-122478 [https://github.com/python/cpython/issues/122478]: Remove internal frames from tracebacks shown in `code.InteractiveInterpreter` with non-default `sys.excepthook()`. Save correct tracebacks in `sys.last_traceback` and update `__traceback__` attribute of `sys.last_value` and `sys.last_exc`.

- gh-116622 [https://github.com/python/cpython/issues/116622]: No Android, as constantes `FICLONE` e `FICLONERANGE` não são mais expostas por `fcntl`, pois esses ioctls são bloqueados pelo SELinux.

- gh-82378 [https://github.com/python/cpython/issues/82378]: Garante que o novo REPL interprete `sys.tracebacklimit` da mesma forma que o REPL clássico.

- gh-122334 [https://github.com/python/cpython/issues/122334]: Corrige o travamento ao importar `ssl` após a reinicialização do interpretador principal.

- gh-122459 [https://github.com/python/cpython/issues/122459]: Optimize `pickling` by name objects without the `__module__` attribute.

- gh-87320 [https://github.com/python/cpython/issues/87320]: Em `code.InteractiveInterpreter`, trata exceções causadas pela chamada de um `sys.excepthook()` não padrão. Antes, a exceção era apresentada para o chamador, encerrando o REPL.

- gh-122272 [https://github.com/python/cpython/issues/122272]: On some platforms such as Linux, year with century was not 0-padded when formatted by `strftime()` with C99-specific specifiers `'%C'` or `'%F'`. The 0-padding behavior is now guaranteed when the format specifiers `'%C'` and `'%F'` are supported by the C library. Patch by Ben Hsing

- gh-122400 [https://github.com/python/cpython/issues/122400]: Lida com `ValueError`s levantados por `os.stat()` em `filecmp.dircmp` e `filecmp.cmpfiles()`. Patch por Bénédikt Tran.

- gh-122311 [https://github.com/python/cpython/issues/122311]: Corrige algumas mensagens de erro em `pickle`.

- gh-121650 [https://github.com/python/cpython/issues/121650]: Cabeçalhos `email` com quebras de linha incorporadas agora são citados na saída. O `generator` agora se recusará a serializar (escrever) cabeçalhos que são dobrados ou delimitados de forma insegura; consulte `verify_generated_headers`. (Contribuição de Bas Bloemsaat e Petr Viktorin em gh-121650 [https://github.com/python/cpython/issues/121650].)

- gh-122332 [https://github.com/python/cpython/issues/122332]: Corrigido segfault com `asyncio.Task.get_coro()` ao usar uma fábrica de tarefas ansiosa.

- gh-105733 [https://github.com/python/cpython/issues/105733]: a função `ctypes.ARRAY()` agora está suavemente descontinuada: ela não emite mais avisos de descontinuação e não está programada para remoção.

- gh-122213 [https://github.com/python/cpython/issues/122213]: Add notes for pickle serialization errors that allow to identify the source of the error.

- gh-119180 [https://github.com/python/cpython/issues/119180]: As part of **PEP 749** [https://peps.python.org/pep-0749/], add the following attributes for customizing evaluation of annotation scopes:

  - `evaluate_value` on `typing.TypeAliasType`
  - `evaluate_bound`, `evaluate_constraints`, and `evaluate_default` on `typing.TypeVar`

  - `evaluate_default` on `typing.ParamSpec`
  - `evaluate_default` on `typing.TypeVarTuple`

- gh-119180 [https://github.com/python/cpython/issues/119180]: Fix handling of classes with custom metaclasses in `annotationlib.get_annotations`.

- gh-122170 [https://github.com/python/cpython/issues/122170]: Lida com `ValueError`s levantadas por `os.stat()` em `linecache`. Patch por Bénédikt Tran.

- gh-122163 [https://github.com/python/cpython/issues/122163]: Add notes for JSON serialization errors that allow to identify the source of the error.

- gh-122129 [https://github.com/python/cpython/issues/122129]: Improve support of method descriptors and wrappers in the help title.

- gh-122145 [https://github.com/python/cpython/issues/122145]: Corrige um problema ao relatar tracebacks correspondentes ao código Python emitindo um corpo AST vazio. Patch de Nikita Sobolev e Bénédikt Tran.

- gh-121723 [https://github.com/python/cpython/issues/121723]: Faz `logging.config.dictConfig()` aceitar qualquer objeto que implemente a API pública Queue. Veja a seção configuração de fila para detalhes. Patch por Bénédikt Tran.

- gh-82951 [https://github.com/python/cpython/issues/82951]: Serializa objetos com `__qualname__` complexo (como métodos não vinculados e classes aninhadas) por nome não envolve mais serializar objetos pais por valor em protocolos pickle < 4.

- gh-120754 [https://github.com/python/cpython/issues/120754]: `Pathlib.read_bytes` no longer opens the file in Python's buffered I/O mode. This reduces overheads as the code reads a file in whole leading to a modest speedup.

- gh-113785 [https://github.com/python/cpython/issues/113785]: `csv` agora analisa corretamente campos numéricos (quando usados com

`csv.QUOTE_NONNUMERIC` ou `csv.QUOTE_STRINGS`) que começam com um caractere de escape.

- gh-122088 [https://github.com/python/cpython/issues/122088]: `@warnings.deprecated` now copies the coroutine status of functions and methods so that `inspect.iscoroutinefunction()` returns the correct result.

- gh-122081 [https://github.com/python/cpython/issues/122081]: Corrige uma falha na função opcional **`decimal.IEEEContext()`** disponível por meio do sinalizador de configuração `EXTRA_FUNCTIONALITY`.

- gh-73991 [https://github.com/python/cpython/issues/73991]: Add `pathlib.Path.move()`, which moves a file or directory tree.

- gh-121268 [https://github.com/python/cpython/issues/121268]: Remove workarounds for non-IEEE 754 systems in `cmath`.

- gh-119698 [https://github.com/python/cpython/issues/119698]: Due to the lack of interest for `symtable.Class.get_methods()`, the method is marked as deprecated and will be removed in Python 3.16. Patch by Bénédikt Tran.

- gh-121889 [https://github.com/python/cpython/issues/121889]: Adjusts `cmath.acosh(complex('0+nanj'))` for recent C standards.

- gh-121804 [https://github.com/python/cpython/issues/121804]: Mostra corretamente os locais de erro quando `SyntaxError` é levantada em um novo repl. Patch por Sergey B Kirpichev.

- gh-121797 [https://github.com/python/cpython/issues/121797]: Add alternative `Fraction` constructor `Fraction.from_number()`.

- gh-121798 [https://github.com/python/cpython/issues/121798]: Add alternative `Decimal` constructor `Decimal.from_number()`.

- gh-120930 [https://github.com/python/cpython/issues/120930]: Corrigido um bug introduzido por gh-92081 [https://github.com/python/cpython/issues/92081] que

adicionava um espaço extra incorreto para palavras codificadas ocorrendo em cabeçalhos encapsulados.

- gh-57141 [https://github.com/python/cpython/issues/57141]: O argumento *shallow* para **`filecmp.dircmp`** (novo no Python 3.13) agora é somente-nomeado.

- gh-121245 [https://github.com/python/cpython/issues/121245]: Simplifica o tratamento do arquivo de histórico no auxiliar `site.register_readline()`. A variável `CAN_USE_PYREPL` agora será inicializada quando importada. Patch por Sergey B Kirpichev.

- gh-121249 [https://github.com/python/cpython/issues/121249]: Support the float complex and double complex C types in the **`struct`** module if the compiler has C11 complex arithmetic. Patch by Sergey B Kirpichev.

- gh-121486 [https://github.com/python/cpython/issues/121486]: **`math`** functions **`isqrt()`**, **`log()`**, **`log2()`** and **`log10()`** now support integers larger than `2**2**32` on 32-bit platforms.

- gh-121474 [https://github.com/python/cpython/issues/121474]: Corrigida a verificação de sanidade ausente para o argumento `parties` no construtor **`threading.Barrier`**. Patch por Clinton Christian (pygeek).

- gh-121450 [https://github.com/python/cpython/issues/121450]: Hard-coded breakpoints (**`breakpoint()`** and **`pdb.set_trace()`**) now reuse the most recent `Pdb` instance that calls `Pdb.set_trace()`, instead of creating a new one each time. As a result, all the instance specific data like `display` and `commands` are preserved across Hard-coded breakpoints.

- gh-119169 [https://github.com/python/cpython/issues/119169]: Slightly speed up **`os.walk()`** by simplifying exception handling.

- gh-121423 [https://github.com/python/cpython/issues/121423]: Improve import time of **`socket`** by lazy importing modules and writing **`socket.errorTab`** as a constant.

- gh-59110 [https://github.com/python/cpython/issues/59110]: `zipimport` supports now namespace packages when no directory entry exists.

- gh-119004 [https://github.com/python/cpython/issues/119004]: Corrige uma falha em OrderedDict.__eq__ quando operandos são mutados durante a verificação. Patch por Bénédikt Tran.

- gh-121313 [https://github.com/python/cpython/issues/121313]: Limit the reading size in the `multiprocessing.connection.Connection` class to 64 KiB to prevent memory overallocation and unnecessary memory management system calls.

- gh-121332 [https://github.com/python/cpython/issues/121332]: Corrige o construtor de nós `ast` com `_attributes` personalizados. Anteriormente, passar atributos personalizados levantaria `DeprecationWarning`. Passar argumentos para o construtor que não estão em `_fields` ou `_attributes` continua descontinuado. Patch por Jelle Zijlstra.

- gh-121245 [https://github.com/python/cpython/issues/121245]: Corrige um bug no tratamento do histórico de comandos do novo REPL que fazia com que o arquivo de histórico fosse apagado na saída do REPL.

- gh-121210 [https://github.com/python/cpython/issues/121210]: Handle AST nodes with missing runtime fields or attributes in `ast.compare()`. Patch by Bénédikt Tran.

- gh-121163 [https://github.com/python/cpython/issues/121163]: Add support for `all` as an valid `action` for `warnings.simplefilter()` and `warnings.filterwarnings()`.

- gh-121151 [https://github.com/python/cpython/issues/121151]: Corrige o encapsulamento de texto de uso longo de argumentos dentro de um grupo mutuamente exclusivo em `argparse`.

- gh-121141 [https://github.com/python/cpython/issues/121141]: Add support for `copy.replace()` to AST nodes. Patch by Bénédikt Tran.

- gh-87744 [https://github.com/python/cpython/issues/87744]: Corrige a corrida waitpid ao chamar **`send_signal()`** em asyncio. Patch por Kumar Aditya.

- gh-121027 [https://github.com/python/cpython/issues/121027]: Adiciona um aviso futuro em **`functools.partial.__get__()`**. Em versões futuras do Python, **`functools.partial`** será um descritor de método.

- gh-121027 [https://github.com/python/cpython/issues/121027]: Make the **`functools.partial`** object a method descriptor.

- gh-117784 [https://github.com/python/cpython/issues/117784]: CPython now detects whether its linked TLS library supports TLSv1.3 post-handshake authentication and disables that feature if support is lacking.

- gh-121025 [https://github.com/python/cpython/issues/121025]: Melhora o **`__repr__()`** de **`functools.partialmethod`**. Patch por Bénédikt Tran.

- gh-121018 [https://github.com/python/cpython/issues/121018]: Fixed issues where **`argparse.ArgumentParser.parse_args()`** did not honor `exit_on_error=False`. Based on patch by Ben Hsing.

- gh-119614 [https://github.com/python/cpython/issues/119614]: Corrige truncamento de strings com caracteres nulos incorporados em algumas operações internas em **`tkinter`**.

- gh-120910 [https://github.com/python/cpython/issues/120910]: Ao ler arquivos instalados de um egg, usa `relative_to(walk_up=True)` para honrar arquivos instalados fora da raiz da instalação.

- gh-61103 [https://github.com/python/cpython/issues/61103]: Support float complex, double complex and long double complex C types in **`ctypes`** as **`c_float_complex`**, **`c_double_complex`** and **`c_longdouble_complex`** if the compiler has C11 complex arithmetic. Patch by Sergey B Kirpichev.

- gh-120888 [https://github.com/python/cpython/issues/120888]: Atualiza pip wheel empacotado com ensurepip (pip 24.1.1)

- gh-101830 [https://github.com/python/cpython/issues/101830]: Acessar a representação de string do objeto `tkinter` não converte mais o objeto Tcl subjacente em uma string no Windows.

- gh-120678 [https://github.com/python/cpython/issues/120678]: Corrigida a regressão no novo REPL que significava que globais de arquivos passados usando o argumento `-i` não seriam incluídos no namespace global do REPL. Patch por Alex Waygood.

- gh-120811 [https://github.com/python/cpython/issues/120811]: Corrige possível vazamento de memória em `contextvars.Context.run()`.

- gh-120782 [https://github.com/python/cpython/issues/120782]: Corrigidas referências erradas dos tipos `datetime` após recarregar o módulo.

- gh-120713 [https://github.com/python/cpython/issues/120713]: `datetime.datetime.strftime()` agora preenche com 0 anos com menos de quatro dígitos para os especificadores de formato `%Y` e `%G` no Linux. Patch por Ben Hsing

- gh-120769 [https://github.com/python/cpython/issues/120769]: Faz com que uma linha vazia em `pdb` repita o último comando mesmo quando o comando é de `cmdqueue`.

- gh-120780 [https://github.com/python/cpython/issues/120780]: Show string value of LOAD_SPECIAL oparg in `dis` output.

- gh-41431 [https://github.com/python/cpython/issues/41431]: Add `datetime.time.strptime()` and `datetime.date.strptime()`. Contributed by Wannes Boeykens.

- gh-120743 [https://github.com/python/cpython/issues/120743]: Soft deprecate `os.popen()` and `os.spawn*` functions. They should no longer be used to write new code. The `subprocess` module is recommended instead. Patch by Victor Stinner.

- gh-120732 [https://github.com/python/cpython/issues/120732]: Corrige `name` passando para o objeto **`unittest.mock.Mock`** ao usar **`unittest.mock.create_autospec()`**.

- gh-111259 [https://github.com/python/cpython/issues/111259]: **`re`** now handles patterns like `"[\s\S]"` or `"\s|\S"` which match any character as effectively as a dot with the `DOTALL` modifier (`"(?s:.)"`).

- gh-120683 [https://github.com/python/cpython/issues/120683]: Corrige um erro em **`logging.LogRecord`**, quando a parte inteira do registro de data e hora é arredondada para cima, enquanto o cálculo de milissegundos trunca, fazendo com que o registro de data e hora do log esteja errado em até 999 ms (afetou aproximadamente 1 em 8 milhões de registros de data e hora).

- gh-118710 [https://github.com/python/cpython/issues/118710]: **`ipaddress.IPv4Address`** and **`ipaddress.IPv6Address`** attributes `version` and `max_prefixlen` are now available on the class.

- gh-120633 [https://github.com/python/cpython/issues/120633]: Move a barra de rolagem e remova os menus destacáveis no turtledemo.

- gh-120606 [https://github.com/python/cpython/issues/120606]: Allow users to use EOF to exit `commands` definition in **`pdb`**

- gh-120284 [https://github.com/python/cpython/issues/120284]: Allow **`asyncio.Runner.run()`** to accept awaitable objects instead of simply coroutines.

- gh-120541 [https://github.com/python/cpython/issues/120541]: Melhora o prompt no pager “less” quando **`help()`** é chamado com argumento não-string.

- gh-120495 [https://github.com/python/cpython/issues/120495]: Corrige o tratamento de exceção incorreto no Tab Nanny. Patch por Wulian233.

- gh-120388 [https://github.com/python/cpython/issues/120388]: Improve a warning message when a test method in **`unittest`** returns something other than

`None`. Now we show the returned object type and optional asyncio-related tip.

- gh-120381 [https://github.com/python/cpython/issues/120381]: Corrige **`inspect.ismethoddescriptor()`** para verificar também a falta de **`__delete__()`**. Patch por Jan Kaliszewski.

- gh-90425 [https://github.com/python/cpython/issues/90425]: O byte do SO em cabeçalhos gzip agora é sempre definido como 255 ao usar **`gzip.compress()`**.

- gh-120343 [https://github.com/python/cpython/issues/120343]: Corrige o relatório de deslocamento de coluna para tokens que vêm depois de f-strings multilinhas no módulo **`tokenize`**.

- gh-119180 [https://github.com/python/cpython/issues/119180]: As part of implementing **PEP 649** [https://peps.python.org/pep-0649/] and **PEP 749** [https://peps.python.org/pep-0749/], add a new module `annotationlib`. Add support for unresolved forward references in annotations to **`dataclasses`**, **`typing.TypedDict`**, and **`typing.NamedTuple`**.

- gh-119600 [https://github.com/python/cpython/issues/119600]: Corrige **`unittest.mock.patch()`** para não ler atributos do alvo quando `new_callable` estiver definido. Patch por Robert Collins.

- gh-120289 [https://github.com/python/cpython/issues/120289]: Fixed the use-after-free issue in **`cProfile`** by disallowing `disable()` and `clear()` in external timers.

- gh-82017 [https://github.com/python/cpython/issues/82017]: Added support for converting any objects that have the **`as_integer_ratio()`** method to a **`Fraction`**.

- gh-114053 [https://github.com/python/cpython/issues/114053]: Corrige bug de caso extremo em que **`typing.get_type_hints()`** produziria resultados incorretos se parâmetros de tipo em um escopo de classe fossem substituídos por atribuições em um escopo de classe e a semântica

`from __future__ import annotations` estivesse habilitada. Patch por Alex Waygood.

- gh-114053 [https://github.com/python/cpython/issues/114053]: Corrige **`NameError`** errôneo ao chamar **`inspect.get_annotations()`** com `eval_str=True` em uma classe que fazia uso de parâmetros do tipo **PEP 695** [https://peps.python.org/pep-0695/] em um módulo que tinha `from __future__ import annotations` no topo do arquivo. Patch por Alex Waygood.

- gh-120268 [https://github.com/python/cpython/issues/120268]: Proíbe passar `None` para **`datetime.date.fromtimestamp()`** Python puro para obter consistência com a implementação da extensão C.

- gh-120244 [https://github.com/python/cpython/issues/120244]: Corrige vazamento de memória em **`re.sub()`** quando a string de substituição contém referências anteriores.

- gh-120254 [https://github.com/python/cpython/issues/120254]: Added `commands` argument to **`pdb.set_trace()`** which allows users to send debugger commands from the source file.

- gh-120211 [https://github.com/python/cpython/issues/120211]: Corrige **`tkinter.ttk`** com Tcl/Tk 9.0.

- gh-71587 [https://github.com/python/cpython/issues/71587]: Corrige travamento na versão C de **`datetime.datetime.strptime()`** quando chamado novamente no interpretador reiniciado.

- gh-117983 [https://github.com/python/cpython/issues/117983]: Adia a importação de `threading` em `importlib.util` até que o carregamento preguiçoso seja usado.

- gh-120157 [https://github.com/python/cpython/issues/120157]: Remove unused constant `concurrent.futures._base._FUTURE_STATES` in **`concurrent.futures`**. Patch by Clinton Christian (pygeek).

- gh-120161 [https://github.com/python/cpython/issues/120161]: `datetime` não trava mais em certas situações complexas de ciclo de referência.

- gh-119698 [https://github.com/python/cpython/issues/119698]: Corrige `symtable.Class.get_methods()` e documenta seu comportamento. Patch por Bénédikt Tran.

- gh-120121 [https://github.com/python/cpython/issues/120121]: Adiciona `concurrent.futures.InvalidStateError` ao módulo `__all__`.

- gh-119933 [https://github.com/python/cpython/issues/119933]: Adiciona a enumeração `symtable.SymbolTableType` para representar as possíveis saídas do método `symtable.SymbolTable.get_type`. Patch por Bénédikt Tran.

- gh-120029 [https://github.com/python/cpython/issues/120029]: Expose `symtable.Symbol` methods `is_free_class()`, `is_comp_iter()` and `is_comp_cell()`. Patch by Bénédikt Tran.

- gh-120108 [https://github.com/python/cpython/issues/120108]: Corrige chamada de `copy.deepcopy()` em árvores `ast` que foram modificadas para ter referências a nós pais. Patch por Jelle Zijlstra.

- gh-120056 [https://github.com/python/cpython/issues/120056]: Add **`socket.IP_RECVERR`** and **`socket.IP_RECVTTL`** constants (both available since Linux 2.2). And **`socket.IP_RECVORIGDSTADDR`** constant (available since Linux 2.6.29).

- gh-112672 [https://github.com/python/cpython/issues/112672]: Suporte para construção de `tkinter` com Tcl 9.0.

- gh-120057 [https://github.com/python/cpython/issues/120057]: Added the `os.environ.refresh()` method to update **`os.environ`** with changes to the environment made by `os.putenv()`, by `os.unsetenv()`, or made outside Python in the same process. Patch by Victor Stinner.

- gh-120029 [https://github.com/python/cpython/issues/120029]: Expose `symtable.Symbol.is_type_parameter()` in the `symtable` module.

Patch by Bénédikt Tran.

- gh-119819 [https://github.com/python/cpython/issues/119819]: Corrige regressão para permitir configuração de registro com tipos de fila de multiprocessamento.

- gh-65454 [https://github.com/python/cpython/issues/65454]: **`unittest.mock.Mock.attach_mock()`** não dispara mais uma chamada para um `PropertyMock` sendo anexado.

- gh-117142 [https://github.com/python/cpython/issues/117142]: O módulo **`ctypes`** agora pode ser importado em todos os subinterpretadores, incluindo aqueles que têm sua própria GIL.

- gh-118835 [https://github.com/python/cpython/issues/118835]: Corrige falha do _pyrepl ao usar prompt personalizado com códigos de escape ANSI.

- gh-81936 [https://github.com/python/cpython/issues/81936]: Os métodos **`help()`** e **`showtopic()`** agora respeitam um argumento *output* configurado para **`pydoc.Helper`** e não usam o pager em tais casos. Patch por Enrico Tröger.

- gh-117398 [https://github.com/python/cpython/issues/117398]: O módulo `_datetime` (implementação C para **`datetime`**) agora oferece suporte a importação em múltiplos interpretadores.

- gh-119824 [https://github.com/python/cpython/issues/119824]: Print stack entry in **`pdb`** when and only when user input is needed.

- gh-119838 [https://github.com/python/cpython/issues/119838]: In mixed arithmetic operations with **`Fraction`** and complex, the fraction is now converted to **`float`** instead of **`complex`**.

- gh-119770 [https://github.com/python/cpython/issues/119770]: Make **`termios`** `ioctl()` constants positive. Patch by Victor Stinner.

- gh-89727 [https://github.com/python/cpython/issues/89727]: Corrige problema com **`shutil.rmtree()`** onde uma **`RecursionError`** é levantada em

árvores de diretório profundas.

- gh-119577 [https://github.com/python/cpython/issues/119577]: O **`DeprecationWarning`** emitido ao testar o valor verdade de um **`xml.etree.ElementTree.Element`** agora descreve o retorno incondicional de `True` em uma versão futura em vez de gerar uma exceção no Python 3.14.

- gh-89727 [https://github.com/python/cpython/issues/89727]: Corrige parcialmente o problema com **`shutil.rmtree()`** onde uma **`RecursionError`** é levantada em árvores de diretório profundas. Um erro de recursão não é mais levantado quando **`rmtree.avoids_symlink_attacks`** é falso.

- gh-93963 [https://github.com/python/cpython/issues/93963]: Remove deprecated names from `importlib.abc` as found in `importlib.resources.abc`.

- gh-119118 [https://github.com/python/cpython/issues/119118]: Corrige regressão de desempenho no módulo **`tokenize`** armazenando em cache o atributo de token `line` e calculando o deslocamento da coluna de forma mais eficiente.

- gh-89727 [https://github.com/python/cpython/issues/89727]: Corrige problema com **`os.fwalk()`** onde uma **`RecursionError`** era levantada em árvores de diretório profundas ajustando a implementação para ser iterativa em vez de recursiva.

- gh-119594 [https://github.com/python/cpython/issues/119594]: If one calls pow(fractions.Fraction, x, module) with modulo not None, the error message now says that the types are incompatible rather than saying pow only takes 2 arguments. Patch by Wim Jeantine-Glenn and Mark Dickinson.

- gh-119588 [https://github.com/python/cpython/issues/119588]: `zipfile.Path.is_symlink` agora avalia se o caminho fornecido é um link simbólico.

- gh-119562 [https://github.com/python/cpython/issues/119562]: Remove **`ast.Num`**, **`ast.Str`**, **`ast.Bytes`**, **`ast.NameConstant`** and **`ast.Ellipsis`**. They had

all emitted deprecation warnings since Python 3.12. Patch by Alex Waygood.

- gh-119555 [https://github.com/python/cpython/issues/119555]: Captura **`SyntaxError`** de **`compile()`** no método runsource() do InteractiveColoredConsole. Patch por Sergey B Kirpichev.

- gh-118908 [https://github.com/python/cpython/issues/118908]: Limita os globais expostos de importações e definições internas na inicialização do novo REPL. Patch por Eugene Triguba e Pablo Galindo.

- gh-117865 [https://github.com/python/cpython/issues/117865]: Improve the import time of the **`ast`** module by deferring the import of **`re`**. Patch by Jelle Zijlstra.

- gh-119127 [https://github.com/python/cpython/issues/119127]: Positional arguments of **`functools.partial()`** objects now support placeholders via **`functools.Placeholder`**.

- gh-113892 [https://github.com/python/cpython/issues/113892]: Agora, o método `sock_connect` de **`asyncio.ProactorEventLoop`** levanta um **`ValueError`** se o soquete fornecido não estiver no modo não bloqueante, bem como em outras implementações de laço.

- gh-119506 [https://github.com/python/cpython/issues/119506]: Corrige o método **`io.TextIOWrapper.write()`** que quebra o buffer interno quando o método é chamado novamente durante a descarga do buffer interno.

- gh-69214 [https://github.com/python/cpython/issues/69214]: Fix `fcntl.ioctl()` *request* parameter: use an `unsigned long` instead of an `unsigned int` for the *request* parameter of **`fcntl.ioctl()`** to support requests larger than `UINT_MAX`. Patch by Victor Stinner.

- gh-119105 [https://github.com/python/cpython/issues/119105]: `difflib`'s `DIffer.compare()` (and so also `ndiff`) can no longer be provoked into cubic-time behavior, or into unbounded recursion, and should generally be faster in ordinary cases too. Results may change in some cases, although that should be rare. Correctness of diffs is not affected.

Some similar lines far apart may be reported as deleting one and adding the other, where before they were displayed on adjacent output lines with markup showing the intraline differences.

- gh-119443 [https://github.com/python/cpython/issues/119443]: O REPL interativo não é mais executado com `from __future__ import annotations` habilitado. Patch por Jelle Zijlstra.

- gh-119461 [https://github.com/python/cpython/issues/119461]: Add `socket.VMADDR_CID_LOCAL` constant. Patch by Victor Stinner.

- gh-117398 [https://github.com/python/cpython/issues/117398]: Os objetos na C-API datetime agora são alocados estaticamente, o que significa melhor segurança de memória, especialmente quando o módulo é recarregado. Isso deve ser transparente para os usuários.

- gh-118894 [https://github.com/python/cpython/issues/118894]: **`asyncio`** O REPL agora tem os mesmos recursos do PyREPL.

- gh-118830 [https://github.com/python/cpython/issues/118830]: Bump **`pickle`** default protocol to `5`.

- gh-118911 [https://github.com/python/cpython/issues/118911]: No PyREPL, a lógica do `maybe-accept` foi atualizada para que, se o usuário pressionar `Enter` duas vezes, ele possa encerrar o bloco mesmo se houver um espaço em branco à direita. Além disso, agora, quando o usuário pressiona a seta para cima, o cursor fica na última linha da função. Isso corresponde ao comportamento do IPython. Patch por Aya Elsayed.

- gh-115225 [https://github.com/python/cpython/issues/115225]: Raise error on certain technically valid but pathological ISO 8601 strings passed to **`datetime.time.fromisoformat()`** that were previously parsed incorrectly.

- gh-111201 [https://github.com/python/cpython/issues/111201]: Remove dependência para **`readline`** do novo REPL do Python.

- gh-119189 [https://github.com/python/cpython/issues/119189]: Ao usar o operador `**` ou `pow()` com `Fraction` como base e um expoente que não seja racional, um float ou um complexo, a fração não é mais convertida em um float.

- gh-119174 [https://github.com/python/cpython/issues/119174]: Corrige DPI alto que causa janelas borradas no turtledemo (exemplos do turtle-graphics) Patch por Wulian233 e Terry Jan Reedy

- gh-119121 [https://github.com/python/cpython/issues/119121]: Corrige um NameError acontecendo em `asyncio.staggered.staggered_race`. Esta função agora está testada.

- gh-119105 [https://github.com/python/cpython/issues/119105]: `difflib.Differ` is much faster for some cases of diffs where many pairs of lines are equally similar.

- gh-119113 [https://github.com/python/cpython/issues/119113]: Corrige problema em que `pathlib.PurePath.with_suffix()` não levantava `TypeError` quando dado `None` como sufixo.

- gh-118643 [https://github.com/python/cpython/issues/118643]: Corrige um AttributeError no módulo `email` ao redobrar uma longa lista de endereços. Também corrige mais casos de codificação incorreta do separador de endereços na lista de endereços.

- gh-73991 [https://github.com/python/cpython/issues/73991]: Add `pathlib.Path.copy()`, which copies a file or directory to another.

- gh-58933 [https://github.com/python/cpython/issues/58933]: Faz `pdb` retornar ao quadro do chamador corretamente quando `f_trace` do quadro do chamador não estiver definido

- gh-82805 [https://github.com/python/cpython/issues/82805]: Support single-dot file extensions in `pathlib.PurePath.suffix` and related attributes and methods. For example, the `suffixes` of `PurePath('foo.bar.')` are now `['.bar', '.']` rather than `[]`. This brings file extension splitting in line with `os.path.splitext()`.

- gh-118924 [https://github.com/python/cpython/issues/118924]: Remove **`version`** and **`version_info`** from **`sqlite3`**. Patch by Hugo van Kemenade.

- gh-118928 [https://github.com/python/cpython/issues/118928]: Disallow using a sequence of parameters with named placeholders in **`sqlite3`** queries. Patch by Erlend E. Aasland.

- gh-118895 [https://github.com/python/cpython/issues/118895]: Define atributos em **`typing.NoDefault`** agora levanta **`AttributeError`** em vez de **`TypeError`**.

- gh-118868 [https://github.com/python/cpython/issues/118868]: Corrigido o problema em que kwargs não eram mais passados para o manipulador de registro QueueHandler

- gh-118827 [https://github.com/python/cpython/issues/118827]: Remove deprecated **`Quoter`** class from **`urllib.parse`**. It had previously raised a **`DeprecationWarning`** since Python 3.11. Patch by Nikita Sobolev.

- gh-118824 [https://github.com/python/cpython/issues/118824]: Remove deprecated **`pty.master_open()`** and **`pty.slave_open()`**. Use **`pty.openpty()`** instead. Patch by Nikita Sobolev.

- gh-118851 [https://github.com/python/cpython/issues/118851]: Argumentos `ctx` para os construtores de classes de nó **`ast`** agora são padronizados para **`ast.Load()`**. Patch por Jelle Zijlstra.

- gh-101588 [https://github.com/python/cpython/issues/101588]: Remove copy, deepcopy, and pickle from itertools. These had previously raised a DeprecationWarning since Python 3.12.

- gh-118805 [https://github.com/python/cpython/issues/118805]: Remove *type*, *choices*, and *metavar* parameters of **`argparse.BooleanOptionalAction`**. They were deprecated since Python 3.12.

- gh-118803 [https://github.com/python/cpython/issues/118803]: **`typing.ByteString`** and **`collections.abc.ByteString`** are removed. They had previously

raised a `DeprecationWarning` since Python 3.12.

- gh-118798 [https://github.com/python/cpython/issues/118798]: The *isdst* parameter has been removed from `email.utils.localtime()`. Patch by Hugo van Kemenade.

- gh-118760 [https://github.com/python/cpython/issues/118760]: Restaura o valor padrão de `tkiter.wantobjects` para `1`.

- gh-118760 [https://github.com/python/cpython/issues/118760]: Corrige erros ao chamar ligações Tkinter no Windows.

- gh-74033 [https://github.com/python/cpython/issues/74033]: Drop support for passing keyword arguments to `pathlib.Path`.

- gh-101357 [https://github.com/python/cpython/issues/101357]: Suppress all `OSError` exceptions from `pathlib.Path.exists()` and `is_*()` methods, rather than a selection of more common errors. The new behaviour is consistent with `os.path.exists()`, `os.path.isdir()`, etc. Use `pathlib.Path.stat()` to retrieve the file status without suppressing exceptions.

- gh-78707 [https://github.com/python/cpython/issues/78707]: Drop support for passing additional positional arguments to `pathlib.PurePath.relative_to()` and `is_relative_to()`.

- gh-118507 [https://github.com/python/cpython/issues/118507]: Corrige `os.path.isfile()` no Windows para pipes. Acelera `os.path.isjunction()` e `os.path.lexists()` no Windows com uma implementação nativa.

- gh-118772 [https://github.com/python/cpython/issues/118772]: Permite que instâncias `typing.TypeVar` sem um padrão sigam instâncias sem um padrão em alguns casos. Patch por Jelle Zijlstra.

- gh-118714 [https://github.com/python/cpython/issues/118714]: Permite `restart` na depuração post-mortem de `pdb`. Removida a mensagem de reinicialização quando o usuário sai do pdb do modo post-mortem.

- gh-118673 [https://github.com/python/cpython/issues/118673]: Removed executable bits and shebang from stdlib modules.

- gh-110863 [https://github.com/python/cpython/issues/110863]: **os.path.realpath()** agora suprime qualquer **OSError** de **os.readlink()** quando o modo *strict* está desabilitado (o padrão).

- gh-118263 [https://github.com/python/cpython/issues/118263]: Acelera **os.path.splitroot()** e **os.path.normpath()** com uma chamada C direta.

- gh-110190 [https://github.com/python/cpython/issues/110190]: Fix ctypes structs with array on SPARC by setting `MAX_STRUCT_SIZE` to 32 in stgdict. Patch by Jakub Kulik

- gh-118033 [https://github.com/python/cpython/issues/118033]: Corrige **dataclasses.dataclass()** não criando um slot `__weakref__` ao subclassificar **typing.Generic**.

- gh-117766 [https://github.com/python/cpython/issues/117766]: Sempre usa **str()** para exibir `choices` em **argparse**.

- gh-106531 [https://github.com/python/cpython/issues/106531]: Em **importlib.resources**, sincronizar com importlib_resources 6.3.2 [https://importlib-resources.readthedocs.io/en/latest/history.html#v6-3-2], incluindo: `MultiplexedPath` agora espera caminhos `Traversable`, descontinuando argumentos de string para `MultiplexedPath`; Suporte habilitado para recursos em pacotes de espaço de nomes em arquivos zip; Corrigido `NotADirectoryError` ao chamar arquivos em um subdiretório de um pacote de espaço de nomes.

- gh-115937 [https://github.com/python/cpython/issues/115937]: Removed extra preprocessing for the `__signature__` attribute: the code just check if it's a **inspect.Signature** instance. Patch by Sergey B Kirpichev.

- gh-115808 [https://github.com/python/cpython/issues/115808]: Add **operator.is_none()** and **operator.is_not_none()** functions.

- gh-114264 [https://github.com/python/cpython/issues/114264]: Improve performance of `copy.deepcopy()` by adding a fast path for atomic types.

- gh-113878 [https://github.com/python/cpython/issues/113878]: Add *doc* parameter to `dataclasses.field()`, so it can be stored and shown as a documentation / metadata. If `@dataclass(slots=True)` is used, then the supplied string is available in the `__slots__` dict. Otherwise, the supplied string is only available in the corresponding `dataclasses.Field` object.

  In order to support this feature we are changing the `__slots__` format in dataclasses from `tuple` to `dict` when documentation / metadata is present.

- gh-113978 [https://github.com/python/cpython/issues/113978]: Ignora avisos sobre conclusão de texto dentro do REPL.

- gh-113008 [https://github.com/python/cpython/issues/113008]: Correct argparse usage output for required, mutually exclusive groups containing a positional argument

- gh-108172 [https://github.com/python/cpython/issues/108172]: `webbrowser` honra o navegador preferido do sistema no Linux quando a entrada desktop de nome contém o texto de um nome de navegador conhecido.

- gh-109109 [https://github.com/python/cpython/issues/109109]: Agora você pode obter as cadeias de certificados TLS brutas de conexões TLS por meio dos métodos `ssl.SSLSocket.get_verified_chain()` e `ssl.SSLSocket.get_unverified_chain()`.

  Contribuição de Mateusz Nowak.

- gh-105623 [https://github.com/python/cpython/issues/105623]: Corrige degradação de desempenho em `logging.handlers.RotatingFileHandler`. Patch por Craig Robson.

- gh-102450 [https://github.com/python/cpython/issues/102450]: Add missing ISO-8601 24:00 alternative to midnight of next day to **`datetime.datetime.fromisoformat()`** and **`datetime.time.fromisoformat()`**. Patch by Izan "TizzySaurus" Robinson (tizzysaurus@gmail.com)

- gh-103956 [https://github.com/python/cpython/issues/103956]: Corrige a falta de caracteres de nova linha na saída do módulo **`trace`** quando o rastreamento de linha está habilitado, mas a linha de código-fonte para o quadro atual não está disponível.

- gh-92081 [https://github.com/python/cpython/issues/92081]: Corrige espaços ausentes em cabeçalhos de e-mail quando os espaços são misturados com caracteres codificados de 8 bits.

- gh-103194 [https://github.com/python/cpython/issues/103194]: Prepara o Tkinter para alterações na API C no Tcl 8.7/9.0 para evitar que **`_tkinter.Tcl_Obj`** seja retornado inesperadamente em vez de **`bool`**, **`str`**, **`bytearray`** ou **`int`**.

- gh-87106 [https://github.com/python/cpython/issues/87106]: Corrigida a manipulação em **`inspect.Signature.bind()`** de argumentos nomeados com o mesmo nome que argumentos somente-posicionais quando um argumento nomeado variádico (por exemplo, `**kwargs`) estiver presente.

- gh-103134 [https://github.com/python/cpython/issues/103134]: Add additional methods to proxy objects in the **`multiprocessing`** module:

  - **`clear()`** e **`copy()`** para proxies de **`list`**
  - **`fromkeys()`**, `reversed(d)`, `d | {}`, `{} | d`, `d |= {'b': 2}` para proxies de **`dict`**

- bpo-45767 [https://bugs.python.org/issue?@action=redirect&bpo=45767]: Corrige conversão de inteiros em **`os.major()`**, **`os.minor()`** e **`os.makedev()`**. Suporte a números de dispositivos maiores que `2**63-1`. Suporte a número de dispositivos inexistente (`NODEV`).

- bpo-44864 [https://bugs.python.org/issue?@action=redirect&bpo=44864]: Não traduz strings fornecidas pelo usuário em `argparse.ArgumentParser`.

- bpo-41843 [https://bugs.python.org/issue?@action=redirect&bpo=41843]: Solaris now uses `os.sendfile()` fast-copy syscall for more efficient `shutil` file copy related functions.

- bpo-15987 [https://bugs.python.org/issue?@action=redirect&bpo=15987]: Implemented `ast.compare()` for comparing two ASTs. Patch by Batuhan Taskaya with some help from Jeremy Hylton.

- gh-67693 [https://github.com/python/cpython/issues/67693]: Corrige `urllib.parse.urlunparse()` e `urllib.parse.urlunsplit()` para URIs com caminho começando com várias barras e sem autoridade. Baseado no patch de Ashwin Ramaswami.

- bpo-14074 [https://bugs.python.org/issue?@action=redirect&bpo=14074]: Corrige o processamento de metavariável de `argparse` para permitir que argumentos posicionais tenham uma metavar de tupla.

IDLE

- gh-122392 [https://github.com/python/cpython/issues/122392]: Aumenta o espaçamento vertical atualmente inadequado para os navegadores IDLE (caminho, módulo e pilha) em monitores de alta resolução.
- gh-112938 [https://github.com/python/cpython/issues/112938]: Corrige travamento ininterrupto quando o console obtém saída rápida e contínua.
- gh-122482 [https://github.com/python/cpython/issues/122482]: Alterado o Sobre do IDLE para direcionar os usuários para discuss.python.org em vez do e-mail e lista de discussão idle-dev, agora não utilizados.
- gh-78889 [https://github.com/python/cpython/issues/78889]: Para o congelamentos do Shell bloqueando o acesso do usuário a atributos sys.stdout.shell que não são métodos, que são todos privados.
- gh-120083 [https://github.com/python/cpython/issues/120083]: Adiciona a cor preta explícita como primeiro plano de dicas flutuantes (hovertips) do IDLE, necessário para macOS recentes. Corrige o Sonoma mostrando branco ilegível em amarelo claro. Patch por John Riggles.

- gh-120104 [https://github.com/python/cpython/issues/120104]: Corrige preenchimento em janelas de diálogo de configuração e pesquisa no IDLE.

Documentação

- gh-124872 [https://github.com/python/cpython/issues/124872]: Adicionadas as definições para contexto, contexto atual e protocolo de gerenciamento de contexto, atualizadas definições relacionadas para serem consistentes e expandida a documentação para `contextvars.Context`.

- gh-125018 [https://github.com/python/cpython/issues/125018]: A documentação `importlib.metadata` agora inclui alvos de referência cruzada semântica para as APIs documentadas significativas. Isso significa que referências intersphinx como `importlib.metadata.version()` agora funcionarão conforme o esperado.

- gh-124720 [https://github.com/python/cpython/issues/124720]: Atualiza a seção “Usando Python em um Mac” do documento “Configuração e uso do Python” e incluir informações sobre a instalação do suporte a threads livres.

- gh-124457 [https://github.com/python/cpython/issues/124457]: Remove coverity scan from the CPython repo. It has not been used since 2020 and is currently unmaintained.

- gh-116622 [https://github.com/python/cpython/issues/116622]: Adiciona um guia de plataforma Android e sinaliza módulos não disponíveis no Android.

- gh-123976 [https://github.com/python/cpython/issues/123976]: Refresh docs around custom providers.

- gh-70870 [https://github.com/python/cpython/issues/70870]: Esclarecido o uso duplo do termo “variável livre” (tanto o significado formal de qualquer referência a nomes definidos fora do escopo local, quanto o significado pragmático mais restrito de variáveis não locais nomeadas em `co_freevars`).

- gh-121277 [https://github.com/python/cpython/issues/121277]: Os escritores da documentação do CPython agora podem usar `next` como a versão para as diretivas `versionchanged`, `versionadded`, `deprecated`.

- gh-117765 [https://github.com/python/cpython/issues/117765]: Improved documentation for **`unittest.mock.patch.dict()`**

- gh-121749 [https://github.com/python/cpython/issues/121749]: Corrige documentação para **`PyModule_AddObjectRef()`**.

- gh-120012 [https://github.com/python/cpython/issues/120012]: Esclarece os comportamentos de **`multiprocessing.Queue.empty()`** e **`multiprocessing.SimpleQueue.empty()`** em filas fechadas. Patch por Bénédikt Tran.

- gh-119574 [https://github.com/python/cpython/issues/119574]: Added some missing environment variables to the output of **`--help-env`**.

- bpo-34008 [https://bugs.python.org/issue?@action=redirect&bpo=34008]: A documentação **`Py_Main()`** foi movida da seção “API de nível muito alto” para a seção “Inicialização e finalização”.

  Também explicita que esperamos que `Py_Main` seja chamado tipicamente em vez de `Py_Initialize` em vez de depois dele (já que `Py_Main` faz sua própria chamada para `Py_Initialize`). Documente que chamar ambos é suportado, mas depende da versão de quais configurações serão aplicadas corretamente.

Núcleo e embutidos

- gh-124375 [https://github.com/python/cpython/issues/124375]: Corrige uma falha na construção com threads livres quando o GC é executado simultaneamente com uma nova thread iniciando.
- gh-125221 [https://github.com/python/cpython/issues/125221]: Corrige possível condição de corrida ao chamar **`__reduce_ex__()`** pela primeira vez na construção com threads livres.
- gh-125174 [https://github.com/python/cpython/issues/125174]: Make the handling of reference counts of immortal objects more robust. Immortal objects

with reference counts that deviate from their original reference count by up to a billion (half a billion on 32 bit builds) are still counted as immortal.
- gh-125039 [https://github.com/python/cpython/issues/125039]: Make `this_instr` and `prev_instr` const in cases generator.

Biblioteca

- gh-125008 [https://github.com/python/cpython/issues/125008]: Corrige `tokenize.untokenize()` produzindo sintaxe inválida para chaves duplas precedidas por certos caracteres de escape.

Núcleo e embutidos

- gh-124871 [https://github.com/python/cpython/issues/124871]: Corrige bug do compilador (em algumas versões do 3.13) onde uma asserção falha durante a análise de acessibilidade.
- gh-123378 [https://github.com/python/cpython/issues/123378]: Corrige uma falha no método `__str__()` de objetos `UnicodeError` quando os valores `UnicodeError.start` e `UnicodeError.end` são inválidos ou estão fora do intervalo. Patch por Bénédikt Tran.
- gh-118093 [https://github.com/python/cpython/issues/118093]: Improve the experimental JIT compiler's ability to stay "on trace" when encountering highly-biased branches.
- gh-124642 [https://github.com/python/cpython/issues/124642]: Corrigido problema de escalabilidade em construções com threads livres para leituras sem travas de dicionários em cenários de multithread
- gh-116510 [https://github.com/python/cpython/issues/116510]: Corrige uma falha causada por strings internas imortais sendo compartilhadas entre subinterpretadores que usam inicialização básica de fase única. Nesse caso, a string pode ser usada por um interpretador que sobrevive ao interpretador que a criou e a internou. Para interpretadores que compartilham o estado obmalloc, compartilhe também o dict interno com o interpretador principal.
- gh-116510 [https://github.com/python/cpython/issues/116510]: Corrige um bug que pode causar uma falha quando subinterpretadores usam módulos "básicos" de extensão de fase única. Objetos compartilhados podem se

referir a nós PyGC_Head que foram liberados como parte da limpeza do interpretador.

- gh-119180 [https://github.com/python/cpython/issues/119180]: The `__main__` module no longer always contains an `__annotations__` dictionary in its global namespace.
- gh-124547 [https://github.com/python/cpython/issues/124547]: Ao desalocar um objeto com valores inline cujo `__dict__` ainda está ativo: se a alocação de memória para os valores inline falhar, limpe o dicionário. Evita uma falha do interpretador.
- gh-124513 [https://github.com/python/cpython/issues/124513]: Corrige uma falha no construtor FrameLocalsProxy: verifica o número de argumentos. Patch por Victor Stinner.
- gh-124442 [https://github.com/python/cpython/issues/124442]: Corrige não determinismo na compilação classificando o valor de `__static_attributes__`. Patch por kp2pml30.
- gh-124285 [https://github.com/python/cpython/issues/124285]: Fix bug where `bool(a)` can be invoked more than once during the evaluation of a compound boolean expression.
- gh-123856 [https://github.com/python/cpython/issues/123856]: Corrige falha do PyREPL quando uma interrupção de teclado é acionada após usar uma pesquisa de histórico
- gh-65961 [https://github.com/python/cpython/issues/65961]: Deprecate the setting and using `__package__` and `__cached__`.
- gh-119726 [https://github.com/python/cpython/issues/119726]: The JIT now generates more efficient code for calls to C functions resulting in up to 0.8% memory savings and 1.5% speed improvement on AArch64. Patch by Diego Russo.
- gh-122878 [https://github.com/python/cpython/issues/122878]: Usa o binário `pager`, se disponível (por exemplo, no Debian e derivados), para exibir REPL `help()`.
- gh-124188 [https://github.com/python/cpython/issues/124188]: Fix reading and decoding a line from the source file with non-UTF-8 encoding for syntax errors raised in the compiler.
- gh-124027 [https://github.com/python/cpython/issues/124027]: Suporte às chaves `<page up>`, `<page down>` e `<delete>` no Python REPL quando `$TERM` estiver definido como `vt100`.

- gh-124022 [https://github.com/python/cpython/issues/124022]: Fix bug where docstring is removed from classes in interactive mode.
- gh-123958 [https://github.com/python/cpython/issues/123958]: docstrings are now removed from the optimized AST in optimization level 2.
- gh-123923 [https://github.com/python/cpython/issues/123923]: The `f_executable` field in the internal `_PyInterpreterFrame` struct now uses a tagged pointer. Profilers and debuggers that uses this field should clear the least significant bit to recover the PyObject* pointer.
- gh-77894 [https://github.com/python/cpython/issues/77894]: Corrige possível travamento no coletor de lixo quando ele tenta quebrar um laço de referência contendo um objeto `memoryview`. Agora, um objeto `memoryview` só pode ser limpo se não houver buffers que o referenciem.
- gh-120221 [https://github.com/python/cpython/issues/120221]: o REPL do asyncio agora está reconhecendo corretamente KeyboardInterrupts. A exibição de exceções levantadas em threads secundárias foi corrigida.
- gh-119310 [https://github.com/python/cpython/issues/119310]: Permite que o novo console interativo leia arquivos de histórico escritos com a biblioteca editline que usam entradas com escape unicode. Patch por aorcajo e Łukasz Langa.
- gh-123572 [https://github.com/python/cpython/issues/123572]: Corrige mapeamentos de teclas para várias teclas F no Windows para o novo REPL. Patch por devdanzin

Biblioteca

- gh-123614 [https://github.com/python/cpython/issues/123614]: Add `turtle.save()` to easily save Turtle drawings as PostScript files. Patch by Marie Roald and Yngve Mardal Moe.

Núcleo e embutidos

- gh-123339 [https://github.com/python/cpython/issues/123339]: Define o atributo `__module__` para uma classe agora remove o item `__firstlineno__` do dict do tipo, então eles não serão mais inconsistentes.

- gh-119034 [https://github.com/python/cpython/issues/119034]: Altera as teclas `<page up>` e `<page down>` do REPL do Python para pesquisa de histórico para frente/para trás. Patch por Victor Stinner.

- gh-123562 [https://github.com/python/cpython/issues/123562]: Improve **`SyntaxError`** message for using `case ... as ...` with not a name.

- gh-123545 [https://github.com/python/cpython/issues/123545]: Corrige um decref duplo em casos raros em construções JIT experimentais.

- gh-123484 [https://github.com/python/cpython/issues/123484]: Corrige `_Py_DebugOffsets` para objetos longos para serem relativos ao início do objeto em vez do início de um subobjeto.

- gh-123446 [https://github.com/python/cpython/issues/123446]: Fix empty function name in **`TypeError`** when builtin magic methods are used without the required args.

- gh-123440 [https://github.com/python/cpython/issues/123440]: Improve **`SyntaxError`** message for using `except as` with not a name.

- gh-116017 [https://github.com/python/cpython/issues/116017]: Improved JIT memory consumption by periodically freeing memory used by infrequently-executed code. This change is especially likely to improve the memory footprint of long-running programs.

- gh-123344 [https://github.com/python/cpython/issues/123344]: Adiciona otimizações AST para padrões de parâmetros de tipo.

- gh-123321 [https://github.com/python/cpython/issues/123321]: Impede que a condição de corrida Parser/myreadline apresente falhas de segmentação no uso multithread. Patch por Bar Harel e Amit Wienner.

- gh-123177 [https://github.com/python/cpython/issues/123177]: Corrige um bug que causava o aparecimento de prompts perdidos no meio de linhas quebradas no novo REPL.

- gh-122982 [https://github.com/python/cpython/issues/122982]: Estende o período de descontinuação para inversão de bool (~) em dois anos.

- gh-123271 [https://github.com/python/cpython/issues/123271]: Make concurrent iterations over the same `zip()` iterator safe under free-threading.

- gh-123275 [https://github.com/python/cpython/issues/123275]: Suporte a `-X gil=1` e a `PYTHON_GIL=1` em construções sem threads livres.

- gh-123177 [https://github.com/python/cpython/issues/123177]: Desativa quebra de linha no Apple Terminal por meio de um código de escape ANSI. Patch por Pablo Galindo

- gh-123229 [https://github.com/python/cpython/issues/123229]: Corrige o aviso valgrind inicializando os buffers f-string para 0 no tokenizador. Patch por Pablo Galindo

- gh-122298 [https://github.com/python/cpython/issues/122298]: Restore printout of GC stats when `gc.set_debug(gc.DEBUG_STATS)` is called. This feature was accidentally removed when implementing incremental GC.

- gh-121804 [https://github.com/python/cpython/issues/121804]: Mostra corretamente os locais de erro quando uma `SyntaxError` é levanta no REPL básico. Patch por Sergey B Kirpichev.

- gh-115776 [https://github.com/python/cpython/issues/115776]: Enables inline values (Python's equivalent of hidden classes) on any class who's instances are of a fixed size.

- gh-123142 [https://github.com/python/cpython/issues/123142]: Corrige o local de origem muito amplo em tracebacks de exceção provenientes de iteráveis quebrados em compreensões.

- gh-123048 [https://github.com/python/cpython/issues/123048]: Corrige um bug em que o código de correspondência de padrões poderia emitir um `JUMP_FORWARD` sem local de origem.

- gh-118093 [https://github.com/python/cpython/issues/118093]: Break up `CALL_ALLOC_AND_ENTER_INIT` into micro-ops and relax requirement for exact args, in order to increase the amount of code supported by tier 2.

- gh-123123 [https://github.com/python/cpython/issues/123123]: Corrige a exibição de exceções **`SyntaxError`** cobrindo várias linhas. Patch por Pablo Galindo

- gh-123083 [https://github.com/python/cpython/issues/123083]: Corrige um possível uso após liberação em `STORE_ATTR_WITH_HINT`.

- gh-123022 [https://github.com/python/cpython/issues/123022]: Corrige falha em construção com threads livres ao chamar **`Py_Initialize()`** de uma thread não principal.

- gh-118093 [https://github.com/python/cpython/issues/118093]: Add three specializations for **`CALL_KW`**:

  - **`CALL_KW_PY`** for calls to Python functions
  - **`CALL_KW_BOUND_METHOD`** for calls to bound methods
  - **`CALL_KW_NON_PY`** for all other calls

- gh-122821 [https://github.com/python/cpython/issues/122821]: Make sure that branches in **`while`** statements have consistent offsets for **`sys.monitoring`**. **`while`** statements are now compiled with a simple jump at the end of the body, instead of duplicating the test.

- gh-122907 [https://github.com/python/cpython/issues/122907]: A construção com `HAVE_DYNAMIC_LOADING` agora funciona tão bem quanto no 3.12. As deficiências existentes serão abordadas separadamente. (Consulte https://github.com/python/cpython/issues/122950.)

- gh-122888 [https://github.com/python/cpython/issues/122888]: Corrige falha em certas chamadas para `str()` com argumentos posicionais do tipo errado. Patch por Jelle Zijlstra.

- gh-118093 [https://github.com/python/cpython/issues/118093]: Improve the experimental JIT's handling of polymorphic code.

- gh-122697 [https://github.com/python/cpython/issues/122697]: Fixed memory leaks at interpreter shutdown in the free-threaded build, and also reporting of leaked memory blocks via `-X showrefcount`.

- gh-116622 [https://github.com/python/cpython/issues/116622]: Corrige mensagens stdout e stderr do Android sendo truncadas ou perdidas.

- gh-122527 [https://github.com/python/cpython/issues/122527]: Corrige uma falha que ocorria quando um `PyStructSequence` era desalocado após o dicionário do seu tipo ser limpo pelo GC. O `tp_basicsize` do tipo agora considera campos não sequenciais que não estão incluídos no `Py_SIZE` da sequência.

- gh-122445 [https://github.com/python/cpython/issues/122445]: Adiciona somente campos que são modificados via self.* para `__static_attributes__`.

- gh-122417 [https://github.com/python/cpython/issues/122417]: In the free-threaded build, the reference counts for heap type objects are now partially stored in a distributed manner in per-thread arrays. This reduces contention on the heap type's reference count fields when creating or destroying instances of the same type from multiple threads concurrently.

- gh-116090 [https://github.com/python/cpython/issues/116090]: Corrige um problema em construções JIT que impediam alguns loops `for` de disparar corretamente eventos de monitoramento `RAISE`.

- gh-122300 [https://github.com/python/cpython/issues/122300]: Preserva nós AST para f-string com especificadores de formato de elemento único. Patch por Pablo Galindo

- gh-120906 [https://github.com/python/cpython/issues/120906]: `frame.f_locals` agora oferece suporte a objetos hash arbitrários como chaves.

- gh-122239 [https://github.com/python/cpython/issues/122239]: When a `list`, `tuple` or `dict` with too many elements is unpacked, show the actual length in the error message.

- gh-122245 [https://github.com/python/cpython/issues/122245]: Detection of writes to `__debug__` is moved from the compiler's codegen stage to the symtable. This means that these errors are now detected even in code that is optimized away before codegen (such as assertions with the `-O` command line option).

- gh-122234 [https://github.com/python/cpython/issues/122234]: Specializations for sums with float and complex inputs in `sum()` now always use compensated summation. Also, for integer items in above specializations: `PyLong_AsDouble()` is used, instead of `PyLong_AsLongAndOverflow()`. Patch by Sergey B Kirpichev.

- gh-122208 [https://github.com/python/cpython/issues/122208]: Os monitoradores de dicionário agora só entregam o evento PyDict_EVENT_ADDED quando a inserção está em um estado conhecido como bom para ser bem-sucedida.

- gh-122160 [https://github.com/python/cpython/issues/122160]: Remove the `BUILD_CONST_KEY_MAP` opcode. Use `BUILD_MAP` instead.

- gh-122029 [https://github.com/python/cpython/issues/122029]: Emite eventos `c_call` em `sys.setprofile()` quando um `PyMethodObject` apontando para um `PyCFunction` é chamado.

- gh-122026 [https://github.com/python/cpython/issues/122026]: Corrige um bug que fazia com que o tokenizador não identificasse corretamente parênteses incompatíveis dentro de f-strings em algumas situações. Patch por Pablo Galindo

- gh-99108 [https://github.com/python/cpython/issues/99108]: Python's hashlib now unconditionally uses the vendored HACL* library for Blake2. Python no longer accepts libb2 as an optional dependency for Blake2.

We refreshed HACL* to the latest version, and now vendor HACL*'s 128-bit and 256-bit wide vector implementations for Blake2, which are used on x86/x64 toolchains when the required CPU features are available at runtime.

HACL*'s 128-bit wide vector implementation of Blake2 can also run on ARM NEON and Power8, but lacking evidence of a performance gain, these are not enabled (yet).

Biblioteca

- gh-121999 [https://github.com/python/cpython/issues/121999]: The default extraction filter for the `tarfile` module is now set to `'data'`.

Núcleo e embutidos

- gh-121860 [https://github.com/python/cpython/issues/121860]: Corrige travamento ao rematerializar um dicionário gerenciado após ele ter sido excluído.
- gh-121795 [https://github.com/python/cpython/issues/121795]: Improve performance of set membership testing, `set.remove()` and `set.discard()` when the argument is a set.
- gh-121814 [https://github.com/python/cpython/issues/121814]: Corrigido o SegFault quando `PyEval_SetTrace()` é usado sem nenhum quadro Python na pilha.
- gh-121295 [https://github.com/python/cpython/issues/121295]: Corrigido o console PyREPL entrando em um estado bloqueado após interromper uma colagem longa
- gh-121794 [https://github.com/python/cpython/issues/121794]: Corrigido o bug no Python de threads livres onde um objeto ressuscitado poderia levar a uma falha de asserção de contagem de referência negativa.
- gh-121657 [https://github.com/python/cpython/issues/121657]: Melhorada a mensagem de `SyntaxError` se o usuário tentar usar `yield from` fora de uma função.
- gh-121609 [https://github.com/python/cpython/issues/121609]: Corrigido o ato de colar caracteres contendo unicode character joiners no novo REPL. Patch por Marta Gomez Macias

- gh-121297 [https://github.com/python/cpython/issues/121297]: Previously, incorrect usage of `await` or asynchronous comprehensions in code removed by the `-O` option was not flagged by the Python compiler. Now, such code raises `SyntaxError`. Patch by Jelle Zijlstra.
- gh-117482 [https://github.com/python/cpython/issues/117482]: Invólucros de slot inesperados não são mais criados para tipos estáticos embutidos em subinterpretadores.
- gh-121562 [https://github.com/python/cpython/issues/121562]: Optimized performance of hex_from_char by replacing switch-case with a lookup table
- gh-121499 [https://github.com/python/cpython/issues/121499]: Corrige um bug que afetava como o histórico multilinha estava sendo renderizado no novo REPL após interagir com o novo cache de tela. Patch por Pablo Galindo
- gh-121497 [https://github.com/python/cpython/issues/121497]: Corrige um bug que impedia o REPL de respeitar corretamente o histórico quando um gancho de entrada era definido. Patch por Pablo Galindo
- gh-121012 [https://github.com/python/cpython/issues/121012]: A execução do nível 2 agora garante que os iteradores de lista permaneçam esgotados, uma vez que se esgotem.
- gh-121439 [https://github.com/python/cpython/issues/121439]: Permite que tuplas de comprimento 20 na lista de liberação sejam reutilizadas.
- gh-121288 [https://github.com/python/cpython/issues/121288]: `ValueError` messages for `list.index()`, `range.index()`, `deque.index()`, `deque.remove()` and `ShareableList.index()` no longer contain the repr of the searched value (which can be arbitrary large) and are consistent with error messages for other `index()` and `remove()` methods.
- gh-121368 [https://github.com/python/cpython/issues/121368]: Corrige a condição de corrida em `_PyType_Lookup` na construção com threads livres devido a uma cerca de memória ausente. Isso pode levar `_PyType_Lookup` a retornar resultados incorretos no arm64.
- gh-121149 [https://github.com/python/cpython/issues/121149]: Added specialization for summation of complexes, this also improves accuracy of builtin `sum()` for such inputs. Patch by Sergey B Kirpichev.

- gh-121130 [https://github.com/python/cpython/issues/121130]: Corrige f-strings com expressões de depuração em especificadores de formato. Patch por Pablo Galindo

Biblioteca

- gh-121381 [https://github.com/python/cpython/issues/121381]: Remove `subprocess._USE_VFORK` escape hatch code and documentation. It was added just in case, and doesn't have any known cases that require it.

Núcleo e embutidos

- gh-119726 [https://github.com/python/cpython/issues/119726]: Optimize code layout for calls to C functions from the JIT on AArch64. Patch by Diego Russo.
- gh-121115 [https://github.com/python/cpython/issues/121115]: **`PyLong_AsNativeBytes()`** não usa mais métodos **`__index__()`** por padrão. O sinalizador `Py_ASNATIVEBYTES_ALLOW_INDEX` foi adicionado para permitir isso.
- gh-120838 [https://github.com/python/cpython/issues/120838]: **`Py_Finalize()`** e **`Py_FinalizeEx()`** agora sempre são executados com o interpretador principal ativo.
- gh-113433 [https://github.com/python/cpython/issues/113433]: Subinterpretadores agora são limpos automaticamente durante a finalização do tempo de execução.
- gh-119726 [https://github.com/python/cpython/issues/119726]: Improve the speed and memory use of C function calls from JIT code on AArch64. Patch by Diego Russo
- gh-116017 [https://github.com/python/cpython/issues/116017]: Simplify the warmup mechanism used for "side exits" in JIT code, resulting in slightly better performance and slightly lower memory usage for most platforms.
- gh-98442 [https://github.com/python/cpython/issues/98442]: Corrige localizações de origem muito amplas das instruções de limpeza de uma declaração with.
- gh-120754 [https://github.com/python/cpython/issues/120754]: Reduce the number of system calls invoked when reading a whole file (ex.

`open('a.txt').read()`). For a sample program that reads the contents of the 400+ `.rst` files in the cpython repository `Doc` folder, there is an over 10% reduction in system call count.

- gh-119462 [https://github.com/python/cpython/issues/119462]: Make sure that invariants of type versioning are maintained: * Superclasses always have their version number assigned before subclasses * The version tag is always zero if the tag is not valid. * The version tag is always non-if the tag is valid.
- gh-120437 [https://github.com/python/cpython/issues/120437]: Corrige problemas de otimização `_CHECK_STACK_SPACE` introduzidos em gh-118322 [https://github.com/python/cpython/issues/118322].
- gh-120722 [https://github.com/python/cpython/issues/120722]: Define corretamente a posição do bytecode em instruções de retorno dentro de lambdas. Patch por Jelle Zijlstra.
- gh-120367 [https://github.com/python/cpython/issues/120367]: Corrige bug onde o compilador cria um salto redundante durante a substituição de pseudo-op. Só pode acontecer com um AST sintético que tenha uma tentativa na mesma linha que a instrução após o manipulador de exceção.
- gh-120507 [https://github.com/python/cpython/issues/120507]: Remove the `BEFORE_WITH` and `BEFORE_ASYNC_WITH` instructions. Add the new **`LOAD_SPECIAL`** instruction. Generate code for `with` and `async with` statements using the new instruction.
- gh-113993 [https://github.com/python/cpython/issues/113993]: Strings internalizadas com **`sys.intern()`** são novamente coletadas como lixo quando não são mais usadas, conforme a documentação. Strings internadas com a função C **`PyUnicode_InternInPlace()`** ainda são imortais. Os internos do mecanismo de internação de strings foram alterados. Isso pode afetar o desempenho e as identidades dos objetos **`str`**.

Biblioteca

- gh-120485 [https://github.com/python/cpython/issues/120485]: Add an override of `allow_reuse_port` on classes subclassing `socketserver.TCPServer` where `allow_reuse_address` is also overridden.

Núcleo e embutidos

- gh-120384 [https://github.com/python/cpython/issues/120384]: Fix an array out of bounds crash in `list_ass_subscript`, which could be invoked via some specifically tailored input: including concurrent modification of a list object, where one thread assigns a slice and another clears it.
- gh-120367 [https://github.com/python/cpython/issues/120367]: Corrige falha no compilador em código com NOPs e JUMPs redundantes que aparecem depois que os manipuladores de exceção são movidos para o final do código.

Biblioteca

- gh-120380 [https://github.com/python/cpython/issues/120380]: Corrige implementação Python de **`pickle.Pickler`** para objetos **`bytes`** e **`bytearray`** ao usar o protocolo versão 5. Patch por Bénédikt Tran.

Núcleo e embutidos

- gh-120397 [https://github.com/python/cpython/issues/120397]: Improve the throughput by up to two times for the **`str.count()`**, **`bytes.count()`** and **`bytearray.count()`** methods for counting single characters.

- gh-120221 [https://github.com/python/cpython/issues/120221]: Entrega sinais reais em Ctrl-C e Ctrl-Z no novo REPL. Patch por Pablo Galindo

- gh-120346 [https://github.com/python/cpython/issues/120346]: Respect **`PYTHON_BASIC_REPL`** when running in interactive inspect mode (`python -i`). Patch by Pablo Galindo

- gh-93691 [https://github.com/python/cpython/issues/93691]: Corrige os locais de origem das instruções geradas para o iterador de uma instrução for.

- gh-120198 [https://github.com/python/cpython/issues/120198]: Corrige uma falha quando vários threads leem e escrevem na mesma `__class__` de um objeto simultaneamente.

- gh-120298 [https://github.com/python/cpython/issues/120298]: Fix use-after free in `list_richcompare_impl` which can be invoked via some specifically

tailored evil input.

- gh-119666 [https://github.com/python/cpython/issues/119666]: Corrige uma falha do compilador no caso em que duas compreensões no escopo da classe referenciam `__class__`.

- gh-119726 [https://github.com/python/cpython/issues/119726]: JIT: Re-use trampolines on AArch64 when creating stencils. Patch by Diego Russo

- gh-120225 [https://github.com/python/cpython/issues/120225]: Corrige a falha no compilador em bloco vazio no final do manipulador de exceção.

- gh-93691 [https://github.com/python/cpython/issues/93691]: Corrige localizações de origem de instruções geradas para declarações with.

- gh-120097 [https://github.com/python/cpython/issues/120097]: `FrameLocalsProxy` agora é uma subclasse de `collections.abc.Mapping` e pode ser correspondido como um mapeamento em instruções `match`

- gh-120080 [https://github.com/python/cpython/issues/120080]: Direct call to the **`int.__round__()`** now accepts `None` as a valid argument.

- gh-119933 [https://github.com/python/cpython/issues/119933]: Melhora as mensagens **`SyntaxError`** para expressões inválidas em um limite de parâmetros de tipo, uma tupla de restrição de parâmetro de tipo ou um parâmetro de tipo padrão. Patch por Bénédikt Tran.

- gh-119724 [https://github.com/python/cpython/issues/119724]: Revertidas as melhorias para mensagens de erro para instruções `elif`/`else` que não correspondem a nenhuma instrução válida, o que torna difícil localizar os erros de sintaxe dentro desses blocos `elif`/`else`.

- gh-119879 [https://github.com/python/cpython/issues/119879]: String search is now slightly faster for certain cases. It now utilizes last character gap (good suffix rule) for two-way periodic needles.

- gh-119842 [https://github.com/python/cpython/issues/119842]: Honra **`PyOS_InputHook()`** no novo REPL. Patch por Pablo Galindo

- gh-119180 [https://github.com/python/cpython/issues/119180]: **`classmethod()`** and **`staticmethod()`** now wrap the **`__annotations__`** and **`__annotate__`** attributes of their underlying callable lazily. See **PEP 649** [https://peps.python.org/pep-0649/]. Patch by Jelle Zijlstra.

- gh-119821 [https://github.com/python/cpython/issues/119821]: Corrige a execução de escopos de anotação dentro de classes quando `globals` estiver definido como um não-dict. Patch por Jelle Zijlstra.

- gh-118934 [https://github.com/python/cpython/issues/118934]: Faz `PyEval_GetLocals` retornar referência emprestada

- gh-119740 [https://github.com/python/cpython/issues/119740]: Remove the previously-deprecated delegation of **`int()`** to **`__trunc__()`**.

- gh-119689 [https://github.com/python/cpython/issues/119689]: Generate stack effect metadata for pseudo instructions from bytecodes.c.

- gh-109218 [https://github.com/python/cpython/issues/109218]: **`complex()`** accepts now a string only as a positional argument. Passing a complex number as the “real” or “imag” argument is deprecated; it should only be passed as a single positional argument.

- gh-119548 [https://github.com/python/cpython/issues/119548]: Adiciona um comando `clear` ao REPL. Patch por Pablo Galindo

- gh-111999 [https://github.com/python/cpython/issues/111999]: Corrige a assinatura de **`str.format_map()`**.

- gh-119560 [https://github.com/python/cpython/issues/119560]: Uma asserção inválida no beta 1 foi removida. A asserção falharia se `PyState_FindModule()` fosse usada na função init de um módulo de extensão antes que o módulo def tivesse sido inicializado.

- gh-119369 [https://github.com/python/cpython/issues/119369]: Corrige um impasse (deadlock) durante a exclusão de thread em construção com threads livres, que poderia ocorrer quando a GIL foi habilitada em tempo de execução.

- gh-119525 [https://github.com/python/cpython/issues/119525]: Corrige o impasse envolvendo o cache `_PyType_Lookup()` na construção com threads livres quando a GIL é habilitada dinamicamente em tempo de execução.

- gh-119258 [https://github.com/python/cpython/issues/119258]: Eliminate type version guards in the tier two interpreter.

  Note that setting the `tp_version_tag` manually (which has never been supported) may result in crashes.

- gh-119311 [https://github.com/python/cpython/issues/119311]: Corrige bug onde nomes são inesperadamente cortados nas bases de classes genéricas.

- gh-119395 [https://github.com/python/cpython/issues/119395]: Corrige bug onde nomes aparecendo após uma classe genérica são cortados como se estivessem na classe genérica.

- gh-119372 [https://github.com/python/cpython/issues/119372]: Correct invalid corner cases in complex division (resulted in `(nan+nanj)` output), e.g. `1/complex('(inf+infj)')`. Patch by Sergey B Kirpichev.

- gh-119180 [https://github.com/python/cpython/issues/119180]: Evaluation of annotations is now deferred. See **PEP 649** [https://peps.python.org/pep-0649/] for details.

- gh-119180 [https://github.com/python/cpython/issues/119180]: Replace **`LOAD_ASSERTION_ERROR`** opcode with **`LOAD_COMMON_CONSTANT`** and add support for **`NotImplementedError`**.

- gh-119213 [https://github.com/python/cpython/issues/119213]: Módulos não embutidos construídos com a clínica de argumentos estavam travando se usados em um subinterpretador antes do interpretador principal. Os objetos que estavam causando o problema por vazamento entre os interpretadores descuidadamente foram corrigidos.

- gh-119011 [https://github.com/python/cpython/issues/119011]: Corrige `type.__type_params__` para retornar uma tupla vazia em vez de um

descritor.

- gh-118692 [https://github.com/python/cpython/issues/118692]: Evita criar instâncias `StopIteration` desnecessárias para monitoramento.

- gh-119180 [https://github.com/python/cpython/issues/119180]: Add an `__annotate__` attribute to functions, classes, and modules as part of **PEP 649** [https://peps.python.org/pep-0649/]. Patch by Jelle Zijlstra.

- gh-119049 [https://github.com/python/cpython/issues/119049]: Corrige a exibição da linha de origem para avisos criados pela API C se o módulo `warnings` ainda não tiver sido importado.

- gh-119057 [https://github.com/python/cpython/issues/119057]: Improve `ZeroDivisionError` error message. Now, all error messages are harmonized: all `/`, `//`, and `%` operations just use “division by zero” message. And `0 ** -1` operation uses “zero to a negative power”.

- gh-118844 [https://github.com/python/cpython/issues/118844]: Corrige falhas de construção ao configurar com `--disable-gil` e `--enable-experimental-jit`.

- gh-118921 [https://github.com/python/cpython/issues/118921]: Adiciona o método `copy()` para `FrameLocalsProxy` que retorna um `dict` para variáveis locais.

- gh-117657 [https://github.com/python/cpython/issues/117657]: Corrige corridas de dados no campo que armazena um ponteiro para a thread principal do interpretador que ocorrem em construções com threads livres.

- gh-118750 [https://github.com/python/cpython/issues/118750]: If the C version of the `decimal` module is available, `int(str)` now uses it to supply an asymptotically much faster conversion. However, this only applies if the string contains over about 2 million digits.

- gh-118767 [https://github.com/python/cpython/issues/118767]: Using `NotImplemented` in a boolean context now raises `TypeError`. Contributed by Jelle Zijlstra.

- gh-118561 [https://github.com/python/cpython/issues/118561]: Fix race condition in free-threaded build where `list.extend()` could expose uninitialised memory to concurrent readers.

- gh-117139 [https://github.com/python/cpython/issues/117139]: Convert the Python evaluation stack to use internal stack references. The purpose is to support tagged pointers. In **PEP 703** [https://peps.python.org/pep-0703/], this will allow for its form of deferred reference counting. For both the default and free-threaded builds, this sets up the infrastructure for unboxed integers in the future.

Biblioteca

- gh-115801 [https://github.com/python/cpython/issues/115801]: Raise `TypeError` when passing a string to `difflib.unified_diff()` and `difflib.context_diff()`.

Núcleo e embutidos

- gh-117558 [https://github.com/python/cpython/issues/117558]: Improve error messages when a string, bytes or bytearray object of length 1 is expected.
- gh-117195 [https://github.com/python/cpython/issues/117195]: Evita falha de asserção para construções de depuração ao chamar `object.__sizeof__(1)`
- gh-116022 [https://github.com/python/cpython/issues/116022]: Improve the `__repr__()` output of `AST` nodes.
- gh-114091 [https://github.com/python/cpython/issues/114091]: Changed the error message for awaiting something that can't be awaited from "object <type> can't be used in an await expression" to "'<type>' object can't be awaited".
- gh-113190 [https://github.com/python/cpython/issues/113190]: `Py_Finalize()` now deletes all interned strings.
- gh-84978 [https://github.com/python/cpython/issues/84978]: Add class methods `float.from_number()` and `complex.from_number()`.
- gh-95144 [https://github.com/python/cpython/issues/95144]: Improve the error message from `a in b` when `b` is not a container to mention the term

“container”.

- bpo-24766 [https://bugs.python.org/issue?@action=redirect&bpo=24766]: Corrige manipulação do argumento `doc` para subclasses de `property`.

API C

- gh-123961 [https://github.com/python/cpython/issues/123961]: Convert the curses.window static type exposed by the **`PyCursesWindow_Type`** macro in `Include/py_curses.h` to a heap type. Patch by Bénédikt Tran.

- gh-124502 [https://github.com/python/cpython/issues/124502]: Add **`PyUnicode_Equal()`** function to the limited C API: test if two strings are equal. Patch by Victor Stinner.

- gh-124296 [https://github.com/python/cpython/issues/124296]: **`PyDictObject`** no longer maintains a private version tag field `ma_version_tag` per **PEP 699** [https://peps.python.org/pep-0699/]. This field was originally added in Python 3.6 (**PEP 509** [https://peps.python.org/pep-0509/]) and deprecated in Python 3.12.

- gh-124160 [https://github.com/python/cpython/issues/124160]: Corrige travamento ao importar módulos contendo inicialização de estado e fase única em um subinterpretador.

- gh-124153 [https://github.com/python/cpython/issues/124153]: Add **`PyType_GetBaseByToken()`** and **`Py_tp_token`** slot for easier type checking, related to **PEP 489** [https://peps.python.org/pep-0489/] and **PEP 630** [https://peps.python.org/pep-0630/].

- gh-124127 [https://github.com/python/cpython/issues/124127]: In the limited C API 3.14 and newer, **`Py_REFCNT()`** is now implemented as an opaque function call to hide implementation details. Patch by Victor Stinner.

- gh-123880 [https://github.com/python/cpython/issues/123880]: Corrige um bug que impedia importações circulares de módulos de extensão que usam inicialização de fase única.

- gh-123909 [https://github.com/python/cpython/issues/123909]: `PyType_FromSpec()`, `PyType_FromSpecWithBases()` and `PyType_FromModuleAndSpec()` will now fail if the metaclass of the new type has custom `tp_new`.

- gh-107954 [https://github.com/python/cpython/issues/107954]: Add functions to configure the Python initialization (**PEP 741** [https://peps.python.org/pep-0741/]):

  - `PyInitConfig_Create()`
  - `PyInitConfig_Free()`
  - `PyInitConfig_GetError()`
  - `PyInitConfig_GetExitCode()`
  - `PyInitConfig_HasOption()`
  - `PyInitConfig_GetInt()`
  - `PyInitConfig_GetStr()`
  - `PyInitConfig_GetStrList()`
  - `PyInitConfig_FreeStrList()`
  - `PyInitConfig_SetInt()`
  - `PyInitConfig_SetStr()`
  - `PyInitConfig_SetStrList()`
  - `PyInitConfig_AddModule()`
  - `Py_InitializeFromInitConfig()`

  Patch por Victor Stinner.

- gh-107954 [https://github.com/python/cpython/issues/107954]: Add functions to get and set the current runtime Python configuration:

  - `PyConfig_Get()`
  - `PyConfig_GetInt()`
  - `PyConfig_Set()`
  - `PyConfig_Names()`

  Patch por Victor Stinner.

- gh-123465 [https://github.com/python/cpython/issues/123465]: `Py_RELATIVE_OFFSET` is now allowed in `PyMemberDef` for the special

offset member "`__vectorcalloffset__`", as well as the discouraged special offset members "`__dictoffset__`" and "`__weaklistoffset__`"

- gh-100554 [https://github.com/python/cpython/issues/100554]: Added a slot `Py_tp_vectorcall` to set **`tp_vectorcall`** via the **`PyType_FromSpec()`** function family. Limited API extensions can use this feature to provide more efficient vector call-based implementation of `__new__` and `__init__`.

- gh-122854 [https://github.com/python/cpython/issues/122854]: Add **`Py_HashBuffer()`** to compute and return the hash value of a buffer. Patch by Antoine Pitrou and Victor Stinner.

- gh-122728 [https://github.com/python/cpython/issues/122728]: Corrige **`PyEval_GetLocals()`** para evitar **`SystemError`** (“argumento ruim para função interna”). Patch por Victor Stinner.

- gh-116622 [https://github.com/python/cpython/issues/116622]: Faz **`PyObject_Print`** contornar um bug no Android e OpenBSD que o impedia de levantar uma exceção ao tentar gravar em um fluxo somente leitura.

- gh-105201 [https://github.com/python/cpython/issues/105201]: Add **`PyIter_NextItem()`** to replace **`PyIter_Next()`**, which has an ambiguous return value. Patch by Irit Katriel and Erlend Aasland.

- gh-121489 [https://github.com/python/cpython/issues/121489]: Exporta **`_PyBytes_Join()`** privada novamente.

- gh-121645 [https://github.com/python/cpython/issues/121645]: Add **`PyBytes_Join(sep, iterable)`** function, similar to `sep.join(iterable)` in Python. Patch by Victor Stinner.

- gh-89364 [https://github.com/python/cpython/issues/89364]: Exporta a função **`PySignal_SetWakeupFd()`**. Anteriormente, a função era documentada, mas não podia ser usada em código de terceiros. Patch por Victor Stinner.

- gh-113993 [https://github.com/python/cpython/issues/113993]: **`PyUnicode_InternInPlace()`** não impede mais que seu argumento seja coletado como lixo.

  Várias funções que usam `char *` agora são documentadas como possivelmente impedindo que objetos de string sejam coletados como lixo; consulte a documentação para obter detalhes: **`PyUnicode_InternFromString()`**, **`PyDict_SetItemString()`**, **`PyObject_SetAttrString()`**, **`PyObject_DelAttrString()`**, **`PyUnicode_InternFromString()`** e funções de conveniência `PyModule_Add*`.

- gh-113601 [https://github.com/python/cpython/issues/113601]: Removidas asserções de construção de depuração relacionadas a strings internas, que eram falsamente acionadas por extensões ABI estáveis.

- gh-112136 [https://github.com/python/cpython/issues/112136]: Restaura a estrutura privada `_PyArg_Parser` e a função privada `_PyArg_ParseTupleAndKeywordsFast()`, removidas anteriormente no Python 3.13 alpha 1. Patch por Victor Stinner.

- gh-120642 [https://github.com/python/cpython/issues/120642]: Remova o tipo privado `_Py_CODEUNIT` da API C pública. O cabeçalho interno `pycore_code.h` agora deve ser usado para obter esse tipo interno. Patch por Victor Stinner.

- gh-120858 [https://github.com/python/cpython/issues/120858]: **`PyDict_Next()`** não trava mais o dicionário na construção com threads livres. O bloqueio precisa ser feito pelo chamador em todo o laço de iteração.

- gh-120642 [https://github.com/python/cpython/issues/120642]: Remove as seguintes funções instáveis:

  - `PyUnstable_Replace_Executor()`
  - `PyUnstable_SetOptimizer()`
  - `PyUnstable_GetOptimizer()`
  - `PyUnstable_GetExecutor()`
  - `PyUnstable_Optimizer_NewCounter()`

- `PyUnstable_Optimizer_NewUOpOptimizer()`

Patch por Victor Stinner.

- gh-120389 [https://github.com/python/cpython/issues/120389]: Add new functions to convert C `<stdint.h>` numbers from/to Python `int`:

  - `PyLong_FromInt32()`
  - `PyLong_FromUInt32()`
  - `PyLong_FromInt64()`
  - `PyLong_FromUInt64()`
  - `PyLong_AsInt32()`
  - `PyLong_AsUInt32()`
  - `PyLong_AsInt64()`
  - `PyLong_AsUInt64()`

  Patch por Victor Stinner.

- gh-120600 [https://github.com/python/cpython/issues/120600]: In the limited C API 3.14 and newer, `Py_TYPE()` is now implemented as an opaque function call to hide implementation details. Patch by Victor Stinner.

- gh-70278 [https://github.com/python/cpython/issues/70278]: `PyUnicode_FromFormat()` no longer produces the ending `\ufffd` character for truncated C string when use precision with `%s` and `%V`. It now truncates the string before the start of truncated multibyte sequences.

- gh-119771 [https://github.com/python/cpython/issues/119771]: Set `errno` in `_Py_c_pow()` on overflows. Patch by Sergey B Kirpichev.

- gh-119182 [https://github.com/python/cpython/issues/119182]: Add a new `PyUnicodeWriter` API to create a Python `str` object:

  - `PyUnicodeWriter_Create()`.
  - `PyUnicodeWriter_Discard()`.
  - `PyUnicodeWriter_Finish()`.
  - `PyUnicodeWriter_WriteChar()`.

  - `PyUnicodeWriter_WriteUTF8()`.
  - `PyUnicodeWriter_WriteUCS4()`.
  - `PyUnicodeWriter_WriteWideChar()`.
  - `PyUnicodeWriter_WriteStr()`.
  - `PyUnicodeWriter_WriteRepr()`.
  - `PyUnicodeWriter_WriteSubstring()`.
  - `PyUnicodeWriter_Format()`.
  - `PyUnicodeWriter_DecodeUTF8Stateful()`.

  Patch por Victor Stinner.

- gh-119613 [https://github.com/python/cpython/issues/119613]: Soft deprecate the `Py_MEMCPY` macro: use directly `memcpy()` instead. Patch by Victor Stinner.

- gh-119775 [https://github.com/python/cpython/issues/119775]: Creating `immutable types` with mutable bases was deprecated since 3.12 and now raises a `TypeError`.

- gh-119585 [https://github.com/python/cpython/issues/119585]: Corrige travamento quando um estado de thread que foi criado por `PyGILState_Ensure()` chama um destruidor que durante `PyThreadState_Clear()` chama de volta para **`PyGILState_Ensure()`** e `PyGILState_Release()`. Isso pode ocorrer quando na construção com threads livres ou ao usar variáveis de thread local cujos destruidores chamam **`PyGILState_Ensure()`**.

- gh-119613 [https://github.com/python/cpython/issues/119613]: Macros `Py_IS_NAN`, `Py_IS_INFINITY` and `Py_IS_FINITE` are soft deprecated.

- gh-119336 [https://github.com/python/cpython/issues/119336]: Restaura a função `_PyLong_NumBits()` removida. Ela é usada pelo projeto pywin32. Patch por Ethan Smith

- gh-119344 [https://github.com/python/cpython/issues/119344]: A API da seção crítica agora é pública como parte da API C não limitada.

- gh-119333 [https://github.com/python/cpython/issues/119333]: Add `PyContext_AddWatcher()` and `PyContext_ClearWatcher()` APIs to

register callbacks to receive notification on enter and exit of context objects.

- gh-119247 [https://github.com/python/cpython/issues/119247]: Adicionadas as macros `Py_BEGIN_CRITICAL_SECTION_SEQUENCE_FAST` e `Py_END_CRITICAL_SECTION_SEQUENCE_FAST` para tornar possível usar as APIs PySequence_Fast com segurança quando de threads livres, e atualizar str.join para funcionar sem que a GIL as use.

- gh-111389 [https://github.com/python/cpython/issues/111389]: Adicionar constante **`PyHASH_MULTIPLIER`**: multiplicador primo usado em string e vários outros hashes. Patch por Victor Stinner.

- gh-118771 [https://github.com/python/cpython/issues/118771]: Several C declarations with names that didn't start with the `Py` or `_Py` prefixes, which were added by mistake in 3.13 alpha and beta releases, were moved to internal headers.

- gh-116984 [https://github.com/python/cpython/issues/116984]: Faz mimalloc incluir relativo ao arquivo atual para evitar que incorporadores ou extensões precisem incluir `Internal/mimalloc` se eles já estiverem incluindo cabeçalhos CPython internos.

- gh-118789 [https://github.com/python/cpython/issues/118789]: Adiciona **`PyUnstable_Object_ClearWeakRefsNoCallbacks()`**, que limpa weakrefs sem chamar seus callbacks.

- gh-118789 [https://github.com/python/cpython/issues/118789]: Restaura `_PyWeakref_ClearRef` que foi removido anteriormente no Python 3.13 alpha 1.

- gh-117511 [https://github.com/python/cpython/issues/117511]: Torna o **`PyMutex`** público na API C não limitada.

- gh-116560 [https://github.com/python/cpython/issues/116560]: Add **`PyLong_GetSign()`** function. Patch by Sergey B Kirpichev.

- gh-97588 [https://github.com/python/cpython/issues/97588]: Fix creating bitfields in **`ctypes`** structures and unions. Fields no longer overlap.

- gh-87135 [https://github.com/python/cpython/issues/87135]: Attempting to acquire the GIL after runtime finalization has begun in a different thread now causes the thread to hang rather than terminate, which avoids potential crashes or memory corruption caused by attempting to terminate a thread that is running code not specifically designed to support termination. In most cases this hanging is harmless since the process will soon exit anyway.

  The `PyThread_exit_thread` function is now deprecated. Its behavior is inconsistent across platforms, and it can only be used safely in the unlikely case that every function in the entire call stack has been designed to support the platform-dependent termination mechanism. It is recommended that users of this function change their design to not require thread termination. In the unlikely case that thread termination is needed and can be done safely, users may migrate to calling platform-specific APIs such as `pthread_exit` (POSIX) or `_endthreadex` (Windows) directly.

- bpo-34008 [https://bugs.python.org/issue?@action=redirect&bpo=34008]: Adiciona `Py_IsInitialized` à lista de APIs que são seguras para chamar antes que o interpretador seja inicializado e atualiza os testes de incorporação para cobri-lo.

Construção

- gh-125269 [https://github.com/python/cpython/issues/125269]: Corrige a detecção de se `-latomic` é necessário ao fazer compilação cruzada do CPython usando o script configure.
- gh-123990 [https://github.com/python/cpython/issues/123990]: Remove `WITH_FREELISTS` macro and `--without-freelists` build configuration
- gh-124102 [https://github.com/python/cpython/issues/124102]: Update internal documentation under PCbuild, so it now correctly states that Windows requires VS2017 or later and Python 3.10 or later

- gh-124043 [https://github.com/python/cpython/issues/124043]: Construir usando `--with-trace-refs` é (temporariamente) proibido quando a GIL está desabilitado.
- gh-123418 [https://github.com/python/cpython/issues/123418]: Atualizada construção do Android para usar OpenSSL 3.0.15.
- gh-123297 [https://github.com/python/cpython/issues/123297]: Propaga o valor de `LDFLAGS` para `LDCXXSHARED` em `sysconfig`. Patch por Pablo Galindo
- gh-121634 [https://github.com/python/cpython/issues/121634]: Permite especificar o triplo de compilação de destino para WASI.
- gh-122578 [https://github.com/python/cpython/issues/122578]: Usa WASI SDK 24 para testes.
- gh-116622 [https://github.com/python/cpython/issues/116622]: Renomeia variável de construção `MODULE_LDFLAGS` de volta para `LIBPYTHON`, como é usada por sistemas de construção de pacotes (por exemplo, Meson).
- gh-118943 [https://github.com/python/cpython/issues/118943]: Corrige um problema em que o JIT experimental poderia ser construído várias vezes pelo alvo `make regen-all`, levando a possíveis condições de corrida em construções fortemente paralelizadas.
- gh-121996 [https://github.com/python/cpython/issues/121996]: Introduce ./configure –disable-safety and –enable-slower-safety options. Patch by Donghee Na.
- gh-120522 [https://github.com/python/cpython/issues/120522]: Adicionada uma opção `--with-app-store-compliance` para corrigir problemas conhecidos com processos de revisão da App Store do macOS/iOS.
- gh-120371 [https://github.com/python/cpython/issues/120371]: Suporta WASI SDK 22 ignorando explicitamente funções que são apenas stubs em wasi-libc.
- gh-121731 [https://github.com/python/cpython/issues/121731]: Corrige erro de construção mimalloc no GNU/Hurd
- gh-121487 [https://github.com/python/cpython/issues/121487]: Corrige aviso de descontinuação para ATOMIC_VAR_INIT em mimalloc.
- gh-121467 [https://github.com/python/cpython/issues/121467]: Corrige um bug do Makefile que impedia a instalação de arquivos de cabeçalho mimalloc.
- gh-121103 [https://github.com/python/cpython/issues/121103]: Em sistemas POSIX, excluindo instalações do framework macOS, o diretório lib para a construção com threads livres agora inclui um sufixo “t” para

evitar conflitos com uma instalação de construção padrão co-localizada.

- gh-120831 [https://github.com/python/cpython/issues/120831]: A versão mínima padrão do iOS foi aumentada para 13.0.
- gh-121082 [https://github.com/python/cpython/issues/121082]: Fix build failure when the developer use `--enable-pystats` arguments in configuration command after #118450.
- gh-120671 [https://github.com/python/cpython/issues/120671]: Corrige testes de configuração com falha devido a um espaço ausente ao anexar a CFLAGS.
- gh-120602 [https://github.com/python/cpython/issues/120602]: Manipula corretamente instalações LLVM com `LLVM_VERSION_SUFFIX` ao construir com `--enable-experimental-jit`.
- gh-120688 [https://github.com/python/cpython/issues/120688]: On WASI in debug mode, Python is now built with compiler flag `-O3` instead of `-Og`, to support more recursive calls. Patch by Victor Stinner.
- gh-118943 [https://github.com/python/cpython/issues/118943]: Corrige uma possível condição de corrida afetando construções paralelas configuradas com `--enable-experimental-jit`, em que **`FileNotFoundError`** poderia ser causado por outro processo que já estava movendo `jit_stencils.h.new` para `jit_stencils.h`.
- gh-120326 [https://github.com/python/cpython/issues/120326]: No Windows, corrige o erro de construção quando as opções `--disable-gil` e `--experimental-jit` são combinadas.
- gh-120291 [https://github.com/python/cpython/issues/120291]: Torna o script de shell `python-config` compatível com shells além do bash.
- gh-113565 [https://github.com/python/cpython/issues/113565]: Melhora as verificações de dependência **`curses`** e **`curses.panel`** em **configure**.
- gh-119729 [https://github.com/python/cpython/issues/119729]: Em sistemas POSIX, os nomes de arquivo pkg-config (`.pc`) agora incluem os sinalizadores ABI, que podem incluir debug ("d") e free-threaded ("t"). Por exemplo: * `python-3.14.pc` (padrão, construção não-debug) * `python-3.14d.pc` (padrão, construção debug) * `python-3.14t.pc` (construção com threads livres)
- gh-119400 [https://github.com/python/cpython/issues/119400]: `make_ssl_certs`, the script that prepares certificate data for the test suite, now allows specifying expiration dates.

- gh-115119 [https://github.com/python/cpython/issues/115119]: Retorna ao libmpdec empacotado se uma versão do sistema não puder ser encontrada.
- gh-119132 [https://github.com/python/cpython/issues/119132]: Atualiza **`sys.version`** para identificar se a construção é a construção padrão ou a construção com threads livres. Patch por Donghee Na.
- gh-118836 [https://github.com/python/cpython/issues/118836]: Corrige um `AssertionError` ao construir com `--enable-experimental-jit` e o compilador emite uma seção `SHT_NOTE`.
- gh-118943 [https://github.com/python/cpython/issues/118943]: Corrige uma possível condição de corrida que afeta construções paralelas configuradas com `--enable-experimental-jit`, nas quais erros de construção podem ser causados por um arquivo de cabeçalho gerado de forma incompleta.

Python 3.13.0 beta 1

Data de lançamento: 2024-05-08

Segurança

- gh-116741 [https://github.com/python/cpython/issues/116741]: Atualiza libexpat empacotado para 2.6.2
- gh-117233 [https://github.com/python/cpython/issues/117233]: Detecta suporte a BLAKE2, SHA3, Shake e SHA512 truncado na biblioteca libcrypto do OpenSSL no momento da construção. Isso permite que **`hashlib`** seja usado com bibliotecas que não suportam todos os algoritmos que o OpenSSL upstream suporta.

Núcleo e embutidos

- gh-118414 [https://github.com/python/cpython/issues/118414]: Adiciona opcodes instrumentados à asserção YIELD_VALUE para rastrear casos.
- gh-117953 [https://github.com/python/cpython/issues/117953]: Quando um módulo embutido ou de extensão é importado pela primeira vez, enquanto um subinterpretador está ativo, a função init do módulo agora é executada

pelo interpretador principal antes que a importação continue no subinterpretador. Consequentemente, os módulos init de fase única agora falham em um subinterpretador isolado sem a função init em execução sob esse interpretador, enquanto antes ele seria executado sob o subinterpretador *antes* de falhar, potencialmente deixando para trás o estado global e os retornos de chamada e, de outra forma, deixando o módulo em um estado inconsistente.

- gh-117549 [https://github.com/python/cpython/issues/117549]: Não usa a sintaxe do inicializador designado em funções inline em cabeçalhos internos. Eles causam problemas para usuários de C++ ou MSVC que ainda não estão usando o padrão C++ mais recente (C++20). Enquanto interno, pycore_backoff.h, é incluído (indiretamente, via pycore_code.h) por algum software de terceiros importante que faz isso para velocidade.

Biblioteca

- gh-95382 [https://github.com/python/cpython/issues/95382]: Melhora o desempenho de **`json.dumps()`** e **`json.dump()`** ao usar o argumento *indent*. Dependendo dos dados, a codificação usando **`json.dumps()`** com *indent* pode ser de 2 a 3 vezes mais rápida.

Núcleo e embutidos

- gh-116322 [https://github.com/python/cpython/issues/116322]: Em construções com `--disable-gil`, a GIL será habilitada durante o carregamento de módulos de extensão C. Se o módulo indicar que suporta execução sem a GIL, a GIL será desabilitada quando o carregamento for concluído. Caso contrário, a GIL permanecerá habilitada pelo restante da vida útil do interpretador. Esse comportamento não se aplica se a GIL tiver sido explicitamente habilitada ou desabilitada com `PYTHON_GIL` ou `-Xgil`.

- gh-118513 [https://github.com/python/cpython/issues/118513]: Corrige **`UnboundLocalError`** incorreta quando duas compreensões na mesma função referenciam o mesmo nome e, em uma compreensão, o nome é vinculado, enquanto na outra é um global implícito.

- gh-118518 [https://github.com/python/cpython/issues/118518]: Permite que o suporte Linux perf funcione sem ponteiros de quadro usando o suporte JIT avançado do perf. O recurso é ativado ao usar a variável de ambiente `PYTHON_PERF_JIT_SUPPORT` ou ao executar Python com `-Xperf_jit`. Patch por Pablo Galindo.

- gh-117514 [https://github.com/python/cpython/issues/117514]: Adiciona a função `sys._is_gil_enabled()` que retorna se a GIL está habilitada no momento. Na construção padrão, ela sempre retorna `True` porque a GIL está sempre habilitada. Na construção com threads livres, ela pode retornar `True` ou `False`.

- gh-118164 [https://github.com/python/cpython/issues/118164]: Interrompe um laço entre a implementação Python do módulo **`decimal`** e o código Python para conversão de inteiro para string. Também otimize a conversão de inteiro para string para valores no intervalo de 9_000 a 135_000 dígitos decimais.

- gh-118473 [https://github.com/python/cpython/issues/118473]: Corrige **`sys.set_asyncgen_hooks()`** para não ser parcialmente definido ao levantar **`TypeError`**.

- gh-118465 [https://github.com/python/cpython/issues/118465]: O compilador preenche o novo campo `__firstlineno__` em uma classe com o número da linha da primeira linha da definição da classe.

- gh-118492 [https://github.com/python/cpython/issues/118492]: Corrige um problema em que o cache de tipos pode expor um atributo acessado anteriormente quando um finalizador é executado.

- gh-117714 [https://github.com/python/cpython/issues/117714]: atualiza `async_generator.athrow().close()` e `async_generator.asend().close()` para fechar sua seção do gerador assíncrono subjacente

- gh-111201 [https://github.com/python/cpython/issues/111201]: O interpretador interativo agora é implementado em Python, o que permite uma série de novos recursos como cores, entrada multilinha, visualização de

histórico e modo de colagem. Contribuído por Pablo Galindo, Łukasz Langa e Lysandros Nikolaou com base no código do projeto PyPy.

- gh-74929 [https://github.com/python/cpython/issues/74929]: Implementa PEP 667: convertido `FrameType.f_locals` e `PyFrame_GetLocals()` para retornar um objeto proxy de gravação quando o quadro se refere a uma função ou compreensão.

- gh-116767 [https://github.com/python/cpython/issues/116767]: Corrige travamento no compilador em ‘async with’ que tem muitos gerenciadores de contexto.

- gh-118335 [https://github.com/python/cpython/issues/118335]: Altera como usar o interpretador de nível 2. Em vez de executar Python com `-X uops` ou definir a variável de ambiente `PYTHON_UOPS=1`, essa escolha agora é feita em tempo de construção configurando com `--enable-experimental-jit=interpreter`.

  Cuidado! Isso altera a variável de ambiente para habilitar ou desabilitar micro-ops para `PYTHON_JIT`. O antigo `PYTHON_UOPS` não é mais usado.

- gh-118306 [https://github.com/python/cpython/issues/118306]: Atualiza a compilação JIT para usar LLVM 18

- gh-118160 [https://github.com/python/cpython/issues/118160]: Escopos de anotação dentro de classes agora podem conter compreensões. No entanto, tais compreensões não são incorporadas em seu escopo pai em tempo de execução. Patch por Jelle Zijlstra.

- gh-118272 [https://github.com/python/cpython/issues/118272]: Corrige bug onde `generator.close` não libera os locais do quadro do gerador.

- gh-118216 [https://github.com/python/cpython/issues/118216]: Não considera importações `__future__` com pontos antes do nome do módulo.

- gh-118074 [https://github.com/python/cpython/issues/118074]: Garante que os objetos Executor no array COLD_EXITS não sejam considerados GC-

able (o que acessaria bytes fora do objeto).

- gh-107674 [https://github.com/python/cpython/issues/107674]: Usa carregamento preguiçoso do número da linha do quadro para melhorar o desempenho do rastreamento

- gh-118082 [https://github.com/python/cpython/issues/118082]: Melhora a mensagem **`SyntaxError`** para importações sem nomes, como nos casos `from x import` e `import`. Agora, ele indica aos usuários que **`import`** espera pelo menos um nome depois dele.

- gh-118090 [https://github.com/python/cpython/issues/118090]: Melhora a mensagem **`SyntaxError`** para colchetes de parâmetros de tipo vazios.

- gh-117958 [https://github.com/python/cpython/issues/117958]: Adicionado um método `get_jit_code()` para acessar o código de máquina compilado JIT do UOp Executor quando o JIT experimental estiver habilitado. Patch por Anthony Shaw.

- gh-117901 [https://github.com/python/cpython/issues/117901]: Adiciona uma opção para o codegen do compilador para salvar sequências de instruções aninhadas para introspecção.

- gh-116622 [https://github.com/python/cpython/issues/116622]: Redireciona stdout e stderr para o log do sistema quando incorporado em um aplicativo Android.

- gh-109118 [https://github.com/python/cpython/issues/109118]: o escopo de anotação dentro de escopos de classe agora pode conter lambdas.

- gh-117894 [https://github.com/python/cpython/issues/117894]: Impede que objetos `agen.aclose()` sejam reutilizados após `.throw()`.

- gh-117881 [https://github.com/python/cpython/issues/117881]: impede acesso simultâneo a um gerador assíncrono via athrow().throw() ou asend().throw()

- gh-117536 [https://github.com/python/cpython/issues/117536]: Corrige um **`RuntimeWarning`** ao chamar `agen.aclose().throw(Exception)`.

- gh-117755 [https://github.com/python/cpython/issues/117755]: Corrige alocador mimalloc para alocação de memória enorme (em torno de 8.589.934.592 GiB) no s390x. Patch por Victor Stinner.

- gh-117750 [https://github.com/python/cpython/issues/117750]: Corrige problema em que o dict de um objeto ficava fora de sincronia com os valores internos do objeto ao ser limpo. `obj.__dict__.clear()` agora limpa os valores internos, mas deixa o dict anexado ao objeto.

- gh-117431 [https://github.com/python/cpython/issues/117431]: Melhora o desempenho dos seguintes métodos **`bytes`** e **`bytearray`** adaptando-os à convenção de chamada **`METH_FASTCALL`**:

  - **`count()`**
  - **`find()`**
  - **`index()`**
  - **`rfind()`**
  - **`rindex()`**

- gh-117709 [https://github.com/python/cpython/issues/117709]: Acelera as chamadas para **`str()`** com argumento somente posicional, usando a convenção de chamada **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall`. Patch por Erlend Aasland.

- gh-117680 [https://github.com/python/cpython/issues/117680]: Dá a `_PyInstructionSequence` uma interface Python e use-a em testes.

- gh-115776 [https://github.com/python/cpython/issues/115776]: Objetos alocados estaticamente são, por definição, imortais, portanto, devem ser marcados como tal, independentemente de estarem em módulos de extensão ou não.

- gh-117385 [https://github.com/python/cpython/issues/117385]: Remove eventos `PY_MONITORING_EVENT_BRANCH` e

`PY_MONITORING_EVENT_EXCEPTION_HANDLED` não manipulados de `sys.settrace()`.

- gh-116322 [https://github.com/python/cpython/issues/116322]: Módulos de extensão podem indicar ao tempo de execução que podem ser executados sem a GIL. Módulos init multifásicos fazem isso chamando o fornecimento de `Py_MOD_GIL_NOT_USED` para o slot `Py_mod_gil`, enquanto módulos init monofásicos chamam `PyUnstable_Module_SetGIL(mod, Py_MOD_GIL_NOT_USED)` de sua função init.

- gh-116129 [https://github.com/python/cpython/issues/116129]: Implementa **PEP 696** [https://peps.python.org/pep-0696/], adicionando suporte para padrões em parâmetros de tipo. Patch por Jelle Zijlstra.

- gh-93502 [https://github.com/python/cpython/issues/93502]: Adiciona duas novas funções à API C, `PyRefTracer_SetTracer()` e `PyRefTracer_GetTracer()`, que permite rastrear a criação e destruição de objetos da mesma forma que o módulo `tracemalloc` faz. Patch por Pablo Galindo

- gh-107674 [https://github.com/python/cpython/issues/107674]: Melhora significativamente o desempenho de `sys.settrace()`

- gh-95754 [https://github.com/python/cpython/issues/95754]: Melhora a mensagem de erro quando um script que faz shadowing de um módulo da biblioteca padrão faz com que `AttributeError` seja levantada. Da mesma forma, melhorar a mensagem de erro quando um script que faz shadowing de um módulo de terceiros tenta acessar um atributo desse módulo de terceiros enquanto ainda está inicializando.

- gh-99180 [https://github.com/python/cpython/issues/99180]: Elimina indicadores de traceback não informativos em instruções `return` e `assignment` simples. Patch por Pablo Galindo.

- gh-105879 [https://github.com/python/cpython/issues/105879]: Permite que os argumentos *globals* e *locals* para `exec()` e `eval()` sejam passados como nomeados.

Biblioteca

- gh-118418 [https://github.com/python/cpython/issues/118418]: Uma **`DeprecationWarning`** agora é emitida se você não passar um valor para o novo parâmetro *type_params* de `typing._eval_type()` ou `typing.ForwardRef._evaluate()`. (Usar qualquer uma dessas funções privadas e não documentadas é desencorajado para começar, mas deixar de passar um valor para o parâmetro `type_params` pode levar a um comportamento incorreto no Python 3.12 ou mais recente.)

- gh-118660 [https://github.com/python/cpython/issues/118660]: Adiciona um segundo parâmetro de tipo opcional para **`typing.ContextManager`** e **`typing.AsyncContextManager`**, representando os tipos de retorno de **`__exit__()`** e **`__aexit__()`** respectivamente. Este parâmetro tem como padrão `bool | None`.

- gh-118650 [https://github.com/python/cpython/issues/118650]: O módulo `enum` permite que o método chamado `_repr_*` seja definido em tipos `Enum`.

- gh-118648 [https://github.com/python/cpython/issues/118648]: Adiciona padrões de parâmetros de tipo para **`typing.Generator`** e **`typing.AsyncGenerator`**.

- gh-101137 [https://github.com/python/cpython/issues/101137]: O tipo MIME `text/x-rst` agora é suportado por **`mimetypes`**.

- gh-118164 [https://github.com/python/cpython/issues/118164]: A implementação Python do módulo `decimal` poderia parecer travar em casos de potência relativamente pequena (como `2**117`) se a precisão do contexto fosse definida como um valor muito alto. Um método diferente para verificar resultados exatamente representáveis é usado agora que não depende da computação de `10**precision` (que poderia ser efetivamente muito grande para ser computada).

- gh-111744 [https://github.com/python/cpython/issues/111744]: `breakpoint()` e `pdb.set_trace()` agora entram no depurador imediatamente após a chamada, em vez de antes da próxima linha ser executada.

- gh-118500 [https://github.com/python/cpython/issues/118500]: Adiciona suporte `pdb` para zipapps

- gh-118406 [https://github.com/python/cpython/issues/118406]: Adiciona assinatura para objetos `sqlite3.Connection`.

- gh-101732 [https://github.com/python/cpython/issues/101732]: Usa uma função de tempo openssl compatível com Y2038 quando disponível.

- gh-118404 [https://github.com/python/cpython/issues/118404]: Corrige `inspect.signature()` para chamáveis não comparáveis.

- gh-118402 [https://github.com/python/cpython/issues/118402]: Corrige `inspect.signature()` para o resultado da chamada `functools.cmp_to_key()`.

- gh-116622 [https://github.com/python/cpython/issues/116622]: No Android, `sysconfig.get_platform` agora retorna o formato especificado por **PEP 738** [https://peps.python.org/pep-0738/].

- gh-118285 [https://github.com/python/cpython/issues/118285]: Permite especificar a assinatura de instâncias personalizadas do tipo de extensão pelo atributo `__text_signature__`. Especifica assinaturas de instâncias `operator.attrgetter`, `operator.itemgetter` e `operator.methodcaller`.

- gh-118314 [https://github.com/python/cpython/issues/118314]: Corrige um caso extremo no modo estrito `binascii.a2b_base64()`, onde o preenchimento excessivo não é detectado quando nenhum preenchimento é necessário.

- gh-118271 [https://github.com/python/cpython/issues/118271]: Adiciona os métodos **`PhotoImage read()`** para ler uma imagem de um arquivo e **`data()`** para obter os dados da imagem. Adiciona os parâmetros *background* e *grayscale* ao método **`write()`** de **`PhotoImage`**.

- gh-118225 [https://github.com/python/cpython/issues/118225]: Adiciona o método **`PhotoImage copy_replace()`** para copiar uma região de uma imagem

para outra, possivelmente com zoom de pixel e/ou subamostragem. Adicione o parâmetro *from_coords* aos métodos **`copy()`**, **`zoom()`** e **`subsample()`** da **`PhotoImage`**. Adiciona os parâmetros *zoom* e *subsample* ao método **`copy()`** de **`PhotoImage`**.

- gh-118221 [https://github.com/python/cpython/issues/118221]: Corrige um bug em que **`sqlite3.Connection.iterdump()`** poderia falhar se um **`row factory`** personalizado fosse usado. Patch por Erlend Aasland.

- gh-118013 [https://github.com/python/cpython/issues/118013]: Corrige regressão introduzida em gh-103193 [https://github.com/python/cpython/issues/103193] que significava que chamar **`inspect.getattr_static()`** em uma instância faria com que uma referência forte à classe dessa instância persistisse em um cache interno no módulo **`inspect`**. Isso causava consumo inesperado de memória se a classe fosse criada dinamicamente, a classe mantivesse referências fortes a outros objetos que ocupassem uma quantidade significativa de memória e o cache contivesse a única referência forte à classe. A correção para a regressão leva a uma lentidão em **`getattr_static()`**, mas a função ainda deve ser significativamente mais rápida do que era no Python 3.11. Patch por Alex Waygood.

- gh-118218 [https://github.com/python/cpython/issues/118218]: Acelera **`itertools.pairwise()`** no caso comum em até 1,8x.

- gh-117486 [https://github.com/python/cpython/issues/117486]: Melhora o comportamento das subclasses definidas pelo usuário de **`ast.AST`**. Essas classes agora não exigirão alterações no caso usual para estar em conformidade com as alterações de comportamento do módulo **`ast`** no Python 3.13. Patch por Jelle Zijlstra.

- gh-90848 [https://github.com/python/cpython/issues/90848]: Corrigida **`unittest.mock.create_autospec()`** para configurar o mock pai com argumentos nomeados.

- gh-118168 [https://github.com/python/cpython/issues/118168]: Corrige a substituição incorreta de argumentos quando **`typing.Unpack`** é usado

com a `tuple` embutido. `typing.Unpack` agora levanta `TypeError` quando usado com certos tipos inválidos. Patch por Jelle Zijlstra.

- gh-118131 [https://github.com/python/cpython/issues/118131]: Adiciona a interface de linha de comando para o módulo `random`. Patch por Hugo van Kemenade.

- gh-118107 [https://github.com/python/cpython/issues/118107]: Corrige `zipimport` leitura de arquivos ZIP64 com entradas de arquivo muito grandes ou muito deslocadas.

- gh-102511 [https://github.com/python/cpython/issues/102511]: Corrige `os.path.normpath()` para caminhos UNC no Windows. Acelera `os.path.splitroot()` com uma implementação nativa.

- gh-117535 [https://github.com/python/cpython/issues/117535]: Altera o nome de arquivo desconhecido de `warnings` de `sys` para `<sys>` para esclarecer que não é um nome de arquivo real.

- gh-114053 [https://github.com/python/cpython/issues/114053]: Corrige `NameError` errôneo ao chamar `typing.get_type_hints()` em uma classe que fazia uso de parâmetros de tipo **PEP 695** [https://peps.python.org/pep-0695/] em um módulo que tinha `from __future__ import annotations` no topo do arquivo. Patch por Alex Waygood.

- gh-116931 [https://github.com/python/cpython/issues/116931]: Adiciona verificação do parâmetro *fileobj* para `tarfile.TarFile.addfile()`

- gh-117995 [https://github.com/python/cpython/issues/117995]: Não levanta `DeprecationWarning` quando uma sequência de parâmetros for usada para vincular marcadores de posição indexados e sem nome. Veja também gh-100668 [https://github.com/python/cpython/issues/100668].

- gh-80361 [https://github.com/python/cpython/issues/80361]: Corrige TypeError em `email.message.Message.get_payload()` quando o charset é codificado em **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html].

- gh-86650 [https://github.com/python/cpython/issues/86650]: Corrige IndexError ao analisar alguns e-mails com Message-ID inválido (incluindo endereços únicos gerados pelo Microsoft Outlook).

- gh-117691 [https://github.com/python/cpython/issues/117691]: Melhora as mensagens de erro emitidas pelos avisos de descontinuação de **`tarfile`** relacionados ao PEP 706. Se um argumento `filter` não for fornecido para `extract()` ou `extractall`, o aviso de descontinuação agora aponta para a linha no código do usuário onde a função relevante foi chamada. Patch por Alex Waygood.

- gh-115874 [https://github.com/python/cpython/issues/115874]: Corrige um possível segfault durante a coleta de lixo de objetos `_asyncio.FutureIter`. Patch por Savannah Ostrowski.

- gh-115060 [https://github.com/python/cpython/issues/115060]: Acelera **`pathlib.Path.glob()`** omitindo uma chamada inicial **`is_dir()`**. Como resultado dessa alteração, **`glob()`** não pode mais levantar **`OSError`**.

- gh-77102 [https://github.com/python/cpython/issues/77102]: O módulo **`site`** agora analisa o arquivo `.pth` com UTF-8 primeiro, e codificação da localidade se `UnicodeDecodeError` acontecer. Ele suportava apenas codificação da localidade antes.

- gh-76785 [https://github.com/python/cpython/issues/76785]: Nós expusemos o módulo de baixo nível **`_interpreters`** para o bem da implementação PyPI de **PEP 734** [https://peps.python.org/pep-0734/]. Às vezes, ele estava disponível como o módulo **`_xxsubinterpreters`** e era usado anteriormente apenas para testes. Na maior parte, ele deve ser considerado um módulo interno, como **`_thread`** e **`_imp`**. Veja https://discuss.python.org/t/pep-734-multiple-interpreters-in-the-stdlib/41147/26.

- gh-115060 [https://github.com/python/cpython/issues/115060]: Acelera **`pathlib.Path.glob()`** não escaneando diretórios para segmentos de padrões não curinga.

- gh-117727 [https://github.com/python/cpython/issues/117727]: Acelera **`pathlib.Path.iterdir()`** usando **`os.scandir()`** internamente.

- gh-117586 [https://github.com/python/cpython/issues/117586]: Acelera **`pathlib.Path.walk()`** trabalhando com strings internamente.

- gh-117722 [https://github.com/python/cpython/issues/117722]: Altera o novo suporte a múltiplos separadores em **`asyncio.StreamReader.readuntil()`** para aceitar apenas tuplas de separadores em vez de iteráveis arbitrários.

- gh-117692 [https://github.com/python/cpython/issues/117692]: Corrige um bug quando **`doctest.DocTestFinder`** estava falhando em `builtin_function_or_method` encapsulado.

- gh-117348 [https://github.com/python/cpython/issues/117348]: Desempenho de tempo de importação amplamente restaurado do configparser evitando dataclasses.

- gh-117641 [https://github.com/python/cpython/issues/117641]: Aceleração de **`os.path.commonpath()`** no Unix.

- gh-117663 [https://github.com/python/cpython/issues/117663]: Corrige `_simple_enum` para detectar apelidos quando vários argumentos estão presentes, mas apenas um é o valor do membro.

- gh-117636 [https://github.com/python/cpython/issues/117636]: Aceleração de **`os.path.join()`**.

- gh-117618 [https://github.com/python/cpython/issues/117618]: Suporte a `package.module` como `filename` para o comando `break` de **`pdb`**

- gh-102247 [https://github.com/python/cpython/issues/102247]: os códigos de status enum com constantes em http.HTTPStatus são atualizados para incluir os nomes do RFC9110. Este RFC inclui alguns status HTTP usados anteriormente apenas para WEBDAV e atribui nomes mais genéricos a eles.

As constantes antigas são preservadas para retrocompatibilidade.

- gh-117607 [https://github.com/python/cpython/issues/117607]: Aceleração de `os.path.relpath()`.

- gh-117586 [https://github.com/python/cpython/issues/117586]: Aceleração de `pathlib.Path.glob()` trabalhando com strings internamente.

- gh-117225 [https://github.com/python/cpython/issues/117225]: Adiciona cor à saída do doctest. Patch por Hugo van Kemenade.

- gh-117566 [https://github.com/python/cpython/issues/117566]: `ipaddress.IPv6Address.is_loopback()` agora retornará `True` para endereços de loopback mapeados em IPv4, ou seja, endereços no espaço de endereço `::ffff:127.0.0.0/104`.

- gh-117546 [https://github.com/python/cpython/issues/117546]: Corrige o problema em que `os.path.realpath()` parou de resolver links simbólicos após encontrar um loop de links simbólicos no POSIX.

- gh-116720 [https://github.com/python/cpython/issues/116720]: Comportamento aprimorado de `asyncio.TaskGroup` quando um cancelamento externo colide com um cancelamento interno. Por exemplo, quando dois grupos de tarefas são aninhados e ambos experimentam uma exceção em uma tarefa filha simultaneamente, era possível que o grupo de tarefas externo se comportasse mal, porque seu cancelamento interno foi engolido pelo grupo de tarefas interno.

  No caso em que um grupo de tarefas é cancelado externamente e também deve levantar uma `ExceptionGroup`, ele agora chamará o método `cancel()` da tarefa pai. Isso garante que uma `asyncio.CancelledError` será levantada no próximo `await`, para que o cancelamento não seja perdido.

  Um benefício adicional dessas mudanças é que os grupos de tarefas agora preservam a contagem de cancelamentos (`asyncio.Task.cancelling()`).

Para lidar com alguns casos extremos, `asyncio.Task.uncancel()` agora pode redefinir o sinalizador não documentado `_must_cancel` quando a contagem de cancelamentos chegar a zero.

- gh-117516 [https://github.com/python/cpython/issues/117516]: Adiciona `typing.TypeIs`, implementando **PEP 742** [https://peps.python.org/pep-0742/]. Patch por Jelle Zijlstra.

- gh-117503 [https://github.com/python/cpython/issues/117503]: Corrige suporte de nomes de usuários não-ASCII em caminhos de bytes em `os.path.expanduser()` no Posix.

- gh-117394 [https://github.com/python/cpython/issues/117394]: `os.path.ismount()` agora é 2-3 vezes mais rápido se o usuário tiver permissões.

- gh-117313 [https://github.com/python/cpython/issues/117313]: Trata apenas `'\\n'`, `'\\r'` e `'\\r\\n'` como separadores de linha ao redobrar as mensagens `email`. Preserva os caracteres de controle `'\\v'`, `'\\f'`, `'\\x1c'`, `'\\x1d'` e `'\\x1e'` e os separadores de linha Unicode `'\\x85'`, `'\\u2028'` e `'\\u2029'` como estão.

- gh-117142 [https://github.com/python/cpython/issues/117142]: Converte `_ctypes` para inicialização multifásica (**PEP 489** [https://peps.python.org/pep-0489/]).

- gh-66543 [https://github.com/python/cpython/issues/66543]: Adiciona a função `mimetypes.guess_file_type()` que funciona com o caminho do arquivo. Passar o caminho do arquivo em vez da URL em `guess_type()` está suavemente descontinuado.

- gh-68583 [https://github.com/python/cpython/issues/68583]: CLI de webbrowser: substitui getopt por argparse, adiciona opções longas. Patch de Hugo van Kemenade.

- gh-116871 [https://github.com/python/cpython/issues/116871]: Sugestões de nome para `AttributeError` e `ImportError` agora incluem apenas nomes sublinhados se o nome original estiver sublinhado.

- gh-116023 [https://github.com/python/cpython/issues/116023]: Não mostra campos vazios (valor `None` ou `[]`) em **`ast.dump()`** por padrão. Adiciona o parâmetro `show_empty=False` para mostrá-los opcionalmente.

- gh-115961 [https://github.com/python/cpython/issues/115961]: Adicionados atributos **`name`** e **`mode`** para objetos arquivo ou similares compactados e arquivados nos módulos **`bz2`**, **`lzma`**, **`tarfile`** e **`zipfile`**. O valor do atributo **`mode`** de **`gzip.GzipFile`** foi alterado de inteiro (`1` ou `2`) para string (`'rb'` ou `'wb'`). O valor do atributo **`mode`** do objeto arquivo ou similar legível retornado por **`zipfile.ZipFile.open()`** foi alterado de `'r'` para `'rb'`.

- gh-82062 [https://github.com/python/cpython/issues/82062]: Corrige **`inspect.signature()`** para manipular corretamente os padrões de parâmetros em métodos em módulos de extensão que usam nomes definidos no espaço de nomes do módulo.

- gh-83856 [https://github.com/python/cpython/issues/83856]: Honra **`atexit`** para todos os métodos de inicialização **`multiprocessing`**

- gh-113081 [https://github.com/python/cpython/issues/113081]: Imprime exceção colorida assim como o traceback embutido em **`pdb`**

- gh-112855 [https://github.com/python/cpython/issues/112855]: Acelera a serialização com pickle de objetos **`pathlib.PurePath`**. Patch por Barney Gale.

- gh-111744 [https://github.com/python/cpython/issues/111744]: Suporte a eventos de opcode em **`bdb`**

- gh-109617 [https://github.com/python/cpython/issues/109617]: **`ncurses`**: corrigiu uma falha que poderia ocorrer no macOS 13 ou anterior quando o Python era criado com o SDK do Apple Xcode 15.

- gh-83151 [https://github.com/python/cpython/issues/83151]: Ativadas declarações e avaliações arbitrárias no shell **`pdb`** para acessar as variáveis locais do quadro atual, o que tornou possível que códigos multiescopo, como geradores ou funções aninhadas, funcionassem.

- gh-110209 [https://github.com/python/cpython/issues/110209]: Adicionado **`__class_getitem__()`** a **`types.GeneratorType`** e **`types.CoroutineType`** para fins de dicas de tipo. Patch por James Hilton-Balfe.

- gh-108191 [https://github.com/python/cpython/issues/108191]: O **`types.SimpleNamespace`** agora aceita um argumento posicional opcional que especifica valores iniciais de atributos como um dict ou um iterável de pares de chave-valor.

- gh-62090 [https://github.com/python/cpython/issues/62090]: Corrige erros de asserção causados por espaços em branco em metavars ou grupos suprimidos com `SUPPRESS` em **`argparse`** simplificando a formatação de uso. Patch por Ali Hamdan.

- gh-102402 [https://github.com/python/cpython/issues/102402]: Ajusta `logging.LogRecord` para usar `time.time_ns()` e corrige um pequeno bug relacionado à matemática de ponto flutuante.

- gh-100242 [https://github.com/python/cpython/issues/100242]: Traz a implementação Python pura `functools.partial.__new__` mais alinhada com a implementação C ao não apenas verificar sempre a presença do atributo `'func'` no primeiro argumento de `partial`. Em vez disso, tanto a versão Python quanto a versão C realizam uma verificação `isinstance(func, partial)` no primeiro argumento de `partial`.

- gh-99730 [https://github.com/python/cpython/issues/99730]: Solicitações HEAD não são mais atualizadas para solicitações GET durante redirecionamentos em urllib.

- gh-66410 [https://github.com/python/cpython/issues/66410]: Define o módulo global **`wantobjects`** **`tkinter`** como `2` antes de criar o objeto **`Tk`** ou chamar o método **`wantobjects()`** do objeto **`Tk`** com o argumento `2` agora faz com que argumentos para retornos de chamada registrados no módulo **`tkinter`** sejam passados como vários objetos Python (`int`, `float`, `bytes`, `tuple`), dependendo de sua representação interna em

Tcl, em vez de sempre `str`. **`tkinter.wantobjects`** agora está definido como `2` por padrão.

- bpo-40943 [https://bugs.python.org/issue?@action=redirect&bpo=40943]: Corrige vários IndexError ao analisar e-mails com Message-ID, endereço, rotas, etc. truncados, por exemplo, `example@`.

- bpo-39324 [https://bugs.python.org/issue?@action=redirect&bpo=39324]: Adiciona mapeamento de tipo mime para .md <-> text/markdown

- bpo-18108 [https://bugs.python.org/issue?@action=redirect&bpo=18108]: **`shutil.chown()`** agora suporta argumentos nomeados *dir_fd* e *follow_symlinks*.

- bpo-30988 [https://bugs.python.org/issue?@action=redirect&bpo=30988]: Corrige análise de e-mails com cabeçalhos de endereço inválidos com um ponto inicial ou final. Patch por tsufeki.

- bpo-32839 [https://bugs.python.org/issue?@action=redirect&bpo=32839]: Adiciona o método **`after_info()`** para widgets Tkinter.

Documentação

- gh-117928 [https://github.com/python/cpython/issues/117928]: A versão mínima do Sphinx necessária para a documentação agora é 6.2.1.

Construção

- gh-118734 [https://github.com/python/cpython/issues/118734]: Corrige a construção do Windows quando invocada diretamente (não por meio do script `build.bat`) sem especificar um valor para `UseTIER2`.
- gh-115119 [https://github.com/python/cpython/issues/115119]: A opção `configure` **`--with-system-libmpdec`** agora tem como padrão `yes`. A cópia empacotada de `libmpdecimal` será removida no Python 3.15.
- gh-117845 [https://github.com/python/cpython/issues/117845]: Corrige a construção em versões recentes do libedit detectando assinaturas de hook readline em **configure**.

- gh-116622 [https://github.com/python/cpython/issues/116622]: Um projeto de testbed foi adicionado para executar o conjunto de testes no Android.
- gh-117645 [https://github.com/python/cpython/issues/117645]: Aumenta o tamanho da pilha WASI de 512 KiB para 8 MiB e a memória inicial de 10 MiB para 20 MiB. Patch por Victor Stinner.
- gh-115119 [https://github.com/python/cpython/issues/115119]: **configure** agora usa **pkg-config** para detectar dependências `decimal` se a opção `--with-system-libmpdec` for fornecida.

Windows

- gh-115119 [https://github.com/python/cpython/issues/115119]: Atualiza o instalador do Windows para usar libmpdecimal 4.0.0.
- gh-118486 [https://github.com/python/cpython/issues/118486]: `os.mkdir()` agora aceita *modo* de `0o700` para restringir o novo diretório ao usuário atual.
- gh-118347 [https://github.com/python/cpython/issues/118347]: Corrige atualizações do iniciador que não sendo instaladas.
- gh-118293 [https://github.com/python/cpython/issues/118293]: O módulo `multiprocessing` agora passa o sinalizador `STARTF_FORCEOFFFEEDBACK` ao gerar processos para informar ao Windows para não alterar o cursor do mouse.
- gh-115009 [https://github.com/python/cpython/issues/115009]: Atualiza o instalador do Windows para usar o SQLite 3.45.3.
- gh-90329 [https://github.com/python/cpython/issues/90329]: Suprime o aviso exibido na criação do ambiente virtual quando os caminhos solicitados e criados diferem apenas por um nome curto (estilo 8.3). Os avisos continuarão a ser exibidos se uma junção ou link simbólico no caminho fez com que o venv fosse criado em um local diferente do solicitado originalmente.
- gh-117786 [https://github.com/python/cpython/issues/117786]: Corrige ambientes virtuais que não iniciam corretamente quando criados a partir de uma instalação da Store.

macOS

- gh-115119 [https://github.com/python/cpython/issues/115119]: Atualiza o instalador do macOS para usar libmpdecimal 4.0.0.

- gh-114099 [https://github.com/python/cpython/issues/114099]: O uso do símbolo do pré-processador iOS foi tornado compatível com SDKs macOS mais antigos.
- gh-115009 [https://github.com/python/cpython/issues/115009]: Atualiza o instalador macOS para usar o SQLite 3.45.3.
- gh-91629 [https://github.com/python/cpython/issues/91629]: Use as configurações `~/.config/fish/conf.d` e **fish_add_path** para definir **`PATH`** ao instalar para o shell Fish.

IDLE

- bpo-34774 [https://bugs.python.org/issue?@action=redirect&bpo=34774]: Use o tema de cores selecionado pelo usuário para Help => IDLE Doc.

API C

- gh-118124 [https://github.com/python/cpython/issues/118124]: Corrige **`Py_BUILD_ASSERT`** e **`Py_BUILD_ASSERT_EXPR`** para expressões não constantes: use `static_assert()` no C11 e mais recentes. Patch por Victor Stinner.

- gh-110850 [https://github.com/python/cpython/issues/110850]: Adiciona variante "Raw" das funções PyTime

  - **`PyTime_MonotonicRaw()`**
  - **`PyTime_PerfCounterRaw()`**
  - **`PyTime_TimeRaw()`**

  Patch por Victor Stinner.

- gh-117987 [https://github.com/python/cpython/issues/117987]: Restaura funções removidas no Python 3.13 alpha 1:

  - **`Py_SetPythonHome()`**
  - **`Py_SetProgramName()`**
  - **`PySys_SetArgvEx()`**
  - **`PySys_SetArgv()`**

Patch por Victor Stinner.

- gh-117929 [https://github.com/python/cpython/issues/117929]: Restaura função removida `PyEval_InitThreads()`. Patch por Victor Stinner.

- gh-117534 [https://github.com/python/cpython/issues/117534]: Melhora a lógica de validação na implementação C de `datetime.datetime.fromisoformat()` para lidar melhor com anos inválidos. Patch por Vlad Efanov.

- gh-68114 [https://github.com/python/cpython/issues/68114]: Corrigido o tratamento de skipitem() dos antigos formatadores ‘w’ e ‘w#’. Eles não são mais suportados e agora levantam uma exceção se usados.

- gh-111997 [https://github.com/python/cpython/issues/111997]: Adiciona uma API C para disparar eventos de monitoramento.

Python 3.13.0 alpha 6

Data de lançamento: 2024-04-09

Núcleo e embutidos

- gh-117494 [https://github.com/python/cpython/issues/117494]: Refatorada a estrutura de dados da sequência de instruções de compile.c para instruction_sequence.c.

- gh-116968 [https://github.com/python/cpython/issues/116968]: Introduzido um tipo de contador de backoff unificado de 16 bits (`_Py_BackoffCounter`), compartilhado entre o especializador adaptativo de Nível 1 e o otimizador de Nível 2. A API usada para contadores de especialização adaptável é alterada, mas o comportamento é (supostamente) idêntico.

  O comportamento dos contadores de Nível 2 é alterado:

  - Não há mais limites dinâmicos (nunca os variamos).
  - Todos os contadores agora usam o mesmo backoff exponencial.

    - O contador para `JUMP_BACKWARD` começa a contagem regressiva a partir de 16.
    - A `temperature` nas saídas laterais começa a contagem regressiva a partir de 64.

- gh-117431 [https://github.com/python/cpython/issues/117431]: Melhora o desempenho dos seguintes métodos **`bytes`** e **`bytearray`** adaptando-os à convenção de chamada **`METH_FASTCALL`**:

    - **`endswith()`**
    - **`startswith()`**

- gh-117431 [https://github.com/python/cpython/issues/117431]: Melhora o desempenho dos seguintes métodos **`str`** adaptando-os à convenção de chamada **`METH_FASTCALL`**:

    - **`count()`**
    - **`endswith()`**
    - **`find()`**
    - **`index()`**
    - **`rfind()`**
    - **`rindex()`**
    - **`startswith()`**

- gh-117411 [https://github.com/python/cpython/issues/117411]: Move `PyFutureFeatures` para um cabeçalho interno e torna-o privado.

- gh-109120 [https://github.com/python/cpython/issues/109120]: Adicionado identificador de expressões de estrela incorretas, por exemplo `f(3, *)`. Patch por Grigoryev Semyon

- gh-117266 [https://github.com/python/cpython/issues/117266]: Corrige travamentos para certas subclasses criadas pelo usuário de **`ast.AST`**. Essas classes agora devem definir o atributo `_field_types`.

- gh-99108 [https://github.com/python/cpython/issues/99108]: Atualizado o código C do projeto HACL* [https://github.com/hacl-star/hacl-star] do **`hashlib`** embutido a partir do upstream que usamos para muitas

implementações quando elas não estão presentes via OpenSSL em uma determinada construção. Isso também evita o raro potencial de um problema de vinculação de regra de definição de nome de símbolo C.

- gh-117108 [https://github.com/python/cpython/issues/117108]: Altera o antigo bit de espaço de objetos na geração jovem de 0 para gcstate->visited, para que quaisquer objetos criados durante o GC tenham o antigo bit definido corretamente se forem movidos para a geração antiga.

- gh-117108 [https://github.com/python/cpython/issues/117108]: O GC do ciclo agora escolhe o tamanho dos incrementos com base no tamanho total do heap, em vez da taxa de criação de objetos. Isso garante que ele possa acompanhar o crescimento dos heaps.

- gh-116735 [https://github.com/python/cpython/issues/116735]: Para `INSTRUMENTED_CALL_FUNCTION_EX`, define `arg0` como `sys.monitoring.MISSING` em vez de `None` para o evento **`CALL`**.

- gh-113964 [https://github.com/python/cpython/issues/113964]: Iniciar novas threads e a criação de processos por meio de **`os.fork()`** agora são impedidos somente quando todas as threads não daemon são encerradas.

- gh-116626 [https://github.com/python/cpython/issues/116626]: Garante que `INSTRUMENTED_CALL_FUNCTION_EX` sempre emita **`CALL`**

- gh-116554 [https://github.com/python/cpython/issues/116554]: `list.sort()` agora explora mais casos de ordenação parcial, particularmente aqueles com longas execuções descendentes com subexecuções de valores iguais. Elas são reconhecidas como execuções únicas agora (anteriormente, cada bloco de valores repetidos fazia com que uma nova execução fosse criada).

- gh-114099 [https://github.com/python/cpython/issues/114099]: Adicionado um carregador que pode descobrir módulos de extensão em uma pasta Frameworks no estilo iOS.

- gh-115775 [https://github.com/python/cpython/issues/115775]: O compilador preenche o novo campo `__static_attributes__` em uma classe com os nomes dos atributos desta classe que são acessados por meio de self.X de qualquer função em seu corpo.

- gh-115776 [https://github.com/python/cpython/issues/115776]: A array de valores, a estrutura `PyDictValues` agora é incorporada ao objeto durante a alocação. Isso fornece melhor desempenho no caso comum e não degrada tanto quando o `__dict__` do objeto é materializado.

- gh-108362 [https://github.com/python/cpython/issues/108362]: Implementa um coletor de lixo cíclico incremental. Ao coletar a geração antiga em incrementos, não há necessidade de uma varredura completa do heap. Isso pode reduzir enormemente o tempo máximo de pausa para programas com grandes heaps.

  Reduz o número de gerações de três para duas. A geração antiga é dividida em dois espaços, “visitado” e “pendente”.

  A coleta acontece em duas etapas:: * Um incremento é formado a partir da geração jovem e uma pequena parte do espaço pendente. * Este incremento é escaneado e os sobreviventes são movidos para o final do espaço visitado.

  Quando o espaço de coleta fica vazio, os dois espaços são trocados.

Biblioteca

- gh-109870 [https://github.com/python/cpython/issues/109870]: Dataclasses agora chamam **`exec()`** uma vez por dataclass, em vez de uma vez por método adicionado. Isso pode acelerar a criação de dataclass em até 20%.
- gh-97901 [https://github.com/python/cpython/issues/97901]: O tipo MIME `text/rtf` agora é suportado por **`mimetypes`**.

Núcleo e embutidos

- bpo-24612 [https://bugs.python.org/issue?@action=redirect&bpo=24612]: Melhora o `SyntaxError` que acontece quando ‘not’ aparece após um operador. Patch por Pablo Galindo

Biblioteca

- gh-117648 [https://github.com/python/cpython/issues/117648]: Melhora o desempenho de `os.path.join()` e `os.path.expanduser()`.

- gh-117584 [https://github.com/python/cpython/issues/117584]: Raise `TypeError` for non-paths in `posixpath.relpath`.

- gh-117467 [https://github.com/python/cpython/issues/117467]: Preserve mailbox ownership when rewriting in `mailbox.mbox.flush`. Patch by Tony Mountifield.

- gh-114848 [https://github.com/python/cpython/issues/114848]: Levanta `FileNotFoundError` quando `getcwd()` retornar ‘(unreachable)’, o que pode acontecer no Linux >= 2.6.36 com glibc < 2.27.

- gh-117459 [https://github.com/python/cpython/issues/117459]: **`asyncio.asyncio.run_coroutine_threadsafe()`** agora mantém o traceback de **`CancelledError`**, `TimeoutError` e **`InvalidStateError`** que são levantados na corrotina.

- gh-117381 [https://github.com/python/cpython/issues/117381]: Corrige mensagem de erro para **`ntpath.commonpath()`**.

- gh-117337 [https://github.com/python/cpython/issues/117337]: Descontinua funções não documentadas **`glob.glob0()`** e **`glob.glob1()`**. Use `glob.glob()` e passe um diretório para seu argumento *root_dir*.

- gh-117349 [https://github.com/python/cpython/issues/117349]: Otimiza várias funções em `os.path`.

- gh-117348 [https://github.com/python/cpython/issues/117348]: Refatorado **`configparser.RawConfigParser._read()`** para reduzir a complexidade ciclométrica e melhorar a compreensão.

- gh-117335 [https://github.com/python/cpython/issues/117335]: Levanta TypeError para não sequências para **`ntpath.commonpath()`**.

- gh-66449 [https://github.com/python/cpython/issues/66449]: **`configparser.ConfigParser`** agora aceita seções sem nome antes das nomeadas, se configurado para isso.

- gh-88014 [https://github.com/python/cpython/issues/88014]: Na documentação de **`gzip.GzipFile`** no módulo gzip, explique o tipo de dados do argumento construtor opcional *mtime* e recomende `mtime = 0` para gerar fluxos determinísticos.

- gh-117310 [https://github.com/python/cpython/issues/117310]: Corrigida uma falha improvável inicial e extra `Py_DECREF` em **`ssl`** ao criar um novo `_ssl._SSLContext` se o CPython foi construído de forma implausível, de modo que a lista de cifras padrão está vazia **ou** a biblioteca SSL à qual foi vinculado relata uma falha de sua API C `SSL_CTX_set_cipher_list()`.

- gh-117294 [https://github.com/python/cpython/issues/117294]: Um `DocTestCase` agora relata como ignorado se todos os exemplos no doctest forem ignorados.

- gh-98966 [https://github.com/python/cpython/issues/98966]: Em **`subprocess`**, levanta uma mensagem mais informativa quando `stdout=STDOUT`.

- gh-117225 [https://github.com/python/cpython/issues/117225]: doctest: imprime apenas “e X failed” quando não for zero, não pluraliza “1 items”. Patch de Hugo van Kemenade.

- gh-117205 [https://github.com/python/cpython/issues/117205]: Acelera **`compileall.compile_dir()`** em 20% ao usar multiprocessamento aumentando `chunksize`.

- gh-117178 [https://github.com/python/cpython/issues/117178]: Corrige a regressão no carregamento lento de módulos autorreferenciais, introduzida em gh-114781 [https://github.com/python/cpython/issues/114781].

- gh-112383 [https://github.com/python/cpython/issues/112383]: Corrige o tratamento do módulo `dis` de instruções `ENTER_EXECUTOR`.

- gh-117182 [https://github.com/python/cpython/issues/117182]: O carregamento preguiçoso de módulos que modificam seu próprio `__class__` não reverte mais o `__class__` para `types.ModuleType`.

- gh-117084 [https://github.com/python/cpython/issues/117084]: Corrige a extração de `zipfile` para entradas de diretório com o nome contendo barras invertidas no Windows.

- gh-117114 [https://github.com/python/cpython/issues/117114]: Torna `os.path.isdevdrive()` disponível em todas as plataformas. Para aqueles que não oferecem Dev Drives, ele sempre retornará `False`.

- gh-117110 [https://github.com/python/cpython/issues/117110]: Corrige um bug que impede que subclasses de `typing.Any` sejam instanciadas com argumentos. Patch por Chris Fu.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Adiadas as importações de seleção em importlib.metadata e importlib.resources para uma aceleração de 14%.

- gh-70647 [https://github.com/python/cpython/issues/70647]: Inicia o período de descontinuação para o comportamento atual de `datetime.datetime.strptime()` e `time.strptime()` que sempre falha ao analisar uma sequência de datas com um `ValueError` envolvendo um dia do mês, como `strptime("02-29", "%m-%d")` quando um ano **não** é especificado e a data acontece de ser 29 de fevereiro. Isso deve ajudar a evitar que os usuários encontrem novos bugs a cada quatro anos devido a uma suposição natural equivocada sobre a API ao analisar valores de data parciais.

- gh-116987 [https://github.com/python/cpython/issues/116987]: Corrigida `inspect.findsource()` para objetos de código de classe.

- gh-114099 [https://github.com/python/cpython/issues/114099]: Modifica a biblioteca padrão para permitir diferenças de plataforma iOS.

- gh-90872 [https://github.com/python/cpython/issues/90872]: No Windows, **`subprocess.Popen.wait()`** não chama mais `WaitForSingleObject()` com um tempo limite negativo: passe `0` ms se o tempo limite for negativo. Patch por Victor Stinner.

- gh-116957 [https://github.com/python/cpython/issues/116957]: configparser: Não deixa os valores ConfigParser em um estado inválido (armazenados como uma lista em vez de uma str) após uma leitura anterior gerar DuplicateSectionError ou DuplicateOptionError.

- gh-115538 [https://github.com/python/cpython/issues/115538]: **`_io.WindowsConsoleIO`** agora emite um aviso se um valor booleano for passado como um argumento filedescriptor.

- gh-90095 [https://github.com/python/cpython/issues/90095]: Ignora linhas vazias e comentários em `.pdbrc`

- gh-106531 [https://github.com/python/cpython/issues/106531]: Atualizado zipfile._path de zipp 3.18 [https://zipp.readthedocs.io/en/latest/history.html#v3-18-0], fornecendo melhor compatibilidade para PyPy, melhor desempenho glob para arquivos zip profundamente aninhados e fornecendo acesso interno a `CompleteDirs.inject` para uso em outros testes (como importlib.resources).

- gh-63207 [https://github.com/python/cpython/issues/63207]: No Windows, **`time.time()`** agora usa o relógio `GetSystemTimePreciseAsFileTime()` para ter uma resolução melhor que 1 us, em vez do relógio `GetSystemTimeAsFileTime()` que tem uma resolução de 15,6 ms. Patch por Victor Stinner.

- gh-116764 [https://github.com/python/cpython/issues/116764]: Restaura o suporte de `None` e outros valores falsos nas funções **`urllib.parse`** **`parse_qs()`** e **`parse_qsl()`**. Além disso, agora eles levantam um TypeError para inteiros diferentes de zero e sequências não vazias.

- gh-116811 [https://github.com/python/cpython/issues/116811]: Em `PathFinder.invalidate_caches`, delega para `MetadataPathFinder.invalidate_caches`.

- gh-116647 [https://github.com/python/cpython/issues/116647]: Corrige filho recursivo em dataclasses

- gh-113171 [https://github.com/python/cpython/issues/113171]: Corrige vários falsos positivos e falsos negativos em

  - `ipaddress.IPv4Address.is_private` (veja estas documentações para detalhes)
  - `ipaddress.IPv4Address.is_global`
  - `ipaddress.IPv6Address.is_private`
  - `ipaddress.IPv6Address.is_global`

  Também nos atributos correspondentes `ipaddress.IPv4Network` e `ipaddress.IPv6Network`.

- gh-63283 [https://github.com/python/cpython/issues/63283]: Em `encodings.idna`, qualquer capitalização do prefixo ACE (`xn--`) agora é aceitável. Patch por Pepijn de Vos e Zackery Spytz.

- gh-71042 [https://github.com/python/cpython/issues/71042]: Adiciona `platform.android_ver()`, que fornece informações sobre o dispositivo e o sistema operacional no Android.

- gh-73468 [https://github.com/python/cpython/issues/73468]: Adicionada a nova função `math.fma()`, envolvendo a operação `fma()` do C99: função de multiplicação-adição fundida. Patch por Mark Dickinson e Victor Stinner.

- gh-116608 [https://github.com/python/cpython/issues/116608]: As funções `importlib.resources` `is_resource()`, `open_binary()`, `open_text()`, `path()`, `read_binary()` e `read_text()` não estão mais descontinuadas e oferecem suporte a subdiretórios por meio de vários argumentos posicionais. A função `contents()` também permite subdiretórios, mas continua descontinuada.

- gh-116484 [https://github.com/python/cpython/issues/116484]: Altera os nomes de widgets `tkinter.Checkbutton` gerados automaticamente para evitar

colisões com nomes de widgets **`tkinter.ttk.Checkbutton`** gerados automaticamente dentro do mesmo widget pai.

- gh-114314 [https://github.com/python/cpython/issues/114314]: Em **`ctypes`**, os dados de ctype agora são armazenados em objetos type diretamente em vez de em uma subclasse dict. Esta é uma alteração interna que não deve afetar o uso.

- gh-116401 [https://github.com/python/cpython/issues/116401]: Corrige o bloqueio de **`os.fwalk()`** e **`shutil.rmtree()`** ao abrir o encadeamento nomeado.

- gh-71052 [https://github.com/python/cpython/issues/71052]: Implementa **`ctypes.util.find_library()`** no Android.

- gh-90535 [https://github.com/python/cpython/issues/90535]: Corrige suporte de valores de *interval* > 1 em **`logging.TimedRotatingFileHandler`** para `when='MIDNIGHT'` e `when='Wx'`.

- gh-113308 [https://github.com/python/cpython/issues/113308]: Remove algumas partes internas protegidas de **`uuid`**: `_has_uuid_generate_time_safe`, `_netbios_getnode`, `_ipconfig_getnode` e `_load_system_functions`. Elas não eram usadas.

- gh-115627 [https://github.com/python/cpython/issues/115627]: Corrige o tratamento de erro do módulo **`ssl`** de término de conexão por peer. Agora ele lança um OSError com o código de erro apropriado em vez de um EOFError.

- gh-114847 [https://github.com/python/cpython/issues/114847]: Acelera **`os.path.realpath()`** em plataformas não Windows.

- gh-114271 [https://github.com/python/cpython/issues/114271]: Corrige uma corrida em `threading.Thread.join()`.

  `threading._MainThread` agora sempre representa o thread principal do interpretador principal.

`PyThreadState.on_delete` e `PyThreadState.on_delete_data` foram removidos.

- gh-113538 [https://github.com/python/cpython/issues/113538]: Adiciona os métodos **`asyncio.Server.close_clients()`** e **`asyncio.Server.abort_clients()`** que permitem fechar um servidor asyncio de forma mais forçada.

- gh-85287 [https://github.com/python/cpython/issues/85287]: Altera os codecs Unicode para retornar UnicodeEncodeError ou UnicodeDecodeError, em vez de apenas UnicodeError.

- gh-113548 [https://github.com/python/cpython/issues/113548]: **`pdb`** agora permite argumentos CLI para `pdb -m`.

- gh-112948 [https://github.com/python/cpython/issues/112948]: Torna a conclusão de **`pdb`** semelhante ao Python REPL

- gh-105866 [https://github.com/python/cpython/issues/105866]: Corrigido o bug `_get_slots` que causava erro ao definir dataclasses com slots e um weakref_slot.

- gh-96471 [https://github.com/python/cpython/issues/96471]: Adiciona a terminação **`asyncio.Queue`** com o método **`shutdown()`**.

- gh-89739 [https://github.com/python/cpython/issues/89739]: O módulo **`zipimport`** agora pode ler arquivos ZIP64.

- bpo-33533 [https://bugs.python.org/issue?@action=redirect&bpo=33533]: **`asyncio.as_completed()`** agora retorna um objeto que é um iterador assíncrono e um iterador simples. O novo padrão de iteração assíncrona permite uma correlação mais fácil entre tarefas anteriores e seus resultados concluídos. Esta é uma correspondência mais próxima ao padrão de iteração de **`concurrent.futures.as_completed()`**. Patch por Justin Arthur.

- bpo-27578 [https://bugs.python.org/issue?@action=redirect&bpo=27578]: **`inspect.getsource()`** (e funções relacionadas) funcionam com

arquivos de módulo vazios, retornando `'\\n'` (ou equivalente razoável) em vez de gerar `OSError`. Patch por Kernc.

- bpo-37141 [https://bugs.python.org/issue?@action=redirect&bpo=37141]: Aceita um iterável de separadores em **`asyncio.StreamReader.readuntil()`**, parando quando um deles for encontrado.

- gh-66543 [https://github.com/python/cpython/issues/66543]: Faz **`mimetypes.guess_type()`** analisar corretamente URLs com apenas um nome de host, URLs contendo fragmento ou consulta e nomes de arquivo com apenas um sharepoint UNC no Windows. Baseado no patch por Dong-hee Na.

- bpo-15010 [https://bugs.python.org/issue?@action=redirect&bpo=15010]: **`unittest.TestLoader.discover()`** agora salva o valor original de `unittest.TestLoader._top_level_dir` e o restaura no final da chamada.

Documentação

- gh-115977 [https://github.com/python/cpython/issues/115977]: Remove referências de compatibilidade para Emscripten.
- gh-114099 [https://github.com/python/cpython/issues/114099]: Adiciona um guia de plataforma iOS e sinaliza módulos não disponíveis no iOS.
- gh-91565 [https://github.com/python/cpython/issues/91565]: Alterações nos arquivos de documentação e saídas de configuração para refletir o novo local para relatar bugs - ou seja, GitHub em vez de bugs.python.org.

Testes

- gh-83434 [https://github.com/python/cpython/issues/83434]: Desativa a saída XML do JUnit (opção de linha de comando `--junit-xml=FILE`) no regrtest ao procurar vazamentos de referência (opção `-R`). Patch por Victor Stinner.
- gh-117187 [https://github.com/python/cpython/issues/117187]: Corrige testes XML para vanilla Expat <2.6.0.

- gh-116333 [https://github.com/python/cpython/issues/116333]: Testes de coisas relacionadas a TLS (códigos de erro, etc.) foram atualizados para serem mais tolerantes sobre strings e comportamentos de mensagens de erro específicos, como visto nos forks BoringSSL e AWS-LC do OpenSSL.
- gh-117089 [https://github.com/python/cpython/issues/117089]: Testes consolidados para importlib.metadata em seu próprio pacote `metadata`.
- gh-115979 [https://github.com/python/cpython/issues/115979]: Atualiza test_importlib para que passe no WASI SDK 21.
- gh-112536 [https://github.com/python/cpython/issues/112536]: Adiciona –tsan ao test.regrtest para executar testes TSAN em tempos de execução razoáveis. Patch por Donghee Na.
- gh-116307 [https://github.com/python/cpython/issues/116307]: Adicionado o auxiliar de importação `isolated_modules`, pois `CleanImport` não remove módulos importados durante o contexto. Use-o em testes importlib.resources para evitar deixar `mod` por aí para impedir testes importlib.metadata.

Construção

- gh-114736 [https://github.com/python/cpython/issues/114736]: Faz construções WASI usar WASI SDK 21.
- gh-115983 [https://github.com/python/cpython/issues/115983]: Pula a construção de módulos de teste que devem ser construídos como compartilhados sob WASI.
- gh-71052 [https://github.com/python/cpython/issues/71052]: Adiciona script de construção e instruções do Android.

Windows

- gh-117267 [https://github.com/python/cpython/issues/117267]: Garante que `DirEntry.stat().st_ctime` se comporte consistentemente com **`os.stat()`** durante o período de descontinuação de `st_ctime` contendo o mesmo valor que `st_birthtime`. Após o período de descontinuação, `st_ctime` será o tempo de alteração de metadados (ou indisponível por meio de `DirEntry`), e apenas `st_birthtime` conterá o tempo de criação.

- gh-116195 [https://github.com/python/cpython/issues/116195]: Melhora o desempenho de **`os.getppid()`** usando uma API de sistema alternativa quando disponível. Contribuição de vxiiduu.
- gh-88494 [https://github.com/python/cpython/issues/88494]: No Windows, **`time.monotonic()`** agora usa o relógio `QueryPerformanceCounter()` para ter uma resolução melhor que 1 us, em vez do relógio `GetTickCount64()` que tem uma resolução de 15,6 ms. Patch por Victor Stinner.
- gh-116773 [https://github.com/python/cpython/issues/116773]: Corrige instâncias de `<_overlapped.Overlapped object at 0xXXX> still has pending operation at deallocation, the process may crash`.
- gh-91227 [https://github.com/python/cpython/issues/91227]: Corrige a implementação asyncio ProactorEventLoop para que o envio de um datagrama para um endereço que não esteja escutando não impeça o recebimento de mais datagramas.
- gh-115119 [https://github.com/python/cpython/issues/115119]: Trocado do código `libmpdecimal` fornecido para um pacote externo hospedado separadamente no repositório `cpython-source-deps` ao construir o módulo `_decimal`.

API C

- gh-117642 [https://github.com/python/cpython/issues/117642]: Corrige a implementação de **PEP 737** [https://peps.python.org/pep-0737/] para `%#T` e `%#N`.
- gh-87193 [https://github.com/python/cpython/issues/87193]: **`_PyBytes_Resize()`** agora pode ser chamada para objetos bytes com contagem de referências > 1, incluindo objetos bytes de 1 byte. Ele cria um novo objeto bytes e destrói o antigo se ele tiver contagem de referências > 1.
- gh-117021 [https://github.com/python/cpython/issues/117021]: Corrige estouro de inteiro em **`PyLong_AsPid()`** em plataformas de 64 bits que não sejam Windows.
- gh-115756 [https://github.com/python/cpython/issues/115756]: **`PyCode_GetFirstFree()`** é uma API utilizável agora e foi renomeada para **`PyUnstable_Code_GetFirstFree()`**. (Contribuição de Bogdan Romanyuk em gh-115781 [https://github.com/python/cpython/issues/115781])
- gh-116869 [https://github.com/python/cpython/issues/116869]: Adiciona o teste `test_cext`: cria uma extensão C para verificar se a API Python C

emite avisos do compilador C. Patch por Victor Stinner.

- gh-116869 [https://github.com/python/cpython/issues/116869]: Torna a API C compatível com o sinalizador do compilador `-Werror=declaration-after-statement` novamente. Patch por Victor Stinner.
- gh-116936 [https://github.com/python/cpython/issues/116936]: Adiciona **`PyType_GetModuleByDef()`** à API C limitada. Patch por Victor Stinner.
- gh-116809 [https://github.com/python/cpython/issues/116809]: Restaura a função privada `_PyErr_ChainExceptions1()` removida. Patch por Victor Stinner.
- gh-115754 [https://github.com/python/cpython/issues/115754]: Na versão limitada 3.13 da API C, obter Singletons `Py_None`, `Py_False`, `Py_True`, `Py_Ellipsis` e `Py_NotImplemented` agora é implementado como chamadas de função no nível ABI estável para ocultar detalhes de implementação. Obter essas constantes ainda retorna referências emprestadas. Patch por Victor Stinner.
- gh-115754 [https://github.com/python/cpython/issues/115754]: Adiciona as funções **`Py_GetConstant()`** e **`Py_GetConstantBorrowed()`** para obter constantes. Por exemplo, `Py_GetConstant(Py_CONSTANT_ZERO)` retorna uma referência forte para a constante zero. Patch por Victor Stinner.
- gh-111696 [https://github.com/python/cpython/issues/111696]: Adiciona suporte para os formatos `%T`, `%T#`, `%N` e `%N#` para **`PyUnicode_FromFormat()`**: formata o nome totalmente qualificado de um tipo de objeto e de um tipo: chama **`PyType_GetModuleName()`**. Veja **PEP 737** [https://peps.python.org/pep-0737/] para mais informações. Patch por Victor Stinner.
- gh-111696 [https://github.com/python/cpython/issues/111696]: Adiciona a função **`PyType_GetModuleName()`** para obter o nome do módulo do tipo. Equivalente a obter o atributo `type.__module__`. Patch por Eric Snow e Victor Stinner.
- gh-111696 [https://github.com/python/cpython/issues/111696]: Adiciona a função **`PyType_GetFullyQualifiedName()`** para obter o nome totalmente qualificado do tipo. Equivalente a `f"{type.__module__}.{type.__qualname__}"`, ou `type.__qualname__` se `type.__module__` não for uma string ou for igual a `"builtins"`. Patch por Victor Stinner.
- gh-85283 [https://github.com/python/cpython/issues/85283]: As extensões C `fcntl`, `grp`, `pwd`, `termios`, `_statistics` e `_testconsole` agora são

construídas com a API C limitada. Patch por Victor Stinner.

- gh-111140 [https://github.com/python/cpython/issues/111140]: Adiciona sinalizadores adicionais a `PyLong_AsNativeBytes()` e `PyLong_FromNativeBytes()` para permitir que o chamador determine como lidar com casos extremos em torno de valores que preenchem o buffer inteiro.
- gh-113024 [https://github.com/python/cpython/issues/113024]: Adiciona a função `PyObject_GenericHash()`.

Python 3.13.0 alpha 5

Data de lançamento: 2024-03-12

Segurança

- gh-115398 [https://github.com/python/cpython/issues/115398]: Permite controlar o adiamento da nova análise Expat >=2.6.0 (**CVE 2023-52425** [https://www.cve.org/CVERecord?id=CVE-2023-52425]) adicionando cinco novos métodos:
  - `xml.etree.ElementTree.XMLParser.flush()`
  - `xml.etree.ElementTree.XMLPullParser.flush()`
  - `xml.parsers.expat.xmlparser.GetReparseDeferralEnabled()`
  - `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()`
  - `xml.sax.expatreader.ExpatParser.flush()`
- gh-114572 [https://github.com/python/cpython/issues/114572]: `ssl.SSLContext.cert_store_stats()` e `ssl.SSLContext.get_ca_certs()` agora travam corretamente o acesso ao armazenamento de certificados, quando o `ssl.SSLContext` é compartilhado entre várias threads.

Núcleo e embutidos

- gh-116604 [https://github.com/python/cpython/issues/116604]: Respeita o status do coletor de lixo quando chamadas indiretas são feitas via `PyErr_CheckSignals()` e o disjuntor de avaliação. Patch por Pablo Galindo

- gh-112087 [https://github.com/python/cpython/issues/112087]: **`list`** agora é compatível com a implementação da **PEP 703** [https://peps.python.org/pep-0703/].
- gh-116381 [https://github.com/python/cpython/issues/116381]: Adiciona especialização para `CONTAINS_OP`.
- gh-116296 [https://github.com/python/cpython/issues/116296]: Corrige possível vazamento de referência no tratamento de erros internos de **`object.__reduce__()`**.
- gh-115823 [https://github.com/python/cpython/issues/115823]: Calcula corretamente intervalos de erro no analisador ao levantar exceções **`SyntaxError`** causadas por sequências de bytes inválidas. Patch por Pablo Galindo
- gh-115778 [https://github.com/python/cpython/issues/115778]: Adiciona anotação `tierN` para definição de instrução no interpretador DSL.
- gh-115733 [https://github.com/python/cpython/issues/115733]: Corrige travamento ao chamar `next()` em iteradores de lista esgotados.
- gh-115700 [https://github.com/python/cpython/issues/115700]: O estágio de construção de regen-cases agora funciona no Windows.
- gh-115347 [https://github.com/python/cpython/issues/115347]: Corrige um bug em que docstring era substituído por um NOP redundante quando Python é executado com `-OO`.
- gh-115323 [https://github.com/python/cpython/issues/115323]: Torna a mensagem de erro mais significativa para quando **`bytearray.extend()`** é chamado com um objeto **`str`**.
- gh-112175 [https://github.com/python/cpython/issues/112175]: Cada `PyThreadState` agora tem seu próprio `eval_breaker`, permitindo que threads específicos sejam interrompidos.

Biblioteca

- gh-115154 [https://github.com/python/cpython/issues/115154]: Corrige um bug que estava fazendo com que a função **`tokenize.untokenize()`** manipulasse literais nomeados unicode incorretamente. Patch por Pablo Galindo

Núcleo e embutidos

- gh-115154 [https://github.com/python/cpython/issues/115154]: Corrige um bug que estava fazendo com que a função `tokenize.untokenize()` manipulasse literais nomeados unicode incorretamente. Patch por Pablo Galindo

Biblioteca

- gh-104090 [https://github.com/python/cpython/issues/104090]: O rastreador de recursos de multiprocessamento agora sai com código de status diferente de zero se um vazamento de recurso for detectado. Ele ainda sai com código de status 0 caso contrário.

Núcleo e embutidos

- gh-105858 [https://github.com/python/cpython/issues/105858]: Melhora os construtores para nós `ast`. Os argumentos de tipos de lista agora assumem como padrão uma lista vazia se omitidos, e os campos opcionais assumem como padrão `None`. Os nós AST agora têm um atributo `__annotations__` com os tipos esperados de seus atributos. Passar argumentos extras não reconhecidos para nós AST está descontinuado e se tornará um erro no Python 3.15. Omitir um argumento obrigatório para um nó AST está descontinuado e se tornará um erro no Python 3.15. Patch por Jelle Zijlstra.
- gh-101860 [https://github.com/python/cpython/issues/101860]: Expor atributo `__name__` na propriedade.
- gh-96497 [https://github.com/python/cpython/issues/96497]: Corrige resolução incorreta de variáveis de classe mutiladas usadas em expressões de atribuição em compreensões.

Biblioteca

- gh-116600 [https://github.com/python/cpython/issues/116600]: Corrige `repr()` para membros globais `Flag`.

- gh-116349 [https://github.com/python/cpython/issues/116349]: **`platform.java_ver()`** está descontinuado e será removido na versão

3.15. Ele não foi testado em grande parte, tinha uma API confusa e era útil apenas para suporte a Jython.

- gh-116143 [https://github.com/python/cpython/issues/116143]: Corrige uma corrida no pydoc `_start_server`, eliminando uma janela na qual `_start_server` pode retornar um thread que está "servindo", mas sem um conjunto `docserver`.

- gh-116127 [https://github.com/python/cpython/issues/116127]: **`typing`**: implementa **PEP 705** [https://peps.python.org/pep-0705/] que adiciona suporte a **`typing.ReadOnly`** para **`typing.TypedDict`**.

- gh-116325 [https://github.com/python/cpython/issues/116325]: **`typing`**: levanta **`SyntaxError`** em vez de **`AttributeError`** em referências de encaminhamento como strings vazias.

- gh-115957 [https://github.com/python/cpython/issues/115957]: Quando `asyncio.TaskGroup.create_task` é chamado em um `asyncio.TaskGroup` inativo, a corrotina fornecida será fechada (o que impede um `RuntimeWarning`).

- gh-115978 [https://github.com/python/cpython/issues/115978]: Desabilita preadv(), readv(), pwritev() e writev() em WASI.

  No wasmtime para WASI 0.2, essas funções não passam no test_posix (https://github.com/bytecodealliance/wasmtime/issues/7830).

- gh-88352 [https://github.com/python/cpython/issues/88352]: Corrige o cálculo do próximo tempo de rollover no manipulador **`logging.TimedRotatingFileHandler.computeRollover()`** agora sempre retorna um registro de data e hora maior que o tempo especificado e funciona corretamente durante a mudança de horário de verão. **`doRollover()`** não substitui mais o arquivo já rolado, evitando perda de dados quando executado à meia-noite ou durante tempo repetido na mudança de horário de verão.

- gh-87115 [https://github.com/python/cpython/issues/87115]: Define `__main__.__spec__` como `None` ao executar um script com **`pdb`**

- gh-76511 [https://github.com/python/cpython/issues/76511]: Corrige UnicodeEncodeError em **`email.Message.as_string()`** que resulta quando uma mensagem que afirma estar no conjunto de caracteres ASCII na verdade tem caracteres não ASCII. Caracteres não ASCII agora são substituídos pelo caractere de substituição U+FFFD, como no manipulador de erros `replace`.

- gh-89547 [https://github.com/python/cpython/issues/89547]: Adiciona suporte para formulários especiais de digitação aninhada como Final[ClassVar[int]].

- gh-65824 [https://github.com/python/cpython/issues/65824]: Melhora o prompt `less` em **`pydoc`**.

- gh-116040 [https://github.com/python/cpython/issues/116040]: [Enum] corrige chamadas por valor quando o segundo valor for falso; por exemplo. Cardinal(1, 0)

- gh-115821 [https://github.com/python/cpython/issues/115821]: [Enum] Melhora a mensagem de erro ao chamar super().__new__() em __new__ personalizado.

- gh-85644 [https://github.com/python/cpython/issues/85644]: Usa a variável de ambiente `XDG_CURRENT_DESKTOP` em **`webbrowser`** para verificar a área de trabalho. Prefira-a ao descontinuado `GNOME_DESKTOP_SESSION_ID` para detecção do GNOME.

- gh-75988 [https://github.com/python/cpython/issues/75988]: Corrigida **`unittest.mock.create_autospec()`** para passar a chamada para o objeto encapsulado para retornar o resultado real.

- gh-115881 [https://github.com/python/cpython/issues/115881]: Corrige problema em que **`ast.parse()`** sinalizaria incorretamente gerenciadores de contexto condicional (como `with (x() if y else z()): ...`) como sintaxe inválida se `feature_version=(3, 8)` fosse passado. Isso reverte as alterações na gramática feitas como parte de gh-94949 [https://github.com/python/cpython/issues/94949].

- gh-115886 [https://github.com/python/cpython/issues/115886]: Corrige truncamento silencioso do nome com um caractere nulo incorporado em **`multiprocessing.shared_memory.SharedMemory`**.

- gh-115532 [https://github.com/python/cpython/issues/115532]: Adiciona estimativa de densidade do kernel ao módulo de estatísticas.

- gh-115714 [https://github.com/python/cpython/issues/115714]: No WASI, o módulo **`time`** não obtém mais o tempo de processo usando `times()` ou `CLOCK_PROCESS_CPUTIME_ID`, a API do sistema não é confiável e provavelmente será removida do WASI. As funções de relógio afetadas retornam para chamar `clock()`.

- gh-115809 [https://github.com/python/cpython/issues/115809]: Melhora o algoritmo para calcular quais arquivos de log revertidos devem ser excluídos em **`logging.TimedRotatingFileHandler`**. Agora é confiável para manipuladores sem `namer` e com `namer` determinístico arbitrário que deixa a parte de data e hora no nome do arquivo inalterada.

- gh-74668 [https://github.com/python/cpython/issues/74668]: As funções **`urllib.parse`** **`parse_qs()`** e **`parse_qsl()`** agora suportam argumentos de bytes contendo dados não ASCII brutos e codificados em porcentagem.

- gh-67044 [https://github.com/python/cpython/issues/67044]: **`csv.writer()`** agora sempre coloca aspas ou escapa `'\r'` e `'\n'`, independentemente do valor *lineterminator*.

- gh-115712 [https://github.com/python/cpython/issues/115712]: Restaura o suporte do delimitador de espaço com `skipinitialspace=True` em **`csv`**. **`csv.writer()`** agora coloca aspas em campos vazios se o delimitador for um espaço e skipinitialspace for true e levanta uma exceção se aspas não forem possíveis.

- gh-112364 [https://github.com/python/cpython/issues/112364]: Corrigido **`ast.unparse()`** para manipular format_spec com `"`, `'` ou `\\`. Corrigido por Frank Hoffmann.

- gh-112997 [https://github.com/python/cpython/issues/112997]: Para de registrar argumentos de retorno de chamada potencialmente sensíveis em **`asyncio`** a menos que o modo de depuração esteja ativo.

- gh-114914 [https://github.com/python/cpython/issues/114914]: Corrige um problema em que um **`StreamWriter`** abandonado não seria coletado como lixo.

- gh-111358 [https://github.com/python/cpython/issues/111358]: Corrige um bug em **`asyncio.BaseEventLoop.shutdown_default_executor()`** para garantir que o tempo limite passado para a corrotina se comporte conforme o esperado.

- gh-115618 [https://github.com/python/cpython/issues/115618]: Corrige a diminuição inadequada da contagem de referência para o argumento `None` em métodos **`property`** **`getter()`**, **`setter()`** e **`deleter()`**.

- gh-112720 [https://github.com/python/cpython/issues/112720]: Refatora **`dis.ArgResolver`** para tornar possível fazer subclasse e alterar a maneira como argumentos de salto são interpretados.

- gh-112006 [https://github.com/python/cpython/issues/112006]: Corrige **`inspect.unwrap()`** para tipos com o descritor de dados `__wrapper__`. Corrige **`inspect.Signature.from_callable()`** para embutido **`classmethod()`** e **`staticmethod()`**.

- gh-101293 [https://github.com/python/cpython/issues/101293]: Adiciona suporte a chamáveis com o método `__call__()` e tipos com métodos `__new__()` e `__init__()` definidos como métodos de classe, métodos estáticos, métodos vinculados, funções parciais e outros tipos de métodos e descritores em **`inspect.Signature.from_callable()`**.

- gh-103092 [https://github.com/python/cpython/issues/103092]: Isola **`_lsprof`** (aplica a **PEP 687** [https://peps.python.org/pep-0687/]).

- gh-113942 [https://github.com/python/cpython/issues/113942]: **`pydoc`** não ignora mais funções globais implementadas como métodos embutidos, como **`MethodDescriptorType`** e **`WrapperDescriptorType`**.

- gh-115256 [https://github.com/python/cpython/issues/115256]: Adicionado DeprecationWarning ao acessar o atributo tarfile de objetos TarInfo. O atributo nunca é usado internamente e só é anexado ao TarInfos quando o tarfile é aberto no modo de gravação, não no modo de leitura. O atributo cria um ciclo de referência desnecessário que pode causar corrupção ao não fechar o identificador após gravar um tarfile.

- gh-115197 [https://github.com/python/cpython/issues/115197]: `urllib.request` não resolve mais o nome do host antes de verificá-lo em relação à lista de bypass de proxy do sistema no macOS e Windows.

- gh-113812 [https://github.com/python/cpython/issues/113812]: **`DatagramTransport.sendto()`** agora enviará datagramas de comprimento zero se for chamado com um objeto de bytes vazio. O controle de fluxo de transporte agora também considera o cabeçalho do datagrama ao calcular o tamanho do buffer.

- gh-114763 [https://github.com/python/cpython/issues/114763]: Protege os módulos carregados com **`importlib.util.LazyLoader`** de condições de corrida quando vários threads tentam acessar atributos antes que o carregamento seja concluído.

- gh-114709 [https://github.com/python/cpython/issues/114709]: **`posixpath.commonpath()`** agora levanta uma exceção **`ValueError`** quando passado um iterável vazio. Anteriormente, **`IndexError`** era levantada.

  `posixpath.commonpath()` agora levanta uma exceção **`TypeError`** quando passado `None`. Anteriormente, **`ValueError`** era levantada.

- gh-114610 [https://github.com/python/cpython/issues/114610]: Corrige bug onde **`pathlib.PurePath.with_stem()`** convertia um sufixo de caminho não vazio em um radical quando recebia um argumento *stem* vazio. Agora levanta **`ValueError`**, assim como **`pathlib.PurePath.with_suffix()`** faz quando chamado em um caminho com um radical vazio, dado um argumento *suffix* não vazio.

- gh-107361 [https://github.com/python/cpython/issues/107361]: Adiciona **`ssl.VERIFY_X509_PARTIAL_CHAIN`** e **`VERIFY_X509_STRICT`** ao contexto SSL padrão criado com **`ssl.create_default_context()`**.

- gh-112281 [https://github.com/python/cpython/issues/112281]: Permite a criação de união de tipos para **`typing.Annotated`** com metadados não-estruturáveis.

- gh-111775 [https://github.com/python/cpython/issues/111775]: Corrige **`importlib.resources.simple.ResourceHandle.open()`** para modo texto, adicionado argumento `stream` ausente.

- gh-90095 [https://github.com/python/cpython/issues/90095]: Faz .pdbrc e -c funcionarem com quaisquer comandos pdb válidos.

- gh-107625 [https://github.com/python/cpython/issues/107625]: Levanta **`configparser.ParsingError`** dos métodos **`read()`** e **`read_file()`** de **`configparser.ConfigParser`** se uma chave sem um valor correspondente for continuada (isto é, seguida por uma linha indentada).

- gh-107155 [https://github.com/python/cpython/issues/107155]: Corrige saída incorreta de `help(x)` onde `x` é uma função **`lambda`**, que tem um atributo de dicionário `__annotations__` com uma chave `"return"`.

- gh-57141 [https://github.com/python/cpython/issues/57141]: Adiciona opção para comparações *não rasas* para **`filecmp.dircmp`** como **`filecmp.cmp()`**. Patch original por Steven Ward. Aprimorado por Tobias Rautenkranz

- gh-69990 [https://github.com/python/cpython/issues/69990]: **`Profile.print_stats()`** foi melhorado para aceitar vários argumentos de classificação. Correção por Chiu-Hsiang Hsu e Furkan Onder.

- gh-104061 [https://github.com/python/cpython/issues/104061]: Adiciona constante **`socket.SO_BINDTOIFINDEX`**.

- gh-60346 [https://github.com/python/cpython/issues/60346]: Corrige ArgumentParser inconsistente com parse_known_args.

- gh-102389 [https://github.com/python/cpython/issues/102389]: Adiciona `windows_31j` aos apelidos para o codec `cp932`

- gh-72249 [https://github.com/python/cpython/issues/72249]: Always include the module name in the **`repr()`** of **`functools.partial()`** objects. Patch by Furkan Onder and Anilyka Barry.

- gh-100985 [https://github.com/python/cpython/issues/100985]: Atualiza HTTPSConnection para encapsular consistentemente endereços IPv6 ao usar um proxy.

- gh-100884 [https://github.com/python/cpython/issues/100884]: email: corrige dobramento incorreto de vírgula em listas de endereços em várias linhas em combinação com codificação unicode.

- gh-95782 [https://github.com/python/cpython/issues/95782]: Corrige **`io.BufferedReader.tell()`**, **`io.BufferedReader.seek()`**, **`_pyio.BufferedReader.tell()`**, **`io.BufferedRandom.tell()`**, **`io.BufferedRandom.seek()`** e **`_pyio.BufferedRandom.tell()`** sendo capazes de retornar deslocamentos negativos.

- gh-96310 [https://github.com/python/cpython/issues/96310]: Corrige um traceback em **`argparse`** quando todas as opções em um grupo mutuamente exclusivo são suprimidas.

- gh-93205 [https://github.com/python/cpython/issues/93205]: Corrige um bug em **`logging.handlers.TimedRotatingFileHandler`** onde várias instâncias de manipulador rotativo apontando para arquivos com o mesmo nome, mas extensões diferentes, entrariam em conflito e não excluiriam os arquivos corretos.

- bpo-31116 [https://bugs.python.org/issue?@action=redirect&bpo=31116]: Adiciona codificação Z85 a `base64`.

- bpo-44865 [https://bugs.python.org/issue?@action=redirect&bpo=44865]: Adiciona chamada ausente para função de localização em **`argparse`**.

- bpo-43952 [https://bugs.python.org/issue?@action=redirect&bpo=43952]: Corrige `multiprocessing.connection.Listener.accept()` para aceitar bytes vazios como authkey. Não aceitar bytes vazios como chave faz com que ele trave indefinidamente.

- bpo-42125 [https://bugs.python.org/issue?@action=redirect&bpo=42125]: linecache: obtém nome do módulo de `__spec__` se disponível. Isso permite obter código-fonte para o módulo `__main__` quando um carregador personalizado é usado.

- bpo-41122 [https://bugs.python.org/issue?@action=redirect&bpo=41122]: Não passar argumentos corretamente para `functools.singledispatchmethod()` agora levanta um TypeError em vez de atingir um índice fora dos limites internamente.

- bpo-40818 [https://bugs.python.org/issue?@action=redirect&bpo=40818]: O asyncio REPL agora executa `sys.__interactivehook__` na inicialização. A implementação padrão de `sys.__interactivehook__` fornece preenchimento automático para o asyncio REPL. O patch é uma contribuição por Rémi Lapeyre.

- bpo-33775 [https://bugs.python.org/issue?@action=redirect&bpo=33775]: Adiciona o texto de ajuda ‘default’ e ‘version’ para localização no argparse.

Documentação

- gh-115399 [https://github.com/python/cpython/issues/115399]: Documenta **CVE 2023-52425** [https://www.cve.org/CVERecord?id=CVE-2023-52425] do Expat <2.6.0 em “Vulnerabilidades em XML”.
- gh-109653 [https://github.com/python/cpython/issues/109653]: Melhora o tempo de importação de `uuid` no Linux.

Testes

- gh-71052 [https://github.com/python/cpython/issues/71052]: Adiciona exclusões de teste para dar suporte à execução do conjunto de testes no Android.

- gh-71052 [https://github.com/python/cpython/issues/71052]: Habilita `test_concurrent_futures` em plataformas que oferecem suporte a threads, mas não multiprocessamento.
- gh-115796 [https://github.com/python/cpython/issues/115796]: Faz com que '_testinternalcapi.assemble_code_object' construa a tabela de exceções para o objeto de código.
- gh-115720 [https://github.com/python/cpython/issues/115720]: Testes de vazamento (`-R`, `--huntrleaks`) agora mostram um resumo do número de vazamentos encontrados em cada iteração.
- gh-115122 [https://github.com/python/cpython/issues/115122]: Adiciona a opção `--bisect` ao executor de teste regrtest: execute testes com falha com `test.bisect_cmd` para identificar testes com falha. Patch por Victor Stinner.
- gh-115596 [https://github.com/python/cpython/issues/115596]: Corrige `ProgramPriorityTests` em `test_os` alterando permanentemente a prioridade do processo.
- gh-115556 [https://github.com/python/cpython/issues/115556]: No Windows, vírgulas passadas em argumentos para `Tools\buildbot\test.bat` e `PCbuild\\rt.bat` agora são manipuladas corretamente.
- gh-115420 [https://github.com/python/cpython/issues/115420]: Corrige tradução de alvos de manipuladores de exceção por `_testinternalcapi.optimize_cfg`.
- gh-115376 [https://github.com/python/cpython/issues/115376]: Corrige falha de segmentação em `_testinternalcapi.compiler_codegen` em entrada incorreta.

Construção

- gh-116313 [https://github.com/python/cpython/issues/116313]: Faz com que construções WASI funcionem no wasmtime 18 com primitivos WASI 0.2/preview2.
- gh-71052 [https://github.com/python/cpython/issues/71052]: Altera **`sys.platform`** do Android de `"linux"` para `"android"`.
- gh-116117 [https://github.com/python/cpython/issues/116117]: Faz backport da PR #42 da `libb2` para corrigir a compilação do CPython no Windows de 32 bits com `clang-cl`.

- gh-71052 [https://github.com/python/cpython/issues/71052]: Corrige vários problemas de construção do Android
- gh-114099 [https://github.com/python/cpython/issues/114099]: Um projeto de testbed foi adicionado para executar o conjunto de testes no iOS.
- gh-115350 [https://github.com/python/cpython/issues/115350]: Corrige o módulo de construção ctypes com -DWIN32_LEAN_AND_MEAN definido
- gh-111225 [https://github.com/python/cpython/issues/111225]: Vincula módulos de extensão ao libpython no Android.
- gh-115737 [https://github.com/python/cpython/issues/115737]: O nome de instalação do libPython agora está definido corretamente para compilações macOS sem framework.
- gh-114099 [https://github.com/python/cpython/issues/114099]: Os alvos do Makefile foram adicionados para oferecer suporte à compilação de uma construção de framework compatível com iOS.

Windows

- gh-116012 [https://github.com/python/cpython/issues/116012]: Garante que o valor de `GetLastError()` seja preservado em todas as operações com a GIL.
- gh-115582 [https://github.com/python/cpython/issues/115582]: Construir extensões destinadas a construções com threads livres do CPython agora exigem a compilação manual com `/DPy_GIL_DISABLED` ao usar uma instalação regular. Espera-se que isso mude em versões futuras.
- gh-115554 [https://github.com/python/cpython/issues/115554]: O instalador agora tem regras mais rígidas sobre a atualização do Gerenciador de instalação do Python. Em geral, a maioria dos usuários tem apenas um único iniciador instalado e não verá nenhuma diferença. Quando vários iniciadores foram instalados, a opção de instalar o iniciador é desabilitada até que todos, exceto um, tenham sido removidos. O downgrade do iniciador (que nunca foi permitido) agora é mais obviamente bloqueado.
- gh-115543 [https://github.com/python/cpython/issues/115543]: Gerenciador de instalação do Python agora pode detectar o Python 3.13 quando instalado da Microsoft Store e instalará o Python 3.12 por padrão quando **`PYLAUNCHER_ALLOW_INSTALL`** estiver definido.

macOS

- gh-116145 [https://github.com/python/cpython/issues/116145]: Atualiza instalador do macOS para Tcl/Tk 8.6.14.

IDLE

- gh-88516 [https://github.com/python/cpython/issues/88516]: No macOS, mostra um ícone de proxy na barra de título das janelas do editor para corresponder ao comportamento da plataforma.

Ferramentas/Demos

- gh-100176 [https://github.com/python/cpython/issues/100176]: Remove Tools/{io,cc,string}bench desatualizado
- bpo-45101 [https://bugs.python.org/issue?@action=redirect&bpo=45101]: Adiciona consistência na mensagem de uso de E/S entre 2 versões do python-config.

API C

- gh-114626 [https://github.com/python/cpython/issues/114626]: Adiciona novamente o nome `_PyCFunctionFastWithKeywords`, removido no Python 3.13 alpha 4 por engano. Manter o antigo nome privado `_PyCFunctionFastWithKeywords` (Python 3.7) como um apelido para o novo nome público `PyCFunctionFastWithKeywords` (Python 3.13a4). Patch por Victor Stinner.
- gh-111418 [https://github.com/python/cpython/issues/111418]: Adiciona as macros C **`PyHASH_MODULUS`**, **`PyHASH_BITS`**, **`PyHASH_INF`** e **`PyHASH_IMAG`**. Patch por Sergey B Kirpichev.

Python 3.13.0 alpha 4

Data de lançamento: 2024-02-15

Segurança

- gh-115399 [https://github.com/python/cpython/issues/115399]: Atualiza libexpat empacotado para 2.6.0
- gh-115243 [https://github.com/python/cpython/issues/115243]: Corrige possíveis travamentos em `collections.deque.index()` quando o deque é modificado simultaneamente.

Núcleo e embutidos

- gh-112087 [https://github.com/python/cpython/issues/112087]: Para um iterador reverso vazio for, lista será reduzida para `reversed()`. Patch por Donghee Na
- gh-114570 [https://github.com/python/cpython/issues/114570]: Adiciona exceção `PythonFinalizationError`. Esta exceção derivada de `RuntimeError` é levantada quando uma operação é bloqueada durante a finalização do Python. Patch por Victor Stinner.
- gh-114695 [https://github.com/python/cpython/issues/114695]: Adiciona `sys._clear_internal_caches()`, que limpa todos os caches internos relacionados ao desempenho (e descontinua a função menos geral `sys._clear_type_cache()`).
- gh-114828 [https://github.com/python/cpython/issues/114828]: Corrige travamentos de compilação em exemplos de código incomuns usando `super()` dentro de uma compreensão em um corpo de classe.
- gh-112069 [https://github.com/python/cpython/issues/112069]: Adapta os métodos `set` e `frozenset` para Argument Clinic.
- gh-115011 [https://github.com/python/cpython/issues/115011]: Setters para membros com um tipo inteiro sem sinal agora oferecem suporte ao mesmo intervalo de valores válidos para objetos que têm um método `__index__()` como para `int`.
- gh-114887 [https://github.com/python/cpython/issues/114887]: Alterada a validação do tipo de soquete em `create_datagram_endpoint()` para aceitar todos os soquetes não-stream. Isso corrige uma regressão na compatibilidade com soquetes brutos.
- gh-114944 [https://github.com/python/cpython/issues/114944]: Corrige uma corrida entre `PyParkingLot_Park` e `_PyParkingLot_UnparkAll`.
- gh-113462 [https://github.com/python/cpython/issues/113462]: Limita o número de versões que uma única classe pode usar. Impede que algumas classes rebeldes usem todos os números de versão.

- gh-76763 [https://github.com/python/cpython/issues/76763]: A função embutido **`chr()`** agora sempre levanta **`ValueError`** para valores fora do intervalo válido. Anteriormente, levantava **`OverflowError`** para valores muito grandes ou pequenos.
- gh-114806 [https://github.com/python/cpython/issues/114806]: Não especializa mais chamadas para classes, se essas classes tiverem metaclasses. Corrige bug onde o método `__call__` da metaclasse não estava sendo chamado.
- gh-107944 [https://github.com/python/cpython/issues/107944]: Melhora mensagem de erro para chamadas de função com argumentos nomeados inválidos via getargs
- gh-112529 [https://github.com/python/cpython/issues/112529]: A construção com threads livres não aloca mais espaço para a estrutura `PyGC_Head` em objetos que oferecem suporte a coleta de lixo cíclica. Vários outros campos e estruturas de dados são usados como substituições, incluindo `ob_gc_bits`, `ob_tid` e estruturas de dados internas mimalloc.
- gh-114456 [https://github.com/python/cpython/issues/114456]: Reduz o limite de recursão em uma construção de depuração do WASI.
- gh-114083 [https://github.com/python/cpython/issues/114083]: O compilador aplica o dobramento de LOAD_CONST com a instrução a seguir em uma passagem separada antes de outras otimizações. Isso permite o jump threading em certas circunstâncias.
- gh-114388 [https://github.com/python/cpython/issues/114388]: Corrige um **`RuntimeWarning`** emitido ao atribuir um valor inteiro que não é uma instância de **`int`** a um atributo que corresponde a um membro struct C de type T_UINT e T_ULONG. Corrige um **`RuntimeWarning`** duplo emitido ao atribuir um valor inteiro negativo a um atributo que corresponde a um membro struct C do tipo T_UINT.
- gh-114265 [https://github.com/python/cpython/issues/114265]: O compilador propaga números de linha antes da otimização, levando a mais oportunidades de otimização e removendo a necessidade do hack `guarantee_lineno_for_exits`.
- gh-112529 [https://github.com/python/cpython/issues/112529]: A construção com threads livres agora tem sua própria implementação de GC segura para thread que usa mimalloc para encontrar objetos rastreados por GC. Não é geracional, diferente da implementação GC existente.

- gh-114050 [https://github.com/python/cpython/issues/114050]: Corrige falha de segmentação causada por uma string de formato incorreta na exceção `TypeError` quando mais de dois argumentos são passados para `int`.
- gh-112354 [https://github.com/python/cpython/issues/112354]: A instrução `END_FOR` agora exibe apenas um valor. Isso é para oferecer melhor suporte a saídas laterais em laços.
- gh-113884 [https://github.com/python/cpython/issues/113884]: Torna **`queue.SimpleQueue`** segura para thread quando a GIL estiver desabilitada.
- gh-114058 [https://github.com/python/cpython/issues/114058]: Implementa as bases do eliminador de redundância de Nível 2.
- gh-113939 [https://github.com/python/cpython/issues/113939]: frame.clear(): Limpa frame.f_locals também, e não apenas os locals rápidos. Isso é relevante quando frame.f_locals foi acessado, o que também conteria referências a todos os locais.
- gh-112050 [https://github.com/python/cpython/issues/112050]: Converte **`collections.deque`** para usar Argument Clinic.
- gh-112050 [https://github.com/python/cpython/issues/112050]: Torna os métodos em **`collections.deque`** seguros para thread quando a GIL estiver desabilitada.
- gh-113464 [https://github.com/python/cpython/issues/113464]: Adiciona uma opção (`--enable-experimental-jit` para construções baseadas em `configure` ou `--experimental-jit` para construções baseadas em `PCbuild`) para construir um compilador *experimental* just-in-time, baseado em copy-and-patch [https://fredrikbk.com/publications/copy-and-patch.pdf]
- gh-113055 [https://github.com/python/cpython/issues/113055]: Torna interp->obmalloc um ponteiro. Para interpretadores que compartilham estado com o interpretador principal, isso aponta para a mesma estrutura de memória estática. Para interpretadores com seu próprio estado obmalloc, ele é alocado em heap. Adiciona free_obmalloc_arenas() que liberará as arenas obmalloc e as estruturas de árvore radix para interpretadores com seu próprio estado obmalloc.
- gh-55664 [https://github.com/python/cpython/issues/55664]: Adiciona um aviso ao criar **`type`** usando um dicionário de espaço de nomes com chaves não string. Patch por Daniel Urban e Furkan Onder.
- gh-104530 [https://github.com/python/cpython/issues/104530]: Usa variáveis de condição nativas do Win32.

Biblioteca

- gh-115392 [https://github.com/python/cpython/issues/115392]: Corrige um bug em `doctest` onde números de linha incorretos seriam relatados para funções decoradas.

- gh-114563 [https://github.com/python/cpython/issues/114563]: Corrige vários bugs de `format()` ao usar a implementação C de `Decimal`: * vazamento de memória em alguns casos raros ao usar a opção de formato `z` (coagir negativo 0) * saída incorreta ao aplicar a opção de formato `z` ao tipo `F` (ponto fixo com `NAN` maiúsculo / `INF`) * saída incorreta ao aplicar a opção de formato `#` (formato alternativo)

- gh-102840 [https://github.com/python/cpython/issues/102840]: Corrige traceback confuso quando as operações floordiv, mod ou divmod acontecem entre instâncias de `fractions.Fraction` e `complex`.

- gh-115165 [https://github.com/python/cpython/issues/115165]: A maioria das exceções agora são ignoradas ao tentar definir o atributo `__orig_class__` em objetos retornados ao chamar apelidos genéricos `typing` (incluindo aliases genéricos criados usando `typing.Annotated`). Anteriormente, apenas `AttributeError` era ignorada. Patch por Dave Shawley.

- gh-112903 [https://github.com/python/cpython/issues/112903]: Corrige erros de “issubclass() arg 1 must be a class” em certos casos de herança múltipla com aliases genéricos (regressão nas primeiras versões alfa 3.13).

- gh-115133 [https://github.com/python/cpython/issues/115133]: Corrige testes para `XMLPullParser` com Expat 2.6.0.

- gh-115059 [https://github.com/python/cpython/issues/115059]: `io.BufferedRandom.read1()` agora limpa o buffer de gravação subjacente.

- gh-79382 [https://github.com/python/cpython/issues/79382]: `**` no final não permite mais corresponder arquivos e caminhos não existentes em

glob() recursivo.

- gh-67837 [https://github.com/python/cpython/issues/67837]: Evita condições de corrida na criação de diretórios durante a extração simultânea em **tarfile** e **zipfile**.

- gh-115060 [https://github.com/python/cpython/issues/115060]: Acelera **pathlib.Path.glob()** removendo a correspondência de regex redundante.

- gh-97928 [https://github.com/python/cpython/issues/97928]: Reverte parcialmente o comportamento de **tkinter.Text.count()**. Por padrão, ele preserva o comportamento de versões mais antigas do Python, exceto que definir `wantobjects` como 0 não tem mais efeito. Adiciona um novo parâmetro *return_ints*: especificar `return_ints=True` faz com que `Text.count()` sempre retorna a contagem única como um inteiro em vez de uma tupla de 1 elemento ou `None`.

- gh-114628 [https://github.com/python/cpython/issues/114628]: Quando csv.Error é levantada ao manipular TypeError, não exibir o traceback de TypeError.

- gh-85984 [https://github.com/python/cpython/issues/85984]: Adicionado `_POSIX_VDISABLE` de `<unistd.h>` de C para **termios**.

- gh-114965 [https://github.com/python/cpython/issues/114965]: Atualiza pip empacotado para 24.0

- gh-114959 [https://github.com/python/cpython/issues/114959]: **tarfile** não ignora mais erros ao tentar extrair um diretório em cima de um arquivo.

- gh-114894 [https://github.com/python/cpython/issues/114894]: Adiciona **array.array.clear()**.

- gh-114071 [https://github.com/python/cpython/issues/114071]: Suporte a subclasses de tupla usando auto() para valor de membro enum.

- gh-109475 [https://github.com/python/cpython/issues/109475]: Corrige suporte ao valor de opção explícito "–" em **argparse** (por exemplo, `--option=--`).

- gh-49766 [https://github.com/python/cpython/issues/49766]: Corrige comparação **date**-**datetime**. Agora, os métodos de comparação especiais como `__eq__` e `__lt__` retornam **NotImplemented** se um dos comparandos for **date** e o outro for **datetime** em vez de ignorar a parte do tempo e o fuso horário ou retornar forçadamente "não igual" ou levantar **TypeError**. Ele torna a comparação das subclasses **date** e **datetime** mais simétrica e permite alterar o comportamento padrão substituindo os métodos de comparação especiais em subclasses.

- gh-110190 [https://github.com/python/cpython/issues/110190]: Corrige estruturas ctypes com vetor na plataforma Windows ARM64 definindo `MAX_STRUCT_SIZE` como 32 em stgdict. Patch por Diego Russo

- gh-114678 [https://github.com/python/cpython/issues/114678]: Garante que o aviso de descontinuação para o especificador 'N' no formato **Decimal** não seja levantada para casos em que 'N' aparece em outros lugares no especificador de formato. Baseado no patch por Stefan Krah.

- gh-70303 [https://github.com/python/cpython/issues/70303]: Retorna arquivos e diretórios de **pathlib.Path.glob()** se um padrão termina com "`**`". Anteriormente, apenas diretórios eram retornados.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Melhora o tempo de importação de **importlib.metadata** e **email.utils**.

- gh-113280 [https://github.com/python/cpython/issues/113280]: Corrige um vazamento de soquete aberto em casos raros quando ocorria um erro na criação de **ssl.SSLSocket**.

- gh-77749 [https://github.com/python/cpython/issues/77749]: **email.policy.EmailPolicy.fold()** agora sempre codifica caracteres não ASCII em cabeçalhos se **utf8** for falso.

- gh-83383 [https://github.com/python/cpython/issues/83383]: A sincronização do banco de dados **`dbm.dumb`** agora é no-op se não houver modificação desde a abertura ou última sincronização. O arquivo de diretório para um banco de dados **`dbm.dumb`** vazio recém-criado agora é criado imediatamente após a abertura, em vez de adiar isso até a sincronização ou o fechamento.

- gh-91602 [https://github.com/python/cpython/issues/91602]: Adiciona o parâmetro somente-nomeado *filter* a **`sqlite3.Connection.iterdump()`** para filtrar objetos de banco de dados para despejar. Patch por Mariusz Felisiak.

- gh-112451 [https://github.com/python/cpython/issues/112451]: Proíbe criar subclasse de Python puro **`datetime.timezone`**. Isso é consistente com a implementação da extensão C. Patch por Mariusz Felisiak.

- gh-69893 [https://github.com/python/cpython/issues/69893]: Adiciona o método **`close()`** para o iterador retornado por **`xml.etree.ElementTree.iterparse()`**.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Reduz o tempo de importação do módulo **`threading`** em ~50%. Patch por Daniel Hollas.

- gh-114492 [https://github.com/python/cpython/issues/114492]: Torna o resultado de **`termios.tcgetattr()`** reproduzível no Alpine Linux. Anteriormente, ele poderia deixar um lixo aleatório em alguns campos.

- gh-114315 [https://github.com/python/cpython/issues/114315]: Torna **`threading.Lock`** uma classe real, não uma função de fábrica. Adiciona `__new__` ao tipo `_thread.lock`.

- gh-100414 [https://github.com/python/cpython/issues/100414]: Adiciona **`dbm.sqlite3`** como um backend para **`dbm`**, e torna-o o novo backend padrão **`dbm`**. Patch por Raymond Hettinger e Erlend E. Aasland.

- gh-113267 [https://github.com/python/cpython/issues/113267]: Reverte alterações em gh-106584 [https://github.com/python/cpython/issues/106584] que tornavam as

chamadas dos métodos `TestResult` `startTest()` e `stopTest()` desbalanceadas.

- gh-75128 [https://github.com/python/cpython/issues/75128]: Ignora uma **`OSError`** em **`asyncio.BaseEventLoop.create_server()`** quando o IPv6 estiver disponível, mas a interface não puder realmente suportá-lo.

- gh-114423 [https://github.com/python/cpython/issues/114423]: As entradas `_DummyThread` em `threading._active` agora são removidas automaticamente quando o thread relacionado morre.

- gh-114257 [https://github.com/python/cpython/issues/114257]: Descarta o erro **`FileNotFound`** em **`ctypes.util.find_library()`** e retorna apenas `None` no Linux.

- gh-114321 [https://github.com/python/cpython/issues/114321]: Expõe mais constantes específicas da plataforma no módulo **`fcntl`** no Linux, macOS, FreeBSD e NetBSD.

- gh-114328 [https://github.com/python/cpython/issues/114328]: A **`tty.setcbreak()`** e a nova **`tty.cfmakecbreak()`** não limpam mais o sinalizador ICRLF de entrada do terminal. Isso corrige uma regressão introduzida na 3.12 que não correspondia mais a como os sistemas operacionais definem o modo cbreak em suas páginas de manual `stty(1)`.

- gh-114281 [https://github.com/python/cpython/issues/114281]: Remove dicas de tipo de `Lib/asyncio/staggered.py`. As anotações no projeto typeshed [https://github.com/python/typeshed] devem ser usadas em vez disso.

- gh-101438 [https://github.com/python/cpython/issues/101438]: Evite o ciclo de referência em ElementTree.iterparse. O iterador retornado por `ElementTree.iterparse` pode manter um descritor de arquivo. O ciclo de referência impedia a limpeza rápida do descritor de arquivo se o iterador retornado não estivesse esgotado.

- gh-114198 [https://github.com/python/cpython/issues/114198]: A assinatura para o método `__replace__` em **`dataclasses`** agora tem o primeiro argumento chamado `self`, em vez de `obj`.

- gh-104522 [https://github.com/python/cpython/issues/104522]: **`OSError`** levantada ao executar um subprocesso agora tem apenas o atributo *filename* definido como *cwd* se o erro foi causado por uma tentativa malsucedida de alterar o diretório atual.

- gh-114149 [https://github.com/python/cpython/issues/114149]: Enum: manipula corretamente subclasses de tuplas em `__new__` personalizado.

- gh-83648 [https://github.com/python/cpython/issues/83648]: Suporta descontinuação de opções, argumentos posicionais e subcomandos em **`argparse`**.

- gh-114087 [https://github.com/python/cpython/issues/114087]: Acelera `dataclasses.asdict` em até 1,35x.

- gh-109534 [https://github.com/python/cpython/issues/109534]: Corrige um vazamento de referência em **`asyncio.selector_events.BaseSelectorEventLoop`** quando negociações SSL falham. Patch foi contribuição de Jamie Phan.

- gh-79634 [https://github.com/python/cpython/issues/79634]: Aceita objetos caminho ou similares como padrões em **`pathlib.Path.glob()`** e **`rglob()`**.

- gh-112202 [https://github.com/python/cpython/issues/112202]: Garante que uma chamada **`asyncio.Condition.notify()`** não seja perdida se a `Task` despertada for cancelada simultaneamente ou encontrar qualquer outro erro.

- gh-113951 [https://github.com/python/cpython/issues/113951]: Corrige o comportamento dos métodos `tag_unbind()` das classes **`tkinter.Text`** e **`tkinter.Canvas`** com três argumentos. Anteriormente, `widget.tag_unbind(tag, sequence, funcid)` destruía a vinculação atual para *sequence*, deixando *sequence* desvinculado, e excluía o comando *funcid*. Agora, ele remove apenas *funcid* da vinculação para *sequence*, mantendo outros comandos, e exclui o comando *funcid*. Ele deixa *sequence* desvinculado somente se *funcid* foi o último comando vinculado.

- gh-97959 [https://github.com/python/cpython/issues/97959]: Corrige métodos de classe de renderização, métodos vinculados, métodos e apelidos de função em `pydoc`. Métodos de classe não têm mais a nota “método de instância builtins.type”. Notas correspondentes agora são adicionadas para métodos de classe e não vinculados. Apelidos de método e função agora têm referências ao módulo ou à classe onde a origem foi definida se for diferente da atual. Métodos vinculados agora são listados na seção de métodos estáticos. Métodos de classes internas agora são suportados, bem como métodos de classes Python.

- gh-113796 [https://github.com/python/cpython/issues/113796]: Adiciona mais verificações de validação no construtor `csv.Dialect`. `ValueError` agora é levantada se o mesmo caractere for usado em funções diferentes.

- gh-113732 [https://github.com/python/cpython/issues/113732]: Corrige suporte de `QUOTE_NOTNULL` e `QUOTE_STRINGS` em `csv.reader()`.

- gh-113225 [https://github.com/python/cpython/issues/113225]: Acelera `pathlib.Path.walk()` usando `os.DirEntry.path` onde possível.

- gh-89039 [https://github.com/python/cpython/issues/89039]: Quando o método replace() é chamado em uma subclasse de datetime, date ou time, chamar corretamente o construtor derivado. Anteriormente, apenas o construtor da classe base era chamado.

  Além disso, certifica-se de passar valores de fold diferentes de zero ao criar subclasses em vários métodos. Anteriormente, fold era ignorado silenciosamente.

- gh-112919 [https://github.com/python/cpython/issues/112919]: Aceleração de `datetime.datetime.replace()`, `datetime.date.replace()` e `datetime.time.replace()`.

- gh-59013 [https://github.com/python/cpython/issues/59013]: Define o ponto de interrupção na primeira linha executável da função, em vez da linha de definição da função quando o usuário faz `break func` usando `pdb`

- gh-112343 [https://github.com/python/cpython/issues/112343]: Melhora o tratamento de variáveis de conveniência pdb para evitar a substituição de conteúdo de string.

- gh-112240 [https://github.com/python/cpython/issues/112240]: Adiciona opção à CLI do módulo de calendar para especificar o dia da semana para começar cada semana. Patch por Steven Ward.

- gh-111741 [https://github.com/python/cpython/issues/111741]: Reconhece `image/webp` como um formato padrão no módulo **`mimetypes`**.

- gh-43457 [https://github.com/python/cpython/issues/43457]: Corrige o método do widget **`tkinter`** **`wm_attributes()`**. Agora ele aceita o nome do atributo sem o prefixo menos para obter atributos de janela e permite especificar atributos e valores para ser definido como argumentos nomeados. Adiciona um novo argumento nomeado opcional *return_python_dict*: chamar `w.wm_attributes(return_python_dict=True)` retorna os atributos como um dict em vez de uma tupla. Chamar `w.wm_attributes()` agora retorna uma tupla em vez de uma string se *wantobjects* foi definido como 0.

- gh-82626 [https://github.com/python/cpython/issues/82626]: Muitas funções agora emitem um aviso se um valor booleano for passado como um argumento de descritor de arquivo.

- gh-111051 [https://github.com/python/cpython/issues/111051]: Adicionada verificação para modificação de arquivo durante a depuração com **`pdb`**

- gh-110345 [https://github.com/python/cpython/issues/110345]: Mostra o patchlevel Tcl/Tk (em vez da versão) em **`tkinter._test()`**.

- gh-38807 [https://github.com/python/cpython/issues/38807]: Corrige a condição de corrida em **`trace`**. Em vez de verificar se um diretório existe e criá-lo, chame diretamente **`os.makedirs()`** com o kwarg `exist_ok=True`.

- gh-75705 [https://github.com/python/cpython/issues/75705]: Define o envelope unixfrom em **`mailbox.mbox`** e **`mailbox.MMDF`**.

- gh-106233 [https://github.com/python/cpython/issues/106233]: Corrige o stacklevel em `InvalidTZPathWarning` durante a importação do módulo **`zoneinfo`**.

- gh-105102 [https://github.com/python/cpython/issues/105102]: Permita que **`ctypes.Union`** seja aninhado em **`ctypes.Structure`** quando o endianness do sistema for o oposto das classes.

- gh-104282 [https://github.com/python/cpython/issues/104282]: Corrige a desreferência de ponteiro nulo em **`lzma._decode_filter_properties()`** devido ao tratamento impróprio de filtros BCJ com propriedades de comprimento zero. Patch por Radislav Chugunov.

- gh-96471 [https://github.com/python/cpython/issues/96471]: Adiciona terminação **`queue.Queue`** com **`shutdown()`**.

- gh-101599 [https://github.com/python/cpython/issues/101599]: Altera a formatação de opções de sinalizador argparse para remover redundância.

- gh-85984 [https://github.com/python/cpython/issues/85984]: Adiciona funções pseudoterminais POSIX **`os.posix_openpt()`**, **`os.grantpt()`**, **`os.unlockpt()`** e **`os.ptsname()`**.

- gh-102512 [https://github.com/python/cpython/issues/102512]: Quando **`os.fork()`** é chamado de um thread estrangeiro (também conhecido como `_DummyThread`), o tipo do thread em um processo filho é alterado para `_MainThread`. Também alterou seu nome e status como daemon, agora pode ser unido.

- gh-88569 [https://github.com/python/cpython/issues/88569]: Adiciona **`os.path.isreserved()`**, que identifica nomes de caminho reservados como “NUL”, “AUX” e “CON”. Esta função está disponível somente no Windows.

  Deprecate **`pathlib.PurePath.is_reserved()`**.

- bpo-38364 [https://bugs.python.org/issue?@action=redirect&bpo=38364]: As funções `inspect isgeneratorfunction`, `iscoroutinefunction`, `isasyncgenfunction` agora oferecem suporte a funções encapsuladas `functools.partialmethod` da mesma forma que oferecem suporte a `functools.partial`.

Documentação

- gh-115233 [https://github.com/python/cpython/issues/115233]: Corrige um exemplo para **`LoggerAdapter`** no Logging Cookbook.

- gh-114123 [https://github.com/python/cpython/issues/114123]: Move a docstring do módulo **`csv`** para o módulo **`csv`** em vez de reexportá-la do módulo interno **`_csv`** e remove `__doc__` de `csv.__all__`.

  Move **`csv.__version__`** para o módulo **`csv`** em vez de reexportá-lo do módulo interno **`_csv`** e remove `__version__` de `csv.__all__`.

Testes

- gh-114099 [https://github.com/python/cpython/issues/114099]: Adicionadas exclusões de teste necessárias para executar o conjunto de testes no iOS.
- gh-105089 [https://github.com/python/cpython/issues/105089]: Corrige o teste `test.test_zipfile.test_core.TestWithDirectory.test_create_directory_with_write` no AIX fazendo um AND bit a bit de 0xFFFF no modo , para que ele fique sincronizado com `zinfo.external_attr`

Construção

- gh-115167 [https://github.com/python/cpython/issues/115167]: Evita incluir `vcruntime140_threads.dll` ao Python ao construir com o Visual Studio 2022 versão 17.8.
- gh-113632 [https://github.com/python/cpython/issues/113632]: Promove o WASI para uma plataforma de nível 2 e remove o Emscripten do nível 3 em configure.ac.

- gh-114099 [https://github.com/python/cpython/issues/114099]: configure e Makefile foram refatorados para acomodar construções de framework em plataformas Apple diferentes do macOS.
- gh-114875 [https://github.com/python/cpython/issues/114875]: Adiciona **`getgrent()`** como um pré-requisito para construir o módulo **`grp`**.

Windows

- gh-115049 [https://github.com/python/cpython/issues/115049]: Corrige a falha do iniciador `py.exe` quando executado como usuários sem perfis de usuário.
- gh-115009 [https://github.com/python/cpython/issues/115009]: Atualize o instalador do Windows para usar o SQLite 3.45.1.
- gh-109991 [https://github.com/python/cpython/issues/109991]: Atualize a construção do Windows para usar o OpenSSL 3.0.13.
- gh-111239 [https://github.com/python/cpython/issues/111239]: Atualiza as construções do Windows para usar o zlib v1.3.1.
- gh-100107 [https://github.com/python/cpython/issues/100107]: O iniciador `py.exe` não tentará mais executar o redirecionador da Microsoft Store ao iniciar um script contendo um shebang `/usr/bin/env`
- gh-112984 [https://github.com/python/cpython/issues/112984]: Adiciona binários de threads livres ao instalador do Windows como um componente opcional.
- gh-89240 [https://github.com/python/cpython/issues/89240]: Permite que **`multiprocessing`** crie pools com mais de 62 processos.

macOS

- gh-115009 [https://github.com/python/cpython/issues/115009]: Atualiza o instalador do macOS para usar o SQLite 3.45.1.
- gh-109991 [https://github.com/python/cpython/issues/109991]: Atualiza o instalador do macOS para usar OpenSSL 3.0.13.
- gh-114490 [https://github.com/python/cpython/issues/114490]: Adiciona suporte de ligação Mach-O para **`platform.architecture()`**.
- gh-87804 [https://github.com/python/cpython/issues/87804]: No macOS, o resultado de `os.statvfs` e `os.fstatvfs` agora relata corretamente o tamanho de discos muito grandes, em versões anteriores o número

relatado de blocos estava errado para discos com pelo menos 2**32 blocos.

IDLE

- gh-96905 [https://github.com/python/cpython/issues/96905]: No código de idlelib, para de redefinir os tipos embutidos ‘dict’ e ‘object’.
- gh-103820 [https://github.com/python/cpython/issues/103820]: Revisa as ligações do IDLE para que os eventos do botão do mouse 4/5 em sistemas de janelas não X11 (por exemplo, Win32 e Aqua) não sejam confundidos com rolagem.

Ferramentas/Demos

- gh-113516 [https://github.com/python/cpython/issues/113516]: Não define `LDSHARED` ao construir para WASI.
- gh-109991 [https://github.com/python/cpython/issues/109991]: Atualiza fluxos de trabalho do GitHub CI para usar OpenSSL 3.0.13 e multissltests para usar 1.1.1w, 3.0.13, 3.1.5 e 3.2.1.
- gh-115015 [https://github.com/python/cpython/issues/115015]: Corrige um bug no Argument Clinic que gerava código incorreto para métodos sem parâmetros que usam a convenção de chamada METH_METHOD | METH_FASTCALL | METH_KEYWORDS. Apenas a contagem de parâmetros posicionais foi verificada; qualquer argumento nomeado passado seria aceito silenciosamente.

API C

- gh-111140 [https://github.com/python/cpython/issues/111140]: Adiciona as funções **`PyLong_AsNativeBytes()`**, **`PyLong_FromNativeBytes()`** e **`PyLong_FromUnsignedNativeBytes()`**.

- gh-114685 [https://github.com/python/cpython/issues/114685]: **`PyBuffer_FillInfo()`** agora levanta uma **`SystemError`** se chamada com **`PyBUF_READ`** ou **`PyBUF_WRITE`** como sinalizadores. Esses sinalizadores devem ser usados apenas com a API C `PyMemoryView_*`.

- gh-114685 [https://github.com/python/cpython/issues/114685]: `PyObject_GetBuffer()` agora levanta uma `SystemError` se chamado com `PyBUF_READ` ou `PyBUF_WRITE` como sinalizadores. Esses sinalizadores devem ser usados somente com a API C `PyMemoryView_*`.

- gh-114626 [https://github.com/python/cpython/issues/114626]: Adiciona os typedefs `PyCFunctionFast` e `PyCFunctionFastWithKeywords` (idênticos aos typedefs `_PyCFunctionFast` e `_PyCFunctionFastWithKeywords` existentes, apenas sem um prefixo _ inicial).

- gh-114329 [https://github.com/python/cpython/issues/114329]: Adiciona `PyList_GetItemRef()`, que é semelhante a `PyList_GetItem()`, mas retorna uma referência forte em vez de uma referência emprestada.

- gh-110850 [https://github.com/python/cpython/issues/110850]: Adiciona a API C de PyTime:

  - Tipo `PyTime_t`.
  - Constantes `PyTime_MIN` e `PyTime_MAX`.
  - Funções `PyTime_AsSecondsDouble()`, `PyTime_Monotonic()`, `PyTime_PerfCounter()` e `PyTime_Time()`.

  Patch por Victor Stinner.

- gh-112066 [https://github.com/python/cpython/issues/112066]: Adiciona `PyDict_SetDefaultRef()`: insere uma chave e um valor em um dicionário se a chave ainda não estiver presente. Isto é similar a `dict.setdefault()`, mas retorna um valor inteiro indicando se a chave já estava presente. Também é similar a `PyDict_SetDefault()`, mas retorna uma referência forte em vez de uma referência emprestada.

Python 3.13.0 alpha 3

Data de lançamento: 2024-01-17

Segurança

- gh-113659 [https://github.com/python/cpython/issues/113659]: Pula arquivos `.pth` com nomes começando com um ponto ou atributo de arquivo oculto.
- gh-112302 [https://github.com/python/cpython/issues/112302]: Criado um documento de lista de materiais de software e ferramentas para rastrear dependências.

Núcleo e embutidos

- gh-107901 [https://github.com/python/cpython/issues/107901]: O compilador duplica blocos básicos que têm uma verificação de quebra de avaliação, nenhum número de linha e vários predecessores.

- gh-107901 [https://github.com/python/cpython/issues/107901]: Um salto que deixa um manipulador de exceção de volta ao código normal não verifica mais o quebra de avaliação.

- gh-113655 [https://github.com/python/cpython/issues/113655]: Define o limite de recursão C para 4000 no Windows e 10000 no Linux/OSX. Isso parece estar perto do ponto ideal para manter a segurança, mas não comprometer a retrocompatibilidade.

- gh-113710 [https://github.com/python/cpython/issues/113710]: Adiciona efeitos de pilha digitados ao DSL do interpretador, junto com várias anotações de instruções.

- gh-77046 [https://github.com/python/cpython/issues/77046]: No Windows, os descritores de arquivo que envolvem identificadores do Windows agora são criados não herdáveis por padrão (**PEP 446** [https://peps.python.org/pep-0446/]). Patch por Zackery Spytz e Victor Stinner.

- gh-113853 [https://github.com/python/cpython/issues/113853]: Garante que todos os executores progridam. Isso garante que a execução de nível 2 sempre progrida.

- gh-113753 [https://github.com/python/cpython/issues/113753]: Corrige um problema em que o finalizador de objetos `PyAsyncGenASend` pode não

ser chamado se eles forem alocados de uma lista livre.

- gh-107901 [https://github.com/python/cpython/issues/107901]: Compilador alterado para que saltos sintéticos que não estejam no final do laço não verifiquem mais o disjuntor de avaliação.

- gh-113703 [https://github.com/python/cpython/issues/113703]: Corrige uma regressão no módulo **`codeop`** que estava fazendo com que ele identificasse incorretamente f-strings incompletas. Patch por Pablo Galindo

- gh-89811 [https://github.com/python/cpython/issues/89811]: Verifica se há uma `tp_version_tag` válida antes de executar especializações de bytecode que dependem desse valor ser utilizável.

- gh-111488 [https://github.com/python/cpython/issues/111488]: Mensagem de erro alterada em caso de nenhuma palavra reservada ‘in’ após ‘for’ em compreensões de lista

- gh-113657 [https://github.com/python/cpython/issues/113657]: Corrige um problema que fazia com que atualizações importantes de ponteiros de instruções fossem otimizadas fora dos rastreamentos de nível dois.

- gh-113603 [https://github.com/python/cpython/issues/113603]: Corrige um bug em que um NOP redundante não é removido, causando falha em uma asserção no compilador no modo de depuração.

- gh-113602 [https://github.com/python/cpython/issues/113602]: Corrige um erro que fazia com que o analisador sintático tentasse sobrescrever erros existentes e travasse no processo. Patch por Pablo Galindo

- gh-113486 [https://github.com/python/cpython/issues/113486]: Não emite mais eventos `PY_UNWIND` espúrios para chamadas otimizadas para classes.

- gh-113297 [https://github.com/python/cpython/issues/113297]: Corrige falha de segmentação no compilador na instrução with com 19 gerenciadores de contexto.

- gh-113212 [https://github.com/python/cpython/issues/113212]: Melhora as mensagens de erro de `super`.

- gh-111375 [https://github.com/python/cpython/issues/111375]: Usa somente `NULL` na pilha de exceções para indicar que uma exceção foi tratada. Patch por Carey Metcalfe.

- gh-112215 [https://github.com/python/cpython/issues/112215]: Aumenta o limite de recursão C por um fator de 3 para compilações não depuradas, exceto para plataformas webassembly e s390 que não foram alteradas. Isso atenua algumas regressões no 3.12 com recursão profunda misturando código interno (C) e Python.

- gh-113054 [https://github.com/python/cpython/issues/113054]: Corrigido bug onde um NOP redundante não é removido, causando falha em uma asserção no compilador no modo de depuração.

- gh-106905 [https://github.com/python/cpython/issues/106905]: Usa o estado por analisador sintático AST em vez do estado global para rastrear a profundidade da recursão dentro do analisador sintático AST para evitar uma possível condição de corrida devido à análise simultânea.

  O problema apareceu principalmente na versão 3.11 por usuários multithread de `ast.parse()`. Na versão 3.12, uma alteração em quando a coleta de lixo pode ser acionada impediu que a condição de corrida ocorresse.

- gh-108866 [https://github.com/python/cpython/issues/108866]: Altera a API e o contrato de `_PyExecutorObject` para retornar o ponteiro next_instr, em vez do quadro, e para sempre executar pelo menos uma instrução.

- gh-90350 [https://github.com/python/cpython/issues/90350]: Otimiza as funções embutidas `min()` e `max()`.

- gh-112943 [https://github.com/python/cpython/issues/112943]: Calcula corretamente os deslocamentos de coluna final para tokens multilinha no módulo `tokenize`. Patch por Pablo Galindo

- gh-112125 [https://github.com/python/cpython/issues/112125]: Corrige `None.__ne__(None)` retornando `NotImplemented` em vez de `False`.

- gh-74616 [https://github.com/python/cpython/issues/74616]: **`input()`** agora levanta uma ValueError quando emitido no terminal se o prompt contiver caracteres nulos incorporados em vez de truncá-lo silenciosamente.

- gh-112716 [https://github.com/python/cpython/issues/112716]: Corrige SystemError na instrução `import` e nos métodos `__reduce__()` de tipos embutidos quando `__builtins__` não é um dict.

- gh-112730 [https://github.com/python/cpython/issues/112730]: Usa cor para destacar locais de erro em tracebacks. Patch por Pablo Galindo

- gh-112625 [https://github.com/python/cpython/issues/112625]: Corrige um bug em que um objeto bytearray poderia ser limpo durante a iteração sobre um argumento no método `bytearray.join()` que poderia resultar na leitura de memória após ser liberado.

- gh-112660 [https://github.com/python/cpython/issues/112660]: Não limpa erros inesperados durante mensagens de erro de formatação para ImportError e AttributeError para módulos.

- gh-105967 [https://github.com/python/cpython/issues/105967]: Solução alternativa para um bug na biblioteca zlib da plataforma macOS da Apple, em que **`zlib.crc32()`** e **`binascii.crc32()`** poderiam produzir resultados incorretos em entradas de vários gigabytes. Incluindo ao usar **`zipfile`** em zips contendo dados grandes.

- gh-95754 [https://github.com/python/cpython/issues/95754]: Fornece uma mensagem de erro melhor ao acessar atributos inválidos em módulos parcialmente inicializados. A origem do módulo que está sendo acessado agora está incluída na mensagem para ajudar com o problema comum de sombreamento de outros módulos.

- gh-112217 [https://github.com/python/cpython/issues/112217]: Adiciona verificação para o tipo de `__cause__` retornado da chamada do tipo `T` em `raise from T`.

- gh-111058 [https://github.com/python/cpython/issues/111058]: Altera coro.cr_frame/gen.gi_frame para retornar `None` após a corrotina/gerador ter sido fechado. Isso corrige um bug em que **`getcoroutinestate()`** e **`getgeneratorstate()`** retornam o estado errado para uma corrotina/gerador fechado.

- gh-112388 [https://github.com/python/cpython/issues/112388]: Corrige um erro que estava fazendo o analisador tentar sobrescrever erros do tokenizador. Patch por pablo Galindo

- gh-112387 [https://github.com/python/cpython/issues/112387]: Corrige posições de erro para strings decodificadas com erros de análise léxica reversa. Patch por Pablo Galindo

- gh-99606 [https://github.com/python/cpython/issues/99606]: Torna o código gerado para uma f-string vazia idêntico ao código de uma string normal vazia.

- gh-112367 [https://github.com/python/cpython/issues/112367]: Evita comportamento indefinido ao usar os trampolins do perf não liberando as arenas de código até o desligamento. Patch por Pablo Galindo

- gh-112320 [https://github.com/python/cpython/issues/112320]: O tradutor de nível 2 agora rastreia o nível de confiança para permanecer “no rastreamento” (ou seja, não sair de volta para o interpretador de nível 1) para instruções de ramificação com base no número de bits definidos no “contador” de ramificação. A tradução de rastreamento termina quando a confiança cai abaixo de 1/3.

- gh-109598 [https://github.com/python/cpython/issues/109598]: **`PyComplex_RealAsDouble()`**/**`PyComplex_ImagAsDouble()`** agora tenta converter um objeto para uma instância **`complex`** usando seu método `__complex__()` antes de retornar ao método `__float__()`. Patch por Sergey B Kirpichev.

- gh-94606 [https://github.com/python/cpython/issues/94606]: Corrige UnicodeEncodeError quando **`email.message.get_payload()`** lê uma mensagem com um caractere substituto Unicode e o conteúdo da

mensagem não está bem formado para codificação de escape substituto. Patch por Sidney Markowitz.

- bpo-21861 [https://bugs.python.org/issue?@action=redirect&bpo=21861]: Usa o nome de classe real do objeto em **`_io.FileIO.__repr__()`**, **`_io._WindowsConsoleIO()`** e **`_io.TextIOWrapper.__repr__()`**, para tornar esses métodos amigáveis à subclasse.

- bpo-45369 [https://bugs.python.org/issue?@action=redirect&bpo=45369]: Remove soluções alternativas do LibreSSL conforme a **PEP 644** [https://peps.python.org/pep-0644/].

- bpo-34392 [https://bugs.python.org/issue?@action=redirect&bpo=34392]: Adicionada **`sys._is_interned()`**.

Biblioteca

- gh-114077 [https://github.com/python/cpython/issues/114077]: Corrige possível **`OverflowError`** em **`socket.socket.sendfile()`** ao passar *count* maior que 2 GiB em plataforma de 32 bits.

- gh-111803 [https://github.com/python/cpython/issues/111803]: **`plistlib`** agora oferece suporte ao carregamento de listas aninhadas mais profundamente em formato binário.

- gh-114014 [https://github.com/python/cpython/issues/114014]: Corrige um bug em **`fractions.Fraction`** onde uma string inválida usando `d` na parte decimais cria um erro diferente comparado a outras letras/caracteres inválidos. Patch por Jeremiah Gabriel Pascual.

- gh-108364 [https://github.com/python/cpython/issues/108364]: **`sqlite3.Connection.iterdump()`** agora garante que o suporte a chave estrangeira seja desabilitado antes de despejar o esquema do banco de dados, se houver qualquer violação de chave estrangeira. Patch por Erlend E. Aasland e Mariusz Felisiak.

- gh-113971 [https://github.com/python/cpython/issues/113971]: O atributo `._compresslevel` protegido anteriormente por **`zipfile.ZipInfo`** foi

tornado público como `.compress_level` com o antigo nome `_compresslevel` permanecendo disponível como uma propriedade para manter a compatibilidade.

- gh-113877 [https://github.com/python/cpython/issues/113877]: Corrige o método `winfo_pathname()` do **`tkinter`** no Windows de 64 bits.

- gh-113868 [https://github.com/python/cpython/issues/113868]: Adicionados **`mmap.MAP_NORESERVE`**, **`mmap.MAP_NOEXTEND`**, **`mmap.MAP_HASSEMAPHORE`**, **`mmap.MAP_NOCACHE`**, **`mmap.MAP_JIT`**, **`mmap.MAP_RESILIENT_CODESIGN`**, **`mmap.MAP_RESILIENT_MEDIA`**, **`mmap.MAP_32BIT`**, **`mmap.MAP_TRANSLATED_ALLOW_EXECUTE`**, **`mmap.MAP_UNIX03`** e **`mmap.MAP_TPRO`**. Todos eles são sinalizadores `mmap(2)` no macOS.

- gh-113848 [https://github.com/python/cpython/issues/113848]: Os gerenciadores de contexto **`asyncio.TaskGroup()`** e **`asyncio.timeout()`** agora manipulam as subclasses **`CancelledError`**, bem como **`CancelledError`** exato.

- gh-113661 [https://github.com/python/cpython/issues/113661]: unittest runner: Não sai com 5 se os testes foram ignorados. A intenção de sair com 5 era detectar problemas em que o conjunto de testes não foi descoberto. Se pulamos os testes, ele foi descoberto corretamente.

- gh-96037 [https://github.com/python/cpython/issues/96037]: Insere **`TimeoutError`** no contexto da exceção que foi levantada durante a saída de um bloco **`asyncio.timeout()`** expirado.

- gh-113781 [https://github.com/python/cpython/issues/113781]: Silencia AttributeError não levantável quando avisos são emitidos durante a finalização do Python.

- gh-113238 [https://github.com/python/cpython/issues/113238]: Adiciona `Anchor` a `importlib.resources` (para que o código esteja em conformidade com a documentação)

- gh-111693 [https://github.com/python/cpython/issues/111693]: **`asyncio.Condition.wait()`** agora levanta novamente a mesma

instância **`CancelledError`** que pode ter causado sua interrupção. Corrigida a condição de corrida em **`asyncio.Semaphore.acquire()`** quando interrompido com uma **`CancelledError`**.

- gh-113791 [https://github.com/python/cpython/issues/113791]: Adiciona `CLOCK_MONOTONIC_RAW_APPROX` e `CLOCK_UPTIME_RAW_APPROX` a **`time`** no macOS. Esses são relógios disponíveis no macOS 10.12 ou posterior.

- gh-112932 [https://github.com/python/cpython/issues/112932]: Restaura a capacidade de **`zipfile`** de `extractall` de arquivos zip com uma entrada de diretório "/" neles, como é comumente adicionado a zips por alguns exportadores de dados de wiki ou rastreadores de bugs.

- gh-113568 [https://github.com/python/cpython/issues/113568]: Levanta avisos de descontinuação de **`pathlib.PurePath`** e não de sua classe base privada `PurePathBase`.

- gh-113594 [https://github.com/python/cpython/issues/113594]: Corrige **`UnicodeEncodeError`** em **`email`** ao redobrar linhas que contêm parte codificada de 8 bits desconhecida seguida por parte codificada de 8 bits não desconhecida.

- gh-113538 [https://github.com/python/cpython/issues/113538]: Em **`asyncio.StreamReaderProtocol.connection_made()`**, há um retorno de chamada que registra um erro se a tarefa que envolve o "retorno de chamada de conectado" falhar. Este retorno de chamada falharia se a tarefa fosse cancelada. Evite isso verificando se a tarefa foi cancelada primeiro. Se sim, feche o transporte, mas não registre um erro.

- gh-113626 [https://github.com/python/cpython/issues/113626]: Adiciona suporte para o argumento *allow_code* no módulo **`marshal`**. Passar `allow_code=False` previne a serialização e desserialização de objetos de código, o que é incompatível entre versões do Python.

- gh-85567 [https://github.com/python/cpython/issues/85567]: Corrige avisos de recursos para arquivos não fechados nas interfaces de linha de comando **`pickle`** e **`pickletools`**.

- gh-113537 [https://github.com/python/cpython/issues/113537]: Suporte a cargas de `str` em **`plistlib.loads()`**.

- gh-89850 [https://github.com/python/cpython/issues/89850]: Adiciona implementações padrão dos métodos **`pickle.Pickler.persistent_id()`** e **`pickle.Unpickler.persistent_load()`** na implementação C. Chamar `super().persistent_id()` e `super().persistent_load()` em subclasses da implementação C das classes **`pickle.Pickler`** e **`pickle.Unpickler`** não causa mais recursão infinita.

- gh-113569 [https://github.com/python/cpython/issues/113569]: Indica se não houve chamadas reais na falha do unittest **`assert_has_calls()`**.

- gh-101225 [https://github.com/python/cpython/issues/101225]: Aumenta o backlog para objetos **`multiprocessing.connection.Listener`** criados por **`multiprocessing.manager`** e **`multiprocessing.resource_sharer`** para reduzir significativamente o risco de obter um erro de conexão recusada ao criar um **`multiprocessing.connection.Connection`** para eles.

- gh-113568 [https://github.com/python/cpython/issues/113568]: Levanta eventos de auditoria de **`pathlib.Path`** e não de sua classe base privada `PathBase`.

- gh-113543 [https://github.com/python/cpython/issues/113543]: Garante que `webbrowser.MacOSXOSAScript` envie o evento de auditoria `webbrowser.open`.

- gh-113028 [https://github.com/python/cpython/issues/113028]: Quando uma segunda referência a uma string aparece na entrada para **`pickle`**, e a implementação Python está em uso, temos a garantia de que uma única cópia é decapada e um único objeto é compartilhado quando recarregado. Anteriormente, no protocolo 0, quando uma string continha certos caracteres (por exemplo, nova linha), isso resultava em objetos duplicados.

- gh-113421 [https://github.com/python/cpython/issues/113421]: Corrige o logger de multiprocessamento para `%(filename)s`.

- gh-111784 [https://github.com/python/cpython/issues/111784]: Corrige falhas de segmentação no módulo `_elementtree`. Corrige a primeira falha de segmentação durante a desalocação de instâncias `_elementtree.XMLParser` mantendo uma referência forte ao módulo `pyexpat` no estado do módulo para o tempo de vida da cápsula. Corrige a segunda falha de segmentação que acontece no mesmo processo de desalocação mantendo uma referência forte ao módulo `_elementtree` na estrutura `XMLParser` para o tempo de vida do módulo `_elementtree`.

- gh-113407 [https://github.com/python/cpython/issues/113407]: Corrige a importação de **`unittest.mock`** quando o CPython é construído sem docstrings.

- gh-113320 [https://github.com/python/cpython/issues/113320]: Corrige uma regressão no Python 3.12 onde classes **`Protocol`** que não foram marcadas como **`runtime-checkable`** seriam introspectadas desnecessariamente, potencialmente causando exceções se o protocolo tivesse membros problemáticos. Patch por Alex Waygood.

- gh-53502 [https://github.com/python/cpython/issues/53502]: Adiciona uma nova opção `aware_datetime` em **`plistlib`** para carregar ou despejar datetime consciente.

- gh-113358 [https://github.com/python/cpython/issues/113358]: Corrige renderização de tracebacks com exceções com um __getattr__ quebrado

- gh-113214 [https://github.com/python/cpython/issues/113214]: Corrige um `AttributeError` durante término do protocolo SSL asyncio em cenários SSL sobre SSL.

- gh-113246 [https://github.com/python/cpython/issues/113246]: Atualiza o pip empacotado para 23.3.2.

- gh-87264 [https://github.com/python/cpython/issues/87264]: Corrigido o método list() do tarfile para mostrar o tipo de arquivo.

- gh-112182 [https://github.com/python/cpython/issues/112182]: **`asyncio.futures.Future.set_exception()`** agora transforma **`StopIteration`** em **`RuntimeError`** em vez de travar ou apresentar outro comportamento incorreto. Contribuição de patch de Jamie Phan.

- gh-113225 [https://github.com/python/cpython/issues/113225]: Acelera **`pathlib.Path.glob()`** usando **`os.DirEntry.path`** sempre que possível.

- gh-113149 [https://github.com/python/cpython/issues/113149]: Melhora a mensagem de erro quando um vetor JSON ou objeto contém uma vírgula final. Patch por Carson Radtke.

- gh-113117 [https://github.com/python/cpython/issues/113117]: O módulo **`subprocess`** agora pode usar a função **`os.posix_spawn()`** com `close_fds=True` em plataformas onde `posix_spawn_file_actions_addclosefrom_np` está disponível. Patch por Jakub Kulik.

- gh-113199 [https://github.com/python/cpython/issues/113199]: Faz `http.client.HTTPResponse.read1` e `http.client.HTTPResponse.readline` fecharem E/S após ler todos os dados quando o comprimento do conteúdo for conhecido. Patch por Illia Volochii.

- gh-113191 [https://github.com/python/cpython/issues/113191]: Adiciona suporte a **`os.fchmod()`** e um descritor de arquivo em **`os.chmod()`** no Windows.

- gh-113188 [https://github.com/python/cpython/issues/113188]: Corrige **`shutil.copymode()`** e **`shutil.copystat()`** no Windows. Anteriormente, eles funcionavam de forma diferente se *dst* fosse um link simbólico: eles modificavam os bits de permissão do próprio *dst* em vez do arquivo para o qual ele aponta se *follow_symlinks* fosse true ou *src* não fosse um link simbólico, e não modificavam os bits de permissão se *follow_symlinks* fosse false e *src* fosse um link simbólico.

- gh-113119 [https://github.com/python/cpython/issues/113119]: **`os.posix_spawn()`** agora aceita `env=None`, o que faz com que o processo recém-gerado

use o ambiente de processo atual. Patch por Jakub Kulik.

- gh-113202 [https://github.com/python/cpython/issues/113202]: Adiciona uma opção `strict` para `batched()` no módulo `itertools`.

- gh-61648 [https://github.com/python/cpython/issues/61648]: Detecta números de linha de propriedades em doctests.

- gh-113175 [https://github.com/python/cpython/issues/113175]: Sincroniza com importlib_metadata 7.0, incluindo anotações de tipo aprimoradas, problema corrigido com pacotes com links simbólicos em `package_distributions`, adicionado `EntryPoints.__repr__`, introduzido o script `diagnose`, adicionada a propriedade `Distribution.origin` e removido o acesso `EntryPoint` descontinuado por índice numérico (comportamento de tupla).

- gh-59616 [https://github.com/python/cpython/issues/59616]: Adiciona suporte para **`os.lchmod()`** e o argumento *follow_symlinks* em **`os.chmod()`** no Windows. Observe que o valor padrão de *follow_symlinks* em **`os.lchmod()`** é `False` no Windows.

- gh-112559 [https://github.com/python/cpython/issues/112559]: **`signal.signal()`** e **`signal.getsignal()`** não chamam mais `repr` em manipuladores chamáveis. **`asyncio.run()`** e **`asyncio.Runner.run()`** não chamam mais `repr` nos resultados da tarefa. Patch por Yilei Yang.

- gh-112962 [https://github.com/python/cpython/issues/112962]: As funções do módulo **`dis`** adicionam informações de cache à instância **`Instruction`** em vez de criar instâncias falsas **`Instruction`** para representar as entradas de cache.

- gh-112989 [https://github.com/python/cpython/issues/112989]: Reduz a sobrecarga para conectar soquetes com SelectorEventLoop do **`asyncio`**.

- gh-112970 [https://github.com/python/cpython/issues/112970]: Usa **`closefrom()`** no Linux onde disponível (por exemplo, glibc-2.34), em vez de apenas FreeBSD.

- gh-110190 [https://github.com/python/cpython/issues/110190]: Corrige estruturas ctypes com array na plataforma PPC64LE definindo `MAX_STRUCT_SIZE` como 64 em stgdict. Patch por Diego Russo.

- gh-112540 [https://github.com/python/cpython/issues/112540]: A função statistics.geometric_mean() agora retorna zero para conjuntos de dados contendo um zero. Anteriormente, isso levantaria uma exceção.

- gh-87286 [https://github.com/python/cpython/issues/87286]: Adicionados **`LOG_FTP`**, **`LOG_NETINFO`**, **`LOG_REMOTEAUTH`**, **`LOG_INSTALL`**, **`LOG_RAS`** e **`LOG_LAUNCHD`** ao módulo **`syslog`**, todos eles constantes usados no macOS.

- gh-112800 [https://github.com/python/cpython/issues/112800]: Corrige **`asyncio`** `SubprocessTransport.close()` para não lançar `PermissionError` quando usado com executáveis setuid.

- gh-51944 [https://github.com/python/cpython/issues/51944]: Adiciona as seguintes constantes ao módulo **`termios`**. Esses valores estão presentes nos cabeçalhos do sistema macOS: `ALTWERASE`, `B14400`, `B28800`, `B7200`, `B76800`, `CCAR_OFLOW`, `CCTS_OFLOW`, `CDSR_OFLOW`, `CDTR_IFLOW`, `CIGNORE`, `CRTS_IFLOW`, `EXTPROC`, `IUTF8`, `MDMBUF`, `NL2`, `NL3`, `NOKERNINFO`, `ONOEOT`, `OXTABS`, `VDSUSP`, `VSTATUS`.

- gh-79325 [https://github.com/python/cpython/issues/79325]: Corrige um erro de recursão infinita na limpeza de **`tempfile.TemporaryDirectory()`** no Windows.

- gh-94692 [https://github.com/python/cpython/issues/94692]: **`shutil.rmtree()`** agora captura apenas exceções OSError. Anteriormente, uma versão resistente a ataques de symlink de `shutil.rmtree()` podia ignorar ou passar para o manipulador de erros uma exceção arbitrária quando argumentos inválidos eram fornecidos.

- gh-112736 [https://github.com/python/cpython/issues/112736]: O uso de símbolos del-safe em `subprocess` foi refatorado para permitir o uso em ambientes de construção multiplataforma.

- gh-112727 [https://github.com/python/cpython/issues/112727]: Acelera `pathlib.Path.absolute()`. Patch por Barney Gale.

- gh-74690 [https://github.com/python/cpython/issues/74690]: Aceleração de verificações de `issubclass()` em relação a `protocolos runtime-checkable` simples em cerca de 6%. Patch por Alex Waygood.

- gh-74690 [https://github.com/python/cpython/issues/74690]: Aceleração de verificações de `isinstance()` em cerca de 20% para `protocolos verificáveis em tempo de execução` que têm apenas um membro chamável. Aceleração de verificações de `issubclass()` para esses protocolos em cerca de 10%. Patch por Alex Waygood.

- gh-112645 [https://github.com/python/cpython/issues/112645]: Remoção de erro de descontinuação ao passar `onerror` para `shutil.rmtree()`.

- gh-112640 [https://github.com/python/cpython/issues/112640]: Adiciona o parâmetro `kwdefaults` a `types.FunctionType` para definir valores padrão de argumento nomeado.

- gh-112622 [https://github.com/python/cpython/issues/112622]: Garante que o parâmetro `name` seja passado para o laço de eventos em `asyncio.create_task()`.

- gh-112618 [https://github.com/python/cpython/issues/112618]: Corrige um bug de armazenamento em cache relacionado a `typing.Annotated`. `Annotated[str, True]` não é mais idêntico a `Annotated[str, 1]`.

- gh-112334 [https://github.com/python/cpython/issues/112334]: Corrige uma regressão de desempenho no `subprocess` do 3.12 no Linux, onde ele não usaria mais a chamada de sistema de caminho rápido `vfork()` quando poderia devido a um bug lógico, em vez disso, voltando para o seguro, mas mais lento `fork()`.

  Também foi corrigido um segundo bug de segurança potencial do 3.12.0. Se um valor de `extra_groups=[]` fosse passado para `subprocess.Popen` ou APIs relacionadas, a chamada de sistema

`setgroups(0, NULL)` subjacente para limpar a lista de grupos não seria feita no processo filho antes de `exec()`.

Isso foi identificado por meio de inspeção de código no processo de correção do primeiro bug.

- gh-110190 [https://github.com/python/cpython/issues/110190]: Corrige estruturas ctypes com array na plataforma Arm definindo `MAX_STRUCT_SIZE` como 32 em stgdict. Patch por Diego Russo.

- gh-81194 [https://github.com/python/cpython/issues/81194]: Corrige uma falha em **`socket.if_indextoname()`** com valor específico (UINT_MAX). Corrigir um estouro de inteiro em **`socket.if_indextoname()`** em plataformas não Windows de 64 bits.

- gh-112578 [https://github.com/python/cpython/issues/112578]: Corrige uma **`RuntimeWarning`** espúrio ao executar o módulo **`zipfile`**.

- gh-112516 [https://github.com/python/cpython/issues/112516]: Atualiza a cópia empacotada do pip para a versão 23.3.1.

- gh-112510 [https://github.com/python/cpython/issues/112510]: Adiciona **`readline.backend`** para os usos do backend readline (`editline` ou `readline`)

- gh-112328 [https://github.com/python/cpython/issues/112328]: [Enum] Torna `EnumDict`, `EnumDict.member_names`, `EnumType._add_alias_` e `EnumType._add_value_alias_` públicos.

- gh-112509 [https://github.com/python/cpython/issues/112509]: Corrige casos extremos que poderiam fazer com que uma chave estivesse presente nos atributos `__required_keys__` e `__optional_keys__` de um **`typing.TypedDict`**. Patch por Jelle Zijlstra.

- gh-101336 [https://github.com/python/cpython/issues/101336]: Adiciona o parâmetro nomeado `keep_alive` para **`AbstractEventLoop.create_server()`** e **`BaseEventLoop.create_server()`**.

- gh-63284 [https://github.com/python/cpython/issues/63284]: Adicionado suporte para modo TLS-PSK (chave pré-compartilhada) para :mod: módulo `ssl`.

- gh-112414 [https://github.com/python/cpython/issues/112414]: Corrige regressão no Python 3.12 onde chamar `repr()` em um módulo que foi importado usando um loader personalizado poderia falhar com `AttributeError`. Patch por Alex Waygood.

- gh-112358 [https://github.com/python/cpython/issues/112358]: Reverte alteração na inicialização `struct.Struct` que quebrou alguns casos de extensão de classes.

- gh-112405 [https://github.com/python/cpython/issues/112405]: Otimiza `pathlib.PurePath.relative_to()`. Patch por Alex Waygood.

- gh-94722 [https://github.com/python/cpython/issues/94722]: Corrige bug onde a comparação entre instâncias de `DocTest` falha se uma delas tiver `None` como seu lineno.

- gh-112361 [https://github.com/python/cpython/issues/112361]: Acelera um pequeno punhado de métodos `pathlib` removendo alguns objetos temporários.

- gh-112345 [https://github.com/python/cpython/issues/112345]: Melhora a mensagem de erro ao tentar chamar `issubclass()` contra um `typing.Protocol` que tem membros não-método. Patch por Randolf Scholz.

- gh-112137 [https://github.com/python/cpython/issues/112137]: Altera a saída `dis` para exibir no-lineno como “–” em vez de “None”.

- gh-112332 [https://github.com/python/cpython/issues/112332]: Descontinua o campo `exc_type` de `traceback.TracebackException`. Adiciona `exc_type_str` para substituí-lo.

- gh-81620 [https://github.com/python/cpython/issues/81620]: Adiciona testes extras para `random.binomialvariate()`

- gh-112292 [https://github.com/python/cpython/issues/112292]: Corrige uma falha em **`readline`** quando importado de um subinterpretador. Patch por Anthony Shaw

- gh-77621 [https://github.com/python/cpython/issues/77621]: Melhora ligeiramente o tempo de importação do módulo **`pathlib`** adiando algumas importações. Patch por Barney Gale.

- gh-112137 [https://github.com/python/cpython/issues/112137]: Altera a saída **`dis`** para exibir rótulos lógicos para alvos de salto em vez de deslocamentos.

- gh-112139 [https://github.com/python/cpython/issues/112139]: Adiciona **`Signature.format()`** para formatar assinaturas para string com opções extras. E usá-lo em **`pydoc`** para renderizar assinaturas mais legíveis que tenham novas linhas entre parâmetros.

- gh-112105 [https://github.com/python/cpython/issues/112105]: Faz **`readline.set_completer_delims()`** funcionar com libedit

- gh-106922 [https://github.com/python/cpython/issues/106922]: Exibe várias linhas com `traceback` quando os erros abrangerem várias linhas.

- gh-111874 [https://github.com/python/cpython/issues/111874]: Ao criar uma classe **`typing.NamedTuple`**, garante que **`__set_name__()`** seja chamado em todos os objetos que definem `__set_name__` e existam nos valores do dicionário de classes da classe `NamedTuple`. Patch por Alex Waygood.

- gh-68166 [https://github.com/python/cpython/issues/68166]: Adiciona suporte ao tipo de elemento “vsapi” em **`tkinter.ttk.Style.element_create()`**.

- gh-110275 [https://github.com/python/cpython/issues/110275]: Os métodos `_replace()` e `__replace__()` da tupla nomeada agora levantam TypeError em vez de ValueError para argumentos nomeados inválidos.

- gh-99367 [https://github.com/python/cpython/issues/99367]: Não desorganiza `sys.path[0]` em **`pdb`** se safe_path estiver definido

- gh-111615 [https://github.com/python/cpython/issues/111615]: Corrige uma regressão causada por uma correção em gh-93162 [https://github.com/python/cpython/issues/93162], na qual você não conseguia configurar uma **`QueueHandler`** sem especificar manipuladores.

- gh-75666 [https://github.com/python/cpython/issues/75666]: Corrige o comportamento do método `unbind()` do widget **`tkinter`** com dois argumentos. Anteriormente, `widget.unbind(sequence, funcid)` destruía a vinculação atual para *sequence*, deixando *sequence* desvinculado, e excluía o comando *funcid*. Agora ele remove apenas *funcid* da vinculação para *sequence*, mantendo outros comandos, e exclui o comando *funcid*. Ele deixa *sequence* desvinculado somente se *funcid* foi o último comando vinculado.

- gh-67790 [https://github.com/python/cpython/issues/67790]: Implementa suporte básico de formatação (largura mínima, alinhamento, preenchimento) para **`fractions.Fraction`**.

- gh-111049 [https://github.com/python/cpython/issues/111049]: Corrige travamento durante coleta de lixo do objeto buffer **`io.BytesIO`**.

- gh-102980 [https://github.com/python/cpython/issues/102980]: Redireciona a saída do comando `interact` de **`pdb`** para o mesmo canal do depurador. Adiciona testes e melhore a documentação.

- gh-102988 [https://github.com/python/cpython/issues/102988]: **`email.utils.getaddresses()`** e **`email.utils.parseaddr()`** agora retornam tuplas de dois elementos `('', '')` em mais situações onde endereços de e-mail inválidos são encontrados em vez de potencialmente imprecisos valores. Adiciona o parâmetro *strict* opcional a estas duas funções: use `strict=False` para obter o comportamento antigo, aceitar entradas malformadas. `getattr(email.utils, 'supports_strict_parsing', False)` pode usar para verificar se o parâmetro *strict* está disponível. Patch por Thomas Dwyer e Victor Stinner para melhorar a correção **CVE 2023-27043** [https://www.cve.org/CVERecord?id=CVE-2023-27043].

- gh-52161 [https://github.com/python/cpython/issues/52161]: **`cmd.Cmd.do_help()`** agora limpa docstrings com **`inspect.cleandoc()`** antes de escrevê-las. Patch por Filip Łapkiewicz.

- gh-82300 [https://github.com/python/cpython/issues/82300]: Adiciona o parâmetro `track` a **`multiprocessing.shared_memory.SharedMemory`** que permite usar blocos de memória compartilhada sem precisar registrar com o rastreador de recursos POSIX que os libera automaticamente na saída do processo.

- gh-110109 [https://github.com/python/cpython/issues/110109]: Adiciona a classe privada `pathlib._PurePathBase`: uma classe base para **`pathlib.PurePath`** que omite certos métodos mágicos. Pode ser tornado público (junto com `_PathBase`) no futuro.

- gh-109858 [https://github.com/python/cpython/issues/109858]: Protege **`zipfile`** de zipbomb “quoted-overlap”. Agora ele levanta BadZipFile ao tentar ler uma entrada que se sobrepõe a outra entrada ou diretório central.

- gh-109786 [https://github.com/python/cpython/issues/109786]: Corrige possíveis vazamentos de referência e travamento ao reinserir o método `__next__()` de **`itertools.pairwise`**.

- gh-91539 [https://github.com/python/cpython/issues/91539]: Melhoria de desempenho pequena (10 - 20%) e trivial de **`urllib.request.getproxies_environment()`**, normalmente útil quando há muitas variáveis de ambiente para analisar.

- gh-103363 [https://github.com/python/cpython/issues/103363]: Adiciona o argumento somente-nomeado *follow_symlinks* a **`pathlib.Path.owner()`** e **`group()`**, com o padrão `True`.

- gh-102130 [https://github.com/python/cpython/issues/102130]: Suporte para conclusão de tabulação em **`cmd`** para `editline`.

- gh-99437 [https://github.com/python/cpython/issues/99437]: **`runpy.run_path()`** agora decodifica objetos caminho ou similares, garantindo que

__file__ e sys.argv[0] do módulo que está sendo executado sejam sempre strings.

- gh-104003 [https://github.com/python/cpython/issues/104003]: Adiciona **`warnings.deprecated()`**, um decorador para marcar funções descontinuadas para verificadores de tipo estático e para avisar sobre o uso de classes e funções obsoletas. Veja **PEP 702** [https://peps.python.org/pep-0702/]. Patch por Jelle Zijlstra.

- gh-103708 [https://github.com/python/cpython/issues/103708]: Torna o nome python codificado, um parâmetro configurável para que diferentes implementações do python possam substituí-lo em vez de fazer grandes diferenças em sysconfig.py

- gh-66515 [https://github.com/python/cpython/issues/66515]: **`mailbox.MH`** agora oferece suporte a pastas que não contêm um arquivo `.mh_sequences` (por exemplo, pastas de cache IMAP do Claws Mail). Patch por Serhiy Storchaka.

- gh-83162 [https://github.com/python/cpython/issues/83162]: Renomeado **`re.error`** para **`PatternError`** para maior clareza, e mantido **`re.error`** para compatibilidade com versões anteriores. Patch por Matthias Bussonnier e Adam Chhina.

- gh-91133 [https://github.com/python/cpython/issues/91133]: Corrige um bug na limpeza de **`tempfile.TemporaryDirectory`**, que agora não desreferencia mais links simbólicos ao contornar erros de permissão do sistema de arquivos.

- bpo-43153 [https://bugs.python.org/issue?@action=redirect&bpo=43153]: No Windows, `tempfile.TemporaryDirectory` mascarava anteriormente um `PermissionError` com `NotADirectoryError` durante a limpeza de diretório. Agora ele levanta corretamente `PermissionError` se os erros não forem ignorados. Patch por Andrei Kulakov e Ken Jin.

- bpo-32731 [https://bugs.python.org/issue?@action=redirect&bpo=32731]: **`getpass.getuser()`** agora levanta **`OSError`** para todas as falhas em

vez de **`ImportError`** em sistemas sem o módulo **`pwd`** ou **`KeyError`** se o banco de dados de senhas estiver vazio.

- bpo-34321 [https://bugs.python.org/issue?@action=redirect&bpo=34321]: **`mmap.mmap`** agora tem um parâmetro *trackfd* no Unix; se for `False`, o descritor de arquivo especificado por *fileno* não será duplicado.

- bpo-35332 [https://bugs.python.org/issue?@action=redirect&bpo=35332]: A função **`shutil.rmtree()`** agora ignora erros ao chamar **`os.close()`** quando *ignore_errors* é `True`, e **`os.close()`** não é mais tentado novamente após erro.

- bpo-35928 [https://bugs.python.org/issue?@action=redirect&bpo=35928]: **`io.TextIOWrapper`** agora manipula corretamente o buffer de decodificação após `read()` e `write()`.

- bpo-26791 [https://bugs.python.org/issue?@action=redirect&bpo=26791]: **`shutil.move()`** agora move um link simbólico para um diretório quando esse diretório é o alvo do link simbólico. Isso fornece o mesmo comportamento do comando mv shell. O comportamento anterior gerou uma exceção. Patch por Jeffrey Kintscher.

- bpo-41422 [https://bugs.python.org/issue?@action=redirect&bpo=41422]: Corrigidos vazamentos de memória de **`pickle.Pickler`** e **`pickle.Unpickler`** envolvendo referências cíclicas por meio do mapeamento de memorando interno.

- bpo-19821 [https://bugs.python.org/issue?@action=redirect&bpo=19821]: A função **`pydoc.ispackage()`** foi descontinuada.

- bpo-40262 [https://bugs.python.org/issue?@action=redirect&bpo=40262]: O método **`ssl.SSLSocket.recv_into()`** não requer mais o argumento *buffer* para implementar `__len__` e suporta buffers com tamanho de item arbitrário.

- bpo-39912 [https://bugs.python.org/issue?@action=redirect&bpo=39912]: **`warnings.filterwarnings()`** e **`warnings.simplefilter()`** agora

levantam exceções apropriadas em vez de `AssertionError`. Patch foi contribuição de Rémi Lapeyre.

- bpo-37260 [https://bugs.python.org/issue?@action=redirect&bpo=37260]: Corrigida uma condição de corrida em **`shutil.rmtree()`** na qual entradas de diretório removidas por outro processo ou thread enquanto `shutil.rmtree()` está em execução podem fazer com que ele levante FileNotFoundError. Patch por Jeffrey Kintscher.

- bpo-36959 [https://bugs.python.org/issue?@action=redirect&bpo=36959]: Corrigidas algumas mensagens de erro para combinações de strings de formato ISO inválidas em `strptime()` que se referiam a diretivas não contidas na string de formato. Patch por Gordon P. Hemsley.

- bpo-18060 [https://bugs.python.org/issue?@action=redirect&bpo=18060]: Corrigido um problema de herança de classe que pode causar falhas de segmentação ao derivar dois ou mais níveis de subclasses de uma classe base de Structure ou Union.

- bpo-29779 [https://bugs.python.org/issue?@action=redirect&bpo=29779]: Adicionada uma nova variável de ambiente **`PYTHON_HISTORY`** para definir o local de um arquivo `.python_history`.

- bpo-21360 [https://bugs.python.org/issue?@action=redirect&bpo=21360]: **`mailbox.Maildir`** agora ignora arquivos com um ponto inicial.

Documentação

- gh-111699 [https://github.com/python/cpython/issues/111699]: Realoca o aviso de descontinuação de `smtpd` para sua própria seção em vez de em `locale` no documento O que há de novo no Python 3.12
- gh-110746 [https://github.com/python/cpython/issues/110746]: Marcação aprimorada para opções/valores válidos para os métodos ttk.treeview.column e ttk.treeview.heading, e para Layouts.
- gh-95649 [https://github.com/python/cpython/issues/95649]: Document que o módulo **`asyncio`** contém código retirado de v0.16.0 do projeto uvloop [https://github.com/MagicStack/uvloop/tree/v0.16.0], bem como as informações de licenciamento do MIT necessárias.

Testes

- gh-111798 [https://github.com/python/cpython/issues/111798]: Desabilita `test_super_deep()` de `test_call` em construções pydebug em WASI; a profundidade da pilha é muito pequena para tornar o teste útil.

- gh-111801 [https://github.com/python/cpython/issues/111801]: Reduz o limite de recursão em `test_isinstance` para `test_infinitely_many_bases()`. Isso evita um estouro de pilha em uma construção pydebug de WASI.

- gh-111802 [https://github.com/python/cpython/issues/111802]: Especifica uma profundidade de recursão baixa para `test_bad_getattr()` em `test.pickletester` para evitar esgotar a pilha em uma construção pydebug para WASI.

- gh-44626 [https://github.com/python/cpython/issues/44626]: Corrige **`os.path.isabs()`** retornando incorretamente `True` quando recebe um caminho que começa com exatamente uma barra (invertida) no Windows.

  Corrige **`pathlib.PureWindowsPath.is_absolute()`** retornando incorretamente `False` para alguns caminhos que começam com duas barras (invertidas).

- gh-113633 [https://github.com/python/cpython/issues/113633]: Use estado do módulo para o módulo de extensão _testcapi.

- gh-109980 [https://github.com/python/cpython/issues/109980]: Corrige `test_tarfile_vs_tar` em `test_shutil` para macOS, onde o tar do sistema pode incluir mais informações no arquivo do que **`shutil.make_archive`**.

- gh-112769 [https://github.com/python/cpython/issues/112769]: Os testes agora comparam corretamente a versão zlib quando **`zlib.ZLIB_RUNTIME_VERSION`** contém sufixos não inteiros. Por exemplo, zlib-ng define a versão como `1.3.0.zlib-ng`.

- gh-112334 [https://github.com/python/cpython/issues/112334]: Adiciona um teste de regressão para verificar se `vfork()` é usado quando esperado por **`subprocess`** em sistemas POSIX habilitados para vfork (Linux).

- gh-108927 [https://github.com/python/cpython/issues/108927]: Corrigida a dependência de ordem na execução de testes no mesmo processo quando um teste que tem submódulos (por exemplo, test_importlib) segue um teste que importa seu submódulo (por exemplo, test_importlib.util) e precede um teste (por exemplo, test_unittest ou test_compileall) que usa esse submódulo.

- bpo-40648 [https://bugs.python.org/issue?@action=redirect&bpo=40648]: Modos de teste que o arquivo pode obter com chmod() no Windows.

Construção

- gh-114013 [https://github.com/python/cpython/issues/114013]: Corrige `Tools/wasm/wasi.py` para não incluir o caminho para `python.wasm` como parte de `HOSTRUNNER`. A variável de ambiente tem como objetivo especificar como executar apenas o host WASI, tendo `python.wasm` e sinalizadores relevantes anexados ao `HOSTRUNNER`. Isso corrige o funcionamento de `make test`.
- gh-113258 [https://github.com/python/cpython/issues/113258]: Alterada a construção do Windows para gravar módulos congelados gerados na árvore de construção em vez da árvore de fontes.
- gh-112305 [https://github.com/python/cpython/issues/112305]: Corrigida a etapa `check-clean-src` realizada em construções fora da árvore para detectar arquivos `$(srcdir)/Python/frozen_modules/*.h` errantes e recomendou etapas de limpeza apropriadas da árvore de fontes para obter uma construção funcional novamente.
- gh-112536 [https://github.com/python/cpython/issues/112536]: Adiciona suporte para sanitizador de thread (TSAN)
- gh-112867 [https://github.com/python/cpython/issues/112867]: Corrige a construção para o caso em que WITH_PYMALLOC_RADIX_TREE=0 definido.
- gh-103065 [https://github.com/python/cpython/issues/103065]: Apresenta `Tools/wasm/wasi.py` para simplificar a execução de uma construção

WASI.

- bpo-11102 [https://bugs.python.org/issue?@action=redirect&bpo=11102]: As funções `os.major()`, `os.makedev()` e `os.minor()` agora estão disponíveis no HP-UX v3.
- bpo-36351 [https://bugs.python.org/issue?@action=redirect&bpo=36351]: Não defina ipv6type ao fazer compilação cruzada.

Windows

- gh-114096 [https://github.com/python/cpython/issues/114096]: Os privilégios de processo que são ativados para criar junções de diretório agora são restaurados posteriormente, evitando alterações de comportamento em outras partes do programa.
- gh-111877 [https://github.com/python/cpython/issues/111877]: As chamadas `os.stat()` estavam retornando valores de tempo incorretos para arquivos que não podiam ser acessados diretamente.
- gh-111973 [https://github.com/python/cpython/issues/111973]: Atualiza o instalador do Windows para usar o SQLite 3.44.2.
- gh-113009 [https://github.com/python/cpython/issues/113009]: `multiprocessing`: No Windows, corrige uma condição de corrida em `Process.terminate()`: não define mais o atributo `returncode` para sempre chamar `WaitForSingleObject()` em `Process.wait()`. Anteriormente, às vezes o processo ainda estava em execução após `TerminateProcess()` mesmo se `GetExitCodeProcess()` não fosse `STILL_ACTIVE`. Patch por Victor Stinner.
- gh-86179 [https://github.com/python/cpython/issues/86179]: Corrige cálculos de caminho ao iniciar o Python no Windows por meio de um link simbólico.
- gh-71383 [https://github.com/python/cpython/issues/71383]: Atualiza Tcl/Tk no instalador do Windows para 8.6.13 com um patch para suprimir avisos incorretos de ThemeChanged.
- gh-111650 [https://github.com/python/cpython/issues/111650]: Garante que a variável do pré-processador `Py_GIL_DISABLED` seja definida em `pyconfig.h` para que módulos de extensão escritos em C possam usá-la.
- gh-112278 [https://github.com/python/cpython/issues/112278]: Reduz o custo de tempo para algumas funções em `platform` no Windows se o usuário

atual não tiver permissão para o WMI.

- gh-73427 [https://github.com/python/cpython/issues/73427]: Descontinua **`sys.enablelegacywindowsfsencoding()`**. Use **`PYTHONLEGACYWINDOWSFSENCODING`** em vez disso. Patch por Inada Naoki.
- gh-87868 [https://github.com/python/cpython/issues/87868]: Classifica e remove corretamente variáveis de ambiente duplicadas em **`_winapi.CreateProcess()`**.
- bpo-37308 [https://bugs.python.org/issue?@action=redirect&bpo=37308]: Corrige mojibake em **`mmap.mmap`** ao usar um argumento *tagname* não ASCII no Windows.

macOS

- gh-113666 [https://github.com/python/cpython/issues/113666]: Adiciona as seguintes constantes ao módulo **`stat`**: `UF_SETTABLE`, `UF_TRACKED`, `UF_DATAVAULT`, `SF_SUPPORTED`, `SF_SETTABLE`, `SF_SYNTHETIC`, `SF_RESTRICTED`, `SF_FIRMLINK` e `SF_DATALESS`. Os valores `UF_SETTABLE`, `SF_SUPPORTED`, `SF_SETTABLE` e `SF_SYNTHETIC` estão disponíveis apenas no macOS.
- gh-113536 [https://github.com/python/cpython/issues/113536]: **`os.waitid()`** agora está disponível no macOS
- gh-110459 [https://github.com/python/cpython/issues/110459]: Executar `configure ... --with-openssl-rpath=X/Y/Z` não falha mais em detectar OpenSSL no macOS.
- gh-74573 [https://github.com/python/cpython/issues/74573]: Documenta que **`dbm.ndbm`** pode corromper silenciosamente arquivos DBM em atualizações ao exceder limites de plataforma não documentados e pode travar (falha de segmentação) ao ler um arquivo corrompido. (FB8919203)
- gh-65701 [https://github.com/python/cpython/issues/65701]: A ferramenta **freeze** não funciona com construções de framework do Python. Documente isso e faça sair mais cedo ao executar a ferramenta com tal construção.
- gh-87277 [https://github.com/python/cpython/issues/87277]: webbrowser: Não procura navegadores X11 no macOS. Geralmente, eles não são usados e procurá-los pode resultar na inicialização do XQuartz, mesmo que ele não seja usado de outra forma.

- gh-111973 [https://github.com/python/cpython/issues/111973]: Atualiza o instalador do macOS para usar o SQLite 3.44.2.
- gh-108269 [https://github.com/python/cpython/issues/108269]: Define `CFBundleAllowMixedLocalizations` como true no Info.plist para o framework, Python.app incorporado e IDLE.app com instalações do framework no macOS. Isso permite que as aplicações selecionem o local preferido do usuário quando for diferente do inglês.
- gh-102362 [https://github.com/python/cpython/issues/102362]: Garante que o resultado de **`sysconfig.get_plaform()`** inclua pelo menos uma versão principal e secundária, mesmo que `MACOSX_DEPLOYMENT_TARGET` esteja definido como apenas uma versão principal durante a construção para corresponder ao formato esperado pelo pip.
- gh-110017 [https://github.com/python/cpython/issues/110017]: Desabilita um teste de estresse de tratamento de sinal no macOS devido a um bug no macOS (FB13453490).
- gh-110820 [https://github.com/python/cpython/issues/110820]: Garante que as definições do pré-processador para `ALIGNOF_MAX_ALIGN_T`, `SIZEOF_LONG_DOUBLE` e `HAVE_GCC_ASM_FOR_X64` estejam corretas para construções Universal 2 no macOS.
- gh-109981 [https://github.com/python/cpython/issues/109981]: Usa `/dev/fd` no macOS para determinar o número de arquivos abertos em `test.support.os_helper.fd_count` para evitar uma falha com descritores de arquivo “guarded” ao sondar arquivos abertos.

IDLE

- gh-72284 [https://github.com/python/cpython/issues/72284]: Melhora as listas de recursos, atalhos de teclado do editor e atalhos de teclado do shell no documento do IDLE.
- gh-113903 [https://github.com/python/cpython/issues/113903]: Corrige a falha rara de test.test_idle, em test_configdialog.
- gh-113729 [https://github.com/python/cpython/issues/113729]: Corrige o bug do menu “Help -> IDLE Doc” em 3.11.7 e 3.12.1.
- gh-113269 [https://github.com/python/cpython/issues/113269]: Corrige o travamento do test_editor no macOS Catalina.
- gh-112898 [https://github.com/python/cpython/issues/112898]: Corrige o processamento de arquivos não salvos ao sair do IDLE no macOS.

- bpo-13586 [https://bugs.python.org/issue?@action=redirect&bpo=13586]: Insere o texto selecionado ao abrir a caixa de diálogo “Replace”.

API C

- gh-106560 [https://github.com/python/cpython/issues/106560]: Corrige declarações redundantes na API C pública. Declare PyBool_Type, PyLong_Type e PySys_Audit() apenas uma vez. Patch por Victor Stinner.
- gh-112438 [https://github.com/python/cpython/issues/112438]: Corrige o suporte de unidades de formato “es”, “et”, “es#” e “et#” em tuplas aninhadas em funções do tipo `PyArg_ParseTuple()`.
- gh-111545 [https://github.com/python/cpython/issues/111545]: Adiciona a função `Py_HashPointer()` para fazer hash de um ponteiro. Patch por Victor Stinner.
- gh-65210 [https://github.com/python/cpython/issues/65210]: Altera a declaração do parâmetro *keywords* de `PyArg_ParseTupleAndKeywords()` e `PyArg_VaParseTupleAndKeywords()` para melhor compatibilidade com C++.

Python 3.13.0 alpha 2

Data de lançamento: 2023-11-22

Núcleo e embutidos

- gh-112243 [https://github.com/python/cpython/issues/112243]: Não inclua comentários nas expressões do depurador com f-string. Patch por Pablo Galindo

- gh-112287 [https://github.com/python/cpython/issues/112287]: Otimiza ligeiramente o interpretador nível 2 (uop) carregando apenas `oparg` e `operand` quando necessário. Também dobra o limite de tamanho de rastreamento novamente, para 512 desta vez.

- gh-112266 [https://github.com/python/cpython/issues/112266]: Altera as docstrings de `__dict__` e `__weakref__`.

- gh-111807 [https://github.com/python/cpython/issues/111807]: Reduz a profundidade máxima da pilha do analisador para 1000 em construções de depuração WASI.

- gh-111798 [https://github.com/python/cpython/issues/111798]: Quando o Python é construído no modo de depuração, define o limite de recursão C para 500 em vez de 1500. Uma construção de depuração provavelmente é construída com baixo nível de otimização, o que implica maior uso de memória de pilha do que uma construção de lançamento. Patch por Victor Stinner.

- gh-106529 [https://github.com/python/cpython/issues/106529]: Habilita a tradução de `FOR_ITER` não especializado para o Nível 2.

- gh-111916 [https://github.com/python/cpython/issues/111916]: Torna os módulos relacionados ao hashlib seguros para thread sem a GIL

- gh-81137 [https://github.com/python/cpython/issues/81137]: Descontinua a atribuição ao campo `__code__` de uma função quando o novo objeto de código for de um tipo incompatível (por exemplo, de um gerador para uma função simples).

- gh-79932 [https://github.com/python/cpython/issues/79932]: Levanta uma exceção se `frame.clear()` for chamado em um quadro suspenso.

- gh-81925 [https://github.com/python/cpython/issues/81925]: Implementa IDs de thread nativas para GNU KFreeBSD.

- gh-111843 [https://github.com/python/cpython/issues/111843]: Usa o backoff exponencial para reduzir o número de tentativas de otimização de nível 2 com falha em mais de 99%.

- gh-110829 [https://github.com/python/cpython/issues/110829]: Unir uma thread agora garante que a thread do sistema operacional subjacente tenha saído. Isso é necessário para fork() mais seguro em processos multithread.

- gh-109369 [https://github.com/python/cpython/issues/109369]: Certifica-se de que os rastreamentos de nível 2 sejam desotimizados se o código for instrumentado

- gh-111772 [https://github.com/python/cpython/issues/111772]: Especializa cargas e armazenamentos de slot para _Py_T_OBJECT, bem como Py_T_OBJECT_EX

- gh-111666 [https://github.com/python/cpython/issues/111666]: Acelera **`BaseExceptionGroup.derive()`**, **`BaseExceptionGroup.subgroup()`** e **`BaseExceptionGroup.split()`** alterando como eles analisam os argumentos passados.

- gh-111654 [https://github.com/python/cpython/issues/111654]: Corrige o travamento do tempo de execução quando algum erro acontece no opcode `LOAD_FROM_DICT_OR_DEREF`.

- gh-111623 [https://github.com/python/cpython/issues/111623]: Adiciona suporte para compartilhar tuplas entre interpretadores usando a API de interpretador cruzado. Patch por Anthony Shaw.

- gh-111354 [https://github.com/python/cpython/issues/111354]: O oparg de **`YIELD_VALUE`** agora é `1` se a instrução for parte de um yield-from ou await, e `0` caso contrário.

  O estado do quadro SUSPENDED agora é dividido em `SUSPENDED` e `SUSPENDED_YIELD_FROM`. Isso simplifica o código em `_PyGen_yf`.

- gh-111520 [https://github.com/python/cpython/issues/111520]: Mescla os interpretadores nível 1 (bytecode) e nível 2 (micro-ops), movendo o laço do interpretador nível 2 e alterne para `_PyEval_EvalFrameDefault()` em `Python/ceval.c`. O arquivo `Python/executor.c` desapareceu. Além disso, as macros `TIER_ONE` e `TIER_TWO` agora são manipuladas pelo gerador de código.

  Cuidado! Isso altera as variáveis de ambiente para habilitar micro-ops e sua depuração para `PYTHON_UOPS` e `PYTHON_LLTRACE`.

- gh-109181 [https://github.com/python/cpython/issues/109181]: Acelera a criação do objeto **`Traceback`** calculando preguiçosamente o número da linha. Patch por Pablo Galindo

- gh-111420 [https://github.com/python/cpython/issues/111420]: Permite comentários de tipo em instruções `with` entre parênteses

- gh-111438 [https://github.com/python/cpython/issues/111438]: Adicione suporte para compartilhar floats entre interpretadores usando a API de interpretador cruzado. Patch por Anthony Shaw.

- gh-111435 [https://github.com/python/cpython/issues/111435]: Adiciona suporte para compartilhar `True` e `False` entre interpretadores usando a API de interpretador cruzado. Patch por Anthony Shaw.

- gh-102388 [https://github.com/python/cpython/issues/102388]: Corrige um bug em que os codecs `iso2022_jp_3` e `iso2022_jp_2004` liam fora dos limites

- gh-111366 [https://github.com/python/cpython/issues/111366]: Corrige um problema no **`codeop`** que estava fazendo com que exceções **`SyntaxError`** geradas na presença de sintaxe inválida não contivessem mensagens de erro precisas. Patch por Pablo Galindo

- gh-111380 [https://github.com/python/cpython/issues/111380]: Corrige um bug que estava fazendo com que **`SyntaxWarning`** aparecesse duas vezes ao analisar se uma sintaxe inválida fosse encontrada posteriormente. Patch por Pablo Galindo

- gh-111374 [https://github.com/python/cpython/issues/111374]: Adicionada uma nova variável de ambiente **`PYTHON_FROZEN_MODULES`**. Determina se os módulos congelados são ou não ignorados pelo mecanismo de importação, equivalente à opção de linha de comando **`-X frozen_modules`**.

- gh-111354 [https://github.com/python/cpython/issues/111354]: Remove `oparg` de **`YIELD_VALUE`**. Altera `oparg` de **`RESUME`** para incluir informações sobre o except-depth. Essas alterações possibilitam simplificar o código no fechamento do gerador.

- gh-94438 [https://github.com/python/cpython/issues/94438]: Corrige uma regressão que impedia o salto entre `is None` e `is not None` durante a depuração. Patch por Savannah Ostrowski.

- gh-67224 [https://github.com/python/cpython/issues/67224]: Mostra linhas de origem em tracebacks ao usar a opção `-c` ao executar Python. Patch por Pablo Galindo

- gh-111123 [https://github.com/python/cpython/issues/111123]: Corrige um bug em que uma declaração **`global`** em um bloco **`except`** é rejeitada quando o global é usado no bloco **`else`**.

- gh-110938 [https://github.com/python/cpython/issues/110938]: Corrige mensagens de erro para blocos indentados com funções e classes com parâmetros de tipo genérico. Patch por Pablo Galindo

- gh-109214 [https://github.com/python/cpython/issues/109214]: Remove atualizações desnecessárias de ponteiros de instruções antes de retornar de quadros.

- gh-110912 [https://github.com/python/cpython/issues/110912]: Exibe corretamente o traceback para exceções **`MemoryError`** usando o módulo **`traceback`**. Patch por Pablo Galindo

- gh-109894 [https://github.com/python/cpython/issues/109894]: Corrige falha devido a **`MemoryError`** estático inicializado incorretamente no subinterpretador.

- gh-110892 [https://github.com/python/cpython/issues/110892]: Retorna `NULL` para eventos `PyTrace_RETURN` causados por uma exceção

- gh-110864 [https://github.com/python/cpython/issues/110864]: Corrige análise de argumento por `_PyArg_UnpackKeywordsWithVararg` para funções que definem parâmetros posicionais-ou-nomeados, argumentos variáveis e somente-nomeados.

- gh-109094 [https://github.com/python/cpython/issues/109094]: Substitui `prev_instr` no quadro do interpretador por `instr_ptr` que aponta para

o início da instrução que está sendo executada no momento (ou será executada quando o quadro for retomado).

- gh-110805 [https://github.com/python/cpython/issues/110805]: Permite que o repl mostre o código-fonte e tracebacks completos. Patch por Pablo Galindo

- gh-110722 [https://github.com/python/cpython/issues/110722]: Adiciona **`PYTHON_PRESITE=package.module`** para importar um módulo no início do ciclo de vida do interpretador antes de `site.py` ser executado. O Python precisa ser construído no modo de depuração para que esta opção exista.

- gh-110481 [https://github.com/python/cpython/issues/110481]: Implementa a contagem de referências tendenciosa em construções `--disable-gil`.

- gh-110543 [https://github.com/python/cpython/issues/110543]: Corrige regressão no Python 3.12 onde **`types.CodeType.replace()`** produziria um objeto de código quebrado se chamado em um objeto código de módulo ou classe que contém uma compreensão. Patch por Jelle Zijlstra.

- gh-89519 [https://github.com/python/cpython/issues/89519]: Removidos os descritores encadeados de **`classmethod`** (introduzidos em bpo-19072 [https://bugs.python.org/issue?@action=redirect&bpo=19072]). Isso não pode mais ser usado para encapsular outros descritores como **`property`**. O design principal desse recurso era falho e causava uma série de problemas posteriores. Para "passar" um **`classmethod`**, considere usar o atributo **`__wrapped__`** que foi adicionado no Python 3.10.

- gh-103615 [https://github.com/python/cpython/issues/103615]: Usa eventos locais para rastreamento de opcode

- bpo-46657 [https://bugs.python.org/issue?@action=redirect&bpo=46657]: Adiciona suporte ao alocador de memória mimalloc.

- gh-106718 [https://github.com/python/cpython/issues/106718]: Quando PyConfig.stdlib_dir é definido explicitamente, ele agora é respeitado e

não será substituído por PyConfig.home.

- gh-106905 [https://github.com/python/cpython/issues/106905]: Corrige o SystemError incorreto sobre a incompatibilidade de profundidade de recursão do construtor AST.

- gh-100445 [https://github.com/python/cpython/issues/100445]: Melhora a mensagem de erro para strings não terminadas com escapes.

- bpo-45759 [https://bugs.python.org/issue?@action=redirect&bpo=45759]: Melhoradas as mensagens de erro para instruções `elif`/`else` que não correspondem a nenhuma instrução válida. Patch por Jeremiah Vivian.

Biblioteca

- gh-111942 [https://github.com/python/cpython/issues/111942]: Corrige o SystemError no construtor TextIOWrapper com o argumento “errors” não codificável no modo não de depuração.

- gh-111995 [https://github.com/python/cpython/issues/111995]: Adicionada a constante `NI_IDN` ao módulo `socket` quando presente em C no momento da compilação para uso com `socket.getnameinfo()`.

- gh-109538 [https://github.com/python/cpython/issues/109538]: Emite mensagem de aviso em vez de exibir `RuntimeError` quando o laço de eventos já foi fechado em `StreamWriter.__del__()`.

- gh-111942 [https://github.com/python/cpython/issues/111942]: Corrige travamentos em `io.TextIOWrapper.reconfigure()` ao passar argumentos inválidos, por exemplo, codificação não string.

- gh-111460 [https://github.com/python/cpython/issues/111460]: `curses`: restaura o suporte a caracteres amplos (incluindo `curses.unget_wch()` e `get_wch()`) no macOS, que estava indisponível devido a uma regressão no Python 3.12.

- gh-103791 [https://github.com/python/cpython/issues/103791]: `contextlib.suppress` agora oferece suporte à supressão de exceções

levantadas como parte de um `BaseExceptionGroup`, além do suporte recente para `ExceptionGroup`.

- gh-111835 [https://github.com/python/cpython/issues/111835]: A classe `mmap.mmap` agora tem um método `seekable()` que pode ser usado quando um objeto arquivo ou similar pesquisável é necessário. O método `seek()` agora retorna a nova posição absoluta. Patch por Donghee Na.

- gh-111804 [https://github.com/python/cpython/issues/111804]: Remove posix.fallocate() em WASI, pois o posix_fallocate() subjacente não está disponível na visualização WASI2.

- gh-111841 [https://github.com/python/cpython/issues/111841]: Corrige argumentos truncados em um caractere nulo incorporado em `os.putenv()` e `os.unsetenv()` no Windows.

- gh-111768 [https://github.com/python/cpython/issues/111768]: `wsgiref.util.is_hop_by_hop()` agora é exposto corretamente em `__all__`.

- gh-80731 [https://github.com/python/cpython/issues/80731]: Evita executar a função padrão em `cmd.Cmd` em um bloco except

- gh-111541 [https://github.com/python/cpython/issues/111541]: Corrige `doctest` para subclasses não embutidas de `SyntaxError`.

- gh-111719 [https://github.com/python/cpython/issues/111719]: Adiciona validação de argumento extra para o comando `alias` em `pdb`

- gh-111482 [https://github.com/python/cpython/issues/111482]: `time`: Torna as funções `time.clock_gettime()` e `time.clock_gettime_ns()` até 2x mais rápidas por convenção de chamada mais rápida. Patch por Victor Stinner.

- gh-110894 [https://github.com/python/cpython/issues/110894]: Chama manipulador de exceção de laço para exceções em `client_connected_cb` de `asyncio.start_server()` para que as aplicações possam lidar com isso. Patch por Kumar Aditya.

- gh-111531 [https://github.com/python/cpython/issues/111531]: Corrige vazamentos de referência nos métodos `bind_class()` e `bind_all()` dos widgets **`tkinter`**.

- gh-111246 [https://github.com/python/cpython/issues/111246]: **`asyncio.loop.create_unix_server()`** agora removerá automaticamente o soquete Unix quando o servidor for fechado.

- gh-111356 [https://github.com/python/cpython/issues/111356]: Adicionados **`io.text_encoding()`**, **`io.DEFAULT_BUFFER_SIZE`** e **`io.IncrementalNewlineDecoder`** para `io.__all__`.

- gh-66425 [https://github.com/python/cpython/issues/66425]: Remove o código para definir o cabeçalho REMOTE_HOST do módulo wsgiref, pois ele é inacessível. Este cabeçalho é usado por motivos de desempenho, o que não é necessário no módulo wsgiref.

- gh-111429 [https://github.com/python/cpython/issues/111429]: Acelera **`pathlib.PurePath.relative_to()`** e **`is_relative_to()`**.

- gh-111342 [https://github.com/python/cpython/issues/111342]: Corrigido erro de digitação em **`math.sumprod()`**.

- gh-68166 [https://github.com/python/cpython/issues/68166]: Remove menção de tipo de elemento “vsapi” não suportado em **`tkinter.ttk.Style.element_create()`**. Adiciona testes para `element_create()` e outros métodos `ttk.Style`. Adiciona exemplos para `element_create()` na documentação.

- gh-111388 [https://github.com/python/cpython/issues/111388]: Adiciona parâmetro `show_group` para **`traceback.format_exception_only()`**, que permite formatar instâncias **`ExceptionGroup`**.

- gh-79033 [https://github.com/python/cpython/issues/79033]: Outra tentativa de corrigir **`asyncio.Server.wait_closed()`**. Agora ela bloqueia até que ambas as condições sejam verdadeiras: o servidor é fechado, *e* não há mais conexões ativas. (Isso significa que em alguns casos onde na versão 3.12.0 essa função retornaria *incorretamente* imediatamente,

agora ela bloqueará; em particular, quando não há conexões ativas, mas o servidor ainda não foi fechado.)

- gh-111259 [https://github.com/python/cpython/issues/111259]: Otimiza curingas recursivos em `pathlib`.

- gh-111295 [https://github.com/python/cpython/issues/111295]: Corrige `time` não verificando erros ao inicializar.

- gh-111253 [https://github.com/python/cpython/issues/111253]: Adiciona verificação de erros durante a inicialização do módulo `_socket`.

- gh-111251 [https://github.com/python/cpython/issues/111251]: Corrige `_blake2` não verificando erros ao inicializar.

- gh-111233 [https://github.com/python/cpython/issues/111233]: Corrige `select` não verificando erros ao inicializar.

- gh-111230 [https://github.com/python/cpython/issues/111230]: Corrige `ssl` não verificando erros ao inicializar.

- gh-111174 [https://github.com/python/cpython/issues/111174]: Corrige travamento em `io.BytesIO.getbuffer()` chamado repetidamente para BytesIO vazio.

- gh-111187 [https://github.com/python/cpython/issues/111187]: Adia versão de remoção para locale.getdefaultlocale() para Python 3.15.

- gh-111159 [https://github.com/python/cpython/issues/111159]: Corrige comparação de saída `doctest` para exceções com notas.

- gh-110910 [https://github.com/python/cpython/issues/110910]: Corrige manipulação de estado inválido em `asyncio.TaskGroup` e `asyncio.Timeout`. Eles agora geram RuntimeError adequado se forem usados incorretamente e são deixados em estado consistente depois disso.

- gh-111092 [https://github.com/python/cpython/issues/111092]: Faz turtledemo ser executado sem root padrão habilitado.

- gh-110944 [https://github.com/python/cpython/issues/110944]: Adiciona suporte a apelido e variáveis de conveniência para completamento automático de **`pdb`**

- gh-110745 [https://github.com/python/cpython/issues/110745]: Adicionado o parâmetro *newline* para **`pathlib.Path.read_text()`**. Patch por Junya Okabe.

- gh-84583 [https://github.com/python/cpython/issues/84583]: Faz **`pdb`** entrar no modo post-mortem mesmo para **`SyntaxError`**

- gh-80675 [https://github.com/python/cpython/issues/80675]: Define `f_trace_lines = True` em todos os quadros em **`pdb.set_trace()`**

- gh-110771 [https://github.com/python/cpython/issues/110771]: Expõe as partes de configuração e limpeza de `asyncio.run_forever()` como os métodos autônomos `asyncio.run_forever_setup()` e `asyncio.run_forever_cleanup()`. Isso permite uma integração mais estreita com laços de eventos da GUI.

- gh-110774 [https://github.com/python/cpython/issues/110774]: Suporte para definir o kwarg loop_factory **`asyncio.Runner`** em **`unittest.IsolatedAsyncioTestCase`**

- gh-110392 [https://github.com/python/cpython/issues/110392]: Corrige **`tty.setraw()`** e **`tty.setcbreak()`**: anteriormente eles retornavam uma lista parcialmente modificada dos atributos tty originais. **`tty.cfmakeraw()`** e **`tty.cfmakecbreak()`** agora fazem uma cópia da lista de caracteres especiais antes de modificá-la.

- gh-59013 [https://github.com/python/cpython/issues/59013]: Torna o número de linhas do ponto de interrupção da função mais preciso em **`pdb`**

- gh-88434 [https://github.com/python/cpython/issues/88434]: Emite aviso de descontinuação para números não inteiros em funções e métodos **`gettext`** que consideram formas plurais, mesmo que a tradução não tenha sido encontrada.

- gh-110395 [https://github.com/python/cpython/issues/110395]: Garante que os objetos **`select.kqueue()`** apareçam corretamente como fechados em filhos bifurcados, para evitar operações em um descritor de arquivo inválido.

- gh-110196 [https://github.com/python/cpython/issues/110196]: Adiciona o método `__reduce__` a **`IPv6Address`** para manter `scope_id`

- gh-109747 [https://github.com/python/cpython/issues/109747]: Melhora os erros para padrões *look-behind* não suportados. Agora re.error é levantada em vez de OverflowError ou RuntimeError para largura muito grande do padrão *look-behind*.

- gh-109466 [https://github.com/python/cpython/issues/109466]: Adiciona a propriedade **`ipaddress.IPv4Address.ipv6_mapped`**, que retorna o endereço IPv6 mapeado para IPv4.

- gh-85098 [https://github.com/python/cpython/issues/85098]: Implementa a CLI do módulo **`symtable`** e melhorar o repr de **`Symbol`**.

- gh-108791 [https://github.com/python/cpython/issues/108791]: Tratamento de erros aprimorado na interface de linha de comando **`pdb`**, fazendo com que ela produza mensagens de erro mais concisas.

- gh-105931 [https://github.com/python/cpython/issues/105931]: Altera **`compileall`** para remover apenas o prefixo stripdir do caminho completo registrado no arquivo `.pyc` compilado, quando o prefixo corresponder ao início do caminho completo em sua totalidade. Quando o prefixo não corresponder, nenhuma remoção será realizada e um aviso para esse efeito será exibido.

  Anteriormente, todos os componentes de caminho do prefixo stripdir que correspondiam ao caminho completo eram removidos, enquanto aqueles que não correspondiam eram deixados sozinhos (incluindo aqueles intercalados entre componentes correspondentes).

- gh-107431 [https://github.com/python/cpython/issues/107431]: Cria os tipos `DictProxy` e `ListProxy` em tipos Generic Alias de

`multiprocessing.managers` para uso `[]` em contextos de tipagem.

- gh-72904 [https://github.com/python/cpython/issues/72904]: Adiciona `glob.translate()`. Esta função converte um nome de caminho com curingas de estilo shell em uma expressão regular.

- gh-90026 [https://github.com/python/cpython/issues/90026]: Define `USE_XATTRS` no Cygwin para que as funções relacionadas a XATTR no módulo `os` fiquem disponíveis.

- gh-90890 [https://github.com/python/cpython/issues/90890]: Novos métodos `mailbox.Maildir.get_info()`, `mailbox.Maildir.set_info()`, `mailbox.Maildir.get_flags()`, `mailbox.Maildir.set_flags()`, `mailbox.Maildir.add_flag()`, `mailbox.Maildir.remove_flag()`. Esses métodos aceleram o acesso às informações e/ou sinalizadores de uma mensagem e são úteis quando não é necessário acessar o conteúdo da mensagem, como ao iterar sobre um Maildir para encontrar mensagens com sinalizadores específicos.

- gh-102956 [https://github.com/python/cpython/issues/102956]: Corrige o retorno de strings de bytes vazias após uma busca no módulo zipfile

- gh-102895 [https://github.com/python/cpython/issues/102895]: Adicionado um parâmetro `local_exit` para `code.interact()` para evitar que `exit()` e `quit` fechem `sys.stdin` e levantem `SystemExit`.

- gh-97928 [https://github.com/python/cpython/issues/97928]: Altera o comportamento de `tkinter.Text.count()`. Agora ele sempre retorna um inteiro se uma ou menos opções de contagem forem especificadas. Anteriormente, ele podia retornar uma única contagem como uma tupla de 1 elemento, um inteiro (somente se a opção `"update"` fosse especificada) ou `None` se nenhum item fosse encontrado. O resultado agora é o mesmo se `wantobjects` for definido como `0`.

- gh-96954 [https://github.com/python/cpython/issues/96954]: Troca o armazenamento dos nomes de pontos de código unicode para usar uma estrutura de dados diferente, um grafo de palavras acíclicas direcionadas

[https://pt.wikipedia.org/wiki/Aut%C3%B4mato_finito_determinístico_acíclico]. Isso torna a biblioteca compartilhada unicodedata cerca de 440 KiB menor. Contribuição de Carl Friedrich Bolz-Tereick usando código do projeto PyPy.

- gh-73561 [https://github.com/python/cpython/issues/73561]: Omite o escopo da interface de um endereço IPv6 quando usado como cabeçalho de host por **`http.client`**.

- gh-86826 [https://github.com/python/cpython/issues/86826]: **`zipinfo`** agora oferece suporte ao intervalo completo de valores na string TZ determinado pelo RFC 8536 e detecta todos os formatos inválidos. Ambas as implementações Python e C agora levantam exceções do mesmo tipo em dados inválidos.

Testes

- gh-111808 [https://github.com/python/cpython/issues/111808]: Faz o valor padrão de `test.support.infinite_recursion()` ser condicional com base em se otimizações foram usadas ao compilar o interpretador. Isso ajuda com plataformas como WASI cujo tamanho de pilha é muito restrito em construções de depuração.
- gh-110722 [https://github.com/python/cpython/issues/110722]: A coleta de cobertura de linha de bibliotecas padrão dentro do conjunto de testes de regressão agora é precisa, bem como muito mais rápida. Patch por Łukasz Langa.
- gh-110367 [https://github.com/python/cpython/issues/110367]: Torna a opção regrtest `--verbose3` compatível com as opções `--huntrleaks -jN`. O comando `./python -m test -j1 -R 3:3 --verbose3` agora funciona como esperado. Patch por Victor Stinner.
- gh-111165 [https://github.com/python/cpython/issues/111165]: Remove as funções não mais utilizadas `run_unittest()` e `run_doctest()` do módulo **`test.support`**.
- gh-110932 [https://github.com/python/cpython/issues/110932]: Corrige regrtest se a variável de ambiente `SOURCE_DATE_EPOCH` estiver definida: usar o valor da variável como a semente aleatória. Patch por Victor Stinner.

- gh-110995 [https://github.com/python/cpython/issues/110995]: test_gdb: Corrige a detecção de gdb construído sem suporte a scripts Python. Patch por Victor Stinner.
- gh-110918 [https://github.com/python/cpython/issues/110918]: Os padrões de correspondência de casos de teste especificados pelas opções `--match`, `--ignore`, `--matchfile` e `--ignorefile` agora são testados na ordem de especificação, e a última correspondência determina se o caso de teste deve ser executado ou ignorado.
- gh-108747 [https://github.com/python/cpython/issues/108747]: Adiciona o teste de unidade para os ganchos `usercustomize` e `sitecustomize` de **`site`**.

Construção

- gh-96954 [https://github.com/python/cpython/issues/96954]: Faz `make regen-unicodedata` funcionar para construções fora da árvore do CPython.
- gh-112088 [https://github.com/python/cpython/issues/112088]: Adiciona o script `Tools/build/regen-configure.sh` para regenerar o `configure` com uma imagem de contêiner do Ubuntu. A imagem do contêiner `quay.io/tiran/cpython_autoconf:271` (tiran/cpython_autoconf [https://github.com/tiran/cpython_autoconf]) não é mais usada. Patch por Victor Stinner.
- gh-111046 [https://github.com/python/cpython/issues/111046]: Para wasi-threads, a memória agora é exportada para corrigir problemas de compatibilidade com alguns tempos de execução wasm.
- gh-110828 [https://github.com/python/cpython/issues/110828]: AIX 32 bits precisa de `-latomic` para construir o módulo de extensão **`_testcapi`**.
- gh-85283 [https://github.com/python/cpython/issues/85283]: As extensões C `errno`, `md5`, `resource`, `winsound`, `_ctypes_test`, `_multiprocessing.posixshmem`, `_scproxy`, `_stat`, `_testimportmultiple` e `_uuid` agora são criadas com a API C limitada. Patch por Victor Stinner.

Windows

- gh-111856 [https://github.com/python/cpython/issues/111856]: Corrige **`fstat()`** em sistemas de arquivos que não oferecem suporte a solicitações de ID de arquivo. Isso inclui FAT32 e exFAT.

- gh-111293 [https://github.com/python/cpython/issues/111293]: Corrige **`os.DirEntry.inode`** eliminando 64 bits mais altos de uma ID de arquivo em alguns sistemas de arquivos no Windows.
- gh-110913 [https://github.com/python/cpython/issues/110913]: O WindowsConsoleIO agora divide corretamente buffers grandes sem dividir sequências UTF-8.

macOS

- gh-59703 [https://github.com/python/cpython/issues/59703]: Para construções de framework macOS, em `getpath.c` use a função `dladdr` do sistema para encontrar o caminho para a biblioteca compartilhada em vez de depender de APIs macOS descontinuadas.
- gh-110950 [https://github.com/python/cpython/issues/110950]: Atualiza o instalador macOS para incluir uma correção Tcl/Tk upstream para o aviso `Secure coding is not enabled for restorable state!` encontrado no Tkinter no macOS 14 Sonoma.
- gh-111015 [https://github.com/python/cpython/issues/111015]: Garante que IDLE.app e Python Launcher.app sejam instalados com as permissões apropriadas em construções macOS.
- gh-71383 [https://github.com/python/cpython/issues/71383]: Atualiza o instalador do macOS para incluir uma correção Tcl/Tk upstream para o erro `ttk::ThemeChanged` encontrado no Tkinter.
- gh-92603 [https://github.com/python/cpython/issues/92603]: Atualiza o instalador do macOS para incluir uma correção aceita pelo Tcl/Tk upstream para uma falha encontrada após a destruição da primeira instância **`tkinter.Tk()`**.

IDLE

- bpo-35668 [https://bugs.python.org/issue?@action=redirect&bpo=35668]: Adiciona docstrings ao módulo depurador IDLE. Corrigir dois bugs: inicializar `Idb.botframe` (deve estar em Bdb); em `Idb.in_rpc_code`, verificar se `prev_frame` é `None` antes de tentar usá-lo. Expandir bastante o test_debugger.

Ferramentas/Demos

- gh-111903 [https://github.com/python/cpython/issues/111903]: Clínica de Argumentos agora oferece suporte à diretiva `@critical_section` que instrui a Clínica de Argumentos a gerar uma seção crítica em torno da chamada de função, que trava o objeto `self` em construções `--disable-gil`. Patch por Sam Gross.

API C

- gh-112026 [https://github.com/python/cpython/issues/112026]: Adiciona novamente a função privada `_PyThreadState_UncheckedGet()` como um apelido para a nova função pública **`PyThreadState_GetUnchecked()`**. Patch por Victor Stinner.

- gh-112026 [https://github.com/python/cpython/issues/112026]: Restaure a função `_PyDict_GetItemStringWithError()` removida. Ela é usada pelo numpy. Patch por Victor Stinner.

- gh-112026 [https://github.com/python/cpython/issues/112026]: Restaure as funções privadas, macros e estruturas da API C removidas que não têm substituição simples por enquanto:

  - _PyDict_GetItem_KnownHash()
  - _PyDict_NewPresized()
  - _PyHASH_BITS
  - _PyHASH_IMAG
  - _PyHASH_INF
  - _PyHASH_MODULUS
  - _PyHASH_MULTIPLIER
  - _PyLong_Copy()
  - _PyLong_FromDigits()
  - _PyLong_New()
  - _PyLong_Sign()
  - _PyObject_CallMethodId()
  - _PyObject_CallMethodNoArgs()
  - _PyObject_CallMethodOneArg()
  - _PyObject_CallOneArg()
  - _PyObject_EXTRA_INIT

    - _PyObject_FastCallDict()
    - _PyObject_GetAttrId()
    - _PyObject_Vectorcall()
    - _PyObject_VectorcallMethod()
    - _PyStack_AsDict()
    - _PyThread_CurrentFrames()
    - _PyUnicodeWriter structure
    - _PyUnicodeWriter_Dealloc()
    - _PyUnicodeWriter_Finish()
    - _PyUnicodeWriter_Init()
    - _PyUnicodeWriter_Prepare()
    - _PyUnicodeWriter_PrepareKind()
    - _PyUnicodeWriter_WriteASCIIString()
    - _PyUnicodeWriter_WriteChar()
    - _PyUnicodeWriter_WriteLatin1String()
    - _PyUnicodeWriter_WriteStr()
    - _PyUnicodeWriter_WriteSubstring()
    - _PyUnicode_AsString()
    - _PyUnicode_FromId()
    - _PyVectorcall_Function()
    - _Py_IDENTIFIER()
    - _Py_c_abs()
    - _Py_c_diff()
    - _Py_c_neg()
    - _Py_c_pow()
    - _Py_c_prod()
    - _Py_c_quot()
    - _Py_c_sum()
    - _Py_static_string()
    - _Py_static_string_init()

  Patch por Victor Stinner.

- gh-112026 [https://github.com/python/cpython/issues/112026]: Adiciona novamente os includes `<ctype.h>` e `<unistd.h>` em `Python.h`, mas não os inclui na versão limitada da API C 3.13 e mais recentes. Patch por Victor Stinner.

- gh-111956 [https://github.com/python/cpython/issues/111956]: Adiciona API de inicialização única interna: `_PyOnceFlag` e `_PyOnceFlag_CallOnce`.

- gh-111262 [https://github.com/python/cpython/issues/111262]: Adiciona funções **`PyDict_Pop()`** e **`PyDict_PopString()`**: remover uma chave de um dicionário e, opcionalmente, retornar o valor removido. Isso é semelhante a **`dict.pop()`**, mas sem o valor padrão e não levantando **`KeyError`** se a chave estiver faltando. Patch por Stefan Behnel e Victor Stinner.

- gh-111863 [https://github.com/python/cpython/issues/111863]: Renomeia `Py_NOGIL` para `Py_GIL_DISABLED`. Patch por Hugo van Kemenade.

- gh-111138 [https://github.com/python/cpython/issues/111138]: Adiciona as funções **`PyList_Extend()`** e **`PyList_Clear()`**: semelhantes aos métodos Python `list.extend()` e `list.clear()`. Patch por Victor Stinner.

- gh-108765 [https://github.com/python/cpython/issues/108765]: No Windows, `Python.h` não inclui mais o arquivo de cabeçalho padrão `<stddef.h>`. Se necessário, ele deve ser incluído explicitamente. Patch por Victor Stinner.

- gh-111569 [https://github.com/python/cpython/issues/111569]: Implementa "Python Critical Sections" de **PEP 703** [https://peps.python.org/pep-0703/]. Essas são macros para ajudar a substituir a GIL por travas por objeto na construção com `--disable-gil` do CPython. As macros são no-ops na construção padrão.

- gh-111506 [https://github.com/python/cpython/issues/111506]: Na versão limitada 3.13 da API C, a função **`Py_SET_REFCNT()`** agora é implementada como uma chamada de função opaca. Patch por Victor Stinner.

- gh-108082 [https://github.com/python/cpython/issues/108082]: Adiciona a função **`PyErr_FormatUnraisable()`**.

- gh-110964 [https://github.com/python/cpython/issues/110964]: Move as funções privadas não documentadas _PyArg e a estrutura _PyArg_Parser para a API C interna (`pycore_modsupport.h`). Patch por Victor Stinner.

- gh-110815 [https://github.com/python/cpython/issues/110815]: Suporte a nomes de palavras reservadas não ASCII em `PyArg_ParseTupleAndKeywords()`.

- gh-109587 [https://github.com/python/cpython/issues/109587]: Introduzida `PyUnstable_PerfTrampoline_CompileCode()`, `PyUnstable_PerfTrampoline_SetPersistAfterFork()` e `PyUnstable_CopyPerfMapFile()`. Essas funções permitem que módulos de extensão inicializem trampolins avidamente, depois que o aplicativo é "aquecido". Isso torna possível ter perf-trampolines em execução de forma sempre habilitada.

- gh-85283 [https://github.com/python/cpython/issues/85283]: Adiciona a função `PySys_Audit()` à API C limitada. Patch por Victor Stinner.

- gh-85283 [https://github.com/python/cpython/issues/85283]: Adiciona `PyMem_RawMalloc()`, `PyMem_RawCalloc()`, `PyMem_RawRealloc()` e `PyMem_RawFree()` à API C limitada. Patch por Victor Stinner.

- gh-106672 [https://github.com/python/cpython/issues/106672]: Funções `PyDict_GetItem()`, `PyDict_GetItemString()`, `PyMapping_HasKey()`, `PyMapping_HasKeyString()`, `PyObject_HasAttr()`, `PyObject_HasAttrString()` e `PySys_GetObject()`, que limpam todos os erros ocorridos durante a chamada da função, agora os relatam usando `sys.unraisablehook()`.

- gh-67565 [https://github.com/python/cpython/issues/67565]: Remove a verificação redundante de contiguidade C em `getargs.c`, `binascii`, `ssl` e Clínica de Argumentos. Remendado por Stefan Krah e Furkan Onder

Python 3.13.0 alpha 1

Data de lançamento: 2023-10-13

Segurança

- gh-108310 [https://github.com/python/cpython/issues/108310]: Corrigido um problema em que instâncias de `ssl.SSLSocket` eram vulneráveis a um

desvio do handshake TLS e incluíam proteções (como verificação de certificado) e tratavam dados não criptografados enviados como se fossem dados criptografados TLS pós-handshake. Problema de segurança relatado como **CVE 2023-40217** [https://www.cve.org/CVERecord?id=CVE-2023-40217] por Aapo Oksman. Patch por Gregory P. Smith.

- gh-107774 [https://github.com/python/cpython/issues/107774]: PEP 669 especifica que `sys.monitoring.register_callback` gerará um evento de auditoria. Pré-lançamentos do Python 3.12 não geravam o evento de auditoria. Isso agora foi corrigido.
- gh-102988 [https://github.com/python/cpython/issues/102988]: Revertida a alteração de melhoria de segurança **`email.utils`** lançada na 3.12beta4 que involuntariamente fez com que **`email.utils.getaddresses`** falhasse ao analisar endereços de e-mail com uma vírgula no campo de nome entre aspas. Veja gh-106669 [https://github.com/python/cpython/issues/106669].
- gh-99108 [https://github.com/python/cpython/issues/99108]: Atualiza nosso novo código HACL* embutido **`hashlib`** do upstream. O SHA2 embutido deve ser mais rápido e um problema com SHA3 em plataformas de 32 bits foi corrigido.
- gh-102509 [https://github.com/python/cpython/issues/102509]: Começa a inicializar `ob_digit` durante a criação de objetos **`PyLongObject`**. Patch por Illia Volochii.

Núcleo e embutidos

- gh-110782 [https://github.com/python/cpython/issues/110782]: Corrige o travamento quando **`typing.TypeVar`** é construído com um argumento nomeado. Patch por Jelle Zijlstra.

- gh-110752 [https://github.com/python/cpython/issues/110752]: Redefine `ceval.eval_breaker` em **`interpreter_clear()`**

- gh-110721 [https://github.com/python/cpython/issues/110721]: Usa a implementação de **`traceback`** para a funcionalidade padrão **`PyErr_Display()`**. Patch por Pablo Galindo

- gh-110696 [https://github.com/python/cpython/issues/110696]: Corrige mensagem de erro incorreta para descompactação de argumento inválido. Patch por Pablo Galindo

- gh-104169 [https://github.com/python/cpython/issues/104169]: Divide o tokenizador em dois diretórios separados: - Uma parte inclui a lógica de produção do lexema real e fica em `Parser/lexer`. - A segunda parte encapsula o lexer de acordo com os diferentes modos de tokenização que temos (string, utf-8, file, interactive, readline) e fica em `Parser/tokenizer`.

- gh-110688 [https://github.com/python/cpython/issues/110688]: Remove o método não documentado `test_c_api` de **`set`**, que foi definido apenas para fins de teste em `Py_DEBUG`. Agora temos testes CAPI adequados.

- gh-104584 [https://github.com/python/cpython/issues/104584]: Corrige um vazamento de referência ao executar com **`PYTHONUOPS`** ou **`-X uops 1`** habilitado.

- gh-110514 [https://github.com/python/cpython/issues/110514]: Adiciona `PY_THROW` para eventos **`sys.setprofile()`**

- gh-110489 [https://github.com/python/cpython/issues/110489]: Otimiza **`math.ceil()`** quando a entrada for exatamente um float, resultando em uma melhoria de cerca de 10%.

- gh-110455 [https://github.com/python/cpython/issues/110455]: Guard `assert(tstate->thread_id > 0)` with `#ifndef HAVE_PTHREAD_STUBS`. This allows for pydebug builds to work under WASI which (currently) lacks thread support.

- gh-110309 [https://github.com/python/cpython/issues/110309]: Remove nós constantes vazios desnecessários no final das especificações da f-strings.

- gh-110259 [https://github.com/python/cpython/issues/110259]: Identifica corretamente a especificação de formato em f-strings (com aspas

simples ou triplas) que tenham várias linhas na parte da expressão e incluam uma especificação de formatação. Patch por Pablo Galindo

- gh-110237 [https://github.com/python/cpython/issues/110237]: Corrige verificações de erro ausentes para chamadas para `PyList_Append` em `_PyEval_MatchClass`.

- gh-110164 [https://github.com/python/cpython/issues/110164]: regrtest: Se a variável de ambiente `SOURCE_DATE_EPOCH` for definida, o regrtest agora desabilita a aleatorização de testes. Patch por Victor Stinner.

- gh-109889 [https://github.com/python/cpython/issues/109889]: Corrige o algoritmo de detecção de NOP redundante do compilador para ignorar NOPs sem número de linha ao procurar o número de linha da próxima instrução.

- gh-109853 [https://github.com/python/cpython/issues/109853]: `sys.path[0]` agora está definido corretamente para subinterpretadores.

- gh-109923 [https://github.com/python/cpython/issues/109923]: Define o número da linha no `POP_TOP` que segue um `RETURN_GENERATOR`.

- gh-105716 [https://github.com/python/cpython/issues/105716]: Subinterpretadores agora lidam corretamente com o caso em que eles têm threads em execução em segundo plano. Antes, tais threads interferiam na limpeza e destruição delas, assim como impediam a execução de outro script.

- gh-109369 [https://github.com/python/cpython/issues/109369]: O eval_breaker interno e os sinalizadores de suporte, além da versão de monitoramento, foram mesclados em um único inteiro atômico para acelerar as verificações.

- gh-109823 [https://github.com/python/cpython/issues/109823]: Corrige bug em que o compilador não ajusta rótulos ao remover um bloco básico vazio que é um alvo de salto.

- gh-109793 [https://github.com/python/cpython/issues/109793]: O thread principal não sai mais prematuramente quando um subinterpretador é limpo durante a finalização do tempo de execução. O bug era um problema

particularmente porque, quando acionado, o processo Python sempre retornava com um exitcode 0, mesmo se falhasse.

- gh-109719 [https://github.com/python/cpython/issues/109719]: Corrige rótulos de alvos de salto ausentes quando o compilador reordena blocos frios/quentes.

- gh-109595 [https://github.com/python/cpython/issues/109595]: Adiciona a opção de linha de comando `-X cpu count` para substituir os resultados de retorno de `os.cpu_count()` e `os.process_cpu_count()`. Esta opção é útil para usuários que precisam limitar os recursos da CPU de um sistema de contêiner sem ter que modificar o contêiner (código da aplicação). Patch por Donghee Na.

- gh-109627 [https://github.com/python/cpython/issues/109627]: Corrige bug em que o compilador não atribui um novo rótulo de alvo de salto a um pequeno bloco de saída duplicado.

- gh-109596 [https://github.com/python/cpython/issues/109596]: Corrige alguns tokens na gramática que estavam marcados incorretamente como palavras-chave suaves. Corrigir também alguns nomes de regras repetidos e garantir que regras repetidas não sejam permitidas. Patch por Pablo Galindo.

- gh-109496 [https://github.com/python/cpython/issues/109496]: Em um Python construído em modo de depuração, `Py_DECREF()` agora chama `_Py_NegativeRefcount()` se o objeto for um ponteiro pendente para memória desalocada: memória preenchida com `0xDD` “byte morto” pelo gancho de depuração nos alocadores de memória. A correção consiste em verificar a contagem de referências *antes* de verificar `_Py_IsImmortal()`. Patch por Victor Stinner.

- gh-107265 [https://github.com/python/cpython/issues/107265]: Faz deopt de opcodes ocultos pelo executor quando o opcode base é necessário

- gh-109371 [https://github.com/python/cpython/issues/109371]: Feito deopt corretamente nas instruções para inicialização de ferramenta e

modificada a asserção incorreta na instrumentação, quando uma ferramenta anterior já define eventos INSTRUCTION

- gh-105658 [https://github.com/python/cpython/issues/105658]: Corrige bug em que o rastreamento de linha de um bloco except terminando com uma condicional inclui um evento em excesso com a linha da expressão condicional.

- gh-109219 [https://github.com/python/cpython/issues/109219]: Corrige escopos de parâmetros de tipo de compilação que usam um nome que também é livre em um escopo interno.

- gh-109351 [https://github.com/python/cpython/issues/109351]: Corrige falha ao compilar um AST inválido envolvendo uma expressão nomeada (morsa).

- gh-109341 [https://github.com/python/cpython/issues/109341]: Corrige falha ao compilar um AST inválido envolvendo um **`ast.TypeAlias`**.

- gh-109195 [https://github.com/python/cpython/issues/109195]: Corrige o local de origem para a instrução `LOAD_*` que precede um `LOAD_SUPER_ATTR` para carregar a variável global `super` (ou variável de sombreamento) para que ela abranja apenas o nome `super` e não os parênteses seguintes.

- gh-109256 [https://github.com/python/cpython/issues/109256]: Os IDs de opcode para opcodes especializados são alocados em seu próprio intervalo para melhorar a estabilidade dos IDs para os opcodes ‘reais’.

- gh-109216 [https://github.com/python/cpython/issues/109216]: Corrige possível vazamento de memória em **`BUILD_MAP`**.

- gh-109207 [https://github.com/python/cpython/issues/109207]: Corrige um SystemError em `__repr__` do objeto de entrada da tabela de símbolos.

- gh-109179 [https://github.com/python/cpython/issues/109179]: Corrige bug em que a exibição do traceback C descarta notas de **`SyntaxError`**.

- gh-109118 [https://github.com/python/cpython/issues/109118]: Não permitir escopos aninhados (lambdas, expressões geradoras e compreensões) dentro de escopos de anotação PEP 695 que estejam aninhados dentro de classes.

- gh-109156 [https://github.com/python/cpython/issues/109156]: Adiciona testes para instruções de desinstrumentação, mantendo a instrumentação para linhas

- gh-109114 [https://github.com/python/cpython/issues/109114]: Relaxa a detecção da mensagem de erro para lambdas inválidas dentro de f-strings para não procurar campos de substituição arbitrários e evitar falsos positivos. Patch por Pablo Galindo

- gh-105848 [https://github.com/python/cpython/issues/105848]: Adiciona um novo opcode `CALL_KW`, usado para chamadas que contêm argumentos nomeados. Além disso, corrige uma possível falha ao pular chamadas de método em um depurador.

- gh-109052 [https://github.com/python/cpython/issues/109052]: Use o opcode base ao comparar objetos de código para evitar interferência da instrumentação

- gh-109118 [https://github.com/python/cpython/issues/109118]: Corrige falha do interpretador quando uma exceção NameError é levantada dentro dos parâmetros de tipo de uma classe genérica.

- gh-88943 [https://github.com/python/cpython/issues/88943]: Melhora o erro de sintaxe para caracteres não ASCII que seguem um literal numérico. Agora, ele aponta para o caractere não ASCII inválido, não para o literal numérico válido.

- gh-108976 [https://github.com/python/cpython/issues/108976]: Corrige falha que ocorre após desinstrumentar um objeto código em uma função de retorno de monitoramento.

- gh-108732 [https://github.com/python/cpython/issues/108732]: Torna as variáveis de iteração de compreensões de escopo de módulo e classe visíveis

para pdb e outras ferramentas que usam `frame.f_locals` novamente.

- gh-108959 [https://github.com/python/cpython/issues/108959]: Corrigir o posicionamento do cursor para locais de erro em operações binárias e de subscrito que envolvem parênteses e espaços não semânticos. Patch por Pablo Galindo.

- gh-104584 [https://github.com/python/cpython/issues/104584]: Corrige uma falha ao executar com **`PYTHONUOPS`** ou **`-X uops`** habilitado e ocorre um erro durante a otimização.

- gh-108727 [https://github.com/python/cpython/issues/108727]: Define `tp_dealloc` para `CounterOptimizer_Type`. Isso corrige uma falha de segmentação na desalocação.

- gh-108520 [https://github.com/python/cpython/issues/108520]: Corrige **`multiprocessing.synchronize.SemLock.__setstate__()`** para inicializar corretamente **`multiprocessing.synchronize.SemLock._is_fork_ctx`**. Isso corrige uma regressão ao passar um SemLock entre processos aninhados.

  Renomeia **`multiprocessing.synchronize.SemLock.is_fork_ctx`** para **`multiprocessing.synchronize.SemLock._is_fork_ctx`** para evitar expô-lo como API pública.

- gh-108654 [https://github.com/python/cpython/issues/108654]: Restaura os locais sombreados por uma compreensão embutida se a compreensão levantar uma exceção.

- gh-108488 [https://github.com/python/cpython/issues/108488]: Altera a inicialização das entradas de cache em linha para que a entrada de cache para `JUMP_BACKWARD` seja inicializada com zero, em vez do valor `adaptive_counter_warmup()` usado para todas as outras instruções. Este contador, único entre as instruções, conta a partir de zero.

- gh-108716 [https://github.com/python/cpython/issues/108716]: Desativa o congelamento profundo de objetos código. Os módulos permanecem

congelados, de modo que não é necessária uma pesquisa no sistema de arquivos para módulos comuns.

- gh-108614 [https://github.com/python/cpython/issues/108614]: Adiciona a instrução RESUME_CHECK para evitar ter que manipular instrumentação, sinais e alternância de contextos no mecanismo de execução de nível 2.

- gh-108487 [https://github.com/python/cpython/issues/108487]: Move um assert que causaria uma falha espúria em um caso tortuoso que deveria apenas acionar a desotimização.

- gh-106176 [https://github.com/python/cpython/issues/106176]: Usa um `WeakValueDictionary` para rastrear as listas que contêm os módulos que cada thread está importando no momento. Isso ajuda a evitar um vazamento de referência, que mantém a lista ativa por mais tempo do que o necessário. Referências fracas são usadas, pois o GC não pode interromper a limpeza.

- gh-105481 [https://github.com/python/cpython/issues/105481]: O estágio de construção regen-opcode foi removido e seu trabalho agora é feito em regen-cases.

- gh-107901 [https://github.com/python/cpython/issues/107901]: Corrige número de linha ausente em **`JUMP_BACKWARD`** no final de um laço for.

- gh-108113 [https://github.com/python/cpython/issues/108113]: A função embutida **`compile()`** agora pode aceitar um novo sinalizador, `ast.PyCF_OPTIMIZED_AST`, que é semelhante a `ast.PyCF_ONLY_AST` exceto que o `AST` retornado é otimizado de acordo com o valor do argumento `optimize`.

  `ast.parse()` agora aceita um argumento opcional `optimize` que é passado para o **`compile()`** embutido. Isto torna possível obter um `AST` otimizado.

- gh-107971 [https://github.com/python/cpython/issues/107971]: Os IDs de opcode são gerados a partir de bytecodes.c em vez de serem codificados em

opcode.py.

- gh-107944 [https://github.com/python/cpython/issues/107944]: Melhora a mensagem de erro para chamadas de função com argumentos nomeados inválidos. Patch por Pablo Galindo.

- gh-108390 [https://github.com/python/cpython/issues/108390]: Levanta uma exceção ao definir um evento não local (`RAISE`, `EXCEPTION_HANDLED`, etc.) em `sys.monitoring.set_local_events`.

  Corrige travamento ao rastrear chamadas recursivas para classes Python.

- gh-108035 [https://github.com/python/cpython/issues/108035]: Remove a estrutura `_PyCFrame`, movendo o ponteiro para o quadro do interpretador atual de volta ao estado da thread, como era na versão 3.10 e anteriores. O `_PyCFrame` existia como uma otimização de desempenho para rastreamento. Desde que a PEP 669 foi implementada, essa otimização não se aplica mais.

- gh-91051 [https://github.com/python/cpython/issues/91051]: Corrige abort/segfault ao usar todos os oito slots do observador de tipo, em plataformas onde `char` tem sinal por padrão.

- gh-106581 [https://github.com/python/cpython/issues/106581]: Corrige possíveis falhas de asserção e eventos de instrumentação ausentes quando **`PYTHONUOPS`** ou **`-X uops`** estiver habilitado.

- gh-107526 [https://github.com/python/cpython/issues/107526]: Reverte a conversão de `vars`, `dir`, `next`, `getattr` e `iter` para clínica de argumentos.

- gh-84805 [https://github.com/python/cpython/issues/84805]: Gera assinatura automaticamente para funções de extensão **`METH_NOARGS`** e **`METH_O`**.

- gh-107758 [https://github.com/python/cpython/issues/107758]: Torna a rotina `dump_stack()` usada pelo recurso `lltrace` (depuração de interpretador de baixo nível) robusta contra recursão, garantindo que ela nunca chame um método `__repr__` implementado em Python. Além disso,

faça com que a saída semelhante para UOPs de Nível 2 apareça em `stdout` (em vez de `stderr`), para corresponder ao código `lltrace` em ceval.c.

- gh-107659 [https://github.com/python/cpython/issues/107659]: Adiciona docstrings para **`ctypes.pointer()`** e **`ctypes.POINTER()`**.

- gh-105848 [https://github.com/python/cpython/issues/105848]: Modifica o bytecode para que o chamável real para um **`CALL`** esteja em uma posição consistente na pilha (independentemente de as otimizações de chamada de método vinculado estarem ativas ou não).

- gh-107674 [https://github.com/python/cpython/issues/107674]: Regressão de desempenho corrigida em `sys.settrace`.

- gh-107724 [https://github.com/python/cpython/issues/107724]: Nas versões de pré-lançamento da 3.12, até a rc1, a função de retorno de chamada sys.monitoring para o evento `PY_THROW` não continha o terceiro argumento, exception. Isso já foi corrigido.

- gh-84436 [https://github.com/python/cpython/issues/84436]: Ignora modificações na contagem de referências para muitos objetos imortais conhecidos.

- gh-107596 [https://github.com/python/cpython/issues/107596]: Especializa a subscrição de objetos **`str`** por índices **`int`**.

- gh-107080 [https://github.com/python/cpython/issues/107080]: As construções de referências de rastreamento (`--with-trace-refs`) estavam travando quando usadas com subinterpretadores isolados. O estado global problemático foi isolado para cada interpretador. Além de corrigir as falhas, essa alteração não afeta os usuários.

- gh-107557 [https://github.com/python/cpython/issues/107557]: Gera os casos necessários para o interpretador abstrato barebones de nível 2 para passagens de otimização em CPython.

- gh-106608 [https://github.com/python/cpython/issues/106608]: Torna `_PyUOpExecutorObject` de comprimento variável.

- gh-100964 [https://github.com/python/cpython/issues/100964]: Limpa o estado de exceção dos geradores após `return` para interromper os ciclos de referência.

- gh-107455 [https://github.com/python/cpython/issues/107455]: Melhora as mensagens de erro ao converter um tipo incompatível para **`ctypes.c_char_p`**, **`ctypes.c_wchar_p`** e **`ctypes.c_void_p`**.

- gh-107263 [https://github.com/python/cpython/issues/107263]: Aumenta o limite de recursão em C para funções que não sejam o interpretador principal de 800 para 1500. Isso deve permitir que funções como `list.__repr__` e `json.dumps` manipulem todas as entradas possíveis antes da versão 3.12.

- gh-104584 [https://github.com/python/cpython/issues/104584]: Corrige um problema que causava a leitura incorreta de caches inline ao executar com **`PYTHONUOPS`** ou **`-X uops`** habilitados.

- gh-104432 [https://github.com/python/cpython/issues/104432]: Corrige possível acesso desalinhado à memória em APIs C envolvendo sequências retornadas de ponteiros `char *` dentro dos módulos **`grp`** e **`socket`**. Isso foi revelado usando uma compilação `-fsaniziter=alignment` no macOS ARM. Patch por Christopher Chavez.

- gh-106078 [https://github.com/python/cpython/issues/106078]: Isola **`_decimal`** (aplicar **PEP 687** [https://peps.python.org/pep-0687/]). Patch por Charlie Zhao.

- gh-106898 [https://github.com/python/cpython/issues/106898]: Adiciona a exceção como terceiro argumento às funções de retorno `PY_UNIND` em `sys.monitoring`. Isso torna o callback `PY_UNWIND` consistente com as outras funções de retorno de tratamento de exceções.

- gh-106895 [https://github.com/python/cpython/issues/106895]: Levanta um `ValueError` quando uma função de callback de monitoramento retornar `DISABLE` para eventos que não podem ser desabilitados localmente.

- gh-106897 [https://github.com/python/cpython/issues/106897]: Adiciona um evento `RERAISE` a `sys.monitoring`, que ocorre quando uma exceção é reativada, seja explicitamente por uma instrução `raise` simples ou implicitamente em um bloco `except` ou `finally`.

- gh-77377 [https://github.com/python/cpython/issues/77377]: Garante que objetos de sincronização multiprocessamento criados em um contexto de fork não sejam enviados para um processo diferente criado em um contexto de geração. Isso transforma um erro de segmentação em um erro de tempo de execução acionável no processo pai.

- gh-106931 [https://github.com/python/cpython/issues/106931]: Objetos string alocados estaticamente agora são internados globalmente em vez de por interpretador. Isso corrige uma situação em que tal string só seria internada em um único interpretador. Objetos string normais não são afetados.

- gh-104621 [https://github.com/python/cpython/issues/104621]: Módulos não suportados agora sempre falham na importação.

- gh-107122 [https://github.com/python/cpython/issues/107122]: Adicionado **`dbm.ndbm.ndbm.clear()`** a **`dbm.ndbm`**. Patch por Donghee Na.

- gh-107122 [https://github.com/python/cpython/issues/107122]: Adiciona **`dbm.gnu.gdbm.clear()`** a **`dbm.gnu`**. Patch por Donghee Na.

- gh-107015 [https://github.com/python/cpython/issues/107015]: Os tokens ASYNC e AWAIT foram removidos da gramática, o que elimina a possibilidade de criar as palavras reservadas contextuais `async` e `await` ao usar `feature_version<7` em **`ast.parse()`**.

- gh-106917 [https://github.com/python/cpython/issues/106917]: Corrige chamadas ao método **`super()`** no estilo classmethod (ou seja, onde o segundo argumento para **`super()`**, ou o segundo argumento implícito extraído de `self/cls` no caso de super com argumento zero, é um tipo) quando o alvo da chamada não é um classmethod.

- gh-105699 [https://github.com/python/cpython/issues/105699]: O Python não trava mais devido a uma corrida pouco frequente ao inicializar strings internas por interpretador. A falha se manifestava quando o interpretador era finalizado.

- gh-105699 [https://github.com/python/cpython/issues/105699]: O Python não trava mais devido a uma corrida pouco frequente na configuração de `Py_FileSystemDefaultEncoding` e `Py_FileSystemDefaultEncodeErrors` (ambos descontinuados), ao inicializar simultaneamente dois subinterpretadores isolados. Agora, eles são definidos apenas durante a inicialização em tempo de execução.

- gh-106908 [https://github.com/python/cpython/issues/106908]: Corrige vários travamentos, vazamentos de referência, falhas de teste e bugs de rastreamento/introspecção ao executar com **`PYTHONUOPS`** ou **`-X uops`** habilitado.

- gh-106092 [https://github.com/python/cpython/issues/106092]: Corrige uma falha de segmentação causada por um bug de uso após liberação em `frame_dealloc` quando a lixeira atrasa a desalocação de um `PyFrameObject`.

- gh-106485 [https://github.com/python/cpython/issues/106485]: Reduz o número de dicionários de instâncias materializadas, desmaterializando-os quando possível.

- gh-106719 [https://github.com/python/cpython/issues/106719]: Não suprime mais erros arbitrários nos getters e setters `__annotations__` nos tipos type e module.

- gh-106723 [https://github.com/python/cpython/issues/106723]: Propaga `frozen_modules` para interpretadores de processos gerados por multiprocessamento.

- gh-104909 [https://github.com/python/cpython/issues/104909]: Divide **`LOAD_ATTR_INSTANCE_VALUE`** em micro-ops.

- gh-104909 [https://github.com/python/cpython/issues/104909]: Divide as especializações `LOAD_GLOBAL` em micro-ops.

- gh-106597 [https://github.com/python/cpython/issues/106597]: Uma nova estrutura de depuração de offsets foi adicionada ao `_PyRuntimeState` que ajudará depuradores e profilers fora do processo a obter as posições para estruturas relevantes do interpretador de uma forma independente de como o Python foi compilado e que não exija a cópia dos cabeçalhos. Patch por Pablo Galindo

- gh-106487 [https://github.com/python/cpython/issues/106487]: Permite que o argumento *count* de `str.replace()` seja nomeado. Patch por Hugo van Kemenade.

- gh-96844 [https://github.com/python/cpython/issues/96844]: Melhora a mensagem de erro de `list.remove()`. Patch por Donghee Na.

- gh-81283 [https://github.com/python/cpython/issues/81283]: O compilador agora remove recuos de docstrings. Reduz o tamanho do arquivo `pyc` em 5% quando o módulo é amplamente documentado. Essa alteração afeta `__doc__`, portanto, ferramentas como doctest serão afetadas.

- gh-106396 [https://github.com/python/cpython/issues/106396]: Quando a especificação de formato de uma expressão de f-string está vazia, o analisador agora gera um nó `ast.JoinedStr` vazio para ela, em vez de um nó **`ast.JoinedStr`** de um elemento com uma string vazia `ast.Constant`.

- gh-100288 [https://github.com/python/cpython/issues/100288]: Especializa `LOAD_ATTR` para não descritores na classe. Adiciona **`LOAD_ATTR_NONDESCRIPTOR_WITH_VALUES`** e **`LOAD_ATTR_NONDESCRIPTOR_NO_DICT`**.

- gh-106008 [https://github.com/python/cpython/issues/106008]: Corrige possíveis vazamentos de referência ao não otimizar comparações com `None` no compilador de bytecode.

- gh-106145 [https://github.com/python/cpython/issues/106145]: Torna `end_lineno` e `end_col_offset` obrigatórios em nós AST `type_param`.

- gh-106213 [https://github.com/python/cpython/issues/106213]: Alterada a forma como os trampolins de chamada do Emscripten funcionam para compatibilidade com a integração Wasm/JS Promise.

- gh-106182 [https://github.com/python/cpython/issues/106182]: **`sys.getfilesystemencoding()`** e **`sys.getfilesystemencodeerrors`** agora retornam objetos Unicode internalizados.

- gh-106210 [https://github.com/python/cpython/issues/106210]: Removido o trampolim de importação do Emscripten, pois não era mais necessário para o Pyodide.

- gh-104584 [https://github.com/python/cpython/issues/104584]: Adicionado um novo otimizador e interpretador de rastreamento experimental (também conhecido como “nível 2”). Atualmente, isso é pessimista, então não use ainda — esta é uma infraestrutura para que possamos experimentar com passes de otimização. Para habilitá-lo, passe `-Xuops` ou defina `PYTHONUOPS=1`. Para obter a saída de depuração, defina `PYTHONUOPSDEBUG=N`, onde `N` é um nível de depuração (0-4, onde 0 indica ausência de saída de depuração e 4 indica excesso de verbosidade).

- gh-105775 [https://github.com/python/cpython/issues/105775]: **`LOAD_CLOSURE`** agora é uma pseudo-op.

- gh-105730 [https://github.com/python/cpython/issues/105730]: Permite qualquer chamada que não seja de objetos de tipo como predicado de condição em **`BaseExceptionGroup.split()`** e **`BaseExceptionGroup.subgroup()`**.

- gh-105979 [https://github.com/python/cpython/issues/105979]: Corrige falha em **`_imp.get_frozen_object()`** devido a tratamento incorreto de exceções.

- gh-106003 [https://github.com/python/cpython/issues/106003]: Adiciona uma nova instrução `TO_BOOL`, que realiza conversões booleanas para `POP_JUMP_IF_TRUE`, `POP_JUMP_IF_FALSE` e `UNARY_NOT` (que agora esperam valores `bool` exatos). Além disso, modifique o oparg de `COMPARE_OP` para incluir um sinalizador opcional de "conversão booleana".

- gh-98931 [https://github.com/python/cpython/issues/98931]: Garante que mensagens de erro personalizadas `SyntaxError` sejam levantadas para importações inválidas com múltiplos destinos. Patch por Pablo Galindo

- gh-105724 [https://github.com/python/cpython/issues/105724]: Melhora as mensagens de erro `assert` fornecendo o intervalo exato de erro.

- gh-105908 [https://github.com/python/cpython/issues/105908]: Corrige o bug em que gh-99111 [https://github.com/python/cpython/issues/99111] quebrava a importação futura `barry_as_FLUFL` no REPL do Python.

- gh-105840 [https://github.com/python/cpython/issues/105840]: Corrige possíveis travamentos ao especializar chamadas de função com muitos `__defaults__`.

- gh-105831 [https://github.com/python/cpython/issues/105831]: Corrige um bug em uma f-string, onde o uso de uma expressão de depuração (o sinal =) que aparece na última linha de um arquivo resulta no buffer de depuração que contém o texto da expressão sendo um caractere a menos.

- gh-105800 [https://github.com/python/cpython/issues/105800]: Emite corretamente `SyntaxWarning` em f-strings se sequências inválidas forem usadas. Patch por Pablo Galindo

- gh-105340 [https://github.com/python/cpython/issues/105340]: Inclui a variável de iteração de compreensão em `locals()` dentro de uma compreensão de escopo de módulo ou classe.

- gh-105331 [https://github.com/python/cpython/issues/105331]: Levanta `ValueError` se o argumento `delay` para `asyncio.sleep()` for um NaN (correspondente a `time.sleep()`).

- gh-105587 [https://github.com/python/cpython/issues/105587]: O tempo de execução não pode garantir que objetos imortais não serão mutados por extensões. Portanto, isso modifica _PyStaticObject_CheckRefcnt para avisar em vez de declarar.

- gh-105564 [https://github.com/python/cpython/issues/105564]: Não inclui quebras de linha artificiais no atributo `line` de tokens nas APIs do módulo `tokenize`. Patch por Pablo Galindo

- gh-105549 [https://github.com/python/cpython/issues/105549]: Tokeniza separadamente os tokens `NUMBER` e `NAME` que não sejam ambíguos. Patch por Pablo Galindo.

- gh-105588 [https://github.com/python/cpython/issues/105588]: Corrige um problema que poderia resultar em travamentos ao compilar nós `ast` malformados.

- gh-100987 [https://github.com/python/cpython/issues/100987]: Permite objetos que não são objetos código como o “executável” em quadros internos. A longo prazo, isso pode ajudar ferramentas como Cython e PySpy a interagirem de forma mais eficiente. A curto prazo, nos permite realizar algumas otimizações de forma mais simples.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs no módulo `builtins`, onde exceções poderiam ser sobrescritas.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bug no compilador, onde uma exceção poderia ser sobrescrita.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Melhora o tratamento de erros em `PyUnicode_BuildEncodingMap()`, onde uma exceção poderia ser sobrescrita.

- gh-105486 [https://github.com/python/cpython/issues/105486]: Altera a representação da lista de argumentos `ParamSpec` em `types.GenericAlias`.

- gh-105678 [https://github.com/python/cpython/issues/105678]: Divide a instrução `MAKE_FUNCTION` em duas partes: `MAKE_FUNCTION`, que cria a função, e `SET_FUNCTION_ATTRIBUTE`, que define os atributos da função. Isso torna o efeito de pilha de `MAKE_FUNCTION` regular para facilitar a otimização e a geração de código.

- gh-105435 [https://github.com/python/cpython/issues/105435]: Corrige o caractere de quebra de linha falso se o arquivo terminar em um comentário sem quebra de linha. Patch por Pablo Galindo

- gh-105390 [https://github.com/python/cpython/issues/105390]: Levanta corretamente as exceções **`tokenize.TokenError`** em vez de **`SyntaxError`** para erros de tokenização, como entrada incompleta. Patch por Pablo Galindo

- gh-105259 [https://github.com/python/cpython/issues/105259]: Não inclui o caractere de quebra de linha para tokens `NEWLINE` finais emitidos no módulo **`tokenize`**. Patch por Pablo Galindo

- gh-104635 [https://github.com/python/cpython/issues/104635]: Elimina instruções redundantes **`STORE_FAST`** no compilador. Patch por Donghee Na e Carl Meyer.

- gh-105324 [https://github.com/python/cpython/issues/105324]: Corrige a função principal do módulo **`tokenize`** ao ler de `sys.stdin`. Patch por Pablo Galindo

- gh-33092 [https://github.com/python/cpython/issues/33092]: Simplifica e acelera o interpretador para f-strings. Remove o opcode `FORMAT_VALUE`. Adiciona opcodes `CONVERT_VALUE`, `FORMAT_SIMPLE` e `FORMAT_WITH_SPEC`. O compilador emite uma sequência mais eficiente para cada expressão de formato.

- gh-105229 [https://github.com/python/cpython/issues/105229]: Remove as superinstruções restantes de duas unidades de código. Todas as superinstruções restantes usam apenas uma unidade de código, simplificando a instrumentação e agilizando o processamento.

- gh-105235 [https://github.com/python/cpython/issues/105235]: Impede o acesso à memória fora dos limites durante chamadas `mmap.find()`.

- gh-98963 [https://github.com/python/cpython/issues/98963]: Restaura a capacidade de uma subclasse de **`property`** definir `__slots__` ou, de outra forma, ser livre de dicionários, ignorando falhas na definição de uma docstring em tal classe. Esse comportamento havia regredido na versão 3.12beta1. Um **`AttributeError`** onde não havia um anteriormente era prejudicial ao código existente.

- gh-104812 [https://github.com/python/cpython/issues/104812]: O mecanismo de "chamada pendente" agora funciona para todos os interpretadores, não apenas para o interpretador principal, e é executado em todas as threads, não apenas na thread principal. Algumas chamadas ainda são feitas apenas na thread principal, ou seja, no interpretador principal. Esta alteração não afeta o tratamento de sinais nem a C-API pública existente (`Py_AddPendingCall()`), que ainda têm como alvo apenas a thread principal. A nova funcionalidade destina-se estritamente ao uso interno por enquanto, uma vez que as consequências de seu uso ainda não são bem compreendidas, exceto em alguns casos muito restritos. Esta alteração alinha a capacidade com a intenção quando o estado foi definido por interpretador há vários anos.

- gh-105194 [https://github.com/python/cpython/issues/105194]: Não escapa com barras invertidas os especificadores de formato de f-string. Patch por Pablo Galindo

- gh-105229 [https://github.com/python/cpython/issues/105229]: Substitui algumas superinstruções dinâmicas por equivalentes de instrução única.

- gh-105162 [https://github.com/python/cpython/issues/105162]: Corrigido bug em generator.close()/throw(), onde um iterador interno era ignorado quando o iterador externo era instrumentado.

- gh-105164 [https://github.com/python/cpython/issues/105164]: Garante que as anotações sejam configuradas corretamente se a única anotação em um bloco estiver dentro de um bloco `match`. Patch por Jelle Zijlstra.

- gh-105148 [https://github.com/python/cpython/issues/105148]: Torna `_PyASTOptimizeState` interno a ast_opt.c. Torna `_PyAST_Optimize` com dois inteiros em vez de um ponteiro para esta estrutura. Isso evita a necessidade de incluir pycore_compile.h em ast_opt.c.

- gh-104799 [https://github.com/python/cpython/issues/104799]: Atributos de nós `ast` que são listas agora assumem como padrão a lista vazia se omitidos. Isso significa que alguns códigos que anteriormente geravam `TypeError` quando o nó AST era usado agora prosseguirão com a lista vazia. Patch por Jelle Zijlstra.

- gh-105111 [https://github.com/python/cpython/issues/105111]: Remove as antigas macros da lixeira `Py_TRASHCAN_SAFE_BEGIN` e `Py_TRASHCAN_SAFE_END`. Elas devem ser substituídas pelas novas macros `Py_TRASHCAN_BEGIN` e `Py_TRASHCAN_END`.

- gh-105035 [https://github.com/python/cpython/issues/105035]: Corrige chamadas de `super()` em tipos com implementação personalizada de `tp_getattro` (por exemplo, metatipos).

- gh-105017 [https://github.com/python/cpython/issues/105017]: Exibe linhas CRLF no atributo de string tokenize em tokens NL e NEWLINE. Patch por Marta Gómez.

- gh-105013 [https://github.com/python/cpython/issues/105013]: Corrige o tratamento de lambdas entre parênteses de várias linhas em `inspect.getsource()`. Patch por Pablo Galindo

- gh-105017 [https://github.com/python/cpython/issues/105017]: Não inclui um token final `NL` adicional ao analisar arquivos com linhas CRLF. Patch por Marta Gómez.

- gh-104976 [https://github.com/python/cpython/issues/104976]: Garante que os objetos `tokenize.TokenInfo` `DEDENT` finais emitidos pelo módulo

`tokenize` sejam reportados como no Python 3.11. Patch por Pablo Galindo.

- gh-104972 [https://github.com/python/cpython/issues/104972]: Garante que o atributo `line` em objetos `tokenize.TokenInfo` no módulo `tokenize` estejam sempre corretos. Patch por Pablo Galindo

- gh-104955 [https://github.com/python/cpython/issues/104955]: Corrige a assinatura para o novo slot de `__release_buffer__()`. Patch por Jelle Zijlstra.

- gh-104690 [https://github.com/python/cpython/issues/104690]: Iniciar novas threads e criar processos através de `os.fork()` durante o desligamento do interpretador (como a partir de manipuladores `atexit`) não é mais suportado. Isso pode levar a uma condição de corrida entre a thread principal do tempo de execução do Python, liberando estados de threads enquanto rotinas internas de `threading` tentam alocar e usar o estado de threads recém-criadas. Ou filhos do fork tentando usar o tempo de execução e o estado da thread no meio do desligamento no processo filho.

- gh-104879 [https://github.com/python/cpython/issues/104879]: Corrige a falha ao acessar o atributo `__module__` de apelidos de tipo definidos fora de um módulo. Patch por Jelle Zijlstra.

- gh-104825 [https://github.com/python/cpython/issues/104825]: Tokens emitidos pelo módulo `tokenize` não incluem mais um caractere `\\n` implícito no atributo `line`. Patch por Pablo Galindo

- gh-104770 [https://github.com/python/cpython/issues/104770]: Se um gerador retornar um valor ao ser fechado, o valor agora é retornado por `generator.close()`.

- gh-89091 [https://github.com/python/cpython/issues/89091]: Levanta `RuntimeWarning` para métodos de gerador assíncronos não aguardados, como `asend()`, `athrow()` e `aclose()`. Patch por Kumar Aditya.

- gh-96663 [https://github.com/python/cpython/issues/96663]: Adiciona uma mensagem de erro melhor e mais introspectiva ao definir atributos em

classes sem um `__dict__` e sem membro de slot para o atributo.

- gh-93627 [https://github.com/python/cpython/issues/93627]: Atualiza a implementação do módulo pickle em Python para corresponder à implementação em C do módulo pickle. Para objetos que definem métodos de redução como `__reduce_ex__()` ou `__reduce__()` como `None`, a operação de pickle resultará em um `TypeError`.

- gh-101006 [https://github.com/python/cpython/issues/101006]: Melhora o tratamento de erros ao ler dados `marshal`.

- gh-91095 [https://github.com/python/cpython/issues/91095]: Especializa chamadas para a maioria das classes Python. Especificamente, qualquer classe que herde de `object` ou de outra classe Python e não sobrescreva `__new__`.

  A instrução especializada faz o seguinte:

  1. Cria o objeto (chamando `object.__new__`)
  2. Envia um quadro de correção para a pilha de quadros (para limpeza após `__init__`)
  3. Envia o quadro de `__init__` para a pilha de quadros

  Acelera a instanciação da maioria das classes Python.

Biblioteca

- gh-110786 [https://github.com/python/cpython/issues/110786]: A CLI de `sysconfig` agora ignora `BrokenPipeError`, fazendo com que ele saia normalmente se sua saída estiver sendo canalizada e o canal for fechado.

- gh-103480 [https://github.com/python/cpython/issues/103480]: O módulo `sysconfig` agora é um pacote, em vez de um módulo de arquivo único.

- gh-110733 [https://github.com/python/cpython/issues/110733]: Microotimização: Evite chamar `min()`, `max()` em `BaseEventLoop._run_once()`.

- gh-94597 [https://github.com/python/cpython/issues/94597]: Adicionado **`asyncio.EventLoop`** para uso com o kwarg **`asyncio.run()`** *loop_factory* para evitar chamar o sistema de políticas asyncio.

- gh-110682 [https://github.com/python/cpython/issues/110682]: **`protocolos runtime-checkable`** usados para considerar `__match_args__` um membro do protocolo em `__instancecheck__` se estivesse presente no protocolo. Agora, este atributo é ignorado se estiver presente.

- gh-110488 [https://github.com/python/cpython/issues/110488]: Corrige alguns problemas em **`pathlib.PurePath.with_name()`**: um único ponto era considerado incorretamente um nome válido e, em **`PureWindowsPath`**, um nome com um fluxo de dados NTFS alternativo, como `a:b`, era considerado incorretamente inválido.

- gh-110590 [https://github.com/python/cpython/issues/110590]: Corrige um bug em **`_sre.compile()`**, onde **`TypeError`** era substituído por **`OverflowError`** quando o argumento *code* era uma lista de não-ints.

- gh-65052 [https://github.com/python/cpython/issues/65052]: Impede que **`pdb`** trave ao tentar exibir objetos não exibíveis

- gh-110519 [https://github.com/python/cpython/issues/110519]: Aviso de descontinuação sobre números não inteiros em **`gettext`** agora sempre se refere à linha no código do usuário onde a função ou método gettext é usado. Anteriormente, ele podia se referir a uma linha no código `gettext`.

- gh-89902 [https://github.com/python/cpython/issues/89902]: Descontinua o especificador de formato não padrão “N” para **`decimal.Decimal`**. Ele não foi documentado e suportado apenas na implementação em C.

- gh-110378 [https://github.com/python/cpython/issues/110378]: Os gerenciadores de contexto **`contextmanager()`** e **`asynccontextmanager()`** agora fecham um objeto gerador subjacente inválido que produz mais de um valor.

- gh-106670 [https://github.com/python/cpython/issues/106670]: Em **`pdb`**, defina a variável de conveniência `$_exception` para depuração post mortem.

- gh-110365 [https://github.com/python/cpython/issues/110365]: Corrige o bug em `termios.tcsetattr()` que estava sobrescrevendo erros existentes durante a análise de inteiros da lista `term`.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Melhora ligeiramente o tempo de importação de vários módulos da biblioteca padrão, adiando as importações de `warnings` dentro desses módulos. Patch por Alex Waygood.

- gh-110273 [https://github.com/python/cpython/issues/110273]: `dataclasses.replace()` agora levanta TypeError em vez de ValueError se especificar o argumento nomeado para um campo declarado com init=False ou se o argumento nomeado não for especificado para o campo InitVar obrigatório.

- gh-110249 [https://github.com/python/cpython/issues/110249]: Adiciona o sinalizador `--inline-caches` à linha de comando `dis`.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Corrige uma regressão do Python 3.12 no tempo de importação de `random`. Patch por Alex Waygood.

- gh-110222 [https://github.com/python/cpython/issues/110222]: Adiciona suporte a objetos sequência struct em `copy.replace()`. Patch por Xuehai Pan.

- gh-109649 [https://github.com/python/cpython/issues/109649]: `multiprocessing`, `concurrent.futures`, `compileall`: Substitui `os.cpu_count()` por `os.process_cpu_count()` para selecionar o número padrão de threads e processos de trabalho. Obter a afinidade da CPU, se compatível. Patch por Victor Stinner.

- gh-110150 [https://github.com/python/cpython/issues/110150]: Corrige o tratamento de caso base em statistics.quantiles. Agora permite um único ponto de dados.

- gh-110036 [https://github.com/python/cpython/issues/110036]: No Windows, o multiprocessamento `Popen.terminate()` agora captura `PermissionError` e obtém o código de saída do processo. Se o

processo ainda estiver em execução, lança novamente o **`PermissionError`**. Caso contrário, o processo é encerrado conforme o esperado: armazena seu código de saída. Patch por Victor Stinner.

- gh-110038 [https://github.com/python/cpython/issues/110038]: Corrige um problema que fazia com que **`KqueueSelector.select()`** não retornasse todos os eventos de prontidão em alguns casos quando um descritor de arquivo é registrado para leitura e escrita.

- gh-110045 [https://github.com/python/cpython/issues/110045]: Atualiza o módulo **`symtable`** para suportar os novos escopos introduzidos por **PEP 695** [https://peps.python.org/pep-0695/].

- gh-88402 [https://github.com/python/cpython/issues/88402]: Adiciona novas variáveis a **`sysconfig.get_config_vars()`** no Windows: `LIBRARY`, `LDLIBRARY`, `LIBDIR`, `SOABI` e `Py_NOGIL`.

- gh-109631 [https://github.com/python/cpython/issues/109631]: Funções do **`re`** como **`re.findall()`**, **`re.split()`**, **`re.search()`** e **`re.sub()`**, que realizam correspondências curtas e repetidas, agora podem ser interrompidas pelo usuário.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Reduz o tempo de importação de **`email.utils`** em cerca de 43%. Isso resulta em uma redução de cerca de 18% no tempo de importação de **`email.message`**, o que, por sua vez, reduz o tempo de importação de **`importlib.metadata`** em cerca de 6%. Patch por Alex Waygood.

- gh-109818 [https://github.com/python/cpython/issues/109818]: Corrige **`reprlib.recursive_repr()`** que não copiava `__type_params__` da função decorada.

- gh-109047 [https://github.com/python/cpython/issues/109047]: **`concurrent.futures`**: A *thread do gerenciador de executores* agora captura exceções ao adicionar um item à *fila de chamadas*. Durante a finalização do Python, a criação de uma nova thread agora pode gerar **`RuntimeError`**. Captura a chamada à exceção `terminate_broken()` neste caso. Patch por Victor Stinner.

- gh-109782 [https://github.com/python/cpython/issues/109782]: Garante que a assinatura de **os.path.isdir()** seja idêntica em todas as plataformas. Patch por Amin Alaee.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Melhora o tempo de importação de **functools** em cerca de 13%. Patch por Alex Waygood.

- gh-109590 [https://github.com/python/cpython/issues/109590]: **shutil.which()** dará preferência a arquivos com a extensão `PATHEXT` se o modo fornecido incluir `os.X_OK` no Win32. Se nenhuma correspondência com `PATHEXT` for encontrada, um arquivo sem a extensão `PATHEXT` poderá ser retornado. Esta alteração fará com que **shutil.which()** aja de forma mais semelhante ao comportamento anterior no Python 3.11.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Reduz o tempo de importação de **enum** em mais de 50%. Patch por Alex Waygood.

- gh-109593 [https://github.com/python/cpython/issues/109593]: Evita impasses em uma chamada reentrante para o rastreador de recursos de multiprocessamento. Tal chamada reentrante, embora improvável, pode ocorrer se uma passagem de GC invocar o finalizador para um objeto de multiprocessamento, como SemLock.

- gh-109653 [https://github.com/python/cpython/issues/109653]: Reduz o tempo de importação de **typing** em cerca de um terço. Patch por Alex Waygood.

- gh-109649 [https://github.com/python/cpython/issues/109649]: Adiciona a função **os.process_cpu_count()** para obter o número de CPUs lógicas utilizáveis pela thread de chamada do processo atual. Patch por Victor Stinner.

- gh-74481 [https://github.com/python/cpython/issues/74481]: Adiciona constantes relacionadas a `set_error_mode` no módulo `msvcrt` na construção de depuração do Python.

- gh-109613 [https://github.com/python/cpython/issues/109613]: Corrige **os.stat()** e **os.DirEntry.stat()**: verificar exceções. Anteriormente, no Python construído em modo de depuração, essas funções podiam disparar um

erro fatal do Python (e abortar o processo) quando uma função era bem-sucedida com uma exceção definida. Patch por Victor Stinner.

- gh-109599 [https://github.com/python/cpython/issues/109599]: Expõe o tipo de objetos PyCapsule como `types.CapsuleType`.

- gh-109109 [https://github.com/python/cpython/issues/109109]: Agora você pode obter as cadeias de certificados TLS brutas de conexões TLS por meio dos métodos **`ssl.SSLSocket.get_verified_chain()`** e **`ssl.SSLSocket.get_unverified_chain()`**.

  Contribuição de Mateusz Nowak.

- gh-109559 [https://github.com/python/cpython/issues/109559]: Atualiza o banco de dados **`unicodedata`** para Unicode 15.1.0.

- gh-109543 [https://github.com/python/cpython/issues/109543]: Remove a verificação desnecessária de **`hasattr()`** durante a criação de **`typing.TypedDict`**.

- gh-109495 [https://github.com/python/cpython/issues/109495]: Remove `__slots__` extras desnecessários na implementação em Python puro de **`datetime`** para reduzir o tamanho da memória, pois são definidos na superclasse. Patch por James Hilton-Balfe

- gh-109461 [https://github.com/python/cpython/issues/109461]: **`logging`**: Usa um gerenciador de contexto para aquisição de travas.

- gh-109096 [https://github.com/python/cpython/issues/109096]: **`http.server.CGIHTTPRequestHandler`** has been deprecated for removal in 3.15. Its design is old and the web world has long since moved beyond CGI.

- gh-109409 [https://github.com/python/cpython/issues/109409]: Corrige o erro que permitia herdar uma classe de dados congelada de vários pais, alguns dos quais possivelmente não estavam congelados.

- gh-109375 [https://github.com/python/cpython/issues/109375]: O comando **`pdb`** `alias` agora impede o registro de apelidos sem argumentos.

- gh-109319 [https://github.com/python/cpython/issues/109319]: Descontinua o campo `dis.HAVE_ARGUMENT` em favor de `dis.hasarg`.

- gh-107219 [https://github.com/python/cpython/issues/107219]: Uma condição de corrida foi corrigida em `concurrent.futures`. Quando um processo no pool de processos era encerrado abruptamente (enquanto o future estava em execução ou pendente), a conexão era encerrada. Se a fila de chamadas estiver bloqueada no envio de bytes para um processo de trabalho, o encerramento da conexão de escrita interrompe o envio, permitindo o fechamento da fila. Patch por Victor Stinner.

- gh-66143 [https://github.com/python/cpython/issues/66143]: O objeto **`codecs.CodecInfo`** agora é copiável e pode ser selecionado. Patch por Robert Lehmann e Furkan Onder.

- gh-109187 [https://github.com/python/cpython/issues/109187]: **`pathlib.Path.resolve()`** agora trata laços de links simbólicos como outros erros: no modo estrito, **`OSError`** é levantada, e no modo não estrito, nenhuma exceção é levantada.

- gh-50644 [https://github.com/python/cpython/issues/50644]: Tenta serializar com pickle ou criar uma cópia rasa ou profunda de fluxos **`codecs`** agora levantam um TypeError. Anteriormente, a cópia falhava com um RecursionError, enquanto o pickle produzia resultados incorretos que eventualmente causavam a falha ao desserializar com pickle com um RecursionError.

- gh-109174 [https://github.com/python/cpython/issues/109174]: Adicionado suporte a **`types.SimpleNamespace`** em **`copy.replace()`**.

- gh-109164 [https://github.com/python/cpython/issues/109164]: **`pdb`**: Substitui **`getopt`** por **`argparse`** para analisar argumentos de linha de comando.

- gh-109151 [https://github.com/python/cpython/issues/109151]: Habilita os recursos de edição `readline` na interface de linha de comando sqlite3 (`python`

`-m sqlite3`).

- gh-108987 [https://github.com/python/cpython/issues/108987]: Corrige a condição de corrida `_thread.start_new_thread()`. Se uma thread for criada durante a finalização do Python, a thread recém-gerada agora sai imediatamente em vez de tentar acessar a memória liberada e causar um travamento. Patch por Victor Stinner.

- gh-108682 [https://github.com/python/cpython/issues/108682]: Enum: requer `names=()` ou `type=...` para criar uma enumeração vazia usando a sintaxe funcional.

- gh-109033 [https://github.com/python/cpython/issues/109033]: Exceções levantadas pela função embutida os.utime agora incluem o nome do arquivo relacionado.

- gh-108843 [https://github.com/python/cpython/issues/108843]: Corrige um problema em `ast.unparse()` ao desfazer a análise de f-strings contendo muitos tipos de aspas.

- gh-108469 [https://github.com/python/cpython/issues/108469]: `ast.unparse()` agora oferece suporte à nova sintaxe f-string introduzida no Python 3.12. Observe que as aspas f-string foram selecionadas novamente para simplificar a nova sintaxe. (Patch por Steven Sun)

- gh-108751 [https://github.com/python/cpython/issues/108751]: Adiciona a função `copy.replace()`, que permite criar uma cópia modificada de um objeto. Ela oferece suporte às tuplas nomeadas, classes de dados e muitos outros objetos.

- gh-108682 [https://github.com/python/cpython/issues/108682]: Enum: Levanta `TypeError` se `super().__new__()` for chamado de um `__new__` personalizado.

- gh-108278 [https://github.com/python/cpython/issues/108278]: Descontinua a passagem do retorno de chamada por argumento nomeado para as seguintes APIs `sqlite3.Connection`:

  - `set_authorizer()`
  - `set_progress_handler()`
  - `set_trace_callback()`

  Os parâmetros afetados se tornarão somente-posicionais no Python 3.15.

  Patch por Erlend E. Aasland.

- gh-105829 [https://github.com/python/cpython/issues/105829]: Corrige impasse (deadlock) em concurrent.futures.ProcessPoolExecutor

- gh-108295 [https://github.com/python/cpython/issues/108295]: Corrige travamentos relacionados ao uso de referências fracas em `typing.TypeVar`.

- gh-108463 [https://github.com/python/cpython/issues/108463]: Faz com que expressões/instruções funcionem conforme o esperado no pdb

- gh-108277 [https://github.com/python/cpython/issues/108277]: Adiciona `os.timerfd_create()`, `os.timerfd_settime()`, `os.timerfd_gettime()`, `os.timerfd_settime_ns()` e `os.timerfd_gettime_ns()` para fornecer uma interface de baixo nível para o descritor de arquivo de notificação de timer do Linux.

- gh-107811 [https://github.com/python/cpython/issues/107811]: `tarfile`: a extração de membros com UID ou GID muito grande (por exemplo, em um sistema operacional com `id_t` de 32 bits) agora falha da mesma forma que a falha na definição do ID.

- gh-64662 [https://github.com/python/cpython/issues/64662]: Corrige o suporte para tabelas virtuais em `sqlite3.Connection.iterdump()`. Patch por Aviv Palivoda.

- gh-108111 [https://github.com/python/cpython/issues/108111]: Corrige uma regressão introduzida em gh-101251 [https://github.com/python/cpython/issues/101251] para a versão 3.12, resultando em um cálculo de deslocamento incorreto em `gzip.GzipFile.seek()`.

- gh-108294 [https://github.com/python/cpython/issues/108294]: **`time.sleep()`** agora levanta um evento de auditoria.

- gh-108278 [https://github.com/python/cpython/issues/108278]: Descontinua a passagem de nome, número de argumentos e o chamável como argumentos nomeados para as seguintes APIs de **`sqlite3.Connection`**:

  - **`create_function()`**
  - **`create_aggregate()`**

  Os parâmetros afetados se tornarão somente-posicionais no Python 3.15.

  Patch por Erlend E. Aasland.

- gh-108322 [https://github.com/python/cpython/issues/108322]: Aceleração de NormalDist.samples() usando o método CDF inverso em vez de chamar random.gauss().

- gh-83417 [https://github.com/python/cpython/issues/83417]: Adiciona a capacidade do venv de criar um arquivo `.gitignore`, que faz com que o ambiente criado seja ignorado pelo Git. Essa configuração é ativada por padrão quando o venv é chamado por meio de sua CLI.

- gh-105736 [https://github.com/python/cpython/issues/105736]: Harmonização da versão Python pura de **`OrderedDict`** com a versão C. Agora, ambas as versões configuram seu estado interno em `__new__`. Anteriormente, a versão Python pura fazia a configuração em `__init__`.

- gh-108083 [https://github.com/python/cpython/issues/108083]: Corrige bugs no construtor de **`sqlite3.Connection`** e **`sqlite3.Connection.close()`**, onde exceções poderiam vazar. Patch por Erlend E. Aasland.

- gh-107932 [https://github.com/python/cpython/issues/107932]: Corrige o módulo `dis` para relatar e exibir corretamente o bytecode que não possui linhas de código-fonte.

- gh-105539 [https://github.com/python/cpython/issues/105539]: **`sqlite3`** agora emite uma **`ResourceWarning`** se um objeto **`sqlite3.Connection`** não for **`fechado`** explicitamente. Patch por Erlend E. Aasland.

- gh-107995 [https://github.com/python/cpython/issues/107995]: O atributo `__module__` em instâncias de **`functools.cached_property`** agora é definido como o nome do módulo no qual a cached_property está definida, em vez de “functools”. Isso significa que os doctests em docstrings `cached_property` agora são coletados corretamente pelo módulo **`doctest`**. Patch por Tyler Smart.

- gh-107963 [https://github.com/python/cpython/issues/107963]: Corrige **`multiprocessing.set_forkserver_preload()`** para verificar a lista de nomes de módulos fornecida. Patch por Donghee Na.

- gh-106242 [https://github.com/python/cpython/issues/106242]: Corrige **`os.path.normpath()`** para manipular caracteres nulos incorporados sem truncar o caminho.

- gh-81555 [https://github.com/python/cpython/issues/81555]: **`xml.dom.minidom`** agora coloca apenas aspas `"` em atributos.

- gh-50002 [https://github.com/python/cpython/issues/50002]: **`xml.dom.minidom`** agora preserva espaços em branco em atributos.

- gh-93057 [https://github.com/python/cpython/issues/93057]: Passa mais de um argumento posicional para **`sqlite3.connect()`** e o construtor **`sqlite3.Connection`** está obsoleto. Os parâmetros restantes se tornarão somente-nomeados no Python 3.15. Patch por Erlend E. Aasland.

- gh-76913 [https://github.com/python/cpython/issues/76913]: Adiciona o parâmetro/recurso *merge_extra* a **`logging.LoggerAdapter`**

- gh-107913 [https://github.com/python/cpython/issues/107913]: Corrige possíveis perdas dos valores `errno` e `winerror` em exceções **`OSError`** se eles fossem limpados ou modificados pelo código de limpeza antes de criar o objeto de exceção.

- gh-107845 [https://github.com/python/cpython/issues/107845]: **`tarfile.data_filter()`** agora leva em consideração a localização dos links simbólicos ao determinar seu destino, portanto, não rejeitará mais alguns tarballs válidos com `LinkOutsideDestinationError`.

- gh-107812 [https://github.com/python/cpython/issues/107812]: Estende o suporte a netlink do socket para a plataforma FreeBSD.

- gh-107805 [https://github.com/python/cpython/issues/107805]: Corrige assinaturas de funções geradas em nível de módulo no **`turtle`**.

- gh-107782 [https://github.com/python/cpython/issues/107782]: **`pydoc`** agora é capaz de exibir assinaturas que não são representáveis em Python, por exemplo, para `getattr` e `dict.pop`.

- gh-56166 [https://github.com/python/cpython/issues/56166]: Descontinua a passagem de argumentos opcionais *maxsplit*, *count* e *flags* em funções de nível de módulo **`re.split()`**, **`re.sub()`** e **`re.subn()`** como posicionais. Eles devem ser passados apenas como nomeados.

- gh-107710 [https://github.com/python/cpython/issues/107710]: Acelera **`logging.getHandlerNames()`**.

- gh-107715 [https://github.com/python/cpython/issues/107715]: Corrige **`doctest.DocTestFinder.find()`** na presença de nomes de classe com caracteres especiais. Patch por Gertjan van Zwieten.

- gh-100814 [https://github.com/python/cpython/issues/100814]: Passa um objeto chamável como um valor de opção para uma imagem Tkinter agora levanta o TclError esperado em vez de um AttributeError.

- gh-72684 [https://github.com/python/cpython/issues/72684]: Adiciona métodos de widget **`tkinter`**: **`tk_busy_hold()`**, **`tk_busy_configure()`**, **`tk_busy_cget()`**, **`tk_busy_forget()`**, **`tk_busy_current()`** e **`tk_busy_status()`**.

- gh-106684 [https://github.com/python/cpython/issues/106684]: Levanta **`ResourceWarning`** quando **`asyncio.StreamWriter`** não for fechado,

causando vazamentos de memória. Patch por Kumar Aditya.

- gh-107465 [https://github.com/python/cpython/issues/107465]: Adiciona método de classe **`pathlib.Path.from_uri()`**.

- gh-107077 [https://github.com/python/cpython/issues/107077]: Parece que em algumas condições, o OpenSSL retornará `SSL_ERROR_SYSCALL` em vez de `SSL_ERROR_SSL` quando uma verificação de certificação falhar, mas os parâmetros de erro ainda conterão `ERR_LIB_SSL` e `SSL_R_CERTIFICATE_VERIFY_FAILED`. Estamos detectando essa situação e levantando o erro `ssl.SSLCertVerificationError` apropriado. Patch por Pablo Galindo

- gh-107576 [https://github.com/python/cpython/issues/107576]: Corrige **`types.get_original_bases()`** para retornar **`__orig_bases__`** somente se estiver presente diretamente em `cls`. Patch por James Hilton-Balfe.

- gh-105481 [https://github.com/python/cpython/issues/105481]: Remove `opcode.is_pseudo`, `opcode.MIN_PSEUDO_OPCODE` e `opcode.MAX_PSEUDO_OPCODE`, que foram adicionados na versão 3.12, nunca foram documentados e não foram projetados para uso externo.

- gh-105481 [https://github.com/python/cpython/issues/105481]: **`opcode.ENABLE_SPECIALIZATION`** (que foi adicionado na versão 3.12, mas nunca foi documentado ou destinado ao uso externo) foi movido para **`_opcode.ENABLE_SPECIALIZATION`**, onde os testes podem acessá-lo.

- gh-107396 [https://github.com/python/cpython/issues/107396]: tarfiles; Corrigido o uso antes da atribuição de self.exception para descompactação gzip.

- gh-107409 [https://github.com/python/cpython/issues/107409]: Definido o atributo **`__wrapped__`** em **`reprlib.recursive_repr()`**.

- gh-107406 [https://github.com/python/cpython/issues/107406]: Implementa o novo método **`__repr__()`** para **`struct.Struct`**. Agora ele retorna `Struct(<format repr>)`.

- gh-107369 [https://github.com/python/cpython/issues/107369]: Otimiza **`textwrap.indent()`**. Ele está ~30% mais rápido para entradas grandes. Patch por Inada Naoki.

- gh-78722 [https://github.com/python/cpython/issues/78722]: Corrige o problema em que **`pathlib.Path.iterdir()`** não levantava **`OSError`** até ser iterado.

- gh-105578 [https://github.com/python/cpython/issues/105578]: Descontinua **`typing.AnyStr`** em favor da nova sintaxe de parâmetro de tipo. Consulte PEP 695.

- gh-62519 [https://github.com/python/cpython/issues/62519]: Faz com que **`gettext.pgettext()`** busque definições no plural quando a tradução não for encontrada.

- gh-107089 [https://github.com/python/cpython/issues/107089]: Prateleiras abertas com **`shelve.open()`** têm um método **`clear()`** muito mais rápido. Patch por James Cave.

- gh-82500 [https://github.com/python/cpython/issues/82500]: Corrige estouro em sistemas de 32 bits com a implementação de **`asyncio`** **`os.sendfile()`**.

- gh-83006 [https://github.com/python/cpython/issues/83006]: Documenta o comportamento de **`shutil.disk_usage()`** para sistemas de arquivos não montados no Unix.

- gh-65495 [https://github.com/python/cpython/issues/65495]: Usa `mail from` e `rcpt to` em letras minúsculas em **`smptlib.SMTP`**.

- gh-106186 [https://github.com/python/cpython/issues/106186]: Não reporta o defeito `MultipartInvariantViolationDefect` quando a classe **`email.parser.Parser`** for usada para analisar e-mails com `headersonly=True`.

- gh-105002 [https://github.com/python/cpython/issues/105002]: Corrige resultado inválido do método **`PurePath.relative_to()`** ao tentar percorrer um

segmento “. .” em *other* com *walk_up* habilitado. Uma exceção **`ValueError`** agora é levantada neste caso.

- gh-106739 [https://github.com/python/cpython/issues/106739]: Adiciona `rtype_cache` à mensagem de aviso (como um complemento ao tipo de objetos vazados e ao número de objetos vazados já incluídos na mensagem) para facilitar a depuração de objetos vazados quando o processo rastreador de recursos de multiprocessamento os encontra no desligamento. Isso ajuda a identificar mais rapidamente o que vazou e/ou por que o objeto vazado não foi limpo corretamente.

- gh-106751 [https://github.com/python/cpython/issues/106751]: Otimiza **`SelectSelector.select()`** para muitos casos de iteração. Patch por Donghee Na.

- gh-106751 [https://github.com/python/cpython/issues/106751]: Otimiza **`_PollLikeSelector.select()`** para o caso de muitas iterações.

- gh-106751 [https://github.com/python/cpython/issues/106751]: Otimiza **`KqueueSelector.select()`** para o caso de muitas iterações. Patch por Donghee Na.

- gh-106831 [https://github.com/python/cpython/issues/106831]: Corrige a possível ausência da verificação `NULL` do resultado `d2i_SSL_SESSION` em `_ssl.c`.

- gh-105481 [https://github.com/python/cpython/issues/105481]: As diversas listas de opcodes no módulo **`dis`** agora são geradas a partir de bytecodes.c em vez de construídas explicitamente em opcode.py.

- gh-106727 [https://github.com/python/cpython/issues/106727]: Torna **`inspect.getsource()`** mais inteligente para classes com definições de mesmo nome

- gh-106789 [https://github.com/python/cpython/issues/106789]: Remove importação de **`pprint`** de **`sysconfig`**.

- gh-105726 [https://github.com/python/cpython/issues/105726]: Adiciona `__slots__` a **`contextlib.AbstractContextManager`** e **`contextlib.AbstractAsyncContextManager`** para que classes filhas possam usar `__slots__`.

- gh-106774 [https://github.com/python/cpython/issues/106774]: Atualiza a cópia do pacote do pip para a versão 23.2.1.

- gh-106751 [https://github.com/python/cpython/issues/106751]: **`selectors`**: Otimiza o código `EpollSelector.select()` movendo parte do código para fora do laço.

- gh-106752 [https://github.com/python/cpython/issues/106752]: Corrigidos vários bugs em zipfile.Path, incluindo: em **`zipfile.Path.match()`**, os separadores do Windows não são mais respeitados (e nunca foram pensados para serem); Corrigidas as operações `name`/`suffix`/`suffixes`/`stem` quando nenhum nome de arquivo está presente e o Path não está na raiz do arquivo zip; Glob reformulado para melhor desempenho e comportamento de correspondência mais correto.

- gh-105293 [https://github.com/python/cpython/issues/105293]: Removida a chamada para `SSL_CTX_set_session_id_context` durante a criação de contexto do lado do cliente no módulo **`ssl`**.

- gh-106734 [https://github.com/python/cpython/issues/106734]: Desabilita o preenchimento automático com tabulação no modo multilinha de **`pdb`**

- gh-105481 [https://github.com/python/cpython/issues/105481]: Exibido metadados do opcode por meio de **`_opcode`**.

- gh-106670 [https://github.com/python/cpython/issues/106670]: Adiciona o novo comando `exceptions` ao depurador Pdb. Ele possibilita a navegação entre exceções encadeadas ao usar a depuração post mortem.

- gh-106602 [https://github.com/python/cpython/issues/106602]: Adiciona __copy__ e __deepcopy__ em **`enum`**

- gh-106664 [https://github.com/python/cpython/issues/106664]: **`selectors`**: Adiciona o método `_SelectorMapping.get()` e otimiza `_SelectorMapping.__getitem__()`.

- gh-106628 [https://github.com/python/cpython/issues/106628]: Acelera a análise de e-mails em cerca de 20%, eliminando a necessidade de compilar uma nova expressão regular para cada e-mail.

- gh-89427 [https://github.com/python/cpython/issues/89427]: Define a variável de ambiente `VIRTUAL_ENV_PROMPT` na ativação de **`venv`**, mesmo quando `VIRTUAL_ENV_DISABLE_PROMPT` estiver definido.

- gh-106530 [https://github.com/python/cpython/issues/106530]: Reverte uma alteração em **`colorsys.rgb_to_hls()`** que causava divisão por zero para certas entradas quase brancas. Patch por Terry Jan Reedy.

- gh-106584 [https://github.com/python/cpython/issues/106584]: Corrige o código de saída para `unittest` caso todos os testes sejam ignorados. Patch por Egor Eliseev.

- gh-106566 [https://github.com/python/cpython/issues/106566]: Otimiza `(?!)` (padrão que sempre falha) em expressões regulares.

- gh-106554 [https://github.com/python/cpython/issues/106554]: **`selectors`**: Reduz a sobrecarga do seletor usando `dict.get()` para procurar descritores de arquivo.

- gh-106558 [https://github.com/python/cpython/issues/106558]: Remove o ciclo de referência em chamadores de **`convert_to_error()`** excluindo `result` do escopo em um bloco `finally`.

- gh-100502 [https://github.com/python/cpython/issues/100502]: Adiciona o atributo de classe **`pathlib.PurePath.pathmod`** que armazena a implementação de **`os.path`** usado para operações de caminho de baixo nível: `posixpath` ou `ntpath`.

- gh-106527 [https://github.com/python/cpython/issues/106527]: Reduz a sobrecarga para adicionar e remover leitores e escritores **`asyncio`**.

- gh-106524 [https://github.com/python/cpython/issues/106524]: Corrige a falha em **`_sre.template()`** com modelos contendo índices de grupo inválidos.

- gh-106531 [https://github.com/python/cpython/issues/106531]: Removido `_legacy` e os nomes que ele fornecia de `importlib.resources`: `Resource`, `contents`, `is_resource`, `open_binary`, `open_text`, `path`, `read_binary` e `read_text`.

- gh-106052 [https://github.com/python/cpython/issues/106052]: Módulo **`re`**: corrige a correspondência de quantificadores possessivos no caso de um subpadrão contendo retrocesso.

- gh-106510 [https://github.com/python/cpython/issues/106510]: Melhora a saída de depuração para grupos atômicos em expressões regulares.

- gh-106503 [https://github.com/python/cpython/issues/106503]: Corrige o ciclo de referência em **`asyncio._SelectorSocketTransport`** removendo `_write_ready` em `close`.

- gh-105497 [https://github.com/python/cpython/issues/105497]: Corrige a inversão da máscara de sinalizador quando existem flags sem nome.

- gh-90876 [https://github.com/python/cpython/issues/90876]: Impede que **`multiprocessing.spawn`** falhe na *importação* em ambientes onde `sys.executable` é `None`. Isso regrediu na versão 3.11 com a adição do suporte a objetos caminho ou similares em multiprocessamento.

- gh-106403 [https://github.com/python/cpython/issues/106403]: Instâncias de **`typing.TypeVar`**, **`typing.ParamSpec`**, **`typing.ParamSpecArgs`**, **`typing.ParamSpecKwargs`** e **`typing.TypeVarTuple`** novamente oferecem suporte a referências fracas, corrigindo uma regressão introduzida no Python 3.12.0 beta 1. Patch por Jelle Zijlstra.

- gh-89812 [https://github.com/python/cpython/issues/89812]: Adiciona a classe privada `pathlib._PathBase`, que fornece suporte experimental para sistemas de arquivos virtuais e pode ser disponibilizada publicamente em uma versão futura do Python.

- gh-106292 [https://github.com/python/cpython/issues/106292]: Verifica se há um valor armazenado em cache do tipo dicionário instanciado no método **`__get__()`** de **`functools.cached_property()`**. Isso corresponde melhor ao comportamento anterior à versão 3.12 e melhora a compatibilidade para usuários que subclassificam **`functools.cached_property()`** e adicionam um método **`__set__()`**.

- gh-106350 [https://github.com/python/cpython/issues/106350]: Detecta possível falha de alocação de memória na função libtommath **`mp_init()`** usada pelo módulo `_tkinter`.

- gh-106330 [https://github.com/python/cpython/issues/106330]: Corrige a correspondência incorreta de caminhos vazios em **`pathlib.PurePath.match()`**. Este bug foi introduzido no Python 3.12.0 beta 1.

- gh-106309 [https://github.com/python/cpython/issues/106309]: Deprecate **`typing.no_type_check_decorator()`**. No major type checker ever added support for this decorator. Patch by Alex Waygood.

- gh-102541 [https://github.com/python/cpython/issues/102541]: Faz com que **`pydoc.doc()`** capture o módulo inválido **`ImportError`** quando o fluxo de saída não for `None`.

- gh-106263 [https://github.com/python/cpython/issues/106263]: Corrige a falha ao chamar `repr` com um objeto SignalDict construído manualmente. Patch por Charlie Zhao.

- gh-106236 [https://github.com/python/cpython/issues/106236]: Substitui instruções `assert` por `raise RuntimeError` em **`threading`**, para que `_DummyThread` não possa ser unido nem mesmo com `-OO`.

- gh-106238 [https://github.com/python/cpython/issues/106238]: Corrige bug raro de simultaneidade na aquisição de trava pelo pacote de registro.

- gh-106152 [https://github.com/python/cpython/issues/106152]: Added PY_THROW event hook for **`cProfile`** for generators

- gh-106075 [https://github.com/python/cpython/issues/106075]: Adicionado `asyncio.taskgroups.__all__` a `asyncio.__all__` para exportação em importações estrela.

- gh-104527 [https://github.com/python/cpython/issues/104527]: O Zipapp agora ignora a anexação de um arquivo a si mesmo.

- gh-106046 [https://github.com/python/cpython/issues/106046]: Melhoria na mensagem de erro de **`os.fspath()`** se chamada em um objeto onde `__fspath__` está definido como `None`. Patch por Alex Waygood.

- gh-105987 [https://github.com/python/cpython/issues/105987]: Corrige falha devido à contagem de referências incorreta nas rotinas internas ansiosas de fábrica de tarefas do do **`asyncio`**.

- gh-105974 [https://github.com/python/cpython/issues/105974]: Corrige bug em que uma classe **`typing.Protocol`** que tinha um ou mais membros não-chamáveis levantava **`TypeError`** quando **`issubclass()`** era chamada contra ela, mesmo se definisse um método `__subclasshook__` personalizado. O comportamento no Python 3.11 e versões anteriores — que agora foi restaurado — era não levantar **`TypeError`** nessas situações se um método `__subclasshook__` personalizado fosse definido. Patch por Alex Waygood.

- gh-96145 [https://github.com/python/cpython/issues/96145]: Revertida a adição de `json.AttrDict`.

- gh-89812 [https://github.com/python/cpython/issues/89812]: Adiciona **`pathlib.UnsupportedOperation`**, que é levantada em vez de **`NotImplementedError`** quando uma operação de caminho não é suportada.

- gh-105808 [https://github.com/python/cpython/issues/105808]: Corrigida uma regressão introduzida em gh-101251 [https://github.com/python/cpython/issues/101251] para a versão 3.12, que fazia com que **`gzip.GzipFile.flush()`** não descarregasse o compactador (nem passasse o argumento `zip_mode`).

- gh-105481 [https://github.com/python/cpython/issues/105481]: **`stack_effect()`** não levanta mais uma exceção se um `oparg` for fornecido para um `opcode` que não usa seu argumento, ou quando não for fornecido para um `opcode` que o utiliza. Neste último caso, o efeito de pilha é retornado para `oparg=0`.

- gh-104799 [https://github.com/python/cpython/issues/104799]: Habilita **`ast.unparse()`** para desfazer o analisador sintático de definições de função e classe criadas sem o novo campo `type_params` de **PEP 695** [https://peps.python.org/pep-0695/]. Patch por Jelle Zijlstra.

- gh-105793 [https://github.com/python/cpython/issues/105793]: Adiciona o argumento somente-nomeado *follow_symlinks* para **`pathlib.Path.is_dir()`** e **`is_file()`**, com o padrão `True`.

- gh-105570 [https://github.com/python/cpython/issues/105570]: Descontinua dois métodos de criação de classes **`typing.TypedDict`** com 0 campos usando a sintaxe funcional: `TD = TypedDict("TD")` e `TD = TypedDict("TD", None)`. Ambos serão descontinuados no Python 3.15. Para criar uma classe `TypedDict` com 0 campos, use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})`.

- gh-105745 [https://github.com/python/cpython/issues/105745]: Corrige o método `webbrowser.Konqueror.open`.

- gh-105733 [https://github.com/python/cpython/issues/105733]: **`ctypes`**: Descontinua as funções não documentadas **`ctypes.SetPointerType()`** e **`ctypes.ARRAY()`**. Patch por Victor Stinner.

- gh-105687 [https://github.com/python/cpython/issues/105687]: Remove as funções descontinuadas `re.template`, `re.T`, `re.TEMPLATE` e `sre_constans.SRE_FLAG_TEMPLATE`.

- gh-105684 [https://github.com/python/cpython/issues/105684]: O suporte a **`asyncio.Task.set_name()`** agora é obrigatório para implementações de tarefas de terceiros. A função não documentada **`_set_task_name()`** (descontinuada desde a versão 3.8) foi removida. Patch por Kumar Aditya.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige um bug em **`_Unpickler_SetInputStream()`**, onde uma exceção poderia ser sobrescrita em caso de falha.

- gh-105626 [https://github.com/python/cpython/issues/105626]: Altera o valor de retorno padrão de **`http.client.HTTPConnection.get_proxy_response_headers()`** para `None` e não `{}`.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`sys`**, onde exceções poderiam ser sobrescritas devido ao tratamento de erros adiado.

- gh-105605 [https://github.com/python/cpython/issues/105605]: Reforça o tratamento de erros em **`pyexpat`** durante a inicialização do módulo para evitar que exceções sejam sobrescritas e que objetos sejam desreferenciados duas vezes.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bug em **`decimal`**, onde uma exceção poderia acabar sendo sobrescrita.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`_datetime`**, onde exceções poderiam ser sobrescritas em caso de falha na inicialização do módulo.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs na inicialização de **`_ssl`**, que poderiam levar a referências vazadas e exceções sobrescritas.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige um bug em **`array.array`**, onde uma exceção poderia acabar sendo sobrescrita.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`_ctypes`**, onde exceções poderiam ser sobrescritas.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige um bug no módulo **`posix`**, onde uma exceção poderia ser sobrescrita.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`_elementtree`**, onde exceções poderiam ser sobrescritas.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`zoneinfo`**, onde exceções poderiam ser sobrescritas.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`errno`**, onde exceções poderiam ser sobrescritas.

- gh-105566 [https://github.com/python/cpython/issues/105566]: Descontinua a criação de uma classe **`typing.NamedTuple`** usando argumentos nomeados para denotar os campos (`NT = NamedTuple("NT", x=int, y=str)`). Isso não será permitido no Python 3.15. Use a sintaxe baseada em classe ou a sintaxe funcional.

  Dois métodos para criar classes `NamedTuple` com 0 campos usando a sintaxe funcional também estão descontinuados e não serão permitidos no Python 3.15: `NT = NamedTuple("NT")` e `NT = NamedTuple("NT", None)`. Para criar uma classe `NamedTuple` com 0 campos, use `class NT(NamedTuple): pass` ou `NT = NamedTuple("NT", [])`.

- gh-105545 [https://github.com/python/cpython/issues/105545]: Remove o atributo `webbrowser.MacOSXOSAScript._name`, que estava descontinuado na versão 3.11.

- gh-105497 [https://github.com/python/cpython/issues/105497]: Corrige a inversão de sinalizadores quando existem membros de apelido/máscara.

- gh-105509 [https://github.com/python/cpython/issues/105509]: **`typing.Annotated`** agora é implementado como uma instância de `typing._SpecialForm` em vez de uma classe. Isso não deve ter impacto direto para os usuários da API pública do módulo **`typing`**.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige bugs em **`pickle`**, onde exceções podiam ser sobrescritas.

- gh-70303 [https://github.com/python/cpython/issues/70303]: Emite **`FutureWarning`** de **`pathlib.Path.glob()`** e **`rglob()`** se o padrão fornecido terminar

com “`**`”. Em uma versão futura do Python, padrões com essa terminação corresponderão tanto a arquivos quanto a diretórios. Adicione uma barra final para corresponder apenas a diretórios.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Corrige um bug em **`sqlite3`** onde uma exceção poderia ser sobrescrita no retorno de chamada de **`collation`**.

- gh-105382 [https://github.com/python/cpython/issues/105382]: Remove os parâmetros *cafile*, *capath* e *cadefault* da função **`urllib.request.urlopen()`**, descontinuados no Python 3.6. Patch por Victor Stinner.

- gh-105376 [https://github.com/python/cpython/issues/105376]: **`logging`**: Remove os métodos `Logger.warn()` e `LoggerAdapter.warn()` não documentados e não testados, bem como a função `logging.warn()`. Descontinuados desde o Python 3.3, eles eram apelidos dos métodos **`logging.Logger.warning()`**, **`logging.LoggerAdapter.warning()`** e **`logging.warning()`**. Patch por Victor Stinner.

- gh-105332 [https://github.com/python/cpython/issues/105332]: Reverte o método de serialização com pickle de by-name para by-value.

- gh-104554 [https://github.com/python/cpython/issues/104554]: Adiciona suporte ao esquema RTSPS em urllib.parse

- gh-105292 [https://github.com/python/cpython/issues/105292]: Adiciona opção a **`traceback.format_exception_only()`** para recursivamente retornar à exceção aninhada de um **`BaseExceptionGroup`**.

- gh-105280 [https://github.com/python/cpython/issues/105280]: Corrige bug em que `isinstance([], collections.abc.Mapping)` poderia ser avaliado como `True` se a coleta de lixo ocorresse no momento errado. O bug foi causado por alterações na implementação de **`typing.Protocol`** no Python 3.12.

- gh-80480 [https://github.com/python/cpython/issues/80480]: **`array`**: Adiciona o typecode `'w'` que representa `Py_UCS4`.

- gh-105239 [https://github.com/python/cpython/issues/105239]: Corrige bug antigo em que `issubclass(object, typing.Protocol)` era avaliado como `True` em alguns casos extremos. Patch por Alex Waygood.

- gh-104310 [https://github.com/python/cpython/issues/104310]: Na versão beta 1, adicionamos uma função utilitária para autores de módulos de extensão, para usar ao testar seus módulos quanto ao suporte em múltiplos interpretadores ou sob uma GIL por interpretador. O nome dessa função mudou de `allowing_all_extensions` para `_incompatible_extension_module_restrictions`. O padrão para o argumento "disable_check" foi alterado de "True" para "False", para corresponder melhor ao novo nome da função.

- gh-105080 [https://github.com/python/cpython/issues/105080]: Assinatura inconsistente corrigida em classes derivadas para **`inspect.signature()`**

- gh-105144 [https://github.com/python/cpython/issues/105144]: Uma regressão recente foi corrigida no módulo **`typing`**. A regressão significava que executar `class Foo(X, typing.Protocol)`, onde `X` era uma classe que tinha **`abc.ABCMeta`** como metaclasse, faria com que chamadas subsequentes de `isinstance(1, X)` levantassem erroneamente **`TypeError`**. Patch por Alex Waygood.

- gh-62948 [https://github.com/python/cpython/issues/62948]: O finalizador **`io.IOBase`** agora registra os erros do método `close()` com **`sys.unraisablehook`**. Anteriormente, os erros eram ignorados silenciosamente por padrão e registrados apenas no Modo de Desenvolvimento Python ou no Python construído em modo de depuração. Patch por Victor Stinner.

- gh-105096 [https://github.com/python/cpython/issues/105096]: **`wave`**: Descontinua os métodos `getmark()`, `setmark()` e `getmarkers()` das classes **`wave.Wave_read`** e **`wave.Wave_write`**. Eles serão removidos no Python 3.15. Patch por Victor Stinner.

- gh-104992 [https://github.com/python/cpython/issues/104992]: Remove o método **`unittest.TestProgram.usageExit()`** não testado e não documentado,

descontinuado no Python 3.11. Patch por Hugo van Kemenade.

- gh-104996 [https://github.com/python/cpython/issues/104996]: Melhore o desempenho da inicialização de **`pathlib.PurePath`** adiando a junção de caminhos quando múltiplos argumentos são fornecidos.

- gh-101588 [https://github.com/python/cpython/issues/101588]: Descontinua o suporte não documentado a copy/deepcopy/pickle para itertools.

- gh-103631 [https://github.com/python/cpython/issues/103631]: Corrige `pathlib.PurePosixPath(pathlib.PureWindowsPath(...))` que não convertia separadores de caminho para restaurar o comportamento compatível com a versão 3.11.

- gh-104947 [https://github.com/python/cpython/issues/104947]: Torna as comparações entre objetos **`pathlib.PureWindowsPath`** consistentes no Windows e no Posix para corresponder ao comportamento da versão 3.11.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`audioop`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`aifc`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`uu`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104935 [https://github.com/python/cpython/issues/104935]: Corrige bugs na interação entre **`typing.runtime_checkable()`** e **`typing.Generic`** que foram introduzidos pela implementação **PEP 695** [https://peps.python.org/pep-0695/]. Patch por Jelle Zijlstra.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`crypt`** e sua extensão

privada `_crypt`, descontinuados no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo `nis`, obsoletos no Python 3.11. Patch por Victor Stinner.

- gh-104898 [https://github.com/python/cpython/issues/104898]: Adiciona `__slots__` ausente a `os.PathLike`.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo `xdrlib`, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo `nntplib`, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104886 [https://github.com/python/cpython/issues/104886]: Remove a classe não documentada `configparser.LegacyInterpolation`, descontinuada na documentação desde o Python 3.2 e com um aviso de descontinuação desde o Python 3.11. Patch por Hugo van Kemenade.

- gh-104786 [https://github.com/python/cpython/issues/104786]: Remove a criação de `typing.TypedDict` baseada em kwargs.

- gh-104876 [https://github.com/python/cpython/issues/104876]: Remove o método `turtle.RawTurtle.settiltangle()`, descontinuado na documentação desde o Python 3.1 e com um aviso de descontinuação desde o Python 3.11. Patch por Hugo van Kemenade.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Removido o pacote `msilib`, descontinuado no Python 3.11.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Removido o módulo `spwd`, descontinuado

no Python 3.11: o projeto python-pam [https://pypi.org/project/python-pam/] pode ser usado em seu lugar. Patch por Victor Stinner.

- gh-75552 [https://github.com/python/cpython/issues/75552]: Removido o módulo `tkinter.tix`, descontinuado desde o Python 3.6.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`chunk`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`mailcap`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`sunau`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104780 [https://github.com/python/cpython/issues/104780]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`ossaudiodev`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove o módulo **`pipes`**, descontinuado no Python 3.11. Patch por Victor Stinner.

- gh-104873 [https://github.com/python/cpython/issues/104873]: Adiciona **`typing.get_protocol_members()`** para retornar o conjunto de membros que definem um **`typing.Protocol`**. Adiciona **`typing.is_protocol()`** para verificar se uma classe é um **`typing.Protocol`**. Patch por Jelle Zijlstra.

- gh-104874 [https://github.com/python/cpython/issues/104874]: Documenta os atributos `__name__` e `__supertype__` de **`typing.NewType`**. Patch por Jelle Zijlstra.

- gh-104835 [https://github.com/python/cpython/issues/104835]: Removidas as seguintes funções **`unittest`**, descontinuadas no Python 3.11:

- **`unittest.findTestCases()`**
- **`unittest.makeSuite()`**
- **`unittest.getTestCaseNames()`**

Em vez delas, use os métodos de **`TestLoader`**:

- **`unittest.TestLoader.loadTestsFromModule()`**
- **`unittest.TestLoader.loadTestsFromTestCase()`**
- **`unittest.TestLoader.getTestCaseNames()`**

Patch por Hugo van Kemenade.

- gh-104804 [https://github.com/python/cpython/issues/104804]: Remove the untested and undocumented **`webbrowser`** **`MacOSX`** class, deprecated in Python 3.11. Patch by Hugo van Kemenade.

- gh-83863 [https://github.com/python/cpython/issues/83863]: Support for using **`pathlib.Path`** objects as context managers has been removed. Before Python 3.9, exiting the context manager marked a path as “closed”, which caused some (but not all!) methods to raise when called. Since Python 3.9, using a path as a context manager does nothing.

- gh-104799 [https://github.com/python/cpython/issues/104799]: Adjust the location of the (see **PEP 695** [https://peps.python.org/pep-0695/]) `type_params` field on **`ast.ClassDef`**, **`ast.AsyncFunctionDef`**, and **`ast.FunctionDef`** to better preserve backward compatibility. Patch by Jelle Zijlstra

- gh-104797 [https://github.com/python/cpython/issues/104797]: Allow **`typing.Protocol`** classes to inherit from **`collections.abc.Buffer`**. Patch by Jelle Zijlstra.

- gh-104783 [https://github.com/python/cpython/issues/104783]: Remove `locale.resetlocale()` function deprecated in Python 3.11. Patch by Victor Stinner.

- gh-104780 [https://github.com/python/cpython/issues/104780]: Remove the `2to3` program and the **`lib2to3`** module, deprecated in Python 3.11. Patch by Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove the **`telnetlib`** module, deprecated in Python 3.11. Patch by Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove the **`imghdr`** module, deprecated in Python 3.11. Patch by Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove the **`cgi`** and **`cgitb`** modules, deprecated in Python 3.11. Patch by Victor Stinner.

- gh-104773 [https://github.com/python/cpython/issues/104773]: **PEP 594** [https://peps.python.org/pep-0594/]: Remove the **`sndhdr`** module, deprecated in Python 3.11. Patch by Victor Stinner.

- gh-104372 [https://github.com/python/cpython/issues/104372]: On Linux where **`subprocess`** can use the `vfork()` syscall for faster spawning, prevent the parent process from blocking other threads by dropping the GIL while it waits for the vfork'ed child process `exec()` outcome. This prevents spawning a binary from a slow filesystem from blocking the rest of the application.

- gh-99108 [https://github.com/python/cpython/issues/99108]: We now release the GIL around built-in **`hashlib`** computations of reasonable size for the SHA families and MD5 hash functions, matching what our OpenSSL backed hash computations already does.

- gh-102613 [https://github.com/python/cpython/issues/102613]: Improve performance of **`pathlib.Path.glob()`** when expanding a pattern with a non-terminal "`**`" component by filtering walked paths through a regular expression, rather than calling **`os.scandir()`** more than once on each directory.

- gh-104399 [https://github.com/python/cpython/issues/104399]: Prepare the `_tkinter` module for building with Tcl 9.0 and future libtommath by replacing usage of deprecated functions **`mp_to_unsigned_bin_n()`** and **`mp_unsigned_bin_size()`** when necessary.

- gh-102676 [https://github.com/python/cpython/issues/102676]: Add fields `start_offset`, `cache_offset`, `end_offset`, `baseopname`, `baseopcode`, `jump_target` and `oparg` to **`dis.Instruction`**.

- gh-103558 [https://github.com/python/cpython/issues/103558]: Fixed `parent` argument validation mechanism of **`argparse`**. Improved test coverage.

- gh-103464 [https://github.com/python/cpython/issues/103464]: Provide helpful usage messages when parsing incorrect **`pdb`** commands.

- gh-103384 [https://github.com/python/cpython/issues/103384]: Generalize the regex pattern `BaseConfigurator.INDEX_PATTERN` to allow spaces and non-alphanumeric characters in keys.

- gh-103124 [https://github.com/python/cpython/issues/103124]: Added multiline statement support for **`pdb`**

- gh-101162 [https://github.com/python/cpython/issues/101162]: Forbid using **`builtins.issubclass()`** with **`types.GenericAlias`** as the first argument.

- gh-103200 [https://github.com/python/cpython/issues/103200]: Fix cache repopulation semantics of zipimport.invalidate_caches(). The cache is now repopulated upon retrieving files with an invalid cache, not when the cache is invalidated.

- gh-100061 [https://github.com/python/cpython/issues/100061]: Fix a bug that causes wrong matches for regular expressions with possessive qualifier.

- gh-77609 [https://github.com/python/cpython/issues/77609]: Add *follow_symlinks* argument to **`pathlib.Path.glob()`** and **`rglob()`**, defaulting to false.

- gh-102541 [https://github.com/python/cpython/issues/102541]: Hide traceback in **`help()`** prompt, when import failed.

- gh-102120 [https://github.com/python/cpython/issues/102120]: Added a stream mode to `tarfile` that allows for reading archives without caching info

about the inner files.

- gh-102029 [https://github.com/python/cpython/issues/102029]: Deprecate passing any arguments to **threading.RLock()**.

- gh-88233 [https://github.com/python/cpython/issues/88233]: Refactored `zipfile._strip_extra` to use higher level abstractions for extras instead of a heavy-state loop.

- gh-102024 [https://github.com/python/cpython/issues/102024]: Reduce calls of `_idle_semaphore.release()` in **concurrent.futures.thread._worker()**.

- gh-73435 [https://github.com/python/cpython/issues/73435]: Add support for recursive wildcards in **pathlib.PurePath.match()**.

- gh-84867 [https://github.com/python/cpython/issues/84867]: **unittest.TestLoader** no longer loads test cases from exact **unittest.TestCase** and **unittest.FunctionTestCase** classes.

- gh-99203 [https://github.com/python/cpython/issues/99203]: Restore following CPython <= 3.10.5 behavior of **shutil.make_archive()**: do not create an empty archive if `root_dir` is not a directory, and, in that case, raise **FileNotFoundError** or **NotADirectoryError** regardless of `format` choice. Beyond the brought-back behavior, the function may now also raise these exceptions in `dry_run` mode.

- gh-80480 [https://github.com/python/cpython/issues/80480]: Emit **DeprecationWarning** for **array**'s `'u'` type code, deprecated in docs since Python 3.3.

- gh-94924 [https://github.com/python/cpython/issues/94924]: **unittest.mock.create_autospec()** now properly returns coroutine functions compatible with **inspect.iscoroutinefunction()**

- gh-94777 [https://github.com/python/cpython/issues/94777]: Fix hanging **multiprocessing** `ProcessPoolExecutor` when a child process crashes while data is being written in the call queue.

- gh-92871 [https://github.com/python/cpython/issues/92871]: Remove the `typing.io` and `typing.re` namespaces, deprecated since Python 3.8. All items are still available from the main **`typing`** module.

- bpo-43633 [https://bugs.python.org/issue?@action=redirect&bpo=43633]: Improve the textual representation of IPv4-mapped IPv6 addresses (**RFC 4291** [https://datatracker.ietf.org/doc/html/rfc4291.html] Sections 2.2, 2.5.5.2) in **`ipaddress`**. Patch by Oleksandr Pavliuk.

- bpo-44850 [https://bugs.python.org/issue?@action=redirect&bpo=44850]: Improve performance of **`operator.methodcaller()`** using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` convention. Patch by Anthony Lee and Pieter Eendebak.

- bpo-44185 [https://bugs.python.org/issue?@action=redirect&bpo=44185]: **`unittest.mock.mock_open()`** will call the **`close()`** method of the file handle mock when it is exiting from the context manager. Patch by Samet Yaslan.

- bpo-40988 [https://bugs.python.org/issue?@action=redirect&bpo=40988]: Improve performance of **`functools.singledispatchmethod`** by caching the generated dispatch wrapper. Optimization suggested by frederico. Patch by @mental32, Alex Waygood and Pieter Eendebak.

- bpo-41768 [https://bugs.python.org/issue?@action=redirect&bpo=41768]: **`unittest.mock`** speccing no longer calls class properties. Patch by Melanie Witt.

- bpo-18319 [https://bugs.python.org/issue?@action=redirect&bpo=18319]: Ensure `gettext(msg)` retrieve translations even if a plural form exists. In other words: `gettext(msg) == ngettext(msg, '', 1)`.

- bpo-17013 [https://bugs.python.org/issue?@action=redirect&bpo=17013]: Add `ThreadingMock` to **`unittest.mock`** that can be used to create Mock objects that can wait until they are called. Patch by Karthikeyan Singaravelan and Mario Corchero.

Documentação

- gh-109209 [https://github.com/python/cpython/issues/109209]: The minimum Sphinx version required for the documentation is now 4.2.
- gh-108826 [https://github.com/python/cpython/issues/108826]: **`dis`** module command-line interface is now mentioned in documentation.
- gh-107305 [https://github.com/python/cpython/issues/107305]: Add documentation for **`PyInterpreterConfig`** and **`Py_NewInterpreterFromConfig()`**. Also clarify some of the nearby docs relative to per-interpreter GIL.
- gh-107008 [https://github.com/python/cpython/issues/107008]: Document the **`curses`** module variables **`LINES`** and **`COLS`**.
- gh-106948 [https://github.com/python/cpython/issues/106948]: Add a number of standard external names to `nitpick_ignore`.
- gh-106232 [https://github.com/python/cpython/issues/106232]: Make timeit doc command lines compatible with Windows by using double quotes for arguments. This works on linux and macOS also.
- gh-105172 [https://github.com/python/cpython/issues/105172]: Fixed **`functools.lru_cache()`** docstring accounting for `typed` argument's different handling of str and int. Patch by Bar Harel.
- gh-105052 [https://github.com/python/cpython/issues/105052]: Update `timeit` doc to specify that time in seconds is just the default.
- gh-89455 [https://github.com/python/cpython/issues/89455]: Add missing documentation for the `max_group_depth` and `max_group_width` parameters and the `exceptions` attribute of the **`traceback.TracebackException`** class.
- gh-89412 [https://github.com/python/cpython/issues/89412]: Add missing documentation for the `end_lineno` and `end_offset` attributes of the **`traceback.TracebackException`** class.
- gh-104943 [https://github.com/python/cpython/issues/104943]: Remove mentions of old Python versions in **`typing.NamedTuple`**.
- gh-54738 [https://github.com/python/cpython/issues/54738]: Add documentation on how to localize the **`argparse`** module.
- gh-102823 [https://github.com/python/cpython/issues/102823]: Document the return type of `x // y` when `x` and `y` have type **`float`**.
- gh-102759 [https://github.com/python/cpython/issues/102759]: Align function signature for `functools.reduce` in documentation and docstring with the C implementation.

Testes

- gh-110647 [https://github.com/python/cpython/issues/110647]: Fix test_stress_modifying_handlers() of test_signal. Patch by Victor Stinner.
- gh-103053 [https://github.com/python/cpython/issues/103053]: Fix test_tools.test_freeze on FreeBSD: run “make distclean” instead of “make clean” in the copied source directory to remove also the “python” program. Patch by Victor Stinner.
- gh-110167 [https://github.com/python/cpython/issues/110167]: Fix a deadlock in test_socket when server fails with a timeout but the client is still running in its thread. Don’t hold a lock to call cleanup functions in doCleanups(). One of the cleanup function waits until the client completes, whereas the client could deadlock if it called addCleanup() in such situation. Patch by Victor Stinner.
- gh-110388 [https://github.com/python/cpython/issues/110388]: Add tests for **`tty`**.
- gh-81002 [https://github.com/python/cpython/issues/81002]: Add tests for **`termios`**.
- gh-110367 [https://github.com/python/cpython/issues/110367]: regrtest: When using worker processes (-jN) with –verbose3 option, regrtest can now display the worker output even if a worker process does crash. Previously, sys.stdout and sys.stderr were replaced and so the worker output was lost on a crash. Patch by Victor Stinner.
- gh-110267 [https://github.com/python/cpython/issues/110267]: Add tests for pickling and copying PyStructSequence objects. Patched by Xuehai Pan.
- gh-110171 [https://github.com/python/cpython/issues/110171]: `libregrtest` now always sets and shows `random.seed`, so tests are more reproducible. Use `--randseed` flag to pass the explicit random seed for tests.
- gh-110152 [https://github.com/python/cpython/issues/110152]: Remove `Tools/scripts/run_tests.py` and `make hostrunnertest`. Just run `./python -m test --slow-ci`, `make buildbottest` or `make test` instead. Python test runner (regrtest) now handles cross-compilation and HOSTRUNNER. It also adds options to Python such fast `-u -E -W default -bb` when `--fast-ci` or `--slow-ci` option is used. Patch by Victor Stinner.
- gh-110031 [https://github.com/python/cpython/issues/110031]: Skip test_threading tests using thread+fork if Python is built with Address Sanitizer (ASAN). Patch by Victor Stinner.

- gh-110088 [https://github.com/python/cpython/issues/110088]: Fix test_asyncio timeouts: don't measure the maximum duration, a test should not measure a CI performance. Only measure the minimum duration when a task has a timeout or delay. Add `CLOCK_RES` to `test_asyncio.utils`. Patch by Victor Stinner.
- gh-109974 [https://github.com/python/cpython/issues/109974]: Fix race conditions in test_threading lock tests. Wait until a condition is met rather than using **`time.sleep()`** with a hardcoded number of seconds. Patch by Victor Stinner.
- gh-110033 [https://github.com/python/cpython/issues/110033]: Fix `test_interprocess_signal()` of `test_signal`. Make sure that the `subprocess.Popen` object is deleted before the test raising an exception in a signal handler. Otherwise, `Popen.__del__()` can get the exception which is logged as `Exception ignored in: ...` and the test fails. Patch by Victor Stinner.
- gh-109594 [https://github.com/python/cpython/issues/109594]: Fix test_timeout() of test_concurrent_futures.test_wait. Remove the future which may or may not complete depending if it takes longer than the timeout or not. Keep the second future which does not complete before wait() timeout. Patch by Victor Stinner.
- gh-109972 [https://github.com/python/cpython/issues/109972]: Split test_gdb.py file into a test_gdb package made of multiple tests, so tests can now be run in parallel. Patch by Victor Stinner.
- gh-109566 [https://github.com/python/cpython/issues/109566]: regrtest: When `--fast-ci` or `--slow-ci` option is used, regrtest now replaces the current process with a new process to add `-u -W default -bb -E` options to Python. Patch by Victor Stinner.
- gh-109748 [https://github.com/python/cpython/issues/109748]: Fix `test_zippath_from_non_installed_posix()` of test_venv: don't copy `__pycache__/` sub-directories, because they can be modified by other Python tests running in parallel. Patch by Victor Stinner.
- gh-109739 [https://github.com/python/cpython/issues/109739]: regrtest: Fix reference leak check on Windows. Disable the load tracker on Windows in the reference leak check mode (-R option). Patch by Victor Stinner.
- gh-109276 [https://github.com/python/cpython/issues/109276]: regrtest: When a test fails with "env changed" and the –rerun option is used, the test is

now re-run in verbose mode in a fresh process. Patch by Victor Stinner.

- gh-103053 [https://github.com/python/cpython/issues/103053]: Skip test_freeze_simple_script() of test_tools.test_freeze if Python is built with `./configure --enable-optimizations`, which means with Profile Guided Optimization (PGO): it just makes the test too slow. The freeze tool is tested by many other CIs with other (faster) compiler flags. Patch by Victor Stinner.
- gh-109580 [https://github.com/python/cpython/issues/109580]: Skip `test_perf_profiler` if Python is built with ASAN, MSAN or UBSAN sanitizer. Python does crash randomly in this test on such build. Patch by Victor Stinner.
- gh-109566 [https://github.com/python/cpython/issues/109566]: regrtest: Add `--fast-ci` and `--slow-ci` options. `--fast-ci` uses a default timeout of 10 minutes and `-u all,-cpu` (skip slowest tests). `--slow-ci` uses a default timeout of 20 minutes and `-u all` (run all tests). Patch by Victor Stinner.
- gh-109425 [https://github.com/python/cpython/issues/109425]: libregrtest now decodes stdout of test worker processes with the “backslashreplace” error handler to log corrupted stdout, instead of failing with an error and not logging the stdout. Patch by Victor Stinner.
- gh-109396 [https://github.com/python/cpython/issues/109396]: Fix `test_socket.test_hmac_sha1()` in FIPS mode. Use a longer key: FIPS mode requires at least of at least 112 bits. The previous key was only 32 bits. Patch by Victor Stinner.
- gh-104736 [https://github.com/python/cpython/issues/104736]: Fix test_gdb on Python built with LLVM clang 16 on Linux ppc64le (ex: Fedora 38). Search patterns in gdb “bt” command output to detect when gdb fails to retrieve the traceback. For example, skip a test if `Backtrace stopped: frame did not save the PC` is found. Patch by Victor Stinner.
- gh-109276 [https://github.com/python/cpython/issues/109276]: libregrtest now calls **`random.seed()`** before running each test file when `-r/--randomize` command line option is used. Moreover, it’s also called in worker processes. It should help to make tests more deterministic. Previously, it was only called once in the main process before running all test files and it was not called in worker processes. Patch by Victor Stinner.

- gh-109276 [https://github.com/python/cpython/issues/109276]: libregrtest now uses a separated file descriptor to write test result as JSON. Previously, if a test wrote debug messages late around the JSON, the main test process failed to parse JSON. Patch by Victor Stinner.
- gh-108996 [https://github.com/python/cpython/issues/108996]: Fix and enable `test_msvcrt`.
- gh-109237 [https://github.com/python/cpython/issues/109237]: Fix `test_site.test_underpth_basic()` when the working directory contains at least one non-ASCII character: encode the `._pth` file to UTF-8 and enable the UTF-8 Mode to use UTF-8 for the child process stdout. Patch by Victor Stinner.
- gh-109230 [https://github.com/python/cpython/issues/109230]: Fix `test_pyexpat.test_exception()`: it can now be run from a directory different than Python source code directory. Before, the test failed in this case. Skip the test if Modules/pyexpat.c source is not available. Skip also the test on Python implementations other than CPython. Patch by Victor Stinner.
- gh-108996 [https://github.com/python/cpython/issues/108996]: Add tests for `msvcrt`.
- gh-109015 [https://github.com/python/cpython/issues/109015]: Fix test_asyncio, test_imaplib and test_socket tests on FreeBSD if the TCP blackhole is enabled (`sysctl net.inet.tcp.blackhole`). Skip the few tests which failed with `ETIMEDOUT` which such non standard configuration. Currently, the FreeBSD GCP image enables TCP and UDP blackhole [https://reviews.freebsd.org/D41751] (`sysctl net.inet.tcp.blackhole=2` and `sysctl net.inet.udp.blackhole=1`). Patch by Victor Stinner.
- gh-91960 [https://github.com/python/cpython/issues/91960]: Skip `test_gdb` if gdb is unable to retrieve Python frame objects: if a frame is `<optimized out>`. When Python is built with “clang -Og”, gdb can fail to retrieve the *frame* parameter of `_PyEval_EvalFrameDefault()`. In this case, tests like `py_bt()` are likely to fail. Without getting access to Python frames, `python-gdb.py` is mostly clueless on retrieving the Python traceback. Moreover, `test_gdb` is no longer skipped on macOS if Python is built with Clang. Patch by Victor Stinner.
- gh-108962 [https://github.com/python/cpython/issues/108962]: Skip `test_tempfile.test_flags()` if `chflags()` fails with “OSError:

[Errno 45] Operation not supported" (ex: on FreeBSD 13). Patch by Victor Stinner.

- gh-91960 [https://github.com/python/cpython/issues/91960]: FreeBSD 13.2 CI coverage for pull requests is now provided by Cirrus-CI (a hosted CI service that supports Linux, macOS, Windows, and FreeBSD).
- gh-89392 [https://github.com/python/cpython/issues/89392]: Removed support of `test_main()` function in tests. They now always use normal unittest test runner.
- gh-108851 [https://github.com/python/cpython/issues/108851]: Fix `test_tomllib` recursion tests for WASI buildbots: reduce the recursion limit and compute the maximum nested array/dict depending on the current available recursion limit. Patch by Victor Stinner.
- gh-108851 [https://github.com/python/cpython/issues/108851]: Add `get_recursion_available()` and `get_recursion_depth()` functions to the **`test.support`** module. Patch by Victor Stinner.
- gh-108834 [https://github.com/python/cpython/issues/108834]: Add `--fail-rerun option` option to regrtest: if a test failed when then passed when rerun in verbose mode, exit the process with exit code 2 (error), instead of exit code 0 (success). Patch by Victor Stinner.
- gh-108834 [https://github.com/python/cpython/issues/108834]: Rename regrtest `--verbose2` option (`-w`) to `--rerun`. Keep `--verbose2` as a deprecated alias. Patch by Victor Stinner.
- gh-108834 [https://github.com/python/cpython/issues/108834]: When regrtest reruns failed tests in verbose mode (`./python -m test --rerun`), tests are now rerun in fresh worker processes rather than being executed in the main process. If a test does crash or is killed by a timeout, the main process can detect and handle the killed worker process. Tests are rerun in parallel if the `-jN` option is used to run tests in parallel. Patch by Victor Stinner.
- gh-108822 [https://github.com/python/cpython/issues/108822]: `regrtest` now computes statistics on all tests: successes, failures and skipped. `test_netrc`, `test_pep646_syntax` and `test_xml_etree` now return results in their `test_main()` function. Patch by Victor Stinner and Alex Waygood.
- gh-108794 [https://github.com/python/cpython/issues/108794]: The **`doctest.DocTestRunner.run()`** method now counts the number of

skipped tests. Add **`doctest.DocTestRunner.skips`** and **`doctest.TestResults.skipped`** attributes. Patch by Victor Stinner.

- gh-108388 [https://github.com/python/cpython/issues/108388]: Convert test_concurrent_futures to a package of 7 sub-tests. Patch by Victor Stinner.
- gh-108388 [https://github.com/python/cpython/issues/108388]: Split test_multiprocessing_fork, test_multiprocessing_forkserver and test_multiprocessing_spawn into test packages. Each package is made of 4 sub-tests: processes, threads, manager and misc. It allows running more tests in parallel and so reduce the total test duration. Patch by Victor Stinner.
- gh-105776 [https://github.com/python/cpython/issues/105776]: Fix test_cppext when the C compiler command `-std=c11` option: remove `-std=` options from the compiler command. Patch by Victor Stinner.
- gh-107652 [https://github.com/python/cpython/issues/107652]: Set up CIFuzz to run fuzz targets in GitHub Actions. Patch by Illia Volochii.
- gh-107237 [https://github.com/python/cpython/issues/107237]: `test_logging`: Fix `test_udp_reconnection()` by increasing the timeout from 100 ms to 5 minutes (LONG_TIMEOUT). Patch by Victor Stinner.
- gh-107178 [https://github.com/python/cpython/issues/107178]: Add the C API test for functions in the Mapping Protocol, the Sequence Protocol and some functions in the Object Protocol.
- gh-106714 [https://github.com/python/cpython/issues/106714]: test_capi: Fix test_no_FatalError_infinite_loop() to no longer write a coredump, by using test.support.SuppressCrashReport. Patch by Victor Stinner.
- gh-104090 [https://github.com/python/cpython/issues/104090]: Avoid creating a reference to the test object in **`collectedDurations()`**.
- gh-106752 [https://github.com/python/cpython/issues/106752]: Moved tests for `zipfile.Path` into `Lib/test/test_zipfile/_path`. Made `zipfile._path` a package.
- gh-106690 [https://github.com/python/cpython/issues/106690]: Add .coveragerc to cpython repository for use with coverage package.
- gh-101634 [https://github.com/python/cpython/issues/101634]: When running the Python test suite with `-jN` option, if a worker stdout cannot be decoded from the locale encoding report a failed testn so the exitcode is non-zero. Patch by Victor Stinner.

- gh-105084 [https://github.com/python/cpython/issues/105084]: When the Python build is configured `--with-wheel-pkg-dir`, tests requiring the `setuptools` and `wheel` wheels will search for the wheels in `WHEEL_PKG_DIR`.
- gh-81005 [https://github.com/python/cpython/issues/81005]: String tests are modified to reflect that `str` and `unicode` are merged in Python 3. Patch by Daniel Fortunov.
- gh-103186 [https://github.com/python/cpython/issues/103186]: Suppress and assert expected RuntimeWarnings in test_sys_settrace.py
- gh-69714 [https://github.com/python/cpython/issues/69714]: Add additional tests to **`calendar`** to achieve full test coverage.

Construção

- gh-103053 [https://github.com/python/cpython/issues/103053]: “make check-clean-src” now also checks if the “python” program is found in the source directory: fail with an error if it does exist. Patch by Victor Stinner.

- gh-109191 [https://github.com/python/cpython/issues/109191]: Fix compile error when building with recent versions of libedit.

- gh-110276 [https://github.com/python/cpython/issues/110276]: No longer ignore **`PROFILE_TASK`** failure silently: command used by Profile Guided Optimization (PGO). Patch by Victor Stinner.

- gh-109566 [https://github.com/python/cpython/issues/109566]: Remove `make testall` target: use `make buildbottest` instead. Patch by Victor Stinner.

- gh-109740 [https://github.com/python/cpython/issues/109740]: The experimental `--disable-gil` configure flag now includes “t” (for “threaded”) in extension ABI tags.

- gh-109054 [https://github.com/python/cpython/issues/109054]: Fix building the `_testcapi` extension on Linux AArch64 which requires linking to libatomic when `<cpython/pyatomic.h>` is used: the `_Py_atomic_or_uint64()` function requires libatomic `__atomic_fetch_or_8()` on this platform. The configure script now

checks if linking to libatomic is needed and generates a new LIBATOMIC variable used to build the _testcapi extension. Patch by Victor Stinner.

- gh-63760 [https://github.com/python/cpython/issues/63760]: Fix Solaris build: no longer redefine the `gethostname()` function. Solaris defines the function since 2005. Patch by Victor Stinner, original patch by Jakub Kulík.

- gh-108740 [https://github.com/python/cpython/issues/108740]: Fix a race condition in `make regen-all`. The `deepfreeze.c` source and files generated by Argument Clinic are now generated or updated before generating "global objects". Previously, some identifiers may miss depending on the order in which these files were generated. Patch by Victor Stinner.

- gh-108634 [https://github.com/python/cpython/issues/108634]: Python built with `configure` **`--with-trace-refs`** (tracing references) is now ABI compatible with Python release build and debug build. Patch by Victor Stinner.

- gh-85283 [https://github.com/python/cpython/issues/85283]: The `_stat` C extension is now built with the limited C API. Patch by Victor Stinner.

- gh-108447 [https://github.com/python/cpython/issues/108447]: Fix x86_64 GNU/Hurd build

- gh-107814 [https://github.com/python/cpython/issues/107814]: When calling `find_python.bat` with `-q` it did not properly silence the output of nuget. That is now fixed.

- gh-105481 [https://github.com/python/cpython/issues/105481]: Remove the make target `regen-opcode-targets`, merge its work into `regen-opcode` which repeats most of the calculation. This simplifies the code for the build and reduces code duplication.

- gh-106881 [https://github.com/python/cpython/issues/106881]: Check for `linux/limits.h` before including it in `Modules/posixmodule.c`.

- gh-95855 [https://github.com/python/cpython/issues/95855]: Refactor platform triplet detection code and add detection for MIPS soft float and musl libc.

- gh-106962 [https://github.com/python/cpython/issues/106962]: Detect MPI compilers in `configure`.

- gh-106118 [https://github.com/python/cpython/issues/106118]: Fix compilation for platforms without **`O_CLOEXEC`**. The issue was introduced with Python 3.12b1 in gh-103295 [https://github.com/python/cpython/issues/103295]. Patch by Erlend Aasland.

- gh-105875 [https://github.com/python/cpython/issues/105875]: SQLite 3.15.2 or newer is required to build the **`sqlite3`** extension module. Patch by Erlend Aasland.

- gh-90005 [https://github.com/python/cpython/issues/90005]: Fix a regression in `configure` where we could end up unintentionally linking with `libbsd`.

- gh-102404 [https://github.com/python/cpython/issues/102404]: Document how to perform a WASI build on Linux. Also add Tools/wasm/build_wasi.sh as a reference implementation of the docs.

- gh-89886 [https://github.com/python/cpython/issues/89886]: Autoconf 2.71 and aclocal 1.16.4 is now required to regenerate `configure`.

- gh-104692 [https://github.com/python/cpython/issues/104692]: Include `commoninstall` as a prerequisite for `bininstall`

  This ensures that `commoninstall` is completed before `bininstall` is started when parallel builds are used (`make -j install`), and so the `python3` symlink is only installed after all standard library modules are installed.

- gh-101538 [https://github.com/python/cpython/issues/101538]: Add experimental wasi-threads support. Patch by Takashi Yamamoto.

Windows

- gh-110437 [https://github.com/python/cpython/issues/110437]: Allows overriding the source of VC redistributables so that releases can be guaranteed to never downgrade between updates.
- gh-109286 [https://github.com/python/cpython/issues/109286]: Update Windows installer to use SQLite 3.43.1.
- gh-82367 [https://github.com/python/cpython/issues/82367]: **`os.path.realpath()`** now resolves MS-DOS style file names even if the file is not accessible. Patch by Moonsik Park.
- gh-109991 [https://github.com/python/cpython/issues/109991]: Update Windows build to use OpenSSL 3.0.11.
- gh-106242 [https://github.com/python/cpython/issues/106242]: Fixes **`realpath()`** to behave consistently when passed a path containing an embedded null character on Windows. In strict mode, it now raises **`OSError`** instead of the unexpected **`ValueError`**, and in non-strict mode will make the path absolute.
- gh-83180 [https://github.com/python/cpython/issues/83180]: Changes the Gerenciador de instalação do Python to prefer an active virtual environment when the launched script has a shebang line using a Unix-like virtual command, even if the command requests a specific version of Python.
- gh-106844 [https://github.com/python/cpython/issues/106844]: Fix integer overflow and truncating by the null character in **`_winapi.LCMapStringEx()`** which affects **`ntpath.normcase()`**.
- gh-105436 [https://github.com/python/cpython/issues/105436]: Ensure that an empty environment block is terminated by two null characters, as is required by Windows.
- gh-105146 [https://github.com/python/cpython/issues/105146]: Updated the links at the end of the installer to point to Discourse rather than the mailing lists.
- gh-103646 [https://github.com/python/cpython/issues/103646]: When installed from the Microsoft Store, `pip` no longer defaults to per-user installs. However, as the install directory is unwritable, it should automatically decide to do a per-user install anyway. This should resolve issues when `pip` is passed an option that conflicts with `--user`.

- gh-88745 [https://github.com/python/cpython/issues/88745]: Improve performance of `shutil.copy2()` by using the operating system's `CopyFile2` function. This may result in subtle changes to metadata copied along with some files, bringing them in line with normal OS behavior.
- gh-104820 [https://github.com/python/cpython/issues/104820]: Fixes `stat()` and related functions on file systems that do not support file ID requests. This includes FAT32 and exFAT.
- gh-104803 [https://github.com/python/cpython/issues/104803]: Add `os.path.isdevdrive()` to detect whether a path is on a Windows Dev Drive. Returns `False` on platforms that do not support Dev Drive, and is absent on non-Windows platforms.

macOS

- gh-109286 [https://github.com/python/cpython/issues/109286]: Update macOS installer to use SQLite 3.43.1.
- gh-109991 [https://github.com/python/cpython/issues/109991]: Atualiza o instalador do macOS para usar OpenSSL 3.0.11.
- gh-99079 [https://github.com/python/cpython/issues/99079]: Atualiza o instalador do macOS para usar OpenSSL 3.0.9.

IDLE

- gh-104719 [https://github.com/python/cpython/issues/104719]: Remove IDLE's modification of tokenize.tabsize and test other uses of tokenize data and methods.

Ferramentas/Demos

- gh-109991 [https://github.com/python/cpython/issues/109991]: Update GitHub CI workflows to use OpenSSL 3.0.11 and multissltests to use 1.1.1w, 3.0.11, and 3.1.3.
- gh-108494 [https://github.com/python/cpython/issues/108494]: Argument Clinic [https://devguide.python.org/development-tools/clinic/] now has a partial support of the Limited API: see documentation in the Python Developer's Guide [https://devguide.python.org/development-tools/clinic/#clinic-howto-limited-capi] Patch by Victor Stinner.

- gh-107704 [https://github.com/python/cpython/issues/107704]: It is now possible to deprecate passing keyword arguments for keyword-or-positional parameters with Argument Clinic, using the new `/ [from X.Y]` syntax. (To be read as *"positional-only from Python version X.Y"*.) See documentation in the Python Developer's Guide [https://devguide.python.org/development-tools/clinic/#clinic-howto-deprecate-keyword] for more information.
- gh-107880 [https://github.com/python/cpython/issues/107880]: Argument Clinic can now clone **`__init__()`** and **`__new__()`** methods.
- gh-104683 [https://github.com/python/cpython/issues/104683]: Add `--exclude` option to Argument Clinic CLI.
- gh-95065 [https://github.com/python/cpython/issues/95065]: Argument Clinic now supports overriding automatically generated signature by using directive `@text_signature`. See documentation in the Python Developer's Guide [https://devguide.python.org/development-tools/clinic/#clinic-howto-override-signature]
- gh-107609 [https://github.com/python/cpython/issues/107609]: Fix duplicate module check in Argument Clinic. Previously, a duplicate definition would incorrectly be silently accepted. Patch by Erlend E. Aasland.
- gh-107467 [https://github.com/python/cpython/issues/107467]: The Argument Clinic command-line tool now prints to stderr instead of stdout on failure.
- gh-106970 [https://github.com/python/cpython/issues/106970]: Fix bugs in the Argument Clinic `destination <name> clear` command; the destination buffers would never be cleared, and the `destination` directive parser would simply continue to the fault handler after processing the command. Patch by Erlend E. Aasland.
- gh-106706 [https://github.com/python/cpython/issues/106706]: Change bytecode syntax for families to remove redundant name matching pseudo syntax.
- gh-106359 [https://github.com/python/cpython/issues/106359]: Argument Clinic now explicitly forbids "kwarg splats" in function calls used as annotations.
- gh-103186 [https://github.com/python/cpython/issues/103186]: `freeze` now fetches `CONFIG_ARGS` from the original CPython instance the Makefile uses to call utility scripts. Patch by Ijtaba Hussain.

- gh-95065 [https://github.com/python/cpython/issues/95065]: It is now possible to deprecate passing parameters positionally with Argument Clinic, using the new `* [from X.Y]` syntax. (To be read as *"keyword-only from Python version X.Y"*.) See documentation in the Python Developer's Guide [https://devguide.python.org/development-tools/clinic/#clinic-howto-deprecate-positional] for more information. Patch by Erlend E. Aasland with help from Alex Waygood, Nikita Sobolev, and Serhiy Storchaka.

API C

- gh-85283 [https://github.com/python/cpython/issues/85283]: If the **`Py_LIMITED_API`** macro is defined, **`Py_BUILD_CORE`**, **`Py_BUILD_CORE_BUILTIN`** and **`Py_BUILD_CORE_MODULE`** macros are now undefined by `<Python.h>`. Patch by Victor Stinner.

- gh-110289 [https://github.com/python/cpython/issues/110289]: Add **`PyUnicode_EqualToUTF8AndSize()`** and **`PyUnicode_EqualToUTF8()`** functions.

- gh-110235 [https://github.com/python/cpython/issues/110235]: Raise **`TypeError`** for duplicate/unknown fields in `PyStructSequence` constructor. Patched by Xuehai Pan.

- gh-110014 [https://github.com/python/cpython/issues/110014]: Remove undocumented `PY_TIMEOUT_MAX` constant from the limited C API. Patch by Victor Stinner.

- gh-109521 [https://github.com/python/cpython/issues/109521]: **`PyImport_GetImporter()`** now sets RuntimeError if it fails to get **`sys.path_hooks`** or **`sys.path_importer_cache`** or they are not list and dict correspondingly. Previously it could return NULL without setting error in obscure cases, crash or raise SystemError if these attributes have wrong type.

- gh-108724 [https://github.com/python/cpython/issues/108724]: Add **`PyMutex`** internal-only lightweight locking API.

- gh-85283 [https://github.com/python/cpython/issues/85283]: Add **`PySys_AuditTuple()`** function: similar to **`PySys_Audit()`**, but pass event arguments as a Python **`tuple`** object. Patch by Victor Stinner.

- gh-108867 [https://github.com/python/cpython/issues/108867]: Add **`PyThreadState_GetUnchecked()`** function: similar to **`PyThreadState_Get()`**, but don't kill the process with a fatal error if it is NULL. The caller is responsible to check if the result is NULL. Previously, the function was private and known as `_PyThreadState_UncheckedGet()`. Patch by Victor Stinner.

- gh-108765 [https://github.com/python/cpython/issues/108765]: `Python.h` no longer includes the `<ctype.h>` standard header file. If needed, it should now be included explicitly. For example, it provides `isalpha()` and `tolower()` functions which are locale dependent. Python provides locale independent functions, like **`Py_ISALPHA()`** and **`Py_TOLOWER()`**. Patch by Victor Stinner.

- gh-108765 [https://github.com/python/cpython/issues/108765]: `Python.h` no longer includes the `<unistd.h>` standard header file. If needed, it should now be included explicitly. For example, it provides the functions: `close()`, `getpagesize()`, `getpid()` and `sysconf()`. Patch by Victor Stinner.

- gh-108765 [https://github.com/python/cpython/issues/108765]: `Python.h` no longer includes the `<ieeefp.h>` standard header. It was included for the `finite()` function which is now provided by the `<math.h>` header. It should now be included explicitly if needed. Remove also the `HAVE_IEEEFP_H` macro. Patch by Victor Stinner.

- gh-108765 [https://github.com/python/cpython/issues/108765]: `Python.h` no longer includes these standard header files: `<time.h>`, `<sys/select.h>` and `<sys/time.h>`. If needed, they should now be included explicitly. For example, `<time.h>` provides the `clock()` and `gmtime()` functions, `<sys/select.h>` provides the `select()` function, and `<sys/time.h>` provides the `futimes()`, `gettimeofday()` and `setitimer()` functions. Patch by Victor Stinner.

- gh-108511 [https://github.com/python/cpython/issues/108511]: Add functions `PyObject_HasAttrWithError()`, `PyObject_HasAttrStringWithError()`, `PyMapping_HasKeyWithError()` and `PyMapping_HasKeyStringWithError()`.

- gh-107073 [https://github.com/python/cpython/issues/107073]: Add `PyObject_VisitManagedDict()` and `PyObject_ClearManagedDict()` functions which must be called by the traverse and clear functions of a type using `Py_TPFLAGS_MANAGED_DICT` flag. Patch by Victor Stinner.

- gh-108634 [https://github.com/python/cpython/issues/108634]: Python built with `configure --with-trace-refs` (tracing references) now supports the Limited API. Patch by Victor Stinner.

- gh-108014 [https://github.com/python/cpython/issues/108014]: Add `PyLong_AsInt()` function: similar to `PyLong_AsLong()`, but store the result in a C int instead of a C long. Previously, it was known as the private function `_PyLong_AsInt()` (with an underscore prefix). Patch by Victor Stinner.

- gh-108314 [https://github.com/python/cpython/issues/108314]: Add `PyDict_ContainsString()` function: same as `PyDict_Contains()`, but *key* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*. Patch by Victor Stinner.

- gh-108337 [https://github.com/python/cpython/issues/108337]: Adiciona operações atômicas em tipos de dados adicionais em pyatomic.h.

- gh-108014 [https://github.com/python/cpython/issues/108014]: Add `Py_IsFinalizing()` function: check if the main Python interpreter is shutting down. Patch by Victor Stinner.

- gh-107916 [https://github.com/python/cpython/issues/107916]: C API functions `PyErr_SetFromErrnoWithFilename()`, `PyErr_SetExcFromWindowsErrWithFilename()` and `PyErr_SetFromWindowsErrWithFilename()` save now the error code before calling `PyUnicode_DecodeFSDefault()`.

- gh-107915 [https://github.com/python/cpython/issues/107915]: Such C API functions as `PyErr_SetString()`, `PyErr_Format()`, `PyErr_SetFromErrnoWithFilename()` and many others no longer crash or ignore errors if it failed to format the error message or decode the filename. Instead, they keep a corresponding error.

- gh-107810 [https://github.com/python/cpython/issues/107810]: Improve **`DeprecationWarning`** for uses of **`PyType_Spec`** with metaclasses that have custom `tp_new`.

- gh-107249 [https://github.com/python/cpython/issues/107249]: Implement the **`Py_UNUSED`** macro for Windows MSVC compiler. Patch by Victor Stinner.

- gh-107226 [https://github.com/python/cpython/issues/107226]: **`PyModule_AddObjectRef()`** is now only available in the limited API version 3.10 or later.

- gh-106320 [https://github.com/python/cpython/issues/106320]: Remove private `_PyUnicode_AsString()` alias to **`PyUnicode_AsUTF8()`**. It was kept for backward compatibility with Python 3.0 - 3.2. The **`PyUnicode_AsUTF8()`** is available since Python 3.3. The **`PyUnicode_AsUTF8String()`** function can be used to keep compatibility with Python 3.2 and older. Patch by Victor Stinner.

- gh-106572 [https://github.com/python/cpython/issues/106572]: Convert **`PyObject_DelAttr()`** and **`PyObject_DelAttrString()`** macros to functions. Patch by Victor Stinner.

- gh-106307 [https://github.com/python/cpython/issues/106307]: Add **`PyMapping_GetOptionalItem()`** function.

- gh-106521 [https://github.com/python/cpython/issues/106521]: Add **`PyObject_GetOptionalAttr()`** and **`PyObject_GetOptionalAttrString()`** functions.

- gh-106320 [https://github.com/python/cpython/issues/106320]: Remove `_PyInterpreterState_Get()` alias to **`PyInterpreterState_Get()`**

which was kept for backward compatibility with Python 3.8. Patch by Victor Stinner.

- gh-106316 [https://github.com/python/cpython/issues/106316]: Remove `cpython/pytime.h` header file: it only contained private functions. Patch by Victor Stinner.

- gh-106023 [https://github.com/python/cpython/issues/106023]: Remove private `_PyObject_FastCall()` function: use `PyObject_Vectorcall()` which is available since Python 3.8 (**PEP 590** [https://peps.python.org/pep-0590/]). Patch by Victor Stinner.

- gh-106168 [https://github.com/python/cpython/issues/106168]: If Python is built in debug mode or **`with assertions`**, **`PyTuple_SET_ITEM()`** and **`PyList_SET_ITEM()`** now check the index argument with an assertion. If the assertion fails, make sure that the size is set before. Patch by Victor Stinner.

- gh-106084 [https://github.com/python/cpython/issues/106084]: Remove the old aliases to functions calling functions which were kept for backward compatibility with Python 3.8 provisional API:

  - `_PyObject_CallMethodNoArgs()`: use `PyObject_CallMethodNoArgs()`
  - `_PyObject_CallMethodOneArg()`: use `PyObject_CallMethodOneArg()`
  - `_PyObject_CallOneArg()`: use `PyObject_CallOneArg()`
  - `_PyObject_FastCallDict()`: use `PyObject_VectorcallDict()`
  - `_PyObject_Vectorcall()`: use `PyObject_Vectorcall()`
  - `_PyObject_VectorcallMethod()`: use `PyObject_VectorcallMethod()`
  - `_PyVectorcall_Function()`: use `PyVectorcall_Function()`

  Just remove the underscore prefix to update your code. Patch by Victor Stinner.

- gh-106004 [https://github.com/python/cpython/issues/106004]: Adds **`PyDict_GetItemRef()`** and **`PyDict_GetItemStringRef()`** functions:

similar to **PyDict_GetItemWithError()** but returning a strong reference instead of a borrowed reference. Patch by Victor Stinner.

- gh-105927 [https://github.com/python/cpython/issues/105927]: Deprecate the **PyWeakref_GetObject()** and **PyWeakref_GET_OBJECT()** functions: use the new **PyWeakref_GetRef()** function instead. Patch by Victor Stinner.

- gh-105927 [https://github.com/python/cpython/issues/105927]: Add **PyWeakref_GetRef()** function: similar to **PyWeakref_GetObject()** but returns a strong reference, or `NULL` if the referent is no longer live. Patch by Victor Stinner.

- gh-105922 [https://github.com/python/cpython/issues/105922]: Add **PyImport_AddModuleRef()**: similar to **PyImport_AddModule()**, but return a strong reference instead of a borrowed reference. Patch by Victor Stinner.

- gh-105227 [https://github.com/python/cpython/issues/105227]: The new **PyType_GetDict()** provides the dictionary for the given type object that is normally exposed by `cls.__dict__`. Normally it's sufficient to use **tp_dict**, but for the static builtin types **tp_dict** is now always `NULL`. **PyType_GetDict()** provides the correct dict object instead.

- gh-105375 [https://github.com/python/cpython/issues/105375]: Fix a bug in **PyErr_WarnExplicit()** where an exception could end up being overwritten if the API failed internally.

- gh-105603 [https://github.com/python/cpython/issues/105603]: We've renamed the new (in 3.12) `PyInterpreterConfig.own_gil` to `PyInterpreterConfig.gil` and changed the meaning of the value from "bool" to an integer with supported values of `PyInterpreterConfig_DEFAULT_GIL`, `PyInterpreterConfig_SHARED_GIL`, and `PyInterpreterConfig_OWN_GIL`. The default is "shared".

- gh-105387 [https://github.com/python/cpython/issues/105387]: In the limited C API version 3.12, **Py_INCREF()** and **Py_DECREF()** functions are now

implemented as opaque function calls to hide implementation details. Patch by Victor Stinner.

- gh-105396 [https://github.com/python/cpython/issues/105396]: Deprecate the **`PyImport_ImportModuleNoBlock()`** function which is just an alias to **`PyImport_ImportModule()`** since Python 3.3. Patch by Victor Stinner.

- gh-103968 [https://github.com/python/cpython/issues/103968]: **`PyType_FromMetaclass()`** now allows metaclasses with `tp_new` set to `NULL`.

- gh-105268 [https://github.com/python/cpython/issues/105268]: Remove the old private, undocumented and untested `_PyGC_FINALIZED()` macro which was kept for backward compatibility with Python 3.8 and older. Patch by Victor Stinner.

- gh-105182 [https://github.com/python/cpython/issues/105182]: Remove `PyEval_AcquireLock()` and `PyEval_ReleaseLock()` functions, deprecated in Python 3.2. Patch by Victor Stinner.

- gh-105182 [https://github.com/python/cpython/issues/105182]: Remove `PyEval_InitThreads()` and `PyEval_ThreadsInitialized()` functions, deprecated in Python 3.9. Patch by Victor Stinner.

- gh-105145 [https://github.com/python/cpython/issues/105145]: Deprecate old Python initialization functions:

  - **`PySys_ResetWarnOptions()`**
  - **`Py_GetExecPrefix()`**
  - **`Py_GetPath()`**
  - **`Py_GetPrefix()`**
  - **`Py_GetProgramFullPath()`**
  - **`Py_GetProgramName()`**
  - **`Py_GetPythonHome()`**

  Patch por Victor Stinner.

- gh-85275 [https://github.com/python/cpython/issues/85275]: `PyObject_AsCharBuffer()`, `PyObject_AsReadBuffer()`, `PyObject_CheckReadBuffer()`, and `PyObject_AsWriteBuffer()` are removed. Please migrate to new buffer protocol; **`PyObject_GetBuffer()`** and **`PyBuffer_Release()`**.

- gh-105156 [https://github.com/python/cpython/issues/105156]: Deprecate the old `Py_UNICODE` and `PY_UNICODE_TYPE` types: use directly the **`wchar_t`** type instead. Since Python 3.3, `Py_UNICODE` and `PY_UNICODE_TYPE` are just aliases to **`wchar_t`**. Patch by Victor Stinner.

- gh-105145 [https://github.com/python/cpython/issues/105145]: Remove the following old functions to configure the Python initialization, deprecated in Python 3.11:

  - `PySys_AddWarnOptionUnicode()`
  - `PySys_AddWarnOption()`
  - `PySys_AddXOption()`
  - `PySys_HasWarnOptions()`
  - `PySys_SetArgvEx()`
  - `PySys_SetArgv()`
  - `PySys_SetPath()`
  - `Py_SetPath()`
  - `Py_SetProgramName()`
  - `Py_SetPythonHome()`
  - `Py_SetStandardStreamEncoding()`
  - `_Py_SetProgramFullPath()`

  Patch por Victor Stinner.

- gh-105107 [https://github.com/python/cpython/issues/105107]: Remove functions deprecated in Python 3.9.

  - `PyEval_CallObject()`, `PyEval_CallObjectWithKeywords()`: use **`PyObject_CallNoArgs()`** and **`PyObject_Call()`** (positional arguments must not be *NULL*) instead.
  - `PyEval_CallFunction()`: use **`PyObject_CallFunction()`**.
  - `PyEval_CallMethod()`: use **`PyObject_CallMethod()`**.

- `PyCFunction_Call()`: use **`PyObject_Call()`**.

Patch por Victor Stinner.

- gh-105115 [https://github.com/python/cpython/issues/105115]: `PyTypeObject.tp_bases` (and `tp_mro`) for builtin static types are now shared by all interpreters, whereas in 3.12-beta1 they were stored on `PyInterpreterState`. Also note that now the tuples are immortal objects.

- gh-105071 [https://github.com/python/cpython/issues/105071]: Add `PyUnstable_Exc_PrepReraiseStar` to the unstable C api to expose the implementation of **`except*`**.

- gh-104922 [https://github.com/python/cpython/issues/104922]: `PY_SSIZE_T_CLEAN` is no longer required to use `'#'` formats in APIs like **`PyArg_ParseTuple()`** and **`Py_BuildValue()`**. They uses `Py_ssize_t` for `'#'` regardless `PY_SSIZE_T_CLEAN`.

- gh-104584 [https://github.com/python/cpython/issues/104584]: Add an unstable C API for hooking in an optimizer. This is mainly internal, but marked “unstable” to allow third-party experimentation.

- gh-104668 [https://github.com/python/cpython/issues/104668]: Don’t call **`PyOS_InputHook`** or **`PyOS_ReadlineFunctionPointer`** in subinterpreters, since it’s generally difficult to avoid using global state in their registered callbacks. This also avoids situations where extensions may find themselves running in a subinterpreter they don’t support (or haven’t yet been loaded in).

- bpo-42327 [https://bugs.python.org/issue?@action=redirect&bpo=42327]: Add **`PyModule_Add()`** function: similar to **`PyModule_AddObjectRef()`** and **`PyModule_AddObject()`**, but always steals a reference to the value.

- bpo-40309 [https://bugs.python.org/issue?@action=redirect&bpo=40309]: Properly handle trailing spaces before closing parenthesis in **`Py_BuildValue()`** format strings.

Python 3.12.0 beta 1

Data de lançamento: 2023-05-22

Segurança

- gh-99889 [https://github.com/python/cpython/issues/99889]: Fixed a security in flaw in **`uu.decode()`** that could allow for directory traversal based on the input if no `out_file` was specified.
- gh-104049 [https://github.com/python/cpython/issues/104049]: Do not expose the local on-disk location in directory indexes produced by **`http.client.SimpleHTTPRequestHandler`**.
- gh-99108 [https://github.com/python/cpython/issues/99108]: Upgrade built-in **`hashlib`** SHA3 implementation to a verified implementation from the `HACL*` project. Used when OpenSSL is not present or lacks SHA3.
- gh-102153 [https://github.com/python/cpython/issues/102153]: **`urllib.parse.urlsplit()`** now strips leading C0 control and space characters following the specification for URLs defined by WHATWG in response to **CVE 2023-24329** [https://www.cve.org/CVERecord?id=CVE-2023-24329]. Patch by Illia Volochii.

Biblioteca

- gh-102856 [https://github.com/python/cpython/issues/102856]: Implement PEP 701 changes in the **`tokenize`** module. Patch by Marta Gómez Macías and Pablo Galindo Salgado

Núcleo e embutidos

- gh-104615 [https://github.com/python/cpython/issues/104615]: Fix wrong ordering of assignments in code like `a, a = x, y`. Contributed by Carl Meyer.

- gh-104572 [https://github.com/python/cpython/issues/104572]: Improve syntax error message for invalid constructs in **PEP 695** [https://peps.python.org/pep-0695/] contexts and in annotations when `from __future__ import annotations` is active.

- gh-104482 [https://github.com/python/cpython/issues/104482]: Fix three error handling bugs in ast.c's validation of pattern matching statements.

- gh-102818 [https://github.com/python/cpython/issues/102818]: Do not add a frame to the traceback in the `sys.setprofile` and `sys.settrace` trampoline functions. This ensures that frames are not duplicated if an exception is raised in the callback function, and ensures that frames are not omitted if a C callback is used and that does not add the frame.

- gh-104405 [https://github.com/python/cpython/issues/104405]: Fix an issue where some bytecode instructions could ignore **PEP 523** [https://peps.python.org/pep-0523/] when "inlining" calls.

- gh-103082 [https://github.com/python/cpython/issues/103082]: Change behavior of `sys.monitoring.events.LINE` events in `sys.monitoring`: Line events now occur when a new line is reached dynamically, instead of using a static approximation, as before. This makes the behavior very similar to that of "line" events in `sys.settrace`. This should ease porting of tools from 3.11 to 3.12.

- gh-104263 [https://github.com/python/cpython/issues/104263]: Fix `float("nan")` to produce a quiet NaN on platforms (like MIPS) where the meaning of the signalling / quiet bit is inverted from its usual meaning. Also introduce a new macro `Py_INFINITY` matching C99's `INFINITY`, and refactor internals to rely on C99's `NAN` and `INFINITY` macros instead of hard-coding bit patterns for infinities and NaNs. Thanks Sebastian Berg.

- gh-99113 [https://github.com/python/cpython/issues/99113]: Multi-phase init extension modules may now indicate that they support running in subinterpreters that have their own GIL. This is done by using `Py_MOD_PER_INTERPRETER_GIL_SUPPORTED` as the value for the `Py_mod_multiple_interpreters` module def slot. Otherwise the module, by default, cannot be imported in such subinterpreters. (This does not affect the main interpreter or subinterpreters that do not have their own GIL.) In addition to the isolation that multi-phase init already normally requires, support for per-interpreter GIL involves one additional constraint: thread-safety. If the module has external (linked)

dependencies and those libraries have any state that isn't thread-safe then the module must do the additional work to add thread-safety. This should be an uncommon case.

- gh-99113 [https://github.com/python/cpython/issues/99113]: The GIL is now (optionally) per-interpreter. This is the fundamental change for PEP 684. This is all made possible by virtue of the isolated state of each interpreter in the process. The behavior of the main interpreter remains unchanged. Likewise, interpreters created using `Py_NewInterpreter()` are not affected. To get an interpreter with its own GIL, call `Py_NewInterpreterFromConfig()`.

- gh-104108 [https://github.com/python/cpython/issues/104108]: Multi-phase init extension modules may now indicate whether or not they actually support multiple interpreters. By default such modules are expected to support use in multiple interpreters. In the uncommon case that one does not, it may use the new `Py_mod_multiple_interpreters` module def slot. A value of `0` means the module does not support them. `1` means it does. The default is `1`.

- gh-104142 [https://github.com/python/cpython/issues/104142]: Fix an issue where **`list`** or **`tuple`** repetition could fail to respect **PEP 683** [https://peps.python.org/pep-0683/].

- gh-104078 [https://github.com/python/cpython/issues/104078]: Improve the performance of **`PyObject_HasAttrString()`**

- gh-104066 [https://github.com/python/cpython/issues/104066]: Improve the performance of **`hasattr()`** for module objects with a missing attribute.

- gh-104028 [https://github.com/python/cpython/issues/104028]: Reduce object creation while calling callback function from gc. Patch by Donghee Na.

- gh-104018 [https://github.com/python/cpython/issues/104018]: Disallow the "z" format specifier in %-format of bytes objects.

- gh-102213 [https://github.com/python/cpython/issues/102213]: Fix performance loss when accessing an object's attributes with `__getattr__` defined.

- gh-103895 [https://github.com/python/cpython/issues/103895]: Improve handling of edge cases in showing `Exception.__notes__`. Ensures that the messages always end with a newline and that string/bytes are not exploded over multiple lines. Patch by Carey Metcalfe.

- gh-103907 [https://github.com/python/cpython/issues/103907]: Don't modify the refcounts of known immortal objects (**`True`**, **`False`**, and **`None`**) in the main interpreter loop.

- gh-103899 [https://github.com/python/cpython/issues/103899]: Provide a helpful hint in the **`TypeError`** message when accidentally calling a module object that has a callable attribute of the same name (such as **`dis.dis()`** or **`datetime.datetime`**).

- gh-103845 [https://github.com/python/cpython/issues/103845]: Remove both line and instruction instrumentation before adding new ones for monitoring, to avoid newly added instrumentation being removed immediately.

- gh-103763 [https://github.com/python/cpython/issues/103763]: Implement **PEP 695** [https://peps.python.org/pep-0695/], adding syntactic support for generic classes, generic functions, and type aliases.

  A new `type X = ...` syntax is added for type aliases, which resolves at runtime to an instance of the new class `typing.TypeAliasType`. The value is lazily evaluated and is accessible through the `.__value__` attribute. This is implemented as a new AST node `ast.TypeAlias`.

  New syntax (`class X[T]: ...`, `def func[T]():  ...`) is added for defining generic functions and classes. This is implemented as a new `type_params` attribute on the AST nodes for classes and functions. This node holds instances of the new AST classes `ast.TypeVar`, `ast.ParamSpec`, and `ast.TypeVarTuple`.

`typing.TypeVar`, `typing.ParamSpec`, `typing.ParamSpecArgs`, `typing.ParamSpecKwargs`, `typing.TypeVarTuple`, and `typing.Generic` are now implemented in C rather than Python.

There are new bytecode instructions `LOAD_LOCALS`, `LOAD_CLASSDICT_OR_GLOBAL`, and `LOAD_CLASSDICT_OR_DEREF` to support correct resolution of names in class namespaces.

Patch by Eric Traut, Larry Hastings, and Jelle Zijlstra.

Biblioteca

- gh-103801 [https://github.com/python/cpython/issues/103801]: Adds three minor linting fixes to the wasm module caught that were caught by ruff.
- gh-103793 [https://github.com/python/cpython/issues/103793]: Optimized asyncio Task creation by deferring expensive string formatting (task name generation) from Task creation to the first time `get_name` is called. This makes asyncio benchmarks up to 5% faster.

Núcleo e embutidos

- gh-102310 [https://github.com/python/cpython/issues/102310]: Change the error range for invalid bytes literals.
- gh-103590 [https://github.com/python/cpython/issues/103590]: Do not wrap a single exception raised from a `try-except*` construct in an **`ExceptionGroup`**.
- gh-103650 [https://github.com/python/cpython/issues/103650]: Change the perf map format to remove the ‘0x’ prefix from the addresses
- gh-102856 [https://github.com/python/cpython/issues/102856]: Implement the required C tokenizer changes for PEP 701. Patch by Pablo Galindo Salgado, Lysandros Nikolaou, Batuhan Taskaya, Marta Gómez Macías and sunmy2019.
- gh-100530 [https://github.com/python/cpython/issues/100530]: Clarify the error message raised when the called part of a class pattern isn’t actually a class.
- gh-101517 [https://github.com/python/cpython/issues/101517]: Fix bug in line numbers of instructions emitted for **`except*`**.

- gh-103492 [https://github.com/python/cpython/issues/103492]: Clarify **`SyntaxWarning`** with literal `is` comparison by specifying which literal is problematic, since comparisons using `is` with e.g. `None` and bool literals are idiomatic.
- gh-87729 [https://github.com/python/cpython/issues/87729]: Add **`LOAD_SUPER_ATTR`** (and a specialization for `super().method()`) to speed up `super().method()` and `super().attr`. This makes `super().method()` roughly 2.3x faster and brings it within 20% of the performance of a simple method call. Patch by Vladimir Matveev and Carl Meyer.
- gh-103488 [https://github.com/python/cpython/issues/103488]: Change the internal offset distinguishing yield and return target addresses, so that the instruction pointer is correct for exception handling and other stack unwinding.
- gh-82012 [https://github.com/python/cpython/issues/82012]: The bitwise inversion operator (~) on bool is deprecated. It returns the bitwise inversion of the underlying `int` representation such that `bool(~True) == True`, which can be confusing. Use `not` for logical negation of bools. In the rare case that you really need the bitwise inversion of the underlying `int`, convert to int explicitly `~int(x)`.
- gh-77757 [https://github.com/python/cpython/issues/77757]: Exceptions raised in a typeobject's `__set_name__` method are no longer wrapped by a **`RuntimeError`**. Context information is added to the exception as a **PEP 678** [https://peps.python.org/pep-0678/] note.
- gh-103333 [https://github.com/python/cpython/issues/103333]: **`AttributeError`** now retains the `name` attribute when pickled and unpickled.
- gh-103242 [https://github.com/python/cpython/issues/103242]: Migrate **`set_ecdh_curve()`** method not to use deprecated OpenSSL APIs. Patch by Donghee Na.
- gh-103323 [https://github.com/python/cpython/issues/103323]: We've replaced our use of `_PyRuntime.tstate_current` with a thread-local variable. This is a fairly low-level implementation detail, and there should be no change in behavior.
- gh-84436 [https://github.com/python/cpython/issues/84436]: The implementation of PEP-683 which adds Immortal Objects by using a fixed reference count that skips reference counting to make objects truly immutable.

- gh-102700 [https://github.com/python/cpython/issues/102700]: Allow built-in modules to be submodules. This allows submodules to be statically linked into a CPython binary.
- gh-103082 [https://github.com/python/cpython/issues/103082]: Implement **PEP 669** [https://peps.python.org/pep-0669/] Low Impact Monitoring for CPython.
- gh-88691 [https://github.com/python/cpython/issues/88691]: Reduce the number of inline `CACHE` entries for `CALL`.
- gh-102500 [https://github.com/python/cpython/issues/102500]: Make the buffer protocol accessible in Python code using the new `__buffer__` and `__release_buffer__` magic methods. See **PEP 688** [https://peps.python.org/pep-0688/] for details. Patch by Jelle Zijlstra.
- gh-97933 [https://github.com/python/cpython/issues/97933]: **PEP 709** [https://peps.python.org/pep-0709/]: inline list, dict and set comprehensions to improve performance and reduce bytecode size.
- gh-99184 [https://github.com/python/cpython/issues/99184]: Bypass instance attribute access of `__name__` in `repr` of `weakref.ref`.
- gh-98003 [https://github.com/python/cpython/issues/98003]: Complex function calls are now faster and consume no C stack space.
- bpo-39610 [https://bugs.python.org/issue?@action=redirect&bpo=39610]: `len()` for 0-dimensional `memoryview` objects (such as `memoryview(ctypes.c_uint8(42))`) now raises a `TypeError`. Previously this returned `1`, which was not consistent with `mem_0d[0]` raising an `IndexError`.

Biblioteca

- bpo-31821 [https://bugs.python.org/issue?@action=redirect&bpo=31821]: Fix **`pause_reading()`** to work when called from **`connection_made()`** in `asyncio`.
- gh-104600 [https://github.com/python/cpython/issues/104600]: `functools.update_wrapper()` now sets the `__type_params__` attribute (added by **PEP 695** [https://peps.python.org/pep-0695/]).
- gh-104340 [https://github.com/python/cpython/issues/104340]: When an `asyncio` pipe protocol loses its connection due to an error, and the caller doesn't await `wait_closed()` on the corresponding `StreamWriter`, don't log a warning about an exception that was never retrieved. After all,

according to the `StreamWriter.close()` docs, the `wait_closed()` call is optional ("not mandatory").

- gh-104555 [https://github.com/python/cpython/issues/104555]: Fix issue where an **`issubclass()`** check comparing a class `X` against a **`runtime-checkable protocol`** `Y` with non-callable members would not cause **`TypeError`** to be raised if an **`isinstance()`** call had previously been made comparing an instance of `X` to `Y`. This issue was present in edge cases on Python 3.11, but became more prominent in 3.12 due to some unrelated changes that were made to runtime-checkable protocols. Patch by Alex Waygood.
- gh-104372 [https://github.com/python/cpython/issues/104372]: Refactored the `_posixsubprocess` internals to avoid Python C API usage between fork and exec when marking `pass_fds=` file descriptors inheritable.
- gh-104484 [https://github.com/python/cpython/issues/104484]: Added *case_sensitive* argument to **`pathlib.PurePath.match()`**
- gh-75367 [https://github.com/python/cpython/issues/75367]: Fix data descriptor detection in **`inspect.getattr_static()`**.
- gh-104536 [https://github.com/python/cpython/issues/104536]: Fix a race condition in the internal **`multiprocessing.process`** cleanup logic that could manifest as an unintended `AttributeError` when calling `process.close()`.
- gh-103857 [https://github.com/python/cpython/issues/103857]: Update datetime deprecations' stracktrace to point to the calling line
- gh-101520 [https://github.com/python/cpython/issues/101520]: Move the core functionality of the `tracemalloc` module in the `Python/` folder, leaving just the module wrapper in `Modules/`.
- gh-104392 [https://github.com/python/cpython/issues/104392]: Remove undocumented and unused `_paramspec_tvars` attribute from some classes in **`typing`**.
- gh-102613 [https://github.com/python/cpython/issues/102613]: Fix issue where **`pathlib.Path.glob()`** raised **`RecursionError`** when walking deep directory trees.
- gh-103000 [https://github.com/python/cpython/issues/103000]: Improve performance of **`dataclasses.asdict()`** for the common case where *dict_factory* is `dict`. Patch by David C Ellis.
- gh-104301 [https://github.com/python/cpython/issues/104301]: Allow leading whitespace in disambiguated statements in **`pdb`**.

- gh-104139 [https://github.com/python/cpython/issues/104139]: Teach **`urllib.parse.unsplit()`** to retain the `"//"` when assembling `itms-services://?action=generate-bugs` style Apple Platform Deployment [https://support.apple.com/en-gb/guide/deployment/depce7cefc4d/web] URLs.
- gh-104307 [https://github.com/python/cpython/issues/104307]: **`socket.getnameinfo()`** now releases the GIL while contacting the DNS server
- gh-104310 [https://github.com/python/cpython/issues/104310]: Users may now use `importlib.util.allowing_all_extensions()` (a context manager) to temporarily disable the strict compatibility checks for importing extension modules in subinterpreters.
- gh-87695 [https://github.com/python/cpython/issues/87695]: Fix issue where **`pathlib.Path.glob()`** raised **`OSError`** when it encountered a symlink to an overly long path.
- gh-104265 [https://github.com/python/cpython/issues/104265]: Prevent possible crash by disallowing instantiation of the **`_csv.Reader`** and **`_csv.Writer`** types. The regression was introduced in 3.10.0a4 with PR 23224 (bpo-14935 [https://bugs.python.org/issue?@action=redirect&bpo=14935]). Patch by Radislav Chugunov.
- gh-102613 [https://github.com/python/cpython/issues/102613]: Improve performance of **`pathlib.Path.glob()`** when expanding recursive wildcards (”`**`”) by merging adjacent wildcards and de-duplicating results only when necessary.
- gh-65772 [https://github.com/python/cpython/issues/65772]: Remove unneeded comments and code in turtle.py.
- gh-90208 [https://github.com/python/cpython/issues/90208]: Fixed issue where **`pathlib.Path.glob()`** returned incomplete results when it encountered a **`PermissionError`**. This method now suppresses all **`OSError`** exceptions, except those raised from calling **`is_dir()`** on the top-level path.
- gh-104144 [https://github.com/python/cpython/issues/104144]: Optimize **`asyncio.TaskGroup`** when using **`asyncio.eager_task_factory()`**. Skip scheduling a done callback if a TaskGroup task completes eagerly.
- gh-104144 [https://github.com/python/cpython/issues/104144]: Optimize **`asyncio.gather()`** when using **`asyncio.eager_task_factory()`** to

complete eagerly if all fututres completed eagerly. Avoid scheduling done callbacks for futures that complete eagerly.

- gh-104114 [https://github.com/python/cpython/issues/104114]: Fix issue where **`pathlib.Path.glob()`** returns paths using the case of non-wildcard segments for corresponding path segments, rather than the real filesystem case.
- gh-104104 [https://github.com/python/cpython/issues/104104]: Improve performance of **`pathlib.Path.glob()`** by using **`re.IGNORECASE`** to implement case-insensitive matching.
- gh-104102 [https://github.com/python/cpython/issues/104102]: Improve performance of **`pathlib.Path.glob()`** when evaluating patterns that contain `'../'` segments.
- gh-103822 [https://github.com/python/cpython/issues/103822]: Update the return type of `weekday` to the newly added Day attribute
- gh-103629 [https://github.com/python/cpython/issues/103629]: Update the `repr` of **`typing.Unpack`** according to **PEP 692** [https://peps.python.org/pep-0692/].
- gh-103963 [https://github.com/python/cpython/issues/103963]: Make **`dis`** display the names of the args for **`CALL_INTRINSIC_*`**.
- gh-104035 [https://github.com/python/cpython/issues/104035]: Do not ignore user-defined `__getstate__` and `__setstate__` methods for slotted frozen dataclasses.
- gh-103987 [https://github.com/python/cpython/issues/103987]: In **`mmap`**, fix several bugs that could lead to access to memory-mapped files after they have been invalidated.
- gh-103977 [https://github.com/python/cpython/issues/103977]: Improve import time of **`platform`** module.
- gh-88773 [https://github.com/python/cpython/issues/88773]: Added **`turtle.teleport()`** to the **`turtle`** module to move a turtle to a new point without tracing a line, visible or invisible. Patch by Liam Gersten.
- gh-103935 [https://github.com/python/cpython/issues/103935]: Use **`io.open_code()`** for files to be executed instead of raw **`open()`**
- gh-68968 [https://github.com/python/cpython/issues/68968]: Fixed garbled output of **`assertEqual()`** when an input lacks final newline.
- gh-100370 [https://github.com/python/cpython/issues/100370]: Fix potential **`OverflowError`** in **`sqlite3.Connection.blobopen()`** for 32-bit builds. Patch by Erlend E. Aasland.

- gh-102628 [https://github.com/python/cpython/issues/102628]: Substitute CTRL-D with CTRL-Z in `sqlite3` CLI banner when running on Windows.
- gh-103636 [https://github.com/python/cpython/issues/103636]: Module-level attributes `January` and `February` are deprecated from `calendar`.
- gh-103583 [https://github.com/python/cpython/issues/103583]: Isolate `_multibytecodec` and codecs extension modules. Patches by Erlend E. Aasland.
- gh-103848 [https://github.com/python/cpython/issues/103848]: Add checks to ensure that `[` bracketed `]` hosts found by `urllib.parse.urlsplit()` are of IPv6 or IPvFuture format.
- gh-103872 [https://github.com/python/cpython/issues/103872]: Update the bundled copy of pip to version 23.1.2.
- gh-99944 [https://github.com/python/cpython/issues/99944]: Make `dis` display the value of oparg of `KW_NAMES`.
- gh-74940 [https://github.com/python/cpython/issues/74940]: The C.UTF-8 locale is no longer converted to en_US.UTF-8, enabling the use of UTF-8 encoding on systems which have no locales installed.
- gh-103861 [https://github.com/python/cpython/issues/103861]: Fix `zipfile.Zipfile` creating invalid zip files when `force_zip64` was used to add files to them. Patch by Carey Metcalfe.
- gh-103857 [https://github.com/python/cpython/issues/103857]: Deprecated `datetime.datetime.utcnow()` and `datetime.datetime.utcfromtimestamp()`. (Patch by Paul Ganssle)
- gh-103839 [https://github.com/python/cpython/issues/103839]: Avoid compilation error due to tommath.h not being found when building Tkinter against Tcl 8.7 built with bundled libtommath.
- gh-103791 [https://github.com/python/cpython/issues/103791]: `contextlib.suppress` now supports suppressing exceptions raised as part of an `ExceptionGroup`. If other exceptions exist on the group, they are re-raised in a group that does not contain the suppressed exceptions.
- gh-90750 [https://github.com/python/cpython/issues/90750]: Use `datetime.datetime.fromisocalendar()` in the implementation of `datetime.datetime.strptime()`, which should now accept only valid ISO dates. (Patch by Paul Ganssle)
- gh-103685 [https://github.com/python/cpython/issues/103685]: Prepare `tkinter.Menu.index()` for Tk 8.7 so that it does not raise `TclError`:

`expected integer but got ""` when it should return `None`.

- gh-81403 [https://github.com/python/cpython/issues/81403]: **`urllib.request.CacheFTPHandler`** no longer raises **`URLError`** if a cached FTP instance is reused. ftplib's endtransfer method calls voidresp to drain the connection to handle FTP instance reuse properly.
- gh-103699 [https://github.com/python/cpython/issues/103699]: Add `__orig_bases__` to non-generic TypedDicts, call-based TypedDicts, and call-based NamedTuples. Other TypedDicts and NamedTuples already had the attribute.
- gh-103693 [https://github.com/python/cpython/issues/103693]: Add convenience variable feature to **`pdb`**
- gh-92248 [https://github.com/python/cpython/issues/92248]: Deprecate `type`, `choices`, and `metavar` parameters of `argparse.BooleanOptionalAction`.
- gh-89415 [https://github.com/python/cpython/issues/89415]: Add **`socket`** constants for source-specific multicast. Patch by Reese Hyde.
- gh-103673 [https://github.com/python/cpython/issues/103673]: **`socketserver`** gains `ForkingUnixStreamServer` and `ForkingUnixDatagramServer` classes. Patch by Jay Berry.
- gh-103636 [https://github.com/python/cpython/issues/103636]: Added Enum for months and days in the calendar module.
- gh-84976 [https://github.com/python/cpython/issues/84976]: Create a new `Lib/_pydatetime.py` file that defines the Python version of the `datetime` module, and make `datetime` import the contents of the new library only if the C implementation is missing. Currently, the full Python implementation is defined and then deleted if the C implementation is not available, slowing down `import datetime` unnecessarily.
- gh-103596 [https://github.com/python/cpython/issues/103596]: Attributes/methods are no longer shadowed by same-named enum members, although they may be shadowed by enum.property's.
- gh-103584 [https://github.com/python/cpython/issues/103584]: Updated `importlib.metadata` with changes from `importlib_metadata` 5.2 through 6.5.0, including: Support `installed-files.txt` for `Distribution.files` when present. `PackageMetadata` now stipulates an additional `get` method allowing for easy querying of metadata keys that may not be present. `packages_distributions` now honors

packages and modules with Python modules that not `.py` sources (e.g. `.pyc`, `.so`). Expand protocol for `PackageMetadata.get_all` to match the upstream implementation of `email.message.Message.get_all` in python/typeshed#9620. Deprecated use of `Distribution` without defining abstract methods. Deprecated expectation that `PackageMetadata.__getitem__` will return `None` for missing keys. In the future, it will raise a `KeyError`.

- gh-103578 [https://github.com/python/cpython/issues/103578]: Fixed a bug where **`pdb`** crashes when reading source file with different encoding by replacing **`io.open()`** with **`io.open_code()`**. The new method would also call into the hook set by **`PyFile_SetOpenCodeHook()`**.
- gh-103556 [https://github.com/python/cpython/issues/103556]: Now creating **`inspect.Signature`** objects with positional-only parameter with a default followed by a positional-or-keyword parameter without one is impossible.
- gh-103559 [https://github.com/python/cpython/issues/103559]: Update the bundled copy of pip to version 23.1.1.
- gh-103548 [https://github.com/python/cpython/issues/103548]: Improve performance of **`pathlib.Path.absolute()`** and **`cwd()`** by joining paths only when necessary. Also improve performance of **`pathlib.PurePath.is_absolute()`** on Posix by skipping path parsing and normalization.
- gh-103538 [https://github.com/python/cpython/issues/103538]: Remove `_tkinter` module code guarded by definition of the `TK_AQUA` macro which was only needed for Tk 8.4.7 or earlier and was never actually defined by any build system or documented for manual use.
- gh-103533 [https://github.com/python/cpython/issues/103533]: Update **`cProfile`** to use PEP 669 API
- gh-103525 [https://github.com/python/cpython/issues/103525]: Fix misleading exception message when mixed `str` and `bytes` arguments are supplied to **`pathlib.PurePath`** and **`Path`**.
- gh-103489 [https://github.com/python/cpython/issues/103489]: Add **`getconfig()`** and **`setconfig()`** to **`Connection`** to make configuration changes to a database connection. Patch by Erlend E. Aasland.
- gh-103365 [https://github.com/python/cpython/issues/103365]: Set default Flag boundary to `STRICT` and fix bitwise operations.

- gh-103472 [https://github.com/python/cpython/issues/103472]: Avoid a potential `ResourceWarning` in `http.client.HTTPConnection` by closing the proxy / tunnel's CONNECT response explicitly.
- gh-103462 [https://github.com/python/cpython/issues/103462]: Fixed an issue with using `writelines()` in `asyncio` to send very large payloads that exceed the amount of data that can be written in one call to `socket.socket.send()` or `socket.socket.sendmsg()`, resulting in the remaining buffer being left unwritten.
- gh-103449 [https://github.com/python/cpython/issues/103449]: Fix a bug in doc string generation in `dataclasses.dataclass()`.
- gh-103092 [https://github.com/python/cpython/issues/103092]: Isolate `_collections` (apply **PEP 687** [https://peps.python.org/pep-0687/]). Patch by Erlend E. Aasland.
- gh-103357 [https://github.com/python/cpython/issues/103357]: Added support for `logging.Formatter` `defaults` parameter to `logging.config.dictConfig()` and `logging.config.fileConfig()`. Patch by Bar Harel.
- gh-103092 [https://github.com/python/cpython/issues/103092]: Adapt the `winreg` extension module to **PEP 687** [https://peps.python.org/pep-0687/].
- gh-74690 [https://github.com/python/cpython/issues/74690]: The performance of `isinstance()` checks against `runtime-checkable protocols` has been considerably improved for protocols that only have a few members. To achieve this improvement, several internal implementation details of the `typing` module have been refactored, including `typing._ProtocolMeta.__instancecheck__`, `typing._is_callable_members_only`, and `typing._get_protocol_attrs`. Patches by Alex Waygood.
- gh-74690 [https://github.com/python/cpython/issues/74690]: The members of a runtime-checkable protocol are now considered "frozen" at runtime as soon as the class has been created. See "What's new in Python 3.12" for more details.
- gh-103256 [https://github.com/python/cpython/issues/103256]: Fixed a bug that caused `hmac` to raise an exception when the requested hash algorithm was not available in OpenSSL despite being available separately as part of `hashlib` itself. It now falls back properly to the built-in. This could happen when, for example, your OpenSSL does not include

SHA3 support and you want to compute `hmac.digest(b'K', b'M', 'sha3_256')`.

- gh-102778 [https://github.com/python/cpython/issues/102778]: Support `sys.last_exc` in **`idlelib`**.
- gh-103285 [https://github.com/python/cpython/issues/103285]: Improve performance of **`ast.get_source_segment()`**.
- gh-103225 [https://github.com/python/cpython/issues/103225]: Fix a bug in **`pdb`** when displaying line numbers of module-level source code.
- gh-103092 [https://github.com/python/cpython/issues/103092]: Adapt the **`msvcrt`** extension module to **PEP 687** [https://peps.python.org/pep-0687/].
- gh-103092 [https://github.com/python/cpython/issues/103092]: Adapt the **`winsound`** extension module to **PEP 687** [https://peps.python.org/pep-0687/].
- gh-93910 [https://github.com/python/cpython/issues/93910]: Remove deprecation of enum `member.member` access.
- gh-102978 [https://github.com/python/cpython/issues/102978]: Fixes **`unittest.mock.patch()`** not enforcing function signatures for methods decorated with `@classmethod` or `@staticmethod` when patch is called with `autospec=True`.
- gh-103092 [https://github.com/python/cpython/issues/103092]: Isolate **`_socket`** (apply **PEP 687** [https://peps.python.org/pep-0687/]). Patch by Erlend E. Aasland.
- gh-100479 [https://github.com/python/cpython/issues/100479]: Add **`pathlib.PurePath.with_segments()`**, which creates a path object from arguments. This method is called whenever a derivative path is created, such as from **`pathlib.PurePath.parent`**. Subclasses may override this method to share information between path objects.
- gh-103220 [https://github.com/python/cpython/issues/103220]: Fix issue where **`os.path.join()`** added a slash when joining onto an incomplete UNC drive with a trailing slash on Windows.
- gh-103204 [https://github.com/python/cpython/issues/103204]: Fixes **`http.server`** accepting HTTP requests with HTTP version numbers preceded by '+', or '-', or with digit-separating '_' characters. The length of the version numbers is also constrained.
- gh-75586 [https://github.com/python/cpython/issues/75586]: Fix various Windows-specific issues with `shutil.which`.
- gh-103193 [https://github.com/python/cpython/issues/103193]: Improve performance of **`inspect.getattr_static()`**. Patch by Alex Waygood.

- gh-103176 [https://github.com/python/cpython/issues/103176]: `sys._current_exceptions()` now returns a mapping from thread-id to an exception instance, rather than to a `(typ, exc, tb)` tuple.
- gh-103143 [https://github.com/python/cpython/issues/103143]: Polish the help messages and docstrings of `pdb`.
- gh-103015 [https://github.com/python/cpython/issues/103015]: Add *entrypoint* keyword-only parameter to `sqlite3.Connection.load_extension()`, for overriding the SQLite extension entry point. Patch by Erlend E. Aasland.
- gh-103000 [https://github.com/python/cpython/issues/103000]: Improve performance of `dataclasses.astuple()` and `dataclasses.asdict()` in cases where the contents are common Python types.
- gh-102953 [https://github.com/python/cpython/issues/102953]: The extraction methods in `tarfile`, and `shutil.unpack_archive()`, have a new a *filter* argument that allows limiting tar features than may be surprising or dangerous, such as creating files outside the destination directory. See Filtros de extração for details.
- gh-97696 [https://github.com/python/cpython/issues/97696]: Implemented an eager task factory in asyncio. When used as a task factory on an event loop, it performs eager execution of coroutines. Coroutines that are able to complete synchronously (e.g. return or raise without blocking) are returned immediately as a finished task, and the task is never scheduled to the event loop. If the coroutine blocks, the (pending) task is scheduled and returned.
- gh-81079 [https://github.com/python/cpython/issues/81079]: Add *case_sensitive* keyword-only argument to `pathlib.Path.glob()` and `rglob()`.
- gh-101819 [https://github.com/python/cpython/issues/101819]: Isolate the `io` extension module by applying **PEP 687** [https://peps.python.org/pep-0687/]. Patch by Kumar Aditya, Victor Stinner, and Erlend E. Aasland.
- gh-91896 [https://github.com/python/cpython/issues/91896]: Deprecate `collections.abc.ByteString`
- gh-101362 [https://github.com/python/cpython/issues/101362]: Speed up `pathlib.Path` construction by omitting the path anchor from the internal list of path parts.
- gh-102114 [https://github.com/python/cpython/issues/102114]: Functions in the `dis` module that accept a source code string as argument now print a more

concise traceback when the string contains a syntax or indentation error.

- gh-62432 [https://github.com/python/cpython/issues/62432]: The **unittest** runner will now exit with status code 5 if no tests were run. It is common for test runner misconfiguration to fail to find any tests, this should be an error.
- gh-78079 [https://github.com/python/cpython/issues/78079]: Fix incorrect normalization of UNC device path roots, and partial UNC share path roots, in **pathlib.PurePath**. Pathlib no longer appends a trailing slash to such paths.
- gh-85984 [https://github.com/python/cpython/issues/85984]: Add **tty.cfmakeraw()** and **tty.cfmakecbreak()** to **tty** and modernize, the behavior of **tty.setraw()** and **tty.setcbreak()** to use POSIX.1-2017 Chapter 11 “General Terminal Interface” flag masks by default.
- gh-101688 [https://github.com/python/cpython/issues/101688]: Implement **types.get_original_bases()** to provide further introspection for types.
- gh-101640 [https://github.com/python/cpython/issues/101640]: **argparse.ArgumentParser** now catches errors when writing messages, such as when **sys.stderr** is `None`. Patch by Oleg Iarygin.
- gh-83861 [https://github.com/python/cpython/issues/83861]: Fix datetime.astimezone method return value when invoked on a naive datetime instance that represents local time falling in a timezone transition gap. PEP 495 requires that instances with fold=1 produce earlier times than those with fold=0 in this case.
- gh-89550 [https://github.com/python/cpython/issues/89550]: Decrease execution time of some **gzip** file writes by 15% by adding more appropriate buffering.
- gh-95299 [https://github.com/python/cpython/issues/95299]: Remove the bundled setuptools wheel from `ensurepip`, and stop installing setuptools in environments created by `venv`.
- gh-99353 [https://github.com/python/cpython/issues/99353]: Respect the **http.client.HTTPConnection** `.debuglevel` flag in **urllib.request.AbstractHTTPHandler** when its constructor parameter `debuglevel` is not set. And do the same for `*HTTPS*`.
- gh-98040 [https://github.com/python/cpython/issues/98040]: Remove the long-deprecated `imp` module.

- gh-97850 [https://github.com/python/cpython/issues/97850]: Deprecate **`pkgutil.find_loader()`** and **`pkgutil.get_loader()`** in favor of **`importlib.util.find_spec()`**.
- gh-94473 [https://github.com/python/cpython/issues/94473]: Flatten arguments in **`tkinter.Canvas.coords()`**. It now accepts not only `x1, y1, x2, y2, ...` and `[x1, y1, x2, y2, ...]`, but also `(x1, y1), (x2, y2), ...` and `[(x1, y1), (x2, y2), ...]`.
- gh-98040 [https://github.com/python/cpython/issues/98040]: Remove more deprecated importlib APIs: `find_loader()`, `find_module()`, `importlib.abc.Finder`, `pkgutil.ImpImporter`, `pkgutil.ImpLoader`.
- gh-96522 [https://github.com/python/cpython/issues/96522]: Fix potential deadlock in pty.spawn()
- gh-96534 [https://github.com/python/cpython/issues/96534]: Support divert(4) added in FreeBSD 14.
- gh-87474 [https://github.com/python/cpython/issues/87474]: Fix potential file descriptor leaks in **`subprocess.Popen`**.
- gh-94906 [https://github.com/python/cpython/issues/94906]: Support multiple steps in **`math.nextafter()`**. Patch by Shantanu Jain and Matthias Gorgens.
- gh-51574 [https://github.com/python/cpython/issues/51574]: Make **`tempfile.mkdtemp()`** return absolute paths when its *dir* parameter is relative.
- gh-94518 [https://github.com/python/cpython/issues/94518]: Convert private **`_posixsubprocess.fork_exec()`** to use Argument Clinic.
- gh-92184 [https://github.com/python/cpython/issues/92184]: When creating zip files using **`zipfile`**, `os.altsep`, if not `None`, will always be treated as a path separator even when it is not `/`. Patch by Carey Metcalfe.
- bpo-46797 [https://bugs.python.org/issue?@action=redirect&bpo=46797]: Deprecation warnings are now emitted for **`ast.Num`**, **`ast.Bytes`**, **`ast.Str`**, **`ast.NameConstant`** and **`ast.Ellipsis`**. These have been documented as deprecated since Python 3.8, and will be removed in Python 3.14.
- bpo-44844 [https://bugs.python.org/issue?@action=redirect&bpo=44844]: Enables **`webbrowser`** to detect and launch Microsoft Edge browser.
- bpo-45606 [https://bugs.python.org/issue?@action=redirect&bpo=45606]: Fixed the bug in **`pathlib.Path.glob()`** – previously a dangling symlink would not be found by this method when the pattern is an exact match, but would be found when the pattern contains a wildcard or the recursive

wildcard (`**`). With this change, a dangling symlink will be found in both cases.

- bpo-23041 [https://bugs.python.org/issue?@action=redirect&bpo=23041]: Add **`QUOTE_STRINGS`** and **`QUOTE_NOTNULL`** to the suite of **`csv`** module quoting styles.
- bpo-24964 [https://bugs.python.org/issue?@action=redirect&bpo=24964]: Added **`http.client.HTTPConnection.get_proxy_response_headers()`** that provides access to the HTTP headers on a proxy server response to the `CONNECT` request.
- bpo-17258 [https://bugs.python.org/issue?@action=redirect&bpo=17258]: **`multiprocessing`** now supports stronger HMAC algorithms for inter-process connection authentication rather than only HMAC-MD5.
- bpo-39744 [https://bugs.python.org/issue?@action=redirect&bpo=39744]: Make **`asyncio.subprocess.Process.communicate()`** close the subprocess's stdin even when called with `input=None`.
- bpo-22708 [https://bugs.python.org/issue?@action=redirect&bpo=22708]: http.client CONNECT method tunnel improvements: Use HTTP 1.1 protocol; send a matching Host: header with CONNECT, if one is not provided; convert IDN domain names to Punycode. Patch by Michael Handler.

Documentação

- gh-67056 [https://github.com/python/cpython/issues/67056]: Document that the effect of registering or unregistering an **`atexit`** cleanup function from within a registered cleanup function is undefined.
- gh-103629 [https://github.com/python/cpython/issues/103629]: Mention the new way of typing `**kwargs` with `Unpack` and `TypedDict` introduced in **PEP 692** [https://peps.python.org/pep-0692/].
- gh-48241 [https://github.com/python/cpython/issues/48241]: Clarifying documentation about the url parameter to urllib.request.urlopen and urllib.request.Request needing to be encoded properly.
- gh-86094 [https://github.com/python/cpython/issues/86094]: Add support for Unicode Path Extra Field in ZipFile. Patch by Yeojin Kim and Andrea Giudiceandrea
- gh-99202 [https://github.com/python/cpython/issues/99202]: Fix extension type from documentation for compiling in C++20 mode

Testes

- gh-104494 [https://github.com/python/cpython/issues/104494]: Update `test_pack_configure_in` and `test_place_configure_in` for changes to error message formatting in Tk 8.7.
- gh-104461 [https://github.com/python/cpython/issues/104461]: Run test_configure_screen on X11 only, since the `DISPLAY` environment variable and `-screen` option for toplevels are not useful on Tk for Win32 or Aqua.
- gh-86275 [https://github.com/python/cpython/issues/86275]: Added property-based tests to the **`zoneinfo`** tests, along with stubs for the `hypothesis` interface. (Patch by Paul Ganssle)
- gh-103329 [https://github.com/python/cpython/issues/103329]: Regression tests for the behaviour of `unittest.mock.PropertyMock` were added.
- gh-102795 [https://github.com/python/cpython/issues/102795]: fix use of poll in test_epoll's test_control_and_wait
- gh-75729 [https://github.com/python/cpython/issues/75729]: Fix the **`os.spawn*`** tests failing on Windows when the working directory or interpreter path contains spaces.

Construção

- gh-101282 [https://github.com/python/cpython/issues/101282]: BOLT optimization is now applied to the libpython shared library if building a shared library. BOLT instrumentation and application settings can now be influenced via the `BOLT_INSTRUMENT_FLAGS` and `BOLT_APPLY_FLAGS` configure variables.

- gh-99017 [https://github.com/python/cpython/issues/99017]: `PYTHON_FOR_REGEN` now require Python 3.10 or newer.

- gh-104490 [https://github.com/python/cpython/issues/104490]: Define `.PHONY` / virtual make targets consistently and properly.

- gh-104106 [https://github.com/python/cpython/issues/104106]: Add gcc fallback of mkfifoat/mknodat for macOS. Patch by Donghee Na.

- gh-103532 [https://github.com/python/cpython/issues/103532]: The `TKINTER_PROTECT_LOADTK` macro is no longer defined or used in the `_tkinter` module. It was previously only defined when building against Tk 8.4.13 and older, but Tk older than 8.5.12 has been unsupported since gh-91152 [https://github.com/python/cpython/issues/91152].

- gh-99069 [https://github.com/python/cpython/issues/99069]: Extended workaround defining `static_assert` when missing from the libc headers to all clang and gcc builds. In particular, this fixes building on macOS <= 10.10.

- gh-100220 [https://github.com/python/cpython/issues/100220]: Changed the default value of the `SHELL` Makefile variable from `/bin/sh` to `/bin/sh -e` to ensure that complex recipes correctly fail after an error. Previously, `make install` could fail to install some files and yet return a successful result.

- gh-90656 [https://github.com/python/cpython/issues/90656]: Add platform triplets for 64-bit LoongArch:

  - loongarch64-linux-gnusf
  - loongarch64-linux-gnuf32
  - loongarch64-linux-gnu

  Patch by Zhang Na.

Windows

- gh-104623 [https://github.com/python/cpython/issues/104623]: Update Windows installer to use SQLite 3.42.0.
- gh-82814 [https://github.com/python/cpython/issues/82814]: Fix a potential `[Errno 13] Permission denied` when using `shutil.copystat()` within Windows Subsystem for Linux (WSL) on a mounted filesystem by adding `errno.EACCES` to the list of ignored errors within the internal implementation.
- gh-103088 [https://github.com/python/cpython/issues/103088]: Fix virtual environment `activate` script having incorrect line endings for Cygwin.

- gh-103088 [https://github.com/python/cpython/issues/103088]: Fixes venvs not working in bash on Windows across different disks
- gh-102997 [https://github.com/python/cpython/issues/102997]: Update Windows installer to use SQLite 3.41.2.
- gh-88013 [https://github.com/python/cpython/issues/88013]: Fixed a bug where **`TypeError`** was raised when calling **`ntpath.realpath()`** with a bytes parameter in some cases.

macOS

- gh-99834 [https://github.com/python/cpython/issues/99834]: Update macOS installer to Tcl/Tk 8.6.13.
- gh-104623 [https://github.com/python/cpython/issues/104623]: Update macOS installer to SQLite 3.42.0.
- gh-103545 [https://github.com/python/cpython/issues/103545]: Add `os.PRIO_DARWIN_THREAD`, `os.PRIO_DARWIN_PROCESS`, `os.PRIO_DARWIN_BG` and `os.PRIO_DARWIN_NONUI`. These can be used with `os.setpriority` to run the process at a lower priority and make use of the efficiency cores on Apple Silicon systems.
- gh-104180 [https://github.com/python/cpython/issues/104180]: Support reading SOCKS proxy configuration from macOS System Configuration. Patch by Sam Schott.
- gh-60436 [https://github.com/python/cpython/issues/60436]: update curses textbox to additionally handle backspace using the `curses.ascii.DEL` key press.
- gh-102997 [https://github.com/python/cpython/issues/102997]: Update macOS installer to SQLite 3.41.2.

IDLE

- gh-104499 [https://github.com/python/cpython/issues/104499]: Fix completions for Tk Aqua 8.7 (currently blank).
- gh-104496 [https://github.com/python/cpython/issues/104496]: About prints both tcl and tk versions if different (expected someday).
- gh-88496 [https://github.com/python/cpython/issues/88496]: Fix IDLE test hang on macOS.

Ferramentas/Demos

- gh-104389 [https://github.com/python/cpython/issues/104389]: Argument Clinic C converters now accept the `unused` keyword, for wrapping a parameter with `Py_UNUSED`. Patch by Erlend E. Aasland.

API C

- gh-101291 [https://github.com/python/cpython/issues/101291]: Added unstable C API for extracting the value of "compact" integers: `PyUnstable_Long_IsCompact()` and `PyUnstable_Long_CompactValue()`.
- gh-104109 [https://github.com/python/cpython/issues/104109]: We've added `Py_NewInterpreterFromConfig()` and `PyInterpreterConfig` to the public C-API (but not the stable ABI; not yet at least). The new function may be used to create a new interpreter with various features configured. The function was added to support PEP 684 (per-interpreter GIL).
- gh-103968 [https://github.com/python/cpython/issues/103968]: `PyType_FromSpec()` and its variants now allow creating classes whose metaclass overrides `tp_new`. The `tp_new` is ignored. This behavior is deprecated and will be disallowed in 3.14+. The new `PyType_FromMetaclass()` already disallows it.
- gh-103743 [https://github.com/python/cpython/issues/103743]: Add `PyUnstable_Object_GC_NewWithExtraData()` function that can be used to allocate additional memory after an object for data not managed by Python.
- gh-103295 [https://github.com/python/cpython/issues/103295]: Introduced `PyUnstable_WritePerfMapEntry()`, `PyUnstable_PerfMapState_Init()` and `PyUnstable_PerfMapState_Fini()`. These allow extension modules (JIT compilers in particular) to write to perf-map files in a thread safe manner. The Python support for the perf map compatible profilers also uses these APIs to write entries in the perf-map file.
- gh-103509 [https://github.com/python/cpython/issues/103509]: Added C API for extending types whose instance memory layout is opaque: `PyType_Spec.basicsize` can now be zero or negative,

`PyObject_GetTypeData()` can be used to get subclass-specific data, and `Py_TPFLAGS_ITEMS_AT_END` can be used to safely extend variable-size objects. See **PEP 697** [https://peps.python.org/pep-0697/] for details.

- gh-103091 [https://github.com/python/cpython/issues/103091]: Add a new C-API function to eagerly assign a version tag to a PyTypeObject: `PyUnstable_Type_AssignVersionTag()`.
- gh-101408 [https://github.com/python/cpython/issues/101408]: `PyObject_GC_Resize` should calculate preheader size if needed. Patch by Donghee Na.
- gh-98836 [https://github.com/python/cpython/issues/98836]: Add support of more formatting options (left aligning, octals, uppercase hexadecimals, **`intmax_t`**, **`ptrdiff_t`**, **`wchar_t`** C strings, variable width and precision) in `PyUnicode_FromFormat()` and `PyUnicode_FromFormatV()`.
- gh-96803 [https://github.com/python/cpython/issues/96803]: Add unstable C-API functions to get the code object, lasti and line number from the internal `_PyInterpreterFrame` in the limited API. The functions are:
  - `PyCodeObject * PyUnstable_InterpreterFrame_GetCode(struct _PyInterpreterFrame *frame)`
  - `int PyUnstable_InterpreterFrame_GetLasti(struct _PyInterpreterFrame *frame)`
  - `int PyUnstable_InterpreterFrame_GetLine(struct _PyInterpreterFrame *frame)`

Python 3.12.0 alpha 7

Data de lançamento: 2023-04-04

Núcleo e embutidos

- gh-102192 [https://github.com/python/cpython/issues/102192]: Deprecated `_PyErr_ChainExceptions` in favour of `_PyErr_ChainExceptions1`.
- gh-89987 [https://github.com/python/cpython/issues/89987]: Reduce the number of inline `CACHE` entries for **`BINARY_SUBSCR`**.

- gh-102859 [https://github.com/python/cpython/issues/102859]: Removed **`JUMP_IF_FALSE_OR_POP`** and **`JUMP_IF_TRUE_OR_POP`** instructions.
- gh-101975 [https://github.com/python/cpython/issues/101975]: Fixed `stacktop` value on tracing entries to avoid corruption on garbage collection.
- gh-102778 [https://github.com/python/cpython/issues/102778]: Add **`sys.last_exc`** and deprecate **`sys.last_type`**, **`sys.last_value`** and **`sys.last_traceback`**, which hold the same information in its legacy form.
- gh-100982 [https://github.com/python/cpython/issues/100982]: Replace all occurrences of `COMPARE_AND_BRANCH` with **`COMPARE_OP`**.
- gh-102701 [https://github.com/python/cpython/issues/102701]: Fix overflow when creating very large dict.
- gh-102755 [https://github.com/python/cpython/issues/102755]: Add **`PyErr_DisplayException()`** which takes just an exception instance, to replace the legacy **`PyErr_Display()`** which takes the `(typ, exc, tb)` triplet.
- gh-102594 [https://github.com/python/cpython/issues/102594]: Add note to exception raised in `PyErr_SetObject` when normalization fails.
- gh-90997 [https://github.com/python/cpython/issues/90997]: Shrink the number of inline **`CACHE`** entries used by **`LOAD_GLOBAL`**.

Biblioteca

- gh-102491 [https://github.com/python/cpython/issues/102491]: Improve import time of `platform` by removing IronPython version parsing. The IronPython version parsing was not functional (see https://github.com/IronLanguages/ironpython3/issues/1667).

Núcleo e embutidos

- gh-101291 [https://github.com/python/cpython/issues/101291]: Rearrage bits in first field (after header) of PyLongObject. * Bits 0 and 1: 1 - sign. I.e. 0 for positive numbers, 1 for zero and 2 for negative numbers. * Bit 2 reserved (probably for the immortal bit) * Bits 3+ the unsigned size.

  This makes a few operations slightly more efficient, and will enable a more compact and faster 2s-complement representation of most ints in

future.

- gh-102397 [https://github.com/python/cpython/issues/102397]: Fix segfault from race condition in signal handling during garbage collection. Patch by Kumar Aditya.

- gh-102406 [https://github.com/python/cpython/issues/102406]: **`codecs`** encoding/decoding errors now get the context information (which operation and which codecs) attached as **PEP 678** [https://peps.python.org/pep-0678/] notes instead of through chaining a new instance of the exception.

- gh-102281 [https://github.com/python/cpython/issues/102281]: Fix potential nullptr dereference and use of uninitialized memory in fileutils. Patch by Max Bachmann.

- gh-102300 [https://github.com/python/cpython/issues/102300]: Reuse operands with refcount of 1 in float specializations of BINARY_OP.

- gh-102213 [https://github.com/python/cpython/issues/102213]: Fix performance loss when accessing an object's attributes with `__getattr__` defined.

- gh-102255 [https://github.com/python/cpython/issues/102255]: Improve build support for the Xbox. Patch by Max Bachmann.

Construção

- gh-102027 [https://github.com/python/cpython/issues/102027]: Fix SSE2 and SSE3 detection in `_blake2` internal module. Patch by Max Bachmann.

Núcleo e embutidos

- gh-101865 [https://github.com/python/cpython/issues/101865]: Deprecate `co_lnotab` in code objects, schedule it for removal in Python 3.14

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Adapt `_pickle` to **PEP 687** [https://peps.python.org/pep-0687/]. Patch by Mohamed Koubaa and Erlend Aasland.

- gh-103085 [https://github.com/python/cpython/issues/103085]: Pure python `locale.getencoding()` will not warn deprecation.

- gh-103068 [https://github.com/python/cpython/issues/103068]: It's no longer possible to register conditional breakpoints in `Pdb` that raise `SyntaxError`. Patch by Tian Gao.

- gh-102549 [https://github.com/python/cpython/issues/102549]: Don't ignore exceptions in member type creation.

- gh-103056 [https://github.com/python/cpython/issues/103056]: Ensure final `_generate_next_value_` is a `staticmethod`.

- gh-103046 [https://github.com/python/cpython/issues/103046]: Display current line label correctly in `dis` when `show_caches` is False and `lasti` points to a CACHE entry.

- gh-102433 [https://github.com/python/cpython/issues/102433]: `isinstance()` checks against `runtime-checkable protocols` now use `inspect.getattr_static()` rather than `hasattr()` to lookup whether attributes exist. This means that descriptors and `__getattr__()` methods are no longer unexpectedly evaluated during `isinstance()` checks against runtime-checkable protocols. However, it may also mean that some objects which used to be considered instances of a runtime-checkable protocol may no longer be considered instances of that protocol on Python 3.12+, and vice versa. Most users are unlikely to be affected by this change. Patch by Alex Waygood.

- gh-103023 [https://github.com/python/cpython/issues/103023]: It's no longer possible to register expressions to display in `Pdb` that raise `SyntaxError`. Patch by Tian Gao.

- gh-102947 [https://github.com/python/cpython/issues/102947]: Improve traceback when `dataclasses.fields()` is called on a non-dataclass. Patch by

Alex Waygood

- gh-102780 [https://github.com/python/cpython/issues/102780]: The **asyncio.Timeout** context manager now works reliably even when performing cleanup due to task cancellation. Previously it could raise a **CancelledError** instead of an **TimeoutError** in such cases.

- gh-102871 [https://github.com/python/cpython/issues/102871]: Remove support for obsolete browsers from **webbrowser**. Removed browsers include Grail, Mosaic, Netscape, Galeon, Skipstone, Iceape, Firebird, and Firefox versions 35 and below.

- gh-102839 [https://github.com/python/cpython/issues/102839]: Improve performance of **math.log()** arguments handling by removing the argument clinic.

- gh-102828 [https://github.com/python/cpython/issues/102828]: Add the `onexc` arg to **shutil.rmtree()**, which is like `onerror` but expects an exception instance rather than an exc_info tuple. Deprecate `onerror`.

- gh-88965 [https://github.com/python/cpython/issues/88965]: typing: Fix a bug relating to substitution in custom classes generic over a **ParamSpec**. Previously, if the `ParamSpec` was substituted with a parameters list that itself contained a **TypeVar**, the `TypeVar` in the parameters list could not be subsequently substituted. This is now fixed.

  Patch by Nikita Sobolev.

- gh-76846 [https://github.com/python/cpython/issues/76846]: Fix issue where `__new__()` and `__init__()` methods of **pathlib.PurePath** and **Path** subclasses were not called in some circumstances.

- gh-78530 [https://github.com/python/cpython/issues/78530]: **asyncio.wait()** now accepts generators yielding tasks. Patch by Kumar Aditya.

- gh-102748 [https://github.com/python/cpython/issues/102748]: **asyncio.iscoroutine()** now returns `False` for generators as **asyncio**

does not support legacy generator-based coroutines. Patch by Kumar Aditya.

- gh-102670 [https://github.com/python/cpython/issues/102670]: Optimized fmean(), correlation(), covariance(), and linear_regression() using the new math.sumprod() function.

- gh-102615 [https://github.com/python/cpython/issues/102615]: Typing: Improve the `repr` of generic aliases for classes generic over a **`ParamSpec`**. (Use square brackets to represent a parameter list.)

- gh-100112 [https://github.com/python/cpython/issues/100112]: **`asyncio.Task.get_coro()`** now always returns a coroutine when wrapping an awaitable object. Patch by Kumar Aditya.

- gh-102578 [https://github.com/python/cpython/issues/102578]: Speed up setting or deleting mutable attributes on non-dataclass subclasses of frozen dataclasses. Due to the implementation of `__setattr__` and `__delattr__` for frozen dataclasses, this previously had a time complexity of $O(n)$. It now has a time complexity of $O(1)$.

- gh-102519 [https://github.com/python/cpython/issues/102519]: Add **`os.listdrives()`**, **`os.listvolumes()`** and **`os.listmounts()`** functions on Windows for enumerating drives, volumes and mount points

- gh-74468 [https://github.com/python/cpython/issues/74468]: Attribute name of the extracted **`tarfile`** file object now holds filename of itself rather than of the archive it is contained in. Patch by Oleg Iarygin.

- gh-102378 [https://github.com/python/cpython/issues/102378]: Private helper method `inspect._signature_strip_non_python_syntax` will no longer strip `/` from the input string.

- gh-79940 [https://github.com/python/cpython/issues/79940]: Add **`inspect.getasyncgenstate()`** and **`inspect.getasyncgenlocals()`**. Patch by Thomas Krennwallner.

- gh-102103 [https://github.com/python/cpython/issues/102103]: Add `module` argument to **`dataclasses.make_dataclass()`** and make classes produced by it pickleable.

- gh-102069 [https://github.com/python/cpython/issues/102069]: Fix `__weakref__` descriptor generation for custom dataclasses.

- gh-102038 [https://github.com/python/cpython/issues/102038]: Skip a `stat` in **`site`** if we have already found a `pyvenv.cfg`

- gh-98886 [https://github.com/python/cpython/issues/98886]: Fix issues when defining dataclasses that have fields with specific underscore names that aren't clearly reserved by **`dataclasses`**.

- gh-101673 [https://github.com/python/cpython/issues/101673]: Fix a **`pdb`** bug where `ll` clears the changes to local variables.

- gh-101313 [https://github.com/python/cpython/issues/101313]: Added -h and –help arguments to the webbrowser CLI

- gh-100372 [https://github.com/python/cpython/issues/100372]: **`ssl.SSLContext.load_verify_locations()`** no longer incorrectly accepts some cases of trailing data when parsing DER.

- gh-89727 [https://github.com/python/cpython/issues/89727]: Fix pathlib.Path.walk RecursionError on deep directory trees by rewriting it using iteration instead of recursion.

- gh-100131 [https://github.com/python/cpython/issues/100131]: Added an optional `delete` keyword argument to **`tempfile.TemporaryDirectory`**.

- gh-48330 [https://github.com/python/cpython/issues/48330]: Added `--durations` command line option, showing the N slowest test cases. **`unittest.TextTestRunner`** and **`unittest.TextTestResult`** constructors accept a new *durations* keyword argument. Subclasses should take this into account or accept `**kwargs`. Added **`unittest.TestResult.addDuration()`** method and **`unittest.TestResult.collectedDurations`** attribute.

(Contribuição de Giampaolo Rodola)

- gh-98169 [https://github.com/python/cpython/issues/98169]: Fix `dataclasses.astuple()` crash when `collections.defaultdict` is present in the attributes.

- gh-96931 [https://github.com/python/cpython/issues/96931]: Fix incorrect results from `ssl.SSLSocket.shared_ciphers()`

- gh-95495 [https://github.com/python/cpython/issues/95495]: When built against OpenSSL 3.0, the `ssl` module had a bug where it reported unauthenticated EOFs (i.e. without close_notify) as a clean TLS-level EOF. It now raises `SSLEOFError`, matching the behavior in previous versions of OpenSSL. The `options` attribute on `SSLContext` also no longer includes `OP_IGNORE_UNEXPECTED_EOF` by default. This option may be set to specify the previous OpenSSL 3.0 behavior.

- gh-94684 [https://github.com/python/cpython/issues/94684]: Now `uuid.uuid3()` and `uuid.uuid5()` functions support `bytes` objects as their *name* argument.

- gh-94440 [https://github.com/python/cpython/issues/94440]: Fix a **`concurrent.futures.process`** bug where `ProcessPoolExecutor` shutdown could hang after a future has been quickly submitted and canceled.

- gh-72346 [https://github.com/python/cpython/issues/72346]: Added deprecation warning to *isdst* parameter of `email.utils.localtime()`.

- bpo-36305 [https://bugs.python.org/issue?@action=redirect&bpo=36305]: Fix handling of Windows filenames that resemble drives, such as `./a:b`, in `pathlib`.

Documentação

- gh-103112 [https://github.com/python/cpython/issues/103112]: Add docstring to `http.client.HTTPResponse.read()` to fix `pydoc` output.

Testes

- gh-102980 [https://github.com/python/cpython/issues/102980]: Improve test coverage on **`pdb`**.
- gh-102537 [https://github.com/python/cpython/issues/102537]: Adjust the error handling strategy in `test_zoneinfo.TzPathTest.python_tzpath_context`. Patch by Paul Ganssle.
- gh-101377 [https://github.com/python/cpython/issues/101377]: Improved test_locale_calendar_formatweekday of calendar.

Construção

- gh-102973 [https://github.com/python/cpython/issues/102973]: Add a dev container (along with accompanying Dockerfile) for development purposes.
- gh-102711 [https://github.com/python/cpython/issues/102711]: Fix `-Wstrict-prototypes` compiler warnings.

Windows

- gh-102690 [https://github.com/python/cpython/issues/102690]: Update **`webbrowser`** to fall back to Microsoft Edge instead of Internet Explorer.
- gh-99726 [https://github.com/python/cpython/issues/99726]: Improves correctness of stat results for Windows, and uses faster API when available

Ferramentas/Demos

- gh-102809 [https://github.com/python/cpython/issues/102809]: `Misc/gdbinit` was removed.

API C

- gh-102013 [https://github.com/python/cpython/issues/102013]: Add a new (unstable) C-API function for iterating over GC'able objects using a callback: `PyUnstable_VisitObjects`.

Python 3.12.0 alpha 6

Data de lançamento: 2023-03-07

Segurança

- gh-99108 [https://github.com/python/cpython/issues/99108]: Replace builtin hashlib implementations of MD5 and SHA1 with verified ones from the HACL* project.
- gh-101727 [https://github.com/python/cpython/issues/101727]: Updated the OpenSSL version used in Windows and macOS binary release builds to 1.1.1t to address **CVE 2023-0286** [https://www.cve.org/CVERecord?id=CVE-2023-0286], **CVE 2022-4303** [https://www.cve.org/CVERecord?id=CVE-2022-4303], and **CVE 2022-4303** [https://www.cve.org/CVERecord?id=CVE-2022-4303] per the OpenSSL 2023-02-07 security advisory [https://openssl-library.org/news/secadv/20230207.txt].
- gh-99108 [https://github.com/python/cpython/issues/99108]: Replace the builtin **`hashlib`** implementations of SHA2-384 and SHA2-512 originally from LibTomCrypt with formally verified, side-channel resistant code from the HACL* [https://github.com/hacl-star/hacl-star/] project. The builtins remain a fallback only used when OpenSSL does not provide them.
- gh-101283 [https://github.com/python/cpython/issues/101283]: **`subprocess.Popen`** now uses a safer approach to find `cmd.exe` when launching with `shell=True`. Patch by Eryk Sun, based on a patch by Oleg Iarygin.

Núcleo e embutidos

- gh-102493 [https://github.com/python/cpython/issues/102493]: Fix regression in semantics of normalisation in `PyErr_SetObject`.
- gh-102416 [https://github.com/python/cpython/issues/102416]: Do not memoize incorrectly automatically generated loop rules in the parser. Patch by Pablo Galindo.
- gh-102356 [https://github.com/python/cpython/issues/102356]: Fix a bug that caused a crash when deallocating deeply nested filter objects. Patch by Marta Gómez Macías.

- gh-102336 [https://github.com/python/cpython/issues/102336]: Cleanup Windows 7 specific special handling. Patch by Max Bachmann.
- gh-102250 [https://github.com/python/cpython/issues/102250]: Fixed a segfault occurring when the interpreter calls a `__bool__` method that raises.
- gh-102126 [https://github.com/python/cpython/issues/102126]: Fix deadlock at shutdown when clearing thread states if any finalizer tries to acquire the runtime head lock. Patch by Kumar Aditya.
- gh-102027 [https://github.com/python/cpython/issues/102027]: Use `GetCurrentProcessId` on Windows when `getpid` is unavailable. Patch by Max Bachmann.
- gh-102056 [https://github.com/python/cpython/issues/102056]: Fix error handling bugs in interpreter's exception printing code, which could cause a crash on infinite recursion.
- gh-100982 [https://github.com/python/cpython/issues/100982]: Restrict the scope of the **`FOR_ITER_RANGE`** instruction to the scope of the original **`FOR_ITER`** instruction, to allow instrumentation.
- gh-101967 [https://github.com/python/cpython/issues/101967]: Fix possible segfault in `positional_only_passed_as_keyword` function, when new list created.
- gh-101952 [https://github.com/python/cpython/issues/101952]: Fix possible segfault in `BUILD_SET` opcode, when new set created.
- gh-74895 [https://github.com/python/cpython/issues/74895]: **`socket.getaddrinfo`** no longer raises **`OverflowError`** for **`int`** **port** values outside of the C long range. Out of range values are left up to the underlying string based C library API to report. A **`socket.gaierror`** `SAI_SERVICE` may occur instead, or no error at all as not all platform C libraries generate an error.
- gh-101799 [https://github.com/python/cpython/issues/101799]: Add **`CALL_INTRINSIC_2`** and use it instead of **`PREP_RERAISE_STAR`**.
- gh-101857 [https://github.com/python/cpython/issues/101857]: Fix xattr support detection on Linux systems by widening the check to linux, not just glibc. This fixes support for musl.
- gh-84783 [https://github.com/python/cpython/issues/84783]: Make the slice object hashable. Patch by Will Bradshaw and Furkan Onder.
- gh-87849 [https://github.com/python/cpython/issues/87849]: Change the `SEND` instruction to leave the receiver on the stack. This allows the

specialized form of `SEND` to skip the chain of C calls and jump directly to the `RESUME` in the generator or coroutine.

- gh-101765 [https://github.com/python/cpython/issues/101765]: Fix SystemError / segmentation fault in iter `__reduce__` when internal access of `builtins.__dict__` keys mutates the iter object.
- gh-101430 [https://github.com/python/cpython/issues/101430]: Update **`tracemalloc`** to handle presize of object properly. Patch by Donghee Na.
- gh-101696 [https://github.com/python/cpython/issues/101696]: Invalidate type version tag in `_PyStaticType_Dealloc` for static types, avoiding bug where a false cache hit could crash the interpreter. Patch by Kumar Aditya.
- gh-101632 [https://github.com/python/cpython/issues/101632]: Adds a new **`RETURN_CONST`** instruction.
- gh-100719 [https://github.com/python/cpython/issues/100719]: Remove gi_code field from generator (and coroutine and async generator) objects as it is redundant. The frame already includes a reference to the code object.
- gh-98627 [https://github.com/python/cpython/issues/98627]: When an interpreter is configured to check (and only then), importing an extension module will now fail when the extension does not support multiple interpreters (i.e. doesn't implement PEP 489 multi-phase init). This does not apply to the main interpreter, nor to subinterpreters created with `Py_NewInterpreter()`.

Biblioteca

- gh-102302 [https://github.com/python/cpython/issues/102302]: Micro-optimise hashing of **`inspect.Parameter`**, reducing the time it takes to hash an instance by around 40%.

- gh-101979 [https://github.com/python/cpython/issues/101979]: Fix a bug where parentheses in the `metavar` argument to **`argparse.ArgumentParser.add_argument()`** were dropped. Patch by Yeojin Kim.

- gh-91038 [https://github.com/python/cpython/issues/91038]: **platform.platform()** now has boolean default arguments.

- gh-81652 [https://github.com/python/cpython/issues/81652]: Add **mmap.MAP_ALIGNED_SUPER** FreeBSD and **mmap.MAP_CONCEAL** OpenBSD constants to **mmap**. Patch by Yeojin Kim.

- gh-102179 [https://github.com/python/cpython/issues/102179]: Fix **os.dup2()** error message for negative fds.

- gh-101961 [https://github.com/python/cpython/issues/101961]: For the binary mode, **fileinput.hookcompressed()** doesn't set the `encoding` value even if the value is `None`. Patch by Gihwan Kim.

- gh-101936 [https://github.com/python/cpython/issues/101936]: The default value of `fp` becomes **io.BytesIO** if **HTTPError** is initialized without a designated `fp` parameter. Patch by Long Vo.

- gh-101566 [https://github.com/python/cpython/issues/101566]: In zipfile, sync Path with zipp 3.14 [https://zipp.readthedocs.io/en/latest/history.html#v3-14-0], including fix for extractall on the underlying zipfile after being wrapped in `Path`.

- gh-97930 [https://github.com/python/cpython/issues/97930]: Apply changes from importlib_resources 5.12 [https://importlib-resources.readthedocs.io/en/latest/history.html#v5-12-0], including fix for `MultiplexedPath` to support directories in multiple namespaces (python/importlib_resources#265).

- gh-101997 [https://github.com/python/cpython/issues/101997]: Upgrade pip wheel bundled with ensurepip (pip 23.0.1)

- gh-99108 [https://github.com/python/cpython/issues/99108]: The built-in extension modules for **hashlib** SHA2 algorithms, used when OpenSSL does not provide them, now live in a single internal `_sha2` module instead of separate `_sha256` and `_sha512` modules.

- gh-101892 [https://github.com/python/cpython/issues/101892]: Callable iterators no longer raise **SystemError** when the callable object exhausts the iterator

but forgets to either return a sentinel value or raise `StopIteration`.

- gh-87634 [https://github.com/python/cpython/issues/87634]: Remove locking behavior from `functools.cached_property()`.

- gh-97786 [https://github.com/python/cpython/issues/97786]: Fix potential undefined behaviour in corner cases of floating-point-to-time conversions.

- gh-101517 [https://github.com/python/cpython/issues/101517]: Fixed bug where `bdb` looks up the source line with `linecache` with a `lineno=None`, which causes it to fail with an unhandled exception.

- gh-101773 [https://github.com/python/cpython/issues/101773]: Optimize `fractions.Fraction` for small components. The private argument `_normalize` of the `fractions.Fraction` constructor has been removed.

- gh-101693 [https://github.com/python/cpython/issues/101693]: In `sqlite3.Cursor.execute()`, `DeprecationWarning` is now emitted when named placeholders are used together with parameters supplied as a sequence instead of as a `dict`. Starting from Python 3.14, using named placeholders with parameters supplied as a sequence will raise a `ProgrammingError`. Patch by Erlend E. Aasland.

- gh-101446 [https://github.com/python/cpython/issues/101446]: Change repr of `collections.OrderedDict` to use regular dictionary formatting instead of pairs of keys and values.

- gh-101362 [https://github.com/python/cpython/issues/101362]: Speed up `pathlib.PurePath` construction by handling arguments more uniformly. When a `pathlib.Path` argument is supplied, we use its string representation rather than joining its parts with `os.path.join()`.

- gh-101362 [https://github.com/python/cpython/issues/101362]: Speed up `pathlib.PurePath` construction by calling `os.path.join()` only when two or more arguments are given.

- gh-101362 [https://github.com/python/cpython/issues/101362]: Speed up `pathlib.Path` construction by running the path flavour compatibility check only when pathlib is imported.

- gh-85984 [https://github.com/python/cpython/issues/85984]: Refactored the implementation of `pty.fork()` to use `os.login_tty()`.

  A `DeprecationWarning` is now raised by `pty.master_open()` and `pty.slave_open()`. They were undocumented and deprecated long long ago in the docstring in favor of `pty.openpty()`.

- gh-101561 [https://github.com/python/cpython/issues/101561]: Add a new decorator `typing.override()`. See **PEP 698** [https://peps.python.org/pep-0698/] for details. Patch by Steven Troxler.

- gh-63301 [https://github.com/python/cpython/issues/63301]: Set exit code when `tabnanny` CLI exits on error.

- gh-101360 [https://github.com/python/cpython/issues/101360]: Fix anchor matching in **`pathlib.PureWindowsPath.match()`**. Path and pattern anchors are now matched with `fnmatch`, just like other path parts. This allows patterns such as `"*:/Users/*"` to be matched.

- gh-101277 [https://github.com/python/cpython/issues/101277]: Remove global state from `itertools` module (**PEP 687** [https://peps.python.org/pep-0687/]). Patches by Erlend E. Aasland.

- gh-100809 [https://github.com/python/cpython/issues/100809]: Fix handling of drive-relative paths (like ‘C:’ and ‘C:foo’) in `pathlib.Path.absolute()`. This method now uses the OS API to retrieve the correct current working directory for the drive.

- gh-99138 [https://github.com/python/cpython/issues/99138]: Apply **PEP 687** [https://peps.python.org/pep-0687/] to `zoneinfo`. Patch by Erlend E. Aasland.

- gh-96764 [https://github.com/python/cpython/issues/96764]: `asyncio.wait_for()` now uses `asyncio.timeout()` as its underlying implementation. Patch by Kumar Aditya.

- gh-88233 [https://github.com/python/cpython/issues/88233]: Correctly preserve “extra” fields in `zipfile` regardless of their ordering relative to a zip64 “extra.”

- bpo-23224 [https://bugs.python.org/issue?@action=redirect&bpo=23224]: Fix segfaults when creating **`lzma.LZMADecompressor`** and **`bz2.BZ2Decompressor`** objects without calling `__init__()`, and fix leakage of locks and internal buffers when calling the `__init__()` methods of **`lzma.LZMADecompressor`**, **`lzma.LZMACompressor`**, **`bz2.BZ2Compressor`**, and **`bz2.BZ2Decompressor`** objects multiple times.

Documentação

- gh-85417 [https://github.com/python/cpython/issues/85417]: Update **`cmath`** documentation to clarify behaviour on branch cuts.
- gh-97725 [https://github.com/python/cpython/issues/97725]: Fix **`asyncio.Task.print_stack()`** description for `file=None`. Patch by Oleg Iarygin.

Testes

- gh-102019 [https://github.com/python/cpython/issues/102019]: Fix deadlock on shutdown if `test_current_{exception,frames}` fails. Patch by Jacob Bower.
- gh-85984 [https://github.com/python/cpython/issues/85984]: Utilize new “winsize” functions from termios in pty tests.
- gh-89792 [https://github.com/python/cpython/issues/89792]: `test_tools` now copies up to 10x less source data to a temporary directory during the `freeze` test by ignoring git metadata and other artifacts. It also limits its python build parallelism based on os.cpu_count instead of hard coding it as 8 cores.

Construção

- gh-99942 [https://github.com/python/cpython/issues/99942]: On Android, in a static build, python-config in embed mode no longer incorrectly

reports a library to link to.

- gh-99942 [https://github.com/python/cpython/issues/99942]: On Android, python.pc now correctly reports the library to link to, the same as python-config.sh.
- gh-100221 [https://github.com/python/cpython/issues/100221]: Fix creating install directories in `make sharedinstall` if they exist outside `DESTDIR` already.
- gh-96821 [https://github.com/python/cpython/issues/96821]: Explicitly mark C extension modules that need defined signed integer overflow, and add a configure option **`--with-strict-overflow`**. Patch by Matthias Görgens and Shantanu Jain.

Windows

- gh-102344 [https://github.com/python/cpython/issues/102344]: Implement `winreg.QueryValue` using `QueryValueEx` and `winreg.SetValue` using `SetValueEx`. Patch by Max Bachmann.
- gh-101881 [https://github.com/python/cpython/issues/101881]: Handle read and write operations on non-blocking pipes properly on Windows.
- gh-101881 [https://github.com/python/cpython/issues/101881]: Add support for the os.get_blocking() and os.set_blocking() functions on Windows.
- gh-101849 [https://github.com/python/cpython/issues/101849]: Ensures installer will correctly upgrade existing `py.exe` launcher installs.
- gh-101763 [https://github.com/python/cpython/issues/101763]: Updates copy of libffi bundled with Windows installs to 3.4.4.
- gh-101759 [https://github.com/python/cpython/issues/101759]: Update Windows installer to SQLite 3.40.1.
- gh-101614 [https://github.com/python/cpython/issues/101614]: Correctly handle extensions built against debug binaries that reference `python3_d.dll`.
- gh-101196 [https://github.com/python/cpython/issues/101196]: The functions `os.path.isdir`, `os.path.isfile`, `os.path.islink` and `os.path.exists` are now 13% to 28% faster on Windows, by making fewer Win32 API calls.

macOS

- gh-101759 [https://github.com/python/cpython/issues/101759]: Update macOS installer to SQLite 3.40.1.

API C

- gh-101907 [https://github.com/python/cpython/issues/101907]: Removes use of non-standard C++ extension in public header files.

- gh-99293 [https://github.com/python/cpython/issues/99293]: Document that the Py_TPFLAGS_VALID_VERSION_TAG is an internal feature, should not be used, and will be removed.

- gh-101578 [https://github.com/python/cpython/issues/101578]: Add `PyErr_GetRaisedException()` and `PyErr_SetRaisedException()` for saving and restoring the current exception. These functions return and accept a single exception object, rather than the triple arguments of the now-deprecated `PyErr_Fetch()` and `PyErr_Restore()`. This is less error prone and a bit more efficient.

  Add `PyException_GetArgs()` and `PyException_SetArgs()` as convenience functions for retrieving and modifying the `args` passed to the exception's constructor.

- gh-91744 [https://github.com/python/cpython/issues/91744]: Introduced the *Unstable C API tier*, marking APi that is allowed to change in minor releases without a deprecation period. See **PEP 689** [https://peps.python.org/pep-0689/] for details.

Python 3.12.0 alpha 5

Data de lançamento: 2023-02-07

Segurança

- gh-99108 [https://github.com/python/cpython/issues/99108]: Replace the builtin `hashlib` implementations of SHA2-224 and SHA2-256 originally from LibTomCrypt with formally verified, side-channel resistant code

from the HACL* [https://github.com/hacl-star/hacl-star/] project. The builtins remain a fallback only used when OpenSSL does not provide them.

Núcleo e embutidos

- gh-92173 [https://github.com/python/cpython/issues/92173]: Fix the `defs` and `kwdefs` arguments to **`PyEval_EvalCodeEx()`** and a reference leak in that function.
- gh-59956 [https://github.com/python/cpython/issues/59956]: The GILState API is now partially compatible with subinterpreters. Previously, `PyThreadState_GET()` and `PyGILState_GetThisThreadState()` would get out of sync, causing inconsistent behavior and crashes.
- gh-101400 [https://github.com/python/cpython/issues/101400]: Fix wrong lineno in exception message on **`continue`** or **`break`** which are not in a loop. Patch by Donghee Na.
- gh-101372 [https://github.com/python/cpython/issues/101372]: Fix **`is_normalized()`** to properly handle the UCD 3.2.0 cases. Patch by Donghee Na.
- gh-101266 [https://github.com/python/cpython/issues/101266]: Fix **`sys.getsizeof()`** reporting for **`int`** subclasses.
- gh-101291 [https://github.com/python/cpython/issues/101291]: Refactor the `PyLongObject` struct into a normal Python object header and a `PyLongValue` struct.
- gh-101046 [https://github.com/python/cpython/issues/101046]: Fix a possible memory leak in the parser when raising **`MemoryError`**. Patch by Pablo Galindo
- gh-101037 [https://github.com/python/cpython/issues/101037]: Fix potential memory underallocation issue for instances of **`int`** subclasses with value zero.
- gh-100762 [https://github.com/python/cpython/issues/100762]: Record the (virtual) exception block depth in the oparg of **`YIELD_VALUE`**. Use this to avoid the expensive `throw()` when closing generators (and coroutines) that can be closed trivially.
- gh-100982 [https://github.com/python/cpython/issues/100982]: Adds a new **`COMPARE_AND_BRANCH`** instruction. This is a bit more efficient when performing a comparison immediately followed by a branch, and

restores the design intent of PEP 659 that specializations are local to a single instruction.

- gh-100942 [https://github.com/python/cpython/issues/100942]: Fixed segfault in property.getter/setter/deleter that occurred when a property subclass overrode the `__new__` method to return a non-property instance.
- gh-100923 [https://github.com/python/cpython/issues/100923]: Remove the `mask` cache entry for the **`COMPARE_OP`** instruction and embed the mask into the oparg.
- gh-100892 [https://github.com/python/cpython/issues/100892]: Fix race while iterating over thread states in clearing **`threading.local`**. Patch by Kumar Aditya.
- gh-91351 [https://github.com/python/cpython/issues/91351]: Fix a case where re-entrant imports could corrupt the import deadlock detection code and cause a **`KeyError`** to be raised out of **`importlib/_bootstrap`**. In addition to the straightforward cases, this could also happen when garbage collection leads to a warning being emitted – as happens when it collects an open socket or file)
- gh-100726 [https://github.com/python/cpython/issues/100726]: Optimize construction of `range` object for medium size integers.
- gh-100712 [https://github.com/python/cpython/issues/100712]: Added option to build cpython with specialization disabled, by setting `ENABLE_SPECIALIZATION=False` in **`opcode`**, followed by `make regen-all`.

Biblioteca

- bpo-32780 [https://bugs.python.org/issue?@action=redirect&bpo=32780]: Inter-field padding is now inserted into the PEP3118 format strings obtained from **`ctypes.Structure`** objects, reflecting their true representation in memory.
- gh-101541 [https://github.com/python/cpython/issues/101541]: [Enum] - fix psuedo-flag creation
- gh-101570 [https://github.com/python/cpython/issues/101570]: Upgrade pip wheel bundled with ensurepip (pip 23.0)
- gh-101323 [https://github.com/python/cpython/issues/101323]: Fix a bug where errors where not thrown by zlib._ZlibDecompressor if encountered during decompressing.

- gh-101317 [https://github.com/python/cpython/issues/101317]: Add *ssl_shutdown_timeout* parameter for **`asyncio.StreamWriter.start_tls()`**.
- gh-101326 [https://github.com/python/cpython/issues/101326]: Fix regression when passing `None` as second or third argument to `FutureIter.throw`.
- gh-92123 [https://github.com/python/cpython/issues/92123]: Adapt the `_elementtree` extension module to multi-phase init (**PEP 489** [https://peps.python.org/pep-0489/]). Patches by Erlend E. Aasland.
- gh-100795 [https://github.com/python/cpython/issues/100795]: Avoid potential unexpected `freeaddrinfo` call (double free) in **`socket`** when a libc `getaddrinfo()` implementation leaves garbage in an output pointer when returning an error. Original patch by Sergey G. Brester.
- gh-101143 [https://github.com/python/cpython/issues/101143]: Remove unused references to **`TimerHandle`** in `asyncio.base_events.BaseEventLoop._add_callback`.
- gh-101144 [https://github.com/python/cpython/issues/101144]: Make **`zipfile.Path.open()`** and **`zipfile.Path.read_text()`** also accept `encoding` as a positional argument. This was the behavior in Python 3.9 and earlier. 3.10 introduced a regression where supplying it as a positional argument would lead to a **`TypeError`**.
- gh-94518 [https://github.com/python/cpython/issues/94518]: Group-related variables of `_posixsubprocess` module are renamed to stress that supplementary group affinity is added to a fork, not replace the inherited ones. Patch by Oleg Iarygin.
- gh-101015 [https://github.com/python/cpython/issues/101015]: Fix **`typing.get_type_hints()`** on `'*tuple[...]'` and `*tuple[...]`. It must not drop the `Unpack` part.
- gh-101000 [https://github.com/python/cpython/issues/101000]: Add **`os.path.splitroot()`**, which splits a path into a 3-item tuple `(drive, root, tail)`. This new function is used by **`pathlib`** to improve the performance of path construction by up to a third.
- gh-100573 [https://github.com/python/cpython/issues/100573]: Fix a Windows **`asyncio`** bug with named pipes where a client doing `os.stat()` on the pipe would cause an error in the server that disabled serving future requests.
- gh-39615 [https://github.com/python/cpython/issues/39615]: **`warnings.warn()`** now has the ability to skip stack frames based on code filename prefix

rather than only a numeric `stacklevel` via the new `skip_file_prefixes` keyword argument.

- gh-100750 [https://github.com/python/cpython/issues/100750]: pass encoding kwarg to subprocess in platform
- gh-100160 [https://github.com/python/cpython/issues/100160]: Emit a deprecation warning in **`asyncio.DefaultEventLoopPolicy.get_event_loop()`** if there is no current event loop set and it decides to create one.
- gh-96290 [https://github.com/python/cpython/issues/96290]: Fix handling of partial and invalid UNC drives in `ntpath.splitdrive()`, and in `ntpath.normpath()` on non-Windows systems. Paths such as '\server' and '\' are now considered by `splitdrive()` to contain only a drive, and consequently are not modified by `normpath()` on non-Windows systems. The behaviour of `normpath()` on Windows systems is unaffected, as native OS APIs are used. Patch by Eryk Sun, with contributions by Barney Gale.
- gh-99952 [https://github.com/python/cpython/issues/99952]: Fix a reference undercounting issue in **`ctypes.Structure`** with `from_param()` results larger than a C pointer.
- gh-67790 [https://github.com/python/cpython/issues/67790]: Add float-style formatting support for **`fractions.Fraction`** instances.
- gh-99266 [https://github.com/python/cpython/issues/99266]: Preserve more detailed error messages in **`ctypes`**.
- gh-86682 [https://github.com/python/cpython/issues/86682]: Ensure runtime-created collections have the correct module name using the newly added (internal) **`sys._getframemodulename()`**.
- gh-88597 [https://github.com/python/cpython/issues/88597]: **`uuid`** now has a command line interface. Try `python -m uuid -h`.
- gh-60580 [https://github.com/python/cpython/issues/60580]: **`ctypes.wintypes.BYTE`** definition changed from **`c_byte`** to **`c_ubyte`** to match Windows SDK. Patch by Anatoly Techtonik and Oleg Iarygin.
- gh-94518 [https://github.com/python/cpython/issues/94518]: `_posixsubprocess` now initializes all UID and GID variables using a reserved `-1` value instead of a separate flag. Patch by Oleg Iarygin.
- bpo-38941 [https://bugs.python.org/issue?@action=redirect&bpo=38941]: The **`xml.etree.ElementTree`** module now emits **`DeprecationWarning`** when testing the truth value of an **`xml.etree.ElementTree.Element`**.

Before, the Python implementation emitted `FutureWarning`, and the C implementation emitted nothing.

- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert `elementtree` types to heap types. Patch by Erlend E. Aasland.
- bpo-29847 [https://bugs.python.org/issue?@action=redirect&bpo=29847]: Fix a bug where `pathlib.Path` accepted and ignored keyword arguments. Patch provided by Yurii Karabas.
- gh-77772 [https://github.com/python/cpython/issues/77772]: `ctypes.CDLL`, `ctypes.OleDLL`, `ctypes.WinDLL`, and `ctypes.PyDLL` now accept path-like objects as their `name` argument. Patch by Robert Hoelzl.

Documentação

- gh-88324 [https://github.com/python/cpython/issues/88324]: Reword `subprocess` to emphasize default behavior of *stdin*, *stdout*, and *stderr* arguments. Remove inaccurate statement about child file handle inheritance.

Testes

- gh-101334 [https://github.com/python/cpython/issues/101334]: `test_tarfile` has been updated to pass when run as a high UID.

Construção

- gh-101282 [https://github.com/python/cpython/issues/101282]: Update BOLT configuration not to use deprecated usage of `--split functions`. Patch by Donghee Na.
- gh-101522 [https://github.com/python/cpython/issues/101522]: Allow overriding Windows dependencies versions and paths using MSBuild properties.
- gh-77532 [https://github.com/python/cpython/issues/77532]: Minor fixes to allow building with `PlatformToolset=ClangCL` on Windows.
- gh-101152 [https://github.com/python/cpython/issues/101152]: In accordance with **PEP 699** [https://peps.python.org/pep-0699/], the `ma_version_tag` field in `PyDictObject` is deprecated for extension modules. Accessing this field will generate a compiler warning at compile time. This field will be removed in Python 3.14.

- gh-100340 [https://github.com/python/cpython/issues/100340]: Allows -Wno-int-conversion for wasm-sdk 17 and onwards, thus enables building WASI builds once against the latest sdk.
- gh-101060 [https://github.com/python/cpython/issues/101060]: Conditionally add `-fno-reorder-blocks-and-partition` in configure. Effectively fixes `--enable-bolt` when using Clang, as this appears to be a GCC-only flag.
- gh-98705 [https://github.com/python/cpython/issues/98705]: `__bool__` is defined in AIX system header files which breaks the build in AIX, so undefine it.
- gh-98636 [https://github.com/python/cpython/issues/98636]: Fix a regression in detecting `gdbm_compat` library for the `_gdbm` module build.
- gh-96305 [https://github.com/python/cpython/issues/96305]: `_aix_support` now uses a simple code to get platform details rather than the now non-existent `_bootsubprocess` during bootstrap.

Windows

- gh-101543 [https://github.com/python/cpython/issues/101543]: Ensure the install path in the registry is only used when the standard library hasn't been located in any other way.
- gh-101467 [https://github.com/python/cpython/issues/101467]: The `py.exe` launcher now correctly filters when only a single runtime is installed. It also correctly handles prefix matches on tags so that `-3.1` does not match `3.11`, but would still match `3.1-32`.
- gh-99834 [https://github.com/python/cpython/issues/99834]: Updates bundled copy of Tcl/Tk to 8.6.13.0
- gh-101135 [https://github.com/python/cpython/issues/101135]: Restore ability to launch older 32-bit versions from the `py.exe` launcher when both 32-bit and 64-bit installs of the same version are available.
- gh-82052 [https://github.com/python/cpython/issues/82052]: Fixed an issue where writing more than 32K of Unicode output to the console screen in one go can result in mojibake.
- gh-100320 [https://github.com/python/cpython/issues/100320]: Ensures the `PythonPath` registry key from an install is used when launching from a different copy of Python that relies on an existing install to provide a copy of its modules and standard library.

- gh-100247 [https://github.com/python/cpython/issues/100247]: Restores support for the `py.exe` launcher finding shebang commands in its configuration file using the full command name.

Python 3.12.0 alpha 4

Data de lançamento: 2023-01-10

Núcleo e embutidos

- gh-100776 [https://github.com/python/cpython/issues/100776]: Fix misleading default value in **`input()`**'s `__text_signature__`.

- gh-99005 [https://github.com/python/cpython/issues/99005]: Remove **`UNARY_POSITIVE`**, **`ASYNC_GEN_WRAP`** and **`LIST_TO_TUPLE`**, replacing them with intrinsics.

- gh-99005 [https://github.com/python/cpython/issues/99005]: Add new **`CALL_INTRINSIC_1`** instruction. Remove **`IMPORT_STAR`**, **`PRINT_EXPR`** and **`STOPITERATION_ERROR`**, replacing them with the **`CALL_INTRINSIC_1`** instruction.

- gh-100288 [https://github.com/python/cpython/issues/100288]: Remove the LOAD_ATTR_METHOD_WITH_DICT specialized instruction. Stats show it is not useful.

- gh-100720 [https://github.com/python/cpython/issues/100720]: Added `_PyFrame_NumSlotsForCodeObject`, which returns the number of slots needed in a frame for a given code object.

- gh-100719 [https://github.com/python/cpython/issues/100719]: Removed the co_nplaincellvars field from the code object, as it is redundant.

- gh-100637 [https://github.com/python/cpython/issues/100637]: Fix **`int.__sizeof__()`** calculation to include the 1-element `ob_digit` array for `0` and `False`.

- gh-100649 [https://github.com/python/cpython/issues/100649]: Update the native_thread_id field of PyThreadState after fork.

- gh-100126 [https://github.com/python/cpython/issues/100126]: Fix an issue where “incomplete” frames could be briefly visible to C code while other frames are being torn down, possibly resulting in corruption or hard crashes of the interpreter while running finalizers.

- gh-87447 [https://github.com/python/cpython/issues/87447]: Fix `SyntaxError` on comprehension rebind checking with names that are not actually redefined.

  Now reassigning `b` in `[(b := 1) for a, b.prop in some_iter]` is allowed. Reassigning `a` is still disallowed as per **PEP 572** [https://peps.python.org/pep-0572/].

- gh-100268 [https://github.com/python/cpython/issues/100268]: Add `int.is_integer()` to improve duck type compatibility between `int` and `float`.

- gh-100425 [https://github.com/python/cpython/issues/100425]: Improve the accuracy of `sum()` with compensated summation.

Biblioteca

- gh-100374 [https://github.com/python/cpython/issues/100374]: Fix incorrect result and delay in `socket.getfqdn()`. Patch by Dominic Socular.

Núcleo e embutidos

- gh-100357 [https://github.com/python/cpython/issues/100357]: Convert `vars`, `dir`, `next`, `getattr`, and `iter` to argument clinic.
- gh-100117 [https://github.com/python/cpython/issues/100117]: Improve the output of `codeobject.co_lines()` by emitting only one entry for each line range.
- gh-90043 [https://github.com/python/cpython/issues/90043]: Handle NaNs when specializing `COMPARE_OP` for `float` values.

- gh-100222 [https://github.com/python/cpython/issues/100222]: Redefine the `_Py_CODEUNIT` typedef as a union to describe its layout to the C compiler, avoiding type punning and improving clarity.
- gh-99955 [https://github.com/python/cpython/issues/99955]: Internal compiler functions (in compile.c) now consistently return -1 on error and 0 on success.
- gh-100188 [https://github.com/python/cpython/issues/100188]: The `BINARY_SUBSCR_LIST_INT` and `BINARY_SUBSCR_TUPLE_INT` instructions are no longer used for negative integers because those instructions always miss when encountering negative integers.
- gh-99110 [https://github.com/python/cpython/issues/99110]: Initialize frame->previous in frameobject.c to fix a segmentation fault when accessing frames created by **`PyFrame_New()`**.
- gh-94155 [https://github.com/python/cpython/issues/94155]: Improved the hashing algorithm for code objects, mitigating some hash collisions.
- gh-99540 [https://github.com/python/cpython/issues/99540]: `None` now hashes to a constant value. This is not a requirements change.
- gh-100143 [https://github.com/python/cpython/issues/100143]: When built with `--enable-pystats`, stats collection is now off by default. To enable it early at startup, pass the `-Xpystats` flag. Stats are now always dumped, even if switched off.
- gh-100146 [https://github.com/python/cpython/issues/100146]: Improve `BUILD_LIST` opcode so that it works similarly to the `BUILD_TUPLE` opcode, by stealing references from the stack rather than repeatedly using stack operations to set list elements. Implementation details are in a new private API **`_PyList_FromArraySteal()`**.
- gh-100110 [https://github.com/python/cpython/issues/100110]: Specialize `FOR_ITER` for tuples.
- gh-100050 [https://github.com/python/cpython/issues/100050]: Honor existing errors obtained when searching for mismatching parentheses in the tokenizer. Patch by Pablo Galindo
- gh-92216 [https://github.com/python/cpython/issues/92216]: Improve the performance of **`hasattr()`** for type objects with a missing attribute.
- gh-99582 [https://github.com/python/cpython/issues/99582]: Freeze **`zipimport`** module into `_bootstrap_python`.
- gh-99554 [https://github.com/python/cpython/issues/99554]: Pack debugging location tables more efficiently during bytecode compilation.

- gh-98522 [https://github.com/python/cpython/issues/98522]: Add an internal version number to code objects, to give better versioning of inner functions and comprehensions, and thus better specialization of those functions. This change is invisible to both Python and C extensions.
- gh-94603 [https://github.com/python/cpython/issues/94603]: Improve performance of `list.pop` for small lists.

Biblioteca

- gh-89051 [https://github.com/python/cpython/issues/89051]: Add **`ssl.OP_LEGACY_SERVER_CONNECT`**

Núcleo e embutidos

- bpo-32782 [https://bugs.python.org/issue?@action=redirect&bpo=32782]: `ctypes` arrays of length 0 now report a correct itemsize when a `memoryview` is constructed from them, rather than always giving a value of 0.

Biblioteca

- gh-100833 [https://github.com/python/cpython/issues/100833]: Speed up **`math.fsum()`** by removing defensive `volatile` qualifiers.
- gh-100805 [https://github.com/python/cpython/issues/100805]: Modify **`random.choice()`** implementation to once again work with NumPy arrays.
- gh-100813 [https://github.com/python/cpython/issues/100813]: Add **`socket.IP_PKTINFO`** constant.
- gh-100792 [https://github.com/python/cpython/issues/100792]: Make **`email.message.Message.__contains__()`** twice as fast.
- gh-91851 [https://github.com/python/cpython/issues/91851]: Microoptimizations for **`fractions.Fraction.__round__()`**, **`fractions.Fraction.__ceil__()`** and **`fractions.Fraction.__floor__()`**.
- gh-90104 [https://github.com/python/cpython/issues/90104]: Avoid RecursionError on `repr` if a dataclass field definition has a cyclic reference.
- gh-100689 [https://github.com/python/cpython/issues/100689]: Fix crash in **`pyexpat`** by statically allocating `PyExpat_CAPI` capsule.

- gh-100740 [https://github.com/python/cpython/issues/100740]: Fix `unittest.mock.Mock` not respecting the spec for attribute names prefixed with `assert`.
- gh-91219 [https://github.com/python/cpython/issues/91219]: Change `SimpleHTTPRequestHandler` to support subclassing to provide a different set of index file names instead of using `__init__` parameters.
- gh-100690 [https://github.com/python/cpython/issues/100690]: `Mock` objects which are not unsafe will now raise an `AttributeError` when accessing an attribute that matches the name of an assertion but without the prefix `assert_`, e.g. accessing `called_once` instead of `assert_called_once`. This is in addition to this already happening for accessing attributes with prefixes `assert`, `assret`, `asert`, `aseert`, and `assrt`.
- gh-89727 [https://github.com/python/cpython/issues/89727]: Simplify and optimize **`os.walk()`** by using **`isinstance()`** checks to check the top of the stack.
- gh-100485 [https://github.com/python/cpython/issues/100485]: Add math.sumprod() to compute the sum of products.
- gh-86508 [https://github.com/python/cpython/issues/86508]: Fix **`asyncio.open_connection()`** to skip binding to local addresses of different family. Patch by Kumar Aditya.
- gh-97930 [https://github.com/python/cpython/issues/97930]: `importlib.resources.files` now accepts a module as an anchor instead of only accepting packages. If a module is passed, resources are resolved adjacent to that module (in the same package or at the package root). The parameter was renamed from `package` to `anchor` with a compatibility shim for those passing by keyword. Additionally, the new `anchor` parameter is now optional and will default to the caller's module.
- gh-100585 [https://github.com/python/cpython/issues/100585]: Fixed a bug where importlib.resources.as_file was leaving file pointers open
- gh-100562 [https://github.com/python/cpython/issues/100562]: Improve performance of **`pathlib.Path.absolute()`** by nearly 2x. This comes at the cost of a performance regression in **`pathlib.Path.cwd()`**, which is generally used less frequently in user code.
- gh-100519 [https://github.com/python/cpython/issues/100519]: Small simplification of **`http.cookiejar.eff_request_host()`** that improves readability and better matches the RFC wording.

- gh-100287 [https://github.com/python/cpython/issues/100287]: Fix the interaction of **`unittest.mock.seal()`** with **`unittest.mock.AsyncMock`**.
- gh-100488 [https://github.com/python/cpython/issues/100488]: Add **`Fraction.is_integer()`** to check whether a **`fractions.Fraction`** is an integer. This improves duck type compatibility with **`float`** and **`int`**.
- gh-100474 [https://github.com/python/cpython/issues/100474]: **`http.server`** now checks that an index page is actually a regular file before trying to serve it. This avoids issues with directories named `index.html`.
- gh-100363 [https://github.com/python/cpython/issues/100363]: Speed up **`asyncio.get_running_loop()`** by removing redundant `getpid` checks. Patch by Kumar Aditya.
- gh-78878 [https://github.com/python/cpython/issues/78878]: Fix crash when creating an instance of **`_ctypes.CField`**.
- gh-100348 [https://github.com/python/cpython/issues/100348]: Fix ref cycle in **`asyncio._SelectorSocketTransport`** by removing `_read_ready_cb` in `close`.
- gh-100344 [https://github.com/python/cpython/issues/100344]: Provide C implementation for **`asyncio.current_task()`** for a 4x-6x speedup.
- gh-100272 [https://github.com/python/cpython/issues/100272]: Fix JSON serialization of OrderedDict. It now preserves the order of keys.
- gh-83076 [https://github.com/python/cpython/issues/83076]: Instantiation of `Mock()` and `AsyncMock()` is now 3.8x faster.
- gh-100234 [https://github.com/python/cpython/issues/100234]: Set a default value of 1.0 for the `lambd` parameter in random.expovariate().
- gh-100228 [https://github.com/python/cpython/issues/100228]: A **`DeprecationWarning`** may be raised when **`os.fork()`** or **`os.forkpty()`** is called from multi-threaded processes. Forking with threads is unsafe and can cause deadlocks, crashes and subtle problems. Lack of a warning does not indicate that the fork call was actually safe, as Python may not be aware of all threads.
- gh-100039 [https://github.com/python/cpython/issues/100039]: Improve signatures for enums and flags.
- gh-100133 [https://github.com/python/cpython/issues/100133]: Fix regression in **`asyncio`** where a subprocess would sometimes lose data received from pipe.
- bpo-44592 [https://bugs.python.org/issue?@action=redirect&bpo=44592]: Fixes inconsistent handling of case sensitivity of *extrasaction* arg in

csv.DictWriter.

- gh-100098 [https://github.com/python/cpython/issues/100098]: Fix `tuple` subclasses being cast to `tuple` when used as enum values.
- gh-85432 [https://github.com/python/cpython/issues/85432]: Rename the *fmt* parameter of the pure-Python implementation of **datetime.time.strftime()** to *format*. Rename the *t* parameter of **datetime.datetime.fromtimestamp()** to *timestamp*. These changes mean the parameter names in the pure-Python implementation now match the parameter names in the C implementation. Patch by Alex Waygood.
- gh-98778 [https://github.com/python/cpython/issues/98778]: Update **HTTPError** to be initialized properly, even if the `fp` is `None`. Patch by Donghee Na.
- gh-99925 [https://github.com/python/cpython/issues/99925]: Unify error messages in JSON serialization between `json.dumps(float('nan'), allow_nan=False)` and `json.dumps(float('nan'), allow_nan=False, indent=<SOMETHING>)`. Now both include the representation of the value that could not be serialized.
- gh-89727 [https://github.com/python/cpython/issues/89727]: Fix issue with **os.walk()** where a **RecursionError** would occur on deep directory structures by adjusting the implementation of **os.walk()** to be iterative instead of recursive.
- gh-94943 [https://github.com/python/cpython/issues/94943]: Add Suporte a dataclass to the **Enum __repr__()**. When inheriting from a **dataclass**, only show the field names in the value section of the member **repr()**, and not the dataclass' class name.
- gh-83035 [https://github.com/python/cpython/issues/83035]: Fix **inspect.getsource()** handling of decorator calls with nested parentheses.
- gh-99576 [https://github.com/python/cpython/issues/99576]: Fix `.save()` method for `LWPCookieJar` and `MozillaCookieJar`: saved file was not truncated on repeated save.
- gh-94912 [https://github.com/python/cpython/issues/94912]: Add **inspect.markcoroutinefunction()** decorator which manually marks a function as a coroutine for the benefit of **iscoroutinefunction()**.
- gh-99509 [https://github.com/python/cpython/issues/99509]: Add **PEP 585** [https://peps.python.org/pep-0585/] support for **multiprocessing.queues.Queue**.

- gh-99482 [https://github.com/python/cpython/issues/99482]: Remove `Jython` partial compatibility code from several stdlib modules.
- gh-99433 [https://github.com/python/cpython/issues/99433]: Fix **`doctest`** failure on **`types.MethodWrapperType`** in modules.
- gh-85267 [https://github.com/python/cpython/issues/85267]: Several improvements to **`inspect.signature()`**'s handling of `__text_signature`. - Fixes a case where **`inspect.signature()`** dropped parameters - Fixes a case where **`inspect.signature()`** raised **`tokenize.TokenError`** - Allows **`inspect.signature()`** to understand defaults involving binary operations of constants - **`inspect.signature()`** is documented as only raising **`TypeError`** or **`ValueError`**, but sometimes raised **`RuntimeError`**. These cases now raise **`ValueError`** - Removed a dead code path
- gh-91166 [https://github.com/python/cpython/issues/91166]: **`asyncio`** is optimized to avoid excessive copying when writing to socket and use **`sendmsg()`** if the platform supports it. Patch by Kumar Aditya.
- gh-98030 [https://github.com/python/cpython/issues/98030]: Add missing TCP socket options from Linux: `TCP_MD5SIG`, `TCP_THIN_LINEAR_TIMEOUTS`, `TCP_THIN_DUPACK`, `TCP_REPAIR`, `TCP_REPAIR_QUEUE`, `TCP_QUEUE_SEQ`, `TCP_REPAIR_OPTIONS`, `TCP_TIMESTAMP`, `TCP_CC_INFO`, `TCP_SAVE_SYN`, `TCP_SAVED_SYN`, `TCP_REPAIR_WINDOW`, `TCP_FASTOPEN_CONNECT`, `TCP_ULP`, `TCP_MD5SIG_EXT`, `TCP_FASTOPEN_KEY`, `TCP_FASTOPEN_NO_COOKIE`, `TCP_ZEROCOPY_RECEIVE`, `TCP_INQ`, `TCP_TX_DELAY`.
- gh-88500 [https://github.com/python/cpython/issues/88500]: Reduced the memory usage of **`urllib.parse.unquote()`** and **`urllib.parse.unquote_to_bytes()`** on large values.
- gh-96127 [https://github.com/python/cpython/issues/96127]: `inspect.signature` was raising `TypeError` on call with mock objects. Now it correctly returns `(*args, **kwargs)` as inferred signature.
- gh-95882 [https://github.com/python/cpython/issues/95882]: Fix a 3.11 regression in **`asynccontextmanager()`**, which caused it to propagate exceptions with incorrect tracebacks and fix a 3.11 regression in **`contextmanager()`**, which caused it to propagate exceptions with incorrect tracebacks for **`StopIteration`**.
- gh-78707 [https://github.com/python/cpython/issues/78707]: Deprecate passing more than one positional argument to

`pathlib.PurePath.relative_to()` and `is_relative_to()`.

- gh-92122 [https://github.com/python/cpython/issues/92122]: Fix reStructuredText syntax errors in docstrings in the `enum` module.
- gh-91851 [https://github.com/python/cpython/issues/91851]: Optimize the `Fraction` arithmetics for small components.
- bpo-24132 [https://bugs.python.org/issue?@action=redirect&bpo=24132]: Make `pathlib.PurePath` and `Path` subclassable (private to start). Previously, attempting to instantiate a subclass resulted in an `AttributeError` being raised. Patch by Barney Gale.
- bpo-40447 [https://bugs.python.org/issue?@action=redirect&bpo=40447]: Accept `os.PathLike` (such as `pathlib.Path`) in the `stripdir` arguments of `compileall.compile_file()` and `compileall.compile_dir()`.
- bpo-36880 [https://bugs.python.org/issue?@action=redirect&bpo=36880]: Fix a reference counting issue when a `ctypes` callback with return type `py_object` returns `None`, which could cause crashes.

Documentação

- gh-100616 [https://github.com/python/cpython/issues/100616]: Document existing `attr` parameter to `curses.window.vline()` function in `curses`.
- gh-100472 [https://github.com/python/cpython/issues/100472]: Remove claim in documentation that the `stripdir`, `prependdir` and `limit_sl_dest` parameters of `compileall.compile_dir()` and `compileall.compile_file()` could be `bytes`.
- bpo-25377 [https://bugs.python.org/issue?@action=redirect&bpo=25377]: Clarify use of octal format of mode argument in help(os.chmod) as well as help(os.fchmod)

Testes

- gh-100454 [https://github.com/python/cpython/issues/100454]: Start running SSL tests with OpenSSL 3.1.0-beta1.
- gh-100086 [https://github.com/python/cpython/issues/100086]: The Python test runner (libregrtest) now logs Python build information like “debug” vs “release” build, or LTO and PGO optimizations. Patch by Victor Stinner.

- gh-93018 [https://github.com/python/cpython/issues/93018]: Make two tests forgiving towards host system libexpat with backported security fixes applied.

Construção

- gh-100540 [https://github.com/python/cpython/issues/100540]: Removed the `--with-system-ffi` `configure` option; `libffi` must now always be supplied by the system on all non-Windows platforms. The option has had no effect on non-Darwin platforms for several releases, and in 3.11 only had the non-obvious effect of invoking `pkg-config` to find `libffi` and never setting `-DUSING_APPLE_OS_LIBFFI`. Now on Darwin platforms `configure` will first check for the OS `libffi` and then fall back to the same processing as other platforms if it is not found.
- gh-88267 [https://github.com/python/cpython/issues/88267]: Avoid exporting Python symbols in linked Windows applications when the core is built as static.
- bpo-41916 [https://bugs.python.org/issue?@action=redirect&bpo=41916]: Allow override of ac_cv_cxx_thread so that cross compiled python can set -pthread for CXX.

Windows

- gh-100180 [https://github.com/python/cpython/issues/100180]: Update Windows installer to OpenSSL 1.1.1s
- gh-99191 [https://github.com/python/cpython/issues/99191]: Use `_MSVC_LANG >= 202002L` instead of less-precise `_MSC_VER >=1929` to more accurately test for C++20 support in `PC/_wmimodule.cpp`.
- gh-79218 [https://github.com/python/cpython/issues/79218]: Define `MS_WIN64` for Mingw-w64 64bit, fix cython compilation failure.
- gh-99941 [https://github.com/python/cpython/issues/99941]: Ensure that **`asyncio.Protocol.data_received()`** receives an immutable **`bytes`** object (as documented), instead of **`bytearray`**.
- bpo-43984 [https://bugs.python.org/issue?@action=redirect&bpo=43984]: **`winreg.SetValueEx()`** now leaves the target value untouched in the case of conversion errors. Previously, `-1` would be written in case of such errors.

- bpo-34816 [https://bugs.python.org/issue?@action=redirect&bpo=34816]: `hasattr(ctypes.windll, 'nonexistant')` now returns `False` instead of raising `OSError`.

macOS

- gh-100180 [https://github.com/python/cpython/issues/100180]: Update macOS installer to OpenSSL 1.1.1s
- gh-100540 [https://github.com/python/cpython/issues/100540]: Removed obsolete `dlfcn.h` shim from the `_ctypes` extension module, which has not been necessary since Mac OS X 10.2.

Ferramentas/Demos

- bpo-45256 [https://bugs.python.org/issue?@action=redirect&bpo=45256]: Fix a bug that caused an `AttributeError` to be raised in `python-gdb.py` when `py-locals` is used without a frame.
- gh-100342 [https://github.com/python/cpython/issues/100342]: Add missing `NULL` check for possible allocation failure in `*args` parsing in Argument Clinic.

API C

- gh-99947 [https://github.com/python/cpython/issues/99947]: Raising SystemError on import will now have its cause be set to the original unexpected exception.
- gh-99240 [https://github.com/python/cpython/issues/99240]: In argument parsing, after deallocating newly allocated memory, reset its pointer to NULL.
- gh-98724 [https://github.com/python/cpython/issues/98724]: The `Py_CLEAR`, `Py_SETREF` and `Py_XSETREF` macros now only evaluate their arguments once. If an argument has side effects, these side effects are no longer duplicated. Patch by Victor Stinner.

Python 3.12.0 alpha 3

Data de lançamento: 2022-12-06

Segurança

- gh-100001 [https://github.com/python/cpython/issues/100001]: `python -m http.server` no longer allows terminal control characters sent within a garbage request to be printed to the stderr server log.

  This is done by changing the **`http.server`** **`BaseHTTPRequestHandler`** `.log_message` method to replace control characters with a `\x`*`HH`* hex escape before printing.

- gh-87604 [https://github.com/python/cpython/issues/87604]: Avoid publishing list of active per-interpreter audit hooks via the **`gc`** module

Núcleo e embutidos

- gh-99891 [https://github.com/python/cpython/issues/99891]: Fix a bug in the tokenizer that could cause infinite recursion when showing syntax warnings that happen in the first line of the source. Patch by Pablo Galindo
- gh-91054 [https://github.com/python/cpython/issues/91054]: Add **`PyCode_AddWatcher()`** and **`PyCode_ClearWatcher()`** APIs to register callbacks to receive notification on creation and destruction of code objects.
- gh-99729 [https://github.com/python/cpython/issues/99729]: Fix an issue that could cause frames to be visible to Python code as they are being torn down, possibly leading to memory corruption or hard crashes of the interpreter.
- gh-99708 [https://github.com/python/cpython/issues/99708]: Fix bug where compiler crashes on an if expression with an empty body block.
- gh-99578 [https://github.com/python/cpython/issues/99578]: Fix a reference bug in **`_imp.create_builtin()`** after the creation of the first sub-interpreter for modules `builtins` and `sys`. Patch by Victor Stinner.
- gh-99581 [https://github.com/python/cpython/issues/99581]: Fixed a bug that was causing a buffer overflow if the tokenizer copies a line missing the newline character from a file that is as long as the available tokenizer buffer. Patch by Pablo galindo

- gh-99553 [https://github.com/python/cpython/issues/99553]: Fix bug where an `ExceptionGroup` subclass can wrap a `BaseException`.

Biblioteca

- gh-99547 [https://github.com/python/cpython/issues/99547]: Add a function to os.path to check if a path is a junction: isjunction. Add similar functionality to pathlib.Path as is_junction.
- gh-99370 [https://github.com/python/cpython/issues/99370]: Fix zip path for venv created from a non-installed python on POSIX platforms.

Núcleo e embutidos

- gh-99377 [https://github.com/python/cpython/issues/99377]: Add audit events for thread creation and clear operations.
- gh-98686 [https://github.com/python/cpython/issues/98686]: Remove the `BINARY_OP_GENERIC` and `COMPARE_OP_GENERIC` "specializations".
- gh-99298 [https://github.com/python/cpython/issues/99298]: Remove the remaining error paths for attribute specializations, and refuse to specialize attribute accesses on types that haven't had `PyType_Ready()` called on them yet.
- gh-99127 [https://github.com/python/cpython/issues/99127]: Allow some features of `syslog` to the main interpreter only. Patch by Donghee Na.
- gh-91053 [https://github.com/python/cpython/issues/91053]: Optimizing interpreters and JIT compilers may need to invalidate internal metadata when functions are modified. This change adds the ability to provide a callback that will be invoked each time a function is created, modified, or destroyed.
- gh-90994 [https://github.com/python/cpython/issues/90994]: Improve error messages when there's a syntax error with call arguments. The following three cases are covered: - No value is assigned to a named argument, eg `foo(a=)`. - A value is assigned to a star argument, eg `foo(*args=[0])`. - A value is assigned to a double-star keyword argument, eg `foo(**kwarg={'a': 0})`.
- bpo-45026 [https://bugs.python.org/issue?@action=redirect&bpo=45026]: Optimize the `range` object iterator. It is now smaller, faster iteration of ranges containing large numbers. Smaller pickles, faster unpickling.

- bpo-31718 [https://bugs.python.org/issue?@action=redirect&bpo=31718]: Raise `ValueError` instead of `SystemError` when methods of uninitialized `io.IncrementalNewlineDecoder` objects are called. Patch by Oren Milman.
- bpo-38031 [https://bugs.python.org/issue?@action=redirect&bpo=38031]: Fix a possible assertion failure in `io.FileIO` when the opener returns an invalid file descriptor.

Biblioteca

- gh-100001 [https://github.com/python/cpython/issues/100001]: Also escape s in the http.server BaseHTTPRequestHandler.log_message so that it is technically possible to parse the line and reconstruct what the original data was. Without this a xHH is ambiguous as to if it is a hex replacement we put in or the characters r"x" came through in the original request line.

- gh-99957 [https://github.com/python/cpython/issues/99957]: Add `frozen_default` parameter to `typing.dataclass_transform()`.

- gh-79033 [https://github.com/python/cpython/issues/79033]: Fix `asyncio.Server.wait_closed()` to actually do what the docs promise – wait for all existing connections to complete, after closing the server.

- gh-51524 [https://github.com/python/cpython/issues/51524]: Fix bug when calling trace.CoverageResults with valid infile.

- gh-99645 [https://github.com/python/cpython/issues/99645]: Fix a bug in handling class cleanups in `unittest.TestCase`. Now `addClassCleanup()` uses separate lists for different `TestCase` subclasses, and `doClassCleanups()` only cleans up the particular class.

- gh-99508 [https://github.com/python/cpython/issues/99508]: Fix `TypeError` in `Lib/importlib/_bootstrap_external.py` while calling `_imp.source_hash()`.

- gh-66285 [https://github.com/python/cpython/issues/66285]: Fix `asyncio` to not share event loop and signal wakeupfd in forked processes. Patch by

Kumar Aditya.

- gh-97001 [https://github.com/python/cpython/issues/97001]: Release the GIL when calling termios APIs to avoid blocking threads.

- gh-92647 [https://github.com/python/cpython/issues/92647]: Use final status of an enum to determine lookup or creation branch of functional API.

- gh-99388 [https://github.com/python/cpython/issues/99388]: Add *loop_factory* parameter to **`asyncio.run()`** to allow specifying a custom event loop factory. Patch by Kumar Aditya.

- gh-99341 [https://github.com/python/cpython/issues/99341]: Fix **`ast.increment_lineno()`** to also cover **`ast.TypeIgnore`** when changing line numbers.

- gh-99382 [https://github.com/python/cpython/issues/99382]: Check the number of arguments in substitution in user generics containing a **`TypeVarTuple`** and one or more **`TypeVar`**.

- gh-99379 [https://github.com/python/cpython/issues/99379]: Fix substitution of **`ParamSpec`** followed by **`TypeVarTuple`** in generic aliases.

- gh-99344 [https://github.com/python/cpython/issues/99344]: Fix substitution of **`TypeVarTuple`** and **`ParamSpec`** together in user generics.

- gh-99284 [https://github.com/python/cpython/issues/99284]: Remove `_use_broken_old_ctypes_structure_semantics_` old untested and undocumented hack from **`ctypes`**.

- gh-99201 [https://github.com/python/cpython/issues/99201]: Fix **`IndexError`** when initializing the config variables on Windows if `HAVE_DYNAMIC_LOADING` is not set.

- gh-99240 [https://github.com/python/cpython/issues/99240]: Fix double-free bug in Argument Clinic `str_converter` by extracting memory clean up to a new `post_parsing` section.

- gh-64490 [https://github.com/python/cpython/issues/64490]: Fix refcount error when arguments are packed to tuple in Argument Clinic.

- gh-99029 [https://github.com/python/cpython/issues/99029]: **`pathlib.PurePath.relative_to()`** now treats naked Windows drive paths as relative. This brings its behaviour in line with other parts of pathlib.

- gh-98253 [https://github.com/python/cpython/issues/98253]: The implementation of the typing module is now more resilient to reference leaks in binary extension modules.

  Previously, a reference leak in a typed C API-based extension module could leak internals of the typing module, which could in turn introduce leaks in essentially any other package with typed function signatures. Although the typing package is not the original source of the problem, such non-local dependences exacerbate debugging of large-scale projects, and the implementation was therefore changed to reduce harm by providing better isolation.

- gh-98458 [https://github.com/python/cpython/issues/98458]: Fix infinite loop in unittest when a self-referencing chained exception is raised

- gh-93453 [https://github.com/python/cpython/issues/93453]: **`asyncio.get_event_loop()`** and many other **`asyncio`** functions like **`asyncio.ensure_future()`**, **`asyncio.shield()`** or **`asyncio.gather()`**, and also the **`get_event_loop()`** method of **`asyncio.BaseDefaultEventLoopPolicy`** now raise a **`RuntimeError`** if called when there is no running event loop and the current event loop was not set. Previously they implicitly created and set a new current event loop. **`DeprecationWarning`** is no longer emitted if there is no running event loop but the current event loop was set.

- gh-97966 [https://github.com/python/cpython/issues/97966]: On **`os.uname_result`**, restored expectation that `_fields` and `_asdict` would include all six properties including `processor`.

- gh-98248 [https://github.com/python/cpython/issues/98248]: Provide informative error messages in **`struct.pack()`** when its integral arguments are not in range.

- gh-98108 [https://github.com/python/cpython/issues/98108]: `zipfile.Path` is now pickleable if its initialization parameters were pickleable (e.g. for file system paths).

- gh-98098 [https://github.com/python/cpython/issues/98098]: Created packages from zipfile and test_zipfile modules, separating `zipfile.Path` functionality.

- gh-82836 [https://github.com/python/cpython/issues/82836]: Fix **`is_private`** properties in the **`ipaddress`** module. Previously non-private networks (0.0.0.0/0) would return `True` from this method; now they correctly return `False`.

- gh-96828 [https://github.com/python/cpython/issues/96828]: Add an **`OP_ENABLE_KTLS`** option for enabling the use of the kernel TLS (kTLS). Patch by Illia Volochii.

- gh-88863 [https://github.com/python/cpython/issues/88863]: To avoid apparent memory leaks when **`asyncio.open_connection()`** raises, break reference cycles generated by local exception and future instances (which has exception instance as its member var). Patch by Dong Uk, Kang.

- gh-91078 [https://github.com/python/cpython/issues/91078]: **`TarFile.next()`** now returns `None` when called on an empty tarfile.

- bpo-47220 [https://bugs.python.org/issue?@action=redirect&bpo=47220]: Document the optional *callback* parameter of **`WeakMethod`**. Patch by Géry Ogam.

- bpo-44817 [https://bugs.python.org/issue?@action=redirect&bpo=44817]: Ignore WinError 53 (ERROR_BAD_NETPATH), 65 (ERROR_NETWORK_ACCESS_DENIED) and 161 (ERROR_BAD_PATHNAME) when using ntpath.realpath().

- bpo-41260 [https://bugs.python.org/issue?@action=redirect&bpo=41260]: Rename the *fmt* parameter of the pure Python implementation of **`datetime.date.strftime()`** to *format*.

- bpo-15999 [https://bugs.python.org/issue?@action=redirect&bpo=15999]: All built-in functions now accept arguments of any type instead of just `bool` and `int` for boolean parameters.

Documentação

- gh-99931 [https://github.com/python/cpython/issues/99931]: Use sphinxext-opengraph [https://sphinxext-opengraph.readthedocs.io/] to generate OpenGraph metadata [https://ogp.me/].
- gh-89682 [https://github.com/python/cpython/issues/89682]: Reworded docstring of the default `__contains__` to clarify that it returns a **`bool`**.
- gh-88330 [https://github.com/python/cpython/issues/88330]: Improved the description of what a resource is in importlib.resources docs.
- gh-92892 [https://github.com/python/cpython/issues/92892]: Document that calling variadic functions with ctypes requires special care on macOS/arm64 (and possibly other platforms).
- bpo-41825 [https://bugs.python.org/issue?@action=redirect&bpo=41825]: Restructured the documentation for the **`os.wait*`** family of functions, and improved the docs for **`os.waitid()`** with more explanation of the possible argument constants.

Testes

- gh-99892 [https://github.com/python/cpython/issues/99892]: Skip test_normalization() of test_unicodedata if it fails to download NormalizationTest.txt file from pythontest.net. Patch by Victor Stinner.
- gh-99934 [https://github.com/python/cpython/issues/99934]: Correct test_marsh on (32 bit) x86: test_deterministic sets was failing.
- gh-99741 [https://github.com/python/cpython/issues/99741]: We've implemented multi-phase init (PEP 489/630/687) for the internal (for testing) _xxsubinterpreters module.
- gh-99659 [https://github.com/python/cpython/issues/99659]: Optional big memory tests in `test_sqlite3` now catch the correct **`sqlite.DataError`**

exception type in case of too large strings and/or blobs passed.

- gh-99593 [https://github.com/python/cpython/issues/99593]: Cover the Unicode C API with tests.
- gh-96002 [https://github.com/python/cpython/issues/96002]: Add functional test for Argument Clinic.

Construção

- gh-99086 [https://github.com/python/cpython/issues/99086]: Fix `-Wimplicit-int`, `-Wstrict-prototypes`, and `-Wimplicit-function-declaration` compiler warnings in **configure** checks.
- gh-99337 [https://github.com/python/cpython/issues/99337]: Fix a compilation issue with GCC 12 on macOS.
- gh-99289 [https://github.com/python/cpython/issues/99289]: Add a `COMPILEALL_OPTS` variable in Makefile to override **`compileall`** options (default: `-j0`) in `make install`. Also merged the `compileall` commands into a single command building .pyc files for the all optimization levels (0, 1, 2) at once. Patch by Victor Stinner.
- gh-98872 [https://github.com/python/cpython/issues/98872]: Fix a possible fd leak in `Programs/_freeze_module.c` introduced in Python 3.11.
- gh-88226 [https://github.com/python/cpython/issues/88226]: Always define `TARGET_*` labels in `Python/ceval.c`, even if `USE_COMPUTED_GOTOS` is disabled. This allows breakpoints to be set at those labels in (for instance) `gdb`.

Windows

- gh-99345 [https://github.com/python/cpython/issues/99345]: Use faster initialization functions to detect install location for Windows Store package
- gh-98629 [https://github.com/python/cpython/issues/98629]: Fix initialization of **`sys.version`** and `sys._git` on Windows
- gh-99442 [https://github.com/python/cpython/issues/99442]: Fix handling in Gerenciador de instalação do Python when `argv[0]` does not include a file extension.
- bpo-40882 [https://bugs.python.org/issue?@action=redirect&bpo=40882]: Fix a memory leak in **`multiprocessing.shared_memory.SharedMemory`** on

Windows.

macOS

- gh-87235 [https://github.com/python/cpython/issues/87235]: On macOS `python3 /dev/fd/9 9</path/to/script.py` failed for any script longer than a couple of bytes.
- gh-98940 [https://github.com/python/cpython/issues/98940]: Fix `Mac/Extras.install.py` file filter bug.

Ferramentas/Demos

- gh-64490 [https://github.com/python/cpython/issues/64490]: Argument Clinic varargs bugfixes
  - Fix out-of-bounds error in **`_PyArg_UnpackKeywordsWithVararg()`**.
  - Fix incorrect check which allowed more than one varargs in clinic.py.
  - Fix miscalculation of `noptargs` in generated code.
  - Do not generate `noptargs` when there is a vararg argument and no optional argument.

API C

- gh-98680 [https://github.com/python/cpython/issues/98680]: `PyBUF_*` constants were marked as part of Limited API of Python 3.11+. These were available in 3.11.0 with **`Py_LIMITED_API`** defined for 3.11, and are necessary to use the buffer API.
- gh-99612 [https://github.com/python/cpython/issues/99612]: Fix **`PyUnicode_DecodeUTF8Stateful()`** for ASCII-only data: `*consumed` was not set.
- gh-47146 [https://github.com/python/cpython/issues/47146]: The `structmember.h` header is deprecated. Its non-deprecated contents are now available just by including `Python.h`, with a `Py_` prefix added if it was missing. (Deprecated contents are **`T_OBJECT`**, **`T_NONE`**, and no-op flags.) Patch by Petr Viktorin, based on earlier work by Alexander Belopolsky and Matthias Braun.

Python 3.12.0 alpha 2

Data de lançamento: 2022-11-14

Segurança

- gh-98433 [https://github.com/python/cpython/issues/98433]: The IDNA codec decoder used on DNS hostnames by `socket` or `asyncio` related name resolution functions no longer involves a quadratic algorithm. This prevents a potential CPU denial of service if an out-of-spec excessive length hostname involving bidirectional characters were decoded. Some protocols such as `urllib` http `3xx` redirects potentially allow for an attacker to supply such a name.

  Individual labels within an IDNA encoded DNS name will now raise an error early during IDNA decoding if they are longer than 1024 unicode characters given that each decoded DNS label must be 63 or fewer characters and the entire decoded DNS name is limited to 255. Only an application presenting a hostname or label consisting primarily of **RFC 3454** [https://datatracker.ietf.org/doc/html/rfc3454.html] section 3.1 “Nothing” characters to be removed would run into of this new limit. See also **RFC 5894** [https://datatracker.ietf.org/doc/html/rfc5894.html] section 6 and **RFC 3491** [https://datatracker.ietf.org/doc/html/rfc3491.html].

- gh-98739 [https://github.com/python/cpython/issues/98739]: Update bundled libexpat to 2.5.0

Núcleo e embutidos

- gh-81057 [https://github.com/python/cpython/issues/81057]: The docs clearly say that `PyImport_Inittab`, `PyImport_AppendInittab()`, and `PyImport_ExtendInittab()` should not be used after `Py_Initialize()` has been called. We now enforce this for the two functions. Additionally, the runtime now uses an internal copy of `PyImport_Inittab`, to guard against modification.

- gh-99298 [https://github.com/python/cpython/issues/99298]: Fix an issue that could potentially cause incorrect error handling for some bytecode instructions.

- gh-99254 [https://github.com/python/cpython/issues/99254]: The compiler now removes all unused constants from code objects (except the first one, which may be a docstring).

- gh-99205 [https://github.com/python/cpython/issues/99205]: Fix an issue that prevented **`PyThreadState`** and **`PyInterpreterState`** memory from being freed properly.

- gh-81057 [https://github.com/python/cpython/issues/81057]: The 18 global C variables holding the state of the allocators have been moved to `_PyRuntimeState`. This is a strictly internal change with no change in behavior.

- gh-99181 [https://github.com/python/cpython/issues/99181]: Fix failure in **`except*`** with unhashable exceptions.

- gh-99204 [https://github.com/python/cpython/issues/99204]: Fix calculation of **`sys._base_executable`** when inside a POSIX virtual environment using copies of the python binary when the base installation does not provide the executable name used by the venv. Calculation will fall back to alternative names (“python<MAJOR>”, “python<MAJOR>.<MINOR>”).

- gh-96055 [https://github.com/python/cpython/issues/96055]: Update **`faulthandler`** to emit an error message with the proper unexpected signal number. Patch by Donghee Na.

- gh-99153 [https://github.com/python/cpython/issues/99153]: Fix location of **`SyntaxError`** for a **`try`** block with both **`except`** and **`except*`**.

- gh-98686 [https://github.com/python/cpython/issues/98686]: Merge the adaptive opcode logic into each instruction’s unquickened variant, and merge the logic in `EXTENDED_ARG_QUICK` into **`EXTENDED_ARG`**. With these changes, the quickening that happens at code object creation is now

only responsible for initializing warmup counters and inserting superinstructions.

- gh-99103 [https://github.com/python/cpython/issues/99103]: Fix the error reporting positions of specialized traceback anchors when the source line contains Unicode characters.

- gh-99139 [https://github.com/python/cpython/issues/99139]: Improve the error suggestion for **`NameError`** exceptions for instances. Now if a **`NameError`** is raised in a method and the instance has an attribute that's exactly equal to the name in the exception, the suggestion will include `self.<NAME>` instead of the closest match in the method scope. Patch by Pablo Galindo

- gh-98401 [https://github.com/python/cpython/issues/98401]: Octal escapes with value larger than `0o377` (ex: `"\477"`), deprecated in Python 3.11, now produce a **`SyntaxWarning`**, instead of **`DeprecationWarning`**. In a future Python version they will be eventually a **`SyntaxError`**. Patch by Victor Stinner.

- gh-98401 [https://github.com/python/cpython/issues/98401]: A backslash-character pair that is not a valid escape sequence now generates a **`SyntaxWarning`**, instead of **`DeprecationWarning`**. For example, `re.compile("\d+\.\d+")` now emits a **`SyntaxWarning`** (`"\d"` is an invalid escape sequence), use raw strings for regular expression: `re.compile(r"\d+\.\d+")`. In a future Python version, **`SyntaxError`** will eventually be raised, instead of **`SyntaxWarning`**. Patch by Victor Stinner.

- gh-96793 [https://github.com/python/cpython/issues/96793]: Handle StopIteration and StopAsyncIteration raised in generator or coroutines in the bytecode, rather than in wrapping C code.

- gh-98931 [https://github.com/python/cpython/issues/98931]: Improve the **`SyntaxError`** error message when the user types `import x from y` instead of `from y import x`. Patch by Pablo Galindo

- gh-98852 [https://github.com/python/cpython/issues/98852]: Fix subscription of type aliases containing bare generic types or types like `TypeVar`: for example `tuple[A, T][int]` and `tuple[TypeVar, T][int]`, where `A` is a generic type, and `T` is a type variable.

- gh-98925 [https://github.com/python/cpython/issues/98925]: Lower the recursion depth for marshal on WASI to support (in-development) wasmtime 2.0.

- gh-98783 [https://github.com/python/cpython/issues/98783]: Fix multiple crashes in debug mode when `str` subclasses are used instead of `str` itself.

- gh-98811 [https://github.com/python/cpython/issues/98811]: Use complete source locations to simplify detection of `__future__` imports which are not at the beginning of the file. Also corrects the offset in the exception raised in one case, which was off by one and impeded highlighting.

- gh-96793 [https://github.com/python/cpython/issues/96793]: Add specialization of `FOR_ITER` for generators. Saves multiple layers of dispatch and checking to get from the `FOR_ITER` instruction in the caller to the `RESUME` in the generator.

- gh-98762 [https://github.com/python/cpython/issues/98762]: Fix source locations of `match` sub-patterns.

- gh-98586 [https://github.com/python/cpython/issues/98586]: Added the methods `PyObject_Vectorcall()` and `PyObject_VectorcallMethod()` to the Limited API along with the auxiliary macro constant `PY_VECTORCALL_ARGUMENTS_OFFSET`.

  The availability of these functions enables more efficient **PEP 590** [https://peps.python.org/pep-0590/] vector calls from binary extension modules that avoid argument boxing/unboxing overheads.

- gh-99257 [https://github.com/python/cpython/issues/99257]: Fix an issue where member descriptors (such as those for `__slots__`) could behave incorrectly or crash instead of raising a `TypeError` when accessed via an instance of an invalid type.

- gh-93143 [https://github.com/python/cpython/issues/93143]: Rather than changing **co_code**, the interpreter will now display a **RuntimeWarning** and assign **None** to any fast locals that are left unbound after jumps or **del** statements executed while tracing.

- gh-96421 [https://github.com/python/cpython/issues/96421]: When calling into Python code from C code, through **PyEval_EvalFrameEx()** or a related C-API function, a shim frame in inserted into the call stack. This occurs in the `_PyEval_EvalFrameDefault()` function. The extra frame should be invisible to all Python and most C extensions, but out-of-process profilers and debuggers need to be aware of it. These shim frames can be detected by checking `frame->owner == FRAME_OWNED_BY_CSTACK`.

  Extensions implementing their own interpreters using PEP 523 need to be aware of this shim frame and the changes to the semantics of **RETURN_VALUE**, **YIELD_VALUE**, and **RETURN_GENERATOR**, which now clear the frame.

Construção

- gh-98415 [https://github.com/python/cpython/issues/98415]: Fix detection of MAC addresses for **uuid** on certain OSs. Patch by Chaim Sanders

Núcleo e embutidos

- gh-98686 [https://github.com/python/cpython/issues/98686]: Quicken all code objects, and specialize adaptive bytecode instructions more aggressively.
- gh-92119 [https://github.com/python/cpython/issues/92119]: Print exception class name instead of its string representation when raising errors from **ctypes** calls.
- gh-91058 [https://github.com/python/cpython/issues/91058]: **ImportError** raised from failed `from <module> import <name>` now include suggestions for the value of `<name>` based on the available names in `<module>`. Patch by Pablo Galindo

- gh-96793 [https://github.com/python/cpython/issues/96793]: The **`FOR_ITER`** now leaves the iterator on the stack on termination of the loop. This is to assist specialization of loops for generators.
- gh-90716 [https://github.com/python/cpython/issues/90716]: Add _pylong.py module. It includes asymptotically faster algorithms that can be used for operations on integers with many digits. It is used by longobject.c to speed up some operations.
- gh-95389 [https://github.com/python/cpython/issues/95389]: Expose **`ETH_P_ALL`** and some of the ETHERTYPE_* constants in **`socket`**. Patch by Noam Cohen.

Biblioteca

- gh-93696 [https://github.com/python/cpython/issues/93696]: Allow **`pdb`** to locate source for frozen modules in the standard library.
- gh-99418 [https://github.com/python/cpython/issues/99418]: Fix bug in **`urllib.parse.urlparse()`** that causes URL schemes that begin with a digit, a plus sign, or a minus sign to be parsed incorrectly.
- gh-94597 [https://github.com/python/cpython/issues/94597]: Deprecate **`asyncio.AbstractChildWatcher`** to be removed in Python 3.14. Patch by Kumar Aditya.
- gh-99305 [https://github.com/python/cpython/issues/99305]: Improve performance of **`secrets.token_hex()`**.
- gh-74044 [https://github.com/python/cpython/issues/74044]: Fixed bug where **`inspect.signature()`** reported incorrect arguments for decorated methods.
- gh-99275 [https://github.com/python/cpython/issues/99275]: Fix `SystemError` in **`ctypes`** when exception was not set during `__initsubclass__`.
- gh-99277 [https://github.com/python/cpython/issues/99277]: Remove older version of `_SSLProtocolTransport.get_write_buffer_limits` in **`asyncio.sslproto`**
- gh-99248 [https://github.com/python/cpython/issues/99248]: fix negative numbers failing in verify()
- gh-99155 [https://github.com/python/cpython/issues/99155]: Fix **`statistics.NormalDist`** pickle with `0` and `1` protocols.
- gh-93464 [https://github.com/python/cpython/issues/93464]: `enum.auto()` is now correctly activated when combined with other assignment values. E.g.

`ONE = auto(), 'some text'` will now evaluate as `(1, 'some text')`.

- gh-99134 [https://github.com/python/cpython/issues/99134]: Update the bundled copy of pip to version 22.3.1.
- gh-92584 [https://github.com/python/cpython/issues/92584]: Remove the `distutils` package. It was deprecated in Python 3.10 by **PEP 632** [https://peps.python.org/pep-0632/] "Deprecate distutils module". For projects still using `distutils` and cannot be updated to something else, the `setuptools` project can be installed: it still provides `distutils`. Patch by Victor Stinner.
- gh-98999 [https://github.com/python/cpython/issues/98999]: Now **`_pyio`** is consistent with **`_io`** in raising `ValueError` when executing methods over closed buffers.
- gh-83004 [https://github.com/python/cpython/issues/83004]: Clean up refleak on failed module initialisation in **`_zoneinfo`**
- gh-83004 [https://github.com/python/cpython/issues/83004]: Clean up refleaks on failed module initialisation in **`_pickle`**
- gh-83004 [https://github.com/python/cpython/issues/83004]: Clean up refleak on failed module initialisation in **`_io`**.
- gh-98897 [https://github.com/python/cpython/issues/98897]: Fix memory leak in **`math.dist()`** when both points don't have the same dimension. Patch by Kumar Aditya.
- gh-98878 [https://github.com/python/cpython/issues/98878]: Use the frame bound builtins when offering a name suggestion in **`traceback`** to prevent crashing when `__builtins__` is not a dict.
- gh-98139 [https://github.com/python/cpython/issues/98139]: In **`importlib._bootstrap`**, enhance namespace package repr to `<module 'x' (namespace) from ['path']>`.
- gh-90352 [https://github.com/python/cpython/issues/90352]: Fix `_SelectorDatagramTransport` to inherit from **`DatagramTransport`** in **`asyncio`**. Patch by Kumar Aditya.
- gh-98793 [https://github.com/python/cpython/issues/98793]: Fix argument typechecks in **`_overlapped.WSAConnect()`** and **`_overlapped.Overlapped.WSASendTo()`** functions.
- gh-98744 [https://github.com/python/cpython/issues/98744]: Prevent crashing in **`traceback`** when retrieving the byte-offset for some source files that contain certain unicode characters.

- gh-98740 [https://github.com/python/cpython/issues/98740]: Fix internal error in the `re` module which in very rare circumstances prevented compilation of a regular expression containing a conditional expression without the "else" branch.
- gh-98703 [https://github.com/python/cpython/issues/98703]: Fix `asyncio.StreamWriter.drain()` to call `protocol.connection_lost` callback only once on Windows.
- gh-98624 [https://github.com/python/cpython/issues/98624]: Add a mutex to unittest.mock.NonCallableMock to protect concurrent access to mock attributes.
- gh-98658 [https://github.com/python/cpython/issues/98658]: The `array.array` class now supports subscripting, making it a generic type.
- gh-98284 [https://github.com/python/cpython/issues/98284]: Improved `TypeError` message for undefined abstract methods of a `abc.ABC` instance. The names of the missing methods are surrounded by single-quotes to highlight them.
- gh-96151 [https://github.com/python/cpython/issues/96151]: Allow `BUILTINS` to be a valid field name for frozen dataclasses.
- gh-98086 [https://github.com/python/cpython/issues/98086]: Make sure `patch.dict()` can be applied on async functions.
- gh-72719 [https://github.com/python/cpython/issues/72719]: Remove modules **`asyncore`** and **`asynchat`**, which were deprecated by **PEP 594** [https://peps.python.org/pep-0594/].
- gh-96192 [https://github.com/python/cpython/issues/96192]: Fix handling of `bytes` path-like objects in **`os.ismount()`**.
- gh-94172 [https://github.com/python/cpython/issues/94172]: `ftplib`: Remove the `FTP_TLS.ssl_version` class attribute: use the *context* parameter instead. Patch by Victor Stinner
- gh-94172 [https://github.com/python/cpython/issues/94172]: Remove the *keyfile* and *certfile* parameters from the `ftplib`, `imaplib`, `poplib` and `smtplib` modules, and the *key_file*, *cert_file* and *check_hostname* parameters from the `http.client` module, all deprecated since Python 3.6. Use the *context* parameter (*ssl_context* in **`imaplib`**) instead. Patch by Victor Stinner.
- gh-83638 [https://github.com/python/cpython/issues/83638]: Add the `autocommit` attribute to `sqlite3.Connection` and the *autocommit* parameter to

`sqlite3.connect()` to control **PEP 249** [https://peps.python.org/pep-0249/]-compliant transaction handling. Patch by Erlend E. Aasland.

- gh-92452 [https://github.com/python/cpython/issues/92452]: Fixed a race condition that could cause `sysconfig.get_config_var()` to incorrectly return `None` in multi-threaded programs.
- gh-91803 [https://github.com/python/cpython/issues/91803]: Fix an error when using a method of objects mocked with `unittest.mock.create_autospec()` after it was sealed with `unittest.mock.seal()` function.
- bpo-38523 [https://bugs.python.org/issue?@action=redirect&bpo=38523]: `shutil.copytree()` now applies the *ignore_dangling_symlinks* argument recursively.
- bpo-40358 [https://bugs.python.org/issue?@action=redirect&bpo=40358]: Add walk_up argument in `pathlib.PurePath.relative_to()`.
- bpo-36267 [https://bugs.python.org/issue?@action=redirect&bpo=36267]: Fix IndexError in `argparse.ArgumentParser` when a `store_true` action is given an explicit argument.

Documentação

- gh-98832 [https://github.com/python/cpython/issues/98832]: Changes wording of docstring for `pathlib.Path.iterdir()`.
- gh-97966 [https://github.com/python/cpython/issues/97966]: Update uname docs to clarify the special nature of the platform attribute and to indicate when it became late-bound.

Testes

- gh-98903 [https://github.com/python/cpython/issues/98903]: The Python test suite now fails with exit code 4 if no tests ran. It should help detecting typos in test names and test methods.
- gh-98713 [https://github.com/python/cpython/issues/98713]: Fix a bug in the `typing` tests where a test relying on CPython-specific implementation details was not decorated with `@cpython_only` and was not skipped on other implementations.
- gh-87390 [https://github.com/python/cpython/issues/87390]: Add tests for star-unpacking with PEP 646, and some other miscellaneous PEP 646 tests.

- gh-96853 [https://github.com/python/cpython/issues/96853]: Added explicit coverage of `Py_Initialize` (and hence `Py_InitializeEx`) back to the embedding tests (all other embedding tests migrated to `Py_InitializeFromConfig` in Python 3.11)
- bpo-34272 [https://bugs.python.org/issue?@action=redirect&bpo=34272]: Some C API tests were moved into the new Lib/test/test_capi/ directory.

Construção

- gh-99086 [https://github.com/python/cpython/issues/99086]: Fix `-Wimplicit-int` compiler warning in **configure** check for `PTHREAD_SCOPE_SYSTEM`.
- gh-99016 [https://github.com/python/cpython/issues/99016]: Fix build with `PYTHON_FOR_REGEN=python3.8`.
- gh-97731 [https://github.com/python/cpython/issues/97731]: Specify the full path to the source location for `make docclean` (needed for cross-builds).
- gh-98949 [https://github.com/python/cpython/issues/98949]: Drop unused build dependency on `readelf`.
- gh-98989 [https://github.com/python/cpython/issues/98989]: Use `python3.11`, if available, for regeneration and freezing.
- gh-98831 [https://github.com/python/cpython/issues/98831]: Add new tooling, in `Tools/cases_generator`, to generate the interpreter switch statement from a list of opcode definitions. This only affects adding, modifying or removing instruction definitions. The instruction definitions now live in `Python/bytecodes.c`, in the form of a custom DSL (under development) [https://github.com/faster-cpython/ideas/blob/main/3.12/interpreter_definition.md]. The tooling reads this file and writes `Python/generated_cases.c.h`, which is then included by `Python/ceval.c` to provide most of the cases of the main interpreter switch.
- gh-98817 [https://github.com/python/cpython/issues/98817]: Remove PCbuild/lib.pyproj: it's not used for anything, is only a minor convenience for Visual Studio users (who probably mostly don't even know about it), and it takes a lot of maintenance effort to keep updated.
- gh-98776 [https://github.com/python/cpython/issues/98776]: Fix `make regen-test-levenshtein` for out-of-tree builds.

- gh-98707 [https://github.com/python/cpython/issues/98707]: Don't use vendored `libmpdec` headers if **`--with-system-libmpdec`** is passed to **configure**. Don't use vendored `libexpat` headers if **`--with-system-expat`** is passed to **configure**.

Windows

- gh-98689 [https://github.com/python/cpython/issues/98689]: Update Windows builds to zlib v1.2.13. v1.2.12 has **CVE 2022-37434** [https://www.cve.org/CVERecord?id=CVE-2022-37434], but the vulnerable `inflateGetHeader` API is not used by Python.
- gh-98790 [https://github.com/python/cpython/issues/98790]: Assumes that a missing `DLLs` directory means that standard extension modules are in the executable's directory.
- gh-98745 [https://github.com/python/cpython/issues/98745]: Update `py.exe` launcher to install 3.11 by default and 3.12 on request.
- gh-98692 [https://github.com/python/cpython/issues/98692]: Fix the Gerenciador de instalação do Python ignoring unrecognized shebang lines instead of treating them as local paths
- gh-94328 [https://github.com/python/cpython/issues/94328]: Update Windows installer to use SQLite 3.39.4.

macOS

- gh-94328 [https://github.com/python/cpython/issues/94328]: Update macOS installer to SQLite 3.39.4.

API C

- gh-98724 [https://github.com/python/cpython/issues/98724]: The **`Py_CLEAR`**, **`Py_SETREF`** and **`Py_XSETREF`** macros now only evaluate their argument once. If the argument has side effects, these side effects are no longer duplicated. Patch by Victor Stinner.
- gh-98978 [https://github.com/python/cpython/issues/98978]: Fix use-after-free in `Py_SetPythonHome(NULL)`, `Py_SetProgramName(NULL)` and `_Py_SetProgramFullPath(NULL)` function calls. Issue reported by Benedikt Reinartz. Patch by Victor Stinner.

- gh-98410 [https://github.com/python/cpython/issues/98410]: Add `getbufferproc` and `releasebufferproc` to the stable API.
- gh-98610 [https://github.com/python/cpython/issues/98610]: Some configurable capabilities of sub-interpreters have changed. They always allow subprocesses (**`subprocess`**) now, whereas before subprocesses could be optionally disallowed for a sub-interpreter. Instead **`os.exec()`** can now be disallowed. Disallowing daemon threads is now supported. Disallowing all threads is still allowed, but is never done by default. Note that the optional restrictions are only available through `_Py_NewInterpreterFromConfig()`, which isn't a public API. They do not affect the main interpreter, nor **`Py_NewInterpreter()`**.
- gh-98608 [https://github.com/python/cpython/issues/98608]: A `_PyInterpreterConfig` has been added and `_Py_NewInterpreter()` has been renamed to `_Py_NewInterpreterFromConfig()`. The "isolated_subinterpreters" argument is now a granular config that captures the previous behavior. Note that this is all "private" API.
- gh-96853 [https://github.com/python/cpython/issues/96853]: `Py_InitializeEx` now correctly calls `PyConfig_Clear` after initializing the interpreter (the omission didn't cause a memory leak only because none of the dynamically allocated config fields are populated by the wrapper function)
- gh-91248 [https://github.com/python/cpython/issues/91248]: Add **`PyFrame_GetVar()`** and **`PyFrame_GetVarString()`** functions to get a frame variable by its name. Patch by Victor Stinner.

Python 3.12.0 alpha 1

Data de lançamento: 2022-10-25

Segurança

- gh-97616 [https://github.com/python/cpython/issues/97616]: Fix multiplying a list by an integer (`list *= int`): detect the integer overflow when the new allocated length is close to the maximum size. Issue reported by Jordan Limor. Patch by Victor Stinner.

- gh-97514 [https://github.com/python/cpython/issues/97514]: On Linux the **`multiprocessing`** module returns to using filesystem backed unix domain sockets for communication with the *forkserver* process instead of the Linux abstract socket namespace. Only code that chooses to use the "forkserver" start method is affected.

  Abstract sockets have no permissions and could allow any user on the system in the same network namespace [https://man7.org/linux/man-pages/man7/network_namespaces.7.html] (often the whole system) to inject code into the multiprocessing *forkserver* process. This was a potential privilege escalation. Filesystem based socket permissions restrict this to the *forkserver* process user as was the default in Python 3.8 and earlier.

  This prevents Linux **CVE 2022-42919** [https://www.cve.org/CVERecord?id=CVE-2022-42919].

- gh-87389 [https://github.com/python/cpython/issues/87389]: **`http.server`**: Fix an open redirection vulnerability in the HTTP server when an URI path starts with `//`. Vulnerability discovered, and initial fix proposed, by Hamza Avvan.

- gh-79096 [https://github.com/python/cpython/issues/79096]: LWPCookieJar and MozillaCookieJar create files with file mode 600 instead of 644 (Microsoft Windows is not affected)

- gh-92888 [https://github.com/python/cpython/issues/92888]: Fix `memoryview` use after free when accessing the backing buffer in certain cases.

- gh-68966 [https://github.com/python/cpython/issues/68966]: The deprecated mailcap module now refuses to inject unsafe text (filenames, MIME types, parameters) into shell commands. Instead of using such text, it will warn and act as if a match was not found (or for test commands, as if the test failed).

Núcleo e embutidos

- gh-98374 [https://github.com/python/cpython/issues/98374]: Suppress ImportError for invalid query for help() command. Patch by Donghee Na.
- gh-98461 [https://github.com/python/cpython/issues/98461]: Fix source location in bytecode for list, set and dict comprehensions as well as generator expressions.
- gh-98354 [https://github.com/python/cpython/issues/98354]: Added unicode check for `name` attribute of `spec` argument passed in **`_imp.create_builtin()`** function.
- gh-98398 [https://github.com/python/cpython/issues/98398]: Fix source location of 'assert' bytecodes.
- gh-98390 [https://github.com/python/cpython/issues/98390]: Fix location of sub-expressions of boolean expressions, by reducing their scope to that of the sub-expression.
- gh-98254 [https://github.com/python/cpython/issues/98254]: Modules from the standard library are now potentially suggested as part of the error messages displayed by the interpreter when an **`NameError`** is raised to the top level. Patch by Pablo Galindo
- gh-97997 [https://github.com/python/cpython/issues/97997]: Add running column offset to the tokenizer state to avoid calculating AST column information with pointer arithmetic.
- gh-97973 [https://github.com/python/cpython/issues/97973]: Modify the tokenizer to return all necessary information the parser needs to set location information in the AST nodes, so that the parser does not have to calculate those doing pointer arithmetic.

Biblioteca

- gh-96078 [https://github.com/python/cpython/issues/96078]: **`os.sched_yield()`** now release the GIL while calling sched_yield(2). Patch by Donghee Na.
- gh-97955 [https://github.com/python/cpython/issues/97955]: Migrate **`zoneinfo`** to Argument Clinic.

Núcleo e embutidos

- gh-97912 [https://github.com/python/cpython/issues/97912]: The compiler now avoids quadratic behavior when finding which instructions should use

the **LOAD_FAST_CHECK** opcode.

- gh-97002 [https://github.com/python/cpython/issues/97002]: Fix an issue where several frame objects could be backed by the same interpreter frame, possibly leading to corrupted memory and hard crashes of the interpreter.

- gh-97943 [https://github.com/python/cpython/issues/97943]: Bugfix: **PyFunction_GetAnnotations()** should return a borrowed reference. It was returning a new reference.

- gh-97922 [https://github.com/python/cpython/issues/97922]: The Garbage Collector now runs only on the eval breaker mechanism of the Python bytecode evaluation loop instead on object allocations. The GC can also run when **PyErr_CheckSignals()** is called so C extensions that need to run for a long time without executing any Python code also have a chance to execute the GC periodically.

- gh-65961 [https://github.com/python/cpython/issues/65961]: When `__package__` is different than `__spec__.parent`, raise a `DeprecationWarning` instead of `ImportWarning`.

  Also remove `importlib.util.set_package()` which was scheduled for removal.

- gh-97850 [https://github.com/python/cpython/issues/97850]: Long deprecated, `module_repr()` should now be completely eradicated.

- gh-86298 [https://github.com/python/cpython/issues/86298]: In cases where `warnings.warn_explicit()` consults the module's loader, an `DeprecationWarning` is issued when `m.__loader__` differs from `m.__spec__.loader`.

- gh-97779 [https://github.com/python/cpython/issues/97779]: Ensure that all Python frame objects are backed by "complete" frames.

- gh-91052 [https://github.com/python/cpython/issues/91052]: Add API for subscribing to modification events on selected dictionaries.

- gh-97752 [https://github.com/python/cpython/issues/97752]: Fix possible data corruption or crashes when accessing the `f_back` member of newly-created generator or coroutine frames.

- gh-97591 [https://github.com/python/cpython/issues/97591]: Fixed a missing incref/decref pair in `Exception.__setstate__()`. Patch by Ofey Chan.

- gh-97670 [https://github.com/python/cpython/issues/97670]: Remove the **`sys.getdxp()`** function and the `Tools/scripts/analyze_dxp.py` script. DXP stands for "dynamic execution pairs". They were related to `DYNAMIC_EXECUTION_PROFILE` and `DXPAIRS` macros which have been removed in Python 3.11. Python can now be built with **`./configure --enable-pystats`** to gather statistics on Python opcodes. Patch by Victor Stinner.

- gh-94526 [https://github.com/python/cpython/issues/94526]: Fix the Python path configuration used to initialized **`sys.path`** at Python startup. Paths are no longer encoded to UTF-8/strict to avoid encoding errors if it contains surrogate characters (bytes paths are decoded with the surrogateescape error handler). Patch by Victor Stinner.

- gh-96670 [https://github.com/python/cpython/issues/96670]: The parser now raises **`SyntaxError`** when parsing source code containing null bytes. Patch by Pablo Galindo

- gh-96975 [https://github.com/python/cpython/issues/96975]: Fix a crash occurring when **`PyEval_GetFrame()`** is called while the topmost Python frame is in a partially-initialized state.

- gh-96848 [https://github.com/python/cpython/issues/96848]: Fix command line parsing: reject **`-X int_max_str_digits`** option with no value (invalid) when the **`PYTHONINTMAXSTRDIGITS`** environment variable is set to a valid limit. Patch by Victor Stinner.

- gh-95921 [https://github.com/python/cpython/issues/95921]: Fix overly-broad source position information for chained comparisons used as branching conditions.

Biblioteca

- gh-96821 [https://github.com/python/cpython/issues/96821]: Fix undefined behaviour in `audioop.c`.

Núcleo e embutidos

- gh-96821 [https://github.com/python/cpython/issues/96821]: Fix undefined behaviour in `_testcapimodule.c`.
- gh-95778 [https://github.com/python/cpython/issues/95778]: When **`ValueError`** is raised if an integer is larger than the limit, mention the **`sys.set_int_max_str_digits()`** function in the error message. Patch by Victor Stinner.
- gh-96387 [https://github.com/python/cpython/issues/96387]: At Python exit, sometimes a thread holding the GIL can wait forever for a thread (usually a daemon thread) which requested to drop the GIL, whereas the thread already exited. To fix the race condition, the thread which requested the GIL drop now resets its request before exiting. Issue discovered and analyzed by Mingliang ZHAO. Patch by Victor Stinner.
- gh-96864 [https://github.com/python/cpython/issues/96864]: Fix a possible assertion failure, fatal error, or **`SystemError`** if a line tracing event raises an exception while opcode tracing is enabled.
- gh-95778 [https://github.com/python/cpython/issues/95778]: The `PyLong_FromString` function was refactored to make it more maintainable and extensible.
- gh-96678 [https://github.com/python/cpython/issues/96678]: Fix undefined behaviour in C code of null pointer arithmetic.
- gh-96754 [https://github.com/python/cpython/issues/96754]: Make sure that all frame objects created are created from valid interpreter frames. Prevents the possibility of invalid frames in backtraces and signal handlers.
- gh-90997 [https://github.com/python/cpython/issues/90997]: Improve the performance of reading and writing inline bytecode caches on some platforms.
- gh-96751 [https://github.com/python/cpython/issues/96751]: Remove dead code from `CALL_FUNCTION_EX` opcode.

- gh-90751 [https://github.com/python/cpython/issues/90751]: **`memoryview`** now supports half-floats. Patch by Donghee Na and Antoine Pitrou.
- gh-96678 [https://github.com/python/cpython/issues/96678]: Fix case of undefined behavior in ceval.c

Biblioteca

- gh-64373 [https://github.com/python/cpython/issues/64373]: Convert **`_functools`** to argument clinic.
- gh-96641 [https://github.com/python/cpython/issues/96641]: Do not expose `KeyWrapper` in **`_functools`**.

Núcleo e embutidos

- gh-96636 [https://github.com/python/cpython/issues/96636]: Ensure that tracing, `sys.setrace()`, is turned on immediately. In pre-release versions of 3.11, some tracing events might have been lost when turning on tracing in a `__del__` method or interrupt.

- gh-96572 [https://github.com/python/cpython/issues/96572]: Fix use after free in trace refs build mode. Patch by Kumar Aditya.

- gh-96611 [https://github.com/python/cpython/issues/96611]: When loading a file with invalid UTF-8 inside a multi-line string, a correct SyntaxError is emitted.

- gh-96612 [https://github.com/python/cpython/issues/96612]: Make sure that incomplete frames do not show up in tracemalloc traces.

- gh-90230 [https://github.com/python/cpython/issues/90230]: Fix compiler warnings and test failures when building with `--enable-pystats`.

- gh-96587 [https://github.com/python/cpython/issues/96587]: Correctly raise `SyntaxError` on exception groups (**PEP 654** [https://peps.python.org/pep-0654/]) on python versions prior to 3.11

- gh-96569 [https://github.com/python/cpython/issues/96569]: Remove two cases of undefined behavior, by adding NULL checks.

- gh-96582 [https://github.com/python/cpython/issues/96582]: Fix possible `NULL` pointer dereference in `_PyThread_CurrentFrames`. Patch by Kumar Aditya.

- gh-91079 [https://github.com/python/cpython/issues/91079]: Separate Python recursion checking from C recursion checking which reduces the chance of C stack overflow and allows the recursion limit to be increased safely.

- gh-93911 [https://github.com/python/cpython/issues/93911]: Fix an issue that could prevent **`LOAD_ATTR`** from specializing properly when accessing properties.

- gh-96348 [https://github.com/python/cpython/issues/96348]: Emit a DeprecationWarning when **`throw()`**, **`throw()`** or **`athrow()`** are called with more than one argument.

- gh-95196 [https://github.com/python/cpython/issues/95196]: Disable incorrect pickling of the C implemented classmethod descriptors.

- gh-96364 [https://github.com/python/cpython/issues/96364]: Fix text signatures of `list.__getitem__` and `dict.__getitem__`.

- gh-96352 [https://github.com/python/cpython/issues/96352]: Fix **`AttributeError`** missing `name` and `obj` attributes in **`object.__getattribute__()`**. Patch by Philip Georgi.

- gh-93554 [https://github.com/python/cpython/issues/93554]: Change the jump opcodes so that all conditional jumps are forward jumps. Backward jumps are converted by the assembler into a conditional forward jump whose target is the fallthrough block (and with a reversed condition), followed by an unconditional backward jump. For example:

  `POP_JUMP_IF_TRUE BACKWARD_TARGET` becomes `POP_JUMP_IF_FALSE NEXT_BLOCK; JUMP BACKWARD_TARGET`.

  All the directed conditional jump opcodes were removed: `POP_JUMP_FORWARD_IF_TRUE`, `POP_JUMP_BACKWARD_IF_TRUE`,

`POP_JUMP_FORWARD_IF_FALSE`, `POP_JUMP_BACKWARD_IF_FALSE`, `POP_JUMP_FORWARD_IF_NONE`, `POP_JUMP_BACKWARD_IF_NONE`, `POP_JUMP_FORWARD_IF_NOT_NONE`, `POP_JUMP_BACKWARD_IF_NOT_NONE`.

The corresponding opcodes without direction are no longer pseudo-instructions, and they implement the forward conditional jumps.

- gh-96268 [https://github.com/python/cpython/issues/96268]: Loading a file with invalid UTF-8 will now report the broken character at the correct location.

- gh-96237 [https://github.com/python/cpython/issues/96237]: The internal field `_PyInterpreterFrame.f_func` is renamed to `_PyInterpreterFrame.f_funcobj` and may be any object. The `f_globals` and `f_builtin` fields may hold junk values.

  It is safest to treat the `_PyInterpreterFrame` struct as opaque.

- gh-96187 [https://github.com/python/cpython/issues/96187]: Fixed a bug that caused `_PyCode_GetExtra` to return garbage for negative indexes. Patch by Pablo Galindo

- gh-96143 [https://github.com/python/cpython/issues/96143]: Add a new `-X perf` Python command line option as well as **`sys.activate_stack_trampoline()`** and **`sys.deactivate_stack_trampoline()`** function in the **`sys`** module that allows to set/unset the interpreter in a way that the Linux `perf` profiler can detect Python calls. The new **`sys.is_stack_trampoline_active()`** function allows to query the state of the perf trampoline. Design by Pablo Galindo. Patch by Pablo Galindo and Christian Heimes with contributions from Gregory P. Smith [Google] and Mark Shannon.

- gh-96071 [https://github.com/python/cpython/issues/96071]: Fix a deadlock in **`PyGILState_Ensure()`** when allocating new thread state. Patch by Kumar Aditya.

- gh-96046 [https://github.com/python/cpython/issues/96046]: **`PyType_Ready()`** now initializes `ht_cached_keys` and performs additional checks to ensure that type objects are properly configured. This avoids crashes in 3rd party packages that don't use regular API to create new types.

- gh-96005 [https://github.com/python/cpython/issues/96005]: On WASI **`ENOTCAPABLE`** is now mapped to **`PermissionError`**. The **`errno`** modules exposes the new error number. `getpath.py` now ignores **`PermissionError`** when it cannot open landmark files `pybuilddir.txt` and `pyenv.cfg`.

- gh-93678 [https://github.com/python/cpython/issues/93678]: Added test a harness for direct unit tests of the compiler's optimization stage. The `_testinternalcapi.optimize_cfg()` function runs the optimiser on a sequence of instructions. The `CfgOptimizationTestCase` class in `test.support` has utilities for invoking the optimizer and checking the output.

- gh-95245 [https://github.com/python/cpython/issues/95245]: Reduces the size of a "simple" Python object from 8 to 6 words by moving the weakreflist pointer into the pre-header directly before the object's dict/values pointer.

- gh-90997 [https://github.com/python/cpython/issues/90997]: Compile virtual **`try`**/**`except`** blocks to handle exceptions raised during **`close()`** or **`throw()`** calls through a suspended frame.

- gh-95977 [https://github.com/python/cpython/issues/95977]: Optimized calling **`__get__()`** with vectorcall. Patch by Kumar Aditya.

- gh-91210 [https://github.com/python/cpython/issues/91210]: Improve error message when a parameter without a default value follows one with a default value, and show the same message, even when the non-default/default sequence is preceded by positional-only parameters.

- gh-95922 [https://github.com/python/cpython/issues/95922]: Fixed bug where the compiler's `eliminate_empty_basic_blocks` function ignores the last block of the code unit.

- gh-95818 [https://github.com/python/cpython/issues/95818]: Skip over incomplete frames in `PyThreadState_GetFrame()`.

- gh-95876 [https://github.com/python/cpython/issues/95876]: Fix format string in `_PyPegen_raise_error_known_location` that can lead to memory corruption on some 64bit systems. The function was building a tuple with `i` (int) instead of `n` (Py_ssize_t) for Py_ssize_t arguments.

- gh-95605 [https://github.com/python/cpython/issues/95605]: Fix misleading contents of error message when converting an all-whitespace string to `float`.

- gh-95150 [https://github.com/python/cpython/issues/95150]: Update code object hashing and equality to consider all debugging and exception handling tables. This fixes an issue where certain non-identical code objects could be “deduplicated” during compilation.

- gh-91146 [https://github.com/python/cpython/issues/91146]: Reduce allocation size of `list` from `str.split()` and `str.rsplit()`. Patch by Donghee Na and Inada Naoki.

- gh-87092 [https://github.com/python/cpython/issues/87092]: Create a ‘jump target label’ abstraction in the compiler so that the compiler’s codegen stage does not work directly with basic blocks. This prepares the code for changes to the underlying CFG generation mechanism.

- gh-95355 [https://github.com/python/cpython/issues/95355]: `_PyPegen_Parser_New` now properly detects token memory allocation errors. Patch by Honglin Zhu.

- gh-90081 [https://github.com/python/cpython/issues/90081]: Run Python code in tracer/profiler function at full speed. Fixes slowdown in earlier versions of 3.11.

- gh-95324 [https://github.com/python/cpython/issues/95324]: Emit a warning in debug mode if an object does not call `PyObject_GC_UnTrack()` before deallocation. Patch by Pablo Galindo.

- gh-95245 [https://github.com/python/cpython/issues/95245]: Merge managed dict and values pointer into a single tagged pointer to save one word in the pre-header.

- gh-93678 [https://github.com/python/cpython/issues/93678]: Add cfg_builder struct and refactor the relevant code so that a cfg can be constructed without an instance of the compiler struct.

- gh-95185 [https://github.com/python/cpython/issues/95185]: Prevented crashes in the AST constructor when compiling some absurdly long expressions like `"+0"*1000000`. **`RecursionError`** is now raised instead. Patch by Pablo Galindo

- gh-93351 [https://github.com/python/cpython/issues/93351]: **`ast.AST`** node positions are now validated when provided to **`compile()`** and other related functions. If invalid positions are detected, a **`ValueError`** will be raised.

- gh-94438 [https://github.com/python/cpython/issues/94438]: Fix an issue that caused extended opcode arguments and some conditional pops to be ignored when calculating valid jump targets for assignments to the `f_lineno` attribute of frame objects. In some cases, this could cause inconsistent internal state, resulting in a hard crash of the interpreter.

- gh-95060 [https://github.com/python/cpython/issues/95060]: Undocumented `PyCode_Addr2Location` function now properly returns when `addrq` argument is less than zero.

- gh-95113 [https://github.com/python/cpython/issues/95113]: Replace all `EXTENDED_ARG_QUICK` instructions with basic **`EXTENDED_ARG`** instructions in unquickened code. Consumers of non-adaptive bytecode should be able to handle extended arguments the same way they were handled in CPython 3.10 and older.

- gh-91409 [https://github.com/python/cpython/issues/91409]: Fix incorrect source location info caused by certain optimizations in the bytecode compiler.

Biblioteca

- gh-95023 [https://github.com/python/cpython/issues/95023]: Implement `os.setns()` and `os.unshare()` for Linux. Patch by Noam Cohen.

Núcleo e embutidos

- gh-94036 [https://github.com/python/cpython/issues/94036]: Fix incorrect source location info for some multi-line attribute accesses and method calls.
- gh-94938 [https://github.com/python/cpython/issues/94938]: Fix error detection in some builtin functions when keyword argument name is an instance of a str subclass with overloaded `__eq__` and `__hash__`. Previously it could cause SystemError or other undesired behavior.

Biblioteca

- gh-94996 [https://github.com/python/cpython/issues/94996]: `ast.parse()` will no longer parse function definitions with positional-only params when passed `feature_version` less than `(3, 8)`. Patch by Shantanu Jain.

Núcleo e embutidos

- gh-94739 [https://github.com/python/cpython/issues/94739]: Allow jumping within, out of, and across exception handlers in the debugger.

Biblioteca

- gh-94949 [https://github.com/python/cpython/issues/94949]: `ast.parse()` will no longer parse parenthesized context managers when passed `feature_version` less than `(3, 9)`. Patch by Shantanu Jain.
- gh-94947 [https://github.com/python/cpython/issues/94947]: `ast.parse()` will no longer parse assignment expressions when passed `feature_version` less than `(3, 8)`. Patch by Shantanu Jain.

Núcleo e embutidos

- gh-91256 [https://github.com/python/cpython/issues/91256]: Ensures the program name is known for help text during interpreter startup.

- gh-94869 [https://github.com/python/cpython/issues/94869]: Fix the column offsets for some expressions in multi-line f-strings **`ast`** nodes. Patch by Pablo Galindo.
- gh-94893 [https://github.com/python/cpython/issues/94893]: Fix an issue where frame object manipulations could corrupt inline bytecode caches.
- gh-94822 [https://github.com/python/cpython/issues/94822]: Fix an issue where lookups of metaclass descriptors may be ignored when an identically-named attribute also exists on the class itself.
- gh-91153 [https://github.com/python/cpython/issues/91153]: Fix an issue where a **`bytearray`** item assignment could crash if it's resized by the new value's **`__index__()`** method.
- gh-90699 [https://github.com/python/cpython/issues/90699]: Fix reference counting bug in **`bool.__repr__()`**. Patch by Kumar Aditya.
- gh-94694 [https://github.com/python/cpython/issues/94694]: Fix an issue that could cause code with multi-line method lookups to have misleading or incorrect column offset information. In some cases (when compiling a hand-built AST) this could have resulted in a hard crash of the interpreter.
- gh-93252 [https://github.com/python/cpython/issues/93252]: Fix an issue that caused internal frames to outlive failed Python function calls, possibly resulting in memory leaks or hard interpreter crashes.
- gh-94215 [https://github.com/python/cpython/issues/94215]: Fix an issue where exceptions raised by line-tracing events would cause frames to be left in an invalid state, possibly resulting in a hard crash of the interpreter.
- gh-92228 [https://github.com/python/cpython/issues/92228]: Disable the compiler's inline-small-exit-blocks optimization for exit blocks that are associated with source code lines. This fixes a bug where the debugger cannot tell where an exception handler ends and the following code block begins.
- gh-94485 [https://github.com/python/cpython/issues/94485]: Line number of a module's `RESUME` instruction is set to 0 as specified in **PEP 626** [https://peps.python.org/pep-0626/].
- gh-94438 [https://github.com/python/cpython/issues/94438]: Account for instructions that can push NULL to the stack when setting line number in a frame. Prevents some (unlikely) crashes.
- gh-91719 [https://github.com/python/cpython/issues/91719]: Reload `opcode` when raising `unknown opcode error` in the interpreter main loop, for C

compilers to generate dispatching code independently.

- gh-94329 [https://github.com/python/cpython/issues/94329]: Compile and run code with unpacking of extremely large sequences (1000s of elements). Such code failed to compile. It now compiles and runs correctly.
- gh-94360 [https://github.com/python/cpython/issues/94360]: Fixed a tokenizer crash when reading encoded files with syntax errors from `stdin` with non utf-8 encoded text. Patch by Pablo Galindo
- gh-88116 [https://github.com/python/cpython/issues/88116]: Fix an issue when reading line numbers from code objects if the encoded line numbers are close to `INT_MIN`. Patch by Pablo Galindo
- gh-94262 [https://github.com/python/cpython/issues/94262]: Don't create frame objects for incomplete frames. Prevents the creation of generators and closures from being observable to Python and C extensions, restoring the behavior of 3.10 and earlier.
- gh-94192 [https://github.com/python/cpython/issues/94192]: Fix error for dictionary literals with invalid expression as value.
- gh-87995 [https://github.com/python/cpython/issues/87995]: **`types.MappingProxyType`** instances are now hashable if the underlying mapping is hashable.
- gh-93883 [https://github.com/python/cpython/issues/93883]: Revise the display strategy of traceback enhanced error locations. The indicators are only shown when the location doesn't span the whole line.
- gh-94163 [https://github.com/python/cpython/issues/94163]: Add **`BINARY_SLICE`** and **`STORE_SLICE`** instructions for more efficient handling and better specialization of slicing operations, where the slice is explicit in the source code.
- gh-94021 [https://github.com/python/cpython/issues/94021]: Fix unreachable code warning in `Python/specialize.c`.
- gh-93911 [https://github.com/python/cpython/issues/93911]: Specialize `LOAD_ATTR` for objects with custom `__getattribute__`.
- gh-93955 [https://github.com/python/cpython/issues/93955]: Improve performance of attribute lookups on objects with custom `__getattribute__` and `__getattr__`. Patch by Ken Jin.
- gh-93911 [https://github.com/python/cpython/issues/93911]: Specialize `LOAD_ATTR` for `property()` attributes.
- gh-93678 [https://github.com/python/cpython/issues/93678]: Refactor compiler optimisation code so that it no longer needs the `struct assembler` and

`struct compiler` passed around. Instead, each function takes the CFG and other data that it actually needs. This will make it possible to test this code directly.

- gh-93841 [https://github.com/python/cpython/issues/93841]: When built with `-enable-pystats`, `sys._stats_on()`, `sys._stats_off()`, `sys._stats_clear()` and `sys._stats_dump()` functions have been added to enable gathering stats for parts of programs.
- gh-93516 [https://github.com/python/cpython/issues/93516]: Store offset of first traceable instruction in code object to avoid having to recompute it for each instruction when tracing.
- gh-93516 [https://github.com/python/cpython/issues/93516]: Lazily create a table mapping bytecode offsets to line numbers to speed up calculation of line numbers when tracing.
- gh-89828 [https://github.com/python/cpython/issues/89828]: **`types.GenericAlias`** no longer relays the `__class__` attribute. For example, `isinstance(list[int], type)` no longer returns `True`.
- gh-93678 [https://github.com/python/cpython/issues/93678]: Refactor the compiler to reduce boilerplate and repetition.
- gh-93671 [https://github.com/python/cpython/issues/93671]: Fix some exponential backtrace case happening with deeply nested sequence patterns in match statements. Patch by Pablo Galindo
- gh-93662 [https://github.com/python/cpython/issues/93662]: Make sure that the end column offsets are correct in multi-line method calls. Previously, the end column could precede the column offset.

Biblioteca

- gh-93461 [https://github.com/python/cpython/issues/93461]: **`importlib.invalidate_caches()`** now drops entries from **`sys.path_importer_cache`** with a relative path as name. This solves a caching issue when a process changes its current working directory.

  `FileFinder` no longer inserts a dot in the path, e.g. `/egg/./spam` is now `/egg/spam`.

Núcleo e embutidos

- gh-93621 [https://github.com/python/cpython/issues/93621]: Change order of bytecode instructions emitted for **`with`** and **`async with`** to reduce the number of entries in the exception table.

- gh-93533 [https://github.com/python/cpython/issues/93533]: Reduce the size of the inline cache for `LOAD_METHOD` by 2 bytes.

- gh-93444 [https://github.com/python/cpython/issues/93444]: Removed redundant fields from the compiler's basicblock struct: `b_nofallthrough`, `b_exit`, `b_return`. They can be easily calculated from the opcode of the last instruction of the block.

- gh-93429 [https://github.com/python/cpython/issues/93429]: `LOAD_METHOD` instruction has been removed. It was merged back into `LOAD_ATTR`.

- gh-93418 [https://github.com/python/cpython/issues/93418]: Fixed an assert where an f-string has an equal sign '=' following an expression, but there's no trailing brace. For example, f"{i=".

- gh-93382 [https://github.com/python/cpython/issues/93382]: Cache the result of **`PyCode_GetCode()`** function to restore the $O(1)$ lookup of the **`co_code`** attribute.

- gh-93359 [https://github.com/python/cpython/issues/93359]: Ensure that custom **`ast`** nodes without explicit end positions can be compiled. Patch by Pablo Galindo.

- gh-93356 [https://github.com/python/cpython/issues/93356]: Code for exception handlers is emitted at the end of the code unit's bytecode. This avoids one jump when no exception is raised.

- gh-93354 [https://github.com/python/cpython/issues/93354]: Use exponential backoff for specialization counters in the interpreter. Can reduce the number of failed specializations significantly and avoid slowdown for those parts of a program that are not suitable for specialization.

- gh-93283 [https://github.com/python/cpython/issues/93283]: Improve error message for invalid syntax of conversion character in f-string

expressions.

- gh-93345 [https://github.com/python/cpython/issues/93345]: Fix a crash in substitution of a `TypeVar` in nested generic alias after `TypeVarTuple`.

- gh-93223 [https://github.com/python/cpython/issues/93223]: When a bytecode instruction jumps to an unconditional jump instruction, the first instruction can often be optimized to target the unconditional jump's target directly. For tracing reasons, this would previously only occur if both instructions have the same line number. This also now occurs if the unconditional jump is artificial, i.e., if it has no associated line number.

- gh-84694 [https://github.com/python/cpython/issues/84694]: The `--experimental-isolated-subinterpreters` configure option and `EXPERIMENTAL_ISOLATED_SUBINTERPRETERS` macro have been removed.

- gh-91924 [https://github.com/python/cpython/issues/91924]: Fix `__lltrace__` debug feature if the stdout encoding is not UTF-8. Patch by Victor Stinner.

- gh-93040 [https://github.com/python/cpython/issues/93040]: Wraps unused parameters in `Objects/obmalloc.c` with `Py_UNUSED`.

- gh-93143 [https://github.com/python/cpython/issues/93143]: Avoid `NULL` checks for uninitialized local variables by determining at compile time which variables must be initialized.

- gh-93061 [https://github.com/python/cpython/issues/93061]: Backward jumps after `async for` loops are no longer given dubious line numbers.

- gh-93065 [https://github.com/python/cpython/issues/93065]: Fix contextvars HAMT implementation to handle iteration over deep trees.

  The bug was discovered and fixed by Eli Libman. See MagicStack/immutables#84 [https://github.com/MagicStack/immutables/issues/84] for more details.

- gh-93012 [https://github.com/python/cpython/issues/93012]: Added the new function **`PyType_FromMetaclass()`**, which generalizes the existing **`PyType_FromModuleAndSpec()`** using an additional metaclass argument. This is useful for language binding tools, where it can be used to intercept type-related operations like subclassing or static attribute access by specifying a metaclass with custom slots.

  Importantly, **`PyType_FromMetaclass()`** is available in the Limited API, which provides a path towards migrating more binding tools onto the Stable ABI.

- gh-93021 [https://github.com/python/cpython/issues/93021]: Fix the **`__text_signature__`** for **`__get__()`** methods implemented in C. Patch by Jelle Zijlstra.

- gh-89914 [https://github.com/python/cpython/issues/89914]: The operand of the `YIELD_VALUE` instruction is set to the stack depth. This is done to help frame handling on `yield` and may assist debuggers.

- gh-92955 [https://github.com/python/cpython/issues/92955]: Fix memory leak in code object's lines and positions iterators as they were not finalized at exit. Patch by Kumar Aditya.

- gh-92930 [https://github.com/python/cpython/issues/92930]: Fixed a crash in `_pickle.c` from mutating collections during `__reduce__` or `persistent_id`.

- gh-90690 [https://github.com/python/cpython/issues/90690]: The PRECALL instruction has been removed. It offered only a small advantage for specialization and is not needed in the vast majority of cases.

- gh-92914 [https://github.com/python/cpython/issues/92914]: Always round the allocated size for lists up to the nearest even number.

- gh-92858 [https://github.com/python/cpython/issues/92858]: Improve error message for some suites with syntax error before ':'

- gh-90473 [https://github.com/python/cpython/issues/90473]: Decrease default recursion limit on WASI to address limited call stack size.

- gh-92804 [https://github.com/python/cpython/issues/92804]: Fix memory leak in `memoryview` iterator as it was not finalized at exit. Patch by Kumar Aditya.

- gh-92777 [https://github.com/python/cpython/issues/92777]: Specialize `LOAD_METHOD` for objects with lazy dictionaries. Patch by Ken Jin.

- gh-92658 [https://github.com/python/cpython/issues/92658]: Add support for connecting and binding to Hyper-V sockets on Windows Hyper-V hosts and guests.

- gh-92236 [https://github.com/python/cpython/issues/92236]: Remove spurious "LINE" event when starting a generator or coroutine, visible tracing functions implemented in C.

Biblioteca

- gh-91102 [https://github.com/python/cpython/issues/91102]: **`_warnings.warn_explicit()`** is ported to Argument Clinic.

Núcleo e embutidos

- gh-92619 [https://github.com/python/cpython/issues/92619]: Make the compiler duplicate an exit block only if none of its instructions have a lineno (previously only the first instruction in the block was checked, leading to unnecessarily duplicated blocks).
- gh-88750 [https://github.com/python/cpython/issues/88750]: The deprecated debug build only `PYTHONTHREADDEBUG` environment variable no longer does anything.

Biblioteca

- gh-92261 [https://github.com/python/cpython/issues/92261]: Fix hang when trying to iterate over a `typing.Union`.

Núcleo e embutidos

- gh-91432 [https://github.com/python/cpython/issues/91432]: Specialized the `FOR_ITER` opcode using the PEP 659 machinery
- gh-91399 [https://github.com/python/cpython/issues/91399]: Removed duplicate '{0, 0, 0, 0, 0, 0}' entry in 'Objects/unicodetype_db.h'.
- gh-91578 [https://github.com/python/cpython/issues/91578]: Updates the error message for abstract class.
- bpo-47091 [https://bugs.python.org/issue?@action=redirect&bpo=47091]: Improve performance of repetition of `list` and `tuple` by using `memcpy` to copy data and performing the reference increments in one step.
- bpo-46142 [https://bugs.python.org/issue?@action=redirect&bpo=46142]: Make `--help` output shorter by moving some info to the new `--help-env` and `--help-xoptions` command-line options. Also add `--help-all` option to print complete usage.
- bpo-42316 [https://bugs.python.org/issue?@action=redirect&bpo=42316]: Document some places where an assignment expression needs parentheses.

Biblioteca

- gh-89237 [https://github.com/python/cpython/issues/89237]: Fix hang on Windows in `subprocess.wait_closed()` in `asyncio` with `ProactorEventLoop`. Patch by Kumar Aditya.

- gh-97928 [https://github.com/python/cpython/issues/97928]: `tkinter.Text.count()` raises now an exception for options starting with "-" instead of silently ignoring them.

- gh-98393 [https://github.com/python/cpython/issues/98393]: The `os` module no longer accepts bytes-like paths, like `bytearray` and `memoryview` types: only the exact `bytes` type is accepted for bytes strings. Patch by Victor Stinner.

- gh-98363 [https://github.com/python/cpython/issues/98363]: Added itertools.batched() to batch data into lists of a given length with the last list possibly being shorter than the others.

- gh-98331 [https://github.com/python/cpython/issues/98331]: Update the bundled copies of pip and setuptools to versions 22.3 and 65.5.0 respectively.

- gh-98307 [https://github.com/python/cpython/issues/98307]: A **`createSocket()`** method was added to **`SysLogHandler`**.

- gh-96035 [https://github.com/python/cpython/issues/96035]: Fix bug in **`urllib.parse.urlparse()`** that causes certain port numbers containing whitespace, underscores, plus and minus signs, or non-ASCII digits to be incorrectly accepted.

- gh-98257 [https://github.com/python/cpython/issues/98257]: Make **`sys.setprofile()`** and **`sys.settrace()`** functions reentrant. They can no long fail with: `RuntimeError("Cannot install a trace function while another trace function is being installed")`. Patch by Victor Stinner.

- gh-98251 [https://github.com/python/cpython/issues/98251]: Allow **`venv`** to pass along **`PYTHON*`** variables to `ensurepip` and `pip` when they do not impact path resolution

- gh-94597 [https://github.com/python/cpython/issues/94597]: Deprecated **`asyncio.AbstractEventLoopPolicy.get_child_watcher()`** and **`asyncio.AbstractEventLoopPolicy.set_child_watcher()`** methods to be removed in Python 3.14. Patch by Kumar Aditya.

- gh-98178 [https://github.com/python/cpython/issues/98178]: On macOS, fix a crash in **`syslog.syslog()`** in multi-threaded applications. On macOS, the libc `syslog()` function is not thread-safe, so **`syslog.syslog()`** no longer releases the GIL to call it. Patch by Victor Stinner.

- gh-44098 [https://github.com/python/cpython/issues/44098]: Release the GIL when creating **`mmap.mmap`** objects on Unix.

- gh-87730 [https://github.com/python/cpython/issues/87730]: Wrap network errors consistently in urllib FTP support, so the test suite doesn’t fail when a network is available but the public internet is not reachable.

- gh-94597 [https://github.com/python/cpython/issues/94597]: The child watcher classes **`MultiLoopChildWatcher`**, **`FastChildWatcher`** and **`SafeChildWatcher`** are deprecated and will be removed in Python 3.14. Patch by Kumar Aditya.

- gh-98023 [https://github.com/python/cpython/issues/98023]: Change default child watcher to **`PidfdChildWatcher`** on Linux systems which supports it. Patch by Kumar Aditya.

- gh-90985 [https://github.com/python/cpython/issues/90985]: Earlier in 3.11 we deprecated `asyncio.Task.cancel("message")`. We realized we were too harsh, and have undeprecated it.

- gh-65961 [https://github.com/python/cpython/issues/65961]: Do not rely solely on `__cached__` on modules; code will also support `__spec__.cached`.

- gh-97646 [https://github.com/python/cpython/issues/97646]: Replace deprecated `application/javascript` with `text/javascript` in **`mimetypes`**. See **RFC 9239** [https://datatracker.ietf.org/doc/html/rfc9239.html]. Patch by Noam Cohen.

- gh-97930 [https://github.com/python/cpython/issues/97930]: Apply changes from importlib_resources 5.8 and 5.9: `Traversable.joinpath` provides a concrete implementation. `as_file` now supports directories of resources.

- gh-97850 [https://github.com/python/cpython/issues/97850]: Remove deprecated **`importlib.util.set_loader()`** and **`importlib.util.module_for_loader()`** from **`importlib.util`**.

- gh-97837 [https://github.com/python/cpython/issues/97837]: Change deprecate warning message in **`unittest`** from

  ```
  It is deprecated to return a value!=None
  ```

  to

`It is deprecated to return a value that is not None from a test case`

- gh-97825 [https://github.com/python/cpython/issues/97825]: Fixes **`AttributeError`** when **`subprocess.check_output()`** is used with argument `input=None` and either of the arguments *encoding* or *errors* are used.

- gh-97008 [https://github.com/python/cpython/issues/97008]: **`NameError`** and **`AttributeError`** spelling suggestions provided since gh-82711 [https://github.com/python/cpython/issues/82711] are now also emitted by the pure Python **`traceback`** module. Tests for those suggestions now exercise both implementations to ensure they are equivalent. Patch by Carl Friedrich Bolz-Tereick and Łukasz Langa.

- gh-97799 [https://github.com/python/cpython/issues/97799]: **`dataclass`** now uses **`inspect.get_annotations()`** to examine the annotations on class objects.

- gh-97781 [https://github.com/python/cpython/issues/97781]: Removed deprecated interfaces in `importlib.metadata` (entry points accessed as dictionary, implicit dictionary construction of sequence of `EntryPoint` objects, mutablility of `EntryPoints` result, access of entry point by index). `entry_points` now has a simpler, more straightforward API (returning `EntryPoints`).

- gh-96827 [https://github.com/python/cpython/issues/96827]: Avoid spurious tracebacks from **`asyncio`** when default executor cleanup is delayed until after the event loop is closed (e.g. as the result of a keyboard interrupt).

- gh-95534 [https://github.com/python/cpython/issues/95534]: **`gzip.GzipFile.read()`** reads 10% faster.

- gh-97592 [https://github.com/python/cpython/issues/97592]: Avoid a crash in the C version of **`asyncio.Future.remove_done_callback()`** when an evil argument is passed.

- gh-97639 [https://github.com/python/cpython/issues/97639]: Remove `tokenize.NL` check from **`tabnanny`**.

- gh-97545 [https://github.com/python/cpython/issues/97545]: Make Semaphore run faster.

- gh-73588 [https://github.com/python/cpython/issues/73588]: Fix generation of the default name of **`tkinter.Checkbutton`**. Previously, checkbuttons in different parent widgets could have the same short name and share the same state if arguments “name” and “variable” are not specified. Now they are globally unique.

- gh-96865 [https://github.com/python/cpython/issues/96865]: fix Flag to use boundary CONFORM

  This restores previous Flag behavior of allowing flags with non-sequential values to be combined; e.g.

  class Skip(Flag): TWO = 2 EIGHT = 8

  Skip.TWO | Skip.EIGHT -> <Skip.TWO|EIGHT: 10>

- gh-97005 [https://github.com/python/cpython/issues/97005]: Update bundled libexpat to 2.4.9

- gh-85760 [https://github.com/python/cpython/issues/85760]: Fix race condition in **`asyncio`** where **`process_exited()`** called before the **`pipe_data_received()`** leading to inconsistent output. Patch by Kumar Aditya.

- gh-96704 [https://github.com/python/cpython/issues/96704]: Pass the correct `contextvars.Context` when a `asyncio` exception handler is called on behalf of a task or callback handle. This adds a new `Task` method, `get_context`, and also a new `Handle` method with the same name. If this method is not found on a task object (perhaps because it is a third-party library that does not yet provide this method), the context prevailing at the time the exception handler is called is used.

- gh-96819 [https://github.com/python/cpython/issues/96819]: Fixed check in **`multiprocessing.resource_tracker`** that guarantees that the length of a write to a pipe is not greater than `PIPE_BUF`.

- gh-95865 [https://github.com/python/cpython/issues/95865]: Reduce **`urllib.parse.quote_from_bytes()`** memory use on large values.

  Contribuição de Dennis Sweeney.

- gh-96741 [https://github.com/python/cpython/issues/96741]: Corrected type annotation for dataclass attribute `pstats.FunctionProfile.ncalls` to be `str`.

- gh-96734 [https://github.com/python/cpython/issues/96734]: Update **`unicodedata`** database to Unicode 15.0.0.

- gh-96735 [https://github.com/python/cpython/issues/96735]: Fix undefined behaviour in **`struct.unpack()`**.

- gh-46412 [https://github.com/python/cpython/issues/46412]: Improve performance of `bool(db)` for large ndb/gdb databases. Previously this would call `len(db)` which would iterate over all keys – the answer (empty or not) is known after the first key.

- gh-96652 [https://github.com/python/cpython/issues/96652]: Fix the faulthandler implementation of `faulthandler.register(signal, chain=True)` if the `sigaction()` function is not available: don't call the previous signal handler if it's NULL. Patch by Victor Stinner.

- gh-68163 [https://github.com/python/cpython/issues/68163]: Correct conversion of **`numbers.Rational`**'s to **`float`**.

- gh-96538 [https://github.com/python/cpython/issues/96538]: Speed up `bisect.bisect()` functions by taking advantage of type-stability.

- gh-96465 [https://github.com/python/cpython/issues/96465]: Fraction hashes are now cached.

- gh-96079 [https://github.com/python/cpython/issues/96079]: In **typing**, fix missing field `name` and incorrect `__module__` in _AnnotatedAlias.

- gh-96415 [https://github.com/python/cpython/issues/96415]: Remove `types._cell_factory` from module namespace.

- gh-95987 [https://github.com/python/cpython/issues/95987]: Fix `repr` of `Any` subclasses.

- gh-96388 [https://github.com/python/cpython/issues/96388]: Work around missing socket functions in **socket**'s `__repr__`.

- gh-96385 [https://github.com/python/cpython/issues/96385]: Fix `TypeVarTuple.__typing_prepare_subst__`. `TypeError` was not raised when using more than one `TypeVarTuple`, like `[*T, *V]` in type alias substitutions.

- gh-96142 [https://github.com/python/cpython/issues/96142]: Add `match_args`, `kw_only`, `slots`, and `weakref_slot` to `_DataclassParams`.

- gh-96073 [https://github.com/python/cpython/issues/96073]: In **inspect**, fix overeager replacement of "`typing.`" in formatting annotations.

- gh-89258 [https://github.com/python/cpython/issues/89258]: Added a **getChildren()** method to **logging.Logger**, to get the immediate child loggers of a logger.

- gh-96346 [https://github.com/python/cpython/issues/96346]: Use double caching for compiled RE patterns.

- gh-96349 [https://github.com/python/cpython/issues/96349]: Fixed a minor performance regression in **threading.Event.__init__()**

- gh-90467 [https://github.com/python/cpython/issues/90467]: Fix **asyncio.streams.StreamReaderProtocol** to keep a strong reference to the created task, so that it's not garbage collected

- gh-96172 [https://github.com/python/cpython/issues/96172]: Fix a bug in `unicodedata`: `east_asian_width` used to return the wrong value for

unassigned characters; and for yet unassigned, but reserved characters.

- gh-96159 [https://github.com/python/cpython/issues/96159]: Fix a performance regression in logging TimedRotatingFileHandler. Only check for special files when the rollover time has passed.

- gh-96175 [https://github.com/python/cpython/issues/96175]: Fix unused `localName` parameter in the `Attr` class in **`xml.dom.minidom`**.

- gh-96145 [https://github.com/python/cpython/issues/96145]: Add AttrDict to JSON module for use with object_hook.

- gh-96052 [https://github.com/python/cpython/issues/96052]: Fix handling compiler warnings (SyntaxWarning and DeprecationWarning) in **`codeop.compile_command()`** when checking for incomplete input. Previously it emitted warnings and raised a SyntaxError. Now it always returns `None` for incomplete input without emitting any warnings.

- gh-96125 [https://github.com/python/cpython/issues/96125]: Fix incorrect condition that causes `sys.thread_info.name` to be wrong on pthread platforms.

- gh-96019 [https://github.com/python/cpython/issues/96019]: Fix a bug in the `makeunicodedata.py` script leading to about 13 KiB of space saving in the `unicodedata` module, specifically the character decomposition data.

- gh-95463 [https://github.com/python/cpython/issues/95463]: Remove an incompatible change from bpo-28080 [https://bugs.python.org/issue?@action=redirect&bpo=28080] that caused a regression that ignored the utf8 in `ZipInfo.flag_bits`. Patch by Pablo Galindo.

- gh-69142 [https://github.com/python/cpython/issues/69142]: Add `%:z` strftime format code (generates tzoffset with colons as separator), see strftime() and strptime() behavior.

- gh-95899 [https://github.com/python/cpython/issues/95899]: Fix `asyncio.Runner` to call `asyncio.set_event_loop()` only once to avoid calling `attach_loop()` multiple times on child watchers. Patch by Kumar Aditya.

- gh-95736 [https://github.com/python/cpython/issues/95736]: Fix `unittest.IsolatedAsyncioTestCase` to set event loop before calling setup functions. Patch by Kumar Aditya.

- gh-95865 [https://github.com/python/cpython/issues/95865]: Speed up `urllib.parse.quote_from_bytes()` by replacing a list comprehension with `map()`.

- gh-95861 [https://github.com/python/cpython/issues/95861]: Add support for computing Spearman's correlation coefficient to the existing statistics.correlation() function.

- gh-95804 [https://github.com/python/cpython/issues/95804]: Fix `logging` shutdown handler so it respects `MemoryHandler.flushOnClose`.

- gh-95704 [https://github.com/python/cpython/issues/95704]: When a task catches `asyncio.CancelledError` and raises some other error, the other error should generally not silently be suppressed.

- gh-95149 [https://github.com/python/cpython/issues/95149]: The `HTTPStatus` enum offers a couple of properties to indicate the HTTP status category e.g. `HTTPStatus.OK.is_success`.

- gh-95609 [https://github.com/python/cpython/issues/95609]: Update bundled pip to 22.2.2.

- gh-95289 [https://github.com/python/cpython/issues/95289]: Fix `asyncio.TaskGroup` to propagate exception when `asyncio.CancelledError` was replaced with another exception by a context manager. Patch by Kumar Aditya and Guido van Rossum.

- gh-94909 [https://github.com/python/cpython/issues/94909]: Fix incorrect joining of relative Windows paths with drives in `pathlib.PurePath` initializer.

- gh-95385 [https://github.com/python/cpython/issues/95385]: Faster `json.dumps()` when sorting of keys is not requested (default).

- gh-83901 [https://github.com/python/cpython/issues/83901]: Improve **`Signature.bind`** error message for missing keyword-only arguments.

- gh-95339 [https://github.com/python/cpython/issues/95339]: Update bundled pip to 22.2.1.

- gh-95045 [https://github.com/python/cpython/issues/95045]: Fix GC crash when deallocating `_lsprof.Profiler` by untracking it before calling any callbacks. Patch by Kumar Aditya.

- gh-95231 [https://github.com/python/cpython/issues/95231]: Fail gracefully if **`EPERM`** or **`ENOSYS`** is raised when loading **`crypt`** methods. This may happen when trying to load `MD5` on a Linux kernel with FIPS enabled.

- gh-95097 [https://github.com/python/cpython/issues/95097]: Fix **`asyncio.run()`** for **`asyncio.Task`** implementations without **`uncancel()`** method. Patch by Kumar Aditya.

- gh-95087 [https://github.com/python/cpython/issues/95087]: Fix IndexError in parsing invalid date in the **`email`** module.

- gh-95199 [https://github.com/python/cpython/issues/95199]: Upgrade bundled setuptools to 63.2.0.

- gh-95194 [https://github.com/python/cpython/issues/95194]: Upgrade bundled pip to 22.2.

- gh-93899 [https://github.com/python/cpython/issues/93899]: Fix check for existence of **`os.EFD_CLOEXEC`**, **`os.EFD_NONBLOCK`** and **`os.EFD_SEMAPHORE`** flags on older kernel versions where these flags are not present. Patch by Kumar Aditya.

- gh-95166 [https://github.com/python/cpython/issues/95166]: Fix **`concurrent.futures.Executor.map()`** to cancel the currently waiting on future on an error - e.g. TimeoutError or KeyboardInterrupt.

- gh-95132 [https://github.com/python/cpython/issues/95132]: Fix a **`sqlite3`** regression where `*args` and `**kwds` were incorrectly relayed from **`connect()`** to the **`Connection`** factory. The regression was introduced in 3.11a1 with PR 24421 (gh-85128 [https://github.com/python/cpython/issues/85128]). Patch by Erlend E. Aasland.

- gh-93157 [https://github.com/python/cpython/issues/93157]: Fix **`fileinput`** module didn't support `errors` option when `inplace` is true.

- gh-91212 [https://github.com/python/cpython/issues/91212]: Fixed flickering of the turtle window when the tracer is turned off. Patch by Shin-myoung-serp.

- gh-95077 [https://github.com/python/cpython/issues/95077]: Add deprecation warning for enum `member.member` access (e.g. `Color.RED.BLUE`). Remove `EnumMeta.__getattr__`.

- gh-95109 [https://github.com/python/cpython/issues/95109]: Ensure that timeouts scheduled with **`asyncio.Timeout`** that have already expired are delivered promptly.

- gh-95105 [https://github.com/python/cpython/issues/95105]: **`wsgiref.types.InputStream.__iter__()`** should return `Iterator[bytes]`, not `Iterable[bytes]`. Patch by Shantanu Jain.

- gh-95066 [https://github.com/python/cpython/issues/95066]: Replaced assert with exception in **`ast.parse()`**, when `feature_version` has an invalid major version. Patch by Shantanu Jain.

- gh-77617 [https://github.com/python/cpython/issues/77617]: Add **`sqlite3`** command-line interface. Patch by Erlend Aasland.

- gh-95005 [https://github.com/python/cpython/issues/95005]: Replace _PyAccu with _PyUnicodeWriter in JSON encoder and StringIO and remove the _PyAccu implementation.

- gh-90085 [https://github.com/python/cpython/issues/90085]: Remove `-c/--clock` and `-t/--time` CLI options of **`timeit`**. The options had been

deprecated since Python 3.3 and the functionality was removed in Python 3.7. Patch by Shantanu Jain.

- gh-94857 [https://github.com/python/cpython/issues/94857]: Fix refleak in `_io.TextIOWrapper.reconfigure`. Patch by Kumar Aditya.

- gh-94821 [https://github.com/python/cpython/issues/94821]: Fix binding of unix socket to empty address on Linux to use an available address from the abstract namespace, instead of “0”.

- gh-94736 [https://github.com/python/cpython/issues/94736]: Fix crash when deallocating an instance of a subclass of `_multiprocessing.SemLock`. Patch by Kumar Aditya.

- gh-81620 [https://github.com/python/cpython/issues/81620]: Add random.binomialvariate().

- gh-74116 [https://github.com/python/cpython/issues/74116]: Allow **`asyncio.StreamWriter.drain()`** to be awaited concurrently by multiple tasks. Patch by Kumar Aditya.

- gh-87822 [https://github.com/python/cpython/issues/87822]: When called with `capture_locals=True`, the **`traceback`** module functions swallow exceptions raised from calls to `repr()` on local variables of frames. This is in order to prioritize the original exception over rendering errors. An indication of the failure is printed in place of the missing value. (Patch by Simon-Martin Schroeder).

- gh-88050 [https://github.com/python/cpython/issues/88050]: Fix **`asyncio`** subprocess transport to kill process cleanly when process is blocked and avoid `RuntimeError` when loop is closed. Patch by Kumar Aditya.

- gh-94637 [https://github.com/python/cpython/issues/94637]: **`SSLContext.set_default_verify_paths()`** now releases the GIL around `SSL_CTX_set_default_verify_paths` call. The function call performs I/O and CPU intensive work.

- gh-94309 [https://github.com/python/cpython/issues/94309]: Deprecate aliases **`typing.Hashable`** and **`typing.Sized`**

- gh-92546 [https://github.com/python/cpython/issues/92546]: An undocumented `python -m pprint` benchmark is moved into `pprint` suite of pyperformance. Patch by Oleg Iarygin.

- gh-94607 [https://github.com/python/cpython/issues/94607]: Fix subclassing complex generics with type variables in **`typing`**. Previously an error message saying `Some type variables ... are not listed in Generic[...]` was shown. **`typing`** no longer populates `__parameters__` with the `__parameters__` of a Python class.

- gh-94619 [https://github.com/python/cpython/issues/94619]: Remove the long-deprecated `module_repr()` from **`importlib`**.

- gh-93910 [https://github.com/python/cpython/issues/93910]: The ability to access the other values of an enum on an enum (e.g. `Color.RED.BLUE`) has been restored in order to fix a performance regression.

- gh-93896 [https://github.com/python/cpython/issues/93896]: Fix **`asyncio.run()`** and **`unittest.IsolatedAsyncioTestCase`** to always the set event loop as it was done in Python 3.10 and earlier. Patch by Kumar Aditya.

- gh-94343 [https://github.com/python/cpython/issues/94343]: Allow setting the attributes of `reprlib.Repr` during object initialization

- gh-94382 [https://github.com/python/cpython/issues/94382]: Port static types of `_multiprocessing` module to heap types. Patch by Kumar Aditya.

- gh-78724 [https://github.com/python/cpython/issues/78724]: Fix crash in **`struct.Struct`** when it was not completely initialized by initializing it in **`__new__()`**. Patch by Kumar Aditya.

- gh-94510 [https://github.com/python/cpython/issues/94510]: Re-entrant calls to **`sys.setprofile()`** and **`sys.settrace()`** now raise **`RuntimeError`**. Patch by Pablo Galindo.

- gh-92336 [https://github.com/python/cpython/issues/92336]: Fix bug where `linecache.getline()` fails on bad files with `UnicodeDecodeError` or `SyntaxError`. It now returns an empty string as per the documentation.

- gh-94398 [https://github.com/python/cpython/issues/94398]: Once a `asyncio.TaskGroup` has started shutting down (i.e., at least one task has failed and the task group has started cancelling the remaining tasks), it should not be possible to add new tasks to the task group.

- gh-94383 [https://github.com/python/cpython/issues/94383]: **`xml.etree`**: Remove the `ElementTree.Element.copy()` method of the pure Python implementation, deprecated in Python 3.10, use the `copy.copy()` function instead. The C implementation of **`xml.etree`** has no `copy()` method, only a `__copy__()` method. Patch by Victor Stinner.

- gh-94379 [https://github.com/python/cpython/issues/94379]: `zipimport`: Remove `find_loader()` and `find_module()` methods, deprecated in Python 3.10: use the `find_spec()` method instead. See **PEP 451** [https://peps.python.org/pep-0451/] for the rationale. Patch by Victor Stinner.

- gh-94352 [https://github.com/python/cpython/issues/94352]: `shlex.split()`: Passing `None` for *s* argument now raises an exception, rather than reading `sys.stdin`. The feature was deprecated in Python 3.9. Patch by Victor Stinner.

- gh-94318 [https://github.com/python/cpython/issues/94318]: Strip trailing spaces in `pydoc` text output.

- gh-89988 [https://github.com/python/cpython/issues/89988]: Fix memory leak in `pickle.Pickler` when looking up **`dispatch_table`**. Patch by Kumar Aditya.

- gh-90016 [https://github.com/python/cpython/issues/90016]: Deprecate `sqlite3` default adapters and converters. Patch by Erlend E. Aasland.

- gh-94254 [https://github.com/python/cpython/issues/94254]: Fixed types of `struct` module to be immutable. Patch by Kumar Aditya.

- gh-93259 [https://github.com/python/cpython/issues/93259]: Now raise `ValueError` when `None` or an empty string are passed to `Distribution.from_name` (and other callers).

- gh-74696 [https://github.com/python/cpython/issues/74696]: **`shutil.make_archive()`** now passes the *root_dir* argument to custom archivers which support it.

- gh-94216 [https://github.com/python/cpython/issues/94216]: The **`dis`** module now has the opcodes for pseudo instructions (those which are used by the compiler during code generation but then removed or replaced by real opcodes before the final bytecode is emitted).

- gh-93096 [https://github.com/python/cpython/issues/93096]: Removed undocumented `python -m codecs`. Use `python -m unittest test.test_codecs.EncodedFileTest` instead.

- gh-94207 [https://github.com/python/cpython/issues/94207]: Made **`_struct.Struct`** GC-tracked in order to fix a reference leak in the **`_struct`** module.

- gh-93096 [https://github.com/python/cpython/issues/93096]: Removed undocumented `-t` argument of `python -m base64`. Use `python -m unittest test.test_base64.LegacyBase64TestCase.test_encodebytes` instead.

- gh-94226 [https://github.com/python/cpython/issues/94226]: Remove the **`locale.format()`** function, deprecated in Python 3.7: use **`locale.format_string()`** instead. Patch by Victor Stinner.

- gh-94199 [https://github.com/python/cpython/issues/94199]: Remove the **`ssl.match_hostname()`** function. The **`ssl.match_hostname()`** was deprecated in Python 3.7. OpenSSL performs hostname matching since Python 3.7, Python no longer uses the **`ssl.match_hostname()`** function. Patch by Victor Stinner.

- gh-94214 [https://github.com/python/cpython/issues/94214]: Document the `context` object used in the `venv.EnvBuilder` class, and add the new environment's library path to it.

- gh-94199 [https://github.com/python/cpython/issues/94199]: Remove the **`ssl.wrap_socket()`** function, deprecated in Python 3.7: instead, create a **`ssl.SSLContext`** object and call its **`ssl.SSLContext.wrap_socket`** method. Any package that still uses **`ssl.wrap_socket()`** is broken and insecure. The function neither sends a SNI TLS extension nor validates server hostname. Code is subject to **CWE 295** [https://cwe.mitre.org/data/definitions/295.html] Improper Certificate Validation. Patch by Victor Stinner.

- gh-94199 [https://github.com/python/cpython/issues/94199]: Remove the **`ssl.RAND_pseudo_bytes()`** function, deprecated in Python 3.6: use **`os.urandom()`** or **`ssl.RAND_bytes()`** instead. Patch by Victor Stinner.

- gh-94199 [https://github.com/python/cpython/issues/94199]: **`hashlib`**: Remove the pure Python implementation of **`hashlib.pbkdf2_hmac()`**, deprecated in Python 3.10. Python 3.10 and newer requires OpenSSL 1.1.1 (**PEP 644** [https://peps.python.org/pep-0644/]): this OpenSSL version provides a C implementation of **`pbkdf2_hmac()`** which is faster. Patch by Victor Stinner.

- gh-94196 [https://github.com/python/cpython/issues/94196]: **`gzip`**: Remove the `filename` attribute of **`gzip.GzipFile`**, deprecated since Python 2.6, use the **`name`** attribute instead. In write mode, the `filename` attribute added `'.gz'` file extension if it was not present. Patch by Victor Stinner.

- gh-94182 [https://github.com/python/cpython/issues/94182]: run the **`asyncio.PidfdChildWatcher`** on the running loop, this allows event loops to run subprocesses when there is no default event loop running on the main thread

- gh-94169 [https://github.com/python/cpython/issues/94169]: Remove `io.OpenWrapper` and `_pyio.OpenWrapper`, deprecated in Python 3.10: just use **`open()`** instead. The **`open()`** (**`io.open()`**) function is a built-in

function. Since Python 3.10, **`_pyio.open()`** is also a static method. Patch by Victor Stinner.

- gh-91742 [https://github.com/python/cpython/issues/91742]: Fix **`pdb`** crash after jump caused by a null pointer dereference. Patch by Kumar Aditya.

- gh-94101 [https://github.com/python/cpython/issues/94101]: Manual instantiation of **`ssl.SSLSession`** objects is no longer allowed as it lead to misconfigured instances that crashed the interpreter when attributes where accessed on them.

- gh-84753 [https://github.com/python/cpython/issues/84753]: **`inspect.iscoroutinefunction()`**, **`inspect.isgeneratorfunction()`**, and **`inspect.isasyncgenfunction()`** now properly return `True` for duck-typed function-like objects like instances of **`unittest.mock.AsyncMock`**.

  This makes **`inspect.iscoroutinefunction()`** consistent with the behavior of **`asyncio.iscoroutinefunction()`**. Patch by Mehdi ABAAKOUK.

- gh-94028 [https://github.com/python/cpython/issues/94028]: Fix a regression in the **`sqlite3`** where statement objects were not properly cleared and reset after use in cursor iters. The regression was introduced by PR 27884 in Python 3.11a1. Patch by Erlend E. Aasland.

- gh-93973 [https://github.com/python/cpython/issues/93973]: Add keyword argument `all_errors` to `asyncio.create_connection` so that multiple connection errors can be raised as an `ExceptionGroup`.

- gh-93963 [https://github.com/python/cpython/issues/93963]: Officially deprecate from `importlib.abc` classes moved to `importlib.resources.abc`.

- gh-93858 [https://github.com/python/cpython/issues/93858]: Prevent error when activating venv in nested fish instances.

- gh-93820 [https://github.com/python/cpython/issues/93820]: Pickle `enum.Flag` by name.

- gh-93847 [https://github.com/python/cpython/issues/93847]: Fix repr of enum of generic aliases.

- gh-91404 [https://github.com/python/cpython/issues/91404]: Revert the `re` memory leak when a match is terminated by a signal or memory allocation failure as the implemented fix caused a major performance regression.

- gh-83499 [https://github.com/python/cpython/issues/83499]: Fix double closing of file description in `tempfile`.

- gh-93820 [https://github.com/python/cpython/issues/93820]: Fixed a regression when `copy.copy()`-ing `enum.Flag` with multiple flag members.

- gh-79512 [https://github.com/python/cpython/issues/79512]: Fixed names and `__module__` value of `weakref` classes `ReferenceType`, `ProxyType`, `CallableProxyType`. It makes them pickleable.

- gh-91389 [https://github.com/python/cpython/issues/91389]: Fix an issue where `dis` utilities could report missing or incorrect position information in the presence of `CACHE` entries.

- gh-93626 [https://github.com/python/cpython/issues/93626]: Set `__future__.annotations` to have a `None` mandatoryRelease to indicate that it is currently 'TBD'.

- gh-90473 [https://github.com/python/cpython/issues/90473]: Emscripten and WASI have no home directory and cannot provide **PEP 370** [https://peps.python.org/pep-0370/] user site directory.

- gh-90494 [https://github.com/python/cpython/issues/90494]: `copy.copy()` and `copy.deepcopy()` now always raise a TypeError if `__reduce__()` returns a tuple with length 6 instead of silently ignore the 6th item or produce incorrect result.

- gh-90549 [https://github.com/python/cpython/issues/90549]: Fix a multiprocessing bug where a global named resource (such as a semaphore) could leak when a child process is spawned (as opposed to forked).

- gh-93521 [https://github.com/python/cpython/issues/93521]: Fixed a case where dataclasses would try to add `__weakref__` into the `__slots__` for a dataclass that specified `weakref_slot=True` when it was already defined in one of its bases. This resulted in a `TypeError` upon the new class being created.

- gh-79579 [https://github.com/python/cpython/issues/79579]: **`sqlite3`** now correctly detects DML queries with leading comments. Patch by Erlend E. Aasland.

- gh-93421 [https://github.com/python/cpython/issues/93421]: Update **`sqlite3.Cursor.rowcount`** when a DML statement has run to completion. This fixes the row count for SQL queries like `UPDATE ... RETURNING`. Patch by Erlend E. Aasland.

- gh-93475 [https://github.com/python/cpython/issues/93475]: Expose `FICLONE` and `FICLONERANGE` constants in **`fcntl`**. Patch by Illia Volochii.

- gh-93370 [https://github.com/python/cpython/issues/93370]: Deprecate **`version`** and **`version_info`**.

- gh-91810 [https://github.com/python/cpython/issues/91810]: Suppress writing an XML declaration in open files in `ElementTree.write()` with `encoding='unicode'` and `xml_declaration=None`.

- gh-91162 [https://github.com/python/cpython/issues/91162]: Support splitting of unpacked arbitrary-length tuple over `TypeVar` and `TypeVarTuple` parameters. For example:

  - `A[T, *Ts][*tuple[int, ...]]` -> `A[int, *tuple[int, ...]]`
  - `A[*Ts, T][*tuple[int, ...]]` -> `A[*tuple[int, ...], int]`

- gh-93353 [https://github.com/python/cpython/issues/93353]: Fix the **`importlib.resources.as_file()`** context manager to remove the

temporary file if destroyed late during Python finalization: keep a local reference to the **os.remove()** function. Patch by Victor Stinner.

- gh-83658 [https://github.com/python/cpython/issues/83658]: Make **multiprocessing.Pool** raise an exception if `maxtasksperchild` is not `None` or a positive int.

- gh-93312 [https://github.com/python/cpython/issues/93312]: Add **os.PIDFD_NONBLOCK** flag to open a file descriptor for a process with **os.pidfd_open()** in non-blocking mode. Patch by Kumar Aditya.

- gh-88123 [https://github.com/python/cpython/issues/88123]: Implement `Enum.__contains__` that returns `True` or `False` to replace the deprecated behaviour that would sometimes raise a **TypeError**.

- gh-93297 [https://github.com/python/cpython/issues/93297]: Make asyncio task groups prevent child tasks from being GCed

- gh-85308 [https://github.com/python/cpython/issues/85308]: Changed **argparse.ArgumentParser** to use filesystem encoding and error handler instead of default text encoding to read arguments from file (e.g. `fromfile_prefix_chars` option). This change affects Windows; argument file should be encoded with UTF-8 instead of ANSI Codepage.

- gh-93156 [https://github.com/python/cpython/issues/93156]: Accessing the **pathlib.PurePath.parents** sequence of an absolute path using negative index values produced incorrect results.

- gh-93162 [https://github.com/python/cpython/issues/93162]: Add the ability for **logging.config.dictConfig()** to usefully configure **QueueHandler** and **QueueListener** as a pair, and add **logging.getHandlerByName()** and **logging.getHandlerNames()** APIs to allow access to handlers by name.

- gh-93243 [https://github.com/python/cpython/issues/93243]: The **smtpd** module was removed per the schedule in **PEP 594** [https://peps.python.org/pep-0594/].

- gh-92886 [https://github.com/python/cpython/issues/92886]: Replace `assert` statements with `raise AssertionError()` in **`BaseHandler`** so that the tested behaviour is maintained running with optimizations `(-O)`.

- gh-90155 [https://github.com/python/cpython/issues/90155]: Fix broken **`asyncio.Semaphore`** when acquire is cancelled.

- gh-90817 [https://github.com/python/cpython/issues/90817]: The **`locale.resetlocale()`** function is deprecated and will be removed in Python 3.13. Use `locale.setlocale(locale.LC_ALL, "")` instead. Patch by Victor Stinner.

- gh-91513 [https://github.com/python/cpython/issues/91513]: Added `taskName` attribute to **`logging`** module for use with **`asyncio`** tasks.

- gh-74696 [https://github.com/python/cpython/issues/74696]: **`shutil.make_archive()`** no longer temporarily changes the current working directory during creation of standard `.zip` or tar archives.

- gh-92728 [https://github.com/python/cpython/issues/92728]: The **`re.template()`** function and the corresponding **`re.TEMPLATE`** and **`re.T`** flags are restored after they were removed in 3.11.0b1, but they are now deprecated, so they might be removed from Python 3.13.

- gh-93033 [https://github.com/python/cpython/issues/93033]: Search in some strings (platform dependent i.e [U+0xFFFF, U+0x0100] on Windows or [U+0xFFFFFFFF, U+0x00010000] on Linux 64-bit) are now up to 10 times faster.

- gh-89973 [https://github.com/python/cpython/issues/89973]: Fix **`re.error`** raised in **`fnmatch`** if the pattern contains a character range with upper bound lower than lower bound (e.g. `[c-a]`). Now such ranges are interpreted as empty ranges.

- gh-93044 [https://github.com/python/cpython/issues/93044]: No longer convert the database argument of **`sqlite3.connect()`** to bytes before passing it to the factory.

- gh-93010 [https://github.com/python/cpython/issues/93010]: In a very special case, the email package tried to append the nonexistent `InvalidHeaderError` to the defect list. It should have been `InvalidHeaderDefect`.

- gh-92986 [https://github.com/python/cpython/issues/92986]: Fix **`ast.unparse()`** when `ImportFrom.level` is `None`

- gh-92932 [https://github.com/python/cpython/issues/92932]: Now **`dis()`** and **`get_instructions()`** handle operand values for instructions prefixed by `EXTENDED_ARG_QUICK`. Patch by Sam Gross and Donghee Na.

- gh-92675 [https://github.com/python/cpython/issues/92675]: Fix **`venv.ensure_directories()`** to accept **`pathlib.Path`** arguments in addition to **`str`** paths. Patch by David Foster.

- gh-87901 [https://github.com/python/cpython/issues/87901]: Removed the `encoding` argument from **`os.popen()`** that was added in 3.11b1.

- gh-91922 [https://github.com/python/cpython/issues/91922]: Fix function **`sqlite.connect()`** and the **`sqlite.Connection`** constructor on non-UTF-8 locales. Also, they now support bytes paths non-decodable with the current FS encoding.

- gh-92869 [https://github.com/python/cpython/issues/92869]: Added **`c_time_t`** to **`ctypes`**, which has the same size as the **`time_t`** type in C.

- gh-92839 [https://github.com/python/cpython/issues/92839]: Fixed crash resulting from calling bisect.insort() or bisect.insort_left() with the key argument not equal to `None`.

- gh-90473 [https://github.com/python/cpython/issues/90473]: **`subprocess`** now fails early on Emscripten and WASI platforms to work around missing **`os.pipe()`** on WASI.

- gh-89325 [https://github.com/python/cpython/issues/89325]: Removed many old deprecated **`unittest`** features: **`TestCase`** method aliases, undocumented and broken **`TestCase`** method

`assertDictContainsSubset`, undocumented `TestLoader.loadTestsFromModule` parameter *use_load_tests*, and an underscored alias of the `TextTestResult` class.

- gh-92734 [https://github.com/python/cpython/issues/92734]: Allow multi-element reprs emitted by `reprlib` to be pretty-printed using configurable indentation.

- gh-92671 [https://github.com/python/cpython/issues/92671]: Fixed `ast.unparse()` for empty tuples in the assignment target context.

- gh-91581 [https://github.com/python/cpython/issues/91581]: `utcfromtimestamp()` no longer attempts to resolve `fold` in the pure Python implementation, since the fold is never 1 in UTC. In addition to being slightly faster in the common case, this also prevents some errors when the timestamp is close to `datetime.min`. Patch by Paul Ganssle.

- gh-86388 [https://github.com/python/cpython/issues/86388]: Removed randrange() functionality deprecated since Python 3.10. Formerly, randrange(10.0) losslessly converted to randrange(10). Now, it raises a TypeError. Also, the exception raised for non-integral values such as randrange(10.5) or randrange('10') has been changed from ValueError to TypeError.

- gh-90385 [https://github.com/python/cpython/issues/90385]: Add `pathlib.Path.walk()` as an alternative to `os.walk()`.

- gh-92550 [https://github.com/python/cpython/issues/92550]: Fix `pathlib.Path.rglob()` for empty pattern.

- gh-92591 [https://github.com/python/cpython/issues/92591]: Allow `logging` filters to return a `logging.LogRecord` instance so that filters attached to `logging.Handler`s can enrich records without side effects on other handlers.

- gh-92445 [https://github.com/python/cpython/issues/92445]: Fix a bug in `argparse` where `nargs="*"` would raise an error instead of returning an empty

list when 0 arguments were supplied if choice was also defined in `parser.add_argument`.

- gh-92547 [https://github.com/python/cpython/issues/92547]: Remove undocumented **`sqlite3`** features deprecated in Python 3.10:

  - `sqlite3.enable_shared_cache()`
  - `sqlite3.OptimizedUnicode`

  Patch por Erlend E. Aasland.

- gh-92530 [https://github.com/python/cpython/issues/92530]: Fix an issue that occurred after interrupting **`threading.Condition.notify()`**.

- gh-92531 [https://github.com/python/cpython/issues/92531]: The statistics.median_grouped() function now always return a float. Formerly, it did not convert the input type when for sequences of length one.

- gh-84131 [https://github.com/python/cpython/issues/84131]: The **`pathlib.Path`** deprecated method `link_to` has been removed. Use 3.10's **`hardlink_to()`** method instead as its semantics are consistent with that of **`symlink_to()`**.

- gh-89336 [https://github.com/python/cpython/issues/89336]: Removed **`configparser`** module APIs: the `SafeConfigParser` class alias, the `ParsingError.filename` property and parameter, and the `ConfigParser.readfp` method, all of which were deprecated since Python 3.2.

- gh-92391 [https://github.com/python/cpython/issues/92391]: Add **`__class_getitem__()`** to **`csv.DictReader`** and **`csv.DictWriter`**, allowing them to be parameterized at runtime. Patch by Marc Mueller.

- gh-91968 [https://github.com/python/cpython/issues/91968]: Add `SO_RTABLE` and `SO_USER_COOKIE` constants to **`socket`**.

- gh-91810 [https://github.com/python/cpython/issues/91810]: **`ElementTree`** method **`write()`** and function **`tostring()`** now use the text file's encoding ("UTF-8" if not available) instead of locale encoding in XML declaration when `encoding="unicode"` is specified.

- gh-81790 [https://github.com/python/cpython/issues/81790]: **`os.path.splitdrive()`** now understands DOS device paths with UNC links (beginning `\\?\UNC\`). Contributed by Barney Gale.

- gh-91760 [https://github.com/python/cpython/issues/91760]: Apply more strict rules for numerical group references and group names in regular expressions. Only sequence of ASCII digits is now accepted as a numerical reference. The group name in bytes patterns and replacement strings can now only contain ASCII letters and digits and underscore.

- gh-90622 [https://github.com/python/cpython/issues/90622]: Worker processes for **`concurrent.futures.ProcessPoolExecutor`** are no longer spawned on demand (a feature added in 3.9) when the multiprocessing context start method is `"fork"` as that can lead to deadlocks in the child processes due to a fork happening while threads are running.

- gh-91577 [https://github.com/python/cpython/issues/91577]: Move imports in **`SharedMemory`** methods to module level so that they can be executed late in python finalization.

- gh-91581 [https://github.com/python/cpython/issues/91581]: Remove an unhandled error case in the C implementation of calls to **`datetime.fromtimestamp`** with no time zone (i.e. getting a local time from an epoch timestamp). This should have no user-facing effect other than giving a possibly more accurate error message when called with timestamps that fall on 10000-01-01 in the local time. Patch by Paul Ganssle.

- gh-91539 [https://github.com/python/cpython/issues/91539]: Improve performance of `urllib.request.getproxies_environment` when there are many environment variables

- gh-91524 [https://github.com/python/cpython/issues/91524]: Speed up the regular expression substitution (functions `re.sub()` and `re.subn()` and corresponding `re.Pattern` methods) for replacement strings containing group references by 2–3 times.

- gh-91447 [https://github.com/python/cpython/issues/91447]: Fix findtext in the xml module to only give an empty string when the text attribute is set to `None`.

- gh-91456 [https://github.com/python/cpython/issues/91456]: Deprecate current default auto() behavior: In 3.13 the default will be for for auto() to always return the largest member value incremented by 1, and to raise if incompatible value types are used.

- bpo-47231 [https://bugs.python.org/issue?@action=redirect&bpo=47231]: Fixed an issue with inconsistent trailing slashes in tarfile longname directories.

- bpo-39064 [https://bugs.python.org/issue?@action=redirect&bpo=39064]: `zipfile.ZipFile` now raises `zipfile.BadZipFile` instead of `ValueError` when reading a corrupt zip file in which the central directory offset is negative.

- bpo-41287 [https://bugs.python.org/issue?@action=redirect&bpo=41287]: Fix handling of the `doc` argument in subclasses of `property()`.

- gh-90005 [https://github.com/python/cpython/issues/90005]: `ctypes` dependency `libffi` is now detected with `pkg-config`.

- bpo-32547 [https://bugs.python.org/issue?@action=redirect&bpo=32547]: The constructors for `DictWriter` and `DictReader` now coerce the `fieldnames` argument to a `list` if it is an iterator.

- bpo-35540 [https://bugs.python.org/issue?@action=redirect&bpo=35540]: Fix `dataclasses.asdict()` crash when `collections.defaultdict` is present in the attributes.

- bpo-47063 [https://bugs.python.org/issue?@action=redirect&bpo=47063]: Add an index_pages parameter to support using non-default index page names.

- bpo-47025 [https://bugs.python.org/issue?@action=redirect&bpo=47025]: Drop support for `bytes` on `sys.path`.

- bpo-46951 [https://bugs.python.org/issue?@action=redirect&bpo=46951]: Order the contents of zipapp archives, to make builds more reproducible.

- bpo-42777 [https://bugs.python.org/issue?@action=redirect&bpo=42777]: Implement `pathlib.Path.is_mount()` for Windows paths.

- bpo-46755 [https://bugs.python.org/issue?@action=redirect&bpo=46755]: In `QueueHandler`, clear `stack_info` from `LogRecord` to prevent stack trace from being written twice.

- bpo-45393 [https://bugs.python.org/issue?@action=redirect&bpo=45393]: Fix the formatting for `await x` and `not x` in the operator precedence table when using the `help()` system.

- bpo-46642 [https://bugs.python.org/issue?@action=redirect&bpo=46642]: Improve error message when trying to subclass an instance of `typing.TypeVar`, `typing.ParamSpec`, `typing.TypeVarTuple`, etc. Based on patch by Gregory Beauregard.

- bpo-46364 [https://bugs.python.org/issue?@action=redirect&bpo=46364]: Restrict use of sockets instead of pipes for stdin of subprocesses created by `asyncio` to AIX platform only.

- bpo-28249 [https://bugs.python.org/issue?@action=redirect&bpo=28249]: Set `doctest.DocTest.lineno` to `None` when an object does not have `__doc__`.

- bpo-46197 [https://bugs.python.org/issue?@action=redirect&bpo=46197]: Fix `ensurepip` environment isolation for subprocess running `pip`.

- bpo-45924 [https://bugs.python.org/issue?@action=redirect&bpo=45924]: Fix `asyncio` incorrect traceback when future's exception is raised multiple times. Patch by Kumar Aditya.

- bpo-45046 [https://bugs.python.org/issue?@action=redirect&bpo=45046]: Add support of context managers in `unittest`: methods `enterContext()` and `enterClassContext()` of class `TestCase`, method `enterAsyncContext()` of class `IsolatedAsyncioTestCase` and function `unittest.enterModuleContext()`.

- bpo-44173 [https://bugs.python.org/issue?@action=redirect&bpo=44173]: Enable fast seeking of uncompressed unencrypted **`zipfile.ZipExtFile`**

- bpo-42627 [https://bugs.python.org/issue?@action=redirect&bpo=42627]: Fix incorrect parsing of Windows registry proxy settings

- bpo-42047 [https://bugs.python.org/issue?@action=redirect&bpo=42047]: Add `threading.get_native_id()` support for DragonFly BSD. Patch by David Carlier.

- bpo-14243 [https://bugs.python.org/issue?@action=redirect&bpo=14243]: The `tempfile.NamedTemporaryFile` function has a new optional parameter *delete_on_close*

- bpo-41246 [https://bugs.python.org/issue?@action=redirect&bpo=41246]: Give the same callback function for when the overlapped operation is done to the functions `recv`, `recv_into`, `recvfrom`, `sendto`, `send` and `sendfile` inside `IocpProactor`.

- bpo-39264 [https://bugs.python.org/issue?@action=redirect&bpo=39264]: Fixed **`collections.UserDict.get()`** to not call `__missing__()` when a value is not found. This matches the behavior of `dict`. Patch by Bar Harel.

- bpo-38693 [https://bugs.python.org/issue?@action=redirect&bpo=38693]: `importlib` now uses f-strings internally instead of `str.format`.

- bpo-38267 [https://bugs.python.org/issue?@action=redirect&bpo=38267]: Add *timeout* parameter to `asyncio.loop.shutdown_default_executor()`. The default value is `None`, which means the executor will be given an unlimited amount of time. When called from `asyncio.Runner` or `asyncio.run()`, the default timeout is 5 minutes.

- bpo-34828 [https://bugs.python.org/issue?@action=redirect&bpo=34828]: **`sqlite3.Connection.iterdump()`** now handles databases that use `AUTOINCREMENT` in one or more tables.

- bpo-32990 [https://bugs.python.org/issue?@action=redirect&bpo=32990]: Support reading wave files with the `WAVE_FORMAT_EXTENSIBLE` format in the **`wave`** module.

- bpo-26253 [https://bugs.python.org/issue?@action=redirect&bpo=26253]: Allow adjustable compression level for tarfile streams in **`tarfile.open()`**.

Documentação

- gh-85525 [https://github.com/python/cpython/issues/85525]: Remove extra row
- gh-86404 [https://github.com/python/cpython/issues/86404]: Deprecated tools `make suspicious` and `rstlint.py` are now removed. They have been replaced by sphinx-lint [https://pypi.org/project/sphinx-lint/].
- gh-97741 [https://github.com/python/cpython/issues/97741]: Fix `!` in `c` domain ref target syntax via a `conf.py` patch, so it works as intended to disable ref target resolution.
- gh-96432 [https://github.com/python/cpython/issues/96432]: Fraction literals now support whitespace around the forward slash, `Fraction('2 / 3')`.
- gh-96098 [https://github.com/python/cpython/issues/96098]: Improve discoverability of the higher level concurrent.futures module by providing clearer links from the lower level threading and multiprocessing modules.
- gh-95957 [https://github.com/python/cpython/issues/95957]: What's New 3.11 now has instructions for how to provide compiler and linker flags for Tcl/Tk and OpenSSL on RHEL 7 and CentOS 7.
- gh-95588 [https://github.com/python/cpython/issues/95588]: Clarified the conflicting advice given in the **`ast`** documentation about **`ast.literal_eval()`** being "safe" for use on untrusted input while at the same time warning that it can crash the process. The latter statement is true and is deemed unfixable without a large amount of work unsuitable for a bugfix. So we keep the warning and no longer claim that `literal_eval` is safe.

- gh-91207 [https://github.com/python/cpython/issues/91207]: Fix stylesheet not working in Windows CHM htmlhelp docs and add warning that they are deprecated. Contributed by C.A.M. Gerlach.
- gh-95454 [https://github.com/python/cpython/issues/95454]: Replaced incorrectly written true/false values in documentation. Patch by Robert O'Shea
- gh-95451 [https://github.com/python/cpython/issues/95451]: Atualiza a documentação da biblioteca com informações de disponibilidade para as plataformas WebAssembly `wasm32-emscripten` e `wasm32-wasi`.
- gh-95415 [https://github.com/python/cpython/issues/95415]: Use consistent syntax for platform availability. The directive now supports a content body and emits a warning when it encounters an unknown platform.
- gh-94321 [https://github.com/python/cpython/issues/94321]: Document the **PEP 246** [https://peps.python.org/pep-0246/] style protocol type **`sqlite3.PrepareProtocol`**.
- gh-86128 [https://github.com/python/cpython/issues/86128]: Document a limitation in ThreadPoolExecutor where its exit handler is executed before any handlers in atexit.
- gh-61162 [https://github.com/python/cpython/issues/61162]: Clarify **`sqlite3`** behavior when How to use the connection context manager.
- gh-87260 [https://github.com/python/cpython/issues/87260]: Align **`sqlite3`** argument specs with the actual implementation.
- gh-86986 [https://github.com/python/cpython/issues/86986]: The minimum Sphinx version required to build the documentation is now 3.2.
- gh-88831 [https://github.com/python/cpython/issues/88831]: Augmented documentation of asyncio.create_task(). Clarified the need to keep strong references to tasks and added a code snippet detailing how to do this.
- gh-86438 [https://github.com/python/cpython/issues/86438]: Clarify that **`-W`** and **`PYTHONWARNINGS`** are matched literally and case-insensitively, rather than as regular expressions, in **`warnings`**.
- gh-93031 [https://github.com/python/cpython/issues/93031]: Update tutorial introduction output to use 3.10+ SyntaxError invalid range.
- gh-92240 [https://github.com/python/cpython/issues/92240]: Added release dates for "What's New in Python 3.X" for 3.0, 3.1, 3.2, 3.8 and 3.10
- bpo-47161 [https://bugs.python.org/issue?@action=redirect&bpo=47161]: Document that **`pathlib.PurePath`** does not collapse initial double slashes because they denote UNC paths.

- bpo-40838 [https://bugs.python.org/issue?@action=redirect&bpo=40838]: Document that `inspect.getdoc()`, `inspect.getmodule()`, and `inspect.getsourcefile()` might return `None`.
- bpo-43689 [https://bugs.python.org/issue?@action=redirect&bpo=43689]: The `Differ` documentation now also mentions other whitespace characters, which make it harder to understand the diff output.
- bpo-38056 [https://bugs.python.org/issue?@action=redirect&bpo=38056]: Overhaul the Error Handlers documentation in `codecs`.
- bpo-13553 [https://bugs.python.org/issue?@action=redirect&bpo=13553]: Document tkinter.Tk args.

Testes

- gh-95027 [https://github.com/python/cpython/issues/95027]: On Windows, when the Python test suite is run with the `-jN` option, the ANSI code page is now used as the encoding for the stdout temporary file, rather than using UTF-8 which can lead to decoding errors. Patch by Victor Stinner.
- gh-96624 [https://github.com/python/cpython/issues/96624]: Fixed the failure of repeated runs of `test.test_unittest` caused by side effects in `test_dotted_but_module_not_loaded`.
- gh-95243 [https://github.com/python/cpython/issues/95243]: Mitigate the inherent race condition from using find_unused_port() in testSockName() by trying to find an unused port a few times before failing. Patch by Ross Burton.
- gh-95573 [https://github.com/python/cpython/issues/95573]: Lib/test/test_asyncio/test_ssl.py [https://github.com/python/cpython/tree/3.15/Lib/test/test_asyncio/test_ssl.py] exposed a bug in the macOS kernel where intense concurrent load on non-blocking sockets occasionally causes `errno.ENOBUFS` ("No buffer space available") to be emitted. FB11063974 filed with Apple, in the mean time as a workaround buffer size used in tests on macOS is decreased to avoid intermittent failures. Patch by Fantix King.
- gh-95280 [https://github.com/python/cpython/issues/95280]: Fix problem with `test_ssl test_get_ciphers` on systems that require perfect forward secrecy (PFS) ciphers.

- gh-95212 [https://github.com/python/cpython/issues/95212]: Make multiprocessing test case `test_shared_memory_recreate` parallel-safe.
- gh-95218 [https://github.com/python/cpython/issues/95218]: Move tests for importlib.resources into test_importlib.resources.
- gh-93963 [https://github.com/python/cpython/issues/93963]: Updated tests to use preferred location for `importlib.resources` ABCs.
- gh-94675 [https://github.com/python/cpython/issues/94675]: Add a regression test for **`re`** exponentional slowdown when using rjsmin.
- gh-91330 [https://github.com/python/cpython/issues/91330]: Added more tests for **`dataclasses`** to cover behavior with data descriptor-based fields.
- gh-94208 [https://github.com/python/cpython/issues/94208]: `test_ssl` is now checking for supported TLS version and protocols in more tests.
- gh-94315 [https://github.com/python/cpython/issues/94315]: Tests now check for DAC override capability instead of relying on **`os.geteuid()`**.
- gh-54781 [https://github.com/python/cpython/issues/54781]: Rename test_tk to test_tkinter, and rename test_ttk_guionly to test_ttk. Patch by Victor Stinner.
- gh-93839 [https://github.com/python/cpython/issues/93839]: Move `Lib/ctypes/test/` to `Lib/test/test_ctypes/`. Patch by Victor Stinner.
- gh-93951 [https://github.com/python/cpython/issues/93951]: In test_bdb.StateTestCase.test_skip, avoid including auxiliary importers.
- gh-93957 [https://github.com/python/cpython/issues/93957]: Provide nicer error reporting from subprocesses in test_venv.EnsurePipTest.test_with_pip.
- gh-93884 [https://github.com/python/cpython/issues/93884]: Add test cases for **`PyNumber_ToBase()`** that take a large number or a non-int object as parameter.
- gh-93852 [https://github.com/python/cpython/issues/93852]: test_asyncio, test_logging, test_socket and test_socketserver now create AF_UNIX domains in the current directory to no longer fail with `OSError("AF_UNIX path too long")` if the temporary directory (the **`TMPDIR`** environment variable) is too long. Patch by Victor Stinner.
- gh-93353 [https://github.com/python/cpython/issues/93353]: regrtest now checks if a test leaks temporary files or directories if run with -jN option. Patch by Victor Stinner.
- gh-84461 [https://github.com/python/cpython/issues/84461]: `run_tests.py` now handles cross compiling env vars correctly and pass `HOSTRUNNER` to

regression tests.

- gh-93616 [https://github.com/python/cpython/issues/93616]: `test_modulefinder` now creates a temporary directory in `ModuleFinderTest.setUp()` instead of module scope.
- gh-93575 [https://github.com/python/cpython/issues/93575]: Fix issue with test_unicode test_raiseMemError. The test case now use `test.support.calcobjsize` to calculate size of PyUnicode structs. **`sys.getsizeof()`** may return different size when string has UTF-8 memory.
- gh-90473 [https://github.com/python/cpython/issues/90473]: WASI does not have a `chmod(2)` syscall. **`os.chmod()`** is now a dummy function on WASI. Skip all tests that depend on working **`os.chmod()`**.
- gh-90473 [https://github.com/python/cpython/issues/90473]: Skip tests on WASI that require symlinks with absolute paths.
- gh-57539 [https://github.com/python/cpython/issues/57539]: Increase calendar test coverage for **`calendar.LocaleTextCalendar.formatweekday()`**.
- gh-90473 [https://github.com/python/cpython/issues/90473]: Skip symlink tests on WASI. wasmtime uses `openat2(2)` with `RESOLVE_BENEATH` flag, which prevents symlinks with absolute paths.
- gh-89858 [https://github.com/python/cpython/issues/89858]: Fix `test_embed` for out-of-tree builds. Patch by Kumar Aditya.
- gh-92886 [https://github.com/python/cpython/issues/92886]: Fixing tests that fail when running with optimizations (`-O`) in `test_imaplib.py`.
- gh-92886 [https://github.com/python/cpython/issues/92886]: Fixing tests that fail when running with optimizations (`-O`) in `test_zipimport.py`
- gh-92886 [https://github.com/python/cpython/issues/92886]: Fixing tests that fail when running with optimizations (`-O`) in `test_py_compile.py`
- gh-92886 [https://github.com/python/cpython/issues/92886]: Fixing tests that fail when running with optimizations (`-O`) in `test_sys_settrace.py`.
- gh-92886 [https://github.com/python/cpython/issues/92886]: Fixing tests that fail when running with optimizations (`-O`) in `_test_multiprocessing.py`
- gh-92670 [https://github.com/python/cpython/issues/92670]: Skip `test_shutil.TestCopy.test_copyfile_nonexistent_dir` test on AIX as the test uses a trailing slash to force the OS consider the path as a directory, but on AIX the trailing slash has no effect and is considered as a file.

- gh-92514 [https://github.com/python/cpython/issues/92514]: Remove unused `test.support.BasicTestRunner`. Patch by Jelle Zijlstra.
- bpo-47016 [https://bugs.python.org/issue?@action=redirect&bpo=47016]: Create a GitHub Actions workflow for verifying bundled pip and setuptools. Patch by Illia Volochii and Adam Turner.

Construção

- gh-96761 [https://github.com/python/cpython/issues/96761]: Fix the build process of clang compiler for **_bootstrap_python** if LTO optimization is applied. Patch by Matthias Görgens and Donghee Na.
- gh-96883 [https://github.com/python/cpython/issues/96883]: `wasm32-emscripten` builds for browsers now include **`concurrent.futures`** for **`asyncio`** and **`unittest.mock`**.
- gh-85936 [https://github.com/python/cpython/issues/85936]: CPython now uses the ThinLTO option as the default policy if the Clang compiler accepts the flag. Patch by Donghee Na.
- gh-96729 [https://github.com/python/cpython/issues/96729]: Ensure that Windows releases built with `Tools\msi\buildrelease.bat` are upgradable to and from official Python releases.
- gh-96269 [https://github.com/python/cpython/issues/96269]: Shared module targets now depend on new `MODULE_DEPS` variable, which includes `EXPORTSYMS`. This fixes a build order issue on unsupported AIX platform.
- gh-84461 [https://github.com/python/cpython/issues/84461]: `wasm32-emscripten` platform no longer builds **`resource`** module, **`getresuid()`**, **`getresgid()`**, and their setters. The APIs are stubs and not functional.
- gh-95973 [https://github.com/python/cpython/issues/95973]: Add a new `--with-dsymutil` configure option to link debug information in macOS. Patch by Pablo Galindo.
- gh-90536 [https://github.com/python/cpython/issues/90536]: Use the BOLT post-link optimizer to improve performance, particularly on medium-to-large applications.
- gh-93744 [https://github.com/python/cpython/issues/93744]: Remove the `configure --with-cxx-main` build option: it didn't work for many years. Remove the `MAINCC` variable from `configure` and `Makefile`. Patch by Victor Stinner.

- gh-94801 [https://github.com/python/cpython/issues/94801]: Fix a regression in `configure` script that caused some header checks to ignore custom `CPPFLAGS`. The regression was introduced in gh-94802 [https://github.com/python/cpython/issues/94802].
- gh-95145 [https://github.com/python/cpython/issues/95145]: wasm32-wasi builds no longer depend on WASIX's pthread stubs. Python now has its own stubbed pthread API.
- gh-95174 [https://github.com/python/cpython/issues/95174]: Python now detects missing `dup` function in WASI and works around some missing **`errno`**, **`select`**, and **`socket`** constants.
- gh-95174 [https://github.com/python/cpython/issues/95174]: Python now skips missing **`socket`** functions and methods on WASI. WASI can only create sockets from existing fd / accept and has no netdb.
- gh-95085 [https://github.com/python/cpython/issues/95085]: Platforms `wasm32-unknown-emscripten` and `wasm32-unknown-wasi` have been promoted to **PEP 11** [https://peps.python.org/pep-0011/] tier 3 platform support.
- gh-94847 [https://github.com/python/cpython/issues/94847]: Fixed `_decimal` module build issue on GCC when compiling with LTO and pydebug. Debug builds no longer force inlining of functions.
- gh-94841 [https://github.com/python/cpython/issues/94841]: Fix the possible performance regression of **`PyObject_Free()`** compiled with MSVC version 1932.
- gh-94801 [https://github.com/python/cpython/issues/94801]: `configure` now uses custom flags like `ZLIB_CFLAGS` and `ZLIB_LIBS` when searching for headers and libraries.
- gh-94773 [https://github.com/python/cpython/issues/94773]: `deepfreeze.py` now supports code object with frozensets that contain incompatible, unsortable types.
- gh-94682 [https://github.com/python/cpython/issues/94682]: Build and test with OpenSSL 1.1.1q
- gh-90005 [https://github.com/python/cpython/issues/90005]: Dependencies of **`readline`** and **`curses`** module are now detected in `configure` script with `pkg-config`. Only `ncurses` / `ncursesw` are detected automatically. The old `curses` library is not configured automatically. Workaround for missing `termcap` or `tinfo` library has been removed.
- gh-90005 [https://github.com/python/cpython/issues/90005]: Fix building `_ctypes` extension without `pkg-config`.

- gh-90005 [https://github.com/python/cpython/issues/90005]: `_dbm` module dependencies are now detected by configure.
- gh-94404 [https://github.com/python/cpython/issues/94404]: `makesetup` now works around an issue with sed on macOS and uses correct CFLAGS for object files that end up in a shared extension. Module CFLAGS are used before PY_STDMODULE_CFLAGS to avoid clashes with system headers.
- gh-93939 [https://github.com/python/cpython/issues/93939]: C extension modules are now built by `configure` and `make` instead of `distutils` and `setup.py`.
- gh-93939 [https://github.com/python/cpython/issues/93939]: The `2to3`, `idle`, and `pydoc` scripts are now generated and installed by `Makefile` instead of `setup.py`.
- gh-94280 [https://github.com/python/cpython/issues/94280]: Updated pegen regeneration script on Windows to find and use Python 3.9 or higher. Prior to this, pegen regeneration already required 3.9 or higher, but the script may have used lower versions of Python.
- gh-93584 [https://github.com/python/cpython/issues/93584]: Address race condition in `Makefile` when installing a PGO build. All `test` and `install` targets now depend on `all` target.
- gh-93491 [https://github.com/python/cpython/issues/93491]: `configure` now detects and reports **PEP 11** [https://peps.python.org/pep-0011/] support tiers.
- gh-69093 [https://github.com/python/cpython/issues/69093]: Fix `Modules/Setup.stdlib.in` rule for `_sqlite3` extension.
- gh-93207 [https://github.com/python/cpython/issues/93207]: `va_start()` with two parameters, like `va_start(args, format),` is now required to build Python. `va_start()` is no longer called with a single parameter. Patch by Kumar Aditya.
- gh-93202 [https://github.com/python/cpython/issues/93202]: Python now always use the `%zu` and `%zd` printf formats to format a **`size_t`** or `Py_ssize_t` number. Building Python 3.12 requires a C11 compiler, so these printf formats are now always supported. Patch by Victor Stinner.
- gh-90473 [https://github.com/python/cpython/issues/90473]: Disable pymalloc and increase stack size on `wasm32-wasi`.
- bpo-34449 [https://bugs.python.org/issue?@action=redirect&bpo=34449]: Drop invalid compiler switch `-fPIC` for HP aCC on HP-UX. Patch by Michael Osipov.

Windows

- gh-98360 [https://github.com/python/cpython/issues/98360]: Fixes **`multiprocessing`** spawning child processes on Windows from a virtual environment to ensure that child processes that also use **`multiprocessing`** to spawn more children will recognize that they are in a virtual environment.
- gh-98414 [https://github.com/python/cpython/issues/98414]: Fix `py.exe` launcher handling of `-V:<company>/` option when default preferences have been set in environment variables or configuration files.
- gh-97728 [https://github.com/python/cpython/issues/97728]: Fix possible crashes caused by the use of uninitialized variables when pass invalid arguments in **`os.system()`** on Windows and in Windows-specific modules (like `winreg`).
- gh-90989 [https://github.com/python/cpython/issues/90989]: Made Gerenciador de instalação do Python install per-user by default (unless an all users install already exists), and clarify some text in the installer.
- gh-97649 [https://github.com/python/cpython/issues/97649]: The `Tools` directory is no longer installed on Windows
- gh-96965 [https://github.com/python/cpython/issues/96965]: Update libffi to 3.4.3
- gh-96577 [https://github.com/python/cpython/issues/96577]: Fixes a potential buffer overrun in **`msilib`**.
- gh-96559 [https://github.com/python/cpython/issues/96559]: Fixes the Windows launcher not using the compatible interpretation of default tags found in configuration files when no tag was passed to the command.
- gh-94781 [https://github.com/python/cpython/issues/94781]: Fix `pcbuild.proj` to clean previous instances of output files in `Python\deepfreeze` and `Python\frozen_modules` directories on Windows. Patch by Charlie Zhao.
- gh-89545 [https://github.com/python/cpython/issues/89545]: Updates **`platform`** code getting the Windows version to use native Windows Management Instrumentation (WMI) queries to determine OS version, type, and architecture.
- gh-95733 [https://github.com/python/cpython/issues/95733]: Make certain requirements of the Windows Store package optional to allow installing on earlier updates of Windows.

- gh-95656 [https://github.com/python/cpython/issues/95656]: Enable the **`enable_load_extension()`** **`sqlite3`** API.
- gh-95587 [https://github.com/python/cpython/issues/95587]: Fixes some issues where the Windows installer would incorrectly detect certain features of an existing install when upgrading.
- gh-94399 [https://github.com/python/cpython/issues/94399]: Restores the behaviour of Gerenciador de instalação do Python for `/usr/bin/env` shebang lines, which will now search **`PATH`** for an executable matching the given command. If none is found, the usual search process is used.
- gh-95445 [https://github.com/python/cpython/issues/95445]: Fixes the unsuccessful removal of the HTML document directory when uninstalling with Windows msi.
- gh-95359 [https://github.com/python/cpython/issues/95359]: Fix Gerenciador de instalação do Python handling of `py.ini` commands (it was incorrectly expecting a `py_` prefix on keys) and crashes when reading per-user configuration file.
- gh-95285 [https://github.com/python/cpython/issues/95285]: Fix Gerenciador de instalação do Python handling of command lines where it is only passed a short executable name.
- gh-90844 [https://github.com/python/cpython/issues/90844]: Allow virtual environments to correctly launch when they have spaces in the path.
- gh-94772 [https://github.com/python/cpython/issues/94772]: Fix incorrect handling of shebang lines in py.exe launcher
- gh-94018 [https://github.com/python/cpython/issues/94018]: **`zipfile`** will now remove trailing spaces from path components when extracting files on Windows.
- gh-93824 [https://github.com/python/cpython/issues/93824]: Drag and drop of files onto Python files in Windows Explorer has been enabled for Windows ARM64.
- gh-43414 [https://github.com/python/cpython/issues/43414]: **`os.get_terminal_size()`** now attempts to read the size from any provided handle, rather than only supporting file descriptors 0, 1 and 2.
- gh-92817 [https://github.com/python/cpython/issues/92817]: Ensures that `py.exe` will prefer an active virtual environment over default tags specified with environment variables or through a `py.ini` file.
- gh-92984 [https://github.com/python/cpython/issues/92984]: Explicitly disable incremental linking for non-Debug builds

- gh-92841 [https://github.com/python/cpython/issues/92841]: **`asyncio`** no longer throws `RuntimeError: Event loop is closed` on interpreter exit after asynchronous socket activity. Patch by Oleg Iarygin.
- bpo-46907 [https://bugs.python.org/issue?@action=redirect&bpo=46907]: Update Windows installer to use SQLite 3.38.4.
- gh-91061 [https://github.com/python/cpython/issues/91061]: Accept os.PathLike for the argument to winsound.PlaySound
- bpo-42658 [https://bugs.python.org/issue?@action=redirect&bpo=42658]: Support native Windows case-insensitive path comparisons by using `LCMapStringEx` instead of **`str.lower()`** in **`ntpath.normcase()`**. Add `LCMapStringEx` to the **`_winapi`** module.
- bpo-38704 [https://bugs.python.org/issue?@action=redirect&bpo=38704]: Prevent installation on unsupported Windows versions.

macOS

- gh-97897 [https://github.com/python/cpython/issues/97897]: The macOS 13 SDK includes support for the `mkfifoat` and `mknodat` system calls. Using the `dir_fd` option with either **`os.mkfifo()`** or **`os.mknod()`** could result in a segfault if cpython is built with the macOS 13 SDK but run on an earlier version of macOS. Prevent this by adding runtime support for detection of these system calls (“weaklinking”) as is done for other newer syscalls on macOS.

IDLE

- gh-97527 [https://github.com/python/cpython/issues/97527]: Fix a bug in the previous bugfix that caused IDLE to not start when run with 3.10.8, 3.12.0a1, and at least Microsoft Python 3.10.2288.0 installed without the Lib/test package. 3.11.0 was never affected.
- gh-65802 [https://github.com/python/cpython/issues/65802]: Document handling of extensions in Save As dialogs.
- gh-95191 [https://github.com/python/cpython/issues/95191]: Include prompts when saving Shell (interactive input and output).
- gh-95511 [https://github.com/python/cpython/issues/95511]: Fix the Shell context menu copy-with-prompts bug of copying an extra line when one selects whole lines.

- gh-95471 [https://github.com/python/cpython/issues/95471]: In the Edit menu, move `Select All` and add a new separator.
- gh-95411 [https://github.com/python/cpython/issues/95411]: Enable using IDLE's module browser with .pyw files.
- gh-89610 [https://github.com/python/cpython/issues/89610]: Add .pyi as a recognized extension for IDLE on macOS. This allows opening stub files by double clicking on them in the Finder.

Ferramentas/Demos

- gh-68686 [https://github.com/python/cpython/issues/68686]: Remove ptags and eptags scripts.
- gh-97681 [https://github.com/python/cpython/issues/97681]: Remove the `Tools/demo/` directory which contained old demo scripts. A copy can be found in the old-demos project [https://github.com/gvanrossum/old-demos]. Patch by Victor Stinner.
- gh-97669 [https://github.com/python/cpython/issues/97669]: Remove outdated example scripts of the `Tools/scripts/` directory. A copy can be found in the old-demos project [https://github.com/gvanrossum/old-demos]. Patch by Victor Stinner.
- gh-95853 [https://github.com/python/cpython/issues/95853]: The `wasm_build.py` script now pre-builds Emscripten ports, checks for broken EMSDK versions, and warns about pkg-config env vars.
- gh-95853 [https://github.com/python/cpython/issues/95853]: A nova ferramenta `Tools/wasm/wasm_builder.py` automatiza as etapas de configuração, compilação e teste para a construção do CPython em plataformas WebAssembly.
- gh-95731 [https://github.com/python/cpython/issues/95731]: Fix handling of module docstrings in `Tools/i18n/pygettext.py`.
- gh-93939 [https://github.com/python/cpython/issues/93939]: Add script `Tools/scripts/check_modules.py` to check and validate builtin and shared extension modules. The script also handles `Modules/Setup` and will eventually replace `setup.py`.
- gh-94538 [https://github.com/python/cpython/issues/94538]: Fix Argument Clinic output to custom file destinations. Patch by Erlend E. Aasland.
- gh-94430 [https://github.com/python/cpython/issues/94430]: Allow parameters named `module` and `self` with custom C names in Argument Clinic.

Patch by Erlend E. Aasland

- gh-86087 [https://github.com/python/cpython/issues/86087]: The `Tools/scripts/parseentities.py` script used to parse HTML4 entities has been removed.

API C

- gh-98393 [https://github.com/python/cpython/issues/98393]: The **`PyUnicode_FSDecoder()`** function no longer accepts bytes-like paths, like **`bytearray`** and **`memoryview`** types: only the exact **`bytes`** type is accepted for bytes strings. Patch by Victor Stinner.

- gh-91051 [https://github.com/python/cpython/issues/91051]: Add **`PyType_Watch()`** and related APIs to allow callbacks on **`PyType_Modified()`**.

- gh-95756 [https://github.com/python/cpython/issues/95756]: Lazily create and cache `co_` attributes for better performance for code getters.

- gh-96512 [https://github.com/python/cpython/issues/96512]: Configuration for the integer string conversion length limitation now lives in the PyConfig C API struct.

- gh-95589 [https://github.com/python/cpython/issues/95589]: Extensions classes that set `tp_dictoffset` and `tp_weaklistoffset` lose the support for multiple inheritance, but are now safe. Extension classes should use **`Py_TPFLAGS_MANAGED_DICT`** and **`Py_TPFLAGS_MANAGED_WEAKREF`** instead.

- gh-95781 [https://github.com/python/cpython/issues/95781]: An unrecognized format character in **`PyUnicode_FromFormat()`** and **`PyUnicode_FromFormatV()`** now sets a **`SystemError`**. In previous versions it caused all the rest of the format string to be copied as-is to the result string, and any extra arguments discarded.

- gh-92678 [https://github.com/python/cpython/issues/92678]: Restore the 3.10 behavior for multiple inheritance of C extension classes that store their dictionary at the end of the struct.

- gh-92678 [https://github.com/python/cpython/issues/92678]: Support C extensions using managed dictionaries by setting the `Py_TPFLAGS_MANAGED_DICT` flag.

- gh-93274 [https://github.com/python/cpython/issues/93274]: API for implementing vectorcall (**`Py_TPFLAGS_HAVE_VECTORCALL`**, **`PyVectorcall_NARGS()`** and **`PyVectorcall_Call()`**) was added to the limited API and stable ABI.

- gh-95504 [https://github.com/python/cpython/issues/95504]: Fix sign placement when specifying width or precision in **`PyUnicode_FromFormat()`** and **`PyUnicode_FromFormatV()`**. Patch by Philip Georgi.

- gh-93012 [https://github.com/python/cpython/issues/93012]: The **`Py_TPFLAGS_HAVE_VECTORCALL`** flag is now removed from a class when the class's **`__call__()`** method is reassigned. This makes vectorcall safe to use with mutable types (i.e. heap types without the **`immutable`** flag). Mutable types that do not override **`tp_call`** now inherit the **`Py_TPFLAGS_HAVE_VECTORCALL`** flag.

- gh-95388 [https://github.com/python/cpython/issues/95388]: Creating **`immutable types`** with mutable bases is deprecated and is planned to be disabled in Python 3.14.

- gh-92678 [https://github.com/python/cpython/issues/92678]: Adds unstable C-API functions `_PyObject_VisitManagedDict` and `_PyObject_ClearManagedDict` to allow C extensions to allow the VM to manage their object's dictionaries.

- gh-94936 [https://github.com/python/cpython/issues/94936]: Added **`PyCode_GetVarnames()`**, **`PyCode_GetCellvars()`** and **`PyCode_GetFreevars()`** for accessing `co_varnames`, `co_cellvars` and `co_freevars` respectively via the C API.

- gh-94930 [https://github.com/python/cpython/issues/94930]: Fix `SystemError` raised when **`PyArg_ParseTupleAndKeywords()`** is used with `#` in `(...)` but without `PY_SSIZE_T_CLEAN` defined.

- gh-94731 [https://github.com/python/cpython/issues/94731]: Python again uses C-style casts for most casting operations when compiled with C++. This may trigger compiler warnings, if they are enabled with e.g. `-Wold-style-cast` or `-Wzero-as-null-pointer-constant` options for `g++`.

- gh-93937 [https://github.com/python/cpython/issues/93937]: The following frame functions and type are now directly available with `#include <Python.h>`, it's no longer needed to add `#include <frameobject.h>`:

  - **`PyFrame_Check()`**
  - **`PyFrame_GetBack()`**
  - **`PyFrame_GetBuiltins()`**
  - **`PyFrame_GetGenerator()`**
  - **`PyFrame_GetGlobals()`**
  - **`PyFrame_GetLasti()`**
  - **`PyFrame_GetLocals()`**
  - **`PyFrame_Type`**

  Patch por Victor Stinner.

- gh-91321 [https://github.com/python/cpython/issues/91321]: Fix the compatibility of the Python C API with C++ older than C++11. Patch by Victor Stinner.

- gh-91731 [https://github.com/python/cpython/issues/91731]: Avoid defining the `static_assert` when compiling with C++ 11, where this is a keyword and redefining it can lead to undefined behavior. Patch by Pablo Galindo

- gh-89546 [https://github.com/python/cpython/issues/89546]: **`PyType_FromMetaclass()`** (and other `PyType_From*` functions) now check that offsets and the base class's **`tp_basicsize`** fit in the new class's `tp_basicsize`.

- gh-93503 [https://github.com/python/cpython/issues/93503]: Add two new public functions to the public C-API, **`PyEval_SetProfileAllThreads()`** and **`PyEval_SetTraceAllThreads()`**, that allow to set tracing and profiling functions in all running threads in addition to the calling one. Also,

two analogous functions to the `threading` module (`threading.setprofile_all_threads()` and `threading.settrace_all_threads()`) that allow to do the same from Python. Patch by Pablo Galindo

- gh-93442 [https://github.com/python/cpython/issues/93442]: Add C++ overloads for _Py_CAST_impl() to handle 0/NULL. This will allow C++ extensions that pass 0 or NULL to macros using _Py_CAST() to continue to compile.

- gh-93466 [https://github.com/python/cpython/issues/93466]: Slot IDs in PyType_Spec may not be repeated. The documentation was updated to mention this. For some cases of repeated slots, PyType_FromSpec and related functions will now raise an exception.

- gh-92898 [https://github.com/python/cpython/issues/92898]: Fix C++ compiler warnings when casting function arguments to `PyObject*`. Patch by Serge Guelton.

- gh-93103 [https://github.com/python/cpython/issues/93103]: Deprecate global configuration variables, like `Py_IgnoreEnvironmentFlag`, in the documentation: the `Py_InitializeFromConfig()` API should be instead. Patch by Victor Stinner.

- gh-77782 [https://github.com/python/cpython/issues/77782]: Deprecate global configuration variable like `Py_IgnoreEnvironmentFlag`: the `Py_InitializeFromConfig()` API should be instead. Patch by Victor Stinner.

- gh-92913 [https://github.com/python/cpython/issues/92913]: Ensures changes to `PyConfig.module_search_paths` are ignored unless `PyConfig.module_search_paths_set` is set

- gh-92781 [https://github.com/python/cpython/issues/92781]: Avoid mixing declarations and code in the C API to fix the compiler warning: “ISO C90 forbids mixed declarations and code” [-Werror=declaration-after-statement]. Patch by Victor Stinner.

- gh-92651 [https://github.com/python/cpython/issues/92651]: Remove the `token.h` header file. There was never any public tokenizer C API. The `token.h` header file was only designed to be used by Python internals. Patch by Victor Stinner.

- gh-92536 [https://github.com/python/cpython/issues/92536]: Remove legacy Unicode APIs based on `Py_UNICODE*`.

- gh-85858 [https://github.com/python/cpython/issues/85858]: Remove the `PyUnicode_InternImmortal()` function and the `SSTATE_INTERNED_IMMORTAL` macro. Patch by Victor Stinner.

- gh-92193 [https://github.com/python/cpython/issues/92193]: Add new function **`PyFunction_SetVectorcall()`** to the C API which sets the vectorcall field of a given **`PyFunctionObject`**.

  Warning: extensions using this API must preserve the behavior of the unaltered function!

- gh-59121 [https://github.com/python/cpython/issues/59121]: Fixed an assert that prevented `PyRun_InteractiveOne` from providing tracebacks when parsing from the provided FD.

- bpo-45383 [https://bugs.python.org/issue?@action=redirect&bpo=45383]: The **`PyType_FromSpec()`** API will now find and use a metaclass based on the provided bases. An error will be raised if there is a metaclass conflict.

Python 3.11.0 beta 1

Data de lançamento: 2022-05-06

Segurança

- gh-57684 [https://github.com/python/cpython/issues/57684]: Add the **`-P`** command line option and the **`PYTHONSAFEPATH`** environment variable to not prepend a potentially unsafe path to **`sys.path`**. Patch by Victor Stinner.

Núcleo e embutidos

- gh-89519 [https://github.com/python/cpython/issues/89519]: Chaining classmethod descriptors (introduced in bpo-19072 [https://bugs.python.org/issue?@action=redirect&bpo=19072]) is deprecated. It can no longer be used to wrap other descriptors such as property(). The core design of this feature was flawed, and it caused a number of downstream problems.
- gh-92345 [https://github.com/python/cpython/issues/92345]: `pymain_run_python()` now imports `readline` and `rlcompleter` before sys.path is extended to include the current working directory of an interactive interpreter. Non-interactive interpreters are not affected.
- bpo-43857 [https://bugs.python.org/issue?@action=redirect&bpo=43857]: Improve the **`AttributeError`** message when deleting a missing attribute. Patch by Géry Ogam.
- gh-92245 [https://github.com/python/cpython/issues/92245]: Make sure that PEP 523 is respected in all cases. In 3.11a7, specialization may have prevented Python-to-Python calls respecting PEP 523.
- gh-92203 [https://github.com/python/cpython/issues/92203]: Add a closure keyword-only parameter to **`exec()`**. It can only be specified when exec-ing a code object that uses free variables. When specified, it must be a tuple, with exactly the number of cell variables referenced by the code object. closure has a default value of `None`, and it must be `None` if the code object doesn't refer to any free variables.
- gh-91173 [https://github.com/python/cpython/issues/91173]: Disable frozen modules in debug builds. Patch by Kumar Aditya.
- gh-92114 [https://github.com/python/cpython/issues/92114]: Improve error message when subscript a type with `__class_getitem__` set to `None`.
- gh-92112 [https://github.com/python/cpython/issues/92112]: Fix crash triggered by an evil custom `mro()` on a metaclass.
- gh-92063 [https://github.com/python/cpython/issues/92063]: The `PRECALL_METHOD_DESCRIPTOR_FAST_WITH_KEYWORDS` instruction now ensures methods are called only on objects of the correct type.
- gh-92031 [https://github.com/python/cpython/issues/92031]: Deoptimize statically allocated code objects during `Py_FINALIZE()` so that future `_PyCode_Quicken` calls always start with unquickened code.
- gh-92036 [https://github.com/python/cpython/issues/92036]: Fix a crash in subinterpreters related to the garbage collector. When a subinterpreter

is deleted, untrack all objects tracked by its GC. To prevent a crash in deallocator functions expecting objects to be tracked by the GC, leak a strong reference to these objects on purpose, so they are never deleted and their deallocator functions are not called. Patch by Victor Stinner.

- gh-92032 [https://github.com/python/cpython/issues/92032]: The interpreter can now autocomplete soft keywords, as of now `match`, `case`, and `_` (wildcard pattern) from **PEP 634** [https://peps.python.org/pep-0634/].
- gh-87999 [https://github.com/python/cpython/issues/87999]: The warning emitted by the Python parser for a numeric literal immediately followed by keyword has been changed from deprecation warning to syntax warning.
- gh-91869 [https://github.com/python/cpython/issues/91869]: Fix an issue where specialized opcodes with extended arguments could produce incorrect tracing output or lead to assertion failures.
- gh-91603 [https://github.com/python/cpython/issues/91603]: Speed up **`types.UnionType`** instantiation. Based on patch provided by Yurii Karabas.
- gh-89373 [https://github.com/python/cpython/issues/89373]: If Python is built in debug mode, Python now ensures that deallocator functions leave the current exception unchanged. Patch by Victor Stinner.
- gh-91632 [https://github.com/python/cpython/issues/91632]: Fix a minor memory leak at exit: release the memory of the **`generic_alias_iterator`** type. Patch by Donghee Na.
- gh-81548 [https://github.com/python/cpython/issues/81548]: Octal escapes with value larger than `0o377` now produce a **`DeprecationWarning`**. In a future Python version they will be a **`SyntaxWarning`** and eventually a **`SyntaxError`**.
- bpo-43950 [https://bugs.python.org/issue?@action=redirect&bpo=43950]: Use a single compact table for line starts, ends and column offsets. Reduces memory consumption for location info by half
- gh-91102 [https://github.com/python/cpython/issues/91102]: Use Argument Clinic for **`EncodingMap`**. Patch by Oleg Iarygin.
- gh-91636 [https://github.com/python/cpython/issues/91636]: Fixed a crash in a garbage-collection edge-case, in which a `PyFunction_Type.tp_clear` function could leave a python function object in an inconsistent state.
- gh-91603 [https://github.com/python/cpython/issues/91603]: Speed up **`isinstance()`** and **`issubclass()`** checks for **`types.UnionType`**. Patch

by Yurii Karabas.

- gh-91625 [https://github.com/python/cpython/issues/91625]: Fixed a bug in which adaptive opcodes ignored any preceding `EXTENDED_ARG`s on specialization failure.
- gh-78607 [https://github.com/python/cpython/issues/78607]: The LLTRACE special build now looks for the name `__lltrace__` defined in module globals, rather than the name `__ltrace__`, which had been introduced as a typo.
- gh-91576 [https://github.com/python/cpython/issues/91576]: Speed up iteration of ascii strings by 50%. Patch by Kumar Aditya.
- gh-89279 [https://github.com/python/cpython/issues/89279]: Improve interpreter performance on Windows by inlining a few specific macros.
- gh-91502 [https://github.com/python/cpython/issues/91502]: Add a new **`_PyFrame_IsEntryFrame()`** API function, to check if a **`PyFrameObject`** is an entry frame. Patch by Pablo Galindo.
- gh-91266 [https://github.com/python/cpython/issues/91266]: Refactor the `bytearray` strip methods `strip`, `lstrip` and `rstrip` to use a common implementation.
- gh-91479 [https://github.com/python/cpython/issues/91479]: Replaced the `__note__` field of **`BaseException`** (added in an earlier version of 3.11) with the final design of **PEP 678** [https://peps.python.org/pep-0678/]. Namely, **`BaseException`** gets an **`add_note()`** method, and its `__notes__` field is created when necessary.
- gh-46055 [https://github.com/python/cpython/issues/46055]: Speed up right shift of negative integers, by removing unnecessary creation of temporaries. Original patch by Xinhang Xu, reworked by Mark Dickinson.
- gh-91462 [https://github.com/python/cpython/issues/91462]: Make the interpreter's low-level tracing (lltrace) feature output more readable by displaying opcode names (rather than just numbers), and by displaying stack contents before each opcode.
- gh-89455 [https://github.com/python/cpython/issues/89455]: Fixed an uninitialized bool value in the traceback printing code path that was introduced by the initial bpo-45292 [https://bugs.python.org/issue?@action=redirect&bpo=45292] exception groups work.
- gh-91421 [https://github.com/python/cpython/issues/91421]: Fix a potential integer overflow in _Py_DecodeUTF8Ex.

- gh-91428 [https://github.com/python/cpython/issues/91428]: Add `static const char *const _PyOpcode_OpName[256] = {...};` to `opcode.h` for debug builds to assist in debugging the Python interpreter. It is now more convenient to make various forms of debugging output more human-readable by including opcode names rather than just the corresponding decimal digits.
- bpo-47120 [https://bugs.python.org/issue?@action=redirect&bpo=47120]: Make **`POP_JUMP_IF_TRUE`**, **`POP_JUMP_IF_FALSE`**, **`POP_JUMP_IF_NONE`** and **`POP_JUMP_IF_NOT_NONE`** virtual, mapping to new relative jump opcodes.
- bpo-45317 [https://bugs.python.org/issue?@action=redirect&bpo=45317]: Add internal documentation explaining design of new (for 3.11) frame stack.

Biblioteca

- bpo-47197 [https://bugs.python.org/issue?@action=redirect&bpo=47197]: ctypes used to mishandle `void` return types, so that for instance a function declared like `ctypes.CFUNCTYPE(None, ctypes.c_int)` would be called with signature `int f(int)` instead of `void f(int)`. Wasm targets require function pointers to be called with the correct signatures so this led to crashes. The problem is now fixed.

Núcleo e embutidos

- bpo-47120 [https://bugs.python.org/issue?@action=redirect&bpo=47120]: Make opcodes **`JUMP_IF_TRUE_OR_POP`** and **`JUMP_IF_FALSE_OR_POP`** relative rather than absolute.

- bpo-47177 [https://bugs.python.org/issue?@action=redirect&bpo=47177]: Replace the `f_lasti` member of the internal `_PyInterpreterFrame` structure with a `prev_instr` pointer, which reduces overhead in the main interpreter loop. The `f_lasti` attribute of Python-layer frame objects is preserved for backward-compatibility.

- bpo-46961 [https://bugs.python.org/issue?@action=redirect&bpo=46961]: Integer mod/remainder operations, including the three-argument form of

`pow()`, now consistently return ints from the global small integer cache when applicable.

- bpo-46962 [https://bugs.python.org/issue?@action=redirect&bpo=46962]: Classes and functions that unconditionally declared their docstrings ignoring the `--without-doc-strings` compilation flag no longer do so.

  The classes affected are **`ctypes.UnionType`**, `pickle.PickleBuffer`, **`testcapi.RecursingInfinitelyError`**, and `types.GenericAlias`.

  The functions affected are 24 methods in `ctypes`.

  Patch by Oleg Iarygin.

- bpo-46942 [https://bugs.python.org/issue?@action=redirect&bpo=46942]: Use Argument Clinic for the `types.MethodType` constructor. Patch by Oleg Iarygin.

- bpo-46764 [https://bugs.python.org/issue?@action=redirect&bpo=46764]: Fix wrapping bound methods with @classmethod

- bpo-43464 [https://bugs.python.org/issue?@action=redirect&bpo=43464]: Optimize `set.intersection()` for non-set arguments.

- bpo-46721 [https://bugs.python.org/issue?@action=redirect&bpo=46721]: Optimize `set.issuperset()` for non-set argument.

- bpo-46509 [https://bugs.python.org/issue?@action=redirect&bpo=46509]: Add type-specialized versions of the `Py_DECREF()`, and use them for `float`, `int`, `str`, `bool`, and `None` to avoid pointer-chasing at runtime where types are known at C compile time.

Construção

- bpo-46045 [https://bugs.python.org/issue?@action=redirect&bpo=46045]: Do not use POSIX semaphores on NetBSD

Núcleo e embutidos

- bpo-36819 [https://bugs.python.org/issue?@action=redirect&bpo=36819]: Fix crashes in built-in encoders with error handlers that return position less or equal than the starting position of non-encodable characters.

- bpo-34093 [https://bugs.python.org/issue?@action=redirect&bpo=34093]: `marshal.dumps()` uses `FLAG_REF` for all interned strings. This makes output more deterministic and helps reproducible build.

- bpo-26579 [https://bugs.python.org/issue?@action=redirect&bpo=26579]: Added `object.__getstate__` which provides the default implementation of the `__getstate__()` method.

  Copying and pickling instances of subclasses of builtin types bytearray, set, frozenset, collections.OrderedDict, collections.deque, weakref.WeakSet, and datetime.tzinfo now copies and pickles instance attributes implemented as slots.

Biblioteca

- gh-87901 [https://github.com/python/cpython/issues/87901]: Add the *encoding* parameter to **`os.popen()`**.

- gh-90997 [https://github.com/python/cpython/issues/90997]: Fix an issue where **`dis`** utilities may interpret populated inline cache entries as valid instructions.

- gh-92332 [https://github.com/python/cpython/issues/92332]: Deprecate **`typing.Text`** (removal of the class is currently not planned). Patch by Alex Waygood.

- gh-78157 [https://github.com/python/cpython/issues/78157]: Deprecate nested classes in enum definitions becoming members – in 3.13 they will be normal classes; add `member` and `nonmember` functions to allow control over results now.

- gh-92356 [https://github.com/python/cpython/issues/92356]: Fixed a performance regression in ctypes function calls.

- gh-90997 [https://github.com/python/cpython/issues/90997]: Show the actual named values stored in inline caches when `show_caches=True` is passed to **`dis`** utilities.

- gh-92301 [https://github.com/python/cpython/issues/92301]: Prefer `close_range()` to iterating over procfs for file descriptor closing in **`subprocess`** for better performance.

- gh-67248 [https://github.com/python/cpython/issues/67248]: Sort the miscellaneous topics in Cmd.do_help()

- gh-92210 [https://github.com/python/cpython/issues/92210]: Port `socket.__init__` to Argument Clinic. Patch by Cinder.

- gh-80010 [https://github.com/python/cpython/issues/80010]: Add support for generalized ISO 8601 parsing to **`datetime.datetime.fromisoformat()`**, **`datetime.date.fromisoformat()`** and **`datetime.time.fromisoformat()`**. Patch by Paul Ganssle.

- gh-92118 [https://github.com/python/cpython/issues/92118]: Fix a 3.11 regression in **`contextmanager()`**, which caused it to propagate exceptions with incorrect tracebacks.

- gh-90887 [https://github.com/python/cpython/issues/90887]: Adding `COPYFILE_STAT`, `COPYFILE_ACL` and `COPYFILE_XATTR` constants for **`os.fcopyfile()`** available in macOs.

- gh-91215 [https://github.com/python/cpython/issues/91215]: For **`@~dataclasses.dataclass`**, add *weakref_slot*. The new parameter defaults to `False`. If true, and if `slots=True`, add a slot named `"__weakref__"`, which will allow instances to be weakref'd. Contributed by Eric V. Smith

- gh-85984 [https://github.com/python/cpython/issues/85984]: New function os.login_tty() for Unix.

- gh-92128 [https://github.com/python/cpython/issues/92128]: Add `__class_getitem__()` to `logging.LoggerAdapter` and `logging.StreamHandler`, allowing them to be parameterized at runtime. Patch by Alex Waygood.

- gh-92049 [https://github.com/python/cpython/issues/92049]: Forbid pickling constants `re._constants.SUCCESS` etc. Previously, pickling did not fail, but the result could not be unpickled.

- gh-92062 [https://github.com/python/cpython/issues/92062]: `inspect.Parameter` now raises `ValueError` if `name` is a keyword, in addition to the existing check that it is an identifier.

- gh-87390 [https://github.com/python/cpython/issues/87390]: Add an `__unpacked__` attribute to `types.GenericAlias`. Patch by Jelle Zijlstra.

- gh-88089 [https://github.com/python/cpython/issues/88089]: Add support for generic `typing.NamedTuple`.

- gh-91996 [https://github.com/python/cpython/issues/91996]: New http.HTTPMethod enum to represent all the available HTTP request methods in a convenient way

- gh-91984 [https://github.com/python/cpython/issues/91984]: Modified test strings in test_argparse.py to not contain trailing spaces before end of line.

- gh-91952 [https://github.com/python/cpython/issues/91952]: Add `encoding="locale"` support to `TextIOWrapper.reconfigure()`.

- gh-91954 [https://github.com/python/cpython/issues/91954]: Add *encoding* and *errors* arguments to `subprocess.getoutput()` and `subprocess.getstatusoutput()`.

- bpo-47029 [https://bugs.python.org/issue?@action=redirect&bpo=47029]: Always close the read end of the pipe used by `multiprocessing.Queue` *after* the last write of buffered data to the write end of the pipe to avoid `BrokenPipeError` at garbage collection and at `multiprocessing.Queue.close()` calls. Patch by Géry Ogam.

- gh-91928 [https://github.com/python/cpython/issues/91928]: Add `datetime.UTC` alias for `datetime.timezone.utc`.

  Patch by Kabir Kwatra.

- gh-68966 [https://github.com/python/cpython/issues/68966]: The **`mailcap`** module is now deprecated and will be removed in Python 3.13. See **PEP 594** [https://peps.python.org/pep-0594/] for the rationale and the **`mimetypes`** module for an alternative. Patch by Victor Stinner.

- gh-91401 [https://github.com/python/cpython/issues/91401]: Provide a way to disable **`subprocess`** use of `vfork()` just in case it is ever needed and document the existing mechanism for `posix_spawn()`.

- gh-64783 [https://github.com/python/cpython/issues/64783]: Fix **`signal.NSIG`** value on FreeBSD to accept signal numbers greater than 32, like **`signal.SIGRTMIN`** and **`signal.SIGRTMAX`**. Patch by Victor Stinner.

- gh-91910 [https://github.com/python/cpython/issues/91910]: Add missing f prefix to f-strings in error messages from the **`multiprocessing`** and **`asyncio`** modules.

- gh-91860 [https://github.com/python/cpython/issues/91860]: Add **`typing.dataclass_transform()`**, implementing **PEP 681** [https://peps.python.org/pep-0681/]. Patch by Jelle Zijlstra.

- gh-91832 [https://github.com/python/cpython/issues/91832]: Add `required` attribute to **`argparse.Action`** repr output.

- gh-91827 [https://github.com/python/cpython/issues/91827]: In the **`tkinter`** module add method `info_patchlevel()` which returns the exact version of the Tcl library as a named tuple similar to **`sys.version_info`**.

- gh-84461 [https://github.com/python/cpython/issues/84461]: Add **`--enable-wasm-pthreads`** to enable pthreads support for WASM builds. `Emscripten/node` no longer has threading enabled by default. Include additional file systems.

- gh-91821 [https://github.com/python/cpython/issues/91821]: Fix unstable `test_from_tuple` test in `test_decimal.py`.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the xdrlib module.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the uu module.

- gh-91760 [https://github.com/python/cpython/issues/91760]: More strict rules will be applied for numerical group references and group names in regular expressions. For now, a deprecation warning is emitted for group references and group names which will be errors in future Python versions.

- gh-84461 [https://github.com/python/cpython/issues/84461]: Add provisional **`sys._emscripten_info`** named tuple with build-time and run-time information about Emscripten platform.

- gh-90623 [https://github.com/python/cpython/issues/90623]: **`signal.raise_signal()`** and **`os.kill()`** now check immediately for pending signals. Patch by Victor Stinner.

- gh-91734 [https://github.com/python/cpython/issues/91734]: Fix OSS audio support on Solaris.

- gh-90633 [https://github.com/python/cpython/issues/90633]: Include the passed value in the exception thrown by **`typing.assert_never()`**. Patch by Jelle Zijlstra.

- gh-91700 [https://github.com/python/cpython/issues/91700]: Compilation of regular expression containing a conditional expression `(?(group)...)` now raises an appropriate **`re.error`** if the group number refers to not defined group. Previously an internal RuntimeError was raised.

- gh-91231 [https://github.com/python/cpython/issues/91231]: Add an optional keyword *shutdown_timeout* parameter to the

multiprocessing.BaseManager constructor. Kill the process if terminate() takes longer than the timeout. Patch by Victor Stinner.

- gh-91621 [https://github.com/python/cpython/issues/91621]: Fix **typing.get_type_hints()** for **collections.abc.Callable**. Patch by Shantanu Jain.

- gh-90568 [https://github.com/python/cpython/issues/90568]: Parsing `\N` escapes of Unicode Named Character Sequences in a **regular expression** raises now **re.error** instead of `TypeError`.

- gh-91670 [https://github.com/python/cpython/issues/91670]: Remove deprecated `SO` config variable in **sysconfig**.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the telnetlib module.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the sunau module.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the spwd module.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the sndhdr module, as well as inline needed functionality for `email.mime.MIMEAudio`.

- gh-91616 [https://github.com/python/cpython/issues/91616]: **re** module, fix **fullmatch()** mismatch when using Atomic Grouping or Possessive Quantifiers.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the ‘pipes’ module.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the ossaudiodev module.

- bpo-47256 [https://bugs.python.org/issue?@action=redirect&bpo=47256]: **re** module, limit the maximum capturing group to 1,073,741,823 in 64-bit build,

this increases the depth of backtracking.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the nis module.

- gh-91595 [https://github.com/python/cpython/issues/91595]: Fix the comparison of character and integer inside **`Tools.gdb.libpython.write_repr()`**. Patch by Yu Liu.

- gh-74166 [https://github.com/python/cpython/issues/74166]: Add option to raise all errors from **`create_connection()`** in an **`ExceptionGroup`** when it fails to create a connection. The default remains to raise only the last error that had occurred when multiple addresses were tried.

- gh-91487 [https://github.com/python/cpython/issues/91487]: Optimize asyncio UDP speed, over 100 times faster when transferring a large file.

- gh-91575 [https://github.com/python/cpython/issues/91575]: Update case-insensitive matching in the **`re`** module to the latest Unicode version.

- gh-90622 [https://github.com/python/cpython/issues/90622]: In `concurrent.futures.process.ProcessPoolExecutor` disallow the "fork" multiprocessing start method when the new `max_tasks_per_child` feature is used as the mix of threads+fork can hang the child processes. Default to using the safe "spawn" start method in that circumstance if no `mp_context` was supplied.

- gh-89022 [https://github.com/python/cpython/issues/89022]: In **`sqlite3`**, `SQLITE_MISUSE` result codes are now mapped to **`InterfaceError`** instead of **`ProgrammingError`**. Also, more accurate exceptions are raised when binding parameters fail. Patch by Erlend E. Aasland.

- gh-91526 [https://github.com/python/cpython/issues/91526]: Stop calling `os.device_encoding(file.fileno())` in **`TextIOWrapper`**. It was complex, never documented, and didn't work for most cases. (Patch by Inada Naoki.)

- gh-88116 [https://github.com/python/cpython/issues/88116]: Change the frame-related functions in the `inspect` module to return a regular object (that is backwards compatible with the old tuple-like interface) that include the extended **PEP 657** [https://peps.python.org/pep-0657/] position information (end line number, column and end column). The affected functions are: `inspect.getframeinfo()`, `inspect.getouterframes()`, `inspect.getinnerframes()`, `inspect.stack()` and `inspect.trace()`. Patch by Pablo Galindo.

- gh-69093 [https://github.com/python/cpython/issues/69093]: Add indexing and slicing support to `sqlite3.Blob`. Patch by Aviv Palivoda and Erlend E. Aasland.

- gh-69093 [https://github.com/python/cpython/issues/69093]: Add context manager support to `sqlite3.Blob`. Patch by Aviv Palivoda and Erlend E. Aasland.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate nntplib.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate msilib.

- gh-91404 [https://github.com/python/cpython/issues/91404]: Improve the performance of `re` matching by using computed gotos (or “threaded code”) on supported platforms and removing expensive pointer indirections.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the imghdr module.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the crypt module.

- gh-91276 [https://github.com/python/cpython/issues/91276]: Make space for longer opcodes in `dis` output.

- bpo-47000 [https://bugs.python.org/issue?@action=redirect&bpo=47000]: Make `TextIOWrapper` uses locale encoding when `encoding="locale"` is specified even in UTF-8 mode.

- gh-91230 [https://github.com/python/cpython/issues/91230]: **`warnings.catch_warnings()`** now accepts arguments for **`warnings.simplefilter()`**, providing a more concise way to locally ignore warnings or convert them to errors.

- gh-91217 [https://github.com/python/cpython/issues/91217]: Deprecate the chunk module.

- gh-91498 [https://github.com/python/cpython/issues/91498]: Add the `TCP_CONNECTION_INFO` option (available on macOS) to **`socket`**.

- bpo-47260 [https://bugs.python.org/issue?@action=redirect&bpo=47260]: Fix `os.closerange()` potentially being a no-op in a Linux seccomp sandbox.

- bpo-47087 [https://bugs.python.org/issue?@action=redirect&bpo=47087]: Implement `typing.Required` and `typing.NotRequired` (**PEP 655** [https://peps.python.org/pep-0655/]). Patch by David Foster and Jelle Zijlstra.

- bpo-47061 [https://bugs.python.org/issue?@action=redirect&bpo=47061]: Deprecate cgi and cgitb.

- bpo-47061 [https://bugs.python.org/issue?@action=redirect&bpo=47061]: Deprecate audioop.

- bpo-47000 [https://bugs.python.org/issue?@action=redirect&bpo=47000]: Add **`locale.getencoding()`** to get the current locale encoding. It is similar to `locale.getpreferredencoding(False)` but ignores the Python UTF-8 Mode.

- bpo-42012 [https://bugs.python.org/issue?@action=redirect&bpo=42012]: Add **`wsgiref.types`**, containing WSGI-specific types for static type checking.

- bpo-47227 [https://bugs.python.org/issue?@action=redirect&bpo=47227]: Suppress expression chaining for more **`re`** parsing errors.

- bpo-47211 [https://bugs.python.org/issue?@action=redirect&bpo=47211]: Remove undocumented and never working function `re.template()` and flag `re.TEMPLATE`. This was later reverted in 3.11.0b2 and deprecated instead.

- bpo-47135 [https://bugs.python.org/issue?@action=redirect&bpo=47135]: **`decimal.localcontext()`** now accepts context attributes via keyword arguments

- bpo-43323 [https://bugs.python.org/issue?@action=redirect&bpo=43323]: Fix errors in the **`email`** module if the charset itself contains undecodable/unencodable characters.

- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Disassembly of quickened code.

- bpo-46681 [https://bugs.python.org/issue?@action=redirect&bpo=46681]: Forward gzip.compress() compresslevel to zlib.

- bpo-45100 [https://bugs.python.org/issue?@action=redirect&bpo=45100]: Add **`typing.get_overloads()`** and **`typing.clear_overloads()`**. Patch by Jelle Zijlstra.

- bpo-44807 [https://bugs.python.org/issue?@action=redirect&bpo=44807]: **`typing.Protocol`** no longer silently replaces **`__init__()`** methods defined on subclasses. Patch by Adrian Garcia Badaracco.

- bpo-46787 [https://bugs.python.org/issue?@action=redirect&bpo=46787]: Fix **`concurrent.futures.ProcessPoolExecutor`** exception memory leak

- bpo-46720 [https://bugs.python.org/issue?@action=redirect&bpo=46720]: Add support for path-like objects to **`multiprocessing.set_executable()`** for Windows to be on a par with Unix-like systems. Patch by Géry Ogam.

- bpo-46696 [https://bugs.python.org/issue?@action=redirect&bpo=46696]: Add `SO_INCOMING_CPU` constant to **`socket`**.

- bpo-46053 [https://bugs.python.org/issue?@action=redirect&bpo=46053]: Fix OSS audio support on NetBSD.

- bpo-45639 [https://bugs.python.org/issue?@action=redirect&bpo=45639]: `image/avif` and `image/webp` were added to **`mimetypes`**.

- bpo-46285 [https://bugs.python.org/issue?@action=redirect&bpo=46285]: Add command-line option `-p`/`--protocol` to module **`http.server`** which specifies the HTTP version to which the server is conformant (HTTP/1.1 conformant servers can now be run from the command-line interface of module **`http.server`**). Patch by Géry Ogam.

- bpo-44791 [https://bugs.python.org/issue?@action=redirect&bpo=44791]: Accept ellipsis as the last argument of **`typing.Concatenate`**.

- bpo-46547 [https://bugs.python.org/issue?@action=redirect&bpo=46547]: Remove variables leaking into `pydoc.Helper` class namespace.

- bpo-46415 [https://bugs.python.org/issue?@action=redirect&bpo=46415]: Fix ipaddress.ip_{address,interface,network} raising TypeError instead of ValueError if given invalid tuple as address parameter.

- bpo-46075 [https://bugs.python.org/issue?@action=redirect&bpo=46075]: `CookieJar` with `DefaultCookiePolicy` now can process cookies from localhost with domain=localhost explicitly specified in Set-Cookie header.

- bpo-45995 [https://bugs.python.org/issue?@action=redirect&bpo=45995]: Add a “z” option to the string formatting specification that coerces negative zero floating-point values to positive zero after rounding to the format precision. Contributed by John Belmonte.

- bpo-26175 [https://bugs.python.org/issue?@action=redirect&bpo=26175]: Fully implement the **`io.BufferedIOBase`** or **`io.TextIOBase`** interface for **`tempfile.SpooledTemporaryFile`** objects. This lets them work correctly with higher-level layers (like compression modules). Patch by Carey Metcalfe.

- bpo-45138 [https://bugs.python.org/issue?@action=redirect&bpo=45138]: Fix a regression in the **`sqlite3`** trace callback where bound parameters were not expanded in the passed statement string. The regression was introduced in Python 3.10 by bpo-40318 [https://bugs.python.org/issue?@action=redirect&bpo=40318]. Patch by Erlend E. Aasland.

- bpo-44863 [https://bugs.python.org/issue?@action=redirect&bpo=44863]: Allow **`TypedDict`** subclasses to also include **`Generic`** as a base class in class based syntax. Thereby allowing the user to define a generic `TypedDict`, just like a user-defined generic but with `TypedDict` semantics.

- bpo-44587 [https://bugs.python.org/issue?@action=redirect&bpo=44587]: Fix BooleanOptionalAction to not automatically add a default string. If a default string is desired, use a formatter to add it.

- bpo-43827 [https://bugs.python.org/issue?@action=redirect&bpo=43827]: All positional-or-keyword parameters to `ABCMeta.__new__` are now positional-only to avoid conflicts with keyword arguments to be passed to **`__init_subclass__()`**.

- bpo-43218 [https://bugs.python.org/issue?@action=redirect&bpo=43218]: Prevent creation of a venv whose path contains the PATH separator. This could affect the usage of the activate script. Patch by Dustin Rodrigues.

- bpo-38435 [https://bugs.python.org/issue?@action=redirect&bpo=38435]: Add a `process_group` parameter to **`subprocess.Popen`** to help move more things off of the unsafe `preexec_fn` parameter.

- bpo-42066 [https://bugs.python.org/issue?@action=redirect&bpo=42066]: Fix cookies getting sorted in **`CookieJar.__iter__()`** which is an extra behavior and not mentioned in RFC 2965 or Netscape cookie protocol. Now the cookies in `CookieJar` follows the order of the `Set-Cookie` header. Patch by Iman Kermani.

- bpo-40617 [https://bugs.python.org/issue?@action=redirect&bpo=40617]: Add **`create_window_function()`** to **`sqlite3.Connection`** for creating aggregate window functions. Patch by Erlend E. Aasland.

- bpo-40676 [https://bugs.python.org/issue?@action=redirect&bpo=40676]: Convert **csv** to use Argument Clinic for **csv.field_size_limit()**, **csv.get_dialect()**, **csv.unregister_dialect()** and **csv.list_dialects()**.

- bpo-39716 [https://bugs.python.org/issue?@action=redirect&bpo=39716]: Raise an `ArgumentError` when the same subparser name is added twice to an `argparse.ArgumentParser`. This is consistent with the (default) behavior when the same option string is added twice to an `ArgumentParser`.

- bpo-36073 [https://bugs.python.org/issue?@action=redirect&bpo=36073]: Raise **ProgrammingError** instead of segfaulting on recursive usage of cursors in **sqlite3** converters. Patch by Sergey Fedoseev.

- bpo-34975 [https://bugs.python.org/issue?@action=redirect&bpo=34975]: Adds a `start_tls()` method to **StreamWriter**, which upgrades the connection with TLS using the given **SSLContext**.

- bpo-22276 [https://bugs.python.org/issue?@action=redirect&bpo=22276]: **Path** methods **glob()** and **rglob()** return only directories if *pattern* ends with a pathname components separator (`/` or **sep**). Patch by Eisuke Kawashima.

- bpo-24905 [https://bugs.python.org/issue?@action=redirect&bpo=24905]: Add **blobopen()** to **sqlite3.Connection**. **sqlite3.Blob** allows incremental I/O operations on blobs. Patch by Aviv Palivoda and Erlend E. Aasland.

Documentação

- gh-91888 [https://github.com/python/cpython/issues/91888]: Add a new `gh` role to the documentation to link to GitHub issues.
- gh-91783 [https://github.com/python/cpython/issues/91783]: Document security issues concerning the use of the function **shutil.unpack_archive()**
- gh-91547 [https://github.com/python/cpython/issues/91547]: Remove “Undocumented modules” page.

- gh-91298 [https://github.com/python/cpython/issues/91298]: In `importlib.resources.abc`, refined the documentation of the Traversable Protocol, applying changes from importlib_resources 5.7.1.
- bpo-44347 [https://bugs.python.org/issue?@action=redirect&bpo=44347]: Clarify the meaning of *dirs_exist_ok*, a kwarg of **`shutil.copytree()`**.
- bpo-36329 [https://bugs.python.org/issue?@action=redirect&bpo=36329]: Remove 'make -C Doc serve' in favour of 'make -C Doc htmlview'
- bpo-47189 [https://bugs.python.org/issue?@action=redirect&bpo=47189]: Add a What's New in Python 3.11 entry for the Faster CPython project. Documentation by Ken Jin and Kumar Aditya.
- bpo-38668 [https://bugs.python.org/issue?@action=redirect&bpo=38668]: Update the introduction to documentation for **`os.path`** to remove warnings that became irrelevant after the implementations of **PEP 383** [https://peps.python.org/pep-0383/] and **PEP 529** [https://peps.python.org/pep-0529/].
- bpo-47115 [https://bugs.python.org/issue?@action=redirect&bpo=47115]: The documentation now lists which members of C structs are part of the Limited API/Stable ABI.
- bpo-46962 [https://bugs.python.org/issue?@action=redirect&bpo=46962]: All docstrings in code snippets are now wrapped into **`PyDoc_STR`** to follow the guideline of **PEP 7's Documentation Strings paragraph** [https://peps.python.org/pep-0007/#documentation-strings]. Patch by Oleg Iarygin.
- bpo-26792 [https://bugs.python.org/issue?@action=redirect&bpo=26792]: Improve the docstrings of **`runpy.run_module()`** and **`runpy.run_path()`**. Original patch by Andrew Brezovsky.

Testes

- gh-92169 [https://github.com/python/cpython/issues/92169]: Use `warnings_helper.import_deprecated()` to import deprecated modules uniformly in tests. Patch by Hugo van Kemenade.
- gh-84461 [https://github.com/python/cpython/issues/84461]: When multiprocessing is enabled, libregrtest can now use a Python executable other than `sys.executable` via the `--python` flag.
- gh-91904 [https://github.com/python/cpython/issues/91904]: Fix initialization of **`PYTHONREGRTEST_UNICODE_GUARD`** which prevented running regression tests on non-UTF-8 locale.

- gh-91752 [https://github.com/python/cpython/issues/91752]: Added @requires_zlib to test.test_tools.test_freeze.TestFreeze.
- gh-91607 [https://github.com/python/cpython/issues/91607]: Fix `test_concurrent_futures` to test the correct multiprocessing start method context in several cases where the test logic mixed this up.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Threading tests are now skipped on WASM targets without pthread support.
- bpo-47109 [https://bugs.python.org/issue?@action=redirect&bpo=47109]: Test for **`ctypes.macholib.dyld`**, **`ctypes.macholib.dylib`**, and **`ctypes.macholib.framework`** are brought from manual pre-**`unittest`** times to **`ctypes.test`** location and structure. Patch by Oleg Iarygin.
- bpo-29890 [https://bugs.python.org/issue?@action=redirect&bpo=29890]: Add tests for **`ipaddress.IPv4Interface`** and **`ipaddress.IPv6Interface`** construction with tuple arguments. Original patch and tests by louisom.

Construção

- gh-89452 [https://github.com/python/cpython/issues/89452]: gdbm-compat is now preferred over ndbm if both are available on the system. This allows avoiding the problematic ndbm.h on macOS.
- gh-91731 [https://github.com/python/cpython/issues/91731]: Python is now built with `-std=c11` compiler option, rather than `-std=c99`. Patch by Victor Stinner.
- bpo-47152 [https://bugs.python.org/issue?@action=redirect&bpo=47152]: Add script and make target for generating `sre_constants.h`.
- bpo-47103 [https://bugs.python.org/issue?@action=redirect&bpo=47103]: As construções de `PGInstrument` do Windows agora copiam uma DLL necessária para o diretório de saída, facilitando a execução do estágio de perfil de uma construção do PGO.

Windows

- bpo-46907 [https://bugs.python.org/issue?@action=redirect&bpo=46907]: Update Windows installer to use SQLite 3.38.3.
- bpo-47239 [https://bugs.python.org/issue?@action=redirect&bpo=47239]: Fixed –list and –list-paths output for Gerenciador de instalação do Python when

used in an active virtual environment.

- bpo-46907 [https://bugs.python.org/issue?@action=redirect&bpo=46907]: Update Windows installer to use SQLite 3.38.2.
- bpo-46785 [https://bugs.python.org/issue?@action=redirect&bpo=46785]: Fix race condition between **`os.stat()`** and unlinking a file on Windows, by using errors codes returned by `FindFirstFileW()` when appropriate in `win32_xstat_impl`.
- bpo-40859 [https://bugs.python.org/issue?@action=redirect&bpo=40859]: Update Windows build to use xz-5.2.5

macOS

- bpo-46907 [https://bugs.python.org/issue?@action=redirect&bpo=46907]: Update macOS installer to SQLite 3.38.4.

Ferramentas/Demos

- gh-91583 [https://github.com/python/cpython/issues/91583]: Fix regression in the code generated by Argument Clinic for functions with the `defining_class` parameter.
- gh-91575 [https://github.com/python/cpython/issues/91575]: Add script `Tools/scripts/generate_re_casefix.py` and the make target `regen-re` for generating additional data for case-insensitive matching according to the current Unicode version.
- gh-91551 [https://github.com/python/cpython/issues/91551]: Remove the ancient Pynche color editor. It has moved to https://gitlab.com/warsaw/pynche

API C

- gh-88279 [https://github.com/python/cpython/issues/88279]: Deprecate the C functions: **`PySys_SetArgv()`**, **`PySys_SetArgvEx()`**, **`PySys_SetPath()`**. Patch by Victor Stinner.

- gh-92154 [https://github.com/python/cpython/issues/92154]: Added the **`PyCode_GetCode()`** function. This function does the equivalent of the Python code `getattr(code_object, 'co_code')`.

- gh-92173 [https://github.com/python/cpython/issues/92173]: Fix the `closure` argument to **`PyEval_EvalCodeEx()`**.

- gh-91320 [https://github.com/python/cpython/issues/91320]: Fix C++ compiler warnings about "old-style cast" (`g++ -Wold-style-cast`) in the Python C API. Use C++ `reinterpret_cast<>` and `static_cast<>` casts when the Python C API is used in C++. Patch by Victor Stinner.

- gh-80527 [https://github.com/python/cpython/issues/80527]: Mark functions as deprecated by **PEP 623** [https://peps.python.org/pep-0623/]: **`PyUnicode_AS_DATA()`**, **`PyUnicode_AS_UNICODE()`**, **`PyUnicode_GET_DATA_SIZE()`**, **`PyUnicode_GET_SIZE()`**. Patch by Victor Stinner.

- gh-91768 [https://github.com/python/cpython/issues/91768]: **`Py_REFCNT()`**, **`Py_TYPE()`**, **`Py_SIZE()`** and **`Py_IS_TYPE()`** functions argument type is now `PyObject*`, rather than `const PyObject*`. Patch by Victor Stinner.

- gh-91020 [https://github.com/python/cpython/issues/91020]: Add `PyBytes_Type.tp_alloc` to initialize `PyBytesObject.ob_shash` for bytes subclasses.

- bpo-40421 [https://bugs.python.org/issue?@action=redirect&bpo=40421]: Add `PyFrame_GetLasti` C-API function to access frame object's `f_lasti` attribute safely from C code.

- bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134]: Remove the `Include/code.h` header file. C extensions should only include the main `<Python.h>` header file. Patch by Victor Stinner.

- bpo-47169 [https://bugs.python.org/issue?@action=redirect&bpo=47169]: **`PyOS_CheckStack()`** is now exported in the Stable ABI on Windows.

- bpo-47169 [https://bugs.python.org/issue?@action=redirect&bpo=47169]: **`PyThread_get_thread_native_id()`** is excluded from the stable ABI on platforms where it doesn't exist (like Solaris).

- bpo-46343 [https://bugs.python.org/issue?@action=redirect&bpo=46343]: Added `PyErr_GetHandledException()` and `PyErr_SetHandledException()` as simpler alternatives to `PyErr_GetExcInfo()` and `PyErr_SetExcInfo()`.

  They are included in the stable ABI.

Python 3.11.0 alpha 7

Data de lançamento: 2022-04-05

Núcleo e embutidos

- bpo-47212 [https://bugs.python.org/issue?@action=redirect&bpo=47212]: Raise `IndentationError` instead of `SyntaxError` for a bare `except` with no following indent. Improve `SyntaxError` locations for an un-parenthesized generator used as arguments. Patch by Matthieu Dartiailh.
- bpo-47186 [https://bugs.python.org/issue?@action=redirect&bpo=47186]: Replace `JUMP_IF_NOT_EG_MATCH` by `CHECK_EG_MATCH` + jump.
- bpo-47176 [https://bugs.python.org/issue?@action=redirect&bpo=47176]: Emscripten builds cannot handle signals in the usual way due to platform limitations. Python can now handle signals. To use, set Module.Py_EmscriptenSignalBuffer to be a single byte SharedArrayBuffer and set Py_EMSCRIPTEN_SIGNAL_HANDLING to 1. Writing a number into the SharedArrayBuffer will cause the corresponding signal to be raised into the Python thread.
- bpo-47186 [https://bugs.python.org/issue?@action=redirect&bpo=47186]: Replace `JUMP_IF_NOT_EXC_MATCH` by `CHECK_EXC_MATCH` + jump.
- bpo-47120 [https://bugs.python.org/issue?@action=redirect&bpo=47120]: Replace the absolute jump opcode `JUMP_NO_INTERRUPT` by the relative `JUMP_BACKWARD_NO_INTERRUPT`.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Avoid unnecessary allocations when comparing code objects.
- bpo-47182 [https://bugs.python.org/issue?@action=redirect&bpo=47182]: Fix a crash when using a named unicode character like `"\N{digit nine}"` after

the main interpreter has been initialized a second time.

- bpo-47162 [https://bugs.python.org/issue?@action=redirect&bpo=47162]: O WebAssembly não consegue lidar com conversões de ponteiros para função ruins (contagem ou tipos diferentes de argumento). O Python agora pode usar trampolins de chamada para atenuar o problema. Defina **`PY_CALL_TRAMPOLINE`** para ativar os trampolins de chamada.
- bpo-46775 [https://bugs.python.org/issue?@action=redirect&bpo=46775]: Some Windows system error codes(>= 10000) are now mapped into the correct errno and may now raise a subclass of **`OSError`**. Patch by Donghee Na.
- bpo-47129 [https://bugs.python.org/issue?@action=redirect&bpo=47129]: Improve error messages in f-string syntax errors concerning empty expressions.
- bpo-47117 [https://bugs.python.org/issue?@action=redirect&bpo=47117]: Fix a crash if we fail to decode characters in interactive mode if the tokenizer buffers are uninitialized. Patch by Pablo Galindo.
- bpo-47127 [https://bugs.python.org/issue?@action=redirect&bpo=47127]: Speed up calls to c functions with keyword arguments by 25% with specialization. Patch by Kumar Aditya.
- bpo-47120 [https://bugs.python.org/issue?@action=redirect&bpo=47120]: Replaced **`JUMP_ABSOLUTE`** by the relative jump **`JUMP_BACKWARD`**.
- bpo-42197 [https://bugs.python.org/issue?@action=redirect&bpo=42197]: **`PyFrame_FastToLocalsWithError()`** and **`PyFrame_LocalsToFast()`** are no longer called during profiling nor tracing. C code can access the `f_locals` attribute of **`PyFrameObject`** by calling **`PyFrame_GetLocals()`**.
- bpo-47070 [https://bugs.python.org/issue?@action=redirect&bpo=47070]: Improve performance of `array_inplace_repeat` by reducing the number of invocations of `memcpy`. Refactor the `repeat` and inplace `repeat` methods of `array`, `bytes`, `bytearray` and `unicodeobject` to use the common `_PyBytes_Repeat`.
- bpo-47053 [https://bugs.python.org/issue?@action=redirect&bpo=47053]: Reduce de-optimization in the specialized `BINARY_OP_INPLACE_ADD_UNICODE` opcode.
- bpo-47045 [https://bugs.python.org/issue?@action=redirect&bpo=47045]: Remove the `f_state` field from the _PyInterpreterFrame struct. Add the `owner` field to the _PyInterpreterFrame struct to make ownership explicit to simplify clearing and deallocing frames and generators.

Biblioteca

- bpo-46968 [https://bugs.python.org/issue?@action=redirect&bpo=46968]: Check for the existence of the “sys/auxv.h” header in `faulthandler` to avoid compilation problems in systems where this header doesn’t exist. Patch by Pablo Galindo

Núcleo e embutidos

- bpo-46329 [https://bugs.python.org/issue?@action=redirect&bpo=46329]: Use low bit of `LOAD_GLOBAL` to indicate whether to push a `NULL` before the global. Helps streamline the call sequence a bit.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Quicken bytecode in-place by storing it as part of the corresponding `PyCodeObject`.
- bpo-47012 [https://bugs.python.org/issue?@action=redirect&bpo=47012]: Speed up iteration of `bytes` and `bytearray` by 30%. Patch by Kumar Aditya.
- bpo-47009 [https://bugs.python.org/issue?@action=redirect&bpo=47009]: Improved the performance of `list.append()` and list comprehensions by optimizing for the common case, where no resize is needed. Patch by Dennis Sweeney.
- bpo-47005 [https://bugs.python.org/issue?@action=redirect&bpo=47005]: Improve performance of `bytearray_repeat` and `bytearray_irepeat` by reducing the number of invocations of `memcpy`.

Biblioteca

- bpo-46829 [https://bugs.python.org/issue?@action=redirect&bpo=46829]: Deprecate passing a message into `asyncio.Future.cancel()` and `asyncio.Task.cancel()`

Núcleo e embutidos

- bpo-46993 [https://bugs.python.org/issue?@action=redirect&bpo=46993]: Speed up `bytearray` creation from `list` and `tuple` by 40%. Patch by Kumar Aditya.

- bpo-39829 [https://bugs.python.org/issue?@action=redirect&bpo=39829]: Removed the `__len__()` call when initializing a list and moved initializing to `list_extend`. Patch by Jeremiah Pascual.
- bpo-46944 [https://bugs.python.org/issue?@action=redirect&bpo=46944]: Speed up throwing exception in generator with **`METH_FASTCALL`** calling convention. Patch by Kumar Aditya.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Modify **`STORE_SUBSCR`** to use an inline cache entry (rather than its oparg) as an adaptive counter.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use inline caching for **`PRECALL`** and **`CALL`**, and remove the internal machinery for managing the (now unused) non-inline caches.
- bpo-46881 [https://bugs.python.org/issue?@action=redirect&bpo=46881]: Statically allocate and initialize the latin1 characters.
- bpo-46838 [https://bugs.python.org/issue?@action=redirect&bpo=46838]: Improve syntax errors for incorrect function definitions. Patch by Pablo Galindo
- bpo-43721 [https://bugs.python.org/issue?@action=redirect&bpo=43721]: Fix docstrings of **`getter`**, **`setter`**, and **`deleter`** to clarify that they create a new copy of the property.
- bpo-43224 [https://bugs.python.org/issue?@action=redirect&bpo=43224]: Make grammar changes required for PEP 646.

Biblioteca

- bpo-47208 [https://bugs.python.org/issue?@action=redirect&bpo=47208]: Allow vendors to override **`CTYPES_MAX_ARGCOUNT`**.

- bpo-23689 [https://bugs.python.org/issue?@action=redirect&bpo=23689]: **`re`** module: fix memory leak when a match is terminated by a signal or memory allocation failure. Patch by Ma Lin.

- bpo-47167 [https://bugs.python.org/issue?@action=redirect&bpo=47167]: Allow overriding a future compliance check in **`asyncio.Task`**.

- bpo-47151 [https://bugs.python.org/issue?@action=redirect&bpo=47151]: When subprocess tries to use vfork, it now falls back to fork if vfork returns

an error. This allows use in situations where vfork isn't allowed by the OS kernel.

- bpo-47152 [https://bugs.python.org/issue?@action=redirect&bpo=47152]: Convert the `re` module into a package. Deprecate modules `sre_compile`, `sre_constants` and `sre_parse`.

- bpo-4833 [https://bugs.python.org/issue?@action=redirect&bpo=4833]: Add **`ZipFile.mkdir()`**

- bpo-27929 [https://bugs.python.org/issue?@action=redirect&bpo=27929]: Fix **`asyncio.loop.sock_connect()`** to only resolve names for **`socket.AF_INET`** or **`socket.AF_INET6`** families. Resolution may not make sense for other families, like **`socket.AF_BLUETOOTH`** and **`socket.AF_UNIX`**.

- bpo-14265 [https://bugs.python.org/issue?@action=redirect&bpo=14265]: Adds the fully qualified test name to unittest output

- bpo-47061 [https://bugs.python.org/issue?@action=redirect&bpo=47061]: Deprecate the aifc module.

- bpo-39622 [https://bugs.python.org/issue?@action=redirect&bpo=39622]: Handle Ctrl+C in asyncio programs to interrupt the main task.

- bpo-47101 [https://bugs.python.org/issue?@action=redirect&bpo=47101]: **`hashlib.algorithms_available`** now lists only algorithms that are provided by activated crypto providers on OpenSSL 3.0. Legacy algorithms are not listed unless the legacy provider has been loaded into the default OSSL context.

- bpo-47099 [https://bugs.python.org/issue?@action=redirect&bpo=47099]: All **`URLError`** exception messages raised in **`urllib.request.URLopener`** now contain a colon between `ftp error` and the rest of the message. Previously, **`open_ftp()`** missed the colon. Patch by Oleg Iarygin.

- bpo-47099 [https://bugs.python.org/issue?@action=redirect&bpo=47099]: Exception chaining is changed from

`Exception.with_traceback()`/**`sys.exc_info()`** to **PEP 3134** [https://peps.python.org/pep-3134/]. Patch by Oleg Iarygin.

- bpo-47095 [https://bugs.python.org/issue?@action=redirect&bpo=47095]: **`hashlib`**'s internal `_blake2` module now prefers `libb2` from https://www.blake2.net/ over Python's vendored copy of blake2.

- bpo-47098 [https://bugs.python.org/issue?@action=redirect&bpo=47098]: The Keccak Code Package for **`hashlib`**'s internal `_sha3` module has been replaced with tiny_sha3. The module is used as fallback when Python is built without OpenSSL.

- bpo-47088 [https://bugs.python.org/issue?@action=redirect&bpo=47088]: Implement **`typing.LiteralString`**, part of **PEP 675** [https://peps.python.org/pep-0675/]. Patch by Jelle Zijlstra.

- bpo-42885 [https://bugs.python.org/issue?@action=redirect&bpo=42885]: Optimize **`re.search()`**, **`re.split()`**, **`re.findall()`**, **`re.finditer()`** and **`re.sub()`** for regular expressions starting with `\A` or `^`.

- bpo-23691 [https://bugs.python.org/issue?@action=redirect&bpo=23691]: Protect the **`re.finditer()`** iterator from re-entering.

- bpo-47067 [https://bugs.python.org/issue?@action=redirect&bpo=47067]: Optimize calling `GenericAlias` objects by using **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` and by replacing `PyObject_SetAttrString` with `PyObject_SetAttr`.

- bpo-28080 [https://bugs.python.org/issue?@action=redirect&bpo=28080]: Add the *metadata_encoding* parameter in the **`zipfile.ZipFile`** constructor and the `--metadata-encoding` option in the **`zipfile`** CLI to allow reading zipfiles using non-standard codecs to encode the filenames within the archive.

- bpo-47000 [https://bugs.python.org/issue?@action=redirect&bpo=47000]: Make **`io.text_encoding()`** returns "utf-8" when UTF-8 mode is enabled.

- bpo-42369 [https://bugs.python.org/issue?@action=redirect&bpo=42369]: Fix thread safety of **`zipfile._SharedFile.tell()`** to avoid a "zipfile.BadZipFile: Bad CRC-32 for file" exception when reading a **`ZipFile`** from multiple threads.

- bpo-38256 [https://bugs.python.org/issue?@action=redirect&bpo=38256]: Fix **`binascii.crc32()`** when it is compiled to use zlib'c crc32 to work properly on inputs 4+GiB in length instead of returning the wrong result. The workaround prior to this was to always feed the function data in increments smaller than 4GiB or to just call the zlib module function.

  We also have **`binascii.crc32()`** release the GIL when computing on larger inputs as **`zlib.crc32()`** and **`hashlib`** do.

  This also boosts performance on Windows as it now uses the zlib crc32 implementation for **`binascii.crc32()`** for a 2-3x speedup.

  That the stdlib has a crc32 API in two modules is a known historical oddity. This moves us closer to a single implementation behind them.

- bpo-47066 [https://bugs.python.org/issue?@action=redirect&bpo=47066]: Global inline flags (e.g. `(?i)`) can now only be used at the start of the regular expressions. Using them not at the start of expression was deprecated since Python 3.6.

- bpo-39394 [https://bugs.python.org/issue?@action=redirect&bpo=39394]: A warning about inline flags not at the start of the regular expression now contains the position of the flag.

- bpo-433030 [https://bugs.python.org/issue?@action=redirect&bpo=433030]: Add support of atomic grouping (`(?>...)`) and possessive quantifiers (`*+`, `++`, `?+`, `{m,n}+`) in **`regular expressions`**.

- bpo-47062 [https://bugs.python.org/issue?@action=redirect&bpo=47062]: Implement **`asyncio.Runner`** context manager.

- bpo-46382 [https://bugs.python.org/issue?@action=redirect&bpo=46382]: **`dataclass()`** `slots=True` now correctly omits slots already defined in base classes. Patch by Arie Bovenberg.

- bpo-47057 [https://bugs.python.org/issue?@action=redirect&bpo=47057]: Use FASTCALL convention for `FutureIter.throw()`

- bpo-47061 [https://bugs.python.org/issue?@action=redirect&bpo=47061]: Deprecate the various modules listed by **PEP 594** [https://peps.python.org/pep-0594/]:

  aifc, asynchat, asyncore, audioop, cgi, cgitb, chunk, crypt, imghdr, msilib, nntplib, nis, ossaudiodev, pipes, smtpd, sndhdr, spwd, sunau, telnetlib, uu, xdrlib

- bpo-34790 [https://bugs.python.org/issue?@action=redirect&bpo=34790]: Remove passing coroutine objects to **`asyncio.wait()`**.

- bpo-47039 [https://bugs.python.org/issue?@action=redirect&bpo=47039]: Normalize `repr()` of asyncio future and task objects.

- bpo-2604 [https://bugs.python.org/issue?@action=redirect&bpo=2604]: Fix bug where doctests using globals would fail when run multiple times.

- bpo-45150 [https://bugs.python.org/issue?@action=redirect&bpo=45150]: Add **`hashlib.file_digest()`** helper for efficient hashing of file object.

- bpo-34861 [https://bugs.python.org/issue?@action=redirect&bpo=34861]: Made cumtime the default sorting key for cProfile

- bpo-45997 [https://bugs.python.org/issue?@action=redirect&bpo=45997]: Fix **`asyncio.Semaphore`** re-aquiring FIFO order.

- bpo-47022 [https://bugs.python.org/issue?@action=redirect&bpo=47022]: The **`asynchat`**, **`asyncore`** and **`smtpd`** modules have been deprecated since at least Python 3.6. Their documentation and deprecation warnings and have now been updated to note they will removed in Python 3.12 (**PEP 594** [https://peps.python.org/pep-0594/]).

- bpo-43253 [https://bugs.python.org/issue?@action=redirect&bpo=43253]: Fix a crash when closing transports where the underlying socket handle is already invalid on the Proactor event loop.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: `select.select()` now passes `NULL` to `select` for each empty fdset.

- bpo-47004 [https://bugs.python.org/issue?@action=redirect&bpo=47004]: Apply bugfixes from importlib_metadata 4.11.3, including bugfix for EntryPoint.extras, which was returning match objects and not the extras strings.

- bpo-46998 [https://bugs.python.org/issue?@action=redirect&bpo=46998]: Allow subclassing of `typing.Any`. Patch by Shantanu Jain.

- bpo-46995 [https://bugs.python.org/issue?@action=redirect&bpo=46995]: Deprecate missing `asyncio.Task.set_name()` for third-party task implementations, schedule making it mandatory in Python 3.13.

- bpo-46994 [https://bugs.python.org/issue?@action=redirect&bpo=46994]: Accept explicit contextvars.Context in `asyncio.create_task()` and `asyncio.loop.create_task()`.

- bpo-46981 [https://bugs.python.org/issue?@action=redirect&bpo=46981]: `typing.get_args(typing.Tuple[()])` now returns `()` instead of `((),)`.

- bpo-46968 [https://bugs.python.org/issue?@action=redirect&bpo=46968]: Add `os.sysconf_names['SC_MINSIGSTKSZ']`.

- bpo-46985 [https://bugs.python.org/issue?@action=redirect&bpo=46985]: Upgrade pip wheel bundled with ensurepip (pip 22.0.4)

- bpo-46968 [https://bugs.python.org/issue?@action=redirect&bpo=46968]: `faulthandler`: On Linux 5.14 and newer, dynamically determine size of signal handler stack size CPython allocates using `getauxval(AT_MINSIGSTKSZ)`. This changes allows for Python extension's request to Linux kernel to use AMX_TILE instruction set

on Sapphire Rapids Xeon processor to succeed, unblocking use of the ISA in frameworks.

- bpo-46917 [https://bugs.python.org/issue?@action=redirect&bpo=46917]: The `math.nan` value is now always available. Patch by Victor Stinner.

- bpo-46955 [https://bugs.python.org/issue?@action=redirect&bpo=46955]: Expose `asyncio.base_events.Server` as `asyncio.Server`. Patch by Stefan Zabka.

- bpo-23325 [https://bugs.python.org/issue?@action=redirect&bpo=23325]: The `signal` module no longer assumes that `SIG_IGN` and `SIG_DFL` are small int singletons.

- bpo-46932 [https://bugs.python.org/issue?@action=redirect&bpo=46932]: Update bundled libexpat to 2.4.7

- bpo-46933 [https://bugs.python.org/issue?@action=redirect&bpo=46933]: The `pwd` module is now optional. `os.path.expanduser()` returns the path when the `pwd` module is not available.

- bpo-40059 [https://bugs.python.org/issue?@action=redirect&bpo=40059]: **PEP 680** [https://peps.python.org/pep-0680/], the `tomllib` module. Adds support for parsing TOML.

- bpo-464471 [https://bugs.python.org/issue?@action=redirect&bpo=464471]: `asyncio.timeout()` and `asyncio.timeout_at()` context managers added. Patch by Tin Tvrtković and Andrew Svetlov.

- bpo-46805 [https://bugs.python.org/issue?@action=redirect&bpo=46805]: Added raw datagram socket functions for asyncio: `sock_sendto()`, `sock_recvfrom()` and `sock_recvfrom_into()`.

- bpo-46644 [https://bugs.python.org/issue?@action=redirect&bpo=46644]: No longer require valid typeforms to be callable. This allows `typing.Annotated` to wrap `typing.ParamSpecArgs` and `dataclasses.InitVar`. Patch by Gregory Beauregard.

- bpo-46581 [https://bugs.python.org/issue?@action=redirect&bpo=46581]: Brings **`ParamSpec`** propagation for **`GenericAlias`** in line with **`Concatenate`** (and others).

- bpo-45413 [https://bugs.python.org/issue?@action=redirect&bpo=45413]: Define *posix_venv* and *nt_venv* sysconfig installation schemes to be used for bootstrapping new virtual environments. Add *venv* sysconfig installation scheme to get the appropriate one of the above. The schemes are identical to the pre-existing *posix_prefix* and *nt* install schemes. The **`venv`** module now uses the *venv* scheme to create new virtual environments instead of hardcoding the paths depending only on the platform. Downstream Python distributors customizing the *posix_prefix* or *nt* install scheme in a way that is not compatible with the install scheme used in virtual environments are encouraged not to customize the *venv* schemes. When Python itself runs in a virtual environment, **`sysconfig.get_default_scheme()`** and **`sysconfig.get_preferred_scheme()`** with `key="prefix"` returns *venv*.

- bpo-43224 [https://bugs.python.org/issue?@action=redirect&bpo=43224]: Implement support for PEP 646 in typing.py.

- bpo-43224 [https://bugs.python.org/issue?@action=redirect&bpo=43224]: Allow unpacking types.GenericAlias objects, e.g. `*tuple[int, str]`.

- bpo-46557 [https://bugs.python.org/issue?@action=redirect&bpo=46557]: Warnings captured by the logging module are now logged without a format string to prevent systems that group logs by the msg argument from grouping captured warnings together.

- bpo-41370 [https://bugs.python.org/issue?@action=redirect&bpo=41370]: **`typing.get_type_hints()`** now supports evaluating strings as forward references in PEP 585 generic aliases.

- bpo-46607 [https://bugs.python.org/issue?@action=redirect&bpo=46607]: Add **`DeprecationWarning`** to **`LegacyInterpolation`**, deprecated in the docstring since Python 3.2. Will be removed in Python 3.13. Use **`BasicInterpolation`** or **`ExtendedInterpolation`** instead.

- bpo-26120 [https://bugs.python.org/issue?@action=redirect&bpo=26120]: **`pydoc`** now excludes __future__ imports from the module's data items.

- bpo-46480 [https://bugs.python.org/issue?@action=redirect&bpo=46480]: Add **`typing.assert_type()`**. Patch by Jelle Zijlstra.

- bpo-46421 [https://bugs.python.org/issue?@action=redirect&bpo=46421]: Fix a unittest issue where if the command was invoked as `python -m unittest` and the filename(s) began with a dot (.), a `ValueError` is returned.

- bpo-46245 [https://bugs.python.org/issue?@action=redirect&bpo=46245]: Add optional parameter *dir_fd* in **`shutil.rmtree()`**.

- bpo-22859 [https://bugs.python.org/issue?@action=redirect&bpo=22859]: **`unittest.TestProgram.usageExit()`** is marked as deprecated, to be removed in Python 3.13.

- bpo-46170 [https://bugs.python.org/issue?@action=redirect&bpo=46170]: Improve the error message when you try to subclass an instance of **`typing.NewType`**.

- bpo-40296 [https://bugs.python.org/issue?@action=redirect&bpo=40296]: Fix supporting generic aliases in **`pydoc`**.

- bpo-20392 [https://bugs.python.org/issue?@action=redirect&bpo=20392]: Fix inconsistency with uppercase file extensions in **`MimeTypes.guess_type()`**. Patch by Kumar Aditya.

- bpo-46030 [https://bugs.python.org/issue?@action=redirect&bpo=46030]: Add `LOCAL_CREDS`, `LOCAL_CREDS_PERSISTENT` and `SCM_CREDS2` FreeBSD constants to the socket module.

- bpo-44439 [https://bugs.python.org/issue?@action=redirect&bpo=44439]: Fix `.write()` method of a member file in `ZipFile`, when the input data is an object that supports the buffer protocol, the file length may be wrong.

- bpo-45171 [https://bugs.python.org/issue?@action=redirect&bpo=45171]: Fix handling of the `stacklevel` argument to logging functions in the `logging` module so that it is consistent across all logging functions and, as advertised, similar to the `stacklevel` argument used in `warn()`.

- bpo-24959 [https://bugs.python.org/issue?@action=redirect&bpo=24959]: Fix bug where `unittest` sometimes drops frames from tracebacks of exceptions raised in tests.

- bpo-44859 [https://bugs.python.org/issue?@action=redirect&bpo=44859]: Raise more accurate and **PEP 249** [https://peps.python.org/pep-0249/] compatible exceptions in `sqlite3`.

    - Raise `InterfaceError` instead of `ProgrammingError` for `SQLITE_MISUSE` errors.
    - Don't overwrite `BufferError` with `ValueError` when conversion to BLOB fails.
    - Raise `ProgrammingError` instead of `Warning` if user tries to `execute()` more than one SQL statement.
    - Raise `ProgrammingError` instead of `ValueError` if an SQL query contains null characters.

- bpo-44493 [https://bugs.python.org/issue?@action=redirect&bpo=44493]: Add missing terminated NUL in sockaddr_un's length

  This was potentially observable when using non-abstract AF_UNIX datagram sockets to processes written in another programming language.

- bpo-41930 [https://bugs.python.org/issue?@action=redirect&bpo=41930]: Add `serialize()` and `deserialize()` support to `sqlite3`. Patch by Erlend E. Aasland.

- bpo-33178 [https://bugs.python.org/issue?@action=redirect&bpo=33178]: Added `ctypes.BigEndianUnion` and `ctypes.LittleEndianUnion` classes, as originally documented in the library docs but not yet implemented.

- bpo-43352 [https://bugs.python.org/issue?@action=redirect&bpo=43352]: Add an Barrier object in synchronization primitives of *asyncio* Lib in order to be consistent with Barrier from *threading* and *multiprocessing* libs*

- bpo-35859 [https://bugs.python.org/issue?@action=redirect&bpo=35859]: `re` module, fix a few bugs about capturing group. In rare cases, capturing group gets an incorrect string. Patch by Ma Lin.

Documentação

- bpo-45099 [https://bugs.python.org/issue?@action=redirect&bpo=45099]: Document internal `asyncio` API.
- bpo-47126 [https://bugs.python.org/issue?@action=redirect&bpo=47126]: Update PEP URLs to **PEP 676** [https://peps.python.org/pep-0676/]'s new canonical form.
- bpo-47040 [https://bugs.python.org/issue?@action=redirect&bpo=47040]: Clarified the old Python versions compatibility note of `binascii.crc32()` / `zlib.adler32()` / `zlib.crc32()` functions.
- bpo-46033 [https://bugs.python.org/issue?@action=redirect&bpo=46033]: Clarify `for` statement execution in its doc.
- bpo-45790 [https://bugs.python.org/issue?@action=redirect&bpo=45790]: Adjust inaccurate phrasing in Definindo Tipos de Extensão: Tutorial about the `ob_base` field and the macros used to access its contents.
- bpo-42340 [https://bugs.python.org/issue?@action=redirect&bpo=42340]: Document that in some circumstances `KeyboardInterrupt` may cause the code to enter an inconsistent state. Provided a sample workaround to avoid it if needed.
- bpo-41233 [https://bugs.python.org/issue?@action=redirect&bpo=41233]: Link the errnos referenced in `Doc/library/exceptions.rst` to their respective section in `Doc/library/errno.rst`, and vice versa. Previously this was only done for EINTR and InterruptedError. Patch by Yan "yyyyyyyan" Orestes.

Testes

- bpo-47205 [https://bugs.python.org/issue?@action=redirect&bpo=47205]: Skip test for `sched_getaffinity()` and `sched_setaffinity()` error case on

FreeBSD.

- bpo-46126 [https://bugs.python.org/issue?@action=redirect&bpo=46126]: Restore ‘descriptions’ when running tests internally.
- bpo-47104 [https://bugs.python.org/issue?@action=redirect&bpo=47104]: Rewrite **`asyncio.to_thread()`** tests to use **`unittest.IsolatedAsyncioTestCase`**.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: The test suite is now passing on the Emscripten platform. All fork, socket, and subprocess-based tests are skipped.
- bpo-47037 [https://bugs.python.org/issue?@action=redirect&bpo=47037]: Skip `strftime("%4Y")` feature test on Windows. It can cause an assertion error in debug builds.
- bpo-46587 [https://bugs.python.org/issue?@action=redirect&bpo=46587]: Skip tests if platform’s `strftime` does not support non-portable glibc extensions.
- bpo-47015 [https://bugs.python.org/issue?@action=redirect&bpo=47015]: A test case for **`os.sendfile()`** is converted from deprecated **`asyncore`** (see **PEP 594** [https://peps.python.org/pep-0594/]) to **`asyncio`**. Patch by Oleg Iarygin.

Construção

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Add configure option **`--enable-wasm-dynamic-linking`** to enable `dlopen` and MAIN_MODULE / SIDE_MODULE on `wasm32-emscripten`.
- bpo-46023 [https://bugs.python.org/issue?@action=redirect&bpo=46023]: `makesetup` now detects and skips all duplicated module definitions. The first entry wins.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Add SOABI `wasm32-emscripten` for Emscripten and `wasm32-wasi` for WASI on 32bit WASM as well as `wasm64` counter parts.
- bpo-47032 [https://bugs.python.org/issue?@action=redirect&bpo=47032]: Ensure Windows install builds fail correctly with a non-zero exit code when part of the build fails.
- bpo-47024 [https://bugs.python.org/issue?@action=redirect&bpo=47024]: Update OpenSSL to 1.1.1n for macOS installers and all Windows builds.
- bpo-46996 [https://bugs.python.org/issue?@action=redirect&bpo=46996]: The **`tkinter`** package now requires Tcl/Tk version 8.5.12 or newer.

- bpo-46973 [https://bugs.python.org/issue?@action=redirect&bpo=46973]: Add `regen-configure` make target to regenerate configure script with Christian's container image `quay.io/tiran/cpython_autoconf:269`.
- bpo-46917 [https://bugs.python.org/issue?@action=redirect&bpo=46917]: Building Python now requires support of IEEE 754 floating-point numbers. Patch by Victor Stinner.
- bpo-45774 [https://bugs.python.org/issue?@action=redirect&bpo=45774]: `configure` now verifies that all SQLite C APIs needed for the **`sqlite3`** extension module are found.

Windows

- bpo-47194 [https://bugs.python.org/issue?@action=redirect&bpo=47194]: Update `zlib` to v1.2.12 to resolve **CVE 2018-25032** [https://www.cve.org/CVERecord?id=CVE-2018-25032].
- bpo-47171 [https://bugs.python.org/issue?@action=redirect&bpo=47171]: Enables installing the `py.exe` launcher on Windows ARM64.
- bpo-46566 [https://bugs.python.org/issue?@action=redirect&bpo=46566]: Upgraded Gerenciador de instalação do Python to support a new `-V:company/tag` argument for full **PEP 514** [https://peps.python.org/pep-0514/] support and to detect ARM64 installs. The `-64` suffix on arguments is deprecated, but still selects any non-32-bit install. Setting **`PYLAUNCHER_ALLOW_INSTALL`** and specifying a version that is not installed will attempt to install the requested version from the Microsoft Store.
- bpo-47086 [https://bugs.python.org/issue?@action=redirect&bpo=47086]: The installer for Windows now includes documentation as loose HTML files rather than a single compiled `.chm` file.
- bpo-46907 [https://bugs.python.org/issue?@action=redirect&bpo=46907]: Update Windows installer to use SQLite 3.38.1.
- bpo-44549 [https://bugs.python.org/issue?@action=redirect&bpo=44549]: Update bzip2 to 1.0.8 in Windows builds to mitigate **CVE 2016-3189** [https://www.cve.org/CVERecord?id=CVE-2016-3189] and **CVE 2019-12900** [https://www.cve.org/CVERecord?id=CVE-2019-12900].
- bpo-46948 [https://bugs.python.org/issue?@action=redirect&bpo=46948]: Prevent **CVE 2022-26488** [https://www.cve.org/CVERecord?id=CVE-2022-26488] by

ensuring the Add to PATH option in the Windows installer uses the correct path when being repaired.

macOS

- bpo-46890 [https://bugs.python.org/issue?@action=redirect&bpo=46890]: Fix a regression in the setting of `sys._base_executable` in framework builds, and thereby fix a regression in **venv** virtual environments with such builds.
- bpo-46907 [https://bugs.python.org/issue?@action=redirect&bpo=46907]: Update macOS installer to SQLite 3.38.1.

Ferramentas/Demos

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Replace Emscripten's limited shell with Katie Bell's browser-ui REPL from python-wasm project.

API C

- bpo-40421 [https://bugs.python.org/issue?@action=redirect&bpo=40421]: Add `PyFrame_GetBuiltins`, `PyFrame_GetGenerator` and `PyFrame_GetGlobals` C-API functions to access frame object attributes safely from C code.

- bpo-46850 [https://bugs.python.org/issue?@action=redirect&bpo=46850]: Move the private `_PyFrameEvalFunction` type, and private `_PyInterpreterState_GetEvalFrameFunc()` and `_PyInterpreterState_SetEvalFrameFunc()` functions to the internal C API. The `_PyFrameEvalFunction` callback function type now uses the `_PyInterpreterFrame` type which is part of the internal C API. Patch by Victor Stinner.

- bpo-46850 [https://bugs.python.org/issue?@action=redirect&bpo=46850]: Move the private undocumented `_PyEval_EvalFrameDefault()` function to the internal C API. The function now uses the `_PyInterpreterFrame` type which is part of the internal C API. Patch by Victor Stinner.

- bpo-46850 [https://bugs.python.org/issue?@action=redirect&bpo=46850]: Remove the private undocumented function `_PyEval_CallTracing()` from the C API. Call the public **`sys.call_tracing()`** function instead. Patch by Victor Stinner.

- bpo-46850 [https://bugs.python.org/issue?@action=redirect&bpo=46850]: Remove the private undocumented function `_PyEval_GetCoroutineOriginTrackingDepth()` from the C API. Call the public **`sys.get_coroutine_origin_tracking_depth()`** function instead. Patch by Victor Stinner.

- bpo-46850 [https://bugs.python.org/issue?@action=redirect&bpo=46850]: Remove the following private undocumented functions from the C API:

  - `_PyEval_GetAsyncGenFirstiter()`
  - `_PyEval_GetAsyncGenFinalizer()`
  - `_PyEval_SetAsyncGenFirstiter()`
  - `_PyEval_SetAsyncGenFinalizer()`

  Call the public **`sys.get_asyncgen_hooks()`** and **`sys.set_asyncgen_hooks()`** functions instead. Patch by Victor Stinner.

- bpo-46987 [https://bugs.python.org/issue?@action=redirect&bpo=46987]: Remove private functions `_PySys_GetObjectId()` and `_PySys_SetObjectId()`. Patch by Donghee Na.

- bpo-46906 [https://bugs.python.org/issue?@action=redirect&bpo=46906]: Add new functions to pack and unpack C double (serialize and deserialize): **`PyFloat_Pack2()`**, **`PyFloat_Pack4()`**, **`PyFloat_Pack8()`**, **`PyFloat_Unpack2()`**, **`PyFloat_Unpack4()`** and **`PyFloat_Unpack8()`**. Patch by Victor Stinner.

Python 3.11.0 alpha 6

Data de lançamento: 2022-03-07

Núcleo e embutidos

- bpo-46940 [https://bugs.python.org/issue?@action=redirect&bpo=46940]: Avoid overriding `AttributeError` metadata information for nested attribute access calls. Patch by Pablo Galindo.
- bpo-46927 [https://bugs.python.org/issue?@action=redirect&bpo=46927]: Include the type's name in the error message for subscripting non-generic types.
- bpo-46921 [https://bugs.python.org/issue?@action=redirect&bpo=46921]: Support vectorcall for `super()`. Patch by Ken Jin.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Fix incorrect handling of inline cache entries when specializing `BINARY_OP`.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use an oparg to simplify the construction of helpful error messages in `GET_AWAITABLE`.
- bpo-46903 [https://bugs.python.org/issue?@action=redirect&bpo=46903]: Make sure that str subclasses can be used as attribute names for instances with virtual dictionaries. Fixes regression in 3.11alpha
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Add more detailed specialization failure stats for `COMPARE_OP` followed by `EXTENDED_ARG`.
- bpo-46891 [https://bugs.python.org/issue?@action=redirect&bpo=46891]: Fix bug introduced during 3.11alpha where subclasses of `types.ModuleType` with `__slots__` were not initialized correctly, resulting in an interpreter crash.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use inline caching for `LOAD_ATTR`, `LOAD_METHOD`, and `STORE_ATTR`.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use inline cache for `BINARY_SUBSCR`.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use inline caching for `COMPARE_OP`.
- bpo-46864 [https://bugs.python.org/issue?@action=redirect&bpo=46864]: Deprecate `PyBytesObject.ob_shash`. It will be removed in Python 3.13.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use inline caching for `UNPACK_SEQUENCE`.
- bpo-46845 [https://bugs.python.org/issue?@action=redirect&bpo=46845]: Reduces dict size by removing hash value from hash table when all inserted

keys are Unicode. For example, `sys.getsizeof(dict.fromkeys("abcdefg"))` becomes 272 bytes from 352 bytes on 64bit platform.

- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Use inline cache for **`LOAD_GLOBAL`**.
- bpo-46852 [https://bugs.python.org/issue?@action=redirect&bpo=46852]: Rename the private undocumented `float.__set_format__()` method to `float.__setformat__()` to fix a typo introduced in Python 3.7. The method is only used by test_float. Patch by Victor Stinner.
- bpo-46852 [https://bugs.python.org/issue?@action=redirect&bpo=46852]: Remove the undocumented private `float.__set_format__()` method, previously known as `float.__setformat__()` in Python 3.7. Its docstring said: “You probably don’t want to use this function. It exists mainly to be used in Python’s test suite.” Patch by Victor Stinner.
- bpo-40116 [https://bugs.python.org/issue?@action=redirect&bpo=40116]: Fix regression that dict.update(other) may don’t respect iterate order of other when other is key sharing dict.
- bpo-46712 [https://bugs.python.org/issue?@action=redirect&bpo=46712]: Share global string identifiers in deep-frozen modules.
- bpo-46430 [https://bugs.python.org/issue?@action=redirect&bpo=46430]: Fix memory leak in interned strings of deep-frozen modules.
- bpo-46841 [https://bugs.python.org/issue?@action=redirect&bpo=46841]: Store **`BINARY_OP`** caches inline using a new **`CACHE`** instruction.
- bpo-45107 [https://bugs.python.org/issue?@action=redirect&bpo=45107]: Specialize `LOAD_METHOD` for instances with a dict.
- bpo-44337 [https://bugs.python.org/issue?@action=redirect&bpo=44337]: Reduce the memory usage of specialized **`LOAD_ATTR`** and **`STORE_ATTR`** instructions.
- bpo-46729 [https://bugs.python.org/issue?@action=redirect&bpo=46729]: Add number of sub-exceptions to **`BaseException.__str__()`**.
- bpo-45885 [https://bugs.python.org/issue?@action=redirect&bpo=45885]: Don’t un-adapt **`COMPARE_OP`** when collecting specialization stats.
- bpo-46329 [https://bugs.python.org/issue?@action=redirect&bpo=46329]: Fix specialization stats gathering for **`PRECALL`** instructions.

Biblioteca

- bpo-46794 [https://bugs.python.org/issue?@action=redirect&bpo=46794]: Bump up the libexpat version into 2.4.6

Núcleo e embutidos

- bpo-46823 [https://bugs.python.org/issue?@action=redirect&bpo=46823]: Implement a specialized combined opcode `LOAD_FAST__LOAD_ATTR_INSTANCE_VALUE`. Patch by Dennis Sweeney.

- bpo-46820 [https://bugs.python.org/issue?@action=redirect&bpo=46820]: Fix parsing a numeric literal immediately (without spaces) followed by “not in” keywords, like in `1not in x`. Now the parser only emits a warning, not a syntax error.

- bpo-46329 [https://bugs.python.org/issue?@action=redirect&bpo=46329]: Move `KW_NAMES` before `PRECALL` instruction in call sequence. Change `operand` of `CALL` to match `PRECALL` for easier specialization.

- bpo-46808 [https://bugs.python.org/issue?@action=redirect&bpo=46808]: Remove the `NEXT_BLOCK` macro from compile.c, and make the compiler automatically generate implicit blocks when they are needed.

- bpo-46329 [https://bugs.python.org/issue?@action=redirect&bpo=46329]: Add `PUSH_NULL` instruction. This is used as a prefix when evaluating a callable, so that the stack has the same shape for methods and other calls. `PRECALL_FUNCTION` and `PRECALL_METHOD` are merged into a single `PRECALL` instruction.

  There is no change in semantics.

- bpo-46762 [https://bugs.python.org/issue?@action=redirect&bpo=46762]: Fix an assert failure in debug builds when a ‘<’, ‘>’, or ‘=’ is the last character in an f-string that’s missing a closing right brace.

- bpo-46730 [https://bugs.python.org/issue?@action=redirect&bpo=46730]: Message of AttributeError caused by getting, setting or deleting a property without the corresponding function now mentions that the attribute is in fact a property and also specifies type of the class that it belongs to.

- bpo-46724 [https://bugs.python.org/issue?@action=redirect&bpo=46724]: Make sure that all backwards jumps use the `JUMP_ABSOLUTE` instruction, rather than `JUMP_FORWARD` with an argument of `(2**32)+offset`.

- bpo-46732 [https://bugs.python.org/issue?@action=redirect&bpo=46732]: Correct the docstring for the **`__bool__()`** method. Patch by Jelle Zijlstra.

- bpo-46072 [https://bugs.python.org/issue?@action=redirect&bpo=46072]: Add more detailed specialization failure statistics for **`BINARY_OP`**.

- bpo-46707 [https://bugs.python.org/issue?@action=redirect&bpo=46707]: Avoid potential exponential backtracking when producing some syntax errors involving lots of brackets. Patch by Pablo Galindo.

Biblioteca

- bpo-46323 [https://bugs.python.org/issue?@action=redirect&bpo=46323]: **`ctypes`** now allocates memory on the stack instead of on the heap to pass arguments while calling a Python callback function. Patch by Donghee Na.

Núcleo e embutidos

- bpo-45923 [https://bugs.python.org/issue?@action=redirect&bpo=45923]: Add a quickened form of **`RESUME`** that skips quickening checks.
- bpo-46702 [https://bugs.python.org/issue?@action=redirect&bpo=46702]: Specialize **`UNPACK_SEQUENCE`** for **`tuple`** and **`list`** unpackings.
- bpo-46072 [https://bugs.python.org/issue?@action=redirect&bpo=46072]: Opcode pair stats are now gathered with `--enable-pystats`. Defining `DYNAMIC_EXECUTION_PROFILE` or `DXPAIRS` no longer has any effect.
- bpo-46675 [https://bugs.python.org/issue?@action=redirect&bpo=46675]: Allow more than 16 items in a split dict before it is combined. The limit is now 254.

Biblioteca

- bpo-40479 [https://bugs.python.org/issue?@action=redirect&bpo=40479]: Add a missing call to `va_end()` in `Modules/_hashopenssl.c`.
- bpo-46323 [https://bugs.python.org/issue?@action=redirect&bpo=46323]: Use **`PyObject_Vectorcall()`** while calling ctypes callback function. Patch by Donghee Na.

Núcleo e embutidos

- bpo-46615 [https://bugs.python.org/issue?@action=redirect&bpo=46615]: When iterating over sets internally in `setobject.c`, acquire strong references to the resulting items from the set. This prevents crashes in corner-cases of various set operations where the set gets mutated.
- bpo-45828 [https://bugs.python.org/issue?@action=redirect&bpo=45828]: The bytecode compiler now attempts to apply runtime stack manipulations at compile-time (whenever it is feasible to do so).
- bpo-30496 [https://bugs.python.org/issue?@action=redirect&bpo=30496]: Fixed a minor portability issue in the implementation of **`PyLong_FromLong()`**, and added a fast path for single-digit integers to **`PyLong_FromLongLong()`**.

Biblioteca

- bpo-25707 [https://bugs.python.org/issue?@action=redirect&bpo=25707]: Fixed a file leak in **`xml.etree.ElementTree.iterparse()`** when the iterator is not exhausted. Patch by Jacob Walls.

- bpo-46877 [https://bugs.python.org/issue?@action=redirect&bpo=46877]: Export **`unittest.doModuleCleanups()`** in **`unittest`**. Patch by Kumar Aditya.

- bpo-46848 [https://bugs.python.org/issue?@action=redirect&bpo=46848]: For performance, use the optimized string-searching implementations from **`find()`** and **`rfind()`** for **`find()`** and **`rfind()`**.

- bpo-46736 [https://bugs.python.org/issue?@action=redirect&bpo=46736]: **`SimpleHTTPRequestHandler`** now uses HTML5 grammar. Patch by Donghee Na.

- bpo-44886 [https://bugs.python.org/issue?@action=redirect&bpo=44886]: Inherit asyncio proactor datagram transport from `asyncio.DatagramTransport`.

- bpo-46827 [https://bugs.python.org/issue?@action=redirect&bpo=46827]: Support UDP sockets in `asyncio.loop.sock_connect()` for selector-based event loops. Patch by Thomas Grainger.

- bpo-46811 [https://bugs.python.org/issue?@action=redirect&bpo=46811]: Make test suite support Expat >=2.4.5

- bpo-46252 [https://bugs.python.org/issue?@action=redirect&bpo=46252]: Raise `TypeError` if `ssl.SSLSocket` is passed to transport-based APIs.

- bpo-46784 [https://bugs.python.org/issue?@action=redirect&bpo=46784]: Fix libexpat symbols collisions with user dynamically loaded or statically linked libexpat in embedded Python.

- bpo-46786 [https://bugs.python.org/issue?@action=redirect&bpo=46786]: The HTML serialisation in xml.etree.ElementTree now writes `embed`, `source`, `track` and `wbr` as empty tags, as defined in HTML 5.

- bpo-39327 [https://bugs.python.org/issue?@action=redirect&bpo=39327]: `shutil.rmtree()` can now work with VirtualBox shared folders when running from the guest operating-system.

- bpo-45390 [https://bugs.python.org/issue?@action=redirect&bpo=45390]: Propagate `asyncio.CancelledError` message from inner task to outer awaiter.

- bpo-46756 [https://bugs.python.org/issue?@action=redirect&bpo=46756]: Fix a bug in `urllib.request.HTTPPasswordMgr.find_user_password()` and `urllib.request.HTTPPasswordMgrWithPriorAuth.is_authenticated()` which allowed to bypass authorization. For example, access to URI `example.org/foobar` was allowed if the user was authorized for URI `example.org/foo`.

- bpo-46737 [https://bugs.python.org/issue?@action=redirect&bpo=46737]: `random.gauss()` and `random.normalvariate()` now have default

arguments.

- bpo-46752 [https://bugs.python.org/issue?@action=redirect&bpo=46752]: Add task groups to asyncio (structured concurrency, inspired by Trio's nurseries). This also introduces a change to task cancellation, where a cancelled task can't be cancelled again until it calls .uncancel().

- bpo-46724 [https://bugs.python.org/issue?@action=redirect&bpo=46724]: Fix **`dis`** behavior on negative jump offsets.

- bpo-46333 [https://bugs.python.org/issue?@action=redirect&bpo=46333]: The **`__repr__()`** method of **`typing.ForwardRef`** now includes the `module` parameter of **`typing.ForwardRef`** when it is set.

- bpo-46643 [https://bugs.python.org/issue?@action=redirect&bpo=46643]: In **`typing.get_type_hints()`**, support evaluating stringified `ParamSpecArgs` and `ParamSpecKwargs` annotations. Patch by Gregory Beauregard.

- bpo-45863 [https://bugs.python.org/issue?@action=redirect&bpo=45863]: When the **`tarfile`** module creates a pax format archive, it will put an integer representation of timestamps in the ustar header (if possible) for the benefit of older unarchivers, in addition to the existing full-precision timestamps in the pax extended header.

- bpo-46066 [https://bugs.python.org/issue?@action=redirect&bpo=46066]: Deprecate kwargs-based syntax for **`typing.TypedDict`** definitions. It had confusing semantics when specifying totality, and was largely unused. Patch by Jingchen Ye.

- bpo-46676 [https://bugs.python.org/issue?@action=redirect&bpo=46676]: Make **`typing.ParamSpec`** args and kwargs equal to themselves. Patch by Gregory Beauregard.

- bpo-46323 [https://bugs.python.org/issue?@action=redirect&bpo=46323]: `ctypes.CFUNCTYPE()` and `ctypes.WINFUNCTYPE()` now fail to create the type if its `_argtypes_` member contains too many arguments.

Previously, the error was only raised when calling a function. Patch by Victor Stinner.

- bpo-46672 [https://bugs.python.org/issue?@action=redirect&bpo=46672]: Fix `NameError` in `asyncio.gather()` when initial type check fails.

- bpo-46659 [https://bugs.python.org/issue?@action=redirect&bpo=46659]: The `calendar.LocaleTextCalendar` and `calendar.LocaleHTMLCalendar` classes now use `locale.getlocale()`, instead of using `locale.getdefaultlocale()`, if no locale is specified. Patch by Victor Stinner.

- bpo-46659 [https://bugs.python.org/issue?@action=redirect&bpo=46659]: The `locale.getdefaultlocale()` function is deprecated and will be removed in Python 3.13. Use `locale.setlocale()`, `locale.getpreferredencoding(False)` and `locale.getlocale()` functions instead. Patch by Victor Stinner.

- bpo-46655 [https://bugs.python.org/issue?@action=redirect&bpo=46655]: In `typing.get_type_hints()`, support evaluating bare stringified `TypeAlias` annotations. Patch by Gregory Beauregard.

- bpo-45948 [https://bugs.python.org/issue?@action=redirect&bpo=45948]: Fixed a discrepancy in the C implementation of the `xml.etree.ElementTree` module. Now, instantiating an `xml.etree.ElementTree.XMLParser` with a `target=None` keyword provides a default `xml.etree.ElementTree.TreeBuilder` target as the Python implementation does.

- bpo-46626 [https://bugs.python.org/issue?@action=redirect&bpo=46626]: Expose Linux's `IP_BIND_ADDRESS_NO_PORT` option in `socket`.

- bpo-46521 [https://bugs.python.org/issue?@action=redirect&bpo=46521]: Fix a bug in the `codeop` module that was incorrectly identifying invalid code involving string quotes as valid code.

- bpo-46571 [https://bugs.python.org/issue?@action=redirect&bpo=46571]: Improve `typing.no_type_check()`.

Now it does not modify external classes and functions. We also now correctly mark classmethods as not to be type checked.

- bpo-46400 [https://bugs.python.org/issue?@action=redirect&bpo=46400]: expat: Update libexpat from 2.4.1 to 2.4.4

- bpo-46556 [https://bugs.python.org/issue?@action=redirect&bpo=46556]: Deprecate undocumented support for using a **`pathlib.Path`** object as a context manager.

- bpo-46534 [https://bugs.python.org/issue?@action=redirect&bpo=46534]: Implement **PEP 673** [https://peps.python.org/pep-0673/] **`typing.Self`**. Patch by James Hilton-Balfe.

- bpo-46522 [https://bugs.python.org/issue?@action=redirect&bpo=46522]: Make various module `__getattr__` AttributeErrors more closely match a typical AttributeError

- bpo-46475 [https://bugs.python.org/issue?@action=redirect&bpo=46475]: Add **`typing.Never`** and **`typing.assert_never()`**. Patch by Jelle Zijlstra.

- bpo-46333 [https://bugs.python.org/issue?@action=redirect&bpo=46333]: The **`__eq__()`** and **`__hash__()`** methods of **`typing.ForwardRef`** now honor the `module` parameter of **`typing.ForwardRef`**. Forward references from different modules are now differentiated.

- bpo-46246 [https://bugs.python.org/issue?@action=redirect&bpo=46246]: Add missing `__slots__` to `importlib.metadata.DeprecatedList`. Patch by Arie Bovenberg.

- bpo-46232 [https://bugs.python.org/issue?@action=redirect&bpo=46232]: The **`ssl`** module now handles certificates with bit strings in DN correctly.

- bpo-46195 [https://bugs.python.org/issue?@action=redirect&bpo=46195]: **`typing.get_type_hints()`** no longer adds `Optional` to parameters with `None` as a default. This aligns to changes to PEP 484 in https://github.com/python/peps/pull/689

- bpo-31369 [https://bugs.python.org/issue?@action=redirect&bpo=31369]: Add **`RegexFlag`** to `re.__all__` and documented it. Add **`NOFLAG`** to indicate no flags being set.

- bpo-45898 [https://bugs.python.org/issue?@action=redirect&bpo=45898]: **`ctypes`** no longer defines `ffi_type_*` symbols in `cfield.c`. The symbols have been provided by libffi for over a decade.

- bpo-44953 [https://bugs.python.org/issue?@action=redirect&bpo=44953]: Calling `operator.itemgetter` objects and `operator.attrgetter` objects is now faster due to use of the vectorcall calling convention.

- bpo-44289 [https://bugs.python.org/issue?@action=redirect&bpo=44289]: Fix an issue with **`is_tarfile()`** method when using *fileobj* argument: position in the *fileobj* was advanced forward which made it unreadable with **`tarfile.TarFile.open()`**.

- bpo-44011 [https://bugs.python.org/issue?@action=redirect&bpo=44011]: Reimplement SSL/TLS support in asyncio, borrow the implementation from uvloop library.

- bpo-41086 [https://bugs.python.org/issue?@action=redirect&bpo=41086]: Make the **`configparser.ConfigParser`** constructor raise **`TypeError`** if the `interpolation` parameter is not of type **`configparser.Interpolation`**

- bpo-29418 [https://bugs.python.org/issue?@action=redirect&bpo=29418]: Implement **`inspect.ismethodwrapper()`** and fix **`inspect.isroutine()`** for cases where methodwrapper is given. Patch by Hakan Çelik.

- bpo-14156 [https://bugs.python.org/issue?@action=redirect&bpo=14156]: argparse.FileType now supports an argument of ‘-’ in binary mode, returning the .buffer attribute of sys.stdin/sys.stdout as appropriate. Modes including ‘x’ and ‘a’ are treated equivalently to ‘w’ when argument is ‘-’. Patch contributed by Josh Rosenberg

Documentação

- bpo-42238 [https://bugs.python.org/issue?@action=redirect&bpo=42238]: `Doc/tools/rstlint.py` has moved to its own repository and is now packaged on PyPI as `sphinx-lint`.

Testes

- bpo-46913 [https://bugs.python.org/issue?@action=redirect&bpo=46913]: Fix test_faulthandler.test_sigfpe() if Python is built with undefined behavior sanitizer (UBSAN): disable UBSAN on the faulthandler_sigfpe() function. Patch by Victor Stinner.
- bpo-46760 [https://bugs.python.org/issue?@action=redirect&bpo=46760]: Remove bytecode offsets from expected values in test.test_dis module. Reduces the obstacles to modifying the VM or compiler.
- bpo-46708 [https://bugs.python.org/issue?@action=redirect&bpo=46708]: Prevent default asyncio event loop policy modification warning after `test_asyncio` execution.
- bpo-46678 [https://bugs.python.org/issue?@action=redirect&bpo=46678]: The function `make_legacy_pyc` in `Lib/test/support/import_helper.py` no longer fails when `PYTHONPYCACHEPREFIX` is set to a directory on a different device from where tempfiles are stored.
- bpo-46623 [https://bugs.python.org/issue?@action=redirect&bpo=46623]: Skip test_pair() and test_speech128() of test_zlib on s390x since they fail if zlib uses the s390x hardware accelerator. Patch by Victor Stinner.

Construção

- bpo-46860 [https://bugs.python.org/issue?@action=redirect&bpo=46860]: Respect `--with-suffix` when building on case-insensitive file systems.
- bpo-46656 [https://bugs.python.org/issue?@action=redirect&bpo=46656]: Building Python now requires a C11 compiler. Optional C11 features are not required. Patch by Victor Stinner.
- bpo-46656 [https://bugs.python.org/issue?@action=redirect&bpo=46656]: Building Python now requires support for floating-point Not-a-Number (NaN): remove the `Py_NO_NAN` macro. Patch by Victor Stinner.
- bpo-46640 [https://bugs.python.org/issue?@action=redirect&bpo=46640]: Building Python now requires a C99 `<math.h>` header file providing a `NAN`

constant, or the `__builtin_nan()` built-in function. Patch by Victor Stinner.

- bpo-46608 [https://bugs.python.org/issue?@action=redirect&bpo=46608]: Exclude marshalled-frozen data if deep-freezing to save 300 KB disk space. This includes adding a new `is_package` field to `_frozen`. Patch by Kumar Aditya.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Fix wasm32-emscripten test failures and platform issues. - Disable syscalls that are not supported or don't work, e.g. wait, getrusage, prlimit, mkfifo, mknod, setres[gu]id, setgroups. - Use fd_count to count open fds. - Add more checks for subprocess and fork. - Add workarounds for missing _multiprocessing and failing socket.accept(). - Enable bzip2. - Disable large file support. - Disable signal.alarm.
- bpo-46430 [https://bugs.python.org/issue?@action=redirect&bpo=46430]: Intern strings in deep-frozen modules. Patch by Kumar Aditya.

Windows

- bpo-46744 [https://bugs.python.org/issue?@action=redirect&bpo=46744]: The default all users install directory for ARM64 is now under the native `Program Files` folder, rather than `Program Files (Arm)` which is intended for ARM (32-bit) files.
- bpo-46567 [https://bugs.python.org/issue?@action=redirect&bpo=46567]: Adds Tcl and Tk support for Windows ARM64. This also adds IDLE to the installation.
- bpo-46638 [https://bugs.python.org/issue?@action=redirect&bpo=46638]: Ensures registry virtualization is consistently disabled. For 3.10 and earlier, it remains enabled (some registry writes are protected), while for 3.11 and later it is disabled (registry modifications affect all applications).

IDLE

- bpo-46630 [https://bugs.python.org/issue?@action=redirect&bpo=46630]: Make query dialogs on Windows start with a cursor in the entry box.
- bpo-45447 [https://bugs.python.org/issue?@action=redirect&bpo=45447]: Apply IDLE syntax highlighting to `.pyi` files. Patch by Alex Waygood and Terry Jan Reedy.

API C

- bpo-46748 [https://bugs.python.org/issue?@action=redirect&bpo=46748]: Python's public headers no longer import `<stdbool.h>`, leaving code that embeds/extends Python free to define `bool`, `true` and `false`.
- bpo-46836 [https://bugs.python.org/issue?@action=redirect&bpo=46836]: Move the **`PyFrameObject`** type definition (`struct _frame`) to the internal C API `pycore_frame.h` header file. Patch by Victor Stinner.
- bpo-45459 [https://bugs.python.org/issue?@action=redirect&bpo=45459]: Rename `Include/buffer.h` header file to `Include/pybuffer.h` to avoid conflicts with projects having an existing `buffer.h` header file. Patch by Victor Stinner.
- bpo-45412 [https://bugs.python.org/issue?@action=redirect&bpo=45412]: Remove the `HAVE_PY_SET_53BIT_PRECISION` macro (moved to the internal C API). Patch by Victor Stinner.
- bpo-46613 [https://bugs.python.org/issue?@action=redirect&bpo=46613]: Added function **`PyType_GetModuleByDef()`**, which allows access to module state when a method's defining class is not available.

Python 3.11.0 alpha 5

Data de lançamento: 2022-02-03

Núcleo e embutidos

- bpo-45773 [https://bugs.python.org/issue?@action=redirect&bpo=45773]: Remove two invalid "peephole" optimizations from the bytecode compiler.

- bpo-46564 [https://bugs.python.org/issue?@action=redirect&bpo=46564]: Do not create frame objects when creating **`super`** object. Patch by Kumar Aditya.

- bpo-45885 [https://bugs.python.org/issue?@action=redirect&bpo=45885]: Added more fined-grained specialization failure stats regarding the `COMPARE_OP` bytecode.

- bpo-44977 [https://bugs.python.org/issue?@action=redirect&bpo=44977]: The delegation of `int()` to `__trunc__()` is now deprecated. Calling `int(a)` when `type(a)` implements `__trunc__()` but not `__int__()` or `__index__()` now raises a `DeprecationWarning`.

- bpo-46458 [https://bugs.python.org/issue?@action=redirect&bpo=46458]: Reorder code emitted by the compiler for a `try`-`except` block so that the `else` block's code immediately follows the `try` body (without a jump). This is more optimal for the happy path.

- bpo-46527 [https://bugs.python.org/issue?@action=redirect&bpo=46527]: Allow passing `iterable` as a keyword argument to `enumerate()` again. Patch by Jelle Zijlstra.

- bpo-46528 [https://bugs.python.org/issue?@action=redirect&bpo=46528]: Replace several stack manipulation instructions (`DUP_TOP`, `DUP_TOP_TWO`, `ROT_TWO`, `ROT_THREE`, `ROT_FOUR`, and `ROT_N`) with new `COPY` and `SWAP` instructions.

- bpo-46329 [https://bugs.python.org/issue?@action=redirect&bpo=46329]: Use two or three bytecodes to implement most calls.

  Calls without named arguments are implemented as a sequence of two instructions: `PRECALL; CALL`. Calls with named arguments are implemented as a sequence of three instructions: `PRECALL; KW_NAMES; CALL`. There are two different `PRECALL` instructions: `PRECALL_FUNTION` and `PRECALL_METHOD`. The latter pairs with `LOAD_METHOD`.

  This partition into pre-call and call allows better specialization, and thus better performance ultimately.

  There is no change in semantics.

- bpo-46503 [https://bugs.python.org/issue?@action=redirect&bpo=46503]: Fix an assert when parsing some invalid N escape sequences in f-strings.

- bpo-46431 [https://bugs.python.org/issue?@action=redirect&bpo=46431]: Improve error message on invalid calls to `BaseExceptionGroup.__new__()`.

- bpo-46476 [https://bugs.python.org/issue?@action=redirect&bpo=46476]: Fix memory leak in code objects generated by deepfreeze. Patch by Kumar Aditya.

- bpo-46481 [https://bugs.python.org/issue?@action=redirect&bpo=46481]: Speed up calls to **`weakref.ref.__call__()`** by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

- bpo-46417 [https://bugs.python.org/issue?@action=redirect&bpo=46417]: Fix a race condition on setting a type `__bases__` attribute: the internal function `add_subclass()` now gets the `PyTypeObject.tp_subclasses` member after calling **`PyWeakref_NewRef()`** which can trigger a garbage collection which can indirectly modify `PyTypeObject.tp_subclasses`. Patch by Victor Stinner.

- bpo-46417 [https://bugs.python.org/issue?@action=redirect&bpo=46417]: `python -X showrefcount` now shows the total reference count after clearing and destroyed the main Python interpreter. Previously, it was shown before. Patch by Victor Stinner.

- bpo-43683 [https://bugs.python.org/issue?@action=redirect&bpo=43683]: Add ASYNC_GEN_WRAP opcode to wrap the value to be yielded in async generators. Removes the need to special case async generators in the `YIELD_VALUE` instruction.

- bpo-46407 [https://bugs.python.org/issue?@action=redirect&bpo=46407]: Optimize some modulo operations in `Objects/longobject.c`. Patch by Jeremiah Vivian.

- bpo-46409 [https://bugs.python.org/issue?@action=redirect&bpo=46409]: Add new `RETURN_GENERATOR` bytecode to make generators. Simplifies calling Python functions in the VM, as they no longer any need to special case generator functions.

  Also add `JUMP_NO_INTERRUPT` bytecode that acts like `JUMP_ABSOLUTE`, but does not check for interrupts.

- bpo-46406 [https://bugs.python.org/issue?@action=redirect&bpo=46406]: The integer division `//` implementation has been optimized to better let the compiler understand its constraints. It can be 20% faster on the amd64 platform when dividing an int by a value smaller than `2**30`.

Biblioteca

- bpo-46383 [https://bugs.python.org/issue?@action=redirect&bpo=46383]: Fix invalid signature of `_zoneinfo`'s `module_free` function to resolve a crash on wasm32-emscripten platform.

Núcleo e embutidos

- bpo-46361 [https://bugs.python.org/issue?@action=redirect&bpo=46361]: Ensure that "small" integers created by **`int.from_bytes()`** and **`decimal.Decimal`** are properly cached.
- bpo-46161 [https://bugs.python.org/issue?@action=redirect&bpo=46161]: Fix the class building error when the arguments are constants and CALL_FUNCTION_EX is used.
- bpo-46028 [https://bugs.python.org/issue?@action=redirect&bpo=46028]: Fixes calculation of **`sys._base_executable`** when inside a virtual environment that uses symlinks with different binary names than the base environment provides.
- bpo-46091 [https://bugs.python.org/issue?@action=redirect&bpo=46091]: Correctly calculate indentation levels for lines with whitespace character that are ended by line continuation characters. Patch by Pablo Galindo

Biblioteca

- bpo-30512 [https://bugs.python.org/issue?@action=redirect&bpo=30512]: Add CAN Socket support for NetBSD.

Construção

- bpo-46045 [https://bugs.python.org/issue?@action=redirect&bpo=46045]: Do not use POSIX semaphores on NetBSD

Núcleo e embutidos

- bpo-44024 [https://bugs.python.org/issue?@action=redirect&bpo=44024]: Improve the `TypeError` message for non-string second arguments passed to the built-in functions `getattr()` and `hasattr()`. Patch by Géry Ogam.

Biblioteca

- bpo-46624 [https://bugs.python.org/issue?@action=redirect&bpo=46624]: Restore support for non-integer arguments of `random.randrange()` and `random.randint()`.
- bpo-46591 [https://bugs.python.org/issue?@action=redirect&bpo=46591]: Make the IDLE doc URL on the About IDLE dialog clickable.
- bpo-46565 [https://bugs.python.org/issue?@action=redirect&bpo=46565]: Remove loop variables that are leaking into modules' namespaces.
- bpo-46553 [https://bugs.python.org/issue?@action=redirect&bpo=46553]: In `typing.get_type_hints()`, support evaluating bare stringified `ClassVar` annotations. Patch by Gregory Beauregard.
- bpo-46544 [https://bugs.python.org/issue?@action=redirect&bpo=46544]: Don't leak `x` & `uspace` intermediate vars in `textwrap.TextWrapper`.
- bpo-46487 [https://bugs.python.org/issue?@action=redirect&bpo=46487]: Add the `get_write_buffer_limits` method to **`asyncio.transports.WriteTransport`** and to the SSL transport.
- bpo-45173 [https://bugs.python.org/issue?@action=redirect&bpo=45173]: Note the configparser deprecations will be removed in Python 3.12.
- bpo-45162 [https://bugs.python.org/issue?@action=redirect&bpo=45162]: The deprecated `unittest` APIs removed in 3.11a1 have been temporarily restored to be removed in 3.12 while cleanups in external projects go in.
- bpo-46539 [https://bugs.python.org/issue?@action=redirect&bpo=46539]: In `typing.get_type_hints()`, support evaluating stringified `ClassVar` and `Final` annotations inside `Annotated`. Patch by Gregory Beauregard.
- bpo-46510 [https://bugs.python.org/issue?@action=redirect&bpo=46510]: Add missing test for `types.TracebackType` and `types.FrameType`. Calculate them directly from the caught exception without calling `sys.exc_info()`.

- bpo-46491 [https://bugs.python.org/issue?@action=redirect&bpo=46491]: Allow **`typing.Annotated`** to wrap **`typing.Final`** and **`typing.ClassVar`**. Patch by Gregory Beauregard.
- bpo-46483 [https://bugs.python.org/issue?@action=redirect&bpo=46483]: Remove **`__class_getitem__()`** from **`pathlib.PurePath`** as this class was not supposed to be generic.
- bpo-46436 [https://bugs.python.org/issue?@action=redirect&bpo=46436]: Fix command-line option `-d`/`--directory` in module **`http.server`** which is ignored when combined with command-line option `--cgi`. Patch by Géry Ogam.
- bpo-41403 [https://bugs.python.org/issue?@action=redirect&bpo=41403]: Make **`mock.patch()`** raise a **`TypeError`** with a relevant error message on invalid arg. Previously it allowed a cryptic **`AttributeError`** to escape.
- bpo-46474 [https://bugs.python.org/issue?@action=redirect&bpo=46474]: In `importlib.metadata.EntryPoint.pattern`, avoid potential REDoS by limiting ambiguity in consecutive whitespace.
- bpo-46474 [https://bugs.python.org/issue?@action=redirect&bpo=46474]: Removed private method from `importlib.metadata.Path`. Sync with importlib_metadata 4.10.0.
- bpo-46470 [https://bugs.python.org/issue?@action=redirect&bpo=46470]: Remove unused branch from `typing._remove_dups_flatten`
- bpo-46469 [https://bugs.python.org/issue?@action=redirect&bpo=46469]: **`asyncio`** generic classes now return **`types.GenericAlias`** in `__class_getitem__` instead of the same class.
- bpo-41906 [https://bugs.python.org/issue?@action=redirect&bpo=41906]: Support passing filter instances in the `filters` values of `handlers` and `loggers` in the dictionary passed to **`logging.config.dictConfig()`**.
- bpo-46422 [https://bugs.python.org/issue?@action=redirect&bpo=46422]: Use `dis.Positions` in `dis.Instruction` instead of a regular `tuple`.
- bpo-46434 [https://bugs.python.org/issue?@action=redirect&bpo=46434]: **`pdb`** now gracefully handles `help` when **`__doc__`** is missing, for example when run with pregenerated optimized `.pyc` files.
- bpo-43869 [https://bugs.python.org/issue?@action=redirect&bpo=43869]: Python uses the same time Epoch on all platforms. Add an explicit unit test to ensure that it's the case. Patch by Victor Stinner.
- bpo-46414 [https://bugs.python.org/issue?@action=redirect&bpo=46414]: Add **`typing.reveal_type()`**. Patch by Jelle Zijlstra.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: **`subprocess`** now imports Windows-specific imports when `msvcrt` module is available, and POSIX-specific imports on all other platforms. This gives a clean exception when `_posixsubprocess` is not available (e.g. Emscripten browser target).
- bpo-40066 [https://bugs.python.org/issue?@action=redirect&bpo=40066]: `IntEnum`, `IntFlag`, and `StrEnum` use the mixed-in type for their `str()` and `format()` output.
- bpo-46316 [https://bugs.python.org/issue?@action=redirect&bpo=46316]: Optimize **`pathlib.Path.iterdir()`** by removing an unnecessary check for special entries.
- bpo-29688 [https://bugs.python.org/issue?@action=redirect&bpo=29688]: Document **`pathlib.Path.absolute()`** (which has always existed).
- bpo-43012 [https://bugs.python.org/issue?@action=redirect&bpo=43012]: The pathlib module's obsolete and internal `_Accessor` class has been removed to prepare the terrain for upcoming enhancements to the module.
- bpo-46258 [https://bugs.python.org/issue?@action=redirect&bpo=46258]: Speed up **`math.isqrt()`** for small positive integers by replacing two division steps with a lookup table.
- bpo-46242 [https://bugs.python.org/issue?@action=redirect&bpo=46242]: Improve error message when creating a new **`enum.Enum`** type subclassing an existing `Enum` with `_member_names_` using **`enum.Enum.__call__()`**.
- bpo-43118 [https://bugs.python.org/issue?@action=redirect&bpo=43118]: Fix a bug in **`inspect.signature()`** that was causing it to fail on some subclasses of classes with a `__text_signature__` referencing module globals. Patch by Weipeng Hong.
- bpo-26552 [https://bugs.python.org/issue?@action=redirect&bpo=26552]: Fixed case where failing **`asyncio.ensure_future()`** did not close the coroutine. Patch by Kumar Aditya.
- bpo-21987 [https://bugs.python.org/issue?@action=redirect&bpo=21987]: Fix an issue with **`tarfile.TarFile.getmember()`** getting a directory name with a trailing slash.
- bpo-46124 [https://bugs.python.org/issue?@action=redirect&bpo=46124]: Update **`zoneinfo`** to rely on importlib.resources traversable API.
- bpo-46103 [https://bugs.python.org/issue?@action=redirect&bpo=46103]: Now **`inspect.getmembers()`** only gets **`__bases__`** attribute from class type. Patch by Weipeng Hong.

- bpo-46080 [https://bugs.python.org/issue?@action=redirect&bpo=46080]: Fix exception in argparse help text generation if a **`argparse.BooleanOptionalAction`** argument's default is `argparse.SUPPRESS` and it has `help` specified. Patch by Felix Fontein.
- bpo-44791 [https://bugs.python.org/issue?@action=redirect&bpo=44791]: Fix substitution of **`ParamSpec`** in **`Concatenate`** with different parameter expressions. Substitution with a list of types returns now a tuple of types. Substitution with `Concatenate` returns now a `Concatenate` with concatenated lists of arguments.

Documentação

- bpo-46463 [https://bugs.python.org/issue?@action=redirect&bpo=46463]: Fixes `escape4chm.py` script used when building the CHM documentation file

Testes

- bpo-43478 [https://bugs.python.org/issue?@action=redirect&bpo=43478]: Mocks can no longer be provided as the specs for other Mocks. As a result, an already-mocked object cannot be passed to `mock.Mock()`. This can uncover bugs in tests since these Mock-derived Mocks will always pass certain tests (e.g. isinstance) and builtin assert functions (e.g. assert_called_once_with) will unconditionally pass.
- bpo-46616 [https://bugs.python.org/issue?@action=redirect&bpo=46616]: Ensures `test_importlib.test_windows` cleans up registry keys after completion.
- bpo-44359 [https://bugs.python.org/issue?@action=redirect&bpo=44359]: test_ftplib now silently ignores socket errors to prevent logging unhandled threading exceptions. Patch by Victor Stinner.
- bpo-46600 [https://bugs.python.org/issue?@action=redirect&bpo=46600]: Fix test_gdb.test_pycfunction() for Python built with `clang -Og`. Tolerate inlined functions in the gdb traceback. Patch by Victor Stinner.
- bpo-46542 [https://bugs.python.org/issue?@action=redirect&bpo=46542]: Fix a Python crash in test_lib2to3 when using Python built in debug mode: limit the recursion limit. Patch by Victor Stinner.

- bpo-46576 [https://bugs.python.org/issue?@action=redirect&bpo=46576]: test_peg_generator now disables compiler optimization when testing compilation of its own C extensions to significantly speed up the testing on non-debug builds of CPython.
- bpo-46542 [https://bugs.python.org/issue?@action=redirect&bpo=46542]: Fix `test_json` tests checking for **`RecursionError`**: modify these tests to use `support.infinite_recursion()`. Patch by Victor Stinner.
- bpo-13886 [https://bugs.python.org/issue?@action=redirect&bpo=13886]: Skip test_builtin PTY tests on non-ASCII characters if the readline module is loaded. The readline module changes input() behavior, but test_builtin is not intended to test the readline module. Patch by Victor Stinner.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Add **`test.support.requires_fork()`** decorators to mark tests that require a working **`os.fork()`**.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Add **`test.support.requires_subprocess()`** decorator to mark tests which require working **`subprocess`** module or `os.spawn*`. The wasm32-emscripten platform has no support for processes.
- bpo-46126 [https://bugs.python.org/issue?@action=redirect&bpo=46126]: Disable ‘descriptions’ when running tests internally.

Construção

- bpo-46602 [https://bugs.python.org/issue?@action=redirect&bpo=46602]: Tidied up configure.ac so that conftest.c is truncated rather than appended. This assists in the case where the ‘rm’ of conftest.c fails to happen between tests. Downstream issues such as a clobbered SOABI can result.
- bpo-46600 [https://bugs.python.org/issue?@action=redirect&bpo=46600]: Fix the test checking if the C compiler supports `-Og` option in the `./configure` script to also use `-Og` on clang which supports it. Patch by Victor Stinner.
- bpo-38472 [https://bugs.python.org/issue?@action=redirect&bpo=38472]: Fix GCC detection in setup.py when cross-compiling. The C compiler is now run with LC_ALL=C. Previously, the detection failed with a German locale.

- bpo-46513 [https://bugs.python.org/issue?@action=redirect&bpo=46513]: **configure** no longer uses `AC_C_CHAR_UNSIGNED` macro and `pyconfig.h` no longer defines reserved symbol `__CHAR_UNSIGNED__`.
- bpo-46471 [https://bugs.python.org/issue?@action=redirect&bpo=46471]: Use global singletons for single byte bytes objects in deepfreeze.
- bpo-46443 [https://bugs.python.org/issue?@action=redirect&bpo=46443]: Deepfreeze now uses cached small integers as it saves some space for common small integers.
- bpo-46429 [https://bugs.python.org/issue?@action=redirect&bpo=46429]: Merge all deep-frozen files into one for space savings. Patch by Kumar Aditya.
- bpo-45569 [https://bugs.python.org/issue?@action=redirect&bpo=45569]: The build now defaults to using 30-bit digits for Python integers. Previously either 15-bit or 30-bit digits would be selected, depending on the platform. 15-bit digits may still be selected using the `--enable-big-digits=15` option to the `configure` script, or by defining `PYLONG_BITS_IN_DIGIT` in `pyconfig.h`.
- bpo-45925 [https://bugs.python.org/issue?@action=redirect&bpo=45925]: Update Windows installer to use SQLite 3.37.2.
- bpo-43112 [https://bugs.python.org/issue?@action=redirect&bpo=43112]: Detect musl libc as a separate SOABI (tagged as `linux-musl`).

Windows

- bpo-33125 [https://bugs.python.org/issue?@action=redirect&bpo=33125]: The traditional EXE/MSI based installer for Windows is now available for ARM64
- bpo-46362 [https://bugs.python.org/issue?@action=redirect&bpo=46362]: os.path.abspath("C:CON") is now fixed to return "\.CON", not the same path. The regression was true of all legacy DOS devices such as COM1, LPT1, or NUL.
- bpo-44934 [https://bugs.python.org/issue?@action=redirect&bpo=44934]: The installer now offers a command-line only option to add the installation directory to the end of **`PATH`** instead of at the start.

macOS

- bpo-45925 [https://bugs.python.org/issue?@action=redirect&bpo=45925]: Update macOS installer to SQLite 3.37.2.

IDLE

- bpo-45296 [https://bugs.python.org/issue?@action=redirect&bpo=45296]: Clarify close, quit, and exit in IDLE. In the File menu, ‘Close’ and ‘Exit’ are now ‘Close Window’ (the current one) and ‘Exit’ is now ‘Exit IDLE’ (by closing all windows). In Shell, ‘quit()’ and ‘exit()’ mean ‘close Shell’. If there are no other windows, this also exits IDLE.

API C

- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Remove the `PyHeapType_GET_MEMBERS()` macro. It was exposed in the public C API by mistake, it must only be used by Python internally. Use the `PyTypeObject.tp_members` member instead. Patch by Victor Stinner.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Move _Py_GetAllocatedBlocks() and _PyObject_DebugMallocStats() private functions to the internal C API. Patch by Victor Stinner.
- bpo-46433 [https://bugs.python.org/issue?@action=redirect&bpo=46433]: The internal function _PyType_GetModuleByDef now correctly handles inheritance patterns involving static types.
- bpo-45459 [https://bugs.python.org/issue?@action=redirect&bpo=45459]: **`Py_buffer`** and various `Py_buffer` related functions are now part of the limited API and stable ABI.
- bpo-14916 [https://bugs.python.org/issue?@action=redirect&bpo=14916]: Fixed bug in the tokenizer that prevented `PyRun_InteractiveOne` from parsing from the provided FD.

Python 3.11.0 alpha 4

Data de lançamento: 2022-01-13

Núcleo e embutidos

- bpo-46070 [https://bugs.python.org/issue?@action=redirect&bpo=46070]: **`Py_EndInterpreter()`** now explicitly untracks all objects currently tracked by the GC. Previously, if an object was used later by another interpreter, calling **`PyObject_GC_UnTrack()`** on the object crashed if the previous or the next object of the **`PyGC_Head`** structure became a dangling pointer. Patch by Victor Stinner.
- bpo-46347 [https://bugs.python.org/issue?@action=redirect&bpo=46347]: Fix memory leak in PyEval_EvalCodeEx.
- bpo-46339 [https://bugs.python.org/issue?@action=redirect&bpo=46339]: Fix a crash in the parser when retrieving the error text for multi-line f-strings expressions that do not start in the first line of the string. Patch by Pablo Galindo
- bpo-46331 [https://bugs.python.org/issue?@action=redirect&bpo=46331]: Do not set line number of instruction storing doc-string. Fixes regression introduced in 3.11 alpha.
- bpo-46314 [https://bugs.python.org/issue?@action=redirect&bpo=46314]: Remove spurious “call” event when creating a lambda function that was accidentally introduced in 3.11a4.
- bpo-46289 [https://bugs.python.org/issue?@action=redirect&bpo=46289]: ASDL declaration of `FormattedValue` has changed to reflect `conversion` field is not optional.
- bpo-46297 [https://bugs.python.org/issue?@action=redirect&bpo=46297]: Fixed an interpreter crash on bootup with multiple PythonPaths set in the Windows registry. Patch by Derzsi Dániel.
- bpo-46237 [https://bugs.python.org/issue?@action=redirect&bpo=46237]: Fix the line number of tokenizer errors inside f-strings. Patch by Pablo Galindo.
- bpo-46263 [https://bugs.python.org/issue?@action=redirect&bpo=46263]: We always expect the “use_frozen_modules” config to be set, now that getpath.c was rewritten in pure Python and the logic improved.
- bpo-46006 [https://bugs.python.org/issue?@action=redirect&bpo=46006]: Fix a regression when a type method like `__init__()` is modified in a subinterpreter. Fix a regression in `_PyUnicode_EqualToASCIIId()` and type `update_slot()`. Revert the change which made the Unicode dictionary of interned strings compatible with subinterpreters: the internal interned dictionary is shared again by all interpreters. Patch by Victor Stinner.

- bpo-45923 [https://bugs.python.org/issue?@action=redirect&bpo=45923]: Add RESUME opcode. This is a logical no-op. It is emitted by the compiler anywhere a Python function can be entered. It is used by the interpreter to perform tracing and optimizer checks.

Biblioteca

- bpo-46208 [https://bugs.python.org/issue?@action=redirect&bpo=46208]: Fix the regression of os.path.normpath("A/../../B") not returning expected "../B" but "B".

Núcleo e embutidos

- bpo-46240 [https://bugs.python.org/issue?@action=redirect&bpo=46240]: Correct the error message for unclosed parentheses when the tokenizer doesn't reach the end of the source when the error is reported. Patch by Pablo Galindo
- bpo-46009 [https://bugs.python.org/issue?@action=redirect&bpo=46009]: Remove the `GEN_START` opcode.
- bpo-46235 [https://bugs.python.org/issue?@action=redirect&bpo=46235]: Certain sequence multiplication operations like `[0] * 1_000` are now faster due to reference-counting optimizations. Patch by Dennis Sweeney.
- bpo-46221 [https://bugs.python.org/issue?@action=redirect&bpo=46221]: **`PREP_RERAISE_STAR`** no longer pushes `lasti` to the stack.
- bpo-46202 [https://bugs.python.org/issue?@action=redirect&bpo=46202]: Remove **`POP_EXCEPT_AND_RERAISE`** and replace it by an equivalent sequence of other opcodes.

Biblioteca

- bpo-46085 [https://bugs.python.org/issue?@action=redirect&bpo=46085]: Fix iterator cache mechanism of **`OrderedDict`**.

Núcleo e embutidos

- bpo-46055 [https://bugs.python.org/issue?@action=redirect&bpo=46055]: Speed up shifting operation involving integers less than **`PyLong_BASE`**. Patch by

Xinhang Xu.

- bpo-46110 [https://bugs.python.org/issue?@action=redirect&bpo=46110]: Add a maximum recursion check to the PEG parser to avoid stack overflow. Patch by Pablo Galindo

- bpo-46107 [https://bugs.python.org/issue?@action=redirect&bpo=46107]: Fix bug where **`ExceptionGroup.split()`** and **`ExceptionGroup.subgroup()`** did not copy the exception group's `__note__` field to the parts.

- bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711]: The interpreter state's representation of handled exceptions (a.k.a exc_info, or _PyErr_StackItem) now has only the `exc_value` field, `exc_type` and `exc_traceback` have been removed as their values can be derived from `exc_value`.

- bpo-44525 [https://bugs.python.org/issue?@action=redirect&bpo=44525]: Replace the four call bytecode instructions which one pre-call instruction and two call instructions.

  Removes `CALL_FUNCTION`, `CALL_FUNCTION_KW`, `CALL_METHOD` and `CALL_METHOD_KW`.

  Adds `CALL_NO_KW` and `CALL_KW` call instructions, and `PRECALL_METHOD` prefix for pairing with `LOAD_METHOD`.

- bpo-46039 [https://bugs.python.org/issue?@action=redirect&bpo=46039]: Remove the `YIELD_FROM` instruction and replace it with the `SEND` instruction which performs the same operation, but without the loop.

- bpo-45635 [https://bugs.python.org/issue?@action=redirect&bpo=45635]: The code called from **`_PyErr_Display()`** was refactored to improve error handling. It now exits immediately upon an unrecoverable error.

- bpo-46054 [https://bugs.python.org/issue?@action=redirect&bpo=46054]: Fix parser error when parsing non-utf8 characters in source files. Patch by Pablo Galindo.

- bpo-46042 [https://bugs.python.org/issue?@action=redirect&bpo=46042]: Improve the location of the caret in `SyntaxError` exceptions emitted by the symbol table. Patch by Pablo Galindo.

- bpo-46049 [https://bugs.python.org/issue?@action=redirect&bpo=46049]: Ensure `._pth` files work as intended on platforms other than Windows.

- bpo-46048 [https://bugs.python.org/issue?@action=redirect&bpo=46048]: Fixes parsing of `._pth` files on startup so that single-character paths are correctly read.

- bpo-37971 [https://bugs.python.org/issue?@action=redirect&bpo=37971]: Fix a bug where the line numbers given in a traceback when a decorator application raised an exception were wrong.

- bpo-46031 [https://bugs.python.org/issue?@action=redirect&bpo=46031]: Add `POP_JUMP_IF_NOT_NONE` and `POP_JUMP_IF_NONE` opcodes to speed up conditional jumps.

- bpo-45654 [https://bugs.python.org/issue?@action=redirect&bpo=45654]: Deepfreeze `runpy`, patch by Kumar Aditya.

- bpo-46025 [https://bugs.python.org/issue?@action=redirect&bpo=46025]: Fix a crash in the `atexit` module involving functions that unregister themselves before raising exceptions. Patch by Pablo Galindo.

Biblioteca

- bpo-46000 [https://bugs.python.org/issue?@action=redirect&bpo=46000]: Improve compatibility of the `curses` module with NetBSD curses.

Núcleo e embutidos

- bpo-44525 [https://bugs.python.org/issue?@action=redirect&bpo=44525]: Specialize the CALL_FUNCTION instruction for calls to builtin types with a single argument. Speeds up `range(x)`, `list(x)`, and specifically `type(obj)`.

- bpo-42918 [https://bugs.python.org/issue?@action=redirect&bpo=42918]: Fix bug where the built-in `compile()` function did not always raise a `SyntaxError` when passed multiple statements in 'single' mode. Patch by Weipeng Hong.
- bpo-45953 [https://bugs.python.org/issue?@action=redirect&bpo=45953]: The main interpreter in _PyRuntimeState.interpreters is now statically allocated (as part of _PyRuntime). Likewise for the initial thread state of each interpreter. This means less allocation during runtime init, as well as better memory locality for these key state objects.
- bpo-45292 [https://bugs.python.org/issue?@action=redirect&bpo=45292]: Complete the **PEP 654** [https://peps.python.org/pep-0654/] implementation: add `except*`.
- bpo-43413 [https://bugs.python.org/issue?@action=redirect&bpo=43413]: Revert changes in `set.__init__`. Subclass of `set` needs to define a `__init__()` method if it defines a `__new__()` method with additional keyword parameters.
- bpo-43931 [https://bugs.python.org/issue?@action=redirect&bpo=43931]: Added the `Py_Version` constant which bears the same value as `PY_VERSION_HEX`. Patch by Gabriele N. Tornetta.

Biblioteca

- bpo-46342 [https://bugs.python.org/issue?@action=redirect&bpo=46342]: The `@typing.final` decorator now sets the `__final__` attribute on the decorated object to allow runtime introspection. Patch by Jelle Zijlstra.

- bpo-46328 [https://bugs.python.org/issue?@action=redirect&bpo=46328]: Added the `sys.exception()` method which returns the active exception instance.

- bpo-46307 [https://bugs.python.org/issue?@action=redirect&bpo=46307]: Add `string.Template.is_valid()` and `string.Template.get_identifiers()` methods.

- bpo-46306 [https://bugs.python.org/issue?@action=redirect&bpo=46306]: Assume that `types.CodeType` always has `types.CodeType.co_firstlineno` in `doctest`.

- bpo-40479 [https://bugs.python.org/issue?@action=redirect&bpo=40479]: Fix **`hashlib`** *usedforsecurity* option to work correctly with OpenSSL 3.0.0 in FIPS mode.

- bpo-46070 [https://bugs.python.org/issue?@action=redirect&bpo=46070]: Fix possible segfault when importing the **`asyncio`** module from different sub-interpreters in parallel. Patch by Erlend E. Aasland.

- bpo-46244 [https://bugs.python.org/issue?@action=redirect&bpo=46244]: Removed `__slots__` from **`typing.ParamSpec`** and **`typing.TypeVar`**. They served no purpose. Patch by Arie Bovenberg.

- bpo-46278 [https://bugs.python.org/issue?@action=redirect&bpo=46278]: Reflect `context` argument in `AbstractEventLoop.call_*()` methods. Loop implementations already support it.

- bpo-46269 [https://bugs.python.org/issue?@action=redirect&bpo=46269]: Remove special-casing of `__new__` in **`enum.Enum.__dir__()`**.

- bpo-46266 [https://bugs.python.org/issue?@action=redirect&bpo=46266]: Improve day constants in **`calendar`**.

  Now all constants (`MONDAY` … `SUNDAY`) are documented, tested, and added to `__all__`.

- bpo-46257 [https://bugs.python.org/issue?@action=redirect&bpo=46257]: Optimized the mean, variance, and stdev functions in the statistics module. If the input is an iterator, it is consumed in a single pass rather than eating memory by conversion to a list. The single pass algorithm is about twice as fast as the previous two pass code.

- bpo-41011 [https://bugs.python.org/issue?@action=redirect&bpo=41011]: Added two new variables to *pyvenv.cfg* which is generated by **`venv`** module: *executable* for the executable and *command* for the command line used to create the environment.

- bpo-46239 [https://bugs.python.org/issue?@action=redirect&bpo=46239]: Improve error message when importing **`asyncio.windows_events`** on non-

Windows.

- bpo-46238 [https://bugs.python.org/issue?@action=redirect&bpo=46238]: Reuse `_winapi` constants in `asyncio.windows_events`.

- bpo-46222 [https://bugs.python.org/issue?@action=redirect&bpo=46222]: Adding `SF_NOCACHE` sendfile constant for FreeBSD for the posixmodule.

- bpo-37295 [https://bugs.python.org/issue?@action=redirect&bpo=37295]: Add fast path for `0 <= k <= n <= 67` for **`math.comb()`**.

- bpo-46176 [https://bugs.python.org/issue?@action=redirect&bpo=46176]: Adding the `MAP_STACK` constant for the mmap module.

- bpo-43424 [https://bugs.python.org/issue?@action=redirect&bpo=43424]: Deprecate **`webbrowser.MacOSXOSAScript._name`** and use `name` instead.

- bpo-45321 [https://bugs.python.org/issue?@action=redirect&bpo=45321]: Added missing error codes to module `xml.parsers.expat.errors`.

- bpo-46125 [https://bugs.python.org/issue?@action=redirect&bpo=46125]: Refactor tests to test traversable API directly. Includes changes from importlib 5.4.0.

- bpo-46118 [https://bugs.python.org/issue?@action=redirect&bpo=46118]: Moved importlib.resources and its related functionality to a package.

- bpo-37578 [https://bugs.python.org/issue?@action=redirect&bpo=37578]: Add *include_hidden* parameter to **`glob()`** and **`iglob()`** to match hidden files and directories when using special characters like `*`, `**`, `?` and `[]`.

- bpo-20369 [https://bugs.python.org/issue?@action=redirect&bpo=20369]: **`concurrent.futures.wait()`** no longer blocks forever when given duplicate Futures. Patch by Kumar Aditya.

- bpo-46105 [https://bugs.python.org/issue?@action=redirect&bpo=46105]: Honor spec when generating requirement specs with urls and extras (importlib_metadata 4.8.3).

- bpo-44893 [https://bugs.python.org/issue?@action=redirect&bpo=44893]: EntryPoint objects are no longer tuples. Recommended means to access is by attribute ('.name', '.group') or accessor ('.load()'). Access by index is deprecated and will raise deprecation warning.

- bpo-22815 [https://bugs.python.org/issue?@action=redirect&bpo=22815]: Print unexpected successes together with failures and errors in summary in **`unittest.TextTestResult`**.

- bpo-22047 [https://bugs.python.org/issue?@action=redirect&bpo=22047]: Calling **`add_argument_group()`** on an argument group is deprecated. Calling **`add_argument_group()`** or **`add_mutually_exclusive_group()`** on a mutually exclusive group is deprecated.

  These features were never supported and do not always work correctly. The functions exist on the API by accident through inheritance and will be removed in the future.

- bpo-26952 [https://bugs.python.org/issue?@action=redirect&bpo=26952]: **`argparse`** raises **`ValueError`** with clear message when trying to render usage for an empty mutually exclusive group. Previously it raised a cryptic **`IndexError`**.

- bpo-45615 [https://bugs.python.org/issue?@action=redirect&bpo=45615]: Functions in the **`traceback`** module raise **`TypeError`** rather than **`AttributeError`** when an exception argument is not of type **`BaseException`**.

- bpo-16594 [https://bugs.python.org/issue?@action=redirect&bpo=16594]: Add allow allow_reuse_port flag in socketserver.

- bpo-27718 [https://bugs.python.org/issue?@action=redirect&bpo=27718]: Fix help for the **`signal`** module. Some functions (e.g. `signal()` and `getsignal()`) were omitted.

- bpo-46032 [https://bugs.python.org/issue?@action=redirect&bpo=46032]: The `registry()` method of **`functools.singledispatch()`** functions checks now the first argument or the first parameter annotation and raises a TypeError if it is not supported. Previously unsupported

"types" were ignored (e.g. `typing.List[int]`) or caused an error at calling time (e.g. `list[int]`).

- bpo-46014 [https://bugs.python.org/issue?@action=redirect&bpo=46014]: Add ability to use `typing.Union` and `types.UnionType` as dispatch argument to `functools.singledispatch`. Patch provided by Yurii Karabas.

- bpo-27062 [https://bugs.python.org/issue?@action=redirect&bpo=27062]: Add **`__all__`** to **`inspect`**, patch by Kumar Aditya.

- bpo-46018 [https://bugs.python.org/issue?@action=redirect&bpo=46018]: Ensure that **`math.expm1()`** does not raise on underflow.

- bpo-46016 [https://bugs.python.org/issue?@action=redirect&bpo=46016]: Adding **`F_DUP2FD`** and **`F_DUP2FD_CLOEXEC`** constants from FreeBSD into the **`fcntl`** module.

- bpo-45755 [https://bugs.python.org/issue?@action=redirect&bpo=45755]: **`typing`** generic aliases now reveal the class attributes of the original generic class when passed to `dir()`. This was the behavior up to Python 3.6, but was changed in 3.7-3.9.

- bpo-45874 [https://bugs.python.org/issue?@action=redirect&bpo=45874]: The empty query string, consisting of no query arguments, is now handled correctly in `urllib.parse.parse_qsl`. This caused problems before when strict parsing was enabled.

- bpo-44674 [https://bugs.python.org/issue?@action=redirect&bpo=44674]: Change how dataclasses disallows mutable default values. It used to use a list of known types (list, dict, set). Now it disallows unhashable objects to be defaults. It's using unhashability as a proxy for mutability. Patch by Eric V. Smith, idea by Raymond Hettinger.

- bpo-23882 [https://bugs.python.org/issue?@action=redirect&bpo=23882]: Remove namespace package (PEP 420) support from unittest discovery. It was introduced in Python 3.4 but has been broken since Python 3.7.

- bpo-25066 [https://bugs.python.org/issue?@action=redirect&bpo=25066]: Added a **`__repr__()`** method to **`multiprocessing.Event`** objects, patch by Kumar Aditya.

- bpo-45643 [https://bugs.python.org/issue?@action=redirect&bpo=45643]: Added **`signal.SIGSTKFLT`** on platforms where this signal is defined.

- bpo-44092 [https://bugs.python.org/issue?@action=redirect&bpo=44092]: Fetch across rollback no longer raises **`InterfaceError`**. Instead we leave it to the SQLite library to handle these cases. Patch by Erlend E. Aasland.

- bpo-42413 [https://bugs.python.org/issue?@action=redirect&bpo=42413]: Replace `concurrent.futures.TimeoutError` and `asyncio.TimeoutError` with builtin **`TimeoutError`**, keep these names as deprecated aliases.

Documentação

- bpo-46196 [https://bugs.python.org/issue?@action=redirect&bpo=46196]: Document method **`cmd.Cmd.columnize()`**.
- bpo-46120 [https://bugs.python.org/issue?@action=redirect&bpo=46120]: State that `|` is preferred for readability over `Union` in the **`typing`** docs.
- bpo-46109 [https://bugs.python.org/issue?@action=redirect&bpo=46109]: Extracted `importlib.resources` and `importlib.resources.abc` documentation into separate files.
- bpo-19737 [https://bugs.python.org/issue?@action=redirect&bpo=19737]: Update the documentation for the **`globals()`** function.

Testes

- bpo-46296 [https://bugs.python.org/issue?@action=redirect&bpo=46296]: Add a test case for **`enum`** with `_use_args_ == True` and `_member_type_ == object`.
- bpo-46205 [https://bugs.python.org/issue?@action=redirect&bpo=46205]: Fix hang in runtest_mp due to race condition
- bpo-46263 [https://bugs.python.org/issue?@action=redirect&bpo=46263]: Fix test_capi on FreeBSD 14-dev: instruct jemalloc to not fill freed

memory with junk byte.

- bpo-46262 [https://bugs.python.org/issue?@action=redirect&bpo=46262]: Cover `ValueError` path in tests for **`enum.Flag._missing_()`**.
- bpo-46150 [https://bugs.python.org/issue?@action=redirect&bpo=46150]: Now `fakename` in `test_pathlib.PosixPathTest.test_expanduser` is checked to be non-existent.
- bpo-46129 [https://bugs.python.org/issue?@action=redirect&bpo=46129]: Rewrite `asyncio.locks` tests with **`unittest.IsolatedAsyncioTestCase`** usage.
- bpo-23819 [https://bugs.python.org/issue?@action=redirect&bpo=23819]: Fixed **`asyncio`** tests in python optimized mode. Patch by Kumar Aditya.
- bpo-46114 [https://bugs.python.org/issue?@action=redirect&bpo=46114]: Fix test case for OpenSSL 3.0.1 version. OpenSSL 3.0 uses `0xMNN00PP0L`.

Construção

- bpo-44133 [https://bugs.python.org/issue?@action=redirect&bpo=44133]: When Python is configured with **`--without-static-libpython`**, the Python static library (libpython.a) is no longer built. Patch by Victor Stinner.
- bpo-44133 [https://bugs.python.org/issue?@action=redirect&bpo=44133]: When Python is built without **`--enable-shared`**, the `python` program is now linked to object files, rather than being linked to the Python static library (libpython.a), to make sure that all symbols are exported. Previously, the linker omitted some symbols like the **`Py_FrozenMain()`** function. Patch by Victor Stinner.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: The `configure` script has a new option `--with-emscripten-target` to select browser or node as Emscripten build target.
- bpo-46315 [https://bugs.python.org/issue?@action=redirect&bpo=46315]: Added and fixed `#ifdef HAVE_FEATURE` checks for functionality that is not available on WASI platform.
- bpo-45723 [https://bugs.python.org/issue?@action=redirect&bpo=45723]: Fixed a regression in `configure` check for **`select.epoll()`**.
- bpo-46263 [https://bugs.python.org/issue?@action=redirect&bpo=46263]: `configure` no longer sets `MULTIARCH` on FreeBSD platforms.
- bpo-46106 [https://bugs.python.org/issue?@action=redirect&bpo=46106]: Updated OpenSSL to 1.1.1m in Windows builds, macOS installer builds, and

CI. Patch by Kumar Aditya.

- bpo-46088 [https://bugs.python.org/issue?@action=redirect&bpo=46088]: Automatically detect or install bootstrap Python runtime when building from Visual Studio.
- bpo-46072 [https://bugs.python.org/issue?@action=redirect&bpo=46072]: Add a –with-pystats configure option to turn on internal statistics gathering.
- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Um novo diretório `Tools/wasm` contém auxiliares relacionados a WebAssembly, como `config.site` próprio para wasm32-emscripten, arquivos do gerador wasm para empacotar a stdlib e um README.
- bpo-46023 [https://bugs.python.org/issue?@action=redirect&bpo=46023]: **makesetup** no longer builds extensions that have been marked as *disabled*. This allows users to disable modules in `Modules/Setup.local`.
- bpo-45949 [https://bugs.python.org/issue?@action=redirect&bpo=45949]: Use pure Python `freeze_module` for all but importlib bootstrap files. `--with-freeze-module` **configure** option is no longer needed for cross builds.

Windows

- bpo-46217 [https://bugs.python.org/issue?@action=redirect&bpo=46217]: Removed parameter that is unsupported on Windows 8.1 and early Windows 10 and may have caused build or runtime failures.

macOS

- bpo-40477 [https://bugs.python.org/issue?@action=redirect&bpo=40477]: The Python Launcher app for macOS now properly launches scripts and, if necessary, the Terminal app when running on recent macOS releases.

API C

- bpo-46236 [https://bugs.python.org/issue?@action=redirect&bpo=46236]: Fix a bug in **`PyFunction_GetAnnotations()`** that caused it to return a `tuple` instead of a `dict`.
- bpo-46140 [https://bugs.python.org/issue?@action=redirect&bpo=46140]: **`PyBuffer_GetPointer()`**, **`PyBuffer_FromContiguous()`**, **`PyBuffer_ToContiguous()`** and **`PyMemoryView_FromBuffer()`** now

take buffer info by `const Py_buffer *` instead of `Py_buffer *`, as they do not need mutability. **`PyBuffer_FromContiguous()`** also now takes the source buffer as `const void *`, and similarly **`PyBuffer_GetPointer()`** takes the strides as `const Py_ssize_t *`.

- bpo-45855 [https://bugs.python.org/issue?@action=redirect&bpo=45855]: Document that the *no_block* argument to **`PyCapsule_Import()`** is a no-op now.
- bpo-45855 [https://bugs.python.org/issue?@action=redirect&bpo=45855]: Replaced deprecated usage of **`PyImport_ImportModuleNoBlock()`** with **`PyImport_ImportModule()`** in stdlib modules. Patch by Kumar Aditya.
- bpo-46007 [https://bugs.python.org/issue?@action=redirect&bpo=46007]: The **`PyUnicode_CHECK_INTERNED()`** macro has been excluded from the limited C API. It was never usable there, because it used internal structures which are not available in the limited C API. Patch by Victor Stinner.

Python 3.11.0 alpha 3

Data de lançamento: 2021-12-08

Núcleo e embutidos

- bpo-46009 [https://bugs.python.org/issue?@action=redirect&bpo=46009]: Restore behavior from 3.9 and earlier when sending non-None to newly started generator. In 3.9 this did not affect the state of the generator. In 3.10.0 and 3.10.1 `gen_func().send(0)` is equivalent to `gen_func().throw(TypeError(...)` which exhausts the generator. In 3.10.2 onward, the behavior has been reverted to that of 3.9.
- bpo-46004 [https://bugs.python.org/issue?@action=redirect&bpo=46004]: Fix the **`SyntaxError`** location for errors involving for loops with invalid targets. Patch by Pablo Galindo
- bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711]: **`_PyErr_ChainStackItem()`** no longer normalizes `exc_info` (including setting the traceback on the exception instance) because `exc_info` is always normalized.
- bpo-45607 [https://bugs.python.org/issue?@action=redirect&bpo=45607]: The `__note__` field was added to **`BaseException`**. It is `None` by default but

can be set to a string which is added to the exception's traceback.

- bpo-45947 [https://bugs.python.org/issue?@action=redirect&bpo=45947]: Place pointers to dict and values immediately before GC header. This reduces number of dependent memory loads to access either dict or values from 3 to 1.
- bpo-45915 [https://bugs.python.org/issue?@action=redirect&bpo=45915]: `is_valid_fd` now uses faster `fcntl(fd, F_GETFD)` on Linux, macOS, and Windows.
- bpo-44530 [https://bugs.python.org/issue?@action=redirect&bpo=44530]: Reverts a change to the `code.__new__` audit event from an earlier prerelease.
- bpo-42268 [https://bugs.python.org/issue?@action=redirect&bpo=42268]: Fail the configure step if the selected compiler doesn't support memory sanitizer. Patch by Pablo Galindo
- bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711]: The three values of `exc_info` are now always consistent with each other. In particular, the `type` and `traceback` fields are now derived from the exception instance. This impacts the return values of **`sys.exc_info()`** and **`PyErr_GetExcInfo()`** if the exception instance is modified while the exception is handled, as well as **`PyErr_SetExcInfo()`**, which now ignores the `type` and `traceback` arguments provided to it.
- bpo-45727 [https://bugs.python.org/issue?@action=redirect&bpo=45727]: Refine the custom syntax error that suggests that a comma may be missing to trigger only when the expressions are detected between parentheses or brackets. Patch by Pablo Galindo
- bpo-45885 [https://bugs.python.org/issue?@action=redirect&bpo=45885]: Specialized the `COMPARE_OP` opcode using the PEP 659 machinery.
- bpo-45786 [https://bugs.python.org/issue?@action=redirect&bpo=45786]: Allocate space for the interpreter frame in the frame object, to avoid an additional allocation when the frame object outlives the frame activation.

Biblioteca

- bpo-45614 [https://bugs.python.org/issue?@action=redirect&bpo=45614]: Fix **`traceback`** display for exceptions with invalid module name.

Núcleo e embutidos

- bpo-45813 [https://bugs.python.org/issue?@action=redirect&bpo=45813]: Fix crash when calling coro.cr_frame.clear() after coroutine has been freed.

- bpo-45811 [https://bugs.python.org/issue?@action=redirect&bpo=45811]: Improve the tokenizer errors when encountering invisible control characters in the parser. Patch by Pablo Galindo

- bpo-45848 [https://bugs.python.org/issue?@action=redirect&bpo=45848]: Allow the parser to obtain error lines directly from encoded files. Patch by Pablo Galindo

- bpo-45709 [https://bugs.python.org/issue?@action=redirect&bpo=45709]: Restore behavior from 3.10 when tracing an exception raised within a with statement.

- bpo-44525 [https://bugs.python.org/issue?@action=redirect&bpo=44525]: Adds new **`COPY_FREE_VARS`** opcode, to make copying of free variables from function to frame explicit. Helps optimization of calls to Python function.

- bpo-45829 [https://bugs.python.org/issue?@action=redirect&bpo=45829]: Specialize **`BINARY_SUBSCR`** for classes with a `__getitem__` method implemented in Python

- bpo-45826 [https://bugs.python.org/issue?@action=redirect&bpo=45826]: Fixed a crash when calling `.with_traceback(None)` on `NameError`. This occurs internally in `unittest.TestCase.assertRaises()`.

- bpo-45822 [https://bugs.python.org/issue?@action=redirect&bpo=45822]: Fixed a bug in the parser that was causing it to not respect **PEP 263** [https://peps.python.org/pep-0263/] coding cookies when no flags are provided. Patch by Pablo Galindo

- bpo-45820 [https://bugs.python.org/issue?@action=redirect&bpo=45820]: Fix a segfault when the parser fails without reading any input. Patch by Pablo Galindo

- bpo-45636 [https://bugs.python.org/issue?@action=redirect&bpo=45636]: Simplify the implementation of **`BINARY_OP`** by indexing into an array of function pointers (rather than switching on the oparg).

- bpo-42540 [https://bugs.python.org/issue?@action=redirect&bpo=42540]: Fix crash when **`os.fork()`** is called with an active non-default memory allocator.

- bpo-45738 [https://bugs.python.org/issue?@action=redirect&bpo=45738]: Fix computation of error location for invalid continuation characters in the parser. Patch by Pablo Galindo.

- bpo-45636 [https://bugs.python.org/issue?@action=redirect&bpo=45636]: Remove an existing “fast path” for old-style string formatting, since it no longer appears to have any measurable impact.

- bpo-45753 [https://bugs.python.org/issue?@action=redirect&bpo=45753]: Make recursion checks a bit more efficient by tracking amount of calls left before overflow.

- bpo-45773 [https://bugs.python.org/issue?@action=redirect&bpo=45773]: Fix a compiler hang when attempting to optimize certain jump patterns.

- bpo-45764 [https://bugs.python.org/issue?@action=redirect&bpo=45764]: The parser now gives a better error message when leaving out the opening parenthesis `(` after a `def`-statement:

  ```
  >>> def f:
    File "<stdin>", line 1
      def f:
           ^
  SyntaxError: expected '('
  ```

- bpo-45609 [https://bugs.python.org/issue?@action=redirect&bpo=45609]: Specialized the `STORE_SUBSCR` opcode using the PEP 659 machinery.

- bpo-45636 [https://bugs.python.org/issue?@action=redirect&bpo=45636]: Replace all numeric `BINARY_*` and `INPLACE_*` instructions with a single **`BINARY_OP`** implementation.

- bpo-45582 [https://bugs.python.org/issue?@action=redirect&bpo=45582]: Path calculation (known as `getpath`) has been reimplemented as a frozen Python module. This should have no visible impact, but may affect calculation of all paths referenced in **`sys`** and **`sysconfig`**.

- bpo-45450 [https://bugs.python.org/issue?@action=redirect&bpo=45450]: Improve the syntax error message for parenthesized arguments. Patch by Pablo Galindo.

Biblioteca

- bpo-27946 [https://bugs.python.org/issue?@action=redirect&bpo=27946]: Fix possible crash when getting an attribute of **`xml.etree.ElementTree.Element`** simultaneously with replacing the `attrib` dict.

- bpo-45711 [https://bugs.python.org/issue?@action=redirect&bpo=45711]: Make **`asyncio`** normalize exceptions as soon as they are captured with **`PyErr_Fetch()`**, and before they are stored as an exc_info triplet. This brings **`asyncio`** in line with the rest of the codebase, where an exc_info triplet is always normalized.

- bpo-23819 [https://bugs.python.org/issue?@action=redirect&bpo=23819]: Replaced asserts with exceptions in asyncio, patch by Kumar Aditya.

- bpo-13236 [https://bugs.python.org/issue?@action=redirect&bpo=13236]: **`unittest.TextTestResult`** and **`unittest.TextTestRunner`** flush now the output stream more often.

- bpo-45917 [https://bugs.python.org/issue?@action=redirect&bpo=45917]: Added **`math.exp2()`**:, which returns 2 raised to the power of x.

- bpo-37658 [https://bugs.python.org/issue?@action=redirect&bpo=37658]: Fix issue when on certain conditions `asyncio.wait_for()` may allow a coroutine to complete successfully, but fail to return the result, potentially causing memory leaks or other issues.

- bpo-45876 [https://bugs.python.org/issue?@action=redirect&bpo=45876]: Improve the accuracy of stdev() and pstdev() in the statistics module. When the inputs are floats or fractions, the output is a correctly rounded float

- bpo-44649 [https://bugs.python.org/issue?@action=redirect&bpo=44649]: Handle dataclass(slots=True) with a field that has default a default value, but for which init=False.

- bpo-45803 [https://bugs.python.org/issue?@action=redirect&bpo=45803]: Added missing kw_only parameter to dataclasses.make_dataclass().

- bpo-45837 [https://bugs.python.org/issue?@action=redirect&bpo=45837]: The **`turtle.RawTurtle.settiltangle()`** is deprecated since Python 3.1, it now emits a deprecation warning and will be removed in Python 3.13.

  Use **`turtle.RawTurtle.tiltangle()`** instead.

  `turtle.RawTurtle.tiltangle()` was earlier incorrectly marked as deprecated, its docstring has been corrected.

  Patch por Hugo van Kemenade.

- bpo-45831 [https://bugs.python.org/issue?@action=redirect&bpo=45831]: **`faulthandler`** can now write ASCII-only strings (like filenames and function names) with a single write() syscall when dumping a traceback. It reduces the risk of getting an unreadable dump when two threads or two processes dump a traceback to the same file (like stderr) at the same time. Patch by Victor Stinner.

- bpo-45828 [https://bugs.python.org/issue?@action=redirect&bpo=45828]: **`sqlite`** C callbacks now use unraisable exceptions if callback tracebacks are enabled. Patch by Erlend E. Aasland.

- bpo-41735 [https://bugs.python.org/issue?@action=redirect&bpo=41735]: Fix thread lock in `zlib.Decompress.flush()` method before `PyObject_GetBuffer`.

- bpo-45235 [https://bugs.python.org/issue?@action=redirect&bpo=45235]: Reverted an argparse bugfix that caused regression in the handling of default arguments for subparsers. This prevented leaf level arguments from taking precedence over root level arguments.

- bpo-45754 [https://bugs.python.org/issue?@action=redirect&bpo=45754]: Fix a regression in Python 3.11a1 and 3.11a2 where `sqlite3` incorrectly would use `SQLITE_LIMIT_LENGTH` when checking SQL statement lengths. Now, `SQLITE_LIMIT_SQL_LENGTH` is used. Patch by Erlend E. Aasland.

- bpo-45766 [https://bugs.python.org/issue?@action=redirect&bpo=45766]: Added *proportional* option to `statistics.linear_regression()`.

- bpo-45765 [https://bugs.python.org/issue?@action=redirect&bpo=45765]: In importlib.metadata, fix distribution discovery for an empty path.

- bpo-45757 [https://bugs.python.org/issue?@action=redirect&bpo=45757]: Fix bug where `dis` produced an incorrect oparg when `EXTENDED_ARG` is followed by an opcode that does not use its argument.

- bpo-45644 [https://bugs.python.org/issue?@action=redirect&bpo=45644]: In-place JSON file formatting using `python3 -m json.tool infile infile` now works correctly, previously it left the file empty. Patch by Chris Wesseling.

- bpo-45703 [https://bugs.python.org/issue?@action=redirect&bpo=45703]: When a namespace package is imported before another module from the same namespace is created/installed in a different `sys.path` location while the program is running, calling the `importlib.invalidate_caches()` function will now also guarantee the new module is noticed.

- bpo-45535 [https://bugs.python.org/issue?@action=redirect&bpo=45535]: Improve output of `dir()` with Enums.

- bpo-45664 [https://bugs.python.org/issue?@action=redirect&bpo=45664]: Fix `types.resolve_bases()` and `types.new_class()` for `types.GenericAlias` instance as a base.

- bpo-45663 [https://bugs.python.org/issue?@action=redirect&bpo=45663]: Fix `dataclasses.is_dataclass()` for dataclasses which are subclasses of `types.GenericAlias`.

- bpo-45662 [https://bugs.python.org/issue?@action=redirect&bpo=45662]: Fix the repr of `dataclasses.InitVar` with a type alias to the built-in class, e.g. `InitVar[list[int]]`.

- bpo-43137 [https://bugs.python.org/issue?@action=redirect&bpo=43137]: Launch GNOME web browsers via gio tool instead of obsolete gvfs-open

- bpo-45429 [https://bugs.python.org/issue?@action=redirect&bpo=45429]: On Windows, `time.sleep()` now uses a waitable timer which supports high-resolution timers. Patch by Donghee Na and Eryk Sun.

- bpo-37295 [https://bugs.python.org/issue?@action=redirect&bpo=37295]: Optimize `math.comb()` and `math.perm()`.

- bpo-45514 [https://bugs.python.org/issue?@action=redirect&bpo=45514]: Deprecated legacy functions in `importlib.resources`.

- bpo-45507 [https://bugs.python.org/issue?@action=redirect&bpo=45507]: Add tests for truncated/missing trailers in gzip.decompress implementation.

- bpo-45359 [https://bugs.python.org/issue?@action=redirect&bpo=45359]: Implement **PEP 585** [https://peps.python.org/pep-0585/] for `graphlib.TopologicalSorter`.

- bpo-44733 [https://bugs.python.org/issue?@action=redirect&bpo=44733]: Add `max_tasks_per_child` to `concurrent.futures.ProcessPoolExecutor`. This allows users to specify the maximum number of tasks a single process should execute before the process needs to be restarted.

- bpo-28806 [https://bugs.python.org/issue?@action=redirect&bpo=28806]: Improve netrc library. netrc file no longer needs to contain all tokens. And if the login name is anonymous, security check is no longer need.

- bpo-43498 [https://bugs.python.org/issue?@action=redirect&bpo=43498]: Avoid a possible *"RuntimeError: dictionary changed size during iteration"* when adjusting the process count of **`ProcessPoolExecutor`**.

- bpo-42158 [https://bugs.python.org/issue?@action=redirect&bpo=42158]: Add MIME types for N-quads, N-triples, Notation3 and TriG to `mimetypes`.

- bpo-30533 [https://bugs.python.org/issue?@action=redirect&bpo=30533]: Add **`inspect.getmembers_static()`** , it return all members without triggering dynamic lookup via the descriptor protocol. Patch by Weipeng Hong.

Documentação

- bpo-42238 [https://bugs.python.org/issue?@action=redirect&bpo=42238]: `make -C Doc suspicious` will be removed soon in favor of `make -C Doc check`, mark it as deprecated.
- bpo-45840 [https://bugs.python.org/issue?@action=redirect&bpo=45840]: Improve cross-references in the documentation for the data model.
- bpo-45640 [https://bugs.python.org/issue?@action=redirect&bpo=45640]: Properly marked-up grammar tokens in the documentation are now clickable and take you to the definition of a given piece of grammar. Patch by Arthur Milchior.
- bpo-45788 [https://bugs.python.org/issue?@action=redirect&bpo=45788]: Link doc for sys.prefix to sysconfig doc on installation paths.
- bpo-45772 [https://bugs.python.org/issue?@action=redirect&bpo=45772]: `socket.socket` documentation is corrected to a class from a function.
- bpo-45392 [https://bugs.python.org/issue?@action=redirect&bpo=45392]: Update the docstring of the **`type`** built-in to remove a redundant line and to mention keyword arguments for the constructor.
- bpo-45250 [https://bugs.python.org/issue?@action=redirect&bpo=45250]: Update the documentation to note that CPython does not consistently require iterators to define `__iter__`.
- bpo-25381 [https://bugs.python.org/issue?@action=redirect&bpo=25381]: In the extending chapter of the extending doc, update a paragraph about the global variables containing exception information.

- bpo-43905 [https://bugs.python.org/issue?@action=redirect&bpo=43905]: Expanded **`astuple()`** and **`asdict()`** docs, warning about deepcopy being applied and providing a workaround.

Testes

- bpo-45695 [https://bugs.python.org/issue?@action=redirect&bpo=45695]: Out-of-tree builds with a read-only source directory are now tested by CI.
- bpo-19460 [https://bugs.python.org/issue?@action=redirect&bpo=19460]: Add new Test for `Lib/email/mime/nonmultipart.py::MIMENonMultipart`.
- bpo-45835 [https://bugs.python.org/issue?@action=redirect&bpo=45835]: Fix race condition in test_queue tests with multiple “feeder” threads.
- bpo-45783 [https://bugs.python.org/issue?@action=redirect&bpo=45783]: The test for the freeze tool now handles file moves and deletions.
- bpo-45745 [https://bugs.python.org/issue?@action=redirect&bpo=45745]: Remove the `--findleaks` command line option of regrtest: use the `--fail-env-changed` option instead. Since Python 3.7, it was a deprecated alias to the `--fail-env-changed` option.
- bpo-45701 [https://bugs.python.org/issue?@action=redirect&bpo=45701]: Add tests with `tuple` type with **`functools.lru_cache()`** to `test_functools`.

Construção

- bpo-44035 [https://bugs.python.org/issue?@action=redirect&bpo=44035]: CI now verifies that autoconf files have been regenerated with a current and unpatched autoconf package.

- bpo-45950 [https://bugs.python.org/issue?@action=redirect&bpo=45950]: The build system now uses a **_bootstrap_python** interpreter for freezing and deepfreezing again. To speed up build process the build tools **_bootstrap_python** and **_freeze_module** are no longer build with LTO.

- bpo-45881 [https://bugs.python.org/issue?@action=redirect&bpo=45881]: The **configure** script now accepts `--with-build-python` and `--with-freeze-module` options to make cross compiling easier.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Emscripten platform now uses `.wasm` suffix by default.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: Disable unusable core extension modules on WASM/Emscripten targets.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: `configure` now checks for socket `shutdown` function. The check makes it possible to disable `SYS_shutdown` with `ac_cv_func_shutdown=no` in CONFIG_SITE.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: `configure` now checks for functions `fork1, getegid, geteuid, getgid, getppid, getuid, opendir, pipe, system, wait, ttyname`.

- bpo-33393 [https://bugs.python.org/issue?@action=redirect&bpo=33393]: Update `config.guess` to 2021-06-03 and `config.sub` to 2021-08-14. `Makefile` now has an `update-config` target to make updating more convenient.

- bpo-45866 [https://bugs.python.org/issue?@action=redirect&bpo=45866]: `make regen-all` now produces the same output when run from a directory other than the source tree: when building Python out of the source tree. pegen now strips directory of the “generated by pygen from <FILENAME>” header Patch by Victor Stinner.

- bpo-40280 [https://bugs.python.org/issue?@action=redirect&bpo=40280]: `configure` now accepts machine `wasm32` or `wasm64` and OS `wasi` or `emscripten` for cross building, e.g. `wasm32-unknown-emscripten`, `wasm32-wasi`, or `wasm32-unknown-wasi`.

- bpo-41498 [https://bugs.python.org/issue?@action=redirect&bpo=41498]: Python now compiles on platforms without `sigset_t`. Several functions in **`signal`** are not available when `sigset_t` is missing.

  Based on patch by Roman Yurchak for pyodide.

- bpo-45881 [https://bugs.python.org/issue?@action=redirect&bpo=45881]: `setup.py` now uses `CC` from environment first to discover multiarch and cross compile paths.

- bpo-45886 [https://bugs.python.org/issue?@action=redirect&bpo=45886]: The `_freeze_module` program path can now be overridden on the command line, e.g. `make FREEZE_MODULE=../x86_64/Program/_freeze_module`.

- bpo-45873 [https://bugs.python.org/issue?@action=redirect&bpo=45873]: Get rid of the `_bootstrap_python` build step. The deepfreeze.py script is now run using `$(PYTHON_FOR_REGEN)` which can be Python 3.7 or newer (on Windows, 3.8 or newer).

- bpo-45847 [https://bugs.python.org/issue?@action=redirect&bpo=45847]: Port builtin hashlib extensions to `PY_STDLIB_MOD` macro and `addext()`.

- bpo-45723 [https://bugs.python.org/issue?@action=redirect&bpo=45723]: Add `autoconf` helpers for saving and restoring environment variables:

  - `SAVE_ENV`: Save `$CFLAGS`, `$LDFLAGS`, `$LIBS`, and `$CPPFLAGS`.
  - `RESTORE_ENV`: Restore `$CFLAGS`, `$LDFLAGS`, `$LIBS`, and `$CPPFLAGS`.
  - `WITH_SAVE_ENV([SCRIPT])`: Run `SCRIPT` wrapped with `SAVE_ENV` and `RESTORE_ENV`.

  Patch por Erlend E. Aasland.

- bpo-45573 [https://bugs.python.org/issue?@action=redirect&bpo=45573]: Mandatory core modules, that are required to bootstrap Python, are now in `Modules/Setup.bootstrap`.

- bpo-45573 [https://bugs.python.org/issue?@action=redirect&bpo=45573]: `configure` now creates `Modules/Setup.stdlib` with conditionally enabled/disabled extension module lines. The file is not used, yet.

- bpo-45573 [https://bugs.python.org/issue?@action=redirect&bpo=45573]: `configure` now uses a unified format to set state, compiler flags, and linker flags

in Makefile. The new macro `PY_STDLIB_MOD` sets three variables that are consumed by `Modules/Setup` and `setup.py`.

- bpo-45816 [https://bugs.python.org/issue?@action=redirect&bpo=45816]: Python now supports building with Visual Studio 2022 (MSVC v143, VS Version 17.0). Patch by Jeremiah Vivian.

- bpo-45800 [https://bugs.python.org/issue?@action=redirect&bpo=45800]: Settings for **`pyexpat`** C extension are now detected by `configure`. The bundled `expat` library is built in `Makefile`.

- bpo-45798 [https://bugs.python.org/issue?@action=redirect&bpo=45798]: Settings for **`decimal`** internal C extension are now detected by `configure`. The bundled `libmpdec` library is built in `Makefile`.

- bpo-45723 [https://bugs.python.org/issue?@action=redirect&bpo=45723]: **configure** has a new option `--with-pkg-config` to disable or require pkg-config.

- bpo-45774 [https://bugs.python.org/issue?@action=redirect&bpo=45774]: The build dependencies for **`sqlite3`** are now detected by `configure` and `pkg-config`. Patch by Erlend E. Aasland.

- bpo-45763 [https://bugs.python.org/issue?@action=redirect&bpo=45763]: The build dependencies for **`zlib`**, **`bz2`**, and **`lzma`** are now detected by `configure`.

- bpo-45747 [https://bugs.python.org/issue?@action=redirect&bpo=45747]: gdbm and dbm build dependencies are now detected by `configure`.

- bpo-45743 [https://bugs.python.org/issue?@action=redirect&bpo=45743]: On macOS, the build system no longer passes `search_paths_first` to the linker. The flag has been the default since Xcode 4 / macOS 10.6.

- bpo-45723 [https://bugs.python.org/issue?@action=redirect&bpo=45723]: `configure.ac` is now compatible with autoconf 2.71. Deprecated checks `STDC_HEADERS` and `AC_HEADER_TIME` have been removed.

- bpo-45723 [https://bugs.python.org/issue?@action=redirect&bpo=45723]: `configure` now prints a warning when pkg-config is missing.

- bpo-45731 [https://bugs.python.org/issue?@action=redirect&bpo=45731]: `configure --enable-loadable-sqlite-extensions` is now handled by new `PY_SQLITE_ENABLE_LOAD_EXTENSION` macro instead of logic in setup.py.

- bpo-45723 [https://bugs.python.org/issue?@action=redirect&bpo=45723]: configure.ac now uses custom helper macros and `AC_CACHE_CHECK` to simplify and speed up configure runs.

- bpo-45696 [https://bugs.python.org/issue?@action=redirect&bpo=45696]: Skip the marshal step for frozen modules by generating C code that produces a set of ready-to-use code objects. This speeds up startup time by another 10% or more.

- bpo-45561 [https://bugs.python.org/issue?@action=redirect&bpo=45561]: Run smelly.py tool from $(srcdir).

Windows

- bpo-46105 [https://bugs.python.org/issue?@action=redirect&bpo=46105]: Fixed calculation of **`sys.path`** in a venv on Windows.
- bpo-45901 [https://bugs.python.org/issue?@action=redirect&bpo=45901]: When installed through the Microsoft Store and set as the default app for `*.py` files, command line arguments will now be passed to Python when invoking a script without explicitly launching Python (that is, `script.py args` rather than `python script.py args`).
- bpo-45616 [https://bugs.python.org/issue?@action=redirect&bpo=45616]: Fix Python Launcher's ability to distinguish between versions 3.1 and 3.10 when either one is explicitly requested. Previously, 3.1 would be used if 3.10 was requested but not installed, and 3.10 would be used if 3.1 was requested but 3.10 was installed.
- bpo-45850 [https://bugs.python.org/issue?@action=redirect&bpo=45850]: Implement changes to build with deep-frozen modules on Windows. Note that we now require Python 3.10 as the "bootstrap" or "host" Python.
- bpo-45732 [https://bugs.python.org/issue?@action=redirect&bpo=45732]: Updates bundled Tcl/Tk to 8.6.12.

- bpo-45720 [https://bugs.python.org/issue?@action=redirect&bpo=45720]: Internal reference to `shlwapi.dll` was dropped to help improve startup time. This DLL will no longer be loaded at the start of every Python process.

macOS

- bpo-45732 [https://bugs.python.org/issue?@action=redirect&bpo=45732]: Update python.org macOS installer to use Tcl/Tk 8.6.12.

API C

- bpo-39026 [https://bugs.python.org/issue?@action=redirect&bpo=39026]: Fix Python.h to build C extensions with Xcode: remove a relative include from `Include/cpython/pystate.h`.

Python 3.11.0 alpha 2

Data de lançamento: 2021-11-05

Núcleo e embutidos

- bpo-45716 [https://bugs.python.org/issue?@action=redirect&bpo=45716]: Improve the **`SyntaxError`** message when using `True`, `None` or `False` as keywords in a function call. Patch by Pablo Galindo.

- bpo-45688 [https://bugs.python.org/issue?@action=redirect&bpo=45688]: **`sys.stdlib_module_names`** now contains the macOS-specific module **`_scproxy`**.

- bpo-45379 [https://bugs.python.org/issue?@action=redirect&bpo=45379]: Clarify **`ImportError`** message when we try to explicitly import a frozen module but frozen modules are disabled.

- bpo-44525 [https://bugs.python.org/issue?@action=redirect&bpo=44525]: Specialize simple calls to Python functions (no starargs, keyword dict, or closure)

- bpo-45530 [https://bugs.python.org/issue?@action=redirect&bpo=45530]: Cases of sorting using tuples as keys may now be significantly faster in some cases. Patch by Tim Peters.

  The order of the result may differ from earlier releases if the tuple elements don't define a total ordering (see Comparações de valor for information on total ordering). It's generally true that the result of sorting simply isn't well-defined in the absence of a total ordering on list elements.

- bpo-45526 [https://bugs.python.org/issue?@action=redirect&bpo=45526]: In obmalloc, set ADDRESS_BITS to not ignore any bits (ignored 16 before). That is safer in the case that the kernel gives user-space virtual addresses that span a range greater than 48 bits.

- bpo-30570 [https://bugs.python.org/issue?@action=redirect&bpo=30570]: Fixed a crash in `issubclass()` from infinite recursion when searching pathological `__bases__` tuples.

- bpo-45521 [https://bugs.python.org/issue?@action=redirect&bpo=45521]: Fix a bug in the obmalloc radix tree code. On 64-bit machines, the bug causes the tree to hold 46-bits of virtual addresses, rather than the intended 48-bits.

- bpo-45494 [https://bugs.python.org/issue?@action=redirect&bpo=45494]: Fix parser crash when reporting errors involving invalid continuation characters. Patch by Pablo Galindo.

- bpo-45445 [https://bugs.python.org/issue?@action=redirect&bpo=45445]: Python now fails to initialize if it finds an invalid `-X` option in the command line. Patch by Pablo Galindo.

- bpo-45340 [https://bugs.python.org/issue?@action=redirect&bpo=45340]: Object attributes are held in an array instead of a dictionary. An object's dictionary are created lazily, only when needed. Reduces the memory consumption of a typical Python object by about 30%. Patch by Mark Shannon.

- bpo-45408 [https://bugs.python.org/issue?@action=redirect&bpo=45408]: Fix a crash in the parser when reporting tokenizer errors that occur at the same time unclosed parentheses are detected. Patch by Pablo Galindo.

- bpo-29410 [https://bugs.python.org/issue?@action=redirect&bpo=29410]: Add SipHash13 for string hash algorithm and use it by default.

- bpo-45385 [https://bugs.python.org/issue?@action=redirect&bpo=45385]: Fix reference leak from descr_check. Patch by Donghee Na.

- bpo-45367 [https://bugs.python.org/issue?@action=redirect&bpo=45367]: Specialized the `BINARY_MULTIPLY` opcode to `BINARY_MULTIPLY_INT` and `BINARY_MULTIPLY_FLOAT` using the PEP 659 machinery.

- bpo-21736 [https://bugs.python.org/issue?@action=redirect&bpo=21736]: Frozen stdlib modules now have `__file__` to the .py file they would otherwise be loaded from, if possible. For packages, `__path__` now has the correct entry instead of being an empty list, which allows unfrozen submodules to be imported. These are set only if the stdlib directory is known when the runtime is initialized. Note that the file at `__file__` is not guaranteed to exist. None of this affects non-stdlib frozen modules nor, for now, frozen modules imported using `PyImport_ImportFrozenModule()`. Also, at the moment `co_filename` is not updated for the module.

- bpo-45020 [https://bugs.python.org/issue?@action=redirect&bpo=45020]: For frozen stdlib modules, record the original module name as `module.__spec__.loader_state.origname`. If the value is different than `module.__spec__.name` then the module was defined as an alias in Tools/scripts/freeze_modules.py. If it is `None` then the module comes from a source file outside the stdlib.

- bpo-45324 [https://bugs.python.org/issue?@action=redirect&bpo=45324]: In FrozenImporter.find_spec(), we now preserve the information needed in exec_module() to load the module. This change mostly impacts internal details, rather than changing the importer's behavior.

- bpo-45292 [https://bugs.python.org/issue?@action=redirect&bpo=45292]: Implement **PEP 654** [https://peps.python.org/pep-0654/]. Add `ExceptionGroup` and `BaseExceptionGroup`. Update traceback display code.

- bpo-40116 [https://bugs.python.org/issue?@action=redirect&bpo=40116]: Change to the implementation of split dictionaries. Classes where the instances differ either in the exact set of attributes, or in the order in which those attributes are set, can still share keys. This should have no observable effect on users of Python or the C-API. Patch by Mark Shannon.

- bpo-44050 [https://bugs.python.org/issue?@action=redirect&bpo=44050]: Extensions that indicate they use global state (by setting `m_size` to -1) can again be used in multiple interpreters. This reverts to behavior of Python 3.8.

- bpo-44525 [https://bugs.python.org/issue?@action=redirect&bpo=44525]: Setup initial specialization infrastructure for the `CALL_FUNCTION` opcode. Implemented initial specializations for C function calls:

  - `CALL_FUNCTION_BUILTIN_O` for `METH_O` flag.
  - `CALL_FUNCTION_BUILTIN_FAST` for `METH_FASTCALL` flag without keywords.
  - `CALL_FUNCTION_LEN` for `len(o)`.
  - `CALL_FUNCTION_ISINSTANCE` for `isinstance(o, t)`.

- bpo-44511 [https://bugs.python.org/issue?@action=redirect&bpo=44511]: Improve the generated bytecode for class and mapping patterns.

- bpo-43706 [https://bugs.python.org/issue?@action=redirect&bpo=43706]: Speed up calls to `enumerate()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

Biblioteca

- bpo-45679 [https://bugs.python.org/issue?@action=redirect&bpo=45679]: Fix caching of multi-value `typing.Literal`. `Literal[True, 2]` is no longer equal to `Literal[1, 2]`.

- bpo-42064 [https://bugs.python.org/issue?@action=redirect&bpo=42064]: Convert **`sqlite3`** to multi-phase initialisation (PEP 489). Patches by Erlend E. Aasland.

- bpo-45438 [https://bugs.python.org/issue?@action=redirect&bpo=45438]: Fix typing.Signature string representation for generic builtin types.

- bpo-45613 [https://bugs.python.org/issue?@action=redirect&bpo=45613]: **`sqlite3`** now sets **`sqlite3.threadsafety`** based on the default threading mode the underlying SQLite library has been compiled with. Patch by Erlend E. Aasland.

- bpo-45574 [https://bugs.python.org/issue?@action=redirect&bpo=45574]: Fix warning about `print_escape` being unused.

- bpo-45581 [https://bugs.python.org/issue?@action=redirect&bpo=45581]: **`sqlite3.connect()`** now correctly raises **`MemoryError`** if the underlying SQLite API signals memory error. Patch by Erlend E. Aasland.

- bpo-45557 [https://bugs.python.org/issue?@action=redirect&bpo=45557]: **`pprint.pprint()`** now handles *underscore_numbers* correctly. Previously it was always setting it to `False`.

- bpo-44019 [https://bugs.python.org/issue?@action=redirect&bpo=44019]: Add **`operator.call()`** to `operator.__all__`. Patch by Kreusada.

- bpo-42174 [https://bugs.python.org/issue?@action=redirect&bpo=42174]: **`shutil.get_terminal_size()`** now falls back to sane values if the column or line count are 0.

- bpo-35673 [https://bugs.python.org/issue?@action=redirect&bpo=35673]: Improve the introspectability of the `__loader__` attribute for namespace packages. **`importlib.machinery.NamespaceLoader`** is now public, and implements the **`importlib.abc.InspectLoader`** interface. `_NamespaceLoader` is kept for backward compatibility.

- bpo-45515 [https://bugs.python.org/issue?@action=redirect&bpo=45515]: Add references to `zoneinfo` in the `datetime` documentation, mostly replacing outdated references to `dateutil.tz`. Change by Paul Ganssle.

- bpo-45475 [https://bugs.python.org/issue?@action=redirect&bpo=45475]: Reverted optimization of iterating `gzip.GzipFile`, `bz2.BZ2File`, and `lzma.LZMAFile` (see bpo-43787 [https://bugs.python.org/issue?@action=redirect&bpo=43787]) because it caused regression when user iterate them without having reference of them. Patch by Inada Naoki.

- bpo-45489 [https://bugs.python.org/issue?@action=redirect&bpo=45489]: Update `ForwardRef` to support `|` operator. Patch by Donghee Na.

- bpo-42222 [https://bugs.python.org/issue?@action=redirect&bpo=42222]: Removed deprecated support for float arguments in *randrange()*.

- bpo-45428 [https://bugs.python.org/issue?@action=redirect&bpo=45428]: Fix a regression in py_compile when reading filenames from standard input.

- bpo-45467 [https://bugs.python.org/issue?@action=redirect&bpo=45467]: Fix incremental decoder and stream reader in the “raw-unicode-escape” codec. Previously they failed if the escape sequence was split.

- bpo-45461 [https://bugs.python.org/issue?@action=redirect&bpo=45461]: Fix incremental decoder and stream reader in the “unicode-escape” codec. Previously they failed if the escape sequence was split.

- bpo-45239 [https://bugs.python.org/issue?@action=redirect&bpo=45239]: Fixed `email.utils.parsedate_tz()` crashing with `UnboundLocalError` on certain invalid input instead of returning `None`. Patch by Ben Hoyt.

- bpo-45417 [https://bugs.python.org/issue?@action=redirect&bpo=45417]: Fix quadratic behaviour in the enum module: Creation of enum classes with a lot of entries was quadratic.

- bpo-45249 [https://bugs.python.org/issue?@action=redirect&bpo=45249]: Fix the behaviour of `traceback.print_exc()` when displaying the caret when

the `end_offset` in the exception is set to 0. Patch by Pablo Galindo

- bpo-45416 [https://bugs.python.org/issue?@action=redirect&bpo=45416]: Fix use of **`asyncio.Condition`** with explicit **`asyncio.Lock`** objects, which was a regression due to removal of explicit loop arguments. Patch by Joongi Kim.

- bpo-20028 [https://bugs.python.org/issue?@action=redirect&bpo=20028]: Empty escapechar/quotechar is not allowed when initializing **`csv.Dialect`**. Patch by Vajrasky Kok and Donghee Na.

- bpo-44904 [https://bugs.python.org/issue?@action=redirect&bpo=44904]: Fix bug in the **`doctest`** module that caused it to fail if a docstring included an example with a `classmethod` `property`. Patch by Alex Waygood.

- bpo-45406 [https://bugs.python.org/issue?@action=redirect&bpo=45406]: Make **`inspect.getmodule()`** catch `FileNotFoundError` raised by **`inspect.getabsfile()`**, and return `None` to indicate that the module could not be determined.

- bpo-45411 [https://bugs.python.org/issue?@action=redirect&bpo=45411]: Add extensions for files containing subtitles - .srt & .vtt - to the mimetypes.py module.

- bpo-10716 [https://bugs.python.org/issue?@action=redirect&bpo=10716]: Migrated pydoc to HTML5 (without changing the look of it). Side effect is to update xmlrpc's `ServerHTMLDoc` which now uses the CSS too. cgitb now relies less on pydoc (as it can't use the CSS file).

- bpo-27580 [https://bugs.python.org/issue?@action=redirect&bpo=27580]: Add support of null characters in **`csv`**.

- bpo-45262 [https://bugs.python.org/issue?@action=redirect&bpo=45262]: Prevent use-after-free in asyncio. Make sure the cached running loop holder gets cleared on dealloc to prevent use-after-free in get_running_loop

- bpo-45386 [https://bugs.python.org/issue?@action=redirect&bpo=45386]: Make **`xmlrpc.client`** more robust to C runtimes where the underlying C

`strftime` function results in a `ValueError` when testing for year formatting options.

- bpo-20028 [https://bugs.python.org/issue?@action=redirect&bpo=20028]: Improve error message of **`csv.Dialect`** when initializing. Patch by Vajrasky Kok and Donghee Na.

- bpo-45343 [https://bugs.python.org/issue?@action=redirect&bpo=45343]: Update bundled pip to 21.2.4 and setuptools to 58.1.0

- bpo-45328 [https://bugs.python.org/issue?@action=redirect&bpo=45328]: Fixed **`http.client.HTTPConnection`** to work properly in OSs that don't support the `TCP_NODELAY` socket option.

- bpo-45243 [https://bugs.python.org/issue?@action=redirect&bpo=45243]: Add **`setlimit()`** and **`getlimit()`** to **`sqlite3.Connection`** for setting and getting SQLite limits by connection basis. Patch by Erlend E. Aasland.

- bpo-45320 [https://bugs.python.org/issue?@action=redirect&bpo=45320]: Removed from the **`inspect`** module:

  - the `getargspec` function, deprecated since Python 3.0; use **`inspect.signature()`** or **`inspect.getfullargspec()`** instead.

  - the `formatargspec` function, deprecated since Python 3.5; use the **`inspect.signature()`** function and **`Signature`** object directly.
  - the undocumented `Signature.from_builtin` and `Signature.from_function` functions, deprecated since Python 3.5; use the **`Signature.from_callable()`** method instead.

  Patch por Hugo van Kemenade.

- bpo-45192 [https://bugs.python.org/issue?@action=redirect&bpo=45192]: Fix the `tempfile._infer_return_type` function so that the `dir` argument of the **`tempfile`** functions accepts an object implementing the `os.PathLike` protocol.

  Patch by Kyungmin Lee.

- bpo-45160 [https://bugs.python.org/issue?@action=redirect&bpo=45160]: When tracing a tkinter variable used by a ttk OptionMenu, callbacks are no longer made twice.

- bpo-25625 [https://bugs.python.org/issue?@action=redirect&bpo=25625]: Added non parallel-safe `chdir()` context manager to change the current working directory and then restore it on exit. Simple wrapper around `chdir()`.

- bpo-24139 [https://bugs.python.org/issue?@action=redirect&bpo=24139]: Add support for SQLite extended result codes in `sqlite3.Error`. Patch by Erlend E. Aasland.

- bpo-24444 [https://bugs.python.org/issue?@action=redirect&bpo=24444]: Fixed an error raised in `argparse` help display when help for an option is set to 1+ blank spaces or when *choices* arg is an empty container.

- bpo-44547 [https://bugs.python.org/issue?@action=redirect&bpo=44547]: Implement `Fraction.__int__`, so that a `fractions.Fraction` instance `f` passes an `isinstance(f, typing.SupportsInt)` check.

- bpo-40321 [https://bugs.python.org/issue?@action=redirect&bpo=40321]: Adds support for HTTP 308 redirects to `urllib`. See **RFC 7538** [https://datatracker.ietf.org/doc/html/rfc7538.html] for details. Patch by Jochem Schulenklopper.

- bpo-41374 [https://bugs.python.org/issue?@action=redirect&bpo=41374]: Ensure that `socket.TCP_*` constants are exposed on Cygwin 3.1.6 and greater.

- bpo-35970 [https://bugs.python.org/issue?@action=redirect&bpo=35970]: Add help flag to the base64 module's command line interface. Patch contributed by Robert Kuska.

Documentação

- bpo-45726 [https://bugs.python.org/issue?@action=redirect&bpo=45726]: Improve documentation for `functools.singledispatch()` and `functools.singledispatchmethod`.

- bpo-45680 [https://bugs.python.org/issue?@action=redirect&bpo=45680]: Amend the docs on `GenericAlias` objects to clarify that non-container classes can also implement `__class_getitem__`. Patch contributed by Alex Waygood.
- bpo-45618 [https://bugs.python.org/issue?@action=redirect&bpo=45618]: Update Sphinx version used to build the documentation to 4.2.0. Patch by Maciej Olko.
- bpo-45655 [https://bugs.python.org/issue?@action=redirect&bpo=45655]: Add a new "relevant PEPs" section to the top of the documentation for the `typing` module. Patch by Alex Waygood.
- bpo-45604 [https://bugs.python.org/issue?@action=redirect&bpo=45604]: Add `level` argument to `multiprocessing.log_to_stderr` function docs.
- bpo-45516 [https://bugs.python.org/issue?@action=redirect&bpo=45516]: Add protocol description to the **`importlib.abc.TraversableResources`** documentation.
- bpo-45464 [https://bugs.python.org/issue?@action=redirect&bpo=45464]: Mention in the documentation of Built-in Exceptions that inheriting from multiple exception types in a single subclass is not recommended due to possible memory layout incompatibility.
- bpo-45449 [https://bugs.python.org/issue?@action=redirect&bpo=45449]: Add note about **PEP 585** [https://peps.python.org/pep-0585/] in **`collections.abc`**.
- bpo-45516 [https://bugs.python.org/issue?@action=redirect&bpo=45516]: Add protocol description to the **`importlib.abc.Traversable`** documentation.
- bpo-20692 [https://bugs.python.org/issue?@action=redirect&bpo=20692]: Add Programming FAQ entry explaining that int literal attribute access requires either a space after or parentheses around the literal.

Testes

- bpo-45678 [https://bugs.python.org/issue?@action=redirect&bpo=45678]: Add tests for scenarios in which **`functools.singledispatchmethod`** is stacked on top of a method that has already been wrapped by two other decorators. Patch by Alex Waygood.
- bpo-45578 [https://bugs.python.org/issue?@action=redirect&bpo=45578]: Add tests for **`dis.distb()`**

- bpo-45678 [https://bugs.python.org/issue?@action=redirect&bpo=45678]: Add tests to ensure that `functools.singledispatchmethod` correctly wraps the attributes of the target function.
- bpo-45668 [https://bugs.python.org/issue?@action=redirect&bpo=45668]: PGO tests now pass when Python is built without test extension modules.
- bpo-45577 [https://bugs.python.org/issue?@action=redirect&bpo=45577]: Add subtests for all `pickle` protocols in `test_zoneinfo`.
- bpo-45566 [https://bugs.python.org/issue?@action=redirect&bpo=45566]: Fix `test_frozen_pickle` in `test_dataclasses` to check all `pickle` versions.
- bpo-43592 [https://bugs.python.org/issue?@action=redirect&bpo=43592]: **`test.libregrtest`** now raises the soft resource limit for the maximum number of file descriptors when the default is too low for our test suite as was often the case on macOS.
- bpo-39679 [https://bugs.python.org/issue?@action=redirect&bpo=39679]: Add more test cases for `@functools.singledispatchmethod` when combined with `@classmethod` or `@staticmethod`.
- bpo-45410 [https://bugs.python.org/issue?@action=redirect&bpo=45410]: When libregrtest spawns a worker process, stderr is now written into stdout to keep messages order. Use a single pipe for stdout and stderr, rather than two pipes. Previously, messages were out of order which made analysis of buildbot logs harder Patch by Victor Stinner.
- bpo-45402 [https://bugs.python.org/issue?@action=redirect&bpo=45402]: Fix test_tools.test_sundry() when Python is built out of tree: fix how the freeze_modules.py tool locates the _freeze_module program. Patch by Victor Stinner.
- bpo-45403 [https://bugs.python.org/issue?@action=redirect&bpo=45403]: Fix test_sys.test_stdlib_dir() when Python is built outside the source tree: compare normalized paths. Patch by Victor Stinner.
- bpo-45400 [https://bugs.python.org/issue?@action=redirect&bpo=45400]: Fix test_name_error_suggestions_do_not_trigger_for_too_many_locals() of test_exceptions if a directory name contains “a1” (like “Python-3.11.0a1”): use a stricter regular expression. Patch by Victor Stinner.
- bpo-10572 [https://bugs.python.org/issue?@action=redirect&bpo=10572]: Rename **`sqlite3`** tests from `test_sqlite` to `test_sqlite3`, and relocate them to `Lib/test/test_sqlite3`. Patch by Erlend E. Aasland.

Construção

- bpo-43158 [https://bugs.python.org/issue?@action=redirect&bpo=43158]: `setup.py` now uses values from configure script to build the `_uuid` extension module. Configure now detects util-linux's `libuuid`, too.
- bpo-45666 [https://bugs.python.org/issue?@action=redirect&bpo=45666]: Fix warning of `swprintf` and `%s` usage in `_testembed.c`
- bpo-45548 [https://bugs.python.org/issue?@action=redirect&bpo=45548]: `Modules/Setup` and `Modules/makesetup` have been improved. The `Setup` file now contains working rules for all extensions. Outdated comments have been removed. Rules defined by `makesetup` track dependencies correctly.
- bpo-45548 [https://bugs.python.org/issue?@action=redirect&bpo=45548]: The **`math`** and **`cmath`** implementation now require a C99 compatible `libm` and no longer ship with workarounds for missing acosh, asinh, atanh, expm1, and log1p functions.
- bpo-45595 [https://bugs.python.org/issue?@action=redirect&bpo=45595]: `setup.py` and `makesetup` now track build dependencies on all Python header files and module specific header files.
- bpo-45571 [https://bugs.python.org/issue?@action=redirect&bpo=45571]: `Modules/Setup` now use `PY_CFLAGS_NODIST` instead of `PY_CFLAGS` to compile shared modules.
- bpo-45570 [https://bugs.python.org/issue?@action=redirect&bpo=45570]: **`pyexpat`** and **`_elementtree`** no longer define obsolete macros `HAVE_EXPAT_CONFIG_H` and `USE_PYEXPAT_CAPI`. `XML_POOR_ENTROPY` is now defined in `expat_config.h`.
- bpo-43974 [https://bugs.python.org/issue?@action=redirect&bpo=43974]: `setup.py` no longer defines `Py_BUILD_CORE_MODULE`. Instead every module, that uses the internal API, defines the macro.
- bpo-45548 [https://bugs.python.org/issue?@action=redirect&bpo=45548]: Fill in missing entries in Modules/Setup.
- bpo-45532 [https://bugs.python.org/issue?@action=redirect&bpo=45532]: Update **`sys.version`** to use `main` as fallback information. Patch by Jeong YunWon.
- bpo-45536 [https://bugs.python.org/issue?@action=redirect&bpo=45536]: The `configure` script now checks whether OpenSSL headers and libraries

provide required APIs. Most common APIs are verified. The check detects outdated or missing OpenSSL. Failures do not stop configure.

- bpo-45221 [https://bugs.python.org/issue?@action=redirect&bpo=45221]: Fixed regression in handling of `LDFLAGS` and `CPPFLAGS` options where **`argparse.parse_known_args()`** could interpret an option as one of the built-in command line argument, for example `-h` for help.
- bpo-45440 [https://bugs.python.org/issue?@action=redirect&bpo=45440]: Building Python now requires a C99 `<math.h>` header file providing the following functions: `copysign()`, `hypot()`, `isfinite()`, `isinf()`, `isnan()`, `round()`. Patch by Victor Stinner.
- bpo-45405 [https://bugs.python.org/issue?@action=redirect&bpo=45405]: Prevent `internal configure error` when running `configure` with recent versions of non-Apple clang. Patch by David Bohman.
- bpo-45433 [https://bugs.python.org/issue?@action=redirect&bpo=45433]: Avoid linking libpython with libcrypt.

Windows

- bpo-43652 [https://bugs.python.org/issue?@action=redirect&bpo=43652]: Update Tcl/Tk to 8.6.11, actually this time. The previous update incorrectly included 8.6.10.
- bpo-45337 [https://bugs.python.org/issue?@action=redirect&bpo=45337]: venv now warns when the created environment may need to be accessed at a different path, due to redirections, links or junctions. It also now correctly installs or upgrades components when the alternate path is required.
- bpo-43851 [https://bugs.python.org/issue?@action=redirect&bpo=43851]: Build SQLite `SQLITE_OMIT_AUTOINIT` on Windows. Patch by Erlend E. Aasland.

macOS

- bpo-44828 [https://bugs.python.org/issue?@action=redirect&bpo=44828]: Avoid tkinter file dialog failure on macOS 12 Monterey when using the Tk 8.6.11 provided by python.org macOS installers. Patch by Marc Culler of the Tk project.

IDLE

- bpo-45495 [https://bugs.python.org/issue?@action=redirect&bpo=45495]: Add context keywords ‘case’ and ‘match’ to completions list.

API C

- bpo-29103 [https://bugs.python.org/issue?@action=redirect&bpo=29103]: **`PyType_FromSpec*`** now copies the class name from the spec to a buffer owned by the class, so the original can be safely deallocated. Patch by Petr Viktorin.

- bpo-45522 [https://bugs.python.org/issue?@action=redirect&bpo=45522]: The internal freelists for frame, float, list, dict, async generators, and context objects can now be disabled.

- bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134]: Exclude **`PyWeakref_GET_OBJECT()`** from the limited C API. It never worked since the **`PyWeakReference`** structure is opaque in the limited C API.

- bpo-35081 [https://bugs.python.org/issue?@action=redirect&bpo=35081]: Move the `interpreteridobject.h` header file from `Include/` to `Include/internal/`. It only provides private functions. Patch by Victor Stinner.

- bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134]: The non-limited API files `cellobject.h`, `classobject.h`, `context.h`, `funcobject.h`, `genobject.h` and `longintrepr.h` have been moved to the `Include/cpython` directory. Moreover, the `eval.h` header file was removed. These files must not be included directly, as they are already included in `Python.h`: Include Files. If they have been included directly, consider including `Python.h` instead. Patch by Victor Stinner.

- bpo-45474 [https://bugs.python.org/issue?@action=redirect&bpo=45474]: The following items are no longer available when `Py_LIMITED_API` is defined:

  - **`PyMarshal_WriteLongToFile()`**

- **`PyMarshal_WriteObjectToFile()`**
- **`PyMarshal_ReadObjectFromString()`**
- **`PyMarshal_WriteObjectToString()`**
- a macro `Py_MARSHAL_VERSION`

Eles não fazem parte da API limitada.

Patch por Victor Stinner.

- bpo-45434 [https://bugs.python.org/issue?@action=redirect&bpo=45434]: Remove the `pystrhex.h` header file. It only contains private functions. C extensions should only include the main `<Python.h>` header file. Patch by Victor Stinner.

- bpo-45440 [https://bugs.python.org/issue?@action=redirect&bpo=45440]: Remove the `Py_FORCE_DOUBLE()` macro. It was used by the `Py_IS_INFINITY()` macro. Patch by Victor Stinner.

- bpo-45434 [https://bugs.python.org/issue?@action=redirect&bpo=45434]: `<Python.h>` no longer includes the header files `<stdlib.h>`, `<stdio.h>`, `<errno.h>` and `<string.h>` when the `Py_LIMITED_API` macro is set to `0x030b0000` (Python 3.11) or higher. C extensions should explicitly include the header files after `#include <Python.h>`. Patch by Victor Stinner.

- bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123]: Remove `Py_UNICODE_COPY()` and `Py_UNICODE_FILL()` macros, deprecated since Python 3.3. Use `PyUnicode_CopyCharacters()` or `memcpy()` (`wchar_t*` string), and `PyUnicode_Fill()` functions instead. Patch by Victor Stinner.

- bpo-45412 [https://bugs.python.org/issue?@action=redirect&bpo=45412]: Remove the following math macros using the `errno` variable:

  - `Py_ADJUST_ERANGE1()`
  - `Py_ADJUST_ERANGE2()`
  - `Py_OVERFLOWED()`
  - `Py_SET_ERANGE_IF_OVERFLOW()`

- `Py_SET_ERRNO_ON_MATH_ERROR()`

Patch por Victor Stinner.

- bpo-45395 [https://bugs.python.org/issue?@action=redirect&bpo=45395]: Custom frozen modules (the array set to `PyImport_FrozenModules`) are now treated as additions, rather than replacing all the default frozen modules. Frozen stdlib modules can still be disabled by setting the "code" field of the custom array entry to NULL.

- bpo-43760 [https://bugs.python.org/issue?@action=redirect&bpo=43760]: Add new **`PyThreadState_EnterTracing()`**, and **`PyThreadState_LeaveTracing()`** functions to the limited C API to suspend and resume tracing and profiling. Patch by Victor Stinner.

- bpo-44220 [https://bugs.python.org/issue?@action=redirect&bpo=44220]: **`PyStructSequence_UnnamedField`** is added to the Stable ABI.

Python 3.11.0 alpha 1

Data de lançamento: 2021-10-05

Segurança

- bpo-42278 [https://bugs.python.org/issue?@action=redirect&bpo=42278]: Replaced usage of **`tempfile.mktemp()`** with **`TemporaryDirectory`** to avoid a potential race condition.
- bpo-44600 [https://bugs.python.org/issue?@action=redirect&bpo=44600]: Fix incorrect line numbers while tracing some failed patterns in match statements. Patch by Charles Burkland.
- bpo-41180 [https://bugs.python.org/issue?@action=redirect&bpo=41180]: Add auditing events to the **`marshal`** module, and stop raising `code.__init__` events for every unmarshalled code object. Directly instantiated code objects will continue to raise an event, and audit event handlers should inspect or collect the raw marshal data. This reduces a significant performance overhead when loading from `.pyc` files.

- bpo-44394 [https://bugs.python.org/issue?@action=redirect&bpo=44394]: Update the vendored copy of libexpat to 2.4.1 (from 2.2.8) to get the fix for the **CVE 2013-0340** [https://www.cve.org/CVERecord?id=CVE-2013-0340] “Billion Laughs” vulnerability. This copy is most used on Windows and macOS.
- bpo-43124 [https://bugs.python.org/issue?@action=redirect&bpo=43124]: Made the internal `putcmd` function in **`smtplib`** sanitize input for presence of `\r` and `\n` characters to avoid (unlikely) command injection.
- bpo-44022 [https://bugs.python.org/issue?@action=redirect&bpo=44022]: **`http.client`** now avoids infinitely reading potential HTTP headers after a `100 Continue` status response from the server.

Núcleo e embutidos

- bpo-43760 [https://bugs.python.org/issue?@action=redirect&bpo=43760]: The number of hardware branches per instruction dispatch is reduced from two to one by adding a special instruction for tracing. Patch by Mark Shannon.
- bpo-45061 [https://bugs.python.org/issue?@action=redirect&bpo=45061]: Add a deallocator to the bool type to detect refcount bugs in C extensions which call Py_DECREF(Py_True) or Py_DECREF(Py_False) by mistake. Detect also refcount bugs when the empty tuple singleton or the Unicode empty string singleton is destroyed by mistake. Patch by Victor Stinner.
- bpo-24076 [https://bugs.python.org/issue?@action=redirect&bpo=24076]: sum() was further optimised for summing up single digit integers.
- bpo-45190 [https://bugs.python.org/issue?@action=redirect&bpo=45190]: Update Unicode databases to Unicode 14.0.0.
- bpo-45167 [https://bugs.python.org/issue?@action=redirect&bpo=45167]: Fix deepcopying of **`types.GenericAlias`** objects.
- bpo-45155 [https://bugs.python.org/issue?@action=redirect&bpo=45155]: **`int.to_bytes()`** and **`int.from_bytes()`** now take a default value of `"big"` for the `byteorder` argument. **`int.to_bytes()`** also takes a default value of `1` for the `length` argument.

Biblioteca

- bpo-44219 [https://bugs.python.org/issue?@action=redirect&bpo=44219]: Release the GIL while performing `isatty` system calls on arbitrary file descriptors. In particular, this affects **`os.isatty()`**, **`os.device_encoding()`** and **`io.TextIOWrapper`**. By extension, **`io.open()`** in text mode is also affected. This change solves a deadlock in **`os.isatty()`**. Patch by Vincent Michel in bpo-44219 [https://bugs.python.org/issue?@action=redirect&bpo=44219].

Núcleo e embutidos

- bpo-44959 [https://bugs.python.org/issue?@action=redirect&bpo=44959]: Added fallback to extension modules with '.sl' suffix on HP-UX

Biblioteca

- bpo-45121 [https://bugs.python.org/issue?@action=redirect&bpo=45121]: Fix issue where `Protocol.__init__` raises `RecursionError` when it's called directly or via `super()`. Patch provided by Yurii Karabas.

Núcleo e embutidos

- bpo-44348 [https://bugs.python.org/issue?@action=redirect&bpo=44348]: The deallocator function of the **`BaseException`** type now uses the trashcan mechanism to prevent stack overflow. For example, when a **`RecursionError`** instance is raised, it can be linked to another RecursionError through the `__context__` attribute or the `__traceback__` attribute, and then a chain of exceptions is created. When the chain is destroyed, nested deallocator function calls can crash with a stack overflow if the chain is too long compared to the available stack memory. Patch by Victor Stinner.

- bpo-45123 [https://bugs.python.org/issue?@action=redirect&bpo=45123]: Fix PyAiter_Check to only check for the __anext__ presence (not for __aiter__). Rename PyAiter_Check to PyAIter_Check, PyObject_GetAiter -> PyObject_GetAIter.

- bpo-1514420 [https://bugs.python.org/issue?@action=redirect&bpo=1514420]: Interpreter no longer attempts to open files with names in angle brackets (like “<string>” or “<stdin>”) when formatting an exception.

- bpo-41031 [https://bugs.python.org/issue?@action=redirect&bpo=41031]: Match C and Python code formatting of unprintable exceptions and exceptions in the `__main__` module.

- bpo-37330 [https://bugs.python.org/issue?@action=redirect&bpo=37330]: `open()`, `io.open()`, `codecs.open()` and `fileinput.FileInput` no longer accept `'U'` (“universal newline”) in the file mode. This flag was deprecated since Python 3.3. Patch by Victor Stinner.

- bpo-45083 [https://bugs.python.org/issue?@action=redirect&bpo=45083]: When the interpreter renders an exception, its name now has a complete qualname. Previously only the class name was concatenated to the module name, which sometimes resulted in an incorrect full name being displayed.

  (This issue impacted only the C code exception rendering, the `traceback` module was using qualname already).

- bpo-34561 [https://bugs.python.org/issue?@action=redirect&bpo=34561]: List sorting now uses the merge-ordering strategy from Munro and Wild’s `powersort()`. Unlike the former strategy, this is provably near-optimal in the entropy of the distribution of run lengths. Most uses of `list.sort()` probably won’t see a significant time difference, but may see significant improvements in cases where the former strategy was exceptionally poor. However, as these are all fast linear-time approximations to a problem that’s inherently at best quadratic-time to solve truly optimally, it’s also possible to contrive cases where the former strategy did better.

- bpo-45056 [https://bugs.python.org/issue?@action=redirect&bpo=45056]: Compiler now removes trailing unused constants from co_consts.

- bpo-45020 [https://bugs.python.org/issue?@action=redirect&bpo=45020]: Add a new command line option, “-X frozen_modules=[on|off]” to opt out of (or

into) using optional frozen modules. This defaults to “on” (or “off” if it’s running out of the source tree).

Biblioteca

- bpo-45012 [https://bugs.python.org/issue?@action=redirect&bpo=45012]: In **`posix`**, release GIL during `stat()`, `lstat()`, and `fstatat()` syscalls made by **`os.DirEntry.stat()`**. Patch by Stanisław Skonieczny.

Núcleo e embutidos

- bpo-45018 [https://bugs.python.org/issue?@action=redirect&bpo=45018]: Fixed pickling of range iterators that iterated for over `2**32` times.
- bpo-45000 [https://bugs.python.org/issue?@action=redirect&bpo=45000]: A **`SyntaxError`** is now raised when trying to delete **`__debug__`**. Patch by Donghee Na.
- bpo-44963 [https://bugs.python.org/issue?@action=redirect&bpo=44963]: Implement `send()` and `throw()` methods for `anext_awaitable` objects. Patch by Pablo Galindo.

Biblioteca

- bpo-44962 [https://bugs.python.org/issue?@action=redirect&bpo=44962]: Fix a race in WeakKeyDictionary, WeakValueDictionary and WeakSet when two threads attempt to commit the last pending removal. This fixes asyncio.create_task and fixes a data loss in asyncio.run where shutdown_asyncgens is not run

Núcleo e embutidos

- bpo-24234 [https://bugs.python.org/issue?@action=redirect&bpo=24234]: Implement the **`__bytes__()`** special method on the **`bytes`** type, so a bytes object `b` passes an `isinstance(b, typing.SupportsBytes)` check.
- bpo-24234 [https://bugs.python.org/issue?@action=redirect&bpo=24234]: Implement the **`__complex__()`** special method on the **`complex`** type, so a complex number `z` passes an `isinstance(z, typing.SupportsComplex)` check.

- bpo-44954 [https://bugs.python.org/issue?@action=redirect&bpo=44954]: Fixed a corner case bug where the result of `float.fromhex('0x.8p-1074')` was rounded the wrong way.
- bpo-44947 [https://bugs.python.org/issue?@action=redirect&bpo=44947]: Refine the syntax error for trailing commas in import statements. Patch by Pablo Galindo.
- bpo-44945 [https://bugs.python.org/issue?@action=redirect&bpo=44945]: Specialize the BINARY_ADD instruction using the PEP 659 machinery. Adds five new instructions:
  - BINARY_ADD_ADAPTIVE
  - BINARY_ADD_FLOAT
  - BINARY_ADD_INT
  - BINARY_ADD_UNICODE
  - BINARY_ADD_UNICODE_INPLACE_FAST

Biblioteca

- bpo-44929 [https://bugs.python.org/issue?@action=redirect&bpo=44929]: Fix some edge cases of `enum.Flag` string representation in the REPL. Patch by Pablo Galindo.

Núcleo e embutidos

- bpo-44914 [https://bugs.python.org/issue?@action=redirect&bpo=44914]: Class version tags are no longer recycled.

  This means that a version tag serves as a unique identifier for the state of a class. We rely on this for effective specialization of the LOAD_ATTR and other instructions.

- bpo-44698 [https://bugs.python.org/issue?@action=redirect&bpo=44698]: Restore behaviour of complex exponentiation with integer-valued exponent of type **`float`** or **`complex`**.

- bpo-44895 [https://bugs.python.org/issue?@action=redirect&bpo=44895]: A debug variable **`PYTHONDUMPREFSFILE`** is added for creating a dump file which is generated by **`--with-trace-refs`**. Patch by Donghee Na.

- bpo-44900 [https://bugs.python.org/issue?@action=redirect&bpo=44900]: Add five superinstructions for PEP 659 quickening:
    - LOAD_FAST LOAD_FAST
    - STORE_FAST LOAD_FAST
    - LOAD_FAST LOAD_CONST
    - LOAD_CONST LOAD_FAST
    - STORE_FAST STORE_FAST
- bpo-44889 [https://bugs.python.org/issue?@action=redirect&bpo=44889]: Initial implementation of adaptive specialization of `LOAD_METHOD`. The following specialized forms were added:
    - `LOAD_METHOD_CACHED`
    - `LOAD_METHOD_MODULE`
    - `LOAD_METHOD_CLASS`
- bpo-44890 [https://bugs.python.org/issue?@action=redirect&bpo=44890]: Specialization stats are always collected in debug builds.
- bpo-44885 [https://bugs.python.org/issue?@action=redirect&bpo=44885]: Correct the ast locations of f-strings with format specs and repeated expressions. Patch by Pablo Galindo
- bpo-44878 [https://bugs.python.org/issue?@action=redirect&bpo=44878]: Remove the loop from the bytecode interpreter. All instructions end with a DISPATCH macro, so the loop is now redundant.
- bpo-44878 [https://bugs.python.org/issue?@action=redirect&bpo=44878]: Remove switch statement for interpreter loop when using computed gotos. This makes sure that we only have one dispatch table in the interpreter.
- bpo-44874 [https://bugs.python.org/issue?@action=redirect&bpo=44874]: Deprecate the old trashcan macros (`Py_TRASHCAN_SAFE_BEGIN`/`Py_TRASHCAN_SAFE_END`). They should be replaced by the new macros `Py_TRASHCAN_BEGIN` and `Py_TRASHCAN_END`.

- bpo-44872 [https://bugs.python.org/issue?@action=redirect&bpo=44872]: Use new trashcan macros (Py_TRASHCAN_BEGIN/END) in frameobject.c instead of the old ones (Py_TRASHCAN_SAFE_BEGIN/END).

- bpo-33930 [https://bugs.python.org/issue?@action=redirect&bpo=33930]: Fix segmentation fault with deep recursion when cleaning method objects. Patch by Augusto Goulart and Pablo Galindo.

- bpo-25782 [https://bugs.python.org/issue?@action=redirect&bpo=25782]: Fix bug where `PyErr_SetObject` hangs when the current exception has a cycle in its context chain.

- bpo-44856 [https://bugs.python.org/issue?@action=redirect&bpo=44856]: Fix reference leaks in the error paths of `update_bases()` and `__build_class__`. Patch by Pablo Galindo.

- bpo-44826 [https://bugs.python.org/issue?@action=redirect&bpo=44826]: Initial implementation of adaptive specialization of STORE_ATTR

  Three specialized forms of STORE_ATTR are added:

  - STORE_ATTR_SLOT
  - STORE_ATTR_SPLIT_KEYS
  - STORE_ATTR_WITH_HINT

- bpo-44838 [https://bugs.python.org/issue?@action=redirect&bpo=44838]: Fixed a bug that was causing the parser to raise an incorrect custom **`SyntaxError`** for invalid ‘if’ expressions. Patch by Pablo Galindo.

- bpo-44821 [https://bugs.python.org/issue?@action=redirect&bpo=44821]: Create instance dictionaries (__dict__) eagerly, to improve regularity of object layout and assist specialization.

- bpo-44792 [https://bugs.python.org/issue?@action=redirect&bpo=44792]: Improve syntax errors for if expressions. Patch by Miguel Brito

- bpo-34013 [https://bugs.python.org/issue?@action=redirect&bpo=34013]: Generalize the invalid legacy statement custom error message (like the one

generated when “print” is called without parentheses) to include more generic expressions. Patch by Pablo Galindo

- bpo-44732 [https://bugs.python.org/issue?@action=redirect&bpo=44732]: Rename `types.Union` to `types.UnionType`.

- bpo-44725 [https://bugs.python.org/issue?@action=redirect&bpo=44725]: Expose specialization stats in python via **`_opcode.get_specialization_stats()`**.

- bpo-44717 [https://bugs.python.org/issue?@action=redirect&bpo=44717]: Improve AttributeError on circular imports of submodules.

- bpo-44698 [https://bugs.python.org/issue?@action=redirect&bpo=44698]: Fix undefined behaviour in complex object exponentiation.

- bpo-44653 [https://bugs.python.org/issue?@action=redirect&bpo=44653]: Support **`typing`** types in parameter substitution in the union type.

- bpo-44676 [https://bugs.python.org/issue?@action=redirect&bpo=44676]: Add ability to serialise `types.Union` objects. Patch provided by Yurii Karabas.

- bpo-44633 [https://bugs.python.org/issue?@action=redirect&bpo=44633]: Parameter substitution of the union type with wrong types now raises `TypeError` instead of returning **`NotImplemented`**.

- bpo-44661 [https://bugs.python.org/issue?@action=redirect&bpo=44661]: Update `property_descr_set` to use vectorcall if possible. Patch by Donghee Na.

- bpo-44662 [https://bugs.python.org/issue?@action=redirect&bpo=44662]: Add `__module__` to `types.Union`. This also fixes `types.Union` issues with `typing.Annotated`. Patch provided by Yurii Karabas.

- bpo-44655 [https://bugs.python.org/issue?@action=redirect&bpo=44655]: Include the name of the type in unset __slots__ attribute errors. Patch by Pablo Galindo

- bpo-44655 [https://bugs.python.org/issue?@action=redirect&bpo=44655]: Don't include a missing attribute with the same name as the failing one when offering suggestions for missing attributes. Patch by Pablo Galindo

- bpo-44646 [https://bugs.python.org/issue?@action=redirect&bpo=44646]: Fix the hash of the union type: it no longer depends on the order of arguments.

- bpo-44636 [https://bugs.python.org/issue?@action=redirect&bpo=44636]: Collapse union of equal types. E.g. the result of `int | int` is now `int`. Fix comparison of the union type with non-hashable objects. E.g. `int | str == {}` no longer raises a TypeError.

Biblioteca

- bpo-44611 [https://bugs.python.org/issue?@action=redirect&bpo=44611]: On Windows, **`os.urandom()`**: uses BCryptGenRandom API instead of CryptGenRandom API which is deprecated from Microsoft Windows API. Patch by Donghee Na.

Núcleo e embutidos

- bpo-44635 [https://bugs.python.org/issue?@action=redirect&bpo=44635]: Convert `None` to `type(None)` in the union type constructor.

- bpo-26280 [https://bugs.python.org/issue?@action=redirect&bpo=26280]: Implement adaptive specialization for BINARY_SUBSCR

  Three specialized forms of BINARY_SUBSCR are added:

  - BINARY_SUBSCR_LIST_INT
  - BINARY_SUBSCR_TUPLE_INT
  - BINARY_SUBSCR_DICT

- bpo-44589 [https://bugs.python.org/issue?@action=redirect&bpo=44589]: Mapping patterns in `match` statements with two or more equal literal keys will now raise a **`SyntaxError`** at compile-time.

- bpo-44606 [https://bugs.python.org/issue?@action=redirect&bpo=44606]: Fix `__instancecheck__` and `__subclasscheck__` for the union type.

- bpo-42073 [https://bugs.python.org/issue?@action=redirect&bpo=42073]: The `@classmethod` decorator can now wrap other classmethod-like descriptors.

- bpo-41972 [https://bugs.python.org/issue?@action=redirect&bpo=41972]: Tuned the string-searching algorithm of fastsearch.h to have a shorter inner loop for most cases.

- bpo-44590 [https://bugs.python.org/issue?@action=redirect&bpo=44590]: All necessary data for executing a Python function (local variables, stack, etc) is now kept in a per-thread stack. Frame objects are lazily allocated on demand. This increases performance by about 7% on the standard benchmark suite. Introspection and debugging are unaffected as frame objects are always available when needed. Patch by Mark Shannon.

- bpo-44584 [https://bugs.python.org/issue?@action=redirect&bpo=44584]: The threading debug (**`PYTHONTHREADDEBUG`** environment variable) is deprecated in Python 3.10 and will be removed in Python 3.12. This feature requires a debug build of Python. Patch by Victor Stinner.

- bpo-43895 [https://bugs.python.org/issue?@action=redirect&bpo=43895]: An obsolete internal cache of shared object file handles added in 1995 that attempted, but did not guarantee, that a .so would not be dlopen'ed twice to work around flaws in mid-1990s posix-ish operating systems has been removed from dynload_shlib.c.

Biblioteca

- bpo-44490 [https://bugs.python.org/issue?@action=redirect&bpo=44490]: **`typing`** now searches for type parameters in `types.Union` objects. `get_type_hints` will also properly resolve annotations with nested `types.Union` objects. Patch provided by Yurii Karabas.

Núcleo e embutidos

- bpo-43950 [https://bugs.python.org/issue?@action=redirect&bpo=43950]: Code objects can now provide the column information for instructions when available. This is levaraged during traceback printing to show the expressions responsible for errors.

  Contributed by Pablo Galindo, Batuhan Taskaya and Ammar Askar as part of **PEP 657** [https://peps.python.org/pep-0657/].

- bpo-44562 [https://bugs.python.org/issue?@action=redirect&bpo=44562]: Remove uses of `PyObject_GC_Del()` in error path when initializing `types.GenericAlias`.

Biblioteca

- bpo-41486 [https://bugs.python.org/issue?@action=redirect&bpo=41486]: Fix a memory consumption and copying performance regression in earlier 3.10 beta releases if someone used an output buffer larger than 4GiB with zlib.decompress on input data that expands that large.

Núcleo e embutidos

- bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908]: Heap types with the `Py_TPFLAGS_IMMUTABLETYPE` flag can now inherit the **PEP 590** [https://peps.python.org/pep-0590/] vectorcall protocol. Previously, this was only possible for static types. Patch by Erlend E. Aasland.

- bpo-44553 [https://bugs.python.org/issue?@action=redirect&bpo=44553]: Implement GC methods for `types.Union` to break reference cycles and prevent memory leaks.

- bpo-44490 [https://bugs.python.org/issue?@action=redirect&bpo=44490]: Add `__parameters__` attribute and `__getitem__` operator to `types.Union`. Patch provided by Yurii Karabas.

- bpo-44523 [https://bugs.python.org/issue?@action=redirect&bpo=44523]: Remove the pass-through for `hash()` of `weakref.proxy` objects to prevent unintended consequences when the original referred object dies while the proxy is part of a hashable object. Patch by Pablo Galindo.

- bpo-44483 [https://bugs.python.org/issue?@action=redirect&bpo=44483]: Fix a crash in `types.Union` objects when creating a union of an object with bad `__module__` field.

- bpo-44486 [https://bugs.python.org/issue?@action=redirect&bpo=44486]: Modules will always have a dictionary, even when created by `types.ModuleType.__new__()`

- bpo-44472 [https://bugs.python.org/issue?@action=redirect&bpo=44472]: Fix ltrace functionality when exceptions are raised. Patch by Pablo Galindo

- bpo-12022 [https://bugs.python.org/issue?@action=redirect&bpo=12022]: A `TypeError` is now raised instead of an `AttributeError` in `with` and `async with` statements for objects which do not support the context manager or asynchronous context manager protocols correspondingly.

- bpo-44297 [https://bugs.python.org/issue?@action=redirect&bpo=44297]: Make sure that the line number is set when entering a comprehension scope. This ensures that backtraces including generator expressions show the correct line number.

- bpo-44456 [https://bugs.python.org/issue?@action=redirect&bpo=44456]: Improve the syntax error when mixing positional and keyword patterns. Patch by Pablo Galindo.

- bpo-44409 [https://bugs.python.org/issue?@action=redirect&bpo=44409]: Fix error location information for tokenizer errors raised on initialization of the tokenizer. Patch by Pablo Galindo.

- bpo-44396 [https://bugs.python.org/issue?@action=redirect&bpo=44396]: Fix a possible crash in the tokenizer when raising syntax errors for unclosed strings. Patch by Pablo Galindo.

- bpo-44376 [https://bugs.python.org/issue?@action=redirect&bpo=44376]: Exact integer exponentiation (like `i**2` or `pow(i, 2)`) with a small exponent is much faster, due to reducing overhead in such cases.

- bpo-44313 [https://bugs.python.org/issue?@action=redirect&bpo=44313]: Directly imported objects and modules (through import and from import statements) don't generate `LOAD_METHOD`/`CALL_METHOD` for directly accessed objects on their namespace. They now use the regular `LOAD_ATTR`/`CALL_FUNCTION`.

- bpo-44338 [https://bugs.python.org/issue?@action=redirect&bpo=44338]: Implement adaptive specialization for LOAD_GLOBAL

  Two specialized forms of LOAD_GLOBAL are added:

  - LOAD_GLOBAL_MODULE
  - LOAD_GLOBAL_BUILTIN

- bpo-44368 [https://bugs.python.org/issue?@action=redirect&bpo=44368]: Improve syntax errors for invalid "as" targets. Patch by Pablo Galindo

- bpo-44349 [https://bugs.python.org/issue?@action=redirect&bpo=44349]: Fix an edge case when displaying text from files with encoding in syntax errors. Patch by Pablo Galindo.

- bpo-44337 [https://bugs.python.org/issue?@action=redirect&bpo=44337]: Initial implementation of adaptive specialization of LOAD_ATTR

  Four specialized forms of LOAD_ATTR are added:

  - LOAD_ATTR_SLOT
  - LOAD_ATTR_SPLIT_KEYS
  - LOAD_ATTR_WITH_HINT
  - LOAD_ATTR_MODULE

- bpo-44335 [https://bugs.python.org/issue?@action=redirect&bpo=44335]: Fix a regression when identifying incorrect characters in syntax errors. Patch by Pablo Galindo

- bpo-43693 [https://bugs.python.org/issue?@action=redirect&bpo=43693]: Computation of the offsets of cell variables is done in the compiler instead of at runtime. This reduces the overhead of handling cell and free variables, especially in the case where a variable is both an argument and cell variable.

- bpo-44317 [https://bugs.python.org/issue?@action=redirect&bpo=44317]: Improve tokenizer error with improved locations. Patch by Pablo Galindo.

Biblioteca

- bpo-44304 [https://bugs.python.org/issue?@action=redirect&bpo=44304]: Fix a crash in the **`sqlite3`** module that happened when the garbage collector clears **`sqlite.Statement`** objects. Patch by Pablo Galindo

Núcleo e embutidos

- bpo-44305 [https://bugs.python.org/issue?@action=redirect&bpo=44305]: Improve error message for `try` blocks without `except` or `finally` blocks. Patch by Pablo Galindo.

- bpo-43413 [https://bugs.python.org/issue?@action=redirect&bpo=43413]: Constructors of subclasses of some builtin classes (e.g. **`tuple`**, **`list`**, **`frozenset`**) no longer accept arbitrary keyword arguments. [reverted in 3.11a4] Subclass of **`set`** can now define a `__new__()` method with additional keyword parameters without overriding also `__init__()`.

- bpo-43667 [https://bugs.python.org/issue?@action=redirect&bpo=43667]: Improve Unicode support in non-UTF locales on Oracle Solaris. This issue does not affect other Solaris systems.

- bpo-43693 [https://bugs.python.org/issue?@action=redirect&bpo=43693]: A new opcode MAKE_CELL has been added that effectively moves some of the work done on function entry into the compiler and into the eval loop. In addition to creating the required cell objects, the new opcode converts relevant arguments (and other locals) to cell variables on function entry.

- bpo-44232 [https://bugs.python.org/issue?@action=redirect&bpo=44232]: Fix a regression in **`type()`** when a metaclass raises an exception. The C function **`type_new()`** must properly report the exception when a metaclass constructor raises an exception and the winner class is not the metaclass. Patch by Victor Stinner.

- bpo-44201 [https://bugs.python.org/issue?@action=redirect&bpo=44201]: Avoid side effects of checking for specialized syntax errors in the REPL that was causing it to ask for extra tokens after a syntax error had been detected. Patch by Pablo Galindo

- bpo-43693 [https://bugs.python.org/issue?@action=redirect&bpo=43693]: `PyCodeObject` gained `co_fastlocalnames` and `co_fastlocalkinds` as the authoritative source of fast locals info. Marshaled code objects have changed accordingly.

- bpo-44184 [https://bugs.python.org/issue?@action=redirect&bpo=44184]: Fix a crash at Python exit when a deallocator function removes the last strong reference to a heap type. Patch by Victor Stinner.

- bpo-44187 [https://bugs.python.org/issue?@action=redirect&bpo=44187]: Implement quickening in the interpreter. This offers no advantages as yet, but is an enabler of future optimizations. See PEP 659 for full explanation.

- bpo-44180 [https://bugs.python.org/issue?@action=redirect&bpo=44180]: The parser doesn't report generic syntax errors that happen in a position further away that the one it reached in the first pass. Patch by Pablo Galindo

- bpo-44168 [https://bugs.python.org/issue?@action=redirect&bpo=44168]: Fix error message in the parser involving keyword arguments with invalid expressions. Patch by Pablo Galindo

- bpo-44156 [https://bugs.python.org/issue?@action=redirect&bpo=44156]: String caches in `compile.c` are now subinterpreter compatible.

- bpo-44143 [https://bugs.python.org/issue?@action=redirect&bpo=44143]: Fixed a crash in the parser that manifest when raising tokenizer errors when an existing exception was present. Patch by Pablo Galindo.

- bpo-44032 [https://bugs.python.org/issue?@action=redirect&bpo=44032]: Move ‘fast’ locals and other variables from the frame object to a per-thread datastack.

- bpo-44114 [https://bugs.python.org/issue?@action=redirect&bpo=44114]: Correção de assinaturas incorretas das funções dictkeys_reversed e dictitems_reversed no código C, que quebrou a construção do WebAssembly.

- bpo-44110 [https://bugs.python.org/issue?@action=redirect&bpo=44110]: Improve `str.__getitem__()` error message

- bpo-26110 [https://bugs.python.org/issue?@action=redirect&bpo=26110]: Add `CALL_METHOD_KW` opcode to speed up method calls with keyword arguments. Idea originated from PyPy. A side effect is executing `CALL_METHOD` is now branchless in the evaluation loop.

- bpo-28307 [https://bugs.python.org/issue?@action=redirect&bpo=28307]: Compiler now optimizes simple C-style formatting with literal format containing only format codes %s, %r and %a by converting them to f-string expressions.

- bpo-43149 [https://bugs.python.org/issue?@action=redirect&bpo=43149]: Correct the syntax error message regarding multiple exception types to not refer to “exception groups”. Patch by Pablo Galindo

- bpo-43822 [https://bugs.python.org/issue?@action=redirect&bpo=43822]: The parser will prioritize tokenizer errors over custom syntax errors when raising exceptions. Patch by Pablo Galindo.

- bpo-40222 [https://bugs.python.org/issue?@action=redirect&bpo=40222]: “Zero cost” exception handling.

    - Uses a lookup table to determine how to handle exceptions.
    - Removes SETUP_FINALLY and POP_TOP block instructions, eliminating the runtime overhead of try statements.
    - Reduces the size of the frame object by about 60%.

Patch by Mark Shannon

- bpo-43918 [https://bugs.python.org/issue?@action=redirect&bpo=43918]: Document the signature and `default` argument in the docstring of the new `anext` builtin.

- bpo-43833 [https://bugs.python.org/issue?@action=redirect&bpo=43833]: Emit a deprecation warning if the numeric literal is immediately followed by one of keywords: and, else, for, if, in, is, or. Raise a syntax error with more informative message if it is immediately followed by other keyword or identifier.

- bpo-43879 [https://bugs.python.org/issue?@action=redirect&bpo=43879]: Add native_thread_id to PyThreadState. Patch by Gabriele N. Tornetta.

- bpo-43693 [https://bugs.python.org/issue?@action=redirect&bpo=43693]: Compute cell offsets relative to locals in compiler. Allows the interpreter to treats locals and cells a single array, which is slightly more efficient. Also make the LOAD_CLOSURE opcode an alias for LOAD_FAST. Preserving LOAD_CLOSURE helps keep bytecode a bit more readable.

- bpo-17792 [https://bugs.python.org/issue?@action=redirect&bpo=17792]: More accurate error messages for access of unbound locals or free vars.

- bpo-28146 [https://bugs.python.org/issue?@action=redirect&bpo=28146]: Fix a confusing error message in **`str.format()`**.

- bpo-11105 [https://bugs.python.org/issue?@action=redirect&bpo=11105]: When compiling **`ast.AST`** objects with recursive references through **`compile()`**, the interpreter doesn't crash anymore instead it raises a **`RecursionError`**.

- bpo-39091 [https://bugs.python.org/issue?@action=redirect&bpo=39091]: Fix crash when using passing a non-exception to a generator's `throw()` method. Patch by Noah Oxer

- bpo-33346 [https://bugs.python.org/issue?@action=redirect&bpo=33346]: Asynchronous comprehensions are now allowed inside comprehensions in asynchronous functions. Outer comprehensions implicitly become asynchronous.

Biblioteca

- bpo-45371 [https://bugs.python.org/issue?@action=redirect&bpo=45371]: Fix clang rpath issue in `distutils`. The UnixCCompiler now uses correct clang option to add a runtime library directory (rpath) to a shared library.

- bpo-45329 [https://bugs.python.org/issue?@action=redirect&bpo=45329]: Fix freed memory access in **`pyexpat.xmlparser`** when building it with an installed expat library <= 2.2.0.

- bpo-41710 [https://bugs.python.org/issue?@action=redirect&bpo=41710]: On Unix, if the `sem_clockwait()` function is available in the C library (glibc 2.30 and newer), the **`threading.Lock.acquire()`** method now uses the monotonic clock (**`time.CLOCK_MONOTONIC`**) for the timeout, rather than using the system clock (**`time.CLOCK_REALTIME`**), to not be affected by system clock changes. Patch by Victor Stinner.

- bpo-1596321 [https://bugs.python.org/issue?@action=redirect&bpo=1596321]: Fix the **`threading._shutdown()`** function when the **`threading`** module was imported first from a thread different than the main thread: no longer log an error at Python exit.

- bpo-45274 [https://bugs.python.org/issue?@action=redirect&bpo=45274]: Fix a race condition in the **`Thread.join()`** method of the **`threading`** module. If the function is interrupted by a signal and the signal handler raises an exception, make sure that the thread remains in a consistent state to prevent a deadlock. Patch by Victor Stinner.

- bpo-21302 [https://bugs.python.org/issue?@action=redirect&bpo=21302]: In Unix operating systems, **`time.sleep()`** now uses the `nanosleep()` function, if `clock_nanosleep()` is not available but `nanosleep()` is available. `nanosleep()` allows to sleep with nanosecond precision.

- bpo-21302 [https://bugs.python.org/issue?@action=redirect&bpo=21302]: On Windows, `time.sleep()` now uses a waitable timer which has a resolution of 100 nanoseconds (10^{-7} seconds). Previously, it had a resolution of 1 millisecond (10^{-3} seconds). Patch by Benjamin Szőke and Victor Stinner.

- bpo-45238 [https://bugs.python.org/issue?@action=redirect&bpo=45238]: Fix **`unittest.IsolatedAsyncioTestCase.debug()`**: it runs now asynchronous methods and callbacks.

- bpo-36674 [https://bugs.python.org/issue?@action=redirect&bpo=36674]: `unittest.TestCase.debug()` raises now a `unittest.SkipTest` if the class or the test method are decorated with the skipping decorator.

- bpo-45235 [https://bugs.python.org/issue?@action=redirect&bpo=45235]: Fix an issue where argparse would not preserve values in a provided namespace when using a subparser with defaults.

- bpo-45183 [https://bugs.python.org/issue?@action=redirect&bpo=45183]: Have zipimport.zipimporter.find_spec() not raise an exception when the underlying zip file has been deleted and the internal cache has been reset via invalidate_cache().

- bpo-45234 [https://bugs.python.org/issue?@action=redirect&bpo=45234]: Fixed a regression in `copyfile()`, `copy()`, `copy2()` raising `FileNotFoundError` when source is a directory, which should raise `IsADirectoryError`

- bpo-45228 [https://bugs.python.org/issue?@action=redirect&bpo=45228]: Fix stack buffer overflow in parsing J1939 network address.

- bpo-45225 [https://bugs.python.org/issue?@action=redirect&bpo=45225]: use map function instead of genexpr in capwords.

- bpo-42135 [https://bugs.python.org/issue?@action=redirect&bpo=42135]: Fix typo: `importlib.find_loader` is really slated for removal in Python 3.12 not 3.10, like the others in PR 25169.

Patch por Hugo van Kemenade.

- bpo-20524 [https://bugs.python.org/issue?@action=redirect&bpo=20524]: Improves error messages on `.format()` operation for `str`, `float`, `int`, and `complex`. New format now shows the problematic pattern and the object type.

- bpo-45168 [https://bugs.python.org/issue?@action=redirect&bpo=45168]: Change **`dis.dis()`** output to omit op arg values that cannot be resolved due to `co_consts`, `co_names` etc not being provided. Previously the oparg itself was repeated in the value field, which is not useful and can be confusing.

- bpo-21302 [https://bugs.python.org/issue?@action=redirect&bpo=21302]: In Unix operating systems, **`time.sleep()`** now uses the `clock_nanosleep()` function, if available, which allows to sleep for an interval specified with nanosecond precision.

- bpo-45173 [https://bugs.python.org/issue?@action=redirect&bpo=45173]: Remove from the **`configparser`** module: the **`SafeConfigParser`** class, the **`filename`** property of the **`ParsingError`** class, the **`readfp()`** method of the **`ConfigParser`** class, deprecated since Python 3.2.

  Patch por Hugo van Kemenade.

- bpo-44987 [https://bugs.python.org/issue?@action=redirect&bpo=44987]: Pure ASCII strings are now normalized in constant time by **`unicodedata.normalize()`**. Patch by Donghee Na.

- bpo-35474 [https://bugs.python.org/issue?@action=redirect&bpo=35474]: Calling **`mimetypes.guess_all_extensions()`** with `strict=False` no longer affects the result of the following call with `strict=True`. Also, mutating the returned list no longer affects the global state.

- bpo-45166 [https://bugs.python.org/issue?@action=redirect&bpo=45166]: **`typing.get_type_hints()`** now works with **`Final`** wrapped in **`ForwardRef`**.

- bpo-45162 [https://bugs.python.org/issue?@action=redirect&bpo=45162]: Remove many old deprecated **`unittest`** features:

  - "`fail*`" and "`assert*`" aliases of **`TestCase`** methods.
  - Broken from start **`TestCase`** method `assertDictContainsSubset()`.
  - Ignored **`<unittest.TestLoader.loadTestsFromModule>`** **`TestLoader.loadTestsFromModule()`** parameter *use_load_tests*.
  - Old alias `_TextTestResult` of **`TextTestResult`**.

- bpo-38371 [https://bugs.python.org/issue?@action=redirect&bpo=38371]: Remove the deprecated `split()` method of **`_tkinter.TkappType`**. Patch by Erlend E. Aasland.

- bpo-20499 [https://bugs.python.org/issue?@action=redirect&bpo=20499]: Improve the speed and accuracy of statistics.pvariance().

- bpo-45132 [https://bugs.python.org/issue?@action=redirect&bpo=45132]: Remove **`__getitem__()`** methods of **`xml.dom.pulldom.DOMEventStream`**, **`wsgiref.util.FileWrapper`** and **`fileinput.FileInput`**, deprecated since Python 3.9.

  Patch por Hugo van Kemenade.

- bpo-45129 [https://bugs.python.org/issue?@action=redirect&bpo=45129]: Due to significant security concerns, the *reuse_address* parameter of **`asyncio.loop.create_datagram_endpoint()`**, disabled in Python 3.9, is now entirely removed. This is because of the behavior of the socket option `SO_REUSEADDR` in UDP.

  Patch por Hugo van Kemenade.

- bpo-45124 [https://bugs.python.org/issue?@action=redirect&bpo=45124]: The `bdist_msi` command, deprecated in Python 3.9, is now removed.

  Use `bdist_wheel` (wheel packages) instead.

  Patch por Hugo van Kemenade.

- bpo-30856 [https://bugs.python.org/issue?@action=redirect&bpo=30856]: **`unittest.TestResult`** methods **`addFailure()`**, **`addError()`**, **`addSkip()`** and **`addSubTest()`** are now called immediately after raising an exception in test or finishing a subtest. Previously they were called only after finishing the test clean up.

- bpo-45034 [https://bugs.python.org/issue?@action=redirect&bpo=45034]: Changes how error is formatted for `struct.pack` with `'H'` and `'h'` modes and too large / small numbers. Now it shows the actual numeric limits, while previously it was showing arithmetic expressions.

- bpo-25894 [https://bugs.python.org/issue?@action=redirect&bpo=25894]: **`unittest`** now always reports skipped and failed subtests separately: separate characters in default mode and separate lines in verbose mode. Also the test description is now output for errors in test method, class and module cleanups.

- bpo-45081 [https://bugs.python.org/issue?@action=redirect&bpo=45081]: Fix issue when dataclasses that inherit from `typing.Protocol` subclasses have wrong `__init__`. Patch provided by Yurii Karabas.

- bpo-45085 [https://bugs.python.org/issue?@action=redirect&bpo=45085]: The `binhex` module, deprecated in Python 3.9, is now removed. The following **`binascii`** functions, deprecated in Python 3.9, are now also removed:

  - `a2b_hqx()`, `b2a_hqx()`;
  - `rlecode_hqx()`, `rledecode_hqx()`.

  A função **`binascii.crc_hqx()`** continua disponível.

  Patch por Victor Stinner.

- bpo-40360 [https://bugs.python.org/issue?@action=redirect&bpo=40360]: The **`lib2to3`** package is now deprecated and may not be able to parse Python 3.10 or newer. See the **PEP 617** [https://peps.python.org/pep-0617/] (New PEG parser for CPython). Patch by Victor Stinner.

- bpo-45075 [https://bugs.python.org/issue?@action=redirect&bpo=45075]: Rename **`traceback.StackSummary.format_frame()`** to **`traceback.StackSummary.format_frame_summary()`**. This method was added for 3.11 so it was not released yet.

  Updated code and docs to better distinguish frame and FrameSummary.

- bpo-31299 [https://bugs.python.org/issue?@action=redirect&bpo=31299]: Add option to completely drop frames from a traceback by returning `None` from a **`format_frame()`** override.

- bpo-41620 [https://bugs.python.org/issue?@action=redirect&bpo=41620]: **`run()`** now always return a **`TestResult`** instance. Previously it returned `None` if the test class or method was decorated with a skipping decorator.

- bpo-45021 [https://bugs.python.org/issue?@action=redirect&bpo=45021]: Fix a potential deadlock at shutdown of forked children when using **`concurrent.futures`** module

- bpo-43913 [https://bugs.python.org/issue?@action=redirect&bpo=43913]: Fix bugs in cleaning up classes and modules in **`unittest`**:

  - Functions registered with **`addModuleCleanup()`** were not called unless the user defines `tearDownModule()` in their test module.
  - Functions registered with **`addClassCleanup()`** were not called if `tearDownClass` is set to `None`.
  - Buffering in **`TestResult`** did not work with functions registered with `addClassCleanup()` and `addModuleCleanup()`.
  - Errors in functions registered with `addClassCleanup()` and `addModuleCleanup()` were not handled correctly in buffered and debug modes.
  - Errors in `setUpModule()` and functions registered with `addModuleCleanup()` were reported in wrong order.
  - And several lesser bugs.

- bpo-45030 [https://bugs.python.org/issue?@action=redirect&bpo=45030]: Fix integer overflow in pickling and copying the range iterator.

- bpo-45001 [https://bugs.python.org/issue?@action=redirect&bpo=45001]: Made email date parsing more robust against malformed input, namely a whitespace-only `Date:` header. Patch by Wouter Bolsterlee.

- bpo-45010 [https://bugs.python.org/issue?@action=redirect&bpo=45010]: Remove support of special method `__div__` in **`unittest.mock`**. It is not used in Python 3.

- bpo-39218 [https://bugs.python.org/issue?@action=redirect&bpo=39218]: Improve accuracy of variance calculations by using `x*x` instead of `x**2`.

- bpo-43613 [https://bugs.python.org/issue?@action=redirect&bpo=43613]: Improve the speed of **`gzip.compress()`** and **`gzip.decompress()`** by compressing and decompressing at once in memory instead of in a streamed fashion.

- bpo-37596 [https://bugs.python.org/issue?@action=redirect&bpo=37596]: Ensure that **`set`** and **`frozenset`** objects are always **`marshalled`** reproducibly.

- bpo-44019 [https://bugs.python.org/issue?@action=redirect&bpo=44019]: A new function `operator.call` has been added, such that `operator.call(obj, *args, **kwargs) == obj(*args, **kwargs)`.

- bpo-42255 [https://bugs.python.org/issue?@action=redirect&bpo=42255]: **`webbrowser.MacOSX`** is deprecated and will be removed in Python 3.13. It is untested and undocumented and also not used by **`webbrowser`** itself. Patch by Donghee Na.

- bpo-44955 [https://bugs.python.org/issue?@action=redirect&bpo=44955]: Method **`stopTestRun()`** is now always called in pair with method **`startTestRun()`** for **`TestResult`** objects implicitly created in **`run()`**. Previously it was not called for test methods and classes decorated with a skipping decorator.

- bpo-39039 [https://bugs.python.org/issue?@action=redirect&bpo=39039]: tarfile.open raises **`ReadError`** when a zlib error occurs during file extraction.

- bpo-44935 [https://bugs.python.org/issue?@action=redirect&bpo=44935]: **`subprocess`** on Solaris now also uses **`os.posix_spawn()`** for better performance.

- bpo-44911 [https://bugs.python.org/issue?@action=redirect&bpo=44911]: **`IsolatedAsyncioTestCase`** will no longer throw an exception while cancelling leaked tasks. Patch by Bar Harel.

- bpo-41322 [https://bugs.python.org/issue?@action=redirect&bpo=41322]: Added `DeprecationWarning` for tests and async tests that return a value!=None (as this may indicate an improperly written test, for example a test written as a generator function).

- bpo-44524 [https://bugs.python.org/issue?@action=redirect&bpo=44524]: Make exception message more useful when subclass from typing special form alias. Patch provided by Yurii Karabas.

- bpo-38956 [https://bugs.python.org/issue?@action=redirect&bpo=38956]: **`argparse.BooleanOptionalAction`**'s default value is no longer printed twice when used with **`argparse.ArgumentDefaultsHelpFormatter`**.

- bpo-44860 [https://bugs.python.org/issue?@action=redirect&bpo=44860]: Fix the `posix_user` scheme in **`sysconfig`** to not depend on **`sys.platlibdir`**.

- bpo-44859 [https://bugs.python.org/issue?@action=redirect&bpo=44859]: Improve error handling in **`sqlite3`** and raise more accurate exceptions.

    - **`MemoryError`** is now raised instead of **`sqlite3.Warning`** when memory is not enough for encoding a statement to UTF-8 in `Connection.__call__()` and `Cursor.execute()`.
    - **`UnicodEncodeError`** is now raised instead of **`sqlite3.Warning`** when the statement contains surrogate characters in `Connection.__call__()` and `Cursor.execute()`.
    - **`TypeError`** is now raised instead of **`ValueError`** for non-string script argument in `Cursor.executescript()`.
    - **`ValueError`** is now raised for script containing the null character instead of truncating it in `Cursor.executescript()`.

  - Correctly handle exceptions raised when getting boolean value of the result of the progress handler.
  - Add many tests covering different corner cases.

- bpo-44581 [https://bugs.python.org/issue?@action=redirect&bpo=44581]: Upgrade bundled pip to 21.2.3 and setuptools to 57.4.0

- bpo-44849 [https://bugs.python.org/issue?@action=redirect&bpo=44849]: Fix the `os.set_inheritable()` function on FreeBSD 14 for file descriptor opened with the `O_PATH` flag: ignore the `EBADF` error on `ioctl()`, fallback on the `fcntl()` implementation. Patch by Victor Stinner.

- bpo-44605 [https://bugs.python.org/issue?@action=redirect&bpo=44605]: The @functools.total_ordering() decorator now works with metaclasses.

- bpo-44524 [https://bugs.python.org/issue?@action=redirect&bpo=44524]: Fixed an issue wherein the `__name__` and `__qualname__` attributes of subscribed specialforms could be `None`.

- bpo-44839 [https://bugs.python.org/issue?@action=redirect&bpo=44839]: `MemoryError` raised in user-defined functions will now produce a `MemoryError` in `sqlite3`. `OverflowError` will now be converted to `DataError`. Previously `OperationalError` was produced in these cases.

- bpo-44822 [https://bugs.python.org/issue?@action=redirect&bpo=44822]: `sqlite3` user-defined functions and aggregators returning `strings` with embedded NUL characters are no longer truncated. Patch by Erlend E. Aasland.

- bpo-44801 [https://bugs.python.org/issue?@action=redirect&bpo=44801]: Ensure that the `ParamSpec` variable in Callable can only be substituted with a parameters expression (a list of types, an ellipsis, ParamSpec or Concatenate).

- bpo-44806 [https://bugs.python.org/issue?@action=redirect&bpo=44806]: Non-protocol subclasses of `typing.Protocol` ignore now the `__init__` method inherited from protocol base classes.

- bpo-27275 [https://bugs.python.org/issue?@action=redirect&bpo=27275]: **`collections.OrderedDict.popitem()`** and **`collections.OrderedDict.pop()`** no longer call `__getitem__` and `__delitem__` methods of the OrderedDict subclasses.

- bpo-44793 [https://bugs.python.org/issue?@action=redirect&bpo=44793]: Fix checking the number of arguments when subscribe a generic type with `ParamSpec` parameter.

- bpo-44784 [https://bugs.python.org/issue?@action=redirect&bpo=44784]: In importlib.metadata tests, override warnings behavior under expected DeprecationWarnings (importlib_metadata 4.6.3).

- bpo-44667 [https://bugs.python.org/issue?@action=redirect&bpo=44667]: The **`tokenize.tokenize()`** doesn't incorrectly generate a `NEWLINE` token if the source doesn't end with a new line character but the last line is a comment, as the function is already generating a `NL` token. Patch by Pablo Galindo

- bpo-44771 [https://bugs.python.org/issue?@action=redirect&bpo=44771]: Added `importlib.simple` module implementing adapters from a low-level resources reader interface to a `TraversableResources` interface. Legacy API (`path`, `contents`, ...) is now supported entirely by the `.files()` API with a compatibility shim supplied for resource loaders without that functionality. Feature parity with `importlib_resources` 5.2.

- bpo-44752 [https://bugs.python.org/issue?@action=redirect&bpo=44752]: **`rcompleter`** does not call **`getattr()`** on **`property`** objects to avoid the side-effect of evaluating the corresponding method.

- bpo-44747 [https://bugs.python.org/issue?@action=redirect&bpo=44747]: Refactor usage of `sys._getframe` in `typing` module. Patch provided by Yurii Karabas.

- bpo-42378 [https://bugs.python.org/issue?@action=redirect&bpo=42378]: Fixes the issue with log file being overwritten when **`logging.FileHandler`** is used in **`atexit`** with *filemode* set to `'w'`. Note this will cause the

message in *atexit* not being logged if the log stream is already closed due to shutdown of logging.

- bpo-44720 [https://bugs.python.org/issue?@action=redirect&bpo=44720]: `weakref.proxy` objects referencing non-iterators now raise `TypeError` rather than dereferencing the null `tp_iternext` slot and crashing.

- bpo-44704 [https://bugs.python.org/issue?@action=redirect&bpo=44704]: The implementation of `collections.abc.Set._hash()` now matches that of `frozenset.__hash__()`.

- bpo-44666 [https://bugs.python.org/issue?@action=redirect&bpo=44666]: Fixed issue in **`compileall.compile_file()`** when `sys.stdout` is redirected. Patch by Stefan Hölzl.

- bpo-44688 [https://bugs.python.org/issue?@action=redirect&bpo=44688]: **`sqlite3.Connection.create_collation()`** now accepts non-ASCII collation names. Patch by Erlend E. Aasland.

- bpo-44690 [https://bugs.python.org/issue?@action=redirect&bpo=44690]: Adopt *binacii.a2b_base64*'s strict mode in *base64.b64decode*.

- bpo-42854 [https://bugs.python.org/issue?@action=redirect&bpo=42854]: Fixed a bug in the **`_ssl`** module that was throwing **`OverflowError`** when using **`_ssl._SSLSocket.write()`** and **`_ssl._SSLSocket.read()`** for a big value of the `len` parameter. Patch by Pablo Galindo.

- bpo-44686 [https://bugs.python.org/issue?@action=redirect&bpo=44686]: Replace `unittest.mock._importer` with `pkgutil.resolve_name`.

- bpo-44353 [https://bugs.python.org/issue?@action=redirect&bpo=44353]: Make `NewType.__call__` faster by implementing it in C. Patch provided by Yurii Karabas.

- bpo-44682 [https://bugs.python.org/issue?@action=redirect&bpo=44682]: Change the **`pdb`** *commands* directive to disallow setting commands for an invalid breakpoint and to display an appropriate error.

- bpo-44353 [https://bugs.python.org/issue?@action=redirect&bpo=44353]: Refactor `typing.NewType` from function into callable class. Patch provided by Yurii Karabas.

- bpo-44678 [https://bugs.python.org/issue?@action=redirect&bpo=44678]: Added a separate error message for discontinuous padding in *binascii.a2b_base64* strict mode.

- bpo-44524 [https://bugs.python.org/issue?@action=redirect&bpo=44524]: Add missing `__name__` and `__qualname__` attributes to `typing` module classes. Patch provided by Yurii Karabas.

- bpo-40897 [https://bugs.python.org/issue?@action=redirect&bpo=40897]: Give priority to using the current class constructor in **`inspect.signature()`**. Patch by Weipeng Hong.

- bpo-44638 [https://bugs.python.org/issue?@action=redirect&bpo=44638]: Add a reference to the zipp project and hint as to how to use it.

- bpo-44648 [https://bugs.python.org/issue?@action=redirect&bpo=44648]: Fixed wrong error being thrown by **`inspect.getsource()`** when examining a class in the interactive session. Instead of **`TypeError`**, it should be **`OSError`** with appropriate error message.

- bpo-44608 [https://bugs.python.org/issue?@action=redirect&bpo=44608]: Fix memory leak in **`_tkinter._flatten()`** if it is called with a sequence or set, but not list or tuple.

- bpo-44594 [https://bugs.python.org/issue?@action=redirect&bpo=44594]: Fix an edge case of **`ExitStack`** and **`AsyncExitStack`** exception chaining. They will now match `with` block behavior when `__context__` is explicitly set to `None` when the exception is in flight.

- bpo-42799 [https://bugs.python.org/issue?@action=redirect&bpo=42799]: In **`fnmatch`**, the cache size for compiled regex patterns (**`functools.lru_cache()`**) was bumped up from 256 to 32768, affecting functions: **`fnmatch.fnmatch()`**, **`fnmatch.fnmatchcase()`**, **`fnmatch.filter()`**.

- bpo-41928 [https://bugs.python.org/issue?@action=redirect&bpo=41928]: Update **`shutil.copyfile()`** to raise **`FileNotFoundError`** instead of confusing **`IsADirectoryError`** when a path ending with a **`os.path.sep`** does not exist; **`shutil.copy()`** and **`shutil.copy2()`** are also affected.

- bpo-44569 [https://bugs.python.org/issue?@action=redirect&bpo=44569]: Added the **`StackSummary.format_frame()`** function in **`traceback`**. This allows users to customize the way individual lines are formatted in tracebacks without re-implementing logic to handle recursive tracebacks.

- bpo-44566 [https://bugs.python.org/issue?@action=redirect&bpo=44566]: handle StopIteration subclass raised from @contextlib.contextmanager generator

- bpo-44558 [https://bugs.python.org/issue?@action=redirect&bpo=44558]: Make the implementation consistency of **`indexOf()`** between C and Python versions. Patch by Donghee Na.

- bpo-41249 [https://bugs.python.org/issue?@action=redirect&bpo=41249]: Fixes `TypedDict` to work with `typing.get_type_hints()` and postponed evaluation of annotations across modules.

- bpo-44554 [https://bugs.python.org/issue?@action=redirect&bpo=44554]: Refactor argument processing in **`pdb.main()`** to simplify detection of errors in input loading and clarify behavior around module or script invocation.

- bpo-34798 [https://bugs.python.org/issue?@action=redirect&bpo=34798]: Break up paragraph about **`pprint.PrettyPrinter`** construction parameters to make it easier to read.

- bpo-44539 [https://bugs.python.org/issue?@action=redirect&bpo=44539]: Added support for recognizing JPEG files without JFIF or Exif markers.

- bpo-44461 [https://bugs.python.org/issue?@action=redirect&bpo=44461]: Fix bug with **`pdb`**'s handling of import error due to a package which does not have a `__main__` module

- bpo-43625 [https://bugs.python.org/issue?@action=redirect&bpo=43625]: Fix a bug in the detection of CSV file headers by `csv.Sniffer.has_header()` and improve documentation of same.

- bpo-44516 [https://bugs.python.org/issue?@action=redirect&bpo=44516]: Update vendored pip to 21.1.3

- bpo-42892 [https://bugs.python.org/issue?@action=redirect&bpo=42892]: Fixed an exception thrown while parsing a malformed multipart email by `email.message.EmailMessage`.

- bpo-44468 [https://bugs.python.org/issue?@action=redirect&bpo=44468]: `typing.get_type_hints()` now finds annotations in classes and base classes with unexpected `__module__`. Previously, it skipped those MRO elements.

- bpo-44491 [https://bugs.python.org/issue?@action=redirect&bpo=44491]: Allow clearing the `sqlite3` authorizer callback by passing `None` to `set_authorizer()`. Patch by Erlend E. Aasland.

- bpo-43977 [https://bugs.python.org/issue?@action=redirect&bpo=43977]: Set the proper `Py_TPFLAGS_MAPPING` and `Py_TPFLAGS_SEQUENCE` flags for subclasses created before a parent has been registered as a `collections.abc.Mapping` or `collections.abc.Sequence`.

- bpo-44482 [https://bugs.python.org/issue?@action=redirect&bpo=44482]: Fix very unlikely resource leak in `glob` in alternate Python implementations.

- bpo-44466 [https://bugs.python.org/issue?@action=redirect&bpo=44466]: The `faulthandler` module now detects if a fatal error occurs during a garbage collector collection. Patch by Victor Stinner.

- bpo-44471 [https://bugs.python.org/issue?@action=redirect&bpo=44471]: A `TypeError` is now raised instead of an `AttributeError` in `contextlib.ExitStack.enter_context()` and `contextlib.AsyncExitStack.enter_async_context()` for objects which do not support the context manager or asynchronous context manager protocols correspondingly.

- bpo-44404 [https://bugs.python.org/issue?@action=redirect&bpo=44404]: **`tkinter`**'s `after()` method now supports callables without the `__name__` attribute.

- bpo-41546 [https://bugs.python.org/issue?@action=redirect&bpo=41546]: Make **`pprint`** (like the builtin `print`) not attempt to write to `stdout` when it is `None`.

- bpo-44458 [https://bugs.python.org/issue?@action=redirect&bpo=44458]: `BUFFER_BLOCK_SIZE` is now declared static, to avoid linking collisions when bz2, lmza or zlib are statically linked.

- bpo-44464 [https://bugs.python.org/issue?@action=redirect&bpo=44464]: Remove exception for flake8 in deprecated importlib.metadata interfaces. Sync with importlib_metadata 4.6.

- bpo-44446 [https://bugs.python.org/issue?@action=redirect&bpo=44446]: Take into account that `lineno` might be `None` in **`traceback.FrameSummary`**.

- bpo-44439 [https://bugs.python.org/issue?@action=redirect&bpo=44439]: Fix in **`bz2.BZ2File.write()`** / **`lzma.LZMAFile.write()`** methods, when the input data is an object that supports the buffer protocol, the file length may be wrong.

- bpo-44434 [https://bugs.python.org/issue?@action=redirect&bpo=44434]: _thread.start_new_thread() no longer calls PyThread_exit_thread() explicitly at the thread exit, the call was redundant. On Linux with the glibc, pthread_exit() aborts the whole process if dlopen() fails to open libgcc_s.so file (ex: EMFILE error). Patch by Victor Stinner.

- bpo-42972 [https://bugs.python.org/issue?@action=redirect&bpo=42972]: The _thread.RLock type now fully implement the GC protocol: add a traverse function and the **`Py_TPFLAGS_HAVE_GC`** flag. Patch by Victor Stinner.

- bpo-44422 [https://bugs.python.org/issue?@action=redirect&bpo=44422]: The **`threading.enumerate()`** function now uses a reentrant lock to prevent a hang on reentrant call. Patch by Victor Stinner.

- bpo-38291 [https://bugs.python.org/issue?@action=redirect&bpo=38291]: Importing typing.io or typing.re now prints a `DeprecationWarning`.

- bpo-37880 [https://bugs.python.org/issue?@action=redirect&bpo=37880]: argparse actions store_const and append_const each receive a default value of `None` when the `const` kwarg is not provided. Previously, this raised a **`TypeError`**.

- bpo-44389 [https://bugs.python.org/issue?@action=redirect&bpo=44389]: Fix deprecation of **`ssl.OP_NO_TLSv1_3`**

- bpo-27827 [https://bugs.python.org/issue?@action=redirect&bpo=27827]: **`pathlib.PureWindowsPath.is_reserved()`** now identifies a greater range of reserved filenames, including those with trailing spaces or colons.

- bpo-44395 [https://bugs.python.org/issue?@action=redirect&bpo=44395]: Fix **`as_string()`** to pass unixfrom properly. Patch by Donghee Na.

- bpo-34266 [https://bugs.python.org/issue?@action=redirect&bpo=34266]: Handle exceptions from parsing the arg of **`pdb`**'s run/restart command.

- bpo-44362 [https://bugs.python.org/issue?@action=redirect&bpo=44362]: Improve **`ssl`** module's deprecation messages, error reporting, and documentation for deprecations.

- bpo-44342 [https://bugs.python.org/issue?@action=redirect&bpo=44342]: [Enum] Change pickling from by-value to by-name.

- bpo-44356 [https://bugs.python.org/issue?@action=redirect&bpo=44356]: [Enum] Allow multiple data-type mixins if they are all the same.

- bpo-44351 [https://bugs.python.org/issue?@action=redirect&bpo=44351]: Restore back **`parse_makefile()`** in `distutils.sysconfig` because it behaves differently than the similar implementation in **`sysconfig`**.

- bpo-35800 [https://bugs.python.org/issue?@action=redirect&bpo=35800]: **`smtpd.MailmanProxy`** is now removed as it is unusable without an

external module, `mailman`. Patch by Donghee Na.

- bpo-44357 [https://bugs.python.org/issue?@action=redirect&bpo=44357]: Added a function that returns cube root of the given number **`math.cbrt()`**

- bpo-44339 [https://bugs.python.org/issue?@action=redirect&bpo=44339]: Change `math.pow(±0.0, -math.inf)` to return `inf` instead of raising `ValueError`. This brings the special-case handling of `math.pow` into compliance with the IEEE 754 standard.

- bpo-44242 [https://bugs.python.org/issue?@action=redirect&bpo=44242]: Remove missing flag check from Enum creation and move into a `verify` decorator.

- bpo-44246 [https://bugs.python.org/issue?@action=redirect&bpo=44246]: In `importlib.metadata`, restore compatibility in the result from `Distribution.entry_points` (`EntryPoints`) to honor expectations in older implementations and issuing deprecation warnings for these cases: A. `EntryPoints` objects are once again mutable, allowing for `sort()` and other list-based mutation operations. Avoid deprecation warnings by casting to a mutable sequence (e.g. `list(dist.entry_points).sort()`). B. `EntryPoints` results once again allow for access by index. To avoid deprecation warnings, cast the result to a Sequence first (e.g. `tuple(dist.entry_points)[0]`).

- bpo-44246 [https://bugs.python.org/issue?@action=redirect&bpo=44246]: In importlib.metadata.entry_points, de-duplication of distributions no longer requires loading the full metadata for PathDistribution objects, improving entry point loading performance by ~10x.

- bpo-43858 [https://bugs.python.org/issue?@action=redirect&bpo=43858]: Added a function that returns a copy of a dict of logging levels: **`logging.getLevelNamesMapping()`**

- bpo-44260 [https://bugs.python.org/issue?@action=redirect&bpo=44260]: The **`random.Random`** constructor no longer reads system entropy without need.

- bpo-44254 [https://bugs.python.org/issue?@action=redirect&bpo=44254]: On Mac, give turtledemo button text a color that works on both light or dark background. Programmers cannot control the latter.

- bpo-44258 [https://bugs.python.org/issue?@action=redirect&bpo=44258]: Support PEP 515 for Fraction's initialization from string.

- bpo-44235 [https://bugs.python.org/issue?@action=redirect&bpo=44235]: Remove deprecated functions in the **`gettext`**. Patch by Donghee Na.

- bpo-38693 [https://bugs.python.org/issue?@action=redirect&bpo=38693]: Prefer f-strings to `.format` in importlib.resources.

- bpo-33693 [https://bugs.python.org/issue?@action=redirect&bpo=33693]: Importlib.metadata now prefers f-strings to .format.

- bpo-44241 [https://bugs.python.org/issue?@action=redirect&bpo=44241]: Incorporate minor tweaks from importlib_metadata 4.1: SimplePath protocol, support for Metadata 2.2.

- bpo-43216 [https://bugs.python.org/issue?@action=redirect&bpo=43216]: Remove the **`@asyncio.coroutine`** decorator enabling legacy generator-based coroutines to be compatible with async/await code; remove **`asyncio.coroutines.CoroWrapper`** used for wrapping legacy coroutine objects in the debug mode. The decorator has been deprecated since Python 3.8 and the removal was initially scheduled for Python 3.10. Patch by Illia Volochii.

- bpo-44210 [https://bugs.python.org/issue?@action=redirect&bpo=44210]: Make importlib.metadata._meta.PackageMetadata public.

- bpo-43643 [https://bugs.python.org/issue?@action=redirect&bpo=43643]: Declare readers.MultiplexedPath.name as a property per the spec.

- bpo-27334 [https://bugs.python.org/issue?@action=redirect&bpo=27334]: The **`sqlite3`** context manager now performs a rollback (thus releasing the database lock) if commit failed. Patch by Luca Citi and Erlend E. Aasland.

- bpo-4928 [https://bugs.python.org/issue?@action=redirect&bpo=4928]: Documented existing behavior on POSIX: NamedTemporaryFiles are not deleted when creating process is killed with SIGKILL

- bpo-44154 [https://bugs.python.org/issue?@action=redirect&bpo=44154]: Optimize `fractions.Fraction` pickling for large components.

- bpo-33433 [https://bugs.python.org/issue?@action=redirect&bpo=33433]: For IPv4 mapped IPv6 addresses (**RFC 4291** [https://datatracker.ietf.org/doc/html/rfc4291.html] Section 2.5.5.2), the `ipaddress.IPv6Address.is_private` check is deferred to the mapped IPv4 address. This solves a bug where public mapped IPv4 addresses were considered private by the IPv6 check.

- bpo-44150 [https://bugs.python.org/issue?@action=redirect&bpo=44150]: Add optional *weights* argument to statistics.fmean().

- bpo-44142 [https://bugs.python.org/issue?@action=redirect&bpo=44142]: `ast.unparse()` will now drop the redundant parentheses when tuples used as assignment targets (e.g in for loops).

- bpo-44145 [https://bugs.python.org/issue?@action=redirect&bpo=44145]: `hmac` computations were not releasing the GIL while calling the OpenSSL `HMAC_Update` C API (a new feature in 3.9). This unintentionally prevented parallel computation as other `hashlib` algorithms support.

- bpo-44095 [https://bugs.python.org/issue?@action=redirect&bpo=44095]: `zipfile.Path` now supports `zipfile.Path.stem`, `zipfile.Path.suffixes`, and `zipfile.Path.suffix` attributes.

- bpo-44077 [https://bugs.python.org/issue?@action=redirect&bpo=44077]: It's now possible to receive the type of service (ToS), a.k.a. differentiated services (DS), a.k.a. differentiated services code point (DSCP) and explicit congestion notification (ECN) IP header fields with `socket.IP_RECVTOS`.

- bpo-37788 [https://bugs.python.org/issue?@action=redirect&bpo=37788]: Fix a reference leak when a Thread object is never joined.

- bpo-38908 [https://bugs.python.org/issue?@action=redirect&bpo=38908]: Subclasses of `typing.Protocol` which only have data variables declared will now raise a `TypeError` when checked with `isinstance` unless they are decorated with **`runtime_checkable()`**. Previously, these checks passed silently. Patch provided by Yurii Karabas.

- bpo-44098 [https://bugs.python.org/issue?@action=redirect&bpo=44098]: `typing.ParamSpec` will no longer be found in the `__parameters__` of most **`typing`** generics except in valid use locations specified by **PEP 612** [https://peps.python.org/pep-0612/]. This prevents incorrect usage like `typing.List[P][int]`. This change means incorrect usage which may have passed silently in 3.10 beta 1 and earlier will now error.

- bpo-44089 [https://bugs.python.org/issue?@action=redirect&bpo=44089]: Allow subclassing `csv.Error` in 3.10 (it was allowed in 3.9 and earlier but was disallowed in early versions of 3.10).

- bpo-44081 [https://bugs.python.org/issue?@action=redirect&bpo=44081]: **`ast.unparse()`** now doesn't use redundant spaces to separate `lambda` and the `:` if there are no parameters.

- bpo-44061 [https://bugs.python.org/issue?@action=redirect&bpo=44061]: Fix regression in previous release when calling **`pkgutil.iter_modules()`** with a list of **`pathlib.Path`** objects

- bpo-44059 [https://bugs.python.org/issue?@action=redirect&bpo=44059]: Register the SerenityOS Browser in the **`webbrowser`** module.

- bpo-36515 [https://bugs.python.org/issue?@action=redirect&bpo=36515]: The **`hashlib`** module no longer does unaligned memory accesses when compiled for ARM platforms.

- bpo-40465 [https://bugs.python.org/issue?@action=redirect&bpo=40465]: Remove random module features deprecated in Python 3.9.

- bpo-44018 [https://bugs.python.org/issue?@action=redirect&bpo=44018]: random.seed() no longer mutates bytearray inputs.

- bpo-38352 [https://bugs.python.org/issue?@action=redirect&bpo=38352]: Add `IO`, `BinaryIO`, `TextIO`, `Match`, and `Pattern` to `typing.__all__`. Patch by Jelle Zijlstra.

- bpo-44002 [https://bugs.python.org/issue?@action=redirect&bpo=44002]: **`urllib.parse`** now uses **`functool.lru_cache()`** for its internal URL splitting and quoting caches instead of rolling its own like its the '90s.

  The undocumented internal **`urllib.parse`** `Quoted` class API is now deprecated, for removal in 3.14.

- bpo-43972 [https://bugs.python.org/issue?@action=redirect&bpo=43972]: When **`http.server.SimpleHTTPRequestHandler`** sends a `301 (Moved Permanently)` for a directory path not ending with `/`, add a `Content-Length: 0` header. This improves the behavior for certain clients.

- bpo-28528 [https://bugs.python.org/issue?@action=redirect&bpo=28528]: Fix a bug in **`pdb`** where **`checkline()`** raises **`AttributeError`** if it is called after **`reset()`**.

- bpo-43853 [https://bugs.python.org/issue?@action=redirect&bpo=43853]: Improved string handling for **`sqlite3`** user-defined functions and aggregates:

  - It is now possible to pass strings with embedded null characters to UDFs
  - Conversion failures now correctly raise **`MemoryError`**

  Patch por Erlend E. Aasland.

- bpo-43666 [https://bugs.python.org/issue?@action=redirect&bpo=43666]: AIX: `Lib/_aix_support.get_platform()` may fail in an AIX WPAR. The fileset bos.rte appears to have a builddate in both LPAR and WPAR so this fileset is queried rather than bos.mp64. To prevent a similar situation (no builddate in ODM) a value (9988) sufficient for completing a build is provided. Patch by M Felt.

- bpo-43650 [https://bugs.python.org/issue?@action=redirect&bpo=43650]: Fix **`MemoryError`** in **`shutil.unpack_archive()`** which fails inside

shutil._unpack_zipfile() on large files. Patch by Igor Bolshakov.

- bpo-43612 [https://bugs.python.org/issue?@action=redirect&bpo=43612]: **zlib.compress()** now accepts a wbits parameter which allows users to compress data as a raw deflate block without zlib headers and trailers in one go. Previously this required instantiating a `zlib.compressobj`. It also provides a faster alternative to `gzip.compress` when wbits=31 is used.

- bpo-43392 [https://bugs.python.org/issue?@action=redirect&bpo=43392]: **importlib._bootstrap._find_and_load()** now implements a two-step check to avoid locking when modules have been already imported and are ready. This improves performance of repeated calls to **importlib.import_module()** and **importlib.__import__()**.

- bpo-43318 [https://bugs.python.org/issue?@action=redirect&bpo=43318]: Fix a bug where **pdb** does not always echo cleared breakpoints.

- bpo-43234 [https://bugs.python.org/issue?@action=redirect&bpo=43234]: Prohibit passing non-**concurrent.futures.ThreadPoolExecutor** executors to **loop.set_default_executor()** following a deprecation in Python 3.8. Patch by Illia Volochii.

- bpo-43232 [https://bugs.python.org/issue?@action=redirect&bpo=43232]: Prohibit previously deprecated potentially disruptive operations on **asyncio.trsock.TransportSocket**. Patch by Illia Volochii.

- bpo-30077 [https://bugs.python.org/issue?@action=redirect&bpo=30077]: Added support for Apple's aifc/sowt pseudo-compression

- bpo-42971 [https://bugs.python.org/issue?@action=redirect&bpo=42971]: Add definition of `errno.EQFULL` for platforms that define this constant (such as macOS).

- bpo-43086 [https://bugs.python.org/issue?@action=redirect&bpo=43086]: Added a new optional `strict_mode` parameter to *binascii.a2b_base64*. When `scrict_mode` is set to `True`, the *a2b_base64* function will accept only

valid base64 content. More details about what “valid base64 content” is, can be found in the function’s documentation.

- bpo-43024 [https://bugs.python.org/issue?@action=redirect&bpo=43024]: Improve the help signature of **`traceback.print_exception()`**, **`traceback.format_exception()`** and **`traceback.format_exception_only()`**.

- bpo-33809 [https://bugs.python.org/issue?@action=redirect&bpo=33809]: Add the **`traceback.TracebackException.print()`** method which prints the formatted exception information.

- bpo-42862 [https://bugs.python.org/issue?@action=redirect&bpo=42862]: **`sqlite3`** now utilizes **`functools.lru_cache()`** to implement the connection statement cache. As a small optimisation, the default statement cache size has been increased from 100 to 128. Patch by Erlend E. Aasland.

- bpo-41818 [https://bugs.python.org/issue?@action=redirect&bpo=41818]: Soumendra Ganguly: add termios.tcgetwinsize(), termios.tcsetwinsize().

- bpo-40497 [https://bugs.python.org/issue?@action=redirect&bpo=40497]: **`subprocess.check_output()`** now raises **`ValueError`** when the invalid keyword argument *check* is passed by user code. Previously such use would fail later with a **`TypeError`**. Patch by Rémi Lapeyre.

- bpo-37449 [https://bugs.python.org/issue?@action=redirect&bpo=37449]: `ensurepip` now uses `importlib.resources.files()` traversable APIs

- bpo-40956 [https://bugs.python.org/issue?@action=redirect&bpo=40956]: Use Argument Clinic in **`sqlite3`**. Patches by Erlend E. Aasland.

- bpo-41730 [https://bugs.python.org/issue?@action=redirect&bpo=41730]: `DeprecationWarning` is now raised when importing **`tkinter.tix`**, which has been deprecated in documentation since Python 3.6.

- bpo-20684 [https://bugs.python.org/issue?@action=redirect&bpo=20684]: Remove unused `_signature_get_bound_param` function from **`inspect`** - by Anthony Sottile.

- bpo-41402 [https://bugs.python.org/issue?@action=redirect&bpo=41402]: Fix `email.message.EmailMessage.set_content()` when called with binary data and `7bit` content transfer encoding.

- bpo-32695 [https://bugs.python.org/issue?@action=redirect&bpo=32695]: The *compresslevel* and *preset* keyword arguments of `tarfile.open()` are now both documented and tested.

- bpo-41137 [https://bugs.python.org/issue?@action=redirect&bpo=41137]: Use utf-8 encoding while reading .pdbrc files. Patch by Srinivas Reddy Thatiparthy

- bpo-24391 [https://bugs.python.org/issue?@action=redirect&bpo=24391]: Improved reprs of `threading` synchronization objects: `Semaphore`, `BoundedSemaphore`, `Event` and `Barrier`.

- bpo-5846 [https://bugs.python.org/issue?@action=redirect&bpo=5846]: Deprecated the following `unittest` functions, scheduled for removal in Python 3.13:

  - `findTestCases()`
  - `makeSuite()`
  - `getTestCaseNames()`

  Em vez delas, use os métodos de `TestLoader`:

  - `unittest.TestLoader.loadTestsFromModule()`
  - `unittest.TestLoader.loadTestsFromTestCase()`
  - `unittest.TestLoader.getTestCaseNames()`

  Patch por Erlend E. Aasland.

- bpo-40563 [https://bugs.python.org/issue?@action=redirect&bpo=40563]: Support pathlike objects on dbm/shelve. Patch by Hakan Çelik and Henry-Joseph Audéoud.

- bpo-34990 [https://bugs.python.org/issue?@action=redirect&bpo=34990]: Fixed a Y2k38 bug in the compileall module where it would fail to compile

files with a modification time after the year 2038.

- bpo-39549 [https://bugs.python.org/issue?@action=redirect&bpo=39549]: Whereas the code for reprlib.Repr had previously used a hardcoded string value of '…', this PR updates it to use of a "fillvalue" attribute, whose value defaults to '…' and can be reset in either individual reprlib.Repr instances or in subclasses thereof.

- bpo-37022 [https://bugs.python.org/issue?@action=redirect&bpo=37022]: **`pdb`** now displays exceptions from `repr()` with its `p` and `pp` commands.

- bpo-38840 [https://bugs.python.org/issue?@action=redirect&bpo=38840]: Fix `test___all__` on platforms lacking a shared memory implementation.

- bpo-39359 [https://bugs.python.org/issue?@action=redirect&bpo=39359]: Add one missing check that the password is a bytes object for an encrypted zipfile.

- bpo-38741 [https://bugs.python.org/issue?@action=redirect&bpo=38741]: **`configparser`**: using ']' inside a section header will no longer cut the section name short at the ']'

- bpo-38415 [https://bugs.python.org/issue?@action=redirect&bpo=38415]: Added missing behavior to **`contextlib.asynccontextmanager()`** to match **`contextlib.contextmanager()`** so decorated functions can themselves be decorators.

- bpo-30256 [https://bugs.python.org/issue?@action=redirect&bpo=30256]: Pass multiprocessing BaseProxy argument `manager_owned` through AutoProxy.

- bpo-27513 [https://bugs.python.org/issue?@action=redirect&bpo=27513]: **`email.utils.getaddresses()`** now accepts **`email.header.Header`** objects along with string values. Patch by Zackery Spytz.

- bpo-16379 [https://bugs.python.org/issue?@action=redirect&bpo=16379]: Add SQLite error code and name to **`sqlite3`** exceptions. Patch by Aviv Palivoda, Daniel Shahaf, and Erlend E. Aasland.

- bpo-26228 [https://bugs.python.org/issue?@action=redirect&bpo=26228]: pty.spawn no longer hangs on FreeBSD, macOS, and Solaris.

- bpo-33349 [https://bugs.python.org/issue?@action=redirect&bpo=33349]: lib2to3 now recognizes async generators everywhere.

- bpo-29298 [https://bugs.python.org/issue?@action=redirect&bpo=29298]: Fix `TypeError` when required subparsers without `dest` do not receive arguments. Patch by Anthony Sottile.

Documentação

- bpo-45216 [https://bugs.python.org/issue?@action=redirect&bpo=45216]: Remove extra documentation listing methods in `difflib`. It was rendering twice in pydoc and was outdated in some places.
- bpo-45024 [https://bugs.python.org/issue?@action=redirect&bpo=45024]: **`collections.abc`** documentation has been expanded to explicitly cover how instance and subclass checks work, with additional doctest examples and an exhaustive list of ABCs which test membership purely by presence of the right special methods. Patch by Raymond Hettinger.
- bpo-44957 [https://bugs.python.org/issue?@action=redirect&bpo=44957]: Promote PEP 604 union syntax by using it where possible. Also, mention `X | Y` more prominently in section about `Union` and mention `X | None` at all in section about `Optional`.
- bpo-16580 [https://bugs.python.org/issue?@action=redirect&bpo=16580]: Added code equivalents for the **`int.to_bytes()`** and **`int.from_bytes()`** methods, as well as tests ensuring that these code equivalents are valid.
- bpo-44903 [https://bugs.python.org/issue?@action=redirect&bpo=44903]: Removed the `othergui.rst` file, any references to it, and the list of GUI frameworks in the FAQ. In their place I've added links to the Python Wiki page on GUI frameworks [https://wiki.python.org/moin/GuiProgramming].
- bpo-33479 [https://bugs.python.org/issue?@action=redirect&bpo=33479]: Tkinter documentation has been greatly expanded with new "Architecture" and "Threading model" sections.
- bpo-36700 [https://bugs.python.org/issue?@action=redirect&bpo=36700]: **`base64`** RFC references were updated to point to **RFC 4648**

[https://datatracker.ietf.org/doc/html/rfc4648.html]; a section was added to point users to the new “security considerations” section of the RFC.

- bpo-44740 [https://bugs.python.org/issue?@action=redirect&bpo=44740]: Replaced occurrences of uppercase “Web” and “Internet” with lowercase versions per the 2016 revised Associated Press Style Book.
- bpo-44693 [https://bugs.python.org/issue?@action=redirect&bpo=44693]: Update the definition of __future__ in the glossary by replacing the confusing word “pseudo-module” with a more accurate description.
- bpo-35183 [https://bugs.python.org/issue?@action=redirect&bpo=35183]: Add typical examples to os.path.splitext docs
- bpo-30511 [https://bugs.python.org/issue?@action=redirect&bpo=30511]: Clarify that **`shutil.make_archive()`** is not thread-safe due to reliance on changing the current working directory.
- bpo-44561 [https://bugs.python.org/issue?@action=redirect&bpo=44561]: Update of three expired hyperlinks in Doc/distributing/index.rst: “Project structure”, “Building and packaging the project”, and “Uploading the project to the Python Packaging Index”.
- bpo-44651 [https://bugs.python.org/issue?@action=redirect&bpo=44651]: Delete entry “coercion” in Doc/glossary.rst for its outdated definition.
- bpo-42958 [https://bugs.python.org/issue?@action=redirect&bpo=42958]: Updated the docstring and docs of **`filecmp.cmp()`** to be more accurate and less confusing especially in respect to *shallow* arg.
- bpo-44631 [https://bugs.python.org/issue?@action=redirect&bpo=44631]: Refactored the `repr()` code of the `_Environ` (os module).
- bpo-44613 [https://bugs.python.org/issue?@action=redirect&bpo=44613]: importlib.metadata is no longer provisional.
- bpo-44558 [https://bugs.python.org/issue?@action=redirect&bpo=44558]: Match the docstring and python implementation of **`countOf()`** to the behavior of its c implementation.
- bpo-44544 [https://bugs.python.org/issue?@action=redirect&bpo=44544]: List all kwargs for **`textwrap.wrap()`**, **`textwrap.fill()`**, and **`textwrap.shorten()`**. Now, there are nav links to attributes of **`TextWrap`**, which makes navigation much easier while minimizing duplication in the documentation.
- bpo-38062 [https://bugs.python.org/issue?@action=redirect&bpo=38062]: Clarify that atexit uses equality comparisons internally.

- bpo-40620 [https://bugs.python.org/issue?@action=redirect&bpo=40620]: Convert examples in tutorial controlflow.rst section 4.3 to be interpreter-demo style.
- bpo-43066 [https://bugs.python.org/issue?@action=redirect&bpo=43066]: Added a warning to **`zipfile`** docs: filename arg with a leading slash may cause archive to be un-openable on Windows systems.
- bpo-39452 [https://bugs.python.org/issue?@action=redirect&bpo=39452]: Rewrote `Doc/library/__main__.rst`. Broadened scope of the document to explicitly discuss and differentiate between `__main__.py` in packages versus the `__name__ == '__main__'` expression (and the idioms that surround it).
- bpo-13814 [https://bugs.python.org/issue?@action=redirect&bpo=13814]: In the Design FAQ, answer "Why don't generators support the with statement?"
- bpo-27752 [https://bugs.python.org/issue?@action=redirect&bpo=27752]: Documentation of csv.Dialect is more descriptive.
- bpo-44453 [https://bugs.python.org/issue?@action=redirect&bpo=44453]: Fix documentation for the return type of **`sysconfig.get_path()`**.
- bpo-44392 [https://bugs.python.org/issue?@action=redirect&bpo=44392]: Added a new section in the C API documentation for types used in type hinting. Documented `Py_GenericAlias` and `Py_GenericAliasType`.
- bpo-38291 [https://bugs.python.org/issue?@action=redirect&bpo=38291]: Mark `typing.io` and `typing.re` as deprecated since Python 3.8 in the documentation. They were never properly supported by type checkers.
- bpo-44322 [https://bugs.python.org/issue?@action=redirect&bpo=44322]: Document that SyntaxError args have a details tuple and that details are adjusted for errors in f-string field replacement expressions.
- bpo-42392 [https://bugs.python.org/issue?@action=redirect&bpo=42392]: Document the deprecation and removal of the `loop` parameter for many functions and classes in **`asyncio`**.
- bpo-44195 [https://bugs.python.org/issue?@action=redirect&bpo=44195]: Corrected references to `TraversableResources` in docs. There is no `TraversableReader`.
- bpo-41963 [https://bugs.python.org/issue?@action=redirect&bpo=41963]: Document that `ConfigParser` strips off comments when reading configuration files.

- bpo-44072 [https://bugs.python.org/issue?@action=redirect&bpo=44072]: Correct where in the numeric ABC hierarchy `**` support is added, i.e., in numbers.Complex, not numbers.Integral.
- bpo-43558 [https://bugs.python.org/issue?@action=redirect&bpo=43558]: Add the remark to **`dataclasses`** documentation that the **`__init__()`** of any base class has to be called in **`__post_init__()`**, along with a code example.
- bpo-44025 [https://bugs.python.org/issue?@action=redirect&bpo=44025]: Clarify when '_' in match statements is a keyword, and when not.
- bpo-41706 [https://bugs.python.org/issue?@action=redirect&bpo=41706]: Fix docs about how methods like `__add__` are invoked when evaluating operator expressions.
- bpo-41621 [https://bugs.python.org/issue?@action=redirect&bpo=41621]: Document that **`collections.defaultdict`** parameter `default_factory` defaults to `None` and is positional-only.
- bpo-41576 [https://bugs.python.org/issue?@action=redirect&bpo=41576]: document BaseException in favor of bare except
- bpo-21760 [https://bugs.python.org/issue?@action=redirect&bpo=21760]: The description for __file__ fixed. Patch by Furkan Onder
- bpo-39498 [https://bugs.python.org/issue?@action=redirect&bpo=39498]: Add a "Security Considerations" index which links to standard library modules that have explicitly documented security considerations.
- bpo-33479 [https://bugs.python.org/issue?@action=redirect&bpo=33479]: Remove the unqualified claim that tkinter is threadsafe. It has not been true for several years and likely never was. An explanation of what is true may be added later, after more discussion, and possibly after patching _tkinter.c,

Testes

- bpo-40173 [https://bugs.python.org/issue?@action=redirect&bpo=40173]: Fix **`test.support.import_helper.import_fresh_module()`**.

- bpo-45280 [https://bugs.python.org/issue?@action=redirect&bpo=45280]: Add a test case for empty **`typing.NamedTuple`**.

- bpo-45269 [https://bugs.python.org/issue?@action=redirect&bpo=45269]: Cover case when invalid `markers` type is supplied to `c_make_encoder`.

- bpo-45128 [https://bugs.python.org/issue?@action=redirect&bpo=45128]: Fix `test_multiprocessing_fork` failure due to `test_logging` and `sys.modules` manipulation.

- bpo-45209 [https://bugs.python.org/issue?@action=redirect&bpo=45209]: Fix `UserWarning: resource_tracker` warning in `_test_multiprocessing._TestSharedMemory.test_shared_memory_cleaned_after_process_termination`

- bpo-45185 [https://bugs.python.org/issue?@action=redirect&bpo=45185]: Enables `TestEnumerations` test cases in `test_ssl` suite.

- bpo-45195 [https://bugs.python.org/issue?@action=redirect&bpo=45195]: Fix test_readline.test_nonascii(): sometimes, the newline character is not written at the end, so don't expect it in the output. Patch by Victor Stinner.

- bpo-45156 [https://bugs.python.org/issue?@action=redirect&bpo=45156]: Fixes infinite loop on **`unittest.mock.seal()`** of mocks created by **`create_autospec()`**.

- bpo-45125 [https://bugs.python.org/issue?@action=redirect&bpo=45125]: Improves pickling tests and docs of `SharedMemory` and `SharableList` objects.

- bpo-44860 [https://bugs.python.org/issue?@action=redirect&bpo=44860]: Update `test_sysconfig.test_user_similar()` for the posix_user scheme: `platlib` doesn't use **`sys.platlibdir`**. Patch by Victor Stinner.

- bpo-45052 [https://bugs.python.org/issue?@action=redirect&bpo=45052]: `WithProcessesTestSharedMemory.test_shared_memory_basics` test was ignored, because `self.assertEqual(sms.size, sms2.size)` line was failing. It is now removed and test is unskipped.

  The main motivation for this line to be removed from the test is that the `size` of `SharedMemory` is not ever guaranteed to be the same. It is

decided by the platform.

- bpo-44895 [https://bugs.python.org/issue?@action=redirect&bpo=44895]: libregrtest now clears the type cache later to reduce the risk of false alarm when checking for reference leaks. Previously, the type cache was cleared too early and libregrtest raised a false alarm about reference leaks under very specific conditions. Patch by Irit Katriel and Victor Stinner.

- bpo-45042 [https://bugs.python.org/issue?@action=redirect&bpo=45042]: Fixes that test classes decorated with `@hashlib_helper.requires_hashdigest` were skipped all the time.

- bpo-25130 [https://bugs.python.org/issue?@action=redirect&bpo=25130]: Add calls of **`gc.collect()`** in tests to support PyPy.

- bpo-45011 [https://bugs.python.org/issue?@action=redirect&bpo=45011]: Made tests relying on the **`_asyncio`** C extension module optional to allow running on alternative Python implementations. Patch by Serhiy Storchaka.

- bpo-44949 [https://bugs.python.org/issue?@action=redirect&bpo=44949]: Fix auto history tests of test_readline: sometimes, the newline character is not written at the end, so don't expect it in the output.

- bpo-44891 [https://bugs.python.org/issue?@action=redirect&bpo=44891]: Tests were added to clarify **`id()`** is preserved when `obj * 1` is used on **`str`** and **`bytes`** objects. Patch by Nikita Sobolev.

- bpo-44852 [https://bugs.python.org/issue?@action=redirect&bpo=44852]: Add ability to wholesale silence DeprecationWarnings while running the regression test suite.

- bpo-40928 [https://bugs.python.org/issue?@action=redirect&bpo=40928]: Notify users running test_decimal regression tests on macOS of potential harmless "malloc can't allocate region" messages spewed by test_decimal.

- bpo-44734 [https://bugs.python.org/issue?@action=redirect&bpo=44734]: Fixed floating-point precision issue in turtle tests.

- bpo-44708 [https://bugs.python.org/issue?@action=redirect&bpo=44708]: Regression tests, when run with -w, are now re-running only the affected test methods instead of re-running the entire test file.

- bpo-42095 [https://bugs.python.org/issue?@action=redirect&bpo=42095]: Added interop tests for Apple plists: generate plist files with Python plistlib and parse with Apple plutil; and the other way round.

- bpo-44647 [https://bugs.python.org/issue?@action=redirect&bpo=44647]: Added a permanent Unicode-valued environment variable to regression tests to ensure they handle this use case in the future. If your test environment breaks because of that, report a bug to us, and temporarily set PYTHONREGRTEST_UNICODE_GUARD=0 in your test environment.

- bpo-44515 [https://bugs.python.org/issue?@action=redirect&bpo=44515]: Adjust recently added contextlib tests to avoid assuming the use of a refcounted GC

- bpo-44287 [https://bugs.python.org/issue?@action=redirect&bpo=44287]: Fix asyncio test_popen() of test_windows_utils by using a longer timeout. Use military grade battle-tested **`test.support.SHORT_TIMEOUT`** timeout rather than a hardcoded timeout of 10 seconds: it's 30 seconds by default, but it is made longer on slow buildbots. Patch by Victor Stinner.

- bpo-44451 [https://bugs.python.org/issue?@action=redirect&bpo=44451]: Reset `DeprecationWarning` filters in `test.test_importlib.test_metadata_api.APITests.test_entry_points_by_index` to avoid `StopIteration` error if `DeprecationWarnings` are ignored.

- bpo-44363 [https://bugs.python.org/issue?@action=redirect&bpo=44363]: Account for address sanitizer in test_capi. test_capi now passes when run GCC address sanitizer.

- bpo-44364 [https://bugs.python.org/issue?@action=redirect&bpo=44364]: Add non integral tests for **`math.sqrt()`** function.

- bpo-43921 [https://bugs.python.org/issue?@action=redirect&bpo=43921]: Fix test_ssl.test_wrong_cert_tls13(): use `suppress_ragged_eofs=False`, since `read()` can raise **`ssl.SSLEOFError`** on Windows. Patch by Victor Stinner.

- bpo-43921 [https://bugs.python.org/issue?@action=redirect&bpo=43921]: Fix test_pha_required_nocert() of test_ssl: catch two more EOF cases (when the `recv()` method returns an empty string). Patch by Victor Stinner.

- bpo-44131 [https://bugs.python.org/issue?@action=redirect&bpo=44131]: Add test_frozenmain to test_embed to test the **`Py_FrozenMain()`** C function. Patch by Victor Stinner.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Ignore error string case in test_file_not_exists().

- bpo-42083 [https://bugs.python.org/issue?@action=redirect&bpo=42083]: Add test to check that `PyStructSequence_NewType` accepts a `PyStructSequence_Desc` with `doc` field set to `NULL`.

- bpo-35753 [https://bugs.python.org/issue?@action=redirect&bpo=35753]: Fix crash in doctest when doctest parses modules that include unwrappable functions by skipping those functions.

- bpo-30256 [https://bugs.python.org/issue?@action=redirect&bpo=30256]: Add test for nested queues when using `multiprocessing` shared objects `AutoProxy[Queue]` inside `ListProxy` and `DictProxy`

Construção

- bpo-45220 [https://bugs.python.org/issue?@action=redirect&bpo=45220]: Avoid building with the Windows 11 SDK previews automatically. This may be overridden by setting the `DefaultWindowsSDKVersion` environment variable before building.

- bpo-45020 [https://bugs.python.org/issue?@action=redirect&bpo=45020]: Freeze stdlib modules that are imported during startup. This provides

significant performance improvements to startup. If necessary, use the previously added “-X frozen_modules=off” commandline option to force importing the source modules.

- bpo-45188 [https://bugs.python.org/issue?@action=redirect&bpo=45188]: Windows builds now regenerate frozen modules as the first part of the build. Previously the regeneration was later in the build, which would require it to be restarted if any modules had changed.

- bpo-45163 [https://bugs.python.org/issue?@action=redirect&bpo=45163]: Fixes Haiku platform build.

- bpo-45067 [https://bugs.python.org/issue?@action=redirect&bpo=45067]: The ncurses function extended_color_content was introduced in 2017

  (https://invisible-island.net/ncurses/NEWS.html#index-t20170401). The

  ncurses-devel package in CentOS 7 had a older version ncurses resulted in compilation error. For compiling ncurses with extended color support, we verify the version of the ncurses library >= 20170401.

- bpo-45019 [https://bugs.python.org/issue?@action=redirect&bpo=45019]: Generate lines in relevant files for frozen modules. Up until now each of the files had to be edited manually. This change makes it easier to add to and modify the frozen modules.

- bpo-44340 [https://bugs.python.org/issue?@action=redirect&bpo=44340]: Add support for building with clang thin lto via –with-lto=thin/full. Patch by Donghee Na and Brett Holman.

- bpo-44535 [https://bugs.python.org/issue?@action=redirect&bpo=44535]: Enable building using a Visual Studio 2022 install on Windows.

- bpo-43298 [https://bugs.python.org/issue?@action=redirect&bpo=43298]: Improved error message when building without a Windows SDK installed.

- bpo-44381 [https://bugs.python.org/issue?@action=redirect&bpo=44381]: The Windows build now accepts **`EnableControlFlowGuard`** set to `guard` to enable CFG.

- bpo-41282 [https://bugs.python.org/issue?@action=redirect&bpo=41282]: Fix broken `make install` that caused standard library extension modules to be unnecessarily and incorrectly rebuilt during the install phase of cpython.

Windows

- bpo-45375 [https://bugs.python.org/issue?@action=redirect&bpo=45375]: Fixes an assertion failure due to searching for the standard library in unnormalised paths.
- bpo-45022 [https://bugs.python.org/issue?@action=redirect&bpo=45022]: Update Windows release to include libffi 3.4.2
- bpo-45007 [https://bugs.python.org/issue?@action=redirect&bpo=45007]: Update to OpenSSL 1.1.1l in Windows build
- bpo-44848 [https://bugs.python.org/issue?@action=redirect&bpo=44848]: Upgrade Windows installer to use SQLite 3.36.0.
- bpo-44572 [https://bugs.python.org/issue?@action=redirect&bpo=44572]: Avoid consuming standard input in the **`platform`** module
- bpo-44582 [https://bugs.python.org/issue?@action=redirect&bpo=44582]: Accelerate speed of **`mimetypes`** initialization using a native implementation of the registry scan.
- bpo-41299 [https://bugs.python.org/issue?@action=redirect&bpo=41299]: Fix 16 milliseconds jitter when using timeouts in **`threading`**, such as with **`threading.Lock.acquire()`** or **`threading.Condition.wait()`**.
- bpo-42686 [https://bugs.python.org/issue?@action=redirect&bpo=42686]: Build **`sqlite3`** with math functions enabled. Patch by Erlend E. Aasland.
- bpo-40263 [https://bugs.python.org/issue?@action=redirect&bpo=40263]: This is a follow-on bug from https://bugs.python.org/issue26903. Once that is applied we run into an off-by-one assertion problem. The assert was not correct.

macOS

- bpo-45007 [https://bugs.python.org/issue?@action=redirect&bpo=45007]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.1l.
- bpo-34602 [https://bugs.python.org/issue?@action=redirect&bpo=34602]: When building CPython on macOS with `./configure --with-undefined-behavior-sanitizer --with-pydebug`, the stack size is now quadrupled to allow for the entire test suite to pass.
- bpo-44848 [https://bugs.python.org/issue?@action=redirect&bpo=44848]: Update macOS installer to use SQLite 3.36.0.
- bpo-44689 [https://bugs.python.org/issue?@action=redirect&bpo=44689]: **`ctypes.util.find_library()`** now works correctly on macOS 11 Big Sur even if Python is built on an older version of macOS. Previously, when built on older macOS systems, `find_library` was not able to find macOS system libraries when running on Big Sur due to changes in how system libraries are stored.
- bpo-41972 [https://bugs.python.org/issue?@action=redirect&bpo=41972]: The framework build's user header path in sysconfig is changed to add a 'pythonX.Y' component to match distutils's behavior.
- bpo-43109 [https://bugs.python.org/issue?@action=redirect&bpo=43109]: Allow –with-lto configure option to work with Apple-supplied Xcode or Command Line Tools.
- bpo-34932 [https://bugs.python.org/issue?@action=redirect&bpo=34932]: Add socket.TCP_KEEPALIVE support for macOS. Patch by Shane Harvey.

IDLE

- bpo-45296 [https://bugs.python.org/issue?@action=redirect&bpo=45296]: On Windows, change exit/quit message to suggest Ctrl-D, which works, instead of <Ctrl-Z Return>, which does not work in IDLE.
- bpo-45193 [https://bugs.python.org/issue?@action=redirect&bpo=45193]: Make completion boxes appear on Ubuntu again.
- bpo-40128 [https://bugs.python.org/issue?@action=redirect&bpo=40128]: Mostly fix completions on macOS when not using tcl/tk 8.6.11 (as with 3.9). The added update_idletask call should be harmless and possibly helpful otherwise.
- bpo-33962 [https://bugs.python.org/issue?@action=redirect&bpo=33962]: Move the indent space setting from the Font tab to the new Windows tab. Patch by Mark Roseman and Terry Jan Reedy.

- bpo-40468 [https://bugs.python.org/issue?@action=redirect&bpo=40468]: Split the settings dialog General tab into Windows and Shell/ED tabs. Move help sources, which extend the Help menu, to the Extensions tab. Make space for new options and shorten the dialog. The latter makes the dialog better fit small screens.
- bpo-41611 [https://bugs.python.org/issue?@action=redirect&bpo=41611]: Avoid uncaught exceptions in `AutoCompleteWindow.winconfig_event()`.
- bpo-41611 [https://bugs.python.org/issue?@action=redirect&bpo=41611]: Fix IDLE sometimes freezing upon tab-completion on macOS.
- bpo-44010 [https://bugs.python.org/issue?@action=redirect&bpo=44010]: Highlight the new match statement's soft keywords: **`match`**, **`case`**, and _. However, this highlighting is not perfect and will be incorrect in some rare cases, including some _-s in `case` patterns.
- bpo-44026 [https://bugs.python.org/issue?@action=redirect&bpo=44026]: Include interpreter's typo fix suggestions in message line for NameErrors and AttributeErrors. Patch by E. Paine.

Ferramentas/Demos

- bpo-44786 [https://bugs.python.org/issue?@action=redirect&bpo=44786]: Fix a warning in regular expression in the c-analyzer script.
- bpo-44967 [https://bugs.python.org/issue?@action=redirect&bpo=44967]: pydoc now returns a non-zero status code when a module cannot be found.
- bpo-44978 [https://bugs.python.org/issue?@action=redirect&bpo=44978]: Allow the Argument Clinic tool to handle `__complex__` special methods.
- bpo-43425 [https://bugs.python.org/issue?@action=redirect&bpo=43425]: Removed the 'test2to3' demo project that demonstrated using lib2to3 to support Python 2.x and Python 3.x from a single source in a distutils package. Patch by Donghee Na
- bpo-44074 [https://bugs.python.org/issue?@action=redirect&bpo=44074]: Make patchcheck automatically detect the correct base branch name (previously it was hardcoded to 'master')
- bpo-20291 [https://bugs.python.org/issue?@action=redirect&bpo=20291]: Added support for variadic positional parameters in Argument Clinic.

API C

- bpo-41710 [https://bugs.python.org/issue?@action=redirect&bpo=41710]: The PyThread_acquire_lock_timed() function now clamps the timeout if it is too large, rather than aborting the process. Patch by Victor Stinner.

- bpo-44687 [https://bugs.python.org/issue?@action=redirect&bpo=44687]: **`BufferedReader.peek()`** no longer raises **`ValueError`** when the entire file has already been buffered.

- bpo-45116 [https://bugs.python.org/issue?@action=redirect&bpo=45116]: Add the **`Py_ALWAYS_INLINE`** macro to ask the compiler to always inline a static inline function. The compiler can ignore it and decides to not inline the function. Patch by Victor Stinner.

- bpo-45094 [https://bugs.python.org/issue?@action=redirect&bpo=45094]: Add the **`Py_NO_INLINE`** macro to disable inlining on a function. Patch by Victor Stinner.

- bpo-45061 [https://bugs.python.org/issue?@action=redirect&bpo=45061]: Add a deallocator to the **`bool`** type to detect refcount bugs in C extensions which call `Py_DECREF(Py_True);` or `Py_DECREF(Py_False);` by mistake. Patch by Victor Stinner.

- bpo-42035 [https://bugs.python.org/issue?@action=redirect&bpo=42035]: Add a new **`PyType_GetQualName()`** function to get type's qualified name.

- bpo-41103 [https://bugs.python.org/issue?@action=redirect&bpo=41103]: Reverts removal of the old buffer protocol because they are part of stable ABI.

- bpo-44751 [https://bugs.python.org/issue?@action=redirect&bpo=44751]: Remove `crypt.h` include from the public `Python.h` header.

- bpo-42747 [https://bugs.python.org/issue?@action=redirect&bpo=42747]: The `Py_TPFLAGS_HAVE_VERSION_TAG` type flag now does nothing. The `Py_TPFLAGS_HAVE_AM_SEND` flag (which was added in 3.10) is removed. Both were unnecessary because it is not possible to have type objects with the relevant fields missing.

- bpo-44530 [https://bugs.python.org/issue?@action=redirect&bpo=44530]: Added the `co_qualname` to the `PyCodeObject` structure to propagate the qualified name from the compiler to code objects.

  Patch by Gabriele N. Tornetta

- bpo-44441 [https://bugs.python.org/issue?@action=redirect&bpo=44441]: **`Py_RunMain()`** now resets **`PyImport_Inittab`** to its initial value at exit. It must be possible to call **`PyImport_AppendInittab()`** or **`PyImport_ExtendInittab()`** at each Python initialization. Patch by Victor Stinner.

- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Remove 4 private trashcan C API functions which were only kept for the backward compatibility of the stable ABI with Python 3.8 and older, since the trashcan API was not usable with the limited C API on Python 3.8 and older. The trashcan API was excluded from the limited C API in Python 3.9.

  Funções removidas:

  - _PyTrash_deposit_object()
  - _PyTrash_destroy_chain()
  - _PyTrash_thread_deposit_object()
  - _PyTrash_thread_destroy_chain()

  The trashcan C API was never usable with the limited C API, since old trashcan macros accessed directly **`PyThreadState`** members like `_tstate->trash_delete_nesting`, whereas the **`PyThreadState`** structure is opaque in the limited C API.

  Exclude also the `PyTrash_UNWIND_LEVEL` constant from the C API.

  Patch por Victor Stinner.

- bpo-40939 [https://bugs.python.org/issue?@action=redirect&bpo=40939]: Removed documentation for the removed `PyParser_*` C API.

- bpo-43795 [https://bugs.python.org/issue?@action=redirect&bpo=43795]: The list in Conteúdo da API Limitada now shows the public name **`PyFrameObject`** rather than `_frame`. The non-existing entry `_node` no longer appears in the list.

- bpo-44378 [https://bugs.python.org/issue?@action=redirect&bpo=44378]: **`Py_IS_TYPE()`** no longer uses **`Py_TYPE()`** to avoid a compiler warning: no longer cast `const PyObject*` to `PyObject*`. Patch by Victor Stinner.

- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Convert the **`Py_TYPE()`** and **`Py_SIZE()`** macros to static inline functions. The **`Py_SET_TYPE()`** and **`Py_SET_SIZE()`** functions must now be used to set an object type and size. Patch by Victor Stinner.

- bpo-44263 [https://bugs.python.org/issue?@action=redirect&bpo=44263]: The **`PyType_Ready()`** function now raises an error if a type is defined with the **`Py_TPFLAGS_HAVE_GC`** flag set but has no traverse function (**`PyTypeObject.tp_traverse`**). Patch by Victor Stinner.

- bpo-43795 [https://bugs.python.org/issue?@action=redirect&bpo=43795]: The undocumented function **`Py_FrozenMain()`** is removed from the Limited API.

- bpo-44113 [https://bugs.python.org/issue?@action=redirect&bpo=44113]: Deprecate the following functions to configure the Python initialization:

  - **`PySys_AddWarnOptionUnicode()`**
  - **`PySys_AddWarnOption()`**
  - **`PySys_AddXOption()`**
  - **`PySys_HasWarnOptions()`**
  - **`Py_SetPath()`**
  - **`Py_SetProgramName()`**
  - **`Py_SetPythonHome()`**
  - **`Py_SetStandardStreamEncoding()`**
  - **`_Py_SetProgramFullPath()`**

Use the new **`PyConfig`** API of the Python Initialization Configuration instead (**PEP 587** [https://peps.python.org/pep-0587/]).

- bpo-44094 [https://bugs.python.org/issue?@action=redirect&bpo=44094]: Remove `PyErr_SetFromErrnoWithUnicodeFilename()`, `PyErr_SetFromWindowsErrWithUnicodeFilename()`, and `PyErr_SetExcFromWindowsErrWithUnicodeFilename()`. They are not documented and have been deprecated since Python 3.3.

- bpo-43795 [https://bugs.python.org/issue?@action=redirect&bpo=43795]: **`PyCodec_Unregister()`** is now properly exported as a function in the Windows Stable ABI DLL.

- bpo-44029 [https://bugs.python.org/issue?@action=redirect&bpo=44029]: Remove deprecated `Py_UNICODE` APIs: `PyUnicode_Encode`, `PyUnicode_EncodeUTF7`, `PyUnicode_EncodeUTF8`, `PyUnicode_EncodeUTF16`, `PyUnicode_EncodeUTF32`, `PyUnicode_EncodeLatin1`, `PyUnicode_EncodeMBCS`, `PyUnicode_EncodeDecimal`, `PyUnicode_EncodeRawUnicodeEscape`, `PyUnicode_EncodeCharmap`, `PyUnicode_EncodeUnicodeEscape`, `PyUnicode_TransformDecimalToASCII`, `PyUnicode_TranslateCharmap`, `PyUnicodeEncodeError_Create`, `PyUnicodeTranslateError_Create`. See **PEP 393** [https://peps.python.org/pep-0393/] and **PEP 624** [https://peps.python.org/pep-0624/] for reference.

- bpo-42035 [https://bugs.python.org/issue?@action=redirect&bpo=42035]: Add a new **`PyType_GetName()`** function to get type's short name.

Python 3.10.0 beta 1

Data de lançamento: 2021-05-03

Segurança

- bpo-43434 [https://bugs.python.org/issue?@action=redirect&bpo=43434]: Creating **`sqlite3.Connection`** objects now also produces `sqlite3.connect`

and `sqlite3.connect/handle` auditing events. Previously these events were only produced by **`sqlite3.connect()`** calls. Patch by Erlend E. Aasland.

- bpo-43998 [https://bugs.python.org/issue?@action=redirect&bpo=43998]: The **`ssl`** module sets more secure cipher suites defaults. Ciphers without forward secrecy and with SHA-1 MAC are disabled by default. Security level 2 prohibits weak RSA, DH, and ECC keys with less than 112 bits of security. **`SSLContext`** defaults to minimum protocol version TLS 1.2. Settings are based on Hynek Schlawack's research.

- bpo-43882 [https://bugs.python.org/issue?@action=redirect&bpo=43882]: The presence of newline or tab characters in parts of a URL could allow some forms of attacks.

  Following the controlling specification for URLs defined by WHATWG **`urllib.parse()`** now removes ASCII newlines and tabs from URLs, preventing such attacks.

- bpo-43472 [https://bugs.python.org/issue?@action=redirect&bpo=43472]: Ensures interpreter-level audit hooks receive the `cpython.PyInterpreterState_New` event when called through the `_xxsubinterpreters` module.

- bpo-43362 [https://bugs.python.org/issue?@action=redirect&bpo=43362]: Fix invalid free in _sha3 module. The issue was introduced in 3.10.0a1. Python 3.9 and earlier are not affected.

- bpo-43762 [https://bugs.python.org/issue?@action=redirect&bpo=43762]: Add audit events for **`sqlite3.connect/handle()`**, **`sqlite3.Connection.enable_load_extension()`**, and **`sqlite3.Connection.load_extension()`**. Patch by Erlend E. Aasland.

- bpo-43756 [https://bugs.python.org/issue?@action=redirect&bpo=43756]: Add new audit event `glob.glob/2` to incorporate the new *root_dir* and *dir_fd* arguments added to **`glob.glob()`** and **`glob.iglob()`**.

- bpo-36384 [https://bugs.python.org/issue?@action=redirect&bpo=36384]: **`ipaddress`** module no longer accepts any leading zeros in IPv4 address strings. Leading zeros are ambiguous and interpreted as octal notation by some libraries. For example the legacy function **`socket.inet_aton()`** treats leading zeros as octal notation. glibc implementation of modern **`inet_pton()`** does not accept any leading zeros. For a while the **`ipaddress`** module used to accept ambiguous leading zeros.

- bpo-43075 [https://bugs.python.org/issue?@action=redirect&bpo=43075]: Fix Regular Expression Denial of Service (ReDoS) vulnerability in **`urllib.request.AbstractBasicAuthHandler`**. The ReDoS-vulnerable regex has quadratic worst-case complexity and it allows cause a denial of service when identifying crafted invalid RFCs. This ReDoS issue is on the client side and needs remote attackers to control the HTTP server.

- bpo-42800 [https://bugs.python.org/issue?@action=redirect&bpo=42800]: Audit hooks are now fired for frame.f_code, traceback.tb_frame, and generator code/frame attribute access.

- bpo-37363 [https://bugs.python.org/issue?@action=redirect&bpo=37363]: Add audit events to the **`http.client`** module.

Núcleo e embutidos

- bpo-43977 [https://bugs.python.org/issue?@action=redirect&bpo=43977]: Prevent classes being both a sequence and a mapping when pattern matching.
- bpo-43977 [https://bugs.python.org/issue?@action=redirect&bpo=43977]: Use **`tp_flags`** on the class object to determine if the subject is a sequence or mapping when pattern matching. Avoids the need to import **`collections.abc`** when pattern matching.
- bpo-43892 [https://bugs.python.org/issue?@action=redirect&bpo=43892]: Restore proper validation of complex literal value patterns when parsing **`match`** blocks.
- bpo-43933 [https://bugs.python.org/issue?@action=redirect&bpo=43933]: Set frame.f_lineno to the line number of the ‘with’ kweyword when executing the call to `__exit__`.

- bpo-43933 [https://bugs.python.org/issue?@action=redirect&bpo=43933]: If the current position in a frame has no line number then set the f_lineno attribute to None, instead of -1, to conform to PEP 626. This should not normally be possible, but might occur in some unusual circumstances.
- bpo-43963 [https://bugs.python.org/issue?@action=redirect&bpo=43963]: Importing the `_signal` module in a subinterpreter has no longer side effects.
- bpo-42739 [https://bugs.python.org/issue?@action=redirect&bpo=42739]: The internal representation of line number tables is changed to not use sentinels, and an explicit length parameter is added to the out of process API function `PyLineTable_InitAddressRange`. This makes the handling of line number tables more robust in some circumstances.

Biblioteca

- bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908]: Make `re` types immutable. Patch by Erlend E. Aasland.
- bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908]: Make the `array.array` type immutable. Patch by Erlend E. Aasland.

Núcleo e embutidos

- bpo-43901 [https://bugs.python.org/issue?@action=redirect&bpo=43901]: Change class and module objects to lazy-create empty annotations dicts on demand. The annotations dicts are stored in the object's __dict__ for backwards compatibility.
- bpo-43892 [https://bugs.python.org/issue?@action=redirect&bpo=43892]: Match patterns now use new dedicated AST nodes (`MatchValue`, `MatchSingleton`, `MatchSequence`, `MatchStar`, `MatchMapping`, `MatchClass`) rather than reusing expression AST nodes. `MatchAs` and `MatchOr` are now defined as pattern nodes rather than as expression nodes. Patch by Nick Coghlan.
- bpo-42725 [https://bugs.python.org/issue?@action=redirect&bpo=42725]: Usage of `await`/`yield`/`yield from` and named expressions within an annotation is now forbidden when PEP 563 is activated.
- bpo-43754 [https://bugs.python.org/issue?@action=redirect&bpo=43754]: When performing structural pattern matching (**PEP 634**

[https://peps.python.org/pep-0634/]), captured names are now left unbound until the *entire* pattern has matched successfully.

- bpo-42737 [https://bugs.python.org/issue?@action=redirect&bpo=42737]: Annotations for complex targets (everything beside simple names) no longer cause any runtime effects with `from __future__ import annotations`.
- bpo-43914 [https://bugs.python.org/issue?@action=redirect&bpo=43914]: **`SyntaxError`** exceptions raised by the interpreter will highlight the full error range of the expression that constitutes the syntax error itself, instead of just where the problem is detected. Patch by Pablo Galindo.
- bpo-38605 [https://bugs.python.org/issue?@action=redirect&bpo=38605]: Revert making `from __future__ import annotations` the default. This follows the Steering Council decision to postpone PEP 563 changes to at least Python 3.11. See the original email for more information regarding the decision: https://mail.python.org/archives/list/python-dev@python.org/thread/CLVXXPQ2T2LQ5MP2Y53VVQFCXYWQJHKZ/. Patch by Pablo Galindo.
- bpo-43475 [https://bugs.python.org/issue?@action=redirect&bpo=43475]: Hashes of NaN values now depend on object identity. Formerly, they always hashed to 0 even though NaN values are not equal to one another. Having the same hash for unequal values caused pile-ups in hash tables.
- bpo-43859 [https://bugs.python.org/issue?@action=redirect&bpo=43859]: Improve the error message for **`IndentationError`** exceptions. Patch by Pablo Galindo
- bpo-41323 [https://bugs.python.org/issue?@action=redirect&bpo=41323]: Constant tuple folding in bytecode optimizer now reuses tuple in constant table.
- bpo-43846 [https://bugs.python.org/issue?@action=redirect&bpo=43846]: Data stack usage is much reduced for large literal and call expressions.
- bpo-38530 [https://bugs.python.org/issue?@action=redirect&bpo=38530]: When printing **`NameError`** raised by the interpreter, **`PyErr_Display()`** will offer suggestions of similar variable names in the function that the exception was raised from. Patch by Pablo Galindo
- bpo-43823 [https://bugs.python.org/issue?@action=redirect&bpo=43823]: Improve syntax errors for invalid dictionary literals. Patch by Pablo Galindo.
- bpo-43822 [https://bugs.python.org/issue?@action=redirect&bpo=43822]: Improve syntax errors in the parser for missing commas between expressions.

Patch by Pablo Galindo.

- bpo-43798 [https://bugs.python.org/issue?@action=redirect&bpo=43798]: `ast.alias` nodes now include source location metadata attributes e.g. lineno, col_offset.
- bpo-43797 [https://bugs.python.org/issue?@action=redirect&bpo=43797]: Improve `SyntaxError` error messages for invalid comparisons. Patch by Pablo Galindo.
- bpo-43760 [https://bugs.python.org/issue?@action=redirect&bpo=43760]: Move the flag for checking whether tracing is enabled to the C stack, from the heap. Should speed up dispatch in the interpreter.
- bpo-43682 [https://bugs.python.org/issue?@action=redirect&bpo=43682]: Static methods (`@staticmethod`) and class methods (`@classmethod`) now inherit the method attributes (`__module__`, `__name__`, `__qualname__`, `__doc__`, `__annotations__`) and have a new `__wrapped__` attribute. Patch by Victor Stinner.
- bpo-43751 [https://bugs.python.org/issue?@action=redirect&bpo=43751]: Fixed a bug where `anext(ait, default)` would erroneously return None.
- bpo-42128 [https://bugs.python.org/issue?@action=redirect&bpo=42128]: `__match_args__` is no longer allowed to be a list.
- bpo-43683 [https://bugs.python.org/issue?@action=redirect&bpo=43683]: Add GEN_START opcode. Marks start of generator, including async, or coroutine and handles sending values to a newly created generator or coroutine.

Biblioteca

- bpo-43105 [https://bugs.python.org/issue?@action=redirect&bpo=43105]: Importlib now resolves relative paths when creating module spec objects from file locations.

Núcleo e embutidos

- bpo-43682 [https://bugs.python.org/issue?@action=redirect&bpo=43682]: Static methods (`@staticmethod`) are now callable as regular functions. Patch by Victor Stinner.
- bpo-42609 [https://bugs.python.org/issue?@action=redirect&bpo=42609]: Prevented crashes in the AST validator and optimizer when compiling some

absurdly long expressions like `"+0"*1000000`. **`RecursionError`** is now raised instead.

- bpo-38530 [https://bugs.python.org/issue?@action=redirect&bpo=38530]: When printing **`AttributeError`**, **`PyErr_Display()`** will offer suggestions of similar attribute names in the object that the exception was raised from. Patch by Pablo Galindo

Biblioteca

- bpo-44015 [https://bugs.python.org/issue?@action=redirect&bpo=44015]: In @dataclass(), raise a TypeError if KW_ONLY is specified more than once.

- bpo-25478 [https://bugs.python.org/issue?@action=redirect&bpo=25478]: Added a *total()* method to collections.Counter() to compute the sum of the counts.

- bpo-43733 [https://bugs.python.org/issue?@action=redirect&bpo=43733]: Change **`netrc.netrc`** to use UTF-8 encoding before using locale encoding.

- bpo-43979 [https://bugs.python.org/issue?@action=redirect&bpo=43979]: Removed an unnecessary list comprehension before looping from **`urllib.parse.parse_qsl()`**. Patch by Christoph Zwerschke and Donghee Na.

- bpo-43993 [https://bugs.python.org/issue?@action=redirect&bpo=43993]: Update bundled pip to 21.1.1.

- bpo-43957 [https://bugs.python.org/issue?@action=redirect&bpo=43957]: [Enum] Deprecate `TypeError` when non-member is used in a containment check; In 3.12 `True` or `False` will be returned instead, and containment will return `True` if the value is either a member of that enum or one of its members' value.

- bpo-42904 [https://bugs.python.org/issue?@action=redirect&bpo=42904]: For backwards compatibility with previous minor versions of Python, if **`typing.get_type_hints()`** receives no namespace dictionary arguments, **`typing.get_type_hints()`** will search through the global

then local namespaces during evaluation of stringized type annotations (string forward references) inside a class.

- bpo-43945 [https://bugs.python.org/issue?@action=redirect&bpo=43945]: [Enum] Deprecate non-standard mixin format() behavior: in 3.12 the enum member, not the member's value, will be used for format() calls.

- bpo-41139 [https://bugs.python.org/issue?@action=redirect&bpo=41139]: Deprecate undocumented `cgi.log()` API.

- bpo-43937 [https://bugs.python.org/issue?@action=redirect&bpo=43937]: Fixed the **`turtle`** module working with non-default root window.

- bpo-43930 [https://bugs.python.org/issue?@action=redirect&bpo=43930]: Update bundled pip to 21.1 and setuptools to 56.0.0

- bpo-43907 [https://bugs.python.org/issue?@action=redirect&bpo=43907]: Fix a bug in the pure-Python pickle implementation when using protocol 5, where bytearray instances that occur several time in the pickled object graph would incorrectly unpickle into repeated copies of the bytearray object.

- bpo-43926 [https://bugs.python.org/issue?@action=redirect&bpo=43926]: In `importlib.metadata`, provide a uniform interface to `Description`, allow for any field to be encoded with multiline values, remove continuation lines from multiline values, and add a `.json` property for easy access to the PEP 566 JSON-compatible form. Sync with `importlib_metadata 4.0`.

- bpo-43920 [https://bugs.python.org/issue?@action=redirect&bpo=43920]: OpenSSL 3.0.0: **`load_verify_locations()`** now returns a consistent error message when cadata contains no valid certificate.

- bpo-43607 [https://bugs.python.org/issue?@action=redirect&bpo=43607]: **`urllib`** can now convert Windows paths with `\\?\` prefixes into URL paths.

- bpo-43817 [https://bugs.python.org/issue?@action=redirect&bpo=43817]: Add **`inspect.get_annotations()`**, which safely computes the annotations

defined on an object. It works around the quirks of accessing the annotations from various types of objects, and makes very few assumptions about the object passed in. **`inspect.get_annotations()`** can also correctly un-stringize stringized annotations.

`inspect.signature()`, **`inspect.from_callable()`**, and **`inspect.from_function()`** now call **`inspect.get_annotations()`** to retrieve annotations. This means **`inspect.signature()`** and **`inspect.from_callable()`** can now un-stringize stringized annotations, too.

- bpo-43284 [https://bugs.python.org/issue?@action=redirect&bpo=43284]: platform.win32_ver derives the windows version from sys.getwindowsversion().platform_version which in turn derives the version from kernel32.dll (which can be of a different version than Windows itself). Therefore change the platform.win32_ver to determine the version using the platform module's _syscmd_ver private function to return an accurate version.

- bpo-42854 [https://bugs.python.org/issue?@action=redirect&bpo=42854]: The **`ssl`** module now uses `SSL_read_ex` and `SSL_write_ex` internally. The functions support reading and writing of data larger than 2 GB. Writing zero-length data no longer fails with a protocol violation error.

- bpo-42333 [https://bugs.python.org/issue?@action=redirect&bpo=42333]: Port `_ssl` extension module to multiphase initialization.

- bpo-43880 [https://bugs.python.org/issue?@action=redirect&bpo=43880]: **`ssl`** now raises DeprecationWarning for OP_NO_SSL/TLS* options, old TLS versions, old protocols, and other features that have been deprecated since Python 3.6, 3.7, or OpenSSL 1.1.0.

- bpo-41559 [https://bugs.python.org/issue?@action=redirect&bpo=41559]: **PEP 612** [https://peps.python.org/pep-0612/] is now implemented purely in Python; builtin `types.GenericAlias` objects no longer include `typing.ParamSpec` in `__parameters__` (with the exception of `collections.abc.Callable`'s `GenericAlias`). This means previously invalid uses of `ParamSpec` (such as `list[P]`) which worked in earlier

versions of Python 3.10 alpha, will now raise `TypeError` during substitution.

- bpo-43867 [https://bugs.python.org/issue?@action=redirect&bpo=43867]: The **`multiprocessing`** `Server` class now explicitly catches **`SystemExit`** and closes the client connection in this case. It happens when the `Server.serve_client()` method reaches the end of file (EOF).

- bpo-40443 [https://bugs.python.org/issue?@action=redirect&bpo=40443]: Remove unused imports: pyclbr no longer uses copy, and typing no longer uses ast. Patch by Victor Stinner.

- bpo-43820 [https://bugs.python.org/issue?@action=redirect&bpo=43820]: Remove an unneeded copy of the namespace passed to dataclasses.make_dataclass().

- bpo-43787 [https://bugs.python.org/issue?@action=redirect&bpo=43787]: Add `__iter__()` method to **`bz2.BZ2File`**, **`gzip.GzipFile`**, and **`lzma.LZMAFile`**. It makes iterating them about 2x faster. Patch by Inada Naoki.

- bpo-43680 [https://bugs.python.org/issue?@action=redirect&bpo=43680]: Deprecate io.OpenWrapper and _pyio.OpenWrapper: use io.open and _pyio.open instead. Until Python 3.9, _pyio.open was not a static method and builtins.open was set to OpenWrapper to not become a bound method when set to a class variable. _io.open is a built-in function whereas _pyio.open is a Python function. In Python 3.10, _pyio.open() is now a static method, and builtins.open() is now io.open().

- bpo-43680 [https://bugs.python.org/issue?@action=redirect&bpo=43680]: The Python **`_pyio.open()`** function becomes a static method to behave as **`io.open()`** built-in function: don't become a bound method when stored as a class variable. It becomes possible since static methods are now callable in Python 3.10. Moreover, **`_pyio.OpenWrapper()`** becomes a simple alias to **`_pyio.open()`**. Patch by Victor Stinner.

- bpo-41515 [https://bugs.python.org/issue?@action=redirect&bpo=41515]: Fix **`KeyError`** raised in **`typing.get_type_hints()`** due to synthetic

modules that don't appear in `sys.modules`.

- bpo-43776 [https://bugs.python.org/issue?@action=redirect&bpo=43776]: When `subprocess.Popen` args are provided as a string or as `pathlib.Path`, the Popen instance repr now shows the right thing.

- bpo-42248 [https://bugs.python.org/issue?@action=redirect&bpo=42248]: [Enum] ensure exceptions raised in `_missing__` are released

- bpo-43744 [https://bugs.python.org/issue?@action=redirect&bpo=43744]: fix issue with enum member name matching the start of a private variable name

- bpo-43772 [https://bugs.python.org/issue?@action=redirect&bpo=43772]: Fixed the return value of `TypeVar.__ror__`. Patch by Jelle Zijlstra.

- bpo-43764 [https://bugs.python.org/issue?@action=redirect&bpo=43764]: Add match_args parameter to @dataclass decorator to allow suppression of __match_args__ generation.

- bpo-43799 [https://bugs.python.org/issue?@action=redirect&bpo=43799]: OpenSSL 3.0.0: define `OPENSSL_API_COMPAT` 1.1.1 to suppress deprecation warnings. Python requires OpenSSL 1.1.1 APIs.

- bpo-43478 [https://bugs.python.org/issue?@action=redirect&bpo=43478]: Mocks can no longer be used as the specs for other Mocks. As a result, an already-mocked object cannot have an attribute mocked using `autospec=True` or be the subject of a `create_autospec(...)` call. This can uncover bugs in tests since these Mock-derived Mocks will always pass certain tests (e.g. `isinstance()`) and builtin assert functions (e.g. assert_called_once_with) will unconditionally pass.

- bpo-43794 [https://bugs.python.org/issue?@action=redirect&bpo=43794]: Add `ssl.OP_IGNORE_UNEXPECTED_EOF` constants (OpenSSL 3.0.0)

- bpo-43785 [https://bugs.python.org/issue?@action=redirect&bpo=43785]: Improve `bz2.BZ2File` performance by removing the RLock from BZ2File. This makes BZ2File thread unsafe in the face of multiple simultaneous

readers or writers, just like its equivalent classes in **`gzip`** and **`lzma`** have always been. Patch by Inada Naoki.

- bpo-43789 [https://bugs.python.org/issue?@action=redirect&bpo=43789]: OpenSSL 3.0.0: Don't call the password callback function a second time when first call has signaled an error condition.

- bpo-43788 [https://bugs.python.org/issue?@action=redirect&bpo=43788]: The header files for **`ssl`** error codes are now OpenSSL version-specific. Exceptions will now show correct reason and library codes. The `make_ssl_data.py` script has been rewritten to use OpenSSL's text file with error codes.

- bpo-43766 [https://bugs.python.org/issue?@action=redirect&bpo=43766]: Implement **PEP 647** [https://peps.python.org/pep-0647/] in the **`typing`** module by adding **`TypeGuard`**.

- bpo-25264 [https://bugs.python.org/issue?@action=redirect&bpo=25264]: **`os.path.realpath()`** now accepts a *strict* keyword-only argument. When set to `True`, **`OSError`** is raised if a path doesn't exist or a symlink loop is encountered.

- bpo-43780 [https://bugs.python.org/issue?@action=redirect&bpo=43780]: In `importlib.metadata`, incorporate changes from importlib_metadata 3.10: Add mtime-based caching during distribution discovery. Flagged use of dict result from `entry_points()` as deprecated.

- gh-47383 [https://github.com/python/cpython/issues/47383]: The `P.args` and `P.kwargs` attributes of **`typing.ParamSpec`** are now instances of the new classes **`typing.ParamSpecArgs`** and **`typing.ParamSpecKwargs`**, which enables a more useful `repr()`. Patch by Jelle Zijlstra.

- bpo-43731 [https://bugs.python.org/issue?@action=redirect&bpo=43731]: Add an `encoding` parameter **`logging.fileConfig()`**.

- bpo-43712 [https://bugs.python.org/issue?@action=redirect&bpo=43712]: Add `encoding` and `errors` parameters to **`fileinput.input()`** and **`fileinput.FileInput`**.

- bpo-38659 [https://bugs.python.org/issue?@action=redirect&bpo=38659]: A `simple_enum` decorator is added to the `enum` module to convert a normal class into an Enum. `test_simple_enum` added to test simple enums against a corresponding normal Enum. Standard library modules updated to use `simple_enum`.

- bpo-43764 [https://bugs.python.org/issue?@action=redirect&bpo=43764]: Fix an issue where **`__match_args__`** generation could fail for some **`dataclasses`**.

- bpo-43752 [https://bugs.python.org/issue?@action=redirect&bpo=43752]: Fix **`sqlite3`** regression for zero-sized blobs with converters, where `b""` was returned instead of `None`. The regression was introduced by PR 24723. Patch by Erlend E. Aasland.

- bpo-43655 [https://bugs.python.org/issue?@action=redirect&bpo=43655]: **`tkinter`** dialog windows are now recognized as dialogs by window managers on macOS and X Window.

- bpo-43723 [https://bugs.python.org/issue?@action=redirect&bpo=43723]: The following `threading` methods are now deprecated and should be replaced:

  - `currentThread` => **`threading.current_thread()`**
  - `activeCount` => **`threading.active_count()`**
  - `Condition.notifyAll` => **`threading.Condition.notify_all()`**
  - `Event.isSet` => **`threading.Event.is_set()`**
  - `Thread.setName` => **`threading.Thread.name`**
  - `thread.getName` => **`threading.Thread.name`**
  - `Thread.isDaemon` => **`threading.Thread.daemon`**
  - `Thread.setDaemon` => **`threading.Thread.daemon`**

  Patch by Jelle Zijlstra.

- bpo-2135 [https://bugs.python.org/issue?@action=redirect&bpo=2135]: Deprecate find_module() and find_loader() implementations in importlib and zipimport.

- bpo-43534 [https://bugs.python.org/issue?@action=redirect&bpo=43534]: **`turtle.textinput()`** and **`turtle.numinput()`** create now a transient window working on behalf of the canvas window.

- bpo-43532 [https://bugs.python.org/issue?@action=redirect&bpo=43532]: Add the ability to specify keyword-only fields to dataclasses. These fields will become keyword-only arguments to the generated __init__.

- bpo-43522 [https://bugs.python.org/issue?@action=redirect&bpo=43522]: Fix problem with **`hostname_checks_common_name`**. OpenSSL does not copy hostflags from *struct SSL_CTX* to *struct SSL*.

- bpo-8978 [https://bugs.python.org/issue?@action=redirect&bpo=8978]: Improve error message for **`tarfile.open()`** when **`lzma`** / **`bz2`** are unavailable. Patch by Anthony Sottile.

- bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967]: Allow **`bytes`** `separator` argument in `urllib.parse.parse_qs` and `urllib.parse.parse_qsl` when parsing **`str`** query strings. Previously, this raised a `TypeError`.

- bpo-43296 [https://bugs.python.org/issue?@action=redirect&bpo=43296]: Improve **`sqlite3`** error handling: `sqlite3_value_blob()` errors that set `SQLITE_NOMEM` now raise **`MemoryError`**. Patch by Erlend E. Aasland.

- bpo-43312 [https://bugs.python.org/issue?@action=redirect&bpo=43312]: New functions **`sysconfig.get_preferred_scheme()`** and **`sysconfig.get_default_scheme()`** are added to query a platform for its preferred “user”, “home”, and “prefix” (default) scheme names.

- bpo-43265 [https://bugs.python.org/issue?@action=redirect&bpo=43265]: Improve **`sqlite3.Connection.backup()`** error handling. The error message for non-existent target database names is now `unknown database <database name>` instead of `SQL logic error`. Patch by Erlend E. Aasland.

- bpo-41282 [https://bugs.python.org/issue?@action=redirect&bpo=41282]: Install schemes in `distutils.command.install` are now loaded from

sysconfig.

- bpo-41282 [https://bugs.python.org/issue?@action=redirect&bpo=41282]: `distutils.sysconfig` has been merged to **sysconfig**.

- bpo-43176 [https://bugs.python.org/issue?@action=redirect&bpo=43176]: Fixed processing of a dataclass that inherits from a frozen dataclass with no fields. It is now correctly detected as an error.

- bpo-43080 [https://bugs.python.org/issue?@action=redirect&bpo=43080]: **pprint** now has support for **dataclasses.dataclass**. Patch by Lewis Gaul.

- bpo-39950 [https://bugs.python.org/issue?@action=redirect&bpo=39950]: Add `pathlib.Path.hardlink_to()` method that supersedes `link_to()`. The new method has the same argument order as `symlink_to()`.

- bpo-42904 [https://bugs.python.org/issue?@action=redirect&bpo=42904]: **typing.get_type_hints()** now checks the local namespace of a class when evaluating **PEP 563** [https://peps.python.org/pep-0563/] annotations inside said class.

- bpo-42269 [https://bugs.python.org/issue?@action=redirect&bpo=42269]: Add `slots` parameter to `dataclasses.dataclass` decorator to automatically generate `__slots__` for class. Patch provided by Yurii Karabas.

- bpo-39529 [https://bugs.python.org/issue?@action=redirect&bpo=39529]: Deprecated use of **asyncio.get_event_loop()** without running event loop. Emit deprecation warning for **asyncio** functions which implicitly create a **Future** or **Task** objects if there is no running event loop and no explicit *loop* argument is passed: **ensure_future()**, **wrap_future()**, **gather()**, **shield()**, **as_completed()** and constructors of **Future**, **Task**, **StreamReader**, **StreamReaderProtocol**.

- bpo-18369 [https://bugs.python.org/issue?@action=redirect&bpo=18369]: Certificate and PrivateKey classes were added to the ssl module. Certificates and keys can now be loaded from memory buffer, too.

- bpo-41486 [https://bugs.python.org/issue?@action=redirect&bpo=41486]: Use a new output buffer management code for **`bz2`** / **`lzma`** / **`zlib`** modules, and add `.readall()` function to `_compression.DecompressReader` class. These bring some performance improvements. Patch by Ma Lin.

- bpo-31870 [https://bugs.python.org/issue?@action=redirect&bpo=31870]: The **`ssl.get_server_certificate()`** function now has a *timeout* parameter.

- bpo-41735 [https://bugs.python.org/issue?@action=redirect&bpo=41735]: Fix thread locks in zlib module may go wrong in rare case. Patch by Ma Lin.

- bpo-36470 [https://bugs.python.org/issue?@action=redirect&bpo=36470]: Fix dataclasses with `InitVars` and **`replace()`**. Patch by Claudiu Popa.

- bpo-40849 [https://bugs.python.org/issue?@action=redirect&bpo=40849]: Expose X509_V_FLAG_PARTIAL_CHAIN ssl flag

- bpo-35114 [https://bugs.python.org/issue?@action=redirect&bpo=35114]: **`ssl.RAND_status()`** now returns a boolean value (as documented) instead of `1` or `0`.

- bpo-39906 [https://bugs.python.org/issue?@action=redirect&bpo=39906]: **`pathlib.Path.stat()`** and **`chmod()`** now accept a *follow_symlinks* keyword-only argument for consistency with corresponding functions in the **`os`** module.

- bpo-39899 [https://bugs.python.org/issue?@action=redirect&bpo=39899]: **`os.path.expanduser()`** now refuses to guess Windows home directories if the basename of current user's home directory does not match their username.

  `pathlib.Path.expanduser()` and **`home()`** now consistently raise **`RuntimeError`** exception when a home directory cannot be resolved. Previously a **`KeyError`** exception could be raised on Windows when the `"USERNAME"` environment variable was unset.

- bpo-36076 [https://bugs.python.org/issue?@action=redirect&bpo=36076]: Added SNI support to `ssl.get_server_certificate()`.

- bpo-38490 [https://bugs.python.org/issue?@action=redirect&bpo=38490]: Covariance, Pearson's correlation, and simple linear regression functionality was added to statistics module. Patch by Tymoteusz Wołodźko.

- bpo-33731 [https://bugs.python.org/issue?@action=redirect&bpo=33731]: Provide a locale.localize() function, which converts a normalized number string into a locale format.

- bpo-32745 [https://bugs.python.org/issue?@action=redirect&bpo=32745]: Fix a regression in the handling of ctypes' `ctypes.c_wchar_p` type: embedded null characters would cause a `ValueError` to be raised. Patch by Zackery Spytz.

Documentação

- bpo-43987 [https://bugs.python.org/issue?@action=redirect&bpo=43987]: Add "Annotations Best Practices" document as a new HOWTO.
- bpo-43977 [https://bugs.python.org/issue?@action=redirect&bpo=43977]: Document the new `Py_TPFLAGS_MAPPING` and `Py_TPFLAGS_SEQUENCE` type flags.
- bpo-43959 [https://bugs.python.org/issue?@action=redirect&bpo=43959]: The documentation on the PyContextVar C-API was clarified.
- bpo-43938 [https://bugs.python.org/issue?@action=redirect&bpo=43938]: Update dataclasses documentation to express that FrozenInstanceError is derived from AttributeError.
- bpo-43778 [https://bugs.python.org/issue?@action=redirect&bpo=43778]: Fix the Sphinx glossary_search extension: create the _static/ sub-directory if it doesn't exist.
- bpo-43755 [https://bugs.python.org/issue?@action=redirect&bpo=43755]: Update documentation to reflect that unparenthesized lambda expressions can no longer be the expression part in an `if` clause in comprehensions and generator expressions since Python 3.9.
- bpo-43739 [https://bugs.python.org/issue?@action=redirect&bpo=43739]: Fixing the example code in Doc/extending/extending.rst to declare and initialize

the pmodule variable to be of the right type.

Testes

- bpo-43961 [https://bugs.python.org/issue?@action=redirect&bpo=43961]: Fix test_logging.test_namer_rotator_inheritance() on Windows: use `os.replace()` rather than `os.rename()`. Patch by Victor Stinner.
- bpo-43842 [https://bugs.python.org/issue?@action=redirect&bpo=43842]: Fix a race condition in the SMTP test of test_logging. Don't close a file descriptor (socket) from a different thread while asyncore.loop() is polling the file descriptor. Patch by Victor Stinner.
- bpo-43843 [https://bugs.python.org/issue?@action=redirect&bpo=43843]: **`test.libregrtest`** now marks a test as ENV_CHANGED (altered the execution environment) if a thread raises an exception but does not catch it. It sets a hook on `threading.excepthook()`. Use `--fail-env-changed` option to mark the test as failed. Patch by Victor Stinner.
- bpo-43811 [https://bugs.python.org/issue?@action=redirect&bpo=43811]: Tests multiple OpenSSL versions on GitHub Actions. Use ccache to speed up testing.
- bpo-43791 [https://bugs.python.org/issue?@action=redirect&bpo=43791]: OpenSSL 3.0.0: Disable testing of legacy protocols TLS 1.0 and 1.1. Tests are failing with TLSV1_ALERT_INTERNAL_ERROR.

Construção

- bpo-43567 [https://bugs.python.org/issue?@action=redirect&bpo=43567]: Improved generated code refresh (AST/tokens/opcodes/keywords) on Windows.
- bpo-43669 [https://bugs.python.org/issue?@action=redirect&bpo=43669]: Implement **PEP 644** [https://peps.python.org/pep-0644/]. Python now requires OpenSSL 1.1.1 or newer.

Windows

- bpo-35306 [https://bugs.python.org/issue?@action=redirect&bpo=35306]: Adds additional arguments to `os.startfile()` function.
- bpo-43538 [https://bugs.python.org/issue?@action=redirect&bpo=43538]: Avoid raising errors from `pathlib.Path.exists()` when passed an invalid

filename.

- bpo-38822 [https://bugs.python.org/issue?@action=redirect&bpo=38822]: Fixed `os.stat()` failing on inaccessible directories with a trailing slash, rather than falling back to the parent directory's metadata. This implicitly affected `os.path.exists()` and `os.path.isdir()`.
- bpo-26227 [https://bugs.python.org/issue?@action=redirect&bpo=26227]: Fixed decoding of host names in `socket.gethostbyaddr()` and `socket.gethostbyname_ex()`.
- bpo-40432 [https://bugs.python.org/issue?@action=redirect&bpo=40432]: Updated pegen regeneration script on Windows to find and use Python 3.8 or higher. Prior to this, pegen regeneration already required 3.8 or higher, but the script may have used lower versions of Python.
- bpo-43745 [https://bugs.python.org/issue?@action=redirect&bpo=43745]: Actually updates Windows release to OpenSSL 1.1.1k. Earlier releases were mislabelled and actually included 1.1.1i again.
- bpo-43652 [https://bugs.python.org/issue?@action=redirect&bpo=43652]: Update Tcl and Tk to 8.6.11 in Windows installer.
- bpo-43492 [https://bugs.python.org/issue?@action=redirect&bpo=43492]: Upgrade Windows installer to use SQLite 3.35.5.
- bpo-30555 [https://bugs.python.org/issue?@action=redirect&bpo=30555]: Fix `WindowsConsoleIO` errors in the presence of fd redirection. Patch by Segev Finer.

macOS

- bpo-42119 [https://bugs.python.org/issue?@action=redirect&bpo=42119]: Fix check for macOS SDK paths when building Python. Narrow search to match contents of SDKs, namely only files in `/System/Library`, `/System/IOSSupport`, and `/usr` other than `/usr/local`. Previously, anything under `/System` was assumed to be in an SDK which causes problems with the new file system layout in 10.15+ where user file systems may appear to be mounted under `/System`. Paths in `/Library` were also incorrectly treated as SDK locations.
- bpo-43568 [https://bugs.python.org/issue?@action=redirect&bpo=43568]: Drop support for MACOSX_DEPLOYMENT_TARGET < 10.3
- bpo-44009 [https://bugs.python.org/issue?@action=redirect&bpo=44009]: Provide "python3.x-intel64" executable to allow reliably forcing macOS

universal2 framework builds to run under Rosetta 2 Intel-64 emulation on Apple Silicon Macs. This can be useful for testing or when universal2 wheels are not yet available.

- bpo-43851 [https://bugs.python.org/issue?@action=redirect&bpo=43851]: Build SQLite with `SQLITE_OMIT_AUTOINIT` on macOS. Patch by Erlend E. Aasland.
- bpo-43492 [https://bugs.python.org/issue?@action=redirect&bpo=43492]: Update macOS installer to use SQLite 3.35.4.
- bpo-42235 [https://bugs.python.org/issue?@action=redirect&bpo=42235]: `Mac/BuildScript/build-installer.py` will now use "–enable-optimizations" and `--with-lto` when building on macOS 10.15 or later.

IDLE

- bpo-37903 [https://bugs.python.org/issue?@action=redirect&bpo=37903]: Add mouse actions to the shell sidebar. Left click and optional drag selects one or more lines, as with the editor line number sidebar. Right click after selecting raises a context menu with 'copy with prompts'. This zips together prompts from the sidebar with lines from the selected text.
- bpo-43981 [https://bugs.python.org/issue?@action=redirect&bpo=43981]: Fix reference leak in test_sidebar and test_squeezer. Patches by Terry Jan Reedy and Pablo Galindo
- bpo-37892 [https://bugs.python.org/issue?@action=redirect&bpo=37892]: Indent IDLE Shell input with spaces instead of tabs
- bpo-43655 [https://bugs.python.org/issue?@action=redirect&bpo=43655]: IDLE dialog windows are now recognized as dialogs by window managers on macOS and X Window.
- bpo-37903 [https://bugs.python.org/issue?@action=redirect&bpo=37903]: IDLE's shell now shows prompts in a separate side-bar.

API C

- bpo-43916 [https://bugs.python.org/issue?@action=redirect&bpo=43916]: Add a new **`Py_TPFLAGS_DISALLOW_INSTANTIATION`** type flag to disallow creating type instances. Patch by Victor Stinner.

- bpo-43774 [https://bugs.python.org/issue?@action=redirect&bpo=43774]: Remove the now unused `PYMALLOC_DEBUG` macro. Debug hooks on memory allocators are now installed by default if Python is built in debug mode (if `Py_DEBUG` macro is defined). Moreover, they can now be used on Python build in release mode (ex: using `PYTHONMALLOC=debug` environment variable).
- bpo-43962 [https://bugs.python.org/issue?@action=redirect&bpo=43962]: _PyInterpreterState_IDIncref() now calls _PyInterpreterState_IDInitref() and always increments id_refcount. Previously, calling _xxsubinterpreters.get_current() could create an id_refcount inconsistency when a _xxsubinterpreters.InterpreterID object was deallocated. Patch by Victor Stinner.
- bpo-28254 [https://bugs.python.org/issue?@action=redirect&bpo=28254]: Add new C-API functions to control the state of the garbage collector: **`PyGC_Enable()`**, **`PyGC_Disable()`**, **`PyGC_IsEnabled()`**, corresponding to the functions in the **`gc`** module.
- bpo-43908 [https://bugs.python.org/issue?@action=redirect&bpo=43908]: Introduce **`Py_TPFLAGS_IMMUTABLETYPE`** flag for immutable type objects, and modify **`PyType_Ready()`** to set it for static types. Patch by Erlend E. Aasland.
- bpo-43795 [https://bugs.python.org/issue?@action=redirect&bpo=43795]: **`PyMem_Calloc()`** is now available in the limited C API (`Py_LIMITED_API`).
- bpo-43868 [https://bugs.python.org/issue?@action=redirect&bpo=43868]: **`PyOS_ReadlineFunctionPointer()`** is no longer exported by limited C API headers and by `python3.dll` on Windows. Like any function that takes `FILE*`, it is not part of the stable ABI.
- bpo-43795 [https://bugs.python.org/issue?@action=redirect&bpo=43795]: Stable ABI and limited API definitions are generated from a central manifest (**PEP 652** [https://peps.python.org/pep-0652/]).
- bpo-43753 [https://bugs.python.org/issue?@action=redirect&bpo=43753]: Add the **`Py_Is(x, y)`** function to test if the *x* object is the *y* object, the same as `x is y` in Python. Add also the **`Py_IsNone()`**, **`Py_IsTrue()`**, **`Py_IsFalse()`** functions to test if an object is, respectively, the `None` singleton, the `True` singleton or the `False` singleton. Patch by Victor Stinner.

Python 3.10.0 alpha 7

Data de lançamento: 2021-04-05

Segurança

- bpo-42988 [https://bugs.python.org/issue?@action=redirect&bpo=42988]: **CVE 2021-3426** [https://www.cve.org/CVERecord?id=CVE-2021-3426]: Remove the `getfile` feature of the **`pydoc`** module which could be abused to read arbitrary files on the disk (directory traversal vulnerability). Moreover, even source code of Python modules can contain sensitive data like passwords. Vulnerability reported by David Schwörer.

- bpo-43285 [https://bugs.python.org/issue?@action=redirect&bpo=43285]: **`ftplib`** no longer trusts the IP address value returned from the server in response to the PASV command by default. This prevents a malicious FTP server from using the response to probe IPv4 address and port combinations on the client network.

  Code that requires the former vulnerable behavior may set a `trust_server_pasv_ipv4_address` attribute on their **`ftplib.FTP`** instances to `True` to re-enable it.

- bpo-43439 [https://bugs.python.org/issue?@action=redirect&bpo=43439]: Add audit hooks for **`gc.get_objects()`**, **`gc.get_referrers()`** and **`gc.get_referents()`**. Patch by Pablo Galindo.

Núcleo e embutidos

- bpo-27129 [https://bugs.python.org/issue?@action=redirect&bpo=27129]: Update CPython bytecode magic number.
- bpo-43672 [https://bugs.python.org/issue?@action=redirect&bpo=43672]: Raise ImportWarning when calling find_loader().
- bpo-43660 [https://bugs.python.org/issue?@action=redirect&bpo=43660]: Fix crash that happens when replacing `sys.stderr` with a callable that can remove the object while an exception is being printed. Patch by Pablo Galindo.

- bpo-27129 [https://bugs.python.org/issue?@action=redirect&bpo=27129]: The bytecode interpreter uses instruction, rather byte, offsets internally. This reduces the number of EXTENDED_ARG instructions needed and streamlines instruction dispatch a bit.

Biblioteca

- bpo-40645 [https://bugs.python.org/issue?@action=redirect&bpo=40645]: Fix reference leak in the **`_hashopenssl`** extension. Patch by Pablo Galindo.

Núcleo e embutidos

- bpo-42134 [https://bugs.python.org/issue?@action=redirect&bpo=42134]: Calls to find_module() by the import system now raise ImportWarning.

- bpo-41064 [https://bugs.python.org/issue?@action=redirect&bpo=41064]: Improve the syntax error for invalid usage of double starred elements ('**') in f-strings. Patch by Pablo Galindo.

- bpo-43575 [https://bugs.python.org/issue?@action=redirect&bpo=43575]: Speed up calls to `map()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

- bpo-42137 [https://bugs.python.org/issue?@action=redirect&bpo=42137]: The import system now prefers using `__spec__` for `ModuleType.__repr__` over `module_repr()`.

- bpo-43452 [https://bugs.python.org/issue?@action=redirect&bpo=43452]: Added micro-optimizations to `_PyType_Lookup()` to improve cache lookup performance in the common case of cache hits.

- bpo-43555 [https://bugs.python.org/issue?@action=redirect&bpo=43555]: Report the column offset for **`SyntaxError`** for invalid line continuation characters. Patch by Pablo Galindo.

- bpo-43517 [https://bugs.python.org/issue?@action=redirect&bpo=43517]: Fix misdetection of circular imports when using `from pkg.mod import attr`, which caused false positives in non-trivial multi-threaded code.

- bpo-43497 [https://bugs.python.org/issue?@action=redirect&bpo=43497]: Emit SyntaxWarnings for assertions with tuple constants, this is a regression introduced in python3.7

- bpo-39316 [https://bugs.python.org/issue?@action=redirect&bpo=39316]: Tracing now has correct line numbers for attribute accesses when the attribute is on a different line from the object. Improves debugging and profiling for multi-line method chains.

- bpo-35883 [https://bugs.python.org/issue?@action=redirect&bpo=35883]: Python no longer fails at startup with a fatal error if a command line argument contains an invalid Unicode character. The **`Py_DecodeLocale()`** function now escapes byte sequences which would be decoded as Unicode characters outside the [U+0000; U+10ffff] range.

- bpo-43410 [https://bugs.python.org/issue?@action=redirect&bpo=43410]: Fix a bug that was causing the parser to crash when emitting syntax errors when reading input from stdin. Patch by Pablo Galindo

- bpo-43406 [https://bugs.python.org/issue?@action=redirect&bpo=43406]: Fix a possible race condition where `PyErr_CheckSignals` tries to execute a non-Python signal handler.

- bpo-42128 [https://bugs.python.org/issue?@action=redirect&bpo=42128]: Add `__match_args__` to struct sequence objects. Patch by Pablo Galindo.

- bpo-43390 [https://bugs.python.org/issue?@action=redirect&bpo=43390]: CPython now sets the `SA_ONSTACK` flag in `PyOS_setsig` for the VM's default signal handlers. This is friendlier to other in-process code that an extension module or embedding use could pull in (such as Golang's cgo) where tiny thread stacks are the norm and `sigaltstack()` has been used to provide for signal handlers. This is a no-op change for the vast majority of processes that don't use sigaltstack.

- bpo-43287 [https://bugs.python.org/issue?@action=redirect&bpo=43287]: Speed up calls to `filter()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

- bpo-37448 [https://bugs.python.org/issue?@action=redirect&bpo=37448]: Add a radix tree based memory map to track in-use obmalloc arenas. Use to replace the old implementation of address_in_range(). The radix tree approach makes it easy to increase pool sizes beyond the OS page size. Boosting the pool and arena size allows obmalloc to handle a significantly higher percentage of requests from its ultra-fast paths.

  It also has the advantage of eliminating the memory unsanitary behavior of the previous address_in_range(). The old address_in_range() was marked with the annotations _Py_NO_SANITIZE_ADDRESS, _Py_NO_SANITIZE_THREAD, and _Py_NO_SANITIZE_MEMORY. Those annotations are no longer needed.

  To disable the radix tree map, set a preprocessor flag as follows: `-DWITH_PYMALLOC_RADIX_TREE=0`.

  Coautoria de: Tim Peters <tim.peters@gmail.com>

- bpo-29988 [https://bugs.python.org/issue?@action=redirect&bpo=29988]: Only handle asynchronous exceptions and requests to drop the GIL when returning from a call or on the back edges of loops. Makes sure that `__exit__()` is always called in with statements, even for interrupts.

Biblioteca

- bpo-43720 [https://bugs.python.org/issue?@action=redirect&bpo=43720]: Document various stdlib deprecations in imp, pkgutil, and importlib.util for removal in Python 3.12.

- bpo-43433 [https://bugs.python.org/issue?@action=redirect&bpo=43433]: `xmlrpc.client.ServerProxy` no longer ignores query and fragment in the URL of the server.

- bpo-31956 [https://bugs.python.org/issue?@action=redirect&bpo=31956]: The `index()` method of `array.array` now has optional *start* and *stop* parameters.

- bpo-40066 [https://bugs.python.org/issue?@action=redirect&bpo=40066]: Enum: adjust `repr()` to show only enum and member name (not value, nor angle brackets) and `str()` to show only member name. Update and improve documentation to match.

- bpo-42136 [https://bugs.python.org/issue?@action=redirect&bpo=42136]: Deprecate all module_repr() methods found in importlib as their use is being phased out by Python 3.12.

- bpo-35930 [https://bugs.python.org/issue?@action=redirect&bpo=35930]: Raising an exception raised in a "future" instance will create reference cycles.

- bpo-41369 [https://bugs.python.org/issue?@action=redirect&bpo=41369]: Finish updating the vendored libmpdec to version 2.5.1. Patch by Stefan Krah.

- bpo-43422 [https://bugs.python.org/issue?@action=redirect&bpo=43422]: Revert the _decimal C API which was added in bpo-41324 [https://bugs.python.org/issue?@action=redirect&bpo=41324].

- bpo-43577 [https://bugs.python.org/issue?@action=redirect&bpo=43577]: Fix deadlock when using **`ssl.SSLContext`** debug callback with **`ssl.SSLContext.sni_callback()`**.

- bpo-43571 [https://bugs.python.org/issue?@action=redirect&bpo=43571]: It's now possible to create MPTCP sockets with IPPROTO_MPTCP

- bpo-43542 [https://bugs.python.org/issue?@action=redirect&bpo=43542]: `image/heic` and `image/heif` were added to **`mimetypes`**.

- bpo-40645 [https://bugs.python.org/issue?@action=redirect&bpo=40645]: The **`hmac`** module now uses OpenSSL's HMAC implementation when digestmod argument is a hash name or builtin hash function.

- bpo-43510 [https://bugs.python.org/issue?@action=redirect&bpo=43510]: Implement **PEP 597** [https://peps.python.org/pep-0597/]: Add `EncodingWarning` warning, `-X warn_default_encoding` option, **`PYTHONWARNDEFAULTENCODING`** environment variable and `encoding="locale"` argument value.

- bpo-43521 [https://bugs.python.org/issue?@action=redirect&bpo=43521]: `ast.unparse` can now render NaNs and empty sets.

- bpo-42914 [https://bugs.python.org/issue?@action=redirect&bpo=42914]: **`pprint.pprint()`** gains a new boolean `underscore_numbers` optional argument to emit integers with thousands separated by an underscore character for improved readability (for example `1_000_000` instead of `1000000`).

- bpo-41361 [https://bugs.python.org/issue?@action=redirect&bpo=41361]: **`rotate()`** calls are now slightly faster due to faster argument parsing.

- bpo-43423 [https://bugs.python.org/issue?@action=redirect&bpo=43423]: **`subprocess.communicate()`** no longer raises an IndexError when there is an empty stdout or stderr IO buffer during a timeout on Windows.

- bpo-27820 [https://bugs.python.org/issue?@action=redirect&bpo=27820]: Fixed long-standing bug of smtplib.SMTP where doing AUTH LOGIN with initial_response_ok=False will fail.

  The cause is that SMTP.auth_login _always_ returns a password if provided with a challenge string, thus non-compliant with the standard for AUTH LOGIN.

  Also fixes bug with the test for smtpd.

- bpo-43445 [https://bugs.python.org/issue?@action=redirect&bpo=43445]: Add frozen modules to **`sys.stdlib_module_names`**. For example, add `"_frozen_importlib"` and `"_frozen_importlib_external"` names.

- bpo-43245 [https://bugs.python.org/issue?@action=redirect&bpo=43245]: Add keyword arguments support to `ChainMap.new_child()`.

- bpo-29982 [https://bugs.python.org/issue?@action=redirect&bpo=29982]: Add optional parameter *ignore_cleanup_errors* to **`tempfile.TemporaryDirectory()`** and allow multiple **`cleanup()`** attempts. Contributed by C.A.M. Gerlach.

- bpo-43428 [https://bugs.python.org/issue?@action=redirect&bpo=43428]: Include changes from importlib_metadata 3.7 [https://importlib-metadata.readthedocs.io/en/latest/history.html#v3-7-0]:

  Performance enhancements to distribution discovery.

  `entry_points` only returns unique distributions.

  Introduces new `EntryPoints` object for containing a set of entry points with convenience methods for selecting entry points by group or name. `entry_points` now returns this object if selection parameters are supplied but continues to return a dict object for compatibility. Users are encouraged to rely on the selection interface. The dict object result is likely to be deprecated in the future.

  Added packages_distributions function to return a mapping of packages to the distributions that provide them.

- bpo-43332 [https://bugs.python.org/issue?@action=redirect&bpo=43332]: Improves the networking efficiency of **`http.client`** when using a proxy via **`set_tunnel()`**. Fewer small send calls are made during connection setup.

- bpo-43420 [https://bugs.python.org/issue?@action=redirect&bpo=43420]: Improve performance of **`fractions.Fraction`** arithmetics for large components. Contributed by Sergey B. Kirpichev.

- bpo-43356 [https://bugs.python.org/issue?@action=redirect&bpo=43356]: Allow passing a signal number to `_thread.interrupt_main()`.

- bpo-43399 [https://bugs.python.org/issue?@action=redirect&bpo=43399]: Fix `ElementTree.extend` not working on iterators when using the Python implementation

- bpo-43369 [https://bugs.python.org/issue?@action=redirect&bpo=43369]: Improve **`sqlite3`** error handling: If `sqlite3_column_text()` and `sqlite3_column_blob()` set `SQLITE_NOMEM`, **`MemoryError`** is now raised. Patch by Erlend E. Aasland.

- bpo-43368 [https://bugs.python.org/issue?@action=redirect&bpo=43368]: Fix a regression introduced in PR 24562, where an empty bytestring was fetched as `None` instead of `b''` in **`sqlite3`**. Patch by Mariusz Felisiak.

- bpo-41282 [https://bugs.python.org/issue?@action=redirect&bpo=41282]: Fixed stacklevel of `DeprecationWarning` emitted from `import distutils`.

- bpo-42129 [https://bugs.python.org/issue?@action=redirect&bpo=42129]: `importlib.resources` now honors namespace packages, merging resources from each location in the namespace as introduced in `importlib_resources` 3.2 and including incidental changes through 5.0.3.

- bpo-43295 [https://bugs.python.org/issue?@action=redirect&bpo=43295]: **`datetime.datetime.strptime()`** now raises `ValueError` instead of `IndexError` when matching `'z'` with the `%z` format specifier.

- bpo-43125 [https://bugs.python.org/issue?@action=redirect&bpo=43125]: Return empty string if base64mime.body_encode receive empty bytes

- bpo-43084 [https://bugs.python.org/issue?@action=redirect&bpo=43084]: **`curses.window.enclose()`** returns now `True` or `False` (as was documented) instead of `1` or `0`.

- bpo-42994 [https://bugs.python.org/issue?@action=redirect&bpo=42994]: Add MIME types for opus, AAC, 3gpp and 3gpp2

- bpo-14678 [https://bugs.python.org/issue?@action=redirect&bpo=14678]: Add an invalidate_caches() method to the zipimport.zipimporter class to support importlib.invalidate_caches(). Patch by Desmond Cheong.

- bpo-42782 [https://bugs.python.org/issue?@action=redirect&bpo=42782]: Fail fast in **`shutil.move()`** to avoid creating destination directories on failure.

- bpo-40066 [https://bugs.python.org/issue?@action=redirect&bpo=40066]: Enum's `repr()` and `str()` have changed: `repr()` is now *EnumClass.MemberName* and `str()` is *MemberName*. Additionally, stdlib Enum's whose contents are available as module attributes, such

as `RegexFlag.IGNORECASE`, have their `repr()` as *module.name*, e.g. `re.IGNORECASE`.

- bpo-26053 [https://bugs.python.org/issue?@action=redirect&bpo=26053]: Fixed bug where the **`pdb`** interactive run command echoed the args from the shell command line, even if those have been overridden at the pdb prompt.

- bpo-24160 [https://bugs.python.org/issue?@action=redirect&bpo=24160]: Fixed bug where breakpoints did not persist across multiple debugger sessions in **`pdb`**'s interactive mode.

- bpo-40701 [https://bugs.python.org/issue?@action=redirect&bpo=40701]: When the **`tempfile.tempdir`** global variable is set to a value of type bytes, it is now handled consistently. Previously exceptions could be raised from some tempfile APIs when the directory did not already exist in this situation. Also ensures that the **`tempfile.gettempdir()`** and **`tempfile.gettempdirb()`** functions *always* return `str` and `bytes` respectively.

- bpo-39342 [https://bugs.python.org/issue?@action=redirect&bpo=39342]: Expose `X509_V_FLAG_ALLOW_PROXY_CERTS` as **`VERIFY_ALLOW_PROXY_CERTS`** to allow proxy certificate validation as explained in https://docs.openssl.org/1.1.1/man7/proxy-certificates/.

- bpo-31861 [https://bugs.python.org/issue?@action=redirect&bpo=31861]: Add builtins.aiter and builtins.anext. Patch by Joshua Bronson (@jab), Daniel Pope (@lordmauve), and Justin Wang (@justin39).

Documentação

- bpo-43199 [https://bugs.python.org/issue?@action=redirect&bpo=43199]: Answer "Why is there no goto?" in the Design and History FAQ.
- bpo-43407 [https://bugs.python.org/issue?@action=redirect&bpo=43407]: Clarified that a result from **`time.monotonic()`**, **`time.perf_counter()`**, **`time.process_time()`**, or **`time.thread_time()`** can be compared with the result from any following call to the same function - not just the next immediate call.

- bpo-43354 [https://bugs.python.org/issue?@action=redirect&bpo=43354]: Fix type documentation for `Fault.faultCode`; the type has to be `int` instead of `str`.
- bpo-41933 [https://bugs.python.org/issue?@action=redirect&bpo=41933]: Clarified wording of s * n in the Common Sequence Operations

Testes

- bpo-37945 [https://bugs.python.org/issue?@action=redirect&bpo=37945]: Fix test_getsetlocale_issue1813() of test_locale: skip the test if `setlocale()` fails. Patch by Victor Stinner.
- bpo-41561 [https://bugs.python.org/issue?@action=redirect&bpo=41561]: Add workaround for Ubuntu's custom OpenSSL security level policy.

Construção

- bpo-43179 [https://bugs.python.org/issue?@action=redirect&bpo=43179]: Introduce and correctly use ALIGNOF_X in place of SIZEOF_X for alignment-related code in optimized string routines. Patch by Jessica Clarke.
- bpo-43631 [https://bugs.python.org/issue?@action=redirect&bpo=43631]: Update macOS, Windows, and CI to OpenSSL 1.1.1k.
- bpo-43617 [https://bugs.python.org/issue?@action=redirect&bpo=43617]: Improve configure.ac: Check for presence of autoconf-archive package and remove our copies of M4 macros.
- bpo-43466 [https://bugs.python.org/issue?@action=redirect&bpo=43466]: The `configure` script now supports `--with-openssl-rpath` option.
- bpo-43372 [https://bugs.python.org/issue?@action=redirect&bpo=43372]: Use `_freeze_importlib` to generate code for the `__hello__` module. This approach ensures the code matches the interpreter version. Previously, PYTHON_FOR_REGEN was used to generate the code, which might be wrong. The marshal format for code objects has changed with bpo-42246 [https://bugs.python.org/issue?@action=redirect&bpo=42246], commit 877df851. Update the code and the expected code sizes in ctypes test_frozentable.

Windows

- bpo-43440 [https://bugs.python.org/issue?@action=redirect&bpo=43440]: Build **`sqlite3`** with the `R*Tree` module enabled. Patch by Erlend E. Aasland.

IDLE

- bpo-42225 [https://bugs.python.org/issue?@action=redirect&bpo=42225]: Document that IDLE can fail on Unix either from misconfigured IP masquerade rules or failure displaying complex colored (non-ascii) characters.

API C

- bpo-43688 [https://bugs.python.org/issue?@action=redirect&bpo=43688]: The limited C API is now supported if Python is built in debug mode (if the `Py_DEBUG` macro is defined). In the limited C API, the **`Py_INCREF()`** and **`Py_DECREF()`** functions are now implemented as opaque function calls, rather than accessing directly the **`PyObject.ob_refcnt`** member, if Python is built in debug mode and the `Py_LIMITED_API` macro targets Python 3.10 or newer. It became possible to support the limited C API in debug mode because the **`PyObject`** structure is the same in release and debug mode since Python 3.8 (see bpo-36465 [https://bugs.python.org/issue?@action=redirect&bpo=36465]).

  The limited C API is still not supported in the `--with-trace-refs` special build (`Py_TRACE_REFS` macro).

  Patch por Victor Stinner.

- bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244]: Remove the `pyarena.h` header file with functions:

  - `PyArena_New()`
  - `PyArena_Free()`
  - `PyArena_Malloc()`
  - `PyArena_AddPyObject()`

  These functions were undocumented, excluded from the limited C API, and were only used internally by the compiler. Patch by Victor

Stinner.

- bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244]: Remove the compiler and parser functions using `struct _mod` type, because the public AST C API was removed:

  - `PyAST_Compile()`
  - `PyAST_CompileEx()`
  - `PyAST_CompileObject()`
  - `PyFuture_FromAST()`
  - `PyFuture_FromASTObject()`
  - `PyParser_ASTFromFile()`
  - `PyParser_ASTFromFileObject()`
  - `PyParser_ASTFromFilename()`
  - `PyParser_ASTFromString()`
  - `PyParser_ASTFromStringObject()`

  These functions were undocumented and excluded from the limited C API. Patch by Victor Stinner.

- bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244]: Remove `ast.h`, `asdl.h`, and `Python-ast.h` header files. These functions were undocumented and excluded from the limited C API. Most names defined by these header files were not prefixed by `Py` and so could create names conflicts. For example, `Python-ast.h` defined a `Yield` macro which was conflict with the `Yield` name used by the Windows `<winbase.h>` header. Use the Python **`ast`** module instead. Patch by Victor Stinner.

- bpo-43541 [https://bugs.python.org/issue?@action=redirect&bpo=43541]: Fix a `PyEval_EvalCodeEx()` regression: fix reference counting on builtins. Patch by Victor Stinner.

- bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244]: Remove the `symtable.h` header file and the undocumented functions:

  - `PyST_GetScope()`
  - `PySymtable_Build()`

  - `PySymtable_BuildObject()`
  - `PySymtable_Free()`
  - `Py_SymtableString()`
  - `Py_SymtableStringObject()`

  A função `Py_SymtableString()` fazia parte da ABI estável por engano, mas não pôde ser usada, porque o arquivo de cabeçalho `symtable.h` foi excluído da API C limitada.

  The Python **`symtable`** module remains available and is unchanged.

  Patch por Victor Stinner.

- bpo-43244 [https://bugs.python.org/issue?@action=redirect&bpo=43244]: Remove the `PyAST_Validate()` function. It is no longer possible to build a AST object (`mod_ty` type) with the public C API. The function was already excluded from the limited C API (**PEP 384** [https://peps.python.org/pep-0384/]). Patch by Victor Stinner.

Python 3.10.0 alpha 6

Data de lançamento: 2021-03-01

Segurança

- bpo-42967 [https://bugs.python.org/issue?@action=redirect&bpo=42967]: Fix web cache poisoning vulnerability by defaulting the query args separator to `&`, and allowing the user to choose a custom separator.

Núcleo e embutidos

- bpo-43321 [https://bugs.python.org/issue?@action=redirect&bpo=43321]: Fix `SystemError` raised when `PyArg_Parse*()` is used with `#` but without `PY_SSIZE_T_CLEAN` defined.
- bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346]: `PyArg_Parse*()` functions now emits `DeprecationWarning` when `u` or `Z` format is used. See **PEP 623** [https://peps.python.org/pep-0623/] for detail.

- bpo-43277 [https://bugs.python.org/issue?@action=redirect&bpo=43277]: Add a new **`PySet_CheckExact()`** function to the C-API to check if an object is an instance of **`set`** but not an instance of a subtype. Patch by Pablo Galindo.
- bpo-42990 [https://bugs.python.org/issue?@action=redirect&bpo=42990]: The **`types.FunctionType`** constructor now inherits the current builtins if the *globals* dictionary has no `"__builtins__"` key, rather than using `{"None": None}` as builtins: same behavior as **`eval()`** and **`exec()`** functions. Defining a function with `def function(...): ...` in Python is not affected, globals cannot be overridden with this syntax: it also inherits the current builtins. Patch by Victor Stinner.
- bpo-42990 [https://bugs.python.org/issue?@action=redirect&bpo=42990]: Functions have a new `__builtins__` attribute which is used to look for builtin symbols when a function is executed, instead of looking into `__globals__['__builtins__']`. Patch by Mark Shannon and Victor Stinner.
- bpo-43149 [https://bugs.python.org/issue?@action=redirect&bpo=43149]: Improve the error message in the parser for exception groups without parentheses. Patch by Pablo Galindo.
- bpo-43121 [https://bugs.python.org/issue?@action=redirect&bpo=43121]: Fixed an incorrect **`SyntaxError`** message for missing comma in literals. Patch by Pablo Galindo.

Biblioteca

- bpo-42819 [https://bugs.python.org/issue?@action=redirect&bpo=42819]: **`readline`**: Explicitly disable bracketed paste in the interactive interpreter, even if it's set in the inputrc, is enabled by default (eg GNU Readline 8.1), or a user calls `readline.read_init_file()`. The Python REPL has not implemented bracketed paste support. Also, bracketed mode writes the `"\x1b[?2004h"` escape sequence into stdout which causes test failures in applications that don't support it. It can still be explicitly enabled by calling `readline.parse_and_bind("set enable-bracketed-paste on")`. Patch by Dustin Rodrigues.

Núcleo e embutidos

- bpo-42808 [https://bugs.python.org/issue?@action=redirect&bpo=42808]: Simple calls to `type(object)` are now faster due to the `vectorcall` calling convention. Patch by Dennis Sweeney.
- bpo-42217 [https://bugs.python.org/issue?@action=redirect&bpo=42217]: Make the compiler merges same co_code and co_linetable objects in a module like already did for co_consts.
- bpo-41972 [https://bugs.python.org/issue?@action=redirect&bpo=41972]: Substring search functions such as `str1 in str2` and `str2.find(str1)` now sometimes use the "Two-Way" string comparison algorithm to avoid quadratic behavior on long strings.
- bpo-42128 [https://bugs.python.org/issue?@action=redirect&bpo=42128]: Implement **PEP 634** [https://peps.python.org/pep-0634/] (structural pattern matching). Patch by Brandt Bucher.

Biblioteca

- bpo-40692 [https://bugs.python.org/issue?@action=redirect&bpo=40692]: In the **`concurrent.futures.ProcessPoolExecutor`**, validate that **`multiprocess.synchronize()`** is available on a given platform and rely on that check in the **`concurrent.futures`** test suite so we can run tests that are unrelated to **`ProcessPoolExecutor`** on those platforms.

Núcleo e embutidos

- bpo-38302 [https://bugs.python.org/issue?@action=redirect&bpo=38302]: If **`object.__ipow__()`** returns **`NotImplemented`**, the operator will correctly fall back to **`object.__pow__()`** and **`object.__rpow__()`** as expected.

Biblioteca

- bpo-43316 [https://bugs.python.org/issue?@action=redirect&bpo=43316]: The `python -m gzip` command line application now properly fails when detecting an unsupported extension. It exits with a non-zero exit code and prints an error message to stderr.
- bpo-43317 [https://bugs.python.org/issue?@action=redirect&bpo=43317]: Set the chunk size for the `gzip` module main function to

io.DEFAULT_BUFFER_SIZE. This is slightly faster than the 1024 bytes constant that was used previously.

- bpo-43146 [https://bugs.python.org/issue?@action=redirect&bpo=43146]: Handle None in single-arg versions of `print_exception()` and `format_exception()`.
- bpo-43260 [https://bugs.python.org/issue?@action=redirect&bpo=43260]: Fix TextIOWrapper can not flush internal buffer forever after very large text is written.
- bpo-43258 [https://bugs.python.org/issue?@action=redirect&bpo=43258]: Prevent needless allocation of `sqlite3` aggregate function context when no rows match an aggregate query. Patch by Erlend E. Aasland.
- bpo-43251 [https://bugs.python.org/issue?@action=redirect&bpo=43251]: Improve `sqlite3` error handling: `sqlite3_column_name()` failures now result in `MemoryError`. Patch by Erlend E. Aasland.
- bpo-40956 [https://bugs.python.org/issue?@action=redirect&bpo=40956]: Fix segfault in `sqlite3.Connection.backup()` if no argument was provided. The regression was introduced by PR 23838. Patch by Erlend E. Aasland.
- bpo-43172 [https://bugs.python.org/issue?@action=redirect&bpo=43172]: The readline module now passes its tests when built directly against libedit. Existing irreconcilable API differences remain in `readline.get_begidx()` and `readline.get_endidx()` behavior based on libreadline vs libedit use.
- bpo-43163 [https://bugs.python.org/issue?@action=redirect&bpo=43163]: Fix a bug in `codeop` that was causing it to not ask for more input when multi-line snippets have unclosed parentheses. Patch by Pablo Galindo
- bpo-43162 [https://bugs.python.org/issue?@action=redirect&bpo=43162]: deprecate unsupported ability to access enum members as attributes of other enum members
- bpo-43146 [https://bugs.python.org/issue?@action=redirect&bpo=43146]: Fix recent regression in None argument handling in `traceback` module functions.
- bpo-43102 [https://bugs.python.org/issue?@action=redirect&bpo=43102]: The namedtuple __new__ method had its __builtins__ set to None instead of an actual dictionary. This created problems for introspection tools.
- bpo-43106 [https://bugs.python.org/issue?@action=redirect&bpo=43106]: Added `O_EVTONLY`, `O_FSYNC`, `O_SYMLINK` and `O_NOFOLLOW_ANY` for macOS. Patch by Donghee Na.

- bpo-42960 [https://bugs.python.org/issue?@action=redirect&bpo=42960]: Adds **`resource.RLIMIT_KQUEUES`** constant from FreeBSD to the **`resource`** module.
- bpo-42151 [https://bugs.python.org/issue?@action=redirect&bpo=42151]: Make the pure Python implementation of **`xml.etree.ElementTree`** behave the same as the C implementation (**`_elementtree`**) regarding default attribute values (by not setting `specified_attributes=1`).
- bpo-29753 [https://bugs.python.org/issue?@action=redirect&bpo=29753]: In ctypes, now packed bitfields are calculated properly and the first item of packed bitfields is now shrank correctly.

Documentação

- bpo-27646 [https://bugs.python.org/issue?@action=redirect&bpo=27646]: Clarify that ‘yield from <expr>’ works with any iterable, not just iterators.
- bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346]: Update some deprecated unicode APIs which are documented as “will be removed in 4.0” to “3.12”. See **PEP 623** [https://peps.python.org/pep-0623/] for detail.

Testes

- bpo-43288 [https://bugs.python.org/issue?@action=redirect&bpo=43288]: Fix test_importlib to correctly skip Unicode file tests if the filesystem does not support them.

Construção

- bpo-43174 [https://bugs.python.org/issue?@action=redirect&bpo=43174]: Windows build now uses `/utf-8` compiler option.

- bpo-43103 [https://bugs.python.org/issue?@action=redirect&bpo=43103]: Add a new configure `--without-static-libpython` option to not build the `libpythonMAJOR.MINOR.a` static library and not install the `python.o` object file.

- bpo-13501 [https://bugs.python.org/issue?@action=redirect&bpo=13501]: The configure script can now use *libedit* instead of *readline* with the command line option `--with-readline=editline`.

- bpo-42603 [https://bugs.python.org/issue?@action=redirect&bpo=42603]: Make configure script use pkg-config to detect the location of Tcl/Tk headers and libraries, used to build tkinter.

  On macOS, a Tcl/Tk configuration provided by pkg-config will be preferred over Tcl/Tk frameworks installed in `/{System/,}Library/Frameworks`. If both exist and the latter is preferred, the appropriate `--with-tcltk-*` configuration options need to be explicitly set.

- bpo-39448 [https://bugs.python.org/issue?@action=redirect&bpo=39448]: Add the "regen-frozen" makefile target that regenerates the code for the frozen `__hello__` module.

Windows

- bpo-43155 [https://bugs.python.org/issue?@action=redirect&bpo=43155]: **`PyCMethod_New()`** is now present in `python3.lib`.

macOS

- bpo-41837 [https://bugs.python.org/issue?@action=redirect&bpo=41837]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.1j.

IDLE

- bpo-43283 [https://bugs.python.org/issue?@action=redirect&bpo=43283]: Document why printing to IDLE's Shell is often slower than printing to a system terminal and that it can be made faster by pre-formatting a single string before printing.

API C

- bpo-43278 [https://bugs.python.org/issue?@action=redirect&bpo=43278]: Always put compiler and system information on the first line of the REPL welcome message.
- bpo-43270 [https://bugs.python.org/issue?@action=redirect&bpo=43270]: Remove the private `_PyErr_OCCURRED()` macro: use the public **`PyErr_Occurred()`** function instead.
- bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134]: Move odictobject.h, parser_interface.h, picklebufobject.h, pydebug.h, and pyfpe.h into the cpython/ directory. They must not be included directly, as they are already included by Python.h: Include Files.
- bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134]: Move pyarena.h, pyctype.h, and pytime.h into the cpython/ directory. They must not be included directly, as they are already included by Python.h: Include Files.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: **`PyExceptionClass_Name()`** is now always declared as a function, in order to hide implementation details. The macro accessed **`PyTypeObject.tp_name`** directly. Patch by Erlend E. Aasland.
- bpo-43239 [https://bugs.python.org/issue?@action=redirect&bpo=43239]: The **`PyCFunction_New()`** function is now exported in the ABI when compiled with `-fvisibility=hidden`.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: **`PyIter_Check()`** is now always declared as a function, in order to hide implementation details. The macro accessed **`PyTypeObject.tp_iternext`** directly. Patch by Erlend E. Aasland.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Convert **`PyDescr_IsData()`** macro to a function to hide implementation details: The macro accessed **`PyTypeObject.tp_descr_set`** directly. Patch by Erlend E. Aasland.
- bpo-43181 [https://bugs.python.org/issue?@action=redirect&bpo=43181]: Convert **`PyObject_TypeCheck()`** macro to a static inline function. Patch by Erlend E. Aasland.

Python 3.10.0 alpha 5

Data de lançamento: 2021-02-02

Segurança

- bpo-42938 [https://bugs.python.org/issue?@action=redirect&bpo=42938]: Avoid static buffers when computing the repr of `ctypes.c_double` and `ctypes.c_longdouble` values.

Núcleo e embutidos

- bpo-42990 [https://bugs.python.org/issue?@action=redirect&bpo=42990]: Refactor the `PyEval_` family of functions.
  - An new function `_PyEval_Vector` is added to simplify calls to Python from C.
  - `_PyEval_EvalCodeWithName` is removed
  - `PyEval_EvalCodeEx` is retained as part of the API, but is not used internally
- bpo-38631 [https://bugs.python.org/issue?@action=redirect&bpo=38631]: Replace `Py_FatalError()` calls in the compiler with regular `SystemError` exceptions. Patch by Victor Stinner.
- bpo-42997 [https://bugs.python.org/issue?@action=redirect&bpo=42997]: Improve error message for missing ":" before blocks. Patch by Pablo Galindo.
- bpo-43017 [https://bugs.python.org/issue?@action=redirect&bpo=43017]: Improve error message in the parser when using un-parenthesised tuples in comprehensions. Patch by Pablo Galindo.
- bpo-42986 [https://bugs.python.org/issue?@action=redirect&bpo=42986]: Fix parser crash when reporting syntax errors in f-string with newlines. Patch by Pablo Galindo.
- bpo-40176 [https://bugs.python.org/issue?@action=redirect&bpo=40176]: Syntax errors for unterminated string literals now point to the start of the string instead of reporting EOF/EOL.
- bpo-42927 [https://bugs.python.org/issue?@action=redirect&bpo=42927]: The inline cache for `LOAD_ATTR` now also optimizes access to attributes defined by `__slots__`. This makes reading such attribute up to 30% faster.
- bpo-42864 [https://bugs.python.org/issue?@action=redirect&bpo=42864]: Improve error messages in the parser when parentheses are not closed. Patch by Pablo Galindo.
- bpo-42924 [https://bugs.python.org/issue?@action=redirect&bpo=42924]: Fix `bytearray` repetition incorrectly copying data from the start of the

buffer, even if the data is offset within the buffer (e.g. after reassigning a slice at the start of the `bytearray` to a shorter byte string).

- bpo-42882 [https://bugs.python.org/issue?@action=redirect&bpo=42882]: Fix the **`_PyUnicode_FromId()`** function (_Py_IDENTIFIER(var) API) when **`Py_Initialize()`** / **`Py_Finalize()`** is called multiple times: preserve `_PyRuntime.unicode_ids.next_index` value.
- bpo-42827 [https://bugs.python.org/issue?@action=redirect&bpo=42827]: Fix a crash when working out the error line of a **`SyntaxError`** in some multi-line expressions.
- bpo-42823 [https://bugs.python.org/issue?@action=redirect&bpo=42823]: frame.f_lineno is correct even if frame.f_trace is set to True

Biblioteca

- bpo-37324 [https://bugs.python.org/issue?@action=redirect&bpo=37324]: Remove deprecated aliases to Classes Base Abstratas de Coleções from the **`collections`** module.

Núcleo e embutidos

- bpo-41994 [https://bugs.python.org/issue?@action=redirect&bpo=41994]: Fixed possible leak in `import` when `sys.modules` is not a `dict`.
- bpo-27772 [https://bugs.python.org/issue?@action=redirect&bpo=27772]: In string formatting, preceding the *width* field by `'0'` no longer affects the default alignment for strings.

Biblioteca

- bpo-43108 [https://bugs.python.org/issue?@action=redirect&bpo=43108]: Fixed a reference leak in the **`curses`** module. Patch by Pablo Galindo
- bpo-43077 [https://bugs.python.org/issue?@action=redirect&bpo=43077]: Update the bundled pip to 21.0.1 and setuptools to 52.0.0.
- bpo-41282 [https://bugs.python.org/issue?@action=redirect&bpo=41282]: Deprecate `distutils` in documentation and add warning on import.
- bpo-43014 [https://bugs.python.org/issue?@action=redirect&bpo=43014]: Improve performance of **`tokenize`** by 20-30%. Patch by Anthony Sottile.

- bpo-42323 [https://bugs.python.org/issue?@action=redirect&bpo=42323]: Fix **`math.nextafter()`** for NaN on AIX.
- bpo-42955 [https://bugs.python.org/issue?@action=redirect&bpo=42955]: Add **`sys.stdlib_module_names`**, containing the list of the standard library module names. Patch by Victor Stinner.
- bpo-42944 [https://bugs.python.org/issue?@action=redirect&bpo=42944]: Fix `random.Random.sample` when `counts` argument is not `None`.
- bpo-42934 [https://bugs.python.org/issue?@action=redirect&bpo=42934]: Use **`TracebackException`**'s new `compact` param in **`TestResult`** to reduce time and memory consumed by traceback formatting.
- bpo-42931 [https://bugs.python.org/issue?@action=redirect&bpo=42931]: Add **`randbytes()`** to `random.__all__`.
- bpo-38250 [https://bugs.python.org/issue?@action=redirect&bpo=38250]: [Enum] Flags consisting of a single bit are now considered canonical, and will be the only flags returned from listing and iterating over a Flag class or a Flag member. Multi-bit flags are considered aliases; they will be returned from lookups and operations that result in their value. Iteration for both Flag and Flag members is in definition order.
- bpo-42877 [https://bugs.python.org/issue?@action=redirect&bpo=42877]: Added the `compact` parameter to the constructor of **`traceback.TracebackException`** to reduce time and memory for use cases that only need to call **`TracebackException.format()`** and **`TracebackException.format_exception_only()`**.
- bpo-42923 [https://bugs.python.org/issue?@action=redirect&bpo=42923]: The **`Py_FatalError()`** function and the **`faulthandler`** module now dump the list of extension modules on a fatal error.
- bpo-42848 [https://bugs.python.org/issue?@action=redirect&bpo=42848]: Removed recursion from **`TracebackException`** to allow it to handle long exception chains.
- bpo-42901 [https://bugs.python.org/issue?@action=redirect&bpo=42901]: [Enum] move member creation from `EnumMeta.__new__` to `_proto_member.__set_name__`, allowing members to be created and visible in `__init_subclass__`.
- bpo-42780 [https://bugs.python.org/issue?@action=redirect&bpo=42780]: Fix os.set_inheritable() for O_PATH file descriptors on Linux.
- bpo-42866 [https://bugs.python.org/issue?@action=redirect&bpo=42866]: Fix a reference leak in the `getcodec()` function of CJK codecs. Patch by

Victor Stinner.

- bpo-42846 [https://bugs.python.org/issue?@action=redirect&bpo=42846]: Convert the 6 CJK codec extension modules (_codecs_cn, _codecs_hk, _codecs_iso2022, _codecs_jp, _codecs_kr and _codecs_tw) to the multiphase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]). Patch by Victor Stinner.
- bpo-42851 [https://bugs.python.org/issue?@action=redirect&bpo=42851]: remove __init_subclass__ support for Enum members
- bpo-42834 [https://bugs.python.org/issue?@action=redirect&bpo=42834]: Make internal caches of the `_json` module compatible with subinterpreters.
- bpo-41748 [https://bugs.python.org/issue?@action=redirect&bpo=41748]: Fix HTMLParser parsing rules for element attributes containing commas with spaces. Patch by Karl Dubost.
- bpo-40810 [https://bugs.python.org/issue?@action=redirect&bpo=40810]: Require SQLite 3.7.15 or newer. Patch by Erlend E. Aasland.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Convert the _multibytecodec extension module (CJK codecs) to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]). Patch by Erlend E. Aasland.
- bpo-42802 [https://bugs.python.org/issue?@action=redirect&bpo=42802]: The distutils `bdist_wininst` command deprecated in Python 3.8 has been removed. The distutils `bdist_wheel` command is now recommended to distribute binary packages on Windows.
- bpo-24464 [https://bugs.python.org/issue?@action=redirect&bpo=24464]: The undocumented built-in function `sqlite3.enable_shared_cache` is now deprecated, scheduled for removal in Python 3.12. Its use is strongly discouraged by the SQLite3 documentation. Patch by Erlend E. Aasland.
- bpo-42384 [https://bugs.python.org/issue?@action=redirect&bpo=42384]: Make pdb populate sys.path[0] exactly the same as regular python execution.
- bpo-42383 [https://bugs.python.org/issue?@action=redirect&bpo=42383]: Fix pdb: previously pdb would fail to restart the debugging target if it was specified using a relative path and the current directory changed.
- bpo-42005 [https://bugs.python.org/issue?@action=redirect&bpo=42005]: Fix CLI of **`cProfile`** and **`profile`** to catch **`BrokenPipeError`**.
- bpo-41604 [https://bugs.python.org/issue?@action=redirect&bpo=41604]: Don’t decrement the reference count of the previous user_ptr when

set_panel_userptr fails.

- bpo-41149 [https://bugs.python.org/issue?@action=redirect&bpo=41149]: Allow executing callables that have a boolean value of `False` when passed to **`Threading.thread`** as the target. Patch contributed by Barney Stratford.
- bpo-38307 [https://bugs.python.org/issue?@action=redirect&bpo=38307]: Add an 'end_lineno' attribute to the Class and Function objects that appear in the tree returned by pyclbr functions. This and the existing 'lineno' attribute define the extent of class and def statements. Patch by Aviral Srivastava.
- bpo-39273 [https://bugs.python.org/issue?@action=redirect&bpo=39273]: The `BUTTON5_*` constants are now exposed in the **`curses`** module if available.
- bpo-33289 [https://bugs.python.org/issue?@action=redirect&bpo=33289]: Correct call to **`tkinter.colorchooser`** to return RGB triplet of ints instead of floats. Patch by Cheryl Sabella.

Documentação

- bpo-40304 [https://bugs.python.org/issue?@action=redirect&bpo=40304]: Fix doc for type(name, bases, dict). Patch by Boris Verkhovskiy and Éric Araujo.
- bpo-42811 [https://bugs.python.org/issue?@action=redirect&bpo=42811]: Updated importlib.util.resolve_name() doc to use __spec__.parent instead of __package__. (Thanks Yair Frid.)

Testes

- bpo-40823 [https://bugs.python.org/issue?@action=redirect&bpo=40823]: Use **`unittest.TestLoader().loadTestsFromTestCase()`** instead of **`unittest.makeSuite()`** in **`sqlite3`** tests. Patch by Erlend E. Aasland.
- bpo-40810 [https://bugs.python.org/issue?@action=redirect&bpo=40810]: In **`sqlite3`**, fix `CheckTraceCallbackContent` for SQLite pre 3.7.15.

Construção

- bpo-43031 [https://bugs.python.org/issue?@action=redirect&bpo=43031]: Pass `--timeout=$(TESTTIMEOUT)` option to the default profile task `./python`

`-m test --pgo` command.

- bpo-36143 [https://bugs.python.org/issue?@action=redirect&bpo=36143]: `make regen-all` now also runs `regen-keyword`. Patch by Victor Stinner.

- bpo-42874 [https://bugs.python.org/issue?@action=redirect&bpo=42874]: Removed the grep -q and -E flags in the tzpath validation section of the configure script to better accommodate users of some platforms (specifically Solaris 10).

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Add library search path by wr-cc in add_cross_compiling_paths() for VxWorks.

- bpo-42856 [https://bugs.python.org/issue?@action=redirect&bpo=42856]: Add `--with-wheel-pkg-dir=PATH` option to the `./configure` script. If specified, the **`ensurepip`** module looks for `setuptools` and `pip` wheel packages in this directory: if both are present, these wheel packages are used instead of ensurepip bundled wheel packages.

  Algumas distribuições Linux possuem políticas de empacotamento recomendando evitar o empacotamento de dependências. Por exemplo, Fedora instala pacotes wheel no diretório `/usr/share/python-wheels/` e não instala o pacote `ensurepip._bundled`.

Windows

- bpo-41837 [https://bugs.python.org/issue?@action=redirect&bpo=41837]: Updated Windows installer to include OpenSSL 1.1.1i
- bpo-42584 [https://bugs.python.org/issue?@action=redirect&bpo=42584]: Upgrade Windows installer to use SQLite 3.34.0.

macOS

- bpo-42504 [https://bugs.python.org/issue?@action=redirect&bpo=42504]: Ensure that the value of sysconfig.get_config_var('MACOSX_DEPLOYMENT_TARGET') is always a string, even in when the value is parsable as an integer.

IDLE

- bpo-43008 [https://bugs.python.org/issue?@action=redirect&bpo=43008]: Make IDLE invoke `sys.excepthook()` in normal, 2-process mode. Patch by Ken Hilton.
- bpo-33065 [https://bugs.python.org/issue?@action=redirect&bpo=33065]: Fix problem debugging user classes with __repr__ method.
- bpo-23544 [https://bugs.python.org/issue?@action=redirect&bpo=23544]: Disable Debug=>Stack Viewer when user code is running or Debugger is active, to prevent hang or crash. Patch by Zackery Spytz.
- bpo-32631 [https://bugs.python.org/issue?@action=redirect&bpo=32631]: Finish zzdummy example extension module: make menu entries work; add docstrings and tests with 100% coverage.

API C

- bpo-42979 [https://bugs.python.org/issue?@action=redirect&bpo=42979]: When Python is built in debug mode (with C assertions), calling a type slot like `sq_length` (`__len__()` in Python) now fails with a fatal error if the slot succeeded with an exception set, or failed with no exception set. The error message contains the slot, the type name, and the current exception (if an exception is set). Patch by Victor Stinner.
- bpo-43030 [https://bugs.python.org/issue?@action=redirect&bpo=43030]: Fixed a compiler warning in `Py_UNICODE_ISSPACE()` on platforms with signed `wchar_t`.

Python 3.10.0 alpha 4

Data de lançamento: 2021-01-04

Núcleo e embutidos

- bpo-42814 [https://bugs.python.org/issue?@action=redirect&bpo=42814]: Fix undefined behavior in `Objects/genericaliasobject.c`.
- bpo-42806 [https://bugs.python.org/issue?@action=redirect&bpo=42806]: Fix the column offsets for f-strings `ast` nodes surrounded by parentheses and

for nodes that spawn multiple lines. Patch by Pablo Galindo.

- bpo-40631 [https://bugs.python.org/issue?@action=redirect&bpo=40631]: Fix regression where a single parenthesized starred expression was a valid assignment target.
- bpo-27794 [https://bugs.python.org/issue?@action=redirect&bpo=27794]: Improve the error message for failed writes/deletes to property objects. When possible, the attribute name is now shown. Patch provided by Yurii Karabas.
- bpo-42745 [https://bugs.python.org/issue?@action=redirect&bpo=42745]: Make the type attribute lookup cache per-interpreter. Patch by Victor Stinner.
- bpo-42246 [https://bugs.python.org/issue?@action=redirect&bpo=42246]: Jumps to jumps are not eliminated when it would break PEP 626.
- bpo-42246 [https://bugs.python.org/issue?@action=redirect&bpo=42246]: Make sure that the `f_lasti` and `f_lineno` attributes of a frame are set correctly when an exception is raised or re-raised. Required for PEP 626.
- bpo-32381 [https://bugs.python.org/issue?@action=redirect&bpo=32381]: The coding cookie (ex: `# coding: latin1`) is now ignored in the command passed to the **`-c`** command line option. Patch by Victor Stinner.
- bpo-30858 [https://bugs.python.org/issue?@action=redirect&bpo=30858]: Improve error location in expressions that contain assignments. Patch by Pablo Galindo and Lysandros Nikolaou.
- bpo-42615 [https://bugs.python.org/issue?@action=redirect&bpo=42615]: Remove jump commands made redundant by the deletion of unreachable bytecode blocks

Biblioteca

- bpo-42639 [https://bugs.python.org/issue?@action=redirect&bpo=42639]: Make the **`atexit`** module state per-interpreter. It is now safe have more than one **`atexit`** module instance. Patch by Donghee Na and Victor Stinner.

Núcleo e embutidos

- bpo-32381 [https://bugs.python.org/issue?@action=redirect&bpo=32381]: Fix encoding name when running a `.pyc` file on Windows: **`PyRun_SimpleFileExFlags()`** now uses the correct encoding to decode the filename.

Biblioteca

- bpo-42195 [https://bugs.python.org/issue?@action=redirect&bpo=42195]: The `__args__` of the parameterized generics for `typing.Callable` and `collections.abc.Callable` are now consistent. The `__args__` for `collections.abc.Callable` are now flattened while `typing.Callable`'s have not changed. To allow this change, `types.GenericAlias` can now be subclassed and `collections.abc.Callable`'s `__class_getitem__` will now return a subclass of `types.GenericAlias`. Tests for typing were also updated to not subclass things like `Callable[..., T]` as that is not a valid base class. Finally, both `Callables` no longer validate their `argtypes`, in `Callable[[argtypes], resulttype]` to prepare for **PEP 612** [https://peps.python.org/pep-0612/]. Patch by Ken Jin.
- bpo-40137 [https://bugs.python.org/issue?@action=redirect&bpo=40137]: Convert functools module to use `PyType_FromModuleAndSpec()`.
- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert `array` to use heap types, and establish module state for these.
- bpo-42008 [https://bugs.python.org/issue?@action=redirect&bpo=42008]: Fix _random.Random() seeding.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `pyexpat` extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

Núcleo e embutidos

- bpo-40521 [https://bugs.python.org/issue?@action=redirect&bpo=40521]: Make the Unicode dictionary of interned strings compatible with subinterpreters. Patch by Victor Stinner.
- bpo-39465 [https://bugs.python.org/issue?@action=redirect&bpo=39465]: Make `_PyUnicode_FromId()` function compatible with subinterpreters. Each interpreter now has an array of identifier objects (interned strings decoded from UTF-8). Patch by Victor Stinner.

Biblioteca

- bpo-42257 [https://bugs.python.org/issue?@action=redirect&bpo=42257]: Handle empty string in variable executable in platform.libc_ver()

- bpo-42772 [https://bugs.python.org/issue?@action=redirect&bpo=42772]: randrange() now raises a TypeError when step is specified without a stop argument. Formerly, it silently ignored the step argument.

- bpo-42759 [https://bugs.python.org/issue?@action=redirect&bpo=42759]: Fixed equality comparison of `tkinter.Variable` and `tkinter.font.Font`. Objects which belong to different Tcl interpreters are now always different, even if they have the same name.

- bpo-42756 [https://bugs.python.org/issue?@action=redirect&bpo=42756]: Configure LMTP Unix-domain socket to use socket global default timeout when a timeout is not explicitly provided.

- bpo-23328 [https://bugs.python.org/issue?@action=redirect&bpo=23328]: Allow / character in username, password fields on _PROXY envars.

- bpo-42740 [https://bugs.python.org/issue?@action=redirect&bpo=42740]: `typing.get_args()` and `typing.get_origin()` now support **PEP 604** [https://peps.python.org/pep-0604/] union types and **PEP 612** [https://peps.python.org/pep-0612/] additions to `Callable`.

- bpo-42655 [https://bugs.python.org/issue?@action=redirect&bpo=42655]: `subprocess` *extra_groups* is now correctly passed into setgroups() system call.

- bpo-42727 [https://bugs.python.org/issue?@action=redirect&bpo=42727]: `EnumMeta.__prepare__` now accepts `**kwds` to properly support `__init_subclass__`

- bpo-38308 [https://bugs.python.org/issue?@action=redirect&bpo=38308]: Add optional *weights* to *statistics.harmonic_mean()*.

- bpo-42721 [https://bugs.python.org/issue?@action=redirect&bpo=42721]: When simple query dialogs (`tkinter.simpledialog`), message boxes (`tkinter.messagebox`) or color choose dialog

(`tkinter.colorchooser`) are created without arguments *master* and *parent*, and the default root window is not yet created, and `NoDefaultRoot()` was not called, a new temporal hidden root window will be created automatically. It will not be set as the default root window and will be destroyed right after closing the dialog window. It will help to use these simple dialog windows in programs which do not need other GUI.

- bpo-25246 [https://bugs.python.org/issue?@action=redirect&bpo=25246]: Optimized `collections.deque.remove()`.

- bpo-35728 [https://bugs.python.org/issue?@action=redirect&bpo=35728]: Added a root parameter to `tkinter.font.nametofont()`.

- bpo-15303 [https://bugs.python.org/issue?@action=redirect&bpo=15303]: `tkinter` supports now widgets with boolean value False.

- bpo-42681 [https://bugs.python.org/issue?@action=redirect&bpo=42681]: Fixed range checks for color and pair numbers in `curses`.

- bpo-42685 [https://bugs.python.org/issue?@action=redirect&bpo=42685]: Improved placing of simple query windows in Tkinter (such as `tkinter.simpledialog.askinteger()`). They are now centered at the center of the parent window if it is specified and shown, otherwise at the center of the screen.

- bpo-9694 [https://bugs.python.org/issue?@action=redirect&bpo=9694]: Argparse help no longer uses the confusing phrase, “optional arguments”. It uses “options” instead.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `_thread` extension module to the multiphase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]) and convert its static types to heap types.

- bpo-37961 [https://bugs.python.org/issue?@action=redirect&bpo=37961]: Fix crash in `tracemalloc.Traceback.__repr__()` (regressed in Python 3.9).

- bpo-42630 [https://bugs.python.org/issue?@action=redirect&bpo=42630]: **`tkinter`** functions and constructors which need a default root window raise now **`RuntimeError`** with descriptive message instead of obscure **`AttributeError`** or **`NameError`** if it is not created yet or cannot be created automatically.

- bpo-42639 [https://bugs.python.org/issue?@action=redirect&bpo=42639]: **`atexit._run_exitfuncs()`** now logs callback exceptions using **`sys.unraisablehook`**, rather than logging them directly into **`sys.stderr`** and raise the last exception.

- bpo-42644 [https://bugs.python.org/issue?@action=redirect&bpo=42644]: `logging.disable` will now validate the types and value of its parameter. It also now accepts strings representing the levels (as does `logging.setLevel`) instead of only the numerical values.

- bpo-42639 [https://bugs.python.org/issue?@action=redirect&bpo=42639]: At Python exit, if a callback registered with **`atexit.register()`** fails, its exception is now logged. Previously, only some exceptions were logged, and the last exception was always silently ignored.

- bpo-36541 [https://bugs.python.org/issue?@action=redirect&bpo=36541]: Fixed lib2to3.pgen2 to be able to parse PEP-570 positional only argument syntax.

- bpo-42382 [https://bugs.python.org/issue?@action=redirect&bpo=42382]: In `importlib.metadata`: - `EntryPoint` objects now expose a `.dist` object referencing the `Distribution` when constructed from a `Distribution`. - Add support for package discovery under package normalization rules. - The object returned by `metadata()` now has a formally defined protocol called `PackageMetadata` with declared support for the `.get_all()` method. - Synced with importlib_metadata 3.3.

- bpo-41877 [https://bugs.python.org/issue?@action=redirect&bpo=41877]: A check is added against misspellings of autospect, auto_spec and set_spec being passed as arguments to patch, patch.object and create_autospec.

- bpo-39717 [https://bugs.python.org/issue?@action=redirect&bpo=39717]: [tarfile] update nested exception raising to use `from None` or `from e`

- bpo-41877 [https://bugs.python.org/issue?@action=redirect&bpo=41877]: AttributeError for suspected misspellings of assertions on mocks are now pointing out that the cause are misspelled assertions and also what to do if the misspelling is actually an intended attribute name. The unittest.mock document is also updated to reflect the current set of recognised misspellings.

- bpo-41559 [https://bugs.python.org/issue?@action=redirect&bpo=41559]: Implemented **PEP 612** [https://peps.python.org/pep-0612/]: added `ParamSpec` and `Concatenate` to **`typing`**. Patch by Ken Jin.

- bpo-42385 [https://bugs.python.org/issue?@action=redirect&bpo=42385]: StrEnum: fix _generate_next_value_ to return a str

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Define THREAD_STACK_SIZE for VxWorks.

- bpo-34750 [https://bugs.python.org/issue?@action=redirect&bpo=34750]: [Enum] `_EnumDict.update()` is now supported.

- bpo-42517 [https://bugs.python.org/issue?@action=redirect&bpo=42517]: Enum: private names do not become members / do not generate errors – they remain normal attributes

- bpo-42678 [https://bugs.python.org/issue?@action=redirect&bpo=42678]: `Enum`: call `__init_subclass__` after members have been added

- bpo-28964 [https://bugs.python.org/issue?@action=redirect&bpo=28964]: **`ast.literal_eval()`** adds line number information (if available) in error message for malformed nodes.

- bpo-42470 [https://bugs.python.org/issue?@action=redirect&bpo=42470]: **`random.sample()`** no longer warns on a sequence which is also a set.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: **`posixpath.expanduser()`** returns the input *path* unchanged if user home directory is None on VxWorks.

- bpo-42388 [https://bugs.python.org/issue?@action=redirect&bpo=42388]: Fix subprocess.check_output(…, input=None) behavior when text=True to be consistent with that of the documentation and universal_newlines=True.

- bpo-34463 [https://bugs.python.org/issue?@action=redirect&bpo=34463]: Fixed discrepancy between **`traceback`** and the interpreter in formatting of SyntaxError with lineno not set (**`traceback`** was changed to match interpreter).

- bpo-42393 [https://bugs.python.org/issue?@action=redirect&bpo=42393]: Raise **`OverflowError`** instead of silent truncation in **`socket.ntohs()`** and **`socket.htons()`**. Silent truncation was deprecated in Python 3.7. Patch by Erlend E. Aasland

- bpo-42222 [https://bugs.python.org/issue?@action=redirect&bpo=42222]: Harmonized **`random.randrange()`** argument handling to match **`range()`**.

  - The integer test and conversion in `randrange()` now uses **`operator.index()`**.
  - Non-integer arguments to `randrange()` are deprecated.
  - The `ValueError` is deprecated in favor of a `TypeError`.
  - It now runs a little faster than before.

  (Contributed by Raymond Hettinger and Serhiy Storchaka.)

- bpo-42163 [https://bugs.python.org/issue?@action=redirect&bpo=42163]: Restore compatibility for **`os.uname_result`** around deepcopy and _replace.

- bpo-42090 [https://bugs.python.org/issue?@action=redirect&bpo=42090]: `zipfile.Path.joinpath` now accepts arbitrary arguments, same as `pathlib.Path.joinpath`.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the _csv module to the multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-42059 [https://bugs.python.org/issue?@action=redirect&bpo=42059]: **`typing.TypedDict`** types created using the alternative call-style syntax now correctly respect the `total` keyword argument when setting their `__required_keys__` and `__optional_keys__` class attributes.

- bpo-41960 [https://bugs.python.org/issue?@action=redirect&bpo=41960]: Add `globalns` and `localns` parameters to the **`inspect.signature()`** and **`inspect.Signature.from_callable()`**.

- bpo-41907 [https://bugs.python.org/issue?@action=redirect&bpo=41907]: fix `format()` behavior for `IntFlag`

- bpo-41891 [https://bugs.python.org/issue?@action=redirect&bpo=41891]: Ensure asyncio.wait_for waits for task completion

- bpo-24792 [https://bugs.python.org/issue?@action=redirect&bpo=24792]: Fixed bug where **`zipimporter`** sometimes reports an incorrect cause of import errors.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix site and sysconfig modules for VxWorks RTOS which has no home directories.

- bpo-41462 [https://bugs.python.org/issue?@action=redirect&bpo=41462]: Add **`os.set_blocking()`** support for VxWorks RTOS.

- bpo-40219 [https://bugs.python.org/issue?@action=redirect&bpo=40219]: Lowered **`tkinter.ttk.LabeledScale`** dummy widget to prevent hiding part of the content label.

- bpo-37193 [https://bugs.python.org/issue?@action=redirect&bpo=37193]: Fixed memory leak in `socketserver.ThreadingMixIn` introduced in Python 3.7.

- bpo-39068 [https://bugs.python.org/issue?@action=redirect&bpo=39068]: Fix initialization race condition in **`a85encode()`** and **`b85encode()`** in **`base64`**. Patch by Brandon Stansbury.

Documentação

- bpo-17140 [https://bugs.python.org/issue?@action=redirect&bpo=17140]: Add documentation for the **`multiprocessing.pool.ThreadPool`** class.
- bpo-34398 [https://bugs.python.org/issue?@action=redirect&bpo=34398]: Prominently feature listings from the glossary in documentation search results. Patch by Ammar Askar.

Testes

- bpo-42794 [https://bugs.python.org/issue?@action=redirect&bpo=42794]: Update test_nntplib to use official group name of news.aioe.org for testing. Patch by Donghee Na.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Skip some asyncio tests on VxWorks.
- bpo-42641 [https://bugs.python.org/issue?@action=redirect&bpo=42641]: Enhance `test_select.test_select()`: it now takes 500 milliseconds rather than 10 seconds. Use Python rather than a shell to make the test more portable.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Skip some tests in _test_all_chown_common() on VxWorks.
- bpo-42199 [https://bugs.python.org/issue?@action=redirect&bpo=42199]: Fix bytecode helper assertNotInBytecode.
- bpo-41443 [https://bugs.python.org/issue?@action=redirect&bpo=41443]: Add more attribute checking in test_posix.py
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Disable os.popen and impacted tests on VxWorks
- bpo-41439 [https://bugs.python.org/issue?@action=redirect&bpo=41439]: Port test_ssl and test_uuid to VxWorks RTOS.

Construção

- bpo-42692 [https://bugs.python.org/issue?@action=redirect&bpo=42692]: Fix __builtin_available check on older compilers. Patch by Joshua Root.
- bpo-27640 [https://bugs.python.org/issue?@action=redirect&bpo=27640]: Added `--disable-test-modules` option to the `configure` script: don't build nor install test modules. Patch by Xavier de Gaye, Thomas Petazzoni and Peixing Xin.
- bpo-42604 [https://bugs.python.org/issue?@action=redirect&bpo=42604]: Now all platforms use a value for the "EXT_SUFFIX" build variable derived from SOABI (for instance in freeBSD, "EXT_SUFFIX" is now ".cpython-310d.so" instead of ".so"). Previously only Linux, Mac and VxWorks were using a value for "EXT_SUFFIX" that included "SOABI".
- bpo-42598 [https://bugs.python.org/issue?@action=redirect&bpo=42598]: Fix implicit function declarations in configure which could have resulted in incorrect configuration checks. Patch contributed by Joshua Root.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Enable libpython3.so for VxWorks.
- bpo-29076 [https://bugs.python.org/issue?@action=redirect&bpo=29076]: Add fish shell support to macOS installer.

macOS

- bpo-42361 [https://bugs.python.org/issue?@action=redirect&bpo=42361]: Update macOS installer build to use Tcl/Tk 8.6.11 (rc2, expected to be final release).
- bpo-41837 [https://bugs.python.org/issue?@action=redirect&bpo=41837]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.1i.
- bpo-42584 [https://bugs.python.org/issue?@action=redirect&bpo=42584]: Update macOS installer to use SQLite 3.34.0.

Ferramentas/Demos

- bpo-42726 [https://bugs.python.org/issue?@action=redirect&bpo=42726]: Fixed Python 3 compatibility issue with gdb/libpython.py handling of attribute dictionaries.
- bpo-42613 [https://bugs.python.org/issue?@action=redirect&bpo=42613]: Fix `freeze.py` tool to use the prope config and library directories. Patch

by Victor Stinner.

API C

- bpo-42591 [https://bugs.python.org/issue?@action=redirect&bpo=42591]: Export the **`Py_FrozenMain()`** function: fix a Python 3.9.0 regression. Python 3.9 uses `-fvisibility=hidden` and the function was not exported explicitly and so not exported.
- bpo-32381 [https://bugs.python.org/issue?@action=redirect&bpo=32381]: Remove the private **`_Py_fopen()`** function which is no longer needed. Use **`_Py_wfopen()`** or **`_Py_fopen_obj()`** instead. Patch by Victor Stinner.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`resource`** extension module to module state
- bpo-42111 [https://bugs.python.org/issue?@action=redirect&bpo=42111]: Update the `xxlimited` module to be a better example of how to use the limited C API.
- bpo-40052 [https://bugs.python.org/issue?@action=redirect&bpo=40052]: Fix an alignment build warning/error in function `PyVectorcall_Function()`. Patch by Andreas Schneider, Antoine Pitrou and Petr Viktorin.

Python 3.10.0 alpha 3

Data de lançamento: 2020-12-07

Segurança

- bpo-40791 [https://bugs.python.org/issue?@action=redirect&bpo=40791]: Add `volatile` to the accumulator variable in `hmac.compare_digest`, making constant-time-defeating optimizations less likely.

Núcleo e embutidos

- bpo-42576 [https://bugs.python.org/issue?@action=redirect&bpo=42576]: `types.GenericAlias` will now raise a `TypeError` when attempting to initialize with a keyword argument. Previously, this would cause the interpreter to crash if the interpreter was compiled with debug

symbols. This does not affect interpreters compiled for release. Patch by Ken Jin.

- bpo-42536 [https://bugs.python.org/issue?@action=redirect&bpo=42536]: Several built-in and standard library types now ensure that their internal result tuples are always tracked by the garbage collector:

  - `collections.OrderedDict.items()`
  - `dict.items()`
  - `enumerate()`
  - `functools.reduce()`
  - `itertools.combinations()`
  - `itertools.combinations_with_replacement()`
  - `itertools.permutations()`
  - `itertools.product()`
  - `itertools.zip_longest()`
  - `zip()`

  Previously, they could have become untracked by a prior garbage collection. Patch by Brandt Bucher.

- bpo-42500 [https://bugs.python.org/issue?@action=redirect&bpo=42500]: Improve handling of exceptions near recursion limit. Converts a number of Fatal Errors in RecursionErrors.

- bpo-42246 [https://bugs.python.org/issue?@action=redirect&bpo=42246]: PEP 626: After a return, the f_lineno attribute of a frame is always the last line executed.

- bpo-42435 [https://bugs.python.org/issue?@action=redirect&bpo=42435]: Speed up comparison of bytes objects with non-bytes objects when option `-b` is specified. Speed up comparison of bytarray objects with non-buffer object.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `_warnings` extension module to the multi-phase initialization API

(**PEP 489** [https://peps.python.org/pep-0489/]). Patch by Victor Stinner.

Núcleo e embutidos

- bpo-41686 [https://bugs.python.org/issue?@action=redirect&bpo=41686]: On Windows, the `SIGINT` event, `_PyOS_SigintEvent()`, is now created even if Python is configured to not install signal handlers (if **`PyConfig.install_signal_handlers`** equals to 0, or `Py_InitializeEx(0)`).
- bpo-42381 [https://bugs.python.org/issue?@action=redirect&bpo=42381]: Allow assignment expressions in set literals and set comprehensions as per PEP 572. Patch by Pablo Galindo.
- bpo-42202 [https://bugs.python.org/issue?@action=redirect&bpo=42202]: Change function parameters annotations internal representation to tuple of strings. Patch provided by Yurii Karabas.
- bpo-42374 [https://bugs.python.org/issue?@action=redirect&bpo=42374]: Fix a regression introduced by the new parser, where an unparenthesized walrus operator was not allowed within generator expressions.
- bpo-42316 [https://bugs.python.org/issue?@action=redirect&bpo=42316]: Allow an unparenthesized walrus in subscript indexes.
- bpo-42349 [https://bugs.python.org/issue?@action=redirect&bpo=42349]: Make sure that the compiler front-end produces a well-formed control flow graph. Be more aggressive in the compiler back-end, as it is now safe to do so.
- bpo-42296 [https://bugs.python.org/issue?@action=redirect&bpo=42296]: On Windows, fix a regression in signal handling which prevented to interrupt a program using CTRL+C. The signal handler can be run in a thread different than the Python thread, in which case the test deciding if the thread can handle signals is wrong.
- bpo-42332 [https://bugs.python.org/issue?@action=redirect&bpo=42332]: **`types.GenericAlias`** objects can now be the targets of weakrefs.
- bpo-42282 [https://bugs.python.org/issue?@action=redirect&bpo=42282]: Optimise constant subexpressions that appear as part of named expressions (previously the AST optimiser did not descend into named expressions). Patch by Nick Coghlan.
- bpo-42266 [https://bugs.python.org/issue?@action=redirect&bpo=42266]: Fixed a bug with the LOAD_ATTR opcode cache that was not respecting monkey-

patching a class-level attribute to make it a descriptor. Patch by Pablo Galindo.

Biblioteca

- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert **`queue`** to use heap types.

Núcleo e embutidos

- bpo-42246 [https://bugs.python.org/issue?@action=redirect&bpo=42246]: Improved accuracy of line tracing events and f_lineno attribute of Frame objects. See PEP 626 for details.

Biblioteca

- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert **`mmap`** to use heap types.

Núcleo e embutidos

- bpo-42233 [https://bugs.python.org/issue?@action=redirect&bpo=42233]: Allow `GenericAlias` objects to use union type expressions. This allows expressions like `list[int] | dict[float, str]` where previously a `TypeError` would have been thrown. This also fixes union type expressions not de-duplicating `GenericAlias` objects. (Contributed by Ken Jin in bpo-42233 [https://bugs.python.org/issue?@action=redirect&bpo=42233].)
- bpo-26131 [https://bugs.python.org/issue?@action=redirect&bpo=26131]: The import system triggers a `ImportWarning` when it falls back to using `load_module()`.

Biblioteca

- bpo-5054 [https://bugs.python.org/issue?@action=redirect&bpo=5054]: CGIHTTPRequestHandler.run_cgi() HTTP_ACCEPT improperly

parsed. Replace the special purpose getallmatchingheaders with generic get_all method and add relevant tests.

Original Patch by Martin Panter. Modified by Senthil Kumaran.

- bpo-42562 [https://bugs.python.org/issue?@action=redirect&bpo=42562]: Fix issue when dis failed to parse function that has no line numbers. Patch provided by Yurii Karabas.

- bpo-17735 [https://bugs.python.org/issue?@action=redirect&bpo=17735]: **`inspect.findsource()`** now raises **`OSError`** instead of **`IndexError`** when **`co_lineno`** of a code object is greater than the file length. This can happen, for example, when a file is edited after it was imported. PR by Irit Katriel.

- bpo-42116 [https://bugs.python.org/issue?@action=redirect&bpo=42116]: Fix handling of trailing comments by **`inspect.getsource()`**.

- bpo-42532 [https://bugs.python.org/issue?@action=redirect&bpo=42532]: Remove unexpected call of `__bool__` when passing a `spec_arg` argument to a Mock.

- bpo-38200 [https://bugs.python.org/issue?@action=redirect&bpo=38200]: Added itertools.pairwise()

- bpo-41818 [https://bugs.python.org/issue?@action=redirect&bpo=41818]: Fix test_master_read() so that it succeeds on all platforms that either raise OSError or return b"" upon reading from master.

- bpo-42487 [https://bugs.python.org/issue?@action=redirect&bpo=42487]: ChainMap.__iter__ no longer calls __getitem__ on underlying maps

- bpo-42482 [https://bugs.python.org/issue?@action=redirect&bpo=42482]: **`TracebackException`** no longer holds a reference to the exception's traceback object. Consequently, instances of TracebackException for equivalent but non-equal exceptions now compare as equal.

- bpo-41818 [https://bugs.python.org/issue?@action=redirect&bpo=41818]: Make test_openpty() avoid unexpected success due to number of rows and/or number of columns being == 0.

- bpo-42392 [https://bugs.python.org/issue?@action=redirect&bpo=42392]: Remove loop parameter from `asyncio.subprocess` and `asyncio.tasks` functions. Patch provided by Yurii Karabas.

- bpo-42392 [https://bugs.python.org/issue?@action=redirect&bpo=42392]: Remove loop parameter from `asyncio.open_connection` and `asyncio.start_server` functions. Patch provided by Yurii Karabas.

- bpo-28468 [https://bugs.python.org/issue?@action=redirect&bpo=28468]: Add **`platform.freedesktop_os_release()`** function to parse freedesktop.org `os-release` files.

- bpo-42299 [https://bugs.python.org/issue?@action=redirect&bpo=42299]: Removed the `formatter` module, which was deprecated in Python 3.4. It is somewhat obsolete, little used, and not tested. It was originally scheduled to be removed in Python 3.6, but such removals were delayed until after Python 2.7 EOL. Existing users should copy whatever classes they use into their code. Patch by Donghee Na and Terry J. Reedy.

- bpo-26131 [https://bugs.python.org/issue?@action=redirect&bpo=26131]: Deprecate zipimport.zipimporter.load_module() in favour of exec_module().

- bpo-41818 [https://bugs.python.org/issue?@action=redirect&bpo=41818]: Updated tests for the pty library. test_basic() has been changed to test_openpty(); this additionally checks if slave termios and slave winsize are being set properly by pty.openpty(). In order to add support for FreeBSD, NetBSD, OpenBSD, and Darwin, this also adds test_master_read(), which demonstrates that pty.spawn() should not depend on an OSError to exit from its copy loop.

- bpo-42392 [https://bugs.python.org/issue?@action=redirect&bpo=42392]: Remove loop parameter from `__init__` in all `asyncio.locks` and `asyncio.Queue` classes. Patch provided by Yurii Karabas.

- bpo-15450 [https://bugs.python.org/issue?@action=redirect&bpo=15450]: Make `filecmp.dircmp` respect subclassing. Now the `filecmp.dircmp.subdirs` behaves as expected when subclassing dircmp.

- bpo-42413 [https://bugs.python.org/issue?@action=redirect&bpo=42413]: The exception `socket.timeout` is now an alias of `TimeoutError`.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Support signal module on VxWorks.

- bpo-42406 [https://bugs.python.org/issue?@action=redirect&bpo=42406]: We fixed an issue in `pickle.whichmodule` in which importing `multiprocessing` could change the how pickle identifies which module an object belongs to, potentially breaking the unpickling of those objects.

- bpo-42403 [https://bugs.python.org/issue?@action=redirect&bpo=42403]: Simplify the `importlib` external bootstrap code: `importlib._bootstrap_external` now uses regular imports to import builtin modules. When it is imported, the builtin `__import__()` function is already fully working and so can be used to import builtin modules like `sys`. Patch by Victor Stinner.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Convert _sre module types to heap types (PEP 384). Patch by Erlend E. Aasland.

- bpo-42375 [https://bugs.python.org/issue?@action=redirect&bpo=42375]: subprocess module update for DragonFlyBSD support.

- bpo-41713 [https://bugs.python.org/issue?@action=redirect&bpo=41713]: Port the `_signal` extension module to the multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]). Patch by Victor Stinner and Mohamed Koubaa.

- bpo-37205 [https://bugs.python.org/issue?@action=redirect&bpo=37205]: `time.time()`, `time.perf_counter()` and `time.monotonic()` functions

can no longer fail with a Python fatal error, instead raise a regular Python exception on failure.

- bpo-42328 [https://bugs.python.org/issue?@action=redirect&bpo=42328]: Fixed **`tkinter.ttk.Style.map()`**. The function accepts now the representation of the default state as empty sequence (as returned by `Style.map()`). The structure of the result is now the same on all platform and does not depend on the value of `wantobjects`.

- bpo-42345 [https://bugs.python.org/issue?@action=redirect&bpo=42345]: Fix various issues with `typing.Literal` parameter handling (flatten, deduplicate, use type to cache key). Patch provided by Yurii Karabas.

- bpo-37205 [https://bugs.python.org/issue?@action=redirect&bpo=37205]: **`time.perf_counter()`** on Windows and **`time.monotonic()`** on macOS are now system-wide. Previously, they used an offset computed at startup to reduce the precision loss caused by the float type. Use **`time.perf_counter_ns()`** and **`time.monotonic_ns()`** added in Python 3.7 to avoid this precision loss.

- bpo-42318 [https://bugs.python.org/issue?@action=redirect&bpo=42318]: Fixed support of non-BMP characters in **`tkinter`** on macOS.

- bpo-42350 [https://bugs.python.org/issue?@action=redirect&bpo=42350]: Fix the **`threading.Thread`** class at fork: do nothing if the thread is already stopped (ex: fork called at Python exit). Previously, an error was logged in the child process.

- bpo-42333 [https://bugs.python.org/issue?@action=redirect&bpo=42333]: Port _ssl extension module to heap types.

- bpo-42014 [https://bugs.python.org/issue?@action=redirect&bpo=42014]: The `onerror` callback from `shutil.rmtree` now receives correct function when `os.open` fails.

- bpo-42237 [https://bugs.python.org/issue?@action=redirect&bpo=42237]: Fix `os.sendfile()` on illumos.

- bpo-42308 [https://bugs.python.org/issue?@action=redirect&bpo=42308]: Add `threading.__excepthook__` to allow retrieving the original value of `threading.excepthook()` in case it is set to a broken or a different value. Patch by Mario Corchero.

- bpo-42131 [https://bugs.python.org/issue?@action=redirect&bpo=42131]: Implement PEP 451/spec methods on zipimport.zipimporter: find_spec(), create_module(), and exec_module().

  This also allows for the documented deprecation of find_loader(), find_module(), and load_module().

- bpo-41877 [https://bugs.python.org/issue?@action=redirect&bpo=41877]: Mock objects which are not unsafe will now raise an AttributeError if an attribute with the prefix asert, aseert, or assrt is accessed, in addition to this already happening for the prefixes assert or assret.

- bpo-42264 [https://bugs.python.org/issue?@action=redirect&bpo=42264]: `sqlite3.OptimizedUnicode` has been undocumented and obsolete since Python 3.3, when it was made an alias to `str`. It is now deprecated, scheduled for removal in Python 3.12.

- bpo-42251 [https://bugs.python.org/issue?@action=redirect&bpo=42251]: Added `threading.gettrace()` and `threading.getprofile()` to retrieve the functions set by `threading.settrace()` and `threading.setprofile()` respectively. Patch by Mario Corchero.

- bpo-42249 [https://bugs.python.org/issue?@action=redirect&bpo=42249]: Fixed writing binary Plist files larger than 4 GiB.

- bpo-42236 [https://bugs.python.org/issue?@action=redirect&bpo=42236]: On Unix, the `os.device_encoding()` function now returns `'UTF-8'` rather than the device encoding if the Python UTF-8 Mode is enabled.

- bpo-41754 [https://bugs.python.org/issue?@action=redirect&bpo=41754]: webbrowser: Ignore *NotADirectoryError* when calling `xdg-settings`.

- bpo-42183 [https://bugs.python.org/issue?@action=redirect&bpo=42183]: Fix a stack overflow error for asyncio Task or Future repr().

  The overflow occurs under some circumstances when a Task or Future recursively returns itself.

- bpo-42140 [https://bugs.python.org/issue?@action=redirect&bpo=42140]: Improve asyncio.wait function to create the futures set just one time.

- bpo-42133 [https://bugs.python.org/issue?@action=redirect&bpo=42133]: Update various modules in the stdlib to fall back on `__spec__.loader` when `__loader__` isn't defined on a module.

- bpo-26131 [https://bugs.python.org/issue?@action=redirect&bpo=26131]: The `load_module()` methods found in `importlib` now trigger a `DeprecationWarning`.

- bpo-39825 [https://bugs.python.org/issue?@action=redirect&bpo=39825]: Windows: Change `sysconfig.get_config_var('EXT_SUFFIX')` to the expected full `platform_tag.extension` format. Previously it was hard-coded to `.pyd`, now it is compatible with `distutils.sysconfig` and will result in something like `.cp38-win_amd64.pyd`. This brings windows into conformance with the other platforms.

- bpo-26389 [https://bugs.python.org/issue?@action=redirect&bpo=26389]: The **`traceback.format_exception()`**, **`traceback.format_exception_only()`**, and **`traceback.print_exception()`** functions can now take an exception object as a positional-only argument.

- bpo-41889 [https://bugs.python.org/issue?@action=redirect&bpo=41889]: Enum: fix regression involving inheriting a multiply inherited enum

- bpo-41861 [https://bugs.python.org/issue?@action=redirect&bpo=41861]: Convert **`sqlite3`** to use heap types (PEP 384). Patch by Erlend E. Aasland.

- bpo-40624 [https://bugs.python.org/issue?@action=redirect&bpo=40624]: Added support for the XPath `!=` operator in xml.etree

- bpo-28850 [https://bugs.python.org/issue?@action=redirect&bpo=28850]: Fix **`pprint.PrettyPrinter.format()`** overrides being ignored for contents of small containers. The **`pprint._safe_repr()`** function was removed.

- bpo-41625 [https://bugs.python.org/issue?@action=redirect&bpo=41625]: Expose the **`splice()`** as **`os.splice()`** in the **`os`** module. Patch by Pablo Galindo

- bpo-34215 [https://bugs.python.org/issue?@action=redirect&bpo=34215]: Clarify the error message for **`asyncio.IncompleteReadError`** when `expected` is `None`.

- bpo-41543 [https://bugs.python.org/issue?@action=redirect&bpo=41543]: Add async context manager support for contextlib.nullcontext.

- bpo-21041 [https://bugs.python.org/issue?@action=redirect&bpo=21041]: **`pathlib.PurePath.parents`** now supports negative indexing. Patch contributed by Yaroslav Pankovych.

- bpo-41332 [https://bugs.python.org/issue?@action=redirect&bpo=41332]: Added missing connect_accepted_socket() method to `asyncio.AbstractEventLoop`.

- bpo-12800 [https://bugs.python.org/issue?@action=redirect&bpo=12800]: Extracting a symlink from a tarball should succeed and overwrite the symlink if it already exists. The fix is to remove the existing file or symlink before extraction. Based on patch by Chris AtLee, Jeffrey Kintscher, and Senthil Kumaran.

- bpo-40968 [https://bugs.python.org/issue?@action=redirect&bpo=40968]: **`urllib.request`** and **`http.client`** now send `http/1.1` ALPN extension during TLS handshake when no custom context is supplied.

- bpo-41001 [https://bugs.python.org/issue?@action=redirect&bpo=41001]: Add **`os.eventfd()`** to provide a low level interface for Linux's event notification file descriptor.

- bpo-40816 [https://bugs.python.org/issue?@action=redirect&bpo=40816]: Add AsyncContextDecorator to contextlib to support async context manager as a decorator.

- bpo-40550 [https://bugs.python.org/issue?@action=redirect&bpo=40550]: Fix time-of-check/time-of-action issue in subprocess.Popen.send_signal.

- bpo-39411 [https://bugs.python.org/issue?@action=redirect&bpo=39411]: Add an `is_async` identifier to **`pyclbr`**'s `Function` objects. Patch by Batuhan Taskaya

- bpo-35498 [https://bugs.python.org/issue?@action=redirect&bpo=35498]: Add slice support to **`pathlib.PurePath.parents`**.

Documentação

- bpo-42238 [https://bugs.python.org/issue?@action=redirect&bpo=42238]: Tentative to deprecate `make suspicious` by first removing it from the CI and documentation builds, but keeping it around for manual uses.
- bpo-42153 [https://bugs.python.org/issue?@action=redirect&bpo=42153]: Fix the URL for the IMAP protocol documents.
- bpo-41028 [https://bugs.python.org/issue?@action=redirect&bpo=41028]: Language and version switchers, previously maintained in every cpython branches, are now handled by docsbuild-script.

Testes

- bpo-41473 [https://bugs.python.org/issue?@action=redirect&bpo=41473]: Re-enable test_gdb on gdb 9.2 and newer: https://bugzilla.redhat.com/show_bug.cgi?id=1866884 bug is fixed in gdb 10.1.
- bpo-42553 [https://bugs.python.org/issue?@action=redirect&bpo=42553]: Fix `test_asyncio.test_call_later()` race condition: don't measure asyncio performance in the `call_later()` unit test. The test failed randomly on the CI.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix test_netrc on VxWorks: create temporary directories using

temp_cwd().

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: skip test_getaddrinfo_ipv6_scopeid_symbolic and test_getnameinfo_ipv6_scopeid_symbolic on VxWorks
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: skip test_test of test_mailcap on VxWorks
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: add shell requirement for test_pipes
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: skip some tests related to fifo on VxWorks
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix test_doctest.py failures for VxWorks.
- bpo-40754 [https://bugs.python.org/issue?@action=redirect&bpo=40754]: Include `_testinternalcapi` module in Windows installer for test suite
- bpo-41561 [https://bugs.python.org/issue?@action=redirect&bpo=41561]: test_ssl: skip test_min_max_version_mismatch when TLS 1.0 is not available
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix os module failures for VxWorks RTOS.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix fifo test cases for VxWorks RTOS.

Construção

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: remove libnet dependency from detect_socket() for VxWorks
- bpo-42398 [https://bugs.python.org/issue?@action=redirect&bpo=42398]: Fix a race condition in “make regen-all” when make -jN option is used to run jobs in parallel. The clinic.py script now only use atomic write to write files. Moveover, generated files are now left unchanged if the content does not change, to not change the file modification time.
- bpo-41617 [https://bugs.python.org/issue?@action=redirect&bpo=41617]: Fix building `pycore_bitutils.h` internal header on old clang version without `__builtin_bswap16()` (ex: Xcode 4.6.3 on Mac OS X 10.7). Patch by Joshua Root and Victor Stinner.
- bpo-38823 [https://bugs.python.org/issue?@action=redirect&bpo=38823]: It is no longer possible to build the `_ctypes` extension module without

`wchar_t` type: remove `CTYPES_UNICODE` macro. Anyway, the **`wchar_t`** type is required to build Python. Patch by Victor Stinner.

- bpo-42087 [https://bugs.python.org/issue?@action=redirect&bpo=42087]: Support was removed for AIX 5.3 and below. See bpo-40680 [https://bugs.python.org/issue?@action=redirect&bpo=40680].
- bpo-40998 [https://bugs.python.org/issue?@action=redirect&bpo=40998]: Addressed three compiler warnings found by undefined behavior sanitizer (ubsan).

Windows

- bpo-42120 [https://bugs.python.org/issue?@action=redirect&bpo=42120]: Remove macro definition of `copysign` (to `_copysign`) in headers.
- bpo-38506 [https://bugs.python.org/issue?@action=redirect&bpo=38506]: The Windows launcher now properly handles Python 3.10 when listing installed Python versions.

macOS

- bpo-42504 [https://bugs.python.org/issue?@action=redirect&bpo=42504]: Fix build on macOS Big Sur when MACOSX_DEPLOYMENT_TARGET=11

- bpo-41116 [https://bugs.python.org/issue?@action=redirect&bpo=41116]: Ensure distutils.unixxcompiler.find_library_file can find system provided libraries on macOS 11.

- bpo-41100 [https://bugs.python.org/issue?@action=redirect&bpo=41100]: Add support for macOS 11 and Apple Silicon systems.

  It is now possible to build “Universal 2” binaries using “–enable-universalsdk –with-universal-archs=universal2”.

  Binaries build on later macOS versions can be deployed back to older versions (tested up to macOS 10.9), when using the correct deployment target. This is tested using Xcode 11 and later.

- bpo-42232 [https://bugs.python.org/issue?@action=redirect&bpo=42232]: Added Darwin specific madvise options to mmap module.

- bpo-38443 [https://bugs.python.org/issue?@action=redirect&bpo=38443]: The `--enable-universalsdk` and `--with-universal-archs` options for the configure script now check that the specified architectures can be used.

IDLE

- bpo-42508 [https://bugs.python.org/issue?@action=redirect&bpo=42508]: Keep IDLE running on macOS. Remove obsolete workaround that prevented running files with shortcuts when using new universal2 installers built on macOS 11.
- bpo-42426 [https://bugs.python.org/issue?@action=redirect&bpo=42426]: Fix reporting offset of the RE error in searchengine.
- bpo-42415 [https://bugs.python.org/issue?@action=redirect&bpo=42415]: Get docstrings for IDLE calltips more often by using inspect.getdoc.

Ferramentas/Demos

- bpo-42212 [https://bugs.python.org/issue?@action=redirect&bpo=42212]: The smelly.py script now also checks the Python dynamic library and extension modules, not only the Python static library. Make also the script more verbose: explain what it does.
- bpo-36310 [https://bugs.python.org/issue?@action=redirect&bpo=36310]: Allow `Tools/i18n/pygettext.py` to detect calls to `gettext` in f-strings.

API C

- bpo-42423 [https://bugs.python.org/issue?@action=redirect&bpo=42423]: The **`PyType_FromSpecWithBases()`** and **`PyType_FromModuleAndSpec()`** functions now accept a single class as the *bases* argument.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`select`** extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _posixsubprocess extension module to multiphase initialization (**PEP**

489 [https://peps.python.org/pep-0489/]).

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _posixshmem extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _struct extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `spwd` extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `gc` extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _queue extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Convert `Py_TYPE()` and `Py_SIZE()` back to macros to allow using them as an l-value. Many third party C extension modules rely on the ability of using Py_TYPE() and Py_SIZE() to set an object type and size: `Py_TYPE(obj) = type;` and `Py_SIZE(obj) = size;`.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `symtable` extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `grp` and `pwd` extension modules to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _random extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _hashlib extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/])
- bpo-41713 [https://bugs.python.org/issue?@action=redirect&bpo=41713]: Removed the undocumented `PyOS_InitInterrupts()` function. Initializing Python already implicitly installs signal handlers: see `PyConfig.install_signal_handlers`. Patch by Victor Stinner.

- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: The `Py_TRASHCAN_BEGIN` macro no longer accesses PyTypeObject attributes, but now can get the condition by calling the new private **`_PyTrash_cond()`** function which hides implementation details.
- bpo-42260 [https://bugs.python.org/issue?@action=redirect&bpo=42260]: **`Py_GetPath()`**, **`Py_GetPrefix()`**, **`Py_GetExecPrefix()`**, **`Py_GetProgramFullPath()`**, **`Py_GetPythonHome()`** and **`Py_GetProgramName()`** functions now return `NULL` if called before **`Py_Initialize()`** (before Python is initialized). Use the new Python Initialization Configuration API to get the Python Path Configuration.. Patch by Victor Stinner.
- bpo-42260 [https://bugs.python.org/issue?@action=redirect&bpo=42260]: The **`PyConfig_Read()`** function now only parses **`PyConfig.argv`** arguments once: **`PyConfig.parse_argv`** is set to `2` after arguments are parsed. Since Python arguments are strippped from **`PyConfig.argv`**, parsing arguments twice would parse the application options as Python options.
- bpo-42262 [https://bugs.python.org/issue?@action=redirect&bpo=42262]: Added **`Py_NewRef()`** and **`Py_XNewRef()`** functions to increment the reference count of an object and return the object. Patch by Victor Stinner.
- bpo-42260 [https://bugs.python.org/issue?@action=redirect&bpo=42260]: When **`Py_Initialize()`** is called twice, the second call now updates more **`sys`** attributes for the configuration, rather than only **`sys.argv`**. Patch by Victor Stinner.
- bpo-41832 [https://bugs.python.org/issue?@action=redirect&bpo=41832]: The **`PyType_FromModuleAndSpec()`** function now accepts NULL `tp_doc` slot.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Added **`PyModule_AddObjectRef()`** function: similar to **`PyModule_AddObject()`** but don't steal a reference to the value on success. Patch by Victor Stinner.
- bpo-42171 [https://bugs.python.org/issue?@action=redirect&bpo=42171]: The **`METH_FASTCALL`** calling convention is added to the limited API. The functions **`PyModule_AddType()`**, **`PyType_FromModuleAndSpec()`**, **`PyType_GetModule()`** and **`PyType_GetModuleState()`** are added to the limited API on Windows.

- bpo-42085 [https://bugs.python.org/issue?@action=redirect&bpo=42085]: Add dedicated entry to PyAsyncMethods for sending values
- bpo-41073 [https://bugs.python.org/issue?@action=redirect&bpo=41073]: **`PyType_GetSlot()`** can now accept static types.
- bpo-30459 [https://bugs.python.org/issue?@action=redirect&bpo=30459]: **`PyList_SET_ITEM()`**, **`PyTuple_SET_ITEM()`** and **`PyCell_SET()`** macros can no longer be used as l-value or r-value. For example, `x = PyList_SET_ITEM(a, b, c)` and `PyList_SET_ITEM(a, b, c) = x` now fail with a compiler error. It prevents bugs like `if (PyList_SET_ITEM (a, b, c) < 0) ...` test. Patch by Zackery Spytz and Victor Stinner.

Python 3.10.0 alpha 2

Data de lançamento: 2020-11-03

Segurança

- bpo-42103 [https://bugs.python.org/issue?@action=redirect&bpo=42103]: Prevented potential DoS attack via CPU and RAM exhaustion when processing malformed Apple Property List files in binary format.
- bpo-42051 [https://bugs.python.org/issue?@action=redirect&bpo=42051]: The **`plistlib`** module no longer accepts entity declarations in XML plist files to avoid XML vulnerabilities. This should not affect users as entity declarations are not used in regular plist files.

Núcleo e embutidos

- bpo-42236 [https://bugs.python.org/issue?@action=redirect&bpo=42236]: If the `nl_langinfo(CODESET)` function returns an empty string, Python now uses UTF-8 as the filesystem encoding. Patch by Victor Stinner.
- bpo-42218 [https://bugs.python.org/issue?@action=redirect&bpo=42218]: Fixed a bug in the PEG parser that was causing crashes in debug mode. Now errors are checked in left-recursive rules to avoid cases where such errors do not get handled in time and appear as long-distance crashes in other places.

- bpo-42214 [https://bugs.python.org/issue?@action=redirect&bpo=42214]: Fixed a possible crash in the PEG parser when checking for the '!=' token in the `barry_as_flufl` rule. Patch by Pablo Galindo.
- bpo-42206 [https://bugs.python.org/issue?@action=redirect&bpo=42206]: Propagate and raise the errors caused by **`PyAST_Validate()`** in the parser.
- bpo-41796 [https://bugs.python.org/issue?@action=redirect&bpo=41796]: The **`ast`** module internal state is now per interpreter. Patch by Victor Stinner.
- bpo-42143 [https://bugs.python.org/issue?@action=redirect&bpo=42143]: Fix handling of errors during creation of `PyFunctionObject`, which resulted in operations on uninitialized memory. Patch by Yonatan Goldschmidt.
- bpo-41659 [https://bugs.python.org/issue?@action=redirect&bpo=41659]: Fix a bug in the parser, where a curly brace following a `primary` didn't fail immediately. This led to invalid expressions like `a {b}` to throw a **`SyntaxError`** with a wrong offset, or invalid expressions ending with a curly brace like `a {` to not fail immediately in the REPL.
- bpo-42150 [https://bugs.python.org/issue?@action=redirect&bpo=42150]: Fix possible buffer overflow in the new parser when checking for continuation lines. Patch by Pablo Galindo.
- bpo-42123 [https://bugs.python.org/issue?@action=redirect&bpo=42123]: Run the parser two times. On the first run, disable all the rules that only generate better error messages to gain performance. If there's a parse failure, run the parser a second time with those enabled.
- bpo-42093 [https://bugs.python.org/issue?@action=redirect&bpo=42093]: The `LOAD_ATTR` instruction now uses new "per opcode cache" mechanism and it is about 36% faster now. Patch by Pablo Galindo and Yury Selivanov.
- bpo-42030 [https://bugs.python.org/issue?@action=redirect&bpo=42030]: Support for the legacy AIX-specific shared library loading support has been removed. All versions of AIX since 4.3 have supported and defaulted to using the common Unix mechanism instead.
- bpo-41984 [https://bugs.python.org/issue?@action=redirect&bpo=41984]: The garbage collector now tracks all user-defined classes. Patch by Brandt Bucher.
- bpo-41993 [https://bugs.python.org/issue?@action=redirect&bpo=41993]: Fixed potential issues with removing not completely initialized module from `sys.modules` when import fails.

- bpo-41979 [https://bugs.python.org/issue?@action=redirect&bpo=41979]: Star-unpacking is now allowed for with item's targets in the PEG parser.
- bpo-41974 [https://bugs.python.org/issue?@action=redirect&bpo=41974]: Removed special methods `__int__`, `__float__`, `__floordiv__`, `__mod__`, `__divmod__`, `__rfloordiv__`, `__rmod__` and `__rdivmod__` of the **`complex`** class. They always raised a **`TypeError`**.
- bpo-41902 [https://bugs.python.org/issue?@action=redirect&bpo=41902]: Micro optimization when compute **`sq_item`** and **`mp_subscript`** of **`range`**. Patch by Donghee Na.
- bpo-41894 [https://bugs.python.org/issue?@action=redirect&bpo=41894]: When loading a native module and a load failure occurs, prevent a possible UnicodeDecodeError when not running in a UTF-8 locale by decoding the load error message using the current locale's encoding.
- bpo-41902 [https://bugs.python.org/issue?@action=redirect&bpo=41902]: Micro optimization for range.index if step is 1. Patch by Donghee Na.

Biblioteca

- bpo-41435 [https://bugs.python.org/issue?@action=redirect&bpo=41435]: Add `sys._current_exceptions()` function to retrieve a dictionary mapping each thread's identifier to the topmost exception currently active in that thread at the time the function is called.

Núcleo e embutidos

- bpo-38605 [https://bugs.python.org/issue?@action=redirect&bpo=38605]: Enable `from __future__ import annotations` (**PEP 563** [https://peps.python.org/pep-0563/]) by default. The values found in **`__annotations__`** dicts are now strings, for example `{"x": "int"}` instead of `{"x": int}`.

Biblioteca

- bpo-35455 [https://bugs.python.org/issue?@action=redirect&bpo=35455]: On Solaris, **`thread_time()`** is now implemented with `gethrvtime()` because `clock_gettime(CLOCK_THREAD_CPUTIME_ID)` is not always available. Patch by Jakub Kulik.

- bpo-42233 [https://bugs.python.org/issue?@action=redirect&bpo=42233]: The **`repr()`** of **`typing`** types containing Generic Alias Types previously did not show the parameterized types in the `GenericAlias`. They have now been changed to do so.

- bpo-29566 [https://bugs.python.org/issue?@action=redirect&bpo=29566]: `binhex.binhex()` consistently writes macOS 9 line endings.

- bpo-26789 [https://bugs.python.org/issue?@action=redirect&bpo=26789]: The **`logging.FileHandler`** class now keeps a reference to the builtin **`open()`** function to be able to open or reopen the file during Python finalization. Fix errors like: `NameError: name 'open' is not defined`. Patch by Victor Stinner.

- bpo-42157 [https://bugs.python.org/issue?@action=redirect&bpo=42157]: Removed the `unicodedata.ucnhash_CAPI` attribute which was an internal PyCapsule object. The related private `_PyUnicode_Name_CAPI` structure was moved to the internal C API. Patch by Victor Stinner.

- bpo-42157 [https://bugs.python.org/issue?@action=redirect&bpo=42157]: Convert the **`unicodedata`** extension module to the multiphase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]) and convert the `unicodedata.UCD` static type to a heap type. Patch by Mohamed Koubaa and Victor Stinner.

- bpo-42146 [https://bugs.python.org/issue?@action=redirect&bpo=42146]: Fix memory leak in **`subprocess.Popen()`** in case an uid (gid) specified in `user` (`group`, `extra_groups`) overflows `uid_t` (`gid_t`).

- bpo-42103 [https://bugs.python.org/issue?@action=redirect&bpo=42103]: **`InvalidFileException`** and **`RecursionError`** are now the only errors caused by loading malformed binary Plist file (previously ValueError and TypeError could be raised in some specific cases).

- bpo-41490 [https://bugs.python.org/issue?@action=redirect&bpo=41490]: In `importlib.resources`, `.path` method is more aggressive about releasing handles to zipfile objects early, enabling use-cases like certifi to leave the context open but delete the underlying zip file.

- bpo-41052 [https://bugs.python.org/issue?@action=redirect&bpo=41052]: Pickling heap types implemented in C with protocols 0 and 1 raises now an error instead of producing incorrect data.

- bpo-42089 [https://bugs.python.org/issue?@action=redirect&bpo=42089]: In `importlib.metadata.PackageNotFoundError`, make reference to the package metadata being missing to improve the user experience.

- bpo-41491 [https://bugs.python.org/issue?@action=redirect&bpo=41491]: plistlib: fix parsing XML plists with hexadecimal integer values

- bpo-42065 [https://bugs.python.org/issue?@action=redirect&bpo=42065]: Fix an incorrectly formatted error from **`_codecs.charmap_decode()`** when called with a mapped value outside the range of valid Unicode code points. PR by Max Bernstein.

- bpo-41966 [https://bugs.python.org/issue?@action=redirect&bpo=41966]: Fix pickling pure Python **`datetime.time`** subclasses. Patch by Dean Inwood.

- bpo-19270 [https://bugs.python.org/issue?@action=redirect&bpo=19270]: **`sched.scheduler.cancel()`** will now cancel the correct event, if two events with same priority are scheduled for the same time. Patch by Bar Harel.

- bpo-28660 [https://bugs.python.org/issue?@action=redirect&bpo=28660]: **`textwrap.wrap()`** now attempts to break long words after hyphens when `break_long_words=True` and `break_on_hyphens=True`.

- bpo-35823 [https://bugs.python.org/issue?@action=redirect&bpo=35823]: Use `vfork()` instead of `fork()` for **`subprocess.Popen()`** on Linux to improve performance in cases where it is deemed safe.

- bpo-42043 [https://bugs.python.org/issue?@action=redirect&bpo=42043]: Add support for `zipfile.Path` inheritance. `zipfile.Path.is_file()` now returns False for non-existent names. `zipfile.Path` objects now expose a `.filename` attribute and rely on that to resolve `.name` and `.parent` when the `Path` object is at the root of the zipfile.

- bpo-42021 [https://bugs.python.org/issue?@action=redirect&bpo=42021]: Fix possible ref leaks in **`sqlite3`** module init.

- bpo-39101 [https://bugs.python.org/issue?@action=redirect&bpo=39101]: Fixed tests using IsolatedAsyncioTestCase from hanging on BaseExceptions.

- bpo-41976 [https://bugs.python.org/issue?@action=redirect&bpo=41976]: Fixed a bug that was causing **`ctypes.util.find_library()`** to return `None` when triying to locate a library in an environment when gcc>=9 is available and `ldconfig` is not. Patch by Pablo Galindo

- bpo-41943 [https://bugs.python.org/issue?@action=redirect&bpo=41943]: Fix bug where TestCase.assertLogs doesn't correctly filter messages by level.

- bpo-41923 [https://bugs.python.org/issue?@action=redirect&bpo=41923]: Implement **PEP 613** [https://peps.python.org/pep-0613/], introducing **`typing.TypeAlias`** annotation.

- bpo-41905 [https://bugs.python.org/issue?@action=redirect&bpo=41905]: A new function in abc: *update_abstractmethods* to re-calculate an abstract class's abstract status. In addition, *dataclass* has been changed to call this function.

- bpo-23706 [https://bugs.python.org/issue?@action=redirect&bpo=23706]: Added *newline* parameter to `pathlib.Path.write_text()`.

- bpo-41876 [https://bugs.python.org/issue?@action=redirect&bpo=41876]: Tkinter font class repr uses font name

- bpo-41831 [https://bugs.python.org/issue?@action=redirect&bpo=41831]: `str()` for the `type` attribute of the `tkinter.Event` object always returns now the numeric code returned by Tk instead of the name of the event type.

- bpo-39337 [https://bugs.python.org/issue?@action=redirect&bpo=39337]: **`encodings.normalize_encoding()`** now ignores non-ASCII characters.

- bpo-41747 [https://bugs.python.org/issue?@action=redirect&bpo=41747]: Ensure all methods that generated from `dataclasses.dataclass()` objects now have the proper `__qualname__` attribute referring to the class they belong to. Patch by Batuhan Taskaya.

- bpo-30681 [https://bugs.python.org/issue?@action=redirect&bpo=30681]: Handle exceptions caused by unparsable date headers when using email "default" policy. Patch by Tim Bell, Georges Toth

- bpo-41586 [https://bugs.python.org/issue?@action=redirect&bpo=41586]: Add F_SETPIPE_SZ and F_GETPIPE_SZ to fcntl module. Allow setting pipesize on subprocess.Popen.

- bpo-41229 [https://bugs.python.org/issue?@action=redirect&bpo=41229]: Add `contextlib.aclosing` for deterministic cleanup of async generators which is analogous to `contextlib.closing` for non-async generators. Patch by Joongi Kim and John Belmonte.

- bpo-16396 [https://bugs.python.org/issue?@action=redirect&bpo=16396]: Allow `ctypes.wintypes` to be imported on non-Windows systems.

- bpo-4356 [https://bugs.python.org/issue?@action=redirect&bpo=4356]: Add a key function to the bisect module.

- bpo-40592 [https://bugs.python.org/issue?@action=redirect&bpo=40592]: `shutil.which()` now ignores empty entries in **`PATHEXT`** instead of treating them as a match.

- bpo-40492 [https://bugs.python.org/issue?@action=redirect&bpo=40492]: Fix `--outfile` for **`cProfile`** / **`profile`** not writing the output file in the original directory when the program being profiled changes the working directory. PR by Anthony Sottile.

- bpo-34204 [https://bugs.python.org/issue?@action=redirect&bpo=34204]: The `shelve` module now uses `pickle.DEFAULT_PROTOCOL` by default instead of `pickle` protocol `3`.

- bpo-27321 [https://bugs.python.org/issue?@action=redirect&bpo=27321]: Fixed KeyError exception when flattening an email to a string attempts to replace a non-existent Content-Transfer-Encoding header.

- bpo-38976 [https://bugs.python.org/issue?@action=redirect&bpo=38976]: The **`http.cookiejar`** module now supports the parsing of cookies in CURL-style cookiejar files through MozillaCookieJar on all platforms. Previously, such cookie entries would be silently ignored when loading a cookiejar with such entries.

  Additionally, the HTTP Only attribute is persisted in the object, and will be correctly written to file if the MozillaCookieJar object is subsequently dumped.

Documentação

- bpo-42061 [https://bugs.python.org/issue?@action=redirect&bpo=42061]: Document __format__ functionality for IP addresses.
- bpo-41910 [https://bugs.python.org/issue?@action=redirect&bpo=41910]: Document the default implementation of `object.__eq__`.
- bpo-42010 [https://bugs.python.org/issue?@action=redirect&bpo=42010]: Clarify that subscription expressions are also valid for certain classes and types in the standard library, and for user-defined classes and types if the classmethod **`__class_getitem__()`** is provided.
- bpo-41805 [https://bugs.python.org/issue?@action=redirect&bpo=41805]: Documented generic alias type and **`types.GenericAlias`**. Also added an entry in glossary for generic types.
- bpo-39693 [https://bugs.python.org/issue?@action=redirect&bpo=39693]: Fix tarfile's extractfile documentation
- bpo-39416 [https://bugs.python.org/issue?@action=redirect&bpo=39416]: Document some restrictions on the default string representations of numeric classes.

Testes

- bpo-41739 [https://bugs.python.org/issue?@action=redirect&bpo=41739]: Fix test_logging.test_race_between_set_target_and_flush(): the test now

waits until all threads complete to avoid leaking running threads.

- bpo-41970 [https://bugs.python.org/issue?@action=redirect&bpo=41970]: Avoid a test failure in `test_lib2to3` if the module has already imported at the time the test executes. Patch by Pablo Galindo.
- bpo-41944 [https://bugs.python.org/issue?@action=redirect&bpo=41944]: Tests for CJK codecs no longer call `eval()` on content received via HTTP.
- bpo-41306 [https://bugs.python.org/issue?@action=redirect&bpo=41306]: Fixed a failure in `test_tk.test_widgets.ScaleTest` happening when executing the test with Tk 8.6.10.

Construção

- bpo-38980 [https://bugs.python.org/issue?@action=redirect&bpo=38980]: Add `-fno-semantic-interposition` to both the compile and link line when building with `--enable-optimizations`. Patch by Victor Stinner and Pablo Galindo.

Windows

- bpo-38439 [https://bugs.python.org/issue?@action=redirect&bpo=38439]: Updates the icons for IDLE in the Windows Store package.
- bpo-38252 [https://bugs.python.org/issue?@action=redirect&bpo=38252]: Use 8-byte step to detect ASCII sequence in 64-bit Windows build.
- bpo-39107 [https://bugs.python.org/issue?@action=redirect&bpo=39107]: Update Tcl and Tk to 8.6.10 in Windows installer.
- bpo-41557 [https://bugs.python.org/issue?@action=redirect&bpo=41557]: Update Windows installer to use SQLite 3.33.0.
- bpo-38324 [https://bugs.python.org/issue?@action=redirect&bpo=38324]: Avoid Unicode errors when accessing certain locale data on Windows.

macOS

- bpo-41471 [https://bugs.python.org/issue?@action=redirect&bpo=41471]: Ignore invalid prefix lengths in system proxy excludes.

IDLE

- bpo-33987 [https://bugs.python.org/issue?@action=redirect&bpo=33987]: Mostly finish using ttk widgets, mainly for editor, settings, and searches. Some patches by Mark Roseman.
- bpo-40511 [https://bugs.python.org/issue?@action=redirect&bpo=40511]: Typing opening and closing parentheses inside the parentheses of a function call will no longer cause unnecessary “flashing” off and on of an existing open call-tip, e.g. when typed in a string literal.
- bpo-38439 [https://bugs.python.org/issue?@action=redirect&bpo=38439]: Add a 256×256 pixel IDLE icon to the Windows .ico file. Created by Andrew Clover. Remove the low-color gif variations from the .ico file.

API C

- bpo-42157 [https://bugs.python.org/issue?@action=redirect&bpo=42157]: The private `_PyUnicode_Name_CAPI` structure of the PyCapsule API `unicodedata.ucnhash_CAPI` has been moved to the internal C API. Patch by Victor Stinner.
- bpo-42015 [https://bugs.python.org/issue?@action=redirect&bpo=42015]: Fix potential crash in deallocating method objects when dynamically allocated **`PyMethodDef`**’s lifetime is managed through the `self` argument of a **`PyCFunction`**.
- bpo-40423 [https://bugs.python.org/issue?@action=redirect&bpo=40423]: The **`subprocess`** module and `os.closerange` will now use the `close_range(low, high, flags)` syscall when it is available for more efficient closing of ranges of descriptors.
- bpo-41845 [https://bugs.python.org/issue?@action=redirect&bpo=41845]: **`PyObject_GenericGetDict()`** is available again in the limited API when targeting 3.10 or later.
- bpo-40422 [https://bugs.python.org/issue?@action=redirect&bpo=40422]: Add `_Py_closerange` function to provide performant closing of a range of file descriptors.
- bpo-41986 [https://bugs.python.org/issue?@action=redirect&bpo=41986]: **`Py_FileSystemDefaultEncodeErrors`** and **`Py_UTF8Mode`** are available again in limited API.
- bpo-41756 [https://bugs.python.org/issue?@action=redirect&bpo=41756]: Add `PyIter_Send` function to allow sending value into

generator/coroutine/iterator without raising StopIteration exception to signal return.
- bpo-41784 [https://bugs.python.org/issue?@action=redirect&bpo=41784]: Added `PyUnicode_AsUTF8AndSize` to the limited C API.

Python 3.10.0 alpha 1

Data de lançamento: 2020-10-05

Segurança

- bpo-41304 [https://bugs.python.org/issue?@action=redirect&bpo=41304]: Fixes `python3x._pth` being ignored on Windows, caused by the fix for bpo-29778 [https://bugs.python.org/issue?@action=redirect&bpo=29778] (**CVE 2020-15801** [https://www.cve.org/CVERecord?id=CVE-2020-15801]).
- bpo-41162 [https://bugs.python.org/issue?@action=redirect&bpo=41162]: Audit hooks are now cleared later during finalization to avoid missing events.
- bpo-29778 [https://bugs.python.org/issue?@action=redirect&bpo=29778]: Ensure `python3.dll` is loaded from correct locations when Python is embedded (**CVE 2020-15523** [https://www.cve.org/CVERecord?id=CVE-2020-15523]).
- bpo-41004 [https://bugs.python.org/issue?@action=redirect&bpo=41004]: The __hash__() methods of ipaddress.IPv4Interface and ipaddress.IPv6Interface incorrectly generated constant hash values of 32 and 128 respectively. This resulted in always causing hash collisions. The fix uses hash() to generate hash values for the tuple of (address, mask length, network address).
- bpo-39603 [https://bugs.python.org/issue?@action=redirect&bpo=39603]: Prevent http header injection by rejecting control characters in http.client.putrequest(…).

Núcleo e embutidos

- bpo-41909 [https://bugs.python.org/issue?@action=redirect&bpo=41909]: Fixed stack overflow in **`issubclass()`** and **`isinstance()`** when getting the

`__bases__` attribute leads to infinite recursion.

- bpo-41922 [https://bugs.python.org/issue?@action=redirect&bpo=41922]: Speed up calls to `reversed()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.
- bpo-41873 [https://bugs.python.org/issue?@action=redirect&bpo=41873]: Calls to `float()` are now faster due to the `vectorcall` calling convention. Patch by Dennis Sweeney.
- bpo-41870 [https://bugs.python.org/issue?@action=redirect&bpo=41870]: Speed up calls to `bool()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the **`_bisect`** module to the multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]).

Núcleo e embutidos

- bpo-39934 [https://bugs.python.org/issue?@action=redirect&bpo=39934]: Correctly count control blocks in ‘except’ in compiler. Ensures that a syntax error, rather a fatal error, occurs for deeply nested, named exception handlers.
- bpo-41780 [https://bugs.python.org/issue?@action=redirect&bpo=41780]: Fix **`__dir__()`** of **`types.GenericAlias`**. Patch by Batuhan Taskaya.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the **`_lsprof`** extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the **`cmath`** extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the **`_scproxy`** extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `termios` extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Convert the `_sha256` extension module types to heap types.

Núcleo e embutidos

- bpo-41690 [https://bugs.python.org/issue?@action=redirect&bpo=41690]: Fix a possible stack overflow in the parser when parsing functions and classes with a huge amount of arguments. Patch by Pablo Galindo.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `_overlapped` extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `_curses_panel` extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `_opcode` extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

Núcleo e embutidos

- bpo-41681 [https://bugs.python.org/issue?@action=redirect&bpo=41681]: Fixes the wrong error description in the error raised by using `2 ,` in format string in f-string and `str.format()`.

Biblioteca

- bpo-41675 [https://bugs.python.org/issue?@action=redirect&bpo=41675]: The implementation of `signal.siginterrupt()` now uses `sigaction()` (if it is available in the system) instead of the deprecated `siginterrupt()`. Patch by Pablo Galindo.

Núcleo e embutidos

- bpo-41670 [https://bugs.python.org/issue?@action=redirect&bpo=41670]: Prevent line trace being skipped on platforms not compiled with `USE_COMPUTED_GOTOS`. Fixes issue where some lines nested within a try-except block were not being traced on Windows.
- bpo-41654 [https://bugs.python.org/issue?@action=redirect&bpo=41654]: Fix a crash that occurred when destroying subclasses of `MemoryError`. Patch by Pablo Galindo.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `zlib` extension module to multi-phase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

Núcleo e embutidos

- bpo-41631 [https://bugs.python.org/issue?@action=redirect&bpo=41631]: The `_ast` module uses again a global state. Using a module state per module instance is causing subtle practical problems. For example, the Mercurial project replaces the `__import__()` function to implement lazy import, whereas Python expected that `import _ast` always return a fully initialized `_ast` module.

Biblioteca

- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert `_operator` to use `PyType_FromSpec()`.

Núcleo e embutidos

- bpo-1653741 [https://bugs.python.org/issue?@action=redirect&bpo=1653741]: Port `_sha3` to multi-phase init. Convert static types to heap types.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the **`_blake2`** extension module to the multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]).

Núcleo e embutidos

- bpo-41533 [https://bugs.python.org/issue?@action=redirect&bpo=41533]: Free the stack allocated in `va_build_stack` if `do_mkstack` fails and the stack is not a `small_stack`.
- bpo-41531 [https://bugs.python.org/issue?@action=redirect&bpo=41531]: Fix a bug that was dropping keys when compiling dict literals with more than 0xFFFF elements. Patch by Pablo Galindo.
- bpo-41525 [https://bugs.python.org/issue?@action=redirect&bpo=41525]: The output of `python --help` contains now only ASCII characters.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the **`_sha1`**, **`_sha512`**, and **`_md5`** extension modules to multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]).

Núcleo e embutidos

- bpo-41431 [https://bugs.python.org/issue?@action=redirect&bpo=41431]: Optimize `dict_merge()` for copying dict (e.g. `dict(d)` and `{}.update(d)`).
- bpo-41428 [https://bugs.python.org/issue?@action=redirect&bpo=41428]: Implement PEP 604. This supports (int | str) etc. in place of Union[str, int].
- bpo-41340 [https://bugs.python.org/issue?@action=redirect&bpo=41340]: Removed fallback implementation for `strdup`.
- bpo-38156 [https://bugs.python.org/issue?@action=redirect&bpo=38156]: Handle interrupts that come after EOF correctly in `PyOS_StdioReadline`.
- bpo-41342 [https://bugs.python.org/issue?@action=redirect&bpo=41342]: **`round()`** with integer argument is now faster (9–60%).
- bpo-41334 [https://bugs.python.org/issue?@action=redirect&bpo=41334]: Constructors **`str()`**, **`bytes()`** and **`bytearray()`** are now faster (around 30–40% for small objects).

- bpo-41295 [https://bugs.python.org/issue?@action=redirect&bpo=41295]: Resolve a regression in CPython 3.8.4 where defining "__setattr__" in a multi-inheritance setup and calling up the hierarchy chain could fail if builtins/extension types were involved in the base types.
- bpo-41323 [https://bugs.python.org/issue?@action=redirect&bpo=41323]: Bytecode optimizations are performed directly on the control flow graph. This will result in slightly more compact code objects in some circumstances.
- bpo-41247 [https://bugs.python.org/issue?@action=redirect&bpo=41247]: Always cache the running loop holder when running `asyncio.set_running_loop`.
- bpo-41252 [https://bugs.python.org/issue?@action=redirect&bpo=41252]: Fix incorrect refcounting in _ssl.c's `_servername_callback()`.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`multiprocessing`** to multi-phase initialization

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`winapi`** to multiphase initialization

Núcleo e embutidos

- bpo-41215 [https://bugs.python.org/issue?@action=redirect&bpo=41215]: Use non-NULL default values in the PEG parser keyword list to overcome a bug that was preventing Python from being properly compiled when using the XLC compiler. Patch by Pablo Galindo.
- bpo-41218 [https://bugs.python.org/issue?@action=redirect&bpo=41218]: Python 3.8.3 had a regression where compiling with ast.PyCF_ALLOW_TOP_LEVEL_AWAIT would aggressively mark list comprehension with CO_COROUTINE. Now only list comprehension making use of async/await will tagged as so.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`faulthandler`** to multiphase initialization.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`sha256`** to multiphase initialization

Núcleo e embutidos

- bpo-41175 [https://bugs.python.org/issue?@action=redirect&bpo=41175]: Guard against a NULL pointer dereference within bytearrayobject triggered by the `bytearray() + bytearray()` operation.
- bpo-41100 [https://bugs.python.org/issue?@action=redirect&bpo=41100]: add arm64 to the allowable Mac OS arches in mpdecimal.h
- bpo-41094 [https://bugs.python.org/issue?@action=redirect&bpo=41094]: Fix decoding errors with audit when open files with non-ASCII names on non-UTF-8 locale.
- bpo-39960 [https://bugs.python.org/issue?@action=redirect&bpo=39960]: The “hackcheck” that prevents sneaking around a type’s __setattr__() by calling the superclass method was rewritten to allow C implemented heap types.
- bpo-41084 [https://bugs.python.org/issue?@action=redirect&bpo=41084]: Prefix the error message with ‘f-string: ‘, when parsing an f-string expression which throws a **`SyntaxError`**.
- bpo-40521 [https://bugs.python.org/issue?@action=redirect&bpo=40521]: Empty frozensets are no longer singletons.
- bpo-41076 [https://bugs.python.org/issue?@action=redirect&bpo=41076]: Pre-feed the parser with the location of the f-string expression, not the f-string itself, which allows us to skip the shifting of the AST node locations after the parsing is completed.
- bpo-41056 [https://bugs.python.org/issue?@action=redirect&bpo=41056]: Fixes a reference to deallocated stack space during startup when constructing sys.path involving a relative symlink when code was supplied via -c. (discovered via Coverity)
- bpo-41061 [https://bugs.python.org/issue?@action=redirect&bpo=41061]: Fix incorrect expressions and asserts in hashtable code and tests.
- bpo-41052 [https://bugs.python.org/issue?@action=redirect&bpo=41052]: Opt out serialization/deserialization for _random.Random
- bpo-40939 [https://bugs.python.org/issue?@action=redirect&bpo=40939]: Rename `PyPegen*` functions to `PyParser*`, so that we can remove the old set of

`PyParser*` functions that were using the old parser, but keep everything backwards-compatible.

- bpo-35975 [https://bugs.python.org/issue?@action=redirect&bpo=35975]: Stefan Behnel reported that cf_feature_version is used even when PyCF_ONLY_AST is not set. This is against the intention and against the documented behavior, so it's been fixed.
- bpo-40939 [https://bugs.python.org/issue?@action=redirect&bpo=40939]: Remove the remaining files from the old parser and the **`symbol`** module.

Biblioteca

- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert **`_bz2`** to use **`PyType_FromSpec()`**.

Núcleo e embutidos

- bpo-41006 [https://bugs.python.org/issue?@action=redirect&bpo=41006]: The `encodings.latin_1` module is no longer imported at startup. Now it is only imported when it is the filesystem encoding or the stdio encoding.
- bpo-40636 [https://bugs.python.org/issue?@action=redirect&bpo=40636]: **`zip()`** now supports **PEP 618** [https://peps.python.org/pep-0618/]'s `strict` parameter, which raises a **`ValueError`** if the arguments are exhausted at different lengths. Patch by Brandt Bucher.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`_gdbm`** to multiphase initialization.
- bpo-40985 [https://bugs.python.org/issue?@action=redirect&bpo=40985]: Fix a bug that caused the **`SyntaxError`** text to be empty when a file ends with a line ending in a line continuation character (i.e. backslash). The error text should contain the text of the last line.
- bpo-40958 [https://bugs.python.org/issue?@action=redirect&bpo=40958]: Fix a possible buffer overflow in the PEG parser when gathering information for emitting syntax errors. Patch by Pablo Galindo.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`_dbm`** to multiphase initialization.
- bpo-40957 [https://bugs.python.org/issue?@action=redirect&bpo=40957]: Fix refleak in _Py_fopen_obj() when PySys_Audit() fails

Biblioteca

- bpo-40950 [https://bugs.python.org/issue?@action=redirect&bpo=40950]: Add a state to the **`nis`** module (**PEP 3121** [https://peps.python.org/pep-3121/]) and apply the multiphase initialization. Patch by Donghee Na.

Núcleo e embutidos

- bpo-40947 [https://bugs.python.org/issue?@action=redirect&bpo=40947]: The Python Path Configuration now takes **`PyConfig.platlibdir`** in account.

- bpo-40939 [https://bugs.python.org/issue?@action=redirect&bpo=40939]: Remove the old parser, the **`parser`** module and all associated support code, command-line options and environment variables. Patch by Pablo Galindo.

- bpo-40847 [https://bugs.python.org/issue?@action=redirect&bpo=40847]: Fix a bug where a line with only a line continuation character is not considered a blank line at tokenizer level. In such cases, more than a single `NEWLINE` token was emitted. The old parser was working around the issue, but the new parser threw a **`SyntaxError`** for valid input due to this. For example, an empty line following a line continuation character was interpreted as a **`SyntaxError`**.

- bpo-40890 [https://bugs.python.org/issue?@action=redirect&bpo=40890]: Each dictionary view now has a `mapping` attribute that provides a **`types.MappingProxyType`** wrapping the original dictionary. Patch contributed by Dennis Sweeney.

- bpo-40889 [https://bugs.python.org/issue?@action=redirect&bpo=40889]: Improved the performance of symmetric difference operations on dictionary item views. Patch by Dennis Sweeney.

- bpo-40904 [https://bugs.python.org/issue?@action=redirect&bpo=40904]: Fix possible segfault in the new PEG parser when parsing f-string containing yield statements with no value (`f"{yield}"`). Patch by Pablo Galindo

- bpo-40903 [https://bugs.python.org/issue?@action=redirect&bpo=40903]: Fixed a possible segfault in the new PEG parser when producing error messages for invalid assignments of the form `p=p=`. Patch by Pablo Galindo

- bpo-40880 [https://bugs.python.org/issue?@action=redirect&bpo=40880]: Fix invalid memory read in the new parser when checking newlines in string literals. Patch by Pablo Galindo.

- bpo-40883 [https://bugs.python.org/issue?@action=redirect&bpo=40883]: Fix memory leak in when parsing f-strings in the new parser. Patch by Pablo Galindo

- bpo-40870 [https://bugs.python.org/issue?@action=redirect&bpo=40870]: Raise **`ValueError`** when validating custom AST's where the constants `True`, `False` and `None` are used within a **`ast.Name`** node.

- bpo-40854 [https://bugs.python.org/issue?@action=redirect&bpo=40854]: Allow overriding **`sys.platlibdir`** via a new **`PYTHONPLATLIBDIR`** environment variable.

- bpo-40826 [https://bugs.python.org/issue?@action=redirect&bpo=40826]: Fix GIL usage in **`PyOS_Readline()`**: lock the GIL to set an exception and pass the Python thread state when checking if there is a pending signal.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`fcntl`** to multiphase initialization.

- bpo-19468 [https://bugs.python.org/issue?@action=redirect&bpo=19468]: Delete unnecessary instance check in importlib.reload(). Patch by Furkan Önder.

- bpo-40824 [https://bugs.python.org/issue?@action=redirect&bpo=40824]: Unexpected errors in calling the `__iter__` method are no longer masked by `TypeError` in the **`in`** operator and functions **`contains()`**, **`indexOf()`** and **`countOf()`** of the **`operator`** module.

- bpo-40792 [https://bugs.python.org/issue?@action=redirect&bpo=40792]: Attributes `start`, `stop` and `step` of the `range` object now always has exact type `int`. Previously, they could have been an instance of a subclass of `int`.

- bpo-40780 [https://bugs.python.org/issue?@action=redirect&bpo=40780]: Fix a corner case where g-style string formatting of a float failed to remove trailing zeros.

- bpo-38964 [https://bugs.python.org/issue?@action=redirect&bpo=38964]: When there's a `SyntaxError` in the expression part of an fstring, the filename attribute of the `SyntaxError` gets correctly set to the name of the file the fstring resides in.

- bpo-40750 [https://bugs.python.org/issue?@action=redirect&bpo=40750]: Support the "-d" debug flag in the new PEG parser. Patch by Pablo Galindo

- bpo-40217 [https://bugs.python.org/issue?@action=redirect&bpo=40217]: Instances of types created with `PyType_FromSpecWithBases()` will no longer automatically visit their class object when traversing references in the garbage collector. The user is expected to manually visit the object's class. Patch by Pablo Galindo.

- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: `Py_TYPE()` is changed to the inline static function. Patch by Donghee Na.

- bpo-40696 [https://bugs.python.org/issue?@action=redirect&bpo=40696]: Fix a hang that can arise after `generator.throw()` due to a cycle in the exception context chain.

- bpo-40521 [https://bugs.python.org/issue?@action=redirect&bpo=40521]: Each interpreter now its has own free lists, singletons and caches:

  - Free lists: float, tuple, list, dict, frame, context, asynchronous generator, MemoryError.
  - Singletons: empty tuple, empty bytes string, empty Unicode string, single byte character, single Unicode (latin1) character.
  - Slice cache.

They are no longer shared by all interpreters.

- bpo-40679 [https://bugs.python.org/issue?@action=redirect&bpo=40679]: Certain `TypeError` messages about missing or extra arguments now include the function's qualified name. Patch by Dennis Sweeney.

- bpo-29590 [https://bugs.python.org/issue?@action=redirect&bpo=29590]: Make the stack trace correct after calling `generator.throw()` on a generator that has yielded from a `yield from`.

- bpo-4022 [https://bugs.python.org/issue?@action=redirect&bpo=4022]: Improve performance of generators by not raising internal StopIteration.

Biblioteca

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `mmap` to multiphase initialization.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `_lzma` to multiphase initialization.

Núcleo e embutidos

- bpo-37999 [https://bugs.python.org/issue?@action=redirect&bpo=37999]: Builtin and extension functions that take integer arguments no longer accept `Decimal`s, `Fraction`s and other objects that can be converted to integers only with a loss (e.g. that have the `__int__()` method but do not have the `__index__()` method).
- bpo-29882 [https://bugs.python.org/issue?@action=redirect&bpo=29882]: Add `int.bit_count()`, counting the number of ones in the binary representation of an integer. Patch by Niklas Fiekas.

Biblioteca

- bpo-36982 [https://bugs.python.org/issue?@action=redirect&bpo=36982]: Use ncurses extended color functions when available to support terminals with 256 colors, and add the new function

`curses.has_extended_color_support()` to indicate whether extended color support is provided by the underlying ncurses library.

Núcleo e embutidos

- bpo-19569 [https://bugs.python.org/issue?@action=redirect&bpo=19569]: Add the private macros `_Py_COMP_DIAG_PUSH`, `_Py_COMP_DIAG_IGNORE_DEPR_DECLS`, and `_Py_COMP_DIAG_POP`.
- bpo-26680 [https://bugs.python.org/issue?@action=redirect&bpo=26680]: The int type now supports the x.is_integer() method for compatibility with float.

Biblioteca

- bpo-41900 [https://bugs.python.org/issue?@action=redirect&bpo=41900]: C14N 2.0 serialisation in xml.etree.ElementTree failed for unprefixed attributes when a default namespace was defined.

- bpo-41887 [https://bugs.python.org/issue?@action=redirect&bpo=41887]: Strip leading spaces and tabs on `ast.literal_eval()`. Also document stripping of spaces and tabs for `eval()`.

- bpo-41773 [https://bugs.python.org/issue?@action=redirect&bpo=41773]: Note in documentation that `random.choices()` doesn't support non-finite weights, raise `ValueError` when given non-finite weights.

- bpo-41840 [https://bugs.python.org/issue?@action=redirect&bpo=41840]: Fix a bug in the `symtable` module that was causing module-scope global variables to not be reported as both local and global. Patch by Pablo Galindo.

- bpo-41842 [https://bugs.python.org/issue?@action=redirect&bpo=41842]: Add `codecs.unregister()` function to unregister a codec search function.

- bpo-40564 [https://bugs.python.org/issue?@action=redirect&bpo=40564]: In `zipfile.Path`, mutate the passed ZipFile object type instead of

making a copy. Prevents issues when both the local copy and the caller's copy attempt to close the same file handle.

- bpo-40670 [https://bugs.python.org/issue?@action=redirect&bpo=40670]: More reliable validation of statements in `timeit.Timer`. It now accepts "empty" statements (only whitespaces and comments) and rejects misindentent statements.

- bpo-41833 [https://bugs.python.org/issue?@action=redirect&bpo=41833]: The `threading.Thread` constructor now uses the target name if the *target* argument is specified but the *name* argument is omitted.

- bpo-41817 [https://bugs.python.org/issue?@action=redirect&bpo=41817]: fix `tkinter.EventType` Enum so all members are strings, and none are tuples

- bpo-41810 [https://bugs.python.org/issue?@action=redirect&bpo=41810]: `types.EllipsisType`, `types.NotImplementedType` and `types.NoneType` have been reintroduced, providing a new set of types readily interpretable by static type checkers.

- bpo-41815 [https://bugs.python.org/issue?@action=redirect&bpo=41815]: Fix SQLite3 segfault when backing up closed database. Patch contributed by Peter David McCormick.

- bpo-41816 [https://bugs.python.org/issue?@action=redirect&bpo=41816]: StrEnum added: it ensures that all members are already strings or string candidates

- bpo-41517 [https://bugs.python.org/issue?@action=redirect&bpo=41517]: fix bug allowing Enums to be extended via multiple inheritance

- bpo-39587 [https://bugs.python.org/issue?@action=redirect&bpo=39587]: use the correct mix-in data type when constructing Enums

- bpo-41792 [https://bugs.python.org/issue?@action=redirect&bpo=41792]: Add is_typeddict function to typing.py to check if a type is a TypedDict class

Previously there was no way to check that without using private API. See the relevant issue in python/typing [https://github.com/python/typing/issues/751].

- bpo-41789 [https://bugs.python.org/issue?@action=redirect&bpo=41789]: Honor `object` overrides in `Enum` class creation (specifically, `__str__`, `__repr__`, `__format__`, and `__reduce_ex__`).

- bpo-32218 [https://bugs.python.org/issue?@action=redirect&bpo=32218]: `enum.Flag` and `enum.IntFlag` members are now iterable.

- bpo-39651 [https://bugs.python.org/issue?@action=redirect&bpo=39651]: Fix a race condition in the `call_soon_threadsafe()` method of `asyncio.ProactorEventLoop`: do nothing if the self-pipe socket has been closed.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `mashal` extension module to the multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port the `_string` extension module to the multi-phase initialization API (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-41732 [https://bugs.python.org/issue?@action=redirect&bpo=41732]: Added an iterator to **`memoryview`**.

- bpo-41720 [https://bugs.python.org/issue?@action=redirect&bpo=41720]: Fixed **`turtle.Vec2D.__rmul__()`** for arguments which are not int or float.

- bpo-41696 [https://bugs.python.org/issue?@action=redirect&bpo=41696]: Fix handling of debug mode in **`asyncio.run()`**. This allows setting `PYTHONASYNCIODEBUG` or `-X dev` to enable asyncio debug mode when using **`asyncio.run()`**.

- bpo-41687 [https://bugs.python.org/issue?@action=redirect&bpo=41687]: Fix implementation of sendfile to be compatible with Solaris.

- bpo-41662 [https://bugs.python.org/issue?@action=redirect&bpo=41662]: No longer override exceptions raised in `__len__()` of a sequence of parameters in **`sqlite3`** with **`ProgrammingError`**.

- bpo-39010 [https://bugs.python.org/issue?@action=redirect&bpo=39010]: Restarting a `ProactorEventLoop` on Windows no longer logs spurious `ConnectionResetErrors`.

- bpo-41638 [https://bugs.python.org/issue?@action=redirect&bpo=41638]: **`ProgrammingError`** message for absent parameter in **`sqlite3`** contains now the name of the parameter instead of its index when parameters are supplied as a dict.

- bpo-41662 [https://bugs.python.org/issue?@action=redirect&bpo=41662]: Fixed crash when mutate list of parameters during iteration in **`sqlite3`**.

- bpo-41513 [https://bugs.python.org/issue?@action=redirect&bpo=41513]: Improved the accuracy of math.hypot(). Internally, each step is computed with extra precision so that the result is now almost always correctly rounded.

- bpo-41609 [https://bugs.python.org/issue?@action=redirect&bpo=41609]: The pdb whatis command correctly reports instance methods as ‘Method’ rather than ‘Function’.

- bpo-39994 [https://bugs.python.org/issue?@action=redirect&bpo=39994]: Fixed pprint’s handling of dict subclasses that override __repr__.

- bpo-32751 [https://bugs.python.org/issue?@action=redirect&bpo=32751]: When cancelling the task due to a timeout, **`asyncio.wait_for()`** will now wait until the cancellation is complete also in the case when *timeout* is <= 0, like it does with positive timeouts.

- bpo-37658 [https://bugs.python.org/issue?@action=redirect&bpo=37658]: **`asyncio.wait_for()`** now properly handles races between cancellation of itself and the completion of the wrapped awaitable.

- bpo-40782 [https://bugs.python.org/issue?@action=redirect&bpo=40782]: Change the method asyncio.AbstractEventLoop.run_in_executor to not be a coroutine.

- bpo-41520 [https://bugs.python.org/issue?@action=redirect&bpo=41520]: Fix `codeop` regression that prevented turning compile warnings into errors.

- bpo-41528 [https://bugs.python.org/issue?@action=redirect&bpo=41528]: turtle uses math module functions to convert degrees to radians and vice versa and to calculate vector norm

- bpo-41513 [https://bugs.python.org/issue?@action=redirect&bpo=41513]: Minor algorithmic improvement to math.hypot() and math.dist() giving small gains in speed and accuracy.

- bpo-41503 [https://bugs.python.org/issue?@action=redirect&bpo=41503]: Fixed a race between setTarget and flush in logging.handlers.MemoryHandler.

- bpo-41497 [https://bugs.python.org/issue?@action=redirect&bpo=41497]: Fix potential UnicodeDecodeError in dis module.

- bpo-41467 [https://bugs.python.org/issue?@action=redirect&bpo=41467]: On Windows, fix asyncio `recv_into()` return value when the socket/pipe is closed (`BrokenPipeError`): return `0` rather than an empty byte string (`b''`).

- bpo-41425 [https://bugs.python.org/issue?@action=redirect&bpo=41425]: Make tkinter doc example runnable.

- bpo-41421 [https://bugs.python.org/issue?@action=redirect&bpo=41421]: Make an algebraic simplification to random.paretovariate(). It now is slightly less subject to round-off error and is slightly faster. Inputs that used to cause ZeroDivisionError now cause an OverflowError instead.

- bpo-41440 [https://bugs.python.org/issue?@action=redirect&bpo=41440]: Add `os.cpu_count()` support for VxWorks RTOS.

- bpo-41316 [https://bugs.python.org/issue?@action=redirect&bpo=41316]: Fix the **`tarfile`** module to write only basename of TAR file to GZIP compression header.

- bpo-41384 [https://bugs.python.org/issue?@action=redirect&bpo=41384]: Raise TclError instead of TypeError when an unknown option is passed to tkinter.OptionMenu.

- bpo-41317 [https://bugs.python.org/issue?@action=redirect&bpo=41317]: Use add_done_callback() in asyncio.loop.sock_accept() to unsubscribe reader early on cancellation.

- bpo-41364 [https://bugs.python.org/issue?@action=redirect&bpo=41364]: Reduce import overhead of **`uuid`**.

- bpo-35328 [https://bugs.python.org/issue?@action=redirect&bpo=35328]: Set the environment variable `VIRTUAL_ENV_PROMPT` at **`venv`** activation.

- bpo-41341 [https://bugs.python.org/issue?@action=redirect&bpo=41341]: Recursive evaluation of `typing.ForwardRef` in `get_type_hints`.

- bpo-41344 [https://bugs.python.org/issue?@action=redirect&bpo=41344]: Prevent creating **`shared_memory.SharedMemory`** objects with `size=0`.

- bpo-41333 [https://bugs.python.org/issue?@action=redirect&bpo=41333]: **`collections.OrderedDict.pop()`** is now 2 times faster.

- bpo-41288 [https://bugs.python.org/issue?@action=redirect&bpo=41288]: Unpickling invalid NEWOBJ_EX opcode with the C implementation raises now UnpicklingError instead of crashing.

- bpo-39017 [https://bugs.python.org/issue?@action=redirect&bpo=39017]: Avoid infinite loop when reading specially crafted TAR files using the tarfile module (**CVE 2019-20907** [https://www.cve.org/CVERecord?id=CVE-2019-20907]).

- bpo-41273 [https://bugs.python.org/issue?@action=redirect&bpo=41273]: Speed up any transport using `_ProactorReadPipeTransport` by calling

`recv_into` instead of `recv`, thus not creating a new buffer for each `recv` call in the transport's read loop.

- bpo-41235 [https://bugs.python.org/issue?@action=redirect&bpo=41235]: Fix the error handling in **`ssl.SSLContext.load_dh_params()`**.

- bpo-41207 [https://bugs.python.org/issue?@action=redirect&bpo=41207]: In distutils.spawn, restore expectation that DistutilsExecError is raised when the command is not found.

- bpo-29727 [https://bugs.python.org/issue?@action=redirect&bpo=29727]: Register **`array.array`** as a **`MutableSequence`**. Patch by Pablo Galindo.

- bpo-39168 [https://bugs.python.org/issue?@action=redirect&bpo=39168]: Remove the `__new__` method of **`typing.Generic`**.

- bpo-41194 [https://bugs.python.org/issue?@action=redirect&bpo=41194]: Fix a crash in the `_ast` module: it can no longer be loaded more than once. It now uses a global state rather than a module state.

- bpo-41195 [https://bugs.python.org/issue?@action=redirect&bpo=41195]: Add read-only ssl.SSLContext.security_level attribute to retrieve the context's security level.

- bpo-41193 [https://bugs.python.org/issue?@action=redirect&bpo=41193]: The `write_history()` atexit function of the readline completer now ignores any **`OSError`** to ignore error if the filesystem is read-only, instead of only ignoring **`FileNotFoundError`** and **`PermissionError`**.

- bpo-41182 [https://bugs.python.org/issue?@action=redirect&bpo=41182]: selector: use DefaultSelector based upon implementation

- bpo-41161 [https://bugs.python.org/issue?@action=redirect&bpo=41161]: The decimal module now requires libmpdec-2.5.0. Users of –with-system-libmpdec should update their system library.

- bpo-40874 [https://bugs.python.org/issue?@action=redirect&bpo=40874]: The decimal module now requires libmpdec-2.5.0.

- bpo-41138 [https://bugs.python.org/issue?@action=redirect&bpo=41138]: Fixed the `trace` module CLI for Python source files with non-UTF-8 encoding.

- bpo-31082 [https://bugs.python.org/issue?@action=redirect&bpo=31082]: Use the term “iterable” in the docstring for `functools.reduce()`.

- bpo-40521 [https://bugs.python.org/issue?@action=redirect&bpo=40521]: Remove freelist from collections.deque().

- bpo-31938 [https://bugs.python.org/issue?@action=redirect&bpo=31938]: Fix default-value signatures of several functions in the `select` module - by Anthony Sottile.

- bpo-41068 [https://bugs.python.org/issue?@action=redirect&bpo=41068]: Fixed reading files with non-ASCII names from ZIP archive directly after writing them.

- bpo-41058 [https://bugs.python.org/issue?@action=redirect&bpo=41058]: `pdb.find_function()` now correctly determines the source file encoding.

- bpo-41056 [https://bugs.python.org/issue?@action=redirect&bpo=41056]: Invalid file descriptor values are now prevented from being passed to os.fpathconf. (discovered by Coverity)

- bpo-41056 [https://bugs.python.org/issue?@action=redirect&bpo=41056]: Fix a NULL pointer dereference within the ssl module during a MemoryError in the keylog callback. (discovered by Coverity)

- bpo-41056 [https://bugs.python.org/issue?@action=redirect&bpo=41056]: Fixed an instance where a MemoryError within the zoneinfo module might not be reported or not reported at its source. (found by Coverity)

- bpo-41048 [https://bugs.python.org/issue?@action=redirect&bpo=41048]: `mimetypes.read_mime_types()` function reads the rule file using UTF-8 encoding, not the locale encoding. Patch by Srinivas Reddy Thatiparthy.

- bpo-41043 [https://bugs.python.org/issue?@action=redirect&bpo=41043]: Fixed the use of **`glob()`** in the stdlib: literal part of the path is now always correctly escaped.

- bpo-41025 [https://bugs.python.org/issue?@action=redirect&bpo=41025]: Fixed an issue preventing the C implementation of **`zoneinfo.ZoneInfo`** from being subclassed.

- bpo-35018 [https://bugs.python.org/issue?@action=redirect&bpo=35018]: Add the **`xml.sax.handler.LexicalHandler`** class that is present in other SAX XML implementations.

- bpo-41002 [https://bugs.python.org/issue?@action=redirect&bpo=41002]: Improve performance of HTTPResponse.read with a given amount. Patch by Bruce Merry.

- bpo-40448 [https://bugs.python.org/issue?@action=redirect&bpo=40448]: **`ensurepip`** now disables the use of `pip` cache when installing the bundled versions of `pip` and `setuptools`. Patch by Krzysztof Konopko.

- bpo-40967 [https://bugs.python.org/issue?@action=redirect&bpo=40967]: Removed **`asyncio.Task.current_task()`** and **`asyncio.Task.all_tasks()`**. Patch contributed by Rémi Lapeyre.

- bpo-40924 [https://bugs.python.org/issue?@action=redirect&bpo=40924]: Ensure `importlib.resources.path` returns an extant path for the SourceFileLoader's resource reader. Avoids the regression identified in master while a long-term solution is devised.

- bpo-40955 [https://bugs.python.org/issue?@action=redirect&bpo=40955]: Fix a minor memory leak in **`subprocess`** module when extra_groups was specified.

- bpo-40855 [https://bugs.python.org/issue?@action=redirect&bpo=40855]: The standard deviation and variance functions in the statistics module were ignoring their mu and xbar arguments.

- bpo-40939 [https://bugs.python.org/issue?@action=redirect&bpo=40939]: Use the new PEG parser when generating the stdlib **`keyword`** module.

- bpo-23427 [https://bugs.python.org/issue?@action=redirect&bpo=23427]: Add `sys.orig_argv` attribute: the list of the original command line arguments passed to the Python executable.

- bpo-33689 [https://bugs.python.org/issue?@action=redirect&bpo=33689]: Ignore empty or whitespace-only lines in .pth files. This matches the documentated behavior. Before, empty lines caused the site-packages dir to appear multiple times in sys.path. By Ido Michael, contributors Malcolm Smith and Tal Einat.

- bpo-40884 [https://bugs.python.org/issue?@action=redirect&bpo=40884]: Added a `defaults` parameter to `logging.Formatter`, to allow specifying default values for custom fields. Patch by Asaf Alon and Bar Harel.

- bpo-40876 [https://bugs.python.org/issue?@action=redirect&bpo=40876]: Clarify error message in the `csv` module.

- bpo-39791 [https://bugs.python.org/issue?@action=redirect&bpo=39791]: Refresh importlib.metadata from importlib_metadata 1.6.1.

- bpo-40807 [https://bugs.python.org/issue?@action=redirect&bpo=40807]: Stop codeop._maybe_compile, used by code.InteractiveInterpreter (and IDLE). from emitting each warning three times.

- bpo-32604 [https://bugs.python.org/issue?@action=redirect&bpo=32604]: Fix reference leak in the `select` module when the module is imported in a subinterpreter.

- bpo-39791 [https://bugs.python.org/issue?@action=redirect&bpo=39791]: Built-in loaders (SourceFileLoader and ZipImporter) now supply `TraversableResources` implementations for `ResourceReader`, and the fallback function has been removed.

- bpo-39314 [https://bugs.python.org/issue?@action=redirect&bpo=39314]: `rlcompleter.Completer` and the standard Python shell now close the parenthesis for functions that take no arguments. Patch contributed by Rémi Lapeyre.

- bpo-17005 [https://bugs.python.org/issue?@action=redirect&bpo=17005]: The topological sort functionality that was introduced initially in the **functools** module has been moved to a new **graphlib** module to better accommodate the new tools and keep the original scope of the **functools** module. Patch by Pablo Galindo

- bpo-40834 [https://bugs.python.org/issue?@action=redirect&bpo=40834]: Fix truncate when sending str object with_xxsubinterpreters.channel_send.

- bpo-40755 [https://bugs.python.org/issue?@action=redirect&bpo=40755]: Add rich comparisons to collections.Counter().

- bpo-26407 [https://bugs.python.org/issue?@action=redirect&bpo=26407]: Unexpected errors in calling the `__iter__` method are no longer masked by `TypeError` in **csv.reader()**, **csv.writer.writerow()** and **csv.writer.writerows()**.

- bpo-39384 [https://bugs.python.org/issue?@action=redirect&bpo=39384]: Fixed email.contentmanager to allow set_content() to set a null string.

- bpo-40744 [https://bugs.python.org/issue?@action=redirect&bpo=40744]: The **sqlite3** module uses SQLite API functions that require SQLite v3.7.3 or higher. This patch removes support for older SQLite versions, and explicitly requires SQLite 3.7.3 both at build, compile and runtime. Patch by Sergey Fedoseev and Erlend E. Aasland.

- bpo-40777 [https://bugs.python.org/issue?@action=redirect&bpo=40777]: Initialize PyDateTime_IsoCalendarDateType.tp_base at run-time to avoid errors on some compilers.

- bpo-38488 [https://bugs.python.org/issue?@action=redirect&bpo=38488]: Update ensurepip to install pip 20.1.1 and setuptools 47.1.0.

- bpo-40792 [https://bugs.python.org/issue?@action=redirect&bpo=40792]: The result of **operator.index()** now always has exact type **int**. Previously, the result could have been an instance of a subclass of `int`.

- bpo-40767 [https://bugs.python.org/issue?@action=redirect&bpo=40767]: **webbrowser** now properly finds the default browser in pure Wayland systems by checking the WAYLAND_DISPLAY environment variable. Patch contributed by Jérémy Attali.

- bpo-40791 [https://bugs.python.org/issue?@action=redirect&bpo=40791]: **hashlib.compare_digest()** uses OpenSSL's `CRYPTO_memcmp()` function when OpenSSL is available.

- bpo-40795 [https://bugs.python.org/issue?@action=redirect&bpo=40795]: **ctypes** module: If ctypes fails to convert the result of a callback or if a ctypes callback function raises an exception, sys.unraisablehook is now called with an exception set. Previously, the error was logged into stderr by **PyErr_Print()**.

- bpo-16995 [https://bugs.python.org/issue?@action=redirect&bpo=16995]: Add **base64.b32hexencode()** and **base64.b32hexdecode()** to support the Base32 Encoding with Extended Hex Alphabet.

- bpo-30008 [https://bugs.python.org/issue?@action=redirect&bpo=30008]: Fix **ssl** code to be compatible with OpenSSL 1.1.x builds that use `no-deprecated` and `--api=1.1.0`.

- bpo-30064 [https://bugs.python.org/issue?@action=redirect&bpo=30064]: Fix asyncio `loop.sock_*` race condition issue

- bpo-40759 [https://bugs.python.org/issue?@action=redirect&bpo=40759]: Deprecate the **symbol** module.

- bpo-40756 [https://bugs.python.org/issue?@action=redirect&bpo=40756]: The second argument (extra) of `LoggerAdapter.__init__` now defaults to None.

- bpo-37129 [https://bugs.python.org/issue?@action=redirect&bpo=37129]: Add a new **os.RWF_APPEND** flag for **os.pwritev()**.

- bpo-40737 [https://bugs.python.org/issue?@action=redirect&bpo=40737]: Fix possible reference leak for **sqlite3** initialization.

- bpo-40726 [https://bugs.python.org/issue?@action=redirect&bpo=40726]: Handle cases where the `end_lineno` is `None` on **`ast.increment_lineno()`**.

- bpo-40698 [https://bugs.python.org/issue?@action=redirect&bpo=40698]: `distutils` upload creates SHA2-256 and Blake2b-256 digests. MD5 digests is skipped if platform blocks MD5.

- bpo-40695 [https://bugs.python.org/issue?@action=redirect&bpo=40695]: **`hashlib`** no longer falls back to builtin hash implementations when OpenSSL provides a hash digest and the algorithm is blocked by security policy.

- bpo-9216 [https://bugs.python.org/issue?@action=redirect&bpo=9216]: **`hashlib.new()`** passed `usedforsecurity` to OpenSSL EVP constructor `_hashlib.new()`. test_hashlib and test_smtplib handle strict security policy better.

- bpo-40614 [https://bugs.python.org/issue?@action=redirect&bpo=40614]: **`ast.parse()`** will not parse self documenting expressions in f-strings when passed `feature_version` is less than `(3, 8)`.

- bpo-40626 [https://bugs.python.org/issue?@action=redirect&bpo=40626]: Add h5 file extension as MIME Type application/x-hdf5, as per HDF Group recommendation for HDF5 formatted data files. Patch contributed by Mark Schwab.

- bpo-25920 [https://bugs.python.org/issue?@action=redirect&bpo=25920]: On macOS, when building Python for macOS 10.4 and older, which wasn't the case for python.org macOS installer, **`socket.getaddrinfo()`** no longer uses an internal lock to prevent race conditions when calling `getaddrinfo()` which is thread-safe since macOS 10.5. Python 3.9 requires macOS 10.6 or newer. The internal lock caused random hang on fork when another thread was calling **`socket.getaddrinfo()`**. The lock was also used on FreeBSD older than 5.3, OpenBSD older than 201311 and NetBSD older than 4.

- bpo-40671 [https://bugs.python.org/issue?@action=redirect&bpo=40671]: Prepare `_hashlib` for **PEP 489** [https://peps.python.org/pep-0489/] and use **`PyModule_AddType()`**.

- bpo-32309 [https://bugs.python.org/issue?@action=redirect&bpo=32309]: Added a new coroutine **`asyncio.to_thread()`**. It is mainly used for running IO-bound functions in a separate thread to avoid blocking the event loop, and essentially works as a high-level version of **`run_in_executor()`** that can directly take keyword arguments.

- bpo-36543 [https://bugs.python.org/issue?@action=redirect&bpo=36543]: Restored the deprecated **`xml.etree.cElementTree`** module.

- bpo-40611 [https://bugs.python.org/issue?@action=redirect&bpo=40611]: **`MAP_POPULATE`** constant has now been added to the list of exported **`mmap`** module flags.

- bpo-39881 [https://bugs.python.org/issue?@action=redirect&bpo=39881]: PEP 554 for use in the test suite. (Patch By Joannah Nanjekye)

- bpo-13097 [https://bugs.python.org/issue?@action=redirect&bpo=13097]: `ctypes` now raises an `ArgumentError` when a callback is invoked with more than 1024 arguments.

- bpo-39385 [https://bugs.python.org/issue?@action=redirect&bpo=39385]: A new test assertion context-manager, **`unittest.assertNoLogs()`** will ensure a given block of code emits no log messages using the logging module. Contributed by Kit Yan Choi.

- bpo-23082 [https://bugs.python.org/issue?@action=redirect&bpo=23082]: Updated the error message and docs of PurePath.relative_to() to better reflect the function behaviour.

- bpo-40318 [https://bugs.python.org/issue?@action=redirect&bpo=40318]: Use SQLite3 trace v2 API, if it is available.

- bpo-40105 [https://bugs.python.org/issue?@action=redirect&bpo=40105]: ZipFile truncates files to avoid corruption when a shorter comment is provided in append (“a”) mode. Patch by Jan Mazur.

- bpo-40084 [https://bugs.python.org/issue?@action=redirect&bpo=40084]: Fix `Enum.__dir__`: dir(Enum.member) now includes attributes as well as

methods.

- bpo-31122 [https://bugs.python.org/issue?@action=redirect&bpo=31122]: ssl.wrap_socket() now raises ssl.SSLEOFError rather than OSError when peer closes connection during TLS negotiation

- bpo-39728 [https://bugs.python.org/issue?@action=redirect&bpo=39728]: fix default `_missing_` so a duplicate `ValueError` is not set as the `__context__` of the original `ValueError`.

- bpo-39244 [https://bugs.python.org/issue?@action=redirect&bpo=39244]: Fixed **`multiprocessing.context.get_all_start_methods`** to properly return the default method first on macOS.

- bpo-39040 [https://bugs.python.org/issue?@action=redirect&bpo=39040]: Fix parsing of invalid mime headers parameters by collapsing whitespace between encoded words in a bare-quote-string.

- bpo-38731 [https://bugs.python.org/issue?@action=redirect&bpo=38731]: Add `--quiet` option to command-line interface of **`py_compile`**. Patch by Gregory Schevchenko.

- bpo-35714 [https://bugs.python.org/issue?@action=redirect&bpo=35714]: **`struct.error`** is now raised if there is a null character in a **`struct`** format string.

- bpo-38144 [https://bugs.python.org/issue?@action=redirect&bpo=38144]: Added the *root_dir* and *dir_fd* parameters in **`glob.glob()`**.

- bpo-26543 [https://bugs.python.org/issue?@action=redirect&bpo=26543]: Fix **`IMAP4.noop()`** when debug mode is enabled (ex: `imaplib.Debug = 3`).

- bpo-12178 [https://bugs.python.org/issue?@action=redirect&bpo=12178]: **`csv.writer()`** now correctly escapes *escapechar* when input contains *escapechar*. Patch by Catalin Iacob, Berker Peksag, and Itay Elbirt.

- bpo-36290 [https://bugs.python.org/issue?@action=redirect&bpo=36290]: AST nodes are now raising **`TypeError`** on conflicting keyword arguments. Patch

contributed by Rémi Lapeyre.

- bpo-33944 [https://bugs.python.org/issue?@action=redirect&bpo=33944]: Added site.py site-packages tracing in verbose mode.

- bpo-35078 [https://bugs.python.org/issue?@action=redirect&bpo=35078]: Refactor formatweekday, formatmonthname methods in LocaleHTMLCalendar and LocaleTextCalendar classes in calendar module to call the base class methods.This enables customizable CSS classes for LocaleHTMLCalendar. Patch by Srinivas Reddy Thatiparthy

- bpo-29620 [https://bugs.python.org/issue?@action=redirect&bpo=29620]: **`assertWarns()`** no longer raises a `RuntimeException` when accessing a module's `__warningregistry__` causes importation of a new module, or when a new module is imported in another thread. Patch by Kernc.

- bpo-31844 [https://bugs.python.org/issue?@action=redirect&bpo=31844]: Remove `ParserBase.error()` method from the private and undocumented `_markupbase` module. **`html.parser.HTMLParser`** is the only subclass of `ParserBase` and its `error()` implementation was deprecated in Python 3.4 and removed in Python 3.5.

- bpo-34226 [https://bugs.python.org/issue?@action=redirect&bpo=34226]: Fix `cgi.parse_multipart` without content_length. Patch by Roger Duran

- bpo-33660 [https://bugs.python.org/issue?@action=redirect&bpo=33660]: Fix pathlib.PosixPath to resolve a relative path located on the root directory properly.

- bpo-28557 [https://bugs.python.org/issue?@action=redirect&bpo=28557]: Improve the error message for a misbehaving `rawio.readinto`

- bpo-26680 [https://bugs.python.org/issue?@action=redirect&bpo=26680]: The d.is_integer() method is added to the Decimal type, for compatibility with other number types.

- bpo-26680 [https://bugs.python.org/issue?@action=redirect&bpo=26680]: The x.is_integer() method is incorporated into the abstract types of the numeric tower, Real, Rational and Integral, with appropriate default implementations.

Documentação

- bpo-41428 [https://bugs.python.org/issue?@action=redirect&bpo=41428]: Add documentation for **PEP 604** [https://peps.python.org/pep-0604/] (Allow writing union types as `X | Y`).
- bpo-41774 [https://bugs.python.org/issue?@action=redirect&bpo=41774]: In Programming FAQ "Sequences (Tuples/Lists)" section, add "How do you remove multiple items from a list".
- bpo-35293 [https://bugs.python.org/issue?@action=redirect&bpo=35293]: Fix RemovedInSphinx40Warning when building the documentation. Patch by Donghee Na.
- bpo-37149 [https://bugs.python.org/issue?@action=redirect&bpo=37149]: Change Shipman tkinter doc link from archive.org to TkDocs. (The doc has been removed from the NMT server.) The new link responds much faster and includes a short explanatory note.
- bpo-41726 [https://bugs.python.org/issue?@action=redirect&bpo=41726]: Update the refcounts info of `PyType_FromModuleAndSpec`.
- bpo-41624 [https://bugs.python.org/issue?@action=redirect&bpo=41624]: Fix the signature of **`typing.Coroutine`**.
- bpo-40204 [https://bugs.python.org/issue?@action=redirect&bpo=40204]: Enable Sphinx 3.2 `c_allow_pre_v3` option and disable `c_warn_on_allowed_pre_v3` option to make the documentation compatible with Sphinx 2 and Sphinx 3.
- bpo-41045 [https://bugs.python.org/issue?@action=redirect&bpo=41045]: Add documentation for debug feature of f-strings.
- bpo-41314 [https://bugs.python.org/issue?@action=redirect&bpo=41314]: Changed the release when `from __future__ import annotations` becomes the default from `4.0` to `3.10` (following a change in PEP 563).
- bpo-40979 [https://bugs.python.org/issue?@action=redirect&bpo=40979]: Refactored typing.rst, arranging more than 70 classes, functions, and decorators into new sub-sections.

- bpo-40552 [https://bugs.python.org/issue?@action=redirect&bpo=40552]: Fix in tutorial section 4.2. Code snippet is now correct.
- bpo-39883 [https://bugs.python.org/issue?@action=redirect&bpo=39883]: Make code, examples, and recipes in the Python documentation be licensed under the more permissive BSD0 license in addition to the existing Python 2.0 license.
- bpo-37703 [https://bugs.python.org/issue?@action=redirect&bpo=37703]: Updated Documentation to comprehensively elaborate on the behaviour of gather.cancel()

Testes

- bpo-41939 [https://bugs.python.org/issue?@action=redirect&bpo=41939]: Fix test_site.test_license_exists_at_url(): call `urllib.request.urlcleanup()` to reset the global `urllib.request._opener`. Patch by Victor Stinner.
- bpo-41731 [https://bugs.python.org/issue?@action=redirect&bpo=41731]: Make test_cmd_line_script pass with option '-vv'.
- bpo-41602 [https://bugs.python.org/issue?@action=redirect&bpo=41602]: Add tests for SIGINT handling in the runpy module.
- bpo-41521 [https://bugs.python.org/issue?@action=redirect&bpo=41521]: **`test.support`**: Rename `blacklist` parameter of **`check_all()`** to `not_exported`.
- bpo-41477 [https://bugs.python.org/issue?@action=redirect&bpo=41477]: Make ctypes optional in test_genericalias.
- bpo-41085 [https://bugs.python.org/issue?@action=redirect&bpo=41085]: Fix integer overflow in the **`array.array.index()`** method on 64-bit Windows for index larger than `2**31`.
- bpo-41069 [https://bugs.python.org/issue?@action=redirect&bpo=41069]: **`test.support.TESTFN`** and the current directory for tests when run via `test.regrtest` contain now non-ascii characters if possible.
- bpo-38377 [https://bugs.python.org/issue?@action=redirect&bpo=38377]: On Linux, skip tests using multiprocessing if the current user cannot create a file in `/dev/shm/` directory. Add the **`skip_if_broken_multiprocessing_synchronize()`** function to the **`test.support`** module.

- bpo-41009 [https://bugs.python.org/issue?@action=redirect&bpo=41009]: Fix use of `support.require_{linux|mac|freebsd}_version()` decorators as class decorator.
- bpo-41003 [https://bugs.python.org/issue?@action=redirect&bpo=41003]: Fix `test_copyreg` when `numpy` is installed: `test.pickletester` now saves/restores warnings filters when importing `numpy`, to ignore filters installed by `numpy`.
- bpo-40964 [https://bugs.python.org/issue?@action=redirect&bpo=40964]: Disable remote **`imaplib`** tests, host cyrus.andrew.cmu.edu is blocking incoming connections.
- bpo-40927 [https://bugs.python.org/issue?@action=redirect&bpo=40927]: Fix test_binhex when run twice: it now uses import_fresh_module() to ensure that it raises DeprecationWarning each time.
- bpo-17258 [https://bugs.python.org/issue?@action=redirect&bpo=17258]: Skip some **`multiprocessing`** tests when MD5 hash digest is blocked.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Increase LOOPBACK_TIMEOUT to 10 for VxWorks RTOS.
- bpo-38169 [https://bugs.python.org/issue?@action=redirect&bpo=38169]: Increase code coverage for SharedMemory and ShareableList
- bpo-34401 [https://bugs.python.org/issue?@action=redirect&bpo=34401]: Make test_gdb properly run on HP-UX. Patch by Michael Osipov.

Construção

- bpo-38249 [https://bugs.python.org/issue?@action=redirect&bpo=38249]: Update **`Py_UNREACHABLE`** to use __builtin_unreachable() if only the compiler is able to use it. Patch by Donghee Na.
- bpo-41617 [https://bugs.python.org/issue?@action=redirect&bpo=41617]: Fix `pycore_bitutils.h` header file to support old clang versions: `__builtin_bswap16()` is not available in LLVM clang 3.0.
- bpo-40204 [https://bugs.python.org/issue?@action=redirect&bpo=40204]: Pin Sphinx version to 2.3.1 in `Doc/Makefile`.
- bpo-36020 [https://bugs.python.org/issue?@action=redirect&bpo=36020]: The C99 functions **`snprintf()`** and **`vsnprintf()`** are now required to build Python.
- bpo-40684 [https://bugs.python.org/issue?@action=redirect&bpo=40684]: `make install` now uses the `PLATLIBDIR` variable for the destination `lib-`

`dynload/` directory when `./configure --with-platlibdir` is used.

- bpo-40683 [https://bugs.python.org/issue?@action=redirect&bpo=40683]: Fixed an issue where the **`zoneinfo`** module and its tests were not included when Python is installed with `make`.

Windows

- bpo-41744 [https://bugs.python.org/issue?@action=redirect&bpo=41744]: Fixes automatic import of props file when using the Nuget package.
- bpo-41627 [https://bugs.python.org/issue?@action=redirect&bpo=41627]: The user site directory for 32-bit now includes a `-32` suffix to distinguish it from the 64-bit interpreter's directory.
- bpo-41526 [https://bugs.python.org/issue?@action=redirect&bpo=41526]: Fixed layout of final page of the installer by removing the special thanks to Mark Hammond (with his permission).
- bpo-41492 [https://bugs.python.org/issue?@action=redirect&bpo=41492]: Fixes the description that appears in UAC prompts.
- bpo-40948 [https://bugs.python.org/issue?@action=redirect&bpo=40948]: Improve post-install message to direct people to the "py" command.
- bpo-41412 [https://bugs.python.org/issue?@action=redirect&bpo=41412]: The installer will now fail to install on Windows 7 and Windows 8. Further, the UCRT dependency is now always downloaded on demand.
- bpo-40741 [https://bugs.python.org/issue?@action=redirect&bpo=40741]: Update Windows release to include SQLite 3.32.3.
- bpo-41142 [https://bugs.python.org/issue?@action=redirect&bpo=41142]: **`msilib`** now supports creating CAB files with non-ASCII file path and adding files with non-ASCII file path to them.
- bpo-41074 [https://bugs.python.org/issue?@action=redirect&bpo=41074]: Fixed support of non-ASCII names in functions **`msilib.OpenDatabase()`** and **`msilib.init_database()`** and non-ASCII SQL in method **`msilib.Database.OpenView()`**.
- bpo-41039 [https://bugs.python.org/issue?@action=redirect&bpo=41039]: Stable ABI redirection DLL (python3.dll) now uses `#pragma comment(linker)` for re-exporting.
- bpo-40164 [https://bugs.python.org/issue?@action=redirect&bpo=40164]: Updates Windows OpenSSL to 1.1.1g

- bpo-39631 [https://bugs.python.org/issue?@action=redirect&bpo=39631]: Changes the registered MIME type for `.py` files on Windows to `text/x-python` instead of `text/plain`.
- bpo-40677 [https://bugs.python.org/issue?@action=redirect&bpo=40677]: Manually define IO_REPARSE_TAG_APPEXECLINK in case some old Windows SDK doesn't have it.
- bpo-37556 [https://bugs.python.org/issue?@action=redirect&bpo=37556]: Extend py.exe help to mention overrides via venv, shebang, environmental variables & ini files.

macOS

- bpo-41557 [https://bugs.python.org/issue?@action=redirect&bpo=41557]: Update macOS installer to use SQLite 3.33.0.
- bpo-39580 [https://bugs.python.org/issue?@action=redirect&bpo=39580]: Avoid opening Finder window if running installer from the command line. Patch contributed by Rick Heil.
- bpo-41100 [https://bugs.python.org/issue?@action=redirect&bpo=41100]: Fix configure error when building on macOS 11. Note that the current Python release was released shortly after the first developer preview of macOS 11 (Big Sur); there are other known issues with building and running on the developer preview. Big Sur is expected to be fully supported in a future bugfix release of Python 3.8.x and with 3.9.0.
- bpo-40741 [https://bugs.python.org/issue?@action=redirect&bpo=40741]: Update macOS installer to use SQLite 3.32.3.
- bpo-41005 [https://bugs.python.org/issue?@action=redirect&bpo=41005]: fixed an XDG settings issue not allowing macos to open browser in webbrowser.py
- bpo-40741 [https://bugs.python.org/issue?@action=redirect&bpo=40741]: Update macOS installer to use SQLite 3.32.2.

IDLE

- bpo-41775 [https://bugs.python.org/issue?@action=redirect&bpo=41775]: Use 'IDLE Shell' as shell title
- bpo-35764 [https://bugs.python.org/issue?@action=redirect&bpo=35764]: Rewrite the Calltips doc section.

- bpo-40181 [https://bugs.python.org/issue?@action=redirect&bpo=40181]: In calltips, stop reminding that '/' marks the end of positional-only arguments.
- bpo-41468 [https://bugs.python.org/issue?@action=redirect&bpo=41468]: Improve IDLE run crash error message (which users should never see).
- bpo-41373 [https://bugs.python.org/issue?@action=redirect&bpo=41373]: Save files loaded with no line ending, as when blank, or different line endings, by setting its line ending to the system default. Fix regression in 3.8.4 and 3.9.0b4.
- bpo-41300 [https://bugs.python.org/issue?@action=redirect&bpo=41300]: Save files with non-ascii chars. Fix regression released in 3.9.0b4 and 3.8.4.
- bpo-37765 [https://bugs.python.org/issue?@action=redirect&bpo=37765]: Add keywords to module name completion list. Rewrite Completions section of IDLE doc.
- bpo-41152 [https://bugs.python.org/issue?@action=redirect&bpo=41152]: The encoding of `stdin`, `stdout` and `stderr` in IDLE is now always UTF-8.
- bpo-41144 [https://bugs.python.org/issue?@action=redirect&bpo=41144]: Make Open Module open a special module such as os.path.
- bpo-39885 [https://bugs.python.org/issue?@action=redirect&bpo=39885]: Make context menu Cut and Copy work again when right-clicking within a selection.
- bpo-40723 [https://bugs.python.org/issue?@action=redirect&bpo=40723]: Make test_idle pass when run after import.

API C

- bpo-41936 [https://bugs.python.org/issue?@action=redirect&bpo=41936]: Removed undocumented macros `Py_ALLOW_RECURSION` and `Py_END_ALLOW_RECURSION` and the `recursion_critical` field of the **`PyInterpreterState`** structure.
- bpo-41692 [https://bugs.python.org/issue?@action=redirect&bpo=41692]: The `PyUnicode_InternImmortal()` function is now deprecated and will be removed in Python 3.12: use **`PyUnicode_InternInPlace()`** instead. Patch by Victor Stinner.
- bpo-41842 [https://bugs.python.org/issue?@action=redirect&bpo=41842]: Add **`PyCodec_Unregister()`** function to unregister a codec search function.
- bpo-41834 [https://bugs.python.org/issue?@action=redirect&bpo=41834]: Remove the `_Py_CheckRecursionLimit` variable: it has been replaced by

ceval.recursion_limit of the **PyInterpreterState** structure. Patch by Victor Stinner.

- bpo-41689 [https://bugs.python.org/issue?@action=redirect&bpo=41689]: Types created with **PyType_FromSpec()** now make any signature in their tp_doc slot accessible from __text_signature__.
- bpo-41524 [https://bugs.python.org/issue?@action=redirect&bpo=41524]: Fix bug in **PyOS_mystrnicmp()** and **PyOS_mystricmp()** that incremented pointers beyond the end of a string.
- bpo-41324 [https://bugs.python.org/issue?@action=redirect&bpo=41324]: Add a minimal decimal capsule API. The API supports fast conversions between Decimals up to 38 digits and their triple representation as a C struct.
- bpo-30155 [https://bugs.python.org/issue?@action=redirect&bpo=30155]: Add **PyDateTime_DATE_GET_TZINFO()** and **PyDateTime_TIME_GET_TZINFO()** macros for accessing the tzinfo attributes of **datetime.datetime** and **datetime.time** objects.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Revert **PyType_HasFeature()** change: it reads again directly the **PyTypeObject.tp_flags** member when the limited C API is not used, rather than always calling **PyType_GetFlags()** which hides implementation details.
- bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123]: Remove PyUnicode_AsUnicodeCopy.
- bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123]: Removed PyLong_FromUnicode().
- bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123]: Removed PyUnicode_GetMax().
- bpo-41123 [https://bugs.python.org/issue?@action=redirect&bpo=41123]: Removed Py_UNICODE_str* functions manipulating Py_UNICODE* strings.
- bpo-41103 [https://bugs.python.org/issue?@action=redirect&bpo=41103]: PyObject_AsCharBuffer(), PyObject_AsReadBuffer(), PyObject_CheckReadBuffer(), and PyObject_AsWriteBuffer() are removed. Please migrate to new buffer protocol; **PyObject_GetBuffer()** and **PyBuffer_Release()**.
- bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346]: Raises DeprecationWarning for PyUnicode_FromUnicode(NULL, size) and PyUnicode_FromStringAndSize(NULL, size) with size > 0.

- bpo-36346 [https://bugs.python.org/issue?@action=redirect&bpo=36346]: Mark `Py_UNICODE_COPY`, `Py_UNICODE_FILL`, `PyUnicode_WSTR_LENGTH`, `PyUnicode_FromUnicode`, `PyUnicode_AsUnicode`, and `PyUnicode_AsUnicodeAndSize` as deprecated in C. Remove `Py_UNICODE_MATCH` which was deprecated and broken since Python 3.3.
- bpo-40989 [https://bugs.python.org/issue?@action=redirect&bpo=40989]: The **`PyObject_INIT()`** and **`PyObject_INIT_VAR()`** macros become aliases to, respectively, **`PyObject_Init()`** and **`PyObject_InitVar()`** functions.
- bpo-36020 [https://bugs.python.org/issue?@action=redirect&bpo=36020]: On Windows, `#include "pyerrors.h"` no longer defines `snprintf` and `vsnprintf` macros.
- bpo-40943 [https://bugs.python.org/issue?@action=redirect&bpo=40943]: The `PY_SSIZE_T_CLEAN` macro must now be defined to use **`PyArg_ParseTuple()`** and **`Py_BuildValue()`** formats which use `#`: `es#`, `et#`, `s#`, `u#`, `y#`, `z#`, `U#` and `Z#`. See Parsing arguments and building values and the **PEP 353** [https://peps.python.org/pep-0353/].
- bpo-40910 [https://bugs.python.org/issue?@action=redirect&bpo=40910]: Export explicitly the **`Py_GetArgcArgv()`** function to the C API and document the function. Previously, it was exported implicitly which no longer works since Python is built with `-fvisibility=hidden`.
- bpo-40724 [https://bugs.python.org/issue?@action=redirect&bpo=40724]: Allow defining buffer slots in type specs.
- bpo-40679 [https://bugs.python.org/issue?@action=redirect&bpo=40679]: Fix a `_PyEval_EvalCode()` crash if *qualname* argument is NULL.
- bpo-40839 [https://bugs.python.org/issue?@action=redirect&bpo=40839]: Calling **`PyDict_GetItem()`** without GIL held had been allowed for historical reason. It is no longer allowed.
- bpo-40826 [https://bugs.python.org/issue?@action=redirect&bpo=40826]: **`PyOS_InterruptOccurred()`** now fails with a fatal error if it is called with the GIL released.
- bpo-40792 [https://bugs.python.org/issue?@action=redirect&bpo=40792]: The result of **`PyNumber_Index()`** now always has exact type **`int`**. Previously, the result could have been an instance of a subclass of `int`.
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Convert **`Py_REFCNT()`** and **`Py_SIZE()`** macros to static inline functions. They

cannot be used as l-value anymore: use `Py_SET_REFCNT()` and `Py_SET_SIZE()` to set an object reference count and size. This change is backward incompatible on purpose, to prepare the C API for an opaque `PyObject` structure.

- bpo-40703 [https://bugs.python.org/issue?@action=redirect&bpo=40703]: The PyType_FromSpec*() functions no longer overwrite the type's "__module__" attribute if it is set via "Py_tp_members" or "Py_tp_getset".
- bpo-39583 [https://bugs.python.org/issue?@action=redirect&bpo=39583]: Remove superfluous "extern C" declarations from `Include/cpython/*.h`.

Python 3.9.0 beta 1

Data de lançamento: 2020-05-19

Segurança

- bpo-40501 [https://bugs.python.org/issue?@action=redirect&bpo=40501]: `uuid` no longer uses `ctypes` to load `libuuid` or `rpcrt4.dll` at runtime.

Núcleo e embutidos

- bpo-40663 [https://bugs.python.org/issue?@action=redirect&bpo=40663]: Correctly generate annotations where parentheses are omitted but required (e.g: `Type[(str, int, *other))]`.
- bpo-40596 [https://bugs.python.org/issue?@action=redirect&bpo=40596]: Fixed `str.isidentifier()` for non-canonicalized strings containing non-BMP characters on Windows.
- bpo-40593 [https://bugs.python.org/issue?@action=redirect&bpo=40593]: Improved syntax errors for invalid characters in source code.
- bpo-40585 [https://bugs.python.org/issue?@action=redirect&bpo=40585]: Fixed a bug when using `codeop.compile_command()` that was causing exceptions to be swallowed with the new parser. Patch by Pablo Galindo
- bpo-40566 [https://bugs.python.org/issue?@action=redirect&bpo=40566]: Apply **PEP 573** [https://peps.python.org/pep-0573/] to `abc`.

- bpo-40502 [https://bugs.python.org/issue?@action=redirect&bpo=40502]: Initialize `n->n_col_offset`. (Patch by Joannah Nanjekye)
- bpo-40527 [https://bugs.python.org/issue?@action=redirect&bpo=40527]: Fix command line argument parsing: no longer write errors multiple times into stderr.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`errno`** to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-40523 [https://bugs.python.org/issue?@action=redirect&bpo=40523]: Add pass-throughs for **`hash()`** and **`reversed()`** to **`weakref.proxy`** objects. Patch by Pablo Galindo.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port **`syslog`** to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-40246 [https://bugs.python.org/issue?@action=redirect&bpo=40246]: Reporting a specialised error message for invalid string prefixes, which was introduced in bpo-40246 [https://bugs.python.org/issue?@action=redirect&bpo=40246], is being reverted due to backwards compatibility concerns for strings that immediately follow a reserved keyword without whitespace between them. Constructs like `bg="#d00" if clear else"#fca"` were failing to parse, which is not an acceptable breakage on such short notice.
- bpo-40417 [https://bugs.python.org/issue?@action=redirect&bpo=40417]: Fix imp module deprecation warning when PyImport_ReloadModule is called. Patch by Robert Rouhani.
- bpo-40408 [https://bugs.python.org/issue?@action=redirect&bpo=40408]: Fixed support of nested type variables in GenericAlias (e.g. `list[list[T]]`).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _stat module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-29587 [https://bugs.python.org/issue?@action=redirect&bpo=29587]: Enable implicit exception chaining when calling **`generator.throw()`**.
- bpo-40328 [https://bugs.python.org/issue?@action=redirect&bpo=40328]: Add tools for generating mappings headers for CJKCodecs.
- bpo-40228 [https://bugs.python.org/issue?@action=redirect&bpo=40228]: Setting frame.f_lineno is now robust w.r.t. changes in the source-to-bytecode compiler

- bpo-38880 [https://bugs.python.org/issue?@action=redirect&bpo=38880]: Added the ability to list interpreters associated with channel ends in the internal subinterpreters module.
- bpo-37986 [https://bugs.python.org/issue?@action=redirect&bpo=37986]: Improve performance of `PyLong_FromDouble()` for values that fit into long.

Biblioteca

- bpo-40662 [https://bugs.python.org/issue?@action=redirect&bpo=40662]: Fixed `ast.get_source_segment()` for ast nodes that have incomplete location information. Patch by Irit Katriel.
- bpo-40665 [https://bugs.python.org/issue?@action=redirect&bpo=40665]: Convert `bisect` to use Argument Clinic.
- bpo-40536 [https://bugs.python.org/issue?@action=redirect&bpo=40536]: Added the `available_timezones()` function to the `zoneinfo` module. Patch by Paul Ganssle.
- bpo-40645 [https://bugs.python.org/issue?@action=redirect&bpo=40645]: The `hmac.HMAC` exposes internal implementation details. The attributes `digest_cons`, `inner`, and `outer` are deprecated and will be removed in the future.
- bpo-40645 [https://bugs.python.org/issue?@action=redirect&bpo=40645]: The internal module `_hashlib` wraps and exposes OpenSSL's HMAC API. The new code will be used in Python 3.10 after the internal implementation details of the pure Python HMAC module are no longer part of the public API.
- bpo-40637 [https://bugs.python.org/issue?@action=redirect&bpo=40637]: Builtin hash modules can now be disabled or selectively enabled with `configure --with-builtin-hashlib-hashes=sha3,blake1` or `--without-builtin-hashlib-hashes`.
- bpo-37630 [https://bugs.python.org/issue?@action=redirect&bpo=37630]: The `hashlib` module can now use SHA3 hashes and SHAKE XOF from OpenSSL when available.
- bpo-40479 [https://bugs.python.org/issue?@action=redirect&bpo=40479]: The `hashlib` now compiles with OpenSSL 3.0.0-alpha2.
- bpo-40257 [https://bugs.python.org/issue?@action=redirect&bpo=40257]: Revert changes to `inspect.getdoc()`.

- bpo-40607 [https://bugs.python.org/issue?@action=redirect&bpo=40607]: When cancelling a task due to timeout, **`asyncio.wait_for()`** will now propagate the exception if an error happens during cancellation. Patch by Roman Skurikhin.
- bpo-40612 [https://bugs.python.org/issue?@action=redirect&bpo=40612]: Fix edge cases in SyntaxError formatting. If the offset is <= 0, no caret is printed. If the offset is > line length, the caret is printed pointing just after the last character.
- bpo-40597 [https://bugs.python.org/issue?@action=redirect&bpo=40597]: If text content lines are longer than policy.max_line_length, always use a content-encoding to make sure they are wrapped.
- bpo-40571 [https://bugs.python.org/issue?@action=redirect&bpo=40571]: Added functools.cache() as a simpler, more discoverable way to access the unbounded cache variant of lru_cache(maxsize=None).
- bpo-40503 [https://bugs.python.org/issue?@action=redirect&bpo=40503]: **PEP 615** [https://peps.python.org/pep-0615/], the **`zoneinfo`** module. Adds support for the IANA time zone database.
- bpo-40397 [https://bugs.python.org/issue?@action=redirect&bpo=40397]: Removed attributes `__args__` and `__parameters__` from special generic aliases like `typing.List` (not subscripted).
- bpo-40549 [https://bugs.python.org/issue?@action=redirect&bpo=40549]: Convert posixmodule.c ("posix" or "nt" module) to the multiphase initialization (PEP 489).
- bpo-31033 [https://bugs.python.org/issue?@action=redirect&bpo=31033]: Add a `msg` argument to **`Future.cancel()`** and **`Task.cancel()`**.
- bpo-40541 [https://bugs.python.org/issue?@action=redirect&bpo=40541]: Added an optional *counts* parameter to random.sample().
- bpo-40515 [https://bugs.python.org/issue?@action=redirect&bpo=40515]: The **`ssl`** and **`hashlib`** modules now actively check that OpenSSL is build with thread support. Python 3.7.0 made thread support mandatory and no longer works safely with a no-thread builds.
- bpo-31033 [https://bugs.python.org/issue?@action=redirect&bpo=31033]: When a **`asyncio.Task`** is cancelled, the exception traceback now chains all the way back to where the task was first interrupted.
- bpo-40504 [https://bugs.python.org/issue?@action=redirect&bpo=40504]: **`functools.lru_cache()`** objects can now be the targets of weakrefs.

- bpo-40559 [https://bugs.python.org/issue?@action=redirect&bpo=40559]: Fix possible memory leak in the C implementation of **`asyncio.Task`**.
- bpo-40480 [https://bugs.python.org/issue?@action=redirect&bpo=40480]: `fnmatch.fnmatch()` could take exponential time in the presence of multiple `*` pattern characters. This was repaired by generating more elaborate regular expressions to avoid futile backtracking.
- bpo-40495 [https://bugs.python.org/issue?@action=redirect&bpo=40495]: **`compileall`** is now able to use hardlinks to prevent duplicates in a case when `.pyc` files for different optimization levels have the same content.
- bpo-40457 [https://bugs.python.org/issue?@action=redirect&bpo=40457]: The ssl module now support OpenSSL builds without TLS 1.0 and 1.1 methods.
- bpo-40355 [https://bugs.python.org/issue?@action=redirect&bpo=40355]: Improve error reporting in **`ast.literal_eval()`** in the presence of malformed **`ast.Dict`** nodes instead of silently ignoring any non-conforming elements. Patch by Curtis Bucher.
- bpo-40465 [https://bugs.python.org/issue?@action=redirect&bpo=40465]: Deprecated the optional *random* argument to *random.shuffle()*.
- bpo-40459 [https://bugs.python.org/issue?@action=redirect&bpo=40459]: **`platform.win32_ver()`** now produces correct *ptype* strings instead of empty strings.
- bpo-39435 [https://bugs.python.org/issue?@action=redirect&bpo=39435]: The first argument of **`pickle.loads()`** is now positional-only.
- bpo-39305 [https://bugs.python.org/issue?@action=redirect&bpo=39305]: Update **`nntplib`** to merge **`nntplib.NNTP`** and **`nntplib._NNTPBase`**. Patch by Donghee Na.
- bpo-32494 [https://bugs.python.org/issue?@action=redirect&bpo=32494]: Update **`dbm.gnu`** to use gdbm_count if possible when calling **`len()`**. Patch by Donghee Na.
- bpo-40453 [https://bugs.python.org/issue?@action=redirect&bpo=40453]: Add `isolated=True` keyword-only parameter to `_xxsubinterpreters.create()`. An isolated subinterpreter cannot spawn threads, spawn a child process or call `os.fork()`.
- bpo-40286 [https://bugs.python.org/issue?@action=redirect&bpo=40286]: Remove `_random.Random.randbytes()`: the C implementation of

`randbytes()`. Implement the method in Python to ease subclassing: `randbytes()` now directly reuses `getrandbits()`.

- bpo-40394 [https://bugs.python.org/issue?@action=redirect&bpo=40394]: Added default arguments to **`difflib.SequenceMatcher.find_longest_match()`**.
- bpo-39995 [https://bugs.python.org/issue?@action=redirect&bpo=39995]: Fix a race condition in concurrent.futures._ThreadWakeup: access to _ThreadWakeup is now protected with the shutdown lock.
- bpo-30966 [https://bugs.python.org/issue?@action=redirect&bpo=30966]: `Process.shutdown(wait=True)` of **`concurrent.futures`** now closes explicitly the result queue.
- bpo-30966 [https://bugs.python.org/issue?@action=redirect&bpo=30966]: Add a new **`close()`** method to the **`SimpleQueue`** class to explicitly close the queue.
- bpo-39966 [https://bugs.python.org/issue?@action=redirect&bpo=39966]: Revert bpo-25597 [https://bugs.python.org/issue?@action=redirect&bpo=25597]. **`unittest.mock.MagicMock`** with wraps' set uses default return values for magic methods.
- bpo-39791 [https://bugs.python.org/issue?@action=redirect&bpo=39791]: Added `files()` function to importlib.resources with support for subdirectories in package data, matching backport in importlib_resources 1.5.
- bpo-40375 [https://bugs.python.org/issue?@action=redirect&bpo=40375]: **`imaplib.IMAP4.unselect()`** is added. Patch by Donghee Na.
- bpo-40389 [https://bugs.python.org/issue?@action=redirect&bpo=40389]: `repr()` now returns `typing.Optional[T]` when called for `typing.Union` of two types, one of which is `NoneType`.
- bpo-40291 [https://bugs.python.org/issue?@action=redirect&bpo=40291]: Add support for CAN_J1939 sockets (available on Linux 5.4+)
- bpo-40273 [https://bugs.python.org/issue?@action=redirect&bpo=40273]: **`types.MappingProxyType`** is now reversible.
- bpo-39075 [https://bugs.python.org/issue?@action=redirect&bpo=39075]: The repr for **`types.SimpleNamespace`** is now insertion ordered rather than alphabetical.
- bpo-40192 [https://bugs.python.org/issue?@action=redirect&bpo=40192]: On AIX, **`thread_time()`** is now implemented with `thread_cputime()` which has nanosecond resolution, rather than

`clock_gettime(CLOCK_THREAD_CPUTIME_ID)` which has a resolution of 10 milliseconds. Patch by Batuhan Taskaya.

- bpo-40025 [https://bugs.python.org/issue?@action=redirect&bpo=40025]: Raise TypeError when _generate_next_value_ is defined after members. Patch by Ethan Onstott.
- bpo-39058 [https://bugs.python.org/issue?@action=redirect&bpo=39058]: In the argparse module, the repr for Namespace() and other argument holders now displayed in the order attributes were added. Formerly, it displayed in alphabetical order even though argument order is preserved the user visible parts of the module.
- bpo-24416 [https://bugs.python.org/issue?@action=redirect&bpo=24416]: The `isocalendar()` methods of **`datetime.date`** and **`datetime.datetime`** now return a named tuple instead of a **`tuple`**.

Documentação

- bpo-34790 [https://bugs.python.org/issue?@action=redirect&bpo=34790]: Add version of removal for explicit passing of coros to `asyncio.wait()`'s documentation
- bpo-40561 [https://bugs.python.org/issue?@action=redirect&bpo=40561]: Provide docstrings for webbrowser open functions.
- bpo-40499 [https://bugs.python.org/issue?@action=redirect&bpo=40499]: Mention that **`asyncio.wait()`** requires a non-empty set of awaitables.
- bpo-39705 [https://bugs.python.org/issue?@action=redirect&bpo=39705]: Tutorial example for sorted() in the Loop Techniques section is given a better explanation. Also a new example is included to explain sorted()'s basic behavior.
- bpo-39435 [https://bugs.python.org/issue?@action=redirect&bpo=39435]: Fix an incorrect signature for **`pickle.loads()`** in the docs

Testes

- bpo-40055 [https://bugs.python.org/issue?@action=redirect&bpo=40055]: distutils.tests now saves/restores warnings filters to leave them unchanged. Importing tests imports docutils which imports pkg_resources which adds a warnings filter.

- bpo-40436 [https://bugs.python.org/issue?@action=redirect&bpo=40436]: test_gdb and test.pythoninfo now check gdb command exit code.

Construção

- bpo-40653 [https://bugs.python.org/issue?@action=redirect&bpo=40653]: Move _dirnameW out of HAVE_SYMLINK to fix a potential compiling issue.
- bpo-40514 [https://bugs.python.org/issue?@action=redirect&bpo=40514]: Add `--with-experimental-isolated-subinterpreters` build option to `configure`: better isolate subinterpreters, experimental build mode.

Windows

- bpo-40650 [https://bugs.python.org/issue?@action=redirect&bpo=40650]: Include winsock2.h in pytime.c for timeval.
- bpo-40458 [https://bugs.python.org/issue?@action=redirect&bpo=40458]: Increase reserved stack space to prevent overflow crash on Windows.
- bpo-39148 [https://bugs.python.org/issue?@action=redirect&bpo=39148]: Add IPv6 support to **`asyncio`** datagram endpoints in ProactorEventLoop. Change the raised exception for unknown address families to ValueError as it's not coming from Windows API.

macOS

- bpo-34956 [https://bugs.python.org/issue?@action=redirect&bpo=34956]: When building Python on macOS from source, `_tkinter` now links with non-system Tcl and Tk frameworks if they are installed in `/Library/Frameworks`, as had been the case on older releases of macOS. If a macOS SDK is explicitly configured, by using `--enable-universalsdk=` or `-isysroot`, only the SDK itself is searched. The default behavior can still be overridden with `--with-tcltk-includes` and `--with-tcltk-libs`.
- bpo-35569 [https://bugs.python.org/issue?@action=redirect&bpo=35569]: Expose RFC 3542 IPv6 socket options.

Ferramentas/Demos

- bpo-40479 [https://bugs.python.org/issue?@action=redirect&bpo=40479]: Update multissltest helper to test with latest OpenSSL 1.0.2, 1.1.0, 1.1.1, and 3.0.0-alpha.
- bpo-40431 [https://bugs.python.org/issue?@action=redirect&bpo=40431]: Fix a syntax typo in `turtledemo` that now raises a `SyntaxError`.
- bpo-40163 [https://bugs.python.org/issue?@action=redirect&bpo=40163]: Fix multissltest tool. OpenSSL has changed download URL for old releases. The multissltest tool now tries to download from current and old download URLs.

API C

- bpo-39465 [https://bugs.python.org/issue?@action=redirect&bpo=39465]: Remove the `_PyUnicode_ClearStaticStrings()` function from the C API.
- bpo-38787 [https://bugs.python.org/issue?@action=redirect&bpo=38787]: Add PyCFunction_CheckExact() macro for exact type checks now that we allow subtypes of PyCFunction, as well as PyCMethod_CheckExact() and PyCMethod_Check() for the new PyCMethod subtype.
- bpo-40545 [https://bugs.python.org/issue?@action=redirect&bpo=40545]: Declare `_PyErr_GetTopmostException()` with `PyAPI_FUNC()` to properly export the function in the C API. The function remains private (`_Py`) prefix.
- bpo-40412 [https://bugs.python.org/issue?@action=redirect&bpo=40412]: Nullify inittab_copy during finalization, preventing future interpreter initializations in an embedded situation from crashing. Patch by Gregory Szorc.
- bpo-40429 [https://bugs.python.org/issue?@action=redirect&bpo=40429]: The **`PyThreadState_GetFrame()`** function now returns a strong reference to the frame.
- bpo-40428 [https://bugs.python.org/issue?@action=redirect&bpo=40428]: Remove the following functions from the C API. Call **`PyGC_Collect()`** explicitly to free all free lists.
  - `PyAsyncGen_ClearFreeLists()`
  - `PyContext_ClearFreeList()`
  - `PyDict_ClearFreeList()`
  - `PyFloat_ClearFreeList()`
  - `PyFrame_ClearFreeList()`

    - `PyList_ClearFreeList()`
    - `PySet_ClearFreeList()`
    - `PyTuple_ClearFreeList()`
- bpo-40421 [https://bugs.python.org/issue?@action=redirect&bpo=40421]: New `PyFrame_GetBack()` function: get the frame next outer frame.
- bpo-40421 [https://bugs.python.org/issue?@action=redirect&bpo=40421]: New `PyFrame_GetCode()` function: return a borrowed reference to the frame code.
- bpo-40217 [https://bugs.python.org/issue?@action=redirect&bpo=40217]: Ensure that instances of types created with `PyType_FromSpecWithBases()` will visit its class object when traversing references in the garbage collector (implemented as an extension of the provided `tp_traverse`). Patch by Pablo Galindo.
- bpo-38787 [https://bugs.python.org/issue?@action=redirect&bpo=38787]: Module C state is now accessible from C-defined heap type methods (**PEP 573** [https://peps.python.org/pep-0573/]). Patch by Marcel Plch and Petr Viktorin.

Python 3.9.0 alpha 6

Data de lançamento: 2020-04-27

Segurança

- bpo-40121 [https://bugs.python.org/issue?@action=redirect&bpo=40121]: Fixes audit events raised on creating a new socket.
- bpo-39073 [https://bugs.python.org/issue?@action=redirect&bpo=39073]: Disallow CR or LF in email.headerregistry.Address arguments to guard against header injection attacks.
- bpo-39503 [https://bugs.python.org/issue?@action=redirect&bpo=39503]: **CVE 2020-8492** [https://www.cve.org/CVERecord?id=CVE-2020-8492]: The `AbstractBasicAuthHandler` class of the `urllib.request` module uses an inefficient regular expression which can be exploited by an attacker to cause a denial of service. Fix the regex to prevent the catastrophic backtracking. Vulnerability reported by Ben Caller and Matt Schwager.

Núcleo e embutidos

- bpo-40313 [https://bugs.python.org/issue?@action=redirect&bpo=40313]: Improve the performance of bytes.hex().
- bpo-40334 [https://bugs.python.org/issue?@action=redirect&bpo=40334]: Switch to a new parser, based on PEG. For more details see PEP 617. To temporarily switch back to the old parser, use `-X oldparser` or `PYTHONOLDPARSER=1`. In Python 3.10 we will remove the old parser completely, including the `parser` module (already deprecated) and anything that depends on it.
- bpo-40267 [https://bugs.python.org/issue?@action=redirect&bpo=40267]: Fix the tokenizer to display the correct error message, when there is a `SyntaxError` on the last input character and no newline follows. It used to be `unexpected EOF while parsing`, while it should be `invalid syntax`.
- bpo-39522 [https://bugs.python.org/issue?@action=redirect&bpo=39522]: Correctly unparse explicit `u` prefix for strings when postponed evaluation for annotations activated. Patch by Batuhan Taskaya.
- bpo-40246 [https://bugs.python.org/issue?@action=redirect&bpo=40246]: Report a specialized error message, `invalid string prefix`, when the tokenizer encounters a string with an invalid prefix.
- bpo-40082 [https://bugs.python.org/issue?@action=redirect&bpo=40082]: Fix the signal handler: it now always uses the main interpreter, rather than trying to get the current Python thread state.
- bpo-37388 [https://bugs.python.org/issue?@action=redirect&bpo=37388]: str.encode() and str.decode() no longer check the encoding and errors in development mode or in debug mode during Python finalization. The codecs machinery can no longer work on very late calls to str.encode() and str.decode().
- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Fix possible refleaks in **`_json`**, memo of PyScannerObject should be traversed.
- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Speed up calls to `dict()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention.
- bpo-40141 [https://bugs.python.org/issue?@action=redirect&bpo=40141]: Add column and line information to `ast.keyword` nodes. Patch by Pablo

Galindo.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `resource` to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port `math` to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _uuid module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-40077 [https://bugs.python.org/issue?@action=redirect&bpo=40077]: Convert json module to use `PyType_FromSpec()`.
- bpo-40067 [https://bugs.python.org/issue?@action=redirect&bpo=40067]: Improve the error message for multiple star expressions in an assignment. Patch by Furkan Onder
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _functools module to multiphase initialization (PEP 489). Patch by Paulo Henrique Silva.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port operator module to multiphase initialization (PEP 489). Patch by Paulo Henrique Silva.
- bpo-20526 [https://bugs.python.org/issue?@action=redirect&bpo=20526]: Fix `PyThreadState_Clear()`. `PyThreadState.frame` is a borrowed reference, not a strong reference: `PyThreadState_Clear()` must not call `Py_CLEAR(tstate->frame)`.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port time module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]). Patch by Paulo Henrique Silva.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _weakref extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-40020 [https://bugs.python.org/issue?@action=redirect&bpo=40020]: Fix a leak and subsequent crash in parsetok.c caused by realloc misuse on a rare codepath.
- bpo-39939 [https://bugs.python.org/issue?@action=redirect&bpo=39939]: Added str.removeprefix and str.removesuffix methods and corresponding bytes, bytearray, and collections.UserString methods to remove affixes

from a string if present. See **PEP 616** [https://peps.python.org/pep-0616/] for a full description. Patch by Dennis Sweeney.

- bpo-39481 [https://bugs.python.org/issue?@action=redirect&bpo=39481]: Implement PEP 585. This supports list[int], tuple[str, …] etc.
- bpo-32894 [https://bugs.python.org/issue?@action=redirect&bpo=32894]: Support unparsing of infinity numbers in postponed annotations. Patch by Batuhan Taşkaya.
- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Speed up calls to `list()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Mark Shannon.

Biblioteca

- bpo-40398 [https://bugs.python.org/issue?@action=redirect&bpo=40398]: **`typing.get_args()`** now always returns an empty tuple for special generic aliases.

- bpo-40396 [https://bugs.python.org/issue?@action=redirect&bpo=40396]: Functions **`typing.get_origin()`**, **`typing.get_args()`** and **`typing.get_type_hints()`** support now generic aliases like `list[int]`.

- bpo-38061 [https://bugs.python.org/issue?@action=redirect&bpo=38061]: Optimize the **`subprocess`** module on FreeBSD using `closefrom()`. A single `close(fd)` syscall is cheap, but when `sysconf(_SC_OPEN_MAX)` is high, the loop calling `close(fd)` on each file descriptor can take several milliseconds.

  The workaround on FreeBSD to improve performance was to load and mount the fdescfs kernel module, but this is not enabled by default.

  Initial patch by Ed Maste (emaste), Conrad Meyer (cem), Kyle Evans (kevans) and Kubilay Kocak (koobs): https://bugs.freebsd.org/bugzilla/show_bug.cgi?id=242274

- bpo-38061 [https://bugs.python.org/issue?@action=redirect&bpo=38061]: On FreeBSD, `os.closerange(fd_low, fd_high)` now calls

`closefrom(fd_low)` if *fd_high* is greater than or equal to `sysconf(_SC_OPEN_MAX)`.

Initial patch by Ed Maste (emaste), Conrad Meyer (cem), Kyle Evans (kevans) and Kubilay Kocak (koobs): https://bugs.freebsd.org/bugzilla/show_bug.cgi?id=242274

- bpo-40360 [https://bugs.python.org/issue?@action=redirect&bpo=40360]: The **`lib2to3`** module is pending deprecation due to **PEP 617** [https://peps.python.org/pep-0617/].

- bpo-40138 [https://bugs.python.org/issue?@action=redirect&bpo=40138]: Fix the Windows implementation of **`os.waitpid()`** for exit code larger than `INT_MAX >> 8`. The exit status is now interpreted as an unsigned number.

- bpo-39942 [https://bugs.python.org/issue?@action=redirect&bpo=39942]: Set "__main__" as the default module name when "__name__" is missing in **`typing.TypeVar`**. Patch by Weipeng Hong.

- bpo-40275 [https://bugs.python.org/issue?@action=redirect&bpo=40275]: The **`logging`** package is now imported lazily in **`unittest`** only when the **`assertLogs()`** assertion is used.

- bpo-40275 [https://bugs.python.org/issue?@action=redirect&bpo=40275]: The **`asyncio`** package is now imported lazily in **`unittest`** only when the **`IsolatedAsyncioTestCase`** class is used.

- bpo-40330 [https://bugs.python.org/issue?@action=redirect&bpo=40330]: In **`ShareableList.__setitem__()`**, check the size of a new string item after encoding it to utf-8, not before.

- bpo-40148 [https://bugs.python.org/issue?@action=redirect&bpo=40148]: Added **`pathlib.Path.with_stem()`** to create a new Path with the stem replaced.

- bpo-40325 [https://bugs.python.org/issue?@action=redirect&bpo=40325]: Deprecated support for set objects in random.sample().

- bpo-40257 [https://bugs.python.org/issue?@action=redirect&bpo=40257]: Improved help for the `typing` module. Docstrings are now shown for all special forms and special generic aliases (like `Union` and `List`). Using `help()` with generic alias like `List[int]` will show the help for the correspondent concrete type (`list` in this case).

- bpo-40257 [https://bugs.python.org/issue?@action=redirect&bpo=40257]: `inspect.getdoc()` no longer returns docstring inherited from the type of the object or from parent class if it is a class if it is not defined in the object itself. In `pydoc` the documentation string is now shown not only for class, function, method etc, but for any object that has its own `__doc__` attribute.

- bpo-40287 [https://bugs.python.org/issue?@action=redirect&bpo=40287]: Fixed `SpooledTemporaryFile.seek()` to return the position.

- bpo-40290 [https://bugs.python.org/issue?@action=redirect&bpo=40290]: Added zscore() to statistics.NormalDist().

- bpo-40282 [https://bugs.python.org/issue?@action=redirect&bpo=40282]: Allow `random.getrandbits(0)` to succeed and to return 0.

- bpo-40286 [https://bugs.python.org/issue?@action=redirect&bpo=40286]: Add `random.randbytes()` function and `random.Random.randbytes()` method to generate random bytes.

- bpo-40277 [https://bugs.python.org/issue?@action=redirect&bpo=40277]: `collections.namedtuple()` now provides a human-readable repr for its field accessors.

- bpo-40270 [https://bugs.python.org/issue?@action=redirect&bpo=40270]: The included copy of sqlite3 on Windows is now compiled with the json extension. This allows the use of functions such as `json_object`.

- bpo-29255 [https://bugs.python.org/issue?@action=redirect&bpo=29255]: Wait in `KqueueSelector.select` when no fds are registered

- bpo-40260 [https://bugs.python.org/issue?@action=redirect&bpo=40260]: Ensure **`modulefinder`** uses **`io.open_code()`** and respects coding comments.

- bpo-40234 [https://bugs.python.org/issue?@action=redirect&bpo=40234]: Allow again to spawn daemon threads in subinterpreters (revert change which denied them).

- bpo-39207 [https://bugs.python.org/issue?@action=redirect&bpo=39207]: Workers in **`ProcessPoolExecutor`** are now spawned on demand, only when there are no available idle workers to reuse. This optimizes startup overhead and reduces the amount of lost CPU time to idle workers. Patch by Kyle Stanley.

- bpo-40091 [https://bugs.python.org/issue?@action=redirect&bpo=40091]: Fix a hang at fork in the logging module: the new private _at_fork_reinit() method is now used to reinitialize locks at fork in the child process.

- bpo-40149 [https://bugs.python.org/issue?@action=redirect&bpo=40149]: Implement traverse and clear slots in _abc._abc_data type.

- bpo-40208 [https://bugs.python.org/issue?@action=redirect&bpo=40208]: Remove deprecated **`symtable.SymbolTable.has_exec()`**.

- bpo-40196 [https://bugs.python.org/issue?@action=redirect&bpo=40196]: Fix a bug in the **`symtable`** module that was causing incorrectly report global variables as local. Patch by Pablo Galindo.

- bpo-40190 [https://bugs.python.org/issue?@action=redirect&bpo=40190]: Add support for `_SC_AIX_REALMEM` to **`posix.sysconf()`**.

- bpo-40182 [https://bugs.python.org/issue?@action=redirect&bpo=40182]: Removed the `_field_types` attribute of the **`typing.NamedTuple`** class.

- bpo-36517 [https://bugs.python.org/issue?@action=redirect&bpo=36517]: Multiple inheritance with **`typing.NamedTuple`** now raises an error instead of silently ignoring other types.

- bpo-40126 [https://bugs.python.org/issue?@action=redirect&bpo=40126]: Fixed reverting multiple patches in unittest.mock. Patcher's `__exit__()` is now never called if its `__enter__()` is failed. Returning true from `__exit__()` silences now the exception.

- bpo-40094 [https://bugs.python.org/issue?@action=redirect&bpo=40094]: CGIHTTPRequestHandler of http.server now logs the CGI script exit code, rather than the CGI script exit status of os.waitpid(). For example, if the script is killed by signal 11, it now logs: "CGI script exit code -11."

- bpo-40108 [https://bugs.python.org/issue?@action=redirect&bpo=40108]: Improve the error message when triying to import a module using **`runpy`** and incorrectly using the ".py" extension at the end of the module name. Patch by Pablo Galindo.

- bpo-40094 [https://bugs.python.org/issue?@action=redirect&bpo=40094]: Add **`os.waitstatus_to_exitcode()`** function: convert a wait status to an exit code.

- bpo-40089 [https://bugs.python.org/issue?@action=redirect&bpo=40089]: Fix threading._after_fork(): if fork was not called by a thread spawned by threading.Thread, threading._after_fork() now creates a _MainThread instance for _main_thread, instead of a _DummyThread instance.

- bpo-40089 [https://bugs.python.org/issue?@action=redirect&bpo=40089]: Add a private `_at_fork_reinit()` method to **`_thread.Lock`**, **`_thread.RLock`**, **`threading.RLock`** and **`threading.Condition`** classes: reinitialize the lock at fork in the child process, reset the lock to the unlocked state. Rename also the private `_reset_internal_locks()` method of **`threading.Event`** to `_at_fork_reinit()`.

- bpo-25780 [https://bugs.python.org/issue?@action=redirect&bpo=25780]: Expose **`CAN_RAW_JOIN_FILTERS`** in the **`socket`** module.

- bpo-39503 [https://bugs.python.org/issue?@action=redirect&bpo=39503]: **`AbstractBasicAuthHandler`** of **`urllib.request`** now parses all

WWW-Authenticate HTTP headers and accepts multiple challenges per header: use the realm of the first Basic challenge.

- bpo-39812 [https://bugs.python.org/issue?@action=redirect&bpo=39812]: Removed daemon threads from `concurrent.futures` by adding an internal `threading._register_atexit()`, which calls registered functions prior to joining all non-daemon threads. This allows for compatibility with subinterpreters, which don't support daemon threads.

- bpo-40050 [https://bugs.python.org/issue?@action=redirect&bpo=40050]: Fix `importlib._bootstrap_external`: avoid creating a new `winreg` builtin module if it's already available in `sys.modules`, and remove redundant imports.

- bpo-40014 [https://bugs.python.org/issue?@action=redirect&bpo=40014]: Fix `os.getgrouplist()`: if `getgrouplist()` function fails because the group list is too small, retry with a larger group list. On failure, the glibc implementation of `getgrouplist()` sets `ngroups` to the total number of groups. For other implementations, double the group list size.

- bpo-40017 [https://bugs.python.org/issue?@action=redirect&bpo=40017]: Add `time.CLOCK_TAI` constant if the operating system support it.

- bpo-40016 [https://bugs.python.org/issue?@action=redirect&bpo=40016]: In re docstring, clarify the relationship between inline and argument compile flags.

- bpo-39953 [https://bugs.python.org/issue?@action=redirect&bpo=39953]: Update internal table of OpenSSL error codes in the `ssl` module.

- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: Added **PEP 584** [https://peps.python.org/pep-0584/] operators to `weakref.WeakValueDictionary`.

- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: Added **PEP 584** [https://peps.python.org/pep-0584/] operators to `weakref.WeakKeyDictionary`.

- bpo-38891 [https://bugs.python.org/issue?@action=redirect&bpo=38891]: Fix linear runtime behaviour of the `__getitem__` and `__setitem__` methods in **`multiprocessing.shared_memory.ShareableList`**. This avoids quadratic performance when iterating a `ShareableList`. Patch by Thomas Krennwallner.

- bpo-39682 [https://bugs.python.org/issue?@action=redirect&bpo=39682]: Remove undocumented support for *closing* a `pathlib.Path` object via its context manager. The context manager magic methods remain, but they are now a no-op, making `Path` objects immutable.

- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: Added **PEP 584** [https://peps.python.org/pep-0584/] operators (`|` and `|=`) to **`collections.ChainMap`**.

- bpo-39011 [https://bugs.python.org/issue?@action=redirect&bpo=39011]: Normalization of line endings in ElementTree attributes was removed, as line endings which were replaced by entity numbers should be preserved in original form.

- bpo-38410 [https://bugs.python.org/issue?@action=redirect&bpo=38410]: Properly handle **`sys.audit()`** failures in **`sys.set_asyncgen_hooks()`**.

- bpo-36541 [https://bugs.python.org/issue?@action=redirect&bpo=36541]: lib2to3 now recognizes named assignment expressions (the walrus operator, `:=`)

- bpo-35967 [https://bugs.python.org/issue?@action=redirect&bpo=35967]: In platform, delay the invocation of ‘uname -p’ until the processor attribute is requested.

- bpo-35113 [https://bugs.python.org/issue?@action=redirect&bpo=35113]: **`inspect.getsource()`** now returns correct source code for inner class with same name as module level class. Decorators are also returned as part of source of the class. Patch by Karthikeyan Singaravelan.

- bpo-33262 [https://bugs.python.org/issue?@action=redirect&bpo=33262]: Deprecate passing None as an argument for **`shlex.split()`**’s `s` parameter. Patch

by Zackery Spytz.

- bpo-31758 [https://bugs.python.org/issue?@action=redirect&bpo=31758]: Prevent crashes when using an uninitialized `_elementtree.XMLParser` object. Patch by Oren Milman.

Documentação

- bpo-27635 [https://bugs.python.org/issue?@action=redirect&bpo=27635]: The pickle documentation incorrectly claimed that `__new__` isn't called by default when unpickling.
- bpo-39879 [https://bugs.python.org/issue?@action=redirect&bpo=39879]: Updated Modelo de dados docs to include **`dict()`** insertion order preservation. Patch by Furkan Onder and Samy Lahfa.
- bpo-38387 [https://bugs.python.org/issue?@action=redirect&bpo=38387]: Document **`PyDoc_STRVAR`** macro in the C-API reference.
- bpo-13743 [https://bugs.python.org/issue?@action=redirect&bpo=13743]: Some methods within xml.dom.minidom.Element class are now better documented.

Testes

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Set expected default encoding in test_c_locale_coercion.py for VxWorks RTOS.
- bpo-40162 [https://bugs.python.org/issue?@action=redirect&bpo=40162]: Update Travis CI configuration to OpenSSL 1.1.1f.
- bpo-40146 [https://bugs.python.org/issue?@action=redirect&bpo=40146]: Update OpenSSL to 1.1.1f in Azure Pipelines.
- bpo-40094 [https://bugs.python.org/issue?@action=redirect&bpo=40094]: Add **`test.support.wait_process()`** function.
- bpo-40003 [https://bugs.python.org/issue?@action=redirect&bpo=40003]: `test.bisect_cmd` now copies Python command line options like `-O` or `-W`. Moreover, emit a warning if `test.bisect_cmd` is used with `-W`/`--verbose2` option.
- bpo-39380 [https://bugs.python.org/issue?@action=redirect&bpo=39380]: Add the encoding in **`ftplib.FTP`** and **`ftplib.FTP_TLS`** to the constructor as

keyword-only and change the default from `latin-1` to `utf-8` to follow **RFC 2640** [https://datatracker.ietf.org/doc/html/rfc2640.html].
- bpo-39793 [https://bugs.python.org/issue?@action=redirect&bpo=39793]: Use the same domain when testing `make_msgid`. Patch by Batuhan Taskaya.
- bpo-1812 [https://bugs.python.org/issue?@action=redirect&bpo=1812]: Fix newline handling in doctest.testfile when loading from a package whose loader has a get_data method. Patch by Peter Donis.

Construção

- bpo-38360 [https://bugs.python.org/issue?@action=redirect&bpo=38360]: Support single-argument form of macOS -isysroot flag.
- bpo-40158 [https://bugs.python.org/issue?@action=redirect&bpo=40158]: Fix CPython MSBuild Properties in NuGet Package (build/native/python.props)
- bpo-38527 [https://bugs.python.org/issue?@action=redirect&bpo=38527]: Fix configure check on Solaris for “float word ordering”: sometimes, the correct “grep” command was not being used. Patch by Arnon Yaari.

Windows

- bpo-40164 [https://bugs.python.org/issue?@action=redirect&bpo=40164]: Updates Windows to OpenSSL 1.1.1f
- bpo-8901 [https://bugs.python.org/issue?@action=redirect&bpo=8901]: Ignore the Windows registry when the `-E` option is used.

macOS

- bpo-38329 [https://bugs.python.org/issue?@action=redirect&bpo=38329]: python.org macOS installers now update the Current version symlink of /Library/Frameworks/Python.framework/Versions for 3.9 installs. Previously, Current was only updated for Python 2.x installs. This should make it easier to embed Python 3 into other macOS applications.
- bpo-40164 [https://bugs.python.org/issue?@action=redirect&bpo=40164]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.1g.

IDLE

- bpo-38439 [https://bugs.python.org/issue?@action=redirect&bpo=38439]: Add a 256×256 pixel IDLE icon to support more modern environments. Created by Andrew Clover. Delete the unused macOS idle.icns icon file.
- bpo-38689 [https://bugs.python.org/issue?@action=redirect&bpo=38689]: IDLE will no longer freeze when inspect.signature fails when fetching a calltip.

Ferramentas/Demos

- bpo-40385 [https://bugs.python.org/issue?@action=redirect&bpo=40385]: Removed the checkpyc.py tool. Please see compileall without force mode as a potential alternative.
- bpo-40179 [https://bugs.python.org/issue?@action=redirect&bpo=40179]: Fixed translation of `#elif` in Argument Clinic.
- bpo-40094 [https://bugs.python.org/issue?@action=redirect&bpo=40094]: Fix `which.py` script exit code: it now uses **`os.waitstatus_to_exitcode()`** to convert **`os.system()`** exit status into an exit code.

API C

- bpo-40241 [https://bugs.python.org/issue?@action=redirect&bpo=40241]: Move the **`PyGC_Head`** structure to the internal C API.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Convert **`PyObject_IS_GC()`** macro to a function to hide implementation details.
- bpo-40241 [https://bugs.python.org/issue?@action=redirect&bpo=40241]: Add the functions **`PyObject_GC_IsTracked()`** and **`PyObject_GC_IsFinalized()`** to the public API to allow to query if Python objects are being currently tracked or have been already finalized by the garbage collector respectively. Patch by Pablo Galindo.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: The **`PyObject_NEW()`** macro becomes an alias to the **`PyObject_New()`** macro, and the **`PyObject_NEW_VAR()`** macro becomes an alias to the **`PyObject_NewVar()`** macro, to hide implementation details. They no longer access directly the **`PyTypeObject.tp_basicsize`** member.

- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: `PyType_HasFeature()` now always calls `PyType_GetFlags()` to hide implementation details. Previously, it accessed directly the `PyTypeObject.tp_flags` member when the limited C API was not used.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Convert the `PyObject_GET_WEAKREFS_LISTPTR()` macro to a function to hide implementation details: the macro accessed directly to the `PyTypeObject.tp_weaklistoffset` member.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Convert `PyObject_CheckBuffer()` macro to a function to hide implementation details: the macro accessed directly the `PyTypeObject.tp_as_buffer` member.
- bpo-40170 [https://bugs.python.org/issue?@action=redirect&bpo=40170]: Always declare `PyIndex_Check()` as an opaque function to hide implementation details: remove `PyIndex_Check()` macro. The macro accessed directly the `PyTypeObject.tp_as_number` member.
- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Add `PyThreadState_GetID()` function: get the unique identifier of a Python thread state.

Python 3.9.0 alpha 5

Data de lançamento: 2020-03-23

Segurança

- bpo-38576 [https://bugs.python.org/issue?@action=redirect&bpo=38576]: Disallow control characters in hostnames in http.client, addressing **CVE 2019-18348** [https://www.cve.org/CVERecord?id=CVE-2019-18348]. Such potentially malicious header injection URLs now cause a InvalidURL to be raised.

Núcleo e embutidos

- bpo-40010 [https://bugs.python.org/issue?@action=redirect&bpo=40010]: Optimize pending calls in multithreaded applications. If a thread different than

the main thread schedules a pending call (`Py_AddPendingCall()`), the bytecode evaluation loop is no longer interrupted at each bytecode instruction to check for pending calls which cannot be executed. Only the main thread can execute pending calls.

Previously, the bytecode evaluation loop was interrupted at each instruction until the main thread executes pending calls.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _weakref extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _collections module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-40010 [https://bugs.python.org/issue?@action=redirect&bpo=40010]: Optimize signal handling in multithreaded applications. If a thread different than the main thread gets a signal, the bytecode evaluation loop is no longer interrupted at each bytecode instruction to check for pending signals which cannot be handled. Only the main thread of the main interpreter can handle signals.

  Previously, the bytecode evaluation loop was interrupted at each instruction until the main thread handles signals.

- bpo-39984 [https://bugs.python.org/issue?@action=redirect&bpo=39984]: If `Py_AddPendingCall()` is called in a subinterpreter, the function is now scheduled to be called from the subinterpreter, rather than being called from the main interpreter. Each subinterpreter now has its own list of scheduled calls.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _heapq module to multiphase initialization.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port itertools module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Speed up calls to `frozenset()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

- bpo-39984 [https://bugs.python.org/issue?@action=redirect&bpo=39984]: subinterpreters: Move `_PyRuntimeState.ceval.tracing_possible` to `PyInterpreterState.ceval.tracing_possible`: each interpreter now has its own variable.

- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Speed up calls to `set()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _statistics module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-39968 [https://bugs.python.org/issue?@action=redirect&bpo=39968]: Use inline function to replace extension modules' get_module_state macros.

- bpo-39965 [https://bugs.python.org/issue?@action=redirect&bpo=39965]: Correctly raise `SyntaxError` if *await* is used inside non-async functions and `PyCF_ALLOW_TOP_LEVEL_AWAIT` is set (like in the asyncio REPL). Patch by Pablo Galindo.

- bpo-39562 [https://bugs.python.org/issue?@action=redirect&bpo=39562]: Allow executing asynchronous comprehensions on the top level when the `PyCF_ALLOW_TOP_LEVEL_AWAIT` flag is given. Patch by Batuhan Taskaya.

- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Speed up calls to `tuple()` by using the **PEP 590** [https://peps.python.org/pep-0590/] `vectorcall` calling convention. Patch by Donghee Na.

- bpo-38373 [https://bugs.python.org/issue?@action=redirect&bpo=38373]: Changed list overallocation strategy. It no longer overallocates if the new size is closer to overallocated size than to the old size and adds padding.

- bpo-39926 [https://bugs.python.org/issue?@action=redirect&bpo=39926]: Update Unicode database to Unicode version 13.0.0.

- bpo-19466 [https://bugs.python.org/issue?@action=redirect&bpo=19466]: Clear the frames of daemon threads earlier during the Python shutdown to call objects destructors. So “unclosed file” resource warnings are now emitted for daemon threads in a more reliable way.

- bpo-38894 [https://bugs.python.org/issue?@action=redirect&bpo=38894]: Fix a bug that was causing incomplete results when calling `pathlib.Path.glob` in the presence of symlinks that point to files where the user does not have read access. Patch by Pablo Galindo and Matt Wozniski.

- bpo-39877 [https://bugs.python.org/issue?@action=redirect&bpo=39877]: Fix **`PyEval_RestoreThread()`** random crash at exit with daemon threads. It now accesses the `_PyRuntime` variable directly instead of using `tstate->interp->runtime`, since `tstate` can be a dangling pointer after **`Py_Finalize()`** has been called. Moreover, the daemon thread now exits before trying to take the GIL.

- bpo-39871 [https://bugs.python.org/issue?@action=redirect&bpo=39871]: Fix a possible **`SystemError`** in `math.{atan2,copysign,remainder}()` when the first argument cannot be converted to a **`float`**. Patch by Zackery Spytz.

- bpo-39776 [https://bugs.python.org/issue?@action=redirect&bpo=39776]: Fix race condition where threads created by PyGILState_Ensure() could get a duplicate id.

  Isso afeta os usuários de tstate->id como o mecanismo de armazenamento em cache da contextvar, que pode retornar objetos em cache inválidos sob uso intenso de threads (observado em cenários de sistemas embarcados).

- bpo-39778 [https://bugs.python.org/issue?@action=redirect&bpo=39778]: Fixed a crash due to incorrect handling of weak references in `collections.OrderedDict` classes. Patch by Pablo Galindo.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port audioop extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-39702 [https://bugs.python.org/issue?@action=redirect&bpo=39702]: Relax decorator grammar restrictions to allow any valid expression (**PEP 614** [https://peps.python.org/pep-0614/]).

- bpo-38091 [https://bugs.python.org/issue?@action=redirect&bpo=38091]: Tweak import deadlock detection code to not deadlock itself.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _locale extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-39087 [https://bugs.python.org/issue?@action=redirect&bpo=39087]: Optimize `PyUnicode_AsUTF8()` and `PyUnicode_AsUTF8AndSize()` slightly when they need to create internal UTF-8 cache.

- bpo-39520 [https://bugs.python.org/issue?@action=redirect&bpo=39520]: Fix unparsing of ext slices with no items (`foo[:,]`). Patch by Batuhan Taskaya.

- bpo-39220 [https://bugs.python.org/issue?@action=redirect&bpo=39220]: Do not optimize annotations if ‘from __future__ import annotations’ is used. Patch by Pablo Galindo.

- bpo-35712 [https://bugs.python.org/issue?@action=redirect&bpo=35712]: Using `NotImplemented` in a boolean context has been deprecated. Patch contributed by Josh Rosenberg.

- bpo-22490 [https://bugs.python.org/issue?@action=redirect&bpo=22490]: Don’t leak environment variable `__PYVENV_LAUNCHER__` into the interpreter session on macOS.

Biblioteca

- bpo-39830 [https://bugs.python.org/issue?@action=redirect&bpo=39830]: Add **`zipfile.Path`** to `__all__` in the **`zipfile`** module.
- bpo-40000 [https://bugs.python.org/issue?@action=redirect&bpo=40000]: Improved error messages for validation of `ast.Constant` nodes. Patch by Batuhan Taskaya.
- bpo-39999 [https://bugs.python.org/issue?@action=redirect&bpo=39999]: `__module__` of the AST node classes is now set to "ast" instead of "_ast". Added docstrings for dummy AST node classes and deprecated attributes.
- bpo-39991 [https://bugs.python.org/issue?@action=redirect&bpo=39991]: **`uuid.getnode()`** now skips IPv6 addresses with the same string length than a MAC address (17 characters): only use MAC addresses.
- bpo-39988 [https://bugs.python.org/issue?@action=redirect&bpo=39988]: Deprecated `ast.AugLoad` and `ast.AugStore` node classes because they are no longer used.
- bpo-39656 [https://bugs.python.org/issue?@action=redirect&bpo=39656]: Ensure `bin/python3.#` is always present in virtual environments on POSIX platforms - by Anthony Sottile.
- bpo-39969 [https://bugs.python.org/issue?@action=redirect&bpo=39969]: Deprecated `ast.Param` node class because it's no longer used. Patch by Batuhan Taskaya.
- bpo-39360 [https://bugs.python.org/issue?@action=redirect&bpo=39360]: Ensure all workers exit when finalizing a **`multiprocessing.Pool`** implicitly via the module finalization handlers of multiprocessing. This fixes a deadlock situation that can be experienced when the Pool is not properly finalized via the context manager or a call to `multiprocessing.Pool.terminate`. Patch by Batuhan Taskaya and Pablo Galindo.
- bpo-35370 [https://bugs.python.org/issue?@action=redirect&bpo=35370]: sys.settrace(), sys.setprofile() and _lsprof.Profiler.enable() now properly report **`PySys_Audit()`** error if "sys.setprofile" or "sys.settrace" audit event is denied.
- bpo-39936 [https://bugs.python.org/issue?@action=redirect&bpo=39936]: AIX: Fix _aix_support module when the subprocess is not available, when building Python from scratch. It now uses new private _bootsubprocess module, rather than having two implementations depending if subprocess is available or not. So

_aix_support.aix_platform() result is now the same if subprocess is available or not.

- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: **`collections.OrderedDict`** now implements `|` and `|=` (**PEP 584** [https://peps.python.org/pep-0584/]).
- bpo-39652 [https://bugs.python.org/issue?@action=redirect&bpo=39652]: The column name found in `sqlite3.Cursor.description` is now truncated on the first '[' only if the PARSE_COLNAMES option is set.
- bpo-39915 [https://bugs.python.org/issue?@action=redirect&bpo=39915]: Ensure **`unittest.mock.AsyncMock.await_args_list`** has call objects in the order of awaited arguments instead of using **`unittest.mock.Mock.call_args`** which has the last value of the call. Patch by Karthikeyan Singaravelan.
- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: Updated **`os.environ`** and **`os.environb`** to support **PEP 584** [https://peps.python.org/pep-0584/]'s merge (`|`) and update (`|=`) operators.
- bpo-38662 [https://bugs.python.org/issue?@action=redirect&bpo=38662]: The `ensurepip` module now invokes `pip` via the `runpy` module. Hence it is no longer tightly coupled with the internal API of the bundled `pip` version, allowing easier updates to a newer `pip` version both internally and for distributors.
- bpo-38075 [https://bugs.python.org/issue?@action=redirect&bpo=38075]: Fix the **`random.Random.seed()`** method when a **`bool`** is passed as the seed.
- bpo-39916 [https://bugs.python.org/issue?@action=redirect&bpo=39916]: More reliable use of `os.scandir()` in `Path.glob()`. It no longer emits a ResourceWarning when interrupted.
- bpo-39850 [https://bugs.python.org/issue?@action=redirect&bpo=39850]: **`multiprocessing`** now supports abstract socket addresses (if abstract sockets are supported in the running platform). When creating arbitrary addresses (like when default-constructing **`multiprocessing.connection.Listener`** objects) abstract sockets are preferred to avoid the case when the temporary-file-generated address is too large for an AF_UNIX socket address. Patch by Pablo Galindo.
- bpo-36287 [https://bugs.python.org/issue?@action=redirect&bpo=36287]: **`ast.dump()`** no longer outputs optional fields and attributes with default values. The default values for optional fields and attributes of

AST nodes are now set as class attributes (e.g. `Constant.kind` is set to `None`).

- bpo-39889 [https://bugs.python.org/issue?@action=redirect&bpo=39889]: Fixed **`ast.unparse()`** for extended slices containing a single element (e.g. `a[i:j,]`). Remove redundant tuples when index with a tuple (e.g. `a[i, j]`).
- bpo-39828 [https://bugs.python.org/issue?@action=redirect&bpo=39828]: Fix **`json.tool`** to catch **`BrokenPipeError`**. Patch by Donghee Na.
- bpo-13487 [https://bugs.python.org/issue?@action=redirect&bpo=13487]: Avoid a possible *"RuntimeError: dictionary changed size during iteration"* from **`inspect.getmodule()`** when it tried to loop through **`sys.modules`**.
- bpo-39674 [https://bugs.python.org/issue?@action=redirect&bpo=39674]: Revert "bpo-37330 [https://bugs.python.org/issue?@action=redirect&bpo=37330]: open() no longer accept 'U' in file mode". The "U" mode of open() is kept in Python 3.9 to ease transition from Python 2.7, but will be removed in Python 3.10.
- bpo-28577 [https://bugs.python.org/issue?@action=redirect&bpo=28577]: The hosts method on 32-bit prefix length IPv4Networks and 128-bit prefix IPv6Networks now returns a list containing the single Address instead of an empty list.
- bpo-39826 [https://bugs.python.org/issue?@action=redirect&bpo=39826]: Add getConnection method to logging HTTPHandler to enable custom connections.
- bpo-39763 [https://bugs.python.org/issue?@action=redirect&bpo=39763]: Reimplement `distutils.spawn.spawn` function with the **`subprocess`** module.
- bpo-39794 [https://bugs.python.org/issue?@action=redirect&bpo=39794]: Add –without-decimal-contextvar build option. This enables a thread-local rather than a coroutine local context.
- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: **`collections.defaultdict`** now implements `|` (**PEP 584** [https://peps.python.org/pep-0584/]).
- bpo-39517 [https://bugs.python.org/issue?@action=redirect&bpo=39517]: Fix runpy.run_path() when using pathlike objects
- bpo-39775 [https://bugs.python.org/issue?@action=redirect&bpo=39775]: Change `inspect.Signature.parameters` back to `collections.OrderedDict`.

This was changed to `dict` in Python 3.9.0a4.

- bpo-39678 [https://bugs.python.org/issue?@action=redirect&bpo=39678]: Refactor queue_manager in **`concurrent.futures.ProcessPoolExecutor`** to make it easier to maintain.
- bpo-39764 [https://bugs.python.org/issue?@action=redirect&bpo=39764]: Fix AttributeError when calling get_stack on a PyAsyncGenObject Task
- bpo-39769 [https://bugs.python.org/issue?@action=redirect&bpo=39769]: The **`compileall.compile_dir()`** function's *ddir* parameter and the compileall command line flag `-d` no longer write the wrong pathname to the generated pyc file for submodules beneath the root of the directory tree being compiled. This fixes a regression introduced with Python 3.5.
- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: **`types.MappingProxyType`** objects now support the merge (`|`) operator from **PEP 584** [https://peps.python.org/pep-0584/].
- bpo-38691 [https://bugs.python.org/issue?@action=redirect&bpo=38691]: The **`importlib`** module now ignores the **`PYTHONCASEOK`** environment variable when the **`-E`** or **`-I`** command line options are being used.
- bpo-39719 [https://bugs.python.org/issue?@action=redirect&bpo=39719]: Remove **`tempfile.SpooledTemporaryFile.softspace()`** as files no longer have the `softspace` attribute in Python 3. Patch by Shantanu.
- bpo-39667 [https://bugs.python.org/issue?@action=redirect&bpo=39667]: Improve pathlib.Path compatibility on zipfile.Path and correct performance degradation as found in zipp 3.0.
- bpo-39638 [https://bugs.python.org/issue?@action=redirect&bpo=39638]: Keep ASDL signatures in the docstrings for `AST` nodes. Patch by Batuhan Taskaya
- bpo-39639 [https://bugs.python.org/issue?@action=redirect&bpo=39639]: Deprecated `ast.Suite` node class because it's no longer used. Patch by Batuhan Taskaya.
- bpo-39609 [https://bugs.python.org/issue?@action=redirect&bpo=39609]: Add thread_name_prefix to default asyncio executor
- bpo-39548 [https://bugs.python.org/issue?@action=redirect&bpo=39548]: Fix handling of header in **`urllib.request.AbstractDigestAuthHandler`** when the optional `qop` parameter is not present.
- bpo-39509 [https://bugs.python.org/issue?@action=redirect&bpo=39509]: HTTP status codes `103 EARLY_HINTS` and `425 TOO_EARLY` are added to

`http.HTTPStatus`. Patch by Donghee Na.

- bpo-39507 [https://bugs.python.org/issue?@action=redirect&bpo=39507]: Adding HTTP status 418 “I’m a Teapot” to HTTPStatus in http library. Patch by Ross Rhodes.
- bpo-39495 [https://bugs.python.org/issue?@action=redirect&bpo=39495]: Remove default value from *attrs* parameter of `xml.etree.ElementTree.TreeBuilder.start()` for consistency between Python and C implementations.
- bpo-38971 [https://bugs.python.org/issue?@action=redirect&bpo=38971]: Open issue in the BPO indicated a desire to make the implementation of codecs.open() at parity with io.open(), which implements a try/except to assure file stream gets closed before an exception is raised.
- bpo-38641 [https://bugs.python.org/issue?@action=redirect&bpo=38641]: Added starred expressions support to `return` and `yield` statements for `lib2to3`. Patch by Vlad Emelianov.
- bpo-37534 [https://bugs.python.org/issue?@action=redirect&bpo=37534]: When using minidom module to generate XML documents the ability to add Standalone Document Declaration is added. All the changes are made to generate a document in compliance with Extensible Markup Language (XML) 1.0 (Fifth Edition) W3C Recommendation (available here: https://www.w3.org/TR/xml/#sec-prolog-dtd).
- bpo-34788 [https://bugs.python.org/issue?@action=redirect&bpo=34788]: Add support for scoped IPv6 addresses to `ipaddress`. Patch by Oleksandr Pavliuk.
- bpo-34822 [https://bugs.python.org/issue?@action=redirect&bpo=34822]: Simplified AST for subscription. Simple indices are now represented by their value, extended slices are represented as tuples. `ast` classes `Index` and `ExtSlice` are considered deprecated and will be removed in future Python versions. In the meantime, `Index(value)` now returns a `value` itself, `ExtSlice(slices)` returns `Tuple(slices, Load())`.

Documentação

- bpo-39868 [https://bugs.python.org/issue?@action=redirect&bpo=39868]: Updated the Language Reference for **PEP 572** [https://peps.python.org/pep-0572/].
- bpo-13790 [https://bugs.python.org/issue?@action=redirect&bpo=13790]: Change ‘string’ to ‘specification’ in format doc.

- bpo-17422 [https://bugs.python.org/issue?@action=redirect&bpo=17422]: The language reference no longer restricts default class namespaces to dicts only.
- bpo-39530 [https://bugs.python.org/issue?@action=redirect&bpo=39530]: Fix misleading documentation about mixed-type numeric comparisons.
- bpo-39718 [https://bugs.python.org/issue?@action=redirect&bpo=39718]: Update **`token`** documentation to reflect additions in Python 3.8
- bpo-39677 [https://bugs.python.org/issue?@action=redirect&bpo=39677]: Changed operand name of **MAKE_FUNCTION** from *argc* to *flags* for module **`dis`**

Testes

- bpo-40019 [https://bugs.python.org/issue?@action=redirect&bpo=40019]: test_gdb now skips tests if it detects that gdb failed to read debug information because the Python binary is optimized.
- bpo-27807 [https://bugs.python.org/issue?@action=redirect&bpo=27807]: `test_site.test_startup_imports()` is now skipped if a path of **`sys.path`** contains a `.pth` file.
- bpo-26067 [https://bugs.python.org/issue?@action=redirect&bpo=26067]: Do not fail test_shutil test_chown test when uid or gid of user cannot be resolved to a name.
- bpo-39855 [https://bugs.python.org/issue?@action=redirect&bpo=39855]: test_subprocess.test_user() now skips the test on an user name if the user name doesn't exist. For example, skip the test if the user "nobody" doesn't exist on Linux.

Construção

- bpo-39761 [https://bugs.python.org/issue?@action=redirect&bpo=39761]: Fix build with DTrace but without additional DFLAGS.
- bpo-39763 [https://bugs.python.org/issue?@action=redirect&bpo=39763]: setup.py now uses a basic implementation of the **`subprocess`** module if the **`subprocess`** module is not available: before required C extension modules are built.
- bpo-1294959 [https://bugs.python.org/issue?@action=redirect&bpo=1294959]: Add `--with-platlibdir` option to the configure script: name of the platform-

specific library directory, stored in the new `sys.platlibdir` attribute. It is used to build the path of platform-specific extension modules and the path of the standard library. It is equal to `"lib"` on most platforms. On Fedora and SuSE, it is equal to `"lib64"` on 64-bit platforms. Patch by Jan Matějek, Matěj Cepl, Charalampos Stratakis and Victor Stinner.

Windows

- bpo-39930 [https://bugs.python.org/issue?@action=redirect&bpo=39930]: Ensures the required `vcruntime140.dll` is included in install packages.
- bpo-39847 [https://bugs.python.org/issue?@action=redirect&bpo=39847]: Avoid hang when computer is hibernated whilst waiting for a mutex (for lock-related objects from `threading`) around 49-day uptime.
- bpo-38597 [https://bugs.python.org/issue?@action=redirect&bpo=38597]: `distutils` will no longer statically link `vcruntime140.dll` when a redistributable version is unavailable. All future releases of CPython will include a copy of this DLL to ensure distributed extensions can continue to load.
- bpo-38380 [https://bugs.python.org/issue?@action=redirect&bpo=38380]: Update Windows builds to use SQLite 3.31.1
- bpo-39789 [https://bugs.python.org/issue?@action=redirect&bpo=39789]: Update Windows release build machines to Visual Studio 2019 (MSVC 14.2).
- bpo-34803 [https://bugs.python.org/issue?@action=redirect&bpo=34803]: Package for nuget.org now includes repository reference and bundled icon image.

macOS

- bpo-38380 [https://bugs.python.org/issue?@action=redirect&bpo=38380]: Update macOS builds to use SQLite 3.31.1

IDLE

- bpo-27115 [https://bugs.python.org/issue?@action=redirect&bpo=27115]: For ‘Go to Line’, use a Query box subclass with IDLE standard behavior and improved error checking.
- bpo-39885 [https://bugs.python.org/issue?@action=redirect&bpo=39885]: Since clicking to get an IDLE context menu moves the cursor, any text selection should be and now is cleared.

- bpo-39852 [https://bugs.python.org/issue?@action=redirect&bpo=39852]: Edit “Go to line” now clears any selection, preventing accidental deletion. It also updates Ln and Col on the status bar.
- bpo-39781 [https://bugs.python.org/issue?@action=redirect&bpo=39781]: Selecting code context lines no longer causes a jump.

Ferramentas/Demos

- bpo-36184 [https://bugs.python.org/issue?@action=redirect&bpo=36184]: Port python-gdb.py to FreeBSD. python-gdb.py now checks for “take_gil” function name to check if a frame tries to acquire the GIL, instead of checking for “pthread_cond_timedwait” which is specific to Linux and can be a different condition than the GIL.
- bpo-38080 [https://bugs.python.org/issue?@action=redirect&bpo=38080]: Added support to fix `getproxies` in the **`lib2to3.fixes.fix_urllib`** module. Patch by José Roberto Meza Cabrera.

API C

- bpo-40024 [https://bugs.python.org/issue?@action=redirect&bpo=40024]: Add **`PyModule_AddType()`** helper function: add a type to a module. Patch by Donghee Na.

- bpo-39946 [https://bugs.python.org/issue?@action=redirect&bpo=39946]: Remove `_PyRuntime.getframe` hook and remove `_PyThreadState_GetFrame` macro which was an alias to `_PyRuntime.getframe`. They were only exposed by the internal C API. Remove also `PyThreadFrameGetter` type.

- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Add **`PyThreadState_GetFrame()`** function: get the current frame of a Python thread state.

- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Add _PyArg_NoKwnames helper function. Patch by Donghee Na.

- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Add `PyThreadState_GetInterpreter()`: get the interpreter of a Python thread state.

- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Add `PyInterpreterState_Get()` function to the limited C API.

- bpo-35370 [https://bugs.python.org/issue?@action=redirect&bpo=35370]: If `PySys_Audit()` fails in `PyEval_SetProfile()` or `PyEval_SetTrace()`, log the error as an unraisable exception.

- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Move the static inline function flavor of Py_EnterRecursiveCall() and Py_LeaveRecursiveCall() to the internal C API: they access PyThreadState attributes. The limited C API provides regular functions which hide implementation details.

- bpo-39947 [https://bugs.python.org/issue?@action=redirect&bpo=39947]: Py_TRASHCAN_BEGIN_CONDITION and Py_TRASHCAN_END macro no longer access PyThreadState attributes, but call new private _PyTrash_begin() and _PyTrash_end() functions which hide implementation details.

- bpo-39884 [https://bugs.python.org/issue?@action=redirect&bpo=39884]: `PyDescr_NewMethod()` and `PyCFunction_NewEx()` now include the method name in the SystemError “bad call flags” error message to ease debug.

- bpo-39877 [https://bugs.python.org/issue?@action=redirect&bpo=39877]: Deprecated **`PyEval_InitThreads()`** and **`PyEval_ThreadsInitialized()`**. Calling **`PyEval_InitThreads()`** now does nothing.

- bpo-38249 [https://bugs.python.org/issue?@action=redirect&bpo=38249]: `Py_UNREACHABLE` is now implemented with `__builtin_unreachable()` and analogs in release mode.

- bpo-38643 [https://bugs.python.org/issue?@action=redirect&bpo=38643]: `PyNumber_ToBase()` now raises a `SystemError` instead of crashing

when called with invalid base.

- bpo-39882 [https://bugs.python.org/issue?@action=redirect&bpo=39882]: The `Py_FatalError()` function is replaced with a macro which logs automatically the name of the current function, unless the `Py_LIMITED_API` macro is defined.

- bpo-39824 [https://bugs.python.org/issue?@action=redirect&bpo=39824]: Extension modules: `m_traverse`, `m_clear` and `m_free` functions of `PyModuleDef` are no longer called if the module state was requested but is not allocated yet. This is the case immediately after the module is created and before the module is executed (`Py_mod_exec` function). More precisely, these functions are not called if `m_size` is greater than 0 and the module state (as returned by `PyModule_GetState()`) is `NULL`.

  Módulos de extensão sem um estado de módulo (`m_size <= 0`) não são afetados.

- bpo-38913 [https://bugs.python.org/issue?@action=redirect&bpo=38913]: Fixed segfault in `Py_BuildValue()` called with a format containing "#" and undefined PY_SSIZE_T_CLEAN whwn an exception is set.

- bpo-38500 [https://bugs.python.org/issue?@action=redirect&bpo=38500]: Add a private API to get and set the frame evaluation function: add `_PyInterpreterState_GetEvalFrameFunc()` and `_PyInterpreterState_SetEvalFrameFunc()` C functions. The `_PyFrameEvalFunction` function type now takes a *tstate* parameter.

Python 3.9.0 alpha 4

Data de lançamento: 2020-02-25

Segurança

- bpo-39184 [https://bugs.python.org/issue?@action=redirect&bpo=39184]: Add audit events to functions in `fcntl`, `msvcrt`, `os`, `resource`, `shutil`, `signal` and `syslog`.

- bpo-39401 [https://bugs.python.org/issue?@action=redirect&bpo=39401]: Avoid unsafe DLL load at startup on Windows 7 and earlier.
- bpo-39184 [https://bugs.python.org/issue?@action=redirect&bpo=39184]: Add audit events to command execution functions in os and pty modules.

Núcleo e embutidos

- bpo-39382 [https://bugs.python.org/issue?@action=redirect&bpo=39382]: Fix a use-after-free in the single inheritance path of `issubclass()`, when the `__bases__` of an object has a single reference, and so does its first item. Patch by Yonatan Goldschmidt.
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Update clinic tool to use **`Py_IS_TYPE()`**. Patch by Donghee Na.
- bpo-39619 [https://bugs.python.org/issue?@action=redirect&bpo=39619]: Enable use of **`os.chroot()`** on HP-UX systems.
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Add **`Py_IS_TYPE()`** static inline function to check whether the object *o* type is *type*.
- bpo-39606 [https://bugs.python.org/issue?@action=redirect&bpo=39606]: Fix regression caused by fix for bpo-39386 [https://bugs.python.org/issue?@action=redirect&bpo=39386], that prevented calling `aclose` on an async generator that had already been closed or exhausted.
- bpo-39579 [https://bugs.python.org/issue?@action=redirect&bpo=39579]: Change the ending column offset of `Attribute` nodes constructed in `ast_for_dotted_name` to point at the end of the current node and not at the end of the last `NAME` node.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _crypt extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _contextvars extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-39510 [https://bugs.python.org/issue?@action=redirect&bpo=39510]: Fix segfault in `readinto()` method on closed BufferedReader.
- bpo-39502 [https://bugs.python.org/issue?@action=redirect&bpo=39502]: Fix **`time.localtime()`** on 64-bit AIX to support years before 1902 and after 2038. Patch by M Felt.

- bpo-39492 [https://bugs.python.org/issue?@action=redirect&bpo=39492]: Fix a reference cycle in the C Pickler that was preventing the garbage collection of deleted, pickled objects.
- bpo-39453 [https://bugs.python.org/issue?@action=redirect&bpo=39453]: Fixed a possible crash in **`list.__contains__()`** when a list is changed during comparing items. Patch by Donghee Na.
- bpo-39434 [https://bugs.python.org/issue?@action=redirect&bpo=39434]: floor division of float operation now has a better performance. Also the message of **`ZeroDivisionError`** for this operation is updated. Patch by Donghee Na.
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _codecs extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _bz2 extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _abc extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).
- bpo-39320 [https://bugs.python.org/issue?@action=redirect&bpo=39320]: Replace two complex bytecodes for building dicts with two simpler ones. The new bytecodes `DICT_MERGE` and `DICT_UPDATE` have been added The old bytecodes `BUILD_MAP_UNPACK` and `BUILD_MAP_UNPACK_WITH_CALL` have been removed.
- bpo-39219 [https://bugs.python.org/issue?@action=redirect&bpo=39219]: Syntax errors raised in the tokenizer now always set correct “text” and “offset” attributes.
- bpo-36051 [https://bugs.python.org/issue?@action=redirect&bpo=36051]: Drop the GIL during large `bytes.join` operations. Patch by Bruce Merry.
- bpo-38960 [https://bugs.python.org/issue?@action=redirect&bpo=38960]: Fix DTrace build issues on FreeBSD. Patch by David Carlier.
- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: Speed up calls to `range()` by about 30%, by using the PEP 590 `vectorcall` calling convention. Patch by Mark Shannon.
- bpo-36144 [https://bugs.python.org/issue?@action=redirect&bpo=36144]: **`dict`** (and **`collections.UserDict`**) objects now support PEP 584’s merge (`|`) and update (`|=`) operators. Patch by Brandt Bucher.

- bpo-32856 [https://bugs.python.org/issue?@action=redirect&bpo=32856]: Optimized the idiom for assignment a temporary variable in comprehensions. Now `for y in [expr]` in comprehensions is as fast as a simple assignment `y = expr`.

Biblioteca

- bpo-30566 [https://bugs.python.org/issue?@action=redirect&bpo=30566]: Fix **`IndexError`** when trying to decode an invalid string with punycode codec.
- bpo-39649 [https://bugs.python.org/issue?@action=redirect&bpo=39649]: Remove obsolete check for `__args__` in `bdb.Bdb.format_stack_entry`.
- bpo-39648 [https://bugs.python.org/issue?@action=redirect&bpo=39648]: Expanded **`math.gcd()`** and **`math.lcm()`** to handle multiple arguments.
- bpo-39681 [https://bugs.python.org/issue?@action=redirect&bpo=39681]: Fix a regression where the C pickle module wouldn't allow unpickling from a file-like object that doesn't expose a readinto() method.
- bpo-35950 [https://bugs.python.org/issue?@action=redirect&bpo=35950]: Raise **`io.UnsupportedOperation`** in **`io.BufferedReader.truncate()`** when it is called on a read-only **`io.BufferedReader`** instance.
- bpo-39479 [https://bugs.python.org/issue?@action=redirect&bpo=39479]: Add **`math.lcm()`** function: least common multiple.
- bpo-39674 [https://bugs.python.org/issue?@action=redirect&bpo=39674]: Revert "Do not expose abstract collection classes in the collections module" change (bpo-25988 [https://bugs.python.org/issue?@action=redirect&bpo=25988]). Aliases to ABC like collections.Mapping are kept in Python 3.9 to ease transition from Python 2.7, but will be removed in Python 3.10.
- bpo-39104 [https://bugs.python.org/issue?@action=redirect&bpo=39104]: Fix hanging ProcessPoolExcutor on `shutdown(wait=False)` when a task has failed pickling.
- bpo-39627 [https://bugs.python.org/issue?@action=redirect&bpo=39627]: Fixed TypedDict totality check for inherited keys.
- bpo-39474 [https://bugs.python.org/issue?@action=redirect&bpo=39474]: Fixed starting position of AST for expressions like `(a)(b)`, `(a)[b]` and `(a).b`.
- bpo-21016 [https://bugs.python.org/issue?@action=redirect&bpo=21016]: The **`pydoc`** and **`trace`** modules now use the **`sysconfig`** module to get the path to

the Python standard library, to support uncommon installation path like `/usr/lib64/python3.9/` on Fedora. Patch by Jan Matějek.

- bpo-39590 [https://bugs.python.org/issue?@action=redirect&bpo=39590]: Collections.deque now holds strong references during deque.__contains__ and deque.count, fixing crashes.
- bpo-39586 [https://bugs.python.org/issue?@action=redirect&bpo=39586]: The distutils `bdist_msi` command is deprecated in Python 3.9, use `bdist_wheel` (wheel packages) instead.
- bpo-39595 [https://bugs.python.org/issue?@action=redirect&bpo=39595]: Improved performance of zipfile.Path for files with a large number of entries. Also improved performance and fixed minor issue as published with importlib_metadata 1.5 [https://importlib-metadata.readthedocs.io/en/latest/history.html#v1-5-0].
- bpo-39350 [https://bugs.python.org/issue?@action=redirect&bpo=39350]: Fix regression in **`fractions.Fraction`** if the numerator and/or the denominator is an **`int`** subclass. The **`math.gcd()`** function is now used to normalize the *numerator* and *denominator*. **`math.gcd()`** always return a **`int`** type. Previously, the GCD type depended on *numerator* and *denominator*.
- bpo-39567 [https://bugs.python.org/issue?@action=redirect&bpo=39567]: Added audit for **`os.walk()`**, **`os.fwalk()`**, **`pathlib.Path.glob()`** and **`pathlib.Path.rglob()`**.
- bpo-39559 [https://bugs.python.org/issue?@action=redirect&bpo=39559]: Remove unused, undocumented argument `getters` from **`uuid.getnode()`**
- bpo-38149 [https://bugs.python.org/issue?@action=redirect&bpo=38149]: **`sys.audit()`** is now called only once per call of **`glob.glob()`** and **`glob.iglob()`**.
- bpo-39546 [https://bugs.python.org/issue?@action=redirect&bpo=39546]: Fix a regression in **`ArgumentParser`** where `allow_abbrev=False` was ignored for long options that used a prefix character other than “-“.
- bpo-39450 [https://bugs.python.org/issue?@action=redirect&bpo=39450]: Striped whitespace from docstring before returning it from **`unittest.case.shortDescription()`**.
- bpo-12915 [https://bugs.python.org/issue?@action=redirect&bpo=12915]: A new function `resolve_name` has been added to the `pkgutil` module. This resolves a string of the form `'a.b.c.d'` or `'a.b:c.d'` to an object. In

the example, `a.b` is a package/module and `c.d` is an object within that package/module reached via recursive attribute access.

- bpo-39353 [https://bugs.python.org/issue?@action=redirect&bpo=39353]: The **`binascii.crc_hqx()`** function is no longer deprecated.
- bpo-39493 [https://bugs.python.org/issue?@action=redirect&bpo=39493]: Mark `typing.IO.closed` as a property
- bpo-39491 [https://bugs.python.org/issue?@action=redirect&bpo=39491]: Add **`typing.Annotated`** and `include_extras` parameter to **`typing.get_type_hints()`** as part of **PEP 593** [https://peps.python.org/pep-0593/]. Patch by Till Varoquaux, documentation by Till Varoquaux and Konstantin Kashin.
- bpo-39485 [https://bugs.python.org/issue?@action=redirect&bpo=39485]: Fix a bug in **`unittest.mock.create_autospec()`** that would complain about the wrong number of arguments for custom descriptors defined in an extension module returning functions.
- bpo-38932 [https://bugs.python.org/issue?@action=redirect&bpo=38932]: Mock fully resets child objects on reset_mock(). Patch by Vegard Stikbakke
- bpo-39082 [https://bugs.python.org/issue?@action=redirect&bpo=39082]: Allow AsyncMock to correctly patch static/class methods
- bpo-39432 [https://bugs.python.org/issue?@action=redirect&bpo=39432]: Implement PEP-489 algorithm for non-ascii “PyInit_…” symbol names in distutils to make it export the correct init symbol also on Windows.
- bpo-18819 [https://bugs.python.org/issue?@action=redirect&bpo=18819]: Omit `devmajor` and `devminor` fields for non-device files in **`tarfile`** archives, enabling bit-for-bit compatibility with GNU `tar(1)`.
- bpo-39349 [https://bugs.python.org/issue?@action=redirect&bpo=39349]: Added a new *cancel_futures* parameter to **`concurrent.futures.Executor.shutdown()`** that cancels all pending futures which have not started running, instead of waiting for them to complete before shutting down the executor.
- bpo-39274 [https://bugs.python.org/issue?@action=redirect&bpo=39274]: `bool(fraction.Fraction)` now returns a boolean even if (numerator != 0) does not return a boolean (ex: numpy number).
- bpo-34793 [https://bugs.python.org/issue?@action=redirect&bpo=34793]: Remove support for `with (await asyncio.lock):` and `with (yield from asyncio.lock):`. The same is correct for `asyncio.Condition` and `asyncio.Semaphore`.

- bpo-25597 [https://bugs.python.org/issue?@action=redirect&bpo=25597]: Ensure, if `wraps` is supplied to **`unittest.mock.MagicMock`**, it is used to calculate return values for the magic methods instead of using the default return values. Patch by Karthikeyan Singaravelan.
- bpo-36350 [https://bugs.python.org/issue?@action=redirect&bpo=36350]: `inspect.Signature.parameters` and `inspect.BoundArguments.arguments` are now dicts instead of OrderedDicts. Patch contributed by Rémi Lapeyre.
- bpo-35727 [https://bugs.python.org/issue?@action=redirect&bpo=35727]: Fix sys.exit() and sys.exit(None) exit code propagation when used in multiprocessing.Process.
- bpo-32173 [https://bugs.python.org/issue?@action=redirect&bpo=32173]: * Add `lazycache` function to `__all__`. * Use `dict.clear` to clear the cache. * Refactoring `getline` function and `checkcache` function.

Documentação

- bpo-17422 [https://bugs.python.org/issue?@action=redirect&bpo=17422]: The language reference now specifies restrictions on class namespaces. Adapted from a patch by Ethan Furman.
- bpo-39572 [https://bugs.python.org/issue?@action=redirect&bpo=39572]: Updated documentation of `total` flag of `TypedDict`.
- bpo-39654 [https://bugs.python.org/issue?@action=redirect&bpo=39654]: In pyclbr doc, update ‘class’ to ‘module’ where appropriate and add readmodule comment. Patch by Hakan Çelik.
- bpo-39153 [https://bugs.python.org/issue?@action=redirect&bpo=39153]: Clarify refcounting semantics for the following functions: - PyObject_SetItem - PyMapping_SetItemString - PyDict_SetItem - PyDict_SetItemString
- bpo-39392 [https://bugs.python.org/issue?@action=redirect&bpo=39392]: Explain that when filling with turtle, overlap regions may be left unfilled.
- bpo-39369 [https://bugs.python.org/issue?@action=redirect&bpo=39369]: Update mmap readline method description. The fact that the readline method does update the file position should not be ignored since this might give the impression for the programmer that it doesn’t update it.
- bpo-9056 [https://bugs.python.org/issue?@action=redirect&bpo=9056]: Include subsection in TOC for PDF version of docs.

Testes

- bpo-38325 [https://bugs.python.org/issue?@action=redirect&bpo=38325]: Skip tests on non-BMP characters of test_winconsoleio.
- bpo-39502 [https://bugs.python.org/issue?@action=redirect&bpo=39502]: Skip test_zipfile.test_add_file_after_2107() if `time.localtime()` fails with `OverflowError`. It is the case on AIX 6.1 for example.

Construção

- bpo-39489 [https://bugs.python.org/issue?@action=redirect&bpo=39489]: Remove `COUNT_ALLOCS` special build.

Windows

- bpo-39553 [https://bugs.python.org/issue?@action=redirect&bpo=39553]: Delete unused code related to SxS manifests.
- bpo-39439 [https://bugs.python.org/issue?@action=redirect&bpo=39439]: Honor the Python path when a virtualenv is active on Windows.
- bpo-39393 [https://bugs.python.org/issue?@action=redirect&bpo=39393]: Improve the error message when attempting to load a DLL with unresolved dependencies.
- bpo-38883 [https://bugs.python.org/issue?@action=redirect&bpo=38883]: `home()` and `expanduser()` on Windows now prefer `USERPROFILE` and no longer use `HOME`, which is not normally set for regular user accounts. This makes them again behave like `os.path.expanduser()`, which was changed to ignore `HOME` in 3.8, see bpo-36264 [https://bugs.python.org/issue?@action=redirect&bpo=36264].
- bpo-39185 [https://bugs.python.org/issue?@action=redirect&bpo=39185]: The build.bat script has additional options for very-quiet output (-q) and very-verbose output (-vv)

IDLE

- bpo-39663 [https://bugs.python.org/issue?@action=redirect&bpo=39663]: Add tests for pyparse find_good_parse_start().

- bpo-39600 [https://bugs.python.org/issue?@action=redirect&bpo=39600]: In the font configuration window, remove duplicated font names.
- bpo-30780 [https://bugs.python.org/issue?@action=redirect&bpo=30780]: Add remaining configdialog tests for buttons and highlights and keys tabs.
- bpo-39388 [https://bugs.python.org/issue?@action=redirect&bpo=39388]: IDLE Settings Cancel button now cancels pending changes
- bpo-38792 [https://bugs.python.org/issue?@action=redirect&bpo=38792]: Close an IDLE shell calltip if a `KeyboardInterrupt` or shell restart occurs. Patch by Zackery Spytz.

API C

- bpo-35081 [https://bugs.python.org/issue?@action=redirect&bpo=35081]: Move the `bytes_methods.h` header file to the internal C API as `pycore_bytes_methods.h`: it only contains private symbols (prefixed by `_Py`), except of the `PyDoc_STRVAR_shared()` macro.
- bpo-35081 [https://bugs.python.org/issue?@action=redirect&bpo=35081]: Move the `dtoa.h` header file to the internal C API as `pycore_dtoa.h`: it only contains private functions (prefixed by `_Py`). The `math` and `cmath` modules must now be compiled with the `Py_BUILD_CORE` macro defined.
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Add `Py_SET_SIZE()` function to set the size of an object.
- bpo-39500 [https://bugs.python.org/issue?@action=redirect&bpo=39500]: `PyUnicode_IsIdentifier()` does not call `Py_FatalError()` anymore if the string is not ready.
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Add `Py_SET_TYPE()` function to set the type of an object.
- bpo-39573 [https://bugs.python.org/issue?@action=redirect&bpo=39573]: Add a `Py_SET_REFCNT()` function to set the reference counter of an object.
- bpo-39542 [https://bugs.python.org/issue?@action=redirect&bpo=39542]: Convert `PyType_HasFeature()`, `PyType_Check()` and `PyType_CheckExact()` macros to static inline functions.
- bpo-39542 [https://bugs.python.org/issue?@action=redirect&bpo=39542]: In the limited C API, `PyObject_INIT()` and `PyObject_INIT_VAR()` are now defined as aliases to `PyObject_Init()` and `PyObject_InitVar()` to make their implementation opaque. It avoids to leak implementation

details in the limited C API. Exclude the following functions from the limited C API: `_Py_NewReference()`, `_Py_ForgetReference()`, `_PyTraceMalloc_NewReference()` and `_Py_GetRefTotal()`.

- bpo-39542 [https://bugs.python.org/issue?@action=redirect&bpo=39542]: Exclude trashcan mechanism from the limited C API: it requires access to PyTypeObject and PyThreadState structure fields, whereas these structures are opaque in the limited C API.
- bpo-39511 [https://bugs.python.org/issue?@action=redirect&bpo=39511]: The **`PyThreadState_Clear()`** function now calls the **`PyThreadState.on_delete`** callback. Previously, that happened in **`PyThreadState_Delete()`**.
- bpo-38076 [https://bugs.python.org/issue?@action=redirect&bpo=38076]: Fix to clear the interpreter state only after clearing module globals to guarantee module state access from C Extensions during runtime destruction
- bpo-39245 [https://bugs.python.org/issue?@action=redirect&bpo=39245]: The Vectorcall API (PEP 590) was made public, adding the functions `PyObject_Vectorcall`, `PyObject_VectorcallMethod`, `PyVectorcall_Function`, `PyObject_CallOneArg`, `PyObject_CallMethodNoArgs`, `PyObject_CallMethodOneArg`, `PyObject_FastCallDict`, and the flag `Py_TPFLAGS_HAVE_VECTORCALL`.

Python 3.9.0 alpha 3

Data de lançamento: 2020-01-24

Núcleo e embutidos

- bpo-39427 [https://bugs.python.org/issue?@action=redirect&bpo=39427]: Document all possibilities for the `-X` options in the command line help section. Patch by Pablo Galindo.

- bpo-39421 [https://bugs.python.org/issue?@action=redirect&bpo=39421]: Fix possible crashes when operating with the functions in the **`heapq`** module and custom comparison operators.

- bpo-39386 [https://bugs.python.org/issue?@action=redirect&bpo=39386]: Prevent double awaiting of async iterator.

- bpo-17005 [https://bugs.python.org/issue?@action=redirect&bpo=17005]: Add **`functools.TopologicalSorter`** to the **`functools`** module to offers functionality to perform topological sorting of graphs. Patch by Pablo Galindo, Tim Peters and Larry Hastings.

- bpo-39320 [https://bugs.python.org/issue?@action=redirect&bpo=39320]: Replace four complex bytecodes for building sequences with three simpler ones.

  The following four bytecodes have been removed:

  - BUILD_LIST_UNPACK
  - BUILD_TUPLE_UNPACK
  - BUILD_SET_UNPACK
  - BUILD_TUPLE_UNPACK_WITH_CALL

  The following three bytecodes have been added:

  - LIST_TO_TUPLE
  - LIST_EXTEND
  - SET_UPDATE

- bpo-39336 [https://bugs.python.org/issue?@action=redirect&bpo=39336]: Import loaders which publish immutable module objects can now publish immutable packages in addition to individual modules.

- bpo-39322 [https://bugs.python.org/issue?@action=redirect&bpo=39322]: Added a new function **`gc.is_finalized()`** to check if an object has been finalized by the garbage collector. Patch by Pablo Galindo.

- bpo-39048 [https://bugs.python.org/issue?@action=redirect&bpo=39048]: Improve the displayed error message when incorrect types are passed to `async with` statements by looking up the **`__aenter__()`** special method before the **`__aexit__()`** special method when entering an asynchronous context manager. Patch by Géry Ogam.

- bpo-39235 [https://bugs.python.org/issue?@action=redirect&bpo=39235]: Fix AST end location for lone generator expression in function call, e.g. f(i for i

in a).

- bpo-39209 [https://bugs.python.org/issue?@action=redirect&bpo=39209]: Correctly handle multi-line tokens in interactive mode. Patch by Pablo Galindo.

- bpo-1635741 [https://bugs.python.org/issue?@action=redirect&bpo=1635741]: Port _json extension module to multiphase initialization (**PEP 489** [https://peps.python.org/pep-0489/]).

- bpo-39216 [https://bugs.python.org/issue?@action=redirect&bpo=39216]: Fix constant folding optimization for positional only arguments - by Anthony Sottile.

- bpo-39215 [https://bugs.python.org/issue?@action=redirect&bpo=39215]: Fix `SystemError` when nested function has annotation on positional-only argument - by Anthony Sottile.

- bpo-39200 [https://bugs.python.org/issue?@action=redirect&bpo=39200]: Correct the error message when calling the `min()` or `max()` with no arguments. Patch by Donghee Na.

- bpo-39200 [https://bugs.python.org/issue?@action=redirect&bpo=39200]: Correct the error message when trying to construct `range` objects with no arguments. Patch by Pablo Galindo.

- bpo-39166 [https://bugs.python.org/issue?@action=redirect&bpo=39166]: Fix incorrect line execution reporting in trace functions when tracing the last iteration of asynchronous for loops. Patch by Pablo Galindo.

- bpo-39114 [https://bugs.python.org/issue?@action=redirect&bpo=39114]: Fix incorrect line execution reporting in trace functions when tracing exception handlers with name binding. Patch by Pablo Galindo.

- bpo-39156 [https://bugs.python.org/issue?@action=redirect&bpo=39156]: Split the COMPARE_OP bytecode instruction into four distinct instructions.

    - COMPARE_OP for rich comparisons
    - IS_OP for ‘is’ and ‘is not’ tests

    - CONTAINS_OP for ‘in’ and ‘is not’ tests
    - JUMP_IF_NOT_EXC_MATCH for checking exceptions in ‘try-except’ statements.

  This improves the clarity of the interpreter and should provide a modest speedup.

- bpo-38588 [https://bugs.python.org/issue?@action=redirect&bpo=38588]: Fix possible crashes in dict and list when calling `PyObject_RichCompareBool()`.

- bpo-13601 [https://bugs.python.org/issue?@action=redirect&bpo=13601]: By default, `sys.stderr` is line-buffered now, even if `stderr` is redirected to a file. You can still make `sys.stderr` unbuffered by passing the `-u` command-line option or setting the `PYTHONUNBUFFERED` environment variable.

  (Contributed by Jendrik Seipp in bpo-13601 [https://bugs.python.org/issue?@action=redirect&bpo=13601].)

- bpo-38610 [https://bugs.python.org/issue?@action=redirect&bpo=38610]: Fix possible crashes in several list methods by holding strong references to list elements when calling `PyObject_RichCompareBool()`.

- bpo-32021 [https://bugs.python.org/issue?@action=redirect&bpo=32021]: Include brotli .br encoding in mimetypes encodings_map

Biblioteca

- bpo-39430 [https://bugs.python.org/issue?@action=redirect&bpo=39430]: Fixed race condition in lazy imports in `tarfile`.
- bpo-39413 [https://bugs.python.org/issue?@action=redirect&bpo=39413]: The `os.unsetenv()` function is now also available on Windows.
- bpo-39390 [https://bugs.python.org/issue?@action=redirect&bpo=39390]: Fixed a regression with the `ignore` callback of `shutil.copytree()`. The argument types are now `str` and `List[str]` again.
- bpo-39395 [https://bugs.python.org/issue?@action=redirect&bpo=39395]: The `os.putenv()` and `os.unsetenv()` functions are now always available.

- bpo-39406 [https://bugs.python.org/issue?@action=redirect&bpo=39406]: If `setenv()` C function is available, **`os.putenv()`** is now implemented with `setenv()` instead of `putenv()`, so Python doesn't have to handle the environment variable memory.
- bpo-39396 [https://bugs.python.org/issue?@action=redirect&bpo=39396]: Fix `math.nextafter(-0.0, +0.0)` on AIX 7.1.
- bpo-29435 [https://bugs.python.org/issue?@action=redirect&bpo=29435]: Allow **`tarfile.is_tarfile()`** to be used with file and file-like objects, like **`zipfile.is_zipfile()`**. Patch by William Woodruff.
- bpo-39377 [https://bugs.python.org/issue?@action=redirect&bpo=39377]: Removed `encoding` option from **`json.loads()`**. It has been deprecated since Python 3.1.
- bpo-39389 [https://bugs.python.org/issue?@action=redirect&bpo=39389]: Write accurate compression level metadata in **`gzip`** archives, rather than always signaling maximum compression.
- bpo-39366 [https://bugs.python.org/issue?@action=redirect&bpo=39366]: The previously deprecated `xpath()` and `xgtitle()` methods of **`nntplib.NNTP`** have been removed.
- bpo-39357 [https://bugs.python.org/issue?@action=redirect&bpo=39357]: Remove the *buffering* parameter of **`bz2.BZ2File`**. Since Python 3.0, it was ignored and using it was emitting **`DeprecationWarning`**. Pass an open file object, to control how the file is opened. The *compresslevel* parameter becomes keyword-only.
- bpo-39353 [https://bugs.python.org/issue?@action=redirect&bpo=39353]: Deprecate binhex4 and hexbin4 standards. Deprecate the **`binhex`** module and the following **`binascii`** functions: **`b2a_hqx()`**, **`a2b_hqx()`**, **`rlecode_hqx()`**, **`rledecode_hqx()`**, **`crc_hqx()`**.
- bpo-39351 [https://bugs.python.org/issue?@action=redirect&bpo=39351]: Remove `base64.encodestring()` and `base64.decodestring()`, aliases deprecated since Python 3.1: use **`base64.encodebytes()`** and **`base64.decodebytes()`** instead.
- bpo-39350 [https://bugs.python.org/issue?@action=redirect&bpo=39350]: Remove `fractions.gcd()` function, deprecated since Python 3.5 (bpo-22486 [https://bugs.python.org/issue?@action=redirect&bpo=22486]): use **`math.gcd()`** instead.
- bpo-39329 [https://bugs.python.org/issue?@action=redirect&bpo=39329]: **`LMTP`** constructor now has an optional *timeout* parameter. Patch by Donghee

Na.

- bpo-39313 [https://bugs.python.org/issue?@action=redirect&bpo=39313]: Add a new `exec_function` option (*–exec-function* in the CLI) to `RefactoringTool` for making `exec` a function. Patch by Batuhan Taskaya.
- bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259]: **`FTP_TLS`** and **`FTP_TLS`** now raise a **`ValueError`** if the given timeout for their constructor is zero to prevent the creation of a non-blocking socket. Patch by Donghee Na.
- bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259]: **`SMTP`** and **`SMTP_SSL`** now raise a **`ValueError`** if the given timeout for their constructor is zero to prevent the creation of a non-blocking socket. Patch by Donghee Na.
- bpo-39310 [https://bugs.python.org/issue?@action=redirect&bpo=39310]: Add **`math.ulp()`**: return the value of the least significant bit of a float.
- bpo-39297 [https://bugs.python.org/issue?@action=redirect&bpo=39297]: Improved performance of importlib.metadata distribution discovery and resilients to inaccessible sys.path entries (importlib_metadata v1.4.0).
- bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259]: **`NNTP`** and **`NNTP_SSL`** now raise a **`ValueError`** if the given timeout for their constructor is zero to prevent the creation of a non-blocking socket. Patch by Donghee Na.
- bpo-38901 [https://bugs.python.org/issue?@action=redirect&bpo=38901]: When you specify prompt='.' or equivalently python -m venv –prompt . … the basename of the current directory is used to set the created venv's prompt when it's activated.
- bpo-39288 [https://bugs.python.org/issue?@action=redirect&bpo=39288]: Add **`math.nextafter()`**: return the next floating-point value after *x* towards *y*.
- bpo-39259 [https://bugs.python.org/issue?@action=redirect&bpo=39259]: **`POP3`** and **`POP3_SSL`** now raise a **`ValueError`** if the given timeout for their constructor is zero to prevent the creation of a non-blocking socket. Patch by Donghee Na.
- bpo-39242 [https://bugs.python.org/issue?@action=redirect&bpo=39242]: Updated the Gmane domain from news.gmane.org to news.gmane.io which is used for examples of **`NNTP`** news reader server and nntplib tests.

- bpo-35292 [https://bugs.python.org/issue?@action=redirect&bpo=35292]: Proxy the `SimpleHTTPRequestHandler.guess_type` to `mimetypes.guess_type` so the `mimetypes.init` is called lazily to avoid unnecessary costs when **`http.server`** module is imported.
- bpo-39239 [https://bugs.python.org/issue?@action=redirect&bpo=39239]: The **`select.epoll.unregister()`** method no longer ignores the **`EBADF`** error.
- bpo-38907 [https://bugs.python.org/issue?@action=redirect&bpo=38907]: In http.server script, restore binding to IPv4 on Windows.
- bpo-39152 [https://bugs.python.org/issue?@action=redirect&bpo=39152]: Fix ttk.Scale.configure([name]) to return configuration tuple for name or all options. Giovanni Lombardo contributed part of the patch.
- bpo-39198 [https://bugs.python.org/issue?@action=redirect&bpo=39198]: If an exception were to be thrown in `Logger.isEnabledFor` (say, by asyncio timeouts or stopit) , the `logging` global lock may not be released appropriately, resulting in deadlock. This change wraps that block of code with `try...finally` to ensure the lock is released.
- bpo-39191 [https://bugs.python.org/issue?@action=redirect&bpo=39191]: Perform a check for running loop before starting a new task in `loop.run_until_complete()` to fail fast; it prevents the side effect of new task spawning before exception raising.
- bpo-38871 [https://bugs.python.org/issue?@action=redirect&bpo=38871]: Correctly parenthesize filter-based statements that contain lambda expressions in **`lib2to3`**. Patch by Donghee Na.
- bpo-39142 [https://bugs.python.org/issue?@action=redirect&bpo=39142]: A change was made to logging.config.dictConfig to avoid converting instances of named tuples to ConvertingTuple. It’s assumed that named tuples are too specialised to be treated like ordinary tuples; if a user of named tuples requires ConvertingTuple functionality, they will have to implement that themselves in their named tuple class.
- bpo-39158 [https://bugs.python.org/issue?@action=redirect&bpo=39158]: ast.literal_eval() now supports empty sets.
- bpo-39129 [https://bugs.python.org/issue?@action=redirect&bpo=39129]: Fix import path for `asyncio.TimeoutError`
- bpo-39057 [https://bugs.python.org/issue?@action=redirect&bpo=39057]: **`urllib.request.proxy_bypass_environment()`** now ignores leading dots and no longer ignores a trailing newline.

- bpo-39056 [https://bugs.python.org/issue?@action=redirect&bpo=39056]: Fixed handling invalid warning category in the -W option. No longer import the re module if it is not needed.
- bpo-39055 [https://bugs.python.org/issue?@action=redirect&bpo=39055]: **`base64.b64decode()`** with `validate=True` raises now a binascii.Error if the input ends with a single `\n`.
- bpo-21600 [https://bugs.python.org/issue?@action=redirect&bpo=21600]: Fix **`mock.patch.stopall()`** to stop active patches that were created with **`mock.patch.dict()`**.
- bpo-39019 [https://bugs.python.org/issue?@action=redirect&bpo=39019]: Implement dummy `__class_getitem__` for **`tempfile.SpooledTemporaryFile`**.
- bpo-39019 [https://bugs.python.org/issue?@action=redirect&bpo=39019]: Implement dummy `__class_getitem__` for `subprocess.Popen`, `subprocess.CompletedProcess`
- bpo-38914 [https://bugs.python.org/issue?@action=redirect&bpo=38914]: Adjusted the wording of the warning issued by distutils' `check` command when the `author` and `maintainer` fields are supplied but no corresponding e-mail field (`author_email` or `maintainer_email`) is found. The wording now reflects the fact that these fields are suggested, but not required. Patch by Juergen Gmach.
- bpo-38878 [https://bugs.python.org/issue?@action=redirect&bpo=38878]: Fixed __subclasshook__ of **`os.PathLike`** to return a correct result upon inheritance. Patch by Bar Harel.
- bpo-38615 [https://bugs.python.org/issue?@action=redirect&bpo=38615]: **`IMAP4`** and **`IMAP4_SSL`** now have an optional *timeout* parameter for their constructors. Also, the **`open()`** method now has an optional *timeout* parameter with this change. The overridden methods of **`IMAP4_SSL`** and **`IMAP4_stream`** were applied to this change. Patch by Donghee Na.
- bpo-35182 [https://bugs.python.org/issue?@action=redirect&bpo=35182]: Fixed **`Popen.communicate()`** subsequent call crash when the child process has already closed any piped standard stream, but still continues to be running. Patch by Andriy Maletsky.
- bpo-38630 [https://bugs.python.org/issue?@action=redirect&bpo=38630]: On Unix, **`subprocess.Popen.send_signal()`** now polls the process status. Polling reduces the risk of sending a signal to the wrong process if the process completed, the **`subprocess.Popen.returncode`** attribute is

still `None`, and the pid has been reassigned (recycled) to a new different process.

- bpo-38536 [https://bugs.python.org/issue?@action=redirect&bpo=38536]: Removes trailing space in formatted currency with `international=True` and a locale with symbol following value. E.g. `locale.currency(12.34, international=True)` returned `'12,34 EUR '` instead of `'12,34 EUR'`.
- bpo-38473 [https://bugs.python.org/issue?@action=redirect&bpo=38473]: Use signature from inner mock for autospecced methods attached with **`unittest.mock.attach_mock()`**. Patch by Karthikeyan Singaravelan.
- bpo-38361 [https://bugs.python.org/issue?@action=redirect&bpo=38361]: Fixed an issue where `ident` could include a leading path separator when **`syslog.openlog()`** was called without arguments.
- bpo-38293 [https://bugs.python.org/issue?@action=redirect&bpo=38293]: Add **`copy.copy()`** and **`copy.deepcopy()`** support to **`property()`** objects.
- bpo-37958 [https://bugs.python.org/issue?@action=redirect&bpo=37958]: Added the pstats.Stats.get_profile_dict() method to return the profile data as a StatsProfile instance.
- bpo-28367 [https://bugs.python.org/issue?@action=redirect&bpo=28367]: Termios magic constants for the following baud rates: - B500000 - B576000 - B921600 - B1000000 - B1152000 - B1500000 - B2000000 - B2500000 - B3000000 - B3500000 - B4000000 Patch by Andrey Smirnov

Documentação

- bpo-39381 [https://bugs.python.org/issue?@action=redirect&bpo=39381]: Mention in docs that **`asyncio.get_event_loop()`** implicitly creates new event loop only if called from the main thread.
- bpo-38918 [https://bugs.python.org/issue?@action=redirect&bpo=38918]: Add an entry for `__module__` in the “function” & “method” sections of the **`inspect`** docs’ Tipos e membros table.
- bpo-3530 [https://bugs.python.org/issue?@action=redirect&bpo=3530]: In the **`ast`** module documentation, fix a misleading `NodeTransformer` example and add advice on when to use the `fix_missing_locations` function.

Construção

- bpo-39395 [https://bugs.python.org/issue?@action=redirect&bpo=39395]: On non-Windows platforms, the **`setenv()`** and **`unsetenv()`** functions are now required to build Python.
- bpo-39160 [https://bugs.python.org/issue?@action=redirect&bpo=39160]: Updated the documentation in `./configure --help` to show default values, reference documentation where required and add additional explanation where needed.
- bpo-39144 [https://bugs.python.org/issue?@action=redirect&bpo=39144]: The ctags and etags build targets both include Modules/_ctypes and Python standard library source files.

IDLE

- bpo-39050 [https://bugs.python.org/issue?@action=redirect&bpo=39050]: Make IDLE Settings dialog Help button work again.
- bpo-34118 [https://bugs.python.org/issue?@action=redirect&bpo=34118]: Tag memoryview, range, and tuple as classes, the same as list, etcetera, in the library manual built-in functions list.
- bpo-32989 [https://bugs.python.org/issue?@action=redirect&bpo=32989]: Add tests for editor newline_and_indent_event method. Remove dead code from pyparse find_good_parse_start method.

API C

- bpo-39372 [https://bugs.python.org/issue?@action=redirect&bpo=39372]: Clean header files of interfaces defined but with no implementation. The public API symbols being removed are: `_PyBytes_InsertThousandsGroupingLocale`, `_PyBytes_InsertThousandsGrouping`, `_Py_InitializeFromArgs`, `_Py_InitializeFromWideArgs`, `_PyFloat_Repr`, `_PyFloat_Digits`, `_PyFloat_DigitsInit`, `PyFrame_ExtendStack`, `_PyAIterWrapper_Type`, `PyNullImporter_Type`, `PyCmpWrapper_Type`, `PySortWrapper_Type`, `PyNoArgsFunction`.
- bpo-39164 [https://bugs.python.org/issue?@action=redirect&bpo=39164]: Add a private `_PyErr_GetExcInfo()` function to retrieve exception information of the specified Python thread state.

Python 3.9.0 alpha 2

Data de lançamento: 2019-12-18

Segurança

- bpo-38945 [https://bugs.python.org/issue?@action=redirect&bpo=38945]: Newline characters have been escaped when performing uu encoding to prevent them from overflowing into to content section of the encoded file. This prevents malicious or accidental modification of data during the decoding process.
- bpo-37228 [https://bugs.python.org/issue?@action=redirect&bpo=37228]: Due to significant security concerns, the *reuse_address* parameter of **`asyncio.loop.create_datagram_endpoint()`** is no longer supported. This is because of the behavior of `SO_REUSEADDR` in UDP. For more details, see the documentation for `loop.create_datagram_endpoint()`. (Contributed by Kyle Stanley, Antoine Pitrou, and Yury Selivanov in bpo-37228 [https://bugs.python.org/issue?@action=redirect&bpo=37228].)
- bpo-38804 [https://bugs.python.org/issue?@action=redirect&bpo=38804]: Fixes a ReDoS vulnerability in **`http.cookiejar`**. Patch by Ben Caller.

Núcleo e embutidos

- bpo-39028 [https://bugs.python.org/issue?@action=redirect&bpo=39028]: Slightly improve the speed of keyword argument parsing with many kwargs by strengthening the assumption that kwargs are interned strings.
- bpo-39080 [https://bugs.python.org/issue?@action=redirect&bpo=39080]: Fix the value of *end_col_offset* for Starred Expression AST nodes when they are among the elements in the *args* attribute of Call AST nodes.
- bpo-39031 [https://bugs.python.org/issue?@action=redirect&bpo=39031]: When parsing an “elif” node, lineno and col_offset of the node now point to the “elif” keyword and not to its condition, making it consistent with the “if” node. Patch by Lysandros Nikolaou.
- bpo-20443 [https://bugs.python.org/issue?@action=redirect&bpo=20443]: In Python 3.9.0a1, sys.argv[0] was made an absolute path if a filename was

specified on the command line. Revert this change, since most users expect sys.argv to be unmodified.

- bpo-39008 [https://bugs.python.org/issue?@action=redirect&bpo=39008]: **`PySys_Audit()`** now requires `Py_ssize_t` to be used for size arguments in the format string, regardless of whether `PY_SSIZE_T_CLEAN` was defined at include time.
- bpo-38673 [https://bugs.python.org/issue?@action=redirect&bpo=38673]: In REPL mode, don't switch to PS2 if the line starts with comment or whitespace. Based on work by Batuhan Taşkaya.
- bpo-38922 [https://bugs.python.org/issue?@action=redirect&bpo=38922]: Calling `replace` on a code object now raises the `code.__new__` audit event.
- bpo-38920 [https://bugs.python.org/issue?@action=redirect&bpo=38920]: Add audit hooks for when **`sys.excepthook()`** and **`sys.unraisablehook()`** are invoked.
- bpo-38892 [https://bugs.python.org/issue?@action=redirect&bpo=38892]: Improve documentation for audit events table and functions.
- bpo-38852 [https://bugs.python.org/issue?@action=redirect&bpo=38852]: Set the thread stack size to 8 Mb for debug builds on android platforms.
- bpo-38858 [https://bugs.python.org/issue?@action=redirect&bpo=38858]: Each Python subinterpreter now has its own "small integer singletons": numbers in [-5; 257] range. It is no longer possible to change the number of small integers at build time by overriding `NSMALLNEGINTS` and `NSMALLPOSINTS` macros: macros should now be modified manually in `pycore_pystate.h` header file.
- bpo-36854 [https://bugs.python.org/issue?@action=redirect&bpo=36854]: The garbage collector state becomes per interpreter (`PyInterpreterState.gc`), rather than being global (`_PyRuntimeState.gc`).
- bpo-38835 [https://bugs.python.org/issue?@action=redirect&bpo=38835]: The `PyFPE_START_PROTECT()` and `PyFPE_END_PROTECT()` macros are empty: they have been doing nothing for the last year, so stop using them.
- bpo-38328 [https://bugs.python.org/issue?@action=redirect&bpo=38328]: Sped up the creation time of constant **`list`** and **`set`** displays. Patch by Brandt Bucher.
- bpo-38707 [https://bugs.python.org/issue?@action=redirect&bpo=38707]: `MainThread.native_id` is now correctly reset in child processes

spawned using `multiprocessing.Process`, instead of retaining the parent's value.

- bpo-38629 [https://bugs.python.org/issue?@action=redirect&bpo=38629]: Added `__floor__` and `__ceil__` methods to float object. Patch by Batuhan Taşkaya.
- bpo-27145 [https://bugs.python.org/issue?@action=redirect&bpo=27145]: int + int and int - int operators can now return small integer singletons. Patch by hongweipeng.
- bpo-38021 [https://bugs.python.org/issue?@action=redirect&bpo=38021]: Provide a platform tag for AIX that is sufficient for PEP425 binary distribution identification. Patch by Michael Felt.
- bpo-35409 [https://bugs.python.org/issue?@action=redirect&bpo=35409]: Ignore GeneratorExit exceptions when throwing an exception into the aclose coroutine of an asynchronous generator.
- bpo-33387 [https://bugs.python.org/issue?@action=redirect&bpo=33387]: Removed WITH_CLEANUP_START, WITH_CLEANUP_FINISH, BEGIN_FINALLY, END_FINALLY, CALL_FINALLY and POP_FINALLY bytecodes. Replaced with RERAISE and WITH_EXCEPT_START bytecodes. The compiler now generates different code for exceptional and non-exceptional branches for 'with' and 'try-except' statements. For 'try-finally' statements the 'finally' block is replicated for each exit from the 'try' body.

Biblioteca

- bpo-39033 [https://bugs.python.org/issue?@action=redirect&bpo=39033]: Fix `NameError` in `zipimport`. Patch by Karthikeyan Singaravelan.

- bpo-39022 [https://bugs.python.org/issue?@action=redirect&bpo=39022]: Update importlib.metadata to include improvements from importlib_metadata 1.3 including better serialization of EntryPoints and improved documentation for custom finders.

- bpo-39006 [https://bugs.python.org/issue?@action=redirect&bpo=39006]: Fix asyncio when the ssl module is missing: only check for ssl.SSLSocket instance if the ssl module is available.

- bpo-38708 [https://bugs.python.org/issue?@action=redirect&bpo=38708]: Fix a potential IndexError in email parser when parsing an empty msg-id.

- bpo-38698 [https://bugs.python.org/issue?@action=redirect&bpo=38698]: Add a new `InvalidMessageID` token to email parser to represent invalid Message-ID headers. Also, add defects when there is remaining value after parsing the header.

- bpo-38994 [https://bugs.python.org/issue?@action=redirect&bpo=38994]: Implement `__class_getitem__` for `os.PathLike`, `pathlib.Path`.

- bpo-38979 [https://bugs.python.org/issue?@action=redirect&bpo=38979]: Return class from `ContextVar.__class_getitem__` to simplify subclassing.

- bpo-38978 [https://bugs.python.org/issue?@action=redirect&bpo=38978]: Implement `__class_getitem__` on asyncio objects (Future, Task, Queue). Patch by Batuhan Taskaya.

- bpo-38916 [https://bugs.python.org/issue?@action=redirect&bpo=38916]: **`array.array`**: Remove `tostring()` and `fromstring()` methods. They were aliases to `tobytes()` and `frombytes()`, deprecated since Python 3.2.

- bpo-38986 [https://bugs.python.org/issue?@action=redirect&bpo=38986]: Make repr of C accelerated TaskWakeupMethWrapper the same as of pure Python version.

- bpo-38982 [https://bugs.python.org/issue?@action=redirect&bpo=38982]: Fix asyncio `PidfdChildWatcher`: handle `waitpid()` error. If `waitpid()` is called elsewhere, `waitpid()` call fails with **`ChildProcessError`**: use return code 255 in this case, and log a warning. It ensures that the pidfd file descriptor is closed if this error occurs.

- bpo-38529 [https://bugs.python.org/issue?@action=redirect&bpo=38529]: Drop too noisy asyncio warning about deletion of a stream without explicit `.close()` call.

- bpo-27413 [https://bugs.python.org/issue?@action=redirect&bpo=27413]: Added ability to pass through `ensure_ascii` options to json.dumps in the `json.tool` command-line interface.

- bpo-38634 [https://bugs.python.org/issue?@action=redirect&bpo=38634]: The **`readline`** module now detects if Python is linked to libedit at runtime on all platforms. Previously, the check was only done on macOS.

- bpo-33684 [https://bugs.python.org/issue?@action=redirect&bpo=33684]: Fix `json.tool` failed to read a JSON file with non-ASCII characters when locale encoding is not UTF-8.

- bpo-38698 [https://bugs.python.org/issue?@action=redirect&bpo=38698]: Prevent UnboundLocalError to pop up in parse_message_id.

  parse_message_id() was improperly using a token defined inside an exception handler, which was raising `UnboundLocalError` on parsing an invalid value. Patch by Claudiu Popa.

- bpo-38927 [https://bugs.python.org/issue?@action=redirect&bpo=38927]: Use `python -m pip` instead of `pip` to upgrade dependencies in venv.

- bpo-26730 [https://bugs.python.org/issue?@action=redirect&bpo=26730]: Fix `SpooledTemporaryFile.rollover()` might corrupt the file when it is in text mode. Patch by Serhiy Storchaka.

- bpo-38881 [https://bugs.python.org/issue?@action=redirect&bpo=38881]: random.choices() now raises a ValueError when all the weights are zero.

- bpo-38876 [https://bugs.python.org/issue?@action=redirect&bpo=38876]: Raise pickle.UnpicklingError when loading an item from memo for invalid input.

  The previous code was raising a `KeyError` for both the Python and C implementation. This was caused by the specified index of an invalid input which did not exist in the memo structure, where the pickle stores what objects it has seen. The malformed input would have

caused either a `BINGET` or `LONG_BINGET` load from the memo, leading to a `KeyError` as the determined index was bogus. Patch by Claudiu Popa.

- bpo-38688 [https://bugs.python.org/issue?@action=redirect&bpo=38688]: Calling func:`shutil.copytree` to copy a directory tree from one directory to another subdirectory resulted in an endless loop and a RecursionError. A fix was added to consume an iterator and create the list of the entries to be copied, avoiding the recursion for newly created directories. Patch by Bruno P. Kinoshita.

- bpo-38863 [https://bugs.python.org/issue?@action=redirect&bpo=38863]: Improve **`is_cgi()`** function in **`http.server`**, which enables processing the case that cgi directory is a child of another directory other than root.

- bpo-37838 [https://bugs.python.org/issue?@action=redirect&bpo=37838]: **`typing.get_type_hints()`** properly handles functions decorated with **`functools.wraps()`**.

- bpo-38870 [https://bugs.python.org/issue?@action=redirect&bpo=38870]: Expose **`ast.unparse()`** as a function of the **`ast`** module that can be used to unparse an **`ast.AST`** object and produce a string with code that would produce an equivalent **`ast.AST`** object when parsed. Patch by Pablo Galindo and Batuhan Taskaya.

- bpo-38859 [https://bugs.python.org/issue?@action=redirect&bpo=38859]: AsyncMock now returns StopAsyncIteration on the exhaustion of a side_effects iterable. Since PEP-479 its Impossible to raise a StopIteration exception from a coroutine.

- bpo-38857 [https://bugs.python.org/issue?@action=redirect&bpo=38857]: AsyncMock fix for return values that are awaitable types. This also covers side_effect iterable values that happened to be awaitable, and wraps callables that return an awaitable type. Before these awaitables were being awaited instead of being returned as is.

- bpo-38834 [https://bugs.python.org/issue?@action=redirect&bpo=38834]: **`typing.TypedDict`** subclasses now track which keys are optional using

the `__required_keys__` and `__optional_keys__` attributes, to enable runtime validation by downstream projects. Patch by Zac Hatfield-Dodds.

- bpo-38821 [https://bugs.python.org/issue?@action=redirect&bpo=38821]: Fix unhandled exceptions in **`argparse`** when internationalizing error messages for arguments with `nargs` set to special (non-integer) values. Patch by Federico Bond.

- bpo-38820 [https://bugs.python.org/issue?@action=redirect&bpo=38820]: Make Python compatible with OpenSSL 3.0.0. **`ssl.SSLSocket.getpeercert()`** no longer returns IPv6 addresses with a trailing new line.

- bpo-38811 [https://bugs.python.org/issue?@action=redirect&bpo=38811]: Fix an unhandled exception in **`pathlib`** when **`os.link()`** is missing. Patch by Toke Høiland-Jørgensen.

- bpo-38686 [https://bugs.python.org/issue?@action=redirect&bpo=38686]: Added support for multiple `qop` values in **`urllib.request.AbstractDigestAuthHandler`**.

- bpo-38712 [https://bugs.python.org/issue?@action=redirect&bpo=38712]: Add the Linux-specific **`signal.pidfd_send_signal()`** function, which allows sending a signal to a process identified by a file descriptor rather than a pid.

- bpo-38348 [https://bugs.python.org/issue?@action=redirect&bpo=38348]: Add `-i` and `--indent` (indentation level), and `--no-type-comments` (type comments) command line options to ast parsing tool.

- bpo-37523 [https://bugs.python.org/issue?@action=redirect&bpo=37523]: Change **`zipfile.ZipExtFile`** to raise `ValueError` when trying to access the underlying file object after it has been closed. This new behavior is consistent with how accessing closed files is handled in other parts of Python.

- bpo-38045 [https://bugs.python.org/issue?@action=redirect&bpo=38045]: Improve the performance of **`enum._decompose()`** in **`enum`**. Patch by hongweipeng.

- bpo-36820 [https://bugs.python.org/issue?@action=redirect&bpo=36820]: Break cycle generated when saving an exception in socket.py, codeop.py and dyld.py as they keep alive not only the exception but user objects through the `__traceback__` attribute. Patch by Mario Corchero.

- bpo-36406 [https://bugs.python.org/issue?@action=redirect&bpo=36406]: Handle namespace packages in **`doctest`**. Patch by Karthikeyan Singaravelan.

- bpo-34776 [https://bugs.python.org/issue?@action=redirect&bpo=34776]: Fix dataclasses to support forward references in type annotations

- bpo-20928 [https://bugs.python.org/issue?@action=redirect&bpo=20928]: ElementTree supports recursive XInclude processing. Patch by Stefan Behnel.

- bpo-29636 [https://bugs.python.org/issue?@action=redirect&bpo=29636]: Add whitespace options for formatting JSON with the `json.tool` CLI. The following mutually exclusive options are now supported: `--indent` for setting the indent level in spaces; `--tab` for indenting with tabs; `--no-indent` for suppressing newlines; and `--compact` for suppressing all whitespace. The default behavior remains the same as `--indent=4`.

Documentação

- bpo-38928 [https://bugs.python.org/issue?@action=redirect&bpo=38928]: Correct when venv's `upgrade_dependencies()` and `--upgrade-deps` are added.
- bpo-38899 [https://bugs.python.org/issue?@action=redirect&bpo=38899]: Update documentation to state that to activate virtual environments under fish one should use `source`, not `.` as documented at https://fishshell.com/docs/current/cmds/source.html.
- bpo-22377 [https://bugs.python.org/issue?@action=redirect&bpo=22377]: Improves documentation of the values that **`datetime.datetime.strptime()`** accepts for `%Z`. Patch by Karl Dubost.

Testes

- bpo-38546 [https://bugs.python.org/issue?@action=redirect&bpo=38546]: Fix test_ressources_gced_in_workers() of test_concurrent_futures: explicitly stop the manager to prevent leaking a child process running in the background after the test completes.
- bpo-38546 [https://bugs.python.org/issue?@action=redirect&bpo=38546]: Multiprocessing and concurrent.futures tests now stop the resource tracker process when tests complete.
- bpo-38614 [https://bugs.python.org/issue?@action=redirect&bpo=38614]: Replace hardcoded timeout constants in tests with new **`test.support`** constants: **`LOOPBACK_TIMEOUT`**, **`INTERNET_TIMEOUT`**, **`SHORT_TIMEOUT`** and **`LONG_TIMEOUT`**. It becomes easier to adjust these four timeout constants for all tests at once, rather than having to adjust every single test file.
- bpo-38547 [https://bugs.python.org/issue?@action=redirect&bpo=38547]: Fix test_pty: if the process is the session leader, closing the master file descriptor raises a SIGHUP signal: simply ignore SIGHUP when running the tests.
- bpo-38992 [https://bugs.python.org/issue?@action=redirect&bpo=38992]: Fix a test for **`math.fsum()`** that was failing due to constant folding.
- bpo-38991 [https://bugs.python.org/issue?@action=redirect&bpo=38991]: **`test.support`**: **`run_python_until_end()`**, **`assert_python_ok()`** and **`assert_python_failure()`** functions no longer strip whitespaces from stderr. Remove `test.support.strip_python_stderr()` function.
- bpo-38965 [https://bugs.python.org/issue?@action=redirect&bpo=38965]: Fix test_faulthandler on GCC 10. Use the “volatile” keyword in `faulthandler._stack_overflow()` to prevent tail call optimization on any compiler, rather than relying on compiler specific pragma.
- bpo-38875 [https://bugs.python.org/issue?@action=redirect&bpo=38875]: test_capi: trashcan tests now require the test “cpu” resource.
- bpo-38841 [https://bugs.python.org/issue?@action=redirect&bpo=38841]: Skip asyncio test_create_datagram_endpoint_existing_sock_unix on platforms lacking a functional bind() for named unix domain sockets.
- bpo-38692 [https://bugs.python.org/issue?@action=redirect&bpo=38692]: Skip the test_posix.test_pidfd_open() test if `os.pidfd_open()` fails with a

`PermissionError`. This situation can happen in a Linux sandbox using a syscall whitelist which doesn't allow the `pidfd_open()` syscall yet.

- bpo-38839 [https://bugs.python.org/issue?@action=redirect&bpo=38839]: Fix some unused functions in tests. Patch by Adam Johnson.
- bpo-38669 [https://bugs.python.org/issue?@action=redirect&bpo=38669]: Raise `TypeError` when passing target as a string with `unittest.mock.patch.object()`.
- bpo-37957 [https://bugs.python.org/issue?@action=redirect&bpo=37957]: test.regrtest now can receive a list of test patterns to ignore (using the -i/–ignore argument) or a file with a list of patterns to ignore (using the –ignore-file argument). Patch by Pablo Galindo.

Construção

- bpo-37404 [https://bugs.python.org/issue?@action=redirect&bpo=37404]: `asyncio` now raises `TypeError` when calling incompatible methods with an `ssl.SSLSocket` socket. Patch by Ido Michael.
- bpo-36500 [https://bugs.python.org/issue?@action=redirect&bpo=36500]: Added an optional "regen" project to the Visual Studio solution that will regenerate all grammar, tokens, and opcodes.

Windows

- bpo-39007 [https://bugs.python.org/issue?@action=redirect&bpo=39007]: Add auditing events to functions in `winreg`.
- bpo-33125 [https://bugs.python.org/issue?@action=redirect&bpo=33125]: Add support for building and releasing Windows ARM64 packages.

macOS

- bpo-37931 [https://bugs.python.org/issue?@action=redirect&bpo=37931]: Fixed a crash on OSX dynamic builds that occurred when re-initializing the posix module after a Py_Finalize if the environment had changed since the previous `import posix`. Patch by Benoît Hudson.

IDLE

- bpo-38944 [https://bugs.python.org/issue?@action=redirect&bpo=38944]: Escape key now closes IDLE completion windows. Patch by Johnny Najera.
- bpo-38943 [https://bugs.python.org/issue?@action=redirect&bpo=38943]: Fix IDLE autocomplete windows not always appearing on some systems. Patch by Johnny Najera.
- bpo-38862 [https://bugs.python.org/issue?@action=redirect&bpo=38862]: 'Strip Trailing Whitespace' on the Format menu removes extra newlines at the end of non-shell files.
- bpo-38636 [https://bugs.python.org/issue?@action=redirect&bpo=38636]: Fix IDLE Format menu tab toggle and file indent width. These functions (default shortcuts Alt-T and Alt-U) were mistakenly disabled in 3.7.5 and 3.8.0.

API C

- bpo-38896 [https://bugs.python.org/issue?@action=redirect&bpo=38896]: Remove `PyUnicode_ClearFreeList()` function: the Unicode free list has been removed in Python 3.3.
- bpo-37340 [https://bugs.python.org/issue?@action=redirect&bpo=37340]: Remove `PyMethod_ClearFreeList()` and `PyCFunction_ClearFreeList()` functions: the free lists of bound method objects have been removed.
- bpo-38835 [https://bugs.python.org/issue?@action=redirect&bpo=38835]: Exclude `PyFPE_START_PROTECT()` and `PyFPE_END_PROTECT()` macros of `pyfpe.h` from `Py_LIMITED_API` (stable API).

Python 3.9.0 alpha 1

Data de lançamento: 2019-11-19

Segurança

- bpo-38722 [https://bugs.python.org/issue?@action=redirect&bpo=38722]: **`runpy`** now uses **`io.open_code()`** to open code files. Patch by Jason Killen.
- bpo-38622 [https://bugs.python.org/issue?@action=redirect&bpo=38622]: Add additional audit events for the **`ctypes`** module.

- bpo-38418 [https://bugs.python.org/issue?@action=redirect&bpo=38418]: Fixes audit event for `os.system()` to be named `os.system`.
- bpo-38243 [https://bugs.python.org/issue?@action=redirect&bpo=38243]: Escape the server title of `xmlrpc.server.DocXMLRPCServer` when rendering the document page as HTML. (Contributed by Donghee Na in bpo-38243 [https://bugs.python.org/issue?@action=redirect&bpo=38243].)
- bpo-38174 [https://bugs.python.org/issue?@action=redirect&bpo=38174]: Update vendorized expat library version to 2.2.8, which resolves **CVE 2019-15903** [https://www.cve.org/CVERecord?id=CVE-2019-15903].
- bpo-37764 [https://bugs.python.org/issue?@action=redirect&bpo=37764]: Fixes email._header_value_parser.get_unstructured going into an infinite loop for a specific case in which the email header does not have trailing whitespace, and the case in which it contains an invalid encoded word. Patch by Ashwin Ramaswami.
- bpo-37461 [https://bugs.python.org/issue?@action=redirect&bpo=37461]: Fix an infinite loop when parsing specially crafted email headers. Patch by Abhilash Raj.
- bpo-37363 [https://bugs.python.org/issue?@action=redirect&bpo=37363]: Adds audit events for the range of supported run commands (see Linha de comando e ambiente).
- bpo-37463 [https://bugs.python.org/issue?@action=redirect&bpo=37463]: ssl.match_hostname() no longer accepts IPv4 addresses with additional text after the address and only quad-dotted notation without trailing whitespaces. Some inet_aton() implementations ignore whitespace and all data after whitespace, e.g. '127.0.0.1 whatever'.
- bpo-37363 [https://bugs.python.org/issue?@action=redirect&bpo=37363]: Adds audit events for `ensurepip`, `ftplib`, `glob`, `imaplib`, `nntplib`, `pdb`, `poplib`, `shutil`, `smtplib`, `sqlite3`, `subprocess`, `telnetlib`, `tempfile` and `webbrowser`, as well as `os.listdir()`, `os.scandir()` and `breakpoint()`.
- bpo-37364 [https://bugs.python.org/issue?@action=redirect&bpo=37364]: `io.open_code()` is now used when reading `.pth` files.
- bpo-34631 [https://bugs.python.org/issue?@action=redirect&bpo=34631]: Updated OpenSSL to 1.1.1c in Windows installer
- bpo-34155 [https://bugs.python.org/issue?@action=redirect&bpo=34155]: Fix parsing of invalid email addresses with more than one `@` (e.g. a@b@c.com.) to

not return the part before 2nd `@` as valid email address. Patch by maxking & jpic.

Núcleo e embutidos

- bpo-38631 [https://bugs.python.org/issue?@action=redirect&bpo=38631]: Replace `Py_FatalError()` call with a regular **`RuntimeError`** exception in **`float.__getformat__()`**.

- bpo-38639 [https://bugs.python.org/issue?@action=redirect&bpo=38639]: Optimized **`math.floor()`**, **`math.ceil()`** and **`math.trunc()`** for floats.

- bpo-38640 [https://bugs.python.org/issue?@action=redirect&bpo=38640]: Fixed a bug in the compiler that was causing to raise in the presence of break statements and continue statements inside always false while loops. Patch by Pablo Galindo.

- bpo-38613 [https://bugs.python.org/issue?@action=redirect&bpo=38613]: Optimized some set operations (e.g. `|`, `^`, and `-`) of `dict_keys`. `d.keys() | other` was slower than `set(d) | other` but they are almost same performance for now.

- bpo-28029 [https://bugs.python.org/issue?@action=redirect&bpo=28029]: `"".replace("", s, n)` now returns `s` instead of an empty string for all non-zero `n`. There are similar changes for **`bytes`** and **`bytearray`** objects.

- bpo-38535 [https://bugs.python.org/issue?@action=redirect&bpo=38535]: Fixed line numbers and column offsets for AST nodes for calls without arguments in decorators.

- bpo-38525 [https://bugs.python.org/issue?@action=redirect&bpo=38525]: Fix a segmentation fault when using reverse iterators of empty `dict` objects. Patch by Donghee Na and Inada Naoki.

- bpo-38465 [https://bugs.python.org/issue?@action=redirect&bpo=38465]: **`bytearray`**, **`array`** and **`mmap`** objects allow now to export more than `2**31` buffers at a time.

- bpo-38469 [https://bugs.python.org/issue?@action=redirect&bpo=38469]: Fixed a bug where the scope of named expressions was not being resolved correctly in the presence of the *global* keyword. Patch by Pablo Galindo.

- bpo-38437 [https://bugs.python.org/issue?@action=redirect&bpo=38437]: Activate the `GC_DEBUG` macro for debug builds of the interpreter (when `Py_DEBUG` is set). Patch by Pablo Galindo.

- bpo-38379 [https://bugs.python.org/issue?@action=redirect&bpo=38379]: When the garbage collector makes a collection in which some objects resurrect (they are reachable from outside the isolated cycles after the finalizers have been executed), do not block the collection of all objects that are still unreachable. Patch by Pablo Galindo and Tim Peters.

- bpo-38379 [https://bugs.python.org/issue?@action=redirect&bpo=38379]: When cyclic garbage collection (gc) runs finalizers that resurrect unreachable objects, the current gc run ends, without collecting any cyclic trash. However, the statistics reported by `collect()` and `get_stats()` claimed that all cyclic trash found was collected, and that the resurrected objects were collected. Changed the stats to report that none were collected.

- bpo-38392 [https://bugs.python.org/issue?@action=redirect&bpo=38392]: In debug mode, **`PyObject_GC_Track()`** now calls `tp_traverse()` of the object type to ensure that the object is valid: test that objects visited by `tp_traverse()` are valid.

- bpo-38210 [https://bugs.python.org/issue?@action=redirect&bpo=38210]: Remove unnecessary intersection and update set operation in dictview with empty set. (Contributed by Donghee Na in bpo-38210 [https://bugs.python.org/issue?@action=redirect&bpo=38210].)

- bpo-38402 [https://bugs.python.org/issue?@action=redirect&bpo=38402]: Check the error from the system's underlying `crypt` or `crypt_r`.

- bpo-37474 [https://bugs.python.org/issue?@action=redirect&bpo=37474]: On FreeBSD, Python no longer calls `fedisableexcept()` at startup to

control the floating-point control mode. The call became useless since FreeBSD 6: it became the default mode.

- bpo-38006 [https://bugs.python.org/issue?@action=redirect&bpo=38006]: Fix a bug due to the interaction of weakrefs and the cyclic garbage collector. We must clear any weakrefs in garbage in order to prevent their callbacks from executing and causing a crash.

- bpo-38317 [https://bugs.python.org/issue?@action=redirect&bpo=38317]: Fix warnings options priority: `PyConfig.warnoptions` has the highest priority, as stated in the **PEP 587** [https://peps.python.org/pep-0587/].

- bpo-38310 [https://bugs.python.org/issue?@action=redirect&bpo=38310]: Predict `BUILD_MAP_UNPACK_WITH_CALL` -> `CALL_FUNCTION_EX` opcode pairs in the main interpreter loop. Patch by Brandt Bucher.

- bpo-36871 [https://bugs.python.org/issue?@action=redirect&bpo=36871]: Improve error handling for the assert_has_calls and assert_has_awaits methods of mocks. Fixed a bug where any errors encountered while binding the expected calls to the mock's spec were silently swallowed, leading to misleading error output.

- bpo-11410 [https://bugs.python.org/issue?@action=redirect&bpo=11410]: Better control over symbol visibility is provided through use of the visibility attributes available in gcc >= 4.0, provided in a uniform way across POSIX and Windows. The POSIX build files have been updated to compile with -fvisibility=hidden, minimising exported symbols.

- bpo-38219 [https://bugs.python.org/issue?@action=redirect&bpo=38219]: Optimized the **`dict`** constructor and the **`update()`** method for the case when the argument is a dict.

- bpo-38236 [https://bugs.python.org/issue?@action=redirect&bpo=38236]: Python now dumps path configuration if it fails to import the Python codecs of the filesystem and stdio encodings.

- bpo-38013 [https://bugs.python.org/issue?@action=redirect&bpo=38013]: Allow to call `async_generator_athrow().throw(...)` even for non-started

async generator helper. It fixes annoying warning at the end of `asyncio.run()` call.

- bpo-38124 [https://bugs.python.org/issue?@action=redirect&bpo=38124]: Fix an off-by-one error in PyState_AddModule that could cause out-of-bounds memory access.

- bpo-38116 [https://bugs.python.org/issue?@action=redirect&bpo=38116]: The select module is now PEP-384 compliant and no longer has static state

- bpo-38113 [https://bugs.python.org/issue?@action=redirect&bpo=38113]: ast module updated to PEP-384 and all statics removed

- bpo-38076 [https://bugs.python.org/issue?@action=redirect&bpo=38076]: The struct module is now PEP-384 compatible

- bpo-38075 [https://bugs.python.org/issue?@action=redirect&bpo=38075]: The random module is now PEP-384 compatible

- bpo-38074 [https://bugs.python.org/issue?@action=redirect&bpo=38074]: zlib module made PEP-384 compatible

- bpo-38073 [https://bugs.python.org/issue?@action=redirect&bpo=38073]: Make pwd extension module PEP-384 compatible

- bpo-38072 [https://bugs.python.org/issue?@action=redirect&bpo=38072]: grp module made PEP-384 compatible

- bpo-38069 [https://bugs.python.org/issue?@action=redirect&bpo=38069]: Make _posixsubprocess PEP-384 compatible

- bpo-38071 [https://bugs.python.org/issue?@action=redirect&bpo=38071]: Make termios extension module PEP-384 compatible

- bpo-38005 [https://bugs.python.org/issue?@action=redirect&bpo=38005]: Fixed comparing and creating of InterpreterID and ChannelID.

- bpo-36946 [https://bugs.python.org/issue?@action=redirect&bpo=36946]: Fix possible signed integer overflow when handling slices. Patch by

hongweipeng.

- bpo-37994 [https://bugs.python.org/issue?@action=redirect&bpo=37994]: Fixed silencing arbitrary errors if an attribute lookup fails in several sites. Only AttributeError should be silenced.

- bpo-8425 [https://bugs.python.org/issue?@action=redirect&bpo=8425]: Optimize set difference_update for the case when the other set is much larger than the base set. (Suggested by Evgeny Kapun with code contributed by Michele Orrù).

- bpo-37966 [https://bugs.python.org/issue?@action=redirect&bpo=37966]: The implementation of `is_normalized()` has been greatly sped up on strings that aren't normalized, by implementing the full normalization-quick-check algorithm from the Unicode standard.

- bpo-37947 [https://bugs.python.org/issue?@action=redirect&bpo=37947]: Adjust correctly the recursion level in the symtable generation for named expressions. Patch by Pablo Galindo.

- bpo-37812 [https://bugs.python.org/issue?@action=redirect&bpo=37812]: The `CHECK_SMALL_INT` macro used inside `Object/longobject.c` has been replaced with an explicit `return` at each call site.

- bpo-37751 [https://bugs.python.org/issue?@action=redirect&bpo=37751]: Fix `codecs.lookup()` to normalize the encoding name the same way than `encodings.normalize_encoding()`, except that **`codecs.lookup()`** also converts the name to lower case.

- bpo-37830 [https://bugs.python.org/issue?@action=redirect&bpo=37830]: Fixed compilation of `break` and `continue` in the `finally` block when the corresponding `try` block contains `return` with a non-constant value.

- bpo-20490 [https://bugs.python.org/issue?@action=redirect&bpo=20490]: Improve import error message for partially initialized module on circular `from` imports - by Anthony Sottile.

- bpo-37840 [https://bugs.python.org/issue?@action=redirect&bpo=37840]: Fix handling of negative indices in `sq_item` of `bytearray`. Patch by Sergey Fedoseev.

- bpo-37802 [https://bugs.python.org/issue?@action=redirect&bpo=37802]: Slightly improve performance of `PyLong_FromUnsignedLong()`, `PyLong_FromUnsignedLongLong()` and `PyLong_FromSize_t()`. Patch by Sergey Fedoseev.

- bpo-37409 [https://bugs.python.org/issue?@action=redirect&bpo=37409]: Ensure explicit relative imports from interactive sessions and scripts (having no parent package) always raise ImportError, rather than treating the current module as the package. Patch by Ben Lewis.

- bpo-32912 [https://bugs.python.org/issue?@action=redirect&bpo=32912]: Reverted bpo-32912 [https://bugs.python.org/issue?@action=redirect&bpo=32912]: emitting `SyntaxWarning` instead of `DeprecationWarning` for invalid escape sequences in string and bytes literals.

- bpo-37757 [https://bugs.python.org/issue?@action=redirect&bpo=37757]: **PEP 572** [https://peps.python.org/pep-0572/]: As described in the PEP, assignment expressions now raise `SyntaxError` when their interaction with comprehension scoping results in an ambiguous target scope.

  The `TargetScopeError` subclass originally proposed by the PEP has been removed in favour of just raising regular syntax errors for the disallowed cases.

- bpo-36279 [https://bugs.python.org/issue?@action=redirect&bpo=36279]: Fix potential use of uninitialized memory in `os.wait3()`.

- bpo-36311 [https://bugs.python.org/issue?@action=redirect&bpo=36311]: Decoding bytes objects larger than 2GiB is faster and no longer fails when a multibyte characters spans a chunk boundary.

- bpo-34880 [https://bugs.python.org/issue?@action=redirect&bpo=34880]: The `assert` statement now works properly if the `AssertionError` exception is being shadowed. Patch by Zackery Spytz.

- bpo-37340 [https://bugs.python.org/issue?@action=redirect&bpo=37340]: Removed object cache (`free_list`) for bound method objects. Temporary bound method objects are less used than before thanks to the `LOAD_METHOD` opcode and the `_PyObject_VectorcallMethod` C API.

- bpo-37648 [https://bugs.python.org/issue?@action=redirect&bpo=37648]: Fixed minor inconsistency in **`list.__contains__()`**, **`tuple.__contains__()`** and a few other places. The collection's item is now always at the left and the needle is on the right of `==`.

- bpo-37444 [https://bugs.python.org/issue?@action=redirect&bpo=37444]: Update differing exception between **`builtins.__import__()`** and **`importlib.__import__()`**.

- bpo-37619 [https://bugs.python.org/issue?@action=redirect&bpo=37619]: When adding a wrapper descriptor from one class to a different class (for example, setting `__add__ = str.__add__` on an `int` subclass), an exception is correctly raised when the operator is called.

- bpo-37593 [https://bugs.python.org/issue?@action=redirect&bpo=37593]: Swap the positions of the *posonlyargs* and *args* parameters in the constructor of **`ast.parameters`** nodes.

- bpo-37543 [https://bugs.python.org/issue?@action=redirect&bpo=37543]: Optimized pymalloc for non PGO build.

- bpo-37537 [https://bugs.python.org/issue?@action=redirect&bpo=37537]: Compute allocated pymalloc blocks inside _Py_GetAllocatedBlocks(). This slows down _Py_GetAllocatedBlocks() but gives a small speedup to _PyObject_Malloc() and _PyObject_Free().

- bpo-37467 [https://bugs.python.org/issue?@action=redirect&bpo=37467]: Fix **`sys.excepthook()`** and **`PyErr_Display()`** if a filename is a bytes string. For example, for a SyntaxError exception where the filename attribute is a bytes string.

- bpo-37433 [https://bugs.python.org/issue?@action=redirect&bpo=37433]: Fix `SyntaxError` indicator printing too many spaces for multi-line strings -

by Anthony Sottile.

- bpo-37417 [https://bugs.python.org/issue?@action=redirect&bpo=37417]: **`bytearray.extend()`** now correctly handles errors that arise during iteration. Patch by Brandt Bucher.

- bpo-37414 [https://bugs.python.org/issue?@action=redirect&bpo=37414]: The undocumented `sys.callstats()` function has been removed. Since Python 3.7, it was deprecated and always returned `None`. It required a special build option `CALL_PROFILE` which was already removed in Python 3.7.

- bpo-37392 [https://bugs.python.org/issue?@action=redirect&bpo=37392]: Remove `sys.getcheckinterval()` and `sys.setcheckinterval()` functions. They were deprecated since Python 3.2. Use **`sys.getswitchinterval()`** and **`sys.setswitchinterval()`** instead. Remove also `check_interval` field of the `PyInterpreterState` structure.

- bpo-37388 [https://bugs.python.org/issue?@action=redirect&bpo=37388]: In development mode and in debug build, *encoding* and *errors* arguments are now checked on string encoding and decoding operations. Examples: **`open()`**, **`str.encode()`** and **`bytes.decode()`**.

  By default, for best performances, the *errors* argument is only checked at the first encoding/decoding error, and the *encoding* argument is sometimes ignored for empty strings.

- bpo-37348 [https://bugs.python.org/issue?@action=redirect&bpo=37348]: Optimized decoding short ASCII string with UTF-8 and ascii codecs. `b"foo".decode()` is about 15% faster. Patch by Inada Naoki.

- bpo-24214 [https://bugs.python.org/issue?@action=redirect&bpo=24214]: Improved support of the surrogatepass error handler in the UTF-8 and UTF-16 incremental decoders.

- bpo-37330 [https://bugs.python.org/issue?@action=redirect&bpo=37330]: **`open()`**, **`io.open()`**, **`codecs.open()`** and **`fileinput.FileInput`** no longer

accept `'U'` (“universal newline”) in the file mode. This flag was deprecated since Python 3.3.

- bpo-35224 [https://bugs.python.org/issue?@action=redirect&bpo=35224]: Reverse evaluation order of key: value in dict comprehensions as proposed in PEP 572. I.e. in `{k: v for ...}`, `k` will be evaluated before `v`.

- bpo-37316 [https://bugs.python.org/issue?@action=redirect&bpo=37316]: Fix the **`PySys_Audit()`** call in **`mmap.mmap`**.

- bpo-37300 [https://bugs.python.org/issue?@action=redirect&bpo=37300]: Remove an unnecessary Py_XINCREF in classobject.c.

- bpo-37269 [https://bugs.python.org/issue?@action=redirect&bpo=37269]: Fix a bug in the peephole optimizer that was not treating correctly constant conditions with binary operators. Patch by Pablo Galindo.

- bpo-20443 [https://bugs.python.org/issue?@action=redirect&bpo=20443]: Python now gets the absolute path of the script filename specified on the command line (ex: “python3 script.py”): the __file__ attribute of the __main__ module and sys.path[0] become an absolute path, rather than a relative path.

- bpo-37257 [https://bugs.python.org/issue?@action=redirect&bpo=37257]: Python’s small object allocator (`obmalloc.c`) now allows (no more than) one empty arena to remain available for immediate reuse, without returning it to the OS. This prevents thrashing in simple loops where an arena could be created and destroyed anew on each iteration.

- bpo-37231 [https://bugs.python.org/issue?@action=redirect&bpo=37231]: The dispatching of type slots to special methods (for example calling `__mul__` when doing `x * y`) has been made faster.

- bpo-36974 [https://bugs.python.org/issue?@action=redirect&bpo=36974]: Implemented separate vectorcall functions for every calling convention of builtin functions and methods. This improves performance for calls.

- bpo-37213 [https://bugs.python.org/issue?@action=redirect&bpo=37213]: Handle correctly negative line offsets in the peephole optimizer. Patch by Pablo Galindo.

- bpo-37219 [https://bugs.python.org/issue?@action=redirect&bpo=37219]: Remove erroneous optimization for empty set differences.

- bpo-15913 [https://bugs.python.org/issue?@action=redirect&bpo=15913]: Implement **`PyBuffer_SizeFromFormat()`** function (previously documented but not implemented): call **`struct.calcsize()`**. Patch by Joannah Nanjekye.

- bpo-36922 [https://bugs.python.org/issue?@action=redirect&bpo=36922]: Slot functions optimize any callable with `Py_TPFLAGS_METHOD_DESCRIPTOR` instead of only instances of `function`.

- bpo-36974 [https://bugs.python.org/issue?@action=redirect&bpo=36974]: The slot `tp_vectorcall_offset` is inherited unconditionally to support `super().__call__()` when the base class uses vectorcall.

- bpo-37160 [https://bugs.python.org/issue?@action=redirect&bpo=37160]: **`threading.get_native_id()`** now also supports NetBSD.

- bpo-37077 [https://bugs.python.org/issue?@action=redirect&bpo=37077]: Add **`threading.get_native_id()`** support for AIX. Patch by M. Felt

- bpo-36781 [https://bugs.python.org/issue?@action=redirect&bpo=36781]: **`sum()`** has been optimized for boolean values.

- bpo-34556 [https://bugs.python.org/issue?@action=redirect&bpo=34556]: Add `--upgrade-deps` to venv module. Patch by Cooper Ry Lees

- bpo-20523 [https://bugs.python.org/issue?@action=redirect&bpo=20523]: `pdb.Pdb` supports ~/.pdbrc in Windows 7. Patch by Tim Hopper and Dan Lidral-Porter.

- bpo-35551 [https://bugs.python.org/issue?@action=redirect&bpo=35551]: Updated encodings: - Removed the “tis260” encoding, which was an alias for

the nonexistent “tactis” codec. - Added “mac_centeuro” as an alias for the mac_latin2 encoding.

- bpo-19072 [https://bugs.python.org/issue?@action=redirect&bpo=19072]: The `classmethod` decorator can now wrap other descriptors such as property objects. Adapted from a patch written by Graham Dumpleton.

- bpo-27575 [https://bugs.python.org/issue?@action=redirect&bpo=27575]: Improve speed of dictview intersection by directly using set intersection logic. Patch by David Su.

- bpo-30773 [https://bugs.python.org/issue?@action=redirect&bpo=30773]: Prohibit parallel running of aclose() / asend() / athrow(). Fix ag_running to reflect the actual running status of the AG.

Biblioteca

- bpo-36589 [https://bugs.python.org/issue?@action=redirect&bpo=36589]: The `curses.update_lines_cols()` function now returns `None` instead of `1` on success.

- bpo-38807 [https://bugs.python.org/issue?@action=redirect&bpo=38807]: Update `TypeError` messages for `os.path.join()` to include `os.PathLike` objects as acceptable input types.

- bpo-38724 [https://bugs.python.org/issue?@action=redirect&bpo=38724]: Add a repr for `subprocess.Popen` objects. Patch by Andrey Doroschenko.

- bpo-38786 [https://bugs.python.org/issue?@action=redirect&bpo=38786]: pydoc now recognizes and parses HTTPS URLs. Patch by python273.

- bpo-38785 [https://bugs.python.org/issue?@action=redirect&bpo=38785]: Prevent asyncio from crashing if parent `__init__` is not called from a constructor of object derived from `asyncio.Future`.

- bpo-38723 [https://bugs.python.org/issue?@action=redirect&bpo=38723]: `pdb` now uses `io.open_code()` to trigger auditing events.

- bpo-27805 [https://bugs.python.org/issue?@action=redirect&bpo=27805]: Allow opening pipes and other non-seekable files in append mode with **`open()`**.

- bpo-38438 [https://bugs.python.org/issue?@action=redirect&bpo=38438]: Simplify the **`argparse`** usage message for `nargs="*"`.

- bpo-38761 [https://bugs.python.org/issue?@action=redirect&bpo=38761]: WeakSet is now registered as a collections.abc.MutableSet.

- bpo-38716 [https://bugs.python.org/issue?@action=redirect&bpo=38716]: logging: change RotatingHandler namer and rotator to class-level attributes. This stops __init__ from setting them to None in the case where a subclass defines them with eponymous methods.

- bpo-38713 [https://bugs.python.org/issue?@action=redirect&bpo=38713]: Add **`os.P_PIDFD`** constant, which may be passed to **`os.waitid()`** to wait on a Linux process file descriptor.

- bpo-38692 [https://bugs.python.org/issue?@action=redirect&bpo=38692]: Add **`asyncio.PidfdChildWatcher`**, a Linux-specific child watcher implementation that polls process file descriptors.

- bpo-38692 [https://bugs.python.org/issue?@action=redirect&bpo=38692]: Expose the Linux `pidfd_open` syscall as **`os.pidfd_open()`**.

- bpo-38602 [https://bugs.python.org/issue?@action=redirect&bpo=38602]: Added constants **`F_OFD_GETLK`**, **`F_OFD_SETLK`** and **`F_OFD_SETLKW`** to the **`fcntl`** module. Patch by Donghee Na.

- bpo-38334 [https://bugs.python.org/issue?@action=redirect&bpo=38334]: Fixed seeking backward on an encrypted **`zipfile.ZipExtFile`**.

- bpo-38312 [https://bugs.python.org/issue?@action=redirect&bpo=38312]: Add **`curses.get_escdelay()`**, **`curses.set_escdelay()`**, **`curses.get_tabsize()`**, and **`curses.set_tabsize()`** functions - by Anthony Sottile.

- bpo-38586 [https://bugs.python.org/issue?@action=redirect&bpo=38586]: Now **`fileConfig()`** correctly sets the .name of handlers loaded.

- bpo-38565 [https://bugs.python.org/issue?@action=redirect&bpo=38565]: Add new cache_parameters() method for functools.lru_cache() to better support pickling.

- bpo-34679 [https://bugs.python.org/issue?@action=redirect&bpo=34679]: asynci.ProactorEventLoop.close() now only calls signal.set_wakeup_fd() in the main thread.

- bpo-31202 [https://bugs.python.org/issue?@action=redirect&bpo=31202]: The case the result of **`pathlib.WindowsPath.glob()`** matches now the case of the pattern for literal parts.

- bpo-36321 [https://bugs.python.org/issue?@action=redirect&bpo=36321]: Remove misspelled attribute. The 3.8 changelog noted that this would be removed in 3.9.

- bpo-38521 [https://bugs.python.org/issue?@action=redirect&bpo=38521]: Fixed erroneous equality comparison in statistics.NormalDist().

- bpo-38493 [https://bugs.python.org/issue?@action=redirect&bpo=38493]: Added **`CLD_KILLED`** and **`CLD_STOPPED`** for **`si_code`**. Patch by Donghee Na.

- bpo-38478 [https://bugs.python.org/issue?@action=redirect&bpo=38478]: Fixed a bug in **`inspect.signature.bind()`** that was causing it to fail when handling a keyword argument with same name as positional-only parameter. Patch by Pablo Galindo.

- bpo-33604 [https://bugs.python.org/issue?@action=redirect&bpo=33604]: Fixed `hmac.new` and `hmac.HMAC` to raise TypeError instead of ValueError when the digestmod parameter, now required in 3.8, is omitted. Also clarified the hmac module documentation and docstrings.

- bpo-38378 [https://bugs.python.org/issue?@action=redirect&bpo=38378]: Parameters *out* and *in* of **`os.sendfile()`** was renamed to *out_fd* and *in_fd*.

- bpo-38417 [https://bugs.python.org/issue?@action=redirect&bpo=38417]: Added support for setting the umask in the child process to the subprocess module on POSIX systems.

- bpo-38449 [https://bugs.python.org/issue?@action=redirect&bpo=38449]: Revert PR 15522, which introduces a regression in `mimetypes.guess_type()` due to improper handling of filenames as urls.

- bpo-38431 [https://bugs.python.org/issue?@action=redirect&bpo=38431]: Fix `__repr__` method for `dataclasses.InitVar` to support typing objects, patch by Samuel Colvin.

- bpo-38109 [https://bugs.python.org/issue?@action=redirect&bpo=38109]: Add missing `stat.S_IFDOOR`, `stat.S_IFPORT`, `stat.S_IFWHT`, `stat.S_ISDOOR()`, `stat.S_ISPORT()`, and `stat.S_ISWHT()` values to the Python implementation of `stat`.

- bpo-38422 [https://bugs.python.org/issue?@action=redirect&bpo=38422]: Clarify docstrings of pathlib suffix(es)

- bpo-38405 [https://bugs.python.org/issue?@action=redirect&bpo=38405]: Nested subclasses of `typing.NamedTuple` are now pickleable.

- bpo-38332 [https://bugs.python.org/issue?@action=redirect&bpo=38332]: Prevent `KeyError` thrown by `_encoded_words.decode()` when given an encoded-word with invalid content-type encoding from propagating all the way to `email.message.get()`.

- bpo-38371 [https://bugs.python.org/issue?@action=redirect&bpo=38371]: Deprecated the `split()` method in `_tkinter.TkappType` in favour of the `splitlist()` method which has more consistent and predictable behavior.

- bpo-38341 [https://bugs.python.org/issue?@action=redirect&bpo=38341]: Add `smtplib.SMTPNotSupportedError` to the `smtplib` exported names.

- bpo-38319 [https://bugs.python.org/issue?@action=redirect&bpo=38319]: sendfile() used in socket and shutil modules was raising OverflowError for files

>= 2GiB on 32-bit architectures. (patch by Giampaolo Rodola)

- bpo-38242 [https://bugs.python.org/issue?@action=redirect&bpo=38242]: Revert the new asyncio Streams API

- bpo-13153 [https://bugs.python.org/issue?@action=redirect&bpo=13153]: OS native encoding is now used for converting between Python strings and Tcl objects. This allows to display, copy and paste to clipboard emoji and other non-BMP characters. Converting strings from Tcl to Python and back now never fails (except MemoryError).

- bpo-38019 [https://bugs.python.org/issue?@action=redirect&bpo=38019]: Correctly handle pause/resume reading of closed asyncio unix pipe.

- bpo-38163 [https://bugs.python.org/issue?@action=redirect&bpo=38163]: Child mocks will now detect their type as either synchronous or asynchronous, asynchronous child mocks will be AsyncMocks and synchronous child mocks will be either MagicMock or Mock (depending on their parent type).

- bpo-38161 [https://bugs.python.org/issue?@action=redirect&bpo=38161]: Removes _AwaitEvent from AsyncMock.

- bpo-38216 [https://bugs.python.org/issue?@action=redirect&bpo=38216]: Allow the rare code that wants to send invalid http requests from the `http.client` library a way to do so. The fixes for bpo-30458 [https://bugs.python.org/issue?@action=redirect&bpo=30458] led to breakage for some projects that were relying on this ability to test their own behavior in the face of bad requests.

- bpo-28286 [https://bugs.python.org/issue?@action=redirect&bpo=28286]: Deprecate opening **`GzipFile`** for writing implicitly. Always specify the *mode* argument for writing.

- bpo-38108 [https://bugs.python.org/issue?@action=redirect&bpo=38108]: Any synchronous magic methods on an AsyncMock now return a MagicMock. Any asynchronous magic methods on a MagicMock now return an AsyncMock.

- bpo-38265 [https://bugs.python.org/issue?@action=redirect&bpo=38265]: Update the *length* parameter of `os.pread()` to accept `Py_ssize_t` instead of int.

- bpo-38112 [https://bugs.python.org/issue?@action=redirect&bpo=38112]: `compileall` has a higher default recursion limit and new command-line arguments for path manipulation, symlinks handling, and multiple optimization levels.

- bpo-38248 [https://bugs.python.org/issue?@action=redirect&bpo=38248]: asyncio: Fix inconsistent immediate Task cancellation

- bpo-38237 [https://bugs.python.org/issue?@action=redirect&bpo=38237]: The arguments for the builtin pow function are more descriptive. They can now also be passed in as keywords.

- bpo-34002 [https://bugs.python.org/issue?@action=redirect&bpo=34002]: Improve efficiency in parts of email package by changing while-pop to a for loop, using isdisjoint instead of set intersections.

- bpo-38191 [https://bugs.python.org/issue?@action=redirect&bpo=38191]: Constructors of `NamedTuple` and `TypedDict` types now accept arbitrary keyword argument names, including “cls”, “self”, “typename”, “_typename”, “fields” and “_fields”.

- bpo-38155 [https://bugs.python.org/issue?@action=redirect&bpo=38155]: Add `__all__` to `datetime`. Patch by Tahia Khan.

- bpo-38185 [https://bugs.python.org/issue?@action=redirect&bpo=38185]: Fixed case-insensitive string comparison in `sqlite3.Row` indexing.

- bpo-38136 [https://bugs.python.org/issue?@action=redirect&bpo=38136]: Changes AsyncMock call count and await count to be two different counters. Now await count only counts when a coroutine has been awaited, not when it has been called, and vice-versa. Update the documentation around this.

- bpo-37828 [https://bugs.python.org/issue?@action=redirect&bpo=37828]: Fix default mock name in `unittest.mock.Mock.assert_called()` exceptions.

Patch by Abraham Toriz Cruz.

- bpo-38175 [https://bugs.python.org/issue?@action=redirect&bpo=38175]: Fix a memory leak in comparison of `sqlite3.Row` objects.

- bpo-33936 [https://bugs.python.org/issue?@action=redirect&bpo=33936]: _hashlib no longer calls obsolete OpenSSL initialization function with OpenSSL 1.1.0+.

- bpo-34706 [https://bugs.python.org/issue?@action=redirect&bpo=34706]: Preserve subclassing in inspect.Signature.from_callable.

- bpo-38153 [https://bugs.python.org/issue?@action=redirect&bpo=38153]: Names of hashing algorithms from OpenSSL are now normalized to follow Python's naming conventions. For example OpenSSL uses sha3-512 instead of sha3_512 or blake2b512 instead of blake2b.

- bpo-38115 [https://bugs.python.org/issue?@action=redirect&bpo=38115]: Fix a bug in dis.findlinestarts() where it would return invalid bytecode offsets. Document that a code object's co_lnotab can contain invalid bytecode offsets.

- bpo-38148 [https://bugs.python.org/issue?@action=redirect&bpo=38148]: Add slots to `asyncio` transport classes, which can reduce memory usage.

- bpo-38142 [https://bugs.python.org/issue?@action=redirect&bpo=38142]: The _hashlib OpenSSL wrapper extension module is now PEP-384 compliant.

- bpo-9216 [https://bugs.python.org/issue?@action=redirect&bpo=9216]: hashlib constructors now support usedforsecurity flag to signal that a hashing algorithm is not used in a security context.

- bpo-36991 [https://bugs.python.org/issue?@action=redirect&bpo=36991]: Fixes a potential incorrect AttributeError exception escaping ZipFile.extract() in some unsupported input error situations.

- bpo-38134 [https://bugs.python.org/issue?@action=redirect&bpo=38134]: Remove obsolete copy of PBKDF2_HMAC_fast. All supported OpenSSL versions contain a fast implementation.

- bpo-38132 [https://bugs.python.org/issue?@action=redirect&bpo=38132]: The OpenSSL hashlib wrapper uses a simpler implementation. Several Macros and pointless caches are gone. The hash name now comes from OpenSSL's EVP. The algorithm name stays the same, except it is now always lower case.

- bpo-38008 [https://bugs.python.org/issue?@action=redirect&bpo=38008]: Fix parent class check in protocols to correctly identify the module that provides a builtin protocol, instead of assuming they all come from the `collections.abc` module

- bpo-34037 [https://bugs.python.org/issue?@action=redirect&bpo=34037]: For `asyncio`, add a new coroutine `loop.shutdown_default_executor()`. The new coroutine provides an API to schedule an executor shutdown that waits on the threadpool to finish closing. Also, `asyncio.run()` has been updated to utilize the new coroutine. Patch by Kyle Stanley.

- bpo-37405 [https://bugs.python.org/issue?@action=redirect&bpo=37405]: Fixed regression bug for socket.getsockname() for non-CAN_ISOTP AF_CAN address family sockets by returning a 1-tuple instead of string.

- bpo-38121 [https://bugs.python.org/issue?@action=redirect&bpo=38121]: Update parameter names on functions in importlib.metadata matching the changes in the 0.22 release of importlib_metadata.

- bpo-38110 [https://bugs.python.org/issue?@action=redirect&bpo=38110]: The os.closewalk() implementation now uses the libc fdwalk() API on platforms where it is available.

- bpo-38093 [https://bugs.python.org/issue?@action=redirect&bpo=38093]: Fixes AsyncMock so it doesn't crash when used with AsyncContextManagers or AsyncIterators.

- bpo-37488 [https://bugs.python.org/issue?@action=redirect&bpo=37488]: Add warning to **`datetime.utctimetuple()`**, **`datetime.utcnow()`** and **`datetime.utcfromtimestamp()`** .

- bpo-35640 [https://bugs.python.org/issue?@action=redirect&bpo=35640]: Allow passing a path-like object as `directory` argument to the **`http.server.SimpleHTTPRequestHandler`** class. Patch by Géry Ogam.

- bpo-38086 [https://bugs.python.org/issue?@action=redirect&bpo=38086]: Update importlib.metadata with changes from importlib_metadata 0.21 [https://gitlab.com/python-devs/importlib_metadata/blob/0.21/importlib_metadata/docs/changelog.rst].

- bpo-37251 [https://bugs.python.org/issue?@action=redirect&bpo=37251]: Remove `__code__` check in AsyncMock that incorrectly evaluated function specs as async objects but failed to evaluate classes with `__await__` but no `__code__` attribute defined as async objects.

- bpo-38037 [https://bugs.python.org/issue?@action=redirect&bpo=38037]: Fix reference counters in the **`signal`** module.

- bpo-38066 [https://bugs.python.org/issue?@action=redirect&bpo=38066]: Hide internal asyncio.Stream methods: feed_eof(), feed_data(), set_exception() and set_transport().

- bpo-38059 [https://bugs.python.org/issue?@action=redirect&bpo=38059]: inspect.py now uses sys.exit() instead of exit()

- bpo-38049 [https://bugs.python.org/issue?@action=redirect&bpo=38049]: Added command-line interface for the **`ast`** module.

- bpo-37953 [https://bugs.python.org/issue?@action=redirect&bpo=37953]: In **`typing`**, improved the `__hash__` and `__eq__` methods for **`ForwardReferences`**.

- bpo-38026 [https://bugs.python.org/issue?@action=redirect&bpo=38026]: Fixed **`inspect.getattr_static()`** used `isinstance` while it should avoid dynamic lookup.

- bpo-35923 [https://bugs.python.org/issue?@action=redirect&bpo=35923]: Update **`importlib.machinery.BuiltinImporter`** to use `loader._ORIGIN` instead of a hardcoded value. Patch by Donghee Na.

- bpo-38010 [https://bugs.python.org/issue?@action=redirect&bpo=38010]: In `importlib.metadata` sync with `importlib_metadata` 0.20, clarifying behavior of `files()` and fixing issue where only one requirement was returned for `requires()` on `dist-info` packages.

- bpo-38006 [https://bugs.python.org/issue?@action=redirect&bpo=38006]: weakref.WeakValueDictionary defines a local remove() function used as callback for weak references. This function was created with a closure. Modify the implementation to avoid the closure.

- bpo-37995 [https://bugs.python.org/issue?@action=redirect&bpo=37995]: Added the *indent* option to **`ast.dump()`** which allows it to produce a multiline indented output.

- bpo-34410 [https://bugs.python.org/issue?@action=redirect&bpo=34410]: Fixed a crash in the **`tee()`** iterator when re-enter it. RuntimeError is now raised in this case.

- bpo-37140 [https://bugs.python.org/issue?@action=redirect&bpo=37140]: Fix a ctypes regression of Python 3.8. When a ctypes.Structure is passed by copy to a function, ctypes internals created a temporary object which had the side effect of calling the structure finalizer (__del__) twice. The Python semantics requires a finalizer to be called exactly once. Fix ctypes internals to no longer call the finalizer twice.

- bpo-37587 [https://bugs.python.org/issue?@action=redirect&bpo=37587]: `_json.scanstring` is now up to 3x faster when there are many backslash escaped characters in the JSON string.

- bpo-37834 [https://bugs.python.org/issue?@action=redirect&bpo=37834]: Prevent shutil.rmtree exception when built on non-Windows system without fd system call support, like older versions of macOS.

- bpo-10978 [https://bugs.python.org/issue?@action=redirect&bpo=10978]: Semaphores and BoundedSemaphores can now release more than one waiting thread at a time.

- bpo-37972 [https://bugs.python.org/issue?@action=redirect&bpo=37972]: Subscripts to the `unittest.mock.call` objects now receive the same chaining mechanism as any other custom attributes, so that the following usage no longer raises a `TypeError`:

  ```
  call().foo().__getitem__('bar')
  ```

  Patch by blhsing

- bpo-37965 [https://bugs.python.org/issue?@action=redirect&bpo=37965]: Fix C compiler warning caused by distutils.ccompiler.CCompiler.has_function.

- bpo-37964 [https://bugs.python.org/issue?@action=redirect&bpo=37964]: Add `F_GETPATH` command to **`fcntl`**.

- bpo-37960 [https://bugs.python.org/issue?@action=redirect&bpo=37960]: `repr()` of buffered and text streams now silences only expected exceptions when get the value of “name” and “mode” attributes.

- bpo-37961 [https://bugs.python.org/issue?@action=redirect&bpo=37961]: Add a `total_nframe` field to the traces collected by the tracemalloc module. This field indicates the original number of frames before it was truncated.

- bpo-37951 [https://bugs.python.org/issue?@action=redirect&bpo=37951]: Most features of the subprocess module now work again in subinterpreters. Only *preexec_fn* is restricted in subinterpreters.

- bpo-36205 [https://bugs.python.org/issue?@action=redirect&bpo=36205]: Fix the rusage implementation of time.process_time() to correctly report the sum of the system and user CPU time.

- bpo-37950 [https://bugs.python.org/issue?@action=redirect&bpo=37950]: Fix **`ast.dump()`** when call with incompletely initialized node.

- bpo-34679 [https://bugs.python.org/issue?@action=redirect&bpo=34679]: Restores instantiation of Windows IOCP event loops from the non-main thread.

- bpo-36917 [https://bugs.python.org/issue?@action=redirect&bpo=36917]: Add default implementation of the **`ast.NodeVisitor.visit_Constant()`** method which emits a deprecation warning and calls corresponding methods `visit_Num()`, `visit_Str()`, etc.

- bpo-37798 [https://bugs.python.org/issue?@action=redirect&bpo=37798]: Update test_statistics.py to verify that the statistics module works well for both C and Python implementations. Patch by Donghee Na

- bpo-26589 [https://bugs.python.org/issue?@action=redirect&bpo=26589]: Added a new status code to the http module: 451 UNAVAILABLE_FOR_LEGAL_REASONS

- bpo-37915 [https://bugs.python.org/issue?@action=redirect&bpo=37915]: Fix a segmentation fault that appeared when comparing instances of `datetime.timezone` and `datetime.tzinfo` objects. Patch by Pablo Galindo.

- bpo-32554 [https://bugs.python.org/issue?@action=redirect&bpo=32554]: Deprecate having random.seed() call hash on arbitrary types.

- bpo-9938 [https://bugs.python.org/issue?@action=redirect&bpo=9938]: Add optional keyword argument `exit_on_error` for **`ArgumentParser`**.

- bpo-37851 [https://bugs.python.org/issue?@action=redirect&bpo=37851]: The **`faulthandler`** module no longer allocates its alternative stack at Python startup. Now the stack is only allocated at the first faulthandler usage.

- bpo-32793 [https://bugs.python.org/issue?@action=redirect&bpo=32793]: Fix a duplicated debug message when **`smtplib.SMTP.connect()`** is called.

- bpo-37885 [https://bugs.python.org/issue?@action=redirect&bpo=37885]: venv: Don't generate unset variable warning on deactivate.

- bpo-37868 [https://bugs.python.org/issue?@action=redirect&bpo=37868]: Fix dataclasses.is_dataclass when given an instance that never raises AttributeError in __getattr__. That is, an object that returns something for __dataclass_fields__ even if it's not a dataclass.

- bpo-37811 [https://bugs.python.org/issue?@action=redirect&bpo=37811]: Fix `socket` module's `socket.connect(address)` function being unable to establish connection in case of interrupted system call. The problem was observed on all OSes which `poll(2)` system call can take only non-negative integers and -1 as a timeout value.

- bpo-37863 [https://bugs.python.org/issue?@action=redirect&bpo=37863]: Optimizations for Fraction.__hash__ suggested by Tim Peters.

- bpo-21131 [https://bugs.python.org/issue?@action=redirect&bpo=21131]: Fix `faulthandler.register(chain=True)` stack. faulthandler now allocates a dedicated stack of `SIGSTKSZ*2` bytes, instead of just `SIGSTKSZ` bytes. Calling the previous signal handler in faulthandler signal handler uses more than `SIGSTKSZ` bytes of stack memory on some platforms.

- bpo-37798 [https://bugs.python.org/issue?@action=redirect&bpo=37798]: Add C fastpath for statistics.NormalDist.inv_cdf() Patch by Donghee Na

- bpo-37804 [https://bugs.python.org/issue?@action=redirect&bpo=37804]: Remove the deprecated method `threading.Thread.isAlive()`. Patch by Donghee Na.

- bpo-37819 [https://bugs.python.org/issue?@action=redirect&bpo=37819]: Add Fraction.as_integer_ratio() to match the corresponding methods in bool, int, float, and decimal.

- bpo-14465 [https://bugs.python.org/issue?@action=redirect&bpo=14465]: Add an xml.etree.ElementTree.indent() function for pretty-printing XML trees. Contributed by Stefan Behnel.

- bpo-37810 [https://bugs.python.org/issue?@action=redirect&bpo=37810]: Fix **`difflib`** `?` hint in diff output when dealing with tabs. Patch by Anthony Sottile.

- bpo-37772 [https://bugs.python.org/issue?@action=redirect&bpo=37772]: In `zipfile.Path`, when adding implicit dirs, ensure that ancestral directories are added and that duplicates are excluded.

- bpo-18578 [https://bugs.python.org/issue?@action=redirect&bpo=18578]: Renamed and documented `test.bytecode_helper` as `test.support.bytecode_helper`. Patch by Joannah Nanjekye.

- bpo-37785 [https://bugs.python.org/issue?@action=redirect&bpo=37785]: Fix xgettext warnings in **`argparse`**.

- bpo-34488 [https://bugs.python.org/issue?@action=redirect&bpo=34488]: **`writelines()`** method of **`io.BytesIO`** is now slightly faster when many small lines are passed. Patch by Sergey Fedoseev.

- bpo-37449 [https://bugs.python.org/issue?@action=redirect&bpo=37449]: `ensurepip` now uses `importlib.resources.read_binary()` to read data instead of `pkgutil.get_data()`. Patch by Joannah Nanjekye.

- bpo-28292 [https://bugs.python.org/issue?@action=redirect&bpo=28292]: Mark calendar.py helper functions as being private. The follows PEP 8 guidance to maintain the style conventions in the module and it addresses a known case of user confusion.

- bpo-18049 [https://bugs.python.org/issue?@action=redirect&bpo=18049]: Add definition of THREAD_STACK_SIZE for AIX in Python/thread_pthread.h The default thread stacksize caused crashes with the default recursion limit Patch by M Felt

- bpo-37742 [https://bugs.python.org/issue?@action=redirect&bpo=37742]: The logging.getLogger() API now returns the root logger when passed the name ‘root’, whereas previously it returned a non-root logger named ‘root’. This could affect cases where user code explicitly wants a non-root logger named ‘root’, or instantiates a logger using

logging.getLogger(__name__) in some top-level module called 'root.py'.

- bpo-37738 [https://bugs.python.org/issue?@action=redirect&bpo=37738]: Fix the implementation of curses `addch(str, color_pair)`: pass the color pair to `setcchar()`, instead of always passing 0 as the color pair.

- bpo-37723 [https://bugs.python.org/issue?@action=redirect&bpo=37723]: Fix performance regression on regular expression parsing with huge character sets. Patch by Yann Vaginay.

- bpo-35943 [https://bugs.python.org/issue?@action=redirect&bpo=35943]: The function **`PyImport_GetModule()`** now ensures any module it returns is fully initialized. Patch by Joannah Nanjekye.

- bpo-32178 [https://bugs.python.org/issue?@action=redirect&bpo=32178]: Fix IndexError in **`email`** package when trying to parse invalid address fields starting with `:`.

- bpo-37268 [https://bugs.python.org/issue?@action=redirect&bpo=37268]: The **`parser`** module is deprecated and will be removed in future versions of Python.

- bpo-11953 [https://bugs.python.org/issue?@action=redirect&bpo=11953]: Completing WSA* error codes in **`socket`**.

- bpo-37685 [https://bugs.python.org/issue?@action=redirect&bpo=37685]: Fixed comparisons of **`datetime.timedelta`** and **`datetime.timezone`**.

- bpo-37697 [https://bugs.python.org/issue?@action=redirect&bpo=37697]: Synchronize `importlib.metadata` with importlib_metadata 0.19 [https://gitlab.com/python-devs/importlib_metadata/-/milestones/20], improving handling of EGG-INFO files and fixing a crash when entry point names contained colons.

- bpo-37695 [https://bugs.python.org/issue?@action=redirect&bpo=37695]: Correct **`curses.unget_wch()`** error message. Patch by Anthony Sottile.

- bpo-37689 [https://bugs.python.org/issue?@action=redirect&bpo=37689]: Add **`is_relative_to()`** in **`PurePath`** to determine whether or not one path is relative to another.

- bpo-29553 [https://bugs.python.org/issue?@action=redirect&bpo=29553]: Fixed **`argparse.ArgumentParser.format_usage()`** for mutually exclusive groups. Patch by Andrew Nester.

- bpo-37691 [https://bugs.python.org/issue?@action=redirect&bpo=37691]: Let math.dist() accept coordinates as sequences (or iterables) rather than just tuples.

- bpo-37685 [https://bugs.python.org/issue?@action=redirect&bpo=37685]: Fixed `__eq__`, `__lt__` etc implementations in some classes. They now return **`NotImplemented`** for unsupported type of the other operand. This allows the other operand to play role (for example the equality comparison with **`ANY`** will return `True`).

- bpo-37354 [https://bugs.python.org/issue?@action=redirect&bpo=37354]: Make Activate.ps1 Powershell script static to allow for signing it.

- bpo-37664 [https://bugs.python.org/issue?@action=redirect&bpo=37664]: Update wheels bundled with ensurepip (pip 19.2.3 and setuptools 41.2.0)

- bpo-37663 [https://bugs.python.org/issue?@action=redirect&bpo=37663]: Bring consistency to venv shell activation scripts by always using __VENV_PROMPT__.

- bpo-37642 [https://bugs.python.org/issue?@action=redirect&bpo=37642]: Allowed the pure Python implementation of **`datetime.timezone`** to represent sub-minute offsets close to minimum and maximum boundaries, specifically in the ranges (23:59, 24:00) and (-23:59, 24:00). Patch by Ngalim Siregar

- bpo-36161 [https://bugs.python.org/issue?@action=redirect&bpo=36161]: In **`posix`**, use `ttyname_r` instead of `ttyname` for thread safety.

- bpo-36324 [https://bugs.python.org/issue?@action=redirect&bpo=36324]: Make internal attributes for statistics.NormalDist() private.

- bpo-37555 [https://bugs.python.org/issue?@action=redirect&bpo=37555]: Fix `NonCallableMock._call_matcher` returning tuple instead of `_Call` object when `self._spec_signature` exists. Patch by Elizabeth Uselton

- bpo-29446 [https://bugs.python.org/issue?@action=redirect&bpo=29446]: Make `from tkinter import *` import only the expected objects.

- bpo-16970 [https://bugs.python.org/issue?@action=redirect&bpo=16970]: Adding a value error when an invalid value in passed to nargs Patch by Robert Leenders

- bpo-34443 [https://bugs.python.org/issue?@action=redirect&bpo=34443]: Exceptions from **`enum`** now use the `__qualname` of the enum class in the exception message instead of the `__name__`.

- bpo-37491 [https://bugs.python.org/issue?@action=redirect&bpo=37491]: Fix `IndexError` when parsing email headers with unexpectedly ending bare-quoted string value. Patch by Abhilash Raj.

- bpo-37587 [https://bugs.python.org/issue?@action=redirect&bpo=37587]: Make json.loads faster for long strings. (Patch by Marco Paolini)

- bpo-18378 [https://bugs.python.org/issue?@action=redirect&bpo=18378]: Recognize “UTF-8” as a valid value for LC_CTYPE in locale._parse_localename.

- bpo-37579 [https://bugs.python.org/issue?@action=redirect&bpo=37579]: Return **`NotImplemented`** in Python implementation of `__eq__` for **`timedelta`** and **`time`** when the other object being compared is not of the same type to match C implementation. Patch by Karthikeyan Singaravelan.

- bpo-21478 [https://bugs.python.org/issue?@action=redirect&bpo=21478]: Record calls to parent when autospecced object is attached to a mock using **`unittest.mock.attach_mock()`**. Patch by Karthikeyan Singaravelan.

- bpo-37531 [https://bugs.python.org/issue?@action=redirect&bpo=37531]: “python3 -m test -jN –timeout=TIMEOUT” now kills a worker process if it runs longer than *TIMEOUT* seconds.

- bpo-37482 [https://bugs.python.org/issue?@action=redirect&bpo=37482]: Fix serialization of display name in originator or destination address fields with both encoded words and special chars.

- bpo-36993 [https://bugs.python.org/issue?@action=redirect&bpo=36993]: Improve error reporting for corrupt zip files with bad zip64 extra data. Patch by Daniel Hillier.

- bpo-37502 [https://bugs.python.org/issue?@action=redirect&bpo=37502]: pickle.loads() no longer raises TypeError when the buffers argument is set to None

- bpo-37520 [https://bugs.python.org/issue?@action=redirect&bpo=37520]: Correct behavior for zipfile.Path.parent when the path object identifies a subdirectory.

- bpo-18374 [https://bugs.python.org/issue?@action=redirect&bpo=18374]: Fix the `.col_offset` attribute of nested **`ast.BinOp`** instances which had a too large value in some situations.

- bpo-37424 [https://bugs.python.org/issue?@action=redirect&bpo=37424]: Fixes a possible hang when using a timeout on `subprocess.run()` while capturing output. If the child process spawned its own children or otherwise connected its stdout or stderr handles with another process, we could hang after the timeout was reached and our child was killed when attempting to read final output from the pipes.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: Fix **`multiprocessing.util.get_temp_dir()`** finalizer: clear also the ‘tempdir’ configuration of the current process, so next call to `get_temp_dir()` will create a new temporary directory, rather than reusing the removed temporary directory.

- bpo-37481 [https://bugs.python.org/issue?@action=redirect&bpo=37481]: The distutils `bdist_wininst` command is deprecated in Python 3.8, use `bdist_wheel` (wheel packages) instead.

- bpo-37479 [https://bugs.python.org/issue?@action=redirect&bpo=37479]: When `Enum.__str__` is overridden in a derived class, the override will be used by `Enum.__format__` regardless of whether mixin classes are present.

- bpo-37440 [https://bugs.python.org/issue?@action=redirect&bpo=37440]: http.client now enables TLS 1.3 post-handshake authentication for default context or if a cert_file is passed to HTTPSConnection.

- bpo-37437 [https://bugs.python.org/issue?@action=redirect&bpo=37437]: Update vendorized expat version to 2.2.7.

- bpo-37428 [https://bugs.python.org/issue?@action=redirect&bpo=37428]: SSLContext.post_handshake_auth = True no longer sets SSL_VERIFY_POST_HANDSHAKE verify flag for client connections. Although the option is documented as ignored for clients, OpenSSL implicitly enables cert chain validation when the flag is set.

- bpo-37420 [https://bugs.python.org/issue?@action=redirect&bpo=37420]: **`os.sched_setaffinity()`** now correctly handles errors that arise during iteration over its `mask` argument. Patch by Brandt Bucher.

- bpo-37412 [https://bugs.python.org/issue?@action=redirect&bpo=37412]: The **`os.getcwdb()`** function now uses the UTF-8 encoding on Windows, rather than the ANSI code page: see **PEP 529** [https://peps.python.org/pep-0529/] for the rationale. The function is no longer deprecated on Windows.

- bpo-37406 [https://bugs.python.org/issue?@action=redirect&bpo=37406]: The sqlite3 module now raises TypeError, rather than ValueError, if operation argument type is not str: execute(), executemany() and calling a connection.

- bpo-29412 [https://bugs.python.org/issue?@action=redirect&bpo=29412]: Fix IndexError in parsing a header value ending unexpectedly. Patch by Abhilash Raj.

- bpo-36546 [https://bugs.python.org/issue?@action=redirect&bpo=36546]: The *dist* argument for statistics.quantiles() is now positional only. The current name doesn't reflect that the argument can be either a dataset or a distribution. Marking the parameter as positional avoids confusion and makes it possible to change the name later.

- bpo-37394 [https://bugs.python.org/issue?@action=redirect&bpo=37394]: Fix a bug that was causing the `queue` module to fail if the accelerator module was not available. Patch by Pablo Galindo.

- bpo-37376 [https://bugs.python.org/issue?@action=redirect&bpo=37376]: `pprint` now has support for `types.SimpleNamespace`. Patch by Carl Bordum Hansen.

- bpo-26967 [https://bugs.python.org/issue?@action=redirect&bpo=26967]: An `ArgumentParser` with `allow_abbrev=False` no longer disables grouping of short flags, such as `-vv`, but only disables abbreviation of long flags as documented. Patch by Zac Hatfield-Dodds.

- bpo-37212 [https://bugs.python.org/issue?@action=redirect&bpo=37212]: `unittest.mock.call()` now preserves the order of keyword arguments in repr output. Patch by Karthikeyan Singaravelan.

- bpo-37372 [https://bugs.python.org/issue?@action=redirect&bpo=37372]: Fix error unpickling datetime.time objects from Python 2 with seconds>=24. Patch by Justin Blanchard.

- bpo-37345 [https://bugs.python.org/issue?@action=redirect&bpo=37345]: Add formal support for UDPLITE sockets. Support was present before, but it is now easier to detect support with `hasattr(socket, 'IPPROTO_UDPLITE')` and there are constants defined for each of the values needed: `socket.IPPROTO_UDPLITE`, `UDPLITE_SEND_CSCOV`, and `UDPLITE_RECV_CSCOV`. Patch by Gabe Appleton.

- bpo-37358 [https://bugs.python.org/issue?@action=redirect&bpo=37358]: Optimized `functools.partial` by using vectorcall.

- bpo-37347 [https://bugs.python.org/issue?@action=redirect&bpo=37347]: **`sqlite3.Connection.create_aggregate()`**, **`sqlite3.Connection.create_function()`**, **`sqlite3.Connection.set_authorizer()`**, **`sqlite3.Connection.set_progress_handler()`** **`sqlite3.Connection.set_trace_callback()`** methods lead to segfaults if some of these methods are called twice with an equal object but not the same. Now callbacks are stored more carefully. Patch by Aleksandr Balezin.

- bpo-37163 [https://bugs.python.org/issue?@action=redirect&bpo=37163]: The *obj* argument of **`dataclasses.replace()`** is positional-only now.

- bpo-37085 [https://bugs.python.org/issue?@action=redirect&bpo=37085]: Add the optional Linux SocketCAN Broadcast Manager constants, used as flags to configure the BCM behaviour, in the socket module. Patch by Karl Ding.

- bpo-37328 [https://bugs.python.org/issue?@action=redirect&bpo=37328]: `HTMLParser.unescape` is removed. It was undocumented and deprecated since Python 3.4.

- bpo-37305 [https://bugs.python.org/issue?@action=redirect&bpo=37305]: Add .webmanifest -> application/manifest+json to list of recognized file types and content type headers

- bpo-37320 [https://bugs.python.org/issue?@action=redirect&bpo=37320]: `aifc.openfp()` alias to `aifc.open()`, `sunau.openfp()` alias to `sunau.open()`, and `wave.openfp()` alias to `wave.open()` have been removed. They were deprecated since Python 3.7.

- bpo-37315 [https://bugs.python.org/issue?@action=redirect&bpo=37315]: Deprecated accepting floats with integral value (like `5.0`) in **`math.factorial()`**.

- bpo-37312 [https://bugs.python.org/issue?@action=redirect&bpo=37312]: `_dummy_thread` and `dummy_threading` modules have been removed. These modules were deprecated since Python 3.7 which requires threading support.

- bpo-33972 [https://bugs.python.org/issue?@action=redirect&bpo=33972]: Email with single part but content-type set to `multipart/*` doesn't raise AttributeError anymore.

- bpo-37280 [https://bugs.python.org/issue?@action=redirect&bpo=37280]: Use threadpool for reading from file for sendfile fallback mode.

- bpo-37279 [https://bugs.python.org/issue?@action=redirect&bpo=37279]: Fix asyncio sendfile support when sendfile sends extra data in fallback mode.

- bpo-19865 [https://bugs.python.org/issue?@action=redirect&bpo=19865]: **`ctypes.create_unicode_buffer()`** now also supports non-BMP characters on platforms with 16-bit **`wchar_t`** (for example, Windows and AIX).

- bpo-37266 [https://bugs.python.org/issue?@action=redirect&bpo=37266]: In a subinterpreter, spawning a daemon thread now raises an exception. Daemon threads were never supported in subinterpreters. Previously, the subinterpreter finalization crashed with a Python fatal error if a daemon thread was still running.

- bpo-37210 [https://bugs.python.org/issue?@action=redirect&bpo=37210]: Allow pure Python implementation of **`pickle`** to work even when the C **`_pickle`** module is unavailable.

- bpo-21872 [https://bugs.python.org/issue?@action=redirect&bpo=21872]: Fix **`lzma`**: module decompresses data incompletely. When decompressing a FORMAT_ALONE format file, and it doesn't have the end marker, sometimes the last one to dozens bytes can't be output. Patch by Ma Lin.

- bpo-35922 [https://bugs.python.org/issue?@action=redirect&bpo=35922]: Fix **`RobotFileParser.crawl_delay()`** and

`RobotFileParser.request_rate()` to return `None` rather than raise **`AttributeError`** when no relevant rule is defined in the robots.txt file. Patch by Rémi Lapeyre.

- bpo-35766 [https://bugs.python.org/issue?@action=redirect&bpo=35766]: Change the format of feature_version to be a (major, minor) tuple.

- bpo-36607 [https://bugs.python.org/issue?@action=redirect&bpo=36607]: Eliminate **`RuntimeError`** raised by **`asyncio.all_tasks()`** if internal tasks weak set is changed by another thread during iteration.

- bpo-18748 [https://bugs.python.org/issue?@action=redirect&bpo=18748]: **`_pyio.IOBase`** destructor now does nothing if getting the `closed` attribute fails to better mimic **`_io.IOBase`** finalizer.

- bpo-36402 [https://bugs.python.org/issue?@action=redirect&bpo=36402]: Fix a race condition at Python shutdown when waiting for threads. Wait until the Python thread state of all non-daemon threads get deleted (join all non-daemon threads), rather than just wait until non-daemon Python threads complete.

- bpo-37206 [https://bugs.python.org/issue?@action=redirect&bpo=37206]: Default values which cannot be represented as Python objects no longer improperly represented as `None` in function signatures.

- bpo-37111 [https://bugs.python.org/issue?@action=redirect&bpo=37111]: Added `encoding` and `errors` keyword parameters to `logging.basicConfig`.

- bpo-12144 [https://bugs.python.org/issue?@action=redirect&bpo=12144]: Ensure cookies with `expires` attribute are handled in **`CookieJar.make_cookies()`**.

- bpo-34886 [https://bugs.python.org/issue?@action=redirect&bpo=34886]: Fix an unintended ValueError from **`subprocess.run()`** when checking for conflicting *input* and *stdin* or *capture_output* and *stdout* or *stderr* args when they were explicitly provided but with `None` values within a passed in `**kwargs` dict rather than as passed directly by name. Patch contributed by Rémi Lapeyre.

- bpo-37173 [https://bugs.python.org/issue?@action=redirect&bpo=37173]: The exception message for `inspect.getfile()` now correctly reports the passed class rather than the builtins module.

- bpo-37178 [https://bugs.python.org/issue?@action=redirect&bpo=37178]: Give math.perm() a one argument form that means the same as math.factorial().

- bpo-37178 [https://bugs.python.org/issue?@action=redirect&bpo=37178]: For math.perm(n, k), let k default to n, giving the same result as factorial.

- bpo-37165 [https://bugs.python.org/issue?@action=redirect&bpo=37165]: Converted _collections._count_elements to use the Argument Clinic.

- bpo-34767 [https://bugs.python.org/issue?@action=redirect&bpo=34767]: Do not always create a **`collections.deque`** in **`asyncio.Lock`**.

- bpo-37158 [https://bugs.python.org/issue?@action=redirect&bpo=37158]: Speed-up statistics.fmean() by switching from a function to a generator.

- bpo-34282 [https://bugs.python.org/issue?@action=redirect&bpo=34282]: Remove `Enum._convert` method, deprecated in 3.8.

- bpo-37150 [https://bugs.python.org/issue?@action=redirect&bpo=37150]: `argparse._ActionsContainer.add_argument` now throws error, if someone accidentally pass FileType class object instead of instance of FileType as `type` argument.

- bpo-28724 [https://bugs.python.org/issue?@action=redirect&bpo=28724]: The socket module now has the **`socket.send_fds()`** and **`socket.recv.fds()`** methods. Contributed by Joannah Nanjekye, Shinya Okano and Victor Stinner.

- bpo-35621 [https://bugs.python.org/issue?@action=redirect&bpo=35621]: Support running asyncio subprocesses when execution event loop in a thread on UNIX.

- bpo-36520 [https://bugs.python.org/issue?@action=redirect&bpo=36520]: Lengthy email headers with UTF-8 characters are now properly encoded when they are folded. Patch by Jeffrey Kintscher.

- bpo-30835 [https://bugs.python.org/issue?@action=redirect&bpo=30835]: Fixed a bug in email parsing where a message with invalid bytes in content-transfer-encoding of a multipart message can cause an AttributeError. Patch by Andrew Donnellan.

- bpo-31163 [https://bugs.python.org/issue?@action=redirect&bpo=31163]: pathlib.Path instance's rename and replace methods now return the new Path instance.

- bpo-25068 [https://bugs.python.org/issue?@action=redirect&bpo=25068]: **`urllib.request.ProxyHandler`** now lowercases the keys of the passed dictionary.

- bpo-26185 [https://bugs.python.org/issue?@action=redirect&bpo=26185]: Fix **`repr()`** on empty **`ZipInfo`** object. Patch by Mickaël Schoentgen.

- bpo-21315 [https://bugs.python.org/issue?@action=redirect&bpo=21315]: Email headers containing RFC2047 encoded words are parsed despite the missing whitespace, and a defect registered. Also missing trailing whitespace after encoded words is now registered as a defect.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Port test_datetime to VxWorks: skip zoneinfo tests on VxWorks

- bpo-35805 [https://bugs.python.org/issue?@action=redirect&bpo=35805]: Add parser for Message-ID header and add it to default HeaderRegistry. This should prevent folding of Message-ID using RFC 2048 encoded words.

- bpo-36871 [https://bugs.python.org/issue?@action=redirect&bpo=36871]: Ensure method signature is used instead of constructor signature of a class while asserting mock object against method calls. Patch by Karthikeyan Singaravelan.

- bpo-35070 [https://bugs.python.org/issue?@action=redirect&bpo=35070]: posix.getgrouplist() now works correctly when the user belongs to NGROUPS_MAX supplemental groups. Patch by Jeffrey Kintscher.

- bpo-31783 [https://bugs.python.org/issue?@action=redirect&bpo=31783]: Fix race condition in ThreadPoolExecutor when worker threads are created during interpreter shutdown.

- bpo-36582 [https://bugs.python.org/issue?@action=redirect&bpo=36582]: Fix `UserString.encode()` to correctly return `bytes` rather than a `UserString` instance.

- bpo-32424 [https://bugs.python.org/issue?@action=redirect&bpo=32424]: Deprecate xml.etree.ElementTree.Element.copy() in favor of copy.copy().

  Patch by Gordon P. Hemsley

- bpo-36564 [https://bugs.python.org/issue?@action=redirect&bpo=36564]: Fix infinite loop in email header folding logic that would be triggered when an email policy's max_line_length is not long enough to include the required markup and any values in the message. Patch by Paul Ganssle

- bpo-36543 [https://bugs.python.org/issue?@action=redirect&bpo=36543]: Removed methods Element.getchildren(), Element.getiterator() and ElementTree.getiterator() and the xml.etree.cElementTree module.

- bpo-36409 [https://bugs.python.org/issue?@action=redirect&bpo=36409]: Remove the old plistlib API deprecated in Python 3.4

- bpo-36302 [https://bugs.python.org/issue?@action=redirect&bpo=36302]: distutils sorts source file lists so that Extension .so files build more reproducibly by default

- bpo-36250 [https://bugs.python.org/issue?@action=redirect&bpo=36250]: Ignore `ValueError` from `signal` with `interaction` in non-main thread.

- bpo-36046 [https://bugs.python.org/issue?@action=redirect&bpo=36046]: Added `user`, `group` and `extra_groups` parameters to the subprocess.Popen

constructor. Patch by Patrick McLean.

- bpo-32627 [https://bugs.python.org/issue?@action=redirect&bpo=32627]: Fix compile error when `_uuid` headers conflicting included.

- bpo-35800 [https://bugs.python.org/issue?@action=redirect&bpo=35800]: Deprecate `smtpd.MailmanProxy` ready for future removal.

- bpo-35168 [https://bugs.python.org/issue?@action=redirect&bpo=35168]: **`shlex.shlex.punctuation_chars`** is now a read-only property.

- bpo-8538 [https://bugs.python.org/issue?@action=redirect&bpo=8538]: Add support for boolean actions like `--foo` and `--no-foo` to argparse. Patch contributed by Rémi Lapeyre.

- bpo-20504 [https://bugs.python.org/issue?@action=redirect&bpo=20504]: Fixes a bug in **`cgi`** module when a multipart/form-data request has no `Content-Length` header.

- bpo-25988 [https://bugs.python.org/issue?@action=redirect&bpo=25988]: The abstract base classes in **`collections.abc`** no longer are exposed in the regular **`collections`** module.

- bpo-11122 [https://bugs.python.org/issue?@action=redirect&bpo=11122]: Distutils won't check for rpmbuild in specified paths only.

- bpo-34775 [https://bugs.python.org/issue?@action=redirect&bpo=34775]: Division handling of PurePath now returns NotImplemented instead of raising a TypeError when passed something other than an instance of str or PurePath. Patch by Roger Aiudi.

- bpo-34749 [https://bugs.python.org/issue?@action=redirect&bpo=34749]: **`binascii.a2b_base64()`** is now up to 2 times faster. Patch by Sergey Fedoseev.

- bpo-34519 [https://bugs.python.org/issue?@action=redirect&bpo=34519]: Add additional aliases for HP Roman 8. Patch by Michael Osipov.

- bpo-28009 [https://bugs.python.org/issue?@action=redirect&bpo=28009]: Fix uuid.getnode() on platforms with '.' as MAC Addr delimiter as well fix for MAC Addr format that omits a leading 0 in MAC Addr values. Currently, AIX is the only know platform with these settings. Patch by Michael Felt.

- bpo-30618 [https://bugs.python.org/issue?@action=redirect&bpo=30618]: Add `readlink()`. Patch by Girts Folkmanis.

- bpo-32498 [https://bugs.python.org/issue?@action=redirect&bpo=32498]: Made `urllib.parse.unquote()` accept bytes in addition to strings. Patch by Stein Karlsen.

- bpo-33348 [https://bugs.python.org/issue?@action=redirect&bpo=33348]: lib2to3 now recognizes expressions after `*` and `**` like in `f(*[] or []).`

- bpo-32689 [https://bugs.python.org/issue?@action=redirect&bpo=32689]: Update `shutil.move()` function to allow for Path objects to be used as source argument. Patch by Emily Morehouse and Maxwell "5.13b" McKinnon.

- bpo-32820 [https://bugs.python.org/issue?@action=redirect&bpo=32820]: Added `__format__` to IPv4 and IPv6 classes. Always outputs a fully zero-padded string. Supports b/x/n modifiers (bin/hex/native format). Native format for IPv4 is bin, native format for IPv6 is hex. Also supports '#' and '_' modifiers.

- bpo-27657 [https://bugs.python.org/issue?@action=redirect&bpo=27657]: Fix urllib.parse.urlparse() with numeric paths. A string like "path:80" is no longer parsed as a path but as a scheme ("path") and a path ("80").

- bpo-4963 [https://bugs.python.org/issue?@action=redirect&bpo=4963]: Fixed non-deterministic behavior related to mimetypes extension mapping and module reinitialization.

Documentação

- bpo-21767 [https://bugs.python.org/issue?@action=redirect&bpo=21767]: Explicitly mention abc support in functools.singledispatch

- bpo-38816 [https://bugs.python.org/issue?@action=redirect&bpo=38816]: Provides more details about the interaction between `fork()` and CPython's runtime, focusing just on the C-API. This includes cautions about where `fork()` should and shouldn't be called.

- bpo-38351 [https://bugs.python.org/issue?@action=redirect&bpo=38351]: Modernize `email` examples from %-formatting to f-strings.

- bpo-38778 [https://bugs.python.org/issue?@action=redirect&bpo=38778]: Document the fact that `RuntimeError` is raised if `os.fork()` is called in a subinterpreter.

- bpo-38592 [https://bugs.python.org/issue?@action=redirect&bpo=38592]: Add Brazilian Portuguese to the language switcher at Python Documentation website.

- bpo-38294 [https://bugs.python.org/issue?@action=redirect&bpo=38294]: Add list of no-longer-escaped chars to re.escape documentation

- bpo-38053 [https://bugs.python.org/issue?@action=redirect&bpo=38053]: Modernized the plistlib documentation

- bpo-26868 [https://bugs.python.org/issue?@action=redirect&bpo=26868]: Fix example usage of `PyModule_AddObject()` to properly handle errors.

- bpo-36797 [https://bugs.python.org/issue?@action=redirect&bpo=36797]: Fix a dead link in the distutils API Reference.

- bpo-37977 [https://bugs.python.org/issue?@action=redirect&bpo=37977]: Warn more strongly and clearly about pickle insecurity

- bpo-37979 [https://bugs.python.org/issue?@action=redirect&bpo=37979]: Added a link to dateutil.parser.isoparse in the datetime.fromisoformat documentation. Patch by Paul Ganssle

- bpo-12707 [https://bugs.python.org/issue?@action=redirect&bpo=12707]: Deprecate info(), geturl(), getcode() methods in favor of the headers, url, and status properties, respectively, for HTTPResponse and addinfourl. Also deprecate the code attribute of addinfourl in favor of the status attribute. Patch by Ashwin Ramaswami

- bpo-37937 [https://bugs.python.org/issue?@action=redirect&bpo=37937]: Mention `frame.f_trace` in **`sys.settrace()`** docs.

- bpo-37878 [https://bugs.python.org/issue?@action=redirect&bpo=37878]: Make **`PyThreadState_DeleteCurrent()`** Internal.

- bpo-37759 [https://bugs.python.org/issue?@action=redirect&bpo=37759]: Beginning edits to Whatsnew 3.8

- bpo-37726 [https://bugs.python.org/issue?@action=redirect&bpo=37726]: Stop recommending getopt in the tutorial for command line argument parsing and promote argparse.

- bpo-32910 [https://bugs.python.org/issue?@action=redirect&bpo=32910]: Remove implementation-specific behaviour of how venv's Deactivate works.

- bpo-37256 [https://bugs.python.org/issue?@action=redirect&bpo=37256]: Fix wording of arguments for **`Request`** in **`urllib.request`**

- bpo-37284 [https://bugs.python.org/issue?@action=redirect&bpo=37284]: Add a brief note to indicate that any new `sys.implementation` required attributes must go through the PEP process.

- bpo-30088 [https://bugs.python.org/issue?@action=redirect&bpo=30088]: Documented that **`mailbox.Maildir`** constructor doesn't attempt to verify the maildir folder layout correctness. Patch by Sviatoslav Sydorenko.

- bpo-37521 [https://bugs.python.org/issue?@action=redirect&bpo=37521]: Fix `importlib` examples to insert any newly created modules via importlib.util.module_from_spec() immediately into sys.modules instead of after calling loader.exec_module().

Thanks to Benjamin Mintz for finding the bug.

- bpo-37456 [https://bugs.python.org/issue?@action=redirect&bpo=37456]: Slash ('/') is now part of syntax.

- bpo-37487 [https://bugs.python.org/issue?@action=redirect&bpo=37487]: Fix PyList_GetItem index description to include 0.

- bpo-37149 [https://bugs.python.org/issue?@action=redirect&bpo=37149]: Replace the dead link to the Tkinter 8.5 reference by John Shipman, New Mexico Tech, with a link to the archive.org copy.

- bpo-37478 [https://bugs.python.org/issue?@action=redirect&bpo=37478]: Added possible exceptions to the description of os.chdir().

- bpo-34903 [https://bugs.python.org/issue?@action=redirect&bpo=34903]: Documented that in `datetime.datetime.strptime()`, the leading zero in some two-digit formats is optional. Patch by Mike Gleen.

- bpo-36260 [https://bugs.python.org/issue?@action=redirect&bpo=36260]: Add decompression pitfalls to zipfile module documentation.

- bpo-37004 [https://bugs.python.org/issue?@action=redirect&bpo=37004]: In the documentation for difflib, a note was added explicitly warning that the results of SequenceMatcher's ratio method may depend on the order of the input strings.

- bpo-36960 [https://bugs.python.org/issue?@action=redirect&bpo=36960]: Restructured the `datetime` docs in the interest of making them more user-friendly and improving readability. Patch by Brad Solomon.

- bpo-36487 [https://bugs.python.org/issue?@action=redirect&bpo=36487]: Make C-API docs clear about what the "main" interpreter is.

- bpo-23460 [https://bugs.python.org/issue?@action=redirect&bpo=23460]: The documentation for decimal string formatting using the `:g` specifier has been updated to reflect the correct exponential notation cutoff point. Original patch contributed by Tuomas Suutari.

- bpo-35803 [https://bugs.python.org/issue?@action=redirect&bpo=35803]: Document and test that `tempfile` functions may accept a path-like object for the `dir` argument. Patch by Anthony Sottile.

- bpo-33944 [https://bugs.python.org/issue?@action=redirect&bpo=33944]: Added a note about the intended use of code in .pth files.

- bpo-34293 [https://bugs.python.org/issue?@action=redirect&bpo=34293]: Fix the Doc/Makefile regarding PAPER environment variable and PDF builds

- bpo-25237 [https://bugs.python.org/issue?@action=redirect&bpo=25237]: Add documentation for tkinter modules

Testes

- bpo-38614 [https://bugs.python.org/issue?@action=redirect&bpo=38614]: Fix test_communicate() of test_asyncio.test_subprocess: use `support.LONG_TIMEOUT` (5 minutes), instead of just 1 minute.

- bpo-38614 [https://bugs.python.org/issue?@action=redirect&bpo=38614]: Add timeout constants to **`test.support`**: **`LOOPBACK_TIMEOUT`**, **`INTERNET_TIMEOUT`**, **`SHORT_TIMEOUT`** and **`LONG_TIMEOUT`**.

- bpo-38502 [https://bugs.python.org/issue?@action=redirect&bpo=38502]: test.regrtest now uses process groups in the multiprocessing mode (-jN command line option) if process groups are available: if **`os.setsid()`** and **`os.killpg()`** functions are available.

- bpo-35998 [https://bugs.python.org/issue?@action=redirect&bpo=35998]: Fix a race condition in test_asyncio.test_start_tls_server_1(). Previously, there was a race condition between the test main() function which replaces the protocol and the test ServerProto protocol which sends ANSWER once it gets HELLO. Now, only the test main() function is responsible to send data, ServerProto no longer sends data.

- bpo-38470 [https://bugs.python.org/issue?@action=redirect&bpo=38470]: Fix `test_compileall.test_compile_dir_maxlevels()` on Windows

without long path support: only create 3 subdirectories instead of between 20 and 100 subdirectories.

- bpo-37531 [https://bugs.python.org/issue?@action=redirect&bpo=37531]: On timeout, regrtest no longer attempts to call `popen.communicate()` again: it can hang until all child processes using stdout and stderr pipes completes. Kill the worker process and ignores its output. Change also the faulthandler timeout of the main process from 1 minute to 5 minutes, for Python slowest buildbots.

- bpo-38239 [https://bugs.python.org/issue?@action=redirect&bpo=38239]: Fix test_gdb for Link Time Optimization (LTO) builds.

- bpo-38275 [https://bugs.python.org/issue?@action=redirect&bpo=38275]: test_ssl now handles disabled TLS/SSL versions better. OpenSSL's crypto policy and run-time settings are recognized and tests for disabled versions are skipped. Tests also accept more TLS minimum_versions for platforms that override OpenSSL's default with strict settings.

- bpo-38271 [https://bugs.python.org/issue?@action=redirect&bpo=38271]: The private keys for test_ssl were encrypted with 3DES in traditional PKCS#5 format. 3DES and the digest algorithm of PKCS#5 are blocked by some strict crypto policies. Use PKCS#8 format with AES256 encryption instead.

- bpo-38270 [https://bugs.python.org/issue?@action=redirect&bpo=38270]: test.support now has a helper function to check for availability of a hash digest function. Several tests are refactored avoid MD5 and use SHA256 instead. Other tests are marked to use MD5 and skipped when MD5 is disabled.

- bpo-37123 [https://bugs.python.org/issue?@action=redirect&bpo=37123]: Multiprocessing test test_mymanager() now also expects -SIGTERM, not only exitcode 0. BaseManager._finalize_manager() sends SIGTERM to the manager process if it takes longer than 1 second to stop, which happens on slow buildbots.

- bpo-38212 [https://bugs.python.org/issue?@action=redirect&bpo=38212]: Multiprocessing tests: increase test_queue_feeder_donot_stop_onexc() timeout from 1 to 60 seconds.

- bpo-38117 [https://bugs.python.org/issue?@action=redirect&bpo=38117]: Test with OpenSSL 1.1.1d

- bpo-38018 [https://bugs.python.org/issue?@action=redirect&bpo=38018]: Increase code coverage for multiprocessing.shared_memory.

- bpo-37805 [https://bugs.python.org/issue?@action=redirect&bpo=37805]: Add tests for json.dump(…, skipkeys=True). Patch by Donghee Na.

- bpo-37531 [https://bugs.python.org/issue?@action=redirect&bpo=37531]: Enhance regrtest multiprocess timeout: write a message when killing a worker process, catch popen.kill() and popen.wait() exceptions, put a timeout on the second call to popen.communicate().

- bpo-37876 [https://bugs.python.org/issue?@action=redirect&bpo=37876]: Add tests for ROT-13 codec.

- bpo-36833 [https://bugs.python.org/issue?@action=redirect&bpo=36833]: Added tests for `PyDateTime_xxx_GET_xxx()` macros of the C API of the **`datetime`** module. Patch by Joannah Nanjekye.

- bpo-37558 [https://bugs.python.org/issue?@action=redirect&bpo=37558]: Fix test_shared_memory_cleaned_after_process_termination name handling

- bpo-37526 [https://bugs.python.org/issue?@action=redirect&bpo=37526]: Add **`test.support.catch_threading_exception()`**: context manager catching **`threading.Thread`** exception using **`threading.excepthook()`**.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: test_concurrent_futures now explicitly stops the ForkServer instance if it’s running.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: multiprocessing tests now stop the ForkServer instance if it's running: close the "alive" file descriptor to ask the server to stop and then remove its UNIX address.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: test_distutils.test_build_ext() is now able to remove the temporary directory on Windows: don't import the newly built C extension ("xx") in the current process, but test it in a separated process.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: test_concurrent_futures now cleans up multiprocessing to remove immediately temporary directories created by multiprocessing.util.get_temp_dir().

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: test_winconsoleio doesn't leak a temporary file anymore: use tempfile.TemporaryFile() to remove it when the test completes.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: multiprocessing tests now explicitly call `_run_finalizers()` to immediately remove temporary directories created by tests.

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: urllib.request tests now call **`urlcleanup()`** to remove temporary files created by `urlretrieve()` tests and to clear the `_opener` global variable set by `urlopen()` and functions calling indirectly `urlopen()`.

- bpo-37472 [https://bugs.python.org/issue?@action=redirect&bpo=37472]: Remove `Lib/test/outstanding_bugs.py`.

- bpo-37199 [https://bugs.python.org/issue?@action=redirect&bpo=37199]: Fix test failures when IPv6 is unavailable or disabled.

- bpo-19696 [https://bugs.python.org/issue?@action=redirect&bpo=19696]: Replace deprecated method "random.choose" with "random.choice" in "test_pkg_import.py".

- bpo-37335 [https://bugs.python.org/issue?@action=redirect&bpo=37335]: Remove no longer necessary code from c locale coercion tests

- bpo-37421 [https://bugs.python.org/issue?@action=redirect&bpo=37421]: Fix test_shutil to no longer leak temporary files.

- bpo-37411 [https://bugs.python.org/issue?@action=redirect&bpo=37411]: Fix test_wsgiref.testEnviron() to no longer depend on the environment variables (don't fail if "X" variable is set).

- bpo-37400 [https://bugs.python.org/issue?@action=redirect&bpo=37400]: Fix test_os.test_chown(): use os.getgroups() rather than grp.getgrall() to get groups. Rename also the test to test_chown_gid().

- bpo-37359 [https://bugs.python.org/issue?@action=redirect&bpo=37359]: Add –cleanup option to python3 -m test to remove `test_python_*` directories of previous failed jobs. Add "make cleantest" to run `python3 -m test --cleanup`.

- bpo-37362 [https://bugs.python.org/issue?@action=redirect&bpo=37362]: test_gdb no longer fails if it gets an "unexpected" message on stderr: it now ignores stderr. The purpose of test_gdb is to test that python-gdb.py commands work as expected, not to test gdb.

- bpo-35998 [https://bugs.python.org/issue?@action=redirect&bpo=35998]: Avoid TimeoutError in test_asyncio: test_start_tls_server_1()

- bpo-37278 [https://bugs.python.org/issue?@action=redirect&bpo=37278]: Fix test_asyncio ProactorLoopCtrlC: join the thread to prevent leaking a running thread and leaking a reference.

- bpo-37261 [https://bugs.python.org/issue?@action=redirect&bpo=37261]: Fix `test.support.catch_unraisable_exception()`: its __exit__() method now ignores unraisable exception raised when clearing its `unraisable` attribute.

- bpo-37069 [https://bugs.python.org/issue?@action=redirect&bpo=37069]: regrtest now uses `sys.unraisablehook()` to mark a test as "environment

altered" (ENV_CHANGED) if it emits an "unraisable exception". Moreover, regrtest logs a warning in this case.

Use `python3 -m test --fail-env-changed` to catch unraisable exceptions in tests.

- bpo-37252 [https://bugs.python.org/issue?@action=redirect&bpo=37252]: Fix assertions in `test_close` and `test_events_mask_overflow` devpoll tests.

- bpo-37169 [https://bugs.python.org/issue?@action=redirect&bpo=37169]: Rewrite `_PyObject_IsFreed()` unit tests.

- bpo-37153 [https://bugs.python.org/issue?@action=redirect&bpo=37153]: `test_venv.test_multiprocessing()` now explicitly calls `pool.terminate()` to wait until the pool completes.

- bpo-34001 [https://bugs.python.org/issue?@action=redirect&bpo=34001]: Make test_ssl pass with LibreSSL. LibreSSL handles minimum and maximum TLS version differently than OpenSSL.

- bpo-36919 [https://bugs.python.org/issue?@action=redirect&bpo=36919]: Make `test_source_encoding.test_issue2301` implementation independent. The test will work now for both CPython and IronPython.

- bpo-30202 [https://bugs.python.org/issue?@action=redirect&bpo=30202]: Update `test.test_importlib.test_abc` to test `find_spec()`.

- bpo-28009 [https://bugs.python.org/issue?@action=redirect&bpo=28009]: Modify the test_uuid logic to test when a program is available AND can be used to obtain a MACADDR as basis for an UUID. Patch by M. Felt

- bpo-34596 [https://bugs.python.org/issue?@action=redirect&bpo=34596]: Fallback to a default reason when **`unittest.skip()`** is uncalled. Patch by Naitree Zhu.

Construção

- bpo-38809 [https://bugs.python.org/issue?@action=redirect&bpo=38809]: On Windows, build scripts will now recognize and use python.exe from an active virtual env.

- bpo-38684 [https://bugs.python.org/issue?@action=redirect&bpo=38684]: Fix _hashlib build when Blake2 is disabled, but OpenSSL supports it.

- bpo-38468 [https://bugs.python.org/issue?@action=redirect&bpo=38468]: Misc/python-config.in now uses `getvar()` for all still existing `sysconfig.get_config_var()` calls. Patch by Joannah Nanjekye.

- bpo-37415 [https://bugs.python.org/issue?@action=redirect&bpo=37415]: Fix stdatomic.h header check for ICC compiler: the ICC implementation lacks atomic_uintptr_t type which is needed by Python.

- bpo-38301 [https://bugs.python.org/issue?@action=redirect&bpo=38301]: In Solaris family, we must be sure to use `-D_REENTRANT`. Patch by Jesús Cea Avión.

- bpo-36002 [https://bugs.python.org/issue?@action=redirect&bpo=36002]: Locate `llvm-profdata` and `llvm-ar` binaries using `AC_PATH_TOOL` rather than `AC_PATH_TARGET_TOOL`.

- bpo-37936 [https://bugs.python.org/issue?@action=redirect&bpo=37936]: The `.gitignore` file systematically keeps “rooted”, with a non-trailing slash, all the rules that are meant to apply to files in a specific place in the repo. Previously, when the intended file to ignore happened to be at the root of the repo, we’d most often accidentally also ignore files and directories with the same name anywhere in the tree.

- bpo-37760 [https://bugs.python.org/issue?@action=redirect&bpo=37760]: The `Tools/unicode/makeunicodedata.py` script, which is used for converting information from the Unicode Character Database into generated code and data used by the methods of **`str`** and by the **`unicodedata`** module, now handles each character’s data as a `dataclass` with named attributes, rather than a length-18 list of different fields.

- bpo-37936 [https://bugs.python.org/issue?@action=redirect&bpo=37936]: The `.gitignore` file no longer applies to any files that are in fact tracked in the Git repository. Patch by Greg Price.

- bpo-37725 [https://bugs.python.org/issue?@action=redirect&bpo=37725]: Change "clean" makefile target to also clean the program guided optimization (PGO) data. Previously you would have to use "make clean" and "make profile-removal", or "make clobber".

- bpo-37707 [https://bugs.python.org/issue?@action=redirect&bpo=37707]: Mark some individual tests to skip when –pgo is used. The tests marked increase the PGO task time significantly and likely don't help improve optimization of the final executable.

- bpo-36044 [https://bugs.python.org/issue?@action=redirect&bpo=36044]: Reduce the number of unit tests run for the PGO generation task. This speeds up the task by a factor of about 15x. Running the full unit test suite is slow. This change may result in a slightly less optimized build since not as many code branches will be executed. If you are willing to wait for the much slower build, the old behavior can be restored using './configure [..] PROFILE_TASK="-m test –pgo-extended"'. We make no guarantees as to which PGO task set produces a faster build. Users who care should run their own relevant benchmarks as results can depend on the environment, workload, and compiler tool chain.

- bpo-37468 [https://bugs.python.org/issue?@action=redirect&bpo=37468]: `make install` no longer installs `wininst-*.exe` files used by distutils bdist_wininst: bdist_wininst only works on Windows.

- bpo-37189 [https://bugs.python.org/issue?@action=redirect&bpo=37189]: Many `PyRun_XXX()` functions like **`PyRun_String()`** were no longer exported in `libpython38.dll` by mistake. Export them again to fix the ABI compatibility.

- bpo-25361 [https://bugs.python.org/issue?@action=redirect&bpo=25361]: Enables use of SSE2 instructions in Windows 32-bit build.

- bpo-36210 [https://bugs.python.org/issue?@action=redirect&bpo=36210]: Update optional extension module detection for AIX. ossaudiodev and spwd are not applicable for AIX, and are no longer reported as missing. 3rd-party packaging of ncurses (with ASIS support) conflicts with officially supported AIX curses library, so configure AIX to use libcurses.a. However, skip trying to build _curses_panel.

  patch by M Felt

Windows

- bpo-38589 [https://bugs.python.org/issue?@action=redirect&bpo=38589]: Fixes HTML Help shortcut when Windows is not installed to C drive
- bpo-38453 [https://bugs.python.org/issue?@action=redirect&bpo=38453]: Ensure ntpath.realpath() correctly resolves relative paths.
- bpo-38519 [https://bugs.python.org/issue?@action=redirect&bpo=38519]: Restores the internal C headers that were missing from the nuget.org and Microsoft Store packages.
- bpo-38492 [https://bugs.python.org/issue?@action=redirect&bpo=38492]: Remove `pythonw.exe` dependency on the Microsoft C++ runtime.
- bpo-38344 [https://bugs.python.org/issue?@action=redirect&bpo=38344]: Fix error message in activate.bat
- bpo-38359 [https://bugs.python.org/issue?@action=redirect&bpo=38359]: Ensures `pyw.exe` launcher reads correct registry key.
- bpo-38355 [https://bugs.python.org/issue?@action=redirect&bpo=38355]: Fixes `ntpath.realpath` failing on `sys.executable`.
- bpo-38117 [https://bugs.python.org/issue?@action=redirect&bpo=38117]: Update bundled OpenSSL to 1.1.1d
- bpo-38092 [https://bugs.python.org/issue?@action=redirect&bpo=38092]: Reduce overhead when using multiprocessing in a Windows virtual environment.
- bpo-38133 [https://bugs.python.org/issue?@action=redirect&bpo=38133]: Allow py.exe launcher to locate installations from the Microsoft Store and improve display of active virtual environments.
- bpo-38114 [https://bugs.python.org/issue?@action=redirect&bpo=38114]: The `pip.ini` is no longer included in the Nuget package.

- bpo-32592 [https://bugs.python.org/issue?@action=redirect&bpo=32592]: Set Windows 8 as the minimum required version for API support
- bpo-36634 [https://bugs.python.org/issue?@action=redirect&bpo=36634]: **`os.cpu_count()`** now returns active processors rather than maximum processors.
- bpo-36634 [https://bugs.python.org/issue?@action=redirect&bpo=36634]: venv activate.bat now works when the existing variables contain double quote characters.
- bpo-38081 [https://bugs.python.org/issue?@action=redirect&bpo=38081]: Prevent error calling **`os.path.realpath()`** on `'NUL'`.
- bpo-38087 [https://bugs.python.org/issue?@action=redirect&bpo=38087]: Fix case sensitivity in test_pathlib and test_ntpath.
- bpo-38088 [https://bugs.python.org/issue?@action=redirect&bpo=38088]: Fixes distutils not finding vcruntime140.dll with only the v142 toolset installed.
- bpo-37283 [https://bugs.python.org/issue?@action=redirect&bpo=37283]: Ensure command-line and unattend.xml setting override previously detected states in Windows installer.
- bpo-38030 [https://bugs.python.org/issue?@action=redirect&bpo=38030]: Fixes **`os.stat()`** failing for block devices on Windows
- bpo-38020 [https://bugs.python.org/issue?@action=redirect&bpo=38020]: Fixes potential crash when calling **`os.readlink()`** (or indirectly through **`realpath()`**) on a file that is not a supported link.
- bpo-37705 [https://bugs.python.org/issue?@action=redirect&bpo=37705]: Improve the implementation of `winerror_to_errno()`.
- bpo-37549 [https://bugs.python.org/issue?@action=redirect&bpo=37549]: **`os.dup()`** no longer fails for standard streams on Windows 7.
- bpo-1311 [https://bugs.python.org/issue?@action=redirect&bpo=1311]: The `nul` file on Windows now returns True from **`exists()`** and a valid result from **`os.stat()`** with `S_IFCHR` set.
- bpo-9949 [https://bugs.python.org/issue?@action=redirect&bpo=9949]: Enable support for following symlinks in **`os.realpath()`**.
- bpo-37834 [https://bugs.python.org/issue?@action=redirect&bpo=37834]: Treat all name surrogate reparse points on Windows in **`os.lstat()`** and other reparse points as regular files in **`os.stat()`**.
- bpo-36266 [https://bugs.python.org/issue?@action=redirect&bpo=36266]: Add the module name in the formatted error message when DLL load fail

happens during module import in `_PyImport_FindSharedFuncptrWindows()`. Patch by Srinivas Nyayapati.

- bpo-25172 [https://bugs.python.org/issue?@action=redirect&bpo=25172]: Trying to import the **`crypt`** module on Windows will result in an **`ImportError`** with a message explaining that the module isn't supported on Windows. On other platforms, if the underlying `_crypt` module is not available, the ImportError will include a message explaining the problem.
- bpo-37778 [https://bugs.python.org/issue?@action=redirect&bpo=37778]: Fixes the icons used for file associations to the Microsoft Store package.
- bpo-37734 [https://bugs.python.org/issue?@action=redirect&bpo=37734]: Fix use of registry values to launch Python from Microsoft Store app.
- bpo-37702 [https://bugs.python.org/issue?@action=redirect&bpo=37702]: Fix memory leak on Windows in creating an SSLContext object or running `urllib.request.urlopen('https://...')`.
- bpo-37672 [https://bugs.python.org/issue?@action=redirect&bpo=37672]: Switch Windows Store package's pip to use bundled `pip.ini` instead of **`PIP_USER`** variable.
- bpo-10945 [https://bugs.python.org/issue?@action=redirect&bpo=10945]: Officially drop support for creating bdist_wininst installers on non-Windows systems.
- bpo-37445 [https://bugs.python.org/issue?@action=redirect&bpo=37445]: Include the `FORMAT_MESSAGE_IGNORE_INSERTS` flag in `FormatMessageW()` calls.
- bpo-37369 [https://bugs.python.org/issue?@action=redirect&bpo=37369]: Fixes path for **`sys.executable`** when running from the Microsoft Store.
- bpo-37380 [https://bugs.python.org/issue?@action=redirect&bpo=37380]: Don't collect unfinished processes with `subprocess._active` on Windows to cleanup later. Patch by Ruslan Kuprieiev.
- bpo-37351 [https://bugs.python.org/issue?@action=redirect&bpo=37351]: Removes libpython38.a from standard Windows distribution.
- bpo-35360 [https://bugs.python.org/issue?@action=redirect&bpo=35360]: Update Windows builds to use SQLite 3.28.0.
- bpo-37267 [https://bugs.python.org/issue?@action=redirect&bpo=37267]: On Windows, **`os.dup()`** no longer creates an inheritable fd when handling a character file.

- bpo-36779 [https://bugs.python.org/issue?@action=redirect&bpo=36779]: Ensure `time.tzname` is correct on Windows when the active code page is set to CP_UTF7 or CP_UTF8.
- bpo-32587 [https://bugs.python.org/issue?@action=redirect&bpo=32587]: Make **`winreg.REG_MULTI_SZ`** support zero-length strings.
- bpo-28269 [https://bugs.python.org/issue?@action=redirect&bpo=28269]: Replace use of **`strcasecmp()`** for the system function **`_stricmp()`**. Patch by Minmin Gong.
- bpo-36590 [https://bugs.python.org/issue?@action=redirect&bpo=36590]: Add native Bluetooth RFCOMM support to socket module.

macOS

- bpo-38117 [https://bugs.python.org/issue?@action=redirect&bpo=38117]: Updated OpenSSL to 1.1.1d in macOS installer.
- bpo-38089 [https://bugs.python.org/issue?@action=redirect&bpo=38089]: Move Azure Pipelines to latest VM versions and make macOS tests optional
- bpo-18049 [https://bugs.python.org/issue?@action=redirect&bpo=18049]: Increase the default stack size of threads from 5MB to 16MB on macOS, to match the stack size of the main thread. This avoids crashes on deep recursion in threads.
- bpo-34602 [https://bugs.python.org/issue?@action=redirect&bpo=34602]: Avoid test suite failures on macOS by no longer calling resource.setrlimit to increase the process stack size limit at runtime. The runtime change is no longer needed since the interpreter is being built with a larger default stack size.
- bpo-35360 [https://bugs.python.org/issue?@action=redirect&bpo=35360]: Update macOS installer to use SQLite 3.28.0.
- bpo-34631 [https://bugs.python.org/issue?@action=redirect&bpo=34631]: Updated OpenSSL to 1.1.1c in macOS installer.

IDLE

- bpo-26353 [https://bugs.python.org/issue?@action=redirect&bpo=26353]: Stop adding newline when saving an IDLE shell window.
- bpo-4630 [https://bugs.python.org/issue?@action=redirect&bpo=4630]: Add an option to toggle IDLE's cursor blink for shell, editor, and output

windows. See Settings, General, Window Preferences, Cursor Blink. Patch by Zackery Spytz.

- bpo-38598 [https://bugs.python.org/issue?@action=redirect&bpo=38598]: Do not try to compile IDLE shell or output windows
- bpo-36698 [https://bugs.python.org/issue?@action=redirect&bpo=36698]: IDLE no longer fails when write non-encodable characters to stderr. It now escapes them with a backslash, as the regular Python interpreter. Added the `errors` field to the standard streams.
- bpo-35379 [https://bugs.python.org/issue?@action=redirect&bpo=35379]: When exiting IDLE, catch any AttributeError. One happens when EditorWindow.close is called twice. Printing a traceback, when IDLE is run from a terminal, is useless and annoying.
- bpo-38183 [https://bugs.python.org/issue?@action=redirect&bpo=38183]: To avoid problems, test_idle ignores the user config directory. It no longer tries to create or access .idlerc or any files within. Users must run IDLE to discover problems with saving settings.
- bpo-38077 [https://bugs.python.org/issue?@action=redirect&bpo=38077]: IDLE no longer adds ‘argv’ to the user namespace when initializing it. This bug only affected 3.7.4 and 3.8.0b2 to 3.8.0b4.
- bpo-38041 [https://bugs.python.org/issue?@action=redirect&bpo=38041]: Shell restart lines now fill the window width, always start with ‘=’, and avoid wrapping unnecessarily. The line will still wrap if the included file name is long relative to the width.
- bpo-35771 [https://bugs.python.org/issue?@action=redirect&bpo=35771]: To avoid occasional spurious test_idle failures on slower machines, increase the `hover_delay` in test_tooltip.
- bpo-37824 [https://bugs.python.org/issue?@action=redirect&bpo=37824]: Properly handle user input warnings in IDLE shell. Cease turning SyntaxWarnings into SyntaxErrors.
- bpo-37929 [https://bugs.python.org/issue?@action=redirect&bpo=37929]: IDLE Settings dialog now closes properly when there is no shell window.
- bpo-37902 [https://bugs.python.org/issue?@action=redirect&bpo=37902]: Add mousewheel scrolling for IDLE module, path, and stack browsers. Patch by George Zhang.
- bpo-37849 [https://bugs.python.org/issue?@action=redirect&bpo=37849]: Fixed completions list appearing too high or low when shown above the current line.

- bpo-36419 [https://bugs.python.org/issue?@action=redirect&bpo=36419]: Refactor IDLE autocomplete and improve testing.
- bpo-37748 [https://bugs.python.org/issue?@action=redirect&bpo=37748]: Reorder the Run menu. Put the most common choice, Run Module, at the top.
- bpo-37692 [https://bugs.python.org/issue?@action=redirect&bpo=37692]: Improve highlight config sample with example shell interaction and better labels for shell elements.
- bpo-37628 [https://bugs.python.org/issue?@action=redirect&bpo=37628]: Settings dialog no longer expands with font size.
- bpo-37627 [https://bugs.python.org/issue?@action=redirect&bpo=37627]: Initialize the Customize Run dialog with the command line arguments most recently entered before. The user can optionally edit before submitting them.
- bpo-33610 [https://bugs.python.org/issue?@action=redirect&bpo=33610]: Fix code context not showing the correct context when first toggled on.
- bpo-37530 [https://bugs.python.org/issue?@action=redirect&bpo=37530]: Optimize code context to reduce unneeded background activity. Font and highlight changes now occur along with text changes instead of after a random delay.
- bpo-27452 [https://bugs.python.org/issue?@action=redirect&bpo=27452]: Cleanup `config.py` by inlining `RemoveFile` and simplifying the handling of `file` in `CreateConfigHandlers`.
- bpo-37325 [https://bugs.python.org/issue?@action=redirect&bpo=37325]: Fix tab focus traversal order for help source and custom run dialogs.
- bpo-37321 [https://bugs.python.org/issue?@action=redirect&bpo=37321]: Both subprocess connection error messages now refer to the ‘Startup failure’ section of the IDLE doc.
- bpo-17535 [https://bugs.python.org/issue?@action=redirect&bpo=17535]: Add optional line numbers for IDLE editor windows. Windows open without line numbers unless set otherwise in the General tab of the configuration dialog.
- bpo-26806 [https://bugs.python.org/issue?@action=redirect&bpo=26806]: To compensate for stack frames added by IDLE and avoid possible problems with low recursion limits, add 30 to limits in the user code execution process. Subtract 30 when reporting recursion limits to make this addition mostly transparent.

- bpo-37177 [https://bugs.python.org/issue?@action=redirect&bpo=37177]: Properly ‘attach’ search dialogs to their main window so that they behave like other dialogs and do not get hidden behind their main window.
- bpo-37039 [https://bugs.python.org/issue?@action=redirect&bpo=37039]: Adjust “Zoom Height” to individual screens by momentarily maximizing the window on first use with a particular screen. Changing screen settings may invalidate the saved height. While a window is maximized, “Zoom Height” has no effect.
- bpo-35763 [https://bugs.python.org/issue?@action=redirect&bpo=35763]: Make calltip reminder about ‘/’ meaning positional-only less obtrusive by only adding it when there is room on the first line.
- bpo-5680 [https://bugs.python.org/issue?@action=redirect&bpo=5680]: Add ‘Run… Customized’ to the Run menu to run a module with customized settings. Any ‘command line arguments’ entered are added to sys.argv. One can suppress the normal Shell main module restart.
- bpo-36390 [https://bugs.python.org/issue?@action=redirect&bpo=36390]: Gather Format menu functions into format.py. Combine paragraph.py, rstrip.py, and format methods from editor.py.

Ferramentas/Demos

- bpo-38118 [https://bugs.python.org/issue?@action=redirect&bpo=38118]: Update Valgrind suppression file to ignore a false alarm in `PyUnicode_Decode()` when using GCC builtin strcmp().
- bpo-38347 [https://bugs.python.org/issue?@action=redirect&bpo=38347]: pathfix.py: Assume all files that end on ‘.py’ are Python scripts when working recursively.
- bpo-37803 [https://bugs.python.org/issue?@action=redirect&bpo=37803]: pdb’s `--help` and `--version` long options now work.
- bpo-37942 [https://bugs.python.org/issue?@action=redirect&bpo=37942]: Improve ArgumentClinic converter for floats.
- bpo-37704 [https://bugs.python.org/issue?@action=redirect&bpo=37704]: Remove `Tools/scripts/h2py.py`: use cffi to access a C API in Python.
- bpo-37675 [https://bugs.python.org/issue?@action=redirect&bpo=37675]: 2to3 now works when run from a zipped standard library.
- bpo-37034 [https://bugs.python.org/issue?@action=redirect&bpo=37034]: Argument Clinic now uses the argument name on errors with keyword-only

argument instead of their position. Patch contributed by Rémi Lapeyre.

- bpo-37064 [https://bugs.python.org/issue?@action=redirect&bpo=37064]: Add option -k to pathscript.py script: preserve shebang flags. Add option -a to pathscript.py script: add flags.

API C

- bpo-37633 [https://bugs.python.org/issue?@action=redirect&bpo=37633]: Re-export some function compatibility wrappers for macros in `pythonrun.h`.
- bpo-38644 [https://bugs.python.org/issue?@action=redirect&bpo=38644]: Provide **`Py_EnterRecursiveCall()`** and **`Py_LeaveRecursiveCall()`** as regular functions for the limited API. Previously, there were defined as macros, but these macros didn't work with the limited API which cannot access `PyThreadState.recursion_depth` field. Remove `_Py_CheckRecursionLimit` from the stable ABI.
- bpo-38650 [https://bugs.python.org/issue?@action=redirect&bpo=38650]: The global variable **`PyStructSequence_UnnamedField`** is now a constant and refers to a constant string.
- bpo-38540 [https://bugs.python.org/issue?@action=redirect&bpo=38540]: Fixed possible leak in **`PyArg_Parse()`** and similar functions for format units `"es#"` and `"et#"` when the macro **`PY_SSIZE_T_CLEAN`** is not defined.
- bpo-38395 [https://bugs.python.org/issue?@action=redirect&bpo=38395]: Fix a crash in **`weakref.proxy`** objects due to incorrect lifetime management when calling some associated methods that may delete the last reference to object being referenced by the proxy. Patch by Pablo Galindo.
- bpo-36389 [https://bugs.python.org/issue?@action=redirect&bpo=36389]: The `_PyObject_CheckConsistency()` function is now also available in release mode. For example, it can be used to debug a crash in the `visit_decref()` function of the GC.
- bpo-38266 [https://bugs.python.org/issue?@action=redirect&bpo=38266]: Revert the removal of PyThreadState_DeleteCurrent() with documentation.
- bpo-38303 [https://bugs.python.org/issue?@action=redirect&bpo=38303]: Update audioop extension module to use the stable ABI (PEP-384). Patch by Tyler Kieft.
- bpo-38234 [https://bugs.python.org/issue?@action=redirect&bpo=38234]: **`Py_SetPath()`** now sets **`sys.executable`** to the program full path

(**`Py_GetProgramFullPath()`**) rather than to the program name (**`Py_GetProgramName()`**).

- bpo-38234 [https://bugs.python.org/issue?@action=redirect&bpo=38234]: Python ignored arguments passed to **`Py_SetPath()`**, **`Py_SetPythonHome()`** and **`Py_SetProgramName()`**: fix Python initialization to use specified arguments.
- bpo-38205 [https://bugs.python.org/issue?@action=redirect&bpo=38205]: The **`Py_UNREACHABLE()`** macro now calls **`Py_FatalError()`**.
- bpo-38140 [https://bugs.python.org/issue?@action=redirect&bpo=38140]: Make dict and weakref offsets opaque for C heap types by passing the offsets through PyMemberDef
- bpo-15088 [https://bugs.python.org/issue?@action=redirect&bpo=15088]: The C function `PyGen_NeedsFinalizing` has been removed. It was not documented, tested or used anywhere within CPython after the implementation of **PEP 442** [https://peps.python.org/pep-0442/]. Patch by Joannah Nanjekye. (Patch by Joannah Nanjekye)
- bpo-36763 [https://bugs.python.org/issue?@action=redirect&bpo=36763]: Options added by `PySys_AddXOption()` are now handled the same way than `PyConfig.xoptions` and command line `-X` options.
- bpo-37926 [https://bugs.python.org/issue?@action=redirect&bpo=37926]: Fix a crash in `PySys_SetArgvEx(0, NULL, 0)`.
- bpo-37879 [https://bugs.python.org/issue?@action=redirect&bpo=37879]: Fix subtype_dealloc to suppress the type decref when the base type is a C heap type
- bpo-37645 [https://bugs.python.org/issue?@action=redirect&bpo=37645]: Add **`_PyObject_FunctionStr()`** to get a user-friendly string representation of a function-like object. Patch by Jeroen Demeyer.
- bpo-29548 [https://bugs.python.org/issue?@action=redirect&bpo=29548]: The functions `PyEval_CallObject`, `PyEval_CallFunction`, `PyEval_CallMethod` and `PyEval_CallObjectWithKeywords` are deprecated. Use **`PyObject_Call()`** and its variants instead.
- bpo-37151 [https://bugs.python.org/issue?@action=redirect&bpo=37151]: `PyCFunction_Call` is now a deprecated alias of **`PyObject_Call()`**.
- bpo-37540 [https://bugs.python.org/issue?@action=redirect&bpo=37540]: The vectorcall protocol now requires that the caller passes only strings as keyword names.

- bpo-37207 [https://bugs.python.org/issue?@action=redirect&bpo=37207]: The vectorcall protocol is now enabled for `type` objects: set `tp_vectorcall` to a vectorcall function to be used instead of `tp_new` and `tp_init` when calling the class itself.
- bpo-21120 [https://bugs.python.org/issue?@action=redirect&bpo=21120]: Exclude Python-ast.h, ast.h and asdl.h from the limited API.
- bpo-37483 [https://bugs.python.org/issue?@action=redirect&bpo=37483]: Add new function `_PyObject_CallOneArg` for calling an object with one positional argument.
- bpo-36763 [https://bugs.python.org/issue?@action=redirect&bpo=36763]: Add **`PyConfig_SetWideStringList()`** function.
- bpo-37337 [https://bugs.python.org/issue?@action=redirect&bpo=37337]: Add fast functions for calling methods: **`_PyObject_VectorcallMethod()`**, **`_PyObject_CallMethodNoArgs()`** and **`_PyObject_CallMethodOneArg()`**.
- bpo-28805 [https://bugs.python.org/issue?@action=redirect&bpo=28805]: The **`METH_FASTCALL`** calling convention has been documented.
- bpo-37221 [https://bugs.python.org/issue?@action=redirect&bpo=37221]: The new function **`PyCode_NewWithPosOnlyArgs()`** allows to create code objects like **`PyCode_New()`**, but with an extra *posonlyargcount* parameter for indicating the number of positonal-only arguments.
- bpo-37215 [https://bugs.python.org/issue?@action=redirect&bpo=37215]: Fix dtrace issue introduce by bpo-36842 [https://bugs.python.org/issue?@action=redirect&bpo=36842]
- bpo-37194 [https://bugs.python.org/issue?@action=redirect&bpo=37194]: Add a new public **`PyObject_CallNoArgs()`** function to the C API: call a callable Python object without any arguments. It is the most efficient way to call a callback without any argument. On x86-64, for example, `PyObject_CallFunctionObjArgs(func, NULL)` allocates 960 bytes on the stack per call, whereas `PyObject_CallNoArgs(func)` only allocates 624 bytes per call.
- bpo-37170 [https://bugs.python.org/issue?@action=redirect&bpo=37170]: Fix the cast on error in **`PyLong_AsUnsignedLongLongMask()`**.
- bpo-35381 [https://bugs.python.org/issue?@action=redirect&bpo=35381]: Convert posixmodule.c statically allocated types `DirEntryType` and `ScandirIteratorType` to heap-allocated types.

- bpo-34331 [https://bugs.python.org/issue?@action=redirect&bpo=34331]: Use singular/plural noun in error message when instantiating an abstract class with non-overridden abstract method(s).

Python 3.8.0 beta 1

Data de lançamento: 2019-06-04

Segurança

- bpo-35907 [https://bugs.python.org/issue?@action=redirect&bpo=35907]: **CVE 2019-9948** [https://www.cve.org/CVERecord?id=CVE-2019-9948]: Avoid file reading by disallowing `local-file://` and `local_file://` URL schemes in `URLopener().open()` and `URLopener().retrieve()` of **`urllib.request`**.
- bpo-33529 [https://bugs.python.org/issue?@action=redirect&bpo=33529]: Prevent fold function used in email header encoding from entering infinite loop when there are too many non-ASCII characters in a header.
- bpo-33164 [https://bugs.python.org/issue?@action=redirect&bpo=33164]: Updated blake2 implementation which uses secure memset implementation provided by platform.

Núcleo e embutidos

- bpo-35814 [https://bugs.python.org/issue?@action=redirect&bpo=35814]: Allow unpacking in the right hand side of annotated assignments. In particular, `t: Tuple[int, ...] = x, y, *z` is now allowed.
- bpo-37126 [https://bugs.python.org/issue?@action=redirect&bpo=37126]: All structseq objects are now tracked by the garbage collector. Patch by Pablo Galindo.
- bpo-37122 [https://bugs.python.org/issue?@action=redirect&bpo=37122]: Make the *co_argcount* attribute of code objects represent the total number of positional arguments (including positional-only arguments). The value of *co_posonlyargcount* can be used to distinguish which arguments are positional only, and the difference (*co_argcount* - *co_posonlyargcount*)

is the number of positional-or-keyword arguments. Patch by Pablo Galindo.

- bpo-20092 [https://bugs.python.org/issue?@action=redirect&bpo=20092]: Constructors of `int`, `float` and `complex` will now use the `__index__()` special method, if available and the corresponding method `__int__()`, `__float__()` or `__complex__()` is not available.
- bpo-37087 [https://bugs.python.org/issue?@action=redirect&bpo=37087]: Add native thread ID (TID) support to OpenBSD.
- bpo-26219 [https://bugs.python.org/issue?@action=redirect&bpo=26219]: Implemented per opcode cache mechanism and `LOAD_GLOBAL` instruction use it. `LOAD_GLOBAL` is now about 40% faster. Contributed by Yury Selivanov, and Inada Naoki.
- bpo-37072 [https://bugs.python.org/issue?@action=redirect&bpo=37072]: Fix crash in PyAST_FromNodeObject() when flags is NULL.
- bpo-37029 [https://bugs.python.org/issue?@action=redirect&bpo=37029]: Freeing a great many small objects could take time quadratic in the number of arenas, due to using linear search to keep `obmalloc.c`'s list of usable arenas sorted by order of number of free memory pools. This is accomplished without search now, leaving the worst-case time linear in the number of arenas. For programs where this quite visibly matters (typically with more than 100 thousand small objects alive simultaneously), this can greatly reduce the time needed to release their memory.
- bpo-26423 [https://bugs.python.org/issue?@action=redirect&bpo=26423]: Fix possible overflow in `wrap_lenfunc()` when `sizeof(long) < sizeof(Py_ssize_t)` (e.g., 64-bit Windows).
- bpo-37050 [https://bugs.python.org/issue?@action=redirect&bpo=37050]: Improve the AST for "debug" f-strings, which use '=' to print out the source of the expression being evaluated. Delete expr_text from the FormattedValue node, and instead use a Constant string node (possibly merged with adjacent constant expressions inside the f-string).
- bpo-22385 [https://bugs.python.org/issue?@action=redirect&bpo=22385]: The `bytes.hex`, `bytearray.hex`, and `memoryview.hex` methods as well as the `binascii.hexlify` and `b2a_hex` functions now have the ability to include an optional separator between hex bytes. This functionality was inspired by MicroPython's hexlify implementation.

- bpo-26836 [https://bugs.python.org/issue?@action=redirect&bpo=26836]: Add **`os.memfd_create()`**.
- bpo-37032 [https://bugs.python.org/issue?@action=redirect&bpo=37032]: Added new `replace()` method to the code type (**`types.CodeType`**).
- bpo-37007 [https://bugs.python.org/issue?@action=redirect&bpo=37007]: Implement **`socket.if_nameindex()`**, **`socket.if_nametoindex()`**, and **`socket.if_indextoname()`** on Windows.
- bpo-36829 [https://bugs.python.org/issue?@action=redirect&bpo=36829]: **`PyErr_WriteUnraisable()`** now creates a traceback object if there is no current traceback. Moreover, call **`PyErr_NormalizeException()`** and **`PyException_SetTraceback()`** to normalize the exception value. Ignore any error.
- bpo-36878 [https://bugs.python.org/issue?@action=redirect&bpo=36878]: Only accept text after `# type: ignore` if the first character is ASCII. This is to disallow things like `# type: ignoreé`.
- bpo-36878 [https://bugs.python.org/issue?@action=redirect&bpo=36878]: Store text appearing after a `# type: ignore` comment in the AST. For example a type ignore like `# type: ignore[E1000]` will have the string `"[E1000]"` stored in its AST node.
- bpo-2180 [https://bugs.python.org/issue?@action=redirect&bpo=2180]: Treat line continuation at EOF as a `SyntaxError` by Anthony Sottile.
- bpo-36907 [https://bugs.python.org/issue?@action=redirect&bpo=36907]: Fix a crash when calling a C function with a keyword dict (`f(**kwargs)`) and changing the dict `kwargs` while that function is running.
- bpo-36946 [https://bugs.python.org/issue?@action=redirect&bpo=36946]: Fix possible signed integer overflow when handling slices.
- bpo-36826 [https://bugs.python.org/issue?@action=redirect&bpo=36826]: Add NamedExpression kind support to ast_unparse.c
- bpo-1875 [https://bugs.python.org/issue?@action=redirect&bpo=1875]: A **`SyntaxError`** is now raised if a code blocks that will be optimized away (e.g. if conditions that are always false) contains syntax errors. Patch by Pablo Galindo.
- bpo-36027 [https://bugs.python.org/issue?@action=redirect&bpo=36027]: Allow computation of modular inverses via three-argument `pow`: the second argument is now permitted to be negative in the case where the first and third arguments are relatively prime.

- bpo-36861 [https://bugs.python.org/issue?@action=redirect&bpo=36861]: Update the Unicode database to version 12.1.0.
- bpo-28866 [https://bugs.python.org/issue?@action=redirect&bpo=28866]: Avoid caching attributes of classes which type defines mro() to avoid a hard cache invalidation problem.
- bpo-36851 [https://bugs.python.org/issue?@action=redirect&bpo=36851]: The `FrameType` stack is now correctly cleaned up if the execution ends with a return and the stack is not empty.
- bpo-34616 [https://bugs.python.org/issue?@action=redirect&bpo=34616]: The `compile()` builtin functions now support the `ast.PyCF_ALLOW_TOP_LEVEL_AWAIT` flag, which allow to compile sources that contains top-level `await`, `async with` or `async for`. This is useful to evaluate async-code from with an already async functions; for example in a custom REPL.
- bpo-36842 [https://bugs.python.org/issue?@action=redirect&bpo=36842]: Implement PEP 578, adding sys.audit, io.open_code and related APIs.
- bpo-27639 [https://bugs.python.org/issue?@action=redirect&bpo=27639]: Correct return type for UserList slicing operations. Patch by Michael Blahay, Erick Cervantes, and vaultah
- bpo-36737 [https://bugs.python.org/issue?@action=redirect&bpo=36737]: Move PyRuntimeState.warnings into per-interpreter state (via “module state”).
- bpo-36793 [https://bugs.python.org/issue?@action=redirect&bpo=36793]: Removed `__str__` implementations from builtin types **`bool`**, **`int`**, **`float`**, **`complex`** and few classes from the standard library. They now inherit `__str__()` from **`object`**.
- bpo-36817 [https://bugs.python.org/issue?@action=redirect&bpo=36817]: Add `a =` feature f-strings for debugging. This can precede `!s`, `!r`, or `!a`. It produces the text of the expression, followed by an equal sign, followed by the repr of the value of the expression. So `f'{3*9+15=}'` would be equal to the string `'3*9+15=42'`. If `=` is specified, the default conversion is set to `!r`, unless a format spec is given, in which case the formatting behavior is unchanged, and __format__ will be used.
- bpo-24048 [https://bugs.python.org/issue?@action=redirect&bpo=24048]: Save the live exception during import.c’s `remove_module()`.
- bpo-27987 [https://bugs.python.org/issue?@action=redirect&bpo=27987]: pymalloc returns memory blocks aligned by 16 bytes, instead of 8 bytes, on 64-

bit platforms to conform x86-64 ABI. Recent compilers assume this alignment more often. Patch by Inada Naoki.

- bpo-36601 [https://bugs.python.org/issue?@action=redirect&bpo=36601]: A long-since-meaningless check for `getpid() == main_pid` was removed from Python's internal C signal handler.
- bpo-36594 [https://bugs.python.org/issue?@action=redirect&bpo=36594]: Fix incorrect use of `%p` in format strings. Patch by Zackery Spytz.
- bpo-36045 [https://bugs.python.org/issue?@action=redirect&bpo=36045]: `builtins.help()` now prefixes `async` for async functions.
- bpo-36084 [https://bugs.python.org/issue?@action=redirect&bpo=36084]: Add native thread ID (TID) to threading.Thread objects (supported platforms: Windows, FreeBSD, Linux, macOS)
- bpo-36035 [https://bugs.python.org/issue?@action=redirect&bpo=36035]: Added fix for broken symlinks in combination with pathlib
- bpo-35983 [https://bugs.python.org/issue?@action=redirect&bpo=35983]: Added new trashcan macros to deal with a double deallocation that could occur when the `tp_dealloc` of a subclass calls the `tp_dealloc` of a base class and that base class uses the trashcan mechanism. Patch by Jeroen Demeyer.
- bpo-20602 [https://bugs.python.org/issue?@action=redirect&bpo=20602]: Do not clear **`sys.flags`** and **`sys.float_info`** during shutdown. Patch by Zackery Spytz.
- bpo-26826 [https://bugs.python.org/issue?@action=redirect&bpo=26826]: Expose **`copy_file_range()`** as a low level API in the **`os`** module.
- bpo-32388 [https://bugs.python.org/issue?@action=redirect&bpo=32388]: Remove cross-version binary compatibility requirement in tp_flags.
- bpo-31862 [https://bugs.python.org/issue?@action=redirect&bpo=31862]: Port binascii to PEP 489 multiphase initialization. Patch by Marcel Plch.

Biblioteca

- bpo-37128 [https://bugs.python.org/issue?@action=redirect&bpo=37128]: Added **`math.perm()`**.

- bpo-37120 [https://bugs.python.org/issue?@action=redirect&bpo=37120]: Add SSLContext.num_tickets to control the number of TLSv1.3 session tickets.

- bpo-12202 [https://bugs.python.org/issue?@action=redirect&bpo=12202]: Fix the error handling in **`msilib.SummaryInformation.GetProperty()`**. Patch by Zackery Spytz.

- bpo-26835 [https://bugs.python.org/issue?@action=redirect&bpo=26835]: The fcntl module now contains file sealing constants for sealing of memfds.

- bpo-29262 [https://bugs.python.org/issue?@action=redirect&bpo=29262]: Add `get_origin()` and `get_args()` introspection helpers to `typing` module.

- bpo-12639 [https://bugs.python.org/issue?@action=redirect&bpo=12639]: **`msilib.Directory.start_component()`** no longer fails if *keyfile* is not `None`.

- bpo-36999 [https://bugs.python.org/issue?@action=redirect&bpo=36999]: Add the `asyncio.Task.get_coro()` method to publicly expose the tasks's coroutine object.

- bpo-35246 [https://bugs.python.org/issue?@action=redirect&bpo=35246]: Make **`asyncio.create_subprocess_exec()`** accept path-like arguments.

- bpo-35279 [https://bugs.python.org/issue?@action=redirect&bpo=35279]: Change default *max_workers* of `ThreadPoolExecutor` from `cpu_count() * 5` to `min(32, cpu_count() + 4)`. Previous value was unreasonably large on many cores machines.

- bpo-37076 [https://bugs.python.org/issue?@action=redirect&bpo=37076]: **`_thread.start_new_thread()`** now logs uncaught exception raised by the function using **`sys.unraisablehook()`**, rather than **`sys.excepthook()`**, so the hook gets access to the function which raised the exception.

- bpo-33725 [https://bugs.python.org/issue?@action=redirect&bpo=33725]: On macOS, the **`multiprocessing`** module now uses *spawn* start method by default.

- bpo-37054 [https://bugs.python.org/issue?@action=redirect&bpo=37054]: Fix destructor **`_pyio.BytesIO`** and **`_pyio.TextIOWrapper`**: initialize their

`_buffer` attribute as soon as possible (in the class body), because it's used by `__del__()` which calls `close()`.

- bpo-37058 [https://bugs.python.org/issue?@action=redirect&bpo=37058]: PEP 544: Add `Protocol` and `@runtime_checkable` to the `typing` module.

- bpo-36933 [https://bugs.python.org/issue?@action=redirect&bpo=36933]: The functions `sys.set_coroutine_wrapper` and `sys.get_coroutine_wrapper` that were deprecated and marked for removal in 3.8 have been removed.

- bpo-37047 [https://bugs.python.org/issue?@action=redirect&bpo=37047]: Handle late binding and attribute access in **`unittest.mock.AsyncMock`** setup for autospeccing. Document newly implemented async methods in **`unittest.mock.MagicMock`**.

- bpo-37049 [https://bugs.python.org/issue?@action=redirect&bpo=37049]: PEP 589: Add `TypedDict` to the `typing` module.

- bpo-37046 [https://bugs.python.org/issue?@action=redirect&bpo=37046]: PEP 586: Add `Literal` to the `typing` module.

- bpo-37045 [https://bugs.python.org/issue?@action=redirect&bpo=37045]: PEP 591: Add `Final` qualifier and `@final` decorator to the `typing` module.

- bpo-37035 [https://bugs.python.org/issue?@action=redirect&bpo=37035]: Don't log OSError based exceptions if a fatal error has occurred in asyncio transport. Peer can generate almost any OSError, user cannot avoid these exceptions by fixing own code. Errors are still propagated to user code, it's just logging them is pointless and pollute asyncio logs.

- bpo-37001 [https://bugs.python.org/issue?@action=redirect&bpo=37001]: **`symtable.symtable()`** now accepts the same input types for source code as the built-in **`compile()`** function. Patch by Dino Viehland.

- bpo-37028 [https://bugs.python.org/issue?@action=redirect&bpo=37028]: Implement asyncio REPL

- bpo-37027 [https://bugs.python.org/issue?@action=redirect&bpo=37027]: Return safe to use proxy socket object from transport.get_extra_info('socket')

- bpo-32528 [https://bugs.python.org/issue?@action=redirect&bpo=32528]: Make asyncio.CancelledError a BaseException.

  This will address the common mistake many asyncio users make: an "except Exception" clause breaking Tasks cancellation.

  In addition to this change, we stop inheriting asyncio.TimeoutError and asyncio.InvalidStateError from their concurrent.futures.* counterparts. There's no point for these exceptions to share the inheritance chain.

- bpo-1230540 [https://bugs.python.org/issue?@action=redirect&bpo=1230540]: Add a new **`threading.excepthook()`** function which handles uncaught **`threading.Thread.run()`** exception. It can be overridden to control how uncaught **`threading.Thread.run()`** exceptions are handled.

- bpo-36996 [https://bugs.python.org/issue?@action=redirect&bpo=36996]: Handle **`unittest.mock.patch()`** used as a decorator on async functions.

- bpo-37008 [https://bugs.python.org/issue?@action=redirect&bpo=37008]: Add support for calling **`next()`** with the mock resulting from **`unittest.mock.mock_open()`**

- bpo-27737 [https://bugs.python.org/issue?@action=redirect&bpo=27737]: Allow whitespace only header encoding in `email.header` - by Batuhan Taskaya

- bpo-36969 [https://bugs.python.org/issue?@action=redirect&bpo=36969]: PDB command `args` now display positional only arguments. Patch contributed by Rémi Lapeyre.

- bpo-36969 [https://bugs.python.org/issue?@action=redirect&bpo=36969]: PDB command `args` now display keyword only arguments. Patch contributed by Rémi Lapeyre.

- bpo-36983 [https://bugs.python.org/issue?@action=redirect&bpo=36983]: Add missing names to `typing.__all__`: `ChainMap`, `ForwardRef`, `OrderedDict` - by Anthony Sottile.

- bpo-36972 [https://bugs.python.org/issue?@action=redirect&bpo=36972]: Add SupportsIndex protocol to the typing module to allow type checking to detect classes that can be passed to `hex()`, `oct()` and `bin()`.

- bpo-32972 [https://bugs.python.org/issue?@action=redirect&bpo=32972]: Implement `unittest.IsolatedAsyncioTestCase` to help testing asyncio-based code.

- bpo-36952 [https://bugs.python.org/issue?@action=redirect&bpo=36952]: **`fileinput.input()`** and **`fileinput.FileInput`** **bufsize** argument has been removed (was deprecated and ignored since Python 3.6), and as a result the **mode** and **openhook** arguments have been made keyword-only.

- bpo-36952 [https://bugs.python.org/issue?@action=redirect&bpo=36952]: Starting with Python 3.3, importing ABCs from **`collections`** is deprecated, and import should be done from **`collections.abc`**. Still being able to import from **`collections`** was marked for removal in 3.8, but has been delayed to 3.9; documentation and `DeprecationWarning` clarified.

- bpo-36949 [https://bugs.python.org/issue?@action=redirect&bpo=36949]: Implement __repr__ for WeakSet objects.

- bpo-36948 [https://bugs.python.org/issue?@action=redirect&bpo=36948]: Fix **`NameError`** in **`urllib.request.URLopener.retrieve()`**. Patch by Karthikeyan Singaravelan.

- bpo-33524 [https://bugs.python.org/issue?@action=redirect&bpo=33524]: Fix the folding of email header when the max_line_length is 0 or None and the header contains non-ascii characters. Contributed by Licht Takeuchi (@Licht-T).

- bpo-24564 [https://bugs.python.org/issue?@action=redirect&bpo=24564]: **`shutil.copystat()`** now ignores **`errno.EINVAL`** on **`os.setxattr()`**

which may occur when copying files on filesystems without extended attributes support.

Original patch by Giampaolo Rodola, updated by Ying Wang.

- bpo-36888 [https://bugs.python.org/issue?@action=redirect&bpo=36888]: Python child processes can now access the status of their parent process using multiprocessing.process.parent_process

- bpo-36921 [https://bugs.python.org/issue?@action=redirect&bpo=36921]: Deprecate `@coroutine` for sake of `async def`.

- bpo-25652 [https://bugs.python.org/issue?@action=redirect&bpo=25652]: Fix bug in `__rmod__` of `UserString` - by Batuhan Taskaya.

- bpo-36916 [https://bugs.python.org/issue?@action=redirect&bpo=36916]: Remove a message about an unhandled exception in a task when writer.write() is used without await and writer.drain() fails with an exception.

- bpo-36889 [https://bugs.python.org/issue?@action=redirect&bpo=36889]: Introduce **`asyncio.Stream`** class that merges **`asyncio.StreamReader`** and **`asyncio.StreamWriter`** functionality. **`asyncio.Stream`** can work in readonly, writeonly and readwrite modes. Provide **`asyncio.connect()`**, **`asyncio.connect_unix()`**, **`asyncio.connect_read_pipe()`** and **`asyncio.connect_write_pipe()`** factories to open **`asyncio.Stream`** connections. Provide **`asyncio.StreamServer`** and **`UnixStreamServer`** to serve servers with asyncio.Stream API. Modify **`asyncio.create_subprocess_shell()`** and **`asyncio.create_subprocess_exec()`** to use **`asyncio.Stream`** instead of deprecated **`StreamReader`** and **`StreamWriter`**. Deprecate **`asyncio.StreamReader`** and **`asyncio.StreamWriter`**. Deprecate usage of private classes, e.g. **`asyncio.FlowControlMixing`** and **`asyncio.StreamReaderProtocol`** outside of asyncio package.

- bpo-36845 [https://bugs.python.org/issue?@action=redirect&bpo=36845]: Added validation of integer prefixes to the construction of IP networks and interfaces in the ipaddress module.

- bpo-23378 [https://bugs.python.org/issue?@action=redirect&bpo=23378]: Add an extend action to argparser.

- bpo-36867 [https://bugs.python.org/issue?@action=redirect&bpo=36867]: Fix a bug making a SharedMemoryManager instance and its parent process use two separate resource_tracker processes.

- bpo-23896 [https://bugs.python.org/issue?@action=redirect&bpo=23896]: Adds a grammar to lib2to3.pygram that contains exec as a function not as statement.

- bpo-36895 [https://bugs.python.org/issue?@action=redirect&bpo=36895]: The function `time.clock()` was deprecated in 3.3 in favor of `time.perf_counter()` and marked for removal in 3.8, it has removed.

- bpo-35545 [https://bugs.python.org/issue?@action=redirect&bpo=35545]: Fix asyncio discarding IPv6 scopes when ensuring hostname resolutions internally

- bpo-36887 [https://bugs.python.org/issue?@action=redirect&bpo=36887]: Add new function **`math.isqrt()`** to compute integer square roots.

- bpo-34632 [https://bugs.python.org/issue?@action=redirect&bpo=34632]: Introduce the `importlib.metadata` module with (provisional) support for reading metadata from third-party packages.

- bpo-36878 [https://bugs.python.org/issue?@action=redirect&bpo=36878]: When using `type_comments=True` in `ast.parse`, treat `# type: ignore` followed by a non-alphanumeric character and then arbitrary text as a type ignore, instead of requiring nothing but whitespace or another comment. This is to permit formations such as `# type: ignore[E1000]`.

- bpo-36778 [https://bugs.python.org/issue?@action=redirect&bpo=36778]: `cp65001` encoding (Windows code page 65001) becomes an alias to `utf_8` encoding.

- bpo-36867 [https://bugs.python.org/issue?@action=redirect&bpo=36867]: The multiprocessing.resource_tracker replaces the

multiprocessing.semaphore_tracker module. Other than semaphores, resource_tracker also tracks shared_memory segments.

- bpo-30262 [https://bugs.python.org/issue?@action=redirect&bpo=30262]: The `Cache` and `Statement` objects of the **`sqlite3`** module are not exposed to the user. Patch by Aviv Palivoda.

- bpo-24538 [https://bugs.python.org/issue?@action=redirect&bpo=24538]: In `shutil.copystat()`, first copy extended file attributes and then file permissions, since extended attributes can only be set on the destination while it is still writeable.

- bpo-36829 [https://bugs.python.org/issue?@action=redirect&bpo=36829]: Add new **`sys.unraisablehook()`** function which can be overridden to control how "unraisable exceptions" are handled. It is called when an exception has occurred but there is no way for Python to handle it. For example, when a destructor raises an exception or during garbage collection (**`gc.collect()`**).

- bpo-36832 [https://bugs.python.org/issue?@action=redirect&bpo=36832]: Introducing `zipfile.Path`, a pathlib-compatible wrapper for traversing zip files.

- bpo-36814 [https://bugs.python.org/issue?@action=redirect&bpo=36814]: Fix an issue where os.posix_spawnp() would incorrectly raise a TypeError when file_actions is None.

- bpo-33110 [https://bugs.python.org/issue?@action=redirect&bpo=33110]: Handle exceptions raised by functions added by concurrent.futures add_done_callback correctly when the Future has already completed.

- bpo-26903 [https://bugs.python.org/issue?@action=redirect&bpo=26903]: Limit `max_workers` in `ProcessPoolExecutor` to 61 to work around a WaitForMultipleObjects limitation.

- bpo-36813 [https://bugs.python.org/issue?@action=redirect&bpo=36813]: Fix **`QueueListener`** to call `queue.task_done()` upon stopping. Patch by Bar Harel.

- bpo-36806 [https://bugs.python.org/issue?@action=redirect&bpo=36806]: Forbid creation of asyncio stream objects like StreamReader, StreamWriter, Process, and their protocols outside of asyncio package.

- bpo-36802 [https://bugs.python.org/issue?@action=redirect&bpo=36802]: Provide both sync and async calls for StreamWriter.write() and StreamWriter.close()

- bpo-36801 [https://bugs.python.org/issue?@action=redirect&bpo=36801]: Properly handle SSL connection closing in asyncio StreamWriter.drain() call.

- bpo-36785 [https://bugs.python.org/issue?@action=redirect&bpo=36785]: Implement PEP 574 (pickle protocol 5 with out-of-band buffers).

- bpo-36772 [https://bugs.python.org/issue?@action=redirect&bpo=36772]: functools.lru_cache() can now be used as a straight decorator in addition to its existing usage as a function that returns a decorator.

- bpo-6584 [https://bugs.python.org/issue?@action=redirect&bpo=6584]: Add a **`BadGzipFile`** exception to the **`gzip`** module.

- bpo-36748 [https://bugs.python.org/issue?@action=redirect&bpo=36748]: Optimized write buffering in C implementation of `TextIOWrapper`. Writing ASCII string to `TextIOWrapper` with ascii, latin1, or utf-8 encoding is about 20% faster. Patch by Inada Naoki.

- bpo-8138 [https://bugs.python.org/issue?@action=redirect&bpo=8138]: Don’t mark `wsgiref.simple_server.SimpleServer` as multi-threaded since `wsgiref.simple_server.WSGIServer` is single-threaded.

- bpo-22640 [https://bugs.python.org/issue?@action=redirect&bpo=22640]: **`py_compile.compile()`** now supports silent mode. Patch by Joannah Nanjekye

- bpo-29183 [https://bugs.python.org/issue?@action=redirect&bpo=29183]: Fix double exceptions in **`wsgiref.handlers.BaseHandler`** by calling its **`close()`** method only when no exception is raised.

- bpo-36548 [https://bugs.python.org/issue?@action=redirect&bpo=36548]: Improved the repr of regular expression flags.

- bpo-36542 [https://bugs.python.org/issue?@action=redirect&bpo=36542]: The signature of Python functions can now be overridden by specifying the `__text_signature__` attribute.

- bpo-36533 [https://bugs.python.org/issue?@action=redirect&bpo=36533]: Reinitialize logging.Handler locks in forked child processes instead of attempting to acquire them all in the parent before forking only to be released in the child process. The acquire/release pattern was leading to deadlocks in code that has implemented any form of chained logging handlers that depend upon one another as the lock acquisition order cannot be guaranteed.

- bpo-35252 [https://bugs.python.org/issue?@action=redirect&bpo=35252]: Throw a TypeError instead of an AssertionError when using an invalid type annotation with singledispatch.

- bpo-35900 [https://bugs.python.org/issue?@action=redirect&bpo=35900]: Allow reduction methods to return a 6-item tuple where the 6th item specifies a custom state-setting method that's called instead of the regular `__setstate__` method.

- bpo-35900 [https://bugs.python.org/issue?@action=redirect&bpo=35900]: enable custom reduction callback registration for functions and classes in _pickle.c, using the new Pickler's attribute `reducer_override`

- bpo-36368 [https://bugs.python.org/issue?@action=redirect&bpo=36368]: Fix a bug crashing SharedMemoryManager instances in interactive sessions after a ctrl-c (KeyboardInterrupt) was sent

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix mmap fail for VxWorks

- bpo-27497 [https://bugs.python.org/issue?@action=redirect&bpo=27497]: **`csv.DictWriter.writeheader()`** now returns the return value of the

underlying **`csv.Writer.writerow()`** method. Patch contributed by Ashish Nitin Patil.

- bpo-36239 [https://bugs.python.org/issue?@action=redirect&bpo=36239]: Parsing .mo files now ignores comments starting and ending with #-#-#-#-#.

- bpo-26707 [https://bugs.python.org/issue?@action=redirect&bpo=26707]: Enable plistlib to read and write binary plist files that were created as a KeyedArchive file. Specifically, this allows the plistlib to process 0x80 tokens as UID objects.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Add posix module support for VxWorks.

- bpo-35125 [https://bugs.python.org/issue?@action=redirect&bpo=35125]: Asyncio: Remove inner callback on outer cancellation in shield

- bpo-35721 [https://bugs.python.org/issue?@action=redirect&bpo=35721]: Fix **`asyncio.SelectorEventLoop.subprocess_exec()`** leaks file descriptors if `Popen` fails and called with `stdin=subprocess.PIPE`. Patch by Niklas Fiekas.

- bpo-31855 [https://bugs.python.org/issue?@action=redirect&bpo=31855]: **`unittest.mock.mock_open()`** results now respects the argument of read([size]). Patch contributed by Rémi Lapeyre.

- bpo-35431 [https://bugs.python.org/issue?@action=redirect&bpo=35431]: Implement **`math.comb()`** that returns binomial coefficient, that computes the number of ways to choose k items from n items without repetition and without order. Patch by Yash Aggarwal and Keller Fuchs.

- bpo-26660 [https://bugs.python.org/issue?@action=redirect&bpo=26660]: Fixed permission errors in **`TemporaryDirectory`** clean up. Previously `TemporaryDirectory.cleanup()` failed when non-writeable or non-searchable files or directories were created inside a temporary directory.

- bpo-34271 [https://bugs.python.org/issue?@action=redirect&bpo=34271]: Add debugging helpers to ssl module. It's now possible to dump key material and to trace TLS protocol. The default and stdlib contexts also support SSLKEYLOGFILE env var.

- bpo-26467 [https://bugs.python.org/issue?@action=redirect&bpo=26467]: Added AsyncMock to support using unittest to mock asyncio coroutines. Patch by Lisa Roach.

- bpo-33569 [https://bugs.python.org/issue?@action=redirect&bpo=33569]: dataclasses.InitVar: Exposes the type used to create the init var.

- bpo-34424 [https://bugs.python.org/issue?@action=redirect&bpo=34424]: Fix serialization of messages containing encoded strings when the policy.linesep is set to a multi-character string. Patch by Jens Troeger.

- bpo-34303 [https://bugs.python.org/issue?@action=redirect&bpo=34303]: Performance of `functools.reduce()` is slightly improved. Patch by Sergey Fedoseev.

- bpo-33361 [https://bugs.python.org/issue?@action=redirect&bpo=33361]: Fix a bug in `codecs.StreamRecoder` where seeking might leave old data in a buffer and break subsequent read calls. Patch by Ammar Askar.

- bpo-22454 [https://bugs.python.org/issue?@action=redirect&bpo=22454]: The `shlex` module now exposes `shlex.join()`, the inverse of `shlex.split()`. Patch by Bo Bayles.

- bpo-31922 [https://bugs.python.org/issue?@action=redirect&bpo=31922]: `asyncio.AbstractEventLoop.create_datagram_endpoint()`: Do not connect UDP socket when broadcast is allowed. This allows to receive replies after a UDP broadcast.

- bpo-24882 [https://bugs.python.org/issue?@action=redirect&bpo=24882]: Change ThreadPoolExecutor to use existing idle threads before spinning up new ones.

- bpo-31961 [https://bugs.python.org/issue?@action=redirect&bpo=31961]: Added support for bytes and path-like objects in **`subprocess.Popen()`** on Windows. The *args* parameter now accepts a path-like object if *shell* is `False` and a sequence containing bytes and path-like objects. The *executable* parameter now accepts a bytes and path-like object. The *cwd* parameter now accepts a bytes object. Based on patch by Anders Lorentsen.

- bpo-33123 [https://bugs.python.org/issue?@action=redirect&bpo=33123]: **`pathlib.Path.unlink`** now accepts a *missing_ok* parameter to avoid a **`FileNotFoundError`** from being raised. Patch by Robert Buchholz.

- bpo-32941 [https://bugs.python.org/issue?@action=redirect&bpo=32941]: Allow **`mmap.mmap`** objects to access the madvise() system call (through **`mmap.mmap.madvise()`**).

- bpo-22102 [https://bugs.python.org/issue?@action=redirect&bpo=22102]: Added support for ZIP files with disks set to 0. Such files are commonly created by builtin tools on Windows when use ZIP64 extension. Patch by Francisco Facioni.

- bpo-32515 [https://bugs.python.org/issue?@action=redirect&bpo=32515]: trace.py can now run modules via python3 -m trace -t –module module_name

- bpo-32299 [https://bugs.python.org/issue?@action=redirect&bpo=32299]: Changed **`unittest.mock.patch.dict()`** to return the patched dictionary when used as context manager. Patch by Vadim Tsander.

- bpo-27141 [https://bugs.python.org/issue?@action=redirect&bpo=27141]: Added a `__copy__()` to `collections.UserList` and `collections.UserDict` in order to correctly implement shallow copying of the objects. Patch by Bar Harel.

- bpo-31829 [https://bugs.python.org/issue?@action=redirect&bpo=31829]: `\r`, `\0` and `\x1a` (end-of-file on Windows) are now escaped in protocol 0 pickles of Unicode strings. This allows to load them without loss from files open in text mode in Python 2.

- bpo-23395 [https://bugs.python.org/issue?@action=redirect&bpo=23395]: `_thread.interrupt_main()` now avoids setting the Python error status if the `SIGINT` signal is ignored or not handled by Python.

Documentação

- bpo-36896 [https://bugs.python.org/issue?@action=redirect&bpo=36896]: Clarify that some types have unstable constructor signature between Python versions.

- bpo-36686 [https://bugs.python.org/issue?@action=redirect&bpo=36686]: Improve documentation of the stdin, stdout, and stderr arguments of the `asyncio.subprocess_exec` function to specify which values are supported. Also mention that decoding as text is not supported.

  Add a few tests to verify that the various values passed to the std* arguments actually work.

- bpo-36984 [https://bugs.python.org/issue?@action=redirect&bpo=36984]: Improve version added references in `typing` module - by Anthony Sottile.

- bpo-36868 [https://bugs.python.org/issue?@action=redirect&bpo=36868]: What's new now mentions SSLContext.hostname_checks_common_name instead of SSLContext.host_flags.

- bpo-35924 [https://bugs.python.org/issue?@action=redirect&bpo=35924]: Add a note to the `curses.addstr()` documentation to warn that multiline strings can cause segfaults because of an ncurses bug.

- bpo-36783 [https://bugs.python.org/issue?@action=redirect&bpo=36783]: Added C API Documentation for Time_FromTimeAndFold and PyDateTime_FromDateAndTimeAndFold as per PEP 495. Patch by Edison Abahurire.

- bpo-36797 [https://bugs.python.org/issue?@action=redirect&bpo=36797]: More of the legacy distutils documentation has been either pruned, or else more clearly marked as being retained solely until the setuptools documentation covers it independently.

- bpo-22865 [https://bugs.python.org/issue?@action=redirect&bpo=22865]: Add detail to the documentation on the `pty.spawn` function.

- bpo-35397 [https://bugs.python.org/issue?@action=redirect&bpo=35397]: Remove deprecation and document urllib.parse.unwrap(). Patch contributed by Rémi Lapeyre.

- bpo-32995 [https://bugs.python.org/issue?@action=redirect&bpo=32995]: Added the context variable in glossary.

- bpo-33519 [https://bugs.python.org/issue?@action=redirect&bpo=33519]: Clarify that `copy()` is not part of the `MutableSequence` ABC.

- bpo-33482 [https://bugs.python.org/issue?@action=redirect&bpo=33482]: Make `codecs.StreamRecoder.writelines` take a list of bytes.

- bpo-25735 [https://bugs.python.org/issue?@action=redirect&bpo=25735]: Added documentation for func factorial to indicate that returns integer values

- bpo-20285 [https://bugs.python.org/issue?@action=redirect&bpo=20285]: Expand object.__doc__ (docstring) to make it clearer. Modify pydoc.py so that help(object) lists object methods (for other classes, help omits methods of the object base class.)

Testes

- bpo-37069 [https://bugs.python.org/issue?@action=redirect&bpo=37069]: Modify test_coroutines, test_cprofile, test_generators, test_raise, test_ssl and test_yield_from to use **`test.support.catch_unraisable_exception()`** rather than **`test.support.captured_stderr()`**.
- bpo-37098 [https://bugs.python.org/issue?@action=redirect&bpo=37098]: Fix test_memfd_create on older Linux Kernels.
- bpo-37081 [https://bugs.python.org/issue?@action=redirect&bpo=37081]: Test with OpenSSL 1.1.1c
- bpo-36829 [https://bugs.python.org/issue?@action=redirect&bpo=36829]: Add **`test.support.catch_unraisable_exception()`**: context manager catching unraisable exception using **`sys.unraisablehook()`**.

- bpo-36915 [https://bugs.python.org/issue?@action=redirect&bpo=36915]: The main regrtest process now always removes all temporary directories of worker processes even if they crash or if they are killed on KeyboardInterrupt (CTRL+c).
- bpo-36719 [https://bugs.python.org/issue?@action=redirect&bpo=36719]: “python3 -m test -jN …” now continues the execution of next tests when a worker process crash (CHILD_ERROR state). Previously, the test suite stopped immediately. Use –failfast to stop at the first error.
- bpo-36816 [https://bugs.python.org/issue?@action=redirect&bpo=36816]: Update Lib/test/selfsigned_pythontestdotnet.pem to match self-signed.pythontest.net’s new TLS certificate.
- bpo-35925 [https://bugs.python.org/issue?@action=redirect&bpo=35925]: Skip httplib and nntplib networking tests when they would otherwise fail due to a modern OS or distro with a default OpenSSL policy of rejecting connections to servers with weak certificates.
- bpo-36782 [https://bugs.python.org/issue?@action=redirect&bpo=36782]: Add tests for several C API functions in the **`datetime`** module. Patch by Edison Abahurire.
- bpo-36342 [https://bugs.python.org/issue?@action=redirect&bpo=36342]: Fix test_multiprocessing in test_venv if platform lacks functioning sem_open.

Construção

- bpo-36721 [https://bugs.python.org/issue?@action=redirect&bpo=36721]: To embed Python into an application, a new `--embed` option must be passed to `python3-config --libs --embed` to get `-lpython3.8` (link the application to libpython). To support both 3.8 and older, try `python3-config --libs --embed` first and fallback to `python3-config --libs` (without `--embed`) if the previous command fails.

  Adicione um módulo pkg-config `python-3.8-embed` para incorporar o Python em uma aplicação: `pkg-config python-3.8-embed --libs` inclui `-lpython3.8`. Para suportar tanto o 3.8 quanto o anterior, tente `pkg-config python-X.Y-embed --libs` primeiro e use como fallback `pkg-config python-X.Y --libs` (sem `--embed`) se o comando anterior falhar (substitua `X.Y` pela versão do Python).

On the other hand, `pkg-config python3.8 --libs` no longer contains `-lpython3.8`. C extensions must not be linked to libpython (except on Android, case handled by the script); this change is backward incompatible on purpose.

- bpo-36786 [https://bugs.python.org/issue?@action=redirect&bpo=36786]: "make install" now runs compileall in parallel.

Windows

- bpo-36965 [https://bugs.python.org/issue?@action=redirect&bpo=36965]: include of STATUS_CONTROL_C_EXIT without depending on MSC compiler
- bpo-35926 [https://bugs.python.org/issue?@action=redirect&bpo=35926]: Update to OpenSSL 1.1.1b for Windows.
- bpo-29883 [https://bugs.python.org/issue?@action=redirect&bpo=29883]: Add Windows support for UDP transports for the Proactor Event Loop. Patch by Adam Meily.
- bpo-33407 [https://bugs.python.org/issue?@action=redirect&bpo=33407]: The `Py_DEPRECATED()` macro has been implemented for MSVC.

macOS

- bpo-36231 [https://bugs.python.org/issue?@action=redirect&bpo=36231]: Support building Python on macOS without /usr/include installed. As of macOS 10.14, system header files are only available within an SDK provided by either the Command Line Tools or the Xcode app.

IDLE

- bpo-35610 [https://bugs.python.org/issue?@action=redirect&bpo=35610]: Replace now redundant .context_use_ps1 with .prompt_last_line. This finishes change started in bpo-31858 [https://bugs.python.org/issue?@action=redirect&bpo=31858].
- bpo-37038 [https://bugs.python.org/issue?@action=redirect&bpo=37038]: Make idlelib.run runnable; add test clause.
- bpo-36958 [https://bugs.python.org/issue?@action=redirect&bpo=36958]: Print any argument other than None or int passed to SystemExit or sys.exit().

- bpo-36807 [https://bugs.python.org/issue?@action=redirect&bpo=36807]: When saving a file, call os.fsync() so bits are flushed to e.g. USB drive.
- bpo-32411 [https://bugs.python.org/issue?@action=redirect&bpo=32411]: In browser.py, remove extraneous sorting by line number since dictionary was created in line number order.

Ferramentas/Demos

- bpo-37053 [https://bugs.python.org/issue?@action=redirect&bpo=37053]: Handle strings like u"bar" correctly in Tools/parser/unparse.py. Patch by Chih-Hsuan Yen.

API C

- bpo-36763 [https://bugs.python.org/issue?@action=redirect&bpo=36763]: Implement the **PEP 587** [https://peps.python.org/pep-0587/] "Python Initialization Configuration".
- bpo-36379 [https://bugs.python.org/issue?@action=redirect&bpo=36379]: Fix crashes when attempting to use the *modulo* parameter when `__ipow__` is implemented in C.
- bpo-37107 [https://bugs.python.org/issue?@action=redirect&bpo=37107]: Update **`PyObject_CallMethodObjArgs()`** and `_PyObject_CallMethodIdObjArgs` to use `_PyObject_GetMethod` to avoid creating a bound method object in many cases. Patch by Michael J. Sullivan.
- bpo-36974 [https://bugs.python.org/issue?@action=redirect&bpo=36974]: Implement **PEP 590** [https://peps.python.org/pep-0590/]: Vectorcall: a fast calling protocol for CPython. This is a new protocol to optimize calls of custom callable objects.
- bpo-36763 [https://bugs.python.org/issue?@action=redirect&bpo=36763]: `Py_Main()` now returns the exitcode rather than calling `Py_Exit(exitcode)` when calling `PyErr_Print()` if the current exception type is `SystemExit`.
- bpo-36922 [https://bugs.python.org/issue?@action=redirect&bpo=36922]: Add new type flag `Py_TPFLAGS_METHOD_DESCRIPTOR` for objects behaving like unbound methods. These are objects supporting the optimization given by the `LOAD_METHOD`/`CALL_METHOD` opcodes. See PEP 590.

- bpo-36728 [https://bugs.python.org/issue?@action=redirect&bpo=36728]: The **`PyEval_ReInitThreads()`** function has been removed from the C API. It should not be called explicitly: use **`PyOS_AfterFork_Child()`** instead.

Python 3.8.0 alpha 4

Data de lançamento: 2019-05-06

Segurança

- bpo-36742 [https://bugs.python.org/issue?@action=redirect&bpo=36742]: Fixes mishandling of pre-normalization characters in urlsplit().
- bpo-30458 [https://bugs.python.org/issue?@action=redirect&bpo=30458]: Address **CVE 2019-9740** [https://www.cve.org/CVERecord?id=CVE-2019-9740] by disallowing URL paths with embedded whitespace or control characters through into the underlying http client request. Such potentially malicious header injection URLs now cause an http.client.InvalidURL exception to be raised.
- bpo-35755 [https://bugs.python.org/issue?@action=redirect&bpo=35755]: **`shutil.which()`** now uses `os.confstr("CS_PATH")` if available and if the **`PATH`** environment variable is not set. Remove also the current directory from **`posixpath.defpath`**. On Unix, **`shutil.which()`** and the **`subprocess`** module no longer search the executable in the current directory if the **`PATH`** environment variable is not set.

Núcleo e embutidos

- bpo-36722 [https://bugs.python.org/issue?@action=redirect&bpo=36722]: In debug build, import now also looks for C extensions compiled in release mode and for C extensions compiled in the stable ABI.
- bpo-32849 [https://bugs.python.org/issue?@action=redirect&bpo=32849]: Fix Python Initialization code on FreeBSD to detect properly when stdin file descriptor (fd 0) is invalid.
- bpo-36623 [https://bugs.python.org/issue?@action=redirect&bpo=36623]: Remove parser headers and related function declarations that lack

implementations after the removal of pgen.

- bpo-20180 [https://bugs.python.org/issue?@action=redirect&bpo=20180]: `dict.pop()` is now up to 33% faster thanks to Argument Clinic. Patch by Inada Naoki.
- bpo-36611 [https://bugs.python.org/issue?@action=redirect&bpo=36611]: Debug memory allocators: disable serialno field by default from debug hooks on Python memory allocators to reduce the memory footprint by 5%. Enable **`tracemalloc`** to get the traceback where a memory block has been allocated when a fatal memory error is logged to decide where to put a breakpoint. Compile Python with `PYMEM_DEBUG_SERIALNO` defined to get back the field.
- bpo-36588 [https://bugs.python.org/issue?@action=redirect&bpo=36588]: On AIX, **`sys.platform`** doesn't contain the major version anymore. Always return `'aix'`, instead of `'aix3'` .. `'aix7'`. Since older Python versions include the version number, it is recommended to always use `sys.platform.startswith('aix')`. Contributed by M. Felt.
- bpo-36549 [https://bugs.python.org/issue?@action=redirect&bpo=36549]: Change str.capitalize to use titlecase for the first character instead of uppercase.
- bpo-36540 [https://bugs.python.org/issue?@action=redirect&bpo=36540]: Implement **PEP 570** [https://peps.python.org/pep-0570/] (Python positional-only parameters). Patch by Pablo Galindo.
- bpo-36475 [https://bugs.python.org/issue?@action=redirect&bpo=36475]: **`PyEval_AcquireLock()`** and **`PyEval_AcquireThread()`** now terminate the current thread if called while the interpreter is finalizing, making them consistent with **`PyEval_RestoreThread()`**, **`Py_END_ALLOW_THREADS()`**, and **`PyGILState_Ensure()`**.
- bpo-36504 [https://bugs.python.org/issue?@action=redirect&bpo=36504]: Fix signed integer overflow in _ctypes.c's `PyCArrayType_new()`.
- bpo-20844 [https://bugs.python.org/issue?@action=redirect&bpo=20844]: Fix running script with encoding cookie and LF line ending may fail on Windows.
- bpo-24214 [https://bugs.python.org/issue?@action=redirect&bpo=24214]: Fixed support of the surrogatepass error handler in the UTF-8 incremental decoder.
- bpo-36452 [https://bugs.python.org/issue?@action=redirect&bpo=36452]: Changing `dict` keys during iteration of the dict itself, `keys()`, `values()`, or `items()` will now be detected in certain corner cases where keys are deleted/added so that the number of keys isn't changed. A

`RuntimeError` will be raised after `len(dict)` iterations. Contributed by Thomas Perl.

- bpo-36459 [https://bugs.python.org/issue?@action=redirect&bpo=36459]: Fix a possible double `PyMem_FREE()` due to tokenizer.c's `tok_nextc()`.
- bpo-36433 [https://bugs.python.org/issue?@action=redirect&bpo=36433]: Fixed TypeError message in classmethoddescr_call.
- bpo-36430 [https://bugs.python.org/issue?@action=redirect&bpo=36430]: Fix a possible reference leak in **`itertools.count()`**.
- bpo-36440 [https://bugs.python.org/issue?@action=redirect&bpo=36440]: Include node names in `ParserError` messages, instead of numeric IDs. Patch by A. Skrobov.
- bpo-36143 [https://bugs.python.org/issue?@action=redirect&bpo=36143]: Regenerate **`keyword`** from the Grammar and Tokens file using pgen. Patch by Pablo Galindo.
- bpo-18372 [https://bugs.python.org/issue?@action=redirect&bpo=18372]: Add missing **`PyObject_GC_Track()`** calls in the **`pickle`** module. Patch by Zackery Spytz.

Biblioteca

- bpo-35952 [https://bugs.python.org/issue?@action=redirect&bpo=35952]: Fix pythoninfo when the compiler is missing.
- bpo-28238 [https://bugs.python.org/issue?@action=redirect&bpo=28238]: The `.find*()` methods of xml.etree.ElementTree can now search for wildcards like `{*}tag` and `{ns}*` that match a tag in any namespace or all tags in a namespace. Patch by Stefan Behnel.
- bpo-26978 [https://bugs.python.org/issue?@action=redirect&bpo=26978]: `pathlib.path.link_to()` is now implemented. It creates a hard link pointing to a path.
- bpo-1613500 [https://bugs.python.org/issue?@action=redirect&bpo=1613500]: **`fileinput.FileInput`** now uses the input file mode to correctly set the output file mode (previously it was hardcoded to `'w'`) when `inplace=True` is passed to its constructor.
- bpo-36734 [https://bugs.python.org/issue?@action=redirect&bpo=36734]: Fix compilation of `faulthandler.c` on HP-UX. Initialize `stack_t current_stack` to zero using `memset()`.

- bpo-13611 [https://bugs.python.org/issue?@action=redirect&bpo=13611]: The xml.etree.ElementTree packages gained support for C14N 2.0 serialisation. Patch by Stefan Behnel.
- bpo-36669 [https://bugs.python.org/issue?@action=redirect&bpo=36669]: Add missing matrix multiplication operator support to weakref.proxy.
- bpo-36676 [https://bugs.python.org/issue?@action=redirect&bpo=36676]: The XMLParser() in xml.etree.ElementTree provides namespace prefix context to the parser target if it defines the callback methods “start_ns()” and/or “end_ns()”. Patch by Stefan Behnel.
- bpo-36673 [https://bugs.python.org/issue?@action=redirect&bpo=36673]: The TreeBuilder and XMLPullParser in xml.etree.ElementTree gained support for parsing comments and processing instructions. Patch by Stefan Behnel.
- bpo-36650 [https://bugs.python.org/issue?@action=redirect&bpo=36650]: The C version of functools.lru_cache() was treating calls with an empty `**kwargs` dictionary as being distinct from calls with no keywords at all. This did not result in an incorrect answer, but it did trigger an unexpected cache miss.
- bpo-28552 [https://bugs.python.org/issue?@action=redirect&bpo=28552]: Fix `distutils.sysconfig` if **`sys.executable`** is `None` or an empty string: use **`os.getcwd()`** to initialize `project_base`. Fix also the distutils build command: don’t use **`sys.executable`** if it is `None` or an empty string.
- bpo-35755 [https://bugs.python.org/issue?@action=redirect&bpo=35755]: **`shutil.which()`** and `distutils.spawn.find_executable` now use `os.confstr("CS_PATH")` if available instead of **`os.defpath`**, if the `PATH` environment variable is not set. Moreover, don’t use `os.confstr("CS_PATH")` nor **`os.defpath`** if the `PATH` environment variable is set to an empty string.
- bpo-25430 [https://bugs.python.org/issue?@action=redirect&bpo=25430]: improve performance of `IPNetwork.__contains__()`
- bpo-30485 [https://bugs.python.org/issue?@action=redirect&bpo=30485]: Path expressions in xml.etree.ElementTree can now avoid explicit namespace prefixes for tags (or the “{namespace}tag” notation) by passing a default namespace with an empty string prefix.
- bpo-36613 [https://bugs.python.org/issue?@action=redirect&bpo=36613]: Fix **`asyncio`** wait() not removing callback if exception

- bpo-36598 [https://bugs.python.org/issue?@action=redirect&bpo=36598]: Fix `isinstance` check for Mock objects with spec when the code is executed under tracing. Patch by Karthikeyan Singaravelan.
- bpo-18748 [https://bugs.python.org/issue?@action=redirect&bpo=18748]: In development mode (**`-X`** `dev`) and in debug build, the **`io.IOBase`** destructor now logs `close()` exceptions. These exceptions are silent by default in release mode.
- bpo-36575 [https://bugs.python.org/issue?@action=redirect&bpo=36575]: The `_lsprof` module now uses internal timer same to `time.perf_counter()` by default. `gettimeofday(2)` was used on Unix. New timer has better resolution on most Unix platforms and timings are no longer impacted by system clock updates since `perf_counter()` is monotonic. Patch by Inada Naoki.
- bpo-33461 [https://bugs.python.org/issue?@action=redirect&bpo=33461]: `json.loads` now emits `DeprecationWarning` when `encoding` option is specified. Patch by Matthias Bussonnier.
- bpo-36559 [https://bugs.python.org/issue?@action=redirect&bpo=36559]: The random module now prefers the lean internal _sha512 module over hashlib for seed(version=2) to optimize import time.
- bpo-17561 [https://bugs.python.org/issue?@action=redirect&bpo=17561]: Set backlog=None as the default for socket.create_server.
- bpo-34373 [https://bugs.python.org/issue?@action=redirect&bpo=34373]: Fix **`time.mktime()`** error handling on AIX for year before 1970.
- bpo-36232 [https://bugs.python.org/issue?@action=redirect&bpo=36232]: Improve error message when trying to open existing DBM database that actually doesn't exist. Patch by Marco Rougeth.
- bpo-36546 [https://bugs.python.org/issue?@action=redirect&bpo=36546]: Add statistics.quantiles()
- bpo-36050 [https://bugs.python.org/issue?@action=redirect&bpo=36050]: Optimized `http.client.HTTPResponse.read()` for large response. Patch by Inada Naoki.
- bpo-36522 [https://bugs.python.org/issue?@action=redirect&bpo=36522]: If *debuglevel* is set to >0 in **`http.client`**, print all values for headers with multiple values for the same header name. Patch by Matt Houglum.
- bpo-36492 [https://bugs.python.org/issue?@action=redirect&bpo=36492]: Deprecated passing required arguments like *func* as keyword arguments in

functions which should accept arbitrary keyword arguments and pass them to other function. Arbitrary keyword arguments (even with names “self” and “func”) can now be passed to these functions if the required arguments are passed as positional arguments.

- bpo-27181 [https://bugs.python.org/issue?@action=redirect&bpo=27181]: Add statistics.geometric_mean().
- bpo-30427 [https://bugs.python.org/issue?@action=redirect&bpo=30427]: `os.path.normcase()` relies on `os.fspath()` to check the type of its argument. Redundant checks have been removed from its `posixpath.normcase()` and `ntpath.normcase()` implementations. Patch by Wolfgang Maier.
- bpo-36385 [https://bugs.python.org/issue?@action=redirect&bpo=36385]: Stop rejecting IPv4 octets for being ambiguously octal. Leading zeros are ignored, and no longer are assumed to specify octal octets. Octets are always decimal numbers. Octets must still be no more than three digits, including leading zeroes.
- bpo-36434 [https://bugs.python.org/issue?@action=redirect&bpo=36434]: Errors during writing to a ZIP file no longer prevent to properly close it.
- bpo-36407 [https://bugs.python.org/issue?@action=redirect&bpo=36407]: Fixed wrong indentation writing for CDATA section in xml.dom.minidom. Patch by Vladimir Surjaninov.
- bpo-36326 [https://bugs.python.org/issue?@action=redirect&bpo=36326]: inspect.getdoc() can now find docstrings for member objects when __slots__ is a dictionary.
- bpo-36366 [https://bugs.python.org/issue?@action=redirect&bpo=36366]: Calling `stop()` on an unstarted or stopped **`unittest.mock.patch()`** object will now return `None` instead of raising **`RuntimeError`**, making the method idempotent. Patch by Karthikeyan Singaravelan.
- bpo-36348 [https://bugs.python.org/issue?@action=redirect&bpo=36348]: The **`imap.IMAP4.logout()`** method no longer ignores silently arbitrary exceptions.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Add time module support and fix test_time failures for VxWorks.
- bpo-36227 [https://bugs.python.org/issue?@action=redirect&bpo=36227]: Added support for keyword arguments `default_namespace` and `xml_declaration` in functions `ElementTree.tostring()` and `ElementTree.tostringlist()`.

- bpo-36004 [https://bugs.python.org/issue?@action=redirect&bpo=36004]: Added new alternate constructors `datetime.date.fromisocalendar()` and `datetime.datetime.fromisocalendar()`, which construct date objects from ISO year, week number and weekday; these are the inverse of each class's `isocalendar` method. Patch by Paul Ganssle.
- bpo-35936 [https://bugs.python.org/issue?@action=redirect&bpo=35936]: `modulefinder` no longer depends on the deprecated `imp` module, and the initializer for `modulefinder.ModuleFinder` now has immutable default arguments. Patch by Brandt Bucher.
- bpo-35376 [https://bugs.python.org/issue?@action=redirect&bpo=35376]: `modulefinder` correctly handles modules that have the same name as a bad package. Patch by Brandt Bucher.
- bpo-17396 [https://bugs.python.org/issue?@action=redirect&bpo=17396]: `modulefinder` no longer crashes when encountering syntax errors in followed imports. Patch by Brandt Bucher.
- bpo-35934 [https://bugs.python.org/issue?@action=redirect&bpo=35934]: Added `create_server()` and `has_dualstack_ipv6()` convenience functions to automate the necessary tasks usually involved when creating a server socket, including accepting both IPv4 and IPv6 connections on the same socket. (Contributed by Giampaolo Rodola in bpo-17561 [https://bugs.python.org/issue?@action=redirect&bpo=17561].)
- bpo-23078 [https://bugs.python.org/issue?@action=redirect&bpo=23078]: Add support for `classmethod()` and `staticmethod()` to `unittest.mock.create_autospec()`. Initial patch by Felipe Ochoa.
- bpo-35416 [https://bugs.python.org/issue?@action=redirect&bpo=35416]: Fix potential resource warnings in distutils. Patch by Mickaël Schoentgen.
- bpo-25451 [https://bugs.python.org/issue?@action=redirect&bpo=25451]: Add transparency methods to `tkinter.PhotoImage`. Patch by Zackery Spytz.
- bpo-35082 [https://bugs.python.org/issue?@action=redirect&bpo=35082]: Don't return deleted attributes when calling dir on a `unittest.mock.Mock`.
- bpo-34547 [https://bugs.python.org/issue?@action=redirect&bpo=34547]: `wsgiref.handlers.BaseHandler` now handles abrupt client connection terminations gracefully. Patch by Petter Strandmark.
- bpo-31658 [https://bugs.python.org/issue?@action=redirect&bpo=31658]: `xml.sax.parse()` now supports path-like. Patch by Mickaël Schoentgen.

- bpo-34139 [https://bugs.python.org/issue?@action=redirect&bpo=34139]: Remove stale unix datagram socket before binding
- bpo-33530 [https://bugs.python.org/issue?@action=redirect&bpo=33530]: Implemented Happy Eyeballs in `asyncio.create_connection()`. Added two new arguments, *happy_eyeballs_delay* and *interleave*, to specify Happy Eyeballs behavior.
- bpo-33291 [https://bugs.python.org/issue?@action=redirect&bpo=33291]: Do not raise AttributeError when calling the inspect functions isgeneratorfunction, iscoroutinefunction, isasyncgenfunction on a method created from an arbitrary callable. Instead, return False.
- bpo-31310 [https://bugs.python.org/issue?@action=redirect&bpo=31310]: Fix the multiprocessing.semaphore_tracker so it is reused by child processes
- bpo-31292 [https://bugs.python.org/issue?@action=redirect&bpo=31292]: Fix `setup.py check --restructuredtext` for files containing `include` directives.

Documentação

- bpo-36625 [https://bugs.python.org/issue?@action=redirect&bpo=36625]: Remove obsolete comments from docstrings in fractions.Fraction
- bpo-30840 [https://bugs.python.org/issue?@action=redirect&bpo=30840]: Document relative imports
- bpo-36523 [https://bugs.python.org/issue?@action=redirect&bpo=36523]: Add docstring for io.IOBase.writelines().
- bpo-36425 [https://bugs.python.org/issue?@action=redirect&bpo=36425]: New documentation translation: Simplified Chinese [https://docs.python.org/zh-cn/].
- bpo-36345 [https://bugs.python.org/issue?@action=redirect&bpo=36345]: Avoid the duplication of code from `Tools/scripts/serve.py` in using the **`literalinclude`** directive for the basic wsgiref-based web server in the documentation of **`wsgiref`**. Contributed by Stéphane Wirtel.
- bpo-36345 [https://bugs.python.org/issue?@action=redirect&bpo=36345]: Using the code of the `Tools/scripts/serve.py` script as an example in the **`wsgiref`** documentation. Contributed by Stéphane Wirtel.
- bpo-36157 [https://bugs.python.org/issue?@action=redirect&bpo=36157]: Added documentation for PyInterpreterState_Main().

- bpo-33043 [https://bugs.python.org/issue?@action=redirect&bpo=33043]: Updates the docs.python.org page with the addition of a ‘Contributing to Docs’ link at the end of the page (between ‘Reporting Bugs’ and ‘About Documentation’). Updates the ‘Found a Bug’ page with additional links and information in the Documentation Bugs section.
- bpo-35581 [https://bugs.python.org/issue?@action=redirect&bpo=35581]: @typing.type_check_only now allows type stubs to mark functions and classes not available during runtime.
- bpo-33832 [https://bugs.python.org/issue?@action=redirect&bpo=33832]: Add glossary entry for ‘magic method’.
- bpo-32913 [https://bugs.python.org/issue?@action=redirect&bpo=32913]: Added re.Match.groupdict example to regex HOWTO.

Testes

- bpo-36719 [https://bugs.python.org/issue?@action=redirect&bpo=36719]: regrtest now always detects uncollectable objects. Previously, the check was only enabled by `--findleaks`. The check now also works with `-jN/--multiprocess N`. `--findleaks` becomes a deprecated alias to `--fail-env-changed`.
- bpo-36725 [https://bugs.python.org/issue?@action=redirect&bpo=36725]: When using multiprocessing mode (-jN), regrtest now better reports errors if a worker process fails, and it exits immediately on a worker thread failure or when interrupted.
- bpo-36454 [https://bugs.python.org/issue?@action=redirect&bpo=36454]: Change test_time.test_monotonic() to test only the lower bound of elapsed time after a sleep command rather than the upper bound. This prevents unnecessary test failures on slow buildbots. Patch by Victor Stinner.
- bpo-32424 [https://bugs.python.org/issue?@action=redirect&bpo=32424]: Improve test coverage for xml.etree.ElementTree. Patch by Gordon P. Hemsley.
- bpo-32424 [https://bugs.python.org/issue?@action=redirect&bpo=32424]: Fix typo in test_cyclic_gc() test for xml.etree.ElementTree. Patch by Gordon P. Hemsley.
- bpo-36635 [https://bugs.python.org/issue?@action=redirect&bpo=36635]: Add a new **`_testinternalcapi`** module to test the internal C API.
- bpo-36629 [https://bugs.python.org/issue?@action=redirect&bpo=36629]: Fix `test_imap4_host_default_value()` of `test_imaplib`: catch also

`errno.ENETUNREACH` error.

- bpo-36611 [https://bugs.python.org/issue?@action=redirect&bpo=36611]: Fix `test_sys.test_getallocatedblocks()` when `tracemalloc` is enabled.
- bpo-36560 [https://bugs.python.org/issue?@action=redirect&bpo=36560]: Fix reference leak hunting in regrtest: compute also deltas (of reference count, allocated memory blocks, file descriptor count) during warmup, to ensure that everything is initialized before starting to hunt reference leaks.
- bpo-36565 [https://bugs.python.org/issue?@action=redirect&bpo=36565]: Fix reference hunting (`python3 -m test -R 3:3`) when Python has no built-in abc module.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Port test_resource to VxWorks: skip tests cases setting RLIMIT_FSIZE and RLIMIT_CPU.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix test_tabnanny on VxWorks: adjust ENOENT error message.
- bpo-36436 [https://bugs.python.org/issue?@action=redirect&bpo=36436]: Fix `_testcapi.pymem_buffer_overflow()`: handle memory allocation failure.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Fix test_utf8_mode on VxWorks: Python always use UTF-8 on VxWorks.
- bpo-36341 [https://bugs.python.org/issue?@action=redirect&bpo=36341]: Fix tests that may fail with PermissionError upon calling bind() on AF_UNIX sockets.

Construção

- bpo-36747 [https://bugs.python.org/issue?@action=redirect&bpo=36747]: Remove the stale scriptsinstall Makefile target.

- bpo-21536 [https://bugs.python.org/issue?@action=redirect&bpo=21536]: On Unix, C extensions are no longer linked to libpython except on Android and Cygwin.

  It is now possible for a statically linked Python to load a C extension built using a shared library Python.

When Python is embedded, `libpython` must not be loaded with `RTLD_LOCAL`, but `RTLD_GLOBAL` instead. Previously, using `RTLD_LOCAL`, it was already not possible to load C extensions which were not linked to `libpython`, such as C extensions of the standard library built by the `*shared*` section of `Modules/Setup`.

distutils, python-config and python-config.py have been modified.

- bpo-36707 [https://bugs.python.org/issue?@action=redirect&bpo=36707]: `./configure --with-pymalloc` no longer adds the `m` flag to SOABI (sys.implementation.cache_tag). Enabling or disabling pymalloc has no impact on the ABI.

- bpo-36635 [https://bugs.python.org/issue?@action=redirect&bpo=36635]: Change `PyAPI_FUNC(type)`, `PyAPI_DATA(type)` and `PyMODINIT_FUNC` macros of `pyport.h` when `Py_BUILD_CORE_MODULE` is defined. The `Py_BUILD_CORE_MODULE` define must be now be used to build a C extension as a dynamic library accessing Python internals: export the `PyInit_`*`xxx`*`()` function in DLL exports on Windows.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Don't build the `_crypt` extension on VxWorks.

- bpo-36618 [https://bugs.python.org/issue?@action=redirect&bpo=36618]: Add `-fmax-type-align=8` to CFLAGS when clang compiler is detected. The pymalloc memory allocator aligns memory on 8 bytes. On x86-64, clang expects alignment on 16 bytes by default and so uses MOVAPS instruction which can lead to segmentation fault. Instruct clang that Python is limited to alignment on 8 bytes to use MOVUPS instruction instead: slower but don't trigger a SIGSEGV if the memory is not aligned on 16 bytes. Sadly, the flag must be added to `CFLAGS` and not just `CFLAGS_NODIST`, since third party C extensions can have the same issue.

- bpo-36605 [https://bugs.python.org/issue?@action=redirect&bpo=36605]: `make tags` and `make TAGS` now also parse `Modules/_io/*.c` and `Modules/_io/*.h`.

- bpo-36465 [https://bugs.python.org/issue?@action=redirect&bpo=36465]: Release builds and debug builds are now ABI compatible: defining the `Py_DEBUG` macro no longer implies the `Py_TRACE_REFS` macro, which introduces the only ABI incompatibility. The `Py_TRACE_REFS` macro, which adds the **`sys.getobjects()`** function and the **`PYTHONDUMPREFS`** environment variable, can be set using the new `./configure --with-trace-refs` build option.

- bpo-36577 [https://bugs.python.org/issue?@action=redirect&bpo=36577]: setup.py now correctly reports missing OpenSSL headers and libraries again.

- bpo-36544 [https://bugs.python.org/issue?@action=redirect&bpo=36544]: Fix regression introduced in bpo-36146 [https://bugs.python.org/issue?@action=redirect&bpo=36146] refactoring setup.py

- bpo-36508 [https://bugs.python.org/issue?@action=redirect&bpo=36508]: `python-config --ldflags` no longer includes flags of the `LINKFORSHARED` variable. The `LINKFORSHARED` variable must only be used to build executables.

- bpo-36503 [https://bugs.python.org/issue?@action=redirect&bpo=36503]: Remove references to “aix3” and “aix4”. Patch by M. Felt.

Windows

- bpo-35920 [https://bugs.python.org/issue?@action=redirect&bpo=35920]: Added platform.win32_edition() and platform.win32_is_iot(). Added support for cross-compiling packages for Windows ARM32. Skip tests that are not expected to work on Windows IoT Core ARM32.
- bpo-36649 [https://bugs.python.org/issue?@action=redirect&bpo=36649]: Remove trailing spaces for registry keys when installed via the Store.
- bpo-34144 [https://bugs.python.org/issue?@action=redirect&bpo=34144]: Fixed activate.bat to correctly update codepage when chcp.com returns dots in output. Patch by Lorenz Mende.
- bpo-36509 [https://bugs.python.org/issue?@action=redirect&bpo=36509]: Added preset-iot layout for Windows IoT ARM containers. This layout doesn’t contain UI components like tkinter or IDLE. It also doesn’t

contain files to support on-target builds since Windows ARM32 builds must be cross-compiled when using MSVC.

- bpo-35941 [https://bugs.python.org/issue?@action=redirect&bpo=35941]: enum_certificates function of the ssl module now returns certificates from all available certificate stores inside windows in a query instead of returning only certificates from the system wide certificate store. This includes certificates from these certificate stores: local machine, local machine enterprise, local machine group policy, current user, current user group policy, services, users. ssl.enum_crls() function is changed in the same way to return all certificate revocation lists inside the windows certificate revocation list stores.
- bpo-36441 [https://bugs.python.org/issue?@action=redirect&bpo=36441]: Fixes creating a venv when debug binaries are installed.
- bpo-36085 [https://bugs.python.org/issue?@action=redirect&bpo=36085]: Enable better DLL resolution on Windows by using safe DLL search paths and adding `os.add_dll_directory()`.
- bpo-36010 [https://bugs.python.org/issue?@action=redirect&bpo=36010]: Add the venv standard library module to the nuget distribution for Windows.
- bpo-29515 [https://bugs.python.org/issue?@action=redirect&bpo=29515]: Add the following socket module constants on Windows: IPPROTO_AH IPPROTO_CBT IPPROTO_DSTOPTS IPPROTO_EGP IPPROTO_ESP IPPROTO_FRAGMENT IPPROTO_GGP IPPROTO_HOPOPTS IPPROTO_ICLFXBM IPPROTO_ICMPV6 IPPROTO_IDP IPPROTO_IGMP IPPROTO_IGP IPPROTO_IPV4 IPPROTO_IPV6 IPPROTO_L2TP IPPROTO_MAX IPPROTO_ND IPPROTO_NONE IPPROTO_PGM IPPROTO_PIM IPPROTO_PUP IPPROTO_RDP IPPROTO_ROUTING IPPROTO_SCTP IPPROTO_ST
- bpo-35947 [https://bugs.python.org/issue?@action=redirect&bpo=35947]: Added current version of libffi to cpython-source-deps. Change _ctypes to use current version of libffi on Windows.
- bpo-34060 [https://bugs.python.org/issue?@action=redirect&bpo=34060]: Report system load when running test suite on Windows. Patch by Ammar Askar. Based on prior work by Jeremy Kloth.
- bpo-31512 [https://bugs.python.org/issue?@action=redirect&bpo=31512]: With the Windows 10 Creators Update, non-elevated users can now create symlinks as long as the computer has Developer Mode enabled.

macOS

- bpo-34602 [https://bugs.python.org/issue?@action=redirect&bpo=34602]: Avoid failures setting macOS stack resource limit with resource.setrlimit. This reverts an earlier fix for bpo-18075 [https://bugs.python.org/issue?@action=redirect&bpo=18075] which forced a non-default stack size when building the interpreter executable on macOS.

IDLE

- bpo-36429 [https://bugs.python.org/issue?@action=redirect&bpo=36429]: Fix starting IDLE with pyshell. Add idlelib.pyshell alias at top; remove pyshell alias at bottom. Remove obsolete __name__=='__main__' command.

Ferramentas/Demos

- bpo-14546 [https://bugs.python.org/issue?@action=redirect&bpo=14546]: Fix the argument handling in Tools/scripts/lll.py.

API C

- bpo-36763 [https://bugs.python.org/issue?@action=redirect&bpo=36763]: Fix memory leak in **`Py_SetStandardStreamEncoding()`**: release memory if the function is called twice.
- bpo-36641 [https://bugs.python.org/issue?@action=redirect&bpo=36641]: PyDoc_VAR(name) and PyDoc_STRVAR(name, str) now create `static const char name[]` instead of `static char name[]`. Patch by Inada Naoki.
- bpo-36389 [https://bugs.python.org/issue?@action=redirect&bpo=36389]: Change the value of `CLEANBYTE`, `DEADDYTE` and `FORBIDDENBYTE` internal constants used by debug hooks on Python memory allocators (**`PyMem_SetupDebugHooks()`** function). Byte patterns `0xCB`, `0xDB` and `0xFB` have been replaced with `0xCD`, `0xDD` and `0xFD` to use the same values than Windows CRT debug `malloc()` and `free()`.
- bpo-36443 [https://bugs.python.org/issue?@action=redirect&bpo=36443]: Since Python 3.7.0, calling **`Py_DecodeLocale()`** before **`Py_Initialize()`** produces mojibake if the `LC_CTYPE` locale is coerced and/or if the UTF-

8 Mode is enabled by the user configuration. The LC_CTYPE coercion and UTF-8 Mode are now disabled by default to fix the mojibake issue. They must now be enabled explicitly (opt-in) using the new **`_Py_PreInitialize()`** API with `_PyPreConfig`.

- bpo-36025 [https://bugs.python.org/issue?@action=redirect&bpo=36025]: Fixed an accidental change to the datetime C API where the arguments to the **`PyDate_FromTimestamp()`** function were incorrectly interpreted as a single timestamp rather than an arguments tuple, which causes existing code to start raising **`TypeError`**. The backwards-incompatible change was only present in alpha releases of Python 3.8. Patch by Paul Ganssle.
- bpo-35810 [https://bugs.python.org/issue?@action=redirect&bpo=35810]: Modify `PyObject_Init` to correctly increase the refcount of heap-allocated Type objects. Also fix the refcounts of the heap-allocated types that were either doing this manually or not decreasing the type's refcount in tp_dealloc

Python 3.8.0 alpha 3

Data de lançamento: 2019-03-25

Segurança

- bpo-36216 [https://bugs.python.org/issue?@action=redirect&bpo=36216]: Changes urlsplit() to raise ValueError when the URL contains characters that decompose under IDNA encoding (NFKC-normalization) into characters that affect how the URL is parsed.
- bpo-35121 [https://bugs.python.org/issue?@action=redirect&bpo=35121]: Don't send cookies of domain A without Domain attribute to domain B when domain A is a suffix match of domain B while using a cookiejar with **`http.cookiejar.DefaultCookiePolicy`** policy. Patch by Karthikeyan Singaravelan.

Núcleo e embutidos

- bpo-36421 [https://bugs.python.org/issue?@action=redirect&bpo=36421]: Fix a possible double decref in _ctypes.c's `PyCArrayType_new()`.
- bpo-36412 [https://bugs.python.org/issue?@action=redirect&bpo=36412]: Fix a possible crash when creating a new dictionary.
- bpo-36398 [https://bugs.python.org/issue?@action=redirect&bpo=36398]: Fix a possible crash in `structseq_repr()`.
- bpo-36256 [https://bugs.python.org/issue?@action=redirect&bpo=36256]: Fix bug in parsermodule when parsing a state in a DFA that has two or more arcs with labels of the same type. Patch by Pablo Galindo.
- bpo-36365 [https://bugs.python.org/issue?@action=redirect&bpo=36365]: repr(structseq) is no longer limited to 512 bytes.
- bpo-36374 [https://bugs.python.org/issue?@action=redirect&bpo=36374]: Fix a possible null pointer dereference in `merge_consts_recursive()`. Patch by Zackery Spytz.
- bpo-36236 [https://bugs.python.org/issue?@action=redirect&bpo=36236]: At Python initialization, the current directory is no longer prepended to **`sys.path`** if it has been removed.
- bpo-36352 [https://bugs.python.org/issue?@action=redirect&bpo=36352]: Python initialization now fails with an error, rather than silently truncating paths, if a path is too long.
- bpo-36301 [https://bugs.python.org/issue?@action=redirect&bpo=36301]: Python initialization now fails if decoding `pybuilddir.txt` configuration file fails at startup.
- bpo-36333 [https://bugs.python.org/issue?@action=redirect&bpo=36333]: Fix leak in _PyRuntimeState_Fini. Contributed by Stéphane Wirtel.
- bpo-36332 [https://bugs.python.org/issue?@action=redirect&bpo=36332]: The builtin **`compile()`** can now handle AST objects that contain assignment expressions. Patch by Pablo Galindo.
- bpo-36282 [https://bugs.python.org/issue?@action=redirect&bpo=36282]: Improved error message for too much positional arguments in some builtin functions.
- bpo-30040 [https://bugs.python.org/issue?@action=redirect&bpo=30040]: New empty dict uses fewer memory for now. It used more memory than empty dict created by `dict.clear()`. And empty dict creation and deletion is about 2x faster. Patch by Inada Naoki.
- bpo-36262 [https://bugs.python.org/issue?@action=redirect&bpo=36262]: Fix an unlikely memory leak on conversion from string to float in the

function `_Py_dg_strtod()` used by `float(str)`, `complex(str)`, `pickle.load()`, `marshal.load()`, etc.

- bpo-36252 [https://bugs.python.org/issue?@action=redirect&bpo=36252]: Update Unicode databases to version 12.0.0.
- bpo-36218 [https://bugs.python.org/issue?@action=redirect&bpo=36218]: Fix a segfault occurring when sorting a list of heterogeneous values. Patch contributed by Rémi Lapeyre and Elliot Gorokhovsky.
- bpo-36188 [https://bugs.python.org/issue?@action=redirect&bpo=36188]: Cleaned up left-over vestiges of Python 2 unbound method handling in method objects and documentation. Patch by Martijn Pieters
- bpo-36124 [https://bugs.python.org/issue?@action=redirect&bpo=36124]: Add a new interpreter-specific dict and expose it in the C-API via PyInterpreterState_GetDict(). This parallels PyThreadState_GetDict(). However, extension modules should continue using PyModule_GetState() for their own internal per-interpreter state.
- bpo-35975 [https://bugs.python.org/issue?@action=redirect&bpo=35975]: Add a `feature_version` flag to `ast.parse()` (documented) and `compile()` (hidden) that allows tweaking the parser to support older versions of the grammar. In particular, if `feature_version` is 5 or 6, the hacks for the `async` and `await` keyword from PEP 492 are reinstated. (For 7 or higher, these are unconditionally treated as keywords, but they are still special tokens rather than `NAME` tokens that the parser driver recognizes.)
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Use UTF-8 as the system encoding on VxWorks.
- bpo-36048 [https://bugs.python.org/issue?@action=redirect&bpo=36048]: The `__index__()` special method will be used instead of `__int__()` for implicit conversion of Python numbers to C integers. Using the `__int__()` method in implicit conversions has been deprecated.
- bpo-35808 [https://bugs.python.org/issue?@action=redirect&bpo=35808]: Retire pgen and use a modified version of pgen2 to generate the parser. Patch by Pablo Galindo.

Biblioteca

- bpo-36401 [https://bugs.python.org/issue?@action=redirect&bpo=36401]: The class documentation created by pydoc now has a separate section for

readonly properties.

- bpo-36320 [https://bugs.python.org/issue?@action=redirect&bpo=36320]: The typing.NamedTuple() class has deprecated the _field_types attribute in favor of the __annotations__ attribute which carried the same information. Also, both attributes were converted from OrderedDict to a regular dict.
- bpo-34745 [https://bugs.python.org/issue?@action=redirect&bpo=34745]: Fix `asyncio` ssl memory issues caused by circular references
- bpo-36324 [https://bugs.python.org/issue?@action=redirect&bpo=36324]: Add method to statistics.NormalDist for computing the inverse cumulative normal distribution.
- bpo-36321 [https://bugs.python.org/issue?@action=redirect&bpo=36321]: collections.namedtuple() misspelled the name of an attribute. To be consistent with typing.NamedTuple, the attribute name should have been "_field_defaults" instead of "_fields_defaults". For backwards compatibility, both spellings are now created. The misspelled version may be removed in the future.
- bpo-36297 [https://bugs.python.org/issue?@action=redirect&bpo=36297]: "unicode_internal" codec is removed. It was deprecated since Python 3.3. Patch by Inada Naoki.
- bpo-36298 [https://bugs.python.org/issue?@action=redirect&bpo=36298]: Raise ModuleNotFoundError in pyclbr when a module can't be found. Thanks to 'mental' for the bug report.
- bpo-36268 [https://bugs.python.org/issue?@action=redirect&bpo=36268]: Switch the default format used for writing tars with `tarfile` to the modern POSIX.1-2001 pax standard, from the vendor-specific GNU. Contributed by C.A.M. Gerlach.
- bpo-36285 [https://bugs.python.org/issue?@action=redirect&bpo=36285]: Fix integer overflows in the array module. Patch by Stephan Hohe.
- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Add _signal module support for VxWorks.
- bpo-36272 [https://bugs.python.org/issue?@action=redirect&bpo=36272]: `logging` does not silently ignore RecursionError anymore. Patch contributed by Rémi Lapeyre.
- bpo-36280 [https://bugs.python.org/issue?@action=redirect&bpo=36280]: Add a kind field to ast.Constant. It is 'u' if the literal has a 'u' prefix (i.e. a Python 2 style unicode literal), else None.

- bpo-35931 [https://bugs.python.org/issue?@action=redirect&bpo=35931]: The **`pdb`** `debug` command now gracefully handles all exceptions.
- bpo-36251 [https://bugs.python.org/issue?@action=redirect&bpo=36251]: Fix format strings used for stderrprinter and re.Match reprs. Patch by Stephan Hohe.
- bpo-36235 [https://bugs.python.org/issue?@action=redirect&bpo=36235]: Fix `CFLAGS` in `customize_compiler()` of `distutils.sysconfig`: when the `CFLAGS` environment variable is defined, don't override `CFLAGS` variable with the `OPT` variable anymore. Initial patch written by David Malcolm.
- bpo-35807 [https://bugs.python.org/issue?@action=redirect&bpo=35807]: Update ensurepip to install pip 19.0.3 and setuptools 40.8.0.
- bpo-36139 [https://bugs.python.org/issue?@action=redirect&bpo=36139]: Release GIL when closing **`mmap`** objects.
- bpo-36179 [https://bugs.python.org/issue?@action=redirect&bpo=36179]: Fix two unlikely reference leaks in _hashopenssl. The leaks only occur in out-of-memory cases.
- bpo-36169 [https://bugs.python.org/issue?@action=redirect&bpo=36169]: Add overlap() method to statistics.NormalDist. Computes the overlapping coefficient for two normal distributions.
- bpo-36103 [https://bugs.python.org/issue?@action=redirect&bpo=36103]: Default buffer size used by `shutil.copyfileobj()` is changed from 16 KiB to 64 KiB on non-Windows platform to reduce system call overhead. Contributed by Inada Naoki.
- bpo-36130 [https://bugs.python.org/issue?@action=redirect&bpo=36130]: Fix `pdb` with `skip=...` when stepping into a frame without a `__name__` global. Patch by Anthony Sottile.
- bpo-35652 [https://bugs.python.org/issue?@action=redirect&bpo=35652]: shutil.copytree(copy_function=…) erroneously pass DirEntry instead of a path string.
- bpo-35178 [https://bugs.python.org/issue?@action=redirect&bpo=35178]: Ensure custom **`warnings.formatwarning()`** function can receive `line` as positional argument. Based on patch by Tashrif Billah.
- bpo-36106 [https://bugs.python.org/issue?@action=redirect&bpo=36106]: Resolve potential name clash with libm's sinpi(). Patch by Dmitrii Pasechnik.
- bpo-36091 [https://bugs.python.org/issue?@action=redirect&bpo=36091]: Clean up reference to async generator in Lib/types. Patch by Henry Chen.

- bpo-36043 [https://bugs.python.org/issue?@action=redirect&bpo=36043]: **`FileCookieJar`** supports path-like object. Contributed by Stéphane Wirtel
- bpo-35899 [https://bugs.python.org/issue?@action=redirect&bpo=35899]: Enum has been fixed to correctly handle empty strings and strings with non-Latin characters (ie. ‘α’, ‘א’) without crashing. Original patch contributed by Maxwell. Assisted by Stéphane Wirtel.
- bpo-21269 [https://bugs.python.org/issue?@action=redirect&bpo=21269]: Add `args` and `kwargs` properties to mock call objects. Contributed by Kumar Akshay.
- bpo-30670 [https://bugs.python.org/issue?@action=redirect&bpo=30670]: `pprint.pp` has been added to pretty-print objects with dictionary keys being sorted with their insertion order by default. Parameter *sort_dicts* has been added to `pprint.pprint`, `pprint.pformat` and `pprint.PrettyPrinter`. Contributed by Rémi Lapeyre.
- bpo-35843 [https://bugs.python.org/issue?@action=redirect&bpo=35843]: Implement `__getitem__` for `_NamespacePath`. Patch by Anthony Sottile.
- bpo-35802 [https://bugs.python.org/issue?@action=redirect&bpo=35802]: Clean up code which checked presence of `os.stat` / `os.lstat` / `os.chmod` which are always present. Patch by Anthony Sottile.
- bpo-35715 [https://bugs.python.org/issue?@action=redirect&bpo=35715]: Librates the return value of a ProcessPoolExecutor _process_worker after it’s no longer needed to free memory
- bpo-35493 [https://bugs.python.org/issue?@action=redirect&bpo=35493]: Use **`multiprocessing.connection.wait()`** instead of polling each 0.2 seconds for worker updates in **`multiprocessing.Pool`**. Patch by Pablo Galindo.
- bpo-35661 [https://bugs.python.org/issue?@action=redirect&bpo=35661]: Store the venv prompt in pyvenv.cfg.
- bpo-35121 [https://bugs.python.org/issue?@action=redirect&bpo=35121]: Don’t set cookie for a request when the request path is a prefix match of the cookie’s path attribute but doesn’t end with “/”. Patch by Karthikeyan Singaravelan.
- bpo-21478 [https://bugs.python.org/issue?@action=redirect&bpo=21478]: Calls to a child function created with **`unittest.mock.create_autospec()`** should propagate to the parent. Patch by Karthikeyan Singaravelan.

- bpo-35198 [https://bugs.python.org/issue?@action=redirect&bpo=35198]: Fix C++ extension compilation on AIX

Documentação

- bpo-36329 [https://bugs.python.org/issue?@action=redirect&bpo=36329]: Declare the path of the Python binary for the usage of `Tools/scripts/serve.py` when executing `make -C Doc/ serve`. Contributed by Stéphane Wirtel
- bpo-36138 [https://bugs.python.org/issue?@action=redirect&bpo=36138]: Improve documentation about converting datetime.timedelta to scalars.
- bpo-21314 [https://bugs.python.org/issue?@action=redirect&bpo=21314]: A new entry was added to the Core Language Section of the Programming FAQ, which explaines the usage of slash(/) in the signature of a function. Patch by Lysandros Nikolaou

Testes

- bpo-36234 [https://bugs.python.org/issue?@action=redirect&bpo=36234]: test_posix.PosixUidGidTests: add tests for invalid uid/gid type (str). Initial patch written by David Malcolm.
- bpo-29571 [https://bugs.python.org/issue?@action=redirect&bpo=29571]: Fix `test_re.test_locale_flag()`: use `locale.getpreferredencoding()` rather than `locale.getlocale()` to get the locale encoding. With some locales, `locale.getlocale()` returns the wrong encoding.
- bpo-36123 [https://bugs.python.org/issue?@action=redirect&bpo=36123]: Fix race condition in test_socket.

Construção

- bpo-36356 [https://bugs.python.org/issue?@action=redirect&bpo=36356]: Fix leaks that led to build failure when configured with address sanitizer.
- bpo-36146 [https://bugs.python.org/issue?@action=redirect&bpo=36146]: Add `TEST_EXTENSIONS` constant to `setup.py` to allow to not build test extensions like `_testcapi`.
- bpo-36146 [https://bugs.python.org/issue?@action=redirect&bpo=36146]: Fix setup.py on macOS: only add `/usr/include/ffi` to include directories of _ctypes, not for all extensions.

- bpo-31904 [https://bugs.python.org/issue?@action=redirect&bpo=31904]: Enable build system to cross-build for VxWorks RTOS.

Windows

- bpo-36312 [https://bugs.python.org/issue?@action=redirect&bpo=36312]: Fixed decoders for the following code pages: 50220, 50221, 50222, 50225, 50227, 50229, 57002 through 57011, 65000 and 42.
- bpo-36264 [https://bugs.python.org/issue?@action=redirect&bpo=36264]: Don't honor POSIX `HOME` in `os.path.expanduser` on windows. Patch by Anthony Sottile.
- bpo-24643 [https://bugs.python.org/issue?@action=redirect&bpo=24643]: Fix name collisions due to `#define timezone _timezone` in PC/pyconfig.h.

IDLE

- bpo-36405 [https://bugs.python.org/issue?@action=redirect&bpo=36405]: Use dict unpacking in idlelib.
- bpo-36396 [https://bugs.python.org/issue?@action=redirect&bpo=36396]: Remove fgBg param of idlelib.config.GetHighlight(). This param was only used twice and changed the return type.
- bpo-36176 [https://bugs.python.org/issue?@action=redirect&bpo=36176]: Fix IDLE autocomplete & calltip popup colors. Prevent conflicts with Linux dark themes (and slightly darken calltip background).
- bpo-23205 [https://bugs.python.org/issue?@action=redirect&bpo=23205]: For the grep module, add tests for findfiles, refactor findfiles to be a module-level function, and refactor findfiles to use os.walk.
- bpo-23216 [https://bugs.python.org/issue?@action=redirect&bpo=23216]: Add docstrings to IDLE search modules.
- bpo-36152 [https://bugs.python.org/issue?@action=redirect&bpo=36152]: Remove colorizer.ColorDelegator.close_when_done and the corresponding argument of .close(). In IDLE, both have always been None or False since 2007.
- bpo-32129 [https://bugs.python.org/issue?@action=redirect&bpo=32129]: Avoid blurry IDLE application icon on macOS with Tk 8.6. Patch by Kevin Walzer.

- bpo-36096 [https://bugs.python.org/issue?@action=redirect&bpo=36096]: Refactor class variables to instance variables in colorizer.
- bpo-30348 [https://bugs.python.org/issue?@action=redirect&bpo=30348]: Increase test coverage of idlelib.autocomplete by 30%. Patch by Louie Lu

Ferramentas/Demos

- bpo-35132 [https://bugs.python.org/issue?@action=redirect&bpo=35132]: Fix py-list and py-bt commands of python-gdb.py on gdb7.
- bpo-32217 [https://bugs.python.org/issue?@action=redirect&bpo=32217]: Fix freeze script on Windows.

API C

- bpo-36381 [https://bugs.python.org/issue?@action=redirect&bpo=36381]: Raise `DeprecationWarning` when '#' formats are used for building or parsing values without `PY_SSIZE_T_CLEAN`.
- bpo-36142 [https://bugs.python.org/issue?@action=redirect&bpo=36142]: The whole coreconfig.h header is now excluded from Py_LIMITED_API. Move functions definitions into a new internal pycore_coreconfig.h header.

Python 3.8.0 alpha 2

Data de lançamento: 2019-02-25

Núcleo e embutidos

- bpo-36052 [https://bugs.python.org/issue?@action=redirect&bpo=36052]: Raise a **`SyntaxError`** when assigning a value to `__debug__` with the Assignment Operator. Contributed by Stéphane Wirtel and Pablo Galindo.
- bpo-36012 [https://bugs.python.org/issue?@action=redirect&bpo=36012]: Doubled the speed of class variable writes. When a non-dunder attribute was updated, there was an unnecessary call to update slots.
- bpo-35942 [https://bugs.python.org/issue?@action=redirect&bpo=35942]: The error message emitted when returning invalid types from `__fspath__` in

interfaces that allow passing **`PathLike`** objects has been improved and now it does explain the origin of the error.

- bpo-36016 [https://bugs.python.org/issue?@action=redirect&bpo=36016]: `gc.get_objects` can now receive an optional parameter indicating a generation to get objects from. Patch by Pablo Galindo.
- bpo-1054041 [https://bugs.python.org/issue?@action=redirect&bpo=1054041]: When the main interpreter exits due to an uncaught KeyboardInterrupt, the process now exits in the appropriate manner for its parent process to detect that a SIGINT or ^C terminated the process. This allows shells and batch scripts to understand that the user has asked them to stop.
- bpo-35992 [https://bugs.python.org/issue?@action=redirect&bpo=35992]: Fix `__class_getitem__()` not being called on a class with a custom non-subscriptable metaclass.
- bpo-35993 [https://bugs.python.org/issue?@action=redirect&bpo=35993]: Fix a crash on fork when using subinterpreters. Contributed by Stéphane Wirtel
- bpo-35991 [https://bugs.python.org/issue?@action=redirect&bpo=35991]: Fix a potential double free in Modules/_randommodule.c.
- bpo-35961 [https://bugs.python.org/issue?@action=redirect&bpo=35961]: Fix a crash in slice_richcompare(): use strong references rather than stolen references for the two temporary internal tuples.
- bpo-35911 [https://bugs.python.org/issue?@action=redirect&bpo=35911]: Enable the creation of cell objects by adding a `cell.__new__` method, and expose the type `cell` in `Lib/types.py` under the name CellType. Patch by Pierre Glaser.
- bpo-12822 [https://bugs.python.org/issue?@action=redirect&bpo=12822]: Use monotonic clock for `pthread_cond_timedwait` when `pthread_condattr_setclock` and `CLOCK_MONOTONIC` are available.
- bpo-15248 [https://bugs.python.org/issue?@action=redirect&bpo=15248]: The compiler emits now syntax warnings in the case when a comma is likely missed before tuple or list.
- bpo-35886 [https://bugs.python.org/issue?@action=redirect&bpo=35886]: The implementation of PyInterpreterState has been moved into the internal header files (guarded by Py_BUILD_CORE).
- bpo-31506 [https://bugs.python.org/issue?@action=redirect&bpo=31506]: Clarify the errors reported when `object.__new__` and `object.__init__` receive more than one argument. Contributed by Sanyam Khurana.

- bpo-35724 [https://bugs.python.org/issue?@action=redirect&bpo=35724]: Signal-handling is now guaranteed to happen relative to the main interpreter.
- bpo-33608 [https://bugs.python.org/issue?@action=redirect&bpo=33608]: We added a new internal _Py_AddPendingCall() that operates relative to the provided interpreter. This allows us to use the existing implementation to ask another interpreter to do work that cannot be done in the current interpreter, like decref an object the other interpreter owns. The existing Py_AddPendingCall() only operates relative to the main interpreter.
- bpo-33989 [https://bugs.python.org/issue?@action=redirect&bpo=33989]: Fix a possible crash in `list.sort()` when sorting objects with `ob_type->tp_richcompare == NULL`. Patch by Zackery Spytz.

Biblioteca

- bpo-35512 [https://bugs.python.org/issue?@action=redirect&bpo=35512]: `unittest.mock.patch.dict()` used as a decorator with string target resolves the target during function call instead of during decorator construction. Patch by Karthikeyan Singaravelan.
- bpo-36018 [https://bugs.python.org/issue?@action=redirect&bpo=36018]: Add statistics.NormalDist, a tool for creating and manipulating normal distributions of random variable. Features a composite class that treats the mean and standard deviation of measurement data as single entity.
- bpo-35904 [https://bugs.python.org/issue?@action=redirect&bpo=35904]: Added statistics.fmean() as a faster, floating-point variant of the existing mean() function.
- bpo-35918 [https://bugs.python.org/issue?@action=redirect&bpo=35918]: Removed broken `has_key` method from multiprocessing.managers.SyncManager.dict. Contributed by Rémi Lapeyre.
- bpo-18283 [https://bugs.python.org/issue?@action=redirect&bpo=18283]: Add support for bytes to `shutil.which()`.
- bpo-35960 [https://bugs.python.org/issue?@action=redirect&bpo=35960]: Fix `dataclasses.field()` throwing away empty mapping objects passed as metadata.
- bpo-35500 [https://bugs.python.org/issue?@action=redirect&bpo=35500]: Write expected and actual call parameters on separate lines in

`unittest.mock.Mock.assert_called_with()` assertion errors. Contributed by Susan Su.

- bpo-35931 [https://bugs.python.org/issue?@action=redirect&bpo=35931]: The `pdb` `debug` command now gracefully handles syntax errors.
- bpo-24209 [https://bugs.python.org/issue?@action=redirect&bpo=24209]: In http.server script, rely on getaddrinfo to bind to preferred address based on the bind parameter. Now default bind or binding to a name may bind to IPv6 or dual-stack, depending on the environment.
- bpo-35321 [https://bugs.python.org/issue?@action=redirect&bpo=35321]: Set `__spec__.origin` of `_frozen_importlib` to frozen so that it matches the behavior of `_frozen_importlib_external`. Patch by Nina Zakharenko.
- bpo-35378 [https://bugs.python.org/issue?@action=redirect&bpo=35378]: Fix a reference issue inside `multiprocessing.Pool` that caused the pool to remain alive if it was deleted without being closed or terminated explicitly. A new strong reference is added to the pool iterators to link the lifetime of the pool to the lifetime of its iterators so the pool does not get destroyed if a pool iterator is still alive.
- bpo-34294 [https://bugs.python.org/issue?@action=redirect&bpo=34294]: re module, fix wrong capturing groups in rare cases. `re.search()`, `re.findall()`, `re.sub()` and other functions that scan through string looking for a match, should reset capturing groups between two match attempts. Patch by Ma Lin.
- bpo-35615 [https://bugs.python.org/issue?@action=redirect&bpo=35615]: `weakref`: Fix a RuntimeError when copying a WeakKeyDictionary or a WeakValueDictionary, due to some keys or values disappearing while iterating.
- bpo-35606 [https://bugs.python.org/issue?@action=redirect&bpo=35606]: Implement `math.prod()` as analogous function to `sum()` that returns the product of a ‘start’ value (default: 1) times an iterable of numbers. Patch by Pablo Galindo.
- bpo-32417 [https://bugs.python.org/issue?@action=redirect&bpo=32417]: Performing arithmetic between `datetime.datetime` subclasses and `datetime.timedelta` now returns an object of the same type as the `datetime.datetime` subclass. As a result, `datetime.datetime.astimezone()` and alternate constructors like

`datetime.datetime.now()` and `datetime.fromtimestamp()` called with a `tz` argument now *also* retain their subclass.
- bpo-35153 [https://bugs.python.org/issue?@action=redirect&bpo=35153]: Add *headers* optional keyword-only parameter to `xmlrpc.client.ServerProxy`, `xmlrpc.client.Transport` and `xmlrpc.client.SafeTransport`. Patch by Cédric Krier.
- bpo-34572 [https://bugs.python.org/issue?@action=redirect&bpo=34572]: Fix C implementation of pickle.loads to use importlib's locking mechanisms, and thereby avoid using partially loaded modules. Patch by Tim Burgess.

Documentação

- bpo-36083 [https://bugs.python.org/issue?@action=redirect&bpo=36083]: Fix formatting of –check-hash-based-pycs options in the manpage Synopsis.
- bpo-36007 [https://bugs.python.org/issue?@action=redirect&bpo=36007]: Bump minimum sphinx version to 1.8. Patch by Anthony Sottile.
- bpo-22062 [https://bugs.python.org/issue?@action=redirect&bpo=22062]: Update documentation and docstrings for pathlib. Original patch by Mike Short.

Testes

- bpo-27313 [https://bugs.python.org/issue?@action=redirect&bpo=27313]: Avoid test_ttk_guionly ComboboxTest failure with macOS Cocoa Tk.

- bpo-36019 [https://bugs.python.org/issue?@action=redirect&bpo=36019]: Add test.support.TEST_HTTP_URL and replace references of http://www.example.com by this new constant. Contributed by Stéphane Wirtel.

- bpo-36037 [https://bugs.python.org/issue?@action=redirect&bpo=36037]: Fix test_ssl for strict OpenSSL configuration like RHEL8 strict crypto policy. Use older TLS version for minimum TLS version of the server SSL context if needed, to test TLS version older than default minimum TLS version.

- bpo-35798 [https://bugs.python.org/issue?@action=redirect&bpo=35798]: Added **`test.support.check_syntax_warning()`**.

- bpo-35505 [https://bugs.python.org/issue?@action=redirect&bpo=35505]: Make test_imap4_host_default_value independent on whether the local IMAP server is running.

- bpo-35917 [https://bugs.python.org/issue?@action=redirect&bpo=35917]: multiprocessing: provide unit tests for SyncManager and SharedMemoryManager classes + all the shareable types which are supposed to be supported by them. (patch by Giampaolo Rodola)

- bpo-35704 [https://bugs.python.org/issue?@action=redirect&bpo=35704]: Skip `test_shutil.test_unpack_archive_xztar` to prevent a MemoryError on 32-bit AIX when MAXDATA setting is less than 0x20000000.

  Patch by Michael Felt (aixtools)

- bpo-34720 [https://bugs.python.org/issue?@action=redirect&bpo=34720]: Assert m_state != NULL to mimic GC traversal functions that do not correctly handle module creation when the module state has not been created.

Windows

- bpo-35976 [https://bugs.python.org/issue?@action=redirect&bpo=35976]: Added ARM build support to Windows build files in PCBuild.
- bpo-35692 [https://bugs.python.org/issue?@action=redirect&bpo=35692]: `pathlib` no longer raises when checking file and directory existence on drives that are not ready
- bpo-35872 [https://bugs.python.org/issue?@action=redirect&bpo=35872]: Uses the base Python executable when invoking venv in a virtual environment
- bpo-35873 [https://bugs.python.org/issue?@action=redirect&bpo=35873]: Prevents venv paths being inherited by child processes
- bpo-35299 [https://bugs.python.org/issue?@action=redirect&bpo=35299]: Fix sysconfig detection of the source directory and distutils handling of pyconfig.h during PGO profiling

IDLE

- bpo-24310 [https://bugs.python.org/issue?@action=redirect&bpo=24310]: IDLE – Document settings dialog font tab sample.
- bpo-35833 [https://bugs.python.org/issue?@action=redirect&bpo=35833]: Revise IDLE doc for control codes sent to Shell. Add a code example block.
- bpo-35689 [https://bugs.python.org/issue?@action=redirect&bpo=35689]: Add docstrings and unittests for colorizer.py.

Python 3.8.0 alpha 1

Data de lançamento: 2019-02-03

Segurança

- bpo-35746 [https://bugs.python.org/issue?@action=redirect&bpo=35746]: **CVE 2019-5010** [https://www.cve.org/CVERecord?id=CVE-2019-5010]: Fix a NULL pointer deref in ssl module. The cert parser did not handle CRL distribution points with empty DP or URI correctly. A malicious or buggy certificate can result into segfault. Vulnerability (TALOS-2018-0758) reported by Colin Read and Nicolas Edet of Cisco.
- bpo-34812 [https://bugs.python.org/issue?@action=redirect&bpo=34812]: The `-I` command line option (run Python in isolated mode) is now also copied by the `multiprocessing` and `distutils` modules when spawning child processes. Previously, only `-E` and `-s` options (enabled by `-I`) were copied.
- bpo-34791 [https://bugs.python.org/issue?@action=redirect&bpo=34791]: The xml.sax and xml.dom.domreg no longer use environment variables to override parser implementations when sys.flags.ignore_environment is set by -E or -I arguments.
- bpo-17239 [https://bugs.python.org/issue?@action=redirect&bpo=17239]: The xml.sax and xml.dom.minidom parsers no longer processes external entities by default. External DTD and ENTITY declarations no longer load files or create network connections.
- bpo-34623 [https://bugs.python.org/issue?@action=redirect&bpo=34623]: **CVE 2018-14647** [https://www.cve.org/CVERecord?id=CVE-2018-14647]: The C accelerated

_elementtree module now initializes hash randomization salt from _Py_HashSecret instead of libexpat's default CSPRNG.

- bpo-34405 [https://bugs.python.org/issue?@action=redirect&bpo=34405]: Updated to OpenSSL 1.1.0i for Windows builds.
- bpo-33871 [https://bugs.python.org/issue?@action=redirect&bpo=33871]: Fixed sending the part of the file in `os.sendfile()` on macOS. Using the *trailers* argument could cause sending more bytes from the input file than was specified.
- bpo-32533 [https://bugs.python.org/issue?@action=redirect&bpo=32533]: Fixed thread-safety of error handling in _ssl.
- bpo-33136 [https://bugs.python.org/issue?@action=redirect&bpo=33136]: Harden ssl module against LibreSSL **CVE 2018-8970** [https://www.cve.org/CVERecord?id=CVE-2018-8970]. X509_VERIFY_PARAM_set1_host() is called with an explicit namelen. A new test ensures that NULL bytes are not allowed.
- bpo-33001 [https://bugs.python.org/issue?@action=redirect&bpo=33001]: Minimal fix to prevent buffer overrun in os.symlink on Windows
- bpo-32981 [https://bugs.python.org/issue?@action=redirect&bpo=32981]: Regexes in difflib and poplib were vulnerable to catastrophic backtracking. These regexes formed potential DOS vectors (REDOS). They have been refactored. This resolves **CVE 2018-1060** [https://www.cve.org/CVERecord?id=CVE-2018-1060] and **CVE 2018-1061** [https://www.cve.org/CVERecord?id=CVE-2018-1061]. Patch by Jamie Davis.
- bpo-28414 [https://bugs.python.org/issue?@action=redirect&bpo=28414]: The ssl module now allows users to perform their own IDN en/decoding when using SNI.

Núcleo e embutidos

- bpo-35877 [https://bugs.python.org/issue?@action=redirect&bpo=35877]: Make parenthesis optional for named expressions in while statement. Patch by Karthikeyan Singaravelan.

- bpo-35814 [https://bugs.python.org/issue?@action=redirect&bpo=35814]: Allow same right hand side expressions in annotated assignments as in normal ones. In particular, `x: Tuple[int, int] = 1, 2` (without parentheses on the right) is now allowed.

- bpo-35766 [https://bugs.python.org/issue?@action=redirect&bpo=35766]: Add the option to parse PEP 484 type comments in the ast module. (Off by default.) This is merging the key functionality of the third party fork thereof, [typed_ast](https://github.com/python/typed_ast).

- bpo-35713 [https://bugs.python.org/issue?@action=redirect&bpo=35713]: Reorganize Python initialization to get working exceptions and sys.stderr earlier.

- bpo-33416 [https://bugs.python.org/issue?@action=redirect&bpo=33416]: Add end line and end column position information to the Python AST nodes. This is a C-level backwards incompatible change.

- bpo-35720 [https://bugs.python.org/issue?@action=redirect&bpo=35720]: Fixed a minor memory leak in pymain_parse_cmdline_impl function in Modules/main.c

- bpo-35634 [https://bugs.python.org/issue?@action=redirect&bpo=35634]: `func(**kwargs)` will now raise an error when `kwargs` is a mapping containing multiple entries with the same key. An error was already raised when other keyword arguments are passed before `**kwargs` since Python 3.6.

- bpo-35623 [https://bugs.python.org/issue?@action=redirect&bpo=35623]: Fix a crash when sorting very long lists. Patch by Stephan Hohe.

- bpo-35214 [https://bugs.python.org/issue?@action=redirect&bpo=35214]: clang Memory Sanitizer build instrumentation was added to work around false positives from posix, socket, time, test_io, and test_faulthandler.

- bpo-35560 [https://bugs.python.org/issue?@action=redirect&bpo=35560]: Fix an assertion error in **`format()`** in debug build for floating-point formatting with “n” format, zero padding and small width. Release build is not impacted. Patch by Karthikeyan Singaravelan.

- bpo-35552 [https://bugs.python.org/issue?@action=redirect&bpo=35552]: Format characters `%s` and `%V` in **`PyUnicode_FromFormat()`** and `%s` in **`PyBytes_FromFormat()`** no longer read memory past the limit if *precision* is specified.

- bpo-35504 [https://bugs.python.org/issue?@action=redirect&bpo=35504]: Fix segfaults and `SystemError`s when deleting certain attributes. Patch by Zackery Spytz.

- bpo-35504 [https://bugs.python.org/issue?@action=redirect&bpo=35504]: Fixed a SystemError when delete the characters_written attribute of an OSError.

- bpo-35494 [https://bugs.python.org/issue?@action=redirect&bpo=35494]: Improved syntax error messages for unbalanced parentheses in f-string.

- bpo-35444 [https://bugs.python.org/issue?@action=redirect&bpo=35444]: Fixed error handling in pickling methods when fail to look up builtin “getattr”. Sped up pickling iterators.

- bpo-35436 [https://bugs.python.org/issue?@action=redirect&bpo=35436]: Fix various issues with memory allocation error handling. Patch by Zackery Spytz.

- bpo-35423 [https://bugs.python.org/issue?@action=redirect&bpo=35423]: Separate the signal handling trigger in the eval loop from the “pending calls” machinery. There is no semantic change and the difference in performance is insignificant.

- bpo-35357 [https://bugs.python.org/issue?@action=redirect&bpo=35357]: Internal attributes’ names of unittest.mock._Call and unittest.mock.MagicProxy (name, parent & from_kall) are now prefixed with _mock_ in order to prevent clashes with widely used object attributes. Fixed minor typo in test function name.

- bpo-35372 [https://bugs.python.org/issue?@action=redirect&bpo=35372]: Fixed the code page decoder for input longer than 2 GiB containing undecodable bytes.

- bpo-35336 [https://bugs.python.org/issue?@action=redirect&bpo=35336]: Fix PYTHONCOERCECLOCALE=1 environment variable: only coerce the C locale if the LC_CTYPE locale is “C”.

- bpo-31241 [https://bugs.python.org/issue?@action=redirect&bpo=31241]: The *lineno* and *col_offset* attributes of AST nodes for list comprehensions, generator expressions and tuples are now point to the opening parenthesis or square brace. For tuples without parenthesis they point to the position of the first item.

- bpo-33954 [https://bugs.python.org/issue?@action=redirect&bpo=33954]: For **`str.format()`**, **`float.__format__()`** and **`complex.__format__()`** methods for non-ASCII decimal point when using the "n" formatter.

- bpo-35269 [https://bugs.python.org/issue?@action=redirect&bpo=35269]: Fix a possible segfault involving a newly created coroutine. Patch by Zackery Spytz.

- bpo-35224 [https://bugs.python.org/issue?@action=redirect&bpo=35224]: Implement **PEP 572** [https://peps.python.org/pep-0572/] (assignment expressions). Patch by Emily Morehouse.

- bpo-32492 [https://bugs.python.org/issue?@action=redirect&bpo=32492]: Speed up **`namedtuple()`** attribute access by 1.6x using a C fast-path for the name descriptors. Patch by Pablo Galindo.

- bpo-35214 [https://bugs.python.org/issue?@action=redirect&bpo=35214]: Fixed an out of bounds memory access when parsing a truncated unicode escape sequence at the end of a string such as `'\N'`. It would read one byte beyond the end of the memory allocation.

- bpo-35214 [https://bugs.python.org/issue?@action=redirect&bpo=35214]: The interpreter and extension modules have had annotations added so that they work properly under clang's Memory Sanitizer. A new configure flag –with-memory-sanitizer has been added to make test builds of this nature easier to perform.

- bpo-35193 [https://bugs.python.org/issue?@action=redirect&bpo=35193]: Fix an off by one error in the bytecode peephole optimizer where it could read bytes beyond the end of bounds of an array when removing unreachable code. This bug was present in every release of Python 3.6 and 3.7 until now.

- bpo-35169 [https://bugs.python.org/issue?@action=redirect&bpo=35169]: Improved error messages for forbidden assignments.

- bpo-34022 [https://bugs.python.org/issue?@action=redirect&bpo=34022]: Fix handling of hash-based bytecode files in `zipimport`. Patch by Elvis Pranskevichus.

- bpo-28401 [https://bugs.python.org/issue?@action=redirect&bpo=28401]: Debug builds will no longer to attempt to import extension modules built for the ABI as they were never compatible to begin with. Patch by Stefano Rivera.

- bpo-29341 [https://bugs.python.org/issue?@action=redirect&bpo=29341]: Clarify in the docstrings of `os` methods that path-like objects are also accepted as input parameters.

- bpo-35050 [https://bugs.python.org/issue?@action=redirect&bpo=35050]: `socket`: Fix off-by-one bug in length check for `AF_ALG` name and type.

- bpo-29743 [https://bugs.python.org/issue?@action=redirect&bpo=29743]: Raise `ValueError` instead of `OverflowError` in case of a negative `_length_` in a `ctypes.Array` subclass. Also raise `TypeError` instead of `AttributeError` for non-integer `_length_`. Original patch by Oren Milman.

- bpo-16806 [https://bugs.python.org/issue?@action=redirect&bpo=16806]: Fix `lineno` and `col_offset` for multi-line string tokens.

- bpo-35029 [https://bugs.python.org/issue?@action=redirect&bpo=35029]: `SyntaxWarning` raised as an exception at code generation time will be now replaced with a `SyntaxError` for better error reporting.

- bpo-34983 [https://bugs.python.org/issue?@action=redirect&bpo=34983]: Expose `symtable.Symbol.is_nonlocal()` in the symtable module. Patch by Pablo Galindo.

- bpo-34974 [https://bugs.python.org/issue?@action=redirect&bpo=34974]: `bytes` and `bytearray` constructors no longer convert unexpected exceptions (e.g.

`MemoryError` and `KeyboardInterrupt`) to `TypeError`.

- bpo-34939 [https://bugs.python.org/issue?@action=redirect&bpo=34939]: Allow annotated names in module namespace that are declared global before the annotation happens. Patch by Pablo Galindo.

- bpo-34973 [https://bugs.python.org/issue?@action=redirect&bpo=34973]: Fixed crash in `bytes()` when the `list` argument is mutated while it is iterated.

- bpo-34876 [https://bugs.python.org/issue?@action=redirect&bpo=34876]: The *lineno* and *col_offset* attributes of the AST for decorated function and class refer now to the position of the corresponding `def`, `async def` and `class` instead of the position of the first decorator. This leads to more correct line reporting in tracing. This is the only case when the position of child AST nodes can precede the position of the parent AST node.

- bpo-34879 [https://bugs.python.org/issue?@action=redirect&bpo=34879]: Fix a possible null pointer dereference in bytesobject.c. Patch by Zackery Spytz.

- bpo-34784 [https://bugs.python.org/issue?@action=redirect&bpo=34784]: Fix the implementation of PyStructSequence_NewType in order to create heap allocated StructSequences.

- bpo-32912 [https://bugs.python.org/issue?@action=redirect&bpo=32912]: A `SyntaxWarning` is now emitted instead of a `DeprecationWarning` for invalid escape sequences in string and bytes literals.

- bpo-34854 [https://bugs.python.org/issue?@action=redirect&bpo=34854]: Fixed a crash in compiling string annotations containing a lambda with a keyword-only argument that doesn't have a default value.

- bpo-34850 [https://bugs.python.org/issue?@action=redirect&bpo=34850]: The compiler now produces a `SyntaxWarning` when identity checks (`is` and `is not`) are used with certain types of literals (e.g. strings, ints). These can often work by accident in CPython, but are not guaranteed by the language spec. The warning advises users to use equality tests (`==` and `!=`) instead.

- bpo-34824 [https://bugs.python.org/issue?@action=redirect&bpo=34824]: Fix a possible null pointer dereference in Modules/_ssl.c. Patch by Zackery Spytz.

- bpo-30156 [https://bugs.python.org/issue?@action=redirect&bpo=30156]: The C function `property_descr_get()` uses a “cached” tuple to optimize function calls. But this tuple can be discovered in debug mode with **`sys.getobjects()`**. Remove the optimization, it’s not really worth it and it causes 3 different crashes last years.

- bpo-34762 [https://bugs.python.org/issue?@action=redirect&bpo=34762]: Fix contextvars C API to use PyObject* pointer types.

- bpo-34751 [https://bugs.python.org/issue?@action=redirect&bpo=34751]: The hash function for tuples is now based on xxHash which gives better collision results on (formerly) pathological cases. Additionally, on 64-bit systems it improves tuple hashes in general. Patch by Jeroen Demeyer with substantial contributions by Tim Peters.

- bpo-34735 [https://bugs.python.org/issue?@action=redirect&bpo=34735]: Fix a memory leak in Modules/timemodule.c. Patch by Zackery Spytz.

- bpo-34683 [https://bugs.python.org/issue?@action=redirect&bpo=34683]: Fixed a bug where some SyntaxError error pointed to locations that were off-by-one.

- bpo-34651 [https://bugs.python.org/issue?@action=redirect&bpo=34651]: Only allow the main interpreter to fork. The avoids the possibility of affecting the main interpreter, which is critical to operation of the runtime.

- bpo-34653 [https://bugs.python.org/issue?@action=redirect&bpo=34653]: Remove unused function PyParser_SimpleParseStringFilename.

- bpo-32236 [https://bugs.python.org/issue?@action=redirect&bpo=32236]: Warn that line buffering is not supported if **`open()`** is called with binary mode and `buffering=1`.

- bpo-34641 [https://bugs.python.org/issue?@action=redirect&bpo=34641]: Further restrict the syntax of the left-hand side of keyword arguments in function calls. In particular, `f((keyword)=arg)` is now disallowed.

- bpo-34637 [https://bugs.python.org/issue?@action=redirect&bpo=34637]: Make the *start* argument to *sum()* visible as a keyword argument.

- bpo-1621 [https://bugs.python.org/issue?@action=redirect&bpo=1621]: Do not assume signed integer overflow behavior (C undefined behavior) when performing set hash table resizing.

- bpo-34588 [https://bugs.python.org/issue?@action=redirect&bpo=34588]: Fix an off-by-one in the recursive call pruning feature of traceback formatting.

- bpo-34485 [https://bugs.python.org/issue?@action=redirect&bpo=34485]: On Windows, the LC_CTYPE is now set to the user preferred locale at startup. Previously, the LC_CTYPE locale was “C” at startup, but changed when calling setlocale(LC_CTYPE, “”) or setlocale(LC_ALL, “”).

- bpo-34485 [https://bugs.python.org/issue?@action=redirect&bpo=34485]: Standard streams like sys.stdout now use the “surrogateescape” error handler, instead of “strict”, on the POSIX locale (when the C locale is not coerced and the UTF-8 Mode is disabled).

- bpo-34485 [https://bugs.python.org/issue?@action=redirect&bpo=34485]: Fix the error handler of standard streams like sys.stdout: PYTHONIOENCODING=”:” is now ignored instead of setting the error handler to “strict”.

- bpo-34485 [https://bugs.python.org/issue?@action=redirect&bpo=34485]: Python now gets the locale encoding with C code to initialize the encoding of standard streams like sys.stdout. Moreover, the encoding is now initialized to the Python codec name to get a normalized encoding name and to ensure that the codec is loaded. The change avoids importing _bootlocale and _locale modules at startup by default.

- bpo-34527 [https://bugs.python.org/issue?@action=redirect&bpo=34527]: On FreeBSD, Py_DecodeLocale() and Py_EncodeLocale() now also forces the ASCII encoding if the LC_CTYPE locale is “POSIX”, not only if the LC_CTYPE locale is “C”.

- bpo-34527 [https://bugs.python.org/issue?@action=redirect&bpo=34527]: The UTF-8 Mode is now also enabled by the “POSIX” locale, not only by the “C” locale.

- bpo-34403 [https://bugs.python.org/issue?@action=redirect&bpo=34403]: On HP-UX with C or POSIX locale, sys.getfilesystemencoding() now returns “ascii” instead of “roman8” (when the UTF-8 Mode is disabled and the C locale is not coerced).

- bpo-34523 [https://bugs.python.org/issue?@action=redirect&bpo=34523]: The Python filesystem encoding is now read earlier during the Python initialization.

- bpo-12458 [https://bugs.python.org/issue?@action=redirect&bpo=12458]: Tracebacks show now correct line number for subexpressions in multiline expressions. Tracebacks show now the line number of the first line for multiline expressions instead of the line number of the last subexpression.

- bpo-34408 [https://bugs.python.org/issue?@action=redirect&bpo=34408]: Prevent a null pointer dereference and resource leakage in `PyInterpreterState_New()`.

- bpo-34400 [https://bugs.python.org/issue?@action=redirect&bpo=34400]: Fix undefined behavior in parsetok.c. Patch by Zackery Spytz.

- bpo-33073 [https://bugs.python.org/issue?@action=redirect&bpo=33073]: Added as_integer_ratio to ints to make them more interoperable with floats.

- bpo-34377 [https://bugs.python.org/issue?@action=redirect&bpo=34377]: Update valgrind suppression list to use `_PyObject_Free`/`_PyObject_Realloc` instead of `PyObject_Free`/`PyObject_Realloc`.

- bpo-34353 [https://bugs.python.org/issue?@action=redirect&bpo=34353]: Added the "socket" option in the `stat.filemode()` Python implementation to match the C implementation.

- bpo-34320 [https://bugs.python.org/issue?@action=redirect&bpo=34320]: Fix `dict(od)` didn't copy iteration order of OrderedDict.

- bpo-34113 [https://bugs.python.org/issue?@action=redirect&bpo=34113]: Fixed crash on debug builds when opcode stack was adjusted with negative numbers. Patch by Constantin Petrisor.

- bpo-34100 [https://bugs.python.org/issue?@action=redirect&bpo=34100]: Compiler now merges constants in tuples and frozensets recursively. Code attributes like `co_names` are merged too.

- bpo-34151 [https://bugs.python.org/issue?@action=redirect&bpo=34151]: Performance of list concatenation, repetition and slicing operations is slightly improved. Patch by Sergey Fedoseev.

- bpo-34170 [https://bugs.python.org/issue?@action=redirect&bpo=34170]: -X dev: it is now possible to override the memory allocator using PYTHONMALLOC even if the developer mode is enabled.

- bpo-33237 [https://bugs.python.org/issue?@action=redirect&bpo=33237]: Improved **`AttributeError`** message for partially initialized module.

- bpo-34149 [https://bugs.python.org/issue?@action=redirect&bpo=34149]: Fix min and max functions to get default behavior when key is None.

- bpo-34125 [https://bugs.python.org/issue?@action=redirect&bpo=34125]: Profiling of unbound built-in methods now works when `**kwargs` is given.

- bpo-34141 [https://bugs.python.org/issue?@action=redirect&bpo=34141]: Optimized pickling atomic types (None, bool, int, float, bytes, str).

- bpo-34126 [https://bugs.python.org/issue?@action=redirect&bpo=34126]: Fix crashes when profiling certain invalid calls of unbound methods. Patch by Jeroen Demeyer.

- bpo-24618 [https://bugs.python.org/issue?@action=redirect&bpo=24618]: Fixed reading invalid memory when create the code object with too small varnames tuple or too large argument counts.

- bpo-34068 [https://bugs.python.org/issue?@action=redirect&bpo=34068]: In `io.IOBase.close()`, ensure that the `closed` attribute is not set with a live exception. Patch by Zackery Spytz and Serhiy Storchaka.

- bpo-34087 [https://bugs.python.org/issue?@action=redirect&bpo=34087]: Fix buffer overflow while converting unicode to numeric values.

- bpo-34080 [https://bugs.python.org/issue?@action=redirect&bpo=34080]: Fixed a memory leak in the compiler when it raised some uncommon errors during tokenizing.

- bpo-34066 [https://bugs.python.org/issue?@action=redirect&bpo=34066]: Disabled interruption by Ctrl-C between calling `open()` and entering a **with** block in `with open()`.

- bpo-34042 [https://bugs.python.org/issue?@action=redirect&bpo=34042]: Fix dict.copy() to maintain correct total refcount (as reported by sys.gettotalrefcount()).

- bpo-33418 [https://bugs.python.org/issue?@action=redirect&bpo=33418]: Fix potential memory leak in function object when it creates reference cycle.

- bpo-33985 [https://bugs.python.org/issue?@action=redirect&bpo=33985]: Implement contextvars.ContextVar.name attribute.

- bpo-33956 [https://bugs.python.org/issue?@action=redirect&bpo=33956]: Update vendored Expat library copy to version 2.2.5.

- bpo-24596 [https://bugs.python.org/issue?@action=redirect&bpo=24596]: Decref the module object in `PyRun_SimpleFileExFlags()` before calling `PyErr_Print()`. Patch by Zackery Spytz.

- bpo-33451 [https://bugs.python.org/issue?@action=redirect&bpo=33451]: Close directly executed pyc files before calling `PyEval_EvalCode()`.

- bpo-1617161 [https://bugs.python.org/issue?@action=redirect&bpo=1617161]: The hash of **`BuiltinMethodType`** instances (methods of built-in classes) now depends on the hash of the identity of *__self__* instead of its value. The hash and equality of **`ModuleType`** and **`MethodWrapperType`** instances (methods of user-defined classes and some methods of built-in classes like `str.__add__`) now depend on the hash and equality of the identity of *__self__* instead of its value. **`MethodWrapperType`** instances no longer support ordering.

- bpo-33824 [https://bugs.python.org/issue?@action=redirect&bpo=33824]: Fix "LC_ALL=C python3.7 -V": reset properly the command line parser when the encoding changes after reading the Python configuration.

- bpo-33803 [https://bugs.python.org/issue?@action=redirect&bpo=33803]: Fix a crash in hamt.c caused by enabling GC tracking for an object that hadn't all of its fields set to NULL.

- bpo-33738 [https://bugs.python.org/issue?@action=redirect&bpo=33738]: Seven macro incompatibilities with the Limited API were fixed, and the macros **`PyIter_Check()`**, **`PyIndex_Check()`** and **`PyExceptionClass_Name()`** were added as functions. A script for automatic macro checks was added.

- bpo-33786 [https://bugs.python.org/issue?@action=redirect&bpo=33786]: Fix asynchronous generators to handle GeneratorExit in athrow() correctly

- bpo-30167 [https://bugs.python.org/issue?@action=redirect&bpo=30167]: `PyRun_SimpleFileExFlags` removes `__cached__` from module in addition to `__file__`.

- bpo-33706 [https://bugs.python.org/issue?@action=redirect&bpo=33706]: Fix a crash in Python initialization when parsing the command line options. Thanks Christoph Gohlke for the bug report and the fix!

- bpo-33597 [https://bugs.python.org/issue?@action=redirect&bpo=33597]: Reduce `PyGC_Head` size from 3 words to 2 words.

- bpo-30654 [https://bugs.python.org/issue?@action=redirect&bpo=30654]: Fixed reset of the SIGINT handler to SIG_DFL on interpreter shutdown even when there was a custom handler set previously. Patch by Philipp Kerling.

- bpo-33622 [https://bugs.python.org/issue?@action=redirect&bpo=33622]: Fixed a leak when the garbage collector fails to add an object with the `__del__` method or referenced by it into the **`gc.garbage`** list. **`PyGC_Collect()`** can now be called when an exception is set and preserves it.

- bpo-33462 [https://bugs.python.org/issue?@action=redirect&bpo=33462]: Make dict and dict views reversible. Patch by Rémi Lapeyre.

- bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722]: A **`RuntimeError`** is now raised when the custom metaclass doesn't provide the `__classcell__` entry in the namespace passed to `type.__new__`. A **`DeprecationWarning`** was emitted in Python 3.6–3.7.

- bpo-33499 [https://bugs.python.org/issue?@action=redirect&bpo=33499]: Add **`PYTHONPYCACHEPREFIX`** environment variable and **`-X`** `pycache_prefix` command-line option to set an alternate root directory for writing module bytecode cache files.

- bpo-25711 [https://bugs.python.org/issue?@action=redirect&bpo=25711]: The **`zipimport`** module has been rewritten in pure Python.

- bpo-33509 [https://bugs.python.org/issue?@action=redirect&bpo=33509]: Fix module_globals parameter of warnings.warn_explicit(): don't crash if module_globals is not a dict.

- bpo-31849 [https://bugs.python.org/issue?@action=redirect&bpo=31849]: Fix signed/unsigned comparison warning in pyhash.c.

- bpo-33475 [https://bugs.python.org/issue?@action=redirect&bpo=33475]: Fixed miscellaneous bugs in converting annotations to strings and optimized parentheses in the string representation.

- bpo-20104 [https://bugs.python.org/issue?@action=redirect&bpo=20104]: Added support for the `setpgroup`, `resetids`, `setsigmask`, `setsigdef` and `scheduler` parameters of `posix_spawn`. Patch by Pablo Galindo.

- bpo-33391 [https://bugs.python.org/issue?@action=redirect&bpo=33391]: Fix a leak in set_symmetric_difference().

- bpo-33363 [https://bugs.python.org/issue?@action=redirect&bpo=33363]: Raise a SyntaxError for `async with` and `async for` statements outside of async functions.

- bpo-28055 [https://bugs.python.org/issue?@action=redirect&bpo=28055]: Fix unaligned accesses in siphash24(). Patch by Rolf Eike Beer.

- bpo-33128 [https://bugs.python.org/issue?@action=redirect&bpo=33128]: Fix a bug that causes PathFinder to appear twice on sys.meta_path. Patch by Pablo Galindo Salgado.

- bpo-33331 [https://bugs.python.org/issue?@action=redirect&bpo=33331]: Modules imported last are now cleared first at interpreter shutdown.

- bpo-33312 [https://bugs.python.org/issue?@action=redirect&bpo=33312]: Fixed clang ubsan (undefined behavior sanitizer) warnings in dictobject.c by adjusting how the internal struct _dictkeysobject shared keys structure is declared.

- bpo-33305 [https://bugs.python.org/issue?@action=redirect&bpo=33305]: Improved syntax error messages for invalid numerical literals.

- bpo-33306 [https://bugs.python.org/issue?@action=redirect&bpo=33306]: Improved syntax error messages for unbalanced parentheses.

- bpo-33234 [https://bugs.python.org/issue?@action=redirect&bpo=33234]: The list constructor will pre-size and not over-allocate when the input length is

known.

- bpo-33270 [https://bugs.python.org/issue?@action=redirect&bpo=33270]: Intern the names for all anonymous code objects. Patch by Zackery Spytz.

- bpo-30455 [https://bugs.python.org/issue?@action=redirect&bpo=30455]: The C and Python code and the documentation related to tokens are now generated from a single source file `Grammar/Tokens`.

- bpo-33176 [https://bugs.python.org/issue?@action=redirect&bpo=33176]: Add a `toreadonly()` method to memoryviews.

- bpo-33231 [https://bugs.python.org/issue?@action=redirect&bpo=33231]: Fix potential memory leak in `normalizestring()`.

- bpo-33205 [https://bugs.python.org/issue?@action=redirect&bpo=33205]: Change dict growth function from `round_up_to_power_2(used*2+hashtable_size/2)` to `round_up_to_power_2(used*3)`. Previously, dict is shrunk only when `used == 0`. Now dict has more chance to be shrunk.

- bpo-29922 [https://bugs.python.org/issue?@action=redirect&bpo=29922]: Improved error messages in 'async with' when `__aenter__()` or `__aexit__()` return non-awaitable object.

- bpo-33199 [https://bugs.python.org/issue?@action=redirect&bpo=33199]: Fix `ma_version_tag` in dict implementation is uninitialized when copying from key-sharing dict.

- bpo-33053 [https://bugs.python.org/issue?@action=redirect&bpo=33053]: When using the -m switch, sys.path[0] is now explicitly expanded as the *starting* working directory, rather than being left as the empty path (which allows imports from the current working directory at the time of the import)

- bpo-33138 [https://bugs.python.org/issue?@action=redirect&bpo=33138]: Changed standard error message for non-pickleable and non-copyable types. It

now says “cannot pickle” instead of “can’t pickle” or “cannot serialize”.

- bpo-33018 [https://bugs.python.org/issue?@action=redirect&bpo=33018]: Improve consistency of errors raised by `issubclass()` when called with a non-class and an abstract base class as the first and second arguments, respectively. Patch by Josh Bronson.

- bpo-33083 [https://bugs.python.org/issue?@action=redirect&bpo=33083]: `math.factorial` no longer accepts arguments that are not int-like. Patch by Pablo Galindo.

- bpo-33041 [https://bugs.python.org/issue?@action=redirect&bpo=33041]: Added new opcode **`END_ASYNC_FOR`** and fixes the following issues:

    - Setting global **`StopAsyncIteration`** no longer breaks `async for` loops.
    - Jumping into an `async for` loop is now disabled.
    - Jumping out of an `async for` loop no longer corrupts the stack.

- bpo-25750 [https://bugs.python.org/issue?@action=redirect&bpo=25750]: Fix rare Python crash due to bad refcounting in `type_getattro()` if a descriptor deletes itself from the class. Patch by Jeroen Demeyer.

- bpo-33041 [https://bugs.python.org/issue?@action=redirect&bpo=33041]: Fixed bytecode generation for “async for” with a complex target. A StopAsyncIteration raised on assigning or unpacking will be now propagated instead of stopping the iteration.

- bpo-33026 [https://bugs.python.org/issue?@action=redirect&bpo=33026]: Fixed jumping out of “with” block by setting f_lineno.

- bpo-33005 [https://bugs.python.org/issue?@action=redirect&bpo=33005]: Fix a crash on fork when using a custom memory allocator (ex: using PYTHONMALLOC env var). _PyGILState_Reinit() and _PyInterpreterState_Enable() now use the default RAW memory allocator to allocate a new interpreters mutex on fork.

- bpo-32911 [https://bugs.python.org/issue?@action=redirect&bpo=32911]: Due to unexpected compatibility issues discovered during downstream beta testing, reverted bpo-29463 [https://bugs.python.org/issue?@action=redirect&bpo=29463]. `docstring` field is removed from Module, ClassDef, FunctionDef, and AsyncFunctionDef ast nodes which was added in 3.7a1. Docstring expression is restored as a first statement in their body. Based on patch by Inada Naoki.

- bpo-17288 [https://bugs.python.org/issue?@action=redirect&bpo=17288]: Prevent jumps from 'return' and 'exception' trace events.

- bpo-32946 [https://bugs.python.org/issue?@action=redirect&bpo=32946]: Importing names from already imported module with "from … import …" is now 30% faster if the module is not a package.

- bpo-32932 [https://bugs.python.org/issue?@action=redirect&bpo=32932]: Make error message more revealing when there are non-str objects in `__all__`.

- bpo-32925 [https://bugs.python.org/issue?@action=redirect&bpo=32925]: Optimized iterating and containing test for literal lists consisting of non-constants: `x in [a, b]` and `for x in [a, b]`. The case of all constant elements already was optimized.

- bpo-32889 [https://bugs.python.org/issue?@action=redirect&bpo=32889]: Update Valgrind suppression list to account for the rename of `Py_ADDRESS_IN_RANG` to `address_in_range`.

- bpo-32836 [https://bugs.python.org/issue?@action=redirect&bpo=32836]: Don't use temporary variables in cases of list/dict/set comprehensions

- bpo-31356 [https://bugs.python.org/issue?@action=redirect&bpo=31356]: Remove the new API added in bpo-31356 [https://bugs.python.org/issue?@action=redirect&bpo=31356] (gc.ensure_disabled() context manager).

- bpo-32305 [https://bugs.python.org/issue?@action=redirect&bpo=32305]: For namespace packages, ensure that both `__file__` and `__spec__.origin` are set to None.

- bpo-32303 [https://bugs.python.org/issue?@action=redirect&bpo=32303]: Make sure `__spec__.loader` matches `__loader__` for namespace packages.

- bpo-32711 [https://bugs.python.org/issue?@action=redirect&bpo=32711]: Fix the warning messages for Python/ast_unparse.c. Patch by Stéphane Wirtel

- bpo-32583 [https://bugs.python.org/issue?@action=redirect&bpo=32583]: Fix possible crashing in builtin Unicode decoders caused by write out-of-bound errors when using customized decode error handlers.

- bpo-32489 [https://bugs.python.org/issue?@action=redirect&bpo=32489]: A **`continue`** statement is now allowed in the **`finally`** clause.

- bpo-17611 [https://bugs.python.org/issue?@action=redirect&bpo=17611]: Simplified the interpreter loop by moving the logic of unrolling the stack of blocks into the compiler. The compiler emits now explicit instructions for adjusting the stack of values and calling the cleaning up code for **`break`**, **`continue`** and **`return`**.

  Removed opcodes **`BREAK_LOOP`**, **`CONTINUE_LOOP`**, **`SETUP_LOOP`** and **`SETUP_EXCEPT`**. Added new opcodes **`ROT_FOUR`**, **`BEGIN_FINALLY`** and **`CALL_FINALLY`** and **`POP_FINALLY`**. Changed the behavior of **`END_FINALLY`** and **`WITH_CLEANUP_START`**.

- bpo-32285 [https://bugs.python.org/issue?@action=redirect&bpo=32285]: New function unicodedata.is_normalized, which can check whether a string is in a specific normal form.

- bpo-10544 [https://bugs.python.org/issue?@action=redirect&bpo=10544]: Yield expressions are now disallowed in comprehensions and generator expressions except the expression for the outermost iterable.

- bpo-32117 [https://bugs.python.org/issue?@action=redirect&bpo=32117]: Iterable unpacking is now allowed without parentheses in yield and return statements, e.g. `yield 1, 2, 3, *rest`. Thanks to David Cuthbert for the change and Jordan Chapman for added tests.

- bpo-31902 [https://bugs.python.org/issue?@action=redirect&bpo=31902]: Fix the `col_offset` attribute for ast nodes `ast.AsyncFor`, `ast.AsyncFunctionDef`, and `ast.AsyncWith`. Previously, `col_offset` pointed to the keyword after `async`.

- bpo-25862 [https://bugs.python.org/issue?@action=redirect&bpo=25862]: Fix assertion failures in the `tell()` method of `io.TextIOWrapper`. Patch by Zackery Spytz.

- bpo-21983 [https://bugs.python.org/issue?@action=redirect&bpo=21983]: Fix a crash in `ctypes.cast()` in case the type argument is a ctypes structured data type. Patch by Eryk Sun and Oren Milman.

- bpo-31577 [https://bugs.python.org/issue?@action=redirect&bpo=31577]: Fix a crash in `os.utime()` in case of a bad ns argument. Patch by Oren Milman.

- bpo-29832 [https://bugs.python.org/issue?@action=redirect&bpo=29832]: Remove references to 'getsockaddrarg' from various socket error messages. Patch by Oren Milman.

Biblioteca

- bpo-35845 [https://bugs.python.org/issue?@action=redirect&bpo=35845]: Add 'order' parameter to memoryview.tobytes().

- bpo-35864 [https://bugs.python.org/issue?@action=redirect&bpo=35864]: The _asdict() method for collections.namedtuple now returns a regular dict instead of an OrderedDict.

- bpo-35537 [https://bugs.python.org/issue?@action=redirect&bpo=35537]: An ExitStack is now used internally within subprocess.Popen to clean up pipe file handles. No behavior change in normal operation. But if closing one handle were ever to cause an exception, the others will now be closed instead of leaked. (patch by Giampaolo Rodola)

- bpo-35847 [https://bugs.python.org/issue?@action=redirect&bpo=35847]: RISC-V needed the CTYPES_PASS_BY_REF_HACK. Fixes ctypes Structure test_pass_by_value.

- bpo-35813 [https://bugs.python.org/issue?@action=redirect&bpo=35813]: Shared memory submodule added to multiprocessing to avoid need for serialization between processes

- bpo-35780 [https://bugs.python.org/issue?@action=redirect&bpo=35780]: Fix lru_cache() errors arising in recursive, reentrant, or multi-threaded code. These errors could result in orphan links and in the cache being trapped in a state with fewer than the specified maximum number of links. Fix handling of negative maxsize which should have been treated as zero. Fix errors in toggling the “full” status flag. Fix misordering of links when errors are encountered. Sync-up the C code and pure Python code for the space saving path in functions with a single positional argument. In this common case, the space overhead of an lru cache entry is reduced by almost half. Fix counting of cache misses. In error cases, the miss count was out of sync with the actual number of times the underlying user function was called.

- bpo-35537 [https://bugs.python.org/issue?@action=redirect&bpo=35537]: **`os.posix_spawn()`** and **`os.posix_spawnp()`** now have a *setsid* parameter.

- bpo-23846 [https://bugs.python.org/issue?@action=redirect&bpo=23846]: **`asyncio.ProactorEventLoop`** now catches and logs send errors when the self-pipe is full.

- bpo-34323 [https://bugs.python.org/issue?@action=redirect&bpo=34323]: **`asyncio`**: Enhance `IocpProactor.close()` log: wait 1 second before the first log, then log every second. Log also the number of seconds since `close()` was called.

- bpo-35674 [https://bugs.python.org/issue?@action=redirect&bpo=35674]: Add a new **`os.posix_spawnp()`** function. Patch by Joannah Nanjekye.

- bpo-35733 [https://bugs.python.org/issue?@action=redirect&bpo=35733]: `ast.Constant(boolean)` no longer an instance of **`ast.Num`**. Patch by Anthony Sottile.

- bpo-35726 [https://bugs.python.org/issue?@action=redirect&bpo=35726]: QueueHandler.prepare() now makes a copy of the record before modifying and enqueueing it, to avoid affecting other handlers in the chain.

- bpo-35719 [https://bugs.python.org/issue?@action=redirect&bpo=35719]: Sped up multi-argument **`math`** functions atan2(), copysign(), remainder() and hypot() by 1.3–2.5 times.

- bpo-35717 [https://bugs.python.org/issue?@action=redirect&bpo=35717]: Fix KeyError exception raised when using enums and compile. Patch contributed by Rémi Lapeyre.

- bpo-35699 [https://bugs.python.org/issue?@action=redirect&bpo=35699]: Fixed detection of Visual Studio Build Tools 2017 in distutils

- bpo-32710 [https://bugs.python.org/issue?@action=redirect&bpo=32710]: Fix memory leaks in asyncio ProactorEventLoop on overlapped operation failure.

- bpo-35702 [https://bugs.python.org/issue?@action=redirect&bpo=35702]: The **`time.CLOCK_UPTIME_RAW`** constant is now available for macOS 10.12.

- bpo-32710 [https://bugs.python.org/issue?@action=redirect&bpo=32710]: Fix a memory leak in asyncio in the ProactorEventLoop when `ReadFile()` or `WSASend()` overlapped operation fail immediately: release the internal buffer.

- bpo-35682 [https://bugs.python.org/issue?@action=redirect&bpo=35682]: Fix `asyncio.ProactorEventLoop.sendfile()`: don't attempt to set the result of an internal future if it's already done.

- bpo-35283 [https://bugs.python.org/issue?@action=redirect&bpo=35283]: Add a deprecated warning for the **`threading.Thread.isAlive()`** method. Patch by Donghee Na.

- bpo-35664 [https://bugs.python.org/issue?@action=redirect&bpo=35664]: Improve operator.itemgetter() performance by 33% with optimized argument

handling and with adding a fast path for the common case of a single non-negative integer index into a tuple (which is the typical use case in the standard library).

- bpo-35643 [https://bugs.python.org/issue?@action=redirect&bpo=35643]: Fixed a SyntaxWarning: invalid escape sequence in Modules/_sha3/cleanup.py. Patch by Mickaël Schoentgen.

- bpo-35619 [https://bugs.python.org/issue?@action=redirect&bpo=35619]: Improved support of custom data descriptors in `help()` and `pydoc`.

- bpo-28503 [https://bugs.python.org/issue?@action=redirect&bpo=28503]: The `crypt` module now internally uses the `crypt_r()` library function instead of `crypt()` when available.

- bpo-35614 [https://bugs.python.org/issue?@action=redirect&bpo=35614]: Fixed help() on metaclasses. Patch by Sanyam Khurana.

- bpo-35568 [https://bugs.python.org/issue?@action=redirect&bpo=35568]: Expose `raise(signum)` as `raise_signal`

- bpo-35588 [https://bugs.python.org/issue?@action=redirect&bpo=35588]: The floor division and modulo operations and the `divmod()` function on `fractions.Fraction` types are 2–4x faster. Patch by Stefan Behnel.

- bpo-35585 [https://bugs.python.org/issue?@action=redirect&bpo=35585]: Speed-up building enums by value, e.g. http.HTTPStatus(200).

- bpo-30561 [https://bugs.python.org/issue?@action=redirect&bpo=30561]: random.gammavariate(1.0, beta) now computes the same result as random.expovariate(1.0 / beta). This synchronizes the two algorithms and eliminates some idiosyncrasies in the old implementation. It does however produce a difference stream of random variables than it used to.

- bpo-35537 [https://bugs.python.org/issue?@action=redirect&bpo=35537]: The `subprocess` module can now use the `os.posix_spawn()` function in some cases for better performance.

- bpo-35526 [https://bugs.python.org/issue?@action=redirect&bpo=35526]: Delaying the ‘joke’ of barry_as_FLUFL.mandatory to Python version 4.0

- bpo-35523 [https://bugs.python.org/issue?@action=redirect&bpo=35523]: Remove **`ctypes`** callback workaround: no longer create a callback at startup. Avoid SELinux alert on `import ctypes` and `import uuid`.

- bpo-31784 [https://bugs.python.org/issue?@action=redirect&bpo=31784]: **`uuid.uuid1()`** now calls **`time.time_ns()`** rather than `int(time.time() * 1e9)`.

- bpo-35513 [https://bugs.python.org/issue?@action=redirect&bpo=35513]: **`TextTestRunner`** of **`unittest.runner`** now uses **`time.perf_counter()`** rather than **`time.time()`** to measure the execution time of a test: **`time.time()`** can go backwards, whereas **`time.perf_counter()`** is monotonic.

- bpo-35502 [https://bugs.python.org/issue?@action=redirect&bpo=35502]: Fixed reference leaks in **`xml.etree.ElementTree.TreeBuilder`** in case of unfinished building of the tree (in particular when an error was raised during parsing XML).

- bpo-35348 [https://bugs.python.org/issue?@action=redirect&bpo=35348]: Make **`platform.architecture()`** parsing of `file` command output more reliable: add the `-b` option to the `file` command to omit the filename, force the usage of the C locale, and search also the “shared object” pattern.

- bpo-35491 [https://bugs.python.org/issue?@action=redirect&bpo=35491]: **`multiprocessing`**: Add `Pool.__repr__()` and enhance `BaseProcess.__repr__()` (add pid and parent pid) to ease debugging. Pool state constant values are now strings instead of integers, for example `RUN` value becomes `'RUN'` instead of `0`.

- bpo-35477 [https://bugs.python.org/issue?@action=redirect&bpo=35477]: **`multiprocessing.Pool.__enter__()`** now fails if the pool is not running: `with pool:` fails if used more than once.

- bpo-31446 [https://bugs.python.org/issue?@action=redirect&bpo=31446]: Copy command line that was passed to CreateProcessW since this function can change the content of the input buffer.

- bpo-35471 [https://bugs.python.org/issue?@action=redirect&bpo=35471]: Python 2.4 dropped MacOS 9 support. The macpath module was deprecated in Python 3.7. The module is now removed.

- bpo-23057 [https://bugs.python.org/issue?@action=redirect&bpo=23057]: Unblock Proactor event loop when keyboard interrupt is received on Windows

- bpo-35052 [https://bugs.python.org/issue?@action=redirect&bpo=35052]: Fix xml.dom.minidom cloneNode() on a document with an entity: pass the correct arguments to the user data handler of an entity.

- bpo-20239 [https://bugs.python.org/issue?@action=redirect&bpo=20239]: Allow repeated assignment deletion of `unittest.mock.Mock` attributes. Patch by Pablo Galindo.

- bpo-17185 [https://bugs.python.org/issue?@action=redirect&bpo=17185]: Set `__signature__` on mock for `inspect` to get signature. Patch by Karthikeyan Singaravelan.

- bpo-35445 [https://bugs.python.org/issue?@action=redirect&bpo=35445]: Memory errors during creating posix.environ no longer ignored.

- bpo-35415 [https://bugs.python.org/issue?@action=redirect&bpo=35415]: Validate fileno= argument to socket.socket().

- bpo-35424 [https://bugs.python.org/issue?@action=redirect&bpo=35424]: `multiprocessing.Pool` destructor now emits `ResourceWarning` if the pool is still running.

- bpo-35330 [https://bugs.python.org/issue?@action=redirect&bpo=35330]: When a `Mock` instance was used to wrap an object, if `side_effect` is used in one of the mocks of it methods, don't call the original implementation and return the result of using the side effect the same way that it is done with return_value.

- bpo-35346 [https://bugs.python.org/issue?@action=redirect&bpo=35346]: Drop Mac OS 9 and Rhapsody support from the **`platform`** module. Rhapsody last release was in 2000. Mac OS 9 last release was in 2001.

- bpo-10496 [https://bugs.python.org/issue?@action=redirect&bpo=10496]: **`check_environ()`** of `distutils.utils` now catches **`KeyError`** on calling **`pwd.getpwuid()`**: don't create the `HOME` environment variable in this case.

- bpo-10496 [https://bugs.python.org/issue?@action=redirect&bpo=10496]: **`posixpath.expanduser()`** now returns the input *path* unchanged if the `HOME` environment variable is not set and the current user has no home directory (if the current user identifier doesn't exist in the password database). This change fix the **`site`** module if the current user doesn't exist in the password database (if the user has no home directory).

- bpo-35389 [https://bugs.python.org/issue?@action=redirect&bpo=35389]: **`platform.libc_ver()`** now uses `os.confstr('CS_GNU_LIBC_VERSION')` if available and the *executable* parameter is not set.

- bpo-35394 [https://bugs.python.org/issue?@action=redirect&bpo=35394]: Add empty slots to asyncio abstract protocols.

- bpo-35310 [https://bugs.python.org/issue?@action=redirect&bpo=35310]: Fix a bug in **`select.select()`** where, in some cases, the file descriptor sequences were returned unmodified after a signal interruption, even though the file descriptors might not be ready yet. **`select.select()`** will now always return empty lists if a timeout has occurred. Patch by Oran Avraham.

- bpo-35380 [https://bugs.python.org/issue?@action=redirect&bpo=35380]: Enable TCP_NODELAY on Windows for proactor asyncio event loop.

- bpo-35341 [https://bugs.python.org/issue?@action=redirect&bpo=35341]: Add generic version of `collections.OrderedDict` to the `typing` module. Patch by Ismo Toijala.

- bpo-35371 [https://bugs.python.org/issue?@action=redirect&bpo=35371]: Fixed possible crash in `os.utime()` on Windows when pass incorrect arguments.

- bpo-35346 [https://bugs.python.org/issue?@action=redirect&bpo=35346]: **`platform.uname()`** now redirects `stderr` to **`os.devnull`** when running external programs like `cmd /c ver`.

- bpo-35066 [https://bugs.python.org/issue?@action=redirect&bpo=35066]: Previously, calling the strftime() method on a datetime object with a trailing '%' in the format string would result in an exception. However, this only occurred when the datetime C module was being used; the python implementation did not match this behavior. Datetime is now PEP-399 compliant, and will not throw an exception on a trailing '%'.

- bpo-35345 [https://bugs.python.org/issue?@action=redirect&bpo=35345]: The function `platform.popen` has been removed, it was deprecated since Python 3.3: use **`os.popen()`** instead.

- bpo-35344 [https://bugs.python.org/issue?@action=redirect&bpo=35344]: On macOS, **`platform.platform()`** now uses **`platform.mac_ver()`**, if it returns a non-empty release string, to get the macOS version rather than the darwin version.

- bpo-35312 [https://bugs.python.org/issue?@action=redirect&bpo=35312]: Make `lib2to3.pgen2.parse.ParseError` round-trip pickle-able. Patch by Anthony Sottile.

- bpo-35308 [https://bugs.python.org/issue?@action=redirect&bpo=35308]: Fix regression in `webbrowser` where default browsers may be preferred over browsers in the `BROWSER` environment variable.

- bpo-24746 [https://bugs.python.org/issue?@action=redirect&bpo=24746]: Avoid stripping trailing whitespace in doctest fancy diff. Original patch by R. David Murray & Jairo Trad. Enhanced by Sanyam Khurana.

- bpo-28604 [https://bugs.python.org/issue?@action=redirect&bpo=28604]: **`locale.localeconv()`** now sets temporarily the `LC_CTYPE` locale to the

`LC_MONETARY` locale if the two locales are different and monetary strings are non-ASCII. This temporary change affects other threads.

- bpo-35277 [https://bugs.python.org/issue?@action=redirect&bpo=35277]: Update ensurepip to install pip 18.1 and setuptools 40.6.2.

- bpo-24209 [https://bugs.python.org/issue?@action=redirect&bpo=24209]: Adds IPv6 support when invoking http.server directly.

- bpo-35226 [https://bugs.python.org/issue?@action=redirect&bpo=35226]: Recursively check arguments when testing for equality of **`unittest.mock.call`** objects and add note that tracking of parameters used to create ancestors of mocks in `mock_calls` is not possible.

- bpo-29564 [https://bugs.python.org/issue?@action=redirect&bpo=29564]: The warnings module now suggests to enable tracemalloc if the source is specified, the tracemalloc module is available, but tracemalloc is not tracing memory allocations.

- bpo-35189 [https://bugs.python.org/issue?@action=redirect&bpo=35189]: Modify the following fnctl function to retry if interrupted by a signal (EINTR): flock, lockf, fnctl

- bpo-30064 [https://bugs.python.org/issue?@action=redirect&bpo=30064]: Use add_done_callback() in sock_* asyncio API to unsubscribe reader/writer early on calcellation.

- bpo-35186 [https://bugs.python.org/issue?@action=redirect&bpo=35186]: Removed the “built with” comment added when `setup.py upload` is used with either `bdist_rpm` or `bdist_dumb`.

- bpo-35152 [https://bugs.python.org/issue?@action=redirect&bpo=35152]: Allow sending more than 2 GB at once on a multiprocessing connection on non-Windows systems.

- bpo-35062 [https://bugs.python.org/issue?@action=redirect&bpo=35062]: Fix incorrect parsing of **`io.IncrementalNewlineDecoder`**’s *translate* argument.

- bpo-35065 [https://bugs.python.org/issue?@action=redirect&bpo=35065]: Remove `StreamReaderProtocol._untrack_reader`. The call to `_untrack_reader` is currently performed too soon, causing the protocol to forget about the reader before `connection_lost` can run and feed the EOF to the reader.

- bpo-34160 [https://bugs.python.org/issue?@action=redirect&bpo=34160]: ElementTree and minidom now preserve the attribute order specified by the user.

- bpo-35079 [https://bugs.python.org/issue?@action=redirect&bpo=35079]: Improve difflib.SequenceManager.get_matching_blocks doc by adding 'non-overlapping' and changing '!=' to '<'.

- bpo-33710 [https://bugs.python.org/issue?@action=redirect&bpo=33710]: Deprecated `l*gettext()` functions and methods in the **`gettext`** module. They return encoded bytes instead of Unicode strings and are artifacts from Python 2 times. Also deprecated functions and methods related to setting the charset for `l*gettext()` functions and methods.

- bpo-35017 [https://bugs.python.org/issue?@action=redirect&bpo=35017]: **`socketserver.BaseServer.serve_forever()`** now exits immediately if it's **`shutdown()`** method is called while it is polling for new events.

- bpo-35024 [https://bugs.python.org/issue?@action=redirect&bpo=35024]: `importlib` no longer logs `wrote <bytecode path>` redundantly after `(created|could not create) <bytecode path>` is already logged. Patch by Quentin Agren.

- bpo-35047 [https://bugs.python.org/issue?@action=redirect&bpo=35047]: `unittest.mock` now includes mock calls in exception messages if `assert_not_called`, `assert_called_once`, or `assert_called_once_with` fails. Patch by Petter Strandmark.

- bpo-31047 [https://bugs.python.org/issue?@action=redirect&bpo=31047]: Fix `ntpath.abspath` regression where it didn't remove a trailing separator on Windows. Patch by Tim Graham.

- bpo-35053 [https://bugs.python.org/issue?@action=redirect&bpo=35053]: tracemalloc now tries to update the traceback when an object is reused from a “free list” (optimization for faster object creation, used by the builtin list type for example).

- bpo-31553 [https://bugs.python.org/issue?@action=redirect&bpo=31553]: Add the –json-lines option to json.tool. Patch by hongweipeng.

- bpo-34794 [https://bugs.python.org/issue?@action=redirect&bpo=34794]: Fixed a leak in Tkinter when pass the Python wrapper around Tcl_Obj back to Tcl/Tk.

- bpo-34909 [https://bugs.python.org/issue?@action=redirect&bpo=34909]: Enum: fix grandchildren subclassing when parent mixed with concrete data types.

- bpo-35022 [https://bugs.python.org/issue?@action=redirect&bpo=35022]: **`unittest.mock.MagicMock`** now supports the `__fspath__` method (from **`os.PathLike`**).

- bpo-35008 [https://bugs.python.org/issue?@action=redirect&bpo=35008]: Fixed references leaks when call the `__setstate__()` method of **`xml.etree.ElementTree.Element`** in the C implementation for already initialized element.

- bpo-23420 [https://bugs.python.org/issue?@action=redirect&bpo=23420]: Verify the value for the parameter ‘-s’ of the cProfile CLI. Patch by Robert Kuska

- bpo-33947 [https://bugs.python.org/issue?@action=redirect&bpo=33947]: dataclasses now handle recursive reprs without raising RecursionError.

- bpo-34890 [https://bugs.python.org/issue?@action=redirect&bpo=34890]: Make **`inspect.iscoroutinefunction()`**, **`inspect.isgeneratorfunction()`** and **`inspect.isasyncgenfunction()`** work with **`functools.partial()`**. Patch by Pablo Galindo.

- bpo-34521 [https://bugs.python.org/issue?@action=redirect&bpo=34521]: Use **`socket.CMSG_SPACE()`** to calculate ancillary data size instead of **`socket.CMSG_LEN()`** in **`multiprocessing.reduction.recvfds()`** as

RFC 3542 [https://datatracker.ietf.org/doc/html/rfc3542.html] requires the use of the former for portable applications.

- bpo-31522 [https://bugs.python.org/issue?@action=redirect&bpo=31522]: The `mailbox.mbox.get_string` function *from_* parameter can now successfully be set to a non-default value.

- bpo-34970 [https://bugs.python.org/issue?@action=redirect&bpo=34970]: Protect tasks weak set manipulation in `asyncio.all_tasks()`

- bpo-34969 [https://bugs.python.org/issue?@action=redirect&bpo=34969]: gzip: Add –fast, –best on the gzip CLI, these parameters will be used for the fast compression method (quick) or the best method compress (slower, but smaller file). Also, change the default compression level to 6 (tradeoff).

- bpo-16965 [https://bugs.python.org/issue?@action=redirect&bpo=16965]: The 2to3 `execfile` fixer now opens the file with mode `'rb'`. Patch by Zackery Spytz.

- bpo-34966 [https://bugs.python.org/issue?@action=redirect&bpo=34966]: **`pydoc`** now supports aliases not only to methods defined in the end class, but also to inherited methods. The docstring is not duplicated for aliases.

- bpo-34926 [https://bugs.python.org/issue?@action=redirect&bpo=34926]: **`mimetypes.MimeTypes.guess_type()`** now accepts path-like object in addition to url strings. Patch by Mayank Asthana.

- bpo-23831 [https://bugs.python.org/issue?@action=redirect&bpo=23831]: Add `moveto()` method to the `tkinter.Canvas` widget. Patch by Juliette Monsel.

- bpo-34941 [https://bugs.python.org/issue?@action=redirect&bpo=34941]: Methods `find()`, `findtext()` and `findall()` of the `Element` class in the **`xml.etree.ElementTree`** module are now able to find children which are instances of `Element` subclasses.

- bpo-32680 [https://bugs.python.org/issue?@action=redirect&bpo=32680]: **`smtplib.SMTP`** objects now always have a `sock` attribute present

- bpo-34769 [https://bugs.python.org/issue?@action=redirect&bpo=34769]: Fix for async generators not finalizing when event loop is in debug mode and garbage collector runs in another thread.

- bpo-34936 [https://bugs.python.org/issue?@action=redirect&bpo=34936]: Fix `TclError` in `tkinter.Spinbox.selection_element()`. Patch by Juliette Monsel.

- bpo-34829 [https://bugs.python.org/issue?@action=redirect&bpo=34829]: Add methods `selection_from`, `selection_range`, `selection_present` and `selection_to` to the `tkinter.Spinbox` for consistency with the `tkinter.Entry` widget. Patch by Juliette Monsel.

- bpo-34911 [https://bugs.python.org/issue?@action=redirect&bpo=34911]: Added *secure_protocols* argument to *http.cookiejar.DefaultCookiePolicy* to allow for tweaking of protocols and also to add support by default for *wss*, the secure websocket protocol.

- bpo-34922 [https://bugs.python.org/issue?@action=redirect&bpo=34922]: Fixed integer overflow in the **`digest()`** and **`hexdigest()`** methods for the SHAKE algorithm in the **`hashlib`** module.

- bpo-34925 [https://bugs.python.org/issue?@action=redirect&bpo=34925]: 25% speedup in argument parsing for the functions in the bisect module.

- bpo-34900 [https://bugs.python.org/issue?@action=redirect&bpo=34900]: Fixed **`unittest.TestCase.debug()`** when used to call test methods with subtests. Patch by Bruno Oliveira.

- bpo-34844 [https://bugs.python.org/issue?@action=redirect&bpo=34844]: logging.Formatter enhancement - Ensure styles and fmt matches in logging.Formatter - Added validate method in each format style class: StrFormatStyle, PercentStyle, StringTemplateStyle. - This method is called in the constructor of logging.Formatter class - Also re-raise the

KeyError in the format method of each style class, so it would a bit clear that it's an error with the invalid format fields.

- bpo-34897 [https://bugs.python.org/issue?@action=redirect&bpo=34897]: Adjust test.support.missing_compiler_executable check so that a nominal command name of "" is ignored. Patch by Michael Felt.

- bpo-34871 [https://bugs.python.org/issue?@action=redirect&bpo=34871]: Fix inspect module polluted `sys.modules` when parsing `__text_signature__` of callable.

- bpo-34898 [https://bugs.python.org/issue?@action=redirect&bpo=34898]: Add `mtime` argument to `gzip.compress` for reproducible output. Patch by Guo Ci Teo.

- bpo-28441 [https://bugs.python.org/issue?@action=redirect&bpo=28441]: On Cygwin and MinGW, ensure that `sys.executable` always includes the full filename in the path, including the `.exe` suffix (unless it is a symbolic link).

- bpo-34866 [https://bugs.python.org/issue?@action=redirect&bpo=34866]: Adding `max_num_fields` to `cgi.FieldStorage` to make DOS attacks harder by limiting the number of `MiniFieldStorage` objects created by `FieldStorage`.

- bpo-34711 [https://bugs.python.org/issue?@action=redirect&bpo=34711]: http.server ensures it reports HTTPStatus.NOT_FOUND when the local path ends with "/" and is not a directory, even if the underlying OS (e.g. AIX) accepts such paths as a valid file reference. Patch by Michael Felt.

- bpo-34872 [https://bugs.python.org/issue?@action=redirect&bpo=34872]: Fix self-cancellation in C implementation of asyncio.Task

- bpo-34849 [https://bugs.python.org/issue?@action=redirect&bpo=34849]: Don't log waiting for `selector.select` in asyncio loop iteration. The waiting is pretty normal for any asyncio program, logging its time just adds a noise to logs without any useful information provided.

- bpo-34022 [https://bugs.python.org/issue?@action=redirect&bpo=34022]: The **`SOURCE_DATE_EPOCH`** environment variable no longer overrides the value of the *invalidation_mode* argument to **`py_compile.compile()`**, and determines its default value instead.

- bpo-34819 [https://bugs.python.org/issue?@action=redirect&bpo=34819]: Use a monotonic clock to compute timeouts in **`Executor.map()`** and **`as_completed()`**, in order to prevent timeouts from deviating when the system clock is adjusted.

- bpo-34758 [https://bugs.python.org/issue?@action=redirect&bpo=34758]: Add .wasm -> application/wasm to list of recognized file types and content type headers

- bpo-34789 [https://bugs.python.org/issue?@action=redirect&bpo=34789]: **`xml.sax.make_parser()`** now accepts any iterable as its *parser_list* argument. Patch by Andrés Delfino.

- bpo-34334 [https://bugs.python.org/issue?@action=redirect&bpo=34334]: In **`QueueHandler`**, clear `exc_text` from **`LogRecord`** to prevent traceback from being written twice.

- bpo-34687 [https://bugs.python.org/issue?@action=redirect&bpo=34687]: On Windows, asyncio now uses ProactorEventLoop, instead of SelectorEventLoop, by default.

- bpo-5950 [https://bugs.python.org/issue?@action=redirect&bpo=5950]: Support reading zip files with archive comments in **`zipimport`**.

- bpo-32892 [https://bugs.python.org/issue?@action=redirect&bpo=32892]: The parser now represents all constants as **`ast.Constant`** instead of using specific constant AST types (`Num`, `Str`, `Bytes`, `NameConstant` and `Ellipsis`). These classes are considered deprecated and will be removed in future Python versions.

- bpo-34728 [https://bugs.python.org/issue?@action=redirect&bpo=34728]: Add deprecation warning when `loop` is used in methods: `asyncio.sleep`, `asyncio.wait` and `asyncio.wait_for`.

- bpo-34738 [https://bugs.python.org/issue?@action=redirect&bpo=34738]: ZIP files created by `distutils` will now include entries for directories.

- bpo-34659 [https://bugs.python.org/issue?@action=redirect&bpo=34659]: Add an optional *initial* argument to itertools.accumulate().

- bpo-29577 [https://bugs.python.org/issue?@action=redirect&bpo=29577]: Support multiple mixin classes when creating Enums.

- bpo-34670 [https://bugs.python.org/issue?@action=redirect&bpo=34670]: Add SSLContext.post_handshake_auth and SSLSocket.verify_client_post_handshake for TLS 1.3's post handshake authentication feature.

- bpo-32718 [https://bugs.python.org/issue?@action=redirect&bpo=32718]: The Activate.ps1 script from venv works with PowerShell Core 6.1 and is now available under all operating systems.

- bpo-31177 [https://bugs.python.org/issue?@action=redirect&bpo=31177]: Fix bug that prevented using **`reset_mock`** on mock instances with deleted attributes

- bpo-34672 [https://bugs.python.org/issue?@action=redirect&bpo=34672]: Add a workaround, so the `'Z'` **`time.strftime()`** specifier on the musl C library can work in some cases.

- bpo-34666 [https://bugs.python.org/issue?@action=redirect&bpo=34666]: Implement `asyncio.StreamWriter.awrite` and `asyncio.StreamWriter.aclose()` coroutines. Methods are needed for providing a consistent stream API with control flow switched on by default.

- bpo-6721 [https://bugs.python.org/issue?@action=redirect&bpo=6721]: Acquire the logging module's commonly used internal locks while fork()ing to avoid deadlocks in the child process.

- bpo-34658 [https://bugs.python.org/issue?@action=redirect&bpo=34658]: Fix a rare interpreter unhandled exception state SystemError only seen when

using subprocess with a preexec_fn while an after_parent handler has been registered with os.register_at_fork and the fork system call fails.

- bpo-34652 [https://bugs.python.org/issue?@action=redirect&bpo=34652]: Ensure **`os.lchmod()`** is never defined on Linux.

- bpo-34638 [https://bugs.python.org/issue?@action=redirect&bpo=34638]: Store a weak reference to stream reader to break strong references loop between reader and protocol. It allows to detect and close the socket if the stream is deleted (garbage collected) without `close()` call.

- bpo-34536 [https://bugs.python.org/issue?@action=redirect&bpo=34536]: `Enum._missing_`: raise `ValueError` if None returned and `TypeError` if non-member is returned.

- bpo-34636 [https://bugs.python.org/issue?@action=redirect&bpo=34636]: Speed up re scanning of many non-matching characters for s w and d within bytes objects. (microoptimization)

- bpo-24412 [https://bugs.python.org/issue?@action=redirect&bpo=24412]: Add **`addModuleCleanup()`** and **`addClassCleanup()`** to unittest to support cleanups for **`setUpModule()`** and **`setUpClass()`**. Patch by Lisa Roach.

- bpo-34630 [https://bugs.python.org/issue?@action=redirect&bpo=34630]: Don’t log SSL certificate errors in asyncio code (connection error logging is skipped already).

- bpo-32490 [https://bugs.python.org/issue?@action=redirect&bpo=32490]: Prevent filename duplication in **`subprocess`** exception messages. Patch by Zackery Spytz.

- bpo-34363 [https://bugs.python.org/issue?@action=redirect&bpo=34363]: dataclasses.asdict() and .astuple() now handle namedtuples correctly.

- bpo-34625 [https://bugs.python.org/issue?@action=redirect&bpo=34625]: Update vendorized expat library version to 2.2.6.

- bpo-32270 [https://bugs.python.org/issue?@action=redirect&bpo=32270]: The subprocess module no longer mistakenly closes redirected fds even when they were in pass_fds when outside of the default {0, 1, 2} set.

- bpo-34622 [https://bugs.python.org/issue?@action=redirect&bpo=34622]: Create a dedicated `asyncio.CancelledError`, `asyncio.InvalidStateError` and `asyncio.TimeoutError` exception classes. Inherit them from corresponding exceptions from `concurrent.futures` package. Extract `asyncio` exceptions into a separate file.

- bpo-34610 [https://bugs.python.org/issue?@action=redirect&bpo=34610]: Fixed iterator of **`multiprocessing.managers.DictProxy`**.

- bpo-34421 [https://bugs.python.org/issue?@action=redirect&bpo=34421]: Fix distutils logging for non-ASCII strings. This caused installation issues on Windows.

- bpo-34604 [https://bugs.python.org/issue?@action=redirect&bpo=34604]: Fix possible mojibake in the error message of `pwd.getpwnam` and `grp.getgrnam` using string representation because of invisible characters or trailing whitespaces. Patch by William Grzybowski.

- bpo-30977 [https://bugs.python.org/issue?@action=redirect&bpo=30977]: Make uuid.UUID use `__slots__` to reduce its memory footprint. Based on original patch by Wouter Bolsterlee.

- bpo-34574 [https://bugs.python.org/issue?@action=redirect&bpo=34574]: OrderedDict iterators are not exhausted during pickling anymore. Patch by Sergey Fedoseev.

- bpo-8110 [https://bugs.python.org/issue?@action=redirect&bpo=8110]: Refactored **`subprocess`** to check for Windows-specific modules rather than `sys.platform == 'win32'`.

- bpo-34530 [https://bugs.python.org/issue?@action=redirect&bpo=34530]: `distutils.spawn.find_executable()` now falls back on **`os.defpath`** if the `PATH` environment variable is not set.

- bpo-34563 [https://bugs.python.org/issue?@action=redirect&bpo=34563]: On Windows, fix multiprocessing.Connection for very large read: fix _winapi.PeekNamedPipe() and _winapi.ReadFile() for read larger than INT_MAX (usually `2**31-1`).

- bpo-34558 [https://bugs.python.org/issue?@action=redirect&bpo=34558]: Correct typo in Lib/ctypes/_aix.py

- bpo-34282 [https://bugs.python.org/issue?@action=redirect&bpo=34282]: Move `Enum._convert` to `EnumMeta._convert_` and fix enum members getting shadowed by parent attributes.

- bpo-22872 [https://bugs.python.org/issue?@action=redirect&bpo=22872]: When the queue is closed, **`ValueError`** is now raised by **`multiprocessing.Queue.put()`** and **`multiprocessing.Queue.get()`** instead of **`AssertionError`** and **`OSError`**, respectively. Patch by Zackery Spytz.

- bpo-34515 [https://bugs.python.org/issue?@action=redirect&bpo=34515]: Fix parsing non-ASCII identifiers in **`lib2to3.pgen2.tokenize`** (**PEP 3131** [https://peps.python.org/pep-3131/]).

- bpo-13312 [https://bugs.python.org/issue?@action=redirect&bpo=13312]: Avoids a possible integer underflow (undefined behavior) in the time module's year handling code when passed a very low negative year value.

- bpo-34472 [https://bugs.python.org/issue?@action=redirect&bpo=34472]: Improved compatibility for streamed files in **`zipfile`**. Previously an optional signature was not being written and certain ZIP applications were not supported. Patch by Silas Sewell.

- bpo-34454 [https://bugs.python.org/issue?@action=redirect&bpo=34454]: Fix the .fromisoformat() methods of datetime types crashing when given unicode with non-UTF-8-encodable code points. Specifically, datetime.fromisoformat() now accepts surrogate unicode code points used as the separator. Report and tests by Alexey Izbyshev, patch by Paul Ganssle.

- bpo-6700 [https://bugs.python.org/issue?@action=redirect&bpo=6700]: Fix inspect.getsourcelines for module level frames/tracebacks. Patch by Vladimir Matveev.

- bpo-34171 [https://bugs.python.org/issue?@action=redirect&bpo=34171]: Running the `trace` module no longer creates the `trace.cover` file.

- bpo-34441 [https://bugs.python.org/issue?@action=redirect&bpo=34441]: Fix crash when an `ABC`-derived class with invalid `__subclasses__` is passed as the second argument to `issubclass()`. Patch by Alexey Izbyshev.

- bpo-34427 [https://bugs.python.org/issue?@action=redirect&bpo=34427]: Fix infinite loop in `a.extend(a)` for `MutableSequence` subclasses.

- bpo-34412 [https://bugs.python.org/issue?@action=redirect&bpo=34412]: Make `signal.strsignal()` work on HP-UX. Patch by Michael Osipov.

- bpo-20849 [https://bugs.python.org/issue?@action=redirect&bpo=20849]: shutil.copytree now accepts a new `dirs_exist_ok` keyword argument. Patch by Josh Bronson.

- bpo-31715 [https://bugs.python.org/issue?@action=redirect&bpo=31715]: Associate `.mjs` file extension with `application/javascript` MIME Type.

- bpo-34384 [https://bugs.python.org/issue?@action=redirect&bpo=34384]: `os.readlink()` now accepts path-like and `bytes` objects on Windows.

- bpo-22602 [https://bugs.python.org/issue?@action=redirect&bpo=22602]: The UTF-7 decoder now raises `UnicodeDecodeError` for ill-formed sequences starting with “+” (as specified in RFC 2152). Patch by Zackery Spytz.

- bpo-2122 [https://bugs.python.org/issue?@action=redirect&bpo=2122]: The `mmap.flush()` method now returns `None` on success, raises an exception on error under all platforms.

- bpo-34341 [https://bugs.python.org/issue?@action=redirect&bpo=34341]: Appending to the ZIP archive with the ZIP64 extension no longer grows the size of extra fields of existing entries.

- bpo-34333 [https://bugs.python.org/issue?@action=redirect&bpo=34333]: Fix %-formatting in `pathlib.PurePath.with_suffix()` when formatting an error message.

- bpo-18540 [https://bugs.python.org/issue?@action=redirect&bpo=18540]: The `imaplib.IMAP4` and `imaplib.IMAP4_SSL` classes now resolve to the local host IP correctly when the default value of *host* parameter (`''`) is used.

- bpo-26502 [https://bugs.python.org/issue?@action=redirect&bpo=26502]: Implement `traceback.FrameSummary.__len__()` method to preserve compatibility with the old tuple API.

- bpo-34318 [https://bugs.python.org/issue?@action=redirect&bpo=34318]: `assertRaises()`, `assertRaisesRegex()`, `assertWarns()` and `assertWarnsRegex()` no longer success if the passed callable is None. They no longer ignore unknown keyword arguments in the context manager mode. A DeprecationWarning was raised in these cases since Python 3.5.

- bpo-9372 [https://bugs.python.org/issue?@action=redirect&bpo=9372]: Deprecate `__getitem__()` methods of `xml.dom.pulldom.DOMEventStream`, `wsgiref.util.FileWrapper` and `fileinput.FileInput`.

- bpo-33613 [https://bugs.python.org/issue?@action=redirect&bpo=33613]: Fix a race condition in `multiprocessing.semaphore_tracker` when the tracker receives SIGINT before it can register signal handlers for ignoring it.

- bpo-34248 [https://bugs.python.org/issue?@action=redirect&bpo=34248]: Report filename in the exception raised when the database file cannot be opened by `dbm.gnu.open()` and `dbm.ndbm.open()` due to OS-related error. Patch by Zsolt Cserna.

- bpo-33089 [https://bugs.python.org/issue?@action=redirect&bpo=33089]: Add math.dist() to compute the Euclidean distance between two points.

- bpo-34246 [https://bugs.python.org/issue?@action=redirect&bpo=34246]: `smtplib.SMTP.send_message()` no longer modifies the content of the

mail_options argument. Patch by Pablo S. Blum de Aguiar.

- bpo-31047 [https://bugs.python.org/issue?@action=redirect&bpo=31047]: Fix `ntpath.abspath` for invalid paths on windows. Patch by Franz Woellert.

- bpo-32321 [https://bugs.python.org/issue?@action=redirect&bpo=32321]: Add pure Python fallback for functools.reduce. Patch by Robert Wright.

- bpo-34270 [https://bugs.python.org/issue?@action=redirect&bpo=34270]: The default asyncio task class now always has a name which can be get or set using two new methods (**`get_name()`** and **`set_name()`**) and is visible in the **`repr()`** output. An initial name can also be set using the new `name` keyword argument to **`asyncio.create_task()`** or the **`create_task()`** method of the event loop. If no initial name is set, the default Task implementation generates a name like `Task-1` using a monotonic counter.

- bpo-34263 [https://bugs.python.org/issue?@action=redirect&bpo=34263]: asyncio's event loop will not pass timeouts longer than one day to epoll/select etc.

- bpo-34035 [https://bugs.python.org/issue?@action=redirect&bpo=34035]: Fix several AttributeError in zipfile seek() methods. Patch by Mickaël Schoentgen.

- bpo-32215 [https://bugs.python.org/issue?@action=redirect&bpo=32215]: Fix performance regression in **`sqlite3`** when a DML statement appeared in a different line than the rest of the SQL query.

- bpo-34075 [https://bugs.python.org/issue?@action=redirect&bpo=34075]: Deprecate passing non-ThreadPoolExecutor instances to **`AbstractEventLoop.set_default_executor()`**.

- bpo-34251 [https://bugs.python.org/issue?@action=redirect&bpo=34251]: Restore `msilib.Win64` to preserve backwards compatibility since it's already used by `distutils'` `bdist_msi` command.

- bpo-19891 [https://bugs.python.org/issue?@action=redirect&bpo=19891]: Ignore errors caused by missing / non-writable homedir while writing history during exit of an interactive session. Patch by Anthony Sottile.

- bpo-33089 [https://bugs.python.org/issue?@action=redirect&bpo=33089]: Enhanced math.hypot() to support more than two dimensions.

- bpo-34228 [https://bugs.python.org/issue?@action=redirect&bpo=34228]: tracemalloc: PYTHONTRACEMALLOC=0 environment variable and -X tracemalloc=0 command line option are now allowed to disable explicitly tracemalloc at startup.

- bpo-13041 [https://bugs.python.org/issue?@action=redirect&bpo=13041]: Use `shutil.get_terminal_size()` to calculate the terminal width correctly in the `argparse.HelpFormatter` class. Initial patch by Zbyszek Jędrzejewski-Szmek.

- bpo-34213 [https://bugs.python.org/issue?@action=redirect&bpo=34213]: Allow frozen dataclasses to have a field named “object”. Previously this conflicted with an internal use of “object”.

- bpo-34052 [https://bugs.python.org/issue?@action=redirect&bpo=34052]: `sqlite3.Connection.create_aggregate()`, `sqlite3.Connection.create_function()`, `sqlite3.Connection.set_authorizer()`, `sqlite3.Connection.set_progress_handler()` methods raises TypeError when unhashable objects are passed as callable. These methods now don’t pass such objects to SQLite API. Previous behavior could lead to segfaults. Patch by Sergey Fedoseev.

- bpo-34197 [https://bugs.python.org/issue?@action=redirect&bpo=34197]: Attributes *skipinitialspace*, *doublequote* and *strict* of the *dialect* attribute of the `csv` reader are now `bool` instances instead of integers 0 or 1.

- bpo-32788 [https://bugs.python.org/issue?@action=redirect&bpo=32788]: Errors other than `TypeError` raised in methods `__adapt__()` and `__conform__()` in the `sqlite3` module are now propagated to the user.

- bpo-21446 [https://bugs.python.org/issue?@action=redirect&bpo=21446]: The `reload` fixer now uses **`importlib.reload()`** instead of deprecated **`imp.reload()`**.

- bpo-940286 [https://bugs.python.org/issue?@action=redirect&bpo=940286]: pydoc's `Helper.showtopic()` method now prints the cross references of a topic correctly.

- bpo-34164 [https://bugs.python.org/issue?@action=redirect&bpo=34164]: **`base64.b32decode()`** could raise UnboundLocalError or OverflowError for incorrect padding. Now it always raises **`base64.Error`** in these cases.

- bpo-33729 [https://bugs.python.org/issue?@action=redirect&bpo=33729]: Fixed issues with arguments parsing in **`hashlib`**.

- bpo-34097 [https://bugs.python.org/issue?@action=redirect&bpo=34097]: ZipFile can zip files older than 1980-01-01 and newer than 2107-12-31 using a new `strict_timestamps` parameter at the cost of setting the timestamp to the limit.

- bpo-34108 [https://bugs.python.org/issue?@action=redirect&bpo=34108]: Remove extraneous CR in 2to3 refactor.

- bpo-34070 [https://bugs.python.org/issue?@action=redirect&bpo=34070]: Make sure to only check if the handle is a tty, when opening a file with `buffering=-1`.

- bpo-27494 [https://bugs.python.org/issue?@action=redirect&bpo=27494]: Reverted bpo-27494 [https://bugs.python.org/issue?@action=redirect&bpo=27494]. 2to3 rejects now a trailing comma in generator expressions.

- bpo-33967 [https://bugs.python.org/issue?@action=redirect&bpo=33967]: functools.singledispatch now raises TypeError instead of IndexError when no positional arguments are passed.

- bpo-34041 [https://bugs.python.org/issue?@action=redirect&bpo=34041]: Add the parameter *deterministic* to the

`sqlite3.Connection.create_function()` method. Patch by Sergey Fedoseev.

- bpo-34056 [https://bugs.python.org/issue?@action=redirect&bpo=34056]: Ensure the loader shim created by `imp.load_module` always returns bytes from its `get_data()` function. This fixes using `imp.load_module` with **PEP 552** [https://peps.python.org/pep-0552/] hash-based pycs.

- bpo-34054 [https://bugs.python.org/issue?@action=redirect&bpo=34054]: The multiprocessing module now uses the monotonic clock `time.monotonic()` instead of the system clock `time.time()` to implement timeout.

- bpo-34043 [https://bugs.python.org/issue?@action=redirect&bpo=34043]: Optimize tarfile uncompress performance about 15% when gzip is used.

- bpo-34044 [https://bugs.python.org/issue?@action=redirect&bpo=34044]: `subprocess.Popen` now copies the *startupinfo* argument to leave it unchanged: it will modify the copy, so that the same `STARTUPINFO` object can be used multiple times.

- bpo-34010 [https://bugs.python.org/issue?@action=redirect&bpo=34010]: Fixed a performance regression for reading streams with tarfile. The buffered read should use a list, instead of appending to a bytes object.

- bpo-34019 [https://bugs.python.org/issue?@action=redirect&bpo=34019]: webbrowser: Correct the arguments passed to Opera Browser when opening a new URL using the `webbrowser` module. Patch by Bumsik Kim.

- bpo-34003 [https://bugs.python.org/issue?@action=redirect&bpo=34003]: csv.DictReader now creates dicts instead of OrderedDicts. Patch by Michael Selik.

- bpo-33978 [https://bugs.python.org/issue?@action=redirect&bpo=33978]: Closed existing logging handlers before reconfiguration via fileConfig and dictConfig. Patch by Karthikeyan Singaravelan.

- bpo-14117 [https://bugs.python.org/issue?@action=redirect&bpo=14117]: Make minor tweaks to turtledemo. The ‘wikipedia’ example is now ‘rosette’, describing what it draws. The ‘penrose’ print output is reduced. The’1024’ output of ‘tree’ is eliminated.

- bpo-33974 [https://bugs.python.org/issue?@action=redirect&bpo=33974]: Fixed passing lists and tuples of strings containing special characters `"`, `\`, `{`, `}` and `\n` as options to `ttk` widgets.

- bpo-27500 [https://bugs.python.org/issue?@action=redirect&bpo=27500]: Fix getaddrinfo to resolve IPv6 addresses correctly.

- bpo-24567 [https://bugs.python.org/issue?@action=redirect&bpo=24567]: Improve random.choices() to handle subnormal input weights that could occasionally trigger an IndexError.

- bpo-33871 [https://bugs.python.org/issue?@action=redirect&bpo=33871]: Fixed integer overflow in `os.readv()`, `os.writev()`, `os.preadv()` and `os.pwritev()` and in `os.sendfile()` with *headers* or *trailers* arguments (on BSD-based OSes and macOS).

- bpo-25007 [https://bugs.python.org/issue?@action=redirect&bpo=25007]: Add `copy.copy()` and `copy.deepcopy()` support to zlib compressors and decompressors. Patch by Zackery Spytz.

- bpo-33929 [https://bugs.python.org/issue?@action=redirect&bpo=33929]: multiprocessing: Fix a race condition in Popen of multiprocessing.popen_spawn_win32. The child process now duplicates the read end of pipe instead of “stealing” it. Previously, the read end of pipe was “stolen” by the child process, but it leaked a handle if the child process had been terminated before it could steal the handle from the parent process.

- bpo-33899 [https://bugs.python.org/issue?@action=redirect&bpo=33899]: Tokenize module now implicitly emits a NEWLINE when provided with input that does not have a trailing new line. This behavior now matches what the C tokenizer does internally. Contributed by Ammar Askar.

- bpo-33897 [https://bugs.python.org/issue?@action=redirect&bpo=33897]: Added a ‘force’ keyword argument to logging.basicConfig().

- bpo-33695 [https://bugs.python.org/issue?@action=redirect&bpo=33695]: **`shutil.copytree()`** uses **`os.scandir()`** function and all copy functions depending from it use cached **`os.stat()`** values. The speedup for copying a directory with 8000 files is around +9% on Linux, +20% on Windows and + 30% on a Windows SMB share. Also the number of **`os.stat()`** syscalls is reduced by 38% making **`shutil.copytree()`** especially faster on network filesystems. (Contributed by Giampaolo Rodola’ in bpo-33695 [https://bugs.python.org/issue?@action=redirect&bpo=33695].)

- bpo-33916 [https://bugs.python.org/issue?@action=redirect&bpo=33916]: bz2 and lzma: When Decompressor.__init__() is called twice, free the old lock to not leak memory.

- bpo-32568 [https://bugs.python.org/issue?@action=redirect&bpo=32568]: Make select.epoll() and its documentation consistent regarding *sizehint* and *flags*.

- bpo-33833 [https://bugs.python.org/issue?@action=redirect&bpo=33833]: Fixed bug in asyncio where ProactorSocketTransport logs AssertionError if force closed during write.

- bpo-33663 [https://bugs.python.org/issue?@action=redirect&bpo=33663]: Convert content length to string before putting to header.

- bpo-33721 [https://bugs.python.org/issue?@action=redirect&bpo=33721]: **`os.path`** functions that return a boolean result like **`exists()`**, **`lexists()`**, **`isdir()`**, **`isfile()`**, **`islink()`**, and **`ismount()`**, and **`pathlib.Path`** methods that return a boolean result like **`exists()`**, **`is_dir()`**, **`is_file()`**, **`is_mount()`**, **`is_symlink()`**, **`is_block_device()`**, **`is_char_device()`**, **`is_fifo()`**, **`is_socket()`** now return `False` instead of raising **`ValueError`** or its subclasses **`UnicodeEncodeError`** and **`UnicodeDecodeError`** for paths that contain characters or bytes unrepresentable at the OS level.

- bpo-26544 [https://bugs.python.org/issue?@action=redirect&bpo=26544]: Fixed implementation of `platform.libc_ver()`. It almost always returned version '2.9' for glibc.

- bpo-33843 [https://bugs.python.org/issue?@action=redirect&bpo=33843]: Remove deprecated `cgi.escape`, `cgi.parse_qs` and `cgi.parse_qsl`.

- bpo-33842 [https://bugs.python.org/issue?@action=redirect&bpo=33842]: Remove `tarfile.filemode` which is deprecated since Python 3.3.

- bpo-30167 [https://bugs.python.org/issue?@action=redirect&bpo=30167]: Prevent site.main() exception if PYTHONSTARTUP is set. Patch by Steve Weber.

- bpo-33805 [https://bugs.python.org/issue?@action=redirect&bpo=33805]: Improve error message of dataclasses.replace() when an InitVar is not specified

- bpo-33687 [https://bugs.python.org/issue?@action=redirect&bpo=33687]: Fix the call to `os.chmod()` for `uu.decode()` if a mode is given or decoded. Patch by Timo Furrer.

- bpo-33812 [https://bugs.python.org/issue?@action=redirect&bpo=33812]: Datetime instance d with non-None tzinfo, but with d.tzinfo.utcoffset(d) returning None is now treated as naive by the astimezone() method.

- bpo-32108 [https://bugs.python.org/issue?@action=redirect&bpo=32108]: In configparser, don't clear section when it is assigned to itself.

- bpo-27397 [https://bugs.python.org/issue?@action=redirect&bpo=27397]: Make email module properly handle invalid-length base64 strings.

- bpo-33578 [https://bugs.python.org/issue?@action=redirect&bpo=33578]: Implement multibyte encoder/decoder state methods

- bpo-30805 [https://bugs.python.org/issue?@action=redirect&bpo=30805]: Avoid race condition with debug logging

- bpo-33476 [https://bugs.python.org/issue?@action=redirect&bpo=33476]: Fix _header_value_parser.py when address group is missing final ';'.

Contributed by Enrique Perez-Terron

- bpo-33694 [https://bugs.python.org/issue?@action=redirect&bpo=33694]: asyncio: Fix a race condition causing data loss on pause_reading()/resume_reading() when using the ProactorEventLoop.

- bpo-32493 [https://bugs.python.org/issue?@action=redirect&bpo=32493]: Correct test for `uuid_enc_be` availability in `configure.ac`. Patch by Michael Felt.

- bpo-33792 [https://bugs.python.org/issue?@action=redirect&bpo=33792]: Add asyncio.WindowsSelectorEventLoopPolicy and asyncio.WindowsProactorEventLoopPolicy.

- bpo-33274 [https://bugs.python.org/issue?@action=redirect&bpo=33274]: W3C DOM Level 1 specifies return value of Element.removeAttributeNode() as “The Attr node that was removed.” xml.dom.minidom now complies with this requirement.

- bpo-33778 [https://bugs.python.org/issue?@action=redirect&bpo=33778]: Update `unicodedata`’s database to Unicode version 11.0.0.

- bpo-33165 [https://bugs.python.org/issue?@action=redirect&bpo=33165]: Added a stacklevel parameter to logging calls to allow use of wrapper/helper functions for logging APIs.

- bpo-33770 [https://bugs.python.org/issue?@action=redirect&bpo=33770]: improve base64 exception message for encoded inputs of invalid length

- bpo-33769 [https://bugs.python.org/issue?@action=redirect&bpo=33769]: asyncio/start_tls: Fix error message; cancel callbacks in case of an unhandled error; mark SSLTransport as closed if it is aborted.

- bpo-33767 [https://bugs.python.org/issue?@action=redirect&bpo=33767]: The concatenation (`+`) and repetition (`*`) sequence operations now raise **`TypeError`** instead of **`SystemError`** when performed on **`mmap.mmap`** objects. Patch by Zackery Spytz.

- bpo-33734 [https://bugs.python.org/issue?@action=redirect&bpo=33734]: asyncio/ssl: Fix AttributeError, increase default handshake timeout

- bpo-31014 [https://bugs.python.org/issue?@action=redirect&bpo=31014]: Fixed creating a controller for `webbrowser` when a user specifies a path to an entry in the BROWSER environment variable. Based on patch by John Still.

- bpo-2504 [https://bugs.python.org/issue?@action=redirect&bpo=2504]: Add gettext.pgettext() and variants.

- bpo-33197 [https://bugs.python.org/issue?@action=redirect&bpo=33197]: Add description property for _ParameterKind

- bpo-32751 [https://bugs.python.org/issue?@action=redirect&bpo=32751]: When cancelling the task due to a timeout, `asyncio.wait_for()` will now wait until the cancellation is complete.

- bpo-32684 [https://bugs.python.org/issue?@action=redirect&bpo=32684]: Fix gather to propagate cancellation of itself even with return_exceptions.

- bpo-33654 [https://bugs.python.org/issue?@action=redirect&bpo=33654]: Support protocol type switching in SSLTransport.set_protocol().

- bpo-33674 [https://bugs.python.org/issue?@action=redirect&bpo=33674]: Pause the transport as early as possible to further reduce the risk of data_received() being called before connection_made().

- bpo-33671 [https://bugs.python.org/issue?@action=redirect&bpo=33671]: `shutil.copyfile()`, `shutil.copy()`, `shutil.copy2()`, `shutil.copytree()` and `shutil.move()` use platform-specific fast-copy syscalls on Linux and macOS in order to copy the file more efficiently. On Windows **`shutil.copyfile()`** uses a bigger default buffer size (1 MiB instead of 16 KiB) and a `memoryview()`-based variant of `shutil.copyfileobj()` is used. The speedup for copying a 512MiB file is about +26% on Linux, +50% on macOS and +40% on Windows. Also, much less CPU cycles are consumed. (Contributed by

Giampaolo Rodola' in bpo-25427 [https://bugs.python.org/issue?@action=redirect&bpo=25427].)

- bpo-33674 [https://bugs.python.org/issue?@action=redirect&bpo=33674]: Fix a race condition in SSLProtocol.connection_made() of asyncio.sslproto: start immediately the handshake instead of using call_soon(). Previously, data_received() could be called before the handshake started, causing the handshake to hang or fail.

- bpo-31647 [https://bugs.python.org/issue?@action=redirect&bpo=31647]: Fixed bug where calling write_eof() on a _SelectorSocketTransport after it's already closed raises AttributeError.

- bpo-32610 [https://bugs.python.org/issue?@action=redirect&bpo=32610]: Make asyncio.all_tasks() return only pending tasks.

- bpo-32410 [https://bugs.python.org/issue?@action=redirect&bpo=32410]: Avoid blocking on file IO in sendfile fallback code

- bpo-33469 [https://bugs.python.org/issue?@action=redirect&bpo=33469]: Fix RuntimeError after closing loop that used run_in_executor

- bpo-33672 [https://bugs.python.org/issue?@action=redirect&bpo=33672]: Fix Task.__repr__ crash with Cython's bogus coroutines

- bpo-33654 [https://bugs.python.org/issue?@action=redirect&bpo=33654]: Fix transport.set_protocol() to support switching between asyncio.Protocol and asyncio.BufferedProtocol. Fix loop.start_tls() to work with asyncio.BufferedProtocols.

- bpo-33652 [https://bugs.python.org/issue?@action=redirect&bpo=33652]: Pickles of type variables and subscripted generics are now future-proof and compatible with older Python versions.

- bpo-32493 [https://bugs.python.org/issue?@action=redirect&bpo=32493]: Fixed `uuid.uuid1()` on FreeBSD.

- bpo-33238 [https://bugs.python.org/issue?@action=redirect&bpo=33238]: Add `InvalidStateError` to **`concurrent.futures`**. `Future.set_result` and `Future.set_exception` now raise `InvalidStateError` if the futures are not pending or running. Patch by Jason Haydaman.

- bpo-33618 [https://bugs.python.org/issue?@action=redirect&bpo=33618]: Finalize and document preliminary and experimental TLS 1.3 support with OpenSSL 1.1.1

- bpo-33625 [https://bugs.python.org/issue?@action=redirect&bpo=33625]: Release GIL on `grp.getgrnam`, `grp.getgrgid`, `pwd.getpwnam` and `pwd.getpwuid` if reentrant variants of these functions are available. Patch by William Grzybowski.

- bpo-33623 [https://bugs.python.org/issue?@action=redirect&bpo=33623]: Fix possible SIGSGV when asyncio.Future is created in __del__

- bpo-11874 [https://bugs.python.org/issue?@action=redirect&bpo=11874]: Use a better regex when breaking usage into wrappable parts. Avoids bogus assertion errors from custom metavar strings.

- bpo-30877 [https://bugs.python.org/issue?@action=redirect&bpo=30877]: Fixed a bug in the Python implementation of the JSON decoder that prevented the cache of parsed strings from clearing after finishing the decoding. Based on patch by c-fos.

- bpo-33604 [https://bugs.python.org/issue?@action=redirect&bpo=33604]: Remove HMAC default to md5 marked for removal in 3.8 (removal originally planned in 3.6, bump to 3.8 in PR 7062).

- bpo-33582 [https://bugs.python.org/issue?@action=redirect&bpo=33582]: Emit a deprecation warning for inspect.formatargspec

- bpo-21145 [https://bugs.python.org/issue?@action=redirect&bpo=21145]: Add `functools.cached_property` decorator, for computed properties cached for the life of the instance.

- bpo-33570 [https://bugs.python.org/issue?@action=redirect&bpo=33570]: Change TLS 1.3 cipher suite settings for compatibility with OpenSSL 1.1.1-pre6 and newer. OpenSSL 1.1.1 will have TLS 1.3 ciphers enabled by default.

- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Do not simplify arguments to `typing.Union`. Now `Union[Manager, Employee]` is not simplified to `Employee` at runtime. Such simplification previously caused several bugs and limited possibilities for introspection.

- bpo-12486 [https://bugs.python.org/issue?@action=redirect&bpo=12486]: **`tokenize.generate_tokens()`** is now documented as a public API to tokenize unicode strings. It was previously present but undocumented.

- bpo-33540 [https://bugs.python.org/issue?@action=redirect&bpo=33540]: Add a new `block_on_close` class attribute to `ForkingMixIn` and `ThreadingMixIn` classes of **`socketserver`**.

- bpo-33548 [https://bugs.python.org/issue?@action=redirect&bpo=33548]: tempfile._candidate_tempdir_list should consider common TEMP locations

- bpo-33109 [https://bugs.python.org/issue?@action=redirect&bpo=33109]: argparse subparsers are once again not required by default, reverting the change in behavior introduced by bpo-26510 [https://bugs.python.org/issue?@action=redirect&bpo=26510] in 3.7.0a2.

- bpo-33541 [https://bugs.python.org/issue?@action=redirect&bpo=33541]: Remove unused private method `_strptime.LocaleTime.__pad` (a.k.a. `_LocaleTime__pad`).

- bpo-33536 [https://bugs.python.org/issue?@action=redirect&bpo=33536]: dataclasses.make_dataclass now checks for invalid field names and duplicate fields. Also, added a check for invalid field specifications.

- bpo-33542 [https://bugs.python.org/issue?@action=redirect&bpo=33542]: Prevent `uuid.get_node` from using a DUID instead of a MAC on Windows.

Patch by Zvi Effron

- bpo-26819 [https://bugs.python.org/issue?@action=redirect&bpo=26819]: Fix race condition with `ReadTransport.resume_reading` in Windows proactor event loop.

- Fix failure in `typing.get_type_hints()` when ClassVar was provided as a string forward reference.

- bpo-33516 [https://bugs.python.org/issue?@action=redirect&bpo=33516]: **`unittest.mock.MagicMock`** now supports the `__round__` magic method.

- bpo-28612 [https://bugs.python.org/issue?@action=redirect&bpo=28612]: Added support for Site Maps to urllib's `RobotFileParser` as **`RobotFileParser.site_maps()`**. Patch by Lady Red, based on patch by Peter Wirtz.

- bpo-28167 [https://bugs.python.org/issue?@action=redirect&bpo=28167]: Remove platform.linux_distribution, which was deprecated since 3.5.

- bpo-33504 [https://bugs.python.org/issue?@action=redirect&bpo=33504]: Switch the default dictionary implementation for **`configparser`** from **`collections.OrderedDict`** to the standard **`dict`** type.

- bpo-33505 [https://bugs.python.org/issue?@action=redirect&bpo=33505]: Optimize asyncio.ensure_future() by reordering if checks: 1.17x faster.

- bpo-33497 [https://bugs.python.org/issue?@action=redirect&bpo=33497]: Add errors param to cgi.parse_multipart and make an encoding in FieldStorage use the given errors (needed for Twisted). Patch by Amber Brown.

- bpo-29235 [https://bugs.python.org/issue?@action=redirect&bpo=29235]: The **`cProfile.Profile`** class can now be used as a context manager. Patch by Scott Sanderson.

- bpo-33495 [https://bugs.python.org/issue?@action=redirect&bpo=33495]: Change dataclasses.Fields repr to use the repr of each of its members, instead

of str. This makes it more clear what each field actually represents. This is especially true for the 'type' member.

- bpo-26103 [https://bugs.python.org/issue?@action=redirect&bpo=26103]: Correct `inspect.isdatadescriptor` to look for `__set__` or `__delete__`. Patch by Aaron Hall.

- bpo-29209 [https://bugs.python.org/issue?@action=redirect&bpo=29209]: Removed the `doctype()` method and the *html* parameter of the constructor of **`XMLParser`**. The `doctype()` method defined in a subclass will no longer be called. Deprecated methods `getchildren()` and `getiterator()` in the **`ElementTree`** module emit now a **`DeprecationWarning`** instead of **`PendingDeprecationWarning`**.

- bpo-33453 [https://bugs.python.org/issue?@action=redirect&bpo=33453]: Fix dataclasses to work if using literal string type annotations or if using PEP 563 "Postponed Evaluation of Annotations". Only specific string prefixes are detected for both ClassVar ("ClassVar" and "typing.ClassVar") and InitVar ("InitVar" and "dataclasses.InitVar").

- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Minor fixes in typing module: add annotations to `NamedTuple.__new__`, pass `*args` and `**kwds` in `Generic.__new__`. Original PRs by Paulius Šarka and Chad Dombrova.

- bpo-33365 [https://bugs.python.org/issue?@action=redirect&bpo=33365]: Print the header values besides the header keys instead just the header keys if *debuglevel* is set to >0 in **`http.client`**. Patch by Marco Strigl.

- bpo-20087 [https://bugs.python.org/issue?@action=redirect&bpo=20087]: Updated alias mapping with glibc 2.27 supported locales.

- bpo-33422 [https://bugs.python.org/issue?@action=redirect&bpo=33422]: Fix trailing quotation marks getting deleted when looking up byte/string literals on pydoc. Patch by Andrés Delfino.

- bpo-28167 [https://bugs.python.org/issue?@action=redirect&bpo=28167]: The function `platform.linux_distribution` and `platform.dist` now

trigger a `DeprecationWarning` and have been marked for removal in Python 3.8

- bpo-33281 [https://bugs.python.org/issue?@action=redirect&bpo=33281]: Fix ctypes.util.find_library regression on macOS.

- bpo-33311 [https://bugs.python.org/issue?@action=redirect&bpo=33311]: Text and html output generated by cgitb does not display parentheses if the current call is done directly in the module. Patch by Stéphane Blondon.

- bpo-27300 [https://bugs.python.org/issue?@action=redirect&bpo=27300]: The file classes in *tempfile* now accept an *errors* parameter that complements the already existing *encoding*. Patch by Stephan Hohe.

- bpo-32933 [https://bugs.python.org/issue?@action=redirect&bpo=32933]: **`unittest.mock.mock_open()`** now supports iteration over the file contents. Patch by Tony Flury.

- bpo-33217 [https://bugs.python.org/issue?@action=redirect&bpo=33217]: Raise **`TypeError`** when looking up non-Enum objects in Enum classes and Enum members.

- bpo-33197 [https://bugs.python.org/issue?@action=redirect&bpo=33197]: Update error message when constructing invalid inspect.Parameters Patch by Donghee Na.

- bpo-33383 [https://bugs.python.org/issue?@action=redirect&bpo=33383]: Fixed crash in the get() method of the **`dbm.ndbm`** database object when it is called with a single argument.

- bpo-33375 [https://bugs.python.org/issue?@action=redirect&bpo=33375]: The warnings module now finds the Python file associated with a warning from the code object, rather than the frame's global namespace. This is consistent with how tracebacks and pdb find filenames, and should work better for dynamically executed code.

- bpo-33336 [https://bugs.python.org/issue?@action=redirect&bpo=33336]: `imaplib` now allows `MOVE` command in `IMAP4.uid()` (RFC 6851: IMAP MOVE

Extension) and potentially as a name of supported method of `IMAP4` object.

- bpo-32455 [https://bugs.python.org/issue?@action=redirect&bpo=32455]: Added *jump* parameter to **`dis.stack_effect()`**.

- bpo-27485 [https://bugs.python.org/issue?@action=redirect&bpo=27485]: Rename and deprecate undocumented functions in **`urllib.parse()`**.

- bpo-33332 [https://bugs.python.org/issue?@action=redirect&bpo=33332]: Add `signal.valid_signals()` to expose the POSIX sigfillset() functionality.

- bpo-33251 [https://bugs.python.org/issue?@action=redirect&bpo=33251]: `ConfigParser.items()` was fixed so that key-value pairs passed in via **`vars()`** are not included in the resulting output.

- bpo-33329 [https://bugs.python.org/issue?@action=redirect&bpo=33329]: Fix multiprocessing regression on newer glibcs

- bpo-33334 [https://bugs.python.org/issue?@action=redirect&bpo=33334]: **`dis.stack_effect()`** now supports all defined opcodes including NOP and EXTENDED_ARG.

- bpo-991266 [https://bugs.python.org/issue?@action=redirect&bpo=991266]: Fix quoting of the `Comment` attribute of **`http.cookies.SimpleCookie`**.

- bpo-33131 [https://bugs.python.org/issue?@action=redirect&bpo=33131]: Upgrade bundled version of pip to 10.0.1.

- bpo-33308 [https://bugs.python.org/issue?@action=redirect&bpo=33308]: Fixed a crash in the **`parser`** module when converting an ST object to a tree of tuples or lists with `line_info=False` and `col_info=True`.

- bpo-23403 [https://bugs.python.org/issue?@action=redirect&bpo=23403]: lib2to3 now uses pickle protocol 4 for pre-computed grammars.

- bpo-33266 [https://bugs.python.org/issue?@action=redirect&bpo=33266]: lib2to3 now recognizes `rf'...'` strings.

- bpo-11594 [https://bugs.python.org/issue?@action=redirect&bpo=11594]: Ensure line-endings are respected when using lib2to3.

- bpo-33254 [https://bugs.python.org/issue?@action=redirect&bpo=33254]: Have **`importlib.resources.contents()`** and **`importlib.abc.ResourceReader.contents()`** return an iterable instead of an iterator.

- bpo-33265 [https://bugs.python.org/issue?@action=redirect&bpo=33265]: `contextlib.ExitStack` and `contextlib.AsyncExitStack` now use a method instead of a wrapper function for exit callbacks.

- bpo-33263 [https://bugs.python.org/issue?@action=redirect&bpo=33263]: Fix FD leak in `_SelectorSocketTransport` Patch by Vlad Starostin.

- bpo-33256 [https://bugs.python.org/issue?@action=redirect&bpo=33256]: Fix display of `<module>` call in the html produced by `cgitb.html()`. Patch by Stéphane Blondon.

- bpo-33144 [https://bugs.python.org/issue?@action=redirect&bpo=33144]: `random.Random()` and its subclassing mechanism got optimized to check only once at class/subclass instantiation time whether its `getrandbits()` method can be relied on by other methods, including `randrange()`, for the generation of arbitrarily large random integers. Patch by Wolfgang Maier.

- bpo-33185 [https://bugs.python.org/issue?@action=redirect&bpo=33185]: Fixed regression when running pydoc with the **`-m`** switch. (The regression was introduced in 3.7.0b3 by the resolution of bpo-33053 [https://bugs.python.org/issue?@action=redirect&bpo=33053])

  This fix also changed pydoc to add `os.getcwd()` to **`sys.path`** when necessary, rather than adding `"."`.

- bpo-29613 [https://bugs.python.org/issue?@action=redirect&bpo=29613]: Added support for the `SameSite` cookie flag to the `http.cookies` module.

- bpo-33169 [https://bugs.python.org/issue?@action=redirect&bpo=33169]: Delete entries of `None` in **`sys.path_importer_cache`** when **`importlib.machinery.invalidate_caches()`** is called.

- bpo-33203 [https://bugs.python.org/issue?@action=redirect&bpo=33203]: `random.Random.choice()` now raises `IndexError` for empty sequences consistently even when called from subclasses without a `getrandbits()` implementation.

- bpo-33224 [https://bugs.python.org/issue?@action=redirect&bpo=33224]: Update difflib.mdiff() for **PEP 479** [https://peps.python.org/pep-0479/]. Convert an uncaught StopIteration in a generator into a return-statement.

- bpo-33209 [https://bugs.python.org/issue?@action=redirect&bpo=33209]: End framing at the end of C implementation of **`pickle.Pickler.dump()`**.

- bpo-32861 [https://bugs.python.org/issue?@action=redirect&bpo=32861]: The urllib.robotparser's `__str__` representation now includes wildcard entries and the "Crawl-delay" and "Request-rate" fields. Also removes extra newlines that were being appended to the end of the string. Patch by Michael Lazar.

- bpo-23403 [https://bugs.python.org/issue?@action=redirect&bpo=23403]: `DEFAULT_PROTOCOL` in **`pickle`** was bumped to 4. Protocol 4 is described in **PEP 3154** [https://peps.python.org/pep-3154/] and available since Python 3.4. It offers better performance and smaller size compared to protocol 3 introduced in Python 3.0.

- bpo-20104 [https://bugs.python.org/issue?@action=redirect&bpo=20104]: Improved error handling and fixed a reference leak in **`os.posix_spawn()`**.

- bpo-33106 [https://bugs.python.org/issue?@action=redirect&bpo=33106]: Deleting a key from a read-only dbm database raises module specific error instead of KeyError.

- bpo-33175 [https://bugs.python.org/issue?@action=redirect&bpo=33175]: In dataclasses, Field.__set_name__ now looks up the __set_name__ special method on the class, not the instance, of the default value.

- bpo-32380 [https://bugs.python.org/issue?@action=redirect&bpo=32380]: Create functools.singledispatchmethod to support generic single dispatch on descriptors and methods.

- bpo-33141 [https://bugs.python.org/issue?@action=redirect&bpo=33141]: Have Field objects pass through __set_name__ to their default values, if they have their own __set_name__.

- bpo-33096 [https://bugs.python.org/issue?@action=redirect&bpo=33096]: Allow ttk.Treeview.insert to insert iid that has a false boolean value. Note iid=0 and iid=False would be same. Patch by Garvit Khatri.

- bpo-32873 [https://bugs.python.org/issue?@action=redirect&bpo=32873]: Treat type variables and special typing forms as immutable by copy and pickle. This fixes several minor issues and inconsistencies, and improves backwards compatibility with Python 3.6.

- bpo-33134 [https://bugs.python.org/issue?@action=redirect&bpo=33134]: When computing dataclass's __hash__, use the lookup table to contain the function which returns the __hash__ value. This is an improvement over looking up a string, and then testing that string to see what to do.

- bpo-33127 [https://bugs.python.org/issue?@action=redirect&bpo=33127]: The ssl module now compiles with LibreSSL 2.7.1.

- bpo-32505 [https://bugs.python.org/issue?@action=redirect&bpo=32505]: Raise TypeError if a member variable of a dataclass is of type Field, but doesn't have a type annotation.

- bpo-33078 [https://bugs.python.org/issue?@action=redirect&bpo=33078]: Fix the failure on OSX caused by the tests relying on sem_getvalue

- bpo-33116 [https://bugs.python.org/issue?@action=redirect&bpo=33116]: Add 'Field' to dataclasses.__all__.

- bpo-32896 [https://bugs.python.org/issue?@action=redirect&bpo=32896]: Fix an error where subclassing a dataclass with a field that uses a default_factory would generate an incorrect class.

- bpo-33100 [https://bugs.python.org/issue?@action=redirect&bpo=33100]: Dataclasses: If a field has a default value that's a MemberDescriptorType, then it's from that field being in __slots__, not an actual default value.

- bpo-32953 [https://bugs.python.org/issue?@action=redirect&bpo=32953]: If a non-dataclass inherits from a frozen dataclass, allow attributes to be added to the derived class. Only attributes from the frozen dataclass cannot be assigned to. Require all dataclasses in a hierarchy to be either all frozen or all non-frozen.

- bpo-33097 [https://bugs.python.org/issue?@action=redirect&bpo=33097]: Raise RuntimeError when `executor.submit` is called during interpreter shutdown.

- bpo-32968 [https://bugs.python.org/issue?@action=redirect&bpo=32968]: Modulo and floor division involving Fraction and float should return float.

- bpo-33061 [https://bugs.python.org/issue?@action=redirect&bpo=33061]: Add missing `NoReturn` to `__all__` in typing.py

- bpo-33078 [https://bugs.python.org/issue?@action=redirect&bpo=33078]: Fix the size handling in multiprocessing.Queue when a pickling error occurs.

- bpo-33064 [https://bugs.python.org/issue?@action=redirect&bpo=33064]: lib2to3 now properly supports trailing commas after `*args` and `**kwargs` in function signatures.

- bpo-33056 [https://bugs.python.org/issue?@action=redirect&bpo=33056]: FIX properly close leaking fds in concurrent.futures.ProcessPoolExecutor.

- bpo-33021 [https://bugs.python.org/issue?@action=redirect&bpo=33021]: Release the GIL during fstat() calls, avoiding hang of all threads when calling mmap.mmap(), os.urandom(), and random.seed(). Patch by Nir Soffer.

- bpo-31804 [https://bugs.python.org/issue?@action=redirect&bpo=31804]: Avoid failing in multiprocessing.Process if the standard streams are closed or None at exit.

- bpo-33034 [https://bugs.python.org/issue?@action=redirect&bpo=33034]: Providing an explicit error message when casting the port property to anything that is not an integer value using `urlparse()` and `urlsplit()`. Patch by Matt Eaton.

- bpo-30249 [https://bugs.python.org/issue?@action=redirect&bpo=30249]: Improve struct.unpack_from() exception messages for problems with the buffer size and offset.

- bpo-33037 [https://bugs.python.org/issue?@action=redirect&bpo=33037]: Skip sending/receiving data after SSL transport closing.

- bpo-27683 [https://bugs.python.org/issue?@action=redirect&bpo=27683]: Fix a regression in **`ipaddress`** that result of **`hosts()`** is empty when the network is constructed by a tuple containing an integer mask and only 1 bit left for addresses.

- bpo-22674 [https://bugs.python.org/issue?@action=redirect&bpo=22674]: Add the strsignal() function in the signal module that returns the system description of the given signal, as returned by strsignal(3).

- bpo-32999 [https://bugs.python.org/issue?@action=redirect&bpo=32999]: Fix C implementation of `ABC.__subclasscheck__(cls, subclass)` crashed when `subclass` is not a type object.

- bpo-33009 [https://bugs.python.org/issue?@action=redirect&bpo=33009]: Fix inspect.signature() for single-parameter partialmethods.

- bpo-32969 [https://bugs.python.org/issue?@action=redirect&bpo=32969]: Expose several missing constants in zlib and fix corresponding documentation.

- bpo-32056 [https://bugs.python.org/issue?@action=redirect&bpo=32056]: Improved exceptions raised for invalid number of channels and sample width when read an audio file in modules **`aifc`**, **`wave`** and **`sunau`**.

- bpo-32970 [https://bugs.python.org/issue?@action=redirect&bpo=32970]: Improved disassembly of the MAKE_FUNCTION instruction.

- bpo-32844 [https://bugs.python.org/issue?@action=redirect&bpo=32844]: Fix wrong redirection of a low descriptor (0 or 1) to stderr in subprocess if another low descriptor is closed.

- bpo-32960 [https://bugs.python.org/issue?@action=redirect&bpo=32960]: For dataclasses, disallow inheriting frozen from non-frozen classes, and also disallow inheriting non-frozen from frozen classes. This restriction will be relaxed at a future date.

- bpo-32713 [https://bugs.python.org/issue?@action=redirect&bpo=32713]: Fixed tarfile.itn handling of out-of-bounds float values. Patch by Joffrey Fuhrer.

- bpo-32257 [https://bugs.python.org/issue?@action=redirect&bpo=32257]: The ssl module now contains OP_NO_RENEGOTIATION constant, available with OpenSSL 1.1.0h or 1.1.1.

- bpo-32951 [https://bugs.python.org/issue?@action=redirect&bpo=32951]: Direct instantiation of SSLSocket and SSLObject objects is now prohibited. The constructors were never documented, tested, or designed as public constructors. Users were suppose to use ssl.wrap_socket() or SSLContext.

- bpo-32929 [https://bugs.python.org/issue?@action=redirect&bpo=32929]: Remove the tri-state parameter “hash”, and add the boolean “unsafe_hash”. If unsafe_hash is True, add a __hash__ function, but if a __hash__ exists, raise TypeError. If unsafe_hash is False, add a __hash__ based on the values of eq= and frozen=. The unsafe_hash=False behavior is the same as the old hash=None behavior. unsafe_hash=False is the default, just as hash=None used to be.

- bpo-32947 [https://bugs.python.org/issue?@action=redirect&bpo=32947]: Add OP_ENABLE_MIDDLEBOX_COMPAT and test workaround for TLSv1.3 for future compatibility with OpenSSL 1.1.1.

- bpo-32146 [https://bugs.python.org/issue?@action=redirect&bpo=32146]: Document the interaction between frozen executables and the spawn and forkserver start methods in multiprocessing.

- bpo-30622 [https://bugs.python.org/issue?@action=redirect&bpo=30622]: The ssl module now detects missing NPN support in LibreSSL.

- bpo-32922 [https://bugs.python.org/issue?@action=redirect&bpo=32922]: dbm.open() now encodes filename with the filesystem encoding rather than default encoding.

- bpo-32759 [https://bugs.python.org/issue?@action=redirect&bpo=32759]: Free unused arenas in multiprocessing.heap.

- bpo-32859 [https://bugs.python.org/issue?@action=redirect&bpo=32859]: In `os.dup2`, don't check every call whether the `dup3` syscall exists or not.

- bpo-32556 [https://bugs.python.org/issue?@action=redirect&bpo=32556]: nt._getfinalpathname, nt._getvolumepathname and nt._getdiskusage now correctly convert from bytes.

- bpo-21060 [https://bugs.python.org/issue?@action=redirect&bpo=21060]: Rewrite confusing message from setup.py upload from "No dist file created in earlier command" to the more helpful "Must create and upload files in one command".

- bpo-32857 [https://bugs.python.org/issue?@action=redirect&bpo=32857]: In **`tkinter`**, `after_cancel(None)` now raises a **`ValueError`** instead of canceling the first scheduled function. Patch by Cheryl Sabella.

- bpo-32852 [https://bugs.python.org/issue?@action=redirect&bpo=32852]: Make sure sys.argv remains as a list when running trace.

- bpo-31333 [https://bugs.python.org/issue?@action=redirect&bpo=31333]: `_abc` module is added. It is a speedup module with C implementations for various functions and methods in `abc`. Creating an ABC subclass and calling `isinstance` or `issubclass` with an ABC subclass are up to 1.5x faster. In addition, this makes Python start-up up to 10% faster.

  Note that the new implementation hides internal registry and caches, previously accessible via private attributes `_abc_registry`, `_abc_cache`, and `_abc_negative_cache`. There are three debugging

helper methods that can be used instead `_dump_registry`, `_abc_registry_clear`, and `_abc_caches_clear`.

- bpo-32841 [https://bugs.python.org/issue?@action=redirect&bpo=32841]: Fixed `asyncio.Condition` issue which silently ignored cancellation after notifying and cancelling a conditional lock. Patch by Bar Harel.

- bpo-32819 [https://bugs.python.org/issue?@action=redirect&bpo=32819]: ssl.match_hostname() has been simplified and no longer depends on re and ipaddress module for wildcard and IP addresses. Error reporting for invalid wildcards has been improved.

- bpo-19675 [https://bugs.python.org/issue?@action=redirect&bpo=19675]: `multiprocessing.Pool` no longer leaks processes if its initialization fails.

- bpo-32394 [https://bugs.python.org/issue?@action=redirect&bpo=32394]: socket: Remove TCP_FASTOPEN,TCP_KEEPCNT,TCP_KEEPIDLE,TCP_KEEPINTVL flags on older version Windows during run-time.

- bpo-31787 [https://bugs.python.org/issue?@action=redirect&bpo=31787]: Fixed refleaks of `__init__()` methods in various modules. (Contributed by Oren Milman)

- bpo-30157 [https://bugs.python.org/issue?@action=redirect&bpo=30157]: Fixed guessing quote and delimiter in csv.Sniffer.sniff() when only the last field is quoted. Patch by Jake Davis.

- bpo-30688 [https://bugs.python.org/issue?@action=redirect&bpo=30688]: Added support of `\N{name}` escapes in regular expressions. Based on patch by Jonathan Eunice.

- bpo-32792 [https://bugs.python.org/issue?@action=redirect&bpo=32792]: collections.ChainMap() preserves the order of the underlying mappings.

- bpo-32775 [https://bugs.python.org/issue?@action=redirect&bpo=32775]: **`fnmatch.translate()`** no longer produces patterns which contain set operations. Sets starting with '[' or containing '–', '&&', '~~' or '||' will be interpreted differently in regular expressions in future versions. Currently they emit warnings. fnmatch.translate() now avoids producing patterns containing such sets by accident.

- bpo-32622 [https://bugs.python.org/issue?@action=redirect&bpo=32622]: Implement native fast sendfile for Windows proactor event loop.

- bpo-32777 [https://bugs.python.org/issue?@action=redirect&bpo=32777]: Fix a rare but potential pre-exec child process deadlock in subprocess on POSIX systems when marking file descriptors inheritable on exec in the child process. This bug appears to have been introduced in 3.4.

- bpo-32647 [https://bugs.python.org/issue?@action=redirect&bpo=32647]: The ctypes module used to depend on indirect linking for dlopen. The shared extension is now explicitly linked against libdl on platforms with dl.

- bpo-32749 [https://bugs.python.org/issue?@action=redirect&bpo=32749]: A **`dbm.dumb`** database opened with flags 'r' is now read-only. **`dbm.dumb.open()`** with flags 'r' and 'w' no longer creates a database if it does not exist.

- bpo-32741 [https://bugs.python.org/issue?@action=redirect&bpo=32741]: Implement `asyncio.TimerHandle.when()` method.

- bpo-32691 [https://bugs.python.org/issue?@action=redirect&bpo=32691]: Use mod_spec.parent when running modules with pdb

- bpo-32734 [https://bugs.python.org/issue?@action=redirect&bpo=32734]: Fixed `asyncio.Lock()` safety issue which allowed acquiring and locking the same lock multiple times, without it being free. Patch by Bar Harel.

- bpo-32727 [https://bugs.python.org/issue?@action=redirect&bpo=32727]: Do not include name field in SMTP envelope from address. Patch by Stéphane Wirtel

- bpo-31453 [https://bugs.python.org/issue?@action=redirect&bpo=31453]: Add TLSVersion constants and SSLContext.maximum_version / minimum_version attributes. The new API wraps OpenSSL 1.1 https://web.archive.org/web/20180309043602/https://www.openssl.org/docs/man1.1.0/ssl/SSL_CTX_set_min_proto_version.html feature.

- bpo-24334 [https://bugs.python.org/issue?@action=redirect&bpo=24334]: Internal implementation details of ssl module were cleaned up. The SSLSocket has one less layer of indirection. Owner and session information are now handled by the SSLSocket and SSLObject constructor. Channel binding implementation has been simplified.

- bpo-31848 [https://bugs.python.org/issue?@action=redirect&bpo=31848]: Fix the error handling in Aifc_read.initfp() when the SSND chunk is not found. Patch by Zackery Spytz.

- bpo-32585 [https://bugs.python.org/issue?@action=redirect&bpo=32585]: Add Ttk spinbox widget to `tkinter.ttk`. Patch by Alan D Moore.

- bpo-32512 [https://bugs.python.org/issue?@action=redirect&bpo=32512]: `profile` CLI accepts `-m module_name` as an alternative to script path.

- bpo-8525 [https://bugs.python.org/issue?@action=redirect&bpo=8525]: help() on a type now displays builtin subclasses. This is intended primarily to help with notification of more specific exception subclasses.

  Patch by Sanyam Khurana.

- bpo-31639 [https://bugs.python.org/issue?@action=redirect&bpo=31639]: http.server now exposes a ThreadingHTTPServer class and uses it when the module is run with `-m` to cope with web browsers pre-opening sockets.

- bpo-29877 [https://bugs.python.org/issue?@action=redirect&bpo=29877]: compileall: import ProcessPoolExecutor only when needed, preventing hangs on low resource platforms

- bpo-32221 [https://bugs.python.org/issue?@action=redirect&bpo=32221]: Various functions returning tuple containing IPv6 addresses now omit `%scope`

part since the same information is already encoded in *scopeid* tuple item. Especially this speeds up **`socket.recvfrom()`** when it receives multicast packet since useless resolving of network interface name is omitted.

- bpo-32147 [https://bugs.python.org/issue?@action=redirect&bpo=32147]: **`binascii.unhexlify()`** is now up to 2 times faster. Patch by Sergey Fedoseev.

- bpo-30693 [https://bugs.python.org/issue?@action=redirect&bpo=30693]: The TarFile class now recurses directories in a reproducible way.

- bpo-30693 [https://bugs.python.org/issue?@action=redirect&bpo=30693]: The ZipFile class now recurses directories in a reproducible way.

- bpo-31680 [https://bugs.python.org/issue?@action=redirect&bpo=31680]: Added **`curses.ncurses_version`**.

- bpo-31908 [https://bugs.python.org/issue?@action=redirect&bpo=31908]: Fix output of cover files for `trace` module command-line tool. Previously emitted cover files only when `--missing` option was used. Patch by Michael Selik.

- bpo-31608 [https://bugs.python.org/issue?@action=redirect&bpo=31608]: Raise a `TypeError` instead of crashing if a `collections.deque` subclass returns a non-deque from `__new__`. Patch by Oren Milman.

- bpo-31425 [https://bugs.python.org/issue?@action=redirect&bpo=31425]: Add support for sockets of the AF_QIPCRTR address family, supported by the Linux kernel. This is used to communicate with services, such as GPS or radio, running on Qualcomm devices. Patch by Bjorn Andersson.

- bpo-22005 [https://bugs.python.org/issue?@action=redirect&bpo=22005]: Implemented unpickling instances of **`datetime`**, **`date`** and **`time`** pickled by Python 2. `encoding='latin1'` should be used for successful decoding.

- bpo-27645 [https://bugs.python.org/issue?@action=redirect&bpo=27645]: **`sqlite3.Connection`** now exposes a **`backup`** method, if the underlying SQLite library is at version 3.6.11 or higher. Patch by Lele Gaifax.

- bpo-16865 [https://bugs.python.org/issue?@action=redirect&bpo=16865]: Support arrays >=2GiB in **`ctypes`**. Patch by Segev Finer.

- bpo-31508 [https://bugs.python.org/issue?@action=redirect&bpo=31508]: Removed support of arguments in `tkinter.ttk.Treeview.selection`. It was deprecated in 3.6. Use specialized methods like `selection_set` for changing the selection.

- bpo-29456 [https://bugs.python.org/issue?@action=redirect&bpo=29456]: Fix bugs in hangul normalization: u1176, u11a7 and u11c3

Documentação

- bpo-21257 [https://bugs.python.org/issue?@action=redirect&bpo=21257]: Document **`http.client.parse_headers()`**.
- bpo-34764 [https://bugs.python.org/issue?@action=redirect&bpo=34764]: Improve example of iter() with 2nd sentinel argument.
- bpo-35564 [https://bugs.python.org/issue?@action=redirect&bpo=35564]: Explicitly set master_doc variable in conf.py for compliance with Sphinx 2.0
- bpo-35511 [https://bugs.python.org/issue?@action=redirect&bpo=35511]: Specified that profile.Profile class doesn’t not support enable or disable methods. Also, elaborated that Profile object as a context manager is only supported in cProfile module.
- bpo-10536 [https://bugs.python.org/issue?@action=redirect&bpo=10536]: Enhance the gettext docs. Patch by Éric Araujo
- bpo-35089 [https://bugs.python.org/issue?@action=redirect&bpo=35089]: Remove mention of `typing.io` and `typing.re`. Their types should be imported from `typing` directly.
- bpo-35038 [https://bugs.python.org/issue?@action=redirect&bpo=35038]: Fix the documentation about an unexisting `f_restricted` attribute in the frame object. Patch by Stéphane Wirtel
- bpo-35042 [https://bugs.python.org/issue?@action=redirect&bpo=35042]: Replace PEP XYZ by the pep role and allow to use the direct links to the PEPs.

- bpo-35044 [https://bugs.python.org/issue?@action=redirect&bpo=35044]: Fix the documentation with the role `exc` for the appropriated exception. Patch by Stéphane Wirtel
- bpo-35035 [https://bugs.python.org/issue?@action=redirect&bpo=35035]: Rename documentation for **`email.utils`** to `email.utils.rst`.
- bpo-34967 [https://bugs.python.org/issue?@action=redirect&bpo=34967]: Use app.add_object_type() instead of the deprecated Sphinx function app.description_unit()
- bpo-34913 [https://bugs.python.org/issue?@action=redirect&bpo=34913]: Add documentation about the new command line interface of the gzip module.
- bpo-32174 [https://bugs.python.org/issue?@action=redirect&bpo=32174]: chm document displays non-ASCII characters properly on some MBCS Windows systems.
- bpo-11233 [https://bugs.python.org/issue?@action=redirect&bpo=11233]: Create availability directive for documentation. Original patch by Georg Brandl.
- bpo-34790 [https://bugs.python.org/issue?@action=redirect&bpo=34790]: Document how passing coroutines to asyncio.wait() can be confusing.
- bpo-34552 [https://bugs.python.org/issue?@action=redirect&bpo=34552]: Make clear that == operator sometimes is equivalent to `is`. The <, <=, > and >= operators are only defined where they make sense.
- bpo-28617 [https://bugs.python.org/issue?@action=redirect&bpo=28617]: Fixed info in the stdtypes docs concerning the types that support membership tests.
- bpo-20177 [https://bugs.python.org/issue?@action=redirect&bpo=20177]: Migrate datetime.date.fromtimestamp to Argument Clinic. Patch by Tim Hoffmann.
- bpo-34065 [https://bugs.python.org/issue?@action=redirect&bpo=34065]: Fix wrongly written basicConfig documentation markup syntax
- bpo-33460 [https://bugs.python.org/issue?@action=redirect&bpo=33460]: replaced ellipsis with correct error codes in tutorial chapter 3.
- bpo-33847 [https://bugs.python.org/issue?@action=redirect&bpo=33847]: Add '@' operator entry to index.
- bpo-33409 [https://bugs.python.org/issue?@action=redirect&bpo=33409]: Clarified the relationship between **PEP 538** [https://peps.python.org/pep-0538/]'s PYTHONCOERCECLOCALE and PEP 540's PYTHONUTF8 mode.

- bpo-33197 [https://bugs.python.org/issue?@action=redirect&bpo=33197]: Add versionadded tag to the documentation of ParameterKind.description
- bpo-17045 [https://bugs.python.org/issue?@action=redirect&bpo=17045]: Improve the C-API doc for PyTypeObject. This includes adding several quick-reference tables and a lot of missing slot/typedef entries. The existing entries were also cleaned up with a slightly more consistent format.
- bpo-33736 [https://bugs.python.org/issue?@action=redirect&bpo=33736]: Improve the documentation of **`asyncio.open_connection()`**, **`asyncio.start_server()`** and their UNIX socket counterparts.
- bpo-23859 [https://bugs.python.org/issue?@action=redirect&bpo=23859]: Document that `asyncio.wait()` does not cancel its futures on timeout.
- bpo-32436 [https://bugs.python.org/issue?@action=redirect&bpo=32436]: Document **PEP 567** [https://peps.python.org/pep-0567/] changes to asyncio.
- bpo-33604 [https://bugs.python.org/issue?@action=redirect&bpo=33604]: Update HMAC md5 default to a DeprecationWarning, bump removal to 3.8.
- bpo-33594 [https://bugs.python.org/issue?@action=redirect&bpo=33594]: Document `getargspec`, `from_function` and `from_builtin` as deprecated in their respective docstring, and include version since deprecation in DeprecationWarning message.
- bpo-33503 [https://bugs.python.org/issue?@action=redirect&bpo=33503]: Fix broken pypi link
- bpo-33421 [https://bugs.python.org/issue?@action=redirect&bpo=33421]: Add missing documentation for `typing.AsyncContextManager`.
- bpo-33487 [https://bugs.python.org/issue?@action=redirect&bpo=33487]: BZ2file now emit a DeprecationWarning when buffering=None is passed, the deprecation message and documentation also now explicitly state it is deprecated since 3.0.
- bpo-33378 [https://bugs.python.org/issue?@action=redirect&bpo=33378]: Add Korean language switcher for https://docs.python.org/3/
- bpo-33276 [https://bugs.python.org/issue?@action=redirect&bpo=33276]: Clarify that the `__path__` attribute on modules cannot be just any value.
- bpo-33201 [https://bugs.python.org/issue?@action=redirect&bpo=33201]: Modernize documentation for writing C extension types.
- bpo-33195 [https://bugs.python.org/issue?@action=redirect&bpo=33195]: Deprecate `Py_UNICODE` usage in `c-api/arg` document. `Py_UNICODE` related APIs are deprecated since Python 3.3, but it is missed in the document.

- bpo-33126 [https://bugs.python.org/issue?@action=redirect&bpo=33126]: Document PyBuffer_ToContiguous().
- bpo-27212 [https://bugs.python.org/issue?@action=redirect&bpo=27212]: Modify documentation for the **`islice()`** recipe to consume initial values up to the start index.
- bpo-28247 [https://bugs.python.org/issue?@action=redirect&bpo=28247]: Update **`zipapp`** documentation to describe how to make standalone applications.
- bpo-18802 [https://bugs.python.org/issue?@action=redirect&bpo=18802]: Documentation changes for ipaddress. Patch by Jon Foster and Berker Peksag.
- bpo-27428 [https://bugs.python.org/issue?@action=redirect&bpo=27428]: Update documentation to clarify that `WindowsRegistryFinder` implements `MetaPathFinder`. (Patch by Himanshu Lakhara)
- bpo-28124 [https://bugs.python.org/issue?@action=redirect&bpo=28124]: The ssl module function ssl.wrap_socket() has been de-emphasized and deprecated in favor of the more secure and efficient SSLContext.wrap_socket() method.
- bpo-17232 [https://bugs.python.org/issue?@action=redirect&bpo=17232]: Clarify docs for -O and -OO. Patch by Terry Reedy.
- bpo-32436 [https://bugs.python.org/issue?@action=redirect&bpo=32436]: Add documentation for the contextvars module (PEP 567).
- bpo-32800 [https://bugs.python.org/issue?@action=redirect&bpo=32800]: Update link to w3c doc for xml default namespaces.
- bpo-11015 [https://bugs.python.org/issue?@action=redirect&bpo=11015]: Update **`test.support`** documentation.
- bpo-32613 [https://bugs.python.org/issue?@action=redirect&bpo=32613]: Update the faq/windows.html to use the py command from PEP 397 instead of python.
- bpo-8722 [https://bugs.python.org/issue?@action=redirect&bpo=8722]: Document **`__getattr__()`** behavior when property **`get()`** method raises **`AttributeError`**.
- bpo-32614 [https://bugs.python.org/issue?@action=redirect&bpo=32614]: Modify RE examples in documentation to use raw strings to prevent **`DeprecationWarning`** and add text to REGEX HOWTO to highlight the deprecation.

- bpo-20709 [https://bugs.python.org/issue?@action=redirect&bpo=20709]: Remove the paragraph where we explain that os.utime() does not support a directory as path under Windows. Patch by Jan-Philip Gehrcke
- bpo-32722 [https://bugs.python.org/issue?@action=redirect&bpo=32722]: Remove the bad example in the tutorial of the Generator Expression. Patch by Stéphane Wirtel
- bpo-31972 [https://bugs.python.org/issue?@action=redirect&bpo=31972]: Improve docstrings for `pathlib.PurePath` subclasses.
- bpo-30607 [https://bugs.python.org/issue?@action=redirect&bpo=30607]: Use the externalized `python-docs-theme` package when building the documentation.
- bpo-8243 [https://bugs.python.org/issue?@action=redirect&bpo=8243]: Add a note about curses.addch and curses.addstr exception behavior when writing outside a window, or pad.
- bpo-32337 [https://bugs.python.org/issue?@action=redirect&bpo=32337]: Update documentation related with `dict` order.
- bpo-25041 [https://bugs.python.org/issue?@action=redirect&bpo=25041]: Document `AF_PACKET` in the **`socket`** module.
- bpo-31432 [https://bugs.python.org/issue?@action=redirect&bpo=31432]: Clarify meaning of CERT_NONE, CERT_OPTIONAL, and CERT_REQUIRED flags for ssl.SSLContext.verify_mode.

Testes

- bpo-35772 [https://bugs.python.org/issue?@action=redirect&bpo=35772]: Fix sparse file tests of test_tarfile on ppc64 with the tmpfs filesystem. Fix the function testing if the filesystem supports sparse files: create a file which contains data and “holes”, instead of creating a file which contains no data. tmpfs effective block size is a page size (tmpfs lives in the page cache). RHEL uses 64 KiB pages on aarch64, ppc64, ppc64le, only s390x and x86_64 use 4 KiB pages, whereas the test punch holes of 4 KiB.

- bpo-35045 [https://bugs.python.org/issue?@action=redirect&bpo=35045]: Make ssl tests less strict and also accept TLSv1 as system default. The changes unbreaks test_min_max_version on Fedora 29.

- bpo-32710 [https://bugs.python.org/issue?@action=redirect&bpo=32710]: `test_asyncio/test_sendfile.py` now resets the event loop policy using **`tearDownModule()`** as done in other tests, to prevent a warning when running tests on Windows.

- bpo-33717 [https://bugs.python.org/issue?@action=redirect&bpo=33717]: test.pythoninfo now logs information of all clocks, not only time.time() and time.perf_counter().

- bpo-35488 [https://bugs.python.org/issue?@action=redirect&bpo=35488]: Add a test to pathlib's Path.match() to verify it does not support glob-style ** recursive pattern matching.

- bpo-31731 [https://bugs.python.org/issue?@action=redirect&bpo=31731]: Fix a race condition in `check_interrupted_write()` of test_io: create directly the thread with SIGALRM signal blocked, rather than blocking the signal later from the thread. Previously, it was possible that the thread gets the signal before the signal is blocked.

- bpo-35424 [https://bugs.python.org/issue?@action=redirect&bpo=35424]: Fix test_multiprocessing_main_handling: use **`multiprocessing.Pool`** with a context manager and then explicitly join the pool.

- bpo-35519 [https://bugs.python.org/issue?@action=redirect&bpo=35519]: Rename **`test.bisect`** module to **`test.bisect_cmd`** to avoid conflict with **`bisect`** module when running directly a test like `./python Lib/test/test_xmlrpc.py`.

- bpo-35513 [https://bugs.python.org/issue?@action=redirect&bpo=35513]: Replace **`time.time()`** with **`time.monotonic()`** in tests to measure time delta.

- bpo-34279 [https://bugs.python.org/issue?@action=redirect&bpo=34279]: **`test.support.run_unittest()`** no longer raise **`TestDidNotRun`** if the test result contains skipped tests. The exception is now only raised if no test have been run and no test have been skipped.

- bpo-35412 [https://bugs.python.org/issue?@action=redirect&bpo=35412]: Add testcase to `test_future4`: check unicode literal.

- bpo-26704 [https://bugs.python.org/issue?@action=redirect&bpo=26704]: Added test demonstrating double-patching of an instance method. Patch by Anthony Sottile.

- bpo-33725 [https://bugs.python.org/issue?@action=redirect&bpo=33725]: test_multiprocessing_fork may crash on recent versions of macOS. Until the issue is resolved, skip the test on macOS.

- bpo-35352 [https://bugs.python.org/issue?@action=redirect&bpo=35352]: Modify test_asyncio to use the certificate set from the test directory.

- bpo-35317 [https://bugs.python.org/issue?@action=redirect&bpo=35317]: Fix `mktime()` overflow error in `test_email`: run `test_localtime_daylight_true_dst_true()` and `test_localtime_daylight_false_dst_true()` with a specific timezone.

- bpo-21263 [https://bugs.python.org/issue?@action=redirect&bpo=21263]: After several reports that test_gdb does not work properly on macOS and since gdb is not shipped by default anymore, test_gdb is now skipped on macOS when LLVM Clang has been used to compile Python. Patch by Lysandros Nikolaou

- bpo-34279 [https://bugs.python.org/issue?@action=redirect&bpo=34279]: regrtest issue a warning when no tests have been executed in a particular test file. Also, a new final result state is issued if no test have been executed across all test files. Patch by Pablo Galindo.

- bpo-34962 [https://bugs.python.org/issue?@action=redirect&bpo=34962]: make docstest in Doc now passes., and is enforced in CI

- bpo-23596 [https://bugs.python.org/issue?@action=redirect&bpo=23596]: Use argparse for the command line of the gzip module. Patch by Antony Lee

- bpo-34537 [https://bugs.python.org/issue?@action=redirect&bpo=34537]: Fix `test_gdb.test_strings()` when `LC_ALL=C` and GDB was compiled with Python 3.6 or earlier.

- bpo-34587 [https://bugs.python.org/issue?@action=redirect&bpo=34587]: test_socket: Remove RDSTest.testCongestion(). The test tries to fill the receiver's socket buffer and expects an error. But the RDS protocol doesn't require that. Moreover, the Linux implementation of RDS expects that the producer of the messages reduces its rate, it's not the role of the receiver to trigger an error. The test fails on Fedora 28 by design, so just remove it.

- bpo-34661 [https://bugs.python.org/issue?@action=redirect&bpo=34661]: Fix test_shutil if unzip doesn't support -t.

- bpo-34200 [https://bugs.python.org/issue?@action=redirect&bpo=34200]: Fixed non-deterministic flakiness of test_pkg by not using the scary test.support.module_cleanup() logic to save and restore sys.modules contents between test cases.

- bpo-34569 [https://bugs.python.org/issue?@action=redirect&bpo=34569]: The experimental PEP 554 data channels now correctly pass negative PyLong objects between subinterpreters on 32-bit systems. Patch by Michael Felt.

- bpo-34594 [https://bugs.python.org/issue?@action=redirect&bpo=34594]: Fix usage of hardcoded `errno` values in the tests.

- bpo-34579 [https://bugs.python.org/issue?@action=redirect&bpo=34579]: Fix test_embed for AIX Patch by Michael Felt

- bpo-34542 [https://bugs.python.org/issue?@action=redirect&bpo=34542]: Use 3072 RSA keys and SHA-256 signature for test certs and keys.

- bpo-11193 [https://bugs.python.org/issue?@action=redirect&bpo=11193]: Remove special condition for AIX in `test_subprocess.test_undecodable_env`

- bpo-34347 [https://bugs.python.org/issue?@action=redirect&bpo=34347]: Fix `test_utf8_mode.test_cmd_line` for AIX

- bpo-34490 [https://bugs.python.org/issue?@action=redirect&bpo=34490]: On AIX with AF_UNIX family sockets getsockname() does not provide ‘sockname’, so skip calls to transport.get_extra_info(‘sockname’)

- bpo-34391 [https://bugs.python.org/issue?@action=redirect&bpo=34391]: Fix ftplib test for TLS 1.3 by reading from data socket.

- bpo-11192 [https://bugs.python.org/issue?@action=redirect&bpo=11192]: Fix `test_socket` on AIX 6.1 and later IPv6 zone id supports only supported by `inet_pton6_zone()`. Switch to runtime-based `platform.system()` to establish current platform rather than build-time based `sys.platform()`

- bpo-34399 [https://bugs.python.org/issue?@action=redirect&bpo=34399]: Update all RSA keys and DH params to use at least 2048 bits.

- bpo-34373 [https://bugs.python.org/issue?@action=redirect&bpo=34373]: Fix `test_mktime` and `test_pthread_getcpuclickid` tests for AIX Add range checking for `_PyTime_localtime` for AIX Patch by Michael Felt

- bpo-11191 [https://bugs.python.org/issue?@action=redirect&bpo=11191]: Skip the distutils test ‘test_search_cpp’ when using XLC as compiler patch by aixtools (Michael Felt)

- Improved an error message when mock assert_has_calls fails.

- bpo-33746 [https://bugs.python.org/issue?@action=redirect&bpo=33746]: Fix test_unittest when run in verbose mode.

- bpo-33901 [https://bugs.python.org/issue?@action=redirect&bpo=33901]: Fix test_dbm_gnu on macOS with gdbm 1.15: add a larger value to make sure that the file size changes.

- bpo-33873 [https://bugs.python.org/issue?@action=redirect&bpo=33873]: Fix a bug in `regrtest` that caused an extra test to run if –huntrleaks/-R was used. Exit with error in case that invalid parameters are specified to –huntrleaks/-R (at least one warmup run and one repetition must be used).

- bpo-33562 [https://bugs.python.org/issue?@action=redirect&bpo=33562]: Check that a global asyncio event loop policy is not left behind by any tests.

- bpo-33655 [https://bugs.python.org/issue?@action=redirect&bpo=33655]: Ignore test_posix_fallocate failures on BSD platforms that might be due to running on ZFS.

- bpo-32962 [https://bugs.python.org/issue?@action=redirect&bpo=32962]: Fixed test_gdb when Python is compiled with flags -mcet -fcf-protection -O0.

- bpo-33358 [https://bugs.python.org/issue?@action=redirect&bpo=33358]: Fix `test_embed.test_pre_initialization_sys_options()` when the interpreter is built with `--enable-shared`.

- bpo-32872 [https://bugs.python.org/issue?@action=redirect&bpo=32872]: Avoid regrtest compatibility issue with namespace packages.

- bpo-32517 [https://bugs.python.org/issue?@action=redirect&bpo=32517]: Fix failing `test_asyncio` on macOS 10.12.2+ due to transport of `KqueueSelector` loop was not being closed.

- bpo-32663 [https://bugs.python.org/issue?@action=redirect&bpo=32663]: Making sure the `SMTPUTF8SimTests` class of tests gets run in `test_smtplib.py`.

- bpo-27643 [https://bugs.python.org/issue?@action=redirect&bpo=27643]: Test_C test case needs “signed short” bitfields, but the IBM XLC compiler (on AIX) does not support this Skip the code and test when AIX and XLC are used

  Applicable to Python2-2.7 and later

- bpo-19417 [https://bugs.python.org/issue?@action=redirect&bpo=19417]: Add test_bdb.py.

- bpo-31809 [https://bugs.python.org/issue?@action=redirect&bpo=31809]: Add tests to verify connection with secp ECDH curves.

Construção

- bpo-34691 [https://bugs.python.org/issue?@action=redirect&bpo=34691]: The _contextvars module is now built into the core Python library on Windows.
- bpo-35683 [https://bugs.python.org/issue?@action=redirect&bpo=35683]: Improved Azure Pipelines build steps and now verifying layouts correctly
- bpo-35642 [https://bugs.python.org/issue?@action=redirect&bpo=35642]: Remove asynciomodule.c from pythoncore.vcxproj
- bpo-35550 [https://bugs.python.org/issue?@action=redirect&bpo=35550]: Fix incorrect Solaris #ifdef checks to look for __sun && __SVR4 instead of sun when compiling.
- bpo-35499 [https://bugs.python.org/issue?@action=redirect&bpo=35499]: `make profile-opt` no longer replaces `CFLAGS_NODIST` with `CFLAGS`. It now adds profile-guided optimization (PGO) flags to `CFLAGS_NODIST`: existing `CFLAGS_NODIST` flags are kept.
- bpo-35257 [https://bugs.python.org/issue?@action=redirect&bpo=35257]: Avoid leaking the linker flags from Link Time Optimizations (LTO) into distutils when compiling C extensions.
- bpo-35351 [https://bugs.python.org/issue?@action=redirect&bpo=35351]: When building Python with clang and LTO, LTO flags are no longer passed into CFLAGS to build third-party C extensions through distutils.
- bpo-35139 [https://bugs.python.org/issue?@action=redirect&bpo=35139]: Fix a compiler error when statically linking `pyexpat` in `Modules/Setup`.
- bpo-35059 [https://bugs.python.org/issue?@action=redirect&bpo=35059]: PCbuild: Set InlineFunctionExpansion to OnlyExplicitInline ("/Ob1" option) in pyproject.props in Debug mode to expand functions marked as inline. This change should make Python compiled in Debug mode a little bit faster on Windows.
- bpo-35011 [https://bugs.python.org/issue?@action=redirect&bpo=35011]: Restores the use of pyexpatns.h to isolate our embedded copy of the expat C library so that its symbols do not conflict at link or dynamic loading time with an embedding application or other extension modules with their own version of libexpat.
- bpo-28015 [https://bugs.python.org/issue?@action=redirect&bpo=28015]: Have –with-lto works correctly with clang.

- bpo-34765 [https://bugs.python.org/issue?@action=redirect&bpo=34765]: Update the outdated install-sh file to the latest revision from automake v1.16.1
- bpo-34585 [https://bugs.python.org/issue?@action=redirect&bpo=34585]: Check for floating-point byte order in configure.ac using compilation tests instead of executing code, so that these checks work in cross-compiled builds.
- bpo-34710 [https://bugs.python.org/issue?@action=redirect&bpo=34710]: Fixed SSL module build with OpenSSL & pedantic CFLAGS.
- bpo-34582 [https://bugs.python.org/issue?@action=redirect&bpo=34582]: Add JUnit XML output for regression tests and update Azure DevOps builds.
- bpo-34081 [https://bugs.python.org/issue?@action=redirect&bpo=34081]: Make Sphinx warnings as errors in the Docs Makefile.
- bpo-34555 [https://bugs.python.org/issue?@action=redirect&bpo=34555]: Fix for case where it was not possible to have both `HAVE_LINUX_VM_SOCKETS_H` and `HAVE_SOCKADDR_ALG` be undefined.
- bpo-33015 [https://bugs.python.org/issue?@action=redirect&bpo=33015]: Fix an undefined behaviour in the pthread implementation of **`PyThread_start_new_thread()`**: add a function wrapper to always return `NULL`.
- bpo-34245 [https://bugs.python.org/issue?@action=redirect&bpo=34245]: The Python shared library is now installed with write permission (mode 0755), which is the standard way of installing such libraries.
- bpo-34121 [https://bugs.python.org/issue?@action=redirect&bpo=34121]: Fix detection of C11 atomic support on clang.
- bpo-32430 [https://bugs.python.org/issue?@action=redirect&bpo=32430]: Rename Modules/Setup.dist to Modules/Setup, and remove the necessity to copy the former manually to the latter when updating the local source tree.
- bpo-30345 [https://bugs.python.org/issue?@action=redirect&bpo=30345]: Add -g to LDFLAGS when compiling with LTO to get debug symbols.
- bpo-5755 [https://bugs.python.org/issue?@action=redirect&bpo=5755]: Move `-Wstrict-prototypes` option to `CFLAGS_NODIST` from `OPT`. This option emitted annoying warnings when building extension modules written in C++.
- bpo-33614 [https://bugs.python.org/issue?@action=redirect&bpo=33614]: Ensures module definition files for the stable ABI on Windows are correctly regenerated.

- bpo-33648 [https://bugs.python.org/issue?@action=redirect&bpo=33648]: The –with-c-locale-warning configuration flag has been removed. It has had no effect for about a year.
- bpo-33522 [https://bugs.python.org/issue?@action=redirect&bpo=33522]: Enable CI builds on Visual Studio Team Services at https://python.visualstudio.com/cpython
- bpo-33512 [https://bugs.python.org/issue?@action=redirect&bpo=33512]: configure's check for "long double" has been simplified
- bpo-33483 [https://bugs.python.org/issue?@action=redirect&bpo=33483]: C compiler is now correctly detected from the standard environment variables. –without-gcc and –with-icc options have been removed.
- bpo-33394 [https://bugs.python.org/issue?@action=redirect&bpo=33394]: Enable the verbose build for extension modules, when GNU make is passed macros on the command line.
- bpo-33393 [https://bugs.python.org/issue?@action=redirect&bpo=33393]: Update config.guess and config.sub files.
- bpo-33377 [https://bugs.python.org/issue?@action=redirect&bpo=33377]: Add new triplets for mips r6 and riscv variants (used in extension suffixes).
- bpo-32232 [https://bugs.python.org/issue?@action=redirect&bpo=32232]: By default, modules configured in `Modules/Setup` are no longer built with `-DPy_BUILD_CORE`. Instead, modules that specifically need that preprocessor definition include it in their individual entries.
- bpo-33182 [https://bugs.python.org/issue?@action=redirect&bpo=33182]: The embedding tests can once again be built with clang 6.0
- bpo-33163 [https://bugs.python.org/issue?@action=redirect&bpo=33163]: Upgrade pip to 9.0.3 and setuptools to v39.0.1.
- bpo-33012 [https://bugs.python.org/issue?@action=redirect&bpo=33012]: gcc 8 has added a new warning heuristic to detect invalid function casts and a stock python build seems to hit that warning quite often. The most common is the cast of a METH_NOARGS function (that uses just one argument) to a PyCFunction. Fix this by adding a dummy argument to all functions that implement METH_NOARGS.
- bpo-32898 [https://bugs.python.org/issue?@action=redirect&bpo=32898]: Fix the python debug build when using COUNT_ALLOCS.
- bpo-29442 [https://bugs.python.org/issue?@action=redirect&bpo=29442]: Replace optparse with argparse in setup.py

Windows

- bpo-35890 [https://bugs.python.org/issue?@action=redirect&bpo=35890]: Fix API calling consistency of GetVersionEx and wcstok.
- bpo-32560 [https://bugs.python.org/issue?@action=redirect&bpo=32560]: The `py` launcher now forwards its `STARTUPINFO` structure to child processes.
- bpo-35854 [https://bugs.python.org/issue?@action=redirect&bpo=35854]: Fix EnvBuilder and –symlinks in venv on Windows
- bpo-35811 [https://bugs.python.org/issue?@action=redirect&bpo=35811]: Avoid propagating venv settings when launching via py.exe
- bpo-35797 [https://bugs.python.org/issue?@action=redirect&bpo=35797]: Fix default executable used by the multiprocessing module
- bpo-35758 [https://bugs.python.org/issue?@action=redirect&bpo=35758]: Allow building on ARM with MSVC.
- bpo-29734 [https://bugs.python.org/issue?@action=redirect&bpo=29734]: Fix handle leaks in os.stat on Windows.
- bpo-35596 [https://bugs.python.org/issue?@action=redirect&bpo=35596]: Use unchecked PYCs for the embeddable distro to avoid zipimport restrictions.
- bpo-35596 [https://bugs.python.org/issue?@action=redirect&bpo=35596]: Fix vcruntime140.dll being added to embeddable distro multiple times.
- bpo-35402 [https://bugs.python.org/issue?@action=redirect&bpo=35402]: Update Windows build to use Tcl and Tk 8.6.9
- bpo-35401 [https://bugs.python.org/issue?@action=redirect&bpo=35401]: Updates Windows build to OpenSSL 1.1.0j
- bpo-34977 [https://bugs.python.org/issue?@action=redirect&bpo=34977]: venv on Windows will now use a python.exe redirector rather than copying the actual binaries from the base environment.
- bpo-34977 [https://bugs.python.org/issue?@action=redirect&bpo=34977]: Adds support for building a Windows App Store package
- bpo-35067 [https://bugs.python.org/issue?@action=redirect&bpo=35067]: Remove _distutils_findvs module and use vswhere.exe instead.
- bpo-32557 [https://bugs.python.org/issue?@action=redirect&bpo=32557]: Allow shutil.disk_usage to take a file path on Windows
- bpo-34770 [https://bugs.python.org/issue?@action=redirect&bpo=34770]: Fix a possible null pointer dereference in pyshellext.cpp.

- bpo-34603 [https://bugs.python.org/issue?@action=redirect&bpo=34603]: Fix returning structs from functions produced by MSVC
- bpo-34581 [https://bugs.python.org/issue?@action=redirect&bpo=34581]: Guard MSVC-specific code in socketmodule.c with `#ifdef _MSC_VER`.
- bpo-34532 [https://bugs.python.org/issue?@action=redirect&bpo=34532]: Fixes exit code of list version arguments for py.exe.
- bpo-34062 [https://bugs.python.org/issue?@action=redirect&bpo=34062]: Fixed the '–list' and '–list-paths' arguments for the py.exe launcher
- bpo-34225 [https://bugs.python.org/issue?@action=redirect&bpo=34225]: Ensure INCLUDE and LIB directories do not end with a backslash.
- bpo-34011 [https://bugs.python.org/issue?@action=redirect&bpo=34011]: A suite of code has been changed which copied across DLLs and init.tcl from the running Python location into a venv being created. These copies are needed only when running from a Python source build, and the copying code is now only run when that is the case, rather than whenever a venv is created.
- bpo-34006 [https://bugs.python.org/issue?@action=redirect&bpo=34006]: Revert line length limit for Windows help docs. The line-length limit is not needed because the pages appear in a separate app rather than on a browser tab. It can also interact badly with the DPI setting.
- bpo-31546 [https://bugs.python.org/issue?@action=redirect&bpo=31546]: Restore running PyOS_InputHook while waiting for user input at the prompt. The restores integration of interactive GUI windows (such as Matplotlib figures) with the prompt on Windows.
- bpo-30237 [https://bugs.python.org/issue?@action=redirect&bpo=30237]: Output error when ReadConsole is canceled by CancelSynchronousIo instead of crashing.
- bpo-33895 [https://bugs.python.org/issue?@action=redirect&bpo=33895]: GIL is released while calling functions that acquire Windows loader lock.
- bpo-33720 [https://bugs.python.org/issue?@action=redirect&bpo=33720]: Reduces maximum marshal recursion depth on release builds.
- bpo-29097 [https://bugs.python.org/issue?@action=redirect&bpo=29097]: Fix bug where **`datetime.fromtimestamp()`** erroneously throws an **`OSError`** on Windows for values between 0 and 86400. Patch by Ammar Askar.
- bpo-33316 [https://bugs.python.org/issue?@action=redirect&bpo=33316]: PyThread_release_lock always fails

- bpo-33184 [https://bugs.python.org/issue?@action=redirect&bpo=33184]: Update Windows installer to use OpenSSL 1.1.0h.
- bpo-32890 [https://bugs.python.org/issue?@action=redirect&bpo=32890]: Fix usage of GetLastError() instead of errno in os.execve() and os.truncate().
- bpo-33016 [https://bugs.python.org/issue?@action=redirect&bpo=33016]: Fix potential use of uninitialized memory in nt._getfinalpathname
- bpo-32903 [https://bugs.python.org/issue?@action=redirect&bpo=32903]: Fix a memory leak in os.chdir() on Windows if the current directory is set to a UNC path.
- bpo-32901 [https://bugs.python.org/issue?@action=redirect&bpo=32901]: Update Tcl and Tk versions to 8.6.8
- bpo-31966 [https://bugs.python.org/issue?@action=redirect&bpo=31966]: Fixed WindowsConsoleIO.write() for writing empty data.
- bpo-32409 [https://bugs.python.org/issue?@action=redirect&bpo=32409]: Ensures activate.bat can handle Unicode contents.
- bpo-32457 [https://bugs.python.org/issue?@action=redirect&bpo=32457]: Improves handling of denormalized executable path when launching Python.
- bpo-32370 [https://bugs.python.org/issue?@action=redirect&bpo=32370]: Use the correct encoding for ipconfig output in the uuid module. Patch by Segev Finer.
- bpo-29248 [https://bugs.python.org/issue?@action=redirect&bpo=29248]: Fix **`os.readlink()`** on Windows, which was mistakenly treating the `PrintNameOffset` field of the reparse data buffer as a number of characters instead of bytes. Patch by Craig Holmquist and SSE4.
- bpo-1104 [https://bugs.python.org/issue?@action=redirect&bpo=1104]: Correctly handle string length in `msilib.SummaryInfo.GetProperty()` to prevent it from truncating the last character.

macOS

- bpo-35401 [https://bugs.python.org/issue?@action=redirect&bpo=35401]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.0j.
- bpo-35025 [https://bugs.python.org/issue?@action=redirect&bpo=35025]: Properly guard the use of the `CLOCK_GETTIME` et al. macros in `timemodule` on macOS.
- bpo-24658 [https://bugs.python.org/issue?@action=redirect&bpo=24658]: On macOS, fix reading from and writing into a file with a size larger than 2 GiB.

- bpo-34405 [https://bugs.python.org/issue?@action=redirect&bpo=34405]: Update to OpenSSL 1.1.0i for macOS installer builds.
- bpo-33635 [https://bugs.python.org/issue?@action=redirect&bpo=33635]: In macOS stat on some file descriptors (/dev/fd/3 f.e) will result in bad file descriptor OSError. Guard against this exception was added in is_dir, is_file and similar methods. DirEntry.is_dir can also throw this exception so _RecursiveWildcardSelector._iterate_directories was also extended with the same error ignoring pattern.
- bpo-13631 [https://bugs.python.org/issue?@action=redirect&bpo=13631]: The .editrc file in user's home directory is now processed correctly during the readline initialization through editline emulation on macOS.
- bpo-33184 [https://bugs.python.org/issue?@action=redirect&bpo=33184]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.0h.
- bpo-32726 [https://bugs.python.org/issue?@action=redirect&bpo=32726]: Build and link with private copy of Tcl/Tk 8.6 for the macOS 10.6+ installer. The 10.9+ installer variant already does this. This means that the Python 3.7 provided by the python.org macOS installers no longer need or use any external versions of Tcl/Tk, either system-provided or user-installed, such as ActiveTcl.
- bpo-32901 [https://bugs.python.org/issue?@action=redirect&bpo=32901]: Update macOS 10.9+ installer to Tcl/Tk 8.6.8.
- bpo-31903 [https://bugs.python.org/issue?@action=redirect&bpo=31903]: In `_scproxy`, drop the GIL when calling into `SystemConfiguration` to avoid deadlocks.

IDLE

- bpo-35770 [https://bugs.python.org/issue?@action=redirect&bpo=35770]: IDLE macosx deletes Options => Configure IDLE. It previously deleted Window => Zoom Height by mistake. (Zoom Height is now on the Options menu). On Mac, the settings dialog is accessed via Preferences on the IDLE menu.
- bpo-35769 [https://bugs.python.org/issue?@action=redirect&bpo=35769]: Change IDLE's new file name from 'Untitled' to 'untitled'
- bpo-35660 [https://bugs.python.org/issue?@action=redirect&bpo=35660]: Fix imports in idlelib.window.

- bpo-35641 [https://bugs.python.org/issue?@action=redirect&bpo=35641]: Proper format `calltip` when the function has no docstring.
- bpo-33987 [https://bugs.python.org/issue?@action=redirect&bpo=33987]: Use ttk Frame for ttk widgets.
- bpo-34055 [https://bugs.python.org/issue?@action=redirect&bpo=34055]: Fix erroneous 'smart' indents and newlines in IDLE Shell.
- bpo-35591 [https://bugs.python.org/issue?@action=redirect&bpo=35591]: Find Selection now works when selection not found.
- bpo-35196 [https://bugs.python.org/issue?@action=redirect&bpo=35196]: Speed up squeezer line counting.
- bpo-35598 [https://bugs.python.org/issue?@action=redirect&bpo=35598]: Update config_key: use PEP 8 names and ttk widgets, make some objects global, and add tests.
- bpo-28097 [https://bugs.python.org/issue?@action=redirect&bpo=28097]: Add Previous/Next History entries to Shell menu.
- bpo-35208 [https://bugs.python.org/issue?@action=redirect&bpo=35208]: Squeezer now properly counts wrapped lines before newlines.
- bpo-35555 [https://bugs.python.org/issue?@action=redirect&bpo=35555]: Gray out Code Context menu entry when it's not applicable.
- bpo-35521 [https://bugs.python.org/issue?@action=redirect&bpo=35521]: Document the IDLE editor code context feature. Add some internal references within the IDLE doc.
- bpo-22703 [https://bugs.python.org/issue?@action=redirect&bpo=22703]: The Code Context menu label now toggles between Show/Hide Code Context. The Zoom Height menu now toggles between Zoom/Restore Height. Zoom Height has moved from the Window menu to the Options menu.
- bpo-35213 [https://bugs.python.org/issue?@action=redirect&bpo=35213]: Where appropriate, use 'macOS' in idlelib.
- bpo-34864 [https://bugs.python.org/issue?@action=redirect&bpo=34864]: On macOS, warn if the system preference "Prefer tabs when opening documents" is set to "Always".
- bpo-34864 [https://bugs.python.org/issue?@action=redirect&bpo=34864]: Document two IDLE on MacOS issues. The System Preferences Dock "prefer tabs always" setting disables some IDLE features. Menus are a bit different than as described for Windows and Linux.
- bpo-35202 [https://bugs.python.org/issue?@action=redirect&bpo=35202]: Remove unused imports from lib/idlelib

- bpo-33000 [https://bugs.python.org/issue?@action=redirect&bpo=33000]: Document that IDLE's shell has no line limit. A program that runs indefinitely can overfill memory.
- bpo-23220 [https://bugs.python.org/issue?@action=redirect&bpo=23220]: Explain how IDLE's Shell displays output.
- bpo-35099 [https://bugs.python.org/issue?@action=redirect&bpo=35099]: Improve the doc about IDLE running user code. The section is renamed from "IDLE – console differences" is renamed "Running user code". It mostly covers the implications of using custom `sys.std`*xxx* objects.
- bpo-35097 [https://bugs.python.org/issue?@action=redirect&bpo=35097]: Add IDLE doc subsection explaining editor windows. Topics include opening, title and status bar, .py* extension, and running.
- bpo-35093 [https://bugs.python.org/issue?@action=redirect&bpo=35093]: Document the IDLE document viewer in the IDLE doc. Add a paragraph in "Help and preferences", "Help sources" subsection.
- bpo-35088 [https://bugs.python.org/issue?@action=redirect&bpo=35088]: Update idlelib.help.copy_string docstring. We now use git and backporting instead of hg and forward merging.
- bpo-35087 [https://bugs.python.org/issue?@action=redirect&bpo=35087]: Update idlelib help files for the current doc build. The main change is the elimination of chapter-section numbers.
- bpo-34548 [https://bugs.python.org/issue?@action=redirect&bpo=34548]: Use configured color theme for read-only text views.
- bpo-1529353 [https://bugs.python.org/issue?@action=redirect&bpo=1529353]: Enable "squeezing" of long outputs in the shell, to avoid performance degradation and to clean up the history without losing it. Squeezed outputs may be copied, viewed in a separate window, and "unsqueezed".
- bpo-34047 [https://bugs.python.org/issue?@action=redirect&bpo=34047]: Fixed mousewheel scrolling direction on macOS.
- bpo-34275 [https://bugs.python.org/issue?@action=redirect&bpo=34275]: Make IDLE calltips always visible on Mac. Some MacOS-tk combinations need .update_idletasks(). Patch by Kevin Walzer.
- bpo-34120 [https://bugs.python.org/issue?@action=redirect&bpo=34120]: Fix unresponsiveness after closing certain windows and dialogs.
- bpo-33975 [https://bugs.python.org/issue?@action=redirect&bpo=33975]: Avoid small type when running htests. Since part of the purpose of human-

viewed tests is to determine that widgets look right, it is important that they look the same for testing as when running IDLE.

- bpo-33905 [https://bugs.python.org/issue?@action=redirect&bpo=33905]: Add test for idlelib.stackview.StackBrowser.
- bpo-33924 [https://bugs.python.org/issue?@action=redirect&bpo=33924]: Change mainmenu.menudefs key ‘windows’ to ‘window’. Every other menudef key is lowercase version of main menu entry.
- bpo-33906 [https://bugs.python.org/issue?@action=redirect&bpo=33906]: Rename idlelib.windows as window Match Window on the main menu and remove last plural module name.
- bpo-33917 [https://bugs.python.org/issue?@action=redirect&bpo=33917]: Fix and document idlelib/idle_test/template.py. The revised file compiles, runs, and tests OK. idle_test/README.txt explains how to use it to create new IDLE test files.
- bpo-33904 [https://bugs.python.org/issue?@action=redirect&bpo=33904]: IDLE: In rstrip, rename class RstripExtension as Rstrip
- bpo-33907 [https://bugs.python.org/issue?@action=redirect&bpo=33907]: For consistency and clarity, rename an IDLE module and classes. Module calltips and its class CallTips are now calltip and Calltip. In module calltip_w, class CallTip is now CalltipWindow.
- bpo-33856 [https://bugs.python.org/issue?@action=redirect&bpo=33856]: Add “help” in the welcome message of IDLE
- bpo-33839 [https://bugs.python.org/issue?@action=redirect&bpo=33839]: IDLE: refactor ToolTip and CallTip and add documentation and tests
- bpo-33855 [https://bugs.python.org/issue?@action=redirect&bpo=33855]: Minimally test all IDLE modules. Add missing files, import module, instantiate classes, and check coverage. Check existing files.
- bpo-33656 [https://bugs.python.org/issue?@action=redirect&bpo=33656]: On Windows, add API call saying that tk scales for DPI. On Windows 8.1+ or 10, with DPI compatibility properties of the Python binary unchanged, and a monitor resolution greater than 96 DPI, this should make text and lines sharper. It should otherwise have no effect.
- bpo-33768 [https://bugs.python.org/issue?@action=redirect&bpo=33768]: Clicking on a context line moves that line to the top of the editor window.
- bpo-33763 [https://bugs.python.org/issue?@action=redirect&bpo=33763]: IDLE: Use read-only text widget for code context instead of label widget.

- bpo-33664 [https://bugs.python.org/issue?@action=redirect&bpo=33664]: Scroll IDLE editor text by lines. Previously, the mouse wheel and scrollbar slider moved text by a fixed number of pixels, resulting in partial lines at the top of the editor box. The change also applies to the shell and grep output windows, but not to read-only text views.
- bpo-33679 [https://bugs.python.org/issue?@action=redirect&bpo=33679]: Enable theme-specific color configuration for Code Context. Use the Highlights tab to see the setting for built-in themes or add settings to custom themes.
- bpo-33642 [https://bugs.python.org/issue?@action=redirect&bpo=33642]: Display up to maxlines non-blank lines for Code Context. If there is no current context, show a single blank line.
- bpo-33628 [https://bugs.python.org/issue?@action=redirect&bpo=33628]: IDLE: Cleanup codecontext.py and its test.
- bpo-33564 [https://bugs.python.org/issue?@action=redirect&bpo=33564]: IDLE's code context now recognizes async as a block opener.
- bpo-21474 [https://bugs.python.org/issue?@action=redirect&bpo=21474]: Update word/identifier definition from ascii to unicode. In text and entry boxes, this affects selection by double-click, movement left/right by control-left/right, and deletion left/right by control-BACKSPACE/DEL.
- bpo-33204 [https://bugs.python.org/issue?@action=redirect&bpo=33204]: IDLE: consistently color invalid string prefixes. A 'u' string prefix cannot be paired with either 'r' or 'f'. Consistently color as much of the prefix, starting at the right, as is valid. Revise and extend colorizer test.
- bpo-32984 [https://bugs.python.org/issue?@action=redirect&bpo=32984]: Set `__file__` while running a startup file. Like Python, IDLE optionally runs one startup file in the Shell window before presenting the first interactive input prompt. For IDLE, `-s` runs a file named in environmental variable **`IDLESTARTUP`** or **`PYTHONSTARTUP`**; `-r file` runs `file`. Python sets `__file__` to the startup file name before running the file and unsets it before the first prompt. IDLE now does the same when run normally, without the `-n` option.
- bpo-32940 [https://bugs.python.org/issue?@action=redirect&bpo=32940]: Simplify and rename StringTranslatePseudoMapping in pyparse.
- bpo-32916 [https://bugs.python.org/issue?@action=redirect&bpo=32916]: Change `str` to `code` in pyparse.

- bpo-32905 [https://bugs.python.org/issue?@action=redirect&bpo=32905]: Remove unused code in pyparse module.
- bpo-32874 [https://bugs.python.org/issue?@action=redirect&bpo=32874]: Add tests for pyparse.
- bpo-32837 [https://bugs.python.org/issue?@action=redirect&bpo=32837]: Using the system and place-dependent default encoding for open() is a bad idea for IDLE's system and location-independent files.
- bpo-32826 [https://bugs.python.org/issue?@action=redirect&bpo=32826]: Add "encoding=utf-8" to open() in IDLE's test_help_about. GUI test test_file_buttons() only looks at initial ascii-only lines, but failed on systems where open() defaults to 'ascii' because readline() internally reads and decodes far enough ahead to encounter a non-ascii character in CREDITS.txt.
- bpo-32831 [https://bugs.python.org/issue?@action=redirect&bpo=32831]: Add docstrings and tests for codecontext.
- bpo-32765 [https://bugs.python.org/issue?@action=redirect&bpo=32765]: Update configdialog General tab docstring to add new widgets to the widget list.

Ferramentas/Demos

- bpo-35884 [https://bugs.python.org/issue?@action=redirect&bpo=35884]: Add a benchmark script for timing various ways to access variables: `Tools/scripts/var_access_benchmark.py`.
- bpo-34989 [https://bugs.python.org/issue?@action=redirect&bpo=34989]: python-gdb.py now handles errors on computing the line number of a Python frame.
- bpo-20260 [https://bugs.python.org/issue?@action=redirect&bpo=20260]: Argument Clinic now has non-bitwise unsigned int converters.
- bpo-32962 [https://bugs.python.org/issue?@action=redirect&bpo=32962]: python-gdb now catches `UnicodeDecodeError` exceptions when calling `string()`.
- bpo-32962 [https://bugs.python.org/issue?@action=redirect&bpo=32962]: python-gdb now catches ValueError on read_var(): when Python has no debug symbols for example.
- bpo-33189 [https://bugs.python.org/issue?@action=redirect&bpo=33189]: **pygettext.py** now recognizes only literal strings as docstrings and translatable strings, and rejects bytes literals and f-string expressions.

- bpo-31920 [https://bugs.python.org/issue?@action=redirect&bpo=31920]: Fixed handling directories as arguments in the `pygettext` script. Based on patch by Oleg Krasnikov.
- bpo-29673 [https://bugs.python.org/issue?@action=redirect&bpo=29673]: Fix pystackv and pystack gdbinit macros.
- bpo-25427 [https://bugs.python.org/issue?@action=redirect&bpo=25427]: Remove the pyvenv script in favor of `python3 -m venv` in order to lower confusion as to what Python interpreter a virtual environment will be created for.
- bpo-32885 [https://bugs.python.org/issue?@action=redirect&bpo=32885]: Add an `-n` flag for `Tools/scripts/pathfix.py` to disable automatic backup creation (files with ~ suffix).
- bpo-32222 [https://bugs.python.org/issue?@action=redirect&bpo=32222]: Fix pygettext not extracting docstrings for functions with type annotated arguments. Patch by Toby Harradine.
- bpo-31583 [https://bugs.python.org/issue?@action=redirect&bpo=31583]: Fix 2to3 for using with –add-suffix option but without –output-dir option for relative path to files in current directory.

API C

- bpo-35713 [https://bugs.python.org/issue?@action=redirect&bpo=35713]: The **`PyByteArray_Init()`** and **`PyByteArray_Fini()`** functions have been removed. They did nothing since Python 2.7.4 and Python 3.2.0, were excluded from the limited API (stable ABI), and were not documented.
- bpo-33817 [https://bugs.python.org/issue?@action=redirect&bpo=33817]: Fixed **`_PyBytes_Resize()`** for empty bytes objects.
- bpo-35322 [https://bugs.python.org/issue?@action=redirect&bpo=35322]: Fix memory leak in **`PyUnicode_EncodeLocale()`** and **`PyUnicode_EncodeFSDefault()`** on error handling.
- bpo-35059 [https://bugs.python.org/issue?@action=redirect&bpo=35059]: The following C macros have been converted to static inline functions: **`Py_INCREF()`**, **`Py_DECREF()`**, **`Py_XINCREF()`**, **`Py_XDECREF()`**, **`PyObject_INIT()`**, **`PyObject_INIT_VAR()`**.
- bpo-35296 [https://bugs.python.org/issue?@action=redirect&bpo=35296]: `make install` now also installs the internal API: `Include/internal/*.h` header files.

- bpo-35081 [https://bugs.python.org/issue?@action=redirect&bpo=35081]: Internal APIs surrounded by `#ifdef Py_BUILD_CORE` have been moved from `Include/*.h` headers to new header files `Include/internal/pycore_*.h`.
- bpo-35259 [https://bugs.python.org/issue?@action=redirect&bpo=35259]: Conditionally declare **`Py_FinalizeEx()`** (new in 3.6) based on Py_LIMITED_API. Patch by Arthur Neufeld.
- bpo-35081 [https://bugs.python.org/issue?@action=redirect&bpo=35081]: The **`_PyObject_GC_TRACK()`** and **`_PyObject_GC_UNTRACK()`** macros have been removed from the public C API.
- bpo-35134 [https://bugs.python.org/issue?@action=redirect&bpo=35134]: Creation of a new `Include/cpython/` subdirectory.
- bpo-34725 [https://bugs.python.org/issue?@action=redirect&bpo=34725]: Adds _Py_SetProgramFullPath so embedders may override sys.executable
- bpo-34910 [https://bugs.python.org/issue?@action=redirect&bpo=34910]: Ensure that **`PyObject_Print()`** always returns `-1` on error. Patch by Zackery Spytz.
- bpo-34523 [https://bugs.python.org/issue?@action=redirect&bpo=34523]: Py_DecodeLocale() and Py_EncodeLocale() now use the UTF-8 encoding on Windows if Py_LegacyWindowsFSEncodingFlag is zero.
- bpo-34193 [https://bugs.python.org/issue?@action=redirect&bpo=34193]: Fix pluralization in TypeError messages in getargs.c and typeobject.c: ‘1 argument’ instead of ‘1 arguments’ and ‘1 element’ instead of ‘1 elements’.
- bpo-34127 [https://bugs.python.org/issue?@action=redirect&bpo=34127]: Return grammatically correct error message based on argument count. Patch by Karthikeyan Singaravelan.
- bpo-23927 [https://bugs.python.org/issue?@action=redirect&bpo=23927]: Fixed **`SystemError`** in **`PyArg_ParseTupleAndKeywords()`** when the `w*` format unit is used for optional parameter.
- bpo-32455 [https://bugs.python.org/issue?@action=redirect&bpo=32455]: Added **`PyCompile_OpcodeStackEffectWithJump()`**.
- bpo-34008 [https://bugs.python.org/issue?@action=redirect&bpo=34008]: Py_Main() can again be called after Py_Initialize(), as in Python 3.6.
- bpo-32500 [https://bugs.python.org/issue?@action=redirect&bpo=32500]: Fixed error messages for **`PySequence_Size()`**, **`PySequence_GetItem()`**,

`PySequence_SetItem()` and `PySequence_DelItem()` called with a mapping and `PyMapping_Size()` called with a sequence.

- bpo-33818 [https://bugs.python.org/issue?@action=redirect&bpo=33818]: `PyExceptionClass_Name()` will now return `const char *` instead of `char *`.
- bpo-33042 [https://bugs.python.org/issue?@action=redirect&bpo=33042]: Embedding applications may once again call PySys_ResetWarnOptions, PySys_AddWarnOption, and PySys_AddXOption prior to calling Py_Initialize.
- bpo-32374 [https://bugs.python.org/issue?@action=redirect&bpo=32374]: Document that m_traverse for multi-phase initialized modules can be called with m_state=NULL, and add a sanity check
- bpo-30863 [https://bugs.python.org/issue?@action=redirect&bpo=30863]: `PyUnicode_AsWideChar()` and `PyUnicode_AsWideCharString()` no longer cache the `wchar_t*` representation of string objects.

Python 3.7.0 final

Data de lançamento: 2018-06-27

Biblioteca

- bpo-33851 [https://bugs.python.org/issue?@action=redirect&bpo=33851]: Fix `ast.get_docstring()` for a node that lacks a docstring.

API C

- bpo-33932 [https://bugs.python.org/issue?@action=redirect&bpo=33932]: Calling Py_Initialize() twice does nothing, instead of failing with a fatal error: restore the Python 3.6 behaviour.

Python 3.7.0 release candidate 1

Data de lançamento: 2018-06-12

Núcleo e embutidos

- bpo-33803 [https://bugs.python.org/issue?@action=redirect&bpo=33803]: Fix a crash in hamt.c caused by enabling GC tracking for an object that hadn't all of its fields set to NULL.
- bpo-33706 [https://bugs.python.org/issue?@action=redirect&bpo=33706]: Fix a crash in Python initialization when parsing the command line options. Thanks Christoph Gohlke for the bug report and the fix!
- bpo-30654 [https://bugs.python.org/issue?@action=redirect&bpo=30654]: Fixed reset of the SIGINT handler to SIG_DFL on interpreter shutdown even when there was a custom handler set previously. Patch by Philipp Kerling.
- bpo-31849 [https://bugs.python.org/issue?@action=redirect&bpo=31849]: Fix signed/unsigned comparison warning in pyhash.c.

Biblioteca

- bpo-30167 [https://bugs.python.org/issue?@action=redirect&bpo=30167]: Prevent site.main() exception if PYTHONSTARTUP is set. Patch by Steve Weber.
- bpo-33812 [https://bugs.python.org/issue?@action=redirect&bpo=33812]: Datetime instance d with non-None tzinfo, but with d.tzinfo.utcoffset(d) returning None is now treated as naive by the astimezone() method.
- bpo-30805 [https://bugs.python.org/issue?@action=redirect&bpo=30805]: Avoid race condition with debug logging
- bpo-33694 [https://bugs.python.org/issue?@action=redirect&bpo=33694]: asyncio: Fix a race condition causing data loss on pause_reading()/resume_reading() when using the ProactorEventLoop.
- bpo-32493 [https://bugs.python.org/issue?@action=redirect&bpo=32493]: Correct test for `uuid_enc_be` availability in `configure.ac`. Patch by Michael Felt.
- bpo-33792 [https://bugs.python.org/issue?@action=redirect&bpo=33792]: Add asyncio.WindowsSelectorEventLoopPolicy and asyncio.WindowsProactorEventLoopPolicy.
- bpo-33778 [https://bugs.python.org/issue?@action=redirect&bpo=33778]: Update `unicodedata`'s database to Unicode version 11.0.0.
- bpo-33770 [https://bugs.python.org/issue?@action=redirect&bpo=33770]: improve base64 exception message for encoded inputs of invalid length
- bpo-33769 [https://bugs.python.org/issue?@action=redirect&bpo=33769]: asyncio/start_tls: Fix error message; cancel callbacks in case of an

unhandled error; mark SSLTransport as closed if it is aborted.

- bpo-33767 [https://bugs.python.org/issue?@action=redirect&bpo=33767]: The concatenation (+) and repetition (*) sequence operations now raise `TypeError` instead of `SystemError` when performed on `mmap.mmap` objects. Patch by Zackery Spytz.
- bpo-33734 [https://bugs.python.org/issue?@action=redirect&bpo=33734]: asyncio/ssl: Fix AttributeError, increase default handshake timeout
- bpo-11874 [https://bugs.python.org/issue?@action=redirect&bpo=11874]: Use a better regex when breaking usage into wrappable parts. Avoids bogus assertion errors from custom metavar strings.
- bpo-33582 [https://bugs.python.org/issue?@action=redirect&bpo=33582]: Emit a deprecation warning for inspect.formatargspec

Documentação

- bpo-33409 [https://bugs.python.org/issue?@action=redirect&bpo=33409]: Clarified the relationship between **PEP 538** [https://peps.python.org/pep-0538/]'s PYTHONCOERCECLOCALE and PEP 540's PYTHONUTF8 mode.
- bpo-33736 [https://bugs.python.org/issue?@action=redirect&bpo=33736]: Improve the documentation of `asyncio.open_connection()`, `asyncio.start_server()` and their UNIX socket counterparts.
- bpo-31432 [https://bugs.python.org/issue?@action=redirect&bpo=31432]: Clarify meaning of CERT_NONE, CERT_OPTIONAL, and CERT_REQUIRED flags for ssl.SSLContext.verify_mode.

Construção

- bpo-5755 [https://bugs.python.org/issue?@action=redirect&bpo=5755]: Move `-Wstrict-prototypes` option to `CFLAGS_NODIST` from `OPT`. This option emitted annoying warnings when building extension modules written in C++.

Windows

- bpo-33720 [https://bugs.python.org/issue?@action=redirect&bpo=33720]: Reduces maximum marshal recursion depth on release builds.

IDLE

- bpo-33656 [https://bugs.python.org/issue?@action=redirect&bpo=33656]: On Windows, add API call saying that tk scales for DPI. On Windows 8.1+ or 10, with DPI compatibility properties of the Python binary unchanged, and a monitor resolution greater than 96 DPI, this should make text and lines sharper. It should otherwise have no effect.
- bpo-33768 [https://bugs.python.org/issue?@action=redirect&bpo=33768]: Clicking on a context line moves that line to the top of the editor window.
- bpo-33763 [https://bugs.python.org/issue?@action=redirect&bpo=33763]: IDLE: Use read-only text widget for code context instead of label widget.
- bpo-33664 [https://bugs.python.org/issue?@action=redirect&bpo=33664]: Scroll IDLE editor text by lines. Previously, the mouse wheel and scrollbar slider moved text by a fixed number of pixels, resulting in partial lines at the top of the editor box. The change also applies to the shell and grep output windows, but not to read-only text views.
- bpo-33679 [https://bugs.python.org/issue?@action=redirect&bpo=33679]: Enable theme-specific color configuration for Code Context. Use the Highlights tab to see the setting for built-in themes or add settings to custom themes.
- bpo-33642 [https://bugs.python.org/issue?@action=redirect&bpo=33642]: Display up to maxlines non-blank lines for Code Context. If there is no current context, show a single blank line.

Python 3.7.0 beta 5

Data de lançamento: 2018-05-30

Núcleo e embutidos

- bpo-33622 [https://bugs.python.org/issue?@action=redirect&bpo=33622]: Fixed a leak when the garbage collector fails to add an object with the `__del__` method or referenced by it into the `gc.garbage` list. `PyGC_Collect()` can now be called when an exception is set and preserves it.
- bpo-33509 [https://bugs.python.org/issue?@action=redirect&bpo=33509]: Fix module_globals parameter of warnings.warn_explicit(): don’t crash if

module_globals is not a dict.

- bpo-20104 [https://bugs.python.org/issue?@action=redirect&bpo=20104]: The new `os.posix_spawn` added in 3.7.0b1 was removed as we are still working on what the API should look like. Expect this in 3.8 instead.
- bpo-33475 [https://bugs.python.org/issue?@action=redirect&bpo=33475]: Fixed miscellaneous bugs in converting annotations to strings and optimized parentheses in the string representation.
- bpo-33391 [https://bugs.python.org/issue?@action=redirect&bpo=33391]: Fix a leak in set_symmetric_difference().
- bpo-28055 [https://bugs.python.org/issue?@action=redirect&bpo=28055]: Fix unaligned accesses in siphash24(). Patch by Rolf Eike Beer.
- bpo-32911 [https://bugs.python.org/issue?@action=redirect&bpo=32911]: Due to unexpected compatibility issues discovered during downstream beta testing, reverted bpo-29463 [https://bugs.python.org/issue?@action=redirect&bpo=29463]. `docstring` field is removed from Module, ClassDef, FunctionDef, and AsyncFunctionDef ast nodes which was added in 3.7a1. Docstring expression is restored as a first statement in their body. Based on patch by Inada Naoki.
- bpo-21983 [https://bugs.python.org/issue?@action=redirect&bpo=21983]: Fix a crash in `ctypes.cast()` in case the type argument is a ctypes structured data type. Patch by Eryk Sun and Oren Milman.

Biblioteca

- bpo-32751 [https://bugs.python.org/issue?@action=redirect&bpo=32751]: When cancelling the task due to a timeout, **`asyncio.wait_for()`** will now wait until the cancellation is complete.
- bpo-32684 [https://bugs.python.org/issue?@action=redirect&bpo=32684]: Fix gather to propagate cancellation of itself even with return_exceptions.
- bpo-33654 [https://bugs.python.org/issue?@action=redirect&bpo=33654]: Support protocol type switching in SSLTransport.set_protocol().
- bpo-33674 [https://bugs.python.org/issue?@action=redirect&bpo=33674]: Pause the transport as early as possible to further reduce the risk of data_received() being called before connection_made().
- bpo-33674 [https://bugs.python.org/issue?@action=redirect&bpo=33674]: Fix a race condition in SSLProtocol.connection_made() of asyncio.sslproto: start immediately the handshake instead of using call_soon(). Previously,

data_received() could be called before the handshake started, causing the handshake to hang or fail.

- bpo-31647 [https://bugs.python.org/issue?@action=redirect&bpo=31647]: Fixed bug where calling write_eof() on a _SelectorSocketTransport after it's already closed raises AttributeError.
- bpo-32610 [https://bugs.python.org/issue?@action=redirect&bpo=32610]: Make asyncio.all_tasks() return only pending tasks.
- bpo-32410 [https://bugs.python.org/issue?@action=redirect&bpo=32410]: Avoid blocking on file IO in sendfile fallback code
- bpo-33469 [https://bugs.python.org/issue?@action=redirect&bpo=33469]: Fix RuntimeError after closing loop that used run_in_executor
- bpo-33672 [https://bugs.python.org/issue?@action=redirect&bpo=33672]: Fix Task.__repr__ crash with Cython's bogus coroutines
- bpo-33654 [https://bugs.python.org/issue?@action=redirect&bpo=33654]: Fix transport.set_protocol() to support switching between asyncio.Protocol and asyncio.BufferedProtocol. Fix loop.start_tls() to work with asyncio.BufferedProtocols.
- bpo-33652 [https://bugs.python.org/issue?@action=redirect&bpo=33652]: Pickles of type variables and subscripted generics are now future-proof and compatible with older Python versions.
- bpo-32493 [https://bugs.python.org/issue?@action=redirect&bpo=32493]: Fixed `uuid.uuid1()` on FreeBSD.
- bpo-33618 [https://bugs.python.org/issue?@action=redirect&bpo=33618]: Finalize and document preliminary and experimental TLS 1.3 support with OpenSSL 1.1.1
- bpo-33623 [https://bugs.python.org/issue?@action=redirect&bpo=33623]: Fix possible SIGSGV when asyncio.Future is created in __del__
- bpo-30877 [https://bugs.python.org/issue?@action=redirect&bpo=30877]: Fixed a bug in the Python implementation of the JSON decoder that prevented the cache of parsed strings from clearing after finishing the decoding. Based on patch by c-fos.
- bpo-33570 [https://bugs.python.org/issue?@action=redirect&bpo=33570]: Change TLS 1.3 cipher suite settings for compatibility with OpenSSL 1.1.1-pre6 and newer. OpenSSL 1.1.1 will have TLS 1.3 ciphers enabled by default.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Do not simplify arguments to `typing.Union`. Now `Union[Manager,`

`Employee]` is not simplified to `Employee` at runtime. Such simplification previously caused several bugs and limited possibilities for introspection.

- bpo-33540 [https://bugs.python.org/issue?@action=redirect&bpo=33540]: Add a new `block_on_close` class attribute to `ForkingMixIn` and `ThreadingMixIn` classes of **`socketserver`**.
- bpo-33548 [https://bugs.python.org/issue?@action=redirect&bpo=33548]: tempfile._candidate_tempdir_list should consider common TEMP locations
- bpo-33109 [https://bugs.python.org/issue?@action=redirect&bpo=33109]: argparse subparsers are once again not required by default, reverting the change in behavior introduced by bpo-26510 [https://bugs.python.org/issue?@action=redirect&bpo=26510] in 3.7.0a2.
- bpo-33536 [https://bugs.python.org/issue?@action=redirect&bpo=33536]: dataclasses.make_dataclass now checks for invalid field names and duplicate fields. Also, added a check for invalid field specifications.
- bpo-33542 [https://bugs.python.org/issue?@action=redirect&bpo=33542]: Prevent `uuid.get_node` from using a DUID instead of a MAC on Windows. Patch by Zvi Effron
- bpo-26819 [https://bugs.python.org/issue?@action=redirect&bpo=26819]: Fix race condition with `ReadTransport.resume_reading` in Windows proactor event loop.
- Fix failure in `typing.get_type_hints()` when ClassVar was provided as a string forward reference.
- bpo-33505 [https://bugs.python.org/issue?@action=redirect&bpo=33505]: Optimize asyncio.ensure_future() by reordering if checks: 1.17x faster.
- bpo-33497 [https://bugs.python.org/issue?@action=redirect&bpo=33497]: Add errors param to cgi.parse_multipart and make an encoding in FieldStorage use the given errors (needed for Twisted). Patch by Amber Brown.
- bpo-33495 [https://bugs.python.org/issue?@action=redirect&bpo=33495]: Change dataclasses.Fields repr to use the repr of each of its members, instead of str. This makes it more clear what each field actually represents. This is especially true for the ‘type’ member.
- bpo-33453 [https://bugs.python.org/issue?@action=redirect&bpo=33453]: Fix dataclasses to work if using literal string type annotations or if using PEP 563 “Postponed Evaluation of Annotations”. Only specific string

prefixes are detected for both ClassVar ("ClassVar" and "typing.ClassVar") and InitVar ("InitVar" and "dataclasses.InitVar").

- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Minor fixes in typing module: add annotations to `NamedTuple.__new__`, pass `*args` and `**kwds` in `Generic.__new__`. Original PRs by Paulius Šarka and Chad Dombrova.
- bpo-20087 [https://bugs.python.org/issue?@action=redirect&bpo=20087]: Updated alias mapping with glibc 2.27 supported locales.
- bpo-33422 [https://bugs.python.org/issue?@action=redirect&bpo=33422]: Fix trailing quotation marks getting deleted when looking up byte/string literals on pydoc. Patch by Andrés Delfino.
- bpo-28167 [https://bugs.python.org/issue?@action=redirect&bpo=28167]: The function `platform.linux_distribution` and `platform.dist` now trigger a `DeprecationWarning` and have been marked for removal in Python 3.8
- bpo-33197 [https://bugs.python.org/issue?@action=redirect&bpo=33197]: Update error message when constructing invalid inspect.Parameters Patch by Donghee Na.
- bpo-33263 [https://bugs.python.org/issue?@action=redirect&bpo=33263]: Fix FD leak in `_SelectorSocketTransport` Patch by Vlad Starostin.
- bpo-32861 [https://bugs.python.org/issue?@action=redirect&bpo=32861]: The urllib.robotparser's `__str__` representation now includes wildcard entries and the "Crawl-delay" and "Request-rate" fields. Patch by Michael Lazar.
- bpo-32257 [https://bugs.python.org/issue?@action=redirect&bpo=32257]: The ssl module now contains OP_NO_RENEGOTIATION constant, available with OpenSSL 1.1.0h or 1.1.1.
- bpo-16865 [https://bugs.python.org/issue?@action=redirect&bpo=16865]: Support arrays >=2GiB in **`ctypes`**. Patch by Segev Finer.

Documentação

- bpo-23859 [https://bugs.python.org/issue?@action=redirect&bpo=23859]: Document that `asyncio.wait()` does not cancel its futures on timeout.
- bpo-32436 [https://bugs.python.org/issue?@action=redirect&bpo=32436]: Document **PEP 567** [https://peps.python.org/pep-0567/] changes to asyncio.

- bpo-33604 [https://bugs.python.org/issue?@action=redirect&bpo=33604]: Update HMAC md5 default to a DeprecationWarning, bump removal to 3.8.
- bpo-33503 [https://bugs.python.org/issue?@action=redirect&bpo=33503]: Fix broken pypi link
- bpo-33421 [https://bugs.python.org/issue?@action=redirect&bpo=33421]: Add missing documentation for `typing.AsyncContextManager`.

Testes

- bpo-33655 [https://bugs.python.org/issue?@action=redirect&bpo=33655]: Ignore test_posix_fallocate failures on BSD platforms that might be due to running on ZFS.
- bpo-32604 [https://bugs.python.org/issue?@action=redirect&bpo=32604]: Remove the _xxsubinterpreters module (meant for testing) and associated helpers. This module was originally added recently in 3.7b1.

Construção

- bpo-33614 [https://bugs.python.org/issue?@action=redirect&bpo=33614]: Ensures module definition files for the stable ABI on Windows are correctly regenerated.
- bpo-33522 [https://bugs.python.org/issue?@action=redirect&bpo=33522]: Enable CI builds on Visual Studio Team Services at https://python.visualstudio.com/cpython
- bpo-33012 [https://bugs.python.org/issue?@action=redirect&bpo=33012]: Add `-Wno-cast-function-type` for gcc 8 for silencing warnings about function casts like casting to PyCFunction in method definition lists.

macOS

- bpo-13631 [https://bugs.python.org/issue?@action=redirect&bpo=13631]: The .editrc file in user's home directory is now processed correctly during the readline initialization through editline emulation on macOS.

IDLE

- bpo-33628 [https://bugs.python.org/issue?@action=redirect&bpo=33628]: IDLE: Cleanup codecontext.py and its test.
- bpo-33564 [https://bugs.python.org/issue?@action=redirect&bpo=33564]: IDLE's code context now recognizes async as a block opener.
- bpo-32831 [https://bugs.python.org/issue?@action=redirect&bpo=32831]: Add docstrings and tests for codecontext.

Python 3.7.0 beta 4

Data de lançamento: 2018-05-02

Núcleo e embutidos

- bpo-33363 [https://bugs.python.org/issue?@action=redirect&bpo=33363]: Raise a SyntaxError for `async with` and `async for` statements outside of async functions.
- bpo-33128 [https://bugs.python.org/issue?@action=redirect&bpo=33128]: Fix a bug that causes PathFinder to appear twice on sys.meta_path. Patch by Pablo Galindo Salgado.
- bpo-33312 [https://bugs.python.org/issue?@action=redirect&bpo=33312]: Fixed clang ubsan (undefined behavior sanitizer) warnings in dictobject.c by adjusting how the internal struct _dictkeysobject shared keys structure is declared.
- bpo-33231 [https://bugs.python.org/issue?@action=redirect&bpo=33231]: Fix potential memory leak in `normalizestring()`.
- bpo-33205 [https://bugs.python.org/issue?@action=redirect&bpo=33205]: Change dict growth function from `round_up_to_power_2(used*2+hashtable_size/2)` to `round_up_to_power_2(used*3)`. Previously, dict is shrunk only when `used == 0`. Now dict has more chance to be shrunk.
- bpo-29922 [https://bugs.python.org/issue?@action=redirect&bpo=29922]: Improved error messages in 'async with' when `__aenter__()` or `__aexit__()` return non-awaitable object.
- bpo-33199 [https://bugs.python.org/issue?@action=redirect&bpo=33199]: Fix `ma_version_tag` in dict implementation is uninitialized when copying from key-sharing dict.

Biblioteca

- bpo-33281 [https://bugs.python.org/issue?@action=redirect&bpo=33281]: Fix ctypes.util.find_library regression on macOS.
- bpo-33383 [https://bugs.python.org/issue?@action=redirect&bpo=33383]: Fixed crash in the get() method of the `dbm.ndbm` database object when it is called with a single argument.
- bpo-33329 [https://bugs.python.org/issue?@action=redirect&bpo=33329]: Fix multiprocessing regression on newer glibcs
- bpo-991266 [https://bugs.python.org/issue?@action=redirect&bpo=991266]: Fix quoting of the `Comment` attribute of `http.cookies.SimpleCookie`.
- bpo-33131 [https://bugs.python.org/issue?@action=redirect&bpo=33131]: Upgrade bundled version of pip to 10.0.1.
- bpo-33308 [https://bugs.python.org/issue?@action=redirect&bpo=33308]: Fixed a crash in the **`parser`** module when converting an ST object to a tree of tuples or lists with `line_info=False` and `col_info=True`.
- bpo-33266 [https://bugs.python.org/issue?@action=redirect&bpo=33266]: lib2to3 now recognizes `rf'...'` strings.
- bpo-11594 [https://bugs.python.org/issue?@action=redirect&bpo=11594]: Ensure line-endings are respected when using lib2to3.
- bpo-33254 [https://bugs.python.org/issue?@action=redirect&bpo=33254]: Have `importlib.resources.contents()` and **`importlib.abc.ResourceReader.contents()`** return an iterable instead of an iterator.
- bpo-33256 [https://bugs.python.org/issue?@action=redirect&bpo=33256]: Fix display of `<module>` call in the html produced by `cgitb.html()`. Patch by Stéphane Blondon.
- bpo-33185 [https://bugs.python.org/issue?@action=redirect&bpo=33185]: Fixed regression when running pydoc with the `-m` switch. (The regression was introduced in 3.7.0b3 by the resolution of bpo-33053 [https://bugs.python.org/issue?@action=redirect&bpo=33053]) This fix also changed pydoc to add `os.getcwd()` to `sys.path` when necessary, rather than adding `"."`.
- bpo-33169 [https://bugs.python.org/issue?@action=redirect&bpo=33169]: Delete entries of `None` in `sys.path_importer_cache` when **`importlib.machinery.invalidate_caches()`** is called.

- bpo-33217 [https://bugs.python.org/issue?@action=redirect&bpo=33217]: Deprecate looking up non-Enum objects in Enum classes and Enum members (will raise `TypeError` in 3.8+).
- bpo-33203 [https://bugs.python.org/issue?@action=redirect&bpo=33203]: `random.Random.choice()` now raises `IndexError` for empty sequences consistently even when called from subclasses without a `getrandbits()` implementation.
- bpo-33224 [https://bugs.python.org/issue?@action=redirect&bpo=33224]: Update difflib.mdiff() for **PEP 479** [https://peps.python.org/pep-0479/]. Convert an uncaught StopIteration in a generator into a return-statement.
- bpo-33209 [https://bugs.python.org/issue?@action=redirect&bpo=33209]: End framing at the end of C implementation of `pickle.Pickler.dump()`.
- bpo-20104 [https://bugs.python.org/issue?@action=redirect&bpo=20104]: Improved error handling and fixed a reference leak in `os.posix_spawn()`.
- bpo-33175 [https://bugs.python.org/issue?@action=redirect&bpo=33175]: In dataclasses, Field.__set_name__ now looks up the __set_name__ special method on the class, not the instance, of the default value.
- bpo-33097 [https://bugs.python.org/issue?@action=redirect&bpo=33097]: Raise RuntimeError when `executor.submit` is called during interpreter shutdown.
- bpo-31908 [https://bugs.python.org/issue?@action=redirect&bpo=31908]: Fix output of cover files for `trace` module command-line tool. Previously emitted cover files only when `--missing` option was used. Patch by Michael Selik.

Documentação

- bpo-33378 [https://bugs.python.org/issue?@action=redirect&bpo=33378]: Add Korean language switcher for https://docs.python.org/3/
- bpo-33276 [https://bugs.python.org/issue?@action=redirect&bpo=33276]: Clarify that the `__path__` attribute on modules cannot be just any value.
- bpo-33201 [https://bugs.python.org/issue?@action=redirect&bpo=33201]: Modernize documentation for writing C extension types.
- bpo-33195 [https://bugs.python.org/issue?@action=redirect&bpo=33195]: Deprecate `Py_UNICODE` usage in `c-api/arg` document. `Py_UNICODE` related APIs are deprecated since Python 3.3, but it is missed in the document.

- bpo-8243 [https://bugs.python.org/issue?@action=redirect&bpo=8243]: Add a note about curses.addch and curses.addstr exception behavior when writing outside a window, or pad.
- bpo-32337 [https://bugs.python.org/issue?@action=redirect&bpo=32337]: Update documentation related with `dict` order.

Testes

- bpo-33358 [https://bugs.python.org/issue?@action=redirect&bpo=33358]: Fix `test_embed.test_pre_initialization_sys_options()` when the interpreter is built with `--enable-shared`.

Construção

- bpo-33394 [https://bugs.python.org/issue?@action=redirect&bpo=33394]: Enable the verbose build for extension modules, when GNU make is passed macros on the command line.
- bpo-33393 [https://bugs.python.org/issue?@action=redirect&bpo=33393]: Update config.guess and config.sub files.
- bpo-33377 [https://bugs.python.org/issue?@action=redirect&bpo=33377]: Add new triplets for mips r6 and riscv variants (used in extension suffixes).
- bpo-32232 [https://bugs.python.org/issue?@action=redirect&bpo=32232]: By default, modules configured in `Modules/Setup` are no longer built with `-DPy_BUILD_CORE`. Instead, modules that specifically need that preprocessor definition include it in their individual entries.
- bpo-33182 [https://bugs.python.org/issue?@action=redirect&bpo=33182]: The embedding tests can once again be built with clang 6.0

Windows

- bpo-33184 [https://bugs.python.org/issue?@action=redirect&bpo=33184]: Update Windows installer to use OpenSSL 1.1.0h.

macOS

- bpo-33184 [https://bugs.python.org/issue?@action=redirect&bpo=33184]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.1.0h.

IDLE

- bpo-21474 [https://bugs.python.org/issue?@action=redirect&bpo=21474]: Update word/identifier definition from ascii to unicode. In text and entry boxes, this affects selection by double-click, movement left/right by control-left/right, and deletion left/right by control-BACKSPACE/DEL.
- bpo-33204 [https://bugs.python.org/issue?@action=redirect&bpo=33204]: IDLE: consistently color invalid string prefixes. A ‘u’ string prefix cannot be paired with either ‘r’ or ‘f’. Consistently color as much of the prefix, starting at the right, as is valid. Revise and extend colorizer test.

Ferramentas/Demos

- bpo-33189 [https://bugs.python.org/issue?@action=redirect&bpo=33189]: **pygettext.py** now recognizes only literal strings as docstrings and translatable strings, and rejects bytes literals and f-string expressions.
- bpo-31920 [https://bugs.python.org/issue?@action=redirect&bpo=31920]: Fixed handling directories as arguments in the `pygettext` script. Based on patch by Oleg Krasnikov.
- bpo-29673 [https://bugs.python.org/issue?@action=redirect&bpo=29673]: Fix pystackv and pystack gdbinit macros.
- bpo-31583 [https://bugs.python.org/issue?@action=redirect&bpo=31583]: Fix 2to3 for using with –add-suffix option but without –output-dir option for relative path to files in current directory.

Python 3.7.0 beta 3

Data de lançamento: 2018-03-29

Segurança

- bpo-33136 [https://bugs.python.org/issue?@action=redirect&bpo=33136]: Harden ssl module against LibreSSL **CVE 2018-8970** [https://www.cve.org/CVERecord?id=CVE-2018-8970]. X509_VERIFY_PARAM_set1_host() is called with

an explicit namelen. A new test ensures that NULL bytes are not allowed.

- bpo-33001 [https://bugs.python.org/issue?@action=redirect&bpo=33001]: Minimal fix to prevent buffer overrun in os.symlink on Windows
- bpo-32981 [https://bugs.python.org/issue?@action=redirect&bpo=32981]: Regexes in difflib and poplib were vulnerable to catastrophic backtracking. These regexes formed potential DOS vectors (REDOS). They have been refactored. This resolves **CVE 2018-1060** [https://www.cve.org/CVERecord?id=CVE-2018-1060] and **CVE 2018-1061** [https://www.cve.org/CVERecord?id=CVE-2018-1061]. Patch by Jamie Davis.

Núcleo e embutidos

- bpo-33053 [https://bugs.python.org/issue?@action=redirect&bpo=33053]: When using the -m switch, sys.path[0] is now explicitly expanded as the *starting* working directory, rather than being left as the empty path (which allows imports from the current working directory at the time of the import)
- bpo-33018 [https://bugs.python.org/issue?@action=redirect&bpo=33018]: Improve consistency of errors raised by `issubclass()` when called with a non-class and an abstract base class as the first and second arguments, respectively. Patch by Josh Bronson.
- bpo-33041 [https://bugs.python.org/issue?@action=redirect&bpo=33041]: Fixed jumping when the function contains an `async for` loop.
- bpo-33026 [https://bugs.python.org/issue?@action=redirect&bpo=33026]: Fixed jumping out of “with” block by setting f_lineno.
- bpo-33005 [https://bugs.python.org/issue?@action=redirect&bpo=33005]: Fix a crash on fork when using a custom memory allocator (ex: using PYTHONMALLOC env var). _PyGILState_Reinit() and _PyInterpreterState_Enable() now use the default RAW memory allocator to allocate a new interpreters mutex on fork.
- bpo-17288 [https://bugs.python.org/issue?@action=redirect&bpo=17288]: Prevent jumps from ‘return’ and ‘exception’ trace events.
- bpo-32836 [https://bugs.python.org/issue?@action=redirect&bpo=32836]: Don’t use temporary variables in cases of list/dict/set comprehensions

Biblioteca

- bpo-33141 [https://bugs.python.org/issue?@action=redirect&bpo=33141]: Have Field objects pass through __set_name__ to their default values, if they have their own __set_name__.
- bpo-33096 [https://bugs.python.org/issue?@action=redirect&bpo=33096]: Allow ttk.Treeview.insert to insert iid that has a false boolean value. Note iid=0 and iid=False would be same. Patch by Garvit Khatri.
- bpo-32873 [https://bugs.python.org/issue?@action=redirect&bpo=32873]: Treat type variables and special typing forms as immutable by copy and pickle. This fixes several minor issues and inconsistencies, and improves backwards compatibility with Python 3.6.
- bpo-33134 [https://bugs.python.org/issue?@action=redirect&bpo=33134]: When computing dataclass's __hash__, use the lookup table to contain the function which returns the __hash__ value. This is an improvement over looking up a string, and then testing that string to see what to do.
- bpo-33127 [https://bugs.python.org/issue?@action=redirect&bpo=33127]: The ssl module now compiles with LibreSSL 2.7.1.
- bpo-32505 [https://bugs.python.org/issue?@action=redirect&bpo=32505]: Raise TypeError if a member variable of a dataclass is of type Field, but doesn't have a type annotation.
- bpo-33078 [https://bugs.python.org/issue?@action=redirect&bpo=33078]: Fix the failure on OSX caused by the tests relying on sem_getvalue
- bpo-33116 [https://bugs.python.org/issue?@action=redirect&bpo=33116]: Add 'Field' to dataclasses.__all__.
- bpo-32896 [https://bugs.python.org/issue?@action=redirect&bpo=32896]: Fix an error where subclassing a dataclass with a field that uses a default_factory would generate an incorrect class.
- bpo-33100 [https://bugs.python.org/issue?@action=redirect&bpo=33100]: Dataclasses: If a field has a default value that's a MemberDescriptorType, then it's from that field being in __slots__, not an actual default value.
- bpo-32953 [https://bugs.python.org/issue?@action=redirect&bpo=32953]: If a non-dataclass inherits from a frozen dataclass, allow attributes to be added to the derived class. Only attributes from the frozen dataclass cannot be assigned to. Require all dataclasses in a hierarchy to be either all frozen or all non-frozen.
- bpo-33061 [https://bugs.python.org/issue?@action=redirect&bpo=33061]: Add missing `NoReturn` to `__all__` in typing.py

- bpo-33078 [https://bugs.python.org/issue?@action=redirect&bpo=33078]: Fix the size handling in multiprocessing.Queue when a pickling error occurs.
- bpo-33064 [https://bugs.python.org/issue?@action=redirect&bpo=33064]: lib2to3 now properly supports trailing commas after `*args` and `**kwargs` in function signatures.
- bpo-33056 [https://bugs.python.org/issue?@action=redirect&bpo=33056]: FIX properly close leaking fds in concurrent.futures.ProcessPoolExecutor.
- bpo-33021 [https://bugs.python.org/issue?@action=redirect&bpo=33021]: Release the GIL during fstat() calls, avoiding hang of all threads when calling mmap.mmap(), os.urandom(), and random.seed(). Patch by Nir Soffer.
- bpo-31804 [https://bugs.python.org/issue?@action=redirect&bpo=31804]: Avoid failing in multiprocessing.Process if the standard streams are closed or None at exit.
- bpo-33037 [https://bugs.python.org/issue?@action=redirect&bpo=33037]: Skip sending/receiving data after SSL transport closing.
- bpo-27683 [https://bugs.python.org/issue?@action=redirect&bpo=27683]: Fix a regression in **`ipaddress`** that result of **`hosts()`** is empty when the network is constructed by a tuple containing an integer mask and only 1 bit left for addresses.
- bpo-32999 [https://bugs.python.org/issue?@action=redirect&bpo=32999]: Fix C implementation of `ABC.__subclasscheck__(cls, subclass)` crashed when `subclass` is not a type object.
- bpo-33009 [https://bugs.python.org/issue?@action=redirect&bpo=33009]: Fix inspect.signature() for single-parameter partialmethods.
- bpo-32969 [https://bugs.python.org/issue?@action=redirect&bpo=32969]: Expose several missing constants in zlib and fix corresponding documentation.
- bpo-32056 [https://bugs.python.org/issue?@action=redirect&bpo=32056]: Improved exceptions raised for invalid number of channels and sample width when read an audio file in modules **`aifc`**, **`wave`** and **`sunau`**.
- bpo-32844 [https://bugs.python.org/issue?@action=redirect&bpo=32844]: Fix wrong redirection of a low descriptor (0 or 1) to stderr in subprocess if another low descriptor is closed.
- bpo-32857 [https://bugs.python.org/issue?@action=redirect&bpo=32857]: In **`tkinter`**, `after_cancel(None)` now raises a **`ValueError`** instead of canceling the first scheduled function. Patch by Cheryl Sabella.
- bpo-31639 [https://bugs.python.org/issue?@action=redirect&bpo=31639]: http.server now exposes a ThreadedHTTPServer class and uses it when the

module is run with `-m` to cope with web browsers pre-opening sockets.

- bpo-27645 [https://bugs.python.org/issue?@action=redirect&bpo=27645]: **`sqlite3.Connection`** now exposes a **`backup`** method, if the underlying SQLite library is at version 3.6.11 or higher. Patch by Lele Gaifax.

Documentação

- bpo-33126 [https://bugs.python.org/issue?@action=redirect&bpo=33126]: Document PyBuffer_ToContiguous().
- bpo-27212 [https://bugs.python.org/issue?@action=redirect&bpo=27212]: Modify documentation for the **`islice()`** recipe to consume initial values up to the start index.
- bpo-28247 [https://bugs.python.org/issue?@action=redirect&bpo=28247]: Update **`zipapp`** documentation to describe how to make standalone applications.
- bpo-18802 [https://bugs.python.org/issue?@action=redirect&bpo=18802]: Documentation changes for ipaddress. Patch by Jon Foster and Berker Peksag.
- bpo-27428 [https://bugs.python.org/issue?@action=redirect&bpo=27428]: Update documentation to clarify that `WindowsRegistryFinder` implements `MetaPathFinder`. (Patch by Himanshu Lakhara)

Testes

- bpo-32872 [https://bugs.python.org/issue?@action=redirect&bpo=32872]: Avoid regrtest compatibility issue with namespace packages.
- bpo-32517 [https://bugs.python.org/issue?@action=redirect&bpo=32517]: Fix failing `test_asyncio` on macOS 10.12.2+ due to transport of `KqueueSelector` loop was not being closed.
- bpo-19417 [https://bugs.python.org/issue?@action=redirect&bpo=19417]: Add test_bdb.py.

Construção

- bpo-33163 [https://bugs.python.org/issue?@action=redirect&bpo=33163]: Upgrade pip to 9.0.3 and setuptools to v39.0.1.

Windows

- bpo-33016 [https://bugs.python.org/issue?@action=redirect&bpo=33016]: Fix potential use of uninitialized memory in nt._getfinalpathname
- bpo-32903 [https://bugs.python.org/issue?@action=redirect&bpo=32903]: Fix a memory leak in os.chdir() on Windows if the current directory is set to a UNC path.

macOS

- bpo-32726 [https://bugs.python.org/issue?@action=redirect&bpo=32726]: Build and link with private copy of Tcl/Tk 8.6 for the macOS 10.6+ installer. The 10.9+ installer variant already does this. This means that the Python 3.7 provided by the python.org macOS installers no longer need or use any external versions of Tcl/Tk, either system-provided or user-installed, such as ActiveTcl.

IDLE

- bpo-32984 [https://bugs.python.org/issue?@action=redirect&bpo=32984]: Set `__file__` while running a startup file. Like Python, IDLE optionally runs one startup file in the Shell window before presenting the first interactive input prompt. For IDLE, `-s` runs a file named in environmental variable **`IDLESTARTUP`** or **`PYTHONSTARTUP`**; `-r file` runs `file`. Python sets `__file__` to the startup file name before running the file and unsets it before the first prompt. IDLE now does the same when run normally, without the `-n` option.
- bpo-32940 [https://bugs.python.org/issue?@action=redirect&bpo=32940]: Simplify and rename StringTranslatePseudoMapping in pyparse.

Ferramentas/Demos

- bpo-32885 [https://bugs.python.org/issue?@action=redirect&bpo=32885]: Add an `-n` flag for `Tools/scripts/pathfix.py` to disable automatic backup creation (files with ~ suffix).

API C

- bpo-33042 [https://bugs.python.org/issue?@action=redirect&bpo=33042]: Embedding applications may once again call PySys_ResetWarnOptions, PySys_AddWarnOption, and PySys_AddXOption prior to calling Py_Initialize.
- bpo-32374 [https://bugs.python.org/issue?@action=redirect&bpo=32374]: Document that m_traverse for multi-phase initialized modules can be called with m_state=NULL, and add a sanity check

Python 3.7.0 beta 2

Data de lançamento: 2018-02-27

Segurança

- bpo-28414 [https://bugs.python.org/issue?@action=redirect&bpo=28414]: The ssl module now allows users to perform their own IDN en/decoding when using SNI.

Núcleo e embutidos

- bpo-32889 [https://bugs.python.org/issue?@action=redirect&bpo=32889]: Update Valgrind suppression list to account for the rename of `Py_ADDRESS_IN_RANG` to `address_in_range`.
- bpo-31356 [https://bugs.python.org/issue?@action=redirect&bpo=31356]: Remove the new API added in bpo-31356 [https://bugs.python.org/issue?@action=redirect&bpo=31356] (gc.ensure_disabled() context manager).
- bpo-32305 [https://bugs.python.org/issue?@action=redirect&bpo=32305]: For namespace packages, ensure that both `__file__` and `__spec__.origin` are set to None.
- bpo-32303 [https://bugs.python.org/issue?@action=redirect&bpo=32303]: Make sure `__spec__.loader` matches `__loader__` for namespace packages.
- bpo-32711 [https://bugs.python.org/issue?@action=redirect&bpo=32711]: Fix the warning messages for Python/ast_unparse.c. Patch by Stéphane Wirtel

- bpo-32583 [https://bugs.python.org/issue?@action=redirect&bpo=32583]: Fix possible crashing in builtin Unicode decoders caused by write out-of-bound errors when using customized decode error handlers.

Biblioteca

- bpo-32960 [https://bugs.python.org/issue?@action=redirect&bpo=32960]: For dataclasses, disallow inheriting frozen from non-frozen classes, and also disallow inheriting non-frozen from frozen classes. This restriction will be relaxed at a future date.
- bpo-32713 [https://bugs.python.org/issue?@action=redirect&bpo=32713]: Fixed tarfile.itn handling of out-of-bounds float values. Patch by Joffrey Fuhrer.
- bpo-32951 [https://bugs.python.org/issue?@action=redirect&bpo=32951]: Direct instantiation of SSLSocket and SSLObject objects is now prohibited. The constructors were never documented, tested, or designed as public constructors. Users were suppose to use ssl.wrap_socket() or SSLContext.
- bpo-32929 [https://bugs.python.org/issue?@action=redirect&bpo=32929]: Remove the tri-state parameter “hash”, and add the boolean “unsafe_hash”. If unsafe_hash is True, add a __hash__ function, but if a __hash__ exists, raise TypeError. If unsafe_hash is False, add a __hash__ based on the values of eq= and frozen=. The unsafe_hash=False behavior is the same as the old hash=None behavior. unsafe_hash=False is the default, just as hash=None used to be.
- bpo-32947 [https://bugs.python.org/issue?@action=redirect&bpo=32947]: Add OP_ENABLE_MIDDLEBOX_COMPAT and test workaround for TLSv1.3 for future compatibility with OpenSSL 1.1.1.
- bpo-30622 [https://bugs.python.org/issue?@action=redirect&bpo=30622]: The ssl module now detects missing NPN support in LibreSSL.
- bpo-32922 [https://bugs.python.org/issue?@action=redirect&bpo=32922]: dbm.open() now encodes filename with the filesystem encoding rather than default encoding.
- bpo-32859 [https://bugs.python.org/issue?@action=redirect&bpo=32859]: In `os.dup2`, don’t check every call whether the `dup3` syscall exists or not.
- bpo-32556 [https://bugs.python.org/issue?@action=redirect&bpo=32556]: nt._getfinalpathname, nt._getvolumepathname and nt._getdiskusage

now correctly convert from bytes.

- bpo-25988 [https://bugs.python.org/issue?@action=redirect&bpo=25988]: Emit a `DeprecationWarning` when using or importing an ABC directly from `collections` rather than from `collections.abc`.
- bpo-21060 [https://bugs.python.org/issue?@action=redirect&bpo=21060]: Rewrite confusing message from setup.py upload from “No dist file created in earlier command” to the more helpful “Must create and upload files in one command”.
- bpo-32852 [https://bugs.python.org/issue?@action=redirect&bpo=32852]: Make sure sys.argv remains as a list when running trace.
- bpo-31333 [https://bugs.python.org/issue?@action=redirect&bpo=31333]: `_abc` module is added. It is a speedup module with C implementations for various functions and methods in `abc`. Creating an ABC subclass and calling `isinstance` or `issubclass` with an ABC subclass are up to 1.5x faster. In addition, this makes Python start-up up to 10% faster. Note that the new implementation hides internal registry and caches, previously accessible via private attributes `_abc_registry`, `_abc_cache`, and `_abc_negative_cache`. There are three debugging helper methods that can be used instead `_dump_registry`, `_abc_registry_clear`, and `_abc_caches_clear`.
- bpo-32841 [https://bugs.python.org/issue?@action=redirect&bpo=32841]: Fixed `asyncio.Condition` issue which silently ignored cancellation after notifying and cancelling a conditional lock. Patch by Bar Harel.
- bpo-32819 [https://bugs.python.org/issue?@action=redirect&bpo=32819]: ssl.match_hostname() has been simplified and no longer depends on re and ipaddress module for wildcard and IP addresses. Error reporting for invalid wildcards has been improved.
- bpo-32394 [https://bugs.python.org/issue?@action=redirect&bpo=32394]: socket: Remove TCP_FASTOPEN,TCP_KEEPCNT,TCP_KEEPIDLE,TCP_KEEPINTVL flags on older version Windows during run-time.
- bpo-31787 [https://bugs.python.org/issue?@action=redirect&bpo=31787]: Fixed refleaks of `__init__()` methods in various modules. (Contributed by Oren Milman)
- bpo-30157 [https://bugs.python.org/issue?@action=redirect&bpo=30157]: Fixed guessing quote and delimiter in csv.Sniffer.sniff() when only the last field is quoted. Patch by Jake Davis.

- bpo-32792 [https://bugs.python.org/issue?@action=redirect&bpo=32792]: collections.ChainMap() preserves the order of the underlying mappings.
- bpo-32775 [https://bugs.python.org/issue?@action=redirect&bpo=32775]: **`fnmatch.translate()`** no longer produces patterns which contain set operations. Sets starting with '[' or containing '–', '&&', '~~' or '||' will be interpreted differently in regular expressions in future versions. Currently they emit warnings. fnmatch.translate() now avoids producing patterns containing such sets by accident.
- bpo-32622 [https://bugs.python.org/issue?@action=redirect&bpo=32622]: Implement native fast sendfile for Windows proactor event loop.
- bpo-32777 [https://bugs.python.org/issue?@action=redirect&bpo=32777]: Fix a rare but potential pre-exec child process deadlock in subprocess on POSIX systems when marking file descriptors inheritable on exec in the child process. This bug appears to have been introduced in 3.4.
- bpo-32647 [https://bugs.python.org/issue?@action=redirect&bpo=32647]: The ctypes module used to depend on indirect linking for dlopen. The shared extension is now explicitly linked against libdl on platforms with dl.
- bpo-32741 [https://bugs.python.org/issue?@action=redirect&bpo=32741]: Implement `asyncio.TimerHandle.when()` method.
- bpo-32691 [https://bugs.python.org/issue?@action=redirect&bpo=32691]: Use mod_spec.parent when running modules with pdb
- bpo-32734 [https://bugs.python.org/issue?@action=redirect&bpo=32734]: Fixed `asyncio.Lock()` safety issue which allowed acquiring and locking the same lock multiple times, without it being free. Patch by Bar Harel.
- bpo-32727 [https://bugs.python.org/issue?@action=redirect&bpo=32727]: Do not include name field in SMTP envelope from address. Patch by Stéphane Wirtel
- bpo-31453 [https://bugs.python.org/issue?@action=redirect&bpo=31453]: Add TLSVersion constants and SSLContext.maximum_version / minimum_version attributes. The new API wraps OpenSSL 1.1 https://web.archive.org/web/20180309043602/https://www.openssl.org/docs/man1.1.0/ssl/SSL_CTX_set_min_proto_version.html feature.
- bpo-24334 [https://bugs.python.org/issue?@action=redirect&bpo=24334]: Internal implementation details of ssl module were cleaned up. The SSLSocket has one less layer of indirection. Owner and session information are

now handled by the SSLSocket and SSLObject constructor. Channel binding implementation has been simplified.

- bpo-31848 [https://bugs.python.org/issue?@action=redirect&bpo=31848]: Fix the error handling in Aifc_read.initfp() when the SSND chunk is not found. Patch by Zackery Spytz.
- bpo-32585 [https://bugs.python.org/issue?@action=redirect&bpo=32585]: Add Ttk spinbox widget to `tkinter.ttk`. Patch by Alan D Moore.
- bpo-32221 [https://bugs.python.org/issue?@action=redirect&bpo=32221]: Various functions returning tuple containing IPv6 addresses now omit `%scope` part since the same information is already encoded in *scopeid* tuple item. Especially this speeds up `socket.recvfrom()` when it receives multicast packet since useless resolving of network interface name is omitted.
- bpo-30693 [https://bugs.python.org/issue?@action=redirect&bpo=30693]: The TarFile class now recurses directories in a reproducible way.
- bpo-30693 [https://bugs.python.org/issue?@action=redirect&bpo=30693]: The ZipFile class now recurses directories in a reproducible way.

Documentação

- bpo-28124 [https://bugs.python.org/issue?@action=redirect&bpo=28124]: The ssl module function ssl.wrap_socket() has been de-emphasized and deprecated in favor of the more secure and efficient SSLContext.wrap_socket() method.
- bpo-17232 [https://bugs.python.org/issue?@action=redirect&bpo=17232]: Clarify docs for -O and -OO. Patch by Terry Reedy.
- bpo-32436 [https://bugs.python.org/issue?@action=redirect&bpo=32436]: Add documentation for the contextvars module (PEP 567).
- bpo-32800 [https://bugs.python.org/issue?@action=redirect&bpo=32800]: Update link to w3c doc for xml default namespaces.
- bpo-11015 [https://bugs.python.org/issue?@action=redirect&bpo=11015]: Update `test.support` documentation.
- bpo-8722 [https://bugs.python.org/issue?@action=redirect&bpo=8722]: Document `__getattr__()` behavior when property `get()` method raises `AttributeError`.
- bpo-32614 [https://bugs.python.org/issue?@action=redirect&bpo=32614]: Modify RE examples in documentation to use raw strings to prevent

`DeprecationWarning` and add text to REGEX HOWTO to highlight the deprecation.
- bpo-31972 [https://bugs.python.org/issue?@action=redirect&bpo=31972]: Improve docstrings for `pathlib.PurePath` subclasses.

Testes

- bpo-31809 [https://bugs.python.org/issue?@action=redirect&bpo=31809]: Add tests to verify connection with secp ECDH curves.

Construção

- bpo-32898 [https://bugs.python.org/issue?@action=redirect&bpo=32898]: Fix the python debug build when using COUNT_ALLOCS.

Windows

- bpo-32901 [https://bugs.python.org/issue?@action=redirect&bpo=32901]: Update Tcl and Tk versions to 8.6.8
- bpo-31966 [https://bugs.python.org/issue?@action=redirect&bpo=31966]: Fixed WindowsConsoleIO.write() for writing empty data.
- bpo-32409 [https://bugs.python.org/issue?@action=redirect&bpo=32409]: Ensures activate.bat can handle Unicode contents.
- bpo-32457 [https://bugs.python.org/issue?@action=redirect&bpo=32457]: Improves handling of denormalized executable path when launching Python.
- bpo-32370 [https://bugs.python.org/issue?@action=redirect&bpo=32370]: Use the correct encoding for ipconfig output in the uuid module. Patch by Segev Finer.
- bpo-29248 [https://bugs.python.org/issue?@action=redirect&bpo=29248]: Fix `os.readlink()` on Windows, which was mistakenly treating the `PrintNameOffset` field of the reparse data buffer as a number of characters instead of bytes. Patch by Craig Holmquist and SSE4.

macOS

- bpo-32901 [https://bugs.python.org/issue?@action=redirect&bpo=32901]: Update macOS 10.9+ installer to Tcl/Tk 8.6.8.

IDLE

- bpo-32916 [https://bugs.python.org/issue?@action=redirect&bpo=32916]: Change `str` to `code` in pyparse.
- bpo-32905 [https://bugs.python.org/issue?@action=redirect&bpo=32905]: Remove unused code in pyparse module.
- bpo-32874 [https://bugs.python.org/issue?@action=redirect&bpo=32874]: Add tests for pyparse.
- bpo-32837 [https://bugs.python.org/issue?@action=redirect&bpo=32837]: Using the system and place-dependent default encoding for open() is a bad idea for IDLE's system and location-independent files.
- bpo-32826 [https://bugs.python.org/issue?@action=redirect&bpo=32826]: Add "encoding=utf-8" to open() in IDLE's test_help_about. GUI test test_file_buttons() only looks at initial ascii-only lines, but failed on systems where open() defaults to 'ascii' because readline() internally reads and decodes far enough ahead to encounter a non-ascii character in CREDITS.txt.
- bpo-32765 [https://bugs.python.org/issue?@action=redirect&bpo=32765]: Update configdialog General tab docstring to add new widgets to the widget list.

Ferramentas/Demos

- bpo-32222 [https://bugs.python.org/issue?@action=redirect&bpo=32222]: Fix pygettext not extracting docstrings for functions with type annotated arguments. Patch by Toby Harradine.

Python 3.7.0 beta 1

Data de lançamento: 2018-01-30

Núcleo e embutidos

- bpo-32703 [https://bugs.python.org/issue?@action=redirect&bpo=32703]: Fix coroutine's ResourceWarning when there's an active error set when it's being finalized.

- bpo-32650 [https://bugs.python.org/issue?@action=redirect&bpo=32650]: Pdb and other debuggers dependent on bdb.py will correctly step over (next command) native coroutines. Patch by Pablo Galindo.
- bpo-28685 [https://bugs.python.org/issue?@action=redirect&bpo=28685]: Optimize list.sort() and sorted() by using type specialized comparisons when possible.
- bpo-32685 [https://bugs.python.org/issue?@action=redirect&bpo=32685]: Improve suggestion when the Python 2 form of print statement is either present on the same line as the header of a compound statement or else terminated by a semi-colon instead of a newline. Patch by Nitish Chandra.
- bpo-32697 [https://bugs.python.org/issue?@action=redirect&bpo=32697]: Python now explicitly preserves the definition order of keyword-only parameters. It's always preserved their order, but this behavior was never guaranteed before; this behavior is now guaranteed and tested.
- bpo-32690 [https://bugs.python.org/issue?@action=redirect&bpo=32690]: The locals() dictionary now displays in the lexical order that variables were defined. Previously, the order was reversed.
- bpo-32677 [https://bugs.python.org/issue?@action=redirect&bpo=32677]: Add `.isascii()` method to `str`, `bytes` and `bytearray`. It can be used to test that string contains only ASCII characters.
- bpo-32670 [https://bugs.python.org/issue?@action=redirect&bpo=32670]: Enforce **PEP 479** [https://peps.python.org/pep-0479/] for all code. This means that manually raising a StopIteration exception from a generator is prohibited for all code, regardless of whether 'from __future__ import generator_stop' was used or not.
- bpo-32591 [https://bugs.python.org/issue?@action=redirect&bpo=32591]: Added built-in support for tracking the origin of coroutine objects; see sys.set_coroutine_origin_tracking_depth and CoroutineType.cr_origin. This replaces the asyncio debug mode's use of coroutine wrapping for native coroutine objects.
- bpo-31368 [https://bugs.python.org/issue?@action=redirect&bpo=31368]: Expose preadv and pwritev system calls in the os module. Patch by Pablo Galindo
- bpo-32544 [https://bugs.python.org/issue?@action=redirect&bpo=32544]: `hasattr(obj, name)` and `getattr(obj, name, default)` are about 4

times faster than before when `name` is not found and `obj` doesn't override `__getattr__` or `__getattribute__`.

- bpo-26163 [https://bugs.python.org/issue?@action=redirect&bpo=26163]: Improved frozenset() hash to create more distinct hash values when faced with datasets containing many similar values.
- bpo-32550 [https://bugs.python.org/issue?@action=redirect&bpo=32550]: Remove the STORE_ANNOTATION bytecode.
- bpo-20104 [https://bugs.python.org/issue?@action=redirect&bpo=20104]: Expose posix_spawn as a low level API in the os module. (removed before 3.7.0rc1)
- bpo-24340 [https://bugs.python.org/issue?@action=redirect&bpo=24340]: Fixed estimation of the code stack size.
- bpo-32436 [https://bugs.python.org/issue?@action=redirect&bpo=32436]: Implement **PEP 567** [https://peps.python.org/pep-0567/] Context Variables.
- bpo-18533 [https://bugs.python.org/issue?@action=redirect&bpo=18533]: `repr()` on a dict containing its own `values()` or `items()` no longer raises `RecursionError`; OrderedDict similarly. Instead, use `...`, as for other recursive structures. Patch by Ben North.
- bpo-20891 [https://bugs.python.org/issue?@action=redirect&bpo=20891]: Py_Initialize() now creates the GIL. The GIL is no longer created "on demand" to fix a race condition when PyGILState_Ensure() is called in a non-Python thread.
- bpo-32028 [https://bugs.python.org/issue?@action=redirect&bpo=32028]: Leading whitespace is now correctly ignored when generating suggestions for converting Py2 print statements to Py3 builtin print function calls. Patch by Sanyam Khurana.
- bpo-31179 [https://bugs.python.org/issue?@action=redirect&bpo=31179]: Make dict.copy() up to 5.5 times faster.
- bpo-31113 [https://bugs.python.org/issue?@action=redirect&bpo=31113]: Get rid of recursion in the compiler for normal control flow.

Biblioteca

- bpo-25988 [https://bugs.python.org/issue?@action=redirect&bpo=25988]: Deprecate exposing the contents of collections.abc in the regular collections module.

- bpo-31429 [https://bugs.python.org/issue?@action=redirect&bpo=31429]: The default cipher suite selection of the ssl module now uses a blacklist approach rather than a hard-coded whitelist. Python no longer re-enables ciphers that have been blocked by OpenSSL security update. Default cipher suite selection can be configured on compile time.
- bpo-30306 [https://bugs.python.org/issue?@action=redirect&bpo=30306]: contextlib.contextmanager now releases the arguments passed to the underlying generator as soon as the context manager is entered. Previously it would keep them alive for as long as the context manager was alive, even when not being used as a function decorator. Patch by Martin Teichmann.
- bpo-21417 [https://bugs.python.org/issue?@action=redirect&bpo=21417]: Added support for setting the compression level for zipfile.ZipFile.
- bpo-32251 [https://bugs.python.org/issue?@action=redirect&bpo=32251]: Implement asyncio.BufferedProtocol (provisional API).
- bpo-32513 [https://bugs.python.org/issue?@action=redirect&bpo=32513]: In dataclasses, allow easier overriding of dunder methods without specifying decorator parameters.
- bpo-32660 [https://bugs.python.org/issue?@action=redirect&bpo=32660]: **`termios`** makes available `FIONREAD`, `FIONCLEX`, `FIOCLEX`, `FIOASYNC` and `FIONBIO` also under Solaris/derivatives.
- bpo-27931 [https://bugs.python.org/issue?@action=redirect&bpo=27931]: Fix email address header parsing error when the username is an empty quoted string. Patch by Xiang Zhang.
- bpo-32659 [https://bugs.python.org/issue?@action=redirect&bpo=32659]: Under Solaris and derivatives, **`os.stat_result`** provides a st_fstype attribute.
- bpo-32662 [https://bugs.python.org/issue?@action=redirect&bpo=32662]: Implement Server.start_serving(), Server.serve_forever(), and Server.is_serving() methods. Add ‘start_serving’ keyword parameter to loop.create_server() and loop.create_unix_server().
- bpo-32391 [https://bugs.python.org/issue?@action=redirect&bpo=32391]: Implement **`asyncio.StreamWriter.wait_closed()`** and **`asyncio.StreamWriter.is_closing()`** methods
- bpo-32643 [https://bugs.python.org/issue?@action=redirect&bpo=32643]: Make Task._step, Task._wakeup and Future._schedule_callbacks methods private.

- bpo-32630 [https://bugs.python.org/issue?@action=redirect&bpo=32630]: Refactor decimal module to use contextvars to store decimal context.
- bpo-32622 [https://bugs.python.org/issue?@action=redirect&bpo=32622]: Add **`asyncio.AbstractEventLoop.sendfile()`** method.
- bpo-32304 [https://bugs.python.org/issue?@action=redirect&bpo=32304]: distutils' upload command no longer corrupts tar files ending with a CR byte, and no longer tries to convert CR to CRLF in any of the upload text fields.
- bpo-32502 [https://bugs.python.org/issue?@action=redirect&bpo=32502]: uuid.uuid1 no longer raises an exception if a 64-bit hardware address is encountered.
- bpo-32596 [https://bugs.python.org/issue?@action=redirect&bpo=32596]: `concurrent.futures` imports `ThreadPoolExecutor` and `ProcessPoolExecutor` lazily (using **PEP 562** [https://peps.python.org/pep-0562/]). It makes `import asyncio` about 15% faster because asyncio uses only `ThreadPoolExecutor` by default.
- bpo-31801 [https://bugs.python.org/issue?@action=redirect&bpo=31801]: Add `_ignore_` to `Enum` so temporary variables can be used during class construction without being turned into members.
- bpo-32576 [https://bugs.python.org/issue?@action=redirect&bpo=32576]: Use queue.SimpleQueue() in places where it can be invoked from a weakref callback.
- bpo-32574 [https://bugs.python.org/issue?@action=redirect&bpo=32574]: Fix memory leak in asyncio.Queue, when the queue has limited size and it is full, the cancellation of queue.put() can cause a memory leak. Patch by: José Melero.
- bpo-32521 [https://bugs.python.org/issue?@action=redirect&bpo=32521]: The nis module is now compatible with new libnsl and headers location.
- bpo-32467 [https://bugs.python.org/issue?@action=redirect&bpo=32467]: collections.abc.ValuesView now inherits from collections.abc.Collection.
- bpo-32473 [https://bugs.python.org/issue?@action=redirect&bpo=32473]: Improve ABCMeta._dump_registry() output readability
- bpo-32102 [https://bugs.python.org/issue?@action=redirect&bpo=32102]: New argument `capture_output` for subprocess.run
- bpo-32521 [https://bugs.python.org/issue?@action=redirect&bpo=32521]: glibc has removed Sun RPC. Use replacement libtirpc headers and library in nis

module.

- bpo-32493 [https://bugs.python.org/issue?@action=redirect&bpo=32493]: UUID module fixes build for FreeBSD/OpenBSD
- bpo-32503 [https://bugs.python.org/issue?@action=redirect&bpo=32503]: Pickling with protocol 4 no longer creates too small frames.
- bpo-29237 [https://bugs.python.org/issue?@action=redirect&bpo=29237]: Create enum for pstats sorting options
- bpo-32454 [https://bugs.python.org/issue?@action=redirect&bpo=32454]: Add close(fd) function to the socket module.
- bpo-25942 [https://bugs.python.org/issue?@action=redirect&bpo=25942]: The subprocess module is now more graceful when handling a Ctrl-C KeyboardInterrupt during subprocess.call, subprocess.run, or a Popen context manager. It now waits a short amount of time for the child (presumed to have also gotten the SIGINT) to exit, before continuing the KeyboardInterrupt exception handling. This still includes a SIGKILL in the call() and run() APIs, but at least the child had a chance first.
- bpo-32433 [https://bugs.python.org/issue?@action=redirect&bpo=32433]: The hmac module now has hmac.digest(), which provides an optimized HMAC digest.
- bpo-28134 [https://bugs.python.org/issue?@action=redirect&bpo=28134]: Sockets now auto-detect family, type and protocol from file descriptor by default.
- bpo-32404 [https://bugs.python.org/issue?@action=redirect&bpo=32404]: Fix bug where **`datetime.datetime.fromtimestamp()`** did not call __new__ in **`datetime.datetime`** subclasses.
- bpo-32403 [https://bugs.python.org/issue?@action=redirect&bpo=32403]: Improved speed of **`datetime.date`** and **`datetime.datetime`** alternate constructors.
- bpo-32228 [https://bugs.python.org/issue?@action=redirect&bpo=32228]: Ensure that `truncate()` preserves the file position (as reported by `tell()`) after writes longer than the buffer size.
- bpo-32410 [https://bugs.python.org/issue?@action=redirect&bpo=32410]: Implement `loop.sock_sendfile` for asyncio event loop.
- bpo-22908 [https://bugs.python.org/issue?@action=redirect&bpo=22908]: Added seek and tell to the ZipExtFile class. This only works if the file object used to open the zipfile is seekable.

- bpo-32373 [https://bugs.python.org/issue?@action=redirect&bpo=32373]: Add socket.getblocking() method.
- bpo-32248 [https://bugs.python.org/issue?@action=redirect&bpo=32248]: Add **`importlib.resources`** and **`importlib.abc.ResourceReader`** as the unified API for reading resources contained within packages. Loaders wishing to support resource reading must implement the **`get_resource_reader()`** method. File-based and zipimport-based loaders both implement these APIs. **`importlib.abc.ResourceLoader`** is deprecated in favor of these new APIs.
- bpo-32320 [https://bugs.python.org/issue?@action=redirect&bpo=32320]: collections.namedtuple() now supports default values.
- bpo-29302 [https://bugs.python.org/issue?@action=redirect&bpo=29302]: Add contextlib.AsyncExitStack. Patch by Alexander Mohr and Ilya Kulakov.
- bpo-31961 [https://bugs.python.org/issue?@action=redirect&bpo=31961]: *Removed in Python 3.7.0b2*. The *args* argument of subprocess.Popen can now be a path-like object. If *args* is given as a sequence, it's first element can now be a path-like object as well.
- bpo-31900 [https://bugs.python.org/issue?@action=redirect&bpo=31900]: The **`locale.localeconv()`** function now sets temporarily the `LC_CTYPE` locale to the `LC_NUMERIC` locale to decode `decimal_point` and `thousands_sep` byte strings if they are non-ASCII or longer than 1 byte, and the `LC_NUMERIC` locale is different than the `LC_CTYPE` locale. This temporary change affects other threads. Same change for the **`str.format()`** method when formatting a number (**`int`**, **`float`**, **`float`** and subclasses) with the `n` type (ex: `'{:n}'.format(1234)`).
- bpo-31853 [https://bugs.python.org/issue?@action=redirect&bpo=31853]: Use super().method instead of socket.method in SSLSocket. They were there most likely for legacy reasons.
- bpo-31399 [https://bugs.python.org/issue?@action=redirect&bpo=31399]: The ssl module now uses OpenSSL's X509_VERIFY_PARAM_set1_host() and X509_VERIFY_PARAM_set1_ip() API to verify hostname and IP addresses. Subject common name fallback can be disabled with SSLContext.hostname_checks_common_name.
- bpo-14976 [https://bugs.python.org/issue?@action=redirect&bpo=14976]: Add a queue.SimpleQueue class, an unbounded FIFO queue with a reentrant C implementation of put().

Documentação

- bpo-32724 [https://bugs.python.org/issue?@action=redirect&bpo=32724]: Add references to some commands in the documentation of Pdb. Patch by Stéphane Wirtel
- bpo-32649 [https://bugs.python.org/issue?@action=redirect&bpo=32649]: Complete the C API documentation, profiling and tracing part with the newly added per-opcode events.
- bpo-17799 [https://bugs.python.org/issue?@action=redirect&bpo=17799]: Explain real behaviour of sys.settrace and sys.setprofile and their C-API counterparts regarding which type of events are received in each function. Patch by Pablo Galindo Salgado.

Testes

- bpo-32721 [https://bugs.python.org/issue?@action=redirect&bpo=32721]: Fix test_hashlib to not fail if the _md5 module is not built.
- bpo-28414 [https://bugs.python.org/issue?@action=redirect&bpo=28414]: Add test cases for IDNA 2003 and 2008 host names. IDNA 2003 internationalized host names are working since bpo-31399 [https://bugs.python.org/issue?@action=redirect&bpo=31399] has landed. IDNA 2008 are still broken.
- bpo-32604 [https://bugs.python.org/issue?@action=redirect&bpo=32604]: Add a new "_xxsubinterpreters" extension module that exposes the existing subinterpreter C-API and a new cross-interpreter data sharing mechanism. The module is primarily intended for more thorough testing of the existing subinterpreter support. Note that the _xxsubinterpreters module has been removed in 3.7.0rc1.
- bpo-32602 [https://bugs.python.org/issue?@action=redirect&bpo=32602]: Add test certs and test for ECDSA cert and EC/RSA dual mode.
- bpo-32549 [https://bugs.python.org/issue?@action=redirect&bpo=32549]: On Travis CI, Python now Compiles and uses a local copy of OpenSSL 1.1.0g for testing.

Construção

- bpo-32635 [https://bugs.python.org/issue?@action=redirect&bpo=32635]: Fix segfault of the crypt module when libxcrypt is provided instead of libcrypt at the system.
- bpo-32598 [https://bugs.python.org/issue?@action=redirect&bpo=32598]: Use autoconf to detect OpenSSL libs, headers and supported features. The ax_check_openssl M4 macro uses pkg-config to locate OpenSSL and falls back to manual search.
- bpo-32593 [https://bugs.python.org/issue?@action=redirect&bpo=32593]: Drop support of FreeBSD 9 and older.
- bpo-29708 [https://bugs.python.org/issue?@action=redirect&bpo=29708]: If the **`SOURCE_DATE_EPOCH`** environment variable is set, **`py_compile`** will always create hash-based `.pyc` files.

Windows

- bpo-32588 [https://bugs.python.org/issue?@action=redirect&bpo=32588]: Create standalone _distutils_findvs module and add missing _queue module to installer.
- bpo-29911 [https://bugs.python.org/issue?@action=redirect&bpo=29911]: Ensure separate Modify and Uninstall buttons are displayed.
- bpo-32507 [https://bugs.python.org/issue?@action=redirect&bpo=32507]: Use app-local UCRT install rather than the proper update for old versions of Windows.

macOS

- bpo-32726 [https://bugs.python.org/issue?@action=redirect&bpo=32726]: Provide an additional, more modern macOS installer variant that supports macOS 10.9+ systems in 64-bit mode only. Upgrade the supplied third-party libraries to OpenSSL 1.1.0g and to SQLite 3.22.0. The 10.9+ installer now links with and supplies its own copy of Tcl/Tk 8.6.
- bpo-28440 [https://bugs.python.org/issue?@action=redirect&bpo=28440]: No longer add /Library/Python/3.x/site-packages to sys.path for macOS framework builds to avoid future conflicts.

API C

- bpo-32681 [https://bugs.python.org/issue?@action=redirect&bpo=32681]: Fix uninitialized variable ‘res’ in the C implementation of os.dup2. Patch by Stéphane Wirtel
- bpo-10381 [https://bugs.python.org/issue?@action=redirect&bpo=10381]: Add C API access to the `datetime.timezone` constructor and `datetime.timezone.UTC` singleton.

Python 3.7.0 alpha 4

Data de lançamento: 2018-01-08

Núcleo e embutidos

- bpo-31975 [https://bugs.python.org/issue?@action=redirect&bpo=31975]: The default warning filter list now starts with a “default::DeprecationWarning:__main__” entry, so deprecation warnings are once again shown by default in single-file scripts and at the interactive prompt.
- bpo-32226 [https://bugs.python.org/issue?@action=redirect&bpo=32226]: `__class_getitem__` is now an automatic class method.
- bpo-32399 [https://bugs.python.org/issue?@action=redirect&bpo=32399]: Add AIX uuid library support for RFC4122 using uuid_create() in libc.a
- bpo-32390 [https://bugs.python.org/issue?@action=redirect&bpo=32390]: Fix the compilation failure on AIX after the f_fsid field has been added to the object returned by os.statvfs() (bpo-32143 [https://bugs.python.org/issue?@action=redirect&bpo=32143]). Original patch by Michael Felt.
- bpo-32379 [https://bugs.python.org/issue?@action=redirect&bpo=32379]: Make MRO computation faster when a class inherits from a single base.
- bpo-32259 [https://bugs.python.org/issue?@action=redirect&bpo=32259]: The error message of a TypeError raised when unpack non-iterable is now more specific.
- bpo-27169 [https://bugs.python.org/issue?@action=redirect&bpo=27169]: The `__debug__` constant is now optimized out at compile time. This fixes also bpo-22091 [https://bugs.python.org/issue?@action=redirect&bpo=22091].
- bpo-32329 [https://bugs.python.org/issue?@action=redirect&bpo=32329]: The `-R` option now turns on hash randomization when the `PYTHONHASHSEED`

environment variable is set to `0`. Previously, the option was ignored. Moreover, `sys.flags.hash_randomization` is now properly set to 0 when hash randomization is turned off by `PYTHONHASHSEED=0`.

- bpo-30416 [https://bugs.python.org/issue?@action=redirect&bpo=30416]: The optimizer is now protected from spending much time doing complex calculations and consuming much memory for creating large constants in constant folding. Increased limits for constants that can be produced in constant folding.
- bpo-32282 [https://bugs.python.org/issue?@action=redirect&bpo=32282]: Fix an unnecessary ifdef in the include of VersionHelpers.h in socketmodule on Windows.
- bpo-30579 [https://bugs.python.org/issue?@action=redirect&bpo=30579]: Implement TracebackType.__new__ to allow Python-level creation of traceback objects, and make TracebackType.tb_next mutable.
- bpo-32260 [https://bugs.python.org/issue?@action=redirect&bpo=32260]: Don't byte swap the input keys to the SipHash algorithm on big-endian platforms. This should ensure siphash gives consistent results across platforms.
- bpo-31506 [https://bugs.python.org/issue?@action=redirect&bpo=31506]: Improve the error message logic for object.__new__ and object.__init__. Patch by Sanyam Khurana.
- bpo-20361 [https://bugs.python.org/issue?@action=redirect&bpo=20361]: `-b` and `-bb` now inject `'default::BytesWarning'` and `error::BytesWarning` entries into `sys.warnoptions`, ensuring that they take precedence over any other warning filters configured via the `-W` option or the `PYTHONWARNINGS` environment variable.
- bpo-32230 [https://bugs.python.org/issue?@action=redirect&bpo=32230]: `-X dev` now injects a `'default'` entry into sys.warnoptions, ensuring that it behaves identically to actually passing `-Wdefault` at the command line.
- bpo-29240 [https://bugs.python.org/issue?@action=redirect&bpo=29240]: Add a new UTF-8 mode: implementation of the **PEP 540** [https://peps.python.org/pep-0540/].
- bpo-32226 [https://bugs.python.org/issue?@action=redirect&bpo=32226]: **PEP 560** [https://peps.python.org/pep-0560/]: Add support for `__mro_entries__` and `__class_getitem__`. Implemented by Ivan Levkivskyi.
- bpo-32225 [https://bugs.python.org/issue?@action=redirect&bpo=32225]: **PEP 562** [https://peps.python.org/pep-0562/]: Add support for module `__getattr__` and

`__dir__`. Implemented by Ivan Levkivskyi.

- bpo-31901 [https://bugs.python.org/issue?@action=redirect&bpo=31901]: The `atexit` module now has its callback stored per interpreter.
- bpo-31650 [https://bugs.python.org/issue?@action=redirect&bpo=31650]: Implement **PEP 552** [https://peps.python.org/pep-0552/] (Deterministic pycs). Python now supports invalidating bytecode cache files bashed on a source content hash rather than source last-modified time.
- bpo-29469 [https://bugs.python.org/issue?@action=redirect&bpo=29469]: Move constant folding from bytecode layer to AST layer. Original patch by Eugene Toder.

Biblioteca

- bpo-32506 [https://bugs.python.org/issue?@action=redirect&bpo=32506]: Now that dict is defined as keeping insertion order, drop OrderedDict and just use plain dict.
- bpo-32279 [https://bugs.python.org/issue?@action=redirect&bpo=32279]: Add params to dataclasses.make_dataclasses(): init, repr, eq, order, hash, and frozen. Pass them through to dataclass().
- bpo-32278 [https://bugs.python.org/issue?@action=redirect&bpo=32278]: Make type information optional on dataclasses.make_dataclass(). If omitted, the string ‘typing.Any’ is used.
- bpo-32499 [https://bugs.python.org/issue?@action=redirect&bpo=32499]: Add dataclasses.is_dataclass(obj), which returns True if obj is a dataclass or an instance of one.
- bpo-32468 [https://bugs.python.org/issue?@action=redirect&bpo=32468]: Improve frame repr() to mention filename, code name and current line number.
- bpo-23749 [https://bugs.python.org/issue?@action=redirect&bpo=23749]: asyncio: Implement loop.start_tls()
- bpo-32441 [https://bugs.python.org/issue?@action=redirect&bpo=32441]: Return the new file descriptor (i.e., the second argument) from `os.dup2`. Previously, `None` was always returned.
- bpo-32422 [https://bugs.python.org/issue?@action=redirect&bpo=32422]: `functools.lru_cache` uses less memory (3 words for each cached key) and takes about 1/3 time for cyclic GC.
- bpo-31721 [https://bugs.python.org/issue?@action=redirect&bpo=31721]: Prevent Python crash from happening when Future._log_traceback is set to

True manually. Now it can only be set to False, or a ValueError is raised.

- bpo-32415 [https://bugs.python.org/issue?@action=redirect&bpo=32415]: asyncio: Add Task.get_loop() and Future.get_loop()
- bpo-26133 [https://bugs.python.org/issue?@action=redirect&bpo=26133]: Don't unsubscribe signals in asyncio UNIX event loop on interpreter shutdown.
- bpo-32363 [https://bugs.python.org/issue?@action=redirect&bpo=32363]: Make asyncio.Task.set_exception() and set_result() raise NotImplementedError. Task._step() and Future.__await__() raise proper exceptions when they are in an invalid state, instead of raising an AssertionError.
- bpo-32357 [https://bugs.python.org/issue?@action=redirect&bpo=32357]: Optimize asyncio.iscoroutine() and loop.create_task() for non-native coroutines (e.g. async/await compiled with Cython). 'loop.create_task(python_coroutine)' used to be 20% faster than 'loop.create_task(cython_coroutine)'. Now, the latter is as fast.
- bpo-32356 [https://bugs.python.org/issue?@action=redirect&bpo=32356]: asyncio.transport.resume_reading() and pause_reading() are now idempotent. New transport.is_reading() method is added.
- bpo-32355 [https://bugs.python.org/issue?@action=redirect&bpo=32355]: Optimize asyncio.gather(); now up to 15% faster.
- bpo-32351 [https://bugs.python.org/issue?@action=redirect&bpo=32351]: Use fastpath in asyncio.sleep if delay<0 (2x boost)
- bpo-32348 [https://bugs.python.org/issue?@action=redirect&bpo=32348]: Optimize asyncio.Future schedule/add/remove callback. The optimization shows 3-6% performance improvements of async/await code.
- bpo-32331 [https://bugs.python.org/issue?@action=redirect&bpo=32331]: Fix socket.settimeout() and socket.setblocking() to keep socket.type as is. Fix socket.socket() constructor to reset any bit flags applied to socket's type. This change only affects OSes that have SOCK_NONBLOCK and/or SOCK_CLOEXEC.
- bpo-32248 [https://bugs.python.org/issue?@action=redirect&bpo=32248]: Add **`importlib.abc.ResourceReader`** as an ABC for loaders to provide a unified API for reading resources contained within packages. Also add **`importlib.resources`** as the port of `importlib_resources`.

- bpo-32311 [https://bugs.python.org/issue?@action=redirect&bpo=32311]: Implement asyncio.create_task(coro) shortcut
- bpo-32327 [https://bugs.python.org/issue?@action=redirect&bpo=32327]: Convert asyncio functions that were documented as coroutines to coroutines. Affected functions: loop.sock_sendall, loop.sock_recv, loop.sock_accept, loop.getaddrinfo, loop.getnameinfo.
- bpo-32323 [https://bugs.python.org/issue?@action=redirect&bpo=32323]: **`urllib.parse.urlsplit()`** does not convert zone-id (scope) to lower case for scoped IPv6 addresses in hostnames now.
- bpo-32302 [https://bugs.python.org/issue?@action=redirect&bpo=32302]: Fix bdist_wininst of distutils for CRT v142: it binary compatible with CRT v140.
- bpo-29711 [https://bugs.python.org/issue?@action=redirect&bpo=29711]: Fix `stop_serving` in asyncio proactor loop kill all listening servers
- bpo-32308 [https://bugs.python.org/issue?@action=redirect&bpo=32308]: **`re.sub()`** now replaces empty matches adjacent to a previous non-empty match.
- bpo-29970 [https://bugs.python.org/issue?@action=redirect&bpo=29970]: Abort asyncio SSLProtocol connection if handshake not complete within 10 seconds.
- bpo-32314 [https://bugs.python.org/issue?@action=redirect&bpo=32314]: Implement asyncio.run().
- bpo-17852 [https://bugs.python.org/issue?@action=redirect&bpo=17852]: Revert incorrect fix based on misunderstanding of _Py_PyAtExit() semantics.
- bpo-32296 [https://bugs.python.org/issue?@action=redirect&bpo=32296]: Implement asyncio._get_running_loop() and get_event_loop() in C. This makes them 4x faster.
- bpo-32250 [https://bugs.python.org/issue?@action=redirect&bpo=32250]: Implement `asyncio.current_task()` and `asyncio.all_tasks()`. Add helpers intended to be used by alternative task implementations: `asyncio._register_task`, `asyncio._enter_task`, `asyncio._leave_task` and `asyncio._unregister_task`. Deprecate `asyncio.Task.current_task()` and `asyncio.Task.all_tasks()`.
- bpo-32255 [https://bugs.python.org/issue?@action=redirect&bpo=32255]: A single empty field is now always quoted when written into a CSV file. This allows to distinguish an empty row from a row consisting of a single empty field. Patch by Licht Takeuchi.

- bpo-32277 [https://bugs.python.org/issue?@action=redirect&bpo=32277]: Raise `NotImplementedError` instead of `SystemError` on platforms where `chmod(..., follow_symlinks=False)` is not supported. Patch by Anthony Sottile.
- bpo-30050 [https://bugs.python.org/issue?@action=redirect&bpo=30050]: New argument warn_on_full_buffer to signal.set_wakeup_fd lets you control whether Python prints a warning on stderr when the wakeup fd buffer overflows.
- bpo-29137 [https://bugs.python.org/issue?@action=redirect&bpo=29137]: The `fpectl` library has been removed. It was never enabled by default, never worked correctly on x86-64, and it changed the Python ABI in ways that caused unexpected breakage of C extensions.
- bpo-32273 [https://bugs.python.org/issue?@action=redirect&bpo=32273]: Move asyncio.test_utils to test.test_asyncio.
- bpo-32272 [https://bugs.python.org/issue?@action=redirect&bpo=32272]: Remove asyncio.async() function.
- bpo-32269 [https://bugs.python.org/issue?@action=redirect&bpo=32269]: Add asyncio.get_running_loop() function.
- bpo-32265 [https://bugs.python.org/issue?@action=redirect&bpo=32265]: All class and static methods of builtin types now are correctly classified by inspect.classify_class_attrs() and grouped in pydoc output. Added types.ClassMethodDescriptorType for unbound class methods of builtin types.
- bpo-32253 [https://bugs.python.org/issue?@action=redirect&bpo=32253]: Deprecate `yield from lock`, `await lock`, `with (yield from lock)` and `with await lock` for asyncio synchronization primitives.
- bpo-22589 [https://bugs.python.org/issue?@action=redirect&bpo=22589]: Changed MIME type of .bmp from ‘image/x-ms-bmp’ to ‘image/bmp’
- bpo-32193 [https://bugs.python.org/issue?@action=redirect&bpo=32193]: Convert asyncio to use *async/await* syntax. Old styled `yield from` is still supported too.
- bpo-32206 [https://bugs.python.org/issue?@action=redirect&bpo=32206]: Add support to run modules with pdb
- bpo-32227 [https://bugs.python.org/issue?@action=redirect&bpo=32227]: `functools.singledispatch` now supports registering implementations using type annotations.

- bpo-15873 [https://bugs.python.org/issue?@action=redirect&bpo=15873]: Added new alternate constructors **`datetime.datetime.fromisoformat()`**, **`datetime.time.fromisoformat()`** and **`datetime.date.fromisoformat()`** as the inverse operation of each classes's respective `isoformat` methods.
- bpo-32199 [https://bugs.python.org/issue?@action=redirect&bpo=32199]: The getnode() ip getter now uses 'ip link' instead of 'ip link list'.
- bpo-32143 [https://bugs.python.org/issue?@action=redirect&bpo=32143]: os.statvfs() includes the f_fsid field from statvfs(2)
- bpo-26439 [https://bugs.python.org/issue?@action=redirect&bpo=26439]: Fix ctypes.util.find_library() for AIX by implementing ctypes._aix.find_library() Patch by: Michael Felt
- bpo-31993 [https://bugs.python.org/issue?@action=redirect&bpo=31993]: The pickler now uses less memory when serializing large bytes and str objects into a file. Pickles created with protocol 4 will require less memory for unpickling large bytes and str objects.
- bpo-27456 [https://bugs.python.org/issue?@action=redirect&bpo=27456]: Ensure TCP_NODELAY is set on Linux. Tests by Victor Stinner.
- bpo-31778 [https://bugs.python.org/issue?@action=redirect&bpo=31778]: ast.literal_eval() is now more strict. Addition and subtraction of arbitrary numbers no longer allowed.
- bpo-31802 [https://bugs.python.org/issue?@action=redirect&bpo=31802]: Importing native path module (`posixpath`, `ntpath`) now works even if the `os` module still is not imported.
- bpo-30241 [https://bugs.python.org/issue?@action=redirect&bpo=30241]: Add contextlib.AbstractAsyncContextManager. Patch by Jelle Zijlstra.
- bpo-31699 [https://bugs.python.org/issue?@action=redirect&bpo=31699]: Fix deadlocks in **`concurrent.futures.ProcessPoolExecutor`** when task arguments or results cause pickling or unpickling errors. This should make sure that calls to the **`ProcessPoolExecutor`** API always eventually return.
- bpo-15216 [https://bugs.python.org/issue?@action=redirect&bpo=15216]: `TextIOWrapper.reconfigure()` supports changing *encoding*, *errors*, and *newline*.

Documentação

- bpo-32418 [https://bugs.python.org/issue?@action=redirect&bpo=32418]: Add get_loop() method to Server and AbstractServer classes.

Testes

- bpo-32252 [https://bugs.python.org/issue?@action=redirect&bpo=32252]: Fix faulthandler_suppress_crash_report() used to prevent core dump files when testing crashes. getrlimit() returns zero on success.
- bpo-32002 [https://bugs.python.org/issue?@action=redirect&bpo=32002]: Adjust C locale coercion testing for the empty locale and POSIX locale cases to more readily adjust to platform dependent behaviour.

Windows

- bpo-19764 [https://bugs.python.org/issue?@action=redirect&bpo=19764]: Implement support for `subprocess.Popen(close_fds=True)` on Windows. Patch by Segev Finer.

Ferramentas/Demos

- bpo-24960 [https://bugs.python.org/issue?@action=redirect&bpo=24960]: 2to3 and lib2to3 can now read pickled grammar files using pkgutil.get_data() rather than probing the filesystem. This lets 2to3 and lib2to3 work when run from a zipfile.

API C

- bpo-32030 [https://bugs.python.org/issue?@action=redirect&bpo=32030]: Py_Initialize() doesn't reset the memory allocators to default if the `PYTHONMALLOC` environment variable is not set.
- bpo-29084 [https://bugs.python.org/issue?@action=redirect&bpo=29084]: Undocumented C API for OrderedDict has been excluded from the limited C API. It was added by mistake and actually never worked in the limited C API.
- bpo-32264 [https://bugs.python.org/issue?@action=redirect&bpo=32264]: Moved the pygetopt.h header into internal/, since it has no public APIs.

- bpo-32241 [https://bugs.python.org/issue?@action=redirect&bpo=32241]: **`Py_SetProgramName()`** and **`Py_SetPythonHome()`** now take the `const wchar *` arguments instead of `wchar *`.

Python 3.7.0 alpha 3

Data de lançamento: 2017-12-05

Núcleo e embutidos

- bpo-32176 [https://bugs.python.org/issue?@action=redirect&bpo=32176]: co_flags.CO_NOFREE is now always set correctly by the code object constructor based on freevars and cellvars, rather than needing to be set correctly by the caller. This ensures it will be cleared automatically when additional cell references are injected into a modified code object and function.
- bpo-10544 [https://bugs.python.org/issue?@action=redirect&bpo=10544]: Yield expressions are now deprecated in comprehensions and generator expressions. They are still permitted in the definition of the outermost iterable, as that is evaluated directly in the enclosing scope.
- bpo-32137 [https://bugs.python.org/issue?@action=redirect&bpo=32137]: The repr of deeply nested dict now raises a RecursionError instead of crashing due to a stack overflow.
- bpo-32096 [https://bugs.python.org/issue?@action=redirect&bpo=32096]: Revert memory allocator changes in the C API: move structures back from _PyRuntime to Objects/obmalloc.c. The memory allocators are once again initialized statically, and so PyMem_RawMalloc() and Py_DecodeLocale() can be called before _PyRuntime_Initialize().
- bpo-32043 [https://bugs.python.org/issue?@action=redirect&bpo=32043]: Add a new “developer mode”: new “-X dev” command line option to enable debug checks at runtime.
- bpo-32023 [https://bugs.python.org/issue?@action=redirect&bpo=32023]: SyntaxError is now correctly raised when a generator expression without parenthesis is used instead of an inheritance list in a class definition. The duplication of the parentheses can be omitted only on calls.

- bpo-32012 [https://bugs.python.org/issue?@action=redirect&bpo=32012]: SyntaxError is now correctly raised when a generator expression without parenthesis is passed as an argument, but followed by a trailing comma. A generator expression always needs to be directly inside a set of parentheses and cannot have a comma on either side.
- bpo-28180 [https://bugs.python.org/issue?@action=redirect&bpo=28180]: A new internal `_Py_SetLocaleFromEnv(category)` helper function has been added in order to improve the consistency of behaviour across different `libc` implementations (e.g. Android doesn't support setting the locale from the environment by default).
- bpo-31949 [https://bugs.python.org/issue?@action=redirect&bpo=31949]: Fixed several issues in printing tracebacks (PyTraceBack_Print()). Setting sys.tracebacklimit to 0 or less now suppresses printing tracebacks. Setting sys.tracebacklimit to None now causes using the default limit. Setting sys.tracebacklimit to an integer larger than LONG_MAX now means using the limit LONG_MAX rather than the default limit. Fixed integer overflows in the case of more than `2**31` traceback items on Windows. Fixed output errors handling.
- bpo-30696 [https://bugs.python.org/issue?@action=redirect&bpo=30696]: Fix the interactive interpreter looping endlessly when no memory.
- bpo-20047 [https://bugs.python.org/issue?@action=redirect&bpo=20047]: Bytearray methods partition() and rpartition() now accept only bytes-like objects as separator, as documented. In particular they now raise TypeError rather of returning a bogus result when an integer is passed as a separator.
- bpo-21720 [https://bugs.python.org/issue?@action=redirect&bpo=21720]: BytesWarning no longer emitted when the *fromlist* argument of `__import__()` or the `__all__` attribute of the module contain bytes instances.
- bpo-31845 [https://bugs.python.org/issue?@action=redirect&bpo=31845]: Environment variables are once more read correctly at interpreter startup.
- bpo-28936 [https://bugs.python.org/issue?@action=redirect&bpo=28936]: Ensure that lexically first syntax error involving a parameter and `global` or `nonlocal` is detected first at a given scope. Patch by Ivan Levkivskyi.
- bpo-31825 [https://bugs.python.org/issue?@action=redirect&bpo=31825]: Fixed OverflowError in the 'unicode-escape' codec and in

codecs.escape_decode() when decode an escaped non-ascii byte.

- bpo-31618 [https://bugs.python.org/issue?@action=redirect&bpo=31618]: The per-frame tracing logic added in 3.7a1 has been altered so that `frame->f_lineno` is updated before either `"line"` or `"opcode"` events are emitted. Previously, opcode events were emitted first, and therefore would occasionally see stale line numbers on the frame. The behavior of this feature has changed slightly as a result: when both `f_trace_lines` and `f_trace_opcodes` are enabled, line events now occur first.
- bpo-28603 [https://bugs.python.org/issue?@action=redirect&bpo=28603]: Print the full context/cause chain of exceptions on interpreter exit, even if an exception in the chain is unhashable or compares equal to later ones. Patch by Zane Bitter.
- bpo-31786 [https://bugs.python.org/issue?@action=redirect&bpo=31786]: Fix timeout rounding in the select module to round correctly negative timeouts between -1.0 and 0.0. The functions now block waiting for events as expected. Previously, the call was incorrectly non-blocking. Patch by Pablo Galindo.
- bpo-31781 [https://bugs.python.org/issue?@action=redirect&bpo=31781]: Prevent crashes when calling methods of an uninitialized `zipimport.zipimporter` object. Patch by Oren Milman.
- bpo-30399 [https://bugs.python.org/issue?@action=redirect&bpo=30399]: Standard repr() of BaseException with a single argument no longer contains redundant trailing comma.
- bpo-31626 [https://bugs.python.org/issue?@action=redirect&bpo=31626]: Fixed a bug in debug memory allocator. There was a write to freed memory after shrinking a memory block.
- bpo-30817 [https://bugs.python.org/issue?@action=redirect&bpo=30817]: `PyErr_PrintEx()` clears now the ignored exception that may be raised by `_PySys_SetObjectId()`, for example when no memory.

Biblioteca

- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Two minor fixes for `typing` module: allow shallow copying instances of generic classes, improve interaction of `__init_subclass__` with generics. Original PRs by Ivan Levkivskyi.

- bpo-32214 [https://bugs.python.org/issue?@action=redirect&bpo=32214]: PEP 557, Data Classes. Provides a decorator which adds boilerplate methods to classes which use type annotations so specify fields.
- bpo-27240 [https://bugs.python.org/issue?@action=redirect&bpo=27240]: The header folding algorithm for the new email policies has been rewritten, which also fixes bpo-30788 [https://bugs.python.org/issue?@action=redirect&bpo=30788], bpo-31831 [https://bugs.python.org/issue?@action=redirect&bpo=31831], and bpo-32182 [https://bugs.python.org/issue?@action=redirect&bpo=32182]. In particular, RFC2231 folding is now done correctly.
- bpo-32186 [https://bugs.python.org/issue?@action=redirect&bpo=32186]: io.FileIO.readall() and io.FileIO.read() now release the GIL when getting the file size. Fixed hang of all threads with inaccessible NFS server. Patch by Nir Soffer.
- bpo-32101 [https://bugs.python.org/issue?@action=redirect&bpo=32101]: Add **`sys.flags.dev_mode`** flag
- bpo-32154 [https://bugs.python.org/issue?@action=redirect&bpo=32154]: The `asyncio.windows_utils.socketpair()` function has been removed: use directly **`socket.socketpair()`** which is available on all platforms since Python 3.5 (before, it wasn't available on Windows). `asyncio.windows_utils.socketpair()` was just an alias to `socket.socketpair` on Python 3.5 and newer.
- bpo-32089 [https://bugs.python.org/issue?@action=redirect&bpo=32089]: warnings: In development (-X dev) and debug mode (pydebug build), use the "default" action for ResourceWarning, rather than the "always" action, in the default warnings filters.
- bpo-32107 [https://bugs.python.org/issue?@action=redirect&bpo=32107]: `uuid.getnode()` now preferentially returns universally administered MAC addresses if available, over locally administered MAC addresses. This makes a better guarantee for global uniqueness of UUIDs returned from `uuid.uuid1()`. If only locally administered MAC addresses are available, the first such one found is returned.
- bpo-23033 [https://bugs.python.org/issue?@action=redirect&bpo=23033]: Wildcard is now supported in hostname when it is one and only character in the left most segment of hostname in second argument of **`ssl.match_hostname()`**. Patch by Mandeep Singh.
- bpo-12239 [https://bugs.python.org/issue?@action=redirect&bpo=12239]: Make **`msilib.SummaryInformation.GetProperty()`** return `None` when the

value of property is `VT_EMPTY`. Initial patch by Mark Mc Mahon.

- bpo-28334 [https://bugs.python.org/issue?@action=redirect&bpo=28334]: Use **`os.path.expanduser()`** to find the `~/.netrc` file in **`netrc.netrc`**. If it does not exist, **`FileNotFoundError`** is raised. Patch by Dimitri Merejkowsky.
- bpo-32121 [https://bugs.python.org/issue?@action=redirect&bpo=32121]: Made `tracemalloc.Traceback` behave more like the traceback module, sorting the frames from oldest to most recent. `Traceback.format()` now accepts negative *limit*, truncating the result to the `abs(limit)` oldest frames. To get the old behaviour, one can use the new *most_recent_first* argument to `Traceback.format()`. (Patch by Jesse Bakker.)
- bpo-31325 [https://bugs.python.org/issue?@action=redirect&bpo=31325]: Fix wrong usage of **`collections.namedtuple()`** in the **`RobotFileParser.parse()`** method. Initial patch by Robin Wellner.
- bpo-12382 [https://bugs.python.org/issue?@action=redirect&bpo=12382]: **`msilib.OpenDatabase()`** now raises a better exception message when it couldn't open or create an MSI file. Initial patch by William Tisäter.
- bpo-19610 [https://bugs.python.org/issue?@action=redirect&bpo=19610]: `setup()` now warns about invalid types for some fields. The `distutils.dist.Distribution` class now warns when `classifiers`, `keywords` and `platforms` fields are not specified as a list or a string.
- bpo-32071 [https://bugs.python.org/issue?@action=redirect&bpo=32071]: Added the `-k` command-line option to `python -m unittest` to run only tests that match the given pattern(s).
- bpo-10049 [https://bugs.python.org/issue?@action=redirect&bpo=10049]: Added *nullcontext* no-op context manager to contextlib. This provides a simpler and faster alternative to ExitStack() when handling optional context managers.
- bpo-28684 [https://bugs.python.org/issue?@action=redirect&bpo=28684]: The new test.support.skip_unless_bind_unix_socket() decorator is used here to skip asyncio tests that fail because the platform lacks a functional bind() function for unix domain sockets (as it is the case for non root users on the recent Android versions that run now SELinux in enforcing mode).
- bpo-32110 [https://bugs.python.org/issue?@action=redirect&bpo=32110]: `codecs.StreamReader.read(n)` now returns not more than *n*

characters/bytes for non-negative *n*. This makes it compatible with `read()` methods of other file-like objects.

- bpo-27535 [https://bugs.python.org/issue?@action=redirect&bpo=27535]: The warnings module doesn't leak memory anymore in the hidden warnings registry for the "ignore" action of warnings filters. warn_explicit() function doesn't add the warning key to the registry anymore for the "ignore" action.
- bpo-32088 [https://bugs.python.org/issue?@action=redirect&bpo=32088]: warnings: When Python is build is debug mode (`Py_DEBUG`), **`DeprecationWarning`**, **`PendingDeprecationWarning`** and **`ImportWarning`** warnings are now displayed by default.
- bpo-1647489 [https://bugs.python.org/issue?@action=redirect&bpo=1647489]: Fixed searching regular expression patterns that could match an empty string. Non-empty string can now be correctly found after matching an empty string.
- bpo-25054 [https://bugs.python.org/issue?@action=redirect&bpo=25054]: Added support of splitting on a pattern that could match an empty string.
- bpo-32072 [https://bugs.python.org/issue?@action=redirect&bpo=32072]: Fixed issues with binary plists: Fixed saving bytearrays. Identical objects will be saved only once. Equal references will be load as identical objects. Added support for saving and loading recursive data structures.
- bpo-32069 [https://bugs.python.org/issue?@action=redirect&bpo=32069]: Drop legacy SSL transport from asyncio, ssl.MemoryBIO is always used anyway.
- bpo-32066 [https://bugs.python.org/issue?@action=redirect&bpo=32066]: asyncio: Support pathlib.Path in create_unix_connection; sock arg should be optional
- bpo-32046 [https://bugs.python.org/issue?@action=redirect&bpo=32046]: Updates 2to3 to convert from operator.isCallable(obj) to callable(obj). Patch by Donghee Na.
- bpo-32018 [https://bugs.python.org/issue?@action=redirect&bpo=32018]: inspect.signature should follow **PEP 8** [https://peps.python.org/pep-0008/], if the parameter has an annotation and a default value. Patch by Donghee Na.
- bpo-32025 [https://bugs.python.org/issue?@action=redirect&bpo=32025]: Add time.thread_time() and time.thread_time_ns()

- bpo-32037 [https://bugs.python.org/issue?@action=redirect&bpo=32037]: Integers that fit in a signed 32-bit integer will be now pickled with protocol 0 using the INT opcode. This will decrease the size of a pickle, speed up pickling and unpickling, and make these integers be unpickled as int instances in Python 2.
- bpo-32034 [https://bugs.python.org/issue?@action=redirect&bpo=32034]: Make asyncio.IncompleteReadError and LimitOverrunError pickleable.
- bpo-32015 [https://bugs.python.org/issue?@action=redirect&bpo=32015]: Fixed the looping of asyncio in the case of reconnection the socket during waiting async read/write from/to the socket.
- bpo-32011 [https://bugs.python.org/issue?@action=redirect&bpo=32011]: Restored support of loading marshal files with the TYPE_INT64 code. These files can be produced in Python 2.7.
- bpo-28369 [https://bugs.python.org/issue?@action=redirect&bpo=28369]: Enhance add_reader/writer check that socket is not used by some transport. Before, only cases when add_reader/writer were called with an int FD were supported. Now the check is implemented correctly for all file-like objects.
- bpo-31976 [https://bugs.python.org/issue?@action=redirect&bpo=31976]: Fix race condition when flushing a file is slow, which can cause a segfault if closing the file from another thread.
- bpo-31985 [https://bugs.python.org/issue?@action=redirect&bpo=31985]: Formally deprecated aifc.openfp, sunau.openfp, and wave.openfp. Since change 7bc817d5ba917528e8bd07ec461c635291e7b06a in 1993, openfp in each of the three modules had been pointing to that module's open function as a matter of backwards compatibility, though it had been both untested and undocumented.
- bpo-21862 [https://bugs.python.org/issue?@action=redirect&bpo=21862]: cProfile command line now accepts `-m module_name` as an alternative to script path. Patch by Sanyam Khurana.
- bpo-31970 [https://bugs.python.org/issue?@action=redirect&bpo=31970]: Reduce performance overhead of asyncio debug mode.
- bpo-31843 [https://bugs.python.org/issue?@action=redirect&bpo=31843]: *database* argument of sqlite3.connect() now accepts a path-like object, instead of just a string.
- bpo-31945 [https://bugs.python.org/issue?@action=redirect&bpo=31945]: Add Configurable *blocksize* to `HTTPConnection` and `HTTPSConnection` for

improved upload throughput. Patch by Nir Soffer.

- bpo-31943 [https://bugs.python.org/issue?@action=redirect&bpo=31943]: Add a `cancelled()` method to **`asyncio.Handle`**. Patch by Marat Sharafutdinov.
- bpo-9678 [https://bugs.python.org/issue?@action=redirect&bpo=9678]: Fixed determining the MAC address in the uuid module: Using ifconfig on NetBSD and OpenBSD. Using arp on Linux, FreeBSD, NetBSD and OpenBSD. Based on patch by Takayuki Shimizukawa.
- bpo-30057 [https://bugs.python.org/issue?@action=redirect&bpo=30057]: Fix potential missed signal in signal.signal().
- bpo-31933 [https://bugs.python.org/issue?@action=redirect&bpo=31933]: Fix Blake2 params leaf_size and node_offset on big endian platforms. Patch by Jack O'Connor.
- bpo-21423 [https://bugs.python.org/issue?@action=redirect&bpo=21423]: Add an initializer argument to {Process,Thread}PoolExecutor
- bpo-31927 [https://bugs.python.org/issue?@action=redirect&bpo=31927]: Fixed compilation of the socket module on NetBSD 8. Fixed assertion failure or reading arbitrary data when parse a AF_BLUETOOTH address on NetBSD and DragonFly BSD.
- bpo-27666 [https://bugs.python.org/issue?@action=redirect&bpo=27666]: Fixed stack corruption in curses.box() and curses.ungetmouse() when the size of types chtype or mmask_t is less than the size of C long. curses.box() now accepts characters as arguments. Based on patch by Steve Fink.
- bpo-31917 [https://bugs.python.org/issue?@action=redirect&bpo=31917]: Add 3 new clock identifiers: **`time.CLOCK_BOOTTIME`**, **`time.CLOCK_PROF`** and **`time.CLOCK_UPTIME`**.
- bpo-31897 [https://bugs.python.org/issue?@action=redirect&bpo=31897]: plistlib now catches more errors when read binary plists and raises InvalidFileException instead of unexpected exceptions.
- bpo-25720 [https://bugs.python.org/issue?@action=redirect&bpo=25720]: Fix the method for checking pad state of curses WINDOW. Patch by Masayuki Yamamoto.
- bpo-31893 [https://bugs.python.org/issue?@action=redirect&bpo=31893]: Fixed the layout of the kqueue_event structure on OpenBSD and NetBSD. Fixed the comparison of the kqueue_event objects.
- bpo-31891 [https://bugs.python.org/issue?@action=redirect&bpo=31891]: Fixed building the curses module on NetBSD.

- bpo-31884 [https://bugs.python.org/issue?@action=redirect&bpo=31884]: added required constants to subprocess module for setting priority on windows
- bpo-28281 [https://bugs.python.org/issue?@action=redirect&bpo=28281]: Remove year (1-9999) limits on the Calendar.weekday() function. Patch by Mark Gollahon.
- bpo-31702 [https://bugs.python.org/issue?@action=redirect&bpo=31702]: crypt.mksalt() now allows to specify the number of rounds for SHA-256 and SHA-512 hashing.
- bpo-30639 [https://bugs.python.org/issue?@action=redirect&bpo=30639]: **`inspect.getfile()`** no longer computes the repr of unknown objects to display in an error message, to protect against badly behaved custom reprs.
- bpo-30768 [https://bugs.python.org/issue?@action=redirect&bpo=30768]: Fix the pthread+semaphore implementation of PyThread_acquire_lock_timed() when called with timeout > 0 and intr_flag=0: recompute the timeout if sem_timedwait() is interrupted by a signal (EINTR). See also the **PEP 475** [https://peps.python.org/pep-0475/].
- bpo-31854 [https://bugs.python.org/issue?@action=redirect&bpo=31854]: Add `mmap.ACCESS_DEFAULT` constant.
- bpo-31834 [https://bugs.python.org/issue?@action=redirect&bpo=31834]: Use optimized code for BLAKE2 only with SSSE3+. The pure SSE2 implementation is slower than the pure C reference implementation.
- bpo-28292 [https://bugs.python.org/issue?@action=redirect&bpo=28292]: Calendar.itermonthdates() will now consistently raise an exception when a date falls outside of the 0001-01-01 through 9999-12-31 range. To support applications that cannot tolerate such exceptions, the new methods itermonthdays3() and itermonthdays4() are added. The new methods return tuples and are not restricted by the range supported by datetime.date.
- bpo-28564 [https://bugs.python.org/issue?@action=redirect&bpo=28564]: The shutil.rmtree() function has been sped up to 20–40%. This was done using the os.scandir() function.
- bpo-28416 [https://bugs.python.org/issue?@action=redirect&bpo=28416]: Instances of pickle.Pickler subclass with the persistent_id() method and pickle.Unpickler subclass with the persistent_load() method no longer create reference cycles.

- bpo-31653 [https://bugs.python.org/issue?@action=redirect&bpo=31653]: Don't release the GIL if we can acquire a multiprocessing semaphore immediately.
- bpo-28326 [https://bugs.python.org/issue?@action=redirect&bpo=28326]: Fix multiprocessing.Process when stdout and/or stderr is closed or None.
- bpo-20825 [https://bugs.python.org/issue?@action=redirect&bpo=20825]: Add `subnet_of` and `superset_of` containment tests to **`ipaddress.IPv6Network`** and **`ipaddress.IPv4Network`**. Patch by Michel Albert and Cheryl Sabella.
- bpo-31827 [https://bugs.python.org/issue?@action=redirect&bpo=31827]: Remove the os.stat_float_times() function. It was introduced in Python 2.3 for backward compatibility with Python 2.2, and was deprecated since Python 3.1.
- bpo-31756 [https://bugs.python.org/issue?@action=redirect&bpo=31756]: Add a `subprocess.Popen(text=False)` keyword argument to `subprocess` functions to be more explicit about when the library should attempt to decode outputs into text. Patch by Andrew Clegg.
- bpo-31819 [https://bugs.python.org/issue?@action=redirect&bpo=31819]: Add AbstractEventLoop.sock_recv_into().
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: If nested log adapters are used, the inner `process()` methods are no longer omitted.
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: The `manager` property on LoggerAdapter objects is now properly settable.
- bpo-31806 [https://bugs.python.org/issue?@action=redirect&bpo=31806]: Fix timeout rounding in time.sleep(), threading.Lock.acquire() and socket.socket.settimeout() to round correctly negative timeouts between -1.0 and 0.0. The functions now block waiting for events as expected. Previously, the call was incorrectly non-blocking. Patch by Pablo Galindo.
- bpo-31803 [https://bugs.python.org/issue?@action=redirect&bpo=31803]: time.clock() and time.get_clock_info('clock') now emit a DeprecationWarning warning.
- bpo-31800 [https://bugs.python.org/issue?@action=redirect&bpo=31800]: Extended support for parsing UTC offsets. strptime '%z' can now parse the output generated by datetime.isoformat, including seconds and microseconds.

- bpo-28603 [https://bugs.python.org/issue?@action=redirect&bpo=28603]: traceback: Fix a TypeError that occurred during printing of exception tracebacks when either the current exception or an exception in its context/cause chain is unhashable. Patch by Zane Bitter.
- bpo-30541 [https://bugs.python.org/issue?@action=redirect&bpo=30541]: Add new function to seal a mock and prevent the automatically creation of child mocks. Patch by Mario Corchero.
- bpo-31784 [https://bugs.python.org/issue?@action=redirect&bpo=31784]: Implement the **PEP 564** [https://peps.python.org/pep-0564/], add new 6 new functions with nanosecond resolution to the `time` module: `clock_gettime_ns()`, `clock_settime_ns()`, `monotonic_ns()`, `perf_counter_ns()`, `process_time_ns()`, `time_ns()`.
- bpo-30143 [https://bugs.python.org/issue?@action=redirect&bpo=30143]: 2to3 now generates a code that uses abstract collection classes from collections.abc rather than collections.
- bpo-31770 [https://bugs.python.org/issue?@action=redirect&bpo=31770]: Prevent a crash when calling the `__init__()` method of a `sqlite3.Cursor` object more than once. Patch by Oren Milman.
- bpo-31764 [https://bugs.python.org/issue?@action=redirect&bpo=31764]: Prevent a crash in `sqlite3.Cursor.close()` in case the `Cursor` object is uninitialized. Patch by Oren Milman.
- bpo-31752 [https://bugs.python.org/issue?@action=redirect&bpo=31752]: Fix possible crash in timedelta constructor called with custom integers.
- bpo-31620 [https://bugs.python.org/issue?@action=redirect&bpo=31620]: an empty asyncio.Queue now doesn’t leak memory when queue.get pollers timeout
- bpo-31690 [https://bugs.python.org/issue?@action=redirect&bpo=31690]: Allow the flags re.ASCII, re.LOCALE, and re.UNICODE to be used as group flags for regular expressions.
- bpo-30349 [https://bugs.python.org/issue?@action=redirect&bpo=30349]: FutureWarning is now emitted if a regular expression contains character set constructs that will change semantically in the future (nested sets and set operations).
- bpo-31664 [https://bugs.python.org/issue?@action=redirect&bpo=31664]: Added support for the Blowfish hashing in the crypt module.
- bpo-31632 [https://bugs.python.org/issue?@action=redirect&bpo=31632]: Fix method set_protocol() of class _SSLProtocolTransport in asyncio module. This

method was previously modifying a wrong reference to the protocol.

- bpo-15037 [https://bugs.python.org/issue?@action=redirect&bpo=15037]: Added a workaround for getkey() in curses for ncurses 5.7 and earlier.
- bpo-31307 [https://bugs.python.org/issue?@action=redirect&bpo=31307]: Allow use of bytes objects for arguments to **`configparser.ConfigParser.read()`**. Patch by Vincent Michel.
- bpo-31334 [https://bugs.python.org/issue?@action=redirect&bpo=31334]: Fix `poll.poll([timeout])` in the `select` module for arbitrary negative timeouts on all OSes where it can only be a non-negative integer or -1. Patch by Riccardo Coccioli.
- bpo-31310 [https://bugs.python.org/issue?@action=redirect&bpo=31310]: multiprocessing's semaphore tracker should be launched again if crashed.
- bpo-31308 [https://bugs.python.org/issue?@action=redirect&bpo=31308]: Make multiprocessing's forkserver process immune to Ctrl-C and other user interruptions. If it crashes, restart it when necessary.
- bpo-31245 [https://bugs.python.org/issue?@action=redirect&bpo=31245]: Added support for AF_UNIX socket in asyncio `create_datagram_endpoint`.
- bpo-30553 [https://bugs.python.org/issue?@action=redirect&bpo=30553]: Add HTTP/2 status code 421 (Misdirected Request) to **`http.HTTPStatus`**. Patch by Vitor Pereira.

Documentação

- bpo-32105 [https://bugs.python.org/issue?@action=redirect&bpo=32105]: Added asyncio.BaseEventLoop.connect_accepted_socket versionadded marker.

Testes

- bpo-31380 [https://bugs.python.org/issue?@action=redirect&bpo=31380]: Skip test_httpservers test_undecodable_file on macOS: fails on APFS.
- bpo-31705 [https://bugs.python.org/issue?@action=redirect&bpo=31705]: Skip test_socket.test_sha256() on Linux kernel older than 4.5. The test fails with ENOKEY on kernel 3.10 (on ppc64le). A fix was merged into the kernel 4.5.

- bpo-32138 [https://bugs.python.org/issue?@action=redirect&bpo=32138]: Skip on Android test_faulthandler tests that raise SIGSEGV and remove the test.support.requires_android_level decorator.
- bpo-32136 [https://bugs.python.org/issue?@action=redirect&bpo=32136]: The runtime embedding tests have been split out from `Lib/test/test_capi.py` into a new `Lib/test/test_embed.py` file.
- bpo-28668 [https://bugs.python.org/issue?@action=redirect&bpo=28668]: test.support.requires_multiprocessing_queue is removed. Skip tests with test.support.import_module('multiprocessing.synchronize') instead when the semaphore implementation is broken or missing.
- bpo-32126 [https://bugs.python.org/issue?@action=redirect&bpo=32126]: Skip test_get_event_loop_new_process in test.test_asyncio.test_events when sem_open() is not functional.
- bpo-31174 [https://bugs.python.org/issue?@action=redirect&bpo=31174]: Fix test_tools.test_unparse: DirectoryTestCase now stores the names sample to always test the same files. It prevents false alarms when hunting reference leaks.

Construção

- bpo-28538 [https://bugs.python.org/issue?@action=redirect&bpo=28538]: Revert the previous changes, the if_nameindex structure is defined by Unified Headers.
- bpo-28762 [https://bugs.python.org/issue?@action=redirect&bpo=28762]: Revert the last commit, the F_LOCK macro is defined by Android Unified Headers.
- bpo-29040 [https://bugs.python.org/issue?@action=redirect&bpo=29040]: Support building Android with Unified Headers. The first NDK release to support Unified Headers is android-ndk-r14.
- bpo-32059 [https://bugs.python.org/issue?@action=redirect&bpo=32059]: `detect_modules()` in `setup.py` now also searches the sysroot paths when cross-compiling.
- bpo-31957 [https://bugs.python.org/issue?@action=redirect&bpo=31957]: Fixes Windows SDK version detection when building for Windows.
- bpo-31609 [https://bugs.python.org/issue?@action=redirect&bpo=31609]: Fixes quotes in PCbuild/clean.bat

- bpo-31934 [https://bugs.python.org/issue?@action=redirect&bpo=31934]: Abort the build when building out of a not clean source tree.
- bpo-31926 [https://bugs.python.org/issue?@action=redirect&bpo=31926]: Fixed Argument Clinic sometimes causing compilation errors when there was more than one function and/or method in a .c file with the same name.
- bpo-28791 [https://bugs.python.org/issue?@action=redirect&bpo=28791]: Update Windows builds to use SQLite 3.21.0.
- bpo-28791 [https://bugs.python.org/issue?@action=redirect&bpo=28791]: Update OS X installer to use SQLite 3.21.0.
- bpo-28643 [https://bugs.python.org/issue?@action=redirect&bpo=28643]: Record profile-opt build progress with stamp files.
- bpo-31866 [https://bugs.python.org/issue?@action=redirect&bpo=31866]: Finish removing support for AtheOS.

Windows

- bpo-1102 [https://bugs.python.org/issue?@action=redirect&bpo=1102]: Return `None` when `View.Fetch()` returns `ERROR_NO_MORE_ITEMS` instead of raising `MSIError`. Initial patch by Anthony Tuininga.
- bpo-31944 [https://bugs.python.org/issue?@action=redirect&bpo=31944]: Fixes Modify button in Apps and Features dialog.
- bpo-20486 [https://bugs.python.org/issue?@action=redirect&bpo=20486]: Implement the `Database.Close()` method to help closing MSI database objects.
- bpo-31857 [https://bugs.python.org/issue?@action=redirect&bpo=31857]: Make the behavior of USE_STACKCHECK deterministic in a multi-threaded environment.

macOS

- bpo-31392 [https://bugs.python.org/issue?@action=redirect&bpo=31392]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.0.2m

IDLE

- bpo-32207 [https://bugs.python.org/issue?@action=redirect&bpo=32207]: Improve tk event exception tracebacks in IDLE. When tk event handling is driven

by IDLE's run loop, a confusing and distracting queue.EMPTY traceback context is no longer added to tk event exception tracebacks. The traceback is now the same as when event handling is driven by user code. Patch based on a suggestion by Serhiy Storchaka.

- bpo-32164 [https://bugs.python.org/issue?@action=redirect&bpo=32164]: Delete unused file idlelib/tabbedpages.py. Use of TabbedPageSet in configdialog was replaced by ttk.Notebook.
- bpo-32100 [https://bugs.python.org/issue?@action=redirect&bpo=32100]: IDLE: Fix old and new bugs in pathbrowser; improve tests. Patch mostly by Cheryl Sabella.
- bpo-31858 [https://bugs.python.org/issue?@action=redirect&bpo=31858]: IDLE – Restrict shell prompt manipulation to the shell. Editor and output windows only see an empty last prompt line. This simplifies the code and fixes a minor bug when newline is inserted. Sys.ps1, if present, is read on Shell start-up, but is not set or changed.
- bpo-31860 [https://bugs.python.org/issue?@action=redirect&bpo=31860]: The font sample in the IDLE configuration dialog is now editable. Changes persist while IDLE remains open
- bpo-31836 [https://bugs.python.org/issue?@action=redirect&bpo=31836]: Test_code_module now passes if run after test_idle, which sets ps1. The code module uses sys.ps1 if present or sets it to '>>> ' if not. Test_code_module now properly tests both behaviors. Ditto for ps2.
- bpo-28603 [https://bugs.python.org/issue?@action=redirect&bpo=28603]: Fix a TypeError that caused a shell restart when printing a traceback that includes an exception that is unhashable. Patch by Zane Bitter.
- bpo-13802 [https://bugs.python.org/issue?@action=redirect&bpo=13802]: Use non-Latin characters in the IDLE's Font settings sample. Even if one selects a font that defines a limited subset of the unicode Basic Multilingual Plane, tcl/tk will use other fonts that define a character. The expanded example give users of non-Latin characters a better idea of what they might see in IDLE's shell and editors. To make room for the expanded sample, frames on the Font tab are re-arranged. The Font/Tabs help explains a bit about the additions.

Ferramentas/Demos

- bpo-32159 [https://bugs.python.org/issue?@action=redirect&bpo=32159]: Remove CVS and Subversion tools: remove svneol.py and treesync.py scripts. CPython migrated from CVS to Subversion, to Mercurial, and then to Git. CVS and Subversion are no longer used to develop CPython.
- bpo-30722 [https://bugs.python.org/issue?@action=redirect&bpo=30722]: Make redemo work with Python 3.6 and newer versions. Also, remove the `LOCALE` option since it doesn't work with string patterns in Python 3. Patch by Christoph Sarnowski.

API C

- bpo-20891 [https://bugs.python.org/issue?@action=redirect&bpo=20891]: Fix PyGILState_Ensure(). When PyGILState_Ensure() is called in a non-Python thread before PyEval_InitThreads(), only call PyEval_InitThreads() after calling PyThreadState_New() to fix a crash.
- bpo-32125 [https://bugs.python.org/issue?@action=redirect&bpo=32125]: The `Py_UseClassExceptionsFlag` flag has been removed. It was deprecated and wasn't used anymore since Python 2.0.
- bpo-25612 [https://bugs.python.org/issue?@action=redirect&bpo=25612]: Move the current exception state from the frame object to the co-routine. This simplifies the interpreter and fixes a couple of obscure bugs caused by having swap exception state when entering or exiting a generator.
- bpo-23699 [https://bugs.python.org/issue?@action=redirect&bpo=23699]: Add Py_RETURN_RICHCOMPARE macro to reduce boilerplate code in rich comparison functions.
- bpo-30697 [https://bugs.python.org/issue?@action=redirect&bpo=30697]: The `PyExc_RecursionErrorInst` singleton is removed and `PyErr_NormalizeException()` does not use it anymore. This singleton is persistent and its members being never cleared may cause a segfault during finalization of the interpreter. See also bpo-22898 [https://bugs.python.org/issue?@action=redirect&bpo=22898].

Python 3.7.0 alpha 2

Data de lançamento: 2017-10-16

Núcleo e embutidos

- bpo-31558 [https://bugs.python.org/issue?@action=redirect&bpo=31558]: `gc.freeze()` is a new API that allows for moving all objects currently tracked by the garbage collector to a permanent generation, effectively removing them from future collection events. This can be used to protect those objects from having their PyGC_Head mutated. In effect, this enables great copy-on-write stability at fork().
- bpo-31642 [https://bugs.python.org/issue?@action=redirect&bpo=31642]: Restored blocking “from package import module” by setting sys.modules[“package.module”] to None.
- bpo-31708 [https://bugs.python.org/issue?@action=redirect&bpo=31708]: Allow use of asynchronous generator expressions in synchronous functions.
- bpo-31709 [https://bugs.python.org/issue?@action=redirect&bpo=31709]: Drop support of asynchronous __aiter__.
- bpo-30404 [https://bugs.python.org/issue?@action=redirect&bpo=30404]: The -u option now makes the stdout and stderr streams unbuffered rather than line-buffered.
- bpo-31619 [https://bugs.python.org/issue?@action=redirect&bpo=31619]: Fixed a ValueError when convert a string with large number of underscores to integer with binary base.
- bpo-31602 [https://bugs.python.org/issue?@action=redirect&bpo=31602]: Fix an assertion failure in `zipimporter.get_source()` in case of a bad `zlib.decompress()`. Patch by Oren Milman.
- bpo-31592 [https://bugs.python.org/issue?@action=redirect&bpo=31592]: Fixed an assertion failure in Python parser in case of a bad `unicodedata.normalize()`. Patch by Oren Milman.
- bpo-31588 [https://bugs.python.org/issue?@action=redirect&bpo=31588]: Raise a `TypeError` with a helpful error message when class creation fails due to a metaclass with a bad `__prepare__()` method. Patch by Oren Milman.
- bpo-31574 [https://bugs.python.org/issue?@action=redirect&bpo=31574]: Importlib was instrumented with two dtrace probes to profile import timing.
- bpo-31566 [https://bugs.python.org/issue?@action=redirect&bpo=31566]: Fix an assertion failure in `_warnings.warn()` in case of a bad `__name__` global. Patch by Oren Milman.

- bpo-31506 [https://bugs.python.org/issue?@action=redirect&bpo=31506]: Improved the error message logic for `object.__new__` and `object.__init__`.
- bpo-31505 [https://bugs.python.org/issue?@action=redirect&bpo=31505]: Fix an assertion failure in `json`, in case `_json.make_encoder()` received a bad `encoder()` argument. Patch by Oren Milman.
- bpo-31492 [https://bugs.python.org/issue?@action=redirect&bpo=31492]: Fix assertion failures in case of failing to import from a module with a bad `__name__` attribute, and in case of failing to access an attribute of such a module. Patch by Oren Milman.
- bpo-31478 [https://bugs.python.org/issue?@action=redirect&bpo=31478]: Fix an assertion failure in `_random.Random.seed()` in case the argument has a bad `__abs__()` method. Patch by Oren Milman.
- bpo-31336 [https://bugs.python.org/issue?@action=redirect&bpo=31336]: Speed up class creation by 10-20% by reducing the overhead in the necessary special method lookups. Patch by Stefan Behnel.
- bpo-31415 [https://bugs.python.org/issue?@action=redirect&bpo=31415]: Add `-X importtime` option to show how long each import takes. It can be used to optimize application's startup time. Support the **`PYTHONPROFILEIMPORTTIME`** as an equivalent way to enable this.
- bpo-31410 [https://bugs.python.org/issue?@action=redirect&bpo=31410]: Optimized calling wrapper and classmethod descriptors.
- bpo-31353 [https://bugs.python.org/issue?@action=redirect&bpo=31353]: **PEP 553** [https://peps.python.org/pep-0553/] - Add a new built-in called `breakpoint()` which calls `sys.breakpointhook()`. By default this imports `pdb` and calls `pdb.set_trace()`, but users may override `sys.breakpointhook()` to call whatever debugger they want. The original value of the hook is saved in `sys.__breakpointhook__`.
- bpo-17852 [https://bugs.python.org/issue?@action=redirect&bpo=17852]: Maintain a list of open buffered files, flush them before exiting the interpreter. Based on a patch from Armin Rigo.
- bpo-31315 [https://bugs.python.org/issue?@action=redirect&bpo=31315]: Fix an assertion failure in imp.create_dynamic(), when spec.name is not a string. Patch by Oren Milman.
- bpo-31311 [https://bugs.python.org/issue?@action=redirect&bpo=31311]: Fix a crash in the `__setstate__()` method of `ctypes._CData`, in case of a bad `__dict__`. Patch by Oren Milman.

- bpo-31293 [https://bugs.python.org/issue?@action=redirect&bpo=31293]: Fix crashes in true division and multiplication of a timedelta object by a float with a bad as_integer_ratio() method. Patch by Oren Milman.
- bpo-31285 [https://bugs.python.org/issue?@action=redirect&bpo=31285]: Fix an assertion failure in `warnings.warn_explicit`, when the return value of the received loader's `get_source()` has a bad `splitlines()` method. Patch by Oren Milman.
- bpo-30406 [https://bugs.python.org/issue?@action=redirect&bpo=30406]: Make `async` and `await` proper keywords, as specified in **PEP 492** [https://peps.python.org/pep-0492/].

Biblioteca

- bpo-30058 [https://bugs.python.org/issue?@action=redirect&bpo=30058]: Fixed buffer overflow in select.kqueue.control().
- bpo-31672 [https://bugs.python.org/issue?@action=redirect&bpo=31672]: `idpattern` in `string.Template` matched some non-ASCII characters. Now it uses `-i` regular expression local flag to avoid non-ASCII characters.
- bpo-31701 [https://bugs.python.org/issue?@action=redirect&bpo=31701]: On Windows, faulthandler.enable() now ignores MSC and COM exceptions.
- bpo-31728 [https://bugs.python.org/issue?@action=redirect&bpo=31728]: Prevent crashes in `_elementtree` due to unsafe cleanup of `Element.text` and `Element.tail`. Patch by Oren Milman.
- bpo-31671 [https://bugs.python.org/issue?@action=redirect&bpo=31671]: Now `re.compile()` converts passed RegexFlag to normal int object before compiling. bm_regex_compile benchmark shows 14% performance improvements.
- bpo-30397 [https://bugs.python.org/issue?@action=redirect&bpo=30397]: The types of compiled regular objects and match objects are now exposed as `re.Pattern` and `re.Match`. This adds information in pydoc output for the `re` module.
- bpo-31675 [https://bugs.python.org/issue?@action=redirect&bpo=31675]: Fixed memory leaks in Tkinter's methods splitlist() and split() when pass a string larger than 2 GiB.
- bpo-31673 [https://bugs.python.org/issue?@action=redirect&bpo=31673]: Fixed typo in the name of Tkinter's method adderrorinfo().

- bpo-31648 [https://bugs.python.org/issue?@action=redirect&bpo=31648]: Improvements to path predicates in ElementTree: Allow whitespace around predicate parts, i.e. “[a = ‘text’]” instead of requiring the less readable “[a=’text’]”. Add support for text comparison of the current node, like “[.=’text’]”. Patch by Stefan Behnel.
- bpo-30806 [https://bugs.python.org/issue?@action=redirect&bpo=30806]: Fix the string representation of a netrc object.
- bpo-31638 [https://bugs.python.org/issue?@action=redirect&bpo=31638]: Add optional argument `compressed` to `zipapp.create_archive`, and add option `--compress` to the command line interface of `zipapp`.
- bpo-25351 [https://bugs.python.org/issue?@action=redirect&bpo=25351]: Avoid venv activate failures with undefined variables
- bpo-20519 [https://bugs.python.org/issue?@action=redirect&bpo=20519]: Avoid ctypes use (if possible) and improve import time for uuid.
- bpo-28293 [https://bugs.python.org/issue?@action=redirect&bpo=28293]: The regular expression cache is no longer completely dumped when it is full.
- bpo-31596 [https://bugs.python.org/issue?@action=redirect&bpo=31596]: Added pthread_getcpuclockid() to the time module
- bpo-27494 [https://bugs.python.org/issue?@action=redirect&bpo=27494]: Make 2to3 accept a trailing comma in generator expressions. For example, `set(x for x in [],)` is now allowed.
- bpo-30347 [https://bugs.python.org/issue?@action=redirect&bpo=30347]: Stop crashes when concurrently iterate over itertools.groupby() iterators.
- bpo-30346 [https://bugs.python.org/issue?@action=redirect&bpo=30346]: An iterator produced by itertools.groupby() iterator now becomes exhausted after advancing the groupby iterator.
- bpo-31556 [https://bugs.python.org/issue?@action=redirect&bpo=31556]: Cancel asyncio.wait_for future faster if timeout <= 0
- bpo-31540 [https://bugs.python.org/issue?@action=redirect&bpo=31540]: Allow passing a context object in **`concurrent.futures.ProcessPoolExecutor`** constructor. Also, free job resources in **`concurrent.futures.ProcessPoolExecutor`** earlier to improve memory usage when a worker waits for new jobs.
- bpo-31516 [https://bugs.python.org/issue?@action=redirect&bpo=31516]: `threading.current_thread()` should not return a dummy thread at shutdown.

- bpo-31525 [https://bugs.python.org/issue?@action=redirect&bpo=31525]: In the sqlite module, require the sqlite3_prepare_v2 API. Thus, the sqlite module now requires sqlite version at least 3.3.9.
- bpo-26510 [https://bugs.python.org/issue?@action=redirect&bpo=26510]: argparse subparsers are now required by default. This matches behaviour in Python 2. For optional subparsers, use the new parameter `add_subparsers(required=False)`. Patch by Anthony Sottile. (As of 3.7.0rc1, the default was changed to not required as had been the case since Python 3.3.)
- bpo-27541 [https://bugs.python.org/issue?@action=redirect&bpo=27541]: Reprs of subclasses of some collection and iterator classes (`bytearray`, `array.array`, `collections.deque`, `collections.defaultdict`, `itertools.count`, `itertools.repeat`) now contain actual type name instead of hardcoded names of the base class.
- bpo-31351 [https://bugs.python.org/issue?@action=redirect&bpo=31351]: python -m ensurepip now exits with non-zero exit code if pip bootstrapping has failed.
- bpo-31389 [https://bugs.python.org/issue?@action=redirect&bpo=31389]: `pdb.set_trace()` now takes an optional keyword-only argument `header`. If given, this is printed to the console just before debugging begins.

Documentação

- bpo-31537 [https://bugs.python.org/issue?@action=redirect&bpo=31537]: Fix incorrect usage of `get_history_length` in readline documentation example code. Patch by Brad Smith.
- bpo-30085 [https://bugs.python.org/issue?@action=redirect&bpo=30085]: The operator functions without double underscores are preferred for clarity. The one with underscores are only kept for back-compatibility.

Construção

- bpo-31696 [https://bugs.python.org/issue?@action=redirect&bpo=31696]: Improve compiler version information in `sys.version` when Python is built with Clang.

- bpo-31625 [https://bugs.python.org/issue?@action=redirect&bpo=31625]: Stop using ranlib on static libraries. Instead, we assume ar supports the ‘s’ flag.
- bpo-31624 [https://bugs.python.org/issue?@action=redirect&bpo=31624]: Remove support for BSD/OS.
- bpo-22140 [https://bugs.python.org/issue?@action=redirect&bpo=22140]: Prevent double substitution of prefix in python-config.sh.
- bpo-31569 [https://bugs.python.org/issue?@action=redirect&bpo=31569]: Correct PCBuild/ case to PCbuild/ in build scripts and documentation.
- bpo-31536 [https://bugs.python.org/issue?@action=redirect&bpo=31536]: Avoid wholesale rebuild after `make regen-all` if nothing changed.

IDLE

- bpo-31460 [https://bugs.python.org/issue?@action=redirect&bpo=31460]: Simplify the API of IDLE’s Module Browser. Passing a widget instead of an flist with a root widget opens the option of creating a browser frame that is only part of a window. Passing a full file name instead of pieces assumed to come from a .py file opens the possibility of browsing python files that do not end in .py.
- bpo-31649 [https://bugs.python.org/issue?@action=redirect&bpo=31649]: IDLE - Make _htest, _utest parameters keyword only.
- bpo-31559 [https://bugs.python.org/issue?@action=redirect&bpo=31559]: Remove test order dependence in idle_test.test_browser.
- bpo-31459 [https://bugs.python.org/issue?@action=redirect&bpo=31459]: Rename IDLE’s module browser from Class Browser to Module Browser. The original module-level class and method browser became a module browser, with the addition of module-level functions, years ago. Nested classes and functions were added yesterday. For back-compatibility, the virtual event <<open-class-browser>>, which appears on the Keys tab of the Settings dialog, is not changed. Patch by Cheryl Sabella.
- bpo-31500 [https://bugs.python.org/issue?@action=redirect&bpo=31500]: Default fonts now are scaled on HiDPI displays.
- bpo-1612262 [https://bugs.python.org/issue?@action=redirect&bpo=1612262]: IDLE module browser now shows nested classes and functions. Original patches for code and tests by Guilherme Polo and Cheryl Sabella, respectively.

API C

- bpo-28280 [https://bugs.python.org/issue?@action=redirect&bpo=28280]: Make `PyMapping_Keys()`, `PyMapping_Values()` and `PyMapping_Items()` always return a `list` (rather than a `list` or a `tuple`). Patch by Oren Milman.
- bpo-31532 [https://bugs.python.org/issue?@action=redirect&bpo=31532]: Fix memory corruption due to allocator mix in getpath.c between Py_GetPath() and Py_SetPath()
- bpo-25658 [https://bugs.python.org/issue?@action=redirect&bpo=25658]: Implement **PEP 539** [https://peps.python.org/pep-0539/] for Thread Specific Storage (TSS) API: it is a new Thread Local Storage (TLS) API to CPython which would supersede use of the existing TLS API within the CPython interpreter, while deprecating the existing API. PEP written by Erik M. Bray, patch by Masayuki Yamamoto.

Python 3.7.0 alpha 1

Data de lançamento: 2017-09-19

Segurança

- bpo-29781 [https://bugs.python.org/issue?@action=redirect&bpo=29781]: SSLObject.version() now correctly returns None when handshake over BIO has not been performed yet.
- bpo-29505 [https://bugs.python.org/issue?@action=redirect&bpo=29505]: Add fuzz tests for float(str), int(str), unicode(str); for oss-fuzz.
- bpo-30947 [https://bugs.python.org/issue?@action=redirect&bpo=30947]: Upgrade libexpat embedded copy from version 2.2.1 to 2.2.3 to get security fixes.
- bpo-30730 [https://bugs.python.org/issue?@action=redirect&bpo=30730]: Prevent environment variables injection in subprocess on Windows. Prevent passing other environment variables and command arguments.
- bpo-30694 [https://bugs.python.org/issue?@action=redirect&bpo=30694]: Upgrade expat copy from 2.2.0 to 2.2.1 to get fixes of multiple security vulnerabilities including: **CVE 2017-9233** [https://www.cve.org/CVERecord?

id=CVE-2017-9233] (External entity infinite loop DoS), **CVE 2016-9063** [https://www.cve.org/CVERecord?id=CVE-2016-9063] (Integer overflow, re-fix), **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718] (Fix regression bugs from 2.2.0's fix to **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718]) and **CVE 2012-0876** [https://www.cve.org/CVERecord?id=CVE-2012-0876] (Counter hash flooding with SipHash). Note: the **CVE 2016-5300** [https://www.cve.org/CVERecord?id=CVE-2016-5300] (Use os-specific entropy sources like getrandom) doesn't impact Python, since Python already gets entropy from the OS to set the expat secret using `XML_SetHashSalt()`.

- bpo-30500 [https://bugs.python.org/issue?@action=redirect&bpo=30500]: Fix urllib.parse.splithost() to correctly parse fragments. For example, `splithost('//127.0.0.1#@evil.com/')` now correctly returns the `127.0.0.1` host, instead of treating `@evil.com` as the host in an authentication (`login@host`).
- bpo-29591 [https://bugs.python.org/issue?@action=redirect&bpo=29591]: Update expat copy from 2.1.1 to 2.2.0 to get fixes of **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718] and **CVE 2016-4472** [https://www.cve.org/CVERecord?id=CVE-2016-4472]. See https://sourceforge.net/p/expat/bugs/537/ for more information.

Núcleo e embutidos

- bpo-31490 [https://bugs.python.org/issue?@action=redirect&bpo=31490]: Fix an assertion failure in `ctypes` class definition, in case the class has an attribute whose name is specified in `_anonymous_` but not in `_fields_`. Patch by Oren Milman.
- bpo-31471 [https://bugs.python.org/issue?@action=redirect&bpo=31471]: Fix an assertion failure in `subprocess.Popen()` on Windows, in case the env argument has a bad `keys()` method. Patch by Oren Milman.
- bpo-31418 [https://bugs.python.org/issue?@action=redirect&bpo=31418]: Fix an assertion failure in `PyErr_WriteUnraisable()` in case of an exception with a bad `__module__` attribute. Patch by Oren Milman.
- bpo-31416 [https://bugs.python.org/issue?@action=redirect&bpo=31416]: Fix assertion failures in case of a bad warnings.filters or warnings.defaultaction. Patch by Oren Milman.

- bpo-28411 [https://bugs.python.org/issue?@action=redirect&bpo=28411]: Change direct usage of PyInterpreterState.modules to PyImport_GetModuleDict(). Also introduce more uniformity in other code that deals with sys.modules. This helps reduce complications when working on sys.modules.
- bpo-28411 [https://bugs.python.org/issue?@action=redirect&bpo=28411]: Switch to the abstract API when dealing with `PyInterpreterState.modules`. This allows later support for all dict subclasses and other Mapping implementations. Also add a `PyImport_GetModule()` function to reduce a bunch of duplicated code.
- bpo-31411 [https://bugs.python.org/issue?@action=redirect&bpo=31411]: Raise a TypeError instead of SystemError in case warnings.onceregistry is not a dictionary. Patch by Oren Milman.
- bpo-31344 [https://bugs.python.org/issue?@action=redirect&bpo=31344]: For finer control of tracing behaviour when testing the interpreter, two new frame attributes have been added to control the emission of particular trace events: `f_trace_lines` (`True` by default) to turn off per-line trace events; and `f_trace_opcodes` (`False` by default) to turn on per-opcode trace events.
- bpo-31373 [https://bugs.python.org/issue?@action=redirect&bpo=31373]: Fix several possible instances of undefined behavior due to floating-point demotions.
- bpo-30465 [https://bugs.python.org/issue?@action=redirect&bpo=30465]: Location information (`lineno` and `col_offset`) in f-strings is now (mostly) correct. This fixes tools like flake8 from showing warnings on the wrong line (typically the first line of the file).
- bpo-30860 [https://bugs.python.org/issue?@action=redirect&bpo=30860]: Consolidate CPython's global runtime state under a single struct. This improves discoverability of the runtime state.
- bpo-31347 [https://bugs.python.org/issue?@action=redirect&bpo=31347]: Fix possible undefined behavior in _PyObject_FastCall_Prepend.
- bpo-31343 [https://bugs.python.org/issue?@action=redirect&bpo=31343]: Include sys/sysmacros.h for major(), minor(), and makedev(). GNU C library plans to remove the functions from sys/types.h.
- bpo-31291 [https://bugs.python.org/issue?@action=redirect&bpo=31291]: Fix an assertion failure in `zipimport.zipimporter.get_data` on Windows,

when the return value of `pathname.replace('/','\\')` isn't a string. Patch by Oren Milman.

- bpo-31271 [https://bugs.python.org/issue?@action=redirect&bpo=31271]: Fix an assertion failure in the `write()` method of `io.TextIOWrapper`, when the encoder doesn't return a bytes object. Patch by Oren Milman.
- bpo-31243 [https://bugs.python.org/issue?@action=redirect&bpo=31243]: Fix a crash in some methods of `io.TextIOWrapper`, when the decoder's state is invalid. Patch by Oren Milman.
- bpo-30721 [https://bugs.python.org/issue?@action=redirect&bpo=30721]: `print` now shows correct usage hint for using Python 2 redirection syntax. Patch by Sanyam Khurana.
- bpo-31070 [https://bugs.python.org/issue?@action=redirect&bpo=31070]: Fix a race condition in importlib _get_module_lock().
- bpo-30747 [https://bugs.python.org/issue?@action=redirect&bpo=30747]: Add a non-dummy implementation of _Py_atomic_store and _Py_atomic_load on MSVC.
- bpo-31095 [https://bugs.python.org/issue?@action=redirect&bpo=31095]: Fix potential crash during GC caused by `tp_dealloc` which doesn't call `PyObject_GC_UnTrack()`.
- bpo-31071 [https://bugs.python.org/issue?@action=redirect&bpo=31071]: Avoid masking original TypeError in call with * unpacking when other arguments are passed.
- bpo-30978 [https://bugs.python.org/issue?@action=redirect&bpo=30978]: str.format_map() now passes key lookup exceptions through. Previously any exception was replaced with a KeyError exception.
- bpo-30808 [https://bugs.python.org/issue?@action=redirect&bpo=30808]: Use _Py_atomic API for concurrency-sensitive signal state.
- bpo-30876 [https://bugs.python.org/issue?@action=redirect&bpo=30876]: Relative import from unloaded package now reimports the package instead of failing with SystemError. Relative import from non-package now fails with ImportError rather than SystemError.
- bpo-30703 [https://bugs.python.org/issue?@action=redirect&bpo=30703]: Improve signal delivery. Avoid using Py_AddPendingCall from signal handler, to avoid calling signal-unsafe functions. The tests I'm adding here fail without the rest of the patch, on Linux and OS X. This means our signal delivery logic had defects (some signals could be lost).

- bpo-30765 [https://bugs.python.org/issue?@action=redirect&bpo=30765]: Avoid blocking in pthread_mutex_lock() when PyThread_acquire_lock() is asked not to block.
- bpo-31161 [https://bugs.python.org/issue?@action=redirect&bpo=31161]: Make sure the 'Missing parentheses' syntax error message is only applied to SyntaxError, not to subclasses. Patch by Martijn Pieters.
- bpo-30814 [https://bugs.python.org/issue?@action=redirect&bpo=30814]: Fixed a race condition when import a submodule from a package.
- bpo-30736 [https://bugs.python.org/issue?@action=redirect&bpo=30736]: The internal unicodedata database has been upgraded to Unicode 10.0.
- bpo-30604 [https://bugs.python.org/issue?@action=redirect&bpo=30604]: Move co_extra_freefuncs from per-thread to per-interpreter to avoid crashes.
- bpo-30597 [https://bugs.python.org/issue?@action=redirect&bpo=30597]: `print` now shows expected input in custom error message when used as a Python 2 statement. Patch by Sanyam Khurana.
- bpo-30682 [https://bugs.python.org/issue?@action=redirect&bpo=30682]: Removed a too-strict assertion that failed for certain f-strings, such as eval("f'\n'") and eval("f'\r'").
- bpo-30501 [https://bugs.python.org/issue?@action=redirect&bpo=30501]: The compiler now produces more optimal code for complex condition expressions in the "if", "while" and "assert" statement, the "if" expression, and generator expressions and comprehensions.
- bpo-28180 [https://bugs.python.org/issue?@action=redirect&bpo=28180]: Implement **PEP 538** [https://peps.python.org/pep-0538/] (legacy C locale coercion). This means that when a suitable coercion target locale is available, both the core interpreter and locale-aware C extensions will assume the use of UTF-8 as the default text encoding, rather than ASCII.
- bpo-30486 [https://bugs.python.org/issue?@action=redirect&bpo=30486]: Allows setting cell values for __closure__. Patch by Lisa Roach.
- bpo-30537 [https://bugs.python.org/issue?@action=redirect&bpo=30537]: itertools.islice now accepts integer-like objects (having an __index__ method) as start, stop, and slice arguments
- bpo-25324 [https://bugs.python.org/issue?@action=redirect&bpo=25324]: Tokens needed for parsing in Python moved to C. `COMMENT`, `NL` and `ENCODING`. This way the tokens and tok_names in the token module don't get changed when you import the tokenize module.

- bpo-29104 [https://bugs.python.org/issue?@action=redirect&bpo=29104]: Fixed parsing backslashes in f-strings.
- bpo-27945 [https://bugs.python.org/issue?@action=redirect&bpo=27945]: Fixed various segfaults with dict when input collections are mutated during searching, inserting or comparing. Based on patches by Duane Griffin and Tim Mitchell.
- bpo-25794 [https://bugs.python.org/issue?@action=redirect&bpo=25794]: Fixed type.__setattr__() and type.__delattr__() for non-interned attribute names. Based on patch by Eryk Sun.
- bpo-30039 [https://bugs.python.org/issue?@action=redirect&bpo=30039]: If a KeyboardInterrupt happens when the interpreter is in the middle of resuming a chain of nested 'yield from' or 'await' calls, it's now correctly delivered to the innermost frame.
- bpo-28974 [https://bugs.python.org/issue?@action=redirect&bpo=28974]: `object.__format__(x, '')` is now equivalent to `str(x)` rather than `format(str(self), '')`.
- bpo-30024 [https://bugs.python.org/issue?@action=redirect&bpo=30024]: Circular imports involving absolute imports with binding a submodule to a name are now supported.
- bpo-12414 [https://bugs.python.org/issue?@action=redirect&bpo=12414]: sys.getsizeof() on a code object now returns the sizes which includes the code struct and sizes of objects which it references. Patch by Donghee Na.
- bpo-29839 [https://bugs.python.org/issue?@action=redirect&bpo=29839]: len() now raises ValueError rather than OverflowError if __len__() returned a large negative integer.
- bpo-11913 [https://bugs.python.org/issue?@action=redirect&bpo=11913]: README.rst is now included in the list of distutils standard READMEs and therefore included in source distributions.
- bpo-29914 [https://bugs.python.org/issue?@action=redirect&bpo=29914]: Fixed default implementations of __reduce__ and __reduce_ex__(). object.__reduce__() no longer takes arguments, object.__reduce_ex__() now requires one argument.
- bpo-29949 [https://bugs.python.org/issue?@action=redirect&bpo=29949]: Fix memory usage regression of set and frozenset object.
- bpo-29935 [https://bugs.python.org/issue?@action=redirect&bpo=29935]: Fixed error messages in the index() method of tuple, list and deque when pass

indices of wrong type.

- bpo-29816 [https://bugs.python.org/issue?@action=redirect&bpo=29816]: Shift operation now has less opportunity to raise OverflowError. ValueError always is raised rather than OverflowError for negative counts. Shifting zero with non-negative count always returns zero.
- bpo-24821 [https://bugs.python.org/issue?@action=redirect&bpo=24821]: Fixed the slowing down to 25 times in the searching of some unlucky Unicode characters.
- bpo-29102 [https://bugs.python.org/issue?@action=redirect&bpo=29102]: Add a unique ID to PyInterpreterState. This makes it easier to identify each subinterpreter.
- bpo-29894 [https://bugs.python.org/issue?@action=redirect&bpo=29894]: The deprecation warning is emitted if __complex__ returns an instance of a strict subclass of complex. In a future versions of Python this can be an error.
- bpo-29859 [https://bugs.python.org/issue?@action=redirect&bpo=29859]: Show correct error messages when any of the pthread_* calls in thread_pthread.h fails.
- bpo-29849 [https://bugs.python.org/issue?@action=redirect&bpo=29849]: Fix a memory leak when an ImportError is raised during from import.
- bpo-28856 [https://bugs.python.org/issue?@action=redirect&bpo=28856]: Fix an oversight that %b format for bytes should support objects follow the buffer protocol.
- bpo-29723 [https://bugs.python.org/issue?@action=redirect&bpo=29723]: The `sys.path[0]` initialization change for bpo-29139 [https://bugs.python.org/issue?@action=redirect&bpo=29139] caused a regression by revealing an inconsistency in how sys.path is initialized when executing `__main__` from a zipfile, directory, or other import location. The interpreter now consistently avoids ever adding the import location's parent directory to `sys.path`, and ensures no other `sys.path` entries are inadvertently modified when inserting the import location named on the command line.
- bpo-29568 [https://bugs.python.org/issue?@action=redirect&bpo=29568]: Escaped percent "%%" in the format string for classic string formatting no longer allows any characters between two percents.
- bpo-29714 [https://bugs.python.org/issue?@action=redirect&bpo=29714]: Fix a regression that bytes format may fail when containing zero bytes

inside.

- bpo-29695 [https://bugs.python.org/issue?@action=redirect&bpo=29695]: bool(), float(), list() and tuple() no longer take keyword arguments. The first argument of int() can now be passes only as positional argument.
- bpo-28893 [https://bugs.python.org/issue?@action=redirect&bpo=28893]: Set correct __cause__ for errors about invalid awaitables returned from __aiter__ and __anext__.
- bpo-28876 [https://bugs.python.org/issue?@action=redirect&bpo=28876]: `bool(range)` works even if `len(range)` raises **`OverflowError`**.
- bpo-29683 [https://bugs.python.org/issue?@action=redirect&bpo=29683]: Fixes to memory allocation in _PyCode_SetExtra. Patch by Brian Coleman.
- bpo-29684 [https://bugs.python.org/issue?@action=redirect&bpo=29684]: Fix minor regression of PyEval_CallObjectWithKeywords. It should raise TypeError when kwargs is not a dict. But it might cause segv when args=NULL and kwargs is not a dict.
- bpo-28598 [https://bugs.python.org/issue?@action=redirect&bpo=28598]: Support __rmod__ for subclasses of str being called before str.__mod__. Patch by Martijn Pieters.
- bpo-29607 [https://bugs.python.org/issue?@action=redirect&bpo=29607]: Fix stack_effect computation for CALL_FUNCTION_EX. Patch by Matthieu Dartiailh.
- bpo-29602 [https://bugs.python.org/issue?@action=redirect&bpo=29602]: Fix incorrect handling of signed zeros in complex constructor for complex subclasses and for inputs having a __complex__ method. Patch by Serhiy Storchaka.
- bpo-29347 [https://bugs.python.org/issue?@action=redirect&bpo=29347]: Fixed possibly dereferencing undefined pointers when creating weakref objects.
- bpo-29463 [https://bugs.python.org/issue?@action=redirect&bpo=29463]: Add `docstring` field to Module, ClassDef, FunctionDef, and AsyncFunctionDef ast nodes. docstring is not first stmt in their body anymore. It affects `co_firstlineno` and `co_lnotab` of code object for module and class. (Reverted in bpo-32911 [https://bugs.python.org/issue?@action=redirect&bpo=32911].)
- bpo-29438 [https://bugs.python.org/issue?@action=redirect&bpo=29438]: Fixed use-after-free problem in key sharing dict.

- bpo-29546 [https://bugs.python.org/issue?@action=redirect&bpo=29546]: Set the ‘path’ and ‘name’ attribute on ImportError for `from ... import ...`.
- bpo-29546 [https://bugs.python.org/issue?@action=redirect&bpo=29546]: Improve from-import error message with location
- bpo-29478 [https://bugs.python.org/issue?@action=redirect&bpo=29478]: If max_line_length=None is specified while using the Compat32 policy, it is no longer ignored. Patch by Mircea Cosbuc.
- bpo-29319 [https://bugs.python.org/issue?@action=redirect&bpo=29319]: Prevent RunMainFromImporter overwriting sys.path[0].
- bpo-29337 [https://bugs.python.org/issue?@action=redirect&bpo=29337]: Fixed possible BytesWarning when compare the code objects. Warnings could be emitted at compile time.
- bpo-29327 [https://bugs.python.org/issue?@action=redirect&bpo=29327]: Fixed a crash when pass the iterable keyword argument to sorted().
- bpo-29034 [https://bugs.python.org/issue?@action=redirect&bpo=29034]: Fix memory leak and use-after-free in os module (path_converter).
- bpo-29159 [https://bugs.python.org/issue?@action=redirect&bpo=29159]: Fix regression in bytes(x) when x.__index__() raises Exception.
- bpo-29049 [https://bugs.python.org/issue?@action=redirect&bpo=29049]: Call _PyObject_GC_TRACK() lazily when calling Python function. Calling function is up to 5% faster.
- bpo-28927 [https://bugs.python.org/issue?@action=redirect&bpo=28927]: bytes.fromhex() and bytearray.fromhex() now ignore all ASCII whitespace, not only spaces. Patch by Robert Xiao.
- bpo-28932 [https://bugs.python.org/issue?@action=redirect&bpo=28932]: Do not include <sys/random.h> if it does not exist.
- bpo-25677 [https://bugs.python.org/issue?@action=redirect&bpo=25677]: Correct the positioning of the syntax error caret for indented blocks. Based on patch by Michael Layzell.
- bpo-29000 [https://bugs.python.org/issue?@action=redirect&bpo=29000]: Fixed bytes formatting of octals with zero padding in alternate form.
- bpo-18896 [https://bugs.python.org/issue?@action=redirect&bpo=18896]: Python function can now have more than 255 parameters. collections.namedtuple() now supports tuples with more than 255 elements.
- bpo-28596 [https://bugs.python.org/issue?@action=redirect&bpo=28596]: The preferred encoding is UTF-8 on Android. Patch written by Chi Hsuan

Yen.

- bpo-22257 [https://bugs.python.org/issue?@action=redirect&bpo=22257]: Clean up interpreter startup (see **PEP 432** [https://peps.python.org/pep-0432/]).
- bpo-26919 [https://bugs.python.org/issue?@action=redirect&bpo=26919]: On Android, operating system data is now always encoded/decoded to/from UTF-8, instead of the locale encoding to avoid inconsistencies with os.fsencode() and os.fsdecode() which are already using UTF-8.
- bpo-28991 [https://bugs.python.org/issue?@action=redirect&bpo=28991]: functools.lru_cache() was susceptible to an obscure reentrancy bug triggerable by a monkey-patched len() function.
- bpo-28147 [https://bugs.python.org/issue?@action=redirect&bpo=28147]: Fix a memory leak in split-table dictionaries: setattr() must not convert combined table into split table. Patch written by INADA Naoki.
- bpo-28739 [https://bugs.python.org/issue?@action=redirect&bpo=28739]: f-string expressions are no longer accepted as docstrings and by ast.literal_eval() even if they do not include expressions.
- bpo-28512 [https://bugs.python.org/issue?@action=redirect&bpo=28512]: Fixed setting the offset attribute of SyntaxError by PyErr_SyntaxLocationEx() and PyErr_SyntaxLocationObject().
- bpo-28918 [https://bugs.python.org/issue?@action=redirect&bpo=28918]: Fix the cross compilation of xxlimited when Python has been built with Py_DEBUG defined.
- bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722]: Rather than silently producing a class that doesn't support zero-argument `super()` in methods, failing to pass the new `__classcell__` namespace entry up to `type.__new__` now results in a `DeprecationWarning` and a class that supports zero-argument `super()`.
- bpo-28797 [https://bugs.python.org/issue?@action=redirect&bpo=28797]: Modifying the class __dict__ inside the __set_name__ method of a descriptor that is used inside that class no longer prevents calling the __set_name__ method of other descriptors.
- bpo-28799 [https://bugs.python.org/issue?@action=redirect&bpo=28799]: Remove the `PyEval_GetCallStats()` function and deprecate the untested and undocumented `sys.callstats()` function. Remove the `CALL_PROFILE` special build: use the **`sys.setprofile()`** function, **`cProfile`** or **`profile`** to profile function calls.

- bpo-12844 [https://bugs.python.org/issue?@action=redirect&bpo=12844]: More than 255 arguments can now be passed to a function.
- bpo-28782 [https://bugs.python.org/issue?@action=redirect&bpo=28782]: Fix a bug in the implementation `yield from` when checking if the next instruction is YIELD_FROM. Regression introduced by WORDCODE (bpo-26647 [https://bugs.python.org/issue?@action=redirect&bpo=26647]).
- bpo-28774 [https://bugs.python.org/issue?@action=redirect&bpo=28774]: Fix error position of the unicode error in ASCII and Latin1 encoders when a string returned by the error handler contains multiple non-encodable characters (non-ASCII for the ASCII codec, characters out of the U+0000-U+00FF range for Latin1).
- bpo-28731 [https://bugs.python.org/issue?@action=redirect&bpo=28731]: Optimize _PyDict_NewPresized() to create correct size dict. Improve speed of dict literal with constant keys up to 30%.
- bpo-28532 [https://bugs.python.org/issue?@action=redirect&bpo=28532]: Show sys.version when -V option is supplied twice.
- bpo-27100 [https://bugs.python.org/issue?@action=redirect&bpo=27100]: The with-statement now checks for __enter__ before it checks for __exit__. This gives less confusing error messages when both methods are missing. Patch by Jonathan Ellington.
- bpo-28746 [https://bugs.python.org/issue?@action=redirect&bpo=28746]: Fix the set_inheritable() file descriptor method on platforms that do not have the ioctl FIOCLEX and FIONCLEX commands.
- bpo-26920 [https://bugs.python.org/issue?@action=redirect&bpo=26920]: Fix not getting the locale's charset upon initializing the interpreter, on platforms that do not have langinfo.
- bpo-28648 [https://bugs.python.org/issue?@action=redirect&bpo=28648]: Fixed crash in Py_DecodeLocale() in debug build on Mac OS X when decode astral characters. Patch by Xiang Zhang.
- bpo-28665 [https://bugs.python.org/issue?@action=redirect&bpo=28665]: Improve speed of the STORE_DEREF opcode by 40%.
- bpo-19398 [https://bugs.python.org/issue?@action=redirect&bpo=19398]: Extra slash no longer added to sys.path components in case of empty compile-time PYTHONPATH components.
- bpo-28621 [https://bugs.python.org/issue?@action=redirect&bpo=28621]: Sped up converting int to float by reusing faster bits counting implementation. Patch by Adrian Wielgosik.

- bpo-28580 [https://bugs.python.org/issue?@action=redirect&bpo=28580]: Optimize iterating split table values. Patch by Xiang Zhang.
- bpo-28583 [https://bugs.python.org/issue?@action=redirect&bpo=28583]: PyDict_SetDefault didn't combine split table when needed. Patch by Xiang Zhang.
- bpo-28128 [https://bugs.python.org/issue?@action=redirect&bpo=28128]: Deprecation warning for invalid str and byte escape sequences now prints better information about where the error occurs. Patch by Serhiy Storchaka and Eric Smith.
- bpo-28509 [https://bugs.python.org/issue?@action=redirect&bpo=28509]: dict.update() no longer allocate unnecessary large memory.
- bpo-28426 [https://bugs.python.org/issue?@action=redirect&bpo=28426]: Fixed potential crash in PyUnicode_AsDecodedObject() in debug build.
- bpo-28517 [https://bugs.python.org/issue?@action=redirect&bpo=28517]: Fixed of-by-one error in the peephole optimizer that caused keeping unreachable code.
- bpo-28214 [https://bugs.python.org/issue?@action=redirect&bpo=28214]: Improved exception reporting for problematic __set_name__ attributes.
- bpo-23782 [https://bugs.python.org/issue?@action=redirect&bpo=23782]: Fixed possible memory leak in _PyTraceback_Add() and exception loss in PyTraceBack_Here().
- bpo-28183 [https://bugs.python.org/issue?@action=redirect&bpo=28183]: Optimize and cleanup dict iteration.
- bpo-26081 [https://bugs.python.org/issue?@action=redirect&bpo=26081]: Added C implementation of asyncio.Future. Original patch by Yury Selivanov.
- bpo-28379 [https://bugs.python.org/issue?@action=redirect&bpo=28379]: Added sanity checks and tests for PyUnicode_CopyCharacters(). Patch by Xiang Zhang.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: The type of long range iterator is now registered as Iterator. Patch by Oren Milman.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: Creating instances of range_iterator by calling range_iterator type now is disallowed. Calling iter() on range instance is the only way. Patch by Oren Milman.
- bpo-26906 [https://bugs.python.org/issue?@action=redirect&bpo=26906]: Resolving special methods of uninitialized type now causes implicit initialization

of the type instead of a fail.

- bpo-18287 [https://bugs.python.org/issue?@action=redirect&bpo=18287]: PyType_Ready() now checks that tp_name is not NULL. Original patch by Niklas Koep.
- bpo-24098 [https://bugs.python.org/issue?@action=redirect&bpo=24098]: Fixed possible crash when AST is changed in process of compiling it.
- bpo-28201 [https://bugs.python.org/issue?@action=redirect&bpo=28201]: Dict reduces possibility of 2nd conflict in hash table when hashes have same lower bits.
- bpo-28350 [https://bugs.python.org/issue?@action=redirect&bpo=28350]: String constants with null character no longer interned.
- bpo-26617 [https://bugs.python.org/issue?@action=redirect&bpo=26617]: Fix crash when GC runs during weakref callbacks.
- bpo-27942 [https://bugs.python.org/issue?@action=redirect&bpo=27942]: String constants now interned recursively in tuples and frozensets.
- bpo-28289 [https://bugs.python.org/issue?@action=redirect&bpo=28289]: ImportError.__init__ now resets not specified attributes.
- bpo-21578 [https://bugs.python.org/issue?@action=redirect&bpo=21578]: Fixed misleading error message when ImportError called with invalid keyword args.
- bpo-28203 [https://bugs.python.org/issue?@action=redirect&bpo=28203]: Fix incorrect type in complex(1.0, {2:3}) error message. Patch by Soumya Sharma.
- bpo-28086 [https://bugs.python.org/issue?@action=redirect&bpo=28086]: Single var-positional argument of tuple subtype was passed unscathed to the C-defined function. Now it is converted to exact tuple.
- bpo-28214 [https://bugs.python.org/issue?@action=redirect&bpo=28214]: Now __set_name__ is looked up on the class instead of the instance.
- bpo-27955 [https://bugs.python.org/issue?@action=redirect&bpo=27955]: Fallback on reading /dev/urandom device when the getrandom() syscall fails with EPERM, for example when blocked by SECCOMP.
- bpo-28192 [https://bugs.python.org/issue?@action=redirect&bpo=28192]: Don’t import readline in isolated mode.
- bpo-27441 [https://bugs.python.org/issue?@action=redirect&bpo=27441]: Remove some redundant assignments to ob_size in longobject.c. Thanks Oren Milman.

- bpo-27222 [https://bugs.python.org/issue?@action=redirect&bpo=27222]: Clean up redundant code in long_rshift function. Thanks Oren Milman.
- Upgrade internal unicode databases to Unicode version 9.0.0.
- bpo-28131 [https://bugs.python.org/issue?@action=redirect&bpo=28131]: Fix a regression in zipimport's compile_source(). zipimport should use the same optimization level as the interpreter.
- bpo-28126 [https://bugs.python.org/issue?@action=redirect&bpo=28126]: Replace Py_MEMCPY with memcpy(). Visual Studio can properly optimize memcpy().
- bpo-28120 [https://bugs.python.org/issue?@action=redirect&bpo=28120]: Fix dict.pop() for split dictionary when trying to remove a "pending key" (Not yet inserted in split-table). Patch by Xiang Zhang.
- bpo-26182 [https://bugs.python.org/issue?@action=redirect&bpo=26182]: Raise DeprecationWarning when async and await keywords are used as variable/attribute/class/function name.
- bpo-26182 [https://bugs.python.org/issue?@action=redirect&bpo=26182]: Fix a refleak in code that raises DeprecationWarning.
- bpo-28721 [https://bugs.python.org/issue?@action=redirect&bpo=28721]: Fix asynchronous generators aclose() and athrow() to handle StopAsyncIteration propagation properly.
- bpo-26110 [https://bugs.python.org/issue?@action=redirect&bpo=26110]: Speed-up method calls: add LOAD_METHOD and CALL_METHOD opcodes.

Biblioteca

- bpo-31499 [https://bugs.python.org/issue?@action=redirect&bpo=31499]: xml.etree: Fix a crash when a parser is part of a reference cycle.
- bpo-31482 [https://bugs.python.org/issue?@action=redirect&bpo=31482]: `random.seed()` now works with bytes in version=1
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: typing.get_type_hints now finds the right globalns for classes and modules by default (when no `globalns` was specified by the caller).
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Speed improvements to the `typing` module. Original PRs by Ivan Levkivskyi and Mitar.
- bpo-31544 [https://bugs.python.org/issue?@action=redirect&bpo=31544]: The C accelerator module of ElementTree ignored exceptions raised when

looking up TreeBuilder target methods in XMLParser().

- bpo-31234 [https://bugs.python.org/issue?@action=redirect&bpo=31234]: socket.create_connection() now fixes manually a reference cycle: clear the variable storing the last exception on success.
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: LoggerAdapter objects can now be nested.
- bpo-31431 [https://bugs.python.org/issue?@action=redirect&bpo=31431]: SSLContext.check_hostname now automatically sets SSLContext.verify_mode to ssl.CERT_REQUIRED instead of failing with a ValueError.
- bpo-31233 [https://bugs.python.org/issue?@action=redirect&bpo=31233]: socketserver.ThreadingMixIn now keeps a list of non-daemonic threads to wait until all these threads complete in server_close().
- bpo-28638 [https://bugs.python.org/issue?@action=redirect&bpo=28638]: Changed the implementation strategy for collections.namedtuple() to substantially reduce the use of exec() in favor of precomputed methods. As a result, the *verbose* parameter and *_source* attribute are no longer supported. The benefits include 1) having a smaller memory footprint for applications using multiple named tuples, 2) faster creation of the named tuple class (approx 4x to 6x depending on how it is measured), and 3) minor speed-ups for instance creation using __new__, _make, and _replace. (The primary patch contributor is Jelle Zijlstra with further improvements by INADA Naoki, Serhiy Storchaka, and Raymond Hettinger.)
- bpo-31400 [https://bugs.python.org/issue?@action=redirect&bpo=31400]: Improves SSL error handling to avoid losing error numbers.
- bpo-27629 [https://bugs.python.org/issue?@action=redirect&bpo=27629]: Make return types of SSLContext.wrap_bio() and SSLContext.wrap_socket() customizable.
- bpo-28958 [https://bugs.python.org/issue?@action=redirect&bpo=28958]: ssl.SSLContext() now uses OpenSSL error information when a context cannot be instantiated.
- bpo-28182 [https://bugs.python.org/issue?@action=redirect&bpo=28182]: The SSL module now raises SSLCertVerificationError when OpenSSL fails to verify the peer’s certificate. The exception contains more information about the error.

- bpo-27340 [https://bugs.python.org/issue?@action=redirect&bpo=27340]: SSLSocket.sendall() now uses memoryview to create slices of data. This fixes support for all bytes-like object. It is also more efficient and avoids costly copies.
- bpo-14191 [https://bugs.python.org/issue?@action=redirect&bpo=14191]: A new function `argparse.ArgumentParser.parse_intermixed_args` provides the ability to parse command lines where there user intermixes options and positional arguments.
- bpo-31178 [https://bugs.python.org/issue?@action=redirect&bpo=31178]: Fix string concatenation bug in rare error path in the subprocess module
- bpo-31350 [https://bugs.python.org/issue?@action=redirect&bpo=31350]: Micro-optimize **`asyncio._get_running_loop()`** to become up to 10% faster.
- bpo-31170 [https://bugs.python.org/issue?@action=redirect&bpo=31170]: expat: Update libexpat from 2.2.3 to 2.2.4. Fix copying of partial characters for UTF-8 input (libexpat bug 115): https://github.com/libexpat/libexpat/issues/115
- bpo-29136 [https://bugs.python.org/issue?@action=redirect&bpo=29136]: Add TLS 1.3 cipher suites and OP_NO_TLSv1_3.
- bpo-1198569 [https://bugs.python.org/issue?@action=redirect&bpo=1198569]: `string.Template` subclasses can optionally define `braceidpattern` if they want to specify different placeholder patterns inside and outside the braces. If None (the default) it falls back to `idpattern`.
- bpo-31326 [https://bugs.python.org/issue?@action=redirect&bpo=31326]: concurrent.futures.ProcessPoolExecutor.shutdown() now explicitly closes the call queue. Moreover, shutdown(wait=True) now also join the call queue thread, to prevent leaking a dangling thread.
- bpo-27144 [https://bugs.python.org/issue?@action=redirect&bpo=27144]: The `map()` and `as_completed()` iterators in `concurrent.futures` now avoid keeping a reference to yielded objects.
- bpo-31281 [https://bugs.python.org/issue?@action=redirect&bpo=31281]: Fix `fileinput.FileInput(files, inplace=True)` when `files` contain `pathlib.Path` objects.
- bpo-10746 [https://bugs.python.org/issue?@action=redirect&bpo=10746]: Fix ctypes producing wrong **PEP 3118** [https://peps.python.org/pep-3118/] type codes for integer types.
- bpo-27584 [https://bugs.python.org/issue?@action=redirect&bpo=27584]: `AF_VSOCK` has been added to the socket interface which allows communication

between virtual machines and their host.

- bpo-22536 [https://bugs.python.org/issue?@action=redirect&bpo=22536]: The subprocess module now sets the filename when FileNotFoundError is raised on POSIX systems due to the executable or cwd not being found.
- bpo-29741 [https://bugs.python.org/issue?@action=redirect&bpo=29741]: Update some methods in the _pyio module to also accept integer types. Patch by Oren Milman.
- bpo-31249 [https://bugs.python.org/issue?@action=redirect&bpo=31249]: concurrent.futures: WorkItem.run() used by ThreadPoolExecutor now breaks a reference cycle between an exception object and the WorkItem object.
- bpo-31247 [https://bugs.python.org/issue?@action=redirect&bpo=31247]: xmlrpc.server now explicitly breaks reference cycles when using sys.exc_info() in code handling exceptions.
- bpo-23835 [https://bugs.python.org/issue?@action=redirect&bpo=23835]: configparser: reading defaults in the `ConfigParser()` constructor is now using `read_dict()`, making its behavior consistent with the rest of the parser. Non-string keys and values in the defaults dictionary are now being implicitly converted to strings. Patch by James Tocknell.
- bpo-31238 [https://bugs.python.org/issue?@action=redirect&bpo=31238]: pydoc: the stop() method of the private ServerThread class now waits until DocServer.serve_until_quit() completes and then explicitly sets its docserver attribute to None to break a reference cycle.
- bpo-5001 [https://bugs.python.org/issue?@action=redirect&bpo=5001]: Many asserts in `multiprocessing` are now more informative, and some error types have been changed to more specific ones.
- bpo-31109 [https://bugs.python.org/issue?@action=redirect&bpo=31109]: Convert zipimport to use Argument Clinic.
- bpo-30102 [https://bugs.python.org/issue?@action=redirect&bpo=30102]: The ssl and hashlib modules now call OPENSSL_add_all_algorithms_noconf() on OpenSSL < 1.1.0. The function detects CPU features and enables optimizations on some CPU architectures such as POWER8. Patch is based on research from Gustavo Serra Scalet.
- bpo-18966 [https://bugs.python.org/issue?@action=redirect&bpo=18966]: Non-daemonic threads created by a multiprocessing.Process are now joined on child exit.

- bpo-31183 [https://bugs.python.org/issue?@action=redirect&bpo=31183]: `dis` now works with asynchronous generator and coroutine objects. Patch by George Collins based on diagnosis by Luciano Ramalho.
- bpo-5001 [https://bugs.python.org/issue?@action=redirect&bpo=5001]: There are a number of uninformative asserts in the `multiprocessing` module, as noted in issue 5001. This change fixes two of the most potentially problematic ones, since they are in error-reporting code, in the `multiprocessing.managers.convert_to_error` function. (It also makes more informative a ValueError message.) The only potentially problematic change is that the AssertionError is now a TypeError; however, this should also help distinguish it from an AssertionError being *reported* by the function/its caller (such as in issue 31169). - Patch by Allen W. Smith (drallensmith on github).
- bpo-31185 [https://bugs.python.org/issue?@action=redirect&bpo=31185]: Fixed miscellaneous errors in asyncio speedup module.
- bpo-31151 [https://bugs.python.org/issue?@action=redirect&bpo=31151]: socketserver.ForkingMixIn.server_close() now waits until all child processes completed to prevent leaking zombie processes.
- bpo-31072 [https://bugs.python.org/issue?@action=redirect&bpo=31072]: Add an `include_file` parameter to `zipapp.create_archive()`
- bpo-24700 [https://bugs.python.org/issue?@action=redirect&bpo=24700]: Optimize array.array comparison. It is now from 10x up to 70x faster when comparing arrays holding values of the same integer type.
- bpo-31135 [https://bugs.python.org/issue?@action=redirect&bpo=31135]: ttk: fix the destroy() method of LabeledScale and OptionMenu classes. Call the parent destroy() method even if the used attribute doesn’t exist. The LabeledScale.destroy() method now also explicitly clears label and scale attributes to help the garbage collector to destroy all widgets.
- bpo-31107 [https://bugs.python.org/issue?@action=redirect&bpo=31107]: Fix `copyreg._slotnames()` mangled attribute calculation for classes whose name begins with an underscore. Patch by Shane Harvey.
- bpo-31080 [https://bugs.python.org/issue?@action=redirect&bpo=31080]: Allow `logging.config.fileConfig` to accept kwargs and/or args.
- bpo-30897 [https://bugs.python.org/issue?@action=redirect&bpo=30897]: `pathlib.Path` objects now include an `is_mount()` method (only implemented on POSIX). This is similar to `os.path.ismount(p)`. Patch by Cooper Ry Lees.

- bpo-31061 [https://bugs.python.org/issue?@action=redirect&bpo=31061]: Fixed a crash when using asyncio and threads.
- bpo-30987 [https://bugs.python.org/issue?@action=redirect&bpo=30987]: Added support for CAN ISO-TP protocol in the socket module.
- bpo-30522 [https://bugs.python.org/issue?@action=redirect&bpo=30522]: Added a `setStream` method to `logging.StreamHandler` to allow the stream to be set after creation.
- bpo-30502 [https://bugs.python.org/issue?@action=redirect&bpo=30502]: Fix handling of long oids in ssl. Based on patch by Christian Heimes.
- bpo-5288 [https://bugs.python.org/issue?@action=redirect&bpo=5288]: Support tzinfo objects with sub-minute offsets.
- bpo-30919 [https://bugs.python.org/issue?@action=redirect&bpo=30919]: Fix shared memory performance regression in multiprocessing in 3.x. Shared memory used anonymous memory mappings in 2.x, while 3.x mmaps actual files. Try to be careful to do as little disk I/O as possible.
- bpo-26732 [https://bugs.python.org/issue?@action=redirect&bpo=26732]: Fix too many fds in processes started with the “forkserver” method. A child process would inherit as many fds as the number of still-running children.
- bpo-29403 [https://bugs.python.org/issue?@action=redirect&bpo=29403]: Fix `unittest.mock`’s autospec to not fail on method-bound builtin functions. Patch by Aaron Gallagher.
- bpo-30961 [https://bugs.python.org/issue?@action=redirect&bpo=30961]: Fix decrementing a borrowed reference in tracemalloc.
- bpo-19896 [https://bugs.python.org/issue?@action=redirect&bpo=19896]: Fix multiprocessing.sharedctypes to recognize typecodes `'q'` and `'Q'`.
- bpo-30946 [https://bugs.python.org/issue?@action=redirect&bpo=30946]: Remove obsolete code in readline module for platforms where GNU readline is older than 2.1 or where select() is not available.
- bpo-25684 [https://bugs.python.org/issue?@action=redirect&bpo=25684]: Change `ttk.OptionMenu` radiobuttons to be unique across instances of `OptionMenu`.
- bpo-30886 [https://bugs.python.org/issue?@action=redirect&bpo=30886]: Fix multiprocessing.Queue.join_thread(): it now waits until the thread completes, even if the thread was started by the same process which created the queue.

- bpo-29854 [https://bugs.python.org/issue?@action=redirect&bpo=29854]: Fix segfault in readline when using readline's history-size option. Patch by Nir Soffer.
- bpo-30794 [https://bugs.python.org/issue?@action=redirect&bpo=30794]: Added multiprocessing.Process.kill method to terminate using the SIGKILL signal on Unix.
- bpo-30319 [https://bugs.python.org/issue?@action=redirect&bpo=30319]: socket.close() now ignores ECONNRESET error.
- bpo-30828 [https://bugs.python.org/issue?@action=redirect&bpo=30828]: Fix out of bounds write in `asyncio.CFuture.remove_done_callback()`.
- bpo-30302 [https://bugs.python.org/issue?@action=redirect&bpo=30302]: Use keywords in the `repr` of `datetime.timedelta`.
- bpo-30807 [https://bugs.python.org/issue?@action=redirect&bpo=30807]: signal.setitimer() may disable the timer when passed a tiny value. Tiny values (such as 1e-6) are valid non-zero values for setitimer(), which is specified as taking microsecond-resolution intervals. However, on some platform, our conversion routine could convert 1e-6 into a zero interval, therefore disabling the timer instead of (re-)scheduling it.
- bpo-30441 [https://bugs.python.org/issue?@action=redirect&bpo=30441]: Fix bug when modifying os.environ while iterating over it
- bpo-29585 [https://bugs.python.org/issue?@action=redirect&bpo=29585]: Avoid importing `sysconfig` from `site` to improve startup speed. Python startup is about 5% faster on Linux and 30% faster on macOS.
- bpo-29293 [https://bugs.python.org/issue?@action=redirect&bpo=29293]: Add missing parameter "n" on multiprocessing.Condition.notify(). The doc claims multiprocessing.Condition behaves like threading.Condition, but its notify() method lacked the optional "n" argument (to specify the number of sleepers to wake up) that threading.Condition.notify() accepts.
- bpo-30532 [https://bugs.python.org/issue?@action=redirect&bpo=30532]: Fix email header value parser dropping folding white space in certain cases.
- bpo-30596 [https://bugs.python.org/issue?@action=redirect&bpo=30596]: Add a `close()` method to `multiprocessing.Process`.
- bpo-9146 [https://bugs.python.org/issue?@action=redirect&bpo=9146]: Fix a segmentation fault in _hashopenssl when standard hash functions such as md5 are not available in the linked OpenSSL library. As in some special FIPS-140 build environments.

- bpo-29169 [https://bugs.python.org/issue?@action=redirect&bpo=29169]: Update zlib to 1.2.11.
- bpo-30119 [https://bugs.python.org/issue?@action=redirect&bpo=30119]: ftplib.FTP.putline() now throws ValueError on commands that contains CR or LF. Patch by Donghee Na.
- bpo-30879 [https://bugs.python.org/issue?@action=redirect&bpo=30879]: os.listdir() and os.scandir() now emit bytes names when called with bytes-like argument.
- bpo-30746 [https://bugs.python.org/issue?@action=redirect&bpo=30746]: Prohibited the '=' character in environment variable names in `os.putenv()` and `os.spawn*()`.
- bpo-30664 [https://bugs.python.org/issue?@action=redirect&bpo=30664]: The description of a unittest subtest now preserves the order of keyword arguments of TestCase.subTest().
- bpo-21071 [https://bugs.python.org/issue?@action=redirect&bpo=21071]: struct.Struct.format type is now **`str`** instead of **`bytes`**.
- bpo-29212 [https://bugs.python.org/issue?@action=redirect&bpo=29212]: Fix concurrent.futures.thread.ThreadPoolExecutor threads to have a non repr() based thread name by default when no thread_name_prefix is supplied. They will now identify themselves as "ThreadPoolExecutor-y_n".
- bpo-29755 [https://bugs.python.org/issue?@action=redirect&bpo=29755]: Fixed the lgettext() family of functions in the gettext module. They now always return bytes.
- bpo-30616 [https://bugs.python.org/issue?@action=redirect&bpo=30616]: Functional API of enum allows to create empty enums. Patched by Donghee Na
- bpo-30038 [https://bugs.python.org/issue?@action=redirect&bpo=30038]: Fix race condition between signal delivery and wakeup file descriptor. Patch by Nathaniel Smith.
- bpo-23894 [https://bugs.python.org/issue?@action=redirect&bpo=23894]: lib2to3 now recognizes `rb'...'` and `f'...'` strings.
- bpo-24744 [https://bugs.python.org/issue?@action=redirect&bpo=24744]: pkgutil.walk_packages function now raises ValueError if *path* is a string. Patch by Sanyam Khurana.
- bpo-24484 [https://bugs.python.org/issue?@action=redirect&bpo=24484]: Avoid race condition in multiprocessing cleanup.

- bpo-30589 [https://bugs.python.org/issue?@action=redirect&bpo=30589]: Fix multiprocessing.Process.exitcode to return the opposite of the signal number when the process is killed by a signal (instead of 255) when using the “forkserver” method.
- bpo-28994 [https://bugs.python.org/issue?@action=redirect&bpo=28994]: The traceback no longer displayed for SystemExit raised in a callback registered by atexit.
- bpo-30508 [https://bugs.python.org/issue?@action=redirect&bpo=30508]: Don’t log exceptions if Task/Future “cancel()” method was called.
- bpo-30645 [https://bugs.python.org/issue?@action=redirect&bpo=30645]: Fix path calculation in `imp.load_package()`, fixing it for cases when a package is only shipped with bytecodes. Patch by Alexandru Ardelean.
- bpo-11822 [https://bugs.python.org/issue?@action=redirect&bpo=11822]: The dis.dis() function now is able to disassemble nested code objects.
- bpo-30624 [https://bugs.python.org/issue?@action=redirect&bpo=30624]: selectors does not take KeyboardInterrupt and SystemExit into account, leaving a fd in a bad state in case of error. Patch by Giampaolo Rodola’.
- bpo-30595 [https://bugs.python.org/issue?@action=redirect&bpo=30595]: multiprocessing.Queue.get() with a timeout now polls its reader in non-blocking mode if it succeeded to acquire the lock but the acquire took longer than the timeout.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Updates to typing module: Add generic AsyncContextManager, add support for ContextManager on all versions. Original PRs by Jelle Zijlstra and Ivan Levkivskyi
- bpo-30605 [https://bugs.python.org/issue?@action=redirect&bpo=30605]: re.compile() no longer raises a BytesWarning when compiling a bytes instance with misplaced inline modifier. Patch by Roy Williams.
- bpo-29870 [https://bugs.python.org/issue?@action=redirect&bpo=29870]: Fix ssl sockets leaks when connection is aborted in asyncio/ssl implementation. Patch by Michaël Sghaïer.
- bpo-29743 [https://bugs.python.org/issue?@action=redirect&bpo=29743]: Closing transport during handshake process leaks open socket. Patch by Nikolay Kim
- bpo-27585 [https://bugs.python.org/issue?@action=redirect&bpo=27585]: Fix waiter cancellation in asyncio.Lock. Patch by Mathieu Sornay.

- bpo-30014 [https://bugs.python.org/issue?@action=redirect&bpo=30014]: modify() method of poll(), epoll() and devpoll() based classes of selectors module is around 10% faster. Patch by Giampaolo Rodola'.
- bpo-30418 [https://bugs.python.org/issue?@action=redirect&bpo=30418]: On Windows, subprocess.Popen.communicate() now also ignore EINVAL on stdin.write() if the child process is still running but closed the pipe.
- bpo-30463 [https://bugs.python.org/issue?@action=redirect&bpo=30463]: Added empty __slots__ to abc.ABC. This allows subclassers to deny __dict__ and __weakref__ creation. Patch by Aaron Hall.
- bpo-30520 [https://bugs.python.org/issue?@action=redirect&bpo=30520]: Loggers are now pickleable.
- bpo-30557 [https://bugs.python.org/issue?@action=redirect&bpo=30557]: faulthandler now correctly filters and displays exception codes on Windows
- bpo-30526 [https://bugs.python.org/issue?@action=redirect&bpo=30526]: Add TextIOWrapper.reconfigure() and a TextIOWrapper.write_through attribute.
- bpo-30245 [https://bugs.python.org/issue?@action=redirect&bpo=30245]: Fix possible overflow when organize struct.pack_into error message. Patch by Yuan Liu.
- bpo-30378 [https://bugs.python.org/issue?@action=redirect&bpo=30378]: Fix the problem that logging.handlers.SysLogHandler cannot handle IPv6 addresses.
- bpo-16500 [https://bugs.python.org/issue?@action=redirect&bpo=16500]: Allow registering at-fork handlers.
- bpo-30470 [https://bugs.python.org/issue?@action=redirect&bpo=30470]: Deprecate invalid ctypes call protection on Windows. Patch by Mariatta Wijaya.
- bpo-30414 [https://bugs.python.org/issue?@action=redirect&bpo=30414]: multiprocessing.Queue._feed background running thread do not break from main loop on exception.
- bpo-30003 [https://bugs.python.org/issue?@action=redirect&bpo=30003]: Fix handling escape characters in HZ codec. Based on patch by Ma Lin.
- bpo-30149 [https://bugs.python.org/issue?@action=redirect&bpo=30149]: inspect.signature() now supports callables with variable-argument parameters wrapped with partialmethod. Patch by Donghee Na.
- bpo-30436 [https://bugs.python.org/issue?@action=redirect&bpo=30436]: importlib.find_spec() raises ModuleNotFoundError instead of

AttributeError if the specified parent module is not a package (i.e. lacks a __path__ attribute).

- bpo-30301 [https://bugs.python.org/issue?@action=redirect&bpo=30301]: Fix AttributeError when using SimpleQueue.empty() under *spawn* and *forkserver* start methods.
- bpo-30375 [https://bugs.python.org/issue?@action=redirect&bpo=30375]: Warnings emitted when compile a regular expression now always point to the line in the user code. Previously they could point into inners of the re module if emitted from inside of groups or conditionals.
- bpo-30329 [https://bugs.python.org/issue?@action=redirect&bpo=30329]: imaplib and poplib now catch the Windows socket WSAEINVAL error (code 10022) on shutdown(SHUT_RDWR): An invalid operation was attempted. This error occurs sometimes on SSL connections.
- bpo-29196 [https://bugs.python.org/issue?@action=redirect&bpo=29196]: Removed previously deprecated in Python 2.4 classes Plist, Dict and _InternalDict in the plistlib module. Dict values in the result of functions readPlist() and readPlistFromBytes() are now normal dicts. You no longer can use attribute access to access items of these dictionaries.
- bpo-9850 [https://bugs.python.org/issue?@action=redirect&bpo=9850]: The **`macpath`** is now deprecated and will be removed in Python 3.8.
- bpo-30299 [https://bugs.python.org/issue?@action=redirect&bpo=30299]: Compiling regular expression in debug mode on CPython now displays the compiled bytecode in human readable form.
- bpo-30048 [https://bugs.python.org/issue?@action=redirect&bpo=30048]: Fixed `Task.cancel()` can be ignored when the task is running coroutine and the coroutine returned without any more `await`.
- bpo-30266 [https://bugs.python.org/issue?@action=redirect&bpo=30266]: contextlib.AbstractContextManager now supports anti-registration by setting __enter__ = None or __exit__ = None, following the pattern introduced in bpo-25958 [https://bugs.python.org/issue?@action=redirect&bpo=25958]. Patch by Jelle Zijlstra.
- bpo-30340 [https://bugs.python.org/issue?@action=redirect&bpo=30340]: Enhanced regular expressions optimization. This increased the performance of matching some patterns up to 25 times.
- bpo-30298 [https://bugs.python.org/issue?@action=redirect&bpo=30298]: Weaken the condition of deprecation warnings for inline modifiers. Now allowed

several subsequential inline modifiers at the start of the pattern (e.g. `'(?i)(?s)...'`). In verbose mode whitespaces and comments now are allowed before and between inline modifiers (e.g. `'(?x) (?i) (?s)...'`).

- bpo-30285 [https://bugs.python.org/issue?@action=redirect&bpo=30285]: Optimized case-insensitive matching and searching of regular expressions.
- bpo-29990 [https://bugs.python.org/issue?@action=redirect&bpo=29990]: Fix range checking in GB18030 decoder. Original patch by Ma Lin.
- bpo-29979 [https://bugs.python.org/issue?@action=redirect&bpo=29979]: rewrite cgi.parse_multipart, reusing the FieldStorage class and making its results consistent with those of FieldStorage for multipart/form-data requests. Patch by Pierre Quentel.
- bpo-30243 [https://bugs.python.org/issue?@action=redirect&bpo=30243]: Removed the __init__ methods of _json's scanner and encoder. Misusing them could cause memory leaks or crashes. Now scanner and encoder objects are completely initialized in the __new__ methods.
- bpo-30215 [https://bugs.python.org/issue?@action=redirect&bpo=30215]: Compiled regular expression objects with the re.LOCALE flag no longer depend on the locale at compile time. Only the locale at matching time affects the result of matching.
- bpo-30185 [https://bugs.python.org/issue?@action=redirect&bpo=30185]: Avoid KeyboardInterrupt tracebacks in forkserver helper process when Ctrl-C is received.
- bpo-30103 [https://bugs.python.org/issue?@action=redirect&bpo=30103]: binascii.b2a_uu() and uu.encode() now support using `'`'` as zero instead of space.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Various updates to typing module: add typing.NoReturn type, use WrapperDescriptorType, minor bug-fixes. Original PRs by Jim Fasarakis-Hilliard and Ivan Levkivskyi.
- bpo-30205 [https://bugs.python.org/issue?@action=redirect&bpo=30205]: Fix getsockname() for unbound AF_UNIX sockets on Linux.
- bpo-30228 [https://bugs.python.org/issue?@action=redirect&bpo=30228]: The seek() and tell() methods of io.FileIO now set the internal seekable attribute to avoid one syscall on open() (in buffered or text mode).
- bpo-30190 [https://bugs.python.org/issue?@action=redirect&bpo=30190]: unittest's assertAlmostEqual and assertNotAlmostEqual provide a better

message in case of failure which includes the difference between left and right arguments. (patch by Giampaolo Rodola')

- bpo-30101 [https://bugs.python.org/issue?@action=redirect&bpo=30101]: Add support for curses.A_ITALIC.
- bpo-29822 [https://bugs.python.org/issue?@action=redirect&bpo=29822]: inspect.isabstract() now works during __init_subclass__. Patch by Nate Soares.
- bpo-29960 [https://bugs.python.org/issue?@action=redirect&bpo=29960]: Preserve generator state when _random.Random.setstate() raises an exception. Patch by Bryan Olson.
- bpo-30070 [https://bugs.python.org/issue?@action=redirect&bpo=30070]: Fixed leaks and crashes in errors handling in the parser module.
- bpo-22352 [https://bugs.python.org/issue?@action=redirect&bpo=22352]: Column widths in the output of dis.dis() are now adjusted for large line numbers and instruction offsets.
- bpo-30061 [https://bugs.python.org/issue?@action=redirect&bpo=30061]: Fixed crashes in IOBase methods __next__() and readlines() when readline() or __next__() respectively return non-sizeable object. Fixed possible other errors caused by not checking results of PyObject_Size(), PySequence_Size(), or PyMapping_Size().
- bpo-30218 [https://bugs.python.org/issue?@action=redirect&bpo=30218]: Fix PathLike support for shutil.unpack_archive. Patch by Jelle Zijlstra.
- bpo-10076 [https://bugs.python.org/issue?@action=redirect&bpo=10076]: Compiled regular expression and match objects in the re module now support copy.copy() and copy.deepcopy() (they are considered atomic).
- bpo-30068 [https://bugs.python.org/issue?@action=redirect&bpo=30068]: _io._IOBase.readlines will check if it's closed first when hint is present.
- bpo-29694 [https://bugs.python.org/issue?@action=redirect&bpo=29694]: Fixed race condition in pathlib mkdir with flags parents=True. Patch by Armin Rigo.
- bpo-29692 [https://bugs.python.org/issue?@action=redirect&bpo=29692]: Fixed arbitrary unchaining of RuntimeError exceptions in contextlib.contextmanager. Patch by Siddharth Velankar.
- bpo-26187 [https://bugs.python.org/issue?@action=redirect&bpo=26187]: Test that sqlite3 trace callback is not called multiple times when schema is changing. Indirectly fixed by switching to use sqlite3_prepare_v2() in

bpo-9303 [https://bugs.python.org/issue?@action=redirect&bpo=9303]. Patch by Aviv Palivoda.

- bpo-30017 [https://bugs.python.org/issue?@action=redirect&bpo=30017]: Allowed calling the close() method of the zip entry writer object multiple times. Writing to a closed writer now always produces a ValueError.
- bpo-29998 [https://bugs.python.org/issue?@action=redirect&bpo=29998]: Pickling and copying ImportError now preserves name and path attributes.
- bpo-29995 [https://bugs.python.org/issue?@action=redirect&bpo=29995]: re.escape() now escapes only regex special characters.
- bpo-29962 [https://bugs.python.org/issue?@action=redirect&bpo=29962]: Add math.remainder operation, implementing remainder as specified in IEEE 754.
- bpo-29649 [https://bugs.python.org/issue?@action=redirect&bpo=29649]: Improve struct.pack_into() exception messages for problems with the buffer size and offset. Patch by Andrew Nester.
- bpo-29654 [https://bugs.python.org/issue?@action=redirect&bpo=29654]: Support If-Modified-Since HTTP header (browser cache). Patch by Pierre Quentel.
- bpo-29931 [https://bugs.python.org/issue?@action=redirect&bpo=29931]: Fixed comparison check for ipaddress.ip_interface objects. Patch by Sanjay Sundaresan.
- bpo-29953 [https://bugs.python.org/issue?@action=redirect&bpo=29953]: Fixed memory leaks in the replace() method of datetime and time objects when pass out of bound fold argument.
- bpo-29942 [https://bugs.python.org/issue?@action=redirect&bpo=29942]: Fix a crash in itertools.chain.from_iterable when encountering long runs of empty iterables.
- bpo-10030 [https://bugs.python.org/issue?@action=redirect&bpo=10030]: Sped up reading encrypted ZIP files by 2 times.
- bpo-29204 [https://bugs.python.org/issue?@action=redirect&bpo=29204]: Element.getiterator() and the html parameter of XMLParser() were deprecated only in the documentation (since Python 3.2 and 3.4 correspondingly). Now using them emits a deprecation warning.
- bpo-27863 [https://bugs.python.org/issue?@action=redirect&bpo=27863]: Fixed multiple crashes in ElementTree caused by race conditions and wrong types.

- bpo-25996 [https://bugs.python.org/issue?@action=redirect&bpo=25996]: Added support of file descriptors in os.scandir() on Unix. os.fwalk() is sped up by 2 times by using os.scandir().
- bpo-28699 [https://bugs.python.org/issue?@action=redirect&bpo=28699]: Fixed a bug in pools in multiprocessing.pool that raising an exception at the very first of an iterable may swallow the exception or make the program hang. Patch by Davin Potts and Xiang Zhang.
- bpo-23890 [https://bugs.python.org/issue?@action=redirect&bpo=23890]: unittest.TestCase.assertRaises() now manually breaks a reference cycle to not keep objects alive longer than expected.
- bpo-29901 [https://bugs.python.org/issue?@action=redirect&bpo=29901]: The zipapp module now supports general path-like objects, not just pathlib.Path.
- bpo-25803 [https://bugs.python.org/issue?@action=redirect&bpo=25803]: Avoid incorrect errors raised by Path.mkdir(exist_ok=True) when the OS gives priority to errors such as EACCES over EEXIST.
- bpo-29861 [https://bugs.python.org/issue?@action=redirect&bpo=29861]: Release references to tasks, their arguments and their results as soon as they are finished in multiprocessing.Pool.
- bpo-19930 [https://bugs.python.org/issue?@action=redirect&bpo=19930]: The mode argument of os.makedirs() no longer affects the file permission bits of newly created intermediate-level directories.
- bpo-29884 [https://bugs.python.org/issue?@action=redirect&bpo=29884]: faulthandler: Restore the old sigaltstack during teardown. Patch by Christophe Zeitouny.
- bpo-25455 [https://bugs.python.org/issue?@action=redirect&bpo=25455]: Fixed crashes in repr of recursive buffered file-like objects.
- bpo-29800 [https://bugs.python.org/issue?@action=redirect&bpo=29800]: Fix crashes in partial.__repr__ if the keys of partial.keywords are not strings. Patch by Michael Seifert.
- bpo-8256 [https://bugs.python.org/issue?@action=redirect&bpo=8256]: Fixed possible failing or crashing input() if attributes “encoding” or “errors” of sys.stdin or sys.stdout are not set or are not strings.
- bpo-28692 [https://bugs.python.org/issue?@action=redirect&bpo=28692]: Using non-integer value for selecting a plural form in gettext is now deprecated.
- bpo-26121 [https://bugs.python.org/issue?@action=redirect&bpo=26121]: Use C library implementation for math functions erf() and erfc().

- bpo-29619 [https://bugs.python.org/issue?@action=redirect&bpo=29619]: os.stat() and os.DirEntry.inode() now convert inode (st_ino) using unsigned integers.
- bpo-28298 [https://bugs.python.org/issue?@action=redirect&bpo=28298]: Fix a bug that prevented array 'Q', 'L' and 'I' from accepting big intables (objects that have __int__) as elements.
- bpo-29645 [https://bugs.python.org/issue?@action=redirect&bpo=29645]: Speed up importing the webbrowser module. webbrowser.register() is now thread-safe.
- bpo-28231 [https://bugs.python.org/issue?@action=redirect&bpo=28231]: The zipfile module now accepts path-like objects for external paths.
- bpo-26915 [https://bugs.python.org/issue?@action=redirect&bpo=26915]: index() and count() methods of collections.abc.Sequence now check identity before checking equality when do comparisons.
- bpo-28682 [https://bugs.python.org/issue?@action=redirect&bpo=28682]: Added support for bytes paths in os.fwalk().
- bpo-29728 [https://bugs.python.org/issue?@action=redirect&bpo=29728]: Add new **`socket.TCP_NOTSENT_LOWAT`** (Linux 3.12) constant. Patch by Nathaniel J. Smith.
- bpo-29623 [https://bugs.python.org/issue?@action=redirect&bpo=29623]: Allow use of path-like object as a single argument in ConfigParser.read(). Patch by David Ellis.
- bpo-9303 [https://bugs.python.org/issue?@action=redirect&bpo=9303]: Migrate sqlite3 module to _v2 API. Patch by Aviv Palivoda.
- bpo-28963 [https://bugs.python.org/issue?@action=redirect&bpo=28963]: Fix out of bound iteration in asyncio.Future.remove_done_callback implemented in C.
- bpo-29704 [https://bugs.python.org/issue?@action=redirect&bpo=29704]: asyncio.subprocess.SubprocessStreamProtocol no longer closes before all pipes are closed.
- bpo-29271 [https://bugs.python.org/issue?@action=redirect&bpo=29271]: Fix Task.current_task and Task.all_tasks implemented in C to accept None argument as their pure Python implementation.
- bpo-29703 [https://bugs.python.org/issue?@action=redirect&bpo=29703]: Fix asyncio to support instantiation of new event loops in child processes.
- bpo-29615 [https://bugs.python.org/issue?@action=redirect&bpo=29615]: SimpleXMLRPCDispatcher no longer chains KeyError (or any other

exception) to exception(s) raised in the dispatched methods. Patch by Petr Motejlek.

- bpo-7769 [https://bugs.python.org/issue?@action=redirect&bpo=7769]: Method register_function() of xmlrpc.server.SimpleXMLRPCDispatcher and its subclasses can now be used as a decorator.
- bpo-29376 [https://bugs.python.org/issue?@action=redirect&bpo=29376]: Fix assertion error in threading._DummyThread.is_alive().
- bpo-28624 [https://bugs.python.org/issue?@action=redirect&bpo=28624]: Add a test that checks that cwd parameter of Popen() accepts PathLike objects. Patch by Sayan Chowdhury.
- bpo-28518 [https://bugs.python.org/issue?@action=redirect&bpo=28518]: Start a transaction implicitly before a DML statement. Patch by Aviv Palivoda.
- bpo-29742 [https://bugs.python.org/issue?@action=redirect&bpo=29742]: get_extra_info() raises exception if get called on closed ssl transport. Patch by Nikolay Kim.
- bpo-16285 [https://bugs.python.org/issue?@action=redirect&bpo=16285]: urllib.parse.quote is now based on RFC 3986 and hence includes '~' in the set of characters that is not quoted by default. Patch by Christian Theune and Ratnadeep Debnath.
- bpo-29532 [https://bugs.python.org/issue?@action=redirect&bpo=29532]: Altering a kwarg dictionary passed to functools.partial() no longer affects a partial object after creation.
- bpo-29110 [https://bugs.python.org/issue?@action=redirect&bpo=29110]: Fix file object leak in aifc.open() when file is given as a filesystem path and is not in valid AIFF format. Patch by Anthony Zhang.
- bpo-22807 [https://bugs.python.org/issue?@action=redirect&bpo=22807]: Add uuid.SafeUUID and uuid.UUID.is_safe to relay information from the platform about whether generated UUIDs are generated with a multiprocessing safe method.
- bpo-29576 [https://bugs.python.org/issue?@action=redirect&bpo=29576]: Improve some deprecations in importlib. Some deprecated methods now emit DeprecationWarnings and have better descriptive messages.
- bpo-29534 [https://bugs.python.org/issue?@action=redirect&bpo=29534]: Fixed different behaviour of Decimal.from_float() for _decimal and _pydecimal. Thanks Andrew Nester.

- bpo-10379 [https://bugs.python.org/issue?@action=redirect&bpo=10379]: locale.format_string now supports the ‘monetary’ keyword argument, and locale.format is deprecated.
- bpo-29851 [https://bugs.python.org/issue?@action=redirect&bpo=29851]: importlib.reload() now raises ModuleNotFoundError if the module lacks a spec.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Various updates to typing module: typing.Counter, typing.ChainMap, improved ABC caching, etc. Original PRs by Jelle Zijlstra, Ivan Levkivskyi, Manuel Krebber, and Łukasz Langa.
- bpo-29100 [https://bugs.python.org/issue?@action=redirect&bpo=29100]: Fix datetime.fromtimestamp() regression introduced in Python 3.6.0: check minimum and maximum years.
- bpo-29416 [https://bugs.python.org/issue?@action=redirect&bpo=29416]: Prevent infinite loop in pathlib.Path.mkdir
- bpo-29444 [https://bugs.python.org/issue?@action=redirect&bpo=29444]: Fixed out-of-bounds buffer access in the group() method of the match object. Based on patch by WGH.
- bpo-29377 [https://bugs.python.org/issue?@action=redirect&bpo=29377]: Add WrapperDescriptorType, MethodWrapperType, and MethodDescriptorType built-in types to types module. Original patch by Manuel Krebber.
- bpo-29218 [https://bugs.python.org/issue?@action=redirect&bpo=29218]: Unused install_misc command is now removed. It has been documented as unused since 2000. Patch by Eric N. Vander Weele.
- bpo-29368 [https://bugs.python.org/issue?@action=redirect&bpo=29368]: The extend() method is now called instead of the append() method when unpickle collections.deque and other list-like objects. This can speed up unpickling to 2 times.
- bpo-29338 [https://bugs.python.org/issue?@action=redirect&bpo=29338]: The help of a builtin or extension class now includes the constructor signature if __text_signature__ is provided for the class.
- bpo-29335 [https://bugs.python.org/issue?@action=redirect&bpo=29335]: Fix subprocess.Popen.wait() when the child process has exited to a stopped instead of terminated state (ex: when under ptrace).
- bpo-29290 [https://bugs.python.org/issue?@action=redirect&bpo=29290]: Fix a regression in argparse that help messages would wrap at non-breaking

spaces.

- bpo-28735 [https://bugs.python.org/issue?@action=redirect&bpo=28735]: Fixed the comparison of mock.MagickMock with mock.ANY.
- bpo-29197 [https://bugs.python.org/issue?@action=redirect&bpo=29197]: Removed deprecated function ntpath.splitunc().
- bpo-29210 [https://bugs.python.org/issue?@action=redirect&bpo=29210]: Removed support of deprecated argument “exclude” in tarfile.TarFile.add().
- bpo-29219 [https://bugs.python.org/issue?@action=redirect&bpo=29219]: Fixed infinite recursion in the repr of uninitialized ctypes.CDLL instances.
- bpo-29192 [https://bugs.python.org/issue?@action=redirect&bpo=29192]: Removed deprecated features in the http.cookies module.
- bpo-29193 [https://bugs.python.org/issue?@action=redirect&bpo=29193]: A format string argument for string.Formatter.format() is now positional-only.
- bpo-29195 [https://bugs.python.org/issue?@action=redirect&bpo=29195]: Removed support of deprecated undocumented keyword arguments in methods of regular expression objects.
- bpo-28969 [https://bugs.python.org/issue?@action=redirect&bpo=28969]: Fixed race condition in C implementation of functools.lru_cache. KeyError could be raised when cached function with full cache was simultaneously called from different threads with the same uncached arguments.
- bpo-20804 [https://bugs.python.org/issue?@action=redirect&bpo=20804]: The unittest.mock.sentinel attributes now preserve their identity when they are copied or pickled.
- bpo-29142 [https://bugs.python.org/issue?@action=redirect&bpo=29142]: In urllib.request, suffixes in no_proxy environment variable with leading dots could match related hostnames again (e.g. .b.c matches a.b.c). Patch by Milan Oberkirch.
- bpo-28961 [https://bugs.python.org/issue?@action=redirect&bpo=28961]: Fix unittest.mock._Call helper: don’t ignore the name parameter anymore. Patch written by Jiajun Huang.
- bpo-15812 [https://bugs.python.org/issue?@action=redirect&bpo=15812]: inspect.getframeinfo() now correctly shows the first line of a context. Patch by Sam Breese.
- bpo-28985 [https://bugs.python.org/issue?@action=redirect&bpo=28985]: Update authorizer constants in sqlite3 module. Patch by Dingyuan Wang.
- bpo-29079 [https://bugs.python.org/issue?@action=redirect&bpo=29079]: Prevent infinite loop in pathlib.resolve() on Windows

- bpo-13051 [https://bugs.python.org/issue?@action=redirect&bpo=13051]: Fixed recursion errors in large or resized curses.textpad.Textbox. Based on patch by Tycho Andersen.
- bpo-9770 [https://bugs.python.org/issue?@action=redirect&bpo=9770]: curses.ascii predicates now work correctly with negative integers.
- bpo-28427 [https://bugs.python.org/issue?@action=redirect&bpo=28427]: old keys should not remove new values from WeakValueDictionary when collecting from another thread.
- bpo-28923 [https://bugs.python.org/issue?@action=redirect&bpo=28923]: Remove editor artifacts from Tix.py.
- bpo-28871 [https://bugs.python.org/issue?@action=redirect&bpo=28871]: Fixed a crash when deallocate deep ElementTree.
- bpo-19542 [https://bugs.python.org/issue?@action=redirect&bpo=19542]: Fix bugs in WeakValueDictionary.setdefault() and WeakValueDictionary.pop() when a GC collection happens in another thread.
- bpo-20191 [https://bugs.python.org/issue?@action=redirect&bpo=20191]: Fixed a crash in resource.prlimit() when passing a sequence that doesn’t own its elements as limits.
- bpo-16255 [https://bugs.python.org/issue?@action=redirect&bpo=16255]: subprocess.Popen uses /system/bin/sh on Android as the shell, instead of /bin/sh.
- bpo-28779 [https://bugs.python.org/issue?@action=redirect&bpo=28779]: multiprocessing.set_forkserver_preload() would crash the forkserver process if a preloaded module instantiated some multiprocessing objects such as locks.
- bpo-26937 [https://bugs.python.org/issue?@action=redirect&bpo=26937]: The chown() method of the tarfile.TarFile class does not fail now when the grp module cannot be imported, as for example on Android platforms.
- bpo-28847 [https://bugs.python.org/issue?@action=redirect&bpo=28847]: dbm.dumb now supports reading read-only files and no longer writes the index file when it is not changed. A deprecation warning is now emitted if the index file is missed and recreated in the ‘r’ and ‘w’ modes (will be an error in future Python releases).
- bpo-27030 [https://bugs.python.org/issue?@action=redirect&bpo=27030]: Unknown escapes consisting of `'\'` and an ASCII letter in re.sub() replacement templates regular expressions now are errors.

- bpo-28835 [https://bugs.python.org/issue?@action=redirect&bpo=28835]: Fix a regression introduced in warnings.catch_warnings(): call warnings.showwarning() if it was overridden inside the context manager.
- bpo-27172 [https://bugs.python.org/issue?@action=redirect&bpo=27172]: To assist with upgrades from 2.7, the previously documented deprecation of `inspect.getfullargspec()` has been reversed. This decision may be revisited again after the Python 2.7 branch is no longer officially supported.
- bpo-28740 [https://bugs.python.org/issue?@action=redirect&bpo=28740]: Add sys.getandroidapilevel(): return the build time API version of Android as an integer. Function only available on Android.
- bpo-26273 [https://bugs.python.org/issue?@action=redirect&bpo=26273]: Add new **`socket.TCP_CONGESTION`** (Linux 2.6.13) and **`socket.TCP_USER_TIMEOUT`** (Linux 2.6.37) constants. Patch written by Omar Sandoval.
- bpo-28752 [https://bugs.python.org/issue?@action=redirect&bpo=28752]: Restored the __reduce__() methods of datetime objects.
- bpo-28727 [https://bugs.python.org/issue?@action=redirect&bpo=28727]: Regular expression patterns, _sre.SRE_Pattern objects created by re.compile(), become comparable (only x==y and x!=y operators). This change should fix the bpo-18383 [https://bugs.python.org/issue?@action=redirect&bpo=18383]: don't duplicate warning filters when the warnings module is reloaded (thing usually only done in unit tests).
- bpo-20572 [https://bugs.python.org/issue?@action=redirect&bpo=20572]: Remove the subprocess.Popen.wait endtime parameter. It was deprecated in 3.4 and undocumented prior to that.
- bpo-25659 [https://bugs.python.org/issue?@action=redirect&bpo=25659]: In ctypes, prevent a crash calling the from_buffer() and from_buffer_copy() methods on abstract classes like Array.
- bpo-28548 [https://bugs.python.org/issue?@action=redirect&bpo=28548]: In the "http.server" module, parse the protocol version if possible, to avoid using HTTP 0.9 in some error responses.
- bpo-19717 [https://bugs.python.org/issue?@action=redirect&bpo=19717]: Makes Path.resolve() succeed on paths that do not exist. Patch by Vajrasky Kok

- bpo-28563 [https://bugs.python.org/issue?@action=redirect&bpo=28563]: Fixed possible DoS and arbitrary code execution when handle plural form selections in the gettext module. The expression parser now supports exact syntax supported by GNU gettext.
- bpo-28387 [https://bugs.python.org/issue?@action=redirect&bpo=28387]: Fixed possible crash in _io.TextIOWrapper deallocator when the garbage collector is invoked in other thread. Based on patch by Sebastian Cufre.
- bpo-27517 [https://bugs.python.org/issue?@action=redirect&bpo=27517]: LZMA compressor and decompressor no longer raise exceptions if given empty data twice. Patch by Benjamin Fogle.
- bpo-28549 [https://bugs.python.org/issue?@action=redirect&bpo=28549]: Fixed segfault in curses's addch() with ncurses6.
- bpo-28449 [https://bugs.python.org/issue?@action=redirect&bpo=28449]: tarfile.open() with mode "r" or "r:" now tries to open a tar file with compression before trying to open it without compression. Otherwise it had 50% chance failed with ignore_zeros=True.
- bpo-23262 [https://bugs.python.org/issue?@action=redirect&bpo=23262]: The webbrowser module now supports Firefox 36+ and derived browsers. Based on patch by Oleg Broytman.
- bpo-24241 [https://bugs.python.org/issue?@action=redirect&bpo=24241]: The webbrowser in an X environment now prefers using the default browser directly. Also, the webbrowser register() function now has a documented 'preferred' argument, to specify browsers to be returned by get() with no arguments. Patch by David Steele
- bpo-27939 [https://bugs.python.org/issue?@action=redirect&bpo=27939]: Fixed bugs in tkinter.ttk.LabeledScale and tkinter.Scale caused by representing the scale as float value internally in Tk. tkinter.IntVar now works if float value is set to underlying Tk variable.
- bpo-28255 [https://bugs.python.org/issue?@action=redirect&bpo=28255]: calendar.TextCalendar.prweek() no longer prints a space after a weeks's calendar. calendar.TextCalendar.pryear() no longer prints redundant newline after a year's calendar. Based on patch by Xiang Zhang.
- bpo-28255 [https://bugs.python.org/issue?@action=redirect&bpo=28255]: calendar.TextCalendar.prmonth() no longer prints a space at the start of new line after printing a month's calendar. Patch by Xiang Zhang.

- bpo-20491 [https://bugs.python.org/issue?@action=redirect&bpo=20491]: The textwrap.TextWrapper class now honors non-breaking spaces. Based on patch by Kaarle Ritvanen.
- bpo-28353 [https://bugs.python.org/issue?@action=redirect&bpo=28353]: os.fwalk() no longer fails on broken links.
- bpo-28430 [https://bugs.python.org/issue?@action=redirect&bpo=28430]: Fix iterator of C implemented asyncio.Future doesn't accept non-None value is passed to it.send(val).
- bpo-27025 [https://bugs.python.org/issue?@action=redirect&bpo=27025]: Generated names for Tkinter widgets now start by the "!" prefix for readability.
- bpo-25464 [https://bugs.python.org/issue?@action=redirect&bpo=25464]: Fixed HList.header_exists() in tkinter.tix module by addin a workaround to Tix library bug.
- bpo-28488 [https://bugs.python.org/issue?@action=redirect&bpo=28488]: shutil.make_archive() no longer adds entry "./" to ZIP archive.
- bpo-25953 [https://bugs.python.org/issue?@action=redirect&bpo=25953]: re.sub() now raises an error for invalid numerical group reference in replacement template even if the pattern is not found in the string. Error message for invalid group reference now includes the group index and the position of the reference. Based on patch by SilentGhost.
- bpo-28469 [https://bugs.python.org/issue?@action=redirect&bpo=28469]: timeit now uses the sequence 1, 2, 5, 10, 20, 50,… instead of 1, 10, 100,… for autoranging.
- bpo-28115 [https://bugs.python.org/issue?@action=redirect&bpo=28115]: Command-line interface of the zipfile module now uses argparse. Added support of long options.
- bpo-18219 [https://bugs.python.org/issue?@action=redirect&bpo=18219]: Optimize csv.DictWriter for large number of columns. Patch by Mariatta Wijaya.
- bpo-28448 [https://bugs.python.org/issue?@action=redirect&bpo=28448]: Fix C implemented asyncio.Future didn't work on Windows.
- bpo-23214 [https://bugs.python.org/issue?@action=redirect&bpo=23214]: In the "io" module, the argument to BufferedReader and BytesIO's read1() methods is now optional and can be -1, matching the BufferedIOBase specification.
- bpo-28480 [https://bugs.python.org/issue?@action=redirect&bpo=28480]: Fix error building socket module when multithreading is disabled.

- bpo-28240 [https://bugs.python.org/issue?@action=redirect&bpo=28240]: timeit: remove `-c/--clock` and `-t/--time` command line options which were deprecated since Python 3.3.
- bpo-28240 [https://bugs.python.org/issue?@action=redirect&bpo=28240]: timeit now repeats the benchmarks 5 times instead of only 3 to make benchmarks more reliable.
- bpo-28240 [https://bugs.python.org/issue?@action=redirect&bpo=28240]: timeit autorange now uses a single loop iteration if the benchmark takes less than 10 seconds, instead of 10 iterations. “python3 -m timeit -s ‘import time’ ‘time.sleep(1)’” now takes 4 seconds instead of 40 seconds.
- Distutils.sdist now looks for README and setup.py files with case sensitivity. This behavior matches that found in Setuptools 6.0 and later. See setuptools 100 [https://github.com/pypa/setuptools/issues/100] for rationale.
- bpo-24452 [https://bugs.python.org/issue?@action=redirect&bpo=24452]: Make webbrowser support Chrome on Mac OS X. Patch by Ned Batchelder.
- bpo-20766 [https://bugs.python.org/issue?@action=redirect&bpo=20766]: Fix references leaked by pdb in the handling of SIGINT handlers.
- bpo-27998 [https://bugs.python.org/issue?@action=redirect&bpo=27998]: Fixed bytes path support in os.scandir() on Windows. Patch by Eryk Sun.
- bpo-28317 [https://bugs.python.org/issue?@action=redirect&bpo=28317]: The disassembler now decodes FORMAT_VALUE argument.
- bpo-28380 [https://bugs.python.org/issue?@action=redirect&bpo=28380]: unittest.mock Mock autospec functions now properly support assert_called, assert_not_called, and assert_called_once.
- bpo-28229 [https://bugs.python.org/issue?@action=redirect&bpo=28229]: lzma module now supports pathlib.
- bpo-28321 [https://bugs.python.org/issue?@action=redirect&bpo=28321]: Fixed writing non-BMP characters with binary format in plistlib.
- bpo-28225 [https://bugs.python.org/issue?@action=redirect&bpo=28225]: bz2 module now supports pathlib. Initial patch by Ethan Furman.
- bpo-28227 [https://bugs.python.org/issue?@action=redirect&bpo=28227]: gzip now supports pathlib. Patch by Ethan Furman.
- bpo-28332 [https://bugs.python.org/issue?@action=redirect&bpo=28332]: Deprecated silent truncations in socket.htons and socket.ntohs. Original patch by Oren Milman.

- bpo-27358 [https://bugs.python.org/issue?@action=redirect&bpo=27358]: Optimized merging var-keyword arguments and improved error message when passing a non-mapping as a var-keyword argument.
- bpo-28257 [https://bugs.python.org/issue?@action=redirect&bpo=28257]: Improved error message when passing a non-iterable as a var-positional argument. Added opcode BUILD_TUPLE_UNPACK_WITH_CALL.
- bpo-28322 [https://bugs.python.org/issue?@action=redirect&bpo=28322]: Fixed possible crashes when unpickle itertools objects from incorrect pickle data. Based on patch by John Leitch.
- bpo-28228 [https://bugs.python.org/issue?@action=redirect&bpo=28228]: imghdr now supports pathlib.
- bpo-28226 [https://bugs.python.org/issue?@action=redirect&bpo=28226]: compileall now supports pathlib.
- bpo-28314 [https://bugs.python.org/issue?@action=redirect&bpo=28314]: Fix function declaration (C flags) for the getiterator() method of xml.etree.ElementTree.Element.
- bpo-28148 [https://bugs.python.org/issue?@action=redirect&bpo=28148]: Stop using localtime() and gmtime() in the time module. Introduced platform independent _PyTime_localtime API that is similar to POSIX localtime_r, but available on all platforms. Patch by Ed Schouten.
- bpo-28253 [https://bugs.python.org/issue?@action=redirect&bpo=28253]: Fixed calendar functions for extreme months: 0001-01 and 9999-12. Methods itermonthdays() and itermonthdays2() are reimplemented so that they don’t call itermonthdates() which can cause datetime.date under/overflow.
- bpo-28275 [https://bugs.python.org/issue?@action=redirect&bpo=28275]: Fixed possible use after free in the decompress() methods of the LZMADecompressor and BZ2Decompressor classes. Original patch by John Leitch.
- bpo-27897 [https://bugs.python.org/issue?@action=redirect&bpo=27897]: Fixed possible crash in sqlite3.Connection.create_collation() if pass invalid string-like object as a name. Patch by Xiang Zhang.
- bpo-18844 [https://bugs.python.org/issue?@action=redirect&bpo=18844]: random.choices() now has k as a keyword-only argument to improve the readability of common cases and come into line with the signature used in other languages.

- bpo-18893 [https://bugs.python.org/issue?@action=redirect&bpo=18893]: Fix invalid exception handling in Lib/ctypes/macholib/dyld.py. Patch by Madison May.
- bpo-27611 [https://bugs.python.org/issue?@action=redirect&bpo=27611]: Fixed support of default root window in the tkinter.tix module. Added the master parameter in the DisplayStyle constructor.
- bpo-27348 [https://bugs.python.org/issue?@action=redirect&bpo=27348]: In the traceback module, restore the formatting of exception messages like “Exception: None”. This fixes a regression introduced in 3.5a2.
- bpo-25651 [https://bugs.python.org/issue?@action=redirect&bpo=25651]: Allow false values to be used for msg parameter of subTest().
- bpo-27778 [https://bugs.python.org/issue?@action=redirect&bpo=27778]: Fix a memory leak in os.getrandom() when the getrandom() is interrupted by a signal and a signal handler raises a Python exception.
- bpo-28200 [https://bugs.python.org/issue?@action=redirect&bpo=28200]: Fix memory leak on Windows in the os module (fix path_converter() function).
- bpo-25400 [https://bugs.python.org/issue?@action=redirect&bpo=25400]: RobotFileParser now correctly returns default values for crawl_delay and request_rate. Initial patch by Peter Wirtz.
- bpo-27932 [https://bugs.python.org/issue?@action=redirect&bpo=27932]: Prevent memory leak in win32_ver().
- Fix UnboundLocalError in socket._sendfile_use_sendfile.
- bpo-28075 [https://bugs.python.org/issue?@action=redirect&bpo=28075]: Check for ERROR_ACCESS_DENIED in Windows implementation of os.stat(). Patch by Eryk Sun.
- bpo-22493 [https://bugs.python.org/issue?@action=redirect&bpo=22493]: Warning message emitted by using inline flags in the middle of regular expression now contains a (truncated) regex pattern. Patch by Tim Graham.
- bpo-25270 [https://bugs.python.org/issue?@action=redirect&bpo=25270]: Prevent codecs.escape_encode() from raising SystemError when an empty bytestring is passed.
- bpo-28181 [https://bugs.python.org/issue?@action=redirect&bpo=28181]: Get antigravity over HTTPS. Patch by Kaartic Sivaraam.
- bpo-25895 [https://bugs.python.org/issue?@action=redirect&bpo=25895]: Enable WebSocket URL schemes in urllib.parse.urljoin. Patch by Gergely

Imreh and Markus Holtermann.

- bpo-28114 [https://bugs.python.org/issue?@action=redirect&bpo=28114]: Fix a crash in parse_envlist() when env contains byte strings. Patch by Eryk Sun.
- bpo-27599 [https://bugs.python.org/issue?@action=redirect&bpo=27599]: Fixed buffer overrun in binascii.b2a_qp() and binascii.a2b_qp().
- bpo-27906 [https://bugs.python.org/issue?@action=redirect&bpo=27906]: Fix socket accept exhaustion during high TCP traffic. Patch by Kevin Conway.
- bpo-28174 [https://bugs.python.org/issue?@action=redirect&bpo=28174]: Handle when SO_REUSEPORT isn't properly supported. Patch by Seth Michael Larson.
- bpo-26654 [https://bugs.python.org/issue?@action=redirect&bpo=26654]: Inspect functools.partial in asyncio.Handle.__repr__. Patch by iceboy.
- bpo-26909 [https://bugs.python.org/issue?@action=redirect&bpo=26909]: Fix slow pipes IO in asyncio. Patch by INADA Naoki.
- bpo-28176 [https://bugs.python.org/issue?@action=redirect&bpo=28176]: Fix callbacks race in asyncio.SelectorLoop.sock_connect.
- bpo-27759 [https://bugs.python.org/issue?@action=redirect&bpo=27759]: Fix selectors incorrectly retain invalid file descriptors. Patch by Mark Williams.
- bpo-28325 [https://bugs.python.org/issue?@action=redirect&bpo=28325]: Remove vestigial MacOS 9 macurl2path module and its tests.
- bpo-28368 [https://bugs.python.org/issue?@action=redirect&bpo=28368]: Refuse monitoring processes if the child watcher has no loop attached. Patch by Vincent Michel.
- bpo-28369 [https://bugs.python.org/issue?@action=redirect&bpo=28369]: Raise RuntimeError when transport's FD is used with add_reader, add_writer, etc.
- bpo-28370 [https://bugs.python.org/issue?@action=redirect&bpo=28370]: Speedup asyncio.StreamReader.readexactly. Patch by Коренберг Марк.
- bpo-28371 [https://bugs.python.org/issue?@action=redirect&bpo=28371]: Deprecate passing asyncio.Handles to run_in_executor.
- bpo-28372 [https://bugs.python.org/issue?@action=redirect&bpo=28372]: Fix asyncio to support formatting of non-python coroutines.
- bpo-28399 [https://bugs.python.org/issue?@action=redirect&bpo=28399]: Remove UNIX socket from FS before binding. Patch by Коренберг Марк.
- bpo-27972 [https://bugs.python.org/issue?@action=redirect&bpo=27972]: Prohibit Tasks to await on themselves.

- bpo-24142 [https://bugs.python.org/issue?@action=redirect&bpo=24142]: Reading a corrupt config file left configparser in an invalid state. Original patch by Florian Höch.
- bpo-29581 [https://bugs.python.org/issue?@action=redirect&bpo=29581]: ABCMeta.__new__ now accepts `**kwargs`, allowing abstract base classes to use keyword parameters in __init_subclass__. Patch by Nate Soares.
- bpo-25532 [https://bugs.python.org/issue?@action=redirect&bpo=25532]: inspect.unwrap() will now only try to unwrap an object sys.getrecursionlimit() times, to protect against objects which create a new object on every attribute access.
- bpo-30177 [https://bugs.python.org/issue?@action=redirect&bpo=30177]: path.resolve(strict=False) no longer cuts the path after the first element not present in the filesystem. Patch by Antoine Pietri.

Documentação

- bpo-31294 [https://bugs.python.org/issue?@action=redirect&bpo=31294]: Fix incomplete code snippet in the ZeroMQSocketListener and ZeroMQSocketHandler examples and adapt them to Python 3.
- bpo-21649 [https://bugs.python.org/issue?@action=redirect&bpo=21649]: Add RFC 7525 and Mozilla server side TLS links to SSL documentation.
- bpo-31128 [https://bugs.python.org/issue?@action=redirect&bpo=31128]: Allow the pydoc server to bind to arbitrary hostnames.
- bpo-30803 [https://bugs.python.org/issue?@action=redirect&bpo=30803]: Clarify doc on truth value testing. Original patch by Peter Thomassen.
- bpo-30176 [https://bugs.python.org/issue?@action=redirect&bpo=30176]: Add missing attribute related constants in curses documentation.
- bpo-30052 [https://bugs.python.org/issue?@action=redirect&bpo=30052]: the link targets for **`bytes()`** and **`bytearray()`** are now their respective type definitions, rather than the corresponding builtin function entries. Use bytes and bytearray to reference the latter. In order to ensure this and future cross-reference updates are applied automatically, the daily documentation builds now disable the default output caching features in Sphinx.
- bpo-26985 [https://bugs.python.org/issue?@action=redirect&bpo=26985]: Add missing info of code object in inspect documentation.

- bpo-19824 [https://bugs.python.org/issue?@action=redirect&bpo=19824]: Improve the documentation for, and links to, template strings by emphasizing their utility for internationalization, and by clarifying some usage constraints. (See also: bpo-20314 [https://bugs.python.org/issue?@action=redirect&bpo=20314], bpo-12518 [https://bugs.python.org/issue?@action=redirect&bpo=12518])
- bpo-28929 [https://bugs.python.org/issue?@action=redirect&bpo=28929]: Link the documentation to its source file on GitHub.
- bpo-25008 [https://bugs.python.org/issue?@action=redirect&bpo=25008]: Document smtpd.py as effectively deprecated and add a pointer to aiosmtpd, a third-party asyncio-based replacement.
- bpo-26355 [https://bugs.python.org/issue?@action=redirect&bpo=26355]: Add canonical header link on each page to corresponding major version of the documentation. Patch by Matthias Bussonnier.
- bpo-29349 [https://bugs.python.org/issue?@action=redirect&bpo=29349]: Fix Python 2 syntax in code for building the documentation.
- bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722]: The data model reference and the porting section in the 3.6 What's New guide now cover the additional `__classcell__` handling needed for custom metaclasses to fully support **PEP 487** [https://peps.python.org/pep-0487/] and zero-argument `super()`.
- bpo-28513 [https://bugs.python.org/issue?@action=redirect&bpo=28513]: Documented command-line interface of zipfile.

Testes

- bpo-29639 [https://bugs.python.org/issue?@action=redirect&bpo=29639]: test.support.HOST is now "localhost", a new HOSTv4 constant has been added for your `127.0.0.1` needs, similar to the existing HOSTv6 constant.
- bpo-31320 [https://bugs.python.org/issue?@action=redirect&bpo=31320]: Silence traceback in test_ssl
- bpo-31346 [https://bugs.python.org/issue?@action=redirect&bpo=31346]: Prefer PROTOCOL_TLS_CLIENT and PROTOCOL_TLS_SERVER protocols for SSLContext.
- bpo-25674 [https://bugs.python.org/issue?@action=redirect&bpo=25674]: Remove sha256.tbs-internet.com ssl test

- bpo-30715 [https://bugs.python.org/issue?@action=redirect&bpo=30715]: Address ALPN callback changes for OpenSSL 1.1.0f. The latest version behaves like OpenSSL 1.0.2 and no longer aborts handshake.
- bpo-30822 [https://bugs.python.org/issue?@action=redirect&bpo=30822]: regrtest: Exclude tzdata from regrtest –all. When running the test suite using –use=all / -u all, exclude tzdata since it makes test_datetime too slow (15-20 min on some buildbots) which then times out on some buildbots. Fix also regrtest command line parser to allow passing -u extralargefile to run test_zipfile64.
- bpo-30695 [https://bugs.python.org/issue?@action=redirect&bpo=30695]: Add the `set_nomemory(start, stop)` and `remove_mem_hooks()` functions to the `_testcapi` module.
- bpo-30357 [https://bugs.python.org/issue?@action=redirect&bpo=30357]: test_thread: setUp() now uses support.threading_setup() and support.threading_cleanup() to wait until threads complete to avoid random side effects on following tests. Initial patch written by Grzegorz Grzywacz.
- bpo-30197 [https://bugs.python.org/issue?@action=redirect&bpo=30197]: Enhanced functions swap_attr() and swap_item() in the test.support module. They now work when delete replaced attribute or item inside the with statement. The old value of the attribute or item (or None if it doesn't exist) now will be assigned to the target of the "as" clause, if there is one.
- bpo-24932 [https://bugs.python.org/issue?@action=redirect&bpo=24932]: Use proper command line parsing in _testembed
- bpo-28950 [https://bugs.python.org/issue?@action=redirect&bpo=28950]: Disallow -j0 to be combined with -T/-l in regrtest command line arguments.
- bpo-28683 [https://bugs.python.org/issue?@action=redirect&bpo=28683]: Fix the tests that bind() a unix socket and raise PermissionError on Android for a non-root user.
- bpo-26936 [https://bugs.python.org/issue?@action=redirect&bpo=26936]: Fix the test_socket failures on Android - getservbyname(), getservbyport() and getaddrinfo() are broken on some Android API levels.
- bpo-28666 [https://bugs.python.org/issue?@action=redirect&bpo=28666]: Now test.support.rmtree is able to remove unwritable or unreadable directories.

- bpo-23839 [https://bugs.python.org/issue?@action=redirect&bpo=23839]: Various caches now are cleared before running every test file.
- bpo-26944 [https://bugs.python.org/issue?@action=redirect&bpo=26944]: Fix test_posix for Android where 'id -G' is entirely wrong or missing the effective gid.
- bpo-28409 [https://bugs.python.org/issue?@action=redirect&bpo=28409]: regrtest: fix the parser of command line arguments.
- bpo-28217 [https://bugs.python.org/issue?@action=redirect&bpo=28217]: Adds _testconsole module to test console input.
- bpo-26939 [https://bugs.python.org/issue?@action=redirect&bpo=26939]: Add the support.setswitchinterval() function to fix test_functools hanging on the Android armv7 qemu emulator.

Construção

- bpo-31354 [https://bugs.python.org/issue?@action=redirect&bpo=31354]: Allow `--with-lto` to be used on all builds, not just `make profile-opt`.
- bpo-31370 [https://bugs.python.org/issue?@action=redirect&bpo=31370]: Remove support for building –without-threads. This option is not really useful anymore in the 21st century. Removing lots of conditional paths allows us to simplify the code base, including in difficult to maintain low-level internal code.
- bpo-31341 [https://bugs.python.org/issue?@action=redirect&bpo=31341]: Per **PEP 11** [https://peps.python.org/pep-0011/], support for the IRIX operating system was removed.
- bpo-30854 [https://bugs.python.org/issue?@action=redirect&bpo=30854]: Fix compile error when compiling –without-threads. Patch by Masayuki Yamamoto.
- bpo-30687 [https://bugs.python.org/issue?@action=redirect&bpo=30687]: Locate msbuild.exe on Windows when building rather than vcvarsall.bat
- bpo-20210 [https://bugs.python.org/issue?@action=redirect&bpo=20210]: Support the *disabled* marker in Setup files. Extension modules listed after this marker are not built at all, neither by the Makefile nor by setup.py.
- bpo-29941 [https://bugs.python.org/issue?@action=redirect&bpo=29941]: Add `--with-assertions` configure flag to explicitly enable C `assert()` checks. Defaults to off. `--with-pydebug` implies `--with-assertions`.

- bpo-28787 [https://bugs.python.org/issue?@action=redirect&bpo=28787]: Fix out-of-tree builds of Python when configured with `--with--dtrace`.
- bpo-29243 [https://bugs.python.org/issue?@action=redirect&bpo=29243]: Prevent unnecessary rebuilding of Python during `make test`, `make install` and some other make targets when configured with `--enable-optimizations`.
- bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404]: Don't regenerate generated files based on file modification time anymore: the action is now explicit. Replace `make touch` with `make regen-all`.
- bpo-29643 [https://bugs.python.org/issue?@action=redirect&bpo=29643]: Fix `--enable-optimization` didn't work.
- bpo-27593 [https://bugs.python.org/issue?@action=redirect&bpo=27593]: sys.version and the platform module python_build(), python_branch(), and python_revision() functions now use git information rather than hg when building from a repo.
- bpo-29572 [https://bugs.python.org/issue?@action=redirect&bpo=29572]: Update Windows build and OS X installers to use OpenSSL 1.0.2k.
- bpo-27659 [https://bugs.python.org/issue?@action=redirect&bpo=27659]: Prohibit implicit C function declarations: use `-Werror=implicit-function-declaration` when possible (GCC and Clang, but it depends on the compiler version). Patch written by Chi Hsuan Yen.
- bpo-29384 [https://bugs.python.org/issue?@action=redirect&bpo=29384]: Remove old Be OS helper scripts.
- bpo-26851 [https://bugs.python.org/issue?@action=redirect&bpo=26851]: Set Android compilation and link flags.
- bpo-28768 [https://bugs.python.org/issue?@action=redirect&bpo=28768]: Fix implicit declaration of function _setmode. Patch by Masayuki Yamamoto
- bpo-29080 [https://bugs.python.org/issue?@action=redirect&bpo=29080]: Removes hard dependency on hg.exe from PCBuild/build.bat
- bpo-23903 [https://bugs.python.org/issue?@action=redirect&bpo=23903]: Added missed names to PC/python3.def.
- bpo-28762 [https://bugs.python.org/issue?@action=redirect&bpo=28762]: lockf() is available on Android API level 24, but the F_LOCK macro is not defined in android-ndk-r13.
- bpo-28538 [https://bugs.python.org/issue?@action=redirect&bpo=28538]: Fix the compilation error that occurs because if_nameindex() is available on Android API level 24, but the if_nameindex structure is not defined.

- bpo-20211 [https://bugs.python.org/issue?@action=redirect&bpo=20211]: Do not add the directory for installing C header files and the directory for installing object code libraries to the cross compilation search paths. Original patch by Thomas Petazzoni.
- bpo-28849 [https://bugs.python.org/issue?@action=redirect&bpo=28849]: Do not define sys.implementation._multiarch on Android.
- bpo-10656 [https://bugs.python.org/issue?@action=redirect&bpo=10656]: Fix out-of-tree building on AIX. Patch by Tristan Carel and Michael Haubenwallner.
- bpo-26359 [https://bugs.python.org/issue?@action=redirect&bpo=26359]: Rename –with-optimiations to –enable-optimizations.
- bpo-28444 [https://bugs.python.org/issue?@action=redirect&bpo=28444]: Fix missing extensions modules when cross compiling.
- bpo-28208 [https://bugs.python.org/issue?@action=redirect&bpo=28208]: Update Windows build and OS X installers to use SQLite 3.14.2.
- bpo-28248 [https://bugs.python.org/issue?@action=redirect&bpo=28248]: Update Windows build and OS X installers to use OpenSSL 1.0.2j.
- bpo-21124 [https://bugs.python.org/issue?@action=redirect&bpo=21124]: Fix building the _struct module on Cygwin by passing `NULL` instead of `&PyType_Type` to PyVarObject_HEAD_INIT. Patch by Masayuki Yamamoto.
- bpo-13756 [https://bugs.python.org/issue?@action=redirect&bpo=13756]: Fix building extensions modules on Cygwin. Patch by Roumen Petrov, based on original patch by Jason Tishler.
- bpo-21085 [https://bugs.python.org/issue?@action=redirect&bpo=21085]: Add configure check for siginfo_t.si_band, which Cygwin does not provide. Patch by Masayuki Yamamoto with review and rebase by Erik Bray.
- bpo-28258 [https://bugs.python.org/issue?@action=redirect&bpo=28258]: Fixed build with Estonian locale (python-config and distclean targets in Makefile). Patch by Arfrever Frehtes Taifersar Arahesis.
- bpo-26661 [https://bugs.python.org/issue?@action=redirect&bpo=26661]: setup.py now detects system libffi with multiarch wrapper.
- bpo-27979 [https://bugs.python.org/issue?@action=redirect&bpo=27979]: A full copy of libffi is no longer bundled for use when building _ctypes on non-OSX UNIX platforms. An installed copy of libffi is now required when building _ctypes on such platforms.

- bpo-15819 [https://bugs.python.org/issue?@action=redirect&bpo=15819]: Remove redundant include search directory option for building outside the source tree.
- bpo-28676 [https://bugs.python.org/issue?@action=redirect&bpo=28676]: Prevent missing 'getentropy' declaration warning on macOS. Patch by Gareth Rees.

Windows

- bpo-31392 [https://bugs.python.org/issue?@action=redirect&bpo=31392]: Update Windows build to use OpenSSL 1.1.0f
- bpo-30389 [https://bugs.python.org/issue?@action=redirect&bpo=30389]: Adds detection of Visual Studio 2017 to distutils on Windows.
- bpo-31358 [https://bugs.python.org/issue?@action=redirect&bpo=31358]: zlib is no longer bundled in the CPython source, instead it is downloaded on demand just like bz2, lzma, OpenSSL, Tcl/Tk, and SQLite.
- bpo-31340 [https://bugs.python.org/issue?@action=redirect&bpo=31340]: Change to building with MSVC v141 (included with Visual Studio 2017)
- bpo-30581 [https://bugs.python.org/issue?@action=redirect&bpo=30581]: os.cpu_count() now returns the correct number of processors on Windows when the number of logical processors is greater than 64.
- bpo-30916 [https://bugs.python.org/issue?@action=redirect&bpo=30916]: Pre-build OpenSSL, Tcl and Tk and include the binaries in the build.
- bpo-30731 [https://bugs.python.org/issue?@action=redirect&bpo=30731]: Add a missing xmlns to python.manifest so that it matches the schema.
- bpo-30291 [https://bugs.python.org/issue?@action=redirect&bpo=30291]: Allow requiring 64-bit interpreters from py.exe using -64 suffix. Contributed by Steve (Gadget) Barnes.
- bpo-30362 [https://bugs.python.org/issue?@action=redirect&bpo=30362]: Adds list options (-0, -0p) to py.exe launcher. Contributed by Steve Barnes.
- bpo-23451 [https://bugs.python.org/issue?@action=redirect&bpo=23451]: Fix socket deprecation warnings in socketmodule.c. Patch by Segev Finer.
- bpo-30450 [https://bugs.python.org/issue?@action=redirect&bpo=30450]: The build process on Windows no longer depends on Subversion, instead pulling external code from GitHub via a Python script. If Python 3.6 is not found on the system (via `py -3.6`), NuGet is used to download a copy of 32-bit Python.

- bpo-29579 [https://bugs.python.org/issue?@action=redirect&bpo=29579]: Removes readme.txt from the installer.
- bpo-25778 [https://bugs.python.org/issue?@action=redirect&bpo=25778]: winreg does not truncate string correctly (Patch by Eryk Sun)
- bpo-28896 [https://bugs.python.org/issue?@action=redirect&bpo=28896]: Deprecate WindowsRegistryFinder and disable it by default
- bpo-28522 [https://bugs.python.org/issue?@action=redirect&bpo=28522]: Fixes mishandled buffer reallocation in getpathp.c
- bpo-28402 [https://bugs.python.org/issue?@action=redirect&bpo=28402]: Adds signed catalog files for stdlib on Windows.
- bpo-28333 [https://bugs.python.org/issue?@action=redirect&bpo=28333]: Enables Unicode for ps1/ps2 and input() prompts. (Patch by Eryk Sun)
- bpo-28251 [https://bugs.python.org/issue?@action=redirect&bpo=28251]: Improvements to help manuals on Windows.
- bpo-28110 [https://bugs.python.org/issue?@action=redirect&bpo=28110]: launcher.msi has different product codes between 32-bit and 64-bit
- bpo-28161 [https://bugs.python.org/issue?@action=redirect&bpo=28161]: Opening CON for write access fails
- bpo-28162 [https://bugs.python.org/issue?@action=redirect&bpo=28162]: WindowsConsoleIO readall() fails if first line starts with Ctrl+Z
- bpo-28163 [https://bugs.python.org/issue?@action=redirect&bpo=28163]: WindowsConsoleIO fileno() passes wrong flags to _open_osfhandle
- bpo-28164 [https://bugs.python.org/issue?@action=redirect&bpo=28164]: _PyIO_get_console_type fails for various paths
- bpo-28137 [https://bugs.python.org/issue?@action=redirect&bpo=28137]: Renames Windows path file to ._pth
- bpo-28138 [https://bugs.python.org/issue?@action=redirect&bpo=28138]: Windows ._pth file should allow import site

IDLE

- bpo-31493 [https://bugs.python.org/issue?@action=redirect&bpo=31493]: IDLE code context – fix code update and font update timers. Canceling timers prevents a warning message when test_idle completes.
- bpo-31488 [https://bugs.python.org/issue?@action=redirect&bpo=31488]: IDLE - Update non-key options in former extension classes. When applying configdialog changes, call .reload for each feature class. Change

ParenMatch so updated options affect existing instances attached to existing editor windows.

- bpo-31477 [https://bugs.python.org/issue?@action=redirect&bpo=31477]: IDLE - Improve rstrip entry in doc. Strip trailing whitespace strips more than blank spaces. Multiline string literals are not skipped.
- bpo-31480 [https://bugs.python.org/issue?@action=redirect&bpo=31480]: IDLE - make tests pass with zzdummy extension disabled by default.
- bpo-31421 [https://bugs.python.org/issue?@action=redirect&bpo=31421]: Document how IDLE runs tkinter programs. IDLE calls tcl/tk update in the background in order to make live interaction and experimentation with tkinter applications much easier.
- bpo-31414 [https://bugs.python.org/issue?@action=redirect&bpo=31414]: IDLE – fix tk entry box tests by deleting first. Adding to an int entry is not the same as deleting and inserting because int(‘’) will fail.
- bpo-31051 [https://bugs.python.org/issue?@action=redirect&bpo=31051]: Rearrange IDLE configdialog GenPage into Window, Editor, and Help sections.
- bpo-30617 [https://bugs.python.org/issue?@action=redirect&bpo=30617]: IDLE - Add docstrings and tests for outwin subclass of editor. Move some data and functions from the class to module level. Patch by Cheryl Sabella.
- bpo-31287 [https://bugs.python.org/issue?@action=redirect&bpo=31287]: IDLE - Do not modify tkinter.message in test_configdialog.
- bpo-27099 [https://bugs.python.org/issue?@action=redirect&bpo=27099]: Convert IDLE’s built-in ‘extensions’ to regular features. About 10 IDLE features were implemented as supposedly optional extensions. Their different behavior could be confusing or worse for users and not good for maintenance. Hence the conversion. The main difference for users is that user configurable key bindings for builtin features are now handled uniformly. Now, editing a binding in a keyset only affects its value in the keyset. All bindings are defined together in the system-specific default keysets in config-extensions.def. All custom keysets are saved as a whole in config-extension.cfg. All take effect as soon as one clicks Apply or Ok. The affected events are ‘<<force-open-completions>>’, ‘<<expand-word>>’, ‘<<force-open-calltip>>’, ‘<<flash-paren>>’, ‘<<format-paragraph>>’, ‘<<run-module>>’, ‘<<check-module>>’, and ‘<<zoom-height>>’. Any (global) customizations made before 3.6.3 will not affect their keyset-specific

customization after 3.6.3. and vice versa. Initial patch by Charles Wohlganger.

- bpo-31206 [https://bugs.python.org/issue?@action=redirect&bpo=31206]: IDLE: Factor HighPage(Frame) class from ConfigDialog. Patch by Cheryl Sabella.
- bpo-31001 [https://bugs.python.org/issue?@action=redirect&bpo=31001]: Add tests for configdialog highlight tab. Patch by Cheryl Sabella.
- bpo-31205 [https://bugs.python.org/issue?@action=redirect&bpo=31205]: IDLE: Factor KeysPage(Frame) class from ConfigDialog. The slightly modified tests continue to pass. Patch by Cheryl Sabella.
- bpo-31130 [https://bugs.python.org/issue?@action=redirect&bpo=31130]: IDLE – stop leaks in test_configdialog. Initial patch by Victor Stinner.
- bpo-31002 [https://bugs.python.org/issue?@action=redirect&bpo=31002]: Add tests for configdialog keys tab. Patch by Cheryl Sabella.
- bpo-19903 [https://bugs.python.org/issue?@action=redirect&bpo=19903]: IDLE: Calltips use `inspect.signature` instead of `inspect.getfullargspec`. This improves calltips for builtins converted to use Argument Clinic. Patch by Louie Lu.
- bpo-31083 [https://bugs.python.org/issue?@action=redirect&bpo=31083]: IDLE - Add an outline of a TabPage class in configdialog. Update existing classes to match outline. Initial patch by Cheryl Sabella.
- bpo-31050 [https://bugs.python.org/issue?@action=redirect&bpo=31050]: Factor GenPage(Frame) class from ConfigDialog. The slightly modified tests continue to pass. Patch by Cheryl Sabella.
- bpo-31004 [https://bugs.python.org/issue?@action=redirect&bpo=31004]: IDLE - Factor FontPage(Frame) class from ConfigDialog. Slightly modified tests continue to pass. Fix General tests. Patch mostly by Cheryl Sabella.
- bpo-30781 [https://bugs.python.org/issue?@action=redirect&bpo=30781]: IDLE - Use ttk widgets in ConfigDialog. Patches by Terry Jan Reedy and Cheryl Sabella.
- bpo-31060 [https://bugs.python.org/issue?@action=redirect&bpo=31060]: IDLE - Finish rearranging methods of ConfigDialog Grouping methods pertaining to each tab and the buttons will aid writing tests and improving the tabs and will enable splitting the groups into classes.
- bpo-30853 [https://bugs.python.org/issue?@action=redirect&bpo=30853]: IDLE – Factor a VarTrace class out of ConfigDialog. Instance tracers manages

pairs consisting of a tk variable and a callback function. When tracing is turned on, setting the variable calls the function. Test coverage for the new class is 100%.

- bpo-31003 [https://bugs.python.org/issue?@action=redirect&bpo=31003]: IDLE: Add more tests for General tab.
- bpo-30993 [https://bugs.python.org/issue?@action=redirect&bpo=30993]: IDLE - Improve configdialog font page and tests. In configdialog: Document causal pathways in create_font_tab docstring. Simplify some attribute names. Move set_samples calls to var_changed_font (idea from Cheryl Sabella). Move related functions to positions after the create widgets function. In test_configdialog: Fix test_font_set so not order dependent. Fix renamed test_indent_scale so it tests the widget. Adjust tests for movement of set_samples call. Add tests for load functions. Put all font tests in one class and tab indent tests in another. Except for two lines, these tests completely cover the related functions.
- bpo-30981 [https://bugs.python.org/issue?@action=redirect&bpo=30981]: IDLE – Add more configdialog font page tests.
- bpo-28523 [https://bugs.python.org/issue?@action=redirect&bpo=28523]: IDLE: replace ‘colour’ with ‘color’ in configdialog.
- bpo-30917 [https://bugs.python.org/issue?@action=redirect&bpo=30917]: Add tests for idlelib.config.IdleConf. Increase coverage from 46% to 96%. Patch by Louie Lu.
- bpo-30934 [https://bugs.python.org/issue?@action=redirect&bpo=30934]: Document coverage details for idlelib tests. Add section to idlelib/idle-test/README.txt. Include check that branches are taken both ways. Exclude IDLE-specific code that does not run during unit tests.
- bpo-30913 [https://bugs.python.org/issue?@action=redirect&bpo=30913]: IDLE: Document ConfigDialog tk Vars, methods, and widgets in docstrings This will facilitate improving the dialog and splitting up the class. Original patch by Cheryl Sabella.
- bpo-30899 [https://bugs.python.org/issue?@action=redirect&bpo=30899]: IDLE: Add tests for ConfigParser subclasses in config. Patch by Louie Lu.
- bpo-30881 [https://bugs.python.org/issue?@action=redirect&bpo=30881]: IDLE: Add docstrings to browser.py. Patch by Cheryl Sabella.
- bpo-30851 [https://bugs.python.org/issue?@action=redirect&bpo=30851]: IDLE: Remove unused variables in configdialog. One is a duplicate, one is set but cannot be altered by users. Patch by Cheryl Sabella.

- bpo-30870 [https://bugs.python.org/issue?@action=redirect&bpo=30870]: IDLE: In Settings dialog, select font with Up, Down keys as well as mouse. Initial patch by Louie Lu.
- bpo-8231 [https://bugs.python.org/issue?@action=redirect&bpo=8231]: IDLE: call config.IdleConf.GetUserCfgDir only once.
- bpo-30779 [https://bugs.python.org/issue?@action=redirect&bpo=30779]: IDLE: Factor ConfigChanges class from configdialog, put in config; test. * In config, put dump test code in a function; run it and unittest in ‘if __name__ == ‘__main__’. * Add class config.ConfigChanges based on changes_class_v4.py on bpo issue. * Add class test_config.ChangesTest, partly using configdialog_tests_v1.py. * Revise configdialog to use ConfigChanges; see tracker msg297804. * Revise test_configdialog to match configdialog changes. * Remove configdialog functions unused or moved to ConfigChanges. Cheryl Sabella contributed parts of the patch.
- bpo-30777 [https://bugs.python.org/issue?@action=redirect&bpo=30777]: IDLE: configdialog - Add docstrings and fix comments. Patch by Cheryl Sabella.
- bpo-30495 [https://bugs.python.org/issue?@action=redirect&bpo=30495]: IDLE: Improve textview with docstrings, PEP8 names, and more tests. Patch by Cheryl Sabella.
- bpo-30723 [https://bugs.python.org/issue?@action=redirect&bpo=30723]: IDLE: Make several improvements to parenmatch. Add ‘parens’ style to highlight both opener and closer. Make ‘default’ style, which is not default, a synonym for ‘opener’. Make time-delay work the same with all styles. Add help for config dialog extensions tab, including help for parenmatch. Add new tests. Original patch by Charles Wohlganger.
- bpo-30674 [https://bugs.python.org/issue?@action=redirect&bpo=30674]: IDLE: add docstrings to grep module. Patch by Cheryl Sabella
- bpo-21519 [https://bugs.python.org/issue?@action=redirect&bpo=21519]: IDLE’s basic custom key entry dialog now detects duplicates properly. Original patch by Saimadhav Heblikar.
- bpo-29910 [https://bugs.python.org/issue?@action=redirect&bpo=29910]: IDLE no longer deletes a character after commenting out a region by a key shortcut. Add `return 'break'` for this and other potential conflicts between IDLE and default key bindings.

- bpo-30728 [https://bugs.python.org/issue?@action=redirect&bpo=30728]: Review and change idlelib.configdialog names. Lowercase method and attribute names. Replace ‘colour’ with ‘color’, expand overly cryptic names, delete unneeded underscores. Replace `import *` with specific imports. Patches by Cheryl Sabella.
- bpo-6739 [https://bugs.python.org/issue?@action=redirect&bpo=6739]: IDLE: Verify user-entered key sequences by trying to bind them with tk. Add tests for all 3 validation functions. Original patch by G Polo. Tests added by Cheryl Sabella.
- bpo-15786 [https://bugs.python.org/issue?@action=redirect&bpo=15786]: Fix several problems with IDLE’s autocompletion box. The following should now work: clicking on selection box items; using the scrollbar; selecting an item by hitting Return. Hangs on MacOSX should no longer happen. Patch by Louie Lu.
- bpo-25514 [https://bugs.python.org/issue?@action=redirect&bpo=25514]: Add doc subsubsection about IDLE failure to start. Popup no-connection message directs users to this section.
- bpo-30642 [https://bugs.python.org/issue?@action=redirect&bpo=30642]: Fix reference leaks in IDLE tests. Patches by Louie Lu and Terry Jan Reedy.
- bpo-30495 [https://bugs.python.org/issue?@action=redirect&bpo=30495]: Add docstrings for textview.py and use PEP8 names. Patches by Cheryl Sabella and Terry Jan Reedy.
- bpo-30290 [https://bugs.python.org/issue?@action=redirect&bpo=30290]: Help-about: use pep8 names and add tests. Increase coverage to 100%. Patches by Louie Lu, Cheryl Sabella, and Terry Jan Reedy.
- bpo-30303 [https://bugs.python.org/issue?@action=redirect&bpo=30303]: Add _utest option to textview; add new tests. Increase coverage to 100%. Patches by Louie Lu and Terry Jan Reedy.
- bpo-29071 [https://bugs.python.org/issue?@action=redirect&bpo=29071]: IDLE colors f-string prefixes (but not invalid ur prefixes).
- bpo-28572 [https://bugs.python.org/issue?@action=redirect&bpo=28572]: Add 10% to coverage of IDLE’s test_configdialog. Update and augment description of the configuration system.

Ferramentas/Demos

- bpo-30983 [https://bugs.python.org/issue?@action=redirect&bpo=30983]: gdb integration commands (py-bt, etc.) work on optimized shared builds now, too. **PEP 523** [https://peps.python.org/pep-0523/] introduced _PyEval_EvalFrameDefault which inlines PyEval_EvalFrameEx on non-debug shared builds. This broke the ability to use py-bt, py-up, and a few other Python-specific gdb integrations. The problem is fixed by only looking for _PyEval_EvalFrameDefault frames in python-gdb.py. Original patch by Bruno "Polaco" Penteado.
- bpo-29748 [https://bugs.python.org/issue?@action=redirect&bpo=29748]: Added the slice index converter in Argument Clinic.
- bpo-24037 [https://bugs.python.org/issue?@action=redirect&bpo=24037]: Argument Clinic now uses the converter `bool(accept={int})` rather than `int` for semantical booleans. This avoids repeating the default value for Python and C and will help in converting to `bool` in future.
- bpo-29367 [https://bugs.python.org/issue?@action=redirect&bpo=29367]: python-gdb.py now supports also `method-wrapper` (`wrapperobject`) objects.
- bpo-28023 [https://bugs.python.org/issue?@action=redirect&bpo=28023]: Fix python-gdb.py didn't support new dict implementation.
- bpo-15369 [https://bugs.python.org/issue?@action=redirect&bpo=15369]: The pybench and pystone microbenchmark have been removed from Tools. Please use the new Python benchmark suite https://github.com/python/pyperformance which is more reliable and includes a portable version of pybench working on Python 2 and Python 3.
- bpo-28102 [https://bugs.python.org/issue?@action=redirect&bpo=28102]: The zipfile module CLI now prints usage to stderr. Patch by Stephen J. Turnbull.

API C

- bpo-31338 [https://bugs.python.org/issue?@action=redirect&bpo=31338]: Added the `Py_UNREACHABLE()` macro for code paths which are never expected to be reached. This and a few other useful macros are now documented in the C API manual.
- bpo-30832 [https://bugs.python.org/issue?@action=redirect&bpo=30832]: Remove own implementation for thread-local storage. CPython has provided the own implementation for thread-local storage (TLS) on Python/thread.c, it's used in the case which a platform has not supplied

native TLS. However, currently all supported platforms (Windows and pthreads) have provided native TLS and defined the Py_HAVE_NATIVE_TLS macro with unconditional in any case.

- bpo-30708 [https://bugs.python.org/issue?@action=redirect&bpo=30708]: PyUnicode_AsWideCharString() now raises a ValueError if the second argument is NULL and the wchar_t* string contains null characters.
- bpo-16500 [https://bugs.python.org/issue?@action=redirect&bpo=16500]: Deprecate PyOS_AfterFork() and add PyOS_BeforeFork(), PyOS_AfterFork_Parent() and PyOS_AfterFork_Child().
- bpo-6532 [https://bugs.python.org/issue?@action=redirect&bpo=6532]: The type of results of PyThread_start_new_thread() and PyThread_get_thread_ident(), and the id parameter of PyThreadState_SetAsyncExc() changed from “long” to “unsigned long”.
- bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867]: Function PySlice_GetIndicesEx() is deprecated and replaced with a macro if Py_LIMITED_API is not set or set to the value between 0x03050400 and 0x03060000 (not including) or 0x03060100 or higher. Added functions PySlice_Unpack() and PySlice_AdjustIndices().
- bpo-29083 [https://bugs.python.org/issue?@action=redirect&bpo=29083]: Fixed the declaration of some public API functions. PyArg_VaParse() and PyArg_VaParseTupleAndKeywords() were not available in limited API. PyArg_ValidateKeywordArguments(), PyArg_UnpackTuple() and Py_BuildValue() were not available in limited API of version < 3.3 when PY_SSIZE_T_CLEAN is defined.
- bpo-28769 [https://bugs.python.org/issue?@action=redirect&bpo=28769]: The result of PyUnicode_AsUTF8AndSize() and PyUnicode_AsUTF8() is now of type `const char *` rather of `char *`.
- bpo-29058 [https://bugs.python.org/issue?@action=redirect&bpo=29058]: All stable API extensions added after Python 3.2 are now available only when Py_LIMITED_API is set to the PY_VERSION_HEX value of the minimum Python version supporting this API.
- bpo-28822 [https://bugs.python.org/issue?@action=redirect&bpo=28822]: The index parameters *start* and *end* of PyUnicode_FindChar() are now adjusted to behave like `str[start:end]`.
- bpo-28808 [https://bugs.python.org/issue?@action=redirect&bpo=28808]: PyUnicode_CompareWithASCIIString() now never raises exceptions.

- bpo-28761 [https://bugs.python.org/issue?@action=redirect&bpo=28761]: The fields name and doc of structures PyMemberDef, PyGetSetDef, PyStructSequence_Field, PyStructSequence_Desc, and wrapperbase are now of type `const char *` rather of `char *`.
- bpo-28748 [https://bugs.python.org/issue?@action=redirect&bpo=28748]: Private variable _Py_PackageContext is now of type `const char *` rather of `char *`.
- bpo-19569 [https://bugs.python.org/issue?@action=redirect&bpo=19569]: Compiler warnings are now emitted if use most of deprecated functions.
- bpo-28426 [https://bugs.python.org/issue?@action=redirect&bpo=28426]: Deprecated undocumented functions PyUnicode_AsEncodedObject(), PyUnicode_AsDecodedObject(), PyUnicode_AsDecodedUnicode() and PyUnicode_AsEncodedUnicode().

Python 3.6.6 final

Data de lançamento: 2018-06-27

There were no new changes in version 3.6.6.

Python 3.6.6 release candidate 1

Data de lançamento: 2018-06-11

Núcleo e embutidos

- bpo-33786 [https://bugs.python.org/issue?@action=redirect&bpo=33786]: Fix asynchronous generators to handle GeneratorExit in athrow() correctly
- bpo-30654 [https://bugs.python.org/issue?@action=redirect&bpo=30654]: Fixed reset of the SIGINT handler to SIG_DFL on interpreter shutdown even when there was a custom handler set previously. Patch by Philipp Kerling.
- bpo-33622 [https://bugs.python.org/issue?@action=redirect&bpo=33622]: Fixed a leak when the garbage collector fails to add an object with the `__del__` method or referenced by it into the **`gc.garbage`** list. **`PyGC_Collect()`** can now be called when an exception is set and preserves it.

- bpo-31849 [https://bugs.python.org/issue?@action=redirect&bpo=31849]: Fix signed/unsigned comparison warning in pyhash.c.
- bpo-33391 [https://bugs.python.org/issue?@action=redirect&bpo=33391]: Fix a leak in set_symmetric_difference().
- bpo-28055 [https://bugs.python.org/issue?@action=redirect&bpo=28055]: Fix unaligned accesses in siphash24(). Patch by Rolf Eike Beer.
- bpo-33231 [https://bugs.python.org/issue?@action=redirect&bpo=33231]: Fix potential memory leak in `normalizestring()`.
- bpo-29922 [https://bugs.python.org/issue?@action=redirect&bpo=29922]: Improved error messages in 'async with' when `__aenter__()` or `__aexit__()` return non-awaitable object.
- bpo-33199 [https://bugs.python.org/issue?@action=redirect&bpo=33199]: Fix `ma_version_tag` in dict implementation is uninitialized when copying from key-sharing dict.
- bpo-33041 [https://bugs.python.org/issue?@action=redirect&bpo=33041]: Fixed jumping when the function contains an `async for` loop.
- bpo-32282 [https://bugs.python.org/issue?@action=redirect&bpo=32282]: Fix an unnecessary ifdef in the include of VersionHelpers.h in socketmodule on Windows.
- bpo-21983 [https://bugs.python.org/issue?@action=redirect&bpo=21983]: Fix a crash in `ctypes.cast()` in case the type argument is a ctypes structured data type. Patch by Eryk Sun and Oren Milman.

Biblioteca

- bpo-30167 [https://bugs.python.org/issue?@action=redirect&bpo=30167]: Prevent site.main() exception if PYTHONSTARTUP is set. Patch by Steve Weber.
- bpo-33812 [https://bugs.python.org/issue?@action=redirect&bpo=33812]: Datetime instance d with non-None tzinfo, but with d.tzinfo.utcoffset(d) returning None is now treated as naive by the astimezone() method.
- bpo-30805 [https://bugs.python.org/issue?@action=redirect&bpo=30805]: Avoid race condition with debug logging
- bpo-33767 [https://bugs.python.org/issue?@action=redirect&bpo=33767]: The concatenation (`+`) and repetition (`*`) sequence operations now raise **`TypeError`** instead of **`SystemError`** when performed on **`mmap.mmap`** objects. Patch by Zackery Spytz.

- bpo-32684 [https://bugs.python.org/issue?@action=redirect&bpo=32684]: Fix gather to propagate cancellation of itself even with return_exceptions.
- bpo-33674 [https://bugs.python.org/issue?@action=redirect&bpo=33674]: Fix a race condition in SSLProtocol.connection_made() of asyncio.sslproto: start immediately the handshake instead of using call_soon(). Previously, data_received() could be called before the handshake started, causing the handshake to hang or fail.
- bpo-31647 [https://bugs.python.org/issue?@action=redirect&bpo=31647]: Fixed bug where calling write_eof() on a _SelectorSocketTransport after it's already closed raises AttributeError.
- bpo-33672 [https://bugs.python.org/issue?@action=redirect&bpo=33672]: Fix Task.__repr__ crash with Cython's bogus coroutines
- bpo-33469 [https://bugs.python.org/issue?@action=redirect&bpo=33469]: Fix RuntimeError after closing loop that used run_in_executor
- bpo-11874 [https://bugs.python.org/issue?@action=redirect&bpo=11874]: Use a better regex when breaking usage into wrappable parts. Avoids bogus assertion errors from custom metavar strings.
- bpo-30877 [https://bugs.python.org/issue?@action=redirect&bpo=30877]: Fixed a bug in the Python implementation of the JSON decoder that prevented the cache of parsed strings from clearing after finishing the decoding. Based on patch by c-fos.
- bpo-33548 [https://bugs.python.org/issue?@action=redirect&bpo=33548]: tempfile._candidate_tempdir_list should consider common TEMP locations
- bpo-33542 [https://bugs.python.org/issue?@action=redirect&bpo=33542]: Prevent `uuid.get_node` from using a DUID instead of a MAC on Windows. Patch by Zvi Effron
- bpo-26819 [https://bugs.python.org/issue?@action=redirect&bpo=26819]: Fix race condition with `ReadTransport.resume_reading` in Windows proactor event loop.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Minor fixes in typing module: add annotations to `NamedTuple.__new__`, pass `*args` and `**kwds` in `Generic.__new__`. Original PRs by Paulius Šarka and Chad Dombrova.
- bpo-20087 [https://bugs.python.org/issue?@action=redirect&bpo=20087]: Updated alias mapping with glibc 2.27 supported locales.

- bpo-33422 [https://bugs.python.org/issue?@action=redirect&bpo=33422]: Fix trailing quotation marks getting deleted when looking up byte/string literals on pydoc. Patch by Andrés Delfino.
- bpo-33197 [https://bugs.python.org/issue?@action=redirect&bpo=33197]: Update error message when constructing invalid inspect.Parameters Patch by Donghee Na.
- bpo-33383 [https://bugs.python.org/issue?@action=redirect&bpo=33383]: Fixed crash in the get() method of the **`dbm.ndbm`** database object when it is called with a single argument.
- bpo-33329 [https://bugs.python.org/issue?@action=redirect&bpo=33329]: Fix multiprocessing regression on newer glibcs
- bpo-991266 [https://bugs.python.org/issue?@action=redirect&bpo=991266]: Fix quoting of the `Comment` attribute of **`http.cookies.SimpleCookie`**.
- bpo-33131 [https://bugs.python.org/issue?@action=redirect&bpo=33131]: Upgrade bundled version of pip to 10.0.1.
- bpo-33308 [https://bugs.python.org/issue?@action=redirect&bpo=33308]: Fixed a crash in the **`parser`** module when converting an ST object to a tree of tuples or lists with `line_info=False` and `col_info=True`.
- bpo-33263 [https://bugs.python.org/issue?@action=redirect&bpo=33263]: Fix FD leak in `_SelectorSocketTransport` Patch by Vlad Starostin.
- bpo-33256 [https://bugs.python.org/issue?@action=redirect&bpo=33256]: Fix display of `<module>` call in the html produced by `cgitb.html()`. Patch by Stéphane Blondon.
- bpo-33203 [https://bugs.python.org/issue?@action=redirect&bpo=33203]: `random.Random.choice()` now raises `IndexError` for empty sequences consistently even when called from subclasses without a `getrandbits()` implementation.
- bpo-33224 [https://bugs.python.org/issue?@action=redirect&bpo=33224]: Update difflib.mdiff() for **PEP 479** [https://peps.python.org/pep-0479/]. Convert an uncaught StopIteration in a generator into a return-statement.
- bpo-33209 [https://bugs.python.org/issue?@action=redirect&bpo=33209]: End framing at the end of C implementation of **`pickle.Pickler.dump()`**.
- bpo-32861 [https://bugs.python.org/issue?@action=redirect&bpo=32861]: The urllib.robotparser's `__str__` representation now includes wildcard entries and the "Crawl-delay" and "Request-rate" fields. Patch by Michael Lazar.

- bpo-33096 [https://bugs.python.org/issue?@action=redirect&bpo=33096]: Allow ttk.Treeview.insert to insert iid that has a false boolean value. Note iid=0 and iid=False would be same. Patch by Garvit Khatri.
- bpo-33127 [https://bugs.python.org/issue?@action=redirect&bpo=33127]: The ssl module now compiles with LibreSSL 2.7.1.
- bpo-33021 [https://bugs.python.org/issue?@action=redirect&bpo=33021]: Release the GIL during fstat() calls, avoiding hang of all threads when calling mmap.mmap(), os.urandom(), and random.seed(). Patch by Nir Soffer.
- bpo-27683 [https://bugs.python.org/issue?@action=redirect&bpo=27683]: Fix a regression in **`ipaddress`** that result of **`hosts()`** is empty when the network is constructed by a tuple containing an integer mask and only 1 bit left for addresses.
- bpo-32844 [https://bugs.python.org/issue?@action=redirect&bpo=32844]: Fix wrong redirection of a low descriptor (0 or 1) to stderr in subprocess if another low descriptor is closed.
- bpo-31908 [https://bugs.python.org/issue?@action=redirect&bpo=31908]: Fix output of cover files for `trace` module command-line tool. Previously emitted cover files only when `--missing` option was used. Patch by Michael Selik.
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: If nested log adapters are used, the inner `process()` methods are no longer omitted.
- bpo-16865 [https://bugs.python.org/issue?@action=redirect&bpo=16865]: Support arrays >=2GiB in **`ctypes`**. Patch by Segev Finer.
- bpo-31238 [https://bugs.python.org/issue?@action=redirect&bpo=31238]: pydoc: the stop() method of the private ServerThread class now waits until DocServer.serve_until_quit() completes and then explicitly sets its docserver attribute to None to break a reference cycle.

Documentação

- bpo-33503 [https://bugs.python.org/issue?@action=redirect&bpo=33503]: Fix broken pypi link
- bpo-33421 [https://bugs.python.org/issue?@action=redirect&bpo=33421]: Add missing documentation for `typing.AsyncContextManager`.
- bpo-33378 [https://bugs.python.org/issue?@action=redirect&bpo=33378]: Add Korean language switcher for https://docs.python.org/3/

- bpo-33276 [https://bugs.python.org/issue?@action=redirect&bpo=33276]: Clarify that the `__path__` attribute on modules cannot be just any value.
- bpo-33201 [https://bugs.python.org/issue?@action=redirect&bpo=33201]: Modernize documentation for writing C extension types.
- bpo-33195 [https://bugs.python.org/issue?@action=redirect&bpo=33195]: Deprecate `Py_UNICODE` usage in `c-api/arg` document. `Py_UNICODE` related APIs are deprecated since Python 3.3, but it is missed in the document.
- bpo-33126 [https://bugs.python.org/issue?@action=redirect&bpo=33126]: Document PyBuffer_ToContiguous().
- bpo-27212 [https://bugs.python.org/issue?@action=redirect&bpo=27212]: Modify documentation for the **`islice()`** recipe to consume initial values up to the start index.
- bpo-28247 [https://bugs.python.org/issue?@action=redirect&bpo=28247]: Update **`zipapp`** documentation to describe how to make standalone applications.
- bpo-18802 [https://bugs.python.org/issue?@action=redirect&bpo=18802]: Documentation changes for ipaddress. Patch by Jon Foster and Berker Peksag.
- bpo-27428 [https://bugs.python.org/issue?@action=redirect&bpo=27428]: Update documentation to clarify that `WindowsRegistryFinder` implements `MetaPathFinder`. (Patch by Himanshu Lakhara)
- bpo-8243 [https://bugs.python.org/issue?@action=redirect&bpo=8243]: Add a note about curses.addch and curses.addstr exception behavior when writing outside a window, or pad.
- bpo-31432 [https://bugs.python.org/issue?@action=redirect&bpo=31432]: Clarify meaning of CERT_NONE, CERT_OPTIONAL, and CERT_REQUIRED flags for ssl.SSLContext.verify_mode.

Testes

- bpo-33655 [https://bugs.python.org/issue?@action=redirect&bpo=33655]: Ignore test_posix_fallocate failures on BSD platforms that might be due to running on ZFS.
- bpo-19417 [https://bugs.python.org/issue?@action=redirect&bpo=19417]: Add test_bdb.py.

Construção

- bpo-5755 [https://bugs.python.org/issue?@action=redirect&bpo=5755]: Move `-Wstrict-prototypes` option to `CFLAGS_NODIST` from `OPT`. This option emitted annoying warnings when building extension modules written in C++.
- bpo-33614 [https://bugs.python.org/issue?@action=redirect&bpo=33614]: Ensures module definition files for the stable ABI on Windows are correctly regenerated.
- bpo-33522 [https://bugs.python.org/issue?@action=redirect&bpo=33522]: Enable CI builds on Visual Studio Team Services at https://python.visualstudio.com/cpython
- bpo-33012 [https://bugs.python.org/issue?@action=redirect&bpo=33012]: Add `-Wno-cast-function-type` for gcc 8 for silencing warnings about function casts like casting to PyCFunction in method definition lists.
- bpo-33394 [https://bugs.python.org/issue?@action=redirect&bpo=33394]: Enable the verbose build for extension modules, when GNU make is passed macros on the command line.

Windows

- bpo-33184 [https://bugs.python.org/issue?@action=redirect&bpo=33184]: Update Windows installer to OpenSSL 1.0.2o.

macOS

- bpo-33184 [https://bugs.python.org/issue?@action=redirect&bpo=33184]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.0.2o.

IDLE

- bpo-33656 [https://bugs.python.org/issue?@action=redirect&bpo=33656]: On Windows, add API call saying that tk scales for DPI. On Windows 8.1+ or 10, with DPI compatibility properties of the Python binary unchanged, and a monitor resolution greater than 96 DPI, this should make text and lines sharper. It should otherwise have no effect.
- bpo-33768 [https://bugs.python.org/issue?@action=redirect&bpo=33768]: Clicking on a context line moves that line to the top of the editor window.

- bpo-33763 [https://bugs.python.org/issue?@action=redirect&bpo=33763]: IDLE: Use read-only text widget for code context instead of label widget.
- bpo-33664 [https://bugs.python.org/issue?@action=redirect&bpo=33664]: Scroll IDLE editor text by lines. Previously, the mouse wheel and scrollbar slider moved text by a fixed number of pixels, resulting in partial lines at the top of the editor box. The change also applies to the shell and grep output windows, but not to read-only text views.
- bpo-33679 [https://bugs.python.org/issue?@action=redirect&bpo=33679]: Enable theme-specific color configuration for Code Context. Use the Highlights tab to see the setting for built-in themes or add settings to custom themes.
- bpo-33642 [https://bugs.python.org/issue?@action=redirect&bpo=33642]: Display up to maxlines non-blank lines for Code Context. If there is no current context, show a single blank line.
- bpo-33628 [https://bugs.python.org/issue?@action=redirect&bpo=33628]: IDLE: Cleanup codecontext.py and its test.
- bpo-33564 [https://bugs.python.org/issue?@action=redirect&bpo=33564]: IDLE's code context now recognizes async as a block opener.
- bpo-29706 [https://bugs.python.org/issue?@action=redirect&bpo=29706]: IDLE now colors async and await as keywords in 3.6. They become full keywords in 3.7.
- bpo-21474 [https://bugs.python.org/issue?@action=redirect&bpo=21474]: Update word/identifier definition from ascii to unicode. In text and entry boxes, this affects selection by double-click, movement left/right by control-left/right, and deletion left/right by control-BACKSPACE/DEL.
- bpo-33204 [https://bugs.python.org/issue?@action=redirect&bpo=33204]: IDLE: consistently color invalid string prefixes. A 'u' string prefix cannot be paired with either 'r' or 'f'. Consistently color as much of the prefix, starting at the right, as is valid. Revise and extend colorizer test.
- bpo-32831 [https://bugs.python.org/issue?@action=redirect&bpo=32831]: Add docstrings and tests for codecontext.

Ferramentas/Demos

- bpo-33189 [https://bugs.python.org/issue?@action=redirect&bpo=33189]: **pygettext.py** now recognizes only literal strings as docstrings and

translatable strings, and rejects bytes literals and f-string expressions.

- bpo-31920 [https://bugs.python.org/issue?@action=redirect&bpo=31920]: Fixed handling directories as arguments in the `pygettext` script. Based on patch by Oleg Krasnikov.
- bpo-29673 [https://bugs.python.org/issue?@action=redirect&bpo=29673]: Fix pystackv and pystack gdbinit macros.
- bpo-32885 [https://bugs.python.org/issue?@action=redirect&bpo=32885]: Add an `-n` flag for `Tools/scripts/pathfix.py` to disable automatic backup creation (files with ~ suffix).
- bpo-31583 [https://bugs.python.org/issue?@action=redirect&bpo=31583]: Fix 2to3 for using with –add-suffix option but without –output-dir option for relative path to files in current directory.

API C

- bpo-32374 [https://bugs.python.org/issue?@action=redirect&bpo=32374]: Document that m_traverse for multi-phase initialized modules can be called with m_state=NULL, and add a sanity check

Python 3.6.5 final

Data de lançamento: 2018-03-28

Testes

- bpo-32872 [https://bugs.python.org/issue?@action=redirect&bpo=32872]: Avoid regrtest compatibility issue with namespace packages.

Construção

- bpo-33163 [https://bugs.python.org/issue?@action=redirect&bpo=33163]: Upgrade pip to 9.0.3 and setuptools to v39.0.1.

Python 3.6.5 release candidate 1

Data de lançamento: 2018-03-13

Segurança

- bpo-33001 [https://bugs.python.org/issue?@action=redirect&bpo=33001]: Minimal fix to prevent buffer overrun in os.symlink on Windows
- bpo-32981 [https://bugs.python.org/issue?@action=redirect&bpo=32981]: Regexes in difflib and poplib were vulnerable to catastrophic backtracking. These regexes formed potential DOS vectors (REDOS). They have been refactored. This resolves **CVE 2018-1060** [https://www.cve.org/CVERecord?id=CVE-2018-1060] and **CVE 2018-1061** [https://www.cve.org/CVERecord?id=CVE-2018-1061]. Patch by Jamie Davis.

Núcleo e embutidos

- bpo-33026 [https://bugs.python.org/issue?@action=redirect&bpo=33026]: Fixed jumping out of “with” block by setting f_lineno.
- bpo-17288 [https://bugs.python.org/issue?@action=redirect&bpo=17288]: Prevent jumps from ‘return’ and ‘exception’ trace events.
- bpo-32889 [https://bugs.python.org/issue?@action=redirect&bpo=32889]: Update Valgrind suppression list to account for the rename of `Py_ADDRESS_IN_RANG` to `address_in_range`.
- bpo-32650 [https://bugs.python.org/issue?@action=redirect&bpo=32650]: Pdb and other debuggers dependent on bdb.py will correctly step over (next command) native coroutines. Patch by Pablo Galindo.
- bpo-32685 [https://bugs.python.org/issue?@action=redirect&bpo=32685]: Improve suggestion when the Python 2 form of print statement is either present on the same line as the header of a compound statement or else terminated by a semi-colon instead of a newline. Patch by Nitish Chandra.
- bpo-32583 [https://bugs.python.org/issue?@action=redirect&bpo=32583]: Fix possible crashing in builtin Unicode decoders caused by write out-of-bound errors when using customized decode error handlers.
- bpo-26163 [https://bugs.python.org/issue?@action=redirect&bpo=26163]: Improved frozenset() hash to create more distinct hash values when faced with datasets containing many similar values.
- bpo-27169 [https://bugs.python.org/issue?@action=redirect&bpo=27169]: The `__debug__` constant is now optimized out at compile time. This fixes also bpo-22091 [https://bugs.python.org/issue?@action=redirect&bpo=22091].

- bpo-32329 [https://bugs.python.org/issue?@action=redirect&bpo=32329]: `sys.flags.hash_randomization` is now properly set to 0 when hash randomization is turned off by `PYTHONHASHSEED=0`.
- bpo-30416 [https://bugs.python.org/issue?@action=redirect&bpo=30416]: The optimizer is now protected from spending much time doing complex calculations and consuming much memory for creating large constants in constant folding.
- bpo-18533 [https://bugs.python.org/issue?@action=redirect&bpo=18533]: `repr()` on a dict containing its own `values()` or `items()` no longer raises `RecursionError`; OrderedDict similarly. Instead, use `...`, as for other recursive structures. Patch by Ben North.
- bpo-32028 [https://bugs.python.org/issue?@action=redirect&bpo=32028]: Leading whitespace is now correctly ignored when generating suggestions for converting Py2 print statements to Py3 builtin print function calls. Patch by Sanyam Khurana.
- bpo-32137 [https://bugs.python.org/issue?@action=redirect&bpo=32137]: The repr of deeply nested dict now raises a RecursionError instead of crashing due to a stack overflow.

Biblioteca

- bpo-33064 [https://bugs.python.org/issue?@action=redirect&bpo=33064]: lib2to3 now properly supports trailing commas after `*args` and `**kwargs` in function signatures.
- bpo-31804 [https://bugs.python.org/issue?@action=redirect&bpo=31804]: Avoid failing in multiprocessing.Process if the standard streams are closed or None at exit.
- bpo-33037 [https://bugs.python.org/issue?@action=redirect&bpo=33037]: Skip sending/receiving data after SSL transport closing.
- bpo-30353 [https://bugs.python.org/issue?@action=redirect&bpo=30353]: Fix ctypes pass-by-value for structs on 64-bit Cygwin/MinGW.
- bpo-33009 [https://bugs.python.org/issue?@action=redirect&bpo=33009]: Fix inspect.signature() for single-parameter partialmethods.
- bpo-32969 [https://bugs.python.org/issue?@action=redirect&bpo=32969]: Expose several missing constants in zlib and fix corresponding documentation.
- bpo-32713 [https://bugs.python.org/issue?@action=redirect&bpo=32713]: Fixed tarfile.itn handling of out-of-bounds float values. Patch by Joffrey

Fuhrer.

- bpo-30622 [https://bugs.python.org/issue?@action=redirect&bpo=30622]: The ssl module now detects missing NPN support in LibreSSL.
- bpo-32922 [https://bugs.python.org/issue?@action=redirect&bpo=32922]: dbm.open() now encodes filename with the filesystem encoding rather than default encoding.
- bpo-32859 [https://bugs.python.org/issue?@action=redirect&bpo=32859]: In `os.dup2`, don't check every call whether the `dup3` syscall exists or not.
- bpo-21060 [https://bugs.python.org/issue?@action=redirect&bpo=21060]: Rewrite confusing message from setup.py upload from "No dist file created in earlier command" to the more helpful "Must create and upload files in one command".
- bpo-32857 [https://bugs.python.org/issue?@action=redirect&bpo=32857]: In **`tkinter`**, `after_cancel(None)` now raises a **`ValueError`** instead of canceling the first scheduled function. Patch by Cheryl Sabella.
- bpo-32852 [https://bugs.python.org/issue?@action=redirect&bpo=32852]: Make sure sys.argv remains as a list when running trace.
- bpo-32841 [https://bugs.python.org/issue?@action=redirect&bpo=32841]: Fixed `asyncio.Condition` issue which silently ignored cancellation after notifying and cancelling a conditional lock. Patch by Bar Harel.
- bpo-31787 [https://bugs.python.org/issue?@action=redirect&bpo=31787]: Fixed refleaks of `__init__()` methods in various modules. (Contributed by Oren Milman)
- bpo-30157 [https://bugs.python.org/issue?@action=redirect&bpo=30157]: Fixed guessing quote and delimiter in csv.Sniffer.sniff() when only the last field is quoted. Patch by Jake Davis.
- bpo-32394 [https://bugs.python.org/issue?@action=redirect&bpo=32394]: socket: Remove TCP_FASTOPEN, TCP_KEEPCNT flags on older version Windows during run-time.
- bpo-32777 [https://bugs.python.org/issue?@action=redirect&bpo=32777]: Fix a rare but potential pre-exec child process deadlock in subprocess on POSIX systems when marking file descriptors inheritable on exec in the child process. This bug appears to have been introduced in 3.4.
- bpo-32647 [https://bugs.python.org/issue?@action=redirect&bpo=32647]: The ctypes module used to depend on indirect linking for dlopen. The shared extension is now explicitly linked against libdl on platforms with dl.

- bpo-32734 [https://bugs.python.org/issue?@action=redirect&bpo=32734]: Fixed `asyncio.Lock()` safety issue which allowed acquiring and locking the same lock multiple times, without it being free. Patch by Bar Harel.
- bpo-32727 [https://bugs.python.org/issue?@action=redirect&bpo=32727]: Do not include name field in SMTP envelope from address. Patch by Stéphane Wirtel
- bpo-27931 [https://bugs.python.org/issue?@action=redirect&bpo=27931]: Fix email address header parsing error when the username is an empty quoted string. Patch by Xiang Zhang.
- bpo-32304 [https://bugs.python.org/issue?@action=redirect&bpo=32304]: distutils' upload command no longer corrupts tar files ending with a CR byte, and no longer tries to convert CR to CRLF in any of the upload text fields.
- bpo-32502 [https://bugs.python.org/issue?@action=redirect&bpo=32502]: uuid.uuid1 no longer raises an exception if a 64-bit hardware address is encountered.
- bpo-31848 [https://bugs.python.org/issue?@action=redirect&bpo=31848]: Fix the error handling in Aifc_read.initfp() when the SSND chunk is not found. Patch by Zackery Spytz.
- bpo-32555 [https://bugs.python.org/issue?@action=redirect&bpo=32555]: On FreeBSD and Solaris, os.strerror() now always decode the byte string from the current locale encoding, rather than using ASCII/surrogateescape in some cases.
- bpo-32521 [https://bugs.python.org/issue?@action=redirect&bpo=32521]: The nis module is now compatible with new libnsl and headers location.
- bpo-32473 [https://bugs.python.org/issue?@action=redirect&bpo=32473]: Improve ABCMeta._dump_registry() output readability
- bpo-32521 [https://bugs.python.org/issue?@action=redirect&bpo=32521]: glibc has removed Sun RPC. Use replacement libtirpc headers and library in nis module.
- bpo-32228 [https://bugs.python.org/issue?@action=redirect&bpo=32228]: Ensure that `truncate()` preserves the file position (as reported by `tell()`) after writes longer than the buffer size.
- bpo-26133 [https://bugs.python.org/issue?@action=redirect&bpo=26133]: Don't unsubscribe signals in asyncio UNIX event loop on interpreter shutdown.

- bpo-32185 [https://bugs.python.org/issue?@action=redirect&bpo=32185]: The SSL module no longer sends IP addresses in SNI TLS extension on platforms with OpenSSL 1.0.2+ or inet_pton.
- bpo-32323 [https://bugs.python.org/issue?@action=redirect&bpo=32323]: **`urllib.parse.urlsplit()`** does not convert zone-id (scope) to lower case for scoped IPv6 addresses in hostnames now.
- bpo-32302 [https://bugs.python.org/issue?@action=redirect&bpo=32302]: Fix bdist_wininst of distutils for CRT v142: it binary compatible with CRT v140.
- bpo-32255 [https://bugs.python.org/issue?@action=redirect&bpo=32255]: A single empty field is now always quoted when written into a CSV file. This allows to distinguish an empty row from a row consisting of a single empty field. Patch by Licht Takeuchi.
- bpo-32277 [https://bugs.python.org/issue?@action=redirect&bpo=32277]: Raise `NotImplementedError` instead of `SystemError` on platforms where `chmod(..., follow_symlinks=False)` is not supported. Patch by Anthony Sottile.
- bpo-32199 [https://bugs.python.org/issue?@action=redirect&bpo=32199]: The getnode() ip getter now uses ‘ip link’ instead of ‘ip link list’.
- bpo-27456 [https://bugs.python.org/issue?@action=redirect&bpo=27456]: Ensure TCP_NODELAY is set on Linux. Tests by Victor Stinner.
- bpo-31900 [https://bugs.python.org/issue?@action=redirect&bpo=31900]: The **`locale.localeconv()`** function now sets temporarily the `LC_CTYPE` locale to the `LC_NUMERIC` locale to decode `decimal_point` and `thousands_sep` byte strings if they are non-ASCII or longer than 1 byte, and the `LC_NUMERIC` locale is different than the `LC_CTYPE` locale. This temporary change affects other threads. Same change for the **`str.format()`** method when formatting a number (**`int`**, **`float`**, **`float`** and subclasses) with the `n` type (ex: `'{:n}'.format(1234)`).
- bpo-31802 [https://bugs.python.org/issue?@action=redirect&bpo=31802]: Importing native path module (`posixpath`, `ntpath`) now works even if the `os` module still is not imported.

Documentação

- bpo-17232 [https://bugs.python.org/issue?@action=redirect&bpo=17232]: Clarify docs for -O and -OO. Patch by Terry Reedy.

- bpo-32800 [https://bugs.python.org/issue?@action=redirect&bpo=32800]: Update link to w3c doc for xml default namespaces.
- bpo-8722 [https://bugs.python.org/issue?@action=redirect&bpo=8722]: Document **`__getattr__()`** behavior when property **`get()`** method raises **`AttributeError`**.
- bpo-32614 [https://bugs.python.org/issue?@action=redirect&bpo=32614]: Modify RE examples in documentation to use raw strings to prevent **`DeprecationWarning`** and add text to REGEX HOWTO to highlight the deprecation.
- bpo-31972 [https://bugs.python.org/issue?@action=redirect&bpo=31972]: Improve docstrings for `pathlib.PurePath` subclasses.
- bpo-17799 [https://bugs.python.org/issue?@action=redirect&bpo=17799]: Explain real behaviour of sys.settrace and sys.setprofile and their C-API counterparts regarding which type of events are received in each function. Patch by Pablo Galindo Salgado.

Testes

- bpo-32517 [https://bugs.python.org/issue?@action=redirect&bpo=32517]: Fix failing `test_asyncio` on macOS 10.12.2+ due to transport of `KqueueSelector` loop was not being closed.
- bpo-32721 [https://bugs.python.org/issue?@action=redirect&bpo=32721]: Fix test_hashlib to not fail if the _md5 module is not built.
- bpo-32252 [https://bugs.python.org/issue?@action=redirect&bpo=32252]: Fix faulthandler_suppress_crash_report() used to prevent core dump files when testing crashes. getrlimit() returns zero on success.
- bpo-31518 [https://bugs.python.org/issue?@action=redirect&bpo=31518]: Debian Unstable has disabled TLS 1.0 and 1.1 for SSLv23_METHOD(). Change TLS/SSL protocol of some tests to PROTOCOL_TLS or PROTOCOL_TLSv1_2 to make them pass on Debian.

Construção

- bpo-32635 [https://bugs.python.org/issue?@action=redirect&bpo=32635]: Fix segfault of the crypt module when libxcrypt is provided instead of libcrypt at the system.

Windows

- bpo-33016 [https://bugs.python.org/issue?@action=redirect&bpo=33016]: Fix potential use of uninitialized memory in nt._getfinalpathname
- bpo-32903 [https://bugs.python.org/issue?@action=redirect&bpo=32903]: Fix a memory leak in os.chdir() on Windows if the current directory is set to a UNC path.
- bpo-31966 [https://bugs.python.org/issue?@action=redirect&bpo=31966]: Fixed WindowsConsoleIO.write() for writing empty data.
- bpo-32409 [https://bugs.python.org/issue?@action=redirect&bpo=32409]: Ensures activate.bat can handle Unicode contents.
- bpo-32457 [https://bugs.python.org/issue?@action=redirect&bpo=32457]: Improves handling of denormalized executable path when launching Python.
- bpo-32370 [https://bugs.python.org/issue?@action=redirect&bpo=32370]: Use the correct encoding for ipconfig output in the uuid module. Patch by Segev Finer.
- bpo-29248 [https://bugs.python.org/issue?@action=redirect&bpo=29248]: Fix **`os.readlink()`** on Windows, which was mistakenly treating the `PrintNameOffset` field of the reparse data buffer as a number of characters instead of bytes. Patch by Craig Holmquist and SSE4.
- bpo-32588 [https://bugs.python.org/issue?@action=redirect&bpo=32588]: Create standalone _distutils_findvs module.

macOS

- bpo-32726 [https://bugs.python.org/issue?@action=redirect&bpo=32726]: Provide an additional, more modern macOS installer variant that supports macOS 10.9+ systems in 64-bit mode only. Upgrade the supplied third-party libraries to OpenSSL 1.0.2n, XZ 5.2.3, and SQLite 3.22.0. The 10.9+ installer now links with and supplies its own copy of Tcl/Tk 8.6.8.

IDLE

- bpo-32984 [https://bugs.python.org/issue?@action=redirect&bpo=32984]: Set `__file__` while running a startup file. Like Python, IDLE optionally runs one startup file in the Shell window before presenting the first interactive input prompt. For IDLE, `-s` runs a file named in

environmental variable **`IDLESTARTUP`** or **`PYTHONSTARTUP`**; `-r file` runs `file`. Python sets `__file__` to the startup file name before running the file and unsets it before the first prompt. IDLE now does the same when run normally, without the `-n` option.

- bpo-32940 [https://bugs.python.org/issue?@action=redirect&bpo=32940]: Simplify and rename StringTranslatePseudoMapping in pyparse.
- bpo-32916 [https://bugs.python.org/issue?@action=redirect&bpo=32916]: Change `str` to `code` in pyparse.
- bpo-32905 [https://bugs.python.org/issue?@action=redirect&bpo=32905]: Remove unused code in pyparse module.
- bpo-32874 [https://bugs.python.org/issue?@action=redirect&bpo=32874]: Add tests for pyparse.
- bpo-32837 [https://bugs.python.org/issue?@action=redirect&bpo=32837]: Using the system and place-dependent default encoding for open() is a bad idea for IDLE's system and location-independent files.
- bpo-32826 [https://bugs.python.org/issue?@action=redirect&bpo=32826]: Add "encoding=utf-8" to open() in IDLE's test_help_about. GUI test test_file_buttons() only looks at initial ascii-only lines, but failed on systems where open() defaults to 'ascii' because readline() internally reads and decodes far enough ahead to encounter a non-ascii character in CREDITS.txt.
- bpo-32765 [https://bugs.python.org/issue?@action=redirect&bpo=32765]: Update configdialog General tab docstring to add new widgets to the widget list.

Ferramentas/Demos

- bpo-24960 [https://bugs.python.org/issue?@action=redirect&bpo=24960]: 2to3 and lib2to3 can now read pickled grammar files using pkgutil.get_data() rather than probing the filesystem. This lets 2to3 and lib2to3 work when run from a zipfile.
- bpo-32222 [https://bugs.python.org/issue?@action=redirect&bpo=32222]: Fix pygettext not extracting docstrings for functions with type annotated arguments. Patch by Toby Harradine.

API C

- bpo-29084 [https://bugs.python.org/issue?@action=redirect&bpo=29084]: Undocumented C API for OrderedDict has been excluded from the limited C API. It was added by mistake and actually never worked in the limited C API.

Python 3.6.4 final

Data de lançamento: 2017-12-18

There were no new code changes in version 3.6.4 since v3.6.4rc1.

Python 3.6.4 release candidate 1

Data de lançamento: 2017-12-05

Núcleo e embutidos

- bpo-32176 [https://bugs.python.org/issue?@action=redirect&bpo=32176]: co_flags.CO_NOFREE is now always set correctly by the code object constructor based on freevars and cellvars, rather than needing to be set correctly by the caller. This ensures it will be cleared automatically when additional cell references are injected into a modified code object and function.
- bpo-31949 [https://bugs.python.org/issue?@action=redirect&bpo=31949]: Fixed several issues in printing tracebacks (PyTraceBack_Print()). Setting sys.tracebacklimit to 0 or less now suppresses printing tracebacks. Setting sys.tracebacklimit to None now causes using the default limit. Setting sys.tracebacklimit to an integer larger than LONG_MAX now means using the limit LONG_MAX rather than the default limit. Fixed integer overflows in the case of more than `2**31` traceback items on Windows. Fixed output errors handling.
- bpo-30696 [https://bugs.python.org/issue?@action=redirect&bpo=30696]: Fix the interactive interpreter looping endlessly when no memory.
- bpo-20047 [https://bugs.python.org/issue?@action=redirect&bpo=20047]: Bytearray methods partition() and rpartition() now accept only bytes-like objects as separator, as documented. In particular they now raise TypeError

rather of returning a bogus result when an integer is passed as a separator.

- bpo-31852 [https://bugs.python.org/issue?@action=redirect&bpo=31852]: Fix a segmentation fault caused by a combination of the async soft keyword and continuation lines.
- bpo-21720 [https://bugs.python.org/issue?@action=redirect&bpo=21720]: BytesWarning no longer emitted when the *fromlist* argument of `__import__()` or the `__all__` attribute of the module contain bytes instances.
- bpo-31825 [https://bugs.python.org/issue?@action=redirect&bpo=31825]: Fixed OverflowError in the 'unicode-escape' codec and in codecs.escape_decode() when decode an escaped non-ascii byte.
- bpo-28603 [https://bugs.python.org/issue?@action=redirect&bpo=28603]: Print the full context/cause chain of exceptions on interpreter exit, even if an exception in the chain is unhashable or compares equal to later ones. Patch by Zane Bitter.
- bpo-31786 [https://bugs.python.org/issue?@action=redirect&bpo=31786]: Fix timeout rounding in the select module to round correctly negative timeouts between -1.0 and 0.0. The functions now block waiting for events as expected. Previously, the call was incorrectly non-blocking. Patch by Pablo Galindo.
- bpo-31642 [https://bugs.python.org/issue?@action=redirect&bpo=31642]: Restored blocking "from package import module" by setting sys.modules["package.module"] to None.
- bpo-31626 [https://bugs.python.org/issue?@action=redirect&bpo=31626]: Fixed a bug in debug memory allocator. There was a write to freed memory after shrinking a memory block.
- bpo-31619 [https://bugs.python.org/issue?@action=redirect&bpo=31619]: Fixed a ValueError when convert a string with large number of underscores to integer with binary base.
- bpo-31592 [https://bugs.python.org/issue?@action=redirect&bpo=31592]: Fixed an assertion failure in Python parser in case of a bad `unicodedata.normalize()`. Patch by Oren Milman.
- bpo-31588 [https://bugs.python.org/issue?@action=redirect&bpo=31588]: Raise a `TypeError` with a helpful error message when class creation fails due to a metaclass with a bad `__prepare__()` method. Patch by Oren Milman.

- bpo-31566 [https://bugs.python.org/issue?@action=redirect&bpo=31566]: Fix an assertion failure in `_warnings.warn()` in case of a bad `__name__` global. Patch by Oren Milman.
- bpo-31505 [https://bugs.python.org/issue?@action=redirect&bpo=31505]: Fix an assertion failure in `json`, in case `_json.make_encoder()` received a bad `encoder()` argument. Patch by Oren Milman.
- bpo-31492 [https://bugs.python.org/issue?@action=redirect&bpo=31492]: Fix assertion failures in case of failing to import from a module with a bad `__name__` attribute, and in case of failing to access an attribute of such a module. Patch by Oren Milman.
- bpo-31490 [https://bugs.python.org/issue?@action=redirect&bpo=31490]: Fix an assertion failure in `ctypes` class definition, in case the class has an attribute whose name is specified in `_anonymous_` but not in `_fields_`. Patch by Oren Milman.
- bpo-31478 [https://bugs.python.org/issue?@action=redirect&bpo=31478]: Fix an assertion failure in `_random.Random.seed()` in case the argument has a bad `__abs__()` method. Patch by Oren Milman.
- bpo-31315 [https://bugs.python.org/issue?@action=redirect&bpo=31315]: Fix an assertion failure in imp.create_dynamic(), when spec.name is not a string. Patch by Oren Milman.
- bpo-31311 [https://bugs.python.org/issue?@action=redirect&bpo=31311]: Fix a crash in the `__setstate__()` method of `ctypes._CData`, in case of a bad `__dict__`. Patch by Oren Milman.
- bpo-31293 [https://bugs.python.org/issue?@action=redirect&bpo=31293]: Fix crashes in true division and multiplication of a timedelta object by a float with a bad as_integer_ratio() method. Patch by Oren Milman.
- bpo-31285 [https://bugs.python.org/issue?@action=redirect&bpo=31285]: Fix an assertion failure in `warnings.warn_explicit`, when the return value of the received loader's `get_source()` has a bad `splitlines()` method. Patch by Oren Milman.
- bpo-30817 [https://bugs.python.org/issue?@action=redirect&bpo=30817]: `PyErr_PrintEx()` clears now the ignored exception that may be raised by `_PySys_SetObjectId()`, for example when no memory.

Biblioteca

- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Two minor fixes for `typing` module: allow shallow copying instances of generic classes, improve interaction of `__init_subclass__` with generics. Original PRs by Ivan Levkivskyi.
- bpo-27240 [https://bugs.python.org/issue?@action=redirect&bpo=27240]: The header folding algorithm for the new email policies has been rewritten, which also fixes bpo-30788 [https://bugs.python.org/issue?@action=redirect&bpo=30788], bpo-31831 [https://bugs.python.org/issue?@action=redirect&bpo=31831], and bpo-32182 [https://bugs.python.org/issue?@action=redirect&bpo=32182]. In particular, RFC2231 folding is now done correctly.
- bpo-32186 [https://bugs.python.org/issue?@action=redirect&bpo=32186]: io.FileIO.readall() and io.FileIO.read() now release the GIL when getting the file size. Fixed hang of all threads with inaccessible NFS server. Patch by Nir Soffer.
- bpo-12239 [https://bugs.python.org/issue?@action=redirect&bpo=12239]: Make **`msilib.SummaryInformation.GetProperty()`** return `None` when the value of property is `VT_EMPTY`. Initial patch by Mark Mc Mahon.
- bpo-31325 [https://bugs.python.org/issue?@action=redirect&bpo=31325]: Fix wrong usage of **`collections.namedtuple()`** in the **`RobotFileParser.parse()`** method. Initial patch by Robin Wellner.
- bpo-12382 [https://bugs.python.org/issue?@action=redirect&bpo=12382]: **`msilib.OpenDatabase()`** now raises a better exception message when it couldn't open or create an MSI file. Initial patch by William Tisäter.
- bpo-32110 [https://bugs.python.org/issue?@action=redirect&bpo=32110]: `codecs.StreamReader.read(n)` now returns not more than *n* characters/bytes for non-negative *n*. This makes it compatible with `read()` methods of other file-like objects.
- bpo-32072 [https://bugs.python.org/issue?@action=redirect&bpo=32072]: Fixed issues with binary plists: Fixed saving bytearrays. Identical objects will be saved only once. Equal references will be load as identical objects. Added support for saving and loading recursive data structures.
- bpo-32034 [https://bugs.python.org/issue?@action=redirect&bpo=32034]: Make asyncio.IncompleteReadError and LimitOverrunError pickleable.
- bpo-32015 [https://bugs.python.org/issue?@action=redirect&bpo=32015]: Fixed the looping of asyncio in the case of reconnection the socket during waiting async read/write from/to the socket.

- bpo-32011 [https://bugs.python.org/issue?@action=redirect&bpo=32011]: Restored support of loading marshal files with the TYPE_INT64 code. These files can be produced in Python 2.7.
- bpo-31970 [https://bugs.python.org/issue?@action=redirect&bpo=31970]: Reduce performance overhead of asyncio debug mode.
- bpo-9678 [https://bugs.python.org/issue?@action=redirect&bpo=9678]: Fixed determining the MAC address in the uuid module: Using ifconfig on NetBSD and OpenBSD. Using arp on Linux, FreeBSD, NetBSD and OpenBSD. Based on patch by Takayuki Shimizukawa.
- bpo-30057 [https://bugs.python.org/issue?@action=redirect&bpo=30057]: Fix potential missed signal in signal.signal().
- bpo-31933 [https://bugs.python.org/issue?@action=redirect&bpo=31933]: Fix Blake2 params leaf_size and node_offset on big endian platforms. Patch by Jack O'Connor.
- bpo-31927 [https://bugs.python.org/issue?@action=redirect&bpo=31927]: Fixed compilation of the socket module on NetBSD 8. Fixed assertion failure or reading arbitrary data when parse a AF_BLUETOOTH address on NetBSD and DragonFly BSD.
- bpo-27666 [https://bugs.python.org/issue?@action=redirect&bpo=27666]: Fixed stack corruption in curses.box() and curses.ungetmouse() when the size of types chtype or mmask_t is less than the size of C long. curses.box() now accepts characters as arguments. Based on patch by Steve Fink.
- bpo-31897 [https://bugs.python.org/issue?@action=redirect&bpo=31897]: plistlib now catches more errors when read binary plists and raises InvalidFileException instead of unexpected exceptions.
- bpo-25720 [https://bugs.python.org/issue?@action=redirect&bpo=25720]: Fix the method for checking pad state of curses WINDOW. Patch by Masayuki Yamamoto.
- bpo-31893 [https://bugs.python.org/issue?@action=redirect&bpo=31893]: Fixed the layout of the kqueue_event structure on OpenBSD and NetBSD. Fixed the comparison of the kqueue_event objects.
- bpo-31891 [https://bugs.python.org/issue?@action=redirect&bpo=31891]: Fixed building the curses module on NetBSD.
- bpo-28416 [https://bugs.python.org/issue?@action=redirect&bpo=28416]: Instances of pickle.Pickler subclass with the persistent_id() method and pickle.Unpickler subclass with the persistent_load() method no longer create reference cycles.

- bpo-28326 [https://bugs.python.org/issue?@action=redirect&bpo=28326]: Fix multiprocessing.Process when stdout and/or stderr is closed or None.
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: If nested log adapters are used, the inner `process()` methods are no longer omitted.
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: The `manager` property on LoggerAdapter objects is now properly settable.
- bpo-31806 [https://bugs.python.org/issue?@action=redirect&bpo=31806]: Fix timeout rounding in time.sleep(), threading.Lock.acquire() and socket.socket.settimeout() to round correctly negative timeouts between -1.0 and 0.0. The functions now block waiting for events as expected. Previously, the call was incorrectly non-blocking. Patch by Pablo Galindo.
- bpo-28603 [https://bugs.python.org/issue?@action=redirect&bpo=28603]: traceback: Fix a TypeError that occurred during printing of exception tracebacks when either the current exception or an exception in its context/cause chain is unhashable. Patch by Zane Bitter.
- bpo-30058 [https://bugs.python.org/issue?@action=redirect&bpo=30058]: Fixed buffer overflow in select.kqueue.control().
- bpo-31770 [https://bugs.python.org/issue?@action=redirect&bpo=31770]: Prevent a crash when calling the `__init__()` method of a `sqlite3.Cursor` object more than once. Patch by Oren Milman.
- bpo-31672 [https://bugs.python.org/issue?@action=redirect&bpo=31672]: `idpattern` in `string.Template` matched some non-ASCII characters. Now it uses `-i` regular expression local flag to avoid non-ASCII characters.
- bpo-31764 [https://bugs.python.org/issue?@action=redirect&bpo=31764]: Prevent a crash in `sqlite3.Cursor.close()` in case the `Cursor` object is uninitialized. Patch by Oren Milman.
- bpo-31752 [https://bugs.python.org/issue?@action=redirect&bpo=31752]: Fix possible crash in timedelta constructor called with custom integers.
- bpo-31701 [https://bugs.python.org/issue?@action=redirect&bpo=31701]: On Windows, faulthandler.enable() now ignores MSC and COM exceptions.
- bpo-31728 [https://bugs.python.org/issue?@action=redirect&bpo=31728]: Prevent crashes in `_elementtree` due to unsafe cleanup of `Element.text` and `Element.tail`. Patch by Oren Milman.

- bpo-31620 [https://bugs.python.org/issue?@action=redirect&bpo=31620]: an empty asyncio.Queue now doesn't leak memory when queue.get pollers timeout
- bpo-31632 [https://bugs.python.org/issue?@action=redirect&bpo=31632]: Fix method set_protocol() of class _SSLProtocolTransport in asyncio module. This method was previously modifying a wrong reference to the protocol.
- bpo-31675 [https://bugs.python.org/issue?@action=redirect&bpo=31675]: Fixed memory leaks in Tkinter's methods splitlist() and split() when pass a string larger than 2 GiB.
- bpo-31673 [https://bugs.python.org/issue?@action=redirect&bpo=31673]: Fixed typo in the name of Tkinter's method adderrorinfo().
- bpo-30806 [https://bugs.python.org/issue?@action=redirect&bpo=30806]: Fix the string representation of a netrc object.
- bpo-15037 [https://bugs.python.org/issue?@action=redirect&bpo=15037]: Added a workaround for getkey() in curses for ncurses 5.7 and earlier.
- bpo-25351 [https://bugs.python.org/issue?@action=redirect&bpo=25351]: Avoid venv activate failures with undefined variables
- bpo-25532 [https://bugs.python.org/issue?@action=redirect&bpo=25532]: inspect.unwrap() will now only try to unwrap an object sys.getrecursionlimit() times, to protect against objects which create a new object on every attribute access.
- bpo-30347 [https://bugs.python.org/issue?@action=redirect&bpo=30347]: Stop crashes when concurrently iterate over itertools.groupby() iterators.
- bpo-31516 [https://bugs.python.org/issue?@action=redirect&bpo=31516]: `threading.current_thread()` should not return a dummy thread at shutdown.
- bpo-31351 [https://bugs.python.org/issue?@action=redirect&bpo=31351]: python -m ensurepip now exits with non-zero exit code if pip bootstrapping has failed.
- bpo-31482 [https://bugs.python.org/issue?@action=redirect&bpo=31482]: `random.seed()` now works with bytes in version=1
- bpo-31334 [https://bugs.python.org/issue?@action=redirect&bpo=31334]: Fix `poll.poll([timeout])` in the `select` module for arbitrary negative timeouts on all OSes where it can only be a non-negative integer or -1. Patch by Riccardo Coccioli.
- bpo-31310 [https://bugs.python.org/issue?@action=redirect&bpo=31310]: multiprocessing's semaphore tracker should be launched again if

crashed.
- bpo-31308 [https://bugs.python.org/issue?@action=redirect&bpo=31308]: Make multiprocessing's forkserver process immune to Ctrl-C and other user interruptions. If it crashes, restart it when necessary.

Documentação

- bpo-32105 [https://bugs.python.org/issue?@action=redirect&bpo=32105]: Added asyncio.BaseEventLoop.connect_accepted_socket versionadded marker.
- bpo-31537 [https://bugs.python.org/issue?@action=redirect&bpo=31537]: Fix incorrect usage of `get_history_length` in readline documentation example code. Patch by Brad Smith.
- bpo-30085 [https://bugs.python.org/issue?@action=redirect&bpo=30085]: The operator functions without double underscores are preferred for clarity. The one with underscores are only kept for back-compatibility.

Testes

- bpo-31380 [https://bugs.python.org/issue?@action=redirect&bpo=31380]: Skip test_httpservers test_undecodable_file on macOS: fails on APFS.
- bpo-31705 [https://bugs.python.org/issue?@action=redirect&bpo=31705]: Skip test_socket.test_sha256() on Linux kernel older than 4.5. The test fails with ENOKEY on kernel 3.10 (on ppc64le). A fix was merged into the kernel 4.5.
- bpo-31174 [https://bugs.python.org/issue?@action=redirect&bpo=31174]: Fix test_tools.test_unparse: DirectoryTestCase now stores the names sample to always test the same files. It prevents false alarms when hunting reference leaks.
- bpo-30695 [https://bugs.python.org/issue?@action=redirect&bpo=30695]: Add the `set_nomemory(start, stop)` and `remove_mem_hooks()` functions to the `_testcapi` module.

Construção

- bpo-32059 [https://bugs.python.org/issue?@action=redirect&bpo=32059]: `detect_modules()` in `setup.py` now also searches the sysroot paths

when cross-compiling.

- bpo-31957 [https://bugs.python.org/issue?@action=redirect&bpo=31957]: Fixes Windows SDK version detection when building for Windows.
- bpo-31609 [https://bugs.python.org/issue?@action=redirect&bpo=31609]: Fixes quotes in PCbuild/clean.bat
- bpo-31934 [https://bugs.python.org/issue?@action=redirect&bpo=31934]: Abort the build when building out of a not clean source tree.
- bpo-31926 [https://bugs.python.org/issue?@action=redirect&bpo=31926]: Fixed Argument Clinic sometimes causing compilation errors when there was more than one function and/or method in a .c file with the same name.
- bpo-28791 [https://bugs.python.org/issue?@action=redirect&bpo=28791]: Update Windows builds to use SQLite 3.21.0.
- bpo-28791 [https://bugs.python.org/issue?@action=redirect&bpo=28791]: Update OS X installer to use SQLite 3.21.0.
- bpo-22140 [https://bugs.python.org/issue?@action=redirect&bpo=22140]: Prevent double substitution of prefix in python-config.sh.
- bpo-31536 [https://bugs.python.org/issue?@action=redirect&bpo=31536]: Avoid wholesale rebuild after `make regen-all` if nothing changed.

Windows

- bpo-1102 [https://bugs.python.org/issue?@action=redirect&bpo=1102]: Return `None` when `View.Fetch()` returns `ERROR_NO_MORE_ITEMS` instead of raising `MSIError`. Initial patch by Anthony Tuininga.
- bpo-31944 [https://bugs.python.org/issue?@action=redirect&bpo=31944]: Fixes Modify button in Apps and Features dialog.

macOS

- bpo-31392 [https://bugs.python.org/issue?@action=redirect&bpo=31392]: Atualizadas as construções do instalador do macOS para utilizar OpenSSL 1.0.2m

IDLE

- bpo-32207 [https://bugs.python.org/issue?@action=redirect&bpo=32207]: Improve tk event exception tracebacks in IDLE. When tk event handling is driven

by IDLE's run loop, a confusing and distracting queue.EMPTY traceback context is no longer added to tk event exception tracebacks. The traceback is now the same as when event handling is driven by user code. Patch based on a suggestion by Serhiy Storchaka.

- bpo-32164 [https://bugs.python.org/issue?@action=redirect&bpo=32164]: Delete unused file idlelib/tabbedpages.py. Use of TabbedPageSet in configdialog was replaced by ttk.Notebook.
- bpo-32100 [https://bugs.python.org/issue?@action=redirect&bpo=32100]: IDLE: Fix old and new bugs in pathbrowser; improve tests. Patch mostly by Cheryl Sabella.
- bpo-31858 [https://bugs.python.org/issue?@action=redirect&bpo=31858]: IDLE – Restrict shell prompt manipulation to the shell. Editor and output windows only see an empty last prompt line. This simplifies the code and fixes a minor bug when newline is inserted. Sys.ps1, if present, is read on Shell start-up, but is not set or changed.
- bpo-31860 [https://bugs.python.org/issue?@action=redirect&bpo=31860]: The font sample in the IDLE configuration dialog is now editable. Changes persist while IDLE remains open
- bpo-31836 [https://bugs.python.org/issue?@action=redirect&bpo=31836]: Test_code_module now passes if run after test_idle, which sets ps1. The code module uses sys.ps1 if present or sets it to '>>> ' if not. Test_code_module now properly tests both behaviors. Ditto for ps2.
- bpo-28603 [https://bugs.python.org/issue?@action=redirect&bpo=28603]: Fix a TypeError that caused a shell restart when printing a traceback that includes an exception that is unhashable. Patch by Zane Bitter.
- bpo-13802 [https://bugs.python.org/issue?@action=redirect&bpo=13802]: Use non-Latin characters in the IDLE's Font settings sample. Even if one selects a font that defines a limited subset of the unicode Basic Multilingual Plane, tcl/tk will use other fonts that define a character. The expanded example give users of non-Latin characters a better idea of what they might see in IDLE's shell and editors. To make room for the expanded sample, frames on the Font tab are re-arranged. The Font/Tabs help explains a bit about the additions.
- bpo-31460 [https://bugs.python.org/issue?@action=redirect&bpo=31460]: Simplify the API of IDLE's Module Browser. Passing a widget instead of an flist with a root widget opens the option of creating a browser frame that is only part of a window. Passing a full file name instead of pieces

assumed to come from a .py file opens the possibility of browsing python files that do not end in .py.

- bpo-31649 [https://bugs.python.org/issue?@action=redirect&bpo=31649]: IDLE - Make _htest, _utest parameters keyword only.
- bpo-31559 [https://bugs.python.org/issue?@action=redirect&bpo=31559]: Remove test order dependence in idle_test.test_browser.
- bpo-31459 [https://bugs.python.org/issue?@action=redirect&bpo=31459]: Rename IDLE's module browser from Class Browser to Module Browser. The original module-level class and method browser became a module browser, with the addition of module-level functions, years ago. Nested classes and functions were added yesterday. For back-compatibility, the virtual event <<open-class-browser>>, which appears on the Keys tab of the Settings dialog, is not changed. Patch by Cheryl Sabella.
- bpo-31500 [https://bugs.python.org/issue?@action=redirect&bpo=31500]: Default fonts now are scaled on HiDPI displays.
- bpo-1612262 [https://bugs.python.org/issue?@action=redirect&bpo=1612262]: IDLE module browser now shows nested classes and functions. Original patches for code and tests by Guilherme Polo and Cheryl Sabella, respectively.

Ferramentas/Demos

- bpo-30722 [https://bugs.python.org/issue?@action=redirect&bpo=30722]: Make redemo work with Python 3.6 and newer versions. Also, remove the `LOCALE` option since it doesn't work with string patterns in Python 3. Patch by Christoph Sarnowski.

API C

- bpo-20891 [https://bugs.python.org/issue?@action=redirect&bpo=20891]: Fix PyGILState_Ensure(). When PyGILState_Ensure() is called in a non-Python thread before PyEval_InitThreads(), only call PyEval_InitThreads() after calling PyThreadState_New() to fix a crash.
- bpo-31532 [https://bugs.python.org/issue?@action=redirect&bpo=31532]: Fix memory corruption due to allocator mix in getpath.c between

Py_GetPath() and Py_SetPath()

- bpo-30697 [https://bugs.python.org/issue?@action=redirect&bpo=30697]: The `PyExc_RecursionErrorInst` singleton is removed and `PyErr_NormalizeException()` does not use it anymore. This singleton is persistent and its members being never cleared may cause a segfault during finalization of the interpreter. See also bpo-22898 [https://bugs.python.org/issue?@action=redirect&bpo=22898].

Python 3.6.3 final

Data de lançamento: 2017-10-03

Biblioteca

- bpo-31641 [https://bugs.python.org/issue?@action=redirect&bpo=31641]: Re-allow arbitrary iterables in `concurrent.futures.as_completed()`. Fixes regression in 3.6.3rc1.

Construção

- bpo-31662 [https://bugs.python.org/issue?@action=redirect&bpo=31662]: Fix typos in Windows `uploadrelease.bat` script. Fix Windows Doc build issues in `Doc/make.bat`.
- bpo-31423 [https://bugs.python.org/issue?@action=redirect&bpo=31423]: Fix building the PDF documentation with newer versions of Sphinx.

Python 3.6.3 release candidate 1

Data de lançamento: 2017-09-18

Segurança

- bpo-29781 [https://bugs.python.org/issue?@action=redirect&bpo=29781]: SSLObject.version() now correctly returns None when handshake over BIO has not been performed yet.

- bpo-30947 [https://bugs.python.org/issue?@action=redirect&bpo=30947]: Upgrade libexpat embedded copy from version 2.2.1 to 2.2.3 to get security fixes.

Núcleo e embutidos

- bpo-31471 [https://bugs.python.org/issue?@action=redirect&bpo=31471]: Fix an assertion failure in `subprocess.Popen()` on Windows, in case the env argument has a bad `keys()` method. Patch by Oren Milman.
- bpo-31418 [https://bugs.python.org/issue?@action=redirect&bpo=31418]: Fix an assertion failure in `PyErr_WriteUnraisable()` in case of an exception with a bad `__module__` attribute. Patch by Oren Milman.
- bpo-31416 [https://bugs.python.org/issue?@action=redirect&bpo=31416]: Fix assertion failures in case of a bad warnings.filters or warnings.defaultaction. Patch by Oren Milman.
- bpo-31411 [https://bugs.python.org/issue?@action=redirect&bpo=31411]: Raise a TypeError instead of SystemError in case warnings.onceregistry is not a dictionary. Patch by Oren Milman.
- bpo-31373 [https://bugs.python.org/issue?@action=redirect&bpo=31373]: Fix several possible instances of undefined behavior due to floating-point demotions.
- bpo-30465 [https://bugs.python.org/issue?@action=redirect&bpo=30465]: Location information (`lineno` and `col_offset`) in f-strings is now (mostly) correct. This fixes tools like flake8 from showing warnings on the wrong line (typically the first line of the file).
- bpo-31343 [https://bugs.python.org/issue?@action=redirect&bpo=31343]: Include sys/sysmacros.h for major(), minor(), and makedev(). GNU C library plans to remove the functions from sys/types.h.
- bpo-31291 [https://bugs.python.org/issue?@action=redirect&bpo=31291]: Fix an assertion failure in `zipimport.zipimporter.get_data` on Windows, when the return value of `pathname.replace('/','\\')` isn't a string. Patch by Oren Milman.
- bpo-31271 [https://bugs.python.org/issue?@action=redirect&bpo=31271]: Fix an assertion failure in the `write()` method of `io.TextIOWrapper`, when the encoder doesn't return a bytes object. Patch by Oren Milman.
- bpo-31243 [https://bugs.python.org/issue?@action=redirect&bpo=31243]: Fix a crash in some methods of `io.TextIOWrapper`, when the decoder's state is

invalid. Patch by Oren Milman.

- bpo-30721 [https://bugs.python.org/issue?@action=redirect&bpo=30721]: `print` now shows correct usage hint for using Python 2 redirection syntax. Patch by Sanyam Khurana.
- bpo-31070 [https://bugs.python.org/issue?@action=redirect&bpo=31070]: Fix a race condition in importlib _get_module_lock().
- bpo-31095 [https://bugs.python.org/issue?@action=redirect&bpo=31095]: Fix potential crash during GC caused by `tp_dealloc` which doesn't call `PyObject_GC_UnTrack()`.
- bpo-31071 [https://bugs.python.org/issue?@action=redirect&bpo=31071]: Avoid masking original TypeError in call with * unpacking when other arguments are passed.
- bpo-30978 [https://bugs.python.org/issue?@action=redirect&bpo=30978]: str.format_map() now passes key lookup exceptions through. Previously any exception was replaced with a KeyError exception.
- bpo-30808 [https://bugs.python.org/issue?@action=redirect&bpo=30808]: Use _Py_atomic API for concurrency-sensitive signal state.
- bpo-30876 [https://bugs.python.org/issue?@action=redirect&bpo=30876]: Relative import from unloaded package now reimports the package instead of failing with SystemError. Relative import from non-package now fails with ImportError rather than SystemError.
- bpo-30703 [https://bugs.python.org/issue?@action=redirect&bpo=30703]: Improve signal delivery. Avoid using Py_AddPendingCall from signal handler, to avoid calling signal-unsafe functions. The tests I'm adding here fail without the rest of the patch, on Linux and OS X. This means our signal delivery logic had defects (some signals could be lost).
- bpo-30765 [https://bugs.python.org/issue?@action=redirect&bpo=30765]: Avoid blocking in pthread_mutex_lock() when PyThread_acquire_lock() is asked not to block.
- bpo-31161 [https://bugs.python.org/issue?@action=redirect&bpo=31161]: Make sure the 'Missing parentheses' syntax error message is only applied to SyntaxError, not to subclasses. Patch by Martijn Pieters.
- bpo-30814 [https://bugs.python.org/issue?@action=redirect&bpo=30814]: Fixed a race condition when import a submodule from a package.
- bpo-30597 [https://bugs.python.org/issue?@action=redirect&bpo=30597]: `print` now shows expected input in custom error message when used as a Python 2 statement. Patch by Sanyam Khurana.

Biblioteca

- bpo-31499 [https://bugs.python.org/issue?@action=redirect&bpo=31499]: xml.etree: Fix a crash when a parser is part of a reference cycle.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: typing.get_type_hints now finds the right globalns for classes and modules by default (when no `globalns` was specified by the caller).
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Speed improvements to the `typing` module. Original PRs by Ivan Levkivskyi and Mitar.
- bpo-31544 [https://bugs.python.org/issue?@action=redirect&bpo=31544]: The C accelerator module of ElementTree ignored exceptions raised when looking up TreeBuilder target methods in XMLParser().
- bpo-31234 [https://bugs.python.org/issue?@action=redirect&bpo=31234]: socket.create_connection() now fixes manually a reference cycle: clear the variable storing the last exception on success.
- bpo-31457 [https://bugs.python.org/issue?@action=redirect&bpo=31457]: LoggerAdapter objects can now be nested.
- bpo-31400 [https://bugs.python.org/issue?@action=redirect&bpo=31400]: Improves SSL error handling to avoid losing error numbers.
- bpo-28958 [https://bugs.python.org/issue?@action=redirect&bpo=28958]: ssl.SSLContext() now uses OpenSSL error information when a context cannot be instantiated.
- bpo-27340 [https://bugs.python.org/issue?@action=redirect&bpo=27340]: SSLSocket.sendall() now uses memoryview to create slices of data. This fixes support for all bytes-like object. It is also more efficient and avoids costly copies.
- bpo-31178 [https://bugs.python.org/issue?@action=redirect&bpo=31178]: Fix string concatenation bug in rare error path in the subprocess module
- bpo-31350 [https://bugs.python.org/issue?@action=redirect&bpo=31350]: Micro-optimize **`asyncio._get_running_loop()`** to become up to 10% faster.
- bpo-31170 [https://bugs.python.org/issue?@action=redirect&bpo=31170]: expat: Update libexpat from 2.2.3 to 2.2.4. Fix copying of partial characters for UTF-8 input (libexpat bug 115): https://github.com/libexpat/libexpat/issues/115
- bpo-29136 [https://bugs.python.org/issue?@action=redirect&bpo=29136]: Add TLS 1.3 cipher suites and OP_NO_TLSv1_3.

- bpo-29212 [https://bugs.python.org/issue?@action=redirect&bpo=29212]: Fix concurrent.futures.thread.ThreadPoolExecutor threads to have a non repr() based thread name by default when no thread_name_prefix is supplied. They will now identify themselves as “ThreadPoolExecutor-y_n”.
- bpo-9146 [https://bugs.python.org/issue?@action=redirect&bpo=9146]: Fix a segmentation fault in _hashopenssl when standard hash functions such as md5 are not available in the linked OpenSSL library. As in some special FIPS-140 build environments.
- bpo-27144 [https://bugs.python.org/issue?@action=redirect&bpo=27144]: The `map()` and `as_completed()` iterators in `concurrent.futures` now avoid keeping a reference to yielded objects.
- bpo-10746 [https://bugs.python.org/issue?@action=redirect&bpo=10746]: Fix ctypes producing wrong **PEP 3118** [https://peps.python.org/pep-3118/] type codes for integer types.
- bpo-22536 [https://bugs.python.org/issue?@action=redirect&bpo=22536]: The subprocess module now sets the filename when FileNotFoundError is raised on POSIX systems due to the executable or cwd not being found.
- bpo-31249 [https://bugs.python.org/issue?@action=redirect&bpo=31249]: concurrent.futures: WorkItem.run() used by ThreadPoolExecutor now breaks a reference cycle between an exception object and the WorkItem object.
- bpo-31247 [https://bugs.python.org/issue?@action=redirect&bpo=31247]: xmlrpc.server now explicitly breaks reference cycles when using sys.exc_info() in code handling exceptions.
- bpo-30102 [https://bugs.python.org/issue?@action=redirect&bpo=30102]: The ssl and hashlib modules now call OPENSSL_add_all_algorithms_noconf() on OpenSSL < 1.1.0. The function detects CPU features and enables optimizations on some CPU architectures such as POWER8. Patch is based on research from Gustavo Serra Scalet.
- bpo-31185 [https://bugs.python.org/issue?@action=redirect&bpo=31185]: Fixed miscellaneous errors in asyncio speedup module.
- bpo-31135 [https://bugs.python.org/issue?@action=redirect&bpo=31135]: ttk: fix the destroy() method of LabeledScale and OptionMenu classes. Call the parent destroy() method even if the used attribute doesn’t exist. The

LabeledScale.destroy() method now also explicitly clears label and scale attributes to help the garbage collector to destroy all widgets.

- bpo-31107 [https://bugs.python.org/issue?@action=redirect&bpo=31107]: Fix `copyreg._slotnames()` mangled attribute calculation for classes whose name begins with an underscore. Patch by Shane Harvey.
- bpo-31061 [https://bugs.python.org/issue?@action=redirect&bpo=31061]: Fixed a crash when using asyncio and threads.
- bpo-30502 [https://bugs.python.org/issue?@action=redirect&bpo=30502]: Fix handling of long oids in ssl. Based on patch by Christian Heimes.
- bpo-30119 [https://bugs.python.org/issue?@action=redirect&bpo=30119]: ftplib.FTP.putline() now throws ValueError on commands that contains CR or LF. Patch by Donghee Na.
- bpo-30595 [https://bugs.python.org/issue?@action=redirect&bpo=30595]: multiprocessing.Queue.get() with a timeout now polls its reader in non-blocking mode if it succeeded to acquire the lock but the acquire took longer than the timeout.
- bpo-29403 [https://bugs.python.org/issue?@action=redirect&bpo=29403]: Fix `unittest.mock`'s autospec to not fail on method-bound builtin functions. Patch by Aaron Gallagher.
- bpo-30961 [https://bugs.python.org/issue?@action=redirect&bpo=30961]: Fix decrementing a borrowed reference in tracemalloc.
- bpo-25684 [https://bugs.python.org/issue?@action=redirect&bpo=25684]: Change `ttk.OptionMenu` radiobuttons to be unique across instances of `OptionMenu`.
- bpo-30886 [https://bugs.python.org/issue?@action=redirect&bpo=30886]: Fix multiprocessing.Queue.join_thread(): it now waits until the thread completes, even if the thread was started by the same process which created the queue.
- bpo-29854 [https://bugs.python.org/issue?@action=redirect&bpo=29854]: Fix segfault in readline when using readline's history-size option. Patch by Nir Soffer.
- bpo-30319 [https://bugs.python.org/issue?@action=redirect&bpo=30319]: socket.close() now ignores ECONNRESET error.
- bpo-30828 [https://bugs.python.org/issue?@action=redirect&bpo=30828]: Fix out of bounds write in `asyncio.CFuture.remove_done_callback()`.
- bpo-30807 [https://bugs.python.org/issue?@action=redirect&bpo=30807]: signal.setitimer() may disable the timer when passed a tiny value. Tiny

values (such as 1e-6) are valid non-zero values for setitimer(), which is specified as taking microsecond-resolution intervals. However, on some platform, our conversion routine could convert 1e-6 into a zero interval, therefore disabling the timer instead of (re-)scheduling it.

- bpo-30441 [https://bugs.python.org/issue?@action=redirect&bpo=30441]: Fix bug when modifying os.environ while iterating over it
- bpo-30532 [https://bugs.python.org/issue?@action=redirect&bpo=30532]: Fix email header value parser dropping folding white space in certain cases.
- bpo-30879 [https://bugs.python.org/issue?@action=redirect&bpo=30879]: os.listdir() and os.scandir() now emit bytes names when called with bytes-like argument.
- bpo-30746 [https://bugs.python.org/issue?@action=redirect&bpo=30746]: Prohibited the '=' character in environment variable names in `os.putenv()` and `os.spawn*()`.
- bpo-29755 [https://bugs.python.org/issue?@action=redirect&bpo=29755]: Fixed the lgettext() family of functions in the gettext module. They now always return bytes.

Documentação

- bpo-31294 [https://bugs.python.org/issue?@action=redirect&bpo=31294]: Fix incomplete code snippet in the ZeroMQSocketListener and ZeroMQSocketHandler examples and adapt them to Python 3.
- bpo-21649 [https://bugs.python.org/issue?@action=redirect&bpo=21649]: Add RFC 7525 and Mozilla server side TLS links to SSL documentation.
- bpo-30803 [https://bugs.python.org/issue?@action=redirect&bpo=30803]: Clarify doc on truth value testing. Original patch by Peter Thomassen.

Testes

- bpo-31320 [https://bugs.python.org/issue?@action=redirect&bpo=31320]: Silence traceback in test_ssl
- bpo-25674 [https://bugs.python.org/issue?@action=redirect&bpo=25674]: Remove sha256.tbs-internet.com ssl test
- bpo-30715 [https://bugs.python.org/issue?@action=redirect&bpo=30715]: Address ALPN callback changes for OpenSSL 1.1.0f. The latest version behaves like OpenSSL 1.0.2 and no longer aborts handshake.

- bpo-30822 [https://bugs.python.org/issue?@action=redirect&bpo=30822]: regrtest: Exclude tzdata from regrtest –all. When running the test suite using –use=all / -u all, exclude tzdata since it makes test_datetime too slow (15-20 min on some buildbots) which then times out on some buildbots. Fix also regrtest command line parser to allow passing -u extralargefile to run test_zipfile64.

Construção

- bpo-30854 [https://bugs.python.org/issue?@action=redirect&bpo=30854]: Fix compile error when compiling –without-threads. Patch by Masayuki Yamamoto.

Windows

- bpo-30389 [https://bugs.python.org/issue?@action=redirect&bpo=30389]: Adds detection of Visual Studio 2017 to distutils on Windows.
- bpo-31340 [https://bugs.python.org/issue?@action=redirect&bpo=31340]: Change to building with MSVC v141 (included with Visual Studio 2017)
- bpo-30581 [https://bugs.python.org/issue?@action=redirect&bpo=30581]: os.cpu_count() now returns the correct number of processors on Windows when the number of logical processors is greater than 64.
- bpo-30731 [https://bugs.python.org/issue?@action=redirect&bpo=30731]: Add a missing xmlns to python.manifest so that it matches the schema.

IDLE

- bpo-31493 [https://bugs.python.org/issue?@action=redirect&bpo=31493]: IDLE code context – fix code update and font update timers. Canceling timers prevents a warning message when test_idle completes.
- bpo-31488 [https://bugs.python.org/issue?@action=redirect&bpo=31488]: IDLE - Update non-key options in former extension classes. When applying configdialog changes, call .reload for each feature class. Change ParenMatch so updated options affect existing instances attached to existing editor windows.
- bpo-31477 [https://bugs.python.org/issue?@action=redirect&bpo=31477]: IDLE - Improve rstrip entry in doc. Strip trailing whitespace strips more than

blank spaces. Multiline string literals are not skipped.

- bpo-31480 [https://bugs.python.org/issue?@action=redirect&bpo=31480]: IDLE - make tests pass with zzdummy extension disabled by default.
- bpo-31421 [https://bugs.python.org/issue?@action=redirect&bpo=31421]: Document how IDLE runs tkinter programs. IDLE calls tcl/tk update in the background in order to make live interaction and experimentation with tkinter applications much easier.
- bpo-31414 [https://bugs.python.org/issue?@action=redirect&bpo=31414]: IDLE – fix tk entry box tests by deleting first. Adding to an int entry is not the same as deleting and inserting because int(‘’) will fail.
- bpo-31051 [https://bugs.python.org/issue?@action=redirect&bpo=31051]: Rearrange IDLE configdialog GenPage into Window, Editor, and Help sections.
- bpo-30617 [https://bugs.python.org/issue?@action=redirect&bpo=30617]: IDLE - Add docstrings and tests for outwin subclass of editor. Move some data and functions from the class to module level. Patch by Cheryl Sabella.
- bpo-31287 [https://bugs.python.org/issue?@action=redirect&bpo=31287]: IDLE - Do not modify tkinter.message in test_configdialog.
- bpo-27099 [https://bugs.python.org/issue?@action=redirect&bpo=27099]: Convert IDLE’s built-in ‘extensions’ to regular features. About 10 IDLE features were implemented as supposedly optional extensions. Their different behavior could be confusing or worse for users and not good for maintenance. Hence the conversion. The main difference for users is that user configurable key bindings for builtin features are now handled uniformly. Now, editing a binding in a keyset only affects its value in the keyset. All bindings are defined together in the system-specific default keysets in config-extensions.def. All custom keysets are saved as a whole in config-extension.cfg. All take effect as soon as one clicks Apply or Ok. The affected events are ‘<<force-open-completions>>’, ‘<<expand-word>>’, ‘<<force-open-calltip>>’, ‘<<flash-paren>>’, ‘<<format-paragraph>>’, ‘<<run-module>>’, ‘<<check-module>>’, and ‘<<zoom-height>>’. Any (global) customizations made before 3.6.3 will not affect their keyset-specific customization after 3.6.3. and vice versa. Initial patch by Charles Wohlganger.
- bpo-31206 [https://bugs.python.org/issue?@action=redirect&bpo=31206]: IDLE: Factor HighPage(Frame) class from ConfigDialog. Patch by Cheryl

Sabella.

- bpo-31001 [https://bugs.python.org/issue?@action=redirect&bpo=31001]: Add tests for configdialog highlight tab. Patch by Cheryl Sabella.
- bpo-31205 [https://bugs.python.org/issue?@action=redirect&bpo=31205]: IDLE: Factor KeysPage(Frame) class from ConfigDialog. The slightly modified tests continue to pass. Patch by Cheryl Sabella.
- bpo-31130 [https://bugs.python.org/issue?@action=redirect&bpo=31130]: IDLE – stop leaks in test_configdialog. Initial patch by Victor Stinner.
- bpo-31002 [https://bugs.python.org/issue?@action=redirect&bpo=31002]: Add tests for configdialog keys tab. Patch by Cheryl Sabella.
- bpo-19903 [https://bugs.python.org/issue?@action=redirect&bpo=19903]: IDLE: Calltips use `inspect.signature` instead of `inspect.getfullargspec`. This improves calltips for builtins converted to use Argument Clinic. Patch by Louie Lu.
- bpo-31083 [https://bugs.python.org/issue?@action=redirect&bpo=31083]: IDLE - Add an outline of a TabPage class in configdialog. Update existing classes to match outline. Initial patch by Cheryl Sabella.
- bpo-31050 [https://bugs.python.org/issue?@action=redirect&bpo=31050]: Factor GenPage(Frame) class from ConfigDialog. The slightly modified tests continue to pass. Patch by Cheryl Sabella.
- bpo-31004 [https://bugs.python.org/issue?@action=redirect&bpo=31004]: IDLE - Factor FontPage(Frame) class from ConfigDialog. Slightly modified tests continue to pass. Fix General tests. Patch mostly by Cheryl Sabella.
- bpo-30781 [https://bugs.python.org/issue?@action=redirect&bpo=30781]: IDLE - Use ttk widgets in ConfigDialog. Patches by Terry Jan Reedy and Cheryl Sabella.
- bpo-31060 [https://bugs.python.org/issue?@action=redirect&bpo=31060]: IDLE - Finish rearranging methods of ConfigDialog Grouping methods pertaining to each tab and the buttons will aid writing tests and improving the tabs and will enable splitting the groups into classes.
- bpo-30853 [https://bugs.python.org/issue?@action=redirect&bpo=30853]: IDLE – Factor a VarTrace class out of ConfigDialog. Instance tracers manages pairs consisting of a tk variable and a callback function. When tracing is turned on, setting the variable calls the function. Test coverage for the new class is 100%.

- bpo-31003 [https://bugs.python.org/issue?@action=redirect&bpo=31003]: IDLE: Add more tests for General tab.
- bpo-30993 [https://bugs.python.org/issue?@action=redirect&bpo=30993]: IDLE - Improve configdialog font page and tests. In configdialog: Document causal pathways in create_font_tab docstring. Simplify some attribute names. Move set_samples calls to var_changed_font (idea from Cheryl Sabella). Move related functions to positions after the create widgets function. In test_configdialog: Fix test_font_set so not order dependent. Fix renamed test_indent_scale so it tests the widget. Adjust tests for movement of set_samples call. Add tests for load functions. Put all font tests in one class and tab indent tests in another. Except for two lines, these tests completely cover the related functions.
- bpo-30981 [https://bugs.python.org/issue?@action=redirect&bpo=30981]: IDLE – Add more configdialog font page tests.
- bpo-28523 [https://bugs.python.org/issue?@action=redirect&bpo=28523]: IDLE: replace ‘colour’ with ‘color’ in configdialog.
- bpo-30917 [https://bugs.python.org/issue?@action=redirect&bpo=30917]: Add tests for idlelib.config.IdleConf. Increase coverage from 46% to 96%. Patch by Louie Lu.
- bpo-30934 [https://bugs.python.org/issue?@action=redirect&bpo=30934]: Document coverage details for idlelib tests. Add section to idlelib/idle-test/README.txt. Include check that branches are taken both ways. Exclude IDLE-specific code that does not run during unit tests.
- bpo-30913 [https://bugs.python.org/issue?@action=redirect&bpo=30913]: IDLE: Document ConfigDialog tk Vars, methods, and widgets in docstrings This will facilitate improving the dialog and splitting up the class. Original patch by Cheryl Sabella.
- bpo-30899 [https://bugs.python.org/issue?@action=redirect&bpo=30899]: IDLE: Add tests for ConfigParser subclasses in config. Patch by Louie Lu.
- bpo-30881 [https://bugs.python.org/issue?@action=redirect&bpo=30881]: IDLE: Add docstrings to browser.py. Patch by Cheryl Sabella.
- bpo-30851 [https://bugs.python.org/issue?@action=redirect&bpo=30851]: IDLE: Remove unused variables in configdialog. One is a duplicate, one is set but cannot be altered by users. Patch by Cheryl Sabella.
- bpo-30870 [https://bugs.python.org/issue?@action=redirect&bpo=30870]: IDLE: In Settings dialog, select font with Up, Down keys as well as mouse. Initial patch by Louie Lu.

- bpo-8231 [https://bugs.python.org/issue?@action=redirect&bpo=8231]: IDLE: call config.IdleConf.GetUserCfgDir only once.
- bpo-30779 [https://bugs.python.org/issue?@action=redirect&bpo=30779]: IDLE: Factor ConfigChanges class from configdialog, put in config; test. * In config, put dump test code in a function; run it and unittest in ‘if __name__ == ‘__main__’. * Add class config.ConfigChanges based on changes_class_v4.py on bpo issue. * Add class test_config.ChangesTest, partly using configdialog_tests_v1.py. * Revise configdialog to use ConfigChanges; see tracker msg297804. * Revise test_configdialog to match configdialog changes. * Remove configdialog functions unused or moved to ConfigChanges. Cheryl Sabella contributed parts of the patch.
- bpo-30777 [https://bugs.python.org/issue?@action=redirect&bpo=30777]: IDLE: configdialog - Add docstrings and fix comments. Patch by Cheryl Sabella.
- bpo-30495 [https://bugs.python.org/issue?@action=redirect&bpo=30495]: IDLE: Improve textview with docstrings, PEP8 names, and more tests. Patch by Cheryl Sabella.
- bpo-30723 [https://bugs.python.org/issue?@action=redirect&bpo=30723]: IDLE: Make several improvements to parenmatch. Add ‘parens’ style to highlight both opener and closer. Make ‘default’ style, which is not default, a synonym for ‘opener’. Make time-delay work the same with all styles. Add help for config dialog extensions tab, including help for parenmatch. Add new tests. Original patch by Charles Wohlganger.
- bpo-30674 [https://bugs.python.org/issue?@action=redirect&bpo=30674]: IDLE: add docstrings to grep module. Patch by Cheryl Sabella
- bpo-21519 [https://bugs.python.org/issue?@action=redirect&bpo=21519]: IDLE’s basic custom key entry dialog now detects duplicates properly. Original patch by Saimadhav Heblikar.
- bpo-29910 [https://bugs.python.org/issue?@action=redirect&bpo=29910]: IDLE no longer deletes a character after commenting out a region by a key shortcut. Add `return 'break'` for this and other potential conflicts between IDLE and default key bindings.
- bpo-30728 [https://bugs.python.org/issue?@action=redirect&bpo=30728]: Review and change idlelib.configdialog names. Lowercase method and attribute names. Replace ‘colour’ with ‘color’, expand overly cryptic names,

delete unneeded underscores. Replace `import *` with specific imports. Patches by Cheryl Sabella.
- bpo-6739 [https://bugs.python.org/issue?@action=redirect&bpo=6739]: IDLE: Verify user-entered key sequences by trying to bind them with tk. Add tests for all 3 validation functions. Original patch by G Polo. Tests added by Cheryl Sabella.

Ferramentas/Demos

- bpo-30983 [https://bugs.python.org/issue?@action=redirect&bpo=30983]: gdb integration commands (py-bt, etc.) work on optimized shared builds now, too. **PEP 523** [https://peps.python.org/pep-0523/] introduced _PyEval_EvalFrameDefault which inlines PyEval_EvalFrameEx on non-debug shared builds. This broke the ability to use py-bt, py-up, and a few other Python-specific gdb integrations. The problem is fixed by only looking for _PyEval_EvalFrameDefault frames in python-gdb.py. Original patch by Bruno "Polaco" Penteado.

Python 3.6.2 final

Data de lançamento: 2017-07-17

No changes since release candidate 2

Python 3.6.2 release candidate 2

Data de lançamento: 2017-07-07

Segurança

- bpo-30730 [https://bugs.python.org/issue?@action=redirect&bpo=30730]: Prevent environment variables injection in subprocess on Windows. Prevent passing other environment variables and command arguments.
- bpo-30694 [https://bugs.python.org/issue?@action=redirect&bpo=30694]: Upgrade expat copy from 2.2.0 to 2.2.1 to get fixes of multiple security vulnerabilities including: **CVE 2017-9233** [https://www.cve.org/CVERecord?

id=CVE-2017-9233] (External entity infinite loop DoS), **CVE 2016-9063** [https://www.cve.org/CVERecord?id=CVE-2016-9063] (Integer overflow, re-fix), **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718] (Fix regression bugs from 2.2.0's fix to **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718]) and **CVE 2012-0876** [https://www.cve.org/CVERecord?id=CVE-2012-0876] (Counter hash flooding with SipHash). Note: the **CVE 2016-5300** [https://www.cve.org/CVERecord?id=CVE-2016-5300] (Use os-specific entropy sources like getrandom) doesn't impact Python, since Python already gets entropy from the OS to set the expat secret using `XML_SetHashSalt()`.
- bpo-30500 [https://bugs.python.org/issue?@action=redirect&bpo=30500]: Fix urllib.parse.splithost() to correctly parse fragments. For example, `splithost('//127.0.0.1#@evil.com/')` now correctly returns the `127.0.0.1` host, instead of treating `@evil.com` as the host in an authentication (`login@host`).

Python 3.6.2 release candidate 1

Data de lançamento: 2017-06-17

Segurança

- bpo-29591 [https://bugs.python.org/issue?@action=redirect&bpo=29591]: Update expat copy from 2.1.1 to 2.2.0 to get fixes of **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718] and **CVE 2016-4472** [https://www.cve.org/CVERecord?id=CVE-2016-4472]. See https://sourceforge.net/p/expat/bugs/537/ for more information.

Núcleo e embutidos

- bpo-30682 [https://bugs.python.org/issue?@action=redirect&bpo=30682]: Removed a too-strict assertion that failed for certain f-strings, such as eval("f'\n'") and eval("f'\r'").
- bpo-30604 [https://bugs.python.org/issue?@action=redirect&bpo=30604]: Move co_extra_freefuncs to not be per-thread to avoid crashes

- bpo-29104 [https://bugs.python.org/issue?@action=redirect&bpo=29104]: Fixed parsing backslashes in f-strings.
- bpo-27945 [https://bugs.python.org/issue?@action=redirect&bpo=27945]: Fixed various segfaults with dict when input collections are mutated during searching, inserting or comparing. Based on patches by Duane Griffin and Tim Mitchell.
- bpo-25794 [https://bugs.python.org/issue?@action=redirect&bpo=25794]: Fixed type.__setattr__() and type.__delattr__() for non-interned attribute names. Based on patch by Eryk Sun.
- bpo-30039 [https://bugs.python.org/issue?@action=redirect&bpo=30039]: If a KeyboardInterrupt happens when the interpreter is in the middle of resuming a chain of nested ‘yield from’ or ‘await’ calls, it’s now correctly delivered to the innermost frame.
- bpo-12414 [https://bugs.python.org/issue?@action=redirect&bpo=12414]: sys.getsizeof() on a code object now returns the sizes which includes the code struct and sizes of objects which it references. Patch by Donghee Na.
- bpo-29949 [https://bugs.python.org/issue?@action=redirect&bpo=29949]: Fix memory usage regression of set and frozenset object.
- bpo-29935 [https://bugs.python.org/issue?@action=redirect&bpo=29935]: Fixed error messages in the index() method of tuple, list and deque when pass indices of wrong type.
- bpo-29859 [https://bugs.python.org/issue?@action=redirect&bpo=29859]: Show correct error messages when any of the pthread_* calls in thread_pthread.h fails.
- bpo-28876 [https://bugs.python.org/issue?@action=redirect&bpo=28876]: `bool(range)` works even if `len(range)` raises **`OverflowError`**.
- bpo-29600 [https://bugs.python.org/issue?@action=redirect&bpo=29600]: Fix wrapping coroutine return values in StopIteration.
- bpo-28856 [https://bugs.python.org/issue?@action=redirect&bpo=28856]: Fix an oversight that %b format for bytes should support objects follow the buffer protocol.
- bpo-29714 [https://bugs.python.org/issue?@action=redirect&bpo=29714]: Fix a regression that bytes format may fail when containing zero bytes inside.
- bpo-29478 [https://bugs.python.org/issue?@action=redirect&bpo=29478]: If max_line_length=None is specified while using the Compat32 policy,

it is no longer ignored. Patch by Mircea Cosbuc.

Biblioteca

- bpo-30616 [https://bugs.python.org/issue?@action=redirect&bpo=30616]: Functional API of enum allows to create empty enums. Patched by Donghee Na
- bpo-30038 [https://bugs.python.org/issue?@action=redirect&bpo=30038]: Fix race condition between signal delivery and wakeup file descriptor. Patch by Nathaniel Smith.
- bpo-23894 [https://bugs.python.org/issue?@action=redirect&bpo=23894]: lib2to3 now recognizes `rb'...'` and `f'...'` strings.
- bpo-23890 [https://bugs.python.org/issue?@action=redirect&bpo=23890]: unittest.TestCase.assertRaises() now manually breaks a reference cycle to not keep objects alive longer than expected.
- bpo-30149 [https://bugs.python.org/issue?@action=redirect&bpo=30149]: inspect.signature() now supports callables with variable-argument parameters wrapped with partialmethod. Patch by Donghee Na.
- bpo-30645 [https://bugs.python.org/issue?@action=redirect&bpo=30645]: Fix path calculation in imp.load_package(), fixing it for cases when a package is only shipped with bytecodes. Patch by Alexandru Ardelean.
- bpo-29931 [https://bugs.python.org/issue?@action=redirect&bpo=29931]: Fixed comparison check for ipaddress.ip_interface objects. Patch by Sanjay Sundaresan.
- bpo-30605 [https://bugs.python.org/issue?@action=redirect&bpo=30605]: re.compile() no longer raises a BytesWarning when compiling a bytes instance with misplaced inline modifier. Patch by Roy Williams.
- bpo-24484 [https://bugs.python.org/issue?@action=redirect&bpo=24484]: Avoid race condition in multiprocessing cleanup (#2159)
- bpo-28994 [https://bugs.python.org/issue?@action=redirect&bpo=28994]: The traceback no longer displayed for SystemExit raised in a callback registered by atexit.
- bpo-30508 [https://bugs.python.org/issue?@action=redirect&bpo=30508]: Don’t log exceptions if Task/Future “cancel()” method was called.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Updates to typing module: Add generic AsyncContextManager, add support for ContextManager on all versions. Original PRs by Jelle Zijlstra and Ivan Levkivskyi

- bpo-29870 [https://bugs.python.org/issue?@action=redirect&bpo=29870]: Fix ssl sockets leaks when connection is aborted in asyncio/ssl implementation. Patch by Michaël Sghaïer.
- bpo-29743 [https://bugs.python.org/issue?@action=redirect&bpo=29743]: Closing transport during handshake process leaks open socket. Patch by Nikolay Kim
- bpo-27585 [https://bugs.python.org/issue?@action=redirect&bpo=27585]: Fix waiter cancellation in asyncio.Lock. Patch by Mathieu Sornay.
- bpo-30418 [https://bugs.python.org/issue?@action=redirect&bpo=30418]: On Windows, subprocess.Popen.communicate() now also ignore EINVAL on stdin.write() if the child process is still running but closed the pipe.
- bpo-29822 [https://bugs.python.org/issue?@action=redirect&bpo=29822]: inspect.isabstract() now works during __init_subclass__. Patch by Nate Soares.
- bpo-29581 [https://bugs.python.org/issue?@action=redirect&bpo=29581]: ABCMeta.__new__ now accepts `**kwargs`, allowing abstract base classes to use keyword parameters in __init_subclass__. Patch by Nate Soares.
- bpo-30557 [https://bugs.python.org/issue?@action=redirect&bpo=30557]: faulthandler now correctly filters and displays exception codes on Windows
- bpo-30378 [https://bugs.python.org/issue?@action=redirect&bpo=30378]: Fix the problem that logging.handlers.SysLogHandler cannot handle IPv6 addresses.
- bpo-29960 [https://bugs.python.org/issue?@action=redirect&bpo=29960]: Preserve generator state when _random.Random.setstate() raises an exception. Patch by Bryan Olson.
- bpo-30414 [https://bugs.python.org/issue?@action=redirect&bpo=30414]: multiprocessing.Queue._feed background running thread do not break from main loop on exception.
- bpo-30003 [https://bugs.python.org/issue?@action=redirect&bpo=30003]: Fix handling escape characters in HZ codec. Based on patch by Ma Lin.
- bpo-30301 [https://bugs.python.org/issue?@action=redirect&bpo=30301]: Fix AttributeError when using SimpleQueue.empty() under *spawn* and *forkserver* start methods.
- bpo-30329 [https://bugs.python.org/issue?@action=redirect&bpo=30329]: imaplib and poplib now catch the Windows socket WSAEINVAL error (code

10022) on shutdown(SHUT_RDWR): An invalid operation was attempted. This error occurs sometimes on SSL connections.

- bpo-30375 [https://bugs.python.org/issue?@action=redirect&bpo=30375]: Warnings emitted when compile a regular expression now always point to the line in the user code. Previously they could point into inners of the re module if emitted from inside of groups or conditionals.
- bpo-30048 [https://bugs.python.org/issue?@action=redirect&bpo=30048]: Fixed `Task.cancel()` can be ignored when the task is running coroutine and the coroutine returned without any more `await`.
- bpo-30266 [https://bugs.python.org/issue?@action=redirect&bpo=30266]: contextlib.AbstractContextManager now supports anti-registration by setting __enter__ = None or __exit__ = None, following the pattern introduced in bpo-25958 [https://bugs.python.org/issue?@action=redirect&bpo=25958]. Patch by Jelle Zijlstra.
- bpo-30298 [https://bugs.python.org/issue?@action=redirect&bpo=30298]: Weaken the condition of deprecation warnings for inline modifiers. Now allowed several subsequential inline modifiers at the start of the pattern (e.g. `'(?i)(?s)...'`). In verbose mode whitespaces and comments now are allowed before and between inline modifiers (e.g. `'(?x) (?i) (?s)...'`).
- bpo-29990 [https://bugs.python.org/issue?@action=redirect&bpo=29990]: Fix range checking in GB18030 decoder. Original patch by Ma Lin.
- bpo-26293 [https://bugs.python.org/issue?@action=redirect&bpo=26293]: Change resulted because of zipfile breakage. (See also: bpo-29094 [https://bugs.python.org/issue?@action=redirect&bpo=29094])
- bpo-30243 [https://bugs.python.org/issue?@action=redirect&bpo=30243]: Removed the __init__ methods of _json's scanner and encoder. Misusing them could cause memory leaks or crashes. Now scanner and encoder objects are completely initialized in the __new__ methods.
- bpo-30185 [https://bugs.python.org/issue?@action=redirect&bpo=30185]: Avoid KeyboardInterrupt tracebacks in forkserver helper process when Ctrl-C is received.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Various updates to typing module: add typing.NoReturn type, use WrapperDescriptorType, minor bug-fixes. Original PRs by Jim Fasarakis-Hilliard and Ivan Levkivskyi.

- bpo-30205 [https://bugs.python.org/issue?@action=redirect&bpo=30205]: Fix getsockname() for unbound AF_UNIX sockets on Linux.
- bpo-30070 [https://bugs.python.org/issue?@action=redirect&bpo=30070]: Fixed leaks and crashes in errors handling in the parser module.
- bpo-30061 [https://bugs.python.org/issue?@action=redirect&bpo=30061]: Fixed crashes in IOBase methods __next__() and readlines() when readline() or __next__() respectively return non-sizeable object. Fixed possible other errors caused by not checking results of PyObject_Size(), PySequence_Size(), or PyMapping_Size().
- bpo-30017 [https://bugs.python.org/issue?@action=redirect&bpo=30017]: Allowed calling the close() method of the zip entry writer object multiple times. Writing to a closed writer now always produces a ValueError.
- bpo-30068 [https://bugs.python.org/issue?@action=redirect&bpo=30068]: _io._IOBase.readlines will check if it's closed first when hint is present.
- bpo-29694 [https://bugs.python.org/issue?@action=redirect&bpo=29694]: Fixed race condition in pathlib mkdir with flags parents=True. Patch by Armin Rigo.
- bpo-29692 [https://bugs.python.org/issue?@action=redirect&bpo=29692]: Fixed arbitrary unchaining of RuntimeError exceptions in contextlib.contextmanager. Patch by Siddharth Velankar.
- bpo-29998 [https://bugs.python.org/issue?@action=redirect&bpo=29998]: Pickling and copying ImportError now preserves name and path attributes.
- bpo-29953 [https://bugs.python.org/issue?@action=redirect&bpo=29953]: Fixed memory leaks in the replace() method of datetime and time objects when pass out of bound fold argument.
- bpo-29942 [https://bugs.python.org/issue?@action=redirect&bpo=29942]: Fix a crash in itertools.chain.from_iterable when encountering long runs of empty iterables.
- bpo-27863 [https://bugs.python.org/issue?@action=redirect&bpo=27863]: Fixed multiple crashes in ElementTree caused by race conditions and wrong types.
- bpo-28699 [https://bugs.python.org/issue?@action=redirect&bpo=28699]: Fixed a bug in pools in multiprocessing.pool that raising an exception at the very first of an iterable may swallow the exception or make the program hang. Patch by Davin Potts and Xiang Zhang.

- bpo-25803 [https://bugs.python.org/issue?@action=redirect&bpo=25803]: Avoid incorrect errors raised by Path.mkdir(exist_ok=True) when the OS gives priority to errors such as EACCES over EEXIST.
- bpo-29861 [https://bugs.python.org/issue?@action=redirect&bpo=29861]: Release references to tasks, their arguments and their results as soon as they are finished in multiprocessing.Pool.
- bpo-29884 [https://bugs.python.org/issue?@action=redirect&bpo=29884]: faulthandler: Restore the old sigaltstack during teardown. Patch by Christophe Zeitouny.
- bpo-25455 [https://bugs.python.org/issue?@action=redirect&bpo=25455]: Fixed crashes in repr of recursive buffered file-like objects.
- bpo-29800 [https://bugs.python.org/issue?@action=redirect&bpo=29800]: Fix crashes in partial.__repr__ if the keys of partial.keywords are not strings. Patch by Michael Seifert.
- bpo-29742 [https://bugs.python.org/issue?@action=redirect&bpo=29742]: get_extra_info() raises exception if get called on closed ssl transport. Patch by Nikolay Kim.
- bpo-8256 [https://bugs.python.org/issue?@action=redirect&bpo=8256]: Fixed possible failing or crashing input() if attributes “encoding” or “errors” of sys.stdin or sys.stdout are not set or are not strings.
- bpo-28298 [https://bugs.python.org/issue?@action=redirect&bpo=28298]: Fix a bug that prevented array ‘Q’, ‘L’ and ‘I’ from accepting big intables (objects that have __int__) as elements. Patch by Oren Milman.
- bpo-28231 [https://bugs.python.org/issue?@action=redirect&bpo=28231]: The zipfile module now accepts path-like objects for external paths.
- bpo-26915 [https://bugs.python.org/issue?@action=redirect&bpo=26915]: index() and count() methods of collections.abc.Sequence now check identity before checking equality when do comparisons.
- bpo-29615 [https://bugs.python.org/issue?@action=redirect&bpo=29615]: SimpleXMLRPCDispatcher no longer chains KeyError (or any other exception) to exception(s) raised in the dispatched methods. Patch by Petr Motejlek.
- bpo-30177 [https://bugs.python.org/issue?@action=redirect&bpo=30177]: path.resolve(strict=False) no longer cuts the path after the first element not present in the filesystem. Patch by Antoine Pietri.

IDLE

- bpo-15786 [https://bugs.python.org/issue?@action=redirect&bpo=15786]: Fix several problems with IDLE's autocompletion box. The following should now work: clicking on selection box items; using the scrollbar; selecting an item by hitting Return. Hangs on MacOSX should no longer happen. Patch by Louie Lu.
- bpo-25514 [https://bugs.python.org/issue?@action=redirect&bpo=25514]: Add doc subsubsection about IDLE failure to start. Popup no-connection message directs users to this section.
- bpo-30642 [https://bugs.python.org/issue?@action=redirect&bpo=30642]: Fix reference leaks in IDLE tests. Patches by Louie Lu and Terry Jan Reedy.
- bpo-30495 [https://bugs.python.org/issue?@action=redirect&bpo=30495]: Add docstrings for textview.py and use PEP8 names. Patches by Cheryl Sabella and Terry Jan Reedy.
- bpo-30290 [https://bugs.python.org/issue?@action=redirect&bpo=30290]: Help-about: use pep8 names and add tests. Increase coverage to 100%. Patches by Louie Lu, Cheryl Sabella, and Terry Jan Reedy.
- bpo-30303 [https://bugs.python.org/issue?@action=redirect&bpo=30303]: Add _utest option to textview; add new tests. Increase coverage to 100%. Patches by Louie Lu and Terry Jan Reedy.

API C

- bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867]: Function PySlice_GetIndicesEx() no longer replaced with a macro if Py_LIMITED_API is not set.

Construção

- bpo-29941 [https://bugs.python.org/issue?@action=redirect&bpo=29941]: Add `--with-assertions` configure flag to explicitly enable C `assert()` checks. Defaults to off. `--with-pydebug` implies `--with-assertions`.
- bpo-28787 [https://bugs.python.org/issue?@action=redirect&bpo=28787]: Fix out-of-tree builds of Python when configured with `--with--dtrace`.
- bpo-29243 [https://bugs.python.org/issue?@action=redirect&bpo=29243]: Prevent unnecessary rebuilding of Python during `make test`, `make install`

and some other make targets when configured with `--enable-optimizations`.

- bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404]: Don't regenerate generated files based on file modification time anymore: the action is now explicit. Replace `make touch` with `make regen-all`.
- bpo-29643 [https://bugs.python.org/issue?@action=redirect&bpo=29643]: Fix `--enable-optimization` didn't work.

Documentação

- bpo-30176 [https://bugs.python.org/issue?@action=redirect&bpo=30176]: Add missing attribute related constants in curses documentation.
- bpo-30052 [https://bugs.python.org/issue?@action=redirect&bpo=30052]: the link targets for **`bytes()`** and **`bytearray()`** are now their respective type definitions, rather than the corresponding builtin function entries. Use bytes and bytearray to reference the latter. In order to ensure this and future cross-reference updates are applied automatically, the daily documentation builds now disable the default output caching features in Sphinx.
- bpo-26985 [https://bugs.python.org/issue?@action=redirect&bpo=26985]: Add missing info of code object in inspect documentation.

Ferramentas/Demos

- bpo-29367 [https://bugs.python.org/issue?@action=redirect&bpo=29367]: python-gdb.py now supports also `method-wrapper` (`wrapperobject`) objects.

Testes

- bpo-30357 [https://bugs.python.org/issue?@action=redirect&bpo=30357]: test_thread: setUp() now uses support.threading_setup() and support.threading_cleanup() to wait until threads complete to avoid random side effects on following tests. Initial patch written by Grzegorz Grzywacz.
- bpo-30197 [https://bugs.python.org/issue?@action=redirect&bpo=30197]: Enhanced functions swap_attr() and swap_item() in the test.support module. They now work when delete replaced attribute or item inside the with

statement. The old value of the attribute or item (or None if it doesn't exist) now will be assigned to the target of the "as" clause, if there is one.

Windows

- bpo-30687 [https://bugs.python.org/issue?@action=redirect&bpo=30687]: Locate msbuild.exe on Windows when building rather than vcvarsall.bat
- bpo-30450 [https://bugs.python.org/issue?@action=redirect&bpo=30450]: The build process on Windows no longer depends on Subversion, instead pulling external code from GitHub via a Python script. If Python 3.6 is not found on the system (via `py -3.6`), NuGet is used to download a copy of 32-bit Python.

Python 3.6.1 final

Data de lançamento: 2017-03-21

Núcleo e embutidos

- bpo-29723 [https://bugs.python.org/issue?@action=redirect&bpo=29723]: The `sys.path[0]` initialization change for bpo-29139 [https://bugs.python.org/issue?@action=redirect&bpo=29139] caused a regression by revealing an inconsistency in how sys.path is initialized when executing `__main__` from a zipfile, directory, or other import location. The interpreter now consistently avoids ever adding the import location's parent directory to `sys.path`, and ensures no other `sys.path` entries are inadvertently modified when inserting the import location named on the command line.

Construção

- bpo-27593 [https://bugs.python.org/issue?@action=redirect&bpo=27593]: fix format of git information used in sys.version
- Corrigir comentário incompatível em python.h

Python 3.6.1 release candidate 1

Data de lançamento: 2017-03-04

Núcleo e embutidos

- bpo-28893 [https://bugs.python.org/issue?@action=redirect&bpo=28893]: Set correct __cause__ for errors about invalid awaitables returned from __aiter__ and __anext__.
- bpo-29683 [https://bugs.python.org/issue?@action=redirect&bpo=29683]: Fixes to memory allocation in _PyCode_SetExtra. Patch by Brian Coleman.
- bpo-29684 [https://bugs.python.org/issue?@action=redirect&bpo=29684]: Fix minor regression of PyEval_CallObjectWithKeywords. It should raise TypeError when kwargs is not a dict. But it might cause segv when args=NULL and kwargs is not a dict.
- bpo-28598 [https://bugs.python.org/issue?@action=redirect&bpo=28598]: Support __rmod__ for subclasses of str being called before str.__mod__. Patch by Martijn Pieters.
- bpo-29607 [https://bugs.python.org/issue?@action=redirect&bpo=29607]: Fix stack_effect computation for CALL_FUNCTION_EX. Patch by Matthieu Dartiailh.
- bpo-29602 [https://bugs.python.org/issue?@action=redirect&bpo=29602]: Fix incorrect handling of signed zeros in complex constructor for complex subclasses and for inputs having a __complex__ method. Patch by Serhiy Storchaka.
- bpo-29347 [https://bugs.python.org/issue?@action=redirect&bpo=29347]: Fixed possibly dereferencing undefined pointers when creating weakref objects.
- bpo-29438 [https://bugs.python.org/issue?@action=redirect&bpo=29438]: Fixed use-after-free problem in key sharing dict.
- bpo-29319 [https://bugs.python.org/issue?@action=redirect&bpo=29319]: Prevent RunMainFromImporter overwriting sys.path[0].
- bpo-29337 [https://bugs.python.org/issue?@action=redirect&bpo=29337]: Fixed possible BytesWarning when compare the code objects. Warnings could be emitted at compile time.

- bpo-29327 [https://bugs.python.org/issue?@action=redirect&bpo=29327]: Fixed a crash when pass the iterable keyword argument to sorted().
- bpo-29034 [https://bugs.python.org/issue?@action=redirect&bpo=29034]: Fix memory leak and use-after-free in os module (path_converter).
- bpo-29159 [https://bugs.python.org/issue?@action=redirect&bpo=29159]: Fix regression in bytes(x) when x.__index__() raises Exception.
- bpo-28932 [https://bugs.python.org/issue?@action=redirect&bpo=28932]: Do not include <sys/random.h> if it does not exist.
- bpo-25677 [https://bugs.python.org/issue?@action=redirect&bpo=25677]: Correct the positioning of the syntax error caret for indented blocks. Based on patch by Michael Layzell.
- bpo-29000 [https://bugs.python.org/issue?@action=redirect&bpo=29000]: Fixed bytes formatting of octals with zero padding in alternate form.
- bpo-26919 [https://bugs.python.org/issue?@action=redirect&bpo=26919]: On Android, operating system data is now always encoded/decoded to/from UTF-8, instead of the locale encoding to avoid inconsistencies with os.fsencode() and os.fsdecode() which are already using UTF-8.
- bpo-28991 [https://bugs.python.org/issue?@action=redirect&bpo=28991]: functools.lru_cache() was susceptible to an obscure reentrancy bug triggerable by a monkey-patched len() function.
- bpo-28739 [https://bugs.python.org/issue?@action=redirect&bpo=28739]: f-string expressions are no longer accepted as docstrings and by ast.literal_eval() even if they do not include expressions.
- bpo-28512 [https://bugs.python.org/issue?@action=redirect&bpo=28512]: Fixed setting the offset attribute of SyntaxError by PyErr_SyntaxLocationEx() and PyErr_SyntaxLocationObject().
- bpo-28918 [https://bugs.python.org/issue?@action=redirect&bpo=28918]: Fix the cross compilation of xxlimited when Python has been built with Py_DEBUG defined.
- bpo-28731 [https://bugs.python.org/issue?@action=redirect&bpo=28731]: Optimize _PyDict_NewPresized() to create correct size dict. Improve speed of dict literal with constant keys up to 30%.

Biblioteca

- bpo-29169 [https://bugs.python.org/issue?@action=redirect&bpo=29169]: Update zlib to 1.2.11.

- bpo-29623 [https://bugs.python.org/issue?@action=redirect&bpo=29623]: Allow use of path-like object as a single argument in ConfigParser.read(). Patch by David Ellis.
- bpo-28963 [https://bugs.python.org/issue?@action=redirect&bpo=28963]: Fix out of bound iteration in asyncio.Future.remove_done_callback implemented in C.
- bpo-29704 [https://bugs.python.org/issue?@action=redirect&bpo=29704]: asyncio.subprocess.SubprocessStreamProtocol no longer closes before all pipes are closed.
- bpo-29271 [https://bugs.python.org/issue?@action=redirect&bpo=29271]: Fix Task.current_task and Task.all_tasks implemented in C to accept None argument as their pure Python implementation.
- bpo-29703 [https://bugs.python.org/issue?@action=redirect&bpo=29703]: Fix asyncio to support instantiation of new event loops in child processes.
- bpo-29376 [https://bugs.python.org/issue?@action=redirect&bpo=29376]: Fix assertion error in threading._DummyThread.is_alive().
- bpo-28624 [https://bugs.python.org/issue?@action=redirect&bpo=28624]: Add a test that checks that cwd parameter of Popen() accepts PathLike objects. Patch by Sayan Chowdhury.
- bpo-28518 [https://bugs.python.org/issue?@action=redirect&bpo=28518]: Start a transaction implicitly before a DML statement. Patch by Aviv Palivoda.
- bpo-29532 [https://bugs.python.org/issue?@action=redirect&bpo=29532]: Altering a kwarg dictionary passed to functools.partial() no longer affects a partial object after creation.
- bpo-29110 [https://bugs.python.org/issue?@action=redirect&bpo=29110]: Fix file object leak in aifc.open() when file is given as a filesystem path and is not in valid AIFF format. Patch by Anthony Zhang.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Various updates to typing module: typing.Counter, typing.ChainMap, improved ABC caching, etc. Original PRs by Jelle Zijlstra, Ivan Levkivskyi, Manuel Krebber, and Łukasz Langa.
- bpo-29100 [https://bugs.python.org/issue?@action=redirect&bpo=29100]: Fix datetime.fromtimestamp() regression introduced in Python 3.6.0: check minimum and maximum years.
- bpo-29519 [https://bugs.python.org/issue?@action=redirect&bpo=29519]: Fix weakref spewing exceptions during interpreter shutdown when used

with a rare combination of multiprocessing and custom codecs.

- bpo-29416 [https://bugs.python.org/issue?@action=redirect&bpo=29416]: Prevent infinite loop in pathlib.Path.mkdir
- bpo-29444 [https://bugs.python.org/issue?@action=redirect&bpo=29444]: Fixed out-of-bounds buffer access in the group() method of the match object. Based on patch by WGH.
- bpo-29335 [https://bugs.python.org/issue?@action=redirect&bpo=29335]: Fix subprocess.Popen.wait() when the child process has exited to a stopped instead of terminated state (ex: when under ptrace).
- bpo-29290 [https://bugs.python.org/issue?@action=redirect&bpo=29290]: Fix a regression in argparse that help messages would wrap at non-breaking spaces.
- bpo-28735 [https://bugs.python.org/issue?@action=redirect&bpo=28735]: Fixed the comparison of mock.MagickMock with mock.ANY.
- bpo-29316 [https://bugs.python.org/issue?@action=redirect&bpo=29316]: Restore the provisional status of typing module, add corresponding note to documentation. Patch by Ivan L.
- bpo-29219 [https://bugs.python.org/issue?@action=redirect&bpo=29219]: Fixed infinite recursion in the repr of uninitialized ctypes.CDLL instances.
- bpo-29011 [https://bugs.python.org/issue?@action=redirect&bpo=29011]: Fix an important omission by adding Deque to the typing module.
- bpo-28969 [https://bugs.python.org/issue?@action=redirect&bpo=28969]: Fixed race condition in C implementation of functools.lru_cache. KeyError could be raised when cached function with full cache was simultaneously called from different threads with the same uncached arguments.
- bpo-29142 [https://bugs.python.org/issue?@action=redirect&bpo=29142]: In urllib.request, suffixes in no_proxy environment variable with leading dots could match related hostnames again (e.g. .b.c matches a.b.c). Patch by Milan Oberkirch.
- bpo-28961 [https://bugs.python.org/issue?@action=redirect&bpo=28961]: Fix unittest.mock._Call helper: don’t ignore the name parameter anymore. Patch written by Jiajun Huang.
- bpo-29203 [https://bugs.python.org/issue?@action=redirect&bpo=29203]: functools.lru_cache() now respects **PEP 468** [https://peps.python.org/pep-0468/] and preserves the order of keyword arguments. f(a=1, b=2) is now cached separately from f(b=2, a=1) since both calls could potentially give different results.

- bpo-15812 [https://bugs.python.org/issue?@action=redirect&bpo=15812]: inspect.getframeinfo() now correctly shows the first line of a context. Patch by Sam Breese.
- bpo-29094 [https://bugs.python.org/issue?@action=redirect&bpo=29094]: Offsets in a ZIP file created with extern file object and modes “w” and “x” now are relative to the start of the file.
- bpo-29085 [https://bugs.python.org/issue?@action=redirect&bpo=29085]: Allow random.Random.seed() to use high quality OS randomness rather than the pid and time.
- bpo-29061 [https://bugs.python.org/issue?@action=redirect&bpo=29061]: Fixed bug in secrets.randbelow() which would hang when given a negative input. Patch by Brendan Donegan.
- bpo-29079 [https://bugs.python.org/issue?@action=redirect&bpo=29079]: Prevent infinite loop in pathlib.resolve() on Windows
- bpo-13051 [https://bugs.python.org/issue?@action=redirect&bpo=13051]: Fixed recursion errors in large or resized curses.textpad.Textbox. Based on patch by Tycho Andersen.
- bpo-29119 [https://bugs.python.org/issue?@action=redirect&bpo=29119]: Fix weakrefs in the pure python version of collections.OrderedDict move_to_end() method. Contributed by Andra Bogildea.
- bpo-9770 [https://bugs.python.org/issue?@action=redirect&bpo=9770]: curses.ascii predicates now work correctly with negative integers.
- bpo-28427 [https://bugs.python.org/issue?@action=redirect&bpo=28427]: old keys should not remove new values from WeakValueDictionary when collecting from another thread.
- bpo-28923 [https://bugs.python.org/issue?@action=redirect&bpo=28923]: Remove editor artifacts from Tix.py.
- bpo-29055 [https://bugs.python.org/issue?@action=redirect&bpo=29055]: Neaten-up empty population error on random.choice() by suppressing the upstream exception.
- bpo-28871 [https://bugs.python.org/issue?@action=redirect&bpo=28871]: Fixed a crash when deallocate deep ElementTree.
- bpo-19542 [https://bugs.python.org/issue?@action=redirect&bpo=19542]: Fix bugs in WeakValueDictionary.setdefault() and WeakValueDictionary.pop() when a GC collection happens in another thread.
- bpo-20191 [https://bugs.python.org/issue?@action=redirect&bpo=20191]: Fixed a crash in resource.prlimit() when passing a sequence that doesn’t own

its elements as limits.

- bpo-28779 [https://bugs.python.org/issue?@action=redirect&bpo=28779]: multiprocessing.set_forkserver_preload() would crash the forkserver process if a preloaded module instantiated some multiprocessing objects such as locks.
- bpo-28847 [https://bugs.python.org/issue?@action=redirect&bpo=28847]: dbm.dumb now supports reading read-only files and no longer writes the index file when it is not changed.
- bpo-26937 [https://bugs.python.org/issue?@action=redirect&bpo=26937]: The chown() method of the tarfile.TarFile class does not fail now when the grp module cannot be imported, as for example on Android platforms.

IDLE

- bpo-29071 [https://bugs.python.org/issue?@action=redirect&bpo=29071]: IDLE colors f-string prefixes (but not invalid ur prefixes).
- bpo-28572 [https://bugs.python.org/issue?@action=redirect&bpo=28572]: Add 10% to coverage of IDLE's test_configdialog. Update and augment description of the configuration system.

Windows

- bpo-29579 [https://bugs.python.org/issue?@action=redirect&bpo=29579]: Removes readme.txt from the installer
- bpo-29326 [https://bugs.python.org/issue?@action=redirect&bpo=29326]: Ignores blank lines in ._pth files (Patch by Alexey Izbyshev)
- bpo-28164 [https://bugs.python.org/issue?@action=redirect&bpo=28164]: Correctly handle special console filenames (patch by Eryk Sun)
- bpo-29409 [https://bugs.python.org/issue?@action=redirect&bpo=29409]: Implement **PEP 529** [https://peps.python.org/pep-0529/] for io.FileIO (Patch by Eryk Sun)
- bpo-29392 [https://bugs.python.org/issue?@action=redirect&bpo=29392]: Prevent crash when passing invalid arguments into msvcrt module.
- bpo-25778 [https://bugs.python.org/issue?@action=redirect&bpo=25778]: winreg does not truncate string correctly (Patch by Eryk Sun)
- bpo-28896 [https://bugs.python.org/issue?@action=redirect&bpo=28896]: Deprecate WindowsRegistryFinder and disable it by default.

API C

- bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867]: Function PySlice_GetIndicesEx() is replaced with a macro if Py_LIMITED_API is not set or set to the value between 0x03050400 and 0x03060000 (not including) or 0x03060100 or higher.
- bpo-29083 [https://bugs.python.org/issue?@action=redirect&bpo=29083]: Fixed the declaration of some public API functions. PyArg_VaParse() and PyArg_VaParseTupleAndKeywords() were not available in limited API. PyArg_ValidateKeywordArguments(), PyArg_UnpackTuple() and Py_BuildValue() were not available in limited API of version < 3.3 when PY_SSIZE_T_CLEAN is defined.
- bpo-29058 [https://bugs.python.org/issue?@action=redirect&bpo=29058]: All stable API extensions added after Python 3.2 are now available only when Py_LIMITED_API is set to the PY_VERSION_HEX value of the minimum Python version supporting this API.

Documentação

- bpo-28929 [https://bugs.python.org/issue?@action=redirect&bpo=28929]: Link the documentation to its source file on GitHub.
- bpo-25008 [https://bugs.python.org/issue?@action=redirect&bpo=25008]: Document smtpd.py as effectively deprecated and add a pointer to aiosmtpd, a third-party asyncio-based replacement.
- bpo-26355 [https://bugs.python.org/issue?@action=redirect&bpo=26355]: Add canonical header link on each page to corresponding major version of the documentation. Patch by Matthias Bussonnier.
- bpo-29349 [https://bugs.python.org/issue?@action=redirect&bpo=29349]: Fix Python 2 syntax in code for building the documentation.

Testes

- bpo-28087 [https://bugs.python.org/issue?@action=redirect&bpo=28087]: Skip test_asyncore and test_eintr poll failures on macOS. Skip some tests of select.poll when running on macOS due to unresolved issues with the underlying system poll function on some macOS versions.

- bpo-29571 [https://bugs.python.org/issue?@action=redirect&bpo=29571]: to match the behaviour of the `re.LOCALE` flag, test_re.test_locale_flag now uses `locale.getpreferredencoding(False)` to determine the candidate encoding for the test regex (allowing it to correctly skip the test when the default locale encoding is a multi-byte encoding)
- bpo-28950 [https://bugs.python.org/issue?@action=redirect&bpo=28950]: Disallow -j0 to be combined with -T/-l in regrtest command line arguments.
- bpo-28683 [https://bugs.python.org/issue?@action=redirect&bpo=28683]: Fix the tests that bind() a unix socket and raise PermissionError on Android for a non-root user.
- bpo-26939 [https://bugs.python.org/issue?@action=redirect&bpo=26939]: Add the support.setswitchinterval() function to fix test_functools hanging on the Android armv7 qemu emulator.

Construção

- bpo-27593 [https://bugs.python.org/issue?@action=redirect&bpo=27593]: sys.version and the platform module python_build(), python_branch(), and python_revision() functions now use git information rather than hg when building from a repo.
- bpo-29572 [https://bugs.python.org/issue?@action=redirect&bpo=29572]: Update Windows build and OS X installers to use OpenSSL 1.0.2k.
- bpo-26851 [https://bugs.python.org/issue?@action=redirect&bpo=26851]: Set Android compilation and link flags.
- bpo-28768 [https://bugs.python.org/issue?@action=redirect&bpo=28768]: Fix implicit declaration of function _setmode. Patch by Masayuki Yamamoto
- bpo-29080 [https://bugs.python.org/issue?@action=redirect&bpo=29080]: Removes hard dependency on hg.exe from PCBuild/build.bat
- bpo-23903 [https://bugs.python.org/issue?@action=redirect&bpo=23903]: Added missed names to PC/python3.def.
- bpo-28762 [https://bugs.python.org/issue?@action=redirect&bpo=28762]: lockf() is available on Android API level 24, but the F_LOCK macro is not defined in android-ndk-r13.
- bpo-28538 [https://bugs.python.org/issue?@action=redirect&bpo=28538]: Fix the compilation error that occurs because if_nameindex() is available on Android API level 24, but the if_nameindex structure is not defined.

- bpo-20211 [https://bugs.python.org/issue?@action=redirect&bpo=20211]: Do not add the directory for installing C header files and the directory for installing object code libraries to the cross compilation search paths. Original patch by Thomas Petazzoni.
- bpo-28849 [https://bugs.python.org/issue?@action=redirect&bpo=28849]: Do not define sys.implementation._multiarch on Android.

Python 3.6.0 final

Data de lançamento: 2016-12-23

No changes since release candidate 2

Python 3.6.0 release candidate 2

Data de lançamento: 2016-12-16

Núcleo e embutidos

- bpo-28147 [https://bugs.python.org/issue?@action=redirect&bpo=28147]: Fix a memory leak in split-table dictionaries: setattr() must not convert combined table into split table. Patch written by INADA Naoki.
- bpo-28990 [https://bugs.python.org/issue?@action=redirect&bpo=28990]: Fix asyncio SSL hanging if connection is closed before handshake is completed. (Patch by HoHo-Ho)

Ferramentas/Demos

- bpo-28770 [https://bugs.python.org/issue?@action=redirect&bpo=28770]: Fix python-gdb.py for fastcalls.

Windows

- bpo-28896 [https://bugs.python.org/issue?@action=redirect&bpo=28896]: Deprecate WindowsRegistryFinder.

Construção

- bpo-28898 [https://bugs.python.org/issue?@action=redirect&bpo=28898]: Prevent gdb build errors due to HAVE_LONG_LONG redefinition.

Python 3.6.0 release candidate 1

Data de lançamento: 2016-12-06

Núcleo e embutidos

- bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722]: Rather than silently producing a class that doesn't support zero-argument `super()` in methods, failing to pass the new `__classcell__` namespace entry up to `type.__new__` now results in a `DeprecationWarning` and a class that supports zero-argument `super()`.
- bpo-28797 [https://bugs.python.org/issue?@action=redirect&bpo=28797]: Modifying the class __dict__ inside the __set_name__ method of a descriptor that is used inside that class no longer prevents calling the __set_name__ method of other descriptors.
- bpo-28782 [https://bugs.python.org/issue?@action=redirect&bpo=28782]: Fix a bug in the implementation `yield from` when checking if the next instruction is YIELD_FROM. Regression introduced by WORDCODE (bpo-26647 [https://bugs.python.org/issue?@action=redirect&bpo=26647]).

Biblioteca

- bpo-27030 [https://bugs.python.org/issue?@action=redirect&bpo=27030]: Unknown escapes in re.sub() replacement template are allowed again. But they still are deprecated and will be disabled in 3.7.
- bpo-28835 [https://bugs.python.org/issue?@action=redirect&bpo=28835]: Fix a regression introduced in warnings.catch_warnings(): call warnings.showwarning() if it was overridden inside the context manager.
- bpo-27172 [https://bugs.python.org/issue?@action=redirect&bpo=27172]: To assist with upgrades from 2.7, the previously documented deprecation of

`inspect.getfullargspec()` has been reversed. This decision may be revisited again after the Python 2.7 branch is no longer officially supported.

- bpo-26273 [https://bugs.python.org/issue?@action=redirect&bpo=26273]: Add new **`socket.TCP_CONGESTION`** (Linux 2.6.13) and **`socket.TCP_USER_TIMEOUT`** (Linux 2.6.37) constants. Patch written by Omar Sandoval.
- bpo-24142 [https://bugs.python.org/issue?@action=redirect&bpo=24142]: Reading a corrupt config file left configparser in an invalid state. Original patch by Florian Höch.
- bpo-28843 [https://bugs.python.org/issue?@action=redirect&bpo=28843]: Fix asyncio C Task to handle exceptions __traceback__.

API C

- bpo-28808 [https://bugs.python.org/issue?@action=redirect&bpo=28808]: PyUnicode_CompareWithASCIIString() now never raises exceptions.

Documentação

- bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722]: The data model reference and the porting section in the What's New guide now cover the additional `__classcell__` handling needed for custom metaclasses to fully support **PEP 487** [https://peps.python.org/pep-0487/] and zero-argument `super()`.

Ferramentas/Demos

- bpo-28023 [https://bugs.python.org/issue?@action=redirect&bpo=28023]: Fix python-gdb.py didn't support new dict implementation.

Python 3.6.0 beta 4

Data de lançamento: 2016-11-21

Núcleo e embutidos

- bpo-28532 [https://bugs.python.org/issue?@action=redirect&bpo=28532]: Show sys.version when -V option is supplied twice.
- bpo-27100 [https://bugs.python.org/issue?@action=redirect&bpo=27100]: The with-statement now checks for __enter__ before it checks for __exit__. This gives less confusing error messages when both methods are missing. Patch by Jonathan Ellington.
- bpo-28746 [https://bugs.python.org/issue?@action=redirect&bpo=28746]: Fix the set_inheritable() file descriptor method on platforms that do not have the ioctl FIOCLEX and FIONCLEX commands.
- bpo-26920 [https://bugs.python.org/issue?@action=redirect&bpo=26920]: Fix not getting the locale's charset upon initializing the interpreter, on platforms that do not have langinfo.
- bpo-28648 [https://bugs.python.org/issue?@action=redirect&bpo=28648]: Fixed crash in Py_DecodeLocale() in debug build on Mac OS X when decode astral characters. Patch by Xiang Zhang.
- bpo-19398 [https://bugs.python.org/issue?@action=redirect&bpo=19398]: Extra slash no longer added to sys.path components in case of empty compile-time PYTHONPATH components.
- bpo-28665 [https://bugs.python.org/issue?@action=redirect&bpo=28665]: Improve speed of the STORE_DEREF opcode by 40%.
- bpo-28583 [https://bugs.python.org/issue?@action=redirect&bpo=28583]: PyDict_SetDefault didn't combine split table when needed. Patch by Xiang Zhang.
- bpo-27243 [https://bugs.python.org/issue?@action=redirect&bpo=27243]: Change PendingDeprecationWarning -> DeprecationWarning. As it was agreed in the issue, __aiter__ returning an awaitable should result in PendingDeprecationWarning in 3.5 and in DeprecationWarning in 3.6.
- bpo-26182 [https://bugs.python.org/issue?@action=redirect&bpo=26182]: Fix a refleak in code that raises DeprecationWarning.
- bpo-28721 [https://bugs.python.org/issue?@action=redirect&bpo=28721]: Fix asynchronous generators aclose() and athrow() to handle StopAsyncIteration propagation properly.

Biblioteca

- bpo-28752 [https://bugs.python.org/issue?@action=redirect&bpo=28752]: Restored the __reduce__() methods of datetime objects.

- bpo-28727 [https://bugs.python.org/issue?@action=redirect&bpo=28727]: Regular expression patterns, _sre.SRE_Pattern objects created by re.compile(), become comparable (only x==y and x!=y operators). This change should fix the bpo-18383 [https://bugs.python.org/issue?@action=redirect&bpo=18383]: don't duplicate warning filters when the warnings module is reloaded (thing usually only done in unit tests).
- bpo-20572 [https://bugs.python.org/issue?@action=redirect&bpo=20572]: The subprocess.Popen.wait method's undocumented endtime parameter now raises a DeprecationWarning.
- bpo-25659 [https://bugs.python.org/issue?@action=redirect&bpo=25659]: In ctypes, prevent a crash calling the from_buffer() and from_buffer_copy() methods on abstract classes like Array.
- bpo-19717 [https://bugs.python.org/issue?@action=redirect&bpo=19717]: Makes Path.resolve() succeed on paths that do not exist. Patch by Vajrasky Kok
- bpo-28563 [https://bugs.python.org/issue?@action=redirect&bpo=28563]: Fixed possible DoS and arbitrary code execution when handle plural form selections in the gettext module. The expression parser now supports exact syntax supported by GNU gettext.
- bpo-28387 [https://bugs.python.org/issue?@action=redirect&bpo=28387]: Fixed possible crash in _io.TextIOWrapper deallocator when the garbage collector is invoked in other thread. Based on patch by Sebastian Cufre.
- bpo-28600 [https://bugs.python.org/issue?@action=redirect&bpo=28600]: Optimize loop.call_soon.
- bpo-28613 [https://bugs.python.org/issue?@action=redirect&bpo=28613]: Fix get_event_loop() return the current loop if called from coroutines/callbacks.
- bpo-28634 [https://bugs.python.org/issue?@action=redirect&bpo=28634]: Fix asyncio.isfuture() to support unittest.Mock.
- bpo-26081 [https://bugs.python.org/issue?@action=redirect&bpo=26081]: Fix refleak in _asyncio.Future.__iter__().throw.
- bpo-28639 [https://bugs.python.org/issue?@action=redirect&bpo=28639]: Fix inspect.isawaitable to always return bool Patch by Justin Mayfield.
- bpo-28652 [https://bugs.python.org/issue?@action=redirect&bpo=28652]: Make loop methods reject socket kinds they do not support.

- bpo-28653 [https://bugs.python.org/issue?@action=redirect&bpo=28653]: Fix a refleak in functools.lru_cache.
- bpo-28703 [https://bugs.python.org/issue?@action=redirect&bpo=28703]: Fix asyncio.iscoroutinefunction to handle Mock objects.
- bpo-28704 [https://bugs.python.org/issue?@action=redirect&bpo=28704]: Fix create_unix_server to support Path-like objects (PEP 519).
- bpo-28720 [https://bugs.python.org/issue?@action=redirect&bpo=28720]: Add collections.abc.AsyncGenerator.

Documentação

- bpo-28513 [https://bugs.python.org/issue?@action=redirect&bpo=28513]: Documented command-line interface of zipfile.

Testes

- bpo-28666 [https://bugs.python.org/issue?@action=redirect&bpo=28666]: Now test.support.rmtree is able to remove unwritable or unreadable directories.
- bpo-23839 [https://bugs.python.org/issue?@action=redirect&bpo=23839]: Various caches now are cleared before running every test file.

Construção

- bpo-10656 [https://bugs.python.org/issue?@action=redirect&bpo=10656]: Fix out-of-tree building on AIX. Patch by Tristan Carel and Michael Haubenwallner.
- bpo-26359 [https://bugs.python.org/issue?@action=redirect&bpo=26359]: Rename –with-optimiations to –enable-optimizations.
- bpo-28676 [https://bugs.python.org/issue?@action=redirect&bpo=28676]: Prevent missing ‘getentropy’ declaration warning on macOS. Patch by Gareth Rees.

Python 3.6.0 beta 3

Data de lançamento: 2016-10-31

Núcleo e embutidos

- bpo-28128 [https://bugs.python.org/issue?@action=redirect&bpo=28128]: Deprecation warning for invalid str and byte escape sequences now prints better information about where the error occurs. Patch by Serhiy Storchaka and Eric Smith.
- bpo-28509 [https://bugs.python.org/issue?@action=redirect&bpo=28509]: dict.update() no longer allocate unnecessary large memory.
- bpo-28426 [https://bugs.python.org/issue?@action=redirect&bpo=28426]: Fixed potential crash in PyUnicode_AsDecodedObject() in debug build.
- bpo-28517 [https://bugs.python.org/issue?@action=redirect&bpo=28517]: Fixed of-by-one error in the peephole optimizer that caused keeping unreachable code.
- bpo-28214 [https://bugs.python.org/issue?@action=redirect&bpo=28214]: Improved exception reporting for problematic __set_name__ attributes.
- bpo-23782 [https://bugs.python.org/issue?@action=redirect&bpo=23782]: Fixed possible memory leak in _PyTraceback_Add() and exception loss in PyTraceBack_Here().
- bpo-28471 [https://bugs.python.org/issue?@action=redirect&bpo=28471]: Fix “Python memory allocator called without holding the GIL” crash in socket.setblocking.

Biblioteca

- bpo-27517 [https://bugs.python.org/issue?@action=redirect&bpo=27517]: LZMA compressor and decompressor no longer raise exceptions if given empty data twice. Patch by Benjamin Fogle.
- bpo-28549 [https://bugs.python.org/issue?@action=redirect&bpo=28549]: Fixed segfault in curses’s addch() with ncurses6.
- bpo-28449 [https://bugs.python.org/issue?@action=redirect&bpo=28449]: tarfile.open() with mode “r” or “r:” now tries to open a tar file with compression before trying to open it without compression. Otherwise it had 50% chance failed with ignore_zeros=True.
- bpo-23262 [https://bugs.python.org/issue?@action=redirect&bpo=23262]: The webbrowser module now supports Firefox 36+ and derived browsers. Based on patch by Oleg Broytman.

- bpo-27939 [https://bugs.python.org/issue?@action=redirect&bpo=27939]: Fixed bugs in tkinter.ttk.LabeledScale and tkinter.Scale caused by representing the scale as float value internally in Tk. tkinter.IntVar now works if float value is set to underlying Tk variable.
- bpo-18844 [https://bugs.python.org/issue?@action=redirect&bpo=18844]: The various ways of specifying weights for random.choices() now produce the same result sequences.
- bpo-28255 [https://bugs.python.org/issue?@action=redirect&bpo=28255]: calendar.TextCalendar().prmonth() no longer prints a space at the start of new line after printing a month's calendar. Patch by Xiang Zhang.
- bpo-20491 [https://bugs.python.org/issue?@action=redirect&bpo=20491]: The textwrap.TextWrapper class now honors non-breaking spaces. Based on patch by Kaarle Ritvanen.
- bpo-28353 [https://bugs.python.org/issue?@action=redirect&bpo=28353]: os.fwalk() no longer fails on broken links.
- bpo-28430 [https://bugs.python.org/issue?@action=redirect&bpo=28430]: Fix iterator of C implemented asyncio.Future doesn't accept non-None value is passed to it.send(val).
- bpo-27025 [https://bugs.python.org/issue?@action=redirect&bpo=27025]: Generated names for Tkinter widgets now start by the "!" prefix for readability.
- bpo-25464 [https://bugs.python.org/issue?@action=redirect&bpo=25464]: Fixed HList.header_exists() in tkinter.tix module by addin a workaround to Tix library bug.
- bpo-28488 [https://bugs.python.org/issue?@action=redirect&bpo=28488]: shutil.make_archive() no longer adds entry "./" to ZIP archive.
- bpo-25953 [https://bugs.python.org/issue?@action=redirect&bpo=25953]: re.sub() now raises an error for invalid numerical group reference in replacement template even if the pattern is not found in the string. Error message for invalid group reference now includes the group index and the position of the reference. Based on patch by SilentGhost.
- bpo-18219 [https://bugs.python.org/issue?@action=redirect&bpo=18219]: Optimize csv.DictWriter for large number of columns. Patch by Mariatta Wijaya.
- bpo-28448 [https://bugs.python.org/issue?@action=redirect&bpo=28448]: Fix C implemented asyncio.Future didn't work on Windows.
- bpo-28480 [https://bugs.python.org/issue?@action=redirect&bpo=28480]: Fix error building socket module when multithreading is disabled.

- bpo-24452 [https://bugs.python.org/issue?@action=redirect&bpo=24452]: Make webbrowser support Chrome on Mac OS X.
- bpo-20766 [https://bugs.python.org/issue?@action=redirect&bpo=20766]: Fix references leaked by pdb in the handling of SIGINT handlers.
- bpo-28492 [https://bugs.python.org/issue?@action=redirect&bpo=28492]: Fix how StopIteration exception is raised in _asyncio.Future.
- bpo-28500 [https://bugs.python.org/issue?@action=redirect&bpo=28500]: Fix asyncio to handle async gens GC from another thread.
- bpo-26923 [https://bugs.python.org/issue?@action=redirect&bpo=26923]: Fix asyncio.Gather to refuse being cancelled once all children are done. Patch by Johannes Ebke.
- bpo-26796 [https://bugs.python.org/issue?@action=redirect&bpo=26796]: Don't configure the number of workers for default threadpool executor. Initial patch by Hans Lawrenz.
- bpo-28544 [https://bugs.python.org/issue?@action=redirect&bpo=28544]: Implement asyncio.Task in C.

Windows

- bpo-28522 [https://bugs.python.org/issue?@action=redirect&bpo=28522]: Fixes mishandled buffer reallocation in getpathp.c

Construção

- bpo-28444 [https://bugs.python.org/issue?@action=redirect&bpo=28444]: Fix missing extensions modules when cross compiling.
- bpo-28208 [https://bugs.python.org/issue?@action=redirect&bpo=28208]: Update Windows build and OS X installers to use SQLite 3.14.2.
- bpo-28248 [https://bugs.python.org/issue?@action=redirect&bpo=28248]: Update Windows build and OS X installers to use OpenSSL 1.0.2j.

Testes

- bpo-26944 [https://bugs.python.org/issue?@action=redirect&bpo=26944]: Fix test_posix for Android where 'id -G' is entirely wrong or missing the effective gid.

- bpo-28409 [https://bugs.python.org/issue?@action=redirect&bpo=28409]: regrtest: fix the parser of command line arguments.

Python 3.6.0 beta 2

Data de lançamento: 2016-10-10

Núcleo e embutidos

- bpo-28183 [https://bugs.python.org/issue?@action=redirect&bpo=28183]: Optimize and cleanup dict iteration.
- bpo-26081 [https://bugs.python.org/issue?@action=redirect&bpo=26081]: Added C implementation of asyncio.Future. Original patch by Yury Selivanov.
- bpo-28379 [https://bugs.python.org/issue?@action=redirect&bpo=28379]: Added sanity checks and tests for PyUnicode_CopyCharacters(). Patch by Xiang Zhang.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: The type of long range iterator is now registered as Iterator. Patch by Oren Milman.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: Creating instances of range_iterator by calling range_iterator type now is deprecated. Patch by Oren Milman.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: The constructor of range_iterator now checks that step is not 0. Patch by Oren Milman.
- bpo-26906 [https://bugs.python.org/issue?@action=redirect&bpo=26906]: Resolving special methods of uninitialized type now causes implicit initialization of the type instead of a fail.
- bpo-18287 [https://bugs.python.org/issue?@action=redirect&bpo=18287]: PyType_Ready() now checks that tp_name is not NULL. Original patch by Niklas Koep.
- bpo-24098 [https://bugs.python.org/issue?@action=redirect&bpo=24098]: Fixed possible crash when AST is changed in process of compiling it.
- bpo-28201 [https://bugs.python.org/issue?@action=redirect&bpo=28201]: Dict reduces possibility of 2nd conflict in hash table when hashes have same lower bits.

- bpo-28350 [https://bugs.python.org/issue?@action=redirect&bpo=28350]: String constants with null character no longer interned.
- bpo-26617 [https://bugs.python.org/issue?@action=redirect&bpo=26617]: Fix crash when GC runs during weakref callbacks.
- bpo-27942 [https://bugs.python.org/issue?@action=redirect&bpo=27942]: String constants now interned recursively in tuples and frozensets.
- bpo-21578 [https://bugs.python.org/issue?@action=redirect&bpo=21578]: Fixed misleading error message when ImportError called with invalid keyword args.
- bpo-28203 [https://bugs.python.org/issue?@action=redirect&bpo=28203]: Fix incorrect type in complex(1.0, {2:3}) error message. Patch by Soumya Sharma.
- bpo-28086 [https://bugs.python.org/issue?@action=redirect&bpo=28086]: Single var-positional argument of tuple subtype was passed unscathed to the C-defined function. Now it is converted to exact tuple.
- bpo-28214 [https://bugs.python.org/issue?@action=redirect&bpo=28214]: Now __set_name__ is looked up on the class instead of the instance.
- bpo-27955 [https://bugs.python.org/issue?@action=redirect&bpo=27955]: Fallback on reading /dev/urandom device when the getrandom() syscall fails with EPERM, for example when blocked by SECCOMP.
- bpo-28192 [https://bugs.python.org/issue?@action=redirect&bpo=28192]: Don’t import readline in isolated mode.
- Upgrade internal unicode databases to Unicode version 9.0.0.
- bpo-28131 [https://bugs.python.org/issue?@action=redirect&bpo=28131]: Fix a regression in zipimport’s compile_source(). zipimport should use the same optimization level as the interpreter.
- bpo-28126 [https://bugs.python.org/issue?@action=redirect&bpo=28126]: Replace Py_MEMCPY with memcpy(). Visual Studio can properly optimize memcpy().
- bpo-28120 [https://bugs.python.org/issue?@action=redirect&bpo=28120]: Fix dict.pop() for split dictionary when trying to remove a “pending key” (Not yet inserted in split-table). Patch by Xiang Zhang.
- bpo-26182 [https://bugs.python.org/issue?@action=redirect&bpo=26182]: Raise DeprecationWarning when async and await keywords are used as variable/attribute/class/function name.

Biblioteca

- bpo-27998 [https://bugs.python.org/issue?@action=redirect&bpo=27998]: Fixed bytes path support in os.scandir() on Windows. Patch by Eryk Sun.
- bpo-28317 [https://bugs.python.org/issue?@action=redirect&bpo=28317]: The disassembler now decodes FORMAT_VALUE argument.
- bpo-26293 [https://bugs.python.org/issue?@action=redirect&bpo=26293]: Fixed writing ZIP files that starts not from the start of the file. Offsets in ZIP file now are relative to the start of the archive in conforming to the specification.
- bpo-28380 [https://bugs.python.org/issue?@action=redirect&bpo=28380]: unittest.mock Mock autospec functions now properly support assert_called, assert_not_called, and assert_called_once.
- bpo-27181 [https://bugs.python.org/issue?@action=redirect&bpo=27181]: remove statistics.geometric_mean and defer until 3.7.
- bpo-28229 [https://bugs.python.org/issue?@action=redirect&bpo=28229]: lzma module now supports pathlib.
- bpo-28321 [https://bugs.python.org/issue?@action=redirect&bpo=28321]: Fixed writing non-BMP characters with binary format in plistlib.
- bpo-28225 [https://bugs.python.org/issue?@action=redirect&bpo=28225]: bz2 module now supports pathlib. Initial patch by Ethan Furman.
- bpo-28227 [https://bugs.python.org/issue?@action=redirect&bpo=28227]: gzip now supports pathlib. Patch by Ethan Furman.
- bpo-27358 [https://bugs.python.org/issue?@action=redirect&bpo=27358]: Optimized merging var-keyword arguments and improved error message when passing a non-mapping as a var-keyword argument.
- bpo-28257 [https://bugs.python.org/issue?@action=redirect&bpo=28257]: Improved error message when passing a non-iterable as a var-positional argument. Added opcode BUILD_TUPLE_UNPACK_WITH_CALL.
- bpo-28322 [https://bugs.python.org/issue?@action=redirect&bpo=28322]: Fixed possible crashes when unpickle itertools objects from incorrect pickle data. Based on patch by John Leitch.
- bpo-28228 [https://bugs.python.org/issue?@action=redirect&bpo=28228]: imghdr now supports pathlib.
- bpo-28226 [https://bugs.python.org/issue?@action=redirect&bpo=28226]: compileall now supports pathlib.
- bpo-28314 [https://bugs.python.org/issue?@action=redirect&bpo=28314]: Fix function declaration (C flags) for the getiterator() method of xml.etree.ElementTree.Element.

- bpo-28148 [https://bugs.python.org/issue?@action=redirect&bpo=28148]: Stop using localtime() and gmtime() in the time module. Introduced platform independent _PyTime_localtime API that is similar to POSIX localtime_r, but available on all platforms. Patch by Ed Schouten.
- bpo-28253 [https://bugs.python.org/issue?@action=redirect&bpo=28253]: Fixed calendar functions for extreme months: 0001-01 and 9999-12. Methods itermonthdays() and itermonthdays2() are reimplemented so that they don't call itermonthdates() which can cause datetime.date under/overflow.
- bpo-28275 [https://bugs.python.org/issue?@action=redirect&bpo=28275]: Fixed possible use after free in the decompress() methods of the LZMADecompressor and BZ2Decompressor classes. Original patch by John Leitch.
- bpo-27897 [https://bugs.python.org/issue?@action=redirect&bpo=27897]: Fixed possible crash in sqlite3.Connection.create_collation() if pass invalid string-like object as a name. Patch by Xiang Zhang.
- bpo-18844 [https://bugs.python.org/issue?@action=redirect&bpo=18844]: random.choices() now has k as a keyword-only argument to improve the readability of common cases and come into line with the signature used in other languages.
- bpo-18893 [https://bugs.python.org/issue?@action=redirect&bpo=18893]: Fix invalid exception handling in Lib/ctypes/macholib/dyld.py. Patch by Madison May.
- bpo-27611 [https://bugs.python.org/issue?@action=redirect&bpo=27611]: Fixed support of default root window in the tkinter.tix module. Added the master parameter in the DisplayStyle constructor.
- bpo-27348 [https://bugs.python.org/issue?@action=redirect&bpo=27348]: In the traceback module, restore the formatting of exception messages like "Exception: None". This fixes a regression introduced in 3.5a2.
- bpo-25651 [https://bugs.python.org/issue?@action=redirect&bpo=25651]: Allow false values to be used for msg parameter of subTest().
- bpo-27778 [https://bugs.python.org/issue?@action=redirect&bpo=27778]: Fix a memory leak in os.getrandom() when the getrandom() is interrupted by a signal and a signal handler raises a Python exception.
- bpo-28200 [https://bugs.python.org/issue?@action=redirect&bpo=28200]: Fix memory leak on Windows in the os module (fix path_converter() function).

- bpo-25400 [https://bugs.python.org/issue?@action=redirect&bpo=25400]: RobotFileParser now correctly returns default values for crawl_delay and request_rate. Initial patch by Peter Wirtz.
- bpo-27932 [https://bugs.python.org/issue?@action=redirect&bpo=27932]: Prevent memory leak in win32_ver().
- Fix UnboundLocalError in socket._sendfile_use_sendfile.
- bpo-28075 [https://bugs.python.org/issue?@action=redirect&bpo=28075]: Check for ERROR_ACCESS_DENIED in Windows implementation of os.stat(). Patch by Eryk Sun.
- bpo-22493 [https://bugs.python.org/issue?@action=redirect&bpo=22493]: Warning message emitted by using inline flags in the middle of regular expression now contains a (truncated) regex pattern. Patch by Tim Graham.
- bpo-25270 [https://bugs.python.org/issue?@action=redirect&bpo=25270]: Prevent codecs.escape_encode() from raising SystemError when an empty bytestring is passed.
- bpo-28181 [https://bugs.python.org/issue?@action=redirect&bpo=28181]: Get antigravity over HTTPS. Patch by Kaartic Sivaraam.
- bpo-25895 [https://bugs.python.org/issue?@action=redirect&bpo=25895]: Enable WebSocket URL schemes in urllib.parse.urljoin. Patch by Gergely Imreh and Markus Holtermann.
- bpo-28114 [https://bugs.python.org/issue?@action=redirect&bpo=28114]: Fix a crash in parse_envlist() when env contains byte strings. Patch by Eryk Sun.
- bpo-27599 [https://bugs.python.org/issue?@action=redirect&bpo=27599]: Fixed buffer overrun in binascii.b2a_qp() and binascii.a2b_qp().
- bpo-27906 [https://bugs.python.org/issue?@action=redirect&bpo=27906]: Fix socket accept exhaustion during high TCP traffic. Patch by Kevin Conway.
- bpo-28174 [https://bugs.python.org/issue?@action=redirect&bpo=28174]: Handle when SO_REUSEPORT isn’t properly supported. Patch by Seth Michael Larson.
- bpo-26654 [https://bugs.python.org/issue?@action=redirect&bpo=26654]: Inspect functools.partial in asyncio.Handle.__repr__. Patch by iceboy.
- bpo-26909 [https://bugs.python.org/issue?@action=redirect&bpo=26909]: Fix slow pipes IO in asyncio. Patch by INADA Naoki.
- bpo-28176 [https://bugs.python.org/issue?@action=redirect&bpo=28176]: Fix callbacks race in asyncio.SelectorLoop.sock_connect.

- bpo-27759 [https://bugs.python.org/issue?@action=redirect&bpo=27759]: Fix selectors incorrectly retain invalid file descriptors. Patch by Mark Williams.
- bpo-28368 [https://bugs.python.org/issue?@action=redirect&bpo=28368]: Refuse monitoring processes if the child watcher has no loop attached. Patch by Vincent Michel.
- bpo-28369 [https://bugs.python.org/issue?@action=redirect&bpo=28369]: Raise RuntimeError when transport's FD is used with add_reader, add_writer, etc.
- bpo-28370 [https://bugs.python.org/issue?@action=redirect&bpo=28370]: Speedup asyncio.StreamReader.readexactly. Patch by Коренберг Марк.
- bpo-28371 [https://bugs.python.org/issue?@action=redirect&bpo=28371]: Deprecate passing asyncio.Handles to run_in_executor.
- bpo-28372 [https://bugs.python.org/issue?@action=redirect&bpo=28372]: Fix asyncio to support formatting of non-python coroutines.
- bpo-28399 [https://bugs.python.org/issue?@action=redirect&bpo=28399]: Remove UNIX socket from FS before binding. Patch by Коренберг Марк.
- bpo-27972 [https://bugs.python.org/issue?@action=redirect&bpo=27972]: Prohibit Tasks to await on themselves.

Windows

- bpo-28402 [https://bugs.python.org/issue?@action=redirect&bpo=28402]: Adds signed catalog files for stdlib on Windows.
- bpo-28333 [https://bugs.python.org/issue?@action=redirect&bpo=28333]: Enables Unicode for ps1/ps2 and input() prompts. (Patch by Eryk Sun)
- bpo-28251 [https://bugs.python.org/issue?@action=redirect&bpo=28251]: Improvements to help manuals on Windows.
- bpo-28110 [https://bugs.python.org/issue?@action=redirect&bpo=28110]: launcher.msi has different product codes between 32-bit and 64-bit
- bpo-28161 [https://bugs.python.org/issue?@action=redirect&bpo=28161]: Opening CON for write access fails
- bpo-28162 [https://bugs.python.org/issue?@action=redirect&bpo=28162]: WindowsConsoleIO readall() fails if first line starts with Ctrl+Z
- bpo-28163 [https://bugs.python.org/issue?@action=redirect&bpo=28163]: WindowsConsoleIO fileno() passes wrong flags to _open_osfhandle

- bpo-28164 [https://bugs.python.org/issue?@action=redirect&bpo=28164]: _PyIO_get_console_type fails for various paths
- bpo-28137 [https://bugs.python.org/issue?@action=redirect&bpo=28137]: Renames Windows path file to ._pth
- bpo-28138 [https://bugs.python.org/issue?@action=redirect&bpo=28138]: Windows ._pth file should allow import site

API C

- bpo-28426 [https://bugs.python.org/issue?@action=redirect&bpo=28426]: Deprecated undocumented functions PyUnicode_AsEncodedObject(), PyUnicode_AsDecodedObject(), PyUnicode_AsDecodedUnicode() and PyUnicode_AsEncodedUnicode().

Construção

- bpo-28258 [https://bugs.python.org/issue?@action=redirect&bpo=28258]: Fixed build with Estonian locale (python-config and distclean targets in Makefile). Patch by Arfrever Frehtes Taifersar Arahesis.
- bpo-26661 [https://bugs.python.org/issue?@action=redirect&bpo=26661]: setup.py now detects system libffi with multiarch wrapper.
- bpo-15819 [https://bugs.python.org/issue?@action=redirect&bpo=15819]: Remove redundant include search directory option for building outside the source tree.

Testes

- bpo-28217 [https://bugs.python.org/issue?@action=redirect&bpo=28217]: Adds _testconsole module to test console input.

Python 3.6.0 beta 1

Data de lançamento: 2016-09-12

Núcleo e embutidos

- bpo-23722 [https://bugs.python.org/issue?@action=redirect&bpo=23722]: The __class__ cell used by zero-argument super() is now initialized from type.__new__ rather than __build_class__, so class methods relying on that will now work correctly when called from metaclass methods during class creation. Patch by Martin Teichmann.
- bpo-25221 [https://bugs.python.org/issue?@action=redirect&bpo=25221]: Fix corrupted result from PyLong_FromLong(0) when Python is compiled with NSMALLPOSINTS = 0.
- bpo-27080 [https://bugs.python.org/issue?@action=redirect&bpo=27080]: Implement formatting support for **PEP 515** [https://peps.python.org/pep-0515/]. Initial patch by Chris Angelico.
- bpo-27199 [https://bugs.python.org/issue?@action=redirect&bpo=27199]: In tarfile, expose copyfileobj bufsize to improve throughput. Patch by Jason Fried.
- bpo-27948 [https://bugs.python.org/issue?@action=redirect&bpo=27948]: In f-strings, only allow backslashes inside the braces (where the expressions are). This is a breaking change from the 3.6 alpha releases, where backslashes are allowed anywhere in an f-string. Also, require that expressions inside f-strings be enclosed within literal braces, and not escapes like `f'\x7b"hi"\x7d'`.
- bpo-28046 [https://bugs.python.org/issue?@action=redirect&bpo=28046]: Remove platform-specific directories from sys.path.
- bpo-28071 [https://bugs.python.org/issue?@action=redirect&bpo=28071]: Add early-out for differencing from an empty set.
- bpo-25758 [https://bugs.python.org/issue?@action=redirect&bpo=25758]: Prevents zipimport from unnecessarily encoding a filename (patch by Eryk Sun)
- bpo-25856 [https://bugs.python.org/issue?@action=redirect&bpo=25856]: The __module__ attribute of extension classes and functions now is interned. This leads to more compact pickle data with protocol 4.
- bpo-27213 [https://bugs.python.org/issue?@action=redirect&bpo=27213]: Rework CALL_FUNCTION* opcodes to produce shorter and more efficient bytecode. Patch by Demur Rumed, design by Serhiy Storchaka, reviewed by Serhiy Storchaka and Victor Stinner.
- bpo-26331 [https://bugs.python.org/issue?@action=redirect&bpo=26331]: Implement tokenizing support for **PEP 515** [https://peps.python.org/pep-0515/]. Patch by Georg Brandl.

- bpo-27999 [https://bugs.python.org/issue?@action=redirect&bpo=27999]: Make “global after use” a SyntaxError, and ditto for nonlocal. Patch by Ivan Levkivskyi.
- bpo-28003 [https://bugs.python.org/issue?@action=redirect&bpo=28003]: Implement **PEP 525** [https://peps.python.org/pep-0525/] – Asynchronous Generators.
- bpo-27985 [https://bugs.python.org/issue?@action=redirect&bpo=27985]: Implement **PEP 526** [https://peps.python.org/pep-0526/] – Syntax for Variable Annotations. Patch by Ivan Levkivskyi.
- bpo-26058 [https://bugs.python.org/issue?@action=redirect&bpo=26058]: Add a new private version to the builtin dict type, incremented at each dictionary creation and at each dictionary change. Implementation of the PEP 509.
- bpo-27364 [https://bugs.python.org/issue?@action=redirect&bpo=27364]: A backslash-character pair that is not a valid escape sequence now generates a DeprecationWarning. Patch by Emanuel Barry.
- bpo-27350 [https://bugs.python.org/issue?@action=redirect&bpo=27350]: `dict` implementation is changed like PyPy. It is more compact and preserves insertion order. (Concept developed by Raymond Hettinger and patch by Inada Naoki.)
- bpo-27911 [https://bugs.python.org/issue?@action=redirect&bpo=27911]: Remove unnecessary error checks in `exec_builtin_or_dynamic()`.
- bpo-27078 [https://bugs.python.org/issue?@action=redirect&bpo=27078]: Added BUILD_STRING opcode. Optimized f-strings evaluation.
- bpo-17884 [https://bugs.python.org/issue?@action=redirect&bpo=17884]: Python now requires systems with inttypes.h and stdint.h
- bpo-27961 [https://bugs.python.org/issue?@action=redirect&bpo=27961]: Require platforms to support `long long`. Python hasn’t compiled without `long long` for years, so this is basically a formality.
- bpo-27355 [https://bugs.python.org/issue?@action=redirect&bpo=27355]: Removed support for Windows CE. It was never finished, and Windows CE is no longer a relevant platform for Python.
- Implement **PEP 523** [https://peps.python.org/pep-0523/].
- bpo-27870 [https://bugs.python.org/issue?@action=redirect&bpo=27870]: A left shift of zero by a large integer no longer attempts to allocate large amounts of memory.
- bpo-25402 [https://bugs.python.org/issue?@action=redirect&bpo=25402]: In int-to-decimal-string conversion, improve the estimate of the intermediate

memory required, and remove an unnecessarily strict overflow check. Patch by Serhiy Storchaka.

- bpo-27214 [https://bugs.python.org/issue?@action=redirect&bpo=27214]: In long_invert, be more careful about modifying object returned by long_add, and remove an unnecessary check for small longs. Thanks Oren Milman for analysis and patch.
- bpo-27506 [https://bugs.python.org/issue?@action=redirect&bpo=27506]: Support passing the bytes/bytearray.translate() “delete” argument by keyword.
- bpo-27812 [https://bugs.python.org/issue?@action=redirect&bpo=27812]: Properly clear out a generator’s frame’s backreference to the generator to prevent crashes in frame.clear().
- bpo-27811 [https://bugs.python.org/issue?@action=redirect&bpo=27811]: Fix a crash when a coroutine that has not been awaited is finalized with warnings-as-errors enabled.
- bpo-27587 [https://bugs.python.org/issue?@action=redirect&bpo=27587]: Fix another issue found by PVS-Studio: Null pointer check after use of ‘def’ in _PyState_AddModule(). Initial patch by Christian Heimes.
- bpo-27792 [https://bugs.python.org/issue?@action=redirect&bpo=27792]: The modulo operation applied to `bool` and other `int` subclasses now always returns an `int`. Previously the return type depended on the input values. Patch by Xiang Zhang.
- bpo-26984 [https://bugs.python.org/issue?@action=redirect&bpo=26984]: int() now always returns an instance of exact int.
- bpo-25604 [https://bugs.python.org/issue?@action=redirect&bpo=25604]: Fix a minor bug in integer true division; this bug could potentially have caused off-by-one-ulp results on platforms with unreliable ldexp implementations.
- bpo-24254 [https://bugs.python.org/issue?@action=redirect&bpo=24254]: Make class definition namespace ordered by default.
- bpo-27662 [https://bugs.python.org/issue?@action=redirect&bpo=27662]: Fix an overflow check in `List_New`: the original code was checking against `Py_SIZE_MAX` instead of the correct upper bound of `Py_SSIZE_T_MAX`. Patch by Xiang Zhang.
- bpo-27782 [https://bugs.python.org/issue?@action=redirect&bpo=27782]: Multi-phase extension module import now correctly allows the `m_methods` field to be used to add module level functions to instances of non-module types returned from `Py_create_mod`. Patch by Xiang Zhang.

- bpo-27936 [https://bugs.python.org/issue?@action=redirect&bpo=27936]: The round() function accepted a second None argument for some types but not for others. Fixed the inconsistency by accepting None for all numeric types.
- bpo-27487 [https://bugs.python.org/issue?@action=redirect&bpo=27487]: Warn if a submodule argument to “python -m” or runpy.run_module() is found in sys.modules after parent packages are imported, but before the submodule is executed.
- bpo-27157 [https://bugs.python.org/issue?@action=redirect&bpo=27157]: Make only type() itself accept the one-argument form. Patch by Eryk Sun and Emanuel Barry.
- bpo-27558 [https://bugs.python.org/issue?@action=redirect&bpo=27558]: Fix a SystemError in the implementation of “raise” statement. In a brand new thread, raise a RuntimeError since there is no active exception to reraise. Patch written by Xiang Zhang.
- bpo-28008 [https://bugs.python.org/issue?@action=redirect&bpo=28008]: Implement **PEP 530** [https://peps.python.org/pep-0530/] – asynchronous comprehensions.
- bpo-27942 [https://bugs.python.org/issue?@action=redirect&bpo=27942]: Fix memory leak in codeobject.c

Biblioteca

- bpo-28732 [https://bugs.python.org/issue?@action=redirect&bpo=28732]: Fix crash in os.spawnv() with no elements in args
- bpo-28485 [https://bugs.python.org/issue?@action=redirect&bpo=28485]: Always raise ValueError for negative compileall.compile_dir(workers=…) parameter, even when multithreading is unavailable.
- bpo-28037 [https://bugs.python.org/issue?@action=redirect&bpo=28037]: Use sqlite3_get_autocommit() instead of setting Connection->inTransaction manually.
- bpo-25283 [https://bugs.python.org/issue?@action=redirect&bpo=25283]: Attributes tm_gmtoff and tm_zone are now available on all platforms in the return values of time.localtime() and time.gmtime().
- bpo-24454 [https://bugs.python.org/issue?@action=redirect&bpo=24454]: Regular expression match object groups are now accessible using __getitem__. “mo[x]” is equivalent to “mo.group(x)”.

- bpo-10740 [https://bugs.python.org/issue?@action=redirect&bpo=10740]: sqlite3 no longer implicitly commit an open transaction before DDL statements.
- bpo-17941 [https://bugs.python.org/issue?@action=redirect&bpo=17941]: Add a *module* parameter to collections.namedtuple().
- bpo-22493 [https://bugs.python.org/issue?@action=redirect&bpo=22493]: Inline flags now should be used only at the start of the regular expression. Deprecation warning is emitted if uses them in the middle of the regular expression.
- bpo-26885 [https://bugs.python.org/issue?@action=redirect&bpo=26885]: xmlrpc now supports unmarshalling additional data types used by Apache XML-RPC implementation for numerics and None.
- bpo-28070 [https://bugs.python.org/issue?@action=redirect&bpo=28070]: Fixed parsing inline verbose flag in regular expressions.
- bpo-19500 [https://bugs.python.org/issue?@action=redirect&bpo=19500]: Add client-side SSL session resumption to the ssl module.
- bpo-28022 [https://bugs.python.org/issue?@action=redirect&bpo=28022]: Deprecate ssl-related arguments in favor of SSLContext. The deprecation include manual creation of SSLSocket and certfile/keyfile (or similar) in ftplib, httplib, imaplib, smtplib, poplib and urllib.
- bpo-28043 [https://bugs.python.org/issue?@action=redirect&bpo=28043]: SSLContext has improved default settings: OP_NO_SSLv2, OP_NO_SSLv3, OP_NO_COMPRESSION, OP_CIPHER_SERVER_PREFERENCE, OP_SINGLE_DH_USE, OP_SINGLE_ECDH_USE and HIGH ciphers without MD5.
- bpo-24693 [https://bugs.python.org/issue?@action=redirect&bpo=24693]: Changed some RuntimeError's in the zipfile module to more appropriate types. Improved some error messages and debugging output.
- bpo-17909 [https://bugs.python.org/issue?@action=redirect&bpo=17909]: `json.load` and `json.loads` now support binary input encoded as UTF-8, UTF-16 or UTF-32. Patch by Serhiy Storchaka.
- bpo-27137 [https://bugs.python.org/issue?@action=redirect&bpo=27137]: the pure Python fallback implementation of `functools.partial` now matches the behaviour of its accelerated C counterpart for subclassing, pickling and text representation purposes. Patch by Emanuel Barry and Serhiy Storchaka.
- Fix possible integer overflows and crashes in the mmap module with unusual usage patterns.

- bpo-1703178 [https://bugs.python.org/issue?@action=redirect&bpo=1703178]: Fix the ability to pass the –link-objects option to the distutils build_ext command.
- bpo-28019 [https://bugs.python.org/issue?@action=redirect&bpo=28019]: itertools.count() no longer rounds non-integer step in range between 1.0 and 2.0 to 1.
- bpo-18401 [https://bugs.python.org/issue?@action=redirect&bpo=18401]: Pdb now supports the 'readrc' keyword argument to control whether .pdbrc files should be read. Patch by Martin Matusiak and Sam Kimbrel.
- bpo-25969 [https://bugs.python.org/issue?@action=redirect&bpo=25969]: Update the lib2to3 grammar to handle the unpacking generalizations added in 3.5.
- bpo-14977 [https://bugs.python.org/issue?@action=redirect&bpo=14977]: mailcap now respects the order of the lines in the mailcap files ("first match"), as required by RFC 1542. Patch by Michael Lazar.
- bpo-28082 [https://bugs.python.org/issue?@action=redirect&bpo=28082]: Convert re flag constants to IntFlag.
- bpo-28025 [https://bugs.python.org/issue?@action=redirect&bpo=28025]: Convert all ssl module constants to IntEnum and IntFlags. SSLContext properties now return flags and enums.
- bpo-23591 [https://bugs.python.org/issue?@action=redirect&bpo=23591]: Add Flag, IntFlag, and auto() to enum module.
- bpo-433028 [https://bugs.python.org/issue?@action=redirect&bpo=433028]: Added support of modifier spans in regular expressions.
- bpo-24594 [https://bugs.python.org/issue?@action=redirect&bpo=24594]: Validates persist parameter when opening MSI database
- bpo-17582 [https://bugs.python.org/issue?@action=redirect&bpo=17582]: xml.etree.ElementTree nows preserves whitespaces in attributes (Patch by Duane Griffin. Reviewed and approved by Stefan Behnel.)
- bpo-28047 [https://bugs.python.org/issue?@action=redirect&bpo=28047]: Fixed calculation of line length used for the base64 CTE in the new email policies.
- bpo-27576 [https://bugs.python.org/issue?@action=redirect&bpo=27576]: Fix call order in OrderedDict.__init__().
- email.generator.DecodedGenerator now supports the policy keyword.
- bpo-28027 [https://bugs.python.org/issue?@action=redirect&bpo=28027]: Remove undocumented modules from `Lib/plat-*`: IN, CDROM, DLFCN, TYPES, CDIO, and STROPTS.

- bpo-27445 [https://bugs.python.org/issue?@action=redirect&bpo=27445]: Don't pass str(_charset) to MIMEText.set_payload(). Patch by Claude Paroz.
- bpo-24277 [https://bugs.python.org/issue?@action=redirect&bpo=24277]: The new email API is no longer provisional, and the docs have been reorganized and rewritten to emphasize the new API.
- bpo-22450 [https://bugs.python.org/issue?@action=redirect&bpo=22450]: urllib now includes an `Accept: */*` header among the default headers. This makes the results of REST API requests more consistent and predictable especially when proxy servers are involved.
- lib2to3.pgen3.driver.load_grammar() now creates a stable cache file between runs given the same Grammar.txt input regardless of the hash randomization setting.
- bpo-28005 [https://bugs.python.org/issue?@action=redirect&bpo=28005]: Allow ImportErrors in encoding implementation to propagate.
- bpo-26667 [https://bugs.python.org/issue?@action=redirect&bpo=26667]: Support path-like objects in importlib.util.
- bpo-27570 [https://bugs.python.org/issue?@action=redirect&bpo=27570]: Avoid zero-length memcpy() etc calls with null source pointers in the "ctypes" and "array" modules.
- bpo-22233 [https://bugs.python.org/issue?@action=redirect&bpo=22233]: Break email header lines *only* on the RFC specified CR and LF characters, not on arbitrary unicode line breaks. This also fixes a bug in HTTP header parsing.
- bpo-27331 [https://bugs.python.org/issue?@action=redirect&bpo=27331]: The email.mime classes now all accept an optional policy keyword.
- bpo-27988 [https://bugs.python.org/issue?@action=redirect&bpo=27988]: Fix email iter_attachments incorrect mutation of payload list.
- bpo-16113 [https://bugs.python.org/issue?@action=redirect&bpo=16113]: Add SHA-3 and SHAKE support to hashlib module.
- Eliminate a tautological-pointer-compare warning in _scproxy.c.
- bpo-27776 [https://bugs.python.org/issue?@action=redirect&bpo=27776]: The **`os.urandom()`** function does now block on Linux 3.17 and newer until the system urandom entropy pool is initialized to increase the security. This change is part of the **PEP 524** [https://peps.python.org/pep-0524/].
- bpo-27778 [https://bugs.python.org/issue?@action=redirect&bpo=27778]: Expose the Linux `getrandom()` syscall as a new **`os.getrandom()`** function. This change is part of the **PEP 524** [https://peps.python.org/pep-0524/].

- bpo-27691 [https://bugs.python.org/issue?@action=redirect&bpo=27691]: Fix ssl module's parsing of GEN_RID subject alternative name fields in X.509 certs.
- bpo-18844 [https://bugs.python.org/issue?@action=redirect&bpo=18844]: Add random.choices().
- bpo-25761 [https://bugs.python.org/issue?@action=redirect&bpo=25761]: Improved error reporting about truncated pickle data in C implementation of unpickler. UnpicklingError is now raised instead of AttributeError and ValueError in some cases.
- bpo-26798 [https://bugs.python.org/issue?@action=redirect&bpo=26798]: Add BLAKE2 (blake2b and blake2s) to hashlib.
- bpo-26032 [https://bugs.python.org/issue?@action=redirect&bpo=26032]: Optimized globbing in pathlib by using os.scandir(); it is now about 1.5–4 times faster.
- bpo-25596 [https://bugs.python.org/issue?@action=redirect&bpo=25596]: Optimized glob() and iglob() functions in the glob module; they are now about 3–6 times faster.
- bpo-27928 [https://bugs.python.org/issue?@action=redirect&bpo=27928]: Add scrypt (password-based key derivation function) to hashlib module (requires OpenSSL 1.1.0).
- bpo-27850 [https://bugs.python.org/issue?@action=redirect&bpo=27850]: Remove 3DES from ssl module's default cipher list to counter measure sweet32 attack (**CVE 2016-2183** [https://www.cve.org/CVERecord?id=CVE-2016-2183]).
- bpo-27766 [https://bugs.python.org/issue?@action=redirect&bpo=27766]: Add ChaCha20 Poly1305 to ssl module's default cipher list. (Required OpenSSL 1.1.0 or LibreSSL).
- bpo-25387 [https://bugs.python.org/issue?@action=redirect&bpo=25387]: Check return value of winsound.MessageBeep.
- bpo-27866 [https://bugs.python.org/issue?@action=redirect&bpo=27866]: Add SSLContext.get_ciphers() method to get a list of all enabled ciphers.
- bpo-27744 [https://bugs.python.org/issue?@action=redirect&bpo=27744]: Add AF_ALG (Linux Kernel crypto) to socket module.
- bpo-26470 [https://bugs.python.org/issue?@action=redirect&bpo=26470]: Port ssl and hashlib module to OpenSSL 1.1.0.
- bpo-11620 [https://bugs.python.org/issue?@action=redirect&bpo=11620]: Fix support for SND_MEMORY in winsound.PlaySound. Based on a patch by Tim Lesher.

- bpo-11734 [https://bugs.python.org/issue?@action=redirect&bpo=11734]: Add support for IEEE 754 half-precision floats to the struct module. Based on a patch by Eli Stevens.
- bpo-27919 [https://bugs.python.org/issue?@action=redirect&bpo=27919]: Deprecated `extra_path` distribution option in distutils packaging.
- bpo-23229 [https://bugs.python.org/issue?@action=redirect&bpo=23229]: Add new `cmath` constants: `cmath.inf` and `cmath.nan` to match `math.inf` and `math.nan`, and also `cmath.infj` and `cmath.nanj` to match the format used by complex repr.
- bpo-27842 [https://bugs.python.org/issue?@action=redirect&bpo=27842]: The csv.DictReader now returns rows of type OrderedDict. (Contributed by Steve Holden.)
- Remove support for passing a file descriptor to os.access. It never worked but previously didn't raise.
- bpo-12885 [https://bugs.python.org/issue?@action=redirect&bpo=12885]: Fix error when distutils encounters symlink.
- bpo-27881 [https://bugs.python.org/issue?@action=redirect&bpo=27881]: Fixed possible bugs when setting sqlite3.Connection.isolation_level. Based on patch by Xiang Zhang.
- bpo-27861 [https://bugs.python.org/issue?@action=redirect&bpo=27861]: Fixed a crash in sqlite3.Connection.cursor() when a factory creates not a cursor. Patch by Xiang Zhang.
- bpo-19884 [https://bugs.python.org/issue?@action=redirect&bpo=19884]: Avoid spurious output on OS X with Gnu Readline.
- bpo-27706 [https://bugs.python.org/issue?@action=redirect&bpo=27706]: Restore deterministic behavior of random.Random().seed() for string seeds using seeding version 1. Allows sequences of calls to random() to exactly match those obtained in Python 2. Patch by Nofar Schnider.
- bpo-10513 [https://bugs.python.org/issue?@action=redirect&bpo=10513]: Fix a regression in Connection.commit(). Statements should not be reset after a commit.
- bpo-12319 [https://bugs.python.org/issue?@action=redirect&bpo=12319]: Chunked transfer encoding support added to http.client.HTTPConnection requests. The urllib.request.AbstractHTTPHandler class does not enforce a Content-Length header any more. If a HTTP request has a file or iterable body, but no Content-Length header, the library now falls back to use chunked transfer-encoding.

- A new version of typing.py from https://github.com/python/typing: - Collection (only for 3.6) (bpo-27598 [https://bugs.python.org/issue?@action=redirect&bpo=27598]) - Add FrozenSet to __all__ (upstream #261) - fix crash in _get_type_vars() (upstream #259) - Remove the dict constraint in ForwardRef._eval_type (upstream #252)
- bpo-27832 [https://bugs.python.org/issue?@action=redirect&bpo=27832]: Make `_normalize` parameter to `Fraction` constructor keyword-only, so that `Fraction(2, 3, 4)` now raises `TypeError`.
- bpo-27539 [https://bugs.python.org/issue?@action=redirect&bpo=27539]: Fix unnormalised `Fraction.__pow__` result in the case of negative exponent and negative base.
- bpo-21718 [https://bugs.python.org/issue?@action=redirect&bpo=21718]: cursor.description is now available for queries using CTEs.
- bpo-27819 [https://bugs.python.org/issue?@action=redirect&bpo=27819]: In distutils sdists, simply produce the “gztar” (gzipped tar format) distributions on all platforms unless “formats” is supplied.
- bpo-2466 [https://bugs.python.org/issue?@action=redirect&bpo=2466]: posixpath.ismount now correctly recognizes mount points which the user does not have permission to access.
- bpo-9998 [https://bugs.python.org/issue?@action=redirect&bpo=9998]: On Linux, ctypes.util.find_library now looks in LD_LIBRARY_PATH for shared libraries.
- bpo-27573 [https://bugs.python.org/issue?@action=redirect&bpo=27573]: exit message for code.interact is now configurable.
- bpo-27930 [https://bugs.python.org/issue?@action=redirect&bpo=27930]: Improved behaviour of logging.handlers.QueueListener. Thanks to Paulo Andrade and Petr Viktorin for the analysis and patch.
- bpo-6766 [https://bugs.python.org/issue?@action=redirect&bpo=6766]: Distributed reference counting added to multiprocessing to support nesting of shared values / proxy objects.
- bpo-21201 [https://bugs.python.org/issue?@action=redirect&bpo=21201]: Improves readability of multiprocessing error message. Thanks to Wojciech Walczak for patch.
- asyncio: Add set_protocol / get_protocol to Transports.
- bpo-27456 [https://bugs.python.org/issue?@action=redirect&bpo=27456]: asyncio: Set TCP_NODELAY by default.

IDLE

- bpo-15308 [https://bugs.python.org/issue?@action=redirect&bpo=15308]: Add ‘interrupt execution’ (^C) to Shell menu. Patch by Roger Serwy, updated by Bayard Randel.
- bpo-27922 [https://bugs.python.org/issue?@action=redirect&bpo=27922]: Stop IDLE tests from ‘flashing’ gui widgets on the screen.
- bpo-27891 [https://bugs.python.org/issue?@action=redirect&bpo=27891]: Consistently group and sort imports within idlelib modules.
- bpo-17642 [https://bugs.python.org/issue?@action=redirect&bpo=17642]: add larger font sizes for classroom projection.
- Add version to title of IDLE help window.
- bpo-25564 [https://bugs.python.org/issue?@action=redirect&bpo=25564]: In section on IDLE – console differences, mention that using exec means that __builtins__ is defined for each statement.
- bpo-27821 [https://bugs.python.org/issue?@action=redirect&bpo=27821]: Fix 3.6.0a3 regression that prevented custom key sets from being selected when no custom theme was defined.

API C

- bpo-26900 [https://bugs.python.org/issue?@action=redirect&bpo=26900]: Excluded underscored names and other private API from limited API.
- bpo-26027 [https://bugs.python.org/issue?@action=redirect&bpo=26027]: Add support for path-like objects in PyUnicode_FSConverter() & PyUnicode_FSDecoder().

Testes

- bpo-27427 [https://bugs.python.org/issue?@action=redirect&bpo=27427]: Additional tests for the math module. Patch by Francisco Couzo.
- bpo-27953 [https://bugs.python.org/issue?@action=redirect&bpo=27953]: Skip math and cmath tests that fail on OS X 10.4 due to a poor libm implementation of tan.
- bpo-26040 [https://bugs.python.org/issue?@action=redirect&bpo=26040]: Improve test_math and test_cmath coverage and rigour. Patch by Jeff Allen.

- bpo-27787 [https://bugs.python.org/issue?@action=redirect&bpo=27787]: Call gc.collect() before checking each test for “dangling threads”, since the dangling threads are weak references.

Construção

- bpo-27566 [https://bugs.python.org/issue?@action=redirect&bpo=27566]: Fix clean target in freeze makefile (patch by Lisa Roach)
- bpo-27705 [https://bugs.python.org/issue?@action=redirect&bpo=27705]: Update message in validate_ucrtbase.py
- bpo-27976 [https://bugs.python.org/issue?@action=redirect&bpo=27976]: Deprecate building _ctypes with the bundled copy of libffi on non-OSX UNIX platforms.
- bpo-27983 [https://bugs.python.org/issue?@action=redirect&bpo=27983]: Cause lack of llvm-profdata tool when using clang as required for PGO linking to be a configure time error rather than make time when `--with-optimizations` is enabled. Also improve our ability to find the llvm-profdata tool on MacOS and some Linuxes.
- bpo-21590 [https://bugs.python.org/issue?@action=redirect&bpo=21590]: Support for DTrace and SystemTap probes.
- bpo-26307 [https://bugs.python.org/issue?@action=redirect&bpo=26307]: The profile-opt build now applies PGO to the built-in modules.
- bpo-26359 [https://bugs.python.org/issue?@action=redirect&bpo=26359]: Add the –with-optimizations flag to turn on LTO and PGO build support when available.
- bpo-27917 [https://bugs.python.org/issue?@action=redirect&bpo=27917]: Set platform triplets for Android builds.
- bpo-25825 [https://bugs.python.org/issue?@action=redirect&bpo=25825]: Update references to the $(LIBPL) installation path on AIX. This path was changed in 3.2a4.
- Update OS X installer to use SQLite 3.14.1 and XZ 5.2.2.
- bpo-21122 [https://bugs.python.org/issue?@action=redirect&bpo=21122]: Fix LTO builds on OS X.
- bpo-17128 [https://bugs.python.org/issue?@action=redirect&bpo=17128]: Build OS X installer with a private copy of OpenSSL. Also provide a sample Install Certificates command script to install a set of root certificates from the third-party certifi module.

Ferramentas/Demos

- bpo-27952 [https://bugs.python.org/issue?@action=redirect&bpo=27952]: Get Tools/scripts/fixcid.py working with Python 3 and the current “re” module, avoid invalid Python backslash escapes, and fix a bug parsing escaped C quote signs.

Windows

- bpo-28065 [https://bugs.python.org/issue?@action=redirect&bpo=28065]: Update xz dependency to 5.2.2 and build it from source.
- bpo-25144 [https://bugs.python.org/issue?@action=redirect&bpo=25144]: Ensures TargetDir is set before continuing with custom install.
- bpo-1602 [https://bugs.python.org/issue?@action=redirect&bpo=1602]: Windows console doesn’t input or print Unicode (PEP 528)
- bpo-27781 [https://bugs.python.org/issue?@action=redirect&bpo=27781]: Change file system encoding on Windows to UTF-8 (PEP 529)
- bpo-27731 [https://bugs.python.org/issue?@action=redirect&bpo=27731]: Opt-out of MAX_PATH on Windows 10
- bpo-6135 [https://bugs.python.org/issue?@action=redirect&bpo=6135]: Adds encoding and errors parameters to subprocess.
- bpo-27959 [https://bugs.python.org/issue?@action=redirect&bpo=27959]: Adds oem encoding, alias ansi to mbcs, move aliasmbcs to codec lookup.
- bpo-27982 [https://bugs.python.org/issue?@action=redirect&bpo=27982]: The functions of the winsound module now accept keyword arguments.
- bpo-20366 [https://bugs.python.org/issue?@action=redirect&bpo=20366]: Build full text search support into SQLite on Windows.
- bpo-27756 [https://bugs.python.org/issue?@action=redirect&bpo=27756]: Adds new icons for Python files and processes on Windows. Designs by Cherry Wang.
- bpo-27883 [https://bugs.python.org/issue?@action=redirect&bpo=27883]: Update sqlite to 3.14.1.0 on Windows.

Python 3.6.0 alpha 4

Data de lançamento: 2016-08-15

Núcleo e embutidos

- bpo-27704 [https://bugs.python.org/issue?@action=redirect&bpo=27704]: Optimized creating bytes and bytearray from byte-like objects and iterables. Speed up to 3 times for short objects. Original patch by Naoki Inada.
- bpo-26823 [https://bugs.python.org/issue?@action=redirect&bpo=26823]: Large sections of repeated lines in tracebacks are now abbreviated as "[Previous line repeated {count} more times]" by the builtin traceback rendering. Patch by Emanuel Barry.
- bpo-27574 [https://bugs.python.org/issue?@action=redirect&bpo=27574]: Decreased an overhead of parsing keyword arguments in functions implemented with using Argument Clinic.
- bpo-22557 [https://bugs.python.org/issue?@action=redirect&bpo=22557]: Now importing already imported modules is up to 2.5 times faster.
- bpo-17596 [https://bugs.python.org/issue?@action=redirect&bpo=17596]: Include <wincrypt.h> to help with Min GW building.
- bpo-17599 [https://bugs.python.org/issue?@action=redirect&bpo=17599]: On Windows, rename the privately defined REPARSE_DATA_BUFFER structure to avoid conflicting with the definition from Min GW.
- bpo-27507 [https://bugs.python.org/issue?@action=redirect&bpo=27507]: Add integer overflow check in bytearray.extend(). Patch by Xiang Zhang.
- bpo-27581 [https://bugs.python.org/issue?@action=redirect&bpo=27581]: Don't rely on wrapping for overflow check in PySequence_Tuple(). Patch by Xiang Zhang.
- bpo-1621 [https://bugs.python.org/issue?@action=redirect&bpo=1621]: Avoid signed integer overflow in list and tuple operations. Patch by Xiang Zhang.
- bpo-27419 [https://bugs.python.org/issue?@action=redirect&bpo=27419]: Standard __import__() no longer look up "__import__" in globals or builtins for importing submodules or "from import". Fixed a crash if raise a warning about unabling to resolve package from __spec__ or __package__.
- bpo-27083 [https://bugs.python.org/issue?@action=redirect&bpo=27083]: Respect the PYTHONCASEOK environment variable under Windows.
- bpo-27514 [https://bugs.python.org/issue?@action=redirect&bpo=27514]: Make having too many statically nested blocks a SyntaxError instead of SystemError.

- bpo-27366 [https://bugs.python.org/issue?@action=redirect&bpo=27366]: Implemented **PEP 487** [https://peps.python.org/pep-0487/] (Simpler customization of class creation). Upon subclassing, the __init_subclass__ classmethod is called on the base class. Descriptors are initialized with __set_name__ after class creation.

Biblioteca

- bpo-26027 [https://bugs.python.org/issue?@action=redirect&bpo=26027]: Add **PEP 519** [https://peps.python.org/pep-0519/]/__fspath__() support to the os and os.path modules. Includes code from Jelle Zijlstra. (See also: bpo-27524 [https://bugs.python.org/issue?@action=redirect&bpo=27524])
- bpo-27598 [https://bugs.python.org/issue?@action=redirect&bpo=27598]: Add Collections to collections.abc. Patch by Ivan Levkivskyi, docs by Neil Girdhar.
- bpo-25958 [https://bugs.python.org/issue?@action=redirect&bpo=25958]: Support "anti-registration" of special methods from various ABCs, like __hash__, __iter__ or __len__. All these (and several more) can be set to None in an implementation class and the behavior will be as if the method is not defined at all. (Previously, this mechanism existed only for __hash__, to make mutable classes unhashable.) Code contributed by Andrew Barnert and Ivan Levkivskyi.
- bpo-16764 [https://bugs.python.org/issue?@action=redirect&bpo=16764]: Support keyword arguments to zlib.decompress(). Patch by Xiang Zhang.
- bpo-27736 [https://bugs.python.org/issue?@action=redirect&bpo=27736]: Prevent segfault after interpreter re-initialization due to ref count problem introduced in code for bpo-27038 [https://bugs.python.org/issue?@action=redirect&bpo=27038] in 3.6.0a3. Patch by Xiang Zhang.
- bpo-25628 [https://bugs.python.org/issue?@action=redirect&bpo=25628]: The *verbose* and *rename* parameters for collections.namedtuple are now keyword-only.
- bpo-12345 [https://bugs.python.org/issue?@action=redirect&bpo=12345]: Add mathematical constant tau to math and cmath. See also **PEP 628** [https://peps.python.org/pep-0628/].
- bpo-26823 [https://bugs.python.org/issue?@action=redirect&bpo=26823]: traceback.StackSummary.format now abbreviates large sections of repeated lines as "[Previous line repeated {count} more times]" (this

change then further affects other traceback display operations in the module). Patch by Emanuel Barry.

- bpo-27664 [https://bugs.python.org/issue?@action=redirect&bpo=27664]: Add to concurrent.futures.thread.ThreadPoolExecutor() the ability to specify a thread name prefix.
- bpo-27181 [https://bugs.python.org/issue?@action=redirect&bpo=27181]: Add geometric_mean and harmonic_mean to statistics module.
- bpo-27573 [https://bugs.python.org/issue?@action=redirect&bpo=27573]: code.interact now prints an message when exiting.
- bpo-6422 [https://bugs.python.org/issue?@action=redirect&bpo=6422]: Add autorange method to timeit.Timer objects.
- bpo-27773 [https://bugs.python.org/issue?@action=redirect&bpo=27773]: Correct some memory management errors server_hostname in _ssl.wrap_socket().
- bpo-26750 [https://bugs.python.org/issue?@action=redirect&bpo=26750]: unittest.mock.create_autospec() now works properly for subclasses of property() and other data descriptors. Removes the never publicly used, never documented unittest.mock.DescriptorTypes tuple.
- bpo-26754 [https://bugs.python.org/issue?@action=redirect&bpo=26754]: Undocumented support of general bytes-like objects as path in compile() and similar functions is now deprecated.
- bpo-26800 [https://bugs.python.org/issue?@action=redirect&bpo=26800]: Undocumented support of general bytes-like objects as paths in os functions is now deprecated.
- bpo-26981 [https://bugs.python.org/issue?@action=redirect&bpo=26981]: Add _order_ compatibility shim to enum.Enum for Python 2/3 code bases.
- bpo-27661 [https://bugs.python.org/issue?@action=redirect&bpo=27661]: Added tzinfo keyword argument to datetime.combine.
- In the curses module, raise an error if window.getstr() or window.instr() is passed a negative value.
- bpo-27783 [https://bugs.python.org/issue?@action=redirect&bpo=27783]: Fix possible usage of uninitialized memory in operator.methodcaller.
- bpo-27774 [https://bugs.python.org/issue?@action=redirect&bpo=27774]: Fix possible Py_DECREF on unowned object in _sre.
- bpo-27760 [https://bugs.python.org/issue?@action=redirect&bpo=27760]: Fix possible integer overflow in binascii.b2a_qp.

- bpo-27758 [https://bugs.python.org/issue?@action=redirect&bpo=27758]: Fix possible integer overflow in the _csv module for large record lengths.
- bpo-27568 [https://bugs.python.org/issue?@action=redirect&bpo=27568]: Prevent HTTPoxy attack (**CVE 2016-1000110** [https://www.cve.org/CVERecord?id=CVE-2016-1000110]). Ignore the HTTP_PROXY variable when REQUEST_METHOD environment is set, which indicates that the script is in CGI mode.
- bpo-7063 [https://bugs.python.org/issue?@action=redirect&bpo=7063]: Remove dead code from the "array" module's slice handling. Patch by Chuck.
- bpo-27656 [https://bugs.python.org/issue?@action=redirect&bpo=27656]: Do not assume sched.h defines any SCHED_* constants.
- bpo-27130 [https://bugs.python.org/issue?@action=redirect&bpo=27130]: In the "zlib" module, fix handling of large buffers (typically 4 GiB) when compressing and decompressing. Previously, inputs were limited to 4 GiB, and compression and decompression operations did not properly handle results of 4 GiB.
- bpo-24773 [https://bugs.python.org/issue?@action=redirect&bpo=24773]: Implemented **PEP 495** [https://peps.python.org/pep-0495/] (Local Time Disambiguation).
- Expose the EPOLLEXCLUSIVE constant (when it is defined) in the select module.
- bpo-27567 [https://bugs.python.org/issue?@action=redirect&bpo=27567]: Expose the EPOLLRDHUP and POLLRDHUP constants in the select module.
- bpo-1621 [https://bugs.python.org/issue?@action=redirect&bpo=1621]: Avoid signed int negation overflow in the "audioop" module.
- bpo-27533 [https://bugs.python.org/issue?@action=redirect&bpo=27533]: Release GIL in nt._isdir
- bpo-17711 [https://bugs.python.org/issue?@action=redirect&bpo=17711]: Fixed unpickling by the persistent ID with protocol 0. Original patch by Alexandre Vassalotti.
- bpo-27522 [https://bugs.python.org/issue?@action=redirect&bpo=27522]: Avoid an unintentional reference cycle in email.feedparser.
- bpo-27512 [https://bugs.python.org/issue?@action=redirect&bpo=27512]: Fix a segfault when os.fspath() called an __fspath__() method that raised an exception. Patch by Xiang Zhang.

IDLE

- bpo-27714 [https://bugs.python.org/issue?@action=redirect&bpo=27714]: text_textview and test_autocomplete now pass when re-run in the same process. This occurs when test_idle fails when run with the -w option but without -jn. Fix warning from test_config.
- bpo-27621 [https://bugs.python.org/issue?@action=redirect&bpo=27621]: Put query response validation error messages in the query box itself instead of in a separate messagebox. Redo tests to match. Add Mac OSX refinements. Original patch by Mark Roseman.
- bpo-27620 [https://bugs.python.org/issue?@action=redirect&bpo=27620]: Escape key now closes Query box as cancelled.
- bpo-27609 [https://bugs.python.org/issue?@action=redirect&bpo=27609]: IDLE: tab after initial whitespace should tab, not autocomplete. This fixes problem with writing docstrings at least twice indented.
- bpo-27609 [https://bugs.python.org/issue?@action=redirect&bpo=27609]: Explicitly return None when there are also non-None returns. In a few cases, reverse a condition and eliminate a return.
- bpo-25507 [https://bugs.python.org/issue?@action=redirect&bpo=25507]: IDLE no longer runs buggy code because of its tkinter imports. Users must include the same imports required to run directly in Python.
- bpo-27173 [https://bugs.python.org/issue?@action=redirect&bpo=27173]: Add 'IDLE Modern Unix' to the built-in key sets. Make the default key set depend on the platform. Add tests for the changes to the config module.
- bpo-27452 [https://bugs.python.org/issue?@action=redirect&bpo=27452]: add line counter and crc to IDLE configHandler test dump.

Testes

- bpo-25805 [https://bugs.python.org/issue?@action=redirect&bpo=25805]: Skip a test in test_pkgutil as needed that doesn't work when `__name__ == __main__`. Patch by SilentGhost.
- bpo-27472 [https://bugs.python.org/issue?@action=redirect&bpo=27472]: Add test.support.unix_shell as the path to the default shell.
- bpo-27369 [https://bugs.python.org/issue?@action=redirect&bpo=27369]: In test_pyexpat, avoid testing an error message detail that changed in Expat 2.2.0.
- bpo-27594 [https://bugs.python.org/issue?@action=redirect&bpo=27594]: Prevent assertion error when running test_ast with coverage enabled: ensure

code object has a valid first line number. Patch suggested by Ivan Levkivskyi.

Windows

- bpo-27647 [https://bugs.python.org/issue?@action=redirect&bpo=27647]: Update bundled Tcl/Tk to 8.6.6.
- bpo-27610 [https://bugs.python.org/issue?@action=redirect&bpo=27610]: Adds **PEP 514** [https://peps.python.org/pep-0514/] metadata to Windows installer
- bpo-27469 [https://bugs.python.org/issue?@action=redirect&bpo=27469]: Adds a shell extension to the launcher so that drag and drop works correctly.
- bpo-27309 [https://bugs.python.org/issue?@action=redirect&bpo=27309]: Enables proper Windows styles in python[w].exe manifest.

Construção

- bpo-27713 [https://bugs.python.org/issue?@action=redirect&bpo=27713]: Suppress spurious build warnings when updating importlib's bootstrap files. Patch by Xiang Zhang
- bpo-25825 [https://bugs.python.org/issue?@action=redirect&bpo=25825]: Correct the references to Modules/python.exp, which is required on AIX. The references were accidentally changed in 3.5.0a1.
- bpo-27453 [https://bugs.python.org/issue?@action=redirect&bpo=27453]: CPP invocation in configure must use CPPFLAGS. Patch by Chi Hsuan Yen.
- bpo-27641 [https://bugs.python.org/issue?@action=redirect&bpo=27641]: The configure script now inserts comments into the makefile to prevent the pgen and _freeze_importlib executables from being cross-compiled.
- bpo-26662 [https://bugs.python.org/issue?@action=redirect&bpo=26662]: Set PYTHON_FOR_GEN in configure as the Python program to be used for file generation during the build.
- bpo-10910 [https://bugs.python.org/issue?@action=redirect&bpo=10910]: Avoid C++ compilation errors on FreeBSD and OS X. Also update FreedBSD version checks for the original ctype UTF-8 workaround.

Python 3.6.0 alpha 3

Data de lançamento: 2016-07-11

Segurança

- bpo-27278 [https://bugs.python.org/issue?@action=redirect&bpo=27278]: Fix os.urandom() implementation using getrandom() on Linux. Truncate size to INT_MAX and loop until we collected enough random bytes, instead of casting a directly Py_ssize_t to int.
- bpo-22636 [https://bugs.python.org/issue?@action=redirect&bpo=22636]: Avoid shell injection problems with ctypes.util.find_library().

Núcleo e embutidos

- bpo-27473 [https://bugs.python.org/issue?@action=redirect&bpo=27473]: Fixed possible integer overflow in bytes and bytearray concatenations. Patch by Xiang Zhang.
- bpo-23034 [https://bugs.python.org/issue?@action=redirect&bpo=23034]: The output of a special Python build with defined COUNT_ALLOCS, SHOW_ALLOC_COUNT or SHOW_TRACK_COUNT macros is now off by default. It can be re-enabled using the "-X showalloccount" option. It now outputs to stderr instead of stdout.
- bpo-27443 [https://bugs.python.org/issue?@action=redirect&bpo=27443]: __length_hint__() of bytearray iterators no longer return a negative integer for a resized bytearray.
- bpo-27007 [https://bugs.python.org/issue?@action=redirect&bpo=27007]: The fromhex() class methods of bytes and bytearray subclasses now return an instance of corresponding subclass.

Biblioteca

- bpo-26844 [https://bugs.python.org/issue?@action=redirect&bpo=26844]: Fix error message for imp.find_module() to refer to 'path' instead of 'name'. Patch by Lev Maximov.
- bpo-23804 [https://bugs.python.org/issue?@action=redirect&bpo=23804]: Fix SSL zero-length recv() calls to not block and not raise an error about unclean EOF.

- bpo-27466 [https://bugs.python.org/issue?@action=redirect&bpo=27466]: Change time format returned by http.cookie.time2netscape, confirming the netscape cookie format and making it consistent with documentation.
- bpo-21708 [https://bugs.python.org/issue?@action=redirect&bpo=21708]: Deprecated dbm.dumb behavior that differs from common dbm behavior: creating a database in ‘r’ and ‘w’ modes and modifying a database in ‘r’ mode.
- bpo-26721 [https://bugs.python.org/issue?@action=redirect&bpo=26721]: Change the socketserver.StreamRequestHandler.wfile attribute to implement BufferedIOBase. In particular, the write() method no longer does partial writes.
- bpo-22115 [https://bugs.python.org/issue?@action=redirect&bpo=22115]: Added methods trace_add, trace_remove and trace_info in the tkinter.Variable class. They replace old methods trace_variable, trace, trace_vdelete and trace_vinfo that use obsolete Tcl commands and might not work in future versions of Tcl. Fixed old tracing methods: trace_vdelete() with wrong mode no longer break tracing, trace_vinfo() now always returns a list of pairs of strings, tracing in the “u” mode now works.
- bpo-26243 [https://bugs.python.org/issue?@action=redirect&bpo=26243]: Only the level argument to zlib.compress() is keyword argument now. The first argument is positional-only.
- bpo-27038 [https://bugs.python.org/issue?@action=redirect&bpo=27038]: Expose the DirEntry type as os.DirEntry. Code patch by Jelle Zijlstra.
- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Update os.fspath()/PyOS_FSPath() to check the return value of __fspath__() to be either str or bytes.
- bpo-18726 [https://bugs.python.org/issue?@action=redirect&bpo=18726]: All optional parameters of the dump(), dumps(), load() and loads() functions and JSONEncoder and JSONDecoder class constructors in the json module are now keyword-only.
- bpo-27319 [https://bugs.python.org/issue?@action=redirect&bpo=27319]: Methods selection_set(), selection_add(), selection_remove() and selection_toggle() of ttk.TreeView now allow passing multiple items as multiple arguments instead of passing them as a tuple. Deprecated undocumented ability of calling the selection() method with arguments.
- bpo-27079 [https://bugs.python.org/issue?@action=redirect&bpo=27079]: Fixed curses.ascii functions isblank(), iscntrl() and ispunct().

- bpo-27294 [https://bugs.python.org/issue?@action=redirect&bpo=27294]: Numerical state in the repr for Tkinter event objects is now represented as a combination of known flags.
- bpo-27177 [https://bugs.python.org/issue?@action=redirect&bpo=27177]: Match objects in the re module now support index-like objects as group indices. Based on patches by Jeroen Demeyer and Xiang Zhang.
- bpo-26754 [https://bugs.python.org/issue?@action=redirect&bpo=26754]: Some functions (compile() etc) accepted a filename argument encoded as an iterable of integers. Now only strings and byte-like objects are accepted.
- bpo-26536 [https://bugs.python.org/issue?@action=redirect&bpo=26536]: socket.ioctl now supports SIO_LOOPBACK_FAST_PATH. Patch by Daniel Stokes.
- bpo-27048 [https://bugs.python.org/issue?@action=redirect&bpo=27048]: Prevents distutils failing on Windows when environment variables contain non-ASCII characters
- bpo-27330 [https://bugs.python.org/issue?@action=redirect&bpo=27330]: Fixed possible leaks in the ctypes module.
- bpo-27238 [https://bugs.python.org/issue?@action=redirect&bpo=27238]: Got rid of bare excepts in the turtle module. Original patch by Jelle Zijlstra.
- bpo-27122 [https://bugs.python.org/issue?@action=redirect&bpo=27122]: When an exception is raised within the context being managed by a contextlib.ExitStack() and one of the exit stack generators catches and raises it in a chain, do not re-raise the original exception when exiting, let the new chained one through. This avoids the **PEP 479** [https://peps.python.org/pep-0479/] bug described in issue25782.
- bpo-16864 [https://bugs.python.org/issue?@action=redirect&bpo=16864]: sqlite3.Cursor.lastrowid now supports REPLACE statement. Initial patch by Alex LordThorsen.
- bpo-26386 [https://bugs.python.org/issue?@action=redirect&bpo=26386]: Fixed ttk.TreeView selection operations with item id's containing spaces.
- bpo-8637 [https://bugs.python.org/issue?@action=redirect&bpo=8637]: Honor a pager set by the env var MANPAGER (in preference to one set by the env var PAGER).
- bpo-16182 [https://bugs.python.org/issue?@action=redirect&bpo=16182]: Fix various functions in the "readline" module to use the locale encoding, and fix get_begidx() and get_endidx() to return code point indexes.

- bpo-27392 [https://bugs.python.org/issue?@action=redirect&bpo=27392]: Add loop.connect_accepted_socket(). Patch by Jim Fulton.

IDLE

- bpo-27477 [https://bugs.python.org/issue?@action=redirect&bpo=27477]: IDLE search dialogs now use ttk widgets.
- bpo-27173 [https://bugs.python.org/issue?@action=redirect&bpo=27173]: Add 'IDLE Modern Unix' to the built-in key sets. Make the default key set depend on the platform. Add tests for the changes to the config module.
- bpo-27452 [https://bugs.python.org/issue?@action=redirect&bpo=27452]: make command line "idle-test> python test_help.py" work. __file__ is relative when python is started in the file's directory.
- bpo-27452 [https://bugs.python.org/issue?@action=redirect&bpo=27452]: add line counter and crc to IDLE configHandler test dump.
- bpo-27380 [https://bugs.python.org/issue?@action=redirect&bpo=27380]: IDLE: add query.py with base Query dialog and ttk widgets. Module had subclasses SectionName, ModuleName, and HelpSource, which are used to get information from users by configdialog and file =>Load Module. Each subclass has itw own validity checks. Using ModuleName allows users to edit bad module names instead of starting over. Add tests and delete the two files combined into the new one.
- bpo-27372 [https://bugs.python.org/issue?@action=redirect&bpo=27372]: Test_idle no longer changes the locale.
- bpo-27365 [https://bugs.python.org/issue?@action=redirect&bpo=27365]: Allow non-ascii chars in IDLE NEWS.txt, for contributor names.
- bpo-27245 [https://bugs.python.org/issue?@action=redirect&bpo=27245]: IDLE: Cleanly delete custom themes and key bindings. Previously, when IDLE was started from a console or by import, a cascade of warnings was emitted. Patch by Serhiy Storchaka.
- bpo-24137 [https://bugs.python.org/issue?@action=redirect&bpo=24137]: Run IDLE, test_idle, and htest with tkinter default root disabled. Fix code and tests that fail with this restriction. Fix htests to not create a second and redundant root and mainloop.
- bpo-27310 [https://bugs.python.org/issue?@action=redirect&bpo=27310]: Fix IDLE.app failure to launch on OS X due to vestigial import.

API C

- bpo-26754 [https://bugs.python.org/issue?@action=redirect&bpo=26754]: PyUnicode_FSDecoder() accepted a filename argument encoded as an iterable of integers. Now only strings and byte-like objects are accepted.

Construção

- bpo-28066 [https://bugs.python.org/issue?@action=redirect&bpo=28066]: Fix the logic that searches build directories for generated include files when building outside the source tree.
- bpo-27442 [https://bugs.python.org/issue?@action=redirect&bpo=27442]: Expose the Android API level that python was built against, in sysconfig.get_config_vars() as ‘ANDROID_API_LEVEL’.
- bpo-27434 [https://bugs.python.org/issue?@action=redirect&bpo=27434]: The interpreter that runs the cross-build, found in PATH, must now be of the same feature version (e.g. 3.6) as the source being built.
- bpo-26930 [https://bugs.python.org/issue?@action=redirect&bpo=26930]: Update Windows builds to use OpenSSL 1.0.2h.
- bpo-23968 [https://bugs.python.org/issue?@action=redirect&bpo=23968]: Rename the platform directory from plat-$(MACHDEP) to plat-$(PLATFORM_TRIPLET). Rename the config directory (LIBPL) from config-$(LDVERSION) to config-$(LDVERSION)-$(PLATFORM_TRIPLET). Install the platform specific _sysconfigdata module into the platform directory and rename it to include the ABIFLAGS.
- Don’t use largefile support for GNU/Hurd.

Ferramentas/Demos

- bpo-27332 [https://bugs.python.org/issue?@action=redirect&bpo=27332]: Fixed the type of the first argument of module-level functions generated by Argument Clinic. Patch by Petr Viktorin.
- bpo-27418 [https://bugs.python.org/issue?@action=redirect&bpo=27418]: Fixed Tools/importbench/importbench.py.

Documentação

- bpo-19489 [https://bugs.python.org/issue?@action=redirect&bpo=19489]: Moved the search box from the sidebar to the header and footer of each page. Patch by Ammar Askar.
- bpo-27285 [https://bugs.python.org/issue?@action=redirect&bpo=27285]: Update documentation to reflect the deprecation of `pyvenv` and normalize on the term “virtual environment”. Patch by Steve Piercy.

Testes

- bpo-27027 [https://bugs.python.org/issue?@action=redirect&bpo=27027]: Added test.support.is_android that is True when this is an Android build.

Python 3.6.0 alpha 2

Data de lançamento: 2016-06-13

Segurança

- bpo-26556 [https://bugs.python.org/issue?@action=redirect&bpo=26556]: Update expat to 2.1.1, fixes **CVE 2015-1283** [https://www.cve.org/CVERecord?id=CVE-2015-1283].
- Fix TLS stripping vulnerability in smtplib, **CVE 2016-0772** [https://www.cve.org/CVERecord?id=CVE-2016-0772]. Reported by Team Oststrom.
- bpo-26839 [https://bugs.python.org/issue?@action=redirect&bpo=26839]: On Linux, **`os.urandom()`** now calls `getrandom()` with `GRND_NONBLOCK` to fall back on reading `/dev/urandom` if the urandom entropy pool is not initialized yet. Patch written by Colm Buckley.

Núcleo e embutidos

- bpo-27095 [https://bugs.python.org/issue?@action=redirect&bpo=27095]: Simplified MAKE_FUNCTION and removed MAKE_CLOSURE opcodes. Patch by Demur Rumed.

- bpo-27190 [https://bugs.python.org/issue?@action=redirect&bpo=27190]: Raise NotSupportedError if sqlite3 is older than 3.3.1. Patch by Dave Sawyer.
- bpo-27286 [https://bugs.python.org/issue?@action=redirect&bpo=27286]: Fixed compiling BUILD_MAP_UNPACK_WITH_CALL opcode. Calling function with generalized unpacking (PEP 448) and conflicting keyword names could cause undefined behavior.
- bpo-27140 [https://bugs.python.org/issue?@action=redirect&bpo=27140]: Added BUILD_CONST_KEY_MAP opcode.
- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Add support for os.PathLike objects to open() (part of **PEP 519** [https://peps.python.org/pep-0519/]).
- bpo-27066 [https://bugs.python.org/issue?@action=redirect&bpo=27066]: Fixed SystemError if a custom opener (for open()) returns a negative number without setting an exception.
- bpo-26983 [https://bugs.python.org/issue?@action=redirect&bpo=26983]: float() now always return an instance of exact float. The deprecation warning is emitted if __float__ returns an instance of a strict subclass of float. In a future versions of Python this can be an error.
- bpo-27097 [https://bugs.python.org/issue?@action=redirect&bpo=27097]: Python interpreter is now about 7% faster due to optimized instruction decoding. Based on patch by Demur Rumed.
- bpo-26647 [https://bugs.python.org/issue?@action=redirect&bpo=26647]: Python interpreter now uses 16-bit wordcode instead of bytecode. Patch by Demur Rumed.
- bpo-23275 [https://bugs.python.org/issue?@action=redirect&bpo=23275]: Allow assigning to an empty target list in round brackets: () = iterable.
- bpo-27243 [https://bugs.python.org/issue?@action=redirect&bpo=27243]: Update the __aiter__ protocol: instead of returning an awaitable that resolves to an asynchronous iterator, the asynchronous iterator should be returned directly. Doing the former will trigger a PendingDeprecationWarning.

Biblioteca

- Comment out socket (SO_REUSEPORT) and posix (O_SHLOCK, O_EXLOCK) constants exposed on the API which are not implemented on GNU/Hurd. They would not work at runtime anyway.

- bpo-27025 [https://bugs.python.org/issue?@action=redirect&bpo=27025]: Generated names for Tkinter widgets are now more meaningful and recognizable.
- bpo-25455 [https://bugs.python.org/issue?@action=redirect&bpo=25455]: Fixed crashes in repr of recursive ElementTree.Element and functools.partial objects.
- bpo-27294 [https://bugs.python.org/issue?@action=redirect&bpo=27294]: Improved repr for Tkinter event objects.
- bpo-20508 [https://bugs.python.org/issue?@action=redirect&bpo=20508]: Improve exception message of IPv{4,6}Network.__getitem__. Patch by Gareth Rees.
- bpo-21386 [https://bugs.python.org/issue?@action=redirect&bpo=21386]: Implement missing IPv4Address.is_global property. It was documented since 07a5610bae9d. Initial patch by Roger Luethi.
- bpo-27029 [https://bugs.python.org/issue?@action=redirect&bpo=27029]: Removed deprecated support of universal newlines mode from ZipFile.open().
- bpo-27030 [https://bugs.python.org/issue?@action=redirect&bpo=27030]: Unknown escapes consisting of `'\'` and an ASCII letter in regular expressions now are errors. The re.LOCALE flag now can be used only with bytes patterns.
- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Add os.PathLike support to DirEntry (part of **PEP 519** [https://peps.python.org/pep-0519/]). Initial patch by Jelle Zijlstra.
- bpo-20900 [https://bugs.python.org/issue?@action=redirect&bpo=20900]: distutils register command now decodes HTTP responses correctly. Initial patch by ingrid.
- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Add os.PathLike support to pathlib, removing its provisional status (part of PEP 519). Initial patch by Dusty Phillips.
- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Add support for os.PathLike objects to os.fsencode() and os.fsdecode() (part of **PEP 519** [https://peps.python.org/pep-0519/]).
- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Introduce os.PathLike and os.fspath() (part of **PEP 519** [https://peps.python.org/pep-0519/]).
- A new version of typing.py provides several new classes and features: @overload outside stubs, Reversible, DefaultDict, Text, ContextManager, Type[], NewType(), TYPE_CHECKING, and

numerous bug fixes (note that some of the new features are not yet implemented in mypy or other static analyzers). Also classes for **PEP 492** [https://peps.python.org/pep-0492/] (Awaitable, AsyncIterable, AsyncIterator) have been added (in fact they made it into 3.5.1 but were never mentioned).

- bpo-25738 [https://bugs.python.org/issue?@action=redirect&bpo=25738]: Stop http.server.BaseHTTPRequestHandler.send_error() from sending a message body for 205 Reset Content. Also, don't send Content header fields in responses that don't have a body. Patch by Susumu Koshiba.
- bpo-21313 [https://bugs.python.org/issue?@action=redirect&bpo=21313]: Fix the "platform" module to tolerate when sys.version contains truncated build information.
- bpo-23883 [https://bugs.python.org/issue?@action=redirect&bpo=23883]: Added missing APIs to __all__ to match the documented APIs for the following modules: cgi, mailbox, mimetypes, plistlib and smtpd. Patches by Jacek Kołodziej.
- bpo-27164 [https://bugs.python.org/issue?@action=redirect&bpo=27164]: In the zlib module, allow decompressing raw Deflate streams with a predefined zdict. Based on patch by Xiang Zhang.
- bpo-24291 [https://bugs.python.org/issue?@action=redirect&bpo=24291]: Fix wsgiref.simple_server.WSGIRequestHandler to completely write data to the client. Previously it could do partial writes and truncate data. Also, wsgiref.handler.ServerHandler can now handle stdout doing partial writes, but this is deprecated.
- bpo-21272 [https://bugs.python.org/issue?@action=redirect&bpo=21272]: Use _sysconfigdata.py to initialize distutils.sysconfig.
- bpo-19611 [https://bugs.python.org/issue?@action=redirect&bpo=19611]: **`inspect`** now reports the implicit `.0` parameters generated by the compiler for comprehension and generator expression scopes as if they were positional-only parameters called `implicit0`. Patch by Jelle Zijlstra.
- bpo-26809 [https://bugs.python.org/issue?@action=redirect&bpo=26809]: Add `__all__` to **`string`**. Patch by Emanuel Barry.
- bpo-26373 [https://bugs.python.org/issue?@action=redirect&bpo=26373]: subprocess.Popen.communicate now correctly ignores BrokenPipeError when the child process dies before .communicate() is called in more/all circumstances.

- signal, socket, and ssl module IntEnum constant name lookups now return a consistent name for values having multiple names. Ex: signal.Signals(6) now refers to itself as signal.SIGALRM rather than flipping between that and signal.SIGIOT based on the interpreter's hash randomization seed.
- bpo-27167 [https://bugs.python.org/issue?@action=redirect&bpo=27167]: Clarify the subprocess.CalledProcessError error message text when the child process died due to a signal.
- bpo-25931 [https://bugs.python.org/issue?@action=redirect&bpo=25931]: Don't define socketserver.Forking* names on platforms such as Windows that do not support os.fork().
- bpo-21776 [https://bugs.python.org/issue?@action=redirect&bpo=21776]: distutils.upload now correctly handles HTTPError. Initial patch by Claudiu Popa.
- bpo-26526 [https://bugs.python.org/issue?@action=redirect&bpo=26526]: Replace custom parse tree validation in the parser module with a simple DFA validator.
- bpo-27114 [https://bugs.python.org/issue?@action=redirect&bpo=27114]: Fix SSLContext._load_windows_store_certs fails with PermissionError
- bpo-18383 [https://bugs.python.org/issue?@action=redirect&bpo=18383]: Avoid creating duplicate filters when using filterwarnings and simplefilter. Based on patch by Alex Shkop.
- bpo-23026 [https://bugs.python.org/issue?@action=redirect&bpo=23026]: winreg.QueryValueEx() now return an integer for REG_QWORD type.
- bpo-26741 [https://bugs.python.org/issue?@action=redirect&bpo=26741]: subprocess.Popen destructor now emits a ResourceWarning warning if the child process is still running.
- bpo-27056 [https://bugs.python.org/issue?@action=redirect&bpo=27056]: Optimize pickle.load() and pickle.loads(), up to 10% faster to deserialize a lot of small objects.
- bpo-21271 [https://bugs.python.org/issue?@action=redirect&bpo=21271]: New keyword only parameters in reset_mock call.

IDLE

- bpo-5124 [https://bugs.python.org/issue?@action=redirect&bpo=5124]: Paste with text selected now replaces the selection on X11. This matches how paste works on Windows, Mac, most modern Linux apps, and ttk widgets. Original patch by Serhiy Storchaka.
- bpo-24750 [https://bugs.python.org/issue?@action=redirect&bpo=24750]: Switch all scrollbars in IDLE to ttk versions. Where needed, minimal tests are added to cover changes.
- bpo-24759 [https://bugs.python.org/issue?@action=redirect&bpo=24759]: IDLE requires tk 8.5 and availability ttk widgets. Delete now unneeded tk version tests and code for older versions. Add test for IDLE syntax colorizer.
- bpo-27239 [https://bugs.python.org/issue?@action=redirect&bpo=27239]: idlelib.macosx.isXyzTk functions initialize as needed.
- bpo-27262 [https://bugs.python.org/issue?@action=redirect&bpo=27262]: move Aqua unbinding code, which enable context menus, to macosx.
- bpo-24759 [https://bugs.python.org/issue?@action=redirect&bpo=24759]: Make clear in idlelib.idle_test.__init__ that the directory is a private implementation of test.test_idle and tool for maintainers.
- bpo-27196 [https://bugs.python.org/issue?@action=redirect&bpo=27196]: Stop 'ThemeChanged' warnings when running IDLE tests. These persisted after other warnings were suppressed in #20567. Apply Serhiy Storchaka's update_idletasks solution to four test files. Record this additional advice in idle_test/README.txt
- bpo-20567 [https://bugs.python.org/issue?@action=redirect&bpo=20567]: Revise idle_test/README.txt with advice about avoiding tk warning messages from tests. Apply advice to several IDLE tests.
- bpo-24225 [https://bugs.python.org/issue?@action=redirect&bpo=24225]: Update idlelib/README.txt with new file names and event handlers.
- bpo-27156 [https://bugs.python.org/issue?@action=redirect&bpo=27156]: Remove obsolete code not used by IDLE.
- bpo-27117 [https://bugs.python.org/issue?@action=redirect&bpo=27117]: Make colorizer htest and turtledemo work with dark themes. Move code for configuring text widget colors to a new function.
- bpo-24225 [https://bugs.python.org/issue?@action=redirect&bpo=24225]: Rename many `idlelib/*.py` and `idle_test/test_*.py` files. Edit files to replace old names with new names when the old name referred to the

module rather than the class it contained. See the issue and IDLE section in What's New in 3.6 for more.

- bpo-26673 [https://bugs.python.org/issue?@action=redirect&bpo=26673]: When tk reports font size as 0, change to size 10. Such fonts on Linux prevented the configuration dialog from opening.
- bpo-21939 [https://bugs.python.org/issue?@action=redirect&bpo=21939]: Add test for IDLE's percolator. Original patch by Saimadhav Heblikar.
- bpo-21676 [https://bugs.python.org/issue?@action=redirect&bpo=21676]: Add test for IDLE's replace dialog. Original patch by Saimadhav Heblikar.
- bpo-18410 [https://bugs.python.org/issue?@action=redirect&bpo=18410]: Add test for IDLE's search dialog. Original patch by Westley Martínez.
- bpo-21703 [https://bugs.python.org/issue?@action=redirect&bpo=21703]: Add test for undo delegator. Patch mostly by Saimadhav Heblikar .
- bpo-27044 [https://bugs.python.org/issue?@action=redirect&bpo=27044]: Add ConfigDialog.remove_var_callbacks to stop memory leaks.
- bpo-23977 [https://bugs.python.org/issue?@action=redirect&bpo=23977]: Add more asserts to test_delegator.

Documentação

- bpo-16484 [https://bugs.python.org/issue?@action=redirect&bpo=16484]: Change the default PYTHONDOCS URL to "https:", and fix the resulting links to use lowercase. Patch by Sean Rodman, test by Kaushik Nadikuditi.
- bpo-24136 [https://bugs.python.org/issue?@action=redirect&bpo=24136]: Document the new **PEP 448** [https://peps.python.org/pep-0448/] unpacking syntax of 3.5.
- bpo-22558 [https://bugs.python.org/issue?@action=redirect&bpo=22558]: Add remaining doc links to source code for Python-coded modules. Patch by Yoni Lavi.

Testes

- bpo-25285 [https://bugs.python.org/issue?@action=redirect&bpo=25285]: regrtest now uses subprocesses when the -j1 command line option is used: each test file runs in a fresh child process. Before, the -j1 option was ignored.
- bpo-25285 [https://bugs.python.org/issue?@action=redirect&bpo=25285]: Tools/buildbot/test.bat script now uses -j1 by default to run each test

file in fresh child process.

Windows

- bpo-27064 [https://bugs.python.org/issue?@action=redirect&bpo=27064]: The py.exe launcher now defaults to Python 3. The Windows launcher `py.exe` no longer prefers an installed Python 2 version over Python 3 by default when used interactively.
- bpo-17500 [https://bugs.python.org/issue?@action=redirect&bpo=17500]: Remove unused and outdated icons. (See also: https://github.com/python/pythondotorg/issues/945)

Construção

- bpo-27229 [https://bugs.python.org/issue?@action=redirect&bpo=27229]: Fix the cross-compiling pgen rule for in-tree builds. Patch by Xavier de Gaye.
- bpo-26930 [https://bugs.python.org/issue?@action=redirect&bpo=26930]: Update OS X 10.5+ 32-bit-only installer to build and link with OpenSSL 1.0.2h.

API C

- bpo-27186 [https://bugs.python.org/issue?@action=redirect&bpo=27186]: Add the PyOS_FSPath() function (part of **PEP 519** [https://peps.python.org/pep-0519/]).
- bpo-26282 [https://bugs.python.org/issue?@action=redirect&bpo=26282]: PyArg_ParseTupleAndKeywords() now supports positional-only parameters.

Ferramentas/Demos

- bpo-26282 [https://bugs.python.org/issue?@action=redirect&bpo=26282]: Argument Clinic now supports positional-only and keyword parameters in the same function.

Python 3.6.0 alpha 1

Data de lançamento: 2016-05-16

Segurança

- bpo-26657 [https://bugs.python.org/issue?@action=redirect&bpo=26657]: Fix directory traversal vulnerability with http.server on Windows. This fixes a regression that was introduced in 3.3.4rc1 and 3.4.0rc1. Based on patch by Philipp Hagemeister.
- bpo-26313 [https://bugs.python.org/issue?@action=redirect&bpo=26313]: ssl.py _load_windows_store_certs fails if windows cert store is empty. Patch by Baji.
- bpo-25939 [https://bugs.python.org/issue?@action=redirect&bpo=25939]: On Windows open the cert store readonly in ssl.enum_certificates.

Núcleo e embutidos

- bpo-20041 [https://bugs.python.org/issue?@action=redirect&bpo=20041]: Fixed TypeError when frame.f_trace is set to None. Patch by Xavier de Gaye.
- bpo-26168 [https://bugs.python.org/issue?@action=redirect&bpo=26168]: Fixed possible refleaks in failing Py_BuildValue() with the “N” format unit.
- bpo-26991 [https://bugs.python.org/issue?@action=redirect&bpo=26991]: Fix possible refleak when creating a function with annotations.
- bpo-27039 [https://bugs.python.org/issue?@action=redirect&bpo=27039]: Fixed bytearray.remove() for values greater than 127. Based on patch by Joe Jevnik.
- bpo-23640 [https://bugs.python.org/issue?@action=redirect&bpo=23640]: int.from_bytes() no longer bypasses constructors for subclasses.
- bpo-27005 [https://bugs.python.org/issue?@action=redirect&bpo=27005]: Optimized the float.fromhex() class method for exact float. It is now 2 times faster.
- bpo-18531 [https://bugs.python.org/issue?@action=redirect&bpo=18531]: Single var-keyword argument of dict subtype was passed unscathed to the C-defined function. Now it is converted to exact dict.
- bpo-26811 [https://bugs.python.org/issue?@action=redirect&bpo=26811]: gc.get_objects() no longer contains a broken tuple with NULL pointer.
- bpo-20120 [https://bugs.python.org/issue?@action=redirect&bpo=20120]: Use RawConfigParser for .pypirc parsing, removing support for interpolation unintentionally added with move to Python 3. Behavior

no longer does any interpolation in .pypirc files, matching behavior in Python 2.7 and Setuptools 19.0.

- bpo-26249 [https://bugs.python.org/issue?@action=redirect&bpo=26249]: Memory functions of the **`PyMem_Malloc()`** domain (**`PYMEM_DOMAIN_MEM`**) now use the pymalloc allocator rather than system **`malloc()`**. Applications calling **`PyMem_Malloc()`** without holding the GIL can now crash: use `PYTHONMALLOC=debug` environment variable to validate the usage of memory allocators in your application.
- bpo-26802 [https://bugs.python.org/issue?@action=redirect&bpo=26802]: Optimize function calls only using unpacking like `func(*tuple)` (no other positional argument, no keyword): avoid copying the tuple. Patch written by Joe Jevnik.
- bpo-26659 [https://bugs.python.org/issue?@action=redirect&bpo=26659]: Make the builtin slice type support cycle collection.
- bpo-26718 [https://bugs.python.org/issue?@action=redirect&bpo=26718]: super.__init__ no longer leaks memory if called multiple times. NOTE: A direct call of super.__init__ is not endorsed!
- bpo-27138 [https://bugs.python.org/issue?@action=redirect&bpo=27138]: Fix the doc comment for FileFinder.find_spec().
- bpo-27147 [https://bugs.python.org/issue?@action=redirect&bpo=27147]: Mention **PEP 420** [https://peps.python.org/pep-0420/] in the importlib docs.
- bpo-25339 [https://bugs.python.org/issue?@action=redirect&bpo=25339]: PYTHONIOENCODING now has priority over locale in setting the error handler for stdin and stdout.
- bpo-26494 [https://bugs.python.org/issue?@action=redirect&bpo=26494]: Fixed crash on iterating exhausting iterators. Affected classes are generic sequence iterators, iterators of str, bytes, bytearray, list, tuple, set, frozenset, dict, OrderedDict, corresponding views and os.scandir() iterator.
- bpo-26574 [https://bugs.python.org/issue?@action=redirect&bpo=26574]: Optimize `bytes.replace(b'', b'.')` and `bytearray.replace(b'', b'.')`. Patch written by Josh Snider.
- bpo-26581 [https://bugs.python.org/issue?@action=redirect&bpo=26581]: If coding cookie is specified multiple times on a line in Python source code file, only the first one is taken to account.
- bpo-19711 [https://bugs.python.org/issue?@action=redirect&bpo=19711]: Add tests for reloading namespace packages.

- bpo-21099 [https://bugs.python.org/issue?@action=redirect&bpo=21099]: Switch applicable importlib tests to use **PEP 451** [https://peps.python.org/pep-0451/] API.
- bpo-26563 [https://bugs.python.org/issue?@action=redirect&bpo=26563]: Debug hooks on Python memory allocators now raise a fatal error if functions of the `PyMem_Malloc()` family are called without holding the GIL.
- bpo-26564 [https://bugs.python.org/issue?@action=redirect&bpo=26564]: On error, the debug hooks on Python memory allocators now use the `tracemalloc` module to get the traceback where a memory block was allocated.
- bpo-26558 [https://bugs.python.org/issue?@action=redirect&bpo=26558]: The debug hooks on Python memory allocator `PyObject_Malloc()` now detect when functions are called without holding the GIL.
- bpo-26516 [https://bugs.python.org/issue?@action=redirect&bpo=26516]: Add `PYTHONMALLOC` environment variable to set the Python memory allocators and/or install debug hooks.
- bpo-26516 [https://bugs.python.org/issue?@action=redirect&bpo=26516]: The `PyMem_SetupDebugHooks()` function can now also be used on Python compiled in release mode.
- bpo-26516 [https://bugs.python.org/issue?@action=redirect&bpo=26516]: The `PYTHONMALLOCSTATS` environment variable can now also be used on Python compiled in release mode. It now has no effect if set to an empty string.
- bpo-26516 [https://bugs.python.org/issue?@action=redirect&bpo=26516]: In debug mode, debug hooks are now also installed on Python memory allocators when Python is configured without pymalloc.
- bpo-26464 [https://bugs.python.org/issue?@action=redirect&bpo=26464]: Fix str.translate() when string is ASCII and first replacements removes character, but next replacement uses a non-ASCII character or a string longer than 1 character. Regression introduced in Python 3.5.0.
- bpo-22836 [https://bugs.python.org/issue?@action=redirect&bpo=22836]: Ensure exception reports from PyErr_Display() and PyErr_WriteUnraisable() are sensible even when formatting them produces secondary errors. This affects the reports produced by sys.__excepthook__() and when __del__() raises an exception.
- bpo-26302 [https://bugs.python.org/issue?@action=redirect&bpo=26302]: Correct behavior to reject comma as a legal character for cookie names.

- bpo-26136 [https://bugs.python.org/issue?@action=redirect&bpo=26136]: Upgrade the warning when a generator raises StopIteration from PendingDeprecationWarning to DeprecationWarning. Patch by Anish Shah.
- bpo-26204 [https://bugs.python.org/issue?@action=redirect&bpo=26204]: The compiler now ignores all constant statements: bytes, str, int, float, complex, name constants (None, False, True), Ellipsis and ast.Constant; not only str and int. For example, `1.0` is now ignored in `def f(): 1.0`.
- bpo-4806 [https://bugs.python.org/issue?@action=redirect&bpo=4806]: Avoid masking the original TypeError exception when using star (`*`) unpacking in function calls. Based on patch by Hagen Fürstenau and Daniel Urban.
- bpo-26146 [https://bugs.python.org/issue?@action=redirect&bpo=26146]: Add a new kind of AST node: `ast.Constant`. It can be used by external AST optimizers, but the compiler does not emit directly such node.
- bpo-23601 [https://bugs.python.org/issue?@action=redirect&bpo=23601]: Sped-up allocation of dict key objects by using Python's small object allocator. (Contributed by Julian Taylor.)
- bpo-18018 [https://bugs.python.org/issue?@action=redirect&bpo=18018]: Import raises ImportError instead of SystemError if a relative import is attempted without a known parent package.
- bpo-25843 [https://bugs.python.org/issue?@action=redirect&bpo=25843]: When compiling code, don't merge constants if they are equal but have a different types. For example, `f1, f2 = lambda: 1, lambda: 1.0` is now correctly compiled to two different functions: `f1()` returns `1` (`int`) and `f2()` returns `1.0` (`float`), even if `1` and `1.0` are equal.
- bpo-26107 [https://bugs.python.org/issue?@action=redirect&bpo=26107]: The format of the `co_lnotab` attribute of code objects changes to support negative line number delta.
- bpo-26154 [https://bugs.python.org/issue?@action=redirect&bpo=26154]: Add a new private _PyThreadState_UncheckedGet() function to get the current Python thread state, but don't issue a fatal error if it is NULL. This new function must be used instead of accessing directly the _PyThreadState_Current variable. The variable is no more exposed since Python 3.5.1 to hide the exact implementation of atomic C types, to avoid compiler issues.

- bpo-25791 [https://bugs.python.org/issue?@action=redirect&bpo=25791]: If __package__ != __spec__.parent or if neither __package__ or __spec__ are defined then ImportWarning is raised.
- bpo-22995 [https://bugs.python.org/issue?@action=redirect&bpo=22995]: [UPDATE] Comment out the one of the pickleability tests in _PyObject_GetState() due to regressions observed in Cython-based projects.
- bpo-25961 [https://bugs.python.org/issue?@action=redirect&bpo=25961]: Disallowed null characters in the type name.
- bpo-25973 [https://bugs.python.org/issue?@action=redirect&bpo=25973]: Fix segfault when an invalid nonlocal statement binds a name starting with two underscores.
- bpo-22995 [https://bugs.python.org/issue?@action=redirect&bpo=22995]: Instances of extension types with a state that aren't subclasses of list or dict and haven't implemented any pickle-related methods (__reduce__, __reduce_ex__, __getnewargs__, __getnewargs_ex__, or __getstate__), can no longer be pickled. Including memoryview.
- bpo-20440 [https://bugs.python.org/issue?@action=redirect&bpo=20440]: Massive replacing unsafe attribute setting code with special macro Py_SETREF.
- bpo-25766 [https://bugs.python.org/issue?@action=redirect&bpo=25766]: Special method __bytes__() now works in str subclasses.
- bpo-25421 [https://bugs.python.org/issue?@action=redirect&bpo=25421]: __sizeof__ methods of builtin types now use dynamic basic size. This allows sys.getsize() to work correctly with their subclasses with __slots__ defined.
- bpo-25709 [https://bugs.python.org/issue?@action=redirect&bpo=25709]: Fixed problem with in-place string concatenation and utf-8 cache.
- bpo-5319 [https://bugs.python.org/issue?@action=redirect&bpo=5319]: New Py_FinalizeEx() API allowing Python to set an exit status of 120 on failure to flush buffered streams.
- bpo-25485 [https://bugs.python.org/issue?@action=redirect&bpo=25485]: telnetlib.Telnet is now a context manager.
- bpo-24097 [https://bugs.python.org/issue?@action=redirect&bpo=24097]: Fixed crash in object.__reduce__() if slot name is freed inside __getattr__.
- bpo-24731 [https://bugs.python.org/issue?@action=redirect&bpo=24731]: Fixed crash on converting objects with special methods __bytes__, __trunc__, and

__float__ returning instances of subclasses of bytes, int, and float to subclasses of bytes, int, and float correspondingly.

- bpo-25630 [https://bugs.python.org/issue?@action=redirect&bpo=25630]: Fix a possible segfault during argument parsing in functions that accept filesystem paths.
- bpo-23564 [https://bugs.python.org/issue?@action=redirect&bpo=23564]: Fixed a partially broken sanity check in the _posixsubprocess internals regarding how fds_to_pass were passed to the child. The bug had no actual impact as subprocess.py already avoided it.
- bpo-25388 [https://bugs.python.org/issue?@action=redirect&bpo=25388]: Fixed tokenizer crash when processing undecodable source code with a null byte.
- bpo-25462 [https://bugs.python.org/issue?@action=redirect&bpo=25462]: The hash of the key now is calculated only once in most operations in C implementation of OrderedDict.
- bpo-22995 [https://bugs.python.org/issue?@action=redirect&bpo=22995]: Default implementation of __reduce__ and __reduce_ex__ now rejects builtin types with not defined __new__.
- bpo-24802 [https://bugs.python.org/issue?@action=redirect&bpo=24802]: Avoid buffer overreads when int(), float(), compile(), exec() and eval() are passed bytes-like objects. These objects are not necessarily terminated by a null byte, but the functions assumed they were.
- bpo-25555 [https://bugs.python.org/issue?@action=redirect&bpo=25555]: Fix parser and AST: fill lineno and col_offset of “arg” node when compiling AST from Python objects.
- bpo-24726 [https://bugs.python.org/issue?@action=redirect&bpo=24726]: Fixed a crash and leaking NULL in repr() of OrderedDict that was mutated by direct calls of dict methods.
- bpo-25449 [https://bugs.python.org/issue?@action=redirect&bpo=25449]: Iterating OrderedDict with keys with unstable hash now raises KeyError in C implementations as well as in Python implementation.
- bpo-25395 [https://bugs.python.org/issue?@action=redirect&bpo=25395]: Fixed crash when highly nested OrderedDict structures were garbage collected.
- bpo-25401 [https://bugs.python.org/issue?@action=redirect&bpo=25401]: Optimize bytes.fromhex() and bytearray.fromhex(): they are now between 2x and 3.5x faster.

- bpo-25399 [https://bugs.python.org/issue?@action=redirect&bpo=25399]: Optimize bytearray % args using the new private _PyBytesWriter API. Formatting is now between 2.5 and 5 times faster.
- bpo-25274 [https://bugs.python.org/issue?@action=redirect&bpo=25274]: sys.setrecursionlimit() now raises a RecursionError if the new recursion limit is too low depending at the current recursion depth. Modify also the “lower-water mark” formula to make it monotonic. This mark is used to decide when the overflowed flag of the thread state is reset.
- bpo-24402 [https://bugs.python.org/issue?@action=redirect&bpo=24402]: Fix input() to prompt to the redirected stdout when sys.stdout.fileno() fails.
- bpo-25349 [https://bugs.python.org/issue?@action=redirect&bpo=25349]: Optimize bytes % args using the new private _PyBytesWriter API. Formatting is now up to 2 times faster.
- bpo-24806 [https://bugs.python.org/issue?@action=redirect&bpo=24806]: Prevent builtin types that are not allowed to be subclassed from being subclassed through multiple inheritance.
- bpo-25301 [https://bugs.python.org/issue?@action=redirect&bpo=25301]: The UTF-8 decoder is now up to 15 times as fast for error handlers: `ignore`, `replace` and `surrogateescape`.
- bpo-24848 [https://bugs.python.org/issue?@action=redirect&bpo=24848]: Fixed a number of bugs in UTF-7 decoding of misformed data.
- bpo-25267 [https://bugs.python.org/issue?@action=redirect&bpo=25267]: The UTF-8 encoder is now up to 75 times as fast for error handlers: `ignore`, `replace`, `surrogateescape`, `surrogatepass`. Patch co-written with Serhiy Storchaka.
- bpo-25280 [https://bugs.python.org/issue?@action=redirect&bpo=25280]: Import trace messages emitted in verbose (-v) mode are no longer formatted twice.
- bpo-25227 [https://bugs.python.org/issue?@action=redirect&bpo=25227]: Optimize ASCII and latin1 encoders with the `surrogateescape` error handler: the encoders are now up to 3 times as fast. Initial patch written by Serhiy Storchaka.
- bpo-25003 [https://bugs.python.org/issue?@action=redirect&bpo=25003]: On Solaris 11.3 or newer, os.urandom() now uses the getrandom() function instead of the getentropy() function. The getentropy() function is

blocking to generate very good quality entropy, os.urandom() doesn't need such high-quality entropy.

- bpo-9232 [https://bugs.python.org/issue?@action=redirect&bpo=9232]: Modify Python's grammar to allow trailing commas in the argument list of a function declaration. For example, "def f(*, a = 3,): pass" is now legal. Patch from Mark Dickinson.
- bpo-24965 [https://bugs.python.org/issue?@action=redirect&bpo=24965]: Implement **PEP 498** [https://peps.python.org/pep-0498/] "Literal String Interpolation". This allows you to embed expressions inside f-strings, which are converted to normal strings at run time. Given x=3, then f'value={x}' == 'value=3'. Patch by Eric V. Smith.
- bpo-26478 [https://bugs.python.org/issue?@action=redirect&bpo=26478]: Fix semantic bugs when using binary operators with dictionary views and tuples.
- bpo-26171 [https://bugs.python.org/issue?@action=redirect&bpo=26171]: Fix possible integer overflow and heap corruption in zipimporter.get_data().
- bpo-25660 [https://bugs.python.org/issue?@action=redirect&bpo=25660]: Fix TAB key behaviour in REPL with readline.
- bpo-26288 [https://bugs.python.org/issue?@action=redirect&bpo=26288]: Optimize PyLong_AsDouble.
- bpo-26289 [https://bugs.python.org/issue?@action=redirect&bpo=26289]: Optimize floor and modulo division for single-digit longs. Microbenchmarks show 2-2.5x improvement. Built-in 'divmod' function is now also ~10% faster. (See also: bpo-26315 [https://bugs.python.org/issue?@action=redirect&bpo=26315])
- bpo-25887 [https://bugs.python.org/issue?@action=redirect&bpo=25887]: Raise a RuntimeError when a coroutine object is awaited more than once.

Biblioteca

- bpo-27057 [https://bugs.python.org/issue?@action=redirect&bpo=27057]: Fix os.set_inheritable() on Android, ioctl() is blocked by SELinux and fails with EACCESS. The function now falls back to fcntl(). Patch written by Michał Bednarski.
- bpo-27014 [https://bugs.python.org/issue?@action=redirect&bpo=27014]: Fix infinite recursion using typing.py. Thanks to Kalle Tuure!

- bpo-27031 [https://bugs.python.org/issue?@action=redirect&bpo=27031]: Removed dummy methods in Tkinter widget classes: tk_menuBar() and tk_bindForTraversal().
- bpo-14132 [https://bugs.python.org/issue?@action=redirect&bpo=14132]: Fix urllib.request redirect handling when the target only has a query string. Original fix by Ján Janech.
- bpo-17214 [https://bugs.python.org/issue?@action=redirect&bpo=17214]: The “urllib.request” module now percent-encodes non-ASCII bytes found in redirect target URLs. Some servers send Location header fields with non-ASCII bytes, but “http.client” requires the request target to be ASCII-encodable, otherwise a UnicodeEncodeError is raised. Based on patch by Christian Heimes.
- bpo-27033 [https://bugs.python.org/issue?@action=redirect&bpo=27033]: The default value of the decode_data parameter for smtpd.SMTPChannel and smtpd.SMTPServer constructors is changed to False.
- bpo-27034 [https://bugs.python.org/issue?@action=redirect&bpo=27034]: Removed deprecated class asynchat.fifo.
- bpo-26870 [https://bugs.python.org/issue?@action=redirect&bpo=26870]: Added readline.set_auto_history(), which can stop entries being automatically added to the history list. Based on patch by Tyler Crompton.
- bpo-26039 [https://bugs.python.org/issue?@action=redirect&bpo=26039]: zipfile.ZipFile.open() can now be used to write data into a ZIP file, as well as for extracting data. Patch by Thomas Kluyver.
- bpo-26892 [https://bugs.python.org/issue?@action=redirect&bpo=26892]: Honor debuglevel flag in urllib.request.HTTPHandler. Patch contributed by Chi Hsuan Yen.
- bpo-22274 [https://bugs.python.org/issue?@action=redirect&bpo=22274]: In the subprocess module, allow stderr to be redirected to stdout even when stdout is not redirected. Patch by Akira Li.
- bpo-26807 [https://bugs.python.org/issue?@action=redirect&bpo=26807]: mock_open ‘files’ no longer error on readline at end of file. Patch from Yolanda Robla.
- bpo-25745 [https://bugs.python.org/issue?@action=redirect&bpo=25745]: Fixed leaking a userptr in curses panel destructor.
- bpo-26977 [https://bugs.python.org/issue?@action=redirect&bpo=26977]: Removed unnecessary, and ignored, call to sum of squares helper in statistics.pvariance.

- bpo-26002 [https://bugs.python.org/issue?@action=redirect&bpo=26002]: Use bisect in statistics.median instead of a linear search. Patch by Upendra Kuma.
- bpo-25974 [https://bugs.python.org/issue?@action=redirect&bpo=25974]: Make use of new Decimal.as_integer_ratio() method in statistics module. Patch by Stefan Krah.
- bpo-26996 [https://bugs.python.org/issue?@action=redirect&bpo=26996]: Add secrets module as described in **PEP 506** [https://peps.python.org/pep-0506/].
- bpo-26881 [https://bugs.python.org/issue?@action=redirect&bpo=26881]: The modulefinder module now supports extended opcode arguments.
- bpo-23815 [https://bugs.python.org/issue?@action=redirect&bpo=23815]: Fixed crashes related to directly created instances of types in _tkinter and curses.panel modules.
- bpo-17765 [https://bugs.python.org/issue?@action=redirect&bpo=17765]: weakref.ref() no longer silently ignores keyword arguments. Patch by Georg Brandl.
- bpo-26873 [https://bugs.python.org/issue?@action=redirect&bpo=26873]: xmlrpc now raises ResponseError on unsupported type tags instead of silently return incorrect result.
- bpo-26915 [https://bugs.python.org/issue?@action=redirect&bpo=26915]: The __contains__ methods in the collections ABCs now check for identity before checking equality. This better matches the behavior of the concrete classes, allows sensible handling of NaNs, and makes it easier to reason about container invariants.
- bpo-26711 [https://bugs.python.org/issue?@action=redirect&bpo=26711]: Fixed the comparison of plistlib.Data with other types.
- bpo-24114 [https://bugs.python.org/issue?@action=redirect&bpo=24114]: Fix an uninitialized variable in `ctypes.util`. The bug only occurs on SunOS when the ctypes implementation searches for the `crle` program. Patch by Xiang Zhang. Tested on SunOS by Kees Bos.
- bpo-26864 [https://bugs.python.org/issue?@action=redirect&bpo=26864]: In urllib.request, change the proxy bypass host checking against no_proxy to be case-insensitive, and to not match unrelated host names that happen to have a bypassed hostname as a suffix. Patch by Xiang Zhang.
- bpo-24902 [https://bugs.python.org/issue?@action=redirect&bpo=24902]: Print server URL on http.server startup. Initial patch by Felix Kaiser.

- bpo-25788 [https://bugs.python.org/issue?@action=redirect&bpo=25788]: fileinput.hook_encoded() now supports an “errors” argument for passing to open. Original patch by Joseph Hackman.
- bpo-26634 [https://bugs.python.org/issue?@action=redirect&bpo=26634]: recursive_repr() now sets __qualname__ of wrapper. Patch by Xiang Zhang.
- bpo-26804 [https://bugs.python.org/issue?@action=redirect&bpo=26804]: urllib.request will prefer lower_case proxy environment variables over UPPER_CASE or Mixed_Case ones. Patch contributed by Hans-Peter Jansen.
- bpo-26837 [https://bugs.python.org/issue?@action=redirect&bpo=26837]: assertSequenceEqual() now correctly outputs non-stringified differing items (like bytes in the -b mode). This affects assertListEqual() and assertTupleEqual().
- bpo-26041 [https://bugs.python.org/issue?@action=redirect&bpo=26041]: Remove “will be removed in Python 3.7” from deprecation messages of platform.dist() and platform.linux_distribution(). Patch by Kumaripaba Miyurusara Athukorala.
- bpo-26822 [https://bugs.python.org/issue?@action=redirect&bpo=26822]: itemgetter, attrgetter and methodcaller objects no longer silently ignore keyword arguments.
- bpo-26733 [https://bugs.python.org/issue?@action=redirect&bpo=26733]: Disassembling a class now disassembles class and static methods. Patch by Xiang Zhang.
- bpo-26801 [https://bugs.python.org/issue?@action=redirect&bpo=26801]: Fix error handling in `shutil.get_terminal_size()`, catch `AttributeError` instead of `NameError`. Patch written by Emanuel Barry.
- bpo-24838 [https://bugs.python.org/issue?@action=redirect&bpo=24838]: tarfile’s ustar and gnu formats now correctly calculate name and link field limits for multibyte character encodings like utf-8.
- bpo-26717 [https://bugs.python.org/issue?@action=redirect&bpo=26717]: Stop encoding Latin-1-ized WSGI paths with UTF-8. Patch by Anthony Sottile.
- bpo-26782 [https://bugs.python.org/issue?@action=redirect&bpo=26782]: Add STARTUPINFO to subprocess.__all__ on Windows.
- bpo-26404 [https://bugs.python.org/issue?@action=redirect&bpo=26404]: Add context manager to socketserver. Patch by Aviv Palivoda.

- bpo-26735 [https://bugs.python.org/issue?@action=redirect&bpo=26735]: Fix `os.urandom()` on Solaris 11.3 and newer when reading more than 1,024 bytes: call `getrandom()` multiple times with a limit of 1024 bytes per call.
- bpo-26585 [https://bugs.python.org/issue?@action=redirect&bpo=26585]: Eliminate http.server._quote_html() and use html.escape(quote=False). Patch by Xiang Zhang.
- bpo-26685 [https://bugs.python.org/issue?@action=redirect&bpo=26685]: Raise OSError if closing a socket fails.
- bpo-16329 [https://bugs.python.org/issue?@action=redirect&bpo=16329]: Add .webm to mimetypes.types_map. Patch by Giampaolo Rodola'.
- bpo-13952 [https://bugs.python.org/issue?@action=redirect&bpo=13952]: Add .csv to mimetypes.types_map. Patch by Geoff Wilson.
- bpo-26587 [https://bugs.python.org/issue?@action=redirect&bpo=26587]: the site module now allows .pth files to specify files to be added to sys.path (e.g. zip files).
- bpo-25609 [https://bugs.python.org/issue?@action=redirect&bpo=25609]: Introduce contextlib.AbstractContextManager and typing.ContextManager.
- bpo-26709 [https://bugs.python.org/issue?@action=redirect&bpo=26709]: Fixed Y2038 problem in loading binary PLists.
- bpo-23735 [https://bugs.python.org/issue?@action=redirect&bpo=23735]: Handle terminal resizing with Readline 6.3+ by installing our own SIGWINCH handler. Patch by Eric Price.
- bpo-25951 [https://bugs.python.org/issue?@action=redirect&bpo=25951]: Change SSLSocket.sendall() to return None, as explicitly documented for plain socket objects. Patch by Aviv Palivoda.
- bpo-26586 [https://bugs.python.org/issue?@action=redirect&bpo=26586]: In http.server, respond with "413 Request header fields too large" if there are too many header fields to parse, rather than killing the connection and raising an unhandled exception. Patch by Xiang Zhang.
- bpo-26676 [https://bugs.python.org/issue?@action=redirect&bpo=26676]: Added missing XMLPullParser to ElementTree.__all__.
- bpo-22854 [https://bugs.python.org/issue?@action=redirect&bpo=22854]: Change BufferedReader.writable() and BufferedWriter.readable() to always return False.
- bpo-26492 [https://bugs.python.org/issue?@action=redirect&bpo=26492]: Exhausted iterator of array.array now conforms with the behavior of iterators of

other mutable sequences: it lefts exhausted even if iterated array is extended.

- bpo-26641 [https://bugs.python.org/issue?@action=redirect&bpo=26641]: doctest.DocFileTest and doctest.testfile() now support packages (module split into multiple directories) for the package parameter.
- bpo-25195 [https://bugs.python.org/issue?@action=redirect&bpo=25195]: Fix a regression in mock.MagicMock. _Call is a subclass of tuple (changeset 3603bae63c13 only works for classes) so we need to implement __ne__ ourselves. Patch by Andrew Plummer.
- bpo-26644 [https://bugs.python.org/issue?@action=redirect&bpo=26644]: Raise ValueError rather than SystemError when a negative length is passed to SSLSocket.recv() or read().
- bpo-23804 [https://bugs.python.org/issue?@action=redirect&bpo=23804]: Fix SSL recv(0) and read(0) methods to return zero bytes instead of up to 1024.
- bpo-26616 [https://bugs.python.org/issue?@action=redirect&bpo=26616]: Fixed a bug in datetime.astimezone() method.
- bpo-26637 [https://bugs.python.org/issue?@action=redirect&bpo=26637]: The **`importlib`** module now emits an **`ImportError`** rather than a **`TypeError`** if **`__import__()`** is tried during the Python shutdown process but **`sys.path`** is already cleared (set to `None`).
- bpo-21925 [https://bugs.python.org/issue?@action=redirect&bpo=21925]: **`warnings.formatwarning()`** now catches exceptions when calling **`linecache.getline()`** and **`tracemalloc.get_object_traceback()`** to be able to log **`ResourceWarning`** emitted late during the Python shutdown process.
- bpo-23848 [https://bugs.python.org/issue?@action=redirect&bpo=23848]: On Windows, faulthandler.enable() now also installs an exception handler to dump the traceback of all Python threads on any Windows exception, not only on UNIX signals (SIGSEGV, SIGFPE, SIGABRT).
- bpo-26530 [https://bugs.python.org/issue?@action=redirect&bpo=26530]: Add C functions **`_PyTraceMalloc_Track()`** and **`_PyTraceMalloc_Untrack()`** to track memory blocks using the **`tracemalloc`** module. Add **`_PyTraceMalloc_GetTraceback()`** to get the traceback of an object.
- bpo-26588 [https://bugs.python.org/issue?@action=redirect&bpo=26588]: The _tracemalloc now supports tracing memory allocations of multiple address spaces (domains).

- bpo-24266 [https://bugs.python.org/issue?@action=redirect&bpo=24266]: Ctrl+C during Readline history search now cancels the search mode when compiled with Readline 7.
- bpo-26590 [https://bugs.python.org/issue?@action=redirect&bpo=26590]: Implement a safe finalizer for the _socket.socket type. It now releases the GIL to close the socket.
- bpo-18787 [https://bugs.python.org/issue?@action=redirect&bpo=18787]: spwd.getspnam() now raises a PermissionError if the user doesn't have privileges.
- bpo-26560 [https://bugs.python.org/issue?@action=redirect&bpo=26560]: Avoid potential ValueError in BaseHandler.start_response. Initial patch by Peter Inglesby.
- bpo-26567 [https://bugs.python.org/issue?@action=redirect&bpo=26567]: Add a new function `PyErr_ResourceWarning()` function to pass the destroyed object. Add a *source* attribute to `warnings.WarningMessage`. Add warnings._showwarnmsg() which uses tracemalloc to get the traceback where source object was allocated.
- bpo-26569 [https://bugs.python.org/issue?@action=redirect&bpo=26569]: Fix `pyclbr.readmodule()` and `pyclbr.readmodule_ex()` to support importing packages.
- bpo-26499 [https://bugs.python.org/issue?@action=redirect&bpo=26499]: Account for remaining Content-Length in HTTPResponse.readline() and read1(). Based on patch by Silent Ghost. Also document that HTTPResponse now supports these methods.
- bpo-25320 [https://bugs.python.org/issue?@action=redirect&bpo=25320]: Handle sockets in directories unittest discovery is scanning. Patch from Victor van den Elzen.
- bpo-16181 [https://bugs.python.org/issue?@action=redirect&bpo=16181]: cookiejar.http2time() now returns None if year is higher than datetime.MAXYEAR.
- bpo-26513 [https://bugs.python.org/issue?@action=redirect&bpo=26513]: Fixes platform module detection of Windows Server
- bpo-23718 [https://bugs.python.org/issue?@action=redirect&bpo=23718]: Fixed parsing time in week 0 before Jan 1. Original patch by Tamás Bence Gedai.
- bpo-26323 [https://bugs.python.org/issue?@action=redirect&bpo=26323]: Add Mock.assert_called() and Mock.assert_called_once() methods to

unittest.mock. Patch written by Amit Saha.

- bpo-20589 [https://bugs.python.org/issue?@action=redirect&bpo=20589]: Invoking Path.owner() and Path.group() on Windows now raise NotImplementedError instead of ImportError.
- bpo-26177 [https://bugs.python.org/issue?@action=redirect&bpo=26177]: Fixed the keys() method for Canvas and Scrollbar widgets.
- bpo-15068 [https://bugs.python.org/issue?@action=redirect&bpo=15068]: Got rid of excessive buffering in fileinput. The bufsize parameter is now deprecated and ignored.
- bpo-19475 [https://bugs.python.org/issue?@action=redirect&bpo=19475]: Added an optional argument timespec to the datetime isoformat() method to choose the precision of the time component.
- bpo-2202 [https://bugs.python.org/issue?@action=redirect&bpo=2202]: Fix UnboundLocalError in AbstractDigestAuthHandler.get_algorithm_impls. Initial patch by Mathieu Dupuy.
- bpo-26167 [https://bugs.python.org/issue?@action=redirect&bpo=26167]: Minimized overhead in copy.copy() and copy.deepcopy(). Optimized copying and deepcopying bytearrays, NotImplemented, slices, short lists, tuples, dicts, sets.
- bpo-25718 [https://bugs.python.org/issue?@action=redirect&bpo=25718]: Fixed pickling and copying the accumulate() iterator with total is None.
- bpo-26475 [https://bugs.python.org/issue?@action=redirect&bpo=26475]: Fixed debugging output for regular expressions with the (?x) flag.
- bpo-26482 [https://bugs.python.org/issue?@action=redirect&bpo=26482]: Allowed pickling recursive dequeues.
- bpo-26335 [https://bugs.python.org/issue?@action=redirect&bpo=26335]: Make mmap.write() return the number of bytes written like other write methods. Patch by Jakub Stasiak.
- bpo-26457 [https://bugs.python.org/issue?@action=redirect&bpo=26457]: Fixed the subnets() methods in IP network classes for the case when resulting prefix length is equal to maximal prefix length. Based on patch by Xiang Zhang.
- bpo-26385 [https://bugs.python.org/issue?@action=redirect&bpo=26385]: Remove the file if the internal open() call in NamedTemporaryFile() fails. Patch by Silent Ghost.

- bpo-26402 [https://bugs.python.org/issue?@action=redirect&bpo=26402]: Fix XML-RPC client to retry when the server shuts down a persistent connection. This was a regression related to the new http.client.RemoteDisconnected exception in 3.5.0a4.
- bpo-25913 [https://bugs.python.org/issue?@action=redirect&bpo=25913]: Leading <~ is optional now in base64.a85decode() with adobe=True. Patch by Swati Jaiswal.
- bpo-26186 [https://bugs.python.org/issue?@action=redirect&bpo=26186]: Remove an invalid type check in importlib.util.LazyLoader.
- bpo-26367 [https://bugs.python.org/issue?@action=redirect&bpo=26367]: importlib.__import__() raises ImportError like builtins.__import__() when `level` is specified but without an accompanying package specified.
- bpo-26309 [https://bugs.python.org/issue?@action=redirect&bpo=26309]: In the "socketserver" module, shut down the request (closing the connected socket) when verify_request() returns false. Patch by Aviv Palivoda.
- bpo-23430 [https://bugs.python.org/issue?@action=redirect&bpo=23430]: Change the socketserver module to only catch exceptions raised from a request handler that are derived from Exception (instead of BaseException). Therefore SystemExit and KeyboardInterrupt no longer trigger the handle_error() method, and will now to stop a single-threaded server.
- bpo-25995 [https://bugs.python.org/issue?@action=redirect&bpo=25995]: os.walk() no longer uses FDs proportional to the tree depth.
- bpo-25994 [https://bugs.python.org/issue?@action=redirect&bpo=25994]: Added the close() method and the support of the context manager protocol for the os.scandir() iterator.
- bpo-23992 [https://bugs.python.org/issue?@action=redirect&bpo=23992]: multiprocessing: make MapResult not fail-fast upon exception.
- bpo-26243 [https://bugs.python.org/issue?@action=redirect&bpo=26243]: Support keyword arguments to zlib.compress(). Patch by Aviv Palivoda.
- bpo-26117 [https://bugs.python.org/issue?@action=redirect&bpo=26117]: The os.scandir() iterator now closes file descriptor not only when the iteration is finished, but when it was failed with error.
- bpo-25949 [https://bugs.python.org/issue?@action=redirect&bpo=25949]: __dict__ for an OrderedDict instance is now created only when needed.
- bpo-25911 [https://bugs.python.org/issue?@action=redirect&bpo=25911]: Restored support of bytes paths in os.walk() on Windows.

- bpo-26045 [https://bugs.python.org/issue?@action=redirect&bpo=26045]: Add UTF-8 suggestion to error message when posting a non-Latin-1 string with http.client.
- bpo-26039 [https://bugs.python.org/issue?@action=redirect&bpo=26039]: Added zipfile.ZipInfo.from_file() and zipinfo.ZipInfo.is_dir(). Patch by Thomas Kluyver.
- bpo-12923 [https://bugs.python.org/issue?@action=redirect&bpo=12923]: Reset FancyURLopener's redirect counter even if there is an exception. Based on patches by Brian Brazil and Daniel Rocco.
- bpo-25945 [https://bugs.python.org/issue?@action=redirect&bpo=25945]: Fixed a crash when unpickle the functools.partial object with wrong state. Fixed a leak in failed functools.partial constructor. "args" and "keywords" attributes of functools.partial have now always types tuple and dict correspondingly.
- bpo-26202 [https://bugs.python.org/issue?@action=redirect&bpo=26202]: copy.deepcopy() now correctly copies range() objects with non-atomic attributes.
- bpo-23076 [https://bugs.python.org/issue?@action=redirect&bpo=23076]: Path.glob() now raises a ValueError if it's called with an invalid pattern. Patch by Thomas Nyberg.
- bpo-19883 [https://bugs.python.org/issue?@action=redirect&bpo=19883]: Fixed possible integer overflows in zipimport.
- bpo-26227 [https://bugs.python.org/issue?@action=redirect&bpo=26227]: On Windows, getnameinfo(), gethostbyaddr() and gethostbyname_ex() functions of the socket module now decode the hostname from the ANSI code page rather than UTF-8.
- bpo-26099 [https://bugs.python.org/issue?@action=redirect&bpo=26099]: The site module now writes an error into stderr if sitecustomize module can be imported but executing the module raise an ImportError. Same change for usercustomize.
- bpo-26147 [https://bugs.python.org/issue?@action=redirect&bpo=26147]: xmlrpc now works with strings not encodable with used non-UTF-8 encoding.
- bpo-25935 [https://bugs.python.org/issue?@action=redirect&bpo=25935]: Garbage collector now breaks reference loops with OrderedDict.
- bpo-16620 [https://bugs.python.org/issue?@action=redirect&bpo=16620]: Fixed AttributeError in msilib.Directory.glob().

- bpo-26013 [https://bugs.python.org/issue?@action=redirect&bpo=26013]: Added compatibility with broken protocol 2 pickles created in old Python 3 versions (3.4.3 and lower).
- bpo-26129 [https://bugs.python.org/issue?@action=redirect&bpo=26129]: Deprecated accepting non-integers in grp.getgrgid().
- bpo-25850 [https://bugs.python.org/issue?@action=redirect&bpo=25850]: Use cross-compilation by default for 64-bit Windows.
- bpo-25822 [https://bugs.python.org/issue?@action=redirect&bpo=25822]: Add docstrings to the fields of urllib.parse results. Patch contributed by Swati Jaiswal.
- bpo-22642 [https://bugs.python.org/issue?@action=redirect&bpo=22642]: Convert trace module option parsing mechanism to argparse. Patch contributed by SilentGhost.
- bpo-24705 [https://bugs.python.org/issue?@action=redirect&bpo=24705]: Fix sysconfig._parse_makefile not expanding ${} vars appearing before $() vars.
- bpo-26069 [https://bugs.python.org/issue?@action=redirect&bpo=26069]: Remove the deprecated apis in the trace module.
- bpo-22138 [https://bugs.python.org/issue?@action=redirect&bpo=22138]: Fix mock.patch behavior when patching descriptors. Restore original values after patching. Patch contributed by Sean McCully.
- bpo-25672 [https://bugs.python.org/issue?@action=redirect&bpo=25672]: In the ssl module, enable the SSL_MODE_RELEASE_BUFFERS mode option if it is safe to do so.
- bpo-26012 [https://bugs.python.org/issue?@action=redirect&bpo=26012]: Don’t traverse into symlinks for `**` pattern in pathlib.Path.[r]glob().
- bpo-24120 [https://bugs.python.org/issue?@action=redirect&bpo=24120]: Ignore PermissionError when traversing a tree with pathlib.Path.[r]glob(). Patch by Ulrich Petri.
- bpo-21815 [https://bugs.python.org/issue?@action=redirect&bpo=21815]: Accept] characters in the data portion of imap responses, in order to handle the flags with square brackets accepted and produced by servers such as gmail.
- bpo-25447 [https://bugs.python.org/issue?@action=redirect&bpo=25447]: fileinput now uses sys.stdin as-is if it does not have a buffer attribute (restores backward compatibility).

- bpo-25971 [https://bugs.python.org/issue?@action=redirect&bpo=25971]: Optimized creating Fractions from floats by 2 times and from Decimals by 3 times.
- bpo-25802 [https://bugs.python.org/issue?@action=redirect&bpo=25802]: Document as deprecated the remaining implementations of importlib.abc.Loader.load_module().
- bpo-25928 [https://bugs.python.org/issue?@action=redirect&bpo=25928]: Add Decimal.as_integer_ratio().
- bpo-25447 [https://bugs.python.org/issue?@action=redirect&bpo=25447]: Copying the lru_cache() wrapper object now always works, independently from the type of the wrapped object (by returning the original object unchanged).
- bpo-25768 [https://bugs.python.org/issue?@action=redirect&bpo=25768]: Have the functions in compileall return booleans instead of ints and add proper documentation and tests for the return values.
- bpo-24103 [https://bugs.python.org/issue?@action=redirect&bpo=24103]: Fixed possible use after free in ElementTree.XMLPullParser.
- bpo-25860 [https://bugs.python.org/issue?@action=redirect&bpo=25860]: os.fwalk() no longer skips remaining directories when error occurs. Original patch by Samson Lee.
- bpo-25914 [https://bugs.python.org/issue?@action=redirect&bpo=25914]: Fixed and simplified OrderedDict.__sizeof__.
- bpo-25869 [https://bugs.python.org/issue?@action=redirect&bpo=25869]: Optimized deepcopying ElementTree; it is now 20 times faster.
- bpo-25873 [https://bugs.python.org/issue?@action=redirect&bpo=25873]: Optimized iterating ElementTree. Iterating elements Element.iter() is now 40% faster, iterating text Element.itertext() is now up to 2.5 times faster.
- bpo-25902 [https://bugs.python.org/issue?@action=redirect&bpo=25902]: Fixed various refcount issues in ElementTree iteration.
- bpo-22227 [https://bugs.python.org/issue?@action=redirect&bpo=22227]: The TarFile iterator is reimplemented using generator. This implementation is simpler that using class.
- bpo-25638 [https://bugs.python.org/issue?@action=redirect&bpo=25638]: Optimized ElementTree.iterparse(); it is now 2x faster. Optimized ElementTree parsing; it is now 10% faster.
- bpo-25761 [https://bugs.python.org/issue?@action=redirect&bpo=25761]: Improved detecting errors in broken pickle data.

- bpo-25717 [https://bugs.python.org/issue?@action=redirect&bpo=25717]: Restore the previous behaviour of tolerating most fstat() errors when opening files. This was a regression in 3.5a1, and stopped anonymous temporary files from working in special cases.
- bpo-24903 [https://bugs.python.org/issue?@action=redirect&bpo=24903]: Fix regression in number of arguments compileall accepts when '-d' is specified. The check on the number of arguments has been dropped completely as it never worked correctly anyway.
- bpo-25764 [https://bugs.python.org/issue?@action=redirect&bpo=25764]: In the subprocess module, preserve any exception caused by fork() failure when preexec_fn is used.
- bpo-25771 [https://bugs.python.org/issue?@action=redirect&bpo=25771]: Tweak the exception message for importlib.util.resolve_name() when 'package' isn't specified but necessary.
- bpo-6478 [https://bugs.python.org/issue?@action=redirect&bpo=6478]: _strptime's regexp cache now is reset after changing timezone with time.tzset().
- bpo-14285 [https://bugs.python.org/issue?@action=redirect&bpo=14285]: When executing a package with the "python -m package" option, and package initialization fails, a proper traceback is now reported. The "runpy" module now lets exceptions from package initialization pass back to the caller, rather than raising ImportError.
- bpo-19771 [https://bugs.python.org/issue?@action=redirect&bpo=19771]: Also in runpy and the "-m" option, omit the irrelevant message "… is a package and cannot be directly executed" if the package could not even be initialized (e.g. due to a bad `*.pyc` file).
- bpo-25177 [https://bugs.python.org/issue?@action=redirect&bpo=25177]: Fixed problem with the mean of very small and very large numbers. As a side effect, statistics.mean and statistics.variance should be significantly faster.
- bpo-25718 [https://bugs.python.org/issue?@action=redirect&bpo=25718]: Fixed copying object with state with boolean value is false.
- bpo-10131 [https://bugs.python.org/issue?@action=redirect&bpo=10131]: Fixed deep copying of minidom documents. Based on patch by Marian Ganisin.
- bpo-7990 [https://bugs.python.org/issue?@action=redirect&bpo=7990]: dir() on ElementTree.Element now lists properties: "tag", "text", "tail" and "attrib". Original patch by Santoso Wijaya.

- bpo-25725 [https://bugs.python.org/issue?@action=redirect&bpo=25725]: Fixed a reference leak in pickle.loads() when unpickling invalid data including tuple instructions.
- bpo-25663 [https://bugs.python.org/issue?@action=redirect&bpo=25663]: In the Readline completer, avoid listing duplicate global names, and search the global namespace before searching builtins.
- bpo-25688 [https://bugs.python.org/issue?@action=redirect&bpo=25688]: Fixed file leak in ElementTree.iterparse() raising an error.
- bpo-23914 [https://bugs.python.org/issue?@action=redirect&bpo=23914]: Fixed SystemError raised by unpickler on broken pickle data.
- bpo-25691 [https://bugs.python.org/issue?@action=redirect&bpo=25691]: Fixed crash on deleting ElementTree.Element attributes.
- bpo-25624 [https://bugs.python.org/issue?@action=redirect&bpo=25624]: ZipFile now always writes a ZIP_STORED header for directory entries. Patch by Dingyuan Wang.
- bpo-25626 [https://bugs.python.org/issue?@action=redirect&bpo=25626]: Change three zlib functions to accept sizes that fit in Py_ssize_t, but internally cap those sizes to UINT_MAX. This resolves a regression in 3.5 where GzipFile.read() failed to read chunks larger than 2 or 4 GiB. The change affects the zlib.Decompress.decompress() max_length parameter, the zlib.decompress() bufsize parameter, and the zlib.Decompress.flush() length parameter.
- bpo-25583 [https://bugs.python.org/issue?@action=redirect&bpo=25583]: Avoid incorrect errors raised by os.makedirs(exist_ok=True) when the OS gives priority to errors such as EACCES over EEXIST.
- bpo-25593 [https://bugs.python.org/issue?@action=redirect&bpo=25593]: Change semantics of EventLoop.stop() in asyncio.
- bpo-6973 [https://bugs.python.org/issue?@action=redirect&bpo=6973]: When we know a subprocess.Popen process has died, do not allow the send_signal(), terminate(), or kill() methods to do anything as they could potentially signal a different process.
- bpo-23883 [https://bugs.python.org/issue?@action=redirect&bpo=23883]: Added missing APIs to __all__ to match the documented APIs for the following modules: calendar, csv, enum, fileinput, ftplib, logging, optparse, tarfile, threading and wave. Also added a test.support.check__all__() helper. Patches by Jacek Kołodziej, Mauro S. M. Rodrigues and Joel Taddei.

- bpo-25590 [https://bugs.python.org/issue?@action=redirect&bpo=25590]: In the Readline completer, only call getattr() once per attribute. Also complete names of attributes such as properties and slots which are listed by dir() but not yet created on an instance.
- bpo-25498 [https://bugs.python.org/issue?@action=redirect&bpo=25498]: Fix a crash when garbage-collecting ctypes objects created by wrapping a memoryview. This was a regression made in 3.5a1. Based on patch by Eryksun.
- bpo-25584 [https://bugs.python.org/issue?@action=redirect&bpo=25584]: Added “escape” to the __all__ list in the glob module.
- bpo-25584 [https://bugs.python.org/issue?@action=redirect&bpo=25584]: Fixed recursive glob() with patterns starting with `**`.
- bpo-25446 [https://bugs.python.org/issue?@action=redirect&bpo=25446]: Fix regression in smtplib’s AUTH LOGIN support.
- bpo-18010 [https://bugs.python.org/issue?@action=redirect&bpo=18010]: Fix the pydoc web server’s module search function to handle exceptions from importing packages.
- bpo-25554 [https://bugs.python.org/issue?@action=redirect&bpo=25554]: Got rid of circular references in regular expression parsing.
- bpo-18973 [https://bugs.python.org/issue?@action=redirect&bpo=18973]: Command-line interface of the calendar module now uses argparse instead of optparse.
- bpo-25510 [https://bugs.python.org/issue?@action=redirect&bpo=25510]: fileinput.FileInput.readline() now returns b’’ instead of ‘’ at the end if the FileInput was opened with binary mode. Patch by Ryosuke Ito.
- bpo-25503 [https://bugs.python.org/issue?@action=redirect&bpo=25503]: Fixed inspect.getdoc() for inherited docstrings of properties. Original patch by John Mark Vandenberg.
- bpo-25515 [https://bugs.python.org/issue?@action=redirect&bpo=25515]: Always use os.urandom as a source of randomness in uuid.uuid4.
- bpo-21827 [https://bugs.python.org/issue?@action=redirect&bpo=21827]: Corrigido textwrap.dedent() para o caso em que o maior espaço em branco comum seja uma substring do menor espaço em branco inicial. Com base no *patch* de Robert Li.
- bpo-25447 [https://bugs.python.org/issue?@action=redirect&bpo=25447]: The lru_cache() wrapper objects now can be copied and pickled (by returning the original object unchanged).

- bpo-25390 [https://bugs.python.org/issue?@action=redirect&bpo=25390]: typing: Don't crash on Union[str, Pattern].
- bpo-25441 [https://bugs.python.org/issue?@action=redirect&bpo=25441]: asyncio: Raise error from drain() when socket is closed.
- bpo-25410 [https://bugs.python.org/issue?@action=redirect&bpo=25410]: Cleaned up and fixed minor bugs in C implementation of OrderedDict.
- bpo-25411 [https://bugs.python.org/issue?@action=redirect&bpo=25411]: Improved Unicode support in SMTPHandler through better use of the email package. Thanks to user simon04 for the patch.
- Move the imp module from a PendingDeprecationWarning to DeprecationWarning.
- bpo-25407 [https://bugs.python.org/issue?@action=redirect&bpo=25407]: Remove mentions of the formatter module being removed in Python 3.6.
- bpo-25406 [https://bugs.python.org/issue?@action=redirect&bpo=25406]: Fixed a bug in C implementation of OrderedDict.move_to_end() that caused segmentation fault or hang in iterating after moving several items to the start of ordered dict.
- bpo-25382 [https://bugs.python.org/issue?@action=redirect&bpo=25382]: pickletools.dis() now outputs implicit memo index for the MEMOIZE opcode.
- bpo-25357 [https://bugs.python.org/issue?@action=redirect&bpo=25357]: Add an optional newline parameter to binascii.b2a_base64(). base64.b64encode() uses it to avoid a memory copy.
- bpo-24164 [https://bugs.python.org/issue?@action=redirect&bpo=24164]: Objects that need calling `__new__` with keyword arguments, can now be pickled using pickle protocols older than protocol version 4.
- bpo-25364 [https://bugs.python.org/issue?@action=redirect&bpo=25364]: zipfile now works in threads disabled builds.
- bpo-25328 [https://bugs.python.org/issue?@action=redirect&bpo=25328]: smtpd's SMTPChannel now correctly raises a ValueError if both decode_data and enable_SMTPUTF8 are set to true.
- bpo-16099 [https://bugs.python.org/issue?@action=redirect&bpo=16099]: RobotFileParser now supports Crawl-delay and Request-rate extensions. Patch by Nikolay Bogoychev.
- bpo-25316 [https://bugs.python.org/issue?@action=redirect&bpo=25316]: distutils raises OSError instead of DistutilsPlatformError when MSVC is not installed.

- bpo-25380 [https://bugs.python.org/issue?@action=redirect&bpo=25380]: Fixed protocol for the STACK_GLOBAL opcode in pickletools.opcodes.
- bpo-23972 [https://bugs.python.org/issue?@action=redirect&bpo=23972]: Updates asyncio datagram create method allowing reuseport and reuseaddr socket options to be set prior to binding the socket. Mirroring the existing asyncio create_server method the reuseaddr option for datagram sockets defaults to True if the O/S is 'posix' (except if the platform is Cygwin). Patch by Chris Laws.
- bpo-25304 [https://bugs.python.org/issue?@action=redirect&bpo=25304]: Add asyncio.run_coroutine_threadsafe(). This lets you submit a coroutine to a loop from another thread, returning a concurrent.futures.Future. By Vincent Michel.
- bpo-25232 [https://bugs.python.org/issue?@action=redirect&bpo=25232]: Fix CGIRequestHandler to split the query from the URL at the first question mark (?) rather than the last. Patch from Xiang Zhang.
- bpo-24657 [https://bugs.python.org/issue?@action=redirect&bpo=24657]: Prevent CGIRequestHandler from collapsing slashes in the query part of the URL as if it were a path. Patch from Xiang Zhang.
- bpo-25287 [https://bugs.python.org/issue?@action=redirect&bpo=25287]: Don't add crypt.METHOD_CRYPT to crypt.methods if it's not supported. Check if it is supported, it may not be supported on OpenBSD for example.
- bpo-23600 [https://bugs.python.org/issue?@action=redirect&bpo=23600]: Default implementation of tzinfo.fromutc() was returning wrong results in some cases.
- bpo-25203 [https://bugs.python.org/issue?@action=redirect&bpo=25203]: Failed readline.set_completer_delims() no longer left the module in inconsistent state.
- bpo-25011 [https://bugs.python.org/issue?@action=redirect&bpo=25011]: rlcompleter now omits private and special attribute names unless the prefix starts with underscores.
- bpo-25209 [https://bugs.python.org/issue?@action=redirect&bpo=25209]: rlcompleter now can add a space or a colon after completed keyword.
- bpo-22241 [https://bugs.python.org/issue?@action=redirect&bpo=22241]: timezone.utc name is now plain 'UTC', not 'UTC-00:00'.
- bpo-23517 [https://bugs.python.org/issue?@action=redirect&bpo=23517]: fromtimestamp() and utcfromtimestamp() methods of datetime.datetime now round microseconds to nearest with ties going

to nearest even integer (ROUND_HALF_EVEN), as round(float), instead of rounding towards -Infinity (ROUND_FLOOR).

- bpo-23552 [https://bugs.python.org/issue?@action=redirect&bpo=23552]: Timeit now warns when there is substantial (4x) variance between best and worst times. Patch from Serhiy Storchaka.
- bpo-24633 [https://bugs.python.org/issue?@action=redirect&bpo=24633]: site-packages/README -> README.txt.
- bpo-24879 [https://bugs.python.org/issue?@action=redirect&bpo=24879]: help() and pydoc can now list named tuple fields in the order they were defined rather than alphabetically. The ordering is determined by the _fields attribute if present.
- bpo-24874 [https://bugs.python.org/issue?@action=redirect&bpo=24874]: Improve speed of itertools.cycle() and make its pickle more compact.
- Fix crash in itertools.cycle.__setstate__() when the first argument wasn't a list.
- bpo-20059 [https://bugs.python.org/issue?@action=redirect&bpo=20059]: urllib.parse raises ValueError on all invalid ports. Patch by Martin Panter.
- bpo-24360 [https://bugs.python.org/issue?@action=redirect&bpo=24360]: Improve __repr__ of argparse.Namespace() for invalid identifiers. Patch by Matthias Bussonnier.
- bpo-23426 [https://bugs.python.org/issue?@action=redirect&bpo=23426]: run_setup was broken in distutils. Patch from Alexander Belopolsky.
- bpo-13938 [https://bugs.python.org/issue?@action=redirect&bpo=13938]: 2to3 converts StringTypes to a tuple. Patch from Mark Hammond.
- bpo-2091 [https://bugs.python.org/issue?@action=redirect&bpo=2091]: open() accepted a 'U' mode string containing '+', but 'U' can only be used with 'r'. Patch from Jeff Balogh and John O'Connor.
- bpo-8585 [https://bugs.python.org/issue?@action=redirect&bpo=8585]: improved tests for zipimporter2. Patch from Mark Lawrence.
- bpo-18622 [https://bugs.python.org/issue?@action=redirect&bpo=18622]: unittest.mock.mock_open().reset_mock would recurse infinitely. Patch from Nicola Palumbo and Laurent De Buyst.
- bpo-24426 [https://bugs.python.org/issue?@action=redirect&bpo=24426]: Fast searching optimization in regular expressions now works for patterns that starts with capturing groups. Fast searching optimization now can't be disabled at compile time.

- bpo-23661 [https://bugs.python.org/issue?@action=redirect&bpo=23661]: unittest.mock side_effects can now be exceptions again. This was a regression vs Python 3.4. Patch from Ignacio Rossi
- bpo-13248 [https://bugs.python.org/issue?@action=redirect&bpo=13248]: Remove deprecated inspect.getmoduleinfo function.
- bpo-25578 [https://bugs.python.org/issue?@action=redirect&bpo=25578]: Fix (another) memory leak in SSLSocket.getpeercer().
- bpo-25530 [https://bugs.python.org/issue?@action=redirect&bpo=25530]: Disable the vulnerable SSLv3 protocol by default when creating ssl.SSLContext.
- bpo-25569 [https://bugs.python.org/issue?@action=redirect&bpo=25569]: Fix memory leak in SSLSocket.getpeercert().
- bpo-25471 [https://bugs.python.org/issue?@action=redirect&bpo=25471]: Sockets returned from accept() shouldn't appear to be nonblocking.
- bpo-25319 [https://bugs.python.org/issue?@action=redirect&bpo=25319]: When threading.Event is reinitialized, the underlying condition should use a regular lock rather than a recursive lock.
- Skip getaddrinfo if host is already resolved. Patch by A. Jesse Jiryu Davis.
- bpo-26050 [https://bugs.python.org/issue?@action=redirect&bpo=26050]: Add asyncio.StreamReader.readuntil() method. Patch by Марк Коренберг.
- bpo-25924 [https://bugs.python.org/issue?@action=redirect&bpo=25924]: Avoid unnecessary serialization of getaddrinfo(3) calls on OS X versions 10.5 or higher. Original patch by A. Jesse Jiryu Davis.
- bpo-26406 [https://bugs.python.org/issue?@action=redirect&bpo=26406]: Avoid unnecessary serialization of getaddrinfo(3) calls on current versions of OpenBSD and NetBSD. Patch by A. Jesse Jiryu Davis.
- bpo-26848 [https://bugs.python.org/issue?@action=redirect&bpo=26848]: Fix asyncio/subprocess.communicate() to handle empty input. Patch by Jack O'Connor.
- bpo-27040 [https://bugs.python.org/issue?@action=redirect&bpo=27040]: Add loop.get_exception_handler method
- bpo-27041 [https://bugs.python.org/issue?@action=redirect&bpo=27041]: asyncio: Add loop.create_future method

IDLE

- bpo-20640 [https://bugs.python.org/issue?@action=redirect&bpo=20640]: Add tests for idlelib.configHelpSourceEdit. Patch by Saimadhav Heblikar.
- In the ‘IDLE-console differences’ section of the IDLE doc, clarify how running with IDLE affects sys.modules and the standard streams.
- bpo-25507 [https://bugs.python.org/issue?@action=redirect&bpo=25507]: fix incorrect change in IOBinding that prevented printing. Augment IOBinding htest to include all major IOBinding functions.
- bpo-25905 [https://bugs.python.org/issue?@action=redirect&bpo=25905]: Revert unwanted conversion of ‘ to ’ RIGHT SINGLE QUOTATION MARK in README.txt and open this and NEWS.txt with ‘ascii’. Re-encode CREDITS.txt to utf-8 and open it with ‘utf-8’.
- bpo-15348 [https://bugs.python.org/issue?@action=redirect&bpo=15348]: Stop the debugger engine (normally in a user process) before closing the debugger window (running in the IDLE process). This prevents the RuntimeErrors that were being caught and ignored.
- bpo-24455 [https://bugs.python.org/issue?@action=redirect&bpo=24455]: Prevent IDLE from hanging when a) closing the shell while the debugger is active (15347); b) closing the debugger with the [X] button (15348); and c) activating the debugger when already active (24455). The patch by Mark Roseman does this by making two changes. 1. Suspend and resume the gui.interaction method with the tcl vwait mechanism intended for this purpose (instead of root.mainloop & .quit). 2. In gui.run, allow any existing interaction to terminate first.
- Change ‘The program’ to ‘Your program’ in an IDLE ‘kill program?’ message to make it clearer that the program referred to is the currently running user program, not IDLE itself.
- bpo-24750 [https://bugs.python.org/issue?@action=redirect&bpo=24750]: Improve the appearance of the IDLE editor window status bar. Patch by Mark Roseman.
- bpo-25313 [https://bugs.python.org/issue?@action=redirect&bpo=25313]: Change the handling of new built-in text color themes to better address the compatibility problem introduced by the addition of IDLE Dark. Consistently use the revised idleConf.CurrentTheme everywhere in idlelib.
- bpo-24782 [https://bugs.python.org/issue?@action=redirect&bpo=24782]: Extension configuration is now a tab in the IDLE Preferences dialog rather than a

separate dialog. The former tabs are now a sorted list. Patch by Mark Roseman.

- bpo-22726 [https://bugs.python.org/issue?@action=redirect&bpo=22726]: Re-activate the config dialog help button with some content about the other buttons and the new IDLE Dark theme.
- bpo-24820 [https://bugs.python.org/issue?@action=redirect&bpo=24820]: IDLE now has an 'IDLE Dark' built-in text color theme. It is more or less IDLE Classic inverted, with a cobalt blue background. Strings, comments, keywords, … are still green, red, orange, … . To use it with IDLEs released before November 2015, hit the 'Save as New Custom Theme' button and enter a new name, such as 'Custom Dark'. The custom theme will work with any IDLE release, and can be modified.
- bpo-25224 [https://bugs.python.org/issue?@action=redirect&bpo=25224]: README.txt is now an idlelib index for IDLE developers and curious users. The previous user content is now in the IDLE doc chapter. 'IDLE' now means 'Integrated Development and Learning Environment'.
- bpo-24820 [https://bugs.python.org/issue?@action=redirect&bpo=24820]: Users can now set breakpoint colors in Settings -> Custom Highlighting. Original patch by Mark Roseman.
- bpo-24972 [https://bugs.python.org/issue?@action=redirect&bpo=24972]: Inactive selection background now matches active selection background, as configured by users, on all systems. Found items are now always highlighted on Windows. Initial patch by Mark Roseman.
- bpo-24570 [https://bugs.python.org/issue?@action=redirect&bpo=24570]: Idle: make calltip and completion boxes appear on Macs affected by a tk regression. Initial patch by Mark Roseman.
- bpo-24988 [https://bugs.python.org/issue?@action=redirect&bpo=24988]: Idle ScrolledList context menus (used in debugger) now work on Mac Aqua. Patch by Mark Roseman.
- bpo-24801 [https://bugs.python.org/issue?@action=redirect&bpo=24801]: Make right-click for context menu work on Mac Aqua. Patch by Mark Roseman.
- bpo-25173 [https://bugs.python.org/issue?@action=redirect&bpo=25173]: Associate tkinter messageboxes with a specific widget. For Mac OSX, make them a 'sheet'. Patch by Mark Roseman.
- bpo-25198 [https://bugs.python.org/issue?@action=redirect&bpo=25198]: Enhance the initial html viewer now used for Idle Help. Properly indent fixed-

pitch text (patch by Mark Roseman). Give code snippet a very Sphinx-like light blueish-gray background. Re-use initial width and height set by users for shell and editor. When the Table of Contents (TOC) menu is used, put the section header at the top of the screen.

- bpo-25225 [https://bugs.python.org/issue?@action=redirect&bpo=25225]: Condense and rewrite Idle doc section on text colors.
- bpo-21995 [https://bugs.python.org/issue?@action=redirect&bpo=21995]: Explain some differences between IDLE and console Python.
- bpo-22820 [https://bugs.python.org/issue?@action=redirect&bpo=22820]: Explain need for *print* when running file from Idle editor.
- bpo-25224 [https://bugs.python.org/issue?@action=redirect&bpo=25224]: Doc: augment Idle feature list and no-subprocess section.
- bpo-25219 [https://bugs.python.org/issue?@action=redirect&bpo=25219]: Update doc for Idle command line options. Some were missing and notes were not correct.
- bpo-24861 [https://bugs.python.org/issue?@action=redirect&bpo=24861]: Most of idlelib is private and subject to change. Use idleib.idle.* to start Idle. See idlelib.__init__.__doc__.
- bpo-25199 [https://bugs.python.org/issue?@action=redirect&bpo=25199]: Idle: add synchronization comments for future maintainers.
- bpo-16893 [https://bugs.python.org/issue?@action=redirect&bpo=16893]: Replace help.txt with help.html for Idle doc display. The new idlelib/help.html is rstripped Doc/build/html/library/idle.html. It looks better than help.txt and will better document Idle as released. The tkinter html viewer that works for this file was written by Rose Roseman. The now unused EditorWindow.HelpDialog class and helt.txt file are deprecated.
- bpo-24199 [https://bugs.python.org/issue?@action=redirect&bpo=24199]: Deprecate unused idlelib.idlever with possible removal in 3.6.
- bpo-24790 [https://bugs.python.org/issue?@action=redirect&bpo=24790]: Remove extraneous code (which also create 2 & 3 conflicts).

Documentação

- bpo-26736 [https://bugs.python.org/issue?@action=redirect&bpo=26736]: Used HTTPS for external links in the documentation if possible.

- bpo-6953 [https://bugs.python.org/issue?@action=redirect&bpo=6953]: Rework the Readline module documentation to group related functions together, and add more details such as what underlying Readline functions and variables are accessed.
- bpo-23606 [https://bugs.python.org/issue?@action=redirect&bpo=23606]: Adds note to ctypes documentation regarding cdll.msvcrt.
- bpo-24952 [https://bugs.python.org/issue?@action=redirect&bpo=24952]: Clarify the default size argument of stack_size() in the “threading” and “_thread” modules. Patch from Mattip.
- bpo-26014 [https://bugs.python.org/issue?@action=redirect&bpo=26014]: Update 3.x packaging documentation: * “See also” links to the new docs are now provided in the legacy pages * links to setuptools documentation have been updated

Testes

- bpo-21916 [https://bugs.python.org/issue?@action=redirect&bpo=21916]: Added tests for the turtle module. Patch by ingrid, Gregory Loyse and Jelle Zijlstra.
- bpo-26295 [https://bugs.python.org/issue?@action=redirect&bpo=26295]: When using “python3 -m test –testdir=TESTDIR”, regrtest doesn’t add “test.” prefix to test module names.
- bpo-26523 [https://bugs.python.org/issue?@action=redirect&bpo=26523]: The multiprocessing thread pool (multiprocessing.dummy.Pool) was untested.
- bpo-26015 [https://bugs.python.org/issue?@action=redirect&bpo=26015]: Added new tests for pickling iterators of mutable sequences.
- bpo-26325 [https://bugs.python.org/issue?@action=redirect&bpo=26325]: Added test.support.check_no_resource_warning() to check that no ResourceWarning is emitted.
- bpo-25940 [https://bugs.python.org/issue?@action=redirect&bpo=25940]: Changed test_ssl to use its internal local server more. This avoids relying on svn.python.org, which recently changed root certificate.
- bpo-25616 [https://bugs.python.org/issue?@action=redirect&bpo=25616]: Tests for OrderedDict are extracted from test_collections into separate file test_ordered_dict.

- bpo-25449 [https://bugs.python.org/issue?@action=redirect&bpo=25449]: Added tests for OrderedDict subclasses.
- bpo-25188 [https://bugs.python.org/issue?@action=redirect&bpo=25188]: Add -P/–pgo to test.regrtest to suppress error output when running the test suite for the purposes of a PGO build. Initial patch by Alecsandru Patrascu.
- bpo-22806 [https://bugs.python.org/issue?@action=redirect&bpo=22806]: Add `python -m test --list-tests` command to list tests.
- bpo-18174 [https://bugs.python.org/issue?@action=redirect&bpo=18174]: `python -m test --huntrleaks ...` now also checks for leak of file descriptors. Patch written by Richard Oudkerk.
- bpo-25260 [https://bugs.python.org/issue?@action=redirect&bpo=25260]: Fix `python -m test --coverage` on Windows. Remove the list of ignored directories.
- `PCbuild\rt.bat` now accepts an unlimited number of arguments to pass along to regrtest.py. Previously there was a limit of 9.
- bpo-26583 [https://bugs.python.org/issue?@action=redirect&bpo=26583]: Skip test_timestamp_overflow in test_import if bytecode files cannot be written.

Construção

- bpo-21277 [https://bugs.python.org/issue?@action=redirect&bpo=21277]: Don’t try to link _ctypes with a ffi_convenience library.
- bpo-26884 [https://bugs.python.org/issue?@action=redirect&bpo=26884]: Fix linking extension modules for cross builds. Patch by Xavier de Gaye.
- bpo-26932 [https://bugs.python.org/issue?@action=redirect&bpo=26932]: Fixed support of RTLD_* constants defined as enum values, not via macros (in particular on Android). Patch by Chi Hsuan Yen.
- bpo-22359 [https://bugs.python.org/issue?@action=redirect&bpo=22359]: Disable the rules for running _freeze_importlib and pgen when cross-compiling. The output of these programs is normally saved with the source code anyway, and is still regenerated when doing a native build. Patch by Xavier de Gaye.
- bpo-21668 [https://bugs.python.org/issue?@action=redirect&bpo=21668]: Link audioop, _datetime, _ctypes_test modules to libm, except on Mac OS X. Patch written by Chi Hsuan Yen.

- bpo-25702 [https://bugs.python.org/issue?@action=redirect&bpo=25702]: A –with-lto configure option has been added that will enable link time optimizations at build time during a make profile-opt. Some compilers and toolchains are known to not produce stable code when using LTO, be sure to test things thoroughly before relying on it. It can provide a few % speed up over profile-opt alone.
- bpo-26624 [https://bugs.python.org/issue?@action=redirect&bpo=26624]: Adds validation of ucrtbase[d].dll version with warning for old versions.
- bpo-17603 [https://bugs.python.org/issue?@action=redirect&bpo=17603]: Avoid error about nonexistent fileblocks.o file by using a lower-level check for st_blocks in struct stat.
- bpo-26079 [https://bugs.python.org/issue?@action=redirect&bpo=26079]: Fixing the build output folder for tix-8.4.3.6. Patch by Bjoern Thiel.
- bpo-26465 [https://bugs.python.org/issue?@action=redirect&bpo=26465]: Update Windows builds to use OpenSSL 1.0.2g.
- bpo-25348 [https://bugs.python.org/issue?@action=redirect&bpo=25348]: Added `--pgo` and `--pgo-job` arguments to `PCbuild\build.bat` for building with Profile-Guided Optimization. The old `PCbuild\build_pgo.bat` script is removed.
- bpo-25827 [https://bugs.python.org/issue?@action=redirect&bpo=25827]: Add support for building with ICC to `configure`, including a new `--with-icc` flag.
- bpo-25696 [https://bugs.python.org/issue?@action=redirect&bpo=25696]: Fix installation of Python on UNIX with make -j9.
- bpo-24986 [https://bugs.python.org/issue?@action=redirect&bpo=24986]: It is now possible to build Python on Windows without errors when external libraries are not available.
- bpo-24421 [https://bugs.python.org/issue?@action=redirect&bpo=24421]: Compile Modules/_math.c once, before building extensions. Previously it could fail to compile properly if the math and cmath builds were concurrent.
- bpo-26465 [https://bugs.python.org/issue?@action=redirect&bpo=26465]: Update OS X 10.5+ 32-bit-only installer to build and link with OpenSSL 1.0.2g.
- bpo-26268 [https://bugs.python.org/issue?@action=redirect&bpo=26268]: Update Windows builds to use OpenSSL 1.0.2f.
- bpo-25136 [https://bugs.python.org/issue?@action=redirect&bpo=25136]: Support Apple Xcode 7’s new textual SDK stub libraries.

- bpo-24324 [https://bugs.python.org/issue?@action=redirect&bpo=24324]: Do not enable unreachable code warnings when using gcc as the option does not work correctly in older versions of gcc and has been silently removed as of gcc-4.5.

Windows

- bpo-27053 [https://bugs.python.org/issue?@action=redirect&bpo=27053]: Updates make_zip.py to correctly generate library ZIP file.
- bpo-26268 [https://bugs.python.org/issue?@action=redirect&bpo=26268]: Update the prepare_ssl.py script to handle OpenSSL releases that don't include the contents of the include directory (that is, 1.0.2e and later).
- bpo-26071 [https://bugs.python.org/issue?@action=redirect&bpo=26071]: bdist_wininst created binaries fail to start and find 32bit Python
- bpo-26073 [https://bugs.python.org/issue?@action=redirect&bpo=26073]: Update the list of magic numbers in launcher
- bpo-26065 [https://bugs.python.org/issue?@action=redirect&bpo=26065]: Excludes venv from library when generating embeddable distro.
- bpo-25022 [https://bugs.python.org/issue?@action=redirect&bpo=25022]: Removed very outdated PC/example_nt/ directory.

Ferramentas/Demos

- bpo-26799 [https://bugs.python.org/issue?@action=redirect&bpo=26799]: Fix python-gdb.py: don't get C types once when the Python code is loaded, but get C types on demand. The C types can change if python-gdb.py is loaded before the Python executable. Patch written by Thomas Ilsche.
- bpo-26271 [https://bugs.python.org/issue?@action=redirect&bpo=26271]: Fix the Freeze tool to properly use flags passed through configure. Patch by Daniel Shaulov.
- bpo-26489 [https://bugs.python.org/issue?@action=redirect&bpo=26489]: Add dictionary unpacking support to Tools/parser/unparse.py. Patch by Guo Ci Teo.
- bpo-26316 [https://bugs.python.org/issue?@action=redirect&bpo=26316]: Fix variable name typo in Argument Clinic.
- bpo-25440 [https://bugs.python.org/issue?@action=redirect&bpo=25440]: Fix output of python-config –extension-suffix.

- bpo-25154 [https://bugs.python.org/issue?@action=redirect&bpo=25154]: The pyvenv script has been deprecated in favour of `python3 -m venv`.

API C

- bpo-26312 [https://bugs.python.org/issue?@action=redirect&bpo=26312]: SystemError is now raised in all programming bugs with using PyArg_ParseTupleAndKeywords(). RuntimeError did raised before in some programming bugs.
- bpo-26198 [https://bugs.python.org/issue?@action=redirect&bpo=26198]: ValueError is now raised instead of TypeError on buffer overflow in parsing “es#” and “et#” format units. SystemError is now raised instead of TypeError on programmatical error in parsing format string.

Python 3.5.5 final

Data de lançamento: 2018-02-04

There were no new changes in version 3.5.5.

Python 3.5.5 release candidate 1

Data de lançamento: 2018-01-23

Segurança

- bpo-32551 [https://bugs.python.org/issue?@action=redirect&bpo=32551]: The `sys.path[0]` initialization change for bpo-29139 [https://bugs.python.org/issue?@action=redirect&bpo=29139] caused a regression by revealing an inconsistency in how sys.path is initialized when executing `__main__` from a zipfile, directory, or other import location. This is considered a potential security issue, as it may lead to privileged processes unexpectedly loading code from user controlled directories in situations where that was not previously the case. The interpreter now consistently avoids ever adding the import location’s parent directory to `sys.path`, and ensures no other `sys.path` entries

are inadvertently modified when inserting the import location named on the command line. (Originally reported as bpo-29723 [https://bugs.python.org/issue?@action=redirect&bpo=29723] against Python 3.6rc1, but it was missed at the time that the then upcoming Python 3.5.4 release would also be affected)

- bpo-30657 [https://bugs.python.org/issue?@action=redirect&bpo=30657]: Fixed possible integer overflow in PyBytes_DecodeEscape, **CVE 2017-1000158** [https://www.cve.org/CVERecord?id=CVE-2017-1000158]. Original patch by Jay Bosamiya; rebased to Python 3 by Miro Hrončok.
- bpo-30947 [https://bugs.python.org/issue?@action=redirect&bpo=30947]: Upgrade libexpat embedded copy from version 2.2.1 to 2.2.3 to get security fixes.

Núcleo e embutidos

- bpo-31095 [https://bugs.python.org/issue?@action=redirect&bpo=31095]: Fix potential crash during GC caused by `tp_dealloc` which doesn't call `PyObject_GC_UnTrack()`.

Biblioteca

- bpo-32072 [https://bugs.python.org/issue?@action=redirect&bpo=32072]: Fixed issues with binary plists: Fixed saving bytearrays. Identical objects will be saved only once. Equal references will be load as identical objects. Added support for saving and loading recursive data structures.
- bpo-31170 [https://bugs.python.org/issue?@action=redirect&bpo=31170]: expat: Update libexpat from 2.2.3 to 2.2.4. Fix copying of partial characters for UTF-8 input (libexpat bug 115): https://github.com/libexpat/libexpat/issues/115

Python 3.5.4 final

Data de lançamento: 2017-08-07

Biblioteca

- bpo-30119 [https://bugs.python.org/issue?@action=redirect&bpo=30119]: ftplib.FTP.putline() now throws ValueError on commands that contains CR or LF. Patch by Donghee Na.

Python 3.5.4 release candidate 1

Data de lançamento: 2017-07-23

Segurança

- bpo-30730 [https://bugs.python.org/issue?@action=redirect&bpo=30730]: Prevent environment variables injection in subprocess on Windows. Prevent passing other environment variables and command arguments.
- bpo-30694 [https://bugs.python.org/issue?@action=redirect&bpo=30694]: Upgrade expat copy from 2.2.0 to 2.2.1 to get fixes of multiple security vulnerabilities including: **CVE 2017-9233** [https://www.cve.org/CVERecord?id=CVE-2017-9233] (External entity infinite loop DoS), **CVE 2016-9063** [https://www.cve.org/CVERecord?id=CVE-2016-9063] (Integer overflow, re-fix), **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718] (Fix regression bugs from 2.2.0's fix to **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718]) and **CVE 2012-0876** [https://www.cve.org/CVERecord?id=CVE-2012-0876] (Counter hash flooding with SipHash). Note: the **CVE 2016-5300** [https://www.cve.org/CVERecord?id=CVE-2016-5300] (Use os-specific entropy sources like getrandom) doesn't impact Python, since Python already gets entropy from the OS to set the expat secret using `XML_SetHashSalt()`.
- bpo-30500 [https://bugs.python.org/issue?@action=redirect&bpo=30500]: Fix urllib.parse.splithost() to correctly parse fragments. For example, `splithost('//127.0.0.1#@evil.com/')` now correctly returns the `127.0.0.1` host, instead of treating `@evil.com` as the host in an authentication (`login@host`).
- bpo-29591 [https://bugs.python.org/issue?@action=redirect&bpo=29591]: Update expat copy from 2.1.1 to 2.2.0 to get fixes of **CVE 2016-0718** [https://www.cve.org/CVERecord?id=CVE-2016-0718] and **CVE 2016-4472** [https://www.cve.org/CVERecord?id=CVE-2016-4472]. See https://sourceforge.net/p/expat/bugs/537/ for more information.

Núcleo e embutidos

- bpo-30876 [https://bugs.python.org/issue?@action=redirect&bpo=30876]: Relative import from unloaded package now reimports the package instead of failing with SystemError. Relative import from non-package now fails with ImportError rather than SystemError.
- bpo-30765 [https://bugs.python.org/issue?@action=redirect&bpo=30765]: Avoid blocking in pthread_mutex_lock() when PyThread_acquire_lock() is asked not to block.
- bpo-27945 [https://bugs.python.org/issue?@action=redirect&bpo=27945]: Fixed various segfaults with dict when input collections are mutated during searching, inserting or comparing. Based on patches by Duane Griffin and Tim Mitchell.
- bpo-25794 [https://bugs.python.org/issue?@action=redirect&bpo=25794]: Fixed type.__setattr__() and type.__delattr__() for non-interned attribute names. Based on patch by Eryk Sun.
- bpo-29935 [https://bugs.python.org/issue?@action=redirect&bpo=29935]: Fixed error messages in the index() method of tuple, list and deque when pass indices of wrong type.
- bpo-28876 [https://bugs.python.org/issue?@action=redirect&bpo=28876]: `bool(range)` works even if `len(range)` raises **`OverflowError`**.
- bpo-29600 [https://bugs.python.org/issue?@action=redirect&bpo=29600]: Fix wrapping coroutine return values in StopIteration.
- bpo-29537 [https://bugs.python.org/issue?@action=redirect&bpo=29537]: Restore runtime compatibility with bytecode files generated by CPython 3.5.0 to 3.5.2, and adjust the eval loop to avoid the problems that could be caused by the malformed variant of the BUILD_MAP_UNPACK_WITH_CALL opcode that they may contain. Patch by Petr Viktorin, Serhiy Storchaka, and Nick Coghlan.
- bpo-28598 [https://bugs.python.org/issue?@action=redirect&bpo=28598]: Support __rmod__ for subclasses of str being called before str.__mod__. Patch by Martijn Pieters.
- bpo-29602 [https://bugs.python.org/issue?@action=redirect&bpo=29602]: Fix incorrect handling of signed zeros in complex constructor for complex subclasses and for inputs having a __complex__ method. Patch by Serhiy Storchaka.

- bpo-29347 [https://bugs.python.org/issue?@action=redirect&bpo=29347]: Fixed possibly dereferencing undefined pointers when creating weakref objects.
- bpo-29438 [https://bugs.python.org/issue?@action=redirect&bpo=29438]: Fixed use-after-free problem in key sharing dict.
- bpo-29319 [https://bugs.python.org/issue?@action=redirect&bpo=29319]: Prevent RunMainFromImporter overwriting sys.path[0].
- bpo-29337 [https://bugs.python.org/issue?@action=redirect&bpo=29337]: Fixed possible BytesWarning when compare the code objects. Warnings could be emitted at compile time.
- bpo-29478 [https://bugs.python.org/issue?@action=redirect&bpo=29478]: If max_line_length=None is specified while using the Compat32 policy, it is no longer ignored. Patch by Mircea Cosbuc.

Biblioteca

- bpo-29403 [https://bugs.python.org/issue?@action=redirect&bpo=29403]: Fix `unittest.mock`'s autospec to not fail on method-bound builtin functions. Patch by Aaron Gallagher.
- bpo-30961 [https://bugs.python.org/issue?@action=redirect&bpo=30961]: Fix decrementing a borrowed reference in tracemalloc.
- bpo-30886 [https://bugs.python.org/issue?@action=redirect&bpo=30886]: Fix multiprocessing.Queue.join_thread(): it now waits until the thread completes, even if the thread was started by the same process which created the queue.
- bpo-29854 [https://bugs.python.org/issue?@action=redirect&bpo=29854]: Fix segfault in readline when using readline's history-size option. Patch by Nir Soffer.
- bpo-30807 [https://bugs.python.org/issue?@action=redirect&bpo=30807]: signal.setitimer() may disable the timer when passed a tiny value. Tiny values (such as 1e-6) are valid non-zero values for setitimer(), which is specified as taking microsecond-resolution intervals. However, on some platform, our conversion routine could convert 1e-6 into a zero interval, therefore disabling the timer instead of (re-)scheduling it.
- bpo-30441 [https://bugs.python.org/issue?@action=redirect&bpo=30441]: Fix bug when modifying os.environ while iterating over it

- bpo-30532 [https://bugs.python.org/issue?@action=redirect&bpo=30532]: Fix email header value parser dropping folding white space in certain cases.
- bpo-29169 [https://bugs.python.org/issue?@action=redirect&bpo=29169]: Update zlib to 1.2.11.
- bpo-30879 [https://bugs.python.org/issue?@action=redirect&bpo=30879]: os.listdir() and os.scandir() now emit bytes names when called with bytes-like argument.
- bpo-30746 [https://bugs.python.org/issue?@action=redirect&bpo=30746]: Prohibited the '=' character in environment variable names in `os.putenv()` and `os.spawn*()`.
- bpo-29755 [https://bugs.python.org/issue?@action=redirect&bpo=29755]: Fixed the lgettext() family of functions in the gettext module. They now always return bytes.
- bpo-30645 [https://bugs.python.org/issue?@action=redirect&bpo=30645]: Fix path calculation in imp.load_package(), fixing it for cases when a package is only shipped with bytecodes. Patch by Alexandru Ardelean.
- bpo-23890 [https://bugs.python.org/issue?@action=redirect&bpo=23890]: unittest.TestCase.assertRaises() now manually breaks a reference cycle to not keep objects alive longer than expected.
- bpo-30149 [https://bugs.python.org/issue?@action=redirect&bpo=30149]: inspect.signature() now supports callables with variable-argument parameters wrapped with partialmethod. Patch by Donghee Na.
- bpo-29931 [https://bugs.python.org/issue?@action=redirect&bpo=29931]: Fixed comparison check for ipaddress.ip_interface objects. Patch by Sanjay Sundaresan.
- bpo-24484 [https://bugs.python.org/issue?@action=redirect&bpo=24484]: Avoid race condition in multiprocessing cleanup.
- bpo-28994 [https://bugs.python.org/issue?@action=redirect&bpo=28994]: The traceback no longer displayed for SystemExit raised in a callback registered by atexit.
- bpo-30508 [https://bugs.python.org/issue?@action=redirect&bpo=30508]: Don't log exceptions if Task/Future "cancel()" method was called.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Updates to typing module: Add generic AsyncContextManager, add support for ContextManager on all versions. Original PRs by Jelle Zijlstra and Ivan Levkivskyi

- bpo-29870 [https://bugs.python.org/issue?@action=redirect&bpo=29870]: Fix ssl sockets leaks when connection is aborted in asyncio/ssl implementation. Patch by Michaël Sghaïer.
- bpo-29743 [https://bugs.python.org/issue?@action=redirect&bpo=29743]: Closing transport during handshake process leaks open socket. Patch by Nikolay Kim
- bpo-27585 [https://bugs.python.org/issue?@action=redirect&bpo=27585]: Fix waiter cancellation in asyncio.Lock. Patch by Mathieu Sornay.
- bpo-30418 [https://bugs.python.org/issue?@action=redirect&bpo=30418]: On Windows, subprocess.Popen.communicate() now also ignore EINVAL on stdin.write() if the child process is still running but closed the pipe.
- bpo-30378 [https://bugs.python.org/issue?@action=redirect&bpo=30378]: Fix the problem that logging.handlers.SysLogHandler cannot handle IPv6 addresses.
- bpo-29960 [https://bugs.python.org/issue?@action=redirect&bpo=29960]: Preserve generator state when _random.Random.setstate() raises an exception. Patch by Bryan Olson.
- bpo-30414 [https://bugs.python.org/issue?@action=redirect&bpo=30414]: multiprocessing.Queue._feed background running thread do not break from main loop on exception.
- bpo-30003 [https://bugs.python.org/issue?@action=redirect&bpo=30003]: Fix handling escape characters in HZ codec. Based on patch by Ma Lin.
- bpo-30301 [https://bugs.python.org/issue?@action=redirect&bpo=30301]: Fix AttributeError when using SimpleQueue.empty() under *spawn* and *forkserver* start methods.
- bpo-30329 [https://bugs.python.org/issue?@action=redirect&bpo=30329]: imaplib and poplib now catch the Windows socket WSAEINVAL error (code 10022) on shutdown(SHUT_RDWR): An invalid operation was attempted. This error occurs sometimes on SSL connections.
- bpo-30375 [https://bugs.python.org/issue?@action=redirect&bpo=30375]: Warnings emitted when compile a regular expression now always point to the line in the user code. Previously they could point into inners of the re module if emitted from inside of groups or conditionals.
- bpo-30048 [https://bugs.python.org/issue?@action=redirect&bpo=30048]: Fixed `Task.cancel()` can be ignored when the task is running coroutine and the coroutine returned without any more `await`.

- bpo-29990 [https://bugs.python.org/issue?@action=redirect&bpo=29990]: Fix range checking in GB18030 decoder. Original patch by Ma Lin.
- bpo-26293 [https://bugs.python.org/issue?@action=redirect&bpo=26293]: Change resulted because of zipfile breakage. (See also: bpo-29094 [https://bugs.python.org/issue?@action=redirect&bpo=29094])
- bpo-30243 [https://bugs.python.org/issue?@action=redirect&bpo=30243]: Removed the __init__ methods of _json's scanner and encoder. Misusing them could cause memory leaks or crashes. Now scanner and encoder objects are completely initialized in the __new__ methods.
- bpo-30185 [https://bugs.python.org/issue?@action=redirect&bpo=30185]: Avoid KeyboardInterrupt tracebacks in forkserver helper process when Ctrl-C is received.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Various updates to typing module: add typing.NoReturn type, use WrapperDescriptorType, minor bug-fixes. Original PRs by Jim Fasarakis-Hilliard and Ivan Levkivskyi.
- bpo-30205 [https://bugs.python.org/issue?@action=redirect&bpo=30205]: Fix getsockname() for unbound AF_UNIX sockets on Linux.
- bpo-30070 [https://bugs.python.org/issue?@action=redirect&bpo=30070]: Fixed leaks and crashes in errors handling in the parser module.
- bpo-30061 [https://bugs.python.org/issue?@action=redirect&bpo=30061]: Fixed crashes in IOBase methods __next__() and readlines() when readline() or __next__() respectively return non-sizeable object. Fixed possible other errors caused by not checking results of PyObject_Size(), PySequence_Size(), or PyMapping_Size().
- bpo-30068 [https://bugs.python.org/issue?@action=redirect&bpo=30068]: _io._IOBase.readlines will check if it's closed first when hint is present.
- bpo-29694 [https://bugs.python.org/issue?@action=redirect&bpo=29694]: Fixed race condition in pathlib mkdir with flags parents=True. Patch by Armin Rigo.
- bpo-29692 [https://bugs.python.org/issue?@action=redirect&bpo=29692]: Fixed arbitrary unchaining of RuntimeError exceptions in contextlib.contextmanager. Patch by Siddharth Velankar.
- bpo-29998 [https://bugs.python.org/issue?@action=redirect&bpo=29998]: Pickling and copying ImportError now preserves name and path attributes.

- bpo-29942 [https://bugs.python.org/issue?@action=redirect&bpo=29942]: Fix a crash in itertools.chain.from_iterable when encountering long runs of empty iterables.
- bpo-27863 [https://bugs.python.org/issue?@action=redirect&bpo=27863]: Fixed multiple crashes in ElementTree caused by race conditions and wrong types.
- bpo-28699 [https://bugs.python.org/issue?@action=redirect&bpo=28699]: Fixed a bug in pools in multiprocessing.pool that raising an exception at the very first of an iterable may swallow the exception or make the program hang. Patch by Davin Potts and Xiang Zhang.
- bpo-25803 [https://bugs.python.org/issue?@action=redirect&bpo=25803]: Avoid incorrect errors raised by Path.mkdir(exist_ok=True) when the OS gives priority to errors such as EACCES over EEXIST.
- bpo-29861 [https://bugs.python.org/issue?@action=redirect&bpo=29861]: Release references to tasks, their arguments and their results as soon as they are finished in multiprocessing.Pool.
- bpo-29884 [https://bugs.python.org/issue?@action=redirect&bpo=29884]: faulthandler: Restore the old sigaltstack during teardown. Patch by Christophe Zeitouny.
- bpo-25455 [https://bugs.python.org/issue?@action=redirect&bpo=25455]: Fixed crashes in repr of recursive buffered file-like objects.
- bpo-29800 [https://bugs.python.org/issue?@action=redirect&bpo=29800]: Fix crashes in partial.__repr__ if the keys of partial.keywords are not strings. Patch by Michael Seifert.
- bpo-29742 [https://bugs.python.org/issue?@action=redirect&bpo=29742]: get_extra_info() raises exception if get called on closed ssl transport. Patch by Nikolay Kim.
- bpo-8256 [https://bugs.python.org/issue?@action=redirect&bpo=8256]: Fixed possible failing or crashing input() if attributes “encoding” or “errors” of sys.stdin or sys.stdout are not set or are not strings.
- bpo-28298 [https://bugs.python.org/issue?@action=redirect&bpo=28298]: Fix a bug that prevented array ‘Q’, ‘L’ and ‘I’ from accepting big intables (objects that have __int__) as elements. Patch by Oren Milman.
- bpo-29615 [https://bugs.python.org/issue?@action=redirect&bpo=29615]: SimpleXMLRPCDispatcher no longer chains KeyError (or any other exception) to exception(s) raised in the dispatched methods. Patch by Petr Motejlek.

- bpo-29704 [https://bugs.python.org/issue?@action=redirect&bpo=29704]: asyncio.subprocess.SubprocessStreamProtocol no longer closes before all pipes are closed.
- bpo-29703 [https://bugs.python.org/issue?@action=redirect&bpo=29703]: Fix asyncio to support instantiation of new event loops in child processes.
- bpo-29376 [https://bugs.python.org/issue?@action=redirect&bpo=29376]: Fix assertion error in threading._DummyThread.is_alive().
- bpo-29110 [https://bugs.python.org/issue?@action=redirect&bpo=29110]: Fix file object leak in aifc.open() when file is given as a filesystem path and is not in valid AIFF format. Patch by Anthony Zhang.
- bpo-28961 [https://bugs.python.org/issue?@action=redirect&bpo=28961]: Fix unittest.mock._Call helper: don't ignore the name parameter anymore. Patch written by Jiajun Huang.
- bpo-29532 [https://bugs.python.org/issue?@action=redirect&bpo=29532]: Altering a kwarg dictionary passed to functools.partial() no longer affects a partial object after creation.
- bpo-28556 [https://bugs.python.org/issue?@action=redirect&bpo=28556]: Various updates to typing module: typing.Counter, typing.ChainMap, improved ABC caching, etc. Original PRs by Jelle Zijlstra, Ivan Levkivskyi, Manuel Krebber, and Łukasz Langa.
- bpo-29100 [https://bugs.python.org/issue?@action=redirect&bpo=29100]: Fix datetime.fromtimestamp() regression introduced in Python 3.6.0: check minimum and maximum years.
- bpo-29519 [https://bugs.python.org/issue?@action=redirect&bpo=29519]: Fix weakref spewing exceptions during interpreter shutdown when used with a rare combination of multiprocessing and custom codecs.
- bpo-29416 [https://bugs.python.org/issue?@action=redirect&bpo=29416]: Prevent infinite loop in pathlib.Path.mkdir
- bpo-29444 [https://bugs.python.org/issue?@action=redirect&bpo=29444]: Fixed out-of-bounds buffer access in the group() method of the match object. Based on patch by WGH.
- bpo-29335 [https://bugs.python.org/issue?@action=redirect&bpo=29335]: Fix subprocess.Popen.wait() when the child process has exited to a stopped instead of terminated state (ex: when under ptrace).
- bpo-29290 [https://bugs.python.org/issue?@action=redirect&bpo=29290]: Fix a regression in argparse that help messages would wrap at non-breaking spaces.

- bpo-28735 [https://bugs.python.org/issue?@action=redirect&bpo=28735]: Fixed the comparison of mock.MagickMock with mock.ANY.
- bpo-29011 [https://bugs.python.org/issue?@action=redirect&bpo=29011]: Fix an important omission by adding Deque to the typing module.
- bpo-29219 [https://bugs.python.org/issue?@action=redirect&bpo=29219]: Fixed infinite recursion in the repr of uninitialized ctypes.CDLL instances.
- bpo-28969 [https://bugs.python.org/issue?@action=redirect&bpo=28969]: Fixed race condition in C implementation of functools.lru_cache. KeyError could be raised when cached function with full cache was simultaneously called from different threads with the same uncached arguments.
- bpo-29142 [https://bugs.python.org/issue?@action=redirect&bpo=29142]: In urllib.request, suffixes in no_proxy environment variable with leading dots could match related hostnames again (e.g. .b.c matches a.b.c). Patch by Milan Oberkirch.

Documentação

- bpo-30176 [https://bugs.python.org/issue?@action=redirect&bpo=30176]: Add missing attribute related constants in curses documentation.
- bpo-26985 [https://bugs.python.org/issue?@action=redirect&bpo=26985]: Add missing info of code object in inspect documentation.
- bpo-28929 [https://bugs.python.org/issue?@action=redirect&bpo=28929]: Link the documentation to its source file on GitHub.
- bpo-25008 [https://bugs.python.org/issue?@action=redirect&bpo=25008]: Document smtpd.py as effectively deprecated and add a pointer to aiosmtpd, a third-party asyncio-based replacement.
- bpo-26355 [https://bugs.python.org/issue?@action=redirect&bpo=26355]: Add canonical header link on each page to corresponding major version of the documentation. Patch by Matthias Bussonnier.
- bpo-29349 [https://bugs.python.org/issue?@action=redirect&bpo=29349]: Fix Python 2 syntax in code for building the documentation.

Testes

- bpo-30822 [https://bugs.python.org/issue?@action=redirect&bpo=30822]: Fix regrtest command line parser to allow passing -u extralargefile to run test_zipfile64.

- bpo-30383 [https://bugs.python.org/issue?@action=redirect&bpo=30383]: regrtest: Enhance regrtest and backport features from the master branch. Add options: –coverage, –testdir, –list-tests (list test files, don't run them), –list-cases (list test identifiers, don't run them, bpo-30523 [https://bugs.python.org/issue?@action=redirect&bpo=30523]), –matchfile (load a list of test filters from a text file, bpo-30540 [https://bugs.python.org/issue?@action=redirect&bpo=30540]), –slowest (alias to –slow). Enhance output: add timestamp, test result, currently running tests, "Tests result: xxx" summary with total duration, etc. Fix reference leak hunting in regrtest, –huntrleaks: regrtest now warms up caches, create explicitly all internal singletons which are created on demand to prevent false positives when checking for reference leaks. (bpo-30675 [https://bugs.python.org/issue?@action=redirect&bpo=30675]).
- bpo-30357 [https://bugs.python.org/issue?@action=redirect&bpo=30357]: test_thread: setUp() now uses support.threading_setup() and support.threading_cleanup() to wait until threads complete to avoid random side effects on following tests. Initial patch written by Grzegorz Grzywacz.
- bpo-28087 [https://bugs.python.org/issue?@action=redirect&bpo=28087]: Skip test_asyncore and test_eintr poll failures on macOS. Skip some tests of select.poll when running on macOS due to unresolved issues with the underlying system poll function on some macOS versions.
- bpo-30197 [https://bugs.python.org/issue?@action=redirect&bpo=30197]: Enhanced functions swap_attr() and swap_item() in the test.support module. They now work when delete replaced attribute or item inside the with statement. The old value of the attribute or item (or None if it doesn't exist) now will be assigned to the target of the "as" clause, if there is one.
- bpo-29571 [https://bugs.python.org/issue?@action=redirect&bpo=29571]: to match the behaviour of the `re.LOCALE` flag, test_re.test_locale_flag now uses `locale.getpreferredencoding(False)` to determine the candidate encoding for the test regex (allowing it to correctly skip the test when the default locale encoding is a multi-byte encoding)

Construção

- bpo-29243 [https://bugs.python.org/issue?@action=redirect&bpo=29243]: Prevent unnecessary rebuilding of Python during `make test`, `make install` and some other make targets when configured with `--enable-optimizations`.
- bpo-23404 [https://bugs.python.org/issue?@action=redirect&bpo=23404]: Don't regenerate generated files based on file modification time anymore: the action is now explicit. Replace `make touch` with `make regen-all`.
- bpo-29643 [https://bugs.python.org/issue?@action=redirect&bpo=29643]: Fix `--enable-optimization` didn't work.

Windows

- bpo-30687 [https://bugs.python.org/issue?@action=redirect&bpo=30687]: Locate msbuild.exe on Windows when building rather than vcvarsall.bat
- bpo-29392 [https://bugs.python.org/issue?@action=redirect&bpo=29392]: Prevent crash when passing invalid arguments into msvcrt module.

API C

- bpo-27867 [https://bugs.python.org/issue?@action=redirect&bpo=27867]: Function PySlice_GetIndicesEx() is replaced with a macro if Py_LIMITED_API is set to the value between 0x03050400 and 0x03060000 (not including) or 0x03060100 or higher.
- bpo-29083 [https://bugs.python.org/issue?@action=redirect&bpo=29083]: Fixed the declaration of some public API functions. PyArg_VaParse() and PyArg_VaParseTupleAndKeywords() were not available in limited API. PyArg_ValidateKeywordArguments(), PyArg_UnpackTuple() and Py_BuildValue() were not available in limited API of version < 3.3 when PY_SSIZE_T_CLEAN is defined.

Python 3.5.3 final

Data de lançamento: 2017-01-17

There were no code changes between 3.5.3rc1 and 3.5.3 final.

Python 3.5.3 release candidate 1

Data de lançamento: 2017-01-02

Segurança

- bpo-27278 [https://bugs.python.org/issue?@action=redirect&bpo=27278]: Fix os.urandom() implementation using getrandom() on Linux. Truncate size to INT_MAX and loop until we collected enough random bytes, instead of casting a directly Py_ssize_t to int.
- bpo-22636 [https://bugs.python.org/issue?@action=redirect&bpo=22636]: Avoid shell injection problems with ctypes.util.find_library().

Núcleo e embutidos

- bpo-29073 [https://bugs.python.org/issue?@action=redirect&bpo=29073]: bytearray formatting no longer truncates on first null byte.
- bpo-28932 [https://bugs.python.org/issue?@action=redirect&bpo=28932]: Do not include <sys/random.h> if it does not exist.
- bpo-28147 [https://bugs.python.org/issue?@action=redirect&bpo=28147]: Fix a memory leak in split-table dictionaries: setattr() must not convert combined table into split table.
- bpo-25677 [https://bugs.python.org/issue?@action=redirect&bpo=25677]: Correct the positioning of the syntax error caret for indented blocks. Based on patch by Michael Layzell.
- bpo-29000 [https://bugs.python.org/issue?@action=redirect&bpo=29000]: Fixed bytes formatting of octals with zero padding in alternate form.
- bpo-28512 [https://bugs.python.org/issue?@action=redirect&bpo=28512]: Fixed setting the offset attribute of SyntaxError by PyErr_SyntaxLocationEx() and PyErr_SyntaxLocationObject().
- bpo-28991 [https://bugs.python.org/issue?@action=redirect&bpo=28991]: functools.lru_cache() was susceptible to an obscure reentrancy bug caused by a monkey-patched len() function.
- bpo-28648 [https://bugs.python.org/issue?@action=redirect&bpo=28648]: Fixed crash in Py_DecodeLocale() in debug build on Mac OS X when decode astral characters. Patch by Xiang Zhang.

- bpo-19398 [https://bugs.python.org/issue?@action=redirect&bpo=19398]: Extra slash no longer added to sys.path components in case of empty compile-time PYTHONPATH components.
- bpo-28426 [https://bugs.python.org/issue?@action=redirect&bpo=28426]: Fixed potential crash in PyUnicode_AsDecodedObject() in debug build.
- bpo-23782 [https://bugs.python.org/issue?@action=redirect&bpo=23782]: Fixed possible memory leak in _PyTraceback_Add() and exception loss in PyTraceBack_Here().
- bpo-28379 [https://bugs.python.org/issue?@action=redirect&bpo=28379]: Added sanity checks and tests for PyUnicode_CopyCharacters(). Patch by Xiang Zhang.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: The type of long range iterator is now registered as Iterator. Patch by Oren Milman.
- bpo-28376 [https://bugs.python.org/issue?@action=redirect&bpo=28376]: The constructor of range_iterator now checks that step is not 0. Patch by Oren Milman.
- bpo-26906 [https://bugs.python.org/issue?@action=redirect&bpo=26906]: Resolving special methods of uninitialized type now causes implicit initialization of the type instead of a fail.
- bpo-18287 [https://bugs.python.org/issue?@action=redirect&bpo=18287]: PyType_Ready() now checks that tp_name is not NULL. Original patch by Niklas Koep.
- bpo-24098 [https://bugs.python.org/issue?@action=redirect&bpo=24098]: Fixed possible crash when AST is changed in process of compiling it.
- bpo-28350 [https://bugs.python.org/issue?@action=redirect&bpo=28350]: String constants with null character no longer interned.
- bpo-26617 [https://bugs.python.org/issue?@action=redirect&bpo=26617]: Fix crash when GC runs during weakref callbacks.
- bpo-27942 [https://bugs.python.org/issue?@action=redirect&bpo=27942]: String constants now interned recursively in tuples and frozensets.
- bpo-21578 [https://bugs.python.org/issue?@action=redirect&bpo=21578]: Fixed misleading error message when ImportError called with invalid keyword args.
- bpo-28203 [https://bugs.python.org/issue?@action=redirect&bpo=28203]: Fix incorrect type in error message from `complex(1.0, {2:3})`. Patch by Soumya Sharma.

- bpo-27955 [https://bugs.python.org/issue?@action=redirect&bpo=27955]: Fallback on reading /dev/urandom device when the getrandom() syscall fails with EPERM, for example when blocked by SECCOMP.
- bpo-28131 [https://bugs.python.org/issue?@action=redirect&bpo=28131]: Fix a regression in zipimport's compile_source(). zipimport should use the same optimization level as the interpreter.
- bpo-25221 [https://bugs.python.org/issue?@action=redirect&bpo=25221]: Fix corrupted result from PyLong_FromLong(0) when Python is compiled with NSMALLPOSINTS = 0.
- bpo-25758 [https://bugs.python.org/issue?@action=redirect&bpo=25758]: Prevents zipimport from unnecessarily encoding a filename (patch by Eryk Sun)
- bpo-28189 [https://bugs.python.org/issue?@action=redirect&bpo=28189]: dictitems_contains no longer swallows compare errors. (Patch by Xiang Zhang)
- bpo-27812 [https://bugs.python.org/issue?@action=redirect&bpo=27812]: Properly clear out a generator's frame's backreference to the generator to prevent crashes in frame.clear().
- bpo-27811 [https://bugs.python.org/issue?@action=redirect&bpo=27811]: Fix a crash when a coroutine that has not been awaited is finalized with warnings-as-errors enabled.
- bpo-27587 [https://bugs.python.org/issue?@action=redirect&bpo=27587]: Fix another issue found by PVS-Studio: Null pointer check after use of 'def' in _PyState_AddModule(). Initial patch by Christian Heimes.
- bpo-26020 [https://bugs.python.org/issue?@action=redirect&bpo=26020]: set literal evaluation order did not match documented behaviour.
- bpo-27782 [https://bugs.python.org/issue?@action=redirect&bpo=27782]: Multi-phase extension module import now correctly allows the `m_methods` field to be used to add module level functions to instances of non-module types returned from `Py_create_mod`. Patch by Xiang Zhang.
- bpo-27936 [https://bugs.python.org/issue?@action=redirect&bpo=27936]: The round() function accepted a second None argument for some types but not for others. Fixed the inconsistency by accepting None for all numeric types.
- bpo-27487 [https://bugs.python.org/issue?@action=redirect&bpo=27487]: Warn if a submodule argument to "python -m" or runpy.run_module() is found in sys.modules after parent packages are imported, but before the submodule is executed.

- bpo-27558 [https://bugs.python.org/issue?@action=redirect&bpo=27558]: Fix a SystemError in the implementation of “raise” statement. In a brand new thread, raise a RuntimeError since there is no active exception to reraise. Patch written by Xiang Zhang.
- bpo-27419 [https://bugs.python.org/issue?@action=redirect&bpo=27419]: Standard __import__() no longer look up “__import__” in globals or builtins for importing submodules or “from import”. Fixed handling an error of non-string package name.
- bpo-27083 [https://bugs.python.org/issue?@action=redirect&bpo=27083]: Respect the PYTHONCASEOK environment variable under Windows.
- bpo-27514 [https://bugs.python.org/issue?@action=redirect&bpo=27514]: Make having too many statically nested blocks a SyntaxError instead of SystemError.
- bpo-27473 [https://bugs.python.org/issue?@action=redirect&bpo=27473]: Fixed possible integer overflow in bytes and bytearray concatenations. Patch by Xiang Zhang.
- bpo-27507 [https://bugs.python.org/issue?@action=redirect&bpo=27507]: Add integer overflow check in bytearray.extend(). Patch by Xiang Zhang.
- bpo-27581 [https://bugs.python.org/issue?@action=redirect&bpo=27581]: Don’t rely on wrapping for overflow check in PySequence_Tuple(). Patch by Xiang Zhang.
- bpo-27443 [https://bugs.python.org/issue?@action=redirect&bpo=27443]: __length_hint__() of bytearray iterators no longer return a negative integer for a resized bytearray.
- bpo-27942 [https://bugs.python.org/issue?@action=redirect&bpo=27942]: Fix memory leak in codeobject.c

Biblioteca

- bpo-15812 [https://bugs.python.org/issue?@action=redirect&bpo=15812]: inspect.getframeinfo() now correctly shows the first line of a context. Patch by Sam Breese.
- bpo-29094 [https://bugs.python.org/issue?@action=redirect&bpo=29094]: Offsets in a ZIP file created with extern file object and modes “w” and “x” now are relative to the start of the file.
- bpo-13051 [https://bugs.python.org/issue?@action=redirect&bpo=13051]: Fixed recursion errors in large or resized curses.textpad.Textbox. Based on

patch by Tycho Andersen.

- bpo-29119 [https://bugs.python.org/issue?@action=redirect&bpo=29119]: Fix weakrefs in the pure python version of collections.OrderedDict move_to_end() method. Contributed by Andra Bogildea.
- bpo-9770 [https://bugs.python.org/issue?@action=redirect&bpo=9770]: curses.ascii predicates now work correctly with negative integers.
- bpo-28427 [https://bugs.python.org/issue?@action=redirect&bpo=28427]: old keys should not remove new values from WeakValueDictionary when collecting from another thread.
- bpo-28923 [https://bugs.python.org/issue?@action=redirect&bpo=28923]: Remove editor artifacts from Tix.py.
- bpo-28871 [https://bugs.python.org/issue?@action=redirect&bpo=28871]: Fixed a crash when deallocate deep ElementTree.
- bpo-19542 [https://bugs.python.org/issue?@action=redirect&bpo=19542]: Fix bugs in WeakValueDictionary.setdefault() and WeakValueDictionary.pop() when a GC collection happens in another thread.
- bpo-20191 [https://bugs.python.org/issue?@action=redirect&bpo=20191]: Fixed a crash in resource.prlimit() when pass a sequence that doesn't own its elements as limits.
- bpo-28779 [https://bugs.python.org/issue?@action=redirect&bpo=28779]: multiprocessing.set_forkserver_preload() would crash the forkserver process if a preloaded module instantiated some multiprocessing objects such as locks.
- bpo-28847 [https://bugs.python.org/issue?@action=redirect&bpo=28847]: dbm.dumb now supports reading read-only files and no longer writes the index file when it is not changed.
- bpo-25659 [https://bugs.python.org/issue?@action=redirect&bpo=25659]: In ctypes, prevent a crash calling the from_buffer() and from_buffer_copy() methods on abstract classes like Array.
- bpo-28732 [https://bugs.python.org/issue?@action=redirect&bpo=28732]: Fix crash in os.spawnv() with no elements in args
- bpo-28485 [https://bugs.python.org/issue?@action=redirect&bpo=28485]: Always raise ValueError for negative compileall.compile_dir(workers=…) parameter, even when multithreading is unavailable.
- bpo-28387 [https://bugs.python.org/issue?@action=redirect&bpo=28387]: Fixed possible crash in _io.TextIOWrapper deallocator when the garbage

collector is invoked in other thread. Based on patch by Sebastian Cufre.

- bpo-27517 [https://bugs.python.org/issue?@action=redirect&bpo=27517]: LZMA compressor and decompressor no longer raise exceptions if given empty data twice. Patch by Benjamin Fogle.
- bpo-28549 [https://bugs.python.org/issue?@action=redirect&bpo=28549]: Fixed segfault in curses's addch() with ncurses6.
- bpo-28449 [https://bugs.python.org/issue?@action=redirect&bpo=28449]: tarfile.open() with mode "r" or "r:" now tries to open a tar file with compression before trying to open it without compression. Otherwise it had 50% chance failed with ignore_zeros=True.
- bpo-23262 [https://bugs.python.org/issue?@action=redirect&bpo=23262]: The webbrowser module now supports Firefox 36+ and derived browsers. Based on patch by Oleg Broytman.
- bpo-27939 [https://bugs.python.org/issue?@action=redirect&bpo=27939]: Fixed bugs in tkinter.ttk.LabeledScale and tkinter.Scale caused by representing the scale as float value internally in Tk. tkinter.IntVar now works if float value is set to underlying Tk variable.
- bpo-28255 [https://bugs.python.org/issue?@action=redirect&bpo=28255]: calendar.TextCalendar().prmonth() no longer prints a space at the start of new line after printing a month's calendar. Patch by Xiang Zhang.
- bpo-20491 [https://bugs.python.org/issue?@action=redirect&bpo=20491]: The textwrap.TextWrapper class now honors non-breaking spaces. Based on patch by Kaarle Ritvanen.
- bpo-28353 [https://bugs.python.org/issue?@action=redirect&bpo=28353]: os.fwalk() no longer fails on broken links.
- bpo-25464 [https://bugs.python.org/issue?@action=redirect&bpo=25464]: Fixed HList.header_exists() in tkinter.tix module by addin a workaround to Tix library bug.
- bpo-28488 [https://bugs.python.org/issue?@action=redirect&bpo=28488]: shutil.make_archive() no longer add entry "./" to ZIP archive.
- bpo-24452 [https://bugs.python.org/issue?@action=redirect&bpo=24452]: Make webbrowser support Chrome on Mac OS X.
- bpo-20766 [https://bugs.python.org/issue?@action=redirect&bpo=20766]: Fix references leaked by pdb in the handling of SIGINT handlers.
- bpo-26293 [https://bugs.python.org/issue?@action=redirect&bpo=26293]: Fixed writing ZIP files that starts not from the start of the file. Offsets in ZIP

file now are relative to the start of the archive in conforming to the specification.

- bpo-28321 [https://bugs.python.org/issue?@action=redirect&bpo=28321]: Fixed writing non-BMP characters with binary format in plistlib.
- bpo-28322 [https://bugs.python.org/issue?@action=redirect&bpo=28322]: Fixed possible crashes when unpickle itertools objects from incorrect pickle data. Based on patch by John Leitch.
- Fix possible integer overflows and crashes in the mmap module with unusual usage patterns.
- bpo-1703178 [https://bugs.python.org/issue?@action=redirect&bpo=1703178]: Fix the ability to pass the –link-objects option to the distutils build_ext command.
- bpo-28253 [https://bugs.python.org/issue?@action=redirect&bpo=28253]: Fixed calendar functions for extreme months: 0001-01 and 9999-12. Methods itermonthdays() and itermonthdays2() are reimplemented so that they don't call itermonthdates() which can cause datetime.date under/overflow.
- bpo-28275 [https://bugs.python.org/issue?@action=redirect&bpo=28275]: Fixed possible use after free in the decompress() methods of the LZMADecompressor and BZ2Decompressor classes. Original patch by John Leitch.
- bpo-27897 [https://bugs.python.org/issue?@action=redirect&bpo=27897]: Fixed possible crash in sqlite3.Connection.create_collation() if pass invalid string-like object as a name. Patch by Xiang Zhang.
- bpo-18893 [https://bugs.python.org/issue?@action=redirect&bpo=18893]: Fix invalid exception handling in Lib/ctypes/macholib/dyld.py. Patch by Madison May.
- bpo-27611 [https://bugs.python.org/issue?@action=redirect&bpo=27611]: Fixed support of default root window in the tkinter.tix module.
- bpo-27348 [https://bugs.python.org/issue?@action=redirect&bpo=27348]: In the traceback module, restore the formatting of exception messages like "Exception: None". This fixes a regression introduced in 3.5a2.
- bpo-25651 [https://bugs.python.org/issue?@action=redirect&bpo=25651]: Allow false values to be used for msg parameter of subTest().
- bpo-27932 [https://bugs.python.org/issue?@action=redirect&bpo=27932]: Prevent memory leak in win32_ver().
- Fix UnboundLocalError in socket._sendfile_use_sendfile.

- bpo-28075 [https://bugs.python.org/issue?@action=redirect&bpo=28075]: Check for ERROR_ACCESS_DENIED in Windows implementation of os.stat(). Patch by Eryk Sun.
- bpo-25270 [https://bugs.python.org/issue?@action=redirect&bpo=25270]: Prevent codecs.escape_encode() from raising SystemError when an empty bytestring is passed.
- bpo-28181 [https://bugs.python.org/issue?@action=redirect&bpo=28181]: Get antigravity over HTTPS. Patch by Kaartic Sivaraam.
- bpo-25895 [https://bugs.python.org/issue?@action=redirect&bpo=25895]: Enable WebSocket URL schemes in urllib.parse.urljoin. Patch by Gergely Imreh and Markus Holtermann.
- bpo-27599 [https://bugs.python.org/issue?@action=redirect&bpo=27599]: Fixed buffer overrun in binascii.b2a_qp() and binascii.a2b_qp().
- bpo-19003 [https://bugs.python.org/issue?@action=redirect&bpo=19003]: m email.generator now replaces only `\r` and/or `\n` line endings, per the RFC, instead of all unicode line endings.
- bpo-28019 [https://bugs.python.org/issue?@action=redirect&bpo=28019]: itertools.count() no longer rounds non-integer step in range between 1.0 and 2.0 to 1.
- bpo-25969 [https://bugs.python.org/issue?@action=redirect&bpo=25969]: Update the lib2to3 grammar to handle the unpacking generalizations added in 3.5.
- bpo-14977 [https://bugs.python.org/issue?@action=redirect&bpo=14977]: mailcap now respects the order of the lines in the mailcap files ("first match"), as required by RFC 1542. Patch by Michael Lazar.
- bpo-24594 [https://bugs.python.org/issue?@action=redirect&bpo=24594]: Validates persist parameter when opening MSI database
- bpo-17582 [https://bugs.python.org/issue?@action=redirect&bpo=17582]: xml.etree.ElementTree nows preserves whitespaces in attributes (Patch by Duane Griffin. Reviewed and approved by Stefan Behnel.)
- bpo-28047 [https://bugs.python.org/issue?@action=redirect&bpo=28047]: Fixed calculation of line length used for the base64 CTE in the new email policies.
- bpo-27445 [https://bugs.python.org/issue?@action=redirect&bpo=27445]: Don't pass str(_charset) to MIMEText.set_payload(). Patch by Claude Paroz.
- bpo-22450 [https://bugs.python.org/issue?@action=redirect&bpo=22450]: urllib now includes an `Accept: */*` header among the default headers. This

makes the results of REST API requests more consistent and predictable especially when proxy servers are involved.

- lib2to3.pgen3.driver.load_grammar() now creates a stable cache file between runs given the same Grammar.txt input regardless of the hash randomization setting.
- bpo-27570 [https://bugs.python.org/issue?@action=redirect&bpo=27570]: Avoid zero-length memcpy() etc calls with null source pointers in the “ctypes” and “array” modules.
- bpo-22233 [https://bugs.python.org/issue?@action=redirect&bpo=22233]: Break email header lines *only* on the RFC specified CR and LF characters, not on arbitrary unicode line breaks. This also fixes a bug in HTTP header parsing.
- bpo-27988 [https://bugs.python.org/issue?@action=redirect&bpo=27988]: Fix email iter_attachments incorrect mutation of payload list.
- bpo-27691 [https://bugs.python.org/issue?@action=redirect&bpo=27691]: Fix ssl module’s parsing of GEN_RID subject alternative name fields in X.509 certs.
- bpo-27850 [https://bugs.python.org/issue?@action=redirect&bpo=27850]: Remove 3DES from ssl module’s default cipher list to counter measure sweet32 attack (**CVE 2016-2183** [https://www.cve.org/CVERecord?id=CVE-2016-2183]).
- bpo-27766 [https://bugs.python.org/issue?@action=redirect&bpo=27766]: Add ChaCha20 Poly1305 to ssl module’s default cipher list. (Required OpenSSL 1.1.0 or LibreSSL).
- bpo-26470 [https://bugs.python.org/issue?@action=redirect&bpo=26470]: Port ssl and hashlib module to OpenSSL 1.1.0.
- Remove support for passing a file descriptor to os.access. It never worked but previously didn’t raise.
- bpo-12885 [https://bugs.python.org/issue?@action=redirect&bpo=12885]: Fix error when distutils encounters symlink.
- bpo-27881 [https://bugs.python.org/issue?@action=redirect&bpo=27881]: Fixed possible bugs when setting sqlite3.Connection.isolation_level. Based on patch by Xiang Zhang.
- bpo-27861 [https://bugs.python.org/issue?@action=redirect&bpo=27861]: Fixed a crash in sqlite3.Connection.cursor() when a factory creates not a cursor. Patch by Xiang Zhang.
- bpo-19884 [https://bugs.python.org/issue?@action=redirect&bpo=19884]: Avoid spurious output on OS X with Gnu Readline.

- bpo-27706 [https://bugs.python.org/issue?@action=redirect&bpo=27706]: Restore deterministic behavior of random.Random().seed() for string seeds using seeding version 1. Allows sequences of calls to random() to exactly match those obtained in Python 2. Patch by Nofar Schnider.
- bpo-10513 [https://bugs.python.org/issue?@action=redirect&bpo=10513]: Fix a regression in Connection.commit(). Statements should not be reset after a commit.
- A new version of typing.py from https://github.com/python/typing: Collection (only for 3.6) (bpo-27598 [https://bugs.python.org/issue?@action=redirect&bpo=27598]). Add FrozenSet to __all__ (upstream #261). Fix crash in _get_type_vars() (upstream #259). Remove the dict constraint in ForwardRef._eval_type (upstream #252).
- bpo-27539 [https://bugs.python.org/issue?@action=redirect&bpo=27539]: Fix unnormalised `Fraction.__pow__` result in the case of negative exponent and negative base.
- bpo-21718 [https://bugs.python.org/issue?@action=redirect&bpo=21718]: cursor.description is now available for queries using CTEs.
- bpo-2466 [https://bugs.python.org/issue?@action=redirect&bpo=2466]: posixpath.ismount now correctly recognizes mount points which the user does not have permission to access.
- bpo-27773 [https://bugs.python.org/issue?@action=redirect&bpo=27773]: Correct some memory management errors server_hostname in _ssl.wrap_socket().
- bpo-26750 [https://bugs.python.org/issue?@action=redirect&bpo=26750]: unittest.mock.create_autospec() now works properly for subclasses of property() and other data descriptors.
- In the curses module, raise an error if window.getstr() or window.instr() is passed a negative value.
- bpo-27783 [https://bugs.python.org/issue?@action=redirect&bpo=27783]: Fix possible usage of uninitialized memory in operator.methodcaller.
- bpo-27774 [https://bugs.python.org/issue?@action=redirect&bpo=27774]: Fix possible Py_DECREF on unowned object in _sre.
- bpo-27760 [https://bugs.python.org/issue?@action=redirect&bpo=27760]: Fix possible integer overflow in binascii.b2a_qp.
- bpo-27758 [https://bugs.python.org/issue?@action=redirect&bpo=27758]: Fix possible integer overflow in the _csv module for large record lengths.

- bpo-27568 [https://bugs.python.org/issue?@action=redirect&bpo=27568]: Prevent HTTPoxy attack (**CVE 2016-1000110** [https://www.cve.org/CVERecord?id=CVE-2016-1000110]). Ignore the HTTP_PROXY variable when REQUEST_METHOD environment is set, which indicates that the script is in CGI mode.
- bpo-27656 [https://bugs.python.org/issue?@action=redirect&bpo=27656]: Do not assume sched.h defines any SCHED_* constants.
- bpo-27130 [https://bugs.python.org/issue?@action=redirect&bpo=27130]: In the “zlib” module, fix handling of large buffers (typically 4 GiB) when compressing and decompressing. Previously, inputs were limited to 4 GiB, and compression and decompression operations did not properly handle results of 4 GiB.
- bpo-27533 [https://bugs.python.org/issue?@action=redirect&bpo=27533]: Release GIL in nt._isdir
- bpo-17711 [https://bugs.python.org/issue?@action=redirect&bpo=17711]: Fixed unpickling by the persistent ID with protocol 0. Original patch by Alexandre Vassalotti.
- bpo-27522 [https://bugs.python.org/issue?@action=redirect&bpo=27522]: Avoid an unintentional reference cycle in email.feedparser.
- bpo-26844 [https://bugs.python.org/issue?@action=redirect&bpo=26844]: Fix error message for imp.find_module() to refer to ‘path’ instead of ‘name’. Patch by Lev Maximov.
- bpo-23804 [https://bugs.python.org/issue?@action=redirect&bpo=23804]: Fix SSL zero-length recv() calls to not block and not raise an error about unclean EOF.
- bpo-27466 [https://bugs.python.org/issue?@action=redirect&bpo=27466]: Change time format returned by http.cookie.time2netscape, confirming the netscape cookie format and making it consistent with documentation.
- bpo-26664 [https://bugs.python.org/issue?@action=redirect&bpo=26664]: Fix activate.fish by removing mis-use of `$`.
- bpo-22115 [https://bugs.python.org/issue?@action=redirect&bpo=22115]: Fixed tracing Tkinter variables: trace_vdelete() with wrong mode no longer break tracing, trace_vinfo() now always returns a list of pairs of strings, tracing in the “u” mode now works.
- Fix a scoping issue in importlib.util.LazyLoader which triggered an UnboundLocalError when lazy-loading a module that was already put into sys.modules.

- bpo-27079 [https://bugs.python.org/issue?@action=redirect&bpo=27079]: Fixed curses.ascii functions isblank(), iscntrl() and ispunct().
- bpo-26754 [https://bugs.python.org/issue?@action=redirect&bpo=26754]: Some functions (compile() etc) accepted a filename argument encoded as an iterable of integers. Now only strings and byte-like objects are accepted.
- bpo-27048 [https://bugs.python.org/issue?@action=redirect&bpo=27048]: Prevents distutils failing on Windows when environment variables contain non-ASCII characters
- bpo-27330 [https://bugs.python.org/issue?@action=redirect&bpo=27330]: Fixed possible leaks in the ctypes module.
- bpo-27238 [https://bugs.python.org/issue?@action=redirect&bpo=27238]: Got rid of bare excepts in the turtle module. Original patch by Jelle Zijlstra.
- bpo-27122 [https://bugs.python.org/issue?@action=redirect&bpo=27122]: When an exception is raised within the context being managed by a contextlib.ExitStack() and one of the exit stack generators catches and raises it in a chain, do not re-raise the original exception when exiting, let the new chained one through. This avoids the **PEP 479** [https://peps.python.org/pep-0479/] bug described in issue25782.
- bpo-26386 [https://bugs.python.org/issue?@action=redirect&bpo=26386]: Fixed ttk.TreeView selection operations with item id's containing spaces.
- bpo-16182 [https://bugs.python.org/issue?@action=redirect&bpo=16182]: Fix various functions in the "readline" module to use the locale encoding, and fix get_begidx() and get_endidx() to return code point indexes.
- bpo-27392 [https://bugs.python.org/issue?@action=redirect&bpo=27392]: Add loop.connect_accepted_socket(). Patch by Jim Fulton.
- bpo-27930 [https://bugs.python.org/issue?@action=redirect&bpo=27930]: Improved behaviour of logging.handlers.QueueListener. Thanks to Paulo Andrade and Petr Viktorin for the analysis and patch.
- bpo-21201 [https://bugs.python.org/issue?@action=redirect&bpo=21201]: Improves readability of multiprocessing error message. Thanks to Wojciech Walczak for patch.
- bpo-27456 [https://bugs.python.org/issue?@action=redirect&bpo=27456]: asyncio: Set TCP_NODELAY by default.
- bpo-27906 [https://bugs.python.org/issue?@action=redirect&bpo=27906]: Fix socket accept exhaustion during high TCP traffic. Patch by Kevin Conway.

- bpo-28174 [https://bugs.python.org/issue?@action=redirect&bpo=28174]: Handle when SO_REUSEPORT isn't properly supported. Patch by Seth Michael Larson.
- bpo-26654 [https://bugs.python.org/issue?@action=redirect&bpo=26654]: Inspect functools.partial in asyncio.Handle.__repr__. Patch by iceboy.
- bpo-26909 [https://bugs.python.org/issue?@action=redirect&bpo=26909]: Fix slow pipes IO in asyncio. Patch by INADA Naoki.
- bpo-28176 [https://bugs.python.org/issue?@action=redirect&bpo=28176]: Fix callbacks race in asyncio.SelectorLoop.sock_connect.
- bpo-27759 [https://bugs.python.org/issue?@action=redirect&bpo=27759]: Fix selectors incorrectly retain invalid file descriptors. Patch by Mark Williams.
- bpo-28368 [https://bugs.python.org/issue?@action=redirect&bpo=28368]: Refuse monitoring processes if the child watcher has no loop attached. Patch by Vincent Michel.
- bpo-28369 [https://bugs.python.org/issue?@action=redirect&bpo=28369]: Raise RuntimeError when transport's FD is used with add_reader, add_writer, etc.
- bpo-28370 [https://bugs.python.org/issue?@action=redirect&bpo=28370]: Speedup asyncio.StreamReader.readexactly. Patch by Коренберг Марк.
- bpo-28371 [https://bugs.python.org/issue?@action=redirect&bpo=28371]: Deprecate passing asyncio.Handles to run_in_executor.
- bpo-28372 [https://bugs.python.org/issue?@action=redirect&bpo=28372]: Fix asyncio to support formatting of non-python coroutines.
- bpo-28399 [https://bugs.python.org/issue?@action=redirect&bpo=28399]: Remove UNIX socket from FS before binding. Patch by Коренберг Марк.
- bpo-27972 [https://bugs.python.org/issue?@action=redirect&bpo=27972]: Prohibit Tasks to await on themselves.
- bpo-26923 [https://bugs.python.org/issue?@action=redirect&bpo=26923]: Fix asyncio.Gather to refuse being cancelled once all children are done. Patch by Johannes Ebke.
- bpo-26796 [https://bugs.python.org/issue?@action=redirect&bpo=26796]: Don't configure the number of workers for default threadpool executor. Initial patch by Hans Lawrenz.
- bpo-28600 [https://bugs.python.org/issue?@action=redirect&bpo=28600]: Optimize loop.call_soon().

- bpo-28613 [https://bugs.python.org/issue?@action=redirect&bpo=28613]: Fix get_event_loop() return the current loop if called from coroutines/callbacks.
- bpo-28639 [https://bugs.python.org/issue?@action=redirect&bpo=28639]: Fix inspect.isawaitable to always return bool Patch by Justin Mayfield.
- bpo-28652 [https://bugs.python.org/issue?@action=redirect&bpo=28652]: Make loop methods reject socket kinds they do not support.
- bpo-28653 [https://bugs.python.org/issue?@action=redirect&bpo=28653]: Fix a refleak in functools.lru_cache.
- bpo-28703 [https://bugs.python.org/issue?@action=redirect&bpo=28703]: Fix asyncio.iscoroutinefunction to handle Mock objects.
- bpo-24142 [https://bugs.python.org/issue?@action=redirect&bpo=24142]: Reading a corrupt config file left the parser in an invalid state. Original patch by Florian Höch.
- bpo-28990 [https://bugs.python.org/issue?@action=redirect&bpo=28990]: Fix SSL hanging if connection is closed before handshake completed. (Patch by HoHo-Ho)

IDLE

- bpo-15308 [https://bugs.python.org/issue?@action=redirect&bpo=15308]: Add ‘interrupt execution’ (^C) to Shell menu. Patch by Roger Serwy, updated by Bayard Randel.
- bpo-27922 [https://bugs.python.org/issue?@action=redirect&bpo=27922]: Stop IDLE tests from ‘flashing’ gui widgets on the screen.
- Add version to title of IDLE help window.
- bpo-25564 [https://bugs.python.org/issue?@action=redirect&bpo=25564]: In section on IDLE – console differences, mention that using exec means that __builtins__ is defined for each statement.
- bpo-27714 [https://bugs.python.org/issue?@action=redirect&bpo=27714]: text_textview and test_autocomplete now pass when re-run in the same process. This occurs when test_idle fails when run with the -w option but without -jn. Fix warning from test_config.
- bpo-25507 [https://bugs.python.org/issue?@action=redirect&bpo=25507]: IDLE no longer runs buggy code because of its tkinter imports. Users must include the same imports required to run directly in Python.

- bpo-27452 [https://bugs.python.org/issue?@action=redirect&bpo=27452]: add line counter and crc to IDLE configHandler test dump.
- bpo-27365 [https://bugs.python.org/issue?@action=redirect&bpo=27365]: Allow non-ascii chars in IDLE NEWS.txt, for contributor names.
- bpo-27245 [https://bugs.python.org/issue?@action=redirect&bpo=27245]: IDLE: Cleanly delete custom themes and key bindings. Previously, when IDLE was started from a console or by import, a cascade of warnings was emitted. Patch by Serhiy Storchaka.

API C

- bpo-28808 [https://bugs.python.org/issue?@action=redirect&bpo=28808]: PyUnicode_CompareWithASCIIString() now never raises exceptions.
- bpo-26754 [https://bugs.python.org/issue?@action=redirect&bpo=26754]: PyUnicode_FSDecoder() accepted a filename argument encoded as an iterable of integers. Now only strings and bytes-like objects are accepted.

Documentação

- bpo-28513 [https://bugs.python.org/issue?@action=redirect&bpo=28513]: Documented command-line interface of zipfile.

Testes

- bpo-28950 [https://bugs.python.org/issue?@action=redirect&bpo=28950]: Disallow -j0 to be combined with -T/-l/-M in regrtest command line arguments.
- bpo-28666 [https://bugs.python.org/issue?@action=redirect&bpo=28666]: Now test.support.rmtree is able to remove unwritable or unreadable directories.
- bpo-23839 [https://bugs.python.org/issue?@action=redirect&bpo=23839]: Various caches now are cleared before running every test file.
- bpo-28409 [https://bugs.python.org/issue?@action=redirect&bpo=28409]: regrtest: fix the parser of command line arguments.
- bpo-27787 [https://bugs.python.org/issue?@action=redirect&bpo=27787]: Call gc.collect() before checking each test for “dangling threads”, since the dangling threads are weak references.

- bpo-27369 [https://bugs.python.org/issue?@action=redirect&bpo=27369]: In test_pyexpat, avoid testing an error message detail that changed in Expat 2.2.0.

Ferramentas/Demos

- bpo-27952 [https://bugs.python.org/issue?@action=redirect&bpo=27952]: Get Tools/scripts/fixcid.py working with Python 3 and the current “re” module, avoid invalid Python backslash escapes, and fix a bug parsing escaped C quote signs.
- bpo-27332 [https://bugs.python.org/issue?@action=redirect&bpo=27332]: Fixed the type of the first argument of module-level functions generated by Argument Clinic. Patch by Petr Viktorin.
- bpo-27418 [https://bugs.python.org/issue?@action=redirect&bpo=27418]: Fixed Tools/importbench/importbench.py.

Windows

- bpo-28251 [https://bugs.python.org/issue?@action=redirect&bpo=28251]: Improvements to help manuals on Windows.
- bpo-28110 [https://bugs.python.org/issue?@action=redirect&bpo=28110]: launcher.msi has different product codes between 32-bit and 64-bit
- bpo-25144 [https://bugs.python.org/issue?@action=redirect&bpo=25144]: Ensures TargetDir is set before continuing with custom install.
- bpo-27469 [https://bugs.python.org/issue?@action=redirect&bpo=27469]: Adds a shell extension to the launcher so that drag and drop works correctly.
- bpo-27309 [https://bugs.python.org/issue?@action=redirect&bpo=27309]: Enabled proper Windows styles in python[w].exe manifest.

Construção

- bpo-29080 [https://bugs.python.org/issue?@action=redirect&bpo=29080]: Removes hard dependency on hg.exe from PCBuild/build.bat
- bpo-23903 [https://bugs.python.org/issue?@action=redirect&bpo=23903]: Added missed names to PC/python3.def.
- bpo-10656 [https://bugs.python.org/issue?@action=redirect&bpo=10656]: Fix out-of-tree building on AIX. Patch by Tristan Carel and Michael

Haubenwallner.

- bpo-26359 [https://bugs.python.org/issue?@action=redirect&bpo=26359]: Rename –with-optimiations to –enable-optimizations.
- bpo-28444 [https://bugs.python.org/issue?@action=redirect&bpo=28444]: Fix missing extensions modules when cross compiling.
- bpo-28248 [https://bugs.python.org/issue?@action=redirect&bpo=28248]: Update Windows build and OS X installers to use OpenSSL 1.0.2j.
- bpo-28258 [https://bugs.python.org/issue?@action=redirect&bpo=28258]: Fixed build with Estonian locale (python-config and distclean targets in Makefile). Patch by Arfrever Frehtes Taifersar Arahesis.
- bpo-26661 [https://bugs.python.org/issue?@action=redirect&bpo=26661]: setup.py now detects system libffi with multiarch wrapper.
- bpo-28066 [https://bugs.python.org/issue?@action=redirect&bpo=28066]: Fix the logic that searches build directories for generated include files when building outside the source tree.
- bpo-15819 [https://bugs.python.org/issue?@action=redirect&bpo=15819]: Remove redundant include search directory option for building outside the source tree.
- bpo-27566 [https://bugs.python.org/issue?@action=redirect&bpo=27566]: Fix clean target in freeze makefile (patch by Lisa Roach)
- bpo-27705 [https://bugs.python.org/issue?@action=redirect&bpo=27705]: Update message in validate_ucrtbase.py
- bpo-27983 [https://bugs.python.org/issue?@action=redirect&bpo=27983]: Cause lack of llvm-profdata tool when using clang as required for PGO linking to be a configure time error rather than make time when –with-optimizations is enabled. Also improve our ability to find the llvm-profdata tool on MacOS and some Linuxes.
- bpo-26307 [https://bugs.python.org/issue?@action=redirect&bpo=26307]: The profile-opt build now applies PGO to the built-in modules.
- bpo-26359 [https://bugs.python.org/issue?@action=redirect&bpo=26359]: Add the –with-optimizations configure flag.
- bpo-27713 [https://bugs.python.org/issue?@action=redirect&bpo=27713]: Suppress spurious build warnings when updating importlib’s bootstrap files. Patch by Xiang Zhang
- bpo-25825 [https://bugs.python.org/issue?@action=redirect&bpo=25825]: Correct the references to Modules/python.exp and ld_so_aix, which are required on AIX. This updates references to an installation path that was

changed in 3.2a4, and undoes changed references to the build tree that were made in 3.5.0a1.

- bpo-27453 [https://bugs.python.org/issue?@action=redirect&bpo=27453]: CPP invocation in configure must use CPPFLAGS. Patch by Chi Hsuan Yen.
- bpo-27641 [https://bugs.python.org/issue?@action=redirect&bpo=27641]: The configure script now inserts comments into the makefile to prevent the pgen and _freeze_importlib executables from being cross-compiled.
- bpo-26662 [https://bugs.python.org/issue?@action=redirect&bpo=26662]: Set PYTHON_FOR_GEN in configure as the Python program to be used for file generation during the build.
- bpo-10910 [https://bugs.python.org/issue?@action=redirect&bpo=10910]: Avoid C++ compilation errors on FreeBSD and OS X. Also update FreedBSD version checks for the original ctype UTF-8 workaround.
- bpo-28676 [https://bugs.python.org/issue?@action=redirect&bpo=28676]: Prevent missing 'getentropy' declaration warning on macOS. Patch by Gareth Rees.

Python 3.5.2 final

Data de lançamento: 2016-06-26

Núcleo e embutidos

- bpo-26930 [https://bugs.python.org/issue?@action=redirect&bpo=26930]: Update Windows builds to use OpenSSL 1.0.2h.

Testes

- bpo-26867 [https://bugs.python.org/issue?@action=redirect&bpo=26867]: Ubuntu's openssl OP_NO_SSLv3 is forced on by default; fix test.

IDLE

- bpo-27365 [https://bugs.python.org/issue?@action=redirect&bpo=27365]: Allow non-ascii in idlelib/NEWS.txt - minimal part for 3.5.2.

Python 3.5.2 release candidate 1

Data de lançamento: 2016-06-12

Segurança

- bpo-26556 [https://bugs.python.org/issue?@action=redirect&bpo=26556]: Update expat to 2.1.1, fixes **CVE 2015-1283** [https://www.cve.org/CVERecord?id=CVE-2015-1283].
- Fix TLS stripping vulnerability in smtplib, **CVE 2016-0772** [https://www.cve.org/CVERecord?id=CVE-2016-0772]. Reported by Team Oststrom.
- bpo-26839 [https://bugs.python.org/issue?@action=redirect&bpo=26839]: On Linux, `os.urandom()` now calls `getrandom()` with `GRND_NONBLOCK` to fall back on reading `/dev/urandom` if the urandom entropy pool is not initialized yet. Patch written by Colm Buckley.
- bpo-26657 [https://bugs.python.org/issue?@action=redirect&bpo=26657]: Fix directory traversal vulnerability with http.server on Windows. This fixes a regression that was introduced in 3.3.4rc1 and 3.4.0rc1. Based on patch by Philipp Hagemeister.
- bpo-26313 [https://bugs.python.org/issue?@action=redirect&bpo=26313]: ssl.py _load_windows_store_certs fails if windows cert store is empty. Patch by Baji.
- bpo-25939 [https://bugs.python.org/issue?@action=redirect&bpo=25939]: On Windows open the cert store readonly in ssl.enum_certificates.

Núcleo e embutidos

- bpo-27066 [https://bugs.python.org/issue?@action=redirect&bpo=27066]: Fixed SystemError if a custom opener (for open()) returns a negative number without setting an exception.
- bpo-20041 [https://bugs.python.org/issue?@action=redirect&bpo=20041]: Fixed TypeError when frame.f_trace is set to None. Patch by Xavier de Gaye.
- bpo-26168 [https://bugs.python.org/issue?@action=redirect&bpo=26168]: Fixed possible refleaks in failing Py_BuildValue() with the “N” format unit.

- bpo-26991 [https://bugs.python.org/issue?@action=redirect&bpo=26991]: Fix possible refleak when creating a function with annotations.
- bpo-27039 [https://bugs.python.org/issue?@action=redirect&bpo=27039]: Fixed bytearray.remove() for values greater than 127. Patch by Joe Jevnik.
- bpo-23640 [https://bugs.python.org/issue?@action=redirect&bpo=23640]: int.from_bytes() no longer bypasses constructors for subclasses.
- bpo-26811 [https://bugs.python.org/issue?@action=redirect&bpo=26811]: gc.get_objects() no longer contains a broken tuple with NULL pointer.
- bpo-20120 [https://bugs.python.org/issue?@action=redirect&bpo=20120]: Use RawConfigParser for .pypirc parsing, removing support for interpolation unintentionally added with move to Python 3. Behavior no longer does any interpolation in .pypirc files, matching behavior in Python 2.7 and Setuptools 19.0.
- bpo-26659 [https://bugs.python.org/issue?@action=redirect&bpo=26659]: Make the builtin slice type support cycle collection.
- bpo-26718 [https://bugs.python.org/issue?@action=redirect&bpo=26718]: super.__init__ no longer leaks memory if called multiple times. NOTE: A direct call of super.__init__ is not endorsed!
- bpo-25339 [https://bugs.python.org/issue?@action=redirect&bpo=25339]: PYTHONIOENCODING now has priority over locale in setting the error handler for stdin and stdout.
- bpo-26494 [https://bugs.python.org/issue?@action=redirect&bpo=26494]: Fixed crash on iterating exhausting iterators. Affected classes are generic sequence iterators, iterators of str, bytes, bytearray, list, tuple, set, frozenset, dict, OrderedDict, corresponding views and os.scandir() iterator.
- bpo-26581 [https://bugs.python.org/issue?@action=redirect&bpo=26581]: If coding cookie is specified multiple times on a line in Python source code file, only the first one is taken to account.
- bpo-26464 [https://bugs.python.org/issue?@action=redirect&bpo=26464]: Fix str.translate() when string is ASCII and first replacements removes character, but next replacement uses a non-ASCII character or a string longer than 1 character. Regression introduced in Python 3.5.0.
- bpo-22836 [https://bugs.python.org/issue?@action=redirect&bpo=22836]: Ensure exception reports from PyErr_Display() and PyErr_WriteUnraisable() are sensible even when formatting them produces secondary errors. This affects the reports produced by sys.__excepthook__() and when __del__() raises an exception.

- bpo-26302 [https://bugs.python.org/issue?@action=redirect&bpo=26302]: Correct behavior to reject comma as a legal character for cookie names.
- bpo-4806 [https://bugs.python.org/issue?@action=redirect&bpo=4806]: Avoid masking the original TypeError exception when using star (`*`) unpacking in function calls. Based on patch by Hagen Fürstenau and Daniel Urban.
- bpo-27138 [https://bugs.python.org/issue?@action=redirect&bpo=27138]: Fix the doc comment for FileFinder.find_spec().
- bpo-26154 [https://bugs.python.org/issue?@action=redirect&bpo=26154]: Add a new private _PyThreadState_UncheckedGet() function to get the current Python thread state, but don't issue a fatal error if it is NULL. This new function must be used instead of accessing directly the _PyThreadState_Current variable. The variable is no more exposed since Python 3.5.1 to hide the exact implementation of atomic C types, to avoid compiler issues.
- bpo-26194 [https://bugs.python.org/issue?@action=redirect&bpo=26194]: Deque.insert() gave odd results for bounded deques that had reached their maximum size. Now an IndexError will be raised when attempting to insert into a full deque.
- bpo-25843 [https://bugs.python.org/issue?@action=redirect&bpo=25843]: When compiling code, don't merge constants if they are equal but have a different types. For example, `f1, f2 = lambda: 1, lambda: 1.0` is now correctly compiled to two different functions: `f1()` returns `1` (`int`) and `f2()` returns `1.0` (`int`), even if `1` and `1.0` are equal.
- bpo-22995 [https://bugs.python.org/issue?@action=redirect&bpo=22995]: [UPDATE] Comment out the one of the pickleability tests in _PyObject_GetState() due to regressions observed in Cython-based projects.
- bpo-25961 [https://bugs.python.org/issue?@action=redirect&bpo=25961]: Disallowed null characters in the type name.
- bpo-25973 [https://bugs.python.org/issue?@action=redirect&bpo=25973]: Fix segfault when an invalid nonlocal statement binds a name starting with two underscores.
- bpo-22995 [https://bugs.python.org/issue?@action=redirect&bpo=22995]: Instances of extension types with a state that aren't subclasses of list or dict and haven't implemented any pickle-related methods (__reduce__,

__reduce_ex__, __getnewargs__, __getnewargs_ex__, or __getstate__), can no longer be pickled. Including memoryview.

- bpo-20440 [https://bugs.python.org/issue?@action=redirect&bpo=20440]: Massive replacing unsafe attribute setting code with special macro Py_SETREF.
- bpo-25766 [https://bugs.python.org/issue?@action=redirect&bpo=25766]: Special method __bytes__() now works in str subclasses.
- bpo-25421 [https://bugs.python.org/issue?@action=redirect&bpo=25421]: __sizeof__ methods of builtin types now use dynamic basic size. This allows sys.getsize() to work correctly with their subclasses with __slots__ defined.
- bpo-25709 [https://bugs.python.org/issue?@action=redirect&bpo=25709]: Fixed problem with in-place string concatenation and utf-8 cache.
- bpo-27147 [https://bugs.python.org/issue?@action=redirect&bpo=27147]: Mention **PEP 420** [https://peps.python.org/pep-0420/] in the importlib docs.
- bpo-24097 [https://bugs.python.org/issue?@action=redirect&bpo=24097]: Fixed crash in object.__reduce__() if slot name is freed inside __getattr__.
- bpo-24731 [https://bugs.python.org/issue?@action=redirect&bpo=24731]: Fixed crash on converting objects with special methods __bytes__, __trunc__, and __float__ returning instances of subclasses of bytes, int, and float to subclasses of bytes, int, and float correspondingly.
- bpo-26478 [https://bugs.python.org/issue?@action=redirect&bpo=26478]: Fix semantic bugs when using binary operators with dictionary views and tuples.
- bpo-26171 [https://bugs.python.org/issue?@action=redirect&bpo=26171]: Fix possible integer overflow and heap corruption in zipimporter.get_data().
- bpo-25660 [https://bugs.python.org/issue?@action=redirect&bpo=25660]: Fix TAB key behaviour in REPL with readline.
- bpo-25887 [https://bugs.python.org/issue?@action=redirect&bpo=25887]: Raise a RuntimeError when a coroutine object is awaited more than once.
- bpo-27243 [https://bugs.python.org/issue?@action=redirect&bpo=27243]: Update the __aiter__ protocol: instead of returning an awaitable that resolves to an asynchronous iterator, the asynchronous iterator should be returned directly. Doing the former will trigger a PendingDeprecationWarning.

Biblioteca

- bpo-21386 [https://bugs.python.org/issue?@action=redirect&bpo=21386]: Implement missing IPv4Address.is_global property. It was documented since 07a5610bae9d. Initial patch by Roger Luethi.
- bpo-20900 [https://bugs.python.org/issue?@action=redirect&bpo=20900]: distutils register command now decodes HTTP responses correctly. Initial patch by ingrid.
- A new version of typing.py provides several new classes and features: @overload outside stubs, Reversible, DefaultDict, Text, ContextManager, Type[], NewType(), TYPE_CHECKING, and numerous bug fixes (note that some of the new features are not yet implemented in mypy or other static analyzers). Also classes for **PEP 492** [https://peps.python.org/pep-0492/] (Awaitable, AsyncIterable, AsyncIterator) have been added (in fact they made it into 3.5.1 but were never mentioned).
- bpo-25738 [https://bugs.python.org/issue?@action=redirect&bpo=25738]: Stop http.server.BaseHTTPRequestHandler.send_error() from sending a message body for 205 Reset Content. Also, don't send Content header fields in responses that don't have a body. Patch by Susumu Koshiba.
- bpo-21313 [https://bugs.python.org/issue?@action=redirect&bpo=21313]: Fix the "platform" module to tolerate when sys.version contains truncated build information.
- bpo-27164 [https://bugs.python.org/issue?@action=redirect&bpo=27164]: In the zlib module, allow decompressing raw Deflate streams with a predefined zdict. Based on patch by Xiang Zhang.
- bpo-24291 [https://bugs.python.org/issue?@action=redirect&bpo=24291]: Fix wsgiref.simple_server.WSGIRequestHandler to completely write data to the client. Previously it could do partial writes and truncate data. Also, wsgiref.handler.ServerHandler can now handle stdout doing partial writes, but this is deprecated.
- bpo-26809 [https://bugs.python.org/issue?@action=redirect&bpo=26809]: Add `__all__` to `string`. Patch by Emanuel Barry.
- bpo-26373 [https://bugs.python.org/issue?@action=redirect&bpo=26373]: subprocess.Popen.communicate now correctly ignores BrokenPipeError when the child process dies before .communicate() is called in more/all circumstances.
- bpo-21776 [https://bugs.python.org/issue?@action=redirect&bpo=21776]: distutils.upload now correctly handles HTTPError. Initial patch by

Claudiu Popa.

- bpo-27114 [https://bugs.python.org/issue?@action=redirect&bpo=27114]: Fix SSLContext._load_windows_store_certs fails with PermissionError
- bpo-18383 [https://bugs.python.org/issue?@action=redirect&bpo=18383]: Avoid creating duplicate filters when using filterwarnings and simplefilter. Based on patch by Alex Shkop.
- bpo-27057 [https://bugs.python.org/issue?@action=redirect&bpo=27057]: Fix os.set_inheritable() on Android, ioctl() is blocked by SELinux and fails with EACCESS. The function now falls back to fcntl(). Patch written by Michał Bednarski.
- bpo-27014 [https://bugs.python.org/issue?@action=redirect&bpo=27014]: Fix infinite recursion using typing.py. Thanks to Kalle Tuure!
- bpo-14132 [https://bugs.python.org/issue?@action=redirect&bpo=14132]: Fix urllib.request redirect handling when the target only has a query string. Original fix by Ján Janech.
- bpo-17214 [https://bugs.python.org/issue?@action=redirect&bpo=17214]: The “urllib.request” module now percent-encodes non-ASCII bytes found in redirect target URLs. Some servers send Location header fields with non-ASCII bytes, but “http.client” requires the request target to be ASCII-encodable, otherwise a UnicodeEncodeError is raised. Based on patch by Christian Heimes.
- bpo-26892 [https://bugs.python.org/issue?@action=redirect&bpo=26892]: Honor debuglevel flag in urllib.request.HTTPHandler. Patch contributed by Chi Hsuan Yen.
- bpo-22274 [https://bugs.python.org/issue?@action=redirect&bpo=22274]: In the subprocess module, allow stderr to be redirected to stdout even when stdout is not redirected. Patch by Akira Li.
- bpo-26807 [https://bugs.python.org/issue?@action=redirect&bpo=26807]: mock_open ‘files’ no longer error on readline at end of file. Patch from Yolanda Robla.
- bpo-25745 [https://bugs.python.org/issue?@action=redirect&bpo=25745]: Fixed leaking a userptr in curses panel destructor.
- bpo-26977 [https://bugs.python.org/issue?@action=redirect&bpo=26977]: Removed unnecessary, and ignored, call to sum of squares helper in statistics.pvariance.
- bpo-26881 [https://bugs.python.org/issue?@action=redirect&bpo=26881]: The modulefinder module now supports extended opcode arguments.

- bpo-23815 [https://bugs.python.org/issue?@action=redirect&bpo=23815]: Fixed crashes related to directly created instances of types in _tkinter and curses.panel modules.
- bpo-17765 [https://bugs.python.org/issue?@action=redirect&bpo=17765]: weakref.ref() no longer silently ignores keyword arguments. Patch by Georg Brandl.
- bpo-26873 [https://bugs.python.org/issue?@action=redirect&bpo=26873]: xmlrpc now raises ResponseError on unsupported type tags instead of silently return incorrect result.
- bpo-26711 [https://bugs.python.org/issue?@action=redirect&bpo=26711]: Fixed the comparison of plistlib.Data with other types.
- bpo-24114 [https://bugs.python.org/issue?@action=redirect&bpo=24114]: Fix an uninitialized variable in `ctypes.util`. The bug only occurs on SunOS when the ctypes implementation searches for the `crle` program. Patch by Xiang Zhang. Tested on SunOS by Kees Bos.
- bpo-26864 [https://bugs.python.org/issue?@action=redirect&bpo=26864]: In urllib.request, change the proxy bypass host checking against no_proxy to be case-insensitive, and to not match unrelated host names that happen to have a bypassed hostname as a suffix. Patch by Xiang Zhang.
- bpo-26634 [https://bugs.python.org/issue?@action=redirect&bpo=26634]: recursive_repr() now sets __qualname__ of wrapper. Patch by Xiang Zhang.
- bpo-26804 [https://bugs.python.org/issue?@action=redirect&bpo=26804]: urllib.request will prefer lower_case proxy environment variables over UPPER_CASE or Mixed_Case ones. Patch contributed by Hans-Peter Jansen.
- bpo-26837 [https://bugs.python.org/issue?@action=redirect&bpo=26837]: assertSequenceEqual() now correctly outputs non-stringified differing items (like bytes in the -b mode). This affects assertListEqual() and assertTupleEqual().
- bpo-26041 [https://bugs.python.org/issue?@action=redirect&bpo=26041]: Remove “will be removed in Python 3.7” from deprecation messages of platform.dist() and platform.linux_distribution(). Patch by Kumaripaba Miyurusara Athukorala.
- bpo-26822 [https://bugs.python.org/issue?@action=redirect&bpo=26822]: itemgetter, attrgetter and methodcaller objects no longer silently ignore keyword

arguments.

- bpo-26733 [https://bugs.python.org/issue?@action=redirect&bpo=26733]: Disassembling a class now disassembles class and static methods. Patch by Xiang Zhang.
- bpo-26801 [https://bugs.python.org/issue?@action=redirect&bpo=26801]: Fix error handling in `shutil.get_terminal_size()`, catch `AttributeError` instead of `NameError`. Patch written by Emanuel Barry.
- bpo-24838 [https://bugs.python.org/issue?@action=redirect&bpo=24838]: tarfile's ustar and gnu formats now correctly calculate name and link field limits for multibyte character encodings like utf-8.
- bpo-26717 [https://bugs.python.org/issue?@action=redirect&bpo=26717]: Stop encoding Latin-1-ized WSGI paths with UTF-8. Patch by Anthony Sottile.
- bpo-26735 [https://bugs.python.org/issue?@action=redirect&bpo=26735]: Fix `os.urandom()` on Solaris 11.3 and newer when reading more than 1,024 bytes: call `getrandom()` multiple times with a limit of 1024 bytes per call.
- bpo-16329 [https://bugs.python.org/issue?@action=redirect&bpo=16329]: Add .webm to mimetypes.types_map. Patch by Giampaolo Rodola'.
- bpo-13952 [https://bugs.python.org/issue?@action=redirect&bpo=13952]: Add .csv to mimetypes.types_map. Patch by Geoff Wilson.
- bpo-26709 [https://bugs.python.org/issue?@action=redirect&bpo=26709]: Fixed Y2038 problem in loading binary PLists.
- bpo-23735 [https://bugs.python.org/issue?@action=redirect&bpo=23735]: Handle terminal resizing with Readline 6.3+ by installing our own SIGWINCH handler. Patch by Eric Price.
- bpo-26586 [https://bugs.python.org/issue?@action=redirect&bpo=26586]: In http.server, respond with "413 Request header fields too large" if there are too many header fields to parse, rather than killing the connection and raising an unhandled exception. Patch by Xiang Zhang.
- bpo-22854 [https://bugs.python.org/issue?@action=redirect&bpo=22854]: Change BufferedReader.writable() and BufferedWriter.readable() to always return False.
- bpo-25195 [https://bugs.python.org/issue?@action=redirect&bpo=25195]: Fix a regression in mock.MagicMock. _Call is a subclass of tuple (changeset 3603bae63c13 only works for classes) so we need to implement __ne__ ourselves. Patch by Andrew Plummer.

- bpo-26644 [https://bugs.python.org/issue?@action=redirect&bpo=26644]: Raise ValueError rather than SystemError when a negative length is passed to SSLSocket.recv() or read().
- bpo-23804 [https://bugs.python.org/issue?@action=redirect&bpo=23804]: Fix SSL recv(0) and read(0) methods to return zero bytes instead of up to 1024.
- bpo-26616 [https://bugs.python.org/issue?@action=redirect&bpo=26616]: Fixed a bug in datetime.astimezone() method.
- bpo-21925 [https://bugs.python.org/issue?@action=redirect&bpo=21925]: **`warnings.formatwarning()`** now catches exceptions on `linecache.getline(...)` to be able to log **`ResourceWarning`** emitted late during the Python shutdown process.
- bpo-24266 [https://bugs.python.org/issue?@action=redirect&bpo=24266]: Ctrl+C during Readline history search now cancels the search mode when compiled with Readline 7.
- bpo-26560 [https://bugs.python.org/issue?@action=redirect&bpo=26560]: Avoid potential ValueError in BaseHandler.start_response. Initial patch by Peter Inglesby.
- bpo-26569 [https://bugs.python.org/issue?@action=redirect&bpo=26569]: Fix **`pyclbr.readmodule()`** and **`pyclbr.readmodule_ex()`** to support importing packages.
- bpo-26499 [https://bugs.python.org/issue?@action=redirect&bpo=26499]: Account for remaining Content-Length in HTTPResponse.readline() and read1(). Based on patch by Silent Ghost. Also document that HTTPResponse now supports these methods.
- bpo-25320 [https://bugs.python.org/issue?@action=redirect&bpo=25320]: Handle sockets in directories unittest discovery is scanning. Patch from Victor van den Elzen.
- bpo-16181 [https://bugs.python.org/issue?@action=redirect&bpo=16181]: cookiejar.http2time() now returns None if year is higher than datetime.MAXYEAR.
- bpo-26513 [https://bugs.python.org/issue?@action=redirect&bpo=26513]: Fixes platform module detection of Windows Server
- bpo-23718 [https://bugs.python.org/issue?@action=redirect&bpo=23718]: Fixed parsing time in week 0 before Jan 1. Original patch by Tamás Bence Gedai.
- bpo-20589 [https://bugs.python.org/issue?@action=redirect&bpo=20589]: Invoking Path.owner() and Path.group() on Windows now raise

NotImplementedError instead of ImportError.

- bpo-26177 [https://bugs.python.org/issue?@action=redirect&bpo=26177]: Fixed the keys() method for Canvas and Scrollbar widgets.
- bpo-15068 [https://bugs.python.org/issue?@action=redirect&bpo=15068]: Got rid of excessive buffering in the fileinput module. The bufsize parameter is no longer used.
- bpo-2202 [https://bugs.python.org/issue?@action=redirect&bpo=2202]: Fix UnboundLocalError in AbstractDigestAuthHandler.get_algorithm_impls. Initial patch by Mathieu Dupuy.
- bpo-25718 [https://bugs.python.org/issue?@action=redirect&bpo=25718]: Fixed pickling and copying the accumulate() iterator with total is None.
- bpo-26475 [https://bugs.python.org/issue?@action=redirect&bpo=26475]: Fixed debugging output for regular expressions with the (?x) flag.
- bpo-26457 [https://bugs.python.org/issue?@action=redirect&bpo=26457]: Fixed the subnets() methods in IP network classes for the case when resulting prefix length is equal to maximal prefix length. Based on patch by Xiang Zhang.
- bpo-26385 [https://bugs.python.org/issue?@action=redirect&bpo=26385]: Remove the file if the internal open() call in NamedTemporaryFile() fails. Patch by Silent Ghost.
- bpo-26402 [https://bugs.python.org/issue?@action=redirect&bpo=26402]: Fix XML-RPC client to retry when the server shuts down a persistent connection. This was a regression related to the new http.client.RemoteDisconnected exception in 3.5.0a4.
- bpo-25913 [https://bugs.python.org/issue?@action=redirect&bpo=25913]: Leading <~ is optional now in base64.a85decode() with adobe=True. Patch by Swati Jaiswal.
- bpo-26186 [https://bugs.python.org/issue?@action=redirect&bpo=26186]: Remove an invalid type check in importlib.util.LazyLoader.
- bpo-26367 [https://bugs.python.org/issue?@action=redirect&bpo=26367]: importlib.__import__() raises SystemError like builtins.__import__() when `level` is specified but without an accompanying package specified.
- bpo-26309 [https://bugs.python.org/issue?@action=redirect&bpo=26309]: In the “socketserver” module, shut down the request (closing the connected socket) when verify_request() returns false. Patch by Aviv Palivoda.

- bpo-25995 [https://bugs.python.org/issue?@action=redirect&bpo=25995]: os.walk() no longer uses FDs proportional to the tree depth.
- bpo-26117 [https://bugs.python.org/issue?@action=redirect&bpo=26117]: The os.scandir() iterator now closes file descriptor not only when the iteration is finished, but when it was failed with error.
- bpo-25911 [https://bugs.python.org/issue?@action=redirect&bpo=25911]: Restored support of bytes paths in os.walk() on Windows.
- bpo-26045 [https://bugs.python.org/issue?@action=redirect&bpo=26045]: Add UTF-8 suggestion to error message when posting a non-Latin-1 string with http.client.
- bpo-12923 [https://bugs.python.org/issue?@action=redirect&bpo=12923]: Reset FancyURLopener's redirect counter even if there is an exception. Based on patches by Brian Brazil and Daniel Rocco.
- bpo-25945 [https://bugs.python.org/issue?@action=redirect&bpo=25945]: Fixed a crash when unpickle the functools.partial object with wrong state. Fixed a leak in failed functools.partial constructor. "args" and "keywords" attributes of functools.partial have now always types tuple and dict correspondingly.
- bpo-26202 [https://bugs.python.org/issue?@action=redirect&bpo=26202]: copy.deepcopy() now correctly copies range() objects with non-atomic attributes.
- bpo-23076 [https://bugs.python.org/issue?@action=redirect&bpo=23076]: Path.glob() now raises a ValueError if it's called with an invalid pattern. Patch by Thomas Nyberg.
- bpo-19883 [https://bugs.python.org/issue?@action=redirect&bpo=19883]: Fixed possible integer overflows in zipimport.
- bpo-26227 [https://bugs.python.org/issue?@action=redirect&bpo=26227]: On Windows, getnameinfo(), gethostbyaddr() and gethostbyname_ex() functions of the socket module now decode the hostname from the ANSI code page rather than UTF-8.
- bpo-26147 [https://bugs.python.org/issue?@action=redirect&bpo=26147]: xmlrpc now works with strings not encodable with used non-UTF-8 encoding.
- bpo-25935 [https://bugs.python.org/issue?@action=redirect&bpo=25935]: Garbage collector now breaks reference loops with OrderedDict.
- bpo-16620 [https://bugs.python.org/issue?@action=redirect&bpo=16620]: Fixed AttributeError in msilib.Directory.glob().

- bpo-26013 [https://bugs.python.org/issue?@action=redirect&bpo=26013]: Added compatibility with broken protocol 2 pickles created in old Python 3 versions (3.4.3 and lower).
- bpo-25850 [https://bugs.python.org/issue?@action=redirect&bpo=25850]: Use cross-compilation by default for 64-bit Windows.
- bpo-17633 [https://bugs.python.org/issue?@action=redirect&bpo=17633]: Improve zipimport's support for namespace packages.
- bpo-24705 [https://bugs.python.org/issue?@action=redirect&bpo=24705]: Fix sysconfig._parse_makefile not expanding ${} vars appearing before $() vars.
- bpo-22138 [https://bugs.python.org/issue?@action=redirect&bpo=22138]: Fix mock.patch behavior when patching descriptors. Restore original values after patching. Patch contributed by Sean McCully.
- bpo-25672 [https://bugs.python.org/issue?@action=redirect&bpo=25672]: In the ssl module, enable the SSL_MODE_RELEASE_BUFFERS mode option if it is safe to do so.
- bpo-26012 [https://bugs.python.org/issue?@action=redirect&bpo=26012]: Don't traverse into symlinks for `**` pattern in pathlib.Path.[r]glob().
- bpo-24120 [https://bugs.python.org/issue?@action=redirect&bpo=24120]: Ignore PermissionError when traversing a tree with pathlib.Path.[r]glob(). Patch by Ulrich Petri.
- bpo-25447 [https://bugs.python.org/issue?@action=redirect&bpo=25447]: fileinput now uses sys.stdin as-is if it does not have a buffer attribute (restores backward compatibility).
- bpo-25447 [https://bugs.python.org/issue?@action=redirect&bpo=25447]: Copying the lru_cache() wrapper object now always works, independently from the type of the wrapped object (by returning the original object unchanged).
- bpo-24103 [https://bugs.python.org/issue?@action=redirect&bpo=24103]: Fixed possible use after free in ElementTree.XMLPullParser.
- bpo-25860 [https://bugs.python.org/issue?@action=redirect&bpo=25860]: os.fwalk() no longer skips remaining directories when error occurs. Original patch by Samson Lee.
- bpo-25914 [https://bugs.python.org/issue?@action=redirect&bpo=25914]: Fixed and simplified OrderedDict.__sizeof__.
- bpo-25902 [https://bugs.python.org/issue?@action=redirect&bpo=25902]: Fixed various refcount issues in ElementTree iteration.

- bpo-25717 [https://bugs.python.org/issue?@action=redirect&bpo=25717]: Restore the previous behaviour of tolerating most fstat() errors when opening files. This was a regression in 3.5a1, and stopped anonymous temporary files from working in special cases.
- bpo-24903 [https://bugs.python.org/issue?@action=redirect&bpo=24903]: Fix regression in number of arguments compileall accepts when '-d' is specified. The check on the number of arguments has been dropped completely as it never worked correctly anyway.
- bpo-25764 [https://bugs.python.org/issue?@action=redirect&bpo=25764]: In the subprocess module, preserve any exception caused by fork() failure when preexec_fn is used.
- bpo-6478 [https://bugs.python.org/issue?@action=redirect&bpo=6478]: _strptime's regexp cache now is reset after changing timezone with time.tzset().
- bpo-14285 [https://bugs.python.org/issue?@action=redirect&bpo=14285]: When executing a package with the "python -m package" option, and package initialization fails, a proper traceback is now reported. The "runpy" module now lets exceptions from package initialization pass back to the caller, rather than raising ImportError.
- bpo-19771 [https://bugs.python.org/issue?@action=redirect&bpo=19771]: Also in runpy and the "-m" option, omit the irrelevant message "… is a package and cannot be directly executed" if the package could not even be initialized (e.g. due to a bad `*.pyc` file).
- bpo-25177 [https://bugs.python.org/issue?@action=redirect&bpo=25177]: Fixed problem with the mean of very small and very large numbers. As a side effect, statistics.mean and statistics.variance should be significantly faster.
- bpo-25718 [https://bugs.python.org/issue?@action=redirect&bpo=25718]: Fixed copying object with state with boolean value is false.
- bpo-10131 [https://bugs.python.org/issue?@action=redirect&bpo=10131]: Fixed deep copying of minidom documents. Based on patch by Marian Ganisin.
- bpo-25725 [https://bugs.python.org/issue?@action=redirect&bpo=25725]: Fixed a reference leak in pickle.loads() when unpickling invalid data including tuple instructions.
- bpo-25663 [https://bugs.python.org/issue?@action=redirect&bpo=25663]: In the Readline completer, avoid listing duplicate global names, and search the global namespace before searching builtins.

- bpo-25688 [https://bugs.python.org/issue?@action=redirect&bpo=25688]: Fixed file leak in ElementTree.iterparse() raising an error.
- bpo-23914 [https://bugs.python.org/issue?@action=redirect&bpo=23914]: Fixed SystemError raised by unpickler on broken pickle data.
- bpo-25691 [https://bugs.python.org/issue?@action=redirect&bpo=25691]: Fixed crash on deleting ElementTree.Element attributes.
- bpo-25624 [https://bugs.python.org/issue?@action=redirect&bpo=25624]: ZipFile now always writes a ZIP_STORED header for directory entries. Patch by Dingyuan Wang.
- Skip getaddrinfo if host is already resolved. Patch by A. Jesse Jiryu Davis.
- bpo-26050 [https://bugs.python.org/issue?@action=redirect&bpo=26050]: Add asyncio.StreamReader.readuntil() method. Patch by Марк Коренберг.
- bpo-25924 [https://bugs.python.org/issue?@action=redirect&bpo=25924]: Avoid unnecessary serialization of getaddrinfo(3) calls on OS X versions 10.5 or higher. Original patch by A. Jesse Jiryu Davis.
- bpo-26406 [https://bugs.python.org/issue?@action=redirect&bpo=26406]: Avoid unnecessary serialization of getaddrinfo(3) calls on current versions of OpenBSD and NetBSD. Patch by A. Jesse Jiryu Davis.
- bpo-26848 [https://bugs.python.org/issue?@action=redirect&bpo=26848]: Fix asyncio/subprocess.communicate() to handle empty input. Patch by Jack O'Connor.
- bpo-27040 [https://bugs.python.org/issue?@action=redirect&bpo=27040]: Add loop.get_exception_handler method
- bpo-27041 [https://bugs.python.org/issue?@action=redirect&bpo=27041]: asyncio: Add loop.create_future method
- bpo-27223 [https://bugs.python.org/issue?@action=redirect&bpo=27223]: asyncio: Fix _read_ready and _write_ready to respect _conn_lost. Patch by Łukasz Langa.
- bpo-22970 [https://bugs.python.org/issue?@action=redirect&bpo=22970]: asyncio: Fix inconsistency cancelling Condition.wait. Patch by David Coles.

IDLE

- bpo-5124 [https://bugs.python.org/issue?@action=redirect&bpo=5124]: Paste with text selected now replaces the selection on X11. This matches how

paste works on Windows, Mac, most modern Linux apps, and ttk widgets. Original patch by Serhiy Storchaka.

- bpo-24759 [https://bugs.python.org/issue?@action=redirect&bpo=24759]: Make clear in idlelib.idle_test.__init__ that the directory is a private implementation of test.test_idle and tool for maintainers.
- bpo-27196 [https://bugs.python.org/issue?@action=redirect&bpo=27196]: Stop 'ThemeChanged' warnings when running IDLE tests. These persisted after other warnings were suppressed in #20567. Apply Serhiy Storchaka's update_idletasks solution to four test files. Record this additional advice in idle_test/README.txt
- bpo-20567 [https://bugs.python.org/issue?@action=redirect&bpo=20567]: Revise idle_test/README.txt with advice about avoiding tk warning messages from tests. Apply advice to several IDLE tests.
- bpo-27117 [https://bugs.python.org/issue?@action=redirect&bpo=27117]: Make colorizer htest and turtledemo work with dark themes. Move code for configuring text widget colors to a new function.
- bpo-26673 [https://bugs.python.org/issue?@action=redirect&bpo=26673]: When tk reports font size as 0, change to size 10. Such fonts on Linux prevented the configuration dialog from opening.
- bpo-21939 [https://bugs.python.org/issue?@action=redirect&bpo=21939]: Add test for IDLE's percolator. Original patch by Saimadhav Heblikar.
- bpo-21676 [https://bugs.python.org/issue?@action=redirect&bpo=21676]: Add test for IDLE's replace dialog. Original patch by Saimadhav Heblikar.
- bpo-18410 [https://bugs.python.org/issue?@action=redirect&bpo=18410]: Add test for IDLE's search dialog. Original patch by Westley Martínez.
- bpo-21703 [https://bugs.python.org/issue?@action=redirect&bpo=21703]: Add test for IDLE's undo delegator. Original patch by Saimadhav Heblikar .
- bpo-27044 [https://bugs.python.org/issue?@action=redirect&bpo=27044]: Add ConfigDialog.remove_var_callbacks to stop memory leaks.
- bpo-23977 [https://bugs.python.org/issue?@action=redirect&bpo=23977]: Add more asserts to test_delegator.
- bpo-20640 [https://bugs.python.org/issue?@action=redirect&bpo=20640]: Add tests for idlelib.configHelpSourceEdit. Patch by Saimadhav Heblikar.
- In the 'IDLE-console differences' section of the IDLE doc, clarify how running with IDLE affects sys.modules and the standard streams.
- bpo-25507 [https://bugs.python.org/issue?@action=redirect&bpo=25507]: fix incorrect change in IOBinding that prevented printing. Augment

IOBinding htest to include all major IOBinding functions.

- bpo-25905 [https://bugs.python.org/issue?@action=redirect&bpo=25905]: Revert unwanted conversion of ‘ to ’ RIGHT SINGLE QUOTATION MARK in README.txt and open this and NEWS.txt with ‘ascii’. Re-encode CREDITS.txt to utf-8 and open it with ‘utf-8’.

Documentação

- bpo-19489 [https://bugs.python.org/issue?@action=redirect&bpo=19489]: Moved the search box from the sidebar to the header and footer of each page. Patch by Ammar Askar.
- bpo-24136 [https://bugs.python.org/issue?@action=redirect&bpo=24136]: Document the new **PEP 448** [https://peps.python.org/pep-0448/] unpacking syntax of 3.5.
- bpo-26736 [https://bugs.python.org/issue?@action=redirect&bpo=26736]: Used HTTPS for external links in the documentation if possible.
- bpo-6953 [https://bugs.python.org/issue?@action=redirect&bpo=6953]: Rework the Readline module documentation to group related functions together, and add more details such as what underlying Readline functions and variables are accessed.
- bpo-23606 [https://bugs.python.org/issue?@action=redirect&bpo=23606]: Adds note to ctypes documentation regarding cdll.msvcrt.
- bpo-25500 [https://bugs.python.org/issue?@action=redirect&bpo=25500]: Fix documentation to not claim that __import__ is searched for in the global scope.
- bpo-26014 [https://bugs.python.org/issue?@action=redirect&bpo=26014]: Update 3.x packaging documentation: * “See also” links to the new docs are now provided in the legacy pages * links to setuptools documentation have been updated

Testes

- bpo-21916 [https://bugs.python.org/issue?@action=redirect&bpo=21916]: Added tests for the turtle module. Patch by ingrid, Gregory Loyse and Jelle Zijlstra.
- bpo-26523 [https://bugs.python.org/issue?@action=redirect&bpo=26523]: The multiprocessing thread pool (multiprocessing.dummy.Pool) was untested.

- bpo-26015 [https://bugs.python.org/issue?@action=redirect&bpo=26015]: Added new tests for pickling iterators of mutable sequences.
- bpo-26325 [https://bugs.python.org/issue?@action=redirect&bpo=26325]: Added test.support.check_no_resource_warning() to check that no ResourceWarning is emitted.
- bpo-25940 [https://bugs.python.org/issue?@action=redirect&bpo=25940]: Changed test_ssl to use self-signed.pythontest.net. This avoids relying on svn.python.org, which recently changed root certificate.
- bpo-25616 [https://bugs.python.org/issue?@action=redirect&bpo=25616]: Tests for OrderedDict are extracted from test_collections into separate file test_ordered_dict.
- bpo-26583 [https://bugs.python.org/issue?@action=redirect&bpo=26583]: Skip test_timestamp_overflow in test_import if bytecode files cannot be written.

Construção

- bpo-26884 [https://bugs.python.org/issue?@action=redirect&bpo=26884]: Fix linking extension modules for cross builds. Patch by Xavier de Gaye.
- bpo-22359 [https://bugs.python.org/issue?@action=redirect&bpo=22359]: Disable the rules for running _freeze_importlib and pgen when cross-compiling. The output of these programs is normally saved with the source code anyway, and is still regenerated when doing a native build. Patch by Xavier de Gaye.
- bpo-27229 [https://bugs.python.org/issue?@action=redirect&bpo=27229]: Fix the cross-compiling pgen rule for in-tree builds. Patch by Xavier de Gaye.
- bpo-21668 [https://bugs.python.org/issue?@action=redirect&bpo=21668]: Link audioop, _datetime, _ctypes_test modules to libm, except on Mac OS X. Patch written by Xavier de Gaye.
- bpo-25702 [https://bugs.python.org/issue?@action=redirect&bpo=25702]: A –with-lto configure option has been added that will enable link time optimizations at build time during a make profile-opt. Some compilers and toolchains are known to not produce stable code when using LTO, be sure to test things thoroughly before relying on it. It can provide a few % speed up over profile-opt alone.
- bpo-26624 [https://bugs.python.org/issue?@action=redirect&bpo=26624]: Adds validation of ucrtbase[d].dll version with warning for old versions.

- bpo-17603 [https://bugs.python.org/issue?@action=redirect&bpo=17603]: Avoid error about nonexistent fileblocks.o file by using a lower-level check for st_blocks in struct stat.
- bpo-26079 [https://bugs.python.org/issue?@action=redirect&bpo=26079]: Fixing the build output folder for tix-8.4.3.6. Patch by Bjoern Thiel.
- bpo-26465 [https://bugs.python.org/issue?@action=redirect&bpo=26465]: Update Windows builds to use OpenSSL 1.0.2g.
- bpo-24421 [https://bugs.python.org/issue?@action=redirect&bpo=24421]: Compile Modules/_math.c once, before building extensions. Previously it could fail to compile properly if the math and cmath builds were concurrent.
- bpo-25348 [https://bugs.python.org/issue?@action=redirect&bpo=25348]: Added `--pgo` and `--pgo-job` arguments to `PCbuild\build.bat` for building with Profile-Guided Optimization. The old `PCbuild\build_pgo.bat` script is now deprecated, and simply calls `PCbuild\build.bat --pgo %*`.
- bpo-25827 [https://bugs.python.org/issue?@action=redirect&bpo=25827]: Add support for building with ICC to `configure`, including a new `--with-icc` flag.
- bpo-25696 [https://bugs.python.org/issue?@action=redirect&bpo=25696]: Fix installation of Python on UNIX with make -j9.
- bpo-26930 [https://bugs.python.org/issue?@action=redirect&bpo=26930]: Update OS X 10.5+ 32-bit-only installer to build and link with OpenSSL 1.0.2h.
- bpo-26268 [https://bugs.python.org/issue?@action=redirect&bpo=26268]: Update Windows builds to use OpenSSL 1.0.2f.
- bpo-25136 [https://bugs.python.org/issue?@action=redirect&bpo=25136]: Support Apple Xcode 7's new textual SDK stub libraries.
- bpo-24324 [https://bugs.python.org/issue?@action=redirect&bpo=24324]: Do not enable unreachable code warnings when using gcc as the option does not work correctly in older versions of gcc and has been silently removed as of gcc-4.5.

Windows

- bpo-27053 [https://bugs.python.org/issue?@action=redirect&bpo=27053]: Updates make_zip.py to correctly generate library ZIP file.
- bpo-26268 [https://bugs.python.org/issue?@action=redirect&bpo=26268]: Update the prepare_ssl.py script to handle OpenSSL releases that don't include the

contents of the include directory (that is, 1.0.2e and later).

- bpo-26071 [https://bugs.python.org/issue?@action=redirect&bpo=26071]: bdist_wininst created binaries fail to start and find 32bit Python
- bpo-26073 [https://bugs.python.org/issue?@action=redirect&bpo=26073]: Update the list of magic numbers in launcher
- bpo-26065 [https://bugs.python.org/issue?@action=redirect&bpo=26065]: Excludes venv from library when generating embeddable distro.
- bpo-17500 [https://bugs.python.org/issue?@action=redirect&bpo=17500]: Remove unused and outdated icons. (See also: https://github.com/python/pythondotorg/issues/945)

Ferramentas/Demos

- bpo-26799 [https://bugs.python.org/issue?@action=redirect&bpo=26799]: Fix python-gdb.py: don't get C types once when the Python code is loaded, but get C types on demand. The C types can change if python-gdb.py is loaded before the Python executable. Patch written by Thomas Ilsche.
- bpo-26271 [https://bugs.python.org/issue?@action=redirect&bpo=26271]: Fix the Freeze tool to properly use flags passed through configure. Patch by Daniel Shaulov.
- bpo-26489 [https://bugs.python.org/issue?@action=redirect&bpo=26489]: Add dictionary unpacking support to Tools/parser/unparse.py. Patch by Guo Ci Teo.
- bpo-26316 [https://bugs.python.org/issue?@action=redirect&bpo=26316]: Fix variable name typo in Argument Clinic.

Python 3.5.1 final

Data de lançamento: 2015-12-06

Núcleo e embutidos

- bpo-25709 [https://bugs.python.org/issue?@action=redirect&bpo=25709]: Fixed problem with in-place string concatenation and utf-8 cache.

Windows

- bpo-25715 [https://bugs.python.org/issue?@action=redirect&bpo=25715]: Python 3.5.1 installer shows wrong upgrade path and incorrect logic for launcher detection.

Python 3.5.1 release candidate 1

Data de lançamento: 2015-11-22

Núcleo e embutidos

- bpo-25630 [https://bugs.python.org/issue?@action=redirect&bpo=25630]: Fix a possible segfault during argument parsing in functions that accept filesystem paths.
- bpo-23564 [https://bugs.python.org/issue?@action=redirect&bpo=23564]: Fixed a partially broken sanity check in the _posixsubprocess internals regarding how fds_to_pass were passed to the child. The bug had no actual impact as subprocess.py already avoided it.
- bpo-25388 [https://bugs.python.org/issue?@action=redirect&bpo=25388]: Fixed tokenizer crash when processing undecodable source code with a null byte.
- bpo-25462 [https://bugs.python.org/issue?@action=redirect&bpo=25462]: The hash of the key now is calculated only once in most operations in C implementation of OrderedDict.
- bpo-22995 [https://bugs.python.org/issue?@action=redirect&bpo=22995]: Default implementation of __reduce__ and __reduce_ex__ now rejects builtin types with not defined __new__.
- bpo-25555 [https://bugs.python.org/issue?@action=redirect&bpo=25555]: Fix parser and AST: fill lineno and col_offset of “arg” node when compiling AST from Python objects.
- bpo-24802 [https://bugs.python.org/issue?@action=redirect&bpo=24802]: Avoid buffer overreads when int(), float(), compile(), exec() and eval() are passed bytes-like objects. These objects are not necessarily terminated by a null byte, but the functions assumed they were.
- bpo-24726 [https://bugs.python.org/issue?@action=redirect&bpo=24726]: Fixed a crash and leaking NULL in repr() of OrderedDict that was mutated by direct calls of dict methods.

- bpo-25449 [https://bugs.python.org/issue?@action=redirect&bpo=25449]: Iterating OrderedDict with keys with unstable hash now raises KeyError in C implementations as well as in Python implementation.
- bpo-25395 [https://bugs.python.org/issue?@action=redirect&bpo=25395]: Fixed crash when highly nested OrderedDict structures were garbage collected.
- bpo-25274 [https://bugs.python.org/issue?@action=redirect&bpo=25274]: sys.setrecursionlimit() now raises a RecursionError if the new recursion limit is too low depending at the current recursion depth. Modify also the “lower-water mark” formula to make it monotonic. This mark is used to decide when the overflowed flag of the thread state is reset.
- bpo-24402 [https://bugs.python.org/issue?@action=redirect&bpo=24402]: Fix input() to prompt to the redirected stdout when sys.stdout.fileno() fails.
- bpo-24806 [https://bugs.python.org/issue?@action=redirect&bpo=24806]: Prevent builtin types that are not allowed to be subclassed from being subclassed through multiple inheritance.
- bpo-24848 [https://bugs.python.org/issue?@action=redirect&bpo=24848]: Fixed a number of bugs in UTF-7 decoding of misformed data.
- bpo-25280 [https://bugs.python.org/issue?@action=redirect&bpo=25280]: Import trace messages emitted in verbose (-v) mode are no longer formatted twice.
- bpo-25003 [https://bugs.python.org/issue?@action=redirect&bpo=25003]: On Solaris 11.3 or newer, os.urandom() now uses the getrandom() function instead of the getentropy() function. The getentropy() function is blocking to generate very good quality entropy, os.urandom() doesn’t need such high-quality entropy.
- bpo-25182 [https://bugs.python.org/issue?@action=redirect&bpo=25182]: The stdprinter (used as sys.stderr before the io module is imported at startup) now uses the backslashreplace error handler.
- bpo-25131 [https://bugs.python.org/issue?@action=redirect&bpo=25131]: Make the line number and column offset of set/dict literals and comprehensions correspond to the opening brace.
- bpo-25150 [https://bugs.python.org/issue?@action=redirect&bpo=25150]: Hide the private `_Py_atomic_`*`xxx`* symbols from the public Python.h header to fix a compilation error with OpenMP. PyThreadState_GET() becomes an alias to PyThreadState_Get() to avoid ABI incompatibilities.

Biblioteca

- bpo-25626 [https://bugs.python.org/issue?@action=redirect&bpo=25626]: Change three zlib functions to accept sizes that fit in Py_ssize_t, but internally cap those sizes to UINT_MAX. This resolves a regression in 3.5 where GzipFile.read() failed to read chunks larger than 2 or 4 GiB. The change affects the zlib.Decompress.decompress() max_length parameter, the zlib.decompress() bufsize parameter, and the zlib.Decompress.flush() length parameter.
- bpo-25583 [https://bugs.python.org/issue?@action=redirect&bpo=25583]: Avoid incorrect errors raised by os.makedirs(exist_ok=True) when the OS gives priority to errors such as EACCES over EEXIST.
- bpo-25593 [https://bugs.python.org/issue?@action=redirect&bpo=25593]: Change semantics of EventLoop.stop() in asyncio.
- bpo-6973 [https://bugs.python.org/issue?@action=redirect&bpo=6973]: When we know a subprocess.Popen process has died, do not allow the send_signal(), terminate(), or kill() methods to do anything as they could potentially signal a different process.
- bpo-25590 [https://bugs.python.org/issue?@action=redirect&bpo=25590]: In the Readline completer, only call getattr() once per attribute.
- bpo-25498 [https://bugs.python.org/issue?@action=redirect&bpo=25498]: Fix a crash when garbage-collecting ctypes objects created by wrapping a memoryview. This was a regression made in 3.5a1. Based on patch by Eryksun.
- bpo-25584 [https://bugs.python.org/issue?@action=redirect&bpo=25584]: Added “escape” to the __all__ list in the glob module.
- bpo-25584 [https://bugs.python.org/issue?@action=redirect&bpo=25584]: Fixed recursive glob() with patterns starting with `**`.
- bpo-25446 [https://bugs.python.org/issue?@action=redirect&bpo=25446]: Fix regression in smtplib’s AUTH LOGIN support.
- bpo-18010 [https://bugs.python.org/issue?@action=redirect&bpo=18010]: Fix the pydoc web server’s module search function to handle exceptions from importing packages.
- bpo-25554 [https://bugs.python.org/issue?@action=redirect&bpo=25554]: Got rid of circular references in regular expression parsing.
- bpo-25510 [https://bugs.python.org/issue?@action=redirect&bpo=25510]: fileinput.FileInput.readline() now returns b’’ instead of ‘’ at the end if

the FileInput was opened with binary mode. Patch by Ryosuke Ito.

- bpo-25503 [https://bugs.python.org/issue?@action=redirect&bpo=25503]: Fixed inspect.getdoc() for inherited docstrings of properties. Original patch by John Mark Vandenberg.
- bpo-25515 [https://bugs.python.org/issue?@action=redirect&bpo=25515]: Always use os.urandom as a source of randomness in uuid.uuid4.
- bpo-21827 [https://bugs.python.org/issue?@action=redirect&bpo=21827]: Corrigido textwrap.dedent() para o caso em que o maior espaço em branco comum seja uma substring do menor espaço em branco inicial. Com base no *patch* de Robert Li.
- bpo-25447 [https://bugs.python.org/issue?@action=redirect&bpo=25447]: The lru_cache() wrapper objects now can be copied and pickled (by returning the original object unchanged).
- bpo-25390 [https://bugs.python.org/issue?@action=redirect&bpo=25390]: typing: Don’t crash on Union[str, Pattern].
- bpo-25441 [https://bugs.python.org/issue?@action=redirect&bpo=25441]: asyncio: Raise error from drain() when socket is closed.
- bpo-25410 [https://bugs.python.org/issue?@action=redirect&bpo=25410]: Cleaned up and fixed minor bugs in C implementation of OrderedDict.
- bpo-25411 [https://bugs.python.org/issue?@action=redirect&bpo=25411]: Improved Unicode support in SMTPHandler through better use of the email package. Thanks to user simon04 for the patch.
- bpo-25407 [https://bugs.python.org/issue?@action=redirect&bpo=25407]: Remove mentions of the formatter module being removed in Python 3.6.
- bpo-25406 [https://bugs.python.org/issue?@action=redirect&bpo=25406]: Fixed a bug in C implementation of OrderedDict.move_to_end() that caused segmentation fault or hang in iterating after moving several items to the start of ordered dict.
- bpo-25364 [https://bugs.python.org/issue?@action=redirect&bpo=25364]: zipfile now works in threads disabled builds.
- bpo-25328 [https://bugs.python.org/issue?@action=redirect&bpo=25328]: smtpd’s SMTPChannel now correctly raises a ValueError if both decode_data and enable_SMTPUTF8 are set to true.
- bpo-25316 [https://bugs.python.org/issue?@action=redirect&bpo=25316]: distutils raises OSError instead of DistutilsPlatformError when MSVC is not installed.

- bpo-25380 [https://bugs.python.org/issue?@action=redirect&bpo=25380]: Fixed protocol for the STACK_GLOBAL opcode in pickletools.opcodes.
- bpo-23972 [https://bugs.python.org/issue?@action=redirect&bpo=23972]: Updates asyncio datagram create method allowing reuseport and reuseaddr socket options to be set prior to binding the socket. Mirroring the existing asyncio create_server method the reuseaddr option for datagram sockets defaults to True if the O/S is 'posix' (except if the platform is Cygwin). Patch by Chris Laws.
- bpo-25304 [https://bugs.python.org/issue?@action=redirect&bpo=25304]: Add asyncio.run_coroutine_threadsafe(). This lets you submit a coroutine to a loop from another thread, returning a concurrent.futures.Future. By Vincent Michel.
- bpo-25232 [https://bugs.python.org/issue?@action=redirect&bpo=25232]: Fix CGIRequestHandler to split the query from the URL at the first question mark (?) rather than the last. Patch from Xiang Zhang.
- bpo-24657 [https://bugs.python.org/issue?@action=redirect&bpo=24657]: Prevent CGIRequestHandler from collapsing slashes in the query part of the URL as if it were a path. Patch from Xiang Zhang.
- bpo-24483 [https://bugs.python.org/issue?@action=redirect&bpo=24483]: C implementation of functools.lru_cache() now calculates key's hash only once.
- bpo-22958 [https://bugs.python.org/issue?@action=redirect&bpo=22958]: Constructor and update method of weakref.WeakValueDictionary now accept the self and the dict keyword arguments.
- bpo-22609 [https://bugs.python.org/issue?@action=redirect&bpo=22609]: Constructor of collections.UserDict now accepts the self keyword argument.
- bpo-25111 [https://bugs.python.org/issue?@action=redirect&bpo=25111]: Fixed comparison of traceback.FrameSummary.
- bpo-25262 [https://bugs.python.org/issue?@action=redirect&bpo=25262]: Added support for BINBYTES8 opcode in Python implementation of unpickler. Highest 32 bits of 64-bit size for BINUNICODE8 and BINBYTES8 opcodes no longer silently ignored on 32-bit platforms in C implementation.
- bpo-25034 [https://bugs.python.org/issue?@action=redirect&bpo=25034]: Fix string.Formatter problem with auto-numbering and nested format_specs. Patch by Anthon van der Neut.

- bpo-25233 [https://bugs.python.org/issue?@action=redirect&bpo=25233]: Rewrite the guts of asyncio.Queue and asyncio.Semaphore to be more understandable and correct.
- bpo-25203 [https://bugs.python.org/issue?@action=redirect&bpo=25203]: Failed readline.set_completer_delims() no longer left the module in inconsistent state.
- bpo-23600 [https://bugs.python.org/issue?@action=redirect&bpo=23600]: Default implementation of tzinfo.fromutc() was returning wrong results in some cases.
- bpo-23329 [https://bugs.python.org/issue?@action=redirect&bpo=23329]: Allow the ssl module to be built with older versions of LibreSSL.
- Prevent overflow in _Unpickler_Read.
- bpo-25047 [https://bugs.python.org/issue?@action=redirect&bpo=25047]: The XML encoding declaration written by Element Tree now respects the letter case given by the user. This restores the ability to write encoding names in uppercase like “UTF-8”, which worked in Python 2.
- bpo-25135 [https://bugs.python.org/issue?@action=redirect&bpo=25135]: Make deque_clear() safer by emptying the deque before clearing. This helps avoid possible reentrancy issues.
- bpo-19143 [https://bugs.python.org/issue?@action=redirect&bpo=19143]: platform module now reads Windows version from kernel32.dll to avoid compatibility shims.
- bpo-25092 [https://bugs.python.org/issue?@action=redirect&bpo=25092]: Fix datetime.strftime() failure when errno was already set to EINVAL.
- bpo-23517 [https://bugs.python.org/issue?@action=redirect&bpo=23517]: Fix rounding in fromtimestamp() and utcfromtimestamp() methods of datetime.datetime: microseconds are now rounded to nearest with ties going to nearest even integer (ROUND_HALF_EVEN), instead of being rounding towards minus infinity (ROUND_FLOOR). It’s important that these methods use the same rounding mode than datetime.timedelta to keep the property: (datetime(1970,1,1) + timedelta(seconds=t)) == datetime.utcfromtimestamp(t). It also the rounding mode used by round(float) for example.
- bpo-25155 [https://bugs.python.org/issue?@action=redirect&bpo=25155]: Fix datetime.datetime.now() and datetime.datetime.utcnow() on Windows to support date after year 2038. It was a regression introduced in Python 3.5.0.

- bpo-25108 [https://bugs.python.org/issue?@action=redirect&bpo=25108]: Omitted internal frames in traceback functions print_stack(), format_stack(), and extract_stack() called without arguments.
- bpo-25118 [https://bugs.python.org/issue?@action=redirect&bpo=25118]: Fix a regression of Python 3.5.0 in os.waitpid() on Windows.
- bpo-24684 [https://bugs.python.org/issue?@action=redirect&bpo=24684]: socket.socket.getaddrinfo() now calls PyUnicode_AsEncodedString() instead of calling the encode() method of the host, to handle correctly custom string with an encode() method which doesn't return a byte string. The encoder of the IDNA codec is now called directly instead of calling the encode() method of the string.
- bpo-25060 [https://bugs.python.org/issue?@action=redirect&bpo=25060]: Correctly compute stack usage of the BUILD_MAP opcode.
- bpo-24857 [https://bugs.python.org/issue?@action=redirect&bpo=24857]: Comparing call_args to a long sequence now correctly returns a boolean result instead of raising an exception. Patch by A Kaptur.
- bpo-23144 [https://bugs.python.org/issue?@action=redirect&bpo=23144]: Make sure that HTMLParser.feed() returns all the data, even when convert_charrefs is True.
- bpo-24982 [https://bugs.python.org/issue?@action=redirect&bpo=24982]: shutil.make_archive() with the "zip" format now adds entries for directories (including empty directories) in ZIP file.
- bpo-25019 [https://bugs.python.org/issue?@action=redirect&bpo=25019]: Fixed a crash caused by setting non-string key of expat parser. Based on patch by John Leitch.
- bpo-16180 [https://bugs.python.org/issue?@action=redirect&bpo=16180]: Exit pdb if file has syntax error, instead of trapping user in an infinite loop. Patch by Xavier de Gaye.
- bpo-24891 [https://bugs.python.org/issue?@action=redirect&bpo=24891]: Fix a race condition at Python startup if the file descriptor of stdin (0), stdout (1) or stderr (2) is closed while Python is creating sys.stdin, sys.stdout and sys.stderr objects. These attributes are now set to None if the creation of the object failed, instead of raising an OSError exception. Initial patch written by Marco Paolini.
- bpo-24992 [https://bugs.python.org/issue?@action=redirect&bpo=24992]: Fix error handling and a race condition (related to garbage collection) in collections.OrderedDict constructor.

- bpo-24881 [https://bugs.python.org/issue?@action=redirect&bpo=24881]: Fixed setting binary mode in Python implementation of FileIO on Windows and Cygwin. Patch from Akira Li.
- bpo-25578 [https://bugs.python.org/issue?@action=redirect&bpo=25578]: Fix (another) memory leak in SSLSocket.getpeercer().
- bpo-25530 [https://bugs.python.org/issue?@action=redirect&bpo=25530]: Disable the vulnerable SSLv3 protocol by default when creating ssl.SSLContext.
- bpo-25569 [https://bugs.python.org/issue?@action=redirect&bpo=25569]: Fix memory leak in SSLSocket.getpeercert().
- bpo-25471 [https://bugs.python.org/issue?@action=redirect&bpo=25471]: Sockets returned from accept() shouldn't appear to be nonblocking.
- bpo-25319 [https://bugs.python.org/issue?@action=redirect&bpo=25319]: When threading.Event is reinitialized, the underlying condition should use a regular lock rather than a recursive lock.
- bpo-21112 [https://bugs.python.org/issue?@action=redirect&bpo=21112]: Fix regression in unittest.expectedFailure on subclasses. Patch from Berker Peksag.
- bpo-24764 [https://bugs.python.org/issue?@action=redirect&bpo=24764]: cgi.FieldStorage.read_multi() now ignores the Content-Length header in part headers. Patch written by Peter Landry and reviewed by Pierre Quentel.
- bpo-24913 [https://bugs.python.org/issue?@action=redirect&bpo=24913]: Fix overrun error in deque.index(). Found by John Leitch and Bryce Darling.
- bpo-24774 [https://bugs.python.org/issue?@action=redirect&bpo=24774]: Fix docstring in http.server.test. Patch from Chiu-Hsiang Hsu.
- bpo-21159 [https://bugs.python.org/issue?@action=redirect&bpo=21159]: Improve message in configparser.InterpolationMissingOptionError. Patch from Łukasz Langa.
- bpo-20362 [https://bugs.python.org/issue?@action=redirect&bpo=20362]: Honour TestCase.longMessage correctly in assertRegex. Patch from Ilia Kurenkov.
- bpo-23572 [https://bugs.python.org/issue?@action=redirect&bpo=23572]: Fixed functools.singledispatch on classes with false metaclasses. Patch by Ethan Furman.
- asyncio: ensure_future() now accepts awaitable objects.

IDLE

- bpo-15348 [https://bugs.python.org/issue?@action=redirect&bpo=15348]: Stop the debugger engine (normally in a user process) before closing the debugger window (running in the IDLE process). This prevents the RuntimeErrors that were being caught and ignored.
- bpo-24455 [https://bugs.python.org/issue?@action=redirect&bpo=24455]: Prevent IDLE from hanging when a) closing the shell while the debugger is active (15347); b) closing the debugger with the [X] button (15348); and c) activating the debugger when already active (24455). The patch by Mark Roseman does this by making two changes. 1. Suspend and resume the gui.interaction method with the tcl vwait mechanism intended for this purpose (instead of root.mainloop & .quit). 2. In gui.run, allow any existing interaction to terminate first.
- Change ‘The program’ to ‘Your program’ in an IDLE ‘kill program?’ message to make it clearer that the program referred to is the currently running user program, not IDLE itself.
- bpo-24750 [https://bugs.python.org/issue?@action=redirect&bpo=24750]: Improve the appearance of the IDLE editor window status bar. Patch by Mark Roseman.
- bpo-25313 [https://bugs.python.org/issue?@action=redirect&bpo=25313]: Change the handling of new built-in text color themes to better address the compatibility problem introduced by the addition of IDLE Dark. Consistently use the revised idleConf.CurrentTheme everywhere in idlelib.
- bpo-24782 [https://bugs.python.org/issue?@action=redirect&bpo=24782]: Extension configuration is now a tab in the IDLE Preferences dialog rather than a separate dialog. The former tabs are now a sorted list. Patch by Mark Roseman.
- bpo-22726 [https://bugs.python.org/issue?@action=redirect&bpo=22726]: Re-activate the config dialog help button with some content about the other buttons and the new IDLE Dark theme.
- bpo-24820 [https://bugs.python.org/issue?@action=redirect&bpo=24820]: IDLE now has an ‘IDLE Dark’ built-in text color theme. It is more or less IDLE Classic inverted, with a cobalt blue background. Strings, comments, keywords, … are still green, red, orange, … . To use it with IDLEs released before November 2015, hit the ‘Save as New Custom Theme’ button and enter a new name, such as ‘Custom Dark’. The custom theme will work with any IDLE release, and can be modified.

- bpo-25224 [https://bugs.python.org/issue?@action=redirect&bpo=25224]: README.txt is now an idlelib index for IDLE developers and curious users. The previous user content is now in the IDLE doc chapter. ‘IDLE’ now means ‘Integrated Development and Learning Environment’.
- bpo-24820 [https://bugs.python.org/issue?@action=redirect&bpo=24820]: Users can now set breakpoint colors in Settings -> Custom Highlighting. Original patch by Mark Roseman.
- bpo-24972 [https://bugs.python.org/issue?@action=redirect&bpo=24972]: Inactive selection background now matches active selection background, as configured by users, on all systems. Found items are now always highlighted on Windows. Initial patch by Mark Roseman.
- bpo-24570 [https://bugs.python.org/issue?@action=redirect&bpo=24570]: Idle: make calltip and completion boxes appear on Macs affected by a tk regression. Initial patch by Mark Roseman.
- bpo-24988 [https://bugs.python.org/issue?@action=redirect&bpo=24988]: Idle ScrolledList context menus (used in debugger) now work on Mac Aqua. Patch by Mark Roseman.
- bpo-24801 [https://bugs.python.org/issue?@action=redirect&bpo=24801]: Make right-click for context menu work on Mac Aqua. Patch by Mark Roseman.
- bpo-25173 [https://bugs.python.org/issue?@action=redirect&bpo=25173]: Associate tkinter messageboxes with a specific widget. For Mac OSX, make them a ‘sheet’. Patch by Mark Roseman.
- bpo-25198 [https://bugs.python.org/issue?@action=redirect&bpo=25198]: Enhance the initial html viewer now used for Idle Help. Properly indent fixed-pitch text (patch by Mark Roseman). Give code snippet a very Sphinx-like light blueish-gray background. Re-use initial width and height set by users for shell and editor. When the Table of Contents (TOC) menu is used, put the section header at the top of the screen.
- bpo-25225 [https://bugs.python.org/issue?@action=redirect&bpo=25225]: Condense and rewrite Idle doc section on text colors.
- bpo-21995 [https://bugs.python.org/issue?@action=redirect&bpo=21995]: Explain some differences between IDLE and console Python.
- bpo-22820 [https://bugs.python.org/issue?@action=redirect&bpo=22820]: Explain need for *print* when running file from Idle editor.
- bpo-25224 [https://bugs.python.org/issue?@action=redirect&bpo=25224]: Doc: augment Idle feature list and no-subprocess section.

- bpo-25219 [https://bugs.python.org/issue?@action=redirect&bpo=25219]: Update doc for Idle command line options. Some were missing and notes were not correct.
- bpo-24861 [https://bugs.python.org/issue?@action=redirect&bpo=24861]: Most of idlelib is private and subject to change. Use idleib.idle.* to start Idle. See idlelib.__init__.__doc__.
- bpo-25199 [https://bugs.python.org/issue?@action=redirect&bpo=25199]: Idle: add synchronization comments for future maintainers.
- bpo-16893 [https://bugs.python.org/issue?@action=redirect&bpo=16893]: Replace help.txt with help.html for Idle doc display. The new idlelib/help.html is rstripped Doc/build/html/library/idle.html. It looks better than help.txt and will better document Idle as released. The tkinter html viewer that works for this file was written by Mark Roseman. The now unused EditorWindow.HelpDialog class and helt.txt file are deprecated.
- bpo-24199 [https://bugs.python.org/issue?@action=redirect&bpo=24199]: Deprecate unused idlelib.idlever with possible removal in 3.6.
- bpo-24790 [https://bugs.python.org/issue?@action=redirect&bpo=24790]: Remove extraneous code (which also create 2 & 3 conflicts).

Documentação

- bpo-22558 [https://bugs.python.org/issue?@action=redirect&bpo=22558]: Add remaining doc links to source code for Python-coded modules. Patch by Yoni Lavi.
- bpo-12067 [https://bugs.python.org/issue?@action=redirect&bpo=12067]: Rewrite Comparisons section in the Expressions chapter of the language reference. Some of the details of comparing mixed types were incorrect or ambiguous. NotImplemented is only relevant at a lower level than the Expressions chapter. Added details of comparing range() objects, and default behaviour and consistency suggestions for user-defined classes. Patch from Andy Maier.
- bpo-24952 [https://bugs.python.org/issue?@action=redirect&bpo=24952]: Clarify the default size argument of stack_size() in the “threading” and “_thread” modules. Patch from Mattip.
- bpo-23725 [https://bugs.python.org/issue?@action=redirect&bpo=23725]: Overhaul tempfile docs. Note deprecated status of mktemp. Patch from

Zbigniew Jędrzejewski-Szmek.

- bpo-24808 [https://bugs.python.org/issue?@action=redirect&bpo=24808]: Update the types of some PyTypeObject fields. Patch by Joseph Weston.
- bpo-22812 [https://bugs.python.org/issue?@action=redirect&bpo=22812]: Fix unittest discovery examples. Patch from Pam McA'Nulty.

Testes

- bpo-25449 [https://bugs.python.org/issue?@action=redirect&bpo=25449]: Added tests for OrderedDict subclasses.
- bpo-25099 [https://bugs.python.org/issue?@action=redirect&bpo=25099]: Make test_compileall not fail when an entry on sys.path cannot be written to (commonly seen in administrative installs on Windows).
- bpo-23919 [https://bugs.python.org/issue?@action=redirect&bpo=23919]: Prevents assert dialogs appearing in the test suite.
- `PCbuild\rt.bat` now accepts an unlimited number of arguments to pass along to regrtest.py. Previously there was a limit of 9.

Construção

- bpo-24915 [https://bugs.python.org/issue?@action=redirect&bpo=24915]: Add LLVM support for PGO builds and use the test suite to generate the profile data. Initial patch by Alecsandru Patrascu of Intel.
- bpo-24910 [https://bugs.python.org/issue?@action=redirect&bpo=24910]: Windows MSIs now have unique display names.
- bpo-24986 [https://bugs.python.org/issue?@action=redirect&bpo=24986]: It is now possible to build Python on Windows without errors when external libraries are not available.

Windows

- bpo-25450 [https://bugs.python.org/issue?@action=redirect&bpo=25450]: Updates shortcuts to start Python in installation directory.
- bpo-25164 [https://bugs.python.org/issue?@action=redirect&bpo=25164]: Changes default all-users install directory to match per-user directory.
- bpo-25143 [https://bugs.python.org/issue?@action=redirect&bpo=25143]: Improves installer error messages for unsupported platforms.

- bpo-25163 [https://bugs.python.org/issue?@action=redirect&bpo=25163]: Display correct directory in installer when using non-default settings.
- bpo-25361 [https://bugs.python.org/issue?@action=redirect&bpo=25361]: Disables use of SSE2 instructions in Windows 32-bit build
- bpo-25089 [https://bugs.python.org/issue?@action=redirect&bpo=25089]: Adds logging to installer for case where launcher is not selected on upgrade.
- bpo-25165 [https://bugs.python.org/issue?@action=redirect&bpo=25165]: Windows uninstallation should not remove launcher if other versions remain
- bpo-25112 [https://bugs.python.org/issue?@action=redirect&bpo=25112]: py.exe launcher is missing icons
- bpo-25102 [https://bugs.python.org/issue?@action=redirect&bpo=25102]: Windows installer does not precompile for -O or -OO.
- bpo-25081 [https://bugs.python.org/issue?@action=redirect&bpo=25081]: Makes Back button in installer go back to upgrade page when upgrading.
- bpo-25091 [https://bugs.python.org/issue?@action=redirect&bpo=25091]: Increases font size of the installer.
- bpo-25126 [https://bugs.python.org/issue?@action=redirect&bpo=25126]: Clarifies that the non-web installer will download some components.
- bpo-25213 [https://bugs.python.org/issue?@action=redirect&bpo=25213]: Restores requestedExecutionLevel to manifest to disable UAC virtualization.
- bpo-25022 [https://bugs.python.org/issue?@action=redirect&bpo=25022]: Removed very outdated PC/example_nt/ directory.

Ferramentas/Demos

- bpo-25440 [https://bugs.python.org/issue?@action=redirect&bpo=25440]: Fix output of python-config –extension-suffix.

Python 3.5.0 final

Data de lançamento: 2015-09-13

Construção

- bpo-25071 [https://bugs.python.org/issue?@action=redirect&bpo=25071]: Windows installer should not require TargetDir parameter when installing

quietly.

Python 3.5.0 release candidate 4

Data de lançamento: 2015-09-09

Biblioteca

- bpo-25029 [https://bugs.python.org/issue?@action=redirect&bpo=25029]: Fixes MemoryError in test_strptime.

Construção

- bpo-25027 [https://bugs.python.org/issue?@action=redirect&bpo=25027]: Reverts partial-static build options and adds vcruntime140.dll to Windows installation.

Python 3.5.0 release candidate 3

Data de lançamento: 2015-09-07

Núcleo e embutidos

- bpo-24305 [https://bugs.python.org/issue?@action=redirect&bpo=24305]: Prevent import subsystem stack frames from being counted by the warnings.warn(stacklevel=) parameter.
- bpo-24912 [https://bugs.python.org/issue?@action=redirect&bpo=24912]: Prevent __class__ assignment to immutable built-in objects.
- bpo-24975 [https://bugs.python.org/issue?@action=redirect&bpo=24975]: Fix AST compilation for **PEP 448** [https://peps.python.org/pep-0448/] syntax.

Biblioteca

- bpo-24917 [https://bugs.python.org/issue?@action=redirect&bpo=24917]: time_strftime() buffer over-read.

- bpo-24748 [https://bugs.python.org/issue?@action=redirect&bpo=24748]: To resolve a compatibility problem found with py2exe and pywin32, imp.load_dynamic() once again ignores previously loaded modules to support Python modules replacing themselves with extension modules. Patch by Petr Viktorin.
- bpo-24635 [https://bugs.python.org/issue?@action=redirect&bpo=24635]: Fixed a bug in typing.py where isinstance([], typing.Iterable) would return True once, then False on subsequent calls.
- bpo-24989 [https://bugs.python.org/issue?@action=redirect&bpo=24989]: Fixed buffer overread in BytesIO.readline() if a position is set beyond size. Based on patch by John Leitch.
- bpo-24913 [https://bugs.python.org/issue?@action=redirect&bpo=24913]: Fix overrun error in deque.index(). Found by John Leitch and Bryce Darling.

Python 3.5.0 release candidate 2

Data de lançamento: 2015-08-25

Núcleo e embutidos

- bpo-24769 [https://bugs.python.org/issue?@action=redirect&bpo=24769]: Interpreter now starts properly when dynamic loading is disabled. Patch by Petr Viktorin.
- bpo-21167 [https://bugs.python.org/issue?@action=redirect&bpo=21167]: NAN operations are now handled correctly when python is compiled with ICC even if -fp-model strict is not specified.
- bpo-24492 [https://bugs.python.org/issue?@action=redirect&bpo=24492]: A “package” lacking a __name__ attribute when trying to perform a `from .. import ...` statement will trigger an ImportError instead of an AttributeError.

Biblioteca

- bpo-24847 [https://bugs.python.org/issue?@action=redirect&bpo=24847]: Removes vcruntime140.dll dependency from Tcl/Tk.

- bpo-24839 [https://bugs.python.org/issue?@action=redirect&bpo=24839]: platform._syscmd_ver raises DeprecationWarning
- bpo-24867 [https://bugs.python.org/issue?@action=redirect&bpo=24867]: Fix Task.get_stack() for ‘async def’ coroutines

Python 3.5.0 release candidate 1

Data de lançamento: 2015-08-09

Núcleo e embutidos

- bpo-24667 [https://bugs.python.org/issue?@action=redirect&bpo=24667]: Resize odict in all cases that the underlying dict resizes.

Biblioteca

- bpo-24824 [https://bugs.python.org/issue?@action=redirect&bpo=24824]: Signatures of codecs.encode() and codecs.decode() now are compatible with pydoc.
- bpo-24634 [https://bugs.python.org/issue?@action=redirect&bpo=24634]: Importing uuid should not try to load libc on Windows
- bpo-24798 [https://bugs.python.org/issue?@action=redirect&bpo=24798]: _msvccompiler.py doesn’t properly support manifests
- bpo-4395 [https://bugs.python.org/issue?@action=redirect&bpo=4395]: Better testing and documentation of binary operators. Patch by Martin Panter.
- bpo-23973 [https://bugs.python.org/issue?@action=redirect&bpo=23973]: Update typing.py from GitHub repo.
- bpo-23004 [https://bugs.python.org/issue?@action=redirect&bpo=23004]: mock_open() now reads binary data correctly when the type of read_data is bytes. Initial patch by Aaron Hill.
- bpo-23888 [https://bugs.python.org/issue?@action=redirect&bpo=23888]: Handle fractional time in cookie expiry. Patch by ssh.
- bpo-23652 [https://bugs.python.org/issue?@action=redirect&bpo=23652]: Make it possible to compile the select module against the libc headers from the Linux Standard Base, which do not include some EPOLL macros. Patch by Matt Frank.

- bpo-22932 [https://bugs.python.org/issue?@action=redirect&bpo=22932]: Fix timezones in email.utils.formatdate. Patch from Dmitry Shachnev.
- bpo-23779 [https://bugs.python.org/issue?@action=redirect&bpo=23779]: imaplib raises TypeError if authenticator tries to abort. Patch from Craig Holmquist.
- bpo-23319 [https://bugs.python.org/issue?@action=redirect&bpo=23319]: Fix ctypes.BigEndianStructure, swap correctly bytes. Patch written by Matthieu Gautier.
- bpo-23254 [https://bugs.python.org/issue?@action=redirect&bpo=23254]: Document how to close the TCPServer listening socket. Patch from Martin Panter.
- bpo-19450 [https://bugs.python.org/issue?@action=redirect&bpo=19450]: Update Windows and OS X installer builds to use SQLite 3.8.11.
- bpo-17527 [https://bugs.python.org/issue?@action=redirect&bpo=17527]: Add PATCH to wsgiref.validator. Patch from Luca Sbardella.
- bpo-24791 [https://bugs.python.org/issue?@action=redirect&bpo=24791]: Fix grammar regression for call syntax: ‘g(*a or b)’.

IDLE

- bpo-23672 [https://bugs.python.org/issue?@action=redirect&bpo=23672]: Allow Idle to edit and run files with astral chars in name. Patch by Mohd Sanad Zaki Rizvi.
- bpo-24745 [https://bugs.python.org/issue?@action=redirect&bpo=24745]: Idle editor default font. Switch from Courier to platform-sensitive TkFixedFont. This should not affect current customized font selections. If there is a problem, edit $HOME/.idlerc/config-main.cfg and remove ‘`fontxxx`’ entries from [Editor Window]. Patch by Mark Roseman.
- bpo-21192 [https://bugs.python.org/issue?@action=redirect&bpo=21192]: Idle editor. When a file is run, put its name in the restart bar. Do not print false prompts. Original patch by Adnan Umer.
- bpo-13884 [https://bugs.python.org/issue?@action=redirect&bpo=13884]: Idle menus. Remove tearoff lines. Patch by Roger Serwy.

Documentação

- bpo-24129 [https://bugs.python.org/issue?@action=redirect&bpo=24129]: Clarify the reference documentation for name resolution. This includes removing the assumption that readers will be familiar with the name resolution scheme Python used prior to the introduction of lexical scoping for function namespaces. Patch by Ivan Levkivskyi.
- bpo-20769 [https://bugs.python.org/issue?@action=redirect&bpo=20769]: Improve reload() docs. Patch by Dorian Pula.
- bpo-23589 [https://bugs.python.org/issue?@action=redirect&bpo=23589]: Remove duplicate sentence from the FAQ. Patch by Yongzhi Pan.
- bpo-24729 [https://bugs.python.org/issue?@action=redirect&bpo=24729]: Correct IO tutorial to match implementation regarding encoding parameter to open function.

Testes

- bpo-24751 [https://bugs.python.org/issue?@action=redirect&bpo=24751]: When running regrtest with the `-w` command line option, a test run is no longer marked as a failure if all tests succeed when re-run.

Python 3.5.0 beta 4

Data de lançamento: 2015-07-26

Núcleo e embutidos

- bpo-23573 [https://bugs.python.org/issue?@action=redirect&bpo=23573]: Restored optimization of bytes.rfind() and bytearray.rfind() for single-byte argument on Linux.
- bpo-24569 [https://bugs.python.org/issue?@action=redirect&bpo=24569]: Make **PEP 448** [https://peps.python.org/pep-0448/] dictionary evaluation more consistent.
- bpo-24583 [https://bugs.python.org/issue?@action=redirect&bpo=24583]: Fix crash when set is mutated while being updated.
- bpo-24407 [https://bugs.python.org/issue?@action=redirect&bpo=24407]: Fix crash when dict is mutated while being updated.
- bpo-24619 [https://bugs.python.org/issue?@action=redirect&bpo=24619]: New approach for tokenizing async/await. As a consequence, it is now

possible to have one-line ‘async def foo(): await ..’ functions.

- bpo-24687 [https://bugs.python.org/issue?@action=redirect&bpo=24687]: Plug refleak on SyntaxError in function parameters annotations.
- bpo-15944 [https://bugs.python.org/issue?@action=redirect&bpo=15944]: memoryview: Allow arbitrary formats when casting to bytes. Patch by Martin Panter.

Biblioteca

- bpo-23441 [https://bugs.python.org/issue?@action=redirect&bpo=23441]: rcompleter now prints a tab character instead of displaying possible completions for an empty word. Initial patch by Martin Sekera.
- bpo-24683 [https://bugs.python.org/issue?@action=redirect&bpo=24683]: Fixed crashes in _json functions called with arguments of inappropriate type.
- bpo-21697 [https://bugs.python.org/issue?@action=redirect&bpo=21697]: shutil.copytree() now correctly handles symbolic links that point to directories. Patch by Eduardo Seabra and Thomas Kluyver.
- bpo-14373 [https://bugs.python.org/issue?@action=redirect&bpo=14373]: Fixed segmentation fault when gc.collect() is called during constructing lru_cache (C implementation).
- bpo-24695 [https://bugs.python.org/issue?@action=redirect&bpo=24695]: Fix a regression in traceback.print_exception(). If exc_traceback is None we shouldn’t print a traceback header like described in the documentation.
- bpo-24620 [https://bugs.python.org/issue?@action=redirect&bpo=24620]: Random.setstate() now validates the value of state last element.
- bpo-22485 [https://bugs.python.org/issue?@action=redirect&bpo=22485]: Fixed an issue that caused `inspect.getsource` to return incorrect results on nested functions.
- bpo-22153 [https://bugs.python.org/issue?@action=redirect&bpo=22153]: Improve unittest docs. Patch from Martin Panter and evilzero.
- bpo-24580 [https://bugs.python.org/issue?@action=redirect&bpo=24580]: Symbolic group references to open group in re patterns now are explicitly forbidden as well as numeric group references.
- bpo-24206 [https://bugs.python.org/issue?@action=redirect&bpo=24206]: Fixed __eq__ and __ne__ methods of inspect classes.
- bpo-24631 [https://bugs.python.org/issue?@action=redirect&bpo=24631]: Fixed regression in the timeit module with multiline setup.

- bpo-18622 [https://bugs.python.org/issue?@action=redirect&bpo=18622]: unittest.mock.mock_open().reset_mock would recurse infinitely. Patch from Nicola Palumbo and Laurent De Buyst.
- bpo-23661 [https://bugs.python.org/issue?@action=redirect&bpo=23661]: unittest.mock side_effects can now be exceptions again. This was a regression vs Python 3.4. Patch from Ignacio Rossi
- bpo-24608 [https://bugs.python.org/issue?@action=redirect&bpo=24608]: chunk.Chunk.read() now always returns bytes, not str.
- bpo-18684 [https://bugs.python.org/issue?@action=redirect&bpo=18684]: Fixed reading out of the buffer in the re module.
- bpo-24259 [https://bugs.python.org/issue?@action=redirect&bpo=24259]: tarfile now raises a ReadError if an archive is truncated inside a data segment.
- bpo-15014 [https://bugs.python.org/issue?@action=redirect&bpo=15014]: SMTP.auth() and SMTP.login() now support RFC 4954's optional initial-response argument to the SMTP AUTH command.
- bpo-24669 [https://bugs.python.org/issue?@action=redirect&bpo=24669]: Fix inspect.getsource() for 'async def' functions. Patch by Kai Groner.
- bpo-24688 [https://bugs.python.org/issue?@action=redirect&bpo=24688]: ast.get_docstring() for 'async def' functions.

Construção

- bpo-24603 [https://bugs.python.org/issue?@action=redirect&bpo=24603]: Update Windows builds and OS X 10.5 installer to use OpenSSL 1.0.2d.

Python 3.5.0 beta 3

Data de lançamento: 2015-07-05

Núcleo e embutidos

- bpo-24467 [https://bugs.python.org/issue?@action=redirect&bpo=24467]: Fixed possible buffer over-read in bytearray. The bytearray object now always allocates place for trailing null byte and it's buffer now is always null-terminated.
- Upgrade to Unicode 8.0.0.

- bpo-24345 [https://bugs.python.org/issue?@action=redirect&bpo=24345]: Add Py_tp_finalize slot for the stable ABI.
- bpo-24400 [https://bugs.python.org/issue?@action=redirect&bpo=24400]: Introduce a distinct type for **PEP 492** [https://peps.python.org/pep-0492/] coroutines; add types.CoroutineType, inspect.getcoroutinestate, inspect.getcoroutinelocals; coroutines no longer use CO_GENERATOR flag; sys.set_coroutine_wrapper works only for 'async def' coroutines; inspect.iscoroutine no longer uses collections.abc.Coroutine, it's intended to test for pure 'async def' coroutines only; add new opcode: GET_YIELD_FROM_ITER; fix generators wrapper used in types.coroutine to be instance of collections.abc.Generator; collections.abc.Awaitable and collections.abc.Coroutine can no longer be used to detect generator-based coroutines–use inspect.isawaitable instead.
- bpo-24450 [https://bugs.python.org/issue?@action=redirect&bpo=24450]: Add gi_yieldfrom to generators and cr_await to coroutines. Contributed by Benno Leslie and Yury Selivanov.
- bpo-19235 [https://bugs.python.org/issue?@action=redirect&bpo=19235]: Add new RecursionError exception. Patch by Georg Brandl.

Biblioteca

- bpo-21750 [https://bugs.python.org/issue?@action=redirect&bpo=21750]: mock_open.read_data can now be read from each instance, as it could in Python 3.3.
- bpo-24552 [https://bugs.python.org/issue?@action=redirect&bpo=24552]: Fix use after free in an error case of the _pickle module.
- bpo-24514 [https://bugs.python.org/issue?@action=redirect&bpo=24514]: tarfile now tolerates number fields consisting of only whitespace.
- bpo-19176 [https://bugs.python.org/issue?@action=redirect&bpo=19176]: Fixed doctype() related bugs in C implementation of ElementTree. A deprecation warning no longer issued by XMLParser subclass with default doctype() method. Direct call of doctype() now issues a warning. Parser's doctype() now is not called if target's doctype() is called. Based on patch by Martin Panter.
- bpo-20387 [https://bugs.python.org/issue?@action=redirect&bpo=20387]: Restore semantic round-trip correctness in tokenize/untokenize for tab-

indented blocks.

- bpo-24456 [https://bugs.python.org/issue?@action=redirect&bpo=24456]: Fixed possible buffer over-read in adpcm2lin() and lin2adpcm() functions of the audioop module.
- bpo-24336 [https://bugs.python.org/issue?@action=redirect&bpo=24336]: The contextmanager decorator now works with functions with keyword arguments called “func” and “self”. Patch by Martin Panter.
- bpo-24522 [https://bugs.python.org/issue?@action=redirect&bpo=24522]: Fix possible integer overflow in json accelerator module.
- bpo-24489 [https://bugs.python.org/issue?@action=redirect&bpo=24489]: ensure a previously set C errno doesn’t disturb cmath.polar().
- bpo-24408 [https://bugs.python.org/issue?@action=redirect&bpo=24408]: Fixed AttributeError in measure() and metrics() methods of tkinter.Font.
- bpo-14373 [https://bugs.python.org/issue?@action=redirect&bpo=14373]: C implementation of functools.lru_cache() now can be used with methods.
- bpo-24347 [https://bugs.python.org/issue?@action=redirect&bpo=24347]: Set KeyError if PyDict_GetItemWithError returns NULL.
- bpo-24348 [https://bugs.python.org/issue?@action=redirect&bpo=24348]: Drop superfluous incref/decref.
- bpo-24359 [https://bugs.python.org/issue?@action=redirect&bpo=24359]: Check for changed OrderedDict size during iteration.
- bpo-24368 [https://bugs.python.org/issue?@action=redirect&bpo=24368]: Support keyword arguments in OrderedDict methods.
- bpo-24362 [https://bugs.python.org/issue?@action=redirect&bpo=24362]: Simplify the C OrderedDict fast nodes resize logic.
- bpo-24377 [https://bugs.python.org/issue?@action=redirect&bpo=24377]: Fix a ref leak in OrderedDict.__repr__.
- bpo-24369 [https://bugs.python.org/issue?@action=redirect&bpo=24369]: Defend against key-changes during iteration.

Testes

- bpo-24373 [https://bugs.python.org/issue?@action=redirect&bpo=24373]: _testmultiphase and xxlimited now use tp_traverse and tp_finalize to avoid reference leaks encountered when combining tp_dealloc with

PyType_FromSpec (see bpo-16690 [https://bugs.python.org/issue?@action=redirect&bpo=16690] for details)

Documentação

- bpo-24458 [https://bugs.python.org/issue?@action=redirect&bpo=24458]: Update documentation to cover multi-phase initialization for extension modules (PEP 489). Patch by Petr Viktorin.
- bpo-24351 [https://bugs.python.org/issue?@action=redirect&bpo=24351]: Clarify what is meant by “identifier” in the context of string.Template instances.

Construção

- bpo-24432 [https://bugs.python.org/issue?@action=redirect&bpo=24432]: Update Windows builds and OS X 10.5 installer to use OpenSSL 1.0.2c.

Python 3.5.0 beta 2

Data de lançamento: 2015-05-31

Núcleo e embutidos

- bpo-24284 [https://bugs.python.org/issue?@action=redirect&bpo=24284]: The startswith and endswith methods of the str class no longer return True when finding the empty string and the indexes are completely out of range.
- bpo-24115 [https://bugs.python.org/issue?@action=redirect&bpo=24115]: Update uses of PyObject_IsTrue(), PyObject_Not(), PyObject_IsInstance(), PyObject_RichCompareBool() and _PyDict_Contains() to check for and handle errors correctly.
- bpo-24328 [https://bugs.python.org/issue?@action=redirect&bpo=24328]: Fix importing one character extension modules.
- bpo-11205 [https://bugs.python.org/issue?@action=redirect&bpo=11205]: In dictionary displays, evaluate the key before the value.

- bpo-24285 [https://bugs.python.org/issue?@action=redirect&bpo=24285]: Fixed regression that prevented importing extension modules from inside packages. Patch by Petr Viktorin.

Biblioteca

- bpo-23247 [https://bugs.python.org/issue?@action=redirect&bpo=23247]: Fix a crash in the StreamWriter.reset() of CJK codecs.
- bpo-24270 [https://bugs.python.org/issue?@action=redirect&bpo=24270]: Add math.isclose() and cmath.isclose() functions as per **PEP 485** [https://peps.python.org/pep-0485/]. Contributed by Chris Barker and Tal Einat.
- bpo-5633 [https://bugs.python.org/issue?@action=redirect&bpo=5633]: Fixed timeit when the statement is a string and the setup is not.
- bpo-24326 [https://bugs.python.org/issue?@action=redirect&bpo=24326]: Fixed audioop.ratecv() with non-default weightB argument. Original patch by David Moore.
- bpo-16991 [https://bugs.python.org/issue?@action=redirect&bpo=16991]: Add a C implementation of OrderedDict.
- bpo-23934 [https://bugs.python.org/issue?@action=redirect&bpo=23934]: Fix inspect.signature to fail correctly for builtin types lacking signature information. Initial patch by James Powell.

Python 3.5.0 beta 1

Data de lançamento: 2015-05-24

Núcleo e embutidos

- bpo-24276 [https://bugs.python.org/issue?@action=redirect&bpo=24276]: Fixed optimization of property descriptor getter.
- bpo-24268 [https://bugs.python.org/issue?@action=redirect&bpo=24268]: PEP 489: Multi-phase extension module initialization. Patch by Petr Viktorin.
- bpo-23955 [https://bugs.python.org/issue?@action=redirect&bpo=23955]: Add pyvenv.cfg option to suppress registry/environment lookup for generating sys.path on Windows.

- bpo-24257 [https://bugs.python.org/issue?@action=redirect&bpo=24257]: Fixed system error in the comparison of faked types.SimpleNamespace.
- bpo-22939 [https://bugs.python.org/issue?@action=redirect&bpo=22939]: Fixed integer overflow in iterator object. Patch by Clement Rouault.
- bpo-23985 [https://bugs.python.org/issue?@action=redirect&bpo=23985]: Fix a possible buffer overrun when deleting a slice from the front of a bytearray and then appending some other bytes data.
- bpo-24102 [https://bugs.python.org/issue?@action=redirect&bpo=24102]: Fixed exception type checking in standard error handlers.
- bpo-15027 [https://bugs.python.org/issue?@action=redirect&bpo=15027]: The UTF-32 encoder is now 3x to 7x faster.
- bpo-23290 [https://bugs.python.org/issue?@action=redirect&bpo=23290]: Optimize set_merge() for cases where the target is empty. (Contributed by Serhiy Storchaka.)
- bpo-2292 [https://bugs.python.org/issue?@action=redirect&bpo=2292]: PEP 448: Additional Unpacking Generalizations.
- bpo-24096 [https://bugs.python.org/issue?@action=redirect&bpo=24096]: Make warnings.warn_explicit more robust against mutation of the warnings.filters list.
- bpo-23996 [https://bugs.python.org/issue?@action=redirect&bpo=23996]: Avoid a crash when a delegated generator raises an unnormalized StopIteration exception. Patch by Stefan Behnel.
- bpo-23910 [https://bugs.python.org/issue?@action=redirect&bpo=23910]: Optimize property() getter calls. Patch by Joe Jevnik.
- bpo-23911 [https://bugs.python.org/issue?@action=redirect&bpo=23911]: Move path-based importlib bootstrap code to a separate frozen module.
- bpo-24192 [https://bugs.python.org/issue?@action=redirect&bpo=24192]: Fix namespace package imports.
- bpo-24022 [https://bugs.python.org/issue?@action=redirect&bpo=24022]: Fix tokenizer crash when processing undecodable source code.
- bpo-9951 [https://bugs.python.org/issue?@action=redirect&bpo=9951]: Added a hex() method to bytes, bytearray, and memoryview.
- bpo-22906 [https://bugs.python.org/issue?@action=redirect&bpo=22906]: PEP 479: Change StopIteration handling inside generators.
- bpo-24017 [https://bugs.python.org/issue?@action=redirect&bpo=24017]: PEP 492: Coroutines with async and await syntax.

Biblioteca

- bpo-14373 [https://bugs.python.org/issue?@action=redirect&bpo=14373]: Added C implementation of functools.lru_cache(). Based on patches by Matt Joiner and Alexey Kachayev.
- bpo-24230 [https://bugs.python.org/issue?@action=redirect&bpo=24230]: The tempfile module now accepts bytes for prefix, suffix and dir parameters and returns bytes in such situations (matching the os module APIs).
- bpo-22189 [https://bugs.python.org/issue?@action=redirect&bpo=22189]: collections.UserString now supports __getnewargs__(), __rmod__(), casefold(), format_map(), isprintable(), and maketrans(). Patch by Joe Jevnik.
- bpo-24244 [https://bugs.python.org/issue?@action=redirect&bpo=24244]: Prevents termination when an invalid format string is encountered on Windows in strftime.
- bpo-23973 [https://bugs.python.org/issue?@action=redirect&bpo=23973]: PEP 484: Add the typing module.
- bpo-23086 [https://bugs.python.org/issue?@action=redirect&bpo=23086]: The collections.abc.Sequence() abstract base class added *start* and *stop* parameters to the index() mixin. Patch by Devin Jeanpierre.
- bpo-20035 [https://bugs.python.org/issue?@action=redirect&bpo=20035]: Replaced the `tkinter._fix` module used for setting up the Tcl/Tk environment on Windows with a private function in the `_tkinter` module that makes no permanent changes to the environment.
- bpo-24257 [https://bugs.python.org/issue?@action=redirect&bpo=24257]: Fixed segmentation fault in sqlite3.Row constructor with faked cursor type.
- bpo-15836 [https://bugs.python.org/issue?@action=redirect&bpo=15836]: assertRaises(), assertRaisesRegex(), assertWarns() and assertWarnsRegex() assertments now check the type of the first argument to prevent possible user error. Based on patch by Daniel Wagner-Hall.
- bpo-9858 [https://bugs.python.org/issue?@action=redirect&bpo=9858]: Add missing method stubs to _io.RawIOBase. Patch by Laura Rupprecht.
- bpo-22955 [https://bugs.python.org/issue?@action=redirect&bpo=22955]: attrgetter, itemgetter and methodcaller objects in the operator module now

support pickling. Added readable and evaluable repr for these objects. Based on patch by Josh Rosenberg.

- bpo-22107 [https://bugs.python.org/issue?@action=redirect&bpo=22107]: tempfile.gettempdir() and tempfile.mkdtemp() now try again when a directory with the chosen name already exists on Windows as well as on Unix. tempfile.mkstemp() now fails early if parent directory is not valid (not exists or is a file) on Windows.
- bpo-23780 [https://bugs.python.org/issue?@action=redirect&bpo=23780]: Improved error message in os.path.join() with single argument.
- bpo-6598 [https://bugs.python.org/issue?@action=redirect&bpo=6598]: Increased time precision and random number range in email.utils.make_msgid() to strengthen the uniqueness of the message ID.
- bpo-24091 [https://bugs.python.org/issue?@action=redirect&bpo=24091]: Fixed various crashes in corner cases in C implementation of ElementTree.
- bpo-21931 [https://bugs.python.org/issue?@action=redirect&bpo=21931]: msilib.FCICreate() now raises TypeError in the case of a bad argument instead of a ValueError with a bogus FCI error number. Patch by Jeffrey Armstrong.
- bpo-13866 [https://bugs.python.org/issue?@action=redirect&bpo=13866]: *quote_via* argument added to urllib.parse.urlencode.
- bpo-20098 [https://bugs.python.org/issue?@action=redirect&bpo=20098]: New mangle_from policy option for email, default True for compat32, but False for all other policies.
- bpo-24211 [https://bugs.python.org/issue?@action=redirect&bpo=24211]: The email library now supports RFC 6532: it can generate headers using utf-8 instead of encoded words.
- bpo-16314 [https://bugs.python.org/issue?@action=redirect&bpo=16314]: Added support for the LZMA compression in distutils.
- bpo-21804 [https://bugs.python.org/issue?@action=redirect&bpo=21804]: poplib now supports RFC 6856 (UTF8).
- bpo-18682 [https://bugs.python.org/issue?@action=redirect&bpo=18682]: Optimized pprint functions for builtin scalar types.
- bpo-22027 [https://bugs.python.org/issue?@action=redirect&bpo=22027]: smtplib now supports RFC 6531 (SMTPUTF8).
- bpo-23488 [https://bugs.python.org/issue?@action=redirect&bpo=23488]: Random generator objects now consume 2x less memory on 64-bit.

- bpo-1322 [https://bugs.python.org/issue?@action=redirect&bpo=1322]: platform.dist() and platform.linux_distribution() functions are now deprecated. Initial patch by Vajrasky Kok.
- bpo-22486 [https://bugs.python.org/issue?@action=redirect&bpo=22486]: Added the math.gcd() function. The fractions.gcd() function now is deprecated. Based on patch by Mark Dickinson.
- bpo-24064 [https://bugs.python.org/issue?@action=redirect&bpo=24064]: Property() docstrings are now writeable. (Patch by Berker Peksag.)
- bpo-22681 [https://bugs.python.org/issue?@action=redirect&bpo=22681]: Added support for the koi8_t encoding.
- bpo-22682 [https://bugs.python.org/issue?@action=redirect&bpo=22682]: Added support for the kz1048 encoding.
- bpo-23796 [https://bugs.python.org/issue?@action=redirect&bpo=23796]: peek and read1 methods of BufferedReader now raise ValueError if they called on a closed object. Patch by John Hergenroeder.
- bpo-21795 [https://bugs.python.org/issue?@action=redirect&bpo=21795]: smtpd now supports the 8BITMIME extension whenever the new *decode_data* constructor argument is set to False.
- bpo-24155 [https://bugs.python.org/issue?@action=redirect&bpo=24155]: optimize heapq.heapify() for better cache performance when heapifying large lists.
- bpo-21800 [https://bugs.python.org/issue?@action=redirect&bpo=21800]: imaplib now supports RFC 5161 (enable), RFC 6855 (utf8/internationalized email) and automatically encodes non-ASCII usernames and passwords to UTF8.
- bpo-20274 [https://bugs.python.org/issue?@action=redirect&bpo=20274]: When calling a _sqlite.Connection, it now complains if passed any keyword arguments. Previously it silently ignored them.
- bpo-20274 [https://bugs.python.org/issue?@action=redirect&bpo=20274]: Remove ignored and erroneous “kwargs” parameters from three METH_VARARGS methods on _sqlite.Connection.
- bpo-24134 [https://bugs.python.org/issue?@action=redirect&bpo=24134]: assertRaises(), assertRaisesRegex(), assertWarns() and assertWarnsRegex() checks now emits a deprecation warning when callable is None or keyword arguments except msg is passed in the context manager mode.

- bpo-24018 [https://bugs.python.org/issue?@action=redirect&bpo=24018]: Add a collections.abc.Generator abstract base class. Contributed by Stefan Behnel.
- bpo-23880 [https://bugs.python.org/issue?@action=redirect&bpo=23880]: Tkinter's getint() and getdouble() now support Tcl_Obj. Tkinter's getdouble() now supports any numbers (in particular int).
- bpo-22619 [https://bugs.python.org/issue?@action=redirect&bpo=22619]: Added negative limit support in the traceback module. Based on patch by Dmitry Kazakov.
- bpo-24094 [https://bugs.python.org/issue?@action=redirect&bpo=24094]: Fix possible crash in json.encode with poorly behaved dict subclasses.
- bpo-9246 [https://bugs.python.org/issue?@action=redirect&bpo=9246]: On POSIX, os.getcwd() now supports paths longer than 1025 bytes. Patch written by William Orr.
- bpo-17445 [https://bugs.python.org/issue?@action=redirect&bpo=17445]: add difflib.diff_bytes() to support comparison of byte strings (fixes a regression from Python 2).
- bpo-23917 [https://bugs.python.org/issue?@action=redirect&bpo=23917]: Fall back to sequential compilation when ProcessPoolExecutor doesn't exist. Patch by Claudiu Popa.
- bpo-23008 [https://bugs.python.org/issue?@action=redirect&bpo=23008]: Fixed resolving attributes with boolean value is False in pydoc.
- Fix asyncio issue 235: LifoQueue and PriorityQueue's put didn't increment unfinished tasks (this bug was introduced when JoinableQueue was merged with Queue).
- bpo-23908 [https://bugs.python.org/issue?@action=redirect&bpo=23908]: os functions now reject paths with embedded null character on Windows instead of silently truncating them.
- bpo-23728 [https://bugs.python.org/issue?@action=redirect&bpo=23728]: binascii.crc_hqx() could return an integer outside of the range 0-0xffff for empty data.
- bpo-23887 [https://bugs.python.org/issue?@action=redirect&bpo=23887]: urllib.error.HTTPError now has a proper repr() representation. Patch by Berker Peksag.
- asyncio: New event loop APIs: set_task_factory() and get_task_factory().
- asyncio: async() function is deprecated in favour of ensure_future().

- bpo-24178 [https://bugs.python.org/issue?@action=redirect&bpo=24178]: asyncio.Lock, Condition, Semaphore, and BoundedSemaphore support new ‘async with’ syntax. Contributed by Yury Selivanov.
- bpo-24179 [https://bugs.python.org/issue?@action=redirect&bpo=24179]: Support ‘async for’ for asyncio.StreamReader. Contributed by Yury Selivanov.
- bpo-24184 [https://bugs.python.org/issue?@action=redirect&bpo=24184]: Add AsyncIterator and AsyncIterable ABCs to collections.abc. Contributed by Yury Selivanov.
- bpo-22547 [https://bugs.python.org/issue?@action=redirect&bpo=22547]: Implement informative __repr__ for inspect.BoundArguments. Contributed by Yury Selivanov.
- bpo-24190 [https://bugs.python.org/issue?@action=redirect&bpo=24190]: Implement inspect.BoundArgument.apply_defaults() method. Contributed by Yury Selivanov.
- bpo-20691 [https://bugs.python.org/issue?@action=redirect&bpo=20691]: Add ‘follow_wrapped’ argument to inspect.Signature.from_callable() and inspect.signature(). Contributed by Yury Selivanov.
- bpo-24248 [https://bugs.python.org/issue?@action=redirect&bpo=24248]: Deprecate inspect.Signature.from_function() and inspect.Signature.from_builtin().
- bpo-23898 [https://bugs.python.org/issue?@action=redirect&bpo=23898]: Fix inspect.classify_class_attrs() to support attributes with overloaded __eq__ and __bool__. Patch by Mike Bayer.
- bpo-24298 [https://bugs.python.org/issue?@action=redirect&bpo=24298]: Fix inspect.signature() to correctly unwrap wrappers around bound methods.

IDLE

- bpo-23184 [https://bugs.python.org/issue?@action=redirect&bpo=23184]: remove unused names and imports in idlelib. Initial patch by Al Sweigart.

Testes

- bpo-21520 [https://bugs.python.org/issue?@action=redirect&bpo=21520]: test_zipfile no longer fails if the word ‘bad’ appears anywhere in the name of the current directory.

- bpo-9517 [https://bugs.python.org/issue?@action=redirect&bpo=9517]: Move script_helper into the support package. Patch by Christie Wilson.

Documentação

- bpo-22155 [https://bugs.python.org/issue?@action=redirect&bpo=22155]: Add File Handlers subsection with createfilehandler to tkinter doc. Remove obsolete example from FAQ. Patch by Martin Panter.
- bpo-24029 [https://bugs.python.org/issue?@action=redirect&bpo=24029]: Document the name binding behavior for submodule imports.
- bpo-24077 [https://bugs.python.org/issue?@action=redirect&bpo=24077]: Fix typo in man page for -I command option: -s, not -S

Ferramentas/Demos

- bpo-24000 [https://bugs.python.org/issue?@action=redirect&bpo=24000]: Improved Argument Clinic's mapping of converters to legacy "format units". Updated the documentation to match.
- bpo-24001 [https://bugs.python.org/issue?@action=redirect&bpo=24001]: Argument Clinic converters now use accept={type} instead of types={'type'} to specify the types the converter accepts.
- bpo-23330 [https://bugs.python.org/issue?@action=redirect&bpo=23330]: h2py now supports arbitrary filenames in #include.
- bpo-24031 [https://bugs.python.org/issue?@action=redirect&bpo=24031]: make patchcheck now supports git checkouts, too.

Python 3.5.0 alpha 4

Data de lançamento: 2015-04-19

Núcleo e embutidos

- bpo-22980 [https://bugs.python.org/issue?@action=redirect&bpo=22980]: Under Linux, GNU/KFreeBSD and the Hurd, C extensions now include the architecture triplet in the extension name, to make it easy to test builds for different ABIs in the same working tree. Under OS X, the

extension name now includes **PEP 3149** [https://peps.python.org/pep-3149/]-style information.

- bpo-22631 [https://bugs.python.org/issue?@action=redirect&bpo=22631]: Added Linux-specific socket constant CAN_RAW_FD_FRAMES. Patch courtesy of Joe Jevnik.
- bpo-23731 [https://bugs.python.org/issue?@action=redirect&bpo=23731]: Implement **PEP 488** [https://peps.python.org/pep-0488/]: removal of .pyo files.
- bpo-23726 [https://bugs.python.org/issue?@action=redirect&bpo=23726]: Don't enable GC for user subclasses of non-GC types that don't add any new fields. Patch by Eugene Toder.
- bpo-23309 [https://bugs.python.org/issue?@action=redirect&bpo=23309]: Avoid a deadlock at shutdown if a daemon thread is aborted while it is holding a lock to a buffered I/O object, and the main thread tries to use the same I/O object (typically stdout or stderr). A fatal error is emitted instead.
- bpo-22977 [https://bugs.python.org/issue?@action=redirect&bpo=22977]: Fixed formatting Windows error messages on Wine. Patch by Martin Panter.
- bpo-23466 [https://bugs.python.org/issue?@action=redirect&bpo=23466]: %c, %o, %x, and %X in bytes formatting now raise TypeError on non-integer input.
- bpo-24044 [https://bugs.python.org/issue?@action=redirect&bpo=24044]: Fix possible null pointer dereference in list.sort in out of memory conditions.
- bpo-21354 [https://bugs.python.org/issue?@action=redirect&bpo=21354]: PyCFunction_New function is exposed by python DLL again.

Biblioteca

- bpo-23840 [https://bugs.python.org/issue?@action=redirect&bpo=23840]: tokenize.open() now closes the temporary binary file on error to fix a resource warning.
- bpo-16914 [https://bugs.python.org/issue?@action=redirect&bpo=16914]: new debuglevel 2 in smtplib adds timestamps to debug output.
- bpo-7159 [https://bugs.python.org/issue?@action=redirect&bpo=7159]: urllib.request now supports sending auth credentials automatically after the first 401. This enhancement is a superset of the enhancement from bpo-19494

[https://bugs.python.org/issue?@action=redirect&bpo=19494] and supersedes that change.

- bpo-23703 [https://bugs.python.org/issue?@action=redirect&bpo=23703]: Fix a regression in urljoin() introduced in 901e4e52b20a. Patch by Demian Brecht.
- bpo-4254 [https://bugs.python.org/issue?@action=redirect&bpo=4254]: Adds _curses.update_lines_cols(). Patch by Arnon Yaari
- bpo-19933 [https://bugs.python.org/issue?@action=redirect&bpo=19933]: Provide default argument for ndigits in round. Patch by Vajrasky Kok.
- bpo-23193 [https://bugs.python.org/issue?@action=redirect&bpo=23193]: Add a numeric_owner parameter to tarfile.TarFile.extract and tarfile.TarFile.extractall. Patch by Michael Vogt and Eric Smith.
- bpo-23342 [https://bugs.python.org/issue?@action=redirect&bpo=23342]: Add a subprocess.run() function than returns a CalledProcess instance for a more consistent API than the existing call* functions.
- bpo-21217 [https://bugs.python.org/issue?@action=redirect&bpo=21217]: inspect.getsourcelines() now tries to compute the start and end lines from the code object, fixing an issue when a lambda function is used as decorator argument. Patch by Thomas Ballinger and Allison Kaptur.
- bpo-24521 [https://bugs.python.org/issue?@action=redirect&bpo=24521]: Fix possible integer overflows in the pickle module.
- bpo-22931 [https://bugs.python.org/issue?@action=redirect&bpo=22931]: Allow '[' and ']' in cookie values.
- The keywords attribute of functools.partial is now always a dictionary.
- bpo-23811 [https://bugs.python.org/issue?@action=redirect&bpo=23811]: Add missing newline to the PyCompileError error message. Patch by Alex Shkop.
- bpo-21116 [https://bugs.python.org/issue?@action=redirect&bpo=21116]: Avoid blowing memory when allocating a multiprocessing shared array that's larger than 50% of the available RAM. Patch by Médéric Boquien.
- bpo-22982 [https://bugs.python.org/issue?@action=redirect&bpo=22982]: Improve BOM handling when seeking to multiple positions of a writable text file.
- bpo-23464 [https://bugs.python.org/issue?@action=redirect&bpo=23464]: Removed deprecated asyncio JoinableQueue.
- bpo-23529 [https://bugs.python.org/issue?@action=redirect&bpo=23529]: Limit the size of decompressed data when reading from GzipFile, BZ2File or

LZMAFile. This defeats denial of service attacks using compressed bombs (i.e. compressed payloads which decompress to a huge size). Patch by Martin Panter and Nikolaus Rath.

- bpo-21859 [https://bugs.python.org/issue?@action=redirect&bpo=21859]: Added Python implementation of io.FileIO.
- bpo-23865 [https://bugs.python.org/issue?@action=redirect&bpo=23865]: close() methods in multiple modules now are idempotent and more robust at shutdown. If they need to release multiple resources, all are released even if errors occur.
- bpo-23400 [https://bugs.python.org/issue?@action=redirect&bpo=23400]: Raise same exception on both Python 2 and 3 if sem_open is not available. Patch by Davin Potts.
- bpo-10838 [https://bugs.python.org/issue?@action=redirect&bpo=10838]: The subprocess now module includes SubprocessError and TimeoutError in its list of exported names for the users wild enough to use `from subprocess import *`.
- bpo-23411 [https://bugs.python.org/issue?@action=redirect&bpo=23411]: Added DefragResult, ParseResult, SplitResult, DefragResultBytes, ParseResultBytes, and SplitResultBytes to urllib.parse.__all__. Patch by Martin Panter.
- bpo-23881 [https://bugs.python.org/issue?@action=redirect&bpo=23881]: urllib.request.ftpwrapper constructor now closes the socket if the FTP connection failed to fix a ResourceWarning.
- bpo-23853 [https://bugs.python.org/issue?@action=redirect&bpo=23853]: **`socket.socket.sendall()`** does no more reset the socket timeout each time data is sent successfully. The socket timeout is now the maximum total duration to send all data.
- bpo-22721 [https://bugs.python.org/issue?@action=redirect&bpo=22721]: An order of multiline pprint output of set or dict containing orderable and non-orderable elements no longer depends on iteration order of set or dict.
- bpo-15133 [https://bugs.python.org/issue?@action=redirect&bpo=15133]: _tkinter.tkapp.getboolean() now supports Tcl_Obj and always returns bool. tkinter.BooleanVar now validates input values (accepted bool, int, str, and Tcl_Obj). tkinter.BooleanVar.get() now always returns bool.
- bpo-10590 [https://bugs.python.org/issue?@action=redirect&bpo=10590]: xml.sax.parseString() now supports string argument.

- bpo-23338 [https://bugs.python.org/issue?@action=redirect&bpo=23338]: Fixed formatting ctypes error messages on Cygwin. Patch by Makoto Kato.
- bpo-15582 [https://bugs.python.org/issue?@action=redirect&bpo=15582]: inspect.getdoc() now follows inheritance chains.
- bpo-2175 [https://bugs.python.org/issue?@action=redirect&bpo=2175]: SAX parsers now support a character stream of InputSource object.
- bpo-16840 [https://bugs.python.org/issue?@action=redirect&bpo=16840]: Tkinter now supports 64-bit integers added in Tcl 8.4 and arbitrary precision integers added in Tcl 8.5.
- bpo-23834 [https://bugs.python.org/issue?@action=redirect&bpo=23834]: Fix socket.sendto(), use the C Py_ssize_t type to store the result of sendto() instead of the C int type.
- bpo-23618 [https://bugs.python.org/issue?@action=redirect&bpo=23618]: `socket.socket.connect()` now waits until the connection completes instead of raising `InterruptedError` if the connection is interrupted by signals, signal handlers don't raise an exception and the socket is blocking or has a timeout. `socket.socket.connect()` still raise `InterruptedError` for non-blocking sockets.
- bpo-21526 [https://bugs.python.org/issue?@action=redirect&bpo=21526]: Tkinter now supports new boolean type in Tcl 8.5.
- bpo-23836 [https://bugs.python.org/issue?@action=redirect&bpo=23836]: Fix the faulthandler module to handle reentrant calls to its signal handlers.
- bpo-23838 [https://bugs.python.org/issue?@action=redirect&bpo=23838]: linecache now clears the cache and returns an empty result on MemoryError.
- bpo-10395 [https://bugs.python.org/issue?@action=redirect&bpo=10395]: Added os.path.commonpath(). Implemented in posixpath and ntpath. Based on patch by Rafik Draoui.
- bpo-23611 [https://bugs.python.org/issue?@action=redirect&bpo=23611]: Serializing more "lookupable" objects (such as unbound methods or nested classes) now are supported with pickle protocols < 4.
- bpo-13583 [https://bugs.python.org/issue?@action=redirect&bpo=13583]: sqlite3.Row now supports slice indexing.
- bpo-18473 [https://bugs.python.org/issue?@action=redirect&bpo=18473]: Fixed 2to3 and 3to2 compatible pickle mappings. Fixed ambiguous reverse mappings. Added many new mappings. Import mapping is no longer applied to modules already mapped with full name mapping.

- bpo-23485 [https://bugs.python.org/issue?@action=redirect&bpo=23485]: select.select() is now retried automatically with the recomputed timeout when interrupted by a signal, except if the signal handler raises an exception. This change is part of the **PEP 475** [https://peps.python.org/pep-0475/].
- bpo-23752 [https://bugs.python.org/issue?@action=redirect&bpo=23752]: When built from an existing file descriptor, io.FileIO() now only calls fstat() once. Before fstat() was called twice, which was not necessary.
- bpo-23704 [https://bugs.python.org/issue?@action=redirect&bpo=23704]: collections.deque() objects now support __add__, __mul__, and __imul__().
- bpo-23171 [https://bugs.python.org/issue?@action=redirect&bpo=23171]: csv.Writer.writerow() now supports arbitrary iterables.
- bpo-23745 [https://bugs.python.org/issue?@action=redirect&bpo=23745]: The new email header parser now handles duplicate MIME parameter names without error, similar to how get_param behaves.
- bpo-22117 [https://bugs.python.org/issue?@action=redirect&bpo=22117]: Fix os.utime(), it now rounds the timestamp towards minus infinity (-inf) instead of rounding towards zero.
- bpo-23310 [https://bugs.python.org/issue?@action=redirect&bpo=23310]: Fix MagicMock's initializer to work with __methods__, just like configure_mock(). Patch by Kasia Jachim.

Construção

- bpo-23817 [https://bugs.python.org/issue?@action=redirect&bpo=23817]: FreeBSD now uses "1.0" in the SOVERSION as other operating systems, instead of just "1".
- bpo-23501 [https://bugs.python.org/issue?@action=redirect&bpo=23501]: Argument Clinic now generates code into separate files by default.

Testes

- bpo-23799 [https://bugs.python.org/issue?@action=redirect&bpo=23799]: Added test.support.start_threads() for running and cleaning up multiple threads.

- bpo-22390 [https://bugs.python.org/issue?@action=redirect&bpo=22390]: test.regrtest now emits a warning if temporary files or directories are left after running a test.

Ferramentas/Demos

- bpo-18128 [https://bugs.python.org/issue?@action=redirect&bpo=18128]: pygettext now uses standard +NNNN format in the POT-Creation-Date header.
- bpo-23935 [https://bugs.python.org/issue?@action=redirect&bpo=23935]: Argument Clinic's understanding of format units accepting bytes, bytearrays, and buffers is now consistent with both the documentation and the implementation.
- bpo-23944 [https://bugs.python.org/issue?@action=redirect&bpo=23944]: Argument Clinic now wraps long impl prototypes at column 78.
- bpo-20586 [https://bugs.python.org/issue?@action=redirect&bpo=20586]: Argument Clinic now ensures that functions without docstrings have signatures.
- bpo-23492 [https://bugs.python.org/issue?@action=redirect&bpo=23492]: Argument Clinic now generates argument parsing code with PyArg_Parse instead of PyArg_ParseTuple if possible.
- bpo-23500 [https://bugs.python.org/issue?@action=redirect&bpo=23500]: Argument Clinic is now smarter about generating the "#ifndef" (empty) definition of the methoddef macro: it's only generated once, even if Argument Clinic processes the same symbol multiple times, and it's emitted at the end of all processing rather than immediately after the first use.

API C

- bpo-23998 [https://bugs.python.org/issue?@action=redirect&bpo=23998]: PyImport_ReInitLock() now checks for lock allocation error

Python 3.5.0 alpha 3

Data de lançamento: 2015-03-28

Núcleo e embutidos

- bpo-23573 [https://bugs.python.org/issue?@action=redirect&bpo=23573]: Increased performance of string search operations (str.find, str.index, str.count, the in operator, str.split, str.partition) with arguments of different kinds (UCS1, UCS2, UCS4).
- bpo-23753 [https://bugs.python.org/issue?@action=redirect&bpo=23753]: Python doesn't support anymore platforms without stat() or fstat(), these functions are always required.
- bpo-23681 [https://bugs.python.org/issue?@action=redirect&bpo=23681]: The -b option now affects comparisons of bytes with int.
- bpo-23632 [https://bugs.python.org/issue?@action=redirect&bpo=23632]: Memoryviews now allow tuple indexing (including for multi-dimensional memoryviews).
- bpo-23192 [https://bugs.python.org/issue?@action=redirect&bpo=23192]: Fixed generator lambdas. Patch by Bruno Cauet.
- bpo-23629 [https://bugs.python.org/issue?@action=redirect&bpo=23629]: Fix the default __sizeof__ implementation for variable-sized objects.

Biblioteca

- bpo-14260 [https://bugs.python.org/issue?@action=redirect&bpo=14260]: The groupindex attribute of regular expression pattern object now is non-modifiable mapping.
- bpo-23792 [https://bugs.python.org/issue?@action=redirect&bpo=23792]: Ignore KeyboardInterrupt when the pydoc pager is active. This mimics the behavior of the standard unix pagers, and prevents pipepager from shutting down while the pager itself is still running.
- bpo-23775 [https://bugs.python.org/issue?@action=redirect&bpo=23775]: pprint() of OrderedDict now outputs the same representation as repr().
- bpo-23765 [https://bugs.python.org/issue?@action=redirect&bpo=23765]: Removed IsBadStringPtr calls in ctypes
- bpo-22364 [https://bugs.python.org/issue?@action=redirect&bpo=22364]: Improved some re error messages using regex for hints.
- bpo-23742 [https://bugs.python.org/issue?@action=redirect&bpo=23742]: ntpath.expandvars() no longer loses unbalanced single quotes.
- bpo-21717 [https://bugs.python.org/issue?@action=redirect&bpo=21717]: The zipfile.ZipFile.open function now supports 'x' (exclusive creation) mode.

- bpo-21802 [https://bugs.python.org/issue?@action=redirect&bpo=21802]: The reader in BufferedRWPair now is closed even when closing writer failed in BufferedRWPair.close().
- bpo-23622 [https://bugs.python.org/issue?@action=redirect&bpo=23622]: Unknown escapes in regular expressions that consist of `'\'` and ASCII letter now raise a deprecation warning and will be forbidden in Python 3.6.
- bpo-23671 [https://bugs.python.org/issue?@action=redirect&bpo=23671]: string.Template now allows specifying the “self” parameter as a keyword argument. string.Formatter now allows specifying the “self” and the “format_string” parameters as keyword arguments.
- bpo-23502 [https://bugs.python.org/issue?@action=redirect&bpo=23502]: The pprint module now supports mapping proxies.
- bpo-17530 [https://bugs.python.org/issue?@action=redirect&bpo=17530]: pprint now wraps long bytes objects and bytearrays.
- bpo-22687 [https://bugs.python.org/issue?@action=redirect&bpo=22687]: Fixed some corner cases in breaking words in tetxtwrap. Got rid of quadratic complexity in breaking long words.
- bpo-4727 [https://bugs.python.org/issue?@action=redirect&bpo=4727]: The copy module now uses pickle protocol 4 (PEP 3154) and supports copying of instances of classes whose __new__ method takes keyword-only arguments.
- bpo-23491 [https://bugs.python.org/issue?@action=redirect&bpo=23491]: Added a zipapp module to support creating executable zip file archives of Python code. Registered “.pyz” and “.pyzw” extensions on Windows for these archives (PEP 441).
- bpo-23657 [https://bugs.python.org/issue?@action=redirect&bpo=23657]: Avoid explicit checks for str in zipapp, adding support for pathlib.Path objects as arguments.
- bpo-23688 [https://bugs.python.org/issue?@action=redirect&bpo=23688]: Added support of arbitrary bytes-like objects and avoided unnecessary copying of memoryview in gzip.GzipFile.write(). Original patch by Wolfgang Maier.
- bpo-23252 [https://bugs.python.org/issue?@action=redirect&bpo=23252]: Added support for writing ZIP files to unseekable streams.
- bpo-23647 [https://bugs.python.org/issue?@action=redirect&bpo=23647]: Increase imaplib’s MAXLINE to accommodate modern mailbox sizes.

- bpo-23539 [https://bugs.python.org/issue?@action=redirect&bpo=23539]: If body is None, http.client.HTTPConnection.request now sets Content-Length to 0 for PUT, POST, and PATCH headers to avoid 411 errors from some web servers.
- bpo-22351 [https://bugs.python.org/issue?@action=redirect&bpo=22351]: The nntplib.NNTP constructor no longer leaves the connection and socket open until the garbage collector cleans them up. Patch by Martin Panter.
- bpo-23704 [https://bugs.python.org/issue?@action=redirect&bpo=23704]: collections.deque() objects now support methods for index(), insert(), and copy(). This allows deques to be registered as a MutableSequence and it improves their substitutability for lists.
- bpo-23715 [https://bugs.python.org/issue?@action=redirect&bpo=23715]: `signal.sigwaitinfo()` and `signal.sigtimedwait()` are now retried when interrupted by a signal not in the *sigset* parameter, if the signal handler does not raise an exception. signal.sigtimedwait() recomputes the timeout with a monotonic clock when it is retried.
- bpo-23001 [https://bugs.python.org/issue?@action=redirect&bpo=23001]: Few functions in modules mmap, ossaudiodev, socket, ssl, and codecs, that accepted only read-only bytes-like object now accept writable bytes-like object too.
- bpo-23646 [https://bugs.python.org/issue?@action=redirect&bpo=23646]: If time.sleep() is interrupted by a signal, the sleep is now retried with the recomputed delay, except if the signal handler raises an exception (PEP 475).
- bpo-23136 [https://bugs.python.org/issue?@action=redirect&bpo=23136]: _strptime now uniformly handles all days in week 0, including Dec 30 of previous year. Based on patch by Jim Carroll.
- bpo-23700 [https://bugs.python.org/issue?@action=redirect&bpo=23700]: Iterator of NamedTemporaryFile now keeps a reference to NamedTemporaryFile instance. Patch by Bohuslav Kabrda.
- bpo-22903 [https://bugs.python.org/issue?@action=redirect&bpo=22903]: The fake test case created by unittest.loader when it fails importing a test module is now picklable.
- bpo-22181 [https://bugs.python.org/issue?@action=redirect&bpo=22181]: On Linux, os.urandom() now uses the new getrandom() syscall if available, syscall introduced in the Linux kernel 3.17. It is more reliable and

more secure, because it avoids the need of a file descriptor and waits until the kernel has enough entropy.

- bpo-2211 [https://bugs.python.org/issue?@action=redirect&bpo=2211]: Updated the implementation of the http.cookies.Morsel class. Setting attributes key, value and coded_value directly now is deprecated. update() and setdefault() now transform and check keys. Comparing for equality now takes into account attributes key, value and coded_value. copy() now returns a Morsel, not a dict. repr() now contains all attributes. Optimized checking keys and quoting values. Added new tests. Original patch by Demian Brecht.
- bpo-18983 [https://bugs.python.org/issue?@action=redirect&bpo=18983]: Allow selection of output units in timeit. Patch by Julian Gindi.
- bpo-23631 [https://bugs.python.org/issue?@action=redirect&bpo=23631]: Fix traceback.format_list when a traceback has been mutated.
- bpo-23568 [https://bugs.python.org/issue?@action=redirect&bpo=23568]: Add rdivmod support to MagicMock() objects. Patch by Håkan Lövdahl.
- bpo-2052 [https://bugs.python.org/issue?@action=redirect&bpo=2052]: Add charset parameter to HtmlDiff.make_file().
- bpo-23668 [https://bugs.python.org/issue?@action=redirect&bpo=23668]: Support os.truncate and os.ftruncate on Windows.
- bpo-23138 [https://bugs.python.org/issue?@action=redirect&bpo=23138]: Fixed parsing cookies with absent keys or values in cookiejar. Patch by Demian Brecht.
- bpo-23051 [https://bugs.python.org/issue?@action=redirect&bpo=23051]: multiprocessing.Pool methods imap() and imap_unordered() now handle exceptions raised by an iterator. Patch by Alon Diamant and Davin Potts.
- bpo-23581 [https://bugs.python.org/issue?@action=redirect&bpo=23581]: Add matmul support to MagicMock. Patch by Håkan Lövdahl.
- bpo-23566 [https://bugs.python.org/issue?@action=redirect&bpo=23566]: enable(), register(), dump_traceback() and dump_traceback_later() functions of faulthandler now accept file descriptors. Patch by Wei Wu.
- bpo-22928 [https://bugs.python.org/issue?@action=redirect&bpo=22928]: Disabled HTTP header injections in http.client. Original patch by Demian Brecht.
- bpo-23615 [https://bugs.python.org/issue?@action=redirect&bpo=23615]: Modules bz2, tarfile and tokenize now can be reloaded with imp.reload(). Patch

by Thomas Kluyver.
- bpo-23605 [https://bugs.python.org/issue?@action=redirect&bpo=23605]: os.walk() now calls os.scandir() instead of os.listdir(). The usage of os.scandir() reduces the number of calls to os.stat(). Initial patch written by Ben Hoyt.

Construção

- bpo-23585 [https://bugs.python.org/issue?@action=redirect&bpo=23585]: make patchcheck will ensure the interpreter is built.

Testes

- bpo-23583 [https://bugs.python.org/issue?@action=redirect&bpo=23583]: Added tests for standard IO streams in IDLE.
- bpo-22289 [https://bugs.python.org/issue?@action=redirect&bpo=22289]: Prevent test_urllib2net failures due to ftp connection timeout.

Ferramentas/Demos

- bpo-22826 [https://bugs.python.org/issue?@action=redirect&bpo=22826]: The result of open() in Tools/freeze/bkfile.py is now better compatible with regular files (in particular it now supports the context management protocol).

Python 3.5.0 alpha 2

Data de lançamento: 2015-03-09

Núcleo e embutidos

- bpo-23571 [https://bugs.python.org/issue?@action=redirect&bpo=23571]: PyObject_Call() and PyCFunction_Call() now raise a SystemError if a function returns a result and raises an exception. The SystemError is chained to the previous exception.

Biblioteca

- bpo-22524 [https://bugs.python.org/issue?@action=redirect&bpo=22524]: New os.scandir() function, part of the **PEP 471** [https://peps.python.org/pep-0471/]: "os.scandir() function – a better and faster directory iterator". Patch written by Ben Hoyt.
- bpo-23103 [https://bugs.python.org/issue?@action=redirect&bpo=23103]: Reduced the memory consumption of IPv4Address and IPv6Address.
- bpo-21793 [https://bugs.python.org/issue?@action=redirect&bpo=21793]: BaseHTTPRequestHandler again logs response code as numeric, not as stringified enum. Patch by Demian Brecht.
- bpo-23476 [https://bugs.python.org/issue?@action=redirect&bpo=23476]: In the ssl module, enable OpenSSL's X509_V_FLAG_TRUSTED_FIRST flag on certificate stores when it is available.
- bpo-23576 [https://bugs.python.org/issue?@action=redirect&bpo=23576]: Avoid stalling in SSL reads when EOF has been reached in the SSL layer but the underlying connection hasn't been closed.
- bpo-23504 [https://bugs.python.org/issue?@action=redirect&bpo=23504]: Added an __all__ to the types module.
- bpo-23563 [https://bugs.python.org/issue?@action=redirect&bpo=23563]: Optimized utility functions in urllib.parse.
- bpo-7830 [https://bugs.python.org/issue?@action=redirect&bpo=7830]: Flatten nested functools.partial.
- bpo-20204 [https://bugs.python.org/issue?@action=redirect&bpo=20204]: Added the __module__ attribute to _tkinter classes.
- bpo-19980 [https://bugs.python.org/issue?@action=redirect&bpo=19980]: Improved help() for non-recognized strings. help('') now shows the help on str. help('help') now shows the help on help(). Original patch by Mark Lawrence.
- bpo-23521 [https://bugs.python.org/issue?@action=redirect&bpo=23521]: Corrected pure python implementation of timedelta division. Eliminated OverflowError from `timedelta * float` for some floats; Corrected rounding in timedelta true division.
- bpo-21619 [https://bugs.python.org/issue?@action=redirect&bpo=21619]: Popen objects no longer leave a zombie after exit in the with statement if the pipe was broken. Patch by Martin Panter.

- bpo-22936 [https://bugs.python.org/issue?@action=redirect&bpo=22936]: Make it possible to show local variables in tracebacks for both the traceback module and unittest.
- bpo-15955 [https://bugs.python.org/issue?@action=redirect&bpo=15955]: Add an option to limit the output size in bz2.decompress(). Patch by Nikolaus Rath.
- bpo-6639 [https://bugs.python.org/issue?@action=redirect&bpo=6639]: Module-level turtle functions no longer raise TclError after closing the window.
- bpo-814253 [https://bugs.python.org/issue?@action=redirect&bpo=814253]: Group references and conditional group references now work in lookbehind assertions in regular expressions. (See also: bpo-9179 [https://bugs.python.org/issue?@action=redirect&bpo=9179])
- bpo-23215 [https://bugs.python.org/issue?@action=redirect&bpo=23215]: Multibyte codecs with custom error handlers that ignores errors consumed too much memory and raised SystemError or MemoryError. Original patch by Aleksi Torhamo.
- bpo-5700 [https://bugs.python.org/issue?@action=redirect&bpo=5700]: io.FileIO() called flush() after closing the file. flush() was not called in close() if closefd=False.
- bpo-23374 [https://bugs.python.org/issue?@action=redirect&bpo=23374]: Fixed pydoc failure with non-ASCII files when stdout encoding differs from file system encoding (e.g. on Mac OS).
- bpo-23481 [https://bugs.python.org/issue?@action=redirect&bpo=23481]: Remove RC4 from the SSL module's default cipher list.
- bpo-21548 [https://bugs.python.org/issue?@action=redirect&bpo=21548]: Fix pydoc.synopsis() and pydoc.apropos() on modules with empty docstrings.
- bpo-22885 [https://bugs.python.org/issue?@action=redirect&bpo=22885]: Fixed arbitrary code execution vulnerability in the dbm.dumb module. Original patch by Claudiu Popa.
- bpo-23239 [https://bugs.python.org/issue?@action=redirect&bpo=23239]: ssl.match_hostname() now supports matching of IP addresses.
- bpo-23146 [https://bugs.python.org/issue?@action=redirect&bpo=23146]: Fix mishandling of absolute Windows paths with forward slashes in pathlib.
- bpo-23096 [https://bugs.python.org/issue?@action=redirect&bpo=23096]: Pickle representation of floats with protocol 0 now is the same for both

Python and C implementations.

- bpo-19105 [https://bugs.python.org/issue?@action=redirect&bpo=19105]: pprint now more efficiently uses free space at the right.
- bpo-14910 [https://bugs.python.org/issue?@action=redirect&bpo=14910]: Add allow_abbrev parameter to argparse.ArgumentParser. Patch by Jonathan Paugh, Steven Bethard, paul j3 and Daniel Eriksson.
- bpo-21717 [https://bugs.python.org/issue?@action=redirect&bpo=21717]: tarfile.open() now supports ‘x’ (exclusive creation) mode.
- bpo-23344 [https://bugs.python.org/issue?@action=redirect&bpo=23344]: marshal.dumps() is now 20-25% faster on average.
- bpo-20416 [https://bugs.python.org/issue?@action=redirect&bpo=20416]: marshal.dumps() with protocols 3 and 4 is now 40-50% faster on average.
- bpo-23421 [https://bugs.python.org/issue?@action=redirect&bpo=23421]: Fixed compression in tarfile CLI. Patch by wdv4758h.
- bpo-23367 [https://bugs.python.org/issue?@action=redirect&bpo=23367]: Fix possible overflows in the unicodedata module.
- bpo-23361 [https://bugs.python.org/issue?@action=redirect&bpo=23361]: Fix possible overflow in Windows subprocess creation code.
- logging.handlers.QueueListener now takes a respect_handler_level keyword argument which, if set to True, will pass messages to handlers taking handler levels into account.
- bpo-19705 [https://bugs.python.org/issue?@action=redirect&bpo=19705]: turtledemo now has a visual sorting algorithm demo. Original patch from Jason Yeo.
- bpo-23801 [https://bugs.python.org/issue?@action=redirect&bpo=23801]: Fix issue where cgi.FieldStorage did not always ignore the entire preamble to a multipart body.

Construção

- bpo-23445 [https://bugs.python.org/issue?@action=redirect&bpo=23445]: pydebug builds now use “gcc -Og” where possible, to make the resulting executable faster.
- bpo-23686 [https://bugs.python.org/issue?@action=redirect&bpo=23686]: Update OS X 10.5 installer build to use OpenSSL 1.0.2a.

API C

- bpo-20204 [https://bugs.python.org/issue?@action=redirect&bpo=20204]: Deprecation warning is now raised for builtin types without the __module__ attribute.

Windows

- bpo-23465 [https://bugs.python.org/issue?@action=redirect&bpo=23465]: Implement **PEP 486** [https://peps.python.org/pep-0486/] - Make the Python Launcher aware of virtual environments. Patch by Paul Moore.
- bpo-23437 [https://bugs.python.org/issue?@action=redirect&bpo=23437]: Make user scripts directory versioned on Windows. Patch by Paul Moore.

Python 3.5.0 alpha 1

Data de lançamento: 2015-02-08

Núcleo e embutidos

- bpo-23285 [https://bugs.python.org/issue?@action=redirect&bpo=23285]: PEP 475 - EINTR handling.
- bpo-22735 [https://bugs.python.org/issue?@action=redirect&bpo=22735]: Fix many edge cases (including crashes) involving custom mro() implementations.
- bpo-22896 [https://bugs.python.org/issue?@action=redirect&bpo=22896]: Avoid using PyObject_AsCharBuffer(), PyObject_AsReadBuffer() and PyObject_AsWriteBuffer().
- bpo-21295 [https://bugs.python.org/issue?@action=redirect&bpo=21295]: Revert some changes (bpo-16795 [https://bugs.python.org/issue?@action=redirect&bpo=16795]) to AST line numbers and column offsets that constituted a regression.
- bpo-22986 [https://bugs.python.org/issue?@action=redirect&bpo=22986]: Allow changing an object's __class__ between a dynamic type and static type in some cases.

- bpo-15859 [https://bugs.python.org/issue?@action=redirect&bpo=15859]: PyUnicode_EncodeFSDefault(), PyUnicode_EncodeMBCS() and PyUnicode_EncodeCodePage() now raise an exception if the object is not a Unicode object. For PyUnicode_EncodeFSDefault(), it was already the case on platforms other than Windows. Patch written by Campbell Barton.
- bpo-21408 [https://bugs.python.org/issue?@action=redirect&bpo=21408]: The default __ne__() now returns NotImplemented if __eq__() returned NotImplemented. Original patch by Martin Panter.
- bpo-23321 [https://bugs.python.org/issue?@action=redirect&bpo=23321]: Fixed a crash in str.decode() when error handler returned replacement string longer than malformed input data.
- bpo-22286 [https://bugs.python.org/issue?@action=redirect&bpo=22286]: The “backslashreplace” error handlers now works with decoding and translating.
- bpo-23253 [https://bugs.python.org/issue?@action=redirect&bpo=23253]: Delay-load ShellExecute[AW] in os.startfile for reduced startup overhead on Windows.
- bpo-22038 [https://bugs.python.org/issue?@action=redirect&bpo=22038]: pyatomic.h now uses stdatomic.h or GCC built-in functions for atomic memory access if available. Patch written by Vitor de Lima and Gustavo Temple.
- bpo-20284 [https://bugs.python.org/issue?@action=redirect&bpo=20284]: %-interpolation (aka printf) formatting added for bytes and bytearray.
- bpo-23048 [https://bugs.python.org/issue?@action=redirect&bpo=23048]: Fix jumping out of an infinite while loop in the pdb.
- bpo-20335 [https://bugs.python.org/issue?@action=redirect&bpo=20335]: bytes constructor now raises TypeError when encoding or errors is specified with non-string argument. Based on patch by Renaud Blanch.
- bpo-22834 [https://bugs.python.org/issue?@action=redirect&bpo=22834]: If the current working directory ends up being set to a non-existent directory then import will no longer raise FileNotFoundError.
- bpo-22869 [https://bugs.python.org/issue?@action=redirect&bpo=22869]: Move the interpreter startup & shutdown code to a new dedicated pylifecycle.c module
- bpo-22847 [https://bugs.python.org/issue?@action=redirect&bpo=22847]: Improve method cache efficiency.

- bpo-22335 [https://bugs.python.org/issue?@action=redirect&bpo=22335]: Fix crash when trying to enlarge a bytearray to 0x7fffffff bytes on a 32-bit platform.
- bpo-22653 [https://bugs.python.org/issue?@action=redirect&bpo=22653]: Fix an assertion failure in debug mode when doing a reentrant dict insertion in debug mode.
- bpo-22643 [https://bugs.python.org/issue?@action=redirect&bpo=22643]: Fix integer overflow in Unicode case operations (upper, lower, title, swapcase, casefold).
- bpo-17636 [https://bugs.python.org/issue?@action=redirect&bpo=17636]: Circular imports involving relative imports are now supported.
- bpo-22604 [https://bugs.python.org/issue?@action=redirect&bpo=22604]: Fix assertion error in debug mode when dividing a complex number by (nan+0j).
- bpo-21052 [https://bugs.python.org/issue?@action=redirect&bpo=21052]: Do not raise ImportWarning when sys.path_hooks or sys.meta_path are set to None.
- bpo-16518 [https://bugs.python.org/issue?@action=redirect&bpo=16518]: Use ‘bytes-like object required’ in error messages that previously used the far more cryptic “‘x’ does not support the buffer protocol.
- bpo-22470 [https://bugs.python.org/issue?@action=redirect&bpo=22470]: Fixed integer overflow issues in “backslashreplace”, “xmlcharrefreplace”, and “surrogatepass” error handlers.
- bpo-22540 [https://bugs.python.org/issue?@action=redirect&bpo=22540]: speed up `PyObject_IsInstance` and `PyObject_IsSubclass` in the common case that the second argument has metaclass `type`.
- bpo-18711 [https://bugs.python.org/issue?@action=redirect&bpo=18711]: Add a new `PyErr_FormatV` function, similar to `PyErr_Format` but accepting a `va_list` argument.
- bpo-22520 [https://bugs.python.org/issue?@action=redirect&bpo=22520]: Fix overflow checking when generating the repr of a unicode object.
- bpo-22519 [https://bugs.python.org/issue?@action=redirect&bpo=22519]: Fix overflow checking in PyBytes_Repr.
- bpo-22518 [https://bugs.python.org/issue?@action=redirect&bpo=22518]: Fix integer overflow issues in latin-1 encoding.
- bpo-16324 [https://bugs.python.org/issue?@action=redirect&bpo=16324]: _charset parameter of MIMEText now also accepts email.charset.Charset

instances. Initial patch by Claude Paroz.

- bpo-1764286 [https://bugs.python.org/issue?@action=redirect&bpo=1764286]: Fix inspect.getsource() to support decorated functions. Patch by Claudiu Popa.
- bpo-18554 [https://bugs.python.org/issue?@action=redirect&bpo=18554]: os.__all__ includes posix functions.
- bpo-21391 [https://bugs.python.org/issue?@action=redirect&bpo=21391]: Use os.path.abspath in the shutil module.
- bpo-11471 [https://bugs.python.org/issue?@action=redirect&bpo=11471]: avoid generating a JUMP_FORWARD instruction at the end of an if-block if there is no else-clause. Original patch by Eugene Toder.
- bpo-22215 [https://bugs.python.org/issue?@action=redirect&bpo=22215]: Now ValueError is raised instead of TypeError when str or bytes argument contains not permitted null character or byte.
- bpo-22258 [https://bugs.python.org/issue?@action=redirect&bpo=22258]: Fix the internal function set_inheritable() on Illumos. This platform exposes the function `ioctl(FIOCLEX)`, but calling it fails with errno is ENOTTY: “Inappropriate ioctl for device”. set_inheritable() now falls back to the slower `fcntl()` (`F_GETFD` and then `F_SETFD`).
- bpo-21389 [https://bugs.python.org/issue?@action=redirect&bpo=21389]: Displaying the __qualname__ of the underlying function in the repr of a bound method.
- bpo-22206 [https://bugs.python.org/issue?@action=redirect&bpo=22206]: Using pthread, PyThread_create_key() now sets errno to ENOMEM and returns -1 (error) on integer overflow.
- bpo-20184 [https://bugs.python.org/issue?@action=redirect&bpo=20184]: Argument Clinic based signature introspection added for 30 of the builtin functions.
- bpo-22116 [https://bugs.python.org/issue?@action=redirect&bpo=22116]: C functions and methods (of the ‘builtin_function_or_method’ type) can now be weakref’ed. Patch by Wei Wu.
- bpo-22077 [https://bugs.python.org/issue?@action=redirect&bpo=22077]: Improve index error messages for bytearrays, bytes, lists, and tuples by adding ‘or slices’. Added ‘, not <typename>’ for bytearrays. Original patch by Claudiu Popa.
- bpo-20179 [https://bugs.python.org/issue?@action=redirect&bpo=20179]: Apply Argument Clinic to bytes and bytearray. Patch by Tal Einat.

- bpo-22082 [https://bugs.python.org/issue?@action=redirect&bpo=22082]: Clear interned strings in slotdefs.
- Upgrade Unicode database to Unicode 7.0.0.
- bpo-21897 [https://bugs.python.org/issue?@action=redirect&bpo=21897]: Fix a crash with the f_locals attribute with closure variables when frame.clear() has been called.
- bpo-21205 [https://bugs.python.org/issue?@action=redirect&bpo=21205]: Add a new `__qualname__` attribute to generator, the qualified name, and use it in the representation of a generator (`repr(gen)`). The default name of the generator (`__name__` attribute) is now get from the function instead of the code. Use `gen.gi_code.co_name` to get the name of the code.
- bpo-21669 [https://bugs.python.org/issue?@action=redirect&bpo=21669]: With the aid of heuristics in SyntaxError.__init__, the parser now attempts to generate more meaningful (or at least more search engine friendly) error messages when “exec” and “print” are used as statements.
- bpo-21642 [https://bugs.python.org/issue?@action=redirect&bpo=21642]: In the conditional if-else expression, allow an integer written with no space between itself and the `else` keyword (e.g. `True if 42else False`) to be valid syntax.
- bpo-21523 [https://bugs.python.org/issue?@action=redirect&bpo=21523]: Fix over-pessimistic computation of the stack effect of some opcodes in the compiler. This also fixes a quadratic compilation time issue noticeable when compiling code with a large number of “and” and “or” operators.
- bpo-21418 [https://bugs.python.org/issue?@action=redirect&bpo=21418]: Fix a crash in the builtin function super() when called without argument and without current frame (ex: embedded Python).
- bpo-21425 [https://bugs.python.org/issue?@action=redirect&bpo=21425]: Fix flushing of standard streams in the interactive interpreter.
- bpo-21435 [https://bugs.python.org/issue?@action=redirect&bpo=21435]: In rare cases, when running finalizers on objects in cyclic trash a bad pointer dereference could occur due to a subtle flaw in internal iteration logic.
- bpo-21377 [https://bugs.python.org/issue?@action=redirect&bpo=21377]: PyBytes_Concat() now tries to concatenate in-place when the first argument has a reference count of 1. Patch by Nikolaus Rath.
- bpo-20355 [https://bugs.python.org/issue?@action=redirect&bpo=20355]: -W command line options now have higher priority than the PYTHONWARNINGS environment variable. Patch by Arfrever.

- bpo-21274 [https://bugs.python.org/issue?@action=redirect&bpo=21274]: Define PATH_MAX for GNU/Hurd in Python/pythonrun.c.
- bpo-20904 [https://bugs.python.org/issue?@action=redirect&bpo=20904]: Support setting FPU precision on m68k.
- bpo-21209 [https://bugs.python.org/issue?@action=redirect&bpo=21209]: Fix sending tuples to custom generator objects with the yield from syntax.
- bpo-21193 [https://bugs.python.org/issue?@action=redirect&bpo=21193]: pow(a, b, c) now raises ValueError rather than TypeError when b is negative. Patch by Josh Rosenberg.
- bpo-21176 [https://bugs.python.org/issue?@action=redirect&bpo=21176]: PEP 465: Add the '@' operator for matrix multiplication.
- bpo-21134 [https://bugs.python.org/issue?@action=redirect&bpo=21134]: Fix segfault when str is called on an uninitialized UnicodeEncodeError, UnicodeDecodeError, or UnicodeTranslateError object.
- bpo-19537 [https://bugs.python.org/issue?@action=redirect&bpo=19537]: Fix PyUnicode_DATA() alignment under m68k. Patch by Andreas Schwab.
- bpo-20929 [https://bugs.python.org/issue?@action=redirect&bpo=20929]: Add a type cast to avoid shifting a negative number.
- bpo-20731 [https://bugs.python.org/issue?@action=redirect&bpo=20731]: Properly position in source code files even if they are opened in text mode. Patch by Serhiy Storchaka.
- bpo-20637 [https://bugs.python.org/issue?@action=redirect&bpo=20637]: Key-sharing now also works for instance dictionaries of subclasses. Patch by Peter Ingebretson.
- bpo-8297 [https://bugs.python.org/issue?@action=redirect&bpo=8297]: Attributes missing from modules now include the module name in the error text. Original patch by ysj.ray.
- bpo-19995 [https://bugs.python.org/issue?@action=redirect&bpo=19995]: %c, %o, %x, and %X now raise TypeError on non-integer input.
- bpo-19655 [https://bugs.python.org/issue?@action=redirect&bpo=19655]: The ASDL parser - used by the build process to generate code for managing the Python AST in C - was rewritten. The new parser is self contained and does not require to carry long the spark.py parser-generator library; spark.py was removed from the source base.
- bpo-12546 [https://bugs.python.org/issue?@action=redirect&bpo=12546]: Allow `\x00` to be used as a fill character when using str, int, float, and complex

__format__ methods.

- bpo-20480 [https://bugs.python.org/issue?@action=redirect&bpo=20480]: Add ipaddress.reverse_pointer. Patch by Leon Weber.
- bpo-13598 [https://bugs.python.org/issue?@action=redirect&bpo=13598]: Modify string.Formatter to support auto-numbering of replacement fields. It now matches the behavior of str.format() in this regard. Patches by Phil Elson and Ramchandra Apte.
- bpo-8931 [https://bugs.python.org/issue?@action=redirect&bpo=8931]: Make alternate formatting ('#') for type 'c' raise an exception. In versions prior to 3.5, '#' with 'c' had no effect. Now specifying it is an error. Patch by Torsten Landschoff.
- bpo-23165 [https://bugs.python.org/issue?@action=redirect&bpo=23165]: Perform overflow checks before allocating memory in the _Py_char2wchar function.

Biblioteca

- bpo-23399 [https://bugs.python.org/issue?@action=redirect&bpo=23399]: pyvenv creates relative symlinks where possible.
- bpo-20289 [https://bugs.python.org/issue?@action=redirect&bpo=20289]: cgi.FieldStorage() now supports the context management protocol.
- bpo-13128 [https://bugs.python.org/issue?@action=redirect&bpo=13128]: Print response headers for CONNECT requests when debuglevel > 0. Patch by Demian Brecht.
- bpo-15381 [https://bugs.python.org/issue?@action=redirect&bpo=15381]: Optimized io.BytesIO to make less allocations and copyings.
- bpo-22818 [https://bugs.python.org/issue?@action=redirect&bpo=22818]: Splitting on a pattern that could match an empty string now raises a warning. Patterns that can only match empty strings are now rejected.
- bpo-23099 [https://bugs.python.org/issue?@action=redirect&bpo=23099]: Closing io.BytesIO with exported buffer is rejected now to prevent corrupting exported buffer.
- bpo-23326 [https://bugs.python.org/issue?@action=redirect&bpo=23326]: Removed __ne__ implementations. Since fixing default __ne__ implementation in bpo-21408 [https://bugs.python.org/issue?@action=redirect&bpo=21408] they are redundant.

- bpo-23363 [https://bugs.python.org/issue?@action=redirect&bpo=23363]: Fix possible overflow in itertools.permutations.
- bpo-23364 [https://bugs.python.org/issue?@action=redirect&bpo=23364]: Fix possible overflow in itertools.product.
- bpo-23366 [https://bugs.python.org/issue?@action=redirect&bpo=23366]: Fixed possible integer overflow in itertools.combinations.
- bpo-23369 [https://bugs.python.org/issue?@action=redirect&bpo=23369]: Fixed possible integer overflow in _json.encode_basestring_ascii.
- bpo-23353 [https://bugs.python.org/issue?@action=redirect&bpo=23353]: Fix the exception handling of generators in PyEval_EvalFrameEx(). At entry, save or swap the exception state even if PyEval_EvalFrameEx() is called with throwflag=0. At exit, the exception state is now always restored or swapped, not only if why is WHY_YIELD or WHY_RETURN. Patch co-written with Antoine Pitrou.
- bpo-14099 [https://bugs.python.org/issue?@action=redirect&bpo=14099]: Restored support of writing ZIP files to tellable but non-seekable streams.
- bpo-14099 [https://bugs.python.org/issue?@action=redirect&bpo=14099]: Writing to ZipFile and reading multiple ZipExtFiles is threadsafe now.
- bpo-19361 [https://bugs.python.org/issue?@action=redirect&bpo=19361]: JSON decoder now raises JSONDecodeError instead of ValueError.
- bpo-18518 [https://bugs.python.org/issue?@action=redirect&bpo=18518]: timeit now rejects statements which can’t be compiled outside a function or a loop (e.g. “return” or “break”).
- bpo-23094 [https://bugs.python.org/issue?@action=redirect&bpo=23094]: Fixed readline with frames in Python implementation of pickle.
- bpo-23268 [https://bugs.python.org/issue?@action=redirect&bpo=23268]: Fixed bugs in the comparison of ipaddress classes.
- bpo-21408 [https://bugs.python.org/issue?@action=redirect&bpo=21408]: Removed incorrect implementations of __ne__() which didn’t returned NotImplemented if __eq__() returned NotImplemented. The default __ne__() now works correctly.
- bpo-19996 [https://bugs.python.org/issue?@action=redirect&bpo=19996]: **`email.feedparser.FeedParser`** now handles (malformed) headers with no key rather than assuming the body has started.
- bpo-20188 [https://bugs.python.org/issue?@action=redirect&bpo=20188]: Support Application-Layer Protocol Negotiation (ALPN) in the ssl module.

- bpo-23133 [https://bugs.python.org/issue?@action=redirect&bpo=23133]: Pickling of ipaddress objects now produces more compact and portable representation.
- bpo-23248 [https://bugs.python.org/issue?@action=redirect&bpo=23248]: Update ssl error codes from latest OpenSSL git master.
- bpo-23266 [https://bugs.python.org/issue?@action=redirect&bpo=23266]: Much faster implementation of ipaddress.collapse_addresses() when there are many non-consecutive addresses.
- bpo-23098 [https://bugs.python.org/issue?@action=redirect&bpo=23098]: 64-bit dev_t is now supported in the os module.
- bpo-21817 [https://bugs.python.org/issue?@action=redirect&bpo=21817]: When an exception is raised in a task submitted to a ProcessPoolExecutor, the remote traceback is now displayed in the parent process. Patch by Claudiu Popa.
- bpo-15955 [https://bugs.python.org/issue?@action=redirect&bpo=15955]: Add an option to limit output size when decompressing LZMA data. Patch by Nikolaus Rath and Martin Panter.
- bpo-23250 [https://bugs.python.org/issue?@action=redirect&bpo=23250]: In the http.cookies module, capitalize “HttpOnly” and “Secure” as they are written in the standard.
- bpo-23063 [https://bugs.python.org/issue?@action=redirect&bpo=23063]: In the distutils’ check command, fix parsing of reST with code or code-block directives.
- bpo-23209 [https://bugs.python.org/issue?@action=redirect&bpo=23209]: selectors.BaseSelector.get_key() now raises a RuntimeError if the selector is closed. And selectors.BaseSelector.close() now clears its internal reference to the selector mapping to break a reference cycle. Initial patch written by Martin Richard. (See also: bpo-23225 [https://bugs.python.org/issue?@action=redirect&bpo=23225])
- bpo-17911 [https://bugs.python.org/issue?@action=redirect&bpo=17911]: Provide a way to seed the linecache for a PEP-302 module without actually loading the code.
- bpo-17911 [https://bugs.python.org/issue?@action=redirect&bpo=17911]: Provide a new object API for traceback, including the ability to not lookup lines at all until the traceback is actually rendered, without any trace of the original objects being kept alive.

- bpo-19777 [https://bugs.python.org/issue?@action=redirect&bpo=19777]: Provide a home() classmethod on Path objects. Contributed by Victor Salgado and Mayank Tripathi.
- bpo-23206 [https://bugs.python.org/issue?@action=redirect&bpo=23206]: Make `json.dumps(..., ensure_ascii=False)` as fast as the default case of `ensure_ascii=True`. Patch by Naoki Inada.
- bpo-23185 [https://bugs.python.org/issue?@action=redirect&bpo=23185]: Add math.inf and math.nan constants.
- bpo-23186 [https://bugs.python.org/issue?@action=redirect&bpo=23186]: Add ssl.SSLObject.shared_ciphers() and ssl.SSLSocket.shared_ciphers() to fetch the client's list ciphers sent at handshake.
- bpo-23143 [https://bugs.python.org/issue?@action=redirect&bpo=23143]: Remove compatibility with OpenSSLs older than 0.9.8.
- bpo-23132 [https://bugs.python.org/issue?@action=redirect&bpo=23132]: Improve performance and introspection support of comparison methods created by functool.total_ordering.
- bpo-19776 [https://bugs.python.org/issue?@action=redirect&bpo=19776]: Add an expanduser() method on Path objects.
- bpo-23112 [https://bugs.python.org/issue?@action=redirect&bpo=23112]: Fix SimpleHTTPServer to correctly carry the query string and fragment when it redirects to add a trailing slash.
- bpo-21793 [https://bugs.python.org/issue?@action=redirect&bpo=21793]: Added http.HTTPStatus enums (i.e. HTTPStatus.OK, HTTPStatus.NOT_FOUND). Patch by Demian Brecht.
- bpo-23093 [https://bugs.python.org/issue?@action=redirect&bpo=23093]: In the io, module allow more operations to work on detached streams.
- bpo-23111 [https://bugs.python.org/issue?@action=redirect&bpo=23111]: In the ftplib, make ssl.PROTOCOL_SSLv23 the default protocol version.
- bpo-22585 [https://bugs.python.org/issue?@action=redirect&bpo=22585]: On OpenBSD 5.6 and newer, os.urandom() now calls getentropy(), instead of reading /dev/urandom, to get pseudo-random bytes.
- bpo-19104 [https://bugs.python.org/issue?@action=redirect&bpo=19104]: pprint now produces evaluable output for wrapped strings.
- bpo-23071 [https://bugs.python.org/issue?@action=redirect&bpo=23071]: Added missing names to codecs.__all__. Patch by Martin Panter.
- bpo-22783 [https://bugs.python.org/issue?@action=redirect&bpo=22783]: Pickling now uses the NEWOBJ opcode instead of the NEWOBJ_EX opcode if

possible.

- bpo-15513 [https://bugs.python.org/issue?@action=redirect&bpo=15513]: Added a __sizeof__ implementation for pickle classes.
- bpo-19858 [https://bugs.python.org/issue?@action=redirect&bpo=19858]: pickletools.optimize() now aware of the MEMOIZE opcode, can produce more compact result and no longer produces invalid output if input data contains MEMOIZE opcodes together with PUT or BINPUT opcodes.
- bpo-22095 [https://bugs.python.org/issue?@action=redirect&bpo=22095]: Fixed HTTPConnection.set_tunnel with default port. The port value in the host header was set to “None”. Patch by Demian Brecht.
- bpo-23016 [https://bugs.python.org/issue?@action=redirect&bpo=23016]: A warning no longer produces an AttributeError when the program is run with pythonw.exe.
- bpo-21775 [https://bugs.python.org/issue?@action=redirect&bpo=21775]: shutil.copytree(): fix crash when copying to VFAT. An exception handler assumed that OSError objects always have a ‘winerror’ attribute. That is not the case, so the exception handler itself raised AttributeError when run on Linux (and, presumably, any other non-Windows OS). Patch by Greg Ward.
- bpo-1218234 [https://bugs.python.org/issue?@action=redirect&bpo=1218234]: Fix inspect.getsource() to load updated source of reloaded module. Initial patch by Berker Peksag.
- bpo-21740 [https://bugs.python.org/issue?@action=redirect&bpo=21740]: Support wrapped callables in doctest. Patch by Claudiu Popa.
- bpo-23009 [https://bugs.python.org/issue?@action=redirect&bpo=23009]: Make sure selectors.EpollSelector.select() works when no FD is registered.
- bpo-22959 [https://bugs.python.org/issue?@action=redirect&bpo=22959]: In the constructor of http.client.HTTPSConnection, prefer the context’s check_hostname attribute over the *check_hostname* parameter.
- bpo-22696 [https://bugs.python.org/issue?@action=redirect&bpo=22696]: Add function `sys.is_finalizing()` to know about interpreter shutdown.
- bpo-16043 [https://bugs.python.org/issue?@action=redirect&bpo=16043]: Add a default limit for the amount of data xmlrpclib.gzip_decode will return. This resolves **CVE 2013-1753** [https://www.cve.org/CVERecord?id=CVE-2013-1753].

- bpo-14099 [https://bugs.python.org/issue?@action=redirect&bpo=14099]: ZipFile.open() no longer reopen the underlying file. Objects returned by ZipFile.open() can now operate independently of the ZipFile even if the ZipFile was created by passing in a file-like object as the first argument to the constructor.
- bpo-22966 [https://bugs.python.org/issue?@action=redirect&bpo=22966]: Fix __pycache__ pyc file name clobber when pyc_compile is asked to compile a source file containing multiple dots in the source file name.
- bpo-21971 [https://bugs.python.org/issue?@action=redirect&bpo=21971]: Update turtledemo doc and add module to the index.
- bpo-21032 [https://bugs.python.org/issue?@action=redirect&bpo=21032]: Fixed socket leak if HTTPConnection.getresponse() fails. Original patch by Martin Panter.
- bpo-22407 [https://bugs.python.org/issue?@action=redirect&bpo=22407]: Deprecated the use of re.LOCALE flag with str patterns or re.ASCII. It was newer worked.
- bpo-22902 [https://bugs.python.org/issue?@action=redirect&bpo=22902]: The “ip” command is now used on Linux to determine MAC address in uuid.getnode(). Patch by Bruno Cauet.
- bpo-22960 [https://bugs.python.org/issue?@action=redirect&bpo=22960]: Add a context argument to xmlrpclib.ServerProxy constructor.
- bpo-22389 [https://bugs.python.org/issue?@action=redirect&bpo=22389]: Add contextlib.redirect_stderr().
- bpo-21356 [https://bugs.python.org/issue?@action=redirect&bpo=21356]: Make ssl.RAND_egd() optional to support LibreSSL. The availability of the function is checked during the compilation. Patch written by Bernard Spil.
- bpo-22915 [https://bugs.python.org/issue?@action=redirect&bpo=22915]: SAX parser now supports files opened with file descriptor or bytes path.
- bpo-22609 [https://bugs.python.org/issue?@action=redirect&bpo=22609]: Constructors and update methods of mapping classes in the collections module now accept the self keyword argument.
- bpo-22940 [https://bugs.python.org/issue?@action=redirect&bpo=22940]: Add readline.append_history_file.
- bpo-19676 [https://bugs.python.org/issue?@action=redirect&bpo=19676]: Added the “namereplace” error handler.

- bpo-22788 [https://bugs.python.org/issue?@action=redirect&bpo=22788]: Add *context* parameter to logging.handlers.HTTPHandler.
- bpo-22921 [https://bugs.python.org/issue?@action=redirect&bpo=22921]: Allow SSLContext to take the *hostname* parameter even if OpenSSL doesn't support SNI.
- bpo-22894 [https://bugs.python.org/issue?@action=redirect&bpo=22894]: TestCase.subTest() would cause the test suite to be stopped when in failfast mode, even in the absence of failures.
- bpo-22796 [https://bugs.python.org/issue?@action=redirect&bpo=22796]: HTTP cookie parsing is now stricter, in order to protect against potential injection attacks.
- bpo-22370 [https://bugs.python.org/issue?@action=redirect&bpo=22370]: Windows detection in pathlib is now more robust.
- bpo-22841 [https://bugs.python.org/issue?@action=redirect&bpo=22841]: Reject coroutines in asyncio add_signal_handler(). Patch by Ludovic.Gasc.
- bpo-19494 [https://bugs.python.org/issue?@action=redirect&bpo=19494]: Added urllib.request.HTTPBasicPriorAuthHandler. Patch by Matej Cepl.
- bpo-22578 [https://bugs.python.org/issue?@action=redirect&bpo=22578]: Added attributes to the re.error class.
- bpo-22849 [https://bugs.python.org/issue?@action=redirect&bpo=22849]: Fix possible double free in the io.TextIOWrapper constructor.
- bpo-12728 [https://bugs.python.org/issue?@action=redirect&bpo=12728]: Different Unicode characters having the same uppercase but different lowercase are now matched in case-insensitive regular expressions.
- bpo-22821 [https://bugs.python.org/issue?@action=redirect&bpo=22821]: Fixed fcntl() with integer argument on 64-bit big-endian platforms.
- bpo-21650 [https://bugs.python.org/issue?@action=redirect&bpo=21650]: Add an `--sort-keys` option to `json.tool` CLI.
- bpo-22824 [https://bugs.python.org/issue?@action=redirect&bpo=22824]: Updated reprlib output format for sets to use set literals. Patch contributed by Berker Peksag.
- bpo-22824 [https://bugs.python.org/issue?@action=redirect&bpo=22824]: Updated reprlib output format for arrays to display empty arrays without an unnecessary empty list. Suggested by Serhiy Storchaka.
- bpo-22406 [https://bugs.python.org/issue?@action=redirect&bpo=22406]: Fixed the uu_codec codec incorrectly ported to 3.x. Based on patch by Martin Panter.

- bpo-17293 [https://bugs.python.org/issue?@action=redirect&bpo=17293]: uuid.getnode() now determines MAC address on AIX using netstat. Based on patch by Aivars Kalvāns.
- bpo-22769 [https://bugs.python.org/issue?@action=redirect&bpo=22769]: Fixed ttk.Treeview.tag_has() when called without arguments.
- bpo-22417 [https://bugs.python.org/issue?@action=redirect&bpo=22417]: Verify certificates by default in httplib (PEP 476).
- bpo-22775 [https://bugs.python.org/issue?@action=redirect&bpo=22775]: Fixed unpickling of http.cookies.SimpleCookie with protocol 2 and above. Patch by Tim Graham.
- bpo-22776 [https://bugs.python.org/issue?@action=redirect&bpo=22776]: Brought excluded code into the scope of a try block in SysLogHandler.emit().
- bpo-22665 [https://bugs.python.org/issue?@action=redirect&bpo=22665]: Add missing get_terminal_size and SameFileError to shutil.__all__.
- bpo-6623 [https://bugs.python.org/issue?@action=redirect&bpo=6623]: Remove deprecated Netrc class in the ftplib module. Patch by Matt Chaput.
- bpo-17381 [https://bugs.python.org/issue?@action=redirect&bpo=17381]: Fixed handling of case-insensitive ranges in regular expressions.
- bpo-22410 [https://bugs.python.org/issue?@action=redirect&bpo=22410]: Module level functions in the re module now cache compiled locale-dependent regular expressions taking into account the locale.
- bpo-22759 [https://bugs.python.org/issue?@action=redirect&bpo=22759]: Query methods on pathlib.Path() (exists(), is_dir(), etc.) now return False when the underlying stat call raises NotADirectoryError.
- bpo-8876 [https://bugs.python.org/issue?@action=redirect&bpo=8876]: distutils now falls back to copying files when hard linking doesn’t work. This allows use with special filesystems such as VirtualBox shared folders.
- bpo-22217 [https://bugs.python.org/issue?@action=redirect&bpo=22217]: Implemented reprs of classes in the zipfile module.
- bpo-22457 [https://bugs.python.org/issue?@action=redirect&bpo=22457]: Honour load_tests in the start_dir of discovery.
- bpo-18216 [https://bugs.python.org/issue?@action=redirect&bpo=18216]: gettext now raises an error when a .mo file has an unsupported major version number. Patch by Aaron Hill.
- bpo-13918 [https://bugs.python.org/issue?@action=redirect&bpo=13918]: Provide a locale.delocalize() function which can remove locale-specific number

formatting from a string representing a number, without then converting it to a specific type. Patch by Cédric Krier.

- bpo-22676 [https://bugs.python.org/issue?@action=redirect&bpo=22676]: Make the pickling of global objects which don't have a __module__ attribute less slow.
- bpo-18853 [https://bugs.python.org/issue?@action=redirect&bpo=18853]: Fixed ResourceWarning in shlex.__nain__.
- bpo-9351 [https://bugs.python.org/issue?@action=redirect&bpo=9351]: Defaults set with set_defaults on an argparse subparser are no longer ignored when also set on the parent parser.
- bpo-7559 [https://bugs.python.org/issue?@action=redirect&bpo=7559]: unittest test loading ImportErrors are reported as import errors with their import exception rather than as attribute errors after the import has already failed.
- bpo-19746 [https://bugs.python.org/issue?@action=redirect&bpo=19746]: Make it possible to examine the errors from unittest discovery without executing the test suite. The new `errors` attribute on `TestLoader` exposes these non-fatal errors encountered during discovery.
- bpo-21991 [https://bugs.python.org/issue?@action=redirect&bpo=21991]: Make email.headerregistry's header 'params' attributes be read-only (MappingProxyType). Previously the dictionary was modifiable but a new one was created on each access of the attribute.
- bpo-22638 [https://bugs.python.org/issue?@action=redirect&bpo=22638]: SSLv3 is now disabled throughout the standard library. It can still be enabled by instantiating a SSLContext manually.
- bpo-22641 [https://bugs.python.org/issue?@action=redirect&bpo=22641]: In asyncio, the default SSL context for client connections is now created using ssl.create_default_context(), for stronger security.
- bpo-17401 [https://bugs.python.org/issue?@action=redirect&bpo=17401]: Include closefd in io.FileIO repr.
- bpo-21338 [https://bugs.python.org/issue?@action=redirect&bpo=21338]: Add silent mode for compileall. quiet parameters of compile_{dir, file, path} functions now have a multilevel value. Also, -q option of the CLI now have a multilevel value. Patch by Thomas Kluyver.
- bpo-20152 [https://bugs.python.org/issue?@action=redirect&bpo=20152]: Convert the array and cmath modules to Argument Clinic.

- bpo-18643 [https://bugs.python.org/issue?@action=redirect&bpo=18643]: Add socket.socketpair() on Windows.
- bpo-22435 [https://bugs.python.org/issue?@action=redirect&bpo=22435]: Fix a file descriptor leak when socketserver bind fails.
- bpo-13096 [https://bugs.python.org/issue?@action=redirect&bpo=13096]: Fixed segfault in CTypes POINTER handling of large values.
- bpo-11694 [https://bugs.python.org/issue?@action=redirect&bpo=11694]: Raise ConversionError in xdrlib as documented. Patch by Filip Gruszczyński and Claudiu Popa.
- bpo-19380 [https://bugs.python.org/issue?@action=redirect&bpo=19380]: Optimized parsing of regular expressions.
- bpo-1519638 [https://bugs.python.org/issue?@action=redirect&bpo=1519638]: Now unmatched groups are replaced with empty strings in re.sub() and re.subn().
- bpo-18615 [https://bugs.python.org/issue?@action=redirect&bpo=18615]: sndhdr.what/whathdr now return a namedtuple.
- bpo-22462 [https://bugs.python.org/issue?@action=redirect&bpo=22462]: Fix pyexpat's creation of a dummy frame to make it appear in exception tracebacks.
- bpo-21965 [https://bugs.python.org/issue?@action=redirect&bpo=21965]: Add support for in-memory SSL to the ssl module. Patch by Geert Jansen.
- bpo-21173 [https://bugs.python.org/issue?@action=redirect&bpo=21173]: Fix len() on a WeakKeyDictionary when .clear() was called with an iterator alive.
- bpo-11866 [https://bugs.python.org/issue?@action=redirect&bpo=11866]: Eliminated race condition in the computation of names for new threads.
- bpo-21905 [https://bugs.python.org/issue?@action=redirect&bpo=21905]: Avoid RuntimeError in pickle.whichmodule() when sys.modules is mutated while iterating. Patch by Olivier Grisel.
- bpo-11271 [https://bugs.python.org/issue?@action=redirect&bpo=11271]: concurrent.futures.Executor.map() now takes a *chunksize* argument to allow batching of tasks in child processes and improve performance of ProcessPoolExecutor. Patch by Dan O'Reilly.
- bpo-21883 [https://bugs.python.org/issue?@action=redirect&bpo=21883]: os.path.join() and os.path.relpath() now raise a TypeError with more helpful error message for unsupported or mismatched types of arguments.

- bpo-22219 [https://bugs.python.org/issue?@action=redirect&bpo=22219]: The zipfile module CLI now adds entries for directories (including empty directories) in ZIP file.
- bpo-22449 [https://bugs.python.org/issue?@action=redirect&bpo=22449]: In the ssl.SSLContext.load_default_certs, consult the environmental variables SSL_CERT_DIR and SSL_CERT_FILE on Windows.
- bpo-22508 [https://bugs.python.org/issue?@action=redirect&bpo=22508]: The email.__version__ variable has been removed; the email code is no longer shipped separately from the stdlib, and __version__ hasn't been updated in several releases.
- bpo-20076 [https://bugs.python.org/issue?@action=redirect&bpo=20076]: Added non derived UTF-8 aliases to locale aliases table.
- bpo-20079 [https://bugs.python.org/issue?@action=redirect&bpo=20079]: Added locales supported in glibc 2.18 to locale alias table.
- bpo-20218 [https://bugs.python.org/issue?@action=redirect&bpo=20218]: Added convenience methods read_text/write_text and read_bytes/ write_bytes to pathlib.Path objects.
- bpo-22396 [https://bugs.python.org/issue?@action=redirect&bpo=22396]: On 32-bit AIX platform, don't expose os.posix_fadvise() nor os.posix_fallocate() because their prototypes in system headers are wrong.
- bpo-22517 [https://bugs.python.org/issue?@action=redirect&bpo=22517]: When an io.BufferedRWPair object is deallocated, clear its weakrefs.
- bpo-22437 [https://bugs.python.org/issue?@action=redirect&bpo=22437]: Number of capturing groups in regular expression is no longer limited by 100.
- bpo-17442 [https://bugs.python.org/issue?@action=redirect&bpo=17442]: InteractiveInterpreter now displays the full chained traceback in its showtraceback method, to match the built in interactive interpreter.
- bpo-23392 [https://bugs.python.org/issue?@action=redirect&bpo=23392]: Added tests for marshal C API that works with FILE*.
- bpo-10510 [https://bugs.python.org/issue?@action=redirect&bpo=10510]: distutils register and upload methods now use HTML standards compliant CRLF line endings.
- bpo-9850 [https://bugs.python.org/issue?@action=redirect&bpo=9850]: Fixed macpath.join() for empty first component. Patch by Oleg Oshmyan.
- bpo-5309 [https://bugs.python.org/issue?@action=redirect&bpo=5309]: distutils' build and build_ext commands now accept a `-j` option to enable parallel building of extension modules.

- bpo-22448 [https://bugs.python.org/issue?@action=redirect&bpo=22448]: Improve canceled timer handles cleanup to prevent unbound memory usage. Patch by Joshua Moore-Oliva.
- bpo-22427 [https://bugs.python.org/issue?@action=redirect&bpo=22427]: TemporaryDirectory no longer attempts to clean up twice when used in the with statement in generator.
- bpo-22362 [https://bugs.python.org/issue?@action=redirect&bpo=22362]: Forbidden ambiguous octal escapes out of range 0-0o377 in regular expressions.
- bpo-20912 [https://bugs.python.org/issue?@action=redirect&bpo=20912]: Now directories added to ZIP file have correct Unix and MS-DOS directory attributes.
- bpo-21866 [https://bugs.python.org/issue?@action=redirect&bpo=21866]: ZipFile.close() no longer writes ZIP64 central directory records if allowZip64 is false.
- bpo-22278 [https://bugs.python.org/issue?@action=redirect&bpo=22278]: Fix urljoin problem with relative urls, a regression observed after changes to issue22118 were submitted.
- bpo-22415 [https://bugs.python.org/issue?@action=redirect&bpo=22415]: Fixed debugging output of the GROUPREF_EXISTS opcode in the re module. Removed trailing spaces in debugging output.
- bpo-22423 [https://bugs.python.org/issue?@action=redirect&bpo=22423]: Unhandled exception in thread no longer causes unhandled AttributeError when sys.stderr is None.
- bpo-21332 [https://bugs.python.org/issue?@action=redirect&bpo=21332]: Ensure that `bufsize=1` in subprocess.Popen() selects line buffering, rather than block buffering. Patch by Akira Li.
- bpo-21091 [https://bugs.python.org/issue?@action=redirect&bpo=21091]: Fix API bug: email.message.EmailMessage.is_attachment is now a method.
- bpo-21079 [https://bugs.python.org/issue?@action=redirect&bpo=21079]: Fix email.message.EmailMessage.is_attachment to return the correct result when the header has parameters as well as a value.
- bpo-22247 [https://bugs.python.org/issue?@action=redirect&bpo=22247]: Add NNTPError to nntplib.__all__.
- bpo-22366 [https://bugs.python.org/issue?@action=redirect&bpo=22366]: urllib.request.urlopen will accept a context object (SSLContext) as an argument which will then be used for HTTPS connection. Patch by Alex Gaynor.

- bpo-4180 [https://bugs.python.org/issue?@action=redirect&bpo=4180]: The warnings registries are now reset when the filters are modified.
- bpo-22419 [https://bugs.python.org/issue?@action=redirect&bpo=22419]: Limit the length of incoming HTTP request in wsgiref server to 65536 bytes and send a 414 error code for higher lengths. Patch contributed by Devin Cook.
- Lax cookie parsing in http.cookies could be a security issue when combined with non-standard cookie handling in some web browsers. Reported by Sergey Bobrov.
- bpo-20537 [https://bugs.python.org/issue?@action=redirect&bpo=20537]: logging methods now accept an exception instance as well as a Boolean value or exception tuple. Thanks to Yury Selivanov for the patch.
- bpo-22384 [https://bugs.python.org/issue?@action=redirect&bpo=22384]: An exception in Tkinter callback no longer crashes the program when it is run with pythonw.exe.
- bpo-22168 [https://bugs.python.org/issue?@action=redirect&bpo=22168]: Prevent turtle AttributeError with non-default Canvas on OS X.
- bpo-21147 [https://bugs.python.org/issue?@action=redirect&bpo=21147]: sqlite3 now raises an exception if the request contains a null character instead of truncating it. Based on patch by Victor Stinner.
- bpo-13968 [https://bugs.python.org/issue?@action=redirect&bpo=13968]: The glob module now supports recursive search in subdirectories using the `**` pattern.
- bpo-21951 [https://bugs.python.org/issue?@action=redirect&bpo=21951]: Fixed a crash in Tkinter on AIX when called Tcl command with empty string or tuple argument.
- bpo-21951 [https://bugs.python.org/issue?@action=redirect&bpo=21951]: Tkinter now most likely raises MemoryError instead of crash if the memory allocation fails.
- bpo-22338 [https://bugs.python.org/issue?@action=redirect&bpo=22338]: Fix a crash in the json module on memory allocation failure.
- bpo-12410 [https://bugs.python.org/issue?@action=redirect&bpo=12410]: imaplib.IMAP4 now supports the context management protocol. Original patch by Tarek Ziadé.
- bpo-21270 [https://bugs.python.org/issue?@action=redirect&bpo=21270]: We now override tuple methods in mock.call objects so that they can be used as normal call attributes.

- bpo-16662 [https://bugs.python.org/issue?@action=redirect&bpo=16662]: `load_tests()` is now unconditionally run when it is present in a package's `__init__.py`. `TestLoader.loadTestsFromModule()` still accepts use_load_tests, but it is deprecated and ignored. A new keyword-only attribute `pattern` is added and documented. Patch given by Robert Collins, tweaked by Barry Warsaw.
- bpo-22226 [https://bugs.python.org/issue?@action=redirect&bpo=22226]: First letter no longer is stripped from the "status" key in the result of Treeview.heading().
- bpo-19524 [https://bugs.python.org/issue?@action=redirect&bpo=19524]: Fixed resource leak in the HTTP connection when an invalid response is received. Patch by Martin Panter.
- bpo-20421 [https://bugs.python.org/issue?@action=redirect&bpo=20421]: Add a .version() method to SSL sockets exposing the actual protocol version in use.
- bpo-19546 [https://bugs.python.org/issue?@action=redirect&bpo=19546]: configparser exceptions no longer expose implementation details. Chained KeyErrors are removed, which leads to cleaner tracebacks. Patch by Claudiu Popa.
- bpo-22051 [https://bugs.python.org/issue?@action=redirect&bpo=22051]: turtledemo no longer reloads examples to re-run them. Initialization of variables and gui setup should be done in main(), which is called each time a demo is run, but not on import.
- bpo-21933 [https://bugs.python.org/issue?@action=redirect&bpo=21933]: Turtledemo users can change the code font size with a menu selection or control(command) '-' or '+' or control-mousewheel. Original patch by Lita Cho.
- bpo-21597 [https://bugs.python.org/issue?@action=redirect&bpo=21597]: The separator between the turtledemo text pane and the drawing canvas can now be grabbed and dragged with a mouse. The code text pane can be widened to easily view or copy the full width of the text. The canvas can be widened on small screens. Original patches by Jan Kanis and Lita Cho.
- bpo-18132 [https://bugs.python.org/issue?@action=redirect&bpo=18132]: Turtledemo buttons no longer disappear when the window is shrunk. Original patches by Jan Kanis and Lita Cho.

- bpo-22043 [https://bugs.python.org/issue?@action=redirect&bpo=22043]: time.monotonic() is now always available. `threading.Lock.acquire()`, `threading.RLock.acquire()` and socket operations now use a monotonic clock, instead of the system clock, when a timeout is used.
- bpo-21527 [https://bugs.python.org/issue?@action=redirect&bpo=21527]: Add a default number of workers to ThreadPoolExecutor equal to 5 times the number of CPUs. Patch by Claudiu Popa.
- bpo-22216 [https://bugs.python.org/issue?@action=redirect&bpo=22216]: smtplib now resets its state more completely after a quit. The most obvious consequence of the previous behavior was a STARTTLS failure during a connect/starttls/quit/connect/starttls sequence.
- bpo-22098 [https://bugs.python.org/issue?@action=redirect&bpo=22098]: ctypes' BigEndianStructure and LittleEndianStructure now define an empty __slots__ so that subclasses don't always get an instance dict. Patch by Claudiu Popa.
- bpo-22185 [https://bugs.python.org/issue?@action=redirect&bpo=22185]: Fix an occasional RuntimeError in threading.Condition.wait() caused by mutation of the waiters queue without holding the lock. Patch by Doug Zongker.
- bpo-22287 [https://bugs.python.org/issue?@action=redirect&bpo=22287]: On UNIX, _PyTime_gettimeofday() now uses clock_gettime(CLOCK_REALTIME) if available. As a side effect, Python now depends on the librt library on Solaris and on Linux (only with glibc older than 2.17).
- bpo-22182 [https://bugs.python.org/issue?@action=redirect&bpo=22182]: Use e.args to unpack exceptions correctly in distutils.file_util.move_file. Patch by Claudiu Popa.
- The webbrowser module now uses subprocess's start_new_session=True rather than a potentially risky preexec_fn=os.setsid call.
- bpo-22042 [https://bugs.python.org/issue?@action=redirect&bpo=22042]: signal.set_wakeup_fd(fd) now raises an exception if the file descriptor is in blocking mode.
- bpo-16808 [https://bugs.python.org/issue?@action=redirect&bpo=16808]: inspect.stack() now returns a named tuple instead of a tuple. Patch by Daniel Shahaf.

- bpo-22236 [https://bugs.python.org/issue?@action=redirect&bpo=22236]: Fixed Tkinter images copying operations in NoDefaultRoot mode.
- bpo-2527 [https://bugs.python.org/issue?@action=redirect&bpo=2527]: Add a *globals* argument to timeit functions, in order to override the globals namespace in which the timed code is executed. Patch by Ben Roberts.
- bpo-22118 [https://bugs.python.org/issue?@action=redirect&bpo=22118]: Switch urllib.parse to use RFC 3986 semantics for the resolution of relative URLs, rather than RFCs 1808 and 2396. Patch by Demian Brecht.
- bpo-21549 [https://bugs.python.org/issue?@action=redirect&bpo=21549]: Added the "members" parameter to TarFile.list().
- bpo-19628 [https://bugs.python.org/issue?@action=redirect&bpo=19628]: Allow compileall recursion depth to be specified with a -r option.
- bpo-15696 [https://bugs.python.org/issue?@action=redirect&bpo=15696]: Add a __sizeof__ implementation for mmap objects on Windows.
- bpo-22068 [https://bugs.python.org/issue?@action=redirect&bpo=22068]: Avoided reference loops with Variables and Fonts in Tkinter.
- bpo-22165 [https://bugs.python.org/issue?@action=redirect&bpo=22165]: SimpleHTTPRequestHandler now supports undecodable file names.
- bpo-15381 [https://bugs.python.org/issue?@action=redirect&bpo=15381]: Optimized line reading in io.BytesIO.
- bpo-8797 [https://bugs.python.org/issue?@action=redirect&bpo=8797]: Raise HTTPError on failed Basic Authentication immediately. Initial patch by Sam Bull.
- bpo-20729 [https://bugs.python.org/issue?@action=redirect&bpo=20729]: Restored the use of lazy iterkeys()/itervalues()/iteritems() in the mailbox module.
- bpo-21448 [https://bugs.python.org/issue?@action=redirect&bpo=21448]: Changed FeedParser feed() to avoid $O(n^2)$ behavior when parsing long line. Original patch by Raymond Hettinger.
- bpo-22184 [https://bugs.python.org/issue?@action=redirect&bpo=22184]: The functools LRU Cache decorator factory now gives an earlier and clearer error message when the user forgets the required parameters.
- bpo-17923 [https://bugs.python.org/issue?@action=redirect&bpo=17923]: glob() patterns ending with a slash no longer match non-dirs on AIX. Based on patch by Delhallt.
- bpo-21725 [https://bugs.python.org/issue?@action=redirect&bpo=21725]: Added support for RFC 6531 (SMTPUTF8) in smtpd.

- bpo-22176 [https://bugs.python.org/issue?@action=redirect&bpo=22176]: Update the ctypes module's libffi to v3.1. This release adds support for the Linux AArch64 and POWERPC ELF ABIv2 little endian architectures.
- bpo-5411 [https://bugs.python.org/issue?@action=redirect&bpo=5411]: Added support for the "xztar" format in the shutil module.
- bpo-21121 [https://bugs.python.org/issue?@action=redirect&bpo=21121]: Don't force 3rd party C extensions to be built with -Werror=declaration-after-statement.
- bpo-21975 [https://bugs.python.org/issue?@action=redirect&bpo=21975]: Fixed crash when using uninitialized sqlite3.Row (in particular when unpickling pickled sqlite3.Row). sqlite3.Row is now initialized in the __new__() method.
- bpo-20170 [https://bugs.python.org/issue?@action=redirect&bpo=20170]: Convert posixmodule to use Argument Clinic.
- bpo-21539 [https://bugs.python.org/issue?@action=redirect&bpo=21539]: Add an *exists_ok* argument to `Pathlib.mkdir()` to mimic `mkdir -p` and `os.makedirs()` functionality. When true, ignore `FileExistsErrors`. Patch by Berker Peksag.
- bpo-22127 [https://bugs.python.org/issue?@action=redirect&bpo=22127]: Bypass IDNA for pure-ASCII host names in the socket module (in particular for numeric IPs).
- bpo-21047 [https://bugs.python.org/issue?@action=redirect&bpo=21047]: set the default value for the *convert_charrefs* argument of HTMLParser to True. Patch by Berker Peksag.
- Add an __all__ to html.entities.
- bpo-15114 [https://bugs.python.org/issue?@action=redirect&bpo=15114]: the strict mode and argument of HTMLParser, HTMLParser.error, and the HTMLParserError exception have been removed.
- bpo-22085 [https://bugs.python.org/issue?@action=redirect&bpo=22085]: Dropped support of Tk 8.3 in Tkinter.
- bpo-21580 [https://bugs.python.org/issue?@action=redirect&bpo=21580]: Now Tkinter correctly handles bytes arguments passed to Tk. In particular this allows initializing images from binary data.
- bpo-22003 [https://bugs.python.org/issue?@action=redirect&bpo=22003]: When initialized from a bytes object, io.BytesIO() now defers making a copy until it is mutated, improving performance and memory use on some use cases. Patch by David Wilson.

- bpo-22018 [https://bugs.python.org/issue?@action=redirect&bpo=22018]: On Windows, signal.set_wakeup_fd() now also supports sockets. A side effect is that Python depends to the WinSock library.
- bpo-22054 [https://bugs.python.org/issue?@action=redirect&bpo=22054]: Add os.get_blocking() and os.set_blocking() functions to get and set the blocking mode of a file descriptor (False if the O_NONBLOCK flag is set, True otherwise). These functions are not available on Windows.
- bpo-17172 [https://bugs.python.org/issue?@action=redirect&bpo=17172]: Make turtledemo start as active on OS X even when run with subprocess. Patch by Lita Cho.
- bpo-21704 [https://bugs.python.org/issue?@action=redirect&bpo=21704]: Fix build error for _multiprocessing when semaphores are not available. Patch by Arfrever Frehtes Taifersar Arahesis.
- bpo-20173 [https://bugs.python.org/issue?@action=redirect&bpo=20173]: Convert sha1, sha256, sha512 and md5 to ArgumentClinic. Patch by Vajrasky Kok.
- Fix repr(_socket.socket) on Windows 64-bit: don't fail with OverflowError on closed socket. repr(socket.socket) already works fine.
- bpo-22033 [https://bugs.python.org/issue?@action=redirect&bpo=22033]: Reprs of most Python implemented classes now contain actual class name instead of hardcoded one.
- bpo-21947 [https://bugs.python.org/issue?@action=redirect&bpo=21947]: The dis module can now disassemble generator-iterator objects based on their gi_code attribute. Patch by Clement Rouault.
- bpo-16133 [https://bugs.python.org/issue?@action=redirect&bpo=16133]: The asynchat.async_chat.handle_read() method now ignores BlockingIOError exceptions.
- bpo-22044 [https://bugs.python.org/issue?@action=redirect&bpo=22044]: Fixed premature DECREF in call_tzinfo_method. Patch by Tom Flanagan.
- bpo-19884 [https://bugs.python.org/issue?@action=redirect&bpo=19884]: readline: Disable the meta modifier key if stdout is not a terminal to not write the ANSI sequence `"\033[1034h"` into stdout. This sequence is used on some terminal (ex: TERM=xterm-256color") to enable support of 8 bit characters.
- bpo-4350 [https://bugs.python.org/issue?@action=redirect&bpo=4350]: Removed a number of out-of-dated and non-working for a long time Tkinter

methods.

- bpo-6167 [https://bugs.python.org/issue?@action=redirect&bpo=6167]: Scrollbar.activate() now returns the name of active element if the argument is not specified. Scrollbar.set() now always accepts only 2 arguments.
- bpo-15275 [https://bugs.python.org/issue?@action=redirect&bpo=15275]: Clean up and speed up the ntpath module.
- bpo-21888 [https://bugs.python.org/issue?@action=redirect&bpo=21888]: plistlib's load() and loads() now work if the fmt parameter is specified.
- bpo-22032 [https://bugs.python.org/issue?@action=redirect&bpo=22032]: __qualname__ instead of __name__ is now always used to format fully qualified class names of Python implemented classes.
- bpo-22031 [https://bugs.python.org/issue?@action=redirect&bpo=22031]: Reprs now always use hexadecimal format with the "0x" prefix when contain an id in form " at 0x…".
- bpo-22018 [https://bugs.python.org/issue?@action=redirect&bpo=22018]: signal.set_wakeup_fd() now raises an OSError instead of a ValueError on `fstat()` failure.
- bpo-21044 [https://bugs.python.org/issue?@action=redirect&bpo=21044]: tarfile.open() now handles fileobj with an integer 'name' attribute. Based on patch by Antoine Pietri.
- bpo-21966 [https://bugs.python.org/issue?@action=redirect&bpo=21966]: Respect -q command-line option when code module is ran.
- bpo-19076 [https://bugs.python.org/issue?@action=redirect&bpo=19076]: Don't pass the redundant 'file' argument to self.error().
- bpo-16382 [https://bugs.python.org/issue?@action=redirect&bpo=16382]: Improve exception message of warnings.warn() for bad category. Initial patch by Phil Elson.
- bpo-21932 [https://bugs.python.org/issue?@action=redirect&bpo=21932]: os.read() now uses a `Py_ssize_t()` type instead of int for the size to support reading more than 2 GB at once. On Windows, the size is truncated to INT_MAX. As any call to os.read(), the OS may read less bytes than the number of requested bytes.
- bpo-21942 [https://bugs.python.org/issue?@action=redirect&bpo=21942]: Fixed source file viewing in pydoc's server mode on Windows.
- bpo-11259 [https://bugs.python.org/issue?@action=redirect&bpo=11259]: asynchat.async_chat().set_terminator() now raises a ValueError if the

number of received bytes is negative.

- bpo-12523 [https://bugs.python.org/issue?@action=redirect&bpo=12523]: asynchat.async_chat.push() now raises a TypeError if it doesn't get a bytes string
- bpo-21707 [https://bugs.python.org/issue?@action=redirect&bpo=21707]: Add missing kwonlyargcount argument to ModuleFinder.replace_paths_in_code().
- bpo-20639 [https://bugs.python.org/issue?@action=redirect&bpo=20639]: calling Path.with_suffix('') allows removing the suffix again. Patch by July Tikhonov.
- bpo-21714 [https://bugs.python.org/issue?@action=redirect&bpo=21714]: Disallow the construction of invalid paths using Path.with_name(). Original patch by Antony Lee.
- bpo-15014 [https://bugs.python.org/issue?@action=redirect&bpo=15014]: Added 'auth' method to smtplib to make implementing auth mechanisms simpler, and used it internally in the login method.
- bpo-21151 [https://bugs.python.org/issue?@action=redirect&bpo=21151]: Fixed a segfault in the winreg module when `None` is passed as a `REG_BINARY` value to SetValueEx. Patch by John Ehresman.
- bpo-21090 [https://bugs.python.org/issue?@action=redirect&bpo=21090]: io.FileIO.readall() does not ignore I/O errors anymore. Before, it ignored I/O errors if at least the first C call read() succeed.
- bpo-5800 [https://bugs.python.org/issue?@action=redirect&bpo=5800]: headers parameter of wsgiref.headers.Headers is now optional. Initial patch by Pablo Torres Navarrete and SilentGhost.
- bpo-21781 [https://bugs.python.org/issue?@action=redirect&bpo=21781]: ssl.RAND_add() now supports strings longer than 2 GB.
- bpo-21679 [https://bugs.python.org/issue?@action=redirect&bpo=21679]: Prevent extraneous fstat() calls during open(). Patch by Bohuslav Kabrda.
- bpo-21863 [https://bugs.python.org/issue?@action=redirect&bpo=21863]: cProfile now displays the module name of C extension functions, in addition to their own name.
- bpo-11453 [https://bugs.python.org/issue?@action=redirect&bpo=11453]: asyncore: emit a ResourceWarning when an unclosed file_wrapper object is destroyed. The destructor now closes the file if needed. The close() method can now be called twice: the second call does nothing.

- bpo-21858 [https://bugs.python.org/issue?@action=redirect&bpo=21858]: Better handling of Python exceptions in the sqlite3 module.
- bpo-21476 [https://bugs.python.org/issue?@action=redirect&bpo=21476]: Make sure the email.parser.BytesParser TextIOWrapper is discarded after parsing, so the input file isn't unexpectedly closed.
- bpo-20295 [https://bugs.python.org/issue?@action=redirect&bpo=20295]: imghdr now recognizes OpenEXR format images.
- bpo-21729 [https://bugs.python.org/issue?@action=redirect&bpo=21729]: Used the "with" statement in the dbm.dumb module to ensure files closing. Patch by Claudiu Popa.
- bpo-21491 [https://bugs.python.org/issue?@action=redirect&bpo=21491]: socketserver: Fix a race condition in child processes reaping.
- bpo-21719 [https://bugs.python.org/issue?@action=redirect&bpo=21719]: Added the `st_file_attributes` field to os.stat_result on Windows.
- bpo-21832 [https://bugs.python.org/issue?@action=redirect&bpo=21832]: Require named tuple inputs to be exact strings.
- bpo-21722 [https://bugs.python.org/issue?@action=redirect&bpo=21722]: The distutils "upload" command now exits with a non-zero return code when uploading fails. Patch by Martin Dengler.
- bpo-21723 [https://bugs.python.org/issue?@action=redirect&bpo=21723]: asyncio.Queue: support any type of number (ex: float) for the maximum size. Patch written by Vajrasky Kok.
- bpo-21711 [https://bugs.python.org/issue?@action=redirect&bpo=21711]: support for "site-python" directories has now been removed from the site module (it was deprecated in 3.4).
- bpo-17552 [https://bugs.python.org/issue?@action=redirect&bpo=17552]: new socket.sendfile() method allowing a file to be sent over a socket by using high-performance os.sendfile() on UNIX. Patch by Giampaolo Rodola'.
- bpo-18039 [https://bugs.python.org/issue?@action=redirect&bpo=18039]: dbm.dump.open() now always creates a new database when the flag has the value 'n'. Patch by Claudiu Popa.
- bpo-21326 [https://bugs.python.org/issue?@action=redirect&bpo=21326]: Add a new is_closed() method to asyncio.BaseEventLoop. run_forever() and run_until_complete() methods of asyncio.BaseEventLoop now raise an exception if the event loop was closed.

- bpo-21766 [https://bugs.python.org/issue?@action=redirect&bpo=21766]: Prevent a security hole in CGIHTTPServer by URL unquoting paths before checking for a CGI script at that path.
- bpo-21310 [https://bugs.python.org/issue?@action=redirect&bpo=21310]: Fixed possible resource leak in failed open().
- bpo-21256 [https://bugs.python.org/issue?@action=redirect&bpo=21256]: Printout of keyword args should be in deterministic order in a mock function call. This will help to write better doctests.
- bpo-21677 [https://bugs.python.org/issue?@action=redirect&bpo=21677]: Fixed chaining nonnormalized exceptions in io close() methods.
- bpo-11709 [https://bugs.python.org/issue?@action=redirect&bpo=11709]: Fix the pydoc.help function to not fail when sys.stdin is not a valid file.
- bpo-21515 [https://bugs.python.org/issue?@action=redirect&bpo=21515]: tempfile.TemporaryFile now uses os.O_TMPFILE flag is available.
- bpo-13223 [https://bugs.python.org/issue?@action=redirect&bpo=13223]: Fix pydoc.writedoc so that the HTML documentation for methods that use ‘self’ in the example code is generated correctly.
- bpo-21463 [https://bugs.python.org/issue?@action=redirect&bpo=21463]: In urllib.request, fix pruning of the FTP cache.
- bpo-21618 [https://bugs.python.org/issue?@action=redirect&bpo=21618]: The subprocess module could fail to close open fds that were inherited by the calling process and already higher than POSIX resource limits would otherwise allow. On systems with a functioning /proc/self/fd or /dev/fd interface the max is now ignored and all fds are closed.
- bpo-20383 [https://bugs.python.org/issue?@action=redirect&bpo=20383]: Introduce importlib.util.module_from_spec() as the preferred way to create a new module.
- bpo-21552 [https://bugs.python.org/issue?@action=redirect&bpo=21552]: Fixed possible integer overflow of too long string lengths in the tkinter module on 64-bit platforms.
- bpo-14315 [https://bugs.python.org/issue?@action=redirect&bpo=14315]: The zipfile module now ignores extra fields in the central directory that are too short to be parsed instead of letting a struct.unpack error bubble up as this “bad data” appears in many real world zip files in the wild and is ignored by other zip tools.
- bpo-13742 [https://bugs.python.org/issue?@action=redirect&bpo=13742]: Added “key” and “reverse” parameters to heapq.merge(). (First draft of patch

contributed by Simon Sapin.)

- bpo-21402 [https://bugs.python.org/issue?@action=redirect&bpo=21402]: tkinter.ttk now works when default root window is not set.
- bpo-3015 [https://bugs.python.org/issue?@action=redirect&bpo=3015]: `_tkinter.create()` now creates `tkapp` object with `wantobjects=1` by default.
- bpo-10203 [https://bugs.python.org/issue?@action=redirect&bpo=10203]: sqlite3.Row now truly supports sequence protocol. In particular it supports reverse() and negative indices. Original patch by Claudiu Popa.
- bpo-18807 [https://bugs.python.org/issue?@action=redirect&bpo=18807]: If copying (no symlinks) specified for a venv, then the python interpreter aliases (python, python3) are now created by copying rather than symlinking.
- bpo-20197 [https://bugs.python.org/issue?@action=redirect&bpo=20197]: Added support for the WebP image type in the imghdr module. Patch by Fabrice Aneche and Claudiu Popa.
- bpo-21513 [https://bugs.python.org/issue?@action=redirect&bpo=21513]: Speedup some properties of IP addresses (IPv4Address, IPv6Address) such as .is_private or .is_multicast.
- bpo-21137 [https://bugs.python.org/issue?@action=redirect&bpo=21137]: Improve the repr for threading.Lock() and its variants by showing the “locked” or “unlocked” status. Patch by Berker Peksag.
- bpo-21538 [https://bugs.python.org/issue?@action=redirect&bpo=21538]: The plistlib module now supports loading of binary plist files when reference or offset size is not a power of two.
- bpo-21455 [https://bugs.python.org/issue?@action=redirect&bpo=21455]: Add a default backlog to socket.listen().
- bpo-21525 [https://bugs.python.org/issue?@action=redirect&bpo=21525]: Most Tkinter methods which accepted tuples now accept lists too.
- bpo-22166 [https://bugs.python.org/issue?@action=redirect&bpo=22166]: With the assistance of a new internal _codecs._forget_codec helping function, test_codecs now clears the encoding caches to avoid the appearance of a reference leak
- bpo-22236 [https://bugs.python.org/issue?@action=redirect&bpo=22236]: Tkinter tests now don’t reuse default root window. New root window is created for every test class.

- bpo-10744 [https://bugs.python.org/issue?@action=redirect&bpo=10744]: Fix **PEP 3118** [https://peps.python.org/pep-3118/] format strings on ctypes objects with a nontrivial shape.
- bpo-20826 [https://bugs.python.org/issue?@action=redirect&bpo=20826]: Optimize ipaddress.collapse_addresses().
- bpo-21487 [https://bugs.python.org/issue?@action=redirect&bpo=21487]: Optimize ipaddress.summarize_address_range() and ipaddress.{IPv4Network,IPv6Network}.subnets().
- bpo-21486 [https://bugs.python.org/issue?@action=redirect&bpo=21486]: Optimize parsing of netmasks in ipaddress.IPv4Network and ipaddress.IPv6Network.
- bpo-13916 [https://bugs.python.org/issue?@action=redirect&bpo=13916]: Disallowed the surrogatepass error handler for non UTF-* encodings.
- bpo-20998 [https://bugs.python.org/issue?@action=redirect&bpo=20998]: Fixed re.fullmatch() of repeated single character pattern with ignore case. Original patch by Matthew Barnett.
- bpo-21075 [https://bugs.python.org/issue?@action=redirect&bpo=21075]: fileinput.FileInput now reads bytes from standard stream if binary mode is specified. Patch by Sam Kimbrel.
- bpo-19775 [https://bugs.python.org/issue?@action=redirect&bpo=19775]: Add a samefile() method to pathlib Path objects. Initial patch by Vajrasky Kok.
- bpo-21226 [https://bugs.python.org/issue?@action=redirect&bpo=21226]: Set up modules properly in PyImport_ExecCodeModuleObject (and friends).
- bpo-21398 [https://bugs.python.org/issue?@action=redirect&bpo=21398]: Fix a unicode error in the pydoc pager when the documentation contains characters not encodable to the stdout encoding.
- bpo-16531 [https://bugs.python.org/issue?@action=redirect&bpo=16531]: ipaddress.IPv4Network and ipaddress.IPv6Network now accept an (address, netmask) tuple argument, so as to easily construct network objects from existing addresses.
- bpo-21156 [https://bugs.python.org/issue?@action=redirect&bpo=21156]: importlib.abc.InspectLoader.source_to_code() is now a staticmethod.
- bpo-21424 [https://bugs.python.org/issue?@action=redirect&bpo=21424]: Simplified and optimized heaqp.nlargest() and nmsmallest() to make fewer tuple comparisons.

- bpo-21396 [https://bugs.python.org/issue?@action=redirect&bpo=21396]: Fix TextIOWrapper(…, write_through=True) to not force a flush() on the underlying binary stream. Patch by akira.
- bpo-18314 [https://bugs.python.org/issue?@action=redirect&bpo=18314]: Unlink now removes junctions on Windows. Patch by Kim Gräsman
- bpo-21088 [https://bugs.python.org/issue?@action=redirect&bpo=21088]: Bugfix for curses.window.addch() regression in 3.4.0. In porting to Argument Clinic, the first two arguments were reversed.
- bpo-21407 [https://bugs.python.org/issue?@action=redirect&bpo=21407]: _decimal: The module now supports function signatures.
- bpo-10650 [https://bugs.python.org/issue?@action=redirect&bpo=10650]: Remove the non-standard 'watchexp' parameter from the Decimal.quantize() method in the Python version. It had never been present in the C version.
- bpo-21469 [https://bugs.python.org/issue?@action=redirect&bpo=21469]: Reduced the risk of false positives in robotparser by checking to make sure that robots.txt has been read or does not exist prior to returning True in can_fetch().
- bpo-19414 [https://bugs.python.org/issue?@action=redirect&bpo=19414]: Have the OrderedDict mark deleted links as unusable. This gives an early failure if the link is deleted during iteration.
- bpo-21421 [https://bugs.python.org/issue?@action=redirect&bpo=21421]: Add __slots__ to the MappingViews ABC. Patch by Josh Rosenberg.
- bpo-21101 [https://bugs.python.org/issue?@action=redirect&bpo=21101]: Eliminate double hashing in the C speed-up code for collections.Counter().
- bpo-21321 [https://bugs.python.org/issue?@action=redirect&bpo=21321]: itertools.islice() now releases the reference to the source iterator when the slice is exhausted. Patch by Anton Afanasyev.
- bpo-21057 [https://bugs.python.org/issue?@action=redirect&bpo=21057]: TextIOWrapper now allows the underlying binary stream's read() or read1() method to return an arbitrary bytes-like object (such as a memoryview). Patch by Nikolaus Rath.
- bpo-20951 [https://bugs.python.org/issue?@action=redirect&bpo=20951]: SSLSocket.send() now raises either SSLWantReadError or SSLWantWriteError on a non-blocking socket if the operation would block. Previously, it would return 0. Patch by Nikolaus Rath.

- bpo-13248 [https://bugs.python.org/issue?@action=redirect&bpo=13248]: removed previously deprecated asyncore.dispatcher __getattr__ cheap inheritance hack.
- bpo-9815 [https://bugs.python.org/issue?@action=redirect&bpo=9815]: assertRaises now tries to clear references to local variables in the exception's traceback.
- bpo-19940 [https://bugs.python.org/issue?@action=redirect&bpo=19940]: ssl.cert_time_to_seconds() now interprets the given time string in the UTC timezone (as specified in RFC 5280), not the local timezone.
- bpo-13204 [https://bugs.python.org/issue?@action=redirect&bpo=13204]: Calling sys.flags.__new__ would crash the interpreter, now it raises a TypeError.
- bpo-19385 [https://bugs.python.org/issue?@action=redirect&bpo=19385]: Make operations on a closed dbm.dumb database always raise the same exception.
- bpo-21207 [https://bugs.python.org/issue?@action=redirect&bpo=21207]: Detect when the os.urandom cached fd has been closed or replaced, and open it anew.
- bpo-21291 [https://bugs.python.org/issue?@action=redirect&bpo=21291]: subprocess's Popen.wait() is now thread safe so that multiple threads may be calling wait() or poll() on a Popen instance at the same time without losing the Popen.returncode value.
- bpo-21127 [https://bugs.python.org/issue?@action=redirect&bpo=21127]: Path objects can now be instantiated from str subclass instances (such as `numpy.str_`).
- bpo-15002 [https://bugs.python.org/issue?@action=redirect&bpo=15002]: urllib.response object to use _TemporaryFileWrapper (and _TemporaryFileCloser) facility. Provides a better way to handle file descriptor close. Patch contributed by Christian Theune.
- bpo-12220 [https://bugs.python.org/issue?@action=redirect&bpo=12220]: mindom now raises a custom ValueError indicating it doesn't support spaces in URIs instead of letting a 'split' ValueError bubble up.
- bpo-21068 [https://bugs.python.org/issue?@action=redirect&bpo=21068]: The ssl.PROTOCOL* constants are now enum members.
- bpo-21276 [https://bugs.python.org/issue?@action=redirect&bpo=21276]: posixmodule: Don't define USE_XATTRS on KFreeBSD and the Hurd.

- bpo-21262 [https://bugs.python.org/issue?@action=redirect&bpo=21262]: New method assert_not_called for Mock. It raises AssertionError if the mock has been called.
- bpo-21238 [https://bugs.python.org/issue?@action=redirect&bpo=21238]: New keyword argument `unsafe` to Mock. It raises `AttributeError` in case of an attribute startswith assert or assret.
- bpo-20896 [https://bugs.python.org/issue?@action=redirect&bpo=20896]: ssl.get_server_certificate() now uses PROTOCOL_SSLv23, not PROTOCOL_SSLv3, for maximum compatibility.
- bpo-21239 [https://bugs.python.org/issue?@action=redirect&bpo=21239]: patch.stopall() didn't work deterministically when the same name was patched more than once.
- bpo-21203 [https://bugs.python.org/issue?@action=redirect&bpo=21203]: Updated fileConfig and dictConfig to remove inconsistencies. Thanks to Jure Koren for the patch.
- bpo-21222 [https://bugs.python.org/issue?@action=redirect&bpo=21222]: Passing name keyword argument to mock.create_autospec now works.
- bpo-21197 [https://bugs.python.org/issue?@action=redirect&bpo=21197]: Add lib64 -> lib symlink in venvs on 64-bit non-OS X POSIX.
- bpo-17498 [https://bugs.python.org/issue?@action=redirect&bpo=17498]: Some SMTP servers disconnect after certain errors, violating strict RFC conformance. Instead of losing the error code when we issue the subsequent RSET, smtplib now returns the error code and defers raising the SMTPServerDisconnected error until the next command is issued.
- bpo-17826 [https://bugs.python.org/issue?@action=redirect&bpo=17826]: setting an iterable side_effect on a mock function created by create_autospec now works. Patch by Kushal Das.
- bpo-7776 [https://bugs.python.org/issue?@action=redirect&bpo=7776]: Fix `Host:` header and reconnection when using http.client.HTTPConnection.set_tunnel(). Patch by Nikolaus Rath.
- bpo-20968 [https://bugs.python.org/issue?@action=redirect&bpo=20968]: unittest.mock.MagicMock now supports division. Patch by Johannes Baiter.
- bpo-21529 [https://bugs.python.org/issue?@action=redirect&bpo=21529]: Fix arbitrary memory access in JSONDecoder.raw_decode with a negative

second parameter. Bug reported by Guido Vranken. (See also: **CVE 2014-4616** [https://www.cve.org/CVERecord?id=CVE-2014-4616])

- bpo-21169 [https://bugs.python.org/issue?@action=redirect&bpo=21169]: getpass now handles non-ascii characters that the input stream encoding cannot encode by re-encoding using the replace error handler.
- bpo-21171 [https://bugs.python.org/issue?@action=redirect&bpo=21171]: Fixed undocumented filter API of the rot13 codec. Patch by Berker Peksag.
- bpo-20539 [https://bugs.python.org/issue?@action=redirect&bpo=20539]: Improved math.factorial error message for large positive inputs and changed exception type (OverflowError -> ValueError) for large negative inputs.
- bpo-21172 [https://bugs.python.org/issue?@action=redirect&bpo=21172]: isinstance check relaxed from dict to collections.Mapping.
- bpo-21155 [https://bugs.python.org/issue?@action=redirect&bpo=21155]: asyncio.EventLoop.create_unix_server() now raises a ValueError if path and sock are specified at the same time.
- bpo-21136 [https://bugs.python.org/issue?@action=redirect&bpo=21136]: Avoid unnecessary normalization of Fractions resulting from power and other operations. Patch by Raymond Hettinger.
- bpo-17621 [https://bugs.python.org/issue?@action=redirect&bpo=17621]: Introduce importlib.util.LazyLoader.
- bpo-21076 [https://bugs.python.org/issue?@action=redirect&bpo=21076]: signal module constants were turned into enums. Patch by Giampaolo Rodola'.
- bpo-20636 [https://bugs.python.org/issue?@action=redirect&bpo=20636]: Improved the repr of Tkinter widgets.
- bpo-19505 [https://bugs.python.org/issue?@action=redirect&bpo=19505]: The items, keys, and values views of OrderedDict now support reverse iteration using reversed().
- bpo-21149 [https://bugs.python.org/issue?@action=redirect&bpo=21149]: Improved thread-safety in logging cleanup during interpreter shutdown. Thanks to Devin Jeanpierre for the patch.
- bpo-21058 [https://bugs.python.org/issue?@action=redirect&bpo=21058]: Fix a leak of file descriptor in `tempfile.NamedTemporaryFile()`, close the file descriptor if `io.open()` fails
- bpo-21200 [https://bugs.python.org/issue?@action=redirect&bpo=21200]: Return None from pkgutil.get_loader() when __spec__ is missing.

- bpo-21013 [https://bugs.python.org/issue?@action=redirect&bpo=21013]: Enhance ssl.create_default_context() when used for server side sockets to provide better security by default.
- bpo-20145 [https://bugs.python.org/issue?@action=redirect&bpo=20145]: `assertRaisesRegex` and `assertWarnsRegex` now raise a `TypeError` if the second argument is not a string or compiled regex.
- bpo-20633 [https://bugs.python.org/issue?@action=redirect&bpo=20633]: Replace relative import by absolute import.
- bpo-20980 [https://bugs.python.org/issue?@action=redirect&bpo=20980]: Stop wrapping exception when using ThreadPool.
- bpo-21082 [https://bugs.python.org/issue?@action=redirect&bpo=21082]: In os.makedirs, do not set the process-wide umask. Note this changes behavior of makedirs when exist_ok=True.
- bpo-20990 [https://bugs.python.org/issue?@action=redirect&bpo=20990]: Fix issues found by pyflakes for multiprocessing.
- bpo-21015 [https://bugs.python.org/issue?@action=redirect&bpo=21015]: SSL contexts will now automatically select an elliptic curve for ECDH key exchange on OpenSSL 1.0.2 and later, and otherwise default to “prime256v1”.
- bpo-21000 [https://bugs.python.org/issue?@action=redirect&bpo=21000]: Improve the command-line interface of json.tool.
- bpo-20995 [https://bugs.python.org/issue?@action=redirect&bpo=20995]: Enhance default ciphers used by the ssl module to enable better security and prioritize perfect forward secrecy.
- bpo-20884 [https://bugs.python.org/issue?@action=redirect&bpo=20884]: Don’t assume that __file__ is defined on importlib.__init__.
- bpo-21499 [https://bugs.python.org/issue?@action=redirect&bpo=21499]: Ignore __builtins__ in several test_importlib.test_api tests.
- bpo-20627 [https://bugs.python.org/issue?@action=redirect&bpo=20627]: xmlrpc.client.ServerProxy is now a context manager.
- bpo-19165 [https://bugs.python.org/issue?@action=redirect&bpo=19165]: The formatter module now raises DeprecationWarning instead of PendingDeprecationWarning.
- bpo-13936 [https://bugs.python.org/issue?@action=redirect&bpo=13936]: Remove the ability of datetime.time instances to be considered false in boolean contexts.

- bpo-18931 [https://bugs.python.org/issue?@action=redirect&bpo=18931]: selectors module now supports /dev/poll on Solaris. Patch by Giampaolo Rodola'.
- bpo-19977 [https://bugs.python.org/issue?@action=redirect&bpo=19977]: When the `LC_TYPE` locale is the POSIX locale (`C` locale), **`sys.stdin`** and **`sys.stdout`** are now using the `surrogateescape` error handler, instead of the `strict` error handler.
- bpo-20574 [https://bugs.python.org/issue?@action=redirect&bpo=20574]: Implement incremental decoder for cp65001 code (Windows code page 65001, Microsoft UTF-8).
- bpo-20879 [https://bugs.python.org/issue?@action=redirect&bpo=20879]: Delay the initialization of encoding and decoding tables for base32, ascii85 and base85 codecs in the base64 module, and delay the initialization of the unquote_to_bytes() table of the urllib.parse module, to not waste memory if these modules are not used.
- bpo-19157 [https://bugs.python.org/issue?@action=redirect&bpo=19157]: Include the broadcast address in the usable hosts for IPv6 in ipaddress.
- bpo-11599 [https://bugs.python.org/issue?@action=redirect&bpo=11599]: When an external command (e.g. compiler) fails, distutils now prints out the whole command line (instead of just the command name) if the environment variable DISTUTILS_DEBUG is set.
- bpo-4931 [https://bugs.python.org/issue?@action=redirect&bpo=4931]: distutils should not produce unhelpful "error: None" messages anymore. distutils.util.grok_environment_error is kept but doc-deprecated.
- bpo-20875 [https://bugs.python.org/issue?@action=redirect&bpo=20875]: Prevent possible gzip "'read' is not defined" NameError. Patch by Claudiu Popa.
- bpo-11558 [https://bugs.python.org/issue?@action=redirect&bpo=11558]: `email.message.Message.attach` now returns a more useful error message if `attach` is called on a message for which `is_multipart` is False.
- bpo-20283 [https://bugs.python.org/issue?@action=redirect&bpo=20283]: RE pattern methods now accept the string keyword parameters as documented. The pattern and source keyword parameters are left as deprecated aliases.
- bpo-20778 [https://bugs.python.org/issue?@action=redirect&bpo=20778]: Fix modulefinder to work with bytecode-only modules.

- bpo-20791 [https://bugs.python.org/issue?@action=redirect&bpo=20791]: copy.copy() now doesn't make a copy when the input is a bytes object. Initial patch by Peter Otten.
- bpo-19748 [https://bugs.python.org/issue?@action=redirect&bpo=19748]: On AIX, time.mktime() now raises an OverflowError for year outsize range [1902; 2037].
- bpo-19573 [https://bugs.python.org/issue?@action=redirect&bpo=19573]: inspect.signature: Use enum for parameter kind constants.
- bpo-20726 [https://bugs.python.org/issue?@action=redirect&bpo=20726]: inspect.signature: Make Signature and Parameter picklable.
- bpo-17373 [https://bugs.python.org/issue?@action=redirect&bpo=17373]: Add inspect.Signature.from_callable method.
- bpo-20378 [https://bugs.python.org/issue?@action=redirect&bpo=20378]: Improve repr of inspect.Signature and inspect.Parameter.
- bpo-20816 [https://bugs.python.org/issue?@action=redirect&bpo=20816]: Fix inspect.getcallargs() to raise correct TypeError for missing keyword-only arguments. Patch by Jeremiah Lowin.
- bpo-20817 [https://bugs.python.org/issue?@action=redirect&bpo=20817]: Fix inspect.getcallargs() to fail correctly if more than 3 arguments are missing. Patch by Jeremiah Lowin.
- bpo-6676 [https://bugs.python.org/issue?@action=redirect&bpo=6676]: Ensure a meaningful exception is raised when attempting to parse more than one XML document per pyexpat xmlparser instance. (Original patches by Hirokazu Yamamoto and Amaury Forgeot d'Arc, with suggested wording by David Gutteridge)
- bpo-21117 [https://bugs.python.org/issue?@action=redirect&bpo=21117]: Fix inspect.signature to better support functools.partial. Due to the specifics of functools.partial implementation, positional-or-keyword arguments passed as keyword arguments become keyword-only.
- bpo-20334 [https://bugs.python.org/issue?@action=redirect&bpo=20334]: inspect.Signature and inspect.Parameter are now hashable. Thanks to Antony Lee for bug reports and suggestions.
- bpo-15916 [https://bugs.python.org/issue?@action=redirect&bpo=15916]: doctest.DocTestSuite returns an empty unittest.TestSuite instead of raising ValueError if it finds no tests
- bpo-21209 [https://bugs.python.org/issue?@action=redirect&bpo=21209]: Fix asyncio.tasks.CoroWrapper to workaround a bug in yield-from

implementation in CPythons prior to 3.4.1.

- asyncio: Add gi_{frame,running,code} properties to CoroWrapper (upstream bpo-163 [https://bugs.python.org/issue?@action=redirect&bpo=163]).
- bpo-21311 [https://bugs.python.org/issue?@action=redirect&bpo=21311]: Avoid exception in _osx_support with non-standard compiler configurations. Patch by John Szakmeister.
- bpo-11571 [https://bugs.python.org/issue?@action=redirect&bpo=11571]: Ensure that the turtle window becomes the topmost window when launched on OS X.
- bpo-21801 [https://bugs.python.org/issue?@action=redirect&bpo=21801]: Validate that __signature__ is None or an instance of Signature.
- bpo-21923 [https://bugs.python.org/issue?@action=redirect&bpo=21923]: Prevent AttributeError in distutils.sysconfig.customize_compiler due to possible uninitialized _config_vars.
- bpo-21323 [https://bugs.python.org/issue?@action=redirect&bpo=21323]: Fix http.server to again handle scripts in CGI subdirectories, broken by the fix for security bpo-19435 [https://bugs.python.org/issue?@action=redirect&bpo=19435]. Patch by Zach Byrne.
- bpo-22733 [https://bugs.python.org/issue?@action=redirect&bpo=22733]: Fix ffi_prep_args not zero-extending argument values correctly on 64-bit Windows.
- bpo-23302 [https://bugs.python.org/issue?@action=redirect&bpo=23302]: Default to TCP_NODELAY=1 upon establishing an HTTPConnection. Removed use of hard-coded MSS as it's an optimization that's no longer needed with Nagle disabled.

IDLE

- bpo-20577 [https://bugs.python.org/issue?@action=redirect&bpo=20577]: Configuration of the max line length for the FormatParagraph extension has been moved from the General tab of the Idle preferences dialog to the FormatParagraph tab of the Config Extensions dialog. Patch by Tal Einat.
- bpo-16893 [https://bugs.python.org/issue?@action=redirect&bpo=16893]: Update Idle doc chapter to match current Idle and add new information.
- bpo-3068 [https://bugs.python.org/issue?@action=redirect&bpo=3068]: Add Idle extension configuration dialog to Options menu. Changes are written

to HOME/.idlerc/config-extensions.cfg. Original patch by Tal Einat.

- bpo-16233 [https://bugs.python.org/issue?@action=redirect&bpo=16233]: A module browser (File : Class Browser, Alt+C) requires an editor window with a filename. When Class Browser is requested otherwise, from a shell, output window, or 'Untitled' editor, Idle no longer displays an error box. It now pops up an Open Module box (Alt+M). If a valid name is entered and a module is opened, a corresponding browser is also opened.
- bpo-4832 [https://bugs.python.org/issue?@action=redirect&bpo=4832]: Save As to type Python files automatically adds .py to the name you enter (even if your system does not display it). Some systems automatically add .txt when type is Text files.
- bpo-21986 [https://bugs.python.org/issue?@action=redirect&bpo=21986]: Code objects are not normally pickled by the pickle module. To match this, they are no longer pickled when running under Idle.
- bpo-17390 [https://bugs.python.org/issue?@action=redirect&bpo=17390]: Adjust Editor window title; remove 'Python', move version to end.
- bpo-14105 [https://bugs.python.org/issue?@action=redirect&bpo=14105]: Idle debugger breakpoints no longer disappear when inserting or deleting lines.
- bpo-17172 [https://bugs.python.org/issue?@action=redirect&bpo=17172]: Turtledemo can now be run from Idle. Currently, the entry is on the Help menu, but it may move to Run. Patch by Ramchandra Apt and Lita Cho.
- bpo-21765 [https://bugs.python.org/issue?@action=redirect&bpo=21765]: Add support for non-ascii identifiers to HyperParser.
- bpo-21940 [https://bugs.python.org/issue?@action=redirect&bpo=21940]: Add unittest for WidgetRedirector. Initial patch by Saimadhav Heblikar.
- bpo-18592 [https://bugs.python.org/issue?@action=redirect&bpo=18592]: Add unittest for SearchDialogBase. Patch by Phil Webster.
- bpo-21694 [https://bugs.python.org/issue?@action=redirect&bpo=21694]: Add unittest for ParenMatch. Patch by Saimadhav Heblikar.
- bpo-21686 [https://bugs.python.org/issue?@action=redirect&bpo=21686]: add unittest for HyperParser. Original patch by Saimadhav Heblikar.
- bpo-12387 [https://bugs.python.org/issue?@action=redirect&bpo=12387]: Add missing upper(lower)case versions of default Windows key bindings for Idle so Caps Lock does not disable them. Patch by Roger Serwy.

- bpo-21695 [https://bugs.python.org/issue?@action=redirect&bpo=21695]: Closing a Find-in-files output window while the search is still in progress no longer closes Idle.
- bpo-18910 [https://bugs.python.org/issue?@action=redirect&bpo=18910]: Add unittest for textView. Patch by Phil Webster.
- bpo-18292 [https://bugs.python.org/issue?@action=redirect&bpo=18292]: Add unittest for AutoExpand. Patch by Saihadhav Heblikar.
- bpo-18409 [https://bugs.python.org/issue?@action=redirect&bpo=18409]: Add unittest for AutoComplete. Patch by Phil Webster.
- bpo-21477 [https://bugs.python.org/issue?@action=redirect&bpo=21477]: htest.py - Improve framework, complete set of tests. Patches by Saimadhav Heblikar
- bpo-18104 [https://bugs.python.org/issue?@action=redirect&bpo=18104]: Add idlelib/idle_test/htest.py with a few sample tests to begin consolidating and improving human-validated tests of Idle. Change other files as needed to work with htest. Running the module as __main__ runs all tests.
- bpo-21139 [https://bugs.python.org/issue?@action=redirect&bpo=21139]: Change default paragraph width to 72, the **PEP 8** [https://peps.python.org/pep-0008/] recommendation.
- bpo-21284 [https://bugs.python.org/issue?@action=redirect&bpo=21284]: Paragraph reformat test passes after user changes reformat width.
- bpo-17654 [https://bugs.python.org/issue?@action=redirect&bpo=17654]: Ensure IDLE menus are customized properly on OS X for non-framework builds and for all variants of Tk.
- bpo-23180 [https://bugs.python.org/issue?@action=redirect&bpo=23180]: Rename IDLE “Windows” menu item to “Window”. Patch by Al Sweigart.

Construção

- bpo-15506 [https://bugs.python.org/issue?@action=redirect&bpo=15506]: Use standard PKG_PROG_PKG_CONFIG autoconf macro in the configure script.
- bpo-22935 [https://bugs.python.org/issue?@action=redirect&bpo=22935]: Allow the ssl module to be compiled if openssl doesn’t support SSL 3.
- bpo-22592 [https://bugs.python.org/issue?@action=redirect&bpo=22592]: Drop support of the Borland C compiler to build Python. The distutils

module still supports it to build extensions.

- bpo-22591 [https://bugs.python.org/issue?@action=redirect&bpo=22591]: Drop support of MS-DOS, especially of the DJGPP compiler (MS-DOS port of GCC).
- bpo-16537 [https://bugs.python.org/issue?@action=redirect&bpo=16537]: Check whether self.extensions is empty in setup.py. Patch by Jonathan Hosmer.
- bpo-22359 [https://bugs.python.org/issue?@action=redirect&bpo=22359]: Remove incorrect uses of recursive make. Patch by Jonas Wagner.
- bpo-21958 [https://bugs.python.org/issue?@action=redirect&bpo=21958]: Define HAVE_ROUND when building with Visual Studio 2013 and above. Patch by Zachary Turner.
- bpo-18093 [https://bugs.python.org/issue?@action=redirect&bpo=18093]: the programs that embed the CPython runtime are now in a separate "Programs" directory, rather than being kept in the Modules directory.
- bpo-15759 [https://bugs.python.org/issue?@action=redirect&bpo=15759]: "make suspicious", "make linkcheck" and "make doctest" in Doc/ now display special message when and only when there are failures.
- bpo-21141 [https://bugs.python.org/issue?@action=redirect&bpo=21141]: The Windows build process no longer attempts to find Perl, instead relying on OpenSSL source being configured and ready to build. The `PCbuild\build_ssl.py` script has been re-written and re-named to `PCbuild\prepare_ssl.py`, and takes care of configuring OpenSSL source for both 32 and 64 bit platforms. OpenSSL sources obtained from svn.python.org will always be pre-configured and ready to build.
- bpo-21037 [https://bugs.python.org/issue?@action=redirect&bpo=21037]: Add a build option to enable AddressSanitizer support.
- bpo-19962 [https://bugs.python.org/issue?@action=redirect&bpo=19962]: The Windows build process now creates "python.bat" in the root of the source tree, which passes all arguments through to the most recently built interpreter.
- bpo-21285 [https://bugs.python.org/issue?@action=redirect&bpo=21285]: Refactor and fix curses configure check to always search in a ncursesw directory.
- bpo-15234 [https://bugs.python.org/issue?@action=redirect&bpo=15234]: For BerkeleyDB and Sqlite, only add the found library and include

directories if they aren't already being searched. This avoids an explicit runtime library dependency.

- bpo-17861 [https://bugs.python.org/issue?@action=redirect&bpo=17861]: Tools/scripts/generate_opcode_h.py automatically regenerates Include/opcode.h from Lib/opcode.py if the latter gets any change.
- bpo-20644 [https://bugs.python.org/issue?@action=redirect&bpo=20644]: OS X installer build support for documentation build changes in 3.4.1: assume externally supplied sphinx-build is available in /usr/bin.
- bpo-20022 [https://bugs.python.org/issue?@action=redirect&bpo=20022]: Eliminate use of deprecated bundlebuilder in OS X builds.
- bpo-15968 [https://bugs.python.org/issue?@action=redirect&bpo=15968]: Incorporated Tcl, Tk, and Tix builds into the Windows build solution.
- bpo-17095 [https://bugs.python.org/issue?@action=redirect&bpo=17095]: Fix Modules/Setup *shared* support.
- bpo-21811 [https://bugs.python.org/issue?@action=redirect&bpo=21811]: Anticipated fixes to support OS X versions > 10.9.
- bpo-21166 [https://bugs.python.org/issue?@action=redirect&bpo=21166]: Prevent possible segfaults and other random failures of python –generate-posix-vars in pybuilddir.txt build target.
- bpo-18096 [https://bugs.python.org/issue?@action=redirect&bpo=18096]: Fix library order returned by python-config.
- bpo-17219 [https://bugs.python.org/issue?@action=redirect&bpo=17219]: Add library build dir for Python extension cross-builds.
- bpo-22919 [https://bugs.python.org/issue?@action=redirect&bpo=22919]: Windows build updated to support VC 14.0 (Visual Studio 2015), which will be used for the official release.
- bpo-21236 [https://bugs.python.org/issue?@action=redirect&bpo=21236]: Build _msi.pyd with cabinet.lib instead of fci.lib
- bpo-17128 [https://bugs.python.org/issue?@action=redirect&bpo=17128]: Use private version of OpenSSL for OS X 10.5+ installer.

API C

- bpo-14203 [https://bugs.python.org/issue?@action=redirect&bpo=14203]: Remove obsolete support for view==NULL in PyBuffer_FillInfo(), bytearray_getbuffer(), bytesiobuf_getbuffer() and array_buffer_getbuf(). All functions now raise BufferError in that case.

- bpo-22445 [https://bugs.python.org/issue?@action=redirect&bpo=22445]: PyBuffer_IsContiguous() now implements precise contiguity tests, compatible with NumPy's NPY_RELAXED_STRIDES_CHECKING compilation flag. Previously the function reported false negatives for corner cases.
- bpo-22079 [https://bugs.python.org/issue?@action=redirect&bpo=22079]: PyType_Ready() now checks that statically allocated type has no dynamically allocated bases.
- bpo-22453 [https://bugs.python.org/issue?@action=redirect&bpo=22453]: Removed non-documented macro PyObject_REPR().
- bpo-18395 [https://bugs.python.org/issue?@action=redirect&bpo=18395]: Rename `_Py_char2wchar()` to **`Py_DecodeLocale()`**, rename `_Py_wchar2char()` to **`Py_EncodeLocale()`**, and document these functions.
- bpo-21233 [https://bugs.python.org/issue?@action=redirect&bpo=21233]: Add new C functions: PyMem_RawCalloc(), PyMem_Calloc(), PyObject_Calloc(), _PyObject_GC_Calloc(). bytes(int) is now using `calloc()` instead of `malloc()` for large objects which is faster and use less memory.
- bpo-20942 [https://bugs.python.org/issue?@action=redirect&bpo=20942]: PyImport_ImportFrozenModuleObject() no longer sets __file__ to match what importlib does; this affects _frozen_importlib as well as any module loaded using imp.init_frozen().

Documentação

- bpo-19548 [https://bugs.python.org/issue?@action=redirect&bpo=19548]: Update the codecs module documentation to better cover the distinction between text encodings and other codecs, together with other clarifications. Patch by Martin Panter.
- bpo-22394 [https://bugs.python.org/issue?@action=redirect&bpo=22394]: Doc/Makefile now supports `make venv PYTHON=../python` to create a venv for generating the documentation, e.g., `make html PYTHON=venv/bin/python3`.
- bpo-21514 [https://bugs.python.org/issue?@action=redirect&bpo=21514]: The documentation of the json module now refers to new JSON RFC 7159 instead of obsoleted RFC 4627.

- bpo-21777 [https://bugs.python.org/issue?@action=redirect&bpo=21777]: The binary sequence methods on bytes and bytearray are now documented explicitly, rather than assuming users will be able to derive the expected behaviour from the behaviour of the corresponding str methods.
- bpo-6916 [https://bugs.python.org/issue?@action=redirect&bpo=6916]: remove documentação da classe asynchat.fifo descontinuada.
- bpo-17386 [https://bugs.python.org/issue?@action=redirect&bpo=17386]: Expandida a funcionalidade do script `Doc/make.bat` para fazê-lo muito mais comparável a `Doc/Makefile`.
- bpo-21312 [https://bugs.python.org/issue?@action=redirect&bpo=21312]: Update the thread_foobar.h template file to include newer threading APIs. Patch by Jack McCracken.
- bpo-21043 [https://bugs.python.org/issue?@action=redirect&bpo=21043]: Remove the recommendation for specific CA organizations and to mention the ability to load the OS certificates.
- bpo-20765 [https://bugs.python.org/issue?@action=redirect&bpo=20765]: Adiciona documentação faltante para PurePath.with_name() e PurePath.with_suffix().
- bpo-19407 [https://bugs.python.org/issue?@action=redirect&bpo=19407]: New package installation and distribution guides based on the Python Packaging Authority tools. Existing guides have been retained as legacy links from the distutils docs, as they still contain some required reference material for tool developers that isn't recorded anywhere else.
- bpo-19697 [https://bugs.python.org/issue?@action=redirect&bpo=19697]: Documenta casos em que __main__.__spec__ é None.

Testes

- bpo-18982 [https://bugs.python.org/issue?@action=redirect&bpo=18982]: Adiciona testes para ILC do módulo calendar.
- bpo-19548 [https://bugs.python.org/issue?@action=redirect&bpo=19548]: Added some additional checks to test_codecs to ensure that statements in the updated documentation remain accurate. Patch by Martin Panter.
- bpo-22838 [https://bugs.python.org/issue?@action=redirect&bpo=22838]: All test_re tests now work with unittest test discovery.

- bpo-22173 [https://bugs.python.org/issue?@action=redirect&bpo=22173]: Update lib2to3 tests to use unittest test discovery.
- bpo-16000 [https://bugs.python.org/issue?@action=redirect&bpo=16000]: Convert test_curses to use unittest.
- bpo-21456 [https://bugs.python.org/issue?@action=redirect&bpo=21456]: Skip two tests in test_urllib2net.py if _ssl module not present. Patch by Remi Pointel.
- bpo-20746 [https://bugs.python.org/issue?@action=redirect&bpo=20746]: Fix test_pdb to run in refleak mode (-R). Patch by Xavier de Gaye.
- bpo-22060 [https://bugs.python.org/issue?@action=redirect&bpo=22060]: test_ctypes has been somewhat cleaned up and simplified; it now uses unittest test discovery to find its tests.
- bpo-22104 [https://bugs.python.org/issue?@action=redirect&bpo=22104]: regrtest.py no longer holds a reference to the suite of tests loaded from test modules that don't define test_main().
- bpo-22111 [https://bugs.python.org/issue?@action=redirect&bpo=22111]: Assorted cleanups in test_imaplib. Patch by Milan Oberkirch.
- bpo-22002 [https://bugs.python.org/issue?@action=redirect&bpo=22002]: Added `load_package_tests` function to test.support and used it to implement/augment test discovery in test_asyncio, test_email, test_importlib, test_json, and test_tools.
- bpo-21976 [https://bugs.python.org/issue?@action=redirect&bpo=21976]: Fix test_ssl to accept LibreSSL version strings. Thanks to William Orr.
- bpo-21918 [https://bugs.python.org/issue?@action=redirect&bpo=21918]: Converted test_tools from a module to a package containing separate test files for each tested script.
- bpo-9554 [https://bugs.python.org/issue?@action=redirect&bpo=9554]: Use modern unittest features in test_argparse. Initial patch by Denver Coneybeare and Radu Voicilas.
- bpo-20155 [https://bugs.python.org/issue?@action=redirect&bpo=20155]: Changed HTTP method names in failing tests in test_httpservers so that packet filtering software (specifically Windows Base Filtering Engine) does not interfere with the transaction semantics expected by the tests.
- bpo-19493 [https://bugs.python.org/issue?@action=redirect&bpo=19493]: Refactored the ctypes test package to skip tests explicitly rather than silently.
- bpo-18492 [https://bugs.python.org/issue?@action=redirect&bpo=18492]: All resources are now allowed when tests are not run by regrtest.py.

- bpo-21634 [https://bugs.python.org/issue?@action=redirect&bpo=21634]: Fix pystone micro-benchmark: use floor division instead of true division to benchmark integers instead of floating-point numbers. Set pystone version to 1.2. Patch written by Lennart Regebro.
- bpo-21605 [https://bugs.python.org/issue?@action=redirect&bpo=21605]: Added tests for Tkinter images.
- bpo-21493 [https://bugs.python.org/issue?@action=redirect&bpo=21493]: Added test for ntpath.expanduser(). Original patch by Claudiu Popa.
- bpo-19925 [https://bugs.python.org/issue?@action=redirect&bpo=19925]: Added tests for the spwd module. Original patch by Vajrasky Kok.
- bpo-21522 [https://bugs.python.org/issue?@action=redirect&bpo=21522]: Added Tkinter tests for Listbox.itemconfigure(), PanedWindow.paneconfigure(), and Menu.entryconfigure().
- bpo-17756 [https://bugs.python.org/issue?@action=redirect&bpo=17756]: Fix test_code test when run from the installed location.
- bpo-17752 [https://bugs.python.org/issue?@action=redirect&bpo=17752]: Fix distutils tests when run from the installed location.
- bpo-18604 [https://bugs.python.org/issue?@action=redirect&bpo=18604]: Consolidated checks for GUI availability. All platforms now at least check whether Tk can be instantiated when the GUI resource is requested.
- bpo-21275 [https://bugs.python.org/issue?@action=redirect&bpo=21275]: Fix a socket test on KFreeBSD.
- bpo-21223 [https://bugs.python.org/issue?@action=redirect&bpo=21223]: Pass test_site/test_startup_imports when some of the extensions are built as builtins.
- bpo-20635 [https://bugs.python.org/issue?@action=redirect&bpo=20635]: Added tests for Tk geometry managers.
- Add test case for freeze.
- bpo-20743 [https://bugs.python.org/issue?@action=redirect&bpo=20743]: Fix a reference leak in test_tcl.
- bpo-21097 [https://bugs.python.org/issue?@action=redirect&bpo=21097]: Move test_namespace_pkgs into test_importlib.
- bpo-21503 [https://bugs.python.org/issue?@action=redirect&bpo=21503]: Use test_both() consistently in test_importlib.
- bpo-20939 [https://bugs.python.org/issue?@action=redirect&bpo=20939]: Avoid various network test failures due to new redirect of

http://www.python.org/ to https://www.python.org: use http://www.example.com instead.

- bpo-20668 [https://bugs.python.org/issue?@action=redirect&bpo=20668]: asyncio tests no longer rely on tests.txt file. (Patch by Vajrasky Kok)
- bpo-21093 [https://bugs.python.org/issue?@action=redirect&bpo=21093]: Prevent failures of ctypes test_macholib on OS X if a copy of libz exists in $HOME/lib or /usr/local/lib.
- bpo-22770 [https://bugs.python.org/issue?@action=redirect&bpo=22770]: Prevent some Tk segfaults on OS X when running gui tests.
- bpo-23211 [https://bugs.python.org/issue?@action=redirect&bpo=23211]: Workaround test_logging failure on some OS X 10.6 systems.
- bpo-23345 [https://bugs.python.org/issue?@action=redirect&bpo=23345]: Prevent test_ssl failures with large OpenSSL patch level values (like 0.9.8zc).

Ferramentas/Demos

- bpo-22314 [https://bugs.python.org/issue?@action=redirect&bpo=22314]: pydoc now works when the LINES environment variable is set.
- bpo-22615 [https://bugs.python.org/issue?@action=redirect&bpo=22615]: Argument Clinic now supports the “type” argument for the int converter. This permits using the int converter with enums and typedefs.
- bpo-20076 [https://bugs.python.org/issue?@action=redirect&bpo=20076]: The makelocalealias.py script no longer ignores UTF-8 mapping.
- bpo-20079 [https://bugs.python.org/issue?@action=redirect&bpo=20079]: The makelocalealias.py script now can parse the SUPPORTED file from glibc sources and supports command line options for source paths.
- bpo-22201 [https://bugs.python.org/issue?@action=redirect&bpo=22201]: Command-line interface of the zipfile module now correctly extracts ZIP files with directory entries. Patch by Ryan Wilson.
- bpo-22120 [https://bugs.python.org/issue?@action=redirect&bpo=22120]: For functions using an unsigned integer return converter, Argument Clinic now generates a cast to that type for the comparison to -1 in the generated code. (This suppresses a compilation warning.)
- bpo-18974 [https://bugs.python.org/issue?@action=redirect&bpo=18974]: Tools/scripts/diff.py now uses argparse instead of optparse.
- bpo-21906 [https://bugs.python.org/issue?@action=redirect&bpo=21906]: Make Tools/scripts/md5sum.py work in Python 3. Patch by Zachary Ware.

- bpo-21629 [https://bugs.python.org/issue?@action=redirect&bpo=21629]: Fix Argument Clinic's "–converters" feature.
- Adiciona suporte a `yield from` para 2to3.
- Adiciona suporte ao operador de multiplicação de matriz **PEP 465** [https://peps.python.org/pep-0465/] para 2to3.
- bpo-16047 [https://bugs.python.org/issue?@action=redirect&bpo=16047]: Fix module exception list and __file__ handling in freeze. Patch by Meador Inge.
- bpo-11824 [https://bugs.python.org/issue?@action=redirect&bpo=11824]: Consider ABI tags in freeze. Patch by Meador Inge.
- bpo-20535 [https://bugs.python.org/issue?@action=redirect&bpo=20535]: PYTHONWARNING no longer affects the run_tests.py script. Patch by Arfrever Frehtes Taifersar Arahesis.

Windows

- bpo-23260 [https://bugs.python.org/issue?@action=redirect&bpo=23260]: Update Windows installer
- A versão do pacote Tcl/Tk foi atualizada para 8.6.3. O resultado mais visível dessa alteração é a adição de novas caixas de diálogo de arquivo nativo ao executar no Windows Vista ou mais recente. Consulte o TIP 432 da Tcl/Tk para obter mais informações. Além disso, esta versão do Tcl/Tk inclui suporte ao Windows 10.
- bpo-17896 [https://bugs.python.org/issue?@action=redirect&bpo=17896]: The Windows build scripts now expect external library sources to be in `PCbuild\..\externals` rather than `PCbuild\..\...`
- bpo-17717 [https://bugs.python.org/issue?@action=redirect&bpo=17717]: The Windows build scripts now use a copy of NASM pulled from svn.python.org to build OpenSSL.
- bpo-21907 [https://bugs.python.org/issue?@action=redirect&bpo=21907]: Improved the batch scripts provided for building Python.
- bpo-22644 [https://bugs.python.org/issue?@action=redirect&bpo=22644]: The bundled version of OpenSSL has been updated to 1.0.1j.
- bpo-10747 [https://bugs.python.org/issue?@action=redirect&bpo=10747]: Use versioned labels in the Windows start menu. Patch by Olive Kilburn.
- bpo-22980 [https://bugs.python.org/issue?@action=redirect&bpo=22980]: arquivos .pyd com identificador de versão e plataforma (por exemplo, ".cp35-

win32.pyd”) agora serão carregados em preferência aos que não tem identificadores.

(Para informações sobre versões mais antigas, consulte o arquivo HISTORY.)

O tutorial do Python

Dica

Este tutorial foi desenvolvido para pessoas *que já programam*, mas são recém-chegadas na linguagem Python, **não** pessoas *iniciantes* em programação.

Python é uma linguagem fácil de aprender e poderosa. Ela tem estruturas de dados de alto nível eficientes e uma abordagem simples mas efetiva de programação orientada a objetos. A sintaxe elegante e a tipagem dinâmica do Python, aliadas com sua natureza interpretativa, o tornam uma linguagem ideal para fazer scripts e desenvolvimento de aplicações rápidas em diversas áreas e na maioria das plataformas.

O interpretador Python e a extensiva biblioteca padrão estão disponíveis gratuitamente em código ou na forma binária para toda as maiores plataformas no endereço eletrônico do Python, https://www.python.org/, e pode ser livremente distribuído. O mesmo endereço contém distribuições de diversos módulos, programas e ferramentas gratuitos produzidos por terceiros e documentação adicional.

O interpretador do Python pode ser facilmente estendido com novas funções e tipos de dados implementados em C ou C++ (ou outras linguagens chamáveis a partir de C). Python também é adequado como uma linguagem de extensão para aplicações personalizáveis.

Este tutorial introduz informalmente o leitor aos conceitos básicos e aos recursos da linguagem e do sistema Python. Esteja ciente de que ele espera que você tenha um conhecimento básico de programação em geral. É mais fácil se você possuir um interpretador Python para uma experiência prática, mas os exemplos são autossuficientes e, portanto, o tutorial pode apenas ser lido off-line também.

Para uma descrição detalhada dos módulos e objetos padrões, veja Referência da bibliotecas embutidas do Python e A biblioteca padrão do Python. Em A Referência da Linguagem Python você encontra uma definição mais formal da linguagem. Para escrever extensões em C ou C++, leia Estendendo e incorporando o interpretador Python e Manual de referência da API C/Python. Existe também uma série de livros que cobrem Python em profundidade.

Este tutorial não espera ser abrangente e cobrir todos os recursos ou mesmo os recursos mais usados. Ele busca introduzir diversos dos recursos mais notáveis do Python e lhe dará uma boa ideia do sabor e estilo da linguagem. Depois de lê-lo, você terá condições de ler e escrever programas e módulos Python e estará pronto para aprender mais sobre os diversos módulos descritos em A biblioteca padrão do Python.

O Glossário também vale a pena ser estudado.

- 1. Abrindo seu apetite
- 2. Utilizando o interpretador Python
  - 2.1. Chamando o interpretador
    - 2.1.1. Passagem de argumentos
    - 2.1.2. Modo interativo
  - 2.2. O interpretador e seu ambiente
    - 2.2.1. Edição de código-fonte
- 3. Uma introdução informal ao Python
  - 3.1. Usando Python como uma calculadora
    - 3.1.1. Números
    - 3.1.2. Texto
    - 3.1.3. Listas
  - 3.2. Primeiros passos para a programação
- 4. Mais ferramentas de controle de fluxo
  - 4.1. Instruções `if`
  - 4.2. Instruções `for`
  - 4.3. A função `range()`
  - 4.4. Instruções `break` e `continue`
  - 4.5. Cláusulas `else` em laços
  - 4.6. Instruções `pass`

- 4.7. Instruções `match`
- 4.8. Definindo funções
- 4.9. Mais sobre definição de funções
  - 4.9.1. Argumentos com valor padrão
  - 4.9.2. Argumentos nomeados
  - 4.9.3. Parâmetros especiais
    - 4.9.3.1. Argumentos posicional-ou-nomeados
    - 4.9.3.2. Parâmetros somente-posicionais
    - 4.9.3.3. Argumentos somente-nomeados
    - 4.9.3.4. Exemplos de funções
    - 4.9.3.5. Recapitulando
  - 4.9.4. Listas de argumentos arbitrárias
  - 4.9.5. Desempacotando listas de argumentos
  - 4.9.6. Expressões lambda
  - 4.9.7. Strings de documentação
  - 4.9.8. Anotações de função
- 4.10. Intermezzo: estilo de codificação

- 5. Estruturas de dados
  - 5.1. Mais sobre listas
    - 5.1.1. Usando listas como pilhas
    - 5.1.2. Usando listas como filas
    - 5.1.3. Compreensões de lista
    - 5.1.4. Compreensões de lista aninhadas
    - 5.1.5. Desempacotamento em Listas e Compreensões de Lista
  - 5.2. A instrução `del`
  - 5.3. Tuplas e Sequências
  - 5.4. Conjuntos
  - 5.5. Dicionários
  - 5.6. Técnicas de iteração
  - 5.7. Mais sobre condições
  - 5.8. Comparando sequências e outros tipos

- 6. Módulos
  - 6.1. Mais sobre módulos
    - 6.1.1. Executando módulos como scripts
    - 6.1.2. O caminho de busca dos módulos
    - 6.1.3. Arquivos Python “compilados”

- 6.2. Módulos padrões
- 6.3. A função `dir()`
- 6.4. Pacotes
  - 6.4.1. Importando * de um pacote
  - 6.4.2. Referências em um mesmo pacote
- 7. Entrada e saída
  - 7.1. Formatação de saída mais elaborada
    - 7.1.1. Literais de string formatados
    - 7.1.2. O método format() de strings
    - 7.1.3. Formatação manual de strings
    - 7.1.4. Formatação de strings à moda antiga
  - 7.2. Lendo e escrevendo arquivos
    - 7.2.1. Métodos de objetos arquivo
    - 7.2.2. Gravando dados estruturados com `json`
- 8. Erros e exceções
  - 8.1. Erros de sintaxe
  - 8.2. Exceções
  - 8.3. Tratamento de exceções
  - 8.4. Levantando exceções
  - 8.5. Encadeamento de exceções
  - 8.6. Exceções definidas pelo usuário
  - 8.7. Definindo ações de limpeza
  - 8.8. Ações de limpeza predefinidas
  - 8.9. Criando e tratando várias exceções não relacionadas
  - 8.10. Enriquecendo exceções com notas
- 9. Classes
  - 9.1. Uma palavra sobre nomes e objetos
  - 9.2. Escopos e espaços de nomes do Python
    - 9.2.1. Exemplo de escopos e espaço de nomes
  - 9.3. Uma primeira olhada nas classes
    - 9.3.1. Sintaxe da definição de classe
    - 9.3.2. Objetos classe
    - 9.3.3. Objetos instância
    - 9.3.4. Objetos método
    - 9.3.5. Variáveis de classe e instância
  - 9.4. Observações aleatórias
  - 9.5. Herança

        - 9.5.1. Herança múltipla
    - 9.6. Variáveis privadas
    - 9.7. Curiosidades e conclusões
    - 9.8. Iteradores
    - 9.9. Geradores
    - 9.10. Expressões geradoras
- 10. Um breve passeio pela biblioteca padrão
    - 10.1. Interface com o sistema operacional
    - 10.2. Caracteres curinga
    - 10.3. Argumentos de linha de comando
    - 10.4. Redirecionamento de erros e encerramento do programa
    - 10.5. Reconhecimento de padrões em strings
    - 10.6. Matemática
    - 10.7. Acesso à internet
    - 10.8. Data e hora
    - 10.9. Compressão de dados
    - 10.10. Medição de desempenho
    - 10.11. Controle de qualidade
    - 10.12. Baterias incluídas
- 11. Um breve passeio pela biblioteca padrão — parte II
    - 11.1. Formatando a saída
    - 11.2. Usando templates
    - 11.3. Trabalhando com formatos binários de dados
    - 11.4. Multi-threading
    - 11.5. Gerando logs
    - 11.6. Referências fracas
    - 11.7. Ferramentas para trabalhar com listas
    - 11.8. Aritmética decimal com ponto flutuante
- 12. Ambientes virtuais e pacotes
    - 12.1. Introdução
    - 12.2. Criando ambientes virtuais
    - 12.3. Gerenciando pacotes com o pip
- 13. E agora?
- 14. Edição de entrada interativa e substituição de histórico
    - 14.1. Tab completion e histórico de edição
    - 14.2. Alternativas ao interpretador interativo
- 15. Aritmética de ponto flutuante: problemas e limitações

- 15.1. Erro de representação
- 16. Anexo
  - 16.1. Modo interativo
    - 16.1.1. Tratamento de erros
    - 16.1.2. Scripts Python executáveis
    - 16.1.3. Arquivo de inicialização do modo interativo
    - 16.1.4. Módulos de customização

1. Abrindo seu apetite

Se você trabalha bastante com computadores, eventualmente perceberá que há alguma tarefa que gostaria de automatizar. Por exemplo, pode querer realizar uma busca e substituição em um grande número de arquivos de texto ou renomear e reorganizar um conjunto de fotos de maneira complexa. Talvez deseje criar um pequeno banco de dados personalizado, uma aplicação GUI especializada ou um jogo simples.

Se você é um desenvolvedor de software profissional, pode ter que trabalhar com várias bibliotecas C/C++/Java, mas o tradicional ciclo escrever/compilar/testar/recompilar é muito lento. Talvez você esteja escrevendo um conjunto de testes para uma biblioteca e está achando tedioso codificar os testes. Ou talvez você tenha escrito um programa que poderia utilizar uma linguagem de extensão, e você não quer conceber e implementar toda uma nova linguagem para sua aplicação.

Python é a linguagem para você.

Você pode escrever um script shell do Unix ou arquivos batch do Windows para algumas dessas tarefas, mas os scripts shell são melhores para mover arquivos e alterar dados de texto, não são adequados para aplicações GUI ou jogos. Você pode escrever um programa C/C++/Java, mas pode levar muito tempo de desenvolvimento para obter até mesmo um primeiro rascunho de programa. O Python é mais simples de usar, está disponível nos sistemas operacionais Windows, macOS e Unix e ajudará você a terminar o trabalho mais rapidamente.

Python é fácil de usar, sem deixar de ser uma linguagem de programação de verdade, oferecendo muito mais estruturação e suporte para programas extensos do que shell scripts ou arquivos de lote oferecem. Por outro lado, Python também oferece melhor verificação de erros do que C, e por ser uma linguagem de *muito alto nível*, ela possui tipos nativos de alto nível, tais como dicionários e vetores (arrays) flexíveis. Devido ao suporte nativo a uma variedade de tipos de dados, Python é aplicável a um domínio de

problemas muito mais vasto do que Awk ou até mesmo Perl, ainda assim muitas tarefas são pelo menos tão fáceis em Python quanto nessas linguagens.

Python permite que você organize seu programa em módulos que podem ser reutilizados em outros programas escritos em Python. A linguagem provê uma vasta coleção de módulos que podem ser utilizados como base para sua aplicação — ou como exemplos para estudo e aprofundamento. Alguns desses módulos implementam manipulação de arquivos, chamadas do sistema, sockets, e até mesmo acesso a bibliotecas de construção de interfaces gráficas, como Tk.

Python é uma linguagem interpretada, por isso você pode economizar um tempo considerável durante o desenvolvimento, uma vez que não há necessidade de compilação e ligação (*linking*). O interpretador pode ser usado interativamente, o que torna fácil experimentar diversas características da linguagem, escrever programas "descartáveis", ou testar funções em um desenvolvimento debaixo para cima (*bottom-up*). É também uma útil calculadora de mesa.

Python permite a escrita de programas compactos e legíveis. Programas escritos em Python são tipicamente mais curtos do que seus equivalentes em C, C++ ou Java, por diversas razões:

- os tipos de alto nível permitem que você expresse operações complexas em um único comando;
- a definição de bloco é feita por indentação ao invés de marcadores de início e fim de bloco;
- não há necessidade de declaração de variáveis ou parâmetros formais;

Python é *extensível*: se você sabe como programar em C, é fácil adicionar módulos ou funções embutidas diretamente no interpretador, seja para desempenhar operações críticas em máxima velocidade, ou para vincular programas Python a bibliotecas que só estejam disponíveis em formato binário (como uma biblioteca gráfica de terceiros). Uma vez que você tenha sido fisgado, você pode vincular o interpretador Python a uma aplicação escrita em C e utilizá-la como linguagem de comandos ou extensão para esta aplicação.

A propósito, a linguagem foi batizada a partir do famoso programa da BBC “Monty Python’s Flying Circus” e não tem nada a ver com répteis. Fazer referências a citações do programa na documentação não é só permitido, como também é encorajado!

Agora que você se entusiasmou com o Python, vai querer conhecê-lo com mais detalhes. Partindo do princípio que a melhor maneira de aprender uma linguagem é usando-a, este tutorial lhe convida para fazê-lo com o interpretador do Python enquanto você o lê.

No próximo capítulo, a mecânica de utilização do interpretador é explicada. Essa informação, ainda que mundana, é essencial para a experimentação dos exemplos apresentados mais tarde.

O resto do tutorial introduz diversos aspectos do sistema e linguagem Python por intermédio de exemplos. Serão abordadas expressões simples, instruções, tipos, funções e módulos. Finalmente, serão explicados alguns conceitos avançados como exceções e classes definidas pelo usuário.

2. Utilizando o interpretador Python

2.1. Chamando o interpretador

O interpretador Python é frequentemente instalado como `/usr/local/bin/python3.15` nas máquinas onde está disponível; adicionando `/usr/local/bin` ao caminho de busca da shell de seu Unix torna-se possível iniciá-lo digitando o comando:

```
python3.15
```

no shell. [[1]] Considerando que a escolha do diretório onde o interpretador está é uma opção de instalação, outros locais são possíveis; verifique com seu guru local de Python ou administrador do sistema local. (Por exemplo, `/usr/local/python` é um local alternativo popular.)

Em máquinas Windows onde você instalou Python a partir da Microsoft Store, o comando `python3.15` estará disponível. Se você tem o lançador py.exe instalado, você pode usar o comando `py`. Veja Gerenciador de instalação do Python para outras maneiras de executar o Python.

Digitar um caractere de fim de arquivo (`Control-D` no Unix, `Control-Z` no Windows) diretamente no prompt força o interpretador a sair com status de saída zero. Se isso não funcionar, você pode sair do interpretador digitando o seguinte comando: `quit()`.

Os recursos de edição de linha do interpretador incluem edição interativa, substituição de histórico e autocompletar código na maioria dos sistemas. Talvez a maneira mais rápida de verificar se a edição da linha de comando é suportada seja digitar uma palavra no prompt do Python e, em seguida, pressionar a seta para a esquerda (ou `Control-b`). Se o cursor mover-se, tem edição de linha de comando; consulte o Apêndice Edição de entrada interativa e substituição de histórico para uma introdução às teclas. Se nada

parecer acontecer, ou se uma sequência como `^[[D` ou `^B` aparecer, a edição de linha de comando não está disponível; só poderá usar a tecla Backspace para remover caracteres da linha atual.

O interpretador trabalha de forma semelhante a uma shell de Unix: quando chamado com a saída padrão conectada a um console de terminal, ele lê e executa comandos interativamente; quando chamado com um nome de arquivo como argumento, ou com redirecionamento da entrada padrão para ler um arquivo, o interpretador lê e executa o *script* contido no arquivo.

Uma segunda forma de iniciar o interpretador é `python -c comando [arg] ...`, que executa uma ou mais instruções especificadas na posição *comando*, analogamente à opção de shell **`-c`**. Considerando que comandos Python frequentemente têm espaços em branco (ou outros caracteres que são especiais para a shell) é aconselhável que todo o *comando* esteja entre as aspas.

Alguns módulos Python são também úteis como scripts. Estes podem ser chamados usando `python -m módulo [arg] ...` que executa o arquivo fonte do *módulo* como se você tivesse digitado seu caminho completo na linha de comando.

Quando um arquivo de script é utilizado, às vezes é útil executá-lo e logo em seguida entrar em modo interativo. Isto pode ser feito acrescentando o argumento **`-i`** antes do nome do script.

Todas as opções de linha de comando são descritas em Linha de comando e ambiente.

2.1.1. Passagem de argumentos

Quando são de conhecimento do interpretador, o nome do script e demais argumentos da linha de comando da shell são acessíveis ao próprio script através da variável `argv` do módulo `sys`. Pode-se acessar essa lista executando `import sys`. Essa lista tem sempre ao menos um elemento; quando nenhum script ou argumento for passado para o interpretador, `sys.argv[0]` será uma string vazia. Quando o nome do script for `'-'` (significando entrada padrão), o conteúdo de `sys.argv[0]` será `'-'`.

Quando for utilizado `-c` *comando*, `sys.argv[0]` conterá `'-c'`. Quando for utilizado `-m` *módulo*, `sys.argv[0]` conterá o caminho completo do módulo localizado. Opções especificadas após **`-c`** *comando* ou **`-m`** *módulo* não serão consumidas pelo interpretador mas deixadas em `sys.argv` para serem tratadas pelo comando ou módulo.

2.1.2. Modo interativo

Quando os comandos são lidos a partir do console, diz-se que o interpretador está em modo interativo. Nesse modo ele solicita um próximo comando através do *prompt primário*, tipicamente três sinais de maior (`>>>`); para linhas de continuação do comando atual, o *prompt secundário* padrão é formado por três pontos (`...`). O interpretador exibe uma mensagem de boas vindas, informando seu número de versão e um aviso de copyright antes de exibir o primeiro prompt:

```
$ python3.15
Python 3.15 (default, May 7 2025, 15:46:04)
[GCC 10.2.0] on linux
Type "help", "copyright", "credits" or "license" for more inform
>>>
```

Linhas de continuação são necessárias em construções multi-linha. Como exemplo, dê uma olhada nesse comando `if`:

```
>>> o_mundo_é_plano = True
>>> if o_mundo_é_plano:
...     print("Cuidado para não cair da borda!")
...
Cuidado para não cair da borda!
```

Para mais informações sobre o modo interativo, veja Modo interativo.

2.2. O interpretador e seu ambiente

2.2.1. Edição de código-fonte

Por padrão, arquivos fonte de Python são tratados com codificação UTF-8. Nessa codificação, caracteres de muitos idiomas no mundo podem ser usados simultaneamente em literais string, identificadores e comentários — embora a biblioteca padrão use apenas caracteres ASCII para identificadores, uma convenção que qualquer código portável deve seguir. Para exibir todos esses caracteres corretamente, seu editor deve reconhecer que o arquivo é UTF-8 e deve usar uma fonte com suporte a todos os caracteres no arquivo.

Para declarar uma codificação diferente da padrão, uma linha de comentário especial deve ser adicionada como *primeira* linha do arquivo. A sintaxe é essa:

```
# -*- coding: encoding -*-
```

onde *encoding* é uma das **codecs** válidas com suporte do Python.

Por exemplo, para declarar que a codificação Windows-1252 deve ser usada, a primeira linha do seu arquivo fonte deve ser:

```
# -*- coding: cp1252 -*-
```

Uma exceção para a regra da *primeira linha* é quando o código-fonte inicia com uma linha com UNIX "shebang". Nesse caso, a declaração de codificação deve ser adicionada como a segunda linha do arquivo. Por exemplo:

```
#!/usr/bin/env python3
# -*- coding: cp1252 -*-
```

Notas de rodapé

[1]

No Unix, o interpretador Python 3.x não é instalado por padrão com o executável nomeado `python`, então não vai conflitar com um executável Python 2.x instalado simultaneamente.

3. Uma introdução informal ao Python

Nos exemplos seguintes, pode-se distinguir entrada e saída pela presença ou ausência dos prompts (>>> e ...): para repetir o exemplo, você deve digitar tudo após o prompt, quando o mesmo aparece; linhas que não começarem com um prompt são na verdade as saídas geradas pelo interpretador. Observe que quando aparece uma linha contendo apenas o prompt secundário você deve digitar uma linha em branco; é assim que se encerra um comando de múltiplas linhas.

Você pode usar o botão “Copy” (que aparece no canto superior direito ao passar o mouse ou tocar em um exemplo de código), que remove os prompts e omite a saída, para copiar e colar as linhas de entrada no seu interpretador.

Muitos exemplos neste manual, mesmo aqueles inscritos na linha de comando interativa, incluem comentários. Comentários em Python começam com o caractere cerquilha `#` e estende até o final da linha. Um comentário pode aparecer no inicio da linha ou após espaço em branco ou código, mas não dentro de uma string literal. O caractere cerquilha dentro de uma string literal é apenas uma cerquilha. Como os comentários são para esclarecer o código e não são interpretados pelo Python, eles podem ser omitidos ao digitar exemplos.

Alguns exemplos:

```
# este é o primeiro comentário
spam = 1  # e este é o segundo comentário
          # ... e agora um terceiro!
texto = "# Este não é um comentário por estar entre aspas."
```

3.1. Usando Python como uma calculadora

Vamos experimentar alguns comandos simples em Python. Inicie o interpretador e aguarde o prompt primário, `>>>`. (Não deve demorar muito.)

3.1.1. Números

O interpretador funciona como uma calculadora bem simples: você pode digitar uma expressão nele e o resultado será apresentado. A sintaxe de expressões é a usual: operadores `+`, `-`, `*` e `/` podem ser usadas para realizar operações aritméticas; parênteses (`()`) podem ser usados para agrupar expressões. Por exemplo:

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5  # divisão sempre retorna um número de ponto flutuante
1.6
```

Os números inteiros (por exemplo, `2`, `4` e `20`) são do tipo **`int`**, aqueles com parte fracionária (por exemplo, `5.0` e `1.6`) são do tipo **`float`**. Veremos mais sobre tipos numéricos posteriormente neste tutorial.

Divisão (`/`) sempre retorna ponto flutuante (float). Para fazer uma divisão pelo piso e receber um inteiro como resultado você pode usar o operador `//`; para calcular o resto você pode usar o `%`:

```
>>> 17 / 3  # divisão clássica retorna um ponto flutuante
5.666666666666667
>>>
>>> 17 // 3  # divisão pelo piso descarta a parte fracionária
5
>>> 17 % 3  # o operador % retorna o resto da divisão
2
>>> 5 * 3 + 2  # quociente do piso * divisor + restante
17
```

Com Python, é possível usar o operador `**` para calcular potências [[1]]:

```
>>> 5 ** 2  # 5 ao quadrado
25
>>> 2 ** 7  # 2 à potência de 7
128
```

O sinal de igual (`'='`) é usado para atribuir um valor a uma variável. Depois de uma atribuição, nenhum resultado é exibido antes do próximo prompt:

```
>>> largura = 20
>>> altura = 5 * 9
>>> largura * altura
900
```

Se uma variável não é “definida” (não tem um valor atribuído), tentar utilizá-la levantará um erro:

```
>>> n  # tenta acessar uma variável não definida
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

Há suporte completo para ponto flutuante (*float*); operadores com operandos de diferentes tipos convertem o inteiro para ponto flutuante:

```
>>> 4 * 3.75 - 1
14.0
```

No modo interativo, o valor da última expressão exibida é atribuída a variável _. Assim, ao utilizar Python como uma calculadora, fica mais fácil prosseguir com os cálculos, por exemplo:

```
>>> taxa = 12.5 / 100
>>> preço = 100.50
>>> preço * taxa
12.5625
>>> preço + _
113.0625
>>> round(_, 2)
113.06
```

Essa variável especial deve ser tratada como *somente para leitura* pelo usuário. Nunca lhe atribua explicitamente um valor — do contrário, estaria criando uma outra variável (homônima) independente, que mascararia a variável especial com seu comportamento mágico.

Além de `int` e `float`, o Python oferece suporte a outros tipos de números, tais como `Decimal` e `Fraction`. O Python também possui suporte embutido a números complexos, e usa os sufixos `j` ou `J` para indicar a parte imaginária (por exemplo, `3+5j`).

3.1.2. Texto

Python pode manipular texto (representado pelo tipo `str`, também chamado de "strings"), bem como números. Isso inclui caracteres "`!`", palavras "`coelho`", nomes "`Paris`", frases "`Eu te protejo.`", etc. "`Oba! :)`". Eles podem ser colocados entre aspas simples (`'...'`) ou aspas duplas (`"..."`) com o mesmo resultado [[2]].

```
>>> 'spam eggs'  # aspas simples
'spam eggs'
>>> "Coelho de Paris, eu te projeto :)! Viva!"  # aspas duplas
'Coelho de Paris, eu te projeto :)! Viva!'
>>> '1975'  # dígitos e numerais envoltos em aspas também são st
'1975'
```

Para colocar aspas entre aspas, precisamos "escapá-la", precedendo-as com `\`. Alternativamente, podemos usar o outro tipo de aspas:

```
>>> 'd\'água'  # use \' para escapar a aspa simples...
"d'água"
>>> "d'água"  # ...ou use aspas duplas
"d'água"
>>> '"Sim", eles disseram.'
'"Sim", eles disseram.'
>>> "\"Sim\", eles disseram."
'"Sim", eles disseram.'
>>> '"Copo d\'água", eles pediram.'
'"Copo d'água", eles pediram.'
```

No shell do Python, a definição de string e a string de saída podem parecer diferentes. A função `print()` produz uma saída mais legível, omitindo as aspas delimitadoras e imprimindo caracteres de escape e especiais:

```
>>> s = 'Primeira linha.\nSegunda linha.'  # \n significa nova l
>>> s  # sem print(), caracteres especiais são incluídos na stri
'Primeira linha.\nSegunda linha.'
>>> print(s)  # com print(), caracteres especiais são interpreta
Primeira linha.
Segunda linha.
```

Caso não queira que os caracteres precedidos por `\` sejam interpretados como caracteres especiais, você pode usar *strings brutas* (*raw*, em inglês) adicionando um `r` antes da primeira aspa:

```
>>> print('C:\this\name')  # aqui \t significa tabulação, \n sig
C:      his
ame
>>> print(r'C:\this\name')  # note o r antes da aspa
C:\this\name
```

Há um aspecto sutil nas strings brutas: uma string bruta não pode terminar em um número ímpar de caracteres `\`; consulte o FAQ relacionado para mais informações e soluções alternativas.

As strings literais podem abranger várias linhas. Uma maneira é usar as aspas triplas: `"""...."""` ou `'''...'''`. Caracteres de fim de linha são incluídos automaticamente na string, mas é possível evitar isso adicionando uma `\` no final. No exemplo a seguir, a nova linha no início não é incluída:

```
>>> print("""\
... Uso: coisinha [OPÇÕES]
...      -h                        Exibe esta mensagem de uso
...      -H hostname               Hostname para se conectar
... """)
Usage: coisinha [OPÇÕES]
     -h                        Exibe esta mensagem de ajuda
     -H hostname               Hostname para se conectar
```

```
>>>
```

Strings podem ser concatenadas (coladas) com o operador +, e repetidas com *:

```
>>> # 3 vezes 'un', seguido por 'ium'
>>> 3 * 'un' + 'ium'
'unununium'
```

Duas ou mais *strings literais* (ou seja, entre aspas) ao lado da outra são automaticamente concatenados.

```
>>> 'Py' 'thon'
'Python'
```

Esse recurso é particularmente útil quando você quer quebrar strings longas:

```
>>> texto = ('Coloque várias strings dentro de parênteses '
...          'para fazer com que elas sejam concatenadas.')
>>> texto
'Coloque várias strings dentro de parênteses para fazer com que
```

Isso só funciona com duas strings literais, não com variáveis ou expressões:

```
>>> prefixo = 'Py'
>>> prefixo 'thon'  # não é possível concatenar uma variável e u
  File "<stdin>", line 1
    prefixi 'thon'
            ^^^^^^
SyntaxError: invalid syntax
>>> ('un' * 3) 'ium'
  File "<stdin>", line 1
    ('un' * 3) 'ium'
               ^^^^^
SyntaxError: invalid syntax
```

Se você quiser concatenar variáveis ou uma variável e uma literal, use +:

```
>>> prefixo + 'thon'
'Python'
```

As strings podem ser *indexadas* (subscritas), com o primeiro caractere como índice 0. Não existe um tipo específico para caracteres; um caractere é simplesmente uma string cujo tamanho é 1:

```
>>> palavra = 'Python'
>>> palavra[0]  # caractere na posição 0
'P'
>>> palavra[5]  # caractere na posição 5
'n'
```

Índices também podem ser números negativos para iniciar a contagem pela direita:

```
>>> palavra[-1]  # último caractere
'n'
>>> palavra[-2]  # penúltimo caractere
'o'
>>> palavra[-6]
'P'
```

Note que dado que -0 é o mesmo que 0, índices negativos começam em -1.

Além da indexação, o *fatiamento* também é permitido. Embora a indexação seja usada para obter caracteres individuais, *fatiar* permite que você obtenha uma substring:

```
>>> palavra[0:2]  # caracteres da posição 0 (incluída) até 2 (ex
'Py'
>>> palavra[2:5]  # caracteres da posição 2 (incluída) até 5 (ex
'tho'
```

Os índices do fatiamento possuem padrões úteis; um primeiro índice omitido padrão é zero, um segundo índice omitido é por padrão o tamanho da string sendo fatiada:

```
>>> palavra[:2]   # caracteres do início até a posição 2 (exclui
'Py'
```

```
>>> palavra[4:]   # caracteres da posição 4 (incluída) até o fim
'on'
>>> palavra[-2:]  # caracteres da penúltima (incluída) até o fim
'on'
```

Observe como o início sempre está incluído, e o fim sempre é excluído. Isso garante que `s[:i] + s[i:]` seja sempre igual a `s`:

```
>>> palavra[:2] + palavra[2:]
'Python'
>>> palavra[:4] + palavra[4:]
'Python'
```

Uma maneira de lembrar como fatias funcionam é pensar que os índices indicam posições *entre* caracteres, onde a borda esquerda do primeiro caractere é 0. Assim, a borda direita do último caractere de uma string de comprimento *n* tem índice *n*, por exemplo:

```
 +---+---+---+---+---+---+
 | P | y | t | h | o | n |
 +---+---+---+---+---+---+
 0   1   2   3   4   5   6
-6  -5  -4  -3  -2  -1
```

A primeira fileira de números indica a posição dos índices 0…6 na string; a segunda fileira indica a posição dos respectivos índices negativos. Uma fatia de *i* a *j* consiste em todos os caracteres entre as bordas *i* e *j*, respectivamente.

Para índices positivos, o comprimento da fatia é a diferença entre os índices, se ambos estão dentro dos limites da string. Por exemplo, o comprimento de `word[1:3]` é 2.

A tentativa de usar um índice que seja muito grande resultará em um erro:

```
>>> palavra[42]  # a palavra só tem 6 caracteres
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
IndexError: string index out of range
```

No entanto, os índices de fatiamento fora do alcance são tratados graciosamente (N.d.T: o termo original “gracefully” indica robustez no tratamento de erros) quando usados para fatiamento. Um índice maior que o comprimento é trocado pelo comprimento, um limite superior menor que o limite inferior produz uma string vazia:

```
>>> palavra[4:42]
'on'
>>> palavra[42:]
''
```

As strings do Python não podem ser alteradas — uma string é imutável. Portanto, atribuir a uma posição indexada na sequência resulta em um erro:

```
>>> palavra[0] = 'J'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
>>> palavra[2:] = 'py'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

Se você precisar de uma string diferente, deverá criar uma nova:

```
>>> 'J' + palavra[1:]
'Jython'
>>> palavra[:2] + 'py'
'Pypy'
```

A função embutida `len()` devolve o comprimento de uma string:

```
>>> s = 'supercalifragilisticexpialidoce'
>>> len(s)
31
```

Ver também

Tipo sequência de texto — str

As strings são exemplos de *tipos de sequências* e suportam as operações comumente suportadas por esses tipos.

String Methods

As strings suportam uma grande quantidade de métodos para transformações básicas e busca.

Literais de strings formatadas

Strings literais que possuem expressões embutidas.

Sintaxe das strings de formato

Informações sobre formatação de string com o método `str.format()`.

Formatação de string no estilo printf

As antigas operações de formatação invocadas quando as strings são o operando esquerdo do operador `%` são descritas com mais detalhes aqui.

3.1.3. Listas

Python inclui diversas estruturas de dados *compostas*, usadas para agrupar outros valores. A mais versátil é *list* (lista), que pode ser escrita como uma lista de valores (itens) separados por vírgula, entre colchetes. Os valores contidos na lista não precisam ser todos do mesmo tipo.

```
>>> quadrados = [1, 4, 9, 16, 25]
>>> quadrados
[1, 4, 9, 16, 25]
```

Como strings (e todos os tipos embutidos de sequência), listas pode ser indexados e fatiados:

```
>>> quadrados[0]  # indexação retorna o item
1
>>> quadrados[-1]
25
>>> quadrados[-3:]  # fatiamento retorna uma nova lista
[9, 16, 25]
```

As listas também suportam operações como concatenação:

```
>>> quadrados + [36, 49, 64, 81, 100]
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100]
```

Diferentemente de strings, que são imutáveis, listas são mutáveis, ou seja, é possível alterar elementos individuais de uma lista:

```
>>> cubos = [1, 8, 27, 65, 125]  # algo errado aqui
>>> 4 ** 3  # o cubo de 4 é 64, não 65!
64
>>> cubos[3] = 64  # substitui o valor errado
>>> cubos
[1, 8, 27, 64, 125]
```

Você também pode adicionar novos itens no final da lista, usando o *método* **`list.append()`** (estudaremos mais a respeito dos métodos posteriormente):

```
>>> cubos.append(216)  # adiciona o cubo de 6
>>> cubos.append(7 ** 3)  # e o cubo de 7
>>> cubos
[1, 8, 27, 64, 125, 216, 343]
```

A atribuição simples em Python nunca copia dados. Quando você atribui uma lista a uma variável, a variável se refere à *lista existente*. Quaisquer alterações que você fizer na lista por meio de uma variável serão vistas por todas as outras variáveis que se referem a ela:

```
>>> rgb = ["Vermelho", "Verde", "Azul"]
>>> rgba = rgb
>>> id(rgb) == id(rgba)  # elas referenciam o mesmo objeto
True
>>> rgba.append("Alf")
>>> rgb
["Vermelho", "Verde", "Azul", "Alf"]
```

Todas as operações de fatiamento devolvem uma nova lista contendo os elementos solicitados. Isso significa que o seguinte fatiamento devolve uma cópia rasa da lista:

```
>>> rgba_correto = rgba[:]
>>> rgba_correto[-1] = "Alfa"
>>> rgba_correto
```

```
["Vermelho", "Verde", "Azul", "Alfa"]
>>> rgba
["Vermelho", "Verde", "Azul", "Alf"]
```

Atribuição a fatias também é possível, e isso pode até alterar o tamanho da lista ou remover todos os itens dela:

```
>>> letras = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letras
['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> # substitui alguns valores
>>> letras[2:5] = ['C', 'D', 'E']
>>> letras
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> # agora remove-os
>>> letras[2:5] = []
>>> letras
['a', 'b', 'f', 'g']
>>> # limpa a lista substituindo todos os elementos por uma list
>>> letras[:] = []
>>> letras
[]
```

A função embutida `len()` também se aplica a listas:

```
>>> letras = ['a', 'b', 'c', 'd']
>>> len(letras)
4
```

É possível aninhar listas (criar listas contendo outras listas), por exemplo:

```
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> x = [a, n]
>>> x
[['a', 'b', 'c'], [1, 2, 3]]
>>> x[0]
['a', 'b', 'c']
>>> x[0][1]
'b'
```

3.2. Primeiros passos para a programação

Claro, podemos usar o Python para tarefas mais complicadas do que somar 2+2. Por exemplo, podemos escrever o início da sequência de Fibonacci [https://pt.wikipedia.org/wiki/Sequência_de_Fibonacci] assim:

```
>>> # Sequência de Fibonacci:
>>> # a soma de dois elementos define a próxima
>>> a, b = 0, 1
>>> while a < 10:
...     print(a)
...     a, b = b, a+b
...
0
1
1
2
3
5
8
```

Este exemplo introduz diversas características ainda não mencionadas.

- A primeira linha contém uma atribuição múltipla: as variáveis `a` e `b` recebem simultaneamente os novos valores 0 e 1. Na última linha há outro exemplo de atribuição múltipla demonstrando que expressões do lado direito são sempre avaliadas primeiro, antes da atribuição. As expressões do lado direito são avaliadas da esquerda para a direita.

- O laço de repetição **`while`** executa enquanto a condição (aqui: `a < 10`) permanece verdadeira. Em Python, como em C, qualquer valor inteiro que não seja zero é considerado verdadeiro; zero é considerado falso. A condição pode também ser uma cadeia de caracteres ou uma lista, ou qualquer sequência; qualquer coisa com um tamanho maior que zero é verdadeiro, enquanto sequências vazias são falsas. O teste usado no exemplo é uma comparação simples. Os operadores padrões de comparação são os mesmos de C: `<` (menor que), `>` (maior que), `==` (igual), `<=` (menor ou igual), `>=` (maior ou igual) e `!=` (diferente).

- O *corpo* do laço é *indentado*: indentação em Python é a maneira de agrupar comandos em blocos. No console interativo padrão você terá que digitar tab ou espaços para indentar cada linha. Na prática você vai preparar scripts Python mais complicados em um editor de texto; a maioria dos editores de texto tem facilidades de indentação automática. Quando um comando composto é digitado interativamente, deve ser finalizado por uma linha em branco (já que o interpretador não tem como adivinhar qual é a última linha do comando). Observe que toda linha de um mesmo bloco de comandos deve ter a mesma indentação.

- A função `print()` escreve o valor dos argumentos fornecidos. É diferente de apenas escrever a expressão no interpretador (como fizemos anteriormente nos exemplos da calculadora) pela forma como lida com múltiplos argumentos, quantidades de ponto flutuante e strings. As strings são impressas sem aspas, e um espaço é inserido entre os itens, assim você pode formatar bem o resultado, dessa forma:

  ```
  >>> i = 256*256
  >>> print('O valor de i é', i)
  O valor de i é 65536
  ```

  O argumento nomeado *end* pode ser usado para evitar uma nova linha após a saída ou finalizar a saída com uma string diferente:

  ```
  >>> a, b = 0, 1
  >>> while a < 1000:
  ...     print(a, end=',')
  ...     a, b = b, a+b
  ...
  0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
  ```

Notas de rodapé

[1]

Uma vez que `**` tem precedência mais alta que `-`, `-3**2` será interpretado como `-(3**2)` e assim resultará em `-9`. Para evitar isso e obter `9`, você pode usar `(-3)**2`.

[2]

Ao contrário de outras linguagens, caracteres especiais como `\n` têm o mesmo significado com as aspas simples (`'...'`) e duplas (`"..."`) . A única diferença entre as duas é que, dentro de aspas simples, você não precisa escapar o `"` (mas você deve escapar a `\'`) e vice-versa.

4. Mais ferramentas de controle de fluxo

Assim como a instrução **`while`** que acabou de ser apresentada, o Python usa mais algumas que encontraremos neste capítulo.

4.1. Instruções `if`

Provavelmente a mais conhecida instrução de controle de fluxo é o **`if`**. Por exemplo:

```
>>> x = int(input("Insira um número inteiro: "))
Insira um número inteiro: 42
>>> if x < 0:
...     x = 0
...     print('Negativo alterado para zero')
... elif x == 0:
...     print('Zero')
... elif x == 1:
...     print('Um')
... else:
...     print('Mais')
...
Mais
```

Pode haver zero ou mais partes **`elif`**, e a parte **`else`** é opcional. A palavra-chave '**`elif`**' é uma abreviação para 'else if', e é útil para evitar indentação excessiva. Uma sequência **`if`** … **`elif`** … **`elif`** … substitui as instruções `switch` ou `case`, encontrados em outras linguagens.

Se você está comparando o mesmo valor com várias constantes, ou verificando por tipos ou atributos específicos, você também pode achar a instrução **`match`** útil. Para mais detalhes veja Instruções match.

4.2. Instruções `for`

A instrução `for` em Python é um pouco diferente do que costuma ser em C ou Pascal. Ao invés de sempre iterar sobre uma progressão aritmética de números (como no Pascal), ou permitir ao usuário definir o passo de iteração e a condição de parada (como C), a instrução **`for`** do Python itera sobre os itens de qualquer sequência (seja uma lista ou uma string), na ordem que aparecem na sequência. Por exemplo:

```
>>> # Mede algumas strings:
>>> palavras = ['gato', 'janela', 'defenestrar']
>>> for p in palavras:
...     print(p, len(p))
...
gato 4
janela 6
defenestrar 11
```

Código que modifica uma coleção sobre a qual está iterando pode ser inseguro. No lugar disso, usualmente você deve iterar sobre uma cópia da coleção ou criar uma nova coleção:

```
# Cria uma amostra de coleção
users = {'Hans': 'active', 'Éléonore': 'inactive', '景太郎': 'act

# Estratégia: iterar por uma cópia
for user, status in users.copy().items():
    if status == 'inactive':
        del users[user]

# Estratégia: criar uma nova coleção
active_users = {}
for user, status in users.items():
    if status == 'active':
        active_users[user] = status
```

4.3. A função `range()`

Se você precisa iterar sobre sequências numéricas, a função embutida `range()` é a resposta. Ela gera progressões aritméticas:

```
>>> for i in range(5):
...     print(i)
...
0
1
2
3
4
```

O ponto de parada fornecido nunca é incluído na lista; `range(10)` gera uma lista com 10 valores, exatamente os índices válidos para uma sequência de comprimento 10. É possível iniciar o intervalo com outro número, ou alterar a razão da progressão (inclusive com passo negativo):

```
>>> list(range(5, 10))
[5, 6, 7, 8, 9]

>>> list(range(0, 10, 3))
[0, 3, 6, 9]

>>> list(range(-10, -100, -30))
[-10, -40, -70]
```

Para iterar sobre os índices de uma sequência, combine **`range()`** e **`len()`** da seguinte forma:

```
>>> a = ['Maria', 'tinha', 'um', 'carneirinho']
>>> for i in range(len(a)):
...     print(i, a[i])
...
0 Maria
1 tinha
2 um
3 carneirinho
```

Na maioria dos casos, porém, é mais conveniente usar a função **`enumerate()`**, veja Técnicas de iteração.

Uma coisa estranha acontece se você imprime um intervalo:

```
>>> range(10)
range(0, 10)
```

Em muitos aspectos, o objeto retornado pela função **range()** se comporta como se fosse uma lista, mas na verdade não é. É um objeto que retorna os itens sucessivos da sequência desejada quando você itera sobre a mesma, mas na verdade ele não gera a lista, economizando espaço.

Dizemos que um objeto é iterável, isso é, candidato a ser alvo de uma função ou construção que espera alguma coisa capaz de retornar sucessivamente seus elementos um de cada vez até ser esgotado. Nós vimos que a instrução **for** é um exemplo de construção, enquanto que um exemplo de função que recebe um iterável é **sum()**:

```
>>> sum(range(4))  # 0 + 1 + 2 + 3
6
```

Mais tarde, veremos mais funções que retornam iteráveis e tomam iteráveis como argumentos. No capítulo Estruturas de dados, iremos discutir **list()** em mais detalhes.

4.4. Instruções `break` e `continue`

A instrução **break** sai imediatamente do laço de repetição mais interno, seja **for** ou **while**:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(f"{n} igual a {x} * {n//x}")
...             break
...
4 igual a 2 * 2
6 igual a 2 * 3
8 igual a 2 * 4
9 equals 3 * 3
```

A instrução **continue** continua com a próxima iteração do laço:

```
>>> for num in range(2, 10):
...     if num % 2 == 0:
...         print(f"Encontrado um número par {num}")
...         continue
...     print(f"Encontrado um número ímpar {num}")
...
Encontrado um número par 2
Encontrado um número ímpar 3
Encontrado um número par 4
Encontrado um número ímpar 5
Encontrado um número par 6
Encontrado um número ímpar 7
Encontrado um número par 8
Encontrado um número ímpar 9
```

4.5. Cláusulas `else` em laços

Em um laço **`for`** ou **`while`** a instrução **`break`** pode ser pareada com uma cláusula **`else`**. Se o laço terminar sem executar o **`break`**, a cláusula **`else`** será executada.

Em um laço **`for`**, a cláusula **`else`** é executada após o laço finalizar sua iteração final, ou seja, se não ocorrer nenhuma interrupção.

Em um laço **`while`**, ele é executado após a condição do laço se tornar falsa.

Em qualquer tipo de laço, a cláusula **`else`** **não** é executada se o laço foi encerrado por um **`break`**. Claro, outras maneiras de encerrar o laço mais cedo, como um **`return`** ou uma exceção levantada, também pularão a execução da cláusula **`else`**.

Isso é exemplificado no seguinte laço **`for`**, que procura por números primos:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(n, 'igual a', x, '*', n//x)
...             break
...     else:
```

```
...         # a iteração passou direto sem encontrar um fator
...         print(n, 'é um número primo')
...
2 é um número primo
3 é um número primo
4 igual a 2 * 2
5 é um número primo
6 igual a 2 * 3
7 é um número primo
8 igual a 2 * 4
9 igual a 3 * 3
```

(Sim, este é o código correto. Observe atentamente: a cláusula `else` pertence ao laço `for`, **não** à instrução `if`.)

Uma maneira de pensar na cláusula else é imaginá-la pareada com o `if` dentro do laço. Conforme o laço é executado, ele executará uma sequência como if/if/if/else. O `if` está dentro do laço, encontrado várias vezes. Se a condição for verdadeira, um `break` acontecerá. Se a condição nunca for verdadeira, a cláusula `else` fora do laço será executada.

Quando usado em um laço, a cláusula `else` tem mais em comum com a cláusula `else` de uma instrução **`try`** do que com a de instruções `if`: a cláusula `else` de uma instrução `try` é executada quando não ocorre exceção, e a cláusula `else` de um laço é executada quando não ocorre um `break`. Para mais informações sobre instrução `try` e exceções, veja Tratamento de exceções.

4.6. Instruções `pass`

A instrução **`pass`** não faz nada. Pode ser usada quando a sintaxe exige uma instrução, mas a semântica do programa não requer nenhuma ação. Por exemplo:

```
>>> while True:
...     pass  # Ocupado, aguardando interrupção por teclado (Ctr
...
```

Isto é usado muitas vezes para se definir classes mínimas:

```
>>> class MinhaClasseVazia:
...     pass
...
```

Outra ocasião em que o **pass** pode ser usado é como um substituto temporário para uma função ou bloco condicional, quando se está trabalhando com código novo, ainda indefinido, permitindo que mantenha-se o pensamento num nível mais abstrato. O **pass** é silenciosamente ignorado:

```
>>> def initlog(*args):
...     pass   # Lembre-se de implementar isso!
...
```

Para este último caso, muitas pessoas usam o literal de reticências `...` em vez de `pass`. Este uso não tem significado especial para Python e não faz parte da definição da linguagem (você pode usar qualquer expressão constante aqui), mas `...` também é usado convencionalmente como um espaço reservado. Consulte O Objeto Ellipsis.

4.7. Instruções `match`

Uma instrução **match** pega uma expressão e compara seu valor com padrões sucessivos fornecidos como um ou mais blocos de case. Isso é superficialmente semelhante a uma instrução switch em C, Java ou JavaScript (e muitas outras linguagens), mas muito mais parecido com a correspondência de padrões em linguages como Rust ou Haskell. Apenas o primeiro padrão que corresponder será executado, podendo também extrair componentes (elementos de sequência ou atributos de objetos) do valor para variáveis. Se nenhum caso corresponder, nenhuma das ramificações será executada.

A forma mais simples compara um valor de assunto com um ou mais literais:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 418:
            return "I'm a teapot"
        case _:
            return "Something's wrong with the internet"
```

Observe o último bloco: o “nome da variável” _ atua como um *curinga* e nunca falha em corresponder.

Você pode combinar vários literais em um único padrão usando `|` (“ou”):

```
case 401 | 403 | 404:
    return "Não permitido"
```

Os padrões podem se parecer com atribuições de desempacotamento e podem ser usados para vincular variáveis:

```
# ponto é uma tupla (x, y)
match ponto:
    case (0, 0):
        print("Origem")
    case (0, y):
        print(f"Y={y}")
    case (x, 0):
        print(f"X={x}")
    case (x, y):
        print(f"X={x}, Y={y}")
    case _:
        raise ValueError("Não é um ponto")
```

Estude isso com cuidado! O primeiro padrão tem dois literais e pode ser considerado uma extensão do padrão literal mostrado acima. Mas os próximos dois padrões combinam um literal e uma variável, e a variável *vincula* um valor do assunto (`ponto`). O quarto padrão captura dois valores, o que o torna conceitualmente semelhante à atribuição de desempacotamento `(x, y) = ponto`.

Se você estiver usando classes para estruturar seus dados, você pode usar o nome da classe seguido por uma lista de argumentos semelhante a um construtor, mas com a capacidade de capturar atributos em variáveis:

```
class Ponto:
    def __init__(self, x, y):
        self.x = x
        self.y = y

def onde_está(ponto):
    match ponto:
        case Ponto(x=0, y=0):
            print("Origem")
        case Ponto(x=0, y=y):
            print(f"Y={y}")
        case Ponto(x=x, y=0):
            print(f"X={x}")
        case Ponto():
            print("Em outro lugar")
        case _:
            print("Não é um ponto")
```

Você pode usar parâmetros posicionais com algumas classes embutidas que fornecem uma ordem para seus atributos (por exemplo, classes de dados). Você também pode definir uma posição específica para atributos em padrões configurando o atributo especial `__match_args__` em suas classes. Se for definido como (“x”, “y”), os seguintes padrões são todos equivalentes (e todos vinculam o atributo `y` à variável `var`):

```
Ponto(1, var)
Ponto(1, y=var)
Ponto(x=1, y=var)
Ponto(y=var, x=1)
```

Uma maneira recomendada de ler padrões é vê-los como uma forma estendida do que você colocaria à esquerda de uma atribuição, para entender quais variáveis seriam definidas para quê. Apenas os nomes autônomos (como `var` acima) são atribuídos por uma instrução de correspondência. Nomes pontilhados (como `foo.bar`), nomes de atributos

(o `x=` e `y=` acima) ou nomes de classes (reconhecidos pelo “(…)” próximo a eles, como `Ponto` acima) nunca são atribuídos.

Os padrões podem ser aninhados arbitrariamente. Por exemplo, se tivermos uma pequena lista de Pontos, com `__match_args__` adicionado, poderíamos correspondê-la assim:

```
class Ponto:
    __match_args__ = ('x', 'y')
    def __init__(self, x, y):
        self.x = x
        self.y = y

match pontos:
    case []:
        print("Sem pontos")
    case [Ponto(0, 0)]:
        print("A origem")
    case [Ponto(x, y)]:
        print(f"Ponto único {x}, {y}")
    case [Ponto(0, y1), Ponto(0, y2)]:
        print(f"Dois do eixo Y em {y1}, {y2}")
    case _:
        print("Outra coisa")
```

Podemos adicionar uma cláusula `if` a um padrão, conhecido como “guarda”. Se a guarda for falsa, `match` continua para tentar o próximo bloco de caso. Observe que a captura de valor ocorre antes que a guarda seja avaliada:

```
match ponto:
    case Ponto(x, y) if x == y:
        print(f"Y=X at {x}")
    case Ponto(x, y):
        print(f"Não está na diagonal")
```

Vários outros recursos importantes desta instrução:

- Assim como desempacotar atribuições, os padrões de tupla e lista têm exatamente o mesmo significado e realmente correspondem a

sequências arbitrárias. Uma exceção importante é que eles não correspondem a iteradores ou strings.

- Os padrões de sequência têm suporte ao desempacotamento estendido: `[x, y, *rest]` e `(x, y, *rest)` funcionam de forma semelhante ao desempacotamento de atribuições. O nome depois de `*` também pode ser `_`, então `(x, y, *_)` corresponde a uma sequência de pelo menos dois itens sem ligar os itens restantes.

- Padrões de mapeamento: `{"bandwidth": b, "latency": l}` captura os valores `"bandwidth"` e `"latency"` de um dicionário. Diferente dos padrões de sequência, chaves extra são ignoradas. Um desempacotamento como `**rest` também é permitido. (Mas `**_` seria redundante, então não é permitido.)

- Subpadrões podem ser capturados usando a palavra reservada `as`:

  ```
  case (Ponto(x1, y1), Ponto(x2, y2) as p2): ...
  ```

  Vai capturar o segundo elemento da entrada como `p2` (se a entrada for uma sequência de dois pontos)

- A maioria dos literais são comparados por igualdade, no entanto os singletons `True`, `False` e `None` são comparados por identidade.

- Padrões podem usar constantes nomeadas. Estas devem ser nomes pontilhados para prevenir que sejam interpretadas como variáveis de captura:

  ```
  from enum import Enum
  class Cor(Enum):
      VERMELHO = 'vermelho'
      VERDE = 'verde'
      AZUL = 'azul'

  cor = Cor(input("Insira sua escolha de 'vermelho, 'azul' ou

  match color:
      case Cor.VERMELHO:
          print("Eu vejo vermelho!")
  ```

```
        case Cor.VERDE:
            print("Grama é verde")
        case Cor.AZUL:
            print("O céu é azul :)")
```

Para uma explicação mais detalhada e exemplos adicionais, você pode olhar **PEP 636** [https://peps.python.org/pep-0636/] que foi escrita em formato de tutorial.

4.8. Definindo funções

Podemos criar uma função que escreve a série de Fibonacci até um limite arbitrário:

```
>>> def fib(n):    # escreve série de Fibonacci menor que n
...     """Imprime uma série de Fibonacci menor que n."""
...     a, b = 0, 1
...     while a < n:
...         print(a, end=' ')
...         a, b = b, a+b
...     print()
...
>>> # Agora chamamos a função que acabamos de definir:
>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597
```

A palavra reservada **`def`** inicia a *definição* de uma função. Ela deve ser seguida do nome da função e da lista de parâmetros formais entre parênteses. As instruções que formam o corpo da função começam na linha seguinte e devem ser indentados.

Opcionalmente, a primeira linha do corpo da função pode ser uma literal string, cujo propósito é documentar a função. Se presente, essa string chama-se *docstring*. (Há mais informação sobre docstrings na seção Strings de documentação.) Existem ferramentas que utilizam docstrings para produzir automaticamente documentação online ou para imprimir, ou ainda, permitir que o usuário navegue interativamente pelo código. É uma boa prática incluir sempre docstrings em suas funções, portanto, tente fazer disso um hábito.

A *execução* de uma função cria uma nova tabela de símbolos para as variáveis locais da função. Mais precisamente, todas as atribuições de variáveis numa função são armazenadas na tabela de símbolos local; referências a variáveis são buscadas primeiro na tabela de símbolos local, em seguida na tabela de símbolos locais de funções delimitadoras ou circundantes, depois na tabela de símbolos global e, finalmente, na tabela de nomes da própria linguagem. Embora possam ser referenciadas, variáveis globais e de funções externas não podem ter atribuições (a menos que seja utilizada a instrução **`global`**, para variáveis globais, ou **`nonlocal`**, para variáveis de funções externas).

Os parâmetros reais (argumentos) de uma chamada de função são introduzidos na tabela de símbolos local da função no momento da chamada; portanto, argumentos são passados *por valor* (onde o *valor* é sempre uma *referência* para objeto, não o valor do objeto). [[1]] Quando uma função chama outra função, ou chama a si mesma recursivamente, uma nova tabela de símbolos é criada para tal chamada.

Uma definição de função associa o nome da função com o objeto função na tabela de símbolos atual. O interpretador reconhece o objeto apontado pelo nome como uma função definida pelo usuário. Outros nomes também podem apontar para o mesmo objeto função e também pode ser usados pra acessar a função:

```
>>> fib
<function fib at 10042ed0>
>>> f = fib
>>> f(100)
0 1 1 2 3 5 8 13 21 34 55 89
```

Conhecendo outras linguagens, pode-se questionar que `fib` não é uma função, mas um procedimento, pois ela não devolve um valor. Na verdade, mesmo funções que não usam a instrução **`return`** devolvem um valor, ainda que pouco interessante. Esse valor é chamado `None` (é um nome embutido). O interpretador interativo evita escrever `None` quando ele é o único resultado de uma expressão. Mas se quiser vê-lo pode usar a função **`print()`**:

```
>>> fib(0)
>>> print(fib(0))
None
```

É fácil escrever uma função que retorna uma lista de números da série de Fibonacci, ao invés de exibi-los:

```
>>> def fib2(n):  # retorna série de  Fibonacci até n
...     """Retorna uma lista contendo a série de Fibonacci até r
...     resultado = []
...     a, b = 0, 1
...     while a < n:
...         resultado.append(a)    # veja abaixo
...         a, b = b, a+b
...     return resultado
...
>>> f100 = fib2(100)    # chama-o
>>> f100                # escreve o resultado
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Este exemplo demonstra novos recursos de Python:

- A instrução **`return`** finaliza a execução e retorna um valor da função. **`return`** sem qualquer expressão como argumento retorna `None`. Atingir o final da função também retorna `None`.
- A instrução `result.append(a)` chama um *método* do objeto lista `result`. Um método é uma função que 'pertence' a um objeto, e é chamada `obj.nomemetodo`, onde `obj` é um objeto qualquer (pode ser uma expressão), e `nomemetodo` é o nome de um método que foi definido pelo tipo do objeto. Tipos diferentes definem métodos diferentes. Métodos de diferentes tipos podem ter o mesmo nome sem ambiguidade. (É possível definir seus próprios tipos de objetos e métodos, utilizando *classes*, veja em Classes) O método **`append()`**, mostrado no exemplo é definido para objetos do tipo lista; adiciona um novo elemento ao final da lista. Neste exemplo, ele equivale a `result = result + [a]`, só que mais eficiente.

4.9. Mais sobre definição de funções

É possível definir funções com um número variável de argumentos. Existem três formas, que podem ser combinadas.

4.9.1. Argumentos com valor padrão

A mais útil das três é especificar um valor padrão para um ou mais argumentos. Isso cria uma função que pode ser invocada com menos argumentos do que os que foram definidos. Por exemplo:

```
def pergunta_ok(mensagem, tentativas=4, lembrete='Por favor, ter
    while True:
        resposta = input(mensagem)
        if resposta in {'s', 'sim', 'é'}:
            return True
        if resposta in {'n', 'não', 'nah'}:
            return False
        tentativas = tentativas - 1
        if tentativas < 0:
            raise ValueError('resposta inválida de usuário')
        print(lembrete)
```

Essa função pode ser chamada de várias formas:

- fornecendo apenas o argumento obrigatório: `ask_ok('Do you really want to quit?')`
- fornecendo um dos argumentos opcionais: `ask_ok('OK to overwrite the file?', 2)`
- ou fornecendo todos os argumentos: `ask_ok('OK to overwrite the file?', 2, 'Come on, only yes or no!')`

Este exemplo também introduz a palavra-chave `in`, que verifica se uma sequência contém ou não um determinado valor.

Os valores padrões são avaliados no momento da definição da função, e no escopo em que a função foi *definida*, portanto:

```
i = 5

def f(arg=i):
```

```
    print(arg)

i = 6
f()
```

irá exibir `5`.

Aviso importante: Valores padrões são avaliados apenas uma vez. Isso faz diferença quando o valor é um objeto mutável, como uma lista, dicionário, ou instâncias de classes. Por exemplo, a função a seguir acumula os argumentos passados, nas chamadas subsequentes:

```
def f(a, L=[]):
    L.append(a)
    return L

print(f(1))
print(f(2))
print(f(3))
```

Isso exibirá:

```
[1]
[1, 2]
[1, 2, 3]
```

Se não quiser que o valor padrão seja compartilhado entre chamadas subsequentes, pode reescrever a função assim:

```
def f(a, L=None):
    if L is None:
        L = []
    L.append(a)
    return L
```

4.9.2. Argumentos nomeados

Funções também podem ser chamadas usando argumentos nomeados da forma `chave=valor`. Por exemplo, a função a seguir:

```
def papagaio(voltagem, estado='é um cadáver', ação='voar', tipo=
    print("-- Este papagaio não conseguiria", ação, end=' ')
    print("nem se você desse um choque de", voltagem, "de volts
    print("-- Plumagem formosa, o", tipo)
    print("-- Ele", estado, "!")
```

aceita um argumento obrigatório (`voltagem`) e três argumentos opcionais (`estado`, `ação`, e `tipo`). Esta função pode ser chamada de qualquer uma dessas formas:

```
papagaio(1000)                                          # 1 argume
papagaio(voltagem=1000)                                 # 1 argume
papagaio(voltagem=1000000, ação='fazer VOOOOOM')        # 2 argume
papagaio(ação='fazer VOOOOOM', voltagem=1000000)        # 2 argume
papagaio('um milhão', 'sem vida', 'pular')              # 3 argume
papagaio('mil', estado='estaria no céu')                # 1 posici
```

mas todas as formas a seguir seriam inválidas:

```
papagaio()                        # faltando argumento obrigatóri
papagaio(voltagem=5.0, 'morto')   # argumento não nomeado após um
papagaio(110, voltagem=220)       # valor duplicado para o mesmo
papagaio(ator='John Cleese')      # argumento nomeado desconhecid
```

Em uma chamada de função, argumentos nomeados devem vir depois dos argumentos posicionais. Todos os argumentos nomeados passados devem corresponder com argumentos aceitos pela função (ex. `ator` não é um argumento nomeado válido para a função `papagaio`), mas sua ordem é irrelevante. Isto também inclui argumentos obrigatórios (ex.: `papagaio(voltagem=1000)` funciona). Nenhum argumento pode receber mais de um valor. Eis um exemplo que não funciona devido a esta restrição:

```
>>> def função(a):
...     pass
...
>>> função(0, a=0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: função() got multiple values for argument 'a'
```

Quando um último parâmetro formal no formato `**nome` está presente, ele recebe um dicionário (veja Tipos mapeamentos — dict, frozendict) contendo todos os argumentos nomeados, exceto aqueles que correspondem a um parâmetro formal. Isto pode ser combinado com um parâmetro formal no formato `*nome` (descrito na próxima subseção) que recebe uma tupla contendo os argumentos posicionais, além da lista de parâmetros formais. (`*nome` deve ocorrer antes de `**nome`.) Por exemplo, se definirmos uma função assim:

```
def loja_de_queijos(tipo, *argumentos, **argumentos_nomeados):
    print("-- Você tem algum", tipo, "?")
    print("-- Lamento, acabou o", tipo)
    for arg in argumentos:
        print(arg)
    print("-" * 40)
    for kw in argumentos_nomeados:
        print(kw, ":", argumentos_nomeados[kw])
```

Pode ser chamada assim:

```
loja_de_queijos("Limburger", "Está muito mole, senhor",
           "Está realmente muito, MUITO mole, senhor.",
           vendedor="Michael Palin",
           cliente="John Cleese",
           sketch="Sketch da Loja de Queijos")
```

e, claro, exibiria:

```
-- Você tem algum Limburger ?
-- Lamento, acabou o Limburger
Está muito mole, senhor.
Está realmente muito, MUITO mole, senhor.
----------------------------------------
vendedor : Michael Palin
cliente : John Cleese
sketch : Cheese Shop Sketch
```

Observe que a ordem em que os argumentos nomeados são exibidos é garantida para corresponder à ordem em que foram fornecidos na chamada da função.

4.9.3. Parâmetros especiais

Por padrão, argumentos podem ser passadas para uma função Python tanto por posição quanto explicitamente pelo nome. Para uma melhor legibilidade e desempenho, faz sentido restringir a maneira pelo qual argumentos possam ser passados, assim um desenvolvedor precisa apenas olhar para a definição da função para determinar se os itens são passados por posição, por posição e nome, ou por nome.

A definição de uma função pode parecer com:

```
def f(pos1, pos2, /, pos_ou_kwd, *, kwd1, kwd2):
      -----------    ----------     ----------
        |             |                  |
        |        Posicional ou nomeado   |
        |                                - Somente nomeado
         -- Somente posicional
```

onde `/` e `*` são opcionais. Se usados, esses símbolos indicam o tipo de parâmetro pelo qual os argumentos podem ser passados para as funções: somente-posicional, posicional-ou-nomeado, e somente-nomeado. Parâmetros nomeados são também conhecidos como parâmetros palavra-chave.

4.9.3.1. Argumentos posicional-ou-nomeados

Se `/` e `*` não estão presentes na definição da função, argumentos podem ser passados para uma função por posição ou por nome.

4.9.3.2. Parâmetros somente-posicionais

Olhando com um pouco mais de detalhes, é possível definir certos parâmetros como *somente-posicional*. Se *somente-posicional*, a ordem do parâmetro importa, e os parâmetros não podem ser passados por nome. Parâmetros somente-posicional são colocados antes de `/` (barra). A `/` é usada para logicamente separar os argumentos somente-posicional dos

demais parâmetros. Se não existe uma `/` na definição da função, não existe parâmetros somente-posicionais.

Parâmetros após a `/` podem ser *posicional-ou-nomeado* ou *somente-nomeado*.

4.9.3.3. Argumentos somente-nomeados

Para definir parâmetros como *somente-nomeado*, indicando que o parâmetro deve ser passado por argumento nomeado, colocamos um `*` na lista de argumentos imediatamente antes do primeiro parâmetro *somente-nomeado*.

4.9.3.4. Exemplos de funções

Considere o seguinte exemplo de definição de função com atenção redobrada para os marcadores `/` e `*`:

```
>>> def arg_padrão(arg):
...     print(arg)
...
>>> def arg_somente_pos(arg, /):
...     print(arg)
...
>>> def arg_somente_nom(*, arg):
...     print(arg)
...
>>> def exemplo_combinado(somente_pos, /, padrão, *, somente_nom
...     print(somente_pos, padrão, somente_nom)
```

A definição da primeira função, `arg_padrão`, a forma mais familiar, não coloca nenhuma restrição para a chamada da função e argumentos podem ser passados por posição ou nome:

```
>>> arg_padrão(2)
2
```

```
>>> arg_padrão(arg=2)
2
```

A segunda função `arg_somente_pos` está restrita ao uso de parâmetros somente posicionais, uma vez que existe uma `/` na definição da função:

```
>>> arg_somente_pos(1)
1

>>> arg_somente_pos(arg=1)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: arg_somente_pos() got some positional-only arguments
```

A terceira função `arg_somente_nom` permite somente argumentos nomeados como indicado pelo `*` na definição da função:

```
>>> arg_somente_nom(3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: arg_somente_nom() takes 0 positional arguments but 1

>>> arg_somente_nom(arg=3)
3
```

E a última usa as três convenções de chamada na mesma definição de função:

```
>>> exemplo_combinado(1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: exemplo_combinado() takes 2 positional arguments but

>>> exemplo_combinado(1, 2, somente_nom=3)
1 2 3

>>> exemplo_combinado(1, padrão=2, somente_nom=3)
1 2 3

>>> exemplo_combinado(somente_pos=1, padrão=2, somente_nom=3)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: exemplo_combinado() got some positional-only argument
```

Finalmente, considere essa definição de função que possui uma potencial colisão entre o argumento posicional `nome` e `**kwds` que possui `nome` como uma chave:

```
def foo(nome, **kwds):
    return 'nome' in kwds
```

Não é possível essa chamada devolver `True`, uma vez que o argumento nomeado `'nome'` sempre será aplicado para o primeiro parâmetro. Por exemplo:

```
>>> foo(1, **{'nome': 2})
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: foo() got multiple values for argument 'nome'
>>>
```

Mas usando `/` (argumentos somente-posicionais), isso é possível já que permite `nome` como um argumento posicional e `'nome'` como uma chave nos argumentos nomeados:

```
>>> def foo(nome, /, **kwds):
...     return 'nome' in kwds
...
>>> foo(1, **{'nome': 2})
True
```

Em outras palavras, o nome de parâmetros somente-posicional podem ser usados em `**kwds` sem ambiguidade.

4.9.3.5. Recapitulando

A situação irá determinar quais parâmetros usar na definição da função:

```
def f(pos1, pos2, /, pos_ou_nom *, nom1, nom2):
```

Como guia:

- Use somente-posicional se você não quer que o nome do parâmetro esteja disponível para o usuário. Isso é útil quando nomes de parâmetros não tem um significado real, se você quer forçar a ordem dos argumentos da função quando ela é chamada ou se você precisa ter alguns parâmetros posicionais e alguns nomeados.
- Use somente-nomeado quando os nomes tem significado e a definição da função fica mais clara deixando esses nomes explícitos ou se você quer evitar que usuários confiem na posição dos argumentos que estão sendo passados.
- Para uma API, use somente-posicional para evitar quebras na mudança da API se os nomes dos parâmetros forem alterados no futuro.

4.9.4. Listas de argumentos arbitrárias

Finalmente, a opção menos usada é especificar que a função pode ser chamada com um número arbitrário de argumentos. Esses argumentos serão empacotados em uma tupla (ver Tuplas e Sequências). Antes dos argumentos em número variável, zero ou mais argumentos normais podem estar presentes.

```
def escreve_vários_itens(arquivo, separador, *args):
    file.write(separador.join(args))
```

Normalmente, esses argumentos *variádicos* estarão no final da lista de parâmetros formais, porque eles englobam todos os argumentos de entrada restantes, que são passados para a função. Quaisquer parâmetros formais que ocorrem após o parâmetro `*args` são argumentos ‘somente-nomeados’ , o que significa que eles só podem ser usados como chave-valor, em vez de argumentos posicionais:

```
>>> def concat(*args, sep="/"):
...     return sep.join(args)
...
>>> concat("terra", "marte", "vênus")
'terra/marte/vênus'
>>> concat("terra", "marte", "vênus", sep=".")
'terra.marte.vênus'
```

4.9.5. Desempacotando listas de argumentos

A situação inversa ocorre quando os argumentos já estão numa lista ou tupla mas ela precisa ser explodida para invocarmos uma função que requer argumentos posicionais separados. Por exemplo, a função `range()` espera argumentos separados, *start* e *stop*. Se os valores já estiverem juntos em uma lista ou tupla, escreva a chamada de função com o operador `*` para desempacotá-los da sequência:

```
>>> list(range(3, 6))            # chamada normal com argumentos
[3, 4, 5]
>>> args = [3, 6]
>>> list(range(*args))           # chamada com argumentos desemp
[3, 4, 5]
```

Da mesma forma, dicionários podem produzir argumentos nomeados com o operador `**`:

```
>>> def papagaio(voltagem, estado='um cadáver', ação='voar'):
...     print("-- Este papagaio não conseguiria", ação, end=' ')
...     print("nem se você desse um choque de", voltagem, "de vo
...     print("Ele", estado, "!")
...
>>> d = {"voltagem": "quatro milhões", "estado": "está realmente
>>> papagaio(**d)
-- Este papagaio não conseguiria voar nem se você desse um choqu
```

4.9.6. Expressões lambda

Pequenas funções anônimas podem ser criadas com a palavra-chave `lambda`. Esta função retorna a soma de seus dois argumentos: `lambda a, b: a+b`. As funções lambda podem ser usadas sempre que objetos função forem necessários. Eles são sintaticamente restritos a uma única expressão. Semanticamente, eles são apenas açúcar sintático para uma definição de função normal. Como definições de funções aninhadas, as funções lambda podem referenciar variáveis contidas no escopo:

```
>>> def cria_incrementador(n):
...     return lambda x: x + n
...
>>> f = cria_incrementador(42)
>>> f(0)
42
>>> f(1)
43
```

O exemplo acima usa uma expressão lambda para retornar uma função. Outro uso é passar uma pequena função como um argumento. Por exemplo, `list.sort()` recebe uma função chave de ordenação *key* que pode ser uma função lambda:

```
>>> pairs = [(1, 'um'), (2, 'dois'), (3, 'três), (4, 'quatro')]
>>> pairs.sort(key=lambda pair: pair[1])
>>> pairs
[(4, 'quatro'), (1, 'um'), (3, 'três'), (2, 'dois')]
```

4.9.7. Strings de documentação

Aqui estão algumas convenções sobre o conteúdo e formatação de strings de documentação, também conhecidas como docstrings.

A primeira linha deve sempre ser curta, um resumo conciso do propósito do objeto. Por brevidade, não deve explicitamente se referir ao nome ou tipo do objeto, uma vez que estas informações estão disponíveis por outros meios (exceto se o nome da função for o próprio verbo que descreve a finalidade da função). Essa linha deve começar com letra maiúscula e terminar com ponto.

Se existem mais linhas na string de documentação, a segunda linha deve estar em branco, separando visualmente o resumo do resto da descrição. As linhas seguintes devem conter um ou mais parágrafos descrevendo as convenções de chamada ao objeto, seus efeitos colaterais, etc.

A analisador sintático do Python remove a indentação de literais de string multilinha quando estas servem como docstrings de módulos, classes ou funções.

Eis um exemplo de uma string de documentação multilinha:

```
>>> def minha_função():
...     """Faz nada, mas documentemos-a.
...
...     Falando sério, ele faz nada:
...
...         >>> minha_função()
...         >>>
...     """
...     pass
...
>>> print(minha_função.__doc__)
Faz nada, mas documentemos-a.

Falando sério, ele faz nada:

    >>> minha_função()
    >>>
```

4.9.8. Anotações de função

Anotações de função são informações de metadados completamente opcionais sobre os tipos usados pelas funções definidas pelo usuário (veja **PEP 3107** [https://peps.python.org/pep-3107/] e **PEP 484** [https://peps.python.org/pep-0484/] para mais informações).

Anotações são armazenadas no atributo `__annotations__` da função como um dicionário e não tem nenhum efeito em qualquer outra parte da função. Anotações de parâmetro são definidas por dois-pontos (“:”) após o nome do parâmetro, seguida por uma expressão que quando avaliada determina o valor da anotação. Anotações do tipo do retorno são definidas por um literal `->`, seguida por uma expressão, entre a lista de parâmetro e os dois-pontos que marcam o fim da instrução `def` . O exemplo a seguir possui um argumento obrigatório, um argumento opcional e o valor de retorno anotados:

```
>>> def f(ham: str, ovos: str = 'ovos') -> str:
...     print("Anotações:", f.__annotations__)
...     print("Argumentos:", ham, ovos)
```

```
...     return ham + ' e ' + ovos
...
>>> f('spam')
Anotações: {'ham': <class 'str'>, 'return': <class 'str'>, 'ovos
Argumentos: spam ovos
'spam e ovos'
```

4.10. Intermezzo: estilo de codificação

Agora que está prestes a escrever códigos mais longos e complexos em Python, é um bom momento para falar sobre *estilo de codificação*. A maioria das linguagens podem ser escritas (ou *formatadas*) em diferentes estilos; alguns são mais legíveis do que outros. Tornar o seu código mais fácil de ler, para os outros, é sempre uma boa ideia, e adotar um estilo de codificação agradável ajuda bastante.

Em Python, a **PEP 8** [https://peps.python.org/pep-0008/] tornou-se o guia de estilo adotado pela maioria dos projetos; promove um estilo de codificação muito legível e visualmente agradável. Todo desenvolvedor Python deve lê-lo em algum momento; eis os pontos mais importantes, selecionados para você:

- Use indentação com 4 espaços, e não use tabulações.

  4 espaços são um bom meio termo entre indentação estreita (permite maior profundidade de aninhamento) e indentação larga (mais fácil de ler). Tabulações trazem complicações, e o melhor é não usar.

- Quebre as linhas de modo que não excedam 79 caracteres.

  Isso ajuda os usuários com telas pequenas e torna possível abrir vários arquivos de código lado a lado em telas maiores.

- Deixe linhas em branco para separar funções e classes, e blocos de código dentro de funções.

- Quando possível, coloque comentários em uma linha própria.

- Escreva strings de documentação.

- Use espaços ao redor de operadores e após vírgulas, mas não diretamente dentro de parênteses, colchetes e chaves: `a = f(1, 2) + g(3, 4)`.

- Nomeie suas classes e funções de forma consistente; a convenção é usar `MaiusculoCamelCase` para classes e `minusculo_com_sublinhado` para funções e métodos. Sempre use `self` como o nome para o primeiro argumento dos métodos (veja Uma primeira olhada nas classes para mais informações sobre classes e métodos).

- Não use codificações exóticas se o seu código é feito para ser usado em um contexto internacional. O padrão do Python, UTF-8, ou mesmo ASCII puro funciona bem em qualquer caso.

- Da mesma forma, não use caracteres não-ASCII em identificadores se houver apenas a menor chance de pessoas falando um idioma diferente ler ou manter o código.

Notas de rodapé

[1]

Na verdade, *passagem por referência para objeto* seria uma descrição melhor, pois, se um objeto mutável for passado, quem chamou verá as alterações feitas por quem foi chamado (por exemplo, a inclusão de itens em uma lista).

5. Estruturas de dados

Esse capítulo descreve algumas coisas que você já aprendeu em detalhes e adiciona algumas coisas novas também.

5.1. Mais sobre listas

O tipo de dado lista tem ainda mais métodos. Aqui estão apresentados todos os métodos de objetos do tipo lista:

list.append(*value*, /)

Adiciona um item ao fim da lista. Similar a `a[len(a):] = [x]`.

list.extend(*iterable*, /)

Estende a lista, adicionando no fim todos os elementos do argumento iterável passado como parâmetro. Similar a `a[len(a):] = iterable`.

list.insert(*index*, *value*, /)

Insere um item em uma dada posição. O primeiro argumento é o índice do elemento antes do qual será feita a inserção, assim `a.insert(0, x)` insere um elemento na frente da lista e `a.insert(len(a), x)` e equivale a `a.append(x)`.

list.remove(*value*, /)

Remove o primeiro item encontrado na lista cujo valor é igual a *value*. Se não existir valor igual, uma exceção **`ValueError`** é levantada.

list.pop(*index=-1*, /)

Remove o item na posição fornecida na lista e retorna. Se nenhum índice for especificado, `a.pop()` remove e retorna o último item da lista. Levanta um **`IndexError`** se a lista estiver vazia ou o índice estiver fora do intervalo da lista.

list.clear()

Remove todos os itens de uma lista. Similar a `del a[:]`.

list.index(*value*[, *start*[, *stop*]])

Retorna o índice de base zero da primeira ocorrência de *value* na lista. Levanta `ValueError` se não houver tal item.

Os argumentos opcionais *start* e *end* são interpretados como nas notações de fatiamento e são usados para limitar a busca para uma subsequência específica da lista. O índice retornado é calculado relativo ao começo da sequência inteira e não referente ao argumento *start*.

list.count(*value*, /)

Devolve o número de vezes em que *value* aparece na lista.

list.sort(*, *key=None*, *reverse=False*)

Ordena os itens na lista (os argumentos podem ser usados para personalizar a ordenação, veja a função `sorted()` para maiores explicações).

list.reverse()

Inverte a ordem dos elementos na lista.

list.copy()

Devolve uma cópia rasa da lista. Similar a `a[:]`.

Um exemplo que usa a maior parte dos métodos das listas:

```
>>> frutas = ['laranja', 'maçã', 'pera', 'banana', 'kiwi', 'maçã
>>> frutas.count('maçã')
2
>>> frutas.count('tangerina')
0
>>> frutas.index('banana')
3
>>> frutas.index('banana', 4) # Encontra a próxima banana inicia
6
```

```
>>> frutas.reverse()
>>> frutas
['banana', 'maçã', 'kiwi', 'banana', 'pera', 'maçã', 'laranja']
>>> frutas.append('uva')
>>> frutas
['banana', 'maçã', 'kiwi', 'banana', 'pera', 'maçã', 'laranja',
>>> frutas.sort()
>>> frutas
['maçã', 'maçã', 'banana', 'banana', 'uva', 'kiwi', 'laranja', '
>>> frutas.pop()
'pera'
```

Você pode ter percebido que métodos como `insert`, `remove` ou `sort`, que apenas modificam a lista, não têm valor de retorno impresso – eles retornam o `None` padrão. [[1]] Isto é um princípio de design para todas as estruturas de dados mutáveis em Python.

Outro aspecto que você pode notar é que nem todos os dados podem ser classificados ou comparados. Por exemplo, `[None, 'hello', 10]` não é ordenável porque os inteiros não podem ser comparados a strings e `None` não pode ser comparado a outros tipos. Além disso, há alguns tipos que não têm uma relação de ordenação definida. Por exemplo, `3+4j < 5+7j` não é uma comparação válida.

5.1.1. Usando listas como pilhas

Os métodos de lista tornam muito fácil utilizar listas como pilhas, onde o item adicionado por último é o primeiro a ser recuperado (política "último a entrar, primeiro a sair"). Para adicionar um item ao topo da pilha, use **`append()`**. Para recuperar um item do topo da pilha use **`pop()`** sem nenhum índice. Por exemplo:

```
>>> pilha = [3, 4, 5]
>>> pilha.append(6)
>>> pilha.append(7)
>>> pilha
[3, 4, 5, 6, 7]
>>> pilha.pop()
7
```

```
>>> pilha
[3, 4, 5, 6]
>>> pilha.pop()
6
>>> pilha.pop()
5
>>> pilha
[3, 4]
```

5.1.2. Usando listas como filas

Você também pode usar uma lista como uma fila, onde o primeiro item adicionado é o primeiro a ser recuperado (política “primeiro a entrar, primeiro a sair”); porém, listas não são eficientes para esta finalidade. Embora *appends* e *pops* no final da lista sejam rápidos, fazer *inserts* ou *pops* no início da lista é lento (porque todos os demais elementos têm que ser deslocados). Consulte Complexidade de tempo de operações em tipos embutidos para mais informações

Para implementar uma fila, use a classe **`collections.deque`** que foi projetada para permitir *appends* e *pops* eficientes nas duas extremidades. Por exemplo:

```
>>> from collections import deque
>>> fila = deque(["Erik", "João", "Miguel"])
>>> fila.append("Tiago")            # Tiago chega
>>> fila.append("George")           # George chega
>>> fila.popleft()                  # O primeiro a chegar agora
'Erik'
>>> fila.popleft()                  # O segundo a chegar agora s
'João'
>>> fila                            # A fila restante na ordem d
deque(['Miguel', 'Tiago', 'George'])
```

5.1.3. Compreensões de lista

Compreensões de lista fornece uma maneira concisa de criar uma lista. Aplicações comuns são criar novas listas onde cada elemento é o resultado de alguma operação aplicada a cada elemento de outra sequência ou

iterável, ou criar uma subsequência de elementos que satisfaçam uma certa condição. (N.d.T. o termo original em inglês é *list comprehensions*, muito utilizado no Brasil; também se usa a abreviação *listcomp*).

Por exemplo, suponha que queremos criar uma lista de quadrados, assim:

```
>>> quadrados = []
>>> for x in range(10):
...     quadrados.append(x**2)
...
>>> quadrados
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Note que isto cria (ou sobrescreve) uma variável chamada `x` que ainda existe após o término do laço. Podemos calcular a lista dos quadrados sem qualquer efeito colateral usando:

```
quadrados = list(map(lambda x: x**2, range(10)))
```

ou, de maneira equivalente:

```
quadrados = [x**2 for x in range(10)]
```

que é mais conciso e legível.

Um compreensão de lista consiste de um par de colchetes contendo uma expressão seguida de uma cláusula **`for`**, e então zero ou mais cláusulas **`for`** ou **`if`**. O resultado será uma nova lista resultante da avaliação da expressão no contexto das cláusulas **`for`** e **`if`**. Por exemplo, essa compreensão combina os elementos de duas listas se eles forem diferentes:

```
>>> [(x, y) for x in [1,2,3] for y in [3,1,4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

e é equivalente a:

```
>>> combos = []
>>> for x in [1,2,3]:
...     for y in [3,1,4]:
...         if x != y:
```

```
...             combos.append((x, y))
...
>>> combos
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

Note como a ordem das instruções **`for`** e **`if`** é a mesma em ambos os trechos.

Se a expressão é uma tupla (ex., `(x, y)` no exemplo anterior), ela deve ser inserida entre parênteses.

```
>>> vec = [-4, -2, 0, 2, 4]
>>> # cria uma nova lista com os valores dobrados
>>> [x*2 for x in vec]
[-8, -4, 0, 4, 8]
>>> # filtra a lista para excluir números negativos
>>> [x for x in vec if x >= 0]
[0, 2, 4]
>>> # aplica uma função para todos os elementos
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
>>> # chama um método em cada elemento
>>> frutafresca = ['  banana', '  baga-de-logan ', 'maracujá  ']
>>> [arma.strip() for arma in frutafresca]
['banana', 'baga-de-logan', 'maracujá']
>>> # cria uma lista de tuplas de 2 elementos como (número, quac
>>> [(x, x**2) for x in range(6)]
[(0, 0), (1, 1), (2, 4), (3, 9), (4, 16), (5, 25)]
>>> # a tupla deve estar entre parênteses, do contrário um erro
>>> [x, x**2 for x in range(6)]
  File "<stdin>", line 1
    [x, x**2 for x in range(6)]
     ^^^^^^^
SyntaxError: did you forget parentheses around the comprehensior
>>> # achatamento de uma lista usando uma compreensão de lista c
>>> vec = [[1,2,3], [4,5,6], [7,8,9]]
>>> [num for elem in vec for num in elem]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Compreensões de lista podem conter expressões complexas e funções aninhadas:

```
>>> from math import pi
>>> [str(round(pi, i)) for i in range(1, 6)]
['3.1', '3.14', '3.142', '3.1416', '3.14159']
```

5.1.4. Compreensões de lista aninhadas

A expressão inicial em uma compreensão de lista pode ser qualquer expressão arbitrária, incluindo outra compreensão de lista.

Observe este exemplo de uma matriz 3x4 implementada como uma lista de 3 listas de comprimento 4:

```
>>> matriz = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]
```

A compreensão de lista abaixo transpõe as linhas e colunas:

```
>>> [[linha[i] for linha in matriz] for i in range(4)]
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Como vimos na seção anterior, a compreensão de lista interna é computada no contexto da cláusula `for` seguinte, portanto o exemplo acima equivale a:

```
>>> transposta = []
>>> for i in range(4):
...     transposta.append([row[i] for linha in matriz])
...
>>> transposta
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

e isso, por sua vez, faz o mesmo que isto:

```
>>> transposta = []
>>> for i in range(4):
...     # as 3 linhas a seguir implementam uma compreensão de li
...     linha_transposta = []
...     for linha in matriz:
...         linha_transposta.append(linha[i])
```

```
...     transposta.append(linha_transposta)
...
>>> transposta
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Na prática, você deve dar preferência a funções embutidas em vez de instruções complexas. A função `zip()` resolve muito bem este caso de uso:

```
>>> list(zip(*matriz))
[(1, 5, 9), (2, 6, 10), (3, 7, 11), (4, 8, 12)]
```

Veja Desempacotando listas de argumentos para entender o uso do asterisco neste exemplo.

5.1.5. Desempacotamento em Listas e Compreensões de Lista

A seção sobre Desempacotando listas de argumentos descreve o uso de `*` para "desempacotar" os elementos de um objeto iterável, fornecendo cada um separadamente como um argumento para uma função. Desempacotamento também pode ser usado noutros contextos, por exemplo, quando criar listas. Ao especificar elementos de uma lista, prefixar uma expressão com um `*` irá desempacotar o resultado dessa expressão, adicionando cada um dos seus elementos à lista que estamos a criar:

```
>>> x = [1, 2, 3]
>>> [0, *x, 4, 5, 6]
[0, 1, 2, 3, 4, 5, 6]
```

Isto apenas funciona se a expressão a seguida de `*` for avaliada como um objeto iterável; tentar desempacotar um objeto não iterável irá levantar uma exceção:

```
>>> x = 1
>>> [0, *x, 2, 3, 4]
Traceback (most recent call last):
  File "<python-input-1>", line 1, in <module>
    [0, *x, 2, 3, 4]
TypeError: Value after * must be an iterable, not int
```

Desempacotamento também pode ser usado em compreensões de lista, como uma forma de construir uma nova lista que representa a concatenação de um número arbitrário de iteráveis:

```
>>> x = [[1, 2, 3], [4, 5, 6], [], [7], [8, 9]]
>>> [*elemento for elemento in x]
[1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Note que o efeito é que cada elemento de `x` é desempacotado. Isto funciona para objetos iteráveis arbitrários, não apenas listas:

```
>>> x = [[1, 2, 3], 'cat', {'spam': 'eggs'}]
>>> [*elemento for elemento in x]
[1, 2, 3, 'c', 'a', 't', 'spam']
```

Mas se os objetos em `x` não forem iteráveis, esta expressão levanta novamente uma exceção.

5.2. A instrução `del`

Existe uma maneira de remover um item de uma lista usando seu índice no lugar do seu valor: a instrução `del`. Ela difere do método `pop()` que devolve um valor. A instrução `del` pode também ser utilizada para remover fatias de uma lista ou limpar a lista inteira (que fizemos antes atribuindo uma lista vazia à fatia `a[:]`). Por exemplo:

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]
```

`del` também pode ser usado para remover totalmente uma variável:

```
>>> del a
```

Referenciar a variável a depois de sua remoção constitui erro (pelo menos até que seja feita uma nova atribuição para ela). Encontraremos outros usos para a instrução **`del`** mais tarde.

5.3. Tuplas e Sequências

Vimos que listas e strings têm muitas propriedades em comum, como indexação e operações de fatiamento. Elas são dois exemplos de *sequências* (veja Tipos sequências — list, tuple, range). Como Python é uma linguagem em evolução, outros tipos de sequências podem ser adicionados. Existe ainda um outro tipo de sequência padrão na linguagem: a *tupla*.

Uma tupla consiste em uma sequência de valores separados por vírgulas, por exemplo:

```
>>> t = 12345, 54321, 'olá!'
>>> t[0]
12345
>>> t
(12345, 54321, 'olá!')
>>> # As tuplas podem ser aninhadas:
>>> u = t, (1, 2, 3, 4, 5)
>>> u
((12345, 54321, 'olá!'), (1, 2, 3, 4, 5))
>>> # As tuplas são imutáveis:
>>> t[0] = 88888
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
>>> # mas podem conter objetos mutáveis:
>>> v = ([1, 2, 3], [3, 2, 1])
>>> v
([1, 2, 3], [3, 2, 1])
>>> # suportam desempacotamento tal como as listas:
>>> x = [1, 2, 3]
>>> 0, *x, 4
(0, 1, 2, 3, 4)
```

Como você pode ver no trecho acima, na saída do console as tuplas são sempre envolvidas por parênteses, assim tuplas aninhadas podem ser lidas

corretamente. Na criação, tuplas podem ser envolvidas ou não por parênteses, desde que o contexto não exija os parênteses (como no caso da tupla dentro de uma expressão maior). Não é possível atribuir itens individuais de uma tupla, contudo é possível criar tuplas que contenham objetos mutáveis, como listas.

Apesar de tuplas serem similares a listas, elas são frequentemente utilizadas em situações diferentes e com propósitos distintos. Tuplas são imutáveis, e usualmente contém uma sequência heterogênea de elementos que são acessados via desempacotamento (ver a seguir nessa seção) ou índice (ou mesmo por um atributo no caso de **`namedtuples`**). Listas são mutáveis, e seus elementos geralmente são homogêneos e são acessados iterando sobre a lista.

Um problema especial é a criação de tuplas contendo 0 ou 1 itens: a sintaxe usa certos truques para acomodar estes casos. Tuplas vazias são construídas por um par de parênteses vazios; uma tupla unitária é construída por um único valor seguido de uma vírgula (não basta colocar um único valor entre parênteses). Feio, mas funciona. Por exemplo:

```
>>> vazio = ()
>>> unitário = 'olá', # <!- note a vírgula ao final
>>> len(vazio)
0
>>> len(unitário)
1
>>> unitário
('olá',)
```

A instrução `t = 12345, 54321, 'bom dia!'` é um exemplo de *empacotamento de tupla*: os valores `12345`, `54321` e `'bom dia!'` são empacotados em uma tupla. A operação inversa também é possível:

```
>>> x, y, z = t
```

Isso é chamado, apropriadamente, de *desempacotamento de sequência* e funciona para qualquer sequência no lado direito. O desempacotamento de sequência requer que haja tantas variáveis no lado esquerdo do sinal de igual, quanto existem de elementos na sequência. Observe que a atribuição

múltipla é, na verdade, apenas uma combinação de empacotamento de tupla e desempacotamento de sequência.

5.4. Conjuntos

Python também inclui um tipo de dados para conjuntos, chamado `set`. Um conjunto é uma coleção desordenada de elementos, sem elementos repetidos. Usos comuns para conjuntos incluem a verificação eficiente da existência de objetos e a eliminação de itens duplicados. Conjuntos também suportam operações matemáticas como união, interseção, diferença e diferença simétrica.

Chaves ou a função **`set()`** podem ser usados para criar conjuntos. Note: para criar um conjunto vazio você precisa usar `set()`, não `{}`; este último cria um dicionário vazio, uma estrutura de dados que discutiremos na próxima seção.

Como os conjuntos não são ordenados, iterá-los ou imprimi-los pode produzir os elementos em uma ordem diferente da esperada.

Uma pequena demonstração:

```
>>> cesta = {'maçã', 'laranja', 'maçã', 'pera', 'laranja', 'bana
>>> print(cesta)                      # mostra que itens duplic
{'laranja', 'banana', 'pera', 'maçã'}
>>> 'laranja' in cesta                # teste de pertinência rá
True
>>> 'crabgrass' in cesta
False

>>> # Demonstra operações de conjunto em letras únicas de duas p
>>>
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                 # letras únicas em a
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                             # letras em a, mas não em
{'r', 'd', 'b'}
>>> a | b                             # letras em a ou em b ou
```

```
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                              # letras em ambos a e b
{'a', 'c'}
>>> a ^ b                              # letras em a ou b, mas r
{'r', 'd', 'b', 'm', 'z', 'l'}
```

Semelhante às compreensões de lista, as compreensões de set também são suportadas, incluindo compreensões com desempacotamento:

```
>>> a = {x for x in 'abracadabra' if x not in 'abc'}
>>> a
{'r', 'd'}

>>> frutas = [{'maçã', 'abacate', 'damasco'}, {'banana', 'mirtil
>>> {*fruta for fruta in frutas}
{'mirtilo', 'banana', 'abacate', 'maçã', 'damasco'}
```

5.5. Dicionários

Outra estrutura de dados embutida muito útil em Python é o *dicionário*, cujo tipo é `dict` (ver Tipos mapeamentos — dict, frozendict). Dicionários são também chamados de “memória associativa” ou “vetor associativo” em outras linguagens. Diferente de sequências que são indexadas por inteiros, dicionários são indexados por chaves (*keys*), que podem ser de qualquer tipo imutável (como strings e inteiros). Tuplas também podem ser chaves se contiverem apenas strings, inteiros ou outras tuplas. Se a tupla contiver, direta ou indiretamente, qualquer valor mutável, não poderá ser chave. Listas não podem ser usadas como chaves porque podem ser modificadas *internamente* pela atribuição em índices ou fatias, e por métodos como **`append()`** e **`extend()`**.

Um bom modelo mental é imaginar um dicionário como um conjunto não-ordenado de pares *chave:valor*, onde as chaves são únicas em uma dada instância do dicionário. Dicionários são delimitados por chaves: `{}`, e contém uma lista de pares chave:valor separada por vírgulas. Dessa forma também será exibido o conteúdo de um dicionário no console do Python. O dicionário vazio é `{}`.

As principais operações em um dicionário são armazenar e recuperar valores a partir de chaves. Também é possível remover um par *chave:valor* com o comando `del`. Se você armazenar um valor utilizando uma chave já presente, o antigo valor será substituído pelo novo.

Extrair um valor para uma chave inexistente por meio de um índice (`d[key]`) levanta **`KeyError`**. Para evitar esse erro ao tentar acessar uma chave possivelmente inexistente, use o método **`get()`**, que retorna `None` (ou um valor padrão especificado) se a chave não estiver no dicionário.

Executar `list(d)` em um dicionário devolve a lista de todas as chaves presentes no dicionário, na ordem de inserção (se desejar ordená-las basta usar a função `sorted(d)`). Para verificar a existência de uma chave, use o operador **`in`**.

A seguir, um exemplo de uso do dicionário:

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']
4098
>>> tel['irv']
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
KeyError: 'irv'
>>> print(tel.get('irv'))
None
>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel
True
>>> 'jack' not in tel
False
```

O construtor `dict()` produz dicionários diretamente de sequências de pares chave-valor:

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}
```

Além disso, as compreensões de dicionários podem ser usadas para criar dicionários a partir de expressões arbitrárias de chave e valor:

```
>>> {x: x**2 for x in (2, 4, 6)}
{2: 4, 4: 16, 6: 36}
```

E o desempacotamento de dicionários (através de `**`) pode ser utilizado para combinar múltiplos dicionários:

```
>>> chances= {i: i**2 for i in (1, 3, 5)}
>>> iguala= {i: i**2 for i in (2, 4, 6)}
>>> {**chances, **iguala}
{1: 1, 3: 9, 5: 25, 2: 4, 4: 16, 6: 36}

>>> todos_os_valores = [chances, iguala, {0: 0}]
>>> {**i for i in todos_os_valores}
{1: 1, 3: 9, 5: 25, 2: 4, 4: 16, 6: 36, 0: 0}
```

Quando chaves são strings simples, é mais fácil especificar os pares usando argumentos nomeados no construtor:

```
>>> dict(sape=4139, guido=4127, jack=4098)
{'sape': 4139, 'guido': 4127, 'jack': 4098}
```

5.6. Técnicas de iteração

Ao iterar sobre dicionários, a chave e o valor correspondente podem ser obtidos simultaneamente usando o método `items()`.

```
>>> cavaleiros = {'gallahad': 'o puro', 'robin': 'o bravo'}
>>> for k, v in cavaleiros.items():
...     print(k, v)
...
```

```
gallahad o puro
robin o bravo
```

Ao iterar sobre sequências, a posição e o valor correspondente podem ser obtidos simultaneamente usando a função **enumerate()**.

```
>>> for i, v in enumerate(['jogo', 'da', 'velha']):
...     print(i, v)
...
0 jogo
1 da
2 velha
```

Para percorrer duas ou mais sequências ao mesmo tempo, as entradas podem ser pareadas com a função **zip()**.

```
>>> perguntas = ['Nome', 'Missão', 'Cor favorita']
>>> respostas = ['Lancelot', 'o santo graal', 'azul']
>>> for q, a in zip(perguntas, respostas):
...     print('{0}? É {1}.'.format(q, a))
...
Nome? É Lancelot.
Missão? É o santo graal.
Cor favorita? É azul.
```

Para percorrer uma sequência em ordem inversa, chame a função **reversed()** com a sequência na ordem original.

```
>>> for i in reversed(range(1, 10, 2)):
...     print(i)
...
9
7
5
3
1
```

Para percorrer uma sequência de maneira ordenada, use a função **sorted()**, que retorna uma lista ordenada com os itens, mantendo a sequência original inalterada.

```
>>> cesta = ['maçã', 'laranja', 'maçã', 'pera', 'laranja', 'bana
>>> for i in sorted(cesta):
...     print(i)
...
maçã
maçã
banana
laranja
laranja
pera
```

Usar `set()` em uma sequência elimina elementos duplicados. O uso de `sorted()` em combinação com **`set()`** sobre uma sequência é uma maneira idiomática de fazer um loop sobre elementos exclusivos da sequência na ordem de classificação.

```
>>> cesta = ['maçã', 'laranja', 'maçã', 'pera', 'laranja', 'bana
>>> for i in sorted(cesta):
...     print(i)
...
maçã
maçã
banana
laranja
laranja
pera
```

Às vezes é tentador alterar uma lista enquanto você itera sobre ela; porém, costuma ser mais simples e seguro criar uma nova lista.

```
>>> import math
>>> dados_brutos = [56.2, float('NaN'), 51.7, 55.3, 52.5, float(
>>> dados_filtrados = []
>>> for valor in dados_brutos:
...     if not math.isnan(value):
...         dados_filtrados.append(valor)
...
>>> dados_filtrados
[56.2, 51.7, 55.3, 52.5, 47.8]
```

5.7. Mais sobre condições

As condições de controle usadas nas instruções `while` e `if` podem conter quaisquer operadores, não apenas comparações.

Os operadores de comparação `in` e `not in` fazem testes de inclusão que determinam se um valor está (ou não está) em um contêiner. Os operadores `is` e `is not` comparam se dois objetos são realmente o mesmo objeto. Todos os operadores de comparação possuem a mesma prioridade, que é menor que a prioridade de todos os operadores numéricos.

Comparações podem ser encadeadas: Por exemplo `a < b == c` testa se `a` é menor que `b` e também se `b` é igual a `c`.

Comparações podem ser combinadas através de operadores booleanos `and` e `or`, e o resultado de uma comparação (ou de qualquer outra expressão), pode ter seu valor booleano negado através de `not`. Estes possuem menor prioridade que os demais operadores de comparação. Entre eles, `not` é o de maior prioridade e `or` o de menor. Dessa forma, a condição `A and not B or C` é equivalente a `(A and (not B)) or C`. Naturalmente, parênteses podem ser usados para expressar o agrupamento desejado.

Os operadores booleanos `and` e `or` são operadores *curto-circuito*: seus argumentos são avaliados da esquerda para a direita, e a avaliação encerra quando o resultado é determinado. Por exemplo, se `A` e `C` são expressões verdadeiras, mas `B` é falsa, então `A and B and C` não chega a avaliar a expressão `C`. Em geral, quando usado sobre valores genéricos e não como booleanos, o valor do resultado de um operador curto-circuito é o último valor avaliado na expressão.

É possível atribuir o resultado de uma comparação ou outra expressão booleana para uma variável. Por exemplo:

```
>>> string1, string2, string3 = '', 'Trondheim', 'Hammer Dance'
>>> non_null = string1 or string2 or string3
>>> non_null
'Trondheim'
```

Observe que no Python, ao contrário de C, a atribuição dentro de expressões deve ser feita explicitamente com o operador morsa `:=`. Isso evita uma classe comum de problemas encontrados nos programas C: digitar `=` em uma expressão quando `==` era o planejado.

5.8. Comparando sequências e outros tipos

Objetos sequência podem ser comparados com outros objetos sequência, desde que o tipo das sequências seja o mesmo. A comparação utiliza a ordem lexicográfica: primeiramente os dois primeiros itens são comparados, e se diferirem isto determinará o resultado da comparação, caso contrário os próximos dois itens serão comparados, e assim por diante até que se tenha exaurido alguma das sequências. Se em uma comparação de itens, os mesmos forem também sequências (aninhadas), então é disparada recursivamente outra comparação lexicográfica. Se todos os itens da sequência forem iguais, então as sequências são ditas iguais. Se uma das sequências é uma subsequência da outra, então a subsequência é a menor. A comparação lexicográfica de strings utiliza codificação Unicode para definir a ordenação. Alguns exemplos de comparações entre sequências do mesmo tipo:

```
(1, 2, 3)              < (1, 2, 4)
[1, 2, 3]              < [1, 2, 4]
'ABC' < 'C' < 'Pascal' < 'Python'
(1, 2, 3, 4)           < (1, 2, 4)
(1, 2)                 < (1, 2, -1)
(1, 2, 3)             == (1.0, 2.0, 3.0)
(1, 2, ('aa', 'ab'))   < (1, 2, ('abc', 'a'), 4)
```

Note que comparar objetos de tipos diferentes com `<` ou `>` é permitido desde que os objetos possuam os métodos de comparação apropriados. Por exemplo, tipos numéricos mistos são comparados de acordo com os seus valores numéricos, portanto 0 é igual a 0.0, etc. Em caso contrário, ao invés de fornecer uma ordenação arbitrária, o interpretador levantará um `TypeError`.

Notas de rodapé

[1]

Outras linguagens podem retornar o objeto modificado, o que permite encadeamento de métodos, como `d->insert("a")->remove("b")->sort();`.

6. Módulos

Ao sair e entrar de novo no interpretador Python, as definições anteriores (funções e variáveis) são perdidas. Portanto, se quiser escrever um programa maior, será mais eficiente usar um editor de texto para preparar as entradas para o interpretador, e executá-lo usando o arquivo como entrada. Isso é conhecido como criar um *script*. Se o programa se torna ainda maior, é uma boa prática dividi-lo em arquivos menores, para facilitar a manutenção. Também é preferível usar um arquivo separado para uma função que você escreveria em vários programas diferentes, para não copiar a definição de função em cada um deles.

Para permitir isso, Python tem uma maneira de colocar as definições em um arquivo e então usá-las em um script ou em uma execução interativa do interpretador. Tal arquivo é chamado de *módulo*; definições de um módulo podem ser *importadas* para outros módulos, ou para o módulo *principal* (a coleção de variáveis a que você tem acesso num script executado como um programa e no modo calculadora).

Um módulo é um arquivo contendo definições e instruções Python. O nome do arquivo é o nome do módulo acrescido do sufixo `.py`. Dentro de um módulo, o nome do módulo (como uma string) está disponível como o valor da variável global `__name__`. Por exemplo, use seu editor de texto favorito para criar um arquivo chamado `fibo.py` no diretório atual com o seguinte conteúdo:

```
# Módulo de números de Fibonacci

def fib(n):
    """Escreve a série de Fibonacci até n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a+b
    print()
```

```
def fib2(n):
    """Retorna a série de Fibonacci até n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a+b
    return result
```

Agora, entre no interpretador Python e importe esse módulo com o seguinte comando:

```
>>> import fibo
```

Isso não adiciona os nomes das funções definidas em `fibo` diretamente ao espaço de nomes atual (veja Escopos e espaços de nomes do Python para mais detalhes); isso adiciona somente o nome do módulo `fibo`. Usando o nome do módulo você pode acessar as funções:

```
>>> fibo.fib(1000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987
>>> fibo.fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
>>> fibo.__name__
'fibo'
```

Se você pretende usar uma função muitas vezes, você pode atribui-lá a um nome local:

```
>>> fib = fibo.fib
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

6.1. Mais sobre módulos

Um módulo pode conter tanto instruções executáveis quanto definições de funções e classes. Essas instruções servem para inicializar o módulo. Eles são executados somente na *primeira* vez que o módulo é encontrado em uma instrução de importação. [[1]] (Também rodam se o arquivo é executado como um script.)

Cada módulo tem seu próprio espaço de nomes privado, que é usado como espaço de nomes global para todas as funções definidas no módulo. Assim, o autor de um módulo pode usar variáveis globais no seu módulo sem se preocupar com conflitos acidentais com as variáveis globais do usuário. Por outro lado, se você precisar usar uma variável global de um módulo, poderá fazê-lo com a mesma notação usada para se referir às suas funções, `nomemodulo.nomeitem`.

Módulos podem importar outros módulos. É costume, porém não obrigatório, colocar todas as instruções **`import`** no início do módulo (ou script , se preferir). As definições do módulo importado, se colocados no nível de um módulo (fora de quaisquer funções ou classes), elas são adicionadas a espaço de nomes global da módulo.

Existe uma variante da instrução **`import`** que importa definições de um módulo diretamente para o espaço de nomes do módulo importador. Por exemplo:

```
>>> from fibo import fib, fib2
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Isso não coloca o nome do módulo de onde foram feitas as importações no espaço de nomes local (assim, no exemplo, `fibo` não está definido).

Existe ainda uma variante que importa todos os nomes definidos em um módulo:

```
>>> from fibo import *
>>> fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Isso importa todos as declarações de nomes, exceto aqueles que iniciam com um sublinhado (_). Na maioria dos casos, programadores Python não usam esta facilidade porque ela introduz um conjunto desconhecido de nomes no ambiente, podendo esconder outros nomes previamente definidos.

Note que, em geral, a prática do `import *` de um módulo ou pacote é desaprovada, uma vez que muitas vezes dificulta a leitura do código. Contudo, é aceitável para diminuir a digitação em sessões interativas.

Se o nome do módulo é seguido pela palavra-chave **`as`**, o nome a seguir é vinculado diretamente ao módulo importado.

```
>>> import fibo as fib
>>> fib.fib(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Isto efetivamente importa o módulo, da mesma maneira que `import fibo` fará, com a única diferença de estar disponível com o nome `fib`.

Também pode ser utilizado com a palavra-chave **`from`**, com efeitos similares:

```
>>> from fibo import fib as fibonacci
>>> fibonacci(500)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377
```

Nota

For efficiency reasons, each module is only imported once per interpreter session. Therefore, if you change your modules, you must restart the interpreter – or, if it's just one module you want to test interactively, use **`importlib.reload()`**, e.g. `import importlib; importlib.reload(modulename)`.

6.1.1. Executando módulos como scripts

Quando você rodar um módulo Python com

```
python fibo.py <argumentos>
```

o código no módulo será executado, da mesma forma que quando é importado, mas com a variável `__name__` com valor `"__main__"`. Isto significa que adicionando este código ao final do seu módulo:

```
if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))
```

você pode tornar o arquivo utilizável tanto como script quanto como um módulo importável, porque o código que analisa a linha de comando só roda se o módulo é executado como arquivo “principal”:

```
$ python fibo.py 50
0 1 1 2 3 5 8 13 21 34
```

Se o módulo é importado, o código não é executado:

```
>>> import fibo
>>>
```

Isso é frequentemente usado para fornecer uma interface de usuário conveniente para um módulo, ou para realizar testes (rodando o módulo como um script executa um conjunto de testes).

6.1.2. O caminho de busca dos módulos

Quando um módulo chamado **`spam`** é importado, o interpretador procura um módulo embutido com este nome. Estes nomes de módulo são listados em **`sys.builtin_module_names`**. Se não encontra, procura um arquivo chamado `spam.py` em uma lista de diretórios incluídos na variável **`sys.path`**. A **`sys.path`** é inicializada com estes locais:

- O diretório que contém o script importador (ou o diretório atual quando nenhum arquivo é especificado).
- A variável de ambiente **`PYTHONPATH`** (uma lista de nomes de diretórios, com a mesma sintaxe da variável de ambiente **`PATH`**).
- O padrão dependente da instalação (por convenção, incluindo um diretório `site-packages`, tratado pelo módulo **`site`**).

Mais detalhes em A inicialização do caminho de pesquisa de módulos sys.path.

Nota

Nos sistemas de arquivos que suportam links simbólicos, o diretório contendo o script de entrada é resultante do diretório apontado pelo link simbólico. Em outras palavras o diretório que contém o link simbólico **não** é adicionado ao caminho de busca de módulos.

Após a inicialização, programas Python podem modificar `sys.path`. O diretório que contém o script sendo executado é colocado no início da lista de caminhos, à frente do caminho da biblioteca padrão. Isto significa que módulos nesse diretório serão carregados, no lugar de módulos com o mesmo nome na biblioteca padrão. Isso costuma ser um erro, a menos que seja intencional. Veja a seção Módulos padrões para mais informações.

6.1.3. Arquivos Python "compilados"

Para acelerar o carregamento de módulos, o Python guarda versões compiladas de cada módulo no diretório `__pycache__` com o nome `modulo.`*`versão`*`.pyc`, onde a versão corresponde ao formato do arquivo compilado; geralmente contêm o número da versão Python utilizada. Por exemplo, no CPython release 3.3 a versão compilada de spam.py será guardada como `__pycache__/spam.cpython-33.pyc`. Esta convenção de nomes permite a coexistência de módulos compilados de diferentes releases e versões de Python.

O Python verifica a data de modificação do arquivo fonte mediante a versão compilada, para ver se está desatualizada e precisa ser recompilada. É um processo completamente automático. Além disso, os módulos compilados são independentes de plataforma, portanto a mesma biblioteca pode ser compartilhada entre sistemas de arquiteturas diferentes.

O Python não verifica as versões compiladas em duas circunstâncias. Primeiro, sempre recompila e não armazena o resultado para módulos carregados diretamente da linha de comando. Segundo, não verifica se não houver um módulo fonte. Para suportar uma distribuição sem fontes (somente as versões compiladas), o módulo compilado deve estar no diretório de fontes, e não deve haver um módulo fonte.

Algumas dicas para especialistas:

- Você pode usar as opções `-O` ou `-OO` no comando Python para reduzir o tamanho de um módulo compilado. A opção `-O` remove as instruções assert, e a opção `-OO` remove, além das instruções assert, as strings de documentações. Como alguns programas podem contar com essa disponibilidade, só use essa opção se souber o que está fazendo. Módulos "otimizados" tem uma marcação `opt-` e são geralmente de menor tamanho. Futuros releases podem mudar os efeitos da otimização.
- Um programa não roda mais rápido quando é lido de um arquivo `.pyc` do que quando lido de um arquivo `.py`; a única coisa que é mais rápida com arquivos `.pyc` é sua velocidade de carregamento.
- O módulo `compileall` pode criar arquivos .pyc para todos os módulos de um diretório.
- Há mais detalhes desse processo, incluindo um fluxograma de decisões, no **PEP 3147** [https://peps.python.org/pep-3147/].

6.2. Módulos padrões

O Python traz uma biblioteca padrão de módulos, descrita em um documento em separado, a Referência da Biblioteca Python (doravante "Referência da Biblioteca"). Alguns módulos estão embutidos no interpretador; estes possibilitam acesso a operações que não são parte do núcleo da linguagem, mas estão no interpretador seja por eficiência ou para permitir o acesso a chamadas do sistema operacional. O conjunto destes módulos é uma opção de configuração que depende também da plataforma utilizada. Por exemplo, o módulo `winreg` só está disponível em sistemas Windows. Existe um módulo que requer especial atenção: `sys`, que é embutido em qualquer interpretador Python. As variáveis `sys.ps1` e `sys.ps2` definem as strings utilizadas como prompt primário e secundário:

```
>>> import sys
>>> sys.ps1
'>>> '
>>> sys.ps2
'... '
>>> sys.ps1 = 'C> '
```

```
C> print('ECA!')
Yuck!
C>
```

Essas variáveis só estão definidas se o interpretador está em modo interativo.

A variável `sys.path` contém uma lista de strings que determina os caminhos de busca de módulos conhecidos pelo interpretador. Ela é inicializada para um caminho padrão, determinado pela variável de ambiente **`PYTHONPATH`**, ou por um valor padrão embutido, se **`PYTHONPATH`** não estiver definida. Você pode modificá-la com as operações típicas de lista, por exemplo:

```
>>> import sys
>>> sys.path.append('/ufs/guido/lib/python')
```

6.3. A função `dir()`

A função embutida `dir()` é usada para descobrir quais nomes são definidos por um módulo. Ela devolve uma lista ordenada de strings:

```
>>> import fibo, sys
>>> dir(fibo)
['__name__', 'fib', 'fib2']
>>> dir(sys)
['__breakpointhook__', '__displayhook__', '__doc__', '__excepthc
 '__interactivehook__', '__loader__', '__name__', '__package__',
 '__stderr__', '__stdin__', '__stdout__', '__unraisablehook__',
 '_clear_type_cache', '_current_frames', '_debugmallocstats', '_
 '_getframe', '_git', '_home', '_xoptions', 'abiflags', 'addaudi
 'api_version', 'argv', 'audit', 'base_exec_prefix', 'base_prefi
 'breakpointhook', 'builtin_module_names', 'byteorder', 'call_tr
 'callstats', 'copyright', 'displayhook', 'dont_write_bytecode',
 'excepthook', 'exec_prefix', 'executable', 'exit', 'flags', 'fl
 'float_repr_style', 'get_asyncgen_hooks', 'get_coroutine_origir
 'getallocatedblocks', 'getdefaultencoding', 'getdlopenflags',
 'getfilesystemencodeerrors', 'getfilesystemencoding', 'getprofi
 'getrecursionlimit', 'getrefcount', 'getsizeof', 'getswitchinte
 'gettrace', 'hash_info', 'hexversion', 'implementation', 'int_i
```

```
 'intern', 'is_finalizing', 'last_traceback', 'last_type', 'last
 'maxsize', 'maxunicode', 'meta_path', 'modules', 'path', 'path_
 'path_importer_cache', 'platform', 'prefix', 'ps1', 'ps2', 'pyc
 'set_asyncgen_hooks', 'set_coroutine_origin_tracking_depth', 's
 'setprofile', 'setrecursionlimit', 'setswitchinterval', 'settra
 'stdin', 'stdout', 'thread_info', 'unraisablehook', 'version',
 'warnoptions']
```

Sem argumentos, `dir()` lista os nomes atualmente definidos:

```
>>> a = [1, 2, 3, 4, 5]
>>> import fibo
>>> fib = fibo.fib
>>> dir()
['__builtins__', '__name__', 'a', 'fib', 'fibo', 'sys']
```

Observe que ela lista todo tipo de nomes: variáveis, módulos, funções, etc.

`dir()` não lista os nomes de variáveis e funções embutidas. Esta lista está disponível no módulo padrão `builtins`:

```
>>> import builtins
>>> dir(builtins)
['ArithmeticError', 'AssertionError', 'AttributeError', 'BaseExc
 'BlockingIOError', 'BrokenPipeError', 'BufferError', 'BytesWarr
 'ChildProcessError', 'ConnectionAbortedError', 'ConnectionError
 'ConnectionRefusedError', 'ConnectionResetError', 'DeprecationW
 'EOFError', 'Ellipsis', 'EnvironmentError', 'Exception', 'False
 'FileExistsError', 'FileNotFoundError', 'FloatingPointError',
 'FutureWarning', 'GeneratorExit', 'IOError', 'ImportError',
 'ImportWarning', 'IndentationError', 'IndexError', 'Interrupted
 'IsADirectoryError', 'KeyError', 'KeyboardInterrupt', 'LookupEr
 'MemoryError', 'NameError', 'None', 'NotADirectoryError', 'Not1
 'NotImplementedError', 'OSError', 'OverflowError',
 'PendingDeprecationWarning', 'PermissionError', 'ProcessLookupE
 'ReferenceError', 'ResourceWarning', 'RuntimeError', 'RuntimeWa
 'StopIteration', 'SyntaxError', 'SyntaxWarning', 'SystemError',
 'SystemExit', 'TabError', 'TimeoutError', 'True', 'TypeError',
 'UnboundLocalError', 'UnicodeDecodeError', 'UnicodeEncodeError'
 'UnicodeError', 'UnicodeTranslateError', 'UnicodeWarning', 'Use
 'ValueError', 'Warning', 'ZeroDivisionError', '_', '__build_cla
```

```
'__debug__', '__doc__', '__import__', '__name__', '__package__'
'all', 'any', 'ascii', 'bin', 'bool', 'bytearray', 'bytes', 'ca
'chr', 'classmethod', 'compile', 'complex', 'copyright', 'credi
'delattr', 'dict', 'dir', 'divmod', 'enumerate', 'eval', 'exec'
'filter', 'float', 'format', 'frozenset', 'getattr', 'globals',
'hash', 'help', 'hex', 'id', 'input', 'int', 'isinstance', 'iss
'iter', 'len', 'license', 'list', 'locals', 'map', 'max', 'memo
'min', 'next', 'object', 'oct', 'open', 'ord', 'pow', 'print',
'quit', 'range', 'repr', 'reversed', 'round', 'set', 'setattr',
'sorted', 'staticmethod', 'str', 'sum', 'super', 'tuple', 'type
'zip']
```

6.4. Pacotes

Os pacotes são uma maneira de estruturar o “espaço de nomes” dos módulos Python, usando “nomes de módulo com pontos”. Por exemplo, o nome do módulo **`A.B`** designa um submódulo chamado `B`, em um pacote chamado `A`. Assim como o uso de módulos evita que os autores de módulos diferentes tenham que se preocupar com nomes de variáveis globais, o uso de nomes de módulos com pontos evita que os autores de pacotes com muitos módulos, como NumPy ou Pillow, tenham que se preocupar com os nomes dos módulos uns dos outros.

Suponha que você queira projetar uma coleção de módulos (um “pacote”) para o gerenciamento uniforme de arquivos de som. Existem muitos formatos diferentes (normalmente identificados pela extensão do nome de arquivo, por exemplo `.wav`, `.aiff`, `.au`), de forma que você pode precisar criar e manter uma crescente coleção de módulos de conversão entre formatos. Ainda podem existir muitas operações diferentes, passíveis de aplicação sobre os arquivos de som (mixagem, eco, equalização, efeito stereo artificial). Logo, possivelmente você também estará escrevendo uma coleção sempre crescente de módulos para aplicar estas operações. Eis uma possível estrutura para o seu pacote (expressa em termos de um sistema de arquivos hierárquico):

```
sound/                          pacote de nível superior
      __init__.py               Inicializa o pacote de som
      formats/                  Subpacote para as conversões ent
```

```
            __init__.py
            wavread.py
            wavwrite.py
            aiffread.py
            aiffwrite.py
            auread.py
            auwrite.py
            ...
      effects/                  Subpacote para efeitos de som
            __init__.py
            echo.py
            surround.py
            reverse.py
            ...
      filters/                  Subpacote para filtros
            __init__.py
            equalizer.py
            vocoder.py
            karaoke.py
            ...
```

Ao importar esse pacote, Python busca pelo subdiretório com mesmo nome, nos diretórios listados em `sys.path`.

Os arquivos `__init__.py` são necessários para que o Python trate diretórios contendo o arquivo como pacotes (a menos que se esteja usando um pacote de espaço de nomes, um recurso relativamente avançado). Isso impede que diretórios com um nome comum, como `string`, ocultem, involuntariamente, módulos válidos que ocorrem posteriormente no caminho de busca do módulo. No caso mais simples, `__init__.py` pode ser apenas um arquivo vazio, mas pode também executar código de inicialização do pacote, ou configurar a variável `__all__`, descrita mais adiante.

Usuários do pacote podem importar módulos individuais, por exemplo:

```
import sound.effects.echo
```

Isso carrega o submódulo **`sound.effects.echo`**. Ele deve ser referenciado com seu nome completo, como em:

```
sound.effects.echo.echofilter(input, output, delay=0.7, atten=4)
```

Uma maneira alternativa para a importação desse módulo é:

```
from sound.effects import echo
```

Isso carrega o submódulo **`echo`** sem necessidade de mencionar o prefixo do pacote no momento da utilização, assim:

```
echo.echofilter(input, output, delay=0.7, atten=4)
```

Também é possível importar diretamente uma única variável ou função:

```
from sound.effects.echo import echofilter
```

Novamente, isso carrega o submódulo **`echo`**, mas a função **`echofilter()`** está acessível diretamente sem prefixo:

```
echofilter(input, output, delay=0.7, atten=4)
```

Observe que ao utilizar `from pacote import item`, o item pode ser um subpacote, submódulo, classe, função ou variável. A instrução `import` primeiro testa se o item está definido no pacote, senão presume que é um módulo e tenta carregá-lo. Se falhar em encontrar o módulo, uma exceção **`ImportError`** é levantada.

Em oposição, em uma construção como `import item.subitem.subsubitem`, cada item, com exceção do último, deve ser um pacote. O último pode ser também um pacote ou módulo, mas nunca uma classe, função ou variável contida em um módulo.

6.4.1. Importando * de um pacote

Agora, o que acontece quando um usuário escreve `from sound.effects import *` ? Idealmente, poderia se esperar que este comando vasculhasse o sistema de arquivos, encontrasse todos os submódulos presentes no pacote, e os importasse. Isso poderia demorar muito e a importação de submódulos

pode ocasionar efeitos colaterais, que somente deveriam ocorrer quando o submódulo é explicitamente importado.

A única solução é o autor do pacote fornecer um índice explícito do pacote. A instrução `import` usa a seguinte convenção: se o arquivo `__init__.py` do pacote define uma lista chamada `__all__`, então esta lista indica os nomes dos módulos a serem importados quando a instrução `from pacote import *` é acionada. Fica a cargo do autor do pacote manter esta lista atualizada, inclusive fica a seu critério excluir inteiramente o suporte a importação direta de todo o pacote através de `from pacote import *`. Por exemplo, o arquivo `sounds/effects/__init__.py` poderia conter apenas:

```
__all__ = ["echo", "surround", "reverse"]
```

Isso significaria que `from sound.effects import *` importaria os três submódulos nomeados do pacote **`sound.effects`**.

Esteja ciente de que os submódulos podem ficar sobrepostos por nomes definidos localmente. Por exemplo, se você adicionou uma função `reverse` ao arquivo `sound/effects/__init__.py`, usar `from sound.effects import *` só importaria os dois submódulos `echo` e `surround`, mas *não* o submódulo `reverse`, porque ele fica sobreposto pela função `reverse` definida localmente:

```
__all__ = [
    "echo",       # refere-se ao arquivo 'echo.py'
    "surround",   # refere-se ao arquivo 'surround.py'
    "reverse",    # !!! refere-se à função 'reverse' agora !!!
]

def reverse(msg: str):  # <-- este nome ofusca o submódulo 'reve
    return msg[::-1]    #     no caso de uma importação 'from sc
```

Se `__all__` não estiver definido, a instrução `from sound.effects import *` não importa todos os submódulos do pacote **`sound.effects`** no espaço de nomes atual. Há apenas garantia que o pacote **`sound.effects`** foi importado (possivelmente executando qualquer código de inicialização em `__init__.py`) juntamente com os nomes definidos no pacote. Isso inclui

todo nome definido em `__init__.py` bem como em qualquer submódulo importado a partir deste. Também inclui quaisquer submódulos do pacote que tenham sido carregados explicitamente por instruções **`import`** anteriores. Considere o código abaixo:

```
import sound.effects.echo
import sound.effects.surround
from sound.effects import *
```

Nesse exemplo, os módulos **`echo`** e **`surround`** são importados no espaço de nomes atual, no momento em que a instrução `from...import` é executada, pois estão definidos no pacote **`sound.effects`**. (Isso também funciona quando `__all__` estiver definida.)

Apesar de que certos módulos são projetados para exportar apenas nomes conforme algum critério quando se faz `import *`, ainda assim essa sintaxe é considerada uma prática ruim em código de produção.

Lembre-se, não há nada errado em usar `from pacote import submodulo_especifico`! De fato, essa é a notação recomendada, a menos que o módulo importado necessite usar submódulos com o mesmo nome, de diferentes pacotes.

6.4.2. Referências em um mesmo pacote

Quando pacotes são estruturados em subpacotes (como no pacote **`sound`** do exemplo), pode-se usar a sintaxe de importações absolutas para se referir aos submódulos de pacotes irmãos (o que na prática é uma forma de fazer um import relativo, a partir da base do pacote). Por exemplo, se o módulo **`sound.filters.vocoder`** precisa usar o módulo **`echo`** do pacote **`sound.effects`**, é preciso importá-lo com `from sound.effects import echo`.

Também é possível escrever imports relativos, com a forma `from module import name`. Esses imports usam pontos para indicar o pacote pai e o atual, envolvidos no import relativo. Do módulo **`surround`**, por exemplo, pode-se usar:

```
from . import echo
from .. import formats
from ..filters import equalizer
```

Observe que as importações relativas são baseadas no nome do pacote do módulo atual. Como o módulo principal não possui um pacote, os módulos destinados ao uso como módulo principal de uma aplicação Python devem sempre usar importações absolutas.

Notas de rodapé

[1]

[#] Na verdade, definições de funções também são ‘instruções’ que são ‘executados’; a execução da definição de uma função adiciona o nome da função no espaço de nomes global do módulo.

7. Entrada e saída

Existem várias maneiras de apresentar a saída de um programa; os dados podem ser exibidos em forma legível para seres humanos, ou escritos em arquivos para uso posterior. Este capítulo apresentará algumas das possibilidades.

7.1. Formatação de saída mais elaborada

Até agora vimos duas maneiras de exibir valores: *expressões* e a função `print()`. (Uma outra maneira é utilizar o método `write()` de objetos do tipo arquivo; o arquivo saída padrão pode ser referenciado como `sys.stdout`. Veja a Referência da Biblioteca Python para mais informações sobre isso.)

Muitas vezes se deseja mais controle sobre a formatação da saída do que simplesmente exibir valores separados por espaço. Existem várias maneiras de formatar a saída.

- Para usar strings literais formatadas, comece uma string com `f` ou `F`, antes de abrir as aspas ou aspas triplas. Dentro dessa string, pode-se escrever uma expressão Python entre caracteres `{` e `}`, que podem se referir a variáveis, ou valores literais.

  ```
  >>> ano = 2016
  >>> evento = 'Referendo'
  >>> f'Resultados do {evento} {ano}'
  'Resultados do Referendo 2016'
  ```

- O método de strings `str.format()` requer mais esforço manual. Ainda será necessário usar `{` e `}` para marcar onde a variável será substituída e pode-se incluir diretivas de formatação detalhadas, mas também precisará incluir a informação a ser formatada. No bloco de código a seguir há dois exemplos de como formatar variáveis:

```
>>> votos_sim = 42_572_654
>>> votos_totais = 85_705_149
>>> porcentagem = votos_sim / votos_totais
>>> '{:-9} votos SIM  {:2.2%}'.format(votos_sim, porcentagem
' 42572654 votos SIM  49.67%'
```

Observe como `yes_votes` são preenchidos com espaços e um sinal negativo apenas para números negativos. O exemplo também imprime `percentage` multiplicado por 100, com 2 casas decimais e seguido por um sinal de porcentagem (veja Minilinguagem de especificação de formato para detalhes).

- Finalmente, pode-se fazer todo o tratamento da saída usando as operações de fatiamento e concatenação de strings para criar qualquer layout que se possa imaginar. O tipo string possui alguns métodos que realizam operações úteis para preenchimento de strings para uma determinada largura de coluna.

Quando não é necessário sofisticar a saída, mas apenas exibir algumas variáveis com propósito de depuração, pode-se converter qualquer valor para uma string com as funções **`repr()`** ou **`str()`**.

A função **`str()`** serve para retornar representações de valores que sejam legíveis para as pessoas, enquanto **`repr()`** é para gerar representações que o interpretador Python consegue ler (ou levantará uma exceção **`SyntaxError`**, se não houver sintaxe equivalente). Para objetos que não têm uma representação adequada para consumo humano, **`str()`** devolve o mesmo valor que **`repr()`**. Muitos valores, tal como números ou estruturas, como listas e dicionários, têm a mesma representação usando quaisquer das funções. Strings, em particular, têm duas representações distintas.

Alguns exemplos:

```
>>> s = 'Olá, mundo.'
>>> str(s)
'Olá, mundo.'
>>> repr(s)
"'Olá, mundo.'"
>>> str(1/7)
```

```
'0.14285714285714285'
>>> x = 10 * 3.25
>>> y = 200 * 200
>>> s = 'O valor de x é ' + repr(x) + ' e de y é ' + repr(y) + '
>>> print(s)
O valor de x é 32.5 e de y é 40000...
>>> # A repr() da string adiciona aspas e contrabarras:
>>> olá = 'olá, mundo\n'
>>> olás = repr(olá)
>>> print(olás)
'olá, mundo\n'
>>> # O argumento de repr() pode ser qualquer objeto Python:
>>> repr((x, y, ('spam', 'ovos')))
"(32.5, 40000, ('spam', 'ovos'))"
```

O módulo `string` contém suporte para uma abordagem simples de criação de modelos baseada em expressões regulares, via `string.Template`. Isso oferece outra maneira de substituir valores em strings, usando espaços reservados como `$x` e substituindo-os por valores de um dicionário. Essa sintaxe é fácil de usar, embora ofereça muito menos controle sobre a formatação.

7.1.1. Literais de string formatados

Strings literais formatadas (também chamadas f-strings, para abreviar) permite que se inclua o valor de expressões Python dentro de uma string, prefixando-a com `f` ou `F` e escrevendo expressões na forma `{expression}`.

Um especificador opcional de formato pode ser incluído após a expressão. Isso permite maior controle sobre como o valor é formatado. O exemplo a seguir arredonda pi para três casas decimais:

```
>>> import math
>>> print(f'O valor de pi é aproximadamente {math.pi:.3f}.')
O valor de pi é aproximadamente 3.142.
```

Passando um inteiro após o `':'` fará com que o campo tenha um número mínimo de caracteres de largura. Isso é útil para alinhar colunas.

```
>>> tabela = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 7678}
>>> for nome, telefone in tabela.items():
...     print(f'{nome:10} ==> {telefone:10d}')
...
Sjoerd     ==>       4127
Jack       ==>       4098
Dcab       ==>       7678
```

Outros modificadores podem ser usados para converter o valor antes de ser formatado. `'!a'` aplica a função **`ascii()`**, `'!s'` aplica a função **`str()`** e `'!r'` aplica a função **`repr()`**

```
>>> animais = 'enguias'
>>> print(f'Meu hovercraft está cheio de {animais}.')
Meu hovercraft está cheio de enguias.
>>> print(f'Meu hovercraft está cheio de {animais!r}.')
Meu hovercraft está cheio de 'enguias'.
```

O especificador `=` pode ser usado para expandir uma expressão para o texto da expressão, um sinal de igual e, então, a representação da expressão avaliada:

```
>>> bugs = 'roaches'
>>> count = 13
>>> area = 'living room'
>>> print(f'Debugging {bugs=} {count=} {area=}')
Debugging bugs='roaches' count=13 area='living room'
```

Veja expressões autodocumentadas para mais informações sobre o especificador `=`. Para obter uma referência sobre essas especificações de formato, consulte o guia de referência para a Minilinguagem de especificação de formato.

7.1.2. O método format() de strings

Um uso básico do método **`str.format()`** tem esta forma:

```
>>> print('Nós somos os {} que dizem "{}!"'.format('cavaleiros',
Nós somos os cavaleiros que dizem "Ni!"
```

As chaves e seus conteúdos (chamados campos de formatação) são substituídos pelos objetos passados para o método **`str.format()`**. Um número nas chaves pode ser usado para referenciar a posição do objeto passado no método **`str.format()`**.

```
>>> print('{0} e {1}'.format('spam', 'ovos'))
spam e eggs
>>> print('{1} e {0}'.format('spam', 'ovos'))
eggs e spam
```

Se argumentos nomeados são passados para o método **`str.format()`**, seus valores serão referenciados usando o nome do argumento:

```
>>> print('Este {comida} está {adjetivo}.'.format(
...       comida='spam', adjetivo='absolutamente horrível'))
Este spam está absolutamente horrível.
```

Argumentos posicionais e nomeados podem ser combinados à vontade:

```
>>> print('A história de {0}, {1} e {outro}.'.format('Bill', 'Ma
...                                                  outro='Ge
A história de Bill, Manfred e Georg.
```

Se uma string de formatação é muito longa, e não se deseja quebrá-la, pode ser bom fazer referência aos valores a serem formatados por nome, em vez de posição. Isto pode ser feito passando um dicionário usando colchetes `'[]'` para acessar as chaves.

```
>>> tabela = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 8637678}
>>> print('Jack: {0[Jack]:d}; Sjoerd: {0[Sjoerd]:d}; '
...       'Dcab: {0[Dcab]:d}'.format(tabela))
Jack: 4098; Sjoerd: 4127; Dcab: 8637678
```

Isto também pode ser feito passando o dicionário `table` como argumentos nomeados com a notação `**`.

```
>>> tabela = {'Sjoerd': 4127, 'Jack': 4098, 'Dcab': 8637678}
>>> print('Jack: {Jack:d}; Sjoerd: {Sjoerd:d}; Dcab: {Dcab:d}'.f
Jack: 4098; Sjoerd: 4127; Dcab: 8637678
```

Isto é particularmente útil em conjunto com a função embutida `vars()`, que devolve um dicionário contendo todas as variáveis locais:

```
>>> tabela = {k: str(v) for k, v in vars().items()}
>>> mensagem = " ".join([f'{k}: ' + '{' + k +'};' for k in tabel
>>> print(mensagem.format(**tabela))
__name__: __main__; __doc__: None; __package__: None; __loader__
```

Como exemplo, as linhas seguintes produzem um conjunto de colunas alinhadas, com alguns inteiros e seus quadrados e cubos:

```
>>> for x in range(1, 11):
...     print('{0:2d} {1:3d} {2:4d}'.format(x, x*x, x*x*x))
...
 1   1    1
 2   4    8
 3   9   27
 4  16   64
 5  25  125
 6  36  216
 7  49  343
 8  64  512
 9  81  729
10 100 1000
```

Para uma visão completa da formatação de strings com `str.format()`, veja a seção Sintaxe das strings de formato.

7.1.3. Formatação manual de strings

Aqui está a mesma tabela de quadrados e cubos, formatados manualmente:

```
>>> for x in range(1, 11):
...     print(repr(x).rjust(2), repr(x*x).rjust(3), end=' ')
...     # Observe o uso do 'end' na linha anterior
...     print(repr(x*x*x).rjust(4))
...
 1   1    1
 2   4    8
 3   9   27
```

```
 4  16   64
 5  25  125
 6  36  216
 7  49  343
 8  64  512
 9  81  729
10 100 1000
```

(Note que o espaço entre cada coluna foi adicionado pela forma que a função `print()` funciona: sempre adiciona espaços entre seus argumentos.)

O método `str.rjust()` justifica uma string à direita, num campo de tamanho definido, acrescentando espaços à esquerda. De forma similar, os métodos `str.ljust()`, justifica à esquerda, e `str.center()`, para centralizar. Esses métodos não escrevem nada, apenas retornam uma nova string. Se a string de entrada é muito longa, os métodos não truncarão a saída, e retornarão a mesma string, sem mudança; isso vai atrapalhar o layout da coluna, mas geralmente é melhor do que a alternativa, que estaria distorcendo o valor. (Se realmente quiser truncar, sempre se pode adicionar uma operação de fatiamento, como em `x.ljust(n)[:n]`.)

Existe ainda o método `str.zfill()` que preenche uma string numérica com zeros à esquerda, e sabe lidar com sinais positivos e negativos:

```
>>> '12'.zfill(5)
'00012'
>>> '-3.14'.zfill(7)
'-003.14'
>>> '3.14159265359'.zfill(5)
'3.14159265359'
```

7.1.4. Formatação de strings à moda antiga

O operador % (módulo) também pode ser usado para formatação de string. Dado `formato % valores` (onde *formato* é uma string), as instâncias de `%` em `formato` serão substituídas por zero ou mais elementos de `valores`. Essa operação é conhecida como interpolação de string. Por exemplo:

```
>>> import math
>>> print('O valor de pi é aproximadamente %5.3f.' % math.pi)
```

```
O valor de pi é aproximadamente 3.142.
```

Mais informação pode ser encontrada na seção Formatação de string no estilo printf.

7.2. Lendo e escrevendo arquivos

`open()` retorna um objeto arquivo, e é mais utilizado com dois argumentos posicionais: `open(filename, mode)`

```
>>> f = open('arquivo_de_trabalho', 'w')
```

O primeiro argumento é uma string contendo o nome do arquivo. O segundo argumento é outra string, contendo alguns caracteres que descrevem o modo como o arquivo será usado. *modo* pode ser `'r'` quando o arquivo será apenas lido, `'w'` para escrever (se o arquivo já existir seu conteúdo prévio será apagado), e `'a'` para abrir o arquivo para adição; qualquer escrita será adicionada ao final do arquivo. A opção `'r+'` abre o arquivo tanto para leitura como para escrita. O argumento *modo* é opcional, em caso de omissão será presumido `'r'`.

Normalmente, arquivos são abertos no *modo texto*, o que significa que você lê strings de e para o arquivo, o qual está em um codificação específica. Se a *codificação* não for especificada, o padrão é UTF-8 (veja **`open()`**). Adicionando `'b'` ao modo irá abrir o o arquivo em *modo binário*. Dados no modo binário são lidos e escritos como objetos **`bytes`**. Você não pode especificar a *codificação* quando estiver abrindo os arquivos em modo binário.

Em modo texto, o padrão durante a leitura é converter terminadores de linha específicos da plataforma (`\n` no Unix, `\r\n` no Windows) para apenas `\n`. Ao escrever no modo de texto, o padrão é converter as ocorrências de `\n` de volta para os finais de linha específicos da plataforma. Essa modificação de bastidores nos dados do arquivo é adequada para arquivos de texto, mas corromperá dados binários, como arquivos `JPEG` ou `EXE`. Tenha muito cuidado para só usar o modo binário, ao ler e gravar esses arquivos.

É uma boa prática usar a palavra-chave **with** ao lidar com arquivos. A vantagem é que o arquivo é fechado corretamente após o término de sua utilização, mesmo que uma exceção seja levantada em algum momento. Usar **with** também é muito mais curto que escrever seu bloco equivalente **try**-**finally**:

```
>>> with open('arquivo_de_trabalho') as f:
...     read_data = f.read()

>>> # Podemos verificar se o arquivo foi fechado automaticamente
>>> f.closed
True
```

Se você não está usando a palavra reservada **with**, então você deveria chamar `f.close()` para fechar o arquivo e imediatamente liberar qualquer recurso do sistema usado por ele.

Aviso

Chamar `f.write()` sem usar a palavra reservada **with** ou chamar `f.close()` **pode** resultar nos argumentos de `f.write()` não serem completamente escritos no disco, mesmo se o programa for encerrado com êxito.

Depois que um arquivo é fechado, seja por uma instrução **with** ou chamando `f.close()`, as tentativas de usar o arquivo falharão automaticamente.

```
>>> f.close()
>>> f.read()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: I/O operation on closed file.
```

7.2.1. Métodos de objetos arquivo

Para simplificar, o resto dos exemplos nesta seção presumem que um objeto arquivo chamado `f` já foi criado.

Para ler o conteúdo de um arquivo, chame `f.read(tamanho)`, que lê um punhado de dados devolvendo-os como uma string (em modo texto) ou bytes (em modo binário). *tamanho* é um argumento numérico opcional. Quando *tamanho* é omitido ou negativo, todo o conteúdo do arquivo é lido e devolvido; se o arquivo é duas vezes maior que memória da máquina, o problema é seu. Caso contrário, no máximo *tamanho* caracteres (em modo texto) ou *tamanho* bytes (em modo binário) são lidos e devolvidos. Se o fim do arquivo for atingido, `f.read()` devolve uma string vazia (`''`).

```
>>> f.read()
'Este é o arquivo inteiro.\n'
>>> f.read()
''
```

O método `f.readline()` lê uma única linha do arquivo; o caractere de quebra de linha (`\n`) é mantido ao final da string, e só é omitido na última linha do arquivo, se o arquivo não terminar com uma quebra de linha. Isso elimina a ambiguidade do valor retornado; se `f.readline()` retorna uma string vazia, o fim do arquivo foi atingido. Linhas em branco são representadas por um `'\n'` – uma string contendo apenas o caractere terminador de linha.

```
>>> f.readline()
'Está é a primeira linha do arquivo.\n'
>>> f.readline()
'Segunda linha do arquivo\n'
>>> f.readline()
''
```

Uma maneira alternativa de ler linhas do arquivo é iterar diretamente pelo objeto arquivo. É eficiente, rápido e resulta em código mais simples:

```
>>> for line in f:
...     print(line, end='')
...
Esta é a primeira linha do arquivo.
Segunda linha do arquivo
```

Se desejar ler todas as linhas de um arquivo em uma lista, pode-se usar `list(f)` ou `f.readlines()`.

`f.write(string)` escreve o conteúdo de *string* para o arquivo, retornando o número de caracteres escritos.

```
>>> f.write('Este é um teste\n')
15
```

Outros tipos de objetos precisam ser convertidos – seja para uma string (em modo texto) ou para bytes (em modo binário) – antes de escrevê-los:

```
>>> value = ('a resposta', 42)
>>> s = str(value)  # converte a tupla para string
>>> f.write(s)
18
```

`f.tell()` retorna um inteiro dando a posição atual do objeto arquivo, no arquivo representado, como número de bytes desde o início do arquivo, no modo binário, e um número ininteligível, quando no modo de texto.

Para mudar a posição, use `f.seek(offset, de_onde)`. A nova posição é computada pela soma do deslocamento *offset* a um ponto de referência especificado pelo argumento *de-onde*. Se o valor de *de_onde* é 0,a referência é o início do arquivo, 1 refere-se à posição atual, e 2 refere-se ao fim do arquivo. Este argumento pode ser omitido e o valor padrão é 0, usando o início do arquivo como referência.

```
>>> f = open('arquivo_de_trabalho', 'rb+')
>>> f.write(b'0123456789abcdef')
16
>>> f.seek(5)      # Vai até o 6º byte no arquivo
5
>>> f.read(1)
b'5'
>>> f.seek(-3, 2)  # Vai até o 3º byte antes do fim
13
>>> f.read(1)
b'd'
```

Em arquivos texto (abertos sem um `b`, em modo string), somente *seeks* relativos ao início do arquivo serão permitidos (exceto se for indicado o final do arquivo, com `seek(0, 2)`) e o único valor válido para *offset* são aqueles retornados por chamada à `f.tell()`, ou zero. Qualquer outro valor para *offset* produz um comportamento indefinido.

Objetos arquivo tem alguns métodos adicionais, como **`isatty()`** e **`truncate()`** que não são usados com frequência; consulte a Biblioteca de Referência para um guia completo de objetos arquivo.

7.2.2. Gravando dados estruturados com `json`

Strings podem ser facilmente gravadas e lidas em um arquivo. Números dão um pouco mais de trabalho, já que o método **`read()`** só retorna strings, que terão que ser passadas para uma função como **`int()`**, que pega uma string como `'123'` e retorna seu valor numérico 123. Quando você deseja salvar tipos de dados mais complexos, como listas e dicionários aninhados, a análise e serialização manual tornam-se complicadas.

Ao invés de ter usuários constantemente escrevendo e depurando código para gravar tipos complicados de dados em arquivos, o Python permite que se use o popular formato de troca de dados chamado JSON (JavaScript Object Notation) [https://json.org]. O módulo padrão chamado **`json`** pode pegar hierarquias de dados em Python e convertê-las em representações de strings; esse processo é chamado *serialização*. Reconstruir os dados estruturados da representação string é chamado *desserialização*. Entre serializar e desserializar, a string que representa o objeto pode ser armazenada em um arquivo, ou estrutura de dados, ou enviada por uma conexão de rede para alguma outra máquina.

Nota

O formato JSON é comumente usado por aplicativos modernos para permitir troca de dados. Pessoas que programam já estão familiarizadas com esse formato, o que o torna uma boa opção para interoperabilidade.

Um objeto `x`, pode ser visualizado na sua representação JSON com uma simples linha de código:

```
>>> import json
>>> x = [1, 'lista', 'simples']
>>> json.dumps(x)
'[1, "lista", "simples"]'
```

Outra variação da função **`dumps()`**, chamada **`dump()`**, serializa o objeto para um arquivo texto. Se `f` é um arquivo texto aberto para escrita, podemos fazer isto:

```
json.dump(x, f)
```

Para decodificar o objeto novamente, se `f` é um objeto arquivo binário ou arquivo texto que foi aberto para leitura:

```
x = json.load(f)
```

Nota

Arquivos JSON devem ser codificados em UTF-8, a codificação padrão para arquivos texto.

Essa técnica de serialização simples pode manipular listas e dicionários, mas a serialização de instâncias de classes arbitrárias no JSON requer um pouco mais de esforço. A referência para o módulo **`json`** contém uma explicação disso.

Ver também

O módulo **`pickle`**

Ao contrário do JSON, *pickle* é um protocolo que permite a serialização de objetos Python arbitrariamente complexos. Por isso, é específico do Python e não pode ser usado para se comunicar com aplicativos escritos em outros linguagens. Também é inseguro por padrão: desserializar dados

de pickle, provenientes de uma fonte não confiável, pode executar código arbitrário, se os dados foram criados por um invasor habilidoso.

8. Erros e exceções

Até agora mensagens de erro foram apenas mencionadas, mas se você testou os exemplos, talvez tenha esbarrado em algumas. Existem pelo menos dois tipos distintos de erros: *erros de sintaxe* e *exceções*.

8.1. Erros de sintaxe

Erros de sintaxe, também conhecidos como erros de parse, são provavelmente os mais frequentes entre aqueles que ainda estão aprendendo Python:

```
>>> while True print('Olá mundo')
  File "<stdin>", line 1
    while True print('Olá mundo')
               ^^^^^
SyntaxError: invalid syntax
```

O analisador sintático repete a linha ofensiva e exibe pequenas setas apontando para o local onde o erro foi detectado. Note que esse nem sempre é o local que precisa ser corrigido. No exemplo, o erro é detectado na função `print()`, já que dois pontos (`':'`) estão faltando logo antes dele.

O nome do arquivo (`<stdin>` no nosso exemplo) e o número da linha são impressos para que você saiba onde procurar caso a entrada tenha vindo de um arquivo.

8.2. Exceções

Mesmo que um comando ou expressão estejam sintaticamente corretos, talvez ocorra um erro na hora de sua execução. Erros detectados durante a execução são chamados *exceções* e não são necessariamente fatais: logo veremos como tratá-las em programas Python. A maioria das exceções não são tratadas pelos programas e acabam resultando em mensagens de erro:

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    10 * (1/0)
          ~^~
ZeroDivisionError: division by zero
>>> 4 + spam*3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    4 + spam*3
        ^^^^
NameError: name 'spam' is not defined
>>> '2' + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    '2' + 2
    ~~~~^~~
TypeError: can only concatenate str (not "int") to str
```

A última linha da mensagem de erro indica o que aconteceu. Exceções surgem com diferentes tipos, e o tipo é exibido como parte da mensagem: os tipos no exemplo são `ZeroDivisionError`, `NameError` e `TypeError`. A string exibida como sendo o tipo da exceção é o nome da exceção embutida que ocorreu. Isso é verdade para todas exceções pré-definidas em Python, mas não é necessariamente verdade para exceções definidas pelo usuário (embora seja uma convenção útil). Os nomes das exceções padrões são identificadores embutidos (não palavras reservadas).

O resto da linha é um detalhamento que depende do tipo da exceção ocorrida e sua causa.

A parte anterior da mensagem de erro apresenta o contexto onde ocorreu a exceção. Essa informação é denominada *stack traceback* (situação da pilha de execução). Em geral, contém uma lista de linhas do código-fonte, sem apresentar, no entanto, linhas lidas da entrada padrão.

Exceções embutidas lista as exceções pré-definidas e seus significados.

8.3. Tratamento de exceções

É possível escrever programas que tratam exceções específicas. Observe o exemplo seguinte, que pede dados ao usuário até que um inteiro válido seja fornecido, ainda permitindo que o programa seja interrompido (utilizando `Control-C` ou seja lá o que o sistema operacional suporte); note que uma interrupção gerada pelo usuário será sinalizada pela exceção **`KeyboardInterrupt`**.

```
>>> while True:
...     try:
...         x = int(input("Insira um número: "))
...         break
...     except ValueError:
...         print("Ops! Esse não é um número válido. Tente novam
...
```

A instrução **`try`** funciona da seguinte maneira:

- Primeiramente, a *cláusula try* (o conjunto de instruções entre as palavras reservadas **`try`** e **`except`**) é executada.
- Se nenhuma exceção ocorrer, a *cláusula except* é ignorada e a execução da instrução **`try`** é finalizada.
- Se ocorrer uma exceção durante a execução de uma cláusula **`try`**, as instruções remanescentes nela são ignoradas. Se o tipo da exceção ocorrida tiver sido previsto em algum **`except`**, essa *cláusula except* é executada, e então depois a execução continua após o bloco try/except.
- Se a exceção levantada não corresponder a nenhuma exceção listada na *cláusula de exceção*, ela é encaminhada a uma instrução **`try`** mais externa. Se não houver nenhum tratador previsto para essa exceção, trata-se de uma *exceção não tratada* e a execução do programa termina com uma mensagem de erro.

A instrução **`try`** pode ter uma ou mais *cláusulas de exceção*, para especificar múltiplos tratadores para diferentes exceções. No máximo um único tratador será executado. Tratadores só são sensíveis às exceções levantadas no interior da *cláusula de tentativa*, e não às que tenham ocorrido no interior de outro tratador numa mesma instrução **`try`**. Uma *cláusula de exceção* pode ser sensível a múltiplas exceções, por exemplo:

```
... except RuntimeError, TypeError, NameError:
...     pass
```

Uma classe em uma cláusula **`except`** corresponde a exceções que são instâncias da própria classe ou de uma de suas classes derivadas (mas o contrário não é válido — uma *cláusula except* listando uma classe derivada não corresponde a instâncias de suas classes base). Por exemplo, o seguinte código irá mostrar B, C, D nessa ordem:

```
class B(Exception):
    pass

class C(B):
    pass

class D(C):
    pass

for cls in [B, C, D]:
    try:
        raise cls()
    except D:
        print("D")
    except C:
        print("C")
    except B:
        print("B")
```

Se a ordem das *cláusulas de exceção* fosse invertida (`except B` no início), seria exibido B, B, B — somente a primeira *cláusula de exceção* compatível é ativada.

Quando uma exceção ocorre, ela pode estar associada a valores chamados *argumentos* da exceção. A presença e os tipos dos argumentos dependem do tipo da exceção.

A *cláusula except* pode especificar uma variável após o nome da exceção. A variável está vinculada à instância de exceção que normalmente possui um atributo `args` que armazena os argumentos. Por conveniência, os tipos

de exceção embutidos definem `__str__()` para exibir todos os argumentos sem acessar explicitamente `.args`.

```
>>> try:
...     raise Exception('spam', 'ovos')
... except Exception as inst:
...     print(type(inst))    # o tipo da exceção
...     print(inst.args)     # argumentos armazenados em .args
...     print(inst)          # __str__ permite imprimir args dir
...                          # mas pode ser substituído em subcl
...     x, y = inst.args     # desempacota args
...     print('x =', x)
...     print('y =', y)
...
<class 'Exception'>
('spam', 'ovos')
('spam', 'ovos')
x = spam
y = ovos
```

A saída `__str__()` da exceção é exibida como a última parte ("detalhe") da mensagem para exceções não tratadas.

`BaseException` é a classe base comum de todas as exceções. Uma de suas subclasses, `Exception`, é a classe base de todas as exceções não fatais. Exceções que não são subclasses de **`Exception`** normalmente não são tratadas, pois são usadas para indicar que o programa deve terminar. Elas incluem `SystemExit`, levantada por `sys.exit()`, e `KeyboardInterrupt`, levantada quando um usuário deseja interromper o programa.

`Exception` pode ser usada como um curinga que captura (quase) tudo. No entanto, é uma boa prática ser o mais específico possível quanto aos tipos de exceções que pretendemos manipular, evitando que quaisquer exceções inesperadas se propaguem.

O padrão mais comum para lidar com `Exception` é imprimir ou registrar a exceção e então levantá-la novamente (permitindo que um chamador lide com a exceção também):

```
import sys

try:
    f = open('meuarquivo.txt')
    s = f.readline()
    i = int(s.strip())
except OSError as err:
    print("Erro de E/S:", err)
except ValueError:
    print("Não foi possível converter dados para um inteiro.")
except Exception as err:
    print(f"Não esperava {err=}, {type(err)=}")
    raise
```

A construção **try** … **except** possui uma *cláusula else* opcional que, quando presente, deve ser colocada após todas as demais cláusulas. É útil para um código que precisa ser executado se nenhuma exceção foi levantada. Por exemplo:

```
for arg in sys.argv[1:]:
    try:
        f = open(arg, 'r')
    except OSError:
        print('não foi possível abrir', arg)
    else:
        print(arg, 'tem', len(f.readlines()), 'linhas')
        f.close()
```

É melhor usar a cláusula **else** do que colocar código adicional na cláusula **try**, porque ela evita que, acidentalmente, seja tratada uma exceção que não foi levantada pelo código protegido pela construção com as instruções **try** … **except**.

Os manipuladores de exceção não tratam apenas exceções que ocorrem imediatamente na *cláusula try*, mas também as que ocorrem dentro de funções chamadas (mesmo indiretamente) na *cláusula try*. Por exemplo:

```
>>> def this_fails():
...     x = 1/0
...
>>> try:
```

```
...     isso_aqui_falha()
... except ZeroDivisionError as err:
...     print('Tratando o erro de tempo de execução:', err)
...
Tratando o erro de tempo de execução: division by zero
```

8.4. Levantando exceções

A instrução `raise` permite ao programador forçar a ocorrência de um determinado tipo de exceção. Por exemplo:

```
>>> raise NameError('Olá')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    raise NameError('Olá')
NameError: Olá
```

O argumento de `raise` indica a exceção a ser levantada. Esse argumento deve ser uma instância de exceção ou uma classe de exceção (uma classe que deriva de `BaseException`, tal como `Exception` ou uma de suas subclasses). Se uma classe de exceção for passada, será implicitamente instanciada invocando o seu construtor sem argumentos:

```
raise ValueError  # abreviação para 'raise ValueError()'
```

Caso você precise determinar se uma exceção foi levantada ou não, mas não quer manipular o erro, uma forma simples de instrução `raise` permite que você levante-a novamente:

```
>>> try:
...     raise NameError('Olá')
... except NameError:
...     print('Uma exceção passou por aqui!')
...     raise
...
Uma exceção passou por aqui!
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    raise NameError('Olá')
NameError: Olá
```

8.5. Encadeamento de exceções

Se uma exceção não tratada ocorrer dentro de uma seção **except**, ela terá a exceção sendo tratada anexada a ela e incluída na mensagem de erro:

```
>>> try:
...     open("database.sqlite")
... except OSError:
...     raise RuntimeError("unable to handle error")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    open("database.sqlite")
    ~~~~^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory: 'databas

During handling of the above exception, another exception occurr

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError("unable to handle error")
RuntimeError: unable to handle error
```

Para indicar que uma exceção é uma consequência direta de outra, a instrução **raise** permite uma cláusula opcional **from**:

```
# exc deve ser uma instância de exceção ou None.
raise RuntimeError from exc
```

Isso pode ser útil quando você está transformando exceções. Por exemplo:

```
>>> def func():
...     raise ConnectionError
...
>>> try:
...     func()
... except ConnectionError as exc:
...     raise RuntimeError('Failed to open database') from exc
...
Traceback (most recent call last):
```

```
  File "<stdin>", line 2, in <module>
    func()
    ~~~~^^
  File "<stdin>", line 2, in func
ConnectionError

The above exception was the direct cause of the following except

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError('Failed to open database') from exc
RuntimeError: Failed to open database
```

Ele também permite desabilitar o encadeamento automático de exceções usando o idioma `from None`:

```
>>> try:
...     open('bancodedados.sqlite')
... except OSError:
...     raise RuntimeError from None
...
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError from None
RuntimeError
```

Para mais informações sobre os mecanismos de encadeamento, veja Exceções embutidas.

8.6. Exceções definidas pelo usuário

Programas podem definir novos tipos de exceções, através da criação de uma nova classe (veja Classes para mais informações sobre classes Python). Exceções devem ser derivadas da classe **`Exception`**, direta ou indiretamente.

As classes de exceção podem ser definidas para fazer qualquer coisa que qualquer outra classe pode fazer, mas geralmente são mantidas simples,

geralmente oferecendo apenas um número de atributos que permitem que informações sobre o erro sejam extraídas por manipuladores para a exceção.

É comum que novas exceções sejam definidas com nomes terminando em “Error”, semelhante a muitas exceções embutidas.

Muitos módulos padrão definem suas próprias exceções para relatar erros que podem ocorrer nas funções que definem.

8.7. Definindo ações de limpeza

A instrução `try` possui outra cláusula opcional, cuja finalidade é permitir a implementação de ações de limpeza, que sempre devem ser executadas independentemente da ocorrência de exceções. Como no exemplo:

```
>>> try:
...     raise KeyboardInterrupt
... finally:
...     print('Adeus, mundo!')
...
Adeus, mundo!
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    raise KeyboardInterrupt
KeyboardInterrupt
```

Se uma cláusula `finally` estiver presente, a cláusula **`finally`** será executada como a última tarefa antes da conclusão da instrução `try`. A cláusula **`finally`** executa se a instrução **`try`** produz uma exceção. Os pontos a seguir discutem casos mais complexos quando ocorre uma exceção:

- Se ocorrer uma exceção durante a execução da cláusula **`try`**, a exceção poderá ser tratada por uma cláusula `except`. Se a exceção não for tratada por uma cláusula **`except`**, a exceção será gerada novamente após a execução da cláusula **`finally`**.
- Uma exceção pode ocorrer durante a execução de uma cláusula **`except`** ou **`else`**. Novamente, a exceção é re-levantada depois que **`finally`** é executada.

- Se a cláusula **`finally`** executar uma instrução **`break`**, **`continue`** ou **`return`**, as exceções não serão levantadas novamente. Isso pode ser confuso e, portanto, não é recomendado. A partir da versão 3.14, o compilador emite um **`SyntaxWarning`** para isso (consulte **PEP 765** [https://peps.python.org/pep-0765/]).
- Se a instrução **`try`** atingir uma instrução **`break`**, **`continue`** ou **`return`**, a cláusula **`finally`** será executada imediatamente antes da execução da instrução **`break`**, **`continue`** ou **`return`**.
- Se uma cláusula **`finally`** incluir uma instrução **`return`**, o valor retornado será aquele da instrução **`return`** da cláusula **`finally`**, não o valor da instrução **`return`** da cláusula **`try`**. A partir da versão 3.14, o compilador emite um **`SyntaxWarning`** para ela (veja **PEP 765** [https://peps.python.org/pep-0765/]).

Por exemplo:

```
>>> def retorna_booleano():
...     try:
...         return True
...     finally:
...         return False
...
>>> retorna_booleano()
False
```

Um exemplo mais complicado:

```
>>> def divide(x, y):
...     try:
...         result = x / y
...     except ZeroDivisionError:
...         print("division by zero!")
...     else:
...         print("result is", result)
...     finally:
...         print("executing finally clause")
...
>>> divide(2, 1)
result is 2.0
executing finally clause
```

```
>>> divide(2, 0)
division by zero!
executing finally clause
>>> divide("2", "1")
executing finally clause
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    divide("2", "1")
    ~~~~~~^^^^^^^^^^
  File "<stdin>", line 3, in divide
    result = x / y
             ~~^~~
TypeError: unsupported operand type(s) for /: 'str' and 'str'
```

Como você pode ver, a cláusula `finally` é executada em todos os casos. A exceção `TypeError` levantada pela divisão de duas strings não é tratada pela cláusula `except` e portanto é re-levantada depois que a cláusula **`finally`** é executada.

Em aplicação do mundo real, a cláusula `finally` é útil para liberar recursos externos (como arquivos ou conexões de rede), independentemente do uso do recurso ter sido bem sucedido ou não.

8.8. Ações de limpeza predefinidas

Alguns objetos definem ações de limpeza padrões para serem executadas quando o objeto não é mais necessário, independentemente da operação que estava usando o objeto ter sido ou não bem sucedida. Veja o exemplo a seguir, que tenta abrir um arquivo e exibir seu conteúdo na tela.

```
for linha in open("meuarquivo.txt"):
    print(linha, end="")
```

O problema com esse código é que ele deixa o arquivo aberto um período indeterminado depois que o código é executado. Isso não chega a ser problema em scripts simples, mas pode ser um problema para grandes aplicações. A palavra reservada `with` permite que objetos como arquivos sejam utilizados com a certeza de que sempre serão prontamente e corretamente finalizados.

```
with open("meuarquivo.txt") as f:
    for linha in f:
        print(linha, end="")
```

Depois que a instrução é executada, o arquivo *f* é sempre fechado, mesmo se ocorrer um problema durante o processamento das linhas. Outros objetos que, como arquivos, fornecem ações de limpeza predefinidas as indicarão em suas documentações.

8.9. Criando e tratando várias exceções não relacionadas

Existem situações em que é necessário relatar várias exceções que ocorreram. Isso geralmente ocorre em estruturas de simultaneidade, quando várias tarefas podem ter falhado em paralelo, mas também há outros casos de uso em que é desejável continuar a execução e coletar vários erros em vez de levantar a primeira exceção.

O `ExceptionGroup` integrado envolve uma lista de instâncias de exceção para que elas possam ser levantadas juntas. É uma exceção em si, portanto, pode ser capturada como qualquer outra exceção.

```
>>> def f():
...     excs = [OSError('error 1'), SystemError('error 2')]
...     raise ExceptionGroup('houve problemas', excs)
...
>>> f()
  + Exception Group Traceback (most recent call last):
  |   File "<stdin>", line 1, in <module>
  |     f()
  |     ~^^
  |   File "<stdin>", line 3, in f
  | ExceptionGroup: houve problemas (2 sub-exceptions)
  +-+---------------- 1 ----------------
    | OSError: error 1
    +---------------- 2 ----------------
    | SystemError: error 2
    +------------------------------------
>>> try:
```

```
...     f()
... except Exception as e:
...     print(f'capturada {type(e)}: {e}')
...
caught <class 'ExceptionGroup'>: houve problemas (2 sub-exceptic
>>>
```

Usando `except*` em vez de `except`, podemos manipular seletivamente apenas as exceções no grupo que correspondem a um determinado tipo. No exemplo a seguir, que mostra um grupo de exceção aninhado, cada cláusula `except*` extrai do grupo exceções de um certo tipo enquanto permite que todas as outras exceções se propaguem para outras cláusulas e eventualmente sejam levantadas novamente.

```
>>> def f():
...     raise ExceptionGroup(
...         "group1",
...         [
...             OSError(1),
...             SystemError(2),
...             ExceptionGroup(
...                 "group2",
...                 [
...                     OSError(3),
...                     RecursionError(4)
...                 ]
...             )
...         ]
...     )
...
>>> try:
...     f()
... except* OSError as e:
...     print("Houve OSErrors")
... except* SystemError as e:
...     print("Houve SystemErrors")
...
Houve OSErrors
Houve SystemErrors
  + Exception Group Traceback (most recent call last):
  |   File "<stdin>", line 2, in <module>
```

```
  |     f()
  |     ~^^
  |   File "<stdin>", line 2, in f
  |     raise ExceptionGroup(
  |     ...<12 lines>...
  |     )
  | ExceptionGroup: group1
  +-+---------------- 1 ----------------
    | ExceptionGroup: group2
    +-+---------------- 1 ----------------
      | RecursionError: 4
      +------------------------------------
>>>
```

Observe que as exceções aninhadas em um grupo de exceções devem ser instâncias, não tipos. Isso ocorre porque, na prática, as exceções normalmente seriam aquelas que já foram levantadas e capturadas pelo programa, seguindo o seguinte padrão:

```
>>> excs = []
... for test in tests:
...     try:
...         test.run()
...     except Exception as e:
...         excs.append(e)
...
>>> if excs:
...    raise ExceptionGroup("Falhas no teste", excs)
...
```

8.10. Enriquecendo exceções com notas

Quando uma exceção é criada para ser levantada, geralmente é inicializada com informações que descrevem o erro ocorrido. Há casos em que é útil adicionar informações após a captura da exceção. Para este propósito, as exceções possuem um método `add_note(note)` que aceita uma string e a adiciona à lista de notas da exceção. A renderização de traceback padrão inclui todas as notas, na ordem em que foram adicionadas, após a exceção.

```
>>> try:
...     raise TypeError('tipo inválido')
... except Exception as e:
...     e.add_note('Adiciona algumas informações')
...     e.add_note('Adiciona mais algumas informações')
...     raise
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    raise TypeError('tipo inválido')
TypeError: tipo inválido
Adiciona algumas informações
Adiciona mais algumas informações
>>>
```

Por exemplo, ao coletar exceções em um grupo de exceções, podemos querer adicionar informações de contexto para os erros individuais. A seguir, cada exceção no grupo tem uma nota indicando quando esse erro ocorreu.

```
>>> def f():
...     raise OSError('operação falhou')
...
>>> excs = []
>>> for i in range(3):
...     try:
...         f()
...     except Exception as e:
...         e.add_note(f'Aconteceu na iteração {i+1}')
...         excs.append(e)
...
>>> raise ExceptionGroup('Nós temos alguns problemas', excs)
  + Exception Group Traceback (most recent call last):
  |   File "<stdin>", line 1, in <module>
  |     raise ExceptionGroup('Nós temos alguns problemas', excs)
  | ExceptionGroup: Nós temos alguns problemas (3 sub-exceptions
  +-+---------------- 1 ----------------
    | Traceback (most recent call last):
    |   File "<stdin>", line 3, in <module>
    |     f()
    |     ~^^
    |   File "<stdin>", line 2, in f
```

```
    |     raise OSError('operação falhou')
    | OSError: operação falhou
    | Aconteceu na iteração 1
    +---------------- 2 ----------------
    | Traceback (most recent call last):
    |   File "<stdin>", line 3, in <module>
    |     f()
    |     ~^^
    |   File "<stdin>", line 2, in f
    |     raise OSError('operação falhou')
    | OSError: operação falhou
    | Aconteceu na iteração 2
    +---------------- 3 ----------------
    | Traceback (most recent call last):
    |   File "<stdin>", line 3, in <module>
    |     f()
    |     ~^^
    |   File "<stdin>", line 2, in f
    |     raise OSError('operação falhou')
    | OSError: operação falhou
    | Aconteceu na iteração 3
    +------------------------------------
>>>
```

9. Classes

Classes proporcionam uma forma de organizar dados e funcionalidades juntos. Criar uma nova classe cria um novo “tipo” de objeto, permitindo que novas “instâncias” desse tipo sejam produzidas. Cada instância da classe pode ter atributos anexados a ela, para manter seu estado. Instâncias da classe também podem ter métodos (definidos pela classe) para modificar seu estado.

Em comparação com outras linguagens de programação, o mecanismo de classes de Python introduz a programação orientada a objetos sem acrescentar muitas novidades de sintaxe ou semântica. É uma mistura de mecanismos equivalentes encontrados em C++ e Modula-3. As classes em Python oferecem todas as características tradicionais da programação orientada a objetos: o mecanismo de herança permite múltiplas classes base (herança múltipla), uma classe derivada pode sobrescrever quaisquer métodos de uma classe ancestral, e um método pode invocar outro método homônimo de uma classe ancestral. Objetos podem armazenar uma quantidade arbitrária de dados de qualquer tipo. Assim como acontece com os módulos, as classes fazem parte da natureza dinâmica de Python: são criadas em tempo de execução, e podem ser alteradas após sua criação.

Usando a terminologia de C++, todos os membros de uma classe (incluindo dados) são *públicos* (veja exceção abaixo Variáveis privadas), e todos as funções membro são *virtuais*. Como em Modula-3, não existem atalhos para referenciar membros do objeto de dentro dos seus métodos: o método (função definida em uma classe) é declarado com um primeiro argumento explícito representando o objeto (instância da classe), que é fornecido implicitamente pela chamada ao método. Como em Smalltalk, classes são objetos. Isso fornece uma semântica para importar e renomear. Ao contrário de C++ ou Modula-3, tipos pré-definidos podem ser utilizados como classes base para extensões por herança pelo usuário. Também, como em C++, a maioria dos operadores (aritméticos, indexação, etc) podem ser redefinidos por instâncias de classe.

(Na falta de uma terminologia universalmente aceita para falar sobre classes, ocasionalmente farei uso de termos comuns em Smalltalk ou C++. Eu usaria termos de Modula-3, já que sua semântica de orientação a objetos é mais próxima da de Python, mas creio que poucos leitores já ouviram falar dessa linguagem.)

9.1. Uma palavra sobre nomes e objetos

Objetos têm individualidade, e vários nomes (em diferentes escopos) podem ser vinculados a um mesmo objeto. Isso é chamado de apelidamento em outras linguagens. Geralmente, esta característica não é muito apreciada, e pode ser ignorada com segurança ao lidar com tipos imutáveis (números, strings, tuplas). Entretanto, apelidamento pode ter um efeito surpreendente na semântica do código Python envolvendo objetos mutáveis como listas, dicionários e a maioria dos outros tipos. Isso pode ser usado em benefício do programa, porque os apelidos funcionam de certa forma como ponteiros. Por exemplo, passar um objeto como argumento é barato, pois só um ponteiro é passado na implementação; e se uma função modifica um objeto passado como argumento, o invocador verá a mudança — isso elimina a necessidade de ter dois mecanismos de passagem de parâmetros como em Pascal.

9.2. Escopos e espaços de nomes do Python

Antes de introduzir classes, é preciso falar das regras de escopo em Python. Definições de classe fazem alguns truques com espaços de nomes. Portanto, primeiro é preciso entender claramente como escopos e espaços de nomes funcionam, para entender o que está acontecendo. Esse conhecimento é muito útil para qualquer programador Python avançado.

Vamos começar com algumas definições.

Um *espaço de nomes* é um mapeamento que associa nomes a objetos. Atualmente, são implementados como dicionários em Python, mas isso não é perceptível (a não ser pelo desempenho), e pode mudar no futuro. Exemplos de espaços de nomes são: o conjunto de nomes pré-definidos

(funções como **abs()** e as exceções pré-definidas); nomes globais em um módulo; e nomes locais na invocação de uma função. De certa forma, os atributos de um objeto também formam um espaço de nomes. O mais importante é saber que não existe nenhuma relação entre nomes em espaços de nomes distintos. Por exemplo, dois módulos podem definir uma função de nome `maximize` sem confusão — usuários dos módulos devem prefixar a função com o nome do módulo, para evitar colisão.

A propósito, utilizo a palavra *atributo* para qualquer nome depois de um ponto. Na expressão `z.real`, por exemplo, `real` é um atributo do objeto `z`. Estritamente falando, referências para nomes em módulos são atributos: na expressão `modname.funcname`, `modname` é um objeto módulo e `funcname` é um de seus atributos. Neste caso, existe um mapeamento direto entre os atributos de um módulo e os nomes globais definidos no módulo: eles compartilham o mesmo espaço de nomes! [[1]]

Atributos podem ser somente leitura ou para leitura e escrita. No segundo caso, é possível atribuir um novo valor ao atributo. Atributos de módulos são passíveis de atribuição: você pode escrever `modname.the_answer = 42`. Atributos que aceitam escrita também podem ser apagados através da instrução **del**. Por exemplo, `del modname.the_answer` removerá o atributo **`the_answer`** do objeto referenciado por `modname`.

Espaços de nomes são criados em momentos diferentes e possuem diferentes ciclos de vida. O espaço de nomes que contém os nomes embutidos é criado quando o interpretador inicializa e nunca é removido. O espaço de nomes global de um módulo é criado quando a definição do módulo é lida, e normalmente duram até a terminação do interpretador. Os comandos executados pela invocação do interpretador, pela leitura de um script com programa principal, ou interativamente, são parte do módulo chamado **__main__**, e portanto possuem seu próprio espaço de nomes. (Os nomes embutidos possuem seu próprio espaço de nomes no módulo chamado **builtins**.).

O espaço de nomes local de uma função é criado quando a função é invocada, e apagado quando a função retorna ou levanta uma exceção que não é tratada na própria função. (Na verdade, uma forma melhor de descrever o que realmente acontece é que o espaço de nomes local é

“esquecido” quando a função termina.) Naturalmente, cada invocação recursiva de uma função tem seu próprio espaço de nomes.

Um *escopo* é uma região textual de um programa Python onde um espaço de nomes é diretamente acessível. Aqui, “diretamente acessível” significa que uma referência sem um prefixo qualificador permite o acesso ao nome.

Ainda que escopos sejam determinados estaticamente, eles são usados dinamicamente. A qualquer momento durante a execução, existem 3 ou 4 escopos aninhados cujos espaços de nomes são diretamente acessíveis:

- o escopo mais interno, que é acessado primeiro, contem os nomes locais
- os escopos das funções que envolvem a função atual, que são acessados a partir do escopo mais próximo, contêm nomes não-locais, mas também não-globais
- o penúltimo escopo contém os nomes globais do módulo atual
- e o escopo mais externo (acessado por último) contém os nomes das funções embutidas e demais objetos pré-definidos do interpretador

Se um nome é declarado no escopo global, então todas as referências e atribuições de valores vão diretamente para o penúltimo escopo, que contém os nomes globais do módulo. Para alterar variáveis declaradas fora do escopo mais interno, a instrução `nonlocal` pode ser usada; caso contrário, todas essas variáveis serão apenas para leitura (a tentativa de atribuir valores a essas variáveis simplesmente criará uma *nova* variável local, no escopo interno, não alterando nada na variável de nome idêntico fora dele).

Normalmente, o escopo local referencia os nomes locais da função corrente no texto do programa. Fora de funções, o escopo local referencia os nomes do escopo global: espaço de nomes do módulo. Definições de classes adicionam um outro espaço de nomes ao escopo local.

É importante perceber que escopos são determinados estaticamente, pelo texto do código-fonte: o escopo global de uma função definida em um módulo é o espaço de nomes deste módulo, sem importar de onde ou por qual apelido a função é invocada. Por outro lado, a busca de nomes é

dinâmica, ocorrendo durante a execução. Porém, a evolução da linguagem está caminhando para uma resolução de nomes estática, em “tempo de compilação”, portanto não conte com a resolução dinâmica de nomes! (De fato, variáveis locais já são resolvidas estaticamente.)

Uma peculiaridade especial do Python é que – se nenhuma instrução **`global`** ou **`nonlocal`** estiver em vigor – as atribuições de nomes sempre entram no escopo mais interno. As atribuições não copiam dados — elas apenas vinculam nomes aos objetos. O mesmo vale para exclusões: a instrução `del x` remove a ligação de `x` do espaço de nomes referenciado pelo escopo local. De fato, todas as operações que introduzem novos nomes usam o escopo local: em particular, instruções **`import`** e definições de funções ligam o módulo ou o nome da função no escopo local.

A instrução **`global`** pode ser usada para indicar que certas variáveis residem no escopo global ao invés do local; a instrução **`nonlocal`** indica que variáveis particulares estão em um espoco mais interno e devem ser recuperadas lá.

9.2.1. Exemplo de escopos e espaço de nomes

Este é um exemplo que demonstra como se referir aos diferentes escopos e aos espaços de nomes, e como **`global`** e **`nonlocal`** pode afetar ligação entre as variáveis:

```
def teste_de_escopo():
    def faz_local():
        spam = "spam local"

    def faz_nonlocal():
        nonlocal spam
        spam = "spam não-local"

    def faz_global():
        global spam
        spam = "spam global"

    spam = "spam teste"
    faz_local()
```

```
    print("Após atribuição local:", spam)
    faz_nonlocal()
    print("Após atribuição não-local:", spam)
    faz_global()
    print("Após atribuição global:", spam)

teste_de_escopo()
print("No escopo global:", spam)
```

A saída do código de exemplo é:

```
Após atribuição local: spam teste
Após atribuição não-local: spam não-local
Após atribuição global: spam não-local
No escopo global: spam global
```

Observe como uma atribuição *local* (que é o padrão) não altera o vínculo de *teste_de_escopo* a *spam*. A instrução `nonlocal` mudou o vínculo de *teste_de_escopo* de *spam* e a atribuição `global` alterou a ligação para o nível do módulo.

Você também pode ver que não havia nenhuma ligação anterior para *spam* antes da atribuição `global`.

9.3. Uma primeira olhada nas classes

Classes introduzem novidades sintáticas, três novos tipos de objetos, e também alguma semântica nova.

9.3.1. Sintaxe da definição de classe

A forma mais simples de definir uma classe é:

```
class NomeClasse:
    <instrução-1>
    .
    .
    .
    <instrução-N>
```

Definições de classe, assim como definições de função (instruções **`def`**), precisam ser executadas antes que tenham qualquer efeito. (Você pode colocar uma definição de classe dentro do teste condicional de um **`if`** ou dentro de uma função.)

Na prática, as instruções dentro da definição de classe geralmente serão definições de funções, mas outras instruções são permitidas, e às vezes são bem úteis — voltaremos a este tema depois. Definições de funções dentro da classe normalmente têm um forma peculiar de lista de argumentos, determinada pela convenção de chamada a métodos — isso também será explicado mais tarde.

Quando se inicia a definição de classe, um novo espaço de nomes é criado, e usado como escopo local — assim, todas atribuições a variáveis locais ocorrem nesse espaço de nomes. Em particular, funções definidas aqui são vinculadas a nomes nesse escopo.

Quando uma definição de classe é finalizada normalmente (até o fim), um *objeto classe* é criado. Este objeto encapsula o conteúdo do espaço de nomes criado pela definição da classe; aprenderemos mais sobre objetos classe na próxima seção. O escopo local (que estava vigente antes da definição da classe) é reativado, e o objeto classe é vinculado ao identificador da classe nesse escopo (**`NomeClasse`** no exemplo).

9.3.2. Objetos classe

Objetos classe suportam dois tipos de operações: *referências a atributos* e *instanciação*.

Referências a atributos de classe utilizam a sintaxe padrão utilizada para quaisquer referências a atributos em Python: `obj.nome`. Nomes de atributos válidos são todos os nomes presentes dentro do espaço de nomes da classe, quando o objeto classe foi criado. Portanto, se a definição de classe tem esta forma:

```
class MinhaClasse:
    """Um exemplo de classe simples"""
    i = 12345
```

```
    def f(self):
        return 'olá mundo'
```

então `MinhaClasse.i` e `MinhaClasse.f` são referências a atributo válidas, retornando, respectivamente, um inteiro e um objeto função. Atributos de classe podem receber valores, pode-se modificar o valor de `MinhaClasse.i` num atribuição. **`__doc__`** também é um atributo válido da classe, retornando a docstring associada à classe: `"Um exemplo de classe simples"`.

Para *instanciar* uma classe, usa-se a mesma sintaxe de invocar uma função. Apenas finja que o objeto classe do exemplo é uma função sem parâmetros, que devolve uma nova instância da classe. Por exemplo (presumindo a classe acima):

```
x = MinhaClasse()
```

cria uma nova *instância* da classe e atribui o objeto resultante à variável local `x`.

A operação de instanciação ("invocar" um objeto classe) cria um objeto vazio. Muitas classes preferem criar novos objetos com um estado inicial predeterminado. Para tanto, a classe pode definir um método especial chamado **`__init__()`**, assim:

```
def __init__(self):
    self.data = []
```

Quando uma classe define um método **`__init__()`**, o processo de instanciação automaticamente invoca **`__init__()`** sobre a instância recém criada. Em nosso exemplo, uma nova instância já inicializada pode ser obtida desta maneira:

```
x = MinhaClasse()
```

Naturalmente, o método **`__init__()`** pode ter parâmetros para maior flexibilidade. Neste caso, os argumentos fornecidos na invocação da classe serão passados para o método **`__init__()`**. Por exemplo,

```
>>> class Complex:
...     def __init__(self, realpart, imagpart):
...         self.r = realpart
...         self.i = imagpart
...
>>> x = Complex(3.0, -4.5)
>>> x.r, x.i
(3.0, -4.5)
```

9.3.3. Objetos instância

Agora o que podemos fazer com objetos de instância? As únicas operações compreendidas por objetos de instância são os atributos de referência. Existem duas maneiras válidas para nomear atributos: atributos de dados e métodos.

Atributos de dados correspondem a "variáveis de instância" em Smalltalk, e a "membros de dados" em C++. Atributos de dados não precisam ser declarados. Assim como variáveis locais, eles passam a existir na primeira vez em que é feita uma atribuição. Por exemplo, se `x` é uma instância da **`MinhaClasse`** criada acima, o próximo trecho de código irá exibir o valor `16`, sem deixar nenhum rastro:

```
x.contador = 1
while x.contador < 10:
    x.contador = x.contador * 2
print(x.contador)
del x.contador
```

O outro tipo de referência a um atributo de instância é um *método*. Um método é uma função que "pertence a" um objeto.

Nomes de métodos válidos de uma instância dependem de sua classe. Por definição, cada atributo de uma classe que é uma função corresponde a um método das instâncias. Em nosso exemplo, `x.f` é uma referência de método válida já que `MinhaClasse.f` é uma função, enquanto `x.i` não é, já que `MinhaClasse.i` não é uma função. Entretanto, `x.f` não é o mesmo que `MinhaClasse.f`. A referência `x.f` acessa um objeto método e a `MinhaClasse.f` acessa um objeto função.

9.3.4. Objetos método

Normalmente, um método é chamado imediatamente após ser referenciado:

```
x.f()
```

Se `x = MinhaClasse()`, como acima, o resultado da expressão acima será a string `"olá mundo"`. No entanto, não é obrigatório invocar o método imediatamente: como `x.f` é também um objeto ele pode ser atribuído a uma variável e invocado depois. Por exemplo:

```
xf = x.f
while True:
    print(xf())
```

exibirá o texto `olá mundo` até o mundo acabar.

O que ocorre precisamente quando um método é invocado? Você deve ter notado que `x.f()` foi chamado sem nenhum argumento, porém a definição da função **`f()`** especificava um argumento. O que aconteceu com esse argumento? Certamente Python levanta uma exceção quando uma função que declara um argumento é invocada sem nenhum argumento — mesmo que o argumento não seja usado no corpo da função…

Na verdade, pode-se supor a resposta: a particularidade sobre os métodos é que o objeto da instância é passado como o primeiro argumento da função. Em nosso exemplo, a chamada `x.f()` é exatamente equivalente a `MinhaClasse.f(x)`. Em geral, chamar um método com uma lista de *n* argumentos é equivalente a chamar a função correspondente com uma lista de argumentos que é criada inserindo o objeto de instância do método antes do primeiro argumento.

Em geral, os métodos funcionam da seguinte forma. Quando um atributo de uma instância, não relacionado a dados, é referenciado, a classe da instância é pesquisada. Se o nome é um atributo de classe válido que é um objeto função, referências ao objeto de instância e ao objeto função serão empacotadas em um objeto método. Quando o objeto método é chamado com uma lista de argumentos, uma nova lista de argumentos é construída a

partir do objeto de instância e da lista de argumentos, e o objeto função é chamado com essa nova lista de argumentos.

9.3.5. Variáveis de classe e instância

De forma geral, variáveis de instância são variáveis que indicam dados que são únicos a cada instância individual, e variáveis de classe são variáveis de atributos e de métodos que são comuns a todas as instâncias de uma classe:

```
class Cachorro:

    tipo = 'canino'         # variável de classe compartilhada p

    def __init__(self, nome):
        self.nome = nome    # variável de instância única para c

>>> d = Cachorro('Fido')
>>> e = Cachorro('Buddy')
>>> d.tipo                  # compartilhada por todos os cachorr
'canino'
>>> e.tipo                  # compartilhada por todos os cachorr
'canino'
>>> d.nome                  # exclusiva do d
'Fido'
>>> e.nome                  # exclusiva do e
'Buddy'
```

Como vimos em Uma palavra sobre nomes e objetos, dados compartilhados podem causar efeitos inesperados quando envolvem objetos mutáveis, como listas ou dicionários. Por exemplo, a lista *tricks* do código abaixo não deve ser usada como variável de classe, pois assim seria compartilhada por todas as instâncias de *Cachorro*:

```
class Cachorro:

    truques = []             # uso incorreto de uma variável de

    def __init__(self, nome):
        self.nome = nome
```

```
    def adicionar_truque(self, truque):
        self.truques.append(truque)

>>> d = Cachorro('Fido')
>>> e = Cachorro('Buddy')
>>> d.adicionar_truque('rolar')
>>> e.adicionar_truque('fingir de morto')
>>> d.truques                # inesperadamente compartilhado por
['rolar', 'fingir de morto']
```

Em vez disso, o modelo correto da classe deve usar uma variável de instância:

```
class Cachorro:

    def __init__(self, nome):
        self.nome = nome
        self.truques = []    # cria uma nova lista vazia para ca

    def adicionar_truque(self, truque):
        self.truques.append(truque)

>>> d = Cachorro('Fido')
>>> e = Cachorro('Buddy')
>>> d.adicionar_truque('rolar')
>>> e.adicionar_truque('fingir de morto')
>>> d.truques
['rolar']
>>> e.truques
['fingir de morto']
```

9.4. Observações aleatórias

Se um mesmo nome de atributo ocorre tanto na instância quanto na classe, a busca pelo atributo prioriza a instância:

```
>>> class Armazém:
...    propósito = 'armazenar'
...    região = 'oeste'
...
```

```
>>> a1 = Armazém()
>>> print(a1.propósito, a1.região)
armazenar oeste
>>> a2 = Armazém()
>>> a2.região = 'leste'
>>> print(a2.propósito, a2.região)
armazenar leste
```

Atributos de dados podem ser referenciados por métodos da própria instância, bem como por qualquer outro usuário do objeto (também chamados “clientes” do objeto). Em outras palavras, classes não servem para implementar tipos puramente abstratos de dados. De fato, nada em Python torna possível assegurar o encapsulamento de dados — tudo é baseado em convenção. (Por outro lado, a implementação de Python, escrita em C, pode esconder completamente detalhes de um objeto e controlar o acesso ao objeto, se necessário; isto pode ser utilizado por extensões de Python escritas em C.)

Clientes devem utilizar atributos de dados com cuidado, pois podem bagunçar invariantes assumidas pelos métodos ao esbarrar em seus atributos de dados. Note que clientes podem adicionar atributos de dados a suas próprias instâncias, sem afetar a validade dos métodos, desde que seja evitado o conflito de nomes. Novamente, uma convenção de nomenclatura poupa muita dor de cabeça.

Não existe atalho para referenciar atributos de dados (ou outros métodos!) de dentro de um método. Isso aumenta a legibilidade dos métodos: não há como confundir variáveis locais com variáveis da instância quando lemos rapidamente um método.

Frequentemente, o primeiro argumento de um método é chamado `self`. Isso não passa de uma convenção: o identificador `self` não é uma palavra reservada nem possui qualquer significado especial em Python. Mas note que, ao seguir essa convenção, seu código se torna legível por uma grande comunidade de desenvolvedores Python e é possível que alguma *IDE* dependa dessa convenção para analisar seu código.

Qualquer objeto função que é atributo de uma classe, define um método para as instâncias dessa classe. Não é necessário que a definição da função

esteja textualmente embutida na definição da classe. Atribuir um objeto função a uma variável local da classe é válido. Por exemplo:

```
# Função definida fora da classe
def f1(self, x, y):
    return min(x, x+y)

class C:
    f = f1

    def g(self):
        return 'olá mundo'

    h = g
```

Agora `f`, `g` e `h` são todos atributos da classe **`C`** que referenciam funções, e consequentemente são todos métodos de instâncias da classe **`C`**, onde `h` é exatamente equivalente a `g`. No entanto, essa prática serve apenas para confundir o leitor do programa.

Métodos podem invocar outros métodos usando atributos de método do argumento `self`:

```
class Bolsa:
    def __init__(self):
        self.data = []

    def adicionar(self, x):
        self.data.append(x)

    def adicionar_em_dobro(self, x):
        self.adicionar(x)
        self.adicionar(x)
```

Métodos podem referenciar nomes globais da mesma forma que funções comuns. O escopo global associado a um método é o módulo contendo sua definição na classe (a classe propriamente dita nunca é usada como escopo global!). Ainda que seja raro justificar o uso de dados globais em um método, há diversos usos legítimos do escopo global. Por exemplo, funções e módulos importados no escopo global podem ser usados por métodos,

bem como as funções e classes definidas no próprio escopo global. Provavelmente, a classe contendo o método em questão também foi definida neste escopo global. Na próxima seção veremos razões pelas quais um método pode querer referenciar sua própria classe.

Cada valor é um objeto e, portanto, tem uma *classe* (também chamada de *tipo*). Ela é armazenada como `object.__class__`.

9.5. Herança

Obviamente, uma característica da linguagem não seria digna do nome "classe" se não suportasse herança. A sintaxe para uma classe derivada é assim:

```
class NomeClasseDerivada(NomeClasseBase):
    <instrução-1>
    .
    .
    .
    <instrução-N>
```

O identificador **`BaseClassName`** deve estar definido em um espaço de nomes acessível do escopo que contém a definição da classe derivada. No lugar do nome da classe base, também são aceitas outras expressões. Isso é muito útil, por exemplo, quando a classe base é definida em outro módulo:

```
class NomeClasseDerivada(nomemódulo.NomeClasseBase):
```

A execução de uma definição de classe derivada procede da mesma forma que a de uma classe base. Quando o objeto classe é construído, a classe base é lembrada. Isso é utilizado para resolver referências a atributos. Se um atributo requisitado não for encontrado na classe, ele é procurado na classe base. Essa regra é aplicada recursivamente se a classe base por sua vez for derivada de outra.

Não há nada de especial sobre instanciação de classes derivadas: `NomeClasseDerivada()` cria uma nova instância da classe. Referências a métodos são resolvidas da seguinte forma: o atributo correspondente é

procurado através da cadeia de classes base, e referências a métodos são válidas se essa procura produzir um objeto função.

Classes derivadas podem sobrescrever métodos das suas classes base. Uma vez que métodos não possuem privilégios especiais quando invocam outros métodos no mesmo objeto, um método na classe base que invoca um outro método da mesma classe base pode, efetivamente, acabar invocando um método sobreposto por uma classe derivada. (Para programadores C++ isso significa que todos os métodos em Python são realmente `virtuais`.)

Um método sobrescrito em uma classe derivada, de fato, pode querer estender, em vez de simplesmente substituir, o método da classe base, de mesmo nome. Existe uma maneira simples de chamar diretamente o método da classe base: apenas chame `BaseClassName.methodname(self, arguments)`. Isso é geralmente útil para os clientes também. (Note que isto só funciona se a classe base estiver acessível como `BaseClassName` no escopo global).

Python tem duas funções embutidas que trabalham com herança:

- Use **`isinstance()`** para verificar o tipo de uma instância: `isinstance(obj, int)` será `True` somente se `obj.__class__` é a classe **`int`** ou alguma classe derivada de **`int`**.
- Use **`issubclass()`** para verificar herança entre classes: `issubclass(bool, int)` é `True` porque **`bool`** é uma subclasse de **`int`**. Porém, `issubclass(float, int)` é `False` porque **`float`** não é uma subclasse de **`int`**.

9.5.1. Herança múltipla

Python também suporta uma forma de herança múltipla. Uma definição de classe com várias classes bases tem esta forma:

```
class NomeClasseDerivada(Base1, Base2, Base3):
    <instrução-1>
    .
    .
```

```
.
<instrução-N>
```

Para a maioria dos casos mais simples, pense na pesquisa de atributos herdados de uma classe pai como o primeiro nível de profundidade, da esquerda para a direita, não pesquisando duas vezes na mesma classe em que há uma sobreposição na hierarquia. Assim, se um atributo não é encontrado em **`DerivedClassName`**, é procurado em **`Base1`**, depois, recursivamente, nas classes base de **`Base1`**, e se não for encontrado lá, é pesquisado em **`Base2`** e assim por diante.

De fato, é um pouco mais complexo que isso; a ordem de resolução de métodos muda dinamicamente para suportar chamadas cooperativas para **`super()`**. Essa abordagem é conhecida em outras linguagens de herança múltipla como chamar-o-próximo-método, e é mais poderosa que a chamada à função super, encontrada em linguagens de herança única.

A ordenação dinâmica é necessária porque todos os casos de herança múltipla exibem um ou mais relacionamentos de diamante (em que pelo menos uma das classes bases pode ser acessada por meio de vários caminhos da classe mais inferior). Por exemplo, todas as classes herdam de **`object`**, portanto, qualquer caso de herança múltipla fornece mais de um caminho para alcançar **`object`**. Para evitar que as classes base sejam acessadas mais de uma vez, o algoritmo dinâmico lineariza a ordem de pesquisa, de forma a preservar a ordenação da esquerda para a direita, especificada em cada classe, que chama cada pai apenas uma vez, e que é monotônica (significando que uma classe pode ser subclassificada sem afetar a ordem de precedência de seus pais). Juntas, essas propriedades tornam possível projetar classes confiáveis e extensíveis com herança múltipla. Para mais detalhes, veja A Ordem de Resolução de Métodos do Python 2.3.

Em alguns casos, a herança múltipla não é permitida; consulte Herança múltipla para mais detalhes.

9.6. Variáveis privadas

Variáveis de instância “privadas”, que não podem ser acessadas, exceto em métodos do próprio objeto, não existem em Python. No entanto, existe uma convenção que é seguida pela maioria dos programas em Python: um nome prefixado com um sublinhado (por exemplo: `_spam`) deve ser tratado como uma parte não-pública da API (seja uma função, um método ou um atributo de dados). Tais nomes devem ser considerados um detalhe de implementação e sujeito a alteração sem aviso prévio.

Uma vez que existe um caso de uso válido para a definição de atributos privados em classes (especificamente para evitar conflitos com nomes definidos em subclasses), existe um suporte limitado a identificadores privados em classes, chamado *desfiguração de nomes*. Qualquer identificador no formato `__spam` (pelo menos dois sublinhados no início, e no máximo um sublinhado no final) é textualmente substituído por `_classname__spam`, onde `classname` é o nome da classe atual com sublinhado(s) iniciais omitidos. Essa desfiguração independe da posição sintática do identificador, desde que ele apareça dentro da definição de uma classe.

Ver também

O especificações de desfiguração de nome privado para detalhes e casos especiais.

A desfiguração de nomes é útil para que subclasses possam sobrescrever métodos sem quebrar invocações de métodos dentro de outra classe. Por exemplo:

```
class Mapeamento:
    def __init__(self, iterável):
        self.lista_itens = []
        self.__atualizar(iterável)

    def atualizar(self, iterável):
        for item in iterável:
            self.lista_itens.append(item)

    __atualizar = atualizar   # cópia privada do método atualiza
```

```
class SubclasseMapeamento(Mapeamento):

    def update(self, chaves, valores):
        # fornece nova assinatura para atualizar(),
        # mas não quebra __init__()
        for item in zip(chaves, valores):
            self.lista_itens.append(item)
```

O exemplo acima deve funcionar mesmo se `SubclasseMapeamento` introduzisse um identificador `__atualizar` uma vez que é substituído por `_Mapeamento__atualizar` na classe `Mapeamento` e `_SubclasseMapeamento__atualizar` na classe `SubclasseMapeamento`, respectivamente.

Note que as regras de desfiguração de nomes foram projetadas para evitar acidentes; ainda é possível acessar ou modificar uma variável que é considerada privada. Isso pode ser útil em certas circunstâncias especiais, como depuração de código.

Código passado para `exec()` ou `eval()` não considera o nome da classe que invocou como sendo a classe corrente; isso é semelhante ao funcionamento da instrução `global`, cujo efeito se aplica somente ao código que é compilado junto. A mesma restrição se aplica às funções `getattr()`, `setattr()` e `delattr()`, e quando acessamos diretamente o `__dict__` da classe.

9.7. Curiosidades e conclusões

Às vezes, é útil ter um tipo semelhante ao “record” de Pascal ou ao “struct” de C, para agrupar alguns itens de dados. A maneira pythônica para este fim é usar **`dataclasses`**:

```
from dataclasses import dataclass

@dataclass
class Empregado:
    nome: str
```

```
    dept: str
    salário: int

>>> joão = Empregado('joão', 'lab de computadores', 1000)
>>> joão.dept
'lab de computadores'
>>> joão.salário
1000
```

Um trecho de código Python que espera um tipo de dado abstrato em particular, pode receber, ao invés disso, uma classe que imita os métodos que aquele tipo suporta. Por exemplo, se você tem uma função que formata dados obtidos de um objeto do tipo "arquivo", pode definir uma classe com métodos `read()` e `readline()` que obtém os dados de um "buffer de caracteres" e passar como argumento.

Métodos de instância têm atributos também: `m.__self__` é o objeto instância com o método `m()`, e `m.__func__` é o objeto função correspondente ao método.

9.8. Iteradores

Você já deve ter notado que pode usar laços `for` com a maioria das coleções em Python:

```
for elemento in [1, 2, 3]:
    print(elemento)
for elemento in (1, 2, 3):
    print(elemento)
for chave in {'um':1, 'dois':2}:
    print(chave)
for char in "123":
    print(char)
for linha in open("meuarquivo.txt"):
    print(linha, end='')
```

Esse estilo de acesso é claro, conciso e conveniente. O uso de iteradores permeia e unifica o Python. Nos bastidores, a instrução `for` chama `iter()` no objeto contêiner. A função retorna um objeto iterador que define o método `__next__()` que acessa elementos no contêiner, um de cada vez.

Quando não há mais elementos, **__next__()** levanta uma exceção **StopIteration** que informa ao **for** para terminar. Você pode chamar o método **__next__()** usando a função embutida **next()**; este exemplo mostra como tudo funciona:

```
>>> s = 'abc'
>>> it = iter(s)
>>> it
<str_iterator object at 0x10c90e650>
>>> next(it)
'a'
>>> next(it)
'b'
>>> next(it)
'c'
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    next(it)
StopIteration
```

Observando o mecanismo por trás do protocolo dos iteradores, fica fácil adicionar esse comportamento às suas classes. Defina um método **__iter__()** que retorna um objeto que tenha um método **__next__()**. Se uma classe já define **__next__()**, então **__iter__()** pode simplesmente retornar `self`:

```
class Inverter:
    """Iterador para fazer um laço em uma sequência de trás para
    def __init__(self, dados):
        self.dados = dados
        self.índice = len(dados)

    def __iter__(self):
        return self

    def __next__(self):
        if self.índice == 0:
            raise StopIteration
        self.índice = self.índice - 1
        return self.dados[self.índice]
```

```
>>> rev = Interverter('spam')
>>> iter(rev)
<__main__.Interverter object at 0x00A1DB50>
>>> for char in rev:
...     print(char)
...
m
a
p
s
```

9.9. Geradores

Geradores são uma ferramenta simples e poderosa para criar iteradores. São escritos como funções normais mas usam a instrução `yield` quando precisam retornar dados. Cada vez que `next()` é chamado, o gerador volta ao ponto onde parou (lembrando todos os valores de dados e qual instrução foi executada pela última vez). Um exemplo mostra como geradores podem ser trivialmente fáceis de criar:

```
def inverter(dados):
    for índice in range(len(dados)-1, -1, -1):
        yield dados[índice]
```

```
>>> for char in inverter('golf'):
...     print(char)
...
f
l
o
g
```

Qualquer coisa que possa ser feita com geradores também pode ser feita com iteradores baseados numa classe, como descrito na seção anterior. O que torna geradores tão compactos é que os métodos `__iter__()` e `__next__()` são criados automaticamente.

Outro ponto chave é que as variáveis locais e o estado da execução são preservados automaticamente entre as chamadas. Isto torna a função mais

fácil de escrever e muito mais clara do que uma implementação usando variáveis de instância como `self.index` e `self.data`.

Além disso, quando geradores terminam, eles levantam **`StopIteration`** automaticamente. Combinados, todos estes aspectos tornam a criação de iteradores tão fácil quanto escrever uma função normal.

9.10. Expressões geradoras

Alguns geradores simples podem ser codificados, de forma sucinta, como expressões, usando uma sintaxe semelhante a compreensões de lista, mas com parênteses em vez de colchetes. Essas expressões são projetadas para situações em que o gerador é usado imediatamente, pela função que o engloba. As expressões geradoras são mais compactas, mas menos versáteis do que as definições completas do gerador, e tendem a usar menos memória do que as compreensões de lista equivalentes.

Exemplos:

```
>>> sum(i*i for i in range(10))                 # soma dos quadr
285

>>> xvec = [10, 20, 30]
>>> yvec = [7, 5, 3]
>>> sum(x*y for x,y in zip(xvec, yvec))         # produto escala
260

>>> palavras_unicas= set(palavra for linha in pagina for palavra

>>> orador_de_formatura = max((aluno.gpa, aluno.nome) for aluno

>>> data = 'golfe'
>>> list(data[i] for i in range(len(data)-1, -1, -1))
['e', 'f', 'l', 'o', 'g']

>>> x = [[1,2,3], [], [4, 5]]
>>> g = (*i for i in x)
>>> list(g)
[1, 2, 3, 4, 5]
```

Na maioria dos casos, as expressões de geradores têm de ser envolvidas em parênteses. No entanto, como caso especial, quando fornecidas como o único argumento para uma função (como nos exemplos que envolvem `sum`, `set`, `max` e `list` acima), a expressão geradora não precisa de ser envolvida num conjunto adicional de parênteses. Ou seja, os seguintes dois trechos de código são semanticamente equivalentes:

```
>>> f(x for x in y)
>>> f((x for x in y))
```

assim como os seguintes:

```
>>> f(*x for x in y)
>>> f((*x for x in y))
```

Notas de rodapé

[1]

Exceto por uma coisa. Os objetos módulo têm um atributo secreto e somente para leitura chamado **`__dict__`** que retorna o dicionário usado para implementar o espaço de nomes do módulo; o nome `__dict__` é um atributo, mas não um nome global. Obviamente, usar isso viola a abstração da implementação do espaço de nomes, e deve ser restrito a coisas como depuradores post-mortem.

10. Um breve passeio pela biblioteca padrão

10.1. Interface com o sistema operacional

O módulo `os` fornece dúzias de funções para interagir com o sistema operacional:

```
>>> import os
>>> os.getcwd()      # Retorna o diretório de trabalho atual
'C:\\Python315'
>>> os.chdir('/server/accesslogs')   # Altera o diretório de tra
>>> os.system('mkdir today')   # Executa o comando mkdir na shel
0
```

Certifique-se de usar a forma `import os` ao invés de `from os import *`. Isso evitará que `os.open()` oculte a função `open()` que opera de forma muito diferente.

As funções embutidas `dir()` e `help()` são úteis como um sistema de ajuda interativa para lidar com módulos grandes como `os`:

```
>>> import os
>>> dir(os)
<retorna uma lista de todas as funções do módulo>
>>> help(os)
<retorna uma página de manual extensa criada a partir das docstr
```

Para tarefas de gerenciamento cotidiano de arquivos e diretórios, o módulo `shutil` fornece uma interface de nível mais alto que é mais simples de usar:

```
>>> import shutil
>>> shutil.copyfile('dados.db', 'arquivo.db')
'arquivo.db'
```

```
>>> shutil.move('/build/executáveis', 'dir_instal')
'dir_instal'
```

10.2. Caracteres curinga

O módulo **glob** fornece uma função para criar listas de arquivos a partir de buscas em diretórios usando caracteres curinga:

```
>>> import glob
>>> glob.glob('*.py')
['primos.py', 'aleatorizar.py', 'aspas.py']
```

10.3. Argumentos de linha de comando

Scripts geralmente precisam processar argumentos passados na linha de comando. Esses argumentos são armazenados como uma lista no atributo *argv* do módulo **sys**. Por exemplo, consideremos o arquivo `demo.py` a seguir:

```
# Arquivo demo.py
import sys
print(sys.argv)
```

Aqui está a saída da execução `python demo.py um dois três` na linha de comando:

```
['demo.py', 'um', 'dois, 'três']
```

O módulo **argparse** fornece um mecanismo mais sofisticado para processar argumentos de linha de comando. O script seguinte extrai e exibe um ou mais nomes de arquivos e um número de linhas opcional:

```
import argparse

parser = argparse.ArgumentParser(
    prog='topo',
    description='Mostra as primeiras linhas de cada arquivo')
parser.add_argument('arquivos, nargs='+')
parser.add_argument('-l', '--linhas', type=int, default=10)
```

```
args = parser.parse_args()
print(args)
```

Quando executada na linha de comando `python topo.py --linhas=5 alfa.txt beta.txt`, o script define `args.linhas` para `5` e `args.arquivos` para `['alfa.txt', 'beta.txt']`.

10.4. Redirecionamento de erros e encerramento do programa

O módulo **sys** também possui atributos para *stdin*, *stdout* e *stderr*. O último é usado para emitir avisos e mensagens de erros visíveis mesmo quando *stdout* foi redirecionado:

```
>>> sys.stderr.write('Aviso, arquivo log não encontrado, iniciar
Aviso, arquivo log não encontrado, iniciando um novo
```

A forma mais direta de encerrar um script é usando `sys.exit()`.

10.5. Reconhecimento de padrões em strings

O módulo **re** fornece ferramentas para lidar com processamento de strings através de expressões regulares. Para reconhecimento de padrões complexos, expressões regulares oferecem uma solução sucinta e eficiente:

```
>>> import re
>>> re.findall(r'\br[a-z]*', 'o rato roeu a roupa do rei de roma
['rato', 'roeu', 'roupa', 'rei', 'roma']
>>> re.sub(r'(\b[a-z]+) \1', r'\1', 'o gato de de chapéu')
'o gato de chapéu'
```

Quando as exigências são simples, métodos de strings são preferíveis por serem mais fáceis de ler e depurar:

```
>>> 'xá para dois'.replace('xá', 'chá')
'chá para dois'
```

10.6. Matemática

O módulo **math** oferece acesso às funções da biblioteca C para matemática de ponto flutuante:

```
>>> import math
>>> math.cos(math.pi / 4)
0.70710678118654757
>>> math.log(1024, 2)
10.0
```

O módulo **random** fornece ferramentas para gerar seleções aleatórias:

```
>>> import random
>>> random.choice(['maçã', 'pera', 'banana'])
'maçã'
>>> random.sample(range(100), 10)   # amostragem sem reposição
[30, 83, 16, 4, 8, 81, 41, 50, 18, 33]
>>> random.random()    # ponto flutuante aleatório do intervalo
0.17970987693706186
>>> random.randrange(6)    # inteiro aleatório escolhido de ranç
4
```

O módulo **statistics** calcula as propriedades estatísticas básicas (a média, a mediana, a variação, etc.) de dados numéricos:

```
>>> import statistics
>>> dados = [2.75, 1.75, 1.25, 0.25, 0.5, 1.25, 3.5]
>>> statistics.mean(dados)
1.6071428571428572
>>> statistics.median(dados)
1.25
>>> statistics.variance(dados)
1.3720238095238095
```

O projeto SciPy <https://scipy.org> tem muitos outros módulos para cálculos numéricos.

10.7. Acesso à internet

Há diversos módulos para acesso e processamento de protocolos da internet. Dois dos mais simples são **urllib.request** para efetuar download de dados a partir de URLs e **smtplib** para enviar mensagens de correio eletrônico:

```
>>> from urllib.request import urlopen
>>> with urlopen('https://docs.python.org/3/') as response:
...     for line in response:
...         line = line.decode()             # Converte bytes pa
...         if 'updated' in line:
...             print(line.rstrip())         # Remove nova linha
...
      Last updated on Nov 11, 2025 (20:11 UTC).

>>> import smtplib
>>> server = smtplib.SMTP('localhost')
>>> server.sendmail('soothsayer@example.org', 'jcaesar@example.c
... """To: jcaesar@example.org
... From: soothsayer@example.org
...
... Cuidado com os Idos de Março.
... """)
>>> server.quit()
```

(Note que o segundo exemplo precisa de um servidor de email rodando em localhost.)

10.8. Data e hora

O módulo **datetime** fornece classes para manipulação de datas e horas nas mais variadas formas. Apesar da disponibilidade de aritmética com data e hora, o foco da implementação é na extração eficiente dos membros para formatação e manipulação. O módulo também oferece objetos que levam os fusos horários em consideração.

```
>>> # datas são facilmente construídas e formatadas
>>> import datetime as dt
>>> now = dt.date.today()
>>> now
```

```
datetime.date(2003, 12, 2)
>>> now.strftime("%m-%d-%y. %d %b %Y is a %A on the %d day of %E
'12-02-03. 02 Dec 2003 is a Tuesday on the 02 day of December.'

>>> # datas têm suporte a matemática de calendário
>>> birthday = date(1964, 7, 31)
>>> age = now - birthday
>>> age.days
14368
```

10.9. Compressão de dados

Formatos comuns de arquivamento e compressão de dados estão disponíveis diretamente através de alguns módulos, entre eles: **zlib**, **gzip**, **bz2**, **lzma**, **zipfile** and **tarfile**.

```
>>> import zlib
>>> s = b'A vaca malhada foi molhada por outra vaca molhada e ma
>>> len(s)
60
>>> t = zlib.compress(s)
>>> len(t)
47
>>> zlib.decompress(t)
b'A vaca malhada foi molhada por outra vaca molhada e malhada.'
>>> zlib.crc32(s)
2444551089
```

10.10. Medição de desempenho

Alguns usuários de Python desenvolvem um interesse profundo pelo desempenho relativo de diferentes abordagens para o mesmo problema. Python oferece uma ferramenta de medição que esclarece essas dúvidas rapidamente.

Por exemplo, pode ser tentador usar o empacotamento e desempacotamento de tuplas ao invés da abordagem tradicional de permutar os argumentos. O

módulo **timeit** rapidamente mostra uma modesta vantagem de desempenho:

```
>>> from timeit import Timer
>>> Timer('t=a; a=b; b=t', 'a=1; b=2').timeit()
0.57535828626024577
>>> Timer('a,b = b,a', 'a=1; b=2').timeit()
0.54962537085770791
```

Em contraste com a granularidade fina do módulo **timeit**, os módulos **profile** e **pstats** oferecem ferramentas para identificar os trechos mais críticos em grandes blocos de código.

10.11. Controle de qualidade

Uma das abordagens usadas no desenvolvimento de software de alta qualidade é escrever testes para cada função à medida que é desenvolvida e executar esses testes frequentemente durante o processo de desenvolvimento.

O módulo **doctest** oferece uma ferramenta para realizar um trabalho de varredura e validação de testes escritos nas strings de documentação (docstrings) de um programa. A construção dos testes é tão simples quanto copiar uma chamada típica juntamente com seus resultados e colá-los na docstring. Isto aprimora a documentação, fornecendo ao usuário um exemplo real, e permite que o módulo doctest verifique se o código continua fiel à documentação:

```
def average(values):
    """Calcula a média aritmética da lista de números.

    >>> print(average([20, 30, 70]))
    40.0
    """
    return sum(values) / len(values)

import doctest
doctest.testmod()   # valida automaticamente os testes incorpora
```

O módulo `unittest` não é tão simples de usar quanto o módulo `doctest`, mas permite que um conjunto muito maior de testes seja mantido em um arquivo separado:

```
import unittest

class TestStatisticalFunctions(unittest.TestCase):

    def test_average(self):
        self.assertEqual(average([20, 30, 70]), 40.0)
        self.assertEqual(round(average([1, 5, 7]), 1), 4.3)
        with self.assertRaises(ZeroDivisionError):
            average([])
        with self.assertRaises(TypeError):
            average(20, 30, 70)

unittest.main()  # Chamar a partir da linha de comando invoca to
```

10.12. Baterias incluídas

Python tem uma filosofia de “baterias incluídas”. Isso fica mais evidente através da sofisticação e robustez dos seus maiores pacotes. Por exemplo:

- Os módulos `xmlrpc.client` e `xmlrpc.server` tornam a implementação de chamadas remotas (remote procedure calls) em uma tarefa quase trivial. Apesar dos nomes dos módulos, nenhum conhecimento direto ou manipulação de XML é necessário.
- O pacote `email` é uma biblioteca para gerenciamento de mensagens de correio eletrônico, incluindo MIME e outros baseados no **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html]. Diferente dos módulos `smtplib` e `poplib` que apenas enviam e recebem mensagens, o pacote de email tem um conjunto completo de ferramentas para construir ou decodificar a estrutura de mensagens complexas (incluindo anexos) e para implementação de protocolos de codificação e cabeçalhos.
- O pacote `json` oferece um suporte robusto para analisar este popular formato para troca de dados. O módulo `csv` oferece suporte para leitura e escrita direta em arquivos no formato Comma-Separated Value, comumente suportado por bancos de dados e planilhas. O

processamento XML é fornecido pelos pacotes `xml.etree.ElementTree`, `xml.dom` e `xml.sax`. Juntos, esses módulos e pacotes simplificam muito a troca de informações entre aplicativos Python e outras ferramentas.

- O módulo `sqlite3` é um wrapper para a biblioteca de banco de dados SQLite, fornecendo um banco de dados persistente que pode ser atualizado e acessado usando sintaxe SQL ligeiramente fora do padrão.
- Internacionalização está disponível através de diversos módulos, como `gettext`, `locale`, e o pacote `codecs`.

11. Um breve passeio pela biblioteca padrão — parte II

Este segundo passeio apresenta alguns módulos avançados que atendem necessidades de programação profissional. Estes módulos raramente aparecem em scripts pequenos.

11.1. Formatando a saída

O módulo `reprlib` fornece uma versão de `repr()` personalizado para exibições abreviadas de contêineres grandes ou profundamente aninhados:

```
>>> import reprlib
>>> reprlib.repr(set('supercalifragilisticexpialidoce'))
"{'a', 'c', 'd', 'e', 'f', 'g', ...}"
```

O módulo `pprint` oferece um controle mais sofisticado na exibição tanto de objetos embutidos quanto aqueles criados pelo usuário de maneira que fique legível para o interpretador. Quando o resultado é maior que uma linha, o "pretty printer" acrescenta quebras de linha e indentação para revelar as estruturas de maneira mais clara:

```
>>> import pprint
>>> t = [[[['preto', 'ciano'], 'branco', ['verde', 'vermelho']],
...     'amarelo'], 'azul']]]
>>>
>>> pprint.pprint(t, width=30)
[[[['preto', 'ciano'],
   'branco',
   ['verde', 'vermelho']],
  [['magenta', 'amarelo'],
   'azul']]]
```

O módulo `textwrap` formata parágrafos de texto para que caibam em uma dada largura de tela:

```
>>> import textwrap
>>> doc = """O método wrap() é como fill(), exceto que ele retor
... uma lista de strings em vez de uma string grande com quebras
... de linha para separar as linhas quebradas."""
...
>>> print(textwrap.fill(doc, width=40))
O método wrap() é como fill(), exceto
que ele retorna uma lista de strings em
vez de uma string grande com quebras de
linha para separar as linhas quebradas.
```

O módulo **locale** acessa uma base de dados de formatos específicos a determinada cultura. O atributo de agrupamento da função “format” oferece uma forma direta de formatar números com separadores de grupo:

```
>>> import locale
>>> locale.setlocale(locale.LC_ALL, 'English_United States.1252'
'English_United States.1252'
>>> conv = locale.localeconv()          # obtém mapeamento de co
>>> x = 1234567.8
>>> locale.format_string("%d", x, grouping=True)
'1,234,567'
>>> locale.format_string("%s%.*f", (conv['currency_symbol'],
...                      conv['frac_digits'], x), grouping=True)
'$1,234,567.80'
```

11.2. Usando templates

módulo **string** inclui a versátil classe **Template** com uma sintaxe simplificada, adequada para ser editada por usuários finais. Isso permite que usuários personalizem suas aplicações sem a necessidade de alterar a aplicação.

Em um template são colocadas marcações indicando o local onde o texto variável deve ser inserido. Uma marcação é formada por `$` seguido de um identificador Python válido (caracteres alfanuméricos e underscores). Envolvendo-se o identificador da marcação entre chaves, permite que ele

seja seguido por mais caracteres alfanuméricos sem a necessidade de espaços. Escrevendo-se `$$` cria-se um único `$`:

```
>>> from string import Template
>>> t = Template('O povo de ${vila} envia $$10 para $causa.')
>>> t.substitute(vila='Nottingham', causa='o fundo de recuperaçã
'O povo de Nottingham envia $10 para o fundo de recuperação.'
```

O método **`substitute()`** levanta uma exceção **`KeyError`** quando o identificador de uma marcação não é fornecido em um dicionário ou em um argumento nomeado (*keyword argument*). Para aplicações que podem receber dados incompletos fornecidos pelo usuário, o método **`safe_substitute()`** pode ser mais apropriado — deixará os marcadores intactos se os dados estiverem faltando:

```
>>> t = Template('Devolva $item para $dono.')
>>> d = dict(item='a andorinha descarregada')
>>> t.substitute(d)
Traceback (most recent call last):
  ...
KeyError: 'dono'
>>> t.safe_substitute(d)
'Devolva a andorinha descarregada para $dono.'
```

Subclasses de Template podem especificar um delimitador personalizado. Por exemplo, um utilitário para renomeação em lote de fotos pode usar o sinal de porcentagem para marcações como a data atual, número sequencial da imagem ou formato do aquivo:

```
>>> import time, os.path
>>> arquivos_fotos = ['img_1074.jpg', 'img_1076.jpg', 'img_1077.
>>> class RenomeiaEmLote(Template):
...     delimiter = '%'
...
>>> formato = input('Informe o estilo de renomeação (%d-data %n-
Informe o estilo de renomeação (%d-data %n-sequencia %f-formato)

>>> t = RenomeiaEmLote(formato)
>>> data = time.strftime('%d%b%y')
>>> for i, nome_arquivo in enumerate(arquivos_fotos):
```

```
...     base, extensão = os.path.splitext(nome_arquivo)
...     novo_nome = t.substitute(d=data, n=i, f=extensão)
...     print('{0} --> {1}'.format(nome_arquivo, novo_nome))

img_1074.jpg --> Ashley_0.jpg
img_1076.jpg --> Ashley_1.jpg
img_1077.jpg --> Ashley_2.jpg
```

Outra aplicação para templates é separar a lógica da aplicação dos detalhes de múltiplos formatos de saída. Assim é possível usar templates personalizados para gerar arquivos XML, relatórios em texto puro e relatórios web em HTML.

11.3. Trabalhando com formatos binários de dados

O módulo **struct** oferece as funções **pack()** e **unpack()** para trabalhar com registros binários de tamanho variável. O exemplo a seguir mostra como iterar através do cabeçalho de informação num aquivo ZIP sem usar o módulo **zipfile**. Os códigos de empacotamento "H" e "I" representam números sem sinal de dois e quatro bytes respectivamente. O "<" indica que os números têm tamanho padrão e são little-endian (bytes menos significativos primeiro):

```
import struct

with open('meuarquivo.zip', 'rb') as arq:
    dados = arq.read()

inicio = 0
for i in range(3):                      # mostra o cabeçalho dos
    inicio += 14
    campos = struct.unpack('<IIIHH', dados[inicio:inicio+16])
    crc32, tamanho_comprimido, tamanho_descomprimido, tamanho_no

    inicio += 16
    nome_arquivo = dados[inicio:inicio+tamanho_nome_arquivo]
    inicio += tamanho_nome_arquivo
    extra = dados[inicio:inicio+tamanho_extra]
```

```
    print(nome_arquivo, hex(crc32), tamanho_comprimido, tamanho_

    inicio += tamanho_extra + tamanho_comprimido     # pula para
```

11.4. Multi-threading

O uso de threads é uma técnica para desacoplar tarefas que não são sequencialmente dependentes. Threads podem ser usadas para melhorar o tempo de resposta de aplicações que aceitam entradas do usuário enquanto outras tarefas são executadas em segundo plano. Um caso relacionado é executar ações de entrada e saída (I/O) em uma thread paralelamente a cálculos em outra thread.

O código a seguir mostra como o módulo de alto nível **threading** pode executar tarefas em segundo plano enquanto o programa principal continua a sua execução:

```
import threading, zipfile

class AsyncZip(threading.Thread):
    def __init__(self, arquivo_entrada, arquivo_saida):
        super().__init__()
        self.arquivo_entrada = arquivo_entrada
        self.arquivo_saida = arquivo_saida

    def run(self):
        with zipfile.ZipFile(self.arquivo_saida, 'w', zipfile.ZI
            f.write(self.arquivo_entrada)
        print('Finalizou compactação em background de:', self.ar

background = AsyncZip('meusdados.txt', 'meuarquivo.zip')
background.start()
print('O programa original continua a executar em primeiro planc

background.join()    # Aguarda a tarefa em background finalizar
print('Programa principal aguardou até a tarefa em background es
```

O principal desafio para as aplicações que usam múltiplas threads é coordenar as threads que compartilham dados ou outros recursos. Para esta

finalidade, o módulo threading oferece alguns mecanismos primitivos de sincronização, como travas, eventos, variáveis de condição e semáforos.

Ainda que todas essas ferramentas sejam poderosas, pequenos erros de design podem resultar em problemas difíceis de serem diagnosticados. Por isso, a abordagem preferida para a coordenação da tarefa é concentrar todo o acesso a um recurso em um único tópico e, em seguida, usar o módulo **queue** para alimentar esse segmento com solicitações de outros tópicos. Aplicações que utilizam objetos **Queue** para comunicação e coordenação inter-thread são mais fáceis de serem projetados, mais legíveis e mais confiáveis.

11.5. Gerando logs

O módulo **logging** oferece um completo e flexível sistema de log. Da maneira mais simples, mensagens de log são enviadas para um arquivo ou para `sys.stderr`:

```
import logging
logging.debug('Depurando informações')
logging.info('Mensagem informática')
logging.warning('Alerta: arquivo de configuração  %s não encontr
logging.error('Ocorreu um erro')
logging.critical('Erro crítico -- desligando')
```

Isso produz a seguinte saída:

```
WARNING:root:Alerta:arquivo de configuração server.conf não enco
ERROR:root:Ocorreu um erro
CRITICAL:root:Erro crítico -- shutting down
```

Por padrão, mensagens informativas e de depuração são suprimidas e a saída é enviada para a saída de erros padrão (stderr). Outras opções de saída incluem envio de mensagens através de correio eletrônico, datagramas, sockets ou para um servidor HTTP. Novos filtros podem selecionar diferentes formas de envio de mensagens, baseadas na prioridade da mensagem: **DEBUG**, **INFO**, **WARNING**, **ERROR** e **CRITICAL**.

O sistema de log pode ser configurado diretamente do Python ou pode ser carregado a partir de um arquivo de configuração editável pelo usuário para logs personalizados sem a necessidade de alterar a aplicação.

11.6. Referências fracas

Python faz gerenciamento automático de memória (contagem de referências para a maioria dos objetos e coleta de lixo para eliminar ciclos). A memória ocupada por um objeto é liberada logo depois da última referência a ele ser eliminada.

Essa abordagem funciona bem para a maioria das aplicações, mas ocasionalmente surge a necessidade de rastrear objetos apenas enquanto estão sendo usados por algum outro. Infelizmente rastreá-los cria uma referência, e isso os fazem permanentes. O módulo **`weakref`** oferece ferramentas para rastrear objetos sem criar uma referência. Quando o objeto não é mais necessário, ele é automaticamente removido de uma tabela de referências fracas e uma chamada (*callback*) é disparada. Aplicações típicas incluem armazenamento em cache de objetos que são muito custosos para criar:

```
>>> import weakref, gc
>>> class A:
...     def __init__(self, value):
...         self.value = value
...     def __repr__(self):
...         return str(self.value)
...
>>> a = A(10)                   # cria a referência
>>> d = weakref.WeakValueDictionary()
>>> d['primário'] = a            # não cria a referência
>>> d['primário']                #  obtém o objeto se ele ainda
10
>>> del a                       # remove a única referência
>>> gc.collect()                # executa coleta de lixo imediat
0
>>> d['primário']                #  a entrada foi removida autom
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
```

```
    d['primário']                # a entrada foi removida automa
  File "C:/python315/lib/weakref.py", line 46, in __getitem__
    o = self.data[key]()
KeyError: 'primário'
```

11.7. Ferramentas para trabalhar com listas

Muitas necessidades envolvendo estruturas de dados podem ser satisfeitas com o tipo embutido lista. Entretanto, algumas vezes há uma necessidade por implementações alternativas que sacrificam algumas facilidades em nome de melhor desempenho.

O módulo **array** oferece uma classe **array**, semelhante a uma lista, mas que armazena apenas dados homogêneos e de maneira mais compacta. O exemplo a seguir mostra um vetor de números armazenados como números binários de dois bytes sem sinal (código de tipo `"H"`) ao invés dos 16 bytes usuais para cada item em uma lista de `int`:

```
>>> from array import array
>>> a = array('H', [4000, 10, 700, 22222])
>>> sum(a)
26932
>>> a[1:3]
array('H', [10, 700])
```

O módulo **collections** oferece um objeto **deque** que comporta-se como uma lista mas com *appends* e *pops* pela esquerda mais rápidos, porém mais lento ao percorrer o meio da sequência. Esses objetos são adequados para implementar filas e buscas em amplitude em árvores de dados:

```
>>> from collections import deque
>>> d = deque(["tarefa1", "tarefa2", "tarefa3"])
>>> d.append("tarefa4")
>>> print("Tratando", d.popleft())
Tratando tarefa1

nao_pesquisada = deque([no_inicial])
def busca_em_profundidade(nao_pesquisada):
    no = nao_pesquisada.popleft()
```

```
    for m in gen_moves(no):
        if eh_o_alvo(m):
            return m
        nao_pesquisada.append(m)
```

Além de implementações alternativas de listas, a biblioteca também oferece outras ferramentas como o módulo `bisect` com funções para manipulação de listas ordenadas:

```
>>> import bisect
>>> notas = [(100, 'perl'), (200, 'tcl'), (400, 'lua'), (500, 'p
>>> bisect.insort(notas, (300, 'ruby'))
>>> notas
[(100, 'perl'), (200, 'tcl'), (300, 'ruby'), (400, 'lua'), (500,
```

O módulo `heapq` oferece funções para implementação de *heaps* baseadas em listas normais. O valor mais baixo é sempre mantido na posição zero. Isso é útil para aplicações que acessam repetidamente o menor elemento, mas não querem reordenar a lista toda a cada acesso:

```
>>> from heapq import heapify, heappop, heappush
>>> dados = [1, 3, 5, 7, 9, 2, 4, 6, 8, 0]
>>> heapify(dados)                      # reorganiza a lista com
>>> heappush(dados, -5)                 # adiciona o valor -5 ac
>>> [heappop(dados) for i in range(3)]  # encontra as 3 menores
[-5, 0, 1]
```

11.8. Aritmética decimal com ponto flutuante

O módulo `decimal` oferece o tipo `Decimal` para aritmética decimal com ponto flutuante. Comparado a implementação embutida `float` que usa aritmética binária de ponto flutuante, a classe é especialmente útil para:

- aplicações financeiras que requerem representação decimal exata,
- controle sobre a precisão,
- controle sobre arredondamento para satisfazer requisitos legais,
- rastreamento de casas decimais significativas, ou

- aplicações onde o usuário espera que os resultados sejam os mesmos que os dos cálculos feitos à mão.

Por exemplo, calcular um imposto de 5% sobre uma chamada telefônica de 70 centavos devolve diferentes resultados com aritmética de ponto flutuante decimal ou binária. A diferença torna-se significativa se os resultados são arredondados para o centavo mais próximo:

```
>>> from decimal import *
>>> round(Decimal('0.70') * Decimal('1.05'), 2)
Decimal('0.74')
>>> round(.70 * 1.05, 2)
0.73
```

O resultado de `Decimal` considera zeros à direita, automaticamente inferindo quatro casas decimais a partir de multiplicandos com duas casas decimais. O módulo Decimal reproduz a aritmética como fazemos à mão e evita problemas que podem ocorrer quando a representação binária do ponto flutuante não consegue representar quantidades decimais com exatidão.

A representação exata permite à classe `Decimal` executar cálculos de módulo e testes de igualdade que não funcionam bem em ponto flutuante binário:

```
>>> Decimal('1.00') % Decimal('.10')
Decimal('0.00')
>>> 1.00 % 0.10
0.09999999999999995

>>> sum([Decimal('0.1')]*10) == Decimal('1.0')
True
>>> 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 ==
False
```

O módulo `decimal` implementa a aritmética com tanta precisão quanto necessária:

```
>>> getcontext().prec = 36
>>> Decimal(1) / Decimal(7)
Decimal('0.142857142857142857142857142857142857')
```

12. Ambientes virtuais e pacotes

12.1. Introdução

Aplicações em Python normalmente usam pacotes e módulos que não vêm como parte da instalação padrão. Aplicações às vezes necessitam uma versão específica de uma biblioteca, porque ela requer que algum problema em particular tenha sido consertado ou foi escrita utilizando-se de uma versão obsoleta da interface da biblioteca.

Isso significa que talvez não seja possível que uma instalação Python preencha os requisitos de qualquer aplicação. Se uma aplicação A necessita a versão 1.0 de um módulo particular mas a aplicação B necessita a versão 2.0, os requisitos entrarão em conflito e instalar qualquer uma das duas versões 1.0 ou 2.0 fará com que uma das aplicações não consiga executar.

A solução para este problema é criar um ambiente virtual, uma árvore de diretórios que contém uma instalação Python para uma versão particular do Python, além de uma série de pacotes adicionais.

Diferentes aplicações podem então usar diferentes ambientes virtuais. Para resolver o exemplo anterior de requisitos conflitantes, a aplicação A deve ter seu próprio ambiente virtual com a versão 1.0 instalada enquanto a aplicação B vai possuir outro ambiente virtual com a versão 2.0. Se a aplicação B precisar fazer uma atualização para a versão 3.0, isso não afetará o ambiente da aplicação A.

12.2. Criando ambientes virtuais

O módulo usado para criar e gerenciar ambientes virtuais é chamado **`venv`**. **`venv`** instalará a versão do Python a partir da qual o comando foi executado (conforme relatado pela opção **`--version`**). Por exemplo, executar o comando com `python3.12` instalará a versão 3.12.

Para criar um ambiente virtual, escolha um diretório onde deseja colocá-lo e execute o módulo **venv** como um script com o caminho do diretório:

```
python -m venv tutorial-env
```

Isso irá criar o diretório `tutorial-env` se ele não existir, e também criará diretórios dentro dele contendo uma cópia do interpretador Python, a biblioteca padrão e diversos arquivos de suporte.

Um diretório de localização comum para um ambiente virtual é `.venv`. Esse nome tipicamente mantém o diretório oculto em seu ambiente, portanto é transparente, ao menos tempo que explica o motivo desse diretório existir. Também previne conflitos com `.env`, arquivos de definição de variáveis de ambiente que algumas ferramentas utilizam.

Uma vez criado seu ambiente virtual, você deve ativá-lo.

No Windows, execute:

```
tutorial-env\Scripts\activate
```

No Unix ou no MacOS, executa:

```
source tutorial-env/bin/activate
```

(Este script é escrito para o shell bash. Se você usa shells **csh** ou **fish**, existem scripts alternativos `activate.csh` e `activate.fish` para utilização.)

Ao ativar seu ambiente virtual haverá uma mudança no prompt do shell para mostrar qual ambiente virtual você está usando e modificará o ambiente para que quando você executar `python` ativar a versão e instalação do Python particular àquele ambiente. Por exemplo:

```
$ source ~/envs/tutorial-env/bin/activate
(tutorial-env) $ python
Python 3.5.1 (default, May  6 2016, 10:59:36)
  ...
>>> import sys
>>> sys.path
```

```
['', '/usr/local/lib/python35.zip', ...,
'~/envs/tutorial-env/lib/python3.5/site-packages']
>>>
```

Note que o ambiente virtual ativado não altera `PYTHONPATH` variáveis de forma alguma. Isso pode levar a resultados inesperados se o path incluir referencia a código incompatível com a versão Python do ambiente virtual em uso. A melhor prática é usar `unset PYTHONPATH` no bash ou no shell que você está usando.

Para desativar um ambiente virtual, digite:

```
deactivate
```

no terminal.

12.3. Gerenciando pacotes com o pip

Você pode instalar, atualizar e remover pacotes usando um programa chamado **pip**. Por padrão `pip` irá instalar pacotes do Python Package Index [https://pypi.org]. Você pode navegar pelo Python Package Index através do seu navegador web.

`pip` tem uma série de subcomandos: “install”, “uninstall”, “freeze”, etc. (Consulte o guia Instalando módulos Python para a documentação completa do `pip`.)

Você pode instalar a última versão de um pacote apenas especificando nome do pacote:

```
(tutorial-env) $ python -m pip install novas
Collecting novas
  Downloading novas-3.1.1.3.tar.gz (136kB)
Installing collected packages: novas
  Running setup.py install for novas
Successfully installed novas-3.1.1.3
```

Você também pode instalar uma versão específica de um pacote dando o nome do pacote seguido por `==` e o número da versão:

```
(tutorial-env) $ python -m pip install requests==2.6.0
Collecting requests==2.6.0
  Using cached requests-2.6.0-py2.py3-none-any.whl
Installing collected packages: requests
Successfully installed requests-2.6.0
```

Se você executar novamente o comando, `pip` será notificado de que a versão já está instalada, e não fará nada. Você pode fornecer um número de versão diferente para obter essa versão ou pode executar `python -m pip install --upgrade` para atualizar o pacote para a versão mais recente:

```
(tutorial-env) $ python -m pip install --upgrade requests
Collecting requests
Installing collected packages: requests
  Found existing installation: requests 2.6.0
    Uninstalling requests-2.6.0:
      Successfully uninstalled requests-2.6.0
Successfully installed requests-2.7.0
```

`python -m pip uninstall` seguido por um ou mais nomes de pacote removerá os pacotes do ambiente virtual.

`python -m pip show` exibirá informações sobre um pacote específico:

```
(tutorial-env) $ python -m pip show requests
---
Metadata-Version: 2.0
Name: requests
Version: 2.7.0
Summary: Python HTTP for Humans.
Home-page: http://python-requests.org
Author: Kenneth Reitz
Author-email: me@kennethreitz.com
License: Apache 2.0
Location: /Users/akuchling/envs/tutorial-env/lib/python3.4/site-
Requires:
```

`python -m pip list` exibirá todos os pacotes instalados no ambiente virtual:

```
(tutorial-env) $ python -m pip list
novas (3.1.1.3)
numpy (1.9.2)
pip (7.0.3)
requests (2.7.0)
setuptools (16.0)
```

`python -m pip freeze` produzirá uma lista semelhante dos pacotes instalados, mas a saída usa o formato que `python -m pip install` espera. Uma convenção comum é colocar esta lista em um arquivo `requirements.txt`:

```
(tutorial-env) $ python -m pip freeze > requirements.txt
(tutorial-env) $ cat requirements.txt
novas==3.1.1.3
numpy==1.9.2
requests==2.7.0
```

O arquivo `requirements.txt` pode ser submetido no controle de versão e adicionado como parte da aplicação. Usuários poderão então instalar todos os pacotes necessários com um `install -r`:

```
(tutorial-env) $ python -m pip install -r requirements.txt
Collecting novas==3.1.1.3 (from -r requirements.txt (line 1))
  ...
Collecting numpy==1.9.2 (from -r requirements.txt (line 2))
  ...
Collecting requests==2.7.0 (from -r requirements.txt (line 3))
  ...
Installing collected packages: novas, numpy, requests
  Running setup.py install for novas
Successfully installed novas-3.1.1.3 numpy-1.9.2 requests-2.7.0
```

`pip` tem inúmeras outras opções. Consulte o guia Instalando módulos Python para a documentação completa do `pip`. Quando você escrever um pacote e desejar deixá-lo disponível no Python Package Index, consulte o guia de usuário para empacotamento de Python [https://packaging.python.org/en/latest/tutorials/packaging-projects/].

13. E agora?

Ler este tutorial provavelmente reforçou seu interesse em usar Python — você deve estar ansioso para aplicar Python para resolver problemas do mundo real. Aonde você deveria ir para aprender mais?

Este tutorial é parte do conjunto de documentação da linguagem Python. Alguns outros documentos neste conjunto são:

- Referência da bibliotecas embutidas do Python: fornece detalhes sobre os tipos e as funções embutidas do Python.
- A biblioteca padrão do Python: fornece material completo (ainda que breve) de referência sobre tipos, funções e módulos na biblioteca padrão. A distribuição padrão do Python inclui *muito* código adicional. Há módulos para ler caixa de correio Unix, baixar documentos via HTTP, gerar números aleatórios, processar opções de linha de comando, comprimir dados a muitas outras tarefas. Uma lida rápida da Referência da Biblioteca lhe dará uma ideia do que está disponível.
- Instalando módulos Python explica como instalar módulos adicionais escritos por outros usuários de Python.
- A Referência da Linguagem Python: Uma explicação detalhada da sintaxe e da semântica de Python. É uma leitura pesada, mas é útil como um guia completo da linguagem propriamente dita.

Mais recursos Python:

- https://www.python.org: o principal site sobre Python. Contém código, documentação, e referências para outras páginas relacionadas a Python ao redor da web.
- https://docs.python.org: acesso rápido à documentação Python.
- https://pypi.org: O Python Package Index, anteriormente apelidado de Cheese Shop [[1]], é um índice de módulos Python criados pelos usuários. Uma vez que você começa a publicar código, pode registrar seus pacotes aqui para que outros possam encontrá-los.

- https://code.activestate.com/recipes/langs/python/: O Python Cookbook (livro de receitas de Python) é uma grande coleção de exemplos de código, módulos maiores e scripts úteis.Contribuições particularmente notáveis são coletadas em um livro também chamado Python Cookbook (O'Reilly & Associates, ISBN 0-596-00797-3.)
- https://pyvideo.org coleta links para vídeo relacionados a Python de conferências e reuniões de grupos de usuários.
- https://scipy.org: o projeto Scientific Python (Python Científico) inclui módulos para computação e manipulação rápida de vetores e também hospeda pacotes para coisas similares como álgebra linear, transformações de Fourier, resolvedores não-lineares, distribuições de números aleatórios, análise estatística e afins.

Para reportar questões e problemas relacionadas a Python, você pode postar no newsgroup *comp.lang.python* ou enviá-los para o grupo de e-mail em python-list@python.org. O newsgroup e a lista são conectados, então mensagens postadas em um são automaticamente encaminhadas ao outro. Há centenas de postagem diárias perguntando (e respondendo) perguntas, sugerindo novas funcionalidades e anunciando novos módulos. E-mails arquivados estão disponíveis em https://mail.python.org/pipermail/. Existe também o grupo de discussão da comunidade brasileira de Python: python-brasil no Google Groups (http://groups.google.com/group/python-brasil), com discussões de ótimo nível técnico.

Antes de postar, certifique-se de checar a lista de perguntas frequentes (também chamada de FAQ). O FAQ responde muitas das perguntas que aparecem com frequência e pode já conter a solução para o seu problema.

Notas de rodapé

[1]

"Cheese Shop" é o título de um quadro do grupo Monty Python: um freguês entra em uma loja especializada em queijos, mas qualquer queijo que ele pede, o balconista diz que está em falta.

14. Edição de entrada interativa e substituição de histórico

Algumas versões do interpretador Python suportam a edição da linha de entrada atual e a substituição da história, semelhante às habilidade encontradas no shell Korn e no shell GNU Bash. Isso é implementado usando a biblioteca GNU Readline [https://tiswww.case.edu/php/chet/readline/rltop.html], que oferece suporte a vários estilos de edição. Esta biblioteca possui sua própria documentação, que não vamos duplicar aqui.

14.1. Tab completion e histórico de edição

A conclusão ("completion") dos nomes de variáveis e módulos é ativado automaticamente na inicialização do interpretador para que a tecla `Tab` invoque a função de conclusão. Ele analisa os nomes das instruções Python, as variáveis locais atuais e os nomes dos módulos disponíveis. Para expressões pontilhadas como `string.a`, ele avaliará a expressão até o `'.'` final e então sugerirá conclusões dos atributos do objeto resultante. Observe que isso pode executar o código definido pela aplicação se um objeto com um método **`__getattr__()`** faz parte da expressão. A configuração padrão também guarda seu histórico em um arquivo chamado `.python_history` no seu diretório de usuário. O histórico estará disponível novamente durante a próxima sessão do interpretador interativo.

14.2. Alternativas ao interpretador interativo

Esta facilidade é um enorme passo em frente em comparação com as versões anteriores do interpretador; No entanto, alguns desejos são deixados: seria bom se um **`INDENT`** adequado fosse sugerido nas linhas de continuação (o analisador sabe se é necessário um token de recuo). O mecanismo de conclusão pode usar a tabela de símbolos do interpretador.

Um comando para verificar (ou mesmo sugerir) parênteses, aspas, etc., também seria útil.

Um interpretador interativo aprimorado e alternativo que existe há algum tempo é o IPython [https://ipython.org/], que apresenta a conclusão da guia, a exploração de objetos e o gerenciamento de histórico avançado. Também pode ser completamente personalizado e incorporado em outras aplicações. Outro ambiente interativo aprimorado similar é bpython [https://bpython-interpreter.org/].

15. Aritmética de ponto flutuante: problemas e limitações

Os números de ponto flutuante são representados no hardware do computador como frações de base 2 (binárias). Por exemplo, a fração **decimal** `0.625` tem valor 6/10 + 2/100 + 5/1000, e da mesma forma a fração **binária** `0.101` tem valor 0/2 + 0/4 + 1/8. Essas duas frações têm valores idênticos, a única diferença real é que a primeira é escrita em notação fracionária de base 10 e a segunda em base 2.

Infelizmente, muitas frações decimais não podem ser representadas precisamente como frações binárias. O resultado é que, em geral, os números decimais de ponto flutuante que você digita acabam sendo armazenados de forma apenas aproximada, na forma de números binários de ponto flutuante.

O problema é mais fácil de entender primeiro em base 10. Considere a fração 1/3. Podemos representá-la aproximadamente como uma fração base 10:

```
0.3
```

ou melhor,

```
0.33
```

ou melhor,

```
0.333
```

e assim por diante. Não importa quantos dígitos você está disposto a escrever, o resultado nunca será exatamente 1/3, mas será uma aproximação de cada vez melhor de 1/3.

Da mesma forma, não importa quantos dígitos de base 2 estejas disposto a usar, o valor decimal 0.1 não pode ser representado exatamente como uma fração de base 2. No sistema de base 2, 1/10 é uma fração binária que se repete infinitamente:

```
0.0001100110011001100110011001100110011001100110011...
```

Se parares em qualquer número finito de bits, obterás uma aproximação. Hoje em dia, na maioria dos computadores, as casas decimais são aproximados usando uma fração binária onde o numerado utiliza os primeiros 53 bits iniciando no bit mais significativo e tendo como denominador uma potência de dois. No caso de 1/10, a fração binária seria `602879701896397 / 2 ** 55` o que chega bem perto, mas mesmo assim, não é igual ao valor original de 1/10.

É fácil esquecer que o valor armazenado é uma aproximação da fração decimal original, devido à forma como os pontos flutuantes são exibidos no interpretador interativo. O Python exibe apenas uma aproximação decimal do verdadeiro valor decimal da aproximação binária armazenada pela máquina. Se o Python exibisse o verdadeiro valor decimal da aproximação binária que representa o decimal 0.1, seria necessário mostrar:

```
>>> 0.1
0.1000000000000000055511151231257827021181583404541015625
```

Isto contém muito mais dígitos do que é o esperado e utilizado pela grande maioria dos desenvolvedores, portanto, o Python limita o número de dígitos exibidos, apresentando um valor arredondado, ao invés de mostrar todas as casas decimais:

```
>>> 1 / 10
0.1
```

Lembre-se, mesmo que o resultado impresso seja o valor exato de 1/10, o valor que verdadeiramente estará armazenado será uma fração binária representável que mais se aproxima.

Curiosamente, existem muitos números decimais diferentes que compartilham a mesma fração binária aproximada. Por exemplo, os

números `0.1` ou o `0.10000000000000001` e `0.1000000000000000055511151231257827021181583404541015625` são todos aproximações de `3602879701896397 / 2 ** 55`. Como todos esses valores decimais compartilham um mesma de aproximação, qualquer um poderá ser exibido enquanto for preservado o invariante `eval(repr(x)) == x`.

Historicamente, o prompt do Python e a função embutida **`repr()`** utilizariam o que contivesse 17 dígitos significativos, `0.10000000000000001`. Desde a versão do Python 3.1, o Python (na maioria dos sistemas) agora é possível optar pela forma mais reduzida, exibindo simplesmente o número `0.1`.

Note que essa é a própria natureza do ponto flutuante binário: não é um bug do Python, e nem é um bug do seu código. Essa situação pode ser observada em todas as linguagens que usam as instruções aritméticas de ponto flutuante do hardware (apesar de algumas linguagens não *mostrarem* a diferença, por padrão, ou em todos os modos de saída).

Para obter um valor mais agradável, poderás utilizar a formatação de sequência de caracteres para produzir um número limitado de dígitos significativos:

```
>>> format(math.pi, '.12g')  # ponto flutuante fornece 12 dígitc
'3.14159265359'

>>> format(math.pi, '.2f')   # ponto flutuante fornece 2 dígitos
'3.14'

>>> repr(math.pi)
'3.141592653589793'
```

É importante perceber que tudo não passa de pura ilusão: estas simplesmente arredondando a *exibição* da verdadeira maquinaria do valor.

Uma ilusão pode gerar outra. Por exemplo, uma vez que 0.1 não é exatamente 1/10, somar três vezes o valor 0.1, não garantirá que o resultado seja exatamente 0.3, isso porque:

```
>>> 0.1 + 0.1 + 0.1 == 0.3
False
```

Inclusive, uma vez que o 0.1 não consegue aproximar-se do valor exato de 1/10 e 0.3 não pode se aproximar mais do valor exato de 3/10, temos então que o pré-arredondamento com a função `round()` não servirá como ajuda:

```
>>> round(0.1, 1) + round(0.1, 1) + round(0.1, 1) == round(0.3,
False
```

Embora os números não possam se aproximar mais dos exatos valores que desejamos, a função `math.isclose()` poderá ser útil para comparar valores inexatos:

```
>>> math.isclose(0.1 + 0.1 + 0.1, 0.3)
True
```

Alternativamente, a função `round()` pode ser usada para comparar aproximações aproximadas:

```
>>> round(math.pi, ndigits=2) == round(22 / 7, ndigits=2)
True
```

A aritmética binária de ponto flutuante contém muitas surpresas como essa. O problema com "0.1" é explicado em detalhes precisos abaixo, na seção "Erro de representação". Consulte Exemplos de problemas de ponto flutuante [https://jvns.ca/blog/2023/01/13/examples-of-floating-point-problems/] (em inglês) para obter um resumo agradável de como o ponto flutuante binário funciona e os tipos de problemas comumente encontrados na prática. Veja também Os perigos do ponto flutuante [http://www.indowsway.com/floatingpoint.htm] (em inglês) para um relato mais completo de outras surpresas comuns.

Como dizemos perto do final, "não há respostas fáceis". Ainda assim, não se percam indevidamente no uso do ponto flutuante! Os erros nas operações do tipo float do Python são heranças do hardware de ponto flutuante e, a maioria dos computadores estão na ordem de não mais do que 1 parte em 2**53 por operação. Isso é mais do que o suficiente para a maioria das tarefas, portanto, é importante lembrar que não se trata de uma aritmética

decimal e que toda operação com o tipo float poderá via a apresentar novos problemas referentes ao arredondamento.

Embora existam casos patológicos, na maior parte das vezes, terás como resultado final o valor esperado, se simplesmente arredondares a exibição final dos resultados para a quantidade de dígitos decimais que esperas a função `str()` geralmente será o suficiente, e , para seja necessário um valor refinado, veja os especificadores de formato `str.format()` contido na seção Sintaxe das strings de formato.

Para as situações que exijam uma representação decimal exata, experimente o módulo `decimal` que possui, a implementação de uma adequada aritmética decimal bastante utilizada nas aplicações contábeis e pelas aplicações que demandam alta precisão.

Uma outra forma de obter uma aritmética exata tem suporte pelo módulo **`fracções`** que implementa a aritmética baseada em números racionais (portanto, os números fracionários como o 1/3 conseguem uma representação precisa).

Caso necessites fazer um intenso uso das operações de ponto flutuante, é importante que conheças o pacote NumPy e, também é importante dizer, que existem diversos pacotes destinados ao trabalho intenso com operações matemáticas e estatísticas que são fornecidas pelo projeto SciPy. Veja <https://scipy.org>.

O Python fornece ferramentas que podem ajudar nessas raras ocasiões em que realmente *faz* necessário conhecer o valor exato de um ponto flutuante. O método `float.as_integer_ratio()` expressa o valor de um número do tipo float em sua forma fracionária:

```
>>> x = 3.14159
>>> x.as_integer_ratio()
(3537115888337719, 1125899906842624)
```

Uma vez que a relação seja exata, será possível utiliza-la para obter, sem que haja quaisquer perda do valor original:

```
>>> x == 3537115888337719 / 1125899906842624
True
```

O método `float.hex()` expressa um ponto flutuante em hexadecimal (base 16), o mesmo também retornará o valor exato pelo computador:

```
>>> x.hex()
'0x1.921f9f01b866ep+1'
```

Sua precisa representação hexadecimal poderá ser utilizada para reconstruir o valor exato do ponto flutuante:

```
>>> x == float.fromhex('0x1.921f9f01b866ep+1')
True
```

Como a representação será exata, é interessante utilizar valores confiáveis em diferentes versões do Python (independente da plataforma) e a troca de dados entre idiomas diferentes que forneçam o mesmo formato (como o Java e o C99).

Uma outra ferramenta que poderá ser útil é a função `sum()` que ajuda a mitigar a perda de precisão durante a soma. Ele usa precisão estendida para etapas intermediárias de arredondamento, pois os valores serão adicionados a um total em execução. Isso poderá fazer a diferença na precisão geral de forma que os erros não se acumulem chegando ao ponto de afetar o resultado final:

```
>>> 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 + 0.1 ==
False
>>> sum([0.1] * 10) == 1.0
True
```

A função `math.fsum()` vai além e rastreia todos os “dígitos perdidos” à medida que os valores são adicionados a um total contínuo, de modo que o resultado tenha apenas um único arredondamento. Isso é mais lento que `sum()`, mas será mais preciso em casos incomuns em que entradas de grande magnitude se cancelam, deixando uma soma final próxima de zero:

```
>>> arr = [-0.10430216751806065, -266310978.67179024, 1434011614
...        -143401161400469.7, 266262841.31058735, -0.0032449368
>>> float(sum(map(Fraction, arr)))   # Soma exata com arredondam
8.042173697819788e-13
>>> math.fsum(arr)                   # Arredondamento simples
8.042173697819788e-13
>>> sum(arr)                         # Arredondamentos múltiplos
8.042178034628478e-13
>>> total = 0.0
>>> for x in arr:
...     total += x                   # Arredondamentos múltiplos
...
>>> total                            # A adição direta não tem c
-0.0051575902860057365
```

15.1. Erro de representação

Esta seção explica o exemplo do “0.1” em detalhes, e mostra como poderás realizar uma análise exata de casos semelhantes. Presumimos que tenhas uma familiaridade básica com a representação binária de ponto flutuante.

Erro de representação refere-se ao fato de que algumas frações decimais (a maioria, na verdade) não podem ser representadas exatamente como frações binárias (base 2). Esta é a principal razão por que o Python (ou Perl, C, C++, Java, Fortran, e muitas outras) frequentemente não exibe o número decimal exato conforme o esperado:

Por que isso acontece? 1/10 não é representado exatamente sendo fração binária. Desde pelo menos 2000, quase todas as máquinas usam aritmética binária de ponto flutuante da IEEE 754 e quase todas as plataformas representam pontos flutuante do Python como valores binary64 de “double precision” (dupla precisão) da IEEE 754. Os valores binary64 da IEEE 754 têm 53 bits de precisão, por isso na entrada o computador se esforça para converter “0.1” para a fração mais próxima que puder, na forma *J*/2***N* onde *J* é um número inteiro contendo exatamente 53 bits. Reescrevendo:

```
1 / 10 ~= J / (2**N)
```

como

```
J ~= 2**N / 10
```

e recordando que *J* tenha exatamente 53 bits (é `>= 2**52`, mas `< 2**53`), o melhor valor para *N* é 56:

```
>>> 2**52 <=  2**56 // 10  < 2**53
True
```

Ou seja, 56 é o único valor de *N* que deixa *J* com exatamente 53 bits. Portanto, o melhor valor que conseguimos obter pra *J* será aquele que possui o quociente arredondado:

```
>>> q, r = divmod(2**56, 10)
>>> r
6
```

Uma vez que o resto seja maior do que a metade de 10, a melhor aproximação que poderá ser obtida se arredondarmos para cima:

```
>>> q+1
7205759403792794
```

Portanto, a melhor aproximação possível de 1/10 como um “IEEE 754 double precision” é:

```
7205759403792794 / 2 ** 56
```

Dividir o numerador e o denominador por dois reduzirá a fração para:

```
3602879701896397 / 2 ** 55
```

Note que, como arredondamos para cima, esse valor é, de fato, um pouco maior que 1/10; se não tivéssemos arredondado para cima, o quociente teria sido um pouco menor que 1/10. Mas em nenhum caso seria possível obter *exatamente* o valor 1/10!

Por isso, o computador nunca “vê” 1/10: o que ele vê é exatamente a fração que é obtida pra cima, a melhor aproximação “IEEE 754 double” possível

é:

```
>>> 0.1 * 2 ** 55
3602879701896397.0
```

Se multiplicarmos essa fração por 10**55, podemos ver o valor contendo os 55 dígitos mais significativos:

```
>>> 3602879701896397 * 10 ** 55 // 2 ** 55
1000000000000000055511151231257827021181583404541015625
```

o que significa que o número exato armazenado no computador é igual ao valor decimal 0.1000000000000000055511151231257827021181583404541015625. Em vez de exibir o valor decimal completo, muitas linguagens (incluindo versões mais antigas do Python), arredondam o resultado para 17 dígitos significativos:

```
>>> format(0.1, '.17f')
'0.10000000000000001'
```

Módulos como o `fractions` e o `decimal` tornam esses cálculos muito mais fáceis:

```
>>> from decimal import Decimal
>>> from fractions import Fraction

>>> Fraction.from_float(0.1)
Fraction(3602879701896397, 36028797018963968)

>>> (0.1).as_integer_ratio()
(3602879701896397, 36028797018963968)

>>> Decimal.from_float(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015
```

```
>>> format(Decimal.from_float(0.1), '.17')
'0.10000000000000001'
```

16. Anexo

16.1. Modo interativo

Existem duas variantes do REPL interativo. O interpretador básico clássico é compatível com todas as plataformas com recursos mínimos de controle de linha.

Desde Python 3.13, um novo console interativo é usado por padrão. Este oferece suporte a cores, edição multilinha, navegação no histórico e modo de colagem. Para desabilitar a cor, veja Controlando cores para mais detalhes. As teclas de função oferecem algumas funcionalidades adicionais. `F1` abre o navegador de ajuda interativo **`pydoc`**. `F2` permite navegar no histórico da linha de comando sem saída nem os prompts `>>>` e `...`. `F3` entra no "modo de colagem", o que torna mais fácil colar blocos de código maiores. Pressione `F3` para retornar ao prompt normal.

Ao usar o novo console interativo, saia do console digitando `exit` ou `quit`. Não é necessário adicionar parênteses de chamada após esses comandos.

Se o novo console interativo não for desejado, ele pode ser desabilitado através da variável de ambiente **`PYTHON_BASIC_REPL`**.

16.1.1. Tratamento de erros

Quando um erro ocorre, o interpretador exibe uma mensagem de erro e um *stack trace* (rastreamento de pilha). Se estiver no modo interativo, ele volta para o prompt primário; se a entrada veio de um arquivo, a execução termina com um status de saída diferente de zero após a exibição do *stack trace*. (Exceções tratadas por uma cláusula **`except`** numa instrução **`try`** não são consideradas erros, nesse contexto.) Alguns erros são irremediavelmente graves e causam términos de execução com status de saída diferente de zero; isso pode ocorrer devido a inconsistências internas e, em alguns casos, por falta de memória. Todas as mensagens de erro são

escritas no fluxo de erros padrão; a saída normal resultante da execução de comandos é escrita no fluxo de saída padrão.

Digitar o caractere de interrupção (geralmente `Control-C` ou `Delete`) em prompts primários ou secundários interrompe a entrada de dados e retorna ao prompt primário. [[1]] Digitar a interrupção durante a execução de um comando levanta a exceção **`KeyboardInterrupt`**, que pode ser tratada por meio de uma declaração **`try`**.

16.1.2. Scripts Python executáveis

Em sistemas Unix similares ao BSD, scripts Python podem ser executados diretamente, tal como scripts shell, se tiverem a linha de código

```
#!/usr/bin/env python3
```

(presumindo que o interpretador está na **`PATH`** do usuário) no começo do script e configurando o arquivo no modo executável. Os dois primeiros caracteres do arquivo devem ser `#!`. Em algumas plataformas, essa primeira linha deve terminar com uma quebra de linha em estilo Unix (`'\n'`), e não em estilo Windows (`'\r\n'`). Note que o caractere cerquilha, `'#'`, é usado em Python para marcar o início de um comentário.

O script pode receber a permissão para atuar em modo executável através do comando **chmod**.

```
$ chmod +x meuscript.py
```

Em sistemas Windows, não existe a noção de “modo executável”. O instalador do Python associa automaticamente os arquivos `.py` ao `python.exe`, de modo que um clique duplo em um arquivo Python o executará como um script. A extensão pode ser também `.pyw`, o que omite a janela de console que normalmente aparece.

16.1.3. Arquivo de inicialização do modo interativo

Quando se usa o Python no modo interativo, pode ser útil definir alguns comandos que sejam executados automaticamente toda vez que o

interpretador for inicializado. Isso pode ser feito configurando-se uma variável de ambiente chamada **PYTHONSTARTUP** para que ela aponte para o arquivo contendo esses comandos. Isso é similar ao recurso `.profile` das shells Unix.

Esse arquivo será lido apenas em sessões do modo interativo, e não quando Python lê comandos de um script, tampouco quando `/dev/tty` é passado explicitamente como a origem dos comandos (neste caso, teremos um comportamento similar a uma sessão interativa padrão). Ele é executado no mesmo espaço de nomes em que os comandos interativos são executados, de modo que os objetos que ele define ou importa possam ser usados sem qualificação na sessão interativa. Também é possível alterar os prompts `sys.ps1` e `sys.ps2` no mesmo arquivo.

Caso deseje usar um arquivo de inicialização adicional a partir do atual diretório de trabalho, você pode programá-lo no arquivo de inicialização global usando um código parecido com `if os.path.isfile('.pythonrc.py'): exec(open('.pythonrc.py').read())`. Se quiser usar o arquivo de inicialização num script, será necessário fazê-lo explicitamente no script:

```
import osfilename = os.environ.get('PYTHONSTARTUP')if filename a
```

16.1.4. Módulos de customização

Python oferece dois ganchos que permitem sua customização: sitecustomize e usercustomize. Para entender como funcionam, primeiro você deve localizar o diretório site-packages do usuário. Inicie o Python e execute este código:

```
>>> import site>>> site.getusersitepackages()'/home/user/.local/
```

Agora você pode criar um arquivo chamado `usercustomize.py` neste diretório e colocar qualquer coisa que quiser dentro. Isto vai afetar toda invocação do Python, a menos que seja iniciado com a opção **-s** para desabilitar a importação automática.

sitecustomize funciona da mesma forma, mas normalmente é criado por um administrador do computador no diretório site-packages global e é importado antes de usercustomize. Veja a documentação do módulo `site` para mais detalhes.

Notas de rodapé

[1]

Um problema com o pacote GNU Readline pode impedir que isso aconteça.

Configurações e Uso do Python

Esta parte da documentação é dedicada a informações gerais sobre a configuração do ambiente Python em diferentes plataformas, a chamada do interpretador e outras coisas que facilitam o trabalho com o Python.

- 1. Linha de comando e ambiente
  - 1.1. Linha de comando
  - 1.2. Variáveis de ambiente
- 2. Utilizando Python em plataformas Unix
  - 2.1. Obtendo e instalando a versão mais recente do Python
  - 2.2. Compilando o Python
  - 2.3. Paths e arquivos relacionados com o Python
  - 2.4. Diversos
  - 2.5. OpenSSL personalizado
- 3. Configurando o Python
  - 3.1. Requisitos de construção
  - 3.2. Arquivos gerados
  - 3.3. Opções de configuração
  - 3.4. Sistema de Construção Python
  - 3.5. Sinalizadores do compilador e do vinculador
- 4. Utilizando Python no Windows
  - 4.1. Gerenciador de instalação do Python
  - 4.2. O pacote embutível
  - 4.3. Os pacotes nuget.org
  - 4.4. Pacotes Alternativos
  - 4.5. Versões do Windows suportadas
  - 4.6. Removendo a limitação de MAX_PATH
  - 4.7. Modo UTF-8
  - 4.8. Encontrando módulos
  - 4.9. Módulos adicionais
  - 4.10. Compilando Python no Windows
  - 4.11. O instalador completo (descontinuado)
  - 4.12. Inicializador Python para Windows (descontinuado)
- 5. Usando Python em macOS

- 5.1. Usando Python para macOS do `python.org`
- 5.2. Distribuições alternativas
- 5.3. Instalar pacotes Python adicionais
- 5.4. Programação de GUI
- 5.5. Tópicos avançados
- 5.6. Outros recursos

- 6. Utilizando Python no Android
  - 6.1. Adicionando Python a uma aplicação Android
  - 6.2. Construindo um pacote Python para Android
- 7. Usando Python no iOS
  - 7.1. Python em tempo de execução no iOS
  - 7.2. Instalando Python no iOS
  - 7.3. Conformidade com a App Store
- 8. Editores e IDEs
  - 8.1. IDLE — editor e console Python
  - 8.2. Outros editores e IDEs

1. Linha de comando e ambiente

O interpretador CPython verifica a linha de comando e o ambiente em busca de várias configurações.

Os esquemas de linha de comando de outras implementações podem ser diferentes. Consulte Implementações Alternativas para mais recursos.

1.1. Linha de comando

Ao invocar o Python, você pode especificar qualquer uma destas opções:

```
python [-bBdEhiIOPqRsSuvVWx?] [-c command | -m module-name | scr
```

O caso de uso mais comum é, obviamente, uma simples invocação de um script:

```
python meuscript.py
```

1.1.1. Opções de interface

A interface do interpretador é semelhante à do console do UNIX, mas fornece alguns métodos adicionais de chamada:

- Quando chamado com a entrada padrão conectada a um dispositivo tty, ele solicita comandos e os executa até um EOF (um caractere de fim de arquivo, você pode produzi-lo com `Ctrl-D` no UNIX ou `Ctrl-Z, Enter` no Windows) ser lido. Para mais informações sobre o modo interativo, veja Modo interativo.
- Quando chamado com um nome de arquivo como argumento ou com um arquivo como entrada padrão, ele lê e executa o script desse arquivo.
- Quando chamado com um nome de diretório como argumento, ele lê e executa um script adequadamente nomeado nesse diretório.

- Quando chamado com `-c command`, ele executa as instruções Python fornecidas como *command*. Aqui *command* pode conter várias instruções separadas por novas linhas.
- Quando chamado com `-m module-name`, o módulo especificado é localizado pelo mecanismo de importação padrão e executado como um script.

No modo não interativo, toda a entrada é analisada antes de ser executada.

Uma opção de interface termina a lista de opções consumidas pelo interpretador, todos os argumentos consecutivos terminam em **`sys.argv`** – observe que o primeiro elemento, índice zero (`sys.argv[0]`) , é uma string que reflete a fonte do programa.

-c <command>

Executa o código Python em *command*. *command* pode ser uma ou mais instruções separadas por novas linhas, com espaços em branco à esquerda significativos, como em um código de um módulo normal.

Se esta opção for fornecida, o primeiro elemento de **`sys.argv`** será `"-c"` e o diretório atual será adicionado ao início de **`sys.path`** (permitindo que módulos nesse diretório sejam importados como módulos de nível superior).

Levanta um evento de auditoria `cpython.run_command` com o argumento `command`.

Alterado na versão 3.14: command é automaticamente desindentado antes da execução.

-m <module-name>

Localiza o módulo usando o mecanismo de importação padrão e executa seu conteúdo como o módulo **`__main__`**.

Como o argumento é o nome de um *módulo*, você não deve fornecer uma extensão de arquivo (`.py`). O nome do módulo deve ser um nome de módulo Python absoluto válido, mas a implementação nem sempre

exige isso (por exemplo, pode permitir que você use um nome que inclua um hífen).

Nomes de pacotes (incluindo pacotes de espaço de nomes) também são permitidos. Quando um nome de pacote é fornecido ao invés de um módulo normal, o interpretador irá executar `<pkg>.__main__` como o módulo principal. Esse comportamento é deliberadamente semelhante ao tratamento de diretórios e arquivos zip que são passados para o interpretador como o argumento do script.

Nota

Esta opção não pode ser usada com módulos embutidos e com módulos de extensão escritos em C, pois não possuem arquivos de módulo em Python. No entanto, ele ainda pode ser usado para módulos pré-compilados, mesmo que o arquivo-fonte original não esteja disponível.

Se esta opção for fornecida, o primeiro elemento de **`sys.argv`** será o caminho completo para o arquivo do módulo (enquanto o arquivo do módulo está sendo localizado, o primeiro elemento será definido como `"-m"`). Como com a opção **`-c`**, o diretório atual será adicionado ao início de **`sys.path`**.

A opção **`-I`** pode ser usada para executar o script em modo isolado, em que **`sys.path`** não contém nem o diretório atual nem o diretório de pacotes do usuário. Todas as variáveis de ambiente `PYTHON*` são ignoradas também.

Muitos módulos de biblioteca padrão contêm código que é executado como um script. Um exemplo é o módulo **`timeit`**:

```
python -m timeit -s "configuração inicial aqui" "código a ser
python -m timeit -h # para detalhes
```

Levanta um evento de auditoria `cpython.run_module` com o argumento `module-name`.

Ver também

`runpy.run_module()`
Funcionalidade equivalente diretamente disponível para código Python

PEP 338 [https://peps.python.org/pep-0338/] – Executing modules as scripts

Alterado na versão 3.1: Forneça o nome do pacote para executar um submódulo `__main__`.

Alterado na versão 3.4: Pacotes de espaço de nomes também são suportados

-

Lê os comandos da entrada padrão (`sys.stdin`). Se a entrada padrão for um terminal, `-i` está implícito.

Se esta opção for fornecida, o primeiro elemento de `sys.argv` será `"-"` e o diretório atual será adicionado ao início de `sys.path`.

Levanta um evento de auditoria `cpython.run_stdin` sem argumentos.

<script>
Executa o código Python contido em *script*, que deve ser um caminho do sistema de arquivos (absoluto ou relativo) referindo-se a um arquivo Python, um diretório contendo um arquivo `__main__.py`, ou um arquivo zip contendo um arquivo `__main__.py`.

Se esta opção for fornecida, o primeiro elemento de `sys.argv` será o nome do script conforme fornecido na linha de comando.

Se o nome do script se referir diretamente a um arquivo Python, o diretório que contém esse arquivo é adicionado ao início de `sys.path`, e o arquivo é executado como o módulo `__main__`.

Se o nome do script se referir a um diretório ou arquivo zip, o nome do script será adicionado ao início de `sys.path` e o arquivo `__main__.py`

nesse local será executado como o módulo `__main__`.

A opção `-I` pode ser usada para executar o script em modo isolado onde `sys.path` não contém nem o diretório do script nem o diretório de pacotes do usuário. Todas as variáveis de ambiente `PYTHON*` são ignoradas também.

Levanta um evento de auditoria `cpython.run_file` com o argumento `filename`.

Ver também

`runpy.run_path()`
Funcionalidade equivalente diretamente disponível para código Python

Se nenhuma opção de interface for fornecida, `-i` está implícito, `sys.argv[0]` é uma string vazia (`""`) e o diretório atual será adicionado ao início de `sys.path`. Além disso, o preenchimento por tab e a edição do histórico são habilitados automaticamente, se disponíveis em sua plataforma (veja Configuração Readline).

Ver também

Chamando o interpretador

Alterado na versão 3.4: Ativação automática de preenchimento com tab e edição de histórico.

1.1.2. Opções genéricas

-?
-h
--help
Imprime uma breve descrição de todas as opções de linha de comando e variáveis de ambiente correspondentes e sai.

--help-env

Imprime uma breve descrição das variáveis de ambiente específicas do Python e sai.

Adicionado na versão 3.11.

--help-xoptions

Imprime uma descrição das opções `-X` específicas da implementação e sai.

Adicionado na versão 3.11.

--help-all

Imprime as informações de uso completas e sai.

Adicionado na versão 3.11.

-V
--version

Exibe o número da versão do Python e sai. O exemplo de saída poderia ser:

```
Python 3.8.0b2+
```

Quando fornecido duas vezes, exibe mais informações sobre a construção, como:

```
Python 3.8.0b2+ (3.8:0c076caaa8, Apr 20 2019, 21:55:00)
[GCC 6.2.0 20161005]
```

Adicionado na versão 3.6: A opção `-vv`.

1.1.3. Opções diversas

-b

Emite um aviso ao converter `bytes` ou `bytearray` para `str` sem especificar a codificação, ou ao comparar **`bytes`** ou **`bytearray`** com **`str`**,

ou **bytes** com **int**. Emite um erro quando a opção é fornecida duas vezes (**-bb**).

Alterado na versão 3.5: Afeta também as comparações de **bytes** com **int**.

Descontinuado desde a versão 3.15: Descontinua as opções **-b** e **-bb**, planejando para se tornarem sem efeito em Python 3.17. Esses foram primariamente auxiliares da transição Python 2 -> 3. A partir de Python 3.17, nenhum **BytesWarning** será levantada para esses casos; em vez disso, use um verificador de tipos.

-B

Se fornecido, Python não tentará escrever arquivos `.pyc` na importação de módulos fonte. Veja também **PYTHONDONTWRITEBYTECODE**.

--check-hash-based-pycs default|always|never

Controla o comportamento de validação de arquivos `.pyc` baseados em hash. Veja Invalidação de bytecode em cache. Quando definido como `default`, os arquivos de cache de bytecode baseados em hash verificados e não verificados são validados de acordo com sua semântica padrão. Quando definido como `always`, todos os arquivos `.pyc` baseados em hash, sejam verificados ou não verificados, são validados para seus arquivos fonte correspondentes. Quando definido como `never`, os arquivos `.pyc` baseados em hash não são validados contra seus respectivos arquivos fonte.

A semântica dos arquivos `.pyc` baseados em marca de tempo não é afetada por esta opção.

-d

Ativa a saída do analisador sintático de depuração (uso avançado). Consulte também a variável de ambiente **PYTHONDEBUG**.

Essa opção requer uma construção de depuração do Python, caso contrário, será ignorada.

-E

Ignora todas as variáveis de ambiente `PYTHON*`, por exemplo **PYTHONPATH** e **PYTHONHOME**, que podem ser definidas.

Veja também as opções **-P** e **-I** (isolado).

-i

Entra no modo interativo após a execução.

Usar a opção **-i** vai entrar no modo interativo em qualquer uma das circunstâncias:

- Quando um script é passado como primeiro argumento
- Quando a opção **-c** é usada
- Quando a opção **-m** é usada

O modo interativo vai ser iniciado mesmo quando a **sys.stdin** não parecer ser um terminal. O arquivo **PYTHONSTARTUP** não é lido.

Isso pode ser útil para inspecionar variáveis globais ou o stack trace (situação da pilha de execução) quando um script levanta uma exceção. Veja também **PYTHONINSPECT**.

-I

Executa o Python em modo isolado. Isso também implica nas opções **-E**, **-P** e **-s**.

No modo isolado, **sys.path** não contém o diretório do script nem o diretório de pacotes do site do usuário. Todas as variáveis de ambiente `PYTHON*` também são ignoradas. Outras restrições podem ser impostas para evitar que o usuário injete código malicioso.

Adicionado na versão 3.4.

-O

Remove as instruções de asserção e qualquer código condicional que envolva o valor de **__debug__**. Aumenta o nome do arquivo de arquivos compilados (bytecode) adicionando `.opt-1` antes da extensão `.pyc`

(veja **PEP 488** [https://peps.python.org/pep-0488/]). Veja também **PYTHONOPTIMIZE**.

Alterado na versão 3.5: Modifica nomes de arquivos `.pyc` conforme a **PEP 488** [https://peps.python.org/pep-0488/].

-OO

Faz o mesmo que **-O** e também descarta as strings de documentação. Aumenta o nome dos arquivos de bytecode compilados (bytecode) adicionando `.opt-2` antes da extensão `.pyc` (veja **PEP 488** [https://peps.python.org/pep-0488/]).

Alterado na versão 3.5: Modifica nomes de arquivos `.pyc` conforme a **PEP 488** [https://peps.python.org/pep-0488/].

-P

Não anexa um caminho potencialmente inseguro a **sys.path**:

- Linha de comando `python -m módulo`: não anexa o diretório atual.
- Linha de comando `python script.py`: não anexa o diretório do próprio script. Se for um link simbólico, resolve links simbólicos.
- Linhas de comando `python -c código` e `python` (REPL): não anexam uma string vazia, o que significa o diretório de trabalho atual.

Veja também a variável de ambiente **PYTHONSAFEPATH** e as opções **-E** e **-I** (isolado).

Adicionado na versão 3.11.

-q

Não exibe as mensagens de direitos autorais e de versão nem mesmo no modo interativo.

Adicionado na versão 3.2.

-R

Habilita a aleatorização com hash. Esta opção só tem efeito se a variável de ambiente **PYTHONHASHSEED** estiver configurada para um valor diferente de `random`, uma vez que a aleatorização com hash é habilitada por padrão.

Em versões anteriores do Python, esta opção ativa a aleatorização com hash, para que os valores **__hash__()** dos objetos str e bytes sejam "salgados" com um valor aleatório imprevisível. Embora permaneçam constantes em um processo Python individual, eles não são previsíveis entre invocações repetidas de Python.

A aleatorização com hash se destina a fornecer proteção contra uma negação de serviço causada por entradas cuidadosamente escolhidas que exploram o pior caso de desempenho de uma inserção de dicionário, de complexidade $O(n^2)$. Consulte https://ocert.org/advisories/ocert-2011-003.html para obter detalhes.

PYTHONHASHSEED permite definir um valor fixo para o hash da semente.

Adicionado na versão 3.2.3.

Alterado na versão 3.7: A opção não é mais ignorada.

-s

Não adiciona o **diretório site-packages do usuário** a **sys.path**.

Veja também **PYTHONNOUSERSITE**.

Ver também

PEP 370 [https://peps.python.org/pep-0370/] – Diretório site-packages por usuário.

-S

Desabilita a importação do módulo **site** e as manipulações do **sys.path** que isso acarreta. Também desabilita essas manipulações se

site for explicitamente importado mais tarde (chame **site.main()** se você quiser que eles sejam acionados).

-u

Força os fluxos de saída e de erro padrão (stdout e stderr) a não terem buffer. Esta opção não tem efeito no fluxo de entrada padrão (stdin).

Veja também **PYTHONUNBUFFERED**.

Alterado na versão 3.7: A camada de texto dos fluxos stdout e stderr agora é sem buffer.

-v

Exibe uma mensagem cada vez que um módulo é inicializado, mostrando o local (nome do arquivo ou módulo embutido) de onde ele é carregado. Quando fornecido duas vezes (**-vv**), exibe uma mensagem para cada arquivo verificado durante a busca por um módulo. Também fornece informações sobre a limpeza do módulo na saída.

Alterado na versão 3.10: O módulo **site** relata os caminhos específicos do site e os arquivos `.pth` sendo processados.

Veja também **PYTHONVERBOSE**.

-W arg

Controle de advertência. O mecanismo de aviso do Python, por padrão, exibe mensagens de aviso para **sys.stderr**.

As configurações mais simples aplicam uma determinada ação incondicionalmente a todos os avisos emitidos por um processo (mesmo aqueles que são ignorados por padrão):

```
-Wdefault  # Avisa uma vez por local de chamada
-Werror    # Converte para exceções
-Walways   # Avisa toda vez
-Wall      # Mesmo que -Walways
-Wmodule   # Avisa uma vez por módulo chamador
-Wonce     # Avisa uma vez por processo do Python
-Wignore   # Nunca avisa
```

Os nomes das ações podem ser abreviados conforme desejado, e o interpretador irá resolvê-los para o nome da ação apropriado. Por exemplo, `-Wi` é equivalente a `-Wignore`.

A forma completa de argumento é:

```
action:message:category:module:lineno
```

Os campos vazios correspondem a todos os valores; campos vazios à direita podem ser omitidos. Por exemplo, `-W ignore::DeprecationWarning` ignora todos os avisos de DeprecationWarning.

O campo *action* é explicado acima, mas aplica-se apenas aos avisos que correspondem aos campos restantes.

O campo *message* deve corresponder a toda a mensagem de aviso; essa correspondência não diferencia maiúsculas de minúsculas. Se começar e terminar com uma barra (`/`), estará especificando uma expressão regular, caso contrário, é uma string literal.

O campo *category* corresponde à categoria de aviso (ex.: `DeprecationWarning`). Deve ser um nome de classe; o teste de correspondência verifica se a categoria de aviso real da mensagem é subclasse da categoria de aviso especificada.

O campo *module* corresponde ao nome do módulo (totalmente qualificado); esta correspondência diferencia maiúsculas de minúsculas. Se começar e terminar com uma barra (`/`), ele especifica uma expressão regular a qual o início do nome do módulo totalmente qualificado deve corresponder; caso contrário, especifica uma string literal à qual o nome do módulo totalmente qualificado deve ser igual.

O campo *lineno* corresponde ao número da linha, em que zero corresponde a todos os números de linha e, portanto, é equivalente a um número de linha omitido.

Múltiplas opções **`-W`** podem ser fornecidas; quando um aviso corresponde a mais de uma opção, a ação da última opção

correspondente é executada. As opções **-W** inválidas são ignoradas (embora uma mensagem de aviso seja exibida sobre opções inválidas quando o primeiro aviso for emitido).

Os avisos também podem ser controlados usando a variável de ambiente **PYTHONWARNINGS** e, dentro de um programa Python, usando o módulo **warnings**. Por exemplo, a função **warnings.filterwarnings()** pode ser usada para aplicar uma expressão regular à mensagem de aviso.

Veja O filtro de avisos e Descrevendo filtros de aviso para mais detalhes.

Alterado na versão 3.15: Adicionado suporte para expressão regular em *message* e *module*.

-x

Pula a primeira linha do código-fonte, permitindo o uso de formas não-Unix de `#!cmd`.

Isso pode ser usado para transformar um script Python em um arquivo de lote (bat) do Windows. De forma semelhante a adicionar uma linha *shebang* e definir o bit de execução no Unix, a extensão do script Python pode ser alterada para `.bat` e a seguinte linha pode ser adicionada ao início do script:

```
@py -x "%~f0" %* & exit /b
```

ou, para especificar ou caminho do interpretador do Python especificamente:

```
@"C:\Caminho\para\python.exe" -x "%~f0" %* & exit /b
```

Ao contrário da linha *shebang*, que é um comentário em Python, esta linha não é uma sintaxe Python válida, e a opção **-x** é necessária para ignorá-la.

-X

Reservado para várias opções específicas de implementação. CPython atualmente define os seguintes valores possíveis:

- `-X faulthandler` para habilitar **`faulthandler`**. Veja também **`PYTHONFAULTHANDLER`**.

  Adicionado na versão 3.3.

- `-X showrefcount` para emitir a contagem de referências total e o número de blocos de memória usados quando o programa termina ou após cada instrução no interpretador interativo. Isso só funciona em construções de depuração.

  Adicionado na versão 3.4.

- `-X tracemalloc` para começar a rastrear alocações de memória do Python usando o módulo **`tracemalloc`**. Por padrão, apenas o quadro mais recente é armazenado no traceback (situação da pilha de execução) de um rastro. Use `-X tracemalloc=NFRAME` para iniciar o rastreamento com um limite de traceback de quadros *NFRAME*. Veja o **`tracemalloc.start()`** e **`PYTHONTRACEMALLOC`** para mais informações.

  Adicionado na versão 3.4.

- `-X int_max_str_digits` configura a limitação de comprimento de string na conversão para inteiro. Veja também **`PYTHONINTMAXSTRDIGITS`**.

  Adicionado na versão 3.11.

- `-X importtime` para mostrar quanto tempo leva cada importação. Mostra o nome do módulo, tempo cumulativo (incluindo importações aninhadas) e tempo próprio (excluindo importações aninhadas). Observe que sua saída pode ser interrompida em aplicações multithread. O uso típico é `python -X importtime -c 'import asyncio'`.

  `-X importtime=2` habilita uma saída adicional que indica quando um módulo importado já foi carregado. Nesses casos, a string `cached` será impressa em ambas as colunas de tempo.

Veja também **PYTHONPROFILEIMPORTTIME**.

Adicionado na versão 3.7.

Alterado na versão 3.14: Adicionado `-X importtime=2` para também rastrear importações de módulos carregados e valores reservados diferentes de `1` e `2` para uso futuro.

- `-X dev`: habilita Modo de Desenvolvimento do Python, introduzindo verificações adicionais de tempo de execução que são muito custosas para serem habilitadas por padrão. Veja também **PYTHONDEVMODE**.

  Adicionado na versão 3.7.

- `-X utf8` habilita o Modo UTF-8 do Python. `-X utf8=0` explicitamente desabilita o Modo UTF-8 do Python (mesmo quando de outra forma seria ativado automaticamente). Veja também **PYTHONUTF8**.

  Adicionado na versão 3.7.

- `-X pycache_prefix=PATH` permite a escrita de arquivos `.pyc` em uma árvore paralela enraizada em um determinado diretório em vez de na árvore de código. Veja também **PYTHONPYCACHEPREFIX**.

  Adicionado na versão 3.8.

- `-X warn_default_encoding` emite uma **EncodingWarning** quando a codificação padrão específica da localidade é usada para abrir arquivos. Veja também **PYTHONWARNDEFAULTENCODING**.

  Adicionado na versão 3.10.

- `-X no_debug_ranges` desabilita a inclusão das tabelas que mapeiam informações de localização extra (linha final, deslocamento da coluna inicial e deslocamento da coluna final) para cada instrução em objetos código. Isso é útil quando objetos código menores e arquivos pyc são desejados, bem como suprimir

os indicadores de localização visual extra quando o interpretador exibe tracebacks (situação da pilha de execução). Veja também **PYTHONNODEBUGRANGES**.

Adicionado na versão 3.11.

- `-X frozen_modules` determina se os módulos congelados são ou não ignorados pelo maquinário de importação. Um valor de `on` significa que eles são importados e `off` significa que eles são ignorados. O padrão é `on` se este for um Python instalado (o caso normal). Se estiver em desenvolvimento (executando a partir da árvore de origem), o padrão é `off`. Observe que os módulos congelados **`importlib_bootstrap`** e **`importlib_bootstrap_external`** são sempre usados, mesmo que esse sinalizador esteja definido como `off`. Veja também **PYTHON_FROZEN_MODULES**.

  Adicionado na versão 3.11.

- `-X perf` habilita o suporte para o perfilador do Linux `perf`. Quando essa opção for fornecida, o perfilador `perf` poderá relatar chamadas Python. Essa opção está disponível apenas em algumas plataformas e não fará nada se não for compatível com o sistema atual. O valor padrão é "off". Consulte também: **PYTHONPERFSUPPORT** e Python support for the perf map compatible profilers.

  Adicionado na versão 3.12.

- `-X perf_jit` habilita o suporte para o perfilador do Linux `perf` com suporte a DWARF. Quando essa opção for fornecida, o perfilador `perf` poderá relatar chamadas Python usando informações de DWARF. Essa opção está disponível somente em algumas plataformas e não fará nada se não for compatível com o sistema atual. O valor padrão é "off". Veja também: **PYTHON_PERF_JIT_SUPPORT** e Python support for the perf map compatible profilers.

Adicionado na versão 3.13.

- `-X disable_remote_debug` desabilita o suporte à depuração remota, conforme descrito em **PEP 768** [https://peps.python.org/pep-0768/]. Isso inclui tanto a funcionalidade de agendar código para execução em outro processo quanto a de receber código para execução no processo atual.

  Esta opção está disponível apenas em algumas plataformas e não fará nada se não for compatível com o sistema atual. Veja também **`PYTHON_DISABLE_REMOTE_DEBUG`** e **PEP 768** [https://peps.python.org/pep-0768/].

  Adicionado na versão 3.14.

- `-X cpu_count=`*`n`* substitui **`os.cpu_count()`**, **`os.process_cpu_count()`** e **`multiprocessing.cpu_count()`**. *n* deve ser maior ou igual a 1. Esta opção pode ser útil para usuários que precisam limitar os recursos da CPU de um sistema contêiner. Veja também **`PYTHON_CPU_COUNT`**. Se *n* for `default`, nada será substituído.

  Adicionado na versão 3.13.

- `-X presite=`*`módulo`* ou `-X presite=`*`módulo:função`* especifica um ponto de entrada que deve ser executado antes de o módulo **`site`** ser executado e antes de o módulo **`__main__`** existir. Portanto, o módulo importado não é **`__main__`**. Isso pode ser usado para executar código antes da inicialização do Python. Python precisa ser construído no modo de depuração para que esta opção exista. Veja também **`PYTHON_PRESITE`**.

  Alterado na versão 3.15: Aceita também o formato `módulo:função` como ponto de entrada.

  Adicionado na versão 3.13.

- `-X gil=0,1` força a GIL a ser desabilitada ou habilitada, respectivamente. A definição para `0` está disponível apenas em construções configuradas com `--disable-gil`. Veja também `PYTHON_GIL` e CPython com threads livres.

  Adicionado na versão 3.13.

- `-X thread_inherit_context=0,1` faz com que `Thread` use, por padrão, uma cópia do contexto do chamador de `Thread.start()` ao iniciar. Caso contrário, as threads iniciarão com um contexto vazio. Se não definida, o valor padrão desta opção será `1` em construções com threads livres e `0` em outras. Veja também `PYTHON_THREAD_INHERIT_CONTEXT`.

  Adicionado na versão 3.14.

- `-X context_aware_warnings=0,1` faz com que o gerenciador de contexto `warnings.catch_warnings` use uma `ContextVar` para armazenar o estado do filtro de avisos. Se não definida, o valor padrão desta opção será `1` em construções com threads livres e `0` em outras. Veja também `PYTHON_CONTEXT_AWARE_WARNINGS`.

  Adicionado na versão 3.14.

- `-X pathconfig_warnings=0,1`, se verdadeiro (`1`), permite que A inicialização do caminho de pesquisa de módulos sys.path registre alertas em stderr. Se falso (`0`), suprime estes alertas. Definido como verdadeiro por padrão. Veja também `PYTHON_PATHCONFIG_WARNINGS` .

  Adicionado na versão 3.15.

- `-X tlbc=0,1` habilita (1, o padrão) ou desabilita (0) o bytecode local da thread em construções configuradas com `--disable-gil`. Quando desabilitado, também desabilita o interpretador especializado. Veja também `PYTHON_TLBC`.

  Adicionado na versão 3.14.

- `-X lazy_imports=all,normal` controla o comportamento de importação preguiçosa. `all` torna todas as importações preguiçosas por padrão e `normal` (o padrão) respeita a palavra-chave `lazy` no código-fonte. Veja também **`PYTHON_LAZY_IMPORTS`** .

  Adicionado na versão 3.15.

Também permite passar valores arbitrários e recuperá-los por meio do dicionário **`sys.xoptions`**.

Adicionado na versão 3.2.

Alterado na versão 3.9: Removida a opção `-X showalloccount`.

Alterado na versão 3.10: Removida a opção `-X oldparser`.

Removido na versão 3.14: **`-J`** não é mais reservado para uso por Jython [https://www.jython.org/] e agora não tem nenhum significado especial.

1.1.4. Controlando cores

O interpretador Python é configurado por padrão para usar cores para destacar a saída em determinadas situações, como ao exibir tracebacks (situação da pilha de execução). Este comportamento pode ser controlado ao definir diferentes variáveis de ambiente.

Definir a variável de ambiente `TERM` como `dumb` desativará a exibição de cores.

Se a variável de ambiente **`FORCE_COLOR`** [https://force-color.org/] estiver definida, a cor será habilitada independentemente do valor de TERM. Isso é útil em sistemas CI que não são terminais, mas ainda podem exibir sequências de escape ANSI.

Se a variável de ambiente **`NO_COLOR`** [https://no-color.org/] estiver definida, o Python desativará todas as cores na saída. Isto tem precedência sobre `FORCE_COLOR`.

Todas essas variáveis de ambiente também são usadas por outras ferramentas para controlar a saída de cores. Para controlar a saída de cores apenas no interpretador Python, a variável de ambiente **PYTHON_COLORS** pode ser usada. Esta variável tem precedência sobre `NO_COLOR`, que, por sua vez, tem precedência sobre `FORCE_COLOR`.

1.2. Variáveis de ambiente

Essas variáveis de ambiente influenciam o comportamento do Python, elas são processadas antes das opções de linha de comando diferentes de -E ou -I. É comum que as opções de linha de comando substituam as variáveis de ambiente em caso de conflito.

PYTHONHOME

Altera a localização das bibliotecas padrão do Python. Por padrão, as bibliotecas são pesquisadas em *prefix*/lib/python*version* e *exec_prefix*/lib/python*version*, onde *prefix* e *exec_prefix* são diretórios dependentes da instalação, ambos padronizados para `/usr/local`.

Quando **PYTHONHOME** é definido como um único diretório, seu valor substitui *prefix* e *exec_prefix*. Para especificar valores diferentes para estes, defina **PYTHONHOME** para *prefix*:*exec_prefix*.

PYTHONPATH

Aumenta o caminho de pesquisa padrão para arquivos de módulo. O formato é o mesmo da variável **PATH** do shell: um ou mais caminhos de diretório separados por **os.pathsep** (por exemplo, dois pontos no Unix ou ponto e vírgula no Windows). Os diretórios inexistentes são ignorados silenciosamente.

Além dos diretórios normais, entradas individuais **PYTHONPATH** podem referir-se a arquivos zip contendo módulos Python puros (tanto no código-fonte quanto na forma compilada). Módulos de extensão não podem ser importados de arquivos zip.

O caminho de pesquisa padrão depende da instalação, mas geralmente começa com `prefix/lib/pythonversion` (veja **PYTHONHOME** acima). É *sempre* anexado a **PYTHONPATH**.

Um diretório adicional será inserido no caminho de pesquisa antes de **PYTHONPATH**, conforme descrito acima em Opções de interface. O caminho de pesquisa pode ser manipulado de partir de um programa em Python, por meio da variável **sys.path**.

PYTHONSAFEPATH

Se for definido como uma string não vazia, não anexa um caminho potencialmente inseguro à **sys.path**: consulte a opção **-P** para obter detalhes.

Adicionado na versão 3.11.

PYTHONPLATLIBDIR

Se for definida como uma string não vazia, ela substitui o valor de **sys.platlibdir**.

Adicionado na versão 3.9.

PYTHONSTARTUP

Se este for o nome de um arquivo legível, os comandos Python nesse arquivo serão executados antes que o primeiro prompt seja exibido no modo interativo. O arquivo é executado no mesmo espaço de nomes em que os comandos interativos são executados, para que os objetos definidos ou importados nele possam ser usados sem qualificação na sessão interativa. Você também pode alterar os prompts **sys.ps1** e **sys.ps2**, bem como o gancho **sys.__interactivehook__**, neste arquivo.

Levanta um evento de auditoria `cpython.run_startup` com o nome de arquivo como argumento quando chamado na inicialização.

PYTHONOPTIMIZE

Se for definido como uma string não vazia, é equivalente a especificar a opção **-O**. Se definido como um inteiro, é equivalente a especificar **-O**

várias vezes.

PYTHONBREAKPOINT

Se estiver definida, ela nomeia um chamável usando a notação de caminho com pontos. O módulo que contém o chamável será importado e, então, o chamável será executado pela implementação padrão de **sys.breakpointhook()**, chamada pelo **breakpoint()** embutido. Se não for definido, ou definido como uma string vazia, é equivalente ao valor de “pdb.set_trace”. Definir isso para a string “0” faz com que a implementação padrão de **sys.breakpointhook()** não faça mais nada além de retornar imediatamente.

Adicionado na versão 3.7.

PYTHONDEBUG

Se for definido como uma string não vazia, é equivalente a especificar a opção **-d**. Se definido como um inteiro, é equivalente a especificar **-d** várias vezes.

Essa variável de ambiente requer uma construção de depuração do Python, caso contrário, será ignorada.

PYTHONINSPECT

Se for definido como uma string não vazia, é equivalente a especificar a opção **-i**.

Esta variável também pode ser modificada pelo código Python usando **os.environ** para forçar o modo de inspeção ao encerrar o programa.

Levanta um evento de auditoria `cpython.run_stdin` sem argumentos.

Alterado na versão 3.12.5: (também 3.11.10, 3.10.15, 3.9.20 e 3.8.20) Emite eventos de auditoria.

Alterado na versão 3.13: Usa PyrePL, se possível, nesse caso **PYTHONSTARTUP** também é executado. Emite eventos de auditoria.

PYTHONUNBUFFERED

Se for definido como uma string não vazia, é equivalente a especificar a opção **-u**.

PYTHONVERBOSE

Se for definido como uma string não vazia, é equivalente a especificar a opção **-v**. Se definido como um inteiro, é equivalente a especificar **-v** várias vezes.

PYTHONCASEOK

Se estiver definido, Python não diferencia letras maiúsculas e minúsculas nas instruções **import**. Isso só funciona no Windows e no OS X.

PYTHONDONTWRITEBYTECODE

Se for definido como uma string não vazia, o Python não tentará escrever arquivos `.pyc` na importação de módulos-fonte. Isso equivale a especificar a opção **-B**.

PYTHONPYCACHEPREFIX

Se estiver definido, o Python escreverá os arquivos `.pyc` em uma árvore de diretório espelho neste caminho, em vez de nos diretórios `__pycache__` dentro da árvore de fontes. Isso equivale a especificar a opção **-X** `pycache_prefix=PATH`.

Adicionado na versão 3.8.

PYTHONHASHSEED

Se esta variável não for definida ou definida como `random`, um valor aleatório é usado para semear os hashes de objetos str e bytes.

Se **PYTHONHASHSEED** for definido como um valor inteiro, ele é usado como semente fixa para gerar o hash() dos tipos cobertos pela aleatorização de hash.

Sua finalidade é permitir hashes repetíveis, como em autotestes do próprio interpretador, ou permitir que um cluster de processos Python compartilhe valores de hash.

O número inteiro deve ser um número decimal no intervalo [0,4294967295]. Especificar o valor 0 desabilitará a aleatorização de hash.

Adicionado na versão 3.2.3.

PYTHONINTMAXSTRDIGITS

Se esta variável estiver definida como um inteiro, é usada para configurar a limitação de comprimento de string na conversão para inteiro global do interpretador.

Adicionado na versão 3.11.

PYTHONIOENCODING

Se for definido antes de executar o interpretador, ele substitui a codificação usada para stdin/stdout/stderr, na sintaxe `encodingname:errorhandler`. Ambas as partes `encodingname` e `:errorhandler` são opcionais e têm o mesmo significado que em **`str.encode()`**.

Para stderr, a parte `:errorhandler` é ignorada; o tratador sempre será `'backslashreplace'`.

Alterado na versão 3.4: A parte `encodingname` agora é opcional.

Alterado na versão 3.6: No Windows, a codificação especificada por esta variável é ignorada para buffers de console interativo, a menos que **`PYTHONLEGACYWINDOWSSTDIO`** também seja especificado. Arquivos e canais redirecionados por meio de fluxos padrão não são afetados.

PYTHONNOUSERSITE

Isso equivale a opção **`-s`**. Se estiver definido, o Python não adicionará o **`diretório site-packages do usuário`** a **`sys.path`**.

Ver também

PEP 370 [https://peps.python.org/pep-0370/] – Diretório site-packages por usuário.

PYTHONUSERBASE

Define o **diretório base do usuário**, que é usado para calcular o caminho do **diretório site-packages do usuário** e caminhos de instalação para `python -m pip install --user`.

Para desativar o diretório site-packages do usuário, consulte **PYTHONNOUSERSITE** ou a opção **-s**.

Ver também

PEP 370 [https://peps.python.org/pep-0370/] – Diretório site-packages por usuário.

PYTHONEXECUTABLE

Se esta variável de ambiente for definida, `sys.argv[0]` será definido com seu valor, em vez do valor obtido a partir do ambiente de execução C. Funciona apenas no macOS.

PYTHONWARNINGS

Isso é equivalente à opção **-W**. Se definido como uma string separada por vírgulas, é equivalente a especificar **-W** várias vezes, com os filtros posteriores na lista tendo precedência sobre os anteriores.

As configurações mais simples aplicam uma determinada ação incondicionalmente a todos os avisos emitidos por um processo (mesmo aqueles que são ignorados por padrão):

```
PYTHONWARNINGS=default  # Avisa uma vez por local de chamada
PYTHONWARNINGS=error    # Converte para exceções
PYTHONWARNINGS=always   # Avisa toda vez
PYTHONWARNINGS=all      # Mesmo que PYTHONWARNINGS=always
PYTHONWARNINGS=module   # Avisa uma vez por módulo chamador
PYTHONWARNINGS=once     # Avisa uma vez por processo do Pytho
PYTHONWARNINGS=ignore   # Nunca avisa
```

Veja O filtro de avisos e Descrevendo filtros de aviso para mais detalhes.

Alterado na versão 3.15: Adicionado suporte para expressão regular em *message* e *module*.

PYTHONFAULTHANDLER

Se esta variável de ambiente for definida como uma string não vazia, `faulthandler.enable()` é chamado na inicialização: instala um tratador para os sinais `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS` e `SIGILL` para despejar o traceback (situação da pilha de execução) do Python. Isso é equivalente à opção `-X` `faulthandler`.

Adicionado na versão 3.3.

PYTHONTRACEMALLOC

Se esta variável de ambiente for definida como uma string não vazia, começa a rastrear as alocações de memória do Python usando o módulo `tracemalloc`. O valor da variável é o número máximo de quadros armazenados em um traceback de um rastreamento. Por exemplo, `PYTHONTRACEMALLOC=1` armazena apenas o quadro mais recente. Veja a função `tracemalloc.start()` para mais informações. Isso é o equivalente a definir a opção `-X` `tracemalloc`.

Adicionado na versão 3.4.

PYTHONPROFILEIMPORTTIME

Se esta variável de ambiente for definida como `1`, o Python mostrará quanto tempo cada importação leva. Se definido como `2`, o Python incluirá a saída para módulos importados que já foram carregados. Isso equivale a definir a opção `-X` `importtime`.

Adicionado na versão 3.7.

Alterado na versão 3.14: Adicionado `PYTHONPROFILEIMPORTTIME=2` para também rastrear importações de módulos carregados.

PYTHONASYNCIODEBUG

Se esta variável de ambiente for definida como uma string não vazia, habilita o modo de depuração do módulo **asyncio**.

Adicionado na versão 3.4.

PYTHONMALLOC

Define os alocadores de memória do Python e/ou instala ganchos de depuração.

Define a família de alocadores de memória usados pelo Python:

- `default`: usa os alocadores padrão de memória.
- `malloc`: usa a função **`malloc()`** da biblioteca C para todos os domínios (**PYMEM_DOMAIN_RAW**, **PYMEM_DOMAIN_MEM**, **PYMEM_DOMAIN_OBJ**).
- `pymalloc`: usa o alocador pymalloc para domínios **PYMEM_DOMAIN_MEM** e **PYMEM_DOMAIN_OBJ** e usa a função **`malloc()`** para o domínio **PYMEM_DOMAIN_RAW**.
- `mimalloc`: usa o alocador mimalloc para domínios **PYMEM_DOMAIN_MEM** e **PYMEM_DOMAIN_OBJ** e usa a função **`malloc()`** para o domínio **PYMEM_DOMAIN_RAW**.

Instala ganchos de depuração:

- `debug`: instala os ganchos de depuração sobre os alocadores de memória padrão.
- `malloc_debug`: o mesmo que `malloc`, mas também instala ganchos de depuração.
- `pymalloc_debug`: o mesmo que `pymalloc`, mas também instala ganchos de depuração.
- `mimalloc_debug`: o mesmo que `mimalloc`, mas também instala ganchos de depuração.

Nota

Na construção com threads livres, os valores `malloc`, `malloc_debug`, `pymalloc` e `pymalloc_debug` não são aceitos. Somente são aceitos `default`, `debug`, `mimalloc` e `mimalloc_debug`.

Adicionado na versão 3.6.

Alterado na versão 3.7: Adicionado o alocador `"default"`.

PYTHONMALLOCSTATS

Se definido como uma string não vazia, o Python exibe estatísticas dos alocadores de memória pymalloc ou do mimalloc (o que estiver em uso) toda vez que uma nova arena de objeto é criada e ao desligamento.

Esta variável é ignorada se a variável de ambiente **`PYTHONMALLOC`** for usada para forçar o alocador **`malloc()`** da biblioteca C, ou se Python estiver configurado sem suporte a `pymalloc` ou `mimalloc`.

Alterado na versão 3.6: Esta variável agora também pode ser usada em Python compilado no modo de lançamento. Agora não tem efeito se definido como uma string vazia.

PYTHON_PYMALLOC_HUGEPAGES

Se definido como um valor inteiro diferente de zero, habilita o suporte a páginas enormes para as arenas pymalloc. Defina como `0` ou não defina para desativar. Python deve ser compilado com **`--with-pymalloc-hugepages`** para que essa variável tenha efeito.

Quando ativada, a alocação de arena usa `MAP_HUGETLB` (Linux) ou `MEM_LARGE_PAGES` (Windows) com retorno automático para páginas regulares se páginas enormes não estiverem disponíveis.

Aviso

No Linux, se o pool de páginas enormes estiver esgotado, as falhas de página - inclusive as falhas de cópia na gravação acionadas por **`os.fork()`** - entregam `SIGBUS` e encerram o processo forçadamente. Só ative essa opção em ambientes em que o pool de páginas enormes tenha tamanho adequado e a segurança contra forks não seja uma preocupação.

No Windows, você precisa de um privilégio especial. Consulte a documentação do Windows para páginas grandes

[https://learn.microsoft.com/pt-br/windows/win32/memory/large-page-support] para obter detalhes. Python falhará na inicialização se o privilégio necessário SeLockMemoryPrivilege [https://learn.microsoft.com/pt-br/previous-versions/windows/it-pro/windows-10/security/threat-protection/security-policy-settings/lock-pages-in-memory] não for concedido ao usuário.

Adicionado na versão 3.15.

PYTHONLEGACYWINDOWSFSENCODING

Se definido como uma string não vazia, o modo padrão do tratador de erros e codificação do sistema de arquivos irá reverter para seus valores pré-3.6 de “mbcs” e “replace”, respectivamente. Caso contrário, os novos padrões “utf-8” e “surrogatepass” serão usados.

Isso também pode ser habilitado em tempo de execução com `sys.enablelegacywindowsfsencoding()`.

Disponibilidade: Windows.

Adicionado na versão 3.6: Veja **PEP 529** [https://peps.python.org/pep-0529/] para mais detalhes.

PYTHONLEGACYWINDOWSSTDIO

Se definido como uma string não vazia, não usa o novo leitor e escritor de console. Isso significa que os caracteres Unicode serão codificados de acordo com a página de código do console ativo, em vez de usar utf-8.

Esta variável é ignorada se os fluxos padrão forem redirecionados (para arquivos ou canais) em vez de se referirem aos buffers do console.

Disponibilidade: Windows.

Adicionado na versão 3.6.

PYTHONCOERCECLOCALE

Se definido como `0`, faz com que a aplicação principal de linha de comando do Python ignore a coerção dos códigos de idioma legados C e POSIX baseados em ASCII para uma alternativa baseada em UTF-8 mais capaz.

Se esta variável *não* estiver definida (ou estiver definida para um valor diferente de `0`), a variável de ambiente de substituição de localidade `LC_ALL` também não será definida, e a localidade atual relatada para a categoria `LC_CTYPE` é a localidade `C` padrão, ou então a localidade `POSIX` explicitamente baseada em ASCII, então a CLI do Python tentará configurar as seguintes localidades para a categoria `LC_CTYPE` na ordem listada antes de carregar o ambiente de execução do interpretador:

- `C.UTF-8`
- `C.utf8`
- `UTF-8`

Se a configuração de uma dessas categorias de localidade for bem-sucedida, a variável de ambiente `LC_CTYPE` também será configurada de acordo no ambiente de processo atual antes que o ambiente de execução do Python seja inicializado. Isso garante que, além de ser visto pelo próprio interpretador e outros componentes com reconhecimento de localidade em execução no mesmo processo (como a biblioteca GNU `readline`), a configuração atualizada também é vista em subprocessos (independentemente ou não desses processos estarem executando um interpretador Python), bem como em operações que consultam o ambiente em vez da localidade C atual (como o **`locale.getdefaultlocale()`** do próprio Python).

Configurar uma dessas localidades (explicitamente ou por meio da coerção de localidade implícita acima) habilita automaticamente o tratador de erros `surrogateescape` para **`sys.stdin`** e **`sys.stdout`** (**`sys.stderr`** continua a usar `backslashreplace`, como em qualquer outra localidade). Este comportamento de tratamento de fluxo pode ser substituído usando **`PYTHONIOENCODING`** como de costume.

Para fins de depuração, definir `PYTHONCOERCECLOCALE=warn` fará com que o Python emita mensagens de aviso em `stderr` se a coerção de

localidade for ativada ou se uma localidade que *teria* acionado a coerção ainda estiver ativa quando o ambiente de execução do Python for inicializado.

Observe também que, mesmo quando a coerção de localidade está desabilitada ou não consegue encontrar uma localidade de destino adequada, **PYTHONUTF8** ainda será ativado por padrão em localidades baseadas em ASCII legadas. Ambos os recursos devem ser desabilitados para forçar o interpretador a usar `ASCII` em vez de `UTF-8` nas interfaces de sistema.

Disponibilidade: Unix.

Adicionado na versão 3.7: Veja **PEP 538** [https://peps.python.org/pep-0538/] para mais detalhes.

PYTHONDEVMODE

Se esta variável de ambiente for definida como uma string não vazia, habilita Modo de Desenvolvimento do Python, introduzindo verificações adicionais em tempo de execução que são muito caras para serem habilitadas por padrão. Isso é o equivalente a definir a opção **-X** `dev`.

Adicionado na versão 3.7.

PYTHONUTF8

Se definido para `1`, habilita o modo UTF-8 do Python.

Se definido para `0`, desabilita o modo UTF-8 do Python.

Definir qualquer outra string não vazia causa um erro durante a inicialização do interpretador.

Adicionado na versão 3.7.

PYTHONWARNDEFAULTENCODING

Se esta variável de ambiente for definida como uma string não vazia, emite uma **EncodingWarning** quando a codificação padrão específica da

localidade é usada.

Veja Ativar EncodingWarning para detalhes.

Adicionado na versão 3.10.

PYTHONNODEBUGRANGES

Se esta variável estiver definida, ela desabilita a inclusão das tabelas que mapeiam informações de localização extra (linha final, deslocamento da coluna inicial e deslocamento da coluna final) para cada instrução em objetos código. Isso é útil quando objetos código menores e arquivos pyc são desejados, bem como para suprimir os indicadores de localização visual extras quando o interpretador exibe tracebacks.

Adicionado na versão 3.11.

PYTHONPERFSUPPORT

Se essa variável for definida como um valor diferente de zero, ela habilitará o suporte ao perfilador do Linux `perf` para que as chamadas Python possam ser detectadas por ele.

Se definido como `0`, desativa o suporte ao perfilador do Linux `perf`.

Consulte também a opção de linha de comando **`-X perf`** e Python support for the perf map compatible profilers.

Adicionado na versão 3.12.

PYTHON_PERF_JIT_SUPPORT

Se essa variável for definida como um valor diferente de zero, ela habilitará o suporte do perfilador do Linux `perf` para que as chamadas Python possam ser detectadas por ele usando informações de DWARF.

Se definido como `0`, desativa o suporte ao perfilador do Linux `perf`.

Veja também a opção de linha de comando **`-X perf jit`** e Python support for the perf map compatible profilers.

Adicionado na versão 3.13.

PYTHON_DISABLE_REMOTE_DEBUG

Se esta variável for definida como uma string não vazia, ela desabilitará o recurso de depuração remota descrito na **PEP 768** [https://peps.python.org/pep-0768/]. Isso inclui tanto a funcionalidade de agendar código para execução em outro processo quanto a de receber código para execução no processo atual.

Veja também a opção de linha de comando `-X disable_remote_debug`.

Adicionado na versão 3.14.

PYTHON_CPU_COUNT

Se esta variável for definida como um número inteiro positivo, ela substituirá os valores de retorno de `os.cpu_count()` e `os.process_cpu_count()`.

Veja também a opção de linha de comando `-X cpu_count`.

Adicionado na versão 3.13.

PYTHON_FROZEN_MODULES

Se esta variável for definida como `on` ou `off`, ela determina se os módulos congelados serão ou não ignorados pelo maquinário de importação. Um valor `on` significa que eles serão importados e `off` significa que serão ignorados. O padrão é `on` para construções sem depuração (o caso normal) e `off` para construções com depuração. Observe que os módulos congelados **`importlib_bootstrap`** e **`importlib_bootstrap_external`** são sempre usados, mesmo que este sinalizador esteja definido como `off`.

Veja também a opção de linha de comando `-X frozen_modules`.

Adicionado na versão 3.13.

PYTHON_COLORS

Se esta variável for definida como `1`, o interpretador irá colorir vários tipos de saída. Definir como `0` desativa este comportamento. Veja também Controlando cores.

Adicionado na versão 3.13.

PYTHON_BASIC_REPL

Se esta variável for definida com qualquer valor, o interpretador não tentará carregar o REPL baseado em Python que requer **`readline`**, e, em vez disso, usará o REPL com o analisador sintático tradicional.

Adicionado na versão 3.13.

PYTHON_BASIC_COMPLETER

Se essa variável for definida como qualquer valor, o PyREPL usará **`rlcompleter`** para implementar o auto-complemento com tab, em vez da padrão, que usa cores.

Adicionado na versão 3.15.

PYTHON_HISTORY

Esta variável de ambiente pode ser usada para definir a localização do arquivo `.python_history` (por padrão, `.python_history` está no diretório pessoal do usuário).

Adicionado na versão 3.13.

PYTHON_GIL

Se esta variável for definida como `1`, a trava global do interpretador (GIL) será forçada. Definir como `0` força a GIL a desligar (precisa do Python configurado com a opção de construção **`--disable-gil`**).

Veja também a opção de linha de comando **`-X gil`**, que tem precedência sobre esta variável, e CPython com threads livres.

Adicionado na versão 3.13.

PYTHON_THREAD_INHERIT_CONTEXT

Se esta variável for definida como `1`, `Thread` usará, por padrão, uma cópia do contexto do chamador de `Thread.start()` ao iniciar. Caso contrário, novas threads serão iniciadas com um contexto vazio. Se não definida, esta variável assume como padrão `1` em construções com threads livres e `0` em outras. Veja também `-X thread_inherit_context`.

Adicionado na versão 3.14.

PYTHON_CONTEXT_AWARE_WARNINGS

Se definido como `1`, o gerenciador de contexto `warnings.catch_warnings` usará uma `ContextVar` para armazenar o estado do filtro de avisos. Se não for definido, o padrão desta variável será `1` em construções com threads livres e `0` em outras. Consulte `-X context_aware_warnings`.

Adicionado na versão 3.14.

PYTHON_PATHCONFIG_WARNINGS

Se verdadeiro (`1`), então A inicialização do caminho de pesquisa de módulos sys.path tem permissão para registrar alertas em stderr. Se falso (`0`), suprime estes alertas. Definido como verdadeiro por padrão. Veja também `-X pathconfig_warnings`.

Adicionado na versão 3.15.

PYTHON_JIT

Em construções em que a compilação experimental just-in-time está disponível, esta variável pode forçar o JIT a ser desabilitado (`0`) ou habilitado (`1`) na inicialização do interpretador.

Adicionado na versão 3.13.

PYTHON_TLBC

Se definido como `1`, habilita o bytecode local da thread. Se definido como `0`, o bytecode local da thread e o interpretador especializado são desabilitados. Aplica-se somente a construções configuradas com `--disable-gil`.

Veja também a opção de linha de comando `-X tlbc`.

Adicionado na versão 3.14.

PYTHON_LAZY_IMPORTS

Controla o comportamento da importação preguiçosa. Aceita dois valores: `all` torna todas as importações preguiçosas por padrão, e `normal` (o padrão) respeita a palavra-chave `lazy` no código-fonte.

Veja também a opção de linha de comando `-X lazy imports` .

Adicionado na versão 3.15.

1.2.1. Variáveis de modo de depuração

PYTHONDUMPREFS

Se definido, Python irá despejar objetos e contagens de referências ainda vivas após desligar o interpretador.

Precisa do Python configurado com a opção de construção `--with-trace-refs`.

PYTHONDUMPREFSFILE

Se definido, o Python irá despejar objetos e contagens de referências ainda ativas após desligar o interpretador em um arquivo no caminho fornecido como valor desta variável de ambiente.

Precisa do Python configurado com a opção de construção `--with-trace-refs`.

Adicionado na versão 3.11.

PYTHON_PRESITE

Se esta variável for definida como um módulo, esse módulo será importado no início do ciclo de vida do interpretador, antes do módulo `site` ser executado, e antes do módulo `__main__` ser criado. Portanto, o módulo importado não é tratado como `__main__`.

Isso pode ser usado para executar código antecipadamente durante a inicialização do Python.

Para importar um submódulo, use `pacote.módulo` como valor, como em uma instrução de importação.

Veja também a opção de linha de comando **`-X presite`**, que tem precedência sobre esta variável.

Precisa do Python configurado com a opção de construção **`--with-pydebug`**.

Alterado na versão 3.15: Aceita também o formato `módulo:função` como ponto de entrada.

Adicionado na versão 3.13.

2. Utilizando Python em plataformas Unix

2.1. Obtendo e instalando a versão mais recente do Python

2.1.1. No Linux

O Python vem pré-instalado na maioria das distribuições Linux e está disponível como um pacote em todas as outras. No entanto, há certos recursos que você pode querer usar que não estão disponíveis no pacote da sua distribuição. Você pode compilar a versão mais recente do Python a partir do código-fonte.

No caso de a versão mais recente do Python não vir pré-instalada e não estar nos repositórios também, você pode criar pacotes para sua própria distro. Dê uma olhada nos seguintes links:

Ver também

https://www.debian.org/doc/manuals/maint-guide/first.en.html
para usuários Debian

https://en.opensuse.org/Portal:Packaging
para usuários OpenSuse

https://docs.fedoraproject.org/en-US/package-maintainers/Packaging_Tutorial_GNU_Hello/
para usuários Fedora

https://slackbook.org/html/package-management-making-packages.html
para usuários do Slackware

2.1.1.1. Instalação do IDLE

Em alguns casos, o IDLE pode não estar incluído em sua instalação do Python.

- Para usuários do Debian e do Ubuntu:

  ```
  sudo apt update
  sudo apt install idle
  ```

- Para usuários do Fedora, RHEL e CentOS:

  ```
  sudo dnf install python3-idle
  ```

- Para usuários do SUSE e do OpenSUSE:

  ```
  sudo zypper install python3-idle
  ```

- Para usuários do Alpine Linux:

  ```
  sudo apk add python3-idle
  ```

2.1.2. No FreeBSD e OpenBSD

- usuários do FreeBSD, para adicionar a utilização do pacote utilize:

  ```
  pkg install python3
  ```

- Usuários do OpenBSD, para adicionar pacotes use:

  ```
  pkg_add -r python

  pkg_add ftp://ftp.openbsd.org/pub/OpenBSD/4.2/packages/<inse
  ```

  Por exemplo, usuários i386 podem pegar a versão 2.5.1 do Python usando o comando:

```
pkg_add ftp://ftp.openbsd.org/pub/OpenBSD/4.2/packages/i386/
```

2.2. Compilando o Python

Ver também

Se você deseja contribuir com o CPython, consulte o devguide [https://devguide.python.org/getting-started/setup-building/], que inclui instruções de construção e outras dicas sobre como configurar o ambiente.

Se você quer compilar o CPython, a primeira coisa que você precisa fazer é baixar o código-fonte [https://www.python.org/downloads/source/]. Você pode baixar a última versão ou obter um clone [https://devguide.python.org/setup/#get-the-source-code] atualizado. Você também pode querer instalar os requisitos de construção.

O processo de compilação consiste nos comandos usuais:

```
./configure
make
make install
```

Opções de configuração e advertências para plataformas específicas do Unix estão amplamente documentadas no arquivo README.rst [https://github.com/python/cpython/tree/3.15/README.rst] na raiz da árvore de fontes Python.

Aviso

`make install` pode sobrescrever ou mascarar o arquivo binário `python3`. `make altinstall` é, portanto, recomendado ao invés de `make install` uma vez que o mesmo apenas instala o arquivo *`exec_prefix`*`/bin/python`*`version`*.

2.3. Paths e arquivos relacionados com o Python

Estes estão sujeitos a diferenças dependendo das convenções de instalação local; **prefix** e **exec_prefix** dependem da instalação e devem ser interpretados da mesma forma que para o software GNU; eles poderão ser os mesmos.

Por exemplo, na maioria dos sistemas Linux, o padrão para ambos é `/usr`.

Arquivo/diretório	Significado
exec_prefix/bin/python3	Localização recomendada do interpretador.
prefix/lib/python*version*, *exec_prefix*/lib/python*version*	A localização recomendada dos diretórios contendo os módulos padrão.
prefix/include/python*version*, *exec_prefix*/include/python*version*	Localizações recomendadas dos diretórios contendo os arquivos de inclusão necessários para o desenvolvimento de extensões Python e incorporação do interpretador.

2.4. Diversos

Para usar facilmente scripts Python no Unix, você precisa torná-los executáveis, por exemplo, com

```
$ chmod +x script
```

e colocar uma linha Shebang apropriada no topo do script. Uma boa escolha normalmente é

```
#!/usr/bin/env python3
```

que procura o interpretador do Python no conjunto **`PATH`**. No entanto, alguns sistemas Unix podem não ter o comando **env**, então você pode precisar codificar `/usr/bin/python3` como o caminho do interpretador.

Para usar comandos Shell em seus scripts Python, veja o módulo **`subprocess`**.

2.5. OpenSSL personalizado

1. Para usar a configuração OpenSSL do seu fornecedor e armazenamento confiável do sistema, localize o diretório com o arquivo `openssl.cnf` ou link simbólico em `/etc`. Na maioria das distribuições o arquivo está em `/etc/ssl` ou em `/etc/pki/tls`. O diretório também deve conter um arquivo `cert.pem` e/ou um diretório `certs`.

   ```
   $ find /etc/ -name openssl.cnf -printf "%h\n"
   /etc/ssl
   ```

2. Baixe, construa e instale o OpenSSL. Certifique-se de usar `install_sw` e não `install`. O destino `install_sw` não substitui o `openssl.cnf`.

   ```
   $ curl -O https://www.openssl.org/source/openssl-VERSÃO.tar
   $ tar xzf openssl-VERSÃO
   $ pushd openssl-VERSÃO
   $ ./config \
       --prefix=/usr/local/custom-openssl \
       --libdir=lib \
       --openssldir=/etc/ssl
   $ make -j1 depend
   $ make -j8
   $ make install_sw
   $ popd
   ```

3. Construa o Python com o OpenSSL personalizado (veja as opções configure `--with-openssl` e `--with-openssl-rpath`)

```
$ pushd python-3.x.x
$ ./configure -C \
    --with-openssl=/usr/local/custom-openssl \
    --with-openssl-rpath=auto \
    --prefix=/usr/local/python-3.x.x
$ make -j8
$ make altinstall
```

Nota

As versões de patches do OpenSSL têm uma ABI compatível com versões anteriores. Você não precisa recompilar o Python para atualizar o OpenSSL. É suficiente substituir a instalação personalizada do OpenSSL por uma versão mais recente.

3. Configurando o Python

3.1. Requisitos de construção

Para construir o CPython, você precisará de:

- Um compilador C11 [https://en.cppreference.com/w/c/11]. Não são necessários recursos opcionais do C11 [https://en.wikipedia.org/wiki/C11_(C_standard_revision)#Optional_features].
- No Windows, é necessário o Microsoft Visual Studio 2017 ou posterior.
- Suporte para números de ponto flutuante do IEEE 754 [https://en.wikipedia.org/wiki/IEEE_754] e Not-a-Number (NaN) de ponto flutuante [https://en.wikipedia.org/wiki/NaN#Floating_point].
- Suporte a threads.

Alterado na versão 3.5: No Windows, o Visual Studio 2015 ou posterior agora é obrigatório.

Alterado na versão 3.6: Alguns recurso do C99, como as funções `<stdint.h>` e `static inline`, agora são obrigatórios.

Alterado na versão 3.7: Suporte a threads agora é obrigatório.

Alterado na versão 3.11: Compilador C11, suporte a IEEE 754 e NaN agora são necessários. No Windows, é necessário o Visual Studio 2017 ou posterior.

Veja também **PEP 7** [https://peps.python.org/pep-0007/] “Guia de estilo para código C” e **PEP 11** [https://peps.python.org/pep-0011/] “Suporte do CPython a plataformas”.

3.1.1. Requisitos para módulos opcionais

Alguns módulos opcionais da biblioteca padrão exigem que bibliotecas de terceiros estejam instaladas para o desenvolvimento (por exemplo, os arquivos de cabeçalho devem estar disponíveis).

Requisitos ausentes são relatados na saída do comando `configure`. Módulos que estão faltando devido a dependências ausentes são listados perto do final da saída do comando `make`, às vezes usando um nome interno, por exemplo, `_ctypes` para o módulo **`ctypes`**.

Se você distribuir um interpretador do CPython sem módulos opcionais, é uma boa prática avisar os usuários, que geralmente esperam que os módulos da biblioteca padrão estejam disponíveis.

As dependências para construir módulos opcionais são:

Dependência	Versão mínima	Módulo Python
libbz2		**`bz2`**
libffi	3.3.0 recomendada	**`ctypes`**
liblzma		**`lzma`**
libmpdec	2.5.0	**`decimal`** [[1]]
libreadline ou libedit [[2]]		**`readline`**
libuuid		`_uuid` [[3]]
ncurses [[4]]		**`curses`**

Dependência	Versão mínima	Módulo Python
OpenSSL	[[6]]	`ssl`, `hashlib` [[5]]
SQLite	3.15.2	`sqlite3`
Tcl/Tk	8.5.12	`tkinter`, IDLE, `turtle`
zlib	1.2.2.1	`zlib`, `gzip`, `ensurepip`
zstd	1.4.5	`compression.zstd`

[1]

Se a biblioteca *libmpdec* não estiver disponível, o módulo `decimal` usará uma implementação em Python puro. Consulte `--with-system-libmpdec` para obter detalhes.

[2]

Consulte `--with-readline` para escolher o backend do módulo `readline`.

[3]

O módulo `uuid` usa `_uuid` para gerar UUIDs “seguros”. Consulte a documentação do módulo para obter detalhes.

[4]

O módulo `curses` requer a biblioteca `libncurses` ou `libncursesw`. O módulo `curses.panel` requer adicionalmente a biblioteca `libpanel` ou `libpanelw`.

[5]

Caso o OpenSSL não esteja disponível, o módulo `hashlib` utilizará implementações integradas de diversas funções de hash. Consulte a `--with-builtin-hashlib-hashes` para *forçar* o uso do OpenSSL.

[6]

O OpenSSL 1.1.1 é a versão mínima necessária para a construção, mas essa série chegou ao fim de sua vida útil e não recebe mais correções de segurança públicas. Utilize a versão de correção (*patch release*) mais recente de uma série LTS atualmente suportada (consulte o OpenSSL Roadmap [https://openssl-library.org/roadmap/index.html]) ou o pacote fornecido pelo seu sistema operacional, caso esteja disponível. Outras bibliotecas que ofereçam uma API compatível com o OpenSSL 1.1.1 ou superior podem funcionar, mas não são oficialmente suportadas.

Note que a tabela não inclui todos os módulos opcionais; em particular, módulos específicos da plataforma, como `winreg`, não estão listados aqui.

Ver também

- O devguide [https://devguide.python.org/getting-started/setup-building/#install-dependencies] inclui uma lista completa das dependências necessárias para construir todos os módulos e instruções sobre como instalá-las em plataformas comuns.
- A `--with-system-expat` permite a construir com uma biblioteca externa libexpat [https://libexpat.github.io/].
- Opções para dependências de terceiros

Alterado na versão 3.1: Tcl/Tk versão 8.3.1 agora é obrigatório para `tkinter`.

Alterado na versão 3.5: Tcl/Tk versão 8.4 agora é obrigatório para `tkinter`.

Alterado na versão 3.7: OpenSSL 1.0.2 agora é obrigatório para `hashlib` e `ssl`.

Alterado na versão 3.10: OpenSSL 1.1.1 agora é obrigatório para **hashlib** e **ssl**. SQLite 3.7.15 agora é obrigatório para **sqlite3**.

Alterado na versão 3.11: Tcl/Tk versão 8.5.12 agora é obrigatório para **tkinter**.

Alterado na versão 3.13: SQLite 3.15.2 agora é obrigatório para **sqlite3**.

3.2. Arquivos gerados

Para reduzir as dependências de construção, o código-fonte do Python contém vários arquivos gerados. Comandos para regenerar todos os arquivos gerados:

```
make regen-all
make regen-stdlib-module-names
make regen-limited-abi
make regen-configure
```

O arquivo `Makefile.pre.in` documenta os arquivos gerados, suas entradas e ferramentas usadas para regenerá-los. Procure por alvos `regen-*` de make.

3.2.1. Script configure

O comando `make regen-configure` regera o arquivo `aclocal.m4` e o script `configure` usando o shell script `Tools/build/regen-configure.sh`, o qual usa um contêiner Ubuntu para obter as mesmas versões de ferramentas e ter uma saída reproduzível.

O contêiner é opcional, o seguinte comando pode ser executado localmente:

```
autoreconf -ivf -Werror
```

Os arquivos gerados podem variar dependendo das versões exatas das ferramentas utilizadas. O contêiner usado pelo CPython possui o Autoconf [https://gnu.org/software/autoconf] 2.72, o `aclocal` do Automake [https://www.gnu.org/software/automake] 1.16.5 e o pkg-config [https://www.freedesktop.org/wiki/Software/pkg-config/] 1.8.1.

Alterado na versão 3.13: O Autoconf 2.71 e o aclocal 1.16.5 agora são usados para regenerar o arquivo `configure`.

Alterado na versão 3.14: O Autoconf 2.72 agora é usado para regenerar o arquivo `configure`.

3.3. Opções de configuração

Liste todas as opções do `configure` usando:

```
./configure --help
```

Veja também o `Misc/SpecialBuilds.txt` na distribuição de código-fonte do Python.

3.3.1. Opções gerais

--enable-loadable-sqlite-extensions

Suporte a extensões carregáveis no módulo de extensão **`_sqlite`** (o padrão é não) do módulo **`sqlite3`**.

Veja o método **`sqlite3.Connection.enable_load_extension()`** do módulo **`sqlite3`**.

Adicionado na versão 3.6.

--disable-ipv6

Desabilita suporte a IPv6 (habilitado por padrão se houver suporte), veja o módulo **`socket`**.

--enable-big-digits=[15|30]

Define o tamanho em bits dos dígitos de **`int`** do Python: 15 ou 30 bits.

Por padrão, o tamanho dos dígitos é 30.

Define o `PYLONG_BITS_IN_DIGIT` para `15` ou `30`.

Veja **sys.int_info.bits_per_digit**.

--with-suffix=SUFFIX

Define o sufixo do executável do Python para *SUFFIX*.

O sufixo padrão é `.exe` no Windows e macOS (executável `python.exe`), `.js` em nó Emscripten, `.html` em navegador Emscripten, `.wasm` em WASI e uma string vazia em outras plataformas (executável `python`).

Alterado na versão 3.11: O sufixo padrão na plataforma WASM é um entre `.js`, `.html` ou `.wasm`

--with-tzpath=<list of absolute paths separated by pathsep>

Seleciona o caminho de pesquisa de fuso horário padrão para **zoneinfo.TZPATH**. Veja a Configuração de tempo de compilação do módulo **zoneinfo**.

Padrão: `/usr/share/zoneinfo:/usr/lib/zoneinfo:/usr/share/lib/zoneinfo:/etc/zoneinfo`.

Veja o separador de caminhos **os.pathsep**.

Adicionado na versão 3.9.

--without-decimal-contextvar

Constrói o módulo de extensão `_decimal` usando um contexto local de thread ao invés de um contexto local de corrotina (padrão), veja o módulo **decimal**.

Veja **decimal.HAVE_CONTEXTVAR** e o módulo **contextvars**.

Adicionado na versão 3.9.

--with-dbmliborder=<list of backend names>

Substitui a ordem de verificação de backends de banco de dados para o módulo **dbm**

Um valor válido é uma string separada por dois pontos (`:`) com os nomes de backend:

- `ndbm`;
- `gdbm`;
- `bdb`.

--without-c-locale-coercion

Desabilita a coerção de localidade C para uma localidade baseada em UTF-8 (ativada por padrão).

Não define a macro `PY_COERCE_C_LOCALE`.

Consulte **`PYTHONCOERCECLOCALE`** e a **PEP 538** [https://peps.python.org/pep-0538/].

--with-platlibdir=DIRNAME

Nome do diretório da biblioteca Python (o padrão é `lib`).

Fedora e SuSE usam `lib64` em plataformas de 64 bits.

Veja **`sys.platlibdir`**.

Adicionado na versão 3.9.

--with-wheel-pkg-dir=PATH

Diretório de pacotes de wheel usados pelo módulo **`ensurepip`** (nenhum por padrão).

Algumas políticas de empacotamento de distribuição do Linux recomendam contra o empacotamento de dependências. Por exemplo, o Fedora instala pacotes wheel no diretório `/usr/share/python-wheels/` e não instala o pacote **`ensurepip._bundled`**.

Adicionado na versão 3.10.

--with-pkg-config=[check|yes|no]

Se o configure deve usar **pkg-config** para detectar dependências de construção.

- `check` (padrão): **pkg-config** é opcional
- `yes`: **pkg-config** é obrigatório
- `no`: configure não usa **pkg-config** mesmo quando presente

Adicionado na versão 3.11.

--with-missing-stdlib-config=FILE

Caminho para um arquivo de configuração JSON [https://www.json.org/json-pt.html] contendo mensagens de erro personalizadas para módulos da biblioteca padrão ausentes.

Esta opção destina-se a distribuidores de Python que desejam fornecer orientações específicas da distribuição quando os usuários encontrarem módulos da biblioteca padrão que estejam ausentes ou empacotados separadamente.

O arquivo JSON deve mapear nomes de módulos ausentes para strings de mensagens de erro personalizadas. Por exemplo, se a sua distribuição empacota **`tkinter`** e **`_tkinter`** separadamente e exclui **`_gdbm`** por motivos legais, a configuração poderia conter:

```
{
    "_gdbm": "The '_gdbm' module is not available in this dis
    "tkinter": "Install the python-tk package to use tkinter'
    "_tkinter": "Install the python-tk package to use tkinter
}
```

Adicionado na versão 3.15.

--enable-pystats

Ativa a coleta de estatísticas internas de desempenho do Python.

Por padrão, a coleta de estatísticas está desativada. Use o comando `python3 -X pystats` ou defina a variável de ambiente `PYTHONSTATS=1` para ativar a coleta de estatísticas na inicialização do Python.

Na saída do Python, despeja as estatísticas se a coleta de estatísticas estiver ativada e não apagada.

Efeitos:

- Adiciona a opção de linha de comando **`-X pystats`**.
- Adiciona a variável de ambiente **`PYTHONSTATS`**.
- Define a macro `Py_STATS`.
- Adiciona funções ao módulo **`sys`**:
  - **`sys._stats_on()`**: Ativa a coleta de estatísticas.
  - **`sys._stats_off()`**: Desativa a coleta de estatísticas.
  - **`sys._stats_clear()`**: Apaga as estatísticas.
  - **`sys._stats_dump()`**: Despeja as estatísticas no arquivo e apaga as estatísticas.

As estatísticas serão despejadas em um arquivo arbitrário (provavelmente único) em `/tmp/py_stats/` (Unix) ou `C:\temp\py_stats\` (Windows). Se aquele diretório não existir, os resultados serão enviados para stderr.

Use `Tools/scripts/summarize_stats.py` para ler as estatísticas.

Estatísticas:

- Código de operação:
  - Especialização: sucesso, fracasso, acerto, adiado, erro, deopt, falhas;
  - Contagem de execuções;
  - Contagem em pares.
- Chamada:
  - Chamadas Python em linha;
  - Chamadas a PyEval;
  - Quadros enviados;
  - Objetos quadro criados;
  - Chamadas de Eval: vetor, gerador, legado, função VECTORCALL, classe de construção, slot, função “ex”, API, método.
- Objeto:

    - incref e decref;
    - incref e decref do interpretador;
    - alocações: tudo, 512 bytes, 4 kiB, big;
    - memória livre;
    - listas de memória livre para/de;
    - Dicionário materializado/desmaterializado;
    - cache de tipo;
    - tentativas de otimização;
    - rastros de otimização criados/executados;
    - uops executados.
  - Coletor de lixo:
    - Coletas de lixo;
    - Objetos visitados;
    - Objetos coletados.

Adicionado na versão 3.11.

--disable-gil

Habilita suporte para execução de Python sem a trava global do interpretador (GIL): construção com threads livres.

Define a macro `Py_GIL_DISABLED` e adiciona `"t"` a **`sys.abiflags`**.

Veja CPython com threads livres para mais detalhes.

Adicionado na versão 3.13.

--enable-experimental-jit=[no|yes|yes-off|interpreter]

Indica como integrar o compilador just-in-time experimental.

- `no`: não constrói o JIT;
- `yes`: habilita o JIT. Para desabilitá-lo em tempo de execução, defina a variável de ambiente **`PYTHON_JIT=0`**.
- `yes-off`: constrói o JIT, mas desabilita-o por padrão. Para habilitá-lo em tempo de execução, defina a variável de ambiente **`PYTHON_JIT=1`**.
- `interpreter`: Habilita o “interpretador JIT” (útil apenas para quem está depurando o próprio JIT). Para desabilitá-lo em tempo de

execução, defina a variável de ambiente **PYTHON_JIT=0**.

`--enable-experimental-jit=no` é o comportamento padrão se a opção não for fornecida, e `--enable-experimental-jit` é uma abreviação de `--enable-experimental-jit=yes`. Consulte `Tools/jit/README.md` para mais informações, incluindo como instalar as dependências necessárias em tempo de construção.

Nota

Ao construir o CPython com o JIT habilitado, certifique-se de que seu sistema tenha o Python 3.11 ou posterior instalado.

Adicionado na versão 3.13.

PKG_CONFIG

Caminho para o utilitário `pkg-config`.

PKG_CONFIG_LIBDIR

PKG_CONFIG_PATH

Opções do `pkg-config`.

--disable-epoll

Construir sem `epoll`, o que significa que **`select.epoll()`** não estará presente, mesmo que o sistema forneça uma função *epoll_create* *[https://manpages.debian.org/epoll_create(2)]*. Isso pode ser utilizado em sistemas onde *epoll_create* ou *epoll_create1* *[https://manpages.debian.org/epoll_create1(2)]* estão disponíveis, mas são incompatíveis com a semântica do Linux.

Adicionado na versão 3.15.

3.3.2. Opções do compilador C

CC

Comando do compilador C.

CFLAGS

Sinalizadores do compilador C.

CPP
: Comando do pré-processador C.

CPPFLAGS
: Sinalizadores do pré-processador C, p.ex., `-I`*`include_dir`*.

3.3.3. Opções da ligação

LDFLAGS
: Sinalizadores do vinculador. p.ex., `-L`*`library_directory`*.

LIBS
: Bibliotecas para passar para o vinculador, p.ex. `-l`*`library`*.

MACHDEP
: Nome para arquivos de biblioteca dependentes de máquina.

3.3.4. Opções para dependências de terceiros

Adicionado na versão 3.11.

BZIP2_CFLAGS

BZIP2_LIBS
: Sinalizadores de compilador C e vinculador para vincular Python a `libbz2`, usados pelo módulo **`bz2`**, substituindo `pkg-config`.

CURSES_CFLAGS

CURSES_LIBS
: Sinalizadores de compilador C e vinculador para `libncurses` ou `libncursesw`, usados pelo módulo **`curses`**, substituindo `pkg-config`.

GDBM_CFLAGS

GDBM_LIBS

Sinalizadores de compilador C e vinculador para `gdbm`.

LIBEDIT_CFLAGS

LIBEDIT_LIBS

Sinalizadores de compilador C e vinculador para `libedit`, usados pelo módulo **`readline`**, substituindo `pkg-config`.

LIBFFI_CFLAGS

LIBFFI_LIBS

Sinalizadores de compilador C e vinculador para `libffi`, usados pelo módulo **`ctypes`**, substituindo `pkg-config`.

LIBMPDEC_CFLAGS

LIBMPDEC_LIBS

Sinalizadores de compilador C e vinculador para `libmpdec`, usados pelo módulo **`decimal`**, substituindo `pkg-config`.

Nota

Estas variáveis de ambiente não têm efeito a menos que **`--with-system-libmpdec`** seja especificado.

LIBLZMA_CFLAGS

LIBLZMA_LIBS

Sinalizadores de compilador C e vinculador para `liblzma`, usados pelo módulo **`lzma`**, substituindo `pkg-config`.

LIBREADLINE_CFLAGS

LIBREADLINE_LIBS

Sinalizadores de compilador C e vinculador para `libreadline`, usados pelo módulo **`readline`**, substituindo `pkg-config`.

LIBSQLITE3_CFLAGS

LIBSQLITE3_LIBS

Sinalizadores de compilador C e vinculador para `libsqlite3`, usados pelo módulo **`sqlite3`**, substituindo `pkg-config`.

LIBUUID_CFLAGS

LIBUUID_LIBS

Sinalizadores de compilador C e vinculador para `libuuid`, usados pelo módulo **`uuid`**, substituindo `pkg-config`.

LIBZSTD_CFLAGS

LIBZSTD_LIBS

Sinalizadores de compilador C e ligador para `libzstd`, usados pelo módulo **`compression.zstd`**, substituindo `pkg-config`.

Adicionado na versão 3.14.

PANEL_CFLAGS

PANEL_LIBS

Sinalizadores de compilador C e vinculador para PANEL, substituindo `pkg-config`.

Sinalizadores de compilador C e vinculador para `libpanel` ou `libpanelw`, usados pelo módulo **`curses.panel`**, substituindo `pkg-config`.

TCLTK_CFLAGS

TCLTK_LIBS

Sinalizadores de compilador C e vinculador para TCLTK, substituindo `pkg-config`.;

ZLIB_CFLAGS

ZLIB_LIBS

Sinalizadores de compilador C e vinculador para `libzlib`, usados pelo módulo **`gzip`**, substituindo `pkg-config`.

3.3.5. Opções de WebAssembly

--enable-wasm-dynamic-linking

Ativa o suporte de vinculação dinâmica para WASM.

A vinculação dinâmica permite `dlopen`. O tamanho do arquivo executável aumenta devido à eliminação limitada de código morto e recursos adicionais.

Adicionado na versão 3.11.

--enable-wasm-pthreads

Ativa o suporte a pthreads para WASM.

Adicionado na versão 3.11.

3.3.6. Opções de instalação

--prefix=PREFIX

Instala arquivos independentes de arquitetura em PREFIX. No Unix, o padrão é `/usr/local`.

Este valor pode ser recuperado em tempo de execução usando **`sys.prefix`**.

Como exemplo, pode-se usar `--prefix="$HOME/.local/"` para instalar um Python em seu diretório pessoal (home).

--exec-prefix=EPREFIX

Instala arquivos dependentes de arquitetura no EPREFIX, o padrão é **`--prefix`**.

Este valor pode ser recuperado em tempo de execução usando **sys.exec_prefix**.

--disable-test-modules

Não constrói nem instala módulos de teste, como o pacote **test** ou o módulo de extensão **_testcapi** (construído e instalado por padrão).

Adicionado na versão 3.10.

--with-ensurepip=[upgrade|install|no]

Seleciona o comando **ensurepip** executado na instalação do Python:

- `upgrade` (padrão): executa o comando `python -m ensurepip --altinstall --upgrade`.
- `install`: executa o comando `python -m ensurepip --altinstall`;
- `no`: não executa ensurepip;

Adicionado na versão 3.6.

3.3.7. Opções de desempenho

Configurar o Python usando `--enable-optimizations --with-lto` (PGO + LTO) é o recomendado para melhor desempenho. O sinalizador experimental `--enable-bolt` também pode ser usado para melhorar o desempenho.

--enable-optimizations

Habilita a otimização guiada por perfil (PGO, do inglês Profile Guided Optimization) usando **PROFILE_TASK** (desabilitado por padrão).

O compilador C Clang requer o programa `llvm-profdata` para PGO. No macOS, o GCC também exige: o GCC é apenas um apelido para o Clang no macOS.

Desabilita também a interposição semântica no libpython se `--enable-shared` e GCC forem usados: adiciona `-fno-semantic-interposition` aos sinalizadores do compilador e do vinculador.

Nota

Durante a construção, você poderá encontrar avisos do compilador sobre a indisponibilidade de dados de perfil para alguns arquivos fonte. Esses avisos são inofensivos, pois apenas um subconjunto do código é exercido durante a aquisição de dados de perfil. Para desativar esses avisos no Clang, suprima-os manualmente adicionando `-Wno-profile-instr-unprofiled` a **`CFLAGS`**.

Adicionado na versão 3.6.

Alterado na versão 3.10: Usa `-fno-semantic-interposition` no GCC.

PROFILE_TASK

Variável de ambiente usada no Makefile: argumentos de linha de comando do Python para a tarefa de geração de PGO.

Padrão: `-m test --pgo --timeout=$(TESTTIMEOUT)`.

Adicionado na versão 3.8.

Alterado na versão 3.13: A falha da tarefa não é mais ignorada silenciosamente.

--with-lto=[full|thin|no|yes]

Habilita o otimização em tempo de vinculação (LTO, do inglês Link Time Optimization) em qualquer construção (desabilitado por padrão).

O compilador C Clang requer `llvm-ar` para LTO (`ar` no macOS), bem como um vinculador compatível com LTO (`ld.gold` ou `lld`).

Adicionado na versão 3.6.

Adicionado na versão 3.11: Para usar o recurso ThinLTO, use `--with-lto=thin` no Clang.

Alterado na versão 3.12: Usa ThinLTO como política de otimização padrão no Clang se o compilador aceitar o sinalizador.

--enable-bolt

Habilita o uso do otimizador binário pós-vinculação BOLT [https://github.com/llvm/llvm-project/tree/main/bolt] (desabilitado por padrão).

BOLT faz parte do projeto LLVM, mas nem sempre está incluído em suas distribuições binárias. Este sinalizador requer que `llvm-bolt` e `merge-fdata` estejam disponíveis.

BOLT ainda é um projeto relativamente novo, então este sinalizador deve ser considerado experimental por enquanto. Como esta ferramenta opera em código de máquina, seu sucesso depende de uma combinação do ambiente de construção + os outros argumentos de configuração de otimização + a arquitetura da CPU, e nem todas as combinações são suportadas. Versões do BOLT anteriores ao LLVM 16 são conhecidas por travar o BOLT em alguns cenários. O uso do LLVM 16 ou mais recente para otimização do BOLT é fortemente incentivado.

As variáveis **`BOLT_INSTRUMENT_FLAGS`** e **`BOLT_APPLY_FLAGS`** do **configure** podem ser definidas para substituir o conjunto padrão de argumentos para **llvm-bolt** para instrumentar e aplicar dados BOLT aos binários , respectivamente.

Adicionado na versão 3.12.

BOLT_APPLY_FLAGS

Argumentos para `llvm-bolt` ao criar um binário otimizado com BOLT [https://github.com/facebookarchive/BOLT].

Adicionado na versão 3.12.

BOLT_INSTRUMENT_FLAGS

Argumentos para `llvm-bolt` ao instrumentar binários.

Adicionado na versão 3.12.

--with-computed-gotos

Habilita “gotos” computados no laço de avaliação (habilitado por padrão em compiladores suportados).

--with-tail-call-interp

Habilita interpretadores usando chamadas de cauda no CPython. Se habilitado, é altamente recomendável habilitar PGO (`--enable-optimizations`). Esta opção requer especificamente um compilador C com suporte adequado a chamadas de cauda e a convenção de chamada preserve_none [https://clang.llvm.org/docs/AttributeReference.html#preserve-none]. Por exemplo, o Clang 19 e versões mais recentes oferecem suporte a esse recurso.

Adicionado na versão 3.14.

--without-frame-pointers

Desabilita os ponteiros de quadro, que estão habilitados por padrão (veja **PEP 831** [https://peps.python.org/pep-0831/]).

Por padrão, o processo de construção acrescenta sinalizadores a `BASECFLAGS` para gerar ponteiros de quadro (*frame pointers*) ou de encadeamento (*backchain pointers*):

- `-fno-omit-frame-pointer` e/ou `-mno-omit-leaf-frame-pointer` são adicionados quando o compilador oferece suporte a eles.
- `-marm` e/ou `-mno-thumb` são adicionados em sistemas ARM de 32 bits quando suportados.
- em plataformas s390x, quando suportado, `-mbackchain` é adicionado *em vez* das flags de ponteiro de quadro mencionadas acima.
- em plataformas ppc64le, não são necessárias sinalizadores de compilador, uma vez que a ABI Power exige que os compiladores mantenham um encadeamento (*back chain*) por padrão.

Ponteiros de quadro (*frame pointers*) permitem que ferramentas de perfilamento, depuração e rastreamento do sistema (como `perf`, `eBPF`, `dtrace` e `gdb`) percorram a pilha de chamadas em C sem a necessidade de metadados DWARF. Os sinalizadores são propagadas para extensões C de terceiros por meio do módulo `sysconfig`. Em compiladores que não as reconhecem, o processo de construção as ignora silenciosamente.

Empacotadores downstream e autores de bibliotecas nativas criadas com sistemas de construção personalizados devem definir as mesmas flags para que a cadeia de desenrolamento (unwind chain) permaneça ininterrupta em todos os frames nativos.

Adicionado na versão 3.15.

--without-mimalloc

Desativa o alocador rápido mimalloc (habilitado por padrão).

Esta opção não pode ser usada junto com **`--disable-gil`** porque a construção com threads livres requer mimalloc.

Veja também a variável de ambiente **`PYTHONMALLOC`**.

--without-pymalloc

Desabilita o alocador de memória especializado do Python pymalloc (habilitado por padrão).

Veja também a variável de ambiente **`PYTHONMALLOC`**.

--with-pymalloc-hugepages

Habilita o suporte a *huge pages* para arenas do pymalloc (desabilitado por padrão). Quando habilitado, o tamanho da arena em plataformas de 64 bits é aumentado para 2 MiB e a alocação da arena utiliza `MAP_HUGETLB` (Linux) ou `MEM_LARGE_PAGES` (Windows), com *fallback* automático para páginas comuns.

Mesmo quando compilado com essa opção, *huge pages* **não** são utilizadas em tempo de execução, a menos que a variável de ambiente **`PYTHON_PYMALLOC_HUGEPAGES`** esteja definida como `1`. Essa ativação explícita é necessária porque *huge pages*

- envolve riscos no Linux: se o pool de *huge pages* se esgotar, falhas de página (incluindo falhas de *copy-on-write* após **`os.fork()`**) geram um sinal `SIGBUS` e encerram forçadamente o processo.
- precisa de um privilégio especial no Windows. Consulte a documentação do Windows para páginas grandes

[https://learn.microsoft.com/pt-br/windows/win32/memory/large-page-support] para obter detalhes. Python falhará na inicialização se o privilégio necessário SeLockMemoryPrivilege [https://learn.microsoft.com/pt-br/previous-versions/windows/it-pro/windows-10/security/threat-protection/security-policy-settings/lock-pages-in-memory] não for concedido ao usuário.

O script de configuração verifica se a plataforma oferece suporte a `MAP_HUGETLB` e emite um aviso caso não esteja disponível.

No Windows, use o sinalizador `--pymalloc-hugepages` com o `build.bat` ou defina a propriedade MSBuild `UsePymallocHugepages`.

Adicionado na versão 3.15.

--without-doc-strings

Desabilita as strings de documentação estática para reduzir o consumo de memória (habilitado por padrão). As strings de documentação definidas em Python não são afetadas.

Não define a macro `WITH_DOC_STRINGS`.

Veja a macro `PyDoc_STRVAR()`.

--enable-profiling

Habilita o perfil de código a nível C com `gprof` (desabilitado por padrão).

--with-strict-overflow

Adiciona `-fstrict-overflow` aos sinalizadores do compilador C (por padrão adicionamos `-fno-strict-overflow`).

--without-remote-debug

Desativa o suporte à depuração remota descrito na **PEP 768** [https://peps.python.org/pep-0768/] (habilitado por padrão). Quando este sinalizador é fornecido, o código que permite ao interpretador agendar a execução de um arquivo Python em um processo separado, conforme descrito na **PEP 768** [https://peps.python.org/pep-0768/], não é compilado. Isso

inclui tanto a funcionalidade de agendar a execução do código quanto a funcionalidade de receber a execução do código.

Py_REMOTE_DEBUG

Esta macro é definida por padrão, a menos que o Python esteja configurado com `--without-remote-debug`.

Observe que, mesmo que a macro esteja definida, a depuração remota pode não estar disponível (por exemplo, em uma plataforma incompatível).

Adicionado na versão 3.14.

3.3.8. Compilação de depuração do Python

Uma compilação de depuração é Python compilada com a opção de configuração `--with-pydebug`.

Efeitos de uma compilação de depuração:

- Exibe todos os avisos por padrão: a lista de filtros de aviso padrão está vazia no módulo `warnings`.
- Adiciona `d` a `sys.abiflags`.
- Adiciona a função **`sys.gettotalrefcount()`**.
- Adiciona a opção de linha de comando `-X showrefcount`.
- Adiciona a opção de linha de comando `-d` e a variável de ambiente `PYTHONDEBUG` para depurar o analisador sintático.
- Adiciona suporte para a variável `__lltrace__`: habilita o rastreamento de baixo nível no laço de avaliação de bytecode se a variável estiver definida.
- Instala ganchos de depuração nos alocadores de memória para detectar estouro de buffer e outros erros de memória.
- Define as macros `Py_DEBUG` e `Py_REF_DEBUG`.
- Adiciona verificações de tempo de execução: código cercado por `#ifdef Py_DEBUG` e `#endif`. Habilita as asserções `assert(...)` e `_PyObject_ASSERT(...)`: não define a macro `NDEBUG` (veja também a

configuração `--with-assertions` opção). Principais verificações de tempo de execução:

- Adiciona verificações de sanidade nos argumentos da função.
- Objetos Unicode e int são criados com sua memória preenchida com um padrão para detectar o uso de objetos não inicializados.
- Garante que as funções que podem limpar ou substituir a exceção atual não sejam chamadas com uma exceção levantada.
- Verifica se as funções desalocadoras não alteram a exceção atual.
- O coletor de lixo (função `gc.collect()`) executa algumas verificações básicas na consistência dos objetos.
- A macro **`Py_SAFE_DOWNCAST()`** verifica o underflow e o overflow de inteiros ao fazer o downcast de tipos largos para tipos estreitos.

Veja também o Modo de Desenvolvimento do Python e a opção de configuração `--with-trace-refs`.

Alterado na versão 3.8: Construções de lançamento agora são compatíveis em ABI com construções de depuração: definir a macro `Py_DEBUG` não implica mais na macro `Py_TRACE_REFS` (consulte a opção `--with-trace-refs`). No entanto, construções de depuração ainda expõem mais símbolos que as construções de lançamento e o código construído no modo depuração não é necessariamente compatível com uma construção de lançamento.

3.3.9. Opções de depuração

--with-pydebug

Construção de depuração do Python: define a macro `Py_DEBUG` (desabilitada por padrão).

--with-trace-refs

Habilita referências de rastreamento para fins de depuração (desabilitado por padrão).

Efeitos:

- Define a macro `Py_TRACE_REFS`.

- Adiciona a função `sys.getobjects()`.
- Adiciona a variável de ambiente `PYTHONDUMPREFS`.

A variável de ambiente `PYTHONDUMPREFS` pode ser usada para despejar objetos e contagens de referências ainda ativas na saída do Python.

Objetos alocados estaticamente não são rastreados.

Adicionado na versão 3.8.

Alterado na versão 3.13: Esta construção agora é compatibilidade de ABI com a construção de lançamento e construção de depuração.

--with-assertions

Constrói com asserções C habilitadas (o padrão é não): `assert(...);` e `_PyObject_ASSERT(...);`.

Se definido, a macro `NDEBUG` não é definida na variável do compilador `OPT`.

Veja também a opção `--with-pydebug` (construção de depuração) que também habilita asserções.

Adicionado na versão 3.6.

--with-valgrind

Habilita suporte ao Valgrind (o padrão é não).

--with-dtrace

Habilita suporte ao DTrace (o padrão é não).

Veja Instrumentando o CPython com DTrace e SystemTap.

Adicionado na versão 3.6.

--with-address-sanitizer

Habilita o detector de erros de memória AddressSanitizer, `asan` (o padrão é não). Para melhorar os recursos de detecção do ASan, você também pode combinar isso com `--without-pymalloc` para desabilitar

o alocador especializado de pequenos objetos cujas alocações não são rastreadas pelo ASan.

Adicionado na versão 3.6.

--with-memory-sanitizer

Habilita o detector de erros de alocação do MemorySanitizer, `msan` (o padrão é não).

Adicionado na versão 3.6.

--with-undefined-behavior-sanitizer

Habilita o detector de comportamento indefinido UndefinedBehaviorSanitizer, `ubsan` (o padrão é não).

Adicionado na versão 3.6.

--with-thread-sanitizer

Habilita o detector de corrida de dados ThreadSanitizer, `tsan` (o padrão é não).

Adicionado na versão 3.13.

3.3.10. Opções da ligação

--enable-shared

Habilita a construção de uma biblioteca Python compartilhada: `libpython` (o padrão é não).

--without-static-libpython

Não constrói `libpythonMAJOR.MINOR.a` e não instala `python.o` (construído e habilitado por padrão).

Adicionado na versão 3.10.

--enable-static-libpython-for-interpreter

Não vincula o binário do interpretador Python (`python3`) à biblioteca compartilhada do Python; em vez disso, vincula o interpretador estaticamente à `libpython` como se a opção `--enable-shared` não tivesse sido utilizada, mas continua a construir a `libpython` compartilhada (para uso por outros programas).

Esta opção faz nada se `--enable-shared` não for usada.

O comportamento padrão (quando `-enable-shared` é utilizado) é vincular o interpretador Python à biblioteca compartilhada compilada.

Adicionado na versão 3.15.

3.3.11. Opções da biblioteca

--with-libs='lib1 ...'

Vincula bibliotecas adicionais (o padrão é não).

--with-system-expat

Constrói o módulo **`pyexpat`** usando uma biblioteca `expat` instalada (o padrão é não).

--with-system-libmpdec

Constrói o módulo de extensão `_decimal` usando uma biblioteca `mpdecimal` instalada, veja o módulo **`decimal`** (o padrão é sim).

Adicionado na versão 3.3.

Alterado na versão 3.13: O padrão é usar a biblioteca `mpdecimal` instalada.

Alterado na versão 3.15: Uma cópia da biblioteca incluída no pacote não será mais selecionada implicitamente se uma biblioteca `mpdecimal` instalada não for encontrada. Somente no Python 3.15, ela ainda pode ser selecionada explicitamente usando `--with-system-libmpdec=no` ou `--without-system-libmpdec`.

Descontinuado desde a versão 3.13, será removido na versão 3.16: Uma cópia dos fontes da biblioteca `mpdecimal` não será mais distribuída com Python 3.16.

Ver também

`LIBMPDEC_CFLAGS` e `LIBMPDEC_LIBS`.

--with-readline=readline|editline

Designa uma biblioteca backend para o módulo `readline`.

- readline: Usa readline como o backend.
- editline: Usa editline como o backend.

Adicionado na versão 3.10.

--without-readline

Não constrói o módulo `readline` (construído por padrão).

Não define a macro `HAVE_LIBREADLINE`.

Adicionado na versão 3.10.

--with-libm=STRING

Substitui a biblioteca matemática `libm` por *STRING* (o padrão depende do sistema).

--with-libc=STRING

Substitui a biblioteca C `libc` por *STRING* (o padrão depende do sistema).

--with-openssl=DIR

Raiz do diretório OpenSSL.

Adicionado na versão 3.7.

--with-openssl-rpath=[no|auto|DIR]

Define o diretório da biblioteca de tempo de execução (rpath) para bibliotecas OpenSSL:

- `no` (padrão): não define o rpath;
- `auto`: detecta automaticamente o rpath de `--with-openssl` e `pkg-config`;
- *DIR*: define um rpath explícito.

Adicionado na versão 3.10.

3.3.12. Opções de segurança

--with-hash-algorithm=[fnv|siphash13|siphash24]

Seleciona o algoritmo de hash para usar em `Python/pyhash.c`:

- `siphash13` (padrão);
- `siphash24`;
- `fnv`.

Adicionado na versão 3.4.

Adicionado na versão 3.11: `siphash13` é adicionado e é o novo padrão.

--with-builtin-hashlib-hashes=md5,sha1,sha256,sha512,sha3,blake2

Módulos embutidos de hash

- `md5`;
- `sha1`;
- `sha256`;
- `sha512`;
- `sha3` (com shake);
- `blake2`.

Adicionado na versão 3.9.

--with-ssl-default-suites=[python|openssl|STRING]

Substitui a string dos conjuntos de criptografia padrão do OpenSSL:

- `python` (padrão): use a seleciona preferida do Python;

- `openssl`: mantém inalterados os padrões do OpenSSL;
- *STRING*: usa uma string personalizada

Veja o módulo **`ssl`**.

Adicionado na versão 3.7.

Alterado na versão 3.10: As configurações `python` e *STRING* também definem TLS 1.2 como versão mínima do protocolo.

--disable-safety

Desabilita as opções do compilador recomendadas pela OpenSSF [https://github.com/ossf/wg-best-practices-os-developers/blob/main/docs/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.md] por motivos de segurança, sem causar sobrecarga de desempenho. Se esta opção não estiver habilitada, o CPython será construído com base nas opções de segurança do compilador, sem causar lentidão. Quando esta opção estiver habilitada, o CPython não será construído com as opções do compilador listadas abaixo.

As seguintes opções do compilador são desabilitadas com **`--disable-safety`**:

- -fstack-protector-strong [https://github.com/ossf/wg-best-practices-os-developers/blob/main/docs/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.md#enable-run-time-checks-for-stack-based-buffer-overflows]: Habilita verificações em tempo de execução para estouros de buffer baseados em pilha.
- -Wtrampolines [https://github.com/ossf/wg-best-practices-os-developers/blob/main/docs/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.md#enable-warning-about-trampolines-that-require-executable-stacks]: Habilita avisos sobre trampolins que exigem pilhas executáveis.

Adicionado na versão 3.14.

--enable-slower-safety

Habilita as opções do compilador recomendadas pela OpenSSF [https://github.com/ossf/wg-best-practices-os-developers/blob/main/docs/Compiler-Hardening-

Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.md] por motivos de segurança, o que causa sobrecarga. Se esta opção não estiver habilitada, o CPython não será construído com base nas opções de segurança do compilador, o que impacta o desempenho. Quando esta opção estiver habilitada, o CPython será construído com as opções do compilador listadas abaixo.

As seguintes opções do compilador são habilitadas com **`--enable-slower-safety`**:

- -D_FORTIFY_SOURCE=3 [https://github.com/ossf/wg-best-practices-os-developers/blob/main/docs/Compiler-Hardening-Guides/Compiler-Options-Hardening-Guide-for-C-and-C++.md#fortify-sources-for-unsafe-libc-usage-and-buffer-overflows]: Fortalece as fontes com verificações de tempo de compilação e execução para uso inseguro de libc e estouros de buffer.

Adicionado na versão 3.14.

3.3.13. Opções do macOS

Veja Mac/README.rst [https://github.com/python/cpython/tree/3.15/Mac/README.rst].

--enable-universalsdk

--enable-universalsdk=SDKDIR

Cria uma construção binária universal. *SDKDIR* especifica qual SDK do macOS deve ser usado para executar a construção (o padrão é não).

--enable-framework

--enable-framework=INSTALLDIR

Cria um Python.framework em vez de uma instalação tradicional do Unix. O *INSTALLDIR* opcional especifica o caminho de instalação (o padrão é não).

--with-universal-archs=ARCH

Especifica o tipo de binário universal que deve ser criado. Esta opção só é válida quando **`--enable-universalsdk`** está definido.

Opções:

- `universal2` (x86-64 e arm64);
- `32-bit` (PPC e i386);
- `64-bit` (PPC64 e x86-64);
- `3-way` (i386, PPC e x86-64);
- `intel` (i386 e x86-64);
- `intel-32` (i386);
- `intel-64` (x86-64);
- `all` (PPC, i386, PPC64 e x86-64).

Observe que os valores para este item de configuração *não* são os mesmos que os identificadores usados para rodas binárias universais no macOS. Veja o Guia de Empacotamento do Python para detalhes sobre as tags de compatibilidade de plataforma de empacotamento usadas no macOS [https://packaging.python.org/pt-br/latest/specifications/platform-compatibility-tags/#macos]

--with-framework-name=FRAMEWORK

Especifica o nome do framework python no macOS válido apenas quando **`--enable-framework`** está definido (padrão: `Python`).

--with-app-store-compliance

--with-app-store-compliance=PATCH-FILE

A biblioteca padrão do Python contém strings que são conhecidas por acionar erros de ferramentas de inspeção automatizadas quando enviadas para distribuição pelas App Stores do macOS e do iOS. Se ativada, esta opção aplicará a lista de patches que corrigem a conformidade da app store. Um arquivo de patch personalizado também pode ser especificado. Esta opção está desativada por padrão.

Adicionado na versão 3.13.

3.3.14. Opções do iOS

Veja Platforms/Apple/iOS/README.md [https://github.com/python/cpython/tree/3.15/Platforms/Apple/iOS/README.md].

--enable-framework=INSTALLDIR

Cria um Python.framework. Ao contrário do macOS, o argumento *INSTALLDIR* que especifica o caminho de instalação é obrigatório.

--with-framework-name=FRAMEWORK

Especifica o nome do framework (padrão: `Python`).

3.3.15. Opções de compilação cruzada

A compilação cruzada, também conhecida como construção cruzada, pode ser usada para construir Python para outra arquitetura ou plataforma de CPU. A compilação cruzada requer um interpretador Python para a plataforma de construção. A versão do Python para construção deve corresponder à versão do Python da compilação cruzada do host.

--build=BUILD

configura para construir em BUILD, geralmente adivinhado por **config.guess**.

--host=HOST

faz compilação cruzada para construir programas para executar no HOST (plataforma de destino)

--with-build-python=path/to/python

caminho para construir o binário `python` para compilação cruzada

Adicionado na versão 3.11.

CONFIG_SITE=file

Uma variável de ambiente que aponta para um arquivo com substituições de configuração.

Exemplo de arquivo *config.site*:

```
# config.site-aarch64
ac_cv_buggy_getaddrinfo=no
ac_cv_file__dev_ptmx=yes
ac_cv_file__dev_ptc=no
```

HOSTRUNNER

Programa para executar CPython para a plataforma host para compilação cruzada.

Adicionado na versão 3.11.

Exemplo de compilação cruzada:

```
CONFIG_SITE=config.site-aarch64 ../configure \
    --build=x86_64-pc-linux-gnu \
    --host=aarch64-unknown-linux-gnu \
    --with-build-python=../x86_64/python
```

3.4. Sistema de Construção Python

3.4.1. Arquivos principais do sistema de construção

- `configure.ac` => `configure`;
- `Makefile.pre.in` => `Makefile` (criado por `configure`);
- `pyconfig.h` (criado por `configure`);
- `Modules/Setup`: Extensões C construídas pelo Makefile usando o shell script `Module/makesetup`;

3.4.2. Principais etapas de construção

- Arquivos C (`.c`) são construídos como arquivos objeto (`.o`).
- Uma biblioteca estática `libpython` (`.a`) é criada a partir de arquivos de objetos.
- `python.o` e a biblioteca estática `libpython` estão vinculadas ao programa final `python`.
- Extensões C são construídas pelo Makefile (veja `Modules/Setup`).

3.4.3. Alvos principais do Makefile

3.4.3.1. make

Na maioria das vezes, ao reconstruir após editar algum código ou atualizar seu checkout do upstream, tudo que você precisa fazer é executar `make`, que (pela semântica do Make) constrói o alvo padrão, o primeiro definido no Makefile. Por tradição (inclusive no projeto CPython) este é geralmente o alvo `all`. O script `configure` expande uma variável `autoconf`, `@DEF_MAKE_ALL_RULE@` para descrever precisamente quais alvos `make all` serão construídos. As três opções são:

- `profile-opt` (configurado com `--enable-optimizations`)
- `build_wasm` (escolhido se a plataforma host corresponder a `wasm32-wasi*` ou `wasm32-emscripten`)
- `build_all` (configurado sem usar explicitamente nenhum dos outros)

Dependendo das alterações mais recentes no arquivo fonte, o Make irá reconstruir quaisquer alvos (arquivos objetos e executáveis) considerados desatualizados, incluindo executar `configure` novamente se necessário. As dependências de origem/alvo são muitas e mantidas manualmente, portanto, às vezes, o Make não tem todas as informações necessárias para detectar corretamente todos os alvos que precisam ser reconstruídos. Dependendo de quais destinos não são reconstruídos, você poderá enfrentar vários problemas. Se você tiver problemas de construção ou teste que você não pode explicar de outra forma, `make clean && make` deve resolver a maioria dos problemas de dependência, às custas de tempos de construção mais longos.

3.4.3.2. make platform

Constrói o programa `python`, mas não constrói os módulos de extensão da biblioteca padrão. Isto gera um arquivo chamado `platform` que contém uma única linha descrevendo os detalhes da plataforma de construção, por exemplo, `macosx-14.3-arm64-3.12` ou `linux-x86_64-3.13`.

3.4.3.3. make profile-opt

Constrói Python usando otimização guiada por perfil (PGO). Você pode usar a opção **`--enable-optimizations`** do configure para tornar este o alvo padrão do comando `make` (`make all` ou apenas `make`).

3.4.3.4. make clean

Remove arquivos construídos.

3.4.3.5. make distclean

Além do trabalho feito por `make clean`, remove os arquivos criados pelo script configure. `configure` terá que ser executado antes de construir novamente. [[7]]

3.4.3.6. make install

Constrói o alvo `all` e instala o Python.

3.4.3.7. make test

Constrói o alvo `all` e execute o conjunto de testes Python com a opção `--fast-ci` sem testes de GUI. Variáveis:

- `TESTOPTS`: opções adicionais de linha de comando regrtest.
- `TESTPYTHONOPTS`: opções adicionais de linha de comando do Python.
- `TESTTIMEOUT`: tempo limite em segundos (padrão: 10 minutos).

3.4.3.8. make ci

Isso é semelhante ao `make test`, mas usa `-ugui` para também executar testes de GUI.

Adicionado na versão 3.14.

3.4.3.9. make buildbottest

Isto é semelhante ao `make test`, mas usa a opção `--slow-ci` e o tempo limite padrão de 20 minutos, em vez da opção `--fast-ci`.

3.4.3.10. make regen-all

Gera novamente (quase) todos os arquivos gerados. Isso inclui (mas não está limitado a) casos de bytecode e arquivo gerador de analisador sintático. `make regen-stdlib-module-names` e `autoconf` devem ser executados separadamente para os restantes arquivos gerados.

3.4.4. Extensões C

Algumas extensões C são construídas como módulos embutidos, como o módulo `sys`. Eles são construídos com a macro `Py_BUILD_CORE_BUILTIN` definida. Módulos embutidos não possuem nenhum atributo `__file__`:

```
>>> import sys
>>> sys
<module 'sys' (built-in)>
>>> sys.__file__
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: module 'sys' has no attribute '__file__'
```

Outras extensões C são construídas como bibliotecas dinâmicas, como o módulo `_asyncio`. Eles são construídos com a macro `Py_BUILD_CORE_MODULE` definida. Exemplo no Linux x86-64:

```
>>> import _asyncio
>>> _asyncio
<module '_asyncio' from '/usr/lib64/python3.9/lib-dynload/_async
>>> _asyncio.__file__
'/usr/lib64/python3.9/lib-dynload/_asyncio.cpython-39-x86_64-lir
```

`Modules/Setup` é usado para gerar alvos Makefile para construir extensões C. No início dos arquivos, as extensões C são construídas como módulos embutidos. Extensões definidas após o marcador `*shared*` são construídas como bibliotecas dinâmicas.

As macros **`PyAPI_FUNC()`**, **`PyAPI_DATA()`** e **`PyMODINIT_FUNC`** de `Include/exports.h` são definidas de forma diferente dependendo se a

macro `Py_BUILD_CORE_MODULE` está definida:

- Usa `Py_EXPORTED_SYMBOL` se `Py_BUILD_CORE_MODULE` estiver definido
- Do contrário, usa `Py_IMPORTED_SYMBOL`.

Se a macro `Py_BUILD_CORE_BUILTIN` for usada por engano em uma extensão C construída como uma biblioteca compartilhada, sua função `PyInit_xxx()` não será exportada, causando um **`ImportError`** na importação.

3.5. Sinalizadores do compilador e do vinculador

Opções definidas pelo script `./configure` e variáveis de ambiente e usadas por `Makefile`.

3.5.1. Sinalizadores do pré-processador

CONFIGURE_CPPFLAGS

Valor da variável **`CPPFLAGS`** passado para o script `./configure`.

Adicionado na versão 3.6.

CPPFLAGS

Sinalizadores de pré-processador C++ / (Objective) C, p.ex. `-I`*`include_dir`* se você tiver cabeçalhos em um diretório não padrão *include_dir*.

Tanto **`CPPFLAGS`** quanto **`LDFLAGS`** precisam conter o valor do shell para poder construir módulos de extensão usando os diretórios especificados nas variáveis de ambiente.

BASECPPFLAGS

Adicionado na versão 3.4.

PY_CPPFLAGS

Sinalizadores extras de pré-processador adicionados para construir os arquivos de objeto do interpretador.

Padrão: `$(BASECPPFLAGS) -I. -I$(srcdir)/Include $(CONFIGURE_CPPFLAGS) $(CPPFLAGS)`.

Adicionado na versão 3.2.

3.5.2. Sinalizadores do compilador

CC

Comando do compilador C.

Exemplo: `gcc -pthread`.

CXX

Comando do compilador C++.

Exemplo: `g++ -pthread`.

CFLAGS

Sinalizadores do compilador C.

CFLAGS_NODIST

`CFLAGS_NODIST` é usado para construir o interpretador e extensões C da stdlib. Use-o quando um sinalizador do compilador *não* deve fazer parte de **`CFLAGS`** depois que o Python estiver instalado (gh-65320 [https://github.com/python/cpython/issues/65320]).

Em particular, **`CFLAGS`** não deve conter:

- o sinalizador do compilador `-I` (para definir o caminho de pesquisa para arquivos incluídos). Os sinalizadores `-I` são processadas da esquerda para a direita, e quaisquer sinalizadores em **`CFLAGS`** terão precedência sobre os sinalizadores `-I` fornecidos pelo usuário e pelo pacote.
- sinalizadores de segurança como `-Werror` porque as distribuições não podem controlar se os pacotes instalados pelos usuários estão em conformidade com esses padrões elevados.

Adicionado na versão 3.5.

COMPILEALL_OPTS

Opções passadas para a linha de comando `compileall` ao construir arquivos PYC em `make install`. Padrão: `-j0`.

Adicionado na versão 3.12.

EXTRA_CFLAGS

Sinalizadores extra do compilador C.

CONFIGURE_CFLAGS

Valor da variável `CFLAGS` passado para o script `./configure`.

Adicionado na versão 3.2.

CONFIGURE_CFLAGS_NODIST

Valor da variável `CFLAGS_NODIST` passado para o script `./configure`.

Adicionado na versão 3.5.

BASECFLAGS

Sinalizadores base do compilador.

OPT

Sinalizadores de otimização.

CFLAGS_ALIASING

Sinalizadores de alias estritos ou não estritos usados para compilar `Python/dtoa.c`.

Adicionado na versão 3.7.

CFLAGS_CEVAL

Sinalizadores usados para compilar `Python/ceval.c`.

Adicionado na versão 3.14.5.

CCSHARED

Sinalizadores de compilador usados para construir uma biblioteca compartilhada.

Por exemplo, `-fPIC` é usado no Linux e no BSD.

CFLAGSFORSHARED

Sinalizadores extras de C adicionados para construir os arquivos de objeto do interpretador.

Padrão: `$(CCSHARED)` quando `--enable-shared` é usado, ou uma string vazia caso contrário.

PY_CFLAGS

Padrão: `$(BASECFLAGS) $(OPT) $(CONFIGURE_CFLAGS) $(CFLAGS) $(EXTRA_CFLAGS)`.

PY_CFLAGS_NODIST

Padrão: `$(CONFIGURE_CFLAGS_NODIST) $(CFLAGS_NODIST) -I$(srcdir)/Include/internal`.

Adicionado na versão 3.5.

PY_STDMODULE_CFLAGS

Sinalizadores do C usados para construir os arquivos de objeto do interpretador.

Padrão: `$(PY_CFLAGS) $(PY_CFLAGS_NODIST) $(PY_CPPFLAGS) $(CFLAGSFORSHARED)`.

Adicionado na versão 3.7.

PY_CORE_CFLAGS

Padrão: `$(PY_STDMODULE_CFLAGS) -DPy_BUILD_CORE`.

Adicionado na versão 3.2.

PY_BUILTIN_MODULE_CFLAGS

Sinalizadores do compilador para construir um módulo de extensão de biblioteca padrão como um módulo embutido, como o módulo **posix**.

Padrão: `$(PY_STDMODULE_CFLAGS) -DPy_BUILD_CORE_BUILTIN`.

Adicionado na versão 3.8.

PURIFY

Comando de Purify. Purify é um programa depurador de memória.

Padrão: string vazia (não usada).

3.5.3. Sinalizadores do vinculador

LINKCC

Comando do vinculador usado para construir programas como `python` e `_testembed`.

Padrão: `$(PURIFY) $(CC)`.

CONFIGURE_LDFLAGS

Valor da variável **LDFLAGS** passado para o script `./configure`.

Evite atribuir **CFLAGS**, **LDFLAGS**, etc. para que os usuários possam usá-los na linha de comando para anexar a esses valores sem pisotear os valores predefinidos.

Adicionado na versão 3.2.

LDFLAGS_NODIST

LDFLAGS_NODIST é usado da mesma maneira que **CFLAGS_NODIST**. Use-o quando um sinalizador de vinculador *não* fizer parte de **LDFLAGS** depois que o Python estiver instalado (gh-65320 [https://github.com/python/cpython/issues/65320]).

Em particular, **LDFLAGS** não deve conter:

- o sinalizador do compilador `-L` (para definir o caminho de pesquisa para arquivos incluídos). Os sinalizadores `-L` são processadas da esquerda para a direita, e quaisquer sinalizadores em **`LDFLAGS`** terão precedência sobre os sinalizadores `-L` fornecidos pelo usuário e pelo pacote.

CONFIGURE_LDFLAGS_NODIST

Valor da variável **`LDFLAGS_NODIST`** passado para o script `./configure`.

Adicionado na versão 3.8.

LDFLAGS

Sinalizadores do vinculador, p.ex. `-L`*`lib_dir`*, se você tiver bibliotecas em um diretório não padrão *lib_dir*.

Tanto **`CPPFLAGS`** quanto **`LDFLAGS`** precisam conter o valor do shell para poder construir módulos de extensão usando os diretórios especificados nas variáveis de ambiente.

Considere utilizar `EXE_LDFLAGS` se os sinalizadores do ligador fornecidas forem específicas para executáveis, como, por exemplo, o sinalizador `-pie` do GCC.

LIBS

Sinalizadores do vinculador para passar bibliotecas para o vinculador ao vincular o executável Python.

Exemplo: `-lrt`.

LDSHARED

Comando para construir uma biblioteca compartilhada.

Padrão: `@LDSHARED@ $(PY_LDFLAGS)`.

BLDSHARED

Comando para construir a biblioteca compartilhada `libpython`.

Padrão: `@BLDSHARED@ $(PY_CORE_LDFLAGS)`.

PY_LDFLAGS

Padrão: `$(CONFIGURE_LDFLAGS) $(LDFLAGS)`.

PY_LDFLAGS_NODIST

Padrão: `$(CONFIGURE_LDFLAGS_NODIST) $(LDFLAGS_NODIST)`.

Adicionado na versão 3.8.

PY_CORE_LDFLAGS

Sinalizadores de vinculador usados para construir os arquivos de objeto do interpretador.

Adicionado na versão 3.8.

EXE_LDFLAGS

Sinalizadores do ligador (linker) usadas para construir alvos executáveis, como o interpretador. Se fornecida, a variável **PY_CORE_EXE_LDFLAGS** será usada em substituição a **PY_CORE_LDFLAGS**.

Adicionado na versão 3.15.

CONFIGURE_EXE_LDFLAGS

Valor da variável **EXE_LDFLAGS** passado ao script `./configure`.

Adicionado na versão 3.15.

PY_CORE_EXE_LDFLAGS

Sinalizadores do ligador (linker) para construir os alvos do interpretador e do executável

Padrão: `$(PY_CORE_LDFLAGS)`

Adicionado na versão 3.15.

Notas de rodapé

[7]

`git clean -fdx` é uma maneira ainda mais extrema de “limpar” seu checkout. Ele remove todos os arquivos não conhecidos pelo Git. Ao procurar bugs usando `git bisect`, isso é recomendado entre testes [https://github.com/python/cpython/issues/114505#issuecomment-1907021718] para garantir uma construção completamente limpa. **Use com cuidado**, pois isso excluirá todos os arquivos não registrados no Git, incluindo seu trabalho novo e não confirmado.

4. Utilizando Python no Windows

Este documento pretende dar uma visão geral do comportamento específico do Windows que você deve conhecer quando fores utilizar o Python no sistema operacional Microsoft Windows.

Ao contrário da maioria dos sistemas e serviços Unix, o Windows não inclui uma instalação do Python com suporte no sistema. Em vez disso, o Python pode ser obtido de diversos distribuidores, inclusive diretamente da equipe do CPython. Cada distribuição do Python terá suas próprias vantagens e desvantagens; no entanto, a consistência com outras ferramentas que você usa geralmente é um benefício valioso. Antes de se comprometer com o processo descrito aqui, recomendamos verificar suas ferramentas existentes para ver se elas podem fornecer Python diretamente.

Para obter o Python da equipe do CPython, use o Python Install Manager (gerenciador de instalação do Python). Esta é uma ferramenta autônoma que disponibiliza o Python como comandos globais na sua máquina Windows, integra-se ao sistema e suporta atualizações ao longo do tempo. Você pode baixar o Python Install Manager. em python.org/downloads [https://www.python.org/downloads/] ou pelo aplicativo da Microsoft Store [https://apps.microsoft.com/detail/9NQ7512CXL7T].

Após instalar o Python Install Manager, o comando global `python` pode ser usado em qualquer terminal para iniciar a versão mais recente do Python. Essa versão pode mudar com o tempo, conforme você adiciona ou remove versões diferentes, e o comando `py list` mostrará qual é a atual.

Em geral, recomendamos que você crie um ambiente virtual para cada projeto e execute `<env>\Scripts\Activate` no seu terminal para usá-lo. Isso proporciona isolamento entre projetos, consistência ao longo do tempo e garante que comandos adicionais adicionados por pacotes também estejam disponíveis na sua sessão. Crie um ambiente virtual usando `python -m venv <caminho do ambiente>`.

Se os comandos `python` ou `py` não estiverem funcionando, consulte a seção Solução de problemas abaixo. Às vezes, são necessárias etapas manuais adicionais para configurar seu PC.

Além de usar o gerenciador de instalação do Python, o Python também pode ser obtido como pacotes NuGet. Veja Os pacotes nuget.org abaixo para mais informações sobre esses pacotes.

As distribuições incorporáveis são pacotes mínimos do Python adequados para incorporação em aplicações maiores. Elas podem ser instaladas usando o gerenciador de instalação do Python. Consulte O pacote embutível abaixo para obter mais informações sobre esses pacotes.

4.1. Gerenciador de instalação do Python

4.1.1. Instalação

O gerenciador de instalação do Python pode ser instalado a partir da aplicação da Microsoft Store [https://apps.microsoft.com/detail/9NQ7512CXL7T] ou baixado e instalado de python.org/downloads [https://www.python.org/downloads/]. As duas versões são idênticas.

Para instalar pela Store, basta clicar em “Install”. Após a conclusão, abra um terminal e digite `python` para começar.

Para instalar o arquivo baixado de python.org, clique duas vezes e selecione “Install” ou execute `Add-AppxPackage <caminho para MSIX>` no Windows Powershell.

Após a instalação, os comandos `python`, `py` e `pymanager` devem estar disponíveis. Se você já possui instalações do Python ou modificou a variável **`PATH`**, pode ser necessário removê-las ou desfazer as modificações. Consulte Solução de problemas para obter mais ajuda sobre como corrigir comandos que não funcionam.

Ao instalar um ambiente de execução pela primeira vez, você provavelmente será solicitado a adicionar um diretório ao seu **`PATH`**. Isso é opcional se você preferir usar o comando `py`, mas é oferecido para aqueles que preferem que todos os apelidos (como `python3.14.exe`) estejam disponíveis. O diretório será `%LocalAppData%\Python\bin` por padrão, mas pode ser personalizado por um administrador. Clique em Iniciar e pesquise “Editar as variáveis de ambiente para sua conta” na página de configurações do sistema para adicionar o caminho.

Cada ambiente de execução do Python que você instalar terá seu próprio diretório para scripts. Estes também precisam ser adicionados a **`PATH`** se você quiser usá-los.

O gerenciador de instalação do Python será atualizado automaticamente para novas versões. Isso não afeta nenhuma instalação de ambientes de execução do Python. Desinstalar o gerenciador de instalação do Python não desinstala nenhum ambiente de execução do Python.

Se você não conseguir instalar um MSIX no seu contexto, por exemplo, se estiver usando um software de implantação automatizada que não oferece suporte a isso ou se estiver mirando o Windows Server 2019, consulte Instalação avançada abaixo para obter mais informações.

4.1.2. Uso básico

O comando recomendado para iniciar o Python é `python`, que iniciará a versão solicitada pelo script em execução, um ambiente virtual ativo ou a versão padrão instalada, que será a versão estável mais recente, a menos que configurado de outra forma. Se nenhuma versão for solicitada especificamente e nenhum ambiente de execução estiver instalado, a versão mais recente será instalada automaticamente.

Para todos os cenários que envolvem múltiplas versões de ambientes de execução do antigo inicializador `py.exe`. Por padrão, `py` corresponde ao comportamento de `python`, mas também permite opções de linha de comando para selecionar uma versão específica, bem como subcomandos para gerenciar instalações. Estes são detalhados abaixo.

Como o comando `py` pode já estar sendo executado pela versão anterior, há também um comando `pymanager` inequívoco. Instalações com script que pretendem usar o gerenciador de instalação Python devem considerar o uso de `pymanager`, devido à menor chance de conflito com instalações existentes. A única diferença entre os dois comandos é que, quando executados sem argumentos, `py` iniciará seu interpretador padrão, enquanto `pymanager` exibirá ajuda (`pymanager exec ...` oferece comportamento equivalente a `py ...`).

Cada um desses comandos também possui uma versão em janela que evita a criação de uma janela de console. São eles `pyw`, `pythonw` e `pywmanager`. Um comando `python3` também está incluído, imitando o comando `python`. Ele visa detectar usos acidentais do comando POSIX típico no Windows, mas não se destina a ser amplamente utilizado ou recomendado.

Para iniciar seu ambiente de execução padrão, execute `python` ou `py` com os argumentos que você deseja passar para o ambiente de execução (como arquivos de script ou o módulo a ser iniciado):

```
$> py
...
$> python meu-script.py
...
$> py -m this
...
```

O ambiente de execução padrão pode ser substituído pela variável de ambiente **`PYTHON_MANAGER_DEFAULT`** ou por um arquivo de configuração. Consulte Configuração para obter informações sobre as configurações.

Para iniciar um ambiente de execução específico, o comando `py` aceita a opção `-V:<TAG>`. Esta opção deve ser especificada antes de quaisquer outras. A tag é parte ou todo o identificador do ambiente; para aqueles da equipe do CPython, parece a versão, potencialmente com a plataforma. Para compatibilidade, o `V:` pode ser omitido nos casos em que a tag se refere a uma versão oficial e começa com `3`.

```
$> py -V:3.14 ...
$> py -V:3-arm64 ...
```

Ambientes de execução de outros distribuidores podem exigir a inclusão da *company* também. Ela deve ser separada da tag por uma barra (`/` ou `\`) e pode ser encurtada a qualquer prefixo de sua valor completo. Especificar a empresa é opcional quando se trata de `PythonCore`, e especificar a tag é opcional (mas não a barra) quando se deseja a versão mais recente de uma empresa específica.

```
$> py -V:Distribuidor\1.0 ...
$> py -V:distrib/ ...
```

Se nenhuma versão for especificada, mas um arquivo de script for passado, o script será inspecionado em busca de uma *linha de shebang*. Este é um formato especial para a primeira linha de um arquivo que permite a substituição do comando. Consulte Linhas shebang para obter mais informações. Quando não houver uma linha shebang ou o problema não puder ser resolvido, o script será iniciado com o ambiente de execução padrão.

Se você estiver executando em um ambiente virtual ativo, não tiver solicitado uma versão específica e não houver uma linha shebang, o ambiente de execução padrão será o ambiente virtual em questão. Nesse cenário, o comando `python` provavelmente já foi substituído e nenhuma dessas verificações ocorreu. No entanto, esse comportamento garante que o comando `py` possa ser usado de forma intercambiável.

Quando nenhum ambiente de execução estiver instalado, qualquer comando de inicialização tentará instalar a versão solicitada e iniciá-la. Entretanto, após alguma versão ser instalada, apenas os comandos `py exec ...` e `pymanager exec ...` serão instalados se a versão solicitada estiver ausente. Outras formas de comandos exibirão um erro e orientarão você a usar `py install` primeiro.

4.1.3. Comando help

O comando `py help` exibirá a lista completa de comandos suportados, juntamente com suas opções. Qualquer comando pode receber a opção `-?` para exibir sua ajuda, ou seu nome pode ser passado para `py help`.

```
$> py help
$> py help install
$> py install /?
```

Todos os comandos oferecem suporte a algumas opções comuns, que serão exibidas por `py help`. Essas opções devem ser especificadas após qualquer subcomando. Especificar `-v` ou `--verbose` aumentará a quantidade de saída exibida, e `-vv` aumentará ainda mais para fins de depuração. Passar `-q` ou `--quiet` reduzirá a saída, e `-qq` a reduzirá ainda mais.

A opção `--config=<PATH>` permite especificar um arquivo de configuração para substituir várias configurações simultaneamente. Consulte Configuração abaixo para obter mais informações sobre esses arquivos.

4.1.4. Listando ambientes de execução

```
$> py list [-f=|--format=<FMT>] [-1|--one] [--online|-s=|--source=<URL>] [<TAG>...]
```

A lista de ambientes de execução instalados pode ser visualizada usando `py list`. Um filtro pode ser adicionado na forma de uma ou mais tags (com ou sem especificador de empresa), e cada uma pode incluir um prefixo `<`, `<=`, `>=` ou `>` para restringir a um intervalo.

Vários formatos são suportados e podem ser passados como a opção `--format=<FMT>` ou `-f <FMT>`. Os formatos incluem `table` (uma visualização de tabela amigável), `csv` (tabela separada por vírgulas), `json` (um único blob JSON), `jsonl` (um blob JSON por resultado), `exe` (apenas o caminho do executável), `prefix` (apenas o caminho do prefixo).

A opção `--one` ou `-1` exibe apenas um único resultado. Se o ambiente de execução padrão estiver incluído, será ele. Caso contrário, o “melhor” resultado será exibido (“melhor” é definido de forma deliberadamente vaga, mas geralmente será a versão mais recente). O resultado exibido por `py list --one <TAG>` corresponderá ao ambiente de execução que seria iniciado por `py -V:<TAG>`.

A opção `--only-managed` exclui resultados que não foram instalados pelo gerenciador de instalação do Python. Isso é útil para determinar quais ambientes de execução podem ser atualizados ou desinstalados por meio do comando `py`.

A opção `--online` é uma abreviação de `--source=<URL>` com a fonte padrão. Passar qualquer uma dessas opções pesquisará no índice online os ambientes de execução que podem ser instalados. O resultado exibido por `py list --online --one <TAG>` corresponderá ao ambiente de execução que seria instalado por `py install <TAG>`.

```
$> py list --online 3.14
```

Para compatibilidade com o inicializador antigo, os comandos `--list`, `--list-paths`, `-0` e `-0p` (por exemplo, `py -0p`) foram mantidos. Eles não permitem opções adicionais e produzirão uma saída com o formato antigo.

4.1.5. Instalando ambientes de execução

```
$> py install [-s=|--source=<URL>] [-f|--force] [-u|--update] [--dry-run] [<TAG>...]
```

Novas versões de ambientes de execução podem ser adicionadas usando `py install`. Uma ou mais tags podem ser especificadas, e a tag especial `default` pode ser usada para selecionar o padrão. Intervalos não são suportados para instalação.

A opção `--source=<URL>` permite substituir o índice online usado para obter ambientes de execução. Isso pode ser usado com um índice offline, como mostrado em Instalações offline.

Passar `--force` ignorará quaisquer arquivos em cache e removerá qualquer instalação existente para substituí-la pela especificada.

Passar `--update` substituirá as instalações existentes se a nova versão for mais recente. Caso contrário, elas serão mantidas. Se nenhuma tag for fornecida com `--update`, todas as instalações gerenciadas pelo gerenciador de instalação do Python serão atualizadas se versões mais recentes estiverem disponíveis. As atualizações removerão quaisquer modificações feitas na instalação, incluindo pacotes instalados globalmente, mas os ambientes virtuais continuarão funcionando.

Passar `--dry-run` gerará saída e logs, mas não modificará nenhuma instalação.

Passar o argumento `--refresh` atualizará todos os registros dos ambientes de execução instalados. Isso recriará atalhos no menu Iniciar, chaves de registro e aliases globais (como `python3.14.exe` ou aqueles referentes a scripts instalados). Esses itens são atualizados automaticamente durante a instalação de qualquer runtime, mas podem precisar de atualização manual após a instalação de pacotes.

In addition to the above options, the `--target` option will extract the runtime to the specified directory instead of doing a normal install. This is useful for embedding runtimes into larger applications. Unlike a normal install, `py` will not be aware of the extracted runtime, and no Start menu or other shortcuts will be created. To launch the runtime, directly execute the main executable (typically `python.exe`) in the target directory.

```
$> py install ... [-t=|--target=<PATH>] <TAG>
```

The `py exec` command will install the requested runtime if it is not already present. This is controlled by the `automatic_install` configuration (**`PYTHON_MANAGER_AUTOMATIC_INSTALL`**), and is enabled by default. If no runtimes are available at all, all launch commands will do an automatic install if the configuration setting allows. This is to ensure a good experience for new users, but should not generally be relied on rather than using the `py exec` command or explicit install commands.

4.1.6. Instalações offline

Para executar instalações offline do Python, você precisará primeiro criar um índice offline em uma máquina que tenha acesso à rede.

```
$> py install --download=<PATH> ... <TAG>...
```

A opção `--download=<PATH>` baixará os pacotes para as tags listadas e criará um diretório contendo-os e um arquivo `index.json` adequado para instalação posterior. Todo esse diretório pode ser movido para a máquina offline e usado para instalar um ou mais dos ambientes de execução incluídos:

```
$> py install --source="<PATH>\index.json" <TAG>...
```

O gerenciador de instalação do Python pode ser instalado baixando seu instalador e movendo-o para outra máquina antes da instalação.

Como alternativa, os arquivos ZIP em um diretório de índice offline podem ser simplesmente transferidos para outra máquina e extraídos. Isso não registrará a instalação de forma alguma e, portanto, ela deve ser iniciada referenciando diretamente os executáveis no diretório extraído, mas às vezes é uma abordagem preferível em casos em que a instalação do gerenciador de instalação do Python não é possível ou conveniente.

Dessa forma, os ambientes de execução do Python podem ser instalados e gerenciados em uma máquina sem acesso à internet.

4.1.7. Desinstalando ambientes de execução

```
$> py uninstall [-y|--yes] <TAG>...
```

Ambientes de execução pode ser removidos com o uso do comando `py uninstall`. Uma ou mais tags devem ser especificadas. Não há suporte para intervalos aqui.

A opção `--yes` ignora o prompt de confirmação antes da desinstalação.

Em vez de passar as tags individualmente, a opção `--purge` pode ser especificada. Isso removerá todos os ambientes de execução gerenciados pelo gerenciador de instalação do Python, incluindo a limpeza do menu Iniciar, do registro e de quaisquer caches de download. Ambientes de execução que não foram instalados pelo gerenciador de instalação do Python não serão afetados, nem os arquivos de configuração criados manualmente.

```
$> py uninstall [-y|--yes] --purge
```

O gerenciador de instalação do Python pode ser desinstalado por meio da página de configurações "Aplicativos instalados" do Windows. Isso não remove nenhum ambiente de execução, e eles ainda poderão ser usados, embora o `python` global e o comando `py` sejam removidos. A reinstalação do gerenciador de instalação do Python permitirá que você gerencie esses ambientes de execução novamente. Para limpar completamente todos os ambientes de execução do Python, execute o `--purge` antes de desinstalar o gerenciador de instalação do Python.

4.1.8. Configuração

O gerenciador de instalação do Python é configurado com uma hierarquia de arquivos de configuração, variáveis de ambiente, opções de linha de comando e configurações de registro. Em geral, os arquivos de configuração têm a capacidade de configurar tudo, incluindo a localização de outros arquivos de configuração, enquanto as configurações de registro são exclusivas do administrador e substituem os arquivos de configuração. As opções de linha de comando substituem todas as outras configurações, mas nem todas as opções estão disponíveis.

Esta seção descreverá os padrões, mas esteja ciente de que instalações modificadas ou substituídas podem resolver as configurações de maneira diferente.

A global configuration file may be configured by an administrator, and would be read first. The user configuration file is stored at `%AppData%\Python\pymanager.json` (note that this location is under `Roaming`, not `Local`) and is read next, overwriting any settings from earlier files. An additional configuration file may be specified as the `PYTHON_MANAGER_CONFIG` environment variable or the `--config` command line option (but not both). These locations may be modified by administrative customization options listed later.

As configurações a seguir são aquelas que provavelmente serão modificadas em uso normal. As seções posteriores listam aquelas destinadas à personalização administrativa.

Opções de configuração padrão

Chave de configuração	Variável de ambiente	Descrição
`default_tag`	PYTHON_MANAGER_DEFAULT	A versão padrão preferida para iniciar ou instalar. Por padrão, esta é interpretada como a versão mais recente, não pré-lançada, da equipe do CPython.
`default_platform`	`PYTHON_MANAGER_DEFAULT_PLATFORM`	A plataforma padrão preferida para iniciar ou instalar. Isso é tratado como um sufixo para a tag especificada, de modo que `py -V:3.14` prefere uma instalação para `3.14-64` se existir (e `default_platform` for `-64`), mas usará `3.14` se não houver nenhuma instalação marcada.
`logs_dir`	`PYTHON_MANAGER_LOGS`	O local onde os arquivos de log são gravados. Por padrão, `%TEMP%`.

Chave de configuração	Variável de ambiente	Descrição
`automatic_install`	PYTHON_MANAGER_AUTOMATIC_INSTALL	True to allow automatic installs when using `py exec` to launch (or `py` when no runtimes are installed yet). Other commands will not automatically install, regardless of this setting. By default, true.
`include_unmanaged`	`PYTHON_MANAGER_INCLUDE_UNMANAGED`	Verdadeiro para permitir a listagem e a execução de ambientes que não foram instalados pelo gerenciador de instalação do Python, ou falso para excluí-los. Por padrão, verdadeiro.
`shebang_can_run_anything`	`PYTHON_MANAGER_SHEBANG_CAN_RUN_ANYTHING`	Verdadeiro para permitir que shebangs em arquivos `.py` iniciem aplicação que não sejam ambientes de execução do Python, ou falso para evitar isso. Por padrão, verdadeiro.

Chave de configuração	Variável de ambiente	Descrição
`shebang_templates`	(nenhuma)	Mapping from shebang line template to alternative command, such as `py -V:<tag>` or a substitute string. See Linhas shebang for more details.
`log_level`	`PYMANAGER_VERBOSE`, `PYMANAGER_DEBUG`	Define o nível padrão de saída (0-50). O padrão é 20. Valores mais baixos produzem mais saída. As variáveis de ambiente são booleanas e podem produzir saída adicional durante a inicialização, que é posteriormente suprimida por outra configuração.
`confirm`	`PYTHON_MANAGER_CONFIRM`	`True` para confirmar certas ações antes de executá-las (como desinstalar) ou `False` para ignorar a confirmação. Por padrão, `True`.
`install.source`	`PYTHON_MANAGER_SOURCE_URL`	Substitui o feed de índice para obter novas instalações.

Chave de configuração	Variável de ambiente	Descrição
`install.enable_entrypoints`	(nenhuma)	True to generate global commands for installed packages (such as `pip.exe`). These are defined by the packages themselves. If set to false, only the Python interpreter has global commands created. By default, true. You should run `py install --refresh` after changing this setting.
`list.format`	`PYTHON_MANAGER_LIST_FORMAT`	Especifica o formato padrão usado pelo comando `py list`. Por padrão, `table`.
`install_dir`	(nenhuma)	Especifica o diretório raiz onde os ambientes de execução serão instalados. Se você alterar essa configuração, os ambientes de execução instalados anteriormente não poderão ser usados, a menos que você os mova para o novo local.

Chave de configuração	Variável de ambiente	Descrição
`global_dir`	(nenhuma)	Specify the directory where global commands (such as `python3.14.exe` and `pip.exe`) are stored. This directory should be added to your **`PATH`** to make the commands available from your terminal.
`download_dir`	(nenhuma)	Especifica o diretório onde os arquivos baixados serão armazenados. Este diretório é um cache temporário e pode ser limpo periodicamente.

Nomes pontilhados devem ser aninhados dentro de objetos JSON, por exemplo, `list.format` seria especificado como `{"list": {"format": "table"}}`.

4.1.9. Linhas shebang

Se a primeira linha de um arquivo de script começar com `#!`, ela é conhecida como uma linha “shebang”. Linux e outros sistemas operacionais semelhantes ao Unix têm suporte nativo para essas linhas, que são comumente usadas nesses sistemas para indicar como um script deve ser executado. Os comandos `python` e `py` permitem que os mesmos recursos sejam usados com scripts Python no Windows.

Para permitir que as linhas shebang em scripts Python sejam portáveis entre Unix e Windows, vários comandos “virtuais” são suportados para especificar qual interpretador usar. Os comandos virtuais suportados são:

- `/usr/bin/env <ALIAS>`
- `/usr/bin/env -S <ALIAS>`
- `/usr/bin/<ALIAS>`
- `/usr/local/bin/<ALIAS>`
- `<ALIAS>`

Por exemplo, se a primeira linha do seu script começa com

```
#! /usr/bin/python
```

O Python padrão ou um ambiente virtual ativo será localizado e utilizado. Como muitos scripts Python escritos para funcionar no Unix já terão essa linha, você deve perceber que esses scripts podem ser usados pelo inicializador sem modificação. Se você está escrevendo um novo script no Windows que você espera que seja útil no Unix, você deve usar uma dessas linhas shebang começando com `/usr`.

Qualquer um dos comandos virtuais acima pode ter o `<ALIAS>` substituído por um apelido de um ambiente de execução instalado. Ou seja, qualquer comando gerado no diretório global de apelidos (que você pode ter adicionado à sua variável de ambiente **`PATH`**) pode ser usado em uma shebang, mesmo que não esteja no seu **`PATH`**. Isso permite o uso de shebangs como `/usr/bin/python3.12` para selecionar um determinado ambiente de execução.

Se nenhum ambiente de execução estiver instalado ou se a instalação automática estiver ativada, o ambiente de execução solicitado será instalado, se necessário. Consulte Configuração para obter informações sobre as definições de configuração.

A forma `/usr/bin/env` da linha shebang também pesquisará a variável de ambiente **`PATH`** em busca de comandos não reconhecidos. Isso corresponde ao comportamento do programa `env` do Unix, que realiza a mesma busca, mas prefere executar comandos Python conhecidos. Um aviso pode ser exibido ao pesquisar executáveis arbitrários, e essa busca pode ser desabilitada pela opção de configuração `shebang_can_run_anything`.

As linhas Shebang que não correspondem a nenhum dos padrões são tratadas como caminhos executáveis do *Windows* que são absolutos ou relativos ao diretório que contém o arquivo de script Isso é uma conveniência para scripts exclusivos do Windows, como aqueles gerados por um instalador, pois o comportamento não é compatível com shells de estilo Unix. Esses caminhos podem ser citados e podem incluir vários argumentos, após os quais o caminho para o script e quaisquer argumentos adicionais serão anexados.

Since version 26.3 of the Python install manager, custom shebang templates may be added to your configuration file. Add the `shebang_templates` object with one member for each template (the string to match) and the command to use when the template is matched. Most commands should be `py -V:<tag>` (or `pyw`) to launch one of your installed runtimes. The `py -3.<version>` form is also allowed, as is a plain `py` to launch the default. No other arguments are supported.

```
{
    "shebang_templates": {
        "/usr/bin/python": "py",
        "/usr/bin/my_custom_python": "py -V:MyCustomPython/3"
    }
}
```

If the substitute command is not `py` or `pyw`, it will be written back into the shebang and regular handling continues. If launching arbitrary executables is permitted, then providing a full path will allow you to redirect from Python to any executable. The template should match either the entire line (ignoring leading and trailing whitespace), or up to the first space in the shebang line.

Nota

O comportamento dos shebangs no gerenciador de instalação do Python é sutilmente diferente do antigo inicializador `py.exe`, e as opções de configuração antigas não se aplicam mais. Se você depende especificamente do comportamento ou configuração antigos, recomendamos instalar o inicializador

legado [https://www.python.org/ftp/python/3.14.0/win32/launcher.msi]. O comando `py` do inicializador legado vai substituir o do PyManager por padrão, e você precisará usar os comandos `pymanager` para instalar e desinstalar.

4.1.10. Instalação avançada

Para situações em que um MSIX não pode ser instalado, como em algumas plataformas de distribuição administrativa mais antigas, há um MSI disponível na página de downloads do python.org. Este MSI não possui interface de usuário e só pode executar instalações por máquina em seu local padrão em Arquivos de Programas. Ele tentará modificar a variável de ambiente do sistema **`PATH`** para incluir este local de instalação, mas certifique-se de validar isso em sua configuração.

Nota

O Windows Server 2019 é a única versão do Windows compatível com o CPython que não oferece suporte a MSIX. Para o Windows Server 2019, você deve usar o MSI.

Esteja ciente que o pacote MSI não inclui nenhum ambiente de execução e, portanto, não é adequado para instalações em ambientes off-line sem criar também um indíce para instalação off-line. Consulte Instalações offline e Configuração administrativa para obter informações sobre como lidar com esses cenários.

Ambientes de execução instalados pelo MSI são compartilhados com os instalados pelo MSIX e são todos apenas por usuário. O gerenciador de instalação do Python não permite a instalação de ambientes de execução por máquina. Para emular uma instalação por máquina, você pode usar o `py install --target=<shared location>` como administrador e adicionar suas próprias modificações em todo o sistema ao **`PATH`**, ao registro ou ao menu Iniciar.

Quando o MSIX é instalado, mas os comandos não estão disponíveis na variável de ambiente **`PATH`**, eles podem ser encontrados em `%LocalAppData%\Microsoft\WindowsApps\PythonSoftwareFoundation.PythonManager_3847v3x7pw1km` ou `%LocalAppData%\Microsoft\WindowsApps\PythonSoftwareFoundation.PythonManager_qbz5n2kfra8p0`, dependendo se foi instalado a partir do python.org ou da Windows Store. Não é recomendável tentar executar o executável diretamente dos Arquivos de Programas.

Para instalar programaticamente o gerenciador de instalação do Python, é mais fácil usar o WinGet, que está incluído em todas as versões suportadas do Windows:

```
$> winget install 9NQ7512CXL7T -e --accept-package-agreements --disable-interactivity

# Opcionalmente execute o verificador de configuraçao e aceite todas as alterações
$> py install --configure -y
```

Para baixar o gerenciador de instalação do Python e instalá-lo em outra máquina, o seguinte comando WinGet baixará os arquivos necessários da Loja para o seu diretório de Downloads (adicione `-d <local>` para personalizar o local de saída). Isso também gera um arquivo YAML que parece desnecessário, pois o MSIX baixado pode ser instalado iniciando ou usando os comandos abaixo.

```
$> winget download 9NQ7512CXL7T -e --skip-license --accept-package-agreements --accept-s
```

Para instalar ou desinstalar programaticamente um MSIX usando apenas o PowerShell, os cmdlets Add-AppxPackage [https://learn.microsoft.com/powershell/module/appx/add-appxpackage] e Remove-AppxPackage [https://learn.microsoft.com/powershell/module/appx/remove-appxpackage] do PowerShell são recomendados:

```
$> Add-AppxPackage C:\Downloads\python-manager-25.0.msix
...
$> Get-AppxPackage PythonSoftwareFoundation.PythonManager | Remove-AppxPackage
```

A versão mais recente pode ser baixada e instalada pelo Windows passando o arquivo AppInstaller para o comando Add-AppxPackage. A instalação é feita usando o MSIX em python.org e é recomendada apenas para casos em que a instalação pela Loja (interativamente ou usando o WinGet) não é possível.

```
$> Add-AppxPackage -AppInstallerFile https://www.python.org/ftp/python/pymanager/pymanage
```

Outras ferramentas e APIs também podem ser usadas para provisionar um pacote MSIX para todos os usuários em uma máquina, mas o Python não considera esse cenário compatível. Sugerimos consultar o cmdlet Add-AppxProvisionedPackage [https://learn.microsoft.com/powershell/module/dism/add-appxprovisionedpackage] do PowerShell, a classe nativa PackageManager [https://learn.microsoft.com/uwp/api/windows.management.deployment.packagemanager] do Windows ou a documentação e o suporte da sua ferramenta de implantação.

Independentemente do método de instalação, os usuários ainda precisarão instalar suas próprias cópias do Python, pois não há como acionar essas instalações sem ser um usuário logado. Ao usar o MSIX, a versão mais recente do Python estará disponível para todos os usuários instalarem sem acesso à rede.

Observe que o MSIX disponível para download na Loja e no site do Python são ligeiramente diferentes e não podem ser instalados ao mesmo tempo. Sempre que possível, sugerimos usar os comandos WinGet acima para baixar o pacote da Loja, a fim de reduzir o risco de instalação conflitante. Não há restrições de licenciamento para o gerenciador de instalação do Python que impeçam o uso do pacote da Loja dessa forma.

4.1.11. Configuração administrativa

Há diversas opções que podem ser úteis para administradores substituírem a configuração do gerenciador de instalação do Python. Elas podem ser usadas para fornecer cache local, desabilitar certos tipos de atalhos e substituir conteúdo agrupado. Todas as opções de configuração acima podem ser definidas, assim como as abaixo.

As opções de configuração podem ser substituídas no registro definindo valores em `HKEY_LOCAL_MACHINE\Software\Policies\Python\PyManager`, onde o nome do valor corresponde à chave de configuração e o tipo de valor é `REG_SZ`. Observe que essa chave pode ser personalizada, mas somente modificando o arquivo de configuração principal distribuído com o gerenciador de instalação do Python. Recomendamos, no entanto, que os valores do registro sejam usados apenas para definir `base_config` como um arquivo JSON contendo o conjunto completo de substituições. As substituições de chave do registro substituirão qualquer outra configuração configurada, enquanto `base_config` permite que os usuários modifiquem ainda mais as configurações necessárias.

Observe que a maioria das configurações com variáveis de ambiente oferece suporte a essas variáveis porque a configuração padrão especifica a variável. Se você as substituir, a variável de ambiente não funcionará mais, a menos que você a substitua por outra. Por exemplo, o valor padrão de `confirm` é literalmente `%PYTHON_MANAGER_CONFIRM%`, o que resolverá a variável no momento do carregamento. Se

você substituir o valor para `yes`, a variável de ambiente não será mais usada. Se você substituir o valor para `%CONFIRM%`, essa variável de ambiente será usada.

As definições de configuração que são caminhos são interpretadas como relativas ao diretório que contém o arquivo de configuração que as especificou.

Opções de configuração administrativa

Chave de configuração	Descrição
`base_config`	O arquivo de configuração de maior prioridade para leitura. Observe que somente o arquivo de configuração embutido e o registro podem modificar essa configuração.
`user_config`	O segundo arquivo de configuração a ser lido.
`additional_config`	O terceiro arquivo de configuração a ser lido.
`registry_override_key`	Local do registro para verificar se há substituições. Observe que somente o arquivo de configuração embutido pode modificar esta configuração.
`bundled_dir`	Diretório somente leitura contendo arquivos armazenados em cache localmente.
`install.fallback_source`	Caminho ou URL para um índice a ser consultado quando o índice principal não puder ser acessado.
`install.enable_shortcut_kinds`	Lista separada por vírgulas de tipos de atalhos permitidos (por exemplo, `"pep514,start"`). Atalhos habilitados ainda podem ser desabilitados por `disable_shortcut_kinds`.
`install.disable_shortcut_kinds`	Lista separada por vírgulas de tipos de atalhos a serem excluídos (por exemplo, `"pep514,start"`). Atalhos desabilitados não são reativados por `enable_shortcut_kinds`.
`install.hard_link_entrypoints`	True to use hard links for global shortcuts to save disk space. If false, each shortcut executable is copied instead. After changing this setting, you must run `py install --refresh --force` to update existing commands. By default, true. Disabling this may be necessary for troubleshooting or systems that have issues with file links.

Chave de configuração	Descrição
`pep514_root`	Local do registro para leitura e gravação de entradas da PEP 514. Por padrão, `HKEY_CURRENT_USER\Software\Python`.
`start_folder`	Pasta do menu Iniciar para gravar atalhos. Por padrão, `Python`. Este caminho é relativo à pasta Programas do usuário.
`virtual_env`	Caminho para o ambiente virtual ativo. Por padrão, é `%VIRTUAL_ENV%`, mas pode ser definido como vazio para desabilitar a detecção de venv.
`shebang_can_run_anything_silently`	Verdadeiro para suprimir avisos visíveis quando um shebang executa uma aplicação que não seja um ambiente de execução Python.
`source_settings`	A mapping from source URL to settings specific to that index. When multiple configuration files include this section, URL settings are added or overwritten, but individual settings are not merged. These settings are currently only for index signatures.

4.1.12. Instalando binários com threads livres

Adicionado na versão 3.13.

Distribuições pré-construídas da construção com threads livres estão disponíveis instalando tags com o sufixo `t`.

```
$> py install 3.14t
$> py install 3.14t-arm64
$> py install 3.14t-32
```

Isso vai instalar e registrar como de costume. Se você não tiver outro ambiente de execução instalado, `python` iniciará este. Caso contrário, você precisará usar `py -V:3.14t ...` ou, se tiver adicionado o diretório global de apelidos à sua variável de ambiente **`PATH`**, o comando `python3.14t.exe`.

4.1.13. Index signatures

Adicionado na versão 26.2.

Index files may be signed to detect tampering. A signature is a catalog file at the same URL as the index with `.cat` added to the filename. The catalog file should contain the hash of its matching index file, and

should be signed with a valid Authenticode signature. This allows standard tooling (on Windows) to generate a signature, and any certificate may be used as long as the client operating system already trusts its certification authority (root CA).

Index signatures are only downloaded and checked when the local configuration's `source_settings` section includes the index URL and `requires_signature` is true, or the index JSON contains `requires_signature` set to true. When the setting exists in local configuration, even when false, settings in the index are ignored.

As well as requiring a valid signature, the `required_root_subject` and `required_publisher_subject` settings can further restrict acceptable signatures based on the certificate Subject fields. Any attribute specified in the configuration must match the attribute in the certificate (additional attributes in the certificate are ignored). Typical attributes are `CN=` for the common name, `O=` for the organizational unit, and `C=` for the publisher's country.

Finally, the `required_publisher_eku` setting allows requiring that a specific Enhanced Key Usage (EKU) has been assigned to the publisher certificate. For example, the EKU `1.3.6.1.5.5.7.3.3` indicates that the certificate was intended for code signing (as opposed to server or client authentication). In combination with a specific root CA, this provides another mechanism to verify a legitimate signature.

This is an example `source_settings` section from a configuration file. In this case, the publisher of the feed is uniquely identified by the combination of the Microsoft Identity Verification root and the EKU assigned by that root. The signature for this case would be found at `https://www.python.org/ftp/python/index-windows.json.cat`.

```
{
  "source_settings": {
    "https://www.python.org/ftp/python/index-windows.json": {
      "requires_signature": true,
      "required_root_subject": "CN=Microsoft Identity Verification Root Certificate Autho
      "required_publisher_subject": "CN=Python Software Foundation",
      "required_publisher_eku": "1.3.6.1.4.1.311.97.608394634.79987812.305991749.5787773
    }
  }
}
```

The same settings could be specified in the `index.json` file instead. In this case, the root and EKU are omitted, meaning that the signature must be valid and have a specific common name in the publisher's certificate, but no other checks are used.

```
{
  "requires_signature": true,
  "required_publisher_subject": "CN=Python Software Foundation",
  "versions": [
    // ...
  ]
}
```

When settings from inside a feed are used, the user is notified and the settings are shown in the log file or verbose output. It is recommended to copy these settings into a local configuration file for feeds that will be used frequently, so that unauthorised modifications to the feed cannot disable verification.

It is not possible to override the location of the signature file in the feed or through a configuration file. Administrators can provide their own `source_settings` in a mandatory configuration file (see

Configuração administrativa).

If signature validation fails, you will be notified and prompted to continue. When interactive confirmation is not allowed (for example, because `--yes` was specified), it will always abort. To use a feed with invalid configuration in this scenario, you must provide a configuration file that disables signature checking for that feed.

```
"source_settings": {
  "https://www.example.com/feed-with-invalid-signature.json": {
    "requires_signature": false
  }
}
```

4.1.14. Proxy settings

Adicionado na versão 26.4.

By default, the Python install manager will use your system-wide proxy settings, including automatic authentication. For most users, this behaves the same as your browser and other applications.

To override the settings, use the `NO_PROXY`, `HTTP_PROXY` and `HTTPS_PROXY` environment variables. These should be set in the terminal session before using the Python install manager. Other applications may also use the variables, so use caution before setting them globally for your entire machine.

Setting `NO_PROXY` to any non-empty value will disable the use of any proxy. Use this to bypass your system's default proxy setting and attempt to access the index server directly.

Setting `HTTPS_PROXY` to a string like `example.com:8080` will connect to that proxy server for all HTTPS connections. For the default index, all connections will be using HTTPS, and so this is the usual setting to override.

The `HTTP_PROXY` variable may be needed if you are using a private index server that does not use encrypted connections, but does require the default proxy server to be overridden. It follows the same format as `HTTPS_PROXY`.

Credentials may be embedded in either setting, however, when `HTTPS_PROXY` has credentials embedded they will be used in preference to other credentials. Only one set of credentials will be used for both types of proxy.

4.1.15. Solução de problemas

If your Python install manager does not seem to be working correctly, please work through these tests and fixes to see if it helps. If not, please report an issue at our bug tracker [https://github.com/python/pymanager/issues], including any relevant log files (written to your `%TEMP%` directory by default).

Solução de problemas

Sintoma	Coisas pra tentar

Sintoma	Coisas pra tentar
`python` retorna um erro “comando não encontrado” ou abre o aplicativo Store quando eu o digito no meu terminal.	Você instalou o gerenciador de instalação do Python?
	Clique em Iniciar, abra “Gerenciar aliases de execução de aplicativo” e verifique se os apelidos (aliases) para “Python (default)” estão ativados. Se já estiverem, tente desativá-los e ativá-los novamente para atualizar o comando. Os comandos “Python (windowed)” e “Python install manager” também podem precisar ser atualizados.
	Verifique se os comandos `py` e `pymanager` funcionam.
	Ensure your **`PATH`** variable contains the entry for `%UserProfile%\AppData\Local\Microsoft\WindowsApps`. The operating system includes this entry once by default, after other user paths. If removed, shortcuts will not be found.
`py` me dá um erro “comando não encontrado” quando eu digito no meu terminal.	Você instalou o gerenciador de instalação do Python?
	Clique em Iniciar, abra “Gerenciar aliases de execução de aplicativo” e verifique se os apelidos (aliases) para “Python (default)” estão ativados. Se já estiverem, tente desativá-los e ativá-los novamente para atualizar o comando. Os comandos “Python (windowed)” e “Python install manager” também podem precisar ser atualizados.
	Ensure your **`PATH`** variable contains the entry for `%UserProfile%\AppData\Local\Microsoft\WindowsApps`. The operating system includes this entry once by default, after other user paths. If removed, shortcuts will not be found.
`py` me dá um erro “não é possível abrir o arquivo” quando digito comandos no meu terminal.	Isso geralmente significa que você tem o inicializador legado instalado e ele tem prioridade sobre o gerenciador de instalação do Python. Para remover, clique em Iniciar, abra “Aplicativos instalados”, procure por “Python launcher” e desinstale-o.

Sintoma	Coisas pra tentar
`python` não inicia o mesmo ambiente de execução que `py`	Clique em Iniciar, abra “Aplicativos instalados”, procure por qualquer ambiente de execução do Python existente e remova-o ou modifique e desative as opções de **`PATH`**.
	Clique em Iniciar, abra “Gerenciar aliases de execução de aplicativo” e verifique se o apelido `python.exe` está definido como “Python (default)”
`python` e `py` não iniciam o ambiente de execução que eu esperava	Verifique a variável de ambiente **`PYTHON_MANAGER_DEFAULT`** ou a configuração `default_tag`. O comando `py list` mostrará o padrão com base nessas configurações.
	As instalações gerenciadas pelo gerenciador de instalação do Python serão escolhidas antes das instalações não gerenciadas. Use `py install` para instalar o ambiente de execução que você espera ou configure sua tag padrão.
	Prerelease and experimental installs that are not managed by the Python install manager may be chosen ahead of stable releases. Configure your default tag or uninstall the prerelease runtime and reinstall it using `py install`.
`pythonw` ou `pyw` não iniciam o mesmo ambiente de execução que `python` ou `py`	Clique em Iniciar, abra “Gerenciar aliases de execução de aplicativo” e verifique se seus apelidos `pythonw.exe` e `pyw.exe` são consistentes com os outros.
`pip` me dá um erro “comando não encontrado” quando eu digito no meu terminal.	Você ativou um ambiente virtual? Execute o script `.venv\Scripts\activate` no seu terminal para ativá-lo.
	The package may be available but missing the generated executable. We recommend using the `python -m pip` command instead. Running `py install --refresh` and ensuring that the global shortcuts directory is on **`PATH`** (it will be shown in the command output if it is not) should make commands such as `pip` (and other installed packages) available.

Sintoma	Coisas pra tentar
I installed a package with `pip` but its command is not found.	Você ativou um ambiente virtual? Execute o script `.venv\Scripts\activate` no seu terminal para ativá-lo.
	New packages do not automatically have global shortcuts created by the Python install manager. Similarly, uninstalled packages do not have their shortcuts removed. Run `py install --refresh` to update the global shortcuts for newly installed packages.
Digitar `script-name.py` no terminal abre em uma nova janela.	Esta é uma limitação conhecida do sistema operacional. Você pode especificar `py` antes do nome do script, criar um arquivo em lote contendo `@py "%~dpn0.py" %*` com o mesmo nome do script, ou instalar o inicializador legado e selecioná-lo como associação para scripts.
Arrastar e soltar arquivos em um script não funciona.	Essa é uma limitação conhecida do sistema operacional. Ela é suportada pelo inicializador legado ou pelo gerenciador de instalação do Python quando instalado a partir do MSI.
I have installed the Python install manager multiple times.	It is possible to install from the Store or WinGet, from the MSIX on the Python website, and from the MSI, all at once. They are all compatible and will share configuration and runtimes.
	See the earlier Instalação avançada section for ways to uninstall the install manager other than the typical Installed Apps (Add and Remove Programs) settings page.
My old `py.ini` settings no longer work.	The new Python install manager no longer supports this configuration file or its settings, and so it will be ignored. See Configuração for information about configuration settings.

4.2. O pacote embutível

Adicionado na versão 3.5.

A distribuição embutida é um arquivo ZIP contendo um ambiente Python mínimo. Ela foi criada para atuar como parte de outra aplicação, ao invés de ser diretamente acessada por usuários finais.

Para instalar uma distribuição incorporada, recomendamos usar o comando `py install` com a opção `--target`:

```
$> py install 3.14-embed --target=<directory>
```

Quando extraída, a distribuição embutida é (quase) completamente isolada do sistema do usuário, incluindo variáveis de ambiente, configurações de registro de sistema, e pacotes instalados. A biblioteca padrão está inclusa como arquivos `.pyc` pré-compilados e otimizados em um ZIP, e `python3.dll`, `python313.dll`, `python.exe` e `pythonw.exe` estão todos disponíveis. Tcl/tk (incluindo todas as dependências, como a Idle), pip e a documentação do Python não estão inclusos.

Um arquivo `._pth` padrão está incluído, o qual restringe ainda mais os caminhos de pesquisa padrão (conforme descrito abaixo em Encontrando módulos). Este arquivo destina-se a ser modificado por desenvolvedores que o incorporam, conforme necessário.

Pacotes de terceiros devem ser instalados pelo instalador da aplicação juntamente com a distribuição embutida. Usar o pip para gerenciar as dependências como em uma instalação regular do Python não é suportado nessa distribuição, apesar de que com algum cuidado pode ser possível incluir e usar o pip para atualizações automáticas. Em geral, pacotes de terceiros devem ser tratados como parte da aplicação ("vendoring") para que o desenvolvedor consiga garantir compatibilidade com versões mais recentes antes de fornecer atualizações para os usuários.

Os dois casos de uso recomendados para essa distribuição são descritos abaixo.

4.2.1. Aplicação Python

Uma aplicação escrita em Python não requer necessariamente que os usuários estejam cientes deste fato. A distribuição embutida pode ser usada neste caso para incluir uma versão privada do pacote de instalação do Python. Dependendo de quão transparente deve ser (ou pelo contrário, quão profissional deve parecer), existem duas opções.

Usar um executável especializado como inicializador requer um pouco de código, mas fornece a experiência mais transparente para os usuários. Com um inicializador personalizado, não existem indicações óbvias que o programa está rodando em Python: ícones podem ser personalizados, informações da empresa e versão podem ser especificadas, e associações de arquivo se comportam de forma apropriada. Na maioria dos casos, um inicializador personalizado deve simplesmente ser capaz de chamar `Py_Main` com uma linha de comando predefinida (hard-coded).

A abordagem mais simples é fornecer um arquivo batch ou um atalho gerado que chama diretamente o `python.exe` ou `pythonw.exe` com os argumentos de linha de comando necessários. Neste caso, a aplicação irá aparecer como Python e não seu nome real, e os usuários podem ter problemas em distinguir ela de outros processos ou associações de arquivo em Python.

Com a abordagem anterior, pacotes devem ser instalados como diretórios juntamente do executável do Python para garantir que estarão disponíveis no caminho do ambiente. Com o instalador especializado, pacotes podem ser armazenados em outras localidades já que há uma oportunidade de especificar o caminho de busca antes de executar a aplicação.

4.2.2. Incorporando Python

Aplicações escritas em código nativo frequentemente precisam de alguma forma de linguagem de script, e a distribuição embutida do Python pode ser usada para esse propósito. Em geral, a maior parte da aplicação é em código nativo, e uma parte irá ou invocar `python.exe` ou usar diretamente `python3.dll`. Para ambos os casos, extrair a distribuição embutida em um subdiretório da instalação da aplicação é suficiente para providenciar um interpretador Python carregável.

Para uso da aplicação, pacotes podem ser instalados em qualquer local já que há uma oportunidade de especificar caminhos de busca antes de inicializar o interpretador. De outra forma, não existem diferenças fundamentais entre usar a distribuição embutida ou a instalação regular.

4.3. Os pacotes nuget.org

Adicionado na versão 3.5.2.

O pacote nuget.org é um ambiente Python de tamanho reduzido criado para uso em integração contínua e construção de sistemas que não precisam de uma instalação de Python por todo o sistema da máquina. Enquanto nuget é o "gerenciador de pacotes para .NET", ele também funciona perfeitamente bem para pacotes contendo ferramentas em tempo de construção.

Visite nuget.org [https://www.nuget.org/] para informações mais atualizadas sobre utilização do nuget. A seguir está um sumário que é suficiente para desenvolvedores Python.

The `nuget.exe` command line tool may be downloaded directly from `https://dist.nuget.org/win-x86-commandline/latest/nuget.exe`, for example, using curl or PowerShell. With the tool, the latest version of Python for 64-bit or 32-bit machines is installed using:

```
nuget.exe install python -ExcludeVersion -OutputDirectory .
nuget.exe install pythonx86 -ExcludeVersion -OutputDirectory .
```

Para selecionar uma versão específica, adicione `-Version 3.x.y`. O diretório de saída pode ser mudado de `.`, e o pacote será instalado em um subdiretório. Por padrão, o subdiretório é nomeado igual ao pacote, e sem a opção `-ExcludeVersion` esse nome irá incluir a versão específica instalada. Dentro do subdiretório está um diretório `tools` que contém a instalação do Python:

```
# Sem -ExcludeVersion
> .\python.3.5.2\tools\python.exe -V
Python 3.5.2

# Com -ExcludeVersion
> .\python\tools\python.exe -V
Python 3.5.2
```

Em geral, pacotes nuget não são atualizáveis, e novas versões devem ser instaladas lado-a-lado e referenciadas usando o caminho completo. Alternativamente, delete o diretório do pacote manualmente e instale novamente. Muitos sistemas CI irão fazer isso automaticamente se eles não preservam arquivos entre construções de projetos.

Juntamente com o diretório `tools` está o diretório `build\native`. Ele contém um arquivo de propriedades MSBuild `python.props` que pode ser usado em um projeto C++ para referenciar a instalação do Python. Incluir as configurações irá automaticamente usar os cabeçalhos e importar as bibliotecas na sua construção de projeto.

As páginas de informação dos pacotes em nuget.org são www.nuget.org/packages/python [https://www.nuget.org/packages/python] para a versão 64 bits, www.nuget.org/packages/pythonx86 [https://www.nuget.org/packages/pythonx86] para a versão 32 bits e www.nuget.org/packages/pythonarm64 [https://www.nuget.org/packages/pythonarm64] para a versão ARM64

4.3.1. Pacotes com threads livres

Adicionado na versão 3.13.

Pacotes contendo binários com threads livres são chamados python-freethreaded [https://www.nuget.org/packages/python-freethreaded] para a versão de 64 bits, pythonx86-freethreaded [https://www.nuget.org/packages/pythonx86-freethreaded] para a versão de 32 bits e pythonarm64-freethreaded [https://www.nuget.org/packages/pythonarm64-freethreaded] para a versão ARM64. Esses dois pacotes contêm os pontos de entrada `python3.13t.exe` e `python.exe`, os quais são executados com threads livres.

4.4. Pacotes Alternativos

À parte da distribuição padrão CPython, existem pacotes modificados incluindo funcionalidades adicionais. A seguir está uma lista de versões populares e seus recursos chave:

ActivePython [https://www.activestate.com/products/python/]
: Instalador com compatibilidade multi-plataforma, documentação, PyWin32

Anaconda [https://www.anaconda.com/download/]
: Módulos científicos populares (como o numpy, scipy e pandas) e o gerenciador de pacotes `conda`.

Enthought Deployment Manager [https://assets.enthought.com/downloads/edm/]
: "A próxima geração de ambiente Python e gerenciador de pacotes".

  Anteriormente, a Enthought fornecia Canopy, mas este chegou ao fim de vida em 2016 [https://support.enthought.com/hc/en-us/articles/360038600051-Canopy-GUI-end-of-life-transition-to-the-Enthought-Deployment-Manager-EDM-and-Visual-Studio-Code].

WinPython [https://winpython.github.io/]
: Distribuição específica do Windows com pacotes científicos pré-construídos e ferramentas para construir pacotes.

Note que esses pacotes podem não incluir as últimas versões do Python ou outras bibliotecas, e não são mantidos ou suportados pelo time do núcleo do Python.

4.5. Versões do Windows suportadas

Como especificado na **PEP 11** [https://peps.python.org/pep-0011/], uma versão Python suporta apenas uma plataforma Windows enquanto a Microsoft considera a plataforma sob suporte estendido. Isso significa que o Python 3.15 suporta Windows 10 ou superiores. Se você precisa de suporte ao Windows 7, por favor instale o Python 3.8. Se você precisa de suporte ao Windows 8.1, por favor instale o Python 3.12.

4.6. Removendo a limitação de MAX_PATH

O Windows historicamente tem limitado os comprimentos dos caminhos de arquivos em 260 caracteres. Isso significava que caminhos maiores que isso não seriam resolvidos e resultariam em erros.

Nas últimas versões do Windows, essa limitação pode ser expandida para mais de 32.000 caracteres. Seu administrador irá precisar ativar a política de grupo "Enable Win32 long paths", ou definir `LongPathsEnabled` para `1` na chave de registro `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem`.

Isso permite que a função `open()`, o módulo `os` e a maior parte das outras funcionalidades de caminho aceitem e retornem caminhos maiores que 260 caracteres quando usando strings.

Após alterar a opção acima e reiniciar, nenhuma configuração adicional é necessária.

4.7. Modo UTF-8

Adicionado na versão 3.7.

Alterado na versão 3.15: Python UTF-8 mode is now enabled by default (**PEP 686** [https://peps.python.org/pep-0686/]).

Windows still uses legacy encodings for the system encoding (the ANSI Code Page). When the Python UTF-8 Mode is disabled, Python uses the ANSI Code Page as the default encoding of text files, as returned by `locale.getencoding()`.

Isso pode causar problemas, porque o UTF-8 é amplamente usado na Internet e na maioria dos sistemas Unix, incluindo o WSL (Subsistema Windows para Linux).

The Python UTF-8 Mode, enabled by default, ignores the system encoding and uses UTF-8 as the default text encoding. When the UTF-8 mode is enabled, you can still use the system encoding (the ANSI Code Page) via the "mbcs" codec.

You can disable the Python UTF-8 Mode via the `-X utf8=0` command line option, or the `PYTHONUTF8=0` environment variable. See `PYTHONUTF8` for disabling UTF-8 mode, and Gerenciador de instalação do Python for how to modify environment variables.

Dica

Adding `PYTHONUTF8={0,1}` to the default environment variables will affect all Python 3.7+ applications on your system. If you have any Python 3.7+ applications which rely on the legacy system encoding, it is recommended to set the environment variable temporarily or use the `-X utf8` command line option.

Nota

Mesmo quando o modo UTF-8 está desativado, o Python usa o UTF-8 por padrão no Windows para:

- E/S do console, incluindo E/S padrão (consulte **PEP 528** [https://peps.python.org/pep-0528/] para detalhes).
- A codificação do sistema de arquivos (veja a **PEP 529** [https://peps.python.org/pep-0529/] para detalhes).

4.8. Encontrando módulos

Estas notas complementam a descrição em A inicialização do caminho de pesquisa de módulos sys.path com notas detalhadas do Windows.

Quando nenhum arquivo `._pth` é encontrado, assim é como `sys.path` é populado no Windows:

- Uma entrada em branco é adicionada ao início, que corresponde ao diretório atual.
- Se a variável de ambiente `PYTHONPATH` existe, como descrito em Variáveis de ambiente, suas entradas são adicionadas em seguida. Note que no Windows, caminhos nessa variável devem ser separados

por ponto e vírgula, para distinguir eles dos dois pontos usados nos identificadores de drivers (`C:\` etc.).

- "Caminhos da aplicação" adicionais podem ser adicionados ao registro como subchaves de `\SOFTWARE\Python\PythonCore{version}\PythonPath` sob ambas `HKEY_CURRENT_USER` e `HKEY_LOCAL_MACHINE`. Subchaves que possuem string de caminhos delimitados por ponto e vírgula como seu valor padrão farão com que cada caminho seja adicionado ao **`sys.path`**. (Note que todos os instaladores conhecidos usam apenas HKLM, portanto HKCU está tipicamente vazio.)
- Se a variável de ambiente **`PYTHONHOME`** está definida, ela é presumida como "Python Home". Caso contrário, o caminho do principal executável do Python é usado para localizar um "arquivo de referência" (ou `Lib\os.py` ou `pythonXY.zip`) para deduzir o "Python Home". Se um Python Home é encontrado, os subdiretórios relevantes adicionados ao **`sys.path`** (`Lib`, `plat-win`, etc) são baseados naquela pasta. Se não, o caminho núcleo do Python é construído a partir do PythonPath armazenado no registro.
- Se o Python Home não puder ser localizado, nenhum **`PYTHONPATH`** está especificado no ambiente, e nenhuma entrada de registro pôde ser encontrada, um caminho padrão com entradas relativas é usado (por exemplo, `.\Lib;.\plat-win`, etc).

Se um arquivo `pyvenv.cfg` for encontrado juntamente com o principal executável ou no diretório um nível acima do executável, as seguintes variações se aplicam:

- Se `home` é um caminho absoluto e **`PYTHONHOME`** não está definido, o caminho é usado ao invés do caminho ao principal executável quando deduzindo a localização do Home.

O resultado final de tudo isso é:

- Quando executando `python.exe`, ou qualquer outro .exe no diretório principal do Python (ou uma versão instalada, ou diretamente do diretório PCbuild), o caminho núcleo é deduzido, e os caminhos núcleo no registro são ignorados. Outros "caminhos da aplicação" no registro são sempre lidos.
- Quando Python é hospedado em outro .exe (diretório diferente, embutido via COM, etc), o "Python Home" não será deduzido, então o caminho núcleo do registro é usado. Outros "caminhos da aplicação" no registro sempre são lidos.
- Se o Python não puder encontrar seu Home e não existem valores no registro (.exe imutáveis, algumas configurações de instalação bem estranhas) você recebe um caminho com alguns caminhos padrão, porém relativos.

Para aqueles que querem empacotar o Python em suas aplicações ou distribuições, o seguinte conselho irá prevenir conflitos com outras instalações:

- Inclua um arquivo `._pth` juntamente do executável contendo os diretórios a serem incluídos. Isso irá ignorar caminhos listados no registro e variáveis de ambiente, e também ignorar **`sites`** a não ser que `import site` seja listado.
- Se você estiver carregando `python3.dll` ou `python37.dll` em seu próprio executável, defina explicitamente **`PyConfig.module_search_paths`** antes de **`Py_InitializeFromConfig()`**.
- Limpe e/ou sobrescreva **`PYTHONPATH`** e defina **`PYTHONHOME`** antes de iniciar o `python.exe` a partir da sua aplicação.
- Se você não puder usar as sugestões anteriores (por exemplo, você é uma distribuição que permite que as pessoas executem o arquivo `python.exe` diretamente), certifique-se de que o arquivo de referência (`Lib\os.py`) existe no seu diretório de instalação. (Note que ele não será detectado dentro de um arquivo ZIP, mas um arquivo ZIP corretamente nomeado será detectado ao invés disso.)

Isso irá garantir que seus arquivos em uma instalação global no sistema não terão precedência sobre uma cópia da biblioteca padrão empacotada com a sua aplicação. Caso contrário, seus usuários podem

experienciar problemas usando sua aplicação. Note que a primeira sugestão é a melhor, pois as outras podem ainda estar suscetíveis a caminhos não-padrão no registro e no site-packages do usuário.

Alterado na versão 3.6: Adiciona suporte a arquivos `._pth` e remove a opção `applocal` do `pyvenv.cfg`.

Alterado na versão 3.6: Adiciona `python`*`XX`*`.zip` como uma possível referência quando diretamente adjacente ao executável.

Descontinuado desde a versão 3.6: Módulos especificados no registro sob `Modules` (não `PythonPath`) podem ser importados por **`importlib.machinery.WindowsRegistryFinder`**. Este localizador está ativo no Windows no 3.6.0 e anteriores, mas pode precisar ser explicitamente adicionado ao **`sys.meta_path`** no futuro.

4.9. Módulos adicionais

Mesmo que o Python tenha como objetivo ser portável através de todas as plataformas, existem recursos que são únicos para o Windows. Alguns módulos, em ambas as bibliotecas padrão e externa, e trechos de código existem para usar esses recursos.

Os módulos padrão específicos para Windows estão documentados em Serviços Específicos do MS Windows.

4.9.1. PyWin32

O módulo PyWin32 [https://pypi.org/project/PyWin32/] de Mark Hammond é uma coleção de módulos para suporte avançado específico para Windows. Isso inclui utilitários para:

- Component Object Model [https://learn.microsoft.com/windows/win32/com/component-object-model--com--portal] (COM)
- Chamadas à API Win32
- Registro
- Log de Eventos
- Interfaces de usuário Microsoft Foundation Classes [https://learn.microsoft.com/cpp/mfc/mfc-desktop-applications] (MFC)

PythonWin [https://web.archive.org/web/20060524042422/https://www.python.org/windows/pythonwin/] é uma aplicação MFC de exemplo enviada com o PyWin32. É uma IDE embutível com um depurador embutido.

Ver também

Win32 How Do I…? **[https://timgolden.me.uk/python/win32_how_do_i.html]**
por Tim Golden

Python e COM **[https://www.boddie.org.uk/python/COM.html]**
by David and Paul Boddie

4.9.2. cx_Freeze

cx_Freeze [https://cx-freeze.readthedocs.io/en/latest/] encapsula scripts Python em programas executáveis do Windows (arquivos `*.exe`). Quando você tiver feito isso, você pode distribuir sua aplicação sem solicitar

que os seus usuários instalem o Python.

4.10. Compilando Python no Windows

Se você quer compilar o CPython por conta própria, a primeira coisa que você precisa ter é a fonte [https://www.python.org/downloads/source/]. Você pode fazer o download tanto da fonte da última versão quanto pegar um novo checkout [https://devguide.python.org/setup/#get-the-source-code].

A árvore de fontes contém uma solução de construção e arquivos de projeto para o Microsoft Visual Studio, que é o compilador usado para construir as versões do Python oficiais. Esses arquivos estão no diretório `PCbuild`.

Confira o `PCbuild/readme.txt` para informações gerais sobre o processo de construção.

Para módulos de extensão, consulte Construindo Extensões C e C++ no Windows.

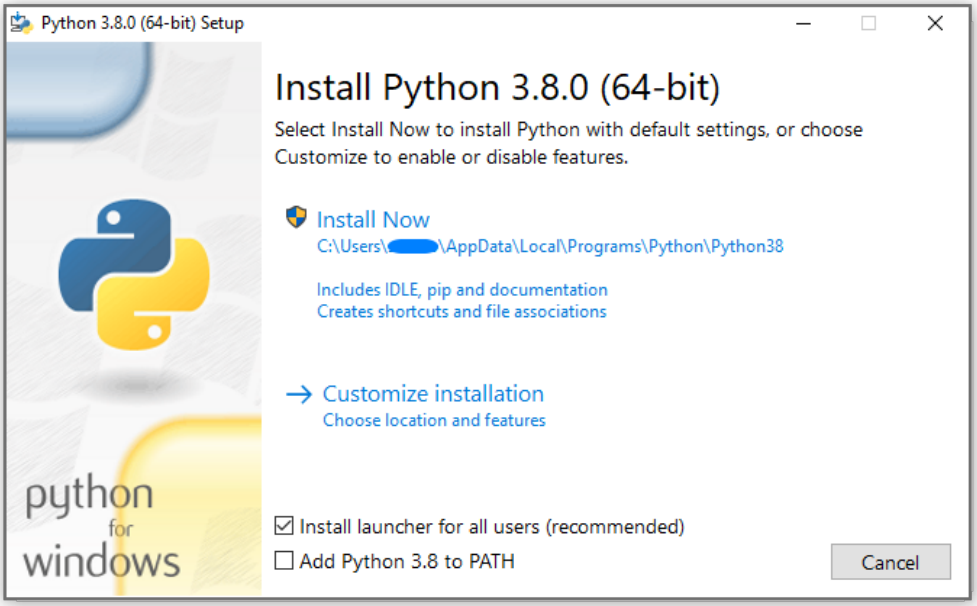
4.11. O instalador completo (descontinuado)

Descontinuado desde a versão 3.14: Este instalador está descontinuado desde a versão 3.14 e não será produzido para Python 3.16 ou posterior. Consulte Gerenciador de instalação do Python para obter o instalador moderno.

4.11.1. Etapas de instalação

Quatro instaladores Python 3.15 estão disponíveis para download - dois de cada para as versões 32-bit e 64-bit do interpretador. O *instalador web* é um download inicial menor, e ele irá automaticamente fazer o download dos componentes solicitados na medida do necessário. O *instalador offline* inclui os componentes necessários para uma instalação padrão e requer apenas uma conexão de internet para recursos opcionais. Veja Instalando sem download para outras formas de evitar o download durante a instalação.

Após iniciar o instalador, uma de duas opções deve ser selecionada:



Se você selecionar “Install Now”:

- Você *não* precisará ser um administrador (a menos que uma atualização de sistema para a Biblioteca em Tempo de Execução C seja necessária ou que você instale o Gerenciador de instalação do Python para todos os usuários)
- Python será instalado dentro do seu diretório de usuário
- O Gerenciador de instalação do Python será instalado de acordo com a opção ao final da primeira página
- A biblioteca padrão, a suíte de testes, o inicializador e o pip serão instalados
- Se selecionado, o diretório de instalação será adicionado no seu `PATH`
- Atalhos serão visíveis apenas para o usuário atual

Selecionando “Customize installation” irá permitir que você selecione os recursos a serem instalados, o local da instalação e outras opções ou ações pós-instalação. Para instalar símbolos de depuração ou binários, você precisará usar essa opção.

Para realizar uma instalação para todos os usuários, você deve selecionar “Customize installation”. Neste caso:

- Você pode ser solicitado a providenciar credenciais administrativas ou aprovação
- Python será instalado dentro do diretório Program Files (Arquivos de Programa)
- O Gerenciador de instalação do Python será instalado dentro do diretório Windows
- Recursos opcionais podem ser selecionados durante a instalação
- A biblioteca padrão pode ser pré-compilada em bytecode
- Se selecionado, o diretório de instalação será adicionado ao `PATH` do sistema
- Atalhos estão disponíveis para todos os usuários

4.11.2. Removendo a limitação de MAX_PATH

O Windows historicamente tem limitado os comprimentos dos caminhos de arquivos em 260 caracteres. Isso significava que caminhos maiores que isso não seriam resolvidos e resultariam em erros.

Nas últimas versões do Windows, essa limitação pode ser expandida para aproximadamente 32.000 caracteres. Seu administrador irá precisar ativar a política de grupo “Enable Win32 long paths”, ou definir `LongPathsEnabled` para `1` na chave de registro `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem`.

Isso permite que a função `open()`, o módulo `os` e a maior parte das outras funcionalidades de caminho aceitem e retornem caminhos maiores que 260 caracteres quando usando strings.

Após alterar a opção acima, nenhuma configuração adicional é necessária.

Alterado na versão 3.6: Suporte para caminhos longos foi ativado no Python.

4.11.3. Instalando sem UI

Todas as opções disponíveis na IU do instalador também podem ser especificadas a partir da linha de comando, permitindo que instaladores por script repliquem uma instalação em várias máquinas sem interação do usuário. Essas opções também podem ser definidas sem suprimir a IU para alterar alguns dos padrões.

As seguintes opções (encontradas executando o instalador com `/?`) podem ser passadas para o instalador:

Nome	Descrição
/passive	para exibir o progresso sem exigir interação do usuário
/quiet	para instalar/desinstalar sem exibir nenhuma UI
/simple	para evitar a personalização do usuário
/uninstall	para remover o Python (sem exigir confirmação)
/layout [diretório]	para pré-baixar todos os componentes
/log [nome-de-arquivo]	para especificar a localização dos arquivos de log

Todas as outras opções são passadas como `name=value`, onde o valor é usualmente `0` para desabilitar o recurso, `1` para ativar o recurso, ou um caminho. A lista completa de opções disponíveis é mostrada abaixo.

Nome	Descrição	Default (padrão)

Nome	Descrição	Default (padrão)
InstallAllUsers (Instalar para Todos usuários)	Execute uma instalação em todo o sistema.	0
TargetDir (Diretório Alvo)	O diretório de instalação	Selecionado com base em “Instalar para Todos os Usuários”
DefaultAllUsersTargetDir (Diretório de Destino Padrão para Todos os usuários)	O diretório de instalação padrão para instalações de todos os usuários	`%ProgramFiles%\Python X.Y` or `%ProgramFiles(x86)%\Python X.Y`
DefaultJustForMeTargetDir (Diretório Alvo Padrão Apenas Para Mim)	O diretório de instalação padrão para instalações just-for-me	`%LocalAppData%\Programs\Python\PythonXY` ou `%LocalAppData%\Programs\Python\PythonXY-32` ou `%LocalAppData%\Programs\Python\PythonXY-64`
DefaultCustomTargetDir (Diretório de destino personalizado padrão)	Diretório de instalação personalizado padrão exibido na interface do usuário	(vazio)
AssociateFiles (Arquivos Associados)	Criar associações de arquivos se o launcher também estiver instalado.	1
CompileAll (Compilar Tudo)	Compile todos os arquivos `.py` para `.pyc`.	0
PrependPath (path a ser percorrido)	Anexa os diretórios de instalação e Scripts ao início de **`PATH`** e adiciona `.PY` a **`PATHEXT`**	0
AppendPath	Anexa os diretórios de instalação e Scripts ao final de **`PATH`** e adiciona `.PY` a **`PATHEXT`**	0

Nome	Descrição	Default (padrão)
Shortcuts (atalhos)	Crie atalhos para o interpretador, documentação e IDLE se instalado.	1
Include_doc	Install Python manual (Instalação Manual do Python)	1
Include_debug (Incluir o Modo de Depuração)	Instalar binários de Depuração	0
Include_dev	Instala cabeçalhos e bibliotecas do desenvolvedor. Omitir isso pode levar a uma instalação inutilizável.	1
Include_exe	Instala `python.exe` e arquivos relacionados. Omitir isso pode levar a uma instalação inutilizável.	1
Include_launcher	Instalar Gerenciador de instalação do Python.	1
InstallLauncherAllUsers	Instala o iniciador para todos os usuários. Também requer `Include_launcher` para ser definido como 1	1
Include_lib	Instala a biblioteca padrão e os módulos de extensão. Omitir isso pode levar a uma instalação inutilizável.	1

Nome	Descrição	Default (padrão)
Include_pip	Instale o pacote pip e setuptools	1
Include_symbols	Instala símbolos de depuração (`*.pdb`)	0
Include_tcltk	Instale o suporte Tcl/Tk e o IDLE	1
Include_test	Instalar o conjunto de testes da biblioteca padrão	1
Include_tools	Instalar scripts com utilitários	1
LauncherOnly	Instala apenas o launcher. Isso substituirá a maioria das outras opções.	0
SimpleInstall	Desativar a maioria das UIs de instalação	0
SimpleInstallDescription	Uma mensagem personalizada para exibir quando a IU de instalação simplificada é usada.	(vazio)

Por exemplo, para instalar silenciosamente uma instalação de Python padrão e em todo o sistema, você pode usar o seguinte comando (a partir de um terminal de comando autorizado):

```
python-3.9.0.exe /quiet InstallAllUsers=1 PrependPath=1 Include_test=0
```

Para permitir que usuários instalem facilmente uma cópia do Python sem a suíte de testes, você pode fornecer um atalho com o seguinte comando. Isso irá exibir uma página inicial simplificado e bloquear a personalização:

```
python-3.9.0.exe InstallAllUsers=0 Include_launcher=0 Include_test=0
    SimpleInstall=1 SimpleInstallDescription="Just for me, no test suite."
```

(Observe que omitir o inicializador também omite associações de arquivos, e só é recomendado para instalações por usuários quando também existe uma instalação por todo o sistema que inclui o inicializador.)

As opções listadas acima também podem ser fornecidas em um arquivo chamado `unattend.xml` juntamente com o executável. Esse arquivo especifica uma lista de opções e valores. Quando um valor é fornecido como um atributo, ele será convertido para um número se possível. Valores fornecidos como elementos de texto são sempre deixados como strings. Esse arquivo de exemplo define as mesmas opções que o exemplo anterior:

```
<Options>
    <Option Name="InstallAllUsers" Value="no" />
    <Option Name="Include_launcher" Value="0" />
    <Option Name="Include_test" Value="no" />
    <Option Name="SimpleInstall" Value="yes" />
    <Option Name="SimpleInstallDescription">Just for me, no test suite</Option>
</Options>
```

4.11.4. Instalando sem download

Como alguns recursos do Python não estão inclusos no download inicial do instalador, selecionar esses recursos pode exigir uma conexão com a internet. Para evitar isso, todos os possíveis componentes podem ser baixados sob demanda para criar um *layout* completo que não irá precisar de uma conexão de internet independentemente dos recursos selecionados. Note que este download pode ser maior que o necessário, mas onde um grande número de instalações serão realizadas é bem útil ter uma cópia em cache local.

Execute o seguinte comando a partir do Prompt de Comando para fazer o download de todos os possíveis arquivos necessários. Lembre-se de substituir `python-3.9.0.exe` pelo nome real do seu instalador, e de criar layouts nos respectivos diretórios para evitar colisão entre arquivos com o mesmo nome.

```
python-3.9.0.exe /layout [diretório alvo opcional]
```

Você também pode especificar a opção `/quiet` para esconder o acompanhamento de progresso.

4.11.5. Modificando uma instalação

Uma vez que o Python foi instalado, você pode adicionar ou remover recursos através da ferramenta Programas e Recursos que é parte do Windows. Selecione o registro do Python e escolha “Uninstall/Change” para abrir o instalador no modo de manutenção.

“Modify” permite que você adicione ou remova recursos modificando as caixas de entrada - caixas de entrada não modificadas não irão instalar ou remover nada. Algumas opções não podem ser modificadas dessa forma, como o diretório de instalação; para modificá-las, você precisará remover e então reinstalar o Python completamente.

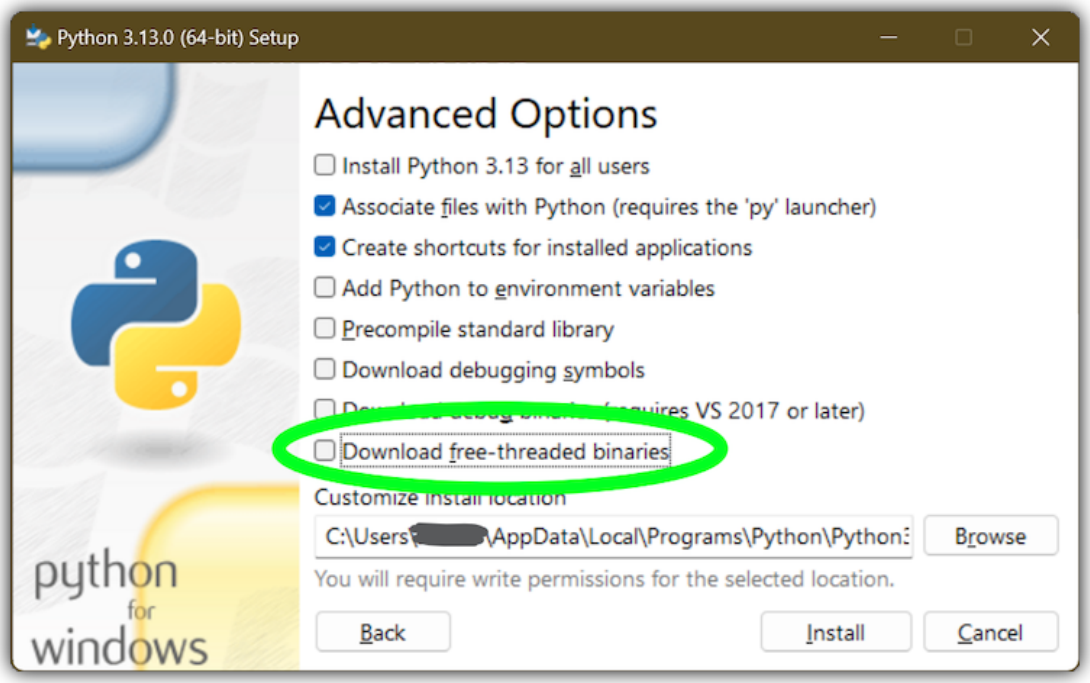
“Repair” irá verificar todos os arquivos que devem ser instalados usando as configurações atuais e substituir qualquer um que tiver sido removido ou modificado.

“Uninstall” irá remover o Python completamente, com a exceção do Gerenciador de instalação do Python, que tem seu próprio registro nos Programas e Recursos.

4.11.6. Instalando binários com threads livres

Adicionado na versão 3.13.

Para instalar binários pré-construídos com recurso de threads livres habilitado (veja **PEP 703** [https://peps.python.org/pep-0703/]), você deve selecionar “Customize installation” (Instalação personalizada). A segunda página de opções inclui a caixa de seleção “Download free-threaded binaries” (Baixar binários de threads livres).



Selecionar esta opção fará o download e instalará binários adicionais no mesmo local da instalação principal do Python. O executável principal é chamado `python3.13t.exe`, e outros binários recebem um sufixo `t` ou um sufixo ABI completo. Arquivos de origem do Python e dependências de terceiros empacotadas são compartilhadas com a instalação principal.

A versão com threads livres é registrada como uma instalação regular do Python com a tag `3.13t` (com um sufixo `-32` ou `-arm64` como normal para essas plataformas). Isso permite que as ferramentas o descubram e que o Gerenciador de instalação do Python ofereça suporte a `py.exe -3.13t`. Observe que o inicializador interpretará `py.exe -3` (ou um shebang `python3`) como “a instalação 3.x mais recente”, que preferirá os binários com threads livres aos normais, enquanto `py.exe -3.13` não. Se você usar o estilo curto da opção, talvez prefira não instalar os binários com threads livres neste momento.

Para especificar a opção de instalação na linha de comando, use `Include_freethreaded=1`. Veja Instalando sem download para instruções sobre como baixar preventivamente os binários adicionais para instalação offline. As opções para incluir símbolos de depuração e binários também se aplicam às construções com threads livres.

Binários com threads livres também estão disponíveis em nuget.org.

4.12. Inicializador Python para Windows (descontinuado)

Descontinuado desde a versão 3.14: O inicializador e esta documentação foram substituídos pelo gerenciador de instalação do Python descrito acima. Este documento é mantido temporariamente por razões históricas.

Adicionado na versão 3.3.

O inicializador Python para Windows é um utilitário que auxilia na localização e execução de diferentes versões do Python. Ele permite que scripts (ou a linha de comando) indiquem uma preferência por uma versão do Python específica, e irá localizar e executar essa versão.

Ao contrário da variável **PATH**, o inicializador irá corretamente selecionar a versão mais apropriada do Python. Ele irá preferir instalações por usuário ao invés de instalações globais no sistema, e ordenará por versão da linguagem ao invés de usar a versão instalada mais recentemente.

O inicializador foi originalmente especificado na **PEP 397** [https://peps.python.org/pep-0397/].

4.12.1. Começando

4.12.1.1. Pela linha de comando

Alterado na versão 3.6.

Instalações globais no sistema do Python 3.3 ou posterior irão colocar o inicializador no seu **PATH**. O inicializador é compatível com todas as versões disponíveis do Python, então não importa qual versão está instalada. Para verificar se o inicializador está disponível, execute o seguinte comando no Prompt de Comando:

```
py
```

Você deve perceber que a última versão do Python que você tem é iniciada - ela pode ser fechada normalmente, e qualquer argumento da linha de comando adicional especificado será enviado diretamente para o Python.

Se você tem múltiplas versões do Python instaladas (por exemplo, 3.7 e 3.15) você deve ter notado que o Python 3.15 foi iniciado – para iniciar o Python 3.7, use o comando:

```
py -3.7
```

Se você quer a versão mais recente do Python 2 que você tem instalada, tente o comando:

```
py -2
```

Se você ver o seguinte erro, você não tem o inicializador instalado:

```
'py' is not recognized as an internal or external command,
operable program or batch file.
```

O comando:

```
py --list
```

exibe a(s) versão(ões) atualmente instalada(s) do Python.

O argumento `-x.y` é a forma abreviada do argumento `-V:Empresa/Tag`, que permite selecionar um ambiente de execução Python específico, incluindo aqueles que podem ter vindo de algum lugar diferente de python.org. Qualquer ambiente de execução registrado seguindo a **PEP 514** [https://peps.python.org/pep-0514/] será detectável. O comando `--list` lista todos os ambientes de execução disponíveis usando o formato `-V:`.

Ao usar o argumento `-V:`, especificar a Empresa limitará a seleção aos ambientes de execução desse provedor, enquanto especificar apenas a Tag selecionará de todos os provedores. Observe que omitir a barra implica uma tag:

```
# Seleciona qualquer ambiente de execução sinalizado como '3.*'
py -V:3

# Seleciona qualquer ambiente de execução lançado como 'PythonCore'
py -V:PythonCore/

# Seleciona o ambiente de execução Python 3 mais recente de PythonCore
py -V:PythonCore/3
```

A forma abreviada do argumento (`-3`) só seleciona a partir de versões principais do Python, e não de outras distribuições. No entanto, a forma mais longa (`-V:3`) selecionará qualquer um.

A Empresa é correspondida na string completa, sem distinção entre maiúsculas e minúsculas. A Tag corresponde a uma string completa ou a um prefixo, desde que o próximo caractere seja um ponto ou um hífen. Isso permite que `-V:3.1` corresponda a `3.1-32`, mas não a `3.10`. As tags são classificadas usando ordenação numérica (`3.10` é mais recente que `3.1`), mas são comparadas usando texto (`-V:3.01` não corresponde a `3.1`).

4.12.1.2. Ambientes virtuais

Adicionado na versão 3.5.

Se o inicializador é executado sem versão explícita do Python especificada, e um ambiente virtual (criado com o módulo da biblioteca padrão **venv** ou da ferramenta externa `virtualenv`) está ativo, o inicializador irá executar o interpretador do ambiente virtual ao invés do global. Para executar o interpretador global, ou desative o ambiente virtual, ou explicitamente especifique a versão global do Python.

4.12.1.3. Por um script

Vamos criar um script teste de Python - crie um arquivo chamado `hello.py` com os seguintes conteúdos:

```
#! python
import sys
sys.stdout.write("olá do Python %s\n" % (sys.version,))
```

A partir do diretório onde hello.py está, execute o comando:

```
py hello.py
```

Você deve notar que o número da versão da sua instalação mais recente do Python 2.x é exibido. Agora tente mudar a primeira linha para ser:

```
#! python3
```

Re-executar o comando agora deve exibir informações da última versão do Python 3.x. Como nos exemplos da linha de comando acima, você pode especificar um qualificador de versão mais explícito. Presumindo que você tem o Python 3.7 instalado, tente mudar a primeira linha para `#! python3.7` e você deve ver as informações da versão 3.7 sendo exibidas.

Note que diferentemente do uso interativo, um simples "python" irá usar a última versão do Python 2.x que você tem instalada. Isso é para retrocompatibilidade e para compatibilidade com Unix, onde o comando `python` tipicamente se refere ao Python 2.

4.12.1.4. Por associação de arquivos

O inicializador deve ter sido associado com arquivos Python (isto é, arquivos `.py`, `.pyw`, `.pyc`) quando foi instalado. Isso significa que quando você clica duas vezes em um desses arquivos a partir do Explorador de Arquivos do Windows o inicializador será usado, e assim você pode usar as mesmas facilidades descritas acima para que o script especifique qual versão deve ser usada.

O benefício chave disso é que um único inicializador pode suportar múltiplas versões do Python ao mesmo tempo dependendo dos conteúdos da primeira linha.

4.12.2. Linhas shebang

Se a primeira linha de um arquivo de script começa com `#!`, ela é conhecida como linha "shebang". Linux e outros tipos de sistemas operacionais Unix têm suporte nativo para essas linhas e elas são comumente usadas nesses sistemas para indicar como um script deve ser executado. Esse inicializador permite que as mesmas facilidades sejam usadas com scripts Python no Windows e os exemplos acima demonstram seu uso.

Para permitir que linhas shebang em scripts Python sejam portáveis entre Unix e Windows, este inicializador suporta um número de comandos 'virtuais' para especificar qual interpretador deve ser usado. Os comandos virtuais suportados são:

- `/usr/bin/env`
- `/usr/bin/python`
- `/usr/local/bin/python`
- `python`

Por exemplo, se a primeira linha do seu script começa com

```
#! /usr/bin/python
```

O Python padrão ou um ambiente virtual ativo será localizado e utilizado. Como muitos scripts Python escritos para funcionar no Unix já terão essa linha, você deve perceber que esses scripts podem ser usados pelo inicializador sem modificação. Se você está escrevendo um novo script no Windows que você espera que seja útil no Unix, você deve usar uma dessas linhas shebang começando com `/usr`.

Qualquer um dos comandos virtuais acima pode ser sufixado com uma versão explícita (ou apenas a maior versão, ou a maior e a menor versão). Além disso, a versão de 32 bits pode ser solicitada adicionando "-32" após a menor versão. Isto é, `/usr/bin/python3.7-32` irá solicitar o uso do python 3.7 32-bit. Se um ambiente virtual estiver ativo, a versão será ignorada e o ambiente será usado.

Adicionado na versão 3.7: A partir do inicializador do python 3.7 é possível solicitar a versão 64-bit adicionando o sufixo "-64". Além disso é possível especificar uma versão maior e arquitetura sem a

menor (isto é, `/usr/bin/python3-64`).

Alterado na versão 3.11: O sufixo “-64” foi descontinuado e agora implica “qualquer arquitetura que não seja comprovadamente i386/32 bits”. Para solicitar um ambiente específico, use o novo argumento `-V:TAG` com a tag completa.

Alterado na versão 3.13: Comandos virtuais que referenciam `python` agora preferem um ambiente virtual ativo em vez de pesquisar **`PATH`**. Isso lida com casos em que o shebang especifica `/usr/bin/env python3`, mas `python3.exe` não está presente no ambiente ativo.

A forma `/usr/bin/env` da linha shebang tem mais uma propriedade especial. Antes de procurar por interpretadores Python instalados, este formulário pesquisará o executável **`PATH`** por um executável Python correspondente ao nome fornecido como o primeiro argumento. Isso corresponde ao comportamento do programa Unix `env`, que executa uma pesquisa **`PATH`**. Se um executável correspondente ao primeiro argumento após o comando `env` não puder ser encontrado, mas o argumento começar com `python`, ele será tratado conforme descrito para os outros comandos virtuais. A variável de ambiente **`PYLAUNCHER_NO_SEARCH_PATH`** pode ser definida (para qualquer valor) para ignorar esta pesquisa de **`PATH`**.

As linhas Shebang que não correspondem a nenhum desses padrões são procuradas na seção `[commands]` do arquivo .INI do lançador. Isso pode ser usado para lidar com certos comandos de uma maneira que faça sentido para o seu sistema. O nome do comando deve ser um único argumento (sem espaços no executável shebang) e o valor substituído é o caminho completo para o executável (argumentos adicionais especificados no .INI serão citados como parte do nome do arquivo).

```
[commands]
/bin/xpython=C:\Program Files\XPython\python.exe
```

Quaisquer comandos não encontrados no arquivo .INI são tratados como caminhos executáveis do **Windows** que são absolutos ou relativos ao diretório que contém o arquivo de script. Isso é uma conveniência para scripts somente do Windows, como aqueles gerados por um instalador, pois o comportamento não é compatível com shells de estilo Unix. Esses caminhos podem ser citados e podem incluir vários argumentos, após os quais o caminho para o script e quaisquer argumentos adicionais serão anexados.

4.12.3. Argumentos em linhas shebang

As linhas shebang também podem especificar opções adicionais a serem passadas ao interpretador Python. Por exemplo, se você tem uma linha shebang:

```
#! /usr/bin/python -v
```

O Python será iniciado com a opção `-v`

4.12.4. Personalização

4.12.4.1. Personalização via arquivos INI

Dois arquivos .ini serão pesquisados pelo inicializador - `py.ini` no diretório de dados de aplicações do usuário atual (`%LOCALAPPDATA%` ou `$env:LocalAppData`) e `py.ini` no mesmo diretório que o inicializador. Os mesmos arquivos .ini são usados para ambas a versão ‘console’ do inicializador (isto é, py.exe) e a versão ‘windows’ (isto é, pyw.exe).

Personalização especificada no “diretório da aplicação” terão precedência sobre àquela especificada junto do executável, portanto um usuário, que pode não ter acesso de escrita ao arquivo .ini junto do inicializador, pode sobrescrever comandos naquele arquivo .ini global.

4.12.4.2. Personalizando versões padrão do Python

Em alguns casos, um qualificador de versão pode ser incluído em um comando para ditar qual versão do Python deve ser usada pelo comando. Um qualificador de versão começa com um número maior de versão e pode opcionalmente ser seguido por um ponto (‘.’) e uma especificação de versão menor. Além disso, é possível especificar se uma implementação de 32 ou 64 bit deve ser solicitada adicionando “-32” ou “-64”.

Por exemplo, uma linha shebang `#!python` não tem qualificador de versão, enquanto `#!python3` tem um qualificador de versão que especifica apenas uma versão maior.

Se nenhum qualificador de versão é encontrado em um comando, a variável de ambiente **`PY_PYTHON`** pode ser definida para especificar o qualificador de versão padrão. Se ela não está definida, o padrão é “3”. A variável pode especificar qualquer valor que pode ser passado para a linha de comando, como “3”, “3.7”, “3.7-32” ou “3.7-64”. (Note que a opção “-64” está disponível apenas com o inicializador incluso no Python 3.7 ou mais recente.)

Se nenhum qualificador de versão menor é encontrado, a variável de ambiente `PY_PYTHON{major}` (onde `{major}` é o qualificador da versão maior atual, como determinado acima) pode ser definida para especificar a versão completa. Se nenhuma opção é encontrada, o inicializador irá enumerar as versões do Python instaladas e usar a última versão menor encontrada como versão maior, o que é provavelmente, ainda que não garantido, a versão instalada mais recentemente naquela família.

Em um Windows 64-bit com ambas as implementações de 32-bit e 64-bit da mesma (maior.menor) versão do Python instaladas, a versão 64-bit sempre será preferida. Isso será verdadeiro para ambas as implementações de 32-bit e 64-bit do inicializador - um inicializador 32-bit irá preferir executar uma instalação 64-bit do Python da versão especificada, se disponível. Isso é para que o comportamento do inicializador possa ser previsto sabendo apenas quais versões estão instaladas no PC e sem considerar a ordem com que elas foram instaladas (isto é, sem saber se a última a ser instalada foi a versão 32 ou 64-bit do Python e do instalador correspondente). Como observado acima, um sufixo opcional “-32” ou “-64” pode ser usado como especificador de versão para mudar esse comportamento.

Exemplos:

- Se nenhuma opção relevante for definida, os comandos `python` e `python2` irão usar a última versão instalada do Python 2.x e o comando `python3` irá usar o último Python 3.x instalado.
- O comando `python3.7` não vai consultar nenhuma opção já que as versões estão completamente especificadas.
- Se `PY_PYTHON=3`, os comandos `python` e `python3` irão ambos usar a última versão do Python 3 instalada.
- Se `PY_PYTHON=3.7-32`, o comando `python` irá usar a implementação 32 bits do 3.7 enquanto o comando `python3` irá usar o último Python instalado (PY_PYTHON não foi considerado porque uma versão maior foi especificada).
- Se `PY_PYTHON=3` e `PY_PYTHON3=3.7`, os comandos `python` e `python3` irão ambos usar especificamente 3.7

Em adição às variáveis de ambiente, as mesmas configurações podem ser feitas no arquivo .INI usado pelo inicializador. A seção nos arquivos INI é chamada `[defaults]` e o nome da chave será o mesmo que

as variáveis de ambiente sem o prefixo `PY_` (e observe que os nomes das chaves no arquivo INI não diferem maiúsculas e minúsculas). Os conteúdos de uma variável de ambiente irão sobrescrever coisas especificadas em um arquivo INI.

Por exemplo:

- Configurar `PY_PYTHON=3.7` é o equivalente ao arquivo INI contendo:

```
[defaults]
python=3.7
```

- Configurar `PY_PYTHON=3` e `PY_PYTHON3=3.7` é o equivalente ao arquivo INI contendo:

```
[defaults]
python=3
python3=3.7
```

4.12.5. Diagnósticos

Se uma variável de ambiente **`PYLAUNCHER_DEBUG`** é configurada (para qualquer valor), o inicializador irá exibir a informação de diagnóstico do stderr (isto é, no console). Enquanto essa informação consegue ser simultaneamente verbosa *e* concisa, ela deve permitir que você veja quais versões do Python foram localizadas, porquê uma versão particular foi escolhida e qual linha de comando exatamente foi usada para executar o Python alvo. Destina-se principalmente para testes e depuração.

4.12.6. Teste de simulação

Se uma variável de ambiente **`PYLAUNCHER_DRYRUN`** for definida (para qualquer valor), o inicializador emitirá o comando que ele teria executado, mas não iniciará o Python de fato. Isso pode ser útil para ferramentas que desejam usar o inicializador para detectar e então iniciar o Python diretamente. Observe que o comando escrito na saída padrão é sempre codificado usando UTF-8 e pode não ser renderizado corretamente no console.

4.12.7. Instalar sob demanda

Se uma variável de ambiente **`PYLAUNCHER_ALLOW_INSTALL`** for definida (para qualquer valor), e a versão do Python solicitada não estiver instalada, mas estiver disponível na Microsoft Store, o inicializador tentará instalá-la. Isso pode exigir interação do usuário para ser concluído, e você pode precisar executar o comando novamente.

Uma variável adicional **`PYLAUNCHER_ALWAYS_INSTALL`** faz com que o inicializador sempre tente instalar o Python, mesmo se for detectado. Isso é destinado principalmente para teste (e deve ser usado com **`PYLAUNCHER_DRYRUN`**).

4.12.8. Códigos de retorno

Os seguintes códigos de saída podem ser retornados pelo inicializador do Python. Infelizmente, não há como distingui-los do código de saída do próprio Python.

Os nomes dos códigos são como usados nas fontes e são apenas para referência. Não há como acessá-los ou resolvê-los além de ler esta página. As entradas são listadas em ordem alfabética de nomes.

Nome	Valor	Descrição
RC_BAD_VENV_CFG	107	Um `pyvenv.cfg` foi encontrado, mas está corrompido.
RC_CREATE_PROCESS	101	Falha ao iniciar o Python.
RC_INSTALLING	111	Uma instalação foi iniciada, mas o comando precisará ser executado novamente após sua conclusão.
RC_INTERNAL_ERROR	109	Erro inesperado. Por favor, relate um bug.
RC_NO_COMMANDLINE	108	Não é possível obter a linha de comando do sistema operacional.
RC_NO_PYTHON	103	Não foi possível localizar a versão solicitada.
RC_NO_VENV_CFG	106	Um `pyvenv.cfg` era necessário, mas não foi encontrado.

5. Usando Python em macOS

Este documento tem o objetivo de fornecer uma visão geral dos comportamentos específicos do macOS que você deve conhecer para começar a usar o Python em computadores Mac. O Python em um Mac com macOS é muito semelhante ao Python em outras plataformas derivadas do Unix, mas há algumas diferenças na instalação e alguns recursos.

Há várias maneiras de obter e instalar o Python para macOS. Versões pré-construídas das versões mais recentes do Python estão disponíveis em vários distribuidores. Grande parte deste documento descreve o uso dos Pythons fornecidos pela equipe de lançamento do CPython, disponíveis para download no site python.org [https://www.python.org/downloads/]. Consulte Distribuições alternativas para conhecer outras opções.

5.1. Usando Python para macOS do `python.org`

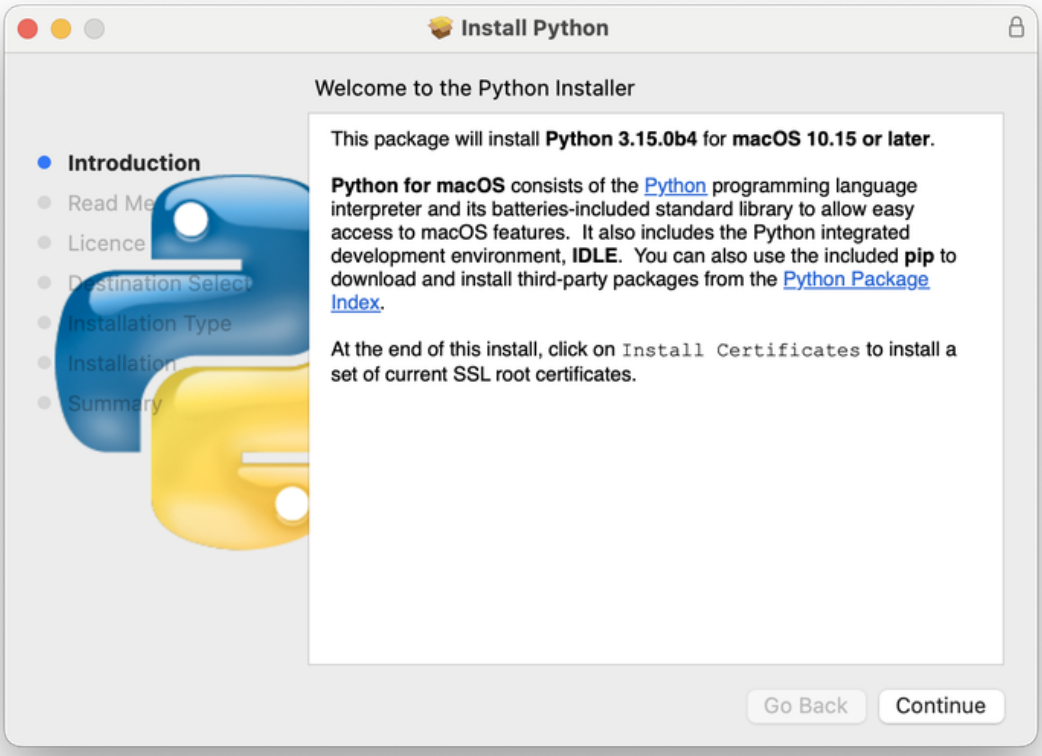
5.1.1. Etapas de instalação

Para as versões atuais de Python [https://www.python.org/downloads/] (exceto as com status `security`), a equipe de lançamento produz um pacote de instalação **Python para macOS** em cada novo lançamento. Uma lista dos instaladores disponíveis é fornecida na página de downloads de macOS [https://www.python.org/downloads/macos/]. Recomendamos usar a versão suportada mais recente de Python sempre que possível. Os instaladores atuais fornecem uma versão de Python como binário universal 2 [https://pt.wikipedia.org/wiki/Binário_universal] que é executado nativamente em todos os Macs (Apple Silicon e Intel) compatíveis com uma ampla gama de versões do macOS, atualmente do **macOS 10.15 Catalina** em diante.

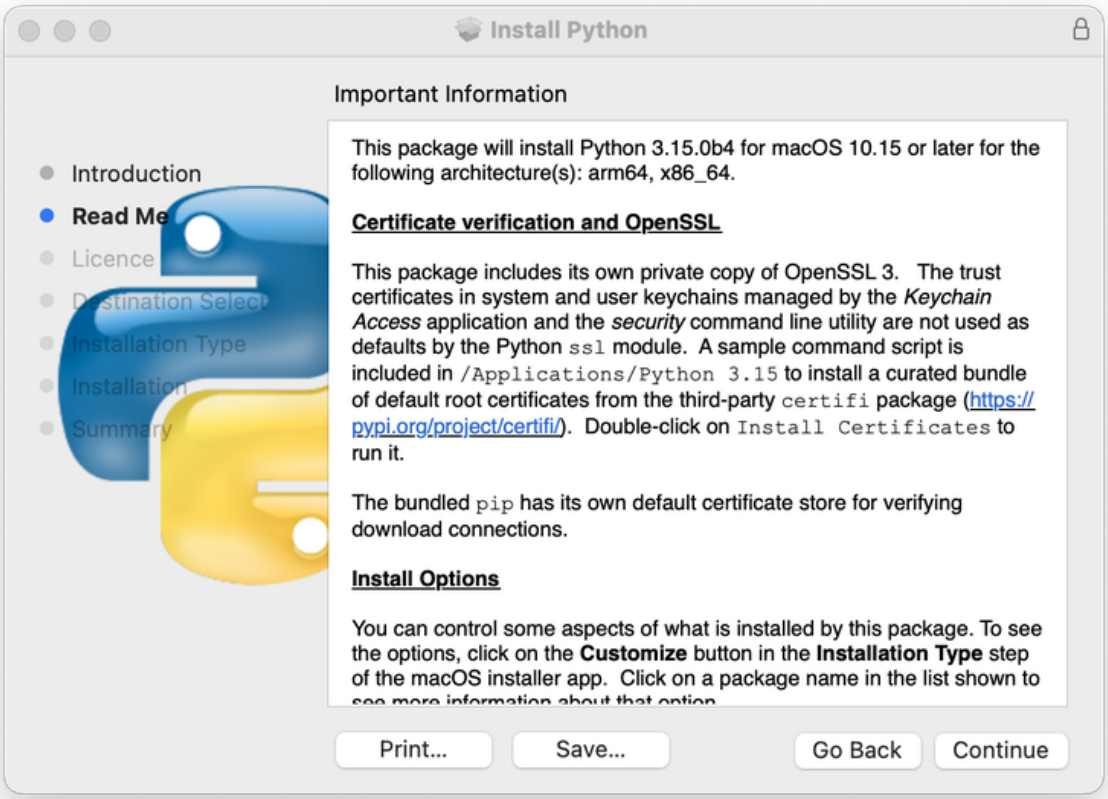
O arquivo baixado é um arquivo padrão de pacote de instalação do macOS (`.pkg`). As informações de integridade do arquivo (tal como a soma de verificação, tamanho e assinatura de Sigstore) de cada arquivo estão inclusas na página de download dos lançamentos. Os pacotes de instalação

e seus conteúdos são assinados e autenticados com certificados Apple Developer ID `Python Software Foundation` para atender aos [requisitos do macOS Gatekeeper](https://support.apple.com/en-us/102445) [https://support.apple.com/en-us/102445].

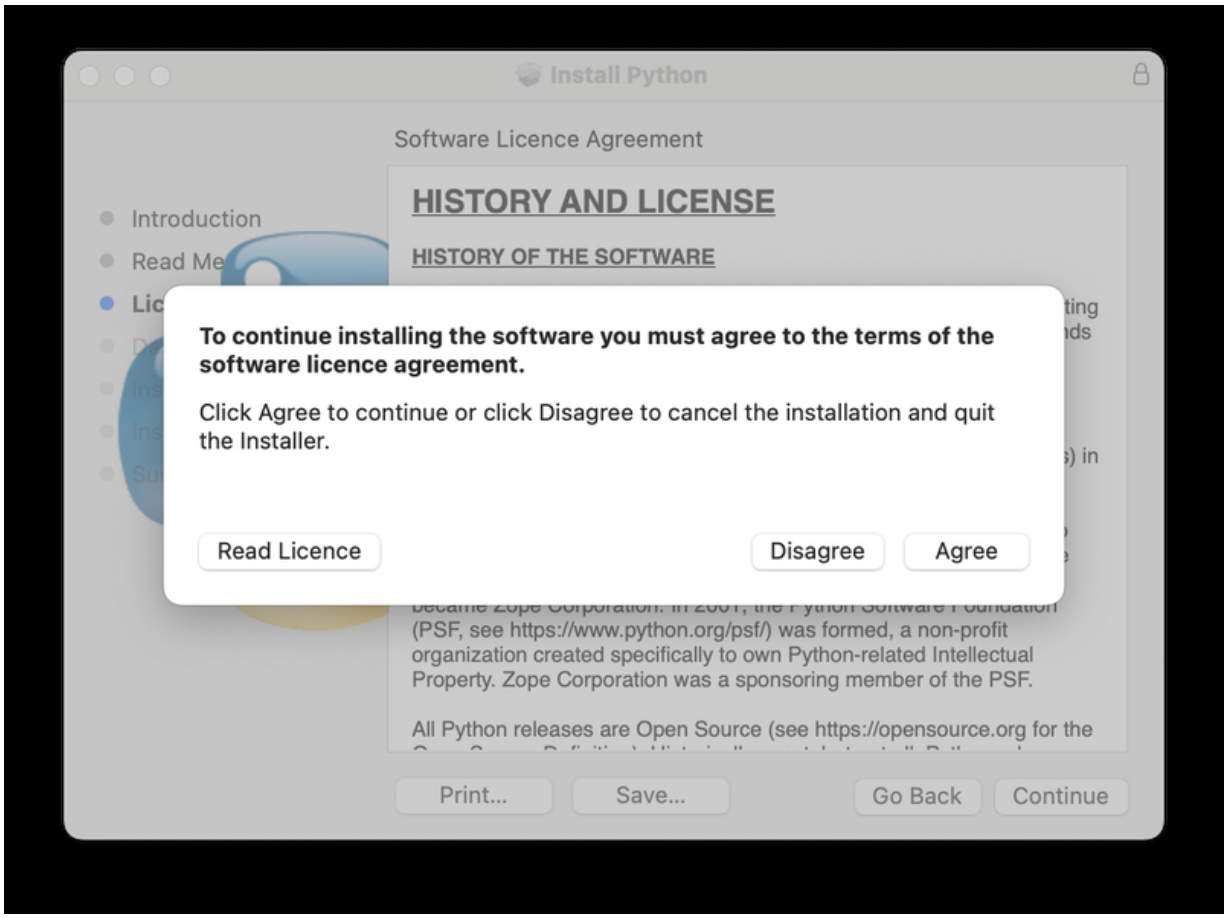
Para uma instalação padrão, clique duas vezes no arquivo baixado de pacote de instalação. Isso deverá abrir o aplicativo padrão de instalação do macOS e exibir o primeiro de vários passos de instalação.



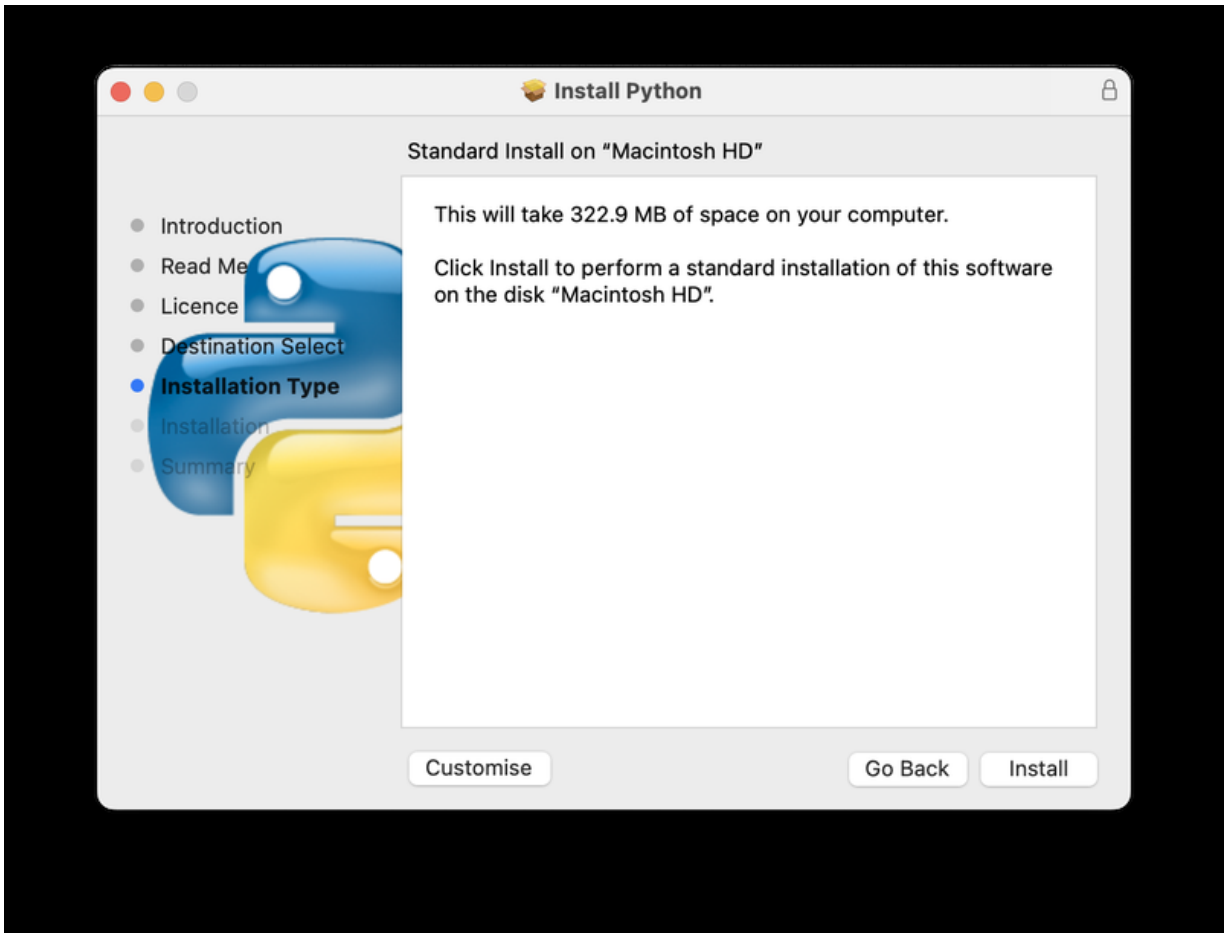
Ao clicar no botão **Continuar**, é exibido o **Read Me** desse instalador. Além de outras informações importantes, o **Read Me** documenta qual versão de Python será instalada e em quais versões do macOS ele é compatível. Talvez você precise rolar a tela para ler o arquivo inteiro. Por padrão, esse **Read Me** também será instalado em `/Applications/Python 3.15/` e estará disponível para leitura a qualquer momento.



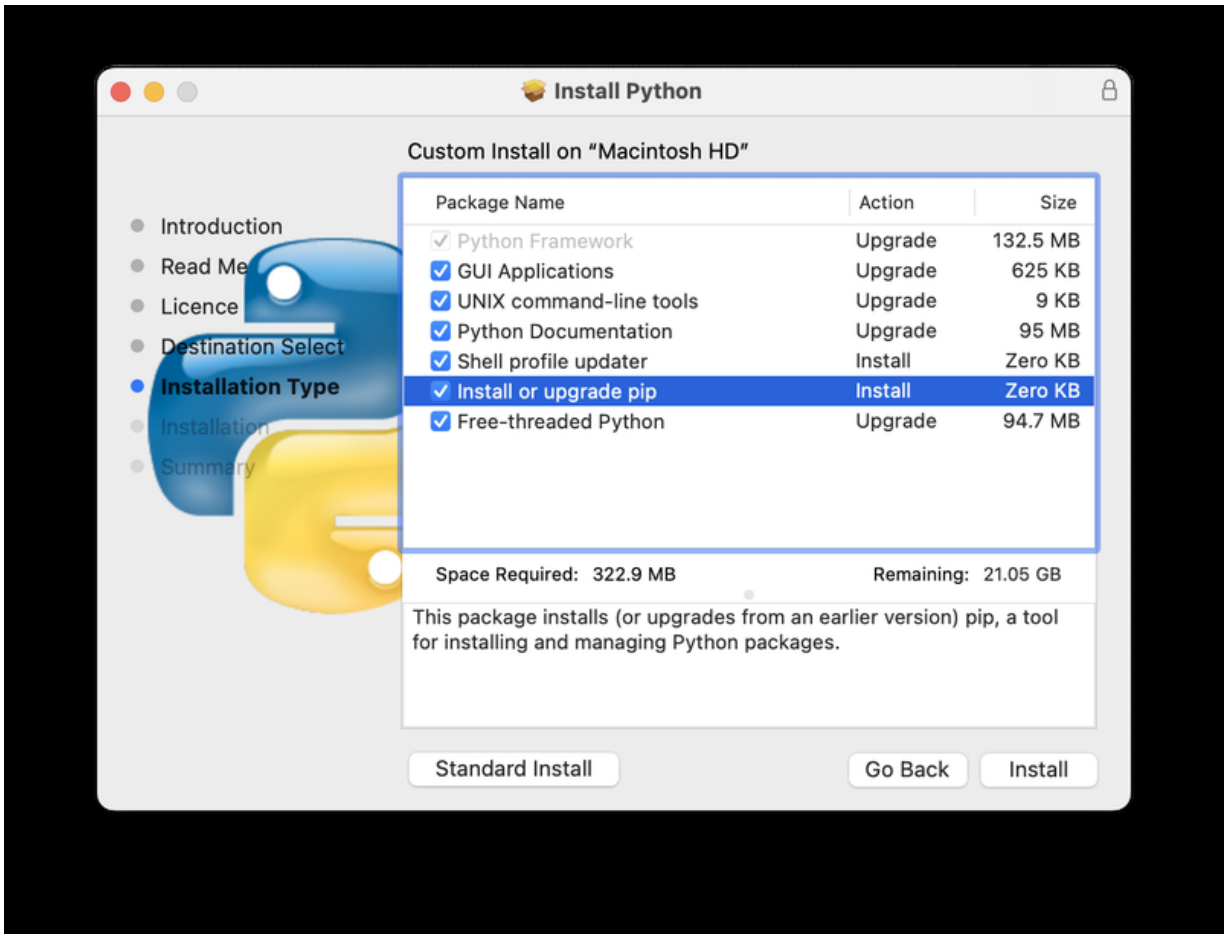
Ao clicar em **Continuar**, o instalador exibe a licença do Python e de outros softwares incluídos. Você precisará **Concordar** com os termos da licença antes de prosseguir para a próxima etapa. Esse arquivo de licença também será instalado e estará disponível para leitura a qualquer momento.



Após aceitar os termos da licença, a próxima etapa será a tela **Tipo de instalação**. Para a maioria dos usos, o conjunto padrão de operações de instalação é adequado.

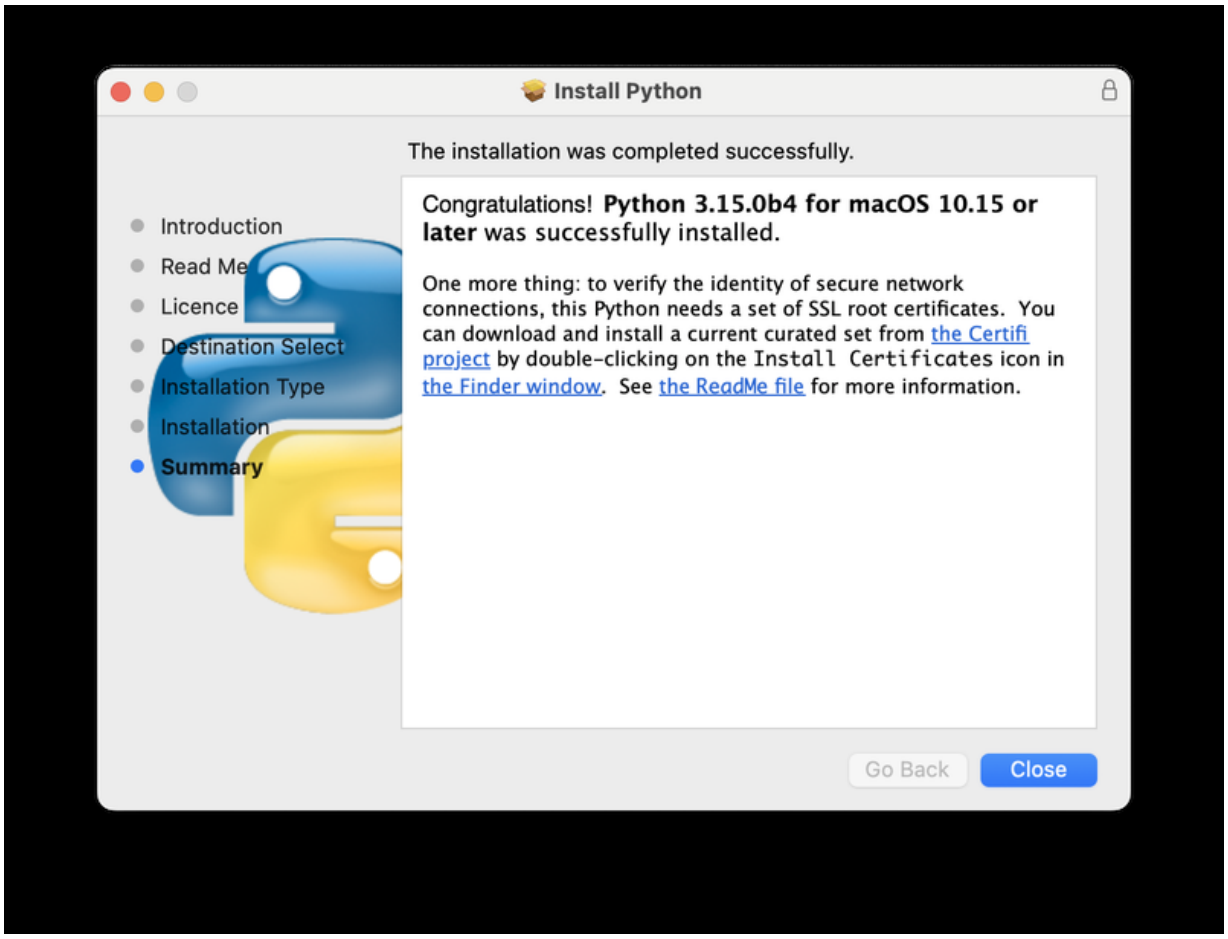


Ao pressionar o botão **Customize**, você pode optar por omitir ou selecionar determinados componentes de pacote do instalador. Clique em cada nome de pacote para ver uma descrição do que ele instala. Para detalhes sobre o recurso opcional de threads livres, o qual é instalado por padrão, consulte Instalando binários com threads livres.

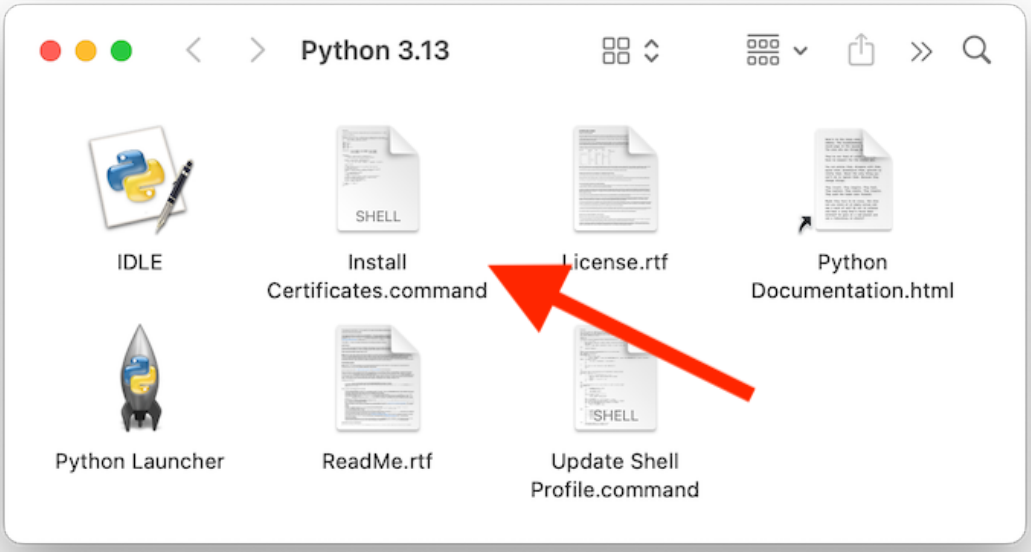


Em ambos os casos, clicar em **Instalar** iniciará o processo de instalação solicitando a permissão para instalar um novo software. É necessário um nome de usuário do macOS com privilégio de `Administrador`, pois o Python instalado estará disponível para todos os usuários do Mac.

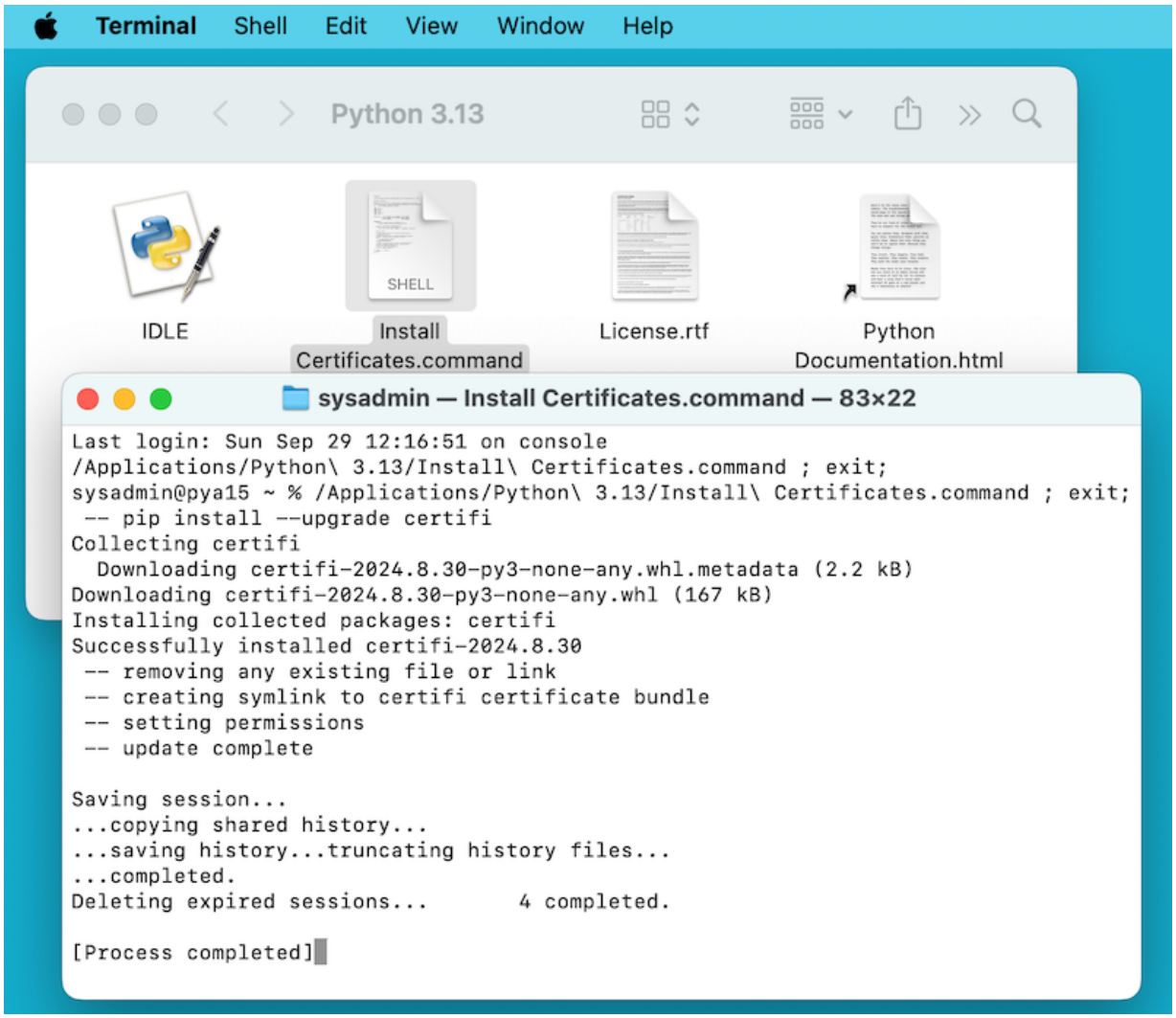
Quando a instalação for concluída, a janela **Resumo** será exibida.



Clique duas vezes no ícone ou arquivo **Install Certificates.command** na janela `/Applications/Python 3.15/` para concluir a instalação.



Isso abrirá uma janela temporária do console **Terminal** que usará o novo Python para baixar e instalar certificados raiz SSL para seu uso.



Se `Successfully installed certifi` e `update complete` aparecerem na janela do terminal, a instalação estará concluída. Feche essa janela do terminal e a janela do instalador.

Uma instalação do padrão incluirá:

- Uma pasta `Python 3.15` em sua pasta `Applications`. Aqui você encontra o **IDLE**, o ambiente de desenvolvimento que é parte das distribuições oficiais do Python; e o **Python Launcher**, que executa scripts Python quando são clicados duas vezes dentro do Finder [https://support.apple.com/en-us/HT201732] .
- Uma framework `/Library/Frameworks/Python.framework`, que inclui o executável Python e suas bibliotecas. O instalador adiciona

esse local ao seu *path* do console. Para desinstalar o Python, você pode remover esses três itens. É um link simbólico para o executável Python armazenado na pasta `/usr/local/bin/`.

Nota

As versões recentes do macOS incluem um comando **python3** em `/usr/bin/python3` que aponta para uma versão geralmente mais antiga e incompleta de Python fornecida por e para uso pelas ferramentas de desenvolvimento da Apple, **Xcode** ou o **Command Line Tools for Xcode**. Você nunca deve modificar ou tentar excluir essa instalação por ser controlada pela Apple e é usada por softwares da Apple ou de terceiros. Se você optar por instalar uma versão mais recente do Python a partir de `python.org`, você terá duas instalações diferentes, mas funcionais, de Python no computador que podem coexistir. As opções do instalador padrão devem garantir que o comando **python3** seja usado em vez do **python3** do sistema.

5.1.2. Como executar um script Python

Há duas maneiras de invocar o interpretador Python. Se você estiver familiarizado com o uso de um console Unix em terminal, você pode invocar `python3.15` ou `python3`, opcionalmente seguido por uma ou mais opções da linha de comando (descritas em Linha de comando e ambiente). O tutorial de Python também tem uma seção útil sobre o uso interativo do Python a partir de um console.

Você também pode invocar o interpretador por um ambiente de desenvolvimento integrado (IDE). O IDLE — Editor e console Python é um editor e ambiente de desenvolvimento básico incluído com a distribuição padrão de Python. **IDLE** inclui um menu Ajuda que te permite acessar a documentação de Python. Se você for completamente iniciante em Python, você pode ler o tutorial de introdução na documentação.

Há muitos outros editores e IDEs disponíveis. Consulte Editores e IDEs para mais informações.

Para executar um script Python na janela do terminal, você pode invocar o interpretador com o nome do arquivo de script:

```
python3.15 meuscript.py
```

Para executar seu script a partir do Finder, você pode:

- Arrastá-lo para o **Python Launcher**.
- Selecione **Python Launcher** como a aplicação padrão para abrir seu script (ou qualquer script `.py`) na janela de informações Finder Info, e clique duas vezes nele. O **Python Launcher** tem várias preferências para controlar como o script é iniciado. Você pode arrastar opções para alterá-las por uma invocação, ou usar o menu `Preferências` para alterar configurações globalmente.

Saiba que executar o script diretamente do Finder pode produzir resultados diferentes daqueles obtidos em uma janela de terminal, pois o script não será executado no ambiente normal do console, incluindo qualquer configuração de variáveis de ambiente nos perfis do console. E, como em qualquer outro script ou programa, certifique-se do que está prestes a executar.

5.2. Distribuições alternativas

Além do instalador padrão do `python.org` para macOS, existem distribuições de terceiros para macOS que podem incluir funcionalidades adicionais. Algumas distribuições populares e seus recursos-chave são:

ActivePython [https://www.activestate.com/products/python/]
: Instalador com compatibilidade multiplataforma, documentação

Anaconda [https://www.anaconda.com/download/]
: Módulos científicos populares (como numpy, scipy e pandas) e o gerenciador de pacotes `conda`.

Homebrew [https://brew.sh]

Gerenciador de pacotes para macOS que inclui várias versões do Python e muitos pacotes Python de terceiros (incluindo numpy, scipy e pandas).

MacPorts [https://www.macports.org]
: Outro gerenciador de pacotes para macOS, incluindo várias versões de Python e muitos pacotes Python de terceiros. Pode incluir versões pré-construídas do Python e muitos pacotes para versões mais antigas do macOS.

Note que distribuições podem não incluir versões mais recentes de Python ou de outras bibliotecas, e não são mantidas ou providas pela equipe do Python.

5.3. Instalar pacotes Python adicionais

Consulte o Guia de Usuário para Empacotamento de Python [https://packaging.python.org/en/latest/tutorials/installing-packages/] para mais informações.

5.4. Programação de GUI

Existem várias opções para criar aplicações GUI no Mac com Python.

O kit de ferramentas GUI padrão de Python é o módulo `tkinter`, baseado no kit de ferramentas multiplataforma Tk (https://www.tcl.tk). Uma versão nativa do Tk para macOS está incluída no instalador.

O PyObjC é uma ligação em Python para o Objective-C/Cocoa da Apple framework. Informações sobre o PyObjC estão disponíveis em pyobjc [https://pypi.org/project/pyobjc/].

Há vários kits de ferramentas GUI alternativas disponíveis para o macOS, incluindo:

- PySide [https://www.qt.io/qt-for-python]: Ligações oficiais em Python ao kit de ferramentas Qt GUI [https://wiki.qt.io/Qt_for_Python] .
- PyQt [https://riverbankcomputing.com/software/pyqt/]: Ligações alternativas em Python ao Qt.

- Kivy [https://kivy.org]: Um kit de ferramentas GUI multiplataforma que dá suporte a plataformas de desktop e dispositivos móveis.
- Toga [https://toga.readthedocs.io]: Parte do BeeWare Project [https://beeware.org]; dá suporte a aplicativos para desktop, dispositivos móveis, Web e console.
- wxPython [https://wxpython.org]: Um kit de ferramentas multiplataforma que dá suporte a sistemas operacionais desktop.

5.5. Tópicos avançados

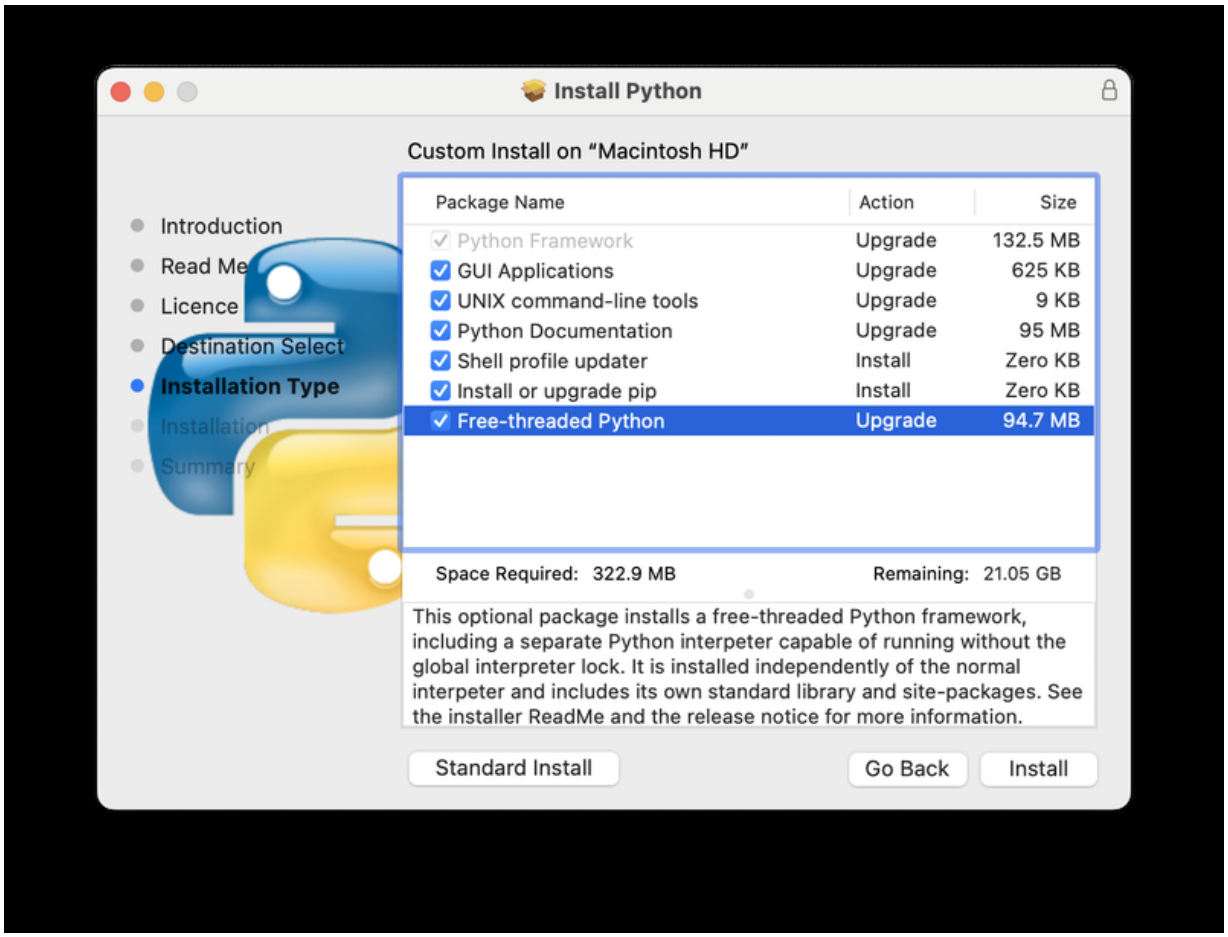
5.5.1. Instalando binários com threads livres

Adicionado na versão 3.13.

Alterado na versão 3.15: O suporte a threads livres é agora instalado por padrão.

O pacote de instalação do Python para macOS em `python.org` pode instalar, por padrão, uma construção adicional do Python 3.15 que dá suporte a threads livres (executar com a trava global do interpretador desativada). Verifique a página de versões em `python.org` para obter informações atualizadas.

O modo com threads livres está funcionando e continua sendo aprimorado, mas há alguma sobrecarga adicional em cargas de trabalho de thread única em comparação com a construção regular. Além disso, pacotes de terceiros, em particular aqueles com um módulo de extensão, podem não estar prontos para uso em uma construção com threads livres e reativarão a GIL. A partir do Python 3.15. o suporte para threads livres é instalado por padrão. Ele é fornecido no pacote como uma opção de instalação separada, que pode ser desmarcada clicando no botão **Customize** na etapa **Installation Type** do instalador, conforme descrito acima.



Se a caixa ao lado do nome do pacote **Free-Threaded Python** for marcada (pois é o padrão), um arquivo `PythonT.framework` também será instalado junto ao arquivo `Python.framework` em `/Library/Frameworks`. Essa configuração permite que a compilação de Python 3.15 com threads livres coexista no seu sistema com a compilação de Python 3.15 tradicional (com GIL) com risco mínimo durante instalação ou testagem. Esse layout de instalação pode mudar em lançamentos futuros.

Precauções e limitações conhecidas:

- O pacote **Unix command-line tools**, que é selecionado por padrão, instalará links em `/usr/local/bin` para `python3.15t`, o interpretador com threads livres e `python3.15t-config`, um utilitário de configuração que pode ser útil para criadores de pacote. Como `/usr/local/bin` é normalmente incluído em seu `PATH` do console,

geralmente não é necessário alterar a variável de ambiente do `PATH` para usar o `python3.15t`.

- Neste lançamento, o pacote **Shell profile updater** e o comando `Update Shell Profile.command` em `/Applications/Python 3.15/` não dão suporte ao pacote com threads livres.

- A construção com threads livres e a compilação tradicional têm caminhos de pesquisa separados e diretórios `site-packages` separados, portanto, por padrão, se você precisar de um pacote disponível em ambas construções, talvez seja necessário instalá-lo em ambos. O pacote com threads livres instalará uma instância separada do **pip** para uso com o `python3.15t`.

    - Para instalar um pacote usando **pip** sem um **venv**:

      ```
      python3.15t -m pip install <package_name>
      ```

- Ao trabalhar com vários ambientes Python, geralmente é mais seguro e mais fácil criar e usar ambientes virtuais. Isso pode tanto evitar possíveis conflitos de comandos, quanto evitar a confusão sobre qual Python está sendo usado:

  ```
  python3.15t -m venv <venv_name>
  ```

  e, em seguida, **activate**.

- Para executar uma versão do IDLE com threads livres:

  ```
  python3.15t -m idlelib
  ```

- Os interpretadores em ambos as construções respondem às mesmas variáveis de ambiente PYTHON, o que pode gerar resultados inesperados, por exemplo, se você tiver `PYTHONPATH` definido em um perfil de console. Se necessário, há opções de linha de comando como `-E` para ignorar essas variáveis de ambiente.

- A construção com threads livres é ligada às bibliotecas compartilhadas de terceiros, como `OpenSSL` e `Tk`, instaladas no framework tradicional.

Isso significa que ambas as compilações também compartilham um conjunto de certificados, como instalados pelo script **Install Certificates.command**, portanto, ele só precisa ser executado uma vez.

- Se você não puder depender do link em `/usr/local/bin` apontando para `python3.15t` com threads livres do `python.org` (por exemplo, se quiser instalar sua própria versão lá ou se alguma outra distribuição instalá-lo lá), você poderá definir explicitamente sua variável de ambiente de console `PATH` para incluir o diretório da framework `PythonT bin`:

  ```
  export PATH="/Library/Frameworks/PythonT.framework/Versions/
  ```

  A instalação tradicional do framework por padrão faz algo semelhante, exceto pelo arquivo `Python.framework`. Observe que ter dois diretórios `bin` do framework em `PATH` pode gerar confusão se houver nomes duplicados, como `python3.15`, em ambos; qual deles é realmente usado depende da ordem em que aparecem em `PATH`. Os comandos `which python3.x` ou `which python3.xt` pode mostrar qual caminho está sendo usado. Usar ambientes virtuais pode ajudar a evitar essas ambiguidades. Outra opção pode ser criar um **alias** no console para o interpretador desejado, como:

  ```
  alias py3.15="/Library/Frameworks/Python.framework/Versions/
  alias py3.15t="/Library/Frameworks/PythonT.framework/Version
  ```

5.5.2. Instalação usando a linha de comando

Se você quiser usar automação para instalar o pacote de instalação `python.org` (em vez de usar o aplicativo GUI **Installer** do macOS), o utilitário para linha de comando **installer** permite que você selecione opções que não sejam do padrão. Se você não estiver familiarizado com o **installer**, ele pode ser um tanto enigmático (consulte **man installer** para obter mais informações). Como exemplo, o seguinte trecho de console mostra uma maneira de fazer isso, usando a versão `3.15.0b2` e desmarcando a opção de interpretador com threads livres:

```
RELEASE="python-3.150b2-macos11.pkg"

# download installer pkg
curl -O https://www.python.org/ftp/python/3.15.0/${RELEASE}

# create installer choicechanges to customize the install:
#    disable the PythonTFramework-3.15 package
#    while accepting the other defaults (install all other packa
cat > ./choicechanges.plist <<EOF
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www
<plist version="1.0">
<array>
        <dict>
                <key>attributeSetting</key>
                <integer>0</integer>
                <key>choiceAttribute</key>
                <string>selected</string>
                <key>choiceIdentifier</key>
                <string>org.python.Python.PythonTFramework-3.15<
        </dict>
</array>
</plist>
EOF

sudo installer -pkg ./${RELEASE} -applyChoiceChangesXML ./choice
```

Após a instalação padrão, você pode testar as duas construções disponíveis com algo como:

```
$ # test that the free-threaded interpreter was installed if the
$ /usr/local/bin/python3.15t -VV
Python 3.15.0b2 free-threading build (v3.15.0b2:3a83b172af, Jun
$ #    and the traditional interpreter
$ /usr/local/bin/python3.15 -VV
Python 3.15.0b2 (v3.15.0b2:3a83b172af, Jun  5 2024, 12:50:24) [C
$ # test that they are also available without the prefix if /usr
$ python3.15t -VV
Python 3.15.0b2 free-threading build (v3.15.0b2:3a83b172af, Jun
$ python3.15 -VV
Python 3.15.0b2 (v3.15.0b2:3a83b172af, Jun  5 2024, 12:50:24) [C
```

Nota

Os instaladores atuais do `python.org` só instalam em locais fixos, como `/Library/Frameworks/`, `/Applications` e `/usr/local/bin`. Não é possível usar a opção `-domain` do **installer** para instalar em outros locais.

5.5.3. Distribuindo aplicações Python

Existe uma variedade de ferramentas para converter seu código Python em uma aplicação distribuível independente:

- py2app [https://pypi.org/project/py2app/]: Oferece suporte à criação de pacotes `.app` do macOS a partir de um projeto Python.
- Briefcase [https://briefcase.readthedocs.io]: Parte do BeeWare Project [https://beeware.org]; uma ferramenta de empacotamento multiplataforma que permite criar *bundles* `.app` no macOS, além de gerenciar assinaturas digitais e a notarização.
- PyInstaller [https://pyinstaller.org/]: Uma ferramenta de empacotamento multiplataforma que cria um único arquivo ou pasta como um artefato distribuível.

5.5.4. Conformidade com a App Store

As aplicações enviadas para distribuição pela macOS App Store devem passar pelo processo de revisão de aplicativos da Apple. Esse processo inclui um conjunto de regras de validação automatizadas que inspecionam o pacote de aplicativos enviado em busca de código problemático.

A biblioteca padrão do Python contém alguns códigos que violam essas regras automatizadas. Embora essas violações pareçam ser falsos positivos, as regras de revisão da Apple não podem ser contestadas. Portanto, é necessário modificar a biblioteca padrão do Python para que uma aplicação passe na revisão da App Store.

A árvore de fontes do Python contém um arquivo de patch [https://github.com/python/cpython/tree/3.15/Mac/Resources/app-store-compliance.patch] que

removerá todo o código que é conhecido por causar problemas no processo de revisão da App Store. Este patch é aplicado automaticamente quando o CPython é configurado com a opção `--with-app-store-compliance`.

Este patch normalmente não é necessário para usar o CPython em um Mac; nem é necessário se você estiver distribuindo uma aplicação *fora* da App Store do macOS. É necessário *apenas* se você estiver usando a App Store do macOS como canal de distribuição.

5.6. Outros recursos

A página de ajuda do python.org [https://www.python.org/about/help/] contém links para vários recursos úteis. A lista de discussão Pythonmac-SIG [https://www.python.org/community/sigs/current/pythonmac-sig/] é outro recurso específico de suporte para usuários e desenvolvedores de Python no Mac.

6. Utilizando Python no Android

Python no Android é diferente do Python em plataformas de desktop. Em uma plataforma desktop, o Python geralmente é instalado como um recurso do sistema que pode ser usado por qualquer usuário daquele computador. Os usuários então interagem com o Python executando um executável **python** e inserindo comandos em um prompt interativo ou executando um script Python.

No Android, não existe o conceito de instalação como recurso do sistema. A única unidade de distribuição de software é uma aplicação, ou "app". Também não há console onde você possa executar um executável **python** ou interagir com um REPL do Python.

Como resultado, a única maneira de usar Python no Andoid é no modo incorporado – ou seja, escrevendo uma aplicação Android nativo, incorporando um interpretador Python usando `libpython` e invocando o código Python usando API de incorporação do Python. O interpretador Python completo, a biblioteca padrão e todo o seu código Python são então empacotados para sua aplicação para seu próprio uso privado.

A biblioteca padrão do Python tem algumas omissões e restrições notáveis no Andoid. Consulte o guia de disponibilidade de API para Android para obter detalhes.

6.1. Adicionando Python a uma aplicação Android

A maioria dos desenvolvedores de aplicações deve usar uma das seguintes ferramentas, o que proporcionará uma experiência muito mais fácil:

- Briefcase [https://briefcase.beeware.org], do projeto BeeWare
- Buildozer [https://buildozer.readthedocs.io], do projeto Kivy
- Chaquopy [https://chaquo.com/chaquopy]
- pyqtdeploy [https://www.riverbankcomputing.com/static/Docs/pyqtdeploy/]
- Termux [https://termux.dev/en/]

Se você tem certeza de que quer fazer tudo isso manualmente, continue lendo. Você pode usar a aplicação de teste [https://github.com/python/cpython/tree/3.15/Platforms/Android/testbed] como um guia; cada passo abaixo contém um link para o arquivo relevante.

- Primeiro, obtenha uma construção do Python para Android:
  - A maneira mais fácil é baixar uma versão do Android em python.org [https://www.python.org/downloads/android/]. O diretório `prefix` mencionado abaixo está no nível raiz do pacote.
  - Ou, se quiser construí-la você mesmo, siga as instruções em Platforms/Android/README.md [https://github.com/python/cpython/tree/3.15/Platforms/Android/README.md]. O diretório `prefix` será criado em `cross-build/HOST`.
- Adicione código ao seu arquivo build.gradle [https://github.com/python/cpython/tree/3.15/Platforms/Android/testbed/app/build.gradle.kts] para copiar os seguintes itens para o seu projeto. Todos, exceto seu próprio código Python, podem ser copiados de `prefix/lib`:
  - Em suas bibliotecas JNI:
    - `libpython*.*.so`
    - `lib*_python.so` (bibliotecas externas, tal como OpenSSL)
  - Em seus ativos (*assets*):
    - `python*.*` (a biblioteca padrão do Python)
    - `python*.*/site-packages` (seu próprio código Python)
- Adicione código ao seu aplicativo para extrair os ativos para o sistema de arquivos [https://github.com/python/cpython/tree/3.15/Platforms/Android/testbed/app/src/main/java/org/python/testbed/MainActivity.kt].
- Adicione código a sua aplicação para iniciar o Python em modo incorporado [https://github.com/python/cpython/tree/3.15/Platforms/Android/testbed/app/src/main/c/main_activity.c]. Isso precisará ser código C chamado via JNI.

6.2. Construindo um pacote Python para Android

Pacotes Python podem ser criados para Android como wheels e lançados no PyPI. A ferramenta recomendada para isso é cibuildwheel [https://cibuildwheel.pypa.io/en/stable/platforms/#android], que automatiza todos os

detalhes da configuração de um ambiente de compilação cruzada, da construção da wheel e dos testes em um emulador.

7. Usando Python no iOS

Autores:: Russell Keith-Magee (2024-03)

Python no iOS é diferente do Python em plataformas de desktop. Em uma plataforma desktop, o Python geralmente é instalado como um recurso do sistema que pode ser usado por qualquer usuário daquele computador. Os usuários então interagem com o Python executando um executável **python** e inserindo comandos em um prompt interativo ou executando um script Python.

No iOS, não existe o conceito de instalação como recurso do sistema. A única unidade de distribuição de software é uma aplicação, ou "app". Também não há console onde você possa executar um executável **python** ou interagir com um REPL do Python.

Como resultado, a única maneira de usar Python no iOS é no modo incorporado – ou seja, escrevendo uma aplicação iOS nativo e incorporando um interpretador Python usando `libPython` e invocando o código Python usando API de incorporação do Python. O interpretador Python completo, a biblioteca padrão e todo o seu código Python são então empacotados como um pacote independente que pode ser distribuído pela iOS App Store.

Se você deseja experimentar pela primeira vez escrever uma aplicação iOS em Python, projetos como BeeWare [https://beeware.org] e Kivy [https://kivy.org] irão fornecer uma experiência de usuário muito mais acessível. Esses projetos gerenciam as complexidades associadas à execução de um projeto iOS, portanto, você só precisa lidar com o próprio código Python.

7.1. Python em tempo de execução no iOS

7.1.1. Compatibilidade com a versão do iOS

A versão mínima suportada do iOS é especificada em tempo de compilação, usando a opção **`--host`** do `configure`. Por padrão, quando compilado para

iOS, Python será compilado com uma versão iOS mínima suportada de 13.0. Para usar uma versão mínima diferente do iOS, forneça o número da versão como parte do argumento **`--host`** - por exemplo, `--host=arm64-apple-ios15.4-simulator` compilaria um simulador ARM64 construído com uma meta de implantação de 15.4.

7.1.2. Identificação da plataforma

Ao executar no iOS, `sys.platform` reportará como `ios`. Este valor será retornado em um iPhone ou iPad, independentemente da aplicação estar executando no simulador ou em um dispositivo físico.

Informações sobre o ambiente de execução específico, incluindo a versão do iOS, modelo do dispositivo e se o dispositivo é um simulador, podem ser obtidas usando **`platform.ios_ver()`**. **`platform.system()`** reportará `iOS` ou `iPadOS`, dependendo do dispositivo.

`os.uname()` reporta detalhes em nível de kernel; ele reportará o nome `Darwin`.

7.1.3. Disponibilidade da biblioteca padrão

A biblioteca padrão do Python tem algumas omissões e restrições notáveis no iOS. Consulte o guia de disponibilidade de API para iOS para obter detalhes.

7.1.4. Módulos de extensão binária

Uma diferença notável sobre o iOS como plataforma é que a distribuição da App Store impõe requisitos rígidos ao empacotamento de uma aplicação. Um desses requisitos rege como os módulos de extensão binária são distribuídos.

A iOS App Store exige que *todos* os módulos binários em uma aplicação iOS sejam bibliotecas dinâmicas, contidas em um framework com metadados apropriados, armazenados na pasta `Frameworks` da aplicação

empacotada. Pode haver apenas um único binário por framework, e não pode haver nenhum material binário executável fora da pasta `Frameworks`.

Isto entra em conflito com a abordagem usual do Python para distribuição de binários, que permite que um módulo de extensão binária seja carregado de qualquer local em `sys.path`. Para garantir a conformidade com as políticas da App Store, um projeto iOS deve pós-processar quaisquer pacotes Python, convertendo módulos binários `.so` em estruturas independentes individuais com metadados e assinatura apropriados. Para obter detalhes sobre como realizar esse pós-processamento, consulte o guia para adicionar Python ao seu projeto.

Para ajudar o Python a descobrir binários em seu novo local, o arquivo `.so` original em `sys.path` é substituído por um arquivo `.fwork`. Este arquivo é um arquivo texto que contém a localização do binário do framework, relativo ao pacote de aplicações. Para permitir que o framework retorne ao local original, o framework deve conter um arquivo `.origin` que contém a localização do arquivo `.fwork`, relativo ao pacote da aplicação.

Por exemplo, considere o caso de uma importação `from foo.bar import _whiz`, onde `_whiz` é implementado com o módulo binário `sources/foo/bar/_whiz.abi3.so`, com `sources` sendo o local registrado em `sys.path`, relativo ao pacote da aplicação. Este módulo *deve* ser distribuído como `Frameworks/foo.bar._whiz.framework/foo.bar._whiz` (criando o nome do framework a partir do caminho de importação completo do módulo), com um arquivo `Info.plist` no diretório `.framework` identificando o binário como um framework. O módulo `foo.bar._whiz` seria representado no local original com um arquivo marcador `sources/foo/bar/_whiz.abi3.fwork`, contendo o caminho `Frameworks/foo.bar._whiz/foo.bar._whiz`. O framework também conteria `Frameworks/foo.bar._whiz.framework/foo.bar._whiz.origin`, contendo o caminho para o arquivo `.fwork`.

Ao executar no iOS, o interpretador Python instalará um **`AppleFrameworkLoader`** que é capaz de ler e importar arquivos `.fwork`. Uma vez importado, o atributo `__file__` do módulo binário reportará como a localização do arquivo `.fwork`. Entretanto, o **`ModuleSpec`** para o módulo

carregado reportará a `origin` como a localização do binário na pasta do framework.

7.1.5. Binários stub do compilador

O Xcode não expõe compiladores explícitos para iOS; em vez disso, ele usa um script `xcrun` que resolve um caminho do compilador (por exemplo, `xcrun --sdk iphoneos clang` para obter o `clang` para um dispositivo iPhone). No entanto, usar este script apresenta dois problemas:

- A saída de `xcrun` inclui caminhos que são específicos da máquina, resultando em um módulo sysconfig que não pode ser compartilhado entre usuários; e
- Isso resulta em definições `CC`/`CPP`/`LD`/`AR` que incluem espaços. Existem muitas ferramentas do ecossistema C que pressupõem que você pode dividir uma linha de comando no primeiro espaço para obter o caminho para o executável do compilador; este não é o caso ao usar `xcrun`.

Para evitar esses problemas, o Python forneceu stubs para essas ferramentas. Esses stubs são wrappers de script de shell em torno das ferramentas subjacentes `xcrun`, distribuídos em uma pasta `bin` distribuída junto com a estrutura iOS compilada. Esses scripts são relocáveis e sempre serão resolvidos para os caminhos apropriados do sistema local. Ao incluir esses scripts na pasta bin que acompanha um framework, o conteúdo do módulo `sysconfig` se torna útil para usuários finais compilarem seus próprios módulos. Ao compilar módulos Python de terceiros para iOS, você deve garantir que esses binários stub estejam no seu caminho.

7.2. Instalando Python no iOS

7.2.1. Ferramentas para construir aplicações de iOS

A construção para iOS requer o uso das ferramentas Xcode da Apple. É altamente recomendável que você use a versão estável mais recente do Xcode. Isso exigirá o uso da versão mais recente (ou segunda) do macOS lançada recentemente, já que a Apple não mantém o Xcode para versões

mais antigas do macOS. As ferramentas de linha de comando do Xcode não são suficientes para o desenvolvimento iOS; você precisa de uma instalação *completa* do Xcode.

Se quiser executar seu código no simulador iOS, você também precisará instalar um iOS Simulator Platform. Você deverá ser solicitado a selecionar um iOS Simulator Platform ao executar o Xcode pela primeira vez. Alternativamente, você pode adicionar um iOS Simulator Platform selecionando na guia Platforms do painel Settings do Xcode.

7.2.2. Adicionando Python a um projeto iOS

Python pode ser adicionado a qualquer projeto iOS, usando Swift ou Objective C. Os exemplos a seguir usarão Objective C; se você estiver usando Swift, poderá achar uma biblioteca como PythonKit [https://github.com/pvieito/PythonKit] útil.

Para adicionar Python a um projeto Xcode de iOS:

1. Construa ou obtenha um `XCFramework` em Python. Veja as instruções em Platforms/Apple/iOS/README.md [https://github.com/python/cpython/tree/3.15/Platforms/Apple/iOS/README.md] (na distribuição fonte do CPython) para detalhes sobre como construir um `XCFramework` em Python. No mínimo, você precisará de uma construção que tenha suporte `arm64-apple-ios`, além de `arm64-apple-ios-simulator` ou `x86_64-apple-ios-simulator`.

2. Arraste o `XCframework` para o seu projeto iOS. Nas instruções a seguir, presumiremos que você colocou o `XCframework` na raiz do seu projeto; no entanto, você pode usar qualquer outro local desejado ajustando os caminhos conforme necessário.

3. Adicione o código da sua aplicação como uma pasta no seu projeto Xcode. Nas instruções a seguir, presumiremos que seu código de usuário está em uma pasta chamada `app` na raiz do seu projeto; você pode usar qualquer outro local ajustando os caminhos conforme necessário. Certifique-se de que esta pasta esteja associada ao destino da sua aplicação.

4. Selecione o destino da aplicação selecionando o nó raiz do seu projeto Xcode e, em seguida, o nome do destino na barra lateral que aparece.

5. Nas configurações de “General”, em “Frameworks, Libraries and Embedded Content”, adicione `Python.xcframework`, com “Embed & Sign” selecionado.

6. Na guia “Build Settings”, modifique o seguinte:

    - Build Options
        - User Script Sandboxing: No
        - Enable Testability: Yes
    - Search Paths
        - Framework Search Paths: `$(PROJECT_DIR)`
        - Header Search Paths: `"$(BUILT_PRODUCTS_DIR)/Python.framework/Headers"`
    - Apple Clang - Warnings - All languages
        - Quoted Include In Framework Header: No

7. Adicione uma etapa de construção que processa a biblioteca padrão do Python e suas dependências de binários Python. Na aba “Build Phases”, adicione uma nova etapa de construção “Run Script” *antes* da etapa “Embed Frameworks”, mas *depois* da etapa “Copy Bundle Resources”. Nomeie a etapa como “Process Python libraries”, desative a caixa de seleção “Based on dependency analysis” e defina o conteúdo do script como:

    ```
    set -e
    source $PROJECT_DIR/Python.xcframework/build/build_utils.sh
    install_python Python.xcframework app
    ```

    Se você colocou seu XCframework em algum lugar diferente da raiz do seu projeto, modifique o caminho para o primeiro argumento.

8. Adicione código Objective C para inicializar e usar um interpretador Python em modo embarcado. Você deve garantir que:

    - Modo UTF-8 (**`PyPreConfig.utf8_mode`**) esteja habilitado;
    - Stdio buffered (**`PyConfig.buffered_stdio`**) esteja desabilitado;

    - Escrita de bytecode (`PyConfig.write_bytecode`) esteja desabilitada;
    - Manipuladores de sinais (`PyConfig.install_signal_handlers`) estejam habilitados;
    - Registro de logs do sistema (`PyConfig.use_system_logger`) esteja desabilitado (opcional, mas fortemente recomendado; isso está habilitado por padrão);
    - `PYTHONHOME` para o interpretador esteja configurado para apontar para a subpasta `python` do pacote da sua aplicação; e
    - O `PYTHONPATH` para o interpretador inclui:
        - a subpasta `python/lib/python3.X` do pacote da sua aplicação,
        - a subpasta `python/lib/python3.X/lib-dynload` do pacote da sua aplicação, e
        - a subpasta `app` do pacote da sua aplicação

    O local do pacote da sua aplicação pode ser determinada usando `[[NSBundle mainBundle] resourcePath]`.

Os passos 7 e 8 destas instruções presumem que você tem uma única pasta de código de aplicação Python puro, chamada `app`. Se você tiver módulos binários de terceiros em sua aplicação, serão necessárias algumas etapas adicionais:

- Você precisa garantir que todas as pastas que contenham binários de terceiros sejam associadas ao destino da aplicação ou sejam explicitamente copiadas como parte da etapa 7. A etapa 7 também deve limpar quaisquer binários que não sejam apropriados para a plataforma que uma construção específica está direcionando (ou seja, exclua todos os binários do dispositivo se estiver criando uma aplicação direcionado ao simulador).
- Se você estiver usando uma pasta separada para pacotes de terceiros, certifique-se de que essa pasta seja adicionada ao final da chamada de `install_python` no passo 7 e que seja parte da configuração `PYTHONPATH` no passo 8.
- Se alguma das pastas que contêm pacotes de terceiros contiver arquivos `.pth`, você deverá adicionar essa pasta como um *diretório de*

site (usando `site.addsitedir()`), em vez de adicioná-la diretamente a `PYTHONPATH` ou `sys.path`.

7.2.3. Testando um pacote Python

A árvore de fontes do CPython contém um projeto de banco de testes [https://github.com/python/cpython/tree/3.15/Platforms/Apple/testbed] que é usado para executar o conjunto de testes do CPython no simulador do iOS. Este banco de testes também pode ser usado como um projeto de banco de testes para executar o conjunto de testes da sua biblioteca Python no iOS.

Depois de criar ou obter um iOS XCFramework (consulte Platforms/Apple/iOS/README.md [https://github.com/python/cpython/tree/3.15/Platforms/Apple/iOS/README.md] para obter detalhes), crie um clone do projeto de banco de testes de iOS do Python. Se você usou o script de construção `Platforms/Apple` para construir o XCframework, você pode executar:

```
$ python cross-build/iOS/testbed clone --app <caminho/para/módul
```

Ou, se você tiver criado seu próprio XCframework, execute:

```
$ python Platforms/Apple/testbed clone --platform iOS --framewor
```

Todas as pastas especificadas com o sinalizador `--app` serão copiadas para o projeto de banco de testes clonado. O banco de testes resultante será criado na pasta `app-testbed`. Neste exemplo, `módulo1` e `módulo2` seriam módulos importáveis em tempo de execução. Se seu projeto tiver dependências adicionais, elas podem ser instaladas na pasta `app-testbed/Testbed/app_packages` (usando `pip install --target app-testbed/Testbed/app_packages` ou similar).

Você pode então usar a pasta `app-testbed` para executar o conjunto de testes para seu aplicativo. Por exemplo, se `módulo1.tests` fosse o ponto de entrada para seu conjunto de testes, você poderia executar:

```
$ python app-testbed run -- módulo1.tests
```

Isso é o equivalente a executar `python -m módulo1.tests` em uma compilação Python de desktop. Quaisquer argumentos após `--` serão passados para o banco de testes como se fossem argumentos para `python -m` em uma máquina de desktop.

Você também pode abrir o projeto de teste no Xcode executando:

```
$ open app-testbed/iOSTestbed.xcodeproj
```

Isso permitirá que você use o conjunto completo de ferramentas do Xcode para depuração.

Os argumentos usados para executar o conjunto de testes são definidos como parte do plano de teste. Para modificar o plano de teste, selecione o nó do plano de teste na árvore do projeto (deve ser o primeiro filho do nó raiz) e selecione a aba “Configurations”. Modifique o valor “Arguments Passed On Launch” para alterar os argumentos de teste.

O plano de teste também desabilita os testes paralelos e especifica o uso do arquivo `Testbed.lldbinit` para fornecer a configuração do depurador. A configuração padrão do depurador desabilita pontos de interrupção automáticos nos sinais `SIGINT`, `SIGUSR1`, `SIGUSR2` e `SIGXFSZ`.

7.3. Conformidade com a App Store

O único mecanismo para distribuir aplicações para dispositivos iOS de terceiros é enviar a aplicação para a App Store do iOS; as aplicações enviadas para distribuição devem passar pelo processo de revisão de aplicações da Apple. Esse processo inclui um conjunto de regras de validação automatizadas que inspecionam o pacote de aplicação enviado em busca de código problemático. Há alguns passos que devem ser seguidas para garantir que sua aplicação consiga passar por esses passos de validação.

7.3.1. Código incompatível na biblioteca padrão

A biblioteca padrão do Python contém alguns códigos que violam essas regras automatizadas. Embora essas violações pareçam ser falsos positivos, as regras de revisão da Apple não podem ser contestadas; portanto, é necessário modificar a biblioteca padrão do Python para que uma aplicação passe na revisão da App Store.

A árvore de fontes do Python contém um arquivo de patch [https://github.com/python/cpython/tree/3.15/Mac/Resources/app-store-compliance.patch] que removerá todo o código que é conhecido por causar problemas no processo de revisão da App Store. Este patch é aplicado automaticamente quando o ao construir para o iOS.

7.3.2. Manifestos de privacidade

Em abril de 2025, a Apple introduziu um requisito para que determinadas bibliotecas de terceiros forneçam um Manifesto de Privacidade [https://developer.apple.com/support/third-party-SDK-requirements]. Como resultado, se você tiver um módulo binário que usa uma das bibliotecas afetadas, deverá fornecer um arquivo `.xcprivacy` para essa biblioteca. O OpenSSL é uma biblioteca afetada por esse requisito, mas existem outras.

Se você produzir um módulo binário chamado `meumódulo.so` e usar o script de construção do Xcode descrito no passo 7 acima, poderá colocar um arquivo `meumódulo.xcprivacy` ao lado de `meumódulo.so`, e o manifesto de privacidade será instalado no local necessário quando o módulo binário for convertido em um framework.

8. Editores e IDEs

Há um grande número de IDEs com suporte à linguagem de programação Python. Vários editores e IDEs provêm destaques coloridos de sintaxe, ferramentas de depuração, e checagem do código conforme a **PEP 8** [https://peps.python.org/pep-0008/].

8.1. IDLE — editor e console Python

IDLE é o Ambiente Integrado de Desenvolvimento e Aprendizagem do Python (em inglês Integrated Development and Learning Environment) e geralmente vem junto com as instalações do Python. Se você estiver no Linux e não tiver o IDLE instalado consulte Instalando o IDLE no Linux. Para obter mais informações consulte, consulte os documentos do IDLE.

8.2. Outros editores e IDEs

As wikis da comunidades Python têm informações enviadas pela comunidade sobre Editores e IDEs. Acesse Ferramentas de Desenvolvimento [https://python.org.br/ferramentas/] para obter uma lista abrangente enviada pela comunidade brasileira ou Python Editors [https://wiki.python.org/moin/PythonEditors] e Integrated Development Environments [https://wiki.python.org/moin/IntegratedDevelopmentEnvironments] para listas enviadas pela comunidade internacional (em inglês).

A Referência da Linguagem Python

Este manual de referência descreve a sintaxe e a semântica fundamental da linguagem. Ele é conciso, mas busca ser preciso e completo.

Em outras partes, os tipos de objeto e funções embutidos são descritos em Referência da bibliotecas embutidas do Python. Os módulos da biblioteca padrão são descritos em A biblioteca padrão do Python.

Para uma introdução informal à linguagem, consulte O tutorial do Python. Para programadores em C ou C++, existem dois manuais adicionais: Estendendo e incorporando o interpretador Python descreve a imagem de alto nível de como escrever um módulo de extensão Python, e o Manual de referência da API C/Python descreve as interfaces disponíveis para programadores C/C++ em detalhes.

- 1. Introdução
  - 1.1. Implementações Alternativas
  - 1.2. Notação
- 2. Análise léxica
  - 2.1. Estrutura das linhas
  - 2.2. Outros tokens
  - 2.3. Nomes (identificadores e palavras reservadas)
  - 2.4. Literais
  - 2.5. Literais de string e bytes
  - 2.6. Literais numéricos
  - 2.7. Operadores e delimitadores
- 3. Modelo de dados
  - 3.1. Objetos, valores e tipos
  - 3.2. A hierarquia de tipos padrão
  - 3.3. Nomes de métodos especiais
  - 3.4. Corrotinas
- 4. Modelo de execução

- 4.1. Estrutura de um programa
- 4.2. Nomeação e ligação
- 4.3. Exceções
- 4.4. Componentes do Ambiente de Execução

- 5. O sistema de importação
  - 5.1. `importlib`
  - 5.2. Pacotes
  - 5.3. Caminho de busca
  - 5.4. Carregando
  - 5.5. O localizador baseado no caminho
  - 5.6. Substituindo o sistema de importação padrão
  - 5.7. Importações relativas ao pacote
  - 5.8. Considerações especiais para __main__
  - 5.9. Referências
- 6. Expressões
  - 6.1. Conversões aritméticas
  - 6.2. Átomos
  - 6.3. Primárias
  - 6.4. Expressão await
  - 6.5. O operador de potência
  - 6.6. Operações aritméticas unárias e bit a bit
  - 6.7. Operações binárias aritméticas
  - 6.8. Operações de deslocamento
  - 6.9. Operações binárias bit a bit
  - 6.10. Comparações
  - 6.11. Operações booleanas
  - 6.12. Expressões de atribuição
  - 6.13. Expressões condicionais
  - 6.14. Lambdas
  - 6.15. Listas de expressões
  - 6.16. Ordem de avaliação
  - 6.17. Precedência de operadores
- 7. Instruções simples
  - 7.1. Instruções de expressão
  - 7.2. Instruções de atribuição
  - 7.3. A instrução `assert`
  - 7.4. A instrução `pass`

- 7.5. A instrução `del`
- 7.6. A instrução `return`
- 7.7. A instrução `yield`
- 7.8. A instrução `raise`
- 7.9. A instrução `break`
- 7.10. A instrução `continue`
- 7.11. A instrução `import`
- 7.12. A instrução `global`
- 7.13. A instrução `nonlocal`
- 7.14. A instrução `type`

- 8. Instruções compostas
  - 8.1. A instrução `if`
  - 8.2. A instrução `while`
  - 8.3. A instrução `for`
  - 8.4. A instrução `try`
  - 8.5. A instrução `with`
  - 8.6. A instrução `match`
  - 8.7. Definições de função
  - 8.8. Definições de classe
  - 8.9. Corrotinas
  - 8.10. Listas de parâmetros de tipo
  - 8.11. Anotações
- 9. Componentes de Alto Nível
  - 9.1. Programas Python completos
  - 9.2. Entrada de arquivo
  - 9.3. Entrada interativa
  - 9.4. Entrada de expressão
- 10. Especificação completa da gramática

1. Introdução

Este manual de referência descreve a linguagem de programação Python. O mesmo não tem como objetivo de ser um tutorial.

Enquanto estou tentando ser o mais preciso possível, optei por usar especificações em inglês e não formal para tudo, exceto para a sintaxe e análise léxica. Isso deve tornar o documento mais compreensível para o leitor intermediário, mas deixará margem para ambiguidades. Consequentemente, caso estivesses vindo de Marte e tentasse reimplementar o Python a partir deste documento, sozinho, talvez precisarias adivinhar algumas coisas e, na verdade, provavelmente acabaria por implementar um linguagem bem diferente. Por outro lado, se estiveres usando o Python e se perguntando quais são as regras precisas sobre uma determinada área da linguagem, você definitivamente encontrá neste documento o que estás procurando. Caso queiras ver uma definição mais formal do linguagem, talvez possas oferecer seu tempo – ou inventar uma máquina de clonagem :-).

É perigoso adicionar muitos detalhes de implementação num documento de referência de uma linguagem – a implementação pode mudar e outras implementações da mesma linguagem podem funcionar de forma diferente. Por outro lado, o CPython é a única implementação de Python em uso de forma generalizada (embora as implementações alternativas continuem a ganhar suporte), e suas peculiaridades e particulares são por vezes dignas de serem mencionadas, especialmente quando a implementação impõe limitações adicionais. Portanto, encontrarás poucas “notas sobre a implementação” espalhadas neste documento.

Cada implementação do Python vem com vários módulos embutidos e por padrão. Estes estão documentados em A biblioteca padrão do Python. Alguns módulos embutidos são mencionados ao interagirem de forma significativa com a definição da linguagem.

1.1. Implementações Alternativas

Embora exista uma implementação do Python que seja, de longe, a mais popular, existem algumas implementações alternativas que são de de interesse particular e para públicos diferentes.

As implementações conhecidas são:

CPython
: Esta é a implementação original e a é a versão do Python que mais vem sendo sendo desenvolvido e a mesma está escrita com a linguagem C. Novas funcionalidades ou recursos da linguagem aparecerão por aqui primeiro.

Jython
: Versão do Python implementado em Java. Esta implementação pode ser usada como linguagem de Script em aplicações Java, ou pode ser usada para criar aplicativos usando as bibliotecas das classes do Java. Também vem sendo bastante utilizado para criar testes unitários para as bibliotecas do Java. Mais informações podem ser encontradas no the Jython website [https://www.jython.org/].

Python for .NET
: Essa implementação utiliza de fato a implementação CPython, mas é uma aplicação gerenciada .NET e disponibilizada como uma bibliotecas .NET. Foi desenvolvida por Brian Lloyd. Para obter mais informações, consulte o site do Python for .NET [https://pythonnet.github.io/].

IronPython
: Um versão alternativa do Python para a plataforma .NET. Ao contrário do Python.NET, esta é uma implementação completa do Python que gera IL e compila o código Python diretamente para assemblies .NET. Foi desenvolvida por Jim Hugunin, o criador original do Jython. Para obter mais informações, consulte o site do IronPython [https://ironpython.net/].

PyPy
: Uma implementação do Python escrita completamente em Python. A mesma suporta vários recursos avançados não encontrados em outras implementações, como suporte sem pilhas e um compilador Just in

Time. Um dos objetivos do projeto é incentivar a construção de experimentos com a própria linguagem, facilitando a modificação do interpretador (uma vez que o mesmos está escrito em Python). Informações adicionais estão disponíveis no site do projeto PyPy [https://pypy.org/].

Cada uma dessas implementações varia em alguma forma a linguagem conforme documentado neste manual, ou introduz informações específicas além do que está coberto na documentação padrão do Python. Consulte a documentação específica da implementação para determinar o que é necessário sobre a implementação específica que você está usando.

1.2. Notação

As descrições de análise lexical e sintaxe utilizam uma notação gramatical que é uma mistura de EBNF [https://pt.m.wikipedia.org/wiki/Formalismo_de_Backus-Naur_Estendido] e GASE [https://pt.m.wikipedia.org/wiki/Gramática_de_análise_sintática_de_expressão] (em inglês, PEG). Por exemplo:

```
name:   letter (letter | digit | "_")*
letter: "a"..."z" | "A"..."Z"
digit:  "0"..."9"
```

Neste exemplo, a primeira linha diz que um `name` é uma `letter` seguida por uma sequência de zero ou mais `letters`, `digits` e sublinhados. Uma `letter`, por sua vez, é qualquer um dos caracteres simples de `'a'` a `'z'` e de `A` a `Z`; um `digit` é um caractere simples de `0` a `9`.

Cada regra começa com um nome (que identifica a regra que está sendo definida) seguido por caractere de dois pontos, `:`. A definição à direita dos dois pontos utiliza os seguintes elementos de sintaxe:

- `name`: Um nome se refere a outra regra. Sempre que possível, é um link para a definição da regra.
  - `TOKEN`: Um nome em maiúsculas refere-se a um token. Para fins de definições gramaticais, tokens são o mesmo que regras.

- `"text"`, `'text'`: O texto entre aspas simples ou duplas deve ser escrito literalmente (sem as aspas). O tipo de citação é escolhido de acordo com o significado de `text`:
  - `'if'`: Um nome entre aspas simples denota uma palavra reservada.
  - `"case"`: Um nome entre aspas duplas denota uma palavra reservada contextual.
  - `'@'`: Um símbolo que não seja uma letra entre aspas simples denota um token **`OP`**, ou seja, um delimitador ou operador.
- `e1 e2`: Itens separados apenas por espaços em branco denotam uma sequência. Aqui, `e1` deve ser seguido por `e2`.
- `e1 | e2`: Uma barra vertical é usada para separar alternativas. Ela denota a "escolha ordenada" do GASE: se `e1` corresponder, `e2` não é considerado. Nas gramáticas GASE tradicionais, isso é escrito como uma barra, `/`, em vez de uma barra vertical. Veja **PEP 617** [https://peps.python.org/pep-0617/] para mais informações e detalhes.
- `e*`: Uma estrela significa zero ou mais repetições do item anterior.
- `e+`: Da mesma forma, um mais significa uma ou mais repetições.
- `[e]`: Uma frase entre colchetes significa zero ou uma ocorrência. Em outras palavras, a frase entre colchetes é opcional.
- `e?`: Um ponto de interrogação tem exatamente o mesmo significado que colchetes: o item anterior é opcional.
- `(e)`: Parênteses são usados para agrupamento.

A notação a seguir é usada somente em definições lexicais.

- `"a"..."z"`: Dois caracteres literais separados por três pontos significam uma escolha de qualquer caractere único no intervalo fornecido (inclusive) de caracteres ASCII.
- `<...>`: Uma frase entre parênteses angulares fornece uma descrição informal do símbolo correspondente (por exemplo, `<qualquer caractere ASCII, exceto "\">`) ou uma abreviação definida no texto próximo (por exemplo, `<Lu>`).

Algumas definições também usam *lookaheads*, que indicam que um elemento deve (ou não) corresponder a uma determinada posição, mas sem consumir nenhuma entrada:

- `&e`: um lookahead positivo (ou seja, `e` é necessário para corresponder)
- `!e`: um lookahead negativo (ou seja, `e` *não* é necessário para corresponder)

Os operadores unários (`*`, `+`, `?`) vinculam-se o mais firmemente possível; a barra vertical (`|`) vincula-se de forma mais frouxa.

O espaço em branco só tem significado para separar tokens.

As regras normalmente ficam contidas em uma única linha, mas regras muito longas podem ser encapsuladas:

```
literal: stringliteral | bytesliteral
         | integer | floatnumber | imagnumber
```

Alternativamente, as regras podem ser formatadas com a primeira linha terminando com dois pontos e cada alternativa começando com uma barra vertical em uma nova linha. Por exemplo:

```
literal:
   | stringliteral
   | bytesliteral
   | integer
   | floatnumber
   | imagnumber
```

Isso *não* significa que exista uma primeira alternativa vazia.

1.2.1. Definições lexicais e sintáticas

Há algumas diferenças entre a análise *léxica* e a *sintática*: o analisador léxico opera nos caracteres individuais da fonte de entrada, enquanto o *parser* (analisador sintático) opera no fluxo de tokens gerados pela análise léxica. No entanto, em alguns casos, o limite exato entre as duas fases é um detalhe da implementação do CPython.

A diferença prática entre os dois é que, em definições *léxicas*, todos os espaços em branco são significativos. O analisador léxico descarta todos os espaços em branco que não são convertidos em tokens como `token.INDENT` ou `NEWLINE`. Definições *sintáticas* então usam esses tokens, em vez dos caracteres de origem.

Esta documentação utiliza a mesma gramática BNF para ambos os estilos de definição. Todos os usos de BNF no próximo capítulo (Análise léxica) são definições lexicais; os usos nos capítulos subsequentes são definições sintáticas.

2. Análise léxica

Um programa Python é lido por um *analisador sintático*. A entrada para o analisador sintático é um fluxo de tokens, gerados pelo *analisador léxico* (também conhecido como *tokenizador*). Este capítulo descreve como o analisador léxico produz esses tokens.

O analisador léxico determina a codificação do texto do programa (UTF-8 por padrão), e o decodifica em caracteres da fonte. Se o texto não puder ser decodificado, uma `SyntaxError` é levantada.

Em seguida, o analisador léxico usa os caracteres da fonte para gerar um fluxo de tokens. Geralmente, o tipo de um token gerado depende do próximo caracter da fonte a ser processado. Além disso, outros comportamentos especiais do analisador também dependem do primeiro caracter da fonte que ainda não foi processado. A seguinte tabela é um resumo desses caracteres da fonte, com links para seções que contém mais informação.

Caractere	Token seguinte (ou outra documentação relevante)
• espaço • tabulação • quebra de página	• Espaço em branco
• CR, LF	• Nova linha • Indentação

Caractere	Token seguinte (ou outra documentação relevante)
• contrabarra (`\`)	• Junção de linha explícita • (Importante também em sequências de escape em strings)
• tralha (`#`)	• Comentário
• aspas (`'`, `"`)	• Literal de string
• Letra ASCII (`a-z`, `A-Z`) • Caractere não-ASCII	• Nome • Literal de string ou bytes prefixado.
• Sublinhado (`_`)	• Nome • (Também pode ser parte de numerais literais)
• número (`0-9`)	• Numeral literal
• ponto (`.`)	• Numeral literal • Operador

Caractere	Token seguinte (ou outra documentação relevante)
• ponto de interrogação (?) • cifrão ($) • acento grave, crase, ou backquote (`) • caractere de controle	• Erro (fora de literais de string e comentários)
• Outro caractere exibível	• Operador ou delimitador
• fim do arquivo	• Marcador de fim

2.1. Estrutura das linhas

Um programa Python é dividido em uma série de *linhas lógicas*.

2.1.1. Linhas lógicas

O fim de uma linha lógica é representado pelo token **NEWLINE**. As instruções não podem cruzar os limites da linha lógica, exceto onde **NEWLINE** for permitido pela sintaxe (por exemplo, entre as instruções de instruções compostas). Uma linha lógica é construída a partir de uma ou mais *linhas físicas* seguindo as regras explícitas ou implícitas que *juntam as linhas*.

2.1.2. Linhas físicas

Uma linha física é uma sequência de caracteres terminada por uma das seguintes sequências de fim de linha:

- o formulário Unix usando ASCII LF (avanço de linha),

- o formulário do Windows usando a sequência ASCII CR LF (retorno seguido de avanço de linha),
- o formato 'Classic Mac OS [https://en.wikipedia.org/wiki/Classic_Mac_OS]' usando o caractere ASCII CR (retorno).

Independentemente da plataforma, cada uma dessas sequências é substituída por um único caractere ASCII LF (avanço de linha). (Isso é feito até mesmo dentro de literais de string.) Cada linha pode usar qualquer uma das sequências; elas não precisam ser consistentes dentro de um arquivo.

O fim da entrada também serve como um terminador implícito para a linha física final.

Formalmente:

```
newline: <ASCII LF> | <ASCII CR> <ASCII LF> | <ASCII CR>
```

2.1.3. Comentários

Um comentário inicia com um caracter cerquilha (#) que não é parte de uma string literal, e termina com o fim da linha física. Um comentário significa o fim da linha lógica a menos que regras de junção de linha implicitas sejam invocadas. Comentários são ignorados pela sintaxe.

2.1.4. Declarações de codificação

Se um comentário na primeira ou segunda linha de um script Python corresponde com a expressão regular `coding[=:]\s*([-\w.]+)`, esse comentário é processado com uma declaração de codificação; o primeiro grupo dessa expressão indica a codificação do arquivo do código-fonte. A declaração de codificação deve aparecer em uma linha exclusiva para tal. Se está na segunda linha, a primeira linha também deve ser uma linha somente com comentário. As formas recomendadas de uma declaração de codificação são:

```
# -*- coding: <nome-codificação> -*-
```

que é reconhecido também por GNU Emacs, e

```
# vim:fileencoding=<nome-codificação>
```

que é reconhecido pelo VIM de Bram Moolenaar.

Se nenhuma codificação é declarada, a codificação padrão é UTF-8. Se a codificação implícita ou explícita de um arquivo é UTF-8, uma marca inicial de ordem de byte UTF-8 (`b'\xef\xbb\xbf'`) será ignorada em vez de ser um erro de sintaxe.

Se uma codificação é declarada, o nome da codificação deve ser reconhecida pelo Python (veja Standard Encodings). A codificação é usada por toda análise léxica, incluindo literais strings, comment and identificadores.

Toda a análise lexical, incluindo literais de string, comentários e identificadores, funciona em texto Unicode decodificado usando a codificação de origem. Qualquer ponto de código Unicode, exceto o caractere de controle NUL, pode aparecer no código-fonte Python.

```
source_character:  <any Unicode code point, except NUL>
```

2.1.5. Junção de linha explícita

Duas ou mais linhas físicas podem ser juntadas em linhas lógicas usando o caractere contrabarra (\) da seguinte forma: quando uma linha física termina com uma contrabarra que não é parte da uma literal string ou comentário, ela é juntada com a linha seguinte formando uma única linha lógica, removendo a contrabarra e o caractere de fim de linha seguinte. Por exemplo:

```
if 1900 < year < 2100 and 1 <= month <= 12 \
   and 1 <= day <= 31 and 0 <= hour < 24 \
   and 0 <= minute < 60 and 0 <= second < 60:   # Parece ser uma
        return 1
```

Uma linha terminada em uma contrabarra não pode conter um comentário. Uma barra invertida não continua um comentário. Uma contrabarra não continua um token, exceto para strings literais (ou seja, tokens diferentes de strings literais não podem ser divididos em linhas físicas usando uma contrabarra). Uma contrabarra é ilegal em qualquer outro lugar em uma linha fora de uma string literal.

2.1.6. Junção de linha implícita

Expressões entre parênteses, colchetes ou chaves podem ser quebradas em mais de uma linha física sem a necessidade do uso de contrabarras. Por exemplo:

```
month_names = ['Januari', 'Februari', 'Maart',      # Estes são
               'April',   'Mei',      'Juni',       # nomes hola
               'Juli',    'Augustus', 'September',  # para os me
               'Oktober', 'November', 'December']   # do ano
```

Linhas continuadas implicitamente podem conter comentários. O recuo das linhas de continuação não é importante. Linhas de continuação em branco são permitidas. Não há token NEWLINE entre linhas de continuação implícitas. Linhas continuadas implicitamente também podem ocorrer dentro de strings com aspas triplas (veja abaixo); nesse caso, eles não podem conter comentários.

2.1.7. Linhas em branco

Uma linha lógica que contém apenas espaços, tabulações, quebras de página e possivelmente um comentário é ignorada (ou seja, nenhum token **NEWLINE** é gerado). Durante a entrada interativa de instruções, o tratamento de uma linha em branco pode diferir dependendo da implementação do interpretador. No interpretador interativo padrão, uma linha lógica totalmente em branco (ou seja, uma que não contenha nem mesmo espaço em branco ou um comentário) encerra uma instrução multilinha.

2.1.8. Indentação

O espaço em branco (espaços e tabulações) no início de uma linha lógica é usado para calcular o nível de indentação da linha, que por sua vez é usado para determinar o agrupamento de instruções.

As tabulações são substituídas (da esquerda para a direita) por um a oito espaços, de modo que o número total de caracteres até e incluindo a substituição seja um múltiplo de oito (essa é intencionalmente a mesma regra usada pelo Unix). O número total de espaços que precedem o primeiro caractere não em branco determina o recuo da linha. O recuo não pode ser dividido em várias linhas físicas usando contrabarra; o espaço em branco até a primeira contrabarra determina a indentação.

A indentação é rejeitada como inconsistente se um arquivo de origem mistura tabulações e espaços de uma forma que torna o significado dependente do valor de uma tabulação em espaços; uma exceção `TabError` é levantada nesse caso.

Nota de compatibilidade entre plataformas: devido à natureza dos editores de texto em plataformas não-UNIX, não é aconselhável usar uma mistura de espaços e tabulações para o recuo em um único arquivo de origem. Deve-se notar também que diferentes plataformas podem limitar explicitamente o nível máximo de indentação.

Um caractere de quebra de página pode estar presente no início da linha; ele será ignorado para os cálculos de indentação acima. Os caracteres de quebra de página que ocorrem em outro lugar além do espaço em branco inicial têm um efeito indefinido (por exemplo, eles podem redefinir a contagem de espaços para zero).

Os níveis de indentação das linhas consecutivas são usados para gerar tokens `INDENT` e `DEDENT`, usando uma pilha, como segue.

Antes da leitura da primeira linha do arquivo, um único zero é colocado na pilha; esse valor nunca mais será exibido. Os números colocados na pilha sempre aumentarão estritamente de baixo para cima. No início de cada linha lógica, o nível de indentação da linha é comparado ao topo da pilha. Se for igual, nada acontece. Se for maior, ele é colocado na pilha e é gerado um token `INDENT`. Se for menor, *deve* ser um dos números que aparecem na

pilha; todos os números maiores na pilha são retirados e, para cada número retirado, é gerado um token **DEDENT**. Ao final do arquivo, um token **DEDENT** é gerado para cada número restante na pilha que seja maior que zero.

Aqui está um exemplo de um trecho de código Python indentado corretamente (embora confuso):

```
def perm(l):
        # Calcula a lista de todas as permutações de l
    if len(l) <= 1:
                  return [l]
    r = []
    for i in range(len(l)):
             s = l[:i] + l[i+1:]
             p = perm(s)
             for x in p:
              r.append(l[i:i+1] + x)
    return r
```

O exemplo a seguir mostra vários erros de indentação:

```
 def perm(l):                       # erro: primeira linha inder
for i in range(len(l)):             # erro: não indentada
    s = l[:i] + l[i+1:]
        p = perm(l[:i] + l[i+1:])   # erro: indentação inesperac
        for x in p:
                r.append(l[i:i+1] + x)
            return r                # erro: desindentação incons
```

(Na verdade, os três primeiros erros são detectados pelo analisador sintático; apenas o último erro é encontrado pelo analisador léxico — o recuo de não corresponde a um nível retirado da pilha.)

2.1.9. Espaços em branco entre tokens

Exceto no início de uma linha lógica ou em literais de string, os caracteres de espaço em branco (espaço, tabulação e avanço de página) podem ser usados indistintamente para separar tokens:

```
whitespace:  ' ' | tab | formfeed
```

O espaço em branco é necessário entre dois tokens somente se sua concatenação puder ser interpretada como um token diferente (por exemplo, `ab` é um token, mas `a b` são dois tokens). Entretanto, `+a` e `+ a` produzem dois tokens, `+` e `a`, pois `+a` não é um token válido.

2.1.10. Marcador de fim

No final da entrada não interativa, o analisador léxico gera um token **`ENDMARKER`**.

2.2. Outros tokens

Além de **`NEWLINE`**, **`INDENT`** e **`DEDENT`**, existem as seguintes categorias de tokens: *identificadores*, *palavras-chave* (**`NAME`**), *literais* (como **`NUMBER`** e **`STRING`**) e outros símbolos (*operadores* e *delimitadores*, **`OP`**). Caracteres de espaço em branco (exceto os terminadores lógicos de linha, discutidos anteriormente) não são tokens, mas servem para delimitar tokens. Onde há ambiguidade, um token compreende a string mais longa possível que forma um token legal, quando lido da esquerda para a direita.

2.3. Nomes (identificadores e palavras reservadas)

Tokens **`NAME`** representam *identificadores*, *palavras reservadas* e *palavras reservadas contextuais*.

Nomes são compostos dos seguintes caracteres:

- letras maiúsculas e minúsculas (`A-Z` e `a-z`),
- o sublinhado (_),
- dígitos (`0` a `9`), que não podem aparecer como o primeiro caractere, e
- caracteres não ASCII. Nomes válidos podem conter apenas caracteres “semelhantes a letras” e “semelhantes a dígitos”; consulte Caracteres não ASCII em nomes para obter detalhes.

Os nomes devem conter pelo menos um caractere, mas não há limite máximo de tamanho. Maiúsculas e minúsculas são relevantes.

Formalmente, nomes são descritos pelas seguintes definições léxicas:

```
NAME:           name_start name_continue*
name_start:     "a"..."z" | "A"..."Z" | "_" | <non-ASCII characte
name_continue:  name_start | "0"..."9"
identifier:     <NAME, except keywords>
```

Observe que nem todos os nomes correspondentes a esta gramática são válidos; consulte Caracteres não ASCII em nomes para obter detalhes.

2.3.1. Palavras reservadas

Os nomes a seguir são usados como palavras reservadas e não podem ser usados como identificadores comuns. Eles devem ser escritos exatamente como estão escritos aqui:

```
False      await      else       import     pass
None       break      except     in         raise
True       class      finally    is         return
and        continue   for        lambda     try
as         def        from       nonlocal   while
assert     del        global     not        with
async      elif       if         or         yield
```

2.3.2. Palavras reservadas contextuais

Adicionado na versão 3.10.

Alguns nomes são reservados apenas em contextos específicos. São conhecidos como *palavras reservadas contextuais*:

- `match`, `case` e `_`, quando usados na instrução `match`.
- `type`, quando usado na instrução `type`.
- `lazy`, quando usado antes de uma instrução `import`.

Elas agem sintaticamente como palavra reservada em seus contextos específicos, mas essa distinção é feita no nível do analisador sintático, não na tokenização.

Como palavras reservadas contextuais, seu uso na gramática é possível preservando a compatibilidade com o código existente que usa esses nomes como identificadores.

Alterado na versão 3.12: `type` é agora uma palavra reservada contextual.

Alterado na versão 3.15: `lazy` agora é uma palavra reservada contextual.

2.3.3. Classes reservadas de identificadores

Certas classes de identificadores (além de palavras reservadas) possuem significados especiais. Essas classes são identificadas pelos padrões de caracteres de sublinhado iniciais e finais:

`_*`

Não importado por `from module import *`.

`_`

Em um padrão `case` de uma instrução **`match`**, _ é uma palavra reservada contextual que denota um curinga.

Isoladamente, o interpretador interativo disponibiliza o resultado da última avaliação na variável _. (Ele é armazenado no módulo **`builtins`**, juntamente com funções embutidas como `print`.)

Em outros lugares, _ é um identificador comum. Muitas vezes é usado para nomear itens “especiais”, mas não é especial para o Python em si.

Nota

O nome _ é frequentemente usado em conjunto com internacionalização; consulte a documentação do módulo **`gettext`** para obter mais informações sobre esta convenção.

Também é comumente usado para variáveis não utilizadas.

`__*__`

Nomes definidos pelo sistema, informalmente conhecidos como nomes “dunder”. Esses nomes e suas implementações são definidos pelo interpretador (incluindo a biblioteca padrão). Os nomes de sistema atuais são discutidos na seção Nomes de métodos especiais e em outros lugares. Provavelmente mais nomes serão definidos em versões futuras do Python. *Qualquer* uso de nomes `__*__`, em qualquer contexto, que não siga o uso explicitamente documentado, está sujeito a quebra sem aviso prévio.

`__*`

Nomes de classes privadas. Os nomes nesta categoria, quando usados no contexto de uma definição de classe, são reescritos para usar uma forma desfigurada para ajudar a evitar conflitos de nomes entre atributos “privados” de classes base e derivadas. Consulte a seção Identificadores (Nomes).

2.3.4. Caracteres não ASCII em nomes

Nomes que contêm caracteres não ASCII precisam de normalização e validação adicionais além das regras e gramática explicadas acima. Por exemplo, `ř_1`, `蛇` ou □□□□ são nomes válidos, mas `r□2`, `€` ou □ não são.

Esta seção explica as exatas regras.

Todos os nomes são convertidos para a forma de normalização [https://www.unicode.org/reports/tr15/#Norm_Forms] NFKC durante a análise sintática. Isso significa que, por exemplo, algumas variantes tipográficas de caracteres são convertidas para sua forma “básica”. Por exemplo, `fiⁿᵃˡᵢᶻᵃᵗᵢᵒⁿ` é normalizado para `finalization`, então o Python os trata como o mesmo nome:

```
>>> fiⁿᵃˡᵢᶻᵃᵗᵢᵒⁿ = 3
>>> finalization
3
```

Nota

A normalização é feita apenas no nível léxico. Funções de tempo de execução que recebem nomes como *strings* geralmente não normalizam seus argumentos. Por exemplo, a variável definida acima é acessível em tempo de execução no dicionário `globals()` como `globals()["finalization"]`, mas não como `globals()["finᵃˡᶻᵃᵗᵢᵒⁿ"]`.

Assim como os nomes que usam apenas ASCII devem conter somente letras, dígitos e o sublinhado, e não podem começar com um dígito, um nome válido deve começar com um caractere do conjunto "semelhante a letras" `xid_start`, e os caracteres restantes devem estar no conjunto "semelhante a letras e dígitos" `xid_continue`.

Esses conjuntos são baseados nos conjuntos *XID_Start* e *XID_Continue*, conforme definidos pelo anexo UAX-31 [https://www.unicode.org/reports/tr31/] do padrão Unicode. O `xid_start` do Python inclui adicionalmente o sublinhado (_). Observe que o Python não está necessariamente em conformidade com o UAX-31 [https://www.unicode.org/reports/tr31/].

Uma lista não normativa de caracteres nos conjuntos *XID_Start* e *XID_Continue*, conforme definidos pelo Unicode, está disponível no arquivo DerivedCoreProperties.txt [https://www.unicode.org/Public/17.0.0/ucd/DerivedCoreProperties.txt] no Banco de Dados de Caracteres Unicode. Para referência, as regras de construção para os conjuntos `xid_*` são apresentadas abaixo.

O conjunto `id_start` é definido como a união de:

- Categoria Unicode `<Lu>` - letras maiúsculas (inclui `A` a `Z`)
- Categoria Unicode `<Ll>` - letras minúsculas (inclui `a` a `z`)
- Categoria Unicode `<Lt>` - letras em titlecase (coloca em maiúsculo a primeira letra de cada palavra principal)
- Categoria Unicode `<Lm>` - letras modificadoras
- Categoria Unicode `<Lo>` - outras letras
- Categoria Unicode `<Nl>` - números de letras

- {"_"} - o underscore (sublinhado)
- `<Other_ID_Start>` - um conjunto explícito de caracteres em PropList.txt [https://www.unicode.org/Public/17.0.0/ucd/PropList.txt] para oferecer suporte à retrocompatibilidade

O conjunto `xid_start` fecha então este conjunto sob a normalização NFKC, removendo todos os caracteres cuja normalização não é da forma `id_start id_continue*`.

O conjunto `id_continue` é definido como a união de:

- `id_start` (veja acima)
- Categoria Unicode `<Nd>` - números decimais (inclui `0` a `9`)
- Categoria Unicode `<Pc>` - pontuações de conectores
- Categoria Unicode `<Mn>` - marcas sem espaçamento
- Categoria Unicode `<Mc>` - espaçamento combinando marcas
- `<Other_ID_Continue>` - outro conjunto explícito de caracteres em PropList.txt [https://www.unicode.org/Public/17.0.0/ucd/PropList.txt] para oferecer suporte à retrocompatibilidade

Novamente, `xid_continue` fecha este conjunto sob a normalização NFKC.

As categorias Unicode usam a versão do Unicode Character Database (em português, banco de dados de caracteres Unicode) incluída no módulo **`unicodedata`**.

Ver também

- **PEP 3131** [https://peps.python.org/pep-3131/] – Suporte a identificadores não-ASCII
- **PEP 672** [https://peps.python.org/pep-0672/] – Considerações de segurança relacionadas ao Unicode para Python

2.4. Literais

Literais são notações para valores constantes de alguns tipos embutidos.

Em termos de análise lexical, Python tem literais de string, de bytes e numéricos.

Outros “literais” são denotados lexicamente usando palavra reservada (`None`, `True`, `False`) e o token de reticências especial (`...`).

2.5. Literais de string e bytes

Literais de string são textos entre aspas simples (`'`) ou aspas duplas (`"`). Por exemplo:

```
"spam"
'eggs'
```

A aspa usada para iniciar o literal também o encerra, portanto, um literal de string só pode conter a outra aspa (exceto com sequências de escape, veja abaixo). Por exemplo:

```
'Diga "Olá", por favor.'
"Não faça isso!"
```

Exceto por essa limitação, a escolha do caractere de aspas (`'` ou `"`) não afeta como o literal é analisado.

Dentro de uma literal de string, o caractere de contrabarra (\) introduz uma sequência de escape , *que tem um significado especial dependendo do caractere após a contrabarra. Por exemplo,* `"`` denota o caractere de aspas duplas e *não* encerra a string:

```
>>> print("Diga \"Olá\" para todo mundo!")
Diga "Olá" para todo mundo!
```

Veja sequências de escape abaixo para uma lista completa dessas sequências e mais detalhes.

2.5.1. Strings entre aspas triplas

Strings também podem ser colocadas emgrupos correspondentes de três aspas simples ou duplas. Geralmente, elas são chamadas de *strings entre aspas triplas*:

```
"""Isso é uma string entre aspas triplas."""
```

Em literais com aspas triplas, aspas sem escape são permitidas (e são mantidas), exceto que três aspas sem escape seguidas encerram o literal, se forem do mesmo tipo (`'` ou `"`) usadas no início:

```
"""Essa string tem "aspas" dentro."""
```

Novas linhas sem escape também são permitidas e mantidas:

```
'''Essa string entre aspas triplas
continua na linha seguinte.'''
```

2.5.2. Prefixos de string

Literais de string podem ter um *prefixo* opcional que influencia como o conteúdo do literal é analisado, por exemplo:

```
b"data"
f'{result=}'
```

Os prefixos permitidos são:

- `b`: Literal de bytes
- `r`: String bruta
- `f`: Literal de string formatada (“f-string”)
- `t`: Literal de string template (“t-string”)
- `u`: Sem efeito (permitido para retrocompatibilidade)

Veja as seções vinculadas para obter detalhes sobre cada tipo.

Prefixos não diferenciam maiúsculas de minúsculas (por exemplo, ‘`B`’ funciona da mesma forma que ‘`b`’). O prefixo ‘`r`’ pode ser combinado com ‘`f`’, ‘`t`’ ou ‘`b`’, portanto, ‘`fr`’, ‘`rf`’, ‘`tr`’, ‘`rt`’, ‘`br`’ e ‘`rb`’ também são prefixos válidos.

Adicionado na versão 3.3: O prefixo `'rb'` de literais de bytes brutos foi adicionado como sinônimo de `'br'`.

O suporte para o literal legado unicode (`u'value'`) foi reintroduzido para simplificar a manutenção de bases de código duplas Python 2.x e 3.x. Consulte **PEP 414** [https://peps.python.org/pep-0414/] para obter mais informações.

2.5.3. Gramática formal

Literais de string, exceto "f-strings" e "t-strings", são descritos pelas seguintes definições lexicais.

Essas definições usam negative lookaheads (`!`) para indicar que uma aspa final encerra o literal.

```
STRING:          [stringprefix] (stringcontent)
stringprefix:    <("r" | "u" | "b" | "br" | "rb"), case-insensit
stringcontent:
   | '''''' ( !''''''' longstringitem)* ''''''
   | """""" ( !"""""" longstringitem)* """""
   | '''' ( !'''' stringitem)* ''''
   | '''' ( !'''' stringitem)* ''''
stringitem:      stringchar | stringescapeseq
stringchar:      <any source character, except backslash and new
longstringitem:  stringitem | newline
stringescapeseq: "\" <any source character>
```

Observe que, como em todas as definições lexicais, os espaços em branco são significativos. Em particular, o prefixo (se houver) deve ser imediatamente seguido pela aspa inicial.

2.5.4. Sequências de escape

A menos que um prefixo '`r`' ou '`R`' esteja presente, as sequências de escape em literais de string e bytes são interpretadas de acordo com regras semelhantes àquelas usadas pelo Standard C. As sequências de escape reconhecidas são:

Sequência de escape	Significado
`\`<newline>	Fim de linha ignorado
`\\`	Contrabarra
`\'`	Aspa simples
`\"`	Aspas duplas
`\a`	ASCII Bell (BEL) - um sinal audível é emitido
`\b`	ASCII Backspace (BS) - apaga caractere à esquerda
`\f`	ASCII Formfeed (FF) - quebra de página
`\n`	ASCII Linefeed (LF) - quebra de linha
`\r`	ASCII Carriage Return (CR) - retorno de carro
`\t`	ASCII Horizontal Tab (TAB) - tabulação horizontal
`\v`	ASCII Vertical Tab (VT) - tabulação vertical

Sequência de escape	Significado
`\ooo`	Caractere octal
`\xhh`	Caractere hexadecimal
`\N{name}`	Caractere nomeado Unicode
`\uxxxx`	Caractere hexadecimal Unicode
`\Uxxxxxxxx`	Caractere hexadecimal Unicode

2.5.4.1. Fim de linha ignorado

Uma contrabarra pode ser adicionada ao fim da linha para ignorar a nova linha:

```
>>> 'Esta string não vai incluir \
... contrabarras e caracteres de nova linha.'
'Esta string não vai incluir contrabarras e caracteres de nova l
```

O mesmo resultado pode ser obtido usando strings com aspas triplas, ou parênteses e concatenação de literal string.

2.5.4.2. Caracteres escapados

Para incluir uma contrabarra em um literal de string não-bruta Python, ela deve ser duplicada. A sequência de escape `\\` denota um único caractere de contrabarra:

```
>>> print('C:\\Program Files')
C:\Program Files
```

Da mesma forma, as sequências `\'` e `\"` denotam os caracteres de aspas simples e duplas, respectivamente:

```
>>> print('\' e \"')
' e "
```

2.5.4.3. Caractere octal

A sequência `\`*ooo* denota um *caractere* com o valor octal (base 8) *ooo*:

```
>>> '\120'
'P'
```

São aceitos até três dígitos octais (0 a 7).

Em um literal de bytes, *caractere* significa um *byte* com o valor fornecido. Em um literal de string, significa um caractere Unicode com o valor fornecido.

Alterado na versão 3.11: Escapes octais com valor maior que `0o377` (255) produz uma **`DeprecationWarning`**.

Alterado na versão 3.12: Escapes octais com valor maior que `0o377` (255) produzem um **`SyntaxWarning`**. Em uma versão futura do Python eles eventualmente vão levantar um **`SyntaxError`**.

2.5.4.4. Caractere hexadecimal

A sequência `\x`*hh* denota um *caractere* com o valor hexa (base 16) *hh*:

```
>>> '\x50'
'P'
```

Ao contrário do padrão C, são necessários exatamente dois dígitos hexadecimais.

Em um literal de bytes, *caractere* significa um *byte* com o valor fornecido. Em um literal de string, significa um caractere Unicode com o valor fornecido.

2.5.4.5. Caractere nomeado Unicode

A sequência `\N{`*`nome`*`}` denota um caractere Unicode com o *nome* fornecido:

```
>>> '\N{LATIN CAPITAL LETTER P}'
'P'
>>> '\N{SNAKE}'
'□'
```

Esta sequência não pode aparecer em literais de bytes.

Alterado na versão 3.3: Foi adicionado suporte a *apelidos para nomes <https://www.unicode.org/Public/17.0.0/ucd/NameAliases.txt>`__*.

2.5.4.6. Caracteres hexadecimais Unicode

As sequências `\u`*`xxxx`* e `\U`*`xxxxxxxx`* denotam o caractere Unicode com o valor hexadecimal fornecido (base 16). São necessários exatamente quatro dígitos para `\u`; são necessários exatamente oito dígitos para `\U`. Este último pode codificar qualquer caractere Unicode.

```
>>> '\u1234'
'□'
>>> '\U0001f40d'
'□'
```

Estas sequências não pode aparecer em literais de bytes.

2.5.4.7. Sequências de espace não reconhecidas

Diferentemente do Standard C, todas as sequências de escape não reconhecidas são deixadas na string inalteradas, ou seja, *a contrabarra é deixada no resultado*:

```
>>> print('\q')
\q
>>> list('\q')
['\\', 'q']
```

Observe que, para literais de bytes, as sequências de escape reconhecidas apenas em literais de string (`\N...`, `\u...`, `\U...`) se enquadram na categoria de escapes não reconhecidos.

Alterado na versão 3.6: Sequências de escape não reconhecidas produzem um **`DeprecationWarning`**.

Alterado na versão 3.12: Sequências de escape não reconhecidas produzem um **`SyntaxWarning`**. Em uma versão futura do Python eles vão levantar um **`SyntaxError`**.

2.5.5. Literais de bytes

Literais de bytes são sempre prefixados com '`b`' ou '`B`'; eles produzem uma instância do tipo **`bytes`** em vez do tipo **`str`**. Eles podem conter apenas caracteres ASCII; bytes com valor numérico igual ou superior a 128 devem ser expressos com sequências de escape (normalmente Caractere hexadecimal ou Caractere octal):

```
>>> b'\x89PNG\r\n\x1a\n'
b'\x89PNG\r\n\x1a\n'
>>> list(b'\x89PNG\r\n\x1a\n')
[137, 80, 78, 71, 13, 10, 26, 10]
```

Da mesma forma, um byte zero deve ser expresso usando uma sequência de escape (normalmente `\0` ou `\x00`).

2.5.6. Literais de string brutas

Literais de string e bytes podem opcionalmente ser prefixados com uma letra '`r`' ou '`R`'; tais construções são chamadas de *literais de strings brutas* e *literais de bytes brutos* tratam as contrabarras como caracteres literais. Como resultado, em literais de string bruta, as sequências de escape não são tratados de maneira especial.

```
>>> r'\d{4}-\d{2}-\d{2}'
'\\d{4}-\\d{2}-\\d{2}'
```

Mesmo em um literal bruto, as aspas podem ser escapadas com uma contrabarra, mas a barra invertida permanece no resultado; por exemplo, `r"\""` é uma literal de string válida que consiste em dois caracteres: uma contrabarra e aspas duplas; `r"\"` não é uma literal de string válida (mesmo uma string bruta não pode terminar em um número ímpar de contrabarras). Especificamente, *um literal bruto não pode terminar em uma única contrabarra* (já que a contrabarra escaparia do seguinte caractere de aspas). Observe também que uma única contrabarra seguida por uma nova linha é interpretada como esses dois caracteres como parte do literal, *não* como uma continuação de linha.

2.5.7. Literais de strings formatadas

Adicionado na versão 3.6.

Alterado na versão 3.7: **`await`** e **`async for`** podem ser usados em expressões dentro de f-strings.

Alterado na versão 3.8: Adicionado o especificador de depuração (=)

Alterado na versão 3.12: Muitas restrições a expressões dentro de f-strings foram removidas. Notavelmente, strings aninhadas, comentários e contrabarras agora são permitidos.

Um *literal de string formatada* ou uma *f-string* é uma string literal que começa com ‘`f`’ ou ‘`F`’. Ao contrário de outras strings literais, as f-strings não têm um valor constante. Elas podem conter *campos de substituição* delimitados por chaves `{}`. Os campos de substituição contêm expressões que são avaliadas em tempo de execução. Por exemplo:

```
>>> who = 'Ninguém'
>>> nationality = 'Espanhola'
>>> f'{who.title()} está à espera da Inquisição {nationality}!'
'Ninguém está à espera da Inquisição Espanhola!'
```

Quaisquer chaves duplas (`{{` ou `}}`) fora dos campos de substituição são substituídas pela chave simples correspondente:

```
>>> print(f'{{...}}')
{...}
```

Outros caracteres fora dos campos de substituição são tratados como em literais de string comuns. Isso significa que sequências de escape são decodificadas (exceto quando um literal também é marcado como uma string bruta) e quebras de linha são possíveis em f-strings entre aspas triplas.

```
>>> nome = 'João'
>>> cor_favorita = 'azul'
>>> print(f'{nome}:\t{cor_favorita}')
João: azul
>>> print(rf"C:\Users\{nome}")
C:\Users\João
>>> print(f'''Três deve ser o número que deves contar
... e o número da contagem deve ser três.''')
Três deve ser o número que deves contar
... e o número da contagem deve ser três.
```

Expressões em literais de strings formatadas são tratadas como expressões Python regulares. Cada expressão é avaliada no contexto em que a literal de string formatada aparece, da esquerda para a direita. Uma expressão vazia não é permitida, e tanto **`lambda`** como expressões `:=` de atribuição devem ser cercadas por parênteses explícitos:

```
>>> f'{(metade := 1/2)}, {metade * 42}'
'0.5, 21.0'
```

É permitido reutilizar o tipo de aspas externas da f-string dentro de um campo de substituição:

```
>>> a = dict(x=2)
>>> f"abc {a["x"]} def"
'abc 2 def'
```

Contrabarras também são permitidas em campos de substituição e são avaliadas da mesma forma que em qualquer outro contexto:

```
>>> a = ["a", "b", "c"]
>>> print(f"A lista a contém:\n{"\n".join(a)}")
A lista a contém:
a
b
c
```

É possível aninhar f-strings:

```
>>> nome = 'mundo'
>>> f'Repetido:{f' olá {nome}' * 3}'
'Repetido: olá mundo olá mundo olá mundo'
```

Programas Python portáteis não devem utilizar mais de 5 níveis de aninhamento.

CPython não possui limite para aninhamento de f-strings.

Expressões de substituição podem conter novas linhas tanto em f-strings com aspas simples quanto nas de aspas triplas, e podem conter comentários. Tudo o que vier após um `#` dentro de um campo de substituição é um comentário (até mesmo chaves de fechamento e aspas). Isso significa que campos de substituição com comentários devem ser fechados em uma linha diferente:

```
>>> a = 2
>>> f"abc{a # Este comentário }" continua até o fim da linha
...      + 3}"
'abc5'
```

Depois da expressão, campos de substituição podem opcionalmente conter:

- um *especificador de depuração* – um sinal de igual (`=`), opcionalmente cercado por espaços em branco em um ou ambos os lados;
- um *especificador de conversão* – `!s`, `!r` ou `!a`; e/ou
- um *especificador de formatação* prefixado com um caractere de dois pontos (`:`).

Consulte o Módulo de f-strings na biblioteca padrão para detalhes sobre como esses campos são avaliados.

Como o módulo explica, *especificadores de formato* são passados como o segundo argumento para a função **`format()`** formatar um valor do campo de substituição. Por exemplo, podem ser usados para especificar a largura de campo e caracteres de preenchimento usando a Minilinguagem de especificação de formato:

```
>>> numero = 14.3
>>> f'{numero:20.7f}'
'           14.3000000'
```

Especificadores de formato de nível superior podem incluir campos de substituição aninhados:

```
>>> tamanho_campo = 20
>>> precisao = 7
>>> f'{numero:{tamanho_campo}.{precisao}f}'
'           14.3000000'
```

Esses campos aninhados podem incluir seus próprios especificadores de conversão campo e especificador de formato

```
>>> numero = 3
>>> f'{numero:{tamanho_campo}}'
'                    3'
>>> f'{numero:{tamanho_campo:05}}'
'00000000000000000003'
```

No entanto, esses campos aninhados não podem incluir campos de substituição mais profundamente aninhados.

Literais de strings formatadas não podem ser usadas como docstrings , mesmo que não incluam expressões:

```
>>> def foo():
... f "Não é um docstring"
...
```

```
>>> print(foo.__doc__)
None
```

Ver também

- **PEP 498** [https://peps.python.org/pep-0498/] – Interpolação de strings literais
- **PEP 701** [https://peps.python.org/pep-0701/] – Formalização sintática de f-strings
- `str.format()`, que usa um mecanismo de string de formato relacionado.

2.5.8. Literais de string template (t-strings)

Adicionado na versão 3.14.

Um *literal de string template* ou *t-string* é um literal de string que é prefixado com '`t`' ou '`T`'. Essas strings seguem as mesmas regras de sintaxe que literais de strings formatadas . Para conhecer as diferenças nas regras de avaliação, consulte o módulo biblioteca padrão em t-strings

2.5.9. Gramática formal para f-strings

F-strings são tratados parcialmente pelo analisador léxico, que produz os tokens `FSTRING_START` , `FSTRING_MIDDLE` e `FSTRING_END`, e parcialmente pelo analisador sintático, que manipula expressões nos campos de substituição. A maneira exata de como trabalho é dividido é um detalhe de implementação do CPython.

Da mesma forma, a gramática f-string é uma mistura de definições léxicas e sintáticas.

Espaços em branco são significativos nessas situações:

- Não pode haver espaços em branco em `FSTRING_START` (entre o prefixo e a aspas).

- Espaços em branco em **FSTRING_MIDDLE** fazem parte do conteúdo literal da string.

- No `fstring_replacement_field`, se `f_debug_specifier` estiver presente, todo espaço em branco após a chave de abertura até o `f_debug_specifier`, bem como o espaço em branco imediatamente após o `f_debug_specifier`, é preservado como parte da expressão.

  A expressão não é tratada na fase de tokenização; ela é recuperada do código-fonte usando as localizações do token `{` token e do token após o `=`.

A definição de `FSTRING_MIDDLE` usa lookaheads negativos (`!`) para indicar caracteres especiais (contrabarra, novas linhas, `{`, `}`) e sequências (`f_quote`).

```
fstring:    FSTRING_START fstring_middle* FSTRING_END

FSTRING_START:      fstringprefix ("'" | '"' | "'''" | '"""')
FSTRING_END:        f_quote
fstringprefix:      <("f" | "fr" | "rf"), case-insensitive>
f_debug_specifier:  '='
f_quote:            <the quote character(s) used in FSTRING_STAR

fstring_middle:
   | fstring_replacement_field
   | FSTRING_MIDDLE
FSTRING_MIDDLE:
   | (!"\" !newline !'{' !'}' !f_quote) source_character
   | stringescapeseq
   | "{{"
   | "}}"
   | <newline, in triple-quoted f-strings only>
fstring_replacement_field:
   | '{' f_expression [f_debug_specifier] [fstring_conversion]
         [fstring_full_format_spec] '}'
fstring_conversion:
   | "!" ("s" | "r" | "a")
fstring_full_format_spec:
   | ':' fstring_format_spec*
```

```
fstring_format_spec:
   | FSTRING_MIDDLE
   | fstring_replacement_field
f_expression:
   | ',' .(conditional_expression | "*" or_expr)+ [","]
   | yield_expression
```

Nota

No trecho de gramática acima, as regras `f_quote` e `FSTRING_MIDDLE` são sensíveis ao contexto – elas dependem do conteúdo de `FSTRING_START` da `fstring` que envolve mais próxima.

A construção de uma gramática formal mais tradicional a partir deste modelo fica como exercício para o leitor.

A gramática para t-strings é idêntica à gramática para f-strings, com *t* em vez de *f* no início dos nomes de regras e tokens e no prefixo.

```
tstring:    TSTRING_START tstring_middle* TSTRING_END

<rest of the t-string grammar is omitted; see above>
```

2.6. Literais numéricos

Tokens **NUMBER** representam literais numéricos, dos quais existem três tipos: inteiros, números de ponto flutuante e números imaginários.

```
NUMBER: integer | floatnumber | imagnumber
```

O valor numérico de um literal numérico é o mesmo que seria se fosse passado como uma string para os construtores de classe **int**, **float** ou **complex**, respectivamente. Observe que nem todas as entradas válidas para esses construtores também são literais válidos.

Literais numéricos não incluem um sinal; uma frase como `-1` é, na verdade, uma expressão composta pelo operador unário '`-`' e o literal `1`.

2.6.1. Inteiros literais

Literais inteiros denotam números inteiros. Por exemplo:

```
7
3
2147483647
```

Não há limite para o comprimento de literais inteiros, exceto o que pode ser armazenado na memória disponível:

```
79228162514264337593543950336792281625142643375935439503367922816251426433759354395033 6
```

Sublinhados podem ser usados para agrupar dígitos para melhor legibilidade e são ignorados para determinar o valor numérico do literal. Por exemplo, os seguintes literais são equivalentes:

```
100_000_000_000
100000000000
1_00_00_00_00_000
```

Sublinhados só podem ocorrer entre dígitos. Por exemplo, `_123`, `321_` e `123__321` *não* são literais válidos.

Os números inteiros podem ser especificados em binário (base 2), octal (base 8) ou hexadecimal (base 16) usando os prefixos `0b`, `0o` e `0x`, respectivamente. Os dígitos hexadecimais de 10 a 15 são representados pelas letras `A-F`, sem distinção entre maiúsculas e minúsculas. Por exemplo:

```
0b100110111
0b_1110_0101
0o177
0o377
0xdeadbeef
0xDead_Beef
```

Um sublinhado pode seguir o especificador base. Por exemplo, `0x_1f` é um literal válido, mas `0_x1f` e `0x__1f` não são.

Zeros à esquerda em um número decimal diferente de zero não são permitidos. Por exemplo, `0123` não é um literal válido. Isso serve para desambiguação com literais octais no estilo C, que o Python usava antes da versão 3.0.

Formalmente, literais inteiros são descritos pelas seguintes definições lexicais:

```
integer:      decinteger | bininteger | octinteger | hexinteger
decinteger:   nonzerodigit (["_"] digit)*
bininteger:   "0" ("b" | "B") (["_"] bindigit)+
octinteger:   "0" ("o" | "O") (["_"] octdigit)+
hexinteger:   "0" ("x" | "X") (["_"] hexdigit)+
zerointeger:  "0"+ (["_"] "0")*
nonzerodigit: "1"..."9"
digit:        "0"..."9"
bindigit:     "0" | "1"
octdigit:     "0"..."7"
hexdigit:     digit | "a"..."f" | "A"..."F"
```

Alterado na versão 3.6: Os sublinhados agora são permitidos para fins de agrupamento de literais.

2.6.2. Literais de ponto flutuante

Literais de ponto flutuante (float), como `3.14` ou `1.5`, denotam aproximações de números reais.

Eles consistem em partes *inteiras* e *fracionárias*, cada uma composta por dígitos decimais. As partes são separadas por um ponto decimal, `.`:

```
2.71828
4.0
```

Ao contrário dos literais inteiros, zeros à esquerda são permitidos nas partes numéricas. Por exemplo, `077.010` é válido e denota o mesmo número que `77.01`.

Assim como em literais inteiros, sublinhados simples podem ocorrer entre dígitos para ajudar na legibilidade:

```
96_485.332_123
3.14_15_93
```

Qualquer uma dessas partes, mas não ambas, pode estar vazia. Por exemplo:

```
10.  # (equivalente a 10.0)
.001  # (equivalente a 0.001)
```

Opcionalmente, o inteiro e a fração podem ser seguidos por um *expoente*: a letra `e` ou `E`, seguida por um sinal opcional, `+` ou `-`, e um número no mesmo formato das partes inteira e fracionária. O `e` ou `E` representa "dez vezes elevado à potência de":

```
1.0e3  # (representa 1.0×10³ ou 1000.0)
1.166e-5  # (representa 1.166×10⁻⁵ ou 0.00001166)
6.02214076e+23  # (representa 6.02214076×10²³ ou 602214076000000
```

Em floats com apenas partes inteiras e expoentes, o ponto decimal pode ser omitido:

```
1e3  # (equivalente a 1.e3 e 1.0e3)
0e0  # (equivalente a 0.)
```

Formalmente, literais de ponto flutuante são descritos pelas seguintes definições léxicas:

```
floatnumber:
   | digitpart "." [digitpart] [exponent]
   | "." digitpart [exponent]
   | digitpart exponent
```

```
digitpart: digit (["_"] digit)*
exponent:  ("e" | "E") ["+" | "-"] digitpart
```

Alterado na versão 3.6: Os sublinhados agora são permitidos para fins de agrupamento de literais.

2.6.3. Literais imaginários

Python possui objetos número complexo, mas não possui literais complexos. Em vez disso, *literais imaginários* denotam números complexos com parte real zero.

Por exemplo, na matemática, o número complexo 3+4.2*i* é escrito como o número real 3 somado ao número imaginário 4.2*i*. Python usa uma sintaxe semelhante, exceto que a unidade imaginária é escrita como `j` em vez de *i*:

```
3+4.2j
```

Esta é uma expressão composta pelo literal inteiro `3`, o operador '`+`' e o literal imaginário `4.2j`. Como esses são três tokens separados, espaços em branco são permitidos entre eles:

```
3 + 4.2j
```

Nenhum espaço em branco é permitido *dentro* de cada token. Em particular, o sufixo `j` não pode ser separado do número que o precede.

O número antes do `j` tem a mesma sintaxe de um literal de ponto flutuante. Portanto, os seguintes são literais imaginários válidos:

```
4.2j
3.14j
10.j
.001j
1e100j
3.14e-10j
3.14_15_93j
```

Ao contrário de um literal de ponto flutuante, o ponto decimal pode ser omitido se o número imaginário tiver apenas uma parte inteira. O número

ainda é avaliado como um número de ponto flutuante, não um inteiro.

```
10j
0j
1000000000000000000000000j   # equivalente a 1e+24j
```

O sufixo `j` não diferencia maiúsculas de minúsculas. Isso significa que você pode usar `J` em vez disso:

```
3.14J   # equivalente a 3.14j
```

Formalmente, literais imaginários são descritos pela seguinte definição léxica:

```
imagnumber: (floatnumber | digitpart) ("j" | "J")
```

2.7. Operadores e delimitadores

A seguinte gramática define os tokens de *operador* e de *delimitador*, ou seja, o tipo de token genérico **OP**. Uma lista destes tokens e seus nomes também está disponível na documentação do módulo **token**.

```
OP:
   | assignment_operator
   | bitwise_operator
   | comparison_operator
   | enclosing_delimiter
   | other_delimiter
   | arithmetic_operator
   | "..."
   | other_op

assignment_operator:   "+=" | "-=" | "*=" | "**=" | "/="  | "//=
                       "&=" | "|=" | "^=" | "<<=" | ">>=" | "@='
bitwise_operator:      "&"  | "|"  | "^"  | "~"   | "<<"  | ">>'
comparison_operator:   "<=" | ">=" | "<"  | ">"   | "==" | "!='
enclosing_delimiter:   "("  | ")"  | "["  | "]"   | "{"  | "}"
other_delimiter:       ","  | ":"  | "!"  | ";"   | "="  | "->'
```

```
arithmetic_operator:   "+"  | "-"  | "**" | "*"   | "//"  | "/"
other_op:              "."  | "@"
```

Nota

Geralmente, *operadores* são usados para combinar expressões, enquanto que os *delimitadores* servem para outras coisas. Entretanto, não há distinção formal clara entre essas duas categorias.

Alguns tokens podem servir tanto como operadores quanto delimitadores, dependendo da forma de uso. Por exemplo, `*` é tanto o operador de multiplicação quanto o delimitador usado para desempacotamento de sequências, e `@` é ao mesmo tempo a multiplicação de matrizes e o delimitador que introduz decoradores.

Para alguns tokens, a distinção não é clara. Por exemplo, alguns consideram `.`, `(` e `)` como sendo delimitadores, enquanto outros enxergam aí o operador **`getattr()`** e o(s) operador(es) de chamada de função.

Alguns operadores do Python, como `and`, `or` e `not in`, usam tokens de palavra reservada ao invés de “símbolos” (tokens de operador).

Uma sequência de três pontos consecutivos (`...`) tem o significado especial de ser um literal de **`Ellipsis`**.

3. Modelo de dados

3.1. Objetos, valores e tipos

Objetos são abstrações do Python para dados. Todos os dados em um programa Python são representados por objetos ou por relações entre objetos. Código também é representado por objetos.

Todo objeto tem uma identidade, um tipo e um valor. A *identidade* de um objeto nunca muda depois de criado; você pode pensar nisso como endereço de objetos em memória. O operador `is` compara as identidades de dois objetos; a função `id()` retorna um inteiro representando sua identidade.

Para CPython, `id(x)` é o endereço de memória em que `x` está armazenado.

O tipo de um objeto determina as operações que o objeto implementa (por exemplo, “ele tem um comprimento?”) e também define os valores possíveis para objetos desse tipo. A função `type()` retorna o tipo de um objeto (que é também um objeto). Como sua identidade, o *tipo* do objeto também é imutável. [[1]]

O *valor* de alguns objetos pode mudar. Objetos cujos valores podem mudar são descritos como *mutáveis*, objetos cujo valor não pode ser mudado uma vez que foram criados são chamados *imutáveis*. (O valor de um objeto contêiner imutável que contém uma referência a um objeto mutável pode mudar quando o valor deste último for mudado; no entanto o contêiner é ainda assim considerada imutável, pois a coleção de objetos que contém não pode ser mudada. Então a imutabilidade não é estritamente o mesmo do que não haver mudanças de valor, é mais sutil.) A mutabilidade de um objeto é determinada pelo seu tipo; por exemplo, números, strings e tuplas são imutáveis, enquanto dicionários e listas são mutáveis.

Os objetos nunca são destruídos explicitamente; no entanto, quando eles se tornam inacessíveis, eles podem ser coletados como lixo. Uma implementação tem permissão para adiar a coleta de lixo ou omiti-la

completamente – é uma questão de detalhe de implementação como a coleta de lixo é implementada, desde que nenhum objeto que ainda esteja acessível seja coletado.

CPython atualmente usa um esquema de contagem de referências com detecção atrasada (opcional) de lixo ligado ciclicamente, que coleta a maioria dos objetos assim que eles se tornam inacessíveis, mas não é garantido que coletará lixo contendo referências circulares. Veja a documentação do módulo `gc` para informações sobre como controlar a coleta de lixo cíclico. Outras implementações agem de forma diferente e o CPython pode mudar. Não dependa da finalização imediata dos objetos quando eles se tornarem inacessíveis (isto é, você deve sempre fechar os arquivos explicitamente).

Observe que o uso dos recursos de rastreamento ou depuração da implementação pode manter os objetos ativos que normalmente seriam coletáveis. Observe também que capturar uma exceção com uma instrução `try`…`except` pode manter os objetos vivos.

Alguns objetos contêm referências a recursos "externos", como arquivos abertos ou janelas. Entende-se que esses recursos são liberados quando o objeto é coletado como lixo, mas como a coleta de lixo não é garantida, tais objetos também fornecem uma maneira explícita de liberar o recurso externo, geralmente um método **`close()`**. Os programas são fortemente recomendados para fechar explicitamente esses objetos. A instrução `try`… `finally` e a instrução `with` fornecem maneiras convenientes de fazer isso.

Alguns objetos contêm referências a outros objetos; eles são chamados de *contêineres*. Exemplos de contêineres são tuplas, listas e dicionários. As referências fazem parte do valor de um contêiner. Na maioria dos casos, quando falamos sobre o valor de um contêiner, nos referimos aos valores, não às identidades dos objetos contidos; entretanto, quando falamos sobre a mutabilidade de um contêiner, apenas as identidades dos objetos contidos imediatamente estão implícitas. Portanto, se um contêiner imutável (como uma tupla) contém uma referência a um objeto mutável, seu valor muda se esse objeto mutável for alterado.

Os tipos afetam quase todos os aspectos do comportamento do objeto. Até mesmo a importância da identidade do objeto é afetada em algum sentido: para tipos imutáveis, as operações que calculam novos valores podem realmente retornar uma referência a qualquer objeto existente com o mesmo tipo e valor, enquanto para objetos mutáveis isso não é permitido. Por exemplo, após `a = 1; b = 1`, *a* e *b* podem ou não se referir ao mesmo objeto com o valor um, dependendo da implementação. Isto ocorre porque `int` é um tipo imutável, então a referência a `1` pode ser reutilizada. Este comportamento depende da implementação usada, então não deve ser considerada confiável, mas é algo para se estar ciente ao fazer uso de testes de identidade de objeto. No entanto, após `c = []; d = []`, *c* e *d* têm a garantia de referir-se a duas listas vazias diferentes e únicas. (Observe que `e = f = []` atribui o *mesmo* objeto para *e* e *f*.)

3.2. A hierarquia de tipos padrão

Abaixo está uma lista dos tipos que são embutidos no Python. Módulos de extensão (escritos em C, Java ou outras linguagens, dependendo da implementação) podem definir tipos adicionais. Versões futuras do Python podem adicionar tipos à hierarquia de tipo (por exemplo, números racionais, matrizes de inteiros armazenadas de forma eficiente, etc.), embora tais adições sejam frequentemente fornecidas por meio da biblioteca padrão.

Algumas das descrições de tipo abaixo contêm um parágrafo listando “atributos especiais”. Esses são atributos que fornecem acesso à implementação e não se destinam ao uso geral. Sua definição pode mudar no futuro.

3.2.1. None

Este tipo possui um único valor. Existe um único objeto com este valor. Este objeto é acessado através do nome embutido `None`. É usado para significar a ausência de um valor em muitas situações, por exemplo, ele é retornado de funções que não retornam nada explicitamente. Seu valor verdade é falso.

3.2.2. NotImplemented

Este tipo possui um único valor. Existe um único objeto com este valor. Este objeto é acessado através do nome embutido `NotImplemented`. Os métodos numéricos e métodos de comparação rica devem retornar esse valor se não implementarem a operação para os operandos fornecidos. (O interpretador tentará então a operação refletida ou alguma outra alternativa, dependendo do operador.) Não deve ser avaliado em um contexto booleano.

Veja a documentação Implementando as operações aritméticas para mais detalhes.

Alterado na versão 3.9: A avaliação de `NotImplemented` em um contexto booleano foi descontinuada.

Alterado na versão 3.14: A avaliação de `NotImplemented` em um contexto booleano agora levanta um `TypeError`. Anteriormente, ele era avaliado como `True` e emitia um `DeprecationWarning` desde o Python 3.9.

3.2.3. Ellipsis

Este tipo possui um único valor. Existe um único objeto com este valor. Este objeto é acessado através do literal `...` ou do nome embutido `Ellipsis` (reticências). Seu valor verdade é verdadeiro.

3.2.4. `numbers.Number`

Esses são criados por literais numéricos e retornados como resultados por operadores aritméticos e funções aritméticas embutidas. Os objetos numéricos são imutáveis; uma vez criado, seu valor nunca muda. Os números do Python são, obviamente, fortemente relacionados aos números matemáticos, mas sujeitos às limitações da representação numérica em computadores.

As representações de string das classes numéricas, calculadas por `__repr__()` e `__str__()`, têm as seguintes propriedades:

- Elas são literais numéricos válidos que, quando passados para seu construtor de classe, produzem um objeto com o valor do numérico original.
- A representação está na base 10, quando possível.
- Os zeros à esquerda, possivelmente com exceção de um único zero antes de um ponto decimal, não são mostrados.
- Os zeros à direita, possivelmente com exceção de um único zero após um ponto decimal, não são mostrados.
- Um sinal é mostrado apenas quando o número é negativo.

Python distingue entre inteiros, números de ponto flutuante e números complexos:

3.2.4.1. `numbers.Integral`

Estes representam elementos do conjunto matemático de inteiros (positivos e negativos).

Nota

As regras para representação de inteiros têm como objetivo fornecer a interpretação mais significativa das operações de deslocamento e máscara envolvendo inteiros negativos.

Existem dois tipos de inteiros:

Inteiros (`int`)

Estes representam números em um intervalo ilimitado, sujeito apenas à memória (virtual) disponível. Para o propósito de operações de deslocamento e máscara, uma representação binária é presumida e os números negativos são representados em uma variante do complemento de 2 que dá a ilusão de uma string infinita de bits de sinal estendendo-se para a esquerda.

Booleanos (`bool`)

Estes representam os valores da verdade Falsos e Verdadeiros. Os dois objetos que representam os valores `False` e `True` são os únicos objetos booleanos. O tipo booleano é um subtipo do tipo inteiro, e os valores booleanos se comportam como os valores 0 e 1, respectivamente, em quase todos os contextos, com exceção de que, quando convertidos em uma string, as strings `"False"` ou `"True"` são retornados, respectivamente.

3.2.4.2. `numbers.Real` (`float`)

Estes representam números de ponto flutuante de precisão dupla no nível da máquina. Você está à mercê da arquitetura da máquina subjacente (e implementação C ou Java) para o intervalo aceito e tratamento de estouro. Python não oferece suporte a números de ponto flutuante de precisão única; a economia no uso do processador e da memória, que normalmente é o motivo de usá-los, é ofuscada pela sobrecarga do uso de objetos em Python, portanto, não há razão para complicar a linguagem com dois tipos de números de ponto flutuante.

3.2.4.3. `numbers.Complex` (`complex`)

Estes representam números complexos como um par de números de ponto flutuante de precisão dupla no nível da máquina. As mesmas advertências se aplicam aos números de ponto flutuante. As partes reais e imaginárias de um número complexo `z` podem ser obtidas através dos atributos somente leitura `z.real` e `z.imag`.

3.2.5. Sequências

Estes representam conjuntos ordenados finitos indexados por números não negativos. A função embutida `len()` retorna o número de itens de uma sequência. Quando o comprimento de uma sequência é *n*, o conjunto de índices contém os números 0, 1, …, *n*-1. O item *i* da sequência *a* é selecionado por `a[i]`. Algumas sequências, incluindo sequências embutidas, interpretam subscritos negativos adicionando o comprimento da

sequência. Por exemplo, `a[-2]` é igual a `a[n-2]`, o penúltimo item da sequência a com comprimento `n`.

O valor resultante deve ser um número inteiro não negativo menor que o número de itens na sequência. Caso contrário, uma **`IndexError`** é levantada.

Sequências também suportam fatiamento: `a[start:stop]``seleciona todos os itens com índice *k* tal que *start*``<=``*k* ``<` *stop*. Quando utilizada como expressão, uma fatia é uma sequência do mesmo tipo. O comentário anterior sobre índices negativos também se aplica a posições negativas em fatiamento. Observe que nenhum erro é gerado se a posição da fatia for menor que zero ou maior que o comprimento da sequência.

Se *start* estiver ausente ou for **`None`** , o fatiamento se comporta como se *start* fosse zero. Se *stop* estiver ausente ou for `None` , o fatiamento se comporta como se *stop* fosse igual ao comprimento da sequência.

Algumas sequências também suportam "fatiamento estendido" com um terceiro parâmetro de "etapa": `a[i:j:k]` seleciona todos os itens de *a* com índice *x* onde `x = i + n*k`, *n* >= `0` e *i* <= *x* < *j*.

As sequências são distinguidas de acordo com sua mutabilidade:

3.2.5.1. Sequências imutáveis

Um objeto de um tipo de sequência imutável não pode ser alterado depois de criado. (Se o objeto contiver referências a outros objetos, esses outros objetos podem ser mutáveis e podem ser alterados; no entanto, a coleção de objetos diretamente referenciada por um objeto imutável não pode ser alterada.)

Os tipos a seguir são sequências imutáveis:

Strings

Uma string (**`str`**) é uma sequência de valores que representam *caracteres* ou, mais formalmente, *pontos de código Unicode*. Todos os

pontos de código no intervalo de `0` a `0x10FFFF` podem ser representados em uma string.

Python não possui um tipo de *caractere* dedicado. Em vez disso, cada ponto de código na string é representado como um objeto de string com comprimento `1`.

A função embutida **`ord()`** converte um ponto de código de sua forma de string para um inteiro no intervalo de `0` a `0x10FFFF`;:func:*chr`converte um inteiro no intervalo de ``0`* a `0x10FFFF``para o objeto de string de comprimento ``1` correspondente. **`str.encode`pode ser usado para converter :class:`str()`** em **`bytes`** usando a codificação do texto, e **`bytes.decode()`** pode ser usado para realizar a operação inversa.

Tuplas

The items of a **`tuple`** are arbitrary Python objects. Tuples of two or more items are formed by comma-separated lists of expressions. A tuple of one item (a ‘singleton’) can be formed by affixing a comma to an expression (an expression by itself does not create a tuple, since parentheses must be usable for grouping of expressions). An empty tuple can be formed by an empty pair of parentheses.

Bytes

A **`bytes`** object is an immutable array. The items are 8-bit bytes, represented by integers in the range 0 <= x < 256. Bytes literals (like `b'abc'`) and the built-in **`bytes()`** constructor can be used to create bytes objects. Also, bytes objects can be decoded to strings via the **`decode()`** method.

3.2.5.2. Sequências mutáveis

As sequências mutáveis podem ser alteradas após serem criadas. As notações de subscrição e fatiamento podem ser usadas como o destino da atribuição e instruções **`del`** (*delete*, exclusão).

Nota

Os módulos `collections` e `array` fornecem exemplos adicionais de tipos de sequência mutáveis.

Atualmente, existem dois tipos de sequência mutável intrínseca:

Listas

Os itens de uma lista são objetos Python arbitrários. As listas são formadas colocando uma lista de expressões separada por vírgulas entre colchetes. (Observe que não há casos especiais necessários para formar listas de comprimento 0 ou 1.)

Vetores de bytes

Um objeto bytearray é um vetor mutável. Eles são criados pelo construtor embutido `bytearray()`. Além de serem mutáveis (e, portanto, não-hasheável), os vetores de bytes fornecem a mesma interface e funcionalidade que os objetos imutáveis `bytes`.

3.2.6. Tipos de conjuntos

Estes representam conjuntos finitos e não ordenados de objetos únicos e imutáveis. Como tal, eles não podem ser indexados por nenhum subscrito. No entanto, eles podem ser iterados, e a função embutida `len()` retorna o número de itens em um conjunto. Os usos comuns para conjuntos são testes rápidos de associação, remoção de duplicatas de uma sequência e computação de operações matemáticas como interseção, união, diferença e diferença simétrica.

Para elementos de conjunto, as mesmas regras de imutabilidade se aplicam às chaves de dicionário. Observe que os tipos numéricos obedecem às regras normais para comparação numérica: se dois números forem iguais (por exemplo, `1` e `1.0`), apenas um deles pode estar contido em um conjunto.

Atualmente, existem dois tipos de conjuntos intrínsecos:

Conjuntos

Estes representam um conjunto mutável. Eles são criados pelo construtor embutido `set()` e podem ser modificados posteriormente por vários métodos, como `add()`.

Conjuntos congelados

Estes representam um conjunto imutável. Eles são criados pelo construtor embutido `frozenset()`. Como um frozenset é imutável e hasheável, ele pode ser usado novamente como um elemento de outro conjunto, ou como uma chave de dicionário.

3.2.7. Mapeamentos

Eles representam conjuntos finitos de objetos indexados por conjuntos de índices arbitrários. A notação subscrito `a[k]` seleciona o item indexado por `k` do mapeamento `a`; isso pode ser usado em expressões e como alvo de atribuições ou instruções `del`. A função embutida `len()` retorna o número de itens em um mapeamento.

Existem dois tipos de mapeamento intrínseco:

3.2.7.1. Dicionários

Eles representam conjuntos finitos de objetos indexados por valores quase arbitrários. Os únicos tipos de valores não aceitáveis como chaves são os valores que contêm listas ou dicionários ou outros tipos mutáveis que são comparados por valor em vez de por identidade de objeto, o motivo é que a implementação eficiente de dicionários requer que o valor de hash de uma chave permaneça constante. Os tipos numéricos usados para chaves obedecem às regras normais para comparação numérica: se dois números forem iguais (por exemplo, `1` e `1.0`), eles podem ser usados alternadamente para indexar a mesma entrada do dicionário.

Dicionários preservam a ordem de inserção, o que significa que as chaves serão produzidas na mesma ordem em que foram adicionadas sequencialmente no dicionário. Substituir uma chave existente não altera a ordem, no entanto, remover uma chave e inseri-la novamente irá adicioná-la ao final em vez de manter seu lugar anterior.

Os dicionários são mutáveis; eles podem ser criados pela notação `{}` (veja a seção Sintaxes de criação de dicionário).

Os módulos de extensão `dbm.ndbm` e `dbm.gnu` fornecem exemplos adicionais de tipos de mapeamento, assim como o módulo `collections`.

Alterado na versão 3.7: Dicionários não preservavam a ordem de inserção nas versões do Python anteriores à 3.6. No CPython 3.6, a ordem de inserção foi preservada, mas foi considerada um detalhe de implementação naquela época, em vez de uma garantia da linguagem.

3.2.7.2. Dicionários congelados

Estes representam um dicionário imutável. Eles são criados pelo construtor embutido `frozendict()`. Um frozendict é hashable se todas as suas chaves e valores forem hasheável, caso em que pode ser usado como um elemento de um conjunto, ou como uma chave em outro mapeamento. `frozendict` não é uma subclasse de `dict` ; ele herda diretamente de `object`.

Adicionado na versão 3.15.

3.2.8. Tipos chamáveis

Estes são os tipos aos quais a operação de chamada de função (veja a seção Chamadas) pode ser aplicada:

3.2.8.1. Funções definidas pelo usuário

Um objeto função definido pelo usuário será criado pela definição de função (veja a seção Definições de função). A mesma deverá ser invocada com uma lista de argumentos contendo o mesmo número de itens que a lista de parâmetros formais da função.

3.2.8.1.1. Atributos especiais de somente leitura

Atributo	Significado
function.__builtins__	A reference to the `dictionary` that holds the function's builtins namespace. *Adicionado na versão 3.10.*
function.__globals__	Uma referência ao `dicionário` que contém as variáveis globais da função – o espaço de nomes global do módulo no qual a função foi definida.
function.__closure__	`None` ou uma `tuple` de células que contêm ligação para os nomes especificados no atributo `co_freevars` do `objeto código` da função. Um objeto de célula tem o atributo `cell_contents`. Isso pode ser usado para obter o valor da célula, bem como definir o valor.

3.2.8.1.2. Atributos especiais graváveis

A maioria desses atributos verifica o tipo do valor atribuído:

Atributo	Significado
function.__doc__	A string de documentação da função, ou `None` se indisponível.

Atributo	Significado
function.__name__	O nome da função. Veja também: `atributos __name__`.
function.__qualname__	O nome qualificado da função. Veja também: `atributos __qualname__`. *Adicionado na versão 3.3.*
function.__module__	O nome do módulo em que a função foi definida ou `None` se indisponível.
function.__defaults__	Uma `tuple` contendo valores de parâmetro padrão para aqueles parâmetros que possuem padrões, ou `None` se nenhum parâmetro tiver um valor padrão.
function.__code__	O objeto código que representa o corpo da função compilada.
function.__dict__	O espaço de nomes que provê atributos de função arbitrários. Veja também: `atributos __dict__`.

Atributo	Significado
function.__annotations__	Um `dicionário` contendo anotações de parâmetros. As chaves do dicionário são os nomes dos parâmetros e `'return'` para a anotação de retorno, se fornecida. Veja também: `object.__annotations__`. *Alterado na versão 3.14:* Anotações são agora avaliados preguiçosamente. Veja a **PEP 649**.
function.__annotate__	A função de anotação para esta função, ou `None` se a função não tiver anotações. Veja `object.__annotate__`. *Adicionado na versão 3.14.*
function.__kwdefaults__	Um `dicionário` contendo padrões apenas para parâmetros somente-nomeados.
function.__type_params__	Uma `tuple` contendo os parâmetros de tipo de uma função genérica. *Adicionado na versão 3.12.*

Os objetos de função também dão suporte à obtenção e definição de atributos arbitrários, que podem ser usados, por exemplo, para anexar metadados a funções. A notação de ponto de atributo regular é usada para obter e definir tais atributos.

A implementação atual do CPython provê apenas atributos de função em funções definidas pelo usuário. Atributos de função em funções embutido podem ser suportados no futuro.

Informações adicionais sobre a definição de uma função podem ser obtidas de seu objeto código (acessível através do atributo `__code__`).

3.2.8.2. Métodos de instância

Um objeto método de instância combina uma classe, uma instância de classe e qualquer objeto chamável (normalmente uma função definida pelo usuário).

Atributos especiais de somente leitura:

method.__self__	Refere-se ao objeto instância da classe ao qual o método é vinculado
method.__func__	Refere-se ao objeto função original
method.__doc__	A documentação do método (igual a `method.__func__.__doc__`). Um `string` se a função original tivesse uma docstring, caso contrário `None`.
method.__name__	O nome do método (mesmo que `method.__func__.__name__`)
method.__module__	O nome do módulo em que o método foi definido ou `None` se indisponível.

Os métodos também implementam o acesso (mas não a configuração) dos atributos arbitrários da função no objeto função subjacente.

Objetos método definidos pelo usuário podem ser criados ao obter um atributo de uma classe (talvez através de uma instância dessa classe), se esse atributo for um objeto função definido pelo usuário ou um objeto **classmethod** .

Quando um objeto método de instância é criado recuperando um objeto função definido pelo usuário de uma classe por meio de uma de suas instâncias, seu atributo **__self__** é a instância, e o objeto método é considerado *vinculado*. O atributo **__func__** do novo método é o objeto da função original.

Quando um objeto método de instância é criado obtendo um objeto **classmethod** de uma classe ou instância, seu atributo **__self__** é a própria classe, e seu atributo **__func__** é o objeto função subjacente ao método de classe.

Quando um objeto método de instância é chamado, a função subjacente (**__func__**) é chamada, inserindo a instância de classe (**__self__**) na frente da lista de argumentos. Por exemplo, quando **C** é uma classe que contém uma definição para uma função **f()**, e `x` é uma instância de **C**, chamando `x.f(1)` é equivalente a chamar `C.f(x, 1)`.

Quando um objeto método de instância é derivado de um objeto **classmethod**, a "instância de classe" armazenada em **__self__** será, na verdade, a própria classe, de modo que chamar `x.f(1)` ou `C.f(1)` é equivalente a chamar `f(C,1)` sendo `f` a função subjacente.

É importante observar que funções definidas pelo usuário que são atributos de uma instância de classe não são convertidas em métodos vinculados; isso *somente* acontece quando a função é um atributo da classe.

3.2.8.3. Funções geradoras

A function or method which contains a **yield** expression (see section Expressões yield) is called a *generator function*. Such a function, when

called, always returns an iterator object which can be used to execute the body of the function: calling the iterator's `iterator.__next__()` method will cause the function to execute until it provides a value using the **`yield`** expression. When the function executes a `return` statement or falls off the end, a `StopIteration` exception is raised and the iterator will have reached the end of the set of values to be returned.

3.2.8.4. Funções de corrotina

Uma função ou um método que é definida(o) usando `async def` é chamado de *função de corrotina*. Tal função, quando chamada, retorna um objeto de corrotina. Ele pode conter expressões `await`, bem como instruções `async with` e `async for`. Veja também a seção Objetos corrotina.

3.2.8.5. Funções geradoras assíncronas

A function or method which is defined using `async def` and which contains a `yield` expression is called a *asynchronous generator function*. Such a function, when called, returns an asynchronous iterator object which can be used in an `async for` statement to execute the body of the function.

Chamar o método `aiterator.__anext__` do iterador assíncrono retornará um aguardável que, quando aguardado, será executado até fornecer um valor usando a expressão `yield`. Quando a função executa uma instrução vazia `return` ou chega ao final, uma exceção `StopAsyncIteration` é levantada e o iterador assíncrono terá alcançado o final do conjunto de valores a serem produzidos.

3.2.8.6. Funções embutidas

Um objeto função embutida é um wrapper em torno de uma função C. Exemplos de funções embutidas são `len()` e `math.sin()` (`math` é um módulo embutido padrão). O número e o tipo dos argumentos são determinados pela função C. Atributos especiais de somente leitura:

- **__doc__** é a string de documentação da função, ou `None` se não estiver disponível. Veja `function.__doc__`.
- **__name__** é o nome da função. Veja `function.__name__`.
- **__self__** é definido para `None` (mas veja o próximo item).
- **__module__** é o nome do módulo no qual a função foi definida ou `None` se não estiver disponível. Veja `function.__module__`.

3.2.8.7. Métodos embutidos

Este é realmente um disfarce diferente de uma função embutida, desta vez contendo um objeto passado para a função C como um argumento extra implícito. Um exemplo de método embutido é `alist.append()`, presumindo que *alist* é um objeto de lista. Nesse caso, o atributo especial de somente leitura **__self__** é definido como o objeto denotado por *alist*. (O atributo tem a mesma semântica de `outros métodos de instância`.)

3.2.8.8. Classes

Classes são chamáveis. Esses objetos normalmente agem como fábricas para novas instâncias de si mesmos, mas variações são possíveis para tipos de classe que substituem `__new__()`. Os argumentos da chamada são passados para **__new__()** e, no caso típico, para `__init__()` para inicializar a nova instância.

3.2.8.9. Instâncias de classe

Instâncias de classes arbitrárias podem ser tornados chamáveis definindo um método `__call__()` em sua classe.

3.2.9. Módulos

Módulos são uma unidade organizacional básica do código Python, e são criados pelo sistema de importação quando invocado pela instrução `import`, ou chamando funções como `importlib.import_module()` e a embutida `__import__()`. Um objeto módulo tem um espaço de nomes implementado por um objeto `dicionário` (este é o dicionário referenciado pelo atributo

`__globals__` das funções definidas no módulo). As referências de atributos são traduzidas para pesquisas neste dicionário, por exemplo, `m.x` é equivalente a `m.__dict__["x"]`. Um objeto módulo não contém o objeto código usado para inicializar o módulo (uma vez que não é necessário depois que a inicialização é concluída).

A atribuição de atributo atualiza o dicionário de espaço de nomes do módulo, por exemplo, `m.x = 1` é equivalente a `m.__dict__["x"] = 1`.

3.2.9.1. Atributos relacionados à importação em objetos de módulo

Objetos de módulo têm os seguintes atributos que se relacionam ao sistema de importação. Quando um módulo é criado usando o maquinário associado ao sistema de importação, esses atributos são preenchidos com base no spec do módulo, antes que o carregador execute e carregue o módulo.

Para criar um módulo dinamicamente em vez de usar o sistema de importação, é recomendado usar `importlib.util.module_from_spec()`, que definirá os vários atributos controlados pela importação para valores apropriados. Também é possível usar o construtor `types.ModuleType` para criar módulos diretamente, mas essa técnica é mais propensa a erros, pois a maioria dos atributos deve ser definida manualmente no objeto do módulo após ele ter sido criado ao usar essa abordagem.

Cuidado

Com exceção de `__name__`, é **fortemente** recomendado que você confie no `__spec__` e seus atributos em vez de qualquer um dos outros atributos individuais listados nesta subseção. Observe que atualizar um atributo em `__spec__` não atualizará o atributo correspondente no próprio módulo:

```
>>> import typing
>>> typing.__name__, typing.__spec__.name
('typing', 'typing')
>>> typing.__spec__.name = 'spelling'
>>> typing.__name__, typing.__spec__.name
('typing', 'spelling')
>>> typing.__name__ = 'keyboard_smashing'
```

```
>>> typing.__name__, typing.__spec__.name
('keyboard_smashing', 'spelling')
```

module.__name__

O nome usado para identificar exclusivamente o módulo no sistema de importação. Para um módulo executado diretamente, isso será definido como `"__main__"`.

Este atributo deve ser definido como o nome totalmente qualificado do módulo. Espera-se que ele corresponda ao valor de **`module.__spec__.name`**.

module.__spec__

Um registro do estado relacionado ao sistema de importação do módulo.

Define com **`spec de módulo`** que foi usado ao importar o módulo. Veja Especificações de módulo para mais detalhes.

Adicionado na versão 3.4.

module.__package__

O pacote ao qual um módulo pertence.

Se o módulo for de nível superior (ou seja, não fizer parte de nenhum pacote específico), o atributo deve ser definido como `''` (a string vazia). Caso contrário, deve ser definido como o nome do pacote do módulo (que pode ser igual a **`module.__name__`** se o módulo em si for um pacote). Veja **PEP 366** [https://peps.python.org/pep-0366/] para mais detalhes.

Este atributo é usado em vez de **`__name__`** para calcular importações relativas explícitas para módulos principais. O padrão é `None` para módulos criados dinamicamente usando o construtor **`types.ModuleType`**; use **`importlib.util.module_from_spec()`** em vez disso para garantir que o atributo seja definido como **`str`**.

É **fortemente** recomendado que você use **`module.__spec__.parent`** em vez de **`module.__package__`**. **`__package__`** agora só é usado como

fallback se **__spec__.parent** não estiver definido, e esse caminho de fallback está descontinuado.

Alterado na versão 3.4: Este atributo agora presume o padrão `None` para módulos criados dinamicamente usando o construtor `types.ModuleType`. Anteriormente, o atributo era opcional.

Alterado na versão 3.6: Espera-se que o valor de **__package__** seja o mesmo que `__spec__.parent`. `__package__` agora é usado apenas como fallback durante a resolução de importação se **__spec__.parent** não estiver definido.

Alterado na versão 3.10: `ImportWarning` é levantada se uma resolução de importação retorna para **__package__** em vez de `__spec__.parent`.

Alterado na versão 3.12: Levanta `DeprecationWarning` em vez de `ImportWarning` ao retornar para **__package__** durante a resolução de importação.

Descontinuado desde a versão 3.13, removido na versão 3.15: **__package__** deixará de ser definido ou levado em consideração pelo sistema de importação ou biblioteca padrão.

module.__loader__

O objeto carregador que o maquinário de importação usou para carregar o módulo.

Este atributo é útil principalmente para introspecção, mas pode ser usado para funcionalidades adicionais específicas do carregador, por exemplo, para obter dados associados a um carregador.

__loader__ assume como padrão `None` para módulos criados dinamicamente usando o construtor `types.ModuleType`; use `importlib.util.module_from_spec()` para garantir que o atributo seja definido como um objeto carregador.

É **fortemetne** recomendado que você use `module.__spec__.loader` em vez de **module.__loader__**.

Alterado na versão 3.4: Este atributo agora presume o padrão `None` para módulos criados dinamicamente usando o construtor `types.ModuleType`. Anteriormente, o atributo era opcional.

Descontinuado desde a versão 3.12, será removido na versão 3.16: A definição **`__loader__`** em um módulo enquanto falha na definição de **`__spec__.loader`** está descontinuado. No Python 3.16, **`__loader__`** deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.

module.__path__

Uma sequência (possivelmente vazia) de strings enumerando os locais onde os submódulos do pacote serão encontrados. Módulos que não sejam de pacote não devem ter um atributo **`__path__`**. Veja Atributo __path__ dos módulos para mais detalhes.

É **fortemente** recomendado que você use `module.__spec__.submodule_search_locations` em vez de **`module.__path__`**.

module.__file__

`__file__` is an optional attribute that may or may not be set. Both attributes should be a `str` when they are available.

An optional attribute, **`__file__`** indicates the pathname of the file from which the module was loaded (if loaded from a file), or the pathname of the shared library file for extension modules loaded dynamically from a shared library. It might be missing for certain types of modules, such as C modules that are statically linked into the interpreter, and the import system may opt to leave it unset if it has no semantic meaning (for example, a module loaded from a database).

Alterado na versão 3.15: The `__cached__` attribute is no longer set on modules or taken into consideration by the import system or standard library.

3.2.9.2. Outros atributos graváveis em objetos de módulo

Além dos atributos relacionados à importação listados acima, os objetos de módulo também têm os seguintes atributos graváveis:

module.__doc__

A string de documentação do módulo, ou `None` se indisponível. Veja também: **`atributos __doc__`**.

module.__annotations__

Um dicionário contendo anotações de variável coletadas durante a execução do corpo do módulo. Para as melhores práticas sobre como trabalhar com **`__annotations__`**. Veja **`annotationlib`**.

Alterado na versão 3.14: Anotações são agora avaliados preguiçosamente. Veja a **PEP 649** [https://peps.python.org/pep-0649/].

module.__annotate__

A função de anotação para este módulo, ou `None` se o módulo não tiver anotações. Veja também: atributos **`__annotate__`**.

Adicionado na versão 3.14.

module.__lazy_modules__

A container (an object implementing **`__contains__()`**) of fully qualified module name strings. When defined at module scope, any regular **`import`** statement in that module whose target module name appears in this container is treated as a lazy import, as if the **`lazy`** keyword had been used. Imports inside functions, class bodies, or **`try`**/**`except`**/**`finally`** blocks are unaffected.

See Compatibility via __lazy_modules__ for details and examples.

Adicionado na versão 3.15.

3.2.9.3. Dicionários do módulo

Os objetos de módulo também têm o seguinte atributo especial somente leitura:

module.__dict__

O espaço de nomes do módulo como um objeto dicionário. Exclusivamente entre os atributos listados aqui, **`__dict__`** não pode ser acessado como uma variável global de dentro de um módulo; ele só pode ser acessado como um atributo em objetos de módulo.

Por causa da maneira como CPython limpa dicionários de módulos, o dicionário do módulo será limpo quando o módulo sair do escopo, mesmo se o dicionário ainda tiver referências ativas. Para evitar isso, copie o dicionário ou mantenha o módulo por perto enquanto usa seu dicionário diretamente.

3.2.10. Classes personalizadas

Tipos de classe personalizados são tipicamente criados por definições de classe (veja a seção Definições de classe). Uma classe possui um espaço de nomes implementado por um objeto dicionário. As referências de atributos de classe são traduzidas para pesquisas neste dicionário, por exemplo, `C.x` é traduzido para `C.__dict__["x"]` (embora haja uma série de ganchos que permitem outros meios de localizar atributos). Quando o nome do atributo não é encontrado lá, a pesquisa do atributo continua nas classes base. Essa pesquisa das classes base usa a ordem de resolução de métodos C3, que se comporta corretamente mesmo na presença de estruturas de herança em losango, onde há vários caminhos de herança que levam de volta a um ancestral comum. Detalhes adicionais sobre a ordem de resolução de métodos (MRO) C3 usado pelo Python podem ser encontrados em A Ordem de Resolução de Métodos do Python 2.3.

Quando uma referência de atributo de classe (para uma classe **`C`**, digamos) produziria um objeto método de classe, ele é transformado em um objeto método de instância cujo atributo **`__self__`** é **`C`**. Quando produziria um objeto **`staticmethod`**, ele é transformado no objeto encapsulado pelo objeto método estático. Veja a seção Implementando descritores para outra maneira em que os atributos recuperados de uma classe podem diferir daqueles realmente contidos em seu **`__dict__`**.

As atribuições de atributos de classe atualizam o dicionário da classe, nunca o dicionário de uma classe base.

Um objeto classe pode ser chamado (veja acima) para produzir uma instância de classe (veja abaixo).

3.2.10.1. Atributos especiais

Atributo	Significado
type.__name__	O nome da classe. Veja também: `atributos __name__`.
type.__qualname__	O nome qualificado da classe. Veja também: `atributos __qualname__`.
type.__module__	O nome do módulo no qual a classe foi definida.
type.__dict__	Um `proxy de mapeamento` fornecendo uma visão somente leitura do espaço de nomes da classe. Veja também: `atributos __dict__`.
type.__bases__	Uma `tuple` contendo as bases da classe. Na maioria dos casos, para uma classe definida como `class X(A, B, C)`, `X.__bases__` será exatamente igual a `(A, B, C)`.

Atributo	Significado
type.__base__	A única classe base na cadeia de herança responsável pelo layout de memória das instâncias. Este atributo corresponde a `tp_base` no nível C.
type.__doc__	A string de documentação da classe, ou `None` se não estiver definida. Não herdado por subclasses.

Atributo	Significado
type.__annotations__	Um dicionário contendo anotações de variáveis coletadas durante a execução do corpo da classe. Veja também: `atributos __annotations__`. Para melhores práticas sobre como trabalhar com `__annotations__`, consulte `annotationlib`. Use `annotationlib.get_annotations()` em vez de acessar este atributo diretamente. **Aviso** Acessar o atributo `__annotations__` diretamente em um objeto classe pode retornar anotações para a classe errada, especialmente em certos casos em que a classe, sua classe base ou uma metaclasse é definida em `from __future__ import annotations`. Consulte a **749** para obter detalhes. Este atributo não existe em certas classes embutidas. Em classes definidas pelo usuário sem `__annotations__`, ele é um dicionário vazio. *Alterado na versão 3.14:* Anotações são agora avaliados preguiçosamente. Veja a **PEP 649**.

Atributo	Significado
type.__annotate__()	A função de anotação para esta classe, ou `None` se a classe não tiver anotações. Veja também: `atributos __annotate__`. *Adicionado na versão 3.14.*
type.__type_params__	Uma `tuple` contendo os parâmetros de tipo de uma classe genérica. *Adicionado na versão 3.12.*
type.__static_attributes__	Uma `tuple` contendo nomes de atributos dessa classe que são atribuídos por meio de `self.X` de qualquer função em seu corpo. *Adicionado na versão 3.13.*
type.__firstlineno__	O número da primeira linha da definição de classe, incluindo decoradores. Definir o atributo `__module__` remove o item `__firstlineno__` do dicionário do tipo. *Adicionado na versão 3.13.*
type.__mro__	A `tuple` de classes que são consideradas ao procurar por classes bases durante resolução de métodos.

3.2.10.2. Métodos especiais

Além dos atributos especiais descritos acima, todas as classes Python também têm os dois métodos a seguir disponíveis:

type.mro()

Este método pode ser substituído por uma metaclasse para personalizar a ordem de resolução de métodos para suas instâncias. Ele é chamado na instanciação da classe, e o seu resultado é armazenado em `__mro__`.

type.__subclasses__()

Cada classe mantém uma lista de referências fracas para suas subclasses imediatas. Este método retorna uma lista de todas essas referências ainda vivas. A lista está na ordem que são definidas. Exemplo:

```
>>> class A: pass
>>> class B(A): pass
>>> A.__subclasses__()
[<class 'B'>]
```

3.2.11. Instâncias de classe

Uma instância de classe é criada chamando um objeto classe (veja acima). Uma instância de classe tem um espaço de nomes implementado como um dicionário que é o primeiro lugar no qual as referências de atributos são pesquisadas. Quando um atributo não é encontrado lá, e a classe da instância possui um atributo com esse nome, a pesquisa continua com os atributos da classe. Se for encontrado um atributo de classe que seja um objeto função definido pelo usuário, ele é transformado em um objeto método de instância cujo atributo `__self__` é a instância. Métodos estáticos e métodos de classe também são transformados; veja acima em “Classes”. Veja a seção Implementando descritores para outra maneira em que os atributos de uma classe recuperados através de suas instâncias podem diferir dos objetos realmente armazenados no `__dict__` da classe. Se nenhum atributo de classe for encontrado, e a classe do objeto tiver um método `__getattr__()`, este é chamado para satisfazer a pesquisa.

As atribuições e exclusões de atributos atualizam o dicionário da instância, nunca o dicionário de uma classe. Se a classe tem um método

`__setattr__()` ou `__delattr__()`, ele é chamado ao invés de atualizar o dicionário da instância diretamente.

As instâncias de classe podem fingir ser números, sequências ou mapeamentos se tiverem métodos com certos nomes especiais. Veja a seção Nomes de métodos especiais.

3.2.11.1. Atributos especiais

object.__class__

A classe à qual pertence uma instância de classe.

object.__dict__

Um dicionário ou outro objeto de mapeamento usado para armazenar atributos (graváveis) de um objeto. Nem todas as instâncias têm um atributo `__dict__`; veja a seção sobre __slots__ para mais detalhes.

3.2.12. Objetos de E/S (também conhecidos como objetos arquivo)

O objeto arquivo representa um arquivo aberto. Vários atalhos estão disponíveis para criar objetos arquivos: a função embutida `open()`, e também `os.popen()`, `os.fdopen()` e o método `makefile()` de objetos soquete (e talvez por outras funções ou métodos fornecidos por módulos de extensão).

File objects implement common methods, listed below, to simplify usage in generic code. They are expected to be Gerenciadores de contexto da instrução with.

Os objetos `sys.stdin`, `sys.stdout` e `sys.stderr` são inicializados para objetos arquivo que correspondem aos fluxos de entrada, saída e erro padrão do interpretador; eles são todos abertos em modo texto e, portanto, seguem a interface definida pela classe abstrata `io.TextIOBase`.

file.read(*size=-1*, /)

Retrieve up to *size* data from the file. As a convenience if *size* is unspecified or -1 retrieve all data available.

file.write(*data*, /)

Store *data* to the file.

file.close()

Flush any buffers and close the underlying file.

3.2.13. Tipos internos

Alguns tipos usados internamente pelo interpretador são expostos ao usuário. Suas definições podem mudar com versões futuras do interpretador, mas são mencionadas aqui para fins de integridade.

3.2.13.1. Objetos código

Objetos código representam código Python executável *compilados em bytes* ou bytecode. A diferença entre um objeto código e um objeto função é que o objeto função contém uma referência explícita aos globais da função (o módulo no qual foi definida), enquanto um objeto código não contém nenhum contexto; também os valores de argumento padrão são armazenados no objeto função, não no objeto código (porque eles representam os valores calculados em tempo de execução). Ao contrário dos objetos função, os objetos código são imutáveis e não contêm referências (direta ou indiretamente) a objetos mutáveis.

3.2.13.1.1. Atributos especiais de somente leitura

codeobject.co_name	O nome da função
codeobject.co_qualname	O nome completo da função *Adicionado na versão 3.11.*

codeobject.co_argcount	O número total de parâmetros posicionais (incluindo parâmetros somente-posicionais e parâmetros com valores padrão) que a função possui
codeobject.co_posonlyargcount	O número de parâmetros somente-posicionais (incluindo argumentos com valores padrão) que a função possui
codeobject.co_kwonlyargcount	O número de parâmetros somente-nomeados (incluindo argumentos com valores padrão) que a função possui
codeobject.co_nlocals	O número de variáveis locais usadas pela função (incluindo parâmetros)
codeobject.co_varnames	Uma `tuple` contendo os nomes das variáveis locais na função (começando com os nomes dos parâmetros)
codeobject.co_cellvars	Uma `tuple` contendo os nomes de variáveis locais que são referenciadas a partir de pelo menos um escopo aninhado de dentro da função
codeobject.co_freevars	Uma `tuple` contendo os nomes de variáveis livres (de clausura) que um escopo aninhado referencia em um escopo externo. Veja também `function.__closure__`.

	Observação: referências a nomes globais e embutidos *não* estão incluídas.
codeobject.co_code	Uma string representando a sequência de instruções bytecode na função
codeobject.co_consts	Um `tuple` contendo os literais usados pelo bytecode na função
codeobject.co_names	Um `tuple` contendo os nomes usados pelo bytecode na função
codeobject.co_filename	O nome do arquivo do qual o código foi compilado
codeobject.co_firstlineno	O número da linha da primeira linha da função
codeobject.co_linetable	A `bytes` object containing encoded source location information. The exact format is an implementation detail and may change between Python versions. Use `co_lines()` and `co_positions()` for supported access to line and position information. To create a modified copy of a code object, use `replace()`. *Adicionado na versão 3.10.*
codeobject.co_stacksize	O tamanho de pilha necessário do objeto código

codeobject.co_flags	Um número **inteiro** codificando uma série de sinalizadores para o interpretador.

Os seguintes bits sinalizadores são definidos para **co_flags**: o bit `0x04` é definido se a função usa a sintaxe `*arguments` para aceitar um número arbitrário de argumentos posicionais; o bit `0x08` é definido se a função usa a sintaxe `**keywords` para aceitar argumentos nomeados arbitrários; o bit `0x20` é definido se a função for um gerador. Veja Code Objects Bit Flags para detalhes na semântica de cada sinalizadores que podem estar presentes.

Declarações de recursos futuros (por exemplo, `from __future__ import division`) também usam bits em **co_flags** para indicar se um objeto código foi compilado com um recurso específico habilitado. Veja **compiler_flag**.

Outros bits em **co_flags** são reservados para uso interno.

Se um objeto código representa uma função e tem uma docstring, o bit **CO_HAS_DOCSTRING** é definido em **co_flags** e o primeiro item em **co_consts** é a docstring da função.

3.2.13.1.2. Métodos de objetos código

codeobject.co_positions()

Retorna um iterável das posições no código-fonte de cada instrução bytecode no objeto código.

O iterador retorna **tuple**s contendo `(start_line, end_line, start_column, end_column)`. A *i-nésima* tupla corresponde à posição do código-fonte que compilou para a *i-nésima* unidade de código. As informações da coluna são deslocamentos de bytes utf-8 indexados em 0 na linha de código fornecida.

A informação posicional pode estar ausente. Veja uma lista não-exaustiva de casos onde isso pode acontecer:

- Executando o interpretador com `no_debug_ranges` `-X`.
- Carregando um arquivo pyc compilado com `no_debug_ranges` `-X`.
- Tuplas posicionais correspondendo a instruções artificiais.
- Números de linha e coluna que não podem ser representados devido a limitações específicas de implementação.

Quando isso ocorre, alguns ou todos elementos da tupla podem ser `None`.

Adicionado na versão 3.11.

Nota

Esse recurso requer o armazenamento de posições de coluna no objeto código, o que pode resultar em um pequeno aumento no uso de memória do interpretador e no uso de disco para arquivos Python compilados. Para evitar armazenar as informações extras e/ou desativar a exibição das informações extras de rastreamento, use a opção de linha de comando `no_debug_ranges` `-X` ou a variável de ambiente `PYTHONNODEBUGRANGES`.

codeobject.co_lines()

Retorna um iterador que produz informações sobre intervalos sucessivos de bytecodes. Cada item gerado é uma `tuple` de `(start, end, lineno)`:

- `start` (um `int`) representa o deslocamento (inclusivo) do início do intervalo bytecode
- `end` (um `int`) representa o deslocamento (exclusivo) do fim do intervalo bytecode
- `lineno` é um `int` representando o número da linha do intervalo do bytecode, ou `None` se os bytecodes no intervalo fornecido não tiverem número de linha

Os itens gerados terão as seguintes propriedades:

- O primeiro intervalo gerado terá um `start` de 0.
- Os intervalos `(start, end)` serão não decrescentes e consecutivos. Ou seja, para qualquer par de **`tuple`**s, o `start` do segundo será igual ao `end` do primeiro.
- Nenhum intervalo será inverso: `end >= start` para todos os trios.
- A última **`tuple`** gerada terá `end` igual ao tamanho do bytecode.

Intervalos de largura zero, onde `start == end`, são permitidos. Intervalos de largura zero são usados para linhas que estão presentes no código-fonte, mas foram eliminadas pelo compilador de bytecode.

Adicionado na versão 3.10.

Ver também

PEP 626 [https://peps.python.org/pep-0626/] - Números de linha precisos para depuração e outras ferramentas.
A PEP que introduziu o método **`co_lines()`**.

codeobject.replace(***kwargs*)

Retorna uma cópia do objeto de código com novos valores para os campos especificados.

Objetos de código também são suportados pela função genérica **`copy.replace()`**.

Adicionado na versão 3.8.

3.2.13.2. Objetos quadro

Objetos quadro representam quadros de execução. Eles podem ocorrer em objetos traceback e também são passados para funções de rastreamento registradas.

3.2.13.2.1. Atributos especiais de somente leitura

frame.f_back	Aponta para o quadro de pilha anterior (em direção ao chamador), ou `None` se este for o quadro de pilha mais abaixo.
frame.f_code	O objeto código sendo executado neste quadro. Acessar este atributo levanta um evento de auditoria `object.__getattr__` com os argumentos `obj` e `"f_code"`.
frame.f_locals	O mapeamento usado pelo quadro para procurar variáveis locais. Se o quadro se referir a um escopo otimizado, isso pode retornar um objeto proxy write-through. *Alterado na versão 3.13:* Retorna um proxy para escopos otimizados.
frame.f_globals	O dicionário usado pelo quadro para procurar variáveis globais
frame.f_builtins	O dicionário usado pelo quadro para procurar nomes embutidos (intrínsecos)
frame.f_lasti	A “instrução precisa” do objeto quadro (este é um índice na string bytecode do objeto código)
frame.f_generator	O objeto gerador ou corrotina que possui este quadro, ou `None` se o quadro for uma função normal. *Adicionado na versão 3.14.*

3.2.13.2.2. Atributos especiais graváveis

frame.f_trace	Se não for `None`, esta é uma função chamada para vários eventos durante a execução do código (isso é usado por depuradores). Normalmente, um evento é disparado para cada nova linha de origem (veja `f_trace_lines`).
frame.f_trace_lines	Defina este atributo como `False` para desabilitar o acionamento de um evento de rastreamento para cada linha de origem.
frame.f_trace_opcodes	Defina este atributo para `True` para permitir que eventos por opcode sejam solicitados. Observe que isso pode levar a um comportamento indefinido do interpretador se as exceções levantadas pela função de rastreamento escaparem para a função que está sendo rastreada.
frame.f_lineno	O número da linha atual do quadro – escrever para isso de dentro de uma função de rastreamento faz saltar para a linha dada (apenas para o quadro mais abaixo). Um depurador pode implementar um comando Jump (também conhecido como Set Next Statement) escrevendo para esse atributo.

3.2.13.2.3. Métodos de objetos quadro

Objetos quadro têm suporte a um método:

frame.clear()

Este método limpa todas as referências a variáveis locais mantidas pelo quadro. Além disso, se o quadro pertencer a um gerador, o gerador é finalizado. Isso ajuda a quebrar os ciclos de referência que envolvem objetos quadro (por exemplo, ao capturar uma exceção e armazenar seu traceback para uso posterior).

`RuntimeError` é levantada se o quadro estiver em execução ou suspenso.

Adicionado na versão 3.4.

Alterado na versão 3.13: Tentar limpar um quadro suspenso levanta `RuntimeError` (como sempre foi o caso para quadros em execução).

3.2.13.3. Objetos traceback

Objetos traceback representam o stack trace (situação da pilha de execução) de uma exceção. Um objeto traceback é criado implicitamente quando ocorre uma exceção e também pode ser criado explicitamente chamando `types.TracebackType`.

Alterado na versão 3.7: Objetos traceback agora podem ser instanciados explicitamente a partir de código Python.

Para tracebacks criados implicitamente, quando a busca por um manipulador de exceção desenrola a pilha de execução, em cada nível desenrolado um objeto traceback é inserido na frente do traceback atual. Quando um manipulador de exceção é inserido, o stack trace é disponibilizado para o programa. (Veja a seção A instrução try.) É acessível como o terceiro item da tupla retornada por `sys.exc_info()`, e como o atributo `__traceback__` da exceção capturada.

Quando o programa não contém um manipulador adequado, o stack trace é escrito (formatado de maneira adequada) no fluxo de erro padrão; se o interpretador for interativo, ele também é disponibilizado ao usuário como `sys.last_traceback`.

Para tracebacks criados explicitamente, cabe ao criador do traceback determinar como os atributos `tb_next` devem ser vinculados para formar um stack trace completo.

Atributos especiais de somente leitura:

traceback.tb_frame	Aponta para o quadro de execução do nível atual. Acessar este atributo levanta um evento de auditoria `object.__getattr__` com os argumentos `obj` e `"tb_frame"`.
traceback.tb_lineno	Fornece o número da linha onde ocorreu a exceção
traceback.tb_lasti	Indica a “instrução precisa”.

O número da linha e a última instrução no traceback podem diferir do número da linha do seu objeto quadro se a exceção ocorreu em uma instrução `try` sem cláusula except correspondente ou com uma cláusula `finally`.

traceback.tb_next

O atributo especial de escrita `tb_next` é o próximo nível no stack trace (em direção ao quadro onde a exceção ocorreu), ou `None` se não houver próximo nível.

Alterado na versão 3.7: Este atributo agora é gravável

3.2.13.4. Objetos slice

Objetos slice são usados para representar fatias para métodos `__getitem__()`. Eles também são criados pela função embutida `slice()`.

Adicionado na versão 3.15: The `slice()` type now supports subscription. For example, `slice[float]` may be used in type annotations to indicate a slice containing `float` objects.

Atributos especiais de somente leitura: `start` é o limite inferior; `stop` é o limite superior; `step` é o valor da diferença entre elementos subjacentes; cada um desses atributos é `None` se omitido. Esses atributos podem ter qualquer tipo.

Objetos slice têm suporte a um método:

slice.indices(*self*, *length*)

Este método recebe um único argumento inteiro *length* e calcula informações sobre a fatia que o objeto slice descreveria se aplicado a uma sequência de itens de *length*. Ele retorna uma tupla de três inteiros; respectivamente, estes são os índices *start* e *stop* e o *step* ou comprimento de avanços da fatia. Índices ausentes ou fora dos limites são tratados de maneira consistente com fatias regulares.

3.2.13.5. Objetos método estático

Objetos método estático fornecem uma forma de transformar objetos função em objetos métodos descritos acima. Um objeto método estático é um invólucro em torno de qualquer outro objeto, comumente um objeto método definido pelo usuário. Quando um objeto método estático é recuperado de uma classe ou de uma instância de classe, o objeto retornado é o objeto encapsulado, do qual não está sujeito a nenhuma transformação adicional. Objetos método estático também são chamáveis. Objetos método estático são criados pelo construtor embutido `staticmethod()`.

3.2.13.6. Objetos método de classe

Um objeto método de classe, como um objeto método estático, é um invólucro em torno de outro objeto que altera a maneira como esse objeto é recuperado de classes e instâncias de classe. O comportamento dos objetos método de classe após tal recuperação é descrito acima, sob “métodos de

instância”. Objetos método de classe são criados pelo construtor embutido `classmethod()`.

3.3. Nomes de métodos especiais

Uma classe pode implementar certas operações que são chamadas por sintaxe especial (como operações aritméticas ou indexação e fatiamento), definindo métodos com nomes especiais. Esta é a abordagem do Python para *sobrecarga de operador*, permitindo que as classes definam seu próprio comportamento em relação aos operadores da linguagem. Por exemplo, se uma classe define um método chamado `__getitem__()`, e `x` é uma instância desta classe, então `x[i]` é aproximadamente equivalente a `type(x).__getitem__(x, i)`. Exceto onde mencionado, as tentativas de executar uma operação levantam uma exceção quando nenhum método apropriado é definido (tipicamente `AttributeError` ou `TypeError`).

Definir um método especial para `None` indica que a operação correspondente não está disponível. Por exemplo, se uma classe define `__iter__()` para `None`, a classe não é iterável, então chamar `iter()` em suas instâncias irá levantar um `TypeError` (sem recorrer a `__getitem__()`). [[2]]

Ao implementar uma classe que emula qualquer tipo embutido, é importante que a emulação seja implementada apenas na medida em que faça sentido para o objeto que está sendo modelado. Por exemplo, algumas sequências podem funcionar bem com a recuperação de elementos individuais, mas extrair uma fatia pode não fazer sentido. (Um exemplo disso é a interface NodeList no Modelo de Objeto de Documento do W3C.)

3.3.1. Personalização básica

object.__new__(*cls*[, ...])

Chamado para criar uma nova instância da classe *cls*. `__new__()` é um método estático (é um caso especial, então você não precisa declará-lo como tal) que recebe a classe da qual uma instância foi solicitada como seu primeiro argumento. Os argumentos restantes são aqueles passados

para a expressão do construtor do objeto (a chamada para a classe). O valor de retorno de **`__new__()`** deve ser a nova instância do objeto (geralmente uma instância de *cls*).

Implementações típicas criam uma nova instância da classe invocando o método **`__new__()`** da superclasse usando `super().__new__(cls[, ...])` com os argumentos apropriados e, em seguida, modificando a instância recém-criada conforme necessário antes de retorná-la.

Se **`__new__()`** é chamado durante a construção do objeto e retorna uma instância de *cls*, então o método **`__init__()`** da nova instância será chamado como `__init__(self[, ...])`, onde *self* é a nova instância e os argumentos restantes são os mesmos que foram passados para o construtor do objeto.

Se **`__new__()`** não retornar uma instância de *cls*, então o método **`__init__()`** da nova instância não será invocado.

`__new__()` destina-se principalmente a permitir que subclasses de tipos imutáveis (como int, str ou tupla) personalizem a criação de instâncias. Também é comumente substituído em metaclasses personalizadas para personalizar a criação de classes.

object.__init__(*self*[, ...])

Chamado após a instância ter sido criada (por **`__new__()`**), mas antes de ser retornada ao chamador. Os argumentos são aqueles passados para a expressão do construtor da classe. Se uma classe base tem um método **`__init__()`**, o método **`__init__()`** da classe derivada, se houver, deve chamá-lo explicitamente para garantir a inicialização apropriada da parte da classe base da instância; por exemplo: `super().__init__([args...])`.

Porque **`__new__()`** e **`__init__()`** trabalham juntos na construção de objetos (**`__new__()`** para criá-lo e **`__init__()`** para personalizá-lo), nenhum valor diferente de `None` pode ser retornado por **`__init__()`**; fazer isso fará com que uma **`TypeError`** seja levantada em tempo de execução.

object.__del__(*self*)

Chamado quando a instância está prestes a ser destruída. Também é chamada de finalizador ou (incorretamente) de destruidor. Se uma classe base tem um método `__del__()`, o método `__del__()` da classe derivada, se houver, deve chamá-lo explicitamente para garantir a exclusão adequada da parte da classe base da instância.

É possível (embora não recomendado!) para o método `__del__()` adiar a destruição da instância criando uma nova referência a ela. Isso é chamado de *ressurreição* de objeto. Depende se a implementação de `__del__()` é chamado uma segunda vez quando um objeto ressuscitado está prestes a ser destruído; a implementação atual do CPython chama-o apenas uma vez.

Não há garantia de que os métodos `__del__()` sejam chamados para objetos que ainda existem quando o interpretador sai. `weakref.finalize` fornece uma maneira direta de registrar uma função de limpeza a ser chamada quando um objeto é coletado como lixo.

Nota

`del x` não chama diretamente `x.__del__()` – o primeiro diminui a contagem de referências para `x` em um, e o segundo só é chamado quando a contagem de referências de `x` atinge zero.

É possível que um ciclo de referência impeça que a contagem de referência de um objeto chegue a zero. Neste caso, mais tarde, o ciclo será detectado e deletado pelo coletor de lixo cíclico. Uma causa comum de referências cíclicas é quando uma exceção foi capturada em uma variável local. O locals do quadro então referencia a exceção, que referencia seu próprio traceback, que referencia o locals de todos os quadros capturados no traceback.

Ver também

Documentação do módulo `gc`.

Aviso

Devido às circunstâncias precárias sob as quais os métodos `__del__()` são invocados, as exceções que ocorrem durante sua execução são ignoradas e um aviso é impresso em `sys.stderr` em seu lugar. Em particular:

- `__del__()` pode ser chamado quando um código arbitrário está sendo executado, incluindo de qualquer thread arbitrária. Se **`__del__()`** precisa bloquear ou invocar qualquer outro recurso de bloqueio, pode ocorrer um impasse, pois o recurso já pode ter sido levado pelo código que é interrompido para executar **`__del__()`**.
- `__del__()` pode ser executado durante o encerramento do interpretador. Como consequência, as variáveis globais que ele precisa acessar (incluindo outros módulos) podem já ter sido excluídas ou definidas como `None`. Python garante que os globais cujo nome comece com um único sublinhado sejam excluídos de seu módulo antes que outros globais sejam excluídos; se nenhuma outra referência a tais globais existir, isso pode ajudar a garantir que os módulos importados ainda estejam disponíveis no momento em que o método **`__del__()`** for chamado.

object.__repr__(*self*)

Chamado pela função embutida `repr()` para calcular a representação da string "oficial" de um objeto. Se possível, isso deve parecer uma expressão Python válida que pode ser usada para recriar um objeto com o mesmo valor (dado um ambiente apropriado). Se isso não for possível, uma string no formato `<...alguma descrição útil...>` deve ser retornada. O valor de retorno deve ser um objeto string. Se uma classe define `__repr__()`, mas não `__str__()`, então **`__repr__()`** também é usado quando uma representação de string "informal" de instâncias daquela classe é necessária.

Isso é normalmente usado para depuração, portanto, é importante que a representação seja rica em informações e inequívoca. Uma implementação padrão é fornecida pela própria classe `object`.

object.__str__(*self*)

Chamado por `str(object)`, a implementação padrão de `__format__()`, e as funções embutidas `print()`, para calcular a representação da string "informal" ou agradável para exibição de um objeto. O valor de retorno deve ser um objeto str.

Este método difere de `object.__repr__()` por não haver expectativa de que `__str__()` retorne uma expressão Python válida: uma representação mais conveniente ou concisa pode ser usada.

A implementação padrão definida pelo tipo embutido `object` chama `object.__repr__()`.

object.__bytes__(*self*)

Chamado por bytes para calcular uma representação de string de bytes de um objeto. Isso deve retornar um objeto `bytes`. A classe `object` em si não fornece este método.

object.__format__(*self*, *format_spec*)

Chamado pela função embutida `format()` e, por extensão, avaliação de literais de string formatadas e o método `str.format()`, para produzir uma representação de string "formatada" de um objeto. O argumento *format_spec* é uma string que contém uma descrição das opções de formatação desejadas. A interpretação do argumento *format_spec* depende do tipo que implementa `__format__()`, entretanto a maioria das classes delegará a formatação a um dos tipos embutidos ou usará uma sintaxe de opção de formatação semelhante.

Consulte Minilinguagem de especificação de formato para uma descrição da sintaxe de formatação padrão.

O valor de retorno deve ser um objeto string.

A implementação padrão pela classe `object` deve receber uma string *format_spec* vazia. Ela delega para `__str__()`.

Alterado na versão 3.4: O método __format__ do próprio `object` levanta uma **`TypeError`** se passada qualquer string não vazia.

Alterado na versão 3.7: `object.__format__(x, '')` é agora equivalente a `str(x)` em vez de `format(str(x), '')`.

object.__lt__(*self*, *other*)
object.__le__(*self*, *other*)
object.__eq__(*self*, *other*)
object.__ne__(*self*, *other*)
object.__gt__(*self*, *other*)
object.__ge__(*self*, *other*)

Esses são os chamados métodos de "comparação rica". A correspondência entre os símbolos do operador e os nomes dos métodos é a seguinte: `x<y` chama `x.__lt__(y)`, `x<=y` chama `x.__le__(y)`, `x==y` chama `x.__eq__(y)`, `x!=y` chama `x.__ne__(y)`, `x>y` chama `x.__gt__(y)` e `x>=y` chama `x.__ge__(y)`.

Um método de comparação rica pode retornar o singleton **`NotImplemented`** se não implementar a operação para um determinado par de argumentos. Por convenção, `False` e `True` são retornados para uma comparação bem-sucedida. No entanto, esses métodos podem retornar qualquer valor, portanto, se o operador de comparação for usado em um contexto booleano (por exemplo, na condição de uma instrução `if`), Python irá chamar **`bool()`** no valor para determinar se o resultado for verdadeiro ou falso.

By default, `object` implements **`__eq__()`** by using `is`, returning **`NotImplemented`** in the case of a false comparison: `True if x is y else NotImplemented`. For **`__ne__()`**, by default it delegates to **`__eq__()`** and inverts the result unless it is **`NotImplemented`**. There are no other implied relationships among the comparison operators or default implementations; for example, the truth of `(x<y or x==y)` does not imply `x<=y`. To automatically generate ordering operations from a single root operation, see **`@functools.total_ordering`**.

Por padrão, a classe `object` fornece implementações consistentes com Comparações de valor: igualdade compara de acordo com a identidade do objeto, e comparações de ordem levantam `TypeError`. Cada método padrão pode gerar esses resultados diretamente, mas também pode retornar `NotImplemented`.

Veja o parágrafo sobre `__hash__()` para algumas notas importantes sobre a criação de objetos hasheáveis que implementam operações de comparação personalizadas e são utilizáveis como chaves de dicionário.

Não há versões de argumentos trocados desses métodos (a serem usados quando o argumento esquerdo não tem suporte à operação, mas o argumento direito sim); em vez disso, `__lt__()` e `__gt__()` são o reflexo um do outro, `__le__()` e `__ge__()` são o reflexo um do outro, e `__eq__()` e `__ne__()` são seu próprio reflexo. Se os operandos são de tipos diferentes e o tipo do operando direito é uma subclasse direta ou indireta do tipo do operando esquerdo, o método refletido do operando direito tem prioridade, caso contrário, o método do operando esquerdo tem prioridade. Subclasse virtual não é considerada.

Quando nenhum método apropriado retorna qualquer valor diferente de `NotImplemented`, os operadores `==` e `!=` retornarão para `is` e `is not`, respectivamente.

object.__hash__(*self*)

Called by built-in function `hash()` and for operations on members of hashed collections including `set`, `frozenset`, `dict`, and `frozendict`. The `__hash__()` method should return an integer. The only required property is that objects which compare equal have the same hash value; it is advised to mix together the hash values of the components of the object that also play a part in comparison of objects by packing them into a tuple and hashing the tuple. Example:

```
def __hash__(self):
    return hash((self.name, self.nick, self.color))
```

Nota

hash() trunca o valor retornado do método **__hash__()** personalizado de um objeto para o tamanho de um **Py_ssize_t**. Isso é normalmente 8 bytes em compilações de 64 bits e 4 bytes em compilações de 32 bits. Se o **__hash__()** de um objeto deve interoperar em compilações de tamanhos de bits diferentes, certifique-se de verificar a largura em todas as compilações com suporte. Uma maneira fácil de fazer isso é com `python -c "import sys; print(sys.hash_info.width)"`.

Se uma classe não define um método **__eq__()**, ela também não deve definir uma operação **__hash__()**; se define **__eq__()** mas não **__hash__()**, suas instâncias não serão utilizáveis como itens em coleções hasheáveis. Se uma classe define objetos mutáveis e implementa um método **__eq__()**, ela não deve implementar **__hash__()**, uma vez que a implementação de coleções hasheáveis requer que o valor hash de uma chave seja imutável (se o valor hash do objeto mudar, estará no balde de hash errado).

As classes definidas pelo usuário têm os métodos **__eq__()** e **__hash__()** por padrão (herdados da classe **object**); com eles, todos os objetos se comparam desiguais (exceto com eles mesmos) e `x.__hash__()` retorna um valor apropriado tal que `x == y` implica que `x is y` e `hash(x) == hash(y)`.

Uma classe que sobrescreve **__eq__()** e não define **__hash__()** terá seu **__hash__()** implicitamente definido como `None`. Quando o método **__hash__()** de uma classe é `None`, as instâncias da classe levantam uma **TypeError** apropriada quando um programa tenta recuperar seu valor hash, e também será identificado corretamente como não-hasheável ao verificar `isinstance(obj, collections.abc.Hashable)`.

Se uma classe que substitui **__eq__()** precisa manter a implementação de **__hash__()** de uma classe base, o interpretador deve ser informado disso explicitamente pela configuração `__hash__ = <ClasseBase>.__hash__`.

Se uma classe que não substitui **__eq__()** deseja suprimir o suporte a hash, deve incluir `__hash__ = None` na definição de classe. Uma classe

que define seu próprio **__hash__()** que levanta explicitamente uma **TypeError** seria incorretamente identificada como hasheável por uma chamada `isinstance(obj, collections.abc.Hashable)`.

Nota

Por padrão, os valores **__hash__()** dos objetos str e bytes são "salgados" com um valor aleatório imprevisível. Embora permaneçam constantes em um processo individual do Python, eles não são previsíveis entre invocações repetidas do Python.

This is intended to provide protection against a denial-of-service caused by carefully chosen inputs that exploit the worst case performance of a dict insertion, $O(n^2)$ complexity. See https://ocert.org/advisories/ocert-2011-003.html for details.

Alterar os valores de hash afeta a ordem de iteração dos conjuntos. Python nunca deu garantias sobre essa ordem (e normalmente varia entre compilações de 32 e 64 bits).

Consulte também **PYTHONHASHSEED**.

Alterado na versão 3.3: Aleatorização de hash está habilitada por padrão.

object.__bool__(*self*)

Chamado para implementar o teste de valor verdade e a operação embutida `bool()`; deve retornar `False` ou `True`. Quando este método não é definido, **__len__()** é chamado, se estiver definido, e o objeto é considerado verdadeiro se seu resultado for diferente de zero. Se uma classe não define **__len__()** nem **__bool__()** (o que é verdadeiro da classe **object** em si), todas as suas instâncias são consideradas verdadeiras.

3.3.2. Personalizando o acesso aos atributos

Os seguintes métodos podem ser definidos para personalizar o significado do acesso aos atributos (uso, atribuição ou exclusão de `x.name`) para instâncias de classe.

object.__getattr__(*self*, *name*)

Chamado quando o acesso padrão ao atributo falha com um **`AttributeError`** (ou **`__getattribute__()`** levanta uma **`AttributeError`** porque *name* não é um atributo de instância ou um atributo na árvore de classes para `self`; ou **`__get__()`** de uma propriedade *name* levanta **`AttributeError`**). Este método deve retornar o valor do atributo (calculado) ou levantar uma exceção **`AttributeError`**. A classe **`object`** em si não fornece este método.

Observe que se o atributo for encontrado através do mecanismo normal, **`__getattr__()`** não é chamado. (Esta é uma assimetria intencional entre **`__getattr__()`** e **`__setattr__()`**.) Isso é feito tanto por razões de eficiência quanto porque **`__getattr__()`** não teria como acessar outros atributos da instância. Observe que pelo menos para variáveis de instâncias, você pode obter controle total não inserindo nenhum valor no dicionário de atributos de instância (mas, em vez disso, inserindo-os em outro objeto). Veja o método **`__getattribute__()`** abaixo para uma maneira de realmente obter controle total sobre o acesso ao atributo.

object.__getattribute__(*self*, *name*)

Chamado incondicionalmente para implementar acessos a atributo para instâncias da classe. Se a classe também define **`__getattr__()`**, o último não será chamado a menos que **`__getattribute__()`** o chame explicitamente ou levante um **`AttributeError`**. Este método deve retornar o valor do atributo (calculado) ou levantar uma exceção **`AttributeError`**. Para evitar recursão infinita neste método, sua implementação deve sempre chamar o método da classe base com o mesmo nome para acessar quaisquer atributos de que necessita, por exemplo, `object.__getattribute__(self, name)`.

Nota

Este método ainda pode ser ignorado ao procurar métodos especiais como resultado de invocação implícita por meio da sintaxe da linguagem ou built-in functions. Consulte Pesquisa de método especial.

Para acessos a certos atributos sensíveis, levanta um evento de auditoria `object.__getattr__` com os argumentos `obj` e `name`.

object.__setattr__(*self*, *name*, *value*)

Chamado quando se tenta efetuar uma atribuição de atributos. Esse método é chamado em vez do mecanismo normal (ou seja, armazena o valor no dicionário da instância). *name* é o nome do atributo, *value* é o valor a ser atribuído a ele.

Se **`__setattr__()`** deseja atribuir a um atributo de instância, ele deve chamar o método da classe base com o mesmo nome, por exemplo, `object.__setattr__(self, name, value)`.

Para atribuições de certos atributos sensíveis, levanta um evento de auditoria `object.__setattr__` com os argumentos `obj`, `name` e `value`.

object.__delattr__(*self*, *name*)

Como **`__setattr__()`**, mas para exclusão de atributo em vez de atribuição. Este método só deve ser implementado se `del obj.name` for significativo para o objeto.

Para exclusões a certos atributos sensíveis, levanta um evento de auditoria `object.__delattr__` com os argumentos `obj` e `name`.

object.__dir__(*self*)

Chamado quando **`dir()`** é chamado com o objeto como argumento. Um iterável deve ser retornada. **`dir()`** converte o iterável retornado em uma lista e o ordena.

3.3.2.1. Personalizando acesso a atributos de módulos

module.__getattr__()
module.__dir__()

Os nomes especiais `__getattr__` e `__dir__` também podem ser usados para personalizar o acesso aos atributos dos módulos. A função `__getattr__` no nível do módulo deve aceitar um argumento que é o nome de um atributo e retornar o valor calculado ou levantar uma exceção **`AttributeError`**. Se um atributo não for encontrado em um objeto de módulo por meio da pesquisa normal, por exemplo **`object.__getattribute__()`**, então `__getattr__` é pesquisado no módulo `__dict__` antes de levantar **`AttributeError`**. Se encontrado, ele é chamado com o nome do atributo e o resultado é retornado.

A função `__dir__` não deve aceitar nenhum argumento e retorna um iterável de strings que representa os nomes acessíveis no módulo. Se presente, esta função substitui a pesquisa padrão **`dir()`** em um módulo.

module.__class__

Para uma personalização mais refinada do comportamento do módulo (definição de atributos, propriedades etc.), pode-se definir o atributo `__class__` de um objeto de módulo para uma subclasse de **`types.ModuleType`**. Por exemplo:

```
import sys
from types import ModuleType

class VerboseModule(ModuleType):
    def __repr__(self):
        return f'Verbose {self.__name__}'

    def __setattr__(self, attr, value):
        print(f'Setting {attr}...')
        super().__setattr__(attr, value)

sys.modules[__name__].__class__ = VerboseModule
```

Nota

Definir `__getattr__` no módulo e configurar o `__class__` do módulo só afeta as pesquisas feitas usando a sintaxe de acesso ao atributo – acessar diretamente os globais do módulo (seja por código dentro do módulo, ou por meio de uma referência ao dicionário global do módulo) não tem efeito.

Alterado na versão 3.5: O atributo de módulo `__class__` pode agora ser escrito.

Adicionado na versão 3.7: Atributos de módulo `__getattr__` e `__dir__`.

Ver também

PEP 562 [https://peps.python.org/pep-0562/] - __getattr__ e __dir__ de módulo
Descreve as funções `__getattr__` e `__dir__` nos módulos.

3.3.2.2. Implementando descritores

Os métodos a seguir se aplicam apenas quando uma instância da classe que contém o método (uma classe chamada *descritora*) aparece em uma classe proprietária *owner* (o descritor deve estar no dicionário de classe do proprietário ou no dicionário de classe para um dos seus pais). Nos exemplos abaixo, "o atributo" refere-se ao atributo cujo nome é a chave da propriedade no `__dict__` da classe proprietária. A classe `object` em si não implementa quaisquer desses protocolos.

object.__get__(*self*, *instance*, *owner=None*)

Chamado para obter o atributo da classe proprietária (acesso ao atributo da classe) ou de uma instância dessa classe (acesso ao atributo da instância). O argumento opcional *owner* é a classe proprietária, enquanto *instance* é a instância pela qual o atributo foi acessado, ou `None` quando o atributo é acessado por meio de *owner*.

Este método deve retornar o valor do atributo calculado ou levantar uma exceção `AttributeError`.

PEP 252 [https://peps.python.org/pep-0252/] especifica que `__get__()` é um chamável com um ou dois argumentos. Os próprios descritores embutidos do Python implementam esta especificação; no entanto, é provável que algumas ferramentas de terceiros tenham descritores que requerem ambos os argumentos. A implementação de `__getattribute__()` do próprio Python sempre passa em ambos os argumentos sejam eles requeridos ou não.

object.__set__(*self*, *instance*, *value*)

Chamado para definir o atributo em uma instância *instance* da classe proprietária para um novo valor, *value*.

Observe que adicionar `__set__()` ou `__delete__()` altera o tipo de descritor para um "descritor de dados". Consulte Invocando descritores para mais detalhes.

object.__delete__(*self*, *instance*)

Chamado para excluir o atributo em uma instância *instance* da classe proprietária.

Instâncias de descritores também podem ter o atributo `__objclass__` presente:

object.__objclass__

O atributo `__objclass__` é interpretado pelo módulo `inspect` como sendo a classe onde este objeto foi definido (configurar isso apropriadamente pode ajudar na introspecção em tempo de execução dos atributos dinâmicos da classe). Para chamáveis, pode indicar que uma instância do tipo fornecido (ou uma subclasse) é esperada ou necessária como o primeiro argumento posicional (por exemplo, CPython define este atributo para métodos não acoplados que são implementados em C).

3.3.2.3. Invocando descritores

Em geral, um descritor é um atributo de objeto com "comportamento de ligação", cujo acesso ao atributo foi substituído por métodos no protocolo

do descritor: `__get__()`, `__set__()` e `__delete__()`. Se qualquer um desses métodos for definido para um objeto, é considerado um descritor.

O comportamento padrão para acesso ao atributo é obter, definir ou excluir o atributo do dicionário de um objeto. Por exemplo, `a.x` tem uma cadeia de pesquisa começando com `a.__dict__['x']`, depois `type(a).__dict__['x']`, e continuando pelas classes bases de `type(a)` excluindo metaclasses.

No entanto, se o valor pesquisado for um objeto que define um dos métodos do descritor, Python pode substituir o comportamento padrão e invocar o método do descritor. Onde isso ocorre na cadeia de precedência depende de quais métodos descritores foram definidos e como eles foram chamados.

O ponto de partida para a invocação do descritor é uma ligação, `a.x`. Como os argumentos são montados depende de `a`:

Chamada direta
: A chamada mais simples e menos comum é quando o código do usuário invoca diretamente um método descritor: `x.__get__(a)`.

Ligação de instâncias
: Se estiver ligando a uma instância de objeto, `a.x` é transformado na chamada: `type(a).__dict__['x'].__get__(a, type(a))`.

Ligação de classes
: Se estiver ligando a uma classe, `A.x` é transformado na chamada: `A.__dict__['x'].__get__(None, A)`.

Ligação de super
: Uma pesquisa pontilhada, ou *dotted lookup*, como `super(A, a).x` procura `a.__class__.__mro__` por uma classe base `B` seguindo `A` e então retorna `B.__dict__['x'].__get__(a, A)`. Se não for um descritor, `x` é retornado inalterado.

Para ligações de instâncias, a precedência de invocação do descritor depende de quais métodos do descritor são definidos. Um descritor pode definir qualquer combinação de `__get__()`, `__set__()` e `__delete__()`. Se

ele não definir **`__get__()`**, então acessar o atributo retornará o próprio objeto descritor, a menos que haja um valor no dicionário de instância do objeto. Se o descritor define **`__set__()`** e/ou **`__delete__()`**, é um descritor de dados; se não definir nenhum, é um descritor sem dados. Normalmente, os descritores de dados definem **`__get__()`** e **`__set__()`**, enquanto os descritores sem dados têm apenas o método **`__get__()`**. Descritores de dados com **`__get__()`** e **`__set__()`** (e/ou **`__delete__()`**) definidos sempre substituem uma redefinição em um dicionário de instância. Em contraste, descritores sem dados podem ser substituídos por instâncias.

Os métodos Python (incluindo aqueles decorados com **`@staticmethod`** e **`@classmethod`**) são implementados como descritores sem dados. Assim, as instâncias podem redefinir e substituir métodos. Isso permite que instâncias individuais adquiram comportamentos que diferem de outras instâncias da mesma classe.

The **`@property`** decorator is implemented as a data descriptor. Accordingly, instances cannot override the behavior of a property.

3.3.2.4. __slots__

__slots__ permite-nos declarar explicitamente membros de dados (como propriedades) e negar a criação de **`__dict__`** e *__weakref__* (a menos que explicitamente declarado em *__slots__* ou disponível em uma classe base.)

O espaço economizado com o uso de **`__dict__`** pode ser significativo. A velocidade de pesquisa de atributos também pode ser significativamente melhorada.

object.__slots__

> Esta variável de classe pode ser atribuída a uma string, iterável ou sequência de strings com nomes de variáveis usados por instâncias. *__slots__* reserva espaço para as variáveis declaradas e evita a criação automática de **`__dict__`** e *__weakref__* para cada instância.

Observações ao uso de *__slots__*:

- Ao herdar de uma classe sem *__slots__*, os atributos `__dict__` e *__weakref__* das instâncias sempre estarão acessíveis.
- Sem uma variável `__dict__`, as instâncias não podem ser atribuídas a novas variáveis não listadas na definição *__slots__*. As tentativas de atribuir a um nome de variável não listado levantam `AttributeError`. Se a atribuição dinâmica de novas variáveis for desejada, então adicione `'__dict__'` à sequência de strings na declaração de *__slots__*.
- Sem uma variável *__weakref__* para cada instância, as classes que definem *__slots__* não suportam `referências fracas` para suas instâncias. Se for necessário um suporte de referência fraca, adicione `'__weakref__'` à sequência de strings na declaração *__slots__*.
- *__slots__* são implementados no nível de classe criando descritores para cada nome de variável. Como resultado, os atributos de classe não podem ser usados para definir valores padrão para variáveis de instância definidas por *__slots__*; caso contrário, o atributo de classe substituiria a atribuição do descritor.
- A ação de uma declaração *__slots__* se limita à classe em que é definida. *__slots__* declarados em uma classe base estão disponíveis nas subclasses. No entanto, instâncias de uma subclasse filha vai receber um `__dict__` e *__weakref__* a menos que a subclasse também defina *__slots__* (que deve conter apenas nomes de quaisquer slots *adicionais*).
- Se uma classe define um slot também definido em uma classe base, a variável de instância definida pelo slot da classe base fica inacessível (exceto por recuperar seu descritor diretamente da classe base). Isso torna o significado do programa indefinido. No futuro, uma verificação pode ser adicionada para evitar isso.
- `TypeError` will be raised if *__slots__* other than *__dict__* and *__weakref__* are defined for a class derived from a `"variable-length" built-in type` such as `int`, `bytes`, and `type`, except `tuple`.
- Qualquer iterável não string pode ser atribuído a *__slots__*.
- Se um `dicionário` for usado para atribuir *__slots__*, as chaves do dicionário serão usadas como os nomes dos slots. Os valores do dicionário podem ser usados para fornecer docstrings por atributo que serão reconhecidos por `inspect.getdoc()` e exibidos na saída de `help()`.

- Atribuição de `__class__` funciona apenas se ambas as classes têm o mesmo *__slots__*.
- Multiple inheritance with multiple slotted parent classes can be used, but only one parent is allowed to have attributes created by slots (the other bases must have empty slot layouts) - violations raise `TypeError`.
- Se um iterador for usado para *__slots__*, um descritor é criado para cada um dos valores do iterador. No entanto, o atributo *__slots__* será um iterador vazio.

Alterado na versão 3.15: Allowed defining the *__dict__* and *__weakref__* *__slots__* for any class. Allowed defining any *__slots__* for a class derived from `tuple`.

3.3.3. Personalizando a criação de classe

Sempre que uma classe herda de outra classe, `__init_subclass__()` é chamado na classe base. Dessa forma, é possível escrever classes que alteram o comportamento das subclasses. Isso está intimamente relacionado aos decoradores de classe, mas onde decoradores de classe afetam apenas a classe específica à qual são aplicados, `__init_subclass__` aplica-se apenas a futuras subclasses da classe que define o método.

classmethod object.__init_subclass__(*cls*)

Este método é chamado sempre que a classe que contém é uma subclasse. *cls* é então a nova subclasse. Se definido como um método de instância normal, esse método é convertido implicitamente em um método de classe.

Argumentos nomeados dados a uma nova classe são passados para `__init_subclass__` da classe base. Para compatibilidade com outras classes usando `__init_subclass__`, deve-se retirar os argumentos nomeados necessários e passar os outros para a classe base, como em:

```
class Philosopher:
    def __init_subclass__(cls, /, default_name, **kwargs):
        super().__init_subclass__(**kwargs)
        cls.default_name = default_name
```

```
class AustralianPhilosopher(Philosopher, default_name="Bruce'
    pass
```

A implementação padrão de `object.__init_subclass__` não faz nada, mas levanta um erro se for chamada com quaisquer argumentos.

Nota

A dica da metaclasse `metaclass` é consumida pelo resto da maquinaria de tipo, e nunca é passada para implementações `__init_subclass__`. A metaclasse real (em vez da dica explícita) pode ser acessada como `type(cls)`.

Adicionado na versão 3.6.

Quando uma classe é criada, **`type.__new__()`** verifica as variáveis de classe e faz retornos de chamada para elas com um gancho **`__set_name__()`**.

object.__set_name__(*self*, *owner*, *name*)

Chamado automaticamente no momento em que a classe proprietária *owner* é criada. O objeto foi atribuído a *name* nessa classe:

```
class A:
    x = C()  # Automaticamente chama: x.__set_name__(A, 'x')
```

Se a variável de classe for atribuída após a criação da classe, **`__set_name__()`** não será chamado automaticamente. Se necessário, **`__set_name__()`** pode ser chamado diretamente:

```
class A:
   pass

c = C()
A.x = c                  # O gancho não é chamado
c.__set_name__(A, 'x')   # Invoca manualmente o gancho
```

Consulte Criando o objeto classe para mais detalhes.

Adicionado na versão 3.6.

3.3.3.1. Metaclasses

Por padrão, as classes são construídas usando `type()`. O corpo da classe é executado em um novo espaço de nomes e o nome da classe é vinculado localmente ao resultado de `type(name, bases, namespace)`.

O processo de criação da classe pode ser personalizado passando o argumento nomeado `metaclass` na linha de definição da classe, ou herdando de uma classe existente que incluiu tal argumento. No exemplo a seguir, `MyClass` e `MySubclass` são instâncias de `Meta`:

```
class Meta(type):
    pass

class MyClass(metaclass=Meta):
    pass

class MySubclass(MyClass):
    pass
```

Quaisquer outros argumentos nomeados especificados na definição de classe são transmitidos para todas as operações de metaclasse descritas abaixo.

Quando uma definição de classe é executada, as seguintes etapas ocorrem:

- entradas de MRO são resolvidas;
- a metaclasse apropriada é determinada;
- o espaço de nomes da classe é preparada;
- o corpo da classe é executado;
- o objeto da classe é criado.

3.3.3.2. Resolvendo entradas de MRO

object.__mro_entries__(*self*, *bases*)

Se uma classe base que aparece em uma definição de classe não é uma instância de `type`, então um método **`__mro_entries__()`** é procurado na base. Se um método **`__mro_entries__()`** é encontrado, a base é

substituída pelo resultado de uma chamada para **`__mro_entries__()`** ao criar a classe. O método é chamado com a tupla de bases original passada como parâmetro *bases*, e deve retornar uma tupla de classes que serão usadas no lugar da base. A tupla retornada pode estar vazia: nesses casos, a base original é ignorada.

Ver também

`types.resolve_bases()`
Dinamicamente resolve bases que não são instâncias de `type`.

`types.get_original_bases()`
Recupera as "bases originais" de uma classe antes das modificações feitas por `__mro_entries__()`.

PEP 560 **[https://peps.python.org/pep-0560/]**
Suporte básico para módulo typing e tipos genéricos.

3.3.3.3. Determinando a metaclasse apropriada

A metaclasse apropriada para uma definição de classe é determinada da seguinte forma:

- se nenhuma classe base e nenhuma metaclasse explícita forem fornecidas, então `type()` é usada;
- se uma metaclasse explícita é fornecida e *não* é uma instância de `type()`, então ela é usada diretamente como a metaclasse;
- se uma instância de `type()` é fornecida como a metaclasse explícita, ou classes bases são definidas, então a metaclasse mais derivada é usada.

A metaclasse mais derivada é selecionada a partir da metaclasse explicitamente especificada (se houver) e das metaclasses (ou seja, `type(cls)`) de todas as classes bases especificadas. A metaclasse mais derivada é aquela que é um subtipo de *todas* essas metaclasses candidatas.

Se nenhuma das metaclasses candidatas atender a esse critério, a definição de classe falhará com `TypeError`.

3.3.3.4. Preparando o espaço de nomes da classe

Uma vez identificada a metaclasse apropriada, o espaço de nomes da classe é preparado. Se a metaclasse tiver um atributo `__prepare__`, ela será chamada como `namespace = metaclass.__prepare__(name, bases, **kwds)` (onde os argumentos nomeados adicionais, se houver, vêm da definição de classe). O método `__prepare__` deve ser implementado como um **`classmethod`**. O espaço de nomes retornado por `__prepare__` é passado para `__new__`, mas quando o objeto classe final é criado, o espaço de nomes é copiado para um novo `dict`.

Se a metaclasse não tiver o atributo `__prepare__`, então o espaço de nomes da classe é inicializado como um mapeamento ordenado vazio.

Ver também

PEP 3115 **[https://peps.python.org/pep-3115/]** **- Metaclasses no Python 3000**
Introduzido o gancho de espaço de nomes `__prepare__`

3.3.3.5. Executando o corpo da classe

O corpo da classe é executado (aproximadamente) como `exec(body, globals(), namespace)`. A principal diferença de uma chamada normal para **`exec()`** é que o escopo léxico permite que o corpo da classe (incluindo quaisquer métodos) faça referência a nomes dos escopos atual e externo quando a definição de classe ocorre dentro de uma função.

No entanto, mesmo quando a definição de classe ocorre dentro da função, os métodos definidos dentro da classe ainda não podem ver os nomes definidos no escopo da classe. Variáveis de classe devem ser acessadas através do primeiro parâmetro de instância ou métodos de classe, ou através

da referência implícita com escopo léxico `__class__` descrita na próxima seção.

3.3.3.6. Criando o objeto classe

Uma vez que o espaço de nomes da classe tenha sido preenchido executando o corpo da classe, o objeto classe é criado chamando `metaclass(name, bases, namespace, **kwds)` (os argumentos adicionais passados aqui são os mesmos passados para `__prepare__`).

Este objeto classe é aquele que será referenciado pela chamada a **`super()`** sem argumentos. `__class__` é uma referência de clausura implícita criada pelo compilador se algum método no corpo da classe se referir a `__class__` ou `super`. Isso permite que a forma de argumento zero de **`super()`** identifique corretamente a classe sendo definida com base no escopo léxico, enquanto a classe ou instância que foi usada para fazer a chamada atual é identificada com base no primeiro argumento passado para o método.

No CPython 3.6 e posterior, a célula `__class__` é passada para a metaclasse como uma entrada de `__classcell__` no espaço de nomes da classe. Se estiver presente, deve ser propagado até a chamada a `type.__new__` para que a classe seja inicializada corretamente. Não fazer isso resultará em um **`RuntimeError`** no Python 3.8.

Quando usada a metaclasse padrão **`type`**, ou qualquer metaclasse que chame `type.__new__`, as seguintes etapas de personalização adicionais são executadas depois da criação do objeto classe:

1. O método `type.__new__` coleta todos os atributos no espaço de nomes da classe que definem um método **`__set_name__()`**;
2. Esses métodos `__set_name__` são chamados com a classe sendo definida e o nome atribuído para este atributo específico;
3. O gancho **`__init_subclass__()`** é chamado na classe base imediata da nova classe em sua ordem de resolução de método.

Depois que o objeto classe é criado, ele é passado para os decoradores de classe incluídos na definição de classe (se houver) e o objeto resultante é

vinculado ao espaço de nomes local como a classe definida.

Quando uma nova classe é criada por `type.__new__`, o objeto fornecido como o parâmetro do espaço de nomes é copiado para um novo mapeamento ordenado e o objeto original é descartado. A nova cópia é envolta em um proxy de somente leitura, que se torna o atributo `__dict__` do objeto classe.

Ver também

PEP 3135 **[https://peps.python.org/pep-3135/]** - **Novo super**
Descreve a referência de clausura implícita de `__class__`

3.3.3.7. Usos para metaclasses

Os usos potenciais para metaclasses são ilimitados. Algumas ideias que foram exploradas incluem enumeradores, criação de log, verificação de interface, delegação automática, criação automática de propriedade, proxies, estruturas e travamento/sincronização automático/a de recursos.

3.3.4. Personalizando verificações de instância e subclasse

Os seguintes métodos são usados para substituir o comportamento padrão das funções embutidas `isinstance()` e `issubclass()`.

Em particular, a metaclasse `abc.ABCMeta` implementa esses métodos a fim de permitir a adição de classes base abstratas (ABCs) como "classes base virtuais" para qualquer classe ou tipo (incluindo tipos embutidos), incluindo outras ABCs.

type.__instancecheck__(*self*, *instance*)
Retorna verdadeiro se *instance* deve ser considerada uma instância (direta ou indireta) da classe *class*. Se definido, chamado para implementar `isinstance(instance, class)`.

type.__subclasscheck__(*self*, *subclass*)

Retorna verdadeiro se *subclass* deve ser considerada uma subclasse (direta ou indireta) da classe *class*. Se definido, chamado para implementar `issubclass(subclass, class)`.

Observe que esses métodos são pesquisados no tipo (metaclasse) de uma classe. Eles não podem ser definidos como métodos de classe na classe real. Isso é consistente com a pesquisa de métodos especiais que são chamados em instâncias, apenas neste caso a própria instância é uma classe.

Ver também

PEP 3119 **[https://peps.python.org/pep-3119/]** **- Introduzindo classes base abstratas**

Inclui a especificação para personalizar o comportamento de `isinstance()` e `issubclass()` através de `__instancecheck__()` e `__subclasscheck__()`, com motivação para esta funcionalidade no contexto da adição de classes base abstratas (veja o módulo `abc`) para a linguagem.

3.3.5. Emulando tipos genéricos

Quando estiver usando anotações de tipo, é frequentemente útil *parametrizar* um tipo genérico usando a notação de colchetes do Python. Por exemplo, a anotação `list[int]` pode ser usada para indicar uma `list` em que todos os seus elementos são do tipo `int`.

Ver também

PEP 484 **[https://peps.python.org/pep-0484/]** **- Dicas de tipo**

Apresenta a estrutura do Python para anotações de tipo

Tipos Generic Alias

Documentação de objetos que representam classes genéricas parametrizadas

Genéricos, genéricos definidos pelo usuário e `typing.Generic`

Documentação sobre como implementar classes genéricas que podem ser parametrizadas em tempo de execução e compreendidas por verificadores de tipo estático.

Uma classe pode *geralmente* ser parametrizada somente se ela define o método de classe especial `__class_getitem__()`.

classmethod object.__class_getitem__(*cls*, *key*)

Retorna um objeto que representa a especialização de uma classe genérica por argumentos de tipo encontrados em *key*.

Quando definido em uma classe, `__class_getitem__()` é automaticamente um método de classe. Assim, não é necessário que seja decorado com `@classmethod` quando de sua definição.

3.3.5.1. O propósito de *__class_getitem__*

O propósito de `__class_getitem__()` é permitir a parametrização em tempo de execução de classes genéricas da biblioteca padrão, a fim de aplicar mais facilmente dicas de tipo a essas classes.

Para implementar classes genéricas personalizadas que podem ser parametrizadas em tempo de execução e compreendidas por verificadores de tipo estáticos, os usuários devem herdar de uma classe da biblioteca padrão que já implementa `__class_getitem__()`, ou herdar de `typing.Generic`, que possui sua própria implementação de `__class_getitem__()`.

Implementações personalizadas de `__class_getitem__()` em classes definidas fora da biblioteca padrão podem não ser compreendidas por verificadores de tipo de terceiros, como o mypy. O uso de `__class_getitem__()` em qualquer classe para fins diferentes de dicas de tipo é desencorajado.

3.3.5.2. *__class_getitem__* versus *__getitem__*

Normalmente, a subscription de um objeto usando colchetes chamará o método de instância **`__getitem__()`** definido na classe do objeto. No entanto, se o objeto sendo subscrito for ele mesmo uma classe, o método de classe **`__class_getitem__()`** pode ser chamado em seu lugar. `__class_getitem__()` deve retornar um objeto GenericAlias se estiver devidamente definido.

Apresentado com a expressão `obj[x]`, o interpretador de Python segue algo parecido com o seguinte processo para decidir se **`__getitem__()`** ou **`__class_getitem__()`** deve ser chamado:

```
from inspect import isclass

def subscribe(obj, x):
    """Return the result of the expression 'obj[x]'"""

    class_of_obj = type(obj)

    # Se a classe de  obj define __getitem__,
    # chama class_of_obj.__getitem__(obj, x)
    if hasattr(class_of_obj, '__getitem__'):
        return class_of_obj.__getitem__(obj, x)

    # Do contrário, se obj for uma classe e define __class_getit
    # chama obj.__class_getitem__(x)
    elif isclass(obj) and hasattr(obj, '__class_getitem__'):
        return obj.__class_getitem__(x)

    # Do contrário, levanta uma exceção
    else:
        raise TypeError(
            f"'{class_of_obj.__name__}' object is not subscripta
        )
```

Em Python, todas as classes são elas mesmas instâncias de outras classes. A classe de uma classe é conhecida como metaclasse dessa classe, e a maioria das classes tem a classe **`type`** como sua metaclasse. **`type`** não define **`__getitem__()`**, o que significa que expressões como `list[int]`, `dict[str, float]` e `tuple[str, bytes]` resultam em chamadas para **`__class_getitem__()`**:

```
>>> # lista tem a classe "type" como sua metaclasse, como a maic
>>> type(list)
<class 'type'>
>>> type(dict) == type(list) == type(tuple) == type(str) == type
True
>>> # "list[int]" chama "list.__class_getitem__(int)"
>>> list[int]
list[int]
>>> # list.__class_getitem__ retorna um objeto GenericAlias:
>>> type(list[int])
<class 'types.GenericAlias'>
```

No entanto, se uma classe tiver uma metaclasse personalizada que define `__getitem__()`, subscrever a classe pode resultar em comportamento diferente. Um exemplo disso pode ser encontrado no módulo `enum`:

```
>>> from enum import Enum
>>> class Menu(Enum):
...     """A breakfast menu"""
...     SPAM = 'spam'
...     BACON = 'bacon'
...
>>> # Classes Enum têm uma metaclasses personalizada:
>>> type(Menu)
<class 'enum.EnumMeta'>
>>> # EnumMeta define __getitem__,
>>> # de forma que __class_getitem__ não é chamado,
>>> # e o resultado não é um objeto GenericAlias:
>>> Menu['SPAM']
<Menu.SPAM: 'spam'>
>>> type(Menu['SPAM'])
<enum 'Menu'>
```

Ver também

PEP 560 [https://peps.python.org/pep-0560/] - Suporte básico para módulo typing e tipos genéricos

Introduz `__class_getitem__()`, e define quando uma subscrição resulta na chamada de `__class_getitem__()` em vez de `__getitem__()`

3.3.6. Emulando objetos chamáveis

object.__call__(*self*[, *args*...])

Chamado quando a instância é "chamada" como uma função; se este método for definido, `x(arg1, arg2, ...)` basicamente traduz para `type(x).__call__(x, arg1, ...)`. A classe **`object`** em si não fornece este método.

3.3.7. Emulando tipos contêineres

Os seguintes métodos podem ser definidos para implementar objetos contêineres. Nenhum deles é fornecido pela própria classe **`object`**. Contêineres geralmente são sequências (como **`listas`** ou **`tuplas`**) ou mapeamentos (como dicionários), mas também podem representar outros contêineres. O primeiro conjunto de métodos é usado para emular uma sequência ou um mapeamento; a diferença é que, para uma sequência, as chaves permitidas devem ser os inteiros *k* para os quais `0 <= k < N`, onde *N* é o comprimento da sequência, ou objetos **`fatiamentos`**, que definem um intervalo de itens. Também é recomendado que os mapeamentos forneçam os métodos **`keys()`**, **`values()`**, **`items()`**, **`get()`**, **`clear()`**, **`setdefault()`**, **`pop()`**, **`popitem()`**, **`copy()`** e **`update()`** se comportando de forma semelhante aos dos objetos **`dicionário`** padrão do Python. O módulo **`collections.abc`** fornece uma classe base abstrata **`MutableMapping`** para ajudar a criar esses métodos a partir de um conjunto base de **`__getitem__()`**, **`__setitem__()`**, **`__delitem__()`** e **`keys()`**.

Sequências mutáveis devem fornecer métodos **`append()`**, **`clear()`**, **`count()`**, **`extend()`**, **`index()`**, **`insert()`**, **`pop()`**, **`remove()`** e **`reverse()`**, como objetos **`list`** padrão do Python. Por fim, os tipos de sequência devem implementar adição (ou seja, concatenação) e multiplicação (ou seja, repetição) definindo os métodos **`__add__()`**, **`__radd__()`**, **`__iadd__()`**, **`__mul__()`**, **`__rmul__()`** e **`__imul__()`** descritos abaixo; eles não devem definir outros operadores numéricos.

Recomenda-se que tanto mapeamentos quanto sequências implementem o método `__contains__()` para permitir o uso eficiente do operador `in`; para mapeamentos, `in` deve pesquisar as chaves do mapeamento; para sequências, deve pesquisar os valores. Recomenda-se ainda que tanto mapeamentos quanto sequências implementem o método `__iter__()` para permitir uma iteração eficiente pelo contêiner; para mapeamentos, **`__iter__()`** deve iterar pelas chaves do objeto; para sequências, deve iterar pelos valores.

object.__len__(*self*)

Chamado para implementar a função embutida `len()`. Deve retornar o comprimento do objeto, um inteiro >= 0. Além disso, um objeto que não define um método `__bool__()` e cujo método **`__len__()`** retorna zero é considerado como falso em um contexto booleano.

No CPython, o comprimento deve ser no máximo `sys.maxsize`. Se o comprimento for maior que **`sys.maxsize`**, alguns recursos (como `len()`) podem levantar `OverflowError`. Para evitar levantar **`OverflowError`** pelo teste de valor de verdade, um objeto deve definir um método `__bool__()`.

object.__length_hint__(*self*)

Chamado para implementar `operator.length_hint()`. Deve retornar um comprimento estimado para o objeto (que pode ser maior ou menor que o comprimento real). O comprimento deve ser um inteiro >= 0. O valor de retorno também pode ser `NotImplemented`, que é tratado da mesma forma como se o método `__length_hint__` não existisse. Este método é puramente uma otimização e nunca é necessário para a correção.

Adicionado na versão 3.4.

object.__getitem__(*self*, *subscript*)

Called to implement *subscription*, that is, `self[subscript]`. See Subscriptions and slicings for details on the syntax.

There are two types of built-in objects that support subscription via **`__getitem__()`**:

- **sequences**, where *subscript* (also called index) should be an integer or a **`slice`** object. See the sequence documentation for the expected behavior, including handling **`slice`** objects and negative indices.
- **mappings**, where *subscript* is also called the key. See mapping documentation for the expected behavior.

If *subscript* is of an inappropriate type, **`__getitem__()`** should raise **`TypeError`**. If *subscript* has an inappropriate value, **`__getitem__()`** should raise an **`LookupError`** or one of its subclasses (**`IndexError`** for sequences; **`KeyError`** for mappings).

Nota

Slicing is handled by **`__getitem__()`**, **`__setitem__()`**, and **`__delitem__()`**. A call like

```
a[1:2] = b
```

é traduzida com

```
a[slice(1, 2, None)] = b
```

and so forth. Missing slice items are always filled in with `None`.

Nota

The sequence iteration protocol (used, for example, in **`for`** loops), expects that an **`IndexError`** will be raised for illegal indexes to allow proper detection of the end of a sequence.

Nota

When subscripting a *class*, the special class method **`__class_getitem__()`** may be called instead of **`__getitem__()`**. See __class_getitem__ versus __getitem__ for more details.

object.__setitem__(*self*, *key*, *value*)

Chamado para implementar a atribuição de `self[key]`. Mesma nota que para **`__getitem__()`**. Isso só deve ser implementado para mapeamentos se os objetos suportarem alterações nos valores das chaves, ou se novas chaves puderem ser adicionadas, ou para sequências se os elementos puderem ser substituídos. As mesmas exceções devem ser levantadas para valores *key* impróprios do método **`__getitem__()`**.

object.__delitem__(*self*, *key*)

Chamado para implementar a exclusão de `self[key]`. Mesma nota que para **`__getitem__()`**. Isso só deve ser implementado para mapeamentos se os objetos suportarem remoções de chaves, ou para sequências se os elementos puderem ser removidos da sequência. As mesmas exceções devem ser levantadas para valores *key* impróprios do método **`__getitem__()`**.

object.__missing__(*self*, *key*)

Chamado por **`dict.__getitem__()`** para implementar `self[key]` para subclasses de dicionário quando a chave não estiver no dicionário.

object.__iter__(*self*)

Este método é chamado quando um iterador é necessário para um contêiner. Este método deve retornar um novo objeto iterador que pode iterar sobre todos os objetos no contêiner. Para mapeamentos, ele deve iterar sobre as chaves do contêiner.

object.__reversed__(*self*)

Chamado (se presente) pelo **`reversed()`** embutido para implementar a iteração reversa. Ele deve retornar um novo objeto iterador que itera sobre todos os objetos no contêiner na ordem reversa.

Se o método **`__reversed__()`** não for fornecido, o **`reversed()`** embutido voltará a usar o protocolo de sequência (**`__len__()`** e **`__getitem__()`**). Objetos que suportam o protocolo de sequência só devem fornecer **`__reversed__()`** se eles puderem fornecer uma

implementação que seja mais eficiente do que aquela fornecida por `reversed()`.

Os operadores de teste de associação (`in` e `not in`) são normalmente implementados como uma iteração através de um contêiner. No entanto, os objetos contêiner podem fornecer o seguinte método especial com uma implementação mais eficiente, que também não requer que o objeto seja iterável.

object.__contains__(*self*, *item*)

Chamado para implementar operadores de teste de associação. Deve retornar verdadeiro se *item* estiver em *self*, falso caso contrário. Para objetos de mapeamento, isso deve considerar as chaves do mapeamento em vez dos valores ou pares de itens-chave.

Para objetos que não definem `__contains__()`, o teste de associação primeiro tenta a iteração via `__iter__()`, depois o protocolo de iteração de sequência antigo via `__getitem__()`, consulte esta seção em a referência da linguagem.

3.3.8. Emulando tipos numéricos

Os métodos a seguir podem ser definidos para emular objetos numéricos. Métodos correspondentes a operações que não são suportadas pelo tipo particular de número implementado (por exemplo, operações bit a bit para números não inteiros) devem ser deixados indefinidos.

object.__add__(*self*, *other*)
object.__sub__(*self*, *other*)
object.__mul__(*self*, *other*)
object.__matmul__(*self*, *other*)
object.__truediv__(*self*, *other*)
object.__floordiv__(*self*, *other*)
object.__mod__(*self*, *other*)
object.__divmod__(*self*, *other*)

object.__pow__(*self*, *other*[, *modulo*])
object.__lshift__(*self*, *other*)
object.__rshift__(*self*, *other*)
object.__and__(*self*, *other*)
object.__xor__(*self*, *other*)
object.__or__(*self*, *other*)

Esses métodos são chamados para implementar as operações aritméticas binárias (`+`, `-`, `*`, `@`, `/`, `//`, `%`, **`divmod()`**, **`pow()`**, `**`, `<<`, `>>`, `&`, `^`, `|`). Por exemplo, para avaliar a expressão `x + y`, onde *x* é uma instância de uma classe que tem um método **`__add__()`**, `type(x).__add__(x, y)` é chamado. O método **`__divmod__()`** deve ser equivalente a usar **`__floordiv__()`** e **`__mod__()`**; não deve estar relacionado a **`__truediv__()`**. Note que **`__pow__()`** deve ser definido para aceitar um terceiro argumento opcional se a versão de três argumentos da função embutida **`pow()`** for suportada.

Se um desses métodos não suporta a operação com os argumentos fornecidos, ele deve retornar **`NotImplemented`**.

object.__radd__(*self*, *other*)
object.__rsub__(*self*, *other*)
object.__rmul__(*self*, *other*)
object.__rmatmul__(*self*, *other*)
object.__rtruediv__(*self*, *other*)
object.__rfloordiv__(*self*, *other*)
object.__rmod__(*self*, *other*)
object.__rdivmod__(*self*, *other*)
object.__rpow__(*self*, *other*[, *modulo*])
object.__rlshift__(*self*, *other*)
object.__rrshift__(*self*, *other*)
object.__rand__(*self*, *other*)
object.__rxor__(*self*, *other*)
object.__ror__(*self*, *other*)

Esses métodos são chamados para implementar as operações aritméticas binárias (`+`, `-`, `*`, `@`, `/`, `//`, `%`, `divmod()`, `pow()`, `**`, `<<`, `>>`, `&`, `^`, `|`) com operandos refletidos (trocados). Essas funções são chamadas apenas se os operandos tiverem tipos diferentes, quando o operando esquerdo não oferecer suporte a operação correspondente [[3]], ou a classe do operando direito é derivada da classe do operando esquerdo. [[4]] Por exemplo, para avaliar a expressão `x - y`, onde *y* é uma instância de uma classe que tem um método `__rsub__()`, `type(y).__rsub__(y, x)` é chamado se `type(x).__sub__(x, y)` retorna `NotImplemented` ou `type(y)` é uma subclasse de `type(x)`. [[5]]

Observe que `__rpow__()` deve ser definido para aceitar um terceiro argumento opcional se a versão de três argumentos da função embutida `pow()` for suportada.

Alterado na versão 3.14: `pow()` de três argumentos agora tenta chamar `__rpow__()` se necessário. Anteriormente, ele só era chamado em **`pow()`** de dois argumentos e no operador binário de potência.

Nota

Se o tipo do operando direito for uma subclasse do tipo do operando esquerdo e essa subclasse fornecer uma implementação diferente do método refletido para a operação, este método será chamado antes do método não refletido do operando esquerdo. Esse comportamento permite que as subclasses substituam as operações de seus ancestrais.

object.__iadd__(*self*, *other*)

object.__isub__(*self*, *other*)

object.__imul__(*self*, *other*)

object.__imatmul__(*self*, *other*)

object.__itruediv__(*self*, *other*)

object.__ifloordiv__(*self*, *other*)

object.__imod__(*self*, *other*)

object.__ipow__(*self*, *other*[, *modulo*])

object.__ilshift__(*self*, *other*)
object.__irshift__(*self*, *other*)
object.__iand__(*self*, *other*)
object.__ixor__(*self*, *other*)
object.__ior__(*self*, *other*)

Esses métodos são chamados para implementar as atribuições aritméticas aumentadas (`+=`, `-=`, `*=`, `@=`, `/=`, `//=`, `%=`, `**=`, `<<=`, `>>=`, `&=`, `^=`, `|=`). Esses métodos devem tentar fazer a operação no local (modificando *self*) e retornar o resultado (que poderia ser, mas não precisa ser, *self*). Se um método específico não for definido, ou se esse método retorna **`NotImplemented`**, a atribuição aumentada volta aos métodos normais. Por exemplo, se *x* é uma instância de uma classe com um método **`__iadd__()`**, `x += y` equivale a `x = x.__iadd__(y)`. Se **`__iadd__()`** não existe, ou se `x.__iadd__(y)` retorna **`NotImplemented`**, `x.__add__(y)` e `y.__radd__(x)` são considerados, como com a avaliação de `x + y`. Em certas situações, a atribuição aumentada pode resultar em erros inesperados (veja Porque uma tupla[i] += ['item'] levanta uma exceção quando a adição funciona?), mas este comportamento é na verdade parte do modelo de dados.

object.__neg__(*self*)
object.__pos__(*self*)
object.__abs__(*self*)
object.__invert__(*self*)

Chamado para implementar as operações aritméticas unárias (`-`, `+`, **`abs()`** e `~`).

object.__complex__(*self*)
object.__int__(*self*)
object.__float__(*self*)

Chamado para implementar as funções embutidas **`complex()`**, **`int()`** e **`float()`**. Deve retornar um valor do tipo apropriado.

object.__index__(*self*)

Chamado para implementar `operator.index()`, e sempre que o Python precisar converter sem perdas o objeto numérico em um objeto inteiro (como no fatiamento ou nas funções embutidas `bin()`, `hex()` e `oct()`). A presença deste método indica que o objeto numérico é do tipo inteiro. Deve retornar um número inteiro.

Se `__int__()`, `__float__()` e `__complex__()` não estiverem definidos, funções embutidas correspondentes `int()`, `float()` e `complex()` recorre a `__index__()`.

object.__round__(*self*[, *ndigits*])
object.__trunc__(*self*)
object.__floor__(*self*)
object.__ceil__(*self*)

Chamado para implementar as funções embutidas `round()` e `trunc()`, `floor()` e `ceil()` de `math`. A menos que *ndigits* sejam passados para `__round__()` todos estes métodos devem retornar o valor do objeto truncado para um `Integral` (tipicamente um `int`).

Alterado na versão 3.14: `int()` não delega mais ao método `__trunc__()`.

3.3.9. Gerenciadores de contexto da instrução with

Um *gerenciador de contexto* é um objeto que define o contexto de tempo de execução a ser estabelecido ao executar uma instrução `with`. O gerenciador de contexto lida com a entrada e a saída do contexto de tempo de execução desejado para a execução do bloco de código. Os gerenciadores de contexto são normalmente invocados usando a instrução `with` (descrita na seção A instrução with), mas também podem ser usados invocando diretamente seus métodos.

Os usos típicos de gerenciadores de contexto incluem salvar e restaurar vários tipos de estado global, travar e destravar recursos, fechar arquivos abertos, etc.

Para mais informações sobre gerenciadores de contexto, veja Tipos de Gerenciador de Contexto. A classe `object` em si não fornece os métodos do gerenciador de contexto.

object.__enter__(*self*)

Insere o contexto de tempo de execução relacionado a este objeto. A instrução `with` vinculará o valor de retorno deste método ao(s) alvo(s) especificado(s) na cláusula `as` da instrução, se houver.

object.__exit__(*self*, *exc_type*, *exc_value*, *traceback*)

Sai do contexto de tempo de execução relacionado a este objeto. Os parâmetros descrevem a exceção que fez com que o contexto fosse encerrado. Se o contexto foi encerrado sem exceção, todos os três argumentos serão `None`.

Se uma exceção for fornecida e o método desejar suprimir a exceção (ou seja, evitar que ela seja propagada), ele deve retornar um valor verdadeiro. Caso contrário, a exceção será processada normalmente ao sair deste método.

Observe que os métodos `__exit__()` não devem relançar a exceção passada; esta é a responsabilidade do chamador.

Ver também

PEP 343 [https://peps.python.org/pep-0343/] - A instrução "with"

A especificação, o histórico e os exemplos para a instrução Python `with`.

3.3.10. Customizando argumentos posicionais na classe correspondência de padrão

Ao usar um nome de classe em um padrão, argumentos posicionais não são permitidos por padrão, ou seja, `case MyClass(x, y)` é tipicamente inválida

sem suporte especial em `MyClass`. Para permitir a utilização desse tipo de padrão, a classe precisa definir um atributo *__match_args__*

object.__match_args__

Essa variável de classe pode ser atribuída a uma tupla de strings. Quando essa classe é usada em uma classe padrão com argumentos posicionais, cada argumento posicional será convertido para um argumento nomeado, usando correspondência de valor em *__match_args__* como palavra reservada. A ausência desse atributo é equivalente a defini-lo como `()`

Por exemplo, se `MyClass.__match_args__` é `("left", "center", "right")` significa que `case MyClass(x, y)` é equivalente a `case MyClass(left=x, center=y)`. Note que o número de argumentos no padrão deve ser menor ou igual ao número de elementos em *__match_args__*; caso seja maior, a tentativa de correspondência de padrão irá levantar uma `TypeError`.

Adicionado na versão 3.10.

Ver também

PEP 634 [https://peps.python.org/pep-0634/] - Correspondência de Padrão Estrutural

A especificação para a instrução Python `match`

3.3.11. Emulando tipos buffer

O protocolo buffer fornece uma maneira para objetos Python exporem acesso eficiente a um vetor de memória de baixo nível. Este protocolo é implementado por tipos embutido como `bytes` e `memoryview`, e bibliotecas de terceiros podem definir tipos de buffer adicionais.

Embora os tipos buffer sejam geralmente implementados em C, também é possível implementar o protocolo em Python.

object.__buffer__(*self*, *flags*)

Chamado quando um buffer é solicitado de *self* (por exemplo, pelo construtor de `memoryview`). O argumento *flags* é um inteiro que representa o tipo de buffer solicitado, afetando, por exemplo, se o buffer retornado é somente leitura ou gravável. `inspect.BufferFlags` fornece uma maneira conveniente de interpretar os sinalizadores. O método deve retornar um objeto `memoryview`.

Thread safety: In free-threaded Python, implementations must manage any internal export counter using atomic operations. The method must be safe to call concurrently from multiple threads, and the returned buffer's underlying data must remain valid until the corresponding `__release_buffer__()` call completes. See Segurança para thread para objetos memoryview for details.

object.__release_buffer__(*self*, *buffer*)

Called when a buffer is no longer needed. The *buffer* argument is a `memoryview` object that was previously returned by `__buffer__()`. The method must release any resources associated with the buffer. This method should return `None`.

Thread safety: In free-threaded Python, any export counter decrement must use atomic operations. Resource cleanup must be thread-safe, as the final release may race with concurrent releases from other threads.

Buffer objects that do not need to perform any cleanup are not required to implement this method.

Adicionado na versão 3.12.

Ver também

PEP 688 [https://peps.python.org/pep-0688/] - Tornando o protocolo de buffer acessível no Python

Introduz os métodos `__buffer__` e `__release_buffer__` no Python

`collections.abc.Buffer`

ABC para tipos buffer.

3.3.12. Anotações

Funções, classes e módulos podem conter anotações, que são uma maneira de associar informações (geralmente dicas de tipo) a um símbolo.

object.__annotations__

Este atributo contém as anotações de um objeto. Ele é avaliado preguiçosamente, portanto, acessá-lo pode executar código arbitrário e levantar exceções. Se a avaliação for bem-sucedida, o atributo é definido como um mapeamento de dicionário de nomes de variáveis para anotações.

Alterado na versão 3.14: Anotações são agora avaliados preguiçosamente.

object.__annotate__(*format*)

Uma função de anotação. Retorna um novo objeto dicionário mapeando nomes de atributos/parâmetros aos seus valores de anotação.

Recebe um parâmetro de formato que especifica o formato em que os valores das anotações devem ser fornecidos. Deve ser um membro da enumeração `annotationlib.Format` ou um inteiro com um valor correspondente a um membro da enumeração.

Se uma função de anotação não oferecer suporte ao formato solicitado, ela deverá levantar `NotImplementedError`. Funções de anotação devem sempre oferecer suporte ao formato `VALUE`; elas não devem levantar `NotImplementedError()` quando chamadas com este formato.

Quando chamada com o formato `VALUE`, uma função de anotação pode levantar `NameError`; ela não deve levantar `NameError` quando chamada solicitando qualquer outro formato.

Se um objeto não tiver nenhuma anotação, `__annotate__` deve ser preferencialmente definido como `None` (não pode ser excluído), em vez de ser definido como uma função que retorna um dicionário vazio.

Adicionado na versão 3.14.

Ver também

PEP 649 [https://peps.python.org/pep-0649/] — Avaliação diferida de anotação usando descritores

Apresenta a avaliação preguiçosa de anotações e a função `__annotate__`.

3.3.13. Pesquisa de método especial

Para classes personalizadas, as invocações implícitas de métodos especiais só têm garantia de funcionar corretamente se definidas em um tipo de objeto, não no dicionário de instância do objeto. Esse comportamento é o motivo pelo qual o código a seguir levanta uma exceção:

```
>>> class C:
...     pass
...
>>> c = C()
>>> c.__len__ = lambda: 5
>>> len(c)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: object of type 'C' has no len()
```

A justificativa por trás desse comportamento está em uma série de métodos especiais como `__hash__()` e `__repr__()` que são implementados por todos os objetos, incluindo objetos de tipo. Se a pesquisa implícita desses métodos usasse o processo de pesquisa convencional, eles falhariam quando invocados no próprio objeto do tipo:

```
>>> 1 .__hash__() == hash(1)
True
```

```
>>> int.__hash__() == hash(int)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: descriptor '__hash__' of 'int' object needs an argume
```

A tentativa incorreta de invocar um método não vinculado de uma classe dessa maneira é às vezes referida como “confusão de metaclasse” e é evitada ignorando a instância ao pesquisar métodos especiais:

```
>>> type(1).__hash__(1) == hash(1)
True
>>> type(int).__hash__(int) == hash(int)
True
```

Além de contornar quaisquer atributos de instância no interesse da correção, a pesquisa de método especial implícita geralmente também contorna o método **`__getattribute__()`** mesmo da metaclasse do objeto:

```
>>> class Meta(type):
...     def __getattribute__(*args):
...         print("Metaclass getattribute invoked")
...         return type.__getattribute__(*args)
...
>>> class C(object, metaclass=Meta):
...     def __len__(self):
...         return 10
...     def __getattribute__(*args):
...         print("Class getattribute invoked")
...         return object.__getattribute__(*args)
...
>>> c = C()
>>> c.__len__()                 # Explicit lookup via instance
Class getattribute invoked
10
>>> type(c).__len__(c)          # Explicit lookup via type
Metaclass getattribute invoked
10
>>> len(c)                      # Implicit lookup
10
```

Ignorar a maquinaria de `__getattribute__()` desta forma fornece um escopo significativo para otimizações de velocidade dentro do interpretador, ao custo de alguma flexibilidade no tratamento de métodos especiais (o método especial *deve* ser definido no próprio objeto classe em ordem ser invocado de forma consistente pelo interpretador).

3.4. Corrotinas

3.4.1. Objetos aguardáveis

Um objeto aguardável geralmente implementa um método `__await__()`. Os objetos corrotina retornados das funções `async def` são aguardáveis.

Nota

Os objetos iteradores geradores retornados de geradores decorados com `types.coroutine()` também são aguardáveis, mas eles não implementam `__await__()`.

object.__await__(*self*)

Deve retornar um iterador. Deve ser usado para implementar objetos aguardáveis. Por exemplo, `asyncio.Future` implementa este método para ser compatível com a expressão `await`. A classe `object` em si não é aguardável e não fornece este método.

Nota

A linguagem não impõe nenhuma restrição ao tipo ou valor dos objetos produzidos pelo iterador retornado por `__await__`, pois isso é específico para a implementação da estrutura de execução assíncrona (por exemplo, `asyncio`) que gerenciará o objeto awaitable.

Adicionado na versão 3.5.

Ver também

PEP 492 [https://peps.python.org/pep-0492/] para informações adicionais sobre objetos aguardáveis.

3.4.2. Objetos corrotina

Objetos corrotina são objetos aguardáveis. A execução de uma corrotina pode ser controlada chamando `__await__()` e iterando sobre o resultado. Quando a corrotina termina a execução e retorna, o iterador levanta `StopIteration`, e o atributo `value` da exceção contém o valor de retorno. Se a corrotina levantar uma exceção, ela será propagada pelo iterador. As corrotinas não devem levantar exceções `StopIteration` diretamente não tratadas.

As corrotinas também têm os métodos listados abaixo, que são análogos aos dos geradores (ver Métodos de iterador gerador). No entanto, ao contrário dos geradores, as corrotinas não suportam diretamente a iteração.

Coroutines are generic over the types of their yield, send, and return values, respectively.

Alterado na versão 3.5.2: É uma `RuntimeError` para aguardar uma corrotina mais de uma vez.

coroutine.send(*value*)

Inicia ou retoma a execução da corrotina. Se *value* for `None`, isso equivale a avançar o iterador retornado por `__await__()`. Se *value* não for `None`, este método delega para o método `send()` do iterador que causou a suspensão da corrotina. O resultado (valor de retorno, `StopIteration` ou outra exceção) é o mesmo de iterar sobre o valor de retorno `__await__()`, descrito acima.

coroutine.throw(*value*)

coroutine.throw(*type*[, *value*[, *traceback*]])

Levanta a exceção especificada na corrotina. Este método delega ao método `throw()` do iterador que causou a suspensão da corrotina, se ela tiver tal método. Caso contrário, a exceção é levantada no ponto de suspensão. O resultado (valor de retorno, `StopIteration` ou outra exceção) é o mesmo de iterar sobre o valor de retorno `__await__()`, descrito acima. Se a exceção não for capturada na corrotina, ela se propagará de volta para o chamador.

Alterado na versão 3.12: A segunda assinatura (tipo[, valor[, traceback]]]) foi descontinuada e pode ser removida em uma versão futura do Python.

coroutine.close()

Faz com que a corrotina se limpe e saia. Se a corrotina for suspensa, este método primeiro delega para o método `close()` do iterador que causou a suspensão da corrotina, se tiver tal método. Então ele levanta `GeneratorExit` no ponto de suspensão, fazendo com que a corrotina se limpe imediatamente. Por fim, a corrotina é marcada como tendo sua execução concluída, mesmo que nunca tenha sido iniciada.

Objetos corrotina são fechados automaticamente usando o processo acima quando estão prestes a ser destruídos.

3.4.3. Iteradores assíncronos

Um *iterador assíncrono* pode chamar código assíncrono em seu método `__anext__`.

Os iteradores assíncronos podem ser usados em uma instrução `async for`.

A classe `object` em si não fornece estes métodos.

object.__aiter__(*self*)

Deve retornar um objeto *iterador assíncrono*.

object.__anext__(*self*)

Deve retornar um *aguardável* resultando em um próximo valor do iterador. Deve levantar um erro **`StopAsyncIteration`** quando a iteração terminar.

Um exemplo de objeto iterável assíncrono:

```
class Reader:
    async def readline(self):
        ...

    def __aiter__(self):
        return self

    async def __anext__(self):
        val = await self.readline()
        if val == b'':
            raise StopAsyncIteration
        return val
```

Adicionado na versão 3.5.

Alterado na versão 3.7: Antes do Python 3.7, **`__aiter__()`** poderia retornar um *aguardável* que resolveria para um iterador assíncrono.

A partir do Python 3.7, **`__aiter__()`** deve retornar um objeto iterador assíncrono. Retornar qualquer outra coisa resultará em um erro **`TypeError`**.

3.4.4. Gerenciadores de contexto assíncronos

Um *gerenciador de contexto assíncrono* é um *gerenciador de contexto* que é capaz de suspender a execução em seus métodos `__aenter__` e `__aexit__`.

Os gerenciadores de contexto assíncronos podem ser usados em uma instrução **`async with`**.

A classe **`object`** em si não fornece estes métodos.

object.__aenter__(*self*)

Semanticamente semelhante a `__enter__()`, a única diferença é que ele deve retornar um *aguardável*.

object.__aexit__(*self*, *exc_type*, *exc_value*, *traceback*)

Semanticamente semelhante a `__exit__()`, a única diferença é que ele deve retornar um *aguardável*.

Um exemplo de uma classe gerenciadora de contexto assíncrona:

```
class AsyncContextManager:
    async def __aenter__(self):
        await log('entrando no contexto')

    async def __aexit__(self, exc_type, exc, tb):
        await log('saindo do contexto')
```

Adicionado na versão 3.5.

Notas de rodapé

[1]

Em alguns casos, *é* possível alterar o tipo de um objeto, sob certas condições controladas. No entanto, geralmente não é uma boa ideia, pois pode levar a um comportamento muito estranho se for tratado incorretamente.

[2]

Os métodos `__hash__()`, `__iter__()`, `__reversed__()` e `__contains__()`, `__class_getitem__()` e `__fspath__()` têm um tratamento especial para isso. Outros ainda irão levantar um `TypeError`, mas podem fazer isso contando com o comportamento de que `None` não é chamável.

[3]

“Não suportar” aqui significa que a classe não possui tal método, ou o método retorna `NotImplemented`. Não defina o método como `None` se quiser

forçar o fallback para o método refletido do operando correto – isso terá o efeito oposto de *bloquear* explicitamente esse fallback.

[4]

Para operandos do mesmo tipo, presume-se que se o método não refletido (como `__add__()`) falhar, a operação não será suportada, razão pela qual o método refletido não é chamado.

[5]

Se o tipo do operando direito for uma subclasse do tipo do operando esquerdo, o método tendo precedência permite que as subclasses substituam as operações de seus ancestrais.

4. Modelo de execução

4.1. Estrutura de um programa

Um programa Python é construído a partir de blocos de código. Um *bloco* é um pedaço do texto do programa Python que é executado como uma unidade. A seguir estão os blocos: um módulo, um corpo de função e uma definição de classe. Cada comando digitado interativamente é um bloco. Um arquivo de script (um arquivo fornecido como entrada padrão para o interpretador ou especificado como argumento de linha de comando para o interpretador) é um bloco de código. Um comando de script (um comando especificado na linha de comando do interpretador com a opção `-c`) é um bloco de código. Um módulo executado sobre um script de nível superior (como o módulo `__main__`) a partir da linha de comando usando um argumento `-m` também é um bloco de código. O argumento da string passado para as funções embutidas `eval()` e `exec()` é um bloco de código.

Um bloco de código é executado em um *quadro de execução*. Um quadro contém algumas informações administrativas (usadas para depuração) e determina onde e como a execução continua após a conclusão do bloco de código.

4.2. Nomeação e ligação

4.2.1. Ligação de nomes

Nomes referem-se a objetos. Os nomes são introduzidos por operações de ligação de nomes.

As seguintes construções ligam nomes:

- parâmetros formais para funções,
- definições de classe,
- definições de função,

- expressões de atribuição,
- alvos que são identificadores se ocorrerem em uma atribuição:
  - cabeçalho de laço `for`,
  - depois de `as` em uma instrução `with`, cláusula `except`, cláusula `except*` ou no padrão as na correspondência de padrões estruturais,
  - em um padrão de captura na correspondência de padrões estruturais
- instruções `import`.
- instruções `type`.
- listas de parâmetros de tipo.

A instrução `import` no formato `from ... import *` liga todos os nomes definidos no módulo importado, exceto aqueles que começam com um sublinhado. Este formulário só pode ser usado no nível do módulo.

Um alvo ocorrendo em uma instrução `del` também é considerado ligado a esse propósito (embora a semântica real seja para desligar do nome).

Cada atribuição ou instrução de importação ocorre dentro de um bloco definido por uma definição de classe ou função ou no nível do módulo (o bloco de código de nível superior).

Se um nome está ligado a um bloco, é uma variável local desse bloco, a menos que declarado como `nonlocal` ou `global`. Se um nome está ligado a nível do módulo, é uma variável global. (As variáveis do bloco de código do módulo são locais e globais.) Se uma variável for usada em um bloco de código, mas não definida lá, é uma variável livre.

Cada ocorrência de um nome no texto do programa se refere à *ligação* daquele nome estabelecido pelas seguintes regras de resolução de nome.

4.2.2. Resolução de nomes

O *escopo* define a visibilidade de um nome dentro de um bloco. Se uma variável local é definida em um bloco, seu escopo inclui esse bloco. Se a definição ocorrer em um bloco de função, o escopo se estende a quaisquer

blocos contidos no bloco de definição, a menos que um bloco contido introduza uma ligação diferente para o nome.

Quando um nome é usado em um bloco de código, ele é resolvido usando o escopo envolvente mais próximo. O conjunto de todos esses escopos visíveis a um bloco de código é chamado de *ambiente* do bloco.

Quando um nome não é encontrado, uma exceção `NameError` é levantada. Se o escopo atual for um escopo de função e o nome se referir a uma variável local que ainda não foi associada a um valor no ponto onde o nome é usado, uma exceção `UnboundLocalError` é levantada. `UnboundLocalError` é uma subclasse de `NameError`.

Se a operação de ligação de nomes ocorre dentro de um bloco de código, todos os usos do nome dentro do bloco são tratadas como referências para o bloco atual. Isso pode. Isso pode levar a erros quando um nome é usado em um bloco antes de ser vinculado. Esta regra é sutil. Python carece de declarações e permite que as operações de ligação de nomes ocorram em qualquer lugar dentro de um bloco de código. As variáveis locais de um bloco de código podem ser determinadas pela varredura de todo o texto do bloco para operações de ligação de nome. Veja o FAQ sobre UnboundLocalError para exemplos.

Se a instrução `global` ocorrer dentro de um bloco, todos os usos dos nomes especificados na instrução referem-se às ligações desses nomes no espaço de nomes de nível superior. Os nomes são resolvidos no espaço de nomes de nível superior pesquisando o espaço de nomes global, ou seja, o espaço de nomes do módulo que contém o bloco de código, e o espaço de nomes embutido, o espaço de nomes do módulo `builtins`. O espaço de nomes global é pesquisado primeiro. Se os nomes não forem encontrados lá, o espaço de nomes embutidos será pesquisado em seguida. Se os nomes também não forem encontrados no espaço de nomes embutido, novas variáveis são criadas no espaço de nomes global. A instrução global deve preceder todos os usos dos nomes listados.

A instrução `global` tem o mesmo escopo que uma operação de ligação de nome no mesmo bloco. Se o escopo mais próximo de uma variável livre contiver uma instrução global, a variável livre será tratada como global.

A instrução **nonlocal** faz com que os nomes correspondentes se refiram a variáveis previamente vinculadas no escopo da função delimitadora mais próxima. A exceção **SyntaxError** é levantada em tempo de compilação se o nome fornecido não existir em nenhum escopo de função delimitador. Parâmetros de tipo não podem ser vinculadas novamente com a instrução **nonlocal**.

O espaço de nomes de um módulo é criado automaticamente na primeira vez que um módulo é importado. O módulo principal de um script é sempre chamado de **__main__**.

Blocos de definição de classe e argumentos para **exec()** e **eval()** são especiais no contexto de resolução de nome. Uma definição de classe é uma instrução executável que pode usar e definir nomes. Essas referências seguem as regras normais para resolução de nome, com exceção de que variáveis locais não vinculadas são pesquisadas no espaço de nomes global global. O espaço de nomes global da definição de classe se torna o dicionário de atributos da classe. O escopo dos nomes definidos em um bloco de classe é limitado ao bloco de classe; ele não se estende aos blocos de código de métodos. Isso inclui compreensões e expressões geradoras, mas não inclui escopos de anotação, que têm acesso a seus escopos de classe delimitadores. Isso significa que o seguinte falhará:

```
class A:
    a = 42
    b = list(a + i for i in range(10))
```

Porém, o seguinte vai funcionar:

```
class A:
    type Alias = Nested
    class Nested: pass

print(A.Alias.__value__)  # <type 'A.Nested'>
```

4.2.3. Escopos de anotação

As instruções de anotações, listas de parâmetros de tipo e **type** introduzem *escopos de anotação*, que se comportam principalmente como escopos de

função, mas com algumas exceções discutidas abaixo.

Os escopos de anotação são usados nos seguintes contextos:

- Anotações de funções.
- Anotações de variável.
- Listas de parâmetros de tipo para apelidos de tipo genérico.
- Listas de parâmetros de tipo para funções genéricas. As anotações de uma função genérica são executadas dentro do escopo de anotação, mas seus padrões e decoradores não.
- Listas de parâmetros de tipo para classes genéricas. As classes base e argumentos nomeados de uma classe genérica são executadas dentro do escopo de anotação, mas seus decoradores não.
- Os limites, restrições e valores padrão para parâmetros de tipo (avaliados preguiçosamente).
- O valor dos apelidos de tipo (avaliado preguiçosamente).

Escopos de anotação diferenciam-se de escopos de função nas seguintes formas:

- Os escopos de anotação têm acesso ao espaço de nomes da classe delimitadora. Se um escopo de anotação estiver imediatamente dentro de um escopo de classe ou dentro de outro escopo de anotação que esteja imediatamente dentro de um escopo de classe, o código no escopo de anotação poderá usar nomes definidos no escopo de classe como se fosse executado diretamente no corpo da classe. Isto contrasta com funções regulares definidas dentro de classes, que não podem acessar nomes definidos no escopo da classe.
- Expressões em escopos de anotação não podem conter expressões `yield`, `yield from`, `await` ou `:= 1`. (Essas expressões são permitidas em outros escopos contidos no escopo de anotação.)
- Nomes definidos em escopos de anotação não podem ser vinculados novamente com instruções `nonlocal` em escopos internos. Isso inclui apenas parâmetros de tipo, pois nenhum outro elemento sintático que pode aparecer nos escopos de anotação pode introduzir novos nomes.
- Embora os escopos de anotação tenham um nome interno, esse nome não é refletido no nome qualificado dos objetos definidos dentro do

escopo. Em vez disso, o `__qualname__` de tais objetos é como se o objeto fosse definido no escopo delimitador.

Adicionado na versão 3.12: Escopos de anotação foram introduzidos no Python 3.12 como parte da **PEP 695** [https://peps.python.org/pep-0695/].

Alterado na versão 3.13: Os escopos de anotação também são usados para padrões de parâmetros de tipo, conforme introduzido pela **PEP 696** [https://peps.python.org/pep-0696/].

Alterado na versão 3.14: Os escopos de anotação agora também são usados para anotações, conforme especificado na **PEP 649** [https://peps.python.org/pep-0649/] e **PEP 749** [https://peps.python.org/pep-0749/].

4.2.4. Avaliação preguiçosa

A maioria dos escopos de anotação são *avaliados preguiçosamente*. Isso inclui anotações, os valores de apelidos de tipo criados por meio da instrução `type` e os limites, restrições e valores padrão de tipos variáveis criados por meio da sintaxe de parâmetro de tipo sintaxe de parâmetro de tipo. Isso significa que eles não são avaliados quando o apelido de tipo ou a tipo variável é criado, ou quando o objeto que contém as anotações é criado. Em vez disso, eles são avaliados apenas quando necessário, por exemplo, quando o atributo `__value__` em um apelido de tipo é acessado.

Exemplo:

```
>>> type Alias = 1/0
>>> Alias.__value__
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> def func[T: 1/0]() pass
>>> T = func.__type_params__[0]
>>> T.__bound__
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
```

Aqui a exceção é levantada apenas quando o atributo `__value__` do apelido de tipo ou o atributo `__bound__` da variável de tipo é acessado.

Esse comportamento é útil principalmente para referências a tipos que ainda não foram definidos quando o alias de tipo ou variável de tipo é criado. Por exemplo, a avaliação preguiçosa permite a criação de apelidos de tipo mutuamente recursivos:

```
from typing import Literal

type SimpleExpr = int | Parenthesized
type Parenthesized = tuple[Literal["("], Expr, Literal[")"]]
type Expr = SimpleExpr | tuple[SimpleExpr, Literal["+", "-"], Ex
```

Valores avaliados preguiçosamente são avaliados em escopo de anotação, o que significa que os nomes que aparecem dentro do valor avaliado preguiçosamente são pesquisados como se fossem usados no escopo imediatamente envolvente.

Adicionado na versão 3.12.

4.2.5. Builtins e execução restrita

Os usuários não devem tocar em `__builtins__`; é estritamente um detalhe de implementação. Usuários que desejam substituir valores no espaço de nomes interno devem **`import`** o módulo **`builtins`** e modificar seus atributos apropriadamente.

O espaço de nomes builtins associado com a execução de um bloco de código é encontrado procurando o nome `__builtins__` em seu espaço de nomes global; este deve ser um dicionário ou um módulo (no último caso, o dicionário do módulo é usado). Por padrão, quando no módulo **`__main__`**, `__builtins__` é o módulo embutido **`builtins`**; quando em qualquer outro módulo, `__builtins__` é um apelido para o dicionário do próprio módulo **`builtins`**.

4.2.6. Interação com recursos dinâmicos

A resolução de nome de variáveis livres ocorre em tempo de execução, não em tempo de compilação. Isso significa que o código a seguir imprimirá 42:

```
i = 10
def f():
    print(i)
i = 42
f()
```

As funções **eval()** e **exec()** não têm acesso ao ambiente completo para resolução de nome. Os nomes podem ser resolvidos nos espaços de nomes locais e globais do chamador. Variáveis livres não são resolvidas no espaço de nomes mais próximo, mas no espaço de nomes global. [[1]] As funções **exec()** e **eval()** possuem argumentos opcionais para substituir o espaço de nomes global e local. Se apenas um espaço de nomes for especificado, ele será usado para ambos.

4.3. Exceções

As exceções são um meio de romper o fluxo normal de controle de um bloco de código para tratar erros ou outras condições excepcionais. Uma exceção é *levantada* no ponto em que o erro é detectado; ele pode ser *tratado* pelo bloco de código circundante ou por qualquer bloco de código que invocou direta ou indiretamente o bloco de código onde ocorreu o erro.

O interpretador Python levanta uma exceção quando detecta um erro em tempo de execução (como divisão por zero). Um programa Python também pode levantar explicitamente uma exceção com a instrução **raise**. Os tratadores de exceção são especificados com a instrução **try** … **except**. A cláusula **finally** de tal declaração pode ser usada para especificar o código de limpeza que não trata a exceção, mas é executado se uma exceção ocorreu ou não no código anterior.

Python usa o modelo de “terminação” da manipulação de erros: um manipulador de exceção pode descobrir o que aconteceu e continuar a execução em um nível externo, mas não pode reparar a causa do erro e tentar novamente a operação com falha (exceto reinserindo a parte incorreta de código de cima).

Quando uma exceção não é manipulada, o interpretador encerra a execução do programa ou retorna ao seu laço principal interativo. Em ambos os casos, ele exeibe um traceback (situação da pilha de execução), exceto quando a exceção é `SystemExit`.

As exceções são identificadas por instâncias de classe. A cláusula `except` é selecionada dependendo da classe da instância: ela deve referenciar a classe da instância ou uma classe base não-virtual dela. A instância pode ser recebida pelo manipulador e pode conter informações adicionais sobre a condição excepcional.

Nota

As mensagens de exceção não fazem parte da API do Python. Seu conteúdo pode mudar de uma versão do Python para outra sem aviso e não deve ser invocado pelo código que será executado em várias versões do interpretador.

Veja também a descrição da declaração `try` na seção A instrução try e a instrução `raise` na seção A instrução raise.

4.4. Componentes do Ambiente de Execução

4.4.1. Modelo Geral de Computação

O modelo de execução do Python não opera no vácuo. Ele é executado em uma máquina hospedeira e através do ambiente de execução desse hospedeiro, incluindo seu sistema operacional (SO), se houver um. Quando um programa é executado, as camadas conceituais de como ele é executado no hospedeiro têm esta forma:

máquina hospedeira
- **processo** (recursos globais)
  - **thread** (executa código de máquina)

Cada processo representa um programa rodando no hospdeiro. Pense em cada processo em si como a parte do programa referente aos dados. Pense nas threads do processo como a parte referente à execução. Essa distinção será importante para entender o ambiente de execução do Python.

O processo, enquanto parte referente a dados, é o contexto de execução sobre o qual o programa roda. Ele consiste majoritariamente no conjunto de recursos alocados paro programa pelo hospedeiro, incluindo memória, sinais, identificadores de arquivos, soquetes, e variáveis de ambiente.

Processos são isolados, e independentes uns dos outros. (O mesmo vale para hospedeiros.) O hospedeiro gerencia o acesso do processo aos recursos a ele alocados, além de orquestrar os vários processos.

Cada thread representa a execução em si do código de máquina do programa, rodando relativamente aos recursos alocados ao processo. Decidir como e quando essa execução acontecerá é prerrogativa exclusiva do hospedeiro.

Do ponto de vista do Python, um programa sempre começa com exatamente uma thread. Entretanto, o programa pode evoluir para rodar em múltiplas threads simultâneas. Nem todo hospedeiro oferece suporte para múltiplas threads por processo, mas a maioria oferece. Diferentemente de processos, threads em um processo não são isoladas nem independentes umas das outras. Especificamente, cada recurso de um processo é compartilhado por todas as suas threads.

O princípio fundamental das threads é que cada uma delas *roda* independentemente, ao mesmo tempo que as outras. Isso pode significar uma simultaneidade conceitual (“concorrência”) ou física (“paralelismo”). De qualquer modo, as threads rodam efetivamente em passos não sincronizados.

Nota

Essa falta de sincronismo significa que não existe nenhuma garantia de consistência em qualquer parte da memória do processo, do ponto de vista do código rodando em qualquer thread. Portanto, programas multi-thread

precisam tomar o cuidado de coordenar o acesso a recursos intencionalmente compartilhados. Da mesma forma, eles precisam ter o máximo cuidado para não acessar nenhum *outro* recurso a partir de múltiplas threads; caso contrário, duas threads rodando ao mesmo tempo podem acidentalmente interferir no uso umas das outras desses recursos compartilhados. Tudo isso vale tanto para programas Python quanto para o ambiente de execução do Python.

Essa exigência ampla e não estruturada é o custo que precisamos pagar em contrapartida pela concorrência bruta que as threads proporcionam. A alternativa à disciplina necessária geralmente envolve lidar com bugs não determinísticos e corrupção de dados.

4.4.2. Modelo do Ambiente de Execução Python

As mesmas camadas conceituais se aplicam a cada programa Python, com algumas camadas de dados específicas para o Python:

máquina hospedeira
- **processo** (recursos globais)
  - Ambiente de execução global do Python (*estado*)
    - Interpretador Python (*estado*)
      - **thread** (roda bytecode Python e a “API C”)
        - *Estado* da thread Python

No nível conceitual: quando um programa Python começa, ele tem o aspecto exato desse diagrama, com uma instância de cada linha. O ambiente de execução pode evoluir para incluir múltiplos interpretadores, e cada interpretador pode evoluir para incluir múltiplos estados de thread.

Nota

Uma implementação do Python não necessariamente vai implementar essas camadas do ambiente de execução de forma distinta, ou mesmo concreta. A única exceção são os casos em que camadas distintas são

diretamente especificadas ou expostas aos usuários, como através do módulo `threading`.

Nota

O interpretador inicial é geralmente chamado de interpretador “principal”. Algumas implementações do Python, como o CPython, dão papéis especiais ao interpretador principal.

Da mesma forma, a thread do hospedeiro na qual o ambiente de execução foi inicializado é chamada de thread “principal”. Ela pode ser diferente da thread inicial do processo, embora geralmente sejam a mesma. Em alguns casos, o termo “thread principal” pode ser ainda mais específico, e se referir ao estado de thread inicial. Um ambiente de execução Python pode atribuir responsabilidades especiais à thread principal, como por exemplo lidar com sinais.

Em sua totalidade, o ambiente de execução Python consiste no seu estado global, seus interpretadores, e seus estados de thread. O ambiente de execução garante que todo esse estado permaneça consistente ao longo do seu tempo de vida, em particular quando usado a partir de múltiplas threads do hospedeiro.

O ambiente de execução global, conceitualmente, é simplesmente um conjunto de interpretadores. Esses interpretadores podem compartilhar alguns dados e outros recursos, mas fora isso são isolados e independentes uns dos outros. O ambiente de execução é responsável por gerenciar esses recursos globais de forma segura. A natureza física desses recursos, bem como o seu gerenciamento, é específica de cada implementação. No fim das contas, a utilidade externa do ambiente de execução global é limitada ao gerenciamento de interpretadores.

Em contraste, conceitualmente um “interpretador” é o que normalmente consideraríamos como sendo o “ambiente de execução Python” (com todas as suas funcionalidades). Quando código de máquina, sendo executado em

uma thread do hospedeiro, interage com o ambiente de execução Python, ele chama o Python no contexto de um interpretador específico.

Nota

O termo interpretador aqui não é a mesma coisa que o “interpretador de bytecode”, que é o que normalmente roda nas threads, executando código Python compilado.

Em um mundo ideal, o termo “ambiente de execução Python” seria usado para se referir ao que atualmente chamamos de “interpretador”. Entretanto, ele vem sendo chamado de “interpretador” pelo menos desde que foi introduzido em 1997 (CPython:a027efa5b [https://github.com/python/cpython/commit/a027efa5b]).

Cada interpretador encapsula completamente todo o estado necesssário para o ambiente de execução do Python funcionar, que não seja global do processo nem específico de threads. Notavelmente, o estado do interpretador persiste entre usos. Ele inclui dados fundamentais, como o `sys.modules`. O ambiente de execução garante que múltiplas threads usando o mesmo interpretador vão compartilhá-lo de maneira segura.

Uma implementação do Python pode oferecer suporte para usar múltiplos interpretadores ao mesmo tempo no mesmo processo. Eles são independentes e isolados uns dos outros. Por exemplo, cada interpretador tem o seu próprio `sys.modules`.

Quanto aos estados específicos ao ambiente de execução de cada thread, cada interpretador gerencia um conjunto de estados de thread, da mesma maneira que o ambiente de execução global contém um conjunto de interpretadores. Cada interpretador pode ter estados de thread para quantas threads do hospedeiro forem necessárias. Ele pode até mesmo ter múltiplos estados de thread para uma mesma thread do hospedeiro, embora isso não seja tão comum.

Conceitualmente, cada estado de thread contém os dados do ambiente de execução específicos àquela thread que são necessários para o interpretador

operar em uma thread do hospedeiro. O estado de thread inclui a exceção atualmente levantada e a pilha de chamadas Python da thread. Ele pode incluir outros recursos específicos à thread.

Nota

O termo “thread Python” pode às vezes se referir ao estado de thread, mas normalmente significa uma thread criada a partir do módulo `threading`.

Cada estado de thread, ao longo do seu tempo de vida, está sempre atrelado a exatamente um interpretador e uma thread do hospedeiro. Ele sempre vai ser usado somente naquela thread e com aquele interpretador.

Múltiplos estados de thread podem estar atrelados à mesma thread do hospedeiro, seja para diferentes interpretadores ou até mesmo para um mesmo interpretador. Entretanto, dada qualquer thread do hospedeiro, somente um dos estados de thread atrelados a ela pode ser usado por ela de cada vez.

Estados de thread são isolados e independentes uns dos outros e não compartilham dados, exceto que podem compartilhar o interpretador e objetos ou outros recursos que pertencem ao interpretador.

Quando um programa está rodando, novas threads Python podem ser criadas a partir do módulo `threading` (em plataformas e implementações do Python que ofereçam suporte a threads). Processos adicionais podem ser criados usando os módulos `os`, `subprocess` e `multiprocessing`. Interpretadores podem ser criados e usados com o módulo `interpreters`. Corrotinas (async) podem ser executadas usando `asyncio` em cada interpretador, tipicamente em uma única thread (geralmente a thread principal).

Notas de rodapé

[1]

Essa limitação ocorre porque o código executado por essas operações não está disponível no momento em que o módulo é compilado.

5. O sistema de importação

O código Python em um módulo obtém acesso ao código em outro módulo pelo processo de importação dele. A instrução `import` é a maneira mais comum de invocar o mecanismo de importação, mas não é a única maneira. Funções como `importlib.import_module()` e a função embutida `__import__()` também podem ser usadas para chamar o mecanismo de importação.

A instrução `import` combina duas operações; ela procura o módulo nomeado e vincula os resultados dessa pesquisa a um nome no escopo local. A operação de busca da instrução **`import`** é definida como uma chamada para a função `__import__()`, com os argumentos apropriados. O valor de retorno de **`__import__()`** é usado para executar a operação de ligação de nome da instrução **`import`**. Veja a instrução **`import`** para os detalhes exatos da operação de ligação desse nome.

Uma chamada direta para `__import__()` realiza apenas a pesquisa do módulo e, se encontrada, a operação de criação do módulo. Embora certos efeitos colaterais possam ocorrer, como a importação de pacotes pai e a atualização de vários caches (incluindo `sys.modules`), apenas a instrução `import` realiza uma operação de ligação de nome.

Quando uma instrução `import` é executada, a função embutida padrão `__import__()` é chamada. Outros mecanismos para chamar o sistema de importação (como `importlib.import_module()`) podem optar por ignorar **`__import__()`** e usar suas próprias soluções para implementar a semântica de importação.

Quando um módulo é importado pela primeira vez, o Python procura pelo módulo e, se encontrado, cria um objeto de módulo [[1]], inicializando-o. Se o módulo nomeado não puder ser encontrado, uma `ModuleNotFoundError` será levantada. O Python implementa várias estratégias para procurar o módulo nomeado quando o mecanismo de

importação é chamado. Essas estratégias podem ser modificadas e estendidas usando vários ganchos descritos nas seções abaixo.

Alterado na versão 3.3: O sistema de importação foi atualizado para implementar completamente a segunda fase da **PEP 302** [https://peps.python.org/pep-0302/]. Não há mais um mecanismo de importação implícito – o sistema completo de importação é exposto através de `sys.meta_path`. Além disso, o suporte nativo a pacote de espaço de nomes foi implementado (consulte **PEP 420** [https://peps.python.org/pep-0420/]).

5.1. `importlib`

O módulo `importlib` fornece uma API rica para interagir com o sistema de importação. Por exemplo, `importlib.import_module()` fornece uma API mais simples e recomendada do que a função embutida `__import__()` para chamar o mecanismo de importação. Consulte a documentação da biblioteca **`importlib`** para obter detalhes adicionais.

5.2. Pacotes

O Python possui apenas um tipo de objeto de módulo e todos os módulos são desse tipo, independentemente de o módulo estar implementado em Python, C ou qualquer outra coisa. Para ajudar a organizar os módulos e fornecer uma hierarquia de nomes, o Python tem o conceito de pacotes.

Você pode pensar em pacotes como os diretórios em um sistema de arquivos e os módulos como arquivos nos diretórios, mas não tome essa analogia muito literalmente, já que pacotes e módulos não precisam se originar do sistema de arquivos. Para os fins desta documentação, usaremos essa analogia conveniente de diretórios e arquivos. Como os diretórios do sistema de arquivos, os pacotes são organizados hierarquicamente e os próprios pacotes podem conter subpacotes e módulos regulares.

É importante ter em mente que todos os pacotes são módulos, mas nem todos os módulos são pacotes. Ou, dito de outra forma, os pacotes são

apenas um tipo especial de módulo. Especificamente, qualquer módulo que contenha um atributo `__path__` é considerado um pacote.

Todo módulo tem um nome. Nomes de subpacotes são separados do nome do pacote por um ponto, semelhante à sintaxe de acesso aos atributos padrão do Python. Assim pode ter um pacote chamado **`email`**, que por sua vez tem um subpacote chamado **`email.mime`** e um módulo dentro dele chamado **`email.mime.text`**.

5.2.1. Pacotes regulares

O Python define dois tipos de pacotes, pacotes regulares e pacotes de espaço de nomes. Pacotes regulares são pacotes tradicionais, como existiam no Python 3.2 e versões anteriores. Um pacote regular é normalmente implementado como um diretório que contém um arquivo `__init__.py`. Quando um pacote regular é importado, esse arquivo `__init__.py` é executado implicitamente, e os objetos que ele define são vinculados aos nomes no espaço de nomes do pacote. O arquivo `__init__.py` pode conter o mesmo código Python que qualquer outro módulo pode conter, e o Python adicionará alguns atributos adicionais ao módulo quando ele for importado.

Por exemplo, o layout do sistema de arquivos a seguir define um pacote `parent` de nível superior com três subpacotes:

```
parent/
    __init__.py
    one/
        __init__.py
    two/
        __init__.py
    three/
        __init__.py
```

A importação de `parent.one` vai executar implicitamente `parent/__init__.py` e `parent/one/__init__.py`. Importações subsequentes de `parent.two` ou `parent.three` vão executar `parent/two/__init__.py` e `parent/three/__init__.py`, respectivamente.

Um subdiretório dentro de um pacote regular que não contém um arquivo `__init__.py` é tratado como um pacote de espaço de nomes implícito (um "subpacote de espaço de nomes") enraizado nesse pacote pai. Consulte a **PEP 420** [https://peps.python.org/pep-0420/] para a especificação subjacente.

5.2.2. Pacotes de espaço de nomes

Um pacote de espaço de nomes é um composto de várias porções, em que cada parte contribui com um subpacote para o pacote pai. Partes podem residir em locais diferentes no sistema de arquivos. Partes também podem ser encontradas em arquivos zip, na rede ou em qualquer outro lugar que o Python pesquisar durante a importação. Os pacotes de espaço de nomes podem ou não corresponder diretamente aos objetos no sistema de arquivos; eles podem ser módulos virtuais que não têm representação concreta.

Os pacotes de espaço de nomes não usam uma lista comum para o atributo `__path__`. Em vez disso, eles usam um tipo iterável personalizado que executará automaticamente uma nova pesquisa por partes do pacote na próxima tentativa de importação dentro desse pacote, se o caminho do pacote pai (ou **`sys.path`** para um pacote de nível superior) for alterado.

Com pacotes de espaço de nomes, não há arquivo `pai/__init__.py`. De fato, pode haver vários diretórios `pai` encontrados durante a pesquisa de importação, onde cada um é fornecido por uma parte diferente. Portanto, `pai/um` pode não estar fisicamente localizado próximo a `pai/dois`. Nesse caso, o Python criará um pacote de espaço de nomes para o pacote `pai` de nível superior sempre que ele ou um de seus subpacotes for importado.

Pacotes de espaços de nomes também podem estar aninhados dentro de um pacote regular. Quando o sistema de importação pesquisa o `__path__` de um pacote regular e encontra um subdiretório que não contém um arquivo `__init__.py`, esse subdiretório torna-se uma porção que contribui para um subpacote de espaço de nomes do pacote regular que o contém.

Veja também **PEP 420** [https://peps.python.org/pep-0420/] para a especificação de pacotes de espaço de nomes.

5.3. Caminho de busca

Para iniciar a busca, o Python precisa do nome completo do módulo (ou pacote, mas para o propósito dessa exposição, não há diferença) que se quer importar. Esse nome vem de vários argumentos passados para a instrução `import`, ou dos parâmetros das funções `importlib.import_module()` ou `__import__()`.

Esse nome será usado em várias fases da busca da importação, e pode ser um nome com pontos para um submódulo como, por exemplo, `foo.bar.baz`. Nesse caso, Python primeiro tenta importar `foo`, depois `foo.bar` e, finalmente, `foo.bar.baz`. Se alguma das importações intermediárias falharem, uma exceção `ModuleNotFoundError` é levantada.

5.3.1. O cache de módulos

A primeira verificação durante a busca da importação é feita no `sys.modules`. Esse mapeamento serve como um cache de todos os módulos que já foram importados previamente, incluindo os caminhos intermediários. Se `foo.bar.baz` foi previamente importado, **`sys.modules`** conterá entradas para `foo`, `foo.bar` e `foo.bar.baz`. Cada chave terá como valor um objeto módulo correspondente.

Durante a importação, o nome do módulo é procurado em `sys.modules` e, se estiver presente, o valor associado é o módulo que satisfaz a importação, e o processo termina. Entretanto, se o valor é `None`, uma exceção `ModuleNotFoundError` é levantada. Se o nome do módulo não foi encontrado, Python continuará a busca pelo módulo.

É possível alterar `sys.modules`. Apagar uma chave pode não destruir o objeto módulo associado (outros módulos podem manter referências para ele), mas a entrada do cache será invalidada para o nome daquele módulo, fazendo Python executar nova busca na próxima importação. Pode ser atribuído `None` para a chave, forçando que a próxima importação do módulo resulte numa exceção `ModuleNotFoundError`.

No entanto, tenha cuidado, pois se você mantiver uma referência para o objeto módulo, invalidar sua entrada de cache em `sys.modules` e, em seguida, reimportar do módulo nomeado, os dois módulo objetos *não* serão os mesmos. Por outro lado, o `importlib.reload()` reutilizará o *mesmo* objeto módulo e simplesmente reinicializará o conteúdo do módulo executando novamente o código do módulo.

5.3.2. Localizadores e carregadores

Se o módulo nomeado não for encontrado em `sys.modules`, então o protocolo de importação do Python é invocado para localizar e carregar o módulo. Este protocolo consiste em dois objetos conceituais, localizadores e carregadores. O trabalho de um localizador é determinar se ele pode localizar o módulo nomeado usando qualquer estratégia que ele conheça. Objetos que implementam ambas essas interfaces são referenciadas como importadores – eles retornam a si mesmos, quando eles descobrem que eles podem carregar o módulo requisitado.

Python inclui um número de localizadores e carregadores padrões. O primeiro sabe como localizar módulos embutidos, e o segundo sabe como localizar módulos congelados. Um terceiro localizador padrão procura em um caminho de importação por módulos. O caminho de importação é uma lista de localizações que podem nomear caminhos de sistema de arquivo ou arquivos zip. Ele também pode ser estendido para buscar por qualquer recurso localizável, tais como aqueles identificados por URLs.

O mecanismo de importação é extensível, então novos localizadores podem ser adicionados para estender o alcance e o escopo de buscar módulos.

Localizadores na verdade não carregam módulos. Se eles conseguirem encontrar o módulo nomeado, eles retornam um *spec de módulo*, um encapsulamento da informação relacionada a importação do módulo, a qual o mecanismo de importação então usa quando o módulo é carregado.

As seguintes seções descrevem o protocolo para localizadores e carregadores em mais detalhes, incluindo como você pode criar e registrar novos para estender o mecanismo de importação.

Alterado na versão 3.4: Em versões anteriores do Python, localizadores retornavam carregadores diretamente, enquanto agora eles retornam especificações de módulo, as qual *contêm* carregadores. Carregadores ainda são usados durante a importação, mas possuem menos responsabilidades.

5.3.3. Ganchos de importação

O mecanismo de importação é desenhado para ser extensível; o mecanismo primário para isso são os *ganchos de importação*. Existem dois tipos de ganchos de importação: *metaganchos* e *ganchos de importação de caminho*.

Metaganchos são chamados no início do processo de importação, antes que qualquer outro processo de importação tenha ocorrido, que não seja busca de cache de `sys.modules`. Isso permite aos metaganchos substituir processamento de `sys.path`, módulos congelados ou mesmo módulos embutidos. Metaganchos são registrados adicionando novos objetos localizadores a `sys.meta_path`, conforme descrito abaixo.

Ganchos de caminho de importação são chamados como parte do processamento de `sys.path` (ou `package.__path__`), no ponto onde é encontrado o item do caminho associado. Ganchos de caminho de importação são registrados adicionando novos chamáveis para `sys.path_hooks`, conforme descrito abaixo.

5.3.4. O metacaminho

Quando o módulo nomeado não é encontrado em `sys.modules`, o Python em seguida o busca em `sys.meta_path`, o qual contém uma lista de objetos localizadores de metacaminho. Esses localizadores são consultados a fim de verificar se eles sabem como manipular o módulo nomeado. Os localizadores de metacaminho devem implementar um método chamado `find_spec()`, o qual recebe três argumentos: um nome, um caminho de importação, e (opcionalmente) um módulo alvo. O localizador de metacaminho pode usar qualquer estratégia que ele quiser para determinar se ele pode manipular o módulo nomeado ou não.

Se o localizador de metacaminho souber como tratar o módulo nomeado, ele retorna um objeto spec. Se ele não puder tratar o módulo nomeado, ele retorna `None`. Se o processamento de **`sys.meta_path`** alcançar o fim da sua lista sem retornar um spec, então **`ModuleNotFoundError`** é levantada. Quaisquer outras exceções levantadas são simplesmente propagadas para cima, abortando o processo de importação.

O método **`find_spec()`** dos localizadores de metacaminhos é chamado com dois ou três argumentos. O primeiro é o nome totalmente qualificado do módulo sendo importado, por exemplo `foo.bar.baz`. O segundo argumento são as entradas do caminho a ser usado na busca do módulo. Para módulos de nível superior, o segundo argumento é `None`, mas para submódulos ou subpacotes, o segundo argumento é o valor do atributo `__path__` do pacote pai. Se o atributo `__path__` apropriado não puder ser acessado, uma exceção **`ModuleNotFoundError`** é levantada. O terceiro argumento é um objeto módulo existente que será o alvo do carregamento posteriormente. O sistema de importação passa um módulo alvo apenas durante o recarregamento.

O metacaminho pode ser percorrido múltiplas vezes para uma requisição de importação individual. Por exemplo, presumindo que nenhum dos módulos envolvidos já tenha sido cacheado, importar `foo.bar.baz` irá primeiro executar uma importação de alto nível, chamando `mpf.find_spec("foo", None, None)` em cada localizador de metacaminho (`mpf`). Depois que `foo` foi importado, `foo.bar` será importado percorrendo o metacaminho uma segunda vez, chamando `mpf.find_spec("foo.bar", foo.__path__, None)`. Uma vez que `foo.bar` tenha sido importado, a travessia final irá chamar `mpf.find_spec("foo.bar.baz", foo.bar.__path__, None)`.

Alguns localizadores de metacaminho apenas dão suporte a importações de alto nível. Estes importadores vão sempre retornar `None` quando qualquer coisa diferente de `None` for passada como o segundo argumento.

O **`sys.meta_path`** padrão do Python possui três localizador de metacaminho, um que sabe como importar módulos embutidos, um que sabe como importar módulos congelados, e outro que sabe como importar módulos de um caminho de importação (isto é, o localizador baseado no caminho).

Alterado na versão 3.4: O método `find_spec()` dos localizador de metacaminho substituiu `find_module()`, o qual agora foi descontinuado. Embora continue a funcionar sem alterações, a mecanismo de importação só tentará fazê-lo se o localizador não implementar `find_spec()`.

Alterado na versão 3.10: O uso de `find_module()` pelo sistema de importação agora levanta `ImportWarning`.

Alterado na versão 3.12: `find_module()` foi removido. Use `find_spec()`.

5.4. Carregando

Se e quando uma spec de módulo é encontrada, o mecanismo de importação vai usá-la (e o carregador que ela contém) durante o carregamento do módulo. Esta é uma aproximação do que acontece durante a etapa de carregamento de uma importação:

```
module = None
if spec.loader is not None and hasattr(spec.loader, 'create_modu
    # Assume-se que 'exec_module' também estará definido no carr
    module = spec.loader.create_module(spec)
if module is None:
    module = ModuleType(spec.name)
# Os atributos de módulo relacionados à importação são definidos
_init_module_attrs(spec, module)

if spec.loader is None:
    # Não suportado
    raise ImportError

sys.modules[spec.name] = module
try:
    spec.loader.exec_module(module)
except BaseException:
    try:
        del sys.modules[spec.name]
    except KeyError:
        pass
    raise
return sys.modules[spec.name]
```

Perceba os seguintes detalhes:

- Se houver um objeto módulo existente com o nome fornecido em `sys.modules`, a importação já tera retornado ele.
- O módulo irá existir em `sys.modules` antes do carregador executar o código do módulo. Isso é crucial porque o código do módulo pode (direta ou indiretamente) importar a si mesmo; adicioná-lo a `sys.modules` antecipadamente previne recursão infinita no pior caso e múltiplos carregamentos no melhor caso.
- Se o carregamento falhar, o módulo com falha – e apenas o módulo com falha – é removido de `sys.modules`. Qualquer módulo já presente no cache de `sys.modules`, e qualquer módulo que tenha sido carregado com sucesso como um efeito colateral, deve permanecer no cache. Isso contrasta com recarregamento, onde mesmo o módulo com falha é mantido em `sys.modules`.
- Depois que o módulo é criado, mas antes da execução, o mecanismo de importação define os atributos de módulo relacionados a importação ("_init_module_attrs" no exemplo de pseudocódigo acima), assim como foi resumido em uma seção posterior.
- Execução de módulo é o momento chave do carregamento, no qual o espaço de nomes do módulo é populado. Execução é inteiramente delegada para o carregador, o qual pode decidir o que será populado e como.
- O módulo criado durante o carregamento e passado para exec_module() pode não ser aquele retornado ao final da importação [[2]].

Alterado na versão 3.4: O sistema de importação tem tomado conta das responsabilidades inerentes dos carregadores. Essas responsabilidades eram anteriormente executadas pelo método `importlib.abc.Loader.load_module`.

Alterado na versão 3.15: O método `load_module` não é mais usado.

5.4.1. Carregadores

Os carregadores de módulo fornecem a função crítica de carregamento: execução do módulo. O mecanismo de importação chama o método **importlib.abc.Loader.exec_module()** com um único argumento, o objeto do módulo a ser executado. Qualquer valor retornado de **exec_module()** é ignorado.

Os carregadores devem atender aos seguintes requisitos:

- Se o módulo for um módulo Python (em oposição a um módulo embutido ou uma extensão carregada dinamicamente), o carregador deve executar o código do módulo no espaço de nomes global do módulo (`module.__dict__`).
- Se o carregador não puder executar o módulo, ele deve levantar uma exceção **ImportError**, embora qualquer outra exceção levantada durante **exec_module()** será propagada.

Em muitos casos, o localizador e o carregador podem ser o mesmo objeto; nesses casos o método **find_spec()** apenas retornaria um spec com o carregador definido como `self`.

Os carregadores de módulo podem optar por criar o objeto do módulo durante o carregamento, implementando um método **create_module()**. Leva um argumento, o spec de módulo e retorna o novo objeto do módulo para usar durante o carregamento. `create_module()` não precisa definir nenhum atributo no objeto do módulo. Se o método retornar `None`, o mecanismo de importação criará ele mesmo o novo módulo.

Adicionado na versão 3.4: O método **create_module()** de carregadores.

Alterado na versão 3.4: O método `importlib.abc.Loader.load_module` foi substituído por **exec_module()** e o mecanismo de importação assumiu todas as responsabilidades inerentes de carregamento.

Para compatibilidade com carregadores existentes, o mecanismo de importação usará o método `load_module()` de carregadores se ele existir e o carregador também não implementar `exec_module()`. No entanto, `load_module()` foi descontinuado e os carregadores devem implementar `exec_module()` em seu lugar.

O método `load_module()` deve implementar toda a funcionalidade inerente de carregamento descrita acima, além de executar o módulo. Todas as mesmas restrições se aplicam, com alguns esclarecimentos adicionais:

- Se houver um objeto de módulo existente com o nome fornecido em **`sys.modules`**, o carregador deverá usar esse módulo existente. (Caso contrário, **`importlib.reload()`** não funcionará corretamente.) Se o módulo nomeado não existir em **`sys.modules`**, o carregador deverá criar um novo objeto de módulo e adicioná-lo a **`sys.modules`**.
- O módulo *deve* existir em **`sys.modules`** antes que o carregador execute o código do módulo, para evitar recursão ilimitada ou carregamento múltiplo.
- Se o carregamento falhar, o carregador deverá remover quaisquer módulos inseridos em **`sys.modules`**, mas deverá remover **apenas** o(s) módulo(s) com falha, e somente se o próprio carregador tiver carregado o(s) módulo(s) explicitamente.

Alterado na versão 3.5: Uma exceção **`DeprecationWarning`** é levantada quando `exec_module()` está definido, mas `create_module()` não.

Alterado na versão 3.6: Uma exceção **`ImportError`** é levantada quando `exec_module()` está definido, mas `create_module()` não.

Alterado na versão 3.10: O uso de `load_module()` vai levantar **`ImportWarning`**.

5.4.2. Submódulos

Quando um submódulo é carregado usando qualquer mecanismo (por exemplo, APIs `importlib`, as instruções `import` ou `import-from`, ou a função `__import__()` embutida) uma ligação é colocada no espaço de nomes do módulo pai para o objeto submódulo. Por exemplo, se o pacote `spam` tiver um submódulo `foo`, após importar `spam.foo`, `spam` terá um atributo `foo` que está vinculado ao submódulo. Digamos que você tenha a seguinte estrutura de diretórios:

```
spam/
    __init__.py
```

```
    foo.py
```

e `spam/__init__.py` tem a seguinte linha:

```
from .foo import Foo
```

então executar o seguinte coloca ligações de nome para `foo` e `Foo` no módulo `spam`:

```
>>> import spam
>>> spam.foo
<module 'spam.foo' from '/tmp/imports/spam/foo.py'>
>>> spam.Foo
<class 'spam.foo.Foo'>
```

Dadas as conhecidas regras de ligação de nomes do Python, isso pode parecer surpreendente, mas na verdade é um recurso fundamental do sistema de importação. A propriedade invariante é que se você tiver `sys.modules['spam']` e `sys.modules['spam.foo']` (como faria após a importação acima), o último deve aparecer como o atributo `foo` do primeiro.

5.4.3. Especificações de módulo

O mecanismo de importação utiliza diversas informações sobre cada módulo durante a importação, principalmente antes do carregamento. A maior parte das informações é comum a todos os módulos. O propósito da spec do módulo é encapsular essas informações relacionadas à importação por módulo.

Usar um spec durante a importação permite que o estado seja transferido entre componentes do sistema de importação, por exemplo entre o localizador que cria o spec de módulo e o carregador que o executa. Mais importante ainda, permite que o mecanismo de importação execute as operações inerentes de carregamento, enquanto que sem um spec de módulo o carregador tinha essa responsabilidade.

O spec do módulo é exposto no atributo **`module.__spec__`**. A configuração oportuna do **`__spec__`** se aplica igualmente aos módulos inicializados

durante a inicialização do interpretador. A única exceção é o `__main__`, cujo **`__spec__`** é definido como None em alguns casos.

Consulte **`ModuleSpec`** para detalhes sobre o conteúdo do spec de módulo.

Adicionado na versão 3.4.

5.4.4. Atributo __path__ dos módulos

O atributo **`__path__`** deve ser uma sequência (possivelmente vazia) de strings enumerando os locais onde os submódulos do pacote serão encontrados. Por definição, um módulo que tenha um atributo **`__path__`** é um pacote.

O atributo **`__path__`** de um pacote é usado durante as importações de seus subpacotes. Dentro do mecanismo de importação, funciona da mesma forma que **`sys.path`**, ou seja, fornecendo uma lista de locais para procurar módulos durante a importação. Entretanto, o **`__path__`** normalmente é muito mais restrito que **`sys.path`**.

As mesmas regras usadas para **`sys.path`** também se aplicam ao **`__path__`** de um pacote. Os **`sys.path_hooks`** (descritos abaixo) são consultados ao percorrer o **`__path__`** de um pacote.

O arquivo `__init__.py` de um pacote pode definir ou alterar o atributo **`__path__`** do pacote, e esta era tipicamente a forma como os pacotes de espaço de nomes eram implementados antes da **PEP 420** [https://peps.python.org/pep-0420/]. Com a adoção da **PEP 420** [https://peps.python.org/pep-0420/], os pacotes de espaço de nomes não precisam mais fornecer arquivos `__init__.py` contendo apenas código de manipulação do **`__path__`**; o mecanismo de importação define automaticamente o **`__path__`** corretamente para o pacote de espaço de nomes.

5.4.5. Representações do módulo

Por padrão, todos os módulos têm uma representação (repr) utilizável, no entanto, dependendo dos atributos definidos acima e do spec do módulo, você pode controlar mais explicitamente a representação dos objetos módulo.

Se o módulo tiver um spec (`__spec__`), o mecanismo de importação tentará gerar uma representação a partir dele. Se isso falhar ou não houver nenhuma especificação, o sistema de importação criará uma representação padrão usando qualquer informação disponível no módulo. Ele tentará usar `module.__name__`, `module.__file__` e `module.__loader__` como entrada para a representação, com padrões para qualquer informação que esteja faltando.

Arquivo estão as exatas regras usadas:

- Se o módulo tiver um atributo `__spec__`, a informação no spec é usada para gerar a representação. Os atributos “name”, “loader”, “origin” e “has_location” são consultados.
- Se o módulo tiver um atributo `__file__`, ele será usado como parte da representação do módulo.
- Se o módulo não tem `__file__` mas tem um `__loader__` que não seja `None`, então a representação do carregador é usado como parte da representação do módulo.
- Caso contrário, basta usar o `__name__` do módulo na representação.

Alterado na versão 3.12: O uso de **`module_repr()`**, descontinuado desde o Python 3.4, foi removido no Python 3.12 e não é mais chamado durante a resolução da representação de um módulo.

5.4.6. Invalidação de bytecode em cache

Antes do Python carregar o bytecode armazenado em cache de um arquivo `.pyc`, ele verifica se o cache está atualizado com o arquivo fonte `.py`. Por padrão, o Python faz isso armazenando o registro de data e hora da última modificação da fonte e o tamanho no arquivo de cache ao escrevê-lo. No tempo de execução, o sistema de importação valida o arquivo de cache

verificando os metadados armazenados no arquivo de cache em relação aos metadados do código-fonte.

Python também oferece suporte a arquivos de cache “baseados em hash”, que armazenam um hash do conteúdo do arquivo fonte em vez de seus metadados. Existem duas variantes de arquivos `.pyc` baseados em hash: verificados e não verificados. Para arquivos `.pyc` baseados em hash verificados, o Python valida o arquivo de cache fazendo hash do arquivo fonte e comparando o hash resultante com o hash no arquivo de cache. Se um arquivo de cache baseado em hash verificado for inválido, o Python o regenerará e gravará um novo arquivo de cache baseado em hash verificado. Para arquivos `.pyc` baseados em hash não verificados, o Python simplesmente presume que o arquivo de cache é válido, se existir. O comportamento de validação de arquivos `.pyc` baseados em hash pode ser substituído pelo sinalizador **`--check-hash-based-pycs`**.

Alterado na versão 3.7: Adicionados arquivos `.pyc` baseados em hash. Anteriormente, o Python oferecia suporte apenas à invalidação de caches de bytecode baseada em registro de data e hora.

5.5. O localizador baseado no caminho

Conforme mencionado anteriormente, Python vem com vários localizadores de metacaminho padrão. Um deles, chamado localizador baseado no caminho (**`PathFinder`**), pesquisa um caminho de importação, que contém uma lista de entradas de caminho. Cada entrada de caminho nomeia um local para procurar módulos.

O próprio localizador baseado no caminho não sabe como importar nada. Em vez disso, ele percorre as entradas de caminho individuais, associando cada uma delas a um localizador de entrada de caminho que sabe como lidar com esse tipo específico de caminho.

O conjunto padrão de localizadores de entrada de caminho implementa toda a semântica para localizar módulos no sistema de arquivos, manipulando tipos de arquivos especiais, como código-fonte Python (arquivos `.py`), código de bytes Python (arquivos `.pyc`) e bibliotecas compartilhadas (por

exemplo, arquivos `.so`). Quando suportado pelo módulo **`zipimport`** na biblioteca padrão, os localizadores de entrada de caminho padrão também lidam com o carregamento de todos esses tipos de arquivos (exceto bibliotecas compartilhadas) de arquivos zip.

As entradas de caminho não precisam ser limitadas aos locais do sistema de arquivos. Eles podem referir-se a URLs, consultas de banco de dados ou qualquer outro local que possa ser especificado como uma string.

O localizador baseado no caminho fornece ganchos e protocolos adicionais para que você possa estender e personalizar os tipos de entradas de caminho pesquisáveis. Por exemplo, se você quiser oferecer suporte a entradas de caminho como URLs de rede, poderá escrever um gancho que implemente a semântica HTTP para localizar módulos na web. Este gancho (um chamável) retornaria um localizador de entrada de caminho suportando o protocolo descrito abaixo, que foi então usado para obter um carregador para o módulo da web.

Uma palavra de advertência: esta seção e a anterior usam o termo *localizador*, distinguindo-os usando os termos localizador de metacaminho e localizador de entrada de caminho. Esses dois tipos de localizadores são muito semelhantes, oferecem suporte a protocolos semelhantes e funcionam de maneira semelhante durante o processo de importação, mas é importante ter em mente que eles são sutilmente diferentes. Em particular, os localizadores de metacaminho operam no início do processo de importação, conforme a travessia de **`sys.meta_path`**.

Por outro lado, os localizadores de entrada de caminho são, em certo sentido, um detalhe de implementação do localizador baseado no caminho e, de fato, se o localizador baseado no caminho fosse removido de **`sys.meta_path`**, nenhuma semântica do localizador de entrada de caminho seria ser invocado.

5.5.1. Localizadores de entrada de caminho

O localizador baseado no caminho é responsável por encontrar e carregar módulos e pacotes Python cuja localização é especificada com uma string

entrada de caminho. A maioria das entradas de caminho nomeiam locais no sistema de arquivos, mas não precisam ser limitadas a isso.

Como um localizador de metacaminho, o localizador baseado no caminho implementa o protocolo **find_spec()** descrito anteriormente, no entanto, ele expõe ganchos adicionais que podem ser usados para personalizar como os módulos são encontrados e carregado do caminho de importação.

Três variáveis são usadas pelo localizador baseado no caminho, **sys.path**, **sys.path_hooks** e **sys.path_importer_cache**. Os atributos `__path__` em objetos de pacote também são usados. Eles fornecem maneiras adicionais de personalizar o mecanismo de importação.

sys.path contém uma lista de strings fornecendo locais de pesquisa para módulos e pacotes. Ele é inicializado a partir da variável de ambiente **PYTHONPATH** e vários outros padrões específicos de instalação e implementação. Entradas em **sys.path** podem nomear diretórios no sistema de arquivos, arquivos zip e potencialmente outros "locais" (veja o módulo **site**) que devem ser pesquisados por módulos, como URLs, ou consultas ao banco de dados. Apenas strings devem estar presentes em **sys.path**; todos os outros tipos de dados são ignorados.

O localizador baseado no caminho é um localizador de metacaminho, então o mecanismo de importação inicia a pesquisa no caminho de importação chamando o método **find_spec()** do localizador baseado no caminho conforme descrito anteriormente. Quando o argumento `path` para **find_spec()** for fornecido, será uma lista de caminhos de string a serem percorridos – normalmente o atributo `__path__` de um pacote para uma importação dentro desse pacote. Se o argumento `path` for `None`, isso indica uma importação de nível superior e **sys.path** é usado.

O localizador baseado no caminho itera sobre cada entrada no caminho de pesquisa e, para cada uma delas, procura um localizador de entrada de caminho (**PathEntryFinder**) apropriado para a entrada do caminho. Como esta pode ser uma operação custosa (por exemplo, pode haver sobrecargas de chamada `stat()` para esta pesquisa), o localizador baseado no caminho mantém um cache mapeando entradas de caminho para localizadores de entrada de caminho. Este cache é mantido em **sys.path_importer_cache**

(apesar do nome, este cache na verdade armazena objetos localizadores em vez de ser limitado a objetos importador). Desta forma, a dispendiosa busca pelo localizador de entrada de caminho de local específico de uma entrada de caminho só precisa ser feita uma vez. O código do usuário é livre para remover entradas de cache de **`sys.path_importer_cache`**, forçando o localizador baseado no caminho a realizar a pesquisa de entrada de caminho novamente.

Se a entrada de caminho não estiver presente no cache, o localizador baseado no caminho itera sobre cada chamável em **`sys.path_hooks`**. Cada um dos ganchos de entrada de caminho nesta lista é chamado com um único argumento, a entrada de caminho a ser pesquisada. Este chamável pode retornar um localizador de entrada de caminho que pode manipular a entrada de caminho ou pode levantar **`ImportError`**. Um **`ImportError`** é usado pelo localizador baseado no caminho para sinalizar que o gancho não consegue encontrar um localizador de entrada de caminho para aquela entrada de caminho. A exceção é ignorada e a iteração com o caminho de importação continua. O gancho deve esperar um objeto string ou bytes; a codificação de objetos bytes depende do gancho (por exemplo, pode ser uma codificação de sistema de arquivos, UTF-8 ou outra coisa) e, se o gancho não puder decodificar o argumento, ele deve levantar **`ImportError`**.

Se a iteração **`sys.path_hooks`** terminar sem que nenhum localizador de entrada de caminho seja retornado, o método **`find_spec()`** do localizador baseado no caminho armazenará `None` em **`sys.path_importer_cache`** (para indicar que não há um localizador para esta entrada de caminho) e retornará `None`, indicando que este localizador de metacaminho não conseguiu encontrar o módulo.

Se um localizador de entrada de caminho *for* retornado por um dos chamáveis de gancho de entrada de caminho em **`sys.path_hooks`**, então o seguinte protocolo é usado para solicitar ao localizador um spec de módulo, que é então usada ao carregar o módulo.

O diretório de trabalho atual – denotado por uma string vazia – é tratado de forma ligeiramente diferente de outras entradas em **`sys.path`**. Primeiro, se o diretório de trabalho atual não puder ser determinado ou for considerado inexistente, nenhum valor será armazenado em **`sys.path_importer_cache`**.

Segundo, o valor para o diretório de trabalho atual é pesquisado novamente para cada pesquisa de módulo. Terceiro, o caminho usado para **`sys.path_importer_cache`** e retornado por **`importlib.machinery.PathFinder.find_spec()`** será o diretório de trabalho atual real e não a string vazia.

5.5.2. Protocolo do localizador de entrada de caminho

Para dar suporte a importações de módulos e pacotes inicializados e também contribuir com partes para pacotes de espaço de nomes, os localizadores de entrada de caminho devem implementar o método **`find_spec()`**.

`find_spec()` recebe dois argumentos: o nome totalmente qualificado do módulo que está sendo importado e o módulo de destino (opcional). `find_spec()` retorna um spec totalmente preenchido para o módulo. Este spec sempre terá "loader" definido (com uma exceção).

Para indicar ao maquinário de importação que o spec representa uma porção de espaço de nomes, o localizador de entrada de caminho define `submodule_search_locations` como uma lista contendo a porção.

Alterado na versão 3.4: **`find_spec()`** substituiu **`find_loader()`** e **`find_module()`**, ambos descontinuados, mas serão usados se `find_spec()` não estiver definido.

Os localizadores de entrada de caminho mais antigos podem implementar um desses dois métodos descontinuados em vez de `find_spec()`. Os métodos ainda são respeitados para fins de compatibilidade com versões anteriores. No entanto, se `find_spec()` for implementado no localizador de entrada de caminho, os métodos legados serão ignorados.

`find_loader()` recebe um argumento, o nome totalmente qualificado do módulo que está sendo importado. `find_loader()` retorna uma tupla 2 onde o primeiro item é o carregador e o segundo item é uma porção de espaço de nomes.

Para compatibilidade com versões anteriores de outras implementações do protocolo de importação, muitos localizadores de entrada de caminho também dão suporte ao mesmo método tradicional `find_module()` que os localizadores de metacaminho. No entanto, os métodos `find_module()` do localizador de entrada de caminho nunca são chamados com um argumento `path` (espera-se que eles registrem as informações de caminho apropriadas da chamada inicial para o gancho de caminho).

O método `find_module()` em localizadores de entrada de caminho foi descontinuado, pois não permite que o localizador de entrada de caminho contribua com porções para pacotes de espaço de nomes. Se `find_loader()` e `find_module()` existirem em um localizador de entrada de caminho, o sistema de importação sempre chamará `find_loader()` em preferência a `find_module()`.

Alterado na versão 3.10: Chamadas para **`find_module()`** e **`find_loader()`** pelo sistema de importação vão levantar **`ImportWarning`**.

Alterado na versão 3.12: `find_module()` e `find_loader()` foram removidos.

5.6. Substituindo o sistema de importação padrão

O mecanismo mais confiável para substituir todo o sistema de importação é excluir o conteúdo padrão de **`sys.meta_path`**, substituindo-o inteiramente por um gancho de metacaminho personalizado.

Se for aceitável alterar apenas o comportamento de instruções de importação sem afetar outras APIs que acessam o sistema de importação, então substituir a função embutida **`__import__()`** pode ser suficiente.

Para impedir seletivamente a importação de alguns módulos de um gancho no início do metacaminho (em vez de desabilitar o sistema de importação padrão completamente), é suficiente levantar **`ModuleNotFoundError`** diretamente de **`find_spec()`** em vez de retornar `None`. O último indica que a busca do metacaminho deve continuar, enquanto levantar uma exceção a encerra imediatamente.

5.7. Importações relativas ao pacote

Importações relativas usam caracteres de ponto no início. Um único ponto no início indica uma importação relativa, começando com o pacote atual. Dois ou mais pontos no início indicam uma importação relativa para o(s) pai(s) do pacote atual, um nível por ponto após o primeiro. Por exemplo, dado o seguinte layout de pacote:

```
package/
    __init__.py
    subpackage1/
        __init__.py
        moduleX.py
        moduleY.py
    subpackage2/
        __init__.py
        moduleZ.py
    moduleA.py
```

Em `subpackage1/moduleX.py` ou `subpackage1/__init__.py`, as seguintes são importações relativas válidas:

```
from .moduleY import spam
from .moduleY import spam as ham
from . import moduleY
from ..subpackage1 import moduleY
from ..subpackage2.moduleZ import eggs
from ..moduleA import foo
```

Importações absolutas podem usar a sintaxe `import <>` ou `from <> import <>`, mas importações relativas podem usar apenas a segunda forma; o motivo para isso é que:

```
import XXX.YYY.ZZZ
```

deve expor `XXX.YYY.ZZZ` como uma expressão utilizável, mas .moduleY não é uma expressão válida.

5.8. Considerações especiais para __main__

O módulo `__main__` é um caso especial em relação ao sistema de importação do Python. Conforme observado em em outro lugar, o módulo `__main__` é inicializado diretamente na inicialização do interpretador, muito parecido com `sys` e `builtins`. No entanto, diferentemente desses dois, ele não se qualifica estritamente como um módulo integrado. Isso ocorre porque a maneira como `__main__` é inicializado depende dos sinalizadores e outras opções com as quais o interpretador é invocado.

5.8.1. __main__.__spec__

Dependendo de como `__main__` é inicializado, `__main__.__spec__` é definido apropriadamente ou como `None`.

Quando o Python é iniciado com a opção `-m`, `__spec__` é definido como o spec de módulo ou pacote correspondente. `__spec__` também é preenchido quando o módulo `__main__` é carregado como parte da execução de um diretório, arquivo zip ou outra entrada `sys.path`.

Nos demais casos, `__main__.__spec__` é definido como `None`, pois o código usado para preencher o `__main__` não corresponde diretamente a um módulo importável:

- prompt interativo
- opção `-c`
- executar a partir de stdin
- executar diretamente de um arquivo de código-fonte ou bytecode

Note que `__main__.__spec__` é sempre `None` no último caso, *mesmo se* o arquivo pudesse ser importado diretamente como um módulo. Use a opção `-m` se metadados de módulo válidos forem desejados em `__main__`.

Note também que mesmo quando `__main__` corresponde a um módulo importável e `__main__.__spec__` é definido adequadamente, eles ainda são considerados módulos *distintos*. Isso se deve ao fato de que os blocos protegidos por verificações `if __name__ == "__main__":` são executados somente quando o módulo é usado para preencher o espaço de nomes `__main__`, e não durante a importação normal.

5.9. Referências

O maquinário de importação evoluiu consideravelmente desde os primeiros dias do Python. A especificação original para pacotes [https://www.python.org/doc/essays/packages/] ainda está disponível para leitura, embora alguns detalhes tenham mudado desde a escrita desse documento.

A especificação original para `sys.meta_path` era **PEP 302** [https://peps.python.org/pep-0302/], com extensão subsequente em **PEP 420** [https://peps.python.org/pep-0420/].

PEP 420 [https://peps.python.org/pep-0420/] introduziu pacotes de espaço de nomes para Python 3.3. **PEP 420** [https://peps.python.org/pep-0420/] também introduziu o protocolo `find_loader()` como uma alternativa ao `find_module()`.

PEP 366 [https://peps.python.org/pep-0366/] descreve a adição do atributo `__package__` para importações relativas explícitas em módulos principais.

PEP 328 [https://peps.python.org/pep-0328/] introduziu importações relativas absolutas e explícitas e inicialmente propôs `__name__` para semântica. **PEP 366** [https://peps.python.org/pep-0366/] eventualmente especificaria `__package__`.

PEP 338 [https://peps.python.org/pep-0338/] define módulos de execução como scripts.

PEP 451 [https://peps.python.org/pep-0451/] adiciona o encapsulamento do estado de importação por módulo em objetos spec. Ele também descarrega a maioria das responsabilidades inerentes dos carregadores de volta para o maquinário de importação. Essas mudanças permitem a descontinuação de várias APIs no sistema de importação e também a adição de novos métodos para localizadores e carregadores.

Notas de rodapé

[1]

Veja `types.ModuleType`.

[2]

A implementação de importlib evita usar o valor de retorno diretamente. Em vez disso, ela obtém o objeto do módulo procurando o nome do módulo em `sys.modules`. O efeito indireto disso é que um módulo importado pode substituir a si mesmo em `sys.modules`. Esse é um comportamento específico da implementação que não tem garantia de funcionar em outras implementações do Python.

6. Expressões

Este capítulo explica o significado dos elementos das expressões em Python.

Notas sobre a sintaxe: Neste capítulo e nos subsequentes, será utilizada a notação de gramática para descrever a sintaxe, em vez da análise léxica.

Quando (uma alternativa de) uma regra de sintaxe tem a forma:

```
name: othername
```

e nenhuma semântica é fornecida, a semântica desta forma de `name` é a mesma que para `othername`.

6.1. Conversões aritméticas

Quando a descrição de um operador aritmético abaixo utiliza a frase "os argumentos numéricos são convertidos para um tipo real comum", isso significa que a implementação do operador para tipos numéricos embutidos funciona conforme descrito na seção Tipos numéricos da documentação da biblioteca padrão.

Algumas regras adicionais se aplicam a certos operadores e operadores não-numéricos (por exemplo, uma string como um argumento à esquerda para o operador `%`). As extensões devem definir seu próprio comportamento de conversão.

6.2. Átomos

Átomos são os elementos mais básicos de expressões. Os átomos mais simples são constantes embutidas, nomes e literais. Átomos mais complexos são delimitados por pares de delimitadores:

- `()` (parênteses): grupos, exibições de tuplas, átomos de yield e expressões geradoras;
- `[]` (colchetes): exibições de lista;
- `{}` (chaves): exibições de dicionário e conjunto.

Formalmente, a sintaxe para átomo é:

```
atom:
   | builtin_constant
   | identifier
   | literal
   | parenthesized_enclosure
   | bracketed_enclosure
   | braced_enclosure
parenthesized_enclosure:
   | group
   | tuple
   | yield_atom
   | generator_expression
bracketed_enclosure:
   | listcomp
   | list
braced_enclosure:
   | dictcomp
   | dict
   | setcomp
   | set
```

6.2.1. Constantes embutidas

As palavras reservadas `True`, `False` e `None` nomeiam constantes embutidas. O token `...` nomeia a constante `Ellipsis`.

A avaliação desses átomos produz o valor correspondente.

Nota

Várias outras constantes embutidas estão disponíveis como variáveis globais, mas apenas aquelas mencionadas aqui são palavras reservadas.

Em particular, esses nomes não podem ser reatribuídos nem usados como atributos:

```
>>> False = 123
  File "<input>", line 1
   False = 123
   ^^^^^
SyntaxError: cannot assign to False
```

Formalmente, a sintaxe para constantes embutidas é:

```
builtin_constant: 'True' | 'False' | 'None' | '...'
```

6.2.2. Identificadores (Nomes)

Um identificador que ocorre como um átomo é um nome. Veja a seção Nomes (identificadores e palavras reservadas) para a definição lexical e a seção Nomeação e ligação para documentação de nomenclatura e ligação.

Quando o nome está vinculado a um objeto, a avaliação do átomo produz esse objeto. Quando um nome não está vinculado, uma tentativa de avaliá-lo levanta uma exceção `NameError`.

6.2.2.1. Desfiguração de nome privado

Quando um identificador que ocorre textualmente em uma definição de classe começa com dois ou mais caracteres de sublinhado e não termina com dois ou mais sublinhados, ele é considerado um *nome privado* daquela classe.

Ver também

As especificações de classe.

Mais precisamente, os nomes privados são transformados em um formato mais longo antes que o código seja gerado para eles. Se o nome transformado tiver mais de 255 caracteres, poderá ocorrer truncamento definido pela implementação.

A transformação é independente do contexto sintático no qual o identificador é usado, mas apenas os seguintes identificadores privados são desfigurados:

- Qualquer nome usado como nome de uma variável que é atribuída ou lida ou qualquer nome de um atributo que está sendo acessado.

  O atributo `__name__` de funções aninhadas, classes e apelidos de tipo, entretanto, não é desfigurado.

- O nome dos módulos importados, por exemplo, `__spam` em `import __spam`. Se o módulo faz parte de um pacote (ou seja, seu nome contém um ponto), o nome *não* é desfigurado, por exemplo, o `__foo` em `import __foo.bar` não é desfigurado.

- O nome de um membro importado, por exemplo, `__f` em `from spam import __f`.

A regra de transformação está definida da seguinte forma:

- O nome da classe, com os sublinhados iniciais removidos e um único sublinhado inicial inserido, é inserido na frente do identificador, por exemplo, o identificador `__spam` ocorrendo em uma classe chamada `Foo`, `_Foo` ou `__Foo` é transformado em `_Foo__spam`.
- Se o nome da classe consiste apenas em sublinhados, a transformação é a identidade, por exemplo, o identificador `__spam` que ocorre em uma classe chamada _ ou __ é deixado como está.

6.2.3. Literais

Um *literal* é uma representação textual de um valor. O Python oferece suporte a literais numéricos, de string e de bytes. strings de formato e string template são tratados como literais de string.

Literais numéricos consistem em um único token **NUMBER**, que representa um número inteiro, um número de ponto flutuante ou um número imaginário. Consulte a seção Literais numéricos na documentação de análise léxica para obter detalhes.

Literais de string e de bytes podem consistir em vários tokens. Consulte a seção Concatenação de literal de string para obter detalhes.

Observe que números negativos e números complexos, como `-3` ou `3+4.2j`, não são sintaticamente literais, mas sim operações aritméticas unárias ou binárias envolvendo o operador `-` ou `+`.

A avaliação de um literal produz um objeto do tipo fornecido (**int**, **float**, **complex**, **str**, **bytes** ou **Template**) com o valor fornecido. O valor pode ser aproximado no caso de literais de ponto flutuante e imaginários.

A gramática formal para literais é:

```
literal: strings | NUMBER
```

6.2.3.1. Identidade de objeto e literais

Todos os literais correspondem a tipos de dados imutáveis e, portanto, a identidade do objeto é menos importante que seu valor. Múltiplas avaliações de literais com o mesmo valor (seja a mesma ocorrência no texto do programa ou uma ocorrência diferente) podem obter o mesmo objeto ou um objeto diferente com o mesmo valor.

Detalhes da implementação do CPython:

Por exemplo, no CPython, números inteiros *pequenos* com o mesmo valor são avaliados como o mesmo objeto:

```
>>> x = 7
>>> y = 7
>>> x is y
True
```

No entanto, números inteiros grandes são avaliados como objetos diferentes:

```
>>> x = 123456789
>>> y = 123456789
>>> x is y
False
```

Esse comportamento pode mudar em versões futuras do CPython. Em particular, o limite entre números inteiros “pequenos” e “grandes” já mudou no passado.

O CPython emitirá um **`SyntaxWarning`** ao comparar literais usando `is`:

```
>>> x = 7
>>> x is 7
<input>:1: SyntaxWarning: "is" with 'int' literal. Did you mea
True
```

Veja Quando eu posso depender dos testes de identidade com o operador is? para mais informações.

Strings template são imutáveis, mas podem referenciar objetos mutáveis como valores de **`Interpolation`**. Para os fins desta seção, duas t-strings têm o “mesmo valor” se tanto a sua estrutura quanto a *identidade* dos valores coincidirem.

Atualmente, cada avaliação de uma string template resulta em um objeto diferente.

6.2.3.2. Concatenação de literal de string

Vários literais de strings ou bytes adjacentes, possivelmente usando convenções de aspas diferentes, são permitidos, e seu significado é o mesmo que sua concatenação:

```
>>> "olá" 'mundo'
"olámundo"
```

Este recurso é definido no nível sintático, portanto, funciona apenas com literais. Para concatenar expressões de string em tempo de execução, o operador '+' pode ser usado:

```
>>> greeting = "Olá"
>>> space = " "
>>> name = "Blaise"
>>> print(greeting + space + name)   # not: print(greeting space
Olá Blaise
```

A concatenação de literais pode misturar livremente strings brutas, strings entre aspas triplas e literais de strings formatadas. Por exemplo:

```
>>> "Olá" r', ' f"{name}!"
"Olá, Blaise!"
```

Este recurso pode ser usado para reduzir o número de contrabarras necessárias, para dividir strings longas convenientemente em linhas longas ou até mesmo para adicionar comentários a partes de strings. Por exemplo:

```
re.compile("[A-Za-z_]"       # letra ou sublinhado
           "[A-Za-z0-9_]*"   # letra, dígito ou sublinhado
          )
```

No entanto, literais de bytes só podem ser combinados com outros literais de bytes; não com literais de string de qualquer tipo. Além disso, literais de strings templates só podem ser combinados com outros literais de strings templates:

```
>>> t"Olá" t"{name}!"
Template(strings=('Olá', '!'), interpolations=(...))
```

Formalmente:

```
strings: (STRING | fstring)+ | tstring+
```

6.2.4. Grupos de parênteses

Um *grupo de parênteses* é uma expressão delimitada por parênteses. O grupo é avaliado com o mesmo valor da expressão contida nele.

Os grupos são usados para substituir ou esclarecer a precedência do operador, da mesma forma que na notação matemática. Por exemplo:

```
>>> 3 << 2 | 4
12
>>> 3 << (2 | 4)   # Substitui a precedência do | (OU bit a bit)
192
>>> (3 << 2) | 4   # Mesmo que sem parênteses (mas mais claro)
12
```

Observe que nem tudo que está entre parênteses é um *grupo*. Especificamente, um grupo entre parênteses deve conter exatamente uma expressão e não pode terminar com uma vírgula. Consulte exibições de tupla e expressões geradoras para outras formas entre parênteses.

Formalmente, a sintaxe para grupos é:

```
group: '(' assignment expression ')'
```

6.2.5. Exibições de contêiner

For constructing builtin containers (lists, sets, tuples or dictionaries), Python provides special syntax called *displays*. There are subtle differences between the four kinds of displays, detailed in the following sections. All displays, however, consist of comma-separated items enclosed in paired delimiters.

For example, a *list display* is a series of expressions enclosed in square brackets:

```
>>> ["one", "two", "three"]
['one', 'two', 'three']
```

```
>>> [1 + 2, 2 + 3]
[3, 5]
```

In list, tuple and dictionary (but not set) displays, the series may be empty:

```
>>> []  # empty list
[]
>>> ()  # empty tuple
()
>>> {}  # empty dictionary
{}
```

If the series is not empty, the items may be followed by an additional comma, which has no effect:

```
>>> ["one", "two", "three",]  # note comma after "three"
['one', 'two', 'three']
```

Nota

The trailing comma is often used for displays that span multiple lines (using implicit line joining), so when a future programmer adds a new entry at the end, they do not need to modify an existing line:

```
>>> [
...     'one',
...     'two',
...     'three',
... ]
['one', 'two', 'three']
```

At runtime, when a display is evaluated, the listed items are evaluated from left to right and placed into a new container of the appropriate type.

For tuple, list and set (but not dict) displays, any item in the display may be prefixed with an asterisk (`*`). This denotes iterable unpacking. At runtime, the asterisk-prefixed expression must evaluate to an iterable, whose contents are inserted into the container at the location of the unpacking. For example:

```
>>> numbers = (1, 2)
>>> [*numbers, 'word', *numbers]
[1, 2, 'word', 1, 2]
```

Dictionary displays use a similar mechanism called *dictionary unpacking*, denoted with a double asterisk (`**`). See Sintaxes de criação de dicionário for details.

A more advanced form of displays are *comprehensions*, where items are computed via a set of looping and filtering instructions. See the Compreensões section for details.

Adicionado na versão 3.5: Iterable and dictionary unpacking in displays, originally proposed by **PEP 448** [https://peps.python.org/pep-0448/].

6.2.5.1. Sintaxes de criação de lista

A *list display* is a possibly empty series of expressions enclosed in square brackets. For example:

```
>>> ["one", "two", "three"]
['one', 'two', 'three']
>>> ["one"]  # One-element list
['one']
>>> []       # empty list
[]
```

See Exibições de contêiner for general information on displays.

The formal grammar for list displays is:

```
list: '[' [flexible expression list] ']'
```

6.2.5.2. Sintaxes de criação de conjunto

A *set display* is a *non-empty* series of expressions enclosed in curly braces. For example:

```
>>> {"one", "two", "three"}
{'one', 'three', 'two'}
>>> {"one"}  # One-element set
{'one'}
```

See Exibições de contêiner for general information on displays.

There is no special syntax for the empty set. The `{}` literal is a dictionary display that constructs an empty dictionary. Call `set()` with no arguments to get an empty set.

The formal grammar for set displays is:

```
set: '{' flexible expression list '}'
```

6.2.5.3. Tuple displays

A *tuple display* is a series of expressions enclosed in parentheses. For example:

```
>>> (1, 2)
(1, 2)
>>> ()  # uma tupla vazia
()
```

See Exibições de contêiner for general information on displays.

To avoid ambiguity, if a tuple display has exactly one element, it requires a trailing comma. Without it, you get a parenthesized group:

```
>>> ('single',)  # single-element tuple
('single',)
>>> ('single')   # no comma: single string
'single'
```

To put it in other words, a tuple display is a parenthesized list of either:

- two or more comma-separated expressions, or
- zero or more expressions, each followed by a comma.

Since tuples are immutable, object identity rules for literals also apply to tuples: at runtime, two occurrences of tuples with the same values may or may not yield the same object.

Nota

Python's syntax also includes expression lists, where a comma-separated list of expressions is *not* enclosed in parentheses but evaluates to tuple.

In other words, when it comes to tuple syntax, the comma is more important that the use of parentheses. Only the empty tuple is spelled without a comma.

The formal grammar for tuple displays is:

```
tuple:
   | '(' flexible expression (',' flexible expression)+ [','] ')
   | '(' flexible expression ',' ')'
   | '(' ')'
```

6.2.5.4. Sintaxes de criação de dicionário

A *dictionary display* is a possibly empty series of *dict items* enclosed in curly braces. Each dict item is a colon-separated pair of expressions: the *key* and its associated *value*. For example:

```
>>> {1: 'one', 2: 'two'}
{1: 'one', 2: 'two'}
```

At runtime, when a dictionary comprehension is evaluated, the expressions are evaluated from left to right. Each key object is used as a key into the dictionary to store the corresponding value. This means that you can specify the same key multiple times in the comprehension, and the final dictionary's value for a given key will be the last one given. For example:

```
>>> {
...     1: 'this will be overridden',
...     2: 'two',
...     1: 'also overridden',
...     1: 'one',
... }
{1: 'one', 2: 'two'}
```

Instead of a key-value pair, a dict item may be an expression prefixed by a double asterisk `**`. This denotes *dictionary unpacking*. At runtime, the expression must evaluate to a mapping; each item of the mapping is added to the new dictionary. As with key-value pairs, later values replace values already set by earlier items and unpackings. This may be used to override a set of defaults:

```
>>> defaults = {'color': 'blue', 'count': 8}
>>> overrides = {'color': 'yellow'}
>>> {**defaults, **overrides}
{'color': 'yellow', 'count': 8}
```

Adicionado na versão 3.5: Desempacotando em sintaxes de criação de dicionário, originalmente proposto pela **PEP 448** [https://peps.python.org/pep-0448/].

The formal grammar for dict displays is:

```
dict:                    '{' [double_starred_kvpairs] '}'
double_starred_kvpairs: ','.double_starred_kvpair+ [',']
double_starred_kvpair:  '**' or_expr | kvpair
kvpair:                 expression ':' expression
```

6.2.6. Compreensões

List, set and dictionary *comprehensions* are a form of container displays where items are computed via a set of looping and filtering instructions rather than listed explicitly.

In its simplest form, a comprehension consists of a single expression followed by a **`for`** clause. The **`for`** clause has the same syntax as the header

of a for statement, without a trailing colon.

For example, a list of the first ten squares is:

```
>>> [x**2 for x in range(10)]
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

At run time, a list comprehension creates a new list. The expression after **`in`** must evaluate to an iterable. For each element of this iterable, the element is bound to the **`for`** clause's target as in a **`for`** statement, then the expression before **`for`** is evaluated with the target in scope and the result is added to the new list. Thus, the example above is roughly equivalent to defining and calling the following function:

```
def make_list_of_squares(iterable):
    result = []
    for x in iterable:
        result.append(x**2)
    return result

make_list_of_squares(range(10))
```

Set comprehensions work similarly. For example, here is a set of lowercase letters:

```
>>> {x.lower() for x in ['a', 'A', 'b', 'C']}
{'c', 'a', 'b'}
```

At run time, this corresponds roughly to calling this function:

```
def make_lowercase_set(iterable):
    result = set(iterable)
    for x in iterable:
        result.append(x.lower())
    return result

make_lowercase_set(['a', 'A', 'b', 'C'])
```

Dictionary comprehensions start with a colon-separated key-value pair instead of an expression. For example:

```
>>> {func.__name__: func for func in [print, hex, any]}
{'print': <built-in function print>,
 'hex': <built-in function hex>,
 'any': <built-in function any>}
```

At run time, this corresponds roughly to:

```
def make_dict_mapping_names_to_functions(iterable):
    result = {}
    for func in iterable:
        result[func.__name__] = func
    return result

iterable([print, hex, any])
```

As in other kinds of dictionary displays, the same key may be specified multiple times. Earlier values are overwritten by ones that are evaluated later.

There are no *tuple comprehensions*. A similar syntax is instead used for generator expressions, from which you can construct a tuple like this:

```
>>> tuple(x**2 for x in range(10))
(0, 1, 4, 9, 16, 25, 36, 49, 64, 81)
```

Alterado na versão 3.8: Antes do Python 3.8, em compreensões de dict, a ordem de avaliação de chave e valor não era bem definida. No CPython, o valor foi avaliado antes da chave. A partir de 3.8, a chave é avaliada antes do valor, conforme proposto pela **PEP 572** [https://peps.python.org/pep-0572/].

6.2.6.1. Filtering in comprehensions

The **`for`** clause may be followed by an **`if`** clause with an expression.

For example, a list of names from the **`math`** module that start with `f` is:

```
>>> [name for name in vars(math) if name.startswith('f')]
['fabs', 'factorial', 'floor', 'fma', 'fmod', 'frexp', 'fsum']
```

At run time, the expression after **`if`** is evaluated before each element is added to the resulting container, and if it is false, the element is skipped. Thus, the above example roughly corresponds to defining and calling the following function:

```
def get_math_f_names(iterable):
    result = []
    for name in iterable:
        if name.startswith('f'):
           result.append(name)
    return result

get_math_f_names(vars(math))
```

Filtering is a special case of more complex comprehensions. See the next section for a more formal description.

6.2.6.2. Complex comprehensions

Generally, a comprehension's initial **`for`** clause may be followed by zero or more additional **`for`** or **`if`** clauses. For example, here is a list of names exposed by two Python modules, filtered to only include names that start with `a`:

```
>>> import array
>>> import math
>>> [
...     name
...     for module in [array, math]
...     for name in vars(module)
...     if name.startswith('a')
... ]
['array', 'acos', 'acosh', 'asin', 'asinh', 'atan', 'atan2', 'at
```

At run time, this roughly corresponds to defining and calling:

```
def get_a_names(iterable):
    result = []
    for module in iterable:
```

```
        for name in vars(module):
            if name.startswith('a'):
                result.append(name)
    return result

get_a_names([array, math])
```

The elements of the new container are those that would be produced by considering each of the **`for`** or **`if`** clauses a block, nesting from left to right, and evaluating the expression to produce an element (or dictionary entry) each time the innermost block is reached.

Aside from the iterable expression in the leftmost **`for`** clause, the comprehension is executed in a separate implicitly nested scope. This ensures that names assigned to in the target list don't "leak" into the enclosing scope. For example:

```
>>> x = 'old value'
>>> [x**2 for x in range(10)]  # this `x` is local to the compre
>>> x
'old value'
```

The iterable expression in the leftmost **`for`** clause is evaluated directly in the enclosing scope and then passed as an argument to the implicitly nested scope.

Subsequent **`for`** clauses and any filter condition in the leftmost **`for`** clause cannot be evaluated in the enclosing scope as they may depend on the values obtained from the leftmost iterable.

Para garantir que a compreensão sempre resulte em um contêiner do tipo apropriado, as expressões `yield` e `yield from` são proibidas no escopo aninhado implicitamente.

Assignment expressions are not allowed inside comprehension iterable expressions (that is, the expressions after the **`in`** keyword), nor anywhere within comprehensions that appear directly in a class definition.

Alterado na versão 3.8: `yield` e `yield from` proibidos no escopo aninhado implícito.

6.2.6.3. Unpacking in comprehensions

If the expression of a list or set comprehension is starred, the result will be unpacked to produce zero or more elements.

This is often used for “flattening” lists, for example:

```
>>> students = ['Petr', 'Blaise', 'Jarka']
>>> teachers = ['Salim', 'Bartosz']
>>> lists_of_people = [students, teachers]
>>> [*people for people in lists_of_people]
['Petr', 'Blaise', 'Jarka', 'Salim', 'Bartosz']
```

At run time, this comprehension roughly corresponds to:

```
def flatten_names(lists_of_people):
    result = []
    for people in lists_of_people:
        result.extend(people)
    return result
```

In dict comprehensions, a double-starred expression will be evaluated and then unpacked using dictionary unpacking, inserting zero or more key/value pairs into the new dictionary. As in other kinds of dictionary displays, if the same key is specified multiple times, the associated value in the resulting dictionary will be the last one specified.

Por exemplo:

```
>>> system_defaults = {'color': 'blue', 'count': 8}
>>> user_defaults = {'color': 'yellow'}
>>> overrides = {'count': 5}

>>> configuration_sets = [system_defaults, user_defaults, overri

>>> {**d for d in configuration_sets}
{'color': 'yellow', 'count': 5}
```

Adicionado na versão 3.15: Unpacking in comprehensions using the `*` and `**` operators was introduced in **PEP 798** [https://peps.python.org/pep-0798/].

6.2.6.4. Asynchronous comprehensions

In an `async def` function, an `async for` clause may be used to iterate over a asynchronous iterator. A comprehension in an `async def` function may consist of either a `for` or `async for` clause following the leading expression, may contain additional `for` or `async for` clauses, and may also use `await` expressions.

If a comprehension contains `async for` clauses, or if it contains `await` expressions or other asynchronous comprehensions anywhere except the iterable expression in the leftmost `for` clause, it is called an *asynchronous comprehension*. An asynchronous comprehension may suspend the execution of the coroutine function in which it appears.

Adicionado na versão 3.6: Asynchronous comprehensions were introduced in **PEP 530** [https://peps.python.org/pep-0530/].

Alterado na versão 3.11: Compreensões assíncronas agora são permitidas dentro de compreensões em funções assíncronas. As compreensões externas tornam-se implicitamente assíncronas.

6.2.6.5. Gramática formal para compreensões

The formal grammar for comprehensions is:

```
listcomp:      '[' comprehension ']'
setcomp:       '{' comprehension '}'
comprehension: flexible_expression for_if_clause+

dictcomp:
    | '{' kvpair for_if_clause+ '}'
    | '{' '**' expression for_if_clause+ '}'
```

```
for_if_clause:
    | ['async'] 'for' target_list 'in' or_test ('if' or_test)*
```

6.2.7. Expressões geradoras

The syntax for *generator expressions* is the same as for list comprehensions, except that they are enclosed in parentheses instead of brackets. For example:

```
>>> iterator = (x ** 2 for x in range(10))
>>> iterator
<generator object <genexpr> at ...>
```

At runtime, a generator expression evaluates to a generator iterator which yields the same values as the corresponding list comprehension:

```
>>> list(iterator)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Thus, the example above is roughly equivalent to defining and calling the following generator function:

```
def make_generator_of_squares(iterator):
    for x in iterator:
        yield x ** 2

make_generator_of_squares(iter(range(10)))
```

The enclosing parentheses can be omitted in calls when the generator expression is the only positional argument and there are no keyword arguments. See the Calls section for details. For example:

```
# The parentheses after `sum` are part of the call syntax:
>>> sum(x ** 2 for x in range(10))
285

# The generator needs its own parentheses if it's not the only a
>>> sum((x ** 2 for x in range(10)), start=1000)
1285
```

The iterable expression in the leftmost **`for`** clause is evaluated immediately, so that an error raised by this expression will be emitted at the point where the generator expression is defined, rather than at the point where the first value is retrieved:

```
>>> (x ** 2 for x in nonexistent_iterable)
Traceback (most recent call last):
  ...
NameError: name 'nonexistent_iterable' is not defined
```

After the expression is evaluated, an iterator is created from the result, as if **`iter()`** was called on it. Any error raised when creating the iterator is also emitted immediately:

```
>>> (x ** 2 for x in None)
Traceback (most recent call last):
  ...
TypeError: 'NoneType' object is not iterable
```

All other expressions are evaluated lazily, in the same fashion as normal generators (that is, when the iterator is asked to yield a value):

```
>>> iterator = (nonexistent_value for x in range(10))
>>> iterator
<generator object <genexpr> at ...>
>>> list(iterator)
Traceback (most recent call last):
  ...
NameError: name 'nonexistent_value' is not defined

>>> iterator = (x * y for x in range(10) for y in nonexistent_it
>>> iterator
<generator object <genexpr> at ...>
>>> list(iterator)
Traceback (most recent call last):
  ...
NameError: name 'nonexistent_iterable' is not defined
```

To avoid interfering with the expected operation of the generator expression itself, `yield` and `yield from` expressions are prohibited inside the

implicitly nested scope.

If a generator expression contains either **`async for`** clauses or **`await`** expressions it is called an *asynchronous generator expression*. An asynchronous generator expression returns a new asynchronous generator object, which is an asynchronous iterator (see Iteradores assíncronos).

The formal grammar for generator expressions is:

```
generator_expression: "(" comprehension ")"
```

Adicionado na versão 3.6: Expressões geradoras assíncronas foram introduzidas.

Alterado na versão 3.7: Antes do Python 3.7, as expressões geradoras assíncronas só podiam aparecer em corrotinas **`async def`**. A partir da versão 3.7, qualquer função pode usar expressões geradoras assíncronas.

Alterado na versão 3.8: `yield` e `yield from` proibidos no escopo aninhado implícito.

6.2.8. Expressões yield

```
yield_atom:       "(" yield_expression ")"
yield_from:       "yield" "from" expression
yield_expression: "yield" yield_list | yield_from
```

A expressão yield é usada ao definir uma função geradora ou uma função geradora assíncrona e, portanto, só pode ser usada no corpo de uma definição de função. Usar uma expressão yield no corpo de uma função faz com que essa função seja uma função geradora, e usá-la no corpo de uma função **`async def`** faz com que essa função de corrotina seja uma função geradora assíncrona. Por exemplo:

```
def gen():  # define uma função geradora
    yield 123
```

```
async def agen(): # define uma função geradora assíncrona
    yield 123
```

Devido a seus efeitos colaterais no escopo recipiente, as expressões `yield` não são permitidas como parte dos escopos definidos implicitamente usados para implementar compreensões e expressões geradoras.

Alterado na versão 3.8: Expressões yield proibidas nos escopos aninhados implicitamente usados para implementar compreensões e expressões geradoras.

As funções geradoras são descritas abaixo, enquanto as funções geradoras assíncronas são descritas separadamente na seção Funções geradoras assíncronas

Quando uma função geradora é chamada, ela retorna um iterador conhecido como gerador. Esse gerador então controla a execução da função geradora. A execução começa quando um dos métodos do gerador é chamado. Nesse momento, a execução segue para a primeira expressão yield, onde é suspensa novamente, retornando o valor de **`yield_list`** ao chamador do gerador, ou `None` se **`yield_list`** é omitido. Por suspenso, queremos dizer que todo o estado local é retido, incluindo as chamadas atuais de variáveis locais, o ponteiro de instrução, a pilha de avaliação interna e o estado de qualquer tratamento de exceção. Quando a execução é retomada chamando um dos métodos do gerador, a função pode prosseguir exatamente como se a expressão yield fosse apenas outra chamada externa. O valor da expressão yield após a retomada depende do método que retomou a execução. Se **`__next__()`** for usado (tipicamente através de uma **`for`** ou do **`next()`** embutido) então o resultado será **`None`**. Caso contrário, se **`send()`** for usado, o resultado será o valor passado para esse método.

Tudo isso torna as funções geradoras bastante semelhantes às corrotinas; cedem múltiplas vezes, possuem mais de um ponto de entrada e sua execução pode ser suspensa. A única diferença é que uma função geradora não pode controlar onde a execução deve continuar após o seu rendimento; o controle é sempre transferido para o chamador do gerador.

Expressões yield são permitidas em qualquer lugar em uma construção `try`. Se o gerador não for retomado antes de ser finalizado (ao atingir uma contagem de referências zero ou ao ser coletado como lixo), o método `close()` do iterador de gerador será chamado, permitindo que quaisquer cláusulas `finally` pendentes sejam executadas.

Quando `yield from <expr>` é usado, a expressão fornecida deve ser iterável. Os valores produzidos pela iteração desse iterável são passados diretamente para o chamador dos métodos do gerador atual. Quaisquer valores passados com `send()` e quaisquer exceções passadas com `throw()` são passados para o iterador subjacente se ele tiver os métodos apropriados. Se este não for o caso, então **`send()`** irá levantar `AttributeError` ou `TypeError`, enquanto **`throw()`** irá apenas levantar a exceção passada imediatamente.

Quando o iterador subjacente estiver completo, o atributo `value` da instância `StopIteration` gerada torna-se o valor da expressão yield. Ele pode ser definido explicitamente ao levantar **`StopIteration`** ou automaticamente quando o subiterador é um gerador (retornando um valor do subgerador).

Alterado na versão 3.3: Adicionado `yield from <expr>` para delegar o fluxo de controle a um subiterador.

Os parênteses podem ser omitidos quando a expressão yield é a única expressão no lado direito de uma instrução de atribuição.

Ver também

PEP 255 [https://peps.python.org/pep-0255/] - Geradores simples

A proposta para adicionar geradores e a instrução `yield` ao Python.

PEP 342 [https://peps.python.org/pep-0342/] - Corrotinas via Geradores Aprimorados

A proposta de aprimorar a API e a sintaxe dos geradores, tornando-os utilizáveis como simples corrotinas.

PEP 380 **[https://peps.python.org/pep-0380/]** **- Sintaxe para Delegar a um Subgerador**

A proposta de introduzir a sintaxe `yield from`, facilitando a delegação a subgeradores.

PEP 525 **[https://peps.python.org/pep-0525/]** **- Geradores assíncronos**

A proposta que se expandiu em **PEP 492** [https://peps.python.org/pep-0492/] adicionando recursos de gerador a funções de corrotina.

6.2.8.1. Métodos de iterador gerador

Esta subseção descreve os métodos de um iterador gerador. Eles podem ser usados para controlar a execução de uma função geradora.

Observe que chamar qualquer um dos métodos do gerador abaixo quando o gerador já estiver em execução levanta uma exceção `ValueError`.

generator.__next__()

Inicia a execução de uma função geradora ou a retoma na última expressão yield executada. Quando uma função geradora é retomada com um método `__next__()`, a expressão yield atual sempre é avaliada como `None`. A execução então continua para a próxima expressão yield, onde o gerador é suspenso novamente, e o valor de `yield list` é retornado para o chamador de `__next__()`. Se o gerador sair sem produzir outro valor, uma exceção `StopIteration` será levantada.

Este método é normalmente chamado implicitamente, por exemplo por um laço `for`, ou pela função embutida `next()`.

generator.send(*value*)

Retoma a execução e “envia” um valor para a função geradora. O argumento *value* torna-se o resultado da expressão yield atual. O método `send()` retorna o próximo valor gerado pelo gerador, ou levanta `StopIteration` se o gerador sair sem produzir outro valor. Quando `send()` é chamado para iniciar o gerador, ele deve ser chamado com

`None` como argumento, porque não há nenhuma expressão yield que possa receber o valor.

generator.throw(*value*)

generator.throw(*type*[, *value*[, *traceback*]])

Levanta uma exceção no ponto em que o gerador foi pausado e retorna o próximo valor gerado pela função geradora. Se o gerador sair sem gerar outro valor, uma exceção `StopIteration` será levantada. Se a função geradora não detectar a exceção passada ou levanta uma exceção diferente, essa exceção se propagará para o chamador.

Em uso típico, isso é chamado com uma única instância de exceção semelhante à forma como a palavra reservada `raise` é usada.

Para compatibilidade com versões anteriores, no entanto, a segunda assinatura é suportada, seguindo uma convenção de versões mais antigas do Python. O argumento *type* deve ser uma classe de exceção e *value* deve ser uma instância de exceção. Se o *valor* não for fornecido, o construtor *tipo* será chamado para obter uma instância. Se *traceback* for fornecido, ele será definido na exceção, caso contrário, qualquer atributo `__traceback__` existente armazenado em *value* poderá ser limpo.

Alterado na versão 3.12: A segunda assinatura (tipo[, valor[, traceback]]) foi descontinuada e pode ser removida em uma versão futura do Python.

generator.close()

Levanta uma exceção `GeneratorExit` no ponto onde a função geradora foi pausada (equivalente a chamar `throw(GeneratorExit)`) A exceção é levantada pela expressão yield onde o gerador foi pausado. Se a função geradora captura a exceção, e retorna um valor, este valor é retornado de `close()`. Se a função geradora já estiver fechada ou levantar **`GeneratorExit`** (por não capturar a exceção), **`close()`** retornará `None`. Se o gerador produzir um valor, uma exceção `RuntimeError` é levantada. Se o gerador levantar qualquer outra exceção, ela será

propagada para o chamador. Se o gerador já saiu devido a uma exceção ou saída normal, **`close()`** retorna **`None`** e tem nenhum outro efeito.

Alterado na versão 3.13: Se um gerador retornar um valor ao ser fechado, o valor será retornado por **`close()`**.

6.2.8.2. Exemplos

Aqui está um exemplo simples que demonstra o comportamento de geradores e funções geradoras:

```
>>> def echo(value=None):
...     print("A execução inicia quando 'next()' é chamada pela
...     try:
...         while True:
...             try:
...                 value = (yield value)
...             except Exception as e:
...                 value = e
...     finally:
...         print("Não se esqueça de fazer uma limpeza quando 'c
...
>>> generator = echo(1)
>>> print(next(generator))
A execução inicia quando 'next()' é chamada pela primeira vez.
1
>>> print(next(generator))
None
>>> print(generator.send(2))
2
>>> generator.throw(TypeError, "spam")
TypeError('spam',)
>>> generator.close()
Não se esqueça de fazer uma limpeza quando 'close()' for chamada
```

Para exemplos usando `yield from`, consulte a PEP 380: Syntax for Delegating to a Subgenerator em “O que há de novo no Python.”

6.2.8.3. Funções geradoras assíncronas

A presença de uma expressão yield em uma função ou método definido usando a `async def` define ainda mais a função como uma função geradora assíncrona.

Quando uma função geradora assíncrona é chamada, ela retorna um iterador assíncrono conhecido como objeto gerador assíncrono. Esse objeto controla a execução da função geradora. Um objeto gerador assíncrono é normalmente usado em uma instrução `async for` em uma função de corrotina de forma análoga a como um objeto gerador seria usado em uma instrução `for`.

A chamada de um dos métodos do gerador assíncrono retorna um objeto aguardável, e a execução começa quando esse objeto é aguardado. Nesse momento, a execução prossegue até a primeira expressão yield, onde é suspensa novamente, retornando o valor de `yield_list` para a corrotina em aguardo. Assim como ocorre com um gerador, a suspensão significa que todo o estado local é mantido, inclusive as ligações atuais das variáveis locais, o ponteiro de instruções, a pilha de avaliação interna e o estado de qualquer tratamento de exceção. Quando a execução é retomada, aguardando o próximo objeto retornado pelos métodos do gerador assíncrono, a função pode prosseguir exatamente como se a expressão de rendimento fosse apenas outra chamada externa. O valor da expressão yield após a retomada depende do método que retomou a execução. Se `__anext__()` for usado, o resultado será `None`. Caso contrário, se `asend()` for usado, o resultado será o valor passado para esse método.

Se um gerador assíncrono encerrar mais cedo por `break`, pela tarefa que fez sua chamada ser cancelada ou por outras exceções, o código de limpeza assíncrona do gerador será executado e possivelmente levantará alguma exceção ou acessará as variáveis de contexto em um contexto inesperado – talvez após o tempo de vida das tarefas das quais ele depende, ou durante o laço de eventos de encerramento quando o gancho de coleta de lixo do gerador assíncrono for chamado. Para prevenir isso, o chamador deve encerrar explicitamente o gerador assíncrono chamando o método `aclose()` para finalizar o gerador e, por fim, desconectá-lo do laço de eventos.

Em uma função geradora assíncrona, expressões de yield são permitidas em qualquer lugar em uma construção `try`. No entanto, se um gerador

assíncrono não for retomado antes de ser finalizado (alcançando uma contagem de referência zero ou sendo coletado pelo coletor de lixo), então uma expressão de yield dentro de um construção **`try`** pode resultar em uma falha na execução das cláusulas pendentes de **`finally`**. Nesse caso, é responsabilidade do laço de eventos ou escalonador que executa o gerador assíncrono chamar o método **`aclose()`** do gerador iterador assíncrono e executar o objeto corrotina resultante, permitindo assim que quaisquer cláusulas pendentes de **`finally`** sejam executadas.

Para cuidar da finalização após o término do laço de eventos, um laço de eventos deve definir uma função *finalizer* que recebe um gerador assíncrono e provavelmente chama **`aclose()`** e executa a corrotina. Este *finalizer* pode ser registrado chamando **`sys.set_asyncgen_hooks()`**. Quando iterado pela primeira vez, um gerador assíncrono armazenará o *finalizer* registrado para ser chamado na finalização. Para um exemplo de referência de um método *finalizer*, consulte a implementação de `asyncio.Loop.shutdown_asyncgens` em Lib/asyncio/base_events.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/base_events.py].

O expressão `yield from <expr>` é um erro de sintaxe quando usado em uma função geradora assíncrona.

6.2.8.4. Métodos geradores-iteradores assíncronos

Esta subseção descreve os métodos de um iterador gerador assíncrono, que são usados para controlar a execução de uma função geradora.

async agen.__anext__()

Retorna um objeto aguardável que, quando executado, começa a executar o gerador assíncrono ou o retoma na última expressão yield executada. Quando uma função geradora assíncrona é retomada com o método **`__anext__()`**, a expressão yield atual sempre avalia para **`None`** no objeto aguardável retornado, que, quando executado, continuará para a próxima expressão yield. O valor de **`yield_list`** da expressão yield é o valor da exceção **`StopIteration`** levantada pela corrotina em conclusão. Se o gerador assíncrono sair sem produzir outro valor, o objeto aguardável em vez disso levanta uma exceção

`StopAsyncIteration`, sinalizando que a iteração assíncrona foi concluída.

Este método é normalmente chamado implicitamente por um laço `async for`.

async agen.asend(*value*)

Retorna um objeto aguardável que, quando executado, retoma a execução do gerador assíncrono. Assim como o método `send()` para um gerador, isso "envia" um valor para a função geradora assíncrona, e o argumento *value* se torna o resultado da expressão de yield atual. O objeto aguardável retornado pelo método `asend()` retornará o próximo valor produzido pelo gerador como o valor da exceção `StopIteration` levantada, ou lança `StopAsyncIteration` se o gerador assíncrono sair sem produzir outro valor. Quando `asend()` é chamado para iniciar o gerador assíncrono, ele deve ser chamado com `None` como argumento, pois não há expressão yield que possa receber o valor.

async agen.athrow(*value*)

async agen.athrow(*type*[, *value*[, *traceback*]])

Retorna um objeto aguardável que gera uma exceção do tipo `type` no ponto em que o gerador assíncrono foi pausado, e retorna o próximo valor produzido pela função geradora como o valor da exceção `StopIteration` levantada. Se o gerador assíncrono terminar sem produzir outro valor, uma exceção `StopAsyncIteration` é levantada pelo objeto aguardável. Se a função geradora não capturar a exceção passada ou gerar uma exceção diferente, então quando o objeto aguardável for executado, essa exceção se propagará para o chamador do objeto aguardável.

Alterado na versão 3.12: A segunda assinatura (tipo[, valor[, traceback]]) foi descontinuada e pode ser removida em uma versão futura do Python.

async agen.aclose()

Retorna um objeto aguardável que, quando executado, levantará uma `GeneratorExit` na função geradora assíncrona no ponto em que foi pausada. Se a função geradora assíncrona sair de forma normal, se estiver já estiver fechada ou levantar `GeneratorExit` (não capturando a exceção), então o objeto aguardável retornado levantará uma exceção `StopIteration`. Quaisquer outros objetos aguardáveis retornados por chamadas subsequentes à função geradora assíncrona levantarão uma exceção `StopAsyncIteration`. Se a função geradora assíncrona levantar um valor, um `RuntimeError` será lançado pelo objeto aguardável. Se a função geradora assíncrona levantar qualquer outra exceção, ela será propagada para o chamador do objeto aguardável. Se a função geradora assíncrona já tiver saído devido a uma exceção ou saída normal, então chamadas posteriores ao método `aclose()` retornarão um objeto aguardável que não faz nada.

6.3. Primárias

Primárias representam as operações mais fortemente vinculadas da linguagem. Sua sintaxe é:

```
primary: atom | attributeref | subscription | call
```

6.3.1. Referências de atributo

Uma referência de atributo é um primário seguido de um ponto e um nome.

```
attributeref: primary "." identifier
```

A primária deve avaliar para um objeto de um tipo que tem suporte a referências de atributo, o que a maioria dos objetos faz. Este objeto é então solicitado a produzir o atributo cujo nome é o identificador. O tipo e o valor produzido são determinados pelo objeto. Várias avaliações da mesma referência de atributo podem produzir diferentes objetos.

Esta produção pode ser personalizada substituindo o método `__getattribute__()` ou o método `__getattr__()`. O método `__getattribute__()` é chamado primeiro e retorna um valor ou levanta uma `AttributeError` se o atributo não estiver disponível.

Se for levantada uma `AttributeError` e o objeto tiver um método `__getattr__()`, esse método será chamado como alternativa.

6.3.2. Subscriptions and slicings

The *subscription* syntax is usually used for selecting an element from a container – for example, to get a value from a `dict`:

```
>>> digits_by_name = {'one': 1, 'two': 2}
>>> digits_by_name['two']  # Subscripting a dictionary using the
2
```

In the subscription syntax, the object being subscribed – a primary – is followed by a *subscript* in square brackets. In the simplest case, the subscript is a single expression.

Depending on the type of the object being subscribed, the subscript is sometimes called a key (for mappings), index (for sequences), or *type argument* (for generic types). Syntactically, these are all equivalent:

```
>>> colors = ['red', 'blue', 'green', 'black']
>>> colors[3]  # Subscripting a list using the index 3
'black'

>>> list[str]  # Parameterizing the list type using the type arg
list[str]
```

At runtime, the interpreter will evaluate the primary and the subscript, and call the primary's `__getitem__()` or `__class_getitem__()` special method with the subscript as argument. For more details on which of these methods is called, see `__class_getitem__` versus `__getitem__`.

To show how subscription works, we can define a custom object that implements `__getitem__()` and prints out the value of the subscript:

```
>>> class SubscriptionDemo:
...     def __getitem__(self, key):
...         print(f'subscripted with: {key!r}')
...
>>> demo = SubscriptionDemo()
>>> demo[1]
subscripted with: 1
>>> demo['a' * 3]
subscripted with: 'aaa'
```

See `__getitem__()` documentation for how built-in types handle subscription.

Subscriptions may also be used as targets in assignment or deletion statements. In these cases, the interpreter will call the subscripted object's `__setitem__()` or `__delitem__()` special method, respectively, instead of `__getitem__()`.

```
>>> colors = ['red', 'blue', 'green', 'black']
>>> colors[3] = 'white'  # Setting item at index
>>> colors
['red', 'blue', 'green', 'white']
>>> del colors[3]  # Deleting item at index 3
>>> colors
['red', 'blue', 'green']
```

All advanced forms of *subscript* documented in the following sections are also usable for assignment and deletion.

6.3.2.1. Fatiamentos

A more advanced form of subscription, *slicing*, is commonly used to extract a portion of a sequence. In this form, the subscript is a slice: up to three expressions separated by colons. Any of the expressions may be omitted, but a slice must contain at least one colon:

```
>>> number_names = ['zero', 'one', 'two', 'three', 'four', 'five
>>> number_names[1:3]
['one', 'two']
>>> number_names[1:]
['one', 'two', 'three', 'four', 'five']
>>> number_names[:3]
['zero', 'one', 'two']
>>> number_names[:]
['zero', 'one', 'two', 'three', 'four', 'five']
>>> number_names[::2]
['zero', 'two', 'four']
>>> number_names[:-3]
['zero', 'one', 'two']
>>> del number_names[4:]
>>> number_names
['zero', 'one', 'two', 'three']
```

When a slice is evaluated, the interpreter constructs a **slice** object whose **start**, **stop** and **step** attributes, respectively, are the results of the expressions between the colons. Any missing expression evaluates to **None**. This **slice** object is then passed to the **__getitem__()** or **__class_getitem__()** special method, as above.

```
# continuing with the SubscriptionDemo instance defined above:
>>> demo[2:3]
subscripted with: slice(2, 3, None)
>>> demo[::'spam']
subscripted with: slice(None, None, 'spam')
```

6.3.2.2. Comma-separated subscripts

The subscript can also be given as two or more comma-separated expressions or slices:

```
# continuing with the SubscriptionDemo instance defined above:
>>> demo[1, 2, 3]
subscripted with: (1, 2, 3)
>>> demo[1:2, 3]
subscripted with: (slice(1, 2, None), 3)
```

This form is commonly used with numerical libraries for slicing multi-dimensional data. In this case, the interpreter constructs a `tuple` of the results of the expressions or slices, and passes this tuple to the `__getitem__()` or `__class_getitem__()` special method, as above.

The subscript may also be given as a single expression or slice followed by a comma, to specify a one-element tuple:

```
>>> demo['spam',]
subscripted with: ('spam',)
```

6.3.2.3. “Starred” subscriptions

Adicionado na versão 3.11: Expressions in *tuple_slices* may be starred. See **PEP 646** [https://peps.python.org/pep-0646/].

The subscript can also contain a starred expression. In this case, the interpreter unpacks the result into a tuple, and passes this tuple to `__getitem__()` or `__class_getitem__()`:

```
# continuing with the SubscriptionDemo instance defined above:
>>> demo[*range(10)]
subscripted with: (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)
```

Starred expressions may be combined with comma-separated expressions and slices:

```
>>> demo['a', 'b', *range(3), 'c']
subscripted with: ('a', 'b', 0, 1, 2, 'c')
```

6.3.2.4. Formal subscription grammar

```
subscription:     primary '[' subscript ']'
subscript:        single_subscript | tuple_subscript
single_subscript: proper_slice | assignment_expression
proper_slice:     [expression] ":" [expression] [ ":" [expressic
tuple_subscript:  ','.(single_subscript | starred_expression)+ [
```

Recall that the `|` operator denotes ordered choice. Specifically, in **`subscript`**, if both alternatives would match, the first (**`single_subscript`**) has priority.

6.3.3. Chamadas

Uma chamada chama um objeto que é um chamável (por exemplo, uma função) com uma série possivelmente vazia de argumentos:

```
call:                  primary "(" [argument list [","] | compreh
argument_list:         positional arguments [","  starred and keyw
                         [","  keywords arguments]
                       | starred and keywords [","  keywords argum
                       | keywords arguments
positional_arguments:  positional item (","  positional item)*
positional_item:       assignment expression | "*" expression
starred_and_keywords:  ("*" expression | keyword item)
                       (","  "*" expression | ","  keyword item)*
keywords_arguments:    (keyword item | "**" expression)
                       (","  keyword item | ","  "**" expression)*
keyword_item:          identifier "=" expression
```

Uma vírgula final opcional pode estar presente após os argumentos posicionais e nomeados, mas não afeta a semântica.

O primário deve ser avaliado como um objeto que pode ser chamado (funções definidas pelo usuário, funções embutidas, métodos de objetos embutidos, objetos de classe, métodos de instâncias de classe e todos os objetos que possuem um método **`__call__()`** são chamáveis). Todas as expressões de argumento são avaliadas antes da tentativa de chamada. Consulte a seção Definições de função para a sintaxe das listas formais de parâmetros.

Se houver argumentos nomeados, eles serão primeiro convertidos em argumentos posicionais, como segue. Primeiro, é criada uma lista de slots não preenchidos para os parâmetros formais. Se houver N argumentos posicionais, eles serão colocados nos primeiros N slots. A seguir, para cada argumento nomeado, o identificador é usado para determinar o slot

correspondente (se o identificador for igual ao primeiro nome formal do parâmetro, o primeiro slot será usado e assim por diante). Se o slot já estiver preenchido, uma exceção `TypeError` será levantada. Caso contrário, o argumento é colocado no slot, preenchendo-o (mesmo que a expressão seja `None`, ela preenche o slot). Quando todos os argumentos forem processados, os slots ainda não preenchidos serão preenchidos com o valor padrão correspondente da definição da função. (Os valores padrão são calculados, uma vez, quando a função é definida; assim, um objeto mutável, como uma lista ou dicionário usado como valor padrão, será compartilhado por todas as chamadas que não especificam um valor de argumento para o slot correspondente; isso deve geralmente ser evitado.) Se houver algum slot não preenchido para o qual nenhum valor padrão for especificado, uma exceção **`TypeError`** será levantada. Caso contrário, a lista de slots preenchidos será usada como lista de argumentos para a chamada.

Uma implementação pode fornecer funções integradas cujos parâmetros posicionais não possuem nomes, mesmo que sejam 'nomeados' para fins de documentação e que, portanto, não possam ser fornecidos por nomes. No CPython, este é o caso de funções implementadas em C que usam `PyArg_ParseTuple()` para analisar seus argumentos.

Se houver mais argumentos posicionais do que slots de parâmetros formais, uma exceção `TypeError` será levantada, a menos que um parâmetro formal usando a sintaxe `*identificador` esteja presente; neste caso, esse parâmetro formal recebe uma tupla contendo os argumentos posicionais em excesso (ou uma tupla vazia se não houver argumentos posicionais em excesso).

Se algum argumento nomeado não corresponder a um nome de parâmetro formal, uma exceção `TypeError` é levantada, a menos que um parâmetro formal usando a sintaxe `**identificador` esteja presente; neste caso, esse parâmetro formal recebe um dicionário contendo os argumentos nomeados em excesso (usando os nomes como chaves e os valores dos argumentos como valores correspondentes), ou um (novo) dicionário vazio se não houver argumentos nomeados em excesso.

Se a sintaxe `*expressão` aparecer na chamada da função, `expressão` deverá ser avaliada como iterável. Os elementos desses iteráveis são tratados como

se fossem argumentos posicionais adicionais. Para a chamada `f(x1, x2, *y, x3, x4)`, se *y* for avaliado como uma sequência *y1*, …, *yM*, isso é equivalente a uma chamada com M+4 argumentos posicionais *x1*, *x2*, *y1*, …, *yM*, *x3*, *x4*.

Uma consequência disso é que embora a sintaxe `*expressão` possa aparecer *depois* de argumentos nomeados explícitos, ela é processada *antes* dos argumentos nomeados (e de quaisquer argumentos de `**expressão` – veja abaixo). Então:

```
>>> def f(a, b):
...     print(a, b)
...
>>> f(b=1, *(2,))
2 1
>>> f(a=1, *(2,))
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: f() got multiple values for keyword argument 'a'
>>> f(1, *(2,))
1 2
```

É incomum que ambos os argumentos nomeados e a sintaxe `*expressão` sejam usados na mesma chamada, portanto, na prática, essa confusão não surge com frequência.

Se a sintaxe `**expressão` aparecer na chamada de função, `expressão` deve ser avaliada como um mapeamento, cujo conteúdo é tratado como argumentos nomeados adicionais. Se um parâmetro que corresponde a uma chave já recebeu um valor (por um argumento nomeado explícito ou de outro desempacotamento), uma exceção **`TypeError`** é levantada.

Quando `**expressão` é usada, cada chave neste mapeamento deve ser uma string. Cada valor do mapeamento é atribuído ao primeiro parâmetro formal elegível para atribuição de nomeas cujo nome é igual à chave. Uma chave não precisa ser um identificador Python (por exemplo, `"max-temp °F"` é aceitável, embora não corresponda a nenhum parâmetro formal que possa ser declarado). Se não houver correspondência com um parâmetro formal, o

par chave-valor é coletado pelo parâmetro `**`, se houver, ou se não houver, uma exceção **`TypeError`** é levantada.

Parâmetros formais usando a sintaxe `*identificador` ou `**identificador` não podem ser usados como slots de argumentos posicionais ou como nomes de argumentos nomeados.

Alterado na versão 3.5: Chamadas de função aceitam qualquer número de desempacotamentos `*` e `**`, argumentos posicionais podem seguir desempacotamentos iteráveis (`*`) e argumentos nomeados podem seguir desempacotamentos de dicionário (`**`). Originalmente proposto pela **PEP 448** [https://peps.python.org/pep-0448/].

Uma chamada sempre retorna algum valor, possivelmente `None`, a menos que levanta uma exceção. A forma como esse valor é calculado depende do tipo do objeto chamável.

Se for…

uma função definida por usuário:

O bloco de código da função é executado, passando-lhe a lista de argumentos. A primeira coisa que o bloco de código fará é vincular os parâmetros formais aos argumentos; isso é descrito na seção Definições de função. Quando o bloco de código executa uma instrução **`return`**, isso especifica o valor de retorno da chamada de função. Se a execução atingir o final do bloco de código sem executar uma instrução **`return`**, o valor de retorno será `None`.

um método embutido ou uma função embutida:

O resultado fica por conta do interpretador; veja Funções embutidas para descrições de funções embutidas e métodos embutidos.

um objeto classe:

Uma nova instância dessa classe é retornada.

um método de instância de classe:

A função correspondente definida pelo usuário é chamada, com uma lista de argumentos que é maior que a lista de argumentos da chamada:

a instância se torna o primeiro argumento.

uma instância de classe:

A classe deve definir um método `__call__()`; o efeito é então o mesmo como se esse método fosse chamado.

6.4. Expressão await

Suspende a execução de corrotina em um objeto aguardável. Só pode ser usado dentro de uma função de corrotina.

```
await_expr: "await" primary
```

Adicionado na versão 3.5.

6.5. O operador de potência

O operador de potência vincula-se com mais força do que os operadores unários à sua esquerda; ele se vincula com menos força do que os operadores unários à sua direita. A sintaxe é:

```
power: (await_expr | primary) ["**" u_expr]
```

Assim, em uma sequência sem parênteses de operadores de potência e unários, os operadores são avaliados da direita para a esquerda (isso não restringe a ordem de avaliação dos operandos): `-1**2` resulta em `-1`.

The power operator has the same semantics as the built-in `pow()` function, when called with two arguments: it yields its left argument raised to the power of its right argument. Numeric arguments are first converted to a common type, and the result is of that type.

Para operandos int, o resultado tem o mesmo tipo que os operandos, a menos que o segundo argumento seja negativo; nesse caso, todos os argumentos são convertidos em ponto flutuante e um resultado ponto

flutuante é entregue. Por exemplo, `10**2` retorna `100`, mas `10**-2` retorna `0.01`.

Elevar `0.0` a uma potência negativa resulta em uma exceção `ZeroDivisionError`. Elevar um número negativo a uma potência fracionária resulta em um número `complex`. (Em versões anteriores, levantava `ValueError`.)

Esta operação pode ser personalizada usando os métodos especial `__pow__()` e `__rpow__()`.

6.6. Operações aritméticas unárias e bit a bit

Todas as operações aritméticas unárias e bit a bit têm a mesma prioridade:

```
u_expr: power | "-" u_expr | "+" u_expr | "~" u_expr
```

O operador unário `-` (menos) produz a negação de seu argumento numérico; a operação pode ser substituída pelo método especial `__neg__()`.

O operador unário `+` (mais) produz seu argumento numérico inalterado; a operação pode ser substituída pelo método especial `__pos__()`.

O operador unário `~` (inverter) produz a inversão bit a bit de seu argumento inteiro. A inversão bit a bit de `x` é definida como `-(x+1)`. Aplica-se apenas a números inteiros ou a objetos personalizados que substituem o método especial `__invert__()`.

Em todos os três casos, se o argumento não tiver o tipo adequado, uma exceção `TypeError` é levantada.

6.7. Operações binárias aritméticas

As operações aritméticas binárias possuem os níveis de prioridade convencionais. Observe que algumas dessas operações também se aplicam a determinados tipos não numéricos. Além do operador potência, existem

apenas dois níveis, um para operadores multiplicativos e outro para operadores aditivos:

```
m_expr: u_expr | m_expr "*" u_expr | m_expr "@" m_expr |
        m_expr "//" u_expr | m_expr "/" u_expr |
        m_expr "%" u_expr
a_expr: m_expr | a_expr "+" m_expr | a_expr "-" m_expr
```

The `*` (multiplication) operator yields the product of its arguments. The arguments must either both be numbers, or one argument must be an integer and the other must be a sequence. In the former case, the numbers are converted to a common real type and then multiplied together. In the latter case, sequence repetition is performed; a negative repetition factor yields an empty sequence.

Esta operação pode ser personalizada usando os métodos especial `__mul__()` e `__rmul__()`.

Alterado na versão 3.14: Se apenas um operando for um número complexo, o outro operando será convertido em um número de ponto flutuante.

O operador `@` (arroba) deve ser usado para multiplicação de matrizes. Nenhum tipo embutido do Python implementa este operador.

Esta operação pode ser personalizada usando os métodos especial `__matmul__()` e `__rmatmul__()`.

Adicionado na versão 3.5.

The `/` (division) and `//` (floor division) operators yield the quotient of their arguments. The numeric arguments are first converted to a common type. Division of integers yields a float, while floor division of integers results in an integer; the result is that of mathematical division with the ‘floor’ function applied to the result. Division by zero raises the `ZeroDivisionError` exception.

A operação de divisão pode ser personalizada usando os métodos especiais `__truediv__()` e `__rtruediv__()`. A operação de divisão pelo piso pode

ser personalizada usando os métodos especiais `__floordiv__()` e `__rfloordiv__()`.

The `%` (modulo) operator yields the remainder from the division of the first argument by the second. The numeric arguments are first converted to a common type. A zero right argument raises the `ZeroDivisionError` exception. The arguments may be floating-point numbers, e.g., `3.14%0.7` equals `0.34` (since `3.14` equals `4*0.7 + 0.34`.) The modulo operator always yields a result with the same sign as its second operand (or zero); the absolute value of the result is strictly smaller than the absolute value of the second operand [[1]].

Os operadores de divisão pelo piso e módulo são conectados pela seguinte identidade: `x == (x//y)*y + (x%y)`. A divisão pelo piso e o módulo também estão conectados com a função embutida `divmod()`: `divmod(x, y) == (x//y, x%y)`. [[2]].

Além de realizar a operação de módulo em números, o operador `%` também é sobrecarregado por objetos string para realizar a formatação de string no estilo antigo (também conhecida como interpolação). A sintaxe para formatação de string é descrita na Referência da Biblioteca Python, seção Formatação de string no estilo printf.

A operação *módulo* pode ser personalizada usando os métodos especial `__mod__()` e `__rmod__()`.

O operador de divisão pelo piso, o operador de módulo e a função `divmod()` não são definidos para números complexos. Em vez disso, converta para um número de ponto flutuante usando a função `abs()` se apropriado.

The `+` (addition) operator yields the sum of its arguments. The arguments must either both be numbers or both be sequences of the same type. In the former case, the numbers are converted to a common real type and then added together. In the latter case, the sequences are concatenated.

Esta operação pode ser personalizada usando os métodos especial `__add__()` e `__radd__()`.

Alterado na versão 3.14: Se apenas um operando for um número complexo, o outro operando será convertido em um número de ponto flutuante.

The `-` (subtraction) operator yields the difference of its arguments. The numeric arguments are first converted to a common real type.

Esta operação pode ser personalizada usando os métodos especial `__sub__()` e `__rsub__()`.

Alterado na versão 3.14: Se apenas um operando for um número complexo, o outro operando será convertido em um número de ponto flutuante.

6.8. Operações de deslocamento

As operações de deslocamento têm menor prioridade que as operações aritméticas:

```
shift_expr: a_expr | shift_expr ("<<" | ">>") a_expr
```

Esses operadores aceitam números inteiros como argumentos. Eles deslocam o primeiro argumento para a esquerda ou para a direita pelo número de bits fornecido pelo segundo argumento.

A operação de deslocamento à esquerda pode ser personalizada usando os métodos especiais `__lshift__()` e `__rlshift__()`. A operação de deslocamento à direita pode ser personalizada usando os métodos especiais `__rshift__()` e `__rrshift__()`.

Um deslocamento para a direita de *n* bits é definido como a divisão pelo piso por `pow(2,n)`. Um deslocamento à esquerda de *n* bits é definido como a multiplicação por `pow(2,n)`.

6.9. Operações binárias bit a bit

Cada uma das três operações bit a bit tem um nível de prioridade diferente:

```
and_expr: shift_expr | and_expr "&" shift_expr
xor_expr: and_expr | xor_expr "^" and_expr
or_expr:  xor_expr | or_expr "|" xor_expr
```

O operador `&` produz o E (AND) bit a bit de seus argumentos, que devem ser inteiros ou um deles deve ser um objeto personalizado substituindo os métodos especiais `__and__()` ou `__rand__()`.

O operador `^` produz o XOR bit a bit (OU exclusivo) de seus argumentos, que devem ser inteiros ou um deles deve ser um objeto personalizado sobrescrevendo os métodos especiais `__xor__()` ou `__rxor__()`.

O operador `|` produz o OU (OR) bit a bit de seus argumentos, que devem ser inteiros ou um deles deve ser um objeto personalizado sobrescrevendo os métodos especiais `__or__()` ou `__ror__()`.

6.10. Comparações

Ao contrário de C, todas as operações de comparação em Python têm a mesma prioridade, que é menor do que qualquer operação aritmética, de deslocamento ou bit a bit. Também diferentemente de C, expressões como `a < b < c` têm a interpretação que é convencional em matemática:

```
comparison:    or_expr (comp_operator or_expr)*
comp_operator: "<" | ">" | "==" | ">=" | "<=" | "!="
               | "is" ["not"] | ["not"] "in"
```

Comparações produzem valores booleanos: `True` ou `False`. *métodos de comparação rica* personalizados podem retornar valores não booleanos. Neste caso, o Python chamará `bool()` nesse valor em contextos booleanos.

As comparações podem ser encadeadas arbitrariamente, por exemplo, `x < y <= z` é equivalente a `x < y and y <= z`, exceto que `y` é avaliado apenas uma vez (mas em ambos os casos `z` não é avaliado quando `x < y` é considerado falso).

Formalmente, se *a*, *b*, *c*, …, *y*, *z* são expressões e *op1*, *op2*, …, *opN* são operadores de comparação, então `a op1 b op2 c ... y opN z` é equivalente a `a op1 b` e `b op2 c` e `... y opN z`, exceto que cada expressão é avaliada no máximo uma vez.

Observe que `a op1 b op2 c` não implica qualquer tipo de comparação entre *a* e *c*, de modo que, por exemplo, `x < y > z` é perfeitamente válido (embora talvez não seja bonito).

6.10.1. Comparações de valor

Os operadores <, >, ==, >=, <= e `!=` comparam os valores de dois objetos. Os objetos não precisam ser do mesmo tipo.

O capítulo Objetos, valores e tipos afirma que os objetos possuem um valor (além do tipo e da identidade). O valor de um objeto é uma noção bastante abstrata em Python: por exemplo, não existe um método de acesso canônico para o valor de um objeto. Além disso, não há exigência de que o valor de um objeto seja construído de uma maneira específica, por exemplo. composto por todos os seus atributos de dados. Os operadores de comparação implementam uma noção específica de qual é o valor de um objeto. Pode-se pensar neles como definindo o valor de um objeto indiretamente, por meio de sua implementação de comparação.

Como todos os tipos são subtipos (diretos ou indiretos) de **`object`**, eles herdam o comportamento de comparação padrão de **`object`**. Os tipos podem personalizar seu comportamento de comparação implementando *métodos de comparação rica* como **`__lt__()`**, descrito em Personalização básica.

O comportamento padrão para comparação de igualdade (`==` e `!=`) é baseado na identidade dos objetos. Consequentemente, a comparação da igualdade de instâncias com a mesma identidade resulta em igualdade, e a comparação da igualdade de instâncias com identidades diferentes resulta em desigualdade. Uma motivação para este comportamento padrão é o desejo de que todos os objetos sejam reflexivos (ou seja, `x is y` implica `x == y`).

Uma comparação de ordem padrão (<, >, <= e >=) não é fornecida; uma tentativa levanta `TypeError`. Uma motivação para este comportamento padrão é a falta de um invariante semelhante ao da igualdade.

O comportamento da comparação de igualdade padrão, de que instâncias com identidades diferentes são sempre desiguais, pode contrastar com o que os tipos precisarão ter uma definição sensata de valor de objeto e igualdade baseada em valor. Esses tipos precisarão personalizar seu comportamento de comparação e, de fato, vários tipos embutidos fizeram isso.

A lista a seguir descreve o comportamento de comparação dos tipos embutidos mais importantes.

- Números de tipos numéricos embutidos (Tipos numéricos — int, float, complex) e dos tipos de biblioteca padrão `fractions.Fraction` e `decimal.Decimal` podem ser comparados dentro e entre seus tipos, com a restrição que os números complexos não oferecem suporte a comparação de ordens. Dentro dos limites dos tipos envolvidos, eles comparam matematicamente (algoritmicamente) corretos sem perda de precisão.

  Os valores não numéricos `float('NaN')` e `decimal.Decimal('NaN')` são especiais. Qualquer comparação ordenada de um número com um valor que não é um número é falsa. Uma implicação contraintuitiva é que os valores que não são numéricos não são iguais a si mesmos. Por exemplo, se `x = float('NaN')`, `3 < x`, `x < 3` e `x == x` são todos falsos, enquanto `x != x` é verdadeiro. Esse comportamento é compatível com IEEE 754.

- `None` e `NotImplemented` são singletons. **PEP 8** [https://peps.python.org/pep-0008/] aconselha que comparações para singletons devem sempre ser feitas com `is` ou `is not`, nunca com os operadores de igualdade.

- Sequências binárias (instâncias de `bytes` ou `bytearray`) podem ser comparadas dentro e entre seus tipos. Eles comparam lexicograficamente usando os valores numéricos de seus elementos.

- Strings (instâncias de `str`) são comparadas lexicograficamente usando os pontos de código Unicode numéricos (o resultado da função embutida `ord()`) de seus caracteres. [[3]]

  Strings e sequências binárias não podem ser comparadas diretamente.

- Sequências (instâncias de `tuple`, `list` ou `range`) podem ser comparadas apenas dentro de cada um de seus tipos, com a restrição de que intervalos não oferecem suporte a comparação de ordem. A comparação de igualdade entre esses tipos resulta em desigualdade, e a comparação ordenada entre esses tipos levanta `TypeError`.

  As sequências são comparadas lexicograficamente usando a comparação de elementos correspondentes. Os contêineres embutidos normalmente presumem que objetos idênticos são iguais a si mesmos. Isso permite ignorar testes de igualdade para objetos idênticos para melhorar o desempenho e manter seus invariantes internos.

  A comparação lexicográfica entre coleções embutidas funciona da seguinte forma:

  - Para que duas coleções sejam comparadas iguais, elas devem ser do mesmo tipo, ter o mesmo comprimento e cada par de elementos correspondentes deve ser comparado igual (por exemplo, `[1,2] == (1,2)` é false porque o tipo não é o mesmo).
  - Coleções que oferecem suporte a comparação de ordem são ordenadas da mesma forma que seus primeiros elementos desiguais (por exemplo, `[1,2,x] <= [1,2,y]` tem o mesmo valor que `x <= y`). Se um elemento correspondente não existir, a coleção mais curta é ordenada primeiro (por exemplo, `[1,2] < [1,2,3]` é verdadeiro).

- Mapeamentos (instâncias de `dict`) comparam iguais se e somente se eles tiverem pares `(chave, valor)` iguais. A comparação de igualdade das chaves e valores reforça a reflexividade.

  Comparações de ordem (`<`, `>`, `<=` e `>=`) levantam `TypeError`.

- Conjuntos (instâncias de `set` ou `frozenset`) podem ser comparados dentro e entre seus tipos.

  Eles definem operadores de comparação de ordem para significar testes de subconjunto e superconjunto. Essas relações não definem ordenações totais (por exemplo, os dois conjuntos `{1,2}` e `{2,3}` não são iguais, nem subconjuntos um do outro, nem superconjuntos um do outro). Consequentemente, conjuntos não são argumentos apropriados para funções que dependem de ordenação total (por exemplo, `min()`, `max()` e `sorted()` produzem resultados indefinidos dada uma lista de conjuntos como entradas) .

  A comparação de conjuntos reforça a reflexividade de seus elementos.

- A maioria dos outros tipos embutidos não possui métodos de comparação implementados, portanto, eles herdam o comportamento de comparação padrão.

As classes definidas pelo usuário que personalizam seu comportamento de comparação devem seguir algumas regras de consistência, se possível:

- A comparação da igualdade deve ser reflexiva. Em outras palavras, objetos idênticos devem ser comparados iguais:

  `x is y` implica em `x == y`

- A comparação deve ser simétrica. Em outras palavras, as seguintes expressões devem ter o mesmo resultado:

  `x == y` e `y == x`

  `x != y` e `y != x`

  `x < y` e `y > x`

  `x <= y` e `y >= x`

- A comparação deve ser transitiva. Os seguintes exemplos (não exaustivos) ilustram isso:

`x > y and y > z` implica em `x > z`

`x < y and y <= z` implica em `x < z`

- A comparação inversa deve resultar na negação booleana. Em outras palavras, as seguintes expressões devem ter o mesmo resultado:

  `x == y` e `not x != y`

  `x < y` e `not x >= y` (pra classificação total)

  `x > y` e `not x <= y` (pra classificação total)

  The last two expressions apply to totally ordered collections (e.g. to sequences, but not to sets or mappings). See also the `@~functools.total_ordering` decorator.

- O resultado `hash()` deve ser consistente com a igualdade. Objetos iguais devem ter o mesmo valor de hash ou ser marcados como não-hasheáveis.

Python não impõe essas regras de consistência. Na verdade, os valores não numéricos são um exemplo de não cumprimento dessas regras.

6.10.2. Operações de teste de pertinência

Os operadores `in` e `not in` testam se um operando é membro ou não de outro. `x in s` é avaliado como `True` se *x* for membro de *s*, e `False` caso contrário. `x not in s` retorna a negação de `x in s`. Todas as sequências e tipos de conjuntos embutidos oferecem suporte a isso, assim como o dicionário, para o qual **`in`** testa se o dicionário tem uma determinada chave. Para tipos de contêiner como list, tuple, set, frozenset, dict ou Collections.deque, a expressão `x in y` é equivalente a `any(x is e or x == e for e in y)`.

Para os tipos string e bytes, `x in y` é `True` se e somente se *x* for uma substring de *y*. Um teste equivalente é `y.find(x) != -1`. Strings vazias são

sempre consideradas uma substring de qualquer outra string, então `"" in "abc"` retornará `True`.

Para classes definidas pelo usuário que definem o método `__contains__()`, `x in y` retorna `True` se `y.__contains__(x)` retorna um valor verdadeiro, e `False` caso contrário.

Para classes definidas pelo usuário que não definem `__contains__()`, mas definem `__iter__()`, `x in y` é `True` se algum valor `z`, para a qual a expressão `x is z or x == z` é verdadeira, é produzida durante a iteração sobre `y`. Se uma exceção for levantada durante a iteração, é como se `in` tivesse levantado essa exceção.

Por último, o protocolo de iteração de estilo antigo é tentado: se uma classe define `__getitem__()`, `x in y` é `True` se, e somente se, houver um índice inteiro não negativo *i* tal que `x is y[i] or x == y[i]`, e nenhum índice inteiro inferior levanta a exceção `IndexError`. (Se qualquer outra exceção for levantada, é como se `in` levantasse essa exceção).

O operador `not in` é definido para ter o valor verdade inverso de `in`.

6.10.3. Comparações de identidade

Os operadores `is` e `is not` testam a identidade de um objeto: `x is y` é verdadeiro se, e somente se, *x* e *y* são o mesmo objeto. A identidade de um objeto é determinada usando a função `id()`. `x is not y` produz o valor verdade inverso. [[4]]

6.11. Operações booleanas

```
or_test:  and_test | or_test "or" and_test
and_test: not_test | and_test "and" not_test
not_test: comparison | "not" not_test
```

No contexto de operações booleanas, e também quando expressões são usadas por instruções de fluxo de controle, os seguintes valores são interpretados como falsos: `False`, `None`, zero numérico de todos os tipos e

strings e contêineres vazios (incluindo strings, tuplas, listas, dicionários, conjuntos e frozensets). Todos os outros valores são interpretados como verdadeiros. Objetos definidos pelo usuário podem personalizar seu valor verdade fornecendo um método `__bool__()`.

O operador `not` produz `True` se seu argumento for falso, `False` caso contrário.

A expressão `x and y` primeiro avalia *x*; se *x* for falso, seu valor será retornado; caso contrário, *y* será avaliado e o valor resultante será retornado.

A expressão `x or y` primeiro avalia *x*; se *x* for verdadeiro, seu valor será retornado; caso contrário, *y* será avaliado e o valor resultante será retornado.

Observe que nem `and` nem `or` restringem o valor e o tipo que retornam para `False` e `True`, mas sim retornam o último argumento avaliado. Isso às vezes é útil, por exemplo, se `s` é uma string que deve ser substituída por um valor padrão se estiver vazia, a expressão `s or 'foo'` produz o valor desejado. Como `not` precisa criar um novo valor, ele retorna um valor booleano independente do tipo de seu argumento (por exemplo, `not 'foo'` produz `False` em vez de `''`.)

6.12. Expressões de atribuição

`assignment_expression`: [`identifier` ":="] `expression`

Uma expressão de atribuição (às vezes também chamada de “expressão nomeada” ou “morsa”) atribui uma `expression` a um `identifier`, ao mesmo tempo que retorna o valor de **`expression`**.

Um caso de uso comum é ao lidar com expressões regulares correspondentes:

```
if matching := pattern.search(data):
    do_something(matching)
```

Ou, ao processar um fluxo de arquivos em partes:

```
while chunk := file.read(9000):
    process(chunk)
```

As expressões de atribuição devem ser colocadas entre parênteses quando usadas como instruções de expressão e quando usadas como subexpressões em expressões de fatiamento, condicionais, de lambda, de argumento nomeado e de `if` de compreensão e em instruções `assert`, `with` e `assignment`. Em todos os outros lugares onde eles podem ser usados, os parênteses não são necessários, inclusive nas instruções `if` e `while`.

Adicionado na versão 3.8: Veja **PEP 572** [https://peps.python.org/pep-0572/] para mais detalhes sobre expressões de atribuição.

6.13. Expressões condicionais

```
conditional_expression: or_test ["if" or_test "else" expression]
expression:             conditional_expression | lambda_expr
```

Uma expressão condicional (às vezes chamada de “operador ternário”) é uma alternativa à instrução if-else. Como é uma expressão, ela retorna um valor e pode aparecer como uma subexpressão.

A expressão `x if C else y` primeiro avalia a condição, *C* em vez de *x*. Se *C* for verdadeiro, *x* é avaliado e seu valor é retornado; caso contrário, *y* será avaliado e seu valor será retornado.

Veja **PEP 308** [https://peps.python.org/pep-0308/] para mais detalhes sobre expressões condicionais.

6.14. Lambdas

```
lambda_expr: "lambda" [parameter_list] ":" expression
```

Expressões lambda (às vezes chamadas de funções lambda) são usadas para criar funções anônimas. A expressão `lambda parameters: expression`

produz um objeto função. O objeto sem nome se comporta como um objeto de função definido com:

```
def <lambda>(parâmetros):
    return expressão
```

Veja a seção Definições de função para a sintaxe das listas de parâmetros. Observe que as funções criadas com expressões lambda não podem conter instruções ou anotações.

6.15. Listas de expressões

```
starred_expression:        "*" or_expr | expression
flexible_expression:       assignment_expression | starred_expres
flexible_expression_list:  flexible_expression ("," flexible_expr
starred_expression_list:   starred_expression ("," starred_expres
expression_list:           expression ("," expression)* [","]
yield_list:                expression_list | starred_expression '
```

Exceto quando parte de uma sintaxe de criação de lista ou conjunto, uma lista de expressões contendo pelo menos uma vírgula produz uma tupla. O comprimento da tupla é o número de expressões na lista. As expressões são avaliadas da esquerda para a direita.

Uma vírgula final é necessária apenas para criar uma tupla de um item, como `1,`; é opcional em todos os outros casos. Uma única expressão sem vírgula final não cria uma tupla, mas produz o valor dessa expressão. (Para criar uma tupla vazia, use um par vazio de parênteses: `()`.)

6.15.1. Iterable unpacking

In an expression list or tuple, list or set display, any expression may be prefixed with an asterisk (`*`). This denotes *iterable unpacking*.

At runtime, the asterisk-prefixed expression must evaluate to an iterable. The iterable is expanded into a sequence of items, which are included in the new tuple, list, or set, at the site of the unpacking.

Adicionado na versão 3.5: Desempacotamento de iterável em listas de expressões, originalmente proposta pela **PEP 448** [https://peps.python.org/pep-0448/].

Adicionado na versão 3.11: Qualquer item em uma lista de expressões pode ser estrelado. Veja a **PEP 646** [https://peps.python.org/pep-0646/].

6.16. Ordem de avaliação

Python avalia expressões da esquerda para a direita. Observe que ao avaliar uma tarefa, o lado direito é avaliado antes do lado esquerdo.

Nas linhas a seguir, as expressões serão avaliadas na ordem aritmética de seus sufixos:

```
expr1, expr2, expr3, expr4
(expr1, expr2, expr3, expr4)
{expr1: expr2, expr3: expr4}
expr1 + expr2 * (expr3 - expr4)
expr1(expr2, expr3, *expr4, **expr5)
expr3, expr4 = expr1, expr2
```

6.17. Precedência de operadores

A tabela a seguir resume a precedência de operadores no Python, da precedência mais alta (mais vinculativa) à precedência mais baixa (menos vinculativa). Os operadores na mesma caixa têm a mesma precedência. A menos que a sintaxe seja fornecida explicitamente, os operadores são binários. Os operadores na mesma caixa agrupam-se da esquerda para a direita (exceto exponenciação e expressões condicionais, que agrupam da direita para a esquerda).

Observe que comparações, testes de pertinência e testes de identidade têm todos a mesma precedência e possuem um recurso de encadeamento da esquerda para a direita, conforme descrito na seção Comparações.

Operador	Descrição
`(expressions...)`, `[expressões...]`, `{chave: valor...}`, `{expressões...}`	Expressão entre parênteses ou de ligação, sintaxe de criação de lista, sintaxe de criação de dicionário, sintaxe de criação de conjunto
`x[index]`, `x[index:index]` `x(arguments...)`, `x.attribute`	Subscription (including slicing), call, attribute reference
`await x`	Expressão await
`**`	Exponenciação [[5]]
`+x`, `-x`, `~x`	positivo, negativo, NEGAÇÃO (NOT) bit a bit
`*`, `@`, `/`, `//`, `%`	Multiplicação, multiplicação de matrizes, divisão, divisão pelo piso, resto [[6]]
`+`, `-`	Adição e subtração
`<<`, `>>`	Deslocamentos
`&`	E (AND) bit a bit

Operador	Descrição
^	OU EXCLUSIVO (XOR) bit a bit
|	OU (OR) bit a bit
`in`, `not in`, `is`, `is not`, <, <=, >, >=, !=, ==	Comparações, incluindo testes de pertinência e testes de identidade
`not x`	NEGAÇÃO (NOT) booleana
`and`	E (AND) booleano
`or`	OU (OR) booleano
`if` – **`else`**	Expressão condicional
`lambda`	Expressão lambda
`:=`	Expressão de atribuição

Notas de rodapé

[1]

Embora `abs(x%y) < abs(y)` seja verdadeiro matematicamente, para números flutuantes pode não ser verdadeiro numericamente devido ao arredondamento. Por exemplo, e presumindo uma plataforma na qual um float Python seja um número de precisão dupla IEEE 754, para que `-1e-100 % 1e100` tenha o mesmo sinal que `1e100`, o resultado calculado é `-1e-100 + 1e100`, que é numericamente exatamente igual a `1e100`. A função **`math.fmod()`** retorna um resultado cujo sinal corresponde ao sinal do primeiro argumento e, portanto, retorna `-1e-100` neste caso. Qual abordagem é mais apropriada depende da aplicação.

[2]

Se x estiver muito próximo de um múltiplo inteiro exato de y, é possível que `x//y` seja maior que `(x-x%y)//y` devido ao arredondamento. Nesses casos, Python retorna o último resultado, para preservar que `divmod(x,y)[0] * y + x % y` esteja muito próximo de `x`.

[3]

O padrão Unicode distingue entre *pontos de código* (por exemplo, U+0041) e *caracteres abstratos* (por exemplo, “LATIN CAPITAL LETTER A”). Embora a maioria dos caracteres abstratos em Unicode sejam representados apenas por meio de um ponto de código, há vários caracteres abstratos que também podem ser representados por meio de uma sequência de mais de um ponto de código. Por exemplo, o caractere abstrato “LATIN CAPITAL LETTER C WITH CEDILLA” pode ser representado como um único *caractere pré-composto* na posição de código U+00C7, ou como uma sequência de um *caractere base* na posição de código U+0043 (LATIN CAPITAL LETTER C), seguido por um *caractere de combinação* na posição de código U+0327 (COMBINING CEDILLA).

Os operadores de comparação em strings são comparados no nível dos pontos de código Unicode. Isso pode ser contraintuitivo para os humanos. Por exemplo, `"\u00C7" == "\u0043\u0327"` é `False`, mesmo que ambas as strings representem o mesmo caractere abstrato “LATIN CAPITAL LETTER C WITH CEDILLA”.

Para comparar strings no nível de caracteres abstratos (ou seja, de uma forma intuitiva para humanos), use `unicodedata.normalize()`.

[4]

Devido à coleta de lixo automática, às listas livres e à natureza dinâmica dos descritores, você pode notar um comportamento aparentemente incomum em certos usos do operador `is`, como aqueles que envolvem comparações entre métodos de instância ou constantes. Confira a documentação para obter mais informações.

[5]

O operador de potência `**` liga-se com menos força do que um operador aritmético ou unário bit a bit à sua direita, ou seja, `2**-1` é `0.5`.

[6]

O operador `%` também é usado para formatação de strings; a mesma precedência se aplica.

7. Instruções simples

Uma instrução simples consiste uma única linha lógica. Várias instruções simples podem ocorrer em uma única linha separada por ponto e vírgula. A sintaxe para instruções simples é:

```
simple_stmt: expression_stmt
           | assert_stmt
           | assignment_stmt
           | augmented_assignment_stmt
           | annotated_assignment_stmt
           | pass_stmt
           | del_stmt
           | return_stmt
           | yield_stmt
           | raise_stmt
           | break_stmt
           | continue_stmt
           | import_stmt
           | future_stmt
           | global_stmt
           | nonlocal_stmt
           | type_stmt
```

7.1. Instruções de expressão

As instruções de expressão são usadas (principalmente interativamente) para calcular e escrever um valor, ou (geralmente) para chamar um procedimento (uma função que não retorna nenhum resultado significativo; em Python, os procedimentos retornam o valor `None`). Outros usos de instruções de expressão são permitidos e ocasionalmente úteis. A sintaxe para uma instrução de expressão é:

```
expression_stmt: starred_expression
```

Uma instrução de expressão avalia a lista de expressões (que pode ser uma única expressão).

No modo interativo, se o valor não for `None`, ele será convertido em uma string usando a função embutida `repr()` e a string resultante será gravada na saída padrão em uma linha sozinha (exceto se o resultado é `None`, de modo que as chamadas de procedimento não causam nenhuma saída.)

7.2. Instruções de atribuição

As instruções de atribuição são usadas para (re)vincular nomes a valores e modificar atributos ou itens de objetos mutáveis:

```
assignment_stmt: (target_list "=")+ (starred_expression | yield_
target_list:     target (","  target)* [","]
target:          identifier
                 | "(" [target_list] ")"
                 | "[" [target_list] "]"
                 | attributeref
                 | subscription
                 | "*" target
```

(See section Primárias for the syntax definitions for *attributeref* and *subscription*.)

Uma instrução de atribuição avalia a lista de expressões (lembre-se de que pode ser uma única expressão ou uma lista separada por vírgulas, a última produzindo uma tupla) e atribui o único objeto resultante a cada uma das listas alvos, da esquerda para a direita.

Assignment is defined recursively depending on the form of the target (list). When a target is part of a mutable object (an attribute reference or subscription), the mutable object must ultimately perform the assignment and decide about its validity, and may raise an exception if the assignment is unacceptable. The rules observed by various types and the exceptions raised are given with the definition of the object types (see section A hierarquia de tipos padrão).

A atribuição de um objeto a uma lista alvo, opcionalmente entre parênteses ou colchetes, é definida recursivamente da maneira a seguir.

- Se a lista alvo contiver um único alvo sem vírgula à direita, opcionalmente entre parênteses, o objeto será atribuído a esse alvo.
- Senão:
  - Se a lista alvo contiver um alvo prefixado com um asterisco, chamado de alvo “com estrela” *(starred)*: o objeto deve ser um iterável com pelo menos tantos itens quantos os alvos na lista alvo, menos um. Os primeiros itens do iterável são atribuídos, da esquerda para a direita, aos alvos antes do alvo com estrela. Os itens finais do iterável são atribuídos aos alvos após o alvo com estrela. Uma lista dos itens restantes no iterável é então atribuída ao alvo com estrela (a lista pode estar vazia).
  - Senão: o objeto deve ser um iterável com o mesmo número de itens que existem alvos na lista alvos, e os itens são atribuídos, da esquerda para a direita, aos alvos correspondentes.

A atribuição de um objeto a um único alvo é definida recursivamente da maneira a seguir.

- Se o alvo for um identificador (nome):

  - Se o nome não ocorrer em uma instrução `global` ou `nonlocal` no bloco de código atual: o nome está vinculado ao objeto no espaço de nomes local atual.
  - Caso contrário: o nome é vinculado ao objeto no espaço de nomes global global ou no espaço de nomes global externo determinado por `nonlocal`, respectivamente.

  O nome é vinculado novamente se já estiver vinculado. Isso pode fazer com que a contagem de referências para o objeto anteriormente vinculado ao nome chegue a zero, fazendo com que o objeto seja desalocado e seu destrutor (se houver) seja chamado.

- Se o alvo for uma referência de atributo: a expressão primária na referência é avaliada. Deve produzir um objeto com atributos atribuíveis; se este não for o caso, a exceção `TypeError` é levanta. Esse

objeto é então solicitado a atribuir o objeto atribuído ao atributo fornecido; se não puder executar a atribuição, ele levanta uma exceção (geralmente, mas não necessariamente `AttributeError`).

Nota: Se o objeto for uma instância de classe e a referência de atributo ocorrer em ambos os lados do operador de atribuição, a expressão do lado direito, `a.x` pode acessar um atributo de instância ou (se não existir nenhum atributo de instância) uma classe atributo. O alvo do lado esquerdo `a.x` é sempre definido como um atributo de instância, criando-o se necessário. Assim, as duas ocorrências de `a.x` não necessariamente se referem ao mesmo atributo: se a expressão do lado direito se refere a um atributo de classe, o lado esquerdo cria um novo atributo de instância como alvo da atribuição:

```
class Cls:
    x = 3             # variável de classe
inst = Cls()
inst.x = inst.x + 1   # escreve inst.x como 4 deixando Cls.x
```

This description does not necessarily apply to descriptor attributes, such as properties created with `@property`.

- If the target is a subscription: The primary expression in the reference is evaluated. Next, the subscript expression is evaluated. Then, the primary's `__setitem__()` method is called with two arguments: the subscript and the assigned object.

  Typically, `__setitem__()` is defined on mutable sequence objects (such as lists) and mapping objects (such as dictionaries), and behaves as follows.

  Se o primário for um objeto de sequência mutável (como uma lista), o subscrito deverá produzir um inteiro. Se for negativo, o comprimento da sequência é adicionado a ela. O valor resultante deve ser um inteiro não negativo menor que o comprimento da sequência, e a sequência é solicitada a atribuir o objeto atribuído ao seu item com esse índice. Se o índice estiver fora do intervalo, a exceção `IndexError` será levantada

(a atribuição a uma sequência subscrita não pode adicionar novos itens a uma lista).

Se o primário for um objeto de mapeamento (como um dicionário), o subscrito deve ter um tipo compatível com o tipo de chave do mapeamento, e o mapeamento é solicitado a criar um par chave/valore que mapeia o subscrito para o objeto atribuído. Isso pode substituir um par de chave/valor existente pelo mesmo valor de chave ou inserir um novo par de chave/valor (se não existir nenhuma chave com o mesmo valor).

If the target is a slicing: The primary expression should evaluate to a mutable sequence object (such as a list). The assigned object should be iterable. The slicing's lower and upper bounds should be integers; if they are `None` (or not present), the defaults are zero and the sequence's length. If either bound is negative, the sequence's length is added to it. The resulting bounds are clipped to lie between zero and the sequence's length, inclusive. Finally, the sequence object is asked to replace the slice with the items of the assigned sequence. The length of the slice may be different from the length of the assigned sequence, thus changing the length of the target sequence, if the target sequence allows it.

Embora a definição de atribuição implique que as sobreposições entre o lado esquerdo e o lado direito sejam "simultâneas" (por exemplo, `a, b = b, a` troca duas variáveis), sobreposições *dentro* da coleção de variáveis atribuídas ocorrem da esquerda para a direita, às vezes resultando em confusão. Por exemplo, o programa a seguir imprime `[0, 2]`:

```
x = [0, 1]
i = 0
i, x[i] = 1, 2         # i é atualizado e, em seguida, x[i] é at
print(x)
```

Ver também

PEP 3132 [https://peps.python.org/pep-3132/] - Desempacotamento estendido de iterável

A especificação para o recurso `*target`.

7.2.1. Instruções de atribuição aumentada

A atribuição aumentada é a combinação, em uma única instrução, de uma operação binária e uma instrução de atribuição:

```
augmented_assignment_stmt: augtarget augop (expression_list | yi
augtarget:                 identifier | attributeref | subscript
augop:                     "+=" | "-=" | "*=" | "@=" | "/=" | "/
                           | ">>=" | "<<=" | "&=" | "^=" | "|="
```

(Veja a seção Primárias para as definições de sintaxe dos últimos três símbolos.)

Uma atribuição aumentada avalia o alvo (que, diferentemente das instruções de atribuição normais, não pode ser um desempacotamento) e a lista de expressões, executa a operação binária específica para o tipo de atribuição nos dois operandos e atribui o resultado ao alvo original. O alvo é avaliado apenas uma vez.

Uma instrução de atribuição aumentada como `x += 1` pode ser reescrita como `x = x + 1` para obter um efeito semelhante, mas não exatamente igual. Na versão aumentada, `x` é avaliado apenas uma vez. Além disso, quando possível, a operação real é executada *no local*, o que significa que, em vez de criar um novo objeto e atribuí-lo ao alvo, o objeto antigo é modificado.

Ao contrário das atribuições normais, as atribuições aumentadas avaliam o lado esquerdo *antes* de avaliar o lado direito. Por exemplo, `a[i] += f(x)` primeiro procura `a[i]`, então avalia `f(x)` e executa a adição e, por último, escreve o resultado de volta para `a[i]`.

Com exceção da atribuição a tuplas e vários alvos em uma única instrução, a atribuição feita por instruções de atribuição aumentada é tratada da mesma maneira que atribuições normais. Da mesma forma, com exceção do

possível comportamento *in-place*, a operação binária executada por atribuição aumentada é a mesma que as operações binárias normais.

Para alvos que são referências de atributos, a mesma advertência sobre atributos de classe e instância se aplica a atribuições regulares.

7.2.2. instruções de atribuição anotado

A atribuição de anotação é a combinação, em uma única instrução, de uma anotação de variável ou atributo e uma instrução de atribuição opcional:

```
annotated_assignment_stmt: augtarget ":" expression
                           ["=" (starred_expression | yield_expr
```

A diferença para as Instruções de atribuição normal é que apenas um único alvo é permitido.

O alvo da atribuição é considerado “simples” se consistir em um único nome que não esteja entre parênteses. Para alvos de atribuição simples, se estiverem no escopo de classe ou módulo, as anotações são reunidas em um escopo de anotação avaliado preguiçosamente. As anotações podem ser avaliadas usando o atributo `__annotations__` de uma classe ou módulo, ou usando os recursos do módulo `annotationlib`.

Se o alvo da atribuição não for simples (um atributo, nó subscrito ou nome entre parênteses), a anotação nunca será avaliada.

Se um nome for anotado em um escopo de função, esse nome será local para esse escopo. As anotações nunca são avaliadas e armazenadas em escopos de função.

Se o lado direito estiver presente, uma atribuição anotada executa a atribuição real como se não houvesse uma anotação presente. Se o lado direito não estiver presente para um alvo de expressão, então o interpretador avalia o alvo, exceto para a última chamada `__setitem__()` ou `__setattr__()`.

Ver também

PEP 526 [https://peps.python.org/pep-0526/] - Sintaxe para Anotações de Variáveis

A proposta que adicionou sintaxe para anotar os tipos de variáveis (incluindo variáveis de classe e variáveis de instância), em vez de expressá-las por meio de comentários.

PEP 484 [https://peps.python.org/pep-0484/] - Dicas de tipo

A proposta que adicionou o módulo `typing` para fornecer uma sintaxe padrão para anotações de tipo que podem ser usadas em ferramentas de análise estática e IDEs.

Alterado na versão 3.8: Agora, as atribuições anotadas permitem as mesmas expressões no lado direito que as atribuições regulares. Anteriormente, algumas expressões (como expressões de tupla sem parênteses) causavam um erro de sintaxe.

Alterado na versão 3.14: As anotações agora têm avaliação preguiçosa em um escopo de anotação separado. Se o alvo da atribuição não for simples, as anotações nunca serão avaliadas.

7.3. A instrução `assert`

As instruções assert são uma maneira conveniente de inserir asserções de depuração em um programa:

`assert_stmt`: `"assert"` **`expression`** `[","` **`expression`**`]`

A forma simples, `assert expression`, é equivalente a

```
if __debug__:
    if not expression: raise AssertionError
```

A forma estendida, `assert expression1, expression2`, é equivalente a

```
if __debug__:
    if not expression1: raise AssertionError(expression2)
```

Essas equivalências presumem que `__debug__` e `AssertionError` referem-se às variáveis embutidas com esses nomes. Na implementação atual, a variável embutida `__debug__` é `True` em circunstâncias normais, `False` quando a otimização é solicitada (opção de linha de comando `-O`). O gerador de código atual não emite código para uma instrução `assert` quando a otimização é solicitada em tempo de compilação. Observe que não é necessário incluir o código-fonte da expressão que falhou na mensagem de erro; ele será exibido como parte do stack trace (situação da pilha de execução).

Atribuições a `__debug__` são ilegais. O valor da variável embutida é determinado quando o interpretador é iniciado.

7.4. A instrução `pass`

```
pass_stmt: "pass"
```

`pass` é uma operação nula — quando é executada, nada acontece. É útil como um espaço reservado quando uma instrução é necessária sintaticamente, mas nenhum código precisa ser executado, por exemplo:

```
def f(arg): pass    # uma função que faz nada (ainda)

class C: pass       # uma classe com nenhum método (ainda)
```

7.5. A instrução `del`

```
del_stmt: "del" target_list
```

A exclusão é definida recursivamente de maneira muito semelhante à maneira como a atribuição é definida. Em vez de explicar em detalhes, aqui estão algumas dicas.

A exclusão de uma lista alvo exclui recursivamente cada alvo, da esquerda para a direita.

A exclusão de um nome remove a ligação desse nome do espaço de nomes local ou global, dependendo se o nome ocorre em uma instrução **`global`** no mesmo bloco de código. Tentar excluir um nome desvinculado levanta uma exceção **`NameError`**.

Deletion of attribute references and subscriptions is passed to the primary object involved; deletion of a slicing is in general equivalent to assignment of an empty slice of the right type (but even this is determined by the sliced object).

Alterado na versão 3.2: Anteriormente, era ilegal excluir um nome do espaço de nomes local se ele ocorresse como uma variável livre em um bloco aninhado.

7.6. A instrução `return`

```
return_stmt: "return" [expression_list]
```

`return` só pode ocorrer sintaticamente aninhado em uma definição de função, não em uma definição de classe aninhada.

Se uma lista de expressões estiver presente, ela será avaliada, caso contrário, será usado o valor `None`.

`return` deixa a chamada da função atual com a lista de expressões (ou `None`) como valor de retorno.

Quando **`return`** passa o controle de uma instrução **`try`** com uma cláusula **`finally`**, essa cláusula **`finally`** é executada antes de realmente sair da função.

Em uma função geradora, a instrução **`return`** indica que o gerador está pronto e fará com que uma exceção **`StopIteration`** seja levantada. O valor

retornado (se houver) é usado como argumento para construir **`StopIteration`** e se torna o atributo **`StopIteration.value`**.

Em uma função de gerador assíncrono, uma instrução **`return`** vazia indica que o gerador assíncrono está pronto e fará com que uma exceção **`StopAsyncIteration`** seja levantada. Uma instrução **`return`** não vazia é um erro de sintaxe em uma função de gerador assíncrono.

7.7. A instrução `yield`

`yield_stmt`: **`yield expression`**

Uma instrução **`yield`** é semanticamente equivalente a uma expressão yield. A instrução `yield` pode ser usada para omitir os parênteses que, de outra forma, seriam necessários na instrução de expressão yield equivalente. Por exemplo, as instruções yield

```
yield <expr>
yield from <expr>
```

são equivalentes às instruções de expressão yield

```
(yield <expr>)
(yield from <expr>)
```

Expressões e instruções yield são usadas apenas ao definir uma função geradora e são usadas apenas no corpo da função geradora. Usar **`yield`** em uma definição de função é suficiente para fazer com que essa definição crie uma função geradora em vez de uma função normal.

Para detalhes completos da semântica **`yield`**, consulte a seção Expressões yield.

7.8. A instrução `raise`

`raise_stmt`: `"raise"` [**`expression`** ["from" **`expression`**]]

Se nenhuma expressão estiver presente, `raise` reativa a exceção em tratamento no momento, também conhecida como *exceção ativa*. Se não houver uma exceção ativa no momento, uma exceção `RuntimeError` é levantada indicando que isso é um erro.

Caso contrário, `raise` avalia a primeira expressão como o objeto de exceção. Deve ser uma subclasse ou uma instância de `BaseException`. Se for uma classe, a instância de exceção será obtida quando necessário, instanciando a classe sem argumentos.

O *tipo* da exceção é a classe da instância de exceção, o *valor* é a própria instância.

Um objeto traceback (situação da pilha de execução) normalmente é criado automaticamente quando uma exceção é levantada e anexada a ele como o atributo `__traceback__`. Você pode criar uma exceção e definir seu próprio traceback em uma etapa usando o método de exceção `with_traceback()` (que retorna a mesma instância de exceção, com seu traceback definido para seu argumento), assim:

```
raise Exception("ocorreu foo").with_traceback(tracebackobj)
```

A cláusula `from` é usada para encadeamento de exceções: se fornecida, a segunda expressão, *expression*, deve ser outra classe ou instância de exceção. Se a segunda expressão for uma instância de exceção, ela será anexada à exceção levantada como o atributo `__cause__` (que é gravável). Se a expressão for uma classe de exceção, a classe será instanciada e a instância resultante será anexada à exceção levantada como o atributo **__cause__**. Se a exceção levantada não for tratada, ambas as exceções serão impressas:

```
>>> try:
...     print(1 / 0)
... except Exception as exc:
...     raise RuntimeError("Something bad happened") from exc
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    print(1 / 0)
```

```
          ~~^~~
ZeroDivisionError: division by zero

The above exception was the direct cause of the following except

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError("Something bad happened") from exc
RuntimeError: Something bad happened
```

Um mecanismo semelhante funciona implicitamente quando uma nova exceção for levantada enquanto outra já estiver sendo tratada. Uma exceção pode ser tratada quando uma cláusula **`except`** ou **`finally`**, ou uma instrução **`with`**, é usada. A exceção anterior é então anexada como o atributo **`__context__`** da nova exceção:

```
>>> try:
...     print(1 / 0)
... except:
...     raise RuntimeError("Something bad happened")
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
    print(1 / 0)
          ~~^~~
ZeroDivisionError: division by zero

During handling of the above exception, another exception occurr

Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
    raise RuntimeError("Something bad happened")
RuntimeError: Something bad happened
```

O encadeamento de exceção pode ser explicitamente suprimido especificando **`None`** na cláusula `from`:

```
>>> try:
...     print(1 / 0)
... except:
```

```
...     raise RuntimeError("Something bad happened") from None
...
Traceback (most recent call last):
  File "<stdin>", line 4, in <module>
RuntimeError: Something bad happened
```

Informações adicionais sobre exceções podem ser encontradas na seção Exceções, e informações sobre como lidar com exceções estão na seção A instrução try.

Alterado na versão 3.3: `None` agora é permitido como `Y` em `raise X from Y`.

Adicionado o atributo `__suppress_context__` para suprimir a exibição automática do contexto de exceção.

Alterado na versão 3.11: Se o traceback da exceção ativa for modificado em uma cláusula `except`, uma instrução `raise` subsequente levantará novamente a exceção com o traceback modificado. Anteriormente, a exceção era levantada novamente com o traceback que tinha quando foi capturada.

7.9. A instrução `break`

```
break_stmt: "break"
```

`break` só pode ocorrer sintaticamente aninhado em um laço `for` ou `while`, mas não aninhado em uma função ou definição de classe dentro desse laço.

Ele termina o laço de repetição mais próximo, pulando a cláusula opcional `else` se o laço tiver uma.

Se um laço `for` é encerrado por `break`, o alvo de controle do laço mantém seu valor atual.

Quando `break` passa o controle de uma instrução `try` com uma cláusula `finally`, essa cláusula `finally` é executada antes de realmente sair do laço.

7.10. A instrução `continue`

```
continue_stmt: "continue"
```

`continue` só pode ocorrer sintaticamente aninhado em um laço `for` ou `while`, mas não em uma função ou definição de classe dentro desse laço. Ele continua com o próximo ciclo do laço de repetição mais próximo.

Quando `continue` passa o controle de uma instrução `try` com uma cláusula `finally`, essa cláusula `finally` é executada antes de realmente iniciar o próximo ciclo do laço.

7.11. A instrução `import`

```
import_stmt:     ["lazy"] "import" module ["as" identifier] (","
                 | ["lazy"] "from" relative_module "import" ider
                 ("," identifier ["as" identifier])*
                 | ["lazy"] "from" relative_module "import" "("
                 ("," identifier ["as" identifier])* [","] ")"
                 | "from" relative_module "import" "*"
module:          (identifier ".")* identifier
relative_module: "."* module | "."+
```

A instrução de importação básica (sem cláusula `from`) é executada em duas etapas:

1. encontra um módulo, carregando e inicializando-o se necessário
2. define a name or names in the current namespace for the scope where the `import` statement occurs, just as an assignment statement would (including `global` and `nonlocal` semantics).

Quando a instrução contém várias cláusulas (separadas por vírgulas), as duas etapas são executadas separadamente para cada cláusula, como se as cláusulas tivessem sido separadas em instruções de importação individuais.

Os detalhes da primeira etapa, encontrar e carregar módulos, estão descritos com mais detalhes na seção sobre o sistema de importação, que também descreve os vários tipos de pacotes e módulos que podem ser importados, bem como todos os os ganchos que podem ser usados para personalizar o sistema de importação. Observe que falhas nesta etapa podem indicar que o módulo não pôde ser localizado *ou* que ocorreu um erro durante a inicialização do módulo, o que inclui a execução do código do módulo.

Se o módulo solicitado for recuperado com sucesso, ele será disponibilizado no espaço de nomes local de três maneiras:

- Se o nome do módulo é seguido pela palavra reservada **`as`**, o nome a seguir é vinculado diretamente ao módulo importado.
- Se nenhum outro nome for especificado e o módulo que está sendo importado for um módulo de nível superior, o nome do módulo será vinculado ao espaço de nomes local como uma referência ao módulo importado
- Se o módulo que está sendo importado *não* for um módulo de nível superior, o nome do pacote de nível superior que contém o módulo será vinculado ao espaço de nomes local como uma referência ao pacote de nível superior. O módulo importado deve ser acessado usando seu nome completo e não diretamente

O formulário **`from`** usa um processo um pouco mais complexo:

1. encontra o módulo especificado na cláusula **`from`**, carregando e inicializando-o se necessário;
2. para cada um dos identificadores especificados nas cláusulas **`import`**:
    1. verifica se o módulo importado tem um atributo com esse nome
    2. caso contrário, tenta importar um submódulo com esse nome e verifica o módulo importado novamente para esse atributo
    3. se o atributo não for encontrado, a exceção **`ImportError`** é levantada.
    4. otherwise, a reference to that value is stored in the current namespace, using the name in the **`as`** clause if it is present, otherwise using the attribute name

Exemplos:

```
import foo                  # foo importado e vinculado localment
import foo.bar.baz          # foo, foo.bar e foo.bar.baz importac
import foo.bar.baz as fbb   # foo, foo.bar e foo.bar.baz importac
from foo.bar import baz     # foo, foo.bar e foo.bar.baz importac
from foo import attr        # foo importado e foo.attr vinculado
```

Se a lista de identificadores for substituída por uma estrela (`'*'`), todos os nomes públicos definidos no módulo serão vinculados ao espaço de nomes local para o escopo onde ocorre a instrução **`import`**.

The *public names* defined by a module are determined by checking the module's namespace for a variable named `__all__`; if defined, it must be a sequence of strings which are names defined or imported by that module. Names containing non-ASCII characters must be in the normalization form [https://www.unicode.org/reports/tr15/#Norm_Forms] NFKC; see Caracteres não ASCII em nomes for details. The names given in `__all__` are all considered public and are required to exist. If `__all__` is not defined, the set of public names includes all names found in the module's namespace which do not begin with an underscore character (`'_'`). `__all__` should contain the entire public API. It is intended to avoid accidentally exporting items that are not part of the API (such as library modules which were imported and used within the module).

A forma curinga de importação — `from module import *` — só é permitida no nível do módulo. Tentar usá-lo em definições de classe ou função irá levantar uma **`SyntaxError`**.

Ao especificar qual módulo importar, você não precisa especificar o nome absoluto do módulo. Quando um módulo ou pacote está contido em outro pacote, é possível fazer uma importação relativa dentro do mesmo pacote superior sem precisar mencionar o nome do pacote. Usando pontos iniciais no módulo ou pacote especificado após **`from`** você pode especificar quão alto percorrer a hierarquia de pacotes atual sem especificar nomes exatos. Um ponto inicial significa o pacote atual onde o módulo que faz a importação existe. Dois pontos significam um nível de pacote acima. Três pontos são dois níveis acima, etc. Então, se você executar `from . import mod` de um módulo no pacote `pkg` então você acabará importando o `pkg.mod`. Se você executar `from ..subpkg2 import mod` de dentro de

`pkg.subpkg1` você irá importar `pkg.subpkg2.mod`. A especificação para importações relativas está contida na seção Importações relativas ao pacote.

`importlib.import_module()` é fornecida para dar suporte a aplicações que determinam dinamicamente os módulos a serem carregados.

Levanta um evento de auditoria `import` com os argumentos `module`, `filename`, `sys.path`, `sys.meta_path`, `sys.path_hooks`.

7.11.1. Lazy imports

The **`lazy`** keyword is a soft keyword that only has special meaning when it appears immediately before an **`import`** or **`from`** statement. When an import statement is preceded by the **`lazy`** keyword, the import becomes *lazy*: the module is not loaded immediately at the import statement. Instead, a lazy proxy object is created and bound to the name. The actual module is loaded on first use of that name.

Lazy imports are only permitted at module scope. Using **`lazy`** inside a function, class body, or **`try`**/**`except`**/**`finally`** block raises a **`SyntaxError`**. Star imports cannot be lazy (`lazy from module import *` is a syntax error), and future statements cannot be lazy.

When using `lazy from ... import`, each imported name is bound to a lazy proxy object. The first access to any of these names triggers loading of the entire module and resolves only that specific name to its actual value. Other names remain as lazy proxies until they are accessed.

Exemplo:

```
lazy import json
import sys

print('json' in sys.modules)  # False - json module not yet loac

# First use triggers loading
result = json.dumps({"hello": "world"})

print('json' in sys.modules)  # True - now loaded
```

If an error occurs during module loading (such as `ImportError` or `SyntaxError`), it is raised at the point where the lazy import is first used, not at the import statement itself.

See **PEP 810** [https://peps.python.org/pep-0810/] for the full specification of lazy imports.

Adicionado na versão 3.15.

7.11.1.1. Compatibility via `__lazy_modules__`

As an alternative to using the `lazy` keyword, a module can opt into lazy loading for specific imports by defining a module-level `__lazy_modules__` variable. When present, it must be a container of fully qualified module name strings. Any regular (non-`lazy`) `import` statement at module scope whose target appears in `__lazy_modules__` is treated as a lazy import, exactly as if the `lazy` keyword had been used.

This provides a way to enable lazy loading for specific dependencies without changing individual `import` statements. This is useful when supporting Python versions older than 3.15 while using lazy imports in 3.15+:

```
__lazy_modules__ = ["json", "pathlib"]

import json     # loaded lazily (name is in __lazy_modules__)
import os       # loaded eagerly (name not in __lazy_modules__)

import pathlib  # loaded lazily
```

Relative imports are resolved to their absolute name before the lookup, so `__lazy_modules__` must always contain fully qualified module names.

For `from`-style imports, the relevant name is the module following `from`, not the names of its members:

```
# In mypackage/mymodule.py
__lazy_modules__ = ["mypackage", "mypackage.sub.utils"]
```

```
from . import helper          # loaded lazily: . resolves to mypa
from .sub.utils import func   # loaded lazily: .sub.utils resolve
import json                   # loaded eagerly (not in __lazy_moc
```

Imports inside functions, class bodies, or `try`/`except`/`finally` blocks are always eager, regardless of **`__lazy_modules__`**.

Adicionado na versão 3.15.

7.11.2. Instruções future

Uma *instrução future* é uma diretiva para o compilador de que um determinado módulo deve ser compilado usando sintaxe ou semântica que estará disponível em uma versão futura especificada do Python, onde o recurso se tornará padrão.

A instrução future destina-se a facilitar a migração para versões futuras do Python que introduzem alterações incompatíveis na linguagem. Ele permite o uso dos novos recursos por módulo antes do lançamento em que o recurso se torna padrão.

```
future_stmt: "from" "__future__" "import" feature ["as" identifi
             ("," feature ["as" identifier])*
             | "from" "__future__" "import" "(" feature ["as" id
             ("," feature ["as" identifier])* [","] ")"
feature:     identifier
```

Uma instrução future deve aparecer perto do topo do módulo. As únicas linhas que podem aparecer antes de uma instrução future são:

- o módulo docstring (se houver),
- comentários,
- linhas vazias e
- outras instruções future.

O único recurso que requer o uso da instrução future é `annotations` (veja **PEP 563** [https://peps.python.org/pep-0563/]).

Todos os recursos históricos habilitados pela instrução future ainda são reconhecidos pelo Python 3. A lista inclui `absolute_import`, `division`, `generators`, `generator_stop`, `unicode_literals`, `print_function`, `nested_scopes` e `with_statement`. Eles são todos redundantes porque estão sempre habilitados e mantidos apenas para compatibilidade com versões anteriores.

Uma instrução future é reconhecida e tratada especialmente em tempo de compilação: as alterações na semântica das construções principais são frequentemente implementadas gerando código diferente. Pode até ser o caso de um novo recurso introduzir uma nova sintaxe incompatível (como uma nova palavra reservada), caso em que o compilador pode precisar analisar o módulo de maneira diferente. Tais decisões não podem ser adiadas até o tempo de execução.

Para qualquer versão, o compilador sabe quais nomes de recursos foram definidos e levanta um erro em tempo de compilação se uma instrução future contiver um recurso desconhecido.

A semântica do tempo de execução direto é a mesma de qualquer instrução de importação: existe um módulo padrão `__future__`, descrito posteriormente, e será importado da maneira usual no momento em que a instrução future for executada.

A semântica interessante do tempo de execução depende do recurso específico ativado pela instrução future.

Observe que não há nada de especial sobre a instrução:

```
import __future__ [as name]
```

Essa não é uma instrução future; é uma instrução de importação comum sem nenhuma semântica especial ou restrições de sintaxe.

O código compilado por chamadas para as funções embutidas `exec()` e `compile()` que ocorrem em um módulo `M` contendo uma instrução future usará, por padrão, a nova sintaxe ou semântica associada com a instrução future. Isso pode ser controlado por argumentos opcionais para `compile()` – veja a documentação dessa função para detalhes.

Uma instrução future tipada digitada em um prompt do interpretador interativo terá efeito no restante da sessão do interpretador. Se um interpretador for iniciado com a opção `-i`, for passado um nome de script para ser executado e o script incluir uma instrução future, ela entrará em vigor na sessão interativa iniciada após a execução do script.

Ver também

PEP 236 [https://peps.python.org/pep-0236/] - De volta ao __future__
A proposta original para o mecanismo do __future__.

7.12. A instrução `global`

```
global_stmt: "global" identifier (","  identifier)*
```

A instrução `global` faz com que os identificadores listados a serem interpretados como globais. Seria impossível atribuir a uma variável global sem `global`, embora variáveis livres possam se referir a globais sem serem declaradas globais.

A instrução `global` se aplica a todo o escopo atual (módulo, corpo da função ou definição de classe). Uma exceção `SyntaxError` é levantada se uma variável for usada ou atribuída antes de sua declaração global no escopo.

No nível do módulo, todas as variáveis são globais, portanto, uma instrução `global` não tem efeito. No entanto, as variáveis ainda não devem ser usadas ou atribuídas antes de sua declaração `global`. Este requisito é flexibilizado no prompt interativo (REPL).

Nota do programador: `global` é uma diretiva para o analisador sintático. Aplica-se apenas ao código analisado ao mesmo tempo que a instrução `global`. Em particular, uma instrução `global` contida em uma string ou objeto código fornecido à função embutida `exec()` não afeta o bloco de código *contendo* a chamada da função e o código contido em tal uma string não é afetada por instruções `global` no código que contém a chamada da função. O mesmo se aplica às funções `eval()` e `compile()`.

7.13. A instrução `nonlocal`

```
nonlocal_stmt: "nonlocal" identifier ("," identifier)*
```

Quando a definição de uma função ou classe está aninhada (incluída) nas definições de outras funções, seus escopos não locais são os escopos locais das funções envolventes. A instrução `nonlocal` faz com que os identificadores listados se refiram a nomes previamente vinculados a escopos não locais. Ele permite que o código encapsulado religue esses identificadores não locais. Se um nome estiver ligado a mais de um escopo não local, a ligação mais próxima será usada. Se um nome não estiver vinculado a nenhum escopo não local, ou se não houver escopo não local, uma exceção `SyntaxError` será levantada.

A instrução `nonlocal` se aplica a todo o escopo de uma função ou corpo da classe. Uma exceção `SyntaxError` é levantada se uma variável for usada ou atribuída antes de sua declaração nonlocal no escopo.

Ver também

PEP 3104 **[https://peps.python.org/pep-3104/]** **- Acesso a nomes em escopos externos**

A especificação para a instrução `nonlocal`.

Nota do programador: `nonlocal` é uma diretiva para o analisador sintático e se aplica apenas ao código analisado junto com ele. Veja a nota para a instrução `global`.

7.14. A instrução `type`

```
type_stmt: 'type' identifier [type_params] "=" expression
```

A instrução **`type`** declara um apelido de tipo, que é uma instância de **`typing.TypeAliasType`**.

Por exemplo, a instrução a seguir cria um apelido de tipo:

```
type Point = tuple[float, float]
```

Este código é aproximadamente equivalente a:

```
annotation-def VALUE_OF_Point():
    return tuple[float, float]
Point = typing.TypeAliasType("Point", VALUE_OF_Point())
```

`annotation-def` indica um escopo de anotação, que se comporta principalmente como uma função, mas com diversas pequenas diferenças.

O valor do apelido de tipo é avaliado no escopo de anotação. Ele não é avaliado quando o apelido de tipo é criado, mas somente quando o valor é acessado através do atributo **`__value__`** do apelido de tipo (veja Avaliação preguiçosa). Isso permite que o apelido de tipo se refira a nomes que ainda não estão definidos.

Apelidos de tipo podem se tornar genéricos adicionando uma lista de parâmetros de tipo após o nome. Veja Apelidos de tipo genérico para mais.

`type` é uma palavra reservada contextual.

Adicionado na versão 3.12.

Ver também

PEP 695 **[https://peps.python.org/pep-0695/]** - **Sintaxe de parâmetros de tipo**
Introduziu a instrução **`type`** e sintaxe para classes e funções genéricas.

8. Instruções compostas

Instruções compostas contém (grupos de) outras instruções; Elas afetam ou controlam a execução dessas outras instruções de alguma maneira. Em geral, instruções compostas abrangem múltiplas linhas, no entanto em algumas manifestações simples uma instrução composta inteira pode estar contida em uma linha.

As instruções `if`, `while` e `for` implementam construções tradicionais de controle do fluxo de execução. `try` especifica tratadores de exceção e/ou código de limpeza para uma instrução ou grupo de instruções, enquanto a palavra reservada `with` permite a execução de código de inicialização e finalização em volta de um bloco de código. Definições de função e classe também são sintaticamente instruções compostas.

Uma instrução composta consiste em uma ou mais “cláusulas”. Uma cláusula consiste em um cabeçalho e um “conjunto”. Os cabeçalhos das cláusulas de uma instrução composta específica estão todos no mesmo nível de indentação. Cada cabeçalho de cláusula começa com uma palavra reservada de identificação exclusiva e termina com dois pontos. Um conjunto é um grupo de instruções controladas por uma cláusula. Um conjunto pode ser uma ou mais instruções simples separadas por ponto e vírgula na mesma linha do cabeçalho, após os dois pontos do cabeçalho, ou pode ser uma ou mais instruções indentadas nas linhas subsequentes. Somente a última forma de conjunto pode conter instruções compostas aninhadas; o seguinte é ilegal, principalmente porque não ficaria claro a qual cláusula `if` a seguinte cláusula `else` pertenceria:

```
if test1: if test2: print(x)
```

Observe também que o ponto e vírgula é mais vinculado que os dois pontos neste contexto, de modo que no exemplo a seguir, todas ou nenhuma das chamadas `print()` são executadas:

```
if x < y < z: print(x); print(y); print(z)
```

Resumindo:

```
compound_stmt: if_stmt
               | while_stmt
               | for_stmt
               | try_stmt
               | with_stmt
               | match_stmt
               | funcdef
               | classdef
               | async_with_stmt
               | async_for_stmt
               | async_funcdef
suite:         stmt_list NEWLINE | NEWLINE INDENT statement+ DED
statement:     stmt_list NEWLINE | compound_stmt
stmt_list:     simple_stmt (";" simple_stmt)* [";"]
```

Note que instruções sempre terminam em uma `NEWLINE` possivelmente seguida por uma `DEDENT`. Note também que cláusulas opcionais de continuação sempre começam com uma palavra reservada que não pode iniciar uma instrução, desta forma não há ambiguidades (o problema do "`else` pendurado" é resolvido em Python obrigando que instruções `if` aninhadas tenham indentação)

A formatação das regras de gramática nas próximas seções põe cada cláusula em uma linha separada para as tornar mais claras.

8.1. A instrução `if`

A instrução `if` é usada para execução condicional:

```
if_stmt: "if" assignment_expression ":" suite
         ("elif" assignment_expression ":" suite)*
         ["else" ":" suite]
```

Ela seleciona exatamente um dos conjuntos *suite* avaliando as expressões *assigment_expression* uma por uma até que uma seja considerada

verdadeira (veja a seção Operações booleanas para a definição de verdadeiro e falso); então esse conjunto é executado (e nenhuma outra parte da instrução `if` é executada ou avaliada). Se todas as expressões forem falsas, o *suite* da cláusula `else`, se presente, é executado.

8.2. A instrução `while`

A instrução `while` é usada para execução repetida desde que uma expressão seja verdadeira:

```
while_stmt: "while" assignment_expression ":" suite
            ["else" ":" suite]
```

Isto testa repetidamente a expressão e, se for verdadeira, executa o primeiro conjunto; se a expressão for falsa (o que pode ser a primeira vez que ela é testada) o conjunto da cláusula **`else`**, se presente, é executado e o laço termina.

Uma instrução `break` executada no primeiro conjunto termina o laço sem executar o conjunto da cláusula **`else`**. Uma instrução `continue` executada no primeiro conjunto ignora o resto do conjunto e volta a testar a expressão.

8.3. A instrução `for`

A instrução `for` é usada para iterar sobre os elementos de uma sequência (como uma string, tupla ou lista) ou outro objeto iterável:

```
for_stmt: "for" target_list "in" starred_expression_list ":" sui
          ["else" ":" suite]
```

A expressão `starred_expression_list` é avaliada uma vez e deve produzir um objeto iterável. Um iterador é criado para esse iterável. O primeiro item fornecido pelo iterador é então atribuído à lista de destino usando as regras padrão de atribuição (veja Instruções de atribuição), e o conjunto é executado. Isso se repete para cada item fornecido pelo iterador.

Quando o iterador está esgotado, o conjunto na cláusula **`else`**, se presente, é executado e o laço termina.

Uma instrução **`break`** executada no primeiro conjunto termina o loop sem executar o conjunto da cláusula **`else`**. Uma instrução **`continue`** executada no primeiro conjunto pula o resto do conjunto e continua com o próximo item, ou com a cláusula **`else`** se não houver próximo item.

O laço for faz atribuições às variáveis na lista de destino. Isso substitui todas as atribuições anteriores a essas variáveis, incluindo aquelas feitas no conjunto do laço for:

```
for i in range(10):
    print(i)
    i = 5             # não afeta o laço for, porque i será
                      # substituída pelo índice seguinte no rang
```

Os nomes na lista de destinos não são excluídos quando o laço termina, mas se a sequência estiver vazia, eles não serão atribuídos pelo laço. Dica: o tipo embutido **`range()`** representa sequências aritméticas imutáveis de inteiros. Por exemplo, iterar `range(3)` sucessivamente produz 0, 1 e depois 2.

Alterado na versão 3.11: Elementos marcados com estrela agora são permitidos na lista de expressões.

8.4. A instrução `try`

A instrução **`try`** especifica manipuladores de exceção e/ou código de limpeza para um grupo de instruções:

```
try_stmt:  try1_stmt | try2_stmt | try3_stmt
try1_stmt: "try" ":" suite
           ("except" [expression ["as" identifier]] ":" suite)+
           ["else" ":" suite]
           ["finally" ":" suite]
try2_stmt: "try" ":" suite
           ("except" "*" expression ["as" identifier] ":" suite)
```

```
            ["else" ":" suite]
            ["finally" ":" suite]
try3_stmt: "try" ":" suite
           "finally" ":" suite
```

Informações adicionais sobre exceções podem ser encontradas na seção Exceções, e informações sobre como usar a instrução **`raise`** para gerar exceções podem ser encontradas na seção A instrução raise.

Alterado na versão 3.14: Suporte para a remoção opcional de parênteses de agrupamento ao usar vários tipos de exceção. Veja **PEP 758** [https://peps.python.org/pep-0758/].

8.4.1. Cláusula `except`

As cláusulas **`except`** especificam um ou mais manipuladores de exceção. Quando nenhuma exceção ocorre na cláusula **`try`**, nenhum manipulador de exceção é executado. Quando uma exceção ocorre no bloco **`try`**, uma busca por um manipulador de exceção é iniciada. Essa busca inspeciona as cláusulas **`except`** por vez até encontrar uma que corresponda à exceção. Uma cláusula **`except`** sem expressão, se presente, deve ser a última; ela corresponde a qualquer exceção.

Para uma cláusula **`except`** com uma expressão, a expressão deve ser avaliada como um tipo de exceção ou como uma tupla de tipos de exceção. Parênteses podem ser omitidos se vários tipos de exceção forem fornecidos e a cláusula `as` não for usada. A exceção levantada corresponde a uma cláusula **`except`** cuja expressão é avaliada como a classe ou uma classe base não virtual do objeto exceção, ou como uma tupla que contém tal classe.

Se nenhuma cláusula **`except`** corresponder à exceção, a busca por um manipulador de exceção continua no código circundante e na pilha de invocação. [[1]]

Se a avaliação de uma expressão no cabeçalho de uma cláusula **`except`** levantar uma exceção, a busca original por um manipulador será cancelada e uma busca pela nova exceção será iniciada no código circundante e na

pilha de chamadas (ela é tratada como se toda a instrução **`try`** levantasse a exceção).

Quando uma cláusula **`except`** correspondente é encontrada, a exceção é atribuída ao destino especificado após a palavra reservada **`as`** nessa cláusula **`except`**, se presente, e o conjunto da cláusula **`except`** é executado. Todas as cláusulas **`except`** devem ter um bloco executável. Quando o final deste bloco é atingido, a execução continua normalmente após toda a instrução **`try`**. (Isso significa que, se existirem dois manipuladores aninhados para a mesma exceção e a exceção ocorrer na cláusula **`try`** do manipulador interno, o manipulador externo não tratará a exceção.)

Quando uma exceção é atribuída usando `as target`, ela é limpa no final da cláusula **`except`**. É como se

```
except E as N:
    foo
```

fosse traduzido para

```
except E as N:
    try:
        foo
    finally:
        del N
```

Isso significa que a exceção deve ser atribuída a um nome diferente para poder ser referenciada após a cláusula **`except`**. As exceções são limpas porque, com o traceback (situação da pilha de execução) anexado a elas, formam um ciclo de referência com o quadro de pilha, mantendo todos os locais nesse quadro vivos até que ocorra a próxima coleta de lixo.

Antes de um conjunto de cláusulas **`except`** ser executado, a exceção é armazenada no módulo **`sys`**, onde pode ser acessada no corpo da cláusula **`except`** chamando **`sys.exception()`**. Ao sair de um manipulador de exceções, a exceção armazenada no módulo **`sys`** é redefinida para seu valor anterior:

```
>>> print(sys.exception())
None
>>> try:
...     raise TypeError
... except:
...     print(repr(sys.exception()))
...     try:
...          raise ValueError
...     except:
...         print(repr(sys.exception()))
...     print(repr(sys.exception()))
...
TypeError()
ValueError()
TypeError()
>>> print(sys.exception())
None
```

8.4.2. Cláusula `except*`

A cláusula **`except*`** especifica um ou mais manipuladores para grupos de exceções (instâncias de **`BaseExceptionGroup`**). Uma instrução **`try`** pode ter cláusulas **`except`** ou **`except*`**, mas não ambas. O tipo de exceção para correspondência é obrigatório no caso de **`except*`**, portanto, `except*:` resulta em um erro de sintaxe. O tipo é interpretado como no caso de **`except`**, mas a correspondência é realizada com as exceções contidas no grupo que está sendo tratado. Uma exceção **`TypeError`** é levantada se um tipo correspondente for uma subclasse de **`BaseExceptionGroup`**, pois isso teria uma semântica ambígua.

Quando um grupo de exceções é levantado dentro de um bloco try, cada cláusula **`except*`** o divide (consulte **`split()`**) em subgrupos de exceções correspondentes e não correspondentes. Se o subgrupo correspondente não estiver vazio, ele se torna a exceção tratada (o valor retornado por **`sys.exception()`**) e é atribuído ao alvo da cláusula **`except*`** (se houver). Em seguida, o corpo da cláusula **`except*`** é executado. Se o subgrupo não correspondente não estiver vazio, ele é processado pela próxima cláusula **`except*`** da mesma maneira. Isso continua até que todas as exceções do

grupo tenham sido tratadas ou até que a última cláusula **`except*`** tenha sido executada.

Após a execução de todas as cláusulas **`except*`**, o grupo de exceções não tratadas é mesclado com quaisquer exceções que foram levantadas uma ou mais vezes em cláusulas **`except*`**. Esse grupo de exceções, mesclado, se propaga:

```
>>> try:
...     raise ExceptionGroup("eg",
...         [ValueError(1), TypeError(2), OSError(3), OSError(4)
... except* TypeError as e:
...     print(f'caught {type(e)} with nested {e.exceptions}')
... except* OSError as e:
...     print(f'caught {type(e)} with nested {e.exceptions}')
...
caught <class 'ExceptionGroup'> with nested (TypeError(2),)
caught <class 'ExceptionGroup'> with nested (OSError(3), OSEror
  + Exception Group Traceback (most recent call last):
  |   File "<doctest default[0]>", line 2, in <module>
  |     raise ExceptionGroup("eg",
  |         [ValueError(1), TypeError(2), OSError(3), OSError(4)
  | ExceptionGroup: eg (1 sub-exception)
  +-+---------------- 1 ----------------
    | ValueError: 1
    +------------------------------------
```

Se a exceção levantada do bloco **`try`** não for um grupo de exceções e seu tipo corresponder a uma das cláusulas **`except*`**, ela será capturada e encapsulada por um grupo de exceções com uma string de mensagem vazia. Isso garante que o tipo do alvo e seja **`BaseExceptionGroup`** de forma consistente:

```
>>> try:
...     raise BlockingIOError
... except* BlockingIOError as e:
...     print(repr(e))
...
ExceptionGroup('', (BlockingIOError(),))
```

break, **continue** e **return** não podem aparecer em uma cláusula **except***.

8.4.3. Cláusula `else`

A cláusula opcional **else** é executada se o fluxo de controle deixar o conjunto **try**, nenhuma exceção foi levantada e nenhuma instrução **return**, **continue** ou **break** foi executada. Exceções na cláusula **else** não são manipuladas pelas cláusulas **except** precedentes.

8.4.4. Cláusula `finally`

Se **finally** estiver presente, especifica um manipulador de "limpeza". A cláusula **try** é executada, incluindo quaisquer cláusulas **except** e **else**. Se uma exceção ocorrer em qualquer uma das cláusulas e não for tratada, a exceção será salva temporariamente. A cláusula **finally** é executada. Se houver uma exceção salva, ela será levantada novamente no final da cláusula **finally**. Se a cláusula **finally** levantar outra exceção, a exceção salva será definida como o contexto da nova exceção. Se a cláusula **finally** executar uma instrução **return**, **break** ou **continue**, a exceção salva será descartada. Por exemplo, esta função retorna 42.

```
def f():
    try:
        1/0
    finally:
        return 42
```

As informações de exceção não estão disponíveis para o programa durante a execução da cláusula **finally**.

Quando uma instrução **return**, **break** ou **continue** é executada no conjunto **try** de uma instrução **try**...**finally**, a cláusula **finally** também é executada "na saída".

O valor de retorno de uma função é determinado pela última instrução **return** executada. Como a cláusula **finally** sempre é executada, uma instrução **return** executada na cláusula **finally** sempre será a última executada. A função a seguir retorna "finally".

```
def foo():
    try:
        return 'try'
    finally:
        return 'finally'
```

Alterado na versão 3.8: Antes do Python 3.8, uma instrução `continue` era ilegal na cláusula `finally` devido a um problema com a implementação.

Alterado na versão 3.14: O compilador emite uma exceção `SyntaxWarning` quando um `return`, `break` ou `continue` aparece em um bloco `finally` (veja **PEP 765** [https://peps.python.org/pep-0765/]).

8.5. A instrução `with`

A instrução `with` envolve a execução de um bloco com métodos definidos por um gerenciador de contexto (veja a seção Gerenciadores de contexto da instrução with). Isso permite que padrões comuns de uso de `try`…`except`…`finally` sejam encapsulados para reutilização conveniente.

```
with_stmt:          "with" ( "(" with_stmt_contents ","? ")" | w
with_stmt_contents: with_item ("," with_item)*
with_item:          expression ["as" target]
```

A execução da instrução `with` com um “item” ocorre da seguinte maneira:

1. A expressão de contexto (a expressão fornecida em `with_item`) é avaliada para obter um gerenciador de contexto.

2. O método `__enter__()` do gerenciador de contexto é carregado para uso posterior.

3. O método `__exit__()` do gerenciador de contexto é carregado para uso posterior.

4. O método `__enter__()` do gerenciador de contexto é invocado.

5. Se um alvo foi incluído na instrução `with`, o valor de retorno de `__enter__()` é atribuído a esse alvo.

   Nota

   A instrução `with` garante que, se o método `__enter__()` retornar sem erro, `__exit__()` sempre será chamado. Assim, se ocorrer um erro durante a atribuição à lista de alvos, ele será tratado da mesma forma que um erro ocorrendo dentro do conjunto seria. Veja a etapa 7 abaixo.

6. O conjunto *suite* é executado.

7. O método `__exit__()` do gerenciador de contexto é invocado. Se uma exceção fez com que o conjunto fosse encerrado, seu tipo, valor e traceback são passados como argumentos para **`__exit__()`**. Caso contrário, três argumentos `None` são fornecidos.

   Se o conjunto foi encerrado devido a uma exceção, e o valor de retorno do método `__exit__()` for falso, a exceção é levantada novamente. Se o valor de retorno era verdadeiro, a exceção é suprimida, e a execução continua com a instrução após a instrução `with`.

   Se o conjunto foi encerrado por qualquer motivo diferente de uma exceção, o valor de retorno de `__exit__()` é ignorado e a execução prossegue no local normal para o tipo de saída que foi gerada.

O seguinte código:

```
with EXPRESSION as TARGET:
    SUITE
```

é semanticamente equivalente a:

```
manager = (EXPRESSION)
enter = manager.__enter__
exit = manager.__exit__
value = enter()
```

```
hit_except = False

try:
    TARGET = value
    SUITE
except:
    hit_except = True
    if not exit(*sys.exc_info()):
        raise
finally:
    if not hit_except:
        exit(None, None, None)
```

exceto que a pesquisa de método especial é usada de forma implícita para `__enter__()` e `__exit__()`.

Com mais de um item, os gerenciadores de contexto são processados como se várias instruções `with` estivessem aninhadas:

```
with A() as a, B() as b:
    SUITE
```

é semanticamente equivalente a:

```
with A() as a:
    with B() as b:
        SUITE
```

Você também pode escrever gerenciadores de contexto multi-item em várias linhas se os itens estiverem entre parênteses. Por exemplo:

```
with (
    A() as a,
    B() as b,
):
    SUITE
```

Alterado na versão 3.1: Suporte para múltiplas expressões de contexto.

Alterado na versão 3.10: Suporte para usar parênteses de agrupamento para dividir a instrução em várias linhas.

Ver também

PEP 343 [https://peps.python.org/pep-0343/] - A instrução "with"
A especificação, o histórico e os exemplos da instrução Python `with`.

8.6. A instrução `match`

Adicionado na versão 3.10.

A instrução match é usada para correspondência de padrões. Sintaxe:

```
match_stmt:   'match' subject_expr ":" NEWLINE INDENT case_block
subject_expr: flexible_expression "," [flexible_expression_list
              | assignment_expression
case_block:   'case' patterns [guard] ":" suite
```

Nota

Esta seção usa aspas simples para denotar palavras reservadas contextuais.

A correspondência de padrões aceita um padrão como entrada (seguindo `case`) e um valor de sujeito (seguindo `match`). O padrão (que pode conter subpadrões) é correspondido ao valor de assunto. Os resultados são:

- Um sucesso ou falha de correspondência (também chamado de sucesso ou falha de padrão).
- Possível vinculação de valores correspondentes a um nome. Os pré-requisitos para isso são discutidos mais adiante.

As palavras reservadas `match` e `case` são palavras reservadas contextuais.

Ver também

- **PEP 634** [https://peps.python.org/pep-0634/] – Structural Pattern Matching: Specification
- **PEP 636** [https://peps.python.org/pep-0636/] – Correspondência de padrões estruturais: Tutorial

8.6.1. Visão Geral

Aqui está uma visão geral do fluxo lógico de uma instrução match:

1. A expressão de sujeito `subject_expr` é avaliada e um valor de sujeito resultante é obtido. Se a expressão de sujeito contiver uma vírgula, uma tupla é construída usando as regras padrão.

2. Cada padrão em um `case_block` é tentado para corresponder ao valor de sujeito. As regras específicas para sucesso ou falha são descritas abaixo. A tentativa de correspondência também pode vincular alguns ou todos os nomes autônomos dentro do padrão. As regras precisas de vinculação de padrão variam por tipo de padrão e são especificadas abaixo. **As vinculações de nome feitas durante uma correspondência de padrão bem-sucedida sobrevivem ao bloco executado e podem ser usadas após a instrução match**.

   Nota

   Durante correspondências de padrões com falha, alguns subpadrões podem ter sucesso. Não confie em vinculações sendo feitas para uma correspondência com falha. Por outro lado, não confie em variáveis permanecendo inalteradas após uma correspondência com falha. O comportamento exato depende da implementação e pode variar. Esta é uma decisão intencional feita para permitir que diferentes implementações adicionem otimizações.

3. Se o padrão for bem-sucedido, o *guard* correspondente (se presente) é avaliado. Neste caso, todas as vinculações de nome são garantidas como tendo acontecido.

- Se o *guard* for avaliado como verdadeiro ou estiver ausente, o `block` dentro de `case_block` será executado.
- Caso contrário, o próximo `case_block` será tentado conforme descrito acima.
- Se não houver mais blocos de caso, a instrução match será concluída.

Nota

Os usuários geralmente nunca devem confiar em um padrão sendo avaliado. Dependendo da implementação, o interpretador pode armazenar valores em cache ou usar outras otimizações que pulam avaliações repetidas.

Um exemplo de instrução match:

```
>>> flag = False
>>> match (100, 200):
...     case (100, 300):  # Não corresponde: 200 != 300
...         print('Case 1')
...     case (100, 200) if flag:  # Corresponde com sucesso, mas
         print('Case 2')
...     case (100, y):  # Corresponde e vincula y a 200
...         print(f'Case 3, y: {y}')
...     case _:  # Padrão não testado
...         print('Case 4, I match anything!')
...
Case 3, y: 200
```

Neste caso, `if flag` é um *guard*. Leia mais sobre isso na próxima seção.

8.6.2. Guards

guard: "if" [assignment expression]

Um `guard` (que faz parte do `case`) deve ter sucesso para que o código dentro do bloco `case` seja executado. Ele assume a forma: `if` seguido por uma expressão.

O fluxo lógico de um bloco `case` com um `guard` é o seguinte:

1. Verifique se o padrão no bloco `case` foi bem-sucedido. Se o padrão falhou, o `guard` não é avaliado e o próximo bloco `case` é verificado.
2. Se o padrão for bem-sucedido, avalia o `guard`.
    - Se a condição `guard` for avaliada como verdadeira, o bloco de caso será selecionado.
    - Se a condição `guard` for avaliada como falsa, o bloco de caso não será selecionado.
    - Se o `guard` levantar uma exceção durante a avaliação, a exceção surgirá.

Guards podem ter efeitos colaterais, pois são expressões. A avaliação de guards deve prosseguir do primeiro ao último bloco de caso, um de cada vez, pulando blocos de caso cujos padrões não são todos bem-sucedidos. (Isto é, a avaliação de guardas deve acontecer em ordem.) A avaliação de guards deve parar quando um bloco de caso for selecionado.

8.6.3. Blocos irrefutáveis de case

Um bloco irrefutável de case é um bloco de case que corresponde a qualquer valor. Uma instrução match pode ter no máximo um bloco irrefutável de case, e ele deve ser o último.

Um bloco de case é considerado irrefutável se não tiver guard e seu padrão for irrefutável. Um padrão é considerado irrefutável se pudermos provar somente por sua sintaxe que ele sempre terá sucesso. Somente os seguintes padrões são irrefutáveis:

- Padrões AS cujo lado esquerdo é irrefutável
- Padrões OR contendo pelo menos um padrão irrefutável
- Padrões de captura
- Padrões curingas
- padrões irrefutáveis entre parêteses

8.6.4. Padrões

Nota

Esta seção usa notações de gramática para além do padrão de EBNF:

- a notação `SEP.RULE+` é uma abreviação para `RULE (SEP RULE)*`
- a notação `!RULE` é uma abreviação para uma asserção de negação antecipada.

Esta é a sintaxe de nível superior para `patterns` (padrões):

```
patterns:       open_sequence_pattern | pattern
pattern:        as_pattern | or_pattern
closed_pattern: | literal_pattern
                | capture_pattern
                | wildcard_pattern
                | value_pattern
                | group_pattern
                | sequence_pattern
                | mapping_pattern
                | class_pattern
```

As descrições abaixo incluirão uma descrição “em termos simples” de o que o padrão faz para fins ilustrativos (créditos a Raymond Hettinger pelo documento que inspirou a maioria das descrições). Note que essas descrições são puramente para fins ilustrativos, e **não necessariamente** refletem a implementação subjacente. Além disso, elas não cobrem todas as formas válidas.

8.6.4.1. Padrões OR

Um padrão OR é composto por dois ou mais padrões separados por barras verticais `|`. Sintaxe:

```
or_pattern: "|".closed_pattern+
```

Somente o último subpadrão pode ser irrefutável, e cada subpadrão deve vincular o mesmo conjunto de nomes para evitar ambiguidades.

Um padrão OR testa a correspondência de cada um dos seus subpadrões, em sequência, ao valor do sujeito, até que uma delas seja bem sucedida. O padrão OR é então considerado bem sucedido. Caso contrário, se todas elas falharam, o padrão OR falhou.

Em termos simples, `P1 | P2 | ...` vai tentar fazer corresponder `P1`, se falhar vai tentar `P2`, declarando sucesso se houver sucesso em qualquer uma das tentativas, e falhando caso contrário.

8.6.4.2. Padrões AS

Um padrão AS corresponde a um padrão OR à esquerda da palavra reservada **`as`** de um assunto. Sintaxe:

```
as_pattern: or_pattern "as" capture_pattern
```

Se o padrão OR falhar, o padrão AS falhará. Caso contrário, o padrão AS vincula o assunto ao nome à direita da palavra-chave as e obtém sucesso. `capture_pattern` não pode ser um _.

Em termos simples, `P as NAME` corresponderá a `P` e, em caso de sucesso, definirá `NAME = <assunto>`.

8.6.4.3. Padrões literais

Um padrão literal corresponde à maioria dos literais em Python. Sintaxe:

```
literal_pattern: signed_number
               | signed_number "+" NUMBER
               | signed_number "-" NUMBER
               | strings
               | "None"
               | "True"
```

```
                  | "False"
signed_number:    ["+" | "-"] NUMBER
```

A regra `strings` e o token `NUMBER` são definidos na gramática padrão do Python. Strings entre aspas triplas são suportadas. Strings brutas e strings de bytes são suportadas. Literais de strings formatadas e Literais de string template (t-strings) não são suportadas.

As formas `signed_number '+' NUMBER` e `signed_number '-' NUMBER` são para expressar números complexos; elas requerem um número real à esquerda e um número imaginário à direita. Por exemplo, `3 + 4j`.

Em termos simples, `LITERAL` terá sucesso somente se `<assunto> == LITERAL`. Para os singletons `None`, `True` e `False`, o operador **`is`** é usado.

8.6.4.4. Padrões de captura

Um padrão de captura vincula o valor do assunto a um nome. Sintaxe:

```
capture_pattern: !'_' NAME
```

Um único sublinhado _ não é um padrão de captura (é o que `!'_'` expressa). Em vez disso, ele é tratado como um **`wildcard pattern`**.

Em um determinado padrão, um determinado nome só pode ser vinculado uma vez. Por exemplo, `case x, x: ...` é inválido enquanto `case [x] | x: ...` é permitido.

Os padrões de captura sempre são bem-sucedidos. A vinculação segue regras de escopo estabelecidas pelo operador de expressão de atribuição na **PEP 572** [https://peps.python.org/pep-0572/]; o nome se torna uma variável local no escopo de função de contenção mais próximo, a menos que haja uma instrução **`global`** ou **`nonlocal`** aplicável.

Em termos simples, `NAME` sempre terá sucesso e definirá `NAME = <assunto>`.

8.6.4.5. Padrões curingas

Um padrão curinga sempre tem sucesso (corresponde a qualquer coisa) e não vincula nenhum nome. Sintaxe:

```
wildcard_pattern: '_'
```

_ é uma palavra reservada contextual dentro de qualquer padrão, mas somente dentro de padrões. É um identificador, como de costume, mesmo dentro de expressões de assunto `match`s, `guard`s e blocos `case`.

Em termos simples, _ sempre terá sucesso.

8.6.4.6. Padrões de valor

Um padrão de valor representa um valor nomeado em Python. Sintaxe:

```
value_pattern: attr
attr:          name_or_attr "." NAME
name_or_attr:  attr | NAME
```

O nome pontilhado no padrão é pesquisado usando as regras de resolução de nomes padrão do Python. O padrão é bem-sucedido se o valor encontrado for comparado igual ao valor do assunto (usando o operador de igualdade ==).

Em termos simples, `NAME1.NAME2` terá sucesso somente se `<assunto> == NAME1.NAME2`

Nota

Se o mesmo valor ocorrer várias vezes na mesma instrução match, o interpretador pode armazenar em cache o primeiro valor encontrado e reutilizá-lo em vez de repetir a mesma pesquisa. Esse cache é estritamente vinculado a uma determinada execução de uma determinada instrução match.

8.6.4.7. Padrões de grupo

Um padrão de grupo permite que os usuários adicionem parênteses em torno de padrões para enfatizar o agrupamento pretendido. Caso contrário, não há sintaxe adicional. Sintaxe:

```
group_pattern: "(" pattern ")"
```

Em termos simples, `(P)` tem o mesmo efeito que `P`.

8.6.4.8. Padrões de sequência

Um padrão de sequência contém vários subpadrões a serem correspondidos com elementos de sequência. A sintaxe é similar ao desempacotamento de uma lista ou tupla.

```
sequence_pattern:        "[" [maybe_sequence_pattern] "]"
                         | "(" [open_sequence_pattern] ")"
open_sequence_pattern:   maybe_star_pattern "," [maybe_sequence_p
maybe_sequence_pattern:  ",".maybe_star_pattern+ ","?
maybe_star_pattern:      star_pattern | pattern
star_pattern:            "*" (capture_pattern | wildcard_pattern)
```

Não há diferença se parênteses ou colchetes são usados para padrões de sequência (por exemplo, `(...)` vs `[...]`).

Nota

Um único padrão entre parênteses sem uma vírgula final (por exemplo, `(3 | 4)`) é um padrão de grupo. Enquanto um único padrão entre colchetes (por exemplo, `[3 | 4]`) ainda é um padrão de sequência.

No máximo um subpadrão de estrela pode estar em um padrão de sequência. O subpadrão de estrela pode ocorrer em qualquer posição. Se nenhum subpadrão de estrela estiver presente, o padrão de sequência é um

padrão de sequência de comprimento fixo; caso contrário, é um padrão de sequência de comprimento variável.

A seguir está o fluxo lógico para corresponder um padrão de sequência com um valor de assunto:

1. Se o valor do assunto não for uma sequência [[2]], o padrão de sequência falhará.

2. Se o valor do assunto for uma instância de `str`, `bytes` ou `bytearray`, o padrão de sequência falhará.

3. As etapas subsequentes dependem se o padrão de sequência é fixo ou de comprimento variável.

   Se o padrão de sequência for de comprimento fixo:

   1. Se o comprimento da sequência do assunto não for igual ao número de subpadrões, o padrão da sequência falha
   2. Subpadrões no padrão de sequência são correspondidos aos seus itens correspondentes na sequência de assunto da esquerda para a direita. A correspondência para assim que um subpadrão falha. Se todos os subpadrões tiverem sucesso em corresponder ao seu item correspondente, o padrão de sequência é bem-sucedido.

   Caso contrário, se o padrão de sequência for de comprimento variável:

   1. Se o comprimento da sequência do assunto for menor que o número de subpadrões não-estrela, o padrão da sequência falha.
   2. Os principais subpadrões não estelares são correspondidos aos seus itens correspondentes, como nas sequências de comprimento fixo.
   3. Se a etapa anterior for bem-sucedida, o subpadrão estrela corresponde a uma lista formada pelos itens de assunto restantes, excluindo os itens restantes correspondentes aos subpadrões não-estrela que seguem o subpadrão estrela.
   4. Os subpadrões não-estrela restantes são correspondidos aos seus itens de assunto correspondentes, como em uma sequência de

comprimento fixo.

Nota

O comprimento da sequência de assunto é obtido via `len()` (ou seja, via protocolo `__len__()`). Esse comprimento pode ser armazenado em cache pelo interpretador de forma similar a padrões de valor.

Em termos simples, `[P1, P2, P3, ... , P<N>]` corresponde somente se tudo o seguinte acontecer:

- verifica se `<subject>` é uma sequência
- `len(subject) == <N>`
- `P1` corresponde a `<subject>[0]` (observe que esta correspondência também pode vincular nomes)
- `P2` corresponde a `<subject>[1]` (observe que esta correspondência também pode vincular nomes)
- … e assim por diante para o padrão/elemento correspondente.

8.6.4.9. Padrões de mapeamento

Um padrão de mapeamento contém um ou mais padrões de chave-valor. A sintaxe é similar à construção de um dicionário. Sintaxe:

```
mapping_pattern:     "{" [items_pattern] "}"
items_pattern:       ",".key_value_pattern+ ","?
key_value_pattern:   (literal_pattern | value_pattern) ":" patte
                     | double_star_pattern
double_star_pattern: "**" capture_pattern
```

No máximo um padrão de estrela dupla pode estar em um padrão de mapeamento. O padrão de estrela dupla deve ser o último subpadrão no padrão de mapeamento.

Chaves duplicadas em padrões de mapeamento não são permitidas. Chaves literais duplicadas levantarão um `SyntaxError`. Duas chaves que de outra

forma têm o mesmo valor levantarão `ValueError` em tempo de execução.

A seguir está o fluxo lógico para comparar um padrão de mapeamento com um valor de assunto:

1. Se o valor do assunto não for um mapeamento [[3]], o padrão de mapeamento falhará.
2. Se cada chave fornecida no padrão de mapeamento estiver presente no mapeamento de assunto, e o padrão para cada chave corresponder ao item correspondente do mapeamento de assunto, o padrão de mapeamento será bem-sucedido.
3. Se chaves duplicadas forem detectadas no padrão de mapeamento, o padrão será considerado inválido. Uma exceção `SyntaxError` é levantada para valores literais duplicados; ou `ValueError` para chaves nomeadas do mesmo valor.

Nota

Os pares de chave-valor são correspondidos usando o formato de dois argumentos do método `get()` do assunto do mapeamento. Os pares de chave-valor correspondidos já devem estar presentes no mapeamento e não devem ser criados em tempo de uso via `__missing__()` ou `__getitem__()`.

Em termos simples, `{KEY1: P1, KEY2: P2, ... }` corresponde somente se tudo o seguinte acontecer:

- verifica se `<assunto>` é um mapeamento
- `KEY1 in <assunto>`
- `P1` corresponde a `<assunto>[KEY1]`
- … e assim por diante para o par KEY/elemento correspondente.

8.6.4.10. Padrões de classe

Um padrão de classe representa uma classe e seus argumentos nomeados e posicionais (se houver). Sintaxe:

```
class_pattern:       name_or_attr "(" [pattern_arguments ","?] '
pattern_arguments:   positional_patterns ["," keyword_patterns]
                     | keyword_patterns
positional_patterns: ",".pattern+
keyword_patterns:    ",".keyword_pattern+
keyword_pattern:     NAME "=" pattern
```

O mesmo argumento nomeado não deve ser repetido em padrões de classe.

A seguir está o fluxo lógico para corresponder a um padrão de classe com um valor de assunto:

1. Se `name_or_attr` não for uma instância do tipo embutido `type` , levanta `TypeError`.

2. Se o valor do assunto não for uma instância de `name_or_attr` (testado via `isinstance()`), o padrão de classe falhará.

3. Se nenhum argumento de padrão estiver presente, o padrão é bem-sucedido. Caso contrário, as etapas subsequentes dependem se os padrões de argumento posicional ou nomeado estão presentes.

   Para vários tipos embutidos (especificados abaixo), um único subpadrão posicional é aceito, o qual corresponderá a todo o assunto; para esses tipos, os padrões de argumentos nomeados também funcionam como para outros tipos.

   Se apenas padrões de argumentos nomeados estiverem presentes, eles serão processados da seguinte forma, um por um:

   I. A palavra reservada é procurada como um atributo no assunto.
      - Se isso levantar uma exceção diferente de `AttributeError`, a exceção será exibida.
      - Se isso levantar `AttributeError`, o padrão de classe falhou.
      - Caso contrário, o subpadrão associado ao padrão de argumento nomeado é correspondido ao valor de atributo do sujeito. Se isso falhar, o padrão de classe falha; se isso for

bem-sucedido, a correspondência prossegue para o próximo argumento nomeado.

II. Se todos os padrões de argumento nomeado forem bem-sucedidos, o padrão de classe será bem-sucedido.

Se houver algum padrão posicional presente, ele será convertido em padrões de argumento nomeado usando o atributo `__match_args__` na classe `name_or_attr` antes da correspondência:

I. O equivalente de `getattr(cls, "__match_args__", ())` é chamado.

- Se isso levantar uma exceção, a exceção surgirá.
- Se o valor retornado não for uma tupla, a conversão falhará e `TypeError` será levantada.
- Se houver mais padrões posicionais do que `len(cls.__match_args__)`, `TypeError` será levantada.
- Caso contrário, o padrão posicional `i` é convertido em um padrão de argumento nomeado usando `__match_args__[i]` como argumento nomeado. `__match_args__[i]` deve ser uma string; caso contrário, `TypeError` é levantada.
- Se houver argumentos nomeados duplicados, `TypeError` será levantada.

Ver também

Customizando argumentos posicionais na classe correspondência de padrão

II. Uma vez que todos os padrões posicionais foram convertidos em padrões de argumentos nomeados, a correspondência continua como se houvesse apenas padrões de palavras reservadas.

Para os seguintes tipos embutidos, o tratamento de subpadrões posicionais é diferente:

- `bool`

- **bytearray**
- **bytes**
- **dict**
- **float**
- **frozendict**
- **frozenset**
- **int**
- **list**
- **set**
- **str**
- **tuple**

Essas classes aceitam um único argumento posicional, e o padrão ali é correspondido ao objeto inteiro em vez de um atributo. Por exemplo, `int(0|1)` corresponde ao valor `0`, mas não ao valor `0.0`.

Em termos simples, `CLS(P1, attr=P2)` corresponde somente se o seguinte acontecer:

- `isinstance(<assunto>, CLS)`
- converte `P1` em um padrão de argumento nomeado usando `CLS.__match_args__`
- Para cada argumento nomeado `attr=P2`:
  - `hasattr(<assunto>, "attr")`
  - `P2` corresponde a `<assunto>.attr`
- … e assim por diante para o par argumento nomeado/elemento correspondente.

Ver também

- **PEP 634** [https://peps.python.org/pep-0634/] – Structural Pattern Matching: Specification
- **PEP 636** [https://peps.python.org/pep-0636/] – Correspondência de padrões estruturais: Tutorial

8.7. Definições de função

Uma definição de função define um objeto de função definido pelo usuário (veja a seção A hierarquia de tipos padrão):

```
funcdef:                    [decorators] "def" funcname [type par
                            ["->" expression] ":" suite
decorators:                 decorator+
decorator:                  "@" assignment_expression NEWLINE
parameter_list:             defparameter ("," defparameter)* ","
                              | parameter_list_no_posonly
parameter_list_no_posonly:  defparameter ("," defparameter)* [","
                            | parameter_list_starargs
parameter_list_starargs:    "*" star_parameter ("," defparameter)
                            | "*" ("," defparameter)+ ["," [param
                            | parameter_star_kwargs
parameter_star_kwargs:      "**" parameter [","]
parameter:                  identifier [":" expression]
star_parameter:             identifier [":" ["*"] expression]
defparameter:               parameter ["=" expression]
funcname:                   identifier
```

Uma definição de função é uma instrução executável. Sua execução vincula o nome da função no espaço de nomes local atual a um objeto função (um invólucro em torno do código executável para a função). Este objeto função contém uma referência ao espaço de nomes global atual como o espaço de nomes global a ser usado quando a função é chamada.

A definição da função não executa o corpo da função; ela é executada somente quando a função é chamada. [[4]]

Uma definição de função pode ser encapsulada por uma ou mais expressões decoradoras. Expressões decoradoras são avaliadas quando a função é definida, no escopo que contém a definição da função. O resultado deve ser um chamável, que é invocado com o objeto de função como o único argumento. O valor retornado é vinculado ao nome da função em vez do objeto de função. Vários decoradores são aplicados de forma aninhada. Por exemplo, o código a seguir

```
@f1(arg)
@f2
```

```
def func(): pass
```

é aproximadamente equivalente a

```
def func(): pass
func = f1(arg)(f2(func))
```

exceto que a função original não está temporariamente vinculada ao nome `func`.

Alterado na versão 3.9: Funções podem ser decoradas com qualquer **`assignment expression`** válida. Anteriormente, a gramática era muito mais restritiva; veja **PEP 614** [https://peps.python.org/pep-0614/] para detalhes.

Uma lista de parâmetros de tipo pode ser dada entre colchetes entre o nome da função e o parêntese de abertura para sua lista de parâmetros. Isso indica aos verificadores de tipo estático que a função é genérica. Em tempo de execução, os parâmetros de tipo podem ser recuperados do atributo **`__type_params__`** da função. Veja Funções genéricas para mais.

Alterado na versão 3.12: Listas de parâmetros de tipo são novas no Python 3.12.

Quando um ou mais parâmetros têm a forma *parameter* = *expression*, diz-se que a função tem "valores de parâmetro padrão". Para um parâmetro com um valor padrão, o argumento correspondente pode ser omitido de uma chamada, em cujo caso o valor padrão do parâmetro é substituído. Se um parâmetro tiver um valor padrão, todos os parâmetros seguintes até "`*`" também devem ter um valor padrão — esta é uma restrição sintática que não é expressa pela gramática.

Os valores dos parâmetros padrão são avaliados da esquerda para a direita quando a definição da função é executada. Isso significa que a expressão é avaliada uma vez, quando a função é definida, e que o mesmo valor "pré-calculado" é usado em cada chamada. Isso é especialmente importante para entender quando um valor de parâmetro padrão é um objeto mutável, como uma lista ou um dicionário: se a função modificar o objeto (por exemplo, anexando um item a uma lista), o valor do parâmetro padrão será efetivamente modificado. Isso geralmente não é o que se pretendia.

Uma maneira de contornar isso é usar `None` como o padrão e testá-lo explicitamente no corpo da função, por exemplo:

```
def whats_on_the_telly(penguin=None):
    if penguin is None:
        penguin = []
    penguin.append("property of the zoo")
    return penguin
```

A semântica de chamada de função é descrita em mais detalhes na seção Chamadas. Uma chamada de função sempre atribui valores a todos os parâmetros mencionados na lista de parâmetros, seja de argumentos posicionais, de argumentos nomeados ou de valores padrão. Se o formato "`*identifier`" estiver presente, ele será inicializado para uma tupla que recebe quaisquer parâmetros posicionais excedentes, padronizando para a tupla vazia. Se o formato "`**identifier`" estiver presente, ele será inicializado para um novo mapeamento ordenado que recebe quaisquer argumentos nomeados excedentes, padronizando para um novo mapeamento vazio do mesmo tipo. Parâmetros após "`*`" ou "`*identifier`" são parâmetros somente-nomeados e podem ser passados somente por argumentos nomeados. Parâmetros antes de "`/`" são parâmetros somente-posicionais e podem ser passados somente por argumentos posicionais.

Alterado na versão 3.8: A sintaxe do parâmetro de função `/` pode ser usada para indicar parâmetros somente-posicionais. Veja a **PEP 570** [https://peps.python.org/pep-0570/] para detalhes.

Os parâmetros podem ter uma anotação do formato "`: expressão`" após o nome do parâmetro. Qualquer parâmetro pode ter uma anotação, mesmo aqueles do formato `*identificador` ou `**identificador`. (Como um caso especial, parâmetros do formato `*identificador` podem ter uma anotação "`: *expressão`".) As funções podem ter uma anotação "return" do formato "`-> expressão`" após a lista de parâmetros. Essas anotações podem ser qualquer expressão Python válida. A presença de anotações não altera a semântica de uma função. Veja Anotações para mais informações sobre anotações.

Alterado na versão 3.11: Parâmetros do formato "`*identificador`" podem ter uma anotação "`: *expressão`". Veja **PEP 646** [https://peps.python.org/pep-0646/].

Também é possível criar funções anônimas (funções não vinculadas a um nome), para uso imediato em expressões. Isso usa expressões lambda, descritas na seção Lambdas. Observe que a expressão lambda é meramente uma abreviação para uma definição de função simplificada; uma função definida em uma instrução "**`def`**" pode ser passada adiante ou atribuída a outro nome, assim como uma função definida por uma expressão lambda. O formato "**`def`**" é, na verdade, mais poderoso, pois permite a execução de várias instruções e anotações.

Nota do programador: Funções são objetos de primeira classe. Uma instrução "`def`" executada dentro de uma definição de função define uma função local que pode ser retornada ou passada adiante. Variáveis livres usadas na função aninhada podem acessar as variáveis locais da função que contém o "def". Veja a seção Nomeação e ligação para detalhes.

Ver também

PEP 3107 **[https://peps.python.org/pep-3107/]** **- Anotações de função**
A especificação original para anotações de funções.

PEP 484 **[https://peps.python.org/pep-0484/]** **- Dicas de tipo**
Definição de um significado padrão para anotações: dicas de tipo.

PEP 526 **[https://peps.python.org/pep-0526/]** **- Sintaxe para anotações de variáveis**
Capacidade de fornecer dica de tipo para declarações de variáveis, incluindo variáveis de classe e variáveis de instância.

PEP 563 **[https://peps.python.org/pep-0563/]** **- Avaliação postergada de anotações**
Suporte para referências futuras dentro de anotações, preservando anotações em um formato de string em tempo de execução em vez de avaliação antecipada.

PEP 318 [https://peps.python.org/pep-0318/] - **Decoradores para funções e métodos**

Decoradores de função e método foram introduzidos. Decoradores de classe foram introduzidos na **PEP 3129** [https://peps.python.org/pep-3129/].

8.8. Definições de classe

Uma definição de classe define um objeto classe (veja a seção A hierarquia de tipos padrão):

```
classdef:    [decorators] "class" classname [type_params] [inher
inheritance: "(" [argument_list] ")"
classname:   identifier
```

Uma definição de classe é uma instrução executável. A lista de herança geralmente fornece uma lista de classes base (veja Metaclasses para usos mais avançados), então cada item na lista deve ser executada como um objeto classe que permite extensão via subclasse. Classes sem uma lista de herança herdam, por padrão, da classe base `object`; portanto,

```
class Foo:
    pass
```

equivale a

```
class Foo(object):
    pass
```

Pode haver uma ou mais classes base; veja Herança múltipla abaixo para mais informações.

O conjunto da classe é então executado em um novo quadro de execução (veja Nomeação e ligação), usando um espaço de nomes local recém-criado e o espaço de nomes global original. (Normalmente, o conjunto contém principalmente definições de função.) Quando o conjunto da classe termina a execução, seu quadro de execução é descartado, mas seu espaço de nomes

local é salvo. [[5]] Um objeto classe é então criado usando a lista de herança para as classes base e o espaço de nomes local salvo para o dicionário de atributos. O nome da classe é vinculado a este objeto classe no espaço de nomes local original.

A ordem em que os atributos são definidos no corpo da classe é preservada no `__dict__` da nova classe. Observe que isso é confiável somente logo após a classe ser criada e somente para classes que foram definidas usando a sintaxe de definição.

A criação de classes pode ser bastante personalizada usando metaclasses.

As classes também podem ser decoradas: assim como na decoração de funções,

```
@f1(arg)
@f2
class Foo: pass
```

é aproximadamente equivalente a

```
class Foo: pass
Foo = f1(arg)(f2(Foo))
```

As regras de execução para as expressões de decorador são as mesmas que para decoradores de função. O resultado é então vinculado ao nome da classe.

Alterado na versão 3.9: Classes podem ser decoradas com qualquer `assignment expression` válida. Anteriormente, a gramática era muito mais restritiva; veja **PEP 614** [https://peps.python.org/pep-0614/] para detalhes.

Uma lista de parâmetros de tipo pode ser dada entre colchetes imediatamente após o nome da classe. Isso indica aos verificadores de tipo estático que a classe é genérica. Em tempo de execução, os parâmetros de tipo podem ser recuperados do atributo `__type_params__` da classe. Veja Classes genéricas para mais.

Alterado na versão 3.12: Listas de parâmetros de tipo são novas no Python 3.12.

Nota do programador: Variáveis definidas na definição de classe são atributos de classe; elas são compartilhadas por instâncias. Atributos de instância podem ser definidos em um método com `self.nome = valor`. Atributos de classe e instância são acessíveis por meio da notação "`self.nome`", e um atributo de instância oculta um atributo de classe com o mesmo nome quando acessado dessa forma. Atributos de classe podem ser usados como padrões para atributos de instância, mas usar valores mutáveis pode levar a resultados inesperados. Descritores podem ser usados para criar variáveis de instância com diferentes detalhes de implementação.

Ver também

PEP 3115 **[https://peps.python.org/pep-3115/]** - **Metaclasses no Python 3000**
A proposta que alterou a declaração de metaclasses para a sintaxe atual e a semântica de como as classes com metaclasses são construídas.

PEP 3129 **[https://peps.python.org/pep-3129/]** - **Decoradores de classe**
A proposta que adicionou decoradores de classe. Decoradores de função e método foram introduzidos na **PEP 318** [https://peps.python.org/pep-0318/].

8.8.1. Herança múltipla

Classes Python podem ter várias classes base, uma técnica conhecida como *herança múltipla*. As classes base são especificadas na definição de classe, listadas entre parênteses após o nome da classe, separadas por vírgulas. Por exemplo, a seguinte definição de classe:

```
>>> class A: pass
>>> class B: pass
>>> class C(A, B): pass
```

define a classe `C` que herda das classes `A` e `B`.

A ordem de resolução de métodos (MRO) é a ordem em que as classes base são pesquisadas ao procurar um atributo em uma classe. Veja A Ordem de Resolução de Métodos do Python 2.3 para uma descrição de como o Python determina a MRO de uma classe.

A herança múltipla nem sempre é permitida. Tentar definir uma classe com herança múltipla levantará um erro se uma das bases não permitir a criação de subclasses, se não for possível criar uma MRO consistente, se nenhuma metaclasse válida puder ser determinada ou se houver um conflito de layout de instância. Discutiremos cada um desses casos a seguir.

Primeiramente, todas as classes base devem permitir subclasse. Embora a maioria das classes permitam subclasse, algumas classes embutidas não permitem, como, por exemplo, **`bool`**:

```
>>> class SubBool(bool):  # TypeError
...    pass
Traceback (most recent call last):
   ...
TypeError: type 'bool' is not an acceptable base type
```

Na MRO resolvida de uma classe, as bases da classe aparecem na ordem em que foram especificadas na lista de bases da classe. Além disso, a MRO sempre lista uma classe filha antes de qualquer uma de suas bases. Uma definição de classe falhará se for impossível resolver, a partir da lista de bases fornecida, uma MRO consistente que satisfaça essas regras:

```
>>> class Base: pass
>>> class Child(Base): pass
>>> class Grandchild(Base, Child): pass  # TypeError
Traceback (most recent call last):
   ...
TypeError: Cannot create a consistent method resolution order (M
```

Na MRO de `Grandchild`, `Base` deve aparecer antes de `Child` porque é a primeira classe base na lista, mas também deve aparecer depois de `Child` porque é pai de `Child`. Isso é uma contradição; portanto, a classe não pode ser definida.

Se algumas das bases tiverem uma metaclasse personalizada, a metaclasse da classe resultante é escolhida entre as metaclasses das bases e a metaclasse explicitamente especificada da classe filha. Ela deve ser uma metaclasse que seja subclasse de todas as outras metaclasses candidatas. Se não existir tal metaclasse entre as candidatas, a classe não pode ser criada, conforme explicado em Determinando a metaclasse apropriada.

Por fim, os layouts de instância das bases devem ser compatíveis entre si. Isso significa que deve ser possível calcular uma *base sólida* para a classe. Exatamente quais classes são consideradas bases sólidas depende da implementação do Python.

No CPython, uma classe é uma base sólida se a definição de `__slots__` não for vazia. Muitas, mas não todas, das classes definidas em C também são bases sólidas, incluindo a maioria das embutidas (como `int` ou `BaseException`), mas excluindo a maioria das classes `Exception` concretas. Em geral, uma classe C é uma base sólida se sua estrutura subjacente tiver um tamanho diferente do de sua classe base.

Toda classe tem uma base sólida. `object`, a classe base de todas as classes, é a sua própria base sólida. Quando há uma única base, a classe filha é a sua própria base sólida se ela mesma for uma base sólida. Se não for, ela herda a base sólida da classe base. Se houver múltiplas bases, primeiro encontramos a base sólida de cada classe base, o que gera uma lista de bases sólidas candidatas. Se exatamente uma dessas candidatas for subclasse de todas as outras, ela será a base sólida. Caso contrário, a criação da classe falha.

Exemplo:

```
>>> class Solid1:
... __slots__ = ("solid1",)
>>>
>>> class Solid2:
... __slots__ = ("solid2",)
>>>
>>> class SolidChild(Solid1):
... __slots__ = ("solid_child",)
>>>
```

```
>>> class C1: # a base sólida é `object`
... pass
>>>
>>> # OK: as bases sólidas são `Solid1` e `object`, e `Solid1` é
>>> class C2(Solid1, C1): # a base sólida é `Solid1`
... pass
>>>
>>> # OK: as bases sólidas são `SolidChild` e `Solid1`, e `Solic
>>> class C3(SolidChild, Solid1): # a base sólida é `SolidChild`
... pass
>>>
>>> # Erro: as bases sólidas são `Solid1` e `Solid2`, mas nenhum
>>> class C4(Solid1, Solid2): # erro: não há uma única base sóli
... pass
Traceback (most recent call last):
...
TypeError: multiple bases have instance lay-out conflict
```

8.9. Corrotinas

Adicionado na versão 3.5.

8.9.1. Definição de função de corrotina

```
async_funcdef: [decorators] "async" "def" funcname "(" [paramete
               ["->" expression] ":" suite
```

A execução de corrotinas do Python pode ser suspensa e retomada em muitos pontos (consulte coroutine). As expressões `await`, `async for` e `async with` só podem ser usadas no corpo de uma função de corrotina.

Funções definidas com a sintaxe `async def` são sempre funções de corrotina, mesmo que não contenham palavras reservadas `await` ou `async`.

Ocorre uma `SyntaxError` se usada uma expressão `yield from` dentro do corpo de uma função de corrotina.

Um exemplo de uma função de corrotina:

```
async def func(param1, param2):
    faz_algo()
    await alguma_corrotina()
```

Alterado na versão 3.7: `await` e `async` agora são palavras reservadas; anteriormente, elas só eram tratadas como tal dentro do corpo de uma função de corrotina.

8.9.2. A instrução `async for`

`async_for_stmt`: `"async"` **`for stmt`**

Um iterável assíncrono fornece um método `__aiter__` que retorna diretamente um iterador assíncrono, que pode chamar código assíncrono em seu método `__anext__`.

A instrução `async for` permite iteração conveniente sobre iteráveis assíncronos.

O seguinte código:

```
async for TARGET in ITER:
    SUITE
else:
    SUITE2
```

É semanticamente equivalente a:

```
iter = (ITER).__aiter__()
running = True

while running:
    try:
        TARGET = await iter.__anext__()
    except StopAsyncIteration:
        running = False
    else:
```

```
        SUITE
else:
    SUITE2
```

exceto que a pesquisa de método especial implícita é usada para **`__aiter__()`** e **`__anext__()`**.

Ocorre uma **`SyntaxError`** se usada uma instrução `async for` fora do corpo de uma função de corrotina.

8.9.3. A instrução `async with`

`async_with_stmt`: `"async"` **`with_stmt`**

Um gerenciador de contexto assíncrono é um gerenciador de contexto que é capaz de suspender a execução em seus métodos *enter* e *exit*.

O seguinte código:

```
async with EXPRESSÃO as ALVO:
    COMANDOS
```

é semanticamente equivalente a:

```
manager = (EXPRESSION)
aenter = manager.__aenter__
aexit = manager.__aexit__
value = await aenter()
hit_except = False

try:
    TARGET = value
    SUITE
except:
    hit_except = True
    if not await aexit(*sys.exc_info()):
        raise
finally:
    if not hit_except:
        await aexit(None, None, None)
```

exceto que a pesquisa de método especial é usada de forma implícita para `__aenter__()` e `__aexit__()`.

Ocorre uma `SyntaxError` se usada uma instrução `async with` fora do corpo de uma função de corrotina.

Ver também

PEP 492 [https://peps.python.org/pep-0492/] **- Corrotina com sintaxe de async e wait**

A proposta que tornou as corrotinas um conceito autônomo em Python e adicionou sintaxe de suporte.

8.10. Listas de parâmetros de tipo

Adicionado na versão 3.12.

Alterado na versão 3.13: Foi adicionado suporte para valores padrão (veja **PEP 696** [https://peps.python.org/pep-0696/]).

```
type_params:  "[" type_param ("," type_param)* "]"
type_param:   typevar | typevartuple | paramspec
typevar:      identifier (":" expression)? ("=" expression)?
typevartuple: "*" identifier ("=" expression)?
paramspec:    "**" identifier ("=" expression)?
```

Funções (incluindo corrotinas), classes e apelidos de tipo podem conter uma lista de parâmetros de tipo:

```
def max[T](args: list[T]) -> T:
    ...

async def amax[T](args: list[T]) -> T:
    ...

class Bag[T]:
    def __iter__(self) -> Iterator[T]:
```

```
        ...

    def add(self, arg: T) -> None:
        ...

type ListOrSet[T] = list[T] | set[T]
```

Semanticamente, isso indica que a função, classe ou apelido de tipo é genérico sobre uma variável de tipo. Essas informações são usadas principalmente por verificadores de tipo estático e, em tempo de execução, objetos genéricos se comportam muito como suas contrapartes não genéricas.

Parâmetros de tipo são declarados entre colchetes (`[]`) imediatamente após o nome da função, classe ou apelido de tipo. Os parâmetros de tipo são acessíveis dentro do escopo do objeto genérico, mas não em outro lugar. Assim, após uma declaração `def func[T]():  pass`, o nome `T` não está disponível no escopo do módulo. Abaixo, a semântica de objetos genéricos é descrita com mais precisão. O escopo de parâmetros de tipo é modelado com uma função especial (tecnicamente, um escopo de anotação) que encapsula a criação do objeto genérico.

Funções genéricas, classes e apelidos de tipo têm um atributo `__type_params__` listando seus parâmetros de tipo.

Existem três espécies de parâmetros de tipo:

- `typing.TypeVar`, introduzido por um nome simples (por exemplo, `T`). Semanticamente, isso representa um único tipo para um verificador de tipos.
- `typing.TypeVarTuple`, introduzido por um nome prefixado com um único asterisco (por exemplo, `*Ts`). Semanticamente, isso representa uma tupla de qualquer número de tipos.
- `typing.ParamSpec`, introduzido por um nome prefixado com dois asteriscos (por exemplo, `**P`). Semanticamente, isso representa os parâmetros de um chamável.

As declarações de `typing.TypeVar` podem definir delimitações (*bounds*) e restrições (*constraints*) com dois pontos (`:`) seguidos por uma expressão.

Uma única expressão após os dois pontos indica uma delimitação (por exemplo, `T: int`). Semanticamente, isso significa que o **`typing.TypeVar`** pode representar apenas tipos que são um subtipo desse limite. Uma tupla de expressões entre parênteses após os dois pontos indica um conjunto de restrições (por exemplo, `T: (str, bytes)`). Cada membro da tupla deve ser um tipo (novamente, isso não é imposto em tempo de execução). Variáveis de tipo restrito podem assumir apenas um dos tipos na lista de restrições.

Para **`typing.TypeVar`**s declarados usando a sintaxe de lista de parâmetros de tipo, a delimitação e as restrições não são avaliados quando o objeto genérico é criado, mas somente quando o valor é acessado explicitamente por meio dos atributos `__bound__` e `__constraints__`. Para fazer isso, as delimitações ou restrições são avaliados em um escopo de anotação separado.

`typing.TypeVarTuple`s e **`typing.ParamSpec`**s não pode ter delimitações e restrições.

Todas as três espécies de parâmetros de tipo também podem ter um *valor padrão*, que é usado quando o parâmetro de tipo não é fornecido explicitamente. Isso é adicionado anexando um único sinal de igual (=) seguido por uma expressão. Como os limites e restrições de tipos variáveis, o valor padrão não é avaliado quando o objeto é criado, mas apenas quando o atributo `__default__` do parâmetro de tipo é acessado. Para esse fim, o valor padrão é avaliado em um escopo de anotação separado. Se nenhum valor padrão for especificado para um parâmetro de tipo, o atributo `__default__` é definido como o objeto sinalizador especial **`typing.NoDefault`**.

O exemplo a seguir indica o conjunto completo de declarações de parâmetros de tipo permitidas:

```
def overly_generic[
   SimpleTypeVar,
   TypeVarWithDefault = int,
   TypeVarWithBound: int,
   TypeVarWithConstraints: (str, bytes),
   *SimpleTypeVarTuple = (int, float),
```

```
   **SimpleParamSpec = (str, bytearray),
](
   a: SimpleTypeVar,
   b: TypeVarWithDefault,
   c: TypeVarWithBound,
   d: Callable[SimpleParamSpec, TypeVarWithConstraints],
   *e: SimpleTypeVarTuple,
): ...
```

8.10.1. Funções genéricas

Funções genéricas são declaradas assim:

```
def func[T](arg: T): ...
```

Essa sintaxe é equivalente a:

```
annotation-def TYPE_PARAMS_OF_func():
    T = typing.TypeVar("T")
    def func(arg: T): ...
    func.__type_params__ = (T,)
    return func
func = TYPE_PARAMS_OF_func()
```

Aqui, `annotation-def` indica um escopo de anotação, que não é vinculado de verdade a nenhum nome em tempo de execução. (Uma outra liberdade foi tomada na tradução: essa sintaxe não faz ocorrer um acesso de atributo no módulo `typing`; ao invés disso é criada uma instância de `typing.TypeVar` diretamente.)

As anotações das funções genéricas são lidas dentro do escopo de anotação usado para declarar os parâmetros de tipo, mas os valores padrão dos argumentos e decoradores da função não são.

Este exemplo ilustra as regras de escopo para estes casos, bem como para outras espécies de parâmetros de tipo:

```
@decorator
def func[T: int, *Ts, **P](*args: *Ts, arg: Callable[P, T] = som
    ...
```

Tirando a avaliação preguiçosa da delimitação da `TypeVar`, isso é equivalente a:

```
DEFAULT_OF_arg = some_default

annotation-def TYPE_PARAMS_OF_func():

    annotation-def BOUND_OF_T():
        return int
    # In reality, BOUND_OF_T() is evaluated only on demand.
    T = typing.TypeVar("T", bound=BOUND_OF_T())

    Ts = typing.TypeVarTuple("Ts")
    P = typing.ParamSpec("P")

    def func(*args: *Ts, arg: Callable[P, T] = DEFAULT_OF_arg):
        ...

    func.__type_params__ = (T, Ts, P)
    return func
func = decorator(TYPE_PARAMS_OF_func())
```

Os nomes com maiúsculas como `DEFAULT_OF_arg` não são realmente vinculados em tempo de execução.

8.10.2. Classes genéricas

Classes genéricas são declaradas assim:

```
class Bag[T]: ...
```

Essa sintaxe é equivalente a:

```
annotation-def TYPE_PARAMS_OF_Bag():
    T = typing.TypeVar("T")
    class Bag(typing.Generic[T]):
        __type_params__ = (T,)
        ...
    return Bag
Bag = TYPE_PARAMS_OF_Bag()
```

Aqui novamente `annotation-def` (não é uma palavra reservada real) indica um escopo de anotação, e o nome `TYPE_PARAMS_OF_Bag` não é realmente vinculado em tempo de execução.

Classes genéricas herdam implicitamente de **`typing.Generic`**. As classes base e os argumentos nomeados de classes genéricas são avaliados dentro do escopo de tipo para os parâmetros de tipo, e os decoradores são avaliados fora desse escopo. Isso é ilustrado por este exemplo:

```
@decorator
class Bag(Base[T], arg=T): ...
```

Isso equivale a:

```
annotation-def TYPE_PARAMS_OF_Bag():
    T = typing.TypeVar("T")
    class Bag(Base[T], typing.Generic[T], arg=T):
        __type_params__ = (T,)
        ...
    return Bag
Bag = decorator(TYPE_PARAMS_OF_Bag())
```

8.10.3. Apelidos de tipo genérico

A instrução **`type`** também pode ser usada para criar um apelido de tipo genérico:

```
type ListOrSet[T] = list[T] | set[T]
```

Exceto pela avaliação preguiçosa do valor, isso é equivalente a:

```
annotation-def TYPE_PARAMS_OF_ListOrSet():
    T = typing.TypeVar("T")

    annotation-def VALUE_OF_ListOrSet():
        return list[T] | set[T]
    # Na realidade, o valor é avaliado preguiçosamente
    return typing.TypeAliasType("ListOrSet", VALUE_OF_ListOrSet(
ListOrSet = TYPE_PARAMS_OF_ListOrSet()
```

Aqui, `annotation-def` (não é uma palavra reservada real) indica um escopo de anotação. Os nomes em maiúsculas como `TYPE_PARAMS_OF_ListOrSet` não são realmente vinculados em tempo de execução.

8.11. Anotações

Alterado na versão 3.14: Anotações são agora avaliados preguiçosamente por padrão.

Variáveis e parâmetros de função podem conter anotações, criadas adicionando dois pontos após o nome, seguidos de uma expressão:

```
x: annotation = 1
def f(param: annotation): ...
```

As funções também podem ter uma anotação de retorno seguindo uma seta:

```
def f() -> annotation: ...
```

Anotações são convencionalmente usadas para dicas de tipo, mas isso não é imposto pela linguagem e, em geral, anotações podem conter expressões arbitrárias. A presença de anotações não altera a semântica de tempo de execução do código, exceto se algum mecanismo for usado para introspectar e utilizar as anotações (como **`dataclasses`** ou **`@functools.singledispatch`**).

Por padrão, as anotações são avaliadas preguiçosamente em um escopo de anotações. Isso significa que elas não são avaliadas quando o código que contém a anotação é avaliado. Em vez disso, o interpretador salva informações que podem ser usadas para avaliar a anotação posteriormente, se solicitado. O módulo **`annotationlib`** fornece ferramentas para avaliar anotações.

Se instrução future `from __future__ import annotations` estiver presente, todas as anotações serão armazenadas como strings:

```
>>> from __future__ import annotations
>>> def f(param: annotation): ...
```

```
>>> f.__annotations__
{'param': 'annotation'}
```

Esta instrução future será descontinuada e removida em uma versão futura do Python, mas não antes do Python 3.13 chegar ao fim de sua vida útil (consulte **PEP 749** [https://peps.python.org/pep-0749/]). Quando usada, ferramentas de introspecção como `annotationlib.get_annotations()` e `typing.get_type_hints()` têm menor probabilidade de resolver anotações em tempo de execução.

Notas de rodapé

[1]

A exceção é propagada para a pilha de invocação, a menos que haja uma cláusula `finally` que por acaso levante outra exceção. Essa nova exceção faz com que a antiga seja perdida.

[2]

Na correspondência de padrões, uma sequência é definida como uma das seguintes:

- uma classe que herda de `collections.abc.Sequence`
- uma classe Python que foi registrada como `collections.abc.Sequence`
- uma classe embutida que tem seu bit `Py_TPFLAGS_SEQUENCE` (CPython) definido
- uma classe que herda de qualquer uma das anteriores

As seguintes classes de biblioteca padrão são sequências:

- `array.array`
- `collections.deque`
- `list`
- `memoryview`
- `range`
- `tuple`

Nota

Valores de assunto do tipo `str`, `bytes` e `bytearray` não correspondem aos padrões de sequência.

[3]

Na correspondência de padrões, um mapeamento é definido como uma das seguintes:

- uma classe que herda de `collections.abc.Mapping`
- uma classe Python que foi registrada como `collections.abc.Mapping`
- uma classe embutida que tem seu bit `Py_TPFLAGS_MAPPING` (CPython) definido
- uma classe que herda de qualquer uma das anteriores

As classes de biblioteca padrão `dict` e `types.MappingProxyType` são mapeamentos.

[4]

Um literal de string que aparece como a primeira instrução no corpo da função é transformado no atributo `__doc__` da função e, portanto, no docstring da função.

[5]

Um literal de string que aparece como a primeira instrução no corpo da classe é transformado no item `__doc__` do espaço de nomes e, portanto, no docstring da classe.

9. Componentes de Alto Nível

O interpretador Python pode receber suas entradas de uma quantidade de fontes: de um script passado a ele como entrada padrão ou como um argumento do programa, digitado interativamente, de um arquivo fonte de um módulo, etc. Este capítulo mostra a sintaxe usada nesses casos.

9.1. Programas Python completos

Ainda que uma especificação de linguagem não precise prescrever como o interpretador da linguagem é invocado, é útil ter uma noção de um programa Python completo. Um programa Python completo é executado em um ambiente minimamente inicializado: todos os módulos embutidos e padrões estão disponíveis, mas nenhum foi inicializado, exceto por `sys` (serviços de sistema diversos), `builtins` (funções embutidas, exceções e `None`) e `__main__`. O último é usado para fornecer o espaço de nomes global e local para execução de um programa completo.

A sintaxe para um programa Python completo é esta para uma entrada de arquivo, descrita na próxima seção.

O interpretador também pode ser invocado no modo interativo; neste caso, ele não lê e executa um programa completo, mas lê e executa uma instrução (possivelmente composta) por vez. O ambiente inicial é idêntico àquele de um programa completo; cada instrução é executada no espaço de nomes de `__main__`.

Um programa completo pode ser passado ao interpretador de três formas: com a opção de linha de comando `-c` *string*, como um arquivo passado como o primeiro argumento da linha de comando, ou como uma entrada padrão. Se o arquivo ou a entrada padrão é um dispositivo tty, o interpretador entra em modo interativo; caso contrário, ele executa o arquivo como um programa completo.

9.2. Entrada de arquivo

Toda entrada lida de arquivos não-interativos têm a mesma forma:

```
file_input: (NEWLINE | statement)* ENDMARKER
```

Essa sintaxe é usada nas seguintes situações:

- quando analisando um programa Python completo (a partir de um arquivo ou de uma string);
- quando analisando um módulo;
- quando analisando uma string passada à função `exec()`;

9.3. Entrada interativa

A entrada em modo interativo é analisada usando a seguinte gramática:

```
interactive_input: [stmt_list] NEWLINE | compound_stmt NEWLINE |
```

Note que uma instrução composta (de alto-nível) deve ser seguida por uma linha em branco no modo interativo; isso é necessário para ajudar o analisador sintático a detectar o fim da entrada.

9.4. Entrada de expressão

A função `eval()` é usada para uma entrada de expressão. Ela ignora espaços à esquerda. O argumento em string para **`eval()`** deve ter a seguinte forma:

```
eval_input: expression_list NEWLINE* ENDMARKER
```

10. Especificação completa da gramática

Esta é a gramática completa do Python, derivada diretamente da gramática usada para gerar o analisador sintático de CPython (consulte Grammar/python.gram [https://github.com/python/cpython/tree/3.15/Grammar/python.gram]). A versão aqui omite detalhes relacionados à geração de código e recuperação de erros.

A notação usada aqui é a mesma dos documentos anteriores e é descrita na seção notação, exceto por uma complicação extra:

- ~ (“cut”): compromete-se com a alternativa atual; falha na regra, se a alternativa falha em analisar.

  O Python utiliza cortes principalmente para otimizações ou para melhorar as mensagens de erro. Na listagem abaixo, eles frequentemente parecem inúteis.

  Atualmente, cortes não aparecem dentro de parênteses, colchetes, *lookaheads* e estruturas semelhantes. Seu comportamento nesses contextos é deliberadamente deixado indefinido.

```
# PEG grammar for Python



# ========================= START OF THE GRAMMAR ===============

# General grammatical elements and rules:
#
# * Strings with double quotes (") denote SOFT KEYWORDS
# * Strings with single quotes (') denote KEYWORDS
# * Upper case names (NAME) denote tokens in the Grammar/Tokens
# * Rule names starting with "invalid_" are used for specializec
#     - These rules are NOT used in the first pass of the parser
```

```
#     - Only if the first pass fails to parse, a second pass inc
#       rules will be executed.
#     - If the parser fails in the second phase with a generic s
#       location of the generic failure of the first pass will b
#       reporting incorrect locations due to the invalid rules).
#     - The order of the alternatives involving invalid rules ma
#       (like any rule in PEG).
#
# Grammar Syntax (see PEP 617 for more information):
#
# rule_name: expression
#   Optionally, a type can be included right after the rule name
#   specifies the return type of the C or Python function corres
#   rule:
# rule_name[return_type]: expression
#   If the return type is omitted, then a void * is returned in
#   Python.
# e1 e2
#   Match e1, then match e2.
# e1 | e2
#   Match e1 or e2.
#   The first alternative can also appear on the line after the
#   formatting purposes. In that case, a | must be used before t
#   alternative, like so:
#       rule_name[return_type]:
#            | first_alt
#            | second_alt
# ( e )
#   Match e (allows also to use other operators in the group lik
# [ e ] or e?
#   Optionally match e.
# e*
#   Match zero or more occurrences of e.
# e+
#   Match one or more occurrences of e.
# s.e+
#   Match one or more occurrences of e, separated by s. The gene
#   does not include the separator. This is otherwise identical
# &e
#   Succeed if e can be parsed, without consuming any input.
# !e
#   Fail if e can be parsed, without consuming any input.
# ~
```

```
#   Commit to the current alternative, even if it fails to parse
# &&e
#   Eager parse e. The parser will not backtrack and will immedi
#   fail with SyntaxError if e cannot be parsed.
#

# STARTING RULES
# ==============

file: [statements] ENDMARKER
interactive: statement_newline
eval: expressions NEWLINE* ENDMARKER
func_type: '(' [type_expressions] ')' '->' expression NEWLINE* E

# GENERAL STATEMENTS
# ==================

statements: statement+

statement:
    | compound_stmt
    | simple_stmts

single_compound_stmt:
    | compound_stmt

statement_newline:
    | single_compound_stmt NEWLINE
    | simple_stmts
    | NEWLINE
    | ENDMARKER

simple_stmts:
    | simple_stmt !';' NEWLINE  # Not needed, there for speedup
    | ';'.simple_stmt+ [';'] NEWLINE

# NOTE: assignment MUST precede expression, else parsing a simpl
# will throw a SyntaxError.
simple_stmt:
    | assignment
    | type_alias
    | import_stmt
    | star_expressions
```

```
    | return_stmt
    | raise_stmt
    | pass_stmt
    | del_stmt
    | yield_stmt
    | assert_stmt
    | break_stmt
    | continue_stmt
    | global_stmt
    | nonlocal_stmt

compound_stmt:
    | function_def
    | if_stmt
    | class_def
    | with_stmt
    | for_stmt
    | try_stmt
    | while_stmt
    | match_stmt

# SIMPLE STATEMENTS
# =================

# NOTE: annotated_rhs may start with 'yield'; yield_expr must st
assignment:
    | NAME ':' expression ['=' annotated_rhs ]
    | ('(' single_target ')'
         | single_subscript_attribute_target) ':' expression ['=
    | (star_targets '=' )+ annotated_rhs !'=' [TYPE_COMMENT]
    | single_target augassign ~ annotated_rhs

annotated_rhs: yield_expr | star_expressions

augassign:
    | '+='
    | '-='
    | '*='
    | '@='
    | '/='
    | '%='
    | '&='
    | '|='
```

```
    | '^='
    | '<<='
    | '>>='
    | '**='
    | '//='

return_stmt:
    | 'return' [star_expressions]

raise_stmt:
    | 'raise' expression 'from' expression
    | 'raise' expression
    | 'raise'

pass_stmt:
    | 'pass'

break_stmt:
    | 'break'

continue_stmt:
    | 'continue'

global_stmt: 'global' ','.NAME+

nonlocal_stmt: 'nonlocal' ','.NAME+

del_stmt:
    | 'del' del_targets &(';' | NEWLINE)

yield_stmt: yield_expr

assert_stmt:
    | 'assert' expression [',' expression ]

import_stmt:
    | import_name
    | import_from

# Import statements
# -----------------

import_name:
```

```
    | "lazy"? 'import' dotted_as_names

# note below: the ('.' | '...') is necessary because '...' is to
import_from:
    | "lazy"? 'from' ('.' | '...')* dotted_name 'import' import_
    | "lazy"? 'from' ('.' | '...')+ 'import' import_from_targets
import_from_targets:
    | '(' import_from_as_names [','] ')'
    | import_from_as_names !','
    | '*'
import_from_as_names:
    | ','.import_from_as_name+
import_from_as_name:
    | NAME ['as' NAME ]

dotted_as_names:
    | ','.dotted_as_name+
dotted_as_name:
    | dotted_name ['as' NAME ]

dotted_name:
    | dotted_name '.' NAME
    | NAME

# COMPOUND STATEMENTS
# ===================

# Common elements
# ---------------

block:
    | NEWLINE INDENT statements DEDENT
    | simple_stmts

decorators: ('@' named_expression NEWLINE )+

# Class definitions
# -----------------

class_def:
    | decorators class_def_raw
    | class_def_raw
```

```
class_def_raw:
    | 'class' NAME [type_params] ['(' [arguments] ')' ] ':' bloc

# Function definitions
# --------------------

function_def:
    | decorators function_def_raw
    | function_def_raw

function_def_raw:
    | 'def' NAME [type_params] '(' [params] ')' ['->' expressior
    | 'async' 'def' NAME [type_params] '(' [params] ')' ['->' ex

# Function parameters
# -------------------

params:
    | parameters

parameters:
    | slash_no_default param_no_default* param_with_default* [st
    | slash_with_default param_with_default* [star_etc]
    | param_no_default+ param_with_default* [star_etc]
    | param_with_default+ [star_etc]
    | star_etc

# Some duplication here because we can't write (',' | &')'),
# which is because we don't support empty alternatives (yet).

slash_no_default:
    | param_no_default+ '/' ','
    | param_no_default+ '/' &')'
slash_with_default:
    | param_no_default* param_with_default+ '/' ','
    | param_no_default* param_with_default+ '/' &')'

star_etc:
    | '*' param_no_default param_maybe_default* [kwds]
    | '*' param_no_default_star_annotation param_maybe_default*
    | '*' ',' param_maybe_default+ [kwds]
    | kwds
```

```
kwds:
    | '**' param_no_default

# One parameter.  This *includes* a following comma and type com
#
# There are three styles:
# - No default
# - With default
# - Maybe with default
#
# There are two alternative forms of each, to deal with type com
# - Ends in a comma followed by an optional type comment
# - No comma, optional type comment, must be followed by close p
# The latter form is for a final parameter without trailing comm
#

param_no_default:
    | param ',' TYPE_COMMENT?
    | param TYPE_COMMENT? &')'
param_no_default_star_annotation:
    | param_star_annotation ',' TYPE_COMMENT?
    | param_star_annotation TYPE_COMMENT? &')'
param_with_default:
    | param default ',' TYPE_COMMENT?
    | param default TYPE_COMMENT? &')'
param_maybe_default:
    | param default? ',' TYPE_COMMENT?
    | param default? TYPE_COMMENT? &')'
param: NAME annotation?
param_star_annotation: NAME star_annotation
annotation: ':' expression
star_annotation: ':' star_expression
default: '=' expression  | invalid_default

# If statement
# ------------

if_stmt:
    | 'if' named_expression ':' block elif_stmt
    | 'if' named_expression ':' block [else_block]
elif_stmt:
    | 'elif' named_expression ':' block elif_stmt
    | 'elif' named_expression ':' block [else_block]
```

```
else_block:
    | 'else' ':' block

# While statement
# ---------------

while_stmt:
    | 'while' named_expression ':' block [else_block]

# For statement
# -------------

for_stmt:
    | 'for' star_targets 'in' ~ star_expressions ':' [TYPE_COMME
    | 'async' 'for' star_targets 'in' ~ star_expressions ':' [TY

# With statement
# --------------

with_stmt:
    | 'with' '(' ','.with_item+ ','? ')' ':' [TYPE_COMMENT] bloc
    | 'with' ','.with_item+ ':' [TYPE_COMMENT] block
    | 'async' 'with' '(' ','.with_item+ ','? ')' ':' block
    | 'async' 'with' ','.with_item+ ':' [TYPE_COMMENT] block

with_item:
    | expression 'as' star_target &(',' | ')' | ':')
    | expression

# Try statement
# -------------

try_stmt:
    | 'try' ':' block finally_block
    | 'try' ':' block except_block+ [else_block] [finally_block]
    | 'try' ':' block except_star_block+ [else_block] [finally_b


# Except statement
# ----------------

except_block:
    | 'except' expression ':' block
```

```
    | 'except' expression 'as' NAME ':' block
    | 'except' expressions ':' block
    | 'except' ':' block
except_star_block:
    | 'except' '*' expression ':' block
    | 'except' '*' expression 'as' NAME ':' block
    | 'except' '*' expressions ':' block
finally_block:
    | 'finally' ':' block

# Match statement
# ---------------

match_stmt:
    | "match" subject_expr ':' NEWLINE INDENT case_block+ DEDENT

subject_expr:
    | star_named_expression ',' star_named_expressions?
    | named_expression

case_block:
    | "case" patterns guard? ':' block

guard: 'if' named_expression

patterns:
    | open_sequence_pattern
    | pattern

pattern:
    | as_pattern
    | or_pattern

as_pattern:
    | or_pattern 'as' pattern_capture_target

or_pattern:
    | '|'.closed_pattern+

closed_pattern:
    | literal_pattern
    | capture_pattern
    | wildcard_pattern
```

```
    | value_pattern
    | group_pattern
    | sequence_pattern
    | mapping_pattern
    | class_pattern

# Literal patterns are used for equality and identity constraint
literal_pattern:
    | signed_number !('+' | '-')
    | complex_number
    | strings
    | 'None'
    | 'True'
    | 'False'

# Literal expressions are used to restrict permitted mapping pat
literal_expr:
    | signed_number !('+' | '-')
    | complex_number
    | strings
    | 'None'
    | 'True'
    | 'False'

complex_number:
    | signed_real_number '+' imaginary_number
    | signed_real_number '-' imaginary_number

signed_number:
    | NUMBER
    | '+' NUMBER
    | '-' NUMBER

signed_real_number:
    | real_number
    | '+' real_number
    | '-' real_number

real_number:
    | NUMBER

imaginary_number:
    | NUMBER
```

```
capture_pattern:
    | pattern_capture_target

pattern_capture_target:
    | !"_" NAME !('.' | '(' | '=')

wildcard_pattern:
    | "_"

value_pattern:
    | attr !('.' | '(' | '=')

attr:
    | name_or_attr '.' NAME

name_or_attr:
    | attr
    | NAME

group_pattern:
    | '(' pattern ')'

sequence_pattern:
    | '[' maybe_sequence_pattern? ']'
    | '(' open_sequence_pattern? ')'

open_sequence_pattern:
    | maybe_star_pattern ',' maybe_sequence_pattern?

maybe_sequence_pattern:
    | ','.maybe_star_pattern+ ','?

maybe_star_pattern:
    | star_pattern
    | pattern

star_pattern:
    | '*' pattern_capture_target
    | '*' wildcard_pattern

mapping_pattern:
    | '{' '}'
```

```
    | '{' double_star_pattern ','? '}'
    | '{' items_pattern ',' double_star_pattern ','? '}'
    | '{' items_pattern ','? '}'

items_pattern:
    | ','.key_value_pattern+

key_value_pattern:
    | (literal_expr | attr) ':' pattern

double_star_pattern:
    | '**' pattern_capture_target

class_pattern:
    | name_or_attr '(' ')'
    | name_or_attr '(' positional_patterns ','? ')'
    | name_or_attr '(' keyword_patterns ','? ')'
    | name_or_attr '(' positional_patterns ',' keyword_patterns

positional_patterns:
    | ','.pattern+

keyword_patterns:
    | ','.keyword_pattern+

keyword_pattern:
    | NAME '=' pattern

# Type statement
# ---------------

type_alias:
    | "type" NAME [type_params] '=' expression

# Type parameter declaration
# --------------------------

type_params:
    | '[' type_param_seq ']'

type_param_seq: ','.type_param+ [',']

type_param:
```

```
    | NAME [type_param_bound] [type_param_default]
    | '*' NAME [type_param_starred_default]
    | '**' NAME [type_param_default]

type_param_bound: ':' expression
type_param_default: '=' expression
type_param_starred_default: '=' star_expression

# EXPRESSIONS
# -----------

expressions:
    | expression (',' expression )+ [',']
    | expression ','
    | expression

expression:
    | if_expression
    | disjunction
    | lambdef

if_expression:
    | disjunction 'if' disjunction 'else' expression

yield_expr:
    | 'yield' 'from' expression
    | 'yield' [star_expressions]

star_expressions:
    | star_expression (',' star_expression )+ [',']
    | star_expression ','
    | star_expression

star_expression:
    | '*' bitwise_or
    | expression

star_named_expressions: ','.star_named_expression+ [',']

star_named_expressions_sequence: ','.star_named_expression_seque

star_named_expression:
    | '*' bitwise_or
```

```
    | named_expression

star_named_expression_sequence:
    | star_named_expression

assignment_expression:
    | NAME ':=' ~ expression

named_expression:
    | assignment_expression
    | expression !':='

disjunction:
    | conjunction ('or' conjunction )+
    | conjunction

conjunction:
    | inversion ('and' inversion )+
    | inversion

inversion:
    | 'not' inversion
    | comparison

# Comparison operators
# --------------------

comparison:
    | bitwise_or compare_op_bitwise_or_pair+
    | bitwise_or

compare_op_bitwise_or_pair:
    | eq_bitwise_or
    | noteq_bitwise_or
    | lte_bitwise_or
    | lt_bitwise_or
    | gte_bitwise_or
    | gt_bitwise_or
    | notin_bitwise_or
    | in_bitwise_or
    | isnot_bitwise_or
    | is_bitwise_or
```

```
eq_bitwise_or: '==' bitwise_or
noteq_bitwise_or:
    | ('!=' ) bitwise_or
lte_bitwise_or: '<=' bitwise_or
lt_bitwise_or: '<' bitwise_or
gte_bitwise_or: '>=' bitwise_or
gt_bitwise_or: '>' bitwise_or
notin_bitwise_or: 'not' 'in' bitwise_or
in_bitwise_or: 'in' bitwise_or
isnot_bitwise_or: 'is' 'not' bitwise_or
is_bitwise_or: 'is' bitwise_or

# Bitwise operators
# -----------------

bitwise_or:
    | bitwise_or '|' bitwise_xor
    | bitwise_xor

bitwise_xor:
    | bitwise_xor '^' bitwise_and
    | bitwise_and

bitwise_and:
    | bitwise_and '&' shift_expr
    | shift_expr

shift_expr:
    | shift_expr '<<' sum
    | shift_expr '>>' sum
    | sum

# Arithmetic operators
# --------------------

sum:
    | sum '+' term
    | sum '-' term
    | term

term:
    | term '*' factor
    | term '/' factor
```

```
    | term '//' factor
    | term '%' factor
    | term '@' factor
    | factor

factor:
    | '+' factor
    | '-' factor
    | '~' factor
    | power

power:
    | await_primary '**' factor
    | await_primary

# Primary elements
# ----------------

# Primary elements are things like "obj.something.something", "c

await_primary:
    | 'await' primary
    | primary

primary:
    | primary '.' NAME
    | primary genexp
    | primary '(' [arguments] ')'
    | primary '[' slices ']'
    | atom

slices:
    | slice !','
    | ','.(slice | starred_expression)+ [',']

slice:
    | [expression] ':' [expression] [':' [expression] ]
    | named_expression

atom:
    | NAME
    | 'True'
    | 'False'
```

```
    | 'None'
    | strings
    | NUMBER
    | (genexp | tuple | group)
    | (listcomp | list)
    | (dictcomp | setcomp | dict | set)
    | '...'

group:
    | '(' (yield_expr | named_expression) ')'

# Lambda functions
# ----------------

lambdef:
    | 'lambda' [lambda_params] ':' expression

lambda_params:
    | lambda_parameters

# lambda_parameters etc. duplicates parameters but without annot
# or type comments, and if there's no comma after a parameter, w
# a colon, not a close parenthesis.  (For more, see parameters a
#
lambda_parameters:
    | lambda_slash_no_default lambda_param_no_default* lambda_pa
    | lambda_slash_with_default lambda_param_with_default* [lamb
    | lambda_param_no_default+ lambda_param_with_default* [lambd
    | lambda_param_with_default+ [lambda_star_etc]
    | lambda_star_etc

lambda_slash_no_default:
    | lambda_param_no_default+ '/' ','
    | lambda_param_no_default+ '/' &':'

lambda_slash_with_default:
    | lambda_param_no_default* lambda_param_with_default+ '/' ',
    | lambda_param_no_default* lambda_param_with_default+ '/' &'

lambda_star_etc:
    | '*' lambda_param_no_default lambda_param_maybe_default* [l
    | '*' ',' lambda_param_maybe_default+ [lambda_kwds]
    | lambda_kwds
```

```
lambda_kwds:
    | '**' lambda_param_no_default

lambda_param_no_default:
    | lambda_param ','
    | lambda_param &':'
lambda_param_with_default:
    | lambda_param default ','
    | lambda_param default &':'
lambda_param_maybe_default:
    | lambda_param default? ','
    | lambda_param default? &':'
lambda_param: NAME

# LITERALS
# ========

fstring_middle:
    | fstring_replacement_field
    | FSTRING_MIDDLE
fstring_replacement_field:
    | '{' annotated_rhs '='? [fstring_conversion] [fstring_full_
fstring_conversion:
    | "!" NAME
fstring_full_format_spec:
    | ':' fstring_format_spec*
fstring_format_spec:
    | FSTRING_MIDDLE
    | fstring_replacement_field
fstring:
    | FSTRING_START fstring_middle* FSTRING_END

tstring_format_spec_replacement_field:
    | '{' annotated_rhs '='? [fstring_conversion] [tstring_full_
tstring_format_spec:
    | TSTRING_MIDDLE
    | tstring_format_spec_replacement_field
tstring_full_format_spec:
    | ':' tstring_format_spec*
tstring_replacement_field:
    | '{' annotated_rhs '='? [fstring_conversion] [tstring_full_
tstring_middle:
```

```
    | tstring_replacement_field
    | TSTRING_MIDDLE
tstring:
    | TSTRING_START tstring_middle* TSTRING_END

string: STRING
strings:
    | (fstring|string)+
    | tstring+

list:
    | '[' [star_named_expressions_sequence] ']'

tuple:
    | '(' [star_named_expression_sequence ',' [star_named_expres

set: '{' star_named_expressions_sequence '}'

# Dicts
# -----

dict:
    | '{' [double_starred_kvpairs] '}'

double_starred_kvpairs: ','.double_starred_kvpair+ [',']

double_starred_kvpair:
    | '**' bitwise_or
    | kvpair

kvpair: expression ':' expression

# Comprehensions & Generators
# ---------------------------

for_if_clauses:
    | for_if_clause+

for_if_clause:
    | 'async' 'for' star_targets 'in' ~ disjunction ('if' disjur
    | 'for' star_targets 'in' ~ disjunction ('if' disjunction )*

listcomp:
```

```
    | '[' star_named_expression for_if_clauses ']'

setcomp:
    | '{' star_named_expression for_if_clauses '}'

genexp:
    | '(' ( assignment_expression | expression !':=' | starred_e

dictcomp:
    | '{' kvpair for_if_clauses '}'
    | '{' '**' expression for_if_clauses '}'

# FUNCTION CALL ARGUMENTS
# =======================

arguments:
    | args [','] &')'

args:
    | ','.(starred_expression | ( assignment_expression | expres
    | kwargs

kwargs:
    | ','.kwarg_or_starred+ ',' ','.kwarg_or_double_starred+
    | ','.kwarg_or_starred+
    | ','.kwarg_or_double_starred+

starred_expression:
    | '*' expression

kwarg_or_starred:
    | NAME '=' expression
    | starred_expression

kwarg_or_double_starred:
    | NAME '=' expression
    | '**' expression

# ASSIGNMENT TARGETS
# ==================

# Generic targets
# ---------------
```

```
# NOTE: star_targets may contain *bitwise_or, targets may not.
star_targets:
    | star_target !','
    | star_target (',' star_target )* [',']

star_targets_list_seq: ','.star_target+ [',']

star_targets_tuple_seq:
    | star_target (',' star_target )+ [',']
    | star_target ','

star_target:
    | '*' (!'*' star_target)
    | target_with_star_atom

target_with_star_atom:
    | t_primary '.' NAME !t_lookahead
    | t_primary '[' slices ']' !t_lookahead
    | star_atom

star_atom:
    | NAME
    | '(' target_with_star_atom ')'
    | '(' [star_targets_tuple_seq] ')'
    | '[' [star_targets_list_seq] ']'

single_target:
    | single_subscript_attribute_target
    | NAME
    | '(' single_target ')'

single_subscript_attribute_target:
    | t_primary '.' NAME !t_lookahead
    | t_primary '[' slices ']' !t_lookahead

t_primary:
    | t_primary '.' NAME &t_lookahead
    | t_primary '[' slices ']' &t_lookahead
    | t_primary genexp &t_lookahead
    | t_primary '(' [arguments] ')' &t_lookahead
    | atom &t_lookahead
```

```
t_lookahead: '(' | '[' | '.'

# Targets for del statements
# --------------------------

del_targets: ','.del_target+ [',']

del_target:
    | t_primary '.' NAME !t_lookahead
    | t_primary '[' slices ']' !t_lookahead
    | del_t_atom

del_t_atom:
    | NAME
    | '(' del_target ')'
    | '(' [del_targets] ')'
    | '[' [del_targets] ']'

# TYPING ELEMENTS
# ---------------

# type_expressions allow */** but ignore them
type_expressions:
    | ','.expression+ ',' '*' expression ',' '**' expression
    | ','.expression+ ',' '*' expression
    | ','.expression+ ',' '**' expression
    | '*' expression ',' '**' expression
    | '*' expression
    | '**' expression
    | ','.expression+

func_type_comment:
    | NEWLINE TYPE_COMMENT &(NEWLINE INDENT)   # Must be followe
    | TYPE_COMMENT

# ========================= END OF THE GRAMMAR ==================



# ========================= START OF INVALID RULES ==============
```

Referência da bibliotecas embutidas do Python

Python vem com diversas funções e classes embutidas.

As classes embutidas incluem tipos de dados que seriam normalmente considerados como parte “central” de uma linguagem, tais como números e listas. Para esses tipos, o núcleo da linguagem Python define a forma de literais e coloca algumas restrições em suas semânticas, mas não define completamente as semânticas.

Os *built-ins* também incluem funções e exceções — objetos que podem ser usados por todo o código Python sem a necessidade de uma instrução `import`. Alguns desses são definidos pelo núcleo da linguagem, mas muitos não são essenciais para as semânticas principais e são apenas descritos aqui.

Ver também

Além das funções embutidas, o Python fornece uma extensa biblioteca padrão importável, veja A biblioteca padrão do Python.

- Tipos embutidos
  - Teste do valor verdade
  - Operações booleanas — `and`, `or`, `not`
  - Comparações
  - Tipos numéricos — `int`, `float`, `complex`
  - Tipo booleano - `bool`
  - Tipos iteradores
  - Tipos sequências — `list`, `tuple`, `range`
  - Resumo dos métodos de tipo de sequência binária e de texto
  - Tipo sequência de texto — `str`
  - Tipos de sequência binária — `bytes`, `bytearray`, `memoryview`
  - Tipo conjuntos — `set`, `frozenset`

    - Tipos mapeamentos — `dict`, `frozendict`
    - Tipos de Gerenciador de Contexto
    - Tipos de anotação de tipo — Apelido genérico, União
    - Outros tipos embutidos
    - Atributos Especiais
    - Limitação de comprimento de string na conversão para inteiro
- Constantes embutidas
    - Constantes adicionadas pelo módulo `site`
- Funções embutidas
- Exceções embutidas
    - Contexto da exceção
    - Herdando de exceções embutidas
    - Classes base
    - Exceções concretas
    - Avisos
    - Grupos de exceções
    - Hierarquia das exceções
- Garantias de segurança de thread
    - Níveis de segurança de threads
    - Segurança de threads para objetos lista
    - Segurança de threads para objetos dict
    - Segurança de thread para objetos conjunto
    - Segurança para thread para objetos bytearray
    - Segurança para thread para objetos memoryview
- Complexidade de tempo de operações em tipos embutidos
    - `list`
    - `tuple`
    - `dict`, `frozendict`
    - `set`, `frozenset`
    - `str`, `bytes`, `bytearray`
    - `memoryview`
    - `range`
    - Notas

Tipos embutidos

As seções a seguir descrevem os tipos padrão que são embutidos ao interpretador.

Os principais tipos embutidos são numéricos, sequências, mapeamentos, classes, instâncias e exceções.

Algumas classes de coleção são mutáveis. Os métodos que adicionam, subtraem ou reorganizam seus membros no lugar, e não retornam um item específico, nunca retornam a instância da coleção propriamente dita, mas um `None`.

Algumas operações são suportadas por vários tipos de objetos; em particular, praticamente todos os objetos podem ser comparados em termos de igualdade, testados quanto ao valor verdade e convertidos em uma string (com a função `repr()` ou a função ligeiramente diferente `str()`). A última função é implicitamente usada quando um objeto é escrito pela função `print()`.

Teste do valor verdade

Qualquer objeto pode ser testado quanto ao valor verdade, para uso em uma condição `if` ou `while` ou como operando das operações booleanas abaixo.

Por predefinição, um objeto é considerado verdadeiro, a menos que a sua classe defina um método `__bool__()` que retorne `False` ou um método `__len__()` que retorne zero, quando chamado com o objeto. [[1]] Se um dos métodos levantar uma exceção quando chamado, a exceção é propagada e o objeto não tem um valor verdadeiro (por exemplo, `NotImplemented`). Aqui estão a maioria dos objetos integrados considerados falsos:

- constantes definidas para serem falsas: `None` e `False`
- zero de qualquer tipo numérico: `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0, 1)`
- sequências e coleções vazias: `''`, `()`, `[]`, `{}`, `set()`, `range(0)`

Operações e funções embutidas que têm um resultado Booleano retornam `0` ou `False` para falso e `1` ou `True` para verdadeiro, salvo indicações ao contrário. (Exceção importante: as operações Booleanas `or` e `and` sempre retornam um de seus operandos.)

Operações booleanas — `and`, `or`, `not`

Esses são as operações booleanas, ordenados por prioridade ascendente:

Operação	Resultado	Notas
`x or y`	se *x* é true, então *x*, do contrário *y*	(1)
`x and y`	se *x* é falso, então *x*, do contrário *y*	(2)
`not x`	se *x* é falso, então `True`, caso contrário `False`	(3)

Notas:

1. Esse é um operador de curto-circuito, por isso só avalia o segundo argumento se o primeiro é falso.
2. Este é um operador de curto-circuito, por isso só avalia o segundo argumento se o primeiro é verdadeiro.
3. `not` tem uma prioridade mais baixa do que operadores não booleanos, então `not a == b` é interpretado como `not (a == b)` e `a == not b` é um erro de sintaxe.

Comparações

Há oito operadores comparativos no Python. Todos eles possuem a mesma prioridade (que é maior do que aquela das operações Booleanas). Comparações podem ser encadeadas arbitrariamente; por exemplo, `x < y <= z` é equivalente a `x < y and y <= z`, exceto que *y* é avaliado apenas uma vez (porém em ambos os casos *z* não é avaliado de todo quando `x < y` é sabido ser falso).

Esta tabela resume as operações de comparação:

Operação	Significado
`<`	estritamente menor que

Operação	Significado
`<=`	menor que ou igual
`>`	estritamente maior que
`>=`	maior que ou igual
`==`	igual
`!=`	não é igual
`is`	identidade do objeto
`is not`	identidade de objeto negada

Salvo indicação em contrário, objetos de tipos diferentes nunca são comparados como iguais. O operador == é sempre definido, mas para alguns tipos de objetos (por exemplo, objetos de classe) é equivalente a `is`. Os operadores <, <=, > e >= só são definidos onde fazem sentido; por exemplo, eles levantam uma exceção `TypeError` quando um dos argumentos é um número complexo.

Instâncias não idênticas de uma classe normalmente comparam-se como desiguais ao menos que a classe defina o método `__eq__()`.

Instâncias de uma classe não podem ser ordenadas com respeito a outras instâncias da mesma classe, ou outros tipos de objeto, ao menos que a classe defina o suficiente dos métodos `__lt__()`, `__le__()`, `__gt__()` e `__ge__()` (no geral, **`__lt__()`** e `__eq__()` são suficientes, se você quiser o significado convencional dos operadores de comparação).

O comportamento dos operadores `is` e `is not` não pode ser personalizado; além disso eles podem ser aplicados a quaisquer dois objetos e nunca levantam uma exceção.

Mais duas operações com a mesma prioridade sintática, `in` e `not in`, são suportadas por tipos que são iteráveis ou implementam o método `__contains__()`.

Tipos numéricos — `int`, `float`, `complex`

Existem três tipos numéricos distintos: *inteiros*, *números de ponto flutuante* e *números complexos*. Além disso, os booleanos são um subtipo de números inteiros. Inteiros têm precisão ilimitada. Números de ponto flutuante são geralmente implementados usando double em C; informações sobre a precisão e representação interna dos números de ponto flutuante para a máquina na qual seu programa está sendo executado estão disponíveis em `sys.float_info`. Números complexos têm uma parte real e imaginária, cada um com um número de ponto flutuante. Para extrair essas partes de um número complexo *z*, use `z.real` e `z.imag`. (A biblioteca padrão inclui os tipos numéricos adicionais `fractions.Fraction`, para racionais, e `decimal.Decimal`, para números de ponto flutuante com precisão definida pelo usuário.)

Números são criados por literais numéricos ou como resultado de operadores e funções embutidas. Integrais literais planos (incluindo números hexadecimais, octais e binários) culminam em integrais. Literais numéricos contendo um ponto decimal ou um sinal exponencial resultam em números de ponto flutuante. Anexando `'j'` ou `'J'` para um literal numérico resulta em um número imaginário (um número complexo com uma parte real zero) com a qual você pode adicionar a um integral ou flutuante para receber um número complexo com partes reais e imaginárias.

Os construtores `int()`, `float()`, e `complex()` podem ser usados para produzir números de um tipo específico.

Python oferece suporte completo a aritmética mista: quando um operador de aritmética binária tem operandos de tipos numéricos embutidos diferentes, o operando com o tipo “mais estreito” é ampliado para o tipo do outro operando.

- Se ambos os argumentos forem números complexos, nenhuma conversão será realizada;
- se qualquer argumento for um número complexo ou de ponto flutuante, o outro será convertido em um número de ponto flutuante;
- caso contrário, ambos devem ser inteiros e nenhuma conversão é necessária.

A aritmética com operandos complexos e reais é definida pela fórmula matemática usual, por exemplo:

```
x + complex(u, v) = complex(x + u, v)
x * complex(u, v) = complex(x * u, x * v)
```

Uma comparação entre números de tipos diferentes se comporta como se os valores exatos desses números estivessem sendo comparados. [[2]]

Todos os tipos numéricos (exceto complexos) suportam as seguintes operações (para prioridades das operações, consulte Precedência de operadores):

Operação	**Resultado**	**Notas**	**Documentação completa**
`x + y`	soma de *x* e *y*		
`x - y`	diferença de *x* e *y*		
`x * y`	produto de *x* e *y*		
`x / y`	quociente de *x* e *y*		
`x // y`	piso do quociente de *x* e *y*	(1)(2)	
`x % y`	restante de `x / y`	(2)	
`-x`	*x* negado		
`+x`	*x* inalterado		
`abs(x)`	valor absoluto ou magnitude de *x*		`abs()`
`int(x)`	*x* convertido em inteiro	(3)(6)	`int()`

Operação	Resultado	Notas	Documentação completa
`float(x)`	*x* convertido em ponto flutuante	(4)(6)	`float()`
`complex(re, im)`	um número complexo com parte real *re*, parte imaginária *im*. *im* tem como padrão zero.	(6)	`complex()`
`c.conjugate()`	conjugado do número complexo *c*		
`divmod(x, y)`	o par `(x // y, x % y)`	(2)	`divmod()`
`pow(x, y)`	*x* elevado a *y*	(5)	`pow()`
`x ** y`	*x* elevado a *y*	(5)	

Notas:

1. Também referido como uma divisão inteira. Para operandos do tipo `int`, o resultado tem o tipo **`int`**; Para operandos do tipo `float`, o resultado tem o tipo **`float`**. Em geral o resultado é inteiro como todo, apesar do tipo do resultado não necessariamente ser **`int`**. O resultado é sempre arredondado para menos infinito: `1//2` é `0`, `(-1)//2` é `-1`, `1//(-2)` é `-1` e `(-1)//(-2)` é `0`.

2. Não para números complexos. Ao invés disso, converte para pontos flutuantes usando `abs()` se for apropriado.

3. Conversão de `float` para `int` trunca, descartando a parte fracionária. Veja as funções `math.floor()` e `math.ceil()` para conversões alternativas.

4. ponto flutuante também aceita a string “nan” e “inf” com um prefixo opcional “+” ou “-” a Não é um Número (NaN) e infinidade positiva ou negativa.

5. Python define `pow(0, 0)` e `0 ** 0` sendo `1`, como é comum para linguagens de programação.

6. Os literais numéricos aceitos incluem os dígitos de `0` a `9` ou qualquer equivalente Unicode (pontos de código com a propriedade `Nd`).

   Veja o Unicode Standard [https://unicode.org/Public/UNIDATA/extracted/DerivedNumericType.txt] para obter uma lista completa de pontos de código com a propriedade `Nd`.

Todos os tipos `numbers.Real` (`int` e `float`) também incluem as seguintes operações.

Operação	Resultado
`math.trunc(x)`	*x* truncado para `Integral`
`round(x[, n])`	*x* arredondado para *n* dígitos, arredondando metade para igualar. Se *n* é omitido, ele toma o padrão de 0.
`math.floor(x)`	o maior `Integral` <= *x*
`math.ceil(x)`	pelo menos `Integral` >= *x*

Para operações numéricas adicionais, consulte os módulos `math` e `cmath`.

Operações de bits em tipos inteiros

Operações bit a bit só fazem sentido para números inteiros. O resultado de operações bit a bit é calculado como se fosse realizado no complemento de dois com um número infinito de bits de sinal.

As prioridades das operações bit a bit binárias são todas menores do que as operações numéricas e maiores que as comparações; a operação unária `~` tem a mesma prioridade que as outras operações numéricas unárias (`+` e `-`).

Esta tabela lista as operações de bits classificadas em prioridade ascendente:

Operação	Resultado	Notas
`x | y`	*ou* bit a bit de *x* e *y*	(4)
`x ^ y`	*ou exclusivo* bit a bit de *x* e *y*	(4)
`x & y`	*e* bit a bit de *x* e *y*	(4)
`x << n`	*x* deslocado para a esquerda por *n* bits	(1)(2)
`x >> n`	*x* deslocado para a direita por *n* bits	(1)(3)
`~x`	os bits de *x* invertidos	

Notas:

1. Contagens de deslocamento negativo são ilegais e levantam uma `ValueError` .
2. Um deslocamento à esquerda por *n* bits é equivalente à multiplicação por `pow(2, n)`.
3. Um deslocamento à direita por *n* bits é equivalente à divisão pelo piso por `pow(2, n)`.
4. Executar esses cálculos com pelo menos um bit extra de extensão de sinal na representação de complemento de dois finitos (uma largura de bit de trabalho `1 + max(x.bit_length(), y.bit_length())` ou mais) é suficiente para obter o mesmo resultado como se houvesse um número infinito de bits de sinal.

Métodos adicionais em tipos inteiros

O tipo int implementa a classe base abstrata `numbers.Integral`. Além disso, ele provê mais alguns métodos:

int.bit_length()

Retorna o número de bits necessários para representar um inteiro em binário, excluindo o sinal e entrelinha zeros:

```
>>> n = -37
>>> bin(n)
'-0b100101'
>>> n.bit_length()
6
```

Mais precisamente, se `x` for diferente de zero, então `x.bit_length()` é o único positivo inteiro `k` tal que `2**(k-1) <= abs(x) < 2**k`. Equivalentemente, quando `abs(x)` for pequeno o suficiente para ter um logaritmo corretamente arrendondado, então `k = 1 + int(log(abs(x), 2))`. Se `x` é zero, então `x.bit_length()` retorna `0`.

Equivalente a:

```
def bit_length(self):
    s = bin(self)       # representação binária:  bin(-37) --> '-0b1
    s = s.lstrip('-0b') # remove zeros no início e sinal negativo
    return len(s)       # len('100101') --> 6
```

Adicionado na versão 3.1.

int.bit_count()

Retorna o número de unidades na representação binária do valor absoluto do inteiro. Isso também é conhecido como contagem da população. Exemplo:

```
>>> n = 19
>>> bin(n)
'0b10011'
>>> n.bit_count()
3
>>> (-n).bit_count()
3
```

Equivalente a:

```
def bit_count(self):
    return bin(self).count("1")
```

Adicionado na versão 3.10.

int.to_bytes(*length=1*, *byteorder='big'*, **, signed=False*)

Retorna um vetor de bytes representando um inteiro.

```
>>> (1024).to_bytes(2, byteorder='big')
b'\x04\x00'
>>> (1024).to_bytes(10, byteorder='big')
b'\x00\x00\x00\x00\x00\x00\x00\x00\x04\x00'
>>> (-1024).to_bytes(10, byteorder='big', signed=True)
b'\xff\xff\xff\xff\xff\xff\xff\xff\xfc\x00'
>>> x = 1000
>>> x.to_bytes((x.bit_length() + 7) // 8, byteorder='little')
b'\xe8\x03'
```

O inteiro é representado usando *length* bytes, e o padrão é 1. Uma `OverflowError` é levantada se um inteiro não é representável com o dado número de bytes.

O argumento *byteorder* determina a ordem de bytes usada para representar um inteiro, e o padrão é `"big"`. Se o *byteorder* é `"big"`, o byte mais significativo está no início do vetor de byte. Se *byteorder* é `"little"`, o byte mais significativo está no final do vetor de byte.

O argumento *signed* determina aonde o modo de complemento de dois é usado para representar o inteiro. Se *signed* é `False` e um inteiro negativo é dado, uma `OverflowError` é levantada. O valor padrão para *signed* é `False`.

Os valores padrão podem ser usados para transformar convenientemente um inteiro em um objeto de byte único:

```
>>> (65).to_bytes()
b'A'
```

No entanto, ao usar os argumentos padrão, não tente converter um valor maior que 255 ou você obterá um `OverflowError`.

Equivalente a:

```
def to_bytes(n, length=1, byteorder='big', signed=False):
    if byteorder == 'little':
        order = range(length)
    elif byteorder == 'big':
        order = reversed(range(length))
    else:
        raise ValueError("byteorder must be either 'little' or 'big'

    return bytes((n >> i*8) & 0xff for i in order)
```

Adicionado na versão 3.2.

Alterado na versão 3.11: Adicionados valores de argumento padrão para `length` e `byteorder`.

classmethod int.from_bytes(*bytes*, *byteorder='big'*, **, signed=False*)

Retorna o inteiro representado pelo vetor de bytes dado.

```
>>> int.from_bytes(b'\x00\x10', byteorder='big')
16
>>> int.from_bytes(b'\x00\x10', byteorder='little')
4096
>>> int.from_bytes(b'\xfc\x00', byteorder='big', signed=True)
-1024
>>> int.from_bytes(b'\xfc\x00', byteorder='big', signed=False)
64512
>>> int.from_bytes([255, 0, 0], byteorder='big')
16711680
```

O argumento *bytes* precisa ou ser um objeto bytes ou similar ou um iterador produzindo bytes.

O argumento *byteorder* determina a ordem de bytes usada para representar um inteiro, e o padrão é `"big"`. Se o *byteorder* é `"big"`, o byte mais significativo está no início do vetor de byte. Se *byteorder* é `"little"`, o byte mais significativo está no final do vetor de byte. Para requisitar a ordem nativa de byte do sistema hospedeiro, use **`sys.byteorder`** como o valor da ordem de byte.

O argumento *signed* indica quando o complemento de dois é usado para representar o inteiro.

Equivalente a:

```
def from_bytes(bytes, byteorder='big', signed=False):
    if byteorder == 'little':
        little_ordered = list(bytes)
    elif byteorder == 'big':
        little_ordered = list(reversed(bytes))
    else:
        raise ValueError("byteorder must be either 'little' or 'big'

    n = sum(b << i*8 for i, b in enumerate(little_ordered))
    if signed and little_ordered and (little_ordered[-1] & 0x80):
        n -= 1 << 8*len(little_ordered)
```

```
    return n
```

Adicionado na versão 3.2.

Alterado na versão 3.11: Adicionado valor de argumento padrão para `byteorder`.

int.as_integer_ratio()

Retorna um par de números inteiros cuja razão é igual ao número inteiro original e tem um denominador positivo. A proporção inteira de números inteiros (números inteiros) é sempre o número inteiro como numerador e `1` como denominador.

Adicionado na versão 3.8.

int.is_integer()

Retorna `True`. Existe para compatibilidade com tipo pato com **`float.is_integer()`**.

Adicionado na versão 3.12.

Métodos adicionais em ponto flutuante

O tipo float implementa a classe base abstrata **`numbers.Real`**. float também possui os seguintes métodos adicionais.

classmethod float.from_number(*x*)

Método de classe para retornar um número de ponto flutuante construído a partir de um número *x*.

Se o argumento é um inteiro ou um número de ponto flutuante, um número de ponto flutuante com o mesmo valor (com a precisão de ponto flutuante de Python) é retornado. Se o argumento está fora do intervalo de um ponto flutuante Python, uma exceção **`OverflowError`** será levantada.

Para um objeto Python genérico `x`, `float.from_number(x)` delega para o método `x.__float__()`. Se **`__float__()`** não estiver definido, então ele delega para o método **`__index__()`**.

Adicionado na versão 3.14.

float.as_integer_ratio()

Retorna um par de inteiros dos quais a proporção é exatamente igual ao float original. A proporção está em seus termos mais baixos e tem um denominador positivo. Levanta um `OverflowError` em infinidades e um `ValueError` em NaNs.

float.is_integer()

Retorna `True` se a instância do float for finita com o valor integral e `False`, caso contrário:

```
>>> (-2.0).is_integer()
True
>>> (3.2).is_integer()
False
```

Dois métodos suportam conversão para e de strings hexadecimais. Uma vez que os pontos flutuantes do Python são armazenados internamente como números binários, a conversão de um ponto flutuante para ou de uma string *decimal* geralmente envolve um pequeno erro de arredondamento. Em contraste, as strings hexadecimais permitem a representação exata e a especificação de números de ponto flutuante. Isso pode ser útil na depuração e no trabalho numérico.

float.hex()

Retorna a representação de um número de ponto flutuante como uma string hexadecimal. Para números de ponto flutuante finitos, essa representação vai sempre incluir um `0x` inicial e um `p` final e expoente.

classmethod float.fromhex(*s*)

Método de classe para retornar um ponto flutuante representado por uma string hexadecimal *s*. A string *s* pode ter espaços em branco iniciais e finais.

Note que `float.hex()` é um método de instância, enquanto `float.fromhex()` é um método de classe.

Uma string hexadecimal toma a forma:

```
[sign] ['0x'] integer ['.' fraction] ['p' exponent]
```

onde o sinal opcional `sign` pode ser tanto `+` or `-`, `integer` e `fraction` são strings de dígitos hexadecimais, e `exponent` é um inteiro decimal com um sinal opcional no início. Maiúsculo ou minúsculo não é significativo, e deve haver ao menos um dígito hexadecimal tanto no inteiro ou na fração. Essa sintaxe é similar à sintaxe

especificada na seção 6.4.4.2 do padrão C99, e também do da sintaxe usada no Java 1.5 em diante. Em particular, a saída de `float.hex()` é usável como um literal hexadecimal de ponto flutuante em código C ou Java, e strings hexadecimais produzidas pelo formato do caractere `%a` do C ou `Double.toHexString` do Java são aceitos pelo `float.fromhex()`.

Note que o expoente é escrito em decimal ao invés de hexadecimal, e que ele dá a potência de 2 pela qual se multiplica o coeficiente. Por exemplo, a string hexadecimal `0x3.a7p10` representa o número de ponto flutuante `(3 + 10./16 + 7./16**2) * 2.0**10` ou `3740.0`:

```
>>> float.fromhex('0x3.a7p10')
3740.0
```

Aplicando a conversão inversa a `3740.0` retorna uma string hexadecimal diferente representada pelo mesmo número:

```
>>> float.hex(3740.0)
'0x1.d380000000000p+11'
```

Métodos adicionais em números complexos

O tipo **`complex`** implementa a classe base abstrata `numbers.Complex`. **`complex`** também possui os seguintes métodos adicionais.

classmethod complex.from_number(*x*)

Método de classe para converter um número para um número complexo.

Para um objeto Python `x` qualquer, `complex.from_number(x)` delega para `x.__complex__()`. Se `__complex__()` não está definido então a chamada é repassada para `__float__()`. Se **`__float__()`** não está definido então a chamada é, novamente, repassada para `__index__()`.

Adicionado na versão 3.14.

Hashing de tipos numéricos

Para números `x` e `y`, possivelmente de diferentes tipos, é um requisito que `hash(x) == hash(y)` sempre que `x == y` (veja a documentação do método `__hash__()` para mais detalhes). Para facilitar a implementação e eficiência através de uma variedade de tipos numéricos (incluindo `int`, `float`, `decimal.Decimal` e `fractions.Fraction`), o hash do Python para tipos numéricos é baseado em uma

única função matemática que é definida para qualquer número racional e, portanto, se aplica para todas as instâncias de **`int`** e **`fractions.Fraction`**, e todas as instâncias finitas das classes **`float`** e **`decimal.Decimal`**. Essencialmente, essa função é dada pelo módulo de redução `P` para um primo fixado `P`. O valor de `P` é disponibilizado ao Python como um atributo **`modulus`** do **`sys.hash_info`**.

Atualmente, o primo usado é `P = 2**31 - 1` em máquinas com longos de 32 bits do C e `P = 2**61 - 1` em máquinas com longos de 64 bits do C.

Aqui estão as regras em detalhe:

- Se `x = m / n` é um número racional não negativo e `n` não é divisível por `P`, define `hash(x)` como `m * invmod(n, P) % P`, onde `invmod(n, P)` retorna o inverso do módulo de `n` com `P`.
- Se `x = m / n` é um número racional não negativo e `n` é divisível por `P` (porém `m` não é) então `n` não possui modulo `P` inverso e a regra acima não se aplica; nesse caso defina `hash(x)` para ser o valor constante `sys.hash_info.inf`.
- Se `x = m / n` é um número racional negativo, defina `hash(x)` como `-hash(-x)`. Se a hash resultante é `-1`, a substituo com `-2`.
- Os valores particulares `sys.hash_info.inf` e `-sys.hash_info.inf` são usados como valores de hash para infinidade positiva ou infinidade negativa (respectivamente).
- Para número **`complex`** `z`, o valor da hash do número real partes imaginárias são combinados calculando `hash(z.real) + sys.hash_info.imag * hash(z.imag)`, módulo reduzido `2**sys.hash_info.width` de modo que isto permaneça em `range(-2**(sys.hash_info.width - 1), 2**(sys.hash_info.width - 1))`. Novamente, se o resultado é `-1`, ele é substituído por `-2`.

Para esclarecer as regras acima, aqui estão alguns exemplos de código em Python, equivalentes ao hash embutido, para calcular o hash de números racionais, **`float`** ou **`complex`**:

```
import sys, math

def hash_fraction(m, n):
    """Calcula o hash de um número racional m / n.

    Presume que m e n são inteiros, com n positivo.
    Equivalente a hash(fractions.Fraction(m, n)).

    """
    P = sys.hash_info.modulus
```

```
    # Remove fatores comuns de P.  (Desnecessário se m e n são primos e
    while m % P == n % P == 0:
        m, n = m // P, n // P

    if n % P == 0:
        hash_value = sys.hash_info.inf
    else:
        # Pequeno teorema de Fermat: pow(n, P-1, P) é 1, então
        # pow(n, P-2, P) dá o inverso de n módulo P.
        hash_value = (abs(m) % P) * pow(n, P - 2, P) % P
    if m < 0:
        hash_value = -hash_value
    if hash_value == -1:
        hash_value = -2
    return hash_value

def hash_float(x):
    """Calcula o hash de um ponto flutuante x."""

    if math.isnan(x):
        return object.__hash__(x)
    elif math.isinf(x):
        return sys.hash_info.inf if x > 0 else -sys.hash_info.inf
    else:
        return hash_fraction(*x.as_integer_ratio())

def hash_complex(z):
    """Calcula o hash de um número complexo z."""

    hash_value = hash_float(z.real) + sys.hash_info.imag * hash_float(z
    # faz um módulo para redução de sinal 2**sys.hash_info.width
    M = 2**(sys.hash_info.width - 1)
    hash_value = (hash_value & (M - 1)) - (hash_value & M)
    if hash_value == -1:
        hash_value = -2
    return hash_value
```

Tipo booleano - `bool`

Booleanos representam valores verdade. O tipo `bool` tem exatamente duas instâncias constantes: `True` e `False`.

A função embutida `bool()` converte qualquer valor em um booleano, se o valor puder ser interpretado como um valor verdade (consultar seção Teste do valor verdade acima).

Para operações lógicas, use operadores booleanos `and`, `or` e `not`. Ao aplicar operadores bit a bit `&`, `|`, `^` a dois booleanos, o resultado é um booleano equivalente às operações lógicas “e”, “ou”, “xor”. Entretanto, é preferível usar os operadores lógicos `and`, `or` e `!=` em relação aos `&`, `|` e `^`.

Descontinuado desde a versão 3.12: O uso do operador de inversão bit a bit ~ foi descontinuado e levantará um erro no Python 3.16.

`bool` é uma subclasse **`int`** (veja Tipos numéricos — int, float, complex). Em diversos contextos numéricos, `False` e `True` se comportam como 0 e 1, respectivamente. Entretanto, basear-se somente nisso, é desencorajado; ao invés disso, a conversão deve ser feita explicitamente usando **`int()`**.

Tipos iteradores

Python suporta o conceito de iteração por contêineres. Isso é implementado usando dois métodos distintos; estes são usados para permitir classes definidas pelo usuário suportem iteração. Sequências, descritas abaixo em mais detalhes, sempre suportam os métodos de iteração.

Um método necessita ser definido para objetos contêineres afim destes proverem suporte a iterável:

container.__iter__()

Retorna um objeto iterador. O objeto deve suportar o protocolo iterador descrito abaixo. Se um contêiner suporta diferentes tipos de iterador, métodos adicionais podem ser providenciados para requisitar especificamente iteradores para aqueles tipos de iterações. (Um exemplo de um objeto suportando múltiplas formas de iteração seria uma estrutura em árvore a qual suporta ambas travessias de travessia em largura e em profundidade.) Esse método corresponde ao slot **`tp_iter`** da estrutura de tipos para objetos Python na API Python/C.

Os próprios objetos iteradores são obrigados a suportarem os dois métodos a seguir, que juntos formam o *protocolo iterador*:

iterator.__iter__()

Retorna o próprio objeto iterador em si. Isso é necessário para permitir que ambos os contêineres e iteradores sejam usados com as instruções **`for`** e **`in`**. Este método corresponde ao slot **`tp_iter`** da estrutura de tipos para objetos Python na API Python/C.

iterator.__next__()

Retorna o próximo item do iterador. Se não houver itens além, a exceção `StopIteration` é levantada. Esse método corresponde ao slot `tp_iternext` da estrutura de tipos para objetos Python na API Python/C.

Python define diversos objetos iteradores para suportar iterações sobre tipos de sequências gerais e específicas, dicionários, e outras formas mais especializadas. Os tipos específicos não são importantes além de sua implementação do protocolo iterador.

Uma vez que o método `__next__()` do iterador levantou `StopIteration`, ele deve continuar fazendo isso em chamadas subsequentes. Implementações que não obedecem essa propriedade são consideradas quebradas.

Tipos geradores

Os geradores do Python proveem uma maneira conveniente para implementar o protocolo iterador. Se o método `__iter__()` de um objeto contêiner é implementado como um gerador, ele irá automaticamente retornar um objeto iterador (tecnicamente, um objeto gerador) fornecendo os métodos `__iter__()` e `__next__()`. Mais informações sobre geradores podem ser encontradas na documentação para a expressão yield.

Tipos sequências — `list`, `tuple`, `range`

Existem três tipos básicos de sequência: listas, tuplas e objetos range. Tipos de sequência adicionais adaptados para o processamento de dados binários e strings de texto são descritos em seções dedicadas.

Operações comuns de sequências

As operações nas seguintes tabelas são suportadas pela maioria dos tipos sequências, ambos mutáveis e imutáveis. A classe ABC `collections.abc.Sequence` é fornecida para tornar fácil a correta implementação desses operadores em tipos sequências personalizados.

Essa tabela lista as operações de sequência ordenadas em prioridade ascendente. Na tabela, *s* e *t* são sequências do mesmo tipo, *n*, *i*, *j* e *k* são inteiros e *x* é um objeto arbitrário que atende a qualquer restrição de valor e tipo imposta por *s*.

As operações `in` e `not in` têm as mesmas prioridades que as operações de comparação. As operações `+` (concatenação) e `*` (repetição) têm a mesma prioridade que as operações numéricas correspondentes. [[3]]

Consulte Complexidade de tempo de operações em tipos embutidos para os custos das várias operações de sequência.

Operação	**Resultado**	**Notas**
`x in s`	`True` caso um item de *s* seja igual a *x*, caso contrário `False`	(1)
`x not in s`	`False` caso um item de *s* for igual a *x*, caso contrário `True`	(1)
`s + t`	a concatenação de *s* e *t*	(6)(7)
`s * n` ou `n * s`	equivalente a adicionar *s* a si mesmo *n* vezes	(2)(7)
`s[i]`	*i*-ésimo item de *s*, origem 0	(3)(8)
`s[i:j]`	fatia de *s* de *i* até *j*	(3)(4)
`s[i:j:k]`	fatia de *s* de *i* até *j* com passo *k*	(3)(5)
`len(s)`	comprimento de *s*	
`min(s)`	menor item de *s*	
`max(s)`	maior item de *s*	

Sequências do mesmo tipo também admitem comparações. Em particular, tuplas e listas são comparadas lexicograficamente por meio da comparação de elementos correspondentes. Isso significa que, para comparar igualmente, cada elemento deve ser comparado igual e as duas sequências devem ser do mesmo tipo e possuir o mesmo comprimento. (Para detalhes completos, veja Comparações na referência da linguagem.)

Iteradores para frente e reversos sobre sequências mutáveis acessam valores usando um índice. Esse índice continuará avançando (ou retrocedendo) mesmo que a sequência subjacente seja alterada. O iterador termina somente quando um **`IndexError`** ou um **`StopIteration`** é encontrado (ou quando o índice cai abaixo de zero).

Notas:

1. Enquanto as operações `in` e `not in` são usadas somente para testes de contenção simples em modo geral, algumas sequências especializadas (tais como **`str`**, **`bytes`** e **`bytearray`**) também usam-nas para testes de subsequências:

   ```
   >>> "gg" in "eggs"
   True
   ```

2. Os valores de *n* menos `0` são tratados como `0` (o que produz uma sequência vazia do mesmo tipo que *s*). Observe que os itens na sequência *s* não são copiados; eles são referenciados várias vezes. Isso frequentemente assombra novos programadores Python; considere então que:

   ```
   >>> lists = [[]] * 3
   >>> lists
   [[], [], []]
   >>> lists[0].append(3)
   >>> lists
   [[3], [3], [3]]
   ```

   O que aconteceu é que `[[]]` é uma lista de um elemento contendo uma lista vazia, então todos os três elementos de `[[]] * 3` são referências a esta única lista vazia. Modificar qualquer um dos elementos de `lists` modifica a lista vazia. Podemos criar uma lista de listas diferentes dessa maneira:

   ```
   >>> lists = [[] for i in range(3)]
   >>> lists[0].append(3)
   >>> lists[1].append(5)
   >>> lists[2].append(7)
   ```

```
>>> lists
[[3], [5], [7]]
```

Mais explicação está disponível no FAQ Como faço para criar uma lista multidimensional?.

3. Se *i* ou *j* for negativo, o índice será relativo ao fim da sequência *s*: `len(s) + i` ou `len(s) + j` será substituído. Mas note que `-0` ainda será `0`.

4. A fatia de *s* de *i* a *j* é definida como a sequência de itens com índice *k* tal que `i <= k < j`.

   - Se *i* for omitido ou `None`, usa `0`.
   - Se *j* for omitido ou `None`, usa `len(s)`.
   - Se *i* ou *j* for menor que `-len(s)`, usa `0`.
   - Se *i* ou *j* for maior que `len(s)`, usa `len(s)`.
   - Se *i* for maior ou igual a *j*, a fatia está vazia.

5. A fatia *s* de *i* até *j* com passo *k* é definida como sendo a sequência de itens com índice `x = i + n*k` tal que `0 <= n < (j-i)/k`. Em outras palavras, os índices são `i`, `i+k`, `i+2*k`, `i+3*k` e assim por diante, parando quando *j* for atingido (mas nunca incluindo *j*). Quando *k* for positivo, *i* e *j* serão reduzidos a `len(s)` se forem maiores. Quando *k* for negativo, *i* e *j* são reduzidos para `len(s) - 1` se forem maiores. Se *i* ou *j* forem omitidos ou `None`, eles se tornam valores "finais" (cujo final depende de *k*). Nota: *k* não pode ser zero. Se *k* for `None`, o mesmo será tratado como sendo igual a `1`.

6. Concatenar sequências imutáveis sempre resulta em um novo objeto. Isso significa que a criação de uma sequência por concatenação repetida terá um custo quadrático de tempo de execução no comprimento total da sequência. Para obter um custo de tempo de execução linear, devemos alternar para uma das alternativas abaixo:
   - Se estiver concatenando objetos `str`, você pode criar uma lista e usar `str.join()` no final ou então escrever numa instância de `io.StringIO` e recuperar o seu valor ao final
   - Se estiver concatenando objetos `bytes`, você também pode usar o método `bytes.join()` ou `io.BytesIO`, ou você pode fazer concatenação interna com um objeto `bytearray`. A classe **`bytearray`** são objetos mutáveis e possuem um eficiente mecanismo de superalocação
   - Se estiver concatenando objetos `tuple`, estenda a classe `list` em vez disso
   - para outros tipos, busque na documentação relevante da classe

7. Alguns tipos de sequências (como `range`) apenas aceitam sequências de itens que seguem padrões específicos e, portanto, não implementam concatenação nem repetição de sequências.
8. Uma `IndexError` é levantada se *i* estiver fora do intervalo da sequência.

Métodos de sequência

Tipos de sequência também oferecem suporte aos seguintes métodos:

sequence.count(*value*, /)

Retorna o número total de ocorrências de *value* em *sequence*.

sequence.index(*value*[, *start*[, *stop*]])

Retorna o índice da primeira ocorrência de *value* em *sequence*.

Levanta `ValueError` se *value* não for encontrado em *sequence*.

Os argumentos *start* ou *stop* permitem uma busca eficiente de subseções da sequência, começando em *start* e terminando em *stop*. Isso é aproximadamente equivalente a `start + sequence[start:stop].index(value)`, só que sem copiar nenhum dado.

Cuidado

Nem todos os tipos de sequência oferecem suporte a passagem dos argumentos *start* e *stop*.

Tipos sequência imutáveis

A única operação que os tipos sequências imutáveis geralmente implementam que também não é implementada pelos tipos sequências mutáveis é suporte para a função embutida `hash()`.

Esse suporte permite sequências imutáveis, tais como instâncias da classe `tuple`, serem usadas como chaves de dicionários `dict` e armazenados em instâncias de `set` e de `frozenset`.

A tentativa de obter um hash de uma sequência imutável que contém valores desnecessários resultará em um erro `TypeError`.

Tipos sequências mutáveis

As operações na tabela a seguir são definidas em tipos sequência mutáveis. A ABC `collections.abc.MutableSequence` é fornecida para tornar mais fácil a implementação correta dessas operações em tipos sequências personalizados.

Na tabela *s* é uma instância de um tipo de sequência mutável, *t* é qualquer objeto iterável e *x* é um objeto arbitrário que atende a qualquer restrição de tipo e valor imposto por *s* (por exemplo `bytearray` só aceita inteiros que atendam a restrição de valor `0 <= x <= 255`).

Operação	Resultado	Notas
`s[i] = x`	item *i* de *s* é substituído por *x*	
`del s[i]`	remove o item *i* de *s*	
`s[i:j] = t`	fatias de *s* de *i* até *j* são substituídas pelo conteúdo do iterável *t*	
`del s[i:j]`	remove os elementos de `s[i:j]` da lista (mesmo que `s[i:j] = []`)	
`s[i:j:k] = t`	os elementos de `s[i:j:k]` são substituídos por aqueles de *t*	(1)
`del s[i:j:k]`	remove os elementos de `s[i:j:k]` desde a listas	
`s += t`	estende *s* com o conteúdo de *t* (na maior parte do mesmo `s[len(s):len(s)] = t`)	
`s *= n`	atualiza *s* com o seu conteúdo por *n* vezes	(2)

Notas:

1. se *k* não for igual a `1`, *t* deve ter o mesmo comprimento que a fatia a qual ele está substituindo.
2. O valor *n* é um inteiro, ou um objeto implementando `__index__()`. Valores zero e negativos de *n* limparão a sequência. Os itens na sequência não são copiados; eles são referenciados várias vezes, como explicado para `s * n` em Operações comuns de sequências.

Métodos de sequências mutáveis

Tipos de sequências mutáveis também oferecem suporte aos seguintes métodos:

sequence.append(*value*, /)

Adiciona *value* ao final da sequência. Isso equivale a escrever `seq[len(seq):len(seq)] = [value]`.

sequence.clear()

Adicionado na versão 3.3.

Remove todos os itens de *sequence*. Isso equivale a escrever `del sequence[:]`.

sequence.copy()

Adicionado na versão 3.3.

Cria uma cópia rasa de *sequence*. Isso equivale a escrever `sequence[:]`.

Dica

O método **`copy()`** não faz parte de `MutableSequence` `ABC`, mas a maioria dos tipos de sequências mutáveis concretas o fornecem.

sequence.extend(*iterable*, /)

Estende *sequence* com o conteúdo de *iterable*. Em geral, isso equivale a escrever `seq[len(seq):len(seq)] = iterable`.

sequence.insert(*index*, *value*, /)

Insire *value* em *sequence* no *index* fornecido. Isso equivale a escrever `sequence[index:index] = [value]`.

sequence.pop(*index=-1*, *[/]*)

Recupera o item em *index* e também o remove de *sequence*. Por padrão, o último item em *sequence* é removido e retornado.

sequence.remove(*value*, *[/]*)

Remove o primeiro item de *sequence* onde `sequence[i] == value`.

Levanta `ValueError` se *value* não for encontrado em *sequence*.

sequence.reverse()

Inverte os itens da *sequence* no lugar. Este método mantém a economia de espaço ao inverter uma sequência grande. Para lembrar aos usuários que ele opera por efeito colateral, ele retorna `None`.

Listas

As listas são sequências mutáveis, normalmente usadas para armazenar coleções de itens homogêneos (onde o grau preciso de similaridade variará de acordo com a aplicação).

class list(*iterable=()*, *[/]*)

As listas podem ser construídas de várias maneiras:

- Usando um par de colchetes para denotar uma lista vazia: `[]`
- Usando colchetes, separando itens por vírgulas: `[a]`, `[a, b, c]`
- Usando uma compreensão de lista: `[x for x in iterable]`
- Usando o construtor de tipo: `list()` ou `list(iterable)`

O construtor produz uma lista cujos itens são iguais e na mesma ordem que os itens de *iterable*. *iterable* pode ser uma sequência, um contêiner que suporte iteração ou um objeto iterador. Se *iterable* já for uma lista, uma cópia será feita e retornada, semelhante a `iterable[:]`. Por exemplo, `list('abc')` retorna `['a', 'b', 'c']` e `list( (1, 2, 3) )` retorna `[1, 2, 3]`. Se nenhum argumento for dado, o construtor criará uma nova lista vazia `[]`.

Muitas outras operações também produzem listas, incluindo a função embutida `sorted()`.

Listas são genéricas em relação aos tipos dos seus itens.

Listas implementam todas as operações de sequências comuns e mutáveis. As listas também fornecem o seguinte método adicional:

sort(*, *key=None*, *reverse=False*)

Esse método classifica a lista in-place, usando apenas comparações < entre itens. As exceções não são suprimidas – se qualquer operação de comparação falhar, toda a operação de ordenação falhará (e a lista provavelmente será deixada em um estado parcialmente modificado).

`sort()` aceita 2 argumentos que só podem ser passados como argumentos somente-nomeados:

key especifica uma função de um argumento que é usado para extrair uma chave de comparação de cada elemento da lista (por exemplo, `key=str.lower`). A chave correspondente a cada item na lista é calculada uma vez e depois usada para todo o processo de classificação. O valor padrão `None` significa que os itens da lista são classificados diretamente sem calcular um valor de chave separado.

A função utilitária `functools.cmp_to_key()` está disponível para converter a função *cmp* no estilo 2.x para uma função *key*.

reverse é um valor booleano. Se definido igual a `True`, então os elementos da lista são classificados como se cada comparação estivesse invertida.

Este método modifica a sequência in-place para economizar espaço ao classificar uma sequência grande. Para lembrar aos usuários que os mesmos operam por efeito colateral, ele não retorna a sequência ordenada (utilize a função `sorted()` para solicitar explicitamente uma nova instância da lista ordenada).

O método `sort()` é garantido como sendo estável. Uma classificação é estável se ela garantir não alterar a ordem relativa de elementos que comparam igual – isso é útil para classificar em várias passagens (por exemplo, classificar por departamento, depois por nota salarial).

Para exemplos de classificação e um breve tutorial de classificação, veja Técnicas de ordenação.

No momento em que uma lista está sendo ordenada, o efeito de tentar alterar, ou mesmo inspecionar, a lista é indefinida. A implementação C do Python faz com que a lista apareça vazia durante o tempo de processamento, e levanta a exceção `ValueError` se puder detectar que a lista foi alterada durante uma ordenação.

Ver também

Para informações detalhadas sobre as garantias de segurança de thread para objetos `list`, consulte Segurança de threads para objetos lista.

Tuplas

Tuplas são sequências imutáveis, tipicamente usadas para armazenar coleções de dados heterogêneos (como as tuplas de 2 elementos produzidas pelo função embutida `enumerate()`). Tuplas também são usadas para casos em que seja necessária uma sequência imutável de dados homogêneos (como permitir o armazenamento em uma instância `set` ou `dict`).

class tuple(*iterable=(), /*)

As tuplas podem ser construídas de várias maneiras:

- Usando um par de parênteses para denotar a tupla vazia: `()`
- Usando uma vírgula à direita para uma tupla singleton: `a,` ou `(a,)`
- Separando os itens com vírgulas: `a, b, c` ou `(a, b, c)`
- Usando a função embutida `tuple()`: `tuple()` ou `tuple(iterable)`

O construtor constrói uma tupla cujos itens são iguais e na mesma ordem dos itens de *iterable*. *iterable* pode ser uma sequência, um contêiner que suporta iteração ou um objeto iterador. Se *iterable* já for uma tupla, este será retornado inalterado. Por exemplo, `tuple('abc')` retorna `('a', 'b', 'c')` e `tuple( [1, 2, 3] )` retorna `(1, 2, 3)`. Se nenhum argumento for dado, o construtor criará uma tupla vazia, `()`.

Observe que, na verdade, é a vírgula que faz uma tupla, e não os parênteses. Os parênteses são opcionais, exceto no caso de tupla vazia, ou quando são necessários para evitar ambiguidades sintáticas. Por exemplo, `f(a, b, c)` é uma chamada da função com três argumentos, enquanto que `f((a, b, c))` é uma chamada de função com uma tupla de 3 elementos com um único argumento.

As tuplas implementam todas as operações comuns de sequência.

Tuplas são genéricas em relação aos tipos de seu conteúdo. Para mais informações, consulte a documentação de tipagem sobre a anotação de tuplas.

Para coleções heterogêneas de dados onde o acesso pelo nome é mais claro do que o acesso pelo índice, `collections.namedtuple()` pode ser uma escolha mais

apropriada do que um objeto tupla simples.

Intervalos

O tipo `range` representa uma sequência imutável de números e é comumente usado para percorrer um número determinado de vezes em um laço `for`.

class range(*stop*, /)
class range(*start*, *stop*, *step=1*, /)

Os argumentos para o construtor de intervalo devem ser inteiros (`int` embutido ou qualquer objeto que implemente o método especial `__index__()`). Se o argumento *step* for omitido, será usado o padrão `1`. Se o argumento *start* for omitido, será usado o padrão `0`. Se *step* for zero, uma exceção `ValueError` será levantada.

Para um *step* positivo, o conteúdo de um intervalo `r` será determinado pela fórmula `r[i] = start + step*i` onde `i >= 0` e `r[i] < stop`.

Para um *step* negativo, o conteúdo do intervalo ainda será determinado pela fórmula `r[i] = start + step*i`, mas as restrições serão `i >= 0` e `r[i] > stop`.

Um objeto intervalo estará vazio se `r[0]` não atender à restrição de valor. Intervalos suportam índices negativos, mas estes são interpretados como indexadores partindo do final da sequência determinada pelos índices positivos.

Intervalos contendo valores absolutos maiores que `sys.maxsize` são permitidos, mas alguns recursos (como `len()`) podem levantar `OverflowError`.

Exemplos de intervalos:

```
>>> list(range(10))
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(range(1, 11))
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> list(range(0, 30, 5))
[0, 5, 10, 15, 20, 25]
>>> list(range(0, 10, 3))
[0, 3, 6, 9]
>>> list(range(0, -10, -1))
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
>>> list(range(0))
[]
```

```
>>> list(range(1, 0))
[]
```

Ranges implementam todas as operações comuns de sequência, exceto a concatenação e a repetição (devido ao fato de que os objetos intervalos só podem representar sequências que seguem um padrão rígido. e a repetição e a concatenação geralmente vão violar esse padrão).

start
: O valor do parâmetro *start* (ou `0` se o parâmetro não for fornecido)

stop
: O valor do parâmetro *stop*

step
: O valor do parâmetro *step* (ou `1` se o parâmetro não for fornecido)

A vantagem do tipo **`range`** sobre um **`list`** ou **`tuple`** regular é que um objeto **`range`** sempre terá a mesma quantidade (pequena) de memória, não importa o tamanho do intervalo o mesmo esteja representando (como ele apenas armazena os valores `start`, `stop` e `step`, calculando itens individuais e subintervalos conforme necessário).

Objetos intervalos implementam a ABC **`collections.abc.Sequence`**, e fornecem recursos como testes de contenção, pesquisa de índice de elemento, fatiamento e suporte a índices negativos (veja Tipos sequências — list, tuple, range):

```
>>> r = range(0, 20, 2)
>>> r
range(0, 20, 2)
>>> 11 in r
False
>>> 10 in r
True
>>> r.index(10)
5
>>> r[5]
10
>>> r[:5]
range(0, 10, 2)
>>> r[-1]
18
```

Testar objetos intervalos por igualdade com `==` e `!=` os compara como sequências. Ou seja, dois objetos intervalos são considerados iguais se representarem a mesma sequência de valores. (Observe que dois objetos intervalos considerados iguais podem ter diferentes atributos `start`, `stop` e `step`, por exemplo `range(0) == range(2, 1, 3)` ou `range(0, 3, 2) == range(0, 4, 2)`.)

Alterado na versão 3.2: Implementa a ABC Sequence. Suporte a fatiamento e a índices negativos. Testa objetos `int` para associação em tempo constante em vez de iterar através de todos os itens.

Alterado na versão 3.3: Define '==' e '!=' para comparar objetos intervalos com base na sequência de valores que eles definem (em vez de comparar com base na identidade do objeto).

Adicionados os atributos `start`, `stop` e `step`.

Ver também

- A receita de linspace [https://code.activestate.com/recipes/579000-equally-spaced-numbers-linspace/] mostra como implementar uma versão preguiçosa de um intervalo adequado para aplicações de ponto flutuante.

Resumo dos métodos de tipo de sequência binária e de texto

A tabela a seguir resume os métodos de tipos de sequência binária e de texto por categoria.

Categoria	Métodos `str`	Métodos `bytes` e `bytearray`
Formatação	`str.format()`	
	`str.format_map()`	
	Literais de strings formatadas	

Categoria	Métodos `str`	Métodos `bytes` e `bytearray`
	Formatação de string no estilo printf	Estilo de Formatação de bytes printf-style
Pesquisa e substituição	`str.find()` `str.rfind()`	`bytes.find()` `bytes.rfind()`
	`str.index()` `str.rindex()`	`bytes.index()` `bytes.rindex()`
	`str.startswith()`	`bytes.startswith()`
	`str.endswith()`	`bytes.endswith()`
	`str.count()`	`bytes.count()`
	`str.replace()`	`bytes.replace()`
Divisão e união	`str.split()` `str.rsplit()`	`bytes.split()` `bytes.rsplit()`
	`str.splitlines()`	`bytes.splitlines()`
	`str.partition()`	`bytes.partition()`
	`str.rpartition()`	`bytes.rpartition()`
	`str.join()`	`bytes.join()`
String Classification	`str.isalpha()`	`bytes.isalpha()`

Categoria	Métodos `str`	Métodos `bytes` e `bytearray`
	`str.isdecimal()`	
	`str.isdigit()`	`bytes.isdigit()`
	`str.isnumeric()`	
	`str.isalnum()`	`bytes.isalnum()`
	`str.isidentifier()`	
	`str.islower()`	`bytes.islower()`
	`str.isupper()`	`bytes.isupper()`
	`str.istitle()`	`bytes.istitle()`
	`str.isspace()`	`bytes.isspace()`
	`str.isprintable()`	
Case Manipulation	`str.lower()`	`bytes.lower()`
	`str.upper()`	`bytes.upper()`
	`str.casefold()`	

Categoria	Métodos `str`	Métodos `bytes` e `bytearray`
	`str.capitalize()`	`bytes.capitalize()`
	`str.title()`	`bytes.title()`
	`str.swapcase()`	`bytes.swapcase()`
Preenchimento e remoção	`str.ljust()` `str.rjust()`	`bytes.ljust()` `bytes.rjust()`
	`str.center()`	`bytes.center()`
	`str.expandtabs()`	`bytes.expandtabs()`
	`str.strip()`	`bytes.strip()`
	`str.lstrip()` `str.rstrip()`	`bytes.lstrip()` `bytes.rstrip()`
	`str.removeprefix()`	`bytes.removeprefix()`
	`str.removesuffix()`	`bytes.removesuffix()`
Tradução e codificação	`str.translate()`	`bytes.translate()`
	`str.maketrans()`	`bytes.maketrans()`
	`str.encode()`	

Categoria	Métodos `str`	Métodos `bytes` e `bytearray`
		`bytes.decode()`

Tipo sequência de texto — `str`

Os dados textuais em Python são tratados com objetos `str`, ou *strings*. Strings são sequências imutáveis de pontos de código Unicode. As strings literais são escritas de diversas maneiras:

- Aspas simples: `'permitem aspas "duplas" internas'`
- Aspas duplas: `"permitem aspas 'simples' internas"`
- Aspas triplas: `'''Três aspas simples'''`, `"""Três aspas duplas"""`

Strings de aspas triplas podem expandir por várias linhas – todos os espaços em branco associados serão incluídos em literal string.

Os literais strings que fazem parte de uma única expressão e têm apenas espaços em branco entre eles serão implicitamente convertidos em um único literal string. Isso é, `("spam " "eggs") == "spam eggs"`.

Veja Literais de string e bytes para mais informações sobre as várias formas de literal de string, incluindo o suporte a sequêncis de escape, e o prefixo `r` (“raw”) que desabilita a maioria dos processos de sequência de escape.

As strings também podem ser criadas a partir de outros objetos usando o construtor `str`.

Uma vez que não há nenhum tipo de “caractere” separador, indexar uma string produz strings de comprimento 1. Ou seja, para uma string não vazia *s*, `s[0] == s[0:1]`.

Também não existe um tipo string mutável, mas o método `str.join()` ou a classe `io.StringIO` podem ser usados para construir strings de forma eficiente a partir de vários partes distintas.

Alterado na versão 3.3: Para a compatibilidade com versões anteriores do Python 2, o prefixo `u` foi mais uma vez permitido em literais strings. Não possui quaisquer

efeito sobre o significado de literais strings e não pode ser combinado com o prefixo `r`.

class str(*, *encoding='utf-8'*, *errors='strict'*)
class str(*object*)
class str(*object*, *encoding*, *errors='strict'*)
class str(*object*, *, *errors*)

Retorna uma versão string de *object*. Se *object* não é fornecido, retorna uma string vazia. Caso contrário, o comportamento de `str()` dependerá se o *encoding* ou *errors* são fornecidos, da seguinte forma.

Se nem *encoding* nem *errors* for fornecido, `str(object)` retornará **`type(object).__str__(object)`**, que é a representação da string "informal" ou que pode ser facilmente imprimível de *object*. Para objetos string, esta é a própria string. Se *object* não tiver um método:**`__str__()`**, então a função **`str()`** retornará **`repr(object)`**.

Se pelo menos um de *encoding* ou *errors* for fornecido, *object* deve ser um objeto bytes ou similar (por exemplo, **`bytes`** ou **`bytearray`**). Nesse caso, se *object* for um objeto **`bytes`** (ou **`bytearray`**), então `str(bytes, encoding, errors)` equivale a **`bytes.decode(encoding, errors)`**. Caso contrário, o objeto bytes subjacente ao objeto buffer é obtido antes de chamar **`bytes.decode()`**. Veja Tipos de sequência binária — bytes, bytearray, memoryview e Protocolo de buffer para obter informações sobre objetos buffer.

Passando um objeto **`bytes`** para **`str()`** sem os argumentos *encoding* ou *errors* se enquadra no primeiro caso de retornar a representação informal de strings (consulte também a opção de linha de comando **`-b`** para Python). Por exemplo:

```
>>> str(b'Zoot!')
"b'Zoot!'"
```

Para mais informações sobre a classe `str` e seus métodos, veja Tipo sequência de texto — str e a seção String Methods abaixo. Para gerar strings formatadas, veja as seções Literais de strings formatadas e Sintaxe das strings de formato. Além disso, veja a seção Serviços de Processamento de Texto.

String Methods

Strings implementam todas as operações comuns de sequências, juntamente com os métodos adicionais descritos abaixo.

Strings também possuem suporte para duas formas de formatação de string, uma fornecendo uma ampla gama de flexibilidade e customização (veja **`str.format()`**, Sintaxe das strings de formato e Formatação personalizada de strings) e a outra baseada no estilo de formatação `printf` da linguagem C, que lida com uma gama menor de tipos e é levemente mais difícil de utilizar corretamente, mas é frequentemente mais rápida para os casos na qual ela consegue manipular (Formatação de string no estilo printf).

A seção Serviços de Processamento de Texto da biblioteca padrão cobre um número de diversos outros módulos que fornecem vários utilitários relacionados a texto (incluindo suporte a expressões regulares no módulo **`re`**).

str.capitalize()

Retorna uma cópia da string com o seu primeiro caractere em maiúsculo e o restante em minúsculo.

Alterado na versão 3.8: O primeiro caractere agora é colocado em *titlecase* ao invés de letras maiúsculas. Isso significa que caracteres como dígrafos apenas terão sua primeira letra alterada para maiúscula, ao invés de todos os caracteres.

str.casefold()

Retorna uma cópia da string em *casefolded*. Strings em *casefold* podem ser usadas para corresponder letras sem importar se são minúsculas ou maiúsculas.

Casefolding é similar a mudar para letras minúsculas, mas mais agressivo porque destina-se a remover todas as diferenças maiúsculas/minúsculas em uma string. Por exemplo, a letra minúscula alemã `'ß'` equivale a `"ss"`. Como ela já é uma minúscula, o método **`lower()`** não irá fazer nada para `'ß'`; já o método **`casefold()`** converte a letra para `"ss"`. Por exemplo:

```
>>> 'straße'.lower()
'straße'
>>> 'straße'.casefold()
'strasse'
```

O algoritmo *casefolding* está descrito na seção 3.13.3 ‘Default Case Folding’ do Padrão Unicode [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G53253].

Adicionado na versão 3.3.

str.center(*width*, *fillchar=' '*, /)

Retorna um texto centralizado em uma string de comprimento *width*. Preenchimento é feito usando o parâmetro *fillchar* especificado (padrão é o caractere de espaço ASCII). A string original é retornada se *width* é menor ou igual que `len(s)`. Por exemplo:

```
>>> 'Python'.center(10)
'  Python  '
>>> 'Python'.center(10, '-')
'--Python--'
>>> 'Python'.center(4)
'Python'
```

str.count(*sub*[, *start*[, *end*]])

Retorna o número de ocorrências da sub-string *sub* que não se sobrepõem no intervalo [*start*, *end*]. Argumentos opcionais *start* e *end* são interpretados como na notação de fatias.

Se *sub* estiver vazio, retorna o número de strings vazias entre os caracteres, que é o comprimento da string mais um. Por exemplo:

```
>>> 'spam, spam, spam'.count('spam')
3
>>> 'spam, spam, spam'.count('spam', 5)
2
>>> 'spam, spam, spam'.count('spam', 5, 10)
1
>>> 'spam, spam, spam'.count('eggs')
0
>>> 'spam, spam, spam'.count('')
17
```

str.encode(*encoding='utf-8'*, *errors='strict'*)

Retorna a string codificada para **`bytes`**.

encoding tem como padrão `'utf-8'`; veja Standard Encodings para valores possíveis.

errors controla como os erros de codificação são tratados. Se `'strict'` (o padrão), uma exceção **`UnicodeError`** é levantada. Outros valores possíveis são `'ignore'`, `'replace'`, `'xmlcharrefreplace'`, `'backslashreplace'` e qualquer outro nome registrado via **`codecs.register_error()`**. Veja Error Handlers para detalhes.

Por motivos de desempenho, o valor de *errors* não é verificado quanto à validade, a menos que um erro de codificação realmente ocorra, Modo de Desenvolvimento do Python esteja ativado ou uma construção de depuração seja usada. Por exemplo:

```
>>> encoded_str_to_bytes = 'Python'.encode()
>>> type(encoded_str_to_bytes)
<class 'bytes'>
>>> encoded_str_to_bytes
b'Python'
```

Alterado na versão 3.1: Adicionado suporte para argumentos nomeados.

Alterado na versão 3.9: O valor do argumento *errors* agora é verificado em Modo de Desenvolvimento do Python e no modo de depuração.

str.endswith(*suffix*[, *start*[, *end*]])

Retorna `True` se a string terminar com o *suffix* especificado, caso contrário retorna `False`. *suffix* também pode ser uma tupla de sufixos para procurar. Com o parâmetro opcional *start*, começamos a testar a partir daquela posição. Com o parâmetro opcional *end*, devemos parar de comparar na posição especificada. Usar *start* e *end* equivale a `str[start:end].endswith(suffix)`. Por exemplo:

```
>>> 'Python'.endswith('on')
True
>>> 'a tuple of suffixes'.endswith(('at', 'in'))
False
>>> 'a tuple of suffixes'.endswith(('at', 'es'))
True
>>> 'Python is amazing'.endswith('is', 0, 9)
True
```

Veja também **`startswith()`** e **`removesuffix()`**.

str.expandtabs(*tabsize=8*)

Retorna uma cópia da string onde todos os caracteres de tabulação são substituídos por um ou mais espaços, dependendo da coluna atual e do tamanho fornecido para a tabulação. Posições de tabulação ocorrem a cada *tabsize* caracteres (o padrão é 8, dada as posições de tabulação nas colunas 0, 8, 16 e assim por diante). Para expandir a string, a coluna atual é definida como zero e a string é examinada caractere por caractere. Se o caractere é uma tabulação (`\t`), um ou mais caracteres de espaço são inseridos no resultado até que a coluna atual seja igual a próxima posição de tabulação. (O caractere de tabulação em si

não é copiado.) Se o caractere é um caractere de nova linha (`\n`) ou de retorno (`\r`), ele é copiado e a coluna atual é redefinida para zero. Qualquer outro caractere é copiado sem ser modificado e a coluna atual é incrementada em uma unidade independentemente de como o caractere é representado quando é impresso. Por exemplo:

```
>>> '01\t012\t0123\t01234'.expandtabs()
'01      012     0123    01234'
>>> '01\t012\t0123\t01234'.expandtabs(4)
'01  012 0123    01234'
>>> print('01\t012\n0123\t01234'.expandtabs(4))
01  012
0123    01234
```

str.find(*sub*[, *start*[, *end*]])

Retorna o índice mais baixo na string onde a substring *sub* é encontrado dentro da fatia `s[start:end]`. Argumentos opcionais como *start* e *end* são interpretados como na notação de fatiamento. Retorna `-1` se *sub* não for localizado. Por exemplo:

```
>>> 'spam, spam, spam'.find('sp')
0
>>> 'spam, spam, spam'.find('sp', 5)
6
```

Veja também **`rfind()`** e **`index()`**.

Nota

O método **`find()`** deve ser usado apenas se precisarmos conhecer a posição de *sub*. Para verificar se *sub* é ou não uma substring, use o operador **`in`**:

```
>>> 'Py' in 'Python'
True
```

str.format(**args*, ***kwargs*)

Executa uma operação de formatação de string. A string na qual este método é chamado pode conter texto literal ou campos para substituição delimitados por chaves `{}`. Cada campo de substituição contém um índice numérico de um argumento posicional ou o nome de um argumento nomeado. Retorna a cópia da string onde cada campo para substituição é substituído com o valor da string do argumento correspondente. Por exemplo:

```
>>> "A soma de 1 + 2 é {0}".format(1+2)
'A soma de 1 + 2 é 3'
>>> "A soma de {a} + {b} é {answer}".format(answer=1+2, a=1, b=2)
'A soma de 1 + 2 é 3'
>>> "{1} espera a inquisição {0}!".format("espanhola", "Ninguém")
'Ninguém espera a inquisição espanhola!'
```

Veja Sintaxe das strings de formato para uma descrição das várias opções de formatação que podem ser especificadas em uma strings de formato.

Nota

Ao formatar um número (`int`, `float`, `complex`, `decimal.Decimal` e sub-classes) com o tipo `n` (ex: `'{:n}'.format(1234)`), a função define temporariamente a localidade `LC_CTYPE` para a localidade `LC_NUMERIC` a fim de decodificar campos `decimal_point` e `thousands_sep` de **`localeconv()`** se eles são caracteres não-ASCII ou maiores que 1 byte, e a localidade `LC_NUMERIC` é diferente da localidade `LC_CTYPE`. Esta mudança temporária afeta outras threads.

Alterado na versão 3.7: Ao formatar um número com o tipo `n`, a função define temporariamente a localidade `LC_CTYPE` para a localidade `LC_NUMERIC` em alguns casos.

str.format_map(*mapping*, /)

Semelhante a `str.format(**mapping)`, exceto pelo fato de que `mapping` é usado diretamente e não copiado para uma classe `dict`. Isso é útil se, por exemplo, `mapping` é uma subclasse de dict:

```
>>> class Default(dict):
...     def __missing__(self, key):
...         return key
...
>>> '{name} was born in {country}'.format_map(Default(name='Guido'))
'Guido was born in country'
```

Adicionado na versão 3.2.

str.index(*sub*[, *start*[, *end*]])

Semelhante a `find()`, mas levanta `ValueError` quando a substring *sub* não é encontrada. Por exemplo:

```
>>> 'spam, spam, spam'.index('spam')
0
>>> 'spam, spam, spam'.index('eggs')
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    'spam, spam, spam'.index('eggs')
    ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
ValueError: substring not found
```

Veja também **`rindex()`**.

str.isalnum()

Retorna `True` se todos os caracteres na string são alfanuméricos e existe pelo menos um caractere, ou `False` caso contrário. Um caractere `c` é alfanumérico se um dos seguintes retorna `True`: `c.isalpha()`, `c.isdecimal()`, `c.isdigit()`, ou `c.isnumeric()`. Por exemplo:

```
>>> 'abc123'.isalnum()
True
>>> 'abc123!@#'.isalnum()
False
>>> ''.isalnum()
False
>>> ' '.isalnum()
False
```

str.isalpha()

Retorna `True` se todos os caracteres na string são alfabéticos e existe pelo menos um caractere, `False` caso contrário. Caracteres alfabéticos são aqueles caracteres definidos na base de dados de caracteres Unicode como “Letra”, isto é, aqueles cuja propriedade na categoria geral é um destes: “Lm”, “Lt”, “Lu”, “Ll” ou “Lo”. Perceba que isso é diferente da propriedade Alfabética definida na seção 4.10 “Letters, Alphabetic, and Ideographic” do Padrão Unicode [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-4/#G91002]. Por exemplo:

```
>>> 'Letras e espaços'.isalpha()
False
>>> 'LetrasApenas'.isalpha()
True
>>> 'µ'.isalpha()  # Caracteres não-ASCII podem ser considerados alf
True
```

Consulte Propriedades Unicode.

str.isascii()

Retorna `True` se a string é vazia ou se todos os caracteres na string são ASCII, `False` caso contrário. Caracteres ASCII têm pontos de código no intervalo U+0000-U+007F. Por exemplo:

```
>>> 'Caracteres ASCII'.isascii()
True
>>> 'µ'.isascii()
False
```

Adicionado na versão 3.7.

str.isdecimal()

Retorna `True` se todos os caracteres na string são caracteres decimais e existe pelo menos um caractere, `False` caso contrário. Caracteres decimais são aqueles que podem ser usados para formar números na base 10, tal como U+0660, ou dígito zero para arábico-índico. Formalmente, um caractere decimal é um caractere em Unicode cuja categoria geral é “Nd”. Por exemplo:

```
>>> '0123456789'.isdecimal()
True
>>> '٠١٢٣٤٥٦٧٨٩'.isdecimal()  # Dígitos árabe-índico zero até nove
True
>>> 'alfabético'.isdecimal()
False
```

str.isdigit()

Retorna `True` se todos os caracteres na string são dígitos e existe pelo menos um caractere, `False` caso contrário. Dígitos incluem caracteres decimais e dígitos que precisam de tratamento especial, tal como a compatibilidade com dígitos sobre-escritos. Isso inclui dígitos que não podem ser usados para formar números na base 10, como por exemplo os *números de Kharosthi <https://en.wikipedia.org/wiki/Kharosthi#Numerals>`__*. Formalmente, um dígito é um caractere que tem a propriedade com valor Numeric_Type=Digit ou Numeric_Type=Decimal.

Por exemplo:

```
>>> '0123456789'.isdigit()
True
>>> '٠١٢٣٤٥٦٧٨٩'.isdigit()  # Algarismos arábicos orientais de zero a
True
>>> '⅕'.isdigit()  # um quinto em fração ordinária
False
```

```
>>> '²'.isdecimal(), '²'.isdigit(),  '²'.isnumeric()
(False, True, True)
```

Veja também **isdecimal()** e **isnumeric()**.

str.isidentifier()

Retorna `True` se a string é um identificador válido conforme a definição da linguagem, seção Nomes (identificadores e palavras reservadas).

keyword.iskeyword() pode ser usada para testar se a string `s` é uma palavra reservada, tal como **def** e **class**.

Exemplo:

```
>>> from keyword import iskeyword

>>> 'olá.isidentifier(), iskeyword('olá')
(True, False)
>>> 'def'.isidentifier(), iskeyword('def')
(True, True)
```

str.islower()

Retorna `True` se todos os caracteres em caixa (que possuem maiúsculo e minúsculo) [[4]] na string são minúsculos e existe pelo menos um caractere em caixa, `False` caso contrário.

str.isnumeric()

Retorna `True` se todos os caracteres na string são caracteres numéricos, e existe pelo menos um caractere, `False` caso contrário. Caracteres numéricos incluem dígitos, e todos os caracteres que têm a propriedade com valor numérico Unicode, isto é: U+2155, um quinto de fração vulgar. Formalmente, caracteres numéricos são aqueles que possuem propriedades com valor Numeric_Type=Digit, Numeric_Type=Decimal ou Numeric_Type=Numeric. Por exemplo:

```
>>> '0123456789'.isnumeric()
True
>>> '٠١٢٣٤٥٦٧٨٩'.isnumeric()  # Algarismos arábicos orientais de zero
True
>>> '⅕'.isnumeric()  # um quinto em fração ordinária
True
>>> '²'.isdecimal(), '²'.isdigit(),  '²'.isnumeric()
(False, True, True)
```

Veja também `isdecimal()` e `isdigit()`.

str.isprintable()

Retorna `True` se todos os caracteres na string forem imprimíveis, `False` se contiver pelo menos um caractere não imprimível.

Aqui, “imprimíveis” significa que o caractere é adequado para `repr()` usar em sua saída; “não imprimível” significa que **`repr()`** em tipos embutidos fará o escape hexadecimal do caractere. Não tem relação com o tratamento de strings escritas em `sys.stdout` ou `sys.stderr`.

Os caracteres imprimíveis são aqueles que no banco de dados de caracteres Unicode (veja `unicodedata`) têm uma categoria geral no grupo Letra, Marca, Número, Pontuação ou Símbolo (L, M, N, P ou S); mais o espaço ASCII 0x20. Caracteres não imprimíveis são aqueles no grupo Separador ou Outro (Z ou C), exceto o espaço ASCII.

Por exemplo:

```
>>> ''.isprintable(), ' '.isprintable()
(True, True)
>>> '\t'.isprintable(), '\n'.isprintable()
(False, False)
```

Veja também `isspace()`.

str.isspace()

Retorna `True` se existem apenas caracteres de espaço em branco na string e existe pelo menos um caractere, `False` caso contrário.

Por exemplo:

```
>>> ''.isspace()
False
>>> ' '.isspace()
True
>>> '\t\n'.isspace() # TAB e QUEBRA DE LINHA
True
>>> '\u3000'.isspace() # ESPAÇO IDEOGRÁFICO
True
```

Um caractere é *espaço em branco* se no banco de dados de caracteres Unicode (veja `unicodedata`), ou pertence a categoria geral `Zs` (“Separador, espaço”), ou

sua classe bidirecional é `WS`, `B` ou `S`.

Veja também **`isprintable()`**.

str.istitle()

Retorna `True` se a string é titlecased e existe pelo menos um caractere, por exemplo caracteres maiúsculos somente podem proceder caracteres que não diferenciam maiúsculas/minúsculas, e caracteres minúsculos somente podem proceder caracteres que permitem ambos. Retorna `False` caso contrário.

Por exemplo:

```
>>> 'Spam, Spam, Spam'.istitle()
True
>>> 'spam, spam, spam'.istitle()
False
>>> 'SPAM, SPAM, SPAM'.istitle()
False
```

Veja também **`title()`**.

str.isupper()

Retorna `True` se todos os caracteres que permitem maiúsculas ou minúsculas [[4]] na string estão com letras maiúsculas, e existe pelo menos um caractere maiúsculo, `False` caso contrário.

```
>>> 'BANANA'.isupper()
True
>>> 'banana'.isupper()
False
>>> 'baNana'.isupper()
False
>>> ' '.isupper()
False
```

str.join(*iterable*, /)

Retorna a string que é a concatenação das strings no *iterável*. Uma **`TypeError`** será levantada se existirem quaisquer valores que não sejam strings no *iterável*, incluindo objetos **`bytes`**. O separador entre elementos é a string que está fornecendo este método. Por exemplo:

```
>>> ', '.join(['spam', 'spam', 'spam'])
'spam, spam, spam'
```

```
>>> '-'.join('Python')
'P-y-t-h-o-n'
```

Veja também **`split()`**.

str.ljust(*width*, *fillchar=' '*, /)

Retorna a string alinhada a esquerda em uma string de comprimento *width*. Preenchimento é feito usando *fillchar* que for especificado (o padrão é o caractere ASCII de espaço). A string original é retornada se *width* é menor ou igual que `len(s)`.

Por exemplo:

```
>>> 'Python'.ljust(10)
'Python    '
>>> 'Python'.ljust(10, '.')
'Python....'
>>> 'Monty Python'.ljust(10, '.')
'Monty Python'
```

Veja também **`rjust()`**

str.lower()

Retorna uma cópia da string com todos os caracteres que permitem maiúsculo E minúsculo [[4]] convertidos para letras minúsculas. Por exemplo:

```
>>> 'Exemplo do Método Lower'.lower()
'exemplo do método lower'
```

O algoritmo de letras minúsculas está descrito na seção 3.13.2 ‘Default Case Folding’ do Padrão Unicode [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G34078].

str.lstrip(*chars=None*, /)

Retorna uma cópia da string com caracteres iniciais removidos. O argumento *chars* é uma string que especifica o conjunto de caracteres a serem removidos. Se for omitido ou se for `None`, o argumento *chars* será considerado como espaço em branco por padrão para a remoção. O argumento *chars* não é um prefixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> '   spacious   '.lstrip()
'spacious   '
>>> 'www.example.com'.lstrip('cmowz.')
'example.com'
```

Consulte `str.removeprefix()` para um método que removerá uma única string de prefixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> 'Arthur: three!'.lstrip('Arthur: ')
'ee!'
>>> 'Arthur: three!'.removeprefix('Arthur: ')
'three!'
```

static str.maketrans(*dict*, /)

static str.maketrans(*from*, *to*, *remove=''*, /)

Este método estático retorna uma tabela de tradução usável para `str.translate()`.

Se existe apenas um argumento, ele deve ser um dicionário mapeando números Unicode (inteiros) ou caracteres (strings de comprimento 1) para números Unicode, strings (de comprimento arbitrário) ou `None`. Caracteres chave serão então convertidos para números ordinais.

Se existirem dois argumentos, eles devem ser strings de igual comprimento, e no dicionário resultante, cada caractere em *from* será mapeado para o caractere na mesma posição em *to*. Se existir um terceiro argumento, ele deve ser uma string, cujos caracteres serão mapeados para `None` no resultado.

Alterado na versão 3.15: dict agora pode ser um `frozendict`.

str.partition(*sep*, /)

Quebra a string na primeira ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo a string, seguido de duas strings vazias.

Por exemplo:

```
>>> 'Monty Python'.partition(' ')
('Monty', ' ', 'Python')
>>> "Monty Python's Flying Circus".partition(' ')
('Monty', ' ', "Python's Flying Circus")
>>> 'Monty Python'.partition('-')
('Monty Python', '', '')
```

Veja também `rpartition()`.

str.removeprefix(*prefix*, /)

Se a string começar com a string *prefix*, retorna `string[len(prefix):]`. Caso contrário, retorna uma cópia da string original:

```
>>> 'TestHook'.removeprefix('Test')
'Hook'
>>> 'BaseTestCase'.removeprefix('Test')
'BaseTestCase'
```

Adicionado na versão 3.9.

Veja também **`removesuffix()`** e **`startswith()`**.

str.removesuffix(*suffix*, *[,]*)

Se a string terminar com a string *suffix* e a *suffix* não estiver vazia, retorna `string[:-len(suffix)]`. Caso contrário, retorna uma cópia da string original:

```
>>> 'MiscTests'.removesuffix('Tests')
'Misc'
>>> 'TmpDirMixin'.removesuffix('Tests')
'TmpDirMixin'
```

Adicionado na versão 3.9.

Veja também **`removeprefix()`** e **`endswith()`**.

str.replace(*old*, *new*, *[,]*, *count=-1*)

Retorna uma cópia da string com todas as ocorrências da substring *old* substituídas por *new*. Se o argumento opcional *count* é fornecido, apenas as primeiras *count* ocorrências são substituídas. Se *count* não é especificado ou é igual a `-1`, então todas as ocorrências serão substituídas. Por exemplo:

```
>>> 'spam, spam, spam'.replace('spam', 'eggs')
'eggs, eggs, eggs'
>>> 'spam, spam, spam'.replace('spam', 'eggs', 1)
'eggs, spam, spam'
```

Alterado na versão 3.13: count agora é suportado como um argumento nomeado.

str.rfind(*sub*[, *start*[, *end*]])

Retorna o maior índice onde a substring *sub* foi encontrada dentro da string, onde *sub* está contida dentro do intervalo `s[start:end]`. Argumentos opcionais *start* e *end* são interpretados usando a notação slice. Retorna `-1` em caso de falha. Por exemplo:

```
>>> 'spam, spam, spam'.rfind('sp')
12
>>> 'spam, spam, spam'.rfind('sp', 0, 10)
6
```

Veja também **find()** e **rindex()**.

str.rindex(*sub*[, *start*[, *end*]])

Semelhante a **rfind()**, mas levanta **ValueError** quando a substring *sub* não é encontrada. Por exemplo:

```
>>> 'spam, spam, spam'.rindex('spam')
12
>>> 'spam, spam, spam'.rindex('eggs')
Traceback (most recent call last):
  File "<stdin-0>", line 1, in <module>
    'spam, spam, spam'.rindex('eggs')
    ~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^^^
ValueError: substring not found
```

Veja também **index()** e **find()**.

str.rjust(*width*, *fillchar=' '*, /)

Retorna a string alinhada à direita em uma string de comprimento *width*. Preenchimento é feito usando o caractere *fillchar* especificado (o padrão é um caractere de espaço ASCII). A string original é retornada se *width* é menor que ou igual a `len(s)`.

Por exemplo:

```
>>> 'Python'.rjust(10)
'    Python'
>>> 'Python'.rjust(10, '.')
'....Python'
>>> 'Monty Python'.rjust(10, '.')
'Monty Python'
```

Veja também **ljust()** e **zfill()**.

str.rpartition(*sep*, /)

Quebra a string na última ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo duas strings vazias, seguido da própria string original.

Por exemplo:

```
>>> 'Monty Python'.rpartition(' ')
('Monty', ' ', 'Python')
>>> "Monty Python's Flying Circus".rpartition(' ')
("Monty Python's Flying", ' ', 'Circus')
>>> 'Monty Python'.rpartition('-')
('', '', 'Monty Python')
```

Veja também **partition()**.

str.rsplit(*sep=None*, *maxsplit=-1*)

Retorna uma lista de palavras na string, usando *sep* como a string delimitadora. Se *maxsplit* é fornecido, no máximo *maxsplit* cortes são feitos, sendo estes mais *à direita*. Se *sep* não foi especificado ou `None` foi informado, qualquer string de **espaço em branco** é um separador. Exceto pelo fato de separar pela direita, **rsplit()** se comporta como **split()**, o qual é descrito em detalhes abaixo.

str.rstrip(*chars=None*, /)

Retorna uma cópia da string com caracteres no final removidos. O argumento *chars* é uma string que especifica o conjunto de caracteres para serem removidos. Se omitidos ou tiver o valor `None`, o argumento *chars* considera como padrão a remoção dos espaços em branco. O argumento *chars* não é um sufixo; ao invés disso, todas as combinações dos seus valores são removidas. Por exemplo:

```
>>> '   spacious   '.rstrip()
'   spacious'
>>> 'mississippi'.rstrip('ipz')
'mississ'
```

Consulte **removesuffix()** para um método que removerá uma única string de sufixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> 'Monty Python'.rstrip(' Python')
'M'
>>> 'Monty Python'.removesuffix(' Python')
'Monty'
```

Veja também **strip()**.

str.split(*sep=None*, *maxsplit=-1*)

Retorna uma lista de palavras na string, usando *sep* como a string delimitadora. Se *maxsplit* é fornecido, no máximo *maxsplit* cortes são feitos (portando, a lista

terá no máximo `maxsplit+1` elementos). Se *maxsplit* não foi especificado ou `-1` foi informado, então não existe limite no número de cortes (todos os cortes possíveis são realizados).

Se *sep* é fornecido, delimitadores consecutivos não são agrupados juntos e eles são destinados para delimitar strings vazias (por exemplo, `'1,,2'.split(',')` retorna `['1', '', '2']`). O argumento *sep* pode consistir em vários caracteres como um único delimitador (para separar com vários delimitadores, use **`re.split()`**). Separar uma string vazia com um separador especificado retorna `['']`.

Por exemplo:

```
>>> '1,2,3'.split(',')
['1', '2', '3']
>>> '1,2,3'.split(',', maxsplit=1)
['1', '2,3']
>>> '1,2,,3,'.split(',')
['1', '2', '', '3', '']
>>> '1<>2<>3<4'.split('<>')
['1', '2', '3<4']
```

Se *sep* não for especificado ou for `None`, um algoritmo diferente de separação é aplicado: ocorrências consecutivas de **`espaços em branco`** são consideradas como um separador único, e o resultado não conterá strings vazias no início ou no final, se a string tiver espaços em branco no início ou no final. Consequentemente, separar uma string vazia ou uma string que consiste apenas de espaços em branco com o separador `None`, retorna `[]`.

Por exemplo:

```
>>> '1 2 3'.split()
['1', '2', '3']
>>> '1 2 3'.split(maxsplit=1)
['1', '2 3']
>>> '   1   2   3   '.split()
['1', '2', '3']
```

Se *sep* não for especificado ou for `None` e *maxsplit* for `0`, somente as sequências iniciais de espaços em branco consecutivos serão consideradas.

Por exemplo:

```
>>> "".split(None, 0)
[]
```

```
>>> "   ".split(None, 0)
[]
>>> "   foo   ".split(maxsplit=0)
['foo   ']
```

Veja também `join()` e `rsplit()`.

str.splitlines(*keepends=False*)

Retorna uma lista das linhas na string, quebrando a mesma nas fronteiras de cada linha. Quebras de linhas não são incluídas na lista resultante a não ser que *keepends* seja fornecido e seja verdadeiro.

Este método divide nos seguintes limites das linhas. Em particular, os limites são um superconjunto de novas linhas universais.

Representação	Descrição
`\n`	Feed de linha
`\r`	Retorno de Carro
`\r\n`	Retorno do Carro + Feed da Linha
`\v` ou `\x0b`	Tabulação de Linha
`\f` ou `\x0c`	Formulário de Feed
`\x1c`	Separador de Arquivos
`\x1d`	Separador de Grupo
`\x1e`	Separador de Registro
`\x85`	Próxima Linha (C1 Control Code)
`\u2028`	Separador de Linha
`\u2029`	Separador de Parágrafo

Alterado na versão 3.2: `\v` e `\f` adicionado à lista de limites de linha.

Por exemplo:

```
>>> 'ab c\n\nde fg\rkl\r\n'.splitlines()
['ab c', '', 'de fg', 'kl']
>>> 'ab c\n\nde fg\rkl\r\n'.splitlines(keepends=True)
['ab c\n', '\n', 'de fg\r', 'kl\r\n']
```

Ao contrário do método **split()** quando um delimitador de String *sep* é fornecido, este método retorna uma lista vazia para a uma String vazia e uma quebra de linha de terminal não resulta numa linha extra:

```
>>> "".splitlines()
[]
>>> "Uma linha\n".splitlines()
['Uma linha']
```

Para comparação, temos `split('\n')`:

```
>>> ''.split('\n')
['']
>>> 'Duas linhas\n'.split('\n')
['Duas linhas', '']
```

str.startswith(*prefix*[, *start*[, *end*]])

Retorne `True` se a String começar com o *prefixo*, caso contrário, retorna `False`. *prefixo* também pode ser uma tupla de prefixos a serem procurados. Com *start* opcional, a String de teste começa nessa posição. Com *fim* opcional, interrompe a comparação de String nessa posição.

Por exemplo:

```
>>> 'Python'.startswith('Py')
True
>>> 'uma tupla de prefixos'.startswith(('em', 'uma'))
True
>>> 'Python é fantástico'.startswith('é', 7)
True
```

Veja também **endswith()** e **removeprefix()**.

str.strip(*chars=None*, /)

Retorna uma cópia da string com caracteres no início e no final removidos. O argumento *chars* é uma string que especifica o conjunto de caracteres a serem removidos. Se for omitido ou for `None`, o argumento *chars* irá remover por padrão os caracteres em branco. O argumento *chars* não é um prefixo, nem um

sufixo; ao contrário disso, todas as combinações dos seus seus valores são removidas

Caracteres de espaço em branco são definidos por `str.isspace()`.

Por exemplo:

```
>>> '   spacious   '.strip()
'spacious'
>>> 'www.example.com'.strip('cmowz.')
'example'
```

Os valores do argumento *chars* são removidos dos extremos inicial e final da string. Caracteres são removidos do extremo inicial até atingir um caractere da string que não está contido no conjunto de caracteres em *chars*. Uma ação similar acontece no extremo final da string.

Por exemplo:

```
>>> comment_string = '#....... Section 3.2.1 Issue #32 .......'
>>> comment_string.strip('.#! ')
'Section 3.2.1 Issue #32'
```

Veja também `rstrip()`.

str.swapcase()

Retorna uma cópia da string com caracteres maiúsculos convertidos para letras minúsculas, e vice-versa. Por exemplo:

```
>>> 'Olá Mundo'.swapcase()
'oLÁ mUNDO'
```

Observe que não é necessariamente verdade que `s.swapcase().swapcase() == s`. Por exemplo:

```
>>> 'straße'.swapcase().swapcase()
'strasse'
```

Veja também `str.lower()` e `str.upper()`.

str.title()

Retorna uma versão titlecased da string, onde palavras iniciam com um caractere com letra maiúscula e os caracteres restantes são em letras minúsculas.

Por exemplo:

```
>>> 'Olá mundo'.title()
'Olá mundo'
```

O algoritmo usa uma definição simples independente de idioma para uma palavra, como grupos de letras consecutivas. A definição funciona em muitos contextos, mas isso significa que apóstrofes em contrações e possessivos formam limites de palavras, os quais podem não ser o resultado desejado:

```
>>> "a caixa d'água do joe's bar está vazia".title()
"A Caixa D'Água Do Joe'S Bar Está Vazia"
```

A função **`string.capwords()`** não tem esse problema, pois ela divide palavras apenas em espaços.

Alternativamente, uma solução alternativa para os apóstrofes pode ser construída usando expressões regulares:

```
>>> import re
>>> def titlecase(s):
...     return re.sub(r"[A-Za-z]+('[A-Za-z]+)?",
...                   lambda mo: mo.group(0).capitalize(),
...                   s)
...
>>> titlecase("o joe's bar está aberto")
"O Joe's Bar Está Aberto"
```

Veja também **`istitle()`**.

str.translate(*table*, /)

Retorna uma cópia da string na qual cada caractere foi mapeado através da tabela de tradução fornecida. A tabela deve ser um objeto que implementa indexação através de **`__getitem__()`**, tipicamente um mapeamento ou uma sequência. Quando indexada por um ordinal unicode (um inteiro), o objeto tabela pode fazer qualquer uma das seguintes ações: retornar um ordinal unicode ou uma string, para mapear o caractere para um ou mais caracteres; retornar `None`, para deletar o caractere da string de retorno; ou levantar uma exceção **`LookupError`**, para mapear o caractere para si mesmo.

Você pode usar **`str.maketrans()`** para criar um mapa de tradução com mapeamentos caractere para caractere em diferentes formatos.

O exemplo a seguir utiliza um mapeamento para substituir `'a'` por `'X'`, `'b'` por `'Y'` e excluir `'c'`:

```
>>> 'abc123'.translate({ord('a'): 'X', ord('b'): 'Y', ord('c'): None
'XY123'
```

Veja também o módulo **codecs** para uma abordagem mais flexível para mapeamento de caractere customizado.

str.upper()

Retorna uma cópia da string com todos os caracteres que permitem maiúsculo e minúsculo [[4]] convertidos para letras maiúsculas. Perceba que `s.upper().isupper()` pode ser `False` se `s` contiver caracteres que não possuem maiúsculas e minúsculas, ou se a categoria Unicode do(s) caractere(s) resultante(s) não for "Lu" (Letra maiúscula), mas por ex "Lt" (Letra em titlecase).

O algoritmo de conversão para letras maiúsculas está descrito na seção 3.13.2 'Default Case Folding' do Padrão Unicode [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G34078].

str.zfill(*width*, /)

Retorna uma cópia da String deixada preenchida com dígitos ASCII `'0'` para fazer uma string de comprimento *width*. Um prefixo sinalizador principal (`'+'/'-'`) será tratado inserindo o preenchimento *após* o caractere de sinal em vez de antes. A String original será retornada se o *width* for menor ou igual a `len(s)`.

Por exemplo:

```
>>> "42".zfill(5)
'00042'
>>> "-42".zfill(5)
'-0042'
```

Veja também **rjust()**

Strings literais formatadas (f-strings)

Adicionado na versão 3.6.

Alterado na versão 3.7: **await** e **async for** podem ser usados em expressões dentro de f-strings.

Alterado na versão 3.8: Adicionado o especificador de depuração (=)

Alterado na versão 3.12: Muitas restrições a expressões dentro de f-strings foram removidas. Notavelmente, strings aninhadas, comentários e contrabarras agora são permitidos.

Uma *f-string* (formalmente uma *literal de string formatada*) é uma literal de string prefixada com `f` ou `F`. Esse tipo de literal de string permite incorporar os resultados de expressões Python arbitrárias dentro de *campos de substituição*, delimitados por chaves (`{}`). Cada campo de substituição deve conter uma expressão, opcionalmente seguido por:

- um *especificador de depuração* – um sinal de igual (=);
- um *especificador de conversão* – `!s`, `!r` ou `!a`; e/ou
- um *especificador de formatação* prefixado com um caractere de dois pontos (`:`).

Veja a seção de análise léxica sobre f-strings para detalhes na sintaxe desses campos.

Especificador de depuração

Adicionado na versão 3.8.

Se um especificador de depuração — um sinal de igual (=) — aparecer após a expressão do campo de substituição, a f-string resultante conterá a origem da expressão, o sinal de igual e o valor da expressão. Isso costuma ser útil para depuração.

```
>>> number = 14.3
>>> f'{number=}'
'number=14.3'
```

Espaços em branco antes, dentro e depois da expressão, bem como espaços em branco após o sinal de igual, são significativos — eles são mantidos no resultado.

```
>>> f'{ number  -  4  = }'
' number  -  4  = 10.3'
```

Especificador de conversão

Por padrão, o valor de uma expressão de campo de substituição é convertido em uma string usando `str()`:

```
>>> from fractions import Fraction
>>> one_third = Fraction(1, 3)
```

```
>>> f'{one_third}'
'1/3'
```

Quando um especificador de depuração é usado, mas nenhum especificador de formato, a conversão padrão usa `repr()`:

```
>>> f'{one_third = }'
'one_third = Fraction(1, 3)'
```

A conversão pode ser especificada explicitamente usando um destes especificadores:

- `!s` para `str()`
- `!r` para `repr()`
- `!a` para `ascii()`

Por exemplo:

```
>>> str(one_third)
'1/3'
>>> repr(one_third)
'Fraction(1, 3)'

>>> f'{one_third!s} is {one_third!r}'
'1/3 is Fraction(1, 3)'

>>> string = "¡kočka 😸!"
>>> ascii(string)
"'\\xa1ko\\u010dka \\U0001f638!'"

>>> f'{string = !a}'
"string = '\\xa1ko\\u010dka \\U0001f638!'"
```

Especificador de formatação

Após a expressão ser avaliada e, possivelmente, convertida usando um especificador de conversão explícito, ela é formatada usando a função `format()`. Se o campo de substituição incluir um *especificador de formato* introduzido por dois pontos (`:`), o especificador é passado para **`format()`** como segundo argumento. O resultado de **`format()`** é então usado como o valor final para o campo de substituição. Por exemplo:

```
>>> from fractions import Fraction
>>> one_third = Fraction(1, 3)
>>> f'{one_third:.6f}'
'0.333333'
```

```
>>> f'{one_third:_^+10}'
'___+1/3___'
>>> f'{one_third!r:_^20}'
'___Fraction(1, 3)___'
>>> f'{one_third = :~>10}~'
'one_third = ~~~~~~~1/3~'
```

literais de strings templates (t-strings)

Uma *t-string* (formalmente um *literal de string template*) é uma string literal que é prefixada com `t` ou `T`.

Essas strings seguem a mesma sintaxe e regras de avaliação que literais de strings formatadas, com as seguintes diferenças:

- Em vez de serem avaliados como um objeto `str`, os literais de string template são avaliados como um objeto **`string.templatelib.Template`**.
- O protocolo **`format()`** não é usado. Em vez disso, o especificador de formato e conversões (se houver alguma) são passados para um novo objeto **`Interpolation`** que é criado para cada expressão avaliada. Cabe ao código que processa o objeto **`Template`** resultante decidir como tratar especificadores de formato e conversões.
- Especificadores de formato contendo campos de substituição aninhados são executados imediatamente, antes de serem passados para o objeto **`Interpolation`**. Por exemplo, uma interpolação da forma `{amount:.{precision}f}` vai executar a expressão interna `{precision}` para determinar o valor do atributo `format_spec`. Se `precision` fosse `2`, o especificador de formato resultante seria `'.2f'`.
- Quando o sinal de igual `'='` aparece em uma expressão de interpolação, o texto da expressão é concatenado ao literal de string que antecede a interpolação relevante. Isso inclui o sinal de igual, e qualquer espaço em branco que o cerque. A instância de **`Interpolation`** para a expressão será criada normalmente, exceto que o **`conversion`** será definido como `r` (**`repr()`**) por padrão. Se uma conversão ou especificador de formato explícito for fornecido, isso substitui esse comportamento padrão.

Formatação de string no estilo `printf`

Nota

As operações de formatação descritas aqui exibem uma variedade de peculiaridades que levam a uma série de erros comuns (como a falha na exibição

de Tuplas e Dicionários corretamente).

O uso de literais de strings formatadas, a interface **`str.format()`** ou **`string.Template`** pode ajudar a evitar esses erros. Cada uma dessas alternativas oferece suas próprias vantagens e desvantagens de simplicidade, flexibilidade e/ou extensibilidade.

Os objetos string possuem um único operador embutido: o operador `%` (módulo). O mesmo também é conhecido como o operador de *formatação* ou *interpolação*. Dado `format % values` (onde *format* é uma string), as especificações de conversão `%` em *format* são substituídas por zero ou mais elementos de *values*. O efeito é semelhante ao uso da função **`sprintf()`** na linguagem C. Por exemplo:

```
>>> print('%s tem %d tipos de aspas.' % ('Python', 2))
Python tem 2 tipos de aspas.
```

Se *format* precisar de um único operador, *valores* podem ser objetos simples ou que não sejam uma tupla. [[5]] Caso contrário, *valores* precisarão ser uma tupla com exatamente o número de itens especificados pela string de formatação, ou um único mapa de objetos (por exemplo, um dicionário).

Um especificador de conversão contém dois ou mais caracteres e tem os seguintes componentes, que devem aparecer nesta ordem:

1. O caractere `'%'`, que determina o início do especificador.
2. Mapeamento de chaves (opcional), consistindo de uma sequência entre parênteses de caracteres (por exemplo, `(algumnome)`).
3. Flags de conversão (opcional), que afetam o resultado de alguns tipos de conversão.
4. Largura mínima do campo (opcional). Se for especificado como um `'*'` (asterisco), a largura real será lida a partir do próximo elemento da tupla em *values* e o objeto a converter virá após a largura mínima do campo e a precisão que é opcional.
5. Precisão (opcional), fornecido como uma `'.'` (ponto) seguido pela precisão. Se determinado como um `'*'` (um asterisco), a precisão real será lida a partir do próximo elemento da tupla em *values*, e o valor a converter virá após a precisão.
6. Modificador de comprimento (opcional).
7. Tipo de conversão.

Quando o argumento certo for um dicionário (ou outro tipo de mapeamento), então os formatos na string *deverão* incluir uma chave de mapeamento entre parênteses nesse dicionário inserido imediatamente após o caractere `'%'`. A key de mapeamento seleciona o valor a ser formatado a partir do mapeamento. Por exemplo:

```
>>> print('%(language)s has %(number)03d quote types.' %
...       {'language': "Python", "number": 2})
Python has 002 quote types.
```

Nesse caso, nenhum especificador `*` poderá ocorrer num formato (uma vez que eles exigem uma lista de parâmetros sequenciais).

Os caracteres flags de conversão são:

Sinalizador	Significado
`'#'`	A conversão de valor usará o “formulário alternativo” (conforme definido abaixo).
`'0'`	A conversão será preenchida por zeros para valores numéricos.
`'-'`	O valor convertido será ajustado à esquerda (substitui a conversão `'0'` se ambos forem fornecidos).
`' '`	(um espaço) Um espaço em branco deverá ser deixado antes de um número positivo (ou uma string vazia) produzido por uma conversão assinada.
`'+'`	Um sinal de caractere (`'+'` ou `'-'`) precederá a conversão (substituindo o sinalizador de “espaço”).

Um modificador de comprimento (`h`, `l`, ou `L`) pode estar presente, mas será ignorado, pois o mesmo não é necessário para o Python – então, por exemplo, `%ld` é idêntico a `%d`.

Os tipos de conversão são:

Conversão	Significado	Notas
`'d'`	Número decimal inteiro sinalizador.	
`'i'`	Número decimal inteiro sinalizador.	
`'o'`	Valor octal sinalizador.	(1)
`'u'`	Tipo obsoleto – é idêntico a `'d'`.	(6)
`'x'`	Hexadecimal com sinal (minúsculo).	(2)
`'X'`	Hexadecimal com sinal (maiúsculo).	(2)
`'e'`	Formato exponencial de ponto flutuante (minúsculo).	(3)
`'E'`	Formato exponencial de ponto flutuante (maiúsculo).	(3)
`'f'`	Formato decimal de ponto flutuante.	(3)
`'F'`	Formato decimal de ponto flutuante.	(3)
`'g'`	O formato de ponto flutuante. Usa o formato exponencial em minúsculo se o expoente for inferior a -4 ou não inferior a precisão, formato decimal, caso contrário.	(4)
`'G'`	Formato de ponto flutuante. Usa o formato exponencial em maiúsculo se o expoente for inferior a -4 ou não inferior que a precisão, formato decimal, caso contrário.	(4)

Conversão	Significado	Notas
`'c'`	Caractere único (aceita inteiro ou um único caractere string).	
`'r'`	String (converte qualquer objeto Python usando a função `repr()`).	(5)
`'s'`	String (converte qualquer objeto Python usando a função `str()`).	(5)
`'a'`	String (converte qualquer objeto Python usando a função `ascii()`).	(5)
`'%'`	Nenhum argumento é convertido, resultando em um caractere `'%'` no resultado.	

Para formatos de ponto flutuante, o resultado deve ser arredondado corretamente para uma determinada precisão `p` de dígitos após o ponto decimal. O modo de arredondamento corresponde ao da função embutida `round()`.

Notas:

1. A forma alternativa faz com que um especificador octal principal (`'0o'`) seja inserido antes do primeiro dígito.

2. O formato alternativo produz um `'0x'` ou `'0X'` (dependendo se o formato `'x'` or `'X'` foi usado) para ser inserido antes do primeiro dígito.

3. A forma alternativa faz com que o resultado sempre contenha um ponto decimal, mesmo que nenhum dígito o siga.

   A precisão determina o número de dígitos após o ponto decimal e o padrão é 6.

4. A forma alternativa faz com que o resultado sempre contenha um ponto decimal e os zeros à direita não sejam removidos, como de outra forma seriam.

   A precisão determina o número de dígitos significativos antes e depois do ponto decimal e o padrão seja 6.

5. Se a precisão for `N`, a saída será truncada em `N` caracteres.

6. Veja **PEP 237** [https://peps.python.org/pep-0237/].

Como as strings no Python possuem comprimento explícito, conversões de `%s` não presumem que `'\0'` seja o fim da string.

Alterado na versão 3.1: `%f` as conversões para números cujo valor absoluto é superior a 1e50 não são mais substituídas pela conversão `%g`.

Tipos de sequência binária — `bytes`, `bytearray`, `memoryview`

Os principais tipos embutidos para manipular dados binários são `bytes` e `bytearray`. Eles são suportados por `memoryview` a qual usa o protocolo buffer para acessar a memória de outros objetos binários sem precisar fazer uma cópia.

O módulo `array` provê suporte a um armazenamento eficiente de tipos de dados básicos como inteiros de 32 bits e valores de ponto flutuante com precisão dupla IEEE754.

Objetos bytes

Objetos bytes são sequências imutáveis de bytes simples. Como muitos protocolos binários importantes são baseados em codificação ASCII de texto, objetos bytes oferecem diversos métodos que são válidos apenas quando trabalhamos com dados compatíveis com ASCII, e são proximamente relacionados com objetos string em uma variedade de outros sentidos.

class bytes(*source=b''*)
class bytes(*source*, *encoding*, *errors='strict'*)

Em primeiro lugar, a sintaxe para literais de bytes é em grande parte a mesma para literais de string, exceto que um prefixo `b` é adicionado:

- Aspas simples: `b'ainda permite aspas "duplas" dentro'`

- Aspas duplas: `b"ainda permite aspas 'simples' dentro"`
- Aspas triplas: `b'''3 aspas simples'''`, `b"""3 aspas duplas"""`

Apenas caracteres ASCII são permitidos em literais de bytes (independentemente da codificação declarada no código-fonte). Qualquer valor binário superior a 127 deverá ser inserido em literais de bytes usando a sequência de escape apropriada.

Assim como string literais, bytes literais também podem usar um prefixo `r` para desabilitar o processamento de sequências de escape. Veja Literais de string e bytes para mais sobre as várias formas de bytes literais, incluindo suporte a sequências de escape.

Enquanto bytes literais e representações são baseados em texto ASCII, objetos bytes na verdade se comportam como sequências imutáveis de inteiros, com cada valor na sequência restrito aos limites `0 <= x < 256` (tentativas de violar essa restrição irão disparar **`ValueError`**). Isto é feito deliberadamente para enfatizar que enquanto muitos formatos binários incluem elementos baseados em ASCII e podem ser utilmente manipulados com alguns algoritmos orientados a texto, esse não é geralmente o caso para dados binários arbitrários (aplicar algoritmos de processamento de texto cegamente em formatos de dados binários que não são compatíveis com ASCII irá usualmente resultar em dados corrompidos).

Além das formas literais, os objetos bytes podem ser criados de várias outras maneiras:

- Um bytes preenchido com objetos zero com um comprimento especificado: `bytes(10)`
- De um iterável de inteiros: `bytes(range(20))`
- Copiando dados binários existentes através do protocolo de Buffer: `bytes(obj)`

Veja também os embutidos bytes.

Como 2 dígitos hexadecimais correspondem precisamente a apenas um byte, números hexadecimais são um formato comumente usado para descrever dados binários. Portanto, o tipo bytes tem um método de classe adicional para ler dados nesse formato:

classmethod fromhex(*string*, /)

Este método de classe da classe **`bytes`** retorna um objeto bytes, decodificando o objeto string fornecido. A string deve conter dois dígitos

hexadecimais por byte, com espaço em branco ASCII sendo ignorado.

```
>>> bytes.fromhex('2Ef0 F1f2  ')
b'.\xf0\xf1\xf2'
```

Alterado na versão 3.7: `bytes.fromhex()` agora ignora todos os espaços em branco em ASCII na string, não apenas espaços.

Alterado na versão 3.14: `bytes.fromhex()` agora aceita objetos `bytes` e similares ASCII como entrada.

Uma função de conversão reversa existe para transformar um objeto bytes na sua representação hexadecimal.

hex(*, *bytes_per_sep=1*)
hex(*sep*, *bytes_per_sep=1*)

Retorna um objeto string contendo dois dígitos hexadecimais para cada byte na instância.

```
>>> b'\xf0\xf1\xf2'.hex()
'f0f1f2'
```

Se você quiser tornar a string hexadecimal mais fácil de ler, você pode especificar um caractere separador através do parâmetro *sep*, que será incluído no resultado. Por padrão, este separador será incluído entre cada byte. Um segundo parâmetro opcional *bytes_per_sep* controla o espaçamento. Valores positivos calculam a posição do separador a partir da direita, valores negativos calculam a partir da esquerda.

```
>>> value = b'\xf0\xf1\xf2'
>>> value.hex('-')
'f0-f1-f2'
>>> value.hex('_', 2)
'f0_f1f2'
>>> b'UUDDLRLRAB'.hex(' ', -4)
'55554444 4c524c52 4142'
```

Adicionado na versão 3.5.

Alterado na versão 3.8: `bytes.hex()` agora suporta os parâmetros opcionais *sep* e *bytes_per_sep* para inserir separadores entre bytes na saída hexadecimal.

Como objetos bytes são sequências de inteiros (certo parentesco a uma tupla), para um objeto bytes *b*, `b[0]` será um inteiro, enquanto `b[0:1]` será um objeto bytes de comprimento 1. (Isso contrasta com strings de texto, onde tanto o uso por índice quanto por fatiamento irão produzir uma string de comprimento 1)

A representação de objetos bytes utiliza o formato literal (`b'...'`), uma vez que ela é frequentemente mais útil do que, por exemplo, `bytes([46, 46, 46])`. Você sempre pode converter um objeto bytes em uma lista de inteiros usando `list(b)`.

Objetos Bytearray

Objetos **`bytearray`** são mutáveis, em contrapartida a objetos **`bytes`**.

class bytearray(*source=b''*)
class bytearray(*source*, *encoding*, *errors='strict'*)

Não existe sintaxe literal dedicada para objetos de bytearray, ao invés disso eles sempre são criados através da chamada do construtor:

- Criando uma instância vazia: `bytearray()`
- Criando uma instância cheia de zero com um determinado comprimento: `bytearray(10)`
- A partir de inteiros iteráveis: `bytearray(range(20))`
- Copiando dados binários existentes através do protocolo de buffer: `bytearray(b'Hi!')`

Como os objetos bytearray são mutáveis, os mesmos suportam as operações de sequência mutable além das operações comuns de bytes e bytearray descritas em Operações com Bytes e Bytearray.

Veja também o tipo embutido bytearray.

Uma vez que 2 dígitos hexadecimais correspondem precisamente a um único byte, os números hexadecimais são um formato comumente usado para descrever dados binários. Consequentemente, o tipo de bytearray tem um método de classe adicional para ler dados nesse formato:

classmethod fromhex(*string*, /)

Este método de classe da classe **`bytearray`** retorna um objeto bytearray, decodificando o objeto string fornecido. A string deve conter dois dígitos hexadecimais por byte, com espaços em branco ASCII sendo ignorados.

```
>>> bytearray.fromhex('2Ef0 F1f2  ')
bytearray(b'.\xf0\xf1\xf2')
```

Alterado na versão 3.7: `bytearray.fromhex()` agora ignora todos os espaços em branco em ASCII na string, não apenas espaços.

Alterado na versão 3.14: `bytearray.fromhex()` agora aceita objetos `bytes` e similares ASCII como entrada.

Existe uma função de conversão reversa para transformar um objeto bytearray em sua representação hexadecimal.

hex(*, *bytes_per_sep=1*)
hex(*sep*, *bytes_per_sep=1*)

Retorna um objeto string contendo dois dígitos hexadecimais para cada byte na instância.

```
>>> bytearray(b'\xf0\xf1\xf2').hex()
'f0f1f2'
```

Adicionado na versão 3.5.

Alterado na versão 3.8: Similar a `bytes.hex()`, `bytearray.hex()` agora suporta os parâmetros opcionais *sep* e *bytes_per_sep* para inserir separadores entre bytes na saída hexadecimal.

resize(*size*, /)

Redimensiona `bytearray` para conter *size* bytes. *size* deve ser maior ou igual a 0.

Se `bytearray` precisar ser reduzido, os bytes além de *size* serão truncados.

Se `bytearray` precisar crescer, todos os novos bytes, aqueles além de *size*, serão definidos como bytes nulos.

Isso equivale a:

```
>>> def resize(ba, size):
...     if len(ba) > size:
...         del ba[size:]
...     else:
...         ba += b'\0' * (size - len(ba))
```

Exemplos:

```
>>> shrink = bytearray(b'abc')
>>> shrink.resize(1)
>>> (shrink, len(shrink))
(bytearray(b'a'), 1)
>>> grow = bytearray(b'abc')
>>> grow.resize(5)
>>> (grow, len(grow))
(bytearray(b'abc\x00\x00'), 5)
```

Adicionado na versão 3.14.

take_bytes(*n=None*, /)

Remove os primeiros *n* bytes do bytearray e retorna-os como um objeto **bytes** imutável. Por padrão (se *n* for `None`), retorna todos os bytes e limpe o bytearray.

Se *n* for negativo, indexa a partir do final e leva em consideração primeiro **len()** somado a *n* bytes. Se *n* estiver fora dos limites, levanta **IndexError**.

Ler menos do que o comprimento completo deixará bytes restantes no **bytearray**, o que exige uma cópia. Se os bytes restantes puderem ser descartados, usa **resize()** ou **del** para truncar e, em seguida, **take_bytes()** sem especificar o tamanho.

Obter todos os bytes é uma operação de cópia zero.

Adicionado na versão 3.15: Consulte a entrada no O que há de novo para ver padrões de código comuns que podem ser otimizados com **bytearray.take_bytes()**.

Como os objetos bytearray são sequências de inteiros (semelhante a uma lista), para um objeto bytearray *b*, `b[0]` será um inteiro, enquanto que `b[0:1]` será um objeto bytearray de comprimento 1. (Isso contrasta com as Strings de texto, onde tanto a indexação como o fatiamento produzirão sequências de comprimento 1)

A representação de objetos bytearray utiliza o formato literal bytes (`bytearray(b'...')`), uma vez que muitas vezes é mais útil do que, por exemplo, `bytearray([46, 46, 46])`. Você sempre pode converter um objeto bytearray em uma lista de inteiros usando `list(b)`.

Ver também

Para informações detalhadas sobre as garantias de segurança de thread para objetos **`bytearray`**, consulte Segurança para thread para objetos bytearray.

Operações com Bytes e Bytearray

Ambos os tipos, bytes e objetos do tipo bytearray, aceitam as operações de sequências comuns. Os mesmos interagem não apenas com operandos do mesmo tipo, mas também com qualquer objeto bytes ou similar. Devido a esta flexibilidade, os mesmos podem ser misturados livremente em operações sem causar erros. No entanto, o tipo de retorno do resultado pode depender da ordem dos operandos.

Nota

Os métodos em bytes e objetos Bytearray não aceitam Strings como argumentos, assim como os métodos de Strings não aceitam bytes como argumentos. Por exemplo, devemos escrever:

```
a = "abc"
b = a.replace("a", "f")
```

e:

```
a = b"abc"
b = a.replace(b"a", b"f")
```

Alguns operações com bytes e bytearray presumem o uso de formatos binários compatíveis com ASCII e, portanto, devem ser evitados ao trabalhar com dados binários arbitrários. Essas restrições são abordadas a seguir.

Nota

O uso dessas operações baseadas em ASCII para manipular dados binários que não são armazenados num formato baseado em ASCII poderá resultar na corrupção de dados.

Os métodos a seguir em Bytes e Bytearray podem ser usados com dados binários arbitrários.

bytes.count(*sub*[, *start*[, *end*]])

bytearray.count(*sub*[, *start*[, *end*]])

Retorna o número de ocorrências não sobrepostas de subsequência *sub* no intervalo [*start*, *end*]. Os argumentos opcionais *start* e *end* são interpretados como na notação de fatiamento.

A subsequência a ser procurada poderá ser qualquer objeto bytes ou similar ou um inteiro no intervalo de 0 a 255.

Se *sub* estiver vazio, retorna o número de fatias vazias entre os caracteres, que é o comprimento do objeto bytes mais um.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.removeprefix(*prefix*, /)
bytearray.removeprefix(*prefix*, /)

Se os dados binários começarem com a string *prefix*, retorna `bytes[len(prefix):]`. Caso contrário, retorna uma cópia dos dados binários originais:

```
>>> b'TestHook'.removeprefix(b'Test')
b'Hook'
>>> b'BaseTestCase'.removeprefix(b'Test')
b'BaseTestCase'
```

O *prefix* pode ser qualquer objeto bytes ou similar.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Adicionado na versão 3.9.

bytes.removesuffix(*suffix*, /)
bytearray.removesuffix(*suffix*, /)

Se os dados binários terminarem com a string *suffix* e a *suffix* não estiver vazia, retorna `bytes[:-len(suffix)]`. Caso contrário, retorna uma cópia dos dados binários originais:

```
>>> b'MiscTests'.removesuffix(b'Tests')
b'Misc'
```

```
>>> b'TmpDirMixin'.removesuffix(b'Tests')
b'TmpDirMixin'
```

O *suffix* pode ser qualquer objeto bytes ou similar.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Adicionado na versão 3.9.

bytes.decode(*encoding='utf-8'*, *errors='strict'*)
bytearray.decode(*encoding='utf-8'*, *errors='strict'*)

Retorna os bytes decodificados para uma `str`.

encoding tem como padrão `'utf-8'`; veja Standard Encodings para valores possíveis.

errors controla como os erros de decodificação são tratados. Se `'strict'` (o padrão), uma exceção `UnicodeError` é levantada. Outros valores possíveis são `'ignore'`, `'replace'` e qualquer outro nome registrado via `codecs.register_error()`. Veja Error Handlers para detalhes.

Por motivos de desempenho, o valor de *errors* não é verificado quanto à validade, a menos que um erro de decodificação realmente ocorra, Modo de Desenvolvimento do Python esteja ativado ou uma construção de depuração seja usada.

Nota

Passar o argumento *encoding* para `str` permite decodificar qualquer objeto objeto bytes ou similar diretamente, sem a necessidade de criar um objeto `bytes` ou `bytearray` temporário.

Alterado na versão 3.1: Adicionado suporte para argumentos nomeados.

Alterado na versão 3.9: O valor do argumento *errors* agora é verificado em Modo de Desenvolvimento do Python e no modo de depuração.

bytes.endswith(*suffix*[, *start*[, *end*]])

bytearray.endswith(*suffix*[, *start*[, *end*]])

Retorna `True` se os dados binários encerra com o parâmetro *suffix* especificado, caso contrário retorna `False`. *suffix* também pode ser uma tupla de sufixos para averiguar. Com o parâmetro opcional *start*, a procura começa naquela posição. Com o parâmetro opcional *end*, o método encerra a comparação na posição fornecida.

O sufixo(es) para buscas pode ser qualquer termos objeto bytes ou similar.

bytes.find(*sub*[, *start*[, *end*]])

bytearray.find(*sub*[, *start*[, *end*]])

Retorna o índice mais baixo nos dados onde a sub-sequência *sub* é encontrada, tal que *sub* está contida na fatia `s[start:end]`. Argumentos opcionais *start* e *end* são interpretados como na notação de fatiamento. Retorna `-1` se *sub* não for localizada.

A subsequência a ser procurada poderá ser qualquer objeto bytes ou similar ou um inteiro no intervalo de 0 a 255.

Nota

O método **`find()`** deve ser usado apenas se você precisa saber a posição de *sub*. Para verificar se *sub* é uma substring ou não, use o operador **`in`**:

```
>>> b'Py' in b'Python'
True
```

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.index(*sub*[, *start*[, *end*]])

bytearray.index(*sub*[, *start*[, *end*]])

Como **`find()`**, mas levanta a exceção **`ValueError`** quando a subsequência não for encontrada.

A subsequência a ser procurada poderá ser qualquer objeto bytes ou similar ou um inteiro no intervalo de 0 a 255.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.join(*iterable*, /)
bytearray.join(*iterable*, /)

Retorna um objeto bytes ou bytearray que é a concatenação das sequências de dados binários no iterável *iterable*. Uma exceção `TypeError` será levantada se existirem quaisquer valores que não sejam objeto bytes ou similar, no *iterable*, incluindo objetos `str`. O separador entre elementos é o conteúdo do objeto bytes ou bytearray que está fornecendo este método.

static bytes.maketrans(*from*, *to*, /)
static bytearray.maketrans(*from*, *to*, /)

Este método estático retorna uma tabela de tradução usável para `bytes.translate()`, que irá mapear cada caractere em *from* no caractere na mesma posição em *to*; *from* e *to* devem ambos ser objetos bytes ou similares e ter o mesmo comprimento.

Adicionado na versão 3.1.

bytes.partition(*sep*, /)
bytearray.partition(*sep*, /)

Quebra a sequência na primeira ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo uma cópia da sequência original, seguido de dois bytes ou objetos bytearray vazios.

O separador para buscar pode ser qualquer objeto bytes ou similar.

bytes.replace(*old*, *new*, /, *count=-1*)
bytearray.replace(*old*, *new*, /, *count=-1*)

Retorna uma cópia da sequência com todas as ocorrências da subsequência *old* substituídas por *new*. Se o argumento opcional *count* é fornecido, apenas as primeiras *count* ocorrências são substituídas. Se *count* não é especificado ou é igual a `-1`, então todas as ocorrências serão substituídas.

A subsequência para buscar e substituição pode ser qualquer termos objeto bytes ou similar.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Alterado na versão 3.15: count agora é suportado como um argumento nomeado.

bytes.rfind(*sub*[, *start*[, *end*]])

bytearray.rfind(*sub*[, *start*[, *end*]])

Retorna o índice mais alto na sequência onde a subsequência *sub* foi encontrada, de modo que *sub* esteja contido dentro de `s[start:end]`. Os argumentos opcionais *start* e *end* são interpretados como na notação de fatiamento. Caso ocorra algum problema será retornado `-1`.

A subsequência a ser procurada poderá ser qualquer objeto bytes ou similar ou um inteiro no intervalo de 0 a 255.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.rindex(*sub*[, *start*[, *end*]])

bytearray.rindex(*sub*[, *start*[, *end*]])

Semelhante a **`rfind()`** mas levanta **`ValueError`** quando a subsequência *sub* não é encontrada.

A subsequência a ser procurada poderá ser qualquer objeto bytes ou similar ou um inteiro no intervalo de 0 a 255.

Alterado na versão 3.3: Também aceita um número inteiro no intervalo de 0 a 255 como subsequência.

bytes.rpartition(*sep*, /)

bytearray.rpartition(*sep*, /)

Quebra a sequência na primeira ocorrência de *sep*, e retorna uma tupla com 3 elementos contendo a parte antes do separador, o próprio separador, e a parte após o separador. Se o separador não for encontrado, retorna uma tupla com 3 elementos contendo dois bytes ou objetos bytearray vazios, seguido por uma cópia da sequência original.

O separador para buscar pode ser qualquer objeto bytes ou similar.

bytes.startswith(*prefix*[, *start*[, *end*]])

bytearray.startswith(*prefix*[, *start*[, *end*]])

Retorna `True` se os dados binários começam com o *prefix* especificado, caso contrário retorna `False`. *prefix* também pode ser uma tupla de prefixos para procurar. Com o parâmetro opcional *start*, começa a procura na posição indicada. Com o parâmetro opcional *end*, encerra a procura na posição indicada.

Os prefixos para pesquisar podem ser qualquer objeto bytes ou similar.

bytes.translate(*table*, /, *delete=b''*)
bytearray.translate(*table*, /, *delete=b''*)

Retorna uma cópia dos bytes ou objeto bytearray, onde todas as ocorrências de bytes do argumento opcional *delete* são removidas, e os bytes restantes foram mapeados através da tabela de tradução fornecida, a qual deve ser um objeto bytes de comprimento 256.

Você pode usar o método **`bytes.maketrans()`** para criar uma tabela de tradução.

Define o argumento *table* como `None` para traduções que excluem apenas caracteres:

```
>>> b'leia este curto texto'.translate(None, b'aeiou')
b'l st crt txt'
```

Alterado na versão 3.6: delete agora é suportado como um argumento nomeado.

Os seguintes métodos de objetos bytes e bytearray tem comportamentos padrões que presumem o uso de formatos binários compatíveis com ASCII, mas ainda podem ser usados com dados binários arbitrários através da passagem argumentos apropriados. Perceba que todos os métodos de bytearray nesta seção *não* alteram os argumentos, e ao invés disso produzem novos objetos.

bytes.center(*width*, *fillbyte=b' '*, /)
bytearray.center(*width*, *fillbyte=b' '*, /)

Retorna uma cópia do objeto centralizado em uma sequência de comprimento *width*. Preenchimento é feito usando o *fillbyte* especificado (o padrão é um espaço ASCII). Para objetos **`bytes`**, a sequência original é retornada se *width* é menor que ou igual a `len(s)`.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.ljust(*width*, *fillbyte=b' '*, /)
bytearray.ljust(*width*, *fillbyte=b' '*, /)

Retorna uma cópia do objeto alinhado a esquerda em uma sequência de comprimento *width*. Preenchimento é feito usando o *fillbyte* especificado (o padrão é um espaço ASCII). Para objetos **`bytes`**, a sequência original é retornada se *width* é menor que ou igual a `len(s)`.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.lstrip(*bytes=None*, /)
bytearray.lstrip(*bytes=None*, /)

Retorna uma cópia da sequência com bytes iniciais especificados removidos. O argumento *bytes* é uma sequência binária que especifica o conjunto de valores de bytes a serem removidos. Se for omitido ou se for `None`, o argumento *bytes* será considerado como **`espaço em branco ASCII`** por padrão para a remoção. O argumento *bytes* não é um prefixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> b'   spacious   '.lstrip()
b'spacious   '
>>> b'www.example.com'.lstrip(b'cmowz.')
b'example.com'
```

A sequência binária de valores de bytes a serem removidos pode ser objeto bytes ou similar. Consulte **`removeprefix()`** para um método que removerá uma única string de prefixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> b'Arthur: three!'.lstrip(b'Arthur: ')
b'ee!'
>>> b'Arthur: three!'.removeprefix(b'Arthur: ')
b'three!'
```

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.rjust(*width*, *fillbyte=b' '*, /)

bytearray.rjust(*width*, *fillbyte=b' '*, /)

Retorna uma cópia do objeto alinhado a direita em uma sequência de comprimento *width*. Preenchimento é feito usando o *fillbyte* especificado (o padrão é um espaço ASCII). Para objetos **`bytes`**, a sequência original é retornada se *width* é menor que ou igual a `len(s)`.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.rsplit(*sep=None*, *maxsplit=-1*)

bytearray.rsplit(*sep=None*, *maxsplit=-1*)

Divide a sequência binária em subsequência do mesmo tipo, usando *sep* como um delimitador de string. Se *maxsplit* é fornecido, no máximo *maxsplit* divisões são feitas, aquelas que estão *mais à direita*. Se *sep* não é especificado ou é `None`, qualquer subsequência consistindo unicamente de **`espaço em branco ASCII`** é um separador. Exceto por dividir pela direita, **`rsplit()`** comporta-se como **`split()`**, o qual é descrito em detalhes abaixo.

bytes.rstrip(*bytes=None*, /)

bytearray.rstrip(*bytes=None*, /)

Retorna uma cópia da sequência com bytes ao final especificados removidos. O argumento *bytes* é uma sequência binária que especifica o conjunto de valores de bytes a serem removidos. Se for omitido ou se for `None`, o argumento *bytes* será considerado como **`espaço em branco ASCII`** por padrão para a remoção. O argumento *bytes* não é um sufixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> b'   spacious   '.rstrip()
b'   spacious'
>>> b'mississippi'.rstrip(b'ipz')
b'mississ'
```

A sequência binária de valores de bytes a serem removidos pode ser objeto bytes ou similar. Consulte **`removesuffix()`** para um método que removerá uma única string de sufixo em vez de todo um conjunto de caracteres. Por exemplo:

```
>>> b'Monty Python'.rstrip(b' Python')
b'M'
```

```
>>> b'Monty Python'.removesuffix(b' Python')
b'Monty'
```

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.split(*sep=None*, *maxsplit=-1*)
bytearray.split(*sep=None*, *maxsplit=-1*)

Divide a sequência binária em subsequências do mesmo tipo, usando *sep* como o delimitador de string. Se *maxsplit* é fornecido e não-negativo, no máximo *maxsplit* divisões são feitas (logo, a lista terá no máximo `maxsplit+1` elementos). Se *maxsplit* não é especificado ou é `-1`, então não existe limite no número de divisões (todas as divisões possíveis são feitas).

Se *sep* é fornecido, delimitadores consecutivos não são agrupados juntos e eles são destinados para delimitar strings vazias (por exemplo, `b'1,,2'.split(b',')` retorna `[b'1', b'', b'2']`). O argumento *sep* pode consistir de uma sequência de múltiplos bytes como um único delimitador. Separar uma sequência vazia com um separador especificado retorna `[b'']` ou `[bytearray(b'')]` dependendo do tipo do objeto que está sendo separado. O argumento *sep* pode ser qualquer objeto bytes ou similar.

Por exemplo:

```
>>> b'1,2,3'.split(b',')
[b'1', b'2', b'3']
>>> b'1,2,3'.split(b',', maxsplit=1)
[b'1', b'2,3']
>>> b'1,2,,3,'.split(b',')
[b'1', b'2', b'', b'3', b'']
>>> b'1<>2<>3<4'.split(b'<>')
[b'1', b'2', b'3<4']
```

Se *sep* não for especificado ou for `None`, um algoritmo diferente de separação é aplicado: ocorrências consecutivas de **`espaços em branco ASCII`** são consideradas como um separador único, e o resultado não conterá strings vazias no início ou no final, se a sequência tiver espaços em branco no início ou no final. Consequentemente, separar uma sequência vazia ou uma sequência que consiste apenas de espaços em branco ASCII sem um separador especificado, retorna `[]`.

Por exemplo:

```
>>> b'1 2 3'.split()
[b'1', b'2', b'3']
>>> b'1 2 3'.split(maxsplit=1)
[b'1', b'2 3']
>>> b'   1   2   3   '.split()
[b'1', b'2', b'3']
```

bytes.strip(*bytes=None*, /)
bytearray.strip(*bytes=None*, /)

Retorna uma cópia da sequência com bytes iniciais e ao final especificados removidos. O argumento *bytes* é uma sequência binária que especifica o conjunto de valores de bytes a serem removidos. Se for omitido ou se for `None`, o argumento *bytes* será considerado como **`espaço em branco ASCII`** por padrão para a remoção. O argumento *bytes* não é um prefixo e sufixo; ao invés disso, todas as combinações dos seus valores são removidas:

```
>>> b'   spacious   '.strip()
b'spacious'
>>> b'www.example.com'.strip(b'cmowz.')
b'example'
```

A sequência binária de valores de bytes a serem removidos pode ser qualquer objeto bytes ou similar.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

O seguintes métodos de bytes e objetos bytearray presumem o uso de formatos binários compatíveis com ASCII e não devem ser aplicados a dados binários arbitrários. Perceba que todos os métodos de bytearray nesta seção *não* alteram os argumentos, e ao invés disso produzem novos objetos.

bytes.capitalize()
bytearray.capitalize()

Retorna uma cópia da sequência com cada byte interpretado como um caractere ASCII, e o primeiro byte em letra maiúscula e o resto em letras minúsculas. Valores de bytes que não são ASCII são passados adiante sem mudanças.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.expandtabs(*tabsize=8*)

bytearray.expandtabs(*tabsize=8*)

Retorna uma cópia da sequência onde todos os caracteres de tabulação ASCII são substituídos por um ou mais espaços ASCII, dependendo da coluna atual e do tamanho fornecido para a tabulação. Posições de tabulação ocorrem a cada *tabsize* bytes (o padrão é 8, dada as posições de tabulação nas colunas 0, 8, 16 e assim por diante). Para expandir a sequência, a coluna atual é definida como zero e a sequência é examinada byte por byte. Se o byte é um caractere ASCII de tabulação (`b'\t'`), um ou mais caracteres de espaço são inseridos no resultado até que a coluna atual seja igual a próxima posição de tabulação. (O caractere de tabulação em si não é copiado.) Se o byte atual é um caractere ASCII de nova linha (`b'\n'`) ou de retorno (`b'\r'`), ele é copiado e a coluna atual é redefinida para zero. Qualquer outro byte é copiado sem ser modificado e a coluna atual é incrementada em uma unidade independentemente de como o byte é representado quanto é impresso:

```
>>> b'01\t012\t0123\t01234'.expandtabs()
b'01      012     0123    01234'
>>> b'01\t012\t0123\t01234'.expandtabs(4)
b'01  012 0123    01234'
```

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.isalnum()

bytearray.isalnum()

Retorna `True` se todos os bytes na sequência são caracteres alfabéticos ASCII ou dígitos decimais ASCII e a sequência não é vazia, `False` caso contrário. Caracteres alfabéticos ASCII são aqueles valores de byte na sequência `b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Dígitos decimais ASCII são aqueles valores de byte na sequência `b'0123456789'`.

Por exemplo:

```
>>> b'ABCabc1'.isalnum()
True
>>> b'ABC abc1'.isalnum()
False
```

bytes.isalpha()
bytearray.isalpha()

Retorna `True` se todos os bytes na sequência são caracteres alfabéticos ASCII e a sequência não é vazia, `False` caso contrário. Caracteres alfabéticos ASCII são aqueles cujo valor dos bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Por exemplo:

```
>>> b'ABCabc'.isalpha()
True
>>> b'ABCabc1'.isalpha()
False
```

bytes.isascii()
bytearray.isascii()

Retorna `True` se a sequência é vazia ou todos os bytes na sequência são ASCII, `False` caso contrário. Bytes ASCII estão no intervalo 0-0x7F.

Adicionado na versão 3.7.

bytes.isdigit()
bytearray.isdigit()

Retorna `True` se todos os bytes na sequência são dígitos decimais ASCII e a sequência não é vazia, `False` caso contrário. Dígitos decimais ASCII são aqueles cujos valores dos bytes estão na sequência `b'0123456789'`.

Por exemplo:

```
>>> b'1234'.isdigit()
True
>>> b'1.23'.isdigit()
False
```

bytes.islower()
bytearray.islower()

Retorna `True` se existe pelo menos um caractere minúsculo ASCII na sequência e nenhum caractere maiúsculo ASCII, `False` caso contrário.

Por exemplo:

```
>>> b'hello world'.islower()
True
>>> b'Hello world'.islower()
False
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

bytes.isspace()
bytearray.isspace()

Retorna `True` se todos os bytes na sequência são espaço em branco ASCII e a sequência não é vazia, `False` caso contrário. Caracteres de espaço em branco ASCII são aqueles cujos valores de bytes estão na sequência `b' \t\n\r\x0b\f'` (espaço, tabulação, nova linha, retorno do cursor, tabulação vertical, formulário de entrada).

bytes.istitle()
bytearray.istitle()

Retorna `True` se a sequência é titlecased ASCII e a sequência não é vazia, `False` caso contrário. Veja **`bytes.title()`** para mais detalhes sobre a definição de “titlecased”.

Por exemplo:

```
>>> b'Hello World'.istitle()
True
>>> b'Hello world'.istitle()
False
```

bytes.isupper()
bytearray.isupper()

Retorna `True` se existe pelo menos um caractere maiúsculo alfabético ASCII na sequência e nenhum caractere minúsculo ASCII, `False` caso contrário.

Por exemplo:

```
>>> b'HELLO WORLD'.isupper()
True
>>> b'Hello world'.isupper()
False
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

bytes.lower()
bytearray.lower()

Retorna uma cópia da sequência com todos os caracteres maiúsculos ASCII convertidos para os seus correspondentes caracteres minúsculos.

Por exemplo:

```
>>> b'Hello World'.lower()
b'hello world'
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.splitlines(*keepends=False*)
bytearray.splitlines(*keepends=False*)

Retorna uma lista das linhas na sequência binária, quebrando nos limites ASCII das linhas. Este método usa a abordagem de novas linhas universais para separar as linhas. Quebras de linhas não são incluídas na lista resultante a não ser que *keepends* seja fornecido e verdadeiro.

Por exemplo:

```
>>> b'ab c\n\nde fg\rkl\r\n'.splitlines()
[b'ab c', b'', b'de fg', b'kl']
```

```
>>> b'ab c\n\nde fg\rkl\r\n'.splitlines(keepends=True)
[b'ab c\n', b'\n', b'de fg\r', b'kl\r\n']
```

Ao contrário de **split()**, quando uma string delimitadora *sep* é fornecida, este método retorna uma lista vazia para a string vazia, e uma quebra de linha terminal não resulta em uma linha extra:

```
>>> b"".split(b'\n'), b"Two lines\n".split(b'\n')
([b''], [b'Two lines', b''])
>>> b"".splitlines(), b"One line\n".splitlines()
([], [b'One line'])
```

bytes.swapcase()
bytearray.swapcase()

Retorna uma cópia da sequência com todos os caracteres minúsculos ASCII convertidos para caracteres maiúsculos correspondentes, e vice-versa.

Por exemplo:

```
>>> b'Hello World'.swapcase()
b'hELLO wORLD'
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Ao contrário de **str.swapcase()**, é sempre fato que `bin.swapcase().swapcase() == bin` para as versões binárias. Conversões maiúsculas/minúsculas são simétricas em ASCII, apesar que isso não é geralmente verdade para pontos de código arbitrários Unicode.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.title()
bytearray.title()

Retorna uma versão titlecased da sequência binária, onde palavras iniciam com um caractere ASCII com letra maiúscula e os caracteres restantes são em letras

minúsculas. Bytes quem não possuem diferença entre maiúscula/minúscula não são alterados.

Por exemplo:

```
>>> b'Hello world'.title()
b'Hello World'
```

Caracteres minúsculos ASCII são aqueles cujos valores de byte estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de byte estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Todos os outros valores de bytes ignoram maiúsculas/minúsculas.

O algoritmo usa uma definição simples independente de idioma para uma palavra, como grupos de letras consecutivas. A definição funciona em muitos contextos, mas isso significa que apóstrofes em contrações e possessivos formam limites de palavras, os quais podem não ser o resultado desejado:

```
>>> b"they're bill's friends from the UK".title()
b"They're Bill'S Friends From The Uk"
```

Uma solução alternativa para os apóstrofes pode ser construída usando expressões regulares:

```
>>> import re
>>> def titlecase(s):
...     return re.sub(rb"[A-Za-z]+('[A-Za-z]+)?",
...                   lambda mo: mo.group(0)[0:1].upper() +
...                              mo.group(0)[1:].lower(),
...                   s)
...
>>> titlecase(b"they're bill's friends.")
b"They're Bill's Friends."
```

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.upper()
bytearray.upper()

Retorna uma cópia da sequência com todos os caracteres minúsculos ASCII convertidos para sua contraparte maiúscula correspondente.

Por exemplo:

```
>>> b'Hello World'.upper()
b'HELLO WORLD'
```

Caracteres minúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'abcdefghijklmnopqrstuvwxyz'`. Caracteres maiúsculos ASCII são aqueles cujos valores de bytes estão na sequência `b'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`.

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

bytes.zfill(*width*, /)
bytearray.zfill(*width*, /)

Retorna uma cópia da sequência preenchida a esquerda com dígitos `b'0'` ASCII para produzir uma sequência se comprimento *width*. Um sinal de prefixo inicial (`b'+'`/ `b'-'`) é controlado através da inserção de preenchimento *depois* do caractere de sinal, ao invés de antes. Para objetos **`bytes`**, a sequência original é retornada se *width* é menor que ou igual a `len(seq)`.

Por exemplo:

```
>>> b"42".zfill(5)
b'00042'
>>> b"-42".zfill(5)
b'-0042'
```

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Estilo de Formatação de bytes `printf`-style

Nota

As operações de formatação descritas aqui exibem uma variedade de peculiaridades que levam a uma série de erros comuns (como a falha na exibição de Tuplas e Dicionários corretamente). Se o valor que está sendo impresso puder ser uma tupla ou dicionário, envolva-o numa tupla.

Objetos Bytes (`bytes`/`bytearray`) possuem uma operação embutida exclusiva: o operador `%` (modulo). Isso também é conhecido como o bytes de *formatação* ou o operador de *interpolação*. Dado `format % values` (onde *format* é um objeto bytes), as especificações de conversão `%` em *format* são substituídas por zero ou mais elementos de *values*. O efeito é semelhante ao uso da função **`sprintf()`** na linguagem C.

Se *format* requer um único argumento, *values* poderá ser um único objeto não-tuple. [[5]] Caso contrário, *values* deverá ser uma tupla com exatamente o número de itens especificados pelo objeto de formatação de Bytes, ou um único objeto de mapeamento (por exemplo, um dicionário).

Um especificador de conversão contém dois ou mais caracteres e tem os seguintes componentes, que devem aparecer nesta ordem:

1. O caractere `'%'`, que determina o início do especificador.
2. Mapeamento de chaves (opcional), consistindo de uma sequência entre parênteses de caracteres (por exemplo, `(algumnome)`).
3. Flags de conversão (opcional), que afetam o resultado de alguns tipos de conversão.
4. Largura mínima do campo (opcional). Se for especificado como um `'*'` (asterisco), a largura real será lida a partir do próximo elemento da tupla em *values* e o objeto a converter virá após a largura mínima do campo e a precisão que é opcional.
5. Precisão (opcional), fornecido como uma `'.'` (ponto) seguido pela precisão. Se determinado como um `'*'` (um asterisco), a precisão real será lida a partir do próximo elemento da tupla em *values*, e o valor a converter virá após a precisão.
6. Modificador de comprimento (opcional).
7. Tipo de conversão.

Quando o argumento a direita é um dicionário (ou outro tipo de mapeamento), então o formato no objeto bytes *deve* incluir um mapeamento de chaves entre parêntesis neste dicionário inserido imediatamente após o caractere `'%'`. O mapeamento de chaves seleciona o valor a ser formatado a partir do mapeamento. Por exemplo:

```
>>> print(b'%(language)s has %(number)03d quote types.' %
...       {b'language': b"Python", b"number": 2})
b'Python has 002 quote types.'
```

Nesse caso, nenhum especificador `*` poderá ocorrer num formato (uma vez que eles exigem uma lista de parâmetros sequenciais).

Os caracteres flags de conversão são:

Sinalizador	Significado
`'#'`	A conversão de valor usará o “formulário alternativo” (conforme definido abaixo).
`'0'`	A conversão será preenchida por zeros para valores numéricos.
`'-'`	O valor convertido será ajustado à esquerda (substitui a conversão `'0'` se ambos forem fornecidos).
`' '`	(um espaço) Um espaço em branco deverá ser deixado antes de um número positivo (ou uma string vazia) produzido por uma conversão assinada.
`'+'`	Um sinal de caractere (`'+'` ou `'-'`) precederá a conversão (substituindo o sinalizador de “espaço”).

Um modificador de comprimento (`h`, `l`, ou `L`) pode estar presente, mas será ignorado, pois o mesmo não é necessário para o Python – então, por exemplo, `%ld` é idêntico a `%d`.

Os tipos de conversão são:

Conversão	Significado	Notas

Conversão	Significado	Notas
`'d'`	Número decimal inteiro sinalizador.	
`'i'`	Número decimal inteiro sinalizador.	
`'o'`	Valor octal sinalizador.	(1)
`'u'`	Tipo obsoleto – é idêntico a `'d'`.	(8)
`'x'`	Hexadecimal com sinal (minúsculo).	(2)
`'X'`	Hexadecimal com sinal (maiúsculo).	(2)
`'e'`	Formato exponencial de ponto flutuante (minúsculo).	(3)
`'E'`	Formato exponencial de ponto flutuante (maiúsculo).	(3)
`'f'`	Formato decimal de ponto flutuante.	(3)
`'F'`	Formato decimal de ponto flutuante.	(3)
`'g'`	O formato de ponto flutuante. Usa o formato exponencial em minúsculo se o expoente for inferior a -4 ou não inferior a precisão, formato decimal, caso contrário.	(4)
`'G'`	Formato de ponto flutuante. Usa o formato exponencial em maiúsculo se o expoente for inferior a -4 ou não inferior que a precisão, formato decimal, caso contrário.	(4)

Conversão	Significado	Notas
`'c'`	Byte simples (aceita objetos inteiros ou de byte único).	
`'b'`	Bytes (qualquer objeto que segue o buffer protocol o que possui `__bytes__()`).	(5)
`'s'`	`'s'` é um alias para `'b'` e só deve ser usado para bases de código Python2/3.	(6)
`'a'`	Bytes (converte qualquer objeto Python usando `repr(obj).encode('ascii', 'backslashreplace')`).	(5)
`'r'`	`'r'` é um alias para `'a'` e só deve ser usado para bases de código Python2/3.	(7)
`'%'`	Nenhum argumento é convertido, resultando em um caractere `'%'` no resultado.	

Notas:

1. A forma alternativa faz com que um especificador octal principal (`'0o'`) seja inserido antes do primeiro dígito.

2. O formato alternativo produz um `'0x'` ou `'0X'` (dependendo se o formato `'x'` or `'X'` foi usado) para ser inserido antes do primeiro dígito.

3. A forma alternativa faz com que o resultado sempre contenha um ponto decimal, mesmo que nenhum dígito o siga.

   A precisão determina o número de dígitos após o ponto decimal e o padrão é 6.

4. A forma alternativa faz com que o resultado sempre contenha um ponto decimal e os zeros à direita não sejam removidos, como de outra forma seriam.

   A precisão determina o número de dígitos significativos antes e depois do ponto decimal e o padrão seja 6.

5. Se a precisão for `N`, a saída será truncada em `N` caracteres.

6. `b'%s'` está descontinuado, mas não será removido durante a versão 3.x.

7. `b'%r'` entrou em desuso, mas não serão removidos na versão 3.x.

8. Veja **PEP 237** [https://peps.python.org/pep-0237/].

Nota

A versão Bytearray deste método *não* opera no local – o mesmo sempre produz um novo objeto, mesmo que não tenha sido feitas alterações.

Ver também

PEP 461 [https://peps.python.org/pep-0461/] - Adicionar formatação % para to bytes e bytearray

Adicionado na versão 3.5.

Visões de memória

Objetos `memoryview` permitem que código Python acesse os dados internos de um objeto que implemente o protocolo buffer sem copiá-lo.

class memoryview(*object*)

Cria uma `memoryview` que referencia *object*. *object* deve ter suporte ao protocolo de buffer. Objetos embutidos com suporte ao protocolo de buffer incluem: `bytes` e `bytearray`.

Uma `memoryview` tem a noção de um *elemento*, o qual é a unidade de memória atômica manipulada pelo objeto de origem *object*. Para muitos tipos simples tais como `bytes` e `bytearray`, um elemento é um byte único, mas outros tipos tais como `array.array` podem ter elementos maiores.

memoryviews são genéricas em relação ao tipo dos seus dados subjacentes.

`len(view)` é igual ao comprimento de **tolist()**, que é a representação de lista aninhada da view. Se `view.ndim == 1`, isso é igual ao número de elementos na visualização.

Alterado na versão 3.12: Se `view.ndim == 0`, `len(view)` agora levanta **TypeError** em vez de retornar 1.

O atributo **itemsize** vai lhe dar um número de bytes em um único elemento.

Um **memoryview** suporta fatiamento e indexação para expor seus dados. Fatiamento unidimensional irá resultar em uma subview:

```
>>> v = memoryview(b'abcefg')
>>> v[1]
98
>>> v[-1]
103
>>> v[1:4]
<memory at 0x7f3ddc9f4350>
>>> bytes(v[1:4])
b'bce'
```

Se **format** é um dos especificadores de formatação nativos do módulo **struct**, indexar com um inteiro ou uma tupla de inteiros também é suportado, e retorna um *element* único com o tipo correto. Visões de memória unidimensionais podem ser indexadas por um inteiro ou por uma tupla contendo um inteiro. Visões de memória multidimensionais podem ser indexadas por tuplas de exatamente *ndim* inteiros, onde *ndim* é o número de dimensões. Visões de memória zero-dimensionais podem ser indexadas por uma tupla vazia.

Aqui temos um exemplo usando um formato não-byte:

```
>>> import array
>>> a = array.array('l', [-11111111, 22222222, -33333333, 44444444])
>>> m = memoryview(a)
>>> m[0]
-11111111
>>> m[-1]
44444444
>>> m[::2].tolist()
[-11111111, -33333333]
```

Se o objeto subjacente é gravável, a memoryview suporta atribuição de fatias unidimensionais. Redimensionamento não é permitido:

```
>>> data = bytearray(b'abcefg')
>>> v = memoryview(data)
>>> v.readonly
False
>>> v[0] = ord(b'z')
>>> data
bytearray(b'zbcefg')
>>> v[1:4] = b'123'
>>> data
bytearray(b'z123fg')
>>> v[2:3] = b'spam'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: memoryview assignment: lvalue and rvalue have different
>>> v[2:6] = b'spam'
>>> data
bytearray(b'z1spam')
```

Visões de memória unidimensionais de tipos hasheáveis (somente leitura) com formatos ‘B’, ‘b’ ou ‘c’ também são hasheáveis. O hash é definido como `hash(m) == hash(m.tobytes())`:

```
>>> v = memoryview(b'abcefg')
>>> hash(v) == hash(b'abcefg')
True
>>> hash(v[2:4]) == hash(b'ce')
True
>>> hash(v[::-2]) == hash(b'abcefg'[::-2])
True
```

Alterado na versão 3.3: Visões de memória unidimensionais agora podem ser fatiadas. Visões de memória unidimensionais com formatos ‘B’, ‘b’ ou ‘c’ agora são hasheáveis.

Alterado na versão 3.4: O memoryview agora é registrado automaticamente como uma classe `collections.abc.Sequence`

Alterado na versão 3.5: Atualmente, os memoryviews podem ser indexadas com uma tupla de números inteiros.

Alterado na versão 3.14: memoryview é agora um tipo genérico.

`memoryview` possui vários métodos:

__eq__(*exporter*)

Uma memoryview e um exportador **PEP 3118** [https://peps.python.org/pep-3118/] são iguais se as suas formas são equivalentes e se todos os valores correspondentes são iguais quando os códigos de formatação dos respectivos operadores são interpretados usando a sintaxe `struct`.

Para o subconjunto `struct` a formatação de Strings atualmente suportadas por `tolist()`, `v` e `w` são iguais se `v.tolist() == w.tolist()`:

```
>>> import array
>>> a = array.array('I', [1, 2, 3, 4, 5])
>>> b = array.array('d', [1.0, 2.0, 3.0, 4.0, 5.0])
>>> c = array.array('b', [5, 3, 1])
>>> x = memoryview(a)
>>> y = memoryview(b)
>>> x == a == y == b
True
>>> x.tolist() == a.tolist() == y.tolist() == b.tolist()
True
>>> z = y[::-2]
>>> z == c
True
>>> z.tolist() == c.tolist()
True
```

Se qualquer string de formatação não for suportada pelo módulo `struct`, então os objetos irão sempre comparar como diferentes (mesmo se as strings de formatação e o conteúdo do buffer são idênticos):

```
>>> from ctypes import BigEndianStructure, c_long
>>> class BEPoint(BigEndianStructure):
...     _fields_ = [("x", c_long), ("y", c_long)]
...
>>> point = BEPoint(100, 200)
>>> a = memoryview(point)
>>> b = memoryview(point)
>>> a == point
False
>>> a == b
False
```

Perceba que, assim como com números de ponto flutuante, `v is w` *não* implica em `v == w` para objetos memoryview.

Alterado na versão 3.3: Versões anteriores comparavam a memória bruta desconsiderando o formato do item e estrutura lógica do array.

tobytes(*order='C'*)

Retorna os dados no buffer como um bytestring. Isso é equivalente a chamar o construtor de **bytes** na memoryview.

```
>>> m = memoryview(b"abc")
>>> m.tobytes()
b'abc'
>>> bytes(m)
b'abc'
```

Para arrays não contíguos, o resultado é igual a representação de lista achatada com todos os elementos convertidos para bytes. **tobytes()** suporta todos os formatos de strings, incluindo aqueles que não estão na sintaxe do módulo **struct**.

Adicionado na versão 3.8: order pode ser {'C', 'F', 'A'}. Quando *order* é 'C' ou 'F', os dados do array original são convertidos para a ordem de C our Fortran. Para views contígua, 'A' retorna uma cópia exata da memória física. Em particular, ordem de Fortran em memória é preservada. Para views não contígua, os dados são convertidos primeiro para C. *order=None* é o mesmo que *order='C'*.

hex(**, bytes_per_sep=1*)

hex(*sep, bytes_per_sep=1*)

Retorna um objeto string contendo dois dígitos hexadecimais para cada byte no buffer.

```
>>> m = memoryview(b"abc")
>>> m.hex()
'616263'
```

Adicionado na versão 3.5.

Alterado na versão 3.8: Similar a **bytes.hex()**, **memoryview.hex()** agora suporta os parâmetros opcionais *sep* e *bytes_per_sep* para inserir separadores entre bytes na saída hexadecimal.

tolist()

Retorna os dados no buffer como uma lista de elementos.

```
>>> memoryview(b'abc').tolist()
[97, 98, 99]
>>> import array
>>> a = array.array('d', [1.1, 2.2, 3.3])
>>> m = memoryview(a)
>>> m.tolist()
[1.1, 2.2, 3.3]
```

Alterado na versão 3.3: `tolist()` agora suporta todos os formatos nativos de caracteres únicos na sintaxe do módulo `struct`, assim como representações multi-dimensionais.

toreadonly()

Retorna uma versão somente leitura do objeto memoryview. O objeto memoryview original não é alterado.

```
>>> m = memoryview(bytearray(b'abc'))
>>> mm = m.toreadonly()
>>> mm.tolist()
[97, 98, 99]
>>> mm[0] = 42
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot modify read-only memory
>>> m[0] = 43
>>> mm.tolist()
[43, 98, 99]
```

Adicionado na versão 3.8.

release()

Libera o buffer subjacente exposto pelo objeto memoryview. Muitos objetos aceitam ações especiais quando a view é mantida com eles (por exemplo, um `bytearray` iria temporariamente proibir o redimensionamento); portanto, chamar release() é útil para remover essas restrições (e liberar quaisquer recursos pendurados) o mais breve possível.

Depois que este método foi chamado, qualquer operação posterior na visão levanta um `ValueError` (exceto `release()`, o qual pode ser chamado múltiplas vezes):

```
>>> m = memoryview(b'abc')
>>> m.release()
>>> m[0]
Traceback (most recent call last):
```

```
  File "<stdin>", line 1, in <module>
ValueError: operation forbidden on released memoryview object
```

O protocolo de gerenciamento de contexto pode ser usado para efeitos similares, usando a instrução `with`:

```
>>> with memoryview(b'abc') as m:
...     m[0]
...
97
>>> m[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: operation forbidden on released memoryview object
```

Adicionado na versão 3.2.

cast(*format*, [..])

cast(*format*, *shape*, [..])

Converte uma memoryview para um novo formato ou forma. *shape* por padrão é `[byte_length//new_itemsize]`, o que significa que a visão resultante será unidimensional. O valor de retorno é uma nova memoryview, mas o buffer por si mesmo não é copiado. Conversões suportadas são 1D -> C-contíguo r C-contíguo -> 1D.

O formato de destino é restrito a um elemento em formato nativo na sintaxe **`struct`**. Um dos formatos deve ser um formato de byte (‘B’, ‘b’ ou ‘c’). O comprimento de bytes do resultado deve ser o mesmo que o comprimento original. Observe que todos os comprimentos de bytes podem depender do sistema operacional.

Converte de 1D/long para 1D/unsigned bytes:

```
>>> import array
>>> a = array.array('l', [1,2,3])
>>> x = memoryview(a)
>>> x.format
'l'
>>> x.itemsize
8
>>> len(x)
3
>>> x.nbytes
24
>>> y = x.cast('B')
```

```
>>> y.format
'B'
>>> y.itemsize
1
>>> len(y)
24
>>> y.nbytes
24
```

Converte de 1D/unsigned bytes para 1D/char:

```
>>> b = bytearray(b'zyz')
>>> x = memoryview(b)
>>> x[0] = b'a'
Traceback (most recent call last):
  ...
TypeError: memoryview: invalid type for format 'B'
>>> y = x.cast('c')
>>> y[0] = b'a'
>>> b
bytearray(b'ayz')
```

Converte de 1D/bytes para 3D/ints para 1D/signed char:

```
>>> import struct
>>> buf = struct.pack("i"*12, *list(range(12)))
>>> x = memoryview(buf)
>>> y = x.cast('i', shape=[2,2,3])
>>> y.tolist()
[[[0, 1, 2], [3, 4, 5]], [[6, 7, 8], [9, 10, 11]]]
>>> y.format
'i'
>>> y.itemsize
4
>>> len(y)
2
>>> y.nbytes
48
>>> z = y.cast('b')
>>> z.format
'b'
>>> z.itemsize
1
>>> len(z)
48
>>> z.nbytes
48
```

Converte 1D/unsigned long para 2D/unsigned long:

```
>>> buf = struct.pack("L"*6, *list(range(6)))
>>> x = memoryview(buf)
>>> y = x.cast('L', shape=[2,3])
>>> len(y)
2
>>> y.nbytes
48
>>> y.tolist()
[[0, 1, 2], [3, 4, 5]]
```

Adicionado na versão 3.3.

Alterado na versão 3.5: O formato fonte não é mais restrito ao converter para uma visão de byte.

count(*value*, *[*)

Conta o número de ocorrências de *value*.

Adicionado na versão 3.14.

index(*value*, *start=0*, *stop=sys.maxsize*, *[*)

Retorna o índice da primeira ocorrência de *value* (no índice *index* ou depois dele e antes do índice *stop*).

Levanta uma exceção `ValueError` se *value* não puder ser encontrado.

Adicionado na versão 3.14.

Existem também diversos atributos somente leitura disponíveis:

obj

O objeto subjacente da memoryview:

```
>>> b  = bytearray(b'xyz')
>>> m = memoryview(b)
>>> m.obj is b
True
```

Adicionado na versão 3.3.

nbytes

`nbytes == product(shape) * itemsize == len(m.tobytes())`. Este é a quantidade de espaço em bytes que o array deve usar em uma representação contígua. Ela não é necessariamente igual a `len(m)`:

```
>>> import array
>>> a = array.array('i', [1,2,3,4,5])
>>> m = memoryview(a)
>>> len(m)
5
>>> m.nbytes
20
>>> y = m[::2]
>>> len(y)
3
>>> y.nbytes
12
>>> len(y.tobytes())
12
```

Arrays Multi-dimensional:

```
>>> import struct
>>> buf = struct.pack("d"*12, *[1.5*x for x in range(12)])
>>> x = memoryview(buf)
>>> y = x.cast('d', shape=[3,4])
>>> y.tolist()
[[0.0, 1.5, 3.0, 4.5], [6.0, 7.5, 9.0, 10.5], [12.0, 13.5, 15.0,
>>> len(y)
3
>>> y.nbytes
96
```

Adicionado na versão 3.3.

readonly

Um bool que indica se a memória é somente leitura.

format

Uma string contendo o formato (no estilo do módulo **`struct`**) para cada elemento na visão. Uma memoryview pode ser criada a partir de exportadores com strings de formato arbitrário, mas alguns métodos (ex: **`tolist()`**) são restritos a formatos de elementos nativos.

Alterado na versão 3.3: formato `'B'` agora é tratado de acordo com a sintaxe do módulo struct. Isso significa que `memoryview(b'abc')[0] == b'abc'[0]`

```
== 97.
```

itemsize

O tamanho em Bytes de cada elemento do memoryview:

```
>>> import array, struct
>>> m = memoryview(array.array('H', [32000, 32001, 32002]))
>>> m.itemsize
2
>>> m[0]
32000
>>> struct.calcsize('H') == m.itemsize
True
```

ndim

Um número inteiro que indica quantas dimensões de uma matriz multidimensional a memória representa.

shape

Uma tupla de inteiros de comprimento **`ndim`** dando a forma da memória como uma matriz N-dimensional.

Alterado na versão 3.3: Uma tupla vazia ao invés de `None` quando ndim = 0.

strides

Uma tupla de inteiros de comprimento de **`ndim`** dando o tamanho em bytes para acessar cada elemento de cada dimensão da matriz.

Alterado na versão 3.3: Uma tupla vazia ao invés de `None` quando ndim = 0.

suboffsets

Usado internamente para estilos de Arrays PIL. O valor é apenas informativo.

c_contiguous

Um bool que indica se a memória é contígua C.

Adicionado na versão 3.3.

f_contiguous

Um bool que indica se a memória é contígua Fortran.

Adicionado na versão 3.3.

contiguous

Um bool que indica se a memória é contígua.

Adicionado na versão 3.3.

Para informações sobre a segurança de thread de objetos `memoryview` na construção com threads livres, consulte Segurança para thread para objetos memoryview.

Tipo conjuntos — `set`, `frozenset`

Um objeto *conjunto* é uma coleção não ordenada de objetos hasheáveis distintos. Usos comuns incluem testes de associação, remover duplicatas de uma sequência e computar operações matemáticas tais como interseção, união, diferença e diferença simétrica. (Para outros tipos de contêineres veja as classes embutidas `dict`, `list` e `tuple`, e o módulo `collections`.) Consulte Complexidade de tempo de operações em tipos embutidos para os custos das várias operações de conjuntos.

Assim como outras coleções, conjuntos suportam `x in set`, `len(set)` e `for x in set`. Sendo uma coleção não ordenada, conjuntos não armazenam posição de elementos ou ordem de inserção. Portanto, conjuntos não suportam indexação, fatiamento ou outros comportamentos de sequências ou similares.

Existem atualmente dois tipos de conjuntos embutidos, `set` e `frozenset`. O tipo **`set`** é mutável – o conteúdo pode ser alterado usando métodos como `add()` e `remove()`. Como ele é mutável, ele não tem valor hash e não pode ser usado como chave de dicionário ou um elemento de um outro conjunto. O tipo **`frozenset`** é imutável e hasheável – seu conteúdo não pode ser alterado depois de ter sido criado; ele pode então ser usado como chave de dicionário ou como um elemento de outro conjunto.

Conjuntos não vazios (que não sejam frozensets) podem ser criados posicionando uma lista de elementos separados por vírgula dentro de chaves, por exemplo: `{'jack', 'sjoerd'}`, além do construtor `set`.

Os construtores de ambas as classes funcionam da mesma forma:

class set(*iterable=()*, /)
class frozenset(*iterable=()*, /)

Retorna um novo objeto set ou frozenset, cujos elementos são obtidos a partir de um *iterable*. Os elementos de um conjunto devem ser hasheáveis. Para representar conjuntos de sets, os sets internos devem ser objetos `frozenset`. Se *iterable* não for especificado, um novo conjunto vazio é retornado.

Conjuntos podem ser criados de várias formas:

- Usar uma lista de elementos separados por vírgulas entre chaves: `{'jack', 'sjoerd'}`
- Usar uma compreensão de conjunto: `{c for c in 'abracadabra' if c not in 'abc'}`
- Usar o construtor da classe: `set()`, `set('foobar')`, `set(['a', 'b', 'foo'])`

Instâncias de **`set`** e **`frozenset`** fornecem as seguintes operações:

len(s)
: Retorna o número de elementos no set *s* (cardinalidade de *s*).

x in s
: Testa se *x* pertence a *s*.

x not in s
: Testa se *x* não pertence a *s*.

frozenset.isdisjoint(*other*, /)
set.isdisjoint(*other*, /)
: Retorna `True` se o conjunto não tem elementos em comum com *other*. Conjuntos são disjuntos se e somente se a sua interseção é o conjunto vazio.

frozenset.issubset(*other*, /)
set.issubset(*other*, /)

set <= other
: Testa se cada elemento do conjunto está contido em *other*.

set < other
: Testa se o conjunto é um subconjunto próprio de *other*, isto é, `set <= other and set != other`.

frozenset.issuperset(*other*, /)
set.issuperset(*other*, /)

set >= other
: Testa se cada elemento em *other* está contido no conjunto.

set > other

Testa se o conjunto é um superconjunto próprio de *other*, isto é, `set >= other and set != other`.

frozenset.union(**others*)
set.union(**others*)

set | other | ...

Retorna um novo conjunto com elementos do conjunto e de todos os outros.

frozenset.intersection(**others*)
set.intersection(**others*)

set & other & ...

Retorna um novo conjunto com elementos comuns do conjunto e de todos os outros.

frozenset.difference(**others*)
set.difference(**others*)

set - other - ...

Retorna um novo conjunto com elementos no conjunto que não estão nos outros.

frozenset.symmetric_difference(*other*, /)
set.symmetric_difference(*other*, /)

set ^ other

Retorna um novo conjunto com elementos estejam ou no conjunto ou em *other*, mas não em ambos.

frozenset.copy()
set.copy()

Retorna uma cópia rasa do conjunto.

Observe que, as versões não-operador dos métodos `union()`, `intersection()`, `difference()`, `symmetric_difference()`, `issubset()` e `issuperset()` irão aceitar qualquer iterável como um argumento. Em contraste, suas contrapartes baseadas em operadores exigem que seus argumentos sejam conjuntos. Isso impede construções suscetíveis a erros como `set('abc') & 'cbs'` e favorece a forma mais legível `set('abc').intersection('cbs')`.

Tanto **`set`** quanto **`frozenset`** aceitam comparações entre conjuntos. Dois conjuntos são iguais se e somente se cada elemento de um está contido no outro (cada um é um subconjunto do outro). Um conjunto é menor que outro se, e somente se, o primeiro conjunto é um subconjunto adequado do segundo (é um subconjunto, mas não é igual). Um conjunto é maior que outro conjunto se, e somente se, o primeiro conjunto é um superconjunto próprio do segundo conjunto (é um superconjunto, mas não é igual).

Instâncias de **`set`** são comparadas a instâncias de **`frozenset`** baseados nos seus membros. Por exemplo, `set('abc') == frozenset('abc')` retorna `True` e assim como `set('abc') in set([frozenset('abc')])`.

O subconjunto e comparações de igualdade não generalizam para a função de ordenamento total. Por exemplo, quaisquer dois conjuntos deslocados não vazios, não são iguais e não são subconjuntos um do outro, então *todos* os seguintes retornam `False`: `a<b`, `a==b` ou `a>b`.

Como conjuntos apenas definem ordenamento parcial (subconjunto de relacionamentos), a saída do método **`list.sort()`** é indefinida para listas e conjuntos.

Elementos de conjuntos, assim como chaves de dicionário, devem ser hasheáveis.

Operações binárias que misturam instâncias de **`set`** com **`frozenset`** retornam o tipo do primeiro operando. Por exemplo: `frozenset('ab') | set('bc')` retorna uma instância de **`frozenset`**.

A seguinte tabela lista operações disponíveis para **`set`** que não se aplicam para instâncias imutáveis de **`frozenset`**:

set.update(**others*)

set |= other | ...
    Atualiza o conjunto, adicionando elementos dos outros.

set.intersection_update(**others*)

set &= other & ...
    Atualiza o conjunto, mantendo somente elementos encontrados nele e em outros.

set.difference_update(**others*)

set -= other | ...

Atualiza o conjunto, removendo elementos encontrados em outros.

set.symmetric_difference_update(*other*, /)

set ^= other

Atualiza o conjunto, mantendo somente elementos encontrados em qualquer conjunto, mas não em ambos.

set.add(*elem*, /)

Adiciona o elemento *elem* ao conjunto.

set.remove(*elem*, /)

Remove o elemento *elem* do conjunto. Levanta `KeyError` se *elem* não estiver contido no conjunto.

set.discard(*elem*, /)

Remove o elemento *elem* do conjunto se ele estiver presente.

set.pop()

Remove e retorna um elemento arbitrário do conjunto. Levanta `KeyError` se o conjunto estiver vazio.

set.clear()

Remove todos os elementos do conjunto.

Perceba, as versões sem operador dos métodos `update()`, `intersection_update()`, `difference_update()` e `symmetric_difference_update()` irão aceitar qualquer iterável como um argumento.

Perceba, o argumento *elem* para os métodos `__contains__()`, `remove()` e `discard()` pode ser um conjunto. Para suportar pesquisas por um frozenset equivalente, um frozenset temporário é criado a partir de *elem*.

Conjuntos e conjuntos congelados (frozensets) são genéricos em relação ao tipo de seus elementos.

Ver também

Para informações detalhadas sobre as garantias de segurança de thread para objetos `set`, consulte Segurança de thread para objetos conjunto.

Tipos mapeamentos — `dict`, `frozendict`

Um objeto mapeamento mapeia valores hasheáveis para objetos arbitrários. Existe atualmente no momento dois tipos de mapeamentos padrão, o *dicionário* e `frozendict`. (Para outros contêineres, veja as classes embutidas `list`, `set` e `tuple`, e o módulo `collections`.) Consulte Complexidade de tempo de operações em tipos embutidos para os custos de várias operações de dicionário.

As chaves de um dicionário são *quase* valores arbitrários. Valores que não são hasheáveis, ou seja, valores contendo listas, dicionários ou outros tipos mutáveis (que são comparados por valor e não por identidade de objeto) não podem ser usados como chaves. Valores que comparam iguais (como `1`, `1.0` e `True`) podem ser usados alternadamente para indexar a mesma entrada do dicionário.

class dict(***kwargs*)
class dict(*mapping*, /, ***kwargs*)
class dict(*iterable*, /, ***kwargs*)

Retorna um novo dicionário inicializado a partir de um argumento posicional opcional, e um conjunto de argumentos nomeados possivelmente vazio.

Os dicionários podem ser criados de várias formas:

- Usar uma lista de pares `key: value` separados por vírgula com chaves: `{'jack': 4098, 'sjoerd': 4127}` ou `{4098: 'jack', 4127: 'sjoerd'}`
- Usar uma compreensão de dicionário: `{}`, `{x: x ** 2 for x in range(10)}`
- Usar o construtor de tipo: `dict()`, `dict([('foo', 100), ('bar', 200)])`, `dict(foo=100, bar=200)`

Se nenhum argumento posicional é fornecido, um dicionário vazio é criado. Se um argumento posicional é fornecido e ele define um método `keys()`, um dicionário é criado chamando `__getitem__()` no argumento com cada chave retornada pelo método. Caso contrário, o argumento posicional deve ser um objeto iterável. Cada item no iterável deve ser por si mesmo um iterável com exatamente dois elementos. O primeiro elemento de cada item torna-se a chave no novo dicionário, e o segundo elemento, o valor correspondente. Se a chave ocorrer mais do que uma vez, o último valor para aquela chave torna-se o valor correspondente no novo dicionário.

Se argumentos nomeados são fornecidos, os argumentos nomeados e seus valores são adicionados ao dicionário criado a partir do argumento posicional. Se uma chave sendo adicionada já está presente, o valor do argumento nomeado substitui o valor do argumento posicional.

Dicionários são iguais se e somente se eles os mesmos pares `(key, value)` (independente de ordem). Comparações de ordem (‘<’, ‘<=’, ‘>=’, ‘>’) levantam `TypeError`. Para ilustrar a criação e igualdade de dicionários, os exemplos a seguir retornam um dicionário igual a `{"one": 1, "two": 2, "three": 3}`:

```
>>> a = dict(one=1, two=2, three=3)
>>> b = {'one': 1, 'two': 2, 'three': 3}
>>> c = dict(zip(['one', 'two', 'three'], [1, 2, 3]))
>>> d = dict([('two', 2), ('one', 1), ('three', 3)])
>>> e = dict({'three': 3, 'one': 1, 'two': 2})
>>> f = dict({'one': 1, 'three': 3}, two=2)
>>> a == b == c == d == e == f
True
```

Fornecer argumentos nomeados conforme no primeiro exemplo somente funciona para chaves que são identificadores válidos no Python. Caso contrário, quaisquer chaves válidas podem ser usadas.

Dicionários preservam a ordem de inserção. Perceba que atualizar a chave não afeta a ordem. Chaves adicionadas após a deleção são inseridas no final.

```
>>> d = {"um": 1, "dois": 2, "três": 3, "quatro": 4}
>>> d
{'um': 1, 'dois': 2, 'três': 3, 'quatro': 4}
>>> list(d)
['um', 'dois', 'três', 'quatro']
>>> list(d.values())
[1, 2, 3, 4]
>>> d["um"] = 42
>>> d
{'um': 42, 'dois': 2, 'três': 3, 'quatro': 4}
>>> del d["dois"]
>>> d["dois"] = None
>>> d
{'um': 42, 'três': 3, 'quatro': 4, 'dois': None}
```

Alterado na versão 3.7: Ordem do dicionário é garantida conforme a ordem de inserção. Este comportamento era um detalhe de implementação do CPython a partir da versão 3.6.

Dicionários são genéricos em relação a dois tipos, representando (respectivamente) os tipos das chaves e dos valores do dicionário.

Estas são as operações que dicionários aceitam (e, portanto, tipos de mapeamento personalizados devem aceitar também):

list(d)

Retorna uma lista de todas as chaves usadas no dicionário *d*.

len(d)

Retorna o número de itens no dicionário *d*.

d[key]

Retorna o item de *d* com a chave *key*. Levanta um **`KeyError`** se *key* não estiver no mapeamento.

Se uma subclasse de um dict define um método **`__missing__()`** e *key* não estiver presente, a operação `d[key]` chama aquele método com a chave *key* como argumento. A operação `d[key]` então retorna ou levanta o que é retornado ou levantado pela chamada de `__missing__(key)`. Nenhuma operação ou métodos invocam **`__missing__()`**. Se **`__missing__()`** não for definido, então **`KeyError`** é levantada. **`__missing__()`** deve ser um método; ele não pode ser uma variável de instância:

```
>>> class Counter(dict):
...     def __missing__(self, key):
...         return 0
...
>>> c = Counter()
>>> c['red']
0
>>> c['red'] += 1
>>> c['red']
1
```

O exemplo acima mostra parte da implementação de **`collections.Counter`**. Um método **`__missing__()`** diferente é usado para **`collections.defaultdict`**.

d[key] = value

Define `d[key]` para *value*.

del d[key]

Remove `d[key]` do *d*. Levanta uma exceção **`KeyError`** se *key* não estiver no mapeamento.

key in d

Retorna `True` se *d* tiver uma chave *key*, caso contrário `False`.

key not in d

Equivalente a `not key in d`.

iter(d)

Retorna um iterador para as chaves do dicionário. Isso é um atalho para `iter(d.keys())`.

clear()

Remove todos os itens do dicionário.

copy()

Retorna uma cópia superficial do dicionário.

classmethod fromkeys(*iterable*, *value=None*, /)

Cria um novo dicionário com chaves provenientes de *iterable* e valores definidos como *value*.

`fromkeys()` é um método de classe que retorna um novo dicionário. *value* tem como valor padrão `None`. Todos os valores referem-se a apenas uma única instância, então geralmente não faz sentido que *value* seja um objeto mutável tal como uma lista vazia. Para obter valores distintos, use uma compreensão de dicionário ao invés.

get(*key*, *default=None*, /)

Retorna o valor para *key* se *key* está no dicionário, caso contrário *default*. Se *default* não é fornecido, será usado o valor padrão `None`, de tal forma que este método nunca levanta um **`KeyError`**.

items()

Retorna uma nova visão dos itens do dicionário (pares de `(key, value)`). Veja a documentação de objetos de visão de dicionário.

keys()

Retorna uma nova visão das chaves do dicionário. Veja a documentação de objetos de visão de dicionário.

pop(*key*, /)
pop(*key*, *default*, /)

Se *key* está no dicionário, remove a mesma e retorna o seu valor, caso contrário retorna *default*. Se *default* não foi fornecido e *key* não está no dicionário, um `KeyError` é levantado.

popitem()

Remove e retorna um par `(key, value)` do dicionário. Pares são retornados como uma pilha, ou seja em ordem LIFO.

`popitem()` é útil para destrutivamente iterar sobre um dicionário, algo comumente usado em algoritmos de conjunto. Se o dicionário estiver vazio, chamar **`popitem()`** levanta um `KeyError`.

Alterado na versão 3.7: Ordem LIFO agora é garantida. Em versões anteriores, `popitem()` iria retornar um par chave/valor arbitrário.

reversed(d)

Retorna um iterador revertido sobre as chaves do dicionário. Isso é um atalho para `reversed(d.keys())`.

Adicionado na versão 3.8.

setdefault(*key*, *default=None*, /)

Se *key* está no dicionário, retorna o seu valor. Se não, insere *key* com o valor *default* e retorna *default*. *default* por padrão usa o valor `None`.

update(***kwargs*)
update(*mapping*, /, ***kwargs*)
update(*iterable*, /, ***kwargs*)

Atualiza o dicionário com os pares chave/valor de *mapping* ou *iterable* e *kwargs*, sobrescrevendo chaves existentes. Retorna `None`.

`update()` aceita ou outro objeto com um método `keys()` (em cada caso `__getitem__()` é chamado com cada chave retornada pelo método) ou um iterável de pares de chave/valor (como tuplas ou outros iteráveis de comprimento dois). Se argumentos nomeados são especificados, o dicionário

é então atualizado com esses pares de chave/valor: `d.update(red=1, blue=2)`.

values()

Retorna uma nova visão dos valores do dicionário. Veja a documentação de objetos de visão de dicionário.

Uma comparação de igualdade entre uma visão de `dict.values()` e outra, sempre irá retornar `False`. Isso também se aplica ao comparar `dict.values()` entre si:

```
>>> d = {'a': 1}
>>> d.values() == d.values()
False
```

d | other

Cria um novo dicionário com as chaves e os valores mesclados de *d* e *other*, que devem ser dicionários. Os valores de *other* têm prioridade quando *d* e *other* compartilham chaves.

Adicionado na versão 3.9.

d |= other

Atualiza o dicionário *d* com chaves e valores de *other*, que podem ser a mapeamento ou um iterável dos pares chave/valor. Os valores de *other* têm prioridade quando *d* e *other* compartilham chaves.

Adicionado na versão 3.9.

Dicionários e visões de dicionários são reversíveis.

```
>>> d = {"um": 1, "dois": 2, "três": 3, "quatro": 4}
>>> d
{'um': 1, 'dois': 2, 'três': 3, 'quatro': 4}
>>> list(reversed(d))
['quatro', 'três', 'dois', 'um']
>>> list(reversed(d.values()))
[4, 3, 2, 1]
>>> list(reversed(d.items()))
[('quatro', 4), ('três', 3), ('dois', 2), ('um', 1)]
```

Alterado na versão 3.8: Dicionários agora são reversíveis.

Ver também

`frozendict` e `types.MappingProxyType` podem ser usados para criar uma visão somente leitura de um `dict`.

Ver também

Para informações detalhadas sobre as garantias de segurança de thread para objetos `dict`, consulte Segurança de threads para objetos dict.

Objetos de visão de dicionário

Os objetos retornados por `dict.keys()`, `dict.values()` e `dict.items()` são *objetos de visão*. Eles fornecem uma visão dinâmica das entradas do dicionário, o que significa que quando o dicionário muda, a visão reflete essas mudanças.

Visões de dicionários podem ser iteradas para apresentar seus respectivos dados, e suportar testes de existência:

len(dictview)

Retorna o número de entradas no dicionário.

iter(dictview)

Retorna um iterador sobre as chaves, valores ou itens (representados como tuplas de `(key, value)`) no dicionário.

Chaves e valores são iterados em ordem de inserção. Isso permite a criação de pares `(value, key)` usando `zip()`: `pairs = zip(d.values(), d.keys())`. Outra maneira de criar a mesma lista é `pairs = [(v, k) for (k, v) in d.items()]`.

Iterar sobre visões enquanto adiciona ou deleta entradas no dicionário pode levantar um `RuntimeError` ou falhar a iteração sobre todas as entradas.

Alterado na versão 3.7: Ordem do dicionário é garantida como a ordem de inserção.

x in dictview

Retorna `True` se *x* está nas chaves, valores ou itens do dicionário subjacente (no último caso, *x* deve ser uma tupla de `(key, value)`).

reversed(dictview)

Retorna um iterador reverso sobre as chaves, valores ou itens do dicionário. A visão será iterada na ordem reversa de inserção.

Alterado na versão 3.8: Visões de dicionário agora são reversíveis.

dictview.mapping

Retorna um `types.MappingProxyType` que envolve o dicionário original ao qual a visão se refere.

Adicionado na versão 3.10.

Visões chave são similar a conjunto, como suas entradas são únicas e hasheáveis. As visões de itens também têm operações semelhantes a conjuntos, uma vez que os pares (chave, valor) são únicos e as chaves são hasheáveis. Se todos os valores em uma visão de itens também forem hasheáveis, a visão de itens poderá interoperar com outros conjuntos. (Visões de valores não são tratadas de como similar a conjunto, pois as entradas geralmente não são únicas.) Para visões similares a conjunto, todas as operações definidas para a classe base abstrata `collections.abc.Set` estão disponíveis (por exemplo, `==`, `<` ou `^`). Ao usar operadores de conjunto, visões de conjunto ou similares aceitam qualquer iterável como outro operando, ao contrário de conjuntos que aceitam apenas conjuntos como entrada.

Um exemplo da utilização da visualização de dicionário:

```
>>> dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
>>> keys = dishes.keys()
>>> values = dishes.values()

>>> # iteration
>>> n = 0
>>> for val in values:
...     n += val
...
>>> print(n)
504

>>> # a iteração sobre chaves e valores é feita na mesma ordem (ordem d
>>> list(keys)
['eggs', 'sausage', 'bacon', 'spam']
>>> list(values)
[2, 1, 1, 500]

>>> # objetos de visualização são dinâmicos e refletem mudanças no dici
>>> del dishes['eggs']
```

```
>>> del dishes['sausage']
>>> list(keys)
['bacon', 'spam']

>>> # operações de conjunto
>>> keys & {'eggs', 'bacon', 'salad'}
{'bacon'}
>>> keys ^ {'sausage', 'juice'} == {'juice', 'sausage', 'bacon', 'spam'
True
>>> keys | ['juice', 'juice', 'juice'] == {'bacon', 'spam', 'juice'}
True

>>> # recupera um proxy somente de leitura para o dicionário original
>>> values.mapping
mappingproxy({'bacon': 1, 'spam': 500})
>>> values.mapping['spam']
500
```

Dicionários congelados

class frozendict(***kwargs*)

class frozendict(*mapping*, *, ***kwargs*)

class frozendict(*iterable*, *, ***kwargs*)

Retorna um novo dicionário congelado inicializado a partir de um argumento posicional opcional, e um conjunto de argumentos nomeados possivelmente vazio.

Um **`frozendict`** possui uma API semelhante à do **`dict`**, com as seguintes diferenças:

- **`dict`** tem mais métodos que **`frozendict`**:
  - **`__delitem__()`**
  - **`__setitem__()`**
  - **`clear()`**
  - **`pop()`**
  - **`popitem()`**
  - **`setdefault()`**
  - **`update()`**
- Um **`frozendict`** pode ser hasheado com `hash(frozendict)` se todas as chaves e valores puderem ser hasheados.

- `frozendict |= other` não modifica o **`frozendict`** no local (*in-place*), mas cria um novo dicionário congelado.

`frozendict` não é uma subclasse de **`dict`**, mas herda diretamente de `object`.

De forma similar aos dicionários, frozendicts são genéricos em relação a dois tipos, representando (respectivamente) os tipos das chaves e dos valores do frozendict.

classmethod fromkeys(*iterable*, *value=None*, /)

Semelhante a **`dict.fromkeys()`**, mas chama novamente o construtor do tipo com um **`frozendict`** inicializado se o tipo for uma subclasse de **`frozendict`** ou se o construtor tiver retornado um **`frozendict`**.

Adicionado na versão 3.15.

Tipos de Gerenciador de Contexto

A instrução **`with`** do Python suporta o conceito de um contexto em tempo de execução definido por um gerenciador de contexto. Isto é implementado usando um par de métodos, que permitem que classes definidas pelo usuário especifiquem um contexto em tempo de execução, o qual é inicializado antes da execução das instruções, e encerrado quando as instruções terminam:

contextmanager.__enter__()

Entra no contexto em tempo de execução e retorna este objeto ou outro objeto relacionado ao contexto em tempo de execução. O valor retornado por este método é ligado ao identificador na cláusula **`as`** das instruções **`with`** usando este gerenciador de contexto.

Um exemplo de gerenciador de contexto que retorna a si mesmo é um objeto arquivo. Objeto arquivos retornam a si mesmos do método __enter__() para permitir que **`open()`** seja usado como a expressão de contexto em uma instrução **`with`**.

Um exemplo de gerenciador de contexto que retorna um objeto relacionado é aquele retornado por **`decimal.localcontext()`**. Esses gerenciadores definem o contexto decimal ativo para uma cópia do contexto decimal original, e então retornam a cópia. Isso permite que mudanças sejam feitas no contexto decimal atual, no corpo contido na instrução **`with`**, sem afetar o código fora da instrução **`with`**.

contextmanager.__exit__(*exc_type*, *exc_val*, *exc_tb*)

Sai do contexto em tempo de execução e retorna um sinalizador booleano indicando se qualquer exceção que ocorreu deve ser suprimida. Se uma exceção ocorreu enquanto era executado o corpo da instrução `with`, os argumentos contêm o tipo da exceção, valor e informação da traceback (situação da pilha de execução). Caso contrário, os três argumentos são `None`.

Retornar um valor verdadeiro deste método fará com que a instrução `with` suprima a exceção e continue a execução com a instrução imediatamente após a instrução **`with`**. Caso contrário a exceção continuará propagando após este método ter encerrado sua execução.

Se este método levantar uma exceção enquanto trata uma exceção anterior do bloco `with`, a nova exceção é levantada e a exceção original é armazenada em seu atributo `__context__`.

A exceção passada nunca deve ser re-levantada explicitamente - ao invés disso, este método deve retornar um valor falso para indicar que o método completou sua execução com sucesso, e não quer suprimir a exceção levantada. Isso permite ao código do gerenciador de contexto facilmente detectar se um método `__exit__()` realmente falhou ou não.

Python define diversos gerenciadores de contexto para suportar facilmente sincronização de threads, solicita o fechamento de arquivos ou outros objetos, e manipula de forma simples o contexto ativo de aritmética decimal. Os tipos especificados não são tratados de forma especial além da sua implementação e do protocolo do gerenciador de contexto. Veja o módulo `contextlib` para alguns exemplos.

Os geradores do Python e o decorador `contextlib.contextmanager` fornecem uma maneira conveniente de implementar esses protocolos. Se uma função geradora for decorada com o decorador **`contextlib.contextmanager`**, ela retornará um gerenciador de contexto implementando os métodos necessários `__enter__()` e `__exit__()`, em vez de o iterador produzido por uma função geradora não decorada.

Observe que não existe nenhum slot específico para qualquer um desses métodos na estrutura de tipos para objetos Python na API Python/C. Tipos de extensão que desejam definir estes métodos devem fornecê-los como um método acessível normal do Python. Comparado com a sobrecarga de configurar o contexto em tempo de execução, a sobrecarga na pesquisa de dicionário em uma única classe é negligenciável.

Tipos de anotação de tipo — Apelido genérico, União

Os principais tipos embutidos para anotações de tipo são Apelido genérico e União.

Tipo Generic Alias

Objetos `GenericAlias` são geralmente criados subscrevendo uma classe. Eles são mais usados com classes contêineres, como **`list`** ou **`dict`**. Por exemplo, `list[int]` é um objeto `GenericAlias` criado pela subscrição da classe `list` com o argumento **`int`**. Objetos `GenericAlias` são destinados principalmente para uso com anotações de tipo.

> **Nota**
>
> Geralmente só é possível subscrever uma classe se a classe implementar o método especial **`__class_getitem__()`**.

Um objeto `GenericAlias` atua como um proxy para um tipo genérico, implementando *genéricos parametrizados*.

Para uma classe contêiner, o(s) argumento(s) fornecido(s) para uma subscrição da classe pode indicar o(s) tipo(s) dos elementos que um objeto contém. Por exemplo, `set[bytes]` pode ser usado em anotações de tipo para significar um **`set`** em que todos os elementos são do tipo **`bytes`**.

Para uma classe que define **`__class_getitem__()`**, mas não é um contêiner, o(s) argumento(s) fornecido(s) para uma subscrição da classe geralmente indicará o(s) tipo(s) de retorno de um ou mais métodos definidos em um objeto. Por exemplo, **`expressões regulares`** podem ser usadas tanto no tipo de dados **`str`** quanto no tipo de dados **`bytes`**:

- Se `x = re.search('foo', 'foo')`, `x` será um objeto re.Match onde os valores de retorno de `x.group(0)` e `x[0]` serão ambos do tipo **`str`**. Podemos representar este tipo de objeto em anotações de tipo com o `GenericAlias` `re.Match[str]`.
- Se `y = re.search(b'bar', b'bar')` (observe o `b` para **`bytes`**), `y` também será uma instância de `re.Match`, mas os valores de retorno de `y.group(0)` e `y[0]` serão ambos do tipo **`bytes`**. Em anotações de tipo, representaríamos esta variedade de objetos re.Match com `re.Match[bytes]`.

Objetos `GenericAlias` são instâncias da classe **`types.GenericAlias`**, que também pode ser usada para criar objetos `GenericAlias` diretamente. Especializações de classes genéricas definidas pelo usuário podem não ser instâncias de **`types.GenericAlias`**, mas oferecem funcionalidade semelhante.

T[X, Y, ...]

Cria um `GenericAlias` representando um tipo `T` parametrizado por tipos *X*, *Y* e mais, dependendo do `T` usados. Por exemplo, uma função esperando uma **`list`** contendo elementos **`float`**:

```
def average(values: list[float]) -> float:
    return sum(values) / len(values)
```

Outro exemplo para mapeamento de objetos, usando um **`dict`**, o qual é um tipo genérico esperando 2 tipos de parâmetros representando o tipo da chave e o tipo do valor. Neste exemplo, a função espera um `dict` com chaves do tipo **`str`** e valores do tipo **`int`**:

```
def send_post_request(url: str, body: dict[str, int]) -> None:
    ...
```

As funções embutidas **`isinstance()`** e **`issubclass()`** não aceitam tipos `GenericAlias` para o seu segundo argumento:

```
>>> isinstance([1, 2], list[str])
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: isinstance() argument 2 cannot be a parameterized generic
```

O runtime Python não obriga anotações de tipo. Isso se aplica a tipos genéricos e seus parâmetros tipados. Ao criar um objeto contêiner a partir de um `GenericAlias`, os elementos no contêiner não são verificados pelo seu tipo. Por exemplo, o seguinte código é desencorajado, mas irá executar sem erros:

```
>>> t = list[str]
>>> t([1, 2, 3])
[1, 2, 3]
```

Além disso, genéricos parametrizados removem parâmetros tipados durante a criação do objeto:

```
>>> t = list[str]
>>> type(t)
<class 'types.GenericAlias'>
```

```
>>> l = t()
>>> type(l)
<class 'list'>
```

Instâncias de `GenericAlias` não são classes em tempo de execução, embora se comportem como classes (podem ser instanciadas e servir de base para subclasses):

```
>>> import inspect
>>> inspect.isclass(list[int])
False
```

Isso é verdadeiro para genéricos definidos pelo usuário também.

Chamar **`repr()`** ou **`str()`** sobre um genérico mostra o tipo parametrizado:

```
>>> repr(list[int])
'list[int]'

>>> str(list[int])
'list[int]'
```

O método **`__getitem__()`** de contêineres genéricos irá levantar uma exceção para não permitir erros como `dict[str][str]`:

```
>>> dict[str][str]
Traceback (most recent call last):
  ...
TypeError: dict[str] is not a generic class
```

Entretanto, tais expressões são válidas quando type variáveis são usadas. O índice deve ter tantos elementos quantos forem os itens de variável de tipo no objeto `GenericAlias` **`__args__`**.

```
>>> from typing import TypeVar
>>> Y = TypeVar('Y')
>>> dict[str, Y][int]
dict[str, int]
```

Classes genéricas padrão

As seguintes classes de biblioteca padrão oferecem suporte a genéricos parametrizados. Esta lista não é exaustiva.

- **`tuple`**
- **`list`**

- dict
- set
- frozendict
- frozenset
- type
- asyncio.Future
- asyncio.Task
- collections.deque
- collections.defaultdict
- collections.OrderedDict
- collections.Counter
- collections.ChainMap
- collections.abc.Awaitable
- collections.abc.Coroutine
- collections.abc.AsyncIterable
- collections.abc.AsyncIterator
- collections.abc.AsyncGenerator
- collections.abc.Iterable
- collections.abc.Iterator
- collections.abc.Generator
- collections.abc.Reversible
- collections.abc.Container
- collections.abc.Collection
- collections.abc.Callable
- collections.abc.Set
- collections.abc.MutableSet
- collections.abc.Mapping
- collections.abc.MutableMapping
- collections.abc.Sequence
- collections.abc.MutableSequence
- collections.abc.ByteString
- collections.abc.MappingView
- collections.abc.KeysView
- collections.abc.ItemsView
- collections.abc.ValuesView
- contextlib.AbstractContextManager
- contextlib.AbstractAsyncContextManager
- dataclasses.Field
- functools.cached_property
- functools.partialmethod
- os.PathLike

- `queue.LifoQueue`
- `queue.Queue`
- `queue.PriorityQueue`
- `queue.SimpleQueue`
- re.Pattern
- re.Match
- `shelve.BsdDbShelf`
- `shelve.DbfilenameShelf`
- `shelve.Shelf`
- `types.MappingProxyType`
- `weakref.WeakKeyDictionary`
- `weakref.WeakMethod`
- `weakref.WeakSet`
- `weakref.WeakValueDictionary`

Atributos especiais de objetos `GenericAlias`

Todos os genéricos parametrizados implementam atributos especiais somente leitura.

genericalias.__origin__

Este atributo aponta para a classe genérica não parametrizada:

```
>>> list[int].__origin__
<class 'list'>
```

genericalias.__args__

Este atributo é uma `tuple` (possivelmente de comprimento 1) de tipos genéricos passado para o método `__class_getitem__()` original da classe genérica:

```
>>> dict[str, list[int]].__args__
(<class 'str'>, list[int])
```

genericalias.__parameters__

O atributo é uma tupla computada preguiçosamente (possivelmente vazia) de variáveis de tipo único encontradas em `__args__`:

```
>>> from typing import TypeVar

>>> T = TypeVar('T')
>>> list[T].__parameters__
(~T,)
```

Nota

Um objeto `GenericAlias` com os parâmetros `typing.ParamSpec` pode não ter `__parameters__` correto após a substituição porque `typing.ParamSpec` se destina principalmente à verificação de tipo estático.

genericalias.__unpacked__

Um booleano que é true se o apelido foi desempacotado usando o operador `*` (consulte `TypeVarTuple`).

Adicionado na versão 3.11.

Ver também

PEP 484 [https://peps.python.org/pep-0484/] - Dicas de tipo

Apresentando a estrutura do Python para anotações de tipo.

PEP 585 [https://peps.python.org/pep-0585/] - Sugestão de tipo para Genéricos em coleções padrão

Apresentando a capacidade de parametrizar nativamente as classes da biblioteca padrão, desde que implementem o método de classe especial `__class_getitem__()`.

Genéricos, genéricos definidos pelo usuário e `typing.Generic`

Documentação sobre como implementar classes genéricas que podem ser parametrizadas em tempo de execução e compreendidas por verificadores de tipo estático.

Adicionado na versão 3.9.

Tipo União

Um objeto união contém o valor da operação `|` (bit a bit ou) em vários objetos tipo. Esses tipos são destinados principalmente para anotações de tipos. A expressão de tipo de união habilita a sintaxe de sugestão de tipo mais limpo em comparação com subscrever `typing.Union`.

X | Y | ...

Define um objeto união que contém os tipos *X*, *Y* e assim por diante. `X | Y` significa X ou Y. É equivalente a `typing.Union[X, Y]`. Por exemplo, a seguinte

função espera um argumento do tipo `int` ou `float`:

```
def square(number: int | float) -> int | float:
    return number ** 2
```

Nota

O operador `|` não pode ser usado em tempo de execução para definir uniões onde um ou mais membros são uma referência direta. Por exemplo, `int | "Foo"`, onde `"Foo"` é uma referência a uma classe ainda não definida, falhará em tempo de execução. Para uniões que incluem referências futuras, apresente a expressão inteira como uma string, por exemplo. `"int | Foo"`.

union_object == other

Os objetos união podem ser testados quanto à igualdade com outros objetos união. Detalhes:

- Uniões de uniões são achatadas:

  ```
  (int | str) | float == int | str | float
  ```

- Tipos redundantes são removidos:

  ```
  int | str | int == int | str
  ```

- Ao comparar uniões, a ordem é ignorada:

  ```
  int | str == str | int
  ```

- Cria instâncias de `typing.Union`:

  ```
  int | str == typing.Union[int, str]
  type(int | str) is typing.Union
  ```

- Tipos opcionais podem ser escritos como uma união com `None`:

  ```
  str | None == typing.Optional[str]
  ```

isinstance(obj, union_object)

issubclass(obj, union_object)

Chamadas para `isinstance()` e `issubclass()` também são suportados com um objeto união:

```
>>> isinstance("", int | str)
True
```

No entanto, genéricos parametrizados em objetos de união não podem ser verificados:

```
>>> isinstance(1, int | list[int])  # cálculo em curto-circuito
True
>>> isinstance([1], int | list[int])
Traceback (most recent call last):
  ...
TypeError: isinstance() argument 2 cannot be a parameterized generic
```

O tipo exposto ao usuário para o objeto união pode ser acessado a partir de **`typing.Union`** e usado por verificações de **`isinstance()`**:

```
>>> import typing
>>> isinstance(int | str, typing.Union)
True
>>> typing.Union()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: cannot create 'typing.Union' instances
```

Nota

O método **`__or__()`** para objetos de tipo foi adicionado para suportar a sintaxe `X | Y`. Se uma metaclasse implementa **`__or__()`**, a União pode substituí-la:

```
>>> class M(type):
...     def __or__(self, other):
...         return "Hello"
...
>>> class C(metaclass=M):
...     pass
...
>>> C | int
'Hello'
>>> int | C
int | C
```

Ver também

PEP 604 [https://peps.python.org/pep-0604/] – PEP propondo a sintaxe `X | Y` e o tipo União.

Adicionado na versão 3.10.

Alterado na versão 3.14: Objetos Union agora são instâncias de `typing.Union`. Anteriormente, eram instâncias de `types.UnionType`, que continua sendo um apelido para `typing.Union`.

Outros tipos embutidos

O interpretador suporta diversos outros tipos de objetos. Maior parte desses, suporta apenas uma ou duas operações.

Módulos

A única operação especial em um módulo é o acesso a um atributo: `m.name`, onde *m* é um módulo e *name* acessa o nome defino na tabela de símbolos de *m*. Atributos de módulo podem receber atribuição. (Perceba que a instrução `import` não é, estritamente falando, uma operação em um objeto do módulo; `import foo` não requer que um objeto do módulo chamado *foo* exista, ao invés disso requer uma *definição* (externa) de um módulo chamado *foo* em algum lugar.)

Um atributo especial de cada módulo é `__dict__`. Este é o dicionário contendo a tabela de símbolos do modulo. Modificar este dicionário vai na verdade modificar a tabela de símbolos do módulo, mas atribuição direta para o atributo `__dict__` não é possível (você pode escrever `m.__dict__['a'] = 1`, o qual define `m.a` para ser `1`, mas você não consegue escrever `m.__dict__ = {}`). Modificar `__dict__` diretamente não é recomendado.

Módulos embutidos no interpretador são escritos desta forma: `<module 'sys' (built-in)>`. Se carregados a partir de um arquivo, eles são escritos como `<module 'os' from '/usr/local/lib/pythonX.Y/os.pyc'>`.

Classes e Instâncias de Classes

Veja Objetos, valores e tipos e Definições de classe para estes.

Funções

Objetos função são criados através da definição de funções. A única operação que pode ser feita em um objeto função é chamá-la: `func(lista-de-argumentos)`.

Existem na verdade duas possibilidades de objetos função: funções embutidas e funções definidas pelo usuário. Ambas suportam a mesma operação (chamar a função), mas a implementação é diferente, portanto os diferentes tipos de objetos.

Veja a função Definições de função para mais informações.

Métodos

Métodos são funções que são chamadas usando a notação de atributo. Existem duas opções: métodos embutidos (tal como **`append()`** em listas) e métodos de instância de classe. Métodos embutidos são descritos com os tipos que os implementam.

Se você acessar um método (uma função definida em um espaço de nomes de uma classe) através de uma instância, você obtém um objeto especial: um objeto com *método vinculado* (também chamado de método de instância). Quando chamado, ele irá adicionar o argumento `self` para a lista de argumentos. Métodos vinculados tem dois atributos somente leitura especiais: **`m.__self__`** é o objeto no qual o método opera, e **`m.__func__`** é a função que implementa o método. Chamar `m(arg-1, arg-2, ..., arg-n)` é completamente equivalente a chamar `m.__func__(m.__self__, arg-1, arg-2, ..., arg-n)`.

Assim como objetos função, objetos de métodos vinculados tem suporte para receber atributos arbitrários. Entretanto, como atributos de método na verdade são armazenados no objeto função subjacente (**`method.__func__`**), definir atributos de método em métodos vinculados não é permitido. Tentar definir um atributo em um método resulta em um **`AttributeError`** sendo levantado. A fim de definir um atributo de método, você precisa definir explicitamente ele no objeto função subjacente:

```
>>> class C:
...     def method(self):
...         pass
...
>>> c = C()
>>> c.method.whoami = 'my name is method'  # proibido atribuir diretame
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'method' object has no attribute 'whoami'
>>> c.method.__func__.whoami = 'my name is method'
```

```
>>> c.method.whoami
'my name is method'
```

Veja Métodos de instância para mais informações.

Objetos código

Objetos código são usados pela implementação para representar código Python executável “pseudo-compilado”, tal como corpo de uma função. Eles são diferentes de objetos função porque eles não contém uma referência para os seus ambientes de execução global. Objetos código são retornados pela função embutida **`compile()`** e podem ser extraídos de objetos função através do seu atributo **`__code__`**. Veja também o módulo **`code`**.

Acessar **`__code__`** levanta um evento de auditoria `object.__getattr__` com os argumentos `obj` e `"__code__"`.

Um objeto de código pode ser executado ou avaliado passando-o (ao invés da string fonte) para as funções embutidas **`exec()`** ou **`eval()`**.

Veja o tipo A hierarquia de tipos padrão para maiores informações.

Objetos tipo

Objetos de tipos representam os vários tipos de objetos. Um tipo de um objeto é acessado pela função embutida **`type()`**. Não existem operações especiais sobre tipos. O módulo padrão **`types`** define nomes para todos os tipos padrão embutidos.

Tipos são escritos como isto: `<class 'int'>`.

O objeto nulo

Este objeto é retornado por funções que não retornam um valor explicitamente. Ele não suporta operações especiais. Existe exatamente um objeto nulo, chamado de `None` (um nome embutido). `type(None)()` produz o mesmo singleton.

Ele é escrito como `None`.

O Objeto Ellipsis

Este objeto é comumente usado para indicar que algo foi omitido. Ela não suporta operações especiais. Existe exatamente um objeto ellipsis, nomeado **`Ellipsis`** (um nome embutido). `type(Ellipsis)()` produz o singleton **`Ellipsis`**.

Está escrito com `Ellipsis` ou `...`.

No uso típico, `...` como o objeto `Ellipsis` aparece em alguns lugares diferentes, por exemplo:

- Em anotações de tipo, como argumentos chamáveis ou elementos de tupla.
- Como o corpo de uma função em vez de uma instrução pass.
- Em bibliotecas de terceiros, como slicing e striding do Numpy [https://numpy.org/doc/stable/user/basics.indexing.html#slicing-and-striding].

Python também usa três pontos de maneiras que não são objetos `Ellipsis`, por exemplo:

- **`ELLIPSIS`** do doctest, como um padrão para conteúdo ausente.
- O prompt Python padrão do console interativo quando a entrada parcial está incompleta.

Por fim, a documentação do Python geralmente usa três pontos no uso convencional do inglês para indicar conteúdo omitido, mesmo em exemplos de código que também os usam como `Ellipsis`.

O Objeto NotImplemented

Este objeto é retornado a partir de comparações e operações binárias quando elas são solicitadas para operar em tipos nos quais eles não suportam. Veja Comparações para mais informações. Existe exatamente um objeto **`NotImplemented`**. `type(NotImplemented)()` produz o mesmo valor.

Está escrito como `NotImplemented`.

Objetos Internos

Veja A hierarquia de tipos padrão para esta informação. Descreve objetos de quadro de pilha, objetos traceback (situação da pilha de execução), e objetos fatia.

Atributos Especiais

A implementação adiciona alguns atributos especiais somente leitura para diversos tipos de objetos, onde eles são relevantes. Alguns desses não são reportados pela função embutida `dir()`.

definition.__name__
: O nome da classe, função, método, descritor, ou instância geradora.

definition.__qualname__
: O nome qualificado da classe, função, método, descritor, ou instância geradora.

Adicionado na versão 3.3.

definition.__module__
: O nome do módulo no qual uma classe ou função foi definida.

definition.__doc__
: A string de documentação de uma classe ou função, ou `None` se não definida.

definition.__type_params__
: Os parâmetros de tipo de classes genéricas, funções e apelidos de tipo. Para classes e funções que não são genéricas, isso será uma tupla vazia.

Adicionado na versão 3.12.

Limitação de comprimento de string na conversão para inteiro

CPython tem um limite global para conversão entre `int` e `str` para mitigar ataques de negação de serviço. Esse limite *somente* se aplica a bases numéricas decimais ou outras que não sejam potência de dois. As conversões hexadecimais, octais e binárias são ilimitadas. O limite pode ser configurado.

O tipo `int` no CPython é um número de comprimento arbitrário armazenado em formato binário (comumente conhecido como “bignum”). Não existe nenhum algoritmo que possa converter uma string em um inteiro binário ou um inteiro binário em uma string em tempo linear, *a menos que* a base seja uma potência de 2. Mesmo os algoritmos mais conhecidos para a base 10 têm complexidade subquadrática. Converter um valor grande como `int('1' * 500_000)` pode levar mais de um segundo em uma CPU rápida.

Limitar o tamanho da conversão oferece uma maneira prática de evitar **CVE 2020-10735** [https://www.cve.org/CVERecord?id=CVE-2020-10735].

O limite é aplicado ao número de caracteres de dígitos na string de entrada ou saída quando um algoritmo de conversão não linear estiver envolvido. Sublinhados e o sinal não são contados para o limite.

Quando uma operação excede o limite, uma exceção `ValueError` é levantada:

```
>>> import sys
>>> sys.set_int_max_str_digits(4300)  # Illustrativo, este é o padrão.
>>> _ = int('2' * 5432)
Traceback (most recent call last):
...
ValueError: Exceeds the limit (4300 digits) for integer string conversi
>>> i = int('2' * 4300)
>>> len(str(i))
4300
>>> i_squared = i*i
>>> len(str(i_squared))
Traceback (most recent call last):
...
ValueError: Exceeds the limit (4300 digits) for integer string conversi
>>> len(hex(i_squared))
7144
>>> assert int(hex(i_squared), base=16) == i*i  # Conversão envolvendo
```

O limite padrão é de 4300 dígitos conforme fornecido em `sys.int_info.default_max_str_digits`. O limite mínimo que pode ser configurado é de 640 dígitos conforme fornecido em **`sys.int_info.str_digits_check_threshold`**.

Verificação:

```
>>> import sys
>>> assert sys.int_info.default_max_str_digits == 4300, sys.int_info
>>> assert sys.int_info.str_digits_check_threshold == 640, sys.int_info
>>> msg = int('578966293710682886880994035146873798396722250538762761 56
...           '92529255143839154833338127435805497794361047062606963666
...           '571186405732').to_bytes(53, 'big')
...
```

Adicionado na versão 3.11.

APIs afetadas

A limitação só se aplica a conversões potencialmente lentas entre `int` e `str` ou `bytes`:

- `int(string)` com padrão sendo base 10.
- `int(string, base)` para todas as bases que não são uma potência de 2.
- `str(integer)`.
- `repr(integer)`.
- qualquer outra conversão de string para base 10 como, por exemplo, `f"{integer}"`, `"{}".format(integer)` ou `b"%d" % integer`.

As limitações não se aplicam a funções com um algoritmo linear:

- `int(string, base)` com base 2, 4, 8, 16 ou 32.
- `int.from_bytes()` e `int.to_bytes()`.
- `hex()`, `oct()`, `bin()`.
- Minilinguagem de especificação de formato para números hexa, octal e binários
- `str` para `float`.
- `str` para `decimal.Decimal`.

Configurando o limiter

Antes de iniciar o Python, você pode usar uma variável de ambiente ou um sinalizador de linha de comando do interpretador para configurar o limite:

- `PYTHONINTMAXSTRDIGITS`, por exemplo `PYTHONINTMAXSTRDIGITS=640 python3` para definir o limite para 640 ou `PYTHONINTMAXSTRDIGITS=0 python3` para desabilitar a limitação.
- `-X int_max_str_digits`, por exemplo `python3 -X int_max_str_digits=640`
- `sys.flags.int_max_str_digits` contém o valor de `PYTHONINTMAXSTRDIGITS` ou `-X int_max_str_digits`. Se a variável env e a opção `-X` estiverem definidas, a opção `-X` terá precedência. Um valor de *-1* indica que ambos não foram definidos, portanto, um valor de `sys.int_info.default_max_str_digits` foi usado durante a inicialização.

A partir do código, você pode inspecionar o limite atual e definir um novo usando estas APIs `sys`:

- `sys.get_int_max_str_digits()` e `sys.set_int_max_str_digits()` são um getter e um setter para o limite de todo o interpretador. Os subinterpretadores têm seu próprio limite.

Informações sobre o padrão e o mínimo podem ser encontradas em `sys.int_info`:

- `sys.int_info.default_max_str_digits` é o limite padrão compilado.
- `sys.int_info.str_digits_check_threshold` é o menor valor aceito para o limite (diferente de 0 que o desabilita).

Adicionado na versão 3.11.

Cuidado

Definir um limite baixo *pode* levar a problemas. Embora raro, existe um código que contém constantes inteiras em decimal em sua origem que excedem o limite mínimo. Uma consequência de definir o limite é que o código-fonte do Python contendo literais inteiros decimais maiores que o limite encontrará um erro durante a análise, geralmente no momento da inicialização ou no momento da importação ou até mesmo no momento da instalação – sempre que um `.pyc` atualizado ainda não existe para o código. Uma solução alternativa para source que contém tais constantes grandes é convertê-las para a forma hexadecimal `0x`, pois não há limite.

Teste sua aplicação completamente se você usar um limite baixo. Certifique-se de que seus testes sejam executados com o limite definido anteriormente por meio do ambiente ou sinalizador para que ele seja aplicado durante a inicialização e até mesmo durante qualquer etapa de instalação que possa invocar o Python para pré-compilar fontes `.py` para arquivos `.pyc`.

Configuração recomendada

Espera-se que o padrão `sys.int_info.default_max_str_digits` seja razoável para a maioria das aplicações. Se sua aplicação exigir um limite diferente, defina-o em seu ponto de entrada principal usando código agnóstico de versão Python, pois essas APIs foram adicionadas em lançamentos de patch de segurança em versões anteriores a 3.12.

Exemplo:

```
>>> import sys
>>> if hasattr(sys, "set_int_max_str_digits"):
...     upper_bound = 68000
...     lower_bound = 4004
...     current_limit = sys.get_int_max_str_digits()
...     if current_limit == 0 or current_limit > upper_bound:
```

```
...         sys.set_int_max_str_digits(upper_bound)
...     elif current_limit < lower_bound:
...         sys.set_int_max_str_digits(lower_bound)
```

Se você precisar desativá-lo totalmente, defina-o como `0`.

Notas de rodapé

[1]

Informações adicionais sobre esses métodos especiais podem ser encontradas no Manual de Referência do Python (Customização básica).

[2]

Como uma consequência, a lista `[1, 2]` é considerada igual a `[1.0, 2.0]`, e similarmente para tuplas.

[3]

Eles precisam ter, já que o analisador sintático não consegue dizer o tipo dos operandos.

[4] (1,2,3,4)

Caracteres que possuem maiúsculo e minúsculo são aqueles com a propriedade de categoria geral igual a “Lu” (Letra, maiúscula), “Ll” (Letra, minúscula), ou “Lt” (Letra, em formato de título).

[5] (1,2)

Para formatar apenas uma tupla, você deve portanto fornecer uma tupla com apenas um elemento, que é a tupla a ser formatada.

Constantes embutidas

Um pequeno número de constantes são definidas no espaço de nomes embutido da linguagem. São elas:

False

O valor falso do tipo `bool`. As atribuições a `False` são ilegais e levantam `SyntaxError`.

True

O valor verdadeiro do tipo `bool`. As atribuições a `True` são ilegais e levantam `SyntaxError`.

None

Um objeto frequentemente usado para representar a ausência de um valor, como quando argumentos padrão não são passados para uma função. Atribuições a `None` são ilegais e levantam um `SyntaxError`. `None` é a única instância do tipo `NoneType`.

NotImplemented

Um valor especial que deve ser retornado pelos métodos especiais binários (por exemplo, `__eq__()`, `__lt__()`, `__add__()`, `__rsub__()`, etc.) para indicar que a operação não foi implementada em relação ao outro tipo; pode ser retornado pelos métodos especiais binários in-place (por exemplo, `__imul__()`, `__iand__()`, etc.) para o mesmo propósito. Ele não deve ser avaliado em um contexto booleano. **`NotImplemented`** é a única instância do tipo `types.NotImplementedType`.

Nota

Quando um método binário (ou local) retorna **`NotImplemented`**, o interpretador tentará a operação refletida no outro tipo (ou algum outro fallback, dependendo do operador). Se todas as tentativas retornarem **`NotImplemented`**, o interpretador levantará uma exceção apropriada. Retornar incorretamente **`NotImplemented`** resultará em

uma mensagem de erro enganosa ou no valor **`NotImplemented`** sendo retornado ao código Python.

Consulte Implementando as operações aritméticas para ver exemplos.

Cuidado

`NotImplemented` e **`NotImplementedError`** não são intercambiáveis. Esta constante deve ser usada somente conforme descrito acima; veja **`NotImplementedError`** para detalhes sobre o uso correto da exceção.

Alterado na versão 3.9: A avaliação de **`NotImplemented`** em um contexto booleano foi descontinuada.

Alterado na versão 3.14: A avaliação de **`NotImplemented`** em um contexto booleano agora levanta um **`TypeError`**. Anteriormente, ele era avaliado como **`True`** e emitia um **`DeprecationWarning`** desde o Python 3.9.

Ellipsis

O mesmo que o literal de elipse "`...`", um objeto frequentemente usado para indicar que algo foi omitido. A atribuição a `Ellipsis` é possível, mas a atribuição a `...` levanta um **`SyntaxError`**. `Ellipsis` é a única instância do tipo **`types.EllipsisType`**.

__debug__

Esta constante é verdadeira se o Python não foi iniciado com uma opção **`-O`**. Veja também a instrução **`assert`**.

Nota

Os nomes **`None`**, **`False`**, **`True`** e **`__debug__`** não podem ser reatribuídos (atribuições a eles, mesmo como um nome de atributo, levantam **`SyntaxError`**), para que possam ser consideradas "verdadeiras" constantes.

Constantes adicionadas pelo módulo `site`

O módulo `site` (que é importado automaticamente durante a inicialização, exceto se a opção de linha de comando `-S` for fornecida) adiciona várias constantes ao espaço de nomes embutido. Eles são úteis para o console do interpretador interativo e não devem ser usados em programas.

quit(*code=None*)
exit(*code=None*)

Objetos que, quando exibidos, exibem uma mensagem como “Use quit() or Ctrl-D (i.e. EOF) to exit” e, quando acessados diretamente no interpretador interativo ou chamados como funções, levantam `SystemExit` com o código de saída especificado.

help

Objeto que, quando exibido, exibe a mensagem “Type help() for interactive help, or help(object) for help about object.” e, quando acessado diretamente no interpretador interativo, invoca o sistema de ajuda embutido (veja `help()`).

copyright
credits

Objetos que ao serem impressos ou chamados, imprimem o texto dos direitos autorais ou créditos, respectivamente.

license

Objeto que, quando impresso, imprime a mensagem “Type license() to see the full license text” e, quando chamado, exibe o texto completo da licença de maneira semelhante a um paginador (uma tela por vez).

Funções embutidas

O interpretador do Python possui várias funções e tipos embutidos que sempre estão disponíveis. A seguir listamos todas as funções em ordem alfabética.

Funções embutidas

Funções embutidas

A
abs()
aiter()
all()
anext()
any()
ascii()

B
bin()
bool()
breakpoint()
bytearray()
bytes()

C
callable()
chr()
classmethod()
compile()
complex()

D
delattr()
dict()
dir()
divmod()

E
enumerate()
eval()
exec()

F
filter()
float()
format()
frozendict()
frozenset()

G
getattr()
globals()

H
hasattr()
hash()
help()
hex()

I
id()
input()
int()
isinstance()
issubclass()
iter()

L
len()
list()
locals()

M
map()
max()
memoryview()
min()

N
next()

O
object()
oct()
open()
ord()

P
pow()
print()
property()

R
range()
repr()
reversed()
round()

S
set()
setattr()
sentinel()
slice()
sorted()
staticmethod()
str()
sum()
super()

T
tuple()
type()

V
vars()

Z
zip()

_
__import__()

---

abs(*number*, /)

Retorna o valor absoluto de um número. O argumento pode ser um inteiro, um número de ponto flutuante ou um objeto implementando

`__abs__()`. Se o argumento é um número complexo, sua magnitude é retornada.

aiter(*async_iterable*, /)

Retorna um iterador assíncrono para um iterável assíncrono. Equivalente a chamar `x.__aiter__()`.

Nota: Ao contrário de `iter()`, `aiter()` não tem uma variante de 2 argumentos.

Adicionado na versão 3.10.

all(*iterable*, /)

Retorna `True` se todos os elementos de *iterable* são verdadeiros (ou se *iterable* estiver vazio). Equivalente a:

```
def all(iterable):
    for elemento in iterable:
        if not elemento:
            return False
    return True
```

awaitable anext(*async_iterator*, /)

awaitable anext(*async_iterator*, *default*, /)

Quando aguardado, retorna o próximo item do iterador assíncrono fornecido, ou *default* se fornecido e o iterador for esgotado.

Esta é a variante assíncrona do `next()` embutido, e se comporta de forma semelhante.

Isso chama o método `__anext__()` de *async_iterator*, retornando um aguardável. Ao aguardar isso, retorna o próximo valor do iterador. Se *default* for fornecido, ele será retornado se o iterador for esgotado, caso contrário, a exceção `StopAsyncIteration` será levantada.

Adicionado na versão 3.10.

any(*iterable*, /)

Retorna `True` se algum elemento de *iterable* for verdadeiro. Se *iterable* estiver vazio, retorna `False`. Equivalente a:

```
def any(iterable):
    for elemento in iterable:
        if elemento:
            return True
    return False
```

ascii(*object*, /)

Como **`repr()`**, retorna uma string contendo uma representação imprimível de um objeto, mas faz escape de caracteres não-ASCII na string retornada por **`repr()`** usando sequências de escapes `\x`, `\u` ou `\U`. Isto gera uma string similar ao que é retornado por **`repr()`** no Python 2.

bin(*integer*, /)

Converte um número inteiro para uma string de binários prefixada com "0b". O resultado é uma expressão Python válida. Se *integer* não é um objeto Python **`int`**, ele tem que definir um método **`__index__()`** que devolve um inteiro. Alguns exemplos:

```
>>> bin(3)
'0b11'
>>> bin(-10)
'-0b1010'
```

Se o prefixo "0b" é desejado ou não, você pode usar uma das seguintes maneiras.

```
>>> format(14, '#b'), format(14, 'b')
('0b1110', '1110')
>>> f'{14:#b}', f'{14:b}'
('0b1110', '1110')
```

Veja também **`enum.bin()`** para representar valores negativos como complemento de dois.

Veja também **`format()`** para mais informações.

class bool(*object=False*, /)

Retorna um valor Booleano, isto é, `True` ou `False`. O argumento é convertido usando o procedimento de teste de verdade padrão. Se o argumento for falso ou foi omitido, isso retorna `False`; senão `True`. A classe **`bool`** é uma subclasse de **`int`** (veja Tipos numéricos — int, float, complex). Ela não pode ser usada para criar outra subclasse. Suas únicas instâncias são `False` e `True` (veja Tipo booleano - bool).

Alterado na versão 3.7: O parâmetro agora é somente-posicional

breakpoint(**args*, ***kws*)

Esta função coloca você no depurador no local da chamada. Especificamente, ela chama **`sys.breakpointhook()`**, passando `args` e `kws` diretamente. Por padrão, `sys.breakpointhook()` chama **`pdb.set_trace()`** não esperando nenhum argumento. Neste caso, isso é puramente uma função de conveniência para você não precisar importar **`pdb`** explicitamente ou digitar mais código para entrar no depurador. Contudo, **`sys.breakpointhook()`** pode ser configurado para alguma outra função e **`breakpoint()`** irá automaticamente chamá-la, permitindo você ir para o depurador de sua escolha. Se **`sys.breakpointhook()`** não estiver acessível, esta função vai levantar **`RuntimeError`**.

Por padrão, o comportamento de **`breakpoint()`** pode ser alterado com a variável de ambiente **`PYTHONBREAKPOINT`**. Veja **`sys.breakpointhook()`** para detalhes de uso.

Observe que isso não é garantido se **`sys.breakpointhook()`** tiver sido substituído.

Levanta um evento de auditoria `builtins.breakpoint` com o argumento `breakpointhook`.

Adicionado na versão 3.7.

class bytearray(*source=b''*)

class bytearray(*source*, *encoding*, *errors='strict'*)

Retorna um novo vetor de bytes. A classe **`bytearray`** é uma sequência mutável de inteiros no intervalo 0 <= x < 256. Ela tem a maior parte dos métodos mais comuns de sequências mutáveis, descritas em Tipos

sequências mutáveis, assim como a maior parte dos métodos que o tipo **`bytes`** tem, veja Operações com Bytes e Bytearray.

O parâmetro opcional *source* pode ser usado para inicializar o vetor de algumas maneiras diferentes:

- Se é uma *string*, você deve informar o parâmetro *encoding* (e opcionalmente, *errors*); **`bytearray()`** então converte a string para bytes usando **`str.encode()`**.
- Se é um *inteiro*, o vetor terá esse tamanho e será inicializado com bytes nulos.
- Se é um objeto em conformidade com a interface de buffer, um buffer somente leitura do objeto será usado para inicializar o vetor de bytes.
- Se é um *iterável*, deve ser um iterável de inteiros no intervalo `0 <= x < 256`, que serão usados como o conteúdo inicial do vetor.

Sem nenhum argumento, um vetor de tamanho 0 é criado.

Veja também Tipos de sequência binária — bytes, bytearray, memoryview e Objetos Bytearray.

class bytes(*source=b''*)
class bytes(*source*, *encoding*, *errors='strict'*)

Retorna um novo objeto “bytes” que é uma sequência imutável de inteiros no intervalo `0 <= x < 256`. **`bytes`** é uma versão imutável de **`bytearray`** – tem os mesmos métodos de objetos imutáveis e o mesmo comportamento de índices e fatiamento.

Consequentemente, argumentos do construtor são interpretados como os de **`bytearray()`**.

Objetos bytes também podem ser criados com literais, veja Literais de string e bytes.

Veja também Tipos de sequência binária — bytes, bytearray, memoryview, Objetos bytes, e Operações com Bytes e Bytearray.

callable(*object*, /)

Retorna `True` se o argumento *object* parece ser chamável, `False` caso contrário. Se retorna `True`, ainda é possível que a chamada falhe, mas se é `False`, chamar *object* nunca será bem sucedido. Note que classes são chamáveis (chamar uma classe devolve uma nova instância); instâncias são chamáveis se suas classes possuem um método `__call__()`.

Adicionado na versão 3.2: Esta função foi removida na versão 3.0, mas retornou no Python 3.2.

chr(*codepoint*, /)

Retorna a string representando um caractere com o ponto de código Unicode especificado. Por exemplo, `chr(97)` retorna a string `'a'`, enquanto `chr(8364)` retorna a string `'€'`. É o inverso de `ord()`.

O intervalo válido para o argumento vai de 0 até 1.114.111 (0x10FFFF na base 16). Será lançada uma exceção `ValueError` se ele estiver fora desse intervalo.

@classmethod

Transforma um método em um método de classe.

Um método de classe recebe a classe como um primeiro argumento implícito, exatamente como um método de instância recebe a instância. Para declarar um método de classe, faça dessa forma:

```
class C:
    @classmethod
    def f(cls, arg1, arg2): ...
```

O termo `@classmethod` é uma função decoradora – veja Definições de função para detalhes.

Um método de classe pode ser chamado tanto da classe (como em `C.f()`) quanto da instância (como em `C().f()`). A instância é ignorada, exceto por sua classe. Se um método de classe é chamado por uma classe derivada, o objeto da classe derivada é passado como primeiro argumento implícito.

Métodos de classe são diferentes de métodos estáticos em C++ ou Java. Se você quer saber desses, veja **staticmethod()** nesta seção. Para mais informações sobre métodos de classe, consulte A hierarquia de tipos padrão.

Alterado na versão 3.9: Métodos de classe agora podem envolver outros descritores tal como **property()**.

Alterado na versão 3.10: Métodos de classe agora herdam os atributos do método (**__module__**, **__name__**, **__qualname__**, **__doc__** e **__annotations__**) e têm um novo atributo `__wrapped__`.

Descontinuado desde a versão 3.11, removido na versão 3.13: Métodos de classe não podem mais envolver outros descritores tal como **property()**.

compile(*source*, *filename*, *mode*, *flags=0*, *dont_inherit=False*, *optimize=-1*, **, module=None*)

Compila o argumento *source* em código ou objeto AST. Objetos código podem ser executados por **exec()** ou **eval()**. *source* pode ser uma string normal, uma string byte, ou um objeto AST. Consulte a documentação do módulo **ast** para saber como trabalhar com objetos AST.

O argumento *filename* deve ser o arquivo de onde o código será lido; passe algum valor reconhecível se isso não foi lido de um arquivo (`'<string>'` é comumente usado).

O argumento *mode* especifica qual o tipo de código deve ser compilado; pode ser `'exec'` se *source* consiste em uma sequência de instruções, `'eval'` se consiste de uma única expressão, ou `'single'` se consiste de uma única instrução interativa (neste último caso, instruções que são avaliadas para alguma coisa diferente de `None` serão exibidas).

Os argumentos opcionais *flags* e *dont_inherit* controlam quais opções do compilador devem ser ativadas e quais recursos futuros devem ser permitidos. Se nenhum estiver presente (ou ambos forem zero), o código é compilado com os mesmos sinalizadores que afetam o código que está chamando **compile()**. Se o argumento *flags* for fornecido e *dont_inherit*

não for (ou for zero), as opções do compilador e as instruções futuras especificadas pelo argumento *flags* são usadas além daquelas que seriam usadas de qualquer maneira. Se *dont_inherit* for um inteiro diferente de zero, então o argumento *flags* é – os sinalizadores (recursos futuros e opções do compilador) no código circundante são ignorados.

Opções de compilador e instruções futuras são especificadas por bits, assim pode ocorrer uma operação *OU* bit a bit para especificar múltiplas instruções. O sinalizador necessário para especificar um dado recurso futuro pode ser encontrado no atributo `compiler_flag` na instância `_Feature` do módulo `__future__`. Sinalizadores de compilador podem ser encontrados no módulo `ast`, com o prefixo `PyCF_`.

O argumento *optimize* especifica o nível de otimização do compilador; o valor padrão de `-1` seleciona o nível de otimização do interpretador dado pela opção `-O`. Níveis explícitos são `0` (nenhuma otimização; `__debug__` é verdadeiro), `1` (instruções `assert` são removidas, `__debug__` é falso) ou `2` (strings de documentação também são removidas).

O argumento opcional *module* especifica o nome do módulo. Ele é necessário para filtrar avisos de sintaxe sem ambiguidade por nome de módulo.

This function raises `SyntaxError` if the compiled source is invalid, including a *source* containing a null character or that cannot be decoded; `ValueError` if *mode* or *flags* is invalid, or if a string *source* contains surrogate characters; `MemoryError` or `RecursionError` if *source* is too complex to parse or compile, for example an expression with many thousands of nested operators; and `OverflowError` if *source* is too large.

Se você quer analisar código Python em sua representação AST, veja `ast.parse()`.

Levanta um evento de auditoria `compile` com os argumentos `source`, `filename`. Esse evento pode também ser levantando por uma compilação implícita.

Nota

Quando compilando uma string com código multi-linhas em modo `'single'` ou `'eval'`, entrada deve ser terminada por ao menos um caractere de nova linhas. Isso é para facilitar a detecção de instruções completas e incompletas no módulo **`code`**.

Aviso

É possível quebrar o interpretador Python com uma string suficientemente grande/complexa ao compilar para um objeto AST, devido limitações do tamanho da pilha no compilador AST do Python.

Alterado na versão 3.2: Permitido o uso de marcadores de novas linhas no estilo Windows e Mac. Além disso, em modo `'exec'` a entrada não precisa mais terminar com uma nova linha. Também foi adicionado o parâmetro *optimize*.

Alterado na versão 3.5: Anteriormente, **`TypeError`** era levantada quando havia bytes nulos em *source*.

Alterado na versão 3.8: `ast.PyCF_ALLOW_TOP_LEVEL_AWAIT` agora pode ser passado em *flags* para habilitar o suporte em nível superior a `await`, `async for`, e `async with`.

Alterado na versão 3.12: **`SyntaxError`** is raised instead of **`ValueError`** when null bytes are encountered in *source*.

Alterado na versão 3.15: Adicionado o parâmetro *module*.

class complex(*number=0*, /)
class complex(*string*, /)
class complex(*real=0*, *imag=0*)

Converte uma única string ou número para um número complexo, ou cria um número complexo a partir de partes real e imaginária.

Exemplos:

```
>>> complex('+1.23')
(1.23+0j)
```

```
>>> complex('-4.5j')
-4.5j
>>> complex('-1.23+4.5j')
(-1.23+4.5j)
>>> complex('\t( -1.23+4.5J )\n')
(-1.23+4.5j)
>>> complex('-Infinity+NaNj')
(-inf+nanj)
>>> complex(1.23)
(1.23+0j)
>>> complex(imag=-4.5)
-4.5j
>>> complex(-1.23, 4.5)
(-1.23+4.5j)
```

Se o argumento for uma string, ele deve conter ou a parte real (com o mesmo formato usado em **float()**) ou uma parte imaginária (com o mesmo formato, mas com um sufixo `'j'` ou `'J'`), ou então ambas as partes real e imaginária (caso no qual o sinal da parte imaginária é obrigatório). A string pode opcionalmente ser cercada por espaços em branco e parênteses `'('` e `')'`, que são ignorados. A string não deve conter espaços em branco entre os símbolos `'+'`, `'-'`, o sufixo `'j'` ou `'J'`, e o número decimal. Por exemplo, `complex('1+2j')` é ok, mas `complex('1 + 2j')` levanta **ValueError**. Mais precisamente, após descartar os parênteses e os espaços em branco do início e do final, a entrada deve ser conforme a regra de produção **complexvalue** da gramática a seguir:

```
complexvalue: floatvalue |
              floatvalue ("j" | "J") |
              floatvalue sign absfloatvalue ("j" | "J")
```

Se o argumento for um número, o construtor serve como uma conversão numérica tal qual **int** e **float**. Para um objeto Python `x` qualquer, `complex(x)` delega para `x.__complex__()`. Se **__complex__()** não está definido então a chamada é repassada para **__float__()**. Se **__float__()** não está definido então a chamada é, novamente, repassada para **__index__()**.

Se dois argumentos forem passados ou argumentos nomeados forem usados, cada argumento pode ser de qualquer tipo numérico (incluindo complexo). Se ambos argumentos forem números reais, é retornado um número complexo com *real* como parte real e *imag* como parte imaginária. Se ambos os argumentos forem números complexos, é retornado um número complexo com parte real `real.real-imag.imag` e parte imaginária `real.imag+imag.real`. Se um dos argumentos for um número real, somente a sua parte real é usada nas expressões anteriores.

Veja também **`complex.from_number()`** que aceita apenas um único argumento numérico.

Se todos os argumentos forem omitidos, retorna `0j`.

O tipo complexo está descrito em Tipos numéricos — int, float, complex.

Alterado na versão 3.6: Agrupar dígitos com sublinhados como em literais de código é permitido.

Alterado na versão 3.8: Chamadas para **`__index__()`** se **`__complex__()`** e **`__float__()`** não estão definidas.

Descontinuado desde a versão 3.14: Passar um número complexo como argumento *real* ou *imag* agora está descontinuado; deve ser passado apenas como um único argumento posicional.

delattr(*object*, *name*, /)

Essa função está relacionada com **`setattr()`**. Os argumentos são um objeto e uma string. A string deve ser o nome de um dos atributos do objeto. A função remove o atributo indicado, desde que o objeto permita. Por exemplo, `delattr(x, 'foobar')` é equivalente a `del x.foobar`. *name* não precisa ser um identificador Python (veja **`setattr()`**).

class dict(***kwargs*)

class dict(*mapping*, /, ***kwargs*)

class dict(*iterable*, /, ***kwargs*)

Cria um novo dicionário. O objeto **`dict`** é a classe do dicionário. Veja também Tipos mapeamentos — dict, frozendict para documentação sobre

esta classe.

Para outros contêineres, consulte as classes embutidas `frozendict`, `list`, `set` e `tuple`, bem como o módulo `collections`.

dir()
dir(*object*, /)

Sem argumentos, devolve a lista de nomes no escopo local atual. Com um argumento, tentará devolver uma lista de atributos válidos para esse objeto.

Se o objeto tiver um método chamado `__dir__()`, esse método será chamado e deve devolver a lista de atributos. Isso permite que objetos que implementam uma função personalizada `__getattr__()` ou `__getattribute__()` personalizem a maneira como `dir()` relata seus atributos.

Se o objeto não fornecer `__dir__()`, a função tentará o melhor possível para coletar informações do atributo `__dict__` do objeto, se definido, e do seu objeto de tipo. A lista resultante não está necessariamente completa e pode ser imprecisa quando o objeto possui um `__getattr__()` personalizado.

O mecanismo padrão `dir()` se comporta de maneira diferente com diferentes tipos de objetos, pois tenta produzir as informações mais relevantes e não completas:

- Se o objeto for um objeto de módulo, a lista conterá os nomes dos atributos do módulo.
- Se o objeto for um objeto de tipo ou classe, a lista conterá os nomes de seus atributos e recursivamente os atributos de suas bases.
- Caso contrário, a lista conterá os nomes dos atributos do objeto, os nomes dos atributos da classe e recursivamente os atributos das classes base da classe.

A lista resultante é alfabeticamente ordenada. Por exemplo:

```
>>> import struct
>>> dir()   # show the names in the module namespace
```

```
['__builtins__', '__name__', 'struct']
>>> dir(struct)   # show the names in the struct module
['Struct', '__all__', '__builtins__', '__doc__', '__file__',
 '__initializing__', '__loader__', '__name__', '__package__',
 '_clearcache', 'calcsize', 'error', 'pack', 'pack_into',
 'unpack', 'unpack_from']
>>> class Shape:
...     def __dir__(self):
...         return ['area', 'perimeter', 'location']
...
>>> s = Shape()
>>> dir(s)
['area', 'location', 'perimeter']
```

Nota

Como `dir()` é fornecido principalmente como uma conveniência para uso em um prompt interativo, ele tenta fornecer um conjunto interessante de nomes mais do que tenta fornecer um conjunto de nomes definido de forma rigorosa ou consistente, e seu comportamento detalhado pode mudar nos lançamentos. Por exemplo, os atributos de metaclasse não estão na lista de resultados quando o argumento é uma classe.

divmod(*a*, *b*, /)

Recebe dois números (não complexos) como argumentos e retorna um par de números que consiste em seu quociente e resto ao usar a divisão inteira. Com tipos de operandos mistos, as regras para operadores aritméticos binários se aplicam. Para números inteiros, o resultado é o mesmo que `(a // b, a % b)`. Para números de ponto flutuante, o resultado é `(q, a % b)`, onde *q* geralmente é `math.floor(a / b)`, mas pode ser 1 a menos que isso. Em qualquer caso, `q * b + a % b` está muito próximo de *a*, se `a % b` é diferente de zero, tem o mesmo sinal que *b* e `0 <= abs(a % b) < abs(b)`.

enumerate(*iterable*, *start=0*)

Devolve um objeto enumerado. *iterable* deve ser uma sequência, um iterador ou algum outro objeto que suporte a iteração. O método `__next__()` do iterador retornado por `enumerate()` devolve uma tupla

contendo uma contagem (a partir de *start*, cujo padrão é 0) e os valores obtidos na iteração sobre *iterable*.

```
>>> seasons = ['Spring', 'Summer', 'Fall', 'Winter']
>>> list(enumerate(seasons))
[(0, 'Spring'), (1, 'Summer'), (2, 'Fall'), (3, 'Winter')]
>>> list(enumerate(seasons, start=1))
[(1, 'Spring'), (2, 'Summer'), (3, 'Fall'), (4, 'Winter')]
```

Equivalente a:

```
def enumerate(iterable, start=0):
    n = start
    for elem in iterable:
        yield n, elem
        n += 1
```

eval(*source*, /, *globals=None*, *locals=None*)

Parâmetros::
- **source** (`str` | code object) – Uma expressão Python.
- **globals** (`dict` | `frozendict` | `None`) – O espaço de nomes global (por padrão, `None`).
- **locals** (mapping | `None`) – O espaço de nomes loca (por padrão, `None`).

Retorna:: O resultado da expressão avaliada.

levanta:: Erros de sintaxe são reportados como exceções.

Aviso

Esta função executa código arbitrário. Chamá-la com entrada fornecida pelo usuário não confiável levará a vulnerabilidades de segurança.

O argumento *source* é analisado e avaliado como uma expressão Python (tecnicamente, uma lista de expressões), usando os dicionários *globals* e *locals* como espaços de nomes global e local. Se o dicionário *globals* estiver presente e não contiver um valor para a chave `__builtins__`, uma referência ao dicionário do módulo `builtins` é inserida sob essa chave antes de *source* ser analisado. A sobrescrita de `__builtins__` pode ser usada para controlar ou alterar os nomes disponíveis, mas isso **não** é um mecanismo de segurança: o código executado ainda pode acessar todos os

embutidos. Se o dicionário *locals* for omitido, o valor predefinido é o dicionário *globals*. Se ambos os mapeamentos forem omitidos, *source* é executado com os *globals* e *locals* do ambiente em que **`eval()`** é chamada. Note que *eval()* só terá acesso aos escopos aninhados (*non-locals*) no ambiente que a contém se eles já estiverem referenciados no escopo que chama **`eval()`** (por exemplo, através de uma instrução **`nonlocal`**).

Exemplo:

```
>>> x = 1
>>> eval('x+1')
2

>>> eval("1, 2")
(1, 2)
```

Esta função também pode ser usada para executar objetos código arbitrários (como os criados por **`compile()`**). Nesse caso, passe um objeto código em vez de uma string. Se o objeto código foi compilado com `'exec'` como o argumento *mode*, o valor de retorno de **`eval()`** será `None`.

Dicas: a execução dinâmica de instruções é implementada pela função **`exec()`**. As funções **`globals()`** e **`locals()`** retornam os dicionários global e local atuais, respectivamente, o que pode ser útil para serem usados por **`eval()`** ou **`exec()`**.

Se a fonte fornecida for uma string, os espaços e tabulações à esquerda ou à direita serão removidos.

Veja **`ast.literal_eval()`** para uma função para avaliar strings com expressões que contêm apenas literais.

Levanta um evento de auditoria `exec` com o código objeto como argumento. Eventos de compilação de código também podem ser levantados.

Alterado na versão 3.13: Os argumentos *globals* e *locals* podem agora ser passados como argumentos nomeados.

Alterado na versão 3.13: A forma de uso do espaço de nomes *locals* padrão foi ajustada conforme descrito na função embutida `locals()`.

Alterado na versão 3.15: globals pode agora ser um `frozendict`.

exec(*source*, /, *globals=None*, *locals=None*, *, *closure=None*)

Aviso

Esta função executa código arbitrário. Chamá-la com entrada fornecida pelo usuário não confiável levará a vulnerabilidades de segurança.

Esta função permite a execução dinâmica de código Python. O parâmetro *source* deve ser ou uma string ou um objeto código. Se for uma string, a mesma é analisada como um conjunto de instruções Python, o qual é então executado (exceto caso um erro de sintaxe ocorra). [[1]] Se for um objeto código, ele é simplesmente executado. Em todos os casos, espera-se que o código a ser executado seja válido como um arquivo de entrada (veja a seção Entrada de arquivo no Manual de Referência). Tenha cuidado, pois as instruções `nonlocal`, `yield` e `return` não podem ser usadas fora das definições de funções mesmo dentro do contexto do código passado para a função `exec()` . O valor de retorno é sempre `None`.

Em todos os casos, se os parâmetros opcionais são omitidos, o código é executado no escopo atual. Se somente *globals* é fornecido, deve ser um dicionário (e não uma subclasse de dicionário), que será usado tanto para as variáveis globais quanto para locais. Se *globals* e *locals* são fornecidos, eles são usados para as variáveis globais e locais, respectivamente. Se fornecido, *locals* pode ser qualquer objeto de mapeamento. Lembre que no nível de módulo, globals e locals são o mesmo dicionário.

Nota

Quando `exec` recebe dois objetos separados como *globals* e *locals*, o código será executado como se estivesse embutido em uma definição de classe. Isso significa que funções e classes definidas no código executado não poderão acessar variáveis que sofreram atribuições no

escopo mais externo (pois, em uma definição de classe, tais variáveis seriam tratadas como variáveis de classe).

Se o dicionário *globals* não contiver um valor para a chave `__builtins__`, a referência ao dicionário do módulo embutido **`builtins`** é inserida com essa chave. A sobrescrita de `__builtins__` pode ser usada para controlar ou alterar os nomes disponíveis, mas isso **não** é um mecanismo de segurança: o código executado ainda pode acessar todos os embutidos.

O argumento *closure* especifica uma clausura – uma tupla de cellvars. Só é válido quando o *objeto* é um objeto código contendo variáveis livres (de clausura). O comprimento da tupla deve corresponder exatamente ao comprimento do atributo **`co_freevars`** do objeto código.

Levanta um evento de auditoria `exec` com o código objeto como argumento. Eventos de compilação de código também podem ser levantados.

Nota

As funções embutidas **`globals()`** e **`locals()`** devolvem os espaços de nomes global e local, respectivamente, o que pode ser útil para passar adiante e usá-los como segundo ou terceiro argumento para **`exec()`**.

Nota

locals padrão atua como descrito na função **`locals()`** abaixo. Se você precisa ver os efeitos do código em *locals* após a função **`exec()`** retornar, passe um dicionário *locals* explícito.

Alterado na versão 3.11: Adicionado o parâmetro *closure*.

Alterado na versão 3.13: Os argumentos *globals* e *locals* podem agora ser passados como argumentos nomeados.

Alterado na versão 3.13: A forma de uso do espaço de nomes *locals* padrão foi ajustada conforme descrito na função embutida **`locals()`**.

Alterado na versão 3.15: globals pode agora ser um `frozendict`.

filter(*function*, *iterable*, /)

Constrói um iterador a partir dos elementos de *iterable* para os quais *function* é verdadeiro. *iterable* pode ser uma sequência, um contêiner que com suporte a iteração, ou um iterador. Se *function* for `None`, a função identidade será usada, isto é, todos os elementos de *iterable* que são falsos são removidos.

Note que `filter(function, iterable)` é equivalente a expressão geradora `(item for item in iterable if function(item))` se *function* não for `None` e `(item for item in iterable if item)` se *function* for `None`.

Veja `itertools.filterfalse()` para a função complementar que devolve elementos de *iterable* para os quais *function* é falso.

class float(*number=0.0*, /)

class float(*string*, /)

Retorna um número de ponto flutuante construído a partir de um número ou string.

Exemplos:

```
>>> float('+1.23')
1.23
>>> float('   -12345\n')
-12345.0
>>> float('1e-003')
0.001
>>> float('+1E6')
1000000.0
>>> float('-Infinity')
-inf
```

Se o argumento for uma string, ele deve conter um número decimal, opcionalmente precedido por um sinal e opcionalmente embutido em um espaço em branco. O sinal opcional pode ser `'+'` ou `'-'`; um sinal `'+'` não tem efeito no valor produzido. O argumento também pode ser uma

string representando um NaN (não um número) ou infinito positivo ou negativo. Mais precisamente, a entrada deve estar de acordo com a regra de produção `floatvalue` na seguinte gramática, depois que os espaços em branco iniciais e finais forem removidos:

```
sign:           "+" | "-"
infinity:       "Infinity" | "inf"
nan:            "nan"
digit:          <a Unicode decimal digit, i.e. characters in Un
digitpart:      digit (["_"] digit)*
number:         [digitpart] "." digitpart | digitpart ["."]
exponent:       ("e" | "E") [sign] digitpart
floatnumber:    number [exponent]
absfloatvalue:  floatnumber | infinity | nan
floatvalue:     [sign] absfloatvalue
```

O caso não é significativo, então, por exemplo, “inf”, “Inf”, “INFINITY” e “iNfINity” são todas grafias aceitáveis para o infinito positivo.

Caso contrário, se o argumento é um inteiro ou um número de ponto flutuante, um número de ponto flutuante com o mesmo valor (com a precisão de ponto flutuante de Python) é retornado. Se o argumento está fora do intervalo de um ponto flutuante Python, uma exceção `OverflowError` será lançada.

Para um objeto Python genérico `x`, `float(x)` delega para o método `x.__float__()`. Se `__float__()` não estiver definido, então ele delega para o método `__index__()`.

Veja também `float.from_number()` que aceita apenas um argumento numérico.

Se nenhum argumento for fornecido, será retornado `0.0`.

O tipo float é descrito em Tipos numéricos — int, float, complex.

Alterado na versão 3.6: Agrupar dígitos com sublinhados como em literais de código é permitido.

Alterado na versão 3.7: O parâmetro agora é somente-posicional

Alterado na versão 3.8: Chamada para `__index__()` se `__float__()` não está definido.

format(*value*, *format_spec=''*, /)

Converte um valor *value* em uma representação "formatada", controlado por *format_spec*. A interpretação de *format_spec* dependerá do tipo do argumento *value*; no entanto há uma sintaxe de formatação padrão usada pela maioria dos tipos embutidos: Minilinguagem de especificação de formato.

O *format_spec* padrão é uma string vazia que geralmente produz o mesmo efeito que chamar `str(value)`.

Uma chamada de `format(value, format_spec)` é convertida em `type(value).__format__(value, format_spec)`, que ignora o dicionário da instância ao pesquisar o método `__format__()` de `value`. Uma exceção `TypeError` é levantada se a pesquisa do método atingir `object` e o *format_spec* não estiver vazio, ou se o *format_spec* ou o valor de retorno não forem strings.

Alterado na versão 3.4: `object().__format__(format_spec)` levanta um `TypeError` se *format_spec* não for uma string vazia.

class frozendict(***kwargs*)

class frozendict(*mapping*, /, ***kwargs*)

class frozendict(*iterable*, /, ***kwargs*)

Cria um novo dicionário imutável. O objeto `frozendict` é uma classe embutida. Veja também Tipos mapeamentos — dict, frozendict para obter a documentação sobre esta classe.

Para outros contêineres, consulte as classes embutidas `dict`, `list`, `set` e `tuple`, bem como o módulo `collections`.

Adicionado na versão 3.15.

class frozenset(*iterable=()*, /)

Retorna um novo objeto **`frozenset`**, opcionalmente com elementos obtidos de *iteráveis*. **`frozenset`** é uma classe embutida. Veja também Tipo conjuntos — set, frozenset para documentação sobre essa classe.

Para outros contêineres veja as classes embutidas **`set`**, **`list`**, **`tuple`**, e **`dict`**, assim como o módulo **`collections`**.

getattr(*object*, *name*, /)
getattr(*object*, *name*, *default*, /)

Devolve o valor do atributo *name* de *object*. *name* deve ser uma string. Se a string é o nome de um dos atributos do objeto, o resultado é o valor de tal atributo. Por exemplo, `getattr(x, 'foobar')` é equivalente a `x.foobar`. Se o atributo não existir, *default* é devolvido se tiver sido fornecido, caso contrário a exceção **`AttributeError`** é levantada. *name* não precisa ser um identificador Python (veja **`setattr()`**).

Nota

Uma vez que desfiguração de nome privado acontece em tempo de compilação, deve-se manualmente mutilar o nome de um atributo privado (atributos com dois sublinhados à esquerda) para recuperá-lo com **`getattr()`**.

globals()

Retorna o dicionário implementando o espaço de nomes do módulo atual. Para código dentro de funções, isso é definido quando a função é definida e permanece o mesmo, independentemente de onde a função é chamada.

hasattr(*object*, *name*, /)

Os argumentos são um objeto e uma string. O resultado é `True` se a string é o nome de um dos atributos do objeto, e `False` se ela não for. (Isto é implementado chamando `getattr(object, name)` e vendo se levanta um **`AttributeError`** ou não.)

hash(*object*, /)

Retorna o valor hash de um objeto (se houver um). Valores hash são números inteiros. Eles são usados para rapidamente comparar chaves de

dicionários durante uma pesquisa em um dicionário. Valores numéricos que ao serem comparados são iguais, possuem o mesmo valor hash (mesmo que eles sejam de tipos diferentes, como é o caso de 1 e 1.0).

Nota

Para objetos com métodos `__hash__()` personalizados, fique atento que `hash()` trunca o valor devolvido baseado no comprimento de bits da máquina hospedeira.

help()
help(*request*)

Invoca o sistema de ajuda embutido. (Esta função é destinada para uso interativo.) Se nenhum argumento é passado, o sistema interativo de ajuda inicia no interpretador do console. Se o argumento é uma string, então a string é pesquisada como o nome de um módulo, função, classe, método, palavra-chave, ou tópico de documentação, e a página de ajuda é exibida no console. Se o argumento é qualquer outro tipo de objeto, uma página de ajuda para o objeto é gerada.

Note que se uma barra(/) aparecer na lista de parâmetros de uma função, quando invocando `help()`, significa que os parâmetros anteriores a barra são apenas posicionais. Para mais informações, veja a entrada no FAQ sobre parâmetros somente-posicionais.

Esta função é adicionada ao espaço de nomes embutido pelo módulo `site`.

Alterado na versão 3.4: Mudanças em `pydoc` e `inspect` significam que as assinaturas reportadas para chamáveis agora são mais compreensíveis e consistentes.

hex(*integer*, /)

Converte um número inteiro para uma string hexadecimal em letras minúsculas prefixada com “0x”. Se *integer* não é um objeto `int` do Python, ele tem que definir um método `__index__()` que retorne um inteiro. Alguns exemplos:

```
>>> hex(255)
'0xff'
>>> hex(-42)
'-0x2a'
```

Se você quer converter um número inteiro para uma string hexadecimal em letras maiúsculas ou minúsculas, com prefixo ou sem, você pode usar qualquer uma das seguintes maneiras:

```
>>> '%#x' % 255, '%x' % 255, '%X' % 255
('0xff', 'ff', 'FF')
>>> format(255, '#x'), format(255, 'x'), format(255, 'X')
('0xff', 'ff', 'FF')
>>> f'{255:#x}', f'{255:x}', f'{255:X}'
('0xff', 'ff', 'FF')
```

Veja também `format()` para mais informações.

Veja também `int()` para converter uma string hexadecimal para um inteiro usando a base 16.

Nota

Para obter uma string hexadecimal de um ponto flutuante, use o método `float.hex()`.

id(*object*, /)

Devolve a “identidade” de um objeto. Ele é um inteiro, o qual é garantido que será único e constante para este objeto durante todo o seu ciclo de vida. Dois objetos com ciclos de vida não sobrepostos podem ter o mesmo valor para `id()`.

Este é o endereço do objeto na memória.

Levanta um evento de auditoria `builtins.id` com o argumento `id`.

input()

input(*prompt*, /)

Se o argumento *prompt* estiver presente, escreve na saída padrão sem uma nova linha ao final. A função então lê uma linha da fonte de entrada, converte a mesma para uma string (removendo o caractere de nova linha ao final), e devolve isso. Quando o final do arquivo (EOF / end-of-file) é encontrado, um erro **`EOFError`** é levantado. Exemplo:

```
>>> s = input('--> ')
--> Monty Python's Flying Circus
>>> s
"Monty Python's Flying Circus"
```

Se o módulo **`readline`** foi carregado, então **`input()`** usará ele para prover edição de linhas elaboradas e funcionalidades de histórico.

Levanta um evento de auditoria `builtins.input` com argumento `prompt` antes de ler a entrada.

Levanta um evento de auditoria `builtins.input/result` com o resultado depois de ler a entrada com sucesso.

class int(*number=0*, *⁄*)
class int(*string*, *⁄*, *base=10*)

Retorna um objeto do tipo inteiro construído a partir de um número ou string, ou retorna `0` caso nenhum argumento seja passado.

Exemplos:

```
>>> int(123.45)
123
>>> int('123')
123
>>> int('   -12_345\n')
-12345
>>> int('FACE', 16)
64206
>>> int('0xface', 0)
64206
>>> int('01110011', base=2)
115
```

Se o argumento definir um método `__int__()`, então `int(x)` retorna `x.__int__()`. Se *x* definir um método `__index__()`, então ele retorna `x.__index__()`. Para números de ponto flutuante, isto trunca o número na direção do zero.

Se o argumento não for um número ou se *base* for fornecido, então o argumento deve ser uma instância de string, `bytes` ou `bytearray` representando um inteiro na base *base*. Opcionalmente, a string pode ser precedida por `+` ou `-` (sem espaço entre eles), ter zeros à esquerda, estar entre espaços em branco e ter sublinhados simples intercalados entre os dígitos.

Uma string de inteiro de base n contém dígitos, cada um representando um valor de 0 a n-1. Os valores 0–9 podem ser representados por qualquer dígito decimal Unicode. Os valores 10–35 podem ser representados por `a` a `z` (ou `A` a `Z`). A *base* padrão é 10. As bases permitidas são 0 e 2–36. As strings base-2, -8 e -16 podem ser opcionalmente prefixadas com `0b`/`0B`, `0o`/`0O` ou `0x`/`0X`, como acontece com literais inteiros no código. Para a base 0, a string é interpretada de maneira semelhante a um literal inteiro no código, em que a base real é 2, 8, 10 ou 16 conforme determinado pelo prefixo. A base 0 também não permite zeros à esquerda: `int('010', 0)` não é válido, enquanto `int('010')` e `int('010', 8)` são.

O tipo inteiro está descrito em Tipos numéricos — int, float, complex.

Alterado na versão 3.4: Se *base* não é uma instância de `int` e o objeto *base* tem um método `base.__index__`, então esse método é chamado para obter um inteiro para a base. Versões anteriores usavam `base.__int__` ao invés de `base.__index__`.

Alterado na versão 3.6: Agrupar dígitos com sublinhados como em literais de código é permitido.

Alterado na versão 3.7: O primeiro parâmetro agora é somente-posicional.

Alterado na versão 3.8: Chamada para `__index__()` se `__int__()` não está definido.

Alterado na versão 3.11: Entradas de strings para `int` e representações em strings podem ser limitadas para ajudar a evitar ataques de negação de serviço. Uma exceção `ValueError` é levantada quando o limite é excedido durante a conversão de uma string em um `int` ou quando a conversão de um `int` em uma string excede o limite. Consulte a documentação sobre limitação de comprimento de conversão de string em inteiro.

Alterado na versão 3.14: `int()` não delega mais ao método `__trunc__()`.

isinstance(*object*, *classinfo*, /)

Retorna `True` se o argumento *object* é uma instância do argumento *classinfo*, ou de uma subclasse dele (direta, indireta ou virtual). Se *object* não é um objeto do tipo dado, a função sempre devolve `False`. Se *classinfo* é uma tupla de tipos de objetos (ou recursivamente, como outras tuplas) ou um Tipo União de vários tipos, retorna `True` se *object* é uma instância de qualquer um dos tipos. Se *classinfo* não é um tipo ou tupla de tipos ou outras tuplas, é levantada uma exceção `TypeError`. `TypeError` pode não ser levantada por um tipo inválido se uma verificação anterior for bem-sucedida.

Alterado na versão 3.10: classinfo pode ser um Tipo União.

issubclass(*cls*, *classinfo*, /)

Retorna `True` se *cls* for uma subclasse (direta, indireta ou virtual) de *classinfo*. Uma classe é considerada uma subclasse de si mesma. *classinfo* pode ser uma tupla de objetos de classe (ou recursivamente, outras tuplas) ou um Tipo União, caso em que retorna `True` se *cls* for uma subclasse de qualquer entrada em *classinfo*. Em qualquer outro caso, é levantada uma exceção `TypeError`.

Alterado na versão 3.10: classinfo pode ser um Tipo União.

iter(*iterable*, /)

iter(*callable*, *sentinel*, /)

Retorna um objeto iterador. O primeiro argumento é interpretado muito diferentemente dependendo da presença do segundo argumento. Sem um segundo argumento, o primeiro argumento deve ser uma coleção de

objetos com suporte ao protocolo iterável (o método `__iter__()`), ou ele deve ter suporte ao protocolo de sequência (o método `__getitem__()` com argumentos inteiros começando em `0`). Se ele não tem suporte nenhum desses protocolos, uma `TypeError` é levantada. Se o segundo argumento, *sentinel*, é fornecido, então o primeiro argumento deve ser um objeto chamável. O iterador criado neste caso irá chamar *callable* sem nenhum argumento para cada chamada para o seu método `__next__()`; se o valor devolvido é igual a *sentinel*, então `StopIteration` será levantado, caso contrário o valor será devolvido.

Veja também Tipos iteradores.

Uma aplicação útil da segunda forma de `iter()` é para construir um bloco de leitura. Por exemplo, ler blocos de comprimento fixo de um arquivo binário de banco de dados até que o final do arquivo seja atingido:

```
from functools import partial
with open('meusdados.db', 'rb') as f:
    for bloco in iter(partial(f.read, 64), b''):
        process_block(bloco)
```

len(*object*, /)

Devolve o comprimento (o número de itens) de um objeto. O argumento pode ser uma sequência (tal como uma string, bytes, tupla, lista, ou um intervalo) ou uma coleção (tal como um dicionário, conjunto, ou conjunto imutável).

`len` levanta `OverflowError` em tamanhos maiores que `sys.maxsize`, tal como `range(2 ** 100)`.

class list(*iterable=()*, /)

Ao invés de ser uma função, `list` é na verdade um tipo de sequência mutável, conforme documentado em Listas e Tipos sequências — list, tuple, range.

locals()

Retorna um objeto de mapeamento representando a tabela de símbolos atual, com nomes de variáveis como as chaves, e as referências às quais cada variável está atrelada no momento como os valores.

Em um escopo de módulo, assim como quando usando **`exec()`** ou **`eval()`** com um único espaço de nomes, esta função retorna o mesmo espaço de nomes que **`globals()`**.

Em um escopo de classe, ela retorna o espaço de nomes que será passado para o construtor da metaclasse.

Quando usando `exec()` ou `eval()` com argumentos de espaço local e global distintos, ela retorna o espaço de nomes local passado na chamada.

Em todos os casos acima, cada chamada a `locals()` em um dado quadro de execução vai retornar o *mesmo* objeto de mapeamento. Mudanças feitas através do objeto de mapeamento retornado por `locals()` serão visíveis tal qual variáveis locais atribuídas, reatribuídas ou deletadas, e atribuir, reatribuir ou deletar variáveis locais afetará imediatamente o conteúdo do objeto de mapeamento retornado.

Em contraste aos casos acima, em um escopo otimizado (incluindo funções, geradores e corrotinas), cada chamada a `locals()` retorna um dicionário novo contendo as ligações atuais das variáveis locais da função e de quaisquer células com referências não-locais. Nesse caso, mudanças em ligações de nomes feitas através do dicionário retornado *não* são escritas de volta nas variáveis locais correspondentes nem nas células com referências não-locais, e atribuir, reatribuir ou deletar variáveis locais e células com referências não-locais *não* afeta o conteúdo dos dicionários que já foram retornados.

Chamar a função `locals()` como parte de uma compreensão em uma função, gerador ou corrotina é equivalente a chamá-la no escopo que contém a compreensão, exceto que as variáveis de iteração inicializadas da compreensão serão incluídas. Em outros escopos, essa função se comporta como se a compreensão estivesse executando como uma função aninhada.

Chamar a função `locals()` como parte de uma expressão geradora é equivalente a chamá-la em uma função geradora aninhada.

Alterado na versão 3.12: O comportamento de `locals()` em uma compreensão foi atualizado conforme descrito na **PEP 709** [https://peps.python.org/pep-0709/].

Alterado na versão 3.13: Como parte da **PEP 667** [https://peps.python.org/pep-0667/], o que acontece após a mutação de objetos de mapeamento retornados desta função está agora definido. O comportamento em escopos otimizados agora é o descrito acima. Além de estar definido, o comportamento em outros escopos não foi modificado em relação aos comportamentos em versões passadas.

map(*function*, *iterable*, *, **iterables*, *strict=False*)

Devolve um iterador que aplica *function* para cada item de *iterable*, gerando os resultados. Se argumentos *iterables* adicionais são passados, *function* deve ter a mesma quantidade de argumentos e ela é aplicada aos itens de todos os iteráveis em paralelo. Com múltiplos iteráveis, o iterador para quando o iterador mais curto é esgotado. Se *strict* for `True` e um dos iteráveis for esgotado antes dos outros, um **`ValueError`** será levanta. Para casos onde os parâmetros de entrada da função já estão organizados em tuplas, veja **`itertools.starmap()`**.

Alterado na versão 3.14: Adicionado o parâmetro *track*.

max(*iterable*, *, *, *key=None*)
max(*iterable*, *, *, *default*, *key=None*)
max(*arg1*, *arg2*, *, **args*, *key=None*)

Devolve o maior item em um iterável ou o maior de dois ou mais argumentos.

Se um argumento posicional é fornecido, ele deve ser um iterável. O maior item no iterável é retornado. Se dois ou mais argumentos posicionais são fornecidos, o maior dos argumentos posicionais é devolvido.

Existem dois parâmetros somente-nomeados opcionais. O parâmetro *key* especifica uma função de ordenamento de um argumento, como aquelas usadas por `list.sort()`. O parâmetro *default* especifica um objeto a ser devolvido se o iterável fornecido estiver vazio. Se o iterável estiver vazio, e *default* não foi fornecido, uma exceção `ValueError` é levantada.

Se múltiplos itens são máximos, a função devolve o primeiro encontrado. Isto é consistente com outras ferramentas de ordenamento que preservam a estabilidade, tais como `sorted(iterable, key=keyfunc, reverse=True)[0]` e `heapq.nlargest(1, iterable, key=keyfunc)`.

Alterado na versão 3.4: Adicionado o parâmetro *default* somente-nomeado.

Alterado na versão 3.8: O valor de *key* pode ser `None`.

class memoryview(*object*)

Devolve um objeto de "visão da memória" criado a partir do argumento fornecido. Veja Visões de memória para mais informações.

min(*iterable*, /, *, *key=None*)
min(*iterable*, /, *, *default*, *key=None*)
min(*arg1*, *arg2*, /, **args*, *key=None*)

Devolve o menor item de um iterável ou o menor de dois ou mais argumentos.

Se um argumento posicional é fornecido, ele deve ser um iterável. O menor item no iterável é devolvido. Se dois ou mais argumentos posicionais são fornecidos, o menor dos argumentos posicionais é devolvido.

Existem dois parâmetros somente-nomeados opcionais. O parâmetro *key* especifica uma função de ordenamento de um argumento, como aquelas usadas por `list.sort()`. O parâmetro *default* especifica um objeto a ser devolvido se o iterável fornecido estiver vazio. Se o iterável estiver vazio, e *default* não foi fornecido, uma exceção `ValueError` é levantada.

Se múltiplos itens são mínimos, a função devolve o primeiro encontrado. Isto é consistente com outras ferramentas de ordenamento que preservam a estabilidade, tais como `sorted(iterable, key=keyfunc)[0]` e `heapq.nsmallest(1, iterable, key=keyfunc)`.

Alterado na versão 3.4: Adicionado o parâmetro *default* somente-nomeado.

Alterado na versão 3.8: O valor de *key* pode ser `None`.

next(*iterator*, /)
next(*iterator*, *default*, /)

Recupera o próximo item do iterador chamando o seu método **`__next__()`**. Se *default* foi fornecido, ele é devolvido caso o iterador tenha sido esgotado, caso contrário **`StopIteration`** é levantada.

class object

Esta é a classe base definitiva de todas as outras classes. Ela tem métodos que são comuns a todas as instâncias de classes Python. Quando o construtor é chamado, ele retorna um novo objeto sem características. O construtor não aceita nenhum argumento.

Nota

Instâncias de **`object`** *não* têm atributos **`__dict__`**, então você não pode atribuir atributos arbitrários a uma instância de **`object`**.

oct(*integer*, /)

Converte um número inteiro para uma string em base octal, prefixada com “0o”. O resultado é uma expressão Python válida. Se *integer* não for um objeto **`int`** Python, ele tem que definir um método **`__index__()`** que devolve um inteiro. Por exemplo:

```
>>> oct(8)
'0o10'
>>> oct(-56)
'-0o70'
```

Se você quiser converter um número inteiro para uma string octal, com o prefixo “0o” ou não, você pode usar qualquer uma das formas a seguir.

```
>>> '%#o' % 10, '%o' % 10
('0o12', '12')
>>> format(10, '#o'), format(10, 'o')
('0o12', '12')
>>> f'{10:#o}', f'{10:o}'
('0o12', '12')
```

Veja também **`format()`** para mais informações.

open(*file*, *mode='r'*, *buffering=-1*, *encoding=None*, *errors=None*, *newline=None*, *closefd=True*, *opener=None*)

Abre *file* e retorna um objeto arquivo correspondente. Se o arquivo não puder ser aberto, uma **`OSError`** é levantada. Veja Lendo e escrevendo arquivos para mais exemplos de como usar esta função.

file é um objeto caminho ou similar fornecendo o caminho (absoluto ou relativo ao diretório de trabalho atual) do arquivo que será aberto, ou de um inteiro descritor de arquivo a ser manipulado (Se um descritor de arquivo é fornecido, ele é fechado quando o objeto de I/O retornado é fechado, a não ser que *closefd* esteja marcado como `False`).

mode é uma string opcional que especifica o modo no qual o arquivo é aberto. O valor padrão é `'r'`, o qual significa abrir para leitura em modo texto. Outros valores comuns são `'w'` para escrever (truncando o arquivo se ele já existe), `'x'` para criação exclusiva e `'a'` para anexar (o qual em *alguns* sistemas Unix, significa que *todas* as escritas anexam ao final do arquivo independentemente da posição de busca atual). No modo texto, se *encoding* não for especificada, UTF-8 é usado por padrão; se Modo UTF-8 do Python estiver desabilitado, **`locale.getencoding()`** é chamada para obter a codificação da localidade atual (Para ler e escrever bytes diretamente, use o modo binário e não especifique *encoding*). Os modos disponíveis são:

Caractere	Significado

Caractere	Significado
`'r'`	abre para leitura (padrão)
`'w'`	abre para escrita, truncando o arquivo primeiro (removendo tudo o que estiver contido no mesmo)
`'x'`	abre para criação exclusiva, falhando caso o arquivo exista
`'a'`	abre para escrita, anexando ao final do arquivo caso o mesmo exista
`'b'`	modo binário
`'t'`	modo texto (padrão)
`'+'`	aberto para atualização (leitura e escrita)

O modo padrão é `'r'` (abre para leitura de texto, um sinônimo de `'rt'`). Modos `'w+'` e `'w+b'` abrem e truncam o arquivo. Modos `'r+'` e `'r+b'` abrem o arquivo sem truncar o mesmo.

Conforme mencionado em Visão Geral, Python diferencia entre entrada/saída binária e de texto. Arquivos abertos em modo binário (incluindo `'b'` no parâmetro *mode*) retornam o conteúdo como objetos **`bytes`** sem usar nenhuma decodificação. No modo texto (o padrão, ou quando `'t'` é incluído no parâmetro *mode*), o conteúdo do arquivo é retornado como **`str`**, sendo os bytes primeiramente decodificados usando a codificação padrão ou usando a codificação definida em *encoding* se fornecida.

Nota

Python não depende da noção básica do sistema operacional sobre arquivos de texto; todo processamento é feito pelo próprio Python, e é então independente de plataforma.

buffering é um número inteiro opcional usado para definir a política de buffering. Passe 0 para desativar o buffer (permitido apenas no modo binário), 1 para selecionar o buffer de linha (usável apenas ao gravar no modo de texto) e um inteiro > 1 para indicar o tamanho em bytes de um buffer de bloco de tamanho fixo. Observe que especificar um tamanho de buffer dessa maneira se aplica a E/S com buffer binário, mas `TextIOWrapper` (ou seja, arquivos abertos com `mode='r+'`) teriam outro buffer. Para desabilitar o buffer em `TextIOWrapper`, considere usar o sinalizador `write_through` para **`io.TextIOWrapper.reconfigure()`**. Quando nenhum argumento *buffering* é fornecido, a política de buffering padrão funciona da seguinte forma:

- Arquivos binários são armazenados em buffer em blocos de tamanho fixo; o tamanho do buffer é `max(min(blocksize, 8 MiB), DEFAULT_BUFFER_SIZE)` quando o tamanho do bloco do dispositivo está disponível. Na maioria dos sistemas, o buffer normalmente tem 128 quilobytes de comprimento.
- Arquivos de texto “interativos” (arquivos para os quais **`isatty()`** retornam `True`) usam buffering de linha. Outros arquivos de texto usam a política descrita acima para arquivos binários.

encoding é o nome da codificação usada para codificar ou decodificar o arquivo. Isto deve ser usado apenas no modo texto. A codificação padrão é UTF-8; se Modo UTF-8 do Python estiver desabilitado, o padrão depende da plataforma (o que **`locale.getencoding()`** retornar). Qualquer codificação do texto suportada pelo Python pode ser usada, e `encoding="locale"` especifica a codificação da localidade atual explicitamente. Veja o módulo **`codecs`** para a lista de codificações suportadas.

errors é uma string opcional que especifica como erros de codificação e de decodificação devem ser tratados — isso não pode ser utilizado no modo binário. Uma variedade de tratadores de erro padrão estão disponíveis, apesar que qualquer nome para tratamento de erro registrado com **`codecs.register_error()`** também é válido. Os nomes padrões podem ser encontrados em Error Handlers.

newline determina como analisar caracteres de nova linha do fluxo. Ele pode ser `None`, `''`, `'\n'`, `'\r'` e `'\r\n'`. Ele funciona da seguinte forma:

- Ao ler a entrada do fluxo, se *newline* for `None`, o modo universal de novas linhas será ativado. As linhas na entrada podem terminar em `'\n'`, `'\r'` ou `'\r\n'`, e são traduzidas para `'\n'` antes de retornar ao chamador. Se for `''`, o modo de novas linhas universais será ativado, mas as terminações de linha serão retornadas ao chamador sem tradução. Se houver algum dos outros valores legais, as linhas de entrada são finalizadas apenas pela string especificada e a finalização da linha é retornada ao chamador sem tradução.
- Ao gravar a saída no fluxo, se *newline* for `None`, quaisquer caracteres `'\n'` gravados serão traduzidos para o separador de linhas padrão do sistema, **`os.linesep`**. Se *newline* for `''` ou `'\n'`, nenhuma tradução ocorrerá. Se *newline* for um dos outros valores legais, qualquer caractere `'\n'` escrito será traduzido para a string especificada.

Se *closefd* for `False` e um descritor de arquivo em vez de um nome de arquivo for fornecido, o descritor de arquivo subjacente será mantido aberto quando o arquivo for fechado. Se um nome de arquivo for fornecido *closefd* deve ser `True` (o padrão), caso contrário, um erro será levantado.

Um abridor personalizado pode ser usado passando um chamável como *opener*. O descritor de arquivo subjacente para o objeto arquivo é obtido chamando *opener* com (*file*, *flags*). *opener* deve retornar um descritor de arquivo aberto (passando **`os.open`** como *opener* resulta em funcionalidade semelhante à passagem de `None`).

O arquivo recém-criado é non-inheritable.

O exemplo a seguir usa o parâmetro dir_fd da função **`os.open()`** para abrir um arquivo relativo a um determinado diretório:

```
>>> import os
>>> dir_fd = os.open('algum_dir', os.O_RDONLY)
>>> def opener(path, flags):
...     return os.open(path, flags, dir_fd=dir_fd)
...
>>> with open('spamspam.txt', 'w', opener=opener) as f:
...     print('Isso será escrito para algum_dir/spamspam.txt',
...
>>> os.close(dir_fd)  # não deixe vazar um descritor de arquiv
```

O tipo de objeto arquivo retornado pela função `open()` depende do modo. Quando `open()` é usado para abrir um arquivo no modo texto (`'w'`, `'r'`, `'wt'`, `'rt'`, etc.), retorna uma subclasse de `io.TextIOBase` (especificamente `io.TextIOWrapper`). Quando usada para abrir um arquivo em modo binário com buffer, a classe retornada é uma subclasse de `io.BufferedIOBase`. A classe exata varia: no modo binário de leitura, ele retorna uma `io.BufferedReader`; nos modos binário de gravação e binário anexado, ele retorna um `io.BufferedWriter` e, no modo leitura/gravação, retorna um `io.BufferedRandom`. Quando o buffer está desativado, o fluxo bruto, uma subclasse de `io.RawIOBase`, `io.FileIO`, é retornado.

Veja também os módulos de para lidar com arquivos, tais como `fileinput`, `io` (onde `open()` é declarado), `os`, `os.path`, `tempfile` e `shutil`.

Levanta um evento de auditoria `open` com os argumentos `path`, `mode`, `flags`.

Os argumentos `mode` e `flags` podem ter sido modificados ou inferidos a partir da chamada original.

Alterado na versão 3.3:

- O parâmetro *opener* foi adicionado.
- O modo `'x'` foi adicionado.
- `IOError` costumava ser levantada, agora ela é um apelido para `OSError`.
- `FileExistsError` agora é levantado se o arquivo aberto no modo de criação exclusivo (`'x'`) já existir.

Alterado na versão 3.4:

- O arquivo agora é não herdável.

Alterado na versão 3.5:

- Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de

sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** [https://peps.python.org/pep-0475/] para entender a justificativa).
- O tratador de erros `'namereplace'` foi adicionado.

Alterado na versão 3.6:

- Suporte adicionado para aceitar objetos implementados `os.PathLike`.
- No Windows, a abertura de um buffer do console pode retornar uma subclasse de `io.RawIOBase` que não seja `io.FileIO`.

Alterado na versão 3.11: O modo `'U'` foi removido.

Alterado na versão 3.15: O UTF-8 agora é a codificação padrão, em vez de uma codificação de localidade dependente da plataforma (**PEP 686** [https://peps.python.org/pep-0686/]).

ord(*character*, /)

Retorna o valor ordinal de um caractere.

Se o argumento for uma string de um caractere, retorna o ponto de código Unicode desse caractere.. Por exemplo, `ord('a')` retorna o número inteiro `97` e `ord('€')` (sinal do Euro) retorna `8364`. Este é o inverso de `chr()`.

Se o argumento for um objeto `bytes` ou `bytearray` de comprimento 1, retorna seu valor de byte único. Por exemplo, `ord(b'a')` retorna o inteiro `97`.

pow(*base*, *exp*, *mod=None*)

Retorna *base* à potência de *exp*; se *mod* estiver presente, retorna *base* à potência *exp*, módulo *mod* (calculado com mais eficiência do que `pow(base, exp) % mod`). A forma de dois argumentos `pow(base, exp)` é equivalente a usar o operador de potência: `base**exp`.

Quando argumentos são tipos numéricos embutidos com tipos de operandos mistos, aplicam-se as regras de coerção para operadores aritméticos binários. Para operandos `int`, o resultado tem o mesmo tipo

que os operandos (após coerção), a menos que o segundo argumento seja negativo; nesse caso, todos os argumentos são convertidos em ponto flutuante e um resultado ponto flutuante é entregue. Por exemplo, `pow(10, 2)` retorna `100`, mas `pow(10, -2)` retorna `0.01`. Para uma base negativa do tipo **`int`** ou **`float`** e um expoente não integral, um resultado complexo é entregue. Por exemplo, `pow(-9, 0.5)` retorna um valor próximo a `3j`. Enquanto isso, para uma base negativa do tipo **`int`** ou **`float`** com um expoente integral, um resultado de ponto flutuante é retornado. Por exemplo, `pow(-9, 2.0)` retorna `81.0`.

Para operandos **`int`** *base* e *exp*, se *mod* estiver presente, *mod* também deve ser do tipo inteiro e *mod* deve ser diferente de zero. Se *mod* estiver presente e *exp* for negativo, *base* deve ser relativamente primo para *mod*. Nesse caso, `pow(inv_base, -exp, mod)` é retornado, onde *inv_base* é um inverso ao *base* módulo *mod*.

Aqui está um exemplo de computação de um inverso para `38` módulo `97`:

```
>>> pow(38, -1, mod=97)
23
>>> 23 * 38 % 97 == 1
True
```

Alterado na versão 3.8: Para operandos **`int`**, a forma de três argumentos de `pow` agora permite que o segundo argumento seja negativo, permitindo o cálculo de inversos modulares.

Alterado na versão 3.8: Permite argumentos de palavra reservada. Anteriormente, apenas argumentos posicionais eram suportados.

print(**objects, sep=' ', end='\n', file=None, flush=False*)

Exibe *objects* no fluxo de texto *arquivo*, separado por *sep* e seguido por *end*. *sep*, *end*, *file* e *flush*, se houver, devem ser fornecidos como argumentos nomeados.

Todos os argumentos que não são nomeados são convertidos em strings como **`str()`** faz e gravados no fluxo, separados por *sep* e seguidos por *end*. *sep* e *end* devem ser strings; eles também podem ser `None`, o que

significa usar os valores padrão. Se nenhum *object* for fornecido, **print()** escreverá apenas *end*.

O argumento *file* deve ser um objeto com um método `write(string)`; se ele não estiver presente ou `None`, então **sys.stdout** será usado. Como argumentos exibidos no console são convertidos para strings de texto, **print()** não pode ser usado com objetos arquivo em modo binário. Para esses casos, use `file.write(...)` ao invés.

O buffer de saída geralmente é determinado por *arquivo*. No entanto, se *flush* for verdadeiro, o fluxo será descarregado à força.

Alterado na versão 3.3: Adicionado o argumento nomeado *flush*.

class property(*fget=None*, *fset=None*, *fdel=None*, *doc=None*)

Retorna um atributo de propriedade.

fget é uma função para obter o valor de um atributo. *fset* é uma função para definir um valor para um atributo. *fdel* é uma função para deletar um valor de um atributo. E *doc* cria um docstring para um atributo.

Um uso comum é para definir um atributo gerenciável `x`:

```
class C:
    def __init__(self):
        self._x = None

    def getx(self):
        return self._x

    def setx(self, valor):
        self._x = valor

    def delx(self):
        del self._x

    x = property(getx, setx, delx, "Eu sou a propriedade de 'x
```

Se *c* é uma instância de *C*, `c.x` irá invocar o método getter, `c.x = valor` irá invocar o método setter, e `del c.x` o método deleter.

Se fornecido, *doc* será a docstring do atributo definido por property. Caso contrário, a property copiará a docstring de *fget* (se ela existir). Isso torna possível criar facilmente propriedades apenas para leitura usando **`@property`** como um decorador:

```
class Parrot:
    def __init__(self):
        self._voltagem = 100000

    @property
    def voltagem(self):
        """Obtém a voltagem atual."""
        return self._voltagem
```

O decorador `@property` transforma o método **`voltage()`** em um "getter" para um atributo somente leitura com o mesmo nome, e define a docstring de *voltage* para "Get the current voltage."

@getter

@setter

@deleter

Um objeto property possui métodos `getter`, `setter` e `deleter` usáveis como decoradores, que criam uma cópia da property com o assessor correspondente a função definida para a função com decorador. Isso é explicado melhor com um exemplo:

```
class C:
    def __init__(self):
        self._x = None

    @property
    def x(self):
        """Sou a propriedade de 'x'."""
        return self._x

    @x.setter
    def x(self, value):
        self._x = value

    @x.deleter
```

```
def x(self):
    del self._x
```

Esse código é exatamente equivalente ao primeiro exemplo. Tenha certeza de nas funções adicionais usar o mesmo nome que a property original (`x` neste caso).

O objeto property retornado também tem os atributos `fget`, `fset`, e `fdel` correspondendo aos argumentos do construtor.

Alterado na versão 3.5: Agora é possível escrever nas docstrings de objetos property.

__name__

Atributo que contém o nome da propriedade. O nome da propriedade pode ser alterado em tempo de execução.

Adicionado na versão 3.13.

class range(*stop*, /)

class range(*start*, *stop*, *step=1*, /)

Em vez de ser uma função, **`range`** é realmente um tipo de sequência imutável, conforme documentado em Intervalos e Tipos sequências — list, tuple, range.

repr(*object*, /)

Retorna uma string contendo uma representação imprimível de um objeto. Para muitos tipos, essa função tenta retornar uma string que produziria um objeto com o mesmo valor quando passado para **`eval()`**, caso contrário, a representação é uma string entre colchetes angulares que contém o nome do tipo do objeto juntamente com informações adicionais, geralmente incluindo o nome e o endereço do objeto. Uma classe pode controlar o que essa função retorna para suas instâncias, definindo um método **`__repr__()`**. Se **`sys.displayhook()`** não estiver acessível, esta função vai levantar **`RuntimeError`**.

Esta classe possui uma representação personalizada que pode ser executada:

```
class Pessoa:
   def __init__(self, nome, idade):
      self.nome = nome
      self.idade = idade

   def __repr__(self):
      return f"Pessoa({self.nome!r}, {self.idade!r})"
```

reversed(*object*, /)

Retorna um iterador reverso. *seq* deve ser um objeto que possui o método `__reversed__()` ou suporta o protocolo de sequência (o método `__len__()` e o método `__getitem__()` com argumentos inteiros começando em `0`).

round(*number*, *ndigits=None*)

Retorna *number* arredondado para *ndigits* precisão após o ponto decimal. Se *ndigits* for omitido ou for `None`, ele retornará o número inteiro mais próximo de sua entrada.

Para os tipos embutidos com suporte a `round()`, os valores são arredondados para o múltiplo mais próximo de 10 para a potência de menos *ndigit*; se dois múltiplos são igualmente próximos, o arredondamento é feito para a opção par (por exemplo, `round(0.5)` e `round(-0.5)` são `0` e `round(1.5)` é `2`). Qualquer valor inteiro é válido para *ndigits* (positivo, zero ou negativo). O valor de retorno é um número inteiro se *ndigits* for omitido ou `None`. Caso contrário, o valor de retorno tem o mesmo tipo que *number*.

Para um objeto Python geral `number`, `round` delega para `number.__round__`.

Nota

O comportamento de `round()` para pontos flutuantes pode ser surpreendente: por exemplo, `round(2.675, 2)` fornece `2.67` em vez do esperado `2.68`. Isso não é um bug: é resultado do fato de que a maioria das frações decimais não pode ser representada exatamente como um ponto flutuante. Veja Aritmética de ponto flutuante: problemas e limitações para mais informações.

class set(*iterable=()*, *[/]*)

Retorna um novo objeto **set**, opcionalmente com elementos obtidos de *iteráveis*. **set** é uma classe embutida. Veja também Tipo conjuntos — set, frozenset para documentação sobre essa classe.

Para outros contêineres, consulte as classes embutidas **frozenset**, **list**, **tuple** e **dict**, bem como o módulo **collections**.

setattr(*object*, *name*, *value*, *[/]*)

Esta é a contrapartida de **getattr()**. Os argumentos são um objeto, uma string e um valor arbitrário. A string pode nomear um atributo existente ou um novo atributo. A função atribui o valor ao atributo, desde que o objeto permita. Por exemplo, `setattr(x, 'foobar', 123)` é equivalente a `x.foobar = 123`.

name não precisa ser um identificador do Python conforme definido em Nomes (identificadores e palavras reservadas) a menos que o objeto opte por impor isso, por exemplo, em um **__getattribute__()** personalizado ou via **__slots__**. Um atributo cujo nome não é um identificador não será acessível usando a notação de ponto, mas pode ser acessado através de **getattr()** etc.

Nota

Uma vez que desfiguração de nome privado acontece em tempo de compilação, deve-se manualmente mutilar o nome de um atributo privado (atributos com dois sublinhados à esquerda) para defini-lo com **setattr()**.

class sentinel(*name*, *[/]*, *[*]*, *repr=None*)

Retorna um novo objeto sinalizador único. *name* deve ser uma **str** e é utilizado, por padrão como a representação do objeto retornado:

```
>>> FALTANDO = sentinel("FALTANDO")
>>> FALTANDO
FALTANDO
```

O argumento opcional *repr* pode ser utilizado para especificar uma representação diferente:

```
>>> MISSING = sentinel("MISSING", repr="<MISSING>")
>>> MISSING
<MISSING>
```

Os objetos sinalizador são sempre verdadeiros e as comparações de igualdade são apenas entre si. Eles são destinados a serem comparados com o operador **`is`**.

`sentinel` não dá suporte a subclasse.

Cópias superficiais ou profundas de um objeto sinalizador retornam o próprio objeto.

Por convenção, sinalizadores são atribuídas a uma variável com um nome correspondente. As sentinelas definidas desta forma podem ser utilizadas em dicas de tipo:

```
MISSING = sentinel("MISSING")

def next_value(default: int | MISSING = MISSING):
    ...
```

Os objetos sinalizador dá suporte ao operador | para utilização em expressões de tipo.

A serialização com **`<pickle>`** é possível para objetos sinalizadores colocados no escopo global de um módulo, com um nome correspondente ao do sinalizador, e para sinalizadores colocados no escopo de classe, com um nome correspondente à nome qualificado do sinalizador. Outros sinalizadores, como os definidos no escopo de uma função, não são serializáveis com pickle. A identidade do sinalizador é preservada após a serialização:

```
import pickle

PICKLABLE = sentinel("PICKLABLE")

assert pickle.loads(pickle.dumps(PICKLABLE)) is PICKLABLE
```

```
class Cls:
    PICKLABLE = sentinel("Cls.PICKLABLE")

assert pickle.loads(pickle.dumps(Cls.PICKLABLE)) is Cls.PICKLA
```

Os objetos sinalizador têm os seguintes atributos:

__name__
O nome do sinalizador.

__module__
O nome do módulo onde o sinalizador foi criado. Este atributo é gravável.

Adicionado na versão 3.15.

class slice(*stop*, /)
class slice(*start*, *stop*, *step=None*, /)

Retorna um objeto fatia representando o conjunto de índices especificados por `range(start, stop, step)`. Os argumentos *start* e *step* têm o padrão `None`.

Objetos fatia também são gerados quando a sintaxe de fatiamento é usada. Por exemplo: `a[start:stop:step]` ou `a[start:stop, i]`.

Veja **`itertools.islice()`** para uma versão alternativa que retorna um iterador.

start
stop
step
Esses atributos somente leitura são definidos para os valores dos argumentos (ou seus valores padrão). Eles não possuem outra funcionalidade explícita; no entanto, eles são usados pelo NumPy e outros pacotes de terceiros.

Alterado na versão 3.12: Os objetos slice agora são hasheáveis (desde que `start`, `stop` e `step` sejam hasheáveis).

sorted(*iterable*, /, *, *key=None*, *reverse=False*)

Retorna uma nova lista classificada dos itens em *iterable*.

Possui dois argumentos opcionais que devem ser especificados como argumentos nomeados.

key especifica a função de um argumento usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). O valor padrão é `None` (compara os elementos diretamente).

reverse é um valor booleano. Se definido igual a `True`, então os elementos da lista são classificados como se cada comparação estivesse invertida.

Usa `functools.cmp_to_key()` para converter a função das antigas *cmp* para uma função *key*.

A função embutida `sorted()` é garantida como estável. Uma ordenação é estável se garantir não alterar a ordem relativa dos elementos que se comparam da mesma forma — isso é útil para ordenar em várias passagens (por exemplo, ordenar por departamento e depois por nível de salário).

O algoritmo de classificação usa apenas comparações < entre itens. Embora definir um método `__lt__()` seja suficiente para ordenação, **PEP 8** [https://peps.python.org/pep-0008/] recomenda que todas as seis comparações ricas sejam implementadas. Isso ajudará a evitar erros ao usar os mesmos dados com outras ferramentas de ordenação, como `max()`, que dependem de um método subjacente diferente. Implementar todas as seis comparações também ajuda a evitar confusão para comparações de tipo misto que podem chamar o método espelhado `__gt__()`.

Para exemplos de classificação e um breve tutorial de classificação, veja Técnicas de ordenação.

@staticmethod

Transforma um método em método estático.

Um método estático não recebe um primeiro argumento implícito. Para declarar um método estático, use este idioma:

```
class C:
    @staticmethod
    def f(arg1, arg2, argN): ...
```

A forma `@staticmethod` é uma função de decorador – veja Definições de função para detalhes.

Um método estático pode ser chamado na classe (tal como `C.f()`) ou em uma instância (tal como `C().f()`). Além disso, o descritor de método estático também é um chamável, então ele pode ser usado na definição de classe (como `f()`).

Métodos estáticos em Python são similares àqueles encontrados em Java ou C++. Veja também **`@classmethod`** para uma variante útil na criação de construtores de classe alternativos.

Como todos os decoradores, também é possível chamar `staticmethod` como uma função regular e fazer algo com seu resultado. Isso é necessário em alguns casos em que você precisa de uma referência para uma função de um corpo de classe e deseja evitar a transformação automática em método de instância. Para esses casos, use este idioma:

```
def função_comum():
    ...

class C:
    método = staticmethod(função_comum)
```

Para mais informações sobre métodos estáticos, consulte A hierarquia de tipos padrão.

Alterado na versão 3.10: Métodos estáticos agora herdam os atributos do método (**`__module__`**, **`__name__`**, **`__qualname__`**, **`__doc__`** e **`__annotations__`**), têm um novo atributo `__wrapped__` e são agora chamáveis como funções regulares.

class str(*, *encoding='utf-8'*, *errors='strict'*)
class str(*object*)
class str(*object*, *encoding*, *errors='strict'*)
class str(*object*, *, *errors*)

Retorna uma versão **`str`** de *object*. Consulte **`str()`** para detalhes.

`str` é uma classe de string embutida. Para informações gerais sobre strings, consulte Tipo sequência de texto — str.

sum(*iterable*, /, *start=0*)

Soma *start* e os itens de um *iterable* da esquerda para a direita e retornam o total. Os itens do *iterable* são normalmente números e o valor inicial não pode ser uma string.

Para alguns casos de uso, existem boas alternativas para **`sum()`**. A maneira rápida e preferida de concatenar uma sequência de strings é chamando `''.join(sequence)`. Para adicionar valores de ponto flutuante com precisão estendida, consulte **`math.fsum()`**. Para concatenar uma série de iteráveis, considere usar **`itertools.chain()`**.

Alterado na versão 3.8: O parâmetro *start* pode ser especificado como um argumento nomeado.

Alterado na versão 3.12: A soma dos pontos flutuantes foi alterada para um algoritmo que oferece maior precisão e melhor comutatividade na maioria das compilações.

Alterado na versão 3.14: Adicionada especialização para soma de complexos, usando o mesmo algoritmo para soma de flutuantes.

class super
class super(*type*, *object_or_type=None*, /)

Retorna um objeto proxy que delega chamadas de método a uma classe pai ou irmão do *type*. Isso é útil para acessar métodos herdados que foram substituídos em uma classe.

O *object_or_type* determina a ordem de resolução de métodos a ser pesquisada. A pesquisa inicia a partir da classe logo após o *type*.

Por exemplo, se `__mro__` de *object_or_type* é `D -> B -> C -> A -> object` e o valor de *type* é `B`, então `super()` procura por `C -> A -> object`.

O atributo `__mro__` da classe correspondente ao *object_or_type* lista a ordem de pesquisa de resolução de método usada por `getattr()` e `super()`. O atributo é dinâmico e pode mudar sempre que a hierarquia da herança é atualizada.

Se o segundo argumento for omitido, o objeto super retornado é desacoplado. Se o segundo argumento é um objeto, `isinstance(obj, type)` deve ser verdadeiro. Se o segundo argumento é um tipo, `issubclass(type2, type)` deve ser verdadeiro (isto é útil para classmethods).

Quando chamado diretamente de dentro de um método comum de uma classe, pode-se omitir ambos os argumentos ("**`super()`** de zero argumentos"). Neste caso, *type* será a classe em questão, e *obj* será o primeiro argumento da função que imediatamente chamou `super()` (geralmente o `self`). (Isso significa que o **`super()`** de zero argumentos não vai funcionar como esperado em funções aninhadas, incluindo expressões geradoras, que criam funções aninhadas implicitamente.)

Existem dois casos de uso típicos para *super*. Em uma hierarquia de classes com herança única, *super* pode ser usado para se referir a classes base sem nomeá-las explicitamente, tornando o código mais sustentável. Esse uso é paralelo ao uso de *super* em outras linguagens de programação.

O segundo caso de uso é oferecer suporte à herança múltipla cooperativa em um ambiente de execução dinâmica. Esse caso de uso é exclusivo do Python e não é encontrado em idiomas ou linguagens compiladas estaticamente que suportam apenas herança única. Isso torna possível implementar "diagramas em losango", onde várias classes base implementam o mesmo método. Um bom design exige que tais implementações tenham a mesma assinatura de chamada em todos os

casos (porque a ordem das chamadas é determinada em tempo de execução, porque essa ordem se adapta às alterações na hierarquia de classes e porque essa ordem pode incluir classes de irmãos desconhecidas antes do tempo de execução).

Nos dois casos de uso, uma chamada típica de superclasse se parece com isso:

```
class C(B):
    def método(self, arg):
        super().método(arg)    # Isso faz o mesmo que:
                               # super(C, self).método(arg)
```

Além das pesquisas de método, **`super()`** também funciona para pesquisas de atributo. Um possível caso de uso para isso é chamar descritores em uma classe pai ou irmã.

Observe que **`super()`** é implementada como parte do processo de vinculação para procura explícita de atributos com ponto, tal como `super().__getitem__(nome)`. Ela faz isso implementando seu próprio método **`__getattribute__()`** para pesquisar classes em uma ordem predizível que possui suporte a herança múltipla cooperativa. Logo, **`super()`** não é definida para procuras implícitas usando instruções ou operadores como `super()[name]`.

Observe também que, além da forma de argumento zero, **`super()`** não se limita ao uso de métodos internos. O formulário de dois argumentos especifica exatamente os argumentos e faz as referências apropriadas. O formulário de argumento zero funciona apenas dentro de uma definição de classe, pois o compilador preenche os detalhes necessários para recuperar corretamente a classe que está sendo definida, além de acessar a instância atual para métodos comuns.

Para sugestões práticas sobre como projetar classes cooperativas usando **`super()`**, consulte o guia para uso de super() [https://rhettinger.wordpress.com/2011/05/26/super-considered-super/].

Alterado na versão 3.14: Os objetos **`super`** agora são **`serializáveis`** e **`copiáveis`**.

class tuple(*iterable=()*, /)

Ao invés de ser uma função, `tuple` é na verdade um tipo de sequência imutável, conforme documentado em Tuplas e Tipos sequências — list, tuple, range.

class type(*object*, /)

class type(*name*, *bases*, *dict*, /, ***kwargs*)

Com um argumento, retorna o tipo de um *object*. O valor de retorno é um tipo de objeto e geralmente o mesmo objeto retornado por `object.__class__`.

A função embutida `isinstance()` é recomendada para testar o tipo de um objeto, porque ela leva sub-classes em consideração.

Com três argumentos, retorna um novo objeto type. Esta é essencialmente a forma dinâmica da instrução `class`. A string *name* é o nome da classe e se torna o atributo `__name__`. A tupla *bases* contém as classes bases e se torna o atributo `__bases__`; se vazio, `object`, a base final de todas as classes é adicionada. O dicionário *dict* contém definições de atributo e método para o corpo da classe; ele pode ser copiado ou envolto antes de se tornar o atributo `__dict__`. As duas instruções a seguir criam objetos `type` idênticos:

```
>>> class X:
...     a = 1
...
>>> X = type('X', (), dict(a=1))
```

Veja também:

- Documentação sobre atributos e métodos em classes.
- Objetos tipo

Argumentos nomeados fornecidos para a forma de três argumentos são passados para a máquina metaclasse apropriada (geralmente `__init_subclass__()`) da mesma forma que palavras-chave em uma definição de classe (além de *metaclasse*) fariam.

Diferentemente de uma instrução `class`, a forma de três argumento não chama o método metaclasse `__prepare__` (veja Preparando o espaço de nomes da classe). Ise `types.new_class()` para criar dinamicamente uma classe usando a metaclasse apropriada.

Veja também Personalizando a criação de classe.

Alterado na versão 3.6: Subclasses de `type` que não substituem `type.__new__` não podem mais usar a forma com apenas um argumento para obter o tipo de um objeto.

Alterado na versão 3.15: dict agora pode ser um `frozendict`.

vars()
vars(*object*, /)

Retorna o atributo `__dict__` para um módulo, classe, instância, or qualquer outro objeto com um atributo `__dict__`.

Objetos como modelos e instâncias têm um atributo atualizável `__dict__`; porém, outros projetos podem ter restrições de escrita em seus atributos `__dict__` (por exemplo, classes usam um `types.MappingProxyType` para prevenir atualizações diretas a dicionário).

Sem nenhum argumento, `vars()` age como `locals()`.

Uma exceção `TypeError` é levantada se um objeto é especificado, mas ela não tem um atributo `__dict__` (por exemplo, se sua classe define o atributo `__slots__`).

Alterado na versão 3.13: O resultado de chamar esta função sem um argumento foi atualizado conforme descrito na embutida `locals()`.

zip(**iterables*, *strict=False*)

Itera sobre vários iteráveis em paralelo, produzindo tuplas com um item de cada um.

Exemplo:

```
>>> for item in zip([1, 2, 3], ['açúcar', 'tempero', 'tudo que
...     print(item)
...
(1, 'açúcar')
(2, 'tempero')
(3, 'tudo que há de bom')
```

Mais formalmente: **`zip()`** retorna um iterador de tuplas, onde a *i*-ésima tupla contém o *i*-ésimo elemento de cada um dos iteráveis do argumento.

Outra maneira de pensar em **`zip()`** é que ela transforma linhas em colunas e colunas em linhas. Isso é semelhante a transpor uma matriz [https://en.wikipedia.org/wiki/Transpose].

`zip()` é preguiçoso: Os elementos não serão processados até que o iterável seja iterado. Por exemplo, por um loop **`for`** ou por um **`list`**.

Uma coisa a considerar é que os iteráveis passados para **`zip()`** podem ter comprimentos diferentes; às vezes por design e às vezes por causa de um bug no código que preparou esses iteráveis. Python oferece três abordagens diferentes para lidar com esse problema:

- Por padrão, **`zip()`** para quando o iterável mais curto tenha sido esgotado. Ele irá ignorar os itens restantes nos iteráveis mais longos, cortando o resultado para o comprimento do iterável mais curto:

  ```
  >>> list(zip(range(3), ['fi', 'fa', 'fo', 'fum']))
  [(0, 'fi'), (1, 'fa'), (2, 'fo')]
  ```

- **`zip()`** é frequentemente usado em casos onde os iteráveis são considerados de tamanho igual. Nesses casos, é recomendado usar a opção `strict=True`. Sua saída é a mesma do **`zip()`**:: normal

  ```
  >>> list(zip(('a', 'b', 'c'), (1, 2, 3), strict=True))
  [('a', 1), ('b', 2), ('c', 3)]
  ```

  Ao contrário do comportamento padrão, ele levanta uma exceção **`ValueError`** se um iterável for esgotado antes dos outros:

  ```
  >>> for item in zip(range(3), ['fee', 'fi', 'fo', 'fum'],
  ...     print(item)
  ```

```
...
(0, 'fee')
(1, 'fi')
(2, 'fo')
Traceback (most recent call last):
  ...
ValueError: zip() argument 2 is longer than argument 1
```

Sem o argumento `strict=True`, qualquer bug que resulte em iteráveis de diferentes comprimentos será silenciado, possivelmente se manifestando como um bug difícil de encontrar em outra parte do programa.

- Iteráveis mais curtos podem ser preenchidos com um valor constante para fazer com que todos os iteráveis tenham o mesmo comprimento. Isso é feito por **`itertools.zip_longest()`**.

Casos extremos: Com um único argumento iterável, **`zip()`** retorna um iterador de tuplas de um elemento. Sem argumentos, retorna um iterador vazio.

Dicas e truques:

- A ordem de avaliação da esquerda para a direita dos iteráveis é garantida. Isso torna possível um idioma para agrupar uma série de dados em grupos de comprimento n usando `zip(*[iter(s)]*n, strict=True)`. Isso repete o *mesmo* iterador `n` vezes para que cada tupla de saída tenha o resultado de chamadas `n` para o iterador. Isso tem o efeito de dividir a entrada em pedaços de n comprimentos.

- **`zip()`** em conjunto com o operador `*` pode ser usado para descompactar uma lista:

```
>>> x = [1, 2, 3]
>>> y = [4, 5, 6]
>>> list(zip(x, y))
[(1, 4), (2, 5), (3, 6)]
>>> x2, y2 = zip(*zip(x, y))
>>> x == list(x2) and y == list(y2)
True
```

Alterado na versão 3.10: Adicionado o argumento `strict`.

__import__(*name*, *globals=None*, *locals=None*, *fromlist=()*, *level=0*)

Nota

Esta é uma função avançada que não é necessária na programação diária do Python, ao contrário de `importlib.import_module()`.

Esta função é chamada pela instrução `import`. Ela pode ser substituída (importando o módulo `builtins` e atribuindo a `builtins.__import__`) para alterar a semântica da instrução `import`, mas isso é **fortemente** desencorajado, pois geralmente é mais simples usar ganchos de importação (consulte **PEP 302** [https://peps.python.org/pep-0302/]) para atingir os mesmos objetivos e não causa problemas com o código que pressupõe que a implementação de importação padrão esteja em uso. O uso direto de `__import__()` também é desencorajado em favor de `importlib.import_module()`.

A função importa o módulo *name*, potencialmente usando os dados *globals* e *locals* para determinar como interpretar o nome em um contexto de pacote. O *fromlist* fornece os nomes de objetos ou submódulos que devem ser importados do módulo, fornecidos por *name*. A implementação padrão não usa seu argumento *locals* e usa seus *globals* apenas para determinar o contexto do pacote da instrução `import`.

level especifica se é necessário usar importações absolutas ou relativas. `0` (o padrão) significa apenas realizar importações absolutas. Valores positivos para *level* indicam o número de diretórios pai a serem pesquisados em relação ao diretório do módulo que chama `__import__()` (consulte **PEP 328** [https://peps.python.org/pep-0328/] para obter detalhes).

Quando a variável *name* está no formato `pacote.módulo`, normalmente, o pacote de nível superior (o nome até o primeiro ponto) é retornado, *não* o módulo nomeado por *name*. No entanto, quando um argumento *fromlist* não vazio é fornecido, o módulo nomeado por *name* é retornado.

Por exemplo, a instrução `import spam` resulta em bytecode semelhante ao seguinte código:

```
spam = __import__('spam', globals(), locals(), [], 0)
```

A instrução `import spam.presunto` resulta nesta chamada:

```
spam = __import__('spam.presunto', globals(), locals(), [], 0)
```

Observe como `__import__()` retorna o módulo de nível superior aqui, porque este é o objeto vinculado a um nome pela instrução `import`.

Por outro lado, a instrução `from spam.presunto import ovos, salsicha as sals` resulta em

```
_temp = __import__('spam.presunto', globals(), locals(), ['ovo
ovos = _temp.ovos
sals = _temp.salsicha
```

Aqui, o módulo `spam.ham` é retornado de `__import__()`. A partir desse objeto, os nomes a serem importados são recuperados e atribuídos aos seus respectivos nomes.

Se você simplesmente deseja importar um módulo (potencialmente dentro de um pacote) pelo nome, use `importlib.import_module()`.

Alterado na versão 3.3: Valores negativos para *level* não são mais suportados (o que também altera o valor padrão para 0).

Alterado na versão 3.9: Quando as opções de linha de comando `-E` ou `-I` estão sendo usadas, a variável de ambiente `PYTHONCASEOK` é agora ignorada.

Notas de rodapé

[1]

Observe que o analisador sintático aceita apenas a convenção de fim de linha no estilo Unix. Se você estiver lendo o código de um arquivo, use o modo de conversão de nova linha para converter novas linhas no estilo Windows ou Mac.

Exceções embutidas

No Python, todas as exceções devem ser instâncias de uma classe derivada de `BaseException`. Em uma instrução `try` com uma cláusula `except` que menciona uma classe específica, essa cláusula também lida com qualquer classe de exceção derivada dessa classe (mas não com as classes de exceção a partir das quais *ela* é derivada). Duas classes de exceção que não são relacionadas por subclasse nunca são equivalentes, mesmo que tenham o mesmo nome.

As exceções embutidas listadas neste capítulo podem ser geradas pelo interpretador ou pelas funções embutidas. Exceto onde mencionado, eles têm um "valor associado" indicando a causa detalhada do erro. Pode ser uma string ou uma tupla de vários itens de informação (por exemplo, um código de erro e uma string que explica o código). O valor associado geralmente é passado como argumentos para o construtor da classe de exceção.

O código do usuário pode gerar exceções embutidas. Isso pode ser usado para testar um manipulador de exceções ou para relatar uma condição de erro "exatamente como" a situação na qual o interpretador gera a mesma exceção; mas lembre-se de que nada impede o código do usuário de gerar um erro inadequado.

As classes de exceções embutidas podem ser usadas como subclasses para definir novas exceções; Os programadores são incentivados a derivar novas exceções da classe `Exception` ou de uma de suas subclasses, e não de `BaseException`. Mais informações sobre a definição de exceções estão disponíveis no Tutorial do Python em Exceções definidas pelo usuário.

Contexto da exceção

Três atributos em objetos exceções fornecem informações sobre o contexto em que a exceção foi levantada.

BaseException.__context__
BaseException.__cause__
BaseException.__suppress_context__

Ao levantar uma nova exceção enquanto outra exceção já está sendo tratada, o atributo **__context__** da nova exceção é automaticamente definido para a exceção tratada. Uma exceção pode ser tratada quando uma cláusula **except** ou **finally**, ou uma instrução **with**, é usada.

Esse contexto implícito da exceção pode ser complementado com uma causa explícita usando **from** com **raise**:

```
raise new_exc from original_exc
```

A expressão a seguir **from** deve ser uma exceção ou `None`. Ela será definida como **__cause__** na exceção levantada. A definição de **__cause__** também define implicitamente o atributo **__suppress_context__** como `True`, de modo que o uso de `raise new_exc from None` substitui efetivamente a exceção antiga pela nova para fins de exibição (por exemplo, convertendo **KeyError** para **AttributeError**), deixando a exceção antiga disponível em **__context__** para introspecção durante a depuração.

O código de exibição padrão do traceback mostra essas exceções encadeadas, além do traceback da própria exceção. Uma exceção explicitamente encadeada em **__cause__** sempre é mostrada quando presente. Uma exceção implicitamente encadeada em **__context__** é mostrada apenas se **__cause__** for **None** e **__suppress_context__** for falso.

Em qualquer um dos casos, a exceção em si sempre é mostrada após todas as exceções encadeadas, de modo que a linha final do traceback sempre mostre a última exceção que foi levantada.

Herdando de exceções embutidas

O código do usuário pode criar subclasses que herdam de um tipo de exceção. É recomendado criar subclasse de apenas um tipo de exceção por

vez para evitar possíveis conflitos entre como as bases tratam o atributo `args`, bem como devido a possíveis incompatibilidades de layout de memória.

A maioria das exceções embutidas são implementadas em C para eficiência, veja: Objects/exceptions.c [https://github.com/python/cpython/tree/3.15/Objects/exceptions.c]. Algumas têm layouts de memória personalizados, o que impossibilita a criação de uma subclasse que herda de vários tipos de exceção. O layout de memória de um tipo é um detalhe de implementação e pode mudar entre as versões do Python, levando a novos conflitos no futuro. Portanto, é recomendável evitar criar subclasses de vários tipos de exceção.

Classes base

As seguintes exceções são usadas principalmente como classes base para outras exceções.

exception BaseException

A classe base para todas as exceções embutidas. Não é para ser herdada diretamente por classes definidas pelo usuário (para isso, use **`Exception`**). Se **`str()`** for chamado em uma instância desta classe, a representação do(s) argumento(s) para a instância será retornada ou a string vazia quando não houver argumentos.

args

A tupla de argumentos fornecidos ao construtor de exceções. Algumas exceções embutidas (como **`OSError`**) esperam um certo número de argumentos e atribuem um significado especial aos elementos dessa tupla, enquanto outras são normalmente chamadas apenas com uma única string que fornece uma mensagem de erro.

with_traceback(*tb*)

Este método define *tb* como o novo traceback (situação da pilha de execução) para a exceção e retorna o objeto exceção. Era mais comumente usado antes que os recursos de encadeamento de exceções de **PEP 3134** [https://peps.python.org/pep-3134/] se tornassem

disponíveis. O exemplo a seguir mostra como podemos converter uma instância de `SomeException` em uma instância de `OtherException` enquanto preservamos o traceback. Uma vez gerado, o quadro atual é empurrado para o traceback de `OtherException`, como teria acontecido com o traceback de `SomeException` original se tivéssemos permitido que ele se propagasse para o chamador.

```
try:
    ...
except SomeException:
    tb = sys.exception().__traceback__
    raise OtherException(...).with_traceback(tb)
```

__traceback__

Um campo gravável que contém o objeto traceback associado a esta exceção. Veja também: A instrução raise.

add_note(*note*)

Adiciona a string `note` às notas da exceção que aparecem no traceback padrão após a string de exceção. Uma exceção **`TypeError`** é levantada se `note` não for uma string.

Adicionado na versão 3.11.

__notes__

Uma lista das notas desta exceção, que foram adicionadas com **`add_note()`**. Este atributo é criado quando **`add_note()`** é chamado.

Adicionado na versão 3.11.

exception Exception

Todas as exceções embutidas que não saem para o sistema são derivadas dessa classe. Todas as exceções definidas pelo usuário também devem ser derivadas dessa classe.

exception ArithmeticError

A classe base para as exceções embutidas levantadas para vários erros aritméticos: `OverflowError`, `ZeroDivisionError`, `FloatingPointError`.

exception BufferError

Levantado quando uma operação relacionada a buffer não puder ser realizada.

exception LookupError

A classe base para as exceções levantadas quando uma chave ou índice usado em um mapeamento ou sequência é inválido: `IndexError`, `KeyError`. Isso pode ser levantado diretamente por `codecs.lookup()`.

Exceções concretas

As seguintes exceções são as que geralmente são levantadas.

exception AssertionError

Levantado quando uma instrução `assert` falha.

exception AttributeError

Levantado quando uma referência de atributo (consulte Referências de atributo) ou atribuição falha. (Quando um objeto não oferece suporte a referências ou atribuições de atributos, `TypeError` é levantado.)

Os argumentos somente-nomeados opcionais *name* e o *obj* definem os atributos correspondente:

name

O nome do atributo a que se tentou obter acesso.

obj

O objeto acessado pelo atributo nomeado.

Alterado na versão 3.10: Adicionado os atributos `name` e `obj`.

exception EOFError

Levantado quando a função `input()` atinge uma condição de fim de arquivo (EOF) sem ler nenhum dado. (Note: os métodos `io.IOBase.read()` e `io.IOBase.readline()` retornam uma string vazia quando o EOF é atingido.)

exception FloatingPointError

Não usado atualmente.

exception GeneratorExit

Levantado quando um gerador ou uma corrotina está fechado(a); veja `generator.close()` e `coroutine.close()`. Herda diretamente de `BaseException` em vez de `Exception`, já que tecnicamente não é um erro.

exception ImportError

Levantada quando a instrução `import` tem problemas ao tentar carregar um módulo. Também é gerado quando o "from list" em `from ... import` tem um nome que não pode ser encontrado.

O argumentos somente-nomeados opcionais *name* e o *path* definem o atributos correspondente:

name

O nome do módulo que tentou-se fazer a importação.

path

O caminho para qualquer arquivo que acionou a exceção.

Alterado na versão 3.3: Adicionados os atributos `name` e `path`.

exception ModuleNotFoundError

Uma subclasse de `ImportError` que é levantada por `import` quando um módulo não pôde ser localizado. Também é levantada quando `None` é encontrado em `sys.modules`.

Adicionado na versão 3.6.

exception ImportCycleError

Uma subclasse de `ImportError` que é levantada quando uma importação preguiçosa (lazy import) falha porque ela (direta ou indiretamente) tenta importar a si mesma.

Adicionado na versão 3.15.

exception IndexError

Levantada quando um índice de alguma sequência está fora do intervalo. (Índices de fatia são truncados silenciosamente para cair num intervalo permitido; se um índice não for um inteiro, `TypeError` é levantada.)

exception KeyError

Levantada quando uma chave de mapeamento (dicionário) não é encontrada no conjunto de chaves existentes.

exception KeyboardInterrupt

Levantada quando um usuário aperta a tecla de interrupção (normalmente `Control-C` ou `Delete`). Durante a execução, uma checagem de interrupção é feita regularmente. A exceção herda de `BaseException` para que não seja capturada acidentalmente por códigos que tratam `Exception` e assim evita que o interpretador saia.

Nota

Capturar uma `KeyboardInterrupt` requer consideração especial. Como pode ser levantada em pontos imprevisíveis, pode, em algumas circunstâncias, deixar o programa em execução em um estado inconsistente. Geralmente é melhor permitir que `KeyboardInterrupt` termine o programa o mais rápido possível ou evitar levantá-la de todo. (Veja Nota sobre manipuladores de sinais e exceções.)

exception MemoryError

Levantada quando uma operação fica sem memória mas a situação ainda pode ser recuperada (excluindo alguns objetos). O valor associado é uma string que indica o tipo de operação (interna) que ficou sem memória. Observe que, por causa da arquitetura de gerenciamento de

memória subjacente (função **`malloc()`** do C), o interpretador pode não ser capaz de se recuperar completamente da situação; no entanto, levanta uma exceção para que um traceback possa ser impresso, no caso de um outro programa ser a causa.

exception NameError

Levantada quando um nome local ou global não é encontrado. Isso se aplica apenas a nomes não qualificados. O valor associado é uma mensagem de erro que inclui o nome que não pode ser encontrado.

O argumento somente-nomeado opcional *name* define o atributo:

name

O nome do variável que tentou-se fazer o acesso.

Alterado na versão 3.10: Adicionado o atributo **`name`**.

exception NotImplementedError

Essa exceção é derivada da **`RuntimeError`**. Em classes base, definidas pelo usuário, os métodos abstratos devem gerar essa exceção quando requerem que classes derivadas substituam o método, ou enquanto a classe está sendo desenvolvida, para indicar que a implementação real ainda precisa ser adicionada.

Nota

Não deve ser usada para indicar que um operador ou método não será mais suportado – nesse caso deixe o operador / método indefinido ou, se é uma subclasse, defina-o como **`None`**.

Cuidado

`NotImplementedError` e **`NotImplemented`** não são intercambiáveis. Esta exceção deve ser usada somente conforme descrito acima; veja **`NotImplemented`** para detalhes sobre o uso correto da constante embutida.

exception OSError([*arg*])

exception OSError(*errno*, *strerror*[, *filename*[, *winerror*[, *filename2*]]])

Esta exceção é levantada quando uma função do sistema retorna um erro relacionado ao sistema, incluindo falhas do tipo E/S como “file not found” ou “disk full” (não para tipos de argumentos não permitidos ou outro erro acessório).

A segunda forma do construtor definir os atributos correspondentes, descritos abaixo. Os atributos usarão o valor padrão **`None`** se não forem especificados. Por compatibilidade com versões anteriores, se três argumentos são passados, o atributo **`args`** contêm somente uma tupla de 2 elementos, os dois primeiros argumentos do construtor.

O construtor geralmente retorna uma subclasse de **`OSError`**, como descrito abaixo em OS exceptions . A subclasse particular depende do valor final de **`errno`**. Este comportamento ocorre apenas durante a construção direta ou por meio de um apelido de **`OSError`**, e não é herdado na criação de subclasses.

errno

Um código de erro numérico da variável C **`errno`**.

winerror

No Windows, isso fornece o código de erro nativo do Windows. O atributo **`errno`** é então uma tradução aproximada, em termos POSIX, desse código de erro nativo.

No Windows, se o argumento de construtor *winerror* for um inteiro, o atributo **`errno`** é determinado a partir do código de erro do Windows e o argumento *errno* é ignorado. Em outras plataformas, o argumento *winerror* é ignorado e o atributo **`winerror`** não existe.

strerror

A mensagem de erro correspondente, conforme fornecida pelo sistema operacional. É formatada pelas funções C **`perror()`** no POSIX e **`FormatMessage()`** no Windows.

filename
filename2

Para exceções que envolvem um caminho do sistema de arquivos (como `open()` ou `os.unlink()`), `filename` é o nome do arquivo passado para a função. Para funções que envolvem dois caminhos de sistema de arquivos (como `os.rename()`), `filename2` corresponde ao segundo nome de arquivo passado para a função.

Alterado na versão 3.3: `EnvironmentError`, `IOError`, `WindowsError`, `socket.error`, `select.error` e `mmap.error` foram fundidos em `OSError`, e o construtor pode retornar uma subclasse.

Alterado na versão 3.4: O atributo `filename` agora é o nome do arquivo original passado para a função, ao invés do nome codificado ou decodificado da tratador de erros e codificação do sistema de arquivos. Além disso, o argumento e o atributo de construtor *filename2* foi adicionado.

exception OverflowError

Levantada quando o resultado de uma operação aritmética é muito grande para ser representada. Isso não pode ocorrer para inteiros (que prefere levantar `MemoryError` a desistir). No entanto, por motivos históricos, OverflowError às vezes é levantada para inteiros que estão fora de um intervalo obrigatório. Devido à falta de padronização do tratamento de exceção de ponto flutuante em C, a maioria das operações de ponto flutuante não são verificadas.

exception PythonFinalizationError

Este exceção é derivada de `RuntimeError`. É levantada quando uma operação é bloqueada durante o encerramento do interpretador e também conhecido como finalização do Python.

Exemplos de operações que podem ser bloqueadas com um `PythonFinalizationError` durante a finalização do Python:

- Criação de uma nova thread no Python.
- `União` de uma thread de daemon em execução.

- `os.fork()`,
- adquirir um lock como `threading.Lock`, quando se sabe que a operação, de outro modo, causaria um deadlock.

Veja também a função `sys.is_finalizing()`.

Adicionado na versão 3.13: Anteriormente, uma `RuntimeError` simples era levantada.

Alterado na versão 3.14: `threading.Thread.join()` pode agora levantar essa exceção.

Alterado na versão 3.15: Esta exceção pode ser levantada ao adquirir `threading.Lock()` ou `threading.RLock()`.

exception RecursionError

Esta exceção é derivada de `RuntimeError`. É levantada quando o interpretador detecta que a profundidade máxima de recursão (veja `sys.getrecursionlimit()`) foi excedida.

Adicionado na versão 3.5: Anteriormente, uma `RuntimeError` simples era levantada.

exception ReferenceError

Esta exceção é levantada quando um intermediário de referência fraca, criado pela função `weakref.proxy()`, é usado para acessar um atributo do referente após ter sido coletado como lixo. Para mais informações sobre referências fracas, veja o módulo `weakref`.

exception RuntimeError

Levantada quando um erro é detectado e não se encaixa em nenhuma das outras categorias. O valor associado é uma string indicando o que precisamente deu errado.

exception StopIteration

Levantada pela função embutida `next()` e o método `__next__()` de um iterador para sinalizar que não há mais itens produzidos pelo iterador.

value

O objeto exceção tem um único atributo **`value`**, que é fornecido como um argumento ao construir a exceção, e o padrão é **`None`**.

Quando uma função geradora ou corrotina retorna, uma nova instância **`StopIteration`** é levantada, e o valor retornado pela função é usado como o parâmetro **`value`** para o construtor da exceção.

Se um código gerador direta ou indiretamente levantar **`StopIteration`**, ele é convertido em uma **`RuntimeError`** (mantendo o **`StopIteration`** como a nova causa da exceção).

Alterado na versão 3.3: Adicionado o atributo `value` e a capacidade das funções geradoras de usá-lo para retornar um valor.

Alterado na versão 3.5: Introduzida a transformação RuntimeError via `from __future__ import generator_stop`, consulte **PEP 479** [https://peps.python.org/pep-0479/].

Alterado na versão 3.7: Habilita **PEP 479** [https://peps.python.org/pep-0479/] para todo o código por padrão: um erro **`StopIteration`** levantado em um gerador é transformado em uma **`RuntimeError`**.

exception StopAsyncIteration

Deve ser levantada pelo método **`__anext__()`** de um objeto iterador assíncrono para parar a iteração.

Adicionado na versão 3.5.

exception SyntaxError(*message*, *details*)

Levantada quando o analisador encontra um erro de sintaxe. Isso pode ocorrer em uma instrução **`import`**, em uma chamada às funções embutidas **`compile()`**, **`exec()`** ou **`eval()`**, ou ao ler o script inicial ou entrada padrão (também interativamente).

A função **`str()`** da instância de exceção retorna apenas a mensagem de erro. Detalhes é uma tupla cujos membros também estão disponíveis como atributos separados.

filename

O nome do arquivo em que ocorreu o erro de sintaxe.

lineno

Em qual número de linha no arquivo o erro ocorreu. Este é indexado em 1: a primeira linha no arquivo tem um `lineno` de 1.

offset

A coluna da linha em que ocorreu o erro. Este é indexado em 1: o primeiro caractere na linha tem um `offset` de 1.

text

O texto do código-fonte envolvido no erro.

end_lineno

Em qual número de linha no arquivo o erro ocorrido termina. Este é indexado em 1: a primeira linha no arquivo tem um `lineno` de 1.

end_offset

A coluna da linha final em que o erro ocorreu termina. Este é indexado em 1: o primeiro caractere na linha tem um `offset` de 1.

Para erros em campos de f-string, a mensagem é prefixada por “f-string: “ e os “offsets” são deslocamentos em um texto construído a partir da expressão de substituição. Por exemplo, compilar o campo f’Bad {a b}’ resulta neste atributo de argumentos: (‘f-string: …’, (‘’, 1, 2, ‘(a b)n’, 1, 5)).

Alterado na versão 3.10: Adicionados os atributos **`end_lineno`** e **`end_offset`**.

exception IndentationError

Classe base para erros de sintaxe relacionados a indentação incorreta. Esta é uma subclasse de **`SyntaxError`**.

exception TabError

Levantada quando o indentação contém um uso inconsistente de tabulações e espaços. Esta é uma subclasse de **`IndentationError`**.

exception SystemError

Levantada quando o interpretador encontra um erro interno, mas a situação não parece tão grave para fazer com que perca todas as esperanças. O valor associado é uma string que indica o que deu errado (em termos de baixo nível). Em CPython, isso pode ser causado pelo uso incorreto da API C do Python, como retornar um valor `NULL` sem um conjunto de exceções.

Se você tem certeza de que essa exceção não foi culpa sua ou de um pacote que você está usando, você deve relatar isso ao autor ou mantenedor do seu interpretador Python. Certifique-se de relatar a versão do interpretador Python (`sys.version`; também é impresso no início de uma sessão Python interativa), a mensagem de erro exata (o valor associado da exceção) e se possível a fonte do programa que acionou o erro.

exception SystemExit

Esta exceção é levantada pela função **`sys.exit()`**. Ele herda de **`BaseException`** em vez de **`Exception`** para que não seja acidentalmente capturado pelo código que captura **`Exception`**. Isso permite que a exceção se propague corretamente e faça com que o interpretador saia. Quando não é tratado, o interpretador Python sai; nenhum traceback (situação da pilha de execução) é impresso. O construtor aceita o mesmo argumento opcional passado para **`sys.exit()`**. Se o valor for um inteiro, ele especifica o status de saída do sistema (passado para a função C **`exit()`**); se for `None`, o status de saída é zero; se tiver outro tipo (como uma string), o valor do objeto é exibido e o status de saída é um.

Uma chamada para **`sys.exit()`** é traduzida em uma exceção para que os tratadores de limpeza (cláusulas **`finally`** das instruções **`try`**) possam ser executados, e para que um depurador possa executar um script sem correr o risco de perder o controle. A função **`os. exit()`** pode ser usada

se for absolutamente necessário sair imediatamente (por exemplo, no processo filho após uma chamada para `os.fork()`).

code

O status de saída ou mensagem de erro transmitida ao construtor. (O padrão é `None`.)

exception TypeError

Levantada quando uma operação ou função é aplicada a um objeto de tipo inadequado. O valor associado é uma string que fornece detalhes sobre a incompatibilidade de tipo.

Essa exceção pode ser levantada pelo código do usuário para indicar que uma tentativa de operação em um objeto não é suportada e não deveria ser. Se um objeto deve ter suporte a uma dada operação, mas ainda não forneceu uma implementação, `NotImplementedError` é a exceção apropriada a ser levantada.

Passar argumentos do tipo errado (por exemplo, passar uma `list` quando um `int` é esperado) deve resultar em uma `TypeError`, mas passar argumentos com o valor errado (por exemplo, um número fora dos limites esperados) deve resultar em uma `ValueError`.

exception UnboundLocalError

Levantada quando uma referência é feita a uma variável local em uma função ou método, mas nenhum valor foi vinculado a essa variável. Esta é uma subclasse de `NameError`.

exception UnicodeError

Levantada quando ocorre um erro de codificação ou decodificação relacionado ao Unicode. É uma subclasse de `ValueError`.

`UnicodeError` possui atributos que descrevem o erro de codificação ou decodificação. Por exemplo, `err.object[err.start:err.end]` fornece a entrada inválida específica na qual o codec falhou.

encoding

O nome da codificação que levantou o erro.

reason
Uma string que descreve o erro de codec específico.

object
O objeto que o codec estava tentando codificar ou decodificar.

start
O primeiro índice de dados inválidos em `object`.

Este valor não deve ser negativo, pois é interpretado como um deslocamento absoluto, mas essa restrição não é aplicada em tempo de execução.

end
O índice após os últimos dados inválidos em `object`.

Este valor não deve ser negativo, pois é interpretado como um deslocamento absoluto, mas essa restrição não é aplicada em tempo de execução.

exception UnicodeEncodeError
Levantada quando ocorre um erro relacionado ao Unicode durante a codificação. É uma subclasse de `UnicodeError`.

exception UnicodeDecodeError
Levantada quando ocorre um erro relacionado ao Unicode durante a decodificação. É uma subclasse de `UnicodeError`.

exception UnicodeTranslateError
Levantada quando ocorre um erro relacionado ao Unicode durante a tradução. É uma subclasse de `UnicodeError`.

exception ValueError
Levantada quando uma operação ou função recebe um argumento que tem o tipo certo, mas um valor inadequado, e a situação não é descrita

por uma exceção mais precisa, como `IndexError`.

exception ZeroDivisionError

Levantada quando o segundo argumento de uma divisão ou operação de módulo é zero. O valor associado é uma string que indica o tipo dos operandos e a operação.

As seguintes exceções são mantidas para compatibilidade com versões anteriores; a partir do Python 3.3, eles são apelidos de `OSError`.

exception EnvironmentError

exception IOError

exception WindowsError

Disponível apenas no Windows.

Exceções de sistema operacional

As seguintes exceções são subclasses de `OSError`, elas são levantadas dependendo do código de erro do sistema.

exception BlockingIOError

Levantada quando uma operação bloquearia em um objeto (por exemplo, um soquete) definido para operação sem bloqueio. Corresponde a **`errno`** `EAGAIN`, `EALREADY`, `EWOULDBLOCK` e `EINPROGRESS`.

Além daquelas de `OSError`, `BlockingIOError` pode ter mais um atributo:

characters_written

Um inteiro contendo o número de **bytes** gravados no fluxo antes de ser bloqueado. Este atributo está disponível ao usar as classes de E/S em buffer do módulo `io`.

exception ChildProcessError

Levantada quando uma operação em um processo filho falha. Corresponde a **errno** **ECHILD**.

exception ConnectionError

Uma classe base para problemas relacionados à conexão.

Suas subclasses são **BrokenPipeError**, **ConnectionAbortedError**, **ConnectionRefusedError** e **ConnectionResetError**.

exception BrokenPipeError

Uma subclasse de **ConnectionError**, levantada ao tentar escrever em um encadeamento, enquanto a outra extremidade foi fechada, ou ao tentar escrever em um soquete que foi desligado para escrita. Corresponde a **errno** **EPIPE** e **ESHUTDOWN**.

exception ConnectionAbortedError

Uma subclasse de **ConnectionError**, levantada quando uma tentativa de conexão é cancelada pelo par. Corresponde a **errno** **ECONNABORTED**.

exception ConnectionRefusedError

Uma subclasse de **ConnectionError**, levantada quando uma tentativa de conexão é recusada pelo par. Corresponde a **errno** **ECONNREFUSED**.

exception ConnectionResetError

Uma subclasse de **ConnectionError**, levantada quando uma conexão é redefinida pelo par. Corresponde a **errno** **ECONNRESET**.

exception FileExistsError

Levantada ao tentar criar um arquivo ou diretório que já existe. Corresponde a **errno** **EEXIST**.

exception FileNotFoundError

Levantada quando um arquivo ou diretório é solicitado, mas não existe. Corresponde a **errno** **ENOENT**.

exception InterruptedError

Levantada quando uma chamada do sistema é interrompida por um sinal de entrada. Corresponde a **errno** **EINTR**.

Alterado na versão 3.5: Python agora tenta novamente as chamadas de sistema quando uma delas é interrompida por um sinal, exceto se o tratador de sinal levantar uma exceção (veja **PEP 475** [https://peps.python.org/pep-0475/] para a justificativa), em vez de levantar **InterruptedError**.

exception IsADirectoryError

Levantada quando uma operação de arquivo (como **os.remove()**) é solicitada em um diretório. Corresponde a **errno** **EISDIR**.

exception NotADirectoryError

Levantada quando uma operação de diretório (como **os.listdir()**) é solicitada em algo que não é um diretório. Na maioria das plataformas POSIX, ela também pode ser levantada se uma operação tentar abrir ou percorrer um arquivo não pertencente ao diretório como se fosse um diretório. Corresponde a **errno** **ENOTDIR**.

exception PermissionError

Levantada ao tentar executar uma operação sem os direitos de acesso adequados - por exemplo, permissões do sistema de arquivos. Corresponde a **errno** **EACCES**, **EPERM**, e **ENOTCAPABLE**.

Alterado na versão 3.11.1: **ENOTCAPABLE** do WASI agora é mapeado para **PermissionError**.

exception ProcessLookupError

Levantada quando um determinado processo não existe. Corresponde a **errno** **ESRCH**.

exception TimeoutError

Levantada quando uma função do sistema expirou no nível do sistema. Corresponde a **errno** **ETIMEDOUT**.

Adicionado na versão 3.3: Todas as subclasses de `OSError` acima foram adicionadas.

Ver também

PEP 3151 [https://peps.python.org/pep-3151/] - Reorganiza a hierarquia de exceções de SO e E/S

Avisos

As seguintes exceções são usadas como categorias de aviso; veja a documentação de Categorias de avisos para mais detalhes.

exception Warning

Classe base para categorias de aviso.

exception UserWarning

Classe base para avisos gerados pelo código do usuário.

exception DeprecationWarning

Classe base para avisos sobre recursos descontinuados destinados a outros desenvolvedores Python.

Ignorado pelos filtros de aviso padrão, exceto no módulo `__main__` (**PEP 565** [https://peps.python.org/pep-0565/]). Habilitar o Modo de Desenvolvimento do Python exibe este aviso.

A política de descontinuação está descrita na **PEP 387** [https://peps.python.org/pep-0387/].

exception PendingDeprecationWarning

Classe base para avisos sobre recursos obsoletos que devem ser descontinuados no futuro, mas ainda não foram.

Esta classe raramente é usada para emitir um aviso de possível descontinuação futura, é incomum, e `DeprecationWarning` é preferível

para descontinuações já ativas.

Ignorado pelos filtros de aviso padrão. Habilitar o Modo de Desenvolvimento do Python exibe este aviso.

A política de descontinuação está descrita na **PEP 387** [https://peps.python.org/pep-0387/].

exception SyntaxWarning

Classe base para avisos sobre sintaxe duvidosa.

Este aviso é tipicamente emitido ao compilar código-fonte Python e, normalmente, não será exibido ao executar código já compilado.

exception RuntimeWarning

Classe base para avisos sobre comportamento duvidoso de tempo de execução.

exception FutureWarning

Classe base para avisos sobre recursos descontinuados destinados a usuários finais de aplicações escritas em Python.

exception ImportWarning

Classe base para avisos sobre prováveis erros na importação de módulos.

Ignorado pelos filtros de aviso padrão. Habilitar o Modo de Desenvolvimento do Python exibe este aviso.

exception UnicodeWarning

Classe base para avisos relacionados a Unicode.

exception EncodingWarning

Classe base para avisos relacionados a codificações.

Veja Ativar EncodingWarning para detalhes.

Adicionado na versão 3.10.

exception BytesWarning

Classe base para avisos relacionados a **bytes** e **bytearray**.

exception ResourceWarning

Classe base para avisos relacionados ao uso de recursos.

Ignorado pelos filtros de aviso padrão. Habilitar o Modo de Desenvolvimento do Python exibe este aviso.

Adicionado na versão 3.2.

Grupos de exceções

Os itens a seguir são usados quando é necessário levantar várias exceções não relacionadas. Eles fazem parte da hierarquia de exceção, portanto, podem ser tratados com **except** como todas as outras exceções. Além disso, eles são reconhecidos por **except***, que corresponde aos seus subgrupos com base nos tipos de exceções contidas.

exception ExceptionGroup(*msg*, *excs*)

exception BaseExceptionGroup(*msg*, *excs*)

Ambos os tipos de exceção agrupam as exceções na sequência `excs`. O parâmetro `msg` deve ser uma string. A diferença entre as duas classes é que **BaseExceptionGroup** estende **BaseException** e pode envolver qualquer exceção, enquanto **ExceptionGroup** estende **Exception** e pode agrupar apenas subclasses de **Exception**. Este design é para que `except Exception` capture um **ExceptionGroup** mas não **BaseExceptionGroup**.

O construtor de **BaseExceptionGroup** retorna uma **ExceptionGroup** ao invés de uma **BaseExceptionGroup** se todas as exceções contidas forem instâncias de **Exception**, então ela pode ser usada para tornar a seleção automática. O construtor de **ExceptionGroup**, por outro lado, levanta **TypeError** se qualquer exceção contida não for uma subclasse de **Exception**.

Grupos de exceções são genéricos em relação ao tipo das exceções que contêm.

O parâmetro `excs` pode ser qualquer sequência, mas as listas e as tuplas são processadas aqui de forma mais eficiente. Para otimizar o desempenho, passe uma tupla como `excs`.

message

O argumento `msg` para o construtor. Este é um atributo somente leitura.

exceptions

Uma tupla de exceções na sequência `excs` passada ao construtor. Este é um atributo somente leitura.

subgroup(*condition*)

Retorna um grupo de exceções que contém apenas as exceções do grupo atual que correspondam à condição *condition*, ou `None` se o resultado estiver vazio.

A condição pode ser um tipo de exceção ou uma tupla de tipos de exceção. Nesse caso, cada exceção é verificada pela mesma regra usada em uma cláusula de `except`. A condição também pode ser um chamável (diferente de um objeto de tipo) que aceita uma exceção como único argumento e retorna verdadeiro se a exceção estiver no subgrupo.

A estrutura de aninhamento da exceção atual é preservada no resultado, assim como os valores de seus campos **`message`**, **`__traceback__`**, **`__cause__`**, **`__context__`** e **`__notes__`**. Grupos aninhados vazios são omitidos do resultado.

A condição é verificada para todas as exceções no grupo de exceções aninhadas, incluindo o nível superior e quaisquer grupos de exceções aninhadas. Se a condição for verdadeira para esse grupo de exceções, ela será incluída no resultado por completo.

Adicionado na versão 3.13: `condition` pode ser qualquer chamável que não seja um objeto de tipo.

split(*condition*)

Como **subgroup()**, mas retorna o par `(match, rest)` em que `match` é `subgroup(condition)` e `rest` é a parte restante não correspondente.

derive(*excs*)

Retorna um grupo de exceções com o mesmo **message**, mas que agrupa as exceções em `excs`.

Este método é usado por **subgroup()** e **split()**, que são usados em vários contextos para dividir um grupo de exceções. Uma subclasse precisa substituí-la para fazer com que **subgroup()** e **split()** retornem instâncias da subclasse em vez de **ExceptionGroup**.

subgroup() e **split()** copiam os campos **__traceback__**, **__cause__**, **__context__** e **__notes__** do grupo de exceções original para o retornado por **derive()**, então esses campos não precisam ser atualizados por **derive()**.

```
>>> class MyGroup(ExceptionGroup):
...     def derive(self, excs):
...         return MyGroup(self.message, excs)
...
>>> e = MyGroup("eg", [ValueError(1), TypeError(2)])
>>> e.add_note("a note")
>>> e.__context__ = Exception("context")
>>> e.__cause__ = Exception("cause")
>>> try:
...    raise e
... except Exception as e:
...    exc = e
...
>>> match, rest = exc.split(ValueError)
>>> exc, exc.__context__, exc.__cause__, exc.__notes__
(MyGroup('eg', [ValueError(1), TypeError(2)]), Exception(
>>> match, match.__context__, match.__cause__, match.__no
```

```
(MyGroup('eg', [ValueError(1)]), Exception('context'), Exc
>>> rest, rest.__context__, rest.__cause__, rest.__notes__
(MyGroup('eg', [TypeError(2)]), Exception('context'), Exc
>>> exc.__traceback__ is match.__traceback__ is rest.__tra
True
```

Observe que `BaseExceptionGroup` define `__new__()`, então subclasses que precisam de uma assinatura de construtor diferente precisam substituir isso em vez de `__init__()`. Por exemplo, a seguir definimos uma subclasse de grupo de exceções que aceita um exit_code e constrói a mensagem do grupo a partir dele.

```
class Errors(ExceptionGroup):
   def __new__(cls, errors, exit_code):
      self = super().__new__(Errors, f"exit code: {exit_code}
      self.exit_code = exit_code
      return self

   def derive(self, excs):
      return Errors(excs, self.exit_code)
```

Como `ExceptionGroup`, qualquer subclasse de `BaseExceptionGroup` que também seja uma subclasse de `Exception` só pode agrupar instâncias de **`Exception`**.

Adicionado na versão 3.11.

Hierarquia das exceções

A hierarquia de classes para exceções embutidas é:

```
BaseException
 ├── BaseExceptionGroup
 ├── GeneratorExit
 ├── KeyboardInterrupt
 ├── SystemExit
 └── Exception
      ├── ArithmeticError
      │    ├── FloatingPointError
```

```
│     ├── OverflowError
│     └── ZeroDivisionError
├── AssertionError
├── AttributeError
├── BufferError
├── EOFError
├── ExceptionGroup [BaseExceptionGroup]
├── ImportError
│     └── ImportCycleError
│     └── ModuleNotFoundError
├── LookupError
│     ├── IndexError
│     └── KeyError
├── MemoryError
├── NameError
│     └── UnboundLocalError
├── OSError
│     ├── BlockingIOError
│     ├── ChildProcessError
│     ├── ConnectionError
│     │     ├── BrokenPipeError
│     │     ├── ConnectionAbortedError
│     │     ├── ConnectionRefusedError
│     │     └── ConnectionResetError
│     ├── FileExistsError
│     ├── FileNotFoundError
│     ├── InterruptedError
│     ├── IsADirectoryError
│     ├── NotADirectoryError
│     ├── PermissionError
│     ├── ProcessLookupError
│     └── TimeoutError
├── ReferenceError
├── RuntimeError
│     ├── NotImplementedError
│     ├── PythonFinalizationError
│     └── RecursionError
├── StopAsyncIteration
├── StopIteration
├── SyntaxError
│     └── IndentationError
│          └── TabError
├── SystemError
```

```
├── TypeError
├── ValueError
│    └── UnicodeError
│         ├── UnicodeDecodeError
│         ├── UnicodeEncodeError
│         └── UnicodeTranslateError
└── Warning
     ├── BytesWarning
     ├── DeprecationWarning
     ├── EncodingWarning
     ├── FutureWarning
     ├── ImportWarning
     ├── PendingDeprecationWarning
     ├── ResourceWarning
     ├── RuntimeWarning
     ├── SyntaxWarning
     ├── UnicodeWarning
     └── UserWarning
```

Garantias de segurança de thread

Esta página documenta as garantias de segurança de thread para tipos embutido na construção com threads livres do Python. As garantias aqui descritas aplicam-se ao utilizar o Python com a GIL desativada (modo threads livres). Quando a GIL está ativada, a maioria das operações é serializada implicitamente.

Para orientações gerais sobre como escrever código seguro para thread no Python com threads livres, consulte Suporte do Python para threads livres.

Níveis de segurança de threads

A documentação da API C usa os seguintes níveis para descrever as garantias de segurança de threads de cada função. Os níveis estão listados do menos para o mais seguro.

Incompatível

Uma função ou operação que não pode se tornar segura para uso concorrente mesmo com sincronização externa. Código incompatível normalmente acessa o estado global de maneira não sincronizada e deve ser chamado apenas a partir de uma única thread durante todo o tempo de vida do programa.

Exemplo: uma função que modifica o estado em todo o processo, como manipuladores de sinais ou variáveis de ambiente, onde chamadas concorrentes de quaisquer threads, mesmo com trava externo, podem conflitar com o ambiente de execução (*runtime*) ou com outras bibliotecas.

Compatível

Uma função ou operação que é segura para ser chamada a partir de múltiplas threads *desde que* o chamador forneça a sincronização externa

apropriada, por exemplo mantendo uma trava durante a duração de cada chamada. Sem tal sincronização, chamadas concorrentes podem produzir condições de corrida ou corridas de dados.

Exemplo: uma função que lê ou escreve em um objeto cujo estado interno não é protegido por uma trava. Os chamadores devem garantir que duas threads não acessem o mesmo objeto ao mesmo tempo.

Seguro em objetos distintos

Uma função ou operação que é segura para ser chamada a partir de múltiplas threads sem sincronização externa, desde que cada thread opere em um objeto **diferente**. Duas threads podem chamar a função ao mesmo tempo, mas não devem passar o mesmo objeto (ou objetos que compartilham estado subjacente) como argumentos.

Exemplo: uma função que modifica campos de uma struct usando escritas não atômicas. Duas threads podem chamar a função cada uma em sua própria instância de struct com segurança, mas chamadas concorrentes na *mesma* instância exigem sincronização externa.

Seguro em objetos compartilhados

Uma função ou operação que é segura para uso concorrente no **mesmo** objeto. A implementação usa sincronização interna (como travas por objeto ou seções críticas) para proteger o estado mutável compartilhado, de modo que os chamadores não precisam fornecer sua própria trava.

Exemplo: `PyList_GetItemRef()` pode ser chamada a partir de múltiplas threads no mesmo `PyListObject` - ela usa sincronização interna para serializar o acesso.

Atômica

Uma função ou operação que parece atômica em relação a outras threads - ela executa instantaneamente da perspectiva de outras threads. Esta é a forma mais forte de segurança de threads.

Exemplo: `PyMutex_IsLocked()` realiza uma leitura atômica do estado do mutex e pode ser chamada a partir de qualquer thread a qualquer momento.

Segurança de threads para objetos lista

Ler um único elemento de uma `list` é atômico:

```
lst[i]   # list.__getitem__
```

Os seguintes métodos percorrem a lista e usam leituras atômicas de cada item para realizar sua função. Isso significa que eles podem retornar resultados afetados por modificações concorrentes:

```
item in lst
lst.index(item)
lst.count(item)
```

Todas as operações acima evitam adquirir travas por objeto. Elas não travam modificações concorrentes. Outras operações que mantêm uma trava não impedirão que estas observem estados intermediários.

Todas as outras operações a partir daqui bloqueiam usando o trava por objeto.

Escrever um único item via `lst[i] = x` é seguro para chamar a partir de múltiplas threads e não corromperá a lista.

As seguintes operações retornam novos objetos e parecem atômicas para outras threads:

```
lst1 + lst2    # concatena duas listas em uma nova lista
x * lst        # repete lst x vezes em uma nova lista
lst.copy()     # retorna uma cópia rasa da lista
```

Os seguintes métodos que operam apenas em um único elemento sem necessidade de deslocamento são atômicos:

```
lst.append(x)  # acrescenta ao final da lista; não é necessário
lst.pop()      # remove o elemento do final da lista; não é nece
```

O método `clear()` também é atômico. Outras threads não podem observar elementos sendo removidos.

O método `sort()` não é atômico. Outras threads não podem observar estados intermediários durante a ordenação, mas a lista parece vazia durante a execução da ordenação.

As seguintes operações podem permitir que operações livres de travas observem estados intermediários, pois modificam múltiplos elementos no local (in-place):

```
lst.insert(idx, item)  # desloca elementos
lst.pop(idx)           # idx não no fim da lista, desloca elemer
lst *= x               # cópia elementos no local
```

O método `remove()` pode permitir modificações concorrentes, pois a comparação de elementos pode executar código Python arbitrário (via `__eq__()`).

`extend()` é seguro para chamar a partir de múltiplas threads. No entanto, suas garantias dependem do iterável passado para ele. Se for uma `list`, uma `tuple`, um `set`, um `frozenset`, um `dict` ou um objeto de visão de dicionário (mas não suas subclasses), a operação `extend` estará segura de modificações concorrentes no iterável. Caso contrário, um iterador é criado, o qual pode ser modificado concorrentemente por outra thread. O mesmo se aplica à concatenação no local de uma lista com outros iteráveis ao usar `lst += iterable`.

De forma semelhante, a atribuição a uma fatia de lista com `lst[i:j] = iterable` é segura para ser chamada a partir de múltiplas threads, mas `iterable` só é travado quando também for uma `list` (mas não suas subclasses).

Operações que envolvem múltiplos acessos, assim como iteração, nunca são atômicas. Por exemplo:

```
# NÃO é atômico: lê-modifica-escreve
lst[i] = lst[i] + 1

# NÃO é atômico: verifica-e-então-age
if lst:
    item = lst.pop()

# NÃO é seguro para thread: iteração enquanto modifica
for item in lst:
    process(item)  # outra thread pode modificar lst
```

Considere a sincronização externa ao compartilhar instâncias de **`list`** entre threads.

Segurança de threads para objetos dict

Criar um dicionário com o construtor **`dict`** é atômico quando o argumento passado for um **`dict`** ou uma **`tuple`**. Ao usar o método **`dict.fromkeys()`**, a criação do dicionário é atômica quando o argumento for um **`dict`**, **`tuple`**, **`set`** ou **`frozenset`**.

As seguintes operações e funções são livres de travas e atômicas.

```
d[key]       # dict.__getitem__
d.get(key)   # dict.get
key in d     # dict.__contains__
len(d)       # dict.__len__
```

Todas as outras operações a partir daqui mantêm a trava por objeto.

Escrever ou remover um único item é seguro para chamar a partir de múltiplas threads e não corromperá o dicionário:

```
d[key] = value        # escreve
del d[key]            # exclui
d.pop(key)            # remove e retorna
d.popitem()           # remove e retorna último item
d.setdefault(key, v)  # insere se não existir
```

Essas operações podem comparar chaves usando `__eq__()`, o que pode executar código Python arbitrário. Durante tais comparações, o dicionário pode ser modificado por outra thread. Para tipos embutidos como `str`, `int` e `float`, que implementam `__eq__()` em C, o trava subjacente não é liberado durante as comparações e isso não é uma preocupação.

As seguintes operações retornam novos objetos e mantêm a trava por objeto durante a duração da operação:

```
d.copy()      # retorna uma cópia rasa do dicionário
d | other     # combina dois dicionários em um novo dicionário
d.keys()      # retorna um novo objeto de visão dict_keys
d.values()    # retorna um novo objeto de visão dict_values
d.items()     # retorna um novo objeto de visão dict_items
```

O método `clear()` mantém a trava durante sua execução. Outras threads não podem observar elementos sendo removidos.

As seguintes operações travariam ambos os dicionários. Para `update()` e `|=`, isso se aplica apenas quando o outro operando for um `dict` que usa o iterador de dicionário padrão (mas não subclasses que substituem a iteração). Para comparação de igualdade, isso se aplica a `dict` e suas subclasses:

```
d.update(outro_dict)  # ambos travados quando outro_dict é um di
d |= outro_dict       # ambos travados quando outro_dict é um di
d == outro_dict       # ambos travados para dict e subclasses
```

Todas as operações de comparação também comparam valores usando `__eq__()`, portanto, para tipos não embutidos, a trava pode ser liberada durante a comparação.

`fromkeys()` trava tanto o novo dicionário quanto o iterável quando o iterável for exatamente um `dict`, `set` ou `frozenset` (não subclasses):

```
dict.fromkeys(a_dict)      # trava ambas
dict.fromkeys(a_set)       # trava ambas
dict.fromkeys(a_frozenset) # trava ambas
```

Ao atualizar a partir de um iterável que não é dicionário, apenas o dicionário de destino é travada. O iterável pode ser modificado concorrentemente por outra thread:

```
d.update(iterável)        # iterável não é um dict: somente d tr
d |= iterável             # iterável não é um dict: somente d tr
dict.fromkeys(iterável)   # iterável não é um dict/conjunto/froz
```

Operações que envolvem múltiplos acessos, assim como iteração, nunca são atômicas:

```
# NÃO é atômico: lê-modifica-escreve
d[key] = d[key] + 1

# NÃO é atômico: verifica-e-então-age (TOCTOU)
if key in d:
    del d[key]

# NÃO é seguro para thread: iteração enquanto modifica
for key, value in d.items():
    process(key)  # outa thread pode modificar d
```

Para evitar problemas de tempo de verificação para tempo de uso (TOCTOU, do inglês *time-of-check to time-of-us*), use operações atômicas ou trata exceções:

```
# Usa pop() com um valor padrão em vez de verificar-e-então-excl
d.pop(key, None)

# ou trata a exceção
try:
    del d[key]
except KeyError:
    pass
```

Para iterar com segurança sobre um dicionário que pode ser modificado por outra thread, itere sobre uma cópia:

```
# Faz uma cópia para iterar com segurança
for key, value in d.copy().items():
```

```
    process(key)
```

Considere a sincronização externa ao compartilhar instâncias de `dict` entre threads.

Segurança de thread para objetos conjunto

A função `len()` é livre de trava e atômica.

A seguinte operação de leitura é livre de trava. Ela não bloqueia modificações concorrentes e pode observar estados intermediários de operações que mantêm a trava por objeto:

```
elem in s    # set.__contains__
```

Esta operação pode comparar elementos usando `__eq__()`, o que pode executar código Python arbitrário. Durante tais comparações, o conjunto pode ser modificado por outra thread. Para tipos embutidos como `str`, `int` e `float`, `__eq__()` não libera a trava subjacente durante as comparações e isso não é uma preocupação.

Todas as outras operações a partir daqui mantêm a trava por objeto.

Adicionar ou remover um único elemento é seguro para chamar a partir de múltiplas threads e não corromperá o conjunto:

```
s.add(elem)      # adiciona elemento
s.remove(elem)   # remove elemento, levanta se estiver faltando
s.discard(elem)  # remove elemento se estiver presente
s.pop()          # remove e retorna elemento arbitrário
```

Essas operações também comparam elementos; portanto, aplicam-se as mesmas considerações sobre `__eq__()` mencionadas acima.

O método `copy()` retorna um novo objeto e mantém a trava por objeto durante toda a execução para que seja sempre atômico.

O método `clear()` mantém a trava durante sua execução. Outras threads não podem observar elementos sendo removidos.

As seguintes operações aceitam apenas `set` ou `frozenset` como operandos e sempre travam ambos os objetos:

```
s |= outro                    # outro deve ser conjunto/frozenset
s &= outro                    # outro deve ser conjunto/frozenset
s -= outro                    # outro deve ser conjunto/frozenset
s ^= outro                    # outro deve ser conjunto/frozenset
s & outro                     # outro deve ser conjunto/frozenset
s | outro                     # outro deve ser conjunto/frozenset
s - outro                     # outro deve ser conjunto/frozenset
s ^ outro                     # outro deve ser conjunto/frozenset
```

`set.update()`, `set.union()`, `set.intersection()` e `set.difference()` podem receber múltiplos iteráveis como argumentos. Todos eles iteram sobre todos os iteráveis fornecidos e realizam o seguinte:

- `set.update()` e `set.union()` travam objetos apenas quando o outro operando é um `set`, `frozenset`, ou `dict`.
- `set.intersection()` e `set.difference()` sempre tentam travar todos os objetos.

`set.symmetric_difference()` tenta travar ambos objetos.

As variantes de atualização dos métodos acima também apresentam algumas diferenças entre si:

- `set.difference_update()` e `set.intersection_update()` tentam travar todos os objetos um por um.
- `set.symmetric_difference_update()` só trava os argumentos se forem do tipo `set`, `frozenset` ou `dict`.

Os métodos a seguir sempre tentam travar ambos objetos:

```
s.isdisjoint(outro)           # ambos travados
s.issubset(outro)             # ambos travados
s.issuperset(outro)           # ambos travados
```

Operações que envolvem múltiplos acessos, assim como iteração, nunca são atômicas:

```
# NÃO é atômico: verifica-e-então-age
if elem in s:
      s.remove(elem)

# NÃO é seguro para thread: iteração enquanto modifica
for elem in s:
      process(elem)  # outra thread pode modificar s
```

Considere a sincronização externa ao compartilhar instâncias de `set` entre threads. Consulte Suporte do Python para threads livres para mais informações.

Segurança para thread para objetos bytearray

A função `len()` é livre de trava e atômica.

A concatenação e as comparações utilizam o protocolo de buffer, o qual impede o redimensionamento, mas não mantém a trava por objeto. Essas operações podem observar estados intermediários resultantes de modificações concorrentes:

```
ba + outro    # pode observar escritas simultâneas
ba == outro   # pode observar escritas simultâneas
ba < outro    # pode observar escritas simultâneas
```

Todas as outras operações a partir daqui mantêm a trava por objeto.

A leitura de um único elemento ou de uma fatia é segura para ser realizada a partir de múltiplas threads:

```
ba[i]        # bytearray.__getitem__
ba[i:j]      # fatia
```

As seguintes operações são seguras para serem chamadas a partir de múltiplas threads e não corromperão o bytearray:

```
ba[i] = x         # escreve um único byte
ba[i:j] = valores # escreve uma fatia
ba.append(x)      # junta um único byte ao final
ba.extend(outro)  # estende com iterável
```

```
ba.insert(i, x)   # insere um único byte
ba.pop()          # remove e retorna o último byte
ba.pop(i)         # remove e retorna o byte no índice
ba.remove(x)      # remove a primeira ocorrência
ba.reverse()      # inverte no local
ba.clear()        # remove todos os bytes
```

A atribuição de fatia TRAVA ambos objetos quando *valores* é um **bytearray**:

```
ba[i:j] = other_bytearray  # ambos travados
```

As operações a seguir retornam novos objetos e mantêm a trava por objeto durante a execução:

```
ba.copy()     # retorna uma cópia rasa
ba * n        # repete em um novo bytearray
```

O teste de pertinência mantém a trava durante a sua execução:

```
x in ba       # bytearray.__contains__
```

Todos os outros métodos de bytearray (como **find()**, **replace()**, **split()**, **decode()**, etc.) mantêm a trava específica do objeto durante sua execução.

Operações que envolvem múltiplos acessos, assim como iteração, nunca são atômicas:

```
# NÃO é atômico: verifica-e-então-age
if x in ba:
    ba.remove(x)

# NÃO é seguro para thread: iteração enquanto modifica
for byte in ba:
    process(byte)  # outra thread por modificar ba
```

Para iterar com segurança sobre um bytearray que possa ser modificado por outra thread, itere sobre uma cópia:

```
# Faz uma cópia para iterar com segurança
for byte in ba.copy():
    process(byte)
```

Considere a sincronização externa ao compartilhar instâncias de **`bytearray`** entre threads. Consulte Suporte do Python para threads livres para obter mais informações.

Segurança para thread para objetos memoryview

Objetos **`memoryview`** fornecem acesso aos dados internos de um objeto subjacente sem realizar cópias. A segurança para thread depende tanto do próprio memoryview quanto do exportador de buffer subjacente.

A implementação do memoryview utiliza operações atômicas para rastrear suas próprias exportações na construção com threads livres (*free-threaded build*). A criação e a liberação de um memoryview são seguras para thread. O acesso a atributos (por exemplo, **`shape`**, **`format`**) lê campos que são imutáveis durante todo o ciclo de vida do memoryview; portanto, leituras concorrentes são seguras, desde que o memoryview não tenha sido liberado.

No entanto, os dados reais acessados por meio do memoryview pertencem ao objeto subjacente. O acesso concorrente a esses dados só é seguro se o objeto subjacente o oferecer suporte a isso:

- Para objetos imutáveis como **`bytes`**, leituras concorrentes por meio de múltiplos memoryviews são seguras.
- Para objetos mutáveis como **`bytearray`**, a leitura e a escrita na mesma região de memória a partir de múltiplas threads, sem sincronização externa, não são seguras e podem resultar em corrupção de dados. Observe que mesmo *memoryviews* somente leitura de objetos mutáveis não impedem condições de corrida se o objeto subjacente for modificado por outra thread.

```
# NÃO é seguro: escritas concorrentes ao mesmo buffer
data = bytearray(1000)
view = memoryview(data)
```

```
# Thread 1: view[0:500] = b'x' * 500
# Thread 2: view[0:500] = b'y' * 500

# Seguro: usa uma trava para acesso concorrente
import threading
lock = threading.Lock()
data = bytearray(1000)
view = memoryview(data)

with lock:
    view[0:500] = b'x' * 500
```

Redimensionar ou realocar o objeto subjacente (como ao chamar `bytearray.resize()`) enquanto um memoryview está exportado levanta uma exceção `BufferError`. Essa regra é aplicada independentemente do uso de threads.

Complexidade de tempo de operações em tipos embutidos

Esta página documenta a complexidade de tempo de várias operações em tipos nativos no CPython. Outras implementações do Python podem apresentar características de desempenho diferentes. Além disso, os custos listados pressupõem os tipos nativos exatos, uma vez que instâncias de subclasses podem ter custos diferentes.

Utilizamos a Big *O* notation [https://en.wikipedia.org/wiki/Big_O_notation] para descrever como o tempo de execução de uma operação cresce em função do tamanho de suas entradas. A menos que indicado de outra forma, *n* representa o número de elementos atualmente no contêiner, e *k* é o valor de um parâmetro numérico, como um índice ou uma contagem de repetições.

`list`

Listas são sequências mutáveis; para mais detalhes sobre a implementação, consulte Como as listas são implementadas no CPython?. Os custos mais elevados ocorrem ao expandir a lista além do tamanho de alocação atual (pois todos os elementos precisam ser movidos) ou ao inserir ou remover itens próximos ao início (pois todos os elementos subsequentes precisam ser movidos). Se você precisar adicionar ou remover itens em ambas as extremidades, considere usar um **`collections.deque`** em vez disso.

Operação	Complexidade
Cópia (`l.copy()`)	$O(n)$

Operação	Complexidade
Anexação ao final (`l.append(x)`) [[1]]	$O(1)$
Pop (`l.pop(k)`) [[1]] [[2]]	$O(n - k)$
Inserção (`l.insert(k, x)`) [[1]] [[2]]	$O(n - k)$
Obtenção de item (`l[k]`)	$O(1)$
Definição de item (`l[k] = x`)	$O(1)$
Exclusão de item (`del l[k]`) [[2]]	$O(n - k)$
Iteração	$O(n)$
Obtenção de fatia (`l[i:j]`)	$O(j - i)$
Definição de fatia (`l[i:j] = t`) [[1]]	$O(j - i)$ if $\text{len}(t) == j - i$, do contrário $O(n - i + \text{len}(t))$
Exclusão de fatia (`del l[i:j]`)	$O(n - i)$

Operação	Complexidade
Estensão (`l.extend(t)`) [[1]] [[3]]	$O(\text{len}(t))$
Ordenação (`l.sort()`) [[4]]	$O(n \log n)$
Concatenação (`l1 + l2`)	$O(\text{len}(l1) + \text{len}(l2))$
Multiplicação (`l * k`)	$O(nk)$
`x in l`	$O(n)$
`min(l)`, `max(l)`	$O(n)$
Obtenção de comprimento (`len(l)`) [[5]]	$O(1)$

`tuple`

Uma **`tuple`** é uma sequência imutável. Como uma tupla nunca pode ser alterada, não há custos de inserção ou remoção, e criar uma cópia simplesmente retorna o mesmo objeto; portanto, a operação ocorre em tempo constante ($O(1)$).

Operação	Complexidade

Operação	Complexidade
Cópia (`tuple(t)`)	$O(1)$
Obtenção de item (`t[k]`)	$O(1)$
Obtenção de fatia (`t[i:j]`)	$O(j - i)$
Concatenação (`t1 + t2`)	$O(\text{len}(t1) + \text{len}(t2))$
Multiplicação (`t * k`)	$O(nk)$
Iteração	$O(n)$
`x in t`	$O(n)$
`min(t)`, `max(t)`	$O(n)$
Obtenção de comprimento (`len(t)`) [[5]]	$O(1)$

`dict`, `frozendict`

Os tempos listados para objetos do tipo *dict* referem-se a casos médios, pois pressupõem que a função de *hash* utilizada seja suficientemente robusta para tornar incomuns as colisões. Também pressupõem que as chaves estejam bem distribuídas no conjunto de chaves possíveis. No pior caso — quando todas as chaves resultam no mesmo valor de *hash* —, cada uma das

operações $O(1)$ listadas abaixo passa a exigir tempo $O(n)$. Além disso, eles pressupõem que as operações de *hashing* e comparação de chaves ocorrem em tempo $O(1)$. Para mais detalhes sobre a implementação, consulte Como são os dicionários implementados no CPython?.

Um `frozendict` é imutável; portanto, não oferece suporte à definição, exclusão ou a atualização de itens. As outras operações listadas abaixo aplicam-se a ele com os mesmos custos.

Operação	**Complexidade**
`key in d`	$O(1)$
Cópia (`d.copy()`) [[6]] [[7]]	$O(n)$
Obtenção de item (`d[key]`, `d.get(key)`)	$O(1)$
Definição de item (`d[key] = value`) [[1]]	$O(1)$
Exclusão de item (`del d[key]`, `d.pop(key)`)	$O(1)$
Atualização (`d.update(t)`, `d |= t`) [[1]] [[3]] [[7]]	$O(\text{len}(t))$
Iteração [[7]]	$O(n)$
Obtenção de comprimento (`len(d)`) [[5]]	$O(1)$

`set`, `frozenset`

Consulte **`dict`**, pois as implementações de **`set`** e **`frozenset`** são semelhantes e as mesmas ressalvas se aplicam. No pior caso, operações de $O(1)$ passam a levar tempo $O(n)$, e operações que percorrem todos os elementos sofrem uma degradação de desempenho correspondente.

Um **`frozenset`** é imutável, portanto, não oferece suporte a operações de adição, descarte ou atualização local (*in-place*). As demais operações listadas abaixo aplicam-se a ele com os mesmos custos.

Operação	**Complexidade**
`x in s`	$O(1)$
Cópia (`s.copy()`) [[6]] [[7]]	$O(n)$
Adição (`s.add(x)`) [[1]]	$O(1)$
Descarte (`s.discard(x)`, `s.remove(x)`)	$O(1)$
União (`s1 | s2`, `s1.union(s2)`) [[7]]	$O(\text{len}(s1) + \text{len}(s2))$
Atualização (`s1 |= s2`, `s1.update(s2)`) [[1]] [[7]]	$O(\text{len}(s2))$
Interseção (`s1 & s2`, `s1.intersection(s2)`) [[7]] [[8]]	$O(\min(\text{len}(s1), \text{len}(s2)))$
Atualização de interseção (`s1 &= s2`, `s1.intersection_update(s2)`) [[1]] [[7]] [[8]]	$O(\min(\text{len}(s1), \text{len}(s2)))$

Operação	Complexidade
Diferença (`s1 - s2`, `s1.difference(s2)`) [[7]] [[9]]	*O*(len(*s1*))
Atualização de diferença (`s1 -= s2`, `s1.difference_update(s2)`) [[1]] [[7]] [[8]]	*O*(min(len(*s1*), len(*s2*)))
Diferença simétrica (`s1 ^ s2`, `s1.symmetric_difference(s2)`) [[7]]	*O*(len(*s1*) + len(*s2*))
Atualização de diferença simétrica (`s1 ^= s2`, `s1.symmetric_difference_update(s2)`) [[1]] [[7]]	*O*(len(*s2*))
Obtenção de comprimento (`len(s)`) [[5]]	*O*(1)

`str`, `bytes`, `bytearray`

Objetos **`str`** e **`bytes`** são sequências imutáveis de caracteres e bytes, respectivamente. Assim como ocorre com as tuplas, copiar um deles retorna o objeto original. Um **`bytearray`** é mutável e, além disso, oferece suporte às operações de modificação de **`list`** (exceto **`sort()`**), com os mesmos custos. No entanto, a exclusão no início utilizando `del` (`del b[0]`, `del b[:k]`) apenas avança o início do buffer, em vez de mover os bytes restantes, resultando em uma complexidade amortizada *O*(1).

Operação	Complexidade
Obtenção de item (`s[k]`)	*O*(1)

Operação	Complexidade
Obtenção de fatia (`s[i:j]`)	$O(j - i)$
Concatenação (`s + t`) [[10]]	$O(\text{len}(s) + \text{len}(t))$
Multiplicação (`s * k`)	$O(nk)$
Busca por substring (`x in s`, `s.find(x)`, `s.index(x)`) [[11]]	$O(n)$
Busca inversa por substring (`s.rfind(x)`, `s.rindex(x)`) [[11]] [[12]]	$O(n \times \text{len}(x))$
Codificação e decodificação [[13]]	$O(n)$
Iteração	$O(n)$
Obtenção de comprimento (`len(s)`) [[5]]	$O(1)$

`memoryview`

Objetos `memoryview` permitem que código Python acesse os dados internos de um objeto que implemente o protocolo buffer sem copiá-lo. Em especial, o fatiamento de uma view de memória retorna uma nova view para o mesmo buffer.

Operação	Complexidade
Criação (`memoryview(obj)`)	$O(1)$
Obtenção de item (`v[k]`)	$O(1)$
Obtenção de fatia (`v[i:j]`)	$O(1)$
Índice (`v.index(x)`) [[11]] [[14]]	$O(n)$
Contagem (`v.count(x)`) [[14]]	$O(n)$
Conversão para bytes (`v.tobytes()`, `bytes(v)`)	$O(n)$
Obtenção de comprimento (`len(v)`) [[5]]	$O(1)$

`range`

Um objeto **`range`** calcula seus itens sob demanda a partir de seus valores de *start*, *stop* e *step*; portanto, a maioria das operações não depende do comprimento do intervalo.

Operação	Complexidade
Obtenção de item (`r[k]`)	$O(1)$

Operação	Complexidade
Obtenção de fatia (`r[i:j]`)	$O(1)$
`x in r` [[15]]	$O(1)$
Índice e contagem (`r.index(x)`, `r.count(x)`) [[15]]	$O(1)$
Iteração	$O(n)$
`min(r)`, `max(r)`	$O(n)$
Obtenção de comprimento (`len(r)`) [[5]]	$O(1)$

Notas

[1] (1,2,3,4,5,6,7,8,9,10,11,12)

Amortizado. Uma operação individual pode, ocasionalmente, ser *O*(*n*) quando o armazenamento subjacente é redimensionado, mas esse custo é distribuído por muitas operações, dependendo do histórico do contêiner.

[2] (1,2,3)

Retirar (*pop*) ou excluir o elemento no índice *k* de uma lista de tamanho *n* desloca todos os elementos após *k* uma posição para a esquerda, movimentando *n* - *k* - 1 elementos; inserir no índice *k* desloca os elementos a partir de *k* uma posição para a direita, movimentando *n* - *k* elementos. O pior caso ocorre no índice 0, onde todo o restante da lista precisa ser deslocado; o caso médio — um índice no meio da lista — requer $O(n/2)$ =

$O(n)$ operações; e realizar a operação no final da lista não exige movimentação, sendo uma operação $O(1)$.

[3] (1,2)

Além do custo de iterar sobre *t*, que pode ser elevado para um iterável arbitrário.

[4]

Este é o pior cenário possível. A ordenação é adaptativa, e entradas já ordenadas ou ordenadas em ordem inversa requerem apenas $O^*(^*n)$ comparações. Consulte Objects/listsort.txt [https://github.com/python/cpython/tree/3.15/Objects/listsort.txt] para mais informações.

[5] (1,2,3,4,5,6,7)

O número de elementos é armazenado no objeto, portanto, `len()` não precisa contá-los.

[6] (1,2)

A cópia de um **`frozendict`** ou um **`frozenset`** é $O(1)$, pois retorna o objeto original.

[7] (1,2,3,4,5,6,7,8,9,10,11,12)

Essas operações percorrem a tabela de hash interna do contêiner, a qual não é reduzida quando elementos são removidos. Mesmo após a remoção da maioria dos elementos, elas continuam levando um tempo proporcional ao tamanho anterior do contêiner, até que uma inserção posterior acione um redimensionamento.

[8] (1,2,3)

$O(\text{len}(t))$ se *t* não é um conjunto.

[9]

O(len(*s*) + len(*t*)) se *t* não é um conjunto.

[10]

Cada concatenação cria um novo objeto; portanto, construir uma string concatenando muitas partes em um laço tem complexidade quadrática em relação ao comprimento total. Consulte a nota sobre a concatenação de sequências imutáveis para ver alternativas.

[11] (1,2,3)

Com os argumentos *start* e *end*, *n* é o comprimento da região pesquisada, e não de *s*, e, ao contrário do fatiamento, nada é copiado.

[12]

Este é o pior caso. As buscas inversas são *O*(*n*) para entradas típicas. Já as buscas diretas utilizam um algoritmo mais elaborado com complexidade linear no pior caso, descrito em Objects/stringlib/stringlib_find_two_way_notes.txt [https://github.com/python/cpython/tree/3.15/Objects/stringlib/stringlib_find_two_way_notes.txt].

[13]

Isso presume um codec que realiza uma quantidade constante de trabalho por caractere.

[14] (1,2)

Eles desempacotam e comparam cada elemento individualmente; portanto, são muito mais lentos do que os métodos equivalentes de `bytes`.

[15] (1,2)

Presumindo os argumentos como sendo do tipo `int` ou `bool`. Para outros tipos, o intervalo é pesquisado como qualquer outra sequência, em tempo *O*(*n*).

A biblioteca padrão do Python

Este manual de referência de bibliotecas descreve a biblioteca padrão que é distribuída com o Python. Ele também descreve alguns dos componentes opcionais que são comumente incluídos nas distribuições do Python.

Em outra parte, A Referência da Linguagem Python descreve a sintaxe e a semântica exatas da linguagem Python, e Referência da bibliotecas embutidas do Python descreve as funções embutidas.

A biblioteca padrão do Python é extensa, oferecendo uma ampla gama de recursos, conforme indicado pelo longo índice listado abaixo. A biblioteca contém módulos embutidos (escritos em C) que fornecem acesso à funcionalidade do sistema, como E/S de arquivos que de outra forma seriam inacessíveis para programadores Python, bem como módulos escritos em Python que fornecem soluções padronizadas para muitos problemas que ocorrem em programação cotidiana. Alguns desses módulos são explicitamente projetados para incentivar e aprimorar a portabilidade de programas em Python, abstraindo os detalhes da plataforma em APIs neutras em plataforma.

Os instaladores do Python para a plataforma Windows geralmente incluem toda a biblioteca padrão e muitas vezes também incluem muitos componentes adicionais. Para sistemas operacionais semelhantes a Unix, o Python é normalmente fornecido como uma coleção de pacotes, portanto, pode ser necessário usar as ferramentas de empacotamento fornecidas com o sistema operacional para obter alguns ou todos os componentes opcionais.

Além da biblioteca padrão, há uma coleção ativa de centenas de milhares de componentes (de programas e módulos individuais a pacotes e frameworks de desenvolvimento de aplicativos inteiros), disponíveis no Python Package Index [https://pypi.org].

- Introdução
  - Observações sobre disponibilidade

- Serviços de Processamento de Texto
  - `string` — Operações comuns de strings
  - `string.templatelib` — Suporte para literais de string template
  - `re` — Operações com expressões regulares
  - `difflib` — Helpers for computing deltas
  - `textwrap` — Quebra automática e preenchimento de texto
  - `unicodedata` — Banco de dados de Unicode
  - `stringprep` — Preparação de string para internet
  - `readline` — Interface para o GNU readline
  - `rlcompleter` — Função de autocomplemento para GNU readline
- Serviços de Dados Binários
  - `struct` — Interpreta bytes como dados binários empacotados
  - `codecs` — Codec registry and base classes
- Tipos de Dados
  - `datetime` — Tipos básicos de data e hora
  - `zoneinfo` — Suporte a fuso horário da IANA
  - `calendar` — Funções gerais relacionadas ao calendário
  - `collections` — Tipos de dados de contêineres
  - `collections.abc` — Classes Base Abstratas para Contêineres
  - `heapq` — Algoritmo de fila heap
  - `bisect` — Algoritmo de bisseção de vetor
  - `array`— Vetores eficientes de valores numéricos
  - `weakref` — Referências fracas
  - `types` — Criação de tipos dinâmicos e nomes para tipos embutidos
  - `copy` — Operações de cópia profunda e cópia rasa
  - `pprint` — Impressão bonita de dados
  - `reprlib` — Implementação alternativa à `repr()`
  - `enum` — Suporte a enumerações
  - `graphlib` — Funcionalidade para operar com estruturas do tipo grafo
- Módulos Matemáticos e Numéricos
  - `numbers` — Classes base abstratas numéricas
  - `math` — Funções matemáticas
  - `math.integer` — integer-specific mathematics functions
  - `cmath` — Funções matemáticas para números complexos
  - `decimal` — Aritmética de ponto fixo decimal e ponto flutuante

    - `fractions` — Números racionais
    - `random` — Gera números pseudoaleatórios
    - `statistics` — Funções estatísticas
- Módulos de Programação Funcional
    - `itertools` — Funções que criam iteradores para laços eficientes
    - `functools` — Funções e operações de ordem superior em objetos chamáveis
    - `operator` — Operadores padrões como funções
- Acesso a arquivos e diretórios
    - `pathlib` — Caminhos do sistema de arquivos orientados a objetos
    - `os.path` — Manipulações comuns de nomes de caminhos
    - `stat` — Interpretando resultados de `stat()`
    - `filecmp` — Comparações de arquivos e diretórios
    - `tempfile` — Gera arquivos temporários e diretórios
    - `glob` — Expansão de padrão de nome de arquivo no estilo Unix
    - `fnmatch` — Correspondência de padrões de nome de arquivo Unix
    - `linecache` — Acesso aleatório a linhas de texto
    - `shutil` — Operações de arquivo de alto nível
- Persistência de Dados
    - `pickle` — Serialização de objetos Python
    - `copyreg` — Registra funções de suporte `pickle`
    - `shelve` — Python object persistence
    - `marshal` — Serialização interna de objetos Python
    - `dbm` — Interfaces para “banco de dados” Unix
    - `sqlite3` —- interface DB-API 2.0 interface para bancos de dados SQLite
- Compressão de Dados e Arquivamento
    - O pacote `compression`
    - `compression.zstd` — Compactação compatível com o formato Zstandard
    - `zlib` — Compactação compatível com **gzip**
    - `gzip` — Suporte para arquivos **gzip**
    - `bz2` — Suporte para compressão **bzip2**
    - `lzma` — Compressão usando o algoritmo LZMA
    - `zipfile` — Trabalha com arquivos ZIP

    - `tarfile` — Ler e gravar arquivos do tipo tar
- Formatos de Arquivo
    - `csv` — Leitura e escrita de arquivos CSV
    - `configparser` — Analisador sintático de arquivo de configuração
    - `tomllib` — Analisa arquivos TOML
    - `netrc` — Arquivo de processamento netrc
    - `plistlib` — Gera e analisa arquivos Apple `.plist`
- Serviços Criptográficos
    - `hashlib` — Resumos de mensagens e hashes seguros
    - `hmac` — Código de autenticação de mensagem com chave hash
    - `secrets` — Gera números aleatórios seguros para gerenciar segredos
- Serviços Genéricos do Sistema Operacional
    - `os` — Diversas interfaces de sistema operacional
    - `io` — Ferramentas essenciais para trabalhar com fluxos
    - `time` — Acesso e conversões de tempo
    - `logging` — Recursos de registro de eventos para Python
    - `logging.config` — Configuração do módulo logging
    - `logging.handlers` — Manipuladores de logging
    - `platform` — Acesso aos dados de identificação da plataforma subjacente
    - `errno` — Símbolos padrão do sistema errno
    - `ctypes` — Uma biblioteca de funções externas para Python
- Bibliotecas de interface de linha de comando
    - `argparse` — Analisador sintático para opções de linha de comando, argumentos e subcomandos
    - `optparse` — Analisador sintático para opções de linha de comando
    - `getpass` — Entrada de senha portátil
    - `fileinput` — Itera sobre linhas de múltiplos fluxos de entrada
    - `curses` — Gerenciador de terminal para visualizadores de células de caracteres.
    - `curses.textpad` — Text input widget for curses programs
    - `curses.ascii` — Utilitários para caracteres ASCII
    - `curses.panel` — Uma extensão de pilha de painéis para o curses
    - `cmd` — Suporte para interpretadores de comando orientado a linhas

- Execução Concorrente
  - `threading` — Paralelismo baseado em threads
  - `multiprocessing` — Paralelismo baseado em processo
  - `multiprocessing.shared_memory` — Memória compartilhada para acesso direto entre processos
  - O pacote `concurrent`
  - `concurrent.futures` — Launching parallel tasks
  - `concurrent.interpreters` — Múltiplos interpretadores no mesmo processo
  - `subprocess` — Gerenciamento de subprocessos
  - `sched` — Agendador de eventos
  - `queue` — Uma classe de fila sincronizada
  - `contextvars` — Variáveis de contexto
  - `_thread` — API de segmentação de baixo nível
- Comunicação em Rede e Interprocesso
  - `asyncio` — E/S assíncrona
  - `socket` — Interface de rede de baixo nível
  - `ssl` — Invólucro de TLS/SSL para objetos socket
  - `select` — Waiting for I/O completion
  - `selectors` — High-level I/O multiplexing
  - `signal` — Define manipuladores para eventos assíncronos
  - `mmap` — Suporte a arquivos mapeados na memória
- Manuseio de Dados na Internet
  - `email` — Um e-mail e um pacote MIME manipulável
  - `json` — Codificador e decodificador JSON
  - `mailbox` — Manipule caixas de entradas em vários formatos
  - `mimetypes` — Mapeia nomes de arquivos para tipos MIME
  - `base64` — Base16, Base32, Base64, Base85 Data Encodings
  - `binascii` — Converte entre binário e ASCII
  - `quopri` — Codifica e decodifica dados MIME imprimidos entre aspas
- Ferramentas de Processamento de Markup Estruturado
  - `html` — Suporte HTML (HyperText Markup Language)
  - `html.parser` — Analisador simples de HTML e XHTML
  - `html.entities` — Definições de entidades gerais de HTML
  - Módulos de Processamento de XML
  - `xml.etree.ElementTree` — A API XML ElementTree

- `xml.dom` — The Document Object Model API
- `xml.dom.minidom` — Minimal DOM implementation
- `xml.dom.pulldom` — Suporte para construir árvores parciais de DOM
- `xml.sax` — Support for SAX2 parsers
- `xml.sax.handler` — Classes base para manipuladores de SAX
- `xml.sax.saxutils` — Utilitários de SAX
- `xml.sax.xmlreader` — Interface for XML parsers
- `xml.parsers.expat` — Fast XML parsing using Expat

- Protocolos de Internet e Suporte
  - `webbrowser` — Controlador de navegador web conveniente
  - `wsgiref` — Implementação de referência e utilitários WSGI
  - `urllib` — Módulos de manipulação de URL
  - `urllib.request` — Biblioteca extensível para abrir URLs
  - `urllib.response` — Response classes used by urllib
  - `urllib.parse` — Analisa URLs para componentes
  - `urllib.error` — Classes de exceção levantadas por urllib.request
  - `urllib.robotparser` — Analisador sintático de robots.txt
  - `http` — Módulos HTTP
  - `http.client` — HTTP protocol client
  - `ftplib` — FTP protocol client
  - `poplib` — Cliente de protocolo POP3
  - `imaplib` — IMAP4 protocol client
  - `smtplib` — Cliente de protocolo SMTP
  - `uuid` — Objetos UUID conforme **RFC 9562**
  - `socketserver` — Um framework para servidores de rede
  - `http.server` — HTTP servers
  - `http.cookies` — HTTP state management
  - `http.cookiejar` — Cookie handling for HTTP clients
  - `xmlrpc` — Módulos de servidor e cliente XMLRPC
  - `xmlrpc.client` — Cliente XML-RPC
  - `xmlrpc.server` — Servidores XML-RPC básicos
  - `ipaddress` — Biblioteca de manipulação de IPv4/IPv6
- Serviços Multimídia
  - `wave` — Read and write WAV files
  - `colorsys` — Conversões entre sistemas de cores
- Internacionalização

    - `gettext` — Serviços de internacionalização multilíngues
    - `locale` — Serviços de internacionalização
- Interfaces gráficas de usuário com Tk
    - `tkinter` — Interface Python para Tcl/Tk
    - `tkinter.colorchooser` — Diálogo de escolha de cor
    - `tkinter.font` — Invólucro de fontes Tkinter
    - Tkinter dialogs
    - `tkinter.messagebox` — Prompts de mensagem do Tkinter
    - `tkinter.scrolledtext` — Widget de texto com *scrollbar*
    - `tkinter.dnd` — Suporte para arrastar e soltar
    - `tkinter.ttk` — Widgets temáticos do Tk
    - IDLE — Editor e console Python
    - `turtle` — Gráficos tartaruga
- Ferramentas de Desenvolvimento
    - `typing` — Suporte para dicas de tipo
    - `pydoc` — Gerador de documentação e sistema de ajuda online
    - Modo de Desenvolvimento do Python
    - `doctest` — Teste exemplos interativos de Python
    - `unittest` — Framework de testes unitários
    - `unittest.mock` — mock object library
    - `unittest.mock` — primeiros passos
    - `test` — Pacote de Testes de Regressão do Python
    - `test.support` — Utilities for the Python test suite
    - `test.support.socket_helper` — Utilities for socket tests
    - `test.support.script_helper` — Utilities for the Python execution tests
    - `test.support.bytecode_helper` — Support tools for testing correct bytecode generation
    - `test.support.threading_helper` — Utilities for threading tests
    - `test.support.os_helper` — Utilities for os tests
    - `test.support.import_helper` — Utilities for import tests
    - `test.support.warnings_helper` — Utilities for warnings tests
- Depuração e perfilamento
    - Tabela de eventos de auditoria
    - `bdb` — Framework do depurador
    - `faulthandler` — Dump the Python traceback
    - `pdb` — O depurador do Python

    - `profiling` — Python profilers
    - `pstats` — Statistics for profilers
    - `timeit` — Mede o tempo de execução de pequenos trechos de código
    - `trace` — Rastreia ou acompanha a execução de instruções Python
    - `tracemalloc` — Trace memory allocations
- Empacotamento e Distribuição de Software
    - `ensurepip` — Inicialização do instalador do `pip`
    - `venv` — Criação de ambientes virtuais
    - `zipapp` — Gerencia arquivos zip executáveis do Python
- Serviços de Tempo de Execução Python
    - `sys` — Parâmetros e funções específicas do sistema
    - `sys.monitoring` — Monitoramento de eventos de execução
    - `sysconfig` — Fornece acesso às informações de configuração do Python
    - `builtins` — Objetos embutidos
    - `__main__` — Ambiente de código principal
    - `warnings` — Controle de avisos
    - `dataclasses` — Data Classes
    - `contextlib` — Utilitários para contextos da instrução `with`
    - `abc` — Classes base abstratas
    - `atexit` — Manipuladores de saída
    - `traceback` — Imprime ou recupera um traceback da pilha
    - `__future__` — Definições de instruções future
    - `gc` — Interface para o coletor de lixo
    - `inspect` — Inspecione objetos vivos
    - `annotationlib` — Funcionalidade para introspecção de anotações
    - `site` — Gancho de configuração específico do site
- Interpretadores Python Personalizados
    - `code` — Classes bases do interpretador
    - `codeop` — Compila código Python
- Importando módulos
    - `zipimport` - Importa módulos de arquivos Zip
    - `pkgutil` — Utilitário de extensão de pacote
    - `modulefinder` — Procura módulos usados por um script
    - `runpy` — Localizando e executando módulos Python

    - `importlib` — A implementação de `import`
    - `importlib.resources` – Leitura, abertura e acesso a recursos de pacotes
    - `importlib.resources.abc` – Classes base abstratas para recursos
    - `importlib.metadata` – Acessando metadados do pacote
    - A inicialização do caminho de pesquisa de módulos `sys.path`
- Serviços da Linguagem Python
    - `ast` — Árvores de sintaxe abstrata
    - `symtable` — Acesso a tabela de símbolos do compilador
    - `token` — Constantes usadas com árvores de análises do Python
    - `keyword` — Testando palavras reservadas do Python
    - `tokenize` — Tokenizador para código-fonte Python
    - `tabnanny` — Detecção de indentação ambígua
    - `pyclbr` — Suporte a navegador de módulos do Python
    - `py_compile` — Compila arquivos fonte do Python
    - `compileall` — Compilar bibliotecas do Python para bytecode
    - `dis` — Disassembler de bytecode do Python
    - `pickletools` — Ferramentas para desenvolvedores pickle
- Serviços Específicos do MS Windows
    - `msvcrt` — Rotinas úteis do runtime MS VC++
    - `winreg` — Acesso aos registros do Windows
    - `winsound` — Interface de reprodução de som para o Windows
- Serviços específicos do Unix
    - `shlex` — Simple lexical analysis
    - `posix` — As chamadas de sistema mais comuns do POSIX
    - `pwd` — A senha do banco de dados
    - `grp` — O banco de dados de grupos
    - `termios` — Controle de tty no estilo POSIX
    - `tty` — Funções de controle de terminal
    - `pty` — Utilitários de pseudoterminal
    - `fcntl` — as chamadas de sistema `fcntl` e `ioctl`
    - `resource` — Informação de uso de recursos
    - `syslog` — Rotinas da biblioteca syslog do Unix
- Interface de linha de comando (CLI) de módulos
- Módulos substituídos
    - `getopt` — Analisador sintático no estilo C para opções de linha de comando

- `profile` — Pure Python profiler
- Módulos removidos
- Considerações de segurança

Introdução

A biblioteca padrão consiste em uma coleção de módulos. Há muitas formas de dissecar essa coleção. A maioria dos módulos são escritos em Python, mas alguns são escritos em C. Todos podem ser importados em seu programa para adicionar funcionalidade Alguns módulos fornecem interfaces que são muito específicas do Python, como imprimir um stack trace (situação da pilha de execução); alguns fornecem interfaces que são específicas para um sistema operacional em particular, tais como acessar hardware específico; outros fornecem interfaces que são específicas de um domínio de aplicação em particular, como desenvolvimento web. Alguns módulos estão disponíveis em todas as versões do Python; outros estão apenas disponíveis quando o sistema subjacente suporta ou necessita deles; e ainda outros estão disponíveis apenas quando uma opção de configuração em particular foi escolhida no momento em que o Python foi compilado e instalado.

Se você começar a ler este manual do início, e pular para o próximo capítulo quando estiver entediado, você terá uma visão geral razoável dos módulos disponíveis e áreas de aplicação que são suportadas pela biblioteca Python. É claro, você não *tem* que ler como se fosse um romance — você também pode navegar pela tabela de conteúdos (no início do manual), ou procurar por uma função, módulo ou termo específicos no índice (na parte final). E finalmente, se você gostar de aprender sobre assuntos diversos, você pode escolher uma página aleatória e leia uma seção ou duas. Independente da ordem na qual você leia as seções deste manual, ajuda primeiro ler Funções embutidas, já que o resto desta seção requer familiaridade com este material.

Ver também

As funções e classes embutidas (que podem ser usadas sem uma instrução `import`) estão descritas em Referência da bibliotecas embutidas do Python.

Observações sobre disponibilidade

- Uma observação “Disponibilidade: Unix” significa que essa função é comumente encontrada em sistemas Unix. Não faz nenhuma reivindicação sobre sua existência em um sistema operacional específico.
- Caso não seja definido separadamente, todas as funções que se definem como “Disponibilidade: Unix” são suportadas no macOS, IOS e Android, todas construídas em um Unix core.
- Se uma nota de disponibilidade contiver uma versão mínima do Kernel e uma versão mínima da libc, ambas as condições deverão ser atendidas. Por exemplo, um recurso com a observação *Disponibilidade: Linux >= 3.17 com glibc >= 2.27* requer Linux 3.17 ou mais recente e glibc 2.27 ou mais recente.

Plataformas WebAssembly

As plataformas WebAssembly [https://webassembly.org/] `wasm32-emscripten` (Emscripten [https://emscripten.org/]) e `wasm32-wasi` (WASI [https://wasi.dev/]) fornecem um subconjunto de APIs POSIX. Os tempos de execução e navegadores do WebAssembly são colocados em área restrita e têm acesso limitado ao host e aos recursos externos. Qualquer módulo de biblioteca padrão do Python que usa processos, encadeamento, rede, sinais ou outras formas de comunicação entre processos (IPC) não está disponível ou pode não funcionar como em outros sistemas semelhantes ao Unix. E/S de arquivo, sistema de arquivos e funções relacionadas à permissão do Unix também são restritas. Emscripten não permite bloqueio de E/S. Outras operações de bloqueio como **`sleep()`** bloqueiam o laço de eventos do navegador.

As propriedades e o comportamento do Python em plataformas WebAssembly dependem da versão Emscripten [https://emscripten.org/]-SDK ou WASI [https://wasi.dev/]-SDK, tempos de execução WASM (navegador, NodeJS, wasmtime [https://wasmtime.dev/]) e sinalizadores de tempo de construção do Python. WebAssembly, Emscripten e WASI são padrões em evolução; alguns recursos como rede podem ser suportados no futuro.

Para Python no navegador, os usuários devem considerar Pyodide [https://pyodide.org/] ou PyScript [https://pyscript.net/]. O PyScript é construído sobre o Pyodide, que por sua vez é construído sobre o CPython e o Emscripten. O Pyodide fornece acesso às APIs JavaScript e DOM dos navegadores, bem como recursos de rede limitados com as APIs `XMLHttpRequest` e `Fetch` do JavaScript.

- As APIs relacionadas a processo não estão disponíveis ou sempre falham com um erro. Isso inclui APIs que geram novos processos (**`fork()`**, **`execve()`**), aguardam processos (**`waitpid()`**), enviam sinais (**`kill()`**), ou interagir com processos. O **`subprocess`** é importável, mas não funciona.
- O módulo **`socket`** está disponível, mas é limitado e se comporta de maneira diferente de outras plataformas. No Emscripten, os soquetes são sempre não bloqueantes e requerem código JavaScript adicional e auxiliares no servidor para intermediar TCP por meio de WebSockets; veja Emscripten Networking [https://emscripten.org/docs/porting/networking.html] para mais informações. A primeira snapshot de preview do WASI permite apenas soquetes de um descritor de arquivo existente.
- Algumas funções são esboço que não fazem nada e sempre retornam valores definidos no código.
- As funções relacionadas a descritores de arquivo, permissões de arquivo, propriedade de arquivo e links são limitadas e não suportam algumas operações. Por exemplo, WASI não permite links simbólicos com nomes de arquivo absolutos.

Plataformas móveis

O Android e o IOS são em linhas gerais sistemas operacionais POSIX. File I/O, socket handling, e threading se comportam como em qualquer sistema operacional POSIX. Entretanto, existem diversas diferenças:

- Plataformas móveis podem usar o Python apenas no modo “incorporado”. Não existe um Python REPL, e não existe possibilidade para usar executáveis em separado como **python** ou **pip**. Para adicionar código Python para seu aplicativo móvel, você deve usar a

API de incorporação do Python. Para maiores detalhes veja Utilizando Python no Android e Usando Python no iOS.

- Subprocessos:
  - No Android, criar subprocessos é possível mas oficialmente sem suporte [https://issuetracker.google.com/issues/128554619#comment4]. Em particular, Android não oferece suporte para nenhuma parte da API System V IPC, assim `multiprocessing` não está disponível.
  - Uma aplicação de iOS não pode usar qualquer forma de subprocessamento, multiprocessamento ou comunicação entre processos. Se uma aplicação de iOS tentar criar um subprocesso, o processo que cria o subprocesso será travado ou encerrado abruptamente. Uma aplicação de iOS não tem visibilidade de outras aplicações em execução, nem capacidade de comunicação com outras aplicações em execução, fora das APIs específicas do iOS que existem para essa finalidade.

- Aplicativos móveis possuem acesso limitado a recursos do sistema (como o relógio do sistema). Esses recursos serão em sua maioria apenas *leitura*, e tentativas de modificar esses recusos geralmente falhará.

- Entrada e saída do console:
  - No Android, o `stdout` e o `stderr` nativos não estão conectados a nada, então o Python instala seus próprios fluxos que redirecionam mensagens para o log do sistema. Esses podem ser vistos sob a tags `python.stdout` e `python.stderr`, respectivamente.

  - As aplicações de iOS têm um conceito limitado de saída do console. `stdout` e `stderr` *existem*, e o conteúdo escrito em `stdout` e `stderr` será visível nos registros (logs) quando executado no Xcode, mas este conteúdo *não* será gravado no registro do sistema. Se um usuário que instalou sua aplicação fornecer os registros da aplicação como ajuda de diagnóstico, eles não incluirão nenhum detalhe escrito em `stdout` ou `stderr`.

- As aplicações para dispositivos móveis não têm nenhum conceito de `stdin`. Embora essas aplicações possam exibir um teclado na tele, este é um recurso de software, não algo anexado ao `stdin`.

  Como consequência, módulos Python que envolvem manipulação de console (como **`curses`** e **`readline`**) não estão disponíveis em plataformas móveis.

Serviços de Processamento de Texto

Os módulos descritos neste capítulo fornecem uma ampla variedade de operações de manipulação de string e outros serviços de processamento de texto.

O módulo `codecs` descrito em Serviços de Dados Binários também é altamente relevante para o processamento de texto. Além disso, consulte a documentação do tipo string do Python em Tipo sequência de texto — str.

- `string` — Operações comuns de strings
  - Constantes de strings
  - Formatação personalizada de strings
  - Sintaxe das strings de formato
    - Minilinguagem de especificação de formato
    - Exemplos de formato
  - Strings template ($-strings)
  - Funções auxiliares
- `string.templatelib` — Suporte para literais de string template
  - Strings template
  - Tipos
  - Funções auxiliares
- `re` — Operações com expressões regulares
  - Sintaxe de expressão regular
  - Conteúdo do módulo
    - Sinalizadores
    - Funções
    - Exceções
  - Regular expression objects
  - Match objects
  - Regular expression examples
    - Checking for a pair
    - Simulando scanf()

        - search() vs. prefixmatch()
        - prefixmatch() vs. match()
        - Making a phonebook
        - Text munging
        - Finding all adverbs
        - Finding all adverbs and their positions
        - Raw string notation
        - Writing a tokenizer
- `difflib` — Helpers for computing deltas
    - Junk heuristic
    - The `difflib` algorithm
    - Diff generation
    - Junk definition functions
    - SequenceMatcher objects
    - Exemplos
        - SequenceMatcher examples
        - Differ example
        - A command-line interface to difflib
        - ndiff example
- `textwrap` — Quebra automática e preenchimento de texto
- `unicodedata` — Banco de dados de Unicode
- `stringprep` — Preparação de string para internet
- `readline` — Interface para o GNU readline
    - Arquivo init
    - Buffer de linha
    - Arquivo de histórico
    - Lista de histórico
    - Ganchos de inicialização
    - Autocomplemento
    - Exemplo
- `rlcompleter` — Função de autocomplemento para GNU readline

`string` — Operações comuns de strings

Código-fonte: Lib/string/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/string/__init__.py]

Ver também

Tipo sequência de texto — str

String Methods

Constantes de strings

As constantes definidas neste módulo são:

string.ascii_letters
: A concatenação das constantes **`ascii_lowercase`** e **`ascii_uppercase`** descritas abaixo. Este valor não depende da localidade.

string.ascii_lowercase
: As letras minúsculas `'abcdefghijklmnopqrstuvwxyz'`. Este valor não depende da localidade e não mudará.

string.ascii_uppercase
: As letras maiúsculas `'ABCDEFGHIJKLMNOPQRSTUVWXYZ'`. Este valor não depende da localidade e não mudará.

string.digits
: A string `'0123456789'`.

string.hexdigits

A string `'0123456789abcdefABCDEF'`.

string.octdigits

A string `'01234567'`.

string.punctuation

String de caracteres ASCII que são considerados caracteres de pontuação na localidade `C`: `!"#$%&'()*+,-./:;<=>?@[\]^_`{|}~`.

string.printable

String de caracteres ASCII que são considerados imprimíveis pelo Python. Esta é uma combinação de **`digits`**, **`ascii_letters`**, **`punctuation`** e **`whitespace`**.

> **Nota**
>
> Por design, **`string.printable.isprintable()`** retorna **`False`**. Em particular, `string.printable` não é imprimível no sentido POSIX (veja *LC_CTYPE* *[https://manpages.debian.org/locale(5)]*).

string.whitespace

Uma string contendo todos os caracteres ASCII que são considerados espaços em branco. Isso inclui espaço de caracteres, tabulação, avanço de linha, retorno, avanço de formulário e tabulação vertical.

Formatação personalizada de strings

A classe embutida de string fornece a capacidade de substituir variáveis complexas e de formatar valores por meio do método **`format()`** descrito na **PEP 3101** [https://peps.python.org/pep-3101/]. A classe **`Formatter`** no módulo **`string`** permite que você crie e personalize seus próprios comportamentos de formatação de strings, usando a mesma implementação do método embutido **`format()`**.

class string.Formatter

A classe **`Formatter`** tem os seguintes métodos públicos:

format(*format_string*, /, **args*, ***kwargs*)

O método principal da API. Ele aceita uma string de formato e um conjunto arbitrário de argumentos posicionais e nomeados. É apenas um invólucro que chama `vformat()`.

Alterado na versão 3.7: Um argumento de string de formato é agora somente-posicional.

vformat(*format_string*, *args*, *kwargs*)

Esta função realiza o trabalho real de formatação. Ela é exposta como uma função separada para casos em que você deseja passar um dicionário predefinido de argumentos, em vez de desempacotar e empacotar novamente o dicionário como argumentos individuais usando a sintaxe `*args` e `**kwargs`. `vformat()` faz o trabalho de quebrar a string de formato em dados de caracteres e campos de substituição. Ela chama os vários métodos descritos a seguir.

Além disso, o `Formatter` define uma série de métodos que devem ser substituídos por subclasses:

parse(*format_string*)

Percorre format_string e retorna um iterável de tuplas (*literal_text*, *field_name*, *format_spec*, *conversion*). Isso é usado por `vformat()` para quebrar a string em texto literal ou em campos de substituição.

Os valores da tupla representam, conceitualmente, um intervalo de texto literal, seguido de um único campo de substituição. Se não houver texto literal (o que pode ocorrer se dois campos de substituição forem consecutivos), *literal_text* será uma string de comprimento zero. Se não houver campo de substituição, os valores de *field_name*, *format_spec* e *conversion* serão `None`. O valor de *field_name* não é modificado, e a numeração automática de campos posicionais não numerados é realizada por `vformat()`.

get_field(*field_name*, *args*, *kwargs*)

Dado *field_name*, converte-o em um objeto a ser formatado. A autonumeração de *field_name* retornada de `parse()` é feita por

`vformat()` antes de chamar este método. Retorna uma tupla (obj, used_key). A versão padrão aceita strings no formato definido na **PEP 3101** [https://peps.python.org/pep-3101/], como “0[name]” ou “label.title”. *args* e *kwargs* são como os argumentos passados para **`vformat()`**. O valor de retorno *used_key* tem o mesmo significado que o parâmetro *key* em `get_value()`.

get_value(*key*, *args*, *kwargs*)

Obtém o valor de um campo. O argumento *key* será um inteiro ou uma string. Se for um inteiro, ele representa o índice do argumento posicional em *args*; se for uma string, representa um argumento nomeado em *kwargs*.

O parâmetro *args* é definido para a lista de argumentos posicionais de `vformat()`, e o parâmetro *kwargs* é definido para o dicionário de argumentos nomeados.

Para nomes de campos compostos, essas funções são chamadas apenas para o primeiro componente do nome do campo; os componentes subsequentes são tratados por meio de operações normais de atributo e de indexação.

Então, por exemplo, a expressão de campo ‘0.name’ faria com que `get_value()` fosse chamado com o argumento *key* 0. O atributo `name` será pesquisado após a função **`get_value()`** retornar, chamando a função embutida `getattr()`.

Se o índice ou palavra-chave se referir a um item que não existe, um `IndexError` ou `KeyError` deve ser levantado.

check_unused_args(*used_args*, *args*, *kwargs*)

Implementa a verificação de argumentos não usados, se desejar. Os argumentos para esta função são o conjunto de todas as chaves de argumento realmente referidas na string de formato (inteiros para argumentos posicionais e strings para argumentos nomeados), e uma referência a *args* e *kwargs* que foi passada para vformat. O conjunto de argumentos não utilizados pode ser calculado com base nesses

parâmetros. Presume-se que `check_unused_args()` levante uma exceção caso a verificação falhe.

format_field(*value*, *format_spec*)

`format_field()` simplesmente chama o global embutido `format()`. O método é fornecido para que as subclasses possam substituí-lo.

convert_field(*value*, *conversion*)

Converte o valor (retornado por `get_field()`) de acordo com um tipo de conversão (como na tupla retornada pelo método `parse()`). A versão padrão entende os tipos de conversão “s” (str), “r” (repr) e “a” (ascii).

Sintaxe das strings de formato

O método `str.format()` e a classe `Formatter` compartilham a mesma sintaxe para strings de formato (embora no caso de `Formatter`, as subclasses possam definir sua própria sintaxe de string de formato). A sintaxe é relacionada a literais de strings formatadas e literais de strings templates, mas é menos sofisticada e, em especial, não oferece suporte a expressões arbitrárias em interpolações.

As strings de formato contêm “campos de substituição” entre chaves `{}`. Tudo o que não estiver entre chaves é considerado texto literal e é copiado inalterado para a saída. Se você precisar incluir um caractere de chave no texto literal, ele pode ser escapado duplicando-o: `{{` e `}}`.

A gramática para um campo de substituição é a seguinte:

```
replacement_field: "{" [field_name] ["!" conversion] [":" format
field_name:        arg_name ("." attribute_name | "[" element_ir
arg_name:          [identifier | digit+]
attribute_name:    identifier
element_index:     digit+ | index_string
index_string:      <any source character except "]"> +
conversion:        "r" | "s" | "a"
format_spec:       format-spec:format_spec
```

Em termos menos formais, o campo de substituição pode começar com um *field_name* que especifica o objeto cujo valor deve ser formatado e inserido na saída, em vez do próprio campo de substituição. O *field_name* é opcionalmente seguido de um campo *conversion*, que é precedido por um ponto de exclamação `'!'`, e de um *format_spec*, que é precedido por dois pontos `':'`. Eles especificam um formato não padrão para o valor de substituição.

Veja também a seção Minilinguagem de especificação de formato.

O próprio *field_name* começa com um *arg_name*, que pode ser um número ou uma palavra-chave. Se for um número, ele se refere a um argumento posicional, e se for uma palavra-chave, ele se refere a um argumento nomeado. Um *arg_name* é tratado como um número se uma chamada à **`str.isdecimal()`** na string retornar verdadeiro. Se os arg_names numéricos em uma string de formato forem 0, 1, 2, … em sequência, eles podem ser todos omitidos (não apenas alguns) e os números 0, 1, 2, … serão inseridos automaticamente nessa ordem. Como *arg_name* não está delimitado por aspas, não é possível especificar chaves de dicionário arbitrárias (por exemplo, as strings `'10'` ou `':-]'`) em uma string de formato. O *arg_name* pode ser seguido por qualquer número de expressões de índice ou de atributo. Uma expressão da forma `'.name'` seleciona o atributo nomeado com **`getattr()`**, enquanto uma expressão da forma `'[index]'` faz uma pesquisa de índice com **`__getitem__()`**.

Alterado na versão 3.1: Os especificadores de argumento posicional podem ser omitidos para **`str.format()`**, de forma que `'{} {}'.format(a, b)` é equivalente a `'{0} {1}'.format(a, b)`.

Alterado na versão 3.4: Os especificadores de argumento posicional podem ser omitidos para **`Formatter`**.

Alguns exemplos simples de string de formato:

```
"Primeiro, tu deverás contar até {0}"  # Referencia o primeiro a
"Traga-me um {}"                       # Referencia implicitamer
"De {} para {}"                        # Mesmo que "De {0} para
```

```
"Minha missão é {nome}"                # Referencia argumento no
"Peso em toneladas {0.peso}"           # O atributo 'peso' do pr
"Unidades destruídas: {jogadores[0]}"  # Primeiro elemento do ar
```

O campo *conversion* causa uma coerção de tipo antes da formatação. Normalmente, o trabalho de formatação de um valor é feito pelo método **`__format__()`** do próprio valor. No entanto, em alguns casos, é desejável forçar um tipo a ser formatado como string, substituindo sua própria definição de formatação. Ao converter o valor em uma string antes de chamar **`__format__()`**, a lógica de formatação normal é contornada.

Atualmente, há suporte para três sinalizadores de conversão: `'!s'`, que chama **`str()`** no valor; `'!r'`, que chama **`repr()`**; e `'!a'`, que chama **`ascii()`**.

Alguns exemplos:

```
"Harold é um {0!s} esperto"        # Chama str() no primeiro arg
"Trazei à luz o {name!r} sagrado"    # Chama repr() no primeiro
"Mais {!a}"                       # Chama ascii() no primeiro arg
```

O campo *format_spec* contém uma especificação de como o valor deve ser apresentado, incluindo detalhes como a largura do campo, o alinhamento, o preenchimento, a precisão decimal e assim por diante. Cada tipo de valor pode definir sua própria “minilinguagem de formatação” ou interpretação de *format_spec*.

A maioria dos tipos embutidos oferece suporte a uma minilinguagem de formatação comum, descrita na próxima seção.

O campo *format_spec* também pode conter campos de substituição aninhados. Esses campos de substituição aninhados podem conter um nome de campo, um sinalizador de conversão e uma especificação de formato, mas um aninhamento mais profundo não é permitido. Os campos de substituição em format_spec são substituídos antes que a string *format_spec* seja interpretada. Isso permite que a formatação de um valor seja especificada dinamicamente.

Veja a seção Exemplos de formato para alguns exemplos.

Minilinguagem de especificação de formato

“Especificações de formato” são usadas nos campos de substituição contidos em uma string de formato para definir como os valores individuais são apresentados (consulte Sintaxe das strings de formato, Literais de strings formatadas e Literais de string template (t-strings)). Elas também podem ser passadas diretamente para a função embutida `format()`. Cada tipo formatável pode definir como a especificação do formato deve ser interpretada.

A maioria dos tipos embutidos implementa as seguintes opções para especificações de formato, embora algumas das opções de formatação sejam válidas apenas para tipos numéricos.

Uma convenção geral é que uma especificação de formato vazia produz o mesmo resultado que se você tivesse chamado `str()` no valor. Uma especificação de formato não vazio geralmente modifica o resultado.

A forma geral de um *especificador de formato padrão* é:

```
format_spec:              [options][width_and_precision][type]
options:                  [[fill]align][sign]["z"]["#"]["0"]
fill:                     <any character>
align:                    "<" | ">" | "=" | "^"
sign:                     "+" | "-" | " "
width_and_precision:      [width_with_grouping][precision_with_gr
width_with_grouping:      [width][grouping]
precision_with_grouping:  "." [precision][grouping] | "." groupir
width:                    digit+
precision:                digit+
grouping:                 "," | "_"
type:                     "b" | "c" | "d" | "e" | "E" | "f" | "F'
                          | "G" | "n" | "o" | "s" | "x" | "X" | '
```

Se um valor *align* válido for especificado, ele pode ser precedido por um caractere de preenchimento *fill* que pode ser qualquer caractere e o padrão é

um espaço se omitido. Não é possível usar uma chave literal (”{” ou “}”) como o caractere *fill* em uma string formatada literal ou ao usar o método `str.format()`. No entanto, é possível inserir uma chave com um campo de substituição aninhado. Esta limitação não afeta a função `format()`.

O significado das várias opções de alinhamento é o seguinte:

Opção	Significado
`'<'`	Força o alinhamento à esquerda do campo no espaço disponível (este é o padrão para a maioria dos objetos).
`'>'`	Força o alinhamento à direita do campo no espaço disponível (este é o padrão para números).
`'='`	Força o preenchimento a ser colocado após o sinal (se houver), mas antes dos dígitos. É usado para imprimir campos na forma “+000000120”. Esta opção de alinhamento só é válida para tipos numéricos, excluindo `complex`. Torna-se o padrão para números quando “0” precede imediatamente a largura do campo.
`'^'`	Força a centralização do campo no espaço disponível.

Observe que, a menos que uma largura de campo mínima seja definida, a largura do campo sempre será igual à largura dos dados que preenchem, de modo que a opção de alinhamento não tem significado neste caso.

A opção *sign* só é válida para tipos numéricos e pode ser um dos seguintes:

Opção	Significado

Opção	Significado
`'+'`	Indica que um sinal deve ser usado para números positivos e negativos.
`'-'`	Indica que um sinal deve ser usado apenas para números negativos (este é o comportamento padrão).
espaço	Indica que um espaço inicial deve ser usado em números positivos e um sinal de menos em números negativos.

A opção `'z'` força valores de ponto flutuante de zero negativo para zero positivo após o arredondamento, conforme a precisão do formato. Esta opção só é válida para tipos de apresentação de ponto flutuante.

Alterado na versão 3.11: Adicionada a opção `'z'` (veja também **PEP 682** [https://peps.python.org/pep-0682/]).

A opção `'#'` faz com que a “forma alternativa” seja usada na conversão. A forma alternativa é definida de forma diferente para cada tipo. Esta opção é válida apenas para tipos inteiros, pontos flutuantes e complexos. Para inteiros, quando a saída binária, octal ou hexadecimal é usada, esta opção adiciona o prefixo respectivo `'0b'`, `'0o'`, `'0x'` ou `'0X'` ao valor de saída. Para pontos flutuantes e complexos, a forma alternativa faz com que o resultado da conversão sempre contenha um caractere de ponto decimal, mesmo que não haja nenhum dígito após. Normalmente, um caractere de ponto decimal aparece no resultado dessas conversões apenas se houver um dígito depois. Além disso, nas conversões `'g'` e `'G'`, os zeros finais não são removidos do resultado.

O *width* é um número inteiro decimal que define a largura mínima total do campo, incluindo quaisquer prefixos, separadores e outros caracteres de

formatação. Se não for especificado, a largura do campo será determinada pelo conteúdo.

Quando nenhum alinhamento explícito é fornecido, preceder o campo *width* com um caractere zero (`'0'`) habilita o preenchimento por zero, com reconhecimento de sinal, para tipos numéricos, exceto `complex`. Isso é equivalente a um caractere *fill* de valor `'0'` e *alignment* de `'='`.

Alterado na versão 3.10: Preceder o campo *width* com `'0'` não afeta mais o alinhamento padrão de strings.

precision é um número decimal que indica quantos dígitos devem ser exibidos depois do ponto decimal para um valor de ponto flutuante formatado com `'f'` e `'F'`, ou antes e depois do ponto decimal para um valor de ponto flutuante formatado com `'g'` ou `'G'`. Para tipos não numéricos, o campo indica o tamanho máximo do campo – em outras palavras, quantos caracteres serão usados do conteúdo do campo. *precision* não é permitido para valores inteiros.

A opção *grouping* após os campos *width* e *precision* especifica um separador de grupo de dígitos para as partes inteira e fracionária de um número, respectivamente. Pode ser um dos seguintes:

Opção	Significado
`','`	Insere uma vírgula a cada 3 dígitos para tipos de apresentação inteiros `'d'` e de ponto flutuante, excluindo `'n'`. Não há suporte para essa opção em outros tipos de apresentação.
`'_'`	Insere um sublinhado a cada 3 dígitos para tipos de apresentação inteiros `'d'` e tipos de apresentação de ponto flutuante, excluindo `'n'`. Para os tipos de apresentação inteiros `'b'`, `'o'`, `'x'` e `'X'`, os sublinhados são inseridos a cada 4 dígitos. Para outros tipos de apresentação, esta opção não é suportada.

Para um separador que respeita as configurações da localidade, utilize em vez disso o `'n'` do tipo de apresentação para ponto flutuante ou do tipo de apresentação para números inteiros.

Alterado na versão 3.1: Adicionada a opção `','` (veja também **PEP 378** [https://peps.python.org/pep-0378/]).

Alterado na versão 3.6: Adicionada a opção `'_'` (veja também **PEP 515** [https://peps.python.org/pep-0515/]).

Alterado na versão 3.14: Suporte a opção *grouping* para a parte fracionária.

Finalmente, o *type* determina como os dados devem ser apresentados.

Os tipos de apresentação de string disponíveis são:

Tipo	Significado
`'s'`	Formato de string. Este é o tipo padrão para strings e pode ser omitido.
None	O mesmo que `'s'`.

Os tipos de apresentação inteira disponíveis são:

Tipo	Significado
`'b'`	Formato binário. Exibe o número na base 2.
`'c'`	Caractere. Converte o inteiro no caractere Unicode correspondente antes de imprimir.

Tipo	Significado
`'d'`	Inteiro decimal. Exibe o número na base 10.
`'o'`	Formato octal. Exibe o número na base 8.
`'x'`	Formato hexadecimal. Exibe o número na base 16, usando letras minúsculas para os dígitos acima de 9.
`'X'`	Formato hexadecimal. Exibe o número na base 16, usando letras maiúsculas para os dígitos acima de 9. No caso de `'#'` ser especificado, o prefixo `'0x'` será maiúsculo para `'0X'` também.
`'n'`	Número. É o mesmo que `'d'`, exceto pelo fato de utilizar a configuração de localidade atual para inserir os separadores de grupo de dígitos apropriados. Observe que a localidade padrão não é a localidade do sistema. Dependendo do seu caso de uso, talvez você queira definir `LC_NUMERIC` com `locale.setlocale()` antes de usar `'n'`.
None	O mesmo que `'d'`.

Além dos tipos de apresentação acima, os inteiros podem ser formatados com os tipos de apresentação de ponto flutuante listados abaixo (exceto `'n'` e `None`). Ao fazer isso, `float()` é usado para converter o inteiro em um número de ponto flutuante antes da formatação.

Os tipos de apresentação disponíveis para valores `float` e `Decimal` são:

Tipo	Significado
`'e'`	Notação científica. Para uma dada precisão `p`, formata o número em notação científica com a letra “e” separando o coeficiente do expoente. O coeficiente tem um dígito antes e `p` dígitos depois do ponto decimal, totalizando `p + 1` dígitos significativos. Sem precisão fornecida, usa uma precisão de `6` dígitos após o ponto decimal para **`float`** e mostra todos os dígitos de coeficiente para **`Decimal`**. Se `p=0`, o ponto decimal é omitido a menos que a opção `#` seja usada. Para **`float`**, o expoente sempre contém pelo menos dois dígitos e é zero se o valor for zero.
`'E'`	Notação científica. O mesmo que `'e'`, exceto que usa um ‘E’ maiúsculo como caractere separador.
`'f'`	Notação de ponto fixo. Para uma dada precisão `p`, formata o número como um número decimal com exatamente `p` dígitos após o ponto decimal. Sem precisão fornecida, usa uma precisão de `6` dígitos após o ponto decimal para **`float`** e uma precisão grande o suficiente para mostrar todos os dígitos de coeficiente para **`Decimal`**. Se `p=0`, o ponto decimal é omitido a menos que a opção `#` seja usada.
`'F'`	Notação de ponto fixo. O mesmo que `'f'`, mas converte `nan` para `NAN` e `inf` para `INF`.

Tipo	Significado
`'g'`	Formato geral. Para uma determinada precisão `p >= 1`, isso arredonda o número para `p` dígitos significativos e, então, formata o resultado em ponto fixo ou em notação científica, conforme sua magnitude. Uma precisão de `0` é tratada como equivalente a uma precisão de `1`. As regras precisas são as seguintes: suponha que o resultado formatado com tipo de apresentação `'e'` e precisão `p-1` teria o expoente `exp`. Então, se `m <= exp < p`, onde `m` é -4 para pontos flutuantes e -6 para **`Decimals`**, o número é formatado com o tipo de apresentação `'f'` e precisão `p-1-exp`. Caso contrário, o número é formatado com o tipo de apresentação `'e'` e a precisão `p-1`. Em ambos os casos, zeros à direita insignificantes são removidos do significando, e o ponto decimal também é removido se não houver dígitos restantes após ele, a menos que a opção `'#'` seja usada. Sem precisão fornecida, usa uma precisão de `6` dígitos significativos para **`float`**. Para **`Decimal`**, o coeficiente do resultado é formado a partir dos dígitos do coeficiente do valor; a notação científica é usada para valores menores que `1e-6` em valor absoluto e valores em que o valor posicional do dígito menos significativo é maior que 1, e a notação de ponto fixo é usada de outra forma. Infinito positivo e negativo, zero positivo e negativo e não-números, são formatados como `inf`, `-inf`, `0`, `-0` e `nan`, respectivamente, independentemente da precisão.
`'G'`	Formato geral. O mesmo que `'g'`, exceto muda para `'E'` se o número ficar muito grande. As representações de infinito e de NaN também são maiúsculas.

Tipo	Significado
`'n'`	Número. É o mesmo que `'g'`, exceto pelo fato de utilizar a configuração de localidade atual para inserir os separadores de grupo de dígitos apropriados para a parte inteira de um número. Observe que a localidade padrão não é a localidade do sistema. Dependendo do seu caso de uso, talvez você queira definir `LC_NUMERIC` com `locale.setlocale()` antes de usar `'n'`.
`'%'`	Porcentagem. Multiplica o número por 100 e exibe no formato fixo (`'f'`), seguido de um sinal de porcentagem.
None	Para `float`, é o como o tipo `'g'`, exceto que quando a notação de ponto fixo é usada para formatar o resultado, ela sempre inclui pelo menos um dígito após o ponto decimal, e troca para a notação científica quando `exp >= p - 1`. Quando a precisão não é especificada, este último será tão grande quanto necessário para representar o valor fornecido fielmente. Para `Decimal`, é o mesmo que `'g'` ou `'G'`, dependendo do valor de `context.capitals` no contexto decimal atual. O efeito geral é combinar a saída de `str()` conforme alterada pelos demais modificadores de formato.

O resultado deve ser arredondado corretamente para uma dada precisão `p` de dígitos após o ponto decimal. O modo de arredondamento para `float` corresponde ao da função embutida `round()`. Para `Decimal`, o modo de arredondamento do contexto atual será aplicado.

Os tipos de apresentação disponíveis para `complex` são os mesmos que para `float` (`'%'` não é permitido). Os componentes real e imaginário de um número complexo são formatados como números de ponto flutuante, conforme o tipo de apresentação especificado. Eles são separados pelo sinal obrigatório da parte imaginária, que é terminada por um sufixo `j`. Se o tipo de apresentação estiver faltando, o resultado corresponderá à saída de `str()` (números complexos com uma parte real diferente de zero também são cercados por parênteses), possivelmente alterado por outros modificadores de formato.

Exemplos de formato

Esta seção contém exemplos da sintaxe de `str.format()` e comparação com a antiga formatação com `%`.

Na maioria dos casos, a sintaxe é semelhante à antiga formatação com `%`, com a adição de `{}` e o uso de `:` em vez de `%`. Por exemplo, `'%03.2f'` pode ser traduzido para `'{:03.2f}'`.

A nova sintaxe de formato também oferece suporte a novas e diferentes opções, como mostradas nos exemplos a seguir.

Acessando argumentos por posição:

```
>>> '{0}, {1}, {2}'.format('a', 'b', 'c')
'a, b, c'
>>> '{}, {}, {}'.format('a', 'b', 'c')  # somente 3.1+
'a, b, c'
>>> '{2}, {1}, {0}'.format('a', 'b', 'c')
'c, b, a'
>>> '{2}, {1}, {0}'.format(*'abc')      # desempacotando sequênc
'c, b, a'
>>> '{0}{1}{0}'.format('abra', 'cad')   # índices dos argumentos
'abracadabra'
```

Acessando argumentos por nome:

```
>>> 'Coordinates: {latitude}, {longitude}'.format(latitude='37.2
'Coordinates: 37.24N, -115.81W'
```

```
>>> coord = {'latitude': '37.24N', 'longitude': '-115.81W'}
>>> 'Coordinates: {latitude}, {longitude}'.format(**coord)
'Coordinates: 37.24N, -115.81W'
```

Acessando atributos dos argumentos:

```
>>> c = 3-5j
>>> ('The complex number {0} is formed from the real part {0.rea
...  'and the imaginary part {0.imag}.').format(c)
'The complex number (3-5j) is formed from the real part 3.0 and
>>> class Point:
...     def __init__(self, x, y):
...         self.x, self.y = x, y
...     def __str__(self):
...         return 'Point({self.x}, {self.y})'.format(self=self)
...
>>> str(Point(4, 2))
'Point(4, 2)'
```

Acessando itens dos argumentos:

```
>>> coord = (3, 5)
>>> 'X: {0[0]};  Y: {0[1]}'.format(coord)
'X: 3;  Y: 5'
```

Substituindo `%s` e `%r`:

```
>>> "repr() mostra aspas: {!r}; str() não mostra: {!s}".format('
"repr() mostra aspas: 'teste1'; str() não mostra: teste2"
```

Alinhando o texto e especificando uma largura:

```
>>> '{:<30}'.format('alinhado à esquerda')
'alinhado à esquerda           '
>>> '{:>30}'.format('alinhado à direita')
'            alinhado à direita'
>>> '{:^30}'.format('centralizado')
'         centralizado         '
>>> '{:*^30}'.format('centralizado')  # usa '*' como preenchimer
'*********centralizado*********'
```

Substituindo `%+f`, `%-f` e `% f` e especificando um sinal:

```
>>> '{:+f}; {:+f}'.format(3.14, -3.14)  # mostra sempre
'+3.140000; -3.140000'
>>> '{: f}; {: f}'.format(3.14, -3.14)  # mostra um espaço para
' 3.140000; -3.140000'
>>> '{:-f}; {:-f}'.format(3.14, -3.14)  # mostra apenas o sinal
'3.140000; -3.140000'
```

Substituindo `%x` e `%o` e convertendo o valor para bases diferentes:

```
>>> # format também tem suporte a números binários
>>> "int: {0:d};  hex: {0:x};  oct: {0:o};  bin: {0:b}".format(4
'int: 42;  hex: 2a;  oct: 52;  bin: 101010'
>>> # com 0x, 0o ou 0b como prefixo:
>>> "int: {0:d};  hex: {0:#x};  oct: {0:#o};  bin: {0:#b}".forma
'int: 42;  hex: 0x2a;  oct: 0o52;  bin: 0b101010'
```

Usando a vírgula ou o sublinhado como separador de grupos de dígitos:

```
>>> '{:,}'.format(1234567890)
'1,234,567,890'
>>> '{:_}'.format(1234567890)
'1_234_567_890'
>>> '{:_b}'.format(1234567890)
'100_1001_1001_0110_0000_0010_1101_0010'
>>> '{:_x}'.format(1234567890)
'4996_02d2'
>>> '{:_}'.format(123456789.123456789)
'123_456_789.12345679'
>>> '{:.,}'.format(123456789.123456789)
'123456789.123,456,79'
>>> '{:,._}'.format(123456789.123456789)
'123,456,789.123_456_79'
```

Expressando uma porcentagem:

```
>>> points = 19
>>> total = 22
```

```
>>> 'Correct answers: {:.2%}'.format(points/total)
'Correct answers: 86.36%'
```

Usando formatação específica do tipo:

```
>>> import datetime as dt
>>> d = dt.datetime(2010, 7, 4, 12, 15, 58)
>>> '{:%Y-%m-%d %H:%M:%S}'.format(d)
'2010-07-04 12:15:58'
```

Aninhando argumentos e exemplos mais complexos:

```
>>> for align, text in zip('<^>', ['left', 'center', 'right']):
...     '{0:{fill}{align}16}'.format(text, fill=align, align=ali
...
'left<<<<<<<<<<<<'
'^^^^^center^^^^^'
'>>>>>>>>>>>>right'
>>>
>>> octets = [192, 168, 0, 1]
>>> '{:02X}{:02X}{:02X}{:02X}'.format(*octets)
'C0A80001'
>>> int(_, 16)
3232235521
>>>
>>> width = 5
>>> for num in range(5,12):
...     for base in 'dXob':
...         print('{0:{width}{base}}'.format(num, base=base, wic
...     print()
...
    5     5     5   101
    6     6     6   110
    7     7     7   111
    8     8    10  1000
    9     9    11  1001
   10     A    12  1010
   11     B    13  1011
```

Strings template ($-strings)

Nota

O recurso descrito aqui foi introduzido no Python 2.4; um método de modelagem simples baseado em expressões regulares. Ele é anterior a `str.format()`, literais de strings formatadas e literais de strings templates.

Não tem relação com os literais de string template (t-strings), introduzidos no Python 3.14. Estes são avaliados como objetos da classe `string.templatelib.Template`, encontrados no módulo `string.templatelib`.

Strings template fornecem substituições de string mais simples, conforme descrito em **PEP 292** [https://peps.python.org/pep-0292/]. Um caso de uso primário para strings template é a internacionalização (i18n), uma vez que, nesse contexto, a sintaxe e a funcionalidade mais simples tornam mais fácil traduzir do que outros recursos embutidos de formatação de strings no Python. Como um exemplo de biblioteca construída sobre strings template para i18n, veja o pacote flufl.i18n [https://flufli18n.readthedocs.io/en/latest/].

Strings template oferecem suporte a substituições baseadas em `$`, usando as seguintes regras:

- `$$` é um escape; é substituído por um único `$`.
- `$identifier` nomeia um espaço reservado de substituição correspondente à chave de mapeamento `"identifier"`. Por padrão, `"identifier"` é restrito a qualquer string ASCII alfanumérica que não distingue maiúsculas e minúsculas (incluindo sublinhados) e que começa com um sublinhado ou com uma letra ASCII. O primeiro caractere não identificador após o caractere `$` termina esta especificação de espaço reservado.
- `${identifier}` é equivalente a `$identifier`. É necessário quando caracteres identificadores válidos seguem o marcador de posição, mas não fazem parte do marcador, como `"${noun}ification"`.

Qualquer outra ocorrência de `$` na string resultará em uma `ValueError` sendo levantada.

O módulo **`string`** fornece uma classe `Template` que implementa essas regras. Os métodos de **`Template`** são:

class string.Template(*template*)

O construtor recebe um único argumento que é a string template

substitute(*mapping={}*, */*, ***kwds*)

Executa a substituição do modelo, retornando uma nova string. *mapping* é qualquer objeto dicionário ou similar com chaves que correspondem aos marcadores de posição no modelo. Como alternativa, você pode fornecer argumentos nomeados, os quais são espaços reservados. Quando *mapping* e *kwds* são fornecidos e há duplicatas, os marcadores de *kwds* têm precedência.

safe_substitute(*mapping={}*, */*, ***kwds*)

Como `substitute()`, exceto que se os espaços reservados estiverem faltando em *mapping* e *kwds*, em vez de levantar uma exceção `KeyError`, o espaço reservado original aparecerá na string resultante intacta. Além disso, ao contrário de **`substitute()`**, qualquer outra ocorrência de `$` simplesmente retornará `$` em vez de levantar `ValueError`.

Embora outras exceções ainda possam ocorrer, esse método é chamado de “seguro” porque sempre tenta retornar uma string utilizável em vez de levantar uma exceção. Em outro sentido, `safe_substitute()` pode ser qualquer coisa diferente de seguro, uma vez que irá ignorar silenciosamente modelos malformados contendo delimitadores pendentes, chaves não correspondidas ou espaços reservados que não são identificadores Python válidos.

is_valid()

Retorna `False` se o modelo tiver espaços reservados inválidos que farão com que `substitute()` levante `ValueError`.

Adicionado na versão 3.11.

get_identifiers()

Retorna uma lista dos identificadores válidos no modelo, na ordem em que aparecem pela primeira vez, ignorando quaisquer identificadores inválidos.

Adicionado na versão 3.11.

Instâncias de **`Template`** também fornecem um atributo de dados públicos:

template

Este é o objeto passado para o argumento *template* do construtor. Em geral, você não deve alterá-lo, mas o acesso somente leitura não é obrigatório.

Aqui está um exemplo de como usar uma instância de Template:

```
>>> from string import Template
>>> s = Template('$who likes $what')
>>> s.substitute(who='tim', what='kung pao')
'tim likes kung pao'
>>> d = dict(who='tim')
>>> Template('Give $who $100').substitute(d)
Traceback (most recent call last):
...
ValueError: Invalid placeholder in string: line 1, col 11
>>> Template('$who likes $what').substitute(d)
Traceback (most recent call last):
...
KeyError: 'what'
>>> Template('$who likes $what').safe_substitute(d)
'tim likes $what'
```

Uso avançado: você pode derivar subclasses de **`Template`** para personalizar a sintaxe do espaço reservado, caractere delimitador ou toda a expressão regular usada para analisar strings template. Para fazer isso, você pode substituir estes atributos de classe:

- *delimiter* – Este é a string literal que descreve um delimitador de introdução do espaço reservado. O valor padrão é `$`. Note que esta *não* deve ser uma expressão regular, já que a implementação irá chamar `re.escape()` nesta string conforme necessário. Observe também que você não pode alterar o delimitador após a criação da classe (ou seja, um delimitador diferente deve ser definido no espaço de nomes da classe da subclasse).

- *idpattern* – Esta é a expressão regular que descreve o padrão para espaço reservado sem envolto em chaves. O valor padrão é a expressão regular `(?a:[_a-z][_a-z0-9]*)`. Se for fornecido e *braceidpattern* for `None`, esse padrão também se aplicará o espaço reservado com chaves.

  Nota

  Uma vez que *flags* padrão é `re.IGNORECASE`, o padrão `[a-z]` pode corresponder a alguns caracteres não ASCII. É por isso que usamos o sinalizador local `a` aqui.

  Alterado na versão 3.7: braceidpattern pode ser usado para definir padrões separados usados dentro e fora das chaves.

- *braceidpattern* – É como *idpattern*, mas descreve o padrão para espaços reservados com chaves. O padrão é `None`, o que significa recorrer a *idpattern* (ou seja, o mesmo padrão é usado dentro e fora das chaves). Se fornecido, permite definir padrões diferentes para espaço reservado com e sem chaves.

  Adicionado na versão 3.7.

- *flags* – Os sinalizadores de expressão regular que serão aplicados ao compilar a expressão regular usada para reconhecer substituições. O valor padrão é `re.IGNORECASE`. Note que `re.VERBOSE` sempre será adicionado aos sinalizadores, então *idpattern*s personalizados devem seguir as convenções para expressões regulares verbosas.

  Adicionado na versão 3.2.

Como alternativa, você pode fornecer todo o padrão de expressão regular substituindo o atributo *pattern* de classe. Se você fizer isso, o valor deve ser uma string de padrão de expressão regular, ou um objeto compilado de expressão regular, com quatro grupos de captura nomeados. Os grupos de captura correspondem às regras fornecidas acima, junto com a regra inválida do espaço reservado:

- *escaped* – Este grupo corresponde à sequência de escape, por exemplo `$$`, no padrão.
- *named* – Este grupo corresponde ao nome do espaço reservado sem chaves; não deve incluir o delimitador no grupo de captura.
- *braced* – Este grupo corresponde ao nome do espaço reservado entre chaves; ele não deve incluir o delimitador ou chaves no grupo de captura.
- *invalid* – Esse grupo corresponde a qualquer outro padrão de delimitador (geralmente um único delimitador) e deve aparecer por último na expressão regular.

Os métodos nesta classe irão levantar `ValueError` se o padrão corresponder ao modelo sem que um desses grupos nomeados corresponda.

Funções auxiliares

string.capwords(*s*, *sep=None*)

Divide o argumento em palavras usando `str.split()`, coloca cada palavra em maiúscula usando `str.capitalize()`, e junte as palavras em maiúsculas usando `str.join()`. Se o segundo argumento opcional *sep* estiver ausente ou `None`, os caracteres de espaço em branco são substituídos por um único espaço e os espaços em branco à esquerda e à direita são removidos, caso contrário *sep* é usado para dividir e unir as palavras.

`string.templatelib` — Suporte para literais de string template

Código-fonte: Lib/string/templatelib.py
[https://github.com/python/cpython/tree/3.15/Lib/string/templatelib.py]

Ver também

- Strings de formatação
- Sintaxe de literal de string template (t-string)
- **PEP 750** [https://peps.python.org/pep-0750/]

Strings template

Adicionado na versão 3.14.

Strings template são um mecanismo para processamento personalizado de strings. Elas têm toda a flexibilidade das Literais de strings formatadas do Python, mas retornam uma instância `Template` que dá acesso às partes estáticas e interpoladas (entre chaves) de uma string *antes* de serem combinadas.

Para escrever uma t-string, use um prefixo `'t'` em vez de `'f'`, assim:

```
>>> pi = 3.14
>>> t't-strings são novas no Python {pi!s}!'
Template(
   strings=('t-strings são novas no Python ', '!'),
   interpolations=(Interpolation(3.14, 'pi', 's', ''),)
)
```

Tipos

class string.templatelib.Template

A classe **`Template`** descreve o conteúdo de uma string template. Ela é imutável, o que significa que os atributos de um template não podem ser reatribuídos.

A maneira mais comum de criar uma instância de **`Template`** é usar a sintaxe de literal de string template. Essa sintaxe é idêntica à de f-strings, exceto pelo fato de usar um prefixo `t` no lugar de um `f`:

```
>>> cheese = 'Red Leicester'
>>> template = t"We're fresh out of {cheese}, sir."
>>> type(template)
<class 'string.templatelib.Template'>
```

Os templates são armazenados como sequências de literais **`strings`** e dinâmicos **`interpolations`**. Um atributo **`values`** contém os valores das interpolações:

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
>>> template.strings
('Ah! We do have ', '.')
>>> template.interpolations
(Interpolation('Camembert', ...),)
>>> template.values
('Camembert',)
```

A tupla **`strings`** tem um elemento a mais que **`interpolations`** e **`values`**; as interpolações "pertencem" às strings. Isso pode ser mais fácil de entender quando as tuplas estão alinhadas

```
template.strings:  ('Ah! We do have ',              '.')
template.values:   (                   'Camembert',    )
```

Atributos

strings*: tuple[str, ...]*

Uma **`tuple`** das strings estáticas no template.

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
```

```
>>> template.strings
('Ah! We do have ', '.')
```

Strings vazias *estão* incluídas na tupla:

```
>>> response = 'We do have '
>>> cheese = 'Camembert'
>>> template = t'Ah! {response}{cheese}.'
>>> template.strings
('Ah! ', '', '.')
```

A tupla `strings` nunca está vazia e sempre contém uma string a mais que as tuplas `interpolations` e `values`:

```
>>> t''.strings
('',)
>>> t''.values
()
>>> t'{'cheese'}'.strings
('', '')
>>> t'{'cheese'}'.values
('cheese',)
```

interpolations*: tuple[Interpolation, ...]*

Uma **`tuple`** das interpolações no template.

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
>>> template.interpolations
(Interpolation('Camembert', 'cheese', None, ''),)
```

A tupla `interpolações` pode estar vazia e sempre contém um valor a menos que a tupla `strings`:

```
>>> t'Red Leicester'.interpolations
()
```

values*: tuple[object, ...]*

Uma tupla de todos os valores interpolados no template.

```
>>> cheese = 'Camembert'
>>> template = t'Ah! We do have {cheese}.'
```

```
>>> template.values
('Camembert',)
```

A tupla `values` sempre tem o mesmo comprimento que a tupla `interpolations`. Isso sempre equivale a `tuple(i.value for i in template.interpolations)`.

Métodos

__new__(**args:* *str* | *Interpolation*)

Embora a sintaxe de literal seja a maneira mais comum de criar um **`Template`**, também é possível criá-las diretamente usando o construtor:

```
>>> from string.templatelib import Interpolation, Template
>>> cheese = 'Camembert'
>>> template = Template(
...     'Ah! We do have ', Interpolation(cheese, 'cheese'),
... )
>>> list(template)
['Ah! We do have ', Interpolation('Camembert', 'cheese', No
```

Se várias strings forem passadas consecutivamente, elas serão concatenadas em um único valor no atributo **`strings`**. Por exemplo, o código a seguir cria uma **`Template`** com uma única string final:

```
>>> from string.templatelib import Template
>>> template = Template('Ah! We do have ', 'Camembert', '.'
>>> template.strings
('Ah! We do have Camembert.',)
```

Se várias interpolações forem passadas consecutivamente, elas serão tratadas como interpolações separadas e uma string vazia será inserida entre elas. Por exemplo, o código a seguir cria um modelo com espaços reservados vazios no atributo **`strings`**:

```
>>> from string.templatelib import Interpolation, Template
>>> template = Template(
...     Interpolation('Camembert', 'cheese'),
...     Interpolation('.', 'punctuation'),
```

```
... )
>>> template.strings
('', '', '')
```

iter(template)

Itera sobre o template, produzindo cada string não-vazia e **`Interpolation`** em ordem correta.

```
>>> cheese = 'Camembert'
>>> list(t'Ah! We do have {cheese}.')
['Ah! We do have ', Interpolation('Camembert', 'cheese', No
```

Cuidado

Strings vazias *não* são incluídas na iteração:

```
>>> response = 'We do have '
>>> cheese = 'Camembert'
>>> list(t'Ah! {response}{cheese}.')
['Ah! ',
 Interpolation('We do have ', 'response', None, ''),
 Interpolation('Camembert', 'cheese', None, ''),
 '.']
```

template + other

template += other

Concatena este template com outro, retornando uma nova instância **`Template`**:

```
>>> cheese = 'Camembert'
>>> list(t'Ah! ' + t'We do have {cheese}.')
['Ah! We do have ', Interpolation('Camembert', 'cheese', No
```

A concatenação de uma **`Template`** e uma `str` **não** é suportada. Isso ocorre porque não está claro se a string deve ser tratada como uma string estática ou uma interpolação. Se você quiser concatenar uma **`Template`** com uma string, você deve encapsular a string diretamente em uma **`Template`** (para tratá-la como uma string estática) ou usar uma **`Interpolation`** (para tratá-la como dinâmica):

```
>>> from string.templatelib import Interpolation, Template
>>> template = t'Ah! '
>>> # Treat 'We do have ' as a static string
>>> template += Template('We do have ')
>>> # Treat cheese as an interpolation
>>> cheese = 'Camembert'
>>> template += Template(Interpolation(cheese, 'cheese'))
>>> list(template)
['Ah! We do have ', Interpolation('Camembert', 'cheese', Nc
```

class string.templatelib.Interpolation

O tipo **`Interpolation`** representa uma expressão dentro de uma string template. É imutável, o que significa que os atributos de uma interpolação não podem ser reatribuídos.

As interpolações oferecem suporte à correspondência de padrões, permitindo que você compare seus atributos com a instrução match:

```
>>> from string.templatelib import Interpolation
>>> interpolation = t'{1. + 2.:.2f}'.interpolations[0]
>>> interpolation
Interpolation(3.0, '1. + 2.', None, '.2f')
>>> match interpolation:
...     case Interpolation(value, expression, conversion, form
...         print(value, expression, conversion, format_spec,
...
3.0 | 1. + 2. | None | .2f
```

As interpolações são genéricas em relação aos tipos dos seus valores.

Atributos

value*: object*

O valor avaliado da interpolação.

```
>>> t'{1 + 2}'.interpolations[0].value
3
```

expression*: str*

Para interpolações criadas por literais de t-string, **`expression`** é o texto da expressão encontrado dentro das chaves (`{` e `}`), incluindo quaisquer espaços em branco, excluindo as próprias chaves e terminando antes do primeiro `!`, `:` ou `=`, se houver algum. Para interpolações criadas manualmente, **`expression`** é a string arbitrária fornecida ao construir a instância de interpolação.

Recomendamos usar expressões Python válidas ou a string vazia para o campo `expression` de instâncias **`Interpolation`** criadas manualmente, embora isso não seja imposto em tempo de execução.

```
>>> t'{1 + 2}'.interpolations[0].expression
'1 + 2'
```

conversion*: Literal['a', 'r', 's'] | None*

A conversão a ser aplicada ao valor, ou `None`.

O **`conversion`** é a conversão opcional a ser aplicada ao valor:

```
>>> t'{1 + 2!a}'.interpolations[0].conversion
'a'
```

Nota

Ao contrário das f-strings, onde as conversões são aplicadas automaticamente, o comportamento esperado com t-strings é que o código que *processa* o **`Template`** decidirá como interpretar e se aplicará o **`conversion`**. Por conveniência, a função **`convert()`** pode ser usada para imitar a semântica de conversão de f-strings.

format_spec*: str*

A especificação de formato a ser aplicada ao valor.

O **`format_spec`** é uma string arbitrária opcional usada como especificação de formato para apresentar o valor:

```
>>> t'{1 + 2:.2f}'.interpolations[0].format_spec
'.2f'
```

Nota

Ao contrário das f-strings, onde as especificações de formato são aplicadas automaticamente por meio do protocolo `format()`, o comportamento esperado com t-strings é que o código que *processa* a interpolação decida como interpretar e se a especificação de formato deve ser aplicada. Como resultado, os valores `format_spec` em interpolações podem ser strings arbitrárias, incluindo aquelas que não estão em conformidade com o protocolo de `format()`.

Métodos

__new__(*value: object, expression: str, conversion: Literal['a', 'r', 's'] | None = None, format_spec: str = ''*)

Cria um novo objeto `Interpolation` a partir de partes do componente.

Parâmetros::

- **value** – O resultado avaliado e dentro do escopo da interpolação.
- **expression** – O texto de uma expressão Pythoı válida ou uma string vazia.
- **conversion** – A conversão a ser usada, uma entre `None`, `'a'`, `'r'` ou `'s'`.
- **format_spec** – Uma string arbitrária opcional usada como especificação de formato para apresentar o valor.

Funções auxiliares

string.templatelib.convert(*obj*, /, *conversion*)

Aplica a semântica de conversão de literal de string formatada ao objeto *obj* fornecido. Isso é frequentemente útil para lógica de processamento de string template personalizada.

Atualmente, há três sinalizadores de conversão suportados:

- `'s'` que chama **`str()`** no valor (como `!s`),
- `'r'` que chama **`repr()`** (como `!r`) e
- `'a'` que chama **`ascii()`** (como `!a`).

Se o sinalizador de conversão é `None`, *obj* é retornado sem alterações.

`re` — Operações com expressões regulares

Código-fonte: Lib/re/ [https://github.com/python/cpython/tree/3.15/Lib/re/]

Este módulo fornece operações para correspondência de expressões regulares semelhantes às encontradas em Perl. O nome do módulo vem das iniciais do termo em inglês *regular expressions*, também frequentemente chamadas de *regex*.

Tanto os padrões quanto as strings a serem pesquisados podem ser strings Unicode (`str`) assim como strings de 8 bits (`bytes`). No entanto, strings Unicode e strings de 8 bits não podem ser misturadas: ou seja, você não pode corresponder uma string Unicode com um padrão de bytes ou vice-versa; da mesma forma, ao solicitar uma substituição, a string de substituição deve ser do mesmo tipo que o padrão e a string de pesquisa.

Expressões regulares usam o caractere de contrabarra (`'\'`) para indicar formas especiais ou para permitir que caracteres especiais sejam usados sem invocar seu significado especial. Isso colide com o uso em Python do mesmo caractere para o mesmo propósito em literais de string; por exemplo, para corresponder a uma contrabarra literal, pode-se ter que escrever `'\\\\'` como a string do padrão, porque a expressão regular deve ser `\\`, e cada contrabarra deve ser expressa como `\\` dentro de um literal de string Python regular. Além disso, observe que quaisquer sequências de escape inválidas no uso do Python da contrabarra em literais de string agora levantam uma exceção `SyntaxWarning` e no futuro isso se tornará um `SyntaxError`. Esse comportamento acontecerá mesmo se for uma sequência de escape válida para uma expressão regular.

A solução é usar a notação de string bruta do Python para padrões de expressão regular; as contrabarras não são tratadas de nenhuma maneira especial em uma string literal com o prefixo `'r'`. Portanto, `r"\n"` é uma

string de dois caracteres contendo `'\'` e `'n'`, enquanto `"\n"` é uma string de um caractere contendo uma nova linha. Normalmente, os padrões serão expressos em código Python usando esta notação de string bruta.

É importante notar que a maioria das operações de expressão regular estão disponíveis como funções e métodos em nível de módulo em expressões regulares compiladas. As funções são atalhos que não exigem que você compile um objeto de expressão regular primeiro, mas perdem-se alguns parâmetros de ajuste fino.

Ver também

O módulo de terceiros regex [https://pypi.org/project/regex/] possui uma API compatível com o módulo da biblioteca padrão **`re`**, mas oferece funcionalidades adicionais e um suporte mais completo a Unicode.

Sintaxe de expressão regular

Uma expressão regular (ou ER) especifica um conjunto de strings que corresponde a ela; as funções neste módulo permitem que você verifique se uma determinada string corresponde a uma determinada expressão regular (ou se uma determinada expressão regular corresponde a uma determinada string, o que resulta na mesma coisa).

As expressões regulares podem ser concatenadas para formar novas expressões regulares; se *A* e *B* forem expressões regulares, então *AB* também será uma expressão regular. Em geral, se uma string *p* corresponder a *A* e outra string *q* corresponder a *B*, a string *pq* corresponderá a AB. Isso é válido, a menos que *A* ou *B* contenham operações de baixa precedência; condições de contorno entre *A* e *B*; ou ter referências de grupo numeradas. Assim, expressões complexas podem ser facilmente construídas a partir de expressões primitivas mais simples, como as descritas aqui. Para obter detalhes sobre a teoria e implementação de expressões regulares, consulte o livro de Friedl [Frie09], ou quase qualquer livro sobre construção de compiladores.

Segue uma breve explicação do formato das expressões regulares. Para mais informações e uma apresentação mais suave, consulte o tutorial de Regular expression HOWTO.

As expressões regulares podem conter caracteres especiais e comuns. A maioria dos caracteres comuns, como `'A'`, `'a'` ou `'0'`, são as expressões regulares mais simples; eles simplesmente se correspondem. Você pode concatenar caracteres comuns, de forma que `último` corresponda à string `'último'`. (No restante desta seção, escreveremos ERs `neste estilo especial`, geralmente sem aspas, e strings para serem correspondidas `'entre aspas simples'`.)

Alguns caracteres, como `'|'` ou `'('`, são especiais. Os caracteres especiais representam classes de caracteres comuns ou afetam como as expressões regulares em torno deles são interpretadas.

Operadores de repetição ou quantificadores (`*`, `+`, `?`, `{m,n}` etc) não podem ser aninhados diretamente. Isso evita ambiguidade com o sufixo modificador não guloso `?`, e com outros modificadores em outras implementações. Para aplicar uma segunda repetição a uma repetição interna, podem ser usados parênteses. Por exemplo, a expressão `(?:a{6})*` corresponde a qualquer múltiplo de seis caracteres `'a'`.

Os caracteres especiais são:

`.`

(Ponto.) No modo padrão, corresponde a qualquer caractere, exceto uma nova linha. Se o sinalizador **`DOTALL`** foi especificado, ele corresponde a qualquer caractere, incluindo uma nova linha. `(?s:.)` corresponde a qualquer caractere independente de sinalizadores.

`^`

(Sinal de circunflexo.) Corresponde ao início da string, e no modo **`MULTILINE`** também corresponde imediatamente após cada nova linha.

`$`

Corresponde ao final da string ou logo antes da nova linha no final da string, e no modo **`MULTILINE`** também corresponde antes de uma nova

linha. `foo` corresponde a ‘foo’ e ‘foobar’, enquanto a expressão regular `foo$` corresponde apenas a ‘foo’. Mais interessante, pesquisar por `foo.$` em `'foo1\nfoo2\n'` corresponde a ‘foo2’ normalmente, mas ‘foo1’ no modo **`MULTILINE`**; procurando por um único `$` em `'foo\n'` encontrará duas correspondências (vazias): uma logo antes da nova linha e uma no final da string.

`*`

Faz a ER resultante corresponder a 0 ou mais repetições da ER anterior, tantas repetições quantas forem possíveis. `ab*` corresponderá a ‘a’, ‘ab’ ou ‘a’ seguido por uma quantidade qualquer de ‘b’s.

`+`

Faz com que a ER resultante corresponda a 1 ou mais repetições da ER anterior. `ab+` irá corresponder a ‘a’ seguido por qualquer número diferente de zero de ‘b’s; não corresponderá apenas a ‘a’.

`?`

Faz a ER resultante corresponder a 0 ou 1 repetição da ER anterior. `ab?` irá corresponder a ‘a’ ou ‘ab’.

`*?`, `+?`, `??`

Os quantificadores `'*'`, `'+'` e `'?'` são todos *gulosos*; eles correspondem ao máximo de texto possível. Às vezes, esse comportamento não é desejado; se a ER `<.*>` for correspondida com `'<a> b <c>'`, ela irá corresponder a toda a string, e não apenas `'<a>'`. Adicionar `?` após o quantificador faz com que ele execute a correspondência da maneira *não gulosa* ou *minimalista*; tão *poucos* caracteres quanto possível serão correspondidos. Usando a ER `<.*?>` irá corresponder apenas `'<a>'`.

`*+`, `++`, `?+`

Como os quantificadores `'*'`, `'+'` e `'?'`, aqueles em que `'+'` é anexado também correspondem tantas vezes quanto possível. No entanto, ao contrário dos quantificadores realmente gulosos, eles não permitem retrocesso quando a expressão que o segue não corresponde. Estes são conhecidos como quantificadores *possessivos*. Por exemplo, `a*a` corresponderá a `'aaaa'` porque o `a*` corresponderá a todos os 4 `'a'`s, mas, quando o final `'a'` for encontrado, a expressão é retrocedida para

que no final o `a*` acabe correspondendo a 3 `'a'`s no total, e o quarto `'a'` seja correspondido por o final `'a'`. No entanto, quando `a*+a` é usado para corresponder a `'aaaa'`, o `a*+` corresponderá a todos os 4 `'a'`, mas quando o `'a'` final não encontrar mais caracteres para corresponder, a expressão não pode ser retrocedida e, portanto, não corresponderá. `x*+`, `x++` e `x?+` são equivalentes a `(?>x*)`, `(?>x+)` e `(?>x?)` respectivamente.

Adicionado na versão 3.11.

`{m}`

Especifica que exatamente *m* cópias da ER anterior devem ser correspondidas; menos correspondências fazem com que toda a ER não seja correspondida. Por exemplo, `a{6}` irá corresponder exatamente a seis caracteres `'a'`, mas não a cinco.

`{m,n}`

Faz a ER resultante corresponder de *m* a *n* repetições da ER anterior, tentando corresponder ao máximo de repetições possível. Por exemplo, `a{3,5}` irá corresponder de 3 a 5 caracteres `'a'`. A omissão de *m* especifica um limite inferior de zero e a omissão de *n* especifica um limite superior infinito. Como exemplo, `a{4,}b` irá corresponder a `'aaaab'` ou mil caracteres `'a'` seguidos por um `'b'`, mas não `'aaab'`. A vírgula não pode ser omitida ou o modificador será confundido com a forma descrita anteriormente.

`{m,n}?`

Faz a ER resultante corresponder de *m* a *n* repetições da ER anterior, tentando corresponder o *mínimo* de repetições possível. Esta é a versão não gulosa do quantificador anterior. Por exemplo, na string de 6 caracteres `'aaaaaa'`, `a{3,5}` irá corresponder a 5 caracteres `'a'`, enquanto `a{3,5}?` corresponderá apenas a 3 caracteres.

`{m,n}+`

Faz a ER resultante corresponder de *m* a *n* repetições da ER anterior, tentando corresponder o maior número possível de repetições *sem* estabelecer nenhum ponto de retrocesso. Esta é a versão possessiva do quantificador acima. Por exemplo, na string de 6 caracteres `'aaaaaa'`,

`a{3,5}+aa` tenta corresponder 5 caracteres `'a'`, então, requerer mais 2 `'a'`s, precisará de mais caracteres do que os disponíveis e, portanto, falhará, enquanto `a{3,5}aa` corresponderá a `a{3,5}` capturando 5, depois 4 `'a'`s por retrocesso e então os 2 `'a'`s finais são combinados com o `aa` final no padrão. `x{m,n}+` é equivalente a `(?>x{m,n})`.

Adicionado na versão 3.11.

`\`

Ou anula o efeito de caracteres especiais (permitindo a você corresponder caracteres como `'*'`, `'?'` e assim por diante), ou sinaliza uma sequência especial; sequências especiais são discutidas abaixo.

Se você não estiver usando uma string bruta para expressar o padrão, lembre-se de que o Python também usa a contrabarra como uma sequência de escape em literais de string; se a sequência de escape não for reconhecida pelo analisador sintático do Python, a contrabarra e o caractere subsequente serão incluídos na string resultante. No entanto, se Python reconhecer a sequência resultante, a contrabarra deve ser repetida duas vezes. Isso é complicado e difícil de entender, portanto, é altamente recomendável que você use strings brutas para todas as expressões, exceto as mais simples.

`[]`

Usado para indicar um conjunto de caracteres. Em um conjunto:

- Caracteres podem ser listados individualmente, por exemplo, `[amk]` vai corresponder a `'a'`, `'m'` ou `'k'`.
- Intervalos de caracteres podem ser indicados fornecendo dois caracteres e separando-os por `'-'`, por exemplo `[a-z]` irá corresponder a qualquer letra ASCII minúscula, `[0-5][0-9]` irá corresponder a todos os números de dois dígitos de `00` a `59`, e `[0-9A-Fa-f]` irá corresponder a qualquer dígito hexadecimal. Se `-` for precedido de uma contrabarra (por exemplo, `[a\-z]`) ou se for colocado como o primeiro ou último caractere (por exemplo, `[-a]` ou `[a-]`), ele corresponderá a um literal `'-'`.

- Os caracteres especiais, exceto contrabarra, perdem seu significado especial dentro dos conjuntos. Por exemplo, `[(+*)]` vai corresponder a qualquer um dos caracteres literais `'('`, `'+'`, `'*'` ou `')'`.
- A contrabarra escapa caracteres que têm significado especial em um conjunto, como `'-'`, `']'`, `'^'` e `'\\'`, ou sinaliza uma sequência especial que representa um único caractere, como `\xa0` ou `\n`, ou uma classe de caracteres como `\w` ou `\S` (definida abaixo). Observe que `\b` representa um único caractere de "backspace", não um limite de palavra como fora de um conjunto, e escapes numéricos como `\1` são sempre escapes octais, não referências de grupo. Sequências especiais que não correspondem a um único caractere, como `\A` e `\Z`, não são permitidas.
- Os caracteres que não estão dentro de um intervalo podem ser correspondidos *complementando* o conjunto. Se o primeiro caractere do conjunto for `'^'`, todos os caracteres que *não* estiverem no conjunto serão correspondidos. Por exemplo, `[^5]` irá corresponder a qualquer caractere exceto `'5'`, e `[^^]` irá corresponder a qualquer caractere exceto `'^'`. `^` não tem nenhum significado especial se não for o primeiro caractere do conjunto.
- Para corresponder a um `']'` literal dentro de um conjunto, preceda-o com uma contrabarra ou coloque-o no início do conjunto. Por exemplo, `[()[\]{}]` e `[]()[{}]` vai corresponder ao colchete à direita, assim como o colchete, chave ou parêntese à esquerda.
- Suporte para conjuntos aninhados e operações de conjunto como no Padrão Técnico do Unicode #18 [https://unicode.org/reports/tr18/] podem ser adicionados no futuro. Isso mudaria a sintaxe, então para facilitar essa mudança uma `FutureWarning` será levantada em casos ambíguos por enquanto. Isso inclui conjuntos que começam com um `'['` literal ou contendo sequências de caracteres literais `'--'`, `'&&'`, `'~~'` e `'||'`. Para evitar um aviso, use uma contrabarra antes deles.

Alterado na versão 3.7: `FutureWarning` é levantada se um conjunto de caracteres contém construções que mudarão semanticamente no futuro.

`|`

`A|B`, onde *A* e *B* podem ser ERs arbitrárias, cria uma expressão regular que corresponderá a *A* ou *B*. Um número arbitrário de ERs pode ser separado por `'|'` desta forma. Isso também pode ser usado dentro de grupos (veja abaixo). Conforme a string alvo é percorrida, ERs separadas por `'|'` são avaliadas da esquerda para a direita. Quando um padrão corresponde completamente, essa ramificação é aceita. Isso significa que, assim que *A* corresponder, *B* não será avaliado posteriormente, mesmo que produza uma correspondência geral mais longa. Em outras palavras, o operador `'|'` nunca é guloso. Para corresponder a um `'|'` literal, use `\|`, ou coloque-o dentro de uma classe de caractere, como em `[|]`.

`(...)`

Corresponde a qualquer expressão regular que esteja entre parênteses e indica o início e o fim de um grupo; o conteúdo de um grupo pode ser recuperado após uma correspondência ter sido realizada e pode ser correspondido posteriormente na string com a sequência especial `\número`, descrita abaixo. Para corresponder aos literais `'('` ou `')'`, use `\(` ou `\)`, ou coloque-os dentro de uma classe de caracteres: `[(]`, `[)]`.

`(?...)`

Esta é uma notação de extensão (um `'?'` após um `'('` não é significativo de outra forma). O primeiro caractere após o `'?'` determina qual o significado e sintaxe posterior da construção. As extensões normalmente não criam um novo grupo; `(?P<nome>...)` é a única exceção a esta regra. A seguir estão as extensões atualmente implementadas.

`(?aiLmsux)`

(Uma ou mais letras do conjunto `'a'`, `'i'`, `'L'`, `'m'`, `'s'`, `'u'`, `'x'`.) O grupo corresponde à string vazia; as letras definem os sinalizadores correspondentes para toda a expressão regular:

- `re.A` (correspondência somente ASCII)
- `re.I` (ignorar maiúsculas/minúsculas)
- `re.L` (dependente da localidade)

- `re.M` (multi-linha)
- `re.S` (ponto corresponde a tudo)
- `re.U` (correspondência Unicode)
- `re.X` (verboso)

(Os sinalizadores são descritos em Conteúdo do módulo.) Isso é útil se você deseja incluir os sinalizadores como parte da expressão regular, em vez de passar um argumento *flag* para a função `re.compile()`. Os sinalizadores devem ser usados primeiro na string de expressão.

Alterado na versão 3.11: Esta construção só pode ser usada no início da expressão.

`(?:...)`

Uma versão de não-captura de parênteses regulares. Corresponde a qualquer expressão regular que esteja entre parênteses, mas a substring correspondida pelo grupo *não pode* ser recuperada após realizar uma correspondência ou referenciada posteriormente no padrão.

`(?aiLmsux-imsx:...)`

(Zero ou mais letras do conjunto `'a'`, `'i'`, `'L'`, `'m'`, `'s'`, `'u'`, `'x'`, opcionalmente seguidas por `'-'` seguido por uma ou mais letras do conjunto `'i'`, `'m'`, `'s'`, `'x'`.) O conjunto de letras define ou remove os sinalizadores correspondentes para a parte da expressão:

- `re.A` (correspondência somente ASCII)
- `re.I` (ignorar maiúsculas/minúsculas)
- `re.L` (dependente da localidade)
- `re.M` (multi-linha)
- `re.S` (ponto corresponde a tudo)
- `re.U` (correspondência Unicode)
- `re.X` (verboso)

(Os sinalizadores são descritos em Conteúdo do módulo.)

As letras `'a'`, `'L'` e `'u'` são mutuamente exclusivas quando usadas como sinalizadores em linha, portanto, não podem ser combinadas ou seguir `'-'`. Em vez disso, quando um deles aparece em um grupo embutido, ele substitui o modo de correspondência no grupo anexo. Em

padrões Unicode `(?a:...)` muda para correspondência somente ASCII, e `(?u:...)` muda para correspondência Unicode (padrão). Em padrões de bytes `(?L:...)` muda para a correspondência dependente da localidade, e `(?a:...)` muda para correspondência apenas ASCII (padrão). Esta substituição só tem efeito para o grupo estreito em linha e o modo de correspondência original é restaurado fora do grupo.

Adicionado na versão 3.6.

Alterado na versão 3.7: As letras `'a'`, `'L'` e `'u'` também podem ser usadas em um grupo.

`(?>...)`

Tenta corresponder `...` como se fosse uma expressão regular separada e, se for bem-sucedida, continua correspondendo ao restante do padrão que a segue. Se o padrão subsequente não corresponder, a pilha só pode ser desenrolada até um ponto *antes* do `(?>...)` porque, uma vez encerrada, a expressão, conhecida como *grupo atômico*, jogou fora todos os pontos de pilha dentro de si. Assim, `(?>.*).` nunca corresponderia a nada porque primeiro o `.*` corresponderia a todos os caracteres possíveis, então, não tendo mais nada para corresponder, o `.` final falharia em corresponder. Como não há pontos de pilha salvos no Grupo Atômico e não há ponto de pilha antes dele, a expressão inteira não corresponderia.

Adicionado na versão 3.11.

`(?P<nome>...)`

Semelhante aos parênteses regulares, mas a substring correspondida pelo grupo é acessível por meio do nome de grupo simbólico *nome*. Os nomes de grupo devem ser identificadores Python válidos e em padrões **`bytes`** eles só podem conter bytes no intervalo ASCII. Cada nome de grupo deve ser definido apenas uma vez em uma expressão regular. Um grupo simbólico também é um grupo numerado, como se o grupo não tivesse um nome.

Grupos nomeados podem ser referenciados em três contextos. Se o padrão for `(?P<citação>['"]).*?(?P=citação)` (ou seja, corresponder

a uma string entre aspas simples ou duplas):

Contexto de referência ao grupo “citação”	Formas de referenciá-lo
no mesmo padrão	• `(?P=citação)` (como mostrado) • `\1`
ao processar a correspondência do objeto *m*	• `m.group('citação')` • `m.end('citação')` (etc.)
em uma string passada para o argumento *repl* de `re.sub()`	• `\g<citação>` • `\g<1>` • `\1`

Alterado na versão 3.12: Nos padrões **bytes**, o grupo *nome* só pode conter bytes no intervalo ASCII (`b'\x00'-b'\x7f'`).

`(?P=nome)`
Uma referência anterior a um grupo nomeado; corresponde a qualquer texto que corresponda ao grupo anterior denominado *nome*.

`(?#...)`
Um comentário; o conteúdo dos parênteses é simplesmente ignorado.

`(?=...)`
Corresponde se `...` corresponder a próxima, mas não consome nada da string. Isso é chamado de *asserção preditiva*. Por exemplo, `Isaac (?=Asimov)` corresponderá a `'Isaac '` apenas se for seguido por `'Asimov'`.

`(?!...)`
Corresponde se `...` não corresponder a próxima. Isso é uma *asserção preditiva negativa*. Por exemplo, `Isaac (?!Asimov)` corresponderá a `'Isaac '` apenas se *não* for seguido por `'Asimov'`.

`(?<=...)`

Corresponde se a posição atual na string for precedida por uma correspondência para `...` que termina na posição atual. Isso é chamado de *asserção retroativa positiva*. `(?<=abc)def` irá encontrar uma correspondência em `'abcdef'`, uma vez que a expressão regular vai voltar 3 caracteres e verificar se o padrão contido corresponde. O padrão contido deve corresponder apenas a strings de algum comprimento fixo, o que significa que `abc` ou `a|b` são permitidos, mas `a*` e `a{3,4}` não são. Observe que os padrões que começam com asserções retroativas positivas não corresponderão ao início da string que está sendo pesquisada; você provavelmente desejará usar a função **`search()`** em vez da função **`match()`**:

```
>>> import re
>>> m = re.search('(?<=abc)def', 'abcdef')
>>> m.group(0)
'def'
```

Este exemplo procura por uma palavra logo após um hífen:

```
>>> m = re.search(r'(?<=-)\w+', 'spam-egg')
>>> m.group(0)
'egg'
```

Alterado na versão 3.5: Adicionado suporte para referências de grupo de comprimento fixo.

`(?<!...)`

Corresponde se a posição atual na string não for precedida por uma correspondência para `...`. Isso é chamado de *asserção retroativa negativa*. Semelhante às asserções retroativas positivas, o padrão contido deve corresponder apenas a strings de algum comprimento fixo. Os padrões que começam com asserções retroativas negativas podem corresponder ao início da string que está sendo pesquisada.

`(?(id/nome)padrão-sim|padrão-não)`

Tentará corresponder com `padrão-sim` se o grupo com determinado *id* ou *nome* existir, e com `padrão-não` se não existir. `padrão-não` é opcional e pode ser omitido. Por exemplo, `(<)?(\w+@\w+(?:\.\w+)+)(?(1)>|$)` é um padrão ruim de correspondência de e-mail, que

corresponde a `'<usuario@host.com>'` e a `'usuario@host.com'`, mas não corresponde a `'<usuario@host.com'` nem `'usuario@host.com>'` totalmente (**`re.search()`** só encontra `'usuario@host.com'` no primeiro caso).

Alterado na versão 3.12: O grupo *id* só podem conter dígitos ASCII. Nos padrões **`bytes`**, o grupo *nome* só pode conter bytes no intervalo ASCII (`b'\x00'-b'\x7f'`).

As sequências especiais consistem em `'\'` e um caractere da lista abaixo. Se o caractere comum não for um dígito ASCII ou uma letra ASCII, a ER resultante corresponderá ao segundo caractere. Por exemplo, `\$` corresponde ao caractere `'$'`.

`\número`

Corresponde ao conteúdo do grupo de mesmo número. Os grupos são numerados a partir de 1. Por exemplo, `(.+) \1` corresponde a `'de de'` ou `'55 55'`, mas não `'dede'` (note o espaço após o grupo). Esta sequência especial só pode ser usada para corresponder a um dos primeiros 99 grupos. Se o primeiro dígito de *número* for 0, ou *número* tiver 3 dígitos octais de comprimento, ele não será interpretado como uma correspondência de grupo, mas como o caractere com *número* de valor octal. Dentro de `'['` e `']'` de uma classe de caracteres, todos os escapes numéricos são tratados como caracteres.

`\A`

Corresponde apenas ao início da string.

`\b`

Corresponde à string vazia, mas apenas no início ou no final de uma palavra. Uma palavra é definida como uma sequência de caracteres de palavras. Observe que, formalmente, `\b` é definido como a fronteira entre um caractere `\w` e um `\W` (ou vice-versa), ou entre `\w` e o início/fim da string. Isso significa que `r'\bat\b'` corresponde a `'at'`, `'at.'`, `'(at)'`, `'as at ay'`, mas não a `'attempt'` ou `'atlas'`.

Os caracteres de palavras predefinidos em padrões Unicode (str) são alfanuméricos Unicode e o sublinhado, mas isso pode ser alterado

usando o sinalizador **ASCII**. Os limites das palavras são determinados pela localidade atual se o sinalizador **LOCALE** for usado.

Nota

Dentro de um intervalo de caracteres, `\b` representa o caractere backspace, para compatibilidade com strings literais do Python.

`\B`

Corresponde à string vazia, mas apenas quando *não* estiver no início ou no final de uma palavra. Isso significa que `r'at\B'` corresponde a `'athens'`, `'atom'`, `'attorney'`, mas não `'at'`, `'at.'` ou `'at!'`. `\B` é exatamente o oposto de `\b`, então caracteres de palavras em padrões Unicode são alfanuméricos Unicode ou o sublinhado, embora isso possa ser alterado usando o sinalizador **ASCII**. Os limites das palavras são determinados pela localidade atual se o sinalizador **LOCALE** for usado.

Alterado na versão 3.14: `\B` agora corresponde string de entrada vazia.

`\d`

Para padrões Unicode (str):

Corresponde a qualquer dígito decimal Unicode (ou seja, qualquer caractere na categoria de caractere Unicode [Nd] [https://www.unicode.org/versions/Unicode15.0.0/ch04.pdf#G134153]). Isso inclui `[0-9]`, e também muitos outros caracteres de dígitos.

Se o sinalizador **ASCII** for usado, apenas `[0-9]` será correspondido.

Para padrões de 8 bits (isto é, bytes):

Corresponde a qualquer dígito decimal no conjunto de caracteres ASCII; isso é equivalente a `[0-9]`.

`\D`

Corresponde a qualquer caractere que não seja um dígito decimal. Isso é o oposto de `\d`.

Se o sinalizador **ASCII** for usado, apenas `[^0-9]` será correspondido.

`\s`

Para padrões Unicode (str):

Corresponde a caracteres de espaço em branco Unicode (como definido por `str.isspace()`) . Isso inclui `[ \t\n\r\f\v]`, e também muitos outros caracteres, como, por exemplo, os espaços não separáveis exigidos pelas regras de tipografia em muitos idiomas.

Se o sinalizador `ASCII` for usado, apenas `[ \t\n\r\f\v]` é correspondido.

Para padrões de 8 bits (isto é, bytes):

Corresponde a caracteres considerados espaços em branco no conjunto de caracteres ASCII; isso é equivalente a `[ \t\n\r\f\v]`.

`\S`

Corresponde a qualquer caractere que não seja um espaço em branco. Isso é o oposto de `\s`.

Se o sinalizador `ASCII` for usado, apenas `[^ \t\n\r\f\v]` é correspondido.

`\w`

Para padrões Unicode (str):

Corresponde a caracteres de palavras Unicode; isso inclui todos os caracteres alfanuméricos Unicode (conforme definido por `str.isalnum()`), bem como o sublinhado (_).

Se o sinalizador `ASCII` for usado, apenas `[a-zA-Z0-9_]` será correspondido.

Para padrões de 8 bits (isto é, bytes):

Corresponde a caracteres considerados alfanuméricos no conjunto de caracteres ASCII; isso é equivalente a `[a-zA-Z0-9_]`. Se o sinalizador `LOCALE` for usado, corresponde aos caracteres considerados alfanuméricos na localidade atual e o sublinhado.

`\W`

Corresponde a qualquer caractere que não seja um caractere de palavra. Isso é o oposto de `\w`. Por padrão, corresponde a caracteres que não são sublinhados (_) para os quais **`str.isalnum()`** retorna `False`.

Se o sinalizador **`ASCII`** for usado, apenas `[^a-zA-Z0-9_]` será correspondido.

Se o sinalizador **`LOCALE`** for usado, corresponde a caracteres que não são alfanuméricos na localidade atual nem ao sublinhado.

`\z`

Corresponde apenas ao final da string.

Adicionado na versão 3.14.

`\Z`

O mesmo que `\z`. Para compatibilidade com versões mais antigas de Python.

A maioria das sequências de escape com suporte das literais de string Python também são aceitos pelo analisador sintático de expressão regular:

```
\a      \b      \f      \n
\N      \r      \t      \u
\U      \v      \x      \\
```

(Observe que `\b` é usado para representar limites de palavras e significa fazer “backspace” apenas dentro das classes de caracteres.)

As sequências de escape `'\u'`, `'\U'` e `'\N'` são reconhecidas apenas em padrões Unicode (str). Em padrões de bytes, eles são erros. Escapes desconhecidos de letras ASCII são reservados para uso futuro e tratados como erros.

Os escapes octais são incluídos em um formulário limitado. Se o primeiro dígito for 0, ou se houver três dígitos octais, é considerado um escape octal.

Caso contrário, é uma referência de grupo. Quanto aos literais de string, os escapes octais têm sempre no máximo três dígitos.

Alterado na versão 3.3: As sequências de escape `'\u'` e `'\U'` foram adicionadas.

Alterado na versão 3.6: Escapes desconhecidos consistindo em `'\'` e uma letra ASCII agora são erros.

Alterado na versão 3.8: A sequência de escape `'\N{name}'` foi adicionada. Como em literais de string, ela se expande para o caractere Unicode nomeado (por exemplo, `'\N{EM DASH}'`).

Conteúdo do módulo

O módulo define várias funções, constantes e uma exceção. Algumas das funções são versões simplificadas dos métodos completos para expressões regulares compiladas. A maioria das aplicações não triviais sempre usa a forma compilada.

Sinalizadores

Alterado na versão 3.6: Constantes de sinalizadores agora são instâncias de **`RegexFlag`**, que é uma subclasse de **`enum.IntFlag`**.

class re.RegexFlag

Uma subclasse de **`enum.IntFlag`** contendo as opções de expressão regular listadas abaixo.

Adicionado na versão 3.11: - added to `__all__`

re.A
re.ASCII

Faz com que `\w`, `\W`, `\b`, `\B`, `\d`, `\D`, `\s` e `\S` executem a correspondência somente ASCII em vez da correspondência Unicode completa. Isso é significativo apenas para padrões Unicode (str) e é ignorado para padrões de bytes.

Corresponde ao sinalizador em linha `(?a)`.

Nota

O sinalizador **U** ainda existe para compatibilidade com versões anteriores, mas é redundante no Python 3, pois as correspondências são feitas em Unicode por padrão para padrões de `str` e correspondência Unicode não é permitida para padrões de “bytes”. **UNICODE** e o sinalizador em linha `(?u)` são igualmente redundantes.

re.DEBUG

Exibe informações de depuração sobre a expressão compilada.

Nenhum sinalizador em linha correspondente.

re.I
re.IGNORECASE

Faz uma correspondência que não diferencia maiúsculas de minúsculas; expressões como `[A-Z]` também corresponderão a letras minúsculas. A correspondência Unicode completa (como `Ü` correspondendo a `ü`) também funciona, a menos que o sinalizador **ASCII** seja usado para desabilitar correspondências não ASCII. A localidade atual não muda o efeito deste sinalizador a menos que o sinalizador **LOCALE** também seja usado.

Corresponde ao sinalizador em linha `(?i)`.

Observe que quando os padrões Unicode `[a-z]` ou `[A-Z]` são usados em combinação com o sinalizador **IGNORECASE**, eles corresponderão às 52 letras ASCII e 4 letras não-ASCII adicionais: ‘İ’ (U+0130, letra latina I maiúscula com ponto em cima), ‘ı’ (U+0131, letra latina i minúscula sem ponto), ‘ſ’ (U+017F, letra latina s minúscula longa) e ‘K’ (U+212A, sinal de Kelvin). Se o sinalizador **ASCII** for usado, apenas as letras ‘a’ a ‘z’ e ‘A’ a ‘Z’ serão correspondidas.

re.L
re.LOCALE

Faz com que `\w`, `\W`, `\b`, `\B` e a correspondência sem diferenciação de maiúsculas e minúsculas dependam da localidade atual. Esse sinalizador só pode ser usado com padrões de bytes.

Corresponde ao sinalizador em linha `(?L)`.

Aviso

Este sinalizador é desencorajado; considere usar correspondência Unicode em vez disso. O mecanismo de localidade é muito pouco confiável, pois só manipula uma "cultura" por vez e só funciona com localidades de 8 bits. A correspondência Unicode é habilitada por padrão para padrões Unicode (str) e é capaz de manipular diferentes localidades e idiomas.

Alterado na versão 3.6: **`LOCALE`** pode ser usado apenas com padrões de bytes e não é compatível com **`ASCII`**.

Alterado na versão 3.7: Objetos de expressão regular compilados com o sinalizador **`LOCALE`** não dependem mais da localidade em tempo de compilação. Apenas a localidade no momento da correspondência afeta o resultado da correspondência.

re.M
re.MULTILINE

Quando especificado, o caractere padrão `'^'` corresponde ao início da string e ao início de cada linha (imediatamente após cada nova linha); e o caractere padrão `'$'` corresponde ao final da string e ao final de cada linha (imediatamente antes de cada nova linha). Por padrão, `'^'` corresponde apenas no início da string, e `'$'` apenas no final da string e imediatamente antes da nova linha (se houver) no final da string.

Corresponde ao sinalizador em linha `(?m)`.

re.NOFLAG

Indica que nenhum sinalizador está sendo aplicado, o valor é `0`. Este sinalizador pode ser usado como um valor padrão para um argumento

nomeado de função ou como um valor base que será condicionalmente OU com outros sinalizadores. Exemplo de uso como valor padrão:

```
def minha_func(padrão, texto, sinalizador=re.NOFLAG):
    return re.search(padrão, texto, sinalizador)
```

Adicionado na versão 3.11.

re.S
re.DOTALL

Faz o caractere especial `'.'` corresponder com qualquer caractere que seja, incluindo uma nova linha; sem este sinalizador, `'.'` irá corresponder a qualquer coisa, *exceto* uma nova linha.

Corresponde ao sinalizador em linha `(?s)`.

re.U
re.UNICODE

Em Python 3, os caracteres Unicode são correspondidos por padrão para padrões `str`. Esse sinalizador é, portanto, redundante e **sem efeito**, sendo mantido apenas para compatibilidade com versões anteriores.

Veja **`ASCII`** para restringir a correspondência apenas a caracteres ASCII.

re.X
re.VERBOSE

Este sinalizador permite que você escreva expressões regulares que parecem mais agradáveis e são mais legíveis, permitindo que você separe visualmente seções lógicas do padrão e adicione comentários. O espaço em branco dentro do padrão é ignorado, exceto quando em uma classe de caractere, ou quando precedido por uma contrabarra sem escape, ou dentro de tokens como `*?`, `(?:` ou `(?P<...>`. Por exemplo, `(?:` e `* ?` não são permitidos. Quando uma linha contém um `#` que não está em uma classe de caractere e não é precedido por uma contrabarra sem escape, todos os caracteres a partir do `#` mais à esquerda até o final da linha são ignorados.

Isso significa que os dois seguintes objetos expressão regular que correspondem a um número decimal são funcionalmente iguais:

```
a = re.compile(r"""\d +  # a parte inteira
                   \.    # o ponto decimal
                   \d *  # alguns digitos fracionários""", re
b = re.compile(r"\d+\.\d*")
```

Corresponde ao sinalizador em linha `(?x)`.

Funções

re.compile(*pattern*, *flags=0*)

Compila um padrão de expressão regular em um objeto expressão regular, que pode ser usado para correspondência usando seu **`prefixmatch()`**, **`search()`** e outros métodos, descritos abaixo.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

A sequência

```
prog = re.compile(pattern)
result = prog.search(string)
```

é equivalente a

```
result = re.search(pattern, string)
```

mas usar **`re.compile()`** e salvar o objeto expressão regular resultante para reutilização é mais eficiente quando a expressão será usada várias vezes em um único programa.

Nota

As versões compiladas dos padrões mais recentes passados para **`re.compile()`** e as funções de correspondência em nível de módulo são armazenadas em cache, de modo que programas que usam apenas

algumas expressões regulares por vez não precisam se preocupar em compilar expressões regulares.

re.search(*pattern*, *string*, *flags=0*)

Percorre a *string* procurando o primeiro local onde o padrão *pattern* de expressão regular produz uma correspondência e retorna um **`Match`** correspondente. Retorna `None` se nenhuma posição na string corresponder ao padrão; observe que isso é diferente de encontrar uma correspondência de comprimento zero em algum ponto da string.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

re.prefixmatch(*pattern*, *string*, *flags=0*)

Se zero ou mais caracteres no início da *string* corresponderem ao padrão *pattern* da expressão regular, retorna um **`Match`** correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

Nota

Mesmo no modo **`MULTILINE`**, isso irá corresponder apenas no início da string e não no início de cada linha.

Se você quiser localizar uma correspondência em qualquer lugar em *string*, use **`search()`** (veja também search() vs. prefixmatch()).

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

Esta função agora possui dois nomes e é conhecida há muito tempo como **`match()`**. Utilize esse nome quando precisar manter a compatibilidade com versões mais antigas do Python.

Adicionado na versão 3.15.

re.match(*pattern*, *string*, *flags=0*)

Suavemente descontinuado desde a versão 3.15: **`match()`** has been soft deprecated in favor of the alternate **`prefixmatch()`** name of this API which is more explicitly descriptive. Use it to better express intent. The norm in other languages and regular expression implementations is to use the term *match* to refer to the behavior of what Python has always called **`search()`**. See prefixmatch() vs. match().

re.fullmatch(*pattern*, *string*, *flags=0*)

Se toda a *string* corresponder ao padrão *pattern* da expressão regular, retorna um **`Match`** correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

Adicionado na versão 3.4.

re.split(*pattern*, *string*, *maxsplit=0*, *flags=0*)

Divide a *string* pelas ocorrências do padrão *pattern*. Se parênteses de captura forem usados em *pattern*, o texto de todos os grupos no padrão também será retornado como parte da lista resultante. Se *maxsplit* for diferente de zero, no máximo *maxsplit* divisões ocorrerão e o restante da string será retornado como o elemento final da lista.

```
>>> re.split(r'\W+', 'Palavras, palavras, palavras.')
['Palavras', 'palavras', 'palavras', '']
>>> re.split(r'(\W+)', 'Palavras, palavras, palavras.')
['Palavras', ', ', 'palavras', ', ', 'palavras', '.', '']
>>> re.split(r'\W+', 'Palavras, palavras, palavras.', maxspli
['Palavras', 'palavras, palavras.']
>>> re.split('[a-f]+', '0a3B9', flags=re.IGNORECASE)
['0', '3', '9']
```

Se houver grupos de captura no separador e ele corresponder ao início da string, o resultado começará com uma string vazia. O mesmo vale para o final da string:

```
>>> re.split(r'(\W+)', '...palavras, palavras...')
['', '...', 'palavras', ', ', 'palavras', '...', '']
```

Dessa forma, os componentes do separador são sempre encontrados nos mesmos índices relativos na lista de resultados.

Correspondências vazias adjacentes não são possíveis, mas uma correspondência vazia pode ocorrer imediatamente após uma correspondência não-vazia.

```
>>> re.split(r'\b', 'Palavras, palavras, palavras.')
['', 'Palavras', ', ', 'palavras', ', ', 'palavras', '.']
>>> re.split(r'\W*', '...palavras...')
['', '', 'p', 'a', 'l', 'a', 'v', 'r', 'a', 's', '', '']
>>> re.split(r'(\W*)', '...palavras...')
['', '...', '', '', 'p', '', 'a', '', 'l', '', 'a', '', 'v',
```

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.1: Adicionado o argumento de sinalizadores opcionais.

Alterado na versão 3.7: Adicionado suporte de divisão em um padrão que pode corresponder a uma string vazia.

Descontinuado desde a versão 3.13: Passar *maxsplit* e *flags* como argumentos posicionais foi descontinuado. Nas versões futuras do Python eles serão parâmetros somente-nomeados.

re.findall(*pattern*, *string*, *flags=0*)

Retorna todas as correspondências não sobrepostas do padrão *pattern* em *string*, como uma lista de strings ou tuplas. A *string* é verificada da

esquerda para a direita e as correspondências são retornadas na ordem encontrada. Correspondências vazias são incluídas no resultado.

O resultado depende do número de grupos de captura no padrão. Se não houver grupos, retorna uma lista de strings que correspondem a todo o padrão. Se houver exatamente um grupo, retorna uma lista de strings correspondentes a esse grupo. Se vários grupos estiverem presentes, retorna uma lista de tuplas de strings correspondentes aos grupos. Grupos de não-captura não afetam a forma do resultado.

```
>>> re.findall(r'\bf[a-z]*', 'which foot or hand fell fastest
['foot', 'fell', 'fastest']
>>> re.findall(r'(\w+)=(\d+)', 'set width=20 and height=10')
[('width', '20'), ('height', '10')]
```

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.7: Correspondências não vazias agora podem começar logo após uma correspondência vazia anterior.

re.finditer(*pattern*, *string*, *flags=0*)

Retorna um iterador produzindo objetos **`Match`** sobre todas as correspondências não sobrepostas para o padrão *pattern* em *string*. A *string* é percorrida da esquerda para a direita e as correspondências são retornadas na ordem encontrada. Correspondências vazias são incluídas no resultado.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.7: Correspondências não vazias agora podem começar logo após uma correspondência vazia anterior.

re.sub(*pattern*, *repl*, *string*, *count=0*, *flags=0*)

Retorna a string obtida substituindo as ocorrências não sobrepostas da extremidade esquerda do padrão *pattern* na *string* por *repl*. Se o padrão não for encontrado, *string* será retornado inalterado. *repl* pode ser uma string ou uma função; se for uma string, qualquer escape de contrabarra será processado. Ou seja, `\n` é convertido em um único caractere de nova linha, `\r` é convertido em um retorno de carro e assim por diante. Escapes desconhecidos de letras ASCII são reservados para uso futuro e tratados como erros. Outros escapes desconhecidos como `\&` são deixados como estão. Referências anteriores, como `\6`, são substituídos pela substring correspondida pelo grupo 6 no padrão. Por exemplo:

```
>>> re.sub(r'def\s+([a-zA-Z_][a-zA-Z_0-9]*)\s*\(\s*\):',
...        r'static PyObject*\npy_\1(void)\n{',
...        'def myfunc():')
'static PyObject*\npy_myfunc(void)\n{'
```

Se *repl* for uma função, ela será chamada para cada ocorrência não sobreposta do padrão *pattern*. A função recebe um único argumento **`Match`** e retorna a string de substituição. Por exemplo:

```
>>> def dashrepl(matchobj):
...     if matchobj.group(0) == '-': return ' '
...     else: return '-'
...
>>> re.sub('-{1,2}', dashrepl, 'arquivos-de-pro----grama')
'arquivos de pro--grama'
>>> re.sub(r'\sE\s', ' & ', 'Feijões queimados e spam', flags
'Feijões queimados & spam'
```

O padrão pode ser uma string ou um **`Pattern`**.

O argumento opcional `count` é o número máximo de ocorrências do padrão a ser substituído; `count` deve ser um número inteiro não negativo. Se omitido ou igual a zero, todas as ocorrências serão substituídas.

Correspondências vazias adjacentes não são possíveis, mas uma correspondência vazia pode ocorrer imediatamente após uma

correspondência não-vazia. Como resultado, `sub('x*', '-', 'abxd')` retorna `'-a-b--d-'` invés de `'-a-b-d-'`.

Em argumentos *repl* do tipo string, além dos escapes de caractere e referências anteriores descritas acima, `\g<nome>` usará a substring correspondida pelo grupo denominado `nome`, conforme definido pela sintaxe `(?P<nome>...)`. `\g<número>` usa o número do grupo correspondente; `\g<2>` é portanto equivalente a `\2`, mas não é ambíguo em uma substituição como `\g<2>0`. `\20` seria interpretado como uma referência ao grupo 20, não uma referência ao grupo 2 seguida pelo caractere literal `'0'`. A referência anterior `\g6` substitui em toda a substring correspondida pela ER.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

Alterado na versão 3.1: Adicionado o argumento de sinalizadores opcionais.

Alterado na versão 3.5: Grupos sem correspondência são substituídos por uma string vazia.

Alterado na versão 3.6: Escapes desconhecidos no padrão *pattern* consistindo em `'\'` e uma letra ASCII agora são erros.

Alterado na versão 3.7: Escapes desconhecidos em *repl* consistindo em `'\'` e uma letra ASCII agora são erros. Uma correspondência vazia pode ocorrer imediatamente após uma correspondência não-vazia.

Alterado na versão 3.12: O grupo *id* só pode conter dígitos ASCII. Nas strings de substituição **`bytes`**, o grupo *name* só pode conter bytes no intervalo ASCII (`b'\x00'-b'\x7f'`).

Descontinuado desde a versão 3.13: Passar *count* e *flags* como argumentos posicionais foi descontinuado. Nas versões futuras do Python eles serão parâmetros somente-nomeados.

re.subn(*pattern*, *repl*, *string*, *count=0*, *flags=0*)

Executa a mesma operação que **sub()**, mas retorna uma tupla `(nova_string, número_de_substituições_feitas)`.

O comportamento da expressão pode ser modificado especificando um valor para o parâmetro *flags*. Os valores podem ser qualquer um dos sinalizadores, combinados usando OU bit a bit (o operador `|`).

re.escape(*pattern*)

Escapa caracteres especiais no padrão *pattern*. Isso é útil se você deseja corresponder uma string literal arbitrária que pode conter metacaracteres de expressão regular. Por exemplo:

```
>>> print(re.escape('https://www.python.org'))
https://www\.python\.org

>>> legal_chars = string.ascii_lowercase + string.digits + "!
>>> print('[%s]+' % re.escape(legal_chars))
[abcdefghijklmnopqrstuvwxyz0123456789!\#\$%\&'\*\+\-\.\^_`\|\

>>> operators = ['+', '-', '*', '/', '**']
>>> print('|'.join(map(re.escape, sorted(operators, reverse=T
/|\-|\+|\*\*|\*
```

Esta função não deve ser usada para a string de substituição em **sub()** e **subn()**, apenas contrabarras devem ser escapadas. Por exemplo:

```
>>> digits_re = r'\d+'
>>> sample = '/usr/sbin/sendmail - 0 errors, 12 warnings'
>>> print(re.sub(digits_re, digits_re.replace('\\', r'\\'), s
/usr/sbin/sendmail - \d+ errors, \d+ warnings
```

Alterado na versão 3.3: O caractere `'_'` não é mais escapado.

Alterado na versão 3.7: Somente caracteres que podem ter um significado especial em uma expressão regular são escapados. Como resultado, `'!'`, `'"'`, `'%'`, `"'"`, `','`, `'/'`, `':'`, `';'`, `'<'`, `'='`, `'>'`, `'@'`, e `"`"` não são mais escapados.

re.purge()

Limpa o cache da expressão regular.

Exceções

exception re.PatternError(*msg*, *pattern=None*, *pos=None*)

Exceção levantada quando uma string passada para uma das funções desde módulo não é uma expressão regular válida (por exemplo, ela pode conter parênteses não correspondentes) ou quando algum outro erro ocorre durante a compilação ou correspondência. Nunca é um erro se uma string não contém correspondência para um padrão. A instância de `PatternError` possui os seguintes atributos adicionais:

msg

A mensagem de erro não formatada.

pattern

O padrão da expressão regular.

pos

O índice no padrão *pattern* no qual a compilação falhou (pode ser `None`).

lineno

A linha correspondente a *pos* (pode ser `None`).

colno

A coluna correspondente a *pos* (pode ser `None`).

Alterado na versão 3.5: Adicionados os atributos adicionais.

Alterado na versão 3.13: `PatternError` era originalmente chamado de `error`; o último é mantido como um apelido para compatibilidade com versões anteriores.

Regular expression objects

class re.Pattern

Objeto expressão regular compilado retornado por **re.compile()**.

Patterns are generic over the type of string they handle (**str** or **bytes**).

Alterado na versão 3.9: **re.Pattern** tem suporte a `[]` para indicar um padrão Unicode (str) ou bytes. Veja Tipo Generic Alias.

Pattern.search(*string*[, *pos*[, *endpos*]])

Percorre a *string* procurando o primeiro local onde esta expressão regular produz uma correspondência e retorna um **Match** correspondente. Retorna `None` se nenhuma posição na string corresponder ao padrão; observe que isso é diferente de encontrar uma correspondência de comprimento zero em algum ponto da string.

O segundo parâmetro opcional *pos* fornece um índice na string onde a pesquisa deve começar; o padrão é `0`. Isso não é totalmente equivalente a fatiar a string; o caractere padrão `'^'` corresponde no início real da string e nas posições logo após uma nova linha, mas não necessariamente no índice onde a pesquisa deve começar.

O parâmetro opcional *endpos* limita o quão longe a string será pesquisada; será como se a string tivesse *endpos* caracteres, então apenas os caracteres de *pos* a `endpos - 1` serão procurados por uma correspondência. Se *endpos* for menor que *pos*, nenhuma correspondência será encontrada; caso contrário, se *rx* é um objeto de expressão regular compilado, `rx.search(string, 0, 50)` é equivalente a `rx.search(string[:50], 0)`.

```
>>> pattern = re.compile("d")
>>> pattern.search("dog")     # Correspondeu no índice 0
<re.Match object; span=(0, 1), match='d'>
>>> pattern.search("dog", 1)  # Não correspondeu; a busca não
```

Pattern.prefixmatch(*string*[, *pos*[, *endpos*]])

Se zero ou mais caracteres no início da *string* corresponderem a esta expressão regular, retorna um **Match** correspondente. Retorna `None` se a

string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

Note that even in **MULTILINE** mode, this will only match at the beginning of the string and not at the beginning of each line.

Os parâmetros opcionais *pos* e *endpos* têm o mesmo significado que para o método **search()**.

```
>>> pattern = re.compile("o")
>>> pattern.prefixmatch("dog")     # No match as "o" is not a
>>> pattern.prefixmatch("dog", 1)  # Match as "o" is the 2nd
<re.Match object; span=(1, 2), match='o'>
```

Se você quiser localizar uma correspondência em qualquer lugar em *string*, use **search()** ao invés (veja também search() vs. prefixmatch()).

This method now has two names and has long been known as **match()**. Use that name when you need to retain compatibility with older Python versions.

Adicionado na versão 3.15.

Pattern.match(*string*[, *pos*[, *endpos*]])

Suavemente descontinuado desde a versão 3.15: **match()** has been soft deprecated in favor of the alternate **prefixmatch()** name of this API which is more explicitly descriptive. Use it to better express intent. The norm in other languages and regular expression implementations is to use the term *match* to refer to the behavior of what Python has always called **search()**. See prefixmatch() vs. match().

Pattern.fullmatch(*string*[, *pos*[, *endpos*]])

Se toda a *string* corresponder a esta expressão regular, retorna um **Match** correspondente. Retorna `None` se a string não corresponder ao padrão; observe que isso é diferente de uma correspondência de comprimento zero.

Os parâmetros opcionais *pos* e *endpos* têm o mesmo significado que para o método `search()`.

```
>>> pattern = re.compile("o[gh]")
>>> pattern.fullmatch("dog")      # Não correspondeu, pois "d
>>> pattern.fullmatch("ogre")     # Não correspondeu, pois nã
>>> pattern.fullmatch("doggie", 1, 3)   # Corresponde dentro
<re.Match object; span=(1, 3), match='og'>
```

Adicionado na versão 3.4.

Pattern.split(*string*, *maxsplit=0*)

Idêntico à função `split()`, usando o padrão compilado.

Pattern.findall(*string*[, *pos*[, *endpos*]])

Semelhante à função `findall()`, usando o padrão compilado, mas também aceita os parâmetros *pos* e *endpos* opcionais que limitam a região de pesquisa como para `search()`.

Pattern.finditer(*string*[, *pos*[, *endpos*]])

Semelhante à função `finditer()`, usando o padrão compilado, mas também aceita os parâmetros *pos* e *endpos* opcionais que limitam a região de pesquisa como para `search()`.

Pattern.sub(*repl*, *string*, *count=0*)

Idêntico à função `sub()`, usando o padrão compilado.

Pattern.subn(*repl*, *string*, *count=0*)

Idêntico à função `subn()`, usando o padrão compilado.

Pattern.flags

Os sinalizadores de correspondência de expressões regulares. Esta é uma combinação dos sinalizadores fornecidos para `compile()`, qualquer sinalizador em linha `(?...)` no padrão e sinalizadores implícitos como `UNICODE` se o padrão for uma string Unicode.

Pattern.groups

O número de grupos de captura no padrão.

Pattern.groupindex

Um dicionário que mapeia qualquer nome de grupo simbólico definido por `(?P<id>)` para números de grupo. O dicionário estará vazio se nenhum grupo simbólico for usado no padrão.

Pattern.pattern

A string de padrão da qual o objeto de padrão foi compilado.

Alterado na versão 3.7: Adicionado suporte de **`copy.copy()`** e **`copy.deepcopy()`**. Os objetos expressão regular compilados são considerados atômicos.

Match objects

Objetos correspondência sempre têm um valor booleano de `True`. Como **`match()`** e **`search()`** retornam `None` quando não há correspondência, você pode testar se houve uma correspondência com uma simples instrução `if`:

```
if match := re.search(pattern, string):
    process(match)
```

class re.Match

Objeto correspondência retornado por `match`s e `search`s bem sucedidos.

Matches are generic over the type of string which was matched (**`str`** or **`bytes`**).

Alterado na versão 3.9: **`re.Match`** tem suporte a `[]` para indicar uma correspondência Unicode (str) ou bytes. Veja Tipo Generic Alias.

Match.expand(*template*)

Retorna a string obtida fazendo a substituição da contrabarra na string template *template*, como feito pelo método **`sub()`**. Escapes como `\n` são convertidos para os caracteres apropriados, e referências anteriores

numéricas (`\1`, `\2`) e referências anteriores nomeadas (`\g<1>`, `\g<nome>`) são substituídas pelo conteúdo do grupo correspondente. A referência anterior `\g<0>` será substituída pela correspondência completa.

Alterado na versão 3.5: Grupos sem correspondência são substituídos por uma string vazia.

Match.group([*group1*, ...])

Returns one or more subgroups of the match. If there is a single argument, the result is a single string; if there are multiple arguments, the result is a tuple with one item per argument. Without arguments, *group1* defaults to zero (the whole match is returned). If a *groupN* argument is zero, the corresponding return value is the entire matching string; if it is a positive integer, it is the string matching the corresponding parenthesized group. If a group number is negative or larger than the number of groups defined in the pattern, an **`IndexError`** exception is raised. If a group is contained in a part of the pattern that did not match, the corresponding result is `None`. If a group is contained in a part of the pattern that matched multiple times, the last match is returned.

```
>>> m = re.search(r"\A(\w+) (\w+)", "Norwegian Blue, pining f
>>> m.group(0)       # The entire match
'Norwegian Blue'
>>> m.group(1)       # The first parenthesized subgroup.
'Norwegian'
>>> m.group(2)       # The second parenthesized subgroup.
'Blue'
>>> m.group(1, 2)    # Multiple arguments give us a tuple.
('Norwegian', 'Blue')
```

Se a expressão regular usa a sintaxe `(?P<nome>...)`, os argumentos *groupN* também podem ser strings que identificam grupos por seus nomes de grupo. Se um argumento string não for usado como um nome de grupo no padrão, uma exceção **`IndexError`** é levantada.

Um exemplo moderadamente complicado:

```
>>> m = re.search(r"(?P<adjective>\w+) (?P<animal>\w+)", "kil
>>> m.group('adjective')
'killer'
>>> m.group('animal')
'rabbit'
```

Grupos nomeados também podem ser referidos por seu índice:

```
>>> m.group(1)
'killer'
>>> m.group(2)
'rabbit'
```

Se um grupo corresponder várias vezes, apenas a última correspondência estará acessível:

```
>>> m = re.search(r"(..)+", "a1b2c3")  # Matches 3 times.
>>> m.group(1)                         # Returns only the las
'c3'
```

Match.__getitem__(*g*)

Isso é idêntico a `m.group(g)`. Isso permite acesso mais fácil a um grupo individual de uma correspondência:

```
>>> m = re.search(r"(\w+) (\w+)", "Norwegian Blue, pining for
>>> m[0]       # The entire match
'Norwegian Blue'
>>> m[1]       # The first parenthesized subgroup.
'Norwegian'
>>> m[2]       # The second parenthesized subgroup.
'Blue'
```

Também há suporte para grupos nomeados:

```
>>> m = re.search(r"(?P<adjective>\w+) (?P<animal>\w+)", "kil
>>> m['adjective']
'killer'
>>> m['animal']
'rabbit'
```

Adicionado na versão 3.6.

Match.groups([*default=None*])

Retorna uma tupla contendo todos os subgrupos da correspondência, de 1 até quantos grupos estiverem no padrão. O argumento *default* é usado para grupos que não participaram da correspondência; o padrão é `None`.

Por exemplo:

```
>>> m = re.search(r"(\d+)\.(\d+)", "24.1632")
>>> m.groups()
('24', '1632')
```

Se colocarmos a casa decimal e tudo depois dela opcional, nem todos os grupos podem participar da correspondência. Esses grupos serão padronizados como `None`, a menos que o argumento *default* seja fornecido:

```
>>> m = re.search(r"(\d+)\.?(\d+)?", "24")
>>> m.groups()      # Second group defaults to None.
('24', None)
>>> m.groups('0')   # Now, the second group defaults to '0'.
('24', '0')
```

Match.groupdict([*default=None*])

Retorna um dicionário contendo todos os subgrupos *nomeados* da correspondência, tendo como chave o nome do subgrupo. O argumento *default* usado para grupos que não participaram da correspondência; o padrão é `None`. Por exemplo:

```
>>> m = re.search(r"(?P<adjective>\w+) (?P<animal>\w+)", "ki
>>> m.groupdict()
{'adjective': 'killer', 'animal': 'rabbit'}
```

Match.start([*group*])

Match.end([*group*])

Retorna os índices de início e fim da substring correspondida pelo grupo *group*; *group* tem como padrão zero (o que significa que toda a substring é correspondida). Retorna `-1` se *group* existe, mas não contribuiu para a correspondência. Para um objeto correspondência *m* e um grupo *g* que contribuiu para a correspondência, a substring correspondida pelo grupo *g* (equivalente a `m.group(g)`) é

```
m.string[m.start(g):m.end(g)]
```

Observe que `m.start(group)` será igual a `m.end(group)` se *group* correspondeu a uma string nula. Por exemplo, após `m = re.search('b(c?)', 'cba')`, `m.start(0)` é 1, `m.end(0)` é 2, `m.start(1)` e `m.end(1)` são 2, e `m.start(2)` levanta uma exceção **`IndexError`**.

Um exemplo que removerá *remova_isto* dos endereços de e-mail:

```
>>> email = "tony@tiremova_istoger.net"
>>> m = re.search("remova_isto", email)
>>> email[:m.start()] + email[m.end():]
'tony@tiger.net'
```

Match.span([*group*])

Para uma correspondência *m*, retorna a tupla com dois elementos `(m.start(group), m.end(group))`. Observe que se *group* não contribuiu para a correspondência, isso é `(-1, -1)`. *group* tem como padrão zero, a correspondência inteira.

Match.pos

O valor de *pos* que foi passado para o método **`search()`** ou **`match()`** de um objeto expressão regular. Este é o índice da string na qual o mecanismo de ER começou a procurar uma correspondência.

Match.endpos

O valor de *endpos* que foi passado para o método **`search()`** ou **`match()`** de um objeto expressão regular. Este é o índice da string após o qual o mecanismo de ER não vai chegar.

Match.lastindex

O índice em número inteiro do último grupo de captura correspondido, ou `None` se nenhum grupo foi correspondido. Por exemplo, as expressões `(a)b`, `((a)(b))` e `((ab))` terão `lastindex == 1` se aplicadas à string `'ab'`, enquanto a expressão `(a)(b)` terá `lastindex == 2`, se aplicada à mesma string.

Match.lastgroup

O nome do último grupo de captura correspondido, ou `None` se o grupo não tinha um nome, ou se nenhum grupo foi correspondido.

Match.re

O objeto expressão regular cujo método **`match()`** ou **`search()`** produziu esta instância de correspondência.

Match.string

A string passada para **`match()`** ou **`search()`**.

Alterado na versão 3.7: Adicionado suporte para **`copy.copy()`** e **`copy.deepcopy()`**. Objetos correspondência são considerados atômicos.

Regular expression examples

Checking for a pair

Neste exemplo, usaremos a seguinte função auxiliar para exibir objetos correspondência com um pouco mais de elegância:

```
def displaymatch(match):
    if match is None:
        return None
    return '<Match: %r, groups=%r>' % (match.group(), match.grou
```

Suponha que você esteja escrevendo um programa de pôquer onde a mão de um jogador é representada como uma string de 5 caracteres com cada caractere representando uma carta, “a” para ás, “k” para rei, “q” para dama, “j” para valete, “t” para 10 e “2” a “9” representando a carta com esse valor.

Para ver se uma determinada string é uma mão válida, pode-se fazer o seguinte:

```
>>> valid_hand_re = re.compile(r"^[a2-9tjqk]{5}$")
>>> displaymatch(valid_hand_re.search("akt5q"))  # Valid.
"<Match: 'akt5q', groups=()>"
>>> displaymatch(valid_hand_re.search("akt5e"))  # Invalid.
>>> displaymatch(valid_hand_re.search("akt"))    # Invalid.
>>> displaymatch(valid_hand_re.search("727ak"))  # Valid.
"<Match: '727ak', groups=()>"
```

Essa última mão, `"727ak"`, continha um par, ou duas cartas com o mesmo valor. Para combinar isso com uma expressão regular, pode-se usar referências anteriores como:

```
>>> pair_re = re.compile(r".*(.).*\1")
>>> displaymatch(pair_re.prefixmatch("717ak"))     # Pair of 7s.
"<Match: '717', groups=('7',)>"
>>> displaymatch(pair_re.prefixmatch("718ak"))     # No pairs.
>>> displaymatch(pair_re.prefixmatch("354aa"))     # Pair of ace
"<Match: '354aa', groups=('a',)>"
```

Para descobrir em que carta o par consiste, pode-se usar o método `group()` do objeto correspondência da seguinte maneira:

```
>>> pair_re = re.compile(r".*(.).*\1")
>>> pair_re.prefixmatch("717ak").group(1)
'7'

# Error because prefixmatch() returns None, which doesn't have a
>>> pair_re.prefixmatch("718ak").group(1)
Traceback (most recent call last):
  File "<pyshell#23>", line 1, in <module>
    pair_re.prefixmatch("718ak").group(1)
AttributeError: 'NoneType' object has no attribute 'group'

>>> pair_re.prefixmatch("354aa").group(1)
'a'
```

Simulando scanf()

Atualmente, Python não possui um equivalente a **`scanf()`**. Expressões regulares são geralmente mais poderosas, embora também mais prolixas, do que strings de formato **`scanf()`**. A tabela abaixo oferece alguns mapeamentos mais ou menos equivalentes entre strings de formato **`scanf()`** e expressões regulares.

Símbolo do `scanf()`	Expressão regular
`%c`	`.`
`%5c`	`.{5}`
`%d`	`[-+]?\d+`
`%e`, `%E`, `%f`, `%g`	`[-+]?(\d+(\.\d*)?|\.\d+)([eE][-+]?\d+)?`
`%i`	`[-+]?(0[xX][\dA-Fa-f]+|0[0-7]*|\d+)`
`%o`	`[-+]?[0-7]+`
`%s`	`\S+`
`%u`	`\d+`
`%x`, `%X`	`[-+]?(0[xX])?[\dA-Fa-f]+`

To extract the filename and numbers from a string like:

```
/usr/sbin/sendmail - 0 errors, 4 warnings
```

you would use a **`scanf()`** format like:

```
%s - %d errors, %d warnings
```

The equivalent regular expression would be:

```
(\S+) - (\d+) errors, (\d+) warnings
```

search() vs. prefixmatch()

Python oferece diferentes operações primitivas baseadas em expressões regulares:

- **`re.prefixmatch()`** checks for a match only at the beginning of the string
- **`re.search()`** verifica uma correspondência em qualquer lugar na string (isso é o que o Perl faz por padrão)
- **`re.fullmatch()`** verifica toda a string para ser uma correspondência

Por exemplo:

```
>>> re.prefixmatch("c", "abcdef")  # No match
>>> re.search("c", "abcdef")       # Match
<re.Match object; span=(2, 3), match='c'>
>>> re.fullmatch("p.*n", "python") # Match
<re.Match object; span=(0, 6), match='python'>
>>> re.fullmatch("r.*n", "python") # No match
```

Expressões regulares começando com `'^'` podem ser usadas com **`search()`** para restringir a correspondência no início da string:

```
>>> re.prefixmatch("c", "abcdef")  # No match
>>> re.search("^c", "abcdef")      # No match
>>> re.search("^a", "abcdef")      # Match
<re.Match object; span=(0, 1), match='a'>
```

Note however that in **`MULTILINE`** mode **`prefixmatch()`** only matches at the beginning of the string, whereas using **`search()`** with a regular expression

beginning with `'^'` will match at the beginning of each line.

```
>>> re.prefixmatch("X", "A\nB\nX", re.MULTILINE)  # No match
>>> re.search("^X", "A\nB\nX", re.MULTILINE)  # Match
<re.Match object; span=(4, 5), match='X'>
```

prefixmatch() vs. match()

Why is the **`match()`** function and method discouraged in favor of the longer **`prefixmatch()`** spelling?

Many other languages have gained regex support libraries since regular expressions were added to Python. However in the most popular of those, they use the term *match* in their APIs to mean the unanchored behavior provided in Python by **`search()`**. Thus use of the plain term *match* can be unclear to those used to other languages when reading or writing code and not familiar with the Python API's divergence from what otherwise become the industry norm.

Quoting from the Zen Of Python (`python3 -m this`): *"Explicit is better than implicit"*. Anyone reading the name **`prefixmatch()`** is likely to understand the intended semantics. When reading **`match()`** there remains a seed of doubt about the intended behavior to anyone not already familiar with this old Python gotcha.

We **do not** plan to remove the older **`match()`** name, as it has been used in code for over 30 years. It has been soft deprecated: code supporting older versions of Python should continue to use **`match()`**, while new code should prefer **`prefixmatch()`**.

Adicionado na versão 3.15: **`prefixmatch()`**

Suavemente descontinuado desde a versão 3.15: **`match()`**

Making a phonebook

`split()` divide uma string em uma lista delimitada pelo padrão passado. O método é inestimável para converter dados textuais em estruturas de dados

que podem ser facilmente lidas e modificadas pelo Python, conforme demonstrado no exemplo a seguir que cria uma lista telefônica.

Primeiro, aqui está a entrada. Normalmente pode vir de um arquivo, aqui estamos usando a sintaxe de string entre aspas triplas.

```
>>> texto = """Ross McFluff: 834.345.1254 155 Elm Street
...
... Ronald Heathmore: 892.345.3428 436 Finley Avenue
... Frank Burger: 925.541.7625 662 South Dogwood Way
...
...
... Heather Albrecht: 548.326.4584 919 Park Place"""
```

Os registros são separados por uma ou mais novas linhas. Agora, convertemos a string em uma lista com cada linha não vazia tendo seu próprio registro:

```
>>> registros = re.split("\n+", texto)
>>> registros
['Ross McFluff: 834.345.1254 155 Elm Street',
'Ronald Heathmore: 892.345.3428 436 Finley Avenue',
'Frank Burger: 925.541.7625 662 South Dogwood Way',
'Heather Albrecht: 548.326.4584 919 Park Place']
```

Finalmente, divida cada registro em uma lista com nome, sobrenome, número de telefone e endereço. Usamos o parâmetro `maxsplit` de **`split()`** porque o endereço contém espaços, nosso padrão de divisão:

```
>>> [re.split(":? ", registro, maxsplit=3) for registro in regis
[['Ross', 'McFluff', '834.345.1254', '155 Elm Street'],
['Ronald', 'Heathmore', '892.345.3428', '436 Finley Avenue'],
['Frank', 'Burger', '925.541.7625', '662 South Dogwood Way'],
['Heather', 'Albrecht', '548.326.4584', '919 Park Place']]
```

O padrão `:?` corresponde ao caractere de dois pontos após o sobrenome, de modo que não ocorre na lista de resultados. Com um `maxsplit` de `4`, podemos separar o número da casa do nome da rua:

```
>>> [re.split(":? ", registro, maxsplit=4) for registro in regis
[['Ross', 'McFluff', '834.345.1254', '155', 'Elm Street'],
['Ronald', 'Heathmore', '892.345.3428', '436', 'Finley Avenue'],
['Frank', 'Burger', '925.541.7625', '662', 'South Dogwood Way'],
['Heather', 'Albrecht', '548.326.4584', '919', 'Park Place']]
```

Text munging

`sub()` substitui cada ocorrência de um padrão por uma string ou o resultado de uma função. Este exemplo demonstra o uso de **`sub()`** com uma função para "mastigar" o texto ou aleatorizar a ordem de todos os caracteres em cada palavra de uma frase, exceto o primeiro e o último caracteres:

```
>>> def repl(m):
...     inner_word = list(m.group(2))
...     random.shuffle(inner_word)
...     return m.group(1) + "".join(inner_word) + m.group(3)
...
>>> text = "Professor Masanori, favor comunicar suas ausências i
>>> re.sub(r"(\w)(\w+)(\w)", repl, text)
'Psesforor Mranoasi, favor cmounacir suas acaêiusns idaaeemttnmi
>>> re.sub(r"(\w)(\w+)(\w)", repl, text)
'Prfseosor Monsaari, fovar camonucir saus acuêsains ientdaammeti
```

Finding all adverbs

`findall()` corresponde *todas* as ocorrências de um padrão, não apenas a primeira como `search()` faz. Por exemplo, se uma pessoa deseja encontrar todos os advérbios em algum texto, ela pode usar **`findall()`** da seguinte maneira:

```
>>> text = "Ele estava cuidadosamente disfarçado, mas foi rapida
>>> re.findall(r"\w+mente\b", text)
['cuidadosamente', 'rapidamente']
```

Finding all adverbs and their positions

Se alguém deseja obter mais informações sobre todas as correspondências de um padrão além do texto correspondido, **`finditer()`** é útil, pois fornece objetos **`Match`** em vez de strings. Continuando com o exemplo anterior, se um escritor quisesse encontrar todos os advérbios *e suas posições* em algum texto, ele usaria **`finditer()`** da seguinte maneira:

```
>>> text = "Ele estava cuidadosamente disfarçado, mas foi rapida
>>> for m in re.finditer(r"\w+mente\b", text):
... print('%02d-%02d: %s' % (m.start(), m.end(), m.group(0)))
11-25: cuidadosamente
46-57: rapidamente
```

Raw string notation

A notação de string bruta (`r"texto"`) mantém as expressões regulares sãs. Sem ela, cada contrabarra (`'\'`) em uma expressão regular teria que ser prefixada com outra para ter seu efeito de caractere especial anulado. Por exemplo, as duas linhas de código a seguir são funcionalmente idênticas:

```
>>> re.search(r"\W(.)\1\W", " ff ")
<re.Match object; span=(0, 4), match=' ff '>
>>> re.search("\\W(.)\\1\\W", " ff ")
<re.Match object; span=(0, 4), match=' ff '>
```

Quando se deseja corresponder a uma contrabarra literal, ela deve ser prefixada com outra contrabarra na expressão regular. Com a notação de string bruta, isso significa `r"\\"`. Sem a notação de string bruta, deve-se usar `"\\\\"`, tornando as seguintes linhas de código funcionalmente idênticas:

```
>>> re.search(r"\\", r"\\")
<re.Match object; span=(0, 1), match='\\'>
>>> re.search("\\\\", r"\\")
<re.Match object; span=(0, 1), match='\\'>
```

Writing a tokenizer

Um tokenizador ou scanner [https://pt.wikipedia.org/wiki/Análise_léxica] analisa uma string para categorizar grupos de caracteres. Este é um primeiro passo útil para escrever um compilador ou interpretador.

As categorias de texto são especificadas com expressões regulares. A técnica é combiná-las em uma única expressão regular mestre e fazer um laço em correspondências sucessivas:

```
from typing import NamedTuple
import re

class Token(NamedTuple):
    type: str
    value: int | float | str
    line: int
    column: int

def tokenize(code):
    keywords = {'IF', 'THEN', 'ENDIF', 'FOR', 'NEXT', 'GOSUB', '
    token_specification = [
        ('NUMBER',   r'\d+(\.\d*)?'),  # Integer or decimal numb
        ('ASSIGN',   r':='),           # Assignment operator
        ('END',      r';'),            # Statement terminator
        ('ID',       r'[A-Za-z]+'),    # Identifiers
        ('OP',       r'[+\-*/]'),      # Arithmetic operators
        ('NEWLINE',  r'\n'),           # Line endings
        ('SKIP',     r'[ \t]+'),       # Skip over spaces and ta
        ('MISMATCH', r'.'),            # Any other character
    ]
    tok_regex = '|'.join('(?P<%s>%s)' % pair for pair in token_s
    line_num = 1
    line_start = 0
    for mo in re.finditer(tok_regex, code):
        kind = mo.lastgroup
        value = mo.group()
        column = mo.start() - line_start
        if kind == 'NUMBER':
            value = float(value) if '.' in value else int(value)
        elif kind == 'ID' and value in keywords:
            kind = value
        elif kind == 'NEWLINE':
            line_start = mo.end()
```

```
            line_num += 1
            continue
        elif kind == 'SKIP':
            continue
        elif kind == 'MISMATCH':
            raise RuntimeError(f'{value!r} unexpected on line {]
        yield Token(kind, value, line_num, column)

statements = '''
    IF quantity THEN
        total := total + price * quantity;
        tax := price * 0.05;
    ENDIF;
'''

for token in tokenize(statements):
    print(token)
```

O tokenizador produz a seguinte saída:

```
Token(type='IF', value='IF', line=2, column=4)
Token(type='ID', value='quantity', line=2, column=7)
Token(type='THEN', value='THEN', line=2, column=16)
Token(type='ID', value='total', line=3, column=8)
Token(type='ASSIGN', value=':=', line=3, column=14)
Token(type='ID', value='total', line=3, column=17)
Token(type='OP', value='+', line=3, column=23)
Token(type='ID', value='price', line=3, column=25)
Token(type='OP', value='*', line=3, column=31)
Token(type='ID', value='quantity', line=3, column=33)
Token(type='END', value=';', line=3, column=41)
Token(type='ID', value='tax', line=4, column=8)
Token(type='ASSIGN', value=':=', line=4, column=12)
Token(type='ID', value='price', line=4, column=15)
Token(type='OP', value='*', line=4, column=21)
Token(type='NUMBER', value=0.05, line=4, column=23)
Token(type='END', value=';', line=4, column=27)
Token(type='ENDIF', value='ENDIF', line=5, column=4)
Token(type='END', value=';', line=5, column=9)
```

[Frie09]

Friedl, Jeffrey. Mastering Regular Expressions. 3ª ed., O'Reilly Media, 2009. A terceira edição do livro não cobre mais o Python, mas a primeira edição cobriu a escrita de bons padrões de expressão regular em grandes detalhes.

`difflib` — Helpers for computing deltas

Código-fonte: Lib/difflib.py [https://github.com/python/cpython/tree/3.15/Lib/difflib.py]

This module provides classes and functions for comparing sequences. Most of them compare sequences of text lines (for example lists of strings, or file objects) and produce *diffs* – reports on the differences. Diffs can be produced in various formats, including HTML and context and unified diffs – formats produced by tools like *diff* *[https://manpages.debian.org/diff(1)]* and *git diff* *[https://manpages.debian.org/git-diff(1)]*.

Comparisons are done using a matching algorithm implemented in `SequenceMatcher` – a flexible class for comparing pairs of sequences of any type, not just text, so long as the sequence elements are hashable.

Junk heuristic

`difflib` uses a *junk* heuristic: some items are deemed to be *junk*, and ignored when searching for similarities. Ideally, these are uninteresting or common items, such as blank lines or whitespace.

This heuristic can speed the algorithm up (because it reduces the number of possible combinations) and it can produce results that are more understandable for humans (typically breaking on whitespace). But it can also cause pathological cases:

- Inappropriately chosen junk items can cause an unexpectedly **large** (but still correct) result.
- The default heuristic is **asymmetric**: only the second sequence is inspected when determining what is considered junk, so comparing A to B can give different results than comparing B to A and reversing the result.

By default, if the second input sequence is at least 200 items long, items that account for more than 1% it are considered *junk*.

Depending on your data, you should consider turning this heuristic off (setting `SequenceMatcher`'s *autojunk* argument to `False`) or tuning it (using the *isjunk* argument, perhaps to one of the predefined functions).

The `difflib` algorithm

The algorithm used in `SequenceMatcher` predates, and is a little fancier than, an algorithm published in the late 1980s by Ratcliff and Obershelp under the hyperbolic name "gestalt pattern matching." The idea is to find the longest contiguous subsequence common to both inputs, then recursively handle the pieces of the sequences to the left and to the right of the matching subsequence.

Ver também

Pattern Matching: The Gestalt Approach
[https://jacobfilipp.com/DrDobbs/articles/DDJ/1988/8807/8807c/8807c.htm]

Discussion of a similar algorithm by John W. Ratcliff and D. E. Metzener. This was published in Dr. Dobb's Journal in July, 1988.

As an extension to the Ratcliff and Obershelp algorithm, `difflib` searches for the longest *junk-free* contiguous subsequence. See the Junk heuristic section for details.

Detalhes da implementação do CPython: Timing

The basic Ratcliff-Obershelp algorithm is cubic time in the worst case and quadratic time in the expected case. `difflib`'s algorithm is quadratic time for the worst case and has expected-case behavior dependent in a complicated way on how many elements the sequences have in common; best case time is linear.

Diff generation

class difflib.Differ

This is a class for comparing sequences of lines of text, and producing human-readable differences or deltas. Differ uses `SequenceMatcher` both to compare sequences of lines, and to compare sequences of characters within similar (near-matching) lines.

Each line of a `Differ` delta begins with a two-letter code:

Código	Significado
`'- '`	line unique to sequence 1
`'+ '`	line unique to sequence 2
`'  '`	line common to both sequences
`'? '`	line not present in either input sequence

Lines beginning with '`?`' attempt to guide the eye to intraline differences, and were not present in either input sequence. These lines can be confusing if the sequences contain whitespace characters, such as spaces, tabs or line breaks.

Note that `Differ`-generated deltas make no claim to be **minimal** diffs. To the contrary, minimal diffs are often counter-intuitive for humans, because they synch up anywhere possible, sometimes at accidental matches 100 pages apart. Restricting synch points to contiguous matches preserves some notion of locality, at the occasional cost of producing a longer diff.

The `Differ` class has this constructor:

__init__(*linejunk=None*, *charjunk=None*)

Optional keyword parameters *linejunk* and *charjunk* are for filter functions (or `None`):

linejunk: A function that accepts a single string argument, and returns true if the string is junk. The default is `None`, meaning that no line is considered junk.

charjunk: A function that accepts a single character argument (a string of length 1), and returns true if the character is junk. The default is `None`, meaning that no character is considered junk.

These junk-filtering functions speed up matching to find differences and do not cause any differing lines or characters to be ignored. Read the description of the **`find_longest_match()`** method's *isjunk* parameter for an explanation.

`Differ` objects are used (deltas generated) via a single method:

compare(*a*, *b*)

Compare two sequences of lines, and generate the delta (a sequence of lines).

Each sequence must contain individual single-line strings ending with newlines. Such sequences can be obtained from the **`readlines()`** method of file-like objects. The generated delta also consists of newline-terminated strings, ready to be printed as-is via the **`writelines()`** method of a file-like object.

class difflib.HtmlDiff

This class can be used to create an HTML table (or a complete HTML file containing the table) showing a side by side, line by line comparison of text with inter-line and intra-line change highlights. The table can be generated in either full or contextual difference mode.

Aviso

The trailing newlines get stripped before the diff, so the result can be incomplete. See gh-71896 [https://github.com/python/cpython/issues/71896] for

details.

The constructor for this class is:

__init__(*tabsize=8*, *wrapcolumn=None*, *linejunk=None*, *charjunk=IS_CHARACTER_JUNK*)

Initializes instance of **`HtmlDiff`**.

tabsize is an optional keyword argument to specify tab stop spacing and defaults to `8`.

wrapcolumn is an optional keyword to specify column number where lines are broken and wrapped, defaults to `None` where lines are not wrapped.

linejunk and *charjunk* are optional keyword arguments passed into **`ndiff()`** (used by **`HtmlDiff`** to generate the side by side HTML differences). See **`ndiff()`** documentation for argument default values and descriptions.

The following methods are public:

make_file(*fromlines*, *tolines*, *fromdesc=''*, *todesc=''*, *context=False*, *numlines=5*, ***, *charset='utf-8'*)

Compares *fromlines* and *tolines* (lists of strings) and returns a string which is a complete HTML file containing a table showing line by line differences with inter-line and intra-line changes highlighted.

fromdesc and *todesc* are optional keyword arguments to specify from/to file column header strings (both default to an empty string).

context and *numlines* are both optional keyword arguments. Set *context* to `True` when contextual differences are to be shown, else the default is `False` to show the full files. *numlines* defaults to `5`. When *context* is `True` *numlines* controls the number of context lines which surround the difference highlights. When *context* is `False` *numlines* controls the number of lines which are shown before a

difference highlight when using the “next” hyperlinks (setting to zero would cause the “next” hyperlinks to place the next difference highlight at the top of the browser without any leading context).

Nota

fromdesc and *todesc* are interpreted as unescaped HTML and should be properly escaped while receiving input from untrusted sources.

Alterado na versão 3.5: charset keyword-only argument was added. The default charset of HTML document changed from `'ISO-8859-1'` to `'utf-8'`.

make_table(*fromlines*, *tolines*, *fromdesc=''*, *todesc=''*, *context=False*, *numlines=5*)

Compares *fromlines* and *tolines* (lists of strings) and returns a string which is a complete HTML table showing line by line differences with inter-line and intra-line changes highlighted.

The arguments for this method are the same as those for the `make_file()` method.

difflib.context_diff(*a*, *b*, *fromfile=''*, *tofile=''*, *fromfiledate=''*, *tofiledate=''*, *n=3*, *lineterm='\n'*)

Compare *a* and *b* (lists of strings); return a delta (a generator generating the delta lines) in context diff format.

Context diffs are a compact way of showing just the lines that have changed plus a few lines of context. The changes are shown in a before/after style. The number of context lines is set by *n* which defaults to three.

By default, the diff control lines (those with `***` or `---`) are created with a trailing newline. This is helpful so that inputs created from `io.IOBase.readlines()` result in diffs that are suitable for use with

`io.IOBase.writelines()` since both the inputs and outputs have trailing newlines.

For inputs that do not have trailing newlines, set the *lineterm* argument to `""` so that the output will be uniformly newline free.

The context diff format normally has a header for filenames and modification times. Any or all of these may be specified using strings for *fromfile*, *tofile*, *fromfiledate*, and *tofiledate*. The modification times are normally expressed in the ISO 8601 format. If not specified, the strings default to blanks.

```
>>> import sys
>>> from difflib import *
>>> s1 = ['bacon\n', 'eggs\n', 'ham\n', 'guido\n']
>>> s2 = ['python\n', 'eggy\n', 'hamster\n', 'guido\n']
>>> sys.stdout.writelines(context_diff(s1, s2, fromfile='befo
...                       tofile='after.py'))
*** before.py
--- after.py
***************
*** 1,4 ****
! bacon
! eggs
! ham
  guido
--- 1,4 ----
! python
! eggy
! hamster
  guido
```

See A command-line interface to difflib for a more detailed example.

difflib.get_close_matches(*word*, *possibilities*, *n=3*, *cutoff=0.6*)

Return a list of the best “good enough” matches. *word* is a sequence for which close matches are desired (typically a string), and *possibilities* is a list of sequences against which to match *word* (typically a list of strings).

Optional argument *n* (default `3`) is the maximum number of close matches to return; *n* must be greater than `0`.

Optional argument *cutoff* (default `0.6`) is a float in the range [0, 1]. Possibilities that don't score at least that similar to *word* are ignored.

The best (no more than *n*) matches among the possibilities are returned in a list, sorted by similarity score, most similar first.

```
>>> get_close_matches('appel', ['ape', 'apple', 'peach', 'pup
['apple', 'ape']
>>> import keyword
>>> get_close_matches('wheel', keyword.kwlist)
['while']
>>> get_close_matches('pineapple', keyword.kwlist)
[]
>>> get_close_matches('accept', keyword.kwlist)
['except']
```

difflib.ndiff(*a*, *b*, *linejunk=None*, *charjunk=IS_CHARACTER_JUNK*)

Compare *a* and *b* (lists of strings); return a **`Differ`**-style delta (a generator generating the delta lines).

Optional keyword parameters *linejunk* and *charjunk* are filtering functions (or `None`):

linejunk: A function that accepts a single string argument, and returns true if the string is junk, or false if not. The default is `None`. There is also a module-level function **`IS_LINE_JUNK()`**, which filters out lines without visible characters, except for at most one hash character (`'#'`) – however the underlying **`SequenceMatcher`** class does a dynamic analysis of which lines are so frequent as to constitute noise, and this usually works better than using this function.

charjunk: A function that accepts a character (a string of length 1), and returns if the character is junk, or false if not. The default is module-level function **`IS_CHARACTER_JUNK()`**, which filters out whitespace characters (a blank or tab; it's a bad idea to include newline in this!).

```
>>> diff = ndiff('one\ntwo\nthree\n'.splitlines(keepends=True
...              'ore\ntree\nemu\n'.splitlines(keepends=True)
>>> print(''.join(diff), end="")
- one
?  ^
+ ore
?  ^
- two
- three
?  -
+ tree
+ emu
```

difflib.restore(*sequence*, *which*)

Return one of the two sequences that generated a delta.

Given a *sequence* produced by `Differ.compare()` or `ndiff()`, extract lines originating from file 1 or 2 (parameter *which*), stripping off line prefixes.

Exemplo:

```
>>> diff = ndiff('one\ntwo\nthree\n'.splitlines(keepends=True
...              'ore\ntree\nemu\n'.splitlines(keepends=True)
>>> diff = list(diff) # materialize the generated delta into
>>> print(''.join(restore(diff, 1)), end="")
one
two
three
>>> print(''.join(restore(diff, 2)), end="")
ore
tree
emu
```

difflib.unified_diff(*a*, *b*, *fromfile=''*, *tofile=''*, *fromfiledate=''*, *tofiledate=''*, *n=3*, *lineterm='\n'*, ***, *color=False*)

Compare *a* and *b* (lists of strings); return a delta (a generator generating the delta lines) in unified diff format.

Unified diffs are a compact way of showing just the lines that have changed plus a few lines of context. The changes are shown in an inline style (instead of separate before/after blocks). The number of context lines is set by *n* which defaults to three.

By default, the diff control lines (those with `---`, `+++`, or `@@`) are created with a trailing newline. This is helpful so that inputs created from **`io.IOBase.readlines()`** result in diffs that are suitable for use with **`io.IOBase.writelines()`** since both the inputs and outputs have trailing newlines.

For inputs that do not have trailing newlines, set the *lineterm* argument to `""` so that the output will be uniformly newline free.

Set *color* to `True` to enable output in color, similar to **git diff --color**. Even if enabled, it can be controlled using environment variables.

The unified diff format normally has a header for filenames and modification times. Any or all of these may be specified using strings for *fromfile*, *tofile*, *fromfiledate*, and *tofiledate*. The modification times are normally expressed in the ISO 8601 format. If not specified, the strings default to blanks.

```
>>> s1 = ['bacon\n', 'eggs\n', 'ham\n', 'guido\n']
>>> s2 = ['python\n', 'eggy\n', 'hamster\n', 'guido\n']
>>> sys.stdout.writelines(unified_diff(s1, s2, fromfile='befo
--- before.py
+++ after.py
@@ -1,4 +1,4 @@
-bacon
-eggs
-ham
+python
+eggy
+hamster
 guido
```

See A command-line interface to difflib for a more detailed example.

Alterado na versão 3.15: Adicionado o parâmetro *color*.

difflib.diff_bytes(*dfunc*, *a*, *b*, *fromfile=b''*, *tofile=b''*, *fromfiledate=b''*, *tofiledate=b''*, *n=3*, *lineterm=b'\n'*)

Compare *a* and *b* (lists of bytes objects) using *dfunc*; yield a sequence of delta lines (also bytes) in the format returned by *dfunc*. *dfunc* must be a callable, typically either `unified_diff()` or `context_diff()`.

Allows you to compare data with unknown or inconsistent encoding. All inputs except *n* must be bytes objects, not str. Works by losslessly converting all inputs (except *n*) to str, and calling `dfunc(a, b, fromfile, tofile, fromfiledate, tofiledate, n, lineterm)`. The output of *dfunc* is then converted back to bytes, so the delta lines that you receive have the same unknown/inconsistent encodings as *a* and *b*.

Adicionado na versão 3.5.

Junk definition functions

difflib.IS_LINE_JUNK(*line*)

Return `True` for ignorable lines. The line *line* is ignorable if *line* is blank or contains a single `'#'`, otherwise it is not ignorable. Used as a default for parameter *linejunk* in `ndiff()` in older versions.

difflib.IS_CHARACTER_JUNK(*ch*)

Return `True` for ignorable characters. The character *ch* is ignorable if *ch* is a space or tab, otherwise it is not ignorable. Used as a default for parameter *charjunk* in `ndiff()`.

SequenceMatcher objects

class difflib.SequenceMatcher(*isjunk=None*, *a=''*, *b=''*, *autojunk=True*)

Optional argument *isjunk* must be `None` (the default) or a one-argument function that takes a sequence element and returns true if and only if the element is “junk” and should be ignored. Passing `None` for *isjunk* is

equivalent to passing `lambda x: False`; in other words, no elements are ignored. For example, pass:

```
lambda x: x in " \t"
```

if you're comparing lines as sequences of characters, and don't want to synch up on blanks or hard tabs.

The optional arguments *a* and *b* are sequences to be compared; both default to empty strings. The elements of both sequences must be hashable.

The optional argument *autojunk* can be used to disable the automatic junk heuristic.

Alterado na versão 3.2: Added the *autojunk* parameter.

SequenceMatcher objects get three data attributes: *bjunk* is the set of elements of *b* for which *isjunk* is `True`; *bpopular* is the set of non-junk elements considered popular by the heuristic (if it is not disabled); *b2j* is a dict mapping the remaining elements of *b* to a list of positions where they occur. All three are reset whenever *b* is reset with **`set_seqs()`** or **`set_seq2()`**.

Adicionado na versão 3.2: The *bjunk* and *bpopular* attributes.

`SequenceMatcher` objects have the following methods:

set_seqs(*a*, *b*)

Set the two sequences to be compared.

`SequenceMatcher` computes and caches detailed information about the second sequence, so if you want to compare one sequence against many sequences, use **`set_seq2()`** to set the commonly used sequence once and call **`set_seq1()`** repeatedly, once for each of the other sequences.

set_seq1(*a*)

Set the first sequence to be compared. The second sequence to be compared is not changed.

set_seq2(*b*)

Set the second sequence to be compared. The first sequence to be compared is not changed.

find_longest_match(*alo=0*, *ahi=None*, *blo=0*, *bhi=None*)

Find longest matching block in `a[alo:ahi]` and `b[blo:bhi]`.

If *isjunk* was omitted or `None`, **`find_longest_match()`** returns `(i, j, k)` such that `a[i:i+k]` is equal to `b[j:j+k]`, where `alo <= i <= i+k <= ahi` and `blo <= j <= j+k <= bhi`. For all `(i', j', k')` meeting those conditions, the additional conditions `k >= k'`, `i <= i'`, and if `i == i'`, `j <= j'` are also met. In other words, of all maximal matching blocks, return one that starts earliest in *a*, and of all those maximal matching blocks that start earliest in *a*, return the one that starts earliest in *b*.

```
>>> s = SequenceMatcher(None, " abcd", "abcd abcd")
>>> s.find_longest_match(0, 5, 0, 9)
Match(a=0, b=4, size=5)
```

If *isjunk* was provided, first the longest matching block is determined as above, but with the additional restriction that no junk element appears in the block. Then that block is extended as far as possible by matching (only) junk elements on both sides. So the resulting block never matches on junk except as identical junk happens to be adjacent to an interesting match.

Here's the same example as before, but considering blanks to be junk. That prevents `' abcd'` from matching the `' abcd'` at the tail end of the second sequence directly. Instead only the `'abcd'` can match, and matches the leftmost `'abcd'` in the second sequence:

```
>>> s = SequenceMatcher(lambda x: x==" ", " abcd", "abcd a
>>> s.find_longest_match(0, 5, 0, 9)
Match(a=1, b=0, size=4)
```

If no blocks match, this returns `(alo, blo, 0)`.

This method returns a named tuple `Match(a, b, size)`.

Alterado na versão 3.9: Added default arguments.

get_matching_blocks()

Return list of triples describing non-overlapping matching subsequences. Each triple is of the form `(i, j, n)`, and means that `a[i:i+n] == b[j:j+n]`. The triples are monotonically increasing in *i* and *j*.

The last triple is a dummy, and has the value `(len(a), len(b), 0)`. It is the only triple with `n == 0`. If `(i, j, n)` and `(i', j', n')` are adjacent triples in the list, and the second is not the last triple in the list, then `i+n < i'` or `j+n < j'`; in other words, adjacent triples always describe non-adjacent equal blocks.

```
>>> s = SequenceMatcher(None, "abxcd", "abcd")
>>> s.get_matching_blocks()
[Match(a=0, b=0, size=2), Match(a=3, b=2, size=2), Match(a
```

get_opcodes()

Return list of 5-tuples describing how to turn *a* into *b*. Each tuple is of the form `(tag, i1, i2, j1, j2)`. The first tuple has `i1 == j1 == 0`, and remaining tuples have *i1* equal to the *i2* from the preceding tuple, and, likewise, *j1* equal to the previous *j2*.

The *tag* values are strings, with these meanings:

Valor	Significado
`'replace'`	`a[i1:i2]` should be replaced by `b[j1:j2]`.
`'delete'`	`a[i1:i2]` should be deleted. Note that `j1 == j2` in this case.

Valor	Significado
`'insert'`	`b[j1:j2]` should be inserted at `a[i1:i1]`. Note that `i1 == i2` in this case.
`'equal'`	`a[i1:i2] == b[j1:j2]` (the sub-sequences are equal).

Por exemplo:

```
>>> a = "qabxcd"
>>> b = "abycdf"
>>> s = SequenceMatcher(None, a, b)
>>> for tag, i1, i2, j1, j2 in s.get_opcodes():
...     print('{:7}   a[{}:{}] --> b[{}:{}] {!r:>8} --> {
...         tag, i1, i2, j1, j2, a[i1:i2], b[j1:j2]))
delete    a[0:1] --> b[0:0]      'q' --> ''
equal     a[1:3] --> b[0:2]     'ab' --> 'ab'
replace   a[3:4] --> b[2:3]      'x' --> 'y'
equal     a[4:6] --> b[3:5]     'cd' --> 'cd'
insert    a[6:6] --> b[5:6]       '' --> 'f'
```

get_grouped_opcodes(*n=3*)

Return a generator of groups with up to *n* lines of context.

Starting with the groups returned by **`get_opcodes()`**, this method splits out smaller change clusters and eliminates intervening ranges which have no changes.

The groups are returned in the same format as **`get_opcodes()`**.

ratio()

Return a measure of the sequences' similarity as a float in the range [0, 1].

Where T is the total number of elements in both sequences, and M is the number of matches, this is 2.0*M / T. Note that this is `1.0` if the sequences are identical, and `0.0` if they have nothing in common.

This is expensive to compute if `get_matching_blocks()` or `get_opcodes()` hasn't already been called, in which case you may want to try `quick_ratio()` or `real_quick_ratio()` first to get an upper bound.

quick_ratio()

Return an upper bound on `ratio()` relatively quickly.

real_quick_ratio()

Return an upper bound on `ratio()` very quickly.

The three methods that return the ratio of matching to total characters can give different results due to differing levels of approximation, although `quick_ratio()` and `real_quick_ratio()` are always at least as large as `ratio()`:

```
>>> s = SequenceMatcher(None, "abcd", "bcde")
>>> s.ratio()
0.75
>>> s.quick_ratio()
0.75
>>> s.real_quick_ratio()
1.0
```

Exemplos

SequenceMatcher examples

This example compares two strings, considering blanks to be "junk":

```
>>> s = SequenceMatcher(lambda x: x == " ",
...                     "private Thread currentThread;",
...                     "private volatile Thread currentThread;'
```

`ratio()` returns a float in [0, 1], measuring the similarity of the sequences. As a rule of thumb, a **`ratio()`** value over 0.6 means the sequences are close matches:

```
>>> print(round(s.ratio(), 3))
0.866
```

If you're only interested in where the sequences match, `get_matching_blocks()` is handy:

```
>>> for block in s.get_matching_blocks():
...     print("a[%d] and b[%d] match for %d elements" % block)
a[0] and b[0] match for 8 elements
a[8] and b[17] match for 21 elements
a[29] and b[38] match for 0 elements
```

Note that the last tuple returned by `get_matching_blocks()` is always a dummy, `(len(a), len(b), 0)`, and this is the only case in which the last tuple element (number of elements matched) is `0`.

If you want to know how to change the first sequence into the second, use `get_opcodes()`:

```
>>> for opcode in s.get_opcodes():
...     print("%6s a[%d:%d] b[%d:%d]" % opcode)
 equal a[0:8] b[0:8]
insert a[8:8] b[8:17]
 equal a[8:29] b[17:38]
```

Ver também

- The `get_close_matches()` function in this module which shows how simple code building on `SequenceMatcher` can be used to do useful work.
- Simple version control recipe [https://code.activestate.com/recipes/576729-simple-version-control/] for a small application built with `SequenceMatcher`.

Differ example

This example compares two texts. First we set up the texts, sequences of individual single-line strings ending with newlines (such sequences can also

be obtained from the **readlines()** method of file-like objects):

```
>>> text1 = '''  1. Beautiful is better than ugly.
...   2. Explicit is better than implicit.
...   3. Simple is better than complex.
...   4. Complex is better than complicated.
... '''.splitlines(keepends=True)
>>> len(text1)
4
>>> text1[0][-1]
'\n'
>>> text2 = '''  1. Beautiful is better than ugly.
...   3.   Simple is better than complex.
...   4. Complicated is better than complex.
...   5. Flat is better than nested.
... '''.splitlines(keepends=True)
```

Next we instantiate a Differ object:

```
>>> d = Differ()
```

Note that when instantiating a **Differ** object we may pass functions to filter out line and character "junk." See the **Differ()** constructor for details.

Finally, we compare the two:

```
>>> result = list(d.compare(text1, text2))
```

`result` is a list of strings, so let's pretty-print it:

```
>>> from pprint import pprint
>>> pprint(result)
['    1. Beautiful is better than ugly.\n',
 '-   2. Explicit is better than implicit.\n',
 '-   3. Simple is better than complex.\n',
 '+   3.   Simple is better than complex.\n',
 '?     ++\n',
 '-   4. Complex is better than complicated.\n',
 '?            ^                     ---- ^\n',
 '+   4. Complicated is better than complex.\n',
 '?           ++++ ^                     ^\n',
 '+   5. Flat is better than nested.\n']
```

As a single multi-line string it looks like this:

```
>>> import sys
>>> sys.stdout.writelines(result)
    1. Beautiful is better than ugly.
-   2. Explicit is better than implicit.
-   3. Simple is better than complex.
+   3.   Simple is better than complex.
?     ++
-   4. Complex is better than complicated.
?            ^                     ---- ^
+   4. Complicated is better than complex.
?           ++++ ^                      ^
+   5. Flat is better than nested.
```

A command-line interface to difflib

This example shows how to use difflib to create a `diff`-like utility.

```
""" Command-line interface to difflib.py providing diffs in four

* ndiff:    lists every line and highlights interline changes.
* context:  highlights clusters of changes in a before/after for
* unified:  highlights clusters of changes in an inline format.
* html:     generates side by side comparison with change highli

"""

import sys, os, difflib, argparse
import datetime as dt

def file_mtime(path):
    t = dt.datetime.fromtimestamp(os.stat(path).st_mtime,
                                  dt.timezone.utc)
    return t.astimezone().isoformat()

def main():

    parser = argparse.ArgumentParser()
    parser.add_argument('-c', action='store_true', default=False
                        help='Produce a context format diff (def
    parser.add_argument('-u', action='store_true', default=False
```

```
                        help='Produce a unified format diff')
    parser.add_argument('-m', action='store_true', default=False
                        help='Produce HTML side by side diff '
                             '(can use -c and -l in conjunction)
    parser.add_argument('-n', action='store_true', default=False
                        help='Produce a ndiff format diff')
    parser.add_argument('-l', '--lines', type=int, default=3,
                        help='Set number of context lines (defau
    parser.add_argument('fromfile')
    parser.add_argument('tofile')
    options = parser.parse_args()

    n = options.lines
    fromfile = options.fromfile
    tofile = options.tofile

    fromdate = file_mtime(fromfile)
    todate = file_mtime(tofile)
    with open(fromfile) as ff:
        fromlines = ff.readlines()
    with open(tofile) as tf:
        tolines = tf.readlines()

    if options.u:
        diff = difflib.unified_diff(fromlines, tolines, fromfile
    elif options.n:
        diff = difflib.ndiff(fromlines, tolines)
    elif options.m:
        diff = difflib.HtmlDiff().make_file(fromlines,tolines,fr
    else:
        diff = difflib.context_diff(fromlines, tolines, fromfile

    sys.stdout.writelines(diff)

if __name__ == '__main__':
    main()
```

ndiff example

This example shows how to use `difflib.ndiff()`.

```
"""ndiff [-q] file1 file2
    or
ndiff (-r1 | -r2) < ndiff_output > file1_or_file2

Print a human-friendly file difference report to stdout.  Both i
and intra-line differences are noted.  In the second form, recre
(-r1) or file2 (-r2) on stdout, from an ndiff report on stdin.

In the first form, if -q ("quiet") is not specified, the first t
of output are

-: file1
+: file2

Each remaining line begins with a two-letter code:

    "- "    line unique to file1
    "+ "    line unique to file2
    "  "    line common to both files
    "? "    line not present in either input file

Lines beginning with "? " attempt to guide the eye to intraline
differences, and were not present in either input file.  These l
confusing if the source files contain tab characters.

The first file can be recovered by retaining only lines that beg
"  " or "- ", and deleting those 2-character prefixes; use ndiff

The second file can be recovered similarly, but by retaining onl
"+ " lines; use ndiff with -r2; or, on Unix, the second file car
recovered by piping the output through

    sed -n '/^[+ ] /s/^..//p'
"""

__version__ = 1, 7, 0

import difflib, sys

def fail(msg):
    out = sys.stderr.write
    out(msg + "\n\n")
    out(__doc__)
```

```
    return 0

# open a file & return the file object; gripe and return 0 if it
# couldn't be opened
def fopen(fname):
    try:
        return open(fname)
    except IOError as detail:
        return fail("couldn't open " + fname + ": " + str(detail

# open two files & spray the diff to stdout; return false iff a
def fcompare(f1name, f2name):
    f1 = fopen(f1name)
    f2 = fopen(f2name)
    if not f1 or not f2:
        return 0

    a = f1.readlines(); f1.close()
    b = f2.readlines(); f2.close()
    for line in difflib.ndiff(a, b):
        print(line, end=' ')

    return 1

# crack args (sys.argv[1:] is normal) & compare;
# return false iff a problem

def main(args):
    import getopt
    try:
        opts, args = getopt.getopt(args, "qr:")
    except getopt.error as detail:
        return fail(str(detail))
    noisy = 1
    qseen = rseen = 0
    for opt, val in opts:
        if opt == "-q":
            qseen = 1
            noisy = 0
        elif opt == "-r":
            rseen = 1
            whichfile = val
    if qseen and rseen:
```

```
        return fail("can't specify both -q and -r")
    if rseen:
        if args:
            return fail("no args allowed with -r option")
        if whichfile in ("1", "2"):
            restore(whichfile)
            return 1
        return fail("-r value must be 1 or 2")
    if len(args) != 2:
        return fail("need 2 filename args")
    f1name, f2name = args
    if noisy:
        print('-:', f1name)
        print('+:', f2name)
    return fcompare(f1name, f2name)

# read ndiff output from stdin, and print file1 (which=='1') or
# file2 (which=='2') to stdout

def restore(which):
    restored = difflib.restore(sys.stdin.readlines(), which)
    sys.stdout.writelines(restored)

if __name__ == '__main__':
    main(sys.argv[1:])
```

`textwrap` — Quebra automática e preenchimento de texto

Código-fonte: Lib/textwrap.py
[https://github.com/python/cpython/tree/3.15/Lib/textwrap.py]

O módulo **`textwrap`** fornece algumas funções úteis, assim como a classe **`TextWrapper`**, que faz todo o trabalho. Se você estiver apenas preenchendo ou quebrando uma ou duas strings, as funções de conveniência deverão ser suficientes; caso contrário, você deve usar uma instância de **`TextWrapper`** para maior eficiência.

textwrap.wrap(*text*, *width=70*, ***, *initial_indent=''*, *subsequent_indent=''*, *expand_tabs=True*, *replace_whitespace=True*, *fix_sentence_endings=False*, *break_long_words=True*, *drop_whitespace=True*, *break_on_hyphens=True*, *tabsize=8*, *max_lines=None*, *placeholder=' [...]'*)

Quebra do parágrafo único em *text* (uma string) para que cada linha tenha, no máximo, *width* caracteres. Retorna uma lista de linhas de saída, sem novas linhas ao final.

Argumentos nomeados opcionais correspondem aos atributos de instância de **`TextWrapper`**, documentados abaixo.

Veja o método **`TextWrapper.wrap()`** para detalhes adicionais sobre como **`wrap()`** se comporta.

textwrap.fill(*text*, *width=70*, ***, *initial_indent=''*, *subsequent_indent=''*, *expand_tabs=True*, *replace_whitespace=True*, *fix_sentence_endings=False*, *break_long_words=True*, *drop_whitespace=True*, *break_on_hyphens=True*, *tabsize=8*, *max_lines=None*, *placeholder=' [...]'*)

Quebra o parágrafo único em *text* e retorna uma única string contendo o parágrafo quebrado. `fill()` é uma abreviação de

```
"\n".join(wrap(text, ...))
```

Em particular, `fill()` aceita exatamente os mesmos argumentos nomeados que `wrap()`.

textwrap.shorten(*text*, *width*, ***, *fix_sentence_endings=False*, *break_long_words=True*, *break_on_hyphens=True*, *placeholder=' [...]'*)

Reduz e trunca o texto *text* para que ele caiba na largura *width* fornecida.

Primeiro, o espaço em branco em *text* é reduzido (todos os espaços em branco são substituídos por espaços simples). Se o resultado couber em *width*, ele será retornado. Caso contrário, palavras suficientes serão eliminadas do final para que as palavras restantes mais o *placeholder* caibam em *largura*:

```
>>> textwrap.shorten("Olá  mundo!", width=12)
'Olá mundo!'
>>> textwrap.shorten("Olá  mundo!", width=11)
'Olá [...]'
>>> textwrap.shorten("Olá mundo", width=10, placeholder="...'
'Olá...'
```

Argumentos nomeados opcionais correspondem aos atributos de instância de `TextWrapper`, documentados abaixo. Observe que o espaço em branco é reduzido antes do texto ser passado para a função **`TextWrapper`** `fill()`, alterando assim o valor de `tabsize`, `expand_tabs`, `drop_whitespace` e `replace_whitespace` não terão efeito.

Adicionado na versão 3.4.

textwrap.dedent(*text*)

Remove qualquer espaço em branco inicial em comum de todas as linhas do *text*.

Isso pode ser usado para alinhar strings entre aspas triplas à borda esquerda da tela, enquanto ainda as apresenta no código-fonte com indentação.

Observe que tabulações e espaços são tratados como espaços em branco, mas não são iguais: as linhas `"  hello"` e `"\thello"` são consideradas como não tendo espaços em branco iniciais comuns.

Linhas contendo apenas espaços em branco são ignoradas na entrada e normalizadas para uma nova linha de um único caractere na saída.

Por exemplo:

```
def test():
    # fim da primeira linha com \ para evitar uma linha vazia
    s = '''\
    olá
      mundo
    '''
    print(repr(s))          # exibe '    olá\n      mundo\n
    print(repr(dedent(s)))  # exibe 'olá\n  mundo\n'
```

Alterado na versão 3.14: A função **`dedent()`** agora normaliza corretamente linhas em branco que contenham apenas caracteres de espaço em branco. Anteriormente, a implementação apenas normalizava linhas em branco que contivessem tabulações e espaços.

textwrap.indent(*text*, *prefix*, *predicate=None*)

Adiciona *prefix* ao início das linhas selecionadas em *text*.

As linhas são separadas chamando `text.splitlines(True)`.

Por padrão, *prefix* é adicionado a todas as linhas que não consistem apenas em espaços em branco (incluindo quaisquer finais de linha).

Por exemplo:

```
>>> s = 'olá\n\n \nmundo'
>>> indent(s, '  ')
```

```
'  olá\n\n \n  mundo'
```

O argumento opcional *predicate* pode ser usado para controlar quais linhas têm indentação. Por exemplo, é fácil adicionar *prefix* até mesmo em linhas vazias e apenas com espaços em branco:

```
>>> print(indent(s, '+ ', lambda line: True))
+ olá
+
+
+ mundo
```

Adicionado na versão 3.3.

wrap(), **fill()** e **shorten()** funcionam criando uma instância **TextWrapper** e chamando um único método nela. Essa instância não é reutilizada, então para aplicações que processam muitas strings de texto usando **wrap()** e/ou **fill()**, pode ser mais eficiente criar seu próprio objeto **TextWrapper**.

O texto é preferencialmente colocado em espaços em branco e logo após os hífens nas palavras hifenizadas; somente então palavras longas serão quebradas se necessário, a menos que **TextWrapper.break_long_words** seja definido como falso.

class textwrap.TextWrapper(**kwargs*)

O construtor **TextWrapper** aceita vários argumentos nomeados opcionais. Cada argumento nomeado corresponde a um atributo de instância, por exemplo:

```
wrapper = TextWrapper(initial_indent="* ")
```

é o mesmo que

```
wrapper = TextWrapper()
wrapper.initial_indent = "* "
```

Você pode reutilizar o mesmo objeto **TextWrapper** muitas vezes e pode alterar qualquer uma de suas opções através de atribuição direta a atributos de instância entre os usos.

Os atributos de instância `TextWrapper` (e argumentos nomeados para o construtor) são os seguintes:

width

(padrão: `70`) O comprimento máximo das linhas quebradas. Contanto que não existam palavras individuais no texto de entrada maiores que `width`, `TextWrapper` garante que nenhuma linha de saída será maior que **`width`** caracteres.

expand_tabs

(padrão: `True`) Se verdadeiro, então todos os caracteres de tabulação em *text* serão expandidos para espaços usando o método `expandtabs()` de *text*.

tabsize

(padrão: `8`) Se `expand_tabs` for verdadeiro, então todos os caracteres de tabulação em *text* serão expandidos para zero ou mais espaços, dependendo da coluna atual e do tamanho da tabulação fornecido.

Adicionado na versão 3.3.

replace_whitespace

(padrão: `True`) Se verdadeiro, após a expansão da tabulação, mas antes da quebra, o método `wrap()` substituirá cada caractere de espaço em branco por um único espaço. Os caracteres de espaço em branco substituídos são os seguintes: tabulação, nova linha, tabulação vertical, feed de formulário e retorno de carro (`'\t\n\v\f\r'`).

Nota

Se `expand_tabs` for falso e `replace_whitespace` for verdadeiro, cada caractere de tabulação será substituído por um único espaço, que *não* é o mesmo que expansão de tabulação.

Nota

Se `replace_whitespace` for falso, novas linhas podem aparecer no meio de uma linha e causar uma saída estranha. Por esta razão, o texto deve ser dividido em parágrafos (usando `str.splitlines()` ou similar) que são agrupados separadamente.

drop_whitespace

(padrão: `True`) Se verdadeiro, os espaços em branco no início e no final de cada linha (após quebra automática, mas antes do recuo) são eliminados. O espaço em branco no início do parágrafo, entretanto, não será eliminado se não houver espaço em branco após ele. Se o espaço em branco eliminado ocupar uma linha inteira, a linha inteira será eliminada.

initial_indent

(padrão: `''`) String que será anexada à primeira linha da saída com quebra de linha. Conta para o comprimento da primeira linha. A string vazia não recebe indentação.

subsequent_indent

(padrão: `''`) String que será anexada a todas as linhas da saída com quebra de linha, exceto a primeira. Conta para o comprimento de cada linha, exceto a primeira.

fix_sentence_endings

(padrão: `False`) Se verdadeiro, `TextWrapper` tenta detectar finais de sentenças e garantir que as sentenças sejam sempre separadas por exatamente dois espaços. Isso geralmente é desejado para texto em fonte monoespaçada. No entanto, o algoritmo de detecção de frase é imperfeito: ele presume que o final de uma frase consiste em uma letra minúscula seguida por uma de `'.'`, `'!'` ou `'?'`, possivelmente seguido por um de `'"'` ou `"'"`, seguido por um espaço. Um problema com este algoritmo é que ele é incapaz de detectar a diferença entre “Dr.” em

```
[...] Dr. Frankenstein's monster [...]
```

e “Spot.” em

`[...] See Spot. See Spot run [...]`

`fix_sentence_endings` é falso por padrão

Como o algoritmo de detecção de frase depende de `string.lowercase` para a definição de “letra minúscula” e de uma convenção de usar dois espaços após um ponto final para separar frases na mesma linha, ele é específico para textos em inglês.

break_long_words

(padrão: `True`) Se verdadeiro, palavras maiores que **`width`** serão quebradas para garantir que nenhuma linha seja maior que **`width`**. Se for falso, palavras longas não serão quebradas e algumas linhas poderão ser maiores que **`width`**. (Palavras longas serão colocadas sozinhas em uma linha, a fim de minimizar o valor pelo qual **`width`** é excedido.)

break_on_hyphens

(padrão: `True`) Se verdadeiro, a quebra ocorrerá preferencialmente em espaços em branco e logo após hífens em palavras compostas, como é habitual em inglês. Se for falso, apenas os espaços em branco serão considerados locais potencialmente bons para quebras de linha, mas você precisa definir **`break_long_words`** como falso se quiser palavras verdadeiramente inseparáveis. O comportamento padrão nas versões anteriores era sempre permitir a quebra de palavras hifenizadas.

max_lines

(padrão: `None`) Se não for `None`, então a saída conterá no máximo *max_lines* linhas, com *placeholder* aparecendo no final da saída.

Adicionado na versão 3.4.

placeholder

(padrão: `' [...]'`) String que aparecerá no final do texto de saída se ele tiver sido truncado.

Adicionado na versão 3.4.

`TextWrapper` também fornece alguns métodos públicos, análogos às funções de conveniência em nível de módulo:

wrap(*text*)

Quebra o parágrafo único em *text* (uma string) para que cada linha tenha no máximo `width` caracteres. Todas as opções de quebra de linha são obtidas dos atributos da instância `TextWrapper`. Retorna uma lista de linhas de saída, sem novas linhas finais. Se a saída agrupada não tiver conteúdo, a lista retornada estará vazia.

fill(*text*)

Quebra o parágrafo único em *text* e retorna uma única string contendo o parágrafo quebrado.

`unicodedata` — Banco de dados de Unicode

This module provides access to the Unicode Character Database (UCD) which defines character properties for all Unicode characters. The data contained in this database is compiled from the UCD version 17.0.0 [https://www.unicode.org/Public/17.0.0/ucd].

The module uses the same names and symbols as defined by Unicode Standard Annex #44, "Unicode Character Database" [https://www.unicode.org/reports/tr44/].

Ver também

O Unicode para mais informações sobre Unicode e como usar este módulo.

Lookup

`lookup(name)`	Look up character by name
`name(chr)`	Return the name assigned to a character

Numeric values

`decimal(chr)`	Decimal value of a character

`digit(chr)`	Digit value of a character
`numeric(chr)`	Numeric value of a character

Properties

`bidirectional(chr)`	Bidirectional class of a character
`block(chr)`	Unicode block of a character
`category(chr)`	General category of a character
`combining(chr)`	Canonical combining class of a character
`decomposition(chr)`	Character decomposition mapping
`east_asian_width(chr)`	East Asian width of a character
`extended_pictographic(chr)`	Check if a character has the Extended_Pictographic property
`grapheme_cluster_break(chr)`	Grapheme_Cluster_Break property of a character
`indic_conjunct_break(chr)`	Indic_Conjunct_Break property of a character

`isxidcontinue(chr)`	Check if a character is a valid identifier continuation
`isxidstart(chr)`	Check if a character is a valid identifier start
`mirrored(chr)`	Mirrored property of a character
Normalization	
`normalize(form, unistr)`	Return the normalized form of a string
`is_normalized(form, unistr)`	Check if a Unicode string is normalized
Text segmentation	
`iter_graphemes(unistr)`	Iterate over grapheme clusters in a string

unicodedata.lookup(*name*, /)

Procurar personagem pelo nome. Se um personagem com o nome fornecido for encontrado, retornar o personagem correspondente. Se não for encontrado, `KeyError` é levantada. Por exemplo:

```
>>> unicodedata.lookup('LEFT CURLY BRACKET')
'{'
```

Os caracteres retornados por esta função são os mesmos produzidos pela sequência de escape `\N` em literais de string. Por exemplo:

```
>>> unicodedata.lookup('MIDDLE DOT') == '\N{MIDDLE DOT}'
True
```

Alterado na versão 3.3: Foi adicionado suporte para apelidos de nomes [[1]] e sequências nomeadas [[2]].

unicodedata.name(*chr*, *default=None*, /)

Retorna o nome atribuído ao caractere *chr* como uma string. Se nenhum nome for definido, *default* será retornado ou, se não for fornecido, **ValueError** será levantada. Por exemplo:

```
>>> unicodedata.name('½')
'VULGAR FRACTION ONE HALF'
>>> unicodedata.name('\uFFFF', 'fallback')
'fallback'
```

unicodedata.decimal(*chr*, *default=None*, /)

Retorna o valor decimal atribuído ao caractere *chr* como inteiro. Se nenhum valor for definido, *default* será retornado ou, se não for fornecido, **ValueError** será levantada. Por exemplo:

```
>>> unicodedata.decimal('\N{ARABIC-INDIC DIGIT NINE}')
9
>>> unicodedata.decimal('\N{SUPERSCRIPT NINE}', -1)
-1
```

unicodedata.digit(*chr*, *default=None*, /)

Retorna o valor de dígito atribuído ao caractere *chr* como inteiro. Se nenhum valor for definido, *default* será retornado ou, se não for fornecido, **ValueError** será levantada.

```
>>> unicodedata.digit('\N{SUPERSCRIPT NINE}')
9
```

unicodedata.numeric(*chr*, *default=None*, /)

Retorna o valor numérico atribuído ao caractere *chr* como float. Se nenhum valor for definido, *default* será retornado ou, se não for fornecido, **ValueError** será levantada.

```
>>> unicodedata.numeric('½')
0.5
```

unicodedata.category(*chr*, /)

Returns the general category assigned to the character *chr* as string. General category names consist of two letters. See the General Category Values section of the Unicode Character Database documentation [https://www.unicode.org/reports/tr44/#General_Category_Values] for a list of category codes. For example:

```
>>> unicodedata.category('A')  # 'L'etter, 'u'ppercase
'Lu'
```

unicodedata.bidirectional(*chr*, /)

Returns the bidirectional class assigned to the character *chr* as string. If no such value is defined, an empty string is returned. See the Bidirectional Class Values section of the Unicode Character Database [https://www.unicode.org/reports/tr44/#Bidi_Class_Values] documentation for a list of bidirectional codes. For example:

```
>>> unicodedata.bidirectional('\N{ARABIC-INDIC DIGIT SEVEN}')
'AN'
```

unicodedata.combining(*chr*, /)

Returns the canonical combining class assigned to the character *chr* as integer. Returns `0` if no combining class is defined. See the Canonical Combining Class Values section of the Unicode Character Database [https://www.unicode.org/reports/tr44/#Canonical_Combining_Class_Values] for more information.

unicodedata.east_asian_width(*chr*, /)

Returns the east asian width assigned to the character *chr* as string. For a list of widths and or more information, see the Unicode Standard Annex #11 [https://www.unicode.org/reports/tr11/].

unicodedata.block(*chr*, /)

Returns the block [https://www.unicode.org/versions/Unicode17.0.0/core-spec/chapter-3/#G64189] assigned to the character *chr*. For example:

```
>>> unicodedata.block('S')
'Basic Latin'
```

Adicionado na versão 3.15.

unicodedata.mirrored(*chr*, /)

Returns the mirrored property assigned to the character *chr* as integer. Returns `1` if the character has been identified as a “mirrored” character in bidirectional text, `0` otherwise. For example:

```
>>> unicodedata.mirrored('>')
1
```

unicodedata.isxidstart(*chr*, /)

Return `True` if *chr* is a valid identifier start per the Unicode Standard Annex #31 [https://www.unicode.org/reports/tr31/], that is, it has the `XID_Start` property. Return `False` otherwise. For example:

```
>>> unicodedata.isxidstart('S')
True
>>> unicodedata.isxidstart('0')
False
```

Adicionado na versão 3.15.

unicodedata.isxidcontinue(*chr*, /)

Return `True` if *chr* is a valid identifier character per the Unicode Standard Annex #31 [https://www.unicode.org/reports/tr31/], that is, it has the `XID_Continue` property. Return `False` otherwise. For example:

```
>>> unicodedata.isxidcontinue('S')
True
>>> unicodedata.isxidcontinue(' ')
False
```

Adicionado na versão 3.15.

unicodedata.decomposition(*chr*, /)

Returns the character decomposition mapping assigned to the character *chr* as string. An empty string is returned in case no such mapping is defined. For example:

```
>>> unicodedata.decomposition('Ã')
'0041 0303'
```

unicodedata.grapheme_cluster_break(*chr*, /)

Returns the Grapheme_Cluster_Break property assigned to the character.

Adicionado na versão 3.15.

unicodedata.indic_conjunct_break(*chr*, /)

Returns the Indic_Conjunct_Break property assigned to the character.

Adicionado na versão 3.15.

unicodedata.extended_pictographic(*chr*, /)

Returns `True` if the character has the Extended_Pictographic property, `False` otherwise.

Adicionado na versão 3.15.

unicodedata.normalize(*form*, *unistr*, /)

Return the normal form *form* for the Unicode string *unistr*. Valid values for *form* are ‘NFC’, ‘NFKC’, ‘NFD’, and ‘NFKD’.

The Unicode standard defines various normalization forms of a Unicode string, based on the definition of canonical equivalence and compatibility equivalence. In Unicode, several characters can be expressed in various way. For example, the character U+00C7 (LATIN CAPITAL LETTER C WITH CEDILLA) can also be expressed as the sequence U+0043 (LATIN CAPITAL LETTER C) U+0327 (COMBINING CEDILLA).

For each character, there are two normal forms: normal form C and normal form D. Normal form D (NFD) is also known as canonical decomposition, and translates each character into its decomposed form. Normal form C (NFC) first applies a canonical decomposition, then composes pre-combined characters again.

In addition to these two forms, there are two additional normal forms based on compatibility equivalence. In Unicode, certain characters are supported which normally would be unified with other characters. For example, U+2160 (ROMAN NUMERAL ONE) is really the same thing as U+0049 (LATIN CAPITAL LETTER I). However, it is supported in Unicode for compatibility with existing character sets (for example, gb2312).

The normal form KD (NFKD) will apply the compatibility decomposition, that is, replace all compatibility characters with their equivalents. The normal form KC (NFKC) first applies the compatibility decomposition, followed by the canonical composition.

Even if two unicode strings are normalized and look the same to a human reader, if one has combining characters and the other doesn't, they may not compare equal.

unicodedata.is_normalized(*form*, *unistr*, */*)

Return whether the Unicode string *unistr* is in the normal form *form*. Valid values for *form* are 'NFC', 'NFKC', 'NFD', and 'NFKD'.

Adicionado na versão 3.8.

unicodedata.iter_graphemes(*unistr*, *start=0*, *end=sys.maxsize*, */*)

Returns an iterator to iterate over grapheme clusters. With optional *start*, iteration begins at that position. With optional *end*, iteration stops at that position.

Converting an emitted item to string returns a substring corresponding to the grapheme cluster. Its `start` and `end` attributes denote the start and end of the grapheme cluster.

It uses extended grapheme cluster rules defined by Unicode Standard Annex #29, "Unicode Text Segmentation" [https://www.unicode.org/reports/tr29/].

Adicionado na versão 3.15.

In addition, the module exposes the following constants:

unicodedata.unidata_version
The version of the Unicode database used in this module.

unicodedata.ucd_3_2_0
This is an object that has most of the methods of the entire module, but uses the Unicode database version 3.2 instead, for applications that require this specific version of the Unicode database (such as IDNA).

Notas de rodapé

[1]

https://www.unicode.org/Public/17.0.0/ucd/NameAliases.txt

[2]

https://www.unicode.org/Public/17.0.0/ucd/NamedSequences.txt

`stringprep` — Preparação de string para internet

Código-fonte: Lib/stringprep.py
[https://github.com/python/cpython/tree/3.15/Lib/stringprep.py]

Ao identificar coisas (como nomes de host) na internet, muitas vezes é necessário comparar tais identificações para “igualdade”. Exatamente como essa comparação é executada pode depender do domínio da aplicação, por exemplo, se deve ser insensível a maiúsculas e minúsculas ou não. Também pode ser necessário restringir as possíveis identificações, para permitir apenas identificações consistindo de caracteres “imprimíveis”.

RFC 3454 [https://datatracker.ietf.org/doc/html/rfc3454.html] define um procedimento para “preparar” strings Unicode em protocolos de internet. Antes de passar strings para o fio, elas são processadas com o procedimento de preparação, após o qual elas têm uma certa forma normalizada. O RFC define um conjunto de tabelas, que podem ser combinadas em perfis. Cada perfil deve definir quais tabelas ele usa, e quais outras partes opcionais do procedimento `stringprep` são parte do perfil. Um exemplo de um perfil `stringprep` é `nameprep`, que é usado para nomes de domínio internacionalizados.

O módulo **`stringprep`** expõe apenas as tabelas do **RFC 3454** [https://datatracker.ietf.org/doc/html/rfc3454.html]. Como essas tabelas seriam muito grandes para representar como dicionários ou listas, o módulo usa o banco de dados de caracteres Unicode internamente. O código-fonte do módulo em si foi gerado usando o utilitário `mkstringprep.py`.

Como resultado, essas tabelas são expostas como funções, não como estruturas de dados. Existem dois tipos de tabelas no RFC: conjuntos e mapeamentos. Para um conjunto, **`stringprep`** fornece a “função característica”, ou seja, uma função que retorna `True` se o parâmetro fizer

parte do conjunto. Para mapeamentos, ele fornece a função de mapeamento: dada a chave, ele retorna o valor associado. Abaixo está uma lista de todas as funções disponíveis no módulo.

stringprep.in_table_a1(*code*)

Determina se *code* está na tabela A.1 (pontos de código não atribuídos no Unicode 3.2).

stringprep.in_table_b1(*code*)

Determina se *code* está na tabela B.1 (comumente mapeado para nada).

stringprep.map_table_b2(*code*)

Retorna o valor mapeado para *code* de acordo com a tabela B.2 (mapeamento para dobramento de casos usado com NFKC).

stringprep.map_table_b3(*code*)

Retorna o valor mapeado para *code* de acordo com a tabela B.3 (mapeamento para dobramento de casos usado com nenhuma normalização).

stringprep.in_table_c11(*code*)

Determina se *code* está na tabela C.1.1 (caracteres de espaço ASCII).

stringprep.in_table_c12(*code*)

Determina se *code* está na tabela C.1.2 (caracteres de espaço não-ASCII).

stringprep.in_table_c11_c12(*code*)

Determina se *code* está na tabela C.1 (caracteres de espaço, união de C.1.1 e C.1.2).

stringprep.in_table_c21(*code*)

Determina se *code* está na tabela C.2.1 (caracteres de controle ASCII).

stringprep.in_table_c22(*code*)

Determina se *code* está na tabela C.2.2 (caracteres de controle não-ASCII).

stringprep.in_table_c21_c22(*code*)

Determina se *code* está na tabela C.2 (caracteres de controle, união de C.2.1 e C.2.2).

stringprep.in_table_c3(*code*)

Determina se *code* está na tabela C.3 (uso privado).

stringprep.in_table_c4(*code*)

Determina se *code* está na tabela C.4 (pontos de código não-caracteres).

stringprep.in_table_c5(*code*)

Determina se *code* está na tabela C.5 (códigos substitutos).

stringprep.in_table_c6(*code*)

Determina se *code* está na tabela C.6 (inapropriado para texto simples).

stringprep.in_table_c7(*code*)

Determina se *code* está na tabela C.7 (inapropriado para para representação canônica).

stringprep.in_table_c8(*code*)

Determina se *code* está na tabela C.8 (alteram propriedades de exibição ou estão descontinuadas).

stringprep.in_table_c9(*code*)

Determina se *code* está na tabela C.9 (caracteres de tag).

stringprep.in_table_d1(*code*)

Determina se *code* está na tabela D.1 (Caracteres com propriedade bidirecional “R” ou “AL”).

stringprep.in_table_d2(*code*)

Determina se *code* está na tabela D.2 (Caracteres com propriedade bidirecional “L”).

`readline` — Interface para o GNU readline

O módulo **`readline`** define uma série de funções para facilitar o autocomplemento e leitura/gravação de arquivos históricos do interpretador Python. Este módulo pode ser usado diretamente ou através do módulo **`rlcompleter`**, que provê o autocomplemento de identificadores Python no prompt interativo. As configurações feitas usando este módulo afetam o comportamento do prompt interativo do interpretador e dos prompts oferecidos pela função embutida **`input()`**.

As combinações de teclas do Readline podem ser configuradas através de um arquivo de inicialização, normalmente `.inputrc` em seu diretório inicial. Consulte Readline Init File [https://tiswww.cwru.edu/php/chet/readline/rluserman.html#Readline-Init-File] no manual do GNU Readline para obter informações sobre o formato e construções permitidas desse arquivo e os recursos da biblioteca do Readline em geral.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Disponibilidade: Unix.

Nota

A API da biblioteca subjacente do Readline pode ser implementada pela biblioteca `editline` (`libedit`) em vez do GNU readline. No macOS, o módulo **`readline`** detecta qual biblioteca está sendo usada em tempo de execução.

O arquivo de configuração do `editline` é diferente daquele do GNU readline. Se você carregar strings de configuração programaticamente, você pode usar **`backend`** para determinar qual biblioteca está sendo usada.

Se você usar a emulação de readline do `editline`/`libedit` no macOS, o arquivo de inicialização localizado em seu diretório inicial será denominado `.editrc`. Por exemplo, o seguinte conteúdo em `~/.editrc` ativará os atalhos de teclado *vi* e o autocomplemento de TAB:

```
python:bind -v
python:bind ^I rl_complete
```

Observe também que bibliotecas diferentes podem usar formatos de arquivo de histórico diferentes. Ao alternar a biblioteca subjacente, os arquivos de histórico existentes podem se tornar inutilizáveis.

readline.backend

O nome da biblioteca Readline subjacente que está sendo usada, seja `"readline"` ou `"editline"`.

Adicionado na versão 3.13.

Arquivo init

As seguintes funções estão relacionadas ao arquivo init e à configuração do usuário:

readline.parse_and_bind(*string*)

Executa a linha de init fornecida no argumento *string*. Isso chama **`rl_parse_and_bind()`** na biblioteca subjacente.

readline.read_init_file([*filename*])

Executa um arquivo de inicialização readline. O nome de arquivo padrão é o último usado. Isso chama **`rl_read_init_file()`** na biblioteca subjacente. Ele levanta um evento de auditoria `open` com o nome do arquivo, se fornecido, e `"<readline_init_file>"` caso contrário, independentemente de qual arquivo a biblioteca resolve.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

Buffer de linha

As seguintes funções operam no buffer de linha:

readline.get_line_buffer()

Retorna o conteúdo atual do buffer de linha (**`rl_line_buffer`** na biblioteca subjacente).

readline.insert_text(*string*)

Insere texto no buffer de linha na posição do cursor. Isso chama **`rl_insert_text()`** na biblioteca subjacente, mas ignora o valor de retorno.

readline.redisplay()

Altera o que é exibido na tela para refletir o conteúdo atual do buffer de linha. Isso chama **`rl_redisplay()`** na biblioteca subjacente.

Arquivo de histórico

As seguintes funções operam em um arquivo histórico:

readline.read_history_file([*filename*])

Carrega um arquivo de histórico do readline e anexa-o à lista de histórico. O nome do arquivo padrão é `~/.history`. Isso chama **`read_history()`** na biblioteca subjacente e levanta um evento de

auditoria `open` com o nome de arquivo se dado e, do contrário, `"~/.history"`.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

readline.write_history_file([*filename*])

Salva a lista de histórico para um arquivo de histórico do readline, sobrescrevendo qualquer arquivo existente. O nome do arquivo padrão é `~/.history`. Isso chama **`write_history()`** na biblioteca subjacente e levanta um evento de auditoria `open` com o nome de arquivo se dado e, do contrário, `"~/.history"`.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

readline.append_history_file(*nelements*[, *filename*])

Anexa os últimos *nelements* itens do histórico a um arquivo. O nome do arquivo padrão é `~/.history`. O arquivo já deve existir. Isso chama **`append_history()`** na biblioteca subjacente. Esta função só existe se o Python foi compilado para uma versão da biblioteca que a suporta. Ela levanta um evento de auditoria `open` com o nome de arquivo se dado e, do contrário, `"~/.history"`.

Adicionado na versão 3.5.

Alterado na versão 3.14: O evento de auditoria foi adicionado.

readline.get_history_length()

readline.set_history_length(*length*)

Define ou retorna o número desejado de linhas para salvar no arquivo de histórico. A função **`write_history_file()`** usa este valor para truncar o arquivo de histórico, chamando **`history_truncate_file()`** na biblioteca subjacente. Valores negativos implicam tamanho ilimitado do arquivo de histórico.

Lista de histórico

As seguintes funções operam em uma lista de histórico global:

readline.clear_history()

Limpa o histórico atual. Isso chama **`clear_history()`** na biblioteca subjacente. A função Python só existe se o Python foi compilado para uma versão da biblioteca que a suporta.

readline.get_current_history_length()

Retorna o número de itens atualmente no histórico. (Isso é diferente de **`get_history_length()`**, que retorna o número máximo de linhas que serão gravadas em um arquivo de histórico.)

readline.get_history_item(*index*)

Retorna o conteúdo atual do item do histórico em *index*. O índice do item é baseado em um. Isso chama **`history_get()`** na biblioteca subjacente.

readline.remove_history_item(*pos*)

Remove o item de histórico especificado por sua posição do histórico. A posição conta a partir de zero. Isso chama **`remove_history()`** na biblioteca subjacente.

readline.replace_history_item(*pos*, *line*)

Substitui o item de histórico especificado pela sua posição por *linha*. A posição conta a partir do zero. Isso chama **`replace_history_entry()`** na biblioteca subjacente.

readline.add_history(*line*)

Acrescenta *line* ao buffer do histórico, como se fosse a última linha digitada. Isso chama **`add_history()`** na biblioteca subjacente.

readline.set_auto_history(*enabled*)

Habilita ou desabilita chamadas automáticas para **`add_history()`** ao ler a entrada via readline. O argumento *enabled* deve ser um valor booleano que, quando verdadeiro, ativa o histórico automático e, quando falso, desativa o histórico automático.

Adicionado na versão 3.6.

O histórico automático está ativado por padrão e as alterações não persistem em várias sessões.

Ganchos de inicialização

readline.set_startup_hook([*function*])

Define ou remove a função invocada pelo retorno de chamada **`rl_startup_hook`** da biblioteca subjacente. Se *function* for especificada, ela será usada como a nova função de gancho; se omitido ou `None`, qualquer função já instalada será removida. O gancho é chamado sem argumentos antes de readline imprimir o primeiro prompt.

readline.set_pre_input_hook([*function*])

Define ou remove a função invocada pelo retorno de chamada **`rl_pre_input_hook`** da biblioteca subjacente. Se *function* for especificada, ela será usada como a nova função de gancho; se omitida ou `None`, qualquer função já instalada será removida. O gancho é chamado sem argumentos após a impressão do primeiro prompt e pouco antes de readline começar a ler os caracteres de entrada. Esta função só existe se o Python foi compilado para uma versão da biblioteca que a suporta.

readline.get_pre_input_hook()

Obtém a função de gancho de pré-entrada atual, ou `None` caso nenhuma função de gancho de pré-entrada tenha sido definida. Esta função só existe se o Python tiver sido compilado para uma versão da biblioteca que ofereça suporte a ela.

Adicionado na versão 3.15.

Autocomplemento

As funções a seguir estão relacionadas à implementação de uma função personalizada de autocomplemento ou *completion*, em inglês, de palavras. Isso normalmente é operado pela tecla Tab e pode sugerir e completar automaticamente uma palavra que está sendo digitada. Por padrão, Readline está configurado para ser usado por **`rlcompleter`** para completar identificadores Python para o interpretador interativo. Se o módulo **`readline`** for usado com um autocomplemento personalizado, um conjunto diferente de delimitadores de palavras deverá ser definido.

readline.set_completer([*function*])

Define ou remove a função de autocomplemento. Se *function* for especificada, ela será usada como a nova função de autocomplemento; se omitido ou `None`, qualquer função de autocomplemento já instalada será removida. A função de autocomplemento é chamada como `function(text, state)`, para *state* em `0`, `1`, `2`, ..., até retornar um valor que não seja string. Deve retornar o próximo autocomplemento possível começando com *text*.

A função de autocomplemento instalada é invocada pelo retorno de chamada *entry_func* passado para **`rl_completion_matches()`** na biblioteca subjacente. A string *text* vem do primeiro parâmetro para o retorno de chamada **`rl_attempted_completion_function`** da biblioteca subjacente.

readline.get_completer()

Obtém a função de autocomplemento ou `None` se nenhuma função de autocomplemento tiver sido definida.

readline.get_completion_type()

Obtém o tipo de autocomplemento que está sendo tentado. Isso retorna a variável **`rl_completion_type`** na biblioteca subjacente como um número inteiro.

readline.get_begidx()

readline.get_endidx()

Obtém o índice inicial ou final do escopo de autocomplemento. Esses índices são os argumentos *start* e *end* passados para o retorno de chamada **`rl_attempted_completion_function`** da biblioteca subjacente. Os valores podem ser diferentes no mesmo cenário de edição de entrada com base na implementação de C readline subjacente. Por exemplo, sabe-se que o libedit se comporta de maneira diferente do libreadline.

readline.set_completer_delims(*string*)
readline.get_completer_delims()

Define ou obtém os delimitadores de palavras para autocomplemento. Estes determinam o início da palavra a ser considerada para autocomplemento (o escopo de autocomplemento). Essas funções acessam a variável **`rl_completer_word_break_characters`** na biblioteca subjacente.

readline.set_completion_display_matches_hook([*function*])

Define ou remove a função de exibição de autocomplemento. Se *function* for especificada, ela será usada como a nova função de exibição de autocomplemento; se omitida ou `None`, qualquer função de exibição de autocomplemento já instalada será removida. Isso define ou limpa o retorno de chamada **`rl_completion_display_matches_hook`** na biblioteca subjacente. A função de exibição de autocomplemento é chamada como `function(substitution, [matches], longest_match_length)` uma vez que cada correspondência precisa ser exibida.

Exemplo

O exemplo a seguir demonstra como usar as funções de leitura e gravação de histórico do módulo **`readline`** para carregar e salvar automaticamente um arquivo de histórico chamado `.python_history` do diretório inicial do usuário. O código abaixo normalmente seria executado automaticamente durante sessões interativas do arquivo **`PYTHONSTARTUP`** do usuário.

```
import atexit
import os
import readline

histfile = os.path.join(os.path.expanduser("~"), ".python_histor
try:
    readline.read_history_file(histfile)
    # comprimento padrão do histórico é -1 (infinito), que pode
    readline.set_history_length(1000)
except FileNotFoundError:
    pass

atexit.register(readline.write_history_file, histfile)
```

Na verdade, este código é executado automaticamente quando o Python é executado no modo interativo (veja Configuração Readline).

O exemplo a seguir atinge o mesmo objetivo, mas oferece suporte a sessões interativas simultâneas, anexando apenas o novo histórico.

```
import atexit
import os
import readline
histfile = os.path.join(os.path.expanduser("~"), ".python_histor

try:
    readline.read_history_file(histfile)
    h_len = readline.get_current_history_length()
except FileNotFoundError:
    open(histfile, 'wb').close()
    h_len = 0

def save(prev_h_len, histfile):
    new_h_len = readline.get_current_history_length()
    readline.set_history_length(1000)
    readline.append_history_file(new_h_len - prev_h_len, histfil
atexit.register(save, h_len, histfile)
```

O exemplo a seguir estende a classe `code.InteractiveConsole` para prover salvamento/restauração do histórico.

```
import atexit
import code
import os
import readline

class HistoryConsole(code.InteractiveConsole):
    def __init__(self, locals=None, filename="<console>",
                 histfile=os.path.expanduser("~/.console-history
        code.InteractiveConsole.__init__(self, locals, filename)
        self.init_history(histfile)

    def init_history(self, histfile):
        readline.parse_and_bind("tab: complete")
        if hasattr(readline, "read_history_file"):
            try:
                readline.read_history_file(histfile)
            except FileNotFoundError:
                pass
            atexit.register(self.save_history, histfile)

    def save_history(self, histfile):
        readline.set_history_length(1000)
        readline.write_history_file(histfile)
```

Nota

O novo REPL introduzido na versão 3.13 não oferece suporte a readline. No entanto, readline ainda pode ser usado definindo a variável de ambiente `PYTHON_BASIC_REPL`.

`rlcompleter` — Função de autocomplemento para GNU readline

Código-fonte: Lib/rlcompleter.py
[https://github.com/python/cpython/tree/3.15/Lib/rlcompleter.py]

O módulo **`rlcompleter`** define uma função de autocompletamento adequada para ser passada para **`set_completer()`** no módulo **`readline`**.

Quando este módulo é importado em uma plataforma Unix com o módulo **`readline`** disponível, uma instância da classe **`Completer`** é criada automaticamente e seu método **`complete()`** é definido como o autocompletamento do readline. O método fornece o autocompletamento de identificadores e palavras-chaves válidas do Python.

Exemplo:

```
>>> import rlcompleter
>>> import readline
>>> readline.parse_and_bind("tab: complete")
>>> readline. <TAB PRESSED>
readline.__doc__          readline.get_line_buffer(  readline.re
readline.__file__         readline.insert_text(      readline.se
readline.__name__         readline.parse_and_bind(
>>> readline.
```

O módulo **`rlcompleter`** é projetado para uso com o modo interativo do Python. A menos que Python seja executado com a opção **`-S`**, o módulo é automaticamente importado e configurado (veja Configuração Readline).

Em plataformas sem **`readline`**, a classe **`Completer`** definida por este módulo ainda pode ser usada para propósitos personalizados.

class rlcompleter.Completer

Os objetos Completer têm o seguinte método:

complete(*text*, *state*)

Retorna o próximo completamento possível para *text*.

Quando chamado pelo módulo `readline`, este método é chamado sucessivamente com `state == 0, 1, 2, ...` até que o método retorne `None`.

Se chamado para *text* que não inclui um caractere de ponto (`'.'`), ele será completado a partir dos nomes atualmente definidos em `__main__`, `builtins` e palavras reservadas (conforme definido pelo módulo `keyword`).

Se chamado por um nome pontilhado, vai tentar avaliar qualquer coisa sem efeitos colaterais óbvios (as funções não serão avaliadas, mas pode levantar chamadas para `__getattr__()`) até a última parte e encontrar correspondências para o resto por meio da função `dir()`. Qualquer exceção levantada durante a avaliação da expressão é capturada, silenciada e `None` é retornado.

Serviços de Dados Binários

Os módulos descritos neste capítulo fornecem algumas operações de serviços básicos para manipulação de dados binários. Outras operações sobre dados binários, especificamente em relação a formatos de arquivo e protocolos de rede, são descritas nas seções relevantes.

Algumas bibliotecas descritas em Serviços de Processamento de Texto também funcionam com formatos binários compatíveis com ASCII (por exemplo `re`) ou com todos os dados binários (por exemplo `difflib`).

Além disso, consulte a documentação dos tipos de dados binários embutidos do Python em Tipos de sequência binária — bytes, bytearray, memoryview.

- `struct` — Interpreta bytes como dados binários empacotados
  - Funções e exceções
  - Format Strings
    - Byte Order, Size, and Alignment
    - Caracteres Formatados
    - Exemplos
  - Applications
    - Native Formats
    - Standard Formats
  - Classes
- `codecs` — Codec registry and base classes
  - Codec Base Classes
    - Error Handlers
    - Stateless Encoding and Decoding
    - Incremental Encoding and Decoding
      - IncrementalEncoder Objects
      - IncrementalDecoder Objects
    - Stream Encoding and Decoding
      - StreamWriter Objects
      - StreamReader Objects

        - StreamReaderWriter Objects
        - StreamRecoder Objects
    - Encodings and Unicode
    - Standard Encodings
    - Python Specific Encodings
        - Text Encodings
        - Binary Transforms
        - Standalone Codec Functions
        - Text Transforms
    - `encodings` — Encodings package
    - `encodings.idna` — Internationalized Domain Names in Applications
    - `encodings.mbcs` — Windows ANSI codepage
    - `encodings.utf_8_sig` — UTF-8 codec with BOM signature

`struct` — Interpreta bytes como dados binários empacotados

Código-fonte: Lib/struct.py [https://github.com/python/cpython/tree/3.15/Lib/struct.py]

Este módulo converte entre valores Python e structs C representados como objetos **`bytes`** do Python. Strings de formato compactas descrevem as conversões pretendidas de/para valores Python. As funções e objetos do módulo podem ser usados para duas aplicações amplamente distintas, troca de dados com fontes externas (arquivos ou conexões de rede) ou transferência de dados entre a aplicação Python e a camada C.

> **Nota**
>
> Quando nenhum caractere de prefixo é fornecido, o modo nativo é o padrão. Ele empacota ou descompacta dados com base na plataforma e no compilador nos quais o interpretador Python foi construído. O resultado do empacotamento de uma dada estrutura C inclui bytes de preenchimento que mantêm o alinhamento adequado para os tipos C envolvidos; similarmente, o alinhamento é levado em conta ao desempacotar. Em contraste, ao comunicar dados entre fontes externas, o programador é responsável por definir a ordenação de bytes e o preenchimento entre os elementos. Veja Byte Order, Size, and Alignment para detalhes.

Several **`struct`** functions (and methods of **`Struct`**) take a *buffer* argument. This refers to objects that implement the Protocolo de buffer and provide either a readable or read-writable buffer. The most common types used for that purpose are **`bytes`** and **`bytearray`**, but many other types that can be viewed as an array of bytes implement the buffer protocol, so that they can be read/filled without additional copying from a **`bytes`** object.

Funções e exceções

O módulo define as seguintes exceções e funções:

exception struct.error

Exceção levantada em várias ocasiões; argumento é uma string que descreve o que está errado.

struct.pack(*format*, *v1*, *v2*, ...)

Return a bytes object containing the values *v1*, *v2*, … packed according to the format string *format*. The arguments must match the values required by the format exactly.

struct.pack_into(*format*, *buffer*, *offset*, *v1*, *v2*, ...)

Pack the values *v1*, *v2*, … according to the format string *format* and write the packed bytes into the writable buffer *buffer* starting at position *offset*. Note that *offset* is a required argument. A negative *offset* counts from the end of *buffer*.

struct.unpack(*format*, *buffer*)

Unpack from the buffer *buffer* (presumably packed by `pack(format, ...)`) according to the format string *format*. The result is a tuple even if it contains exactly one item. The buffer's size in bytes must match the size required by the format, as reflected by **`calcsize()`**.

struct.unpack_from(*format*, /, *buffer*, *offset=0*)

Unpack from *buffer* starting at position *offset*, according to the format string *format*. The result is a tuple even if it contains exactly one item. The buffer's size in bytes, starting at position *offset*, must be at least the size required by the format, as reflected by **`calcsize()`**. A negative *offset* counts from the end of *buffer*.

struct.iter_unpack(*format*, *buffer*)

Iteratively unpack from the buffer *buffer* according to the format string *format*. This function returns an iterator which will read equally sized chunks from the buffer until all its contents have been consumed. The

buffer's size in bytes must be a multiple of the size required by the format, as reflected by `calcsize()`.

Each iteration yields a tuple as specified by the format string.

Adicionado na versão 3.4.

struct.calcsize(*format*)

Return the size of the struct (and hence of the bytes object produced by `pack(format, ...)`) corresponding to the format string *format*.

Format Strings

Format strings describe the data layout when packing and unpacking data. They are built up from format characters, which specify the type of data being packed/unpacked. In addition, special characters control the byte order, size and alignment. Each format string consists of an optional prefix character which describes the overall properties of the data and one or more format characters which describe the actual data values and padding.

Byte Order, Size, and Alignment

By default, C types are represented in the machine's native format and byte order, and properly aligned by skipping pad bytes if necessary (according to the rules used by the C compiler). This behavior is chosen so that the bytes of a packed struct correspond exactly to the memory layout of the corresponding C struct. Whether to use native byte ordering and padding or standard formats depends on the application.

Alternatively, the first character of the format string can be used to indicate the byte order, size and alignment of the packed data, according to the following table:

Caractere	Ordem de bytes	Tamanho	Alinhamento

Caractere	Ordem de bytes	Tamanho	Alinhamento
`@`	nativo	nativo	nativo
`=`	nativo	padrão	nenhum
`<`	little-endian	padrão	nenhum
`>`	big-endian	padrão	nenhum
`!`	rede (= big-endian)	padrão	nenhum

Se o primeiro caractere não for um destes, `'@'` é presumido.

Nota

The number 1023 (`0x3ff` in hexadecimal) has the following byte representations:

- `03 ff` in big-endian (`>`)
- `ff 03` in little-endian (`<`)

Python example:

```
>>> import struct
>>> struct.pack('>h', 1023)
b'\x03\xff'
>>> struct.pack('<h', 1023)
b'\xff\x03'
```

Native byte order is big-endian or little-endian, depending on the host system. For example, Intel x86, AMD64 (x86-64), and Apple M1 are little-endian; IBM z and many legacy architectures are big-endian. Use `sys.byteorder` to check the endianness of your system.

Native size and alignment are determined using the C compiler's `sizeof` expression. This is always combined with native byte order.

Standard size depends only on the format character; see the table in the Caracteres Formatados section.

Note the difference between `'@'` and `'='`: both use native byte order, but the size and alignment of the latter is standardized.

The form `'!'` represents the network byte order which is always big-endian as defined in IETF RFC 1700 [https://datatracker.ietf.org/doc/html/rfc1700].

There is no way to indicate non-native byte order (force byte-swapping); use the appropriate choice of `'<'` or `'>'`.

Notas:

1. Padding is only automatically added between successive structure members. No padding is added at the beginning or the end of the encoded struct.
2. No padding is added when using non-native size and alignment, e.g. with '<', '>', '=', and '!'.
3. To align the end of a structure to the alignment requirement of a particular type, end the format with the code for that type with a repeat count of zero. See Exemplos.

Caracteres Formatados

Format characters have the following meaning; the conversion between C and Python values should be obvious given their types. The 'Standard size' column refers to the size of the packed value in bytes when using standard size; that is, when the format string starts with one of `'<'`, `'>'`, `'!'` or `'='`. When using native size, the size of the packed value is platform-dependent.

Formatação	Tipo em C	Tipo em Python	Standard size	Notas
`x`	pad byte	no value		(7)
`c`	char	bytes of length 1	1	
`b`	signed char	int	1	(2)
`B`	unsigned char	int	1	(2)
`?`	_Bool	bool	1	(1)
`h`	short	int	2	(2)
`H`	unsigned short	int	2	(2)
`i`	int	int	4	(2)
`I`	unsigned int	int	4	(2)
`l`	long	int	4	(2)
`L`	unsigned long	int	4	(2)

Formatação	Tipo em C	Tipo em Python	Standard size	Notas
`q`	long long	int	8	(2)
`Q`	unsigned long long	int	8	(2)
`n`	**`ssize_t`**	int		(2), (3)
`N`	**`size_t`**	int		(2), (3)
`e`	_Float16	float	2	(4), (6)
`f`	float	float	4	(4)
`d`	double	float	8	(4)
`Zf`	float complex	complexo	8	(10)
`Zd`	double complex	complexo	16	(10)
`s`	char[]	bytes		(9)

Formatação	Tipo em C	Tipo em Python	Standard size	Notas
`p`	char[]	bytes		(8)
`P`	void*	int		(2), (5)

Alterado na versão 3.3: Added support for the `'n'` and `'N'` formats.

Alterado na versão 3.6: Added support for the `'e'` format.

Alterado na versão 3.14: Added support for the `'F'` and `'D'` formats.

Alterado na versão 3.15: Added support for the `'Zf'` and `'Zd'` formats. `'F'` and `'D'` formats are soft deprecated.

Ver também

The `array` and ctypes modules, as well as third-party modules like numpy [https://numpy.org/doc/stable/reference/arrays.interface.html#object.__array_interface__], use similar – but slightly different – type codes.

Notas:

1. The `'?'` conversion code corresponds to the _Bool type defined by C standards since C99. In standard mode, it is represented by one byte.

2. When attempting to pack a non-integer using any of the integer conversion codes, if the non-integer has a `__index__()` method then that method is called to convert the argument to an integer before packing.

Alterado na versão 3.2: Added use of the `__index__()` method for non-integers.

3. The `'n'` and `'N'` conversion codes are only available for the native size (selected as the default or with the `'@'` byte order character). For the standard size, you can use whichever of the other integer formats fits your application.

4. For the `'f'`, `'d'` and `'e'` conversion codes, the packed representation uses the IEEE 754 binary32, binary64 or binary16 format (for `'f'`, `'d'` or `'e'` respectively), regardless of the floating-point format used by the platform.

5. The `'P'` format character is only available for the native byte ordering (selected as the default or with the `'@'` byte order character). The byte order character `'='` chooses to use little- or big-endian ordering based on the host system. The struct module does not interpret this as native ordering, so the `'P'` format is not available.

6. The IEEE 754 binary16 "half precision" type was introduced in the 2008 revision of the IEEE 754 standard [https://en.wikipedia.org/wiki/IEEE_754-2008_revision]. It has a sign bit, a 5-bit exponent and 11-bit precision (with 10 bits explicitly stored), and can represent numbers between approximately `6.1e-05` and `6.5e+04` at full precision. This type is not widely supported by C compilers: it's available as _Float16 type, if the compiler supports the Annex H of the C23 standard. On a typical machine, an unsigned short can be used for storage, but not for math operations. See the Wikipedia page on the half-precision floating-point format [https://en.wikipedia.org/wiki/Half-precision_floating-point_format] for more information.

7. When packing, `'x'` inserts one NUL byte.

8. The `'p'` format character encodes a "Pascal string", meaning a short variable-length string stored in a *fixed number of bytes*, given by the count. The first byte stored is the length of the string, or 255, whichever is smaller. The bytes of the string follow. If the byte string passed in to `pack()` is too long (longer than the count minus 1), only

the leading `count-1` bytes of the string are stored. If the byte string is shorter than `count-1`, it is padded with null bytes so that exactly count bytes in all are used. Note that for **`unpack()`**, the `'p'` format character consumes `count` bytes, but that the **`bytes`** object returned can never contain more than 255 bytes. When packing, arguments of types **`bytes`** and **`bytearray`** are accepted.

9. For the `'s'` format character, the count is interpreted as the length of the byte string, not a repeat count like for the other format characters; for example, `'10s'` means a single 10-byte string mapping to or from a single Python byte string, while `'10c'` means 10 separate one byte character elements (e.g., `cccccccccc`) mapping to or from ten different Python byte objects. (See Exemplos for a concrete demonstration of the difference.) If a count is not given, it defaults to 1. For packing, the byte string is truncated or padded with null bytes as appropriate to make it fit. For unpacking, the resulting **`bytes`** object always has exactly the specified number of bytes. As a special case, `'0s'` means a single, empty byte string (while `'0c'` means 0 characters). When packing, arguments of types **`bytes`** and **`bytearray`** are accepted.

10. For the `'Zf'` and `'Zd'` type codes, the packed representation uses the IEEE 754 binary32 and binary64 format for components of the complex number, regardless of the floating-point format used by the platform. Note that complex types are available unconditionally, despite complex types being an optional feature in C. As specified in the C11 standard, each complex type is represented by a two-element C array containing, respectively, the real and imaginary parts. The `'F'` and `'D'` (for `'Zf'` and `'Zd'`, respectively) format characters are supported for compatibility.

A format character may be preceded by an integral repeat count. For example, the format string `'4h'` means exactly the same as `'hhhh'`.

Whitespace characters between formats are ignored; a count and its format must not contain whitespace though.

When packing a value `x` using one of the integer formats (`'b'`, `'B'`, `'h'`, `'H'`, `'i'`, `'I'`, `'l'`, `'L'`, `'q'`, `'Q'`), if `x` is outside the valid range for that

format then `struct.error` is raised.

Alterado na versão 3.1: Previously, some of the integer formats wrapped out-of-range values and raised `DeprecationWarning` instead of `struct.error`.

For the `'?'` format character, the return value is either `True` or `False`. When packing, the truth value of the argument object is used. Either 0 or 1 in the native or standard bool representation will be packed, and any non-zero value will be `True` when unpacking.

Exemplos

Nota

Native byte order examples (designated by the `'@'` format prefix or lack of any prefix character) may not match what the reader's machine produces as that depends on the platform and compiler.

Pack and unpack integers of three different sizes, using big endian ordering:

```
>>> from struct import *
>>> pack(">bhl", 1, 2, 3)
b'\x01\x00\x02\x00\x00\x00\x03'
>>> unpack('>bhl', b'\x01\x00\x02\x00\x00\x00\x03')
(1, 2, 3)
>>> calcsize('>bhl')
7
```

Attempt to pack an integer which is too large for the defined field:

```
>>> pack(">h", 99999)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
struct.error: 'h' format requires -32768 <= number <= 32767
```

Demonstrate the difference between `'s'` and `'c'` format characters:

```
>>> pack("@ccc", b'1', b'2', b'3')
b'123'
>>> pack("@3s", b'123')
b'123'
```

Unpacked fields can be named by assigning them to variables or by wrapping the result in a named tuple:

```
>>> record = b'raymond   \x32\x12\x08\x01\x08'
>>> name, serialnum, school, gradelevel = unpack('<10sHHb', recc

>>> from collections import namedtuple
>>> Student = namedtuple('Student', 'name serialnum school grade
>>> Student._make(unpack('<10sHHb', record))
Student(name=b'raymond   ', serialnum=4658, school=264, gradelev
```

The ordering of format characters may have an impact on size in native mode since padding is implicit. In standard mode, the user is responsible for inserting any desired padding. Note in the first `pack` call below that three NUL bytes were added after the packed `'#'` to align the following integer on a four-byte boundary. In this example, the output was produced on a little endian machine:

```
>>> pack('@ci', b'#', 0x12131415)
b'#\x00\x00\x00\x15\x14\x13\x12'
>>> pack('@ic', 0x12131415, b'#')
b'\x15\x14\x13\x12#'
>>> calcsize('@ci')
8
>>> calcsize('@ic')
5
```

The following format `'llh0l'` results in two pad bytes being added at the end, assuming the platform's longs are aligned on 4-byte boundaries:

```
>>> pack('@llh0l', 1, 2, 3)
b'\x00\x00\x00\x01\x00\x00\x00\x02\x00\x03\x00\x00'
```

Ver também

Module `array`
Packed binary storage of homogeneous data.

Módulo `json`
JSON encoder and decoder.

Módulo `pickle`
Python object serialization.

Applications

Two main applications for the **`struct`** module exist, data interchange between Python and C code within an application or another application compiled using the same compiler (native formats), and data interchange between applications using agreed upon data layout (standard formats). Generally speaking, the format strings constructed for these two domains are distinct.

Native Formats

When constructing format strings which mimic native layouts, the compiler and machine architecture determine byte ordering and padding. In such cases, the `@` format character should be used to specify native byte ordering and data sizes. Internal pad bytes are normally inserted automatically. It is possible that a zero-repeat format code will be needed at the end of a format string to round up to the correct byte boundary for proper alignment of consecutive chunks of data.

Consider these two simple examples (on a 64-bit, little-endian machine):

```
>>> calcsize('@lhl')
24
>>> calcsize('@llh')
18
```

Data is not padded to an 8-byte boundary at the end of the second format string without the use of extra padding. A zero-repeat format code solves that problem:

```
>>> calcsize('@llh0l')
24
```

The `'x'` format code can be used to specify the repeat, but for native formats it is better to use a zero-repeat format like `'0l'`.

By default, native byte ordering and alignment is used, but it is better to be explicit and use the `'@'` prefix character.

Standard Formats

When exchanging data beyond your process such as networking or storage, be precise. Specify the exact byte order, size, and alignment. Do not assume they match the native order of a particular machine. For example, network byte order is big-endian, while many popular CPUs are little-endian. By defining this explicitly, the user need not care about the specifics of the platform their code is running on. The first character should typically be `<` or `>` (or `!`). Padding is the responsibility of the programmer. The zero-repeat format character won't work. Instead, the user must explicitly add `'x'` pad bytes where needed. Revisiting the examples from the previous section, we have:

```
>>> calcsize('<qh6xq')
24
>>> pack('<qh6xq', 1, 2, 3) == pack('@lhl', 1, 2, 3)
True
>>> calcsize('@llh')
18
>>> pack('@llh', 1, 2, 3) == pack('<qqh', 1, 2, 3)
True
>>> calcsize('<qqh6x')
24
>>> calcsize('@llh0l')
24
>>> pack('@llh0l', 1, 2, 3) == pack('<qqh6x', 1, 2, 3)
True
```

The above results (executed on a 64-bit machine) aren't guaranteed to match when executed on different machines. For example, the examples below were executed on a 32-bit machine:

```
>>> calcsize('<qqh6x')
24
>>> calcsize('@llh0l')
12
>>> pack('@llh0l', 1, 2, 3) == pack('<qqh6x', 1, 2, 3)
False
```

Classes

The **`struct`** module also defines the following type:

class struct.Struct(*format*)

Return a new Struct object which writes and reads binary data according to the format string *format*. Creating a `Struct` object once and calling its methods is more efficient than calling module-level functions with the same format since the format string is only compiled once.

Nota

The compiled versions of the most recent format strings passed to the module-level functions are cached, so programs that use only a few format strings needn't worry about reusing a single **`Struct`** instance.

Compiled Struct objects support the following methods and attributes:

pack(*v1*, *v2*, ...)

Identical to the **`pack()`** function, using the compiled format. (`len(result)` will equal **`size`**.)

pack_into(*buffer*, *offset*, *v1*, *v2*, ...)

Identical to the **`pack_into()`** function, using the compiled format.

unpack(*buffer*)

Identical to the `unpack()` function, using the compiled format. The buffer's size in bytes must equal `size`.

unpack_from(*buffer*, *offset=0*)

Identical to the `unpack from()` function, using the compiled format. The buffer's size in bytes, starting at position *offset*, must be at least `size`.

iter_unpack(*buffer*)

Identical to the `iter unpack()` function, using the compiled format. The buffer's size in bytes must be a multiple of `size`.

Adicionado na versão 3.4.

format

The format string used to construct this Struct object.

Alterado na versão 3.7: The format string type is now `str` instead of `bytes`.

size

The calculated size of the struct (and hence of the bytes object produced by the `pack()` method) corresponding to `format`.

Alterado na versão 3.13: The *repr()* of structs has changed. It is now:

```
>>> Struct('i')
Struct('i')
```

codecs — Codec registry and base classes

Código-fonte: Lib/codecs.py [https://github.com/python/cpython/tree/3.15/Lib/codecs.py]

Este módulo define classes base para codecs Python padrão (codificadores e decodificadores) e fornece acesso ao registro interno de codecs Python, que gerencia o codec e o processo de pesquisa de tratamento de erros. A maioria dos codecs padrão são codificações de texto, que codificam texto em bytes (e decodificam bytes em texto), mas também existem codecs fornecidos que codificam texto em texto e bytes em bytes. Codecs personalizados podem codificar e decodificar entre tipos arbitrários, mas alguns recursos do módulo são restritos para serem usados especificamente com codificações do texto ou com codecs que codificam para `bytes`.

The module defines the following functions for encoding and decoding with any codec:

codecs.encode(*obj*, *encoding='utf-8'*, *errors='strict'*)

Encodes *obj* using the codec registered for *encoding*.

Errors may be given to set the desired error handling scheme. The default error handler is `'strict'` meaning that encoding errors raise `ValueError` (or a more codec specific subclass, such as `UnicodeEncodeError`). Refer to Codec Base Classes for more information on codec error handling.

codecs.decode(*obj*, *encoding='utf-8'*, *errors='strict'*)

Decodes *obj* using the codec registered for *encoding*.

Errors may be given to set the desired error handling scheme. The default error handler is `'strict'` meaning that decoding errors raise `ValueError` (or a more codec specific subclass, such as

`UnicodeDecodeError`). Refer to Codec Base Classes for more information on codec error handling.

codecs.charmap_build(*string*)

Return a mapping suitable for encoding with a custom single-byte encoding. Given a `str` *string* of up to 256 characters representing a decoding table, returns either a compact internal mapping object `EncodingMap` or a `dictionary` mapping character ordinals to byte values. Raises a `TypeError` on invalid input.

The full details for each codec can also be looked up directly:

codecs.lookup(*encoding*, /)

Looks up the codec info in the Python codec registry and returns a `CodecInfo` object as defined below.

This function first normalizes the *encoding*: all ASCII letters are converted to lower case, spaces are replaced with hyphens. Then encoding is looked up in the registry's cache. If not found, the list of registered search functions is scanned. If no `CodecInfo` object is found, a `LookupError` is raised. Otherwise, the **`CodecInfo`** object is stored in the cache and returned to the caller.

Alterado na versão 3.9: Any characters except ASCII letters and digits and a dot are converted to underscore.

Alterado na versão 3.15: No characters are converted to underscore anymore. Spaces are converted to hyphens.

class codecs.CodecInfo(*encode*, *decode*, *streamreader=None*, *streamwriter=None*, *incrementalencoder=None*, *incrementaldecoder=None*, *name=None*)

Codec details when looking up the codec registry. The constructor arguments are stored in attributes of the same name:

name

The name of the encoding.

encode
decode

The stateless encoding and decoding functions. These must be functions or methods which have the same interface as the `encode()` and `decode()` methods of Codec instances (see Codec Interface). The functions or methods are expected to work in a stateless mode.

incrementalencoder
incrementaldecoder

Incremental encoder and decoder classes or factory functions. These have to provide the interface defined by the base classes `IncrementalEncoder` and `IncrementalDecoder`, respectively. Incremental codecs can maintain state.

streamwriter
streamreader

Stream writer and reader classes or factory functions. These have to provide the interface defined by the base classes `StreamWriter` and `StreamReader`, respectively. Stream codecs can maintain state.

To simplify access to the various codec components, the module provides these additional functions which use `lookup()` for the codec lookup:

codecs.getencoder(*encoding*)

Look up the codec for the given encoding and return its encoder function.

Raises a `LookupError` in case the encoding cannot be found.

codecs.getdecoder(*encoding*)

Look up the codec for the given encoding and return its decoder function.

Raises a `LookupError` in case the encoding cannot be found.

codecs.getincrementalencoder(*encoding*)

Look up the codec for the given encoding and return its incremental encoder class or factory function.

Raises a `LookupError` in case the encoding cannot be found or the codec doesn't support an incremental encoder.

codecs.getincrementaldecoder(*encoding*)

Look up the codec for the given encoding and return its incremental decoder class or factory function.

Raises a `LookupError` in case the encoding cannot be found or the codec doesn't support an incremental decoder.

codecs.getreader(*encoding*)

Look up the codec for the given encoding and return its `StreamReader` class or factory function.

Raises a `LookupError` in case the encoding cannot be found.

codecs.getwriter(*encoding*)

Look up the codec for the given encoding and return its `StreamWriter` class or factory function.

Raises a `LookupError` in case the encoding cannot be found.

Custom codecs are made available by registering a suitable codec search function:

codecs.register(*search_function*, /)

Register a codec search function. Search functions are expected to take one argument, being the encoding name in all lower case letters with spaces converted to hyphens, and return a `CodecInfo` object. In case a search function cannot find a given encoding, it should return `None`.

codecs.unregister(*search_function*, /)

Unregister a codec search function and clear the registry's cache. If the search function is not registered, do nothing.

Adicionado na versão 3.10.

While the builtin **open()** and the associated **io** module are the recommended approach for working with encoded text files, this module provides additional utility functions and classes that allow the use of a wider range of codecs when working with binary files:

codecs.open(*filename*, *mode='r'*, *encoding=None*, *errors='strict'*, *buffering=-1*)

Open an encoded file using the given *mode* and return an instance of **StreamReaderWriter**, providing transparent encoding/decoding. The default file mode is `'r'`, meaning to open the file in read mode.

Nota

If *encoding* is not `None`, then the underlying encoded files are always opened in binary mode. No automatic conversion of `'\n'` is done on reading and writing. The *mode* argument may be any binary mode acceptable to the built-in **open()** function; the `'b'` is automatically added.

encoding specifies the encoding which is to be used for the file. Any encoding that encodes to and decodes from bytes is allowed, and the data types supported by the file methods depend on the codec used.

errors may be given to define the error handling. It defaults to `'strict'` which causes a **ValueError** to be raised in case an encoding error occurs.

buffering has the same meaning as for the built-in **open()** function. It defaults to -1 which means that the default buffer size will be used.

Alterado na versão 3.11: O modo `'U'` foi removido.

Descontinuado desde a versão 3.14: **codecs.open()** has been superseded by **open()**.

codecs.EncodedFile(*file*, *data_encoding*, *file_encoding=None*, *errors='strict'*)

Return a **`StreamRecoder`** instance, a wrapped version of *file* which provides transparent transcoding. The original file is closed when the wrapped version is closed.

Data written to the wrapped file is decoded according to the given *data_encoding* and then written to the original file as bytes using *file_encoding*. Bytes read from the original file are decoded according to *file_encoding*, and the result is encoded using *data_encoding*.

If *file_encoding* is not given, it defaults to *data_encoding*.

errors may be given to define the error handling. It defaults to `'strict'`, which causes **`ValueError`** to be raised in case an encoding error occurs.

codecs.iterencode(*iterator*, *encoding*, *errors='strict'*, ***kwargs*)

Uses an incremental encoder to iteratively encode the input provided by *iterator*. *iterator* must yield **`str`** objects. This function is a generator. The *errors* argument (as well as any other keyword argument) is passed through to the incremental encoder.

This function requires that the codec accept text **`str`** objects to encode. Therefore it does not support bytes-to-bytes encoders such as `base64_codec`.

codecs.iterdecode(*iterator*, *encoding*, *errors='strict'*, ***kwargs*)

Uses an incremental decoder to iteratively decode the input provided by *iterator*. *iterator* must yield **`bytes`** objects. This function is a generator. The *errors* argument (as well as any other keyword argument) is passed through to the incremental decoder.

This function requires that the codec accept **`bytes`** objects to decode. Therefore it does not support text-to-text encoders such as `rot_13`, although `rot_13` may be used equivalently with **`iterencode()`**.

codecs.readbuffer_encode(*buffer*, *errors=None*, /)

Return a **tuple** containing the raw bytes of *buffer*, a buffer-compatible object or **str** (encoded to UTF-8 before processing), and their length in bytes.

The *errors* argument is ignored.

```
>>> codecs.readbuffer_encode(b"Zito")
(b'Zito', 4)
```

The module also provides the following constants which are useful for reading and writing to platform dependent files:

codecs.BOM
codecs.BOM_BE
codecs.BOM_LE
codecs.BOM_UTF8
codecs.BOM_UTF16
codecs.BOM_UTF16_BE
codecs.BOM_UTF16_LE
codecs.BOM_UTF32
codecs.BOM_UTF32_BE
codecs.BOM_UTF32_LE

These constants define various byte sequences, being Unicode byte order marks (BOMs) for several encodings. They are used in UTF-16 and UTF-32 data streams to indicate the byte order used, and in UTF-8 as a Unicode signature. **BOM_UTF16** is either **BOM_UTF16_BE** or **BOM_UTF16_LE** depending on the platform's native byte order, **BOM** is an alias for **BOM_UTF16**, **BOM_LE** for **BOM_UTF16_LE** and **BOM_BE** for **BOM_UTF16_BE**. The others represent the BOM in UTF-8 and UTF-32 encodings.

Codec Base Classes

The **codecs** module defines a set of base classes which define the interfaces for working with codec objects, and can also be used as the basis for custom codec implementations.

Each codec has to define four interfaces to make it usable as codec in Python: stateless encoder, stateless decoder, stream reader and stream writer. The stream reader and writers typically reuse the stateless encoder/decoder to implement the file protocols. Codec authors also need to define how the codec will handle encoding and decoding errors.

Error Handlers

To simplify and standardize error handling, codecs may implement different error handling schemes by accepting the *errors* string argument:

```
>>> 'German ß, ♬'.encode(encoding='ascii', errors='backslashrepl
b'German \\xdf, \\u266c'
>>> 'German ß, ♬'.encode(encoding='ascii', errors='xmlcharrefrep
b'German &#223;, &#9836;'
```

The following error handlers can be used with all Python Standard Encodings codecs:

Valor	Significado
`'strict'`	Raise `UnicodeError` (or a subclass), this is the default. Implemented in `strict_errors()`.
`'ignore'`	Ignore the malformed data and continue without further notice. Implemented in `ignore_errors()`.
`'replace'`	Replace with a replacement marker. On encoding, use `?` (ASCII character). On decoding, use `\uFFFD` (U+FFFD, the official REPLACEMENT CHARACTER). Implemented in `replace_errors()`.

Valor	Significado
`'backslashreplace'`	Replace with backslashed escape sequences. On encoding, use hexadecimal form of Unicode code point with formats `\x`*`hh`* `\u`*`xxxx`* `\U`*`xxxxxxxx`*. On decoding, use hexadecimal form of byte value with format `\x`*`hh`*. Implemented in **`backslashreplace_errors()`**.
`'surrogateescape'`	On decoding, replace byte with individual surrogate code ranging from `U+DC80` to `U+DCFF`. This code will then be turned back into the same byte when the `'surrogateescape'` error handler is used when encoding the data. (See **PEP 383** for more.)

Os seguintes manipuladores de erro aplicam-se apenas à codificação (dentro de codificações do texto):

Valor	Significado
`'xmlcharrefreplace'`	Replace with XML/HTML numeric character reference, which is a decimal form of Unicode code point with format `&#`*`num`*`;`. Implemented in **`xmlcharrefreplace_errors()`**.
`'namereplace'`	Replace with `\N{...}` escape sequences, what appears in the braces is the Name property from Unicode Character Database. Implemented in **`namereplace_errors()`**.

In addition, the following error handler is specific to the given codecs:

Valor	Codecs	Significado
`'surrogatepass'`	utf-8, utf-16, utf-32, utf-16-be, utf-16-le, utf-32-be, utf-32-le	Allow encoding and decoding surrogate code point (`U+D800` - `U+DFFF`) as normal code point. Otherwise these codecs treat the presence of surrogate code point in `str` as an error.

Adicionado na versão 3.1: The `'surrogateescape'` and `'surrogatepass'` error handlers.

Alterado na versão 3.4: The `'surrogatepass'` error handler now works with utf-16* and utf-32* codecs.

Adicionado na versão 3.5: The `'namereplace'` error handler.

Alterado na versão 3.5: The `'backslashreplace'` error handler now works with decoding and translating.

The set of allowed values can be extended by registering a new named error handler:

codecs.register_error(*name*, *error_handler*, /)

Register the error handling function *error_handler* under the name *name*. The *error_handler* argument will be called during encoding and decoding in case of an error, when *name* is specified as the errors parameter.

For encoding, *error_handler* will be called with a `UnicodeEncodeError` instance, which contains information about the location of the error. The error handler must either raise this or a different exception, or return a tuple with a replacement for the unencodable part of the input and a

position where encoding should continue. The replacement may be either `str` or `bytes`. If the replacement is bytes, the encoder will simply copy them into the output buffer. If the replacement is a string, the encoder will encode the replacement. Encoding continues on original input at the specified position. Negative position values will be treated as being relative to the end of the input string. If the resulting position is out of bound an `IndexError` will be raised.

Decoding and translating works similarly, except `UnicodeDecodeError` or `UnicodeTranslateError` will be passed to the handler and that the replacement from the error handler will be put into the output directly.

Previously registered error handlers (including the standard error handlers) can be looked up by name:

codecs.lookup_error(*name*, /)

Return the error handler previously registered under the name *name*.

Raises a `LookupError` in case the handler cannot be found.

The following standard error handlers are also made available as module level functions:

codecs.strict_errors(*exception*)

Implementa a tratativa de erro `'strict'`.

Each encoding or decoding error raises a `UnicodeError`.

codecs.ignore_errors(*exception*)

Implementa a tratativa de erro `'ignore'`.

Malformed data is ignored; encoding or decoding is continued without further notice.

codecs.replace_errors(*exception*)

Implementa a tratativa de erro `'replace'`.

Substitutes `?` (ASCII character) for encoding errors or ` ` (U+FFFD, the official REPLACEMENT CHARACTER) for decoding errors.

codecs.backslashreplace_errors(*exception*)

Implementa a tratativa de erro `'backslashreplace'`.

Malformed data is replaced by a backslashed escape sequence. On encoding, use the hexadecimal form of Unicode code point with formats `\x`*`hh`* `\u`*`xxxx`* `\U`*`xxxxxxxx`*. On decoding, use the hexadecimal form of byte value with format `\x`*`hh`*.

Alterado na versão 3.5: Works with decoding and translating.

codecs.xmlcharrefreplace_errors(*exception*)

Implements the `'xmlcharrefreplace'` error handling (for encoding within text encoding only).

The unencodable character is replaced by an appropriate XML/HTML numeric character reference, which is a decimal form of Unicode code point with format `&#`*`num`*`;` .

codecs.namereplace_errors(*exception*)

Implements the `'namereplace'` error handling (for encoding within text encoding only).

The unencodable character is replaced by a `\N{...}` escape sequence. The set of characters that appear in the braces is the Name property from Unicode Character Database. For example, the German lowercase letter `'ß'` will be converted to byte sequence `\N{LATIN SMALL LETTER SHARP S}` .

Adicionado na versão 3.5.

Stateless Encoding and Decoding

The base **`Codec`** class defines these methods which also define the function interfaces of the stateless encoder and decoder:

class codecs.Codec

encode(*input*, *errors='strict'*)

Encodes the object *input* and returns a tuple (output object, length consumed). For instance, text encoding converts a string object to a bytes object using a particular character set encoding (e.g., `cp1252` or `iso-8859-1`).

The *errors* argument defines the error handling to apply. It defaults to `'strict'` handling.

The method may not store state in the **`Codec`** instance. Use **`StreamWriter`** for codecs which have to keep state in order to make encoding efficient.

The encoder must be able to handle zero length input and return an empty object of the output object type in this situation.

decode(*input*, *errors='strict'*)

Decodes the object *input* and returns a tuple (output object, length consumed). For instance, for a text encoding, decoding converts a bytes object encoded using a particular character set encoding to a string object.

For text encodings and bytes-to-bytes codecs, *input* must be a bytes object or one which provides the read-only buffer interface – for example, buffer objects and memory mapped files.

The *errors* argument defines the error handling to apply. It defaults to `'strict'` handling.

The method may not store state in the **`Codec`** instance. Use **`StreamReader`** for codecs which have to keep state in order to make decoding efficient.

The decoder must be able to handle zero length input and return an empty object of the output object type in this situation.

Incremental Encoding and Decoding

The `IncrementalEncoder` and `IncrementalDecoder` classes provide the basic interface for incremental encoding and decoding. Encoding/decoding the input isn't done with one call to the stateless encoder/decoder function, but with multiple calls to the `encode()`/`decode()` method of the incremental encoder/decoder. The incremental encoder/decoder keeps track of the encoding/decoding process during method calls.

The joined output of calls to the `encode()`/`decode()` method is the same as if all the single inputs were joined into one, and this input was encoded/decoded with the stateless encoder/decoder.

IncrementalEncoder Objects

The `IncrementalEncoder` class is used for encoding an input in multiple steps. It defines the following methods which every incremental encoder must define in order to be compatible with the Python codec registry.

class codecs.IncrementalEncoder(*errors='strict'*)

Constructor for an **`IncrementalEncoder`** instance.

All incremental encoders must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The **`IncrementalEncoder`** may implement different error handling schemes by providing the *errors* keyword argument. See Error Handlers for possible values.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the **`IncrementalEncoder`** object.

encode(*object*, *final=False*)

Encodes *object* (taking the current state of the encoder into account) and returns the resulting encoded object. If this is the last call to **`encode()`** *final* must be true (the default is false).

reset()

Reset the encoder to the initial state. The output is discarded: call `.encode(object, final=True)`, passing an empty byte or text string if necessary, to reset the encoder and to get the output.

getstate()

Return the current state of the encoder which must be an integer. The implementation should make sure that `0` is the most common state. (States that are more complicated than integers can be converted into an integer by marshaling/pickling the state and encoding the bytes of the resulting string into an integer.)

setstate(*state*)

Set the state of the encoder to *state*. *state* must be an encoder state returned by **`getstate()`**.

IncrementalDecoder Objects

The **`IncrementalDecoder`** class is used for decoding an input in multiple steps. It defines the following methods which every incremental decoder must define in order to be compatible with the Python codec registry.

class codecs.IncrementalDecoder(*errors='strict'*)

Constructor for an **`IncrementalDecoder`** instance.

All incremental decoders must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The **`IncrementalDecoder`** may implement different error handling schemes by providing the *errors* keyword argument. See Error Handlers for possible values.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the **`IncrementalDecoder`** object.

decode(*object*, *final=False*)

Decodes *object* (taking the current state of the decoder into account) and returns the resulting decoded object. If this is the last call to **`decode()`** *final* must be true (the default is false). If *final* is true the decoder must decode the input completely and must flush all buffers. If this isn't possible (e.g. because of incomplete byte sequences at the end of the input) it must initiate error handling just like in the stateless case (which might raise an exception).

reset()

Reset the decoder to the initial state.

getstate()

Return the current state of the decoder. This must be a tuple with two items, the first must be the buffer containing the still undecoded input. The second must be an integer and can be additional state info. (The implementation should make sure that `0` is the most common additional state info.) If this additional state info is `0` it must be possible to set the decoder to the state which has no input buffered and `0` as the additional state info, so that feeding the previously buffered input to the decoder returns it to the previous state without producing any output. (Additional state info that is more complicated than integers can be converted into an integer by marshaling/pickling the info and encoding the bytes of the resulting string into an integer.)

setstate(*state*)

Set the state of the decoder to *state*. *state* must be a decoder state returned by **`getstate()`**.

Stream Encoding and Decoding

The `StreamWriter` and `StreamReader` classes provide generic working interfaces which can be used to implement new encoding submodules very easily. See `encodings.utf_8` for an example of how this is done.

StreamWriter Objects

The `StreamWriter` class is a subclass of `Codec` and defines the following methods which every stream writer must define in order to be compatible with the Python codec registry.

class codecs.StreamWriter(*stream*, *errors='strict'*)

Constructor for a `StreamWriter` instance.

All stream writers must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The *stream* argument must be a file-like object open for writing text or binary data, as appropriate for the specific codec.

The `StreamWriter` may implement different error handling schemes by providing the *errors* keyword argument. See Error Handlers for the standard error handlers the underlying stream codec may support.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the `StreamWriter` object.

write(*object*)

Writes the object's contents encoded to the stream.

writelines(*list*)

Writes the concatenated iterable of strings to the stream (possibly by reusing the `write()` method). Infinite or very large iterables are not supported. The standard bytes-to-bytes codecs do not support this method.

reset()

Resets the codec buffers used for keeping internal state.

Chamar este método deve garantir que os dados na saída estejam num estado limpo, que permite anexar novos dados sem ter que verificar novamente todo o fluxo para recuperar o estado.

In addition to the above methods, the `StreamWriter` must also inherit all other methods and attributes from the underlying stream.

StreamReader Objects

The `StreamReader` class is a subclass of `Codec` and defines the following methods which every stream reader must define in order to be compatible with the Python codec registry.

class codecs.StreamReader(*stream*, *errors='strict'*)

Constructor for a `StreamReader` instance.

All stream readers must provide this constructor interface. They are free to add additional keyword arguments, but only the ones defined here are used by the Python codec registry.

The *stream* argument must be a file-like object open for reading text or binary data, as appropriate for the specific codec.

The `StreamReader` may implement different error handling schemes by providing the *errors* keyword argument. See Error Handlers for the standard error handlers the underlying stream codec may support.

The *errors* argument will be assigned to an attribute of the same name. Assigning to this attribute makes it possible to switch between different error handling strategies during the lifetime of the `StreamReader` object.

The set of allowed values for the *errors* argument can be extended with `register_error()`.

read(*size=-1*, *chars=-1*, *firstline=False*)

Decodes data from the stream and returns the resulting object.

The *chars* argument indicates the number of decoded code points or bytes to return. The `read()` method will never return more data than requested, but it might return less, if there is not enough available.

The *size* argument indicates the approximate maximum number of encoded bytes or code points to read for decoding. The decoder can modify this setting as appropriate. The default value -1 indicates to read and decode as much as possible. This parameter is intended to prevent having to decode huge files in one step.

The *firstline* flag indicates that it would be sufficient to only return the first line, if there are decoding errors on later lines.

The method should use a greedy read strategy meaning that it should read as much data as is allowed within the definition of the encoding and the given size, e.g. if optional encoding endings or state markers are available on the stream, these should be read too.

readline(*size=None*, *keepends=True*)

Read one line from the input stream and return the decoded data.

size, if given, is passed as size argument to the stream's `read()` method.

If *keepends* is false line-endings will be stripped from the lines returned.

readlines(*sizehint=None*, *keepends=True*)

Read all lines available on the input stream and return them as a list of lines.

Line-endings are implemented using the codec's `decode()` method and are included in the list entries if *keepends* is true.

sizehint, if given, is passed as the *size* argument to the stream's `read()` method.

reset()

Resets the codec buffers used for keeping internal state.

Note that no stream repositioning should take place. This method is primarily intended to be able to recover from decoding errors.

In addition to the above methods, the `StreamReader` must also inherit all other methods and attributes from the underlying stream.

StreamReaderWriter Objects

The `StreamReaderWriter` is a convenience class that allows wrapping streams which work in both read and write modes.

The design is such that one can use the factory functions returned by the `lookup()` function to construct the instance.

class codecs.StreamReaderWriter(*stream*, *Reader*, *Writer*, *errors='strict'*)

Creates a `StreamReaderWriter` instance. *stream* must be a file-like object. *Reader* and *Writer* must be factory functions or classes providing the `StreamReader` and `StreamWriter` interface resp. Error handling is done in the same way as defined for the stream readers and writers.

`StreamReaderWriter` instances define the combined interfaces of `StreamReader` and `StreamWriter` classes. They inherit all other methods and attributes from the underlying stream.

StreamRecoder Objects

The `StreamRecoder` translates data from one encoding to another, which is sometimes useful when dealing with different encoding environments.

The design is such that one can use the factory functions returned by the `lookup()` function to construct the instance.

class codecs.StreamRecoder(*stream*, *encode*, *decode*, *Reader*, *Writer*, *errors='strict'*)

Creates a **`StreamRecoder`** instance which implements a two-way conversion: *encode* and *decode* work on the frontend — the data visible to code calling **`read()`** and **`write()`**, while *Reader* and *Writer* work on the backend — the data in *stream*.

You can use these objects to do transparent transcodings, e.g., from Latin-1 to UTF-8 and back.

The *stream* argument must be a file-like object.

The *encode* and *decode* arguments must adhere to the **`Codec`** interface. *Reader* and *Writer* must be factory functions or classes providing objects of the **`StreamReader`** and **`StreamWriter`** interface respectively.

Error handling is done in the same way as defined for the stream readers and writers.

`StreamRecoder` instances define the combined interfaces of **`StreamReader`** and **`StreamWriter`** classes. They inherit all other methods and attributes from the underlying stream.

Encodings and Unicode

Strings are stored internally as sequences of code points in range `U+0000`–`U+10FFFF`. (See **PEP 393** [https://peps.python.org/pep-0393/] for more details about the implementation.) Once a string object is used outside of CPU and memory, endianness and how these arrays are stored as bytes become an issue. As with other codecs, serialising a string into a sequence of bytes is known as *encoding*, and recreating the string from the sequence of bytes is known as *decoding*. There are a variety of different text serialisation codecs, which are collectivity referred to as text encodings.

The simplest text encoding (called `'latin-1'` or `'iso-8859-1'`) maps the code points 0–255 to the bytes `0x0`–`0xff`, which means that a string object that contains code points above `U+00FF` can't be encoded with this codec.

Doing so will raise a **`UnicodeEncodeError`** that looks like the following (although the details of the error message may differ):

```
UnicodeEncodeError: 'latin-1' codec can't encode character
'\u1234' in position 3: ordinal not in range(256).
```

There's another group of encodings (the so called charmap encodings) that choose a different subset of all Unicode code points and how these code points are mapped to the bytes `0x0`–`0xff`. To see how this is done simply open e.g. `encodings/cp1252.py` (which is an encoding that is used primarily on Windows). There's a string constant with 256 characters that shows you which character is mapped to which byte value.

All of these encodings can only encode 256 of the 1114112 code points defined in Unicode. A simple and straightforward way that can store each Unicode code point, is to store each code point as four consecutive bytes. There are two possibilities: store the bytes in big endian or in little endian order. These two encodings are called `UTF-32-BE` and `UTF-32-LE` respectively. Their disadvantage is that if, for example, you use `UTF-32-BE` on a little endian machine you will always have to swap bytes on encoding and decoding. Python's `UTF-16` and `UTF-32` codecs avoid this problem by using the platform's native byte order when no BOM is present. Python follows prevailing platform practice, so native-endian data round-trips without redundant byte swapping, even though the Unicode Standard defaults to big-endian when the byte order is unspecified. When these bytes are read by a CPU with a different endianness, the bytes have to be swapped. To be able to detect the endianness of a `UTF-16` or `UTF-32` byte sequence, a BOM ("Byte Order Mark") is used. This is the Unicode character `U+FEFF`. This character can be prepended to every `UTF-16` or `UTF-32` byte sequence. The byte swapped version of this character (`0xFFFE`) is an illegal character that may not appear in a Unicode text. When the first character of a `UTF-16` or `UTF-32` byte sequence is `U+FFFE`, the bytes have to be swapped on decoding.

Unfortunately the character `U+FEFF` had a second purpose as a `ZERO WIDTH NO-BREAK SPACE`: a character that has no width and doesn't allow a word to be split. It can e.g. be used to give hints to a ligature algorithm. With Unicode 4.0 using `U+FEFF` as a `ZERO WIDTH NO-BREAK SPACE` has been

deprecated (with `U+2060` (`WORD JOINER`) assuming this role). Nevertheless Unicode software still must be able to handle `U+FEFF` in both roles: as a BOM it's a device to determine the storage layout of the encoded bytes, and vanishes once the byte sequence has been decoded into a string; as a `ZERO WIDTH NO-BREAK SPACE` it's a normal character that will be decoded like any other.

There's another encoding that is able to encode the full range of Unicode characters: UTF-8. UTF-8 is an 8-bit encoding, which means there are no issues with byte order in UTF-8. Each byte in a UTF-8 byte sequence consists of two parts: marker bits (the most significant bits) and payload bits. The marker bits are a sequence of zero to four `1` bits followed by a `0` bit. Unicode characters are encoded like this (with x being payload bits, which when concatenated give the Unicode character):

Range	Encoding
`U-00000000 ... U-0000007F`	0xxxxxxx
`U-00000080 ... U-000007FF`	110xxxxx 10xxxxxx
`U-00000800 ... U-0000FFFF`	1110xxxx 10xxxxxx 10xxxxxx
`U-00010000 ... U-0010FFFF`	11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

The least significant bit of the Unicode character is the rightmost x bit.

As UTF-8 is an 8-bit encoding no BOM is required and any `U+FEFF` character in the decoded string (even if it's the first character) is treated as a `ZERO WIDTH NO-BREAK SPACE`.

Without external information it's impossible to reliably determine which encoding was used for encoding a string. Each charmap encoding can decode any random byte sequence. However that's not possible with UTF-8, as UTF-8 byte sequences have a structure that doesn't allow arbitrary byte sequences. To increase the reliability with which a UTF-8 encoding can be detected, Microsoft invented a variant of UTF-8 (that Python calls `"utf-8-sig"`) for its Notepad program: Before any of the Unicode characters is written to the file, a UTF-8 encoded BOM (which looks like this as a byte sequence: `0xef`, `0xbb`, `0xbf`) is written. As it's rather improbable that any charmap encoded file starts with these byte values (which would e.g. map to

> LATIN SMALL LETTER I WITH DIAERESIS
> RIGHT-POINTING DOUBLE ANGLE QUOTATION MARK
> INVERTED QUESTION MARK

in iso-8859-1), this increases the probability that a `utf-8-sig` encoding can be correctly guessed from the byte sequence. So here the BOM is not used to be able to determine the byte order used for generating the byte sequence, but as a signature that helps in guessing the encoding. On encoding the utf-8-sig codec will write `0xef`, `0xbb`, `0xbf` as the first three bytes to the file. On decoding `utf-8-sig` will skip those three bytes if they appear as the first three bytes in the file. In UTF-8, the use of the BOM is discouraged and should generally be avoided.

Standard Encodings

Python comes with a number of codecs built-in, either implemented as C functions or with dictionaries as mapping tables. The following table lists the codecs by name, together with a few common aliases, and the languages for which the encoding is likely used. Neither the list of aliases nor the list of languages is meant to be exhaustive. Notice that spelling alternatives that only differ in case or use a space or a hyphen instead of an underscore are also valid aliases because they are equivalent when normalized by **`normalize_encoding()`**. For example, `'utf-8'` is a valid alias for the `'utf_8'` codec.

Nota

The below table lists the most common aliases, for a complete list refer to the source aliases.py [https://github.com/python/cpython/tree/3.15/Lib/encodings/aliases.py] file.

On Windows, `cpXXX` codecs are available for all code pages. But only codecs listed in the following table are guaranteed to exist on other platforms.

Detalhes da implementação do CPython: Some common encodings can bypass the codecs lookup machinery to improve performance. These optimization opportunities are only recognized by CPython for a limited set of (case insensitive) aliases: utf-8, utf8, latin-1, latin1, iso-8859-1, iso8859-1, mbcs (Windows only), ascii, us-ascii, utf-16, utf16, utf-32, utf32, and the same using underscores instead of dashes. Using alternative aliases for these encodings may result in slower execution.

Alterado na versão 3.6: Optimization opportunity recognized for us-ascii.

Many of the character sets support the same languages. They vary in individual characters (e.g. whether the EURO SIGN is supported or not), and in the assignment of characters to code positions. For the European languages in particular, the following variants typically exist:

- an ISO 8859 codeset
- a Microsoft Windows code page, which is typically derived from an 8859 codeset, but replaces control characters with additional graphic characters
- an IBM EBCDIC code page
- an IBM PC code page, which is ASCII compatible

Codec	Aliases	Idiomas

Codec	Aliases	Idiomas
ascii	646, us-ascii	Inglês
big5	big5-tw, csbig5	Traditional Chinese
big5hkscs	big5-hkscs, hkscs	Traditional Chinese
cp037	IBM037, IBM039	Inglês
cp273	273, IBM273, csIBM273	Alemão *Adicionado na versão 3.4.*
cp424	EBCDIC-CP-HE, IBM424	Hebraico
cp437	437, IBM437	Inglês
cp500	EBCDIC-CP-BE, EBCDIC-CP-CH, IBM500	Western Europe
cp720		Árabe
cp737		Grego

Codec	Aliases	Idiomas
cp775	IBM775	Baltic languages
cp850	850, IBM850	Western Europe
cp852	852, IBM852	Central and Eastern Europe
cp855	855, IBM855	Belarusian, Bulgarian, Macedonian, Russian, Serbian
cp856		Hebraico
cp857	857, IBM857	Turco
cp858	858, IBM00858	Western Europe
cp860	860, IBM860	Português
cp861	861, CP-IS, IBM861	Islandês
cp862	862, IBM862	Hebraico

Codec	Aliases	Idiomas
cp863	863, IBM863	Canadense
cp864	IBM864	Árabe
cp865	865, IBM865	Danish, Norwegian
cp866	866, IBM866	Russo
cp869	869, CP-GR, IBM869	Grego
cp874		Tailandês
cp875		Grego
cp932	932, ms932, mskanji, ms-kanji, windows-31j	Japonês
cp949	949, ms949, uhc	Coreano
cp950	950, ms950	Traditional Chinese
cp1006		Urdu

Codec	Aliases	Idiomas
cp1026	ibm1026	Turco
cp1125	1125, ibm1125, cp866u, ruscii	Ucraniano *Adicionado na versão 3.4.*
cp1140	IBM01140	Western Europe
cp1250	windows-1250	Central and Eastern Europe
cp1251	windows-1251	Belarusian, Bulgarian, Macedonian, Russian, Serbian
cp1252	windows-1252	Western Europe
cp1253	windows-1253	Grego
cp1254	windows-1254	Turco
cp1255	windows-1255	Hebraico

Codec	Aliases	Idiomas
cp1256	windows-1256	Árabe
cp1257	windows-1257	Baltic languages
cp1258	windows-1258	Vietnamita
euc_jp	eucjp, ujis, u-jis	Japonês
euc_jis_2004	jisx0213, eucjis2004	Japonês
euc_jisx0213	eucjisx0213	Japonês
euc_kr	euckr, korean, ksc5601, ks_c-5601, ks_c-5601-1987, ksx1001, ks_x-1001	Coreano
gb2312	chinese, csiso58gb231280, euc-cn, euccn, eucgb2312-cn, gb2312-1980, gb2312-80, iso-ir-58	Simplified Chinese
gbk	936, cp936, ms936	Unified Chinese
gb18030	gb18030-2000	Unified Chinese

Codec	Aliases	Idiomas
hz	hzgb, hz-gb, hz-gb-2312	Simplified Chinese
iso2022_jp	csiso2022jp, iso2022jp, iso-2022-jp	Japonês
iso2022_jp_1	iso2022jp-1, iso-2022-jp-1	Japonês
iso2022_jp_2	iso2022jp-2, iso-2022-jp-2	Japanese, Korean, Simplified Chinese, Western Europe, Greek
iso2022_jp_2004	iso2022jp-2004, iso-2022-jp-2004	Japonês
iso2022_jp_3	iso2022jp-3, iso-2022-jp-3	Japonês
iso2022_jp_ext	iso2022jp-ext, iso-2022-jp-ext	Japonês
iso2022_kr	csiso2022kr, iso2022kr, iso-2022-kr	Coreano
latin_1	iso-8859-1, iso8859-1, 8859, cp819, latin, latin1, L1	Western Europe

Codec	Aliases	Idiomas
iso8859_2	iso-8859-2, latin2, L2	Central and Eastern Europe
iso8859_3	iso-8859-3, latin3, L3	Esperanto, Maltese
iso8859_4	iso-8859-4, latin4, L4	Northern Europe
iso8859_5	iso-8859-5, cyrillic	Belarusian, Bulgarian, Macedonian, Russian, Serbian
iso8859_6	iso-8859-6, arabic	Árabe
iso8859_7	iso-8859-7, greek, greek8	Grego
iso8859_8	iso-8859-8, hebrew	Hebraico
iso8859_9	iso-8859-9, latin5, L5	Turco
iso8859_10	iso-8859-10, latin6, L6	Nordic languages
iso8859_11	iso-8859-11, thai	Thai languages

Codec	Aliases	Idiomas
iso8859_13	iso-8859-13, latin7, L7	Baltic languages
iso8859_14	iso-8859-14, latin8, L8	Celtic languages
iso8859_15	iso-8859-15, latin9, L9	Western Europe
iso8859_16	iso-8859-16, latin10, L10	South-Eastern Europe
johab	cp1361, ms1361	Coreano
koi8_r		Russo
koi8_t		Tajik *Adicionado na versão 3.5.*
koi8_u		Ucraniano
kz1048	kz_1048, strk1048_2002, rk1048	Cazaque *Adicionado na versão 3.5.*

Codec	Aliases	Idiomas
mac_cyrillic	maccyrillic	Belarusian, Bulgarian, Macedonian, Russian, Serbian
mac_greek	macgreek	Grego
mac_iceland	maciceland	Islandês
mac_latin2	maclatin2, maccentraleurope, mac_centeuro	Central and Eastern Europe
mac_roman	macroman, macintosh	Western Europe
mac_turkish	macturkish	Turco
ptcp154	csptcp154, pt154, cp154, cyrillic-asian	Cazaque
shift_jis	csshiftjis, shiftjis, sjis, s_jis	Japonês
shift_jis_2004	shiftjis2004, sjis_2004, sjis2004	Japonês
shift_jisx0213	shiftjisx0213, sjisx0213, s_jisx0213	Japonês

Codec	Aliases	Idiomas
utf_32	U32, utf32	todas linguagens
utf_32_be	UTF-32BE	todas linguagens
utf_32_le	UTF-32LE	todas linguagens
utf_16	U16, utf16	todas linguagens
utf_16_be	UTF-16BE	todas linguagens
utf_16_le	UTF-16LE	todas linguagens
utf_7	U7, unicode-1-1-utf-7	todas linguagens
utf_8	U8, UTF, utf8, cp65001	todas linguagens
utf_8_sig	utf8-sig	todas linguagens

Alterado na versão 3.4: The utf-16* and utf-32* encoders no longer allow surrogate code points (`U+D800`—`U+DFFF`) to be encoded. The utf-32* decoders no longer decode byte sequences that correspond to surrogate code points.

Alterado na versão 3.8: `cp65001` is now an alias to `utf_8`.

Alterado na versão 3.14: On Windows, `cpXXX` codecs are now available for all code pages.

Python Specific Encodings

A number of predefined codecs are specific to Python, so their codec names have no meaning outside Python. These are listed in the tables below based on the expected input and output types (note that while text encodings are the most common use case for codecs, the underlying codec infrastructure supports arbitrary data transforms rather than just text encodings). For asymmetric codecs, the stated meaning describes the encoding direction.

Text Encodings

The following codecs provide **`str`** to **`bytes`** encoding and bytes-like object to **`str`** decoding, similar to the Unicode text encodings.

Codec	Aliases	Significado
idna		Implement **RFC 3490**, see also **`encodings.idna`**. Only `errors='strict'` is supported. **Aviso** This codec builds on `punycode`, whose algorithms scale poorly, so limit the length of untrusted input.
mbcs	ansi, dbcs	Windows only: Encode the operand according to the ANSI codepage (CP_ACP).

Codec	Aliases	Significado
oem		Windows only: Encode the operand according to the OEM codepage (CP_OEMCP). *Adicionado na versão 3.6.*
palmos		Encoding of PalmOS 3.5.
punycode		Implement **RFC 3492**. Stateful codecs are not supported. **Aviso** The decoding and encoding algorithms scale poorly, so limit the length of untrusted input.
raw_unicode_escape		Latin-1 encoding with `\u`*xxxx* and `\U`*xxxxxxxx* for other code points. Existing backslashes are not escaped in any way. It is used in the Python pickle protocol.
undefined		This Codec should only be used for testing purposes. Raise an exception for all conversions, even empty strings. The error handler is ignored.

Codec	Aliases	Significado
unicode_escape		Encoding suitable as the contents of a Unicode literal in ASCII-encoded Python source code, except that quotes are not escaped. Decode from Latin-1 source code. Beware that Python source code actually uses UTF-8 by default.

Alterado na versão 3.8: “unicode_internal” codec is removed.

Binary Transforms

The following codecs provide binary transforms: bytes-like object to `bytes` mappings. They are not supported by `bytes.decode()` (which only produces `str` output).

Codec	Aliases	Significado	Encoder / decoder

Codec	Aliases	Significado	Encoder / decoder
base64_codec [[1]]	base64, base_64	Convert the operand to multiline MIME base64 (the result always includes a trailing `'\n'`). *Alterado na versão 3.4:* accepts any bytes-like object as input for encoding and decoding	`base64.encodebytes()` / `base64.decodebytes()`
bz2_codec	bz2	Compress the operand using bz2.	`bz2.compress()` / `bz2.decompress()`
hex_codec	hex	Convert the operand to hexadecimal representation, with two digits per byte.	`binascii.b2a_hex()` / `binascii.a2b_hex()`

Codec	Aliases	Significado	Encoder / decoder
quopri_codec	quopri, quotedprintable, quoted_printable	Convert the operand to MIME quoted printable.	`quopri.encode()` with `quotetabs=True` / `quopri.decode()`
uu_codec	uu	Convert the operand using uuencode.	
zlib_codec	zip, zlib	Compress the operand using gzip.	`zlib.compress()` / `zlib.decompress()`

[1]

In addition to bytes-like objects, `'base64_codec'` also accepts ASCII-only instances of `str` for decoding

Adicionado na versão 3.2: Restoration of the binary transforms.

Alterado na versão 3.4: Restoration of the aliases for the binary transforms.

Standalone Codec Functions

The following functions provide encoding and decoding functionality similar to codecs, but are not available as named codecs through `codecs.encode()` or `codecs.decode()`. They are used internally (for example, by `pickle`) and behave similarly to the `string_escape` codec that was removed in Python 3.

codecs.escape_encode(*input*, *errors=None*)

Encode *input* using escape sequences. Similar to how `repr()` on bytes produces escaped byte values.

input must be a `bytes` object.

Returns a tuple `(output, length)` where *output* is a `bytes` object and *length* is the number of bytes consumed.

codecs.escape_decode(*input*, *errors=None*)

Decode *input* from escape sequences back to the original bytes.

input must be a bytes-like object.

Returns a tuple `(output, length)` where *output* is a `bytes` object and *length* is the number of bytes consumed.

Text Transforms

The following codec provides a text transform: a `str` to `str` mapping. It is not supported by `str.encode()` (which only produces `bytes` output).

Codec	Aliases	Significado
rot_13	rot13	Return the Caesar-cypher encryption of the operand.

Adicionado na versão 3.2: Restoration of the `rot_13` text transform.

Alterado na versão 3.4: Restoration of the `rot13` alias.

`encodings` — Encodings package

This module implements the following functions:

encodings.normalize_encoding(*encoding*)

Normalize encoding name *encoding*.

Normalization works as follows: all non-alphanumeric characters except the dot used for Python package names are collapsed and replaced with a single underscore, leading and trailing underscores are removed. For example, `'  -;#'` becomes `'_'`.

Note that *encoding* should be ASCII only.

Nota

The following functions should not be used directly, except for testing purposes; **`codecs.lookup()`** should be used instead.

encodings.search_function(*encoding*)

Search for the codec module corresponding to the given encoding name *encoding*.

This function first normalizes the *encoding* using **`normalize_encoding()`**, then looks for a corresponding alias. It attempts to import a codec module from the encodings package using either the alias or the normalized name. If the module is found and defines a valid `getregentry()` function that returns a **`codecs.CodecInfo`** object, the codec is cached and returned.

If the codec module defines a `getaliases()` function any returned aliases are registered for future use.

encodings.win32_code_page_search_function(*encoding*)

Search for a Windows code page encoding *encoding* of the form `cpXXXX`.

If the code page is valid and supported, return a **`codecs.CodecInfo`** object for it.

Disponibilidade: Windows.

Adicionado na versão 3.14.

This module implements the following exception:

exception encodings.CodecRegistryError
Raised when a codec is invalid or incompatible.

`encodings.idna` — Internationalized Domain Names in Applications

This module implements **RFC 3490** [https://datatracker.ietf.org/doc/html/rfc3490.html] (Internationalized Domain Names in Applications) and **RFC 3492** [https://datatracker.ietf.org/doc/html/rfc3492.html] (Nameprep: A Stringprep Profile for Internationalized Domain Names (IDN)). It builds upon the `punycode` encoding and `stringprep`.

Aviso

This module builds on `punycode`, whose algorithms scale poorly, so limit the length of untrusted input.

If you need the IDNA 2008 standard from **RFC 5891** [https://datatracker.ietf.org/doc/html/rfc5891.html] and **RFC 5895** [https://datatracker.ietf.org/doc/html/rfc5895.html], use the third-party idna [https://pypi.org/project/idna/] module.

These RFCs together define a protocol to support non-ASCII characters in domain names. A domain name containing non-ASCII characters (such as `www.Alliancefrançaise.nu`) is converted into an ASCII-compatible encoding (ACE, such as `www.xn--alliancefranaise-npb.nu`). The ACE form of the domain name is then used in all places where arbitrary characters are not allowed by the protocol, such as DNS queries, HTTP *Host* fields, and so on. This conversion is carried out in the application; if possible invisible to the user: The application should transparently convert

Unicode domain labels to IDNA on the wire, and convert back ACE labels to Unicode before presenting them to the user.

Python supports this conversion in several ways: the `idna` codec performs conversion between Unicode and ACE, separating an input string into labels based on the separator characters defined in **section 3.1 of RFC 3490** [https://datatracker.ietf.org/doc/html/rfc3490.html#section-3.1] and converting each label to ACE as required, and conversely separating an input byte string into labels based on the `.` separator and converting any ACE labels found into unicode. Furthermore, the **`socket`** module transparently converts Unicode host names to ACE, so that applications need not be concerned about converting host names themselves when they pass them to the socket module. On top of that, modules that have host names as function parameters, such as **`http.client`** and **`ftplib`**, accept Unicode host names (**`http.client`** then also transparently sends an IDNA hostname in the *Host* field if it sends that field at all).

When receiving host names from the wire (such as in reverse name lookup), no automatic conversion to Unicode is performed: applications wishing to present such host names to the user should decode them to Unicode.

The module **`encodings.idna`** also implements the nameprep procedure, which performs certain normalizations on host names, to achieve case-insensitivity of international domain names, and to unify similar characters. The nameprep functions can be used directly if desired.

encodings.idna.nameprep(*label*)

Return the nameprepped version of *label*. The implementation currently assumes query strings, so `AllowUnassigned` is true.

encodings.idna.ToASCII(*label*)

Convert a label to ASCII, as specified in **RFC 3490** [https://datatracker.ietf.org/doc/html/rfc3490.html]. `UseSTD3ASCIIRules` is assumed to be false.

encodings.idna.ToUnicode(*label*)

Convert a label to Unicode, as specified in **RFC 3490** [https://datatracker.ietf.org/doc/html/rfc3490.html].

`encodings.mbcs` — Windows ANSI codepage

This module implements the ANSI codepage (CP_ACP).

Disponibilidade: Windows.

Alterado na versão 3.2: Before 3.2, the *errors* argument was ignored; `'replace'` was always used to encode, and `'ignore'` to decode.

Alterado na versão 3.3: Support any error handler.

`encodings.utf_8_sig` — UTF-8 codec with BOM signature

This module implements a variant of the UTF-8 codec. On encoding, a UTF-8 encoded BOM will be prepended to the UTF-8 encoded bytes. For the stateful encoder this is only done once (on the first write to the byte stream). On decoding, an optional UTF-8 encoded BOM at the start of the data will be skipped.

Tipos de Dados

Os módulos descritos neste capítulo fornecem uma variedade de tipos de dados especializados, como datas e horas, vetores de tipo fixo, filas de heap, filas de extremidade dupla e enumerações.

O Python também fornece alguns tipos de dados embutidos, em especial `list`, `tuple`, `dict`, `frozendict`, `set` e `frozenset`. A classe `str` é usada para armazenar strings Unicode, e as classes `bytes` e `bytearray` são usadas para armazenar dados binários.

Os seguintes módulos estão documentados neste capítulo:

- `datetime` — Tipos básicos de data e hora
  - Aware and naive objects
  - Constantes
  - Available types
    - Common properties
    - Determining if an object is aware or naive
  - `timedelta` objects
    - Examples of usage: `timedelta`
  - Objetos `date`
    - Examples of usage: `date`
  - `datetime` objects
    - Examples of usage: `datetime`
  - Objetos `time`
    - Examples of usage: `time`
  - `tzinfo` objects
  - `timezone` objects
  - `strftime()` and `strptime()` behavior
    - `strftime()` and `strptime()` format codes
    - Technical detail
- `zoneinfo` — Suporte a fuso horário da IANA
  - Usando `ZoneInfo`
  - Fontes de dados

        - Configurando as fontes de dados
            - Configuração em tempo de compilação
            - Configuração do ambiente
            - Configuração em tempo de execução
    - A classe `ZoneInfo`
        - String representations
        - Pickle serialization
    - Funções
    - Globals
    - Exceptions and warnings
- `calendar` — Funções gerais relacionadas ao calendário
    - Uso na linha de comando
- `collections` — Tipos de dados de contêineres
    - Objetos `ChainMap`
        - Exemplos e Receitas de `ChainMap`
    - Objetos `Counter`
    - Objetos `deque`
        - Receitas de `deque`
    - Objetos `defaultdict`
        - Exemplos de `defaultdict`
    - Função de fábrica para tuplas com campos nomeados `namedtuple()`
    - Objetos `OrderedDict`
        - Exemplos e receitas de `OrderedDict`
    - Objetos `UserDict`
    - Objetos `UserList`
    - Objetos `UserString`
- `collections.abc` — Classes Base Abstratas para Contêineres
    - Classes Base Abstratas de Coleções
    - Classes Base Abstrata de Coleções – Descrições Detalhadas
    - Exemplos e receitas
- `heapq` — Algoritmo de fila heap
    - Exemplos básicos
    - Other Applications
    - Notas de implementação da fila de prioridade
    - Teoria
- `bisect` — Algoritmo de bisseção de vetor

    - Observações sobre desempenho
    - Buscando em listas ordenadas
    - Exemplos
- `array` — Vetores eficientes de valores numéricos
- `weakref` — Referências fracas
    - Objetos referência fraca
    - Exemplo
    - Objetos finalizadores
    - Comparando finalizadores com métodos `__del__()`
- `types` — Criação de tipos dinâmicos e nomes para tipos embutidos
    - Criação de tipos dinâmicos
    - Tipos padrão do interpretador
    - Classes e funções de utilidades adicionais
    - Funções de utilidade de corrotina
- `copy` — Operações de cópia profunda e cópia rasa
- `pprint` — Impressão bonita de dados
    - Funções
    - Objetos PrettyPrinter
    - Exemplo
- `reprlib` — Implementação alternativa à `repr()`
    - Objetos Repr
    - Estendendo objetos Repr
- `enum` — Suporte a enumerações
    - Conteúdo do módulo
    - Data types
        - Nomes `__dunder__` suportados
        - Nomes `_sunder_` suportados
    - Utilities and decorators
    - Notas
- `graphlib` — Funcionalidade para operar com estruturas do tipo grafo
    - Exceções

`datetime` — Tipos básicos de data e hora

Código-fonte: Lib/datetime.py
[https://github.com/python/cpython/tree/3.15/Lib/datetime.py]

O módulo **`datetime`** fornece classes para manipulação de datas e horas.

Ainda que a aritmética de data e hora seja suportada, o foco da implementação é a extração eficiente de atributos para a formatação e manipulação da saída.

Dica

Pule para os códigos de formatação.

Ver também

Módulo `calendar`
Funções gerais relacionadas ao calendário.

Módulo `time`
Acesso e conversões de tempo

Módulo `zoneinfo`
Fusos horários concretos que representam o banco de dados de fusos horários da IANA.

Pacote dateutil [https://dateutil.readthedocs.io/en/stable/]
Biblioteca de terceiros com fuso horário expandido e suporte à análise.

Pacote DateType [https://pypi.org/project/DateType/]

Biblioteca de terceiros que apresenta tipos estáticos distintos para, por exemplo, permitir que verificadores de tipo estático diferenciem datas ingênuas e conscientes.

Aware and naive objects

Objetos de data e hora podem ser categorizados como “consciente” ou “ingênuo” dependendo se eles incluem ou não informação sobre fuso horário.

Com conhecimento suficiente dos ajustes de tempo algorítmicos e políticos aplicáveis, como informações de fuso horário e horário de verão, um objeto **consciente** pode se localizar em relação a outros objetos conscientes. Um objeto consciente representa um momento específico no tempo que não está aberto à interpretação. [[1]]

Um objeto **ingênuo** não contém informações suficientes para se localizar inequivocamente em relação a outros objetos de data/hora. Se um objeto ingênuo representa o Coordinated Universal Time (UTC), a hora local, ou a hora em algum outro fuso horário, isso depende exclusivamente do programa, assim como é tarefa do programa decidir se um número específico representa metros, milhas ou massa. Objetos ingênuos são fáceis de entender e trabalhar, com o custo de ignorar alguns aspectos da realidade.

For applications requiring aware objects, `datetime` and `time` objects have an optional time zone information attribute, `tzinfo`, that can be set to an instance of a subclass of the abstract `tzinfo` class. These `tzinfo` objects capture information about the offset from UTC time, the time zone name, and whether daylight saving time is in effect.

Somente uma classe concreta `tzinfo`, a classe `timezone`, é fornecida pelo módulo `datetime`. A classe `timezone` pode representar fusos horários simples com diferenças fixas do UTC, como o próprio UTC, ou os fusos horários norte-americanos EST e EDT. O suporte a fusos horários em níveis

mais detalhados depende da aplicação. As regras para ajuste de tempo em todo o mundo são mais políticas do que racionais, mudam com frequência e não há um padrão adequado para todas as aplicações além da UTC.

Constantes

O módulo **`datetime`** exporta as seguintes constantes:

datetime.MINYEAR

O menor número de ano permitido em um objeto `date` ou `datetime`. `MINYEAR` é 1.

datetime.MAXYEAR

O maior número de ano permitido no objeto `date` ou `datetime`. `MAXYEAR` é 9999.

datetime.UTC

Apelido para o singleton de fuso horário UTC `datetime.timezone.utc`.

Adicionado na versão 3.11.

Available types

class datetime.date

Uma data ingênua idealizada, presumindo que o atual calendário Gregoriano sempre foi, e sempre estará em vigor. Atributos: `year`, `month` e `day`.

class datetime.time

Um horário ideal, independente de qualquer dia em particular, presumindo que todos os dias tenham exatamente 24*60*60 segundos. (Não há noção de “segundos bissextos” aqui.) Atributos: `hour`, `minute`, `second`, `microsecond` e `tzinfo`.

class datetime.datetime

Uma combinação de uma data e uma hora. Atributos: `year`, `month`, `day`, `hour`, `minute`, `second`, `microsecond` e `tzinfo`.

class datetime.timedelta

Uma duração que expressa a diferença entre duas instâncias `datetime` ou `date` para resolução de microssegundos.

class datetime.tzinfo

Uma classe base abstrata para objetos de informações de fuso horário. Eles são usados pelas classes `datetime` e `time` para fornecer uma noção personalizável de ajuste de horário (por exemplo, para considerar o fuso horário e/ou o horário de verão).

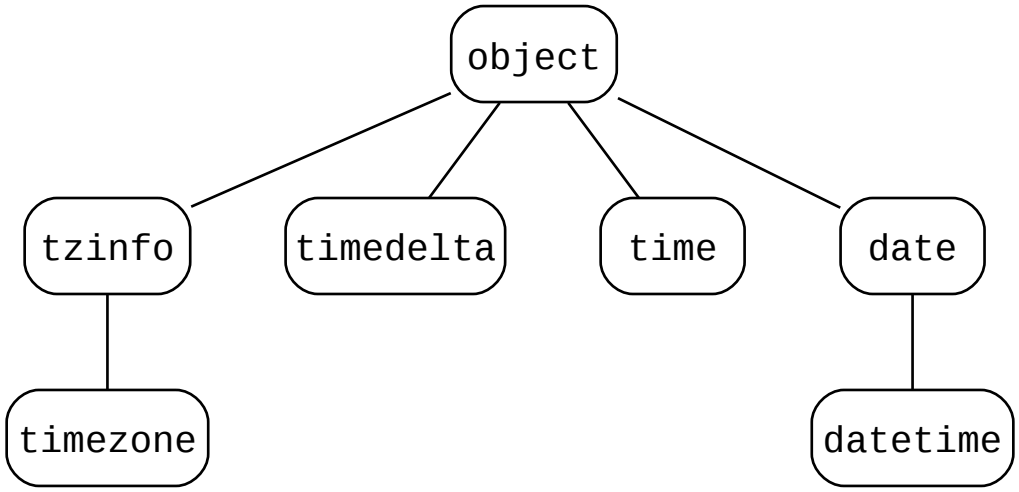
class datetime.timezone

Uma classe que implementa a classe base abstrata `tzinfo` como uma diferença fixa do UTC.

Adicionado na versão 3.2.

Objetos desse tipo são imutáveis.

Subclass relationships:



Common properties

Os tipos `date`, `datetime`, `time` e `timezone` compartilham esses recursos comuns:

- Objetos desse tipo são imutáveis.
- Objetos desses tipos são hasheáveis, o que significa que podem ser usados como chaves de dicionário.
- Objetos desse tipo suportam decapagem eficiente através do módulo `pickle`.

Determining if an object is aware or naive

Objetos do tipo `date` são sempre ingênuos.

Um objeto do tipo `time` ou `datetime` pode ser consciente ou ingênuo.

O objeto `datetime` `d` é consciente se ambos os seguintes itens forem verdadeiros:

1. `d.tzinfo` não é `None`
2. `d.tzinfo.utcoffset(d)` não retorna `None`

Caso contrário, `d` é ingênuo.

O objeto `time` `t` é consciente, se os seguintes itens são verdadeiros:

1. `t.tzinfo` não é `None`
2. `t.tzinfo.utcoffset(None)` não retorna `None`.

Caso contrário, `t` é ingênuo.

A distinção entre consciente e ingênuo não se aplica a objetos `timedelta`.

`timedelta` objects

O objeto `timedelta` representa uma duração, a diferença entre duas instâncias `datetime` ou `date`.

class datetime.timedelta(*days=0*, *seconds=0*, *microseconds=0*, *milliseconds=0*, *minutes=0*, *hours=0*, *weeks=0*)

Todos os argumentos são opcionais e o padrão é 0. Os argumentos podem ser números inteiros ou ponto flutuantes, e podem ser positivos ou negativos.

Apenas *days*, *seconds* e *microseconds* são armazenados internamente. Os argumentos são convertidos para essas unidades:

- Um milissegundo é convertido em 1000 microssegundos.
- Um minuto é convertido em 60 segundos.
- Uma hora é convertida em 3600 segundos.
- Uma semana é convertida para 7 dias.

e dias, segundos e microssegundos são normalizados para que a representação seja única, com

- `0 <= microseconds < 1000000`
- `0 <= seconds < 3600*24` (o número de segundos em um dia)
- `-999999999 <= days <= 999999999`

O exemplo a seguir ilustra como quaisquer argumentos além de *days*, *seconds* e *microseconds* são “mesclados” e normalizados nos três atributos resultantes:

```
>>> import datetime as dt
>>> delta = dt.timedelta(
...     days=50,
...     seconds=27,
...     microseconds=10,
...     milliseconds=29000,
...     minutes=5,
...     hours=8,
...     weeks=2
... )
>>> # Only days, seconds, and microseconds remain
>>> delta
datetime.timedelta(days=64, seconds=29156, microseconds=10)
```

Dica

`import datetime as dt` instead of `import datetime` or `from datetime import datetime` to avoid confusion between the module and the class. See How I Import Python's datetime Module [https://adamj.eu/tech/2019/09/12/how-i-import-pythons-datetime-module/].

Se qualquer argumento for um ponto flutuante e houver microssegundos fracionários, os microssegundos fracionários restantes de todos os argumentos serão combinados e sua soma será arredondada para o microssegundo mais próximo usando o desempatador de metade da metade para o par. Se nenhum argumento é ponto flutuante, os processos de conversão e normalização são exatos (nenhuma informação é perdida).

Se o valor normalizado de dias estiver fora do intervalo indicado, a exceção **`OverflowError`** é levantada.

Observe que a normalização de valores negativos pode ser surpreendente a princípio. Por exemplo:

```
>>> import datetime as dt
>>> d = dt.timedelta(microseconds=-1)
>>> (d.days, d.seconds, d.microseconds)
(-1, 86399, 999999)
```

Como a representação de string dos objetos **`timedelta`** pode ser confusa, use a seguinte receita para produzir um formato mais legível:

```
>>> def pretty_timedelta(td):
...     if td.days >= 0:
...         return str(td)
...     return f'-({-td!s})'
...
>>> d = timedelta(hours=-1)
>>> str(d)  # not human-friendly
'-1 day, 23:00:00'
>>> pretty_timedelta(d)
'-(1:00:00)'
```

Atributos de classe:

timedelta.min

O mais negativo objeto **timedelta**, `timedelta(-999999999)`.

timedelta.max

O mais positivo objeto **timedelta**, `timedelta(days=999999999, hours=23, minutes=59, seconds=59, microseconds=999999)`.

timedelta.resolution

A menor diferença possível entre objetos não iguais **timedelta**, `timedelta(microseconds=1)`.

Observe que, devido à normalização, `timedelta.max` é maior que `-timedelta.min`. `-timedelta.max` não é representável como um objeto **timedelta**.

Atributos de instância (somente leitura):

timedelta.days

Entre -999.999.999 e 999.999.999 inclusive

timedelta.seconds

Entre 0 e 86.399 inclusive.

Cuidado

É um bug relativamente comum utilizar este atributo de forma não intencional quando, na verdade, o objetivo é obter o valor de **total_seconds()**:

```
>>> import datetime as dt
>>> duration = dt.timedelta(seconds=11235813)
>>> duration.days, duration.seconds
(130, 3813)
>>> duration.total_seconds()
11235813.0
```

timedelta.microseconds

Entre 0 e 999.999 inclusive.

Operações suportadas:

Operação	Resultado
`t1 = t2 + t3`	Soma de `t2` e `t3`. Depois `t1 - t2 == t3` e `t1 - t3 == t2` são verdadeiros. (1)
`t1 = t2 - t3`	Diferença de `t2` e `t3`. Depois `t1 == t2 - t3` e `t2 == t1 + t3` são verdadeiros (1)(6)
`t1 = t2 * i or t1 = i * t2`	Delta multiplicado por um número inteiro. Depois `t1 // i == t2` é verdadeiro, desde que `i != 0`.
	Em geral, `t1 * i == t1 * (i-1) + t1` é verdadeiro. (1)
`t1 = t2 * f or t1 = f * t2`	Delta multiplicado por um float, ponto flutuante. O resultado é arredondado para o múltiplo mais próximo de timedelta.resolution usando a metade da metade para o par.
`f = t2 / t3`	Divisão (3) da duração total `t2` por unidade de intervalo `t3`. Retorna um objeto `float`.
`t1 = t2 / f or t1 = t2 / i`	Delta dividido por um float ou um int. O resultado é arredondado para o múltiplo mais próximo de timedelta.resolution usando a metade da metade para o par.

Operação	Resultado
`t1 = t2 // i` ou `t1 = t2 // t3`	O piso é calculado e o restante (se houver) é jogado fora. No segundo caso, um número inteiro é retornado. (3)
`t1 = t2 % t3`	O restante é calculado como um objeto **`timedelta`**. (3)
`q, r = divmod(t1, t2)`	Calcula o quociente e o resto: `q = t1 // t2` (3) e `r = t1 % t2`. `q` é um inteiro e `r` é um objeto **`timedelta`**.
`+t1`	Retorna um objeto **`timedelta`** com o mesmo valor. (2)
`-t1`	Equivalente a `timedelta(-t1.days, -t1.seconds, -t1.microseconds)` e a `t1 * -1`. (1)(4)
`abs(t)`	Equivalente a `+t` quando `t.days >= 0` e a `-t` quando `t.days < 0`. (2)
`str(t)`	Retorna uma string no formato `[D day[s], ][H]H:MM:SS[.UUUUUU]`, onde D é negativo para `t` negativo. (5)
`repr(t)`	Retorna uma representação em string do objeto **`timedelta`** como uma chamada do construtor com valores de atributos canônicos.

Notas:

1. Isso é exato, mas pode transbordar.

2. Isso é exato e não pode transbordar.

3. A divisão por zero levanta `ZeroDivisionError`.

4. `-timedelta.max` não é representável como um objeto `timedelta`.

5. As representações de string de objetos `timedelta` são normalizadas de maneira semelhante à sua representação interna. Isso leva a resultados um tanto incomuns para timedeltas negativos. Por exemplo:

```
>>> timedelta(hours=-5)
datetime.timedelta(days=-1, seconds=68400)
>>> print(_)
-1 day, 19:00:00
```

6. A expressão `t2 - t3` sempre será igual à expressão `t2 + (-t3)` exceto quando t3 for igual a `timedelta.max`; nesse caso, o primeiro produzirá um resultado enquanto o último transbordará.

Além das operações listadas acima, os objetos `timedelta` suportam certas adições e subtrações com os objetos `date` e `datetime` (veja abaixo).

Alterado na versão 3.2: Floor division and true division of a `timedelta` object by another **`timedelta`** object are now supported, as are remainder operations and the `divmod()` function. True division and multiplication of a **`timedelta`** object by a `float` object are now supported.

Objetos `timedelta` dão suporte a comparações de igualdade e ordem.

Em contexto booleano, um objeto `timedelta` é considerado verdadeiro se, e somente se, não for igual a `timedelta(0)`.

Métodos de instância:

timedelta.total_seconds()

Return the total number of seconds contained in the duration. Equivalent to `td / timedelta(seconds=1)`. For interval units other than seconds, use the division form directly (for example, `td / timedelta(microseconds=1)`).

Observe que, em intervalos de tempo muito grandes (mais de 270 anos na maioria das plataformas), esse método perde a precisão de microssegundos.

Adicionado na versão 3.2.

Examples of usage: `timedelta`

Um exemplo adicional de normalização:

```
>>> # Components of another_year add up to exactly 365 days
>>> import datetime as dt
>>> year = dt.timedelta(days=365)
>>> another_year = dt.timedelta(weeks=40, days=84, hours=23,
...                             minutes=50, seconds=600)
>>> year == another_year
True
>>> year.total_seconds()
31536000.0
```

Exemplos de aritmética com `timedelta`:

```
>>> import datetime as dt
>>> year = dt.timedelta(days=365)
>>> ten_years = 10 * year
>>> ten_years
datetime.timedelta(days=3650)
>>> ten_years.days // 365
10
>>> nine_years = ten_years - year
>>> nine_years
datetime.timedelta(days=3285)
>>> three_years = nine_years // 3
>>> three_years, three_years.days // 365
(datetime.timedelta(days=1095), 3)
```

Objetos `date`

Um objeto **`date`** representa uma data (ano, mês e dia) em um calendário idealizado, o calendário gregoriano atual, estendido indefinidamente em ambas as direções.

1º de janeiro do ano 1 é chamado de dia número 1, 2º de janeiro do ano 1 é chamado de dia número 2, e assim por diante. [[2]]

class datetime.date(*year*, *month*, *day*)

Todos os argumentos são obrigatórios. Os argumentos devem ser números inteiros nos seguintes intervalos:

- `MINYEAR <= year <= MAXYEAR`
- `1 <= month <= 12`
- `1 <= day <= número de dias no mês e ano fornecidos`

Se um argumento fora desses intervalos for fornecido, a exceção **`ValueError`** é levantada.

Outros construtores, todos os métodos de classe:

classmethod date.today()

Retorna a data local atual.

Isso é equivalente a `date.fromtimestamp(time.time())`.

classmethod date.fromtimestamp(*timestamp*)

Return the local date corresponding to the POSIX *timestamp*, such as is returned by **`time.time()`**.

Isso pode levantar **`OverflowError`**, se o registro de data e hora estiver fora do intervalo de valores suportados pela função C **`localtime()`** da plataforma, e em caso de falha **`OSError`** em **`localtime()`**. É comum que isso seja restrito a anos de 1970 a 2038. Observe que, em sistemas não POSIX que incluem segundos bissextos na sua notação de registro de data e hora, os segundos bissextos são ignorados por **`fromtimestamp()`**.

Alterado na versão 3.3: Levanta `OverflowError` ao invés de `ValueError` se o registro de data e hora estiver fora do intervalo de valores suportados pela plataforma C **`localtime()`** função. Levanta `OSError` ao invés de **`ValueError`** em falha de **`localtime()`** .

Alterado na versão 3.15: Accepts any real number as *timestamp*, not only integer or float.

classmethod date.fromordinal(*ordinal*)

Return the date corresponding to the proleptic Gregorian *ordinal*, where January 1 of year 1 has ordinal 1.

`ValueError` é levantado a menos que `1 <= ordinal <= date.max.toordinal()`. Para qualquer data `d`, `date.fromordinal(d.toordinal()) == d`.

classmethod date.fromisoformat(*date_string*)

Retorna um `date` correspondendo a *date_string* dada em qualquer formato válido de ISO 8601, com as seguintes exceções:

1. Datas de precisão reduzida não são atualmente suportadas (`YYYY-MM`, `YYYY`).
2. Representações de data estendidas não são atualmente suportadas (`±YYYYYY-MM-DD`).
3. Atualmente, as datas ordinais não são suportadas (`YYYY-OOO`).

Exemplos:

```
>>> import datetime as dt
>>> dt.date.fromisoformat('2019-12-04')
datetime.date(2019, 12, 4)
>>> dt.date.fromisoformat('20191204')
datetime.date(2019, 12, 4)
>>> dt.date.fromisoformat('2021-W01-1')
datetime.date(2021, 1, 4)
```

Adicionado na versão 3.7.

Alterado na versão 3.11: Anteriormente, este método tinha suporte apenas ao formato `YYYY-MM-DD`.

classmethod date.fromisocalendar(*year*, *week*, *day*)

Return a **date** corresponding to the ISO calendar date specified by *year*, *week* and *day*. This is the inverse of the function **date.isocalendar()**.

Adicionado na versão 3.8.

classmethod date.strptime(*date_string*, *format*)

Retorna um **date** correspondente ao *date_string*, analisado conforme *format*. Isso equivale a:

```
date(*(time.strptime(date_string, format)[0:3]))
```

ValueError é levantado se o date_string e o format não puderem ser interpretados por **time.strptime()** ou se este retornar um valor que não é uma tupla temporal. Veja também strftime() and strptime() behavior e **date.fromisoformat()**.

Nota

If *format* specifies a day of month (`%d`) without a year, **ValueError** is raised. This is to avoid a quadrennial leap year bug in code seeking to parse only a month and day as the default year used in absence of one in the format is not a leap year. The workaround is to always include a year in your *format*. If parsing *date_string* values that do not have a year, explicitly add a year that is a leap year before parsing:

```
>>> import datetime as dt
>>> date_string = "02/29"
>>> when = dt.date.strptime(f"{date_string};1984", "%m/%d;%
>>> when.strftime("%B %d")
'February 29'
```

Adicionado na versão 3.14.

Atributos de classe:

date.min

A data representável mais antiga, `date(MINYEAR, 1, 1)`.

date.max
: A data representável mais tardia, `date(MAXYEAR, 12, 31)`.

date.resolution
: A menor diferença possível entre objetos date não iguais, `timedelta(days=1)`.

Atributos de instância (somente leitura):

date.year
: Entre **`MINYEAR`** e **`MAXYEAR`**, incluindo os extremos.

date.month
: Entre 1 e 12, incluindo os extremos.

date.day
: Entre 1 e o número de dias do mês especificado no ano especificado.

Operações suportadas:

Operação	**Resultado**
`date2 = date1 + timedelta`	`date2` terá `timedelta.days` dias após `date1`. (1)
`date2 = date1 - timedelta`	Calcula `date2` de modo que `date2 + timedelta == date1`. (2)
`timedelta = date1 - date2`	(3)

Operação	Resultado
`date1 == date2` `date1 != date2`	Comparação de igualdade. (4)

`date1 < date2`
`date1 > date2`
`date1 <= date2`
`date1 >= date2`
	Comparação de ordem. (5)

Notas:

1. *date2* é movida para frente no tempo se `timedelta.days > 0`, ou para trás se `timedelta.days < 0`. Posteriormente `date2 - date1 == timedelta.days`. `timedelta.seconds` e `timedelta.microseconds` são ignorados. A exceção `OverflowError` é levantada se `date2.year` for menor que `MINYEAR` ou maior que `MAXYEAR`.

2. `timedelta.seconds` e `timedelta.microseconds` são ignoradas.

3. Isso é exato e não pode estourar. `timedelta.seconds` e `timedelta.microseconds` são 0, e `date2 + timedelta == date1` depois.

4. Objetos `date` são iguais se representam a mesma data.

   Objetos **`date`** que não são também instâncias `datetime` nunca são iguais a objetos **`datetime`**, mesmo que representem a mesma data.

5. *date1* é considerado menor que *date2* quando *date1* precede *date2* no tempo. Em outras palavras, `date1 < date2` se e somente se `date1.toordinal() < date2.toordinal()`.

   Order comparison between a `date` object that is not also a `datetime` instance and a **`datetime`** object raises `TypeError`.

Alterado na versão 3.13: A comparação entre o objeto `datetime` e uma instância da subclasse `date` que não é uma subclasse de **`datetime`** não converte mais a última em **`date`**, ignorando a parte horária e o fuso horário. O comportamento padrão pode ser alterado substituindo os métodos especiais de comparação nas subclasses.

Em contextos booleanos, todo objeto `date` é considerado verdadeiro.

Métodos de instância:

date.replace(*year=self.year*, *month=self.month*, *day=self.day*)

Retorna um novo objeto `date` com os mesmos valores, mas com os parâmetros especificados atualizados.

Exemplo:

```
>>> import datetime as dt
>>> d = dt.date(2002, 12, 31)
>>> d.replace(day=26)
datetime.date(2002, 12, 26)
```

A função genérica `copy.replace()` também oferece suporte a objetos `date`.

date.timetuple()

Retorna uma `time.struct_time` tal como retornado por `time.localtime()`.

As horas, minutos e segundos são 0, e o sinalizador de horário de verão é -1.

`d.timetuple()` é equivalente a:

```
time.struct_time((d.year, d.month, d.day, 0, 0, 0, d.weekday(
```

sendo `yday = d.toordinal() - date(d.year, 1, 1).toordinal() + 1` o número do dia no ano atual, começando com 1 para 1º de janeiro.

date.toordinal()

Retorna o ordinal proléptico gregoriano da data, considerando que 1º de janeiro do ano 1 tem o ordinal 1. Para qualquer objeto **`date`** `d`, `date.fromordinal(d.toordinal()) == d`.

date.weekday()

Retorna o dia da semana como um inteiro, onde Segunda é 0 e Domingo é 6. Por exemplo, `date(2002, 12, 4).weekday() == 2`, uma Quarta-feira. Veja também **`isoweekday()`**.

date.isoweekday()

Retorna o dia da semana como um inteiro, onde Segunda é 1 e Domingo é 7. Por exemplo, `date(2002, 12, 4).isoweekday() == 3`, uma Quarta-feira. Veja também **`weekday()`**, **`isocalendar()`**.

date.isocalendar()

Retorna um objeto tupla nomeada com três componentes: `year`, `week` e `weekday`.

O calendário ISO é uma variação amplamente usada do calendário gregoriano. [[3]]

O ano ISO consiste de 52 ou 53 semanas completas, e onde uma semana começa na segunda-feira e termina no domingo. A primeira semana de um ano ISO é a primeira semana no calendário (Gregoriano) de um ano contendo uma quinta-feira. Isso é chamado de semana número 1, e o ano ISO dessa quinta-feira é o mesmo que o seu ano Gregoriano.

Por exemplo, 2004 começa em uma quinta-feira, então a primeira semana do ano ISO 2004 começa na segunda-feira, 29 de dezembro de 2003, e termina no domingo, 4 de janeiro de 2004:

```
>>> import datetime as dt
>>> dt.date(2003, 12, 29).isocalendar()
datetime.IsoCalendarDate(year=2004, week=1, weekday=1)
>>> dt.date(2004, 1, 4).isocalendar()
datetime.IsoCalendarDate(year=2004, week=1, weekday=7)
```

Alterado na versão 3.9: Resultado alterado de uma tupla para uma tupla nomeada.

date.isoformat()

Retorna uma string representando a data no formato ISO 8601, `YYYY-MM-DD`:

```
>>> import datetime as dt
>>> dt.date(2002, 12, 4).isoformat()
'2002-12-04'
```

date.__str__()

Para um data `d`, `str(d)` é equivalente a `d.isoformat()`.

date.ctime()

Retorna uma string representando a data:

```
>>> import datetime as dt
>>> dt.date(2002, 12, 4).ctime()
'Wed Dec  4 00:00:00 2002'
```

`d.ctime()` é equivalente a:

```
time.ctime(time.mktime(d.timetuple()))
```

em plataformas em que a função C nativa **`ctime()`** (que é invocada pela função **`time.ctime()`**, mas não pelo método **`date.ctime()`**) se conforma com o padrão C.

date.strftime(*format*)

Retorna uma string representando a data, controlado por uma string explícita de formatação. Códigos de formatação referenciando horas, minutos ou segundos irão ver valores 0. Veja também strftime() and strptime() behavior e **`date.isoformat()`**.

date.__format__(*format*)

O mesmo que **`date.strftime()`**. Isso torna possível especificar uma string de formatação para o objeto **`date`** em literais de string formatados

e ao usar **`str.format()`**. Veja também strftime() and strptime() behavior e **`date.isoformat()`**.

Examples of usage: `date`

Exemplo de contagem de dias para um evento:

```
>>> import time
>>> import datetime as dt
>>> today = dt.date.today()
>>> today
datetime.date(2007, 12, 5)
>>> today == dt.date.fromtimestamp(time.time())
True
>>> my_birthday = dt.date(today.year, 6, 24)
>>> if my_birthday < today:
...     my_birthday = my_birthday.replace(year=today.year + 1)
...
>>> my_birthday
datetime.date(2008, 6, 24)
>>> time_to_birthday = abs(my_birthday - today)
>>> time_to_birthday.days
202
```

Mais exemplos de uso da classe **`date`**:

```
>>> import datetime as dt
>>> d = dt.date.fromordinal(730920) # 730920th day after 1. 1. 0
>>> d
datetime.date(2002, 3, 11)

>>> # Methods related to formatting string output
>>> d.isoformat()
'2002-03-11'
>>> d.strftime("%d/%m/%y")
'11/03/02'
>>> d.strftime("%A %d. %B %Y")
'Monday 11. March 2002'
>>> d.ctime()
'Mon Mar 11 00:00:00 2002'
>>> 'The {1} is {0:%d}, the {2} is {0:%B}.'.format(d, "day", "mo
'The day is 11, the month is March.'
```

```
>>> # Methods for extracting 'components' under different calenc
>>> t = d.timetuple()
>>> for i in t:
...     print(i)
2002                # year
3                   # month
11                  # day
0
0
0
0                   # weekday (0 = Monday)
70                  # 70th day in the year
-1
>>> ic = d.isocalendar()
>>> for i in ic:
...     print(i)
2002                # ISO year
11                  # ISO week number
1                   # ISO day number ( 1 = Monday )

>>> # A date object is immutable; all operations produce a new c
>>> d.replace(year=2005)
datetime.date(2005, 3, 11)
```

`datetime` objects

Um objeto **`datetime`** é um único objeto contendo todas as informações de um objeto **`date`** e um objeto **`time`**.

Like a **`date`** object, **`datetime`** assumes the current Gregorian calendar extended in both directions; like a **`time`** object, **`datetime`** assumes there are exactly 3600*24 seconds in every day.

Construtor:

class datetime.datetime(*year*, *month*, *day*, *hour=0*, *minute=0*, *second=0*, *microsecond=0*, *tzinfo=None*, *, *fold=0*)

Os argumentos *year*, *month* e *day* são obrigatórios. *tzinfo* pode ser `None`, ou uma instância de subclasse de **`tzinfo`**. Os argumentos remanescentes devem ser inteiros nos seguintes intervalos:

- `MINYEAR <= year <= MAXYEAR`,
- `1 <= month <= 12`,
- `1 <= day <= número de dias no mês e ano fornecidos`,
- `0 <= hour < 24`,
- `0 <= minute < 60`,
- `0 <= second < 60`,
- `0 <= microsecond < 1000000`,
- `fold in [0, 1]`.

Se um argumento fora desses intervalos for fornecido, a exceção **`ValueError`** é levantada.

Alterado na versão 3.6: Adicionado o parâmetro *fold*.

Outros construtores, todos os métodos de classe:

classmethod datetime.today()

Retorna a data e hora local atual, com o atributo **`tzinfo`** definido para `None`.

Equivalente a:

```
datetime.fromtimestamp(time.time())
```

Veja também **`now()`**, **`fromtimestamp()`**.

Este método é funcionalmente equivalente a **`now()`**, mas sem um parâmetro `tz`.

classmethod datetime.now(*tz=None*)

Retorna a data e hora local atual.

Se o argumento opcional *tz* é `None` ou não especificado, isto é o mesmo que **`today()`**, mas, se possível, fornece mais precisão do que pode ser obtido indo por um registro de data e hora da função **`time.time()`** (por

exemplo, isto pode ser possível em plataformas que fornecem a função C **gettimeofday()**).

Se *tz* não for `None`, ele deve ser uma instância de uma subclasse de **tzinfo**, e a data e hora local atual são convertidas para o fuso horário de *tz*.

Esta função é preferida ao invés de **today()** e **utcnow()**.

Nota

Chamadas subsequentes para **datetime.now()** podem retornar o mesmo instante, dependendo da precisão do relógio subjacente.

classmethod datetime.utcnow()

Retorna a data e hora atual em UTC, com **tzinfo** como `None`.

Este é similar a **now()**, mas retorna a data e hora atual em UTC, como um objeto **datetime** ingênuo. Um datetime UTC consciente pode ser obtido chamando `datetime.now(timezone.utc)`. Veja também **now()**.

Aviso

Devido ao fato de objetos `datetime` ingênuos serem tratados por muitos métodos `datetime` como hora local, é preferível usar datetimes conscientes para representar horas em UTC. De tal forma, a maneira recomendada para criar um objeto representando a hora local em UTC é chamando `datetime.now(timezone.utc)`.

Descontinuado desde a versão 3.12: Use **datetime.now()** com **UTC**.

classmethod datetime.fromtimestamp(*timestamp*, *tz=None*)

Retorna a data e hora local correspondente ao registro de data e hora POSIX, como é retornado por **time.time()**. Se o argumento opcional *tz* é `None` ou não especificado, o registro de data e hora é convertido para a data e hora local da plataforma, e o objeto **datetime** retornado é ingênuo.

Se *tz* não for `None`, ela deve ser uma instância de uma subclasse de `tzinfo`, e o registro de data e hora é convertido para o fuso horário de *tz*.

`fromtimestamp()` pode levantar `OverflowError`, se o registro de data e hora estiver fora do intervalo de valores suportados pelas funções em C **`localtime()`** ou **`gmtime()`** da plataforma, e ocorrer uma falha de `OSError` em **`localtime()`** ou **`gmtime()`**. É comum para isso ser restrito aos anos 1970 até 2038. Perceba que em sistemas não-POSIX que incluem segundos bissextos na sua notação de registro de data e hora, segundos bissextos são ignorados por **`fromtimestamp()`**, e então é possível ter dois registros de data e hora com diferença de um segundo que apresentam objetos `datetime` idênticos. Este método é preferido sobre `utcfromtimestamp()`.

Alterado na versão 3.3: Levanta um `OverflowError` ao invés de `ValueError` se o registro de data e hora estiver fora do intervalo dos valores suportados pelas funções C **`localtime()`** ou **`gmtime()`** da plataforma. Levanta `OSError` ao invés de **`ValueError`** em falhas de **`localtime()`** ou **`gmtime()`**.

Alterado na versão 3.6: `fromtimestamp()` pode retornar instâncias com `fold` igual a 1.

Alterado na versão 3.15: Accepts any real number as *timestamp*, not only integer or float.

classmethod datetime.utcfromtimestamp(*timestamp*)

Retorna o `datetime` UTC correspondente ao registro de data e hora POSIX, com `tzinfo` setado para `None`. (O objeto resultante é ingênuo.)

Isso pode levantar `OverflowError`, se o registro de data e hora estiver fora do intervalo de valores suportados pela função C **`gmtime()`** da plataforma, e em caso de falha `OSError` em **`gmtime()`**. É comum que isso seja restrito a anos de 1970 a 2038.

Para conseguir um objeto `datetime` consciente, chame `fromtimestamp()`:

```
datetime.fromtimestamp(timestamp, timezone.utc)
```

Nas plataformas compatíveis com POSIX, é equivalente à seguinte expressão:

```
datetime(1970, 1, 1, tzinfo=timezone.utc) + timedelta(seconds
```

com a exceção de que a última fórmula sempre dá suporte ao intervalo completo de anos: entre **MINYEAR** e **MAXYEAR** inclusive.

> **Aviso**
>
> Devido ao fato de objetos `datetime` ingênuos serem tratados por muitos métodos `datetime` como hora local, é preferível usar datetimes conscientes para representar horas em UTC. De tal forma, a maneira recomendada para criar um objeto representando um registro de data e hora específico em UTC é chamando `datetime.fromtimestamp(timestamp, tz=timezone.utc)`.

Alterado na versão 3.3: Levanta **OverflowError** ao invés de **ValueError** se o registro de data e hora estiver fora do intervalo de valores suportados pela função C **gmtime()** da plataforma. Levanta **OSError** ao invés de **ValueError** em caso de falha **gmtime()**.

Alterado na versão 3.15: Accepts any real number as *timestamp*, not only integer or float.

Descontinuado desde a versão 3.12: Use **datetime.fromtimestamp()** com **UTC**.

classmethod datetime.fromordinal(*ordinal*)

Retorna um **datetime** correspondente ao ordinal proléptico Gregoriano, onde 1º de janeiro do ano 1 tem ordinal 1. **ValueError** é levantado a não ser que `1 <= ordinal <= datetime.max.toordinal()`. As horas, minutos, segundos e micro segundos do resultado são todos 0, e **tzinfo** é `None`.

classmethod datetime.combine(*date*, *time*, *tzinfo=time.tzinfo*)

Return a new **datetime** object whose date components are equal to the given **date** object's, and whose time components are equal to the given **time** object's. If the *tzinfo* argument is provided, its value is used to set the **tzinfo** attribute of the result, otherwise the **tzinfo** attribute of the *time* argument is used. If the *date* argument is a **datetime** object, its time components and **tzinfo** attributes are ignored.

Para qualquer objeto **datetime** `d`, `d == datetime.combine(d.date(), d.time(), d.tzinfo)`.

Alterado na versão 3.6: Adicionado o argumento *tzinfo*.

classmethod datetime.fromisoformat(*date_string*)

Retorna um **datetime** correspondendo a *date_string* em qualquer formato válido de ISO 8601, com as seguintes exceções:

1. Os deslocamentos de fuso horário podem ter segundos fracionários.
2. O separador `T` pode ser substituído por qualquer caractere unicode único.
3. Horas e minutos fracionários não são suportados.
4. Datas de precisão reduzida não são atualmente suportadas (`YYYY-MM`, `YYYY`).
5. Representações de data estendidas não são atualmente suportadas (`±YYYYYY-MM-DD`).
6. Atualmente, as datas ordinais não são suportadas (`YYYY-OOO`).

Exemplos:

```
>>> import datetime as dt
>>> dt.datetime.fromisoformat('2011-11-04')
datetime.datetime(2011, 11, 4, 0, 0)
>>> dt.datetime.fromisoformat('20111104')
datetime.datetime(2011, 11, 4, 0, 0)
>>> dt.datetime.fromisoformat('2011-11-04T00:05:23')
datetime.datetime(2011, 11, 4, 0, 5, 23)
>>> dt.datetime.fromisoformat('2011-11-04T00:05:23Z')
datetime.datetime(2011, 11, 4, 0, 5, 23, tzinfo=datetime.time
```

```
>>> dt.datetime.fromisoformat('20111104T000523')
datetime.datetime(2011, 11, 4, 0, 5, 23)
>>> dt.datetime.fromisoformat('2011-W01-2T00:05:23.283')
datetime.datetime(2011, 1, 4, 0, 5, 23, 283000)
>>> dt.datetime.fromisoformat('2011-11-04 00:05:23.283')
datetime.datetime(2011, 11, 4, 0, 5, 23, 283000)
>>> dt.datetime.fromisoformat('2011-11-04 00:05:23.283+00:00'
datetime.datetime(2011, 11, 4, 0, 5, 23, 283000, tzinfo=datet
>>> dt.datetime.fromisoformat('2011-11-04T00:05:23+04:00')
datetime.datetime(2011, 11, 4, 0, 5, 23,
    tzinfo=datetime.timezone(datetime.timedelta(seconds=14400
```

Adicionado na versão 3.7.

Alterado na versão 3.11: Anteriormente, este método suportava apenas formatos que podiam ser emitidos por `date.isoformat()` ou `datetime.isoformat()`.

classmethod datetime.fromisocalendar(*year*, *week*, *day*)

Return a `datetime` corresponding to the ISO calendar date specified by *year*, *week* and *day*. The non-date components of the datetime are populated with their normal default values. This is the inverse of the function `datetime.isocalendar()`.

Adicionado na versão 3.8.

classmethod datetime.strptime(*date_string*, *format*)

Retorna um `datetime` correspondente ao *date_string*, analisado de acordo com *format*.

Se *format* não contiver microssegundos ou informações de fuso horário, isso é equivalente a:

```
datetime(*(time.strptime(date_string, format)[0:6]))
```

`ValueError` é levantado se o date_string e o format não puderem ser interpretados por `time.strptime()` ou se ele retorna um valor o qual não é uma tupla temporal. Veja também strftime() and strptime() behavior e `datetime.fromisoformat()`.

Alterado na versão 3.15: If *format* specifies a day of month (`%d`) without a year, **`ValueError`** is raised. This is to avoid a quadrennial leap year bug in code seeking to parse only a month and day as the default year used in absence of one in the format is not a leap year. The workaround is to always include a year in your *format*. If parsing *date_string* values that do not have a year, explicitly add a year that is a leap year before parsing:

```
>>> import datetime as dt
>>> date_string = "02/29"
>>> when = dt.datetime.strptime(f"{date_string};1984", "%m/%
>>> when.strftime("%B %d")
'February 29'
```

Atributos de classe:

datetime.min

O primeiro **`datetime`** representável, `datetime(MINYEAR, 1, 1, tzinfo=None)`.

datetime.max

O último **`datetime`** representável, `datetime(MAXYEAR, 12, 31, 23, 59, 59, 999999, tzinfo=None)`.

datetime.resolution

A menor diferença possível entre objetos **`datetime`** diferentes, `timedelta(microseconds=1)`.

Atributos de instância (somente leitura):

datetime.year

Entre **`MINYEAR`** e **`MAXYEAR`**, incluindo os extremos.

datetime.month

Entre 1 e 12, incluindo os extremos.

datetime.day

Entre 1 e o número de dias do mês especificado no ano especificado.

datetime.hour

No intervalo `range(24)`.

datetime.minute

No intervalo `range(60)`.

datetime.second

No intervalo `range(60)`.

datetime.microsecond

No intervalo `range(1000000)`.

datetime.tzinfo

O objeto passado como o argumento *tzinfo* do construtor **`datetime`**, ou `None` se nada foi passado.

datetime.fold

Entre `[0, 1]`. Usado para desambiguar tempos reais durante um intervalo repetido. (Um intervalo repetido ocorre quando relógios são atrasados ao final do horário de verão ou quando a diferença UTC para o fuso horário atual é reduzida por razões políticas.) Os valores 0 e 1 representam, respectivamente, o primeiro e o segundo dos dois momentos com a mesma representação de tempo real.

Adicionado na versão 3.6.

Operações suportadas:

Operação	**Resultado**
`datetime2 = datetime1 + timedelta`	(1)

Operação	Resultado
`datetime2 = datetime1 - timedelta`	(2)
`timedelta = datetime1 - datetime2`	(3)
`datetime1 == datetime2` `datetime1 != datetime2`	Comparação de igualdade. (4)

`datetime1 < datetime2`
`datetime1 > datetime2`
`datetime1 <= datetime2`
`datetime1 >= datetime2`
	Comparação de ordem. (5)

1. `datetime2` representa a duração de `timedelta` removido de `datetime1`, movendo o tempo para frente se `timedelta.days > 0`, ou para trás se `timedelta.days < 0`. O resultado tem o mesmo atributo **`tzinfo`** que o datetime de entrada, e `datetime2 - datetime1 == timedelta` após. **`OverflowError`** é levantada se `datetime2.year` fosse menor que **`MINYEAR`** ou maior que **`MAXYEAR`**. Perceba que nenhum ajuste no fuso horário é feito mesmo se a entrada é um objeto consciente disso.

2. Computa o `datetime2` tal que `datetime2 + timedelta == datetime1`. Assim como para adição, o resultado tem o mesmo atributo **`tzinfo`** que datetime de entrada, e nenhum ajuste de fuso horário é feito mesmo que a entrada seja consciente disso.

3. Subtraction of a **`datetime`** from a **`datetime`** is defined only if both operands are naive, or if both are aware. If one is aware and the other is naive, **`TypeError`** is raised.

Se ambos são ingênuos, ou ambos são conscientes e tiverem o mesmo atributo `tzinfo`, os atributos **`tzinfo`** são ignorados, e o resultado é um objeto `t` do tipo `timedelta`, tal que `datetime2 + t == datetime1`. Nenhum ajuste de fuso horário é feito neste caso.

Se ambas são conscientes e têm atributos `tzinfo` diferentes, `a-b` age como se `a` e `b` tivessem sido primeiro convertidos para datetimes ingênuas em UTC. O resultado é `(a.replace(tzinfo=None) - a.utcoffset()) - (b.replace(tzinfo=None) - b.utcoffset())` exceto que a implementação nunca estoura.

4. Objetos `datetime` são iguais se representarem a mesma data e hora, levando em consideração o fuso horário.

   Naive and aware `datetime` objects are never equal.

   If both comparands are aware, and have the same **`tzinfo`** attribute, the **`tzinfo`** and `fold` attributes are ignored and the base datetimes are compared. If both comparands are aware and have different `tzinfo` attributes, the comparison acts as comparands were first converted to UTC datetimes except that the implementation never overflows. `datetime` instances in a repeated interval are never equal to **`datetime`** instances in other time zone.

5. *datetime1* é considerado menor que *datetime2* quando *datetime1* precede *datetime2* no tempo. levando em consideração o fuso horário.

   A comparação de ordens entre objetos `datetime` ingênuos e conscientes levanta `TypeError`.

   Se ambos os comparandos forem conscientes e tiverem o mesmo atributo **`tzinfo`**, os atributos **`tzinfo`** e `fold` serão ignorados e os datetimes base serão comparados. Se ambos os comparandos forem conscientes e tiverem atributos `tzinfo` diferentes, a comparação atua como se os comparandos fossem primeiro convertidos para datetimes UTC, exceto que a implementação nunca estoura.

Alterado na versão 3.3: Comparações de igualdade entre instâncias de `datetime` conscientes e nativas não levantam `TypeError`.

Alterado na versão 3.13: A comparação entre o objeto `datetime` e uma instância da subclasse `date` que não é uma subclasse de **`datetime`** não converte mais a última em **`date`**, ignorando a parte horária e o fuso horário. O comportamento padrão pode ser alterado substituindo os métodos especiais de comparação nas subclasses.

Métodos de instância:

datetime.date()

Retorna um objeto `date` com o mesmo ano, mês e dia.

datetime.time()

Retorna um objeto `time` com a mesma hora, minuto, segundo, microssegundo e fold. O atributo `tzinfo` é `None`. Veja também o método `timetz()`.

Alterado na versão 3.6: O valor fold é copiado para o objeto `time` retornado.

datetime.timetz()

Retorna um objeto `time` com os mesmos atributos de hora, minuto, segundo, microssegundo, fold e tzinfo. Veja também o método `time()`.

Alterado na versão 3.6: O valor fold é copiado para o objeto `time` retornado.

datetime.replace(*year=self.year*, *month=self.month*, *day=self.day*, *hour=self.hour*, *minute=self.minute*, *second=self.second*, *microsecond=self.microsecond*, *tzinfo=self.tzinfo*, *, *fold=0*)

Retorna um novo objeto `datetime` com os mesmos atributos, mas com os parâmetros especificados atualizados. Perceba que `tzinfo=None` pode ser especificado para criar um datetime ingênuo a partir de um datetime consciente, sem conversão de dados da data ou hora.

Objetos **datetime** também são suportados pela função genérica **copy.replace()**.

Alterado na versão 3.6: Adicionado o parâmetro *fold*.

datetime.astimezone(*tz=None*)

Retorna um objeto **datetime** com novo atributo **tzinfo** definido por *tz*, ajustando a data e hora de forma que o resultado seja o mesmo horário UTC que *self*, mas na hora local de *tz*.

Se fornecido, *tz* deve ser uma instância de uma subclasse **tzinfo**, e seus métodos **utcoffset()** e **dst()** não devem retornar `None`. Se *self* for ingênuo, é presumido que ele representa o tempo no fuso horário do sistema.

Se for chamado sem argumentos (ou com `tz=None`) o fuso horário do sistema local é presumido como o fuso horário desejado. O atributo `.tzinfo` da instância datetime convertida será definido para uma instância de **timezone** com o nome da zona e um deslocamento obtido a partir do sistema operacional.

Se `self.tzinfo` for *tz*, `self.astimezone(tz)` é igual a *self*: nenhum ajuste nos dados de data ou hora é realizado. Caso contrário o resultado é a hora local no fuso horário *tz*, representando a mesma hora UTC que *self*: depois `astz = dt.astimezone(tz)`, `astz - astz.utcoffset()` terá os mesmos dados de data e hora que `dt - dt.utcoffset()`.

Se você quer meramente anexar um objeto **timezone** *tz* a um datetime *dt* sem ajustes de dados de data e hora, use `dt.replace(tzinfo=tz)`. Se você meramente quer remover o objeto **timezone** de um datetime consciente *dt* sem conversão de dados de data e hora, use `dt.replace(tzinfo=None)`.

Perceba que o método padrão **tzinfo.fromutc()** pode ser substituído em uma subclasse **tzinfo** para afetar o resultado retornado por **astimezone()**. Ignorando erros de letras maiúsculas/minúsculas, **astimezone()** funciona como:

```
def astimezone(self, tz):
    if self.tzinfo is tz:
        return self
    # Converte self para UTC e anexa o novo objeto timezone.
    utc = (self - self.utcoffset()).replace(tzinfo=tz)
    # Converte o horário local de UTC para tz.
    return tz.fromutc(utc)
```

Alterado na versão 3.3: tz agora pode ser omitido.

Alterado na versão 3.6: O método **`astimezone()`** agora pode ser chamado em instâncias ingênuas que presumidamente representam a hora local do sistema.

datetime.utcoffset()

Se **`tzinfo`** for `None`, retorna `None`, caso contrário retorna `self.tzinfo.utcoffset(self)`, e levanta uma exceção se o segundo não retornar `None` ou um objeto **`timedelta`** com magnitude menor que um dia.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

datetime.dst()

Se **`tzinfo`** for `None`, retorna `None`, caso contrário retorna `self.tzinfo.dst(self)`, e levanta uma exceção se o segundo não retornar `None` ou um objeto **`timedelta`** com magnitude menor que um dia.

Alterado na versão 3.7: A diferença de horário de verão não é restrita a um número completo de minutos.

datetime.tzname()

Se **`tzinfo`** for `None`, retorna `None`, caso contrário retorna `self.tzinfo.tzname(self)`, levanta uma exceção se o segundo não retornar `None` ou um objeto string.

datetime.timetuple()

Retorna uma `time.struct_time` tal como retornado por `time.localtime()`.

`d.timetuple()` é equivalente a:

```
time.struct_time((d.year, d.month, d.day,
                  d.hour, d.minute, d.second,
                  d.weekday(), yday, dst))
```

onde `yday = d.toordinal() - date(d.year, 1, 1).toordinal() + 1` é o número de dias dentro do ano atual, começando com 1 para 1º de janeiro. O sinalizador `tm_isdst` do resultado é definido conforme o método `dst()`: `tzinfo` é `None` ou **`dst()`** retorna `None`, **`tm_isdst`** é definido para `-1`; caso contrário se **`dst()`** retornar um valor diferente de zero, **`tm_isdst`** é definido para 1; caso contrário **`tm_isdst`** é definido para 0.

datetime.utctimetuple()

Se a instância `datetime` `d` é ingênua, isto é o mesmo que `d.timetuple()`, exceto que `tm_isdst` é forçado para 0, independentemente do que `d.dst()` retornar. O horário de verão nunca é afetado por um horário UTC.

Se `d` é consciente, `d` é normalizado para horário UTC, subtraindo `d.utcoffset()`, e um `time.struct_time` para o horário normalizado é retornado. **`tm_isdst`** é forçado para 0. Observe que uma exceção `OverflowError` pode ser levantada se `d.year` for `MINYEAR` ou `MAXYEAR` e os ajustes UTC ultrapassar o limite de um ano.

Aviso

Por causa que objetos `datetime` ingênuos são tratados por muitos métodos `datetime` como hora local, é preferido usar datetimes conscientes para representar horários em UTC; como resultado, usar `datetime.utctimetuple()` pode dar resultados enganosos. Se você tem um `datetime` ingênuo representando UTC, use `datetime.replace(tzinfo=timezone.utc)` para torná-lo consciente, ponto no qual você pode usar `datetime.timetuple()`.

datetime.toordinal()

Retorna o ordinal proléptico gregoriano da data. o mesmo que `self.date().toordinal()`.

datetime.timestamp()

Retorna o registro de data e hora POSIX correspondente a instância **`datetime`**. O valor de retorno é um **`float`** similar aquele retornado por **`time.time()`**.

Naive **`datetime`** instances are assumed to represent local time and this method relies on platform C functions to perform the conversion. Since **`datetime`** supports a wider range of values than the platform C functions on many platforms, this method may raise **`OverflowError`** or **`OSError`** for times far in the past or far in the future.

Para instâncias conscientes de **`datetime`**, o valor retornado é computado como:

```
(dt - datetime(1970, 1, 1, tzinfo=timezone.utc)).total_second
```

Nota

Não existe método para obter o timestamp POSIX diretamente de uma instância **`datetime`** ingênua representando tempo em UTC. Se a sua aplicação usa esta convenção e o fuso horário do seu sistema não está setado para UTC, você pode obter o registro de data e hora POSIX fornecendo `tzinfo=timezone.utc`:

```
timestamp = dt.replace(tzinfo=timezone.utc).timestamp()
```

ou calculando o registro de data e hora diretamente:

```
timestamp = (dt - datetime(1970, 1, 1)) / timedelta(seconds
```

Adicionado na versão 3.3.

Alterado na versão 3.6: O método **timestamp()** usa o atributo **fold** para desambiguar os tempos durante um intervalo repetido.

Alterado na versão 3.6: This method no longer relies on the platform C **mktime()** function to perform conversions.

datetime.weekday()

Retorna o dia da semana como um inteiro, em que segunda-feira é 0 e domingo é 6. O mesmo que `self.date().weekday()`. Veja também **isoweekday()**.

datetime.isoweekday()

Retorna o dia da semana como um inteiro, em que segunda-feira é 1 e domingo é 7. O mesmo que `self.date().isoweekday()`. Veja também **weekday()**, **isocalendar()**.

datetime.isocalendar()

Retorna uma tupla nomeada com três componentes: `year`, `week` e `weekday`. O mesmo que `self.date().isocalendar()`.

datetime.isoformat(*sep='T', timespec='auto'*)

Retorna uma string representando a data e o tempo no formato ISO 8601:

- `YYYY-MM-DDTHH:MM:SS.ffffff`, se **microsecond** não é 0
- `YYYY-MM-DDTHH:MM:SS`, se **microsecond** é 0

Se **utcoffset()** não retorna `None`, uma string é adicionada com a diferença UTC:

- `YYYY-MM-DDTHH:MM:SS.ffffff+HH:MM[:SS[.ffffff]]`, se **microsecond** não é 0
- `YYYY-MM-DDTHH:MM:SS+HH:MM[:SS[.ffffff]]`, se **microsecond** é 0

Exemplos:

```
>>> import datetime as dt
>>> dt.datetime(2019, 5, 18, 15, 17, 8, 132263).isoformat()
```

```
'2019-05-18T15:17:08.132263'
>>> dt.datetime(2019, 5, 18, 15, 17, tzinfo=dt.timezone.utc).
'2019-05-18T15:17:00+00:00'
```

O argumento opcional *sep* (por padrão, `'T'`) é um separador de caractere único, colocado entre as porções de data e tempo do resultado. Por exemplo:

```
>>> import datetime as dt
>>> class TZ(dt.tzinfo):
...     """A time zone with an arbitrary, constant -06:39 off
...     def utcoffset(self, when):
...         return dt.timedelta(hours=-6, minutes=-39)
...
>>> dt.datetime(2002, 12, 25, tzinfo=TZ()).isoformat(' ')
'2002-12-25 00:00:00-06:39'
>>> dt.datetime(2009, 11, 27, microsecond=100, tzinfo=TZ()).i
'2009-11-27T00:00:00.000100-06:39'
```

O argumento opcional *timespec* especifica o número de componentes adicionais do tempo a incluir (o padrão é `'auto'`). Pode ser uma das seguintes strings:

- `'auto'`: O mesmo que `'seconds'` se **`microsecond`** é 0, o mesmo que `'microseconds'` caso contrário.
- `'hours'`: Inclui o atributo **`hour`** no formato de dois dígitos `HH`.
- `'minutes'`: Inclui os atributos **`hour`** e **`minute`** no formato `HH:MM`.
- `'seconds'`: Inclui os atributos **`hour`**, **`minute`** e **`second`** no formato `HH:MM:SS`.
- `'milliseconds'`: Inclui o tempo completo, mas trunca a parte fracional dos segundos em milissegundos. Formato `HH:MM:SS.sss`.
- `'microseconds'`: Inclui o tempo completo no formato `HH:MM:SS.ffffff`.

Nota

Componentes do tempo excluídos são truncados, não arredondados.

A exceção **ValueError** vai ser levantada no caso de um argumento *timespec* inválido:

```
>>> import datetime as dt
>>> dt.datetime.now().isoformat(timespec='minutes')
'2002-12-25T00:00'
>>> my_datetime = dt.datetime(2015, 1, 1, 12, 30, 59, 0)
>>> my_datetime.isoformat(timespec='microseconds')
'2015-01-01T12:30:59.000000'
```

Alterado na versão 3.6: Adicionado o parâmetro *timespec*.

datetime.__str__()

Para uma instância **datetime** `d`, `str(d)` é equivalente a `d.isoformat(' ')`.

datetime.ctime()

Retorna uma string representando a data e hora:

```
>>> import datetime as dt
>>> dt.datetime(2002, 12, 4, 20, 30, 40).ctime()
'Wed Dec  4 20:30:40 2002'
```

A string de saída *não* irá incluir informações de fuso horário, independente de a entrada ser consciente ou ingênua.

`d.ctime()` é equivalente a:

```
time.ctime(time.mktime(d.timetuple()))
```

em plataformas onde a função nativa em C **ctime()** (a qual **time.ctime()** invoca, mas a qual **datetime.ctime()** não invoca) conforma com o padrão C.

datetime.strftime(*format*)

Retorna uma string representando a data e hora, controladas por uma string com formato explícito. Veja também strftime() and strptime() behavior e **datetime.isoformat()**.

datetime.__format__(*format*)

O mesmo que **`datetime.strftime()`**. Isso torna possível especificar uma string de formatação para o objeto **`datetime`** em literais de string formatados e ao usar **`str.format()`**. Veja também strftime() and strptime() behavior e **`datetime.isoformat()`**.

Examples of usage: `datetime`

Exemplos para trabalhar com objetos **`datetime`**:

```
>>> import datetime as dt

>>> # Using datetime.combine()
>>> d = dt.date(2005, 7, 14)
>>> t = dt.time(12, 30)
>>> dt.datetime.combine(d, t)
datetime.datetime(2005, 7, 14, 12, 30)

>>> # Using datetime.now()
>>> dt.datetime.now()
datetime.datetime(2007, 12, 6, 16, 29, 43, 79043)   # GMT +1
>>> dt.datetime.now(dt.timezone.utc)
datetime.datetime(2007, 12, 6, 15, 29, 43, 79060, tzinfo=datetim

>>> # Using datetime.strptime()
>>> my_datetime = dt.datetime.strptime("21/11/06 16:30", "%d/%m/
>>> my_datetime
datetime.datetime(2006, 11, 21, 16, 30)

>>> # Using datetime.timetuple() to get tuple of all attributes
>>> tt = my_datetime.timetuple()
>>> for it in tt:
...     print(it)
...
2006    # year
11      # month
21      # day
16      # hour
30      # minute
0       # second
1       # weekday (0 = Monday)
```

```
325     # number of days since 1st January
-1      # dst - method tzinfo.dst() returned None

>>> # Date in ISO format
>>> ic = my_datetime.isocalendar()
>>> for it in ic:
...     print(it)
...
2006    # ISO year
47      # ISO week
2       # ISO weekday

>>> # Formatting a datetime
>>> my_datetime.strftime("%A, %d. %B %Y %I:%M%p")
'Tuesday, 21. November 2006 04:30PM'
>>> 'The {1} is {0:%d}, the {2} is {0:%B}, the {3} is {0:%I:%M%p
'The day is 21, the month is November, the time is 04:30PM.'
```

O exemplo abaixo define uma subclasse **tzinfo** capturando informações de fuso horário para Kabul, Afeganistão, o qual usou +4 UTC até 1945 e depois +4:30 UTC após esse período:

```
import datetime as dt

class KabulTz(dt.tzinfo):
    # Kabul used +4 until 1945, when they moved to +4:30
    UTC_MOVE_DATE = dt.datetime(1944, 12, 31, 20, tzinfo=dt.time

    def utcoffset(self, when):
        if when.year < 1945:
            return dt.timedelta(hours=4)
        elif (1945, 1, 1, 0, 0) <= when.timetuple()[:5] < (1945,
            # An ambiguous ("imaginary") half-hour range represe
            # a 'fold' in time due to the shift from +4 to +4:30
            # If when falls in the imaginary range, use fold to
            # to resolve. See PEP 495.
            return dt.timedelta(hours=4, minutes=(30 if when.fol
        else:
            return dt.timedelta(hours=4, minutes=30)

    def fromutc(self, when):
        # Follow same validations as in datetime.tzinfo
```

```
        if not isinstance(when, dt.datetime):
            raise TypeError("fromutc() requires a datetime argum
        if when.tzinfo is not self:
            raise ValueError("when.tzinfo is not self")

        # A custom implementation is required for fromutc as
        # the input to this function is a datetime with utc valu
        # but with a tzinfo set to self.
        # See datetime.astimezone or fromtimestamp.
        if when.replace(tzinfo=dt.timezone.utc) >= self.UTC_MOVE
            return when + dt.timedelta(hours=4, minutes=30)
        else:
            return when + dt.timedelta(hours=4)

    def dst(self, when):
        # Kabul does not observe daylight saving time.
        return dt.timedelta(0)

    def tzname(self, when):
        if when >= self.UTC_MOVE_DATE:
            return "+04:30"
        return "+04"
```

Uso de `KabulTz` mostrado acima:

```
>>> tz1 = KabulTz()

>>> # Datetime before the change
>>> dt1 = dt.datetime(1900, 11, 21, 16, 30, tzinfo=tz1)
>>> print(dt1.utcoffset())
4:00:00

>>> # Datetime after the change
>>> dt2 = dt.datetime(2006, 6, 14, 13, 0, tzinfo=tz1)
>>> print(dt2.utcoffset())
4:30:00

>>> # Convert datetime to another time zone
>>> dt3 = dt2.astimezone(dt.timezone.utc)
>>> dt3
datetime.datetime(2006, 6, 14, 8, 30, tzinfo=datetime.timezone.u
>>> dt2
```

```
datetime.datetime(2006, 6, 14, 13, 0, tzinfo=KabulTz())
>>> dt2 == dt3
True
```

Objetos `time`

Um objeto `time` representa a hora (local) do dia, independentemente de um dia em particular, e sujeito a ajustes através de um objeto `tzinfo`.

class datetime.time(*hour=0*, *minute=0*, *second=0*, *microsecond=0*, *tzinfo=None*, *, *fold=0*)

Todos os argumentos são opcionais. *tzinfo* pode ser `None`, ou uma instância de uma subclasse de `tzinfo`. Os argumentos remanescentes devem ser inteiros nos seguintes intervalos:

- `0 <= hour < 24`,
- `0 <= minute < 60`,
- `0 <= second < 60`,
- `0 <= microsecond < 1000000`,
- `fold in [0, 1]`.

Se um argumento fora desses intervalos for fornecido, `ValueError` é levantada. Todos têm o valor 0 como padrão, exceto *tzinfo*, que tem o valor padrão `None`.

Atributos de classe:

time.min

O `time` mais cedo que pode ser representado, `time(0, 0, 0, 0)`.

time.max

O `time` mais tardio que pode ser representado, `time(23, 59, 59, 999999)`.

time.resolution

A menor diferença possível entre objetos `time` diferentes, `timedelta(microseconds=1)`, embora perceba que aritmética sobre

objetos **`time`** não é suportada.

Atributos de instância (somente leitura):

time.hour
: No intervalo `range(24)`.

time.minute
: No intervalo `range(60)`.

time.second
: No intervalo `range(60)`.

time.microsecond
: No intervalo `range(1000000)`.

time.tzinfo
: O objeto passado como argumento tzinfo para o construtor da classe **`time`**, ou `None` se nada foi passado.

time.fold
: Entre `[0, 1]`. Usado para desambiguar tempos reais durante um intervalo repetido. (Um intervalo repetido ocorre quando relógios são atrasados ao final do horário de verão ou quando a diferença UTC para o fuso horário atual é reduzida por razões políticas.) Os valores 0 e 1 representam, respectivamente, o primeiro e o segundo dos dois momentos com a mesma representação de tempo real.

  Adicionado na versão 3.6.

Objetos **`time`** oferecem suporte para comparações de igualdade e ordem, onde `a` é considerado menor que `b` quando `a` precede `b` no tempo.

Objetos **`time`** ingênuos e conscientes nunca são iguais. A comparação de ordem entre objetos **`time`** ingênuos e conscientes levanta **`TypeError`**.

Se ambos os comparandos são conscientes, e tem o mesmo atributo **`tzinfo`**, os atributos **`tzinfo`** e **`fold`** são ignorados e os tempos base são comparados.

Se ambos os comparandos são conscientes e tem atributos **`tzinfo`** diferentes, os comparandos são primeiro ajustados subtraindo sua diferença em UTC (obtida através de `self.utcoffset()`).

Alterado na versão 3.3: Comparações de igualdade entre instâncias de **`time`** conscientes e nativas não levantam **`TypeError`**.

Em contextos Booleanos, um objeto **`time`** é sempre considerado como verdadeiro.

Alterado na versão 3.5: Before Python 3.5, a **`time`** object was considered to be false if it represented midnight in UTC. This behavior was considered obscure and error-prone and has been removed in Python 3.5. See bpo-13936 [https://bugs.python.org/issue?@action=redirect&bpo=13936] for more information.

Outros construtores:

classmethod time.fromisoformat(*time_string*)

Retorna um **`time`** correspondendo a *time_string* em qualquer formato válido de ISO 8601, com as seguintes exceções:

1. Os deslocamentos de fuso horário podem ter segundos fracionários.
2. O `T` inicial, normalmente exigido nos casos em que pode haver ambiguidade entre uma data e uma hora, não é necessário.
3. Segundos fracionários podem ter qualquer número de dígitos (algo além de 6 será truncado).
4. Horas e minutos fracionários não são suportados.

Exemplos:

```
>>> import datetime as dt
>>> dt.time.fromisoformat('04:23:01')
datetime.time(4, 23, 1)
>>> dt.time.fromisoformat('T04:23:01')
datetime.time(4, 23, 1)
>>> dt.time.fromisoformat('T042301')
datetime.time(4, 23, 1)
>>> dt.time.fromisoformat('04:23:01.000384')
datetime.time(4, 23, 1, 384)
```

```
>>> dt.time.fromisoformat('04:23:01,000384')
datetime.time(4, 23, 1, 384)
>>> dt.time.fromisoformat('04:23:01+04:00')
datetime.time(4, 23, 1, tzinfo=datetime.timezone(datetime.tin
>>> dt.time.fromisoformat('04:23:01Z')
datetime.time(4, 23, 1, tzinfo=datetime.timezone.utc)
>>> dt.time.fromisoformat('04:23:01+00:00')
datetime.time(4, 23, 1, tzinfo=datetime.timezone.utc)
```

Adicionado na versão 3.7.

Alterado na versão 3.11: Anteriormente, este método suportava apenas formatos que podiam ser emitidos por **`time.isoformat()`**.

classmethod time.strptime(*date_string*, *format*)

Retorna um **`time`** correspondente ao *date_string*, analisado de acordo com *format*.

Se *format* não contiver microssegundos ou informações de fuso horário, isso é equivalente a:

```
time(*(time.strptime(date_string, format)[3:6]))
```

`ValueError` é levantado se o *date_string* e o *format* não puderem ser interpretados por **`time.strptime()`** ou se ele retorna um valor o qual não é uma tupla temporal. Veja também strftime() and strptime() behavior e **`time.fromisoformat()`**.

Adicionado na versão 3.14.

Métodos de instância:

time.replace(*hour=self.hour*, *minute=self.minute*, *second=self.second*, *microsecond=self.microsecond*, *tzinfo=self.tzinfo*, *, *fold=0*)

Return a new **`time`** with the same values, but with specified parameters updated. Note that `tzinfo=None` can be specified to create a naive **`time`** from an aware **`time`**, without conversion of the time data.

Objetos `time` também são suportados pela função genérica `copy.replace()`.

Alterado na versão 3.6: Adicionado o parâmetro *fold*.

time.isoformat(*timespec='auto'*)

Retorna uma string representando a hora em formato ISO 8601, um destes:

- `HH:MM:SS.ffffff`, se `microsecond` não é 0
- `HH:MM:SS`, se `microsecond` é 0
- `HH:MM:SS.ffffff+HH:MM[:SS[.ffffff]]`, se `utcoffset()` não retorna `None`
- `HH:MM:SS+HH:MM[:SS[.ffffff]]`, se `microsecond` é 0 e `utcoffset()` não retorna `None`

O argumento opcional *timespec* especifica o número de componentes adicionais do tempo a incluir (o padrão é `'auto'`). Pode ser uma das seguintes strings:

- `'auto'`: O mesmo que `'seconds'` se `microsecond` é 0, o mesmo que `'microseconds'` caso contrário.
- `'hours'`: Inclui o atributo `hour` no formato de dois dígitos `HH`.
- `'minutes'`: Inclui os atributos `hour` e `minute` no formato `HH:MM`.
- `'seconds'`: Inclui os atributos `hour`, `minute` e `second` no formato `HH:MM:SS`.
- `'milliseconds'`: Inclui o tempo completo, mas trunca a parte fracional dos segundos em milissegundos. Formato `HH:MM:SS.sss`.
- `'microseconds'`: Inclui o tempo completo no formato `HH:MM:SS.ffffff`.

Nota

Componentes do tempo excluídos são truncados, não arredondados.

`ValueError` será levantado com um argumento *timespec* inválido.

Exemplo:

```
>>> import datetime as dt
>>> dt.time(hour=12, minute=34, second=56, microsecond=123456
'12:34'
>>> my_time = dt.time(hour=12, minute=34, second=56, microsec
>>> my_time.isoformat(timespec='microseconds')
'12:34:56.000000'
>>> my_time.isoformat(timespec='auto')
'12:34:56'
```

Alterado na versão 3.6: Adicionado o parâmetro *timespec*.

time.__str__()

Para um tempo `t`, `str(t)` é equivalente a `t.isoformat()`.

time.strftime(*format*)

Retorna uma string representando a hora, controladas por uma string com formato explícito. Veja também strftime() and strptime() behavior e **`time.isoformat()`**.

time.__format__(*format*)

O mesmo que **`time.strftime()`**. Isso torna possível especificar uma string de formatação para o objeto **`time`** em literais de string formatados e ao usar **`str.format()`**. Veja também strftime() and strptime() behavior e **`time.isoformat()`**.

time.utcoffset()

Se **`tzinfo`** for `None`, retorna `None`, caso contrário retorna `self.tzinfo.utcoffset(None)`, e levanta uma exceção se o segundo não retornar `None` ou um objeto **`timedelta`** com magnitude menor que um dia.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

time.dst()

Se **`tzinfo`** for `None`, retorna `None`, caso contrário retorna `self.tzinfo.dst(None)`, e levanta uma exceção se o segundo não

retornar `None`, ou um objeto **`timedelta`** com magnitude menor que um dia.

Alterado na versão 3.7: A diferença de horário de verão não é restrita a um número completo de minutos.

time.tzname()

Se **`tzinfo`** for `None`, retorna `None`, caso contrário retorna `self.tzinfo.tzname(None)`, ou levanta uma exceção se o último caso não retornar `None` ou um objeto string.

Examples of usage: `time`

Exemplos para trabalhar com um objeto **`time`**:

```
>>> import datetime as dt
>>> class TZ1(dt.tzinfo):
...     def utcoffset(self, when):
...         return dt.timedelta(hours=1)
...     def dst(self, when):
...         return dt.timedelta(0)
...     def tzname(self, when):
...         return "+01:00"
...     def  __repr__(self):
...         return f"{self.__class__.__name__}()"
...
>>> t = dt.time(12, 10, 30, tzinfo=TZ1())
>>> t
datetime.time(12, 10, 30, tzinfo=TZ1())
>>> t.isoformat()
'12:10:30+01:00'
>>> t.dst()
datetime.timedelta(0)
>>> t.tzname()
'+01:00'
>>> t.strftime("%H:%M:%S %Z")
'12:10:30 +01:00'
>>> 'The {} is {:%H:%M}.'.format("time", t)
'The time is 12:10.'
```

`tzinfo` objects

class datetime.tzinfo

This is an abstract base class, meaning that this class should not be instantiated directly. Define a subclass of `tzinfo` to capture information about a particular time zone.

An instance of (a concrete subclass of) `tzinfo` can be passed to the constructors for `datetime` and `time` objects. The latter objects view their attributes as being in local time, and the `tzinfo` object supports methods revealing offset of local time from UTC, the name of the time zone, and DST offset, all relative to a date or time object passed to them.

You need to derive a concrete subclass, and (at least) supply implementations of the standard `tzinfo` methods needed by the `datetime` methods you use. The `datetime` module provides `timezone`, a simple concrete subclass of `tzinfo` which can represent time zones with fixed offset from UTC such as UTC itself or North American EST and EDT.

Requisito especial para preservação: uma subclasse `tzinfo` deve ter um método `__init__()` que pode ser chamado sem nenhum argumento, caso contrário ele pode ser conservado, mas não alterado novamente. Isso é um requisito técnico que pode ser relaxado no futuro.

Uma subclasse concreta de `tzinfo` pode precisar implementar os seguintes métodos. Exatamente quais métodos são necessários depende do uso feito de objetos `datetime` conscientes. Se estiver em dúvida, simplesmente implemente todos eles.

tzinfo.utcoffset(*dt*)

Retorna a diferença da hora local a partir do UTC, como um objeto `timedelta`, que é positivo a leste do UTC. Se a hora local está a oeste do UTC, isto deve ser negativo.

Isto representa a diferença *total* a partir de UTC; por exemplo, se um objeto `tzinfo` representa fuso horário e ajustes de horário de verão, `utcoffset()` deve retornar a soma deles. Se a diferença UTC não é conhecida, retorna `None`. Caso contrário o valor retornado deve ser um objeto `timedelta` estritamente contido entre `-timedelta(hours=24)` e `timedelta(hours=24)` (a magnitude da diferença deve ser menor que um dia). A maior parte das implementações de **`utcoffset()`** irá provavelmente parecer com um destes dois:

```
return CONSTANT                 # classe de posição fixa
return CONSTANT + self.dst(dt)  # classe consciente de horari
```

Se `utcoffset()` não retorna `None`, `dst()` também não deve retornar `None`.

A implementação padrão de `utcoffset()` levanta `NotImplementedError`.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

tzinfo.dst(*dt*)

Retorna o ajuste para o horário de verão (DST - daylight saving time), como um objeto `timedelta` ou `None` se informação para o horário de verão é desconhecida.

Retorna `timedelta(0)` se o horário de verão não estiver ativo. Se o horário de verão estiver ativo, retorna a diferença como um objeto `timedelta` (veja `utcoffset()` para detalhes). Perceba que a diferença do horário de verão, se aplicável, já foi adicionada a diferença UTC retornada por **`utcoffset()`**, então não existe necessidade de consultar `dst()` a não ser que você esteja interessado em obter a informação de horário de verão separadamente. Por exemplo, `datetime.timetuple()` chama o método **`dst()`** do seu atributo `tzinfo` para determinar como o flag `tm_isdst` deve ser definido, e `tzinfo.fromutc()` chama **`dst()`** para contabilizar as mudanças de horário de verão quando ocorrem mudanças de fuso horário.

Uma instância *tz* de uma subclasse **tzinfo** que modela tanto horário padrão quanto horário de verão deve ser consistente neste sentido:

```
tz.utcoffset(dt) - tz.dst(dt)
```

must return the same result for every **datetime** *dt* with `dt.tzinfo == tz`. For sane **tzinfo** subclasses, this expression yields the time zone's "standard offset", which should not depend on the date or the time, but only on geographic location. The implementation of **datetime.astimezone()** relies on this, but cannot detect violations; it's the programmer's responsibility to ensure it. If a **tzinfo** subclass cannot guarantee this, it may be able to override the default implementation of **tzinfo.fromutc()** to work correctly with **astimezone()** regardless.

Maior parte das implementações de **dst()** provavelmente irá parecer com um destes dois:

```
import datetime as dt

def dst(self, when):
    # a fixed-offset class:  doesn't account for DST
    return dt.timedelta(0)
```

ou:

```
import datetime as dt

def dst(self, when):
    # Code to set dston and dstoff to the time zone's DST
    # transition times based on the input when.year, and expr
    # in standard local time.

    if dston <= when.replace(tzinfo=None) < dstoff:
        return dt.timedelta(hours=1)
    else:
        return dt.timedelta(0)
```

A implementação padrão de **dst()** levanta **NotImplementedError**.

Alterado na versão 3.7: A diferença de horário de verão não é restrita a um número completo de minutos.

tzinfo.tzname(*dt*)

Return the time zone name corresponding to the **datetime** object *dt*, as a string. Nothing about string names is defined by the **datetime** module, and there's no requirement that it mean anything in particular. For example, `"GMT"`, `"UTC"`, `"-500"`, `"-5:00"`, `"EDT"`, `"US/Eastern"`, `"America/New York"` are all valid replies. Return `None` if a string name isn't known. Note that this is a method rather than a fixed string primarily because some **tzinfo** subclasses will wish to return different names depending on the specific value of *dt* passed, especially if the **tzinfo** class is accounting for daylight time.

A implementação padrão de **tzname()** levanta **NotImplementedError**.

These methods are called by a **datetime** or **time** object, in response to their methods of the same names. A **datetime** object passes itself as the argument, and a **time** object passes `None` as the argument. A **tzinfo** subclass's methods should therefore be prepared to accept a *dt* argument of `None`, or of class **datetime**.

Quando `None` é passado, cabe ao projetista da classe decidir a melhor resposta. Por exemplo, retornar `None` é apropriado se a classe deseja dizer que objetos time não participam nos protocolos da classe **tzinfo**. Pode ser mais útil para `utcoffset(None)` retornar a diferença UTC padrão, como não existe outra convenção para descobrir a diferença padrão.

When a **datetime** object is passed in response to a **datetime** method, `dt.tzinfo` is the same object as *self*. **tzinfo** methods can rely on this, unless user code calls **tzinfo** methods directly. The intent is that the **tzinfo** methods interpret *dt* as being in local time, and not need worry about objects in other time zones.

Existe mais um método **tzinfo** que uma subclasse pode desejar substituição:

tzinfo.fromutc(*dt*)

Isto é chamado a partir do padrão implementação de `datetime.astimezone()`. Quando chamado a partir dele, `dt.tzinfo` é *self*, e os dados de data e hora de *dt* devem ser vistos como expressando um Horário Universal Coordenado (UTC). O propósito de `fromutc()` é ajustar os dados de data e hora, retornando um *datetime* equivalente na hora local de *self*.

A maioria das subclasses de `tzinfo` deve ser capaz de herdar a implementação padrão de `fromutc()` sem problemas. Ela é robusta o suficiente para lidar com fusos horários de deslocamento fixo, e fusos horários que contabilizam tanto o horário padrão quanto o horário de verão, sendo que este último é tratado mesmo que os horários de transição do horário de verão sejam diferentes em anos distintos. Um exemplo de fuso horário que a implementação padrão de **`fromutc()`** pode não lidar corretamente em todos os casos é aquele onde o deslocamento padrão (de UTC) depende da data e hora específicas passadas, o que pode acontecer por razões políticas. As implementações padrão de `astimezone()` e **`fromutc()`** podem não produzir o resultado desejado se o resultado estiver entre as horas que abrangem o momento em que o deslocamento padrão muda.

Ignorando o código para casos de erros, a implementação padrão `fromutc()` funciona como:

```
import datetime as dt

def fromutc(self, when):
    # raise ValueError error if when.tzinfo is not self
    dtoff = when.utcoffset()
    dtdst = when.dst()
    # raise ValueError if dtoff is None or dtdst is None
    delta = dtoff - dtdst  # this is self's standard offset
    if delta:
        when += delta   # convert to standard local time
        dtdst = when.dst()
        # raise ValueError if dtdst is None
    if dtdst:
        return when + dtdst
    else:
        return when
```

No seguinte arquivo **`tzinfo_examples.py`** existem alguns exemplos de classes **`tzinfo`**:

```
import datetime as dt

# A class capturing the platform's idea of local time.
# (May result in wrong values on historical times in
#  timezones where UTC offset and/or the DST rules had
#  changed in the past.)
import time

ZERO = dt.timedelta(0)
HOUR = dt.timedelta(hours=1)
SECOND = dt.timedelta(seconds=1)

STDOFFSET = dt.timedelta(seconds=-time.timezone)
if time.daylight:
    DSTOFFSET = dt.timedelta(seconds=-time.altzone)
else:
    DSTOFFSET = STDOFFSET

DSTDIFF = DSTOFFSET - STDOFFSET


class LocalTimezone(dt.tzinfo):

    def fromutc(self, when):
        assert when.tzinfo is self
        stamp = (when - dt.datetime(1970, 1, 1, tzinfo=self)) //
        args = time.localtime(stamp)[:6]
        dst_diff = DSTDIFF // SECOND
        # Detect fold
        fold = (args == time.localtime(stamp - dst_diff))
        return dt.datetime(*args, microsecond=when.microsecond,
                           tzinfo=self, fold=fold)

    def utcoffset(self, when):
        if self._isdst(when):
            return DSTOFFSET
        else:
            return STDOFFSET
```

```
    def dst(self, when):
        if self._isdst(when):
            return DSTDIFF
        else:
            return ZERO

    def tzname(self, when):
        return time.tzname[self._isdst(when)]

    def _isdst(self, when):
        tt = (when.year, when.month, when.day,
              when.hour, when.minute, when.second,
              when.weekday(), 0, 0)
        stamp = time.mktime(tt)
        tt = time.localtime(stamp)
        return tt.tm_isdst > 0


Local = LocalTimezone()


# A complete implementation of current DST rules for major US ti

def first_sunday_on_or_after(when):
    days_to_go = 6 - when.weekday()
    if days_to_go:
        when += dt.timedelta(days_to_go)
    return when


# US DST Rules
#
# This is a simplified (i.e., wrong for a few cases) set of rule
# DST start and end times. For a complete and up-to-date set of
# and timezone definitions, visit the Olson Database (or try pyt
# http://www.twinsun.com/tz/tz-link.htm
# https://sourceforge.net/projects/pytz/ (might not be up-to-dat
#
# In the US, since 2007, DST starts at 2am (standard time) on th
# Sunday in March, which is the first Sunday on or after Mar 8.
DSTSTART_2007 = dt.datetime(1, 3, 8, 2)
# and ends at 2am (DST time) on the first Sunday of Nov.
DSTEND_2007 = dt.datetime(1, 11, 1, 2)
```

```
# From 1987 to 2006, DST used to start at 2am (standard time) or
# Sunday in April and to end at 2am (DST time) on the last
# Sunday of October, which is the first Sunday on or after Oct 2
DSTSTART_1987_2006 = dt.datetime(1, 4, 1, 2)
DSTEND_1987_2006 = dt.datetime(1, 10, 25, 2)
# From 1967 to 1986, DST used to start at 2am (standard time) or
# Sunday in April (the one on or after April 24) and to end at 2
# on the last Sunday of October, which is the first Sunday
# on or after Oct 25.
DSTSTART_1967_1986 = dt.datetime(1, 4, 24, 2)
DSTEND_1967_1986 = DSTEND_1987_2006


def us_dst_range(year):
    # Find start and end times for US DST. For years before 1967
    # start = end for no DST.
    if 2006 < year:
        dststart, dstend = DSTSTART_2007, DSTEND_2007
    elif 1986 < year < 2007:
        dststart, dstend = DSTSTART_1987_2006, DSTEND_1987_2006
    elif 1966 < year < 1987:
        dststart, dstend = DSTSTART_1967_1986, DSTEND_1967_1986
    else:
        return (dt.datetime(year, 1, 1), ) * 2

    start = first_sunday_on_or_after(dststart.replace(year=year)
    end = first_sunday_on_or_after(dstend.replace(year=year))
    return start, end


class USTimeZone(dt.tzinfo):

    def __init__(self, hours, reprname, stdname, dstname):
        self.stdoffset = dt.timedelta(hours=hours)
        self.reprname = reprname
        self.stdname = stdname
        self.dstname = dstname

    def __repr__(self):
        return self.reprname

    def tzname(self, when):
        if self.dst(when):
```

```
        return self.dstname
    else:
        return self.stdname

def utcoffset(self, when):
    return self.stdoffset + self.dst(when)

def dst(self, when):
    if when is None or when.tzinfo is None:
        # An exception may be sensible here, in one or both
        # It depends on how you want to treat them.  The def
        # fromutc() implementation (called by the default as
        # implementation) passes a datetime with when.tzinfo
        return ZERO
    assert when.tzinfo is self
    start, end = us_dst_range(when.year)
    # Can't compare naive to aware objects, so strip the tim
    # when first.
    when = when.replace(tzinfo=None)
    if start + HOUR <= when < end - HOUR:
        # DST is in effect.
        return HOUR
    if end - HOUR <= when < end:
        # Fold (an ambiguous hour): use when.fold to disambi
        return ZERO if when.fold else HOUR
    if start <= when < start + HOUR:
        # Gap (a non-existent hour): reverse the fold rule.
        return HOUR if when.fold else ZERO
    # DST is off.
    return ZERO

def fromutc(self, when):
    assert when.tzinfo is self
    start, end = us_dst_range(when.year)
    start = start.replace(tzinfo=self)
    end = end.replace(tzinfo=self)
    std_time = when + self.stdoffset
    dst_time = std_time + HOUR
    if end <= dst_time < end + HOUR:
        # Repeated hour
        return std_time.replace(fold=1)
    if std_time < start or dst_time >= end:
        # Standard time
```

```
            return std_time
        if start <= std_time < end - HOUR:
            # Daylight saving time
            return dst_time


Eastern  = USTimeZone(-5, "Eastern",  "EST", "EDT")
Central  = USTimeZone(-6, "Central",  "CST", "CDT")
Mountain = USTimeZone(-7, "Mountain", "MST", "MDT")
Pacific  = USTimeZone(-8, "Pacific",  "PST", "PDT")
```

Perceba que existem sutilezas inevitáveis duas vezes por ano em uma subclasse **`tzinfo`** contabilizando tanto hora normal e horário de verão, nos pontos de transição do horário de verão. Para concretude, considere a costa leste dos EUA (UTC -0500), onde o horário de verão EDT começa no minuto após 1:59 (EST, hora padrão) no segundo domingo de Março, e termina no minuto posterior a 1:59 (EDT, horário de verão) no primeiro domingo de Novembro:

```
  UTC   3:MM  4:MM  5:MM  6:MM  7:MM  8:MM
  EST  22:MM 23:MM  0:MM  1:MM  2:MM  3:MM
  EDT  23:MM  0:MM  1:MM  2:MM  3:MM  4:MM

start  22:MM 23:MM  0:MM  1:MM  3:MM  4:MM

  end  23:MM  0:MM  1:MM  1:MM  2:MM  3:MM
```

Quando o horário de verão inicia (a linha de “início”), o relógio de parede local salta de 1:59 para 3:00. Um horário de parede da forma 2:MM realmente não faz sentido nesse dia, então `astimezone(Eastern)` não irá entregar um resultado com `hour == 2` no dia que o horário de verão começar. Por exemplo, na primavera de transição para frente em 2016, nós tivemos:

```
>>> import datetime as dt
>>> from tzinfo_examples import HOUR, Eastern
>>> u0 = dt.datetime(2016, 3, 13, 5, tzinfo=dt.timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
```

```
...     print(u.time(), 'UTC =', t.time(), t.tzname())
...
05:00:00 UTC = 00:00:00 EST
06:00:00 UTC = 01:00:00 EST
07:00:00 UTC = 03:00:00 EDT
08:00:00 UTC = 04:00:00 EDT
```

Quando o horário de verão termina (a linha “end”), há um problema potencialmente pior: existe uma hora que não pode ser expressa de forma inequívoca no horário local: a última hora do horário de verão. No fuso Eastern, isso corresponde a horários no formato 5:MM UTC no dia em que o horário de verão termina. O relógio de parede local salta de 1:59 (horário de verão) para 1:00 (horário padrão) novamente. Horários locais no formato 1:MM são ambíguos. **astimezone()** imita o comportamento do relógio local mapeando então duas horas UTC adjacentes para a mesma hora local. No exemplo do fuso Eastern, horários UTC no formato 5:MM e 6:MM ambos se transformam em 1:MM quando convertidos para o fuso Eastern, mas os horários anteriores têm o atributo **fold** definido como 0 e os horários posteriores têm o atributo definido como 1. Por exemplo, na transição de retorno de 2016, obtemos:

```
>>> import datetime as dt
>>> from tzinfo_examples import HOUR, Eastern
>>> u0 = dt.datetime(2016, 11, 6, 4, tzinfo=dt.timezone.utc)
>>> for i in range(4):
...     u = u0 + i*HOUR
...     t = u.astimezone(Eastern)
...     print(u.time(), 'UTC =', t.time(), t.tzname(), t.fold)
...
04:00:00 UTC = 00:00:00 EDT 0
05:00:00 UTC = 01:00:00 EDT 0
06:00:00 UTC = 01:00:00 EST 1
07:00:00 UTC = 02:00:00 EST 0
```

Observe que as instâncias de **datetime** que diferem apenas pelo valor do atributo **fold** são consideradas iguais nas comparações.

Applications that can’t bear wall-time ambiguities should explicitly check the value of the **fold** attribute or avoid using hybrid **tzinfo** subclasses; there are no ambiguities when using **timezone**, or any other fixed-offset

tzinfo subclass (such as a class representing only EST (fixed offset -5 hours), or only EDT (fixed offset -4 hours)).

Ver também

zoneinfo

O módulo **datetime** possui uma classe básica **timezone** (para lidar com deslocamentos fixos arbitrários do UTC) e seu atributo **timezone.utc** (uma instância UTC **timezone**).

zoneinfo traz o *banco de dados de fuso horário IANA* (também conhecido como banco de dados Olson) para Python, e seu uso é recomendado.

Banco de dados de fuso horário da IANA **[https://www.iana.org/time-zones]**

O banco de dados de fuso horário (comumente chamado de tz, tzdata ou zoneinfo) contém código e dados que representam o histórico de hora local para muitas localizações representativas ao redor do globo. Ele é atualizado periodicamente para refletir mudanças feitas por corpos políticos para limites de fuso horário, diferenças UTC, e regras de horário de verão.

timezone objects

A classe **timezone** é uma subclasse de **tzinfo**, cada instância da qual representa um fuso horário definido por um deslocamento fixo do UTC.

Objetos desta classe não podem ser usados para representar informações de fuso horário em locais onde diferentes deslocamentos são usados em diferentes dias do ano ou onde mudanças históricas foram feitas no horário civil.

class datetime.timezone(*offset*[, *name*])

O argumento *offset* deve ser especificado como um objeto **timedelta** representando a diferença entre o tempo local e o UTC. Ele deve estar

estritamente entre `-timedelta(hours=24)` e `timedelta(hous=24)`, caso contrário a exceção **`ValueError`** será provocada.

O argumento *name* é opcional. Se especificado, deve ser uma string que será usada como o valor retornado pelo método **`datetime.tzname()`**.

Adicionado na versão 3.2.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

timezone.utcoffset(*dt*)

Retorna o valor fixo especificado quando a instância **`timezone`** é construída.

O argumento *dt* é ignorado. O valor de retorno é uma instância **`timedelta`** equivalente à diferença entre o tempo local e o UTC.

Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

timezone.tzname(*dt*)

Retorna o valor fixo especificado quando a instância **`timezone`** é construída.

Se *name* não é passado ao construtor, o nome retornado por `tzname(dt)` é gerado a partir do valor de `offset` do seguinte modo: Se *offset* é `timedelta(0)`, o nome é “UTC”, caso contrário é uma string no formato `UTC±HH:MM`, na qual ± é o sinal do `offset`, HH e MM são dois dígitos de `offset.hours` e `offset.minutes` respectivamente.

Alterado na versão 3.6: Nome gerado de `offset=timedelta(0)` é agora simplesmente `'UTC'`, não `'UTC+00:00'`.

timezone.dst(*dt*)

Sempre retorna `None`.

timezone.fromutc(*dt*)

Retorna `dt + offset`. O argumento *dt* deve ser uma instância **`datetime`** consciente, com `tzinfo` definida para `self`.

Atributos de classe:

timezone.utc

O fuso horário UTC, `timezone(timedelta(0))`.

`strftime()` and `strptime()` behavior

Todos os objetos **`date`**, **`datetime`** e **`time`** dão suporte ao método `strftime(format)`, para criar uma string representando o tempo sob o controle de uma string de formatação explícita.

Em recíproca, os métodos de classe **`date.strptime()`**, **`datetime.strptime()`** e **`time.strptime()`** criam um objeto a partir de uma string representando o tempo e uma string de formatação correspondente.

A tabela abaixo fornece uma comparação de alto nível entre **`strftime()`** e **`strptime()`**:

	`strftime`	**`strptime`**
Uso	Converte objeto para uma string conforme um formato fornecido	Interpreta uma string como um objeto dado um formato correspondente
Tipo de método	Método de instância	Método de classe

	strftime	strptime
Assinatura	`strftime(format)`	`strptime(date_string, format)`

`strftime()` and `strptime()` format codes

Esses métodos aceitam códigos de formato que podem ser usados para analisar e formatar datas:

```
>>> import datetime as dt
>>> dt.datetime.strptime('31/01/22 23:59:59.999999',
...                      '%d/%m/%y %H:%M:%S.%f')
datetime.datetime(2022, 1, 31, 23, 59, 59, 999999)
>>> _.strftime('%a %d %b %Y, %I:%M%p')
'Mon 31 Jan 2022, 11:59PM'
```

The following is a list of all the format codes that the 2011 C standard requires, and these work on all supported platforms.

Diretiva	Significado	Exemplo	Notas
`%a`	Dias da semana como nomes abreviados da localidade.	Sun, Mon, …, Sat (en_US); So, Mo, …, Sa (de_DE)	(1)

Diretiva	Significado	Exemplo	Notas
%A	Dia da semana como nome completo da localidade.	Sunday, Monday, …, Saturday (en_US); Sonntag, Montag, …, Samstag (de_DE)	(1)
%b	Mês como nome da localidade abreviado.	Jan, Feb, …, Dec (en_US); Jan, Feb, …, Dez (de_DE)	(1)
%B	Mês como nome completo da localidade.	January, February, …, December (en_US); janeiro, fevereiro, …, dezembro (pt_BR)	(1)
%c	Representação de data e hora apropriada da localidade.	Tue Aug 16 21:30:00 1988 (en_US); Di 16 Aug 21:30:00 1988 (de_DE)	(1)

Diretiva	Significado	Exemplo	Notas
`%C`	The year divided by 100 and truncated to an integer as a zero-padded decimal number.	01, 02, …, 99	(0)
`%d`	Dia do mês como um número decimal com zeros a esquerda.	01, 02, …, 31	(9), (10)
`%D`	Equivalent to `%m/%d/%y`.	11/28/25	(9)
`%e`	The day of the month as a space-padded decimal number.	␣1, ␣2, …, 31	(10)
`%F`	Equivalent to `%Y-%m-%d`, the ISO 8601 format.	2025-10-11, 1001-12-30	
`%g`	Last 2 digits of ISO 8601 year representing the year that contains the greater part of the ISO week (`%V`).	00, 01, …, 99	(0)
`%G`	Ano ISO 8601 com o século representando o ano que a maior parte da semana ISO (`%V`).	0001, 0002, …, 2013, 2014, …, 9998, 9999	(8)
`%h`	Equivalent to `%b`.	See `%b`.	(0)

Diretiva	Significado	Exemplo	Notas
`%H`	Hora (relógio de 24 horas) como um número decimal com zeros a esquerda.	00, 01, …, 23	(9)
`%I`	Hora (relógio de 12 horas) como um número decimal com zeros a esquerda.	01, 02, …, 12	(9)
`%j`	Dia do ano como um número decimal, com zeros a esquerda.	001, 002, …, 366	(9)
`%m`	Mês como um número decimal com zeros a esquerda.	01, 02, …, 12	(9)
`%M`	Minutos como um número decimal, com zeros a esquerda.	00, 01, …, 59	(9)
`%n`	The newline character (`'\n'`). For **`strptime()`**, zero or more whitespace.	`\n`	
`%p`	Equivalente da localidade a AM ou PM.	AM, PM (en_US); am, pm (de_DE)	(1), (3)

Diretiva	Significado	Exemplo	Notas
`%r`	Locale’s 12-hour clock time.	12:00:00 AM	(1), (0)
`%R`	Equivalent to `%H:%M`.	10:01	
`%S`	Segundos como um número decimal, com zeros a esquerda.	00, 01, …, 59	(4), (9)
`%t`	The tab character (`'\t'`). For **`strptime()`**, zero or more whitespace.	`\t`	
`%T`	ISO 8601 time format, equivalent to `%H:%M:%S`.	10:01:59	
`%u`	Dia de semana ISO 8601 como um número decimal onde 1 é segunda-feira.	1, 2, …, 7	
`%U`	Número da semana do ano (Domingo como o primeiro dia da semana) como um número decimal, com zeros a esquerda. Todos os dias em um ano novo precedendo o primeiro domingo são considerados como estando na semana 0.	00, 01, …, 53	(7), (9)

Diretiva	Significado	Exemplo	Notas
`%V`	Semana ISO 8601 como um número decimal, com segunda-feira como o primeiro dia da semana. A semana 01 é a semana contendo o dia 4 de Janeiro.	01, 02, …, 53	(8), (9)
`%w`	Dia da semana como um número decimal, onde 0 é domingo e 6 é sábado.	0, 1, …, 6	
`%W`	Número da semana do ano (Segunda-feira como o primeiro dia da semana) como um número decimal, com zeros a esquerda. Todos os dias em um ano novo precedendo a primeira segunda-feira são considerados como estando na semana 0.	00, 01, …, 53	(7), (9)
`%x`	Representação de data apropriada de localidade.	08/16/88 (None); 08/16/1988 (en_US); 16.08.1988 (de_DE)	(1)
`%X`	Representação de hora apropriada da localidade.	21:30:00 (en_US); 21:30:00 (de_DE)	(1)

Diretiva	Significado	Exemplo	Notas
`%y`	Ano sem século como um número decimal com zeros a esquerda.	00, 01, …, 99	(9)
`%Y`	Ano com século como um número decimal.	0001, 0002, …, 2013, 2014, …, 9998, 9999	(2)
`%z`	Diferença UTC no formato `±HHMM[SS[.ffffff]]` (string vazia se o objeto é ingênuo).	(vazio), +0000, -0400, +1030, +063415, -030712.345216	(6)
`%Z`	Nome do fuso horário (string vazia se o objeto é ingênuo).	(vazio), UTC, GMT	(6)
`%%`	Um caractere literal `'%'`.	%	

The ISO 8601 year and ISO 8601 week directives are not interchangeable with the year and week number directives above. Calling `strptime()` with incomplete or ambiguous ISO 8601 directives will raise a `ValueError`.

Several additional directives not required by the C11 standard are included for convenience.

Diretiva	Significado	Exemplo	Notas

Diretiva	Significado	Exemplo	Notas
`%f`	Microssegundos como um número decimal, com zeros à esquerda até completar 6 dígitos.	000000, 000001, …, 999999	(5)
`%:z`	Diferença UTC no formato `±HH:MM[:SS[.ffffff]]` (string vazia se o objeto é ingênuo).	(vazio), +00:00, -04:00, +10:30, +06:34:15, -03:07:12.345216	(6)

O conjunto completo de códigos de formato suportados varia entre as plataformas, porque o Python chama a função **`strftime()`** da biblioteca C da plataforma, e variações de plataforma são comuns. Para ver o conjunto completo de códigos de formato suportados em sua plataforma, consulte a documentação *strftime(3)* *[https://manpages.debian.org/strftime(3)]*. Também existem diferenças entre plataformas no tratamento de especificadores de formato não suportados.

Adicionado na versão 3.6: `%G`, `%u` e `%V` foram adicionados.

Adicionado na versão 3.12: `%:z` was added for **`strftime()`**.

Adicionado na versão 3.15: `%D`, `%F`, `%n`, `%t`, and `%:z` were added for **`strptime()`**.

Technical detail

Em termos gerais, `d.strftime(fmt)` age como `time.strftime(fmt, d.timetuple())` do módulo **`time`**, embora nem todos os objetos suportem um método **`timetuple()`**.

For the **`datetime.strptime()`** and **`date.strptime()`** class methods, the default value is `1900-01-01T00:00:00.000`: any components not specified in the format string will be pulled from the default value.

Nota

Format strings without separators can be ambiguous for parsing. For example, with `%Y%m%d`, the string `2026111` may be parsed either as `2026-11-01` or as `2026-01-11`. Use separators to ensure the input is parsed as intended.

Nota

When used to parse partial dates lacking a year, **`datetime.strptime()`** and **`date.strptime()`** will raise when encountering February 29 because the default year of 1900 is *not* a leap year. Always add a default leap year to partial date strings before parsing.

```
>>> import datetime as dt
>>> value = "2/29"
>>> dt.datetime.strptime(value, "%m/%d")
Traceback (most recent call last):
...
ValueError: day 29 must be in range 1..28 for month 2 in year 19
>>> dt.datetime.strptime(f"1904 {value}", "%Y %m/%d")
datetime.datetime(1904, 2, 29, 0, 0)
```

Usar `datetime.strptime(date_string, format)` é equivalente a:

```
datetime(*(time.strptime(date_string, format)[0:6]))
```

exceto quando o formato inclui componentes de sub-segundos ou informações de deslocamento de fuso horário, que são suportados em `datetime.strptime` mas são descartados por `time.strptime`.

Para objetos `time`, os códigos de formato para ano, mês e dia não devem ser usados, pois objetos **`time`** não têm tais valores. Se forem usados de qualquer forma, o ano é substituído por 1900, e o mês e dia por 1.

Para objetos `date`, os códigos de formato para horas, minutos, segundos e microssegundos não devem ser usados, pois objetos **`date`** não têm tais valores. Se forem usados de qualquer forma, são substituídos por 0.

Pela mesma razão, o tratamento de formato de strings contendo pontos de código Unicode que não podem ser representados no conjunto de caracteres da localidade atual também é dependente da plataforma. Em algumas plataformas, tais pontos de código são preservados intactos na saída, enquanto em outros `strftime` pode levantar `UnicodeError` ou retornar uma string vazia ao invés.

Notas:

0. This format code is currently unsupported by `strptime()`.

1. Como o formato depende da localidade atual, deve-se tomar cuidado ao fazer suposições sobre o valor de saída. A ordenação dos campos irá variar (por exemplo, “mês/dia/ano” versus “dia/mês/ano”) e a saída pode conter caracteres não ASCII.

2. O método `strptime()` pode analisar anos no intervalo completo de [1, 9999], mas anos < 1000 devem ser preenchidos com zeros até uma largura de 4 dígitos.

   Alterado na versão 3.2: Em versões anteriores, o método `strftime()` era limitado a anos >= 1900.

   Alterado na versão 3.3: Na versão 3.2, o método `strftime()` era limitado a anos >= 1000.

3. Quando usado com o método `strptime()`, a diretiva `%p` afeta somente o campo de hora de saída se a diretiva `%I` for usada para analisar a hora.

4. Ao contrário do módulo `time`, o módulo **`datetime`** não oferece suporte para segundos intercalares.

5. Quando usado com o método `strptime()`, a diretiva `%f` aceita de um a seis dígitos e zeros à direita. `%f` é uma extensão do conjunto de caracteres de formato no padrão do C (mas implementado separadamente em objetos datetime e, portanto, sempre disponível).

6. Para um objeto ingênuo, os códigos de formatação `%z`, `%:z` e `%Z` são substituídos por strings vazias.

   Para um objeto consciente:

   `%z`

   `utcoffset()` é transformado em uma string do formato `±HHMM[SS[.ffffff]]`, onde `HH` é uma string de 2 dígitos que fornece o número de horas de deslocamento UTC, `MM` é uma string de 2 dígitos que fornece o número de minutos de deslocamento UTC, `SS` é uma string de 2 dígitos que fornece o número de segundos de deslocamento UTC e `ffffff` é uma string de 6 dígitos que fornece o número de microssegundos de deslocamento UTC. A parte `ffffff` é omitida quando o deslocamento é um número inteiro de segundos e tanto a parte `ffffff` quanto a parte `SS` são omitidas quando o deslocamento é um número inteiro de minutos. Por exemplo, se **`utcoffset()`** retornar `timedelta(hours=-3, minutes=-30)`, `%z` será substituído pela string `'-0330'`.

   Alterado na versão 3.7: A diferença UTC não é restrita a um número completo de minutos.

   Alterado na versão 3.7: When the `%z` directive is provided to the `strptime()` method, the UTC offsets can have a colon as a separator between hours, minutes and seconds. For example, both `'+010000'` and `'+01:00:00'` will be parsed as an offset of one hour. In addition, providing `'Z'` is identical to `'+00:00'`.

   `%:z`

When used with **strftime()**, behaves exactly as `%z`, except that a colon separator is added between hours, minutes and seconds.

When used with **strptime()**, the UTC offset is *required* to have a colon as a separator between hours, minutes and seconds. For example, `'+01:00:00'` (but *not* `'+010000'`) will be parsed as an offset of one hour. In addition, providing `'Z'` is identical to `'+00:00'`.

`%Z`

Em **strftime()**, `%Z` é substituído por uma string vazia se **tzname()** retornar `None`; caso contrário, `%Z` é substituído pelo valor retornado, que deve ser uma string.

strptime() aceita apenas certos valores para `%Z`:

1. qualquer valor em `time.tzname` para a localidade da sua máquina
2. os valores codificados `UTC` e `GMT`

Então alguém vivendo no Japão pode ter `JST`, `UTC`, e `GMT` como valores válidos, mas provavelmente não `EST`. Isso levantará `ValueError` para valores inválidos.

Alterado na versão 3.2: Quando a diretiva `%z` é fornecida ao método **strptime()**, um objeto **datetime** consciente será produzido. O `tzinfo` do resultado será definido para uma instância **timezone**.

7. Quando utilizados com o método **strptime()**, `%U` e `%W` são usados somente em cálculos quando o dia da semana e o ano (`%Y`) são especificados.

8. Semelhante a `%U` e `%W`, `%V` é usado apenas em cálculos quando o dia da semana e o ano ISO (`%G`) são especificados em um formato de string **strptime()**. Observe também que `%G` e `%Y` não são intercambiáveis.

9. Quando usado com o método **strptime()**, o zero à esquerda é opcional para os formatos `%d`, `%m`, `%H`, `%I`, `%M`, `%S`, `%j`, `%U`, `%W` e `%V`. O formato `%y` requer um zero à esquerda.

10. When parsing a month and day using `strptime()`, always include a year in the format. If the value you need to parse lacks a year, append an explicit dummy leap year. Otherwise your code will raise an exception when it encounters leap day because the default year used by the parser (1900) is not a leap year. Users run into that bug every leap year.

    ```
    >>> month_day = "02/29"
    >>> dt.datetime.strptime(f"{month_day};1984", "%m/%d;%Y")  #
    datetime.datetime(1984, 2, 29, 0, 0)
    ```

    Alterado na versão 3.15: Using `%d` without a year now raises `ValueError`.

    Descontinuado desde a versão 3.15, será removido na versão 3.17: `strptime()` calls using a format string containing `%e` without a year now emit a `DeprecationWarning`.

Notas de rodapé

[1]

If, that is, we ignore the effects of relativity.

[2]

Isso combina com a definição do calendário “proléptico Gregoriano” no livro *Cálculos Calendáricos* de Dershowitz e Reingold, onde ele é o calendário base para todas as computações. Veja o livro para algoritmos para conversão entre ordinais proléptico Gregoriano e muitos outros sistemas de calendário.

[3]

Veja O guia para a matemática de calendário ISO 8601 [https://web.archive.org/web/20220531051136/https://webspace.science.uu.nl/~gent0113/calendar/isocalendar.htm] de R. H. van Gent para uma boa explicação.

`zoneinfo` — Suporte a fuso horário da IANA

Adicionado na versão 3.9.

Código-fonte: Lib/zoneinfo [https://github.com/python/cpython/tree/3.15/Lib/zoneinfo]

The `zoneinfo` module provides a concrete time zone implementation to support the IANA time zone database as originally specified in **PEP 615** [https://peps.python.org/pep-0615/]. By default, `zoneinfo` uses the system's time zone data if available; if no system time zone data is available, the library will fall back to using the first-party tzdata [https://pypi.org/project/tzdata/] package available on PyPI.

Ver também

Módulo: `datetime`
Fornece os tipos `time` e `datetime` com os quais a classe `ZoneInfo` foi projetada para ser usada.

Pacote tzdata [https://pypi.org/project/tzdata/]
Pacote próprio mantido pelos desenvolvedores principais do CPython para fornecer dados de fuso horário via PyPI.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Usando `ZoneInfo`

ZoneInfo é uma implementação concreta da classe base abstrata **datetime.tzinfo** e deve ser anexada a `tzinfo`, por meio do construtor, do método **datetime.replace** ou **datetime.astimezone**:

```
>>> from zoneinfo import ZoneInfo
>>> import datetime as dt

>>> when = dt.datetime(2020, 10, 31, 12, tzinfo=ZoneInfo("Americ
>>> print(when)
2020-10-31 12:00:00-07:00

>>> when.tzname()
'PDT'
```

Datas e horas construídas dessa maneira são compatíveis com a aritmética de data e hora e lidam com transições de horário de verão sem intervenção adicional:

```
>>> when_add = when + dt.timedelta(days=1)

>>> print(when_add)
2020-11-01 12:00:00-08:00

>>> when_add.tzname()
'PST'
```

Esses fusos horários também oferecem suporte ao atributo **fold** introduzido em **PEP 495** [https://peps.python.org/pep-0495/]. Durante transições de deslocamento que induzem horários ambíguos (como uma transição do horário de verão para o horário padrão), o deslocamento *anterior* à transição é usado quando `fold=0`, e o deslocamento *posterior* à transição é usado quando `fold=1`, por exemplo:

```
>>> when = dt.datetime(2020, 11, 1, 1, tzinfo=ZoneInfo("America/
>>> print(when)
2020-11-01 01:00:00-07:00

>>> print(when.replace(fold=1))
2020-11-01 01:00:00-08:00
```

Ao converter de outro fuso horário, a dobra será definida para o valor correto:

```
>>> LOS_ANGELES = ZoneInfo("America/Los_Angeles")
>>> when_utc = dt.datetime(2020, 11, 1, 8, tzinfo=dt.timezone.ut

>>> # Before the PDT -> PST transition
>>> print(when_utc.astimezone(LOS_ANGELES))
2020-11-01 01:00:00-07:00

>>> # After the PDT -> PST transition
>>> print((when_utc + dt.timedelta(hours=1)).astimezone(LOS_ANGE
2020-11-01 01:00:00-08:00
```

Fontes de dados

O módulo `zoneinfo` não fornece dados de fuso horário diretamente, mas, em vez disso, extrai informações de fuso horário do banco de dados de fuso horário do sistema ou do pacote PyPI original tzdata [https://pypi.org/project/tzdata/], se disponível. Alguns sistemas, incluindo principalmente os Windows, não possuem um banco de dados IANA disponível e, portanto, para projetos que buscam compatibilidade multiplataforma e exigem dados de fuso horário, recomenda-se declarar uma dependência em tzdata. Se nem os dados do sistema nem o tzdata estiverem disponíveis, todas as chamadas para **`ZoneInfo`** levantarão **`ZoneInfoNotFoundError`**.

Configurando as fontes de dados

Quando `ZoneInfo(key)` é chamado, o construtor primeiro pesquisa nos diretórios especificados em **`TZPATH`** por um arquivo que corresponda a `key` e, em caso de falha, procura uma correspondência no pacote tzdata. Esse comportamento pode ser configurado de três maneiras:

1. O padrão **`TZPATH`**, quando não especificado de outra forma, pode ser configurado em tempo de compilação.
2. **`TZPATH`** pode ser configurado usando uma variável de ambiente.

3. Em tempo de execução, o caminho de pesquisa pode ser manipulado usando a função `reset_tzpath()`.

Configuração em tempo de compilação

O `TZPATH` padrão inclui vários locais de implantação comuns para o banco de dados de fuso horário (exceto no Windows, onde não há locais "conhecidos" para dados de fuso horário). Em sistemas POSIX, distribuidores downstream e aqueles que constroem o Python a partir do código-fonte e que sabem onde os dados de fuso horário do sistema estão implantados podem alterar o caminho do fuso horário padrão especificando a opção de tempo de compilação `TZPATH` (ou, mais provavelmente, o `sinalizador de configuração --with-tzpath`), que deve ser uma string delimitada por `os.pathsep`.

Em todas as plataformas, o valor configurado está disponível como a chave `TZPATH` em `sysconfig.get_config_var()`.

Configuração do ambiente

Ao inicializar `TZPATH` (no momento da importação ou sempre que `reset_tzpath()` for chamado sem argumentos), o módulo `zoneinfo` usará a variável de ambiente `PYTHONTZPATH`, se existir, para definir o caminho de pesquisa.

PYTHONTZPATH

Esta é uma string separada por `os.pathsep` contendo o caminho de pesquisa de fuso horário a ser usado. Ela deve consistir apenas em caminhos absolutos, e não relativos. Componentes relativos especificados em `PYTHONTZPATH` não serão usados, mas, caso contrário, o comportamento quando um caminho relativo é especificado é definido pela implementação; o CPython levantará `InvalidTZPathWarning`, mas outras implementações podem ignorar silenciosamente o componente incorreto ou levantar uma exceção.

Para configurar o sistema para ignorar os dados do sistema e usar o pacote tzdata, defina `PYTHONTZPATH=""`.

Configuração em tempo de execução

O caminho de busca TZ também pode ser configurado em tempo de execução usando a função `reset_tzpath()`. Geralmente, essa não é uma operação aconselhável, embora seja razoável usá-la em funções de teste que exigem o uso de um caminho de fuso horário específico (ou que exijam a desativação do acesso aos fusos horários do sistema).

A classe `ZoneInfo`

class zoneinfo.ZoneInfo(*key*)

Uma subclasse concreta de `datetime.tzinfo` que representa um fuso horário IANA especificado pela string `key`. Chamadas ao construtor primário sempre retornarão objetos que sejam idênticos em comparação; em outras palavras, exceto pela invalidação de cache via `ZoneInfo.clear_cache()`, para todos os valores de `key`, a seguinte asserção será sempre verdadeira:

```
a = ZoneInfo(key)
b = ZoneInfo(key)
assert a is b
```

`key` deve estar no formato de um caminho POSIX relativo e normalizado, sem referências de nível superior. O construtor levantará `ValueError` se uma chave não conforme for passada.

Se nenhum arquivo correspondente a `key` for encontrado, o construtor levantará `ZoneInfoNotFoundError`.

A classe `ZoneInfo` tem dois construtores alternativos:

classmethod ZoneInfo.from_file(*file_obj*, */*, *key=None*)

Constructs a `ZoneInfo` object from a file-like object returning bytes (e.g. a file opened in binary mode or an `io.BytesIO` object). Unlike the primary constructor, this always constructs a new object.

The `key` parameter sets the name of the zone for the purposes of `__str__()` and `__repr__()`.

Objects created via this constructor cannot be pickled (see pickling).

`ValueError` is raised if the data read from *file_obj* is not a valid TZif file.

classmethod ZoneInfo.no_cache(*key*)

An alternate constructor that bypasses the constructor's cache. It is identical to the primary constructor, but returns a new object on each call. This is most likely to be useful for testing or demonstration purposes, but it can also be used to create a system with a different cache invalidation strategy.

Objects created via this constructor will also bypass the cache of a deserializing process when unpickled.

Cuidado

Using this constructor may change the semantics of your datetimes in surprising ways, only use it if you know that you need to.

The following class methods are also available:

classmethod ZoneInfo.clear_cache(*, *only_keys=None*)

A method for invalidating the cache on the `ZoneInfo` class. If no arguments are passed, all caches are invalidated and the next call to the primary constructor for each key will return a new instance.

If an iterable of key names is passed to the `only_keys` parameter, only the specified keys will be removed from the cache. Keys passed to `only_keys` but not found in the cache are ignored.

Aviso

Invoking this function may change the semantics of datetimes using `ZoneInfo` in surprising ways; this modifies module state and thus may have wide-ranging effects. Only use it if you know that you need to.

A classe possui um atributo:

ZoneInfo.key

This is a read-only attribute that returns the value of `key` passed to the constructor, which should be a lookup key in the IANA time zone database (e.g. `America/New_York`, `Europe/Paris` or `Asia/Tokyo`).

For zones constructed from file without specifying a `key` parameter, this will be set to `None`.

Nota

Although it is a somewhat common practice to expose these to end users, these values are designed to be primary keys for representing the relevant zones and not necessarily user-facing elements. Projects like CLDR (the Unicode Common Locale Data Repository) can be used to get more user-friendly strings from these keys.

String representations

The string representation returned when calling **`str`** on a **`ZoneInfo`** object defaults to using the **`ZoneInfo.key`** attribute (see the note on usage in the attribute documentation):

```
>>> zone = ZoneInfo("Pacific/Kwajalein")
>>> str(zone)
'Pacific/Kwajalein'

>>> when = dt.datetime(2020, 4, 1, 3, 15, tzinfo=zone)
>>> f"{when.isoformat()} [{when.tzinfo}]"
'2020-04-01T03:15:00+12:00 [Pacific/Kwajalein]'
```

For objects constructed from a file without specifying a `key` parameter, `str` falls back to calling **`repr()`**. `ZoneInfo`'s `repr` is implementation-defined and not necessarily stable between versions, but it is guaranteed not to be a valid `ZoneInfo` key.

Pickle serialization

Rather than serializing all transition data, `ZoneInfo` objects are serialized by key, and `ZoneInfo` objects constructed from files (even those with a value for `key` specified) cannot be pickled.

The behavior of a `ZoneInfo` file depends on how it was constructed:

1. `ZoneInfo(key)`: When constructed with the primary constructor, a `ZoneInfo` object is serialized by key, and when deserialized, the deserializing process uses the primary and thus it is expected that these are the same object as other references to the same time zone. For example, if `europe_berlin_pkl` is a string containing a pickle constructed from `ZoneInfo("Europe/Berlin")`, one would expect the following behavior:

   ```
   >>> a = ZoneInfo("Europe/Berlin")
   >>> b = pickle.loads(europe_berlin_pkl)
   >>> a is b
   True
   ```

2. `ZoneInfo.no_cache(key)`: When constructed from the cache-bypassing constructor, the `ZoneInfo` object is also serialized by key, but when deserialized, the deserializing process uses the cache bypassing constructor. If `europe_berlin_pkl_nc` is a string containing a pickle constructed from `ZoneInfo.no_cache("Europe/Berlin")`, one would expect the following behavior:

   ```
   >>> a = ZoneInfo("Europe/Berlin")
   >>> b = pickle.loads(europe_berlin_pkl_nc)
   >>> a is b
   False
   ```

3. `ZoneInfo.from_file(file_obj, /, key=None)`: When constructed from a file, the `ZoneInfo` object raises an exception on pickling. If an end user wants to pickle a `ZoneInfo` constructed from a file, it is recommended that they use a wrapper type or a custom serialization function: either serializing by key or storing the contents of the file object and serializing that.

This method of serialization requires that the time zone data for the required key be available on both the serializing and deserializing side, similar to the way that references to classes and functions are expected to exist in both the serializing and deserializing environments. It also means that no guarantees are made about the consistency of results when unpickling a `ZoneInfo` pickled in an environment with a different version of the time zone data.

Funções

zoneinfo.available_timezones()

Get a set containing all the valid keys for IANA time zones available anywhere on the time zone path. This is recalculated on every call to the function.

This function only includes canonical zone names and does not include "special" zones such as those under the `posix/` and `right/` directories, the `posixrules` or the `localtime` zone.

Cuidado

This function may open a large number of files, as the best way to determine if a file on the time zone path is a valid time zone is to read the "magic string" at the beginning.

Nota

These values are not designed to be exposed to end-users; for user facing elements, applications should use something like CLDR (the Unicode Common Locale Data Repository) to get more user-friendly strings. See also the cautionary note on `ZoneInfo.key`.

zoneinfo.reset_tzpath(*to=None*)

Sets or resets the time zone search path (**TZPATH**) for the module. When called with no arguments, **TZPATH** is set to the default value.

Calling `reset_tzpath` will not invalidate the **ZoneInfo** cache, and so calls to the primary `ZoneInfo` constructor will only use the new `TZPATH` in the case of a cache miss.

The `to` parameter must be a sequence of strings or **os.PathLike** and not a string, all of which must be absolute paths. **ValueError** will be raised if something other than an absolute path is passed.

Globals

zoneinfo.TZPATH

A read-only sequence representing the time zone search path – when constructing a `ZoneInfo` from a key, the key is joined to each entry in the `TZPATH`, and the first file found is used.

`TZPATH` may contain only absolute paths, never relative paths, regardless of how it is configured.

The object that `zoneinfo.TZPATH` points to may change in response to a call to **reset_tzpath()**, so it is recommended to use `zoneinfo.TZPATH` rather than importing `TZPATH` from `zoneinfo` or assigning a long-lived variable to `zoneinfo.TZPATH`.

For more information on configuring the time zone search path, see Configurando as fontes de dados.

Exceptions and warnings

exception zoneinfo.ZoneInfoNotFoundError

Raised when construction of a **ZoneInfo** object fails because the specified key could not be found on the system. This is a subclass of

`KeyError`.

exception zoneinfo.InvalidTZPathWarning

Raised when `PYTHONTZPATH` contains an invalid component that will be filtered out, such as a relative path.

`calendar` — Funções gerais relacionadas ao calendário

Código-fonte: Lib/calendar.py
[https://github.com/python/cpython/tree/3.15/Lib/calendar.py]

Este módulo permite que você exiba calendários como o programa Unix **cal**, e fornece funções adicionais úteis relacionadas ao calendário. Por padrão, esses calendários têm a segunda-feira como o primeiro dia da semana, e domingo como o último (a convenção europeia). Use `setfirstweekday()` para colocar o primeiro dia da semana como domingo (6) ou para qualquer outro dia da semana. Parâmetros que especificam datas são dados como inteiros. Para funcionalidade relacionada, veja também os módulos `datetime` e `time`.

As funções e classes definidas neste módulo usam um calendário idealizado, o calendário Gregoriano atual estendido indefinidamente nas duas direções. Isso corresponde à definição do calendário "proleptic Gregorian" (gregoriano proléptico) no livro "Calendrical Calculations" de Dershowitz e Reingold, onde está o calendário base para todas os cálculos. Anos com zero ou negativos são interpretados e prescritos pelo padrão ISO 8601. Ano 0 é 1 A.C., ano -1 é 2 A.C, e de assim em diante.

class calendar.Calendar(*firstweekday=0*)

Cria um objeto `Calendar`. *firstweekday* é um inteiro que especifica o primeiro dia da semana. `MONDAY` é `0` (o padrão), `SUNDAY` é `6`.

Um objeto `Calendar` fornece vários métodos que podem ser usados para preparar os dados do calendário para formatação. Esta classe não realiza nenhuma formatação por si mesma. Esse é o trabalho das subclasses.

Instâncias de `Calendar` têm os seguintes métodos e atributos:

firstweekday

O primeiro dia da semana como um inteiro (0–6).

Esta propriedade também pode ser definida e lida usando `setfirstweekday()` e `getfirstweekday()` respectivamente.

getfirstweekday()

Retorna um `int` correspondente ao primeiro dia da semana atual (0-6).

Idêntico à leitura da propriedade `firstweekday`.

setfirstweekday(*firstweekday*)

Set the first weekday to *firstweekday*, passed as an `int` (0–6).

Idêntico à definição da propriedade `firstweekday`.

iterweekdays()

Return an iterator for the weekday numbers that will be used for one week. The first value from the iterator will be the same as the value of the `firstweekday` property.

itermonthdates(*year*, *month*)

Retorna um iterador para o mês *month* (1–12) no ano *year*. Este iterador retornará todos os dias (como objetos `datetime.date`) do mês, bem como os dias antes do início do mês ou após o final do mês, necessários para obter uma semana completa.

itermonthdays(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão simplesmente os números dos dias do mês. Para os dias fora do mês especificado, o número do dia será `0`.

itermonthdays2(*year*, *month*)

Return an iterator for the month *month* in the year *year* similar to `itermonthdates()`, but not restricted by the `datetime.date` range. Days returned will be tuples consisting of a day of the month number and a weekday number.

itermonthdays3(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão tuplas consistindo de números de um ano, um mês e um dia do mês.

Adicionado na versão 3.7.

itermonthdays4(*year*, *month*)

Retorna um iterador para o mês *month* no ano *year* semelhante a `itermonthdates()`, mas não restrito pelo intervalo de `datetime.date`. Os dias retornados serão tuplas consistindo de números de um ano, um mês, um dia do mês e um dia da semana.

Adicionado na versão 3.7.

monthdatescalendar(*year*, *month*)

Retorna uma lista das semanas do mês *month* do *year* como semanas completas. As semanas são listas de sete objetos `datetime.date`.

monthdays2calendar(*year*, *month*)

Retorna uma lista das semanas do mês *month* do ano *year* como semanas completas. As semanas são listas de sete tuplas de números dias e de dias de semanas.

monthdayscalendar(*year*, *month*)

Retorna uma lista das semanas do mês *month* do ano *year* como semanas completas. As semanas são listas de números de sete dias.

yeardatescalendar(*year*, *width=3*)

Retorna os dados para o ano especificado prontos para formatação. O valor de retorno é uma lista de linhas de meses. Cada linha de mês contém até *width* meses (padrão é 3). Cada mês contém entre 4 e 6 semanas, e cada semana contém 1–7 dias. Os dias são objetos `datetime.date`.

yeardays2calendar(*year*, *width=3*)

Retorna os dados para o ano especificado prontos para formatação (semelhante a `yeardatescalendar()`). Entradas nas listas semanais são tuplas de números de dias e números de dias de semana. Números de dias fora deste mês são zero.

yeardayscalendar(*year*, *width=3*)

Retorna a data para o ano especificado prontos para formatação (semelhante a `yeardatescalendar()`). Entradas nas listas de semanas são números de dias. Números de dias fora deste mês são zero.

class calendar.TextCalendar(*firstweekday=0*)

Esta classe pode ser usada para gerar texto plano para calendários.

Instâncias de `TextCalendar` têm os seguintes métodos:

prweek(*theweek*, *width*)

Print a week's calendar as returned by `formatweek()` and without a final newline.

formatday(*theday*, *weekday*, *width*)

Retorna uma string representando um único dia formatado com a largura fornecida em *width*. Se *theday* for `0`, retorna uma string de espaços da largura especificada, representando um dia vazio. O parâmetro *weekday* não é usado.

formatweek(*theweek*, *w=0*)

Retorna uma única semana em uma string sem nova linha. Se *w* for providenciado, isto especifica a largura das colunas de data, que são

centrais. Dependendo do primeiro dia da semana conforme especificado no construtor ou configurado pelo método `setfirstweekday()`.

formatweekday(*weekday*, *width*)

Retorna uma string representando o nome de um único dia da semana formatado para a largura especificada em *width*. O parâmetro *weekday* é um inteiro representando o dia da semana, onde `0` é segunda-feira e `6` é domingo.

formatweekheader(*width*)

Retorna uma string contendo a linha de cabeçalho dos nomes dos dias da semana, formatada com a largura fornecida por *width* para cada coluna. Os nomes dependem das configurações de localidade e são preenchidos até a largura especificada.

formatmonth(*theyear*, *themonth*, *w=0*, *l=0*)

Retorna o calendário do mês em uma string multilinha. Se *w* for providenciado, isto especifica a largura das colunas de data, que são centrais. Se *l* for dado, este especifica o número de linhas que cada semana vai usar. Dependendo do primeiro dia da semana conforme especificado no construtor ou configurado pelo método `setfirstweekday()`.

formatmonthname(*theyear*, *themonth*, *width=0*, *withyear=True*)

Retorna uma string representando o nome do mês centralizado dentro da largura especificada em *width*. Se *withyear* for `True`, inclui o ano na saída. Os parâmetros *theyear* e *themonth* especificam o ano e o mês para o nome a ser formatado, respectivamente.

prmonth(*theyear*, *themonth*, *w=0*, *l=0*)

Imprime um calendário do mês conforme retornado pelo `formatmonth()`.

formatyear(*theyear*, *w=2*, *l=1*, *c=6*, *m=3*)

Retorna um calendário com *m* colunas para um ano inteiro conforme uma string multilinha. Parâmetros opcionais *w*, *l* e *c* definem a largura da coluna data, linhas por semana e números de espaços entre as colunas dos meses, respectivamente. Depende do primeiro dia da semana conforme especificado no construtor ou definido pelo método `setfirstweekday()`. O ano mais novo para o qual o calendário pode ser gerado depende da plataforma.

pryear(*theyear*, *w=2*, *l=1*, *c=6*, *m=3*)

Imprime o calendário para um ano inteiro conforme retornado por `formatyear()`.

class calendar.HTMLCalendar(*firstweekday=0*)

Esta classse pode ser usada para gerar calendários HTML.

Instâncias de **`HTMLCalendar`** têm os seguintes métodos:

formatmonth(*theyear*, *themonth*, *withyear=True*)

Retorna um calendário do mês como uma tabela HTML. Se *withyear* for verdadeiro, o ano será incluído no cabeçalho, senão apenas o nome do mês será utilizado.

formatyear(*theyear*, *width=3*)

Retorna um calendário do ano como uma tabela HTML. *width* (padronizada para 3) especifica o número de meses por linha.

formatyearpage(*theyear*, *width=3*, *css='calendar.css'*, *encoding=None*)

Return a year's calendar as a complete HTML page. *width* (defaulting to 3) specifies the number of months per row. *css* is the name for the cascading style sheet to be used. `None` can be passed if no style sheet should be used. *encoding* specifies the encoding to be used for the output (defaulting to `'utf-8'`).

formatmonthname(*theyear*, *themonth*, *withyear=True*)

Retorna um nome de mês como uma linha de tabela HTML. Se *withyear* for verdadeiro, o ano será incluído na linha, senão apenas o

nome do mês será utilizado.

`HTMLCalendar` tem os seguintes atributos que você pode substituir para personalizar as classes CSS usadas pelo calendário:

cssclasses

Uma lista de classes CSS usadas para cada dia da semana. A lista de classes padrão é:

```
cssclasses = ["mon", "tue", "wed", "thu", "fri", "sat", "s
```

mais estilos podem ser adicionados para cada dia:

```
cssclasses = ["mon text-bold", "tue", "wed", "thu", "fri",
```

Observe que o comprimento desta lista deve ser de sete itens.

cssclass_noday

A classe CSS para um dia da semana que ocorre no mês anterior ou seguinte.

Adicionado na versão 3.7.

cssclasses_weekday_head

Uma lista de classes CSS usadas para nomes de dias da semana na linha de cabeçalho. O padrão é o mesmo que **`cssclasses`**.

Adicionado na versão 3.7.

cssclass_month_head

A classe CSS principal do mês (usada por **`formatmonthname()`**). O valor padrão é `"month"`.

Adicionado na versão 3.7.

cssclass_month

A classe CSS para a tabela do mês inteiro (usada por **formatmonth()**). O valor padrão é `"month"`.

Adicionado na versão 3.7.

cssclass_year

A classe CSS para a tabela de tabelas do ano inteiro (usada por **formatyear()**). O valor padrão é `"year"`.

Adicionado na versão 3.7.

cssclass_year_head

A classe CSS para o cabeçalho da tabela para o ano inteiro (usado por **formatyear()**). O valor padrão é `"year"`.

Adicionado na versão 3.7.

Observe que, embora a nomenclatura dos atributos de classe descritos acima seja singular (por exemplo, `cssclass_month` `cssclass_noday`), é possível substituir a única classe CSS por uma lista de classes CSS separadas por espaços, por exemplo:

```
"text-bold text-red"
```

Aqui está um exemplo de como **HTMLCalendar** pode ser personalizado:

```
class CustomHTMLCal(calendar.HTMLCalendar):
    cssclasses = [style + " text-nowrap" for style in
                  calendar.HTMLCalendar.cssclasses]
    cssclass_month_head = "text-center month-head"
    cssclass_month = "text-center month"
    cssclass_year = "text-italic lead"
```

class calendar.LocaleTextCalendar(*firstweekday=0*, *locale=None*)

Esta subclasse de **TextCalendar** pode receber um nome de localidade no construtor e retornará nomes de meses e dias da semana na localidade especificada.

class calendar.LocaleHTMLCalendar(*firstweekday=0*, *locale=None*)

Esta subclasse de **HTMLCalendar** pode receber um nome de localidade no construtor e retornará nomes de meses e dias da semana na localidade especificada.

Nota

Os métodos construtores **formatweekday()** e **formatmonthname()** dessas duas classes alteram temporariamente a localidade LC_TIME para o *locale* fornecido. Como a localidade atual é uma configuração de todo o processo, eles não são seguros para threads.

Para calendários de texto simples, este módulo fornece as seguintes funções.

calendar.setfirstweekday(*firstweekday*)

Define o dia da semana (0 é segunda-feira, 6 é domingo) para começar cada semana. Os valores **MONDAY**, **TUESDAY**, **WEDNESDAY**, **THURSDAY**, **FRIDAY**, **SATURDAY** e **SUNDAY** são fornecidos para conveniência. Por exemplo, para definir o primeiro dia da semana como domingo:

```
import calendar
calendar.setfirstweekday(calendar.SUNDAY)
```

calendar.firstweekday()

Retorna a configuração atual para o dia da semana que inicia cada semana.

calendar.isleap(*year*)

Retorna **True** se *year* for um ano bissexto, caso contrário, **False**.

calendar.leapdays(*y1*, *y2*)

Retorna o número de anos bissextos no intervalo de *y1* a *y2* (exclusivo), onde *y1* e *y2* são anos.

Esta função funciona para intervalos que abrangem uma mudança de século.

calendar.weekday(*year*, *month*, *day*)

Retorna o dia da semana (`0` é segunda-feira) para *year* (`1970`–…), *month* (`1`–`12`), *day* (`1`–`31`).

calendar.weekheader(*width*)

Return a header containing abbreviated weekday names. *width* specifies the width in characters for one weekday.

calendar.monthrange(*year*, *month*)

Returns weekday of first day of the month and number of days in month, for the specified *year* and *month*.

calendar.monthcalendar(*year*, *month*)

Retorna uma matriz representando o calendário de um mês. Cada linha representa uma semana; dias fora do mês são representados por zeros. Cada semana começa com segunda-feira, a menos que seja definida por `setfirstweekday()`.

calendar.prmonth(*theyear*, *themonth*, *w=0*, *l=0*)

Imprime um calendário do mês conforme retornado pelo `month()`.

calendar.month(*theyear*, *themonth*, *w=0*, *l=0*)

Retorna o calendário de um mês em uma string de várias linhas usando `formatmonth()` da classe `TextCalendar`.

calendar.prcal(*theyear*, *w=0*, *l=0*, *c=6*, *m=3*)

Imprime o calendário para um ano inteiro conforme retornado por `calendar()`.

calendar.calendar(*theyear*, *w=2*, *l=1*, *c=6*, *m=3*)

Retorna um calendário de 3 colunas para um ano inteiro como uma string de várias linhas usando `formatyear()` da classe `TextCalendar`.

calendar.timegm(*tuple*)

An unrelated but handy function that takes a time tuple such as returned by the **gmtime()** function in the **time** module, and returns the corresponding Unix timestamp value, assuming an epoch of 1970, and the POSIX encoding. In fact, **time.gmtime()** and **timegm()** are each other's inverse.

The **calendar** module exports the following data attributes:

calendar.day_name

Uma sequência que representa os dias da semana no local atual, onde Monday é o dia número 0.

```
>>> import calendar
>>> list(calendar.day_name)
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Sat
```

calendar.day_abbr

Uma sequência que representa os dias abreviados da semana no local atual, onde Mon é o dia número 0.

```
>>> import calendar
>>> list(calendar.day_abbr)
['Mon', 'Tue', 'Wed', 'Thu', 'Fri', 'Sat', 'Sun']
```

calendar.MONDAY
calendar.TUESDAY
calendar.WEDNESDAY
calendar.THURSDAY
calendar.FRIDAY
calendar.SATURDAY
calendar.SUNDAY

Apelidos para os dias da semana, onde `MONDAY` é `0` e `SUNDAY` é `6`.

Adicionado na versão 3.12.

class calendar.Day

Enumeração que define os dias da semana como constantes inteiras. Os membros dessa enumeração são exportados para o escopo do módulo

como **MONDAY** até **SUNDAY**.

Adicionado na versão 3.12.

calendar.month_name

Uma sequência que representa os meses do ano na localidade atual. Isso segue a convenção normal de janeiro sendo o mês número 1, então tem um comprimento de 13 e `month_name[0]` é a string vazia.

```
>>> import calendar
>>> list(calendar.month_name)
['', 'January', 'February', 'March', 'April', 'May', 'June',
```

Cuidado

In locales with alternative month names forms, the **`month_name`** sequence may not be suitable when a month name stands by itself and not as part of a date. For instance, in Greek and in many Slavic and Baltic languages, **`month_name`** will produce the month in genitive case. Use **`standalone_month_name`** for a form suitable for standalone use.

calendar.month_abbr

Uma sequência que representa os meses abreviados do ano na localidade atual. Isso segue a convenção normal de janeiro sendo o mês número 1, então tem um comprimento de 13 e `month_abbr[0]` é a string vazia.

```
>>> import calendar
>>> list(calendar.month_abbr)
['', 'Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug',
```

Cuidado

In locales with alternative month names forms, the **`month_abbr`** sequence may not be suitable when a month name stands by itself and not as part of a date. Use **`standalone_month_abbr`** for a form suitable for standalone use.

calendar.standalone_month_name

A sequence that represents the months of the year in the current locale in the standalone form if the locale provides one. Else it is equivalent to **`month_name`**.

Adicionado na versão 3.15.

calendar.standalone_month_abbr

A sequence that represents the abbreviated months of the year in the current locale in the standalone form if the locale provides one. Else it is equivalent to **`month_abbr`**.

Adicionado na versão 3.15.

calendar.JANUARY
calendar.FEBRUARY
calendar.MARCH
calendar.APRIL
calendar.MAY
calendar.JUNE
calendar.JULY
calendar.AUGUST
calendar.SEPTEMBER
calendar.OCTOBER
calendar.NOVEMBER
calendar.DECEMBER

Apelidos para os meses do ano, onde `JANUARY` é `1` e `DECEMBER` é `12`.

Adicionado na versão 3.12.

class calendar.Month

Enumeração que define os meses do ano como constantes inteiras. Os membros dessa enumeração são exportados para o escopo do módulo como **`JANUARY`** até **`DECEMBER`**.

Adicionado na versão 3.12.

The **calendar** module defines the following exceptions:

exception calendar.IllegalMonthError(*month*)

A subclass of **ValueError** and **IndexError**, raised when the given month number is outside of the range 1-12 (inclusive).

Alterado na versão 3.12: **IllegalMonthError** is now also a subclass of **ValueError**. New code should avoid catching **IndexError**.

month

O número de meses inválidos.

exception calendar.IllegalWeekdayError(*weekday*)

Uma subclasse de **ValueError**, levantada quando o número do dia da semana fornecido está fora do intervalo de 0 a 6 (inclusive).

weekday

O número de dias da semana inválidos.

Ver também

Módulo datetime

Interface orientada a objetos para datas e horas com funcionalidade semelhante ao módulo **time**.

Módulo time

Funções de baixo nível relacionadas ao tempo.

Uso na linha de comando

Adicionado na versão 2.5.

The **calendar** module can be executed as a script from the command line to interactively print a calendar.

```
python -m calendar [-h] [-L LOCALE] [-e ENCODING] [-t {text,html
```

Por exemplo, para exibir um calendário para o ano 2000:

```
$ python -m calendar 2000                                    2000
```

As seguintes opções são aceitas:

--help, -h
: Mostra a mensagem de ajuda e sai.

--locale LOCALE, -L LOCALE
: A localidade a ser usada para nomes de meses e dias da semana. O padrão é inglês.

--encoding ENCODING, -e ENCODING
: A codificação a ser usada na saída. `--encoding` é necessário se `--locale` estiver definido.

--type {text,html}, -t {text,html}
: Exibe o calendário no terminal como texto ou como um documento HTML.

--first-weekday FIRST_WEEKDAY, -f FIRST_WEEKDAY
: O dia da semana para começar cada semana. Deve ser um número entre 0 (segunda-feira) e 6 (domingo). O padrão é 0.

  Adicionado na versão 3.13.

year
: O ano para exibir o calendário. O padrão é o ano atual.

month
: The month of the specified `year` to print the calendar for. Must be a number between 1 and 12. Defaults to printing a calendar for the full year.

Opções de modo texto:

--width WIDTH, -w WIDTH

A largura da coluna de data no terminal. A data é exibida centralizada na coluna. Qualquer valor inferior a 2 é ignorado. O padrão é 2.

--lines LINES, -l LINES

O número de linhas por semana no terminal. A data é exibida alinhada no topo. Qualquer valor inferior a 1 é ignorado. O padrão é 1.

--spacing SPACING, -s SPACING

O espaço entre os meses nas colunas. Qualquer valor inferior a 2 é ignorado. O padrão é 6.

--months MONTHS, -m MONTHS

O número de meses exibidos por linha. O padrão é 3.

Alterado na versão 3.14: Por padrão, dia de hoje é destacada em cor e pode ser controlada usando variáveis de ambiente.

Alterado na versão 3.15: By default, the month is now also highlighted in color, and the days of the week are also in color. This behavior can be controlled using environment variables.

Opções do modo HTML:

--css CSS, -c CSS

O caminho de uma folha de estilo CSS a ser usada no calendário. Isso deve ser relativo ao HTML gerado ou a um HTTP absoluto ou a uma URL `file:///`.

`collections` — Tipos de dados de contêineres

Código-fonte: Lib/collections/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/collections/__init__.py]

Este módulo implementa tipos de dados de contêineres especializados que fornecem alternativas aos contêineres embutidos do Python, `dict`, `list`, `set` e `tuple`.

`namedtuple()`	função fábrica (*factory function*) para criar subclasses de tuplas com campos nomeados
`deque`	contêiner lista ou similar com acréscimos e retiradas rápidas nas duas extremidades
`ChainMap`	classe dict ou similar para criar uma visão única de vários mapeamentos
`Counter`	subclasse de dict para contar objetos hasheáveis
`OrderedDict`	subclasse de dict que lembra a ordem que as entradas foram adicionadas
`defaultdict`	subclasse de dict que chama uma função fábrica para fornecer valores não encontrados

`UserDict`	invólucro em torno de objetos dicionário para facilitar fazer subclasse de dict
`UserList`	invólucro em torno de objetos lista para facilitar criação de subclasse de lista
`UserString`	invólucro em torno de objetos string para uma facilitar criação de subclasse de string

Objetos `ChainMap`

Adicionado na versão 3.3.

Uma classe `ChainMap` é fornecida para ligar rapidamente uma série de mapeamentos de forma que possam ser tratados como um só. O que é frequentemente mais rápido do que criar um novo dicionário e executar múltiplas chamadas de `update()`.

A classe pode ser usada para simular escopos aninhados e é útil em modelos.

class collections.ChainMap(**maps*)

`ChainMap` agrupa múltiplos dicts ou outros mapeamentos para criar uma única vista atualizável. Se nenhum *maps* for especificado, um dicionário vazio será fornecido para que uma nova cadeia tenha sempre pelo menos um mapeamento.

Os mapeamentos subjacentes são armazenados em uma lista. Essa lista é pública e pode ser acessada ou atualizada usando o atributo *maps*. Não existe outro estado.

Faz uma busca nos mapeamentos subjacentes sucessivamente até que uma chave seja encontrada. Em contraste, escrita, atualizações e remoções operam apenas no primeiro mapeamento.

Uma **`ChainMap`** incorpora os mapeamentos subjacentes por referência. Então, se um dos mapeamentos subjacentes for atualizado, essas alterações serão refletidas na **`ChainMap`**.

Todos os métodos usuais do dicionário são suportados. Além disso, existe um atributo *maps*, um método para criar novos subcontextos e uma propriedade para acessar todos, exceto o primeiro mapeamento:

maps

Uma lista de mapeamentos atualizáveis pelo usuário. A lista é ordenada do primeiro ao último pesquisado. É o único estado armazenado e pode ser modificado para alterar quais mapeamentos são pesquisados. A lista deve sempre conter pelo menos um mapeamento.

new_child(*m=None*, ***kwargs*)

Retorna uma nova **`ChainMap`** contendo um novo mapa seguido de todos os mapas na instância atual. Se `m` for especificado, torna-se o novo mapa na frente da lista de mapeamentos; Se não especificado, é usado um dicionário vazio, de modo que chamar `d.new_child()` é equivalente a: `ChainMap({}, *d.maps)`. Se algum argumento nomeado for especificado, ele atualizará o mapa passado ou o novo dicionário vazio. Esse método é usado para criar subcontextos que podem ser atualizados sem alterar valores em nenhum dos mapeamentos pai.

Alterado na versão 3.4: O parâmetro opcional `m` foi adicionado.

Alterado na versão 3.10: Suporte a argumentos nomeados foi adicionado.

parents

Propriedade que retorna um novo **`ChainMap`** contendo todos os mapas da instância atual, exceto o primeiro. Isso é útil para pular o primeiro mapa da pesquisa. Os casos de uso são semelhantes aos do argumento nomeado **`nonlocal`** usada em escopos aninhados. Os casos de uso também são paralelos aos da função embutida **`super()`**.

Uma referência a `d.parents` é equivalente a: `ChainMap(*d.maps[1:])`.

Observe, a ordem de iteração de um **`ChainMap`** é determinada pela varredura dos mapeamentos do último ao primeiro:

```
>>> baseline = {'music': 'bach', 'art': 'rembrandt'}
>>> adjustments = {'art': 'van gogh', 'opera': 'carmen'}
>>> list(ChainMap(adjustments, baseline))
['music', 'art', 'opera']
```

Isso dá a mesma ordem de uma série de chamadas de **`dict.update()`** começando com o último mapeamento:

```
>>> combined = baseline.copy()
>>> combined.update(adjustments)
>>> list(combined)
['music', 'art', 'opera']
```

Alterado na versão 3.9: Adicionado suporte para os operadores `|` e `|=`, especificados na **PEP 584** [https://peps.python.org/pep-0584/].

Ver também

- A classe MultiContext [https://github.com/enthought/codetools/blob/4.0.0/codetools/contexts/multi_context.py] no pacote CodeTools [https://github.com/enthought/codetools] de Enthought tem opções para oferecer suporte à escrita em qualquer mapeamento na cadeia.
- A classe Context [https://github.com/django/django/blob/main/django/template/context.py] do Django para modelos é uma cadeia de mapeamentos somente leitura. Ela também oferece o recurso de push e pop (inserir e retirar) contextos semelhantes ao método **`new_child()`** e a propriedade **`parents`**.
- A receita de Contextos Aninhados [https://code.activestate.com/recipes/577434-nested-contexts-a-chain-of-mapping-objects/] possui opções para controlar se escritas e outras mutações se aplicam a apenas o primeiro mapeamento ou para qualquer mapeamento na cadeia.

- Uma versão muito simplificada somente leitura do Chainmap [https://code.activestate.com/recipes/305268/].

Exemplos e Receitas de `ChainMap`

Esta seção mostra várias abordagens para trabalhar com mapas encadeados.

Exemplo de simulação da cadeia de busca interna do Python:

```
import builtins
pylookup = ChainMap(locals(), globals(), vars(builtins))
```

Exemplo de como permitir que os argumentos de linha de comando especificados pelo usuário tenham precedência sobre as variáveis de ambiente que, por sua vez, têm precedência sobre os valores padrão:

```
import os, argparse

defaults = {'color': 'red', 'user': 'guest'}

parser = argparse.ArgumentParser()
parser.add_argument('-u', '--user')
parser.add_argument('-c', '--color')
namespace = parser.parse_args()
command_line_args = {k: v for k, v in vars(namespace).items() if

combined = ChainMap(command_line_args, os.environ, defaults)
print(combined['color'])
print(combined['user'])
```

Padrões de exemplo para utilização da classe `ChainMap` para simular contextos aninhados:

```
c = ChainMap()          # Cria o contexto raiz
d = c.new_child()       # Cria um contexto filho aninhado
e = c.new_child()       # Filho de c, independente de d
e.maps[0]               # Dicionário do contexto atual -- como loc
e.maps[-1]              # Contexto raiz -- como globals() do Pytho
e.parents               # Envolvendo cadeia de contexto -- como os
```

```
d['x'] = 1            # Define valor no contexto atual
d['x']                # Obtém a primeira chave na cadeia de cont
del d['x']            # Exclui do contexto atual
list(d)               # Todos os valores aninhados
k in d                # Verifica todos os valores aninhados
len(d)                # Número de valores aninhados
d.items()             # Todos os itens aninhados
dict(d)               # Achata em um dicionário comum
```

A classe `ChainMap` só faz atualizações (escritas e remoções) no primeiro mapeamento na cadeia, enquanto as pesquisas irão buscar em toda a cadeia. Contudo, se há o desejo de escritas e remoções profundas, é fácil fazer uma subclasse que atualiza chaves encontradas mais a fundo na cadeia:

```
class DeepChainMap(ChainMap):
    'Variant of ChainMap that allows direct updates to inner scc

    def __setitem__(self, key, value):
        for mapping in self.maps:
            if key in mapping:
                mapping[key] = value
                return
        self.maps[0][key] = value

    def __delitem__(self, key):
        for mapping in self.maps:
            if key in mapping:
                del mapping[key]
                return
        raise KeyError(key)

>>> d = DeepChainMap({'zebra': 'black'}, {'elephant': 'blue'}, {
>>> d['lion'] = 'orange'         # atualiza uma chave existente
>>> d['snake'] = 'red'           # novas chaves são adicionadas
>>> del d['elephant']            # remove uma chave existente um
>>> d                            # exibe o resultado
DeepChainMap({'zebra': 'black', 'snake': 'red'}, {}, {'lion': 'c
```

Objetos `Counter`

Uma ferramenta de contagem é fornecida para apoiar contas rápidas e convenientes. Por exemplo:

```
>>> # Conta ocorrências de palavras em uma lista
>>> cnt = Counter()
>>> for word in ['red', 'blue', 'red', 'green', 'blue', 'blue']:
...     cnt[word] += 1
...
>>> cnt
Counter({'blue': 3, 'red': 2, 'green': 1})

>>> # Encontra as dez palavras mais comuns em Hamlet
>>> import re
>>> words = re.findall(r'\w+', open('hamlet.txt').read().lower()
>>> Counter(words).most_common(10)
[('the', 1143), ('and', 966), ('to', 762), ('of', 669), ('i', 63
 ('you', 554),  ('a', 546), ('my', 514), ('hamlet', 471), ('in',
```

class collections.Counter(***kwargs*)

class collections.Counter(*iterable*, /, ***kwargs*)

class collections.Counter(*mapping*, /, ***kwargs*)

Um **`Counter`** é uma subclasse de **`dict`** subclass para contagem de objetos hasheáveis. É uma coleção na qual elementos são armazenados como chaves de dicionário e suas contagens são armazenadas como valores de dicionário. Contagens podem ser qualquer valor inteiro incluindo zero e contagens negativas. A classe **`Counter`** é similar a sacos ou multiconjuntos em outras linguagens.

Os elementos são contados a partir de um iterável *iterable* ou inicializado a partir de um outro mapeamento *mapping* (ou contador):

```
>>> c = Counter()                           # a new, empty co
>>> c = Counter('gallahad')                 # a new counter f
>>> c = Counter({'red': 4, 'blue': 2})      # a new counter f
>>> c = Counter(cats=4, dogs=8)             # a new counter f
```

Objetos Counter tem uma interface de dicionário, com a diferença que devolvem uma contagem zero para itens que não estão presentes em vez

de levantar a exceção **KeyError**:

```
>>> c = Counter(['eggs', 'ham'])
>>> c['bacon']                              # count of a miss
0
```

Definir uma contagem como zero não remove um elemento do contador. Use `del` para o remover completamente.

```
>>> c['sausage'] = 0                        # counter entry w
>>> del c['sausage']                        # del actually re
```

Adicionado na versão 3.1.

Alterado na versão 3.7: Como uma subclasse de **dict**, **Counter** herda a capacidade de lembrar a ordem de inserção. Operações matemáticas em objetos *Counter* também preservam ordem. Os resultados são ordenados de acordo com o momento que um elemento é encontrado pela primeira vez no operando da esquerda e, em seguida, pela ordem encontrada no operando da direita.

Objetos Counter têm suporte a métodos adicionais além daqueles que já estão disponíveis para todos os dicionários:

elements()

Retorna um iterador sobre os elementos, repetindo cada um tantas vezes quanto sua contagem. Os elementos são retornados na ordem em que foram encontrados pela primeira vez. Se a contagem de um elemento é menor que um, ele será ignorado por **elements()** .

```
>>> c = Counter(a=4, b=2, c=0, d=-2)
>>> sorted(c.elements())
['a', 'a', 'a', 'a', 'b', 'b']
```

most_common(*n=None*)

Retorna uma lista dos *n* elementos mais comuns e suas contagens, do mais comum para o menos comum. Se *n* for omitido ou igual a `None`, **most_common()** retorna *todos* os elementos no contador.

Elementos com contagens iguais são ordenados na ordem em que foram encontrados pela primeira vez:

```
>>> Counter('abracadabra').most_common(3)
[('a', 5), ('b', 2), ('r', 2)]
```

subtract(**kwargs)

subtract(*iterable*, /, **kwargs)

subtract(*mapping*, /, **kwargs)

Os elementos são subtraídos de um iterável *iterable* ou de outro mapeamento *mapping* (ou contador). Funciona como `dict.update()`, mas subtraindo contagens ao invés de substituí-las. Tanto as entradas quanto as saídas podem ser zero ou negativas.

```
>>> c = Counter(a=4, b=2, c=0, d=-2)
>>> d = Counter(a=1, b=2, c=3, d=4)
>>> c.subtract(d)
>>> c
Counter({'a': 3, 'b': 0, 'c': -3, 'd': -6})
```

Adicionado na versão 3.2.

total()

Calcula a soma das contagens.

```
>>> c = Counter(a=10, b=5, c=0)
>>> c.total()
15
```

Adicionado na versão 3.10.

The usual dictionary methods are available for `Counter` objects except for these two which work differently for counters:

fromkeys(*iterable*)

Este método de classe não está implementado para objetos `Counter`.

update(**kwargs)

update(*iterable*, /, **kwargs)

update(*mapping*, /, **kwargs)

Elementos são contados a partir de um iterável *iterable* ou adicionados de outro mapeamento *mapping* (ou contador). Funciona como `dict.update()`, mas adiciona contagens em vez de substituí-las. Além disso, é esperado que o *iterable* seja uma sequência de elementos, e não uma sequência de pares `(key, value)`.

Os contadores oferecem suporte a operadores de comparação rica para relacionamentos de igualdade, subconjunto e superconjunto: `==`, `!=`, `<`, `<=`, `>`, `>=`. Todos esses testes tratam os elementos ausentes como tendo contagens zero para que `Counter(a=1) == Counter(a=1, b=0)` retorne verdadeiro.

Alterado na versão 3.10: Foram adicionadas operações de rica comparação.

Alterado na versão 3.10: Em testes de igualdade, os elementos ausentes são tratados como tendo contagens zero. Anteriormente, `Counter(a=3)` e `Counter(a=3, b=0)` eram considerados distintos.

Padrões comuns para trabalhar com objetos `Counter`:

```
c.total()                       # total de todas as contagens
c.clear()                       # redefine todas as contagens
list(c)                         # lista elementos únicos
set(c)                          # converte para um conjunto
dict(c)                         # converte para um dicionário co
c.items()                       # acessa os pares (elem, cnt)
Counter(dict(list_of_pairs))    # converte de uma lista de pares
c.most_common()[:-n-1:-1]       # n últimos elementos comuns
+c                              # remove zero e contagens negati
```

Several mathematical operations are provided for combining `Counter` objects to produce multisets (counters that have counts greater than zero). Addition and subtraction combine counters by adding or subtracting the counts of corresponding elements. Intersection and union return the minimum and maximum of corresponding counts. Symmetric difference returns the difference between the maximum and minimum of the corresponding counts. Equality and inclusion compare corresponding

counts. Each operation can accept inputs with signed counts, but the output will exclude results with counts of zero or below.

```
>>> c = Counter(a=3, b=1)
>>> d = Counter(a=1, b=2)
>>> c + d                       # add two counters together:  c[
Counter({'a': 4, 'b': 3})
>>> c - d                       # subtract (keeping only positiv
Counter({'a': 2})
>>> c & d                       # intersection:  min(c[x], d[x])
Counter({'a': 1, 'b': 1})
>>> c | d                       # union:  max(c[x], d[x])
Counter({'a': 3, 'b': 2})
>>> c ^ d                       # max(c[x], d[x]) - min(c[x], d[
Counter({'a': 2, 'b': 1})
>>> c == d                      # equality:  c[x] == d[x]
False
>>> c <= d                      # inclusion:  c[x] <= d[x]
False
```

A adição e subtração unárias são atalhos para adicionar um contador vazio ou subtrair de um contador vazio.

```
>>> c = Counter(a=2, b=-4)
>>> +c
Counter({'a': 2})
>>> -c
Counter({'b': 4})
```

Adicionado na versão 3.3: Adicionado suporte para operador unário mais, unário menos e operações *in-place* em multiconjuntos.

Adicionado na versão 3.15: Added support for the symmetric difference multiset operation, `c ^ d`.

Nota

Os contadores foram projetados principalmente para funcionar com números inteiros positivos para representar contagens contínuas; no entanto, foi tomado cuidado para não impedir desnecessariamente os

casos de uso que precisassem de outros tipos ou valores negativos. Para ajudar nesses casos de uso, esta seção documenta o intervalo mínimo e as restrições de tipo.

- A própria classe `Counter` é uma subclasse de dicionário sem restrições em suas chaves e valores. Os valores pretendem ser números que representam contagens, mas você *pode* armazenar qualquer coisa no campo de valor.
- O método `most_common()` requer apenas que os valores sejam ordenáveis.
- Para operações in-place, como `c[key] += 1`, o tipo de valor precisa oferecer suporte a apenas adição e subtração. Portanto, frações, números de ponto flutuante e decimais funcionariam e os valores negativos são suportados. O mesmo também é verdadeiro para `update()` e `subtract()` que permitem valores negativos e zero para entradas e saídas.
- Os métodos de multiconjuntos são projetados apenas para casos de uso com valores positivos. As entradas podem ser negativas ou zero, mas apenas saídas com valores positivos são criadas. Não há restrições de tipo, mas o tipo de valor precisa suportar adição, subtração e comparação.
- O método `elements()` requer contagens de inteiros. Ele ignora contagens zero e negativas.

Ver também

- Classe Bag [https://www.gnu.org/software/smalltalk/manual-base/html_node/Bag.html] do Smalltalk.

- Entrada da Wikipédia para Multiconjuntos [https://pt.wikipedia.org/wiki/Multiconjunto].

- Tutorial com exemplos de multiconjuntos no C++ [http://www.java2s.com/Tutorial/Cpp/0380__set-multiset/Catalog0380__set-multiset.htm].

- Para operações matemáticas em multiconjuntos e seus casos de uso, consulte *Knuth, Donald. The Art of Computer Programming Volume*

II, Seção 4.6.3, Exercício 19.

- Para enumerar todos os multiconjuntos distintos de um determinado tamanho em um determinado conjunto de elementos, consulte `itertools.combinations_with_replacement()`:

```
map(Counter, combinations_with_replacement('ABC', 2)) # --
```

Objetos deque

class collections.deque([*iterable*[, *maxlen*]])

Retorna um novo objeto deque inicializado da esquerda para a direita (usando `append()`) com dados do iterável *iterable*. Se *iterable* não for especificado, o novo deque estará vazio.

Deques são uma generalização de pilhas e filas (o nome é pronunciado "deck" e é abreviação de "double-ended queue", conhecida em português como "fila duplamente terminada"). Deques oferecem suporte para inclusões e remoções seguras para thread e eficientes em uso de memória de ambos os lados do deque com aproximadamente o mesmo desempenho $O(1)$ em qualquer direção.

Embora os objetos `list` ofereçam suporte a operações semelhantes, eles são otimizados para operações rápidas de comprimento fixo e sujeitam a custos de movimentação de memória $O(n)$ para as operações `pop(0)` e `insert(0, v)` que alteram o tamanho e a posição da representação de dados subjacente.

Se *maxlen* não for especificado ou for `None`, deques podem crescer para um comprimento arbitrário. Caso contrário, o deque é limitado ao comprimento máximo especificado. Quando um deque de comprimento limitado está cheio, quando novos itens são adicionados, um número correspondente de itens é descartado da extremidade oposta. Deques de comprimento limitado fornecem funcionalidade semelhante ao filtro

`tail` no Unix. Eles também são úteis para rastrear transações e outras pools de dados onde apenas a atividade mais recente é de interesse.

Deques are generic over the type of their contents.

Os objetos deque oferecem suporte aos seguintes métodos:

append(*item*, /)
: Adiciona *item* ao lado direito do deque.

appendleft(*item*, /)
: Adiciona *item* ao lado esquerdo do deque

clear()
: Remove todos os elementos do deque deixando-o com comprimento 0.

copy()
: Cria uma cópia rasa do deque.

Adicionado na versão 3.5.

count(*value*, /)
: Conta o número de elementos em deque igual a *value*.

Adicionado na versão 3.2.

extend(*iterable*, /)
: Estende o lado direito do deque anexando elementos do argumento iterável.

extendleft(*iterable*, /)
: Estende o lado esquerdo do deque anexando elementos de *iterable*. Observe que a série de acréscimos à esquerda resulta na reversão da ordem dos elementos no argumento iterável.

index(*value*[, *start*[, *stop*]])

Retorna a posição de *value* no deque (no ou após o índice *start* e antes do índice *stop*). Retorna a primeira correspondência ou levanta `ValueError` se não for encontrado.

Adicionado na versão 3.5.

insert(*index*, *value*, /)

Insere *value* no deque na posição *index*.

Se a inserção fizer com que um deque limitado cresça além de *maxlen*, uma `IndexError` é levantada.

Adicionado na versão 3.5.

pop()

Remove e devolve um elemento do lado direito do deque. Se nenhum elemento estiver presente, levanta um `IndexError`.

popleft()

Remove e devolve um elemento do lado esquerdo do deque. Se nenhum elemento estiver presente, levanta um `IndexError`.

remove(*value*, /)

Remove a primeira ocorrência de *value*. Se não for encontrado, levanta um `ValueError`.

reverse()

Inverte os elementos do deque no local e, em seguida, retorna `None`.

Adicionado na versão 3.2.

rotate(*n=1*, /)

Gira o deque *n* passos para a direita. Se *n* for negativo, gire para a esquerda.

Quando o deque não está vazio, girar um passo para a direita é equivalente a `d.appendleft(d.pop())` e girar um passo para a esquerda é equivalente a `d.append(d.popleft())`.

Os objetos Deque também fornecem um atributo somente leitura:

maxlen

Tamanho máximo de um deque ou `None` se ilimitado.

Adicionado na versão 3.1.

Além do acima, deques oferecem suporte à iteração, à serialização com pickle, `len(d)`, `reversed(d)`, `copy.copy(d)`, `copy.deepcopy(d)` e ao teste de associação com o operador **`in`** e a referências subscritas, como `d[0]` para acessar o primeiro elemento. O acesso indexado é *O*(1) em ambas as extremidades, mas diminui para *O*(*n*) no meio. Para acesso rápido e aleatório, use listas.

A partir da versão 3.5, deques oferecem suporte a `__add__()`,``__mul__()`` e `__imul__()`.

Exemplo:

```
>>> from collections import deque
>>> d = deque('ghi')                 # torna um novo deque com t
>>> for elem in d:                   # itera os elementos do dec
...     print(elem.upper())
G
H
I

>>> d.append('j')                    # adiciona uma nova entrada
>>> d.appendleft('f')                # adiciona uma nova entrada
>>> d                                # mostra a representação do
deque(['f', 'g', 'h', 'i', 'j'])

>>> d.pop()                          # retorna e remove o item m
'j'
>>> d.popleft()                      # retorna e remove o item m
'f'
```

```
>>> list(d)                          # lista o conteúdo do deque
['g', 'h', 'i']
>>> d[0]                             # exibe o item mais à esque
'g'
>>> d[-1]                            # exibe o item mais à direi
'i'

>>> list(reversed(d))                # lista o conteúdo de um de
['i', 'h', 'g']
>>> 'h' in d                         # pesquisa no deque
True
>>> d.extend('jkl')                  # adiciona vários elementos
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])
>>> d.rotate(1)                      # rotação para direita
>>> d
deque(['l', 'g', 'h', 'i', 'j', 'k'])
>>> d.rotate(-1)                     # rotação para esquerda
>>> d
deque(['g', 'h', 'i', 'j', 'k', 'l'])

>>> deque(reversed(d))               # cria um novo deque na orc
deque(['l', 'k', 'j', 'i', 'h', 'g'])
>>> d.clear()                        # esvazia o deque
>>> d.pop()                          # não é possível retirar um
Traceback (most recent call last):
    File "<pyshell#6>", line 1, in -toplevel-
        d.pop()
IndexError: pop from an empty deque

>>> d.extendleft('abc')              # extendleft() inverte a or
>>> d
deque(['c', 'b', 'a'])
```

Receitas de deque

Esta seção mostra várias abordagens para trabalhar com deques.

Deques de comprimento limitado fornecem funcionalidade semelhante ao filtro `tail` em Unix:

```
def tail(filename, n=10):
    'Return the last n lines of a file'
    with open(filename) as f:
        return deque(f, n)
```

Outra abordagem para usar deques é manter uma sequência de elementos adicionados recentemente, acrescentando à direita e clicando à esquerda:

```
def moving_average(iterable, n=3):
    # moving_average([40, 30, 50, 46, 39, 44]) --> 40.0 42.0 45.
    # https://en.wikipedia.org/wiki/Moving_average
    it = iter(iterable)
    d = deque(itertools.islice(it, n-1))
    d.appendleft(0)
    s = sum(d)
    for elem in it:
        s += elem - d.popleft()
        d.append(elem)
        yield s / n
```

A round-robin scheduler [https://en.wikipedia.org/wiki/Round-robin_scheduling] can be implemented with input iterators stored in a **deque**. Values are yielded from the active iterator in position zero. If that iterator is exhausted, it can be removed with **popleft()**; otherwise, it can be cycled back to the end with the **rotate()** method:

```
def roundrobin(*iterables):
    "roundrobin('ABC', 'D', 'EF') --> A D E B F C"
    iterators = deque(map(iter, iterables))
    while iterators:
        try:
            while True:
                yield next(iterators[0])
                iterators.rotate(-1)
        except StopIteration:
            # Remove um iterador esgotado.
            iterators.popleft()
```

O método **rotate()** fornece uma maneira de implementar o fatiamento e exclusão **deque**. Por exemplo, uma implementação Python pura de `del`

`d[n]` depende do método `rotate()` para posicionar os elementos a serem retirados:

```
def delete_nth(d, n):
    d.rotate(-n)
    d.popleft()
    d.rotate(n)
```

Para implementar o fatiamento de **`deque`**, use uma abordagem semelhante aplicando **`rotate()`** para trazer um elemento alvo para o lado esquerdo do deque. Remova as entradas antigas com **`popleft()`**, adicione novas entradas com **`extend()`**, e, então, inverta a rotação. Com pequenas variações dessa abordagem, é fácil implementar manipulações de pilha de estilo Forth, como `dup`, `drop`, `swap`, `over`, `pick`, `rot` e `roll`.

Objetos `defaultdict`

class collections.defaultdict(*default_factory=None*, /, ***kwargs*)

class collections.defaultdict(*default_factory*, *mapping*, /, ***kwargs*)

class collections.defaultdict(*default_factory*, *iterable*, /, ***kwargs*)

Retorna um novo objeto dicionário ou similar. **`defaultdict`** é uma subclasse da classe embutida **`dict`**. Ele substitui um método e adiciona uma variável de instância gravável. A funcionalidade restante é a mesma da classe **`dict`** e não está documentada aqui.

O primeiro argumento fornece o valor inicial para o atributo **`default_factory`**; o padrão é `None`. Todos os argumentos restantes são tratados da mesma forma como se fossem passados para o construtor **`dict`**, incluindo argumentos nomeados.

`defaultdict`s are generic over two types, signifying (respectively) the types of the dictionary's keys and values.

Os objetos **`defaultdict`** oferecem suporta ao seguinte método além das operações padrão **`dict`**:

__missing__(*key*, /)

Se o atributo `default_factory` for `None`, isso levanta uma exceção `KeyError` com *key* como argumento.

Se `default_factory` não for `None`, ele é chamado sem argumentos para fornecer um valor padrão para a chave *key* fornecida, este valor é inserido no dicionário para *key* e retornado.

Se chamar `default_factory` levanta uma exceção, esta exceção é propagada inalterada.

Este método é chamado pelo método `__getitem__()` da classe `dict` quando a chave solicitada não é encontrada; tudo o que ele retorna ou levanta é então retornado ou levantado por **`__getitem__()`**.

Observe que `__missing__()` *não* é chamado para nenhuma operação além de `__getitem__()`. Isso significa que `get()` irá, como dicionários normais, retornar `None` como padrão ao invés de usar `default_factory`.

Objetos `defaultdict` permitem a seguinte variável de instância:

default_factory

Este atributo é usado pelo método `__missing__()`; ele é inicializado a partir do primeiro argumento para o construtor, se presente, ou para `None`, se ausente.

Alterado na versão 3.9: Adicionado operadores de mesclagem (`|`) e de atualização (`|=`), especificados na **PEP 584** [https://peps.python.org/pep-0584/].

Exemplos de `defaultdict`

Usando `list` como `default_factory`, se for fácil agrupar a sequencia dos pares chave-valores num dicionário de listas

```
>>> s = [('yellow', 1), ('blue', 2), ('yellow', 3), ('blue', 4),
>>> d = defaultdict(list)
>>> for k, v in s:
...     d[k].append(v)
...
```

```
>>> sorted(d.items())
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

Quando cada chave é encontrada pela primeira vez, ela ainda não está no mapeamento; então uma entrada é criada automaticamente usando a função `default_factory` que retorna uma `list` vazia. A operação `list.append()` então anexa o valor à nova lista. Quando as chaves são encontradas novamente, a pesquisa prossegue normalmente (retornando a lista daquela chave) e a operação **`list.append()`** adiciona outro valor à lista. Esta técnica é mais simples e rápida que uma técnica equivalente usando `dict.setdefault()`:

```
>>> d = {}
>>> for k, v in s:
...     d.setdefault(k, []).append(v)
...
>>> sorted(d.items())
[('blue', [2, 4]), ('red', [1]), ('yellow', [1, 3])]
```

Definir `default_factory` como `int` torna `defaultdict` útil para contagem (como um multiconjunto em outras linguagens):

```
>>> s = 'mississippi'
>>> d = defaultdict(int)
>>> for k in s:
...     d[k] += 1
...
>>> sorted(d.items())
[('i', 4), ('m', 1), ('p', 2), ('s', 4)]
```

Quando uma letra é encontrada pela primeira vez, ela está ausente no mapeamento, então a função `default_factory` chama `int()` para fornecer uma contagem padrão de zero. A operação de incremento então cria a contagem para cada letra.

A função `int()` que sempre retorna zero é apenas um caso especial de funções constantes. Uma maneira mais rápida e flexível de criar funções constantes é usar uma função lambda que pode fornecer qualquer valor constante (não apenas zero):

```
>>> def constant_factory(value):
...     return lambda: value
...
>>> d = defaultdict(constant_factory('<missing>'))
>>> d.update(name='John', action='ran')
>>> '%(name)s %(action)s to %(object)s' % d
'John ran to <missing>'
```

Definir `default_factory` como `set` torna `defaultdict` útil para construir um dicionário de conjuntos:

```
>>> s = [('red', 1), ('blue', 2), ('red', 3), ('blue', 4), ('red
>>> d = defaultdict(set)
>>> for k, v in s:
...     d[k].add(v)
...
>>> sorted(d.items())
[('blue', {2, 4}), ('red', {1, 3})]
```

Função de fábrica para tuplas com campos nomeados `namedtuple()`

Tuplas nomeadas determinam o significado de cada posição numa tupla e permitem um código mais legível e autodocumentado. Podem ser usadas sempre que tuplas regulares forem utilizadas, e adicionam a possibilidade de acessar campos pelo nome ao invés da posição do índice.

collections.namedtuple(*typename*, *field_names*, *, *rename=False*, *defaults=None*, *module=None*)

Retorna uma nova subclasse de tupla chamada *typename*. A nova subclasse é usada para criar objetos tupla ou similares que possuem campos acessíveis por pesquisa de atributos, além de serem indexáveis e iteráveis. As instâncias da subclasse também possuem uma docstring útil (com *typename* e *field_names*) e um método útil `__repr__()` que lista o conteúdo da tupla em um formato `nome=valor`.

field_names são uma sequência de strings como `['x', 'y']`. Alternativamente, *field_names* pode ser uma única string com cada nome de campo separado por espaços em branco e/ou vírgulas como, por exemplo, `'x y'` ou `'x, y'`.

Qualquer identificador Python válido pode ser usado para um nome de campo, exceto para nomes que começam com um sublinhado. Identificadores válidos consistem em letras, dígitos e sublinhados, mas não começam com um dígito ou sublinhado e não podem ser uma **`keyword`** como *class*, *for*, *return*, *global*, *pass* ou *raise*.

Se *rename* for verdadeiro, nomes de campos inválidos serão automaticamente substituídos por nomes posicionais. Por exemplo, `['abc', 'def', 'ghi', 'abc']` é convertido para `['abc', '_1', 'ghi', '_3']`, eliminando a palavra reservada `def` e o nome de campo duplicado `abc`.

defaults pode ser `None` ou um iterável de valores padrão. Como os campos com valor padrão devem vir depois de qualquer campo sem padrão, os *padrões* são aplicados aos parâmetros mais à direita. Por exemplo, se os nomes dos campos forem `['x', 'y', 'z']` e os padrões forem `(1, 2)`, então `x` será um argumento obrigatório, `y` será o padrão `1`, e `z` será o padrão `2`.

Se *module* for definido, o atributo **`__module__`** da tupla nomeada será definido com esse valor.

As instâncias de tuplas nomeadas não possuem dicionários por instância, portanto são leves e não requerem mais memória do que as tuplas normais.

Para prover suporte para a serialização com pickle, a classe de tupla nomeada deve ser atribuída a uma variável que corresponda a *typename*.

Alterado na versão 3.1: Adicionado suporte a *rename*.

Alterado na versão 3.6: Os parâmetros *verbose* e *rename* tornaram-se argumentos somente-nomeados.

Alterado na versão 3.6: Adicionado o parâmetro *module*.

Alterado na versão 3.7: Removido o parâmetro *verbose* e o atributo **`_source`**.

Alterado na versão 3.7: Adicionado o parâmetro *defaults* e o atributo **`_field_defaults`**.

```
>>> # Exemplo básico
>>> Point = namedtuple('Point', ['x', 'y'])
>>> p = Point(11, y=22)     # instancia com argumentos nomeados
>>> p[0] + p[1]             # indexável como a tupla plana (11,
33
>>> x, y = p                # desempacota como uma tupla normal
>>> x, y
(11, 22)
>>> p.x + p.y               # campos também acessíveis pelo nome
33
>>> p                       # __repr__ legível com o estilo nome
Point(x=11, y=22)
```

Tuplas nomeadas são especialmente úteis para atribuir nomes de campos a tuplas de resultados retornadas pelos módulos **`csv`** ou **`sqlite3`**:

```
EmployeeRecord = namedtuple('EmployeeRecord', 'name, age, title,

import csv
for emp in map(EmployeeRecord._make, csv.reader(open("employees.
    print(emp.name, emp.title)

import sqlite3
conn = sqlite3.connect('/companydata')
cursor = conn.cursor()
cursor.execute('SELECT name, age, title, department, paygrade FR
for emp in map(EmployeeRecord._make, cursor.fetchall()):
    print(emp.name, emp.title)
```

Além dos métodos herdados das tuplas, as tuplas nomeadas oferecem suporte a três métodos adicionais e dois atributos. Para evitar conflitos com

nomes de campos, os nomes de métodos e atributos começam com um sublinhado.

classmethod somenamedtuple._make(*iterable*, /)

Método de classe que cria uma nova instância a partir de uma sequência existente ou iterável.

```
>>> t = [11, 22]
>>> Point._make(t)
Point(x=11, y=22)
```

somenamedtuple._asdict()

Retorna um novo **dict** que mapeia nomes de campo para seus respectivos valores:

```
>>> p = Point(x=11, y=22)
>>> p._asdict()
{'x': 11, 'y': 22}
```

Alterado na versão 3.1: Retorna um **OrderedDict** em vez de um **dict** normal.

Alterado na versão 3.8: Retorna um **dict** regular em vez de um **OrderedDict**. A partir do Python 3.7, é garantido que os dicionários regulares sejam ordenados. Se os recursos extras de **OrderedDict** forem necessários, a correção sugerida é converter o resultado para o tipo desejado: `OrderedDict(nt._asdict())`.

somenamedtuple._replace(***kwargs*)

Retorna uma nova instância da tupla nomeada substituindo os campos especificados por novos valores:

```
>>> p = Point(x=11, y=22)
>>> p._replace(x=33)
Point(x=33, y=22)

>>> for partnum, record in inventory.items():
...     inventory[partnum] = record._replace(price=newprices|
```

Tuplas nomeadas também são suportados pela função genérica `copy.replace()`.

Alterado na versão 3.13: Levanta `TypeError` em vez de `ValueError` para argumentos nomeados inválidos.

somenamedtuple._fields

Tupla de strings listando os nomes dos campos. Útil para introspecção e para criar novos tipos de tuplas nomeadas a partir de tuplas nomeadas existentes.

```
>>> p._fields            # exibe os nomes de campos
('x', 'y')

>>> Color = namedtuple('Color', 'red green blue')
>>> Pixel = namedtuple('Pixel', Point._fields + Color._fields
>>> Pixel(11, 22, 128, 255, 0)
Pixel(x=11, y=22, red=128, green=255, blue=0)
```

somenamedtuple._field_defaults

Dicionário mapeando nomes de campos para valores padrão.

```
>>> Account = namedtuple('Account', ['type', 'balance'], defa
>>> Account._field_defaults
{'balance': 0}
>>> Account('premium')
Account(type='premium', balance=0)
```

Para recuperar um campo cujo nome está armazenado em uma string, use a função `getattr()`:

```
>>> getattr(p, 'x')
11
```

Para converter um dicionário em uma tupla nomeada, use o operador estrela dupla (conforme descrito em Desempacotando listas de argumentos):

```
>>> d = {'x': 11, 'y': 22}
>>> Point(**d)
```

```
Point(x=11, y=22)
```

Como uma tupla nomeada é uma classe regular do Python, é fácil adicionar ou alterar funcionalidades com uma subclasse. Veja como adicionar um campo calculado e um formato de impressão de largura fixa:

```
>>> class Point(namedtuple('Point', ['x', 'y'])):
...     __slots__ = ()
...     @property
...     def hypot(self):
...         return (self.x ** 2 + self.y ** 2) ** 0.5
...     def __str__(self):
...         return 'Point: x=%6.3f  y=%6.3f  hypot=%6.3f' % (sel

>>> for p in Point(3, 4), Point(14, 5/7):
...     print(p)
Point: x= 3.000  y= 4.000  hypot= 5.000
Point: x=14.000  y= 0.714  hypot=14.018
```

A subclasse mostrada acima define `__slots__` como uma tupla vazia. Isso ajuda a manter baixos os requisitos de memória, evitando a criação de dicionários de instância.

A criação de subclasse não é útil para adicionar novos campos armazenados. Em vez disso, simplesmente crie um novo tipo de tupla nomeado a partir do atributo **`_fields`**:

```
>>> Point3D = namedtuple('Point3D', Point._fields + ('z',))
```

Docstrings podem ser personalizados fazendo atribuições diretas aos campos `__doc__`:

```
>>> Book = namedtuple('Book', ['id', 'title', 'authors'])
>>> Book.__doc__ += ': Hardcover book in active collection'
>>> Book.id.__doc__ = '13-digit ISBN'
>>> Book.title.__doc__ = 'Title of first printing'
>>> Book.authors.__doc__ = 'List of authors sorted by last name'
```

Alterado na versão 3.5: Os docstrings de propriedade tornaram-se graváveis.

Ver também

- Veja `typing.NamedTuple` para uma maneira de adicionar dicas de tipo para tuplas nomeadas. Ele também fornece uma notação elegante usando a palavra reservada `class`:

  ```
  class Component(NamedTuple):
      part_number: int
      weight: float
      description: Optional[str] = None
  ```

- Veja `types.SimpleNamespace()` para um espaço de nomes mutável baseado em um dicionário subjacente em vez de uma tupla.

- O módulo `dataclasses` fornece um decorador e funções para adicionar automaticamente métodos especiais gerados a classes definidas pelo usuário.

Objetos `OrderedDict`

Os dicionários ordenados são como os dicionários normais, mas possuem alguns recursos extras relacionados às operações de pedido. Eles se tornaram menos importantes agora que a classe embutida `dict` ganhou a capacidade de lembrar a ordem de inserção (esse novo comportamento foi garantido no Python 3.7).

Algumas diferenças de `dict` ainda permanecem:

- O `dict` regular foi projetado para ser muito bom em operações de mapeamento. O rastreamento do pedido de inserção era secundário.

- O `OrderedDict` foi projetado para ser bom em operações de reordenação. A eficiência de espaço, a velocidade de iteração e o desempenho das operações de atualização eram secundários.

- O algoritmo `OrderedDict` pode lidar com operações de reordenação frequentes melhor do que `dict`. Conforme mostrado nas receitas abaixo, isso o torna adequado para implementar vários tipos de caches LRU.

- A operação de igualdade para `OrderedDict` verifica a ordem correspondente.

  Um `dict` regular pode emular o teste de igualdade sensível à ordem com `p == q and all(k1 == k2 for k1, k2 in zip(p, q))`.

- O método `popitem()` de `OrderedDict` tem uma assinatura diferente. Ele aceita um argumento opcional para especificar qual item será exibido.

  Um `dict` normal pode emular o `od.popitem(last=True)` do OrderedDict com `d.popitem()` que é garantido para exibir o (último) item mais à direita.

  Um `dict` normal pode emular o `od.popitem(last=False)` do OrderedDict com `(k := next(iter(d)), d.pop(k))` que retornará e remova o item mais à esquerda (primeiro), se existir.

- `OrderedDict` possui um método `move_to_end()` para reposicionar eficientemente um elemento em um endpoint.

  Um `dict` normal pode emular o `od.move_to_end(k, last=True)` do OrderedDict com `d[k] = d.pop(k)` que moverá a chave e seu valor associado para a posição mais à direita (última).

  Um `dict` regular não tem um equivalente eficiente para o `od.move_to_end(k, last=False)` do OrderedDict, que move a chave e seu valor associado para a posição mais à esquerda (primeira).

- Até o Python 3.8, `dict` não tinha um método `__reversed__()`.

class collections.OrderedDict(***kwargs*)

class collections.OrderedDict(*mapping*, /, ***kwargs*)

class collections.OrderedDict(*iterable*, /, ***kwargs*)

Retorna uma instância de uma subclasse `dict` que possui métodos especializados para reorganizar a ordem do dicionário.

Adicionado na versão 3.1.

popitem(*last=True*)

O método `popitem()` para dicionários ordenados retorna e remove um par (chave, valor). Os pares são retornados na ordem LIFO se *last* for verdadeiro ou na ordem FIFO se for falso.

move_to_end(*key*, *last=True*)

Move uma chave *key* existente para qualquer extremidade de um dicionário ordenado. O item é movido para a extremidade direita se *last* for verdadeiro (o padrão) ou para o início se *último* for falso. Levanta `KeyError` se a *key* não existir:

```
>>> d = OrderedDict.fromkeys('abcde')
>>> d.move_to_end('b')
>>> ''.join(d)
'acdeb'
>>> d.move_to_end('b', last=False)
>>> ''.join(d)
'bacde'
```

Adicionado na versão 3.2.

Além dos métodos usuais de mapeamento, dicionários ordenados também oferecem suporte a iteração reversa usando a função `reversed()`.

Testes de igualdade entre objetos `OrderedDict` são sensíveis à ordem e são aproximadamente equivalentes a `list(od1.items())==list(od2.items())`.

Testes de igualdade entre objetos `OrderedDict` e outros objetos `Mapping` não diferenciam a ordem como dicionários regulares. Isso permite que objetos `OrderedDict` sejam substituídos em qualquer lugar que um dicionário regular seja usado.

Alterado na versão 3.5: Os itens, chaves e valores de visões de `OrderedDict` agora oferecem suporte a iteração reversa usando `reversed()`.

Alterado na versão 3.6: Com a aceitação da **PEP 468** [https://peps.python.org/pep-0468/], a ordem é mantida para argumentos nomeados passados para o construtor `OrderedDict` e seu método `update()`.

Alterado na versão 3.9: Adicionado operadores de mesclagem (`|`) e de atualização (`|=`), especificados na **PEP 584** [https://peps.python.org/pep-0584/].

Exemplos e receitas de `OrderedDict`

É simples criar uma variante de dicionário ordenado que lembre a ordem em que as chaves foram inseridas pela *última* vez. Se uma nova entrada substituir uma entrada existente, a posição de inserção original será alterada e movida para o final:

```
class LastUpdatedOrderedDict(OrderedDict):
    'Store items in the order that the keys were last updated.'

    def __setitem__(self, key, value):
        super().__setitem__(key, value)
        self.move_to_end(key)
```

An `OrderedDict` would also be useful for implementing variants of `@functools.lru_cache`:

```
from collections import OrderedDict
from time import monotonic

class TimeBoundedLRU:
    "LRU Cache that invalidates and refreshes old entries."

    def __init__(self, func, maxsize=128, maxage=30):
        self.cache = OrderedDict()      # { args : (timestamp, r
        self.func = func
        self.maxsize = maxsize
        self.maxage = maxage
```

```
    def __call__(self, *args):
        if args in self.cache:
            self.cache.move_to_end(args)
            timestamp, result = self.cache[args]
            if monotonic() - timestamp <= self.maxage:
                return result
        result = self.func(*args)
        self.cache[args] = monotonic(), result
        if len(self.cache) > self.maxsize:
            self.cache.popitem(last=False)
        return result
```

```
class MultiHitLRUCache:
    """ LRU cache that defers caching a result until
        it has been requested multiple times.

        To avoid flushing the LRU cache with one-time requests,
        we don't cache until a request has been made more than c

    """

    def __init__(self, func, maxsize=128, maxrequests=4096, cach
        self.requests = OrderedDict()   # { uncached_key : reque
        self.cache = OrderedDict()      # { cached_key : functic
        self.func = func
        self.maxrequests = maxrequests  # max number of uncachec
        self.maxsize = maxsize          # max number of stored r
        self.cache_after = cache_after

    def __call__(self, *args):
        if args in self.cache:
            self.cache.move_to_end(args)
            return self.cache[args]
        result = self.func(*args)
        self.requests[args] = self.requests.get(args, 0) + 1
        if self.requests[args] <= self.cache_after:
            self.requests.move_to_end(args)
            if len(self.requests) > self.maxrequests:
                self.requests.popitem(last=False)
        else:
            self.requests.pop(args, None)
            self.cache[args] = result
```

```
        if len(self.cache) > self.maxsize:
            self.cache.popitem(last=False)
    return result
```

Objetos `UserDict`

A classe `UserDict` atua como um invólucro em torno de objetos do tipo dicionário. A necessidade desta classe foi parcialmente suplantada pela capacidade de criar subclasses diretamente de `dict`; entretanto, essa classe pode ser mais fácil de trabalhar, pois o dicionário subjacente é acessível como atributo.

class collections.UserDict(**kwargs)

class collections.UserDict(*mapping*, /, **kwargs)

class collections.UserDict(*iterable*, /, **kwargs)

Classe que simula um dicionário. O conteúdo da instância é mantido em um dicionário regular, acessível pelo atributo `data` das instâncias `UserDict`. Se forem fornecidos argumentos, serão usados para inicializar `data` como um dicionário regular.

Além de prover suporte aos métodos e operações de mapeamentos, as instâncias de `UserDict` fornecem o seguinte atributo:

data

Um dicionário real usado para armazenar o conteúdo da classe `UserDict`.

`UserDict` instances also override the following method:

popitem()

Remove and return a `(key, value)` pair from the wrapped dictionary. Pairs are returned in the same order as `data.popitem()`. (For the default `dict.popitem()`, this order is LIFO.) If the dictionary is empty, raises a `KeyError`.

Objetos `UserList`

Esta classe atua como um invólucro em torno de objetos de lista. É uma classe base útil para suas próprias classes semelhantes a listas, que podem herdar delas e substituir métodos existentes ou adicionar novos. Desta forma, é possível adicionar novos comportamentos às listas.

A necessidade desta classe foi parcialmente suplantada pela capacidade de criar subclasses diretamente de `list`; no entanto, pode ser mais fácil trabalhar com essa classe porque a lista subjacente pode ser acessada como um atributo.

class collections.UserList([*list*])

Classe que simula uma lista. O conteúdo da instância é mantido em uma lista regular, que é acessível através do atributo `data` das instâncias `UserList`. O conteúdo da instância é inicialmente definido como uma cópia de *list*, padronizando a lista vazia `[]`. *list* pode ser qualquer iterável, por exemplo, uma lista Python real ou um objeto **`UserList`**.

Além de prover suporte aos métodos e operações de sequências mutáveis, as instâncias `UserList` fornecem o seguinte atributo:

data

Um objeto `list` real usado para armazenar o conteúdo da classe `UserList`.

Requisitos para criar subclasse: Espera-se que as subclasses de `UserList` ofereçam um construtor que pode ser chamado sem argumentos ou com um argumento. Listar operações que retornam uma nova sequência tenta criar uma instância da classe de implementação real. Para isso, presume que o construtor pode ser chamado com um único parâmetro, que é um objeto de sequência usado como fonte de dados.

Se uma classe derivada não desejar atender a este requisito, todos os métodos especiais suportados por esta classe precisarão ser substituídos; consulte as fontes para obter informações sobre os métodos que precisam ser fornecidos nesse caso.

Objetos `UserString`

A classe `UserString` atua como um invólucro em torno de objetos string. A necessidade desta classe foi parcialmente suplantada pela capacidade de criar subclasses diretamente de `str`; entretanto, essa classe pode ser mais fácil de trabalhar porque a string subjacente é acessível como um atributo.

class collections.UserString(*seq*)

Classe que simula um objeto string. O conteúdo da instância é mantido em um objeto string regular, que é acessível através do atributo `data` das instâncias `UserString`. O conteúdo da instância é inicialmente definido como uma cópia de *seq*. O argumento *seq* pode ser qualquer objeto que possa ser convertido em uma string usando a função embutida `str()`.

Além de prover suporte aos métodos e operações de strings, as instâncias `UserString` fornecem o seguinte atributo:

data

Um objeto `str` real usado para armazenar o conteúdo da classe `UserString`.

Alterado na versão 3.5: Novos métodos `__getnewargs__`, `__rmod__`, `casefold`, `format_map`, `isprintable` e `maketrans`.

`collections.abc` — Classes Base Abstratas para Contêineres

Adicionado na versão 3.3: Anteriormente, esse módulo fazia parte do módulo `collections`.

Código-fonte: Lib/_collections_abc.py
[https://github.com/python/cpython/tree/3.15/Lib/_collections_abc.py]

Esse módulo fornece classes base abstratas que podem ser usadas para testar se uma classe fornece uma interface específica; por exemplo, se é hasheável ou se é um mapeamento.

Um teste `issubclass()` ou `isinstance()` para uma interface funciona em uma das três formas.

1. Uma classe recém-escrita pode herdar diretamente de uma das classes base abstratas. A classe deve fornecer os métodos abstratos necessários. Os métodos mixin restantes vêm da herança e podem ser substituídos se desejado. Outros métodos podem ser adicionados conforme necessário:

```
class C(Sequence):                          # Herança direta
    def __init__(self): ...                 # Método extra não e
    def __getitem__(self, index):  ...      # Método abstrato ex
    def __len__(self):  ...                 # Método abstrato ex
    def count(self, value): ...             # Opcionalmente subs
```

```
>>> issubclass(C, Sequence)
True
>>> isinstance(C(), Sequence)
True
```

2. Classes existentes e classes embutidas podem ser registradas como "subclasses virtuais" dos ABCs. Essas classes devem definir a API completa, incluindo todos os métodos abstratos e todos os métodos mixin. Isso permite que os usuários confiem nos testes **`issubclass()`** ou **`isinstance()`** para determinar se a interface completa é suportada. A exceção a essa regra é para métodos que são automaticamente inferidos do restante da API:

```
class D:                                    # Sem herança
    def __init__(self): ...                 # Método extra exig
    def __getitem__(self, index):  ...      # Método abstrato
    def __len__(self):  ...                 # Método abstrato
    def count(self, value): ...             # Método mixin
    def index(self, value): ...             # Método mixin

Sequence.register(D)                        # Registra ao invés
```

```
>>> issubclass(D, Sequence)
True
>>> isinstance(D(), Sequence)
True
```

Neste exemplo, a classe **`D`** não precisa definir `__contains__`, `__iter__` e `__reversed__` porque o operador in, a lógica iteration e a função **`reversed()`** retornam automaticamente para o uso de `__getitem__` e `__len__`.

3. Algumas interfaces simples são diretamente reconhecíveis pela presença dos métodos necessários (a menos que esses métodos tenham sido definidos como **`None`**):

```
class E:
    def __iter__(self): ...
    def __next__(self): ...

>>> issubclass(E, Iterable)
True
>>> isinstance(E(), Iterable)
True
```

Interfaces complexas não oferecem suporte a esta última técnica porque uma interface é mais do que apenas a presença de nomes de métodos. Interfaces especificam semântica e relacionamentos entre métodos que não podem ser inferidos somente da presença de nomes de métodos específicos. Por exemplo, saber que uma classe fornece `__getitem__`, `__len__` e `__iter__` é insuficiente para distinguir uma **`Sequence`** de uma **`Mapping`**.

Adicionado na versão 3.9: Essas classes abstratas agora oferecem suporte a `[]`. Veja Tipo Generic Alias e **PEP 585** [https://peps.python.org/pep-0585/].

Classes Base Abstratas de Coleções

O módulo de coleções oferece o seguinte ABCs:

ABC	Herda de	Métodos abstratos	Métodos mixin
`Container` [[1]]		`__contains__`	
`Hashable` [[1]]		`__hash__`	
`Iterable` [[1]] [[2]]		`__iter__`	
`Iterator` [[1]]	**`Iterable`**	`__next__`	`__iter__`
`Reversible` [[1]]	**`Iterable`**	`__reversed__`	

ABC	Herda de	Métodos abstratos	Métodos mixin
Generator [[1]]	Iterator	send, throw	close, __iter__, __next__
Sized [[1]]		__len__	
Callable [[1]]		__call__	
Collection [[1]]	Sized, Iterable, Container	__contains__, __iter__, __len__	
Sequence	Reversible, Collection	__getitem__, __len__	__contains__, __iter__, __reversed__, index, and count
MutableSequence	Sequence	__getitem__, __setitem__, __delitem__, __len__, insert	Herdados métodos de Sequence e append, clear, reverse, extend, pop, remove e __iadd__
ByteString	Sequence	__getitem__, __len__	Métodos herdados de Sequence

ABC	Herda de	Métodos abstratos	Métodos mixin
Set	Collection	__contains__, __iter__, __len__	__le__, __lt__, __eq__, __ne__, __gt__, __ge__, __and__, __or__, __sub__, __rsub__, __xor__, __rxor__ e isdisjoint
MutableSet	Set	__contains__, __iter__, __len__, add, discard	Herdado Set métodos e clear, pop, remove, __ior__, __iand__, __ixor__, e __isub__
Mapping	Collection	__getitem__, __iter__, __len__	__contains__, keys, items, values, get, __eq__, e __ne__
MutableMapping	Mapping	__getitem__, __setitem__, __delitem__, __iter__, __len__	Herdado Mapping métodos e pop, popitem, clear, update, e setdefault
MappingView	Sized		__init__, __len__ e __repr__

ABC	Herda de	Métodos abstratos	Métodos mixin
ItemsView	MappingView, Set		__contains__, __iter__
KeysView	MappingView, Set		__contains__, __iter__
ValuesView	MappingView, Collection		__contains__, __iter__
Awaitable [[1]]		__await__	
Coroutine [[1]]	Awaitable	send, throw	close
AsyncIterable [[1]]		__aiter__	
AsyncIterator [[1]]	AsyncIterable	__anext__	__aiter__
AsyncGenerator [[1]]	AsyncIterator	asend, athrow	aclose, __aiter__, __anext__
Buffer [[1]]		__buffer__	

Notas de rodapé

[1] (1,2,3,4,5,6,7,8,9,10,11,12,13,14,15)

Essas ABCs substituem `__subclasshook__()` para dar suporte ao teste de uma interface verificando se os métodos necessários estão presentes e não foram definidos como `None`. Isso só funciona para interfaces simples. Interfaces mais complexas exigem registro ou subclassificação direta.

[2]

A verificação `isinstance(obj, Iterable)` detecta classes que são registradas como `Iterable` ou que possuem um método `__iter__()`, mas que não detecta classes que iteram com o método `__getitem__()`. A única maneira confiável de determinar se um objeto é iterável é chamar `iter(obj)`.

Classes Base Abstrata de Coleções – Descrições Detalhadas

class collections.abc.Container

ABC para classes que fornecem o método `__contains__()`.

class collections.abc.Hashable

ABC para classes que fornecem o método `__hash__()`.

class collections.abc.Sized

ABC para classes que fornecem o método `__len__()`.

class collections.abc.Callable

ABC para classes que fornecem o método `__call__()`.

Veja Anotações de objetos chamáveis para detalhes sobre como usar `Callable` em anotações de tipos.

class collections.abc.Iterable

ABC para classes que fornecem o método `__iter__()`.

A verificação `isinstance(obj, Iterable)` detecta classes que são registradas como **`Iterable`** ou que possuem um método **`__iter__()`**, mas que não detecta classes que iteram com o método **`__getitem__()`**. A única maneira confiável de determinar se um objeto é iterável é chamar `iter(obj)`.

class collections.abc.Collection

ABC para classes de contêiner iterável de tamanho.

Adicionado na versão 3.6.

class collections.abc.Iterator

ABC para classes que fornecem os métodos **`__iter__()`** e métodos **`__next__()`**. Veja também a definição de iterator.

class collections.abc.Reversible

ABC para classes iteráveis que também fornecem o método **`__reversed__()`**.

Adicionado na versão 3.6.

class collections.abc.Generator

ABC para classes geradores que implementam o protocolo definido em **PEP 342** [https://peps.python.org/pep-0342/] que estende os iteradores com os métodos **`send()`**, **`throw()`** e **`close()`**.

Consulte Anotando geradores e corrotinas para obter detalhes sobre como usar **`Generator`** em anotações de tipos.

Adicionado na versão 3.5.

class collections.abc.Sequence
class collections.abc.MutableSequence
class collections.abc.ByteString

ABCs para sequências somente de leitura e mutáveis.

Nota de implementação: Alguns métodos mixin, como **`__iter__()`**, **`__reversed__()`** e **`index()`**, fazem chamadas repetidas ao método

subjacente **__getitem__()**. Consequentemente, se **__getitem__()** for implementado com velocidade de acesso constante, os métodos mixin terão desempenho linear; no entanto, se o método subjacente for linear (como seria com uma lista encadeada), os mixins terão desempenho quadrático e provavelmente precisarão ser substituídos.

index(*value*, *start=0*, *stop=None*)

Retorna o primeiro índice de *valor*.

Levanta **ValueError** se o valor não estiver presente.

Suporte aos argumentos *start* e *stop* é opcional, mas recomendado.

Alterado na versão 3.5: O método **index()** ganhou suporte para os argumentos *stop* e *start*.

Descontinuado desde a versão 3.12, será removido na versão 3.17: A ABC **ByteString** foi descontinuada.

Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **Buffer** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

ByteString foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **bytes** e **bytearray**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **ByteString** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **memoryview**, também nunca foram entendidos como subtipos de **ByteString** (seja em tempo de execução ou por verificadores de tipo estáticos).

Consulte **PEP** [https://peps.python.org/pep-0688/#current-options] para mais detalhes.

class collections.abc.Set

class collections.abc.MutableSet

ABCs para conjuntos somente leitura e mutáveis.

class collections.abc.Mapping
class collections.abc.MutableMapping

ABCs para somente leitura e mutável mappings.

class collections.abc.MappingView
class collections.abc.ItemsView
class collections.abc.KeysView
class collections.abc.ValuesView

ABCs para mapeamento, itens, chaves e valores views.

class collections.abc.Awaitable

ABC para objetos aguardáveis, que podem ser usados em expressões de `await`. Implementações personalizadas devem fornecer o método `__await__()`.

Objetos e instâncias de corrotina da ABC `Coroutine` são todas instâncias dessa ABC.

Nota

No CPython, as corrotinas baseados em gerador (geradores decorados com `@types.coroutine`) são *aguardáveis*, embora não possuam o método `__await__()`. Usar `isinstance(gencoro, Awaitable)` para eles retornará `False`. Use `inspect.isawaitable()` para detectá-los.

Adicionado na versão 3.5.

class collections.abc.Coroutine

ABC para classes compatíveis com corrotina. Eles implementam os seguintes métodos, definidos em Objetos corrotina: `send()`, `throw()`, e `close()`. Implementações personalizadas também devem implementar `__await__()`. Todas as instâncias `Coroutine` também são instâncias de `Awaitable`.

Nota

No CPython, as corrotinas baseados em gerador (geradores decorados com `@types.coroutine`) são *aguardáveis*, embora não possuam o método `__await__()`. Usar `isinstance(gencoro, Coroutine)` para eles retornará `False`. Use `inspect.isawaitable()` para detectá-los.

Consulte Anotando geradores e corrotinas para obter detalhes sobre como usar **`Coroutine`** em anotações de tipos. A variância e a ordem dos parâmetros de tipo correspondem às de **`Generator`**.

Adicionado na versão 3.5.

class collections.abc.AsyncIterable

ABC para classes que fornecem um método `__aiter__`. Veja também a definição de iterável assíncrono.

Adicionado na versão 3.5.

class collections.abc.AsyncIterator

ABC para classes que fornecem os métodos `__aiter__` e `__anext__`. Veja também a definição de iterador assíncrono.

Adicionado na versão 3.5.

class collections.abc.AsyncGenerator

ABC para classes de gerador assíncrono que implementam o protocolo definido em **PEP 525** [https://peps.python.org/pep-0525/] e **PEP 492** [https://peps.python.org/pep-0492/].

Consulte Anotando geradores e corrotinas para obter detalhes sobre como usar **`AsyncGenerator`** em anotações de tipos.

Adicionado na versão 3.6.

class collections.abc.Buffer

ABC para classes que fornecem o método `__buffer__()`, implementando o protocolo buffer. Veja **PEP 688** [https://peps.python.org/pep-0688/].

Adicionado na versão 3.12.

Exemplos e receitas

ABCs nos permitem perguntar a classes ou instâncias se elas fornecem funcionalidades específicas, por exemplo:

```
size = None
if isinstance(myvar, collections.abc.Sized):
    size = len(myvar)
```

Vários ABCs também são também úteis como mixins que facilitam o desenvolvimento de classes que suportam APIs de contêiner. Por exemplo, para escrever uma classe que suporte toda a API `Set` , é necessário fornecer apenas os três métodos abstratos subjacentes: `__contains__()`, `__iter__()`, e `__len__()`. O ABC fornece os métodos restantes, como `__and__()` e `isdisjoint()`:

```
class ListBasedSet(collections.abc.Set):
    ''' Alternate set implementation favoring space over speed
        and not requiring the set elements to be hashable. '''
    def __init__(self, iterable):
        self.elements = lst = []
        for value in iterable:
            if value not in lst:
                lst.append(value)

    def __iter__(self):
        return iter(self.elements)

    def __contains__(self, value):
        return value in self.elements

    def __len__(self):
        return len(self.elements)

s1 = ListBasedSet('abcdef')
s2 = ListBasedSet('defghi')
overlap = s1 & s2            # O método __and__() é automaticame
```

Notas sobre o uso de **`Set`** e **`MutableSet`** como um mixin:

1. Como algumas operações de conjunto criam novos conjuntos, os métodos de mixin padrão precisam de uma maneira de criar novas instâncias a partir de um iterável. Supõe-se que a classe construtor tenha uma assinatura no formato `ClassName(iterable)`. Essa suposição é fatorada em um **`classmethod`** interno chamado **`_from_iterable()`** que chama `cls(iterable)` para produzir um novo conjunto. Se o mixin **`Set`** estiver sendo usado em uma classe com uma assinatura de construtor diferente, você precisará substituir **`_from_iterable()`** por um método de classe ou um método regular que possa construir novas instâncias a partir de um argumento iterável.
2. Para substituir as comparações (presumivelmente para velocidade, já que a semântica é fixa), redefina **`__le__()`** e **`__ge__()`**, então as outras operações seguirão o exemplo automaticamente.
3. O mixin **`Set`** fornece um método **`_hash()`** para calcular um valor de hash para o conjunto; no entanto, **`__hash__()`** não é definido porque nem todos os conjuntos são hasheáveis ou imutáveis. Para adicionar hasheabilidade em conjuntos usando mixin, herde de ambos **`Set`** e **`Hashable`**, e então defina `__hash__ = Set._hash`.

Ver também

- OrderedSet receita [https://code.activestate.com/recipes/576694/] para um exemplo baseado em **`MutableSet`**.
- Para mais informações sobre ABCs, consulte o módulo **`abc`** e **PEP 3119** [https://peps.python.org/pep-3119/].

`heapq` — Algoritmo de fila heap

Código-fonte: Lib/heapq.py [https://github.com/python/cpython/tree/3.15/Lib/heapq.py]

Este módulo fornece uma implementação do algoritmo de fila heap, também conhecido como algoritmo de fila de prioridade.

Min-heaps are binary trees for which every parent node has a value less than or equal to any of its children. We refer to this condition as the heap invariant.

For min-heaps, this implementation uses lists for which `heap[k] <= heap[2*k+1]` and `heap[k] <= heap[2*k+2]` for all *k* for which the compared elements exist. Elements are counted from zero. The interesting property of a min-heap is that its smallest element is always the root, `heap[0]`.

Max-heaps satisfy the reverse invariant: every parent node has a value *greater* than any of its children. These are implemented as lists for which `maxheap[2*k+1] <= maxheap[k]` and `maxheap[2*k+2] <= maxheap[k]` for all *k* for which the compared elements exist. The root, `maxheap[0]`, contains the *largest* element; `heap.sort(reverse=True)` maintains the max-heap invariant.

The **`heapq`** API differs from textbook heap algorithms in two aspects: (a) We use zero-based indexing. This makes the relationship between the index for a node and the indexes for its children slightly less obvious, but is more suitable since Python uses zero-based indexing. (b) Textbooks often focus on max-heaps, due to their suitability for in-place sorting. Our implementation favors min-heaps as they better correspond to Python **`lists`**.

These two aspects make it possible to view the heap as a regular Python list without surprises: `heap[0]` is the smallest item, and `heap.sort()` maintains

the heap invariant!

Like `list.sort()`, this implementation uses only the `<` operator for comparisons, for both min-heaps and max-heaps.

In the API below, and in this documentation, the unqualified term *heap* generally refers to a min-heap. The API for max-heaps is named using a `_max` suffix.

To create a heap, use a list initialized as `[]`, or transform an existing list into a min-heap or max-heap using the `heapify()` or `heapify_max()` functions, respectively.

The following functions are provided for min-heaps:

heapq.heapify(*x*)

Transform list *x* into a min-heap, in-place, in linear time.

heapq.heappush(*heap*, *item*)

Push the value *item* onto the *heap*, maintaining the min-heap invariant.

heapq.heappop(*heap*)

Pop and return the smallest item from the *heap*, maintaining the min-heap invariant. If the heap is empty, `IndexError` is raised. To access the smallest item without popping it, use `heap[0]`.

heapq.heappushpop(*heap*, *item*)

Coloca *item* no heap, depois retira e retorna o menor item do *heap*. A ação combinada é executada com mais eficiência do que `heappush()` seguida por uma chamada separada para `heappop()`.

heapq.heapreplace(*heap*, *item*)

Abre e retorna o menor item da *heap* e também coloca o novo *item*. O tamanho do heap não muda. Se o heap estiver vazio, a exceção `IndexError` será levantada.

Esta operação de uma etapa é mais eficiente que `heappop()` seguida por `heappush()` e pode ser mais apropriada ao usar um heap de tamanho fixo. A combinação de retirar/colocar sempre retorna um elemento do heap e o substitui por *item*.

O valor retornado pode ser maior que o *item* adicionado. Se isso não for desejado, considere usar `heappushpop()`. Sua combinação de retirar/colocar retorna o menor dos dois valores, deixando o valor maior no heap.

For max-heaps, the following functions are provided:

heapq.heapify_max(*x*)

Transform list *x* into a max-heap, in-place, in linear time.

Adicionado na versão 3.14.

heapq.heappush_max(*heap*, *item*)

Push the value *item* onto the max-heap *heap*, maintaining the max-heap invariant.

Adicionado na versão 3.14.

heapq.heappop_max(*heap*)

Pop and return the largest item from the max-heap *heap*, maintaining the max-heap invariant. If the max-heap is empty, `IndexError` is raised. To access the largest item without popping it, use `maxheap[0]`.

Adicionado na versão 3.14.

heapq.heappushpop_max(*heap*, *item*)

Push *item* on the max-heap *heap*, then pop and return the largest item from *heap*. The combined action runs more efficiently than `heappush_max()` followed by a separate call to `heappop_max()`.

Adicionado na versão 3.14.

heapq.heapreplace_max(*heap*, *item*)

Pop and return the largest item from the max-heap *heap* and also push the new *item*. The max-heap size doesn't change. If the max-heap is empty, `IndexError` is raised.

The value returned may be smaller than the *item* added. Refer to the analogous function `heapreplace()` for detailed usage notes.

Adicionado na versão 3.14.

O módulo também oferece três funções de propósito geral baseadas em heaps.

heapq.merge(**iterables*, *key=None*, *reverse=False*)

Mescla diversas entradas classificadas em uma única saída classificada (por exemplo, mescla entradas com registro de data e hora de vários arquivos de log). Retorna um iterador sobre os valores classificados.

Semelhante a `sorted(itertools.chain(*iterables))` mas retorna um iterável, não puxa os dados para a memória todos de uma vez e presume que cada um dos fluxos de entrada já está classificado (do menor para o maior).

Possui dois argumentos opcionais que devem ser especificados como argumentos nomeados.

key especifica uma função chave de um argumento que é usado para extrair uma chave de comparação de cada elemento de entrada. O valor padrão é `None` (compare os elementos diretamente).

reverse é um valor booleano. Se definido como `True`, então os elementos de entrada serão mesclados como se cada comparação fosse invertida. Para obter um comportamento semelhante a `sorted(itertools.chain(*iterables), reverse=True)`, todos os iteráveis devem ser classificados do maior para o menor.

Alterado na versão 3.5: Adicionados os parâmetros opcionais *key* e *reverse*.

heapq.nlargest(*n*, *iterable*, *key=None*)

Retorna uma lista com os *n* maiores elementos do conjunto de dados definido por *iterable*. *key*, se fornecido, especifica uma função de um argumento que é usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). Equivalente a: `sorted(iterable, key=key, reverse=True)[:n]`.

heapq.nsmallest(*n*, *iterable*, *key=None*)

Retorna uma lista com os *n* menores elementos do conjunto de dados definido por *iterable*. *key*, se fornecido, especifica uma função de um argumento que é usado para extrair uma chave de comparação de cada elemento em *iterable* (por exemplo, `key=str.lower`). Equivalente a: `sorted(iterable, key=key)[:n]`.

As duas últimas funções têm melhor desempenho para valores menores de *n*. Para valores maiores, é mais eficiente usar a função **`sorted()`**. Além disso, quando `n==1`, é mais eficiente usar as funções embutidas **`min()`** e **`max()`**. Se for necessário o uso repetido dessas funções, considere transformar o iterável em um heap real.

Exemplos básicos

Um heapsort [https://en.wikipedia.org/wiki/Heapsort] pode ser implementado colocando todos os valores em um heap e, em seguida, retirando os menores valores, um de cada vez:

```
>>> def heapsort(iterable):
...     h = []
...     for value in iterable:
...         heappush(h, value)
...     return [heappop(h) for i in range(len(h))]
...
>>> heapsort([1, 3, 5, 7, 9, 2, 4, 6, 8, 0])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Isto é semelhante a `sorted(iterable)`, mas diferente de **`sorted()`**, esta implementação não é estável.

Os elementos de heap podem ser tuplas. Isto é útil para atribuir valores de comparação (como prioridades de tarefas) juntamente com o registro principal que está sendo rastreado:

```
>>> h = []
>>> heappush(h, (5, 'write code'))
>>> heappush(h, (7, 'release product'))
>>> heappush(h, (1, 'write spec'))
>>> heappush(h, (3, 'create tests'))
>>> heappop(h)
(1, 'write spec')
```

Other Applications

Medians [https://en.wikipedia.org/wiki/Median] are a measure of central tendency for a set of numbers. In distributions skewed by outliers, the median provides a more stable estimate than an average (arithmetic mean). A running median is an online algorithm [https://en.wikipedia.org/wiki/Online_algorithm] that updates continuously as new data arrives.

A running median can be efficiently implemented by balancing two heaps, a max-heap for values at or below the midpoint and a min-heap for values above the midpoint. When the two heaps have the same size, the new median is the average of the tops of the two heaps; otherwise, the median is at the top of the larger heap:

```
def running_median(iterable):
    "Yields the cumulative median of values seen so far."

    lo = []  # max-heap
    hi = []  # min-heap (same size as or one smaller than lo)

    for x in iterable:
        if len(lo) == len(hi):
            heappush_max(lo, heappushpop(hi, x))
            yield lo[0]
        else:
            heappush(hi, heappushpop_max(lo, x))
            yield (lo[0] + hi[0]) / 2
```

Por exemplo:

```
>>> list(running_median([5.0, 9.0, 4.0, 12.0, 8.0, 9.0]))
[5.0, 7.0, 5.0, 7.0, 8.0, 8.5]
```

Notas de implementação da fila de prioridade

Uma fila de prioridade [https://en.wikipedia.org/wiki/Priority_queue] é de uso comum para um heap e apresenta vários desafios de implementação:

- Estabilidade de classificação: como fazer com que duas tarefas com prioridades iguais sejam retornadas na ordem em que foram adicionadas originalmente?
- A comparação de tuplas quebra para pares (prioridade, tarefa) se as prioridades forem iguais e as tarefas não tiverem uma ordem de comparação padrão.
- Se a prioridade de uma tarefa mudar, como movê-la para uma nova posição no heap?
- Ou se uma tarefa pendente precisar ser excluída, como encontrá-la e removê-la da fila?

Uma solução para os dois primeiros desafios é armazenar as entradas como uma lista de 3 elementos, incluindo a prioridade, uma contagem de entradas e a tarefa. A contagem de entradas serve de desempate para que duas tarefas com a mesma prioridade sejam retornadas na ordem em que foram adicionadas. E como não há duas contagens de entradas iguais, a comparação de tuplas nunca tentará comparar diretamente duas tarefas.

Outra solução para o problema de tarefas não comparáveis é criar uma classe wrapper que ignore o item da tarefa e compare apenas o campo de prioridade:

```
from dataclasses import dataclass, field
from typing import Any

@dataclass(order=True)
class PrioritizedItem:
```

```
    priority: int
    item: Any=field(compare=False)
```

Os desafios restantes giram em torno de encontrar uma tarefa pendente e fazer alterações em sua prioridade ou removê-la totalmente. Encontrar uma tarefa pode ser feito com um dicionário apontando para uma entrada na fila.

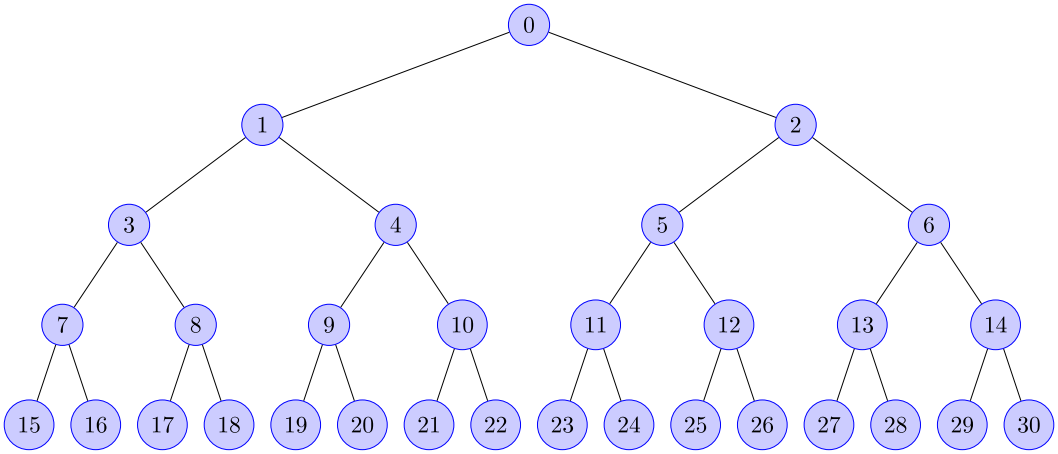
Remover a entrada ou alterar sua prioridade é mais difícil porque quebraria os invariantes da estrutura de heap. Assim, uma possível solução é marcar a entrada como removida e adicionar uma nova entrada com a prioridade revisada:

```
pq = []                         # lista de entradas organizadas
entry_finder = {}               # mapeamentos de tarefas para er
REMOVED = '<removed-task>'      # espaço reservado para uma tare
counter = itertools.count()     # contagem de sequência única

def add_task(task, priority=0):
    'Adiciona uma nova tarefa ou atualiza a prioridade de uma ta
    if task in entry_finder:
        remove_task(task)
    count = next(counter)
    entry = [priority, count, task]
    entry_finder[task] = entry
    heappush(pq, entry)

def remove_task(task):
    'Marca uma tarefa existente com REMOVED (removida). Levanta
    entry = entry_finder.pop(task)
    entry[-1] = REMOVED

def pop_task():
    'Remove e retorna a tarefa de mais baixa prioridade. Levanta
    while pq:
        priority, count, task = heappop(pq)
        if task is not REMOVED:
            del entry_finder[task]
            return task
    raise KeyError('pop from an empty priority queue')
```

Teoria

Heaps são arrays para os quais `a[k] <= a[2*k+1]` e `a[k] <= a[2*k+2]` para todos *k*, contando elementos de 0. Para fins de comparação, os elementos inexistentes são considerados infinitos. A propriedade interessante de um heap é que `a[0]` é sempre seu menor elemento.

The strange invariant above is meant to be an efficient memory representation for a tournament. The numbers below are *k*, not `a[k]`:



Na árvore acima, cada célula *k* está no topo de `2*k+1` e `2*k+2`. Num torneio binário normal que vemos nos desportos, cada célula é a vencedora das duas células que está no topo, e podemos rastrear o vencedor na árvore para ver todos os adversários que teve. Contudo, em muitas aplicações informáticas de tais torneios, não precisamos de traçar a história de um vencedor. Para sermos mais eficientes em termos de memória, quando um vencedor é promovido, tentamos substituí-lo por algo de nível inferior, e a regra passa a ser que uma célula e as duas células que ela cobre contêm três itens diferentes, mas a célula de cima “ganha” sobre as duas células superiores.

Se esse invariante de heap estiver protegido o tempo todo, o índice 0 é claramente o vencedor geral. A maneira algorítmica mais simples de removê-lo e encontrar o “próximo” vencedor é mover algum perdedor (digamos a célula 30 no diagrama acima) para a posição 0 e, em seguida, filtrar esse novo 0 pela árvore, trocando valores, até que o invariante é

restabelecido. Isto é claramente logarítmico do número total de itens na árvore. Ao iterar todos os itens, você obtém uma classificação $O(n \log n)$.

Um recurso interessante desse tipo é que você pode inserir novos itens com eficiência enquanto a classificação está em andamento, desde que os itens inseridos não sejam “melhores” que o último 0º elemento extraído. Isto é especialmente útil em contextos de simulação, onde a árvore contém todos os eventos recebidos e a condição “vitória” significa o menor tempo programado. Quando um evento agenda outros eventos para execução, eles são agendados para o futuro, para que possam entrar facilmente no heap. Portanto, um heap é uma boa estrutura para implementar escalonadores (foi isso que usei no meu sequenciador MIDI :-).

Várias estruturas para implementação de escalonadores foram extensivamente estudadas, e os heaps são bons para isso, pois são razoavelmente rápidos, a velocidade é quase constante e o pior caso não é muito diferente do caso médio. No entanto, existem outras representações que são globalmente mais eficientes, embora os piores casos possam ser terríveis.

Heaps também são muito úteis em classificações de discos grandes. Provavelmente todos vocês sabem que uma classificação grande implica a produção de “execuções” (que são sequências pré-classificadas, cujo tamanho geralmente está relacionado à quantidade de memória da CPU), seguidas de passagens de fusão para essas execuções, cuja fusão geralmente é muito inteligente. organizado [[1]]. É muito importante que a classificação inicial produza as execuções mais longas possíveis. Os torneios são uma boa maneira de conseguir isso. Se, usando toda a memória disponível para realizar um torneio, você substituir e filtrar itens que se encaixem na corrida atual, você produzirá corridas que têm o dobro do tamanho da memória para entradas aleatórias e muito melhores para entradas ordenadas de maneira imprecisa.

Além disso, se você gerar o 0º item no disco e obter uma entrada que pode não caber no torneio atual (porque o valor “ganha” sobre o último valor de saída), ele não poderá caber no heap, então o tamanho do heap diminui. A memória liberada poderia ser reutilizada de maneira inteligente e imediata para a construção progressiva de um segundo heap, que cresce exatamente

na mesma proporção que o primeiro heap está derretendo. Quando o primeiro heap desaparece completamente, você troca os heaps e inicia uma nova execução. Inteligente e bastante eficaz!

Em uma palavra, heaps são estruturas de memória úteis para conhecer. Eu os uso em alguns aplicativos e acho bom manter um módulo “heap” por perto. :-)

Notas de rodapé

[1]

Os algoritmos de balanceamento de disco atuais, hoje em dia, são mais incômodos do que inteligentes, e isso é consequência da capacidade de busca dos discos. Em dispositivos que não são capazes de buscar, como grandes drives de fita, a história era bem diferente, e era preciso ser muito inteligente para garantir (com muita antecedência) que cada movimento da fita seria o mais eficaz possível (isto é, participaria melhor na “progressão” da fusão). Algumas fitas podiam até ser lidas de trás para frente, o que também era usado para evitar o tempo de rebobinar. Acredite em mim, fitas realmente boas eram espetaculares de assistir! Desde sempre, ordenar sempre foi uma Grande Arte! :-)

`bisect` — Algoritmo de bisseção de vetor

Código-fonte: Lib/bisect.py [https://github.com/python/cpython/tree/3.15/Lib/bisect.py]

Este módulo fornece suporte para manter uma lista em ordem de classificação sem ter que classificar a lista após cada inserção. Para longas listas de itens com operações de comparação caras, isso pode ser uma melhoria em relação a pesquisas lineares ou recorrência frequente.

The module is called **`bisect`** because it uses a basic bisection algorithm to do its work. Unlike other bisection tools that search for a specific value, the functions in this module are designed to locate an insertion point. Accordingly, the functions never call an `__eq__()` method to determine whether a value has been found. Instead, the functions only call the `__lt__()` method and will return an insertion point between values in an array.

> **Nota**
>
> The functions in this module are not thread-safe. If multiple threads concurrently use **`bisect`** functions on the same sequence, this may result in undefined behaviour. Likewise, if the provided sequence is mutated by a different thread while a **`bisect`** function is operating on it, the result is undefined. For example, using `insort_left()` on the same list from multiple threads may result in the list becoming unsorted.

As seguintes funções são fornecidas:

bisect.bisect_left(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Localiza o ponto de inserção de *x* em *a* para manter a ordem de classificação. Os parâmetros *lo* e *hi* podem ser usados para especificar

um subconjunto da lista que deve ser considerado; por padrão, toda a lista é usada. Se *x* já estiver presente em *a*, o ponto de inserção estará antes (à esquerda) de qualquer entrada existente. O valor de retorno é adequado para uso como o primeiro parâmetro para `list.insert()` supondo que *a* já esteja ordenado.

O ponto de inserção retornado *ip* particiona o vetor *a* em duas fatias de forma que `all(elem < x for elem in a[lo : ip])` seja verdadeiro para a fatia esquerda e `all(elem >= x for elem in a[ip : hi])` é verdadeiro para a fatia certa.

key especifica uma função chave de um argumento que é usado para extrair uma chave de comparação de cada elemento no vetor. Para oferecer suporte à pesquisa de registros complexos, a função chave não é aplicada ao valor *x*.

Se *key* for `None`, os elementos serão comparados diretamente e nenhuma função chave será chamada.

Alterado na versão 3.10: Adicionado o parâmetro *key*.

bisect.bisect_right(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)
bisect.bisect(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Semelhante a **`bisect_left()`**, mas retorna um ponto de inserção que vem depois (à direita de) qualquer entrada existente de *x* em *a*.

O ponto de inserção retornado *ip* particiona o vetor *a* em duas fatias de forma que `all(elem <= x for elem in a[lo : ip])` seja verdadeiro para a fatia esquerda e `all(elem > x for elem in a[ip : hi])` é verdadeiro para a fatia certa.

Alterado na versão 3.10: Adicionado o parâmetro *key*.

bisect.insort_left(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Insere *x* em *a* na ordem de classificação.

Esta função primeiro executa `bisect_left()` para localizar um ponto de inserção. Em seguida, executa o método `insert()` em *a* para inserir *x* na posição apropriada para manter a ordem de classificação.

Para oferecer suporte à inserção de registros em uma tabela, a função *key* (se houver) é aplicada a *x* para a etapa de pesquisa, mas não para a etapa de inserção.

Tenha em mente que a busca *O*(log *n*) é dominada pelo etapa de inserção lenta O(n).

Alterado na versão 3.10: Adicionado o parâmetro *key*.

bisect.insort_right(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)
bisect.insort(*a*, *x*, *lo=0*, *hi=len(a)*, *, *key=None*)

Semelhante a `insort_left()`, mas inserindo *x* em *a* após qualquer entrada existente de *x*.

Esta função primeiro executa `bisect_right()` para localizar um ponto de inserção. Em seguida, executa o método `insert()` em *a* para inserir *x* na posição apropriada para manter a ordem de classificação.

Para oferecer suporte à inserção de registros em uma tabela, a função *key* (se houver) é aplicada a *x* para a etapa de pesquisa, mas não para a etapa de inserção.

Tenha em mente que a busca *O*(log *n*) é dominada pelo etapa de inserção lenta O(n).

Alterado na versão 3.10: Adicionado o parâmetro *key*.

Observações sobre desempenho

Ao escrever um código sensível ao tempo usando *bisect()* e *insort()*, lembre-se do seguinte:

- A bisseção é eficaz para pesquisar intervalos de valores. Para localizar valores específicos, os dicionários são mais eficientes.
- As funções *insort()* são $O(n)$ porque a etapa de busca logarítmica é dominada pela etapa de inserção de tempo linear.
- The search functions are stateless and discard key function results after they are used. Consequently, if the search functions are used in a loop, the key function may be called again and again on the same array elements. If the key function isn't fast, consider wrapping it with `@functools.cache` to avoid duplicate computations. Alternatively, consider searching an array of precomputed keys to locate the insertion point (as shown in the examples section below).

Ver também

- Sorted Collections [https://grantjenks.com/docs/sortedcollections/] é um módulo de alto desempenho que usa bisseção para gerenciar coleções de dados classificadas.
- A receita de SortedCollection [https://code.activestate.com/recipes/577197-sortedcollection/] usa bisseção para construir uma classe de coleção completa com métodos de busca diretos e suporte para uma função chave. As chaves são pré-calculadas para economizar em chamadas desnecessárias para a função chave durante as buscas.

Buscando em listas ordenadas

As funções de bisseção acima são úteis para encontrar pontos de inserção, mas podem ser complicadas ou difíceis de usar para tarefas comuns de busca. As cinco funções a seguir mostram como transformá-las nas buscas padrão para listas ordenadas:

```
def index(a, x):
    'Encontra o valor mais à esquerda exatamente igual a x'
    i = bisect_left(a, x)
    if i != len(a) and a[i] == x:
        return i
    raise ValueError
```

```
def find_lt(a, x):
    'Encontra o valor mais à direita menor que x'
    i = bisect_left(a, x)
    if i:
        return a[i-1]
    raise ValueError

def find_le(a, x):
    'Encontra o valor mais à direita menor ou igual a x'
    i = bisect_right(a, x)
    if i:
        return a[i-1]
    raise ValueError

def find_gt(a, x):
    'Encontra o valor mais à esquerda maior que x'
    i = bisect_right(a, x)
    if i != len(a):
        return a[i]
    raise ValueError

def find_ge(a, x):
    'Encontra o item mais à esquerda maior ou igual a x'
    i = bisect_left(a, x)
    if i != len(a):
        return a[i]
    raise ValueError
```

Exemplos

A função **`bisect()`** pode ser útil para buscas em tabelas numéricas. Este exemplo usa **`bisect()`** para buscar uma nota em letra para uma pontuação de exame (digamos) com base em um conjunto de pontos de interrupção numéricos ordenados: 90 e acima é um “A”, 80 a 89 é um “B” e por aí vai:

```
>>> def grade(score):
...     i = bisect([60, 70, 80, 90], score)
...     return "FDCBA"[i]
...
```

```
>>> [grade(score) for score in [33, 99, 77, 70, 89, 90, 100]]
['F', 'A', 'C', 'C', 'B', 'A', 'A']
```

As funções **bisect()** e **insort()** também funcionam com listas de tuplas. O argumento *key* pode servir para extrair o campo usado para ordenar registros em uma tabela:

```
>>> from collections import namedtuple
>>> from operator import attrgetter
>>> from bisect import bisect, insort
>>> from pprint import pprint

>>> Movie = namedtuple('Movie', ('name', 'released', 'director')

>>> movies = [
...     Movie('Jaws', 1975, 'Spielberg'),
...     Movie('Titanic', 1997, 'Cameron'),
...     Movie('The Birds', 1963, 'Hitchcock'),
...     Movie('Aliens', 1986, 'Cameron')
... ]

>>> # Encontra o primeiro filme lançado após 1960
>>> by_year = attrgetter('released')
>>> movies.sort(key=by_year)
>>> movies[bisect(movies, 1960, key=by_year)]
Movie(name='The Birds', released=1963, director='Hitchcock')

>>> # Insere um filme enquanto mantém a ordem de classificação
>>> romance = Movie('Love Story', 1970, 'Hiller')
>>> insort(movies, romance, key=by_year)
>>> pprint(movies)
[Movie(name='The Birds', released=1963, director='Hitchcock'),
 Movie(name='Love Story', released=1970, director='Hiller'),
 Movie(name='Jaws', released=1975, director='Spielberg'),
 Movie(name='Aliens', released=1986, director='Cameron'),
 Movie(name='Titanic', released=1997, director='Cameron')]
```

Se a função chave for custosa, é possível evitar chamadas de função repetidas buscando uma lista de chaves pré-calculadas para encontrar o índice de um registro:

```
>>> data = [('red', 5), ('blue', 1), ('yellow', 8), ('black', 0)
>>> data.sort(key=lambda r: r[1])       # Ou use operator.itemge
>>> keys = [r[1] for r in data]         # Pré-calcula uma lista
>>> data[bisect_left(keys, 0)]
('black', 0)
>>> data[bisect_left(keys, 1)]
('blue', 1)
>>> data[bisect_left(keys, 5)]
('red', 5)
>>> data[bisect_left(keys, 8)]
('yellow', 8)
```

`array`— Vetores eficientes de valores numéricos

This module defines an object type which can compactly represent an array of basic values: characters, integers, floating-point numbers, complex numbers. Arrays are mutable sequence types and behave very much like lists, except that the type of objects stored in them is constrained. The type is specified at object creation time by using a *type code*, which is a single character. The following type codes are defined:

Código de tipo	Tipo em C	Tipo em Python	Tamanho mínimo em bytes	Notas
`'b'`	signed char	int	1	
`'B'`	unsigned char	int	1	
`'u'`	wchar_t	Caractere unicode	2	(1)
`'w'`	Py_UCS4	Caractere unicode	4	(2)
`'h'`	signed short	int	2	

Código de tipo	Tipo em C	Tipo em Python	Tamanho mínimo em bytes	Notas
`'H'`	unsigned short	int	2	
`'i'`	signed int	int	2	
`'I'`	unsigned int	int	2	
`'l'`	signed long	int	4	
`'L'`	unsigned long	int	4	
`'q'`	signed long long	int	8	
`'Q'`	unsigned long long	int	8	
`'e'`	_Float16	ponto flutuante	2	(3)
`'f'`	ponto flutuante	ponto flutuante	4	

Código de tipo	Tipo em C	Tipo em Python	Tamanho mínimo em bytes	Notas
`'d'`	double	ponto flutuante	8	
`'Zf'`	float complex	complexo	8	(4)
`'Zd'`	double complex	complexo	16	(4)

Notas:

1. Pode ser de 16 ou 32 bits dependendo da plataforma.

   Alterado na versão 3.9: `array('u')` agora usa **`wchar_t`** como tipo C no lugar do descontinuado `Py_UNICODE`. Essa mudança não afeta o comportamento pois `Py_UNICODE` é um apelido para **`wchar_t`** desde Python 3.3.

   Descontinuado desde a versão 3.3, será removido na versão 3.16: Por favor, migre para o código de tipo `'w'`.

2. *Adicionado na versão 3.13.*

3. The IEEE 754 binary16 “half precision” type was introduced in the 2008 revision of the IEEE 754 standard [https://en.wikipedia.org/wiki/IEEE_754-2008_revision]. This type is not widely supported by C compilers. It’s available as _Float16 type, if the compiler supports the Annex H of the C23 standard.

   Adicionado na versão 3.15.

4. Complex types (`Zf` and `Zd`) are available unconditionally, regardless on support for complex types (the Annex G of the C11 standard) by the C compiler. As specified in the C11 standard, each complex type is represented by a two-element C array containing, respectively, the real and imaginary parts.

   Adicionado na versão 3.15.

Ver também

The ctypes and struct modules, as well as third-party modules like numpy [https://numpy.org/doc/stable/reference/arrays.interface.html#object.__array_interface__], use similar – but slightly different – type codes.

A representação dos valores é definida pela arquitetura da máquina, mais especificamente da implementação do C. O tamanho real pode ser acessado pelo atributo `array.itemsize`.

O módulo define o seguinte item:

array.typecodes

A tuple with all available type codes.

Alterado na versão 3.15: The type changed from `str` to `tuple`.

O módulo define o seguinte tipo:

class array.array(*typecode*[, *initializer*])

Um novo vetor cujos itens são restritos por *typecode* e inicializados a partir do valor opcional *initializer*, que deve ser um objeto `bytes` ou `bytearray`, uma string Unicode ou iterável sobre elementos do tipo apropriado.

Se for fornecido um objeto `bytes` ou `bytearray`, o inicializador é passado para o método `frombytes()` do novo vetor; se for fornecida uma string Unicode, o inicializador é passado para o método

fromunicode(); caso contrário, o iterador do inicializador é passado para o método **extend()** para adicionar itens iniciais ao vetor.

Array objects support the ordinary mutable sequence operations of indexing, slicing, concatenation, and multiplication. When using slice assignment, the assigned value must be an array object with the same type code; in all other cases, **TypeError** is raised. Array objects also implement the buffer interface, and may be used wherever bytes-like objects are supported.

Arrays are generic over the type of their contents.

Levanta um evento de auditoria `array.__new__` com os argumentos `typecode`, `initializer`.

typecode

O caractere de código de tipo usado para criar o vetor.

itemsize

O tamanho em bytes de um item do vetor em representação interna.

append(*value*, /)

Append a new item with the specified value to the end of the array.

buffer_info()

Retorna uma tupla `(address, length)` com o endereço corrente da memória e o tamanho em elementos do buffer usado para armazenar conteúdos do vetor. O tamanho do buffer da memória em bytes pode ser computado como `array.buffer_info()[1] * array.itemsize`. Isso é ocasionalmente útil quando se está trabalhando com interfaces I/O de baixo nível (inerentemente inseguras) que precisam de endereços de memória, como algumas operações **ioctl()**. Os números retornados são válidos enquanto o vetor existir e nenhuma operação de alteração de tamanho for aplicada a ele.

Nota

Quando se está usando vetores de código escrito em C ou C++ (o único jeito efetivo de usar essa informação), faz mais sentido usar a interface do buffer suportada pelos vetores. Esse método é mantido para retrocompatibilidade e deve ser evitado em código novo. A interface de buffers está documentada em Protocolo de buffer.

byteswap()

"Byteswap" all items of the array. This is only supported for values which are 1, 2, 4, 8 or 16 bytes in size; for other types of values, `RuntimeError` is raised. It is useful when reading data from a file written on a machine with a different byte order. Note, that for complex types the order of components (the real part, followed by imaginary part) is preserved.

count(*value*, /)

Return the number of occurrences of *value* in the array.

extend(*iterable*, /)

Acrescenta os itens de *iterable* ao final do vetor. Se *iterable* for outro vetor, ele deve ter *exatamente* o mesmo código de tipo; senão, ocorrerá uma `TypeError`. Se *iterable* não for um vetor, ele deve ser iterável e seus elementos devem ser do tipo correto para ser acrescentado ao vetor.

frombytes(*buffer*, /)

Adiciona itens do objeto bytes ou similar, interpretando seu conteúdo como um vetor de valores de máquina (como se tivesse sido lido de um arquivo usando o método `fromfile()`).

Adicionado na versão 3.2: `fromstring()` foi renomeado para `frombytes()` para maior clareza.

fromfile(*f*, *n*, /)

Lê *n* itens (como valores de máquinas) do objeto arquivo *f* e adiciona-os ao fim do vetor. Se estão disponíveis menos de *n* itens, `EOFError` é levantada, mas os itens disponíveis ainda são inseridos ao final do vetor.

fromlist(*list*, /)

Adiciona itens de *list*. Isso é equivalente a `for x in list: a.append(x)` exceto que se ocorrer um erro de tipo, o vetor não é alterado.

fromunicode(*ustr*, /)

Estende este vetor com dados da string Unicode fornecida. O vetor deve ter o código de tipo `'u'` ou `'w'`; caso contrário, uma `ValueError` é levantada. Use `array.frombytes(unicodestring.encode(enc))` para anexar dados Unicode a um vetor de algum outro tipo.

index(*value*[, *start*[, *stop*]])

Return the smallest *i* such that *i* is the index of the first occurrence of *value* in the array. The optional arguments *start* and *stop* can be specified to search for *value* within a subsection of the array. Raise `ValueError` if *value* is not found.

Alterado na versão 3.10: Adicionados os parâmetros opcionais *start e *stop*.

insert(*index*, *value*, /)

Insert a new item *value* in the array before position *index*. Negative values are treated as being relative to the end of the array.

pop(*index=-1*, /)

Remove o item com o índice *i* do vetor e retorna este item. O valor padrão do argumento opcional é `-1`, assim por padrão o último item é removido e retornado.

remove(*value*, /)

Remove the first occurrence of *value* from the array.

clear()

Remove todos os elementos do vetor.

Adicionado na versão 3.13.

reverse()

Inverte a ordem dos itens no vetor.

tobytes()

Devolve os itens do vetor como um vetor de valores de máquina com a representação em bytes (a mesma sequência de bytes que seria escrita pelo método `tofile()`.)

Adicionado na versão 3.2: **`tostring()`** foi nomeado para `tobytes()` para maior clareza.

tofile(*f*, /)

Escreve todos os itens (como valores de máquinas) para o objeto arquivo *f*.

tolist()

Devolve os itens do vetor como uma lista comum.

tounicode()

Devolve os itens do vetor como uma string Unicode. O vetor deve ser do tipo `'u'` ou `'w'`; caso contrário `ValueError` é levantada. Use `array.tobytes().decode(enc)` para obter uma string Unicode de um vetor de outros tipos.

A representação em string de objetos vetor tem o formato `array(typecode, initializer)`. O *initializer* é omitido se o vetor estiver vazio; caso contrário, é uma string Unicode se o *typecode* for `'u'` ou `'w'`; caso contrário, é uma lista de números. A representação em string pode ser convertida novamente para um array com o mesmo tipo e valor usando

`eval()`, desde que a classe `array` tenha sido importada usando `from array import array`. As variáveis `inf` e `nan` também devem ser definidas se contiverem valores de ponto flutuante correspondentes. Exemplos:

```
array('l')
array('w', 'hello \u2641')
array('l', [1, 2, 3, 4, 5])
array('d', [1.0, 2.0, 3.14, -inf, nan])
```

Ver também

Módulo `struct`

Empacotamento e desempacotamento de dados binários heterogêneos.

NumPy [https://numpy.org/]

O pacote NumPy define outro tipo de vetor.

`weakref` — Referências fracas

Código-fonte: Lib/weakref.py
[https://github.com/python/cpython/tree/3.15/Lib/weakref.py]

O módulo **`weakref`** permite ao programador Python criar *referências fracas* para objetos.

A seguir, o termo *referente* significa o objeto ao qual é referido por uma referência fraca.

Uma referência fraca a um objeto não é suficiente para mantê-lo vivo: quando as únicas referências restantes a um referente são referências fracas, a coleta de lixo está livre para destruir o referente e reutilizar sua memória para outra coisa. Entretanto, até que o objeto seja realmente destruído, a referência fraca poderá retornar o objeto mesmo que não haja referências fortes a ele.

Um uso principal para referências fracas é implementar caches ou mapeamentos contendo objetos grandes, onde é desejado que um objeto grande não seja mantido ativo apenas porque aparece em um cache ou mapeamento.

Por exemplo, se você tiver vários objetos de imagem binária grandes, poderá associar um nome a cada um. Se você usasse um dicionário Python para mapear nomes para imagens, ou imagens para nomes, os objetos de imagem permaneceriam vivos apenas porque apareceriam como valores ou chaves nos dicionários. As classes **`WeakKeyDictionary`** e **`WeakValueDictionary`** fornecidas pelo módulo **`weakref`** são uma alternativa, usando referências fracas para construir mapeamentos que não mantêm objetos vivos apenas porque aparecem nos objetos de mapeamento. Se, por exemplo, um objeto de imagem for um valor em um **`WeakValueDictionary`**, então quando as últimas referências restantes a esse objeto de imagem forem as referências fracas mantidas por mapeamentos

fracos, a coleta de lixo poderá recuperar o objeto e suas entradas correspondentes em mapeamentos fracos são simplesmente excluídos.

`WeakKeyDictionary` e `WeakValueDictionary` usam referências fracas em sua implementação, configurando funções de retorno de chamada nas referências fracas que notificam os dicionários fracos quando uma chave ou valor foi recuperado pela coleta de lixo. `WeakSet` implementa a interface `set`, mas mantém referências fracas aos seus elementos, assim como `WeakKeyDictionary` faz.

`finalize` fornece uma maneira direta de registrar uma função de limpeza a ser chamada quando um objeto é coletado como lixo. Isso é mais simples de usar do que configurar uma função de retorno de chamada em uma referência fraca não tratada, pois o módulo garante automaticamente que o finalizador permaneça ativo até que o objeto seja coletado.

A maioria dos programas deve descobrir que usar um desses tipos de contêineres fracos ou `finalize` é tudo que eles precisam – geralmente não é necessário criar suas próprias referências fracas diretamente. O maquinário de baixo nível é exposto pelo módulo `weakref` para benefício de usos avançados.

Nem todos os objetos podem ser referenciados de maneira fraca. Objetos que oferecem suporte a referências fracas incluem instâncias de classe, funções escritas em Python (mas não em C), métodos de instância, conjuntos, frozensets, alguns objetos arquivos, geradores, objetos tipo, sockets, arrays, deques, objetos padrão de expressão regular e objetos código.

Alterado na versão 3.2: Adicionado suporte para thread.lock, threading.Lock e objetos código.

Vários tipos embutidos como `list` e `dict` não oferecem suporta diretamente a referências fracas, mas podem adicionar suporte através de subclasses:

```
class Dict(dict):
    pass
```

```
obj = Dict(red=1, green=2, blue=3)   # este objeto é potencialme
```

Outros tipos embutidos como **`tuple`** e **`int`** não oferecem suporte a referências fracas mesmo em subclasses.

Os tipos de extensão podem ser facilmente criados para oferecer suporte a referências fracas; veja Weak Reference Support.

Quando `__slots__` são definidos para um determinado tipo, o suporte a referência fraca é desativado a menos que uma string `'__weakref__'` também esteja presente na sequência de strings na declaração de `__slots__`. Veja a documentação de `__slots__` para detalhes.

class weakref.ref(*object*[, *callback*])

Retorna uma referência fraca para *object*. O objeto original pode ser recuperado chamando o objeto referência se o referente ainda estiver ativo; se o referente não estiver mais ativo, chamar o objeto referência fará com que **`None`** seja retornado. Se *callback* for fornecido e não for **`None`**, e o objeto referência fraca retornado ainda estiver ativo, o função de retorno será chamada quando o objeto estiver prestes a ser finalizado; o objeto referência fraca será passado como único parâmetro para a função de retorno; o referente não estará mais disponível.

É permitido que muitas referências fracas sejam construídas para o mesmo objeto. As funções de retorno registradas para cada referência fraca serão chamadas da função de retorno registrada mais recentemente para a função de retorno registrada mais antiga.

As exceções levantadas pela função de retorno serão anotadas na saída de erro padrão, mas não poderão ser propagadas; elas são tratadas exatamente da mesma maneira que as exceções levantadas pelo método **`__del__()`** de um objeto.

Referências fracas são hasheáveis se o *object* for hasheável. Elas manterão seu valor de hash mesmo depois que *object* for excluído. Se

`hash()` for chamada pela primeira vez somente após o *object* ter sido excluído, a chamada vai levantar `TypeError`.

Referências fracas oferecem suporte a testes de igualdade, mas não de ordenação. Se os referentes ainda estiverem vivos, duas referências terão a mesma relação de igualdade que seus referentes (independentemente do *callback*). Se um dos referentes tiver sido excluído, as referências serão iguais somente se os objetos referência forem o mesmo objeto.

Este é um tipo do qual pode ser feita subclasse em vez de uma função de fábrica.

Referências fracas são genéricas em relação ao tipo do objeto ao qual se referem.

__callback__

Este atributo somente leitura retorna a função de retorno atualmente associada à referência fraca. Se não houver função de retorno ou se o referente da referência fraca não estiver mais ativo, então este atributo terá o valor `None`.

Alterado na versão 3.4: Adicionado o atributo `__callback__`.

weakref.proxy(*object*[, *callback*])

Retorna um intermediário (proxy, em inglês) para *object* que usa uma referência fraca. Isto provê suporte ao uso do intermediário na maioria dos contextos, em vez de exigir a desreferenciação explícita usada com objetos de referência fraca. O objeto retornado terá um tipo `ProxyType` ou `CallableProxyType`, dependendo se *object* é chamável. Objetos intermediários não são hasheáveis independentemente do referente; isso evita uma série de problemas relacionados à sua natureza fundamentalmente mutável e impede seu uso como chaves de dicionário. *callback* é igual ao parâmetro de mesmo nome da função `ref()`.

Acessar um atributo do objeto intermediário depois que o referente é coletado como lixo levanta `ReferenceError`.

Alterado na versão 3.8: Estendeu o suporte ao operador em objetos intermediários para incluir os operadores `@` e `@=` para multiplicação de matrizes.

weakref.getweakrefcount(*object*)

Retorna o número de referências fracas e intermediários que fazem referência a *object*.

weakref.getweakrefs(*object*)

Retorna uma lista de todos os objetos intermediários e de referência fraca que fazem referência a *object*.

class weakref.WeakKeyDictionary([*dict*])

Classe de mapeamento que faz referência fraca a chaves. Entradas no dicionário serão descartadas quando não houver mais uma referência forte à chave. Isso pode ser usado para associar dados adicionais a um objeto de propriedade de outras partes de uma aplicação sem adicionar atributos a esses objetos. Isso pode ser especialmente útil com objetos que substituem acessos de atributos.

Note que quando uma chave com valor igual a uma chave existente (mas não identidade igual) é inserida no dicionário, ela substitui o valor, mas não substitui a chave existente. Devido a isso, quando a referência à chave original é excluída, ela também exclui a entrada no dicionário:

```
>>> class T(str): pass
...
>>> k1, k2 = T(), T()
>>> d = weakref.WeakKeyDictionary()
>>> d[k1] = 1   # d = {k1: 1}
>>> d[k2] = 2   # d = {k1: 2}
>>> del k1      # d = {}
```

Uma solução alternativa seria remover a chave antes da reatribuição:

```
>>> class T(str): pass
...
>>> k1, k2 = T(), T()
>>> d = weakref.WeakKeyDictionary()
>>> d[k1] = 1   # d = {k1: 1}
>>> del d[k1]
>>> d[k2] = 2   # d = {k2: 2}
>>> del k1      # d = {k2: 2}
```

Alterado na versão 3.9: Adicionado suporte para operadores `|` e `|=`, conforme especificado na **PEP 584** [https://peps.python.org/pep-0584/].

Objetos `WeakKeyDictionary` têm um método adicional que expõe as referências internas diretamente. Não há garantia de que as referências estejam "ativas" no momento em que são usadas, então o resultado da chamada das referências precisa ser verificado antes de ser usado. Isso pode ser usado para evitar a criação de referências que farão com que o coletor de lixo mantenha as chaves por mais tempo do que o necessário.

WeakKeyDictionary.keyrefs()

Retorna um iterável das referências fracas às chaves.

class weakref.WeakValueDictionary([*dict*])

Classe de mapeamento que faz referência fraca a valores. Entradas no dicionário serão descartadas quando nenhuma referência forte ao valor existir mais.

Alterado na versão 3.9: Adicionado suporte para operadores `|` e `|=`, conforme especificado na **PEP 584** [https://peps.python.org/pep-0584/].

Os objetos `WeakValueDictionary` têm um método adicional que apresenta os mesmos problemas que o método `WeakKeyDictionary.keyrefs()`.

WeakValueDictionary.valuerefs()

Retorna um iterável das referências fracas aos valores.

class weakref.WeakSet([*elements*])

Define a classe que mantém referências fracas para seus elementos. Um elemento será descartado quando nenhuma referência forte a ele existir mais.

class weakref.WeakMethod(*method*[, *callback*])

Uma subclasse personalizada de `ref` que simula uma referência fraca a um método vinculado (ou seja, um método definido em uma classe e pesquisado em uma instância). Como um método vinculado é efêmero, uma referência fraca padrão não pode mantê-lo. `WeakMethod` tem um código especial para recriar o método vinculado até que o objeto ou a função original morra:

```
>>> class C:
...     def method(self):
...         print("method called!")
...
>>> c = C()
>>> r = weakref.ref(c.method)
>>> r()
>>> r = weakref.WeakMethod(c.method)
>>> r()
<bound method C.method of <__main__.C object at 0x7fc8598302
>>> r()()
method called!
>>> del c
>>> gc.collect()
0
>>> r()
>>>
```

callback é o mesmo que o parâmetro de mesmo nome da função `ref()`.

Adicionado na versão 3.4.

class weakref.finalize(*obj*, *func*, /, **args*, ***kwargs*)

Retorna um objeto finalizador chamável que será chamado quando *obj* for coletado como lixo. Diferentemente de uma referência fraca comum, um finalizador sempre sobreviverá até que o objeto referência seja coletado, simplificando muito o gerenciamento do ciclo de vida.

Um finalizador é considerado *alive* (vivo) até ser chamado (explicitamente ou na coleta de lixo), e depois disso ele é *dead* (morto). Chamar um finalizador vivo retorna o resultado da avaliação de `func(*arg, **kwargs)`, enquanto chamar um finalizador morto retorna **`None`**.

Exceções levantadas por funções de retorno do finalizador durante a coleta de lixo serão mostradas na saída de erro padrão, mas não podem ser propagadas. Elas são tratadas da mesma forma que exceções levantadas de um método **`__del__()`** de um objeto ou de uma função de retorno da referência fraca.

Quando o programa é encerrado (ou mais genericamente, no desligamento do interpretador), cada finalizador vivo restante é chamado, a menos que seu atributo **`atexit`** tenha sido definido como falso. Eles são chamados na ordem reversa da criação.

Um finalizador nunca vai invocar sua função de retorno durante a parte posterior do desligamento do interpretador quando os globais do módulo podem ter sido substituídos por **`None`**.

__call__()

Se *self* estiver vivo, marca-o como morto e retorna o resultado da chamada `func(*args, **kwargs)`. Se *self* estiver morto, retorna **`None`**.

detach()

Se *self* estiver vivo, marca-o como morto e retorna a tupla `(obj, func, args, kwargs)`. Se *self* estiver morto, retorna **`None`**.

peek()

Se *self* estiver vivo, retorna a tupla `(obj, func, args, kwargs)`. Se *self* estiver morto, retorna **`None`**.

alive

Propriedade que é verdadeiro se o finalizador estiver ativo, falsa caso contrário.

atexit

Uma propriedade booleana gravável que por padrão é verdadeiro. No desligamento do interpretador, todos os finalizadores vivos restantes para os quais `atexit` é verdadeiro são chamados na ordem reversa da criação.

Nota

É importante garantir que *func*, *args* e *kwargs* não possuam nenhuma referência a *obj*, direta ou indiretamente, pois, caso contrário, *obj* nunca será coletado como lixo. Em particular, *func* não deve ser um método vinculado de *obj*.

Adicionado na versão 3.4.

class weakref.ReferenceType

O objeto tipo para objetos referências fracas.

class weakref.ProxyType

O tipo de objeto para intermediários de objetos que não são chamáveis.

class weakref.CallableProxyType

O objeto de tipo para intermediários de objetos chamáveis.

weakref.ProxyTypes

Sequência contendo todos os objetos de tipo para intermediários. Isso pode tornar mais simples testar se um objeto é um intermediário sem depender da nomeação de ambos os tipos de intermediário.

Ver também

PEP 205 [https://peps.python.org/pep-0205/] - Referências fracas

A proposta e a justificativa para esse recurso, incluindo links para implementações anteriores e informações sobre recursos semelhantes em outras linguagens.

Objetos referência fraca

Objetos referência fraca não têm métodos nem atributos além de `ref.__callback__`. Um objeto referência fraca permite que o referente seja obtido, se ele ainda existir, chamando-o:

```
>>> import weakref
>>> class Object:
...     pass
...
>>> o = Object()
>>> r = weakref.ref(o)
>>> o2 = r()
>>> o is o2
True
```

Se o referente não existir mais, chamar o objeto referência retornará `None`:

```
>>> del o, o2
>>> print(r())
None
```

O teste de se um objeto referência fraca ainda está vivo deve ser feito usando a expressão `ref() is not None`. Normalmente, o código da aplicação que precisa usar um objeto de referência deve seguir este padrão:

```
# r é um objeto referência fraca
o = r()
if o is None:
    # referente foi coletado como lixo
    print("O objeto foi desalocado; não é possível mexer com ele
else:
    print("O objeto ainda está vivo!")
    o.faz_algo_útil()
```

Usar um teste separado para “vivacidade” cria condições de corrida em aplicações que usam mais de uma thread; uma outra thread pode fazer com que uma referência fraca seja invalidada antes que a referência fraca seja

chamada; o idioma mostrado acima é seguro em aplicações que usam mais de uma thread, bem como em aplicações de thread única.

Versões especializadas de objetos `ref` podem ser criadas por meio de subclasse. Isso é usado na implementação do `WeakValueDictionary` para reduzir a sobrecarga de memória para cada entrada no mapeamento. Isso pode ser mais útil para associar informações adicionais a uma referência, mas também pode ser usado para inserir processamento adicional em chamadas para recuperar o referente.

Este exemplo mostra como uma subclasse de `ref` pode ser usada para armazenar informações adicionais sobre um objeto e afetar o valor retornado quando o referente é acessado:

```
import weakref

class ExtendedRef(weakref.ref):
    def __init__(self, ob, callback=None, /, **annotations):
        super().__init__(ob, callback)
        self.__counter = 0
        for k, v in annotations.items():
            setattr(self, k, v)

    def __call__(self):
        """Retorna um par contendo o referente e o número de
        vezes que a referência foi chamada.
        """
        ob = super().__call__()
        if ob is not None:
            self.__counter += 1
            ob = (ob, self.__counter)
        return ob
```

Exemplo

Este exemplo simples mostra como uma aplicação pode usar IDs de objeto para recuperar objetos que ele viu antes. Os IDs dos objetos podem então ser usados em outras estruturas de dados sem forçar os objetos a

permanecerem vivos, mas os objetos ainda podem ser recuperados por ID se o fizerem.

```
import weakref

_id2obj_dict = weakref.WeakValueDictionary()

def remember(obj):
    oid = id(obj)
    _id2obj_dict[oid] = obj
    return oid

def id2obj(oid):
    return _id2obj_dict[oid]
```

Objetos finalizadores

O principal benefício de usar `finalize` é que ele simplifica o registro de um retorno de chamada sem precisar preservar o objeto finalizador retornado. Por exemplo

```
>>> import weakref
>>> class Object:
...     pass
...
>>> kenny = Object()
>>> weakref.finalize(kenny, print, "You killed Kenny!")
<finalize object at ...; for 'Object' at ...>
>>> del kenny
You killed Kenny!
```

O finalizador pode ser chamado diretamente também. No entanto, o finalizador vai invocar a função de retorno no máximo uma vez.

```
>>> def callback(x, y, z):
...     print("CALLBACK")
...     return x + y + z
...
>>> obj = Object()
>>> f = weakref.finalize(obj, callback, 1, 2, z=3)
>>> assert f.alive
```

```
>>> assert f() == 6
CALLBACK
>>> assert not f.alive
>>> f()                      # callback not called because finali
>>> del obj                  # callback not called because finali
```

Você pode desfazer o registro de um finalizador usando seu método `detach()`. Isso elimina o finalizador e retorna os argumentos passados ao construtor quando ele foi criado.

```
>>> obj = Object()
>>> f = weakref.finalize(obj, callback, 1, 2, z=3)
>>> f.detach()
(<...Object object ...>, <function callback ...>, (1, 2), {'z':
>>> newobj, func, args, kwargs = _
>>> assert not f.alive
>>> assert newobj is obj
>>> assert func(*args, **kwargs) == 6
CALLBACK
```

A menos que você defina o atributo `atexit` como `False`, um finalizador será chamado quando o programa sair se ele ainda estiver vivo. Por exemplo

```
>>> obj = Object()
>>> weakref.finalize(obj, print, "obj dead or exiting")
<finalize object at ...; for 'Object' at ...>
>>> exit()
obj dead or exiting
```

Comparando finalizadores com métodos `__del__()`

Suponha que queremos criar uma classe cujas instâncias representam diretórios temporários. Os diretórios devem ser excluídos com seus conteúdos quando o primeiro dos seguintes eventos ocorrer:

- o objeto é um lixo coletado,

- o método **`remove()`** do objeto é chamado, ou
- o programa finaliza.

Podemos tentar implementar a classe usando um método **`__del__()`** da seguinte maneira:

```
class TempDir:
    def __init__(self):
        self.name = tempfile.mkdtemp()

    def remove(self):
        if self.name is not None:
            shutil.rmtree(self.name)
            self.name = None

    @property
    def removed(self):
        return self.name is None

    def __del__(self):
        self.remove()
```

A partir do Python 3.4, os métodos **`__del__()`** não impedem mais que os ciclos de referência sejam coletados como lixo, e os módulos globais não são mais forçados a **`None`** durante desligamento do interpretador. Então, esse código deve funcionar sem problemas no CPython.

Entretanto, o tratamento dos métodos **`__del__()`** é notoriamente específico da implementação, pois depende de detalhes internos da implementação do coletor de lixo do interpretador.

Uma alternativa mais robusta pode ser definir um finalizador que faça referência apenas às funções e objetos específicos de que necessita, em vez de ter acesso ao estado completo do objeto:

```
class TempDir:
    def __init__(self):
        self.name = tempfile.mkdtemp()
        self._finalizer = weakref.finalize(self, shutil.rmtree,

    def remove(self):
```

```
        self._finalizer()

    @property
    def removed(self):
        return not self._finalizer.alive
```

Definido assim, nosso finalizador recebe apenas uma referência aos detalhes que ele precisa para limpar o diretório apropriadamente. Se o objeto nunca for coletado como lixo, o finalizador ainda será chamado na saída.

A outra vantagem dos finalizadores baseados em referências fracas é que eles podem ser usados para registrar finalizadores para classes onde a definição é controlada por terceiros, como executar código quando um módulo é descarregado:

```
import weakref, sys
def unloading_module():
    # referência implícita aos globais do módulo do corpo da fur
weakref.finalize(sys.modules[__name__], unloading_module)
```

Nota

Se você criar um objeto finalizador em uma thread em um daemon assim que o programa sair, então há a possibilidade de que o finalizador não seja chamado na saída. No entanto, em um thread em um daemon `atexit.register()`, `try: ... finally: ...` e `with: ...` não garantem que a limpeza ocorra também.

`types` — Criação de tipos dinâmicos e nomes para tipos embutidos

Código-fonte: Lib/types.py [https://github.com/python/cpython/tree/3.15/Lib/types.py]

Este módulo define funções utilitárias para auxiliar na criação dinâmica de novos tipos.

Também define nomes para alguns tipos de objetos usados pelo interpretador Python padrão, mas não expostos como componentes embutidos como `int` ou `str` são.

Por fim, fornece algumas classes e funções adicionais relacionadas ao tipo que não são fundamentais o suficiente para serem incorporadas.

Criação de tipos dinâmicos

types.new_class(*name*, *bases=()*, *kwds=None*, *exec_body=None*)

Cria um objeto de classe dinamicamente usando a metaclasse apropriada.

Os três primeiros argumentos são os componentes que compõem um cabeçalho de definição de classe: o nome da classe, as classes base (em ordem), os argumentos nomeados (como `metaclass`).

O argumento *exec_body* é um retorno de chamada usado para preencher o espaço para nome da classe recém-criado. Ele deve aceitar o espaço para nome da classe como seu único argumento e atualizar o espaço para nome diretamente com o conteúdo da classe. Se nenhum retorno de

chamada for fornecido, ele terá o mesmo efeito que passar em `lambda ns: None`.

Adicionado na versão 3.3.

types.prepare_class(*name*, *bases=()*, *kwds=None*)

Calcula a metaclasse apropriada e cria o espaço de nomes da classe.

Os argumentos são os componentes que compõem um cabeçalho de definição de classe: o nome da classe, as classes base (em ordem) e os argumentos nomeados (como `metaclass`).

O valor de retorno é uma tupla de 3: `metaclass, namespace, kwds`

metaclass é a metaclasse apropriada, *namespace* é o espaço de nomes da classe preparada e *kwds* é uma cópia atualizada do argumento passado no *kwds* com qualquer entrada `'metaclass'` removida. Se nenhum argumento *kwds* for passado, este será um ditado vazio.

Adicionado na versão 3.3.

Alterado na versão 3.6: O valor padrão para o elemento `namespace` da tupla retornada foi alterado. Agora, um mapeamento preservando-ordem-inserção é usado quando a metaclasse não possui um método `__prepare__`.

Ver também

Metaclasses

Detalhes completos do processo de criação de classe suportado por essas funções

PEP 3115 **[https://peps.python.org/pep-3115/]** - **Metaclasses no Python 3000**

Introduzido o gancho de espaço de nomes `__prepare__`

types.resolve_bases(*bases*)

Resolve entradas MRO dinamicamente, conforme especificado pela **PEP 560** [https://peps.python.org/pep-0560/].

Esta função procura por itens em *bases* que não sejam instâncias de `type` e retorna uma tupla onde cada objeto que possui um método `__mro_entries__()` é substituído por um resultado desempacotado da chamada desse método. Se um item *bases* é uma instância de **`type`**, ou não possui o método **`__mro_entries__()`**, ele é incluído na tupla de retorno inalterada.

Adicionado na versão 3.7.

types.get_original_bases(*cls*, /)

Retorna a tupla de objetos originalmente dados como as bases de *cls* antes do método `__mro_entries__()` ter sido chamado em qualquer base (seguindo os mecanismos apresentados na **PEP 560** [https://peps.python.org/pep-0560/]). Isso é útil para introspecção de Generics.

Para classes que possuem um atributo `__orig_bases__`, esta função retorna o valor de `cls.__orig_bases__`. Para classes sem o atributo `__orig_bases__`, `cls.__bases__` é retornado.

Exemplos:

```
from typing import TypeVar, Generic, NamedTuple, TypedDict

T = TypeVar("T")
class Foo(Generic[T]): ...
class Bar(Foo[int], float): ...
class Baz(list[str]): ...
Eggs = NamedTuple("Eggs", [("a", int), ("b", str)])
Spam = TypedDict("Spam", {"a": int, "b": str})

assert Bar.__bases__ == (Foo, float)
assert get_original_bases(Bar) == (Foo[int], float)

assert Baz.__bases__ == (list,)
assert get_original_bases(Baz) == (list[str],)

assert Eggs.__bases__ == (tuple,)
```

```
assert get_original_bases(Eggs) == (NamedTuple,)

assert Spam.__bases__ == (dict,)
assert get_original_bases(Spam) == (TypedDict,)

assert int.__bases__ == (object,)
assert get_original_bases(int) == (object,)
```

Adicionado na versão 3.12.

Ver também

PEP 560 [https://peps.python.org/pep-0560/] - Suporte básico para módulo typing e tipos genéricos

Tipos padrão do interpretador

Este módulo fornece nomes para muitos dos tipos necessários para implementar um interpretador Python. Evita deliberadamente incluir alguns dos tipos que surgem apenas incidentalmente durante o processamento, como o tipo `listiterator`.

O uso típico desses nomes é para verificações `isinstance()` ou `issubclass()`.

Se você instanciar algum desses tipos, observe que as assinaturas podem variar entre as versões do Python.

Os nomes padrão são definidos para os seguintes tipos:

class types.NoneType

O tipo de `None`.

Adicionado na versão 3.10.

class types.FunctionType
class types.LambdaType

O tipo de funções definidas pelo usuário e funções criadas por expressões **`lambda`**.

Levanta um evento de auditoria `function.__new__` com o argumento `code`.

O evento de auditoria ocorre apenas para instanciação direta de objetos de função e não é levantado para compilação normal.

class types.GeneratorType

O tipo de objetos de iterador gerador, criados pelas funções de gerador.

class types.CoroutineType

O tipo de objetos de corrotina, criado por funções de **`async def`**.

Adicionado na versão 3.5.

class types.AsyncGeneratorType

O tipo de objetos de iterador gerador assíncrono, criados pelas funções do gerador assíncrono.

Adicionado na versão 3.6.

class types.CodeType(****kwargs*)

O tipo de objetos código retornados por **`compile()`**.

Levanta um `code.__new__` de evento de auditoria com os argumentos `code`, `filename`, `name`, `argcount`, `posonlyargcount`, `kwonlyargcount`, `nlocals`, `stacksize`, `flags`.

Observe que os argumentos auditados podem não corresponder aos nomes ou posições exigidos pelo inicializador. O evento de auditoria ocorre apenas para instanciação direta de objetos de código e não é levantado para compilação normal.

class types.CellType

O tipo para objetos de célula: tais objetos são usados como contêineres para as variáveis de clausura de uma função.

Adicionado na versão 3.8.

class types.MethodType

O tipo de método de instâncias de classe definidas pelo usuário.

class types.BuiltinFunctionType
class types.BuiltinMethodType

O tipo de funções embutidas como `len()` ou `sys.exit()`, e métodos de classes embutidas. (Aqui, o termo “embutidas” significa “escrito em C”.)

class types.WrapperDescriptorType

O tipo de método de alguns tipos de dados embutidos e classes base, como `object.__init__()` ou `object.__lt__()`.

Adicionado na versão 3.7.

class types.MethodWrapperType

O tipo de métodos *vinculados* de alguns tipos de dados embutidos e classes base. Por exemplo, é o tipo de `object().__str__`.

Adicionado na versão 3.7.

class types.NotImplementedType

O tipo de `NotImplemented`.

Adicionado na versão 3.10.

class types.MethodDescriptorType

O tipo de método de alguns tipos de dados embutidos, como `str.join()`.

Adicionado na versão 3.7.

class types.ClassMethodDescriptorType

O tipo de métodos de classe *não vinculados* de alguns tipos de dados embutidos, como `dict.__dict__['fromkeys']`.

Adicionado na versão 3.7.

class types.ModuleType(*name*, *doc=None*)

O tipo de módulos. O construtor aceita o nome do módulo a ser criado e, opcionalmente, seu docstring.

Ver também

Documentação em objetos de módulo

Fornece detalhes sobre os atributos especiais que podem ser encontrados em instâncias de **`ModuleType`**.

`importlib.util.module_from_spec()`

Módulos criados usando o construtor **`ModuleType`** são criados com muitos de seus atributos especiais não definidos ou definidos com valores padrão. **`module_from_spec()`** fornece uma maneira mais robusta de criar instâncias **`ModuleType`**, o que garante que os vários atributos sejam definidos adequadamente.

class types.EllipsisType

O tipo de **`Ellipsis`**.

Adicionado na versão 3.10.

class types.GenericAlias(*t_origin*, *t_args*)

O tipo dos genéricos parametrizados como `list[int]`.

`t_origin` deve ser uma classe genérica não parametrizada, como `list`, `tuple` ou `dict`. `t_args` deve ser uma **`tuple`** (possivelmente com comprimento 1) de tipos que parametrizam `t_origin`:

```
>>> from types import GenericAlias

>>> list[int] == GenericAlias(list, (int,))
True
>>> dict[str, int] == GenericAlias(dict, (str, int))
True
```

Adicionado na versão 3.9.

Alterado na versão 3.9.2: Este tipo pode agora ter uma subclasse.

Ver também

Tipos Generic Alias
Documentação detalhada sobre instâncias de `types.GenericAlias`

PEP 585 **[https://peps.python.org/pep-0585/]** - **Sugestão de tipo para Genéricos em coleções padrão**
Apresentação da classe `types.GenericAlias`

class types.UnionType

The type of union type expressions.

Adicionado na versão 3.10.

Alterado na versão 3.14: Isso é agora um apelido para `typing.Union`.

class types.TracebackType(*tb_next*, *tb_frame*, *tb_lasti*, *tb_lineno*)

O tipo de objetos traceback, como encontrados em `sys.exception().__traceback__`.

Veja a referência de linguagem para detalhes dos atributos e operações disponíveis, e orientação sobre como criar tracebacks dinamicamente.

class types.FrameType

O tipo de objetos de quadro como encontrado em `tb.tb_frame` se `tb` é um objeto traceback.

class types.FrameLocalsProxyType

O tipo de objeto proxy para variáveis locais do frame, conforme encontrado no atributo `frame.f_locals`.

Adicionado na versão 3.15.

Ver também

PEP 667 [https://peps.python.org/pep-0667/]

class types.LazyImportType

O tipo de objetos proxy de importação preguiçosa (*lazy*). Esses objetos são criados quando um módulo é importado de forma preguiçosa e servem como espaços reservados até que o módulo seja efetivamente acessado. Esse tipo pode ser usado para detectar importações preguiçosas programaticamente.

Adicionado na versão 3.15.

Ver também

PEP 810 [https://peps.python.org/pep-0810/]

resolve()

Materializa a importação preguiçosa e retorna o objeto "real" que está sendo importado.

class types.GetSetDescriptorType

O tipo de objetos definidos em módulos de extensão com `PyGetSetDef`, como `FrameType.f_locals` ou `array.array.typecode`. Este tipo é usado como descritor para atributos de objeto; tem o mesmo propósito que o tipo `property`, mas para classes definidas em módulos de extensão.

class types.MemberDescriptorType

O tipo de objetos definidos em módulos de extensão com `PyMemberDef`, como `datetime.timedelta.days`. Este tipo é usado como descritor para membros de dados C simples que usam funções de conversão padrão; tem o mesmo propósito que o tipo `property`, mas para classes definidas em módulos de extensão.

Além disso, quando uma classe é definida com um atributo `__slots__`, então para cada atributo, uma instância de **`MemberDescriptorType`** será adicionada como um atributo na classe. Isso permite que o atributo apareça no `__dict__` da classe.

Em outras implementações de Python, este tipo pode ser idêntico a `GetSetDescriptorType`.

class types.MappingProxyType(*mapping*)

Proxy somente leitura de um mapeamento. Ele fornece uma visão dinâmica das entradas do mapeamento, o que significa que quando o mapeamento muda, a visão reflete essas mudanças.

`MappingProxyType`s são genéricos em relação a dois tipos, representando (respectivamente) os tipos das chaves e dos valores do mapeamento subjacente.

Adicionado na versão 3.3.

Alterado na versão 3.9: Atualizado para ter suporte ao novo operador de união (`|`) da **PEP 584** [https://peps.python.org/pep-0584/], que simplesmente delega para o mapeamento subjacente.

key in proxy

Retorna `True` se o mapeamento subjacente tiver uma chave *key*, senão `False`.

proxy[key]

Retorna e o item do mapeamento subjacente com a chave *key*. Levanta um **`KeyError`** se *key* não estiver no mapeamento subjacente.

iter(proxy)

Retorna um iterador sobre as chaves do mapeamento subjacente. Este é um atalho para `iter(proxy.keys())`.

len(proxy)

Retorna o número de itens no mapeamento subjacente.

copy()

Retorna uma cópia rasa do mapeamento subjacente.

get(*key*[, *default*])

Retorna o valor para *key* se *key* estiver no mapeamento subjacente, caso contrário, *default*. Se *default* não for fornecido, o padrão é `None`, de forma que este método nunca levante uma **`KeyError`**.

items()

Retorna uma nova visão dos itens do mapeamento subjacente (pares `(chave, valor)`).

keys()

Retorna uma nova visão das chaves do mapeamento subjacente.

values()

Retorna uma nova visão dos valores do mapeamento subjacente.

reversed(proxy)

Retorna um iterador reverso sobre as chaves do mapeamento subjacente.

Adicionado na versão 3.9.

hash(proxy)

Retorna um hash do mapeamento subjacente.

Adicionado na versão 3.12.

class types.CapsuleType

O tipo de objetos cápsula.

Adicionado na versão 3.13.

Classes e funções de utilidades adicionais

class types.SimpleNamespace

Uma subclasse **object** simples que fornece acesso de atributo ao seu espaço de nomes, bem como um repr significativo.

Ao contrário de **object**, com **SimpleNamespace** você pode adicionar e remover atributos.

Objetos **SimpleNamespace** podem ser inicializados da mesma forma que **dict**: ou com argumentos nomeados, com um único argumento posicional, ou com ambos. Quando inicializados com argumentos nomeados, eles são adicionados diretamente ao espaço de nomes subjacente. Alternativamente, quando inicializado com um argumento posicional, o espaço de nomes subjacente será atualizado com pares de valores-chave desse argumento (um objeto mapeamento ou um objeto iterável produzindo pares de valores-chave). Todas essas chaves devem ser strings.

O tipo é aproximadamente equivalente ao seguinte código:

```
class SimpleNamespace:
    def __init__(self, mapping_or_iterable=(), /, **kwargs):
        self.__dict__.update(mapping_or_iterable)
        self.__dict__.update(kwargs)

    def __repr__(self):
        items = (f"{k}={v!r}" for k, v in self.__dict__.items
        return "{}({})".format(type(self).__name__, ", ".joir

    def __eq__(self, other):
        if isinstance(self, SimpleNamespace) and isinstance(c
           return self.__dict__ == other.__dict__
        return NotImplemented
```

`SimpleNamespace` pode ser útil como um substituto para `class NS: pass`. No entanto, para um tipo de registro estruturado, use **namedtuple()**.

Objetos **SimpleNamespace** são suportados por **copy.replace()**.

Adicionado na versão 3.3.

Alterado na versão 3.9: A ordem dos atributos no repr mudou de alfabética para inserção (como no `dict`).

Alterado na versão 3.13: Adicionado suporte para um argumento posicional opcional.

types.DynamicClassAttribute(*fget=None*, *fset=None*, *fdel=None*, *doc=None*)

Roteia o acesso ao atributo em uma classe para __getattr__.

Este é um descritor, usado para definir atributos que atuam de forma diferente quando acessados por meio de uma instância e por meio de uma classe. O acesso à instância permanece normal, mas o acesso a um atributo por meio de uma classe será roteado para o método __getattr__ da classe; isso é feito levantando AttributeError.

Isso permite ter propriedades ativas em uma instância, e ter atributos virtuais na classe com o mesmo nome (veja **`enum.Enum`** para um exemplo).

Adicionado na versão 3.4.

Funções de utilidade de corrotina

types.coroutine(*gen_func*)

Esta função transforma uma função geradora em uma função de corrotina que retorna uma corrotina baseada em gerador. A corrotina baseada em gerador ainda é um iterador gerador, mas também é considerada um objeto corrotina e é aguardável. No entanto, pode não necessariamente implementar o método **`__await__()`**.

Se *gen_func* for uma função geradora, ela será modificada no local.

Se *gen_func* não for uma função geradora, ela será envolta. Se ele retornar uma instância de **`Collections.abc.Generator`**, a instância será

envolvida em um objeto proxy *aguardável*. Todos os outros tipos de objetos serão retornados como estão.

Adicionado na versão 3.5.

copy — Operações de cópia profunda e cópia rasa

Código-fonte: Lib/copy.py [https://github.com/python/cpython/tree/3.15/Lib/copy.py]

As instruções de atribuição no Python não copiam objetos; elas criam ligações entre um destino e um objeto. Para coleções mutáveis ou que contêm itens mutáveis, às vezes é necessária uma cópia para que seja possível alterar uma sem alterar a outra. Este módulo fornece operações genéricas de cópia profunda e rasa (explicadas abaixo).

Resumo da interface:

copy.copy(*obj*)

Retorna uma cópia rasa de *obj*.

copy.deepcopy(*obj*[, *memo*])

Retorna uma cópia profunda de *obj*.

copy.replace(*obj*, /, **changes*)

Cria um novo objeto do mesmo tipo que *obj*, substituindo os campos pelos valores de *changes*.

Adicionado na versão 3.13.

exception copy.Error

Levantada para erros específicos do módulo.

A diferença entre cópia profunda e rasa é relevante apenas para objetos compostos (objetos que contêm outros objetos, como listas ou instâncias de classe):

- Uma *cópia rasa* constrói um novo objeto composto e, na medida do possível, insere nele *referências* aos objetos encontrados no original.
- Uma *cópia profunda* constrói um novo objeto composto e então, recursivamente, insere nele *cópias* dos objetos encontrados no original.

Frequentemente, existem dois problemas com operações de cópia profunda que não existem com operações de cópia rasa:

- Objetos recursivos (objetos compostos que, direta ou indiretamente, contêm uma referência a si mesmos) podem causar um laço recursivo.
- Como a cópia profunda copia tudo, ela pode copiar muito, incluindo dados que devem ser compartilhados entre as cópias.

A função `deepcopy()` evita esses problemas:

- mantendo um dicionário `memo` de objetos já copiados durante a passagem de cópia atual; e
- permitindo que as classes definidas pelo usuário substituam a operação de cópia ou o conjunto de componentes copiados.

Este módulo não copia tipos como módulo, método, stack trace (situação da pilha de execução), quadro de empilhamento, arquivo, soquete, janela ou outros tipos semelhantes. Ele "copia" funções e classes (rasas e profundamente), devolvendo o objeto original inalterado; isso é compatível com a maneira como estes itens são tratados pelo módulo `pickle`.

Cópias rasas (*shallow copies*) de muitas coleções podem ser feitas utilizando o método **`copy()`** correspondente (como `list.copy()`, `dict.copy()` ou `set.copy()`), e de sequências (como listas ou *bytearrays*) criando uma fatia (*slice*) de toda a sequência (`sequence[:]`). No entanto, esses métodos e o fatiamento podem criar uma instância do tipo base ao copiar uma instância de uma subclasse, enquanto `copy.copy()` normalmente retorna uma instância do mesmo tipo.

As classes podem usar as mesmas interfaces para controlar a cópia que usam para controlar a serialização com pickle. Veja a descrição do módulo `pickle` para obter informações sobre esses métodos. Na verdade, o módulo **`copy`** usa as funções pickle registradas do módulo `copyreg`.

Para que uma classe defina sua própria implementação de cópia, ela pode definir métodos especiais `__copy__()` e `__deepcopy__()`.

object.__copy__(*self*)

Chamado para implementar a operação de cópia rasa; não é fornecido nenhum argumento adicional.

object.__deepcopy__(*self*, *memo*)

Chamado para implementar a operação de cópia profunda; é passado um argumento, o dicionário *memo*. Se a implementação `__deepcopy__` precisar fazer uma cópia profunda de um componente, ela deve chamar a função `deepcopy()` com o componente como primeiro argumento e o dicionário *memo* como segundo argumento. O dicionário *memo* deve ser tratado como um objeto opaco.

A função **`copy.replace()`** é mais limitada que `copy()` e `deepcopy()`, e tem suporte apenas a tuplas nomeadas criadas por `namedtuple()`, `dataclasses` e outras classes que definem o método `__replace__()`.

object.__replace__(*self*, /, ***changes*)

Este método deve criar um novo objeto do mesmo tipo, substituindo os campos pelos valores de *changes*.

Adicionado na versão 3.13.

Ver também

Módulo `pickle`

Discussão dos métodos especiais usados para dar suporte à recuperação e à restauração do estado do objeto.

`pprint` — Impressão bonita de dados

Código-fonte: Lib/pprint.py [https://github.com/python/cpython/tree/3.15/Lib/pprint.py]

O módulo **`pprint`** fornece a capacidade de “imprimir de forma bonita” estruturas de dados Python arbitrárias em um formato que pode ser usado como entrada para o interpretador. Se as estruturas formatadas incluírem objetos que não sejam tipos fundamentais do Python, a representação poderá não ser carregável. Este pode ser o caso se objetos como arquivos, soquetes ou classes forem incluídos, bem como muitos outros objetos que não são representáveis como literais do Python.

A representação formatada mantém os objetos em uma única linha, se possível, e os divide em múltiplas linhas se eles não couberem na largura permitida, ajustável pelo parâmetro *width* padrão para 80 caracteres.

Alterado na versão 3.9: Adicionado suporte para impressão bonita de **`types.SimpleNamespace`**.

Alterado na versão 3.10: Adicionado suporte para impressão bonita de **`dataclasses.dataclass`**.

Funções

pprint.pp(*object*, *stream=None*, *indent=1*, *width=80*, *depth=None*, *, *compact=False*, *expand=False*, *sort_dicts=False*, *underscore_numbers=False*)

Imprime a representação formatada de *object*, seguida por uma nova linha. Esta função pode ser usado no interpretador interativo em vez da função **`print()`** para inspecionar valores. Dica: você pode reatribuir `print = pprint.pp` para uso dentro de um escopo.

Parâmetros::

- **object** – O objeto a ser impresso.
- **stream** (file-like object | None) – Um objeto arquivo ou similar no qual a saída será gravada chamando seu método `write()`. Se `None` (o padrão), `sys.stdout` será usado.
- **indent** (*int*) – A quantidade de indentação adicionado para cada nível de aninhamento.
- **width** (*int*) – O número máximo desejado de caracteres por linha na saída. Se uma estrutura nãc puder ser formatada dentro da restrição de largura, será feito o melhor esforço.
- **depth** (*int* | *None*) – O número de níveis de aninhamento que podem ser impressos. Se a estrutura de dados que está sendo impressa for muito profunda, o próximo nível contido será substituído por `....` Se `None` (o padrão), não há restrição na profundidade dos objetos que estão sendo formatados.
- **compact** (*bool*) – Control the way long sequences are formatted. If `False` (the default), each item of sequence will be formatted on a separate line, otherwise as many items as will fit within the *widt* will be formatted on each output line. Incompatibl with *expand*.
- **expand** (*bool*) – If `True`, opening parentheses and brackets will be followed by a newline and the following content will be indented by one level, similar to pretty-printed JSON. Incompatible with *compact*.
- **sort_dicts** (*bool*) – Se `True`, os dicionários serão formatados com suas chaves classificadas, caso contrário, serão exibidos na ordem de inserção (o padrão).
- **underscore_numbers** (*bool*) – Se `True`, os números inteiros serão formatados com o caracter _ para um separador de milhares, caso contrário, o sublinhados não serão exibidos (o padrão).

```
>>> import pprint
>>> stuff = ['spam', 'eggs', 'lumberjack', 'knights', 'ni']
>>> stuff.insert(0, stuff)
>>> pprint.pp(stuff)
[<Recursion on list with id=...>,
 'spam',
 'eggs',
 'lumberjack',
 'knights',
 'ni']
```

Adicionado na versão 3.8.

pprint.pprint(*object*, *stream=None*, *indent=1*, *width=80*, *depth=None*, *, *compact=False*, *expand=False*, *sort_dicts=True*, *underscore_numbers=False*)

Apelido para **pp()** com *sort_dicts* definido como `True` por padrão, o que classificaria automaticamente as chaves dos dicionários, você pode querer usar **pp()** em vez disso, onde é `False` por padrão.

pprint.pformat(*object*, *indent=1*, *width=80*, *depth=None*, *, *compact=False*, *expand=False*, *sort_dicts=True*, *underscore_numbers=False*)

Return the formatted representation of *object* as a string. *indent*, *width*, *depth*, *compact*, *expand*, *sort_dicts* and *underscore_numbers* are passed to the **PrettyPrinter** constructor as formatting parameters and their meanings are as described in the documentation above.

pprint.isreadable(*object*)

Determina se a representação formatada de *object* é “legível” ou pode ser usada para reconstruir o valor usando **eval()**. Isso sempre retorna `False` para objetos recursivos.

```
>>> pprint.isreadable(stuff)
False
```

pprint.isrecursive(*object*)

Determina se *object* requer uma representação recursiva. Esta função está sujeita às mesmas limitações mencionadas em **saferepr()** abaixo e pode

levantar uma exceção **RecursionError** se falhar em detectar um objeto recursivo.

pprint.saferepr(*object*)

Retorna uma representação de string de *object*, protegida contra recursão em algumas estruturas de dados comuns, nomeadamente instâncias de **dict**, **list** e **tuple** ou subclasses cujo `__repr__` não foi substituído. Se a representação do objeto expõe uma entrada recursiva, a referência recursiva será representada como `<Recursion on typename with id=number>`. A representação não é formatada de outra forma.

```
>>> pprint.saferepr(stuff)
"[<Recursion on list with id=...>, 'spam', 'eggs', 'lumberjack
```

Objetos PrettyPrinter

class pprint.PrettyPrinter(*indent=1*, *width=80*, *depth=None*, *stream=None*, ***, *compact=False*, *expand=False*, *sort_dicts=True*, *underscore_numbers=False*)

Constrói uma instância **PrettyPrinter**.

Os argumentos têm o mesmo significado que para **pp()**. Observe que eles estão em uma ordem diferente e que o padrão *sort_dicts* é `True`.

```
>>> import pprint
>>> stuff = ['spam', 'eggs', 'lumberjack', 'knights', 'ni']
>>> stuff.insert(0, stuff[:])
>>> pp = pprint.PrettyPrinter(indent=4)
>>> pp.pprint(stuff)
[   ['spam', 'eggs', 'lumberjack', 'knights', 'ni'],
    'spam',
    'eggs',
    'lumberjack',
    'knights',
    'ni']
>>> pp = pprint.PrettyPrinter(width=41, compact=True)
>>> pp.pprint(stuff)
[['spam', 'eggs', 'lumberjack',
  'knights', 'ni'],
```

```
 'spam', 'eggs', 'lumberjack', 'knights',
 'ni']
>>> pp = pprint.PrettyPrinter(width=41, expand=True, indent=3)
>>> pp.pprint(stuff)
[
   [
      'spam',
      'eggs',
      'lumberjack',
      'knights',
      'ni',
   ],
   'spam',
   'eggs',
   'lumberjack',
   'knights',
   'ni',
]
>>> tup = ('spam', ('eggs', ('lumberjack', ('knights', ('ni',
... ('parrot', ('fresh fruit',))))))))
>>> pp = pprint.PrettyPrinter(depth=6)
>>> pp.pprint(tup)
('spam', ('eggs', ('lumberjack', ('knights', ('ni', ('dead', (
```

Alterado na versão 3.4: Adicionado o parâmetro *compact*.

Alterado na versão 3.8: Adicionado o parâmetro *sort_dicts*.

Alterado na versão 3.10: Adicionado o parâmetro *underscore_numbers*.

Alterado na versão 3.11: Não tenta mais escrever em **`sys.stdout`** se for `None`.

Alterado na versão 3.15: Added the *expand* parameter.

Instâncias **`PrettyPrinter`** contêm os seguintes métodos:

PrettyPrinter.pformat(*object*)

Retorna a representação formatada de *object*. Isso leva em consideração as opções passadas para o construtor **`PrettyPrinter`**.

PrettyPrinter.pprint(*object*)

Exibe a representação formatada de *object* no fluxo configurado, seguida por uma nova linha.

Os métodos a seguir fornecem as implementações para as funções correspondentes com os mesmos nomes. Usar esses métodos em uma instância é um pouco mais eficiente já que novos objetos `PrettyPrinter` não precisam ser criados.

PrettyPrinter.isreadable(*object*)

Determina se a representação formatada do objeto é "legível" ou pode ser usada para reconstruir o valor usando `eval()`. Observe que isso retorna `False` para objetos recursivos. Se o parâmetro *depth* de `PrettyPrinter` estiver definido e o objeto for mais profundo que o permitido, isso retornará `False`.

PrettyPrinter.isrecursive(*object*)

Determina se o objeto requer uma representação recursiva.

Este método é fornecido como um gancho para permitir que as subclasses modifiquem a maneira como os objetos são convertidos em strings. A implementação padrão usa os componentes internos da implementação de `saferepr()`.

PrettyPrinter.format(*object*, *context*, *maxlevels*, *level*)

Retorna três valores: a versão formatada de *object* como uma string, um sinalizador indicando se o resultado é legível e um sinalizador indicando se a recursão foi detectada. O primeiro argumento é o objeto a ser apresentado. O segundo é um dicionário que contém o `id()` de objetos que fazem parte do contexto de apresentação atual (contêineres diretos e indiretos para *objects* que estão afetando a apresentação) como chaves; se for necessário apresentar um objeto que já esteja representado em *context*, o terceiro valor de retorno deverá ser `True`. Chamadas recursivas ao método `format()` devem adicionar entradas adicionais para contêineres neste dicionário. O terceiro argumento, *maxlevels*, fornece o limite solicitado para recursão; será `0` se não houver limite solicitado. Este argumento deve ser passado sem modificações para chamadas recursivas.

O quarto argumento, *level*, fornece o nível atual; chamadas recursivas devem receber um valor menor que o da chamada atual.

Exemplo

Para demonstrar vários usos da função **pp()** e seus parâmetros, vamos buscar informações sobre um projeto no PyPI [https://pypi.org]:

```
>>> import json
>>> import pprint
>>> from urllib.request import urlopen
>>> with urlopen('https://pypi.org/pypi/sampleproject/1.2.0/json'
...     project_info = json.load(resp)['info']
```

Em sua forma básica, **pp()** mostra o objeto inteiro:

```
>>> pprint.pp(project_info)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': ['Development Status :: 3 - Alpha',
                 'Intended Audience :: Developers',
                 'License :: OSI Approved :: MIT License',
                 'Programming Language :: Python :: 2',
                 'Programming Language :: Python :: 2.6',
                 'Programming Language :: Python :: 2.7',
                 'Programming Language :: Python :: 3',
                 'Programming Language :: Python :: 3.2',
                 'Programming Language :: Python :: 3.3',
                 'Programming Language :: Python :: 3.4',
                 'Topic :: Software Development :: Build Tools'],
 'description': 'A sample Python project\n'
                '=======================\n'
                '\n'
                'This is the description file for the project.\n'
                '\n'
                'The file should use UTF-8 encoding and be writte
                'ReStructured Text. It\n'
                'will be used to generate the project webpage on
                'should be written for\n'
                'that purpose.\n'
```

```
                '\n'
                'Typical contents for this file would include an
                'the project, basic\n'
                'usage examples, etc. Generally, including the pr
                'changelog in here is not\n'
                'a good idea, although a simple "What\'s New" sec
                'most recent version\n'
                'may be appropriate.',
 'description_content_type': None,
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {'last_day': -1, 'last_month': -1, 'last_week': -1}
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
 'maintainer_email': None,
 'name': 'sampleproject',
 'package_url': 'https://pypi.org/project/sampleproject/',
 'platform': 'UNKNOWN',
 'project_url': 'https://pypi.org/project/sampleproject/',
 'project_urls': {'Download': 'UNKNOWN',
                  'Homepage': 'https://github.com/pypa/sampleproj
 'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
 'requires_dist': None,
 'requires_python': None,
 'summary': 'A sample Python project',
 'version': '1.2.0'}
```

O resultado pode ser limitado a uma certa *depth* (reticências são usadas para conteúdos mais profundos):

```
>>> pprint.pp(project_info, depth=1)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': [...],
 'description': 'A sample Python project\n'
                '========================\n'
                '\n'
                'This is the description file for the project.\n'
                '\n'
                'The file should use UTF-8 encoding and be writte
```

```
                 'ReStructured Text. It\n'
                 'will be used to generate the project webpage on
                 'should be written for\n'
                 'that purpose.\n'
                 '\n'
                 'Typical contents for this file would include an
                 'the project, basic\n'
                 'usage examples, etc. Generally, including the pr
                 'changelog in here is not\n'
                 'a good idea, although a simple "What\'s New" sec
                 'most recent version\n'
                 'may be appropriate.',
 'description_content_type': None,
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {...},
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
 'maintainer_email': None,
 'name': 'sampleproject',
 'package_url': 'https://pypi.org/project/sampleproject/',
 'platform': 'UNKNOWN',
 'project_url': 'https://pypi.org/project/sampleproject/',
 'project_urls': {...},
 'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
 'requires_dist': None,
 'requires_python': None,
 'summary': 'A sample Python project',
 'version': '1.2.0'}
```

Além disso, a largura *width* máxima do caractere pode ser sugerida. Se um objeto longo não puder ser dividido, a largura especificada será excedida:

```
>>> pprint.pp(project_info, depth=1, width=60)
{'author': 'The Python Packaging Authority',
 'author_email': 'pypa-dev@googlegroups.com',
 'bugtrack_url': None,
 'classifiers': [...],
 'description': 'A sample Python project\n'
                '=======================\n'
                '\n'
```

```
                 'This is the description file for the '
                 'project.\n'
                 '\n'
                 'The file should use UTF-8 encoding and be '
                 'written using ReStructured Text. It\n'
                 'will be used to generate the project '
                 'webpage on PyPI, and should be written '
                 'for\n'
                 'that purpose.\n'
                 '\n'
                 'Typical contents for this file would '
                 'include an overview of the project, '
                 'basic\n'
                 'usage examples, etc. Generally, including '
                 'the project changelog in here is not\n'
                 'a good idea, although a simple "What\'s '
                 'New" section for the most recent version\n'
                 'may be appropriate.',
 'description_content_type': None,
 'docs_url': None,
 'download_url': 'UNKNOWN',
 'downloads': {...},
 'home_page': 'https://github.com/pypa/sampleproject',
 'keywords': 'sample setuptools development',
 'license': 'MIT',
 'maintainer': None,
 'maintainer_email': None,
 'name': 'sampleproject',
 'package_url': 'https://pypi.org/project/sampleproject/',
 'platform': 'UNKNOWN',
 'project_url': 'https://pypi.org/project/sampleproject/',
 'project_urls': {...},
 'release_url': 'https://pypi.org/project/sampleproject/1.2.0/',
 'requires_dist': None,
 'requires_python': None,
 'summary': 'A sample Python project',
 'version': '1.2.0'}
```

Lastly, we can format like pretty-printed JSON with the *expand* parameter. Best results are achieved with a higher *indent* value:

```
>>> pprint.pp(project_info, indent=4, expand=True)
{
```

'author': 'The Python Packaging Authority',
'author_email': 'pypa-dev@googlegroups.com',
'bugtrack_url': None,
'classifiers': [
   'Development Status :: 3 - Alpha',
   'Intended Audience :: Developers',
   'License :: OSI Approved :: MIT License',
   'Programming Language :: Python :: 2',
   'Programming Language :: Python :: 2.6',
   'Programming Language :: Python :: 2.7',
   'Programming Language :: Python :: 3',
   'Programming Language :: Python :: 3.2',
   'Programming Language :: Python :: 3.3',
   'Programming Language :: Python :: 3.4',
   'Topic :: Software Development :: Build Tools',
],
'description': 'A sample Python project\n'
'=======================\n'
'\n'
'This is the description file for the project.\n'
'\n'
'The file should use UTF-8 encoding and be written using ReStr
'Text. It\n'
'will be used to generate the project webpage on PyPI, and sho
'written for\n'
'that purpose.\n'
'\n'
'Typical contents for this file would include an overview of t
'basic\n'
'usage examples, etc. Generally, including the project changel
'is not\n'
'a good idea, although a simple "What\'s New" section for the
'version\n'
'may be appropriate.',
'description_content_type': None,
'docs_url': None,
'download_url': 'UNKNOWN',
'downloads': {'last_day': -1, 'last_month': -1, 'last_week': -
'dynamic': None,
'home_page': 'https://github.com/pypa/sampleproject',
'keywords': 'sample setuptools development',
'license': 'MIT',
'license_expression': None,
'license_files': None,

```
    'maintainer': None,
    'maintainer_email': None,
    'name': 'sampleproject',
    'package_url': 'https://pypi.org/project/sampleproject/',
    'platform': 'UNKNOWN',
    'project_url': 'https://pypi.org/project/sampleproject/',
    'project_urls': {
       'Download': 'UNKNOWN',
       'Homepage': 'https://github.com/pypa/sampleproject',
    },
    'provides_extra': None,
    'release_url': 'https://pypi.org/project/sampleproject/1.2.0/'
    'requires_dist': None,
    'requires_python': None,
    'summary': 'A sample Python project',
    'version': '1.2.0',
    'yanked': False,
    'yanked_reason': None,
}
```

`reprlib` — Implementação alternativa à `repr()`

Código-fonte: Lib/reprlib.py [https://github.com/python/cpython/tree/3.15/Lib/reprlib.py]

O módulo **`reprlib`** fornece um meio de produzir representações de objetos com limites no tamanho das strings resultantes. Isso é usado no depurador Python e pode ser útil em outros contextos também.

Este módulo fornece uma classe, uma instância e uma função.

class reprlib.Repr(*, *maxlevel=6*, *maxtuple=6*, *maxlist=6*, *maxarray=5*, *maxdict=4*, *maxset=6*, *maxfrozenset=6*, *maxdeque=6*, *maxstring=30*, *maxlong=40*, *maxother=30*, *fillvalue='...'*, *indent=None*)

Classe que fornece serviços de formatação úteis na implementação de funções semelhantes à embutida **`repr()`**; limites de tamanho para diferentes tipos de objetos são adicionados para evitar a geração de representações excessivamente longas.

Os argumentos nomeados do construtor podem ser usados como um atalho para definir os atributos da instância de **`Repr`**. O que significa que a inicialização a seguir:

```
aRepr = reprlib.Repr(maxlevel=3)
```

É equivalente a:

```
aRepr = reprlib.Repr()
aRepr.maxlevel = 3
```

Consulte a seção Objetos Repr para obter mais informações sobre atributos **`Repr`**.

Alterado na versão 3.12: Permite que atributos sejam definidos por meio de argumentos nomeados.

reprlib.aRepr

Esta é uma instância de **Repr** que é usada para fornecer a função **repr()** descrita abaixo. Alterar os atributos deste objeto afetará os limites de tamanho usados por **repr()** e pelo depurador Python.

reprlib.repr(*obj*)

Este é o método **repr()** de `aRepr`. Ele retorna uma string semelhante à retornada pela função embutida de mesmo nome, mas com limites na maioria dos tamanhos.

Além das ferramentas de limitação de tamanho, o módulo também fornece um decorador para detectar chamadas recursivas para **__repr__()** e substituir uma string de espaço reservado.

@reprlib.recursive_repr(*fillvalue='...'*)

Decorador para métodos **__repr__()** para detectar chamadas recursivas dentro da mesma thread. Se uma chamada recursiva for feita, o *fillvalue* é retornado; caso contrário, a chamada usual **__repr__()** é feita. Por exemplo:

```
>>> from reprlib import recursive_repr
>>> class MyList(list):
...     @recursive_repr()
...     def __repr__(self):
...         return '<' + '|'.join(map(repr, self)) + '>'
...
>>> m = MyList('abc')
>>> m.append(m)
>>> m.append('x')
>>> print(m)
<'a'|'b'|'c'|...|'x'>
```

Adicionado na versão 3.2.

Objetos Repr

Instâncias `Repr` fornecem vários atributos que podem ser usados para fornecer limites de tamanho para as representações de diferentes tipos de objetos e métodos que formatam tipos de objetos específicos.

Repr.fillvalue

Esta string é exibida para referências recursivas. O padrão é `....`

Adicionado na versão 3.11.

Repr.maxlevel

Limite de profundidade na criação de representações recursivas. O padrão é `6`.

Repr.maxdict
Repr.maxlist
Repr.maxtuple
Repr.maxset
Repr.maxfrozenset
Repr.maxdeque
Repr.maxarray

Limites no número de entradas representadas para o tipo de objeto nomeado. O padrão é `4` para `maxdict`, `5` para `maxarray` e `6` para os demais.

Repr.maxlong

Número máximo de caracteres na representação para um inteiro. Os dígitos são omitidos a partir do meio. O padrão é `40`.

Repr.maxstring

Limite no número de caracteres na representação da string. Observe que a representação “normal” da string é usada como fonte de caracteres: se sequências de escape forem necessárias na representação, elas poderão ser distorcidas quando a representação for encurtada. O padrão é `30`.

Repr.maxother

Este limite é usado para controlar o tamanho de tipos de objetos para os quais nenhum método de formatação específico está disponível no

objeto **Repr**. Ele é aplicado de maneira semelhante a **maxstring**. O padrão é `20`.

Repr.indent

Se este atributo for definido como `None` (o padrão), a saída será formatada sem quebras de linha ou recuo, como o padrão **repr()**. Por exemplo:

```
>>> example = [
...     1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6
>>> import reprlib
>>> aRepr = reprlib.Repr()
>>> print(aRepr.repr(example))
[1, 'spam', {'a': 2, 'b': 'spam eggs', 'c': {3: 4.5, 6: []}},
```

Se **indent** for definido como uma string, cada nível de recursão será colocado em sua própria linha, recuado por essa string:

```
>>> aRepr.indent = '-->'
>>> print(aRepr.repr(example))
[
-->1,
-->'spam',
-->{
-->-->'a': 2,
-->-->'b': 'spam eggs',
-->-->'c': {
-->-->-->3: 4.5,
-->-->-->6: [],
-->-->},
-->},
-->'ham',
]
```

Definir **indent** como um valor inteiro positivo se comporta como se fosse definido como uma string com esse número de espaços:

```
>>> aRepr.indent = 4
>>> print(aRepr.repr(example))
[
    1,
```

```
        'spam',
        {
            'a': 2,
            'b': 'spam eggs',
            'c': {
                3: 4.5,
                6: [],
            },
        },
        'ham',
    ]
```

Adicionado na versão 3.12.

Repr.repr(*obj*)

O equivalente ao `repr()` embutido que usa a formatação imposta pela instância.

Repr.repr1(*obj*, *level*)

Implementação recursiva usada por `repr()`. Esta usa o tipo de *obj* para determinar qual método de formatação chamar, passando *obj* e *level*. Os métodos específicos de tipo devem chamar `repr1()` para realizar a formatação recursiva, com `level - 1` para o valor de *level* na chamada recursiva.

Repr.repr_TYPE(*obj*, *level*)

Métodos de formatação para tipos específicos são implementados como métodos com um nome baseado no nome do tipo. No nome do método, **TYPE** é substituído por `'_'.join(type(obj).__name__.split())`. O despacho para esses métodos é tratado por `repr1()`. Métodos específicos de tipo que precisam formatar um valor recursivamente devem chamar `self.repr1(subobj, level - 1)`.

Estendendo objetos Repr

O uso de despacho dinâmico por `Repr.repr1()` permite que subclasses de `Repr` adicionem suporte para tipos de objetos embutidos adicionais ou

modifiquem o tratamento de tipos já suportados. Este exemplo mostra como suporte especial para objetos arquivo pode ser adicionado:

```
import reprlib
import sys

class MyRepr(reprlib.Repr):

    def repr_TextIOWrapper(self, obj, level):
        if obj.name in {'<stdin>', '<stdout>', '<stderr>'}:
            return obj.name
        return repr(obj)

aRepr = MyRepr()
print(aRepr.repr(sys.stdin))         # exibe '<stdin>'
```

```
<stdin>
```

`enum` — Suporte a enumerações

Adicionado na versão 3.4.

Código-fonte: Lib/enum.py [https://github.com/python/cpython/tree/3.15/Lib/enum.py]

Uma enumeração:

- é um conjunto de nomes simbólicos (membros) vinculados a valores únicos
- pode ser iterado para retornar seus membros canônicos (ou seja, não incluindo apelidos) na ordem de definição
- usa a sintaxe *call* para retornar membros por valor
- usa a sintaxe *index* para retornar membros por nome

Important

Esta página contêm informação de referência da API. Para informação tutorial e discussão de tópicos mais avançados, consulte

- Tutorial básico
- Tutorial avançado
- Livro de receitas de enum

As enumerações são criadas usando a sintaxe de `class` ou usando a sintaxe de chamada de função:

```
>>> from enum import Enum

>>> # sintaxe de classe
>>> class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3

>>> # sintaxe funcional
>>> Color = Enum('Color', [('RED', 1), ('GREEN', 2), ('BLUE', 3)
```

Embora possamos usar a sintaxe `class` para criar Enums, Enums não são classes Python normais. Veja Como Enums são diferentes? para mais detalhes.

Nota

Nomenclatura

- A classe `Color` é uma *enumeração* (ou *enum*)
- Os atributos `Color.RED`, `Color.GREEN`, etc., são *membros de enumeração* (ou *membros*) e são constantes funcionalmente.
- Os membros enum têm *nomes* e *valores* (o nome de `Color.RED` é `RED`, o valor de `Color.BLUE` é `3`, etc.)

Conteúdo do módulo

`EnumType`

O `type` para Enum e suas subclasses.

`Enum`

Classe base para criação de constantes enumeradas.

`IntEnum`

Classe base para criar constantes enumeradas que também são subclasses de `int`. (Notas)

`StrEnum`

Classe base para criar constantes enumeradas que também são subclasses de `str`. (Notas)

`Flag`

Classe base para criar constantes enumeradas que podem ser combinadas usando operações bit a bit sem perder sua associação `Flag`.

`IntFlag`

Classe base para criar constantes enumeradas que podem ser combinadas usando operadores bit a bit sem perder sua associação `IntFlag`. Membros de **`IntFlag`** também são subclasses de `int`. (Notas)

`ReprEnum`

Usado por `IntEnum`, `StrEnum` e `IntFlag` para manter o `str()` do tipo misto.

`EnumCheck`

Uma enumeração com os valores `CONTINUOUS`, `NAMED_FLAGS` e `UNIQUE`, para uso com `verify()` para garantir que várias restrições sejam atendidas por uma determinada enumeração.

`FlagBoundary`

Uma enumeração com os valores `STRICT`, `CONFORM`, `EJECT` e `KEEP` que permite um controle mais refinado sobre como valores inválidos são tratados em uma enumeração.

`EnumDict`

Uma subclasse de `dict` para uso ao criar subclasse de `EnumType`.

`auto`

As instâncias são substituídas por um valor apropriado para membros Enum. `StrEnum` assume como padrão a versão em minúsculas do nome do membro, enquanto outros Enums assumem como padrão 1 e aumentam a partir daí.

`@~enum.property`

Permite que membros `Enum` tenham atributos sem conflitar com nomes de membros. Os atributos `value` e `name` são implementados dessa forma.

`@unique`

Decorador de classe Enum que garante que apenas um nome seja vinculado a cada valor.

`@verify`

Decorador de classe Enum que verifica restrições selecionáveis pelo usuário em uma enumeração.

`@member`

Torna `obj` um membro. Pode ser usado como um decorador.

`@nonmember`

Não torna `obj` um membro. Pode ser usado como um decorador.

`@global_enum`

Modifica `str()` e `repr()` de uma enumeração para mostrar seus membros como pertencentes ao módulo em vez de sua classe, e exporta os membros da enumeração para o espaço de nomes global.

`show_flag_values()`

Retorna uma lista de todos os inteiros de potência de dois contidos em um sinalizador.

`enum.bin()`

Like built-in **bin()**, except negative values are represented in two's complement, and the leading bit always indicates sign (`0` implies positive, `1` implies negative).

Adicionado na versão 3.6: `Flag`, `IntFlag`, `auto`

Adicionado na versão 3.11: `StrEnum`, `EnumCheck`, `ReprEnum`, `FlagBoundary`, `property`, `member`, `nonmember`, `global_enum`, `show_flag_values`

Adicionado na versão 3.13: `EnumDict`

Data types

class enum.EnumType

EnumType é a metaclasse para enumerações *enum*. É possível criar subclasse de *EnumType* – veja Criando subclasses de EnumType para detalhes.

`EnumType` é responsável por definir os métodos **__repr__()**, **__str__()**, **__format__()** e **__reduce__()** corretos no *enum* final, bem como criar os membros do enum, manipular adequadamente as duplicatas, fornecer iteração sobre a classe do enum, etc.

Adicionado na versão 3.11: Antes da versão 3.11, `EnumType` era chamado de `EnumMeta`, que ainda está disponível como um apelido.

__call__(*cls*, *value*, *names=None*, ***, *module=None*, *qualname=None*, *type=None*, *start=1*, *boundary=None*)

Este método é chamado de formas diferentes:

- para procurar um membro existente:

cls::	A classe enum sendo chamada.
value::	O valor para procurar.

- para usar a enumeração `cls` para criar uma nova enumeração (somente se a enumeração existente não tiver nenhum membro):

cls::	A classe enum sendo chamada.
value::	O nome do novo Enum para criar.
names::	Os nomes/valores dos membros para o novo Enum.
module::	O nome do módulo no qual o novo Enum é criado.
qualname::	O local real no módulo onde este Enum pode ser encontrado.
type::	Um tipo de mix-in para o novo Enum.
start::	O primeiro valor inteiro para o Enum (usado por `auto`).
boundary::	Como lidar com valores fora do intervalo de operações de bits (somente `Flag`).

__contains__(*cls*, *member*)

Retorna `True` se o membro pertencer a `cls`:

```
>>> some_var = Color.RED
>>> some_var in Color
True
>>> Color.RED.value in Color
True
```

Alterado na versão 3.12: Antes do Python 3.12, um `TypeError` era levantado se um membro não Enum fosse usado em uma verificação de contenção.

__dir__(*cls*)

Retorna `['__class__', '__doc__', '__members__', '__module__']` e os nomes dos membros em *cls*:

```
>>> dir(Color)
['BLUE', 'GREEN', 'RED', '__class__', '__contains__', '__
```

__getitem__(*cls*, *name*)

Retorna o membro Enum em *cls* correspondente a *name* ou levanta `KeyError`:

```
>>> Color['BLUE']
<Color.BLUE: 3>
```

__iter__(*cls*)

Retorna cada membro em *cls* na ordem de definição:

```
>>> list(Color)
[<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 3>]
```

__len__(*cls*)

Returns the number of members in *cls*:

```
>>> len(Color)
3
```

__members__

Retorna um mapeamento de cada nome de enumeração para seu membro, incluindo apelidos

__reversed__(*cls*)

Retorna cada membro em *cls* na ordem inversa de definição:

```
>>> list(reversed(Color))
[<Color.BLUE: 3>, <Color.GREEN: 2>, <Color.RED: 1>]
```

class enum.Enum

Enum é a classe base para todas as enumerações *enum*.

name

O nome usado para definir o membro `Enum`:

```
>>> Color.BLUE.name
'BLUE'
```

value

O valor dado ao membro `Enum`:

```
>>> Color.RED.value
1
```

O valor do membro pode ser definido em `__new__()`.

Nota

Valores de membros de Enum

Os valores dos membros podem ser qualquer coisa: `int`, `str`, etc. Se o valor exato não for importante, você pode usar instâncias de `auto` e um valor apropriado será escolhido para você. Consulte `auto` para obter detalhes.

Embora valores mutáveis/não hasheáveis, como `dict`, `list` ou um mutável `dataclass`, possam ser usados, eles terão um impacto quadrático no desempenho durante a criação em relação ao número total de valores mutáveis/não hasheáveis na enumeração.

name

Nome do membro.

value

O valor do membro pode ser definido em `__new__()`.

order

Não é mais usado, mantido para compatibilidade com versões anteriores. (atributo de classe, removido durante a criação da classe).

The `_order_` attribute can be provided to help keep Python 2 / Python 3 code in sync. It will be checked against the actual order of

the enumeration and raise an error if the two do not match:

```
>>> class Color(Enum):
...     _order_ = 'RED GREEN BLUE'
...     RED = 1
...     BLUE = 3
...     GREEN = 2
...
Traceback (most recent call last):
...
TypeError: member order does not match _order_:
   ['RED', 'BLUE', 'GREEN']
   ['RED', 'GREEN', 'BLUE']
```

Nota

In Python 2 code the **`_order_`** attribute is necessary as definition order is lost before it can be recorded.

Adicionado na versão 3.6.

ignore

`_ignore_` é usado somente durante a criação e é removido da enumeração quando a criação é concluída.

`_ignore_` é uma lista de nomes que não se tornarão membros e cujos nomes também serão removidos da enumeração concluída. Veja TimePeriod para um exemplo.

Adicionado na versão 3.7.

__dir__(*self*)

Retorna `['__class__', '__doc__', '__module__', 'name', 'value']` e quaisquer métodos públicos definidos em *self.__class__*:

```
>>> from enum import Enum
>>> import datetime as dt
>>> class Weekday(Enum):
...     MONDAY = 1
```

```
...     TUESDAY = 2
...     WEDNESDAY = 3
...     THURSDAY = 4
...     FRIDAY = 5
...     SATURDAY = 6
...     SUNDAY = 7
...     @classmethod
...     def today(cls):
...         print(f'today is {cls(dt.date.today().isoweek
...
>>> dir(Weekday.SATURDAY)
['__class__', '__doc__', '__eq__', '__hash__', '__module__
```

_generate_next_value_(*name*, *start*, *count*, *last_values*)

nome::	O nome do membro que está sendo definido (por exemplo, 'RED').
start::	O valor inicial do Enum; o padrão é 1.
count::	O número de membros atualmente definido, sem incluir este.
last_values::	Uma lista de valores anteriores.

A *staticmethod* that is used to determine the next value returned by `auto`.

Nota

For standard `Enum` classes the next value chosen is the highest value seen incremented by one.

For `Flag` classes the next value chosen will be the next highest power-of-two.

This method may be overridden, e.g.:

```
>>> from enum import auto, Enum
>>> class PowersOfThree(Enum):
...     @staticmethod
...     def _generate_next_value_(name, start, count, las
```

```
...         return 3 ** (count + 1)
...     FIRST = auto()
...     SECOND = auto()
...
>>> PowersOfThree.SECOND.value
9
```

Adicionado na versão 3.6.

Alterado na versão 3.13: Versões anteriores usariam o último valor visualizado em vez do maior valor.

__init__(*self*, **args*, ***kwds*)

Por padrão, não faz nada. Se vários valores forem fornecidos na atribuição de membro, esses valores se tornarão argumentos separados para `__init__`; por exemplo.:

```
>>> from enum import Enum
>>> class Weekday(Enum):
...     MONDAY = 1, 'Mon'
```

`Weekday.__init__()` seria chamado como `Weekday.__init__(self, 1, 'Mon')`

__init_subclass__(*cls*, ***kwds*)

Um *classmethod* usado para configurar subclasses subsequentes. Por padrão, não faz nada.

missing(*cls*, *value*)

Um *classmethod* para procurar valores não encontrados em *cls*. Por padrão, ele não faz nada, mas pode ser substituído para implementar um comportamento de pesquisa personalizado:

```
>>> from enum import auto, StrEnum
>>> class Build(StrEnum):
...     DEBUG = auto()
...     OPTIMIZED = auto()
...     @classmethod
...     def _missing_(cls, value):
```

```
...         value = value.lower()
...         for member in cls:
...             if member.value == value:
...                 return member
...         return None
...
>>> Build.DEBUG.value
'debug'
>>> Build('deBUG')
<Build.DEBUG: 'debug'>
```

Adicionado na versão 3.6.

__new__(*cls*, **args*, ***kwds*)

By default, doesn't exist. If specified, either in the enum class definition or in a mixin class (such as `int`), all values given in the member assignment will be passed; e.g.

```
>>> from enum import Enum
>>> class MyIntEnum(int, Enum):
...     TWENTYSIX = '1a', 16
```

results in the call `int('1a', 16)` and a value of `26` for the member.

Nota

When writing a custom `__new__`, do not use `super().__new__` – call the appropriate `__new__` instead.

__repr__(*self*)

Returns the string used for *repr()* calls. By default, returns the *Enum* name, member name, and value, but can be overridden:

```
>>> from enum import auto, Enum
>>> class OtherStyle(Enum):
...     ALTERNATE = auto()
...     OTHER = auto()
...     SOMETHING_ELSE = auto()
...     def __repr__(self):
...         cls_name = self.__class__.__name__
```

```
...         return f'{cls_name}.{self.name}'
...
>>> OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f"{Ot
(OtherStyle.ALTERNATE, 'OtherStyle.ALTERNATE', 'OtherStyle
```

__str__(*self*)

Returns the string used for *str()* calls. By default, returns the *Enum* name and member name, but can be overridden:

```
>>> from enum import auto, Enum
>>> class OtherStyle(Enum):
...     ALTERNATE = auto()
...     OTHER = auto()
...     SOMETHING_ELSE = auto()
...     def __str__(self):
...         return f'{self.name}'
...
>>> OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f"{Ot
(<OtherStyle.ALTERNATE: 1>, 'ALTERNATE', 'ALTERNATE')
```

__format__(*self*)

Returns the string used for *format()* and *f-string* calls. By default, returns `__str__()` return value, but can be overridden:

```
>>> from enum import auto, Enum
>>> class OtherStyle(Enum):
...     ALTERNATE = auto()
...     OTHER = auto()
...     SOMETHING_ELSE = auto()
...     def __format__(self, spec):
...         return f'{self.name}'
...
>>> OtherStyle.ALTERNATE, str(OtherStyle.ALTERNATE), f"{Ot
(<OtherStyle.ALTERNATE: 1>, 'OtherStyle.ALTERNATE', 'ALTER
```

Nota

Using `auto` with `Enum` results in integers of increasing value, starting with `1`.

Alterado na versão 3.12: Added Suporte a dataclass.

_add_alias_()

Adds a new name as an alias to an existing member:

```
>>> Color.RED._add_alias_("ERROR")
>>> Color.ERROR
<Color.RED: 1>
```

Raises a **NameError** if the name is already assigned to a different member.

Adicionado na versão 3.13.

_add_value_alias_()

Adds a new value as an alias to an existing member:

```
>>> Color.RED._add_value_alias_(42)
>>> Color(42)
<Color.RED: 1>
```

Raises a **ValueError** if the value is already linked with a different member.
See MultiValueEnum for an example.

Adicionado na versão 3.13.

class enum.IntEnum

IntEnum is the same as **Enum**, but its members are also integers and can be used anywhere that an integer can be used. If any integer operation is performed with an *IntEnum* member, the resulting value loses its enumeration status.

```
>>> from enum import IntEnum
>>> class Number(IntEnum):
...     ONE = 1
...     TWO = 2
...     THREE = 3
...
```

```
>>> Number.THREE
<Number.THREE: 3>
>>> Number.ONE + Number.TWO
3
>>> Number.THREE + 5
8
>>> Number.THREE == 3
True
```

Nota

Using **auto** with **IntEnum** results in integers of increasing value, starting with 1.

Alterado na versão 3.11: **__str__()** is now **int.__str__()** to better support the *replacement of existing constants* use-case. **__format__()** was already **int.__format__()** for that same reason.

class enum.StrEnum

StrEnum is the same as **Enum**, but its members are also strings and can be used in most of the same places that a string can be used. The result of any string operation performed on or with a *StrEnum* member is not part of the enumeration.

```
>>> from enum import StrEnum, auto
>>> class Color(StrEnum):
...     RED = 'r'
...     GREEN = 'g'
...     BLUE = 'b'
...     UNKNOWN = auto()
...
>>> Color.RED
<Color.RED: 'r'>
>>> Color.UNKNOWN
<Color.UNKNOWN: 'unknown'>
>>> str(Color.UNKNOWN)
'unknown'
```

Nota

There are places in the stdlib that check for an exact `str` instead of a `str` subclass (i.e. `type(unknown) == str` instead of `isinstance(unknown, str)`), and in those locations you will need to use `str(MyStrEnum.MY_MEMBER)`.

Nota

Using `auto` with `StrEnum` results in the lower-cased member name as the value.

Nota

`__str__()` is `str.__str__()` to better support the *replacement of existing constants* use-case. `__format__()` is likewise `str.__format__()` for that same reason.

Adicionado na versão 3.11.

class enum.Flag

`Flag` is the same as `Enum`, but its members support the bitwise operators `&` (*AND*), `|` (*OR*), `^` (*XOR*), and `~` (*INVERT*); the results of those operations are (aliases of) members of the enumeration.

__contains__(*self*, *value*)

Returns *True* if value is in self:

```
>>> from enum import Flag, auto
>>> class Color(Flag):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> purple = Color.RED | Color.BLUE
>>> white = Color.RED | Color.GREEN | Color.BLUE
>>> Color.GREEN in purple
False
>>> Color.GREEN in white
True
```

```
>>> purple in white
True
>>> white in purple
False
```

__iter__(*self*)

Returns all contained non-alias members:

```
>>> list(Color.RED)
[<Color.RED: 1>]
>>> list(purple)
[<Color.RED: 1>, <Color.BLUE: 4>]
```

Adicionado na versão 3.11.

__len__(*self*)

Returns number of members in flag:

```
>>> len(Color.GREEN)
1
>>> len(white)
3
```

Adicionado na versão 3.11.

__bool__(*self*)

Returns *True* if any members in flag, *False* otherwise:

```
>>> bool(Color.GREEN)
True
>>> bool(white)
True
>>> black = Color(0)
>>> bool(black)
False
```

__or__(*self*, *other*)

Returns current flag binary or'ed with other:

```
>>> Color.RED | Color.GREEN
<Color.RED|GREEN: 3>
```

__and__(*self*, *other*)

Returns current flag binary and'ed with other:

```
>>> purple & white
<Color.RED|BLUE: 5>
>>> purple & Color.GREEN
<Color: 0>
```

__xor__(*self*, *other*)

Returns current flag binary xor'ed with other:

```
>>> purple ^ white
<Color.GREEN: 2>
>>> purple ^ Color.GREEN
<Color.RED|GREEN|BLUE: 7>
```

__invert__(*self*)

Returns all the flags in *type(self)* that are not in *self*:

```
>>> ~white
<Color: 0>
>>> ~purple
<Color.GREEN: 2>
>>> ~Color.RED
<Color.GREEN|BLUE: 6>
```

_numeric_repr_()

Function used to format any remaining unnamed numeric values. Default is the value's repr; common choices are `hex()` and `oct()`.

Nota

Using `auto` with `Flag` results in integers that are powers of two, starting with `1`.

Alterado na versão 3.11: The *repr()* of zero-valued flags has changed. It is now:

```
>>> Color(0)
<Color: 0>
```

class enum.IntFlag

`IntFlag` is the same as **`Flag`**, but its members are also integers and can be used anywhere that an integer can be used.

```
>>> from enum import IntFlag, auto
>>> class Color(IntFlag):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> Color.RED & 2
<Color: 0>
>>> Color.RED | 2
<Color.RED|GREEN: 3>
```

If any integer operation is performed with an *IntFlag* member, the result is not an *IntFlag*:

```
>>> Color.RED + 2
3
```

If a **`Flag`** operation is performed with an *IntFlag* member and:

- the result is a valid *IntFlag*: an *IntFlag* is returned
- the result is not a valid *IntFlag*: the result depends on the **`FlagBoundary`** setting

The **`repr()`** of unnamed zero-valued flags has changed. It is now:

```
>>> Color(0)
<Color: 0>
```

Nota

Using **`auto`** with **`IntFlag`** results in integers that are powers of two, starting with `1`.

Alterado na versão 3.11: `__str__()` is now `int.__str__()` to better support the *replacement of existing constants* use-case. `__format__()` was already `int.__format__()` for that same reason.

Inversion of an `IntFlag` now returns a positive value that is the union of all flags not in the given flag, rather than a negative value. This matches the existing `Flag` behavior.

class enum.ReprEnum

`ReprEnum` uses the `repr()` of `Enum`, but the `str()` of the mixed-in data type:

- `int.__str__()` for `IntEnum` and `IntFlag`
- `str.__str__()` for `StrEnum`

Inherit from `ReprEnum` to keep the `str()` / `format()` of the mixed-in data type instead of using the `Enum`-default `str()`.

Adicionado na versão 3.11.

class enum.EnumCheck

EnumCheck contains the options used by the `verify()` decorator to ensure various constraints; failed constraints result in a `ValueError`.

UNIQUE

Ensure that each value has only one name:

```
>>> from enum import Enum, verify, UNIQUE
>>> @verify(UNIQUE)
... class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3
...     CRIMSON = 1
Traceback (most recent call last):
...
ValueError: aliases found in <enum 'Color'>: CRIMSON -> RI
```

CONTINUOUS

Ensure that there are no missing values between the lowest-valued member and the highest-valued member:

```
>>> from enum import Enum, verify, CONTINUOUS
>>> @verify(CONTINUOUS)
... class Color(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 5
Traceback (most recent call last):
...
ValueError: invalid enum 'Color': missing values 3, 4
```

NAMED_FLAGS

Ensure that any flag groups/masks contain only named flags – useful when values are specified instead of being generated by `auto()`:

```
>>> from enum import Flag, verify, NAMED_FLAGS
>>> @verify(NAMED_FLAGS)
... class Color(Flag):
...     RED = 1
...     GREEN = 2
...     BLUE = 4
...     WHITE = 15
...     NEON = 31
Traceback (most recent call last):
...
ValueError: invalid Flag 'Color': aliases WHITE and NEON a
```

Nota

CONTINUOUS and NAMED_FLAGS are designed to work with integer-valued members.

Adicionado na versão 3.11.

class enum.FlagBoundary

`FlagBoundary` controls how out-of-range values are handled in `Flag` and its subclasses.

STRICT

Out-of-range values cause a `ValueError` to be raised. This is the default for `Flag`:

```
>>> from enum import Flag, STRICT, auto
>>> class StrictFlag(Flag, boundary=STRICT):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> StrictFlag(2**2 + 2**4)
Traceback (most recent call last):
...
ValueError: <flag 'StrictFlag'> invalid value 20
    given 0b0 10100
  allowed 0b0 00111
```

CONFORM

Out-of-range values have invalid values removed, leaving a valid `Flag` value:

```
>>> from enum import Flag, CONFORM, auto
>>> class ConformFlag(Flag, boundary=CONFORM):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> ConformFlag(2**2 + 2**4)
<ConformFlag.BLUE: 4>
```

EJECT

Out-of-range values lose their `Flag` membership and revert to `int`.

```
>>> from enum import Flag, EJECT, auto
>>> class EjectFlag(Flag, boundary=EJECT):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> EjectFlag(2**2 + 2**4)
20
```

KEEP

Out-of-range values are kept, and the **Flag** membership is kept. This is the default for **IntFlag**:

```
>>> from enum import Flag, KEEP, auto
>>> class KeepFlag(Flag, boundary=KEEP):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...
>>> KeepFlag(2**2 + 2**4)
<KeepFlag.BLUE|16: 20>
```

Adicionado na versão 3.11.

class enum.EnumDict

EnumDict is a subclass of **dict** that is used as the namespace for defining enum classes (see Preparando o espaço de nomes da classe). It is exposed to allow subclasses of **EnumType** with advanced behavior like having multiple values per member. It should be called with the name of the enum class being created, otherwise private names and internal classes will not be handled correctly.

Note that only the **MutableMapping** interface (**__setitem__()** and **update()**) is overridden. It may be possible to bypass the checks using other **dict** operations like **|=**.

member_names

A list of member names.

Adicionado na versão 3.13.

Nomes __dunder__ suportados

__members__ is a read-only ordered mapping of `member_name:member` items. It is only available on the class.

`__new__()`, if specified, must create and return the enum members; it is also a very good idea to set the member's `_value_` appropriately. Once all the members are created it is no longer used.

Nomes `_sunder_` suportados

- `_name_` – name of the member
- `_value_` – value of the member; can be set in `__new__`
- `_missing_()` – a lookup function used when a value is not found; may be overridden
- `_ignore_` – a list of names, either as a `list` or a `str`, that will not be transformed into members, and will be removed from the final class
- `_order_` – no longer used, kept for backward compatibility (class attribute, removed during class creation)
- `_generate_next_value_()` – used to get an appropriate value for an enum member; may be overridden
- `_add_alias_()` – adds a new name as an alias to an existing member.
- `_add_value_alias_()` – adds a new value as an alias to an existing member.
- While `_sunder_` names are generally reserved for the further development of the `Enum` class and can not be used, some are explicitly allowed:
  - `_repr_*` (e.g. `_repr_html_`), as used in IPython's rich display [https://ipython.readthedocs.io/en/stable/config/integrating.html#rich-display]

Adicionado na versão 3.6: `_missing_`, `_order_`, `_generate_next_value_`

Adicionado na versão 3.7: `_ignore_`

Adicionado na versão 3.13: `_add_alias_`, `_add_value_alias_`, `_repr_*`

Utilities and decorators

class enum.auto

auto can be used in place of a value. If used, the *Enum* machinery will call an `Enum`'s `_generate_next_value_()` to get an appropriate value.

For **Enum** and **IntEnum** that appropriate value will be the highest value seen plus one; for **Flag** and **IntFlag** it will be the first power-of-two greater than the highest value seen; for **StrEnum** it will be the lower-cased version of the member's name. Care must be taken if mixing *auto()* with manually specified values.

auto instances are only resolved when at the top level of an assignment, either by itself or as part of a tuple:

- `FIRST = auto()` will work (auto() is replaced with `1`);
- `SECOND = auto(), -2` will work (auto is replaced with `2`, so `2, -2` is used to create the `SECOND` enum member;
- `THIRD = [auto(), -3]` will *not* work (`[<auto instance>, -3]` is used to create the `THIRD` enum member)

Alterado na versão 3.11.1: In prior versions, `auto()` had to be the only thing on the assignment line to work properly.

`_generate_next_value_` can be overridden to customize the values used by *auto*.

Nota

In version 3.13 the default `_generate_next_value_` will always return the highest member value incremented by 1, and will fail if any member is an incompatible type.

@enum.property

A decorator similar to the built-in **@property**, but specifically for enumerations. It allows member attributes to have the same names as members themselves.

Nota

The *property* and the member must be defined in separate classes; for example, the *value* and *name* attributes are defined in the *Enum* class, and *Enum* subclasses can define members with the names `value` and `name`.

Adicionado na versão 3.11.

@enum.unique

A `class` decorator specifically for enumerations. It searches an enumeration's `__members__`, gathering any aliases it finds; if any are found `ValueError` is raised with the details:

```
>>> from enum import Enum, unique
>>> @unique
... class Errado(Enum):
...     UM = 1
...     DOIS = 2
...     TRES = 3
...     QUATRO = 3
...
Traceback (most recent call last):
...
ValueError: duplicate values found in <enum 'Errado'>: QUATRO
```

@enum.verify

A `class` decorator specifically for enumerations. Members from `EnumCheck` are used to specify which constraints should be checked on the decorated enumeration.

Adicionado na versão 3.11.

@enum.member

A decorator for use in enums: its target will become a member.

Adicionado na versão 3.11.

@enum.nonmember

A decorator for use in enums: its target will not become a member.

Adicionado na versão 3.11.

@enum.global_enum

A decorator to change the `str()` and `repr()` of an enum to show its members as belonging to the module instead of its class. Should only be used when the enum members are exported to the module global namespace (see `re.RegexFlag` for an example).

Adicionado na versão 3.11.

enum.show_flag_values(*value*)

Return a list of all power-of-two integers contained in a flag *value*.

Adicionado na versão 3.11.

enum.bin(*num*, *max_bits=None*)

Like built-in **`bin()`**, except negative values are represented in two's complement, and the leading bit always indicates sign (`0` implies positive, `1` implies negative).

```
>>> import enum
>>> enum.bin(10)
'0b0 1010'
>>> enum.bin(~10)   # ~10 is -11
'0b1 0101'
```

Adicionado na versão 3.11.

Notas

`IntEnum`, `StrEnum`, and `IntFlag`

These three enum types are designed to be drop-in replacements for existing integer- and string-based values; as such, they have extra limitations:

- `__str__` uses the value and not the name of the enum member
- `__format__`, because it uses `__str__`, will also use the value of the enum member instead of its name

If you do not need/want those limitations, you can either create your own base class by mixing in the `int` or `str` type yourself:

```
>>> from enum import Enum
>>> class MyIntEnum(int, Enum):
...     pass
```

or you can reassign the appropriate **`str()`**, etc., in your enum:

```
>>> from enum import Enum, IntEnum
>>> class MyIntEnum(IntEnum):
...     __str__ = Enum.__str__
```

`graphlib` — Funcionalidade para operar com estruturas do tipo grafo

Código-fonte: Lib/graphlib.py
[https://github.com/python/cpython/tree/3.15/Lib/graphlib.py]

---

class graphlib.TopologicalSorter(*graph=None*)

Fornece funcionalidade para classificar topologicamente um grafo de nós hasheáveis.

Uma ordem topológica é uma ordenação linear dos vértices em um grafo, de modo que para cada aresta direcionada u -> v do vértice u ao vértice v, o vértice u vem antes do vértice v na ordenação. Por exemplo, os vértices do grafo podem representar tarefas a serem executadas e as arestas podem representar restrições de que uma tarefa deve ser executada antes de outra; neste exemplo, uma ordem topológica é apenas uma sequência válida para as tarefas. Uma ordenação topológica completa é possível se e somente se o grafo não tiver ciclos direcionados, ou seja, se for um grafo acíclico direcionado.

Se o argumento opcional *graph* for fornecido, ele deverá ser um dicionário que represente um grafo acíclico direcionado no qual as chaves sejam nós e os valores sejam iteráveis de todos os predecessores desse nó no grafo (os nós que possuem bordas que apontam para o valor na chave). Nós adicionais podem ser adicionados ao grafo usando o método `add()`.

No caso geral, as etapas necessárias para executar a classificação de um determinado grafo são as seguintes:

- Cria uma instância da classe `TopologicalSorter` com um grafo inicial opcional.
- Adiciona nós adicionais ao grafo.
- Chama o método `prepare()` no grafo.
- Enquanto `is_active()` é `True`, itera pelos nós retornados por `get_ready()` e os processa. Chama `done()` em cada nó na medida em que finaliza o processamento.

Caso apenas uma classificação imediata dos nós no grafo seja necessária e nenhum paralelismo esteja envolvido, o método de conveniência `TopologicalSorter.static_order()` pode ser usado diretamente:

```
>>> graph = {"D": {"B", "C"}, "C": {"A"}, "B": {"A"}}
>>> ts = TopologicalSorter(graph)
>>> tuple(ts.static_order())
('A', 'C', 'B', 'D')
```

A classe foi projetada para suportar facilmente o processamento paralelo dos nós à medida que eles se tornam prontos. Por exemplo:

```
topological_sorter = TopologicalSorter()

# Adiciona nodos para 'topological_sorter'...

topological_sorter.prepare()
while topological_sorter.is_active():
    for node in topological_sorter.get_ready():
        # Processos ou threads de workers obtém nós para
        # trabalhar na fila 'task_queue'.
        task_queue.put(node)

    # Quando o trabalho para um nó é feito, os workers coloca
    # nó em 'finalized_tasks_queue' para que possamos obter n
    # nós para trabalhar. A definição de 'is_active()' garant
    # neste ponto, pelo menos um nó foi colocado em 'task_que
    # que ainda não foi passado para 'done()', então este 'ge
    # bloqueio deve (eventualmente) ter sucesso. Depois de ch
    # 'done()', fazemos um laço de volta para chamar 'get_rea
    # novamente, então colocamos nós recém-liberados em
    # 'task_queue' o mais rápido possível.
```

```
    node = finalized_tasks_queue.get()
    topological_sorter.done(node)
```

add(*node*, **predecessors*)

Adiciona um novo nó e seus predecessores ao grafo. O *node* e todos os elementos em *predecessors* devem ser hasheáveis.

Se chamado várias vezes com o mesmo argumento do nó, o conjunto de dependências será a união de todas as dependências transmitidas.

É possível adicionar um nó sem dependências (*predecessors* não são fornecidos) ou fornecer uma dependência duas vezes. Se um nó que não foi fornecido anteriormente for incluído entre os *predecessors*, ele será automaticamente adicionado ao grafo sem predecessores próprios.

Levanta `ValueError` se chamado após `prepare()`.

prepare()

Marca o grafo como concluído e verifica os ciclos no grafo. Se qualquer ciclo for detectado, `CycleError` será gerado, mas `get_ready()` ainda poderá ser usado para obter o maior número possível de nós até que os ciclos bloqueiem mais progressos. Após uma chamada para esta função, o grafo não pode ser modificado e, portanto, nenhum nó pode ser adicionado usando `add()`.

Uma `ValueError` será levantada se a classificação tiver sido iniciada por `static_order()` ou `get_ready()`.

Alterado na versão 3.14: `prepare()` agora pode ser chamado mais de uma vez, desde que a classificação não tenha sido iniciada. Anteriormente, isso levantava `ValueError`.

is_active()

Retorna `True` se mais progresso puder ser feito e `False` caso contrário. É possível progredir se os ciclos não bloquearem a

resolução e ainda houver nós prontos que ainda não foram retornados por `TopologicalSorter.get_ready()` ou o número de nós marcados `TopologicalSorter.done()` é menor que o número retornado por **`TopologicalSorter.get_ready()`**.

O método `__bool__()` desta classe adia para essa função, então, em vez de:

```
if ts.is_active():
    ...
```

é possível simplesmente fazer:

```
if ts:
    ...
```

Levanta `ValueError` se chamado sem chamar `prepare()` anteriormente.

done(**nodes*)

Marca um conjunto de nós retornados por `TopologicalSorter.get_ready()` como processado, desbloqueando qualquer sucessor de cada nó em *nodes* para ser retornado no futuro por uma chamada para **`TopologicalSorter.get_ready()`**.

Levanta `ValueError` se algum nó em *nodes* já foi marcado como processado por uma chamada anterior a este método ou se um nó não foi adicionado ao grafo usando `TopologicalSorter.add()`, se chamado sem chamar **`prepare()`** ou se o nó ainda não foi retornado por `get_ready()`.

get_ready()

Retorna uma `tupla` com todos os nós que estão prontos. Inicialmente, ele retorna todos os nós sem predecessores e, uma vez marcados como processados, chamando `TopologicalSorter.done()`, novas chamadas retornarão todos os novos nós que já tenham seus antecessores já processados. Quando

não for possível fazer mais progresso, as tuplas vazias serão retornadas.

Levanta `ValueError` se chamado sem chamar `prepare()` anteriormente.

static_order()

Retorna um objeto iterador que irá iterar sobre os nós em uma ordem topológica. Ao usar este método, `prepare()` e `done()` não devem ser chamados. Este método é equivalente a:

```
def static_order(self):
    self.prepare()
    while self.is_active():
        node_group = self.get_ready()
        yield from node_group
        self.done(*node_group)
```

A ordem específica retornada pode depender da ordem específica em que os itens foram inseridos no grafo. Por exemplo:

```
>>> ts = TopologicalSorter()
>>> ts.add(3, 2, 1)
>>> ts.add(1, 0)
>>> print([*ts.static_order()])
[2, 0, 1, 3]

>>> ts2 = TopologicalSorter()
>>> ts2.add(1, 0)
>>> ts2.add(3, 2, 1)
>>> print([*ts2.static_order()])
[0, 2, 1, 3]
```

Isso se deve ao fato de que “0” e “2” estão no mesmo nível no grafo (eles teriam sido retornados na mesma chamada para `get_ready()`) e a ordem entre eles é determinada pela ordem de inserção.

Se qualquer ciclo for detectado, `CycleError` será levantada.

Adicionado na versão 3.9.

Exceções

O módulo **`graphlib`** define as seguintes classes de exceção:

exception graphlib.CycleError

Subclasse de `ValueError` levantada por `TopologicalSorter.prepare()` se houver ciclos no grafo de trabalho. Se existirem vários ciclos, apenas uma opção indefinida entre eles será relatada e incluída na exceção.

O ciclo detectado pode ser acessado através do segundo elemento no atributo `args` da instância de exceção e consiste em uma lista de nós, de modo que cada nó seja, no grafo, um predecessor imediato do próximo nó na lista. Na lista relatada, o primeiro e o último nó serão os mesmos, para deixar claro que é cíclico.

Módulos Matemáticos e Numéricos

Os módulos descritos neste capítulo fornecem funções e tipos de dados relacionados com o numérico e matemática. O módulo `numbers` define uma hierarquia abstrata de tipos numéricos. Os módulos `math` e `cmath` contêm várias funções matemáticas para números de ponto flutuante e números complexos. O módulo `decimal` suporta representações exatas de números decimais, usando aritmética de precisão arbitrária.

Os seguintes módulos estão documentados neste capítulo:

- `numbers` — Classes base abstratas numéricas
  - A torre numérica
  - Notas para implementadores de tipos
    - Adicionando mais ABCs numéricas
    - Implementando as operações aritméticas
- `math` — Funções matemáticas
  - Arimética de ponto flutuante
  - Funções de manipulação de ponto flutuante
  - Funções de potências, exponenciais e logarítmicas
  - Funções de soma e produto
  - Conversão angular
  - Funções trigonométricas
  - Funções hiperbólicas
  - Funções especiais
  - Funções de teoria dos números
  - Constantes
- `math.integer` — integer-specific mathematics functions
- `cmath` — Funções matemáticas para números complexos
  - Conversões de e para coordenadas polares
  - Funções de potência e logarítmicas
  - Funções trigonométricas
  - Funções hiperbólicas
  - Funções de classificação
  - Constantes

- `decimal` — Aritmética de ponto fixo decimal e ponto flutuante
  - Tutorial de início rápido
  - Objetos de Decimal
    - Operandos lógicos
  - Objetos de contexto
  - Constantes
  - Modos de arredondamento
  - Sinais
  - Observações sobre ponto flutuante
    - Atenuando o erro de arredondamento com precisão maior
    - Valores especiais
  - Trabalhando com threads
  - Receitas
  - FAQ sobre Decimal
- `fractions` — Números racionais
- `random` — Gera números pseudoaleatórios
  - Funções de contabilidade
  - Funções para bytes
  - Funções para inteiros
  - Funções para sequências
  - Distribuições discretas
  - Distribuições com valor real
  - Gerador alternativo
  - Notas sobre reprodutibilidade
  - Exemplos
  - Receitas
  - Uso na linha de comando
  - Exemplos na linha de comando
- `statistics` — Funções estatísticas
  - Médias e medidas de valor central
  - Medidas de espalhamento
  - Estatísticas para relações entre duas entradas
  - Detalhes das funções
  - Exceções
  - Objetos `NormalDist`
  - Exemplos e receitas
    - Problemas clássicos de probabilidade

- Entradas de Monte Carlo para simulações
- Aproximação de distribuições binomiais
- Classificador bayesiano ingênuo

`numbers` — Classes base abstratas numéricas

Código-fonte: Lib/numbers.py
[https://github.com/python/cpython/tree/3.15/Lib/numbers.py]

O módulo **`numbers`** (**PEP 3141** [https://peps.python.org/pep-3141/]) define uma hierarquia de classes base abstratas que progressivamente definem mais operações. Nenhum dos tipos definidos neste módulo se destina a ser instanciado.

class numbers.Number

A raiz da hierarquia numérica. Se você quiser apenas verificar se um argumento *x* é um número, sem se importar com o tipo, use `isinstance(x, Number)`.

A torre numérica

class numbers.Complex

As subclasses deste tipo descrevem números complexos e incluem as operações que funcionam no tipo embutido **`complex`**. Elas são: conversões para **`complex`** e **`bool`**, **`real`**, **`imag`**, `+`, `-`, `*`, `/`, `**`, **`abs()`**, **`conjugate()`**, `==` e `!=`. Todos exceto `-` e `!=` são abstratos.

real

Abstrata. Obtém o componente real deste número.

imag

Abstrata. Obtém o componente imaginário deste número.

abstractmethod conjugate()

Abstrata. Retorna o conjugado complexo. Por exemplo, `(1+3j).conjugate() == (1-3j)`.

class numbers.Real

Para `Complex`, **`Real`** adiciona as operações que funcionam em números reais.

Em suma, são: uma conversão para `float`, `math.trunc()`, `round()`, `math.floor()`, `math.ceil()`, `divmod()`, `//`, `%`, `<`, `<=`, `>` e `>=`.

Real também fornece padrão para `complex()`, `real`, `imag` e `conjugate()`.

class numbers.Rational

Estende `Real` e adiciona as propriedades `numerator` e `denominator`. Ele fornece um padrão para `float()`.

Os valores `numerator` e `denominator` devem ser instâncias de `Integral` e devem estar nos termos mais baixos com **`denominator`** positivo.

numerator

Abstrato. O numerador deste número racional.

denominator

Abstrato. O denominador deste número racional.

class numbers.Integral

Estende `Rational` e adiciona uma conversão para `int`. Fornece padrões para `float()`, `numerator` e `denominator`. Adiciona métodos abstratos para `pow()` com operações de módulo e de string de bits: `<<`, `>>`, `&`, `^`, `|`, `~`.

Notas para implementadores de tipos

Os implementadores devem ter o cuidado de tornar iguais números iguais e fazer hash deles com os mesmos valores. Isso pode ser sutil se houver duas

extensões diferentes dos números reais. Veja também Hashing de tipos numéricos.

Adicionando mais ABCs numéricas

Existem, é claro, mais ABCs possíveis para números, e isso seria uma hierarquia pobre se excluísse a possibilidade de adicioná-los. Você pode adicionar `MyFoo` entre **`Complex`** e **`Real`** com:

```
class MyFoo(Complex): ...
MyFoo.register(Real)
```

Implementando as operações aritméticas

Queremos implementar as operações aritméticas de forma que as operações de modo misto chamem uma implementação cujo autor conhecia os tipos de ambos os argumentos ou convertam ambos para o tipo embutido mais próximo e façam a operação lá. Para subtipos de **`Integral`**, isso significa que **`__add__()`** e **`__radd__()`** devem ser definidos com:

```
class MyIntegral(Integral):

    def __add__(self, other):
        if isinstance(other, MyIntegral):
            return do_my_adding_stuff(self, other)
        elif isinstance(other, OtherTypeIKnowAbout):
            return do_my_other_adding_stuff(self, other)
        else:
            return NotImplemented

    def __radd__(self, other):
        if isinstance(other, MyIntegral):
            return do_my_adding_stuff(other, self)
        elif isinstance(other, OtherTypeIKnowAbout):
            return do_my_other_adding_stuff(other, self)
        elif isinstance(other, Integral):
            return int(other) + int(self)
        elif isinstance(other, Real):
            return float(other) + float(self)
        elif isinstance(other, Complex):
```

```
            return complex(other) + complex(self)
        else:
            return NotImplemented
```

Existem 5 casos diferentes para uma operação de tipo misto em subclasses de **`Complex`**. Vou me referir a todo o código acima que não se refere a `MyIntegral` e `OtherTypeIKnowAbout` com um "modelo". `a` será uma instância de `A`, que é um subtipo de **`Complex`** (`a : A <: Complex`) e `b : B <: Complex`. Vou considerar `a + b`:

1. Se `A` define um **`__add__()`**, que aceita `b`, está tudo bem.
2. Se `A` voltar ao código padrão e tivesse que retornar um valor de **`__add__()`**, perderíamos a possibilidade de que `B` definisse um **`__radd__()`** mais inteligente, então o código padrão deve retornar **`NotImplemented`** de **`__add__()`**. (Ou `A` pode não implementar **`__add__()`**.)
3. Então, **`__radd__()`** do `B` consegue uma chance. Se ele aceitar `a`, está tudo bem.
4. Se ele recorrer ao padrão, não há mais métodos possíveis para tentar, então é aqui que a implementação padrão deve residir.
5. Se `B <: A`, Python tenta `B.__radd__` antes de `A.__add__`. Isso está ok, porque foi implementado com conhecimento de `A`, então ele pode lidar com essas instâncias antes de delegar para **`Complex`**.

Se `A <: Complex` e `B <: Real` sem compartilhar nenhum outro conhecimento, então a operação compartilhada apropriada é aquela envolvendo a **`complex`** embutida, e **`__radd__()`** de ambos chegam lá, de forma que `a+b == b+a`.

Como a maioria das operações em qualquer tipo será muito semelhante, pode ser útil definir uma função auxiliar que gera as instâncias de avanço e reversão de qualquer operador. Por exemplo, **`fractions.Fraction`** usa:

```
def _operator_fallbacks(monomorphic_operator, fallback_operator):
    def forward(a, b):
        if isinstance(b, (int, Fraction)):
            return monomorphic_operator(a, b)
        elif isinstance(b, float):
            return fallback_operator(float(a), b)
```

```
        elif isinstance(b, complex):
            return fallback_operator(complex(a), b)
        else:
            return NotImplemented
    forward.__name__ = '__' + fallback_operator.__name__ + '__'
    forward.__doc__ = monomorphic_operator.__doc__

    def reverse(b, a):
        if isinstance(a, Rational):
            # Inclui ints.
            return monomorphic_operator(a, b)
        elif isinstance(a, Real):
            return fallback_operator(float(a), float(b))
        elif isinstance(a, Complex):
            return fallback_operator(complex(a), complex(b))
        else:
            return NotImplemented
    reverse.__name__ = '__r' + fallback_operator.__name__ + '__'
    reverse.__doc__ = monomorphic_operator.__doc__

    return forward, reverse

def _add(a, b):
    """a + b"""
    return Fraction(a.numerator * b.denominator +
                    b.numerator * a.denominator,
                    a.denominator * b.denominator)

__add__, __radd__ = _operator_fallbacks(_add, operator.add)

# ...
```

`math` — Funções matemáticas

Este módulo fornece acesso à constantes e funções matemáticas comuns, incluindo aquelas definidas pelo padrão C.

Essas funções não podem ser usadas com números complexos; use as funções de mesmo nome do módulo `cmath` se você precisar de suporte para números complexos. A distinção entre funções que suportam números complexos e aquelas que não suportam é feita uma vez que a maioria dos usuários não quer aprender a matemática necessária para entender números complexos. Receber uma exceção em vez de um resultado complexo permite a detecção antecipada do número complexo inesperado usado como parâmetro, para que o programador possa determinar como e por que ele foi gerado em primeiro lugar.

As funções a seguir são fornecidas por este módulo. Exceto quando explicitamente indicado de outra forma, todos os valores de retorno são pontos flutuantes.

Arimética de ponto flutuante

`ceil(x)`	Teto de *x*, o menor número inteiro maior ou igual a *x*
`fabs(x)`	Valor absoluto de *x*
`floor(x)`	Piso de *x*, o maior inteiro menor ou igual a *x*
`fma(x, y, z)`	Operação de multiplicação e adição combinadas: `(x * y) + z`

`fmax(x, y)`	Máximo de dois valores de ponto flutuante
`fmin(x, y)`	Mínimo de dois valores de ponto flutuante
`fmod(x, y)`	Resto da divisão `x / y`
`modf(x)`	Partes fracionárias e inteiras de *x*
`remainder(x, y)`	Resto de *x* em relação a *y*
`trunc(x)`	Parte inteira de *x*

Funções de manipulação de ponto flutuante

`copysign(x, y)`	Magnitude (valor absoluto) de *x* com o sinal de *y*
`frexp(x)`	Mantissa e expoente de *x*
`isclose(a, b, rel_tol, abs_tol)`	Verifica se os valores *a* e *b* estão próximos um do outro
`isfinite(x)`	Verifica se *x* não é infinito e nem um NaN
`isnormal(x)`	Verifica se *x* é um número normal

`issubnormal(x)`	Verifica se *x* é um número subnormal
`isinf(x)`	Verifica se *x* é um infinito positivo ou negativo
`isnan(x)`	Verifica se *x* é um NaN (não um número)
`ldexp(x, i)`	`x * (2**i)`, inverso da função `frexp()`
`nextafter(x, y, steps)`	Valor de ponto flutuante de *steps* passos depois de *x* em direção a *y*
`signbit(x)`	Verifica se *x* é um número negativo
`ulp(x)`	Valor do bit menos significativo de *x*

Funções de potências, exponenciais e logarítmicas

`cbrt(x)`	Raiz cúbica de *x*
`exp(x)`	*e* elevado à potência *x*
`exp2(x)`	*2* elevado a potência de *x*
`expm1(x)`	*e* elevado a potência de *x*, menos 1
`log(x, base)`	Logaritmo de *x* para a base fornecida (*e* por padrão)

`log1p(x)`	Logaritmo natural de *1+x* (base *e*)
`log2(x)`	Logaritmo de base 2 de *x*
`log10(x)`	Logaritmo de base 10 de *x*
`pow(x, y)`	*x* elevado à potência *y*
`sqrt(x)`	Raiz quadrada de *x*

Funções de soma e produto

`dist(p, q)`	Distância euclidiana entre dois pontos *p* e *q* dada como um iterável de coordenadas
`fsum(iterable)`	Soma dos valores na entrada *iterable*
`hypot(*coordinates)`	Norma euclidiana de um iterável de coordenadas
`prod(iterable, start)`	Produto de elementos da entrada *iterable* com um valor *start*
`sumprod(p, q)`	Soma dos produtos de dois iteráveis *p* e *q*

Conversão angular

`degrees(x)`	Converte o ângulo *x* de radianos para graus
`radians(x)`	Converte o ângulo *x* de graus para radianos

Funções trigonométricas

`acos(x)`	Arco cosseno de *x*
`asin(x)`	Arco seno de *x*/
`atan(x)`	Arco tangente de *x*
`atan2(y, x)`	`atan(y / x)`
`cos(x)`	Cosseno de *x*
`sin(x)`	Seno de *x*
`tan(x)`	Tangente de *x*

Funções hiperbólicas

`acosh(x)`	Cosseno hiperbólico inverso de *x*
`asinh(x)`	Seno hiperbólico inverso de *x*

`atanh(x)`	Tangente hiperbólico inverso de *x*
`cosh(x)`	Cosseno hiperbólico de *x*
`sinh(x)`	Seno hiperbólico de *x*
`tanh(x)`	Tangente hiperbólico de *x*
Funções especiais	
`erf(x)`	Função erro em *x*
`erfc(x)`	Função erro complementar em *x*
`gamma(x)`	Função gama em *x*
`lgamma(x)`	Logaritmo natural do valor absoluto da função gama em *x*
Constantes	
`pi`	$\pi = 3.141592\ldots$
`e`	$e = 2.718281\ldots$
`tau`	$\tau = 2\pi = 6.283185\ldots$

`inf`	Infinito positivo
`nan`	“Not a number” (NaN)

Arimética de ponto flutuante

math.ceil(*x*)

Retorna o teto de *x*, o menor inteiro maior ou igual que *x*. Se *x* não é um float, delega para `x.__ceil__`, que deve retornar um valor do tipo `Integral`.

math.fabs(*x*)

Retorna o valor absoluto de *x*.

math.floor(*x*)

Retorna o chão de *x*, o maior inteiro menor ou igual a *x*. Se *x* não é um ponto flutuante, delega para `x.__floor__`, que deve retornar um valor do tipo `Integral`

math.fma(*x*, *y*, *z*)

Operação de multiplicação-adição combinada. Retorna `(x * y) + z`, calculado de forma a obter o mesmo resultado que seria obtido se o cálculo desfrutasse de precisão e limites infinitos, seguido de um único arredondamento para o formato `float`. Esta operação frequentemente oferece acurácia melhor do que a expressão direta `(x * y) + z` ofereceria.

Esta função segue a especificação da operação fusedMultiplyAdd descrita no padrão IEEE 754. O padrão deixa um caso a ser definido pela implementação, que são os resultados de `fma(0, inf, nan)` e `fma(inf, 0, nan)`. Nestes casos, `math.fma` retorna NaN, e não levanta nenhuma exceção.

Adicionado na versão 3.13.

math.fmax(*x*, *y*)

Obtém o maior de dois valores de ponto flutuante, tratando NaNs como dados ausentes.

Quando ambos os operandos forem NaNs (com sinal) ou zeros, retorna-se `nan` e `0`, respectivamente, e o sinal do resultado é definido pela implementação; isto é, não se exige que **`fmax()`** leve em consideração o sinal de tais operandos (veja o Anexo F da norma C11, §F.10.0.3 e §F.10.9.2).

Adicionado na versão 3.15.

math.fmin(*x*, *y*)

Obtém o menor de dois valores de ponto flutuante, tratando NaNs como dados ausentes.

Quando ambos os operandos forem NaNs (com sinal) ou zeros, retorna-se `nan` e `0`, respectivamente, e o sinal do resultado é definido pela implementação; isto é, não se exige que **`fmin()`** leve em consideração o sinal de tais operandos (veja o Anexo F da norma C11, §F.10.0.3 e §F.10.9.2).

Adicionado na versão 3.15.

math.fmod(*x*, *y*)

Retorna o resto ponto flutuante de `x / y`, conforme definido pela função `fmod(x, y)` da biblioteca C da plataforma. Observe que a expressão Python `x % y` pode não retornar o mesmo resultado. A intenção do padrão C é que `fmod(x, y)` seja exatamente (matematicamente; com precisão infinita) igual a `x - n*y` para algum inteiro *n* de modo que o resultado tenha o mesmo sinal que *x* e magnitude menor que `abs(y)`. O `x % y` do Python retorna um resultado com o sinal de *y*, e pode não ser exatamente computável para argumentos de ponto flutuante. Por exemplo, `fmod(-1e-100, 1e100)` é `-1e-100`, mas o resultado de `-1e-100 % 1e100` do Python é `1e100-1e-100`, que não pode ser representado exatamente como um ponto flutuante, e é arredondado para o surpreendente `1e100`. Por esta razão, a

função **fmod()** é geralmente preferida ao trabalhar com pontos flutuantes, enquanto o `x % y` do Python é preferido ao trabalhar com inteiros.

math.modf(*x*)

Retorna as partes fracionárias e inteiras de *x*. Ambos os resultados carregam o sinal de *x* e são pontos flutuantes.

Observe que **modf()** tem um padrão de chamada/retorno diferente de seu equivalente C: elas pegam um único argumento e retornam um par de valores, ao invés de retornar seu segundo valor de retorno por meio de um "parâmetro de saída" (não existe tal coisa em Python).

math.remainder(*x*, *y*)

Retorna o resto no estilo IEEE 754 de *x* em relação a *y*. Para o finito *x* e o finito diferente de zero *y*, esta é a diferença `x - n*y`, onde `n` é o número inteiro mais próximo do valor exato do quociente `x / y` . Se `x / y` está exatamente no meio do caminho entre dois inteiros consecutivos, o inteiro *par* mais próximo é usado para `n`. O resto `r = remainder(x, y)` assim sempre satisfaz `abs(r) <= 0.5 * abs(y)`.

Casos especiais seguem IEEE 754: em particular, `remainder(x, math.inf)` é *x* para qualquer *x* finito, e `remainder(x, 0)` e `remainder(math.inf, x)` levantam **ValueError** para qualquer *x* não NaN. Se o resultado da operação `remainder` for zero, esse zero terá o mesmo sinal de *x*.

Em plataformas que usam ponto flutuante binário do IEEE 754, o resultado dessa operação é sempre exatamente representável: nenhum erro de arredondamento é introduzido.

Adicionado na versão 3.7.

math.trunc(*x*)

Retorna *x* com a parte fracionária removida, deixando a parte inteira. Isso arredonda para 0: `trunc()` é equivalente a **floor()** para *x* positivos, e equivalentes a **ceil()** para *x* negativos. Se *x* não é um ponto

flutuante, então delega para `x.__trunc__()`, cujo qual deve retornar um valor do tipo `Integral`.

Para as funções `ceil()`, `floor()` e `modf()`, observe que *todos* os números de ponto flutuante de magnitude suficientemente grande são inteiros exatos. Os pontos flutuantes do Python normalmente não carregam mais do que 53 bits de precisão (o mesmo que o tipo duplo da plataforma C), caso em que qualquer ponto flutuante *x* com `abs(x) >= 2**52` necessariamente não tem bits fracionários.

Funções de manipulação de ponto flutuante

math.copysign(*x*, *y*)

Retorna um ponto flutuante com a magnitude (valor absoluto) de *x*, mas o sinal de *y*. Em plataformas que suportam zeros com sinal, `copysign(1.0, -0.0)` retorna *-1.0*.

math.frexp(*x*)

Retorna a mantissa e o expoente de *x* como o par `(m, e)`. Se *x* for um número finito diferente de zero, então *m* é um float com `0.5 <= abs(m) < 1.0` e um inteiro *e* é tal que `x == m * 2**e` exatamente. Caso contrário, retorna `(x, 0)`. Isso é usado para "desmembrar" a representação interna de um float de maneira portátil.

Observe que `frexp()` tem um padrão de chamada/retorno diferente de seu equivalente C: elas pegam um único argumento e retornam um par de valores, ao invés de retornar seu segundo valor de retorno por meio de um "parâmetro de saída" (não existe tal coisa em Python).

math.isclose(*a*, *b*, ***, *rel_tol=1e-09*, *abs_tol=0.0*)

Retorna `True` se os valores *a* e *b* estiverem próximos e `False` caso contrário.

Se dois valores são considerados próximos ou não é determinado de acordo com tolerâncias absolutas e relativas fornecidas. Se nenhum erro

ocorrer, o resultado será: `abs(a-b) <= max(rel_tol * max(abs(a), abs(b)), abs_tol)`.

rel_tol é a tolerância relativa – é a diferença máxima permitida entre *a* e *b*, em relação ao maior valor absoluto de *a* ou *b*. Por exemplo, para definir uma tolerância de 5%, passe `rel_tol=0.05`. A tolerância padrão é `1e-09`, o que garante que os dois valores sejam iguais em cerca de 9 dígitos decimais. *rel_tol* deve ser não negativo e menor que `1.0`.

abs_tol é a tolerância absoluta; o padrão é `0.0` e deve ser não negativo. Ao comparar `x` com `0.0`, `isclose(x, 0)` é computado como `abs(x) <= rel_tol * abs(x)`, que é `False` para qualquer `x` não zero e *rel_tol* menor que `1.0`. Então adicione um argumento *abs_tol* positivo apropriado à chamada.

Os valores especiais do IEEE 754 de `NaN`, `inf` e `-inf` serão tratados de acordo com as regras do IEEE. Especificamente, `NaN` não é considerado próximo a qualquer outro valor, incluindo `NaN`. `inf` e `-inf` são considerados apenas próximos a si mesmos.

Adicionado na versão 3.5.

Ver também

PEP 485 [https://peps.python.org/pep-0485/] – Uma função para testar igualdade aproximada

math.isfinite(*x*)

Retorna `True` se *x* não for um infinito nem um NaN, e `False` caso contrário. (Observe que `0.0` *é* considerado finito.)

Adicionado na versão 3.2.

math.isnormal(*x*)

Retorna `True` se *x* for um número normal, isto é, um número finito e diferente de zero que não seja subnormal (veja **`issubnormal()`**). Retorna `False` caso contrário.

Adicionado na versão 3.15.

math.issubnormal(*x*)

Retorna `True` se *x* for um número subnormal, isto é, um número finito e diferente de zero com magnitude menor que `sys.float_info.min`. Retorna `False` caso contrário.

Adicionado na versão 3.15.

math.isinf(*x*)

Retorna `True` se *x* for um infinito positivo ou negativo, e `False` caso contrário.

math.isnan(*x*)

Retorna `True` se *x* for um NaN (não um número), e `False` caso contrário.

math.ldexp(*x*, *i*)

Retorna `x * (2**i)`. Este é essencialmente o inverso da função `frexp()`.

math.nextafter(*x*, *y*, *steps=1*)

Retorna o valor de ponto flutuante com *steps* passos após *x* em direção a *y*.

Se *x* for igual a *y*, retorna *y*, a menos que *steps* seja zero.

Exemplos:

- `math.nextafter(x, math.inf)` sobe: em direção ao infinito positivo.
- `math.nextafter(x, -math.inf)` desce: em direção ao menos infinito.
- `math.nextafter(x, 0.0)` vai em direção a zero.
- `math.nextafter(x, math.copysign(math.inf, x))` se afasta do zero.

Veja também **`math.ulp()`**

Adicionado na versão 3.9.

Alterado na versão 3.12: Adicionado o argumento *steps*.

math.signbit(*x*)

Retorna `True` se o sinal de *x* for negativo e `False` caso contrário.

Isso é útil para detectar o bit de sinal de zeros, infinitos e NaNs.

Adicionado na versão 3.15.

math.ulp(*x*)

Retorna o valor do bit menos significativo do ponto flutuante *x*:

- Se *x* for um NaN (não um número), retorna *x*.
- Se *x* for negativo, retorna `ulp(-x)`.
- Se *x* for um infinito positivo, retorna *x*.
- Se *x* for igual a zero, retorna o menor valor flutuante positivo *desnormalizado* representável (menor que o ponto flutuante de valor mínimo positivo *normalizado*, `sys.float_info.min`).
- Se *x* for igual ao maior ponto flutuante positivo representável, retorna o valor do bit menos significativo de *x*, tal que o primeiro ponto flutuante menor que *x* seja `x - ulp(x)`.
- Caso contrário (*x* é um número finito positivo), retorna o valor do bit menos significativo de *x*, de modo que o primeiro ponto flutuante maior que *x* seja `x + ulp(x)`.

ULP significa “Unit in the Last Place” ou, em português, unidade na última posição.

Veja também `math.nextafter()` e `sys.float_info.epsilon`.

Adicionado na versão 3.9.

Funções de potências, exponenciais e logarítmicas

math.cbrt(*x*)

Retorna a raiz cúbica de *x*.

Adicionado na versão 3.11.

math.exp(*x*)

Retorna *e* elevado à potência *x*, onde *e* = 2.718281… é a base dos logaritmos naturais. Isso geralmente é mais preciso do que `math.e ** x` ou `pow(math.e, x)`.

math.exp2(*x*)

Retorna *2* elevado a *x*

Adicionado na versão 3.11.

math.expm1(*x*)

Retorna *e* elevado à potência *x*, menos 1. Aqui *e* é a base dos logaritmos naturais. Para pequenos pontos flutuantes *x*, a subtração em `exp(x) - 1` pode resultar em uma perda significativa de precisão [https://en.wikipedia.org/wiki/Loss_of_significance]; a função **`expm1()`** fornece uma maneira de calcular essa quantidade com precisão total:

```
>>> from math import exp, expm1
>>> exp(1e-5) - 1  # gives result accurate to 11 places
1.0000050000069649e-05
>>> expm1(1e-5)    # result accurate to full precision
1.0000050000166668e-05
```

Adicionado na versão 3.2.

math.log(*x*[, *base*])

Com um argumento, retorna o logaritmo natural de *x* (para base *e*).

Com dois argumentos, retorna o logaritmo de *x* para a *base* fornecida, calculada como `log(x)/log(base)`.

math.log1p(*x*)

Retorna o logaritmo natural de *1+x* (base *e*). O resultado é calculado de forma precisa para *x* próximo a zero.

math.log2(*x*)

Retorna o logaritmo de base 2 de *x*. Isso geralmente é mais preciso do que `log(x, 2)`.

Adicionado na versão 3.3.

Ver também

`int.bit_length()` retorna o número de bits necessários para representar um inteiro em binário, excluindo o sinal e os zeros à esquerda.

math.log10(*x*)

Retorna o logaritmo de base 10 de *x*. Isso geralmente é mais preciso do que `log(x, 10)`.

math.pow(*x*, *y*)

Retorna *x* elevado a potência de *y*. Exceções seguem o padrão IEEE 754 o máximo possível. `pow(1.0, x)` e `pow(x, 0.0)` em particular sempre retornam `1.0`, mesmo quando *x* é ZERO ou NaN. Se ambos *x* e *y* são números finitos, *x* é negativo, e *y* não é um inteiro então `pow(x, y)` é indefinido e levanta `ValueError`.

Ao contrário do operador embutido `**`, `math.pow()` converte ambos os seus argumentos para o tipo `float`. Use `**` ou a função embutida `pow()` para calcular potências inteiras exatas.

Alterado na versão 3.11: Os casos especiais `pow(0.0, -inf)` e `pow(-0.0, -inf)` foram alterados para retornar `inf` ao invés de levantar `ValueError`, para ter consistência com a IEEE 754.

math.sqrt(*x*)

Retorna a raiz quadrada de *x*.

Funções de soma e produto

math.dist(*p*, *q*)

Retorna a distância euclidiana entre dois pontos *p* e *q*, cada um dado como uma sequência (ou iterável) de coordenadas. Os dois pontos devem ter a mesma dimensão.

Aproximadamente equivalente a:

```
sqrt(sum((px - qx) ** 2.0 for px, qx in zip(p, q, strict=True
```

Adicionado na versão 3.8.

math.fsum(*iterable*)

Retorna uma soma precisa dos valores de ponto flutuante no iterável. Evita perda de precisão rastreando várias somas parciais intermediárias.

A precisão do algoritmo depende das garantias aritméticas do IEEE-754 e do caso típico em que o modo de arredondamento é meio par. Em algumas compilações que não são do Windows, a biblioteca C subjacente usa adição de precisão estendida e pode ocasionalmente arredondar uma soma intermediária fazendo com que ela introduza um erro no bit menos significativo.

Para uma discussão mais aprofundada e duas abordagens alternativas, consulte o ASPN cookbook recipes for accurate floating point summation [https://code.activestate.com/recipes/393090-binary-floating-point-summation-accurate-to-full-p/].

math.hypot(**coordinates*)

Retorna a norma euclidiana, `sqrt(sum(x**2 for x in coordinates))`. Este é o comprimento do vetor da origem até o ponto dado pelas coordenadas.

Para um ponto bidimensional `(x, y)`, isso é equivalente a calcular a hipotenusa de um triângulo retângulo usando o teorema de Pitágoras,

```
sqrt(x*x + y*y).
```

Alterado na versão 3.8: Adicionado suporte para pontos n-dimensionais. Anteriormente, apenas o caso bidimensional era suportado.

Alterado na versão 3.10: Melhorou a precisão do algoritmo para que o erro máximo seja inferior a 1 ulp (unidade no último lugar). Mais tipicamente, o resultado é quase sempre arredondado corretamente para 1/2 ulp.

math.prod(*iterable*, *, *start=1*)

Calcula o produto de todos os elementos na entrada *iterable*. O valor *start* padrão para o produto é `1`.

Quando o iterável estiver vazio, retorna o valor de start. Esta função deve ser usada especificamente com valores numéricos e pode rejeitar tipos não numéricos.

Adicionado na versão 3.8.

math.sumprod(*p*, *q*)

Retorna a soma dos produtos dos valores de dois iteráveis *p* e *q*.

Levanta `ValueError` se as entradas não tiverem o mesmo comprimento.

Aproximadamente equivalente a:

```
sum(map(operator.mul, p, q, strict=True))
```

Para entradas float e mistas int/float, os produtos intermediários e as somas são calculados com precisão estendida.

Adicionado na versão 3.12.

Conversão angular

math.degrees(*x*)

Converte o ângulo *x* de radianos para graus.

math.radians(*x*)

Converte o ângulo *x* de graus para radianos.

Funções trigonométricas

math.acos(*x*)

Retorna o arco cosseno de *x*, em radianos. O resultado está entre `0` e `pi`.

math.asin(*x*)

Retorna o arco seno de *x*, em radianos. O resultado está entre `-pi/2` e `pi/2`.

math.atan(*x*)

Retorna o arco tangente de *x*, em radianos. O resultado está entre `-pi/2` e `pi/2`.

math.atan2(*y*, *x*)

Retorna `atan(y / x)`, em radianos. O resultado está entre `-pi` e `pi`. O vetor no plano da origem ao ponto `(x, y)` faz este ângulo com o eixo X positivo. O ponto de **`atan2()`** é que os sinais de ambas as entradas são conhecidos por ele, então ele pode calcular o quadrante correto para o ângulo. Por exemplo, `atan(1)` e `atan2(1, 1)` são ambos `pi/4`, mas `atan2(-1, -1)` é `-3*pi/4`.

math.cos(*x*)

Retorna o cosseno de *x* radianos.

math.sin(*x*)

Retorna o seno de *x* radianos.

math.tan(*x*)

Retorna o tangente de *x* radianos.

Funções hiperbólicas

Funções hiperbólicas [https://en.wikipedia.org/wiki/Hyperbolic_functions] são análogas às funções trigonométricas baseadas em hipérboles em vez de círculos.

math.acosh(*x*)
: Retorna o cosseno hiperbólico inverso de *x*.

math.asinh(*x*)
: Retorna o seno hiperbólico inverso de *x*.

math.atanh(*x*)
: Retorna a tangente hiperbólica inversa de *x*.

math.cosh(*x*)
: Retorna o cosseno hiperbólico de *x*.

math.sinh(*x*)
: Retorna o seno hiperbólico de *x*.

math.tanh(*x*)
: Retorna a tangente hiperbólica de *x*.

Funções especiais

math.erf(*x*)
: Retorna a função erro [https://pt.wikipedia.org/wiki/Função_erro] em *x*.

A função `erf()` pode ser usada para calcular funções estatísticas tradicionais, como a distribuição normal padrão cumulativa [https://en.wikipedia.org/wiki/Cumulative_distribution_function]:

```
def phi(x):
    'Função de distribuição cumulativa para a distribuição no
    return (1.0 + erf(x / sqrt(2.0))) / 2.0
```

Adicionado na versão 3.2.

math.erfc(*x*)

Retorna a função erro complementar em *x*. A função erro complementar [https://en.wikipedia.org/wiki/Error_function] é definida como `1.0 - erf(x)`. É usado para grandes valores de *x* onde uma subtração de um causaria uma perda de significância [https://en.wikipedia.org/wiki/Loss_of_significance].

Adicionado na versão 3.2.

math.gamma(*x*)

Retorna a função gama [https://pt.wikipedia.org/wiki/Função_gama] em *x*.

Adicionado na versão 3.2.

math.lgamma(*x*)

Retorna o logaritmo natural do valor absoluto da função gama em *x*.

Adicionado na versão 3.2.

Funções de teoria dos números

Para retrocompatibilidade, o módulo **`math`** também fornece apelidos para as seguintes funções do módulo **`math.integer`**:

`comb(n, k)`	Número de maneiras de escolher *k* itens a partir de *n* itens sem repetição e sem ordem
`factorial(n)`	Fatorial de *n*
`gcd(*integers)`	Máximo divisor comum dos argumentos inteiros
`isqrt(n)`	Raiz quadrada inteira de um inteiro não negativo *n*

`lcm(*integers)`	Mínimo múltiplo comum dos argumentos inteiros
`perm(n, k)`	Número de maneiras de escolher *k* itens de *n* itens sem repetição e com ordem

Adicionado na versão 3.5: A função `gcd()`.

Adicionado na versão 3.8: As funções `comb()`, `perm()` e `isqrt()`.

Adicionado na versão 3.9: A função `lcm()`.

Alterado na versão 3.9: Adicionado suporte para um número arbitrário de argumentos na função `gcd()`. Anteriormente, apenas dois argumentos eram suportados.

Alterado na versão 3.10: Números de ponto flutuante com valor integral (como `5.0`) não são mais aceitos na função `factorial()`.

Suavemente descontinuado desde a versão 3.15: Usa as funções de `math.integer` ao invés desses apelidos.

Constantes

math.pi

A constante matemática π = 3.141592…, para a precisão disponível.

math.e

A constante matemática e = 2.718281…, para a precisão disponível.

math.tau

A constante matemática τ = 6.283185…, para a precisão disponível. Tau é uma constante de círculo igual a 2π, a razão entre a circunferência de um círculo e seu raio. Para saber mais sobre Tau, confira o vídeo Pi is

(still) Wrong [https://vimeo.com/147792667] de Vi Hart, e comece a comemorar o dia do Tau [https://tauday.com/] comendo duas vezes mais torta!

Adicionado na versão 3.6.

math.inf

Um infinito positivo de ponto flutuante. (Para infinito negativo, use `-math.inf`.) Equivalente à saída de `float('inf')`.

Adicionado na versão 3.5.

math.nan

Um valor de ponto flutuante "não é um número" (NaN). Equivalente à saída de `float('nan')`. Devido aos requisitos do padrão IEEE-754 [https://en.wikipedia.org/wiki/IEEE_754], `math.nan` e `float('nan')` não são considerados para ser igual a qualquer outro valor numérico, incluindo eles próprios. Para verificar se um número é NaN, use a função **`isnan()`** para testar NaNs em vez de `is` ou `==`. Exemplo:

```
>>> import math
>>> math.nan == math.nan
False
>>> float('nan') == float('nan')
False
>>> math.isnan(math.nan)
True
>>> math.isnan(float('nan'))
True
```

Adicionado na versão 3.5.

Alterado na versão 3.11: Agora está sempre disponível

O módulo **`math`** consiste principalmente em invólucros finos em torno das funções da biblioteca matemática C da plataforma. O comportamento em casos excepcionais segue o Anexo F da norma C99 quando apropriado. A implementação atual levantará **`ValueError`** para operações inválidas como `sqrt(-1.0)` ou `log(0.0)` (onde C99 Anexo F recomenda sinalizar operação inválida ou divisão por zero), e **`OverflowError`** para resultados que

estouram (por exemplo, `exp(1000.0)`). Um NaN não será retornado de nenhuma das funções acima, a menos que um ou mais dos argumentos de entrada sejam um NaN; nesse caso, a maioria das funções retornará um NaN, mas (novamente seguindo C99 Anexo F) há algumas exceções a esta regra, por exemplo, `pow(float('nan'), 0.0)` ou `hypot(float('nan'), float('inf'))`.

Observe que o Python não faz nenhum esforço para distinguir NaNs de sinalização de NaNs silenciosos, e o comportamento para NaNs de sinalização permanece não especificado. O comportamento típico é tratar todos os NaNs como se estivessem quietos.

Ver também

Módulo `cmath`
Versões de números complexos de muitas dessas funções.

Módulo `math.integer`
Funções matemáticas específicas para inteiro.

`math.integer` — integer-specific mathematics functions

Adicionado na versão 3.15.

This module provides access to the mathematical functions defined for integer arguments. These functions accept integers and objects that implement the `__index__()` method which is used to convert the object to an integer number.

The following functions are provided by this module. All return values are computed exactly and are integers.

math.integer.comb(*n*, *k*, */*)

Return the number of ways to choose *k* items from *n* items without repetition and without order.

Evaluates to `n! / (k! * (n - k)!)` when `k <= n` and evaluates to zero when `k > n`.

Also called the binomial coefficient because it is equivalent to the coefficient of k-th term in polynomial expansion of $(1 + x)^n$.

Raises `ValueError` if either of the arguments are negative.

math.integer.factorial(*n*, */*)

Return factorial of the nonnegative integer *n*.

math.integer.gcd(**integers*)

Return the greatest common divisor of the specified integer arguments. If any of the arguments is nonzero, then the returned value is the largest positive integer that is a divisor of all arguments. If all arguments are zero, then the returned value is `0`. `gcd()` without arguments returns `0`.

math.integer.isqrt(*n*, /)

Return the integer square root of the nonnegative integer *n*. This is the floor of the exact square root of *n*, or equivalently the greatest integer *a* such that $a^2 \leq n$.

For some applications, it may be more convenient to have the least integer *a* such that $n \leq a^2$, or in other words the ceiling of the exact square root of *n*. For positive *n*, this can be computed using `a = 1 + isqrt(n - 1)`.

math.integer.lcm(**integers*)

Return the least common multiple of the specified integer arguments. If all arguments are nonzero, then the returned value is the smallest positive integer that is a multiple of all arguments. If any of the arguments is zero, then the returned value is `0`. `lcm()` without arguments returns `1`.

math.integer.perm(*n*, *k=None*, /)

Return the number of ways to choose *k* items from *n* items without repetition and with order.

Evaluates to `n! / (n - k)!` when `k <= n` and evaluates to zero when `k > n`.

If *k* is not specified or is `None`, then *k* defaults to *n* and the function returns `n!`.

Raises `ValueError` if either of the arguments are negative.

`cmath` — Funções matemáticas para números complexos

Este módulo fornece acesso a funções matemáticas para números complexos. As funções neste módulo aceitam inteiros, números de ponto flutuante ou números complexos como argumentos. Eles também aceitarão qualquer objeto Python que tenha um método `__complex__()` ou `__float__()`: esses métodos são usados para converter o objeto em um número complexo ou de ponto flutuante, respectivamente, e a função é então aplicada ao resultado da conversão.

Nota

Para funções que envolvem cortes de ramificação, temos o problema de decidir como definir essas funções no próprio corte. Seguindo o artigo de Kahan intitulado “Branch cuts for complex elementary functions” (em tradução livre, “Cortes de ramificação para funções complexas elementares”), bem como o Anexo G do C99 e padrões C posteriores, usamos o sinal de zero para distinguir um lado do outro no corte de ramificação: para um corte de ramificação ao longo (de uma porção) do eixo real olhamos para o sinal da parte imaginária, enquanto para um corte de ramificação ao longo do eixo imaginário olhamos para o sinal da parte real.

Por exemplo, a função `cmath.sqrt()` tem um corte de ramificação ao longo do eixo real negativo. Um argumento de `-2-0j` é tratado como se estivesse *abaixo* do corte de ramificação, e assim dá um resultado no eixo imaginário negativo:

```
>>> cmath.sqrt(-2-0j)
-1.4142135623730951j
```

Mas um argumento de `-2+0j` é tratado como se estivesse acima do corte de ramificação:

```
>>> cmath.sqrt(-2+0j)
1.4142135623730951j
```

Conversões de e para coordenadas polares

`phase(z)`	Retorna a fase de *z*
`polar(z)`	Retorna a representação de *z* em coordenadas polares
`rect(r, phi)`	Retorna o número complexo *z* com coordenadas polares *r* e *phi*

Funções de potência e logarítmicas

`exp(z)`	Retorna *e* elevado à potência *z*
`log(z[, base])`	Retorna o logaritmo de *z* para a *base* fornecida (*e* por padrão)
`log10(z)`	Retorna o logaritmo de base 10 de *z*
`sqrt(z)`	Retorna a raiz quadrada de *z*

Funções trigonométricas

`acos(z)`	Retorna o arco cosseno de *z*
`asin(z)`	Retorna o arco seno de *z*
`atan(z)`	Retorna o arco tangente de *z*
`cos(z)`	Retorna o cosseno de *z*
`sin(z)`	Retorna o seno de *z*
`tan(z)`	Retorna a tangente de *z*

Funções hiperbólicas

`acosh(z)`	Retorna o cosseno hiperbólico inverso de *z*
`asinh(z)`	Retorna o seno hiperbólico inverso de *z*
`atanh(z)`	Retorna a tangente hiperbólica inversa de *z*
`cosh(z)`	Retorna o cosseno hiperbólico de *z*
`sinh(z)`	Retorna o seno hiperbólico de *z*
`tanh(z)`	Retorna a tangente hiperbólica de *z*

Funções de classificação	
`isfinite(z)`	Verifica se todos os componentes de *z* são finitos
`isinf(z)`	Verifica se algum componente de *z* é infinito
`isnan(z)`	Verifica se algum componente de *z* é NaN
`isclose(a, b, *, rel_tol, abs_tol)`	Verifica se os valores *a* e *b* estão próximos um do outro
Constantes	
`pi`	$\pi = 3.141592\ldots$
`e`	$e = 2.718281\ldots$
`tau`	$\tau = 2\pi = 6.283185\ldots$
`inf`	Infinito positivo
`infj`	Infinito imaginário puro
`nan`	“Not a number” (NaN)

Conversões de e para coordenadas polares

Um número complexo Python `z` é armazenado internamente usando coordenadas *retangulares* ou *cartesianas*. É completamente determinado por sua *parte real* `z.real` e sua *parte imaginária* `z.imag`.

Coordenadas polares fornecem uma forma alternativa de representar um número complexo. Em coordenadas polares, um número complexo *z* é definido pelo módulo *r* e pelo ângulo de fase *phi*. O módulo *r* é a distância de *z* à origem, enquanto a fase *phi* é o ângulo anti-horário, medido em radianos, do eixo x positivo ao segmento de reta que une a origem a *z*.

As funções a seguir podem ser usadas para converter coordenadas retangulares nativas em coordenadas polares e vice-versa.

cmath.phase(*z*)

Retorna a fase de *z* (também conhecido como *argumento* de *z*), como um ponto flutuante. `phase(z)` equivale a `math.atan2(z.imag, z.real)`. O resultado está no intervalo [-*π*, *π*], e o corte de ramificação para esta operação está ao longo do eixo real negativo. O sinal do resultado é igual ao sinal de `z.imag`, mesmo quando `z.imag` é zero:

```
>>> phase(-1+0j)
3.141592653589793
>>> phase(-1-0j)
-3.141592653589793
```

Nota

O módulo (valor absoluto) de um número complexo *z* pode ser calculado usando a função embutida **`abs()`**. Não há função do módulo **`cmath`** separada para esta operação.

cmath.polar(*z*)

Retorna a representação de *z* em coordenadas polares. Retorna um par `(r, phi)` onde *r* é o módulo de *z* e phi é a fase de *z*. `polar(z)` equivale a `(abs(z), phase(z))`.

cmath.rect(*r*, *phi*)

Retorna o número complexo *z* com coordenadas polares *r* e *phi*. Equivale a `complex(r * math.cos(phi), r * math.sin(phi))`.

Funções de potência e logarítmicas

cmath.exp(*z*)

Retorna *e* elevado à potência *z*, onde *e* é a base de logaritmos naturais.

cmath.log(*z*[, *base*])

Retorna o logaritmo de *z* para a *base* fornecida. Se a *base* não for especificada, retorna o logaritmo natural de *z*. Há um corte de ramificação, de 0 ao longo do eixo real negativo até -∞.

cmath.log10(*z*)

Retorna o logaritmo de *z* na base 10. Este tem o mesmo corte de ramificação que **`log()`**.

cmath.sqrt(*z*)

Retorna a raiz quadrada de *z*. Este tem o mesmo corte de ramificação que **`log()`**.

Funções trigonométricas

cmath.acos(*z*)

Retorna o arco cosseno de *z*. Existem dois cortes de ramificação: um se estende desde 1 ao longo do eixo real até ∞. O outro se estende para a esquerda de -1 ao longo do eixo real até -∞.

cmath.asin(*z*)

Retorna o arco seno de *z*. Tem os mesmos cortes de ramificação que `acos()`.

cmath.atan(*z*)

Retorna o arco tangente de *z*. Existem dois cortes de ramificação: Um se estende de `1j` ao longo do eixo imaginário até `∞j`. O outro se estende de `-1j` ao longo do eixo imaginário até `-∞j`.

cmath.cos(*z*)

Retorna o cosseno de *z*.

cmath.sin(*z*)

Retorna o seno de *z*.

cmath.tan(*z*)

Retorna a tangente de *z*.

Funções hiperbólicas

cmath.acosh(*z*)

Retorna o cosseno hiperbólico inverso de *z*. Há um corte de ramificação, estendendo-se para a esquerda de 1 ao longo do eixo real até -∞.

cmath.asinh(*z*)

Retorna o seno hiperbólico inverso de *z*. Existem dois cortes de ramificação: Um se estende de `1j` ao longo do eixo imaginário até `∞j`. O outro se estende de `-1j` ao longo do eixo imaginário até `-∞j`.

cmath.atanh(*z*)

Retorna a tangente hiperbólica inversa de *z*. Existem dois cortes de ramificação: Um se estende de `1` ao longo do eixo real até `∞`. O outro se estende de `-1` ao longo do eixo real até `-∞`.

cmath.cosh(*z*)

Retorna o cosseno hiperbólico de *z*.

cmath.sinh(*z*)

Retorna o seno hiperbólico de *z*.

cmath.tanh(*z*)

Retorna a tangente hiperbólica de *z*.

Funções de classificação

cmath.isfinite(*z*)

Retorna `True` se ambas as partes real e imaginária de *z* forem finitas, e `False` caso contrário.

Adicionado na versão 3.2.

cmath.isinf(*z*)

Retorna `True` se ou a parte real ou a imaginária de *z* for infinita, e `False` caso contrário.

cmath.isnan(*z*)

Retorna `True` se ou a parte real ou a imaginária de *z* for NaN, e `False` caso contrário.

cmath.isclose(*a*, *b*, ***, *rel_tol=1e-09*, *abs_tol=0.0*)

Retorna `True` se os valores *a* e *b* estiverem próximos e `False` caso contrário.

Se dois valores são considerados próximos ou não é determinado de acordo com tolerâncias absolutas e relativas fornecidas. Se nenhum erro ocorrer, o resultado será: `abs(a-b) <= max(rel_tol * max(abs(a), abs(b)), abs_tol)`.

rel_tol é a tolerância relativa – é a diferença máxima permitida entre *a* e *b*, em relação ao maior valor absoluto de *a* ou *b*. Por exemplo, para definir uma tolerância de 5%, passe `rel_tol=0.05`. A tolerância padrão

é `1e-09`, o que garante que os dois valores sejam iguais em cerca de 9 dígitos decimais. *rel_tol* deve ser não negativo e menor que `1.0`.

abs_tol é a tolerância absoluta; o padrão é `0.0` e deve ser não negativo. Ao comparar `x` com `0.0`, `isclose(x, 0)` é computado como `abs(x) <= rel_tol * abs(x)`, que é `False` para qualquer `x` e rel_tol menor que `1.0`. Então adicione um argumento abs_tol positivo apropriado à chamada.

Os valores especiais do IEEE 754 de `NaN`, `inf` e `-inf` serão tratados de acordo com as regras do IEEE. Especificamente, `NaN` não é considerado próximo a qualquer outro valor, incluindo `NaN`. `inf` e `-inf` são considerados apenas próximos a si mesmos.

Adicionado na versão 3.5.

Ver também

PEP 485 [https://peps.python.org/pep-0485/] – Uma função para testar igualdade aproximada

Constantes

cmath.pi

A constante matemática *π*, como um ponto flutuante.

cmath.e

A constante matemática *e*, como um ponto flutuante.

cmath.tau

A constante matemática *τ*, como um ponto flutuante.

Adicionado na versão 3.6.

cmath.inf

Infinito positivo de ponto flutuante. Equivale a `float('inf')`.

Adicionado na versão 3.6.

cmath.infj

Número complexo com parte real zero e parte imaginária infinita positiva. Equivale a `complex(0.0, float('inf'))`.

Adicionado na versão 3.6.

cmath.nan

Um valor de ponto flutuante “não um número” (NaN). Equivale a `float('nan')`. Veja também **`math.nan`**.

Adicionado na versão 3.6.

cmath.nanj

Número complexo com parte real zero e parte imaginária NaN. Equivale a `complex(0.0, float('nan'))`.

Adicionado na versão 3.6.

Observe que a seleção de funções é semelhante, mas não idêntica, àquela no módulo **`math`**. A razão para ter dois módulos é que alguns usuários não estão interessados em números complexos e talvez nem saibam o que são. Eles preferem que `math.sqrt(-1)` gere uma exceção do que retorne um número complexo. Observe também que as funções definidas em **`cmath`** sempre retornam um número complexo, mesmo que a resposta possa ser expressa como um número real (nesse caso o número complexo tem uma parte imaginária de zero).

Uma nota sobre cortes de ramificação: são curvas ao longo das quais a função dada não é contínua. Eles são um recurso necessário de muitas funções complexas. Presume-se que se você precisar calcular com funções complexas, você entenderá sobre cortes de ramificação. Consulte quase qualquer livro (não muito elementar) sobre variáveis complexas para obter esclarecimento. Para informações sobre a escolha adequada dos cortes de ramificação para fins numéricos, uma boa referência deve ser a seguinte:

Ver também

Kahan, W: Branch cuts for complex elementary functions; or, Much ado about nothing’s sign bit. Em Iserles, A. e Powell, M. (eds.), The state of the art in numerical analysis. Clarendon Press (1987) pp165–211.

`decimal` — Aritmética de ponto fixo decimal e ponto flutuante

Código-fonte: Lib/decimal.py [https://github.com/python/cpython/tree/3.15/Lib/decimal.py]

O módulo **`decimal`** fornece suporte à aritmética rápida de ponto flutuante decimal, com arredondamento correto. Oferece várias vantagens sobre o tipo de dados **`float`**:

- Decimal "é baseado em um modelo de ponto flutuante [https://speleotrove.com/decimal/damodel.html#refnumber] que foi projetado com as pessoas em mente e necessariamente tem um princípio orientador primordial – os computadores devem fornecer uma aritmética que funcione da mesma maneira que a aritmética que as pessoas aprendem na escola". – Trecho da especificação aritmética decimal.

- Os números decimais podem ser representados exatamente. Por outro lado, números como `1.1` e `2.2` não têm representações exatas em ponto flutuante binário. Os usuários finais normalmente não esperam que `1.1 + 2.2` sejam exibidos como `3.3000000000000003`, como ocorre com o ponto flutuante binário.

- A exatidão transita para a aritmética. No ponto flutuante decimal, `0.1 + 0.1 + 0.1 - 0.3` é exatamente igual a zero. No ponto flutuante binário, o resultado é `5.5511151231257827e-017`. Embora próximas de zero, as diferenças impedem o teste de igualdade confiável, e podem se acumular. Por esse motivo, o decimal é preferido em aplicações de contabilidade que envolvem invariáveis estritos de igualdade.

- O módulo decimal incorpora uma noção de casas significativas para que `1.30 + 1.20` seja `2.50`. O zero à direita é mantido para indicar significância. Esta é a apresentação habitual para aplicações monetárias. Para multiplicação, a abordagem "livro escolar" usa todas as figuras nos multiplicandos. Por exemplo, `1.3 * 1.2` é igual a `1.56` enquanto `1.30 * 1.20` é igual a `1.5600`.

- Diferentemente do ponto flutuante binário baseado em hardware, o módulo decimal possui uma precisão alterável pelo usuário (padrão de 28 casas), que pode ser tão grande quanto necessário para um determinado problema:

```
>>> from decimal import *
>>> getcontext().prec = 6
>>> Decimal(1) / Decimal(7)
Decimal('0.142857')
>>> getcontext().prec = 28
>>> Decimal(1) / Decimal(7)
Decimal('0.1428571428571428571428571429')
```

- O ponto flutuante binário e decimal é implementado em termos de padrões publicados. Enquanto o tipo ponto flutuante embutido expõe apenas uma parte modesta de seus recursos, o módulo decimal expõe todas as partes necessárias do padrão. Quando necessário, o programador tem controle total sobre o arredondamento e o manuseio do sinal. Isso inclui uma opção para impor aritmética exata usando exceções para bloquear quaisquer operações inexatas.

- O módulo decimal foi projetado para dar suporte, “sem prejuízo, à aritmética decimal não arredondada exata (às vezes chamada de aritmética de ponto fixo) e à aritmética arredondada de ponto flutuante”. – Trecho da especificação aritmética decimal.

O design do módulo é centrado em torno de três conceitos: o número decimal, o contexto da aritmética e os sinais.

Um número decimal é imutável. Possui um sinal, dígitos de coeficiente e um expoente. Para preservar a significância, os dígitos do coeficiente não truncam zeros à direita. Os decimais também incluem valores especiais, tais como `Infinity`, `-Infinity` e `NaN`. O padrão também diferencia `-0` de `+0`.

O contexto da aritmética é um ambiente que especifica precisão, regras de arredondamento, limites de expoentes, sinalizadores indicando os resultados das operações e ativadores de interceptação que determinam se os sinais são tratados como exceções. As opções de arredondamento incluem **`ROUND_CEILING`**, **`ROUND_DOWN`**, **`ROUND_FLOOR`**, **`ROUND_HALF_DOWN`**, **`ROUND_HALF_EVEN`**, **`ROUND_HALF_UP`**, **`ROUND_UP`** e **`ROUND_05UP`**.

Sinais são grupos de condições excepcionais que surgem durante o curso da computação. Dependendo das necessidades da aplicação, os sinais podem ser ignorados, considerados informativos ou tratados como exceções. Os sinais no módulo decimal são: `Clamped`, `InvalidOperation`, `DivisionByZero`, `Inexact`, `Rounded`, `Subnormal`, `Overflow`, `Underflow` e `FloatOperation`.

Para cada sinal, há um sinalizador e um ativador de interceptação. Quando um sinal é encontrado, seu sinalizador é definido como um e, se o ativador de interceptação estiver definido como um, uma exceção será gerada. Os sinalizadores são fixos; portanto, o usuário precisa redefini-los antes de monitorar um cálculo.

Ver também

- A especificação geral aritmética decimal da IBM, The General Decimal Arithmetic Specification [https://speleotrove.com/decimal/decarith.html].

Tutorial de início rápido

O início usual do uso de decimais é importar o módulo, exibir o contexto atual com `getcontext()` e, se necessário, definir novos valores para precisão, arredondamento ou armadilhas ativadas:

```
>>> from decimal import *
>>> getcontext()
Context(prec=28, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=99999
        capitals=1, clamp=0, flags=[], traps=[Overflow, DivisionByZe
        InvalidOperation])

>>> getcontext().prec = 7       # Define uma nova precisão
```

Instâncias decimais podem ser construídas a partir de números inteiros, strings, valores ponto flutuante ou tuplas. A construção a partir de um número inteiro ou de um ponto flutuante realiza uma conversão exata do valor desse número inteiro ou ponto flutuante. Os números decimais incluem valores especiais como `NaN`, que significa “Não é um número”, `Infinity` positivo e negativo e `-0`:

```
>>> getcontext().prec = 28
>>> Decimal(10)
```

```
Decimal('10')
>>> Decimal('3.14')
Decimal('3.14')
>>> Decimal(3.14)
Decimal('3.14000000000000012434497875801753252744674682617187 5')
>>> Decimal((0, (3, 1, 4), -2))
Decimal('3.14')
>>> Decimal(str(2.0 ** 0.5))
Decimal('1.4142135623730951')
>>> Decimal(2) ** Decimal('0.5')
Decimal('1.4142135623730950488016887242')
>>> Decimal('NaN')
Decimal('NaN')
>>> Decimal('-Infinity')
Decimal('-Infinity')
```

Se o sinal **FloatOperation** está ativado, a mistura acidental de decimais e pontos flutuantes em construtores ou comparações de ordenação levanta uma exceção:

```
>>> c = getcontext()
>>> c.traps[FloatOperation] = True
>>> Decimal(3.14)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.FloatOperation: [<class 'decimal.FloatOperation'>]
>>> Decimal('3.5') < 3.7
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.FloatOperation: [<class 'decimal.FloatOperation'>]
>>> Decimal('3.5') == 3.5
True
```

Adicionado na versão 3.3.

A significância de um novo Decimal é determinada unicamente pelo número de dígitos introduzidos. A precisão e o arredondamento do contexto só entram em jogo nas operações aritméticas.

```
>>> getcontext().prec = 6
>>> Decimal('3.0')
Decimal('3.0')
>>> Decimal('3.1415926535')
Decimal('3.1415926535')
```

```
>>> Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85987')
>>> getcontext().rounding = ROUND_UP
>>> Decimal('3.1415926535') + Decimal('2.7182818285')
Decimal('5.85988')
```

Se os limites internos da versão C forem excedidos, a construção de um decimal levanta **`InvalidOperation`**:

```
>>> Decimal("1e99999999999999999999")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
decimal.InvalidOperation: [<class 'decimal.InvalidOperation'>]
```

Alterado na versão 3.3.

Os decimais interagem bem com grande parte do resto do Python. Aqui está um pequeno circo voador de ponto flutuante decimal:

```
>>> data = list(map(Decimal, '1.34 1.87 3.45 2.35 1.00 0.03 9.25'.sp
>>> max(data)
Decimal('9.25')
>>> min(data)
Decimal('0.03')
>>> sorted(data)
[Decimal('0.03'), Decimal('1.00'), Decimal('1.34'), Decimal('1.87'),
 Decimal('2.35'), Decimal('3.45'), Decimal('9.25')]
>>> sum(data)
Decimal('19.29')
>>> a,b,c = data[:3]
>>> str(a)
'1.34'
>>> float(a)
1.34
>>> round(a, 1)
Decimal('1.3')
>>> int(a)
1
>>> a * 5
Decimal('6.70')
>>> a * b
Decimal('2.5058')
>>> c % a
Decimal('0.77')
```

Decimais podem ser formatados (com **`format()`** ou Literais de strings formatadas) em notação de ponto fixo ou científica, usando a mesma sintaxe para formatação (consulte Minilinguagem de especificação de formato) do tipo embutido **`float`**:

```
>>> format(Decimal('2.675'), "f")
'2.675'
>>> format(Decimal('2.675'), ".2f")
'2.68'
>>> f"{Decimal('2.675'):.2f}"
'2.68'
>>> format(Decimal('2.675'), ".2e")
'2.68e+0'
>>> with localcontext() as ctx:
...     ctx.rounding = ROUND_DOWN
...     print(format(Decimal('2.675'), ".2f"))
...
2.67
```

E algumas funções matemáticas também estão disponíveis para Decimal:

```
>>> getcontext().prec = 28
>>> Decimal(2).sqrt()
Decimal('1.4142135623730950488016887242')
>>> Decimal(1).exp()
Decimal('2.718281828459045235360287471')
>>> Decimal('10').ln()
Decimal('2.302585092994045684017991455')
>>> Decimal('10').log10()
Decimal('1')
```

O método **`quantize()`** arredonda um número para um expoente fixo. Esse método é útil para aplicações monetárias que geralmente arredondam os resultados para um número fixo de casas:

```
>>> Decimal('7.325').quantize(Decimal('.01'), rounding=ROUND_DOWN)
Decimal('7.32')
>>> Decimal('7.325').quantize(Decimal('1.'), rounding=ROUND_UP)
Decimal('8')
```

Como mostrado acima, a função **`getcontext()`** acessa o contexto atual e permite alterar as configurações. Essa abordagem atende às necessidades da maioria das aplicações.

Para trabalhos mais avançados, pode ser útil criar contextos alternativos usando o construtor `Context()`. Para ativar uma alternativa, use a função `setcontext()`.

De acordo com o padrão, o módulo **`decimal`** fornece dois contextos padrão prontos para uso, `BasicContext` e `ExtendedContext`. O primeiro é especialmente útil para depuração porque muitas das armadilhas estão ativadas:

```
>>> myothercontext = Context(prec=60, rounding=ROUND_HALF_DOWN)
>>> setcontext(myothercontext)
>>> Decimal(1) / Decimal(7)
Decimal('0.142857142857142857142857142857142857142857142857142857142

>>> ExtendedContext
Context(prec=9, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
        capitals=1, clamp=0, flags=[], traps=[])
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(7)
Decimal('0.142857143')
>>> Decimal(42) / Decimal(0)
Decimal('Infinity')

>>> setcontext(BasicContext)
>>> Decimal(42) / Decimal(0)
Traceback (most recent call last):
  File "<pyshell#143>", line 1, in -toplevel-
    Decimal(42) / Decimal(0)
DivisionByZero: x / 0
```

Os contextos também contêm sinalizadores para monitorar condições excepcionais identificadas durante os cálculos. Os sinalizadores permanecem definidos até que sejam explicitamente limpos, portanto, é melhor limpar os sinalizadores antes de cada conjunto de cálculos monitorados usando o método `clear_flags()`.

```
>>> setcontext(ExtendedContext)
>>> getcontext().clear_flags()
>>> Decimal(355) / Decimal(113)
Decimal('3.14159292')
>>> getcontext()
Context(prec=9, rounding=ROUND_HALF_EVEN, Emin=-999999, Emax=999999,
        capitals=1, clamp=0, flags=[Inexact, Rounded], traps=[])
```

A entrada *flags* indica que a aproximação racional de pi foi arredondada (os dígitos além da precisão do contexto foram descartados) e que o resultado é inexato (alguns desses dígitos descartados eram diferentes de zero).

Armadilhas individuais são definidas usando o dicionário no atributo **traps** de um contexto:

```
>>> setcontext(ExtendedContext)
>>> Decimal(1) / Decimal(0)
Decimal('Infinity')
>>> getcontext().traps[DivisionByZero] = 1
>>> Decimal(1) / Decimal(0)
Traceback (most recent call last):
  File "<pyshell#112>", line 1, in -toplevel-
    Decimal(1) / Decimal(0)
DivisionByZero: x / 0
```

A maioria dos programas ajusta o contexto atual apenas uma vez, no início. E, em muitas aplicações, os dados são convertidos para **Decimal** em uma única conversão dentro de um loop. Com o conjunto de contextos e decimais criados, a maior parte do programa manipula os dados de maneira diferente da que se faz com outros tipos numéricos do Python.

Objetos de Decimal

class decimal.Decimal(*value='0', context=None*)

Constrói um novo objeto de **Decimal** a partir de *value*.

value pode ser um inteiro, uma string, uma tupla, um **float** ou outro objeto **Decimal**. Se nenhum *value* for fornecido, retornará `Decimal('0')`. Se *value* for uma string, ele deverá estar em conformidade com a sintaxe da string numérica decimal após caracteres de espaço em branco à esquerda e à direita, bem como sublinhados serem removidos:

```
sign           ::=  '+' | '-'
digit          ::=  '0' | '1' | '2' | '3' | '4' | '5' | '6' | '7
indicator      ::=  'e' | 'E'
digits         ::=  digit [digit]...
decimal-part   ::=  digits '.' [digits] | ['.'] digits
exponent-part  ::=  indicator [sign] digits
infinity       ::=  'Infinity' | 'Inf'
```

```
nan             ::=  'NaN' [digits] | 'sNaN' [digits]
numeric-value   ::=  decimal-part [exponent-part] | infinity
numeric-string ::=  [sign] numeric-value | [sign] nan
```

Outros dígitos decimais Unicode também são permitidos onde `digit` aparece acima. Isso inclui dígitos decimais de vários outros alfabetos (por exemplo, dígitos em árabes-índicos e devanágaris), além dos dígitos de largura total `'\uff10'` a `'\uff19'`. A capitalização não é significativa, então, por exemplo, `inf`, `Inf`, `INFINITY` e `iNfINity` são todas grafias aceitáveis para infinito positivo.

Se *value* for uma **tupla**, ela deverá ter três componentes: um sinal (`0` para positivo ou `1` para negativo), uma **tupla** de dígitos e um expoente inteiro. Por exemplo, `Decimal((0, (1, 4, 1, 4), -3))` retorna `Decimal('1.414')`.

Se *value* é um **float**, o valor do ponto flutuante binário é convertido sem perdas para o seu equivalente decimal exato. Essa conversão geralmente requer 53 ou mais dígitos de precisão. Por exemplo, `Decimal(float('1.1'))` converte para `Decimal('1.100000000000000088817841970012523233890533447265625')`.

A precisão *context* não afeta o número de dígitos armazenados. Isso é determinado exclusivamente pelo número de dígitos de *value*. Por exemplo, `Decimal('3.00000')` registra todos os cinco zeros, mesmo que a precisão do contexto seja de apenas três.

O objetivo do argumento *context* é determinar o que fazer se *value* for uma string malformada. Se o contexto capturar **InvalidOperation**, uma exceção será levantada; caso contrário, o construtor retornará um novo decimal com o valor `NaN`.

Uma vez construídos, objetos **Decimal** são imutáveis.

Alterado na versão 3.2: O argumento para o construtor agora pode ser uma instância de **float**.

Alterado na versão 3.3: Os argumentos de **float** levantam uma exceção se a armadilha **FloatOperation** estiver definida. Por padrão, a armadilha está

desativada.

Alterado na versão 3.6: Sublinhados são permitidos para agrupamento, como nos literais de ponto flutuante e inteiros no código.

Objetos decimais de ponto flutuante compartilham muitas propriedades com outros tipos numéricos embutidos, como **`float`** e **`int`**. Todas as operações matemáticas usuais e métodos especiais se aplicam. Da mesma forma, objetos decimais podem ser copiados, separados, impressos, usados como chaves de dicionário, usados como elementos de conjunto, comparados, classificados e convertidos para outro tipo (como **`float`** ou **`int`**).

Existem algumas pequenas diferenças entre a aritmética de objetos decimais e a de números inteiros e flutuantes. Quando o operador de resto `%` é aplicado a objetos decimais, o sinal do resultado é o sinal do *dividendo* em vez do sinal do divisor:

```
>>> (-7) % 4
1
>>> Decimal(-7) % Decimal(4)
Decimal('-3')
```

O operador de divisão inteira `//` se comporta de maneira análoga, retornando a parte inteira do quociente verdadeiro (truncando em direção a zero) em vez de seu resto, de modo a preservar a identidade usual `x == (x // y) * y + x % y`:

```
>>> -7 // 4
-2
>>> Decimal(-7) // Decimal(4)
Decimal('-1')
```

Os operadores `%` e `//` implementam as operações de `remainder` e `divide-integer` (respectivamente), conforme descrito na especificação.

Objetos decimais geralmente não podem ser combinados com pontos flutuantes ou instâncias de **`fractions.Fraction`** em operações aritméticas: uma tentativa de adicionar um **`Decimal`** a um **`float`**, por exemplo, vai levantar um **`TypeError`**. No entanto, é possível usar os operadores de comparação do Python para comparar uma instância de **`Decimal`** `x` com outro número `y`. Isso evita resultados confusos ao comparar números de tipos diferentes.

Alterado na versão 3.2: As comparações de tipos mistos entre instâncias de **`Decimal`** e outros tipos numéricos agora estão totalmente implementadas.

Além das propriedades numéricas padrões, os objetos de ponto flutuante decimal também possuem vários métodos especializados:

adjusted()

Retorna o expoente ajustado depois de deslocar os dígitos mais à direita do coeficiente até restar apenas o dígito principal: `Decimal('321e+5').adjusted()` retorna sete. Usado para determinar a posição do dígito mais significativo em relação ao ponto decimal.

as_integer_ratio()

Retorna um par `(n, d)` de números inteiros que representam a instância dada **`Decimal`** como uma fração, nos termos mais baixos e com um denominador positivo:

```
>>> Decimal('-3.14').as_integer_ratio()
(-157, 50)
```

A conversão é exata. Levanta OverflowError em infinitos e ValueError em NaNs.

Adicionado na versão 3.6.

as_tuple()

Retorna uma representação de tupla nomeada do número: `DecimalTuple(sign, digits, exponent)`.

canonical()

Retorna a codificação canônica do argumento. Atualmente, a codificação de uma instância de **`Decimal`** é sempre canônica, portanto, esta operação retorna seu argumento inalterado.

compare(*other*, *context=None*)

Compara os valores de duas instâncias decimais. **`compare()`** retorna uma instância decimal, e se qualquer operando for um NaN, o resultado será um NaN:

```
a or b is a NaN  ==> Decimal('NaN')
a < b            ==> Decimal('-1')
a == b           ==> Decimal('0')
a > b            ==> Decimal('1')
```

compare_signal(*other*, *context=None*)

Esta operação é idêntica ao método **compare()**, exceto que todos os NaNs sinalizam. Ou seja, se nenhum operando for um NaN sinalizador, qualquer operando NaN silencioso será tratado como se fosse um NaN sinalizador.

compare_total(*other*, *context=None*)

Compara dois operandos usando sua representação abstrata em vez de seu valor numérico. Semelhante ao método **compare()**, mas o resultado fornece uma ordem total nas instâncias de **Decimal**. Duas instâncias de **Decimal** com o mesmo valor numérico, mas diferentes representações, se comparam desiguais nesta ordem:

```
>>> Decimal('12.0').compare_total(Decimal('12'))
Decimal('-1')
```

Os NaNs silenciosos e sinalizadores também estão incluídos no pedido total. O resultado dessa função é `Decimal('0')` se os dois operandos tiverem a mesma representação, `Decimal('-1')` se o primeiro operando for menor na ordem total que o segundo e `Decimal('1')` se o primeiro operando for maior na ordem total que o segundo operando. Veja a especificação para detalhes da ordem total.

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

compare_total_mag(*other*, *context=None*)

Compara dois operandos usando sua representação abstrata em vez de seu valor, como em **compare_total()**, mas ignorando o sinal de cada operando. `x.compare_total_mag(y)` é equivalente a `x.copy_abs().compare_total(y.copy_abs())`.

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

conjugate()

Apenas retorna a si próprio, sendo esse método apenas para atender à Especificação de Decimal.

copy_abs()

Retorna o valor absoluto do argumento. Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado.

copy_negate()

Retorna a negação do argumento. Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado.

copy_sign(*other*, *context=None*)

Retorna uma cópia do primeiro operando com o sinal definido para ser o mesmo que o sinal do segundo operando. Por exemplo:

```
>>> Decimal('2.3').copy_sign(Decimal('-1.5'))
Decimal('-2.3')
```

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

exp(*context=None*)

Retorna o valor da função exponencial (natural) `e**x` no número especificado. O resultado é arredondado corretamente usando o modo de arredondamento **ROUND_HALF_EVEN**.

```
>>> Decimal(1).exp()
Decimal('2.718281828459045235360287471')
>>> Decimal(321).exp()
Decimal('2.561702493119680037517373933E+139')
```

classmethod from_float(*f*, *./*)

Construtor alternativo que aceita apenas instâncias de **`float`** ou **`int`**.

Observe que `Decimal.from_float(0.1)` não é o mesmo que `Decimal('0.1')`. Como 0.1 não é exatamente representável no ponto flutuante binário, o valor é armazenado como o valor representável mais próximo que é `0x1.999999999999ap-4`.; Esse valor equivalente em decimal é `0.1000000000000000055511151231257827021181583404541015625`.

Nota

A partir do Python 3.2 em diante, uma instância de **`Decimal`** também pode ser construída diretamente a partir de um **`float`**.

```
>>> Decimal.from_float(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015
>>> Decimal.from_float(float('nan'))
Decimal('NaN')
>>> Decimal.from_float(float('inf'))
Decimal('Infinity')
>>> Decimal.from_float(float('-inf'))
Decimal('-Infinity')
```

Adicionado na versão 3.1.

classmethod from_number(*number*, *./*)

Construtor alternativo que aceita apenas instâncias de **`float`**, **`int`** ou **`Decimal`**, mas não de strings ou tuplas.

```
>>> Decimal.from_number(314)
Decimal('314')
>>> Decimal.from_number(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015
>>> Decimal.from_number(Decimal('3.14'))
Decimal('3.14')
```

Adicionado na versão 3.14.

fma(*other*, *third*, *context=None*)

Multiplicação e adição fundidas. Retorna self*other+third sem arredondamento do produto intermediário self*other.

```
>>> Decimal(2).fma(3, 5)
Decimal('11')
```

is_canonical()

Retorna **True** se o argumento for canônico e **False** caso contrário. Atualmente, uma instância de **Decimal** é sempre canônica, portanto, esta operação sempre retorna **True**.

is_finite()

Retorna **True** se o argumento for um número finito e **False** se o argumento for um infinito ou um NaN.

is_infinite()

Retorna **True** se o argumento for infinito positivo ou negativo e **False** caso contrário.

is_nan()

Retorna **True** se o argumento for NaN (silencioso ou sinalizado) e **False** caso contrário.

is_normal(*context=None*)

Retorna **True** se o argumento for um número finito *normal*. Retorna **False** se o argumento for zero, subnormal, infinito ou NaN.

is_qnan()

Retorna **True** se o argumento for um NaN silencioso e **False** caso contrário.

is_signed()

Retorna **True** se o argumento tiver um sinal negativo e **False** caso contrário. Observe que zeros e NaNs podem carregar sinais.

is_snan()

Retorna **True** se o argumento for um sinal NaN e **False** caso contrário.

is_subnormal(*context=None*)

Retorna **`True`** se o argumento for subnormal e **`False`** caso contrário.

is_zero()

Retorna **`True`** se o argumento for um zero (positivo ou negativo) e **`False`** caso contrário.

ln(*context=None*)

Retorna o logaritmo (base e) natural do operando. O resultado é arredondado corretamente usando o modo de arredondamento **`ROUND_HALF_EVEN`**.

log10(*context=None*)

Retorna o logaritmo da base dez do operando. O resultado é arredondado corretamente usando o modo de arredondamento **`ROUND_HALF_EVEN`**.

logb(*context=None*)

Para um número diferente de zero, retorna o expoente ajustado de seu operando como uma instância de **`Decimal`**. Se o operando é zero, `Decimal('-Infinity')` é retornado e o sinalizador **`DivisionByZero`** é levantado. Se o operando for um infinito, `Decimal('Infinity')` será retornado.

logical_and(*other*, *context=None*)

`logical_and()` é uma operação lógica que leva dois *operandos lógicos* (consulte Operandos lógicos). O resultado é o `and` dígito a dígito dos dois operandos.

logical_invert(*context=None*)

`logical_invert()` é uma operação lógica. O resultado é a inversão dígito a dígito do operando.

logical_or(*other*, *context=None*)

`logical_or()` é uma operação lógica que leva dois *operandos lógicos* (consulte Operandos lógicos). O resultado é o `or` dígito a dígito dos dois operandos.

logical_xor(*other*, *context=None*)

`logical xor()` é uma operação lógica que leva dois *operandos lógicos* (consulte Operandos lógicos). O resultado é o “ou exclusivo” dígito a dígito ou dos dois operandos.

max(*other*, *context=None*)

Como `max(self, other)`, exceto que a regra de arredondamento de contexto é aplicada antes de retornar e que os valores `NaN` são sinalizados ou ignorados (dependendo do contexto e se estão sinalizando ou silenciosos).

max_mag(*other*, *context=None*)

Semelhante ao método **`max()`**, mas a comparação é feita usando os valores absolutos dos operandos.

min(*other*, *context=None*)

Como `min(self, other)`, exceto que a regra de arredondamento de contexto é aplicada antes de retornar e que os valores `NaN` são sinalizados ou ignorados (dependendo do contexto e se estão sinalizando ou silenciosos).

min_mag(*other*, *context=None*)

Semelhante ao método **`min()`**, mas a comparação é feita usando os valores absolutos dos operandos.

next_minus(*context=None*)

Retorna o maior número representável no contexto fornecido (ou no contexto atual da thread, se nenhum contexto for fornecido) que seja menor que o operando especificado.

next_plus(*context=None*)

Retorna o menor número representável no contexto fornecido (ou no contexto atual da thread, se nenhum contexto for fornecido) que seja maior que o operando fornecido.

next_toward(*other*, *context=None*)

Se os dois operandos forem desiguais, retorna o número mais próximo ao primeiro operando na direção do segundo operando. Se os dois operandos forem numericamente iguais, retorna uma cópia do primeiro

operando com o sinal configurado para ser o mesmo que o sinal do segundo operando.

normalize(*context=None*)

Usado para produzir valores canônicos de uma classe de equivalência no contexto atual ou no contexto especificado.

Esta tem a mesma semântica que a operação unária mais, exceto que se o resultado final for finito, ele será reduzido à sua forma mais simples, com todos os zeros à direita removidos e seu sinal preservado. Ou seja, enquanto o coeficiente for diferente de zero e múltiplo de dez, o coeficiente é dividido por dez e o expoente é incrementado em 1. Caso contrário (o coeficiente é zero), o expoente é definido como 0. Em todos os casos, o sinal permanece inalterado. .

Por exemplo, `Decimal('32.100')` e `Decimal('0.321000e+2')` ambos normalizam para o valor equivalente `Decimal('32.1')`.

Observe que o arredondamento é aplicado *antes* de reduzir para a forma mais simples.

Nas versões mais recentes da especificação, esta operação também é conhecida como `reduce`.

number_class(*context=None*)

Retorna uma string descrevendo a *classe* do operando. O valor retornado é uma das dez sequências a seguir.

- `"-Infinity"`, indicando que o operando é infinito negativo.
- `"-Normal"`, indicando que o operando é um número normal negativo.
- `"-Subnormal"`, indicando que o operando é negativo e subnormal.
- `"-Zero"`, indicando que o operando é um zero negativo.
- `"+Zero"`, indicando que o operando é um zero positivo.
- `"+Subnormal"`, indicando que o operando é positivo e subnormal.
- `"+Normal"`, indicando que o operando é um número normal positivo.
- `"+Infinity"`, indicando que o operando é infinito positivo.
- `"NaN"`, indicando que o operando é um NaN (“Not a Number”) silencioso.

- `"sNaN"`, indicando que o operando é um NaN sinalizador.

quantize(*exp*, *rounding=None*, *context=None*)

Retorna um valor igual ao primeiro operando após o arredondamento e com o expoente do segundo operando.

```
>>> Decimal('1.41421356').quantize(Decimal('1.000'))
Decimal('1.414')
```

Diferentemente de outras operações, se o comprimento do coeficiente após a operação de quantização for maior que a precisão, então **`InvalidOperation`** é sinalizado. Isso garante que, a menos que haja uma condição de erro, o expoente quantizado é sempre igual ao do operando do lado direito.

Também, diferentemente de outras operações, a quantização nunca sinaliza Underflow, mesmo que o resultado seja subnormal e inexato.

Se o expoente do segundo operando for maior que o do primeiro, o arredondamento poderá ser necessário. Nesse caso, o modo de arredondamento é determinado pelo argumento `rounding`, se fornecido, ou pelo argumento `context` fornecido; se nenhum argumento for fornecido, o modo de arredondamento do contexto da thread atual será usado.

Um erro é retornado sempre que o expoente resultante for maior que **`Emax`** ou menor que **`Etiny()`**.

radix()

Retorna `Decimal(10)`, a raiz (base) na qual a classe **`Decimal`** faz toda a sua aritmética. Incluído para compatibilidade com a especificação.

remainder_near(*other*, *context=None*)

Retorna o resto da divisão de *self* por *other*. Isso é diferente de `self % other`, pois o sinal do resto é escolhido para minimizar seu valor absoluto. Mais precisamente, o valor de retorno é `self - n * other`, onde `n` é o número inteiro mais próximo do valor exato de `self / other`, e se dois números inteiros estiverem igualmente próximos, o par será é escolhido.

Se o resultado for zero, seu sinal será o sinal de *self*.

```
>>> Decimal(18).remainder_near(Decimal(10))
Decimal('-2')
>>> Decimal(25).remainder_near(Decimal(10))
Decimal('5')
>>> Decimal(35).remainder_near(Decimal(10))
Decimal('-5')
```

rotate(*other*, *context=None*)

Retorna o resultado da rotação dos dígitos do primeiro operando em uma quantidade especificada pelo segundo operando. O segundo operando deve ser um número inteiro no intervalo - precisão através da precisão. O valor absoluto do segundo operando fornece o número de locais a serem rotacionados. Se o segundo operando for positivo, a rotação será para a esquerda; caso contrário, a rotação será para a direita. O coeficiente do primeiro operando é preenchido à esquerda com zeros na precisão do comprimento, se necessário. O sinal e o expoente do primeiro operando não são alterados.

same_quantum(*other*, *context=None*)

Testa se “self” e “other” têm o mesmo expoente ou se ambos são `NaN`.

Esta operação não é afetada pelo contexto e é silenciosa: nenhum sinalizador é alterado e nenhum arredondamento é executado. Como uma exceção, a versão C pode levantar InvalidOperation se o segundo operando não puder ser convertido exatamente.

scaleb(*other*, *context=None*)

Retorna o primeiro operando com o expoente ajustado pelo segundo. Da mesma forma, retorna o primeiro operando multiplicado por `10**other`. O segundo operando deve ser um número inteiro.

shift(*other*, *context=None*)

Retorna o resultado da troca dos dígitos do primeiro operando em uma quantidade especificada pelo segundo operando. O segundo operando deve ser um número inteiro no intervalo - precisão através da precisão. O valor absoluto do segundo operando fornece o número de locais a serem deslocados. Se o segundo operando for positivo, o deslocamento será para a esquerda; caso contrário, a mudança é para a direita. Os dígitos

deslocados para o coeficiente são zeros. O sinal e o expoente do primeiro operando não são alterados.

sqrt(*context=None*)

Retorna a raiz quadrada do argumento para a precisão total.

to_eng_string(*context=None*)

Converte em uma string, usando notação de engenharia, se for necessário um expoente.

A notação de engenharia possui um expoente que é múltiplo de 3. Isso pode deixar até 3 dígitos à esquerda da casa decimal e pode exigir a adição de um ou dois zeros à direita.

Por exemplo, isso converte `Decimal('123E+1')` para `Decimal('1.23E+3')`.

to_integral(*rounding=None*, *context=None*)

Idêntico ao método **`to_integral_value()`**. O nome `to_integral` foi mantido para compatibilidade com versões mais antigas.

to_integral_exact(*rounding=None*, *context=None*)

Arredonda para o número inteiro mais próximo, sinalizando **`Inexact`** ou **`Rounded`**, conforme apropriado, se o arredondamento ocorrer. O modo de arredondamento é determinado pelo parâmetro `rounding`, se fornecido, ou pelo `context` especificado. Se nenhum parâmetro for fornecido, o modo de arredondamento do contexto atual será usado.

to_integral_value(*rounding=None*, *context=None*)

Arredonda para o número inteiro mais próximo sem sinalizar **`Inexact`** ou **`Rounding`**. Se fornecido, aplica *rounding*; caso contrário, usa o método de arredondamento no *context* especificado ou no contexto atual.

Os números decimais podem ser arredondados usando a função **`round()`**:

round(number)

round(number, ndigits)

Se *ndigits* não for fornecido ou `None`, retorna o `int` de *number* mais próximo, arredondando até chegar em par, e ignorando o modo de arredondamento do contexto `Decimal`. Levanta `OverflowError` se *number* for um infinito ou `ValueError` se for um NaN (silencioso ou de sinalização).

Se *ndigits* for um `int`, o modo de arredondamento do contexto é respeitado e um `Decimal` representando *número* arredondado para o múltiplo mais próximo de `Decimal('1E-ndigits')` é retornado; neste caso, `round(number, ndigits)` é equivalente a `self.quantize(Decimal('1E-ndigits'))`. Retorna `Decimal('NaN')` se *number* for um NaN silencioso. Levanta `InvalidOperation` se *number* for um infinito, uma sinalização NaN, ou se o comprimento do coeficiente após a operação de quantização for maior que a precisão do contexto atual. Em outras palavras, para os situações comuns:

- se *ndigits* for positivo, retorna *number* arredondado para *ndigits* casas decimais;
- se *ndigits* for zero, retorna *number* arredondado para o número inteiro mais próximo;
- se *ndigits* for negativo, retorna *number* arredondado para o múltiplo mais próximo de `10**abs(ndigits)`.

Por exemplo:

```
>>> from decimal import Decimal, getcontext, ROUND_DOWN
>>> getcontext().rounding = ROUND_DOWN
>>> round(Decimal('3.75'))     # arredondamento de contexto i
4
>>> round(Decimal('3.5'))      # arredondamento para par mais
4
>>> round(Decimal('3.75'), 0)  # usa o arredondamento de cont
Decimal('3')
>>> round(Decimal('3.75'), 1)
Decimal('3.7')
>>> round(Decimal('3.75'), -1)
Decimal('0E+1')
```

Operandos lógicos

Os métodos `logical_and()`, `logical_invert()`, `logical_or()` e `logical_xor()` esperam que seus argumentos sejam *operandos lógicos*. Um *operando lógico* é uma instância de `Decimal` cujo expoente e sinal são zero e cujos dígitos são todos `0` ou `1`.

Objetos de contexto

Contextos são ambientes para operações aritméticas. Eles governam a precisão, estabelecem regras para arredondamento, determinam quais sinais são tratados como exceções e limitam o intervalo dos expoentes.

Cada thread possui seu próprio contexto atual que é acessado ou alterado usando as funções `getcontext()` e `setcontext()`:

decimal.getcontext()

Retorna o contexto atual para a thread ativa.

decimal.setcontext(*c*, *./*)

Define o contexto atual para a thread ativa como *C*.

Você também pode usar a instrução `with` e a função `localcontext()` para alterar temporariamente o contexto ativo.

decimal.localcontext(*ctx=None*, ***kwargs*)

Retorna um gerenciador de contexto que vai definir o contexto atual da thread ativa para uma cópia de *ctx* na entrada da instrução “with” e restaurar o contexto anterior ao sair da instrução “with”. Se nenhum contexto for especificado, uma cópia do contexto atual será usada. O argumento *kwargs* é usado para definir os atributos do novo contexto.

Por exemplo, o código a seguir define a precisão decimal atual para 42 casas, executa um cálculo e restaura automaticamente o contexto anterior:

```
from decimal import localcontext

with localcontext() as ctx:
    ctx.prec = 42   # Efetua um cálculo de alta precisão
    s = calculate_something()
s = +s  # Arredonda o resultado final de volta para a precisão pa
```

Usando argumentos nomeados, o código seria o seguinte:

```
from decimal import localcontext

with localcontext(prec=42) as ctx:
    s = calculate_something()
s = +s
```

Levanta `TypeError` se *kwargs* fornecer um atributo que `Context` não oferecer suporte. Levanta `TypeError` ou `ValueError` se *kwargs* fornecer um valor inválido para um atributo.

Alterado na versão 3.11: `localcontext()` agora tem suporte à configuração de atributos de contexto através do uso de argumentos nomeados.

decimal.IEEEContext(*bits*)

Retorna um objeto de contexto inicializado com os valores apropriados para um dos formatos de intercâmbio IEEE. O argumento deve ser um múltiplo de 32 e menor que `IEEE_CONTEXT_MAX_BITS`.

Adicionado na versão 3.14.

Novos contextos também podem ser criados usando o construtor `Context` descrito abaixo. Além disso, o módulo fornece três contextos pré-criados:

decimal.BasicContext

Este é um contexto padrão definido pela Especificação Aritmética Decimal Geral. A precisão está definida como nove. O arredondamento está definido como `ROUND_HALF_UP`. Todos os sinalizadores estão limpos. Todos as armadilhas estão ativadas (tratadas como exceções), exceto por `Inexact`, `Rounded` e `Subnormal`.

Como muitas das armadilhas estão ativadas, esse contexto é útil para depuração.

decimal.ExtendedContext

Este é um contexto padrão definido pela Especificação Aritmética Decimal Geral. A precisão está definida como nove. O arredondamento está definido como `ROUND_HALF_EVEN`. Todos os sinalizadores estão limpos. Nenhuma armadilha está ativada (de forma que exceções não são levantadas durante os cálculos).

Como as armadilhas estão desativadas, esse contexto é útil para aplicativos que preferem ter o valor de resultado de `NaN` ou `Infinity` em vez de levantar exceções. Isso permite que uma aplicação conclua uma execução na presença de condições que interromperiam o programa.

decimal.DefaultContext

Este contexto é usado pelo construtor **`Context`** como um protótipo para novos contextos. Alterar um campo (tal como precisão) tem o efeito de alterar o padrão para novos contextos criados pelo construtor **`Context`**.

Esse contexto é mais útil em ambientes multithread. A alteração de um dos campos antes do início das threads tem o efeito de definir os padrões para todo o sistema. Não é recomendável alterar os campos após o início das threads, pois exigiria sincronização de threads para evitar condições de corrida.

Em ambientes de thread única, é preferível não usar esse contexto. Em vez disso, basta criar contextos explicitamente, conforme descrito abaixo.

Os valores padrão são **`Context.prec`**=`28`, **`Context.rounding`**=**`ROUND_HALF_EVEN`** e armadilhas ativadas para **`Overflow`**, **`InvalidOperation`** e **`DivisionByZero`**.

Além dos três contextos fornecidos, novos contextos podem ser criados com o construtor **`Context`**.

class decimal.Context(*prec=None*, *rounding=None*, *Emin=None*, *Emax=None*, *capitals=None*, *clamp=None*, *flags=None*, *traps=None*)

Cria um novo contexto. Se um campo não for especificado ou for **`None`**, os valores padrão serão copiados de **`DefaultContext`**. Se o campo *flags* não for especificado ou for **`None`**, todos os sinalizadores serão limpados.

prec

Um inteiro no intervalo [`1`, **`MAX_PREC`**] que define a precisão para operações aritméticas no contexto.

rounding

Uma das constantes listadas na seção Modos de arredondamento.

traps

flags

Listas de todos os sinais a serem configurados. Geralmente, novos contextos devem apenas definir armadilhas e deixar os sinalizadores limpos.

Emin
Emax

Números inteiros que especificam os limites externos permitidos para expoentes. *Emin* deve estar no intervalo [`MIN_EMIN`, `0`], *Emax* no intervalo [`0`, `MAX_EMAX`].

capitals

Assume `0` ou `1` (o padrão). Se definido como `1`, os expoentes serão impressos com um `E` maiúsculo; caso contrário, um `e` minúsculo é usado: `Decimal('6.02e+23')`.

clamp

Assume `0` (o padrão) ou `1`. Se definido como `1`, o expoente `e` de uma instância de `Decimal` representável nesse contexto é estritamente limitado ao intervalo `Emin - prec + 1 <= e <= Emax - prec + 1`. Se *clamp* for `0`, uma condição mais fraca será mantida: o expoente ajustado da instância de **`Decimal`** é no máximo `Emax`. Quando *clamp* é `1`, um grande número normal terá, sempre que possível, seu expoente reduzido e um número correspondente de zeros adicionado ao seu coeficiente, para ajustar as restrições do expoente; isso preserva o valor do número, mas perde informações sobre zeros à direita significativos. Por exemplo:

```
>>> Context(prec=6, Emax=999, clamp=1).create_decimal('1.23e9
Decimal('1.23000E+999')
```

Um valor de *clamp* de `1` permite compatibilidade com os formatos de intercâmbio decimal de largura fixa especificados na IEEE 754.

A classe `Context` define vários métodos de uso geral, bem como um grande número de métodos para fazer aritmética diretamente em um determinado contexto. Além disso, para cada um dos métodos de `Decimal` descritos acima (com exceção dos métodos `adjusted()` e `as_tuple()`) existe um método correspondente em **`Context`**. Por exemplo, para uma instância `C` de **`Context`** e uma instância `x` de **`Decimal`**, `C.exp(x)` é equivalente a `x.exp(context=C)`.

Cada método de **`Context`** aceita um número inteiro do Python (uma instância de **`int`**) em qualquer lugar em que uma instância de Decimal seja aceita.

clear_flags()

Redefine todos os sinalizadores para `0`.

clear_traps()

Redefine todas as armadilhas para `0`.

Adicionado na versão 3.3.

copy()

Retorna uma duplicata do contexto.

copy_decimal(*num*, *./*)

Retorna uma cópia da instância de Decimal *num*.

create_decimal(*num='0'*, *./*)

Cria uma nova instância decimal a partir de *num*, mas usando *self* como contexto. Diferentemente do construtor de **`Decimal`**, a precisão do contexto, o método de arredondamento, os sinalizadores e as armadilhas são aplicadas à conversão.

Isso é útil porque as constantes geralmente são fornecidas com uma precisão maior do que a necessária pela aplicação. Outro benefício é que o arredondamento elimina imediatamente os efeitos indesejados dos dígitos além da precisão atual. No exemplo a seguir, o uso de entradas não arredondadas significa que adicionar zero a uma soma pode alterar o resultado:

```
>>> getcontext().prec = 3
>>> Decimal('3.4445') + Decimal('1.0023')
Decimal('4.45')
>>> Decimal('3.4445') + Decimal(0) + Decimal('1.0023')
Decimal('4.44')
```

Este método implementa a operação “to-number” da especificação IBM. Se o argumento for uma string, nenhum espaço em branco à esquerda ou à direita ou sublinhado serão permitidos.

create_decimal_from_float(*f*, *context*)

Cria uma nova instância de Decimal a partir de um ponto flutuante *f*, mas arredondando usando *self* como contexto. Diferentemente do método da classe **`Decimal.from_float()`**, a precisão do contexto, o método de arredondamento, os sinalizadores e as armadilhas são aplicados à conversão.

```
>>> context = Context(prec=5, rounding=ROUND_DOWN)
>>> context.create_decimal_from_float(math.pi)
Decimal('3.1415')
>>> context = Context(prec=5, traps=[Inexact])
>>> context.create_decimal_from_float(math.pi)
Traceback (most recent call last):
    ...
decimal.Inexact: None
```

Adicionado na versão 3.1.

Etiny()

Retorna um valor igual a `Emin - prec + 1`, que é o valor mínimo do expoente para resultados subnormais. Quando ocorre o estouro negativo, o expoente é definido como **`Etiny`**.

Etop()

Retorna um valor igual a `Emax - prec + 1`.

A abordagem usual para trabalhar com decimais é criar instâncias de **`Decimal`** e depois aplicar operações aritméticas que ocorrem no contexto atual da thread ativa. Uma abordagem alternativa é usar métodos de contexto para calcular dentro de um contexto específico. Os métodos são semelhantes aos da classe **`Decimal`** e são contados apenas brevemente aqui.

abs(*x*, *context*)

Retorna o valor absoluto de *x*.

add(*x*, *y*, *context*)

Retorna a soma de *x* e *y*.

canonical(*x*, *context*)

Retorna o mesmo objeto de Decimal *x*.

compare(*x*, *y*, *context*)

Compara *x* e *y* numericamente.

compare_signal(*x*, *y*, *context*)

Compara os valores dos dois operandos numericamente.

compare_total(*x*, *y*, *context*)

Compara dois operandos usando sua representação abstrata.

compare_total_mag(*x*, *y*, *context*)

Compara dois operandos usando sua representação abstrata, ignorando o sinal.

copy_abs(*x*, *context*)

Retorna uma cópia de *x* com o sinal definido para 0.

copy_negate(*x*, *context*)

Retorna uma cópia de *x* com o sinal invertido.

copy_sign(*x*, *y*, *context*)

Copia o sinal de *y* para *x*.

divide(*x*, *y*, *context*)

Retorna *x* dividido por *y*.

divide_int(*x*, *y*, *context*)

Retorna *x* dividido por *y*, truncado para um inteiro.

divmod(*x*, *y*, *context*)

Divide dois números e retorna a parte inteira do resultado.

exp(*x*, *context*)

Retorna e ** x.

fma(*x*, *y*, *z*, *context*)

Retorna *x* multiplicado por *y*, mais *z*.

is_canonical(*x*, *context*)

Retorna `True` se *x* for canonical; caso contrário, retorna `False`.

is_finite(*x*, *context*)

Retorna `True` se *x* for finito; caso contrário, retorna `False`.

is_infinite(*x*, *context*)

Retorna `True` se *x* for infinito; caso contrário, retorna `False`.

is_nan(*x*, *context*)

Retorna `True` se *x* for qNaN ou sNaN; caso contrário, retorna `False`.

is_normal(*x*, *context*)

Retorna `True` se *x* for um número normal; caso contrário, retorna `False`.

is_qnan(*x*, *context*)

Retorna `True` se *x* for um NaN silencioso; caso contrário, retorna `False`.

is_signed(*x*, *context*)

Retorna `True` se *x* for negativo; caso contrário, retorna `False`.

is_snan(*x*, *context*)

Retorna `True` se *x* for um NaN sinalizador; caso contrário, retorna `False`.

is_subnormal(*x*, *context*)

Retorna `True` se *x* for subnormal; caso contrário, retorna `False`.

is_zero(*x*, *context*)

Retorna `True` se *x* for zero; caso contrário, retorna `False`.

ln(*x*, *context*)

Retorna o logaritmo natural (base e) de *x*.

log10(*x*, *context*)

Retorna o logaritmo de base 10 de *x*.

logb(*x*, *context*)

Retorna o expoente da magnitude do MSD do operando.

logical_and(*x*, *y*, *context*)
Aplica a operação lógica *e* entre cada dígito do operando.

logical_invert(*x*, *context*)
Inverte todos os dígitos em *x*.

logical_or(*x*, *y*, *context*)
Aplica a operação lógica *ou* entre cada dígito do operando.

logical_xor(*x*, *y*, *context*)
Aplica a operação lógica *ou exclusivo* entre cada dígito do operando.

max(*x*, *y*, *context*)
Compara dois valores numericamente e retorna o máximo.

max_mag(*x*, *y*, *context*)
Compara dois valores numericamente com seu sinal ignorado.

min(*x*, *y*, *context*)
Compara dois valores numericamente e retorna o mínimo.

min_mag(*x*, *y*, *context*)
Compara dois valores numericamente com seu sinal ignorado.

minus(*x*, *context*)
Minus corresponde ao operador de subtração de prefixo unário no Python.

multiply(*x*, *y*, *context*)
Retorna o produto de *x* e *y*.

next_minus(*x*, *context*)
Retorna o maior número representável menor que *x*.

next_plus(*x*, *context*)

Retorna o menor número representável maior que *x*.

next_toward(*x*, *y*, *./.*)

Retorna o número mais próximo a *x*, em direção a *y*.

normalize(*x*, *./.*)

Reduz *x* para sua forma mais simples.

number_class(*x*, *./.*)

Retorna uma indicação da classe de *x*.

plus(*x*, *./.*)

Plus corresponde ao operador de soma de prefixo unário no Python. Esta operação aplica a precisão e o arredondamento do contexto, portanto *não* é uma operação de identidade.

power(*x*, *y*, *modulo=None*)

Retorna `x` à potência de `y`, com a redução de módulo `modulo` se fornecido.

Com dois argumentos, calcula `x**y`. Se `x` for negativo, `y` deve ser inteiro. O resultado será inexato, a menos que `y` seja inteiro e o resultado seja finito e possa ser expresso exatamente em "precisão" dígitos. O modo de arredondamento do contexto é usado. Os resultados são sempre arredondados corretamente na versão Python.

`Decimal(0) ** Decimal(0)` resulta em `InvalidOperation`, e se `InvalidOperation` não for capturado, resulta em `Decimal('NaN')`.

Alterado na versão 3.3: O módulo C calcula `power()` em termos das funções corretamente arredondadas `exp()` e `ln()`. O resultado é bem definido, mas apenas "quase sempre corretamente arredondado".

Com três argumentos, calcula `(x**y) % modulo`. Para o formulário de três argumentos, as seguintes restrições nos argumentos são válidas:

- todos os três argumentos devem ser inteiros
- `y` não pode ser negativo
- pelo menos um de `x` ou `y` não pode ser negativo

- `modulo` não pode ser zero e deve ter pelo menos “precisão” dígitos

O valor resultante de `Context.power(x, y, modulo)` é igual ao valor que seria obtido ao computar `(x**y) % modulo` com precisão ilimitada, mas é calculado com mais eficiência . O expoente do resultado é zero, independentemente dos expoentes de `x`, `y` e `modulo`. O resultado é sempre exato.

quantize(*x*, *y*, *context*)

Retorna um valor igual a *x* (arredondado), com o expoente de *y*.

radix()

Só retorna 10, já que isso é Decimal, :)

remainder(*x*, *y*, *context*)

Retorna o resto da divisão inteira.

O sinal do resultado, se diferente de zero, é o mesmo que o do dividendo original.

remainder_near(*x*, *y*, *context*)

Retorna `x - y * n`, onde *n* é o número inteiro mais próximo do valor exato de `x / y` (se o resultado for 0, seu sinal será o sinal de *x*).

rotate(*x*, *y*, *context*)

Retorna uma cópia re de *x*, *y* vezes.

same_quantum(*x*, *y*, *context*)

Retorna `True` se os dois operandos tiverem o mesmo expoente.

scaleb(*x*, *y*, *context*)

Retorna o primeiro operando após adicionar o segundo valor seu exp.

shift(*x*, *y*, *context*)

Retorna uma cópia deslocada de *x*, *y* vezes.

sqrt(*x*, *context*)

Raiz quadrada de um número não negativo para precisão do contexto.

subtract(*x*, *y*, *./*)

Retorna a diferença entre *x* e *y*.

to_eng_string(*x*, *./*)

Converte em uma string, usando notação de engenharia, se for necessário um expoente.

A notação de engenharia possui um expoente que é múltiplo de 3. Isso pode deixar até 3 dígitos à esquerda da casa decimal e pode exigir a adição de um ou dois zeros à direita.

to_integral_exact(*x*, *./*)

Arredonda para um número inteiro.

to_sci_string(*x*, *./*)

Converte um número em uma string usando notação científica.

Constantes

decimal.SPEC_VERSION

A versão mais recente da Especificação de Aritmética Decimal Geral (General Decimal Arithmetic Specification) com a qual esta implementação está em conformidade. Consulte https://speleotrove.com/decimal/decarith.html para acessar a especificação.

Adicionado na versão 3.15.

As seguintes constantes são relevantes apenas para o módulo C. Eles também estão incluídos na versão pura do Python para compatibilidade.

	32 bits	**64 bits**
decimal.MAX_PREC	`425000000`	`999999999999999999`
decimal.MAX_EMAX	`425000000`	`999999999999999999`

	32 bits	64 bits
decimal.MIN_EMIN	`-425000000`	`-999999999999999999`
decimal.MIN_ETINY	`-849999999`	`-1999999999999999997`
decimal.IEEE_CONTEXT_MAX_BITS	`256`	`512`

decimal.HAVE_THREADS

O valor é `True`. Descontinuado porque o Python agora sempre tem threads.

Descontinuado desde a versão 3.9.

decimal.HAVE_CONTEXTVAR

O valor padrão é `True`. Se o Python for **`configurado usando a opção --without-decimal-contextvar`**, a versão C usará um contexto local de thread em vez de local de corrotina e o valor será `False`. Isso é um pouco mais rápido em alguns cenários de contexto aninhados.

Adicionado na versão 3.8.3.

Modos de arredondamento

decimal.ROUND_CEILING

Arredonda para `Infinity`.

decimal.ROUND_DOWN

Arredonda para zero.

decimal.ROUND_FLOOR

Arredonda para `-Infinity`.

decimal.ROUND_HALF_DOWN

Arrendonda para o mais próximo com empates tendendo a zero.

decimal.ROUND_HALF_EVEN

Arredonda para o mais próximo com empates indo para o mais próximo inteiro par.

decimal.ROUND_HALF_UP

Arrendonda para o mais próximo com empates se afastando de zero.

decimal.ROUND_UP

Arredonda se afastando de zero.

decimal.ROUND_05UP

Arredonda se afastando de zero se o último dígito após o arredondamento para zero fosse 0 ou 5; caso contrário, arredonda para zero.

Sinais

Sinais representam condições que surgem durante o cálculo. Cada um corresponde a um sinalizador de contexto e um ativador de armadilha de contexto.

O sinalizador de contexto é definido sempre que a condição é encontrada. Após o cálculo, os sinalizadores podem ser verificados para fins informativos (por exemplo, para determinar se um cálculo era exato). Depois de verificar os sinalizadores, certifique-se de limpar todos os sinalizadores antes de iniciar o próximo cálculo.

Se o ativador de armadilha de contexto estiver definido para o sinal, a condição fará com que uma exceção Python seja levantada. Por exemplo, se a armadilha `DivisionByZero` for configurada, uma exceção `DivisionByZero` será levantada ao encontrar a condição.

class decimal.Clamped

Altera um expoente para ajustar as restrições de representação.

Normalmente, *clamping* ocorre quando um expoente fica fora dos limites do contexto `Emin` e `Emax`. Se possível, o expoente é reduzido para caber adicionando zeros ao coeficiente.

class decimal.DecimalException

Classe base para outros sinais e uma subclasse de `ArithmeticError`.

class decimal.DivisionByZero

Sinaliza a divisão de um número não infinito por zero.

Pode ocorrer com divisão, divisão de módulo ou ao elevar um número a uma potência negativa. Se este sinal não for capturado, retornará `Infinity` ou `-Infinity` com o sinal determinado pelas entradas do cálculo.

class decimal.Inexact

Indica que o arredondamento ocorreu e o resultado não é exato.

Sinaliza quando dígitos diferentes de zero foram descartados durante o arredondamento. O resultado arredondado é retornado. O sinalizador ou armadilha de sinal é usado para detectar quando os resultados são inexatos.

class decimal.InvalidOperation

Uma operação inválida foi realizada.

Indica que uma operação foi solicitada que não faz sentido. Se não for capturado, retorna `NaN`. As possíveis causas incluem:

```
Infinity - Infinity
0 * Infinity
Infinity / Infinity
x % 0
Infinity % x
sqrt(-x) and x > 0
0 ** 0
x ** (non-integer)
x ** Infinity
```

class decimal.Overflow

Estouro numérico.

Indica que o expoente é maior que `Context.Emax` após o arredondamento ocorrer. Se não for capturado, o resultado depende do modo de arredondamento, puxando para dentro para o maior número finito representável ou arredondando para fora para `Infinity`. Nos dois casos, `Inexact` e `Rounded` também são sinalizados.

class decimal.Rounded

O arredondamento ocorreu, embora possivelmente nenhuma informação tenha sido perdida.

Sinalizado sempre que o arredondamento descarta dígitos; mesmo que esses dígitos sejam zero (como arredondamento `5.00` a `5.0`). Se não for capturado, retorna o resultado inalterado. Este sinal é usado para detectar a perda de dígitos significativos.

class decimal.Subnormal

O expoente foi menor que `Emin` antes do arredondamento.

Ocorre quando um resultado da operação é subnormal (o expoente é muito pequeno). Se não for capturado, retorna o resultado inalterado.

class decimal.Underflow

Estouro negativo numérico com resultado arredondado para zero.

Ocorre quando um resultado subnormal é empurrado para zero arredondando. `Inexact` e `Subnormal` também são sinalizados.

class decimal.FloatOperation

Ativa semânticas mais rigorosas para misturar objetos de float com de Decimal.

Se o sinal não for capturado (padrão), a mistura de tipos float e Decimal será permitida no construtor `Decimal`, `create_decimal()` e em todos os operadores de comparação. Tanto a conversão quanto as comparações são exatas. Qualquer ocorrência de uma operação mista é registrada silenciosamente pela configuração `FloatOperation` nos sinalizadores de contexto. Conversões explícitas com `from_float()` ou `create_decimal_from_float()` não definem o sinalizador.

Caso contrário (o sinal é capturado), apenas comparações de igualdade e conversões explícitas são silenciosas. Todas as outras operações mistas levantam `FloatOperation`.

A tabela a seguir resume a hierarquia de sinais:

```
exceptions.ArithmeticError(exceptions.Exception)
    DecimalException
        Clamped
        DivisionByZero(DecimalException, exceptions.ZeroDivisionErr
        Inexact
            Overflow(Inexact, Rounded)
            Underflow(Inexact, Rounded, Subnormal)
        InvalidOperation
        Rounded
        Subnormal
        FloatOperation(DecimalException, exceptions.TypeError)
```

Observações sobre ponto flutuante

Atenuando o erro de arredondamento com precisão maior

O uso do ponto flutuante decimal elimina o erro de representação decimal (possibilitando representar `0.1` de forma exata); no entanto, algumas operações ainda podem sofrer erros de arredondamento quando dígitos diferentes de zero excederem a precisão fixa.

Os efeitos do erro de arredondamento podem ser amplificados pela soma ou subtração de valores que se compensam quase por completo, o que resulta em perda de significância. Knuth fornece dois exemplos instrutivos em que a aritmética de ponto flutuante arredondado com precisão insuficiente causa a quebra das propriedades associativas e distributivas da adição:

```
# Exemplos de Seminumerical Algorithms, Section 4.2.2.
>>> from decimal import Decimal, getcontext
>>> getcontext().prec = 8

>>> u, v, w = Decimal(11111113), Decimal(-11111111), Decimal('7.511
>>> (u + v) + w
Decimal('9.5111111')
>>> u + (v + w)
Decimal('10')

>>> u, v, w = Decimal(20000), Decimal(-6), Decimal('6.0000003')
>>> (u*v) + (u*w)
Decimal('0.01')
>>> u * (v+w)
Decimal('0.0060000')
```

O módulo **decimal** permite restaurar as identidades expandindo a precisão o suficiente para evitar perda de significância:

```
>>> getcontext().prec = 20
>>> u, v, w = Decimal(11111113), Decimal(-11111111), Decimal('7.511
>>> (u + v) + w
Decimal('9.51111111')
>>> u + (v + w)
Decimal('9.51111111')
>>>
>>> u, v, w = Decimal(20000), Decimal(-6), Decimal('6.0000003')
>>> (u*v) + (u*w)
Decimal('0.0060000')
>>> u * (v+w)
Decimal('0.0060000')
```

Valores especiais

O sistema numérico para o módulo **decimal** fornece valores especiais, incluindo `NaN`, `sNaN`, `-Infinity`, `Infinity`, e dois zeros, `+0` e `-0`.

Os infinitos podem ser construídos diretamente com: `Decimal('Infinity')`. Além disso, eles podem resultar da divisão por zero quando o sinal **DivisionByZero** não é capturado. Da mesma forma, quando o sinal **Overflow** não é capturado, o infinito pode resultar do arredondamento para além dos limites do maior número representável.

Os infinitos contêm sinais (afins) e podem ser usados em operações aritméticas, nas quais são tratados como números muito grandes e indeterminados. Por exemplo, adicionar uma constante ao infinito resulta em outro infinito.

Algumas operações são indeterminadas e retornam `NaN` ou, se o sinal **InvalidOperation** for capturado, levantam uma exceção. Por exemplo, `0/0` retorna `NaN`, que significa “não é um número” em inglês. Esta variação de `NaN` é silenciosa e, uma vez criada, fluirá para outros cálculos, sempre resultando em mais um `NaN`. Esse comportamento pode ser útil para uma série de cálculos que, ocasionalmente, têm entradas ausentes — ele permite que o cálculo continue enquanto sinaliza resultados específicos como inválidos.

Uma variante é `sNaN`, que sinaliza, em vez de permanecer em silêncio, após cada operação. Esse é um valor de retorno útil quando um resultado inválido precisa interromper um cálculo para tratamento especial.

O comportamento dos operadores de comparação do Python pode ser um pouco surpreendente quando há um `NaN` envolvido. Um teste de igualdade em que um dos operandos é um `NaN` silencioso ou sinalizador sempre retorna **`False`** (mesmo ao fazer `Decimal('NaN')==Decimal('NaN')`), enquanto um teste de desigualdade sempre retorna **`True`**. Uma tentativa de comparar dois decimais usando qualquer um dos operadores `<`, `<=`, `>` ou `>=` levantará o sinal **`InvalidOperation`** se um dos operandos for um `NaN` e retornará **`False`** se esse sinal não for capturado. Observe que a especificação aritmética decimal geral não especifica o comportamento das comparações diretas; as regras para comparações envolvendo um `NaN` foram retiradas do padrão IEEE 854 (consulte a Tabela 3 na seção 5.7). Para garantir uma conformidade rígida com os padrões, use os métodos **`compare()`** e **`compare_signal()`**.

Os zeros com sinais podem resultar de cálculos insuficientes. Eles mantêm o sinal que teria resultado se o cálculo tivesse sido realizado com maior precisão. Como sua magnitude é zero, os zeros positivos e negativos são tratados como iguais e seu sinal é informacional.

Além dos dois zeros com sinais que são distintos e iguais, existem várias representações de zero com diferentes precisões e ainda com valor equivalente. Isso leva um pouco de tempo para se acostumar. Para um olho acostumado a representações de ponto flutuante normalizadas, não é imediatamente óbvio que o seguinte cálculo retorne um valor igual a zero:

```
>>> 1 / Decimal('Infinity')
Decimal('0E-1000026')
```

Trabalhando com threads

A função **`getcontext()`** acessa um objeto **`Context`** diferente para cada thread. Ter contextos de threads separadas significa que as threads podem fazer alterações (como `getcontext().prec=10`) sem interferir em outras threads.

Da mesma forma, a função **`setcontext()`** atribui automaticamente seu alvo à thread atual.

Se `setcontext()` não tiver sido chamado antes de `getcontext()`, então **`getcontext()`** criará automaticamente um novo contexto para uso na thread atual. Novos objetos de contexto têm valores padrão definidos a partir do objeto `decimal.DefaultContext`.

O sinalizador `sys.flags.thread_inherit_context` afeta o contexto de novas threads. Se o sinalizador for falso, as novas threads começarão com um contexto vazio. Nesse caso, `getcontext()` criará um novo objeto de contexto quando chamado e usará os valores padrão de *DefaultContext*. Se o sinalizador for verdadeiro, as novas threads começarão com uma cópia do contexto do chamador de `threading.Thread.start()`.

Para controlar os padrões para que cada thread, use os mesmos valores em toda a aplicação, modifique diretamente o objeto *DefaultContext*. Isso deve ser feito *antes* de qualquer thread ser iniciada, para que não haja uma condição de corrida entre as threads chamando `getcontext()`. Por exemplo:

```
# Define padrões para todo a aplicação para todas as threads prestes
DefaultContext.prec = 12
DefaultContext.rounding = ROUND_DOWN
DefaultContext.traps = ExtendedContext.traps.copy()
DefaultContext.traps[InvalidOperation] = 1
setcontext(DefaultContext)

# Depois, as threads podem ser iniciadas
t1.start()
t2.start()
t3.start()
 . . .
```

Receitas

Aqui estão algumas receitas que servem como funções utilitárias e que demonstram maneiras de trabalhar com a classe `Decimal`:

```
def moneyfmt(value, places=2, curr='', sep=',', dp='.',
             pos='', neg='-', trailneg=''):
    """Converte Decimal para uma string formatada de moeda.

    places:  número obrigatório de casas após o ponto decimal
    curr:    símbolo de moeda opcional antes do sinal (pode estar em
```

```
    sep:     separador de agrupamento opcional (vírgula, ponto, espa
    dp:      indicador de ponto decimal (vírgula ou ponto)
             especificar como espaço em branco somente quando places
    pos:     sinal opcional para números positivos: '+', espaço ou e
    neg:     sinal opcional para números negativos: '-', '(', espaço
    trailneg:indicador de menos final opcional: '-', ')', espaço ou

    >>> d = Decimal('-1234567.8901')
    >>> moneyfmt(d, curr='$')
    '-$1,234,567.89'
    >>> moneyfmt(d, places=0, sep='.', dp='', neg='', trailneg='-')
    '1.234.568-'
    >>> moneyfmt(d, curr='$', neg='(', trailneg=')')
    '($1,234,567.89)'
    >>> moneyfmt(Decimal(123456789), sep=' ')
    '123 456 789.00'
    >>> moneyfmt(Decimal('-0.02'), neg='<', trailneg='>')
    '<0.02>'

    """
    q = Decimal(10) ** -places      # 2 casas --> '0.01'
    sign, digits, exp = value.quantize(q).as_tuple()
    result = []
    digits = list(map(str, digits))
    build, next = result.append, digits.pop
    if sign:
        build(trailneg)
    for i in range(places):
        build(next() if digits else '0')
    if places:
        build(dp)
    if not digits:
        build('0')
    i = 0
    while digits:
        build(next())
        i += 1
        if i == 3 and digits:
            i = 0
            build(sep)
    build(curr)
    build(neg if sign else pos)
    return ''.join(reversed(result))

def pi():
```

```
    """Calcula Pi com a precisão atual.

    >>> print(pi())
    3.141592653589793238462643383

    """
    getcontext().prec += 2  # dígitos extras para casas intermediár
    three = Decimal(3)      # substitui "three=3.0" para pontos flut
    lasts, t, s, n, na, d, da = 0, three, 3, 1, 0, 0, 24
    while s != lasts:
        lasts = s
        n, na = n+na, na+8
        d, da = d+da, da+32
        t = (t * n) / d
        s += t
    getcontext().prec -= 2
    return +s               # unário com mais aplica a nova precisão

def exp(x):
    """Retorna e elevado à potência de x. O tipo de resultado corres

    >>> print(exp(Decimal(1)))
    2.718281828459045235360287471
    >>> print(exp(Decimal(2)))
    7.389056098930650227230427461
    >>> print(exp(2.0))
    7.38905609893
    >>> print(exp(2+0j))
    (7.38905609893+0j)

    """
    getcontext().prec += 2
    i, lasts, s, fact, num = 0, 0, 1, 1, 1
    while s != lasts:
        lasts = s
        i += 1
        fact *= i
        num *= x
        s += num / fact
    getcontext().prec -= 2
    return +s

def cos(x):
    """Retorna o cosseno de x medido em radianos.
```

```
    A aproximação da série de Taylor funciona melhor para um valor p
    Para valores maiores, primeiro calcule x = x % (2 * pi).

    >>> print(cos(Decimal('0.5')))
    0.8775825618903727161162815826
    >>> print(cos(0.5))
    0.87758256189
    >>> print(cos(0.5+0j))
    (0.87758256189+0j)

    """
    getcontext().prec += 2
    i, lasts, s, fact, num, sign = 0, 0, 1, 1, 1, 1
    while s != lasts:
        lasts = s
        i += 2
        fact *= i * (i-1)
        num *= x * x
        sign *= -1
        s += num / fact * sign
    getcontext().prec -= 2
    return +s

def sin(x):
    """Retorna o seno de x medido em radianos.

    A aproximação da série de Taylor funciona melhor para um valor p
    Para valores maiores, primeiro calcule x = x % (2 * pi).

    >>> print(sin(Decimal('0.5')))
    0.4794255386042030002732879352
    >>> print(sin(0.5))
    0.479425538604
    >>> print(sin(0.5+0j))
    (0.479425538604+0j)

    """
    getcontext().prec += 2
    i, lasts, s, fact, num, sign = 1, 0, x, 1, x, 1
    while s != lasts:
        lasts = s
        i += 2
        fact *= i * (i-1)
        num *= x * x
        sign *= -1
```

```
        s += num / fact * sign
    getcontext().prec -= 2
    return +s
```

FAQ sobre Decimal

P: É complicado digitar `decimal.Decimal('1234.5')`. Existe uma maneira de minimizar a digitação ao usar o interpretador interativo?

R: Alguns usuários abreviam o construtor para apenas uma única letra:

```
>>> D = decimal.Decimal
>>> D('1.23') + D('3.45')
Decimal('4.68')
```

P: Em uma aplicação de ponto fixo com duas casas decimais, algumas entradas têm muitas casas e precisam ser arredondadas. Outros não devem ter dígitos em excesso e precisam ser validados. Quais métodos devem ser usados?

R: O método **`quantize()`** arredonda para um número fixo de casas decimais. Se a armadilha **`Inexact`** estiver configurada, também será útil para validação:

```
>>> TWOPLACES = Decimal(10) ** -2       # same as Decimal('0.01')

>>> # Round to two places
>>> Decimal('3.214').quantize(TWOPLACES)
Decimal('3.21')

>>> # Validate that a number does not exceed two places
>>> Decimal('3.21').quantize(TWOPLACES, context=Context(traps=[Inexa
Decimal('3.21')

>>> Decimal('3.214').quantize(TWOPLACES, context=Context(traps=[Inex
Traceback (most recent call last):
   ...
Inexact: None
```

P: Assim que eu tiver entradas de duas casas válidas, como mantenho essa invariante em uma aplicação?

R: Algumas operações como adição, subtração e multiplicação por um número inteiro preservam automaticamente o ponto fixo. Outras operações, como divisão e multiplicação não inteira, alteram o número de casas decimais e precisam ser seguidas com uma etapa **quantize()**:

```
>>> a = Decimal('102.72')           # Initial fixed-point values
>>> b = Decimal('3.17')
>>> a + b                           # Addition preserves fixed-point
Decimal('105.89')
>>> a - b
Decimal('99.55')
>>> a * 42                          # So does integer multiplication
Decimal('4314.24')
>>> (a * b).quantize(TWOPLACES)     # Must quantize non-integer mult
Decimal('325.62')
>>> (b / a).quantize(TWOPLACES)     # And quantize division
Decimal('0.03')
```

No desenvolvimento de aplicações de ponto fixo, é conveniente definir funções para manipular a etapa **quantize()**:

```
>>> def mul(x, y, fp=TWOPLACES):
...     return (x * y).quantize(fp)
...
>>> def div(x, y, fp=TWOPLACES):
...     return (x / y).quantize(fp)

>>> mul(a, b)                       # Automatically preserve fixed-p
Decimal('325.62')
>>> div(b, a)
Decimal('0.03')
```

P: Existem várias maneiras de expressar o mesmo valor. Os números `200`, `200.000`, `2E2` e `.02E+4` têm todos o mesmo valor em várias precisões. Existe uma maneira de transformá-los em um único valor canônico reconhecível?

R: O método **normalize()** mapeia todos os valores equivalentes para um único representativo:

```
>>> values = map(Decimal, '200 200.000 2E2 .02E+4'.split())
>>> [v.normalize() for v in values]
[Decimal('2E+2'), Decimal('2E+2'), Decimal('2E+2'), Decimal('2E+2')]
```

P: Quando ocorre o arredondamento em um cálculo?

R: Ocorre *após* o cálculo. A filosofia da especificação decimal é que os números são considerados exatos e criados independentemente do contexto atual. Eles podem até ter maior precisão do que o contexto atual. O processo de cálculo com essas entradas exatas e, em seguida, o arredondamento (ou outras operações de contexto) é aplicado ao *resultado* do cálculo:

```
>>> getcontext().prec = 5
>>> pi = Decimal('3.1415926535')   # Mais de 5 dígitos
>>> pi                             # Todos os dígitos são retidos
Decimal('3.1415926535')
>>> pi + 0                         # Arredondado após uma adição
Decimal('3.1416')
>>> pi - Decimal('0.00005')        # Subtrai números não arredondado
Decimal('3.1415')
>>> pi + 0 - Decimal('0.00005').   # Os valores intermediários são a
Decimal('3.1416')
```

P: Alguns valores decimais sempre são exibidas com notação exponencial. Existe uma maneira de obter uma representação não exponencial?

R: Para alguns valores, a notação exponencial é a única maneira de expressar o número de casas significativas no coeficiente. Por exemplo, expressar `5.0E+3` como `5000` mantém o valor constante, mas não pode mostrar a significância de duas casa do original.

Se uma aplicação não se importa com o rastreamento da significância, é fácil remover o expoente e os zeros à direita, perdendo a significância, mas mantendo o valor inalterado:

```
>>> def remove_exponent(d):
...     return d.quantize(Decimal(1)) if d == d.to_integral() else d
```

```
>>> remove_exponent(Decimal('5E+3'))
Decimal('5000')
```

P: Existe uma maneira de converter um float comum em um **`Decimal`**?

R: Sim, qualquer número de ponto flutuante binário pode ser expresso exatamente como um Decimal, embora uma conversão exata possa exigir mais

precisão do que a intuição sugere:

```
>>> Decimal(math.pi)
Decimal('3.141592653589793115997963468544185161590576171875')
```

P: Em um cálculo complexo, como posso ter certeza de que não obtive um resultado falso devido à precisão insuficiente ou a anomalias de arredondamento.

R: O módulo decimal facilita o teste de resultados. Uma boa prática é executar novamente os cálculos usando maior precisão e com vários modos de arredondamento. Resultados amplamente diferentes indicam precisão insuficiente, problemas no modo de arredondamento, entradas mal condicionadas ou um algoritmo numericamente instável.

P: Notei que a precisão do contexto é aplicada aos resultados das operações, mas não às entradas. Há algo a observar ao misturar valores de diferentes precisões?

R: Sim. O princípio é que todos os valores são considerados exatos, assim como a aritmética desses valores. Somente os resultados são arredondados. A vantagem das entradas é que “o que você vê é o que você obtém”. Uma desvantagem é que os resultados podem parecer estranhos se você esquecer que as entradas não foram arredondadas:

```
>>> getcontext().prec = 3
>>> Decimal('3.104') + Decimal('2.104')
Decimal('5.21')
>>> Decimal('3.104') + Decimal('0.000') + Decimal('2.104')
Decimal('5.20')
```

A solução é aumentar a precisão ou forçar o arredondamento das entradas usando a operação unária de mais:

```
>>> getcontext().prec = 3
>>> +Decimal('1.23456789')      # unary plus triggers rounding
Decimal('1.23')
```

Como alternativa, as entradas podem ser arredondadas na criação usando o método **`Context.create_decimal()`**:

```
>>> Context(prec=5, rounding=ROUND_DOWN).create_decimal('1.2345678')
Decimal('1.2345')
```

P: A implementação do CPython é rápida para números grandes?

A: Sim. Nas implementações CPython e PyPy3, as versões C/CFFI do módulo decimal integram a biblioteca de alta velocidade libmpdec [https://www.bytereef.org/mpdecimal/doc/libmpdec/index.html] para precisão arbitrária de aritmética de ponto flutuante decimal corretamente arredondado [[1]]. `libmpdec` usa a multiplicação de Karatsuba [https://pt.wikipedia.org/wiki/Algoritmo_de_Karatsuba] para números com tamanho médio e a Transformada Numérica de Fourier [https://en.wikipedia.org/wiki/Discrete_Fourier_transform_(general)#Number-theoretic_transform] para números muito grandes.

O contexto deve ser adaptado para uma aritmética exata de precisão arbitrária. **`Emin`** e **`Emax`** devem sempre ser configurados com os valores máximos, **`clamp`** deve sempre ser 0 (o padrão). A configuração de **`prec`** requer alguns cuidados.

A abordagem mais fácil para testar a aritmética do bignum é usar o valor máximo para **`prec`** também [[2]]:

```
>>> setcontext(Context(prec=MAX_PREC, Emax=MAX_EMAX, Emin=MIN_EMIN)
>>> x = Decimal(2) ** 256
>>> x / 128
Decimal('904625697166532776746648320380374280103671755200316906558
```

Para resultados inexatos, **`MAX_PREC`** é muito grande em plataformas de 64 bits e a memória disponível será insuficiente:

```
>>> Decimal(1) / 3
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
MemoryError
```

Em sistemas com alocação excessiva (por exemplo, Linux), uma abordagem mais sofisticada é ajustar **`prec`** à quantidade de RAM disponível. Suponha que você tenha 8 GB de RAM e espere 10 operandos simultâneos usando no máximo 500 MB cada:

```
>>> import sys
>>>
>>> # Número máximo de dígitos para um único operando usando 500 MB
>>> # em palavras de 8 bytes com 19 dígitos por palavra (4 bytes e 9
>>> # para a construção de 32 bits):
>>> maxdigits = 19 * ((500 * 1024**2) // 8)
```

```
>>>
>>> # Confere que isso funciona:
>>> c = Context(prec=maxdigits, Emax=MAX_EMAX, Emin=MIN_EMIN)
>>> c.traps[Inexact] = True
>>> setcontext(c)
>>>
>>> # Preenche a precisão disponível com noves:
>>> x = Decimal(0).logical_invert() * 9
>>> sys.getsizeof(x)
524288112
>>> x + 2
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  decimal.Inexact: [<class 'decimal.Inexact'>]
```

Em geral (e especialmente em sistemas sem alocação excessiva), recomenda-se estimar limites ainda mais apertados e definir a armadilha **`Inexact`** se for esperado que todos os cálculos sejam mais precisos.

[1]

Adicionado na versão 3.3.

[2]

Alterado na versão 3.9: Esta abordagem agora funciona para todos os resultados exatos, exceto para potências de números que não sejam inteiros.

`fractions` — Números racionais

Código-fonte: Lib/fractions.py
[https://github.com/python/cpython/tree/3.15/Lib/fractions.py]

O módulo **`fractions`** fornece suporte para aritmética de números racionais.

Uma instância de Fraction pode ser construída a partir de um par de números racionais, de um único número ou de uma string.

class fractions.Fraction(*numerator=0*, *denominator=1*)
class fractions.Fraction(*number*)
class fractions.Fraction(*string*)

A primeira versão requer que *numerator* e *denominator* sejam instâncias de **`numbers.Rational`** e retorna uma nova instância de **`Fraction`** com um valor igual a `numerator/denominator`. Se *denominator* for zero, ele levanta uma **`ZeroDivisionError`**.

A segunda versão requer que *number* seja uma instância de **`numbers.Rational`** ou tenha o método **`as_integer_ratio()`** (isso inclui **`float`** e **`decimal.Decimal`**). Ela retorna uma instância de **`Fraction`** com exatamente o mesmo valor. É presumido que o método **`as_integer_ratio()`** retorne um par de inteiros primos entre si e que o último seja positivo. Observe que, devido aos problemas comuns com pontos binários (consulte Aritmética de ponto flutuante: problemas e limitações), o argumento para `Fraction(1.1)` não é exatamente igual a 11/10 e, portanto, `Fraction(1.1)` *não* retorna `Fraction(11, 10)` como seria de se esperar. (Mas consulte a documentação do método **`limit_denominator()`** abaixo.)

A última versão do construtor espera uma string. O formato usual para esta instância é:

```
[sinal] numerador ['/' denominador]
```

onde o `sign` opcional pode ser '+' ou '-' e `numerator` e `denominator` (se presente) são strings de dígitos decimais (sublinhados podem ser usados para delimitar dígitos como com literais integrais no código). Além disso, qualquer string que represente um valor finito e seja aceita pelo construtor **`float`** também é aceita pelo construtor **`Fraction`**. Em qualquer forma, a string de entrada também pode ter espaços em branco à esquerda e/ou à direita. Aqui estão alguns exemplos:

```
>>> from fractions import Fraction
>>> Fraction(16, -10)
Fraction(-8, 5)
>>> Fraction(123)
Fraction(123, 1)
>>> Fraction()
Fraction(0, 1)
>>> Fraction('3/7')
Fraction(3, 7)
>>> Fraction(' -3/7 ')
Fraction(-3, 7)
>>> Fraction('1.414213 \t\n')
Fraction(1414213, 1000000)
>>> Fraction('-.125')
Fraction(-1, 8)
>>> Fraction('7e-6')
Fraction(7, 1000000)
>>> Fraction(2.25)
Fraction(9, 4)
>>> Fraction(1.1)
Fraction(2476979795053773, 2251799813685248)
>>> from decimal import Decimal
>>> Fraction(Decimal('1.1'))
Fraction(11, 10)
```

A classe **`Fraction`** herda da classe base abstrata **`numbers.Rational`** e implementa todos os métodos e operações dessa classe. As instâncias de **`Fraction`** são hasheável e devem ser tratadas como imutáveis. Além disso, **`Fraction`** tem as seguintes propriedades e métodos:

Alterado na versão 3.2: O construtor **`Fraction`** agora aceita instâncias **`float`** e **`decimal.Decimal`**.

Alterado na versão 3.9: A função `math.gcd()` agora é usada para normalizar o *numerator* e o *denominator*. `math.gcd()` sempre retorna um tipo `int`. Anteriormente, o tipo GCD dependia do *numerator* e do *denominator*.

Alterado na versão 3.11: Sublinhados agora são permitidos ao criar uma instância `Fraction` a partir de uma string, seguindo as regras **PEP 515** [https://peps.python.org/pep-0515/].

Alterado na versão 3.11: `Fraction` implementa `__int__` agora para satisfazer verificações de instância `typing.SupportsInt`.

Alterado na versão 3.12: É permitido espaço ao redor da barra para entradas de string: `Fraction('2 / 3')`.

Alterado na versão 3.12: Instâncias de `Fraction` agora oferecem suporte à formatação no estilo float, com tipos de apresentação `"e"`, `"E"`, `"f"`, `"F"`, `"g"`, `"G"` e `"%""`.

Alterado na versão 3.13: A formatação de instâncias `Fraction` sem um tipo de apresentação agora oferece suporte a preenchimento, alinhamento, tratamento de sinais, largura mínima e agrupamento.

Alterado na versão 3.14: O construtor `Fraction` agora aceita qualquer objeto com o método `as_integer_ratio()`.

numerator

Numerador de Fraction no menor termo.

denominator

Denominador de Fraction em termos mínimos. Garantidamente positivo.

as_integer_ratio()

Retorna uma tupla de dois inteiros, cuja razão é igual a Fraction original. A razão está em termos mais baixos e tem um denominador positivo.

Adicionado na versão 3.8.

is_integer()

Retorna `True` se a Fraction for um inteiro.

Adicionado na versão 3.12.

classmethod from_float(*f*)

Construtor alternativo que aceita apenas instâncias de **`float`** ou **`numbers.Integral`**. Esteja ciente de que `Fraction.from_float(0.3)` não é o mesmo valor que `Fraction(3, 10)`.

Nota

A partir do Python 3.2, você também pode construir uma instância **`Fraction`** diretamente de um **`float`**.

classmethod from_decimal(*dec*)

Construtor alternativo que aceita somente instâncias de **`decimal.Decimal`** ou **`numbers.Integral`**.

Nota

A partir do Python 3.2, você também pode construir uma instância **`Fraction`** diretamente de uma instância de **`decimal.Decimal`**.

classmethod from_number(*number*)

Construtor alternativo que aceita apenas instâncias de **`numbers.Integral`**, **`numbers.Rational`**, **`float`** ou **`decimal.Decimal`** e objetos com o método **`as_integer_ratio()`**, mas não strings.

Adicionado na versão 3.14.

limit_denominator(*max_denominator=1000000*)

Encontra e retorna o **`Fraction`** mais próximo de `self` que tem denominador no máximo max_denominator. Este método é útil para encontrar aproximações racionais para um dado número de ponto flutuante:

```
>>> from fractions import Fraction
>>> Fraction('3.1415926535897932').limit_denominator(1000
Fraction(355, 113)
```

ou para recuperar um número racional que é representado como um ponto flutuante:

```
>>> from math import pi, cos
>>> Fraction(cos(pi/3))
Fraction(4503599627370497, 9007199254740992)
>>> Fraction(cos(pi/3)).limit_denominator()
Fraction(1, 2)
>>> Fraction(1.1).limit_denominator()
Fraction(11, 10)
```

__floor__()

Retorna o maior **`int`** `<= self`. Este método também pode ser acessado por meio da função **`math.floor()`**:

```
>>> from math import floor
>>> floor(Fraction(355, 113))
3
```

__ceil__()

Retorna o menor **`int`** `>= self`. Este método também pode ser acessado por meio da função **`math.ceil()`**.

__round__()

__round__(*ndigits*)

A primeira versão retorna o **`int`** mais próximo de `self`, arredondando a metade para par. A segunda versão arredonda `self` para o múltiplo mais próximo de `Fraction(1, 10**ndigits)` (logicamente, se `ndigits` for negativo), novamente arredondando a

metade para par. Este método também pode ser acessado por meio da função **round()**.

__format__(*format_spec*, /)

Fornece suporte para formatação de instâncias de **Fraction** por meio do método **str.format()**, da função embutida **format()** ou literais de strings formatadas.

Se a string de especificação de formato `format_spec` não terminar com um dos tipos de apresentação `'e'`, `'E'`, `'f'`, `'F'`, `'g'`, `'G'` ou `'%'`, então a formatação segue as regras gerais para preenchimento, alinhamento, tratamento de sinais, largura mínima e agrupamento, conforme descrito na minilinguagem de especificação de formato. O sinalizador de "formato alternativo" `'#'` é suportado: se presente, ele força a string de saída a sempre incluir um denominador explícito, mesmo quando o valor que está sendo formatado é um inteiro exato. O sinalizador de preenchimento de zeros `'0'` não é suportado.

Se a string de especificação de formato `format_spec` terminar com um dos tipos de apresentação `'e'`, `'E'`, `'f'`, `'F'`, `'g'`, `'G'` ou `'%'`, então a formatação segue as regras descritas para o tipo **float** na seção Minilinguagem de especificação de formato.

Veja alguns exemplos:

```
>>> from fractions import Fraction
>>> format(Fraction(103993, 33102), '_')
'103_993/33_102'
>>> format(Fraction(1, 7), '.^+10')
'...+1/7...'
>>> format(Fraction(3, 1), '')
'3'
>>> format(Fraction(3, 1), '#')
'3/1'
>>> format(Fraction(1, 7), '.40g')
'0.1428571428571428571428571428571428571429'
>>> format(Fraction('1234567.855'), '_.2f')
'1_234_567.86'
```

```
>>> f"{Fraction(355, 113):*>20.6e}"
'********3.141593e+00'
>>> old_price, new_price = 499, 672
>>> "{:.2%} price increase".format(Fraction(new_price, ol
'34.67% price increase'
```

Ver também

Módulo `numbers`

As classes base abstratas que compõem a torre numérica.

`random` — Gera números pseudoaleatórios

Código-fonte: Lib/random.py
[https://github.com/python/cpython/tree/3.15/Lib/random.py]

Este módulo implementa geradores de números pseudoaleatórios para várias distribuições.

Para números inteiros, há uma seleção uniforme de um intervalo. Para sequências, há uma seleção uniforme de um elemento aleatório, uma função para gerar uma permutação aleatória de uma lista localmente e uma função para amostragem aleatória sem substituição.

Na linha real, existem funções para calcular distribuições uniforme, normal (gaussiana), log-normal, exponencial negativa, gama e beta. Para gerar distribuições de ângulos, a distribuição de von Mises está disponível.

Quase todas as funções do módulo dependem da função básica `random()`, que gera um ponto flutuante aleatório uniformemente no intervalo semiaberto `0.0 <= X < 1.0`. Python usa o Mersenne Twister como gerador de núcleo. Produz pontos flutuantes de precisão de 53 bits e possui um período de 2**19937-1. A implementação subjacente em C é rápida e segura para threads. O Mersenne Twister é um dos geradores de números aleatórios mais amplamente testados existentes. No entanto, sendo completamente determinístico, não é adequado para todos os fins e é totalmente inadequado para fins criptográficos.

As funções fornecidas por este módulo são, na verdade, métodos vinculados de uma instância oculta da classe `random.Random`. Você pode instanciar suas próprias instâncias de `Random` para obter geradores que não compartilham estado.

A classe `Random` também pode ser subclassificada se você quiser usar um gerador básico diferente de sua própria criação: veja a documentação dessa classe para mais detalhes.

O módulo `random` também fornece a classe `SystemRandom` que usa a função do sistema `os.urandom()` para gerar números aleatórios a partir de fontes fornecidas pelo sistema operacional.

Aviso

Os geradores pseudoaleatórios deste módulo não devem ser usados para fins de segurança. Para segurança ou uso criptográfico, veja o módulo `secrets`.

Ver também

M. Matsumoto and T. Nishimura, "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudorandom number generator", ACM Transactions on Modeling and Computer Simulation Vol. 8, No. 1, January pp.3–30 1998.

Receita de Complementary-Multiply-with-Carry [https://code.activestate.com/recipes/576707-long-period-random-number-generator/] para um gerador de números aleatórios alternativo compatível com um longo período e operações de atualização comparativamente simples.

Nota

O gerador global de números aleatórios e as instâncias de `Random` são seguros para thread. No entanto, na construção com threads livres, chamadas simultâneas ao gerador global ou à mesma instância de `Random` podem encontrar contenção e baixo desempenho. Considere usar instâncias separadas de `Random` por thread.

Funções de contabilidade

random.seed(*a=None*, *version=2*)

Inicializa o gerador de números aleatórios.

Se *a* for omitido ou `None`, a hora atual do sistema será usada. Se fontes de aleatoriedade são fornecidas pelo sistema operacional, elas são usadas no lugar da hora do sistema (consulte a função `os.urandom()` para detalhes sobre disponibilidade).

Se *a* é um int, seu valor absoluto é usado diretamente.

Com a versão 2 (o padrão), o objeto a `str`, `bytes` ou `bytearray` é convertido em um objeto `int` e todos os seus bits são usados.

Com a versão 1 (fornecida para reproduzir sequências aleatórias de versões mais antigas do Python), o algoritmo para `str` e `bytes` gera um intervalo mais restrito de sementes.

Alterado na versão 3.2: Movido para o esquema da versão 2, que usa todos os bits em uma semente de strings.

Alterado na versão 3.11: seed deve ser um dos seguintes tipos: `None`, `int`, `float`, `str`, `bytes` ou `bytearray`.

random.getstate()

Retorna um objeto capturando o estado interno atual do gerador. Este objeto pode ser passado para `setstate()` para restaurar o estado.

random.setstate(*state*)

state deveria ter sido obtido de uma chamada anterior para `getstate()`, e `setstate()` restaura o estado interno do gerador para o que era no momento que `getstate()` foi chamado.

Funções para bytes

random.randbytes(*n*)

Gera *n* bytes aleatórios.

Este método não deve ser usado para gerar tokens de segurança. Use `secrets.token_bytes()`.

Adicionado na versão 3.9.

Funções para inteiros

random.randrange(*stop*)

random.randrange(*start*, *stop*[, *step*])

Retorna um elemento selecionado aleatoriamente de `range(start, stop, step)`.

Isso é aproximadamente equivalente a `choice(range(start, stop, step))` mas provê suporte a intervalos arbitrariamente grandes e é otimizado para casos comuns.

O padrão de argumento posicional corresponde à função `range()`.

Argumentos nomeados não devem ser usados porque podem ser interpretados de maneiras inesperadas. Por exemplo `randrange(start=100)` é interpretado como `randrange(0, 100, 1)`.

Alterado na versão 3.2: `randrange()` é mais sofisticado em produzir valores igualmente distribuídos. Anteriormente, usava um estilo como `int(random()*n)`, que poderia produzir distribuições ligeiramente desiguais.

Alterado na versão 3.12: A conversão automática de tipos não inteiros não é mais suportada. Chamadas como `randrange(10.0)` e `randrange(Fraction(10, 1))` agora levantam uma `TypeError`.

random.randint(*a*, *b*)

Retorna um inteiro aleatório *N* de forma que `a <= N <= b`. Apelido para `randrange(a, b+1)`.

random.getrandbits(*k*)

Retorna um número inteiro Python não-negativo com bits aleatórios *k*. Este método é fornecido com o gerador Mersenne Twister e alguns outros geradores também podem fornecê-lo como uma parte opcional da API. Quando disponível, `getrandbits()` permite que `randrange()` manipule intervalos arbitrariamente grandes.

Alterado na versão 3.9: Este método agora aceita zero em *k*.

Funções para sequências

random.choice(*seq*)

Retorna um elemento aleatório da sequência não vazia *seq*. Se *seq* estiver vazio, levanta `IndexError`.

random.choices(*population*, *weights=None*, **, cum_weights=None*, *k=1*)

Retorna uma lista do tamanho de *k* de elementos escolhidos da *population* com substituição. Se a *population* estiver vazio, levanta `IndexError`.

Se uma sequência *weights* for especificada, as seleções serão feitas de acordo com os pesos relativos. Alternativamente, se uma sequência *cum_weights* for fornecida, as seleções serão feitas de acordo com os pesos cumulativos (talvez calculados usando `itertools.accumulate()`). Por exemplo, os pesos relativos `[10, 5, 30, 5]` são equivalentes aos pesos cumulativos `[10, 15, 45, 50]`. Internamente, os pesos relativos são convertidos em pesos acumulados antes de fazer seleções, portanto, fornecer pesos cumulativos economiza trabalho.

Se nem *weights* nem *cum_weights* forem especificados, as seleções serão feitas com igual probabilidade. Se uma sequência de pesos for fornecida, ela deverá ter o mesmo comprimento que a sequência

population. É um **TypeError** para especificar ambos os *weights* e *cum_weights*.

weights ou *cum_weights* pode usar qualquer tipo numérico que interopera com os valores **float** retornados por **random()** (que inclui inteiros, pontos flutuantes, e frações mas exclui decimais). Presume-se que pesos serão não-negativos e finitos. Uma **ValueError** é levantada se todos os pesos forem zero.

Para uma dada semente, a função **choices()** com igual peso normalmente produz uma sequência diferente das chamadas repetidas para **choice()**. O algoritmo usado por **choice()** usa aritmética de ponto flutuante para consistência e velocidade internas. O algoritmo usado por **choice()** assume como padrão aritmética inteira com seleções repetidas para evitar pequenos vieses de erro de arredondamento.

Adicionado na versão 3.6.

Alterado na versão 3.9: Levanta uma **ValueError** se todos os pesos forem zero.

random.shuffle(*x*)

Embaralha a sequência *x* internamente.

Para embaralhar uma sequência imutável e retornar uma nova lista embaralhada, use `sample(x, k=len(x))`.

Observe que, mesmo para pequenos `len(x)`, o número total de permutações de *x* pode crescer rapidamente maior que o período da maioria dos geradores de números aleatórios. Isso implica que a maioria das permutações de uma longa sequência nunca pode ser gerada. Por exemplo, uma sequência de comprimento 2080 é a maior que pode caber no período do gerador de números aleatórios Mersenne Twister.

Alterado na versão 3.11: Removido o parâmetro opcional *random*.

random.sample(*population*, *k*, *, *counts=None*)

Retorna uma lista de comprimento *k* de elementos únicos escolhidos na sequência populacional. Usado para amostragem aleatória sem reposição.

Retorna uma nova lista contendo elementos da população, mantendo a população original inalterada. A lista resultante está na ordem de seleção, para que todas as subfatias também sejam amostras aleatórias válidas. Isso permite que os vencedores do sorteio (a amostra) sejam divididos em grandes prêmios e vencedores de segundo lugar (as subfatias).

Os membros da população não precisam ser hasheáveis ou únicos. Se a população contiver repetições, cada ocorrência é uma seleção possível na amostra.

Elementos repetidos podem ser especificados um de cada vez ou com o parâmetro somente-nomeado opcional *count*. Por exemplo, `sample(['red', 'blue'], counts=[4, 2], k=5)` é equivalente a `sample(['red', 'red', 'red', 'red', 'blue', 'blue'], k=5)`.

Para escolher uma amostra de um intervalo de números inteiros, use um objeto `range()` como argumento. Isso é especialmente rápido e com eficiência de espaço para amostragem de uma grande população: `sample(range(10000000), k=60)`.

Se o tamanho da amostra for maior que o tamanho da população, uma `ValueError` é levantada.

Alterado na versão 3.9: Adicionado o parâmetro *counts*.

Alterado na versão 3.11: population deve ser uma sequência. A conversão automática de conjuntos em listas não é mais suportada.

Distribuições discretas

A função a seguir gera uma distribuição discreta.

random.binomialvariate(*n=1*, *p=0.5*)

Distribuição binomial [https://mathworld.wolfram.com/BinomialDistribution.html]. Retorna o número de sucessos para *n* tentativas independentes com a probabilidade de sucesso em cada tentativa sendo *p*:

Matematicamente equivalente a:

```
sum(random() < p for i in range(n))
```

O número de tentativas *n* deve ser um número inteiro não negativo. A probabilidade de sucesso *p* deve estar entre `0.0 <= p <= 1.0`. O resultado é um número inteiro no intervalo `0 <= X <= n`.

Adicionado na versão 3.12.

Distribuições com valor real

As funções a seguir geram distribuições específicas com valor real. Os parâmetros de função são nomeados após as variáveis correspondentes na equação da distribuição, conforme usadas na prática matemática comum; a maioria dessas equações pode ser encontrada em qualquer texto estatístico.

random.random()

Retorna o próximo número aleatório de ponto flutuante no intervalo `0.0 <= X < 1.0`.

random.uniform(*a*, *b*)

Retorna um número de ponto flutuante aleatório *N* de forma que `a <= N <= b` para `a <= b` e `b <= N <= a` para `b < a`.

O valor final `b` pode ou não ser incluído no intervalo dependendo do arredondamento de ponto flutuante na expressão `a + (b-a) * random()`.

random.triangular(*low*, *high*, *mode*)

Retorna um número de ponto flutuante aleatório *N* de forma que `low <= N <= high` e com o *mode* especificado entre esses limites. Os limites

low e *high* são padronizados como zero e um. O argumento *mode* assume como padrão o ponto médio entre os limites, fornecendo uma distribuição simétrica.

random.betavariate(*alpha*, *beta*)

Distribuição beta. As condições nos parâmetros são `alpha > 0` e `beta > 0`. Os valores retornados variam entre 0 e 1.

random.expovariate(*lambd=1.0*)

Distribuição exponencial. *lambd* é 1.0 dividido pela média desejada. Deve ser diferente de zero. (O parâmetro seria chamado "lambda", mas é uma palavra reservada em Python.) Os valores retornados variam de 0 a infinito positivo se *lambd* for positivo e de infinito negativo a 0 se *lambd* for negativo.

Alterado na versão 3.12: Adicionado o valor padrão para `lambd`.

random.gammavariate(*alpha*, *beta*)

Distribuição gama. (*Não* a função gama!) Os parâmetros de forma e escala, *alfa* e *beta*, devem ter valores positivos. (As convenções de chamada variam e algumas fontes definem 'beta' como o inverso da escala).

A função de distribuição de probabilidade é:

```
            x ** (alpha - 1) * math.exp(-x / beta)
pdf(x) =  --------------------------------------
              math.gamma(alpha) * beta ** alpha
```

random.gauss(*mu=0.0*, *sigma=1.0*)

Distribuição normal, também chamada de distribuição gaussiana. *mu* é a média e *sigma* é o desvio padrão. Isto é um pouco mais rápido que a função **`normalvariate()`** definida abaixo.

Nota sobre multithreading: quando duas threads chamam esta função simultaneamente, é possível que recebam o mesmo valor de retorno. Isso pode ser evitado de três maneiras. 1) Fazer com que cada thread

use uma instância diferente do gerador de números aleatórios. 2) Colocar travas em todas as chamadas. 3) Usar a função mais lenta, mas segura para thread `normalvariate()`.

Alterado na versão 3.11: mu e *sigma* agora têm argumentos padrão.

random.lognormvariate(*mu*, *sigma*)

Distribuição log normal. Se você usar o logaritmo natural dessa distribuição, obterá uma distribuição normal com média *mu* e desvio padrão *sigma*. *mu* pode ter qualquer valor e *sigma* deve ser maior que zero.

random.normalvariate(*mu=0.0*, *sigma=1.0*)

Distribuição normal. *mu* é a média e *sigma* é o desvio padrão.

Alterado na versão 3.11: mu e *sigma* agora têm argumentos padrão.

random.vonmisesvariate(*mu*, *kappa*)

mu é o ângulo médio, expresso em radianos entre 0 e 2**pi*, e *kappa* é o parâmetro de concentração, que deve ser maior ou igual a zero. Se *kappa* for igual a zero, essa distribuição será reduzida para um ângulo aleatório uniforme no intervalo de 0 a 2**pi*.

random.paretovariate(*alpha*)

Distribuição de Pareto. *alpha* é o parâmetro de forma.

random.weibullvariate(*alpha*, *beta*)

Distribuição Weibull. *alpha* é o parâmetro de escala e *beta* é o parâmetro de forma.

Gerador alternativo

class random.Random([*seed*])

Classe que implementa o gerador de números pseudoaleatórios padrão usado pelo módulo `random`.

Alterado na versão 3.11: Anteriormente, o *seed* poderia ser qualquer objeto hasheável. Agora está limitado a: `None`, `int`, `float`, `str`, `bytes` ou `bytearray`.

Subclasses de **`Random`** devem substituir os seguintes métodos se desejarem fazer uso de um gerador básico diferente:

seed(*a=None*, *version=2*)

Substitua este método nas subclasses para personalizar o comportamento de `seed()` das instâncias de **`Random`**.

getstate()

Substitua este método nas subclasses para personalizar o comportamento de `getstate()` das instâncias de **`Random`**.

setstate(*state*)

Substitua este método nas subclasses para personalizar o comportamento de `setstate()` das instâncias de **`Random`**.

random()

Substitua este método nas subclasses para personalizar o comportamento de `random()` das instâncias de **`Random`**.

Opcionalmente, uma subclasse geradora personalizada também pode fornecer o seguinte método:

getrandbits(*k*)

Substitua este método nas subclasses para personalizar o comportamento de `getrandbits()` das instâncias de **`Random`**.

randbytes(*n*)

Substitua este método nas subclasses para personalizar o comportamento de `randbytes()` das instâncias de **`Random`**.

class random.SystemRandom([*seed*])

Classe que usa a função `os.urandom()` para gerar números aleatórios a partir de fontes fornecidas pelo sistema operacional. Não disponível em todos os sistemas. Não depende do estado do software e as sequências não são reproduzíveis. Assim, o método `seed()` não tem efeito e é ignorado. Os métodos `getstate()` e `setstate()` levantam `NotImplementedError` se chamados.

Notas sobre reprodutibilidade

Às vezes, é útil poder reproduzir as sequências fornecidas por um gerador de números pseudoaleatórios. Reutilizando um valor inicial, a mesma sequência deve ser reproduzível de execução para execução, desde que mais de uma thread não estejam em execução.

A maioria dos algoritmos e funções de propagação do módulo aleatório está sujeita a alterações nas versões de Python, mas dois aspectos são garantidos para não serem alterados:

- Se um novo método de semeadura for adicionado, será oferecida um semeador compatível com versões anteriores.
- O método `random()` do gerador continuará produzindo a mesma sequência quando o semeador compatível receber a mesma semente.

Exemplos

Exemplos básicos:

```
>>> random()                            # Ponto flutuante aleatóri
0.37444887175646646

>>> uniform(2.5, 10.0)                  # Ponto flutuante aleatóri
3.1800146073117523

>>> expovariate(1 / 5)                  # Intervalo entre chegadas
5.148957571865031

>>> randrange(10)                       # Inteiro de 0 até 9 inclu
7
```

```
>>> randrange(0, 101, 2)              # Inteiro par entre 0 até
26

>>> choice(['win', 'lose', 'draw'])   # Um único elemento aleató
'draw'

>>> deck = 'ace two three four'.split()
>>> shuffle(deck)                     # Embaralha uma lista
>>> deck
['four', 'two', 'ace', 'three']

>>> sample([10, 20, 30, 40, 50], k=4) # Quatro amostras sem subs
[40, 10, 50, 30]
```

Simulações:

```
>>> # Seis giros da roleta (amostragem ponderada com substituiçã
>>> choices(['red', 'black', 'green'], [18, 18, 2], k=6)
['red', 'green', 'black', 'black', 'red', 'black']

>>> # Distribui 20 cartas sem reposição de um baralho de
>>> # 52 cartas e determine a proporção de cartas com
>>> # valor dez: dez, valete, rainha ou rei.
>>> deal = sample(['tens', 'low cards'], counts=[16, 36], k=20)
>>> deal.count('tens') / 20
0.15

>>> # Estima a probabilidade de obter 5 ou mais caras em 7 giros
>>> # de uma moeda viciada que cai em cara 60% das vezes.
>>> sum(binomialvariate(n=7, p=0.6) >= 5 for i in range(10_000))
0.4169

>>> # Probabilidade da mediana de 5 amostras estar nos dois quar
>>> def trial():
...     return 2_500 <= sorted(choices(range(10_000), k=5))[2] <
...
>>> sum(trial() for i in range(10_000)) / 10_000
0.7958
```

Exemplo de bootstrapping estatístico [https://pt.wikipedia.org/wiki/Bootstrapping_(estatística)] usando reamostragem com substituição para estimar um intervalo de confiança para a média de uma amostra:

```
# https://www.thoughtco.com/example-of-bootstrapping-3126155
from statistics import fmean as mean
from random import choices

data = [41, 50, 29, 37, 81, 30, 73, 63, 20, 35, 68, 22, 60, 31,
means = sorted(mean(choices(data, k=len(data))) for i in range(1
print(f'The sample mean of {mean(data):.1f} has a 90% confidence
      f'interval from {means[5]:.1f} to {means[94]:.1f}')
```

Exemplo de um teste de permutação de reamostragem [https://en.wikipedia.org/wiki/Resampling_(statistics)#Permutation_tests] para determinar a significância estatística ou valor-p [https://pt.wikipedia.org/wiki/Valor-p] de uma diferença observada entre os efeitos de uma droga em comparação com um placebo:

```
# Exemplo de "Statistics is Easy" de Dennis Shasha e Manda Wilsc
from statistics import fmean as mean
from random import shuffle

drug = [54, 73, 53, 70, 73, 68, 52, 65, 65]
placebo = [54, 51, 58, 44, 55, 52, 42, 47, 58, 46]
observed_diff = mean(drug) - mean(placebo)

n = 10_000
count = 0
combined = drug + placebo
for i in range(n):
    shuffle(combined)
    new_diff = mean(combined[:len(drug)]) - mean(combined[len(dr
    count += (new_diff >= observed_diff)

print(f'{n} label reshufflings produced only {count} instances w
print(f'at least as extreme as the observed difference of {obser
print(f'The one-sided p-value of {count / n:.4f} leads us to rej
print(f'hypothesis that there is no difference between the drug
```

Simulação de tempos de chegada e entregas de serviços para uma fila multisservidor:

```
from heapq import heapify, heapreplace
from random import expovariate, gauss
from statistics import mean, quantiles

average_arrival_interval = 5.6
average_service_time = 15.0
stdev_service_time = 3.5
num_servers = 3

waits = []
arrival_time = 0.0
servers = [0.0] * num_servers  # hora em que cada servidor fica
heapify(servers)
for i in range(1_000_000):
    arrival_time += expovariate(1.0 / average_arrival_interval)
    next_server_available = servers[0]
    wait = max(0.0, next_server_available - arrival_time)
    waits.append(wait)
    service_duration = max(0.0, gauss(average_service_time, stde
    service_completed = arrival_time + wait + service_duration
    heapreplace(servers, service_completed)

print(f'Mean wait: {mean(waits):.1f}   Max wait: {max(waits):.1f
print('Quartiles:', [round(q, 1) for q in quantiles(waits)])
```

Ver também

Statistics for Hackers [https://www.youtube.com/watch?v=Iq9DzN6mvYA] um tutorial em vídeo por Jake Vanderplas [https://us.pycon.org/2016/speaker/profile/295/] sobre análise estatística usando apenas alguns conceitos fundamentais, incluindo simulação, amostragem, embaralhamento e validação cruzada.

Economics Simulation [https://nbviewer.org/url/norvig.com/ipython/Economics.ipynb] uma simulação de um mercado por Peter Norvig [https://norvig.com/bio.html] que mostra o uso eficaz de muitas das ferramentas e distribuições fornecidas por este módulo (gauss, uniform, sample, betavariate, choice, triangular e randrange).

A Concrete Introduction to Probability (using Python) [https://nbviewer.org/url/norvig.com/ipython/Probability.ipynb] um tutorial de Peter Norvig [https://norvig.com/bio.html] cobrindo os fundamentos da teoria das probabilidades, como escrever simulações e como realizar análises de dados usando Python.

Receitas

Estas receitas mostram como fazer seleções aleatórias de forma eficiente a partir dos iteradores combinatórios no módulo **`itertools`** ou no projeto more-itertools [https://pypi.org/project/more-itertools/]:

```
import random

def random_product(*iterables, repeat=1):
    "Random selection from itertools.product(*iterables, repeat=
    pools = tuple(map(tuple, iterables)) * repeat
    return tuple(map(random.choice, pools))

def random_permutation(iterable, r=None):
    "Random selection from itertools.permutations(iterable, r)"
    pool = tuple(iterable)
    r = len(pool) if r is None else r
    return tuple(random.sample(pool, r))

def random_combination(iterable, r):
    "Random selection from itertools.combinations(iterable, r)"
    pool = tuple(iterable)
    n = len(pool)
    indices = sorted(random.sample(range(n), r))
    return tuple(pool[i] for i in indices)

def random_combination_with_replacement(iterable, r):
    "Choose r elements with replacement.  Order the result to ma
    # Result will be in set(itertools.combinations_with_replacem
    pool = tuple(iterable)
    n = len(pool)
    indices = sorted(random.choices(range(n), k=r))
    return tuple(pool[i] for i in indices)
```

```
def random_derangement(iterable):
    "Choose a permutation where no element stays in its original
    seq = tuple(iterable)
    if len(seq) < 2:
        if not seq:
            return ()
        raise IndexError('No derangments to choose from')
    perm = list(range(len(seq)))
    start = tuple(perm)
    while True:
        random.shuffle(perm)
        if all(p != q for p, q in zip(start, perm)):
            return tuple([seq[i] for i in perm])
```

O padrão de `random()` é retornar múltiplos de 2^{-53} no intervalo $0.0 \le x < 1.0$. Todos esses números são espaçados uniformemente e são representáveis exatamente como pontos flutuantes Python. No entanto, muitos outros pontos flutuantes representados nesse intervalo não são seleções possíveis. Por exemplo, `0.05954861408025609` não é um múltiplo inteiro de 2^{-53}.

A receita a seguir tem uma abordagem diferente. Todos os pontos flutuantes no intervalo são seleções possíveis. A mantissa vem de uma distribuição uniforme de inteiros no intervalo $2^{52} \le mantissa < 2^{53}$. O expoente vem de uma distribuição geométrica onde expoentes menores que -53 ocorrem com metade da frequência do próximo expoente maior.

```
from random import Random
from math import ldexp

class FullRandom(Random):

    def random(self):
        mantissa = 0x10_0000_0000_0000 | self.getrandbits(52)
        exponent = -53
        x = 0
        while not x:
            x = self.getrandbits(32)
```

```
            exponent += x.bit_length() - 32
        return ldexp(mantissa, exponent)
```

Todas as distribuições reais valoradas na classe usarão o novo método:

```
>>> fr = FullRandom()
>>> fr.random()
0.05954861408025609
>>> fr.expovariate(0.25)
8.87925541791544
```

A receita é conceitualmente equivalente a um algoritmo que escolhe entre todos os múltiplos de 2^{-1074} no intervalo $0.0 \le x < 1.0$. Todos esses números são espaçados uniformemente, mas a maioria deve ser arredondada para o ponto flutuante Python representável mais próximo. (O valor 2^{-1074} é o menor ponto flutuante positivo não normalizado e é igual a `math.ulp(0.0)`.)

Ver também

Generating Pseudo-random Floating-Point Values [https://allendowney.com/research/rand/downey07randfloat.pdf] um artigo de Allen B. Downey descrevendo formas de gerar pontos flutuantes mais refinados do que normalmente gerados por **`random()`**.

Uso na linha de comando

Adicionado na versão 3.13.

O módulo **`random`** pode ser invocado como um script na linha de comando:

```
python -m random [-h] [-c ESCOLHA [ESCOLHA ...] | -i N | -f N] [
```

As seguintes opções são aceitas:

-h, --help

Mostra a mensagem de ajuda e sai.

-c CHOICE [CHOICE ...]
--choice CHOICE [CHOICE ...]

Imprime uma escolha aleatória, usando `choice()`.

-i <N>
--integer <N>

Imprime um inteiro aleatório entre 1 e N inclusive, usando `randint()`.

-f <N>
--float <N>

Imprime um número de ponto flutuante aleatório entre 0 e N inclusive, usando `uniform()`.

Se nenhuma opção for fornecida, a saída dependerá da entrada:

- String ou vários: mesmo que `--choice`.
- Inteiro mesmo que `--integer`.
- Ponto flutuante: mesmo que `--float`.

Exemplos na linha de comando

Aqui estão alguns exemplos da interface de linha de comando **`random`**:

```
$ # Escolhe um aleatório
$ python -m random egg bacon sausage spam "Lobster Thermidor aux
Lobster Thermidor aux crevettes with a Mornay sauce

$ # Inteiro aleatório
$ python -m random 6
6

$ # Número de ponto flutuante aleatório
$ python -m random 1.8
1.7080016272295635

$ # Com argumentos explícitos
$ python  -m random --choice egg bacon sausage spam "Lobster The
```

```
egg

$ python -m random --integer 6
3

$ python -m random --float 1.8
1.5666339105010318

$ python -m random --integer 6
5

$ python -m random --float 6
3.1942323316565915
```

`statistics` — Funções estatísticas

Adicionado na versão 3.4.

Código-fonte: Lib/statistics.py
[https://github.com/python/cpython/tree/3.15/Lib/statistics.py]

Esse módulo fornece funções para o cálculo de estatísticas matemáticas de dados numéricos (para valores do tipo `Real`).

O módulo não tem a intenção de competir com bibliotecas de terceiros como NumPy [https://numpy.org], SciPy [https://scipy.org/], ou pacotes proprietários de estatísticas com todos os recursos destinados a estatísticos profissionais como Minitab, SAS e Matlab. Ela destina-se ao nível de calculadoras gráficas e científicas.

A menos que seja explicitamente indicado, essas funções suportam `int`, `float`, `Decimal` e `Fraction`. O uso com outros tipos (sejam numéricos ou não) não é atualmente suportado. Coleções com uma mistura de tipos também são indefinidas e dependentes da implementação. Se os seus dados de entrada consistem de tipos misturados, você pode usar `map()` para garantir um resultado consistente, por exemplo `map(float, dado_entrada)`.

Alguns conjuntos de dados usam valores `NaN` (não um número) para representar dados ausentes. Como os NaNs possuem semântica de comparação incomum, eles causam comportamentos surpreendentes ou indefinidos nas funções estatísticas que classificam dados ou contam ocorrências. As funções afetadas são `median()`, `median_low()`, `median_high()`, `median_grouped()`, `mode()`, `multimode()` e `quantiles()`. Os valores `NaN` devem ser removidos antes de chamar estas funções:

```
>>> from statistics import median
>>> from math import isnan
>>> from itertools import filterfalse
```

```
>>> data = [20.7, float('NaN'),19.2, 18.3, float('NaN'), 14.4]
>>> sorted(data)  # Isso tem um comportamento surpreendente
[20.7, nan, 14.4, 18.3, 19.2, nan]
>>> median(data)  # Este resultado é inesperado
16.35

>>> sum(map(isnan, data))    # Número de valores em falta
2
>>> clean = list(filterfalse(isnan, data))  # Remove valores NaN
>>> clean
[20.7, 19.2, 18.3, 14.4]
>>> sorted(clean)  # A classificação agora funciona como esperado
[14.4, 18.3, 19.2, 20.7]
>>> median(clean)       # Este resultado é agora bem definido
18.75
```

Médias e medidas de valor central

Essas funções calculam a média ou o valor típico de uma população ou amostra.

`mean()`	Média aritmética dos dados.
`fmean()`	Média aritmética rápida de ponto flutuante, com ponderação opcional.
`geometric_mean()`	Média geométrica dos dados.
`harmonic_mean()`	Média harmônica dos dados.
`kde()`	Estima a distribuição de densidade de probabilidade dos dados.

`kde_random()`	Amostragem aleatória do PDF gerado por kde().
`median()`	Mediana (valor do meio) dos dados.
`median_low()`	Mediana inferior dos dados.
`median_high()`	Mediana superior dos dados.
`median_grouped()`	Mediana (50º percentil) dos dados agrupados.
`mode()`	Moda (valor mais comum) de dados discretos ou nominais.
`multimode()`	Lista de modas (valores mais comuns) de dados discretos ou nominais.
`quantiles()`	Divide os dados em intervalos com probabilidade igual.

Medidas de espalhamento

Essas funções calculam o quanto a população ou amostra tendem a desviar dos valores típicos ou médios.

`pstdev()`	Desvio padrão populacional dos dados.
`pvariance()`	Variância populacional dos dados.

`stdev()`	Desvio padrão amostral dos dados.
`variance()`	Variância amostral dos dados.

Estatísticas para relações entre duas entradas

Essas funções calculam estatísticas sobre relações entre duas entradas.

`covariance()`	Covariância amostral para duas variáveis.
`correlation()`	Coeficientes de correlação de Pearson e Spearman.
`linear_regression()`	Inclinação e intersecção para regressão linear simples.

Detalhes das funções

Nota: as funções não exigem que os dados estejam ordenados. No entanto, para conveniência do leitor, a maioria dos exemplos mostrará sequências ordenadas.

statistics.mean(*data*)

Retorna a média aritmética amostral de *data* que pode ser uma sequência ou iterável.

A média aritmética é a soma dos dados dividida pela quantidade de dados. É comumente chamada apenas de “média”, apesar de ser uma das diversas médias matemáticas. Ela representa uma medida da localização central dos dados.

Se *data* for vazio, uma exceção do tipo **`StatisticsError`** será levantada.

Alguns exemplos de uso:

```
>>> mean([1, 2, 3, 4, 4])
2.8
>>> mean([-1.0, 2.5, 3.25, 5.75])
2.625

>>> from fractions import Fraction as F
>>> mean([F(3, 7), F(1, 21), F(5, 3), F(1, 3)])
Fraction(13, 21)

>>> from decimal import Decimal as D
>>> mean([D("0.5"), D("0.75"), D("0.625"), D("0.375")])
Decimal('0.5625')
```

Nota

A média é fortemente afetada por outliers [https://pt.wikipedia.org/wiki/Outlier] (valor atípico) e não é necessariamente um exemplo típico dos pontos de dados. Para uma medida mais robusta, embora menos eficiente, de tendência central [https://pt.m.wikipedia.org/wiki/Tendência_central], veja **`median()`**.

A média amostral fornece uma estimativa não enviesada da média populacional verdadeira, ou seja, quando a média é obtida para todas as possíveis amostras, `mean(sample)` converge para a média verdadeira de toda população. Se *data* representa toda população ao invés de uma amostra, então `mean(data)` é equivalente a calcular a verdadeira média populacional μ.

statistics.fmean(*data*, *weights=None*)

Converte valores em *data* para ponto flutuante e calcula a média aritmética.

Essa função executa mais rapidamente do que a função **`mean()`** e sempre retorna um **`float`**. *data* pode ser uma sequência ou iterável. Se o

conjunto de dados de entrada estiver vazio, levanta uma exceção **StatisticsError**.

```
>>> fmean([3.5, 4.0, 5.25])
4.25
```

A ponderação opcional é suportada. Por exemplo, um professor atribui uma nota para um curso ponderando questionários em 20%, trabalhos de casa em 20%, um exame de meio de período em 30% e um exame final em 30%:

```
>>> grades = [85, 92, 83, 91]
>>> weights = [0.20, 0.20, 0.30, 0.30]
>>> fmean(grades, weights)
87.6
```

Se *weights* for fornecido, deve ter o mesmo comprimento que *data* ou uma **ValueError** será levantada.

Adicionado na versão 3.8.

Alterado na versão 3.11: Adicionado suporte a *weights*.

statistics.geometric_mean(*data*)

Converte valores em *data* para ponto flutuante e calcula a média geométrica.

A média geométrica indica a tendência central ou valor típico de *data* usando o produto dos valores (em oposição à média aritmética que usa a soma deles).

Levanta uma exceção **StatisticsError** se a entrada do conjunto de dados for vazia, contiver um zero ou um valor negativo. *data* pode ser uma sequência ou iterável.

Nenhum esforço especial é feito para alcançar resultados exatos. (Mas, isso pode mudar no futuro).

```
>>> round(geometric_mean([54, 24, 36]), 1)
36.0
```

Adicionado na versão 3.8.

statistics.harmonic_mean(*data*, *weights=None*)

Retorna a média harmônica de *data*, uma sequência ou iterável de números de valor real. Se *weights* for omitido ou `None`, então a ponderação igual é presumida.

A média harmônica é a recíproca da média arimética calculada pela função **`mean()`** dos recíprocos dos dados. Por exemplo, a média harmônica de três valores *a*, *b* e *c* será equivalente a `3/(1/a + 1/b + 1/c)`. Se um dos valores for zero, o resultado também será zero.

A média harmônica é um tipo de média, uma medida de localização central dos dados. Ela é geralmente apropriada quando se está calculando a média de razões e taxas; por exemplo, velocidades.

Suponha que um carro viaje 10 km a 40 km/h, e em seguida viaje mais 10 km a 60 km/h. Qual é a velocidade média?

```
>>> harmonic_mean([40, 60])
48.0
```

Suponha que um carro viaja a 40 km/h por 5 km e, quando o trânsito melhora, acelera para 60 km/h pelos 30 km restantes da viagem. Qual é a velocidade média?

```
>>> harmonic_mean([40, 60], weights=[5, 30])
56.0
```

`StatisticsError` é levantada se *data* for vazio, qualquer elemento for menor que zero, ou se a soma ponderada não for positiva.

O algoritmo atual tem uma saída antecipada quando encontra um zero na entrada. Isso significa que as entradas subsequentes não tem a validade testada. (Esse comportamento pode mudar no futuro.)

Adicionado na versão 3.6.

Alterado na versão 3.10: Adicionado suporte a *weights*.

statistics.kde(*data*, *h*, *kernel='normal'*, *, *cumulative=False*)

Estimativa de densidade por Kernel (KDE) [https://www.itm-conferences.org/articles/itmconf/pdf/2018/08/itmconf_sam2018_00037.pdf]: Cria uma função de densidade de probabilidade contínua ou uma função de distribuição cumulativa a partir de amostras discretas.

A ideia básica é suavizar os dados usando uma função kernel [https://en.wikipedia.org/wiki/Kernel_(statistics)]. para ajudar a tirar inferências sobre uma população a partir de uma amostra.

O grau de suavização é controlado pelo parâmetro de escala *h*, que é chamado de largura de banda. Valores menores enfatizam características locais, enquanto valores maiores dão resultados mais suaves.

O *kernel* determina os pesos relativos dos pontos de dados da amostra. Geralmente, a escolha do formato de kernel não importa tanto quanto o parâmetro de suavização de largura de banda mais influente.

Os kernels que dão algum peso a cada ponto de amostra incluem *normal* (*gauss*), *logistic* e *sigmoid*.

Os kernels que apenas dão peso aos pontos de amostra dentro da largura de banda incluem *rectangular* (*uniform*), *triangular*, *parabolic* (*epanechnikov*), *quartic* (*biweight*), *triweight* e *cosine*.

Se *cumulative* for verdadeiro, retornará uma função de distribuição cumulativa.

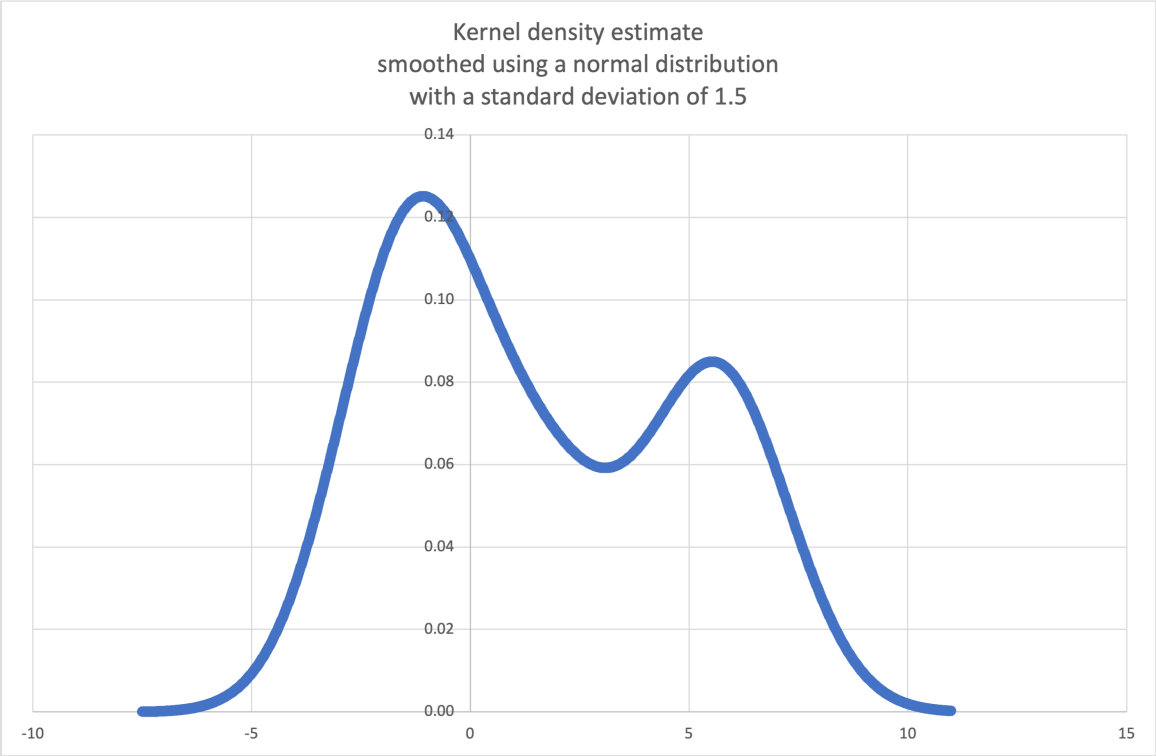
Uma **`StatisticsError`** será levantada se a sequência de dados em *data* estiver vazia.

A Wikipédia tem um exemplo onde podemos usar **`kde()`** para gerar e plotar uma função de densidade de probabilidade estimada a partir de uma pequena amostra:

```
>>> sample = [-2.1, -1.3, -0.4, 1.9, 5.1, 6.2]
>>> f_hat = kde(sample, h=1.5)
```

```
>>> xarr = [i/100 for i in range(-750, 1100)]
>>> yarr = [f_hat(x) for x in xarr]
```

Os pontos em `xarr` e `yarr` podem ser usados para fazer um gráfico PDF:



Adicionado na versão 3.13.

statistics.kde_random(*data*, *h*, *kernel='normal'*, *, *seed=None*)

Retorna uma função que faz uma seleção aleatória da função de densidade de probabilidade estimada produzida por `kde(data, h, kernel)`.

Fornecer uma *seed* permite seleções reproduzíveis. No futuro, os valores podem mudar um pouco conforme estimativas de funções de distribuição acumulada (CDF) inversas de kernel mais precisas forem implementadas. *seed* pode ser um inteiro, float, str ou bytes.

Uma **`StatisticsError`** será levantada se a sequência de dados em *data* estiver vazia.

Continuando o exemplo de `kde()`, podemos usar `kde_random()` para gerar novas seleções aleatórias a partir de uma função de densidade de probabilidade estimada:

```
>>> data = [-2.1, -1.3, -0.4, 1.9, 5.1, 6.2]
>>> rand = kde_random(data, h=1.5, seed=8675309)
>>> new_selections = [rand() for i in range(10)]
>>> [round(x, 1) for x in new_selections]
[0.7, 6.2, 1.2, 6.9, 7.0, 1.8, 2.5, -0.5, -1.8, 5.6]
```

Adicionado na versão 3.13.

statistics.median(*data*)

Retorna a mediana (o valor do meio) de dados numéricos, usando o método comum de "média entre os dois do meio". Se *data* for vazio, é levantada uma exceção `StatisticsError`. *data* pode ser uma sequência ou um iterável.

A mediana é uma medida robusta de localização central e é menos afetada por valores discrepantes. Quando a quantidade de pontos de dados for ímpar, o valor de meio é retornado:

```
>>> median([1, 3, 5])
3
```

Quando o número de elementos for par, a mediana é calculada tomando-se a média entre os dois valores no meio:

```
>>> median([1, 3, 5, 7])
4.0
```

Isso serve quando seus dados forem discretos e você não se importa que a média possa não ser um valor que de fato ocorre nos seus dados.

Caso os dados sejam ordinais (oferecem suporte para operações de ordenação) mas não são numéricos (não oferecem suporte para adição), considere usar a função `median_low()` ou `median_high()` no lugar.

statistics.median_low(*data*)

Retorna a mediana inferior de dados numéricos. Se *data* for vazio, a exceção `StatisticsError` é levantada. *data* pode ser uma sequência ou um iterável.

A mediana inferior sempre é um membro do conjunto de dados. Quando o número de elementos for ímpar, o valor intermediário é retornado. Se houver um número par de elementos, o menor entre os dois valores centrais é retornado.

```
>>> median_low([1, 3, 5])
3
>>> median_low([1, 3, 5, 7])
3
```

Use a mediana inferior caso seus dados forem discretos e você prefira que a mediana seja um valor que de fato existe nos seus dados ao invés de um valor interpolado.

statistics.median_high(*data*)

Retorna a mediana superior de dados numéricos. Se *data* for vazio, a exceção `StatisticsError` é levantada. *data* pode ser uma sequência ou um iterável.

A mediana superior sempre é um membro do conjunto de dados. Quando o número de elementos for ímpar, o valor intermediário é retornado. Se houver um número par de elementos, o maior entre os dois valores centrais é retornado.

```
>>> median_high([1, 3, 5])
3
>>> median_high([1, 3, 5, 7])
5
```

Use a mediana superior caso seus dados forem discretos e você prefira que a mediana seja um valor que de fato existe nos seus dados ao invés de um valor interpolado.

statistics.median_grouped(*data*, *interval=1.0*)

Estima a mediana para dados numéricos que foram agrupados ou "binned" [https://en.wikipedia.org/wiki/Data_binning] em torno dos pontos médios de intervalos consecutivos de largura fixa.

data pode ser qualquer iterável de dados numéricos com cada valor sendo exatamente o ponto médio de um bin. Pelo menos um valor deve estar presente.

interval é a largura de cada compartimento.

Por exemplo, as informações demográficas podem ter sido resumidas em grupos etários consecutivos de dez anos, com cada grupo sendo representado pelos pontos médios de cinco anos dos intervalos:

```
>>> from collections import Counter
>>> demographics = Counter({
...    25: 172,   # 20 to 30 years old
...    35: 484,   # 30 to 40 years old
...    45: 387,   # 40 to 50 years old
...    55:  22,   # 50 to 60 years old
...    65:   6,   # 60 to 70 years old
... })
...
```

O 50º percentil (mediana) é a 536ª pessoa de 1071 membros. Essa pessoa está na faixa etária de 30 a 40 anos.

A função regular `median()` iria presumir que todos na faixa etária tricenariana tinham exatamente 35 anos. Uma suposição mais sustentável é que os 484 membros dessa faixa etária estão distribuídos uniformemente entre 30 e 40. Para isso, usamos `median_grouped()`:

```
>>> data = list(demographics.elements())
>>> median(data)
35
>>> round(median_grouped(data, interval=10), 1)
37.5
```

O chamador é responsável por garantir que os pontos de dados sejam separados por múltiplos exatos de *intervalo*. Isso é essencial para obter

um resultado correto. A função não verifica essa pré-condição.

As entradas podem ser qualquer tipo numérico que possa ser convertido para um ponto flutuante durante a etapa de interpolação.

statistics.mode(*data*)

Retorna o valor mais comum dos dados discretos ou nominais em *data*. A moda (quando existe) é o valor mais típico e serve como uma medida de localização central.

Se existirem múltiplas modas com a mesma frequência, retorna a primeira encontrada em *data*. Se ao invés disso se desejar a menor ou a maior dentre elas, use `min(multimode(data))` ou `max(multimode(data))`. Se a entrada *data* é vazia, a exceção `StatisticsError` é levantada.

`mode` presume que os dados são discretos e retorna um único valor. Esse é o tratamento padrão do conceito de moda normalmente ensinado nas escolas:

```
>>> mode([1, 1, 2, 3, 3, 3, 3, 4])
3
```

A moda é única no sentido que é a única medida estatística nesse módulo que também se aplica a dados nominais (não-numéricos):

```
>>> mode(["red", "blue", "blue", "red", "green", "red", "red'
'red'
```

Somente entradas hasheáveis são suportadas. Para manipular o tipo `set`, considere fazer a conversão para `frozenset`. Para manipular o tipo `list`, considere fazer a conversão para `tuple`. Para entradas mistas ou aninhadas, considere usar este algoritmo quadrático mais lento que depende apenas de testes de igualdade: `max(data, key=data.count)`.

Alterado na versão 3.8: Agora lida com conjunto de dados multimodais retornando a primeira moda encontrada. Anteriormente, ela levantava a

exceção `StatisticsError` quando mais do que uma moda era encontrada.

statistics.multimode(*data*)

Retorna uma lista dos valores mais frequentes na ordem em que eles foram encontrados em *data*. Irá retornar mais do que um resultado se houver múltiplas modas ou uma lista vazia se *data* for vazio.

```
>>> multimode('aabbbbccddddeeffffgg')
['b', 'd', 'f']
>>> multimode('')
[]
```

Adicionado na versão 3.8.

statistics.pstdev(*data*, *mu=None*)

Retorna o desvio padrão populacional (a raiz quadrada da variância populacional). Veja os argumentos e outros detalhes em `pvariance()`.

```
>>> pstdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75])
0.986893273527251
```

statistics.pvariance(*data*, *mu=None*)

Retorna a variância populacional de *data*, que deve ser uma sequência ou iterável não-vazio de números reais. A variância, o segundo momento estatístico a partir da média, é uma medida da variabilidade (espalhamento ou dispersão) dos dados. Uma variância grande indica que os dados são espalhados; uma variância menor indica que os dados estão agrupado em volta da média.

Se o segundo argumento opcional *mu* for fornecido, ele deve ser a média da *população* dos dados em *data*. Ele também pode ser usado para calcular o segundo momento em torno de um ponto que não é a média. Se estiver faltando ou for `None` (o padrão), a média aritmética é calculada automaticamente.

Use essa função para calcular a variância de toda a população. Para estimar a variância de uma amostra, a função `variance()` costuma ser

uma escolha melhor.

Levanta `StatisticsError` se *data* for vazio.

Exemplos:

```
>>> data = [0.0, 0.25, 0.25, 1.25, 1.5, 1.75, 2.75, 3.25]
>>> pvariance(data)
1.25
```

Se você já calculou a média dos seus dados, você pode passar o valor no segundo argumento opcional *mu* para evitar recálculos:

```
>>> mu = mean(data)
>>> pvariance(data, mu)
1.25
```

Decimais e frações são suportadas:

```
>>> from decimal import Decimal as D
>>> pvariance([D("27.5"), D("30.25"), D("30.25"), D("34.5"),
Decimal('24.815')

>>> from fractions import Fraction as F
>>> pvariance([F(1, 4), F(5, 4), F(1, 2)])
Fraction(13, 72)
```

Nota

Quando os dados de entrada representarem toda a população, ele retorna a variância populacional σ^2. Se em vez disso, amostras forem usadas, então a variância amostral enviesada s^2, também conhecida como variância com N graus de liberdade é retornada.

Se de alguma forma você souber a verdadeira média populacional μ, você pode usar essa função para calcular a variância de uma amostra, fornecendo a média populacional conhecida como segundo argumento. Caso seja fornecido um conjunto de amostras aleatórias da população, o resultado será um estimador não enviesado da variância populacional.

statistics.stdev(*data*, *xbar=None*)

Retorna o desvio padrão amostral (a raiz quadrada da variância amostral). Veja **`variance()`** para argumentos e outros detalhes.

```
>>> stdev([1.5, 2.5, 2.5, 2.75, 3.25, 4.75])
1.0810874155219827
```

statistics.variance(*data*, *xbar=None*)

Retorna a variância amostral de *data*, que deve ser um iterável com pelo menos dois números reais. Variância, ou o segundo momento estatístico a partir da média, é uma medida de variabilidade (espalhamento ou dispersão) dos dados. Uma variância grande indica que os dados são espalhados, uma variância menor indica que os dados estão agrupados em volta da média.

Se o segundo argumento opcional *xbar* for dado, ele deve a média *sample* de *data*. Se ele não estiver presente ou for `None` (valor padrão), a média é automaticamente calculada.

Use essa função quando seus dados representarem uma amostra da população. Para calcular a variância de toda população veja **`pvariance()`**.

Levanta a exceção **`StatisticsError`** se *data* tiver menos do que dois valores.

Exemplos:

```
>>> data = [2.75, 1.75, 1.25, 0.25, 0.5, 1.25, 3.5]
>>> variance(data)
1.3720238095238095
```

Se você já calculou a média amostral dos seus dados, você pode passar o valor no segundo argumento opcional *xbar* para evitar recálculos:

```
>>> m = mean(data)
>>> variance(data, m)
1.3720238095238095
```

Essa função não verifica se você passou a média verdadeira como *xbar*. Usar valores arbitrários para *xbar* pode levar a resultados inválidos ou impossíveis.

Decimais e frações são suportados.

```
>>> from decimal import Decimal as D
>>> variance([D("27.5"), D("30.25"), D("30.25"), D("34.5"), D
Decimal('31.01875')

>>> from fractions import Fraction as F
>>> variance([F(1, 6), F(1, 2), F(5, 3)])
Fraction(67, 108)
```

Nota

Essa é a variância amostral s^2 com a correção de Bessel, também conhecida como variância com N-1 graus de liberdade. Desde que os pontos de dados sejam representativos (por exemplo, independentes e distribuídos de forma idêntica), o resultado deve ser uma estimativa não enviesada da verdadeira variação populacional.

Caso você de alguma forma saiba a verdadeira média populacional μ você deveria passar para a função `pvariance()` como o parâmetro *mu* para obter a variância da amostra.

statistics.quantiles(*data*, *, *n=4*, *method='exclusive'*)

Divide *data* em *n* intervalos contínuos com igual probabilidade. Retorna uma lista de `n - 1` pontos de corte separando os intervalos.

Defina *n* como 4 para quartis (o padrão). Defina *n* como 10 para decis. Defina *n* como 100 para percentis, o que fornece os 99 pontos de corte que separam *data* em 100 grupos de tamanhos iguais. Levanta a exceção `StatisticsError` se *n* não for pelo menos 1.

data pode ser qualquer iterável contendo dados amostrais. Para resultados significativos, a quantidade de dados em *data* deve ser maior

do que *n*. Levanta a exceção `StatisticsError` se não houver pelo menos um ponto de dados.

Os pontos de corte são linearmente interpolados a partir dos dois pontos mais próximos. Por exemplo, se um ponto de corte cair em um terço da distância entre dois valores, `100` e `112`, o ponto de corte será avaliado como `104`.

O *method* para computar quantis pode variar dependendo se *data* incluir ou excluir os menores e maiores valores possíveis da população.

O valor padrão do parâmetro *method* é “exclusive” e é usado para dados amostrados de uma população que pode ter valores mais extremos do que os encontrados nas amostras. A porção da população que fica abaixo do *i-ésimo* item de *m* pontos ordenados é calculada como `i / (m + 1)`. Dados nove valores, o método os ordena e atribui a eles os seguintes percentis: 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%.

Definimos o parâmetro *method* para “inclusive” para descrever dados da população ou para amostras que são conhecidas por incluir os valores mais extremos da população. O mínimo valor em *data* é tratado como o percentil 0 e o máximo valor é tratado como percentil 100. A porção da população que fica abaixo do *i-ésimo* item de *m* pontos ordenados é calculada como `(i - 1) / (m - 1)`. Dados 11 valores, o método os ordena e atribui a eles os seguintes percentis: 0%, 10%, 20%, 30%, 40%, 50%, 60%, 70%, 80%, 90%, 100%.

```
# Pontos de corte de decil para dados amostrados empiricament
>>> data = [105, 129, 87, 86, 111, 111, 89, 81, 108, 92, 110,
...         100, 75, 105, 103, 109, 76, 119, 99, 91, 103, 129
...         106, 101, 84, 111, 74, 87, 86, 103, 103, 106, 86,
...         111, 75, 87, 102, 121, 111, 88, 89, 101, 106, 95,
...         103, 107, 101, 81, 109, 104]
>>> [round(q, 1) for q in quantiles(data, n=10)]
[81.0, 86.2, 89.0, 99.4, 102.5, 103.6, 106.0, 109.8, 111.0]
```

Adicionado na versão 3.8.

Alterado na versão 3.13: Não levanta mais uma exceção para uma entrada com apenas um único ponto de dados. Isso permite que estimativas de quantil sejam construídas um ponto de amostra por vez, tornando-se gradualmente mais refinadas com cada novo ponto de dados.

statistics.covariance(*x*, *y*, /)

Retorna a covariância da amostra de duas sequências *x* e *y*. Covariância é uma medida da variabilidade conjunta de duas entradas.

Ambas as entradas devem ter o mesmo comprimento (não menos que dois), caso contrário, **`StatisticsError`** será levantada.

Exemplos:

```
>>> x = [1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> y = [1, 2, 3, 1, 2, 3, 1, 2, 3]
>>> covariance(x, y)
0.75
>>> z = [9, 8, 7, 6, 5, 4, 3, 2, 1]
>>> covariance(x, z)
-7.5
>>> covariance(z, x)
-7.5
```

Adicionado na versão 3.10.

statistics.correlation(*x*, *y*, /, *, *method='linear'*)

Retorna o coeficiente de correlação de Pearson [https://en.wikipedia.org/wiki/Pearson_correlation_coefficient] para duas sequências. O coeficiente de correlação de Pearson *r* assume valores entre -1 e +1. Ele mede a força e a direção de uma relação linear.

Se *method* for “ranked”, calcula coeficiente de correlação de classificação de Spearman [https://en.wikipedia.org/wiki/Spearman%27s_rank_correlation_coefficient] para duas entradas. Os dados são substituídos por classificações. Os empates são calculados em média para que valores iguais recebam a mesma

classificação. O coeficiente resultante mede a força de um relacionamento monotônico.

O coeficiente de correlação de Spearman é apropriado para dados ordinais ou para dados contínuos que não atendem ao requisito de proporção linear para o coeficiente de correlação de Pearson.

Ambas as entradas devem ter o mesmo comprimento (não menos que dois) e não precisam ser constantes, caso contrário, **`StatisticsError`** será levantada.

Exemplo com leis de Kepler do movimento planetário [https://en.wikipedia.org/wiki/Kepler's_laws_of_planetary_motion]:

```
>>> # Mercúrio, Venus, Terra, Marte, Jupiter, Saturno, Urano
>>> orbital_period = [88, 225, 365, 687, 4331, 10_756, 30_687
>>> dist_from_sun = [58, 108, 150, 228, 778, 1_400, 2_900, 4_

>>> # Mostra que existe uma perfeita relação monótona
>>> correlation(orbital_period, dist_from_sun, method='ranked
1.0

>>> # Observe que uma relação linear é imperfeita
>>> round(correlation(orbital_period, dist_from_sun), 4)
0.9882

>>> # Demonstra a terceira lei de Kepler: Há uma correlação 1
>>> # entre o quadrado do período orbital e o cubo da
>>> # distância a partir do sol.
>>> period_squared = [p * p for p in orbital_period]
>>> dist_cubed = [d * d * d for d in dist_from_sun]
>>> round(correlation(period_squared, dist_cubed), 4)
1.0
```

Adicionado na versão 3.10.

Alterado na versão 3.12: Adicionado suporte para o coeficiente de correlação de classificação de Spearman.

statistics.linear_regression(*x*, *y*, */*, ***, *proportional=False*)

Retorna a inclinação e a interceptação dos parâmetros de regressão linear simples [https://en.wikipedia.org/wiki/Simple_linear_regression] estimados usando mínimos quadrados ordinários. A regressão linear simples descreve a relação entre uma variável independente *x* e uma variável dependente *y* em termos desta função linear:

$$y = slope * x + intercept + noise$$

onde `slope` e `intercept` são os parâmetros de regressão que são estimados, e `noise` representa a variabilidade dos dados que não foi explicada pela regressão linear (é igual à diferença entre os valores previstos e reais da variável dependente).

Ambas as sequências devem ter o mesmo comprimento (não menos que dois) e a variável independente *x* não pode ser constante; caso contrário, um **`StatisticsError`** será levantada.

Por exemplo, podemos usar as datas de lançamento dos filmes do Monty Python [https://en.wikipedia.org/wiki/Monty_Python#Films] para prever o número cumulativo de filmes do Monty Python que teriam sido produzidos até 2019, supondo que eles tivessem mantido o ritmo.

```
>>> year = [1971, 1975, 1979, 1982, 1983]
>>> films_total = [1, 2, 3, 4, 5]
>>> slope, intercept = linear_regression(year, films_total)
>>> round(slope * 2019 + intercept)
16
```

Se *proportional* for verdadeiro, a variável independente *x* e a variável dependente *y* são assumidas como diretamente proporcionais. Os dados são ajustados a uma linha que passa pela origem. Como o *intercept* sempre será 0,0, a função linear subjacente simplifica para:

$$y = slope * x + noise$$

Continuando o exemplo de **`correlation()`**, vamos ver o quão bem um modelo baseado em planetas principais pode prever as distâncias orbitais de planetas anões:

```
>>> model = linear_regression(period_squared, dist_cubed, pro
>>> slope = model.slope

>>> # Planetas anões:   Pluto,  Eris,    Makemake, Haumea, Ce
>>> orbital_periods = [90_560, 204_199, 111_845, 103_410, 1_6
>>> predicted_dist = [math.cbrt(slope * (p * p)) for p in orb
>>> list(map(round, predicted_dist))
[5912, 10166, 6806, 6459, 414]

>>> [5_906, 10_152, 6_796, 6_450, 414]  # distância real em m
[5906, 10152, 6796, 6450, 414]
```

Adicionado na versão 3.10.

Alterado na versão 3.11: Adicionado suporte a *proportional*.

Exceções

Uma única exceção é definida:

exception statistics.StatisticsError

Subclasse de **`ValueError`** para exceções relacionadas a estatísticas.

Objetos **`NormalDist`**

`NormalDist` é uma ferramenta para criar e manipular distribuições normais de uma variável aleatória [https://pt.wikipedia.org/wiki/Variável_aleatória]. É uma classe que trata a média e o desvio padrão das medições de dados como uma entidade única.

Distribuições normais surgem do Teorema Central do Limite [https://pt.wikipedia.org/wiki/Teorema_central_do_limite] e possuem uma gama de aplicações em estatísticas.

class statistics.NormalDist(*mu=0.0*, *sigma=1.0*)

Retorna um novo objeto *NormalDist* onde *mu* representa a média aritmética [https://pt.wikipedia.org/wiki/Média_aritmética] e *sigma* representa o

desvio padrão [https://pt.wikipedia.org/wiki/Desvio_padrão].

Se *sigma* for negativo, levanta a exceção `StatisticsError`.

mean

Uma propriedade somente leitura para a média aritmética [https://pt.wikipedia.org/wiki/Média_aritmética] de uma distribuição normal.

median

Uma propriedade somente leitura para a mediana [https://pt.wikipedia.org/wiki/Mediana_(estatística)] de uma distribuição normal.

mode

Uma propriedade somente leitura para a moda [https://pt.wikipedia.org/wiki/Moda_(estatística)] de uma distribuição normal.

stdev

Uma propriedade somente leitura para o desvio padrão [https://pt.wikipedia.org/wiki/Desvio_padrão] de uma distribuição normal.

variance

Uma propriedade somente leitura para a variância [https://pt.wikipedia.org/wiki/Variáncia] de uma distribuição normal. Igual ao quadrado do desvio padrão.

classmethod from_samples(*data*)

Faz uma instância da distribuição normal com os parâmetros *mu* e *sigma* estimados a partir de *data* usando `fmean()` e `stdev()`.

data pode ser qualquer iterável e deve consistir de valores que pode ser convertidos para o tipo `float`. Se *data* não contém pelo menos dois elementos, levanta a exceção `StatisticsError` porque é preciso pelo menos um ponto para estimar um valor central e pelo menos dois pontos para estimar a dispersão.

samples(*n*, *, *seed=None*)

Gera *n* amostras aleatórias para uma dada média e desvio padrão. Retorna uma **`list`** de valores **`float`**.

Se o parâmetro *seed* for fornecido, cria uma nova instância do gerador de número aleatório subjacente. Isso é útil para criar resultados reproduzíveis, mesmo em um contexto multithreading.

Alterado na versão 3.13.

Alterado para um algoritmo mais rápido. Para reproduzir amostras de versões anteriores, use **`random.seed()`** e **`random.gauss()`**.

pdf(*x*)

Usando uma função densidade de probabilidade (fdp) [https://pt.wikipedia.org/wiki/Função_densidade], calcula a probabilidade relativa que uma variável aleatória *X* estará perto do valor dado *x*. Matematicamente, é o limite da razão `P(x <= X < x+dx) / dx` quando *dx* se aproxima de zero.

A probabilidade relativa é calculada como a probabilidade de uma amostra ocorrer em um intervalo estreito dividida pela largura do intervalo (daí a palavra "densidade"). Como a probabilidade é relativa a outros pontos, seu valor pode ser maior que `1.0`.

cdf(*x*)

Usando uma função distribuição acumulada (fda) [https://pt.wikipedia.org/wiki/Função_distribuição_acumulada], calcula a probabilidade de que uma variável aleatória *X* seja menor ou igual a *x*. Matematicamente, é representada da seguinte maneira: `P(X <= x)`.

inv_cdf(*p*)

Calcula a função distribuição acumulada inversa, também conhecida como função quantil [https://pt.wikipedia.org/wiki/Quantil] ou o função ponto percentual [https://web.archive.org/web/20190203145224/https://www.statisticshowto.datasciencecent

ral.com/inverse-distribution-function/]. Matematicamente, é representada como `x : P(X <= x) = p`.

Encontra o valor *x* da variável aleatória *X* de tal forma que a probabilidade da variável ser menor ou igual a esse valor seja igual à probabilidade dada *p*.

overlap(*other*)

Mede a concordância entre duas distribuições de probabilidade normais. Retorna um valor entre 0,0 e 1,0 fornecendo a área de sobreposição para as duas funções de densidade de probabilidade [https://www.rasch.org/rmt/rmt101r.htm].

quantiles(*n=4*)

Divide a distribuição normal em *n* intervalos contínuos com probabilidade igual. Retorna uma lista de (n - 1) pontos de corte separando os intervalos.

Defina *n* como 4 para quartis (o padrão). Defina *n* como 10 para decis. Defina *n* como 100 para percentis, o que dá os 99 pontos de corte que separam a distribuição normal em 100 grupos de tamanhos iguais.

zscore(*x*)

Calcula a Pontuação Padrão (z-score) [https://www.statisticshowto.com/probability-and-statistics/z-score/] descrevendo *x* em termos do número de desvios padrão acima ou abaixo da média da distribuição normal: `(x - mean) / stdev`.

Adicionado na versão 3.9.

Instâncias de **`NormalDist`** suportam adição, subtração, multiplicação e divisão por uma constante. Essas operações são usadas para translação e dimensionamento. Por exemplo:

```
>>> temperature_february = NormalDist(5, 2.5)             # C
>>> temperature_february * (9/5) + 32                     # F
NormalDist(mu=41.0, sigma=4.5)
```

A divisão de uma constante por uma instância de `NormalDist` não é suportada porque o resultado não seria distribuído normalmente.

Uma vez que distribuições normais surgem de efeitos aditivos de variáveis independentes, é possível adicionar e subtrair duas variáveis aleatórias independentes normalmente distribuídas [https://en.wikipedia.org/wiki/Sum_of_normally_distributed_random_variables] representadas como instâncias de `NormalDist`. Por exemplo:

```
>>> birth_weights = NormalDist.from_samples([2.5, 3.1, 2.1, 2
>>> drug_effects = NormalDist(0.4, 0.15)
>>> combined = birth_weights + drug_effects
>>> round(combined.mean, 1)
3.1
>>> round(combined.stdev, 1)
0.5
```

Adicionado na versão 3.8.

Exemplos e receitas

Problemas clássicos de probabilidade

`NormalDist` facilmente resolve problemas de probabilidade clássicos.

Por exemplo, considerando os dados históricos para exames SAT [https://nces.ed.gov/programs/digest/d17/tables/dt17_226.40.asp] mostrando que as pontuações são normalmente distribuídas com média de 1060 e desvio padrão de 195, determine o percentual de alunos com pontuações de teste entre 1100 e 1200, após arredondar para o número inteiro mais próximo:

```
>>> sat = NormalDist(1060, 195)
>>> fraction = sat.cdf(1200 + 0.5) - sat.cdf(1100 - 0.5)
>>> round(fraction * 100.0, 1)
18.4
```

Encontrar os quartis [https://pt.wikipedia.org/wiki/Quartil] e decis [https://pt.wikipedia.org/wiki/Decil] para as pontuações SAT:

```
>>> list(map(round, sat.quantiles()))
[928, 1060, 1192]
>>> list(map(round, sat.quantiles(n=10)))
[810, 896, 958, 1011, 1060, 1109, 1162, 1224, 1310]
```

Entradas de Monte Carlo para simulações

Para estimar a distribuição de um modelo que não seja fácil de resolver analiticamente, **`NormalDist`** pode gerar amostras de entrada para uma simulação Monte Carlo [https://pt.wikipedia.org/wiki/Método_de_Monte_Carlo]:

```
>>> def model(x, y, z):
...     return (3*x + 7*x*y - 5*y) / (11 * z)
...
>>> n = 100_000
>>> X = NormalDist(10, 2.5).samples(n, seed=3652260728)
>>> Y = NormalDist(15, 1.75).samples(n, seed=4582495471)
>>> Z = NormalDist(50, 1.25).samples(n, seed=6582483453)
>>> quantiles(map(model, X, Y, Z))
[1.4591308524824727, 1.8035946855390597, 2.175091447274739]
```

Aproximação de distribuições binomiais

As distribuições normais podem ser usadas para aproximar distribuições binomiais [https://mathworld.wolfram.com/BinomialDistribution.html] quando o tamanho da amostra é grande e quando a probabilidade de um teste bem-sucedido é próximo a 50%.

Por exemplo, uma conferência de código aberto tem 750 participantes e duas salas com capacidade para 500 pessoas. Há uma palestra sobre Python e outra sobre Ruby. Em conferências anteriores, 65% dos participantes preferiram ouvir palestras sobre Python. Supondo que as preferências da população não tenham mudado, qual é a probabilidade da sala de Python permanecer dentro de seus limites de capacidade?

```
>>> n = 750              # Tamanho da amostra
>>> p = 0.65             # Preferência para Python
>>> q = 1.0 - p          # Preferência para Ruby
>>> k = 500              # Capacidade

>>> # Aproximação usando a distribuição normal cumulativa
>>> from math import sqrt
>>> round(NormalDist(mu=n*p, sigma=sqrt(n*p*q)).cdf(k + 0.5), 4)
0.8402

>>> # Solução exata usando a distribuição binomial cumulativa
>>> from math import comb, fsum
>>> round(fsum(comb(n, r) * p**r * q**(n-r) for r in range(k+1)))
0.8402

>>> # Aproximação usando uma simulação
>>> from random import seed, binomialvariate
>>> seed(8675309)
>>> mean(binomialvariate(n, p) <= k for i in range(10_000))
0.8406
```

Classificador bayesiano ingênuo

Distribuições normais geralmente surgem em problemas de aprendizado de máquina.

A Wikipedia tem um bom exemplo de um Classificador Bayesiano Ingênuo [https://en.wikipedia.org/wiki/Naive_Bayes_classifier#Person_classification]. O desafio é prever o sexo de uma pessoa a partir de medidas de características normalmente distribuídas, incluindo altura, peso e tamanho do pé.

Recebemos um conjunto de dados de treinamento com medições para oito pessoas. As medidas são consideradas normalmente distribuídas, então resumimos os dados com **`NormalDist`**:

```
>>> height_male = NormalDist.from_samples([6, 5.92, 5.58, 5.92])
>>> height_female = NormalDist.from_samples([5, 5.5, 5.42, 5.75]
>>> weight_male = NormalDist.from_samples([180, 190, 170, 165])
>>> weight_female = NormalDist.from_samples([100, 150, 130, 150]
```

```
>>> foot_size_male = NormalDist.from_samples([12, 11, 12, 10])
>>> foot_size_female = NormalDist.from_samples([6, 8, 7, 9])
```

Em seguida, encontramos uma nova pessoa cujas características de medidas são conhecidas, mas cujo gênero é desconhecido:

```
>>> ht = 6.0        # altura
>>> wt = 130        # peso
>>> fs = 8          # tamanho do pé
```

Começando com uma probabilidade a priori [https://pt.wikipedia.org/wiki/Probabilidade_a_priori] de 50% de ser homem ou mulher, calculamos a posteriori como a priori vezes o produto das probabilidade para as características de medidas dado o gênero:

```
>>> prior_male = 0.5
>>> prior_female = 0.5
>>> posterior_male = (prior_male * height_male.pdf(ht) *
...                   weight_male.pdf(wt) * foot_size_male.pdf(f

>>> posterior_female = (prior_female * height_female.pdf(ht) *
...                     weight_female.pdf(wt) * foot_size_female
```

A previsão final vai para a probabilidade posterior maior. Isso é conhecido como máximo a posteriori [https://en.wikipedia.org/wiki/Maximum_a_posteriori_estimation] ou MAP:

```
>>> 'male' if posterior_male > posterior_female else 'female'
'female'
```

Módulos de Programação Funcional

Os módulos descritos neste capítulo fornecem funções e classes que suportam um estilo de programação funcional e operações gerais em chamáveis.

Os seguintes módulos estão documentados neste capítulo:

- `itertools` — Funções que criam iteradores para laços eficientes
  - Funções do intertools
  - Receitas com itertools
- `functools` — Funções e operações de ordem superior em objetos chamáveis
  - Objetos `partial`
- `operator` — Operadores padrões como funções
  - Mapeando os operadores para suas respectivas funções
  - Operadores in-place

`itertools` — Funções que criam iteradores para laços eficientes

Esse módulo implementa diversos blocos de instruções com iteradores, inspirados por construções de APL, Haskell, e SML. Cada uma foi adequadamente reformulada para Python.

Esse módulo padroniza um conjunto central de ferramentas rápidas e de uso eficiente da memória, que podem ser utilizadas sozinhas ou combinadas. Juntas, eles formam uma "álgebra de iteradores" tornando possível construir ferramentas sucintas e eficientes em Python puro.

Por exemplo, SML fornece uma ferramenta para tabulação: `tabulate(f)` que produz uma sequência `f(0), f(1), ...`. O mesmo efeito pode ser obtido em Python combinando `map()` e `count()` para formar `map(f, count())`.

General iterators:

Iterador	Argumentos	Resultado	Exemplo
`accumulate()`	p [,func]	p0, p0+p1, p0+p1+p2, …	`accumulate([1,2,3,4,5]) → 1 3 6 10 15`
`batched()`	p, n	(p0, p1, …, p_n-1), …	`batched('ABCDEFG', n=3) → ABC DEF G`
`chain()`	p, q, …	p0, p1, … último elemento de p, q0, q1, …	`chain('ABC', 'DEF') → A B C D E F`
`chain.from_iterable()`	iterável	p0, p1, … último elemento de p, q0, q1, …	`chain.from_iterable(['ABC', 'DEF']) → A B C D E F`
`compress()`	data, selectors	(d[0] if s[0]), (d[1] if s[1]), …	`compress('ABCDEF', [1,0,1,0,1,1]) → A C E F`

Iterador	Argumentos	Resultado	Exemplo
`count()`	[start[, step]]	start, start+step, start+2*step, …	`count(10) → 10 11 12 13 14 ...`
`cycle()`	p	p0, p1, … ultimo elemento de p, p0, p1, …	`cycle('ABCD') → A B C D A B C D ...`
`dropwhile()`	predicate, seq	seq[n], seq[n+1], começando quando o predicate falha	`dropwhile(lambda x: x<5, [1,4,6,3,8]) → 6 3 8`
`filterfalse()`	predicate, seq	elementos de seq onde predicate(elem) falha	`filterfalse(lambda x: x<5, [1,4,6,3,8]) → 6 8`
`groupby()`	iterable[, key]	sub-iteradores agrupados pelo valor de key(v)	`groupby(['A','B','DEF'], len) → (1, A B) (3, DEF)`
`islice()`	seq, [start,] stop [, step]	elementos de seq[start:stop:step]	`islice('ABCDEFG', 2, None) → C D E F G`
`pairwise()`	iterável	(p[0], p[1]), (p[1], p[2])	`pairwise('ABCDEFG') → AB BC CD DE EF FG`
`repeat()`	elem [,n]	elem, elem, elem, … repete infinitamente ou até n vezes	`repeat(10, 3) → 10 10 10`
`starmap()`	func, seq	func(*seq[0]), func(*seq[1]), …	`starmap(pow, [(2,5), (3,2), (10,3)]) → 32 9 1000`

Iterador	Argumentos	Resultado	Exemplo
`takewhile()`	predicate, seq	seq[0], seq[1], até predicate falhar	`takewhile(lambda x: x<5, [1,4,6,3,8]) → 1 4`
`tee()`	it, n	n iteradores it independentes	`tee('ABC', 2) → A B C, A B C`
`zip_longest()`	p, q, …	(p[0], q[0]), (p[1], q[1]), …	`zip_longest('ABCD', 'xy', fillvalue='-') → Ax By C- D-`

Iteradores combinatórios:

Iterador	Argumentos	Resultado
`product()`	p, q, … [repeat=1]	produto cartesiano, equivalente a laços `for` aninhados
`permutations()`	p[, r]	tuplas de tamanho r, com todas ordenações possíveis, sem elementos repetidos
`combinations()`	p, r	tuplas de tamanho r, ordenadas, sem elementos repetidos
`combinations_with_replacement()`	p, r	tuplas de tamanho r, ordenadas, com elementos repetidos

Exemplos	Resultado
`product('ABCD', repeat=2)`	`AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD`

Exemplos	Resultado
`permutations('ABCD', 2)`	`AB AC AD BA BC BD CA CB CD DA DB DC`
`combinations('ABCD', 2)`	`AB AC AD BC BD CD`
`combinations_with_replacement('ABCD', 2)`	`AA AB AC AD BB BC BD CC CD DD`

Funções do intertools

Todas as funções a seguir constroem e retornam iteradores. Algumas fornecem fluxos de comprimento infinito, então elas devem ser acessadas somente por funções ou laços que truncam o fluxo.

itertools.accumulate(*iterable*[, *function*, *, *initial=None*])

Constrói um iterador que retorna somas acumuladas, ou resultados acumulados de outras funções binárias.

A função *function* assume como padrão a adição. *function* deve aceitar dois argumentos, um total acumulado e um valor de *iterable*.

Se um valor inicial *initial* for fornecido, o acúmulo começará com esse valor e a saída terá um elemento a mais que o iterável de entrada.

Aproximadamente equivalente a:

```
def accumulate(iterable, function=operator.add, *, initial=None):
    'Retorna totais atuais'
    # accumulate([1,2,3,4,5]) → 1 3 6 10 15
    # accumulate([1,2,3,4,5], initial=100) → 100 101 103 106 110 115
    # accumulate([1,2,3,4,5], operator.mul) → 1 2 6 24 120

    iterator = iter(iterable)
    total = initial
    if initial is None:
        try:
            total = next(iterator)
        except StopIteration:
            return

    yield total
    for element in iterator:
```

```
        total = function(total, element)
        yield total
```

Para calcular um mínimo atual, defina *function* como `min()`. Para um máximo atual, defina *function* como `max()`. Ou para um produto atual, defina *function* como `operator.mul()`. Para construir uma tabela de amortização [https://www.ramseysolutions.com/real-estate/amortization-schedule], acumular os juros e aplicar os pagamentos:

```
>>> data = [3, 4, 6, 2, 1, 9, 0, 7, 5, 8]
>>> list(accumulate(data, max))              # máximo atual
[3, 4, 6, 6, 6, 9, 9, 9, 9, 9]
>>> list(accumulate(data, operator.mul))     # produto atual
[3, 12, 72, 144, 144, 1296, 0, 0, 0, 0]

# Amortiza um empréstimo de 5% de 1000 com 10 pagamentos anuais de 90
>>> update = lambda balance, payment: round(balance * 1.05) - payment
>>> list(accumulate(repeat(90, 10), update, initial=1_000))
[1000, 960, 918, 874, 828, 779, 728, 674, 618, 559, 497]
```

Veja `functools.reduce()` para uma função similar que devolve apenas o valor acumulado final.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionado o parâmetro opcional *function*.

Alterado na versão 3.8: Adicionado o parâmetro opcional *initial*.

itertools.batched(*iterable*, *n*, ***, *strict=False*)

Dados de lote do *iterable* em tuplas de comprimento *n*. O último lote pode ser menor que *n*.

Se *strict* for verdadeiro, será levantada uma `ValueError` se o lote final for menor que *n*.

Loops over the input iterable and accumulates data into tuples up to size *n*. The input is consumed lazily, just enough to fill a batch. The result is yielded as soon as the batch is full or when the input iterable is exhausted:

```
>>> flattened_data = ['roses', 'red', 'violets', 'blue', 'sugar', 'sweet']
>>> unflattened = list(batched(flattened_data, 2))
>>> unflattened
[('roses', 'red'), ('violets', 'blue'), ('sugar', 'sweet')]
```

Aproximadamente equivalente a:

```
def batched(iterable, n, *, strict=False):
    # batched('ABCDEFG', 3) → ABC DEF G
    if n < 1:
        raise ValueError('n must be at least one')
    iterator = iter(iterable)
    while batch := tuple(islice(iterator, n)):
        if strict and len(batch) != n:
```

```
            raise ValueError('batched(): incomplete batch')
        yield batch
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionada a opção *strict*.

itertools.chain(**iterables*)

Make an iterator that returns elements from the first iterable until it is exhausted, then proceeds to the next iterable, until all of the iterables are exhausted. This combines multiple data sources into a single iterator. Roughly equivalent to:

```
def chain(*iterables):
    # chain('ABC', 'DEF') → A B C D E F
    for iterable in iterables:
        yield from iterable
```

Note that unpacking in comprehensions provides similar functionality so that `list(chain(p, q))` could be written as `[*s for s in (p, q)]`.

classmethod chain.from_iterable(*iterable*)

Construtor alternativo para **`chain()`**. Obtém entradas encadeadas a partir de um único argumento iterável que avaliado preguiçosamente. Aproximadamente equivalente a:

```
def from_iterable(iterables):
    # chain.from_iterable(['ABC', 'DEF']) → A B C D E F
    for iterable in iterables:
        yield from iterable
```

Note that unpacking in comprehensions provides similar functionality so that `list(chain.from_iterable(iterables))` could be written as `[*s for s in iterables]`.

itertools.combinations(*iterable*, *r*)

Devolve subsequências de elementos com comprimento *r* a partir da entrada *iterável*

A saída é uma subsequência de **`product()`** mantendo apenas entradas que são subsequências do *iterable*. O comprimento da saída é dado por **`math.comb()`** que calcula `n! / r! / (n - r)!` quando `0 ≤ r ≤ n` ou zero quando `r > n`.

As tuplas das combinações são emitidas em ordem lexicográfica de acordo com a ordem do iterável de entrada. Se o iterável estiver ordenado, as tuplas de combinação serão produzidas em sequência ordenada.

Os elementos são tratados como únicos baseado em suas posições, não em seus valores. Se os elementos de entrada são únicos, não haverá repetição de valores nas sucessivas combinações.

Aproximadamente equivalente a:

```
def combinations(iterable, r):
    # combinations('ABCD', 2) → AB AC AD BC BD CD
    # combinations(range(4), 3) → 012 013 023 123

    pool = tuple(iterable)
    n = len(pool)
    if r > n:
        return
    indices = list(range(r))

    yield tuple(pool[i] for i in indices)
    while True:
        for i in reversed(range(r)):
            if indices[i] != i + n - r:
                break
        else:
            return
        indices[i] += 1
        for j in range(i+1, r):
            indices[j] = indices[j-1] + 1
        yield tuple(pool[i] for i in indices)
```

itertools.combinations_with_replacement(*iterable*, *r*)

Devolve subsequências de comprimento *r* de elementos do *iterável* de entrada permitindo que elementos individuais sejam repetidos mais de uma vez.

A saída é uma subsequência de **`product()`** que mantém apenas entradas que são subsequências (com possíveis elementos repetidos) do *iterable*. O número de subsequências retornadas é `(n + r - 1)! / r! / (n - 1)!` quando `n > 0`.

As tuplas das combinações são emitidas em ordem lexicográfica de acordo com a ordem do iterável de entrada. Se o iterável estiver ordenado, as tuplas de combinação serão produzidas em sequência ordenada.

Os elementos são tratados como únicos baseado em suas posições, não em seus valores. Se os elementos de entrada forem únicos, não haverá repetição de valores nas combinações geradas.

Aproximadamente equivalente a:

```
def combinations_with_replacement(iterable, r):
    # combinations_with_replacement('ABC', 2) → AA AB AC BB BC CC

    pool = tuple(iterable)
    n = len(pool)
    if not n and r:
        return
    indices = [0] * r

    yield tuple(pool[i] for i in indices)
    while True:
```

```
        for i in reversed(range(r)):
            if indices[i] != n - 1:
                break
        else:
            return
        indices[i:] = [indices[i] + 1] * (r - i)
        yield tuple(pool[i] for i in indices)
```

Adicionado na versão 3.1.

itertools.compress(*data*, *selectors*)

Make an iterator that returns elements from *data* where the corresponding element in *selectors* is true. Stops when either the *data* or *selectors* iterables have been exhausted. Roughly equivalent to:

```
def compress(data, selectors):
    # compress('ABCDEF', [1,0,1,0,1,1]) → A C E F
    return (datum for datum, selector in zip(data, selectors) if selector)
```

Adicionado na versão 3.1.

itertools.count(*start=0*, *step=1*)

Cria um iterador que retorne valores uniformemente espaçados começando com *start*. Pode ser usado com **`map()`** para gerar pontos de dados consecutivos ou com **`zip()`** para adicionar números de sequência. Aproximadamente equivalente a:

```
def count(start=0, step=1):
    # count(10) → 10 11 12 13 14 ...
    # count(2.5, 0.5) → 2.5 3.0 3.5 ...
    n = start
    while True:
        yield n
        n += step
```

Quando é feita uma contagem usando números de ponto flutuante, é possível ter melhor precisão substituindo código multiplicativo como `(start + step * i for i in count())`.

Alterado na versão 3.1: Adicionou argumento *step* e permitiu argumentos não-inteiros.

itertools.cycle(*iterable*)

Make an iterator returning elements from the *iterable* and saving a copy of each. When the iterable is exhausted, return elements from the saved copy. Repeats indefinitely. Roughly equivalent to:

```
def cycle(iterable):
    # cycle('ABCD') → A B C D A B C D A B C D ...

    saved = []
    for element in iterable:
```

```
        yield element
        saved.append(element)

    while saved:
        for element in saved:
            yield element
```

Esta itertool pode exigir armazenamento auxiliar significativo (dependendo do comprimento do iterável).

itertools.dropwhile(*predicate*, *iterable*)

Cria um iterador que remove elementos do *iterable* enquanto o *predicate* for verdadeiro e, em seguida, retorna todos os elementos. Equivalente a:

```
def dropwhile(predicate, iterable):
    # dropwhile(lambda x: x<5, [1,4,6,3,8]) → 6 3 8

    iterator = iter(iterable)
    for x in iterator:
        if not predicate(x):
            yield x
            break

    for x in iterator:
        yield x
```

Observe que isso não produz *nenhuma* saída até que o predicado se torne falso, portanto, esta itertool pode ter um longo tempo de inicialização.

itertools.filterfalse(*predicate*, *iterable*)

Cria um iterador que filtre elementos do *iterable*, retornando apenas aqueles para os quais o *predicate* retorna um valor falso. Se o *predicate* for `None`, retorna os itens que são falsos. Equivalente aproximadamente a:

```
def filterfalse(predicate, iterable):
    # filterfalse(lambda x: x<5, [1,4,6,3,8]) → 6 8

    if predicate is None:
        predicate = bool

    for x in iterable:
        if not predicate(x):
            yield x
```

itertools.groupby(*iterable*, *key=None*)

Cria um iterador que retorna chaves e grupos consecutivos do *iterable*. A *key* é uma função que calcula um valor-chave para cada elemento. Se não for especificado ou for `None`, *key* assume como padrão uma função de identidade e retorna o elemento inalterado. Geralmente, o iterável precisa já estar ordenado na mesma função chave.

O funcionamento de **groupby()** é semelhante ao filtro `uniq` no Unix. Ele gera uma quebra ou um novo grupo sempre que o valor da função da tecla muda (razão pela qual geralmente é necessário ordenar os dados usando a mesma função da tecla). Esse comportamento difere do GROUP BY do SQL, que agrega elementos comuns independentemente da ordem de entrada.

O grupo retornado é, ele próprio, um iterador que compartilha o iterável subjacente com **groupby()**. Como a origem é compartilhada, quando o objeto **groupby()** avança, o grupo anterior não fica mais visível. Portanto, se esses dados forem necessários posteriormente, eles devem ser armazenados como uma lista:

```
groups = []
uniquekeys = []
data = sorted(data, key=keyfunc)
for k, g in groupby(data, keyfunc):
    groups.append(list(g))      # armazena um iterador de grupo como uma list
    uniquekeys.append(k)
```

groupby() é aproximadamente equivalente a:

```
def groupby(iterable, key=None):
    # [k for k, g in groupby('AAAABBBCCDAABBB')] → A B C D A B
    # [list(g) for k, g in groupby('AAAABBBCCD')] → AAAA BBB CC D

    keyfunc = (lambda x: x) if key is None else key
    iterator = iter(iterable)
    exhausted = False

    def _grouper(target_key):
        nonlocal curr_value, curr_key, exhausted
        yield curr_value
        for curr_value in iterator:
            curr_key = keyfunc(curr_value)
            if curr_key != target_key:
                return
            yield curr_value
        exhausted = True

    try:
        curr_value = next(iterator)
    except StopIteration:
        return
    curr_key = keyfunc(curr_value)

    while not exhausted:
        target_key = curr_key
        curr_group = _grouper(target_key)
        yield curr_key, curr_group
        if curr_key == target_key:
            for _ in curr_group:
                pass
```

itertools.islice(*iterable*, *stop*)

itertools.islice(*iterable*, *start*, *stop*[, *step*])

Cria um iterador que retorna os elementos selecionados do iterável. Funciona como o fatiamento de sequência, mas não oferece suporte a valores negativos para *start*, *stop* ou *step*.

Se *start* for zero ou `None`, a iteração começa em zero. Caso contrário, os elementos do iterável são pulados até que *start* seja alcançado.

If *stop* is `None`, iteration continues until the input is exhausted, if at all. Otherwise, it stops at the specified position.

Se *step* for `None`, o padrão do passo é 1. Os elementos são retornados consecutivamente, a menos que *step* seja definido como maior que um, o que resulta na omissão de itens.

Aproximadamente equivalente a:

```
def islice(iterable, *args):
    # islice('ABCDEFG', 2) → A B
    # islice('ABCDEFG', 2, 4) → C D
    # islice('ABCDEFG', 2, None) → C D E F G
    # islice('ABCDEFG', 0, None, 2) → A C E G

    s = slice(*args)
    start = 0 if s.start is None else s.start
    stop = s.stop
    step = 1 if s.step is None else s.step
    if start < 0 or (stop is not None and stop < 0) or step <= 0:
        raise ValueError

    indices = count() if stop is None else range(max(start, stop))
    next_i = start
    for i, element in zip(indices, iterable):
        if i == next_i:
            yield element
            next_i += step
```

Se a entrada for um iterador, o consumo total do *islice* avança o iterador de entrada em `max(start, stop)` passos, independentemente do valor do *step*.

itertools.pairwise(*iterable*)

Return successive overlapping pairs taken from the input *iterable*.

The number of 2-tuples in the output iterator will be one fewer than the number of inputs. It will be empty if the input iterable has fewer than two values.

Aproximadamente equivalente a:

```
def pairwise(iterable):
    # pairwise('ABCDEFG') → AB BC CD DE EF FG
```

```
    iterator = iter(iterable)
    a = next(iterator, None)

    for b in iterator:
        yield a, b
        a = b
```

Adicionado na versão 3.10.

itertools.permutations(*iterable*, *r=None*)

Return successive *r* length permutations of elements [https://www.britannica.com/science/permutation] from the *iterable*.

If *r* is not specified or is `None`, then *r* defaults to the length of the *iterable* and all possible full-length permutations are generated.

The output is a subsequence of **`product()`** where entries with repeated elements have been filtered out. The length of the output is given by **`math.perm()`** which computes `n! / (n - r)!` when `0 ≤ r ≤ n` or zero when `r > n`.

The permutation tuples are emitted in lexicographic order according to the order of the input *iterable*. If the input *iterable* is sorted, the output tuples will be produced in sorted order.

Elements are treated as unique based on their position, not on their value. If the input elements are unique, there will be no repeated values within a permutation.

Aproximadamente equivalente a:

```
def permutations(iterable, r=None):
    # permutations('ABCD', 2) → AB AC AD BA BC BD CA CB CD DA DB DC
    # permutations(range(3)) → 012 021 102 120 201 210

    pool = tuple(iterable)
    n = len(pool)
    r = n if r is None else r
    if r > n:
        return

    indices = list(range(n))
    cycles = list(range(n, n-r, -1))
    yield tuple(pool[i] for i in indices[:r])

    while n:
        for i in reversed(range(r)):
            cycles[i] -= 1
            if cycles[i] == 0:
                indices[i:] = indices[i+1:] + indices[i:i+1]
                cycles[i] = n - i
            else:
```

```
                j = cycles[i]
                indices[i], indices[-j] = indices[-j], indices[i]
                yield tuple(pool[i] for i in indices[:r])
                break
        else:
            return
```

itertools.product(**iterables*, *repeat=1*)

Cartesian product [https://en.wikipedia.org/wiki/Cartesian_product] of the input iterables.

Aproximadamente equivalente a laços for aninhados em uma expressão geradora. Por exemplo, `product(A, B)` devolve o mesmo que `((x,y) for x in A for y in B)`.

Os laços aninhados circulam como um hodômetro com o elemento mais à direita avançando a cada iteração. Este padrão cria uma ordenação lexicográfica de maneira que se os iteráveis de entrada estiverem ordenados, as tuplas produzidas são emitidas de maneira ordenada.

To compute the product of an iterable with itself, specify the number of repetitions with the optional *repeat* keyword argument. For example, `product(A, repeat=4)` means the same as `product(A, A, A, A)`.

This function is roughly equivalent to the following code, except that the actual implementation does not build up intermediate results in memory:

```
def product(*iterables, repeat=1):
    # product('ABCD', 'xy') → Ax Ay Bx By Cx Cy Dx Dy
    # product(range(2), repeat=3) → 000 001 010 011 100 101 110 111

    if repeat < 0:
        raise ValueError('repeat argument cannot be negative')
    pools = [tuple(pool) for pool in iterables] * repeat

    result = [[]]
    for pool in pools:
        result = [x+[y] for x in result for y in pool]

    for prod in result:
        yield tuple(prod)
```

Before **`product()`** runs, it completely consumes the input iterables, keeping pools of values in memory to generate the products. Accordingly, it is only useful with finite inputs.

itertools.repeat(*object*[, *times*])

Make an iterator that returns *object* over and over again. Runs indefinitely unless the *times* argument is specified.

Aproximadamente equivalente a:

```
def repeat(object, times=None):
    # repeat(10, 3) → 10 10 10
    if times is None:
        while True:
            yield object
    else:
        for i in range(times):
            yield object
```

Um uso comum de *repeat* é para fornecer um fluxo de valores constantes para *map* ou *zip*.

```
>>> list(map(pow, range(10), repeat(2)))
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

itertools.starmap(*function*, *iterable*)

Make an iterator that computes the *function* using arguments obtained from the *iterable*. Used instead of **`map()`** when argument parameters have already been "pre-zipped" into tuples.

The difference between **`map()`** and **`starmap()`** parallels the distinction between `function(a,b)` and `function(*c)`. Roughly equivalent to:

```
def starmap(function, iterable):
    # starmap(pow, [(2,5), (3,2), (10,3)]) → 32 9 1000
    for args in iterable:
        yield function(*args)
```

itertools.takewhile(*predicate*, *iterable*)

Make an iterator that returns elements from the *iterable* as long as the *predicate* is true. Roughly equivalent to:

```
def takewhile(predicate, iterable):
    # takewhile(lambda x: x<5, [1,4,6,3,8]) → 1 4
    for x in iterable:
        if not predicate(x):
            break
        yield x
```

Note, the element that first fails the predicate condition is consumed from the input iterator and there is no way to access it. This could be an issue if an application wants to further consume the input iterator after *takewhile* has been run to exhaustion. To work around this problem, consider using more-itertools before_and_after() [https://more-itertools.readthedocs.io/en/stable/api.html#more_itertools.before_and_after] instead.

itertools.tee(*iterable*, *n=2*)

Return *n* independent iterators from a single iterable.

Aproximadamente equivalente a:

```
def tee(iterable, n=2):
    if n < 0:
        raise ValueError
    if n == 0:
        return ()
    iterator = _tee(iterable)
    result = [iterator]
    for _ in range(n - 1):
        result.append(_tee(iterator))
    return tuple(result)

class _tee:

    def __init__(self, iterable):
        it = iter(iterable)
        if isinstance(it, _tee):
            self.iterator = it.iterator
            self.link = it.link
        else:
            self.iterator = it
            self.link = [None, None]

    def __iter__(self):
        return self

    def __next__(self):
        link = self.link
        if link[1] is None:
            link[0] = next(self.iterator)
            link[1] = [None, None]
        value, self.link = link
        return value
```

When the input *iterable* is already a tee iterator object, all members of the return tuple are constructed as if they had been produced by the upstream **`tee()`** call. This “flattening step” allows nested **`tee()`** calls to share the same underlying data chain and to have a single update step rather than a chain of calls.

The flattening property makes tee iterators efficiently peekable:

```
def lookahead(tee_iterator):
     "Return the next value without moving the input forward"
     [forked_iterator] = tee(tee_iterator, 1)
     return next(forked_iterator)

>>> iterator = iter('abcdef')
>>> [iterator] = tee(iterator, 1)   # Make the input peekable
>>> next(iterator)                  # Move the iterator forward
'a'
>>> lookahead(iterator)             # Check next value
'b'
>>> next(iterator)                  # Continue moving forward
'b'
```

`tee` iterators are not threadsafe. A **`RuntimeError`** may be raised when simultaneously using iterators returned by the same **`tee()`** call, even if the original *iterable* is threadsafe.

This itertool may require significant auxiliary storage (depending on how much temporary data needs to be stored). In general, if one iterator uses most or all of the data before another iterator starts, it is faster to use **`list()`** instead of **`tee()`**.

itertools.zip_longest(**iterables*, *fillvalue=None*)

Make an iterator that aggregates elements from each of the *iterables*.

If the iterables are of uneven length, missing values are filled-in with *fillvalue*. If not specified, *fillvalue* defaults to `None`.

Iteration continues until the longest iterable is exhausted.

Aproximadamente equivalente a:

```
def zip_longest(*iterables, fillvalue=None):
    # zip_longest('ABCD', 'xy', fillvalue='-') → Ax By C- D-

    iterators = list(map(iter, iterables))
    num_active = len(iterators)
    if not num_active:
        return

    while True:
        values = []
        for i, iterator in enumerate(iterators):
            try:
                value = next(iterator)
            except StopIteration:
                num_active -= 1
                if not num_active:
                    return
                iterators[i] = repeat(fillvalue)
                value = fillvalue
            values.append(value)
        yield tuple(values)
```

If one of the iterables is potentially infinite, then the **`zip_longest()`** function should be wrapped with something that limits the number of calls (for example **`islice()`** or **`takewhile()`**).

Receitas com itertools

Esta seção mostra receitas para criação de um ferramental ampliado usando as ferramentas existentes de itertools como elementos construtivos.

The primary purpose of the itertools recipes is educational. The recipes show various ways of thinking about individual tools — for example, that `chain.from_iterable` is related to the

concept of flattening. The recipes also give ideas about ways that the tools can be combined — for example, how `starmap()` and `repeat()` can work together. The recipes also show patterns for using itertools with the **`operator`** and **`collections`** modules as well as with the built-in itertools such as `map()`, `filter()`, `reversed()`, and `enumerate()`.

A secondary purpose of the recipes is to serve as an incubator. The `accumulate()`, `compress()`, and `pairwise()` itertools started out as recipes. Currently, the `sliding_window()`, `derangements()`, and `sieve()` recipes are being tested to see whether they prove their worth.

Substantially all of these recipes and many, many others can be installed from the more-itertools [https://pypi.org/project/more-itertools/] project found on the Python Package Index:

```
python -m pip install more-itertools
```

Many of the recipes offer the same high performance as the underlying toolset. Superior memory performance is kept by processing elements one at a time rather than bringing the whole iterable into memory all at once. Code volume is kept small by linking the tools together in a functional style [https://www.cs.kent.ac.uk/people/staff/dat/miranda/whyfp90.pdf]. High speed is retained by preferring "vectorized" building blocks over the use of for-loops and generators which incur interpreter overhead.

```
from itertools import (accumulate, batched, chain, combinations, compress,
     count, cycle, filterfalse, groupby, islice, permutations, product,
     repeat, starmap, tee, zip_longest)
from collections import Counter, deque
from contextlib import suppress
from functools import reduce
from heapq import heappush, heappushpop, heappush_max, heappushpop_max
from math import comb, isqrt, prod, sumprod
from operator import getitem, is_not, itemgetter, mul, neg, truediv


# ==== Basic one liners ====

def take(n, iterable):
    "Return first n items of the iterable as a list."
    return list(islice(iterable, n))

def prepend(value, iterable):
    "Prepend a single value in front of an iterable."
    # prepend(1, [2, 3, 4]) → 1 2 3 4
    return chain([value], iterable)

def repeatfunc(function, times=None, *args):
    "Repeat calls to a function with specified arguments."
    if times is None:
        return starmap(function, repeat(args))
    return starmap(function, repeat(args, times))

def flatten(list_of_lists):
    "Flatten one level of nesting."
```

```
    return chain.from_iterable(list_of_lists)

def ncycles(iterable, n):
    "Returns the sequence elements n times."
    return chain.from_iterable(repeat(tuple(iterable), n))

def loops(n):
    "Loop n times. Like range(n) but without creating integers."
    # for _ in loops(100): ...
    return repeat(None, n)

def tail(n, iterable):
    "Return an iterator over the last n items."
    # tail(3, 'ABCDEFG') → E F G
    return iter(deque(iterable, maxlen=n))

def consume(iterator, n=None):
    "Advance the iterator n-steps ahead. If n is None, consume entirely."
    # Use functions that consume iterators at C speed.
    if n is None:
        deque(iterator, maxlen=0)
    else:
        next(islice(iterator, n, n), None)

def nth(iterable, n, default=None):
    "Returns the nth item or a default value."
    return next(islice(iterable, n, None), default)

def quantify(iterable, predicate=bool):
    "Given a predicate that returns True or False, count the True results."
    return sum(map(predicate, iterable))

def first_true(iterable, default=False, predicate=None):
    "Returns the first true value or the *default* if there is no true value."
    # first_true([a, b, c], x) → a or b or c or x
    # first_true([a, b], x, f) → a if f(a) else b if f(b) else x
    return next(filter(predicate, iterable), default)

def all_equal(iterable, key=None):
    "Returns True if all the elements are equal to each other."
    # all_equal('4ε□□¿', key=int) → True
    return len(take(2, groupby(iterable, key))) <= 1


# ==== Data pipelines ====

def unique_justseen(iterable, key=None):
    "Yield unique elements, preserving order. Remember only the element just see
    # unique_justseen('AAAABBBCCDAABBB') → A B C D A B
    # unique_justseen('ABBcCAD', str.casefold) → A B c A D
    if key is None:
        return map(itemgetter(0), groupby(iterable))
    return map(next, map(itemgetter(1), groupby(iterable, key)))
```

```
def unique_everseen(iterable, key=None):
    "Yield unique elements, preserving order. Remember all elements ever seen."
    # unique_everseen('AAAABBBCCDAABBB') → A B C D
    # unique_everseen('ABBcCAD', str.casefold) → A B c D
    seen = set()
    if key is None:
        for element in filterfalse(seen.__contains__, iterable):
            seen.add(element)
            yield element
    else:
        for element in iterable:
            k = key(element)
            if k not in seen:
                seen.add(k)
                yield element

def unique(iterable, key=None, reverse=False):
    "Yield unique elements in sorted order. Supports unhashable inputs."
    # unique([[1, 2], [3, 4], [1, 2]]) → [1, 2] [3, 4]
    sequenced = sorted(iterable, key=key, reverse=reverse)
    return unique_justseen(sequenced, key=key)

def sliding_window(iterable, n):
    "Collect data into overlapping fixed-length chunks or blocks."
    # sliding_window('ABCDEFG', 3) → ABC BCD CDE DEF EFG
    iterator = iter(iterable)
    window = deque(islice(iterator, n - 1), maxlen=n)
    for x in iterator:
        window.append(x)
        yield tuple(window)

def grouper(iterable, n, *, incomplete='fill', fillvalue=None):
    "Collect data into non-overlapping fixed-length chunks or blocks."
    # grouper('ABCDEFG', 3, fillvalue='x')       → ABC DEF Gxx
    # grouper('ABCDEFG', 3, incomplete='strict') → ABC DEF ValueError
    # grouper('ABCDEFG', 3, incomplete='ignore') → ABC DEF
    iterators = [iter(iterable)] * n
    match incomplete:
        case 'fill':
            return zip_longest(*iterators, fillvalue=fillvalue)
        case 'strict':
            return zip(*iterators, strict=True)
        case 'ignore':
            return zip(*iterators)
        case _:
            raise ValueError('Expected fill, strict, or ignore')

def roundrobin(*iterables):
    "Visit input iterables in a cycle until each is exhausted."
    # roundrobin('ABC', 'D', 'EF') → A D E B F C
    # Algorithm credited to George Sakkis
    iterators = map(iter, iterables)
```

```
    for num_active in range(len(iterables), 0, -1):
        iterators = cycle(islice(iterators, num_active))
        yield from map(next, iterators)

def subslices(seq):
    "Return all contiguous non-empty subslices of a sequence."
    # subslices('ABCD') → A AB ABC ABCD B BC BCD C CD D
    slices = starmap(slice, combinations(range(len(seq) + 1), 2))
    return map(getitem, repeat(seq), slices)

def derangements(iterable, r=None):
    "Produce r length permutations without fixed points."
    # derangements('ABCD') → BADC BCDA BDAC CADB CDAB CDBA DABC DCAB DCBA
    # Algorithm credited to Stefan Pochmann
    seq = tuple(iterable)
    pos = tuple(range(len(seq)))
    have_moved = map(map, repeat(is_not), repeat(pos), permutations(pos, r=r))
    valid_derangements = map(all, have_moved)
    return compress(permutations(seq, r=r), valid_derangements)

def iter_index(iterable, value, start=0, stop=None):
    "Return indices where a value occurs in a sequence or iterable."
    # iter_index('AABCADEAF', 'A') → 0 1 4 7
    seq_index = getattr(iterable, 'index', None)
    if seq_index is None:
        iterator = islice(iterable, start, stop)
        for i, element in enumerate(iterator, start):
            if element is value or element == value:
                yield i
    else:
        stop = len(iterable) if stop is None else stop
        i = start
        with suppress(ValueError):
            while True:
                yield (i := seq_index(value, i, stop))
                i += 1

def iter_except(function, exception, first=None):
    "Convert a call-until-exception interface to an iterator interface."
    # iter_except(d.popitem, KeyError) → non-blocking dictionary iterator
    with suppress(exception):
        if first is not None:
            yield first()
        while True:
            yield function()


# ==== Mathematical operations ====

def multinomial(*counts):
    "Number of distinct arrangements of a multiset."
    # Counter('abracadabra').values() → 5 2 2 1 1
    # multinomial(5, 2, 2, 1, 1) → 83160
```

```
    return prod(map(comb, accumulate(counts), counts))

def powerset(iterable):
    "Subsequences of the iterable from shortest to longest."
    # powerset([1,2,3]) → () (1,) (2,) (3,) (1,2) (1,3) (2,3) (1,2,3)
    s = list(iterable)
    return chain.from_iterable(combinations(s, r) for r in range(len(s)+1))

def sum_of_squares(iterable):
    "Add up the squares of the input values."
    # sum_of_squares([10, 20, 30]) → 1400
    return sumprod(*tee(iterable))


# ==== Matrix operations ====

def reshape(matrix, columns):
    "Reshape a 2-D matrix to have a given number of columns."
    # reshape([(0, 1), (2, 3), (4, 5)], 3) →  (0, 1, 2) (3, 4, 5)
    return batched(chain.from_iterable(matrix), columns, strict=True)

def transpose(matrix):
    "Swap the rows and columns of a 2-D matrix."
    # transpose([(1, 2, 3), (11, 22, 33)]) → (1, 11) (2, 22) (3, 33)
    return zip(*matrix, strict=True)

def matmul(m1, m2):
    "Multiply two matrices."
    # matmul([(7, 5), (3, 5)], [(2, 5), (7, 9)]) → (49, 80) (41, 60)
    n = len(m2[0])
    return batched(starmap(sumprod, product(m1, transpose(m2))), n)


# ==== Polynomial arithmetic ====

def convolve(signal, kernel):
    """Discrete linear convolution of two iterables.
    Equivalent to polynomial multiplication.

    Convolutions are mathematically commutative; however, the inputs are
    evaluated differently.  The signal is consumed lazily and can be
    infinite. The kernel is fully consumed before the calculations begin.

    Article:  https://betterexplained.com/articles/intuitive-convolution/
    Video:    https://www.youtube.com/watch?v=KuXjwB4LzSA
    """
    # convolve([1, -1, -20], [1, -3]) → 1 -4 -17 60
    # convolve(data, [0.25, 0.25, 0.25, 0.25]) → Moving average (blur)
    # convolve(data, [1/2, 0, -1/2]) → 1st derivative estimate
    # convolve(data, [1, -2, 1]) → 2nd derivative estimate
    kernel = tuple(kernel)[::-1]
    n = len(kernel)
    padded_signal = chain(repeat(0, n-1), signal, repeat(0, n-1))
```

```
    windowed_signal = sliding_window(padded_signal, n)
    return map(sumprod, repeat(kernel), windowed_signal)

def polynomial_from_roots(roots):
    """Compute a polynomial's coefficients from its roots.

       (x - 5) (x + 4) (x - 3)  expands to:   x³ -4x² -17x + 60
    """
    # polynomial_from_roots([5, -4, 3]) → [1, -4, -17, 60]
    factors = zip(repeat(1), map(neg, roots))
    return list(reduce(convolve, factors, [1]))

def polynomial_eval(coefficients, x):
    """Evaluate a polynomial at a specific value.

    Computes with better numeric stability than Horner's method.
    """
    # Evaluate x³ -4x² -17x + 60 at x = 5
    # polynomial_eval([1, -4, -17, 60], x=5) → 0
    n = len(coefficients)
    if not n:
        return type(x)(0)
    powers = map(pow, repeat(x), reversed(range(n)))
    return sumprod(coefficients, powers)

def polynomial_derivative(coefficients):
    """Compute the first derivative of a polynomial.

       f(x)  =  x³ -4x² -17x + 60
       f'(x) = 3x² -8x  -17
    """
    # polynomial_derivative([1, -4, -17, 60]) → [3, -8, -17]
    n = len(coefficients)
    powers = reversed(range(1, n))
    return list(map(mul, coefficients, powers))


# ==== Number theory ====

def sieve(n):
    "Primes less than n."
    # sieve(30) → 2 3 5 7 11 13 17 19 23 29
    if n > 2:
        yield 2
    data = bytearray((0, 1)) * (n // 2)
    for p in iter_index(data, 1, start=3, stop=isqrt(n) + 1):
        data[p*p : n : p+p] = bytes(len(range(p*p, n, p+p)))
    yield from iter_index(data, 1, start=3)

def factor(n):
    "Prime factors of n."
    # factor(99) → 3 3 11
    # factor(1_000_000_000_000_007) → 47 59 360620266859
```

```
    # factor(1_000_000_000_000_403) → 1000000000000403
    for prime in sieve(isqrt(n) + 1):
        while not n % prime:
            yield prime
            n //= prime
            if n == 1:
                return
    if n > 1:
        yield n

def is_prime(n):
    "Return True if n is prime."
    # is_prime(1_000_000_000_000_403) → True
    return n > 1 and next(factor(n)) == n

def totient(n):
    "Count of natural numbers up to n that are coprime to n."
    # https://mathworld.wolfram.com/TotientFunction.html
    # totient(12) → 4 because len([1, 5, 7, 11]) == 4
    for prime in set(factor(n)):
        n -= n // prime
    return n


# ==== Running statistics ====

def running_mean(iterable):
    "Average of values seen so far."
    # running_mean([37, 33, 38, 28]) → 37 35 36 34
    return map(truediv, accumulate(iterable), count(1))

def running_min(iterable):
    "Smallest of values seen so far."
    # running_min([37, 33, 38, 28]) → 37 33 33 28
    return accumulate(iterable, func=min)

def running_max(iterable):
    "Largest of values seen so far."
    # running_max([37, 33, 38, 28]) → 37 37 38 38
    return accumulate(iterable, func=max)

def running_median(iterable):
    "Median of values seen so far."
    # running_median([37, 33, 38, 28]) → 37 35 37 35
    read = iter(iterable).__next__
    lo = []  # max-heap
    hi = []  # min-heap the same size as or one smaller than lo
    with suppress(StopIteration):
        while True:
            heappush_max(lo, heappushpop(hi, read()))
            yield lo[0]
            heappush(hi, heappushpop_max(lo, read()))
            yield (lo[0] + hi[0]) / 2
```

```
def running_statistics(iterable):
    "Aggregate statistics for values seen so far."
    # Generate tuples:  (size, minimum, median, maximum, mean)
    t0, t1, t2, t3 = tee(iterable, 4)
    return zip(
        count(1),
        running_min(t0),
        running_median(t1),
        running_max(t2),
        running_mean(t3),
    )
```

`functools` — Funções e operações de ordem superior em objetos chamáveis

Código-fonte: Lib/functools.py
[https://github.com/python/cpython/tree/3.15/Lib/functools.py]

The **`functools`** module is for higher-order functions: functions that act on or return other functions. In general, any callable object can be treated as a function for the purposes of this module.

The **`functools`** module defines the following functions:

@functools.cache(*user_function*)

Cache simples e leve de funções sem vínculo. Às vezes chamado de "memoizar" [https://en.wikipedia.org/wiki/Memoization].

Returns the same as `lru_cache(maxsize=None)`, creating a thin wrapper around a dictionary lookup for the function arguments. Because it never needs to evict old values, this is smaller and faster than **`@lru_cache`** with a size limit.

Por exemplo:

```
@cache
def factorial(n):
    return n * factorial(n-1) if n else 1

>>> factorial(10)   # sem nenhum resultado previamente armaze
3628800
>>> factorial(5)    # sem novas chamadas, apenas retorna o re
120
```

```
>>> factorial(12)   # duas novas chamadas recursivas, factori
479001600
```

O cache é seguro para thread, permitindo que a função encapsulada seja usada em múltiplas threads. Isso significa que a estrutura de dados subjacente permanecerá coerente durante atualizações simultâneas.

É possível que a função encapsulada seja chamada mais de uma vez se outra thread fizer uma chamada adicional antes que a chamada inicial seja concluída e armazenada em cache.

Call-once behavior is not guaranteed because locks are not held during the function call. Potentially another call with the same arguments could occur while the first call is still running.

Adicionado na versão 3.9.

@functools.cached_property(*func*)

Transform a method of a class into a property whose value is computed once and then cached as a normal attribute for the life of the instance. Similar to `@property`, with the addition of caching. Useful for expensive computed properties of instances that are otherwise effectively immutable.

Exemplo:

```
class DataSet:

    def __init__(self, sequence_of_numbers):
        self._data = tuple(sequence_of_numbers)

    @cached_property
    def stdev(self):
        return statistics.stdev(self._data)
```

The mechanics of `@cached_property` are somewhat different from `@property`. A regular property blocks attribute writes unless a setter is defined. In contrast, a *cached_property* allows writes.

O decorador *cached_property* só é executado em pesquisas e apenas quando um atributo com o mesmo nome não existe. Quando executado, o *cached_property* escreve no atributo com o mesmo nome. Leituras e escritas subsequentes do atributo têm precedência sobre o método *cached_property* e ele funciona como um atributo normal.

O valor em cache pode ser apagado excluindo o atributo. Isso permite que o método *cached_property* seja executado novamente.

A propriedade *cached_property* não impede uma possível condição de corrida em uso multithread. A função getter pode ser executada mais de uma vez na mesma instância, com a última execução definindo o valor em cache. Se a propriedade em cache for idempotente ou não causar problemas ao ser executada mais de uma vez em uma instância, isso não será um problema. Caso seja necessária sincronização, implemente a trava necessária dentro da função getter decorada ou em torno do acesso à propriedade em cache.

Observe que este decorador interfere com a operação dos dicionários que compartilham chaves da **PEP 412** [https://peps.python.org/pep-0412/]. Isso significa que os dicionários de instância podem ocupar mais espaço do que o normal.

Also, this decorator requires that the `__dict__` attribute on each instance be a mutable mapping. This means it will not work with some types, such as metaclasses (since the `__dict__` attributes on type instances are read-only proxies for the class namespace), and those that specify `__slots__` without including `__dict__` as one of the defined slots (as such classes don't provide a `__dict__` attribute at all).

If a mutable mapping is not available or if space-efficient key sharing is desired, an effect similar to **`@cached_property`** can also be achieved by stacking **`@property`** on top of **`@lru_cache`**. See Como faço para armazenar em cache as chamadas de um método? for more details on how this differs from **`@cached_property`**.

Adicionado na versão 3.8.

Alterado na versão 3.12: Prior to Python 3.12, **`@cached_property`** included an undocumented lock to ensure that in multi-threaded usage the getter function was guaranteed to run only once per instance. However, the lock was per-property, not per-instance, which could result in unacceptably high lock contention. In Python 3.12+ this locking is removed.

functools.cmp_to_key(*func*)

Transforma uma função de comparação de estilo antigo para um função chave. Usado com ferramentas que aceitam funções chave (como **`sorted()`**, **`min()`**, **`max()`**, **`heapq.nlargest()`**, **`heapq.nsmallest()`**, **`itertools.groupby()`**). Esta função é usada principalmente como uma ferramenta de transição para programas que estão sendo convertidos a partir do Python 2, que suportou o uso de funções de comparação.

A comparison function is any callable that accepts two arguments, compares them, and returns a negative number for less-than, zero for equality, or a positive number for greater-than. A key function is a callable that accepts one argument and returns another value to be used as the sort key.

Exemplo:

```
sorted(iterable, key=cmp_to_key(locale.strcoll))  # locale-aw
```

Para exemplos de classificação e um breve tutorial de classificação, veja Técnicas de ordenação.

Adicionado na versão 3.2.

@functools.lru_cache(*user_function*)

@functools.lru_cache(*maxsize=128, typed=False*)

Decorador para embrulhar uma função com um chamável memoizável que economiza até as chamadas mais recentes *maxsize*. Pode economizar tempo quando uma função cara ou E/S é periodicamente chamada com os mesmos argumentos.

O cache é seguro para thread, permitindo que a função encapsulada seja usada em múltiplas threads. Isso significa que a estrutura de dados subjacente permanecerá coerente durante atualizações simultâneas.

É possível que a função encapsulada seja chamada mais de uma vez se outra thread fizer uma chamada adicional antes que a chamada inicial seja concluída e armazenada em cache.

Since a dictionary is used to cache results, the positional and keyword arguments to the function must be hashable.

Distinct argument patterns may be considered to be distinct calls with separate cache entries. For example, `f(a=1, b=2)` and `f(b=2, a=1)` differ in their keyword argument order and may have two separate cache entries.

Se *user_function* especificado, deve ser um chamável. Isso permite que o decorador *lru_cache* seja aplicado diretamente a uma função do usuário, deixando *maxsize* em seu valor padrão de 128:

```
@lru_cache
def count_vowels(word):
    return sum(word.count(vowel) for vowel in 'AEIOUaeiou')
```

Se *maxsize* for definido como `None`, o recurso LRU é desabilitado e o cache pode crescer sem limites.

If *typed* is set to true, function arguments of different types will be cached separately. If *typed* is false, the implementation will usually regard them as equivalent calls and only cache a single result. (Some types such as *str* and *int* may be cached separately even when *typed* is false.)

Note, type specificity applies only to the function's immediate arguments rather than their contents. The scalar arguments, `Decimal(42)` and `Fraction(42)` are treated as distinct calls with distinct results. In contrast, the tuple arguments `('answer', Decimal(42))` and `('answer', Fraction(42))` are treated as equivalent.

The wrapped function is instrumented with a **`cache_parameters()`** function that returns a new **`dict`** showing the values for *maxsize* and *typed*. This is for information purposes only. Mutating the values has no effect.

To help measure the effectiveness of the cache and tune the *maxsize* parameter, the wrapped function is instrumented with a **`cache_info()`** function that returns a named tuple showing *hits*, *misses*, *maxsize* and *currsize*.

The decorator also provides a **`cache_clear()`** function for clearing or invalidating the cache.

A função subjacente original é acessível através do atributo **`__wrapped__`**. Isso é útil para introspecção, para ignorar o cache, ou para reinstalar a função com um cache diferente.

The cache keeps references to the arguments and return values until they age out of the cache or until the cache is cleared.

If a method is cached, the `self` instance argument is included in the cache. See Como faço para armazenar em cache as chamadas de um método?

An LRU (least recently used) cache [https://en.wikipedia.org/wiki/Cache_replacement_policies#Least_Recently_Used_(LRU)] works best when the most recent calls are the best predictors of upcoming calls (for example, the most popular articles on a news server tend to change each day). The cache's size limit assures that the cache does not grow without bound on long-running processes such as web servers.

In general, the LRU cache should only be used when you want to reuse previously computed values. Accordingly, it doesn't make sense to cache functions with side-effects, functions that need to create distinct mutable objects on each call (such as generators and async functions), or impure functions such as time() or random().

Exemplo de um cache LRU para conteúdo web estático:

```
@lru_cache(maxsize=32)
def get_pep(num):
    'Retrieve text of a Python Enhancement Proposal'
    resource = f'https://peps.python.org/pep-{num:04d}'
    try:
        with urllib.request.urlopen(resource) as s:
            return s.read()
    except urllib.error.HTTPError:
        return 'Not Found'

>>> for n in 8, 290, 308, 320, 8, 218, 320, 279, 289, 320, 9
...     pep = get_pep(n)
...     print(n, len(pep))

>>> get_pep.cache_info()
CacheInfo(hits=3, misses=8, maxsize=32, currsize=8)
```

Exemplo de computação eficiente dos [números Fibonacci](https://pt.wikipedia.org/wiki/Sequência_de_Fibonacci) usando um cache para implementar uma [programação dinâmica](https://pt.wikipedia.org/wiki/Programação_dinâmica) técnica:

```
@lru_cache(maxsize=None)
def fib(n):
    if n < 2:
        return n
    return fib(n-1) + fib(n-2)

>>> [fib(n) for n in range(16)]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610]

>>> fib.cache_info()
CacheInfo(hits=28, misses=16, maxsize=None, currsize=16)
```

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionada a opção *typed*.

Alterado na versão 3.8: Adicionada a opção *user_function*.

Alterado na versão 3.9: Added the function **`cache_parameters()`**

@functools.total_ordering

Given a class defining one or more rich comparison ordering methods, this class decorator supplies the rest. This simplifies the effort involved in specifying all of the possible rich comparison operations:

The class must define one of **`__lt__()`**, **`__le__()`**, **`__gt__()`**, or **`__ge__()`**. In addition, the class should supply an **`__eq__()`** method.

Por exemplo:

```
@total_ordering
class Student:
    def _is_valid_operand(self, other):
        return (hasattr(other, "lastname") and
                hasattr(other, "firstname"))
    def __eq__(self, other):
        if not self._is_valid_operand(other):
            return NotImplemented
        return ((self.lastname.lower(), self.firstname.lower(
                (other.lastname.lower(), other.firstname.lowe
    def __lt__(self, other):
        if not self._is_valid_operand(other):
            return NotImplemented
        return ((self.lastname.lower(), self.firstname.lower(
                (other.lastname.lower(), other.firstname.lowe
```

Nota

While this decorator makes it easy to create well behaved totally ordered types, it *does* come at the cost of slower execution and more complex stack traces for the derived comparison methods. If performance benchmarking indicates this is a bottleneck for a given application, implementing all six rich comparison methods instead is likely to provide an easy speed boost.

Nota

This decorator makes no attempt to override methods that have been declared in the class *or its superclasses*. Meaning that if a superclass defines a comparison operator, *total_ordering* will not implement it again, even if the original method is abstract.

Adicionado na versão 3.2.

Alterado na versão 3.4: Returning `NotImplemented` from the underlying comparison function for unrecognised types is now supported.

functools.Placeholder

A singleton object used as a sentinel to reserve a place for positional arguments when calling **`partial()`** and **`partialmethod()`**.

Adicionado na versão 3.14.

functools.partial(*func*, /, **args*, ***keywords*)

Return a new partial object which when called will behave like *func* called with the positional arguments *args* and keyword arguments *keywords*. If more arguments are supplied to the call, they are appended to *args*. If additional keyword arguments are supplied, they extend and override *keywords*. Roughly equivalent to:

```
def partial(func, /, *args, **keywords):
    def newfunc(*more_args, **more_keywords):
        return func(*args, *more_args, **(keywords | more_key
    newfunc.func = func
    newfunc.args = args
    newfunc.keywords = keywords
    return newfunc
```

The **`partial()`** function is used for partial function application which “freezes” some portion of a function’s arguments and/or keywords resulting in a new object with a simplified signature. For example, **`partial()`** can be used to create a callable that behaves like the **`int()`** function where the *base* argument defaults to `2`:

```
>>> basetwo = partial(int, base=2)
>>> basetwo.__doc__ = 'Convert base 2 string to an int.'
>>> basetwo('10010')
18
```

If **`Placeholder`** sentinels are present in *args*, they will be filled first when **`partial()`** is called. This makes it possible to pre-fill any positional argument with a call to **`partial()`**; without **`Placeholder`**, only the chosen number of leading positional arguments can be pre-filled.

If any **`Placeholder`** sentinels are present, all must be filled at call time:

```
>>> say_to_world = partial(print, Placeholder, Placeholder, '
>>> say_to_world('Hello', 'dear')
Hello dear world!
```

Calling `say_to_world('Hello')` raises a **`TypeError`**, because only one positional argument is provided, but there are two placeholders that must be filled in.

If **`partial()`** is applied to an existing partial object, **`Placeholder`** sentinels of the input object are filled in with new positional arguments. A placeholder can be retained by inserting a new **`Placeholder`** sentinel to the place held by a previous **`Placeholder`**:

```
>>> from functools import partial, Placeholder as _
>>> remove = partial(str.replace, _, _, '')
>>> message = 'Hello, dear dear world!'
>>> remove(message, ' dear')
'Hello, world!'
>>> remove_dear = partial(remove, _, ' dear')
>>> remove_dear(message)
'Hello, world!'
>>> remove_first_dear = partial(remove_dear, _, 1)
>>> remove_first_dear(message)
'Hello, dear world!'
```

`Placeholder` cannot be passed to **`partial()`** as a keyword argument.

Alterado na versão 3.14: Added support for **`Placeholder`** in positional arguments.

class functools.partialmethod(*func*, /, **args*, ***keywords*)

Return a new **`partialmethod`** descriptor which behaves like **`partial`** except that it is designed to be used as a method definition rather than being directly callable.

func must be a descriptor or a callable (objects which are both, like normal functions, are handled as descriptors).

When *func* is a descriptor (such as a normal Python function, **`classmethod()`**, **`staticmethod()`**, **`abstractmethod()`** or another instance of **`partialmethod`**), calls to `__get__` are delegated to the underlying descriptor, and an appropriate partial object returned as the result.

When *func* is a non-descriptor callable, an appropriate bound method is created dynamically. This behaves like a normal Python function when used as a method: the *self* argument will be inserted as the first positional argument, even before the *args* and *keywords* supplied to the **`partialmethod`** constructor.

Exemplo:

```
>>> class Cell:
...     def __init__(self):
...         self._alive = False
...     @property
...     def alive(self):
...         return self._alive
...     def set_state(self, state):
...         self._alive = bool(state)
...     set_alive = partialmethod(set_state, True)
...     set_dead = partialmethod(set_state, False)
...
>>> c = Cell()
>>> c.alive
False
>>> c.set_alive()
```

```
>>> c.alive
True
```

Adicionado na versão 3.4.

functools.reduce(*function*, *iterable*, /[, *initial*])

Apply *function* of two arguments cumulatively to the items of *iterable*, from left to right, so as to reduce the iterable to a single value. For example, `reduce(lambda x, y: x+y, [1, 2, 3, 4, 5])` calculates `((((1+2)+3)+4)+5)`. The left argument, *x*, is the accumulated value and the right argument, *y*, is the update value from the *iterable*. If the optional *initial* is present, it is placed before the items of the iterable in the calculation, and serves as a default when the iterable is empty. If *initial* is not given and *iterable* contains only one item, the first item is returned.

Aproximadamente equivalente a:

```
initial_missing = sentinel('initial_missing')

def reduce(function, iterable, /, initial=initial_missing):
    it = iter(iterable)
    if initial is initial_missing:
        value = next(it)
    else:
        value = initial
    for element in it:
        value = function(value, element)
    return value
```

See **`itertools.accumulate()`** for an iterator that yields all intermediate values.

Alterado na versão 3.14: initial is now supported as a keyword argument.

@functools.singledispatch

Transform a function into a single-dispatch generic function.

To define a generic function, decorate it with the `@singledispatch` decorator. When defining a function using `@singledispatch`, note that the dispatch happens on the type of the first argument:

```
>>> from functools import singledispatch
>>> @singledispatch
... def fun(arg, verbose=False):
...     if verbose:
...         print("Let me just say,", end=" ")
...     print(arg)
```

To add overloaded implementations to the function, use the **`register()`** attribute of the generic function, which can be used as a decorator. For functions annotated with types, the decorator will infer the type of the first argument automatically:

```
>>> @fun.register
... def _(arg: int, verbose=False):
...     if verbose:
...         print("Strength in numbers, eh?", end=" ")
...     print(arg)
...
>>> @fun.register
... def _(arg: list, verbose=False):
...     if verbose:
...         print("Enumerate this:")
...     for i, elem in enumerate(arg):
...         print(i, elem)
```

`typing.Union` can also be used:

```
>>> @fun.register
... def _(arg: int | float, verbose=False):
...     if verbose:
...         print("Strength in numbers, eh?", end=" ")
...     print(arg)
...
>>> from typing import Union
>>> @fun.register
... def _(arg: Union[list, set], verbose=False):
...     if verbose:
...         print("Enumerate this:")
```

```
...     for i, elem in enumerate(arg):
...         print(i, elem)
...
```

For code which doesn't use type annotations, the appropriate type argument can be passed explicitly to the decorator itself:

```
>>> @fun.register(complex)
... def _(arg, verbose=False):
...     if verbose:
...         print("Better than complicated.", end=" ")
...     print(arg.real, arg.imag)
...
```

For code that dispatches on a collections type (e.g., `list`), but wants to typehint the items of the collection (e.g., `list[int]`), the dispatch type should be passed explicitly to the decorator itself with the typehint going into the function definition:

```
>>> @fun.register(list)
... def _(arg: list[int], verbose=False):
...     if verbose:
...         print("Enumerate this:")
...     for i, elem in enumerate(arg):
...         print(i, elem)
```

Nota

At runtime the function will dispatch on an instance of a list regardless of the type contained within the list i.e. `[1,2,3]` will be dispatched the same as `["foo", "bar", "baz"]`. The annotation provided in this example is for static type checkers only and has no runtime impact.

To enable registering lambdas and pre-existing functions, the **`register()`** attribute can also be used in a functional form:

```
>>> def nothing(arg, verbose=False):
...     print("Nothing.")
...
>>> fun.register(type(None), nothing)
```

The **register()** attribute returns the undecorated function. This enables decorator stacking, **pickling**, and the creation of unit tests for each variant independently:

```
>>> @fun.register(float)
... @fun.register(Decimal)
... def fun_num(arg, verbose=False):
...     if verbose:
...         print("Half of your number:", end=" ")
...     print(arg / 2)
...
>>> fun_num is fun
False
```

When called, the generic function dispatches on the type of the first argument:

```
>>> fun("Hello, world.")
Hello, world.
>>> fun("test.", verbose=True)
Let me just say, test.
>>> fun(42, verbose=True)
Strength in numbers, eh? 42
>>> fun(['spam', 'spam', 'eggs', 'spam'], verbose=True)
Enumerate this:
0 spam
1 spam
2 eggs
3 spam
>>> fun(None)
Nothing.
>>> fun(1.23)
0.615
```

Where there is no registered implementation for a specific type, its method resolution order is used to find a more generic implementation. The original function decorated with `@singledispatch` is registered for the base **object** type, which means it is used if no better implementation is found.

If an implementation is registered to an abstract base class, virtual subclasses of the base class will be dispatched to that implementation:

```
>>> from collections.abc import Mapping
>>> @fun.register
... def _(arg: Mapping, verbose=False):
...     if verbose:
...         print("Keys & Values")
...     for key, value in arg.items():
...         print(key, "=>", value)
...
>>> fun({"a": "b"})
a => b
```

To check which implementation the generic function will choose for a given type, use the `dispatch()` attribute:

```
>>> fun.dispatch(float)
<function fun_num at 0x1035a2840>
>>> fun.dispatch(dict)    # note: default implementation
<function fun at 0x103fe0000>
```

To access all registered implementations, use the read-only `registry` attribute:

```
>>> fun.registry.keys()
dict_keys([<class 'NoneType'>, <class 'int'>, <class 'object'
          <class 'decimal.Decimal'>, <class 'list'>,
          <class 'float'>])
>>> fun.registry[float]
<function fun_num at 0x1035a2840>
>>> fun.registry[object]
<function fun at 0x103fe0000>
```

Adicionado na versão 3.4.

Alterado na versão 3.7: The `register()` attribute now supports using type annotations.

Alterado na versão 3.11: The `register()` attribute now supports `typing.Union` as a type annotation.

class functools.singledispatchmethod(*func*)

Transform a method into a single-dispatch generic function.

To define a generic method, decorate it with the `@singledispatchmethod` decorator. When defining a method using `@singledispatchmethod`, note that the dispatch happens on the type of the first non-*self* or non-*cls* argument:

```
class Negator:
    @singledispatchmethod
    def neg(self, arg):
        raise NotImplementedError("Cannot negate a")

    @neg.register
    def _(self, arg: int):
        return -arg

    @neg.register
    def _(self, arg: bool):
        return not arg
```

`@singledispatchmethod` supports nesting with other decorators such as **`@classmethod`**. Note that to allow for `dispatcher.register`, `singledispatchmethod` must be the *outer most* decorator. Here is the `Negator` class with the `neg` methods bound to the class, rather than an instance of the class:

```
class Negator:
    @singledispatchmethod
    @classmethod
    def neg(cls, arg):
        raise NotImplementedError("Cannot negate a")

    @neg.register
    @classmethod
    def _(cls, arg: int):
        return -arg

    @neg.register
    @classmethod
```

```
def _(cls, arg: bool):
    return not arg
```

The same pattern can be used for other similar decorators: `@staticmethod`, `@~abc.abstractmethod`, and others.

Adicionado na versão 3.8.

Alterado na versão 3.15: Added support of non-descriptor callables.

functools.update_wrapper(*wrapper*, *wrapped*, *assigned=WRAPPER_ASSIGNMENTS*, *updated=WRAPPER_UPDATES*)

Update a *wrapper* function to look like the *wrapped* function. The optional arguments are tuples to specify which attributes of the original function are assigned directly to the matching attributes on the wrapper function and which attributes of the wrapper function are updated with the corresponding attributes from the original function. The default values for these arguments are the module level constants `WRAPPER_ASSIGNMENTS` (which assigns to the wrapper function's `__module__`, `__name__`, `__qualname__`, `__annotations__`, `__type_params__`, and `__doc__`, the documentation string) and `WRAPPER_UPDATES` (which updates the wrapper function's `__dict__`, i.e. the instance dictionary).

To allow access to the original function for introspection and other purposes (e.g. bypassing a caching decorator such as `@lru_cache`), this function automatically adds a `__wrapped__` attribute to the wrapper that refers to the function being wrapped.

The main intended use for this function is in decorator functions which wrap the decorated function and return the wrapper. If the wrapper function is not updated, the metadata of the returned function will reflect the wrapper definition rather than the original function definition, which is typically less than helpful.

`update_wrapper()` may be used with callables other than functions. Any attributes named in *assigned* or *updated* that are missing from the object being wrapped are ignored (i.e. this function will not attempt to

set them on the wrapper function). **`AttributeError`** is still raised if the wrapper function itself is missing any attributes named in *updated*.

Alterado na versão 3.2: The `__wrapped__` attribute is now automatically added. The **`__annotations__`** attribute is now copied by default. Missing attributes no longer trigger an **`AttributeError`**.

Alterado na versão 3.4: The `__wrapped__` attribute now always refers to the wrapped function, even if that function defined a `__wrapped__` attribute. (see bpo-17482 [https://bugs.python.org/issue?@action=redirect&bpo=17482])

Alterado na versão 3.12: The **`__type_params__`** attribute is now copied by default.

@functools.wraps(*wrapped*, *assigned=WRAPPER_ASSIGNMENTS*, *updated=WRAPPER_UPDATES*)

This is a convenience function for invoking **`update_wrapper()`** as a function decorator when defining a wrapper function. It is equivalent to `partial(update_wrapper, wrapped=wrapped, assigned=assigned, updated=updated)`. For example:

```
>>> from functools import wraps
>>> def my_decorator(f):
...     @wraps(f)
...     def wrapper(*args, **kwds):
...         print('Calling decorated function')
...         return f(*args, **kwds)
...     return wrapper
...
>>> @my_decorator
... def example():
...     """Docstring"""
...     print('Called example function')
...
>>> example()
Calling decorated function
Called example function
>>> example.__name__
'example'
```

```
>>> example.__doc__
'Docstring'
```

Without the use of this decorator factory, the name of the example function would have been `'wrapper'`, and the docstring of the original **`example()`** would have been lost.

Objetos `partial`

`partial` objects are callable objects created by **`partial()`**. They have three read-only attributes:

partial.func

A callable object or function. Calls to the **`partial`** object will be forwarded to **`func`** with new arguments and keywords.

partial.args

The leftmost positional arguments that will be prepended to the positional arguments provided to a **`partial`** object call.

partial.keywords

The keyword arguments that will be supplied when the **`partial`** object is called.

`partial` objects are like function objects in that they are callable, weak referenceable, and can have attributes. There are some important differences. For instance, the **`__name__`** and **`__doc__`** attributes are not created automatically.

`operator` — Operadores padrões como funções

Código-fonte: Lib/operator.py
[https://github.com/python/cpython/tree/3.15/Lib/operator.py]

The **`operator`** module exports a set of efficient functions corresponding to the intrinsic operators of Python. For example, `operator.add(x, y)` is equivalent to the expression `x+y`. Many function names are those used for special methods, without the double underscores. For backward compatibility, many of these have a variant with the double underscores kept. The variants without the double underscores are preferred for clarity.

As funções se enquadram em categorias que realizam comparações de objetos, operações lógicas, operações matemáticas e operações de sequência.

As funções de comparação de objetos são úteis para todos os objetos e são nomeadas conforme os operadores de comparação que os mesmos suportam:

operator.lt(*a*, *b*)
operator.le(*a*, *b*)
operator.eq(*a*, *b*)
operator.ne(*a*, *b*)
operator.ge(*a*, *b*)
operator.gt(*a*, *b*)
operator.__lt__(*a*, *b*)
operator.__le__(*a*, *b*)
operator.__eq__(*a*, *b*)
operator.__ne__(*a*, *b*)

operator.__ge__(*a*, *b*)

operator.__gt__(*a*, *b*)

Executam “comparações ricas” entre *a* e *b*. Especialmente, `lt(a, b)` é equivalente a `a < b`, `le(a, b)` é equivalente a `a <= b`, `eq(a, b)` é equivalente a `a == b`, `ne(a, b)` é equivalente a `a != b`, `gt(a, b)` é equivalente a `a > b` e `ge(a, b)` é equivalente a `a >= b`. Observe que essas funções podem retornar qualquer valor, que pode ou não ser interpretável como um valor booleano. Consulte Comparações para obter mais informações sobre comparações ricas.

As operações lógicas também são geralmente aplicáveis a todos os objetos e tem suporte para testes de verdade, testes de identidade e operações booleanas:

operator.not_(*obj*)

operator.__not__(*obj*)

Retorna o resultado de **`not`** *obj*. (Veja que não existe nenhum método **`__not__()`** para instâncias de objetos; apenas o núcleo do interpretador definirá esta operação. O resultado será afetado pelos métodos **`__bool__()`** e **`__len__()`**.)

operator.truth(*obj*)

Retorna **`True`** se o *obj* for verdadeiro, e **`False`** caso contrário. Isso é equivalente a utilizar a construção **`bool`**.

operator.is_(*a*, *b*)

Retorna `a is b`. Testa a identidade do objeto.

operator.is_not(*a*, *b*)

Retorna `a is not b`. Testa a identidade do objeto.

operator.is_none(*a*)

Retorna `a is None`. Testa a identidade do objeto.

Adicionado na versão 3.14.

operator.is_not_none(*a*)

Retorna `a is not None`. Testa a identidade do objeto.

Adicionado na versão 3.14.

As operações matemáticas bit a bit são as mais numerosas:

operator.abs(*obj*)
operator.__abs__(*obj*)

Retorna o valor absoluto de *obj*.

operator.add(*a*, *b*)
operator.__add__(*a*, *b*)

Retorna `a + b`, onde *a* e *b* são números.

operator.and_(*a*, *b*)
operator.__and__(*a*, *b*)

Return `a & b`.

operator.floordiv(*a*, *b*)
operator.__floordiv__(*a*, *b*)

Retorna `a // b`.

operator.index(*a*)
operator.__index__(*a*)

Retorna *a* convertendo para um inteiro. Equivalente a `a.__index__()`.

Alterado na versão 3.10: O resultado sempre tem o tipo exato `int`. Anteriormente, o resultado poderia ter sido uma instância de uma subclasse de `int`.

operator.inv(*obj*)
operator.invert(*obj*)
operator.__inv__(*obj*)
operator.__invert__(*obj*)

Return `~obj`.

operator.lshift(*a*, *b*)
operator.__lshift__(*a*, *b*)
Return `a << b`.

operator.mod(*a*, *b*)
operator.__mod__(*a*, *b*)
Retorna `a % b`.

operator.mul(*a*, *b*)
operator.__mul__(*a*, *b*)
Return `a * b`.

operator.matmul(*a*, *b*)
operator.__matmul__(*a*, *b*)
Retorna `a @ b`.

Adicionado na versão 3.5.

operator.neg(*obj*)
operator.__neg__(*obj*)
Retorna *obj* negado (`-obj`).

operator.or_(*a*, *b*)
operator.__or__(*a*, *b*)
Return `a | b`.

operator.pos(*obj*)
operator.__pos__(*obj*)
Return `+obj`.

operator.pow(*a*, *b*)
operator.__pow__(*a*, *b*)

Return `a ** b`.

operator.rshift(*a*, *b*)
operator.__rshift__(*a*, *b*)
Return `a >> b`.

operator.sub(*a*, *b*)
operator.__sub__(*a*, *b*)
Retorna `a - b`.

operator.truediv(*a*, *b*)
operator.__truediv__(*a*, *b*)
Retorna `a / b` onde 2/3 é .66 em vez de 0. Isso também é conhecido como divisão “verdadeira”.

operator.xor(*a*, *b*)
operator.__xor__(*a*, *b*)
Return `a ^ b`.

Operações que funcionam com sequências (algumas delas com mapas também) incluem:

operator.concat(*a*, *b*)
operator.__concat__(*a*, *b*)
Retorna `a + b` para as sequências *a* e *b*.

operator.contains(*a*, *b*)
operator.__contains__(*a*, *b*)
Retorna o resultado do teste `b in a`. Observe os operandos invertidos.

operator.countOf(*a*, *b*)
Retorna o número de ocorrências de *b* em *a*.

operator.delitem(*a*, *b*)
operator.__delitem__(*a*, *b*)

Remove de *a* o valor no índice *b*.

operator.getitem(*a*, *b*)
operator.__getitem__(*a*, *b*)
Retorna de *a* o valor no índice *b*.

operator.indexOf(*a*, *b*)
Retorna o índice da primeira ocorrência de *b* em *a*.

operator.setitem(*a*, *b*, *c*)
operator.__setitem__(*a*, *b*, *c*)
Define em *a* o valor no índice *b* para *c*.

operator.length_hint(*obj*, *default=0*)
Retorna um comprimento estimado para o objeto *obj*. Primeiro tenta retornar o seu comprimento real, em seguida, uma estimativa utilizando `object.__length_hint__()`, e finalmente retorna o valor padrão.

Adicionado na versão 3.4.

Os operadores seguintes funcionam com chamáveis:

operator.call(*obj*, /, **args*, ***kwargs*)
operator.__call__(*obj*, /, **args*, ***kwargs*)
Retorna `obj(*args, **kwargs)`.

Adicionado na versão 3.11.

The **`operator`** module also defines tools for generalized attribute and item lookups. These are useful for making fast field extractors as arguments for `map()`, `sorted()`, `itertools.groupby()`, or other functions that expect a function argument.

operator.attrgetter(*attr*)
operator.attrgetter(**attrs*)

Retorna um objeto chamável que pode buscar o *attr* do seu operando. Caso seja solicitado mais de um atributo, retorna uma tupla de atributos. Os nomes dos atributos também podem conter pontos. Por exemplo:

- Depois de `f = attrgetter('name')`, a chamada a `f(b)` retorna `b.name`.
- Depois de `f = attrgetter('name', 'date')`, a chamada a `f(b)` retorna `(b.name, b.date)`.
- Depois de `f = attrgetter('name.first', 'name.last')`, a chamada a `f(b)` retorna `(b.name.first, b.name.last)`.

Equivalente a:

```
def attrgetter(*items):
    if any(not isinstance(item, str) for item in items):
        raise TypeError('attribute name must be a string')
    if len(items) == 1:
        attr = items[0]
        def g(obj):
            return resolve_attr(obj, attr)
    else:
        def g(obj):
            return tuple(resolve_attr(obj, attr) for attr in
    return g

def resolve_attr(obj, attr):
    for name in attr.split("."):
        obj = getattr(obj, name)
    return obj
```

operator.itemgetter(*item*)
operator.itemgetter(**items*)

Retorna um objeto chamável que busca *item* de seu operando usando o método `__getitem__()` do operando. Se vários itens forem especificados, retorna um tupla de valores de pesquisa. Por exemplo:

- Depois de `f = itemgetter(2)`, a chamada a `f(r)` retorna `r[2]`.
- Depois de `g = itemgetter(2, 5, 3)`, a chamada a `g(r)` retorna `(r[2], r[5], r[3])`.

Equivalente a:

```
def itemgetter(*items):
    if len(items) == 1:
        item = items[0]
        def g(obj):
            return obj[item]
    else:
        def g(obj):
            return tuple(obj[item] for item in items)
    return g
```

Os itens podem ser de qualquer tipo aceito pelo método `__getitem__()` do operando. Dicionários aceitam qualquer valor hasheável . Listas, tuplas e strings aceitam um índice ou um fatiamento:

```
>>> itemgetter(1)('ABCDEFG')
'B'
>>> itemgetter(1, 3, 5)('ABCDEFG')
('B', 'D', 'F')
>>> itemgetter(slice(2, None))('ABCDEFG')
'CDEFG'
>>> soldier = dict(rank='captain', name='dotterbart')
>>> itemgetter('rank')(soldier)
'captain'
```

Exemplo de uso `itemgetter()` para recuperar campos específicos de um registro de tupla:

```
>>> inventory = [('apple', 3), ('banana', 2), ('pear', 5), ('
>>> getcount = itemgetter(1)
>>> list(map(getcount, inventory))
[3, 2, 5, 1]
>>> sorted(inventory, key=getcount)
[('orange', 1), ('banana', 2), ('apple', 3), ('pear', 5)]
```

operator.methodcaller(*name*, /, **args*, ***kwargs*)

Retorna um objeto chamável que invoca o método *name* em seu operando. Se argumentos adicionais e/ou argumentos nomeados forem fornecidos, os mesmos serão passados para o método. Por exemplo:

- Depois de `f = methodcaller('name')`, a chamada a `f(b)` retorna `b.name()`.
- Depois de `f = methodcaller('name', 'foo', bar=1)`, a chamada `f(b)` retorna `b.name('foo', bar=1)`.

Equivalente a:

```
def methodcaller(name, /, *args, **kwargs):
    def caller(obj):
        return getattr(obj, name)(*args, **kwargs)
    return caller
```

Mapeando os operadores para suas respectivas funções

This table shows how abstract operations correspond to operator symbols in the Python syntax and the functions in the **`operator`** module.

Operação	Sintaxe	Função
Adição	`a + b`	`add(a, b)`
Concatenação	`seq1 + seq2`	`concat(seq1, seq2)`
Teste de pertinência	`obj in seq`	`contains(seq, obj)`
Divisão	`a / b`	`truediv(a, b)`
Divisão	`a // b`	`floordiv(a, b)`

Operação	Sintaxe	Função
Bitwise And, or Intersection	`a & b`	`and_(a, b)`
Bitwise Exclusive Or, or Symmetric Difference	`a ^ b`	`xor(a, b)`
Bitwise Inversion, or Complement	`~ a`	`invert(a)`
Bitwise Or, or Union	`a | b`	`or_(a, b)`
Exponenciação	`a ** b`	`pow(a, b)`
Identidade	`a is b`	`is_(a, b)`
Identidade	`a is not b`	`is_not(a, b)`
Identidade	`a is None`	`is_none(a)`
Identidade	`a is not None`	`is_not_none(a)`
Atribuição Indexada	`obj[k] = v`	`setitem(obj, k, v)`
Eliminação indexada	`del obj[k]`	`delitem(obj, k)`

Operação	Sintaxe	Função
Indexação	`obj[k]`	`getitem(obj, k)`
Deslocamento à esquerda	`a << b`	`lshift(a, b)`
Módulo	`a % b`	`mod(a, b)`
Multiplicação	`a * b`	`mul(a, b)`
Multiplicação de matrizes	`a @ b`	`matmul(a, b)`
Negação (aritmética)	`- a`	`neg(a)`
Negação (lógica)	`not a`	`not_(a)`
Positivo	`+ a`	`pos(a)`
Deslocamento à direita	`a >> b`	`rshift(a, b)`
Atribuição de fatia	`seq[i:j] = values`	`setitem(seq, slice(i, j), values)`
Remoção de fatia	`del seq[i:j]`	`delitem(seq, slice(i, j))`

Operação	Sintaxe	Função
Fatiamento	`seq[i:j]`	`getitem(seq, slice(i, j))`
Formatação de strings	`s % obj`	`mod(s, obj)`
Subtração	`a - b`	`sub(a, b)`
Teste verdadeiro	`obj`	`truth(obj)`
Ordenação	`a < b`	`lt(a, b)`
Ordenação	`a <= b`	`le(a, b)`
Igualdade	`a == b`	`eq(a, b)`
Diferença	`a != b`	`ne(a, b)`
Ordenação	`a >= b`	`ge(a, b)`
Ordenação	`a > b`	`gt(a, b)`

Operadores in-place

Muitas operações possuem uma versão “in-place”. Listadas abaixo, as funções fornecem um acesso mais primitivo aos operadores locais do que a sintaxe usual; por exemplo, a instrução `x += y` é equivalente a `x = operator.iadd(x, y)`. Outra maneira de colocá-lo é dizendo que `z = operator.iadd(x, y)` é equivalente à instrução composta `z = x; z += y`.

Nesses exemplos, note que, quando um método in-place é invocado, a computação e a atribuição são realizadas em duas etapas separadas. As funções in-place listadas abaixo apenas fazem o primeiro passo, invocando o método in-place. O segundo passo, a atribuição, não é tratado.

Para os casos imutáveis, como as strings, números e tuplas, o valor atualizado será calculado, mas não será atribuído de volta à variável de entrada:

```
>>> a = 'hello'
>>> iadd(a, ' world')
'hello world'
>>> a
'hello'
```

Para alvos mutáveis tal como listas e dicionários, o método in-place vai realizar a atualização, então nenhuma atribuição subsequente é necessária:

```
>>> s = ['h', 'e', 'l', 'l', 'o']
>>> iadd(s, [' ', 'w', 'o', 'r', 'l', 'd'])
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
>>> s
['h', 'e', 'l', 'l', 'o', ' ', 'w', 'o', 'r', 'l', 'd']
```

operator.iadd(*a*, *b*)

operator.__iadd__(*a*, *b*)

`a = iadd(a, b)` é equivalente a `a += b`.

operator.iand(*a*, *b*)

operator.__iand__(*a*, *b*)

`a = iand(a, b)` é equivalente a `a &= b`.

operator.iconcat(*a*, *b*)
operator.__iconcat__(*a*, *b*)

`a = iconcat(a, b)` é equivalente a `a += b` onde *a* e *b* são sequências.

operator.ifloordiv(*a*, *b*)
operator.__ifloordiv__(*a*, *b*)

`a = ifloordiv(a, b)` é equivalente a `a //= b`.

operator.ilshift(*a*, *b*)
operator.__ilshift__(*a*, *b*)

`a = ilshift(a, b)` é equivalente a `a <<= b`.

operator.imod(*a*, *b*)
operator.__imod__(*a*, *b*)

`a = imod(a, b)` é equivalente a `a %= b`.

operator.imul(*a*, *b*)
operator.__imul__(*a*, *b*)

`a = imul(a, b)` é equivalente a `a *= b`.

operator.imatmul(*a*, *b*)
operator.__imatmul__(*a*, *b*)

`a = imatmul(a, b)` é equivalente a `a @= b`.

Adicionado na versão 3.5.

operator.ior(*a*, *b*)
operator.__ior__(*a*, *b*)

`a = ior(a, b)` é equivalente a `a |= b`.

operator.ipow(*a*, *b*)
operator.__ipow__(*a*, *b*)

`a = ipow(a, b)` é equivalente a `a **= b`.

operator.irshift(*a*, *b*)
operator.__irshift__(*a*, *b*)

`a = irshift(a, b)` é equivalente a `a >>= b`.

operator.isub(*a*, *b*)
operator.__isub__(*a*, *b*)

`a = isub(a, b)` é equivalente a `a -= b`.

operator.itruediv(*a*, *b*)
operator.__itruediv__(*a*, *b*)

`a = itruediv(a, b)` é equivalente a `a /= b`.

operator.ixor(*a*, *b*)
operator.__ixor__(*a*, *b*)

`a = ixor(a, b)` é equivalente a `a ^= b`.

Acesso a arquivos e diretórios

Os módulos descritos neste capítulo dizem respeito aos arquivos e diretórios no disco. Por exemplo, existem módulos para ler as propriedades dos arquivos, manipular o caminhos de forma multiplataforma e para criar arquivos temporários. A lista completa de módulos neste capítulo é:

- `pathlib` — Caminhos do sistema de arquivos orientados a objetos
  - Uso básico
  - Exceções
  - Caminhos puros
    - Propriedades gerais
    - Operadores
    - Acessando partes individuais
    - Métodos e propriedades
  - Caminhos concretos
    - Analisando e gerando URIs
    - Expandindo e resolvendo caminhos
    - Consultando tipo de arquivo e status
    - Lendo e escrevendo arquivos
    - Lendo diretórios
    - Criando arquivos e diretórios
    - Copiando, movendo e excluindo
    - Permissões e propriedade
  - Linguagem de padrões
  - Comparação com o módulo `glob`
  - Comparação com os módulos `os` e `os.path`
    - Ferramentas correspondentes
  - Protocolos
- `os.path` — Manipulações comuns de nomes de caminhos
- `stat` — Interpretando resultados de `stat()`
- `filecmp` — Comparações de arquivos e diretórios
  - A classe `dircmp`
- `tempfile` — Gera arquivos temporários e diretórios
  - Exemplos

    - Deprecated functions and variables
- `glob` — Expansão de padrão de nome de arquivo no estilo Unix
    - Exemplos
- `fnmatch` — Correspondência de padrões de nome de arquivo Unix
- `linecache` — Acesso aleatório a linhas de texto
- `shutil` — Operações de arquivo de alto nível
    - Operações de diretório e arquivos
        - Operações de cópia eficientes dependentes da plataforma
        - Exemplo de copytree
        - exemplo rmtree
    - Operações de arquivamento
        - Exemplo de arquivo
        - Exemplo de arquivamento com *base_dir*
    - Consultando o tamanho do terminal de saída

Ver também

Módulo `os`

Interfaces do sistema operacional, incluindo funções para trabalhar com arquivos num nível inferior a objetos arquivos do Python.

Módulo `io`

A biblioteca embutida de E/S do Python, incluindo as classes abstratas e algumas classes concretas, como E/S de arquivos.

Função embutida `open()`

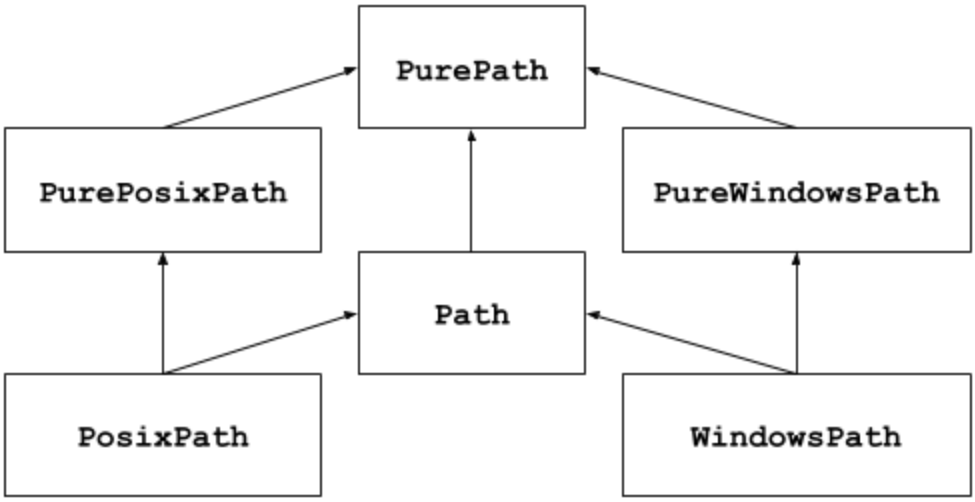
A maneira padrão de abrir arquivos para ler e escrever em Python.

`pathlib` — Caminhos do sistema de arquivos orientados a objetos

Adicionado na versão 3.4.

Código-fonte: Lib/pathlib/ [https://github.com/python/cpython/tree/3.15/Lib/pathlib/]

Este módulo oferece classes que representam caminhos de sistema de arquivos com semântica apropriada para diferentes sistemas operacionais. As classes de caminho são divididas entre caminhos puros, que fornecem operações puramente computacionais sem E/S, e caminhos concretos, que herdam de caminhos puros, mas também fornecem operações de E/S.



Se você nunca usou este módulo antes ou apenas não tem certeza de qual classe é a certa para sua tarefa, provavelmente `Path` é o que você precisa. Esta classe instancia um caminho concreto para a plataforma em que o código está sendo executado.

Caminhos puros são úteis em alguns casos especiais. Por exemplo:

1. Se você deseja manipular os caminhos do Windows em uma máquina Unix (ou vice-versa). Você não pode instanciar `WindowsPath` quando está no Unix, mas você pode instanciar `PureWindowsPath`.
2. Você quer ter certeza de que seu código apenas manipula caminhos, sem realmente acessar o sistema operacional. Nesse caso, instanciar uma das classes puras pode ser útil, pois elas simplesmente não têm nenhuma operação de acesso ao sistema operacional.

Ver também

PEP 428 [https://peps.python.org/pep-0428/]: O módulo pathlib – caminhos de sistema de arquivos orientados a objetos.

Ver também

Para manipulação de caminho de baixo nível em strings, você também pode usar o módulo `os.path`.

Uso básico

Importando a classe principal:

```
>>> from pathlib import Path
```

Listando os subdiretórios:

```
>>> p = Path('.')
>>> [x for x in p.iterdir() if x.is_dir()]
[PosixPath('.hg'), PosixPath('docs'), PosixPath('dist'),
 PosixPath('__pycache__'), PosixPath('build')]
```

Listando os arquivos fontes do Python e sua árvore de diretórios:

```
>>> list(p.glob('**/*.py'))
[PosixPath('test_pathlib.py'), PosixPath('setup.py'),
 PosixPath('pathlib.py'), PosixPath('docs/conf.py'),
 PosixPath('build/lib/pathlib.py')]
```

Navegando dentro da árvore de diretórios:

```
>>> p = Path('/etc')
>>> q = p / 'init.d' / 'reboot'
>>> q
PosixPath('/etc/init.d/reboot')
>>> q.resolve()
PosixPath('/etc/rc.d/init.d/halt')
```

Consultando as propriedades do path:

```
>>> q.exists()
True
>>> q.is_dir()
False
```

Abrindo um arquivo:

```
>>> with q.open() as f: f.readline()
...
'#!/bin/bash\n'
```

Exceções

exception pathlib.UnsupportedOperation

Uma exceção herdada de `NotImplementedError` que é levantada quando uma operação não suportada é chamada em um objeto caminho.

Adicionado na versão 3.13.

Caminhos puros

Objetos de caminho puro fornecem operações de manipulação de caminho que, na verdade, não acessam um sistema de arquivos. Existem três

maneiras de acessar essas classes, que também chamamos de *sabores*:

class pathlib.PurePath(**pathsegments*)

Uma classe genérica que representa o tipo de caminho do sistema (ao instanciá-la, cria-se uma **PurePosixPath** ou uma **PureWindowsPath**):

```
>>> PurePath('setup.py')      # Executando em uma máquina Uni
PurePosixPath('setup.py')
```

Cada elemento de *pathsegments* pode ser uma string representando um segmento de caminho, um objeto que implementa a interface que retorna uma string ou um objeto implementando a interface **os.PathLike**, onde o método **__fspath__()** retorna uma string, como outro objeto caminho:

```
>>> PurePath('foo', 'algum/caminho', 'bar')
PurePosixPath('foo/algum/caminho/bar')
>>> PurePath(Path('foo'), Path('bar'))
PurePosixPath('foo/bar')
```

Quando *pathsegments* está vazio ou consiste am apenas strings vazias, o diretório atual é presumido:

```
>>> PurePath(), PurePath('')
(PurePosixPath('.'), PurePosixPath('.'))
```

Se um segmento for um caminho absoluto, todos os segmentos anteriores serão ignorados (como **os.path.join()**):

```
>>> PurePath('/etc', '/usr', 'lib64')
PurePosixPath('/usr/lib64')
>>> PureWindowsPath('c:/Windows', 'd:bar')
PureWindowsPath('d:bar')
```

No Windows, a unidade não é redefinida quando um segmento de caminho relativo enraizado (por exemplo, `r'\foo'`) é encontrado:

```
>>> PureWindowsPath('c:/Windows', '/Program Files')
PureWindowsPath('c:/Program Files')
```

Barras espúrias e pontos simples são recolhidos, mas pontos duplos (`'..'`) e barras duplas iniciais (`'//'`) não são, pois isso mudaria o significado de um caminho por vários motivos (por exemplo, links simbólicos, caminhos UNC):

```
>>> PurePath('foo//bar')
PurePosixPath('foo/bar')
>>> PurePath('//foo/bar')
PurePosixPath('//foo/bar')
>>> PurePath('foo/./bar')
PurePosixPath('foo/bar')
>>> PurePath('foo/../bar')
PurePosixPath('foo/../bar')
```

(uma abordagem ingênua seria criar `PurePosixPath('foo/../bar')` equivalente a `PurePosixPath('bar')`, o que é errado se `foo` for um link simbólico para outro diretório)

Objetos caminho puro implementam a interface **`os.PathLike`**, permitindo que sejam usados em qualquer lugar em que a interface seja aceita.

Alterado na versão 3.6: Adicionado suporte para a interface **`os.PathLike`**.

class pathlib.PurePosixPath(**pathsegments*)

Uma subclasse de **`PurePath`**, este tipo de caminho representa caminhos de sistema de arquivos não Windows:

```
>>> PurePosixPath('/etc/hosts')
PurePosixPath('/etc/hosts')
```

pathsegments é especificado de forma similar para **`PurePath`**.

class pathlib.PureWindowsPath(**pathsegments*)

Uma subclasse de **`PurePath`**, este tipo de caminho representa os caminhos do sistema de arquivos do Windows, incluindo Caminhos UNC [https://en.wikipedia.org/wiki/Path_(computing)#UNC]:

```
>>> PureWindowsPath('c:/', 'Users', 'Ximénez')
PureWindowsPath('c:/Users/Ximénez')
>>> PureWindowsPath('//servidor/compartilhamento/arquivo')
PureWindowsPath('//servidor/compartilhamento/arquivo')
```

pathsegments é especificado de forma similar para **`PurePath`**.

Independentemente do sistema em que você está usando, você pode instanciar todas essas classes, uma vez que elas não fornecem nenhuma operação que faça chamadas de sistema.

Propriedades gerais

Os caminhos são imutáveis e hasheáveis. Os caminhos do mesmo sabor são comparáveis e ordenáveis. Essas propriedades respeitam a semântica de caixa alta e baixa do sabor:

```
>>> PurePosixPath('foo') == PurePosixPath('FOO')
False
>>> PureWindowsPath('foo') == PureWindowsPath('FOO')
True
>>> PureWindowsPath('FOO') in { PureWindowsPath('foo') }
True
>>> PureWindowsPath('C:') < PureWindowsPath('d:')
True
```

Caminhos de um sabor diferente são comparados de forma desigual e não podem ser ordenados:

```
>>> PureWindowsPath('foo') == PurePosixPath('foo')
False
>>> PureWindowsPath('foo') < PurePosixPath('foo')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: '<' not supported between instances of 'PureWindowsPa
```

Operadores

O operador barra ajuda a criar caminhos filhos, como os.path.join(). Se o argumento for um caminho absoluto, o caminho anterior será ignorado. No Windows, a unidade não é redefinida quando o argumento é um caminho relativo enraizado (por exemplo, `r'\foo'`):

```
>>> p = PurePath('/etc')
>>> p
PurePosixPath('/etc')
>>> p / 'init.d' / 'apache2'
PurePosixPath('/etc/init.d/apache2')
>>> q = PurePath('bin')
>>> '/usr' / q
PurePosixPath('/usr/bin')
>>> p / '/um_caminho_absoluto'
PurePosixPath('/um_caminho_absoluto')
>>> PureWindowsPath('c:/Windows', '/Program Files')
PureWindowsPath('c:/Program Files')
```

Um objeto de caminho pode ser usado em qualquer lugar em que um objeto implementando os.PathLike seja aceito:

```
>>> import os
>>> p = PurePath('/etc')
>>> os.fspath(p)
'/etc'
```

A representação de string de um caminho é o próprio caminho do sistema de arquivos bruto (na forma nativa, por exemplo, com contrabarras no Windows), que você pode passar para qualquer função usando um caminho de arquivo como uma string:

```
>>> p = PurePath('/etc')
>>> str(p)
'/etc'
>>> p = PureWindowsPath('c:/Program Files')
>>> str(p)
'c:\\Program Files'
```

Da mesma forma, chamar bytes em um caminho fornece o caminho do sistema de arquivos bruto como um objeto bytes, codificado por os.fsencode():

```
>>> bytes(p)
b'/etc'
```

Nota

A chamada de `bytes` só é recomendada no Unix. No Windows, a forma Unicode é a representação canônica dos caminhos do sistema de arquivos.

Acessando partes individuais

Para acessar as “partes” individuais (componentes) de um caminho, use a seguinte propriedade:

PurePath.parts

Uma tupla que dá acesso aos vários componentes do caminho:

```
>>> p = PurePath('/usr/bin/python3')
>>> p.parts
('/', 'usr', 'bin', 'python3')

>>> p = PureWindowsPath('c:/Program Files/PSF')
>>> p.parts
('c:\\', 'Program Files', 'PSF')
```

(observe como a unidade e a raiz local são reagrupadas em uma única parte)

Métodos e propriedades

Caminhos puros fornecem os seguintes métodos e propriedades:

PurePath.parser

A implementação do módulo `os.path` usado para análise e junção de caminhos de baixo nível: **`posixpath`** ou **`ntpath`**.

Adicionado na versão 3.13.

PurePath.drive

Uma string que representa a letra ou nome da unidade, se houver:

```
>>> PureWindowsPath('c:/Program Files/').drive
'c:'
>>> PureWindowsPath('/Program Files/').drive
''
>>> PurePosixPath('/etc').drive
''
```

Os compartilhamentos UNC também são considerados unidades:

```
>>> PureWindowsPath('//host/compartilhamento/foo.txt').drive
'\\\\host\\compartilhamento'
```

PurePath.root

Uma string que representa a raiz (local ou global), se houver:

```
>>> PureWindowsPath('c:/Program Files/').root
'\\'
>>> PureWindowsPath('c:Program Files/').root
''
>>> PurePosixPath('/etc').root
'/'
```

Os compartilhamentos UNC sempre têm uma raiz:

```
>>> PureWindowsPath('//host/compartilhamento').root
'\\'
```

Se o caminho começa com mais de duas barras sucessivas, **PurePosixPath** as recolhe:

```
>>> PurePosixPath('//etc').root
'//'
>>> PurePosixPath('///etc').root
'/'
>>> PurePosixPath('////etc').root
'/'
```

Nota

Este comportamento está em conformidade com o parágrafo 4.11 Pathname Resolution [https://pubs.opengroup.org/onlinepubs/009695399/basedefs/xbd_chap04.html#tag_04_11] do *The Open Group Base Specifications Issue 6*:

"Um nome de caminho que começa com duas barras sucessivas pode ser interpretado de maneira definida pela implementação, embora mais de duas barras iniciais devam ser tratadas como uma única barra."

PurePath.anchor

A concatenação da unidade e da raiz:

```
>>> PureWindowsPath('c:/Program Files/').anchor
'c:\\'
>>> PureWindowsPath('c:Program Files/').anchor
'c:'
>>> PurePosixPath('/etc').anchor
'/'
>>> PureWindowsPath('//host/compartilhamento').anchor
'\\\\host\\compartilhamento\\'
```

PurePath.parents

Uma sequência imutável que fornece acesso aos ancestrais lógicos do caminho:

```
>>> p = PureWindowsPath('c:/foo/bar/setup.py')
>>> p.parents[0]
PureWindowsPath('c:/foo/bar')
>>> p.parents[1]
PureWindowsPath('c:/foo')
>>> p.parents[2]
PureWindowsPath('c:/')
```

Alterado na versão 3.10: A sequência pai agora tem suporte fatias e valores de índice negativos.

PurePath.parent

O pai lógico do caminho:

```
>>> p = PurePosixPath('/a/b/c/d')
>>> p.parent
PurePosixPath('/a/b/c')
```

Você não pode passar por uma âncora ou caminho vazio:

```
>>> p = PurePosixPath('/')
>>> p.parent
PurePosixPath('/')
>>> p = PurePosixPath('.')
>>> p.parent
PurePosixPath('.')
```

Nota

Esta é uma operação puramente lexical, daí o seguinte comportamento:

```
>>> p = PurePosixPath('foo/..')
>>> p.parent
PurePosixPath('foo')
```

Se você deseja percorrer um caminho arbitrário do sistema de arquivos para cima, é recomendável primeiro chamar `Path.resolve()` para resolver os links simbólicos e eliminar os componentes `"..."`.

PurePath.name

Uma string que representa o componente do caminho final, excluindo a unidade e a raiz, se houver:

```
>>> PurePosixPath('minha/biblioteca/setup.py').name
'setup.py'
```

Nomes de unidades UNC não são considerados::

```
>>> PureWindowsPath('//algum/compartilhamento/setup.py').name
'setup.py'
>>> PureWindowsPath('//algum/compartilhamento').name
''
```

PurePath.suffix

A última parte separada por pontos do componente final, se houver:

```
>>> PurePosixPath('minha/biblioteca/setup.py').suffix
'.py'
>>> PurePosixPath('minha/biblioteca.tar.gz').suffix
'.gz'
>>> PurePosixPath('minha/biblioteca').suffix
''
```

Isso é comumente chamado de extensão de arquivo.

Alterado na versão 3.14: Um único ponto (”`.`”) é considerado um sufixo válido.

PurePath.suffixes

Uma lista dos sufixos do caminho, com frequência chamados de extensões de arquivo:

```
>>> PurePosixPath('minha/biblioteca.tar.gar').suffixes
['.tar', '.gar']
>>> PurePosixPath('minha/biblioteca.tar.gz').suffixes
['.tar', '.gz']
>>> PurePosixPath('minha/biblioteca').suffixes
[]
```

Alterado na versão 3.14: Um único ponto (”`.`”) é considerado um sufixo válido.

PurePath.stem

O componente final do caminho, sem seu sufixo:

```
>>> PurePosixPath('minha/biblioteca.tar.gz').stem
'library.tar'
>>> PurePosixPath('minha/biblioteca.tar').stem
'library'
>>> PurePosixPath('minha/biblioteca').stem
'library'
```

Alterado na versão 3.14: Um único ponto (”`.`”) é considerado um sufixo válido.

PurePath.as_posix()

Retorna uma representação de string do caminho com barras (`/`):

```
>>> p = PureWindowsPath('c:\\windows')
>>> str(p)
'c:\\windows'
>>> p.as_posix()
'c:/windows'
```

PurePath.is_absolute()

Retorna se o caminho é absoluto ou não. Um caminho é considerado absoluto se tiver uma raiz e (se o tipo permitir) uma unidade:

```
>>> PurePosixPath('/a/b').is_absolute()
True
>>> PurePosixPath('a/b').is_absolute()
False

>>> PureWindowsPath('c:/a/b').is_absolute()
True
>>> PureWindowsPath('/a/b').is_absolute()
False
>>> PureWindowsPath('c:').is_absolute()
False
>>> PureWindowsPath('//some/share').is_absolute()
True
```

PurePath.is_relative_to(*other*)

Retorna se este caminho é ou não relativo a outro caminho, representado por *other*.

```
>>> p = PurePath('/etc/passwd')
>>> p.is_relative_to('/etc')
True
>>> p.is_relative_to('/usr')
False
```

Este método é baseado em string; ele não acessa o sistema de arquivos nem trata segmentos “. .” especialmente. O código a seguir é equivalente:

```
>>> u = PurePath('/usr')
>>> u == p or u in p.parents
False
```

Adicionado na versão 3.9.

Descontinuado desde a versão 3.12, removido na versão 3.14: A passagem de argumentos adicionais está descontinuada; se fornecidos, eles são unidos com *other*.

PurePath.joinpath(**pathsegments*)

Chamar este método é equivalente a combinar o caminho com cada um dos *pathsegments* por vez:

```
>>> PurePosixPath('/etc').joinpath('passwd')
PurePosixPath('/etc/passwd')
>>> PurePosixPath('/etc').joinpath(PurePosixPath('passwd'))
PurePosixPath('/etc/passwd')
>>> PurePosixPath('/etc').joinpath('init.d', 'apache2')
PurePosixPath('/etc/init.d/apache2')
>>> PureWindowsPath('c:').joinpath('/Program Files')
PureWindowsPath('c:/Program Files')
```

PurePath.full_match(*pattern*, *, *case_sensitive=None*)

Compara esse caminho com o padrão de estilo glob fornecido. Retorna `True` se a correspondência for bem-sucedida, `False` caso contrário. Por exemplo:

```
>>> PurePath('a/b.py').full_match('a/*.py')
True
>>> PurePath('a/b.py').full_match('*.py')
False
>>> PurePath('/a/b/c.py').full_match('/a/**')
True
>>> PurePath('/a/b/c.py').full_match('**/*.py')
True
```

Ver também

Documentação de Linguagem de padrões.

Tal como acontece com outros métodos, a distinção entre maiúsculas e minúsculas segue os padrões da plataforma:

```
>>> PurePosixPath('b.py').full_match('*.PY')
False
>>> PureWindowsPath('b.py').full_match('*.PY')
True
```

Defina *case_sensitive* como `True` ou `False` para substituir esse comportamento.

Adicionado na versão 3.13.

PurePath.match(*pattern*, *, *case_sensitive=None*)

Compara esse caminho com o padrão no recursivo de estilo glob fornecido. Retorna `True` se a correspondência for bem-sucedida, `False` caso contrário.

Este método é semelhante a `full_match()`, mas padrões vazios não são permitidos (`ValueError` é levantada), o curinga recursivo "`**`" não é suportado (ele age como o "`*`" não recursivo) e, se um padrão relativo for fornecido, a correspondência será feita da direita:

```
>>> PurePath('a/b.py').match('*.py')
True
>>> PurePath('/a/b/c.py').match('b/*.py')
True
>>> PurePath('/a/b/c.py').match('a/*.py')
False
```

Alterado na versão 3.12: O parâmetro *pattern* aceita um objeto caminho ou similar.

Alterado na versão 3.12: O parâmetro *case_sensitive* foi adicionado.

PurePath.relative_to(*other*, *walk_up=False*)

Calcula uma versão deste caminho em relação ao caminho representado por *other*. Se for impossível, `ValueError` é levantada:

```
>>> p = PurePosixPath('/etc/passwd')
>>> p.relative_to('/')
PurePosixPath('etc/passwd')
>>> p.relative_to('/etc')
PurePosixPath('passwd')
>>> p.relative_to('/usr')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 941, in relative_to
    raise ValueError(error_message.format(str(self), str(form
ValueError: '/etc/passwd' is not in the subpath of '/usr' OR
```

Quando *walk_up* é falso (o padrão), o caminho deve começar com *other*. Quando o argumento é verdadeiro, entradas `..` podem ser adicionadas para formar o caminho relativo. Em todos os outros casos, como os caminhos que fazem referência a unidades diferentes, `ValueError` é levantada.:

```
>>> p.relative_to('/usr', walk_up=True)
PurePosixPath('../etc/passwd')
>>> p.relative_to('foo', walk_up=True)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 941, in relative_to
    raise ValueError(error_message.format(str(self), str(form
ValueError: '/etc/passwd' is not on the same drive as 'foo' (
```

Aviso

Esta função faz parte de `PurePath` e funciona com strings. Ela não verifica ou acessa a estrutura do arquivo subjacente. Isso pode impactar a opção *walk_up*, pois ela presume que não há links simbólicos presentes no caminho; chame `resolve()` primeiro se necessário para resolver links simbólicos.

Alterado na versão 3.12: O parâmetro *walk_up* foi adicionado (o comportamento antigo é o mesmo de `walk_up=False`).

Descontinuado desde a versão 3.12, removido na versão 3.14: A passagem de argumentos posicionais adicionais está descontinuada; se fornecidos, eles são unidos com *other*.

PurePath.with_name(*name*)

Retorna um novo caminho com o `name` alterado. Se o caminho original não tiver um nome, ValueError é levantada:

```
>>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz')
>>> p.with_name('setup.py')
PureWindowsPath('c:/Downloads/setup.py')
>>> p = PureWindowsPath('c:/')
>>> p.with_name('setup.py')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/antoine/cpython/default/Lib/pathlib.py", line 7
    raise ValueError("%r has an empty name" % (self,))
ValueError: PureWindowsPath('c:/') has an empty name
```

PurePath.with_stem(*stem*)

Retorna um novo caminho com o `stem` alterado. Se o caminho original não tiver um nome, ValueError é levantada:

```
>>> p = PureWindowsPath('c:/Downloads/draft.txt')
>>> p.with_stem('final')
PureWindowsPath('c:/Downloads/final.txt')
>>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz')
>>> p.with_stem('lib')
PureWindowsPath('c:/Downloads/lib.gz')
>>> p = PureWindowsPath('c:/')
>>> p.with_stem('')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/home/antoine/cpython/default/Lib/pathlib.py", line 8
    return self.with_name(stem + self.suffix)
  File "/home/antoine/cpython/default/Lib/pathlib.py", line 8
    raise ValueError("%r has an empty name" % (self,))
ValueError: PureWindowsPath('c:/') has an empty name
```

Adicionado na versão 3.9.

PurePath.with_suffix(*suffix*)

Retorna um novo caminho com o **suffix** alterado. Se o caminho original não tiver um sufixo, o novo *suffixo* será anexado. Se o *suffix* for uma string vazia, o sufixo original será removido:

```
>>> p = PureWindowsPath('c:/Downloads/pathlib.tar.gz')
>>> p.with_suffix('.bz2')
PureWindowsPath('c:/Downloads/pathlib.tar.bz2')
>>> p = PureWindowsPath('README')
>>> p.with_suffix('.txt')
PureWindowsPath('README.txt')
>>> p = PureWindowsPath('README.txt')
>>> p.with_suffix('')
PureWindowsPath('README')
```

Alterado na versão 3.14: Um único ponto (”.”) é considerado um sufixo válido. Em versões anteriores, **ValueError** era levantada se um único ponto fosse fornecido.

PurePath.with_segments(**pathsegments*)

Cria um novo objeto caminho do mesmo tipo combinando os *pathsegments* fornecidos. Este método é chamado sempre que um caminho derivado é criado, como de **parent** e **relative_to()**. Subclasses podem substituir este método para passar informações para caminhos derivados, por exemplo:

```
from pathlib import PurePosixPath

class MyPath(PurePosixPath):
    def __init__(self, *pathsegments, session_id):
        super().__init__(*pathsegments)
        self.session_id = session_id

    def with_segments(self, *pathsegments):
        return type(self)(*pathsegments, session_id=self.sess

etc = MyPath('/etc', session_id=42)
```

```
hosts = etc / 'hosts'
print(hosts.session_id)  # 42
```

Adicionado na versão 3.12.

Caminhos concretos

Caminhos concretos são subclasses das classes de caminho puro. Além das operações fornecidas por este último, eles também fornecem métodos para fazer chamadas de sistema em objetos de caminho. Existem três maneiras de instanciar caminhos concretos:

class pathlib.Path(**pathsegments*)

Uma subclase de `PurePath`, esta classe representa caminhos concretos do tipo de caminho do sistema (instanciando-o cria uma `PosixPath` ou uma `WindowsPath`):

```
>>> Path('setup.py')
PosixPath('setup.py')
```

pathsegments é especificado de forma similar para `PurePath`.

class pathlib.PosixPath(**pathsegments*)

Uma subclase de `Path` e `PurePosixPath`, esta classe representa caminhos concretos de sistemas de arquivos não-Windows:

```
>>> PosixPath('/etc/hosts')
PosixPath('/etc/hosts')
```

pathsegments é especificado de forma similar para `PurePath`.

Alterado na versão 3.13: Levanta `UnsupportedOperation` no Windows. Em versões anteriores, `NotImplementedError` era levantada em vez disso.

class pathlib.WindowsPath(**pathsegments*)

Uma subclasse de `Path` e `PureWindowsPath`, esta classe representa caminhos concretos de sistemas de arquivos do Windows:

```
>>> WindowsPath('c:/', 'Users', 'Ximénez')
WindowsPath('c:/Users/Ximénez')
```

pathsegments é especificado de forma similar para `PurePath`.

Alterado na versão 3.13: Levanta `UnsupportedOperation` em plataformas não-Windows. Em versões anteriores, `NotImplementedError` era levantada em vez disso.

Você só pode instanciar o tipo de classe que corresponde ao seu sistema (permitir chamadas de sistema em tipos de caminho não compatíveis pode levar a bugs ou falhas em sua aplicação):

```
>>> import os
>>> os.name
'posix'
>>> Path('setup.py')
PosixPath('setup.py')
>>> PosixPath('setup.py')
PosixPath('setup.py')
>>> WindowsPath('setup.py')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "pathlib.py", line 798, in __new__
    % (cls.__name__,))
UnsupportedOperation: cannot instantiate 'WindowsPath' on your s
```

Alguns métodos de caminho concretos podem levantar um `OSError` se uma chamada de sistema falhar (por exemplo, porque o caminho não existe).

Analisando e gerando URIs

Objetos caminho concretos podem ser criados e representados como URIs de ‘file’ em conformidade com **RFC 8089** [https://datatracker.ietf.org/doc/html/rfc8089.html].

Nota

URIs de arquivos não são portáteis entre máquinas com diferentes codificações de sistema de arquivos.

classmethod Path.from_uri(*uri*)

Retorna um novo objeto de caminho a partir da análise sintática de um URI ‘file’. Por exemplo:

```
>>> p = Path.from_uri('file:///etc/hosts')
PosixPath('/etc/hosts')
```

No Windows, os caminhos do dispositivo DOS e UNC podem ser analisados a partir de URIs:

```
>>> p = Path.from_uri('file:///c:/windows')
WindowsPath('c:/windows')
>>> p = Path.from_uri('file://servidor/compartilhamento')
WindowsPath('//servidor/compartilhamento')
```

Várias formas variantes são suportadas:

```
>>> p = Path.from_uri('file:////servidor/compartilhamento')
WindowsPath('//servidor/compartilhamento')
>>> p = Path.from_uri('file://///servidor/compartilhamento')
WindowsPath('//servidor/compartilhamento')
>>> p = Path.from_uri('file:c:/windows')
WindowsPath('c:/windows')
>>> p = Path.from_uri('file:/c|/windows')
WindowsPath('c:/windows')
```

`ValueError` é levantada se o URI não começar com `file:` ou se o caminho analisado não for absoluto.

Adicionado na versão 3.13.

Alterado na versão 3.14: A autoridade de URL é descartada se corresponder ao nome do host local. Caso contrário, se a autoridade não estiver vazia ou for `localhost`, no Windows um caminho UNC será

retornado (como antes) e, em outras plataformas, um `ValueError` será levantada.

Path.as_uri()

Representa o caminho como um URI ‘file’. `ValueError` é levantada se o caminho não for absoluto.

```
>>> p = PosixPath('/etc/passwd')
>>> p.as_uri()
'file:///etc/passwd'
>>> p = WindowsPath('c:/Windows')
>>> p.as_uri()
'file:///c:/Windows'
```

Descontinuado desde a versão 3.14, será removido na versão 3.19: Chamar este método de `PurePath` em vez de `Path` é possível, mas está descontinuado. O uso de `os.fsencode()` pelo método o torna estritamente impuro.

Expandindo e resolvendo caminhos

classmethod Path.home()

Retorna um novo objeto caminho representando o diretório pessoal do usuário (conforme retornado por `os.path.expanduser()` com a construção ~). Se o diretório pessoal não puder ser resolvido, `RuntimeError` é levantada.

```
>>> Path.home()
PosixPath('/home/antoine')
```

Adicionado na versão 3.5.

Path.expanduser()

Retorna um novo caminho com as construções ~ e `~user` expandidas, como retornado por `os.path.expanduser()`. Se um diretório pessoal não puder ser resolvido, `RuntimeError` é levantada.

```
>>> p = PosixPath('~/filmes/Monty Python')
>>> p.expanduser()
```

```
PosixPath('/home/eric/filmes/Monty Python')
```

Adicionado na versão 3.5.

classmethod Path.cwd()

Retorna um novo objeto caminho que representa o diretório atual (conforme retornado por `os.getcwd()`):

```
>>> Path.cwd()
PosixPath('/home/antoine/pathlib')
```

Path.absolute()

Torna o caminho absoluto, sem normalização ou resolução de links simbólicos. Retorna um novo objeto de caminho:

```
>>> p = Path('tests')
>>> p
PosixPath('tests')
>>> p.absolute()
PosixPath('/home/antoine/pathlib/tests')
```

Path.resolve(*strict=False*)

Torna o caminho absoluto, resolvendo quaisquer links simbólicos. Um novo objeto de caminho é retornado:

```
>>> p = Path()
>>> p
PosixPath('.')
>>> p.resolve()
PosixPath('/home/antoine/pathlib')
```

Componentes "`..`" também são eliminados (este é o único método para fazer isso):

```
>>> p = Path('docs/../setup.py')
>>> p.resolve()
PosixPath('/home/antoine/pathlib/setup.py')
```

Se um caminho não existe ou um laço de link simbólico é encontrado, e *strict* é `True`, `OSError` é levantada. Se *strict* for `False`, o caminho será

resolvido tanto quanto possível e qualquer resto é anexado sem verificar se existe.

Alterado na versão 3.6: O parâmetro *strict* foi adicionado (comportamento pré-3.6 é estrito).

Alterado na versão 3.13: Os laços em link simbólico são tratados como outros erros: `OSError` é levantada no modo estrito, e nenhuma exceção é levantada no modo não estrito. Em versões anteriores, `RuntimeError` é levantada não importando o valor de *strict*.

Path.readlink()

Retorna o caminho para o qual o link simbólico aponta (conforme retornado por `os.readlink()`):

```
>>> p = Path('meulink')
>>> p.symlink_to('setup.py')
>>> p.readlink()
PosixPath('setup.py')
```

Adicionado na versão 3.9.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se `os.readlink()` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Consultando tipo de arquivo e status

Alterado na versão 3.8: `exists()`, `is_dir()`, `is_file()`, `is_mount()`, `is_symlink()`, `is_block_device()`, `is_char_device()`, `is_fifo()`, `is_socket()` agora retornam `False` em vez de levantar uma exceção para caminhos que contêm caracteres não representáveis no nível do sistema operacional.

Alterado na versão 3.14: Os métodos fornecidos acima agora retornam `False` em vez de levantar qualquer exceção `OSError` do sistema operacional. Em versões anteriores, alguns tipos de exceção **`OSError`** eram levantados e outros suprimidos. O novo comportamento é consistente com

`os.path.exists()`, `os.path.isdir()`, etc. Use `stat()` para recuperar o status do arquivo sem suprimir exceções.

Path.stat(*, *follow_symlinks=True*)

Retorna um objeto `os.stat_result` contendo informações sobre este caminho, como `os.stat()`. O resultado é consultado em cada chamada para este método.

Este método normalmente segue links simbólicos; para obter estado de um link simbólico, adicione o argumento `follow_symlinks=False`, ou use `lstat()`.

```
>>> p = Path('setup.py')
>>> p.stat().st_size
956
>>> p.stat().st_mtime
1327883547.852554
```

Alterado na versão 3.10: O parâmetro *follow_symlinks* foi adicionado.

Path.lstat()

Como `Path.stat()`, mas, se o caminho apontar para um link simbólico, retorna as informações do link simbólico ao invés de seu alvo.

Path.exists(*, *follow_symlinks=True*)

Retorna `True` se o caminho apontar para um arquivo ou diretório existente e `False` se o caminho for inválido, inacessível ou ausente. Use `Path.stat()` para distinguir entre esses casos.

Este método normalmente segue links simbólicos; para verificar se um link simbólico existe, adicione o argumento `follow_symlinks=False`.

```
>>> Path('').exists()  # O diretório atual.
True
>>> Path('.').exists()
True
>>> Path('setup.py').exists()
True
>>> Path('/etc').exists()
```

```
True
>>> Path('nonexistentfile').exists()
False
```

Alterado na versão 3.12: O parâmetro *follow_symlinks* foi adicionado.

Path.is_file(*, *follow_symlinks=True*)

Retorna `True` se o caminho apontar para um arquivo comum. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um arquivo comum. Use **`Path.stat()`** para distinguir entre esses casos.

Este método normalmente segue links simbólicos; para excluir links simbólicos, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.is_dir(*, *follow_symlinks=True*)

Retorna `True` se o caminho apontar para um diretório. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um diretório. Use **`Path.stat()`** para distinguir entre esses casos.

Este método normalmente segue links simbólicos; para excluir links simbólicos para diretórios, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.is_symlink()

Retorna `True` se o caminho apontar para um link simbólico, ainda que esteja quebrado. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um arquivo comum. Use **`Path.stat()`** para distinguir entre esses casos.

Path.is_junction()

Retorna `True` se o caminho apontar para uma junção, e `False` para qualquer outro tipo de arquivo. Atualmente, apenas o Windows dá

suporte a junções.

Adicionado na versão 3.12.

Path.is_mount()

Retorna `True` se o caminho for um *ponto de montagem*: um ponto em um sistema de arquivos onde um sistema de arquivos diferente foi montado. No POSIX, a função verifica se o pai do *path*, `path/..`, está em um dispositivo diferente de *path*, ou se `path/..` e *path* apontam para o mesmo nó-i no mesmo dispositivo – isso deve detectar pontos de montagem para todas as variantes Unix e POSIX. No Windows, um ponto de montagem é considerado uma raiz de letra de unidade (por exemplo, `c:\`), um compartilhamento UNC (por exemplo, `\\server\share`) ou um diretório de sistema de arquivos montado.

Adicionado na versão 3.7.

Alterado na versão 3.12: Suporte ao Windows foi adicionado.

Path.is_socket()

Retorna `True` se o caminho apontar para um socket Unix. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um socket Unix. Use **`Path.stat()`** para distinguir entre esses casos.

Path.is_fifo()

Retorna `True` se o caminho apontar para um FIFO. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um FIFO. Use **`Path.stat()`** para distinguir entre esses casos.

Path.is_block_device()

Retorna `True` se o caminho apontar para um dispositivo de bloco. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um de bloco. Use **`Path.stat()`** para distinguir entre esses casos.

Path.is_char_device()

Retorna `True` se o caminho apontar para um dispositivo de caractere. `False` será retornado se o caminho for inválido, inacessível ou ausente, ou se apontar para algo diferente de um dispositivo de caractere. Use **`Path.stat()`** para distinguir entre esses casos.

Path.samefile(*other_path*)

Retorna se este path apontar para o mesmo arquivo como *other_path*, que pode ser um objeto PATH ou uma String. A semântica é semelhante a função **`os.path.samefile()`** e a função **`os.path.samestat()`**.

Um **`OSError`** poderá ser levantado caso algum arquivo não puder ser acessado por alguma razão.

```
>>> p = Path('spam')
>>> q = Path('eggs')
>>> p.samefile(q)
False
>>> p.samefile('spam')
True
```

Adicionado na versão 3.5.

Path.info

Um objeto **`PathInfo`** que oferece suporte a consultas de informações sobre o tipo de arquivo. O objeto expõe métodos que armazenam seus resultados em cache, o que pode ajudar a reduzir o número de chamadas de sistema necessárias ao alternar entre tipos de arquivo. Por exemplo:

```
>>> p = Path('src')
>>> if p.info.is_symlink():
...     print('link simbólico')
... elif p.info.is_dir():
...     print('diretório')
... elif p.info.exists():
...     print('alguma coisa mais')
... else:
...     print('não encontrado')
...
diretório
```

Se o caminho foi gerado a partir de **Path.iterdir()**, este atributo é inicializado com algumas informações sobre o tipo de arquivo obtidas a partir da varredura do diretório pai. O simples acesso a **Path.info** não realiza nenhuma consulta ao sistema de arquivos.

Para obter informações atualizadas, é melhor chamar **Path.is_dir()**, **is_file()** e **is_symlink()** em vez dos métodos deste atributo. Não há como redefinir o cache; em vez disso, você pode criar um novo objeto de caminho com um cache de informações vazio via `p = Path(p)`.

Adicionado na versão 3.14.

Lendo e escrevendo arquivos

Path.open(*mode='r', buffering=-1, encoding=None, errors=None, newline=None*)

Abre o arquivo apontado pelo caminho, como a função embutida **open()** faz:

```
>>> p = Path('setup.py')
>>> with p.open() as f:
...     f.readline()
...
'#!/usr/bin/env python3\n'
```

Path.read_text(*encoding=None, errors=None, newline=None*)

Retorna o conteúdo decodificado do arquivo apontado como uma string:

```
>>> p = Path('meu_arquivo_texto')
>>> p.write_text('Conteúdo de arquivo texto')
18
>>> p.read_text()
'Conteúdo de arquivo texto'
```

O arquivo é aberto e, então, fechado. Os parâmetros opcionais têm o mesmo significado que em **open()**.

Adicionado na versão 3.5.

Alterado na versão 3.13: O parâmetro *newline* foi adicionado.

Path.read_bytes()

Retorna o conteúdo binário do arquivo apontado como um objeto bytes:

```
>>> p = Path('meu_arquivo_binário')
>>> p.write_bytes(b'meu_arquivo_binário')
20
>>> p.read_bytes()
b'Binary file contents'
```

Adicionado na versão 3.5.

Path.write_text(*data*, *encoding=None*, *errors=None*, *newline=None*)

Abre o arquivo apontado no modo de texto, escreve *data* e fecha o arquivo:

```
>>> p = Path('meu_arquivo_texto')
>>> p.write_text('Conteúdo de arquivo texto')
18
>>> p.read_text()
'Conteúdo de arquivo texto'
```

Um arquivo existente com o mesmo nome é sobrescrito. Os parâmetros opcionais têm o mesmo significado que em **open()**.

Adicionado na versão 3.5.

Alterado na versão 3.10: O parâmetro *newline* foi adicionado.

Path.write_bytes(*data*)

Abre o arquivo apontado no modo bytes, escreve *data* e fecha o arquivo:

```
>>> p = Path('meu_arquivo_binário')
>>> p.write_bytes(b'meu_arquivo_binário')
20
>>> p.read_bytes()
b'Binary file contents'
```

Um arquivo existente de mesmo nome será substituído.

Adicionado na versão 3.5.

Lendo diretórios

Path.iterdir()

Quando o caminho aponta para um diretório, produz objetos caminho do conteúdo do diretório:

```
>>> p = Path('docs')
>>> for child in p.iterdir(): child
...
PosixPath('docs/conf.py')
PosixPath('docs/_templates')
PosixPath('docs/make.bat')
PosixPath('docs/index.rst')
PosixPath('docs/_build')
PosixPath('docs/_static')
PosixPath('docs/Makefile')
```

Os filhos são gerados em ordem arbitrária e as entradas especiais `'.'` e `'..'` não são incluídas. Se um arquivo for removido ou adicionado ao diretório após a criação do iterador, não está especificado se um objeto caminho para esse arquivo é incluído.

Se o caminho não for um diretório ou estiver inacessível, **`OSError`** será levantada.

Path.glob(*pattern*, *, *case_sensitive=None*, *recurse_symlinks=False*)

Faz o glob do *pattern* relativo fornecido no diretório representado por este caminho, produzindo todos os arquivos correspondentes (de qualquer tipo):

```
>>> sorted(Path('.').glob('*.py'))
[PosixPath('pathlib.py'), PosixPath('setup.py'), PosixPath('t
>>> sorted(Path('.').glob('*/*.py'))
[PosixPath('docs/conf.py')]
>>> sorted(Path('.').glob('**/*.py'))
```

```
[PosixPath('build/lib/pathlib.py'),
 PosixPath('docs/conf.py'),
 PosixPath('pathlib.py'),
 PosixPath('setup.py'),
 PosixPath('test_pathlib.py')]
```

Nota

Os caminhos são retornados sem ordem específica. Se precisar de uma ordem específica, classifique os resultados.

Ver também

Documentação de Linguagem de padrões.

Por padrão, ou quando o argumento somente-nomeado *case_sensitive* é definido como `None`, esse método corresponde a caminhos usando regras de capitalização específicas da plataforma: normalmente, diferencia maiúsculas de minúsculas no POSIX e não o faz no Windows. Defina *case_sensitive* como `True` ou `False` para substituir esse comportamento.

Por padrão, ou quando o argumento somente-nomeado *recurse_symlinks* é definido como `False`, este método segue links simbólicos, exceto ao expandir curingas "`**`". Defina *recurse_symlinks* como `True` para sempre seguir links simbólicos.

Nota

Quaisquer exceções **`OSError`** levantadas durante a varredura do sistema de arquivos são ignoradas. Isso inclui **`PermissionError`** ao acessar diretórios sem permissão de leitura.

Levanta um evento de auditoria do método `pathlib.Path.glob` com os argumentos `self` e `pattern`.

Alterado na versão 3.12: O parâmetro *case_sensitive* foi adicionado.

Alterado na versão 3.13: O parâmetro *recurse_symlinks* foi adicionado.

Alterado na versão 3.13: O parâmetro *pattern* aceita um objeto caminho ou similar.

Alterado na versão 3.13: Quaisquer exceções `OSError` levantadas durante a varredura do sistema de arquivos são ignoradas. Em versões anteriores, tais exceções eram suprimidas em muitos casos, mas não em todos.

Path.rglob(*pattern*, *, *case_sensitive=None*, *recurse_symlinks=False*)

Faz um glob recursivo no *pattern* fornecido. Isso é como chamar `Path.glob()` com "`**/`" adicionado à frente do *pattern*.

Nota

Os caminhos são retornados sem ordem específica. Se precisar de uma ordem específica, classifique os resultados.

Nota

Quaisquer exceções `OSError` levantadas durante a varredura do sistema de arquivos são ignoradas. Isso inclui `PermissionError` ao acessar diretórios sem permissão de leitura.

Ver também

Documentações de Linguagem de padrões e de `Path.glob()`.

Levanta um evento de auditoria `pathlib.Path.rglob` com argumentos `self`, `pattern`.

Alterado na versão 3.12: O parâmetro *case_sensitive* foi adicionado.

Alterado na versão 3.13: O parâmetro *recurse_symlinks* foi adicionado.

Alterado na versão 3.13: O parâmetro *pattern* aceita um objeto caminho ou similar.

Path.walk(*top_down=True*, *on_error=None*, *follow_symlinks=False*)

Gera os nomes de arquivos em uma árvore de diretórios percorrendo a árvore de cima para baixo ou de baixo para cima.

Para cada diretório na árvore de diretórios com raiz em *self* (incluindo *self*, mas excluindo '.' e '..'), o método produz uma tupla de 3 elementos de `(dirpath, dirnames, filenames)`.

dirpath é um **`Path`** para o diretório que está sendo percorrido no momento, *dirnames* é uma lista de strings para os nomes dos subdiretórios em *dirpath* (excluindo `'.'` e `'..'`), e *filenames* é uma lista de strings para os nomes dos arquivos não diretório em *dirpath*. Para obter um caminho completo (que começa com *self*) para um arquivo ou diretório em *dirpath*, faça `dirpath / name`. Se as listas são ou não classificadas depende do sistema de arquivos.

Se o argumento opcional *top_down* for verdadeiro (que é o padrão), o trio para um diretório é gerado antes dos trios para qualquer um de seus subdiretórios (os diretórios são percorridos de cima para baixo). Se *top_down* for falso, o trio para um diretório é gerado após os trios para todos os seus subdiretórios (os diretórios são percorridos de baixo para cima). Não importa o valor de *top_down*, a lista de subdiretórios é recuperada antes que os trios para o diretório e seus subdiretórios sejam percorridos.

Quando *top_down* é verdadeiro, o chamador pode modificar a lista *dirnames* no local (por exemplo, usando **`del`** ou atribuição de fatias), e **`Path.walk()`** só recursivamente nos subdiretórios cujos nomes permanecem em *dirnames*. Isso pode ser usado para podar a pesquisa, ou para impor uma ordem específica de visita, ou mesmo para informar **`Path.walk()`** sobre diretórios que o chamador cria ou renomeia antes de retomar **`Path.walk()`** novamente. Modificar *dirnames* quando *top_down* é false não tem efeito no comportamento de **`Path.walk()`**,

pois os diretórios em *dirnames* já foram gerados no momento em que *dirnames* é cedido ao chamador.

Por padrão, os erros de `os.scandir()` são ignorados. Se o argumento opcional *on_error* for especificado, deve ser um chamável; ela será chamado com um argumento, uma instância da exceção `OSError`. O chamável pode tratar do erro para continuar a caminhada ou levantá-la novamente para parar a caminhada. Observe que o nome do arquivo está disponível como o atributo `filename` do objeto de exceção.

Por padrão, `Path.walk()` não segue links simbólicos e, em vez disso, os adiciona à lista *filenames*. Defina *follow_symlinks* como true para resolver links simbólicos e colocá-los em *dirnames* e *filenames* conforme apropriado para seus alvos e, consequentemente, visitar diretórios apontados por links simbólicos (quando suportado).

Nota

Esteja ciente de que definir *follow_symlinks* como true pode levar a uma recursão infinita se um link apontar para um diretório pai dele mesmo. `Path.walk()` não rastreia os diretórios que já visitou.

Nota

`Path.walk()` presume que os diretórios que percorre não sejam modificados durante a execução. Por exemplo, se um diretório de *dirnames* tiver sido substituído por um link simbólico e *follow_symlinks* for falso, **`Path.walk()`** ainda tentará descer até ele. Para evitar esse comportamento, remova os diretórios de *dirnames* conforme apropriado.

Nota

Diferentemente de `os.walk()`, `Path.walk()` lista links simbólicos para diretórios em *filenames* se *follow_symlinks* for falso.

Esse exemplo mostra o número de bytes usados por todos os arquivos em cada diretório, ignorando diretórios `__pycache__`:

```
from pathlib import Path
for root, dirs, files in Path("cpython/Lib/concurrent").walk(
  print(
      root,
      "consumes",
      sum((root / file).stat().st_size for file in files),
      "bytes in",
      len(files),
      "non-directory files"
  )
  if '__pycache__' in dirs:
        dirs.remove('__pycache__')
```

O próximo exemplo é uma implementação simples de `shutil.rmtree()`. Percorrer a árvore de baixo para cima é essencial, pois `rmdir()` não permite excluir um diretório antes que ele esteja vazio.

```
# Exclui tudo o que estiver acessível no diretório "top".
# CUIDADO: Isso é perigoso! Por exemplo, se top == Path('/'),
# todos os seus arquivos poderão ser excluídos.
for root, dirs, files in top.walk(top_down=False):
    for name in files:
        (root / name).unlink()
    for name in dirs:
        (root / name).rmdir()
```

Adicionado na versão 3.12.

Criando arquivos e diretórios

Path.touch(*mode=0o666*, *exist_ok=True*)

Cria um arquivo neste caminho especifico. Caso o *modo* for dado, ele será combinado com o valor do processo `umask` para determinar o modo de arquivo e as flags de acesso. Se o arquivo já existir, a função será bem-sucedida quando *exist_ok* for verdadeiro (e o tempo de

modificação for atualizado para a hora atual), caso contrário a exceção `FileExistsError` será levantada.

Ver também

Os métodos `open()`, `write_text()` e `write_bytes()` são frequentemente usados para criar arquivos.

Path.mkdir(*mode=0o777*, *parents=False*, *exist_ok=False*, ***, *parent_mode=None*)

Cria um novo diretório neste caminho fornecido. Se *mode* for fornecido, ele é combinado com o valor `umask` do processo para determinar o modo do arquivo e os sinalizadores de acesso. Se o caminho já existe, `FileExistsError` é levantada.

Se *parents* for verdadeiro, quaisquer pais ausentes neste caminho serão criados conforme necessário; eles são criados com as permissões padrão sem levar o *mode* em consideração (imitando o comando POSIX `mkdir -p`).

Se *parent_mode* não for `None`, ele é usado como o modo para quaisquer diretórios de nível intermediário recém-criados quando *parents* for verdadeiro. Assim como *mode*, ele é combinado com o valor de `umask` do processo. Caso contrário, os diretórios intermediários são criados com as permissões padrão (também sujeitas ao umask).

Se *parents* for falso (o padrão), um pai ausente levanta `FileNotFoundError`.

Se *exist_ok* for falso (o padrão), `FileExistsError` será levantada se o diretório alvo já existir.

Se *exist_ok* for verdadeiro, a exceção `FileExistsError` não será levantada, a menos que o caminho fornecido já exista no sistema de arquivos e não seja um diretório (mesmo comportamento do comando POSIX `mkdir -p`).

Alterado na versão 3.5: O parâmetro *exist_ok* foi adicionado.

Adicionado na versão 3.15: O parâmetro *parent_mode*.

Path.symlink_to(*target*, *target_is_directory=False*)

Faça desse path um symbolic link apontando para um *target*

No Windows, um link simbólico representa um arquivo ou um diretório e não se transforma no destino dinamicamente. Se o alvo estiver presente, será criado um link simbólico de mesmo tipo. Caso contrário, o link simbólico será criado como um diretório se *target_is_directory* for verdadeiro ou um link simbólico de arquivo (o padrão) caso contrário. Em plataformas não Windows, *target_is_directory* é ignorado.

```
>>> p = Path('meulink')
>>> p.symlink_to('setup.py')
>>> p.resolve()
PosixPath('/home/antoine/pathlib/setup.py')
>>> p.stat().st_size
956
>>> p.lstat().st_size
8
```

Nota

A ordem dos argumentos (link, target) é o inverso da função `os.symlink()`'s.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se `os.symlink()` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Path.hardlink_to(*target*)

Faz deste caminho um link físico para o mesmo arquivo que *target*.

Nota

A ordem dos argumentos (link, target) é o inverso da função `os.link()`'s.

Adicionado na versão 3.10.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se `os.link()` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Copiando, movendo e excluindo

Path.copy(*target*, *, *follow_symlinks=True*, *preserve_metadata=False*)

Copia este arquivo ou árvore de diretório para o *target* fornecido e retorna uma nova instância `Path` apontando para *target*.

Se a origem for um arquivo, o destino será substituído, caso seja um arquivo existente. Se a origem for um link simbólico e *follow_symlinks* for verdadeiro (o padrão), o destino do link simbólico será copiado. Caso contrário, o link simbólico será recriado no destino.

Se *preserve_metadata* for falso (o padrão), apenas estruturas de diretório e dados de arquivo terão a garantia de serem copiados. Defina *preserve_metadata* como verdadeiro para garantir que permissões de arquivo e diretório, sinalizadores, horários do último acesso e modificação e atributos estendidos sejam copiados onde suportados. Este argumento não tem efeito ao copiar arquivos no Windows (onde os metadados são sempre preservados).

Nota

Quando suportado pelo sistema operacional e pelo sistema de arquivos, este método realiza uma cópia leve, na qual os blocos de dados são copiados somente quando modificados. Isso é conhecido como cópia na gravação ou, em inglês, copy-on-write.

Adicionado na versão 3.14.

Path.copy_into(*target_dir*, *, *follow_symlinks=True*, *preserve_metadata=False*)

Copia este arquivo ou diretório para o *target_dir* fornecido, que deve ser um diretório existente. Os demais argumentos são tratados de forma idêntica a `Path.copy()`. Retorna uma nova instância de `Path` apontando para a cópia.

Adicionado na versão 3.14.

Path.rename(*target*)

Renomeia este arquivo ou diretório para o *target* fornecido e retorna uma nova instância de `Path` apontando para *target*. No Unix, se *target* existir e for um arquivo, ele será substituído silenciosamente se o usuário tiver permissão. No Windows, se *target* existir, a exceção `FileExistsError` será levantada. *target* pode ser uma string ou outro objeto caminho:

```
>>> p = Path('foo')
>>> p.open('w').write('algum texto')
9
>>> target = Path('bar')
>>> p.rename(target)
PosixPath('bar')
>>> target.open().read()
'algum texto'
```

O caminho de destino pode ser absoluto ou relativo. Caminhos relativos são interpretados em relação ao diretório de trabalho atual, *não* ao diretório do objeto `Path`.

Está implementada em termos de `os.rename()` e fornece as mesmas garantias.

Alterado na versão 3.8: Adicionado valor de retorno, retorna a nova instância de `Path`.

Path.replace(*target*)

Renomeia este arquivo ou diretório para o *target* fornecido e retorna uma nova instância de `Path` apontando para *target*. Se *target* apontar para um arquivo ou diretório vazio, ele será substituído incondicionalmente.

O caminho de destino pode ser absoluto ou relativo. Caminhos relativos são interpretados em relação ao diretório de trabalho atual, *não* ao diretório do objeto `Path`.

Alterado na versão 3.8: Adicionado valor de retorno, retorna a nova instância de `Path`.

Path.move(*target*)

Move este arquivo ou árvore de diretório para o *target* fornecido e retorna uma nova instância `Path` apontando para *target*.

Se o *destino* não existir, ele será criado. Se tanto este caminho quanto o *destino* forem arquivos existentes, o destino será sobrescrito. Se ambos os caminhos apontarem para o mesmo arquivo ou diretório, ou se o *destino* for um diretório não vazio, então `OSError` será levantada.

Se ambos os caminhos estiverem no mesmo sistema de arquivos, a movimentação será realizada com `os.replace()`. Caso contrário, este caminho será copiado (preservando metadados e links simbólicos) e, em seguida, excluído.

Adicionado na versão 3.14.

Path.move_into(*target_dir*)

Move este arquivo ou diretório para o *target_dir* fornecido, que deve ser um diretório existente. Retorna uma nova instância de `Path` apontando para o caminho movido.

Adicionado na versão 3.14.

Path.unlink(*missing_ok=False*)

Remova esse arquivo ou link simbólico. Caso o caminho aponte para um diretório, use a função `Path.rmdir()` em vez disso.

Se *missing_ok* for falso (o padrão), `FileNotFoundError` é levantada se o caminho não existir.

Se *missing_ok* for verdadeiro, exceções de `FileNotFoundError` serão ignoradas (mesmo comportamento que o comando POSIX `rm -f`).

Alterado na versão 3.8: O parâmetro *missing_ok* foi adicionado.

Path.rmdir()

Remove este diretório. O diretório deve estar vazio.

Permissões e propriedade

Path.owner(*, *follow_symlinks=True*)

Retorna o nome do usuário que é o dono do arquivo. `KeyError` é levantada caso o identificador de usuário (UID) não seja encontrado no banco de dados do sistema.

Esse método normalmente segue links simbólicos; para retornar o dono do link simbólico, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se o módulo `pwd` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.group(*, *follow_symlinks=True*)

Retorna o nome do usuário que possui o grupo. `KeyError` é levantada caso o identificador de grupo (GID) não seja encontrado no banco de dados do sistema.

Esse método normalmente segue links simbólicos; para retornar o grupo do link simbólico, adicione o argumento `follow_symlinks=False`.

Alterado na versão 3.13: Levanta `UnsupportedOperation` se o módulo `grp` não estiver disponível. Em versões anteriores, `NotImplementedError` era levantada.

Alterado na versão 3.13: O parâmetro *follow_symlinks* foi adicionado.

Path.chmod(*mode*, *, *follow_symlinks=True*)

Altera o modo de arquivo e as permissões, como `os.chmod()`.

Este método normalmente segue links simbólicos. Alguns tipos de Unix têm suporte à alteração de permissões no próprio link simbólico; nessas plataformas, você pode adicionar o argumento `follow_symlinks=False`, ou usar `lchmod()`.

```
>>> p = Path('setup.py')
>>> p.stat().st_mode
33277
>>> p.chmod(0o444)
>>> p.stat().st_mode
33060
```

Alterado na versão 3.10: O parâmetro *follow_symlinks* foi adicionado.

Path.lchmod(*mode*)

Como `Path.chmod()`, mas se o caminho apontar para um link simbólico, o modo do link simbólico é alterado em vez de seu alvo.

Linguagem de padrões

Os seguintes curingas são suportados em padrões para `full_match()`, `glob()` e `rglob()`:

`**` (todo o segmento)

Corresponde a qualquer número de segmentos de arquivo ou diretório, incluindo zero.

`*` (todo o segmento)

Corresponde a um segmento de arquivo ou diretório.

`*` (parte de um segmento)
: Corresponde a qualquer número de caracteres não separadores, incluindo zero.

`?`
: Corresponde a um caractere não separador.

`[seq]`
: Corresponde a um caractere em *seq*, onde *seq* é uma sequência de caracteres. Expressões de intervalo são suportadas; por exemplo, `[a-z]` corresponde a qualquer letra ASCII minúscula. Vários intervalos podem ser combinados: `[a-zA-Z0-9_]` corresponde a qualquer letra, dígito ou sublinhado ASCII.

`[!seq]`
: Corresponde a um caractere que não está em *seq*, onde *seq* segue as mesmas regras acima.

Para uma correspondência literal, coloque os metacaracteres entre colchetes. Por exemplo, `"[?]"` corresponde ao caractere `"?"`.

O curinga "`**`" habilita fazer glob recursivo. Alguns exemplos:

Padrão	**Significado**
"`**/*`"	Qualquer caminho com pelo menos um segmento.
"`**/*.py`"	Qualquer caminho com um segmento final terminando em "`.py`".
"`assets/**`"	Qualquer caminho que comece com "`assets/`".

Padrão	Significado
“`assets/**/*`”	Qualquer caminho que comece com “`assets/`”, excluindo o próprio “`assets/`”.

Nota

Fazer globs com o curinga “`**`” visita todos os diretórios da árvore. Árvores de diretórios grandes podem levar muito tempo para serem pesquisadas.

Alterado na versão 3.13: Fazer glob com um padrão que termina com “`**`” retorna arquivos e diretórios. Em versões anteriores, apenas diretórios eram retornados.

Em `Path.glob()` e `rglob()`, uma barra final pode ser adicionada ao padrão para corresponder apenas aos diretórios.

Alterado na versão 3.11: Fazer glob com um padrão que termina com um separador de componentes de caminho (`sep` ou `altsep`) retorna apenas diretórios.

Comparação com o módulo `glob`

Os padrões aceitos e os resultados gerados por `Path.glob()` e `Path.rglob()` diferem ligeiramente daqueles pelo módulo `glob`:

1. Arquivos que começam com um ponto não são especiais em pathlib. Isso é como passar `include_hidden=True` para `glob.glob()`.
2. Os componentes do padrão “`**`” são sempre recursivos em pathlib. Isso é como passar `recursive=True` para `glob.glob()`.
3. Os componentes do padrão “`**`” não seguem links simbólicos por padrão em pathlib. Esse comportamento não tem equivalente em

`glob.glob()`, mas você pode passar `recurse_symlinks=True` para `Path.glob()` para obter um comportamento compatível.

4. Como todos os objetos `PurePath` e `Path`, os valores retornados de `Path.glob()` e `Path.rglob()` não incluem barras finais.
5. Os valores retornados de `path.glob()` e `path.rglob()` do pathlib incluem o *path* como prefixo, diferente dos resultados de `glob.glob(root_dir=path)`.
6. Os valores retornados de `path.glob()` e `path.rglob()` do pathlib podem incluir o próprio *path*, por exemplo, ao globar "`**`", enquanto os resultados de `glob.glob(root_dir=path)` nunca incluem uma string vazia que corresponderia a *path*.

Comparação com os módulos `os` e `os.path`

pathlib implementa operações de caminho usando os objetos `PurePath` e `Path`, e por isso é considerado *orientado a objetos*. Por outro lado, os módulos `os` e `os.path` fornecem funções que funcionam com objetos de baixo nível `str` e `bytes`, o que é uma abordagem mais *procedimental*. Alguns usuários consideram o estilo orientado a objetos mais legível.

Muitas funções em `os` e `os.path` oferecem suporte a caminhos `bytes` e caminhos relativos aos descritores de diretório. Esses recursos não estão disponíveis em pathlib.

Os tipos `str` e `bytes` do Python e partes dos módulos `os` e `os.path` são escritos em C e são muito rápidos. pathlib é escrito em Python puro e geralmente é mais lento, mas raramente lento o suficiente para fazer diferença.

A normalização de caminho do pathlib é um pouco mais opinativa e consistente do que a do `os.path`. Por exemplo, enquanto `os.path.abspath()` elimina segmentos "`..`" de um caminho, o que pode alterar seu significado se links simbólicos estiverem envolvidos, `Path.absolute()` preserva esses segmentos para maior segurança.

A normalização de caminho do pathlib pode torná-lo inadequado para algumas aplicações:

1. pathlib normaliza `Path("minha_pasta/")` para `Path("minha_pasta")`, o que altera o significado de um caminho quando fornecido a diversas APIs de sistemas operacionais e utilitários de linha de comando. Especificamente, a ausência de um separador final pode permitir que o caminho seja resolvido como um arquivo ou diretório, em vez de apenas um diretório.
2. pathlib normaliza `Path("./meu_programa")` para `Path("meu_programa")`, o que altera o significado de um caminho quando usado como um caminho de pesquisa executável, como em um shell ou ao gerar um processo filho. Especificamente, a ausência de um separador no caminho pode forçar a busca em **`PATH`** em vez do diretório atual.

Como consequência dessas diferenças, pathlib não é um substituto imediato para `os.path`.

Ferramentas correspondentes

Abaixo está uma tabela mapeando várias funções `os` à sua `PurePath`/`Path` equivalente.

`os` e `os.path`	**pathlib**
`os.path.dirname()`	`PurePath.parent`
`os.path.basename()`	`PurePath.name`
`os.path.splitext()`	`PurePath.stem`, `PurePath.suffix`
`os.path.join()`	`PurePath.joinpath()`

os e os.path	pathlib
os.path.isabs()	PurePath.is_absolute()
os.path.relpath()	PurePath.relative_to() [[1]]
os.path.expanduser()	Path.expanduser() [[2]]
os.path.realpath()	Path.resolve()
os.path.abspath()	Path.absolute() [[3]]
os.path.exists()	Path.exists()
os.path.isfile()	Path.is_file()
os.path.isdir()	Path.is_dir()
os.path.islink()	Path.is_symlink()
os.path.isjunction()	Path.is_junction()
os.path.ismount()	Path.is_mount()
os.path.samefile()	Path.samefile()

os e os.path	pathlib
os.getcwd()	Path.cwd()
os.stat()	Path.stat()
os.lstat()	Path.lstat()
os.listdir()	Path.iterdir()
os.walk()	Path.walk() [[4]]
os.mkdir(), os.makedirs()	Path.mkdir()
os.link()	Path.hardlink_to()
os.symlink()	Path.symlink_to()
os.readlink()	Path.readlink()
os.rename()	Path.rename()
os.replace()	Path.replace()
os.remove(), os.unlink()	Path.unlink()

os e os.path	pathlib
os.rmdir()	Path.rmdir()
os.chmod()	Path.chmod()
os.lchmod()	Path.lchmod()

Notas de rodapé

[1]

os.path.relpath() chama abspath() para tornar os caminhos absolutos e remover partes “. .”, enquanto PurePath.relative_to() é uma operação lexical que levanta ValueError quando as âncoras de suas entradas são diferentes (por exemplo, se um caminho for absoluto e o outro relativo).

[2]

os.path.expanduser() retorna o caminho inalterado se o diretório inicial não puder ser resolvido, enquanto Path.expanduser() levanta RuntimeError.

[3]

os.path.abspath() remove componentes “. .” sem resolver links simbólicos, o que pode alterar o significado do caminho, enquanto Path.absolute() deixa quaisquer componentes “. .” no caminho.

[4]

os.walk() sempre segue links simbólicos ao categorizar caminhos em *dirnames* e *filenames*, enquanto Path.walk() categoriza todos os links

simbólicos em *filenames* quando *follow_symlinks* é falso (o padrão).

Protocolos

O módulo **`pathlib.types`** fornece classes para verificação de tipos estática.

Adicionado na versão 3.14.

class pathlib.types.PathInfo

Um `typing.Protocol` que descreve o atributo `Path.info`. Implementações podem retornar resultados do cache de seus métodos.

exists(*, *follow_symlinks=True*)

Retorna `True` se o caminho for um arquivo ou um diretório existente, ou qualquer outro tipo de arquivo; retorna `False` se o caminho não existir.

Se *follow_symlinks* for `False`, retorna `True` para links simbólicos sem verificar se seus alvos existem.

is_dir(*, *follow_symlinks=True*)

Retorna `True` se o caminho for um diretório ou um link simbólico apontando para um diretório; retorna `False` se o caminho for (ou apontar para) qualquer outro tipo de arquivo ou se não existir.

Se *follow_symlinks* for `False`, retorna `True` apenas se o caminho for um diretório (sem seguir os links simbólicos); retorna `False` se o caminho for qualquer outro tipo de arquivo ou não existir.

is_file(*, *follow_symlinks=True*)

Retorna `True` se este caminho for um arquivo ou um link simbólico apontando para um arquivo; retorna `False` se a entrada for (ou apontar para) um diretório ou outro caminho que não seja de arquivo, ou se ela não existir.

Se *follow_symlinks* for `False`, retorna `True` apenas se o caminho for um arquivo (sem seguir os links simbólicos); retorna `False` se o

caminho for um diretório ou outro que não seja um arquivo, ou se não existir.

is_symlink()

Retorna `True` se este caminho for um link simbólico (mesmo quebrado); retorna `False` se o caminho for um diretório ou qualquer tipo de arquivo, ou se não existir.

`os.path` — Manipulações comuns de nomes de caminhos

Código-fonte: Lib/genericpath.py [https://github.com/python/cpython/tree/3.15/Lib/genericpath.py], Lib/posixpath.py [https://github.com/python/cpython/tree/3.15/Lib/posixpath.py] (para POSIX) e Lib/ntpath.py [https://github.com/python/cpython/tree/3.15/Lib/ntpath.py] (para Windows).

Este módulo implementa algumas funções úteis em nomes de caminho. Para ler ou escrever arquivos veja `open()`, e para acessar o sistema de arquivos veja o módulo `os`. Os parâmetros de caminho podem ser passados como strings, ou bytes, ou qualquer objeto que implemente o protocolo `os.PathLike`.

Ao contrário de um shell Unix, Python não faz nenhuma expansão *automática* de caminho. Funções como `expanduser()` e `expandvars()` podem ser invocadas explicitamente quando uma aplicação deseja uma expansão de caminho no estilo do shell. (Veja também o módulo `glob`.)

Ver também

O módulo `pathlib` oferece objetos de caminho de alto nível.

Nota

Todas essas funções aceitam apenas bytes ou apenas objetos de string como seus parâmetros. O resultado é um objeto do mesmo tipo, se um caminho ou nome de arquivo for retornado.

Nota

Since different operating systems have different path name conventions, there are several versions of this module in the standard library. The **os.path** module is always the path module suitable for the operating system Python is running on, and therefore usable for local paths. However, you can also import and use the individual modules if you want to manipulate a path that is *always* in one of the different formats. They all have the same interface:

- **posixpath** para caminhos no estilo UNIX
- **ntpath** para caminhos do Windows

Alterado na versão 3.8: **exists()**, **lexists()**, **isdir()**, **isfile()**, **islink()** e **ismount()** agora retornam `False` em vez de levantar uma exceção para caminhos que contêm caracteres ou bytes não representáveis no nível de sistema de operacional.

os.path.abspath(*path*)

Return a normalized absolutized version of the pathname *path*. On most platforms, this is equivalent to calling `normpath(join(os.getcwd(), path))`.

On Windows the path is normalized by the operating system, therefore the result can differ from `normpath(join(os.getcwd(), path))`. A drive-relative path is resolved against the current directory of the specified drive, and the drive letter is capitalized. Trailing dots and spaces are stripped. For example:

```
>>> os.path.abspath('c:spam')
'C:\\Temp\\spam'
>>> os.path.abspath('c:/temp/spam. . .')
'c:\\temp\\spam'
```

Ver também

os.path.join() and **os.path.normpath()**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.basename(*path*, /)

Retorna o nome base do caminho *path*. Este é o segundo elemento do par retornado pela passagem de *path* para a função `split()`. Observe que o resultado desta função é diferente do programa Unix **basename**; onde **basename** para `'/foo/bar/'` retorna `'bar'`, a função `basename()` retorna uma string vazia (`''`).

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.commonpath(*paths*)

Retorna o subcaminho comum mais longo de cada nome de caminho no iterável *paths*. Levanta `ValueError` se *paths* contiverem nomes de caminho absolutos e relativos, os *paths* estiverem em unidades diferentes ou se *paths* estiverem vazios. Ao contrário de `commonprefix()`, retorna um caminho válido.

Adicionado na versão 3.5.

Alterado na versão 3.6: Aceita uma sequência de objetos caminho ou similares.

Alterado na versão 3.13: Qualquer iterável agora pode ser passado, em vez de apenas sequências.

os.path.commonprefix(*list*, /)

Return the longest string prefix (taken character-by-character) that is a prefix of all strings in *list*. If *list* is empty, return the empty string (`''`).

Aviso

This function may return invalid paths because it works a character at a time. If you need a **common path prefix**, then the algorithm implemented in this function is not secure. Use `commonpath()` for finding a common path prefix.

```
>>> os.path.commonprefix(['/usr/lib', '/usr/local/lib'])
'/usr/l'
```

```
>>> os.path.commonpath(['/usr/lib', '/usr/local/lib'])
'/usr'
```

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Descontinuado desde a versão 3.15: Deprecated in favor of **os.path.commonpath()** for path prefixes. The **os.path.commonprefix()** function is being deprecated due to having a misleading name and module. The function is not safe to use for path prefixes despite being included in a module about path manipulation, meaning it is easy to accidentally introduce path traversal vulnerabilities into Python programs by using this function.

os.path.dirname(*path*, /)

Retorna o nome do diretório do nome de caminho *path*. Este é o primeiro elemento do par retornado passando *path* para a função **split()**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.exists(*path*)

Retorna `True` se *path* se referir a um caminho existente ou um descritor de arquivo aberto. Retorna `False` para links simbólicos quebrados. Em algumas plataformas, esta função pode retornar `False` se a permissão não for concedida para executar **os.stat()** no arquivo solicitado, mesmo se o *path* existir fisicamente.

Alterado na versão 3.3: path agora pode ser um inteiro: `True` é retornado se for um descritor de arquivo aberto, `False` caso contrário.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.lexists(*path*)

Retorna `True` se *path* se referir a um caminho existente, incluindo links simbólicos quebrados. Equivalente a **exists()** em plataformas sem **os.lstat()**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.expanduser(*path*)

No Unix e no Windows, retorna o argumento com um componente inicial de ~ ou `~user` substituído pelo diretório inicial daquele usuário *user*.

No Unix, um ~ no início é substituído pela variável de ambiente **`HOME`** se estiver definida; caso contrário, o diretório pessoal do usuário atual é procurado no diretório de senha através do módulo embutido **`pwd`**. Um `~user` no início é procurado diretamente no diretório de senhas.

No Windows, **`USERPROFILE`** será usada se definida; caso contrário, uma combinação de **`HOMEPATH`** e **`HOMEDRIVE`** será usada. Um `~user` inicial é tratado verificando se o último componente do diretório home do usuário atual corresponde a **`USERNAME`**, e substituindo-o se for o caso.

Se a expansão falhar ou se o caminho não começar com um til, o caminho será retornado inalterado.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.8: Não usa mais **`HOME`** no Windows.

os.path.expandvars(*path*)

Retorna o argumento com as variáveis de ambiente expandidas. Substrings da forma `$name` ou `${name}` são substituídas pelo valor da variável de ambiente *name*. Nomes de variáveis malformados e referências a variáveis não existentes permanecem inalterados.

No Windows, expansões `%name%` são suportadas juntamente a `$name` e `${name}`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.getatime(*path*, /)

Retorna a hora do último acesso de *path*. O valor de retorno é um número de ponto flutuante dando o número de segundos desde a Era Unix (veja o módulo `time`). Levanta `OSError` se o arquivo não existe ou está inacessível.

os.path.getmtime(*path*, /)

Retorna a hora da última modificação de *path*. O valor de retorno é um número de ponto flutuante dando o número de segundos desde a Era Unix (veja o módulo `time`). Levanta `OSError` se o arquivo não existe ou está inacessível.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.getctime(*path*, /)

Retorna o ctime do sistema que, em alguns sistemas (como Unix) é a hora da última alteração de metadados, e, em outros (como Windows), é a hora de criação de *path*. O valor de retorno é um número que fornece o número de segundos desde a Era Unix (veja o módulo `time`). Levanta `OSError` se o arquivo não existe ou está inacessível.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.getsize(*path*, /)

Retorna o tamanho, em bytes, de *path*. Levanta `OSError` se o arquivo não existe ou está inacessível.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.isabs(*path*, /)

Retorna `True` se *path* for um nome de caminho absoluto. No Unix, isso significa que começa com uma barra, no Windows começa com duas (contra)barras ou uma letra de unidade, caractere de dois pontos e (contra)barra juntos.

Ver também

`abspath()`

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.13: No Windows, retorna `False` se o caminho fornecido começar com exatamente uma (contra)barra.

os.path.isfile(*path*)

Retorna `True` se *path* for um arquivo regular **`existente`**. Isso segue links simbólicos, então **`islink()`** e **`isfile()`** podem ser verdadeiros para o mesmo caminho.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.isdir(*path*, /)

Retorna `True` se *path* for um diretório **`existente`**. Isso segue links simbólicos, então **`islink()`** e **`isdir()`** podem ser verdadeiros para o mesmo caminho.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.isjunction(*path*)

Retorna `True` se *path* se referir a uma entrada de diretório **`existente`** que é uma junção. Sempre retorna `False` se junções não forem suportados na plataforma atual.

Adicionado na versão 3.12.

os.path.islink(*path*)

Retorna `True` se *path* se referir a uma entrada de diretório **`existente`** que é um link simbólico. Sempre `False` se links simbólicos não forem suportados pelo tempo de execução Python.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.ismount(*path*)

Retorna `True` se o nome de caminho *path* for um *ponto de montagem*: um ponto em um sistema de arquivos onde um sistema de arquivos

diferente foi montado. No POSIX, a função verifica se o pai de *path*, `path/..`, está em um dispositivo diferente de *path*, ou se `path/..` e *path* apontam para o mesmo nó-i no mesmo dispositivo – isso deve detectar pontos de montagem para todas as variantes Unix e POSIX. Não é capaz de detectar confiavelmente montagens bind no mesmo sistema de arquivos. Em sistemas Linux, sempre vai retornar `True` para subvolumes btrfs, mesmo se eles não forem pontos de montagem. No Windows, uma raiz de letra de unidade e um UNC de compartilhamento são sempre pontos de montagem e, para qualquer outro caminho, `GetVolumePathName` é chamado para ver se é diferente do caminho de entrada.

Alterado na versão 3.4: Adicionado suporte para detecção de pontos de montagem não raiz no Windows.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.isdevdrive(*path*)

Retorna `True` se o nome do caminho *path* estiver localizado em um Windows Dev Drive. Um Dev Drive é otimizado para cenários de desenvolvedor e oferece desempenho mais rápido para leitura e gravação de arquivos. É recomendado para uso em código-fonte, diretórios de construção temporários, caches de pacotes e outras operações com uso intensivo de ES.

Pode levantar um erro para um caminho inválido, por exemplo, um sem uma unidade reconhecível, mas retorna `False` em plataformas que não provêm suporte a Dev Drives. Consulte a documentação do Windows [https://learn.microsoft.com/windows/dev-drive/] para obter informações sobre como habilitar e criar Dev Drives.

Adicionado na versão 3.12.

Alterado na versão 3.13: A função já está disponível em todas as plataformas, e sempre retornará `False` naquelas que não possuem suporte para Dev Drives

os.path.isreserved(*path*)

Retorna `True` se *path* for um caminho reservado no sistema atual.

No Windows, os nomes de arquivos reservados incluem aqueles que terminam com espaço ou ponto; aqueles que contêm dois pontos (ou seja, fluxos de arquivos como “nome:fluxo”), caracteres curinga (ou seja, `'*?"<>'`), encadeamento (pipe) ou caracteres de controle ASCII; bem como nomes de dispositivos DOS, como “NUL”, “CON”, “CONIN$”, “CONOUT$”, “AUX”, “PRN”, “COM1” e “LPT1”.

Nota

Esta função aproxima regras para caminhos reservados na maioria dos sistemas Windows. Essas regras mudam com o tempo em diversas versões do Windows. Esta função pode ser atualizada em versões futuras do Python à medida que as alterações nas regras se tornarem amplamente disponíveis.

Disponibilidade: Windows.

Adicionado na versão 3.13.

os.path.join(*path*, /, **paths*)

Join one or more path segments intelligently. The return value is the concatenation of *path* and all members of **paths*, with exactly one directory separator following each non-empty part, except the last. That is, the result will only end in a separator if the last part is either empty or ends in a separator.

If a segment is an absolute path (which on Windows requires both a drive and a root), then all previous segments are ignored and joining continues from the absolute path segment. On Linux, for example:

```
>>> os.path.join('/home/foo', 'bar')
'/home/foo/bar'
>>> os.path.join('/home/foo', '/home/bar')
'/home/bar'
```

On Windows, the drive is not reset when a rooted path segment (e.g., `r'\foo'`) is encountered. If a segment is on a different drive or is an absolute path, all previous segments are ignored and the drive is reset. For example:

```
>>> os.path.join('c:\\', 'foo')
'c:\\foo'
>>> os.path.join('c:\\foo', 'd:\\bar')
'd:\\bar'
```

Note that since there is a current directory for each drive, `os.path.join("c:", "foo")` represents a path relative to the current directory on drive `C:` (`c:foo`), not `c:\foo`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *path* e *paths*.

os.path.normcase(*path*, /)

Normaliza o estado de letras maiúsculas/minúsculas de um nome de caminho. No Windows, converte todos os caracteres do nome do caminho em minúsculas e também converte barras normais em barras invertidas. Em outros sistemas operacionais, retorna o caminho inalterado.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.normpath(*path*)

Normaliza um nome de caminho retirando separadores redundantes e referências de nível superior para que `A//B`, `A/B/`, `A/./B` e `A/foo/../B` todos se tornem `A/B`. Essa manipulação de string pode mudar o significado de um caminho que contém links simbólicos. No Windows, ele converte barras normais em barras invertidas. Para normalizar o estado de letras maiúsculas/minúsculas, use **`normcase()`**.

Nota

Em sistemas POSIX, de acordo com IEEE Std 1003.1 2013 Edition; 4.13 Pathname Resolution [https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap04.html#tag_04_13],

se um nome de caminho começa com exatamente duas barras, o primeiro componente após os caracteres iniciais pode ser interpretado em um forma definida pela implementação, embora mais de dois caracteres iniciais devam ser tratados como um único caractere.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.realpath(*path*, *, *, *strict=False*)

Return the canonical path of the specified filename, eliminating any symbolic links encountered in the path (if they are supported by the operating system). On Windows, this function will also resolve MS-DOS (also called 8.3) style names such as `C:\\PROGRA~1` to `C:\\Program Files`. The returned path uses the case reported by the operating system, which can differ from the case of *path*, in particular the drive letter is capitalized.

Por padrão, o caminho é avaliado até o primeiro componente que não existe, é um laço de link simbólico ou cuja avaliação levanta **`OSError`**. Todos esses componentes são anexados inalterados à parte existente do caminho.

Alguns erros tratados dessa forma incluem “acesso negado”, “não é um diretório” ou “argumento inválido para função interna”. Assim, o caminho resultante pode estar ausente ou inacessível, ainda pode conter links ou laços e pode passar por diretórios não especificados.

Esse comportamento pode ser modificado por argumentos nomeados:

If *strict* is `True`, the first error encountered when evaluating the path is re-raised. In particular, **`FileNotFoundError`** is raised if *path* does not exist, or another **`OSError`** if it is otherwise inaccessible. If *strict* is **`ALL_BUT_LAST`**, the last component of the path is allowed to be missing, but all other errors are raised.

Se *strict* for **`os.path.ALLOW_MISSING`**, erros diferentes de **`FileNotFoundError`** serão levantados novamente (como em `strict=True`). Portanto, o caminho retornado não conterá nenhum link

simbólico, mas o arquivo nomeado e alguns de seus diretórios pais podem estar ausentes.

Nota

Esta função emula o procedimento do sistema operacional para tornar um caminho canônico, que difere ligeiramente entre o Windows e o UNIX em relação a como links e componentes de caminho subsequentes interagem.

As APIs do sistema operacional tornam os caminhos canônicos conforme necessário, portanto, normalmente não é necessário chamar esta função.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.8: Links simbólicos e junções agora são resolvidos no Windows.

Alterado na versão 3.10: O parâmetro *strict* foi adicionado.

Alterado na versão 3.15: The `ALL_BUT_LAST` and `ALLOW_MISSING` values for the *strict* parameter was added.

os.path.ALL_BUT_LAST

Valor especial usado para o argumento *strict* em `realpath()`.

Adicionado na versão 3.15.

os.path.ALLOW_MISSING

Valor especial usado para o argumento *strict* em `realpath()`.

Adicionado na versão 3.15.

os.path.relpath(*path*, *start=os.curdir*)

Retorna um caminho de arquivo relativo a *caminho* do diretório atual ou de um diretório *start* opcional. Este é um cálculo de caminho: o sistema de arquivos não é acessado para confirmar a existência ou natureza de

path ou *start*. No Windows, `ValueError` é levantada quando *path* e *start* estão em unidades diferentes.

start tem como padrão `os.curdir`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.samefile(*path1*, *path2*, /)

Retorna `True` se ambos os argumentos de nome de caminho se referem ao mesmo arquivo ou diretório. Isso é determinado pelo número do dispositivo e número do nó-i e levanta uma exceção se uma chamada `os.stat()` em qualquer um dos caminhos falhar.

Alterado na versão 3.2: Adicionado suporte ao Windows.

Alterado na versão 3.4: O Windows agora usa a mesma implementação que todas as outras plataformas.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.sameopenfile(*fp1*, *fp2*)

Retorna `True` se os descritores de arquivo *fp1* e *fp2* fazem referência ao mesmo arquivo.

Alterado na versão 3.2: Adicionado suporte ao Windows.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.samestat(*stat1*, *stat2*, /)

Retorna `True` se as tuplas de estatísticas *stat1* e *stat2* fazem referência ao mesmo arquivo. Essas estruturas podem ter sido retornadas por `os.fstat()`, `os.lstat()` ou `os.stat()`. Esta função implementa a comparação subjacente usada por `samefile()` e `sameopenfile()`.

Alterado na versão 3.4: Adicionado suporte ao Windows.

os.path.split(*path*, /)

Split the pathname *path* into a pair, `(head, tail)` where *tail* is the last pathname component and *head* is everything leading up to that. The *tail* part will never contain a slash; if *path* ends in a slash, *tail* will be empty. If there is no slash in *path*, *head* will be empty. If *path* is empty, both *head* and *tail* are empty. Trailing slashes are stripped from *head* unless it is the root (one or more slashes only). In all cases, `join(head, tail)` returns a path to the same location as *path* (but the strings may differ). Also see the functions **`join()`**, **`dirname()`** and **`basename()`**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.splitdrive(*path*, /)

Divide o nome do caminho *path* em um par `(unidade, rabo)` onde *unidade* é um ponto de montagem ou uma string vazia. Em sistemas que não usam especificações de unidade, *unidade* sempre será a string vazia. Em todos os casos, `unidade + rabo` será o mesmo que *path*.

No Windows, divide um nome de caminho em unidade/ponto de compartilhamento UNC e caminho relativo.

Se o caminho contiver uma letra de unidade, a unidade conterá tudo, incluindo os dois pontos:

```
>>> splitdrive("c:/dir")
("c:", "/dir")
```

Se o caminho contiver um caminho UNC, a unidade conterá o nome do host e o compartilhamento:

```
>>> splitdrive("//host/computador/diretório")
("//host/computador", "/diretório")
```

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.splitroot(*path*, /)

Divide o nome de caminho *path* em uma tupla de 3 itens `(drive, root, tail)` onde *drive* é um nome de dispositivo ou ponto de montagem, *root* é uma string de separadores após a unidade e *tail* é tudo

depois da raiz. Qualquer um desses itens pode ser uma string vazia. Em todos os casos, `drive + root + tail` será igual a *path*.

Em sistemas POSIX, *drive* está sempre vazio. A *root* pode estar vazia (se *path* for relativo), uma única barra (se *path* for absoluto) ou duas barras (definidas pela implementação de acordo com IEEE Std 1003.1-2017; 4.13 Pathname Resolution [https://pubs.opengroup.org/onlinepubs/9699919799/basedefs/V1_chap04.html#tag_04_13].) Por exemplo:

```
>>> splitroot('/home/sam')
('', '/', 'home/sam')
>>> splitroot('//home/sam')
('', '//', 'home/sam')
>>> splitroot('///home/sam')
('', '/', '//home/sam')
```

No Windows, *drive* pode estar vazio, ser um nome de letra de unidade, um compartilhamento UNC ou um nome de dispositivo. *root* pode estar vazio, ser uma barra ou uma barra invertida. Por exemplo:

```
>>> splitroot('C:/Users/Sam')
('C:', '/', 'Users/Sam')
>>> splitroot('//Server/Share/Users/Sam')
('//Server/Share', '/', 'Users/Sam')
```

Adicionado na versão 3.12.

os.path.splitext(*path*, /)

Divida o nome do caminho *path* em um par `(root, ext)` de modo que `root + ext == path`, e a extensão, *ext*, esteja vazia ou comece com um ponto e contenha no máximo um período.

Se o caminho não contiver extensão, *ext* será `''`:

```
>>> splitext('bar')
('bar', '')
```

Se o caminho contiver uma extensão, *ext* será definido para esta extensão, incluindo o ponto inicial. Observe que os períodos anteriores

serão ignorados:

```
>>> splitext('foo.bar.exe')
('foo.bar', '.exe')
>>> splitext('/foo/bar.exe')
('/foo/bar', '.exe')
```

Os períodos iniciais do último componente do caminho são considerados parte da raiz:

```
>>> splitext('.cshrc')
('.cshrc', '')
>>> splitext('/foo/....jpg')
('/foo/....jpg', '')
```

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.path.supports_unicode_filenames

`True` se strings Unicode arbitrárias podem ser usadas como nomes de arquivo (dentro das limitações impostas pelo sistema de arquivos).

`stat` — Interpretando resultados de `stat()`

Código-fonte: Lib/stat.py [https://github.com/python/cpython/tree/3.15/Lib/stat.py]

O módulo **`stat`** define constantes e funções para interpretar os resultados de **`os.stat()`**, **`os.fstat()`** e **`os.lstat()`** (se existirem). Para detalhes completos sobre chamadas **`stat()`**, **`fstat()`** e **`lstat()`**, consulte a documentação do seu sistema.

Alterado na versão 3.4: O módulo stat é apoiado por uma implementação C.

O módulo **`stat`** define as seguintes funções para testar tipos de arquivos específicos:

stat.S_ISDIR(*mode*)

Retorna diferente de zero se o modo for de um diretório.

stat.S_ISCHR(*mode*)

Retorna diferente de zero se o modo for de um arquivo de dispositivo especial de caractere.

stat.S_ISBLK(*mode*)

Retorna diferente de zero se o modo for de um arquivo de dispositivo especial de bloco.

stat.S_ISREG(*mode*)

Retorna diferente de zero se o modo for de um arquivo regular.

stat.S_ISFIFO(*mode*)

Retorna diferente de zero se o modo for de um FIFO (encadeamento nomeado).

stat.S_ISLNK(*mode*)

Retorna diferente de zero se o modo for de um link simbólico.

stat.S_ISSOCK(*mode*)

Retorna diferente de zero se o modo for de um soquete.

stat.S_ISDOOR(*mode*)

Retorna diferente de zero se o modo for de uma porta.

Adicionado na versão 3.4.

stat.S_ISPORT(*mode*)

Retorna diferente de zero se o modo for de uma porta de eventos.

Adicionado na versão 3.4.

stat.S_ISWHT(*mode*)

Retorna diferente de zero se o modo for de um apagamento.

Adicionado na versão 3.4.

Duas funções adicionais são definidas para manipulação mais geral do modo do arquivo:

stat.S_IMODE(*mode*)

Retorna a parte do modo do arquivo que pode ser definido por `os.chmod()` — ou seja, os bits de permissão do arquivo, mais os bits sticky bit, set-group-id e set-user-id (em sistemas que têm suporte a eles).

stat.S_IFMT(*mode*)

Retorna a parte do modo do arquivo que descreve o tipo de arquivo (usado pelas funções **`S_IS*()`** acima).

Normalmente, você usaria as funções **os.path.is*()** para testar o tipo de um arquivo; as funções aqui são úteis quando você está fazendo vários testes do mesmo arquivo e deseja evitar a sobrecarga da chamada de sistema **stat()** para cada teste. Eles também são úteis ao verificar informações sobre um arquivo que não é manipulado por **os.path**, como os testes para dispositivos de blocos e caracteres.

Exemplo:

```
import os, sys
from stat import *

def walktree(top, callback):
    '''desce recursivamente a árvore de diretórios com raiz no t
     chamando a função de retorno de chamada para cada arquivo r

    for f in os.listdir(top):
        pathname = os.path.join(top, f)
        mode = os.lstat(pathname).st_mode
        if S_ISDIR(mode):
            # É um diretório, percorre recursivamente nele
            walktree(pathname, callback)
        elif S_ISREG(mode):
            # É um arquivo, chama a função de retorno
            callback(pathname)
        else:
            # Tipo de arquivo desconhecido, imprime uma mensagem
            print('Pulando %s' % pathname)

def visitfile(file):
    print('visitando, file)

if __name__ == '__main__':
    walktree(sys.argv[1], visitfile)
```

Uma função utilitária adicional é fornecida para converter o modo de um arquivo em uma string legível por humanos:

stat.filemode(*mode*)

Converte o modo de um arquivo em uma string no formato '-rwxrwxrwx'.

Adicionado na versão 3.3.

Alterado na versão 3.4: A função tem suporte a `S_IFDOOR`, `S_IFPORT` e `S_IFWHT`.

Todas as variáveis abaixo são simplesmente índices simbólicos nas 10 tuplas retornadas por `os.stat()`, `os.fstat()` ou `os.lstat()`.

stat.ST_MODE
Modo de proteção de nó-i.

stat.ST_INO
Número de nó-i.

stat.ST_DEV
Nó-i em que o dispositivo reside.

stat.ST_NLINK
Número de links para o nó-i.

stat.ST_UID
O ID de usuário para o proprietário.

stat.ST_GID
O ID de grupo para o proprietário.

stat.ST_SIZE
Tamanho em bytes de um arquivo simples; quantidade de dados aguardando em alguns arquivos especiais.

stat.ST_ATIME
Hora do último acesso.

stat.ST_MTIME

Hora da última modificação.

stat.ST_CTIME
O “ctime” conforme relatado pelo sistema operacional. Em alguns sistemas (como Unix) é o horário da última alteração de metadados e, em outros (como Windows), é o horário de criação (consulte a documentação da plataforma para obter detalhes).

A interpretação do “tamanho do arquivo” muda de acordo com o tipo de arquivo. Para arquivos simples, este é o tamanho do arquivo em bytes. Para FIFOs e soquetes na maioria dos tipos de Unix (incluindo Linux em particular), o “tamanho” é o número de bytes aguardando para serem lidos no momento da chamada para `os.stat()`, `os.fstat()` ou `os.lstat()`; isso às vezes pode ser útil, especialmente para pesquisar um desses arquivos especiais após uma abertura sem bloqueio. O significado do campo de tamanho para outros dispositivos de caracteres e blocos varia mais, dependendo da implementação da chamada de sistema subjacente.

As variáveis abaixo definem os sinalizadores utilizadas no campo `ST_MODE`.

O uso das funções acima é mais portátil do que o uso do primeiro conjunto de sinalizadores:

stat.S_IFSOCK
Soquete.

stat.S_IFLNK
Link simbólico.

stat.S_IFREG
Arquivo regular.

stat.S_IFBLK
Dispositivo de bloco.

stat.S_IFDIR
Diretório.

stat.S_IFCHR
Dispositivo de caracteres.

stat.S_IFIFO
FIFO.

stat.S_IFDOOR
Porta.

Adicionado na versão 3.4.

stat.S_IFPORT
Porta de eventos.

Adicionado na versão 3.4.

stat.S_IFWHT
Apagamento.

Adicionado na versão 3.4.

Nota

`S_IFDOOR`, `S_IFPORT` ou `S_IFWHT` são definidos como 0 quando a plataforma não possui suporte para os tipos de arquivo.

Os seguintes sinalizadores também podem ser usados no argumento *mode* de `os.chmod()`:

stat.S_ISUID
Define o bit de UID.

stat.S_ISGID
Bit de set-group-ID. Este bit tem vários usos especiais. Para um diretório, indica que a semântica BSD deve ser usada para esse diretório: os arquivos criados lá herdam seu ID de grupo do diretório,

não do ID de grupo efetivo do processo de criação, e os diretórios criados lá também receberão o conjunto de bits `S_ISGID`. Para um arquivo que não possui o bit de execução de grupo (`S_IXGRP`) definido, o bit de set-group-ID indica trava obrigatória de arquivo/registro (veja também `S_ENFMT`).

stat.S_ISVTX
: Sticky bit. Quando este bit é definido em um diretório, significa que um arquivo nesse diretório pode ser renomeado ou excluído apenas pelo proprietário do arquivo, pelo proprietário do diretório ou por um processo privilegiado.

stat.S_IRWXU
: Máscara para permissões de proprietário de arquivo.

stat.S_IRUSR
: Proprietário tem permissão de leitura.

stat.S_IWUSR
: Proprietário tem permissão de escrita.

stat.S_IXUSR
: Proprietário tem permissão de execução.

stat.S_IRWXG
: Máscara para permissões de grupo.

stat.S_IRGRP
: Grupo tem permissão de leitura.

stat.S_IWGRP
: Grupo tem permissão de escrita.

stat.S_IXGRP
: Grupo tem permissão de execução.

stat.S_IRWXO

Máscara para permissões para outros (não no grupo).

stat.S_IROTH
Outros têm permissão de leitura.

stat.S_IWOTH
Outros têm permissão de escrita.

stat.S_IXOTH
Outros têm permissão de execução.

stat.S_ENFMT
Aplicação de trava de arquivo do System V. Este sinalizador é compartilhada com `S_ISGID`: a trava de arquivo/registro é aplicada em arquivos que não possuem o bit de execução de grupo (`S_IXGRP`) definido.

stat.S_IREAD
Sinônimo Unix V7 para `S_IRUSR`.

stat.S_IWRITE
Sinônimo Unix V7 para `S_IWUSR`.

stat.S_IEXEC
Sinônimo Unix V7 para `S_IXUSR`.

Os seguintes sinalizadores podem ser usados no argumento *flags* de `os.chflags()`:

stat.UF_SETTABLE
Todos os sinalizadores definíveis pelo usuário

Adicionado na versão 3.13.

stat.UF_NODUMP
Não despeja o arquivo.

stat.UF_IMMUTABLE
O arquivo não pode ser alterado.

stat.UF_APPEND
O arquivo só pode sofrer acréscimos.

stat.UF_OPAQUE
O diretório é opaco quando visualizado por meio de uma pilha de união.

stat.UF_NOUNLINK
O arquivo não pode ser renomeado ou excluído.

stat.UF_COMPRESSED
O arquivo é armazenado compactado (macOS 10.6+).

stat.UF_TRACKED
Usado para manipular IDs de documentos (macOS)

Adicionado na versão 3.13.

stat.UF_DATAVAULT
O arquivo precisa de direito para leitura ou gravação (macOS 10.13+)

Adicionado na versão 3.13.

stat.UF_HIDDEN
O arquivo não deve ser exibido em uma GUI (macOS 10.5+).

stat.SF_SETTABLE
Todos os sinalizadores mutáveis de superusuário

Adicionado na versão 3.13.

stat.SF_SUPPORTED
Todos os sinalizadores válidos de superusuário

Disponibilidade: macOS

Adicionado na versão 3.13.

stat.SF_SYNTHETIC
Todos os sinalizadores sintéticos de somente leitura de superusuário

Disponibilidade: macOS

Adicionado na versão 3.13.

stat.SF_ARCHIVED
O arquivo não pode ser arquivado.

stat.SF_IMMUTABLE
O arquivo não pode ser alterado.

stat.SF_APPEND
O arquivo só pode sofrer acréscimos.

stat.SF_RESTRICTED
O arquivo precisa de um direito para ser escrito (macOS 10.13+)

Adicionado na versão 3.13.

stat.SF_NOUNLINK
O arquivo não pode ser renomeado ou excluído.

stat.SF_SNAPSHOT
O arquivo é um arquivo de captura (snapshot).

stat.SF_FIRMLINK
O arquivo é um firmlink (macOS 10.15+)

Adicionado na versão 3.13.

stat.SF_DATALESS
O arquivo é um objeto sem dados (macOS 10.15+)

Adicionado na versão 3.13.

Consulte a página man dos sistemas *BSD ou macOS *chflags(2)* *[https://manpages.debian.org/chflags(2)]* para obter mais informações.

No Windows, as seguintes constantes de atributos de arquivo estão disponíveis para uso ao testar bits no membro `st_file_attributes` retornado por **`os.stat()`**. Consulte a documentação da API do Windows [https://msdn.microsoft.com/en-us/library/windows/desktop/gg258117.aspx] para obter mais detalhes sobre o significado dessas constantes.

stat.FILE_ATTRIBUTE_ARCHIVE
stat.FILE_ATTRIBUTE_COMPRESSED
stat.FILE_ATTRIBUTE_DEVICE
stat.FILE_ATTRIBUTE_DIRECTORY
stat.FILE_ATTRIBUTE_ENCRYPTED
stat.FILE_ATTRIBUTE_HIDDEN
stat.FILE_ATTRIBUTE_INTEGRITY_STREAM
stat.FILE_ATTRIBUTE_NORMAL
stat.FILE_ATTRIBUTE_NOT_CONTENT_INDEXED
stat.FILE_ATTRIBUTE_NO_SCRUB_DATA
stat.FILE_ATTRIBUTE_OFFLINE
stat.FILE_ATTRIBUTE_READONLY
stat.FILE_ATTRIBUTE_REPARSE_POINT
stat.FILE_ATTRIBUTE_SPARSE_FILE
stat.FILE_ATTRIBUTE_SYSTEM
stat.FILE_ATTRIBUTE_TEMPORARY
stat.FILE_ATTRIBUTE_VIRTUAL

Adicionado na versão 3.5.

No Windows, as seguintes constantes estão disponíveis para comparação com o membro `st_reparse_tag` retornado por **`os.lstat()`**. Estas são constantes bem conhecidas, mas não são uma lista exaustiva.

stat.IO_REPARSE_TAG_SYMLINK
stat.IO_REPARSE_TAG_MOUNT_POINT
stat.IO_REPARSE_TAG_APPEXECLINK

Adicionado na versão 3.8.

No Linux, as seguintes constantes de atributo de arquivo estão disponíveis para uso ao testar bits nos membros `stx_attributes` e `stx_attributes_mask` retornados por `os.statx()`. Consulte a página man *statx(2)* *[https://manpages.debian.org/statx(2)]* para obter mais detalhes sobre o significado dessas constantes.

stat.STATX_ATTR_COMPRESSED
stat.STATX_ATTR_IMMUTABLE
stat.STATX_ATTR_APPEND
stat.STATX_ATTR_NODUMP
stat.STATX_ATTR_ENCRYPTED
stat.STATX_ATTR_AUTOMOUNT
stat.STATX_ATTR_MOUNT_ROOT
stat.STATX_ATTR_VERITY
stat.STATX_ATTR_DAX
stat.STATX_ATTR_WRITE_ATOMIC

Adicionado na versão 3.15.

`filecmp` — Comparações de arquivos e diretórios

Código-fonte: Lib/filecmp.py
[https://github.com/python/cpython/tree/3.15/Lib/filecmp.py]

O módulo **`filecmp`** define funções para comparar arquivos e diretórios, com várias compensações opcionais de tempo/correção. Para comparar arquivos, veja também o módulo **`difflib`**.

O módulo **`filecmp`** define as seguintes funções:

filecmp.cmp(*f1*, *f2*, *shallow=True*)

Compara os arquivos chamados *f1* e *f2*, retornando `True` se eles parecerem iguais, `False` caso contrário.

Se *shallow* for verdadeiro e as assinaturas de **`os.stat()`** (tipo de arquivo, tamanho e hora de modificação) de ambos os arquivos forem idênticas, os arquivos serão considerados iguais.

Caso contrário, os arquivos serão tratados como diferentes se seus tamanhos ou conteúdos forem diferentes.

Observe que nenhum programa externo é chamado a partir desta função, o que lhe confere portabilidade e eficiência.

Esta função usa um cache para comparações passadas e os resultados, com entradas de cache invalidadas se as informações de **`os.stat()`** para o arquivo mudarem. O cache inteiro pode ser limpo usando **`clear_cache()`**.

filecmp.cmpfiles(*a*, *b*, *common*, *shallow=True*)

Compara os arquivos nos dois diretórios *a* e *b* cujos nomes são dados por *common*.

Retorna três listas de nomes de arquivos: *match*, *mismatch*, *errors*. *match* contém a lista de arquivos que correspondem, *mismatch* contém os nomes daqueles que não correspondem e *errors* lista os nomes dos arquivos que não puderam ser comparados. Os arquivos são listados em *errors* se não existirem em um dos diretórios, o usuário não tiver permissão para lê-los ou se a comparação não puder ser feita por algum outro motivo.

O parâmetro *shallow* tem o mesmo significado e valor padrão que `filecmp.cmp()`.

Por exemplo, `cmpfiles('a', 'b', ['c', 'd/e'])` comparará `a/c` com `b/c` e `a/d/e` com `b/d/e`. `'c'` e `'d/e'` estarão cada um em uma das três listas retornadas.

filecmp.clear_cache()

Limpa o cache do filecmp. Isso pode ser útil se um arquivo for comparado tão rapidamente após ser modificado que ele esteja dentro da resolução de mtime do sistema de arquivos subjacente.

Adicionado na versão 3.4.

A classe `dircmp`

class filecmp.dircmp(*a*, *b*, *ignore=None*, *hide=None*, ***, *shallow=True*)

Constrói um novo objeto de comparação de diretórios para comparar os diretórios *a* e *b*. *ignore* é uma lista de nomes a serem ignorados e o padrão é `filecmp.DEFAULT_IGNORES`. *hide* é uma lista de nomes a serem ocultados e o padrão é `[os.curdir, os.pardir]`.

A classe `dircmp` compara arquivos fazendo comparações rasas, isto é, *shallow* conforme descrito para `filecmp.cmp()` por padrão usando o parâmetro *shallow*.

Alterado na versão 3.13: Adicionado o parâmetro *shallow*.

A classe `dircmp` fornece os seguintes métodos:

report()
: Exibe (para `sys.stdout`) uma comparação entre *a* e *b*.

report_partial_closure()
: Exibe uma comparação entre *a* e *b* e subdiretórios imediatos comuns.

report_full_closure()
: Exibe uma comparação entre *a* e *b* e subdiretórios comuns (recursivamente).

A classe `dircmp` oferece uma série de atributos interessantes que podem ser usados para obter várias informações sobre as árvores de diretórios que estão sendo comparadas.

Observe que, por meio dos ganchos `__getattr__()`, todos os atributos são computados preguiçosamente, portanto, não há perda de velocidade se apenas os atributos que são leves para computar forem usados.

left
: O diretório *a*.

right
: O diretório *b*.

left_list
: Arquivos e subdiretórios em *a*, filtrados por *hide* e *ignore*.

right_list
: Arquivos e subdiretórios em *b*, filtrados por *hide* e *ignore*.

common
: Arquivos e subdiretórios em *a* e *b*.

left_only
: Arquivos e subdiretórios em apenas *a*.

right_only
: Arquivos e subdiretórios em apenas *b*.

common_dirs
: Subdiretórios em *a* e *b*.

common_files
: Arquivos em *a* e *b*.

common_funny
: Nomes em *a* e *b*, de modo que o tipo difere entre os diretórios, ou nomes para os quais `os.stat()` relata um erro.

same_files
: Arquivos que são idênticos em *a* e *b*, usando o operador de comparação de arquivos da classe.

diff_files
: Arquivos que estão em *a* e *b*, cujos conteúdos diferem de acordo com o operador de comparação de arquivos da classe.

funny_files
: Arquivos que estão em *a* e *b*, mas não puderam ser comparados.

subdirs
: Um dicionário que mapeia nomes em `common_dirs` para instâncias `dircmp` (ou instâncias MyDirCmp se esta instância for do tipo MyDirCmp, uma subclasse de **`dircmp`**).

  Alterado na versão 3.10: Anteriormente, as entradas eram sempre instâncias de `dircmp`. Agora, as entradas são do mesmo tipo que *self*, se *self* for uma subclasse de **`dircmp`**.

filecmp.DEFAULT_IGNORES

Adicionado na versão 3.4.

Lista de diretórios ignorados por **`dircmp`** por padrão.

Aqui está um exemplo simplificado do uso do atributo `subdirs` para pesquisar recursivamente em dois diretórios para mostrar arquivos diferentes em comum:

```
>>> from filecmp import dircmp
>>> def print_diff_files(dcmp):
...     for name in dcmp.diff_files:
...         print("diff_file %s found in %s and %s" % (name, dcm
...               dcmp.right))
...     for sub_dcmp in dcmp.subdirs.values():
...         print_diff_files(sub_dcmp)
...
>>> dcmp = dircmp('dir1', 'dir2')
>>> print_diff_files(dcmp)
```

`tempfile` — Gera arquivos temporários e diretórios

Código-fonte: Lib/tempfile.py
[https://github.com/python/cpython/tree/3.15/Lib/tempfile.py]

This module creates temporary files and directories. It works on all supported platforms. `TemporaryFile`, `NamedTemporaryFile`, `TemporaryDirectory`, and `SpooledTemporaryFile` are high-level interfaces which provide automatic cleanup and can be used as context managers. `mkstemp()` and `mkdtemp()` are lower-level functions which require manual cleanup.

All the user-callable functions and constructors take additional arguments which allow direct control over the location and name of temporary files and directories. Files names used by this module include a string of random characters which allows those files to be securely created in shared temporary directories. To maintain backward compatibility, the argument order is somewhat odd; it is recommended to use keyword arguments for clarity.

The module defines the following user-callable items:

tempfile.TemporaryFile(*mode='w+b'*, *buffering=-1*, *encoding=None*, *newline=None*, *suffix=None*, *prefix=None*, *dir=None*, ***, *errors=None*)

Return a file-like object that can be used as a temporary storage area. The file is created securely, using the same rules as `mkstemp()`. It will be destroyed as soon as it is closed (including an implicit close when the object is garbage collected). Under Unix, the directory entry for the file is either not created at all or is removed immediately after the file is created. Other platforms do not support this; your code should not rely on a temporary file created using this function having or not having a visible name in the file system.

The resulting object can be used as a context manager (see Exemplos). On completion of the context or destruction of the file object the temporary file will be removed from the filesystem.

The *mode* parameter defaults to `'w+b'` so that the file created can be read and written without being closed. Binary mode is used so that it behaves consistently on all platforms without regard for the data that is stored. *buffering*, *encoding*, *errors* and *newline* are interpreted as for **`open()`**.

The *dir*, *prefix* and *suffix* parameters have the same meaning and defaults as with **`mkstemp()`**.

The returned object is a true file object on POSIX platforms. On other platforms, it is a file-like object whose **`file`** attribute is the underlying true file object.

The **`os.O_TMPFILE`** flag is used if it is available and works (Linux-specific, requires Linux kernel 3.11 or later).

On platforms that are neither Posix nor Cygwin, TemporaryFile is an alias for NamedTemporaryFile.

Levanta um evento de auditoria `tempfile.mkstemp` com o argumento `fullpath`.

Alterado na versão 3.5: The **`os.O_TMPFILE`** flag is now used if available.

Alterado na versão 3.8: Parâmetro *errors* foi adicionado.

tempfile.NamedTemporaryFile(*mode='w+b', buffering=-1, encoding=None, newline=None, suffix=None, prefix=None, dir=None, delete=True, *, errors=None, delete_on_close=True*)

This function operates exactly as **`TemporaryFile()`** does, except the following differences:

- This function returns a file that is guaranteed to have a visible name in the file system.
- To manage the named file, it extends the parameters of `TemporaryFile()` with *delete* and *delete_on_close* parameters that determine whether and how the named file should be automatically deleted.

The returned object is always a file-like object whose **`file`** attribute is the underlying true file object. This file-like object can be used in a `with` statement, just like a normal file. The name of the temporary file can be retrieved from the **`name`** attribute of the returned file-like object. On Unix, unlike with the `TemporaryFile()`, the directory entry does not get unlinked immediately after the file creation.

If *delete* is true (the default) and *delete_on_close* is true (the default), the file is deleted as soon as it is closed. If *delete* is true and *delete_on_close* is false, the file is deleted on context manager exit only, or else when the file-like object is finalized. Deletion is not always guaranteed in this case (see `object.__del__()`). If *delete* is false, the value of *delete_on_close* is ignored.

Therefore to use the name of the temporary file to reopen the file after closing it, either make sure not to delete the file upon closure (set the *delete* parameter to be false) or, in case the temporary file is created in a `with` statement, set the *delete_on_close* parameter to be false. The latter approach is recommended as it provides assistance in automatic cleaning of the temporary file upon the context manager exit.

Opening the temporary file again by its name while it is still open works as follows:

- On POSIX the file can always be opened again.
- On Windows, make sure that at least one of the following conditions are fulfilled:
  - *delete* is false
  - additional open shares delete access (e.g. by calling `os.open()` with the flag `O_TEMPORARY`)

- *delete* is true but *delete_on_close* is false. Note, that in this case the additional opens that do not share delete access (e.g. created via builtin `open()`) must be closed before exiting the context manager, else the `os.unlink()` call on context manager exit will fail with a `PermissionError`.

On Windows, if *delete_on_close* is false, and the file is created in a directory for which the user lacks delete access, then the `os.unlink()` call on exit of the context manager will fail with a `PermissionError`. This cannot happen when *delete_on_close* is true because delete access is requested by the open, which fails immediately if the requested access is not granted.

On POSIX (only), a process that is terminated abruptly with SIGKILL cannot automatically delete any NamedTemporaryFiles it created.

Levanta um evento de auditoria `tempfile.mkstemp` com o argumento `fullpath`.

Alterado na versão 3.8: Parâmetro *errors* foi adicionado.

Alterado na versão 3.12: Added *delete_on_close* parameter.

class tempfile.SpooledTemporaryFile(*max_size=0*, *mode='w+b'*, *buffering=-1*, *encoding=None*, *newline=None*, *suffix=None*, *prefix=None*, *dir=None*, *, *errors=None*)

This class operates exactly as `TemporaryFile()` does, except that data is spooled in memory until the file size exceeds *max_size*, or until the file's `fileno()` method is called, at which point the contents are written to disk and operation proceeds as with **`TemporaryFile()`**.

rollover()

The resulting file has one additional method, **`rollover()`**, which causes the file to roll over to an on-disk file regardless of its size.

The returned object is a file-like object whose **`_file`** attribute is either an `io.BytesIO` or `io.TextIOWrapper` object (depending on whether binary or text *mode* was specified) or a true file object, depending on

whether `rollover()` has been called. This file-like object can be used in a `with` statement, just like a normal file.

Alterado na versão 3.3: the truncate method now accepts a *size* argument.

Alterado na versão 3.8: Parâmetro *errors* foi adicionado.

Alterado na versão 3.11: Fully implements the `io.BufferedIOBase` and `io.TextIOBase` abstract base classes (depending on whether binary or text *mode* was specified).

class tempfile.TemporaryDirectory(*suffix=None*, *prefix=None*, *dir=None*, *ignore_cleanup_errors=False*, *, *delete=True*)

This class securely creates a temporary directory using the same rules as `mkdtemp()`. The resulting object can be used as a context manager (see Exemplos). On completion of the context or destruction of the temporary directory object, the newly created temporary directory and all its contents are removed from the filesystem.

name

The directory name can be retrieved from the `name` attribute of the returned object. When the returned object is used as a context manager, the `name` will be assigned to the target of the `as` clause in the `with` statement, if there is one.

cleanup()

The directory can be explicitly cleaned up by calling the `cleanup()` method. If *ignore_cleanup_errors* is true, any unhandled exceptions during explicit or implicit cleanup (such as a `PermissionError` removing open files on Windows) will be ignored, and the remaining removable items deleted on a “best-effort” basis. Otherwise, errors will be raised in whatever context cleanup occurs (the `cleanup()` call, exiting the context manager, when the object is garbage-collected or during interpreter shutdown).

The *delete* parameter can be used to disable cleanup of the directory tree upon exiting the context. While it may seem unusual for a context manager to disable the action taken when exiting the context, it can be useful during debugging or when you need your cleanup behavior to be conditional based on other logic.

Aviso

Cleanup is not robust against the tree being modified while it is removed. Files outside of the tree may have their permissions and file flags reset.

On systems where `shutil.rmtree.avoids_symlink_attacks` is false, manipulating symbolic links during cleanup may cause files outside of the tree to be removed.

Levanta um evento de auditoria `tempfile.mkdtemp` com o argumento `fullpath`.

Adicionado na versão 3.2.

Alterado na versão 3.10: Added *ignore_cleanup_errors* parameter.

Alterado na versão 3.12: Added the *delete* parameter.

tempfile.mkstemp(*suffix=None*, *prefix=None*, *dir=None*, *text=False*)

Creates a temporary file in the most secure manner possible. There are no race conditions in the file's creation, assuming that the platform properly implements the `os.O_EXCL` flag for `os.open()`. The file is readable and writable only by the creating user ID. If the platform uses permission bits to indicate whether a file is executable, the file is executable by no one.

The file descriptor is not inherited by child processes.

Unlike `TemporaryFile()`, the user of `mkstemp()` is responsible for closing the file descriptor (for example, using `os.close()`) and deleting the temporary file (for example, using `os.remove()`).

If *suffix* is not `None`, the file name will end with that suffix, otherwise there will be no suffix. **`mkstemp()`** does not put a dot between the file name and the suffix; if you need one, put it at the beginning of *suffix*.

If *prefix* is not `None`, the file name will begin with that prefix; otherwise, a default prefix is used. The default is the return value of **`gettempprefix()`** or **`gettempprefixb()`**, as appropriate.

If *dir* is not `None`, the file will be created in that directory; otherwise, a default directory is used. The default directory is chosen from a platform-dependent list, but the user of the application can control the directory location by setting the *TMPDIR*, *TEMP* or *TMP* environment variables. There is thus no guarantee that the generated filename will have any nice properties, such as not requiring quoting when passed to external commands via `os.popen()`.

If any of *suffix*, *prefix*, and *dir* are not `None`, they must be the same type. If they are bytes, the returned name will be bytes instead of str. If you want to force a bytes return value with otherwise default behavior, pass `suffix=b''`.

If *text* is specified and true, the file is opened in text mode. Otherwise, (the default) the file is opened in binary mode.

`mkstemp()` returns a tuple containing an OS-level handle to an open file (as would be returned by **`os.open()`**) and the absolute pathname of that file, in that order.

Levanta um evento de auditoria `tempfile.mkstemp` com o argumento `fullpath`.

Alterado na versão 3.5: suffix, *prefix*, and *dir* may now be supplied in bytes in order to obtain a bytes return value. Prior to this, only str was allowed. *suffix* and *prefix* now accept and default to `None` to cause an appropriate default value to be used.

Alterado na versão 3.6: The *dir* parameter now accepts a path-like object.

tempfile.mkdtemp(*suffix=None*, *prefix=None*, *dir=None*)

Creates a temporary directory in the most secure manner possible. There are no race conditions in the directory's creation. The directory is readable, writable, and searchable only by the creating user ID.

The user of **`mkdtemp()`** is responsible for deleting the temporary directory and its contents when done with it.

The *prefix*, *suffix*, and *dir* arguments are the same as for **`mkstemp()`**.

`mkdtemp()` returns the absolute pathname of the new directory.

Levanta um evento de auditoria `tempfile.mkdtemp` com o argumento `fullpath`.

Alterado na versão 3.5: *suffix*, *prefix*, and *dir* may now be supplied in bytes in order to obtain a bytes return value. Prior to this, only str was allowed. *suffix* and *prefix* now accept and default to `None` to cause an appropriate default value to be used.

Alterado na versão 3.6: The *dir* parameter now accepts a path-like object.

Alterado na versão 3.12: **`mkdtemp()`** now always returns an absolute path, even if *dir* is relative.

tempfile.gettempdir()

Return the name of the directory used for temporary files. This defines the default value for the *dir* argument to all functions in this module.

Python searches a standard list of directories to find one which the calling user can create files in. The list is:

1. The directory named by the **`TMPDIR`** environment variable.
2. The directory named by the **`TEMP`** environment variable.
3. The directory named by the **`TMP`** environment variable.
4. Uma localização específica por plataforma:

- On Windows, the directories `%USERPROFILE%\AppData\Local\Temp`, `%SYSTEMROOT%\Temp`, `C:\TEMP`, `C:\TMP`, `\TEMP`, and `\TMP`, in that order.
- On all other platforms, the directories `/tmp`, `/var/tmp`, and `/usr/tmp`, in that order.

5. As a last resort, the current working directory.

The result of this search is cached, see the description of **`tempdir`** below.

Alterado na versão 3.10: Always returns a str. Previously it would return any **`tempdir`** value regardless of type so long as it was not `None`.

tempfile.gettempdirb()

Same as **`gettempdir()`** but the return value is in bytes.

Adicionado na versão 3.5.

tempfile.gettempprefix()

Return the filename prefix used to create temporary files. This does not contain the directory component.

tempfile.gettempprefixb()

Same as **`gettempprefix()`** but the return value is in bytes.

Adicionado na versão 3.5.

The module uses a global variable to store the name of the directory used for temporary files returned by **`gettempdir()`**. It can be set directly to override the selection process, but this is discouraged. All functions in this module take a *dir* argument which can be used to specify the directory. This is the recommended approach that does not surprise other unsuspecting code by changing global API behavior.

tempfile.tempdir

When set to a value other than `None`, this variable defines the default value for the *dir* argument to the functions defined in this module, including its type, bytes or str. It cannot be a path-like object.

If `tempdir` is `None` (the default) at any call to any of the above functions except **`gettempprefix()`** it is initialized following the algorithm described in **`gettempdir()`**.

Nota

Beware that if you set `tempdir` to a bytes value, there is a nasty side effect: The global default return type of **`mkstemp()`** and **`mkdtemp()`** changes to bytes when no explicit `prefix`, `suffix`, or `dir` arguments of type str are supplied. Please do not write code expecting or depending on this. This awkward behavior is maintained for compatibility with the historical implementation.

Exemplos

Here are some examples of typical usage of the **`tempfile`** module:

```
>>> import tempfile

# create a temporary file and write some data to it
>>> fp = tempfile.TemporaryFile()
>>> fp.write(b'Hello world!')
# read data from file
>>> fp.seek(0)
>>> fp.read()
b'Hello world!'
# close the file, it will be removed
>>> fp.close()

# create a temporary file using a context manager
>>> with tempfile.TemporaryFile() as fp:
...     fp.write(b'Hello world!')
...     fp.seek(0)
...     fp.read()
b'Hello world!'
>>>
# file is now closed and removed

# create a temporary file using a context manager
```

```
# close the file, use the name to open the file again
>>> with tempfile.NamedTemporaryFile(delete_on_close=False) as f
...     fp.write(b'Hello world!')
...     fp.close()
... # the file is closed, but not removed
... # open the file again by using its name
...     with open(fp.name, mode='rb') as f:
...         f.read()
b'Hello world!'
>>>
# file is now removed

# create a temporary directory using the context manager
>>> with tempfile.TemporaryDirectory() as tmpdirname:
...     print('created temporary directory', tmpdirname)
>>>
# directory and contents have been removed
```

Deprecated functions and variables

A historical way to create temporary files was to first generate a file name with the `mktemp()` function and then create a file using this name. Unfortunately this is not secure, because a different process may create a file with this name in the time between the call to **`mktemp()`** and the subsequent attempt to create the file by the first process. The solution is to combine the two steps and create the file immediately. This approach is used by `mkstemp()` and the other functions described above.

tempfile.mktemp(*suffix=''*, *prefix='tmp'*, *dir=None*)

Descontinuado desde a versão 2.3: Use `mkstemp()`.

Return an absolute pathname of a file that did not exist at the time the call is made. The *prefix*, *suffix*, and *dir* arguments are similar to those of `mkstemp()`, except that bytes file names, `suffix=None` and `prefix=None` are not supported.

Aviso

Use of this function may introduce a security hole in your program. By the time you get around to doing anything with the file name it returns, someone else may have beaten you to the punch. **`mktemp()`** usage can be replaced easily with **`NamedTemporaryFile()`**, passing it the `delete=False` parameter:

```
>>> f = NamedTemporaryFile(delete=False)
>>> f.name
'/tmp/tmptjujjt'
>>> f.write(b"Hello World!\n")
13
>>> f.close()
>>> os.unlink(f.name)
>>> os.path.exists(f.name)
False
```

`glob` — Expansão de padrão de nome de arquivo no estilo Unix

Código-fonte: Lib/glob.py [https://github.com/python/cpython/tree/3.15/Lib/glob.py]

O módulo **`glob`** encontra caminhos usando regras de correspondência de padrões semelhante a do shell Unix. Nenhuma expansão de til é feita, mas `*`, `?` e os intervalos de caracteres expressos com `[]` serão correspondidos corretamente. Isso é feito usando as funções **`os.scandir()`** e **`fnmatch.fnmatch()`** em conjunto, e não invocando realmente um subshell.

Nota

Os caminhos são retornados sem uma ordem específica. Se precisar de uma ordem específica, classifique os resultados.

By default, files beginning with a dot (`.`) can only be matched by patterns that also start with a dot, unlike **`fnmatch.fnmatch()`** or **`pathlib.Path.glob()`**. For tilde and shell variable expansion, use **`os.path.expanduser()`** and **`os.path.expandvars()`**.

Para uma correspondência literal, coloque os metacaracteres entre colchetes. Por exemplo, `'[?]'` corresponde ao caractere `'?'`.

O módulo **`glob`** define as seguintes funções:

glob.glob(*pathname*, **, root_dir=None, dir_fd=None, recursive=False, include_hidden=False*)

Retorna uma lista possivelmente vazia de nomes de caminho que correspondem a *pathname*, que deve ser uma string contendo uma especificação de caminho. *pathname* pode ser absoluto (como

`/usr/src/Python-1.5/Makefile`) ou relativo (como `../../Tools/*/*.gif`) e pode conter curingas no estilo shell. Links simbólicos quebrados são incluídos nos resultados (como no shell). Se os resultados são classificados ou não depende do sistema de arquivos. Se um arquivo que satisfaz as condições for removido ou adicionado durante a chamada desta função, não é especificado se um nome de caminho para esse arquivo será incluído.

Se *root_dir* não for `None`, deve ser um objeto caminho ou similar especificando o diretório raiz para pesquisa. Tem o mesmo efeito em **`glob()`** que alterar o diretório atual antes de chamá-lo. Se *pathname* for relativo, o resultado conterá caminhos relativos a *root_dir*.

Esta função oferece suporte para caminhos relativos aos descritores de diretório com o parâmetro *dir_fd*.

Se *recursive* for verdadeiro, o padrão "`**`" corresponderá a qualquer arquivo e zero ou mais diretórios, subdiretórios e links simbólicos para diretórios. Se o padrão for seguido por um **`os.sep`** ou **`os.altsep`**, então os arquivos não irão corresponder.

If *include_hidden* is true, wildcards can match path segments that begin with a dot (`.`).

Levanta um evento de auditoria `glob.glob` com os argumentos `pathname`, `recursive`.

Levanta um evento de auditoria `glob.glob/2` com os argumentos `pathname`, `recursive`, `root_dir`, `dir_fd`.

Nota

Usar o padrão "`**`" em grandes árvores de diretório pode consumir uma quantidade excessiva de tempo.

Nota

Esta função pode retornar nomes de caminhos duplicados se *pathname* contiver vários padrões “`**`” e *recursive* for verdadeiro.

Nota

Qualquer exceção `OSError` levantada durante a varredura do sistema de arquivos é ignorada. Isso inclui `PermissionError` ao acessar diretórios sem permissão de leitura.

Alterado na versão 3.5: Suporte a globs recursivos usando “`**`”.

Alterado na versão 3.10: Adicionados os parâmetros *root_dir* e *dir_fd*.

Alterado na versão 3.11: Adicionado o parâmetro *include_hidden*.

glob.iglob(*pathname*, *, *root_dir=None*, *dir_fd=None*, *recursive=False*, *include_hidden=False*)

Retorna um iterador que produz os mesmos valores que `glob()` sem realmente armazená-los todos simultaneamente.

Levanta um evento de auditoria `glob.glob` com os argumentos `pathname`, `recursive`.

Levanta um evento de auditoria `glob.glob/2` com os argumentos `pathname`, `recursive`, `root_dir`, `dir_fd`.

Nota

Esta função pode retornar nomes de caminhos duplicados se *pathname* contiver vários padrões “`**`” e *recursive* for verdadeiro.

Nota

Qualquer exceção `OSError` levantada durante a varredura do sistema de arquivos é ignorada. Isso inclui `PermissionError` ao acessar diretórios sem permissão de leitura.

Alterado na versão 3.5: Suporte a globs recursivos usando “`**`”.

Alterado na versão 3.10: Adicionados os parâmetros *root_dir* e *dir_fd*.

Alterado na versão 3.11: Adicionado o parâmetro *include_hidden*.

glob.escape(*pathname*)

Escape all special characters (`'?'`, `'*'` and `'['`). This is useful if you want to match an arbitrary literal string that may have special characters in it. Special characters in drive/UNC sharepoints are not escaped, for example on Windows `escape('//?/c:/Quo vadis?.txt')` returns `'//?/c:/Quo vadis[?].txt'`.

Adicionado na versão 3.4.

glob.translate(*pathname*, *, *recursive=False*, *include_hidden=False*, *seps=None*)

Convert the given path specification to a regular expression for use with `re.prefixmatch()`. The path specification can contain shell-style wildcards.

Por exemplo:

```
>>> import glob, re
>>>
>>> regex = glob.translate('**/*.txt', recursive=True, inclu
>>> regex
'(?s:(?:.+/)?[^/]*\\.txt)\\z'
>>> reobj = re.compile(regex)
>>> reobj.prefixmatch('foo/bar/baz.txt')
<re.Match object; span=(0, 15), match='foo/bar/baz.txt'>
```

Separadores de caminho e segmentos são significativos para esta função, diferentemente de `fnmatch.translate()`. Por padrão, curingas não correspondem a separadores de caminho, e segmentos de padrão `*` correspondem precisamente a um segmento de caminho.

Se *recursive* for verdadeiro, o segmento de padrão "`**`" corresponderá a qualquer número de segmentos de caminho.

Se *include_hidden* for verdadeiro, os curingas poderão corresponder a segmentos de caminho que começam com um ponto (`.`).

Uma sequência de separadores de caminho pode ser fornecida ao argumento *seps*. Se não for fornecido, `os.sep` e `altsep` (se disponível) são usados.

Ver também

Os métodos `pathlib.PurePath.full_match()` e `pathlib.Path.glob()`, que chamam esta função para implementar correspondência de padrões e globbing.

Adicionado na versão 3.13.

Exemplos

Considere um diretório contendo os seguintes arquivos: `1.gif`, `2.txt`, `card.gif` e um subdiretório `sub` que contém apenas o arquivo `3.txt`. `glob()` produzirá os seguintes resultados. Observe como todos os componentes principais do caminho são preservados.

```
>>> import glob
>>> glob.glob('./[0-9].*')
['./1.gif', './2.txt']
>>> glob.glob('*.gif')
['1.gif', 'card.gif']
>>> glob.glob('?.gif')
['1.gif']
>>> glob.glob('**/*.txt', recursive=True)
['2.txt', 'sub/3.txt']
>>> glob.glob('./**/', recursive=True)
['./', './sub/']
```

Se o diretório contém arquivos começando com `.` eles não serão correspondidos por padrão. Por exemplo, considere um diretório contendo

`card.gif` e `.card.gif`

```
>>> import glob
>>> glob.glob('*.gif')
['card.gif']
>>> glob.glob('.c*')
['.card.gif']
```

Ver também

O módulo **`fnmatch`** oferece expansão de nome de arquivo (não caminho) no estilo shell.

Ver também

O módulo **`pathlib`** oferece objetos de caminho de alto nível.

`fnmatch` — Correspondência de padrões de nome de arquivo Unix

Código-fonte: Lib/fnmatch.py
[https://github.com/python/cpython/tree/3.15/Lib/fnmatch.py]

Este módulo fornece suporte para curingas no estilo shell do Unix, que *não* são iguais às expressões regulares (documentadas no módulo `re`). Os caracteres especiais usados nos curingas no estilo de shell são:

Padrão	Significado
`*`	corresponde a tudo
`?`	Corresponde a qualquer caractere único
`[seq]`	corresponde a qualquer caractere em *seq*
`[!seq]`	corresponde a qualquer caractere ausente em *seq*

Para uma correspondência literal, coloque os metacaracteres entre colchetes. Por exemplo, `'[?]'` corresponde ao caractere `'?'`.

Note que o separador de nome de arquivo (`'/'` no Unix) *não* é especial para este módulo. Veja o módulo `glob` para expansão do nome do caminho (`glob` usa `filter()` para corresponder aos segmentos do nome do caminho). Da mesma forma, os nomes de arquivos que começam com um ponto final não são especiais para este módulo e são correspondidos pelos padrões `*` e `?`.

A menos que indicado de outra forma, “string de nome de arquivo” e “string de padrão” referem-se a objetos `str` ou `bytes` codificados em `ISO-8859-1`. Observe que as funções documentadas abaixo não permitem misturar um padrão **`bytes`** com um nome de arquivo **`str`** e vice-versa.

Por fim, observe que `@functools.lru_cache` com um *maxsize* de 32768 é usado para armazenar em cache os padrões regex compilados (tipados) nas seguintes funções: `fnmatch()`, `fnmatchcase()`, `filter()`, `filterfalse()`.

fnmatch.fnmatch(*name*, *pat*)

Testa se a string do nome de arquivo *name* corresponde à string de padrão *pat*, retornando `True` ou `False`. Ambos os parâmetros são normalizados em maiúsculas e minúsculas usando `os.path.normcase()`. `fnmatchcase()` pode ser usado para realizar uma comparação com distinção entre maiúsculas e minúsculas, independentemente de ser padrão para o sistema operacional.

Este exemplo vai exibir todos os nomes de arquivos no diretório atual com a extensão `.txt`:

```
import fnmatch
import os

for arquivo in os.listdir('.'):
    if fnmatch.fnmatch(arquivo, '*.txt'):
        print(arquivo)
```

fnmatch.fnmatchcase(*name*, *pat*)

Testa se a string do nome de arquivo *name* corresponde à string de padrão *pat*, retornando `True` ou `False`; a comparação diferencia maiúsculas de minúsculas e não se aplica a `os.path.normcase()`.

fnmatch.filter(*names*, *pat*)

Constrói uma lista a partir daqueles elementos do iterável de strings de nome de arquivo *names* que correspondem à string de padrão *pat*. É o mesmo que `[n for n in names if fnmatch(n, pat)]`, mas implementado com mais eficiência.

fnmatch.filterfalse(*names*, *pat*)

Constrói uma lista a partir daqueles elementos do iterável de strings de nome de arquivo *names* que não correspondem à string de padrão *pat*. É o mesmo que `[n for n in names if not fnmatch(n, pat)]`, mas implementado com mais eficiência.

Adicionado na versão 3.14.

fnmatch.translate(*pat*)

Retorna o padrão *pat* no estilo shell convertido em uma expressão regular para usar com `re.prefixmatch()`. O padrão é esperado ser um `str`.

Exemplo:

```
>>> import fnmatch, re
>>>
>>> regex = fnmatch.translate('*.txt')
>>> regex
'(?s:.*\\.txt)\\z'
>>> reobj = re.compile(regex)
>>> reobj.prefixmatch('foobar.txt')
<re.Match object; span=(0, 10), match='foobar.txt'>
```

Ver também

Módulo `glob`

Expansão de caminho no estilo shell do Unix.

`linecache` — Acesso aleatório a linhas de texto

Código-fonte: Lib/linecache.py
[https://github.com/python/cpython/tree/3.15/Lib/linecache.py]

O módulo **`linecache`** permite obter qualquer linha de um arquivo fonte Python, enquanto tenta otimizar internamente, usando um cache, o caso comum em que muitas linhas são lidas em um único arquivo. Isso é usado pelo módulo **`traceback`** para recuperar as linhas de origem para inclusão no traceback (situação da pilha de execução) formatado.

A função **`tokenize.open()`** é usada para abrir arquivos. Esta função usa **`tokenize.detect_encoding()`** para obter a codificação do arquivo; na ausência de um token de codificação, o padrão de codificação do arquivo é UTF-8.

O módulo **`linecache`** define as seguintes funções:

linecache.getline(*filename*, *lineno*, *module_globals=None*)

Obtém a linha *lineno* do arquivo chamado *filename*. Essa função nunca levanta uma exceção — ela retornará `''` em erros (o caractere de nova linha final será incluído para as linhas encontradas).

Se *filename* indicar um módulo congelado (começando com `'<frozen '`), a função tentará obter o nome real do arquivo de `module_globals['__file__']` se *module_globals* não for `None`.

Se um arquivo chamado *filename* não for encontrado, a função primeiro verifica por um `__loader__` da **PEP 302** [https://peps.python.org/pep-0302/] em *module_globals*. Se existe um carregador e ele define um método `get_source`, isso determina as linhas fonte (se `get_source()` retornar `None`, então `''` será retornado). Por fim, se *filename* for um nome de

arquivo relativo, ele será procurado em relação às entradas no caminho de pesquisa do módulo, `sys.path`.

Alterado na versão 3.14: Suporte a *filename* de módulos congelados.

linecache.clearcache()

Limpa o cache. Use esta função se você não precisar mais de linhas de arquivos lidos anteriormente usando **`getline()`**.

linecache.checkcache(*filename=None*)

Verifica a validade do cache. Use esta função se os arquivos no cache tiverem sido alterados no disco e você precisar da versão atualizada. Se *filename* for omitido, ele verificará todas as entradas no cache.

linecache.lazycache(*filename*, *module_globals*)

Captura detalhes suficientes sobre um módulo não baseado em arquivo para permitir obter suas linhas posteriormente via **`getline()`** mesmo se *module_globals* for `None` na chamada posterior. Isso evita a execução de E/S até que uma linha seja realmente necessária, sem ter que carregar o módulo global indefinidamente.

Adicionado na versão 3.5.

Exemplo:

```
>>> import linecache
>>> linecache.getline(linecache.__file__, 8)
'import sys\n'
```

`shutil` — Operações de arquivo de alto nível

Código-fonte: Lib/shutil.py [https://github.com/python/cpython/tree/3.15/Lib/shutil.py]

The `shutil` module offers a number of high-level operations on files and collections of files. In particular, functions are provided which support file copying and removal. For operations on individual files, see also the `os` module.

Aviso

Mesmo as funções de cópia de arquivos de nível mais alto (`shutil.copy()`, `shutil.copy2()`) não podem copiar todos os metadados do arquivo.

Nas plataformas POSIX, isso significa que o proprietário e o grupo do arquivo são perdidos, bem como as ACLs. No Mac OS, a bifurcação de recursos e outros metadados não são usados. Isso significa que os recursos serão perdidos e o tipo de arquivo e os códigos do criador não estarão corretos. No Windows, os proprietários de arquivos, ACLs e fluxos de dados alternativos não são copiados.

Operações de diretório e arquivos

shutil.copyfileobj(*fsrc*, *fdst*[, *length*])

Copia o conteúdo do objeto arquivo ou similar *fsrc* para o objeto arquivo ou similar *fdst*. O número inteiro *length*, se fornecido, é o tamanho do buffer. Em particular, um valor negativo *length* significa copiar os dados sem repetir os dados de origem em pedaços; por padrão, os dados são lidos em pedaços para evitar o consumo descontrolado de

memória. Observe que, se a posição atual do arquivo do objeto *fsrc* não for 0, apenas o conteúdo da posição atual do arquivo até o final do arquivo será copiado.

`copyfileobj()` *não* garante que o fluxo de destino tenha sido liberado após a conclusão da cópia. Se você quiser ler do destino após a conclusão da operação de cópia (por exemplo, ler o conteúdo de um arquivo temporário copiado de um fluxo HTTP), certifique-se de ter chamado `flush()` ou `close()` no objeto arquivo ou similar antes de tentar ler o arquivo de destino.

shutil.copyfile(*src*, *dst*, *, *follow_symlinks=True*)

Copia o conteúdo (sem metadados) do arquivo chamado *src* para um arquivo chamado *dst* e retorna *dst* da maneira mais eficiente possível. *src* e *dst* são objetos caminho ou similares ou nomes de caminhos dados como strings.

dst deve ser o nome completo do arquivo de destino; veja `copy()` para uma cópia que aceita um caminho de diretório de destino. Se *src* e *dst* especificarem o mesmo arquivo, `SameFileError` será levantada.

O local de destino deve ser gravável; caso contrário, uma exceção `OSError` será levantada. Se o *dst* já existir, ele será substituído. Arquivos especiais como dispositivos de caractere ou bloco e encadeamentos (pipe) não podem ser copiados com esta função.

Se *follow_symlinks* for falso e *src* for um link simbólico, um novo link simbólico será criado em vez de copiar o arquivo *src* para o qual o arquivo aponta.

Levanta um evento de auditoria `shutil.copyfile` com os argumentos `src`, `dst`.

Alterado na versão 3.3: `IOError` costumava ser levantada em vez de `OSError`. Adicionado argumento *follow_symlinks*. Agora retorna *dst*.

Alterado na versão 3.4: Levanta `SameFileError` em vez de `Error`. Como a primeira é uma subclasse da última, essa alteração é compatível

com versões anteriores.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção Operações de cópia eficientes dependentes da plataforma.

exception shutil.SpecialFileError

Esta exceção é levantada quando **`copyfile()`** ou **`copytree()`** tenta copiar um pipe nomeado.

Adicionado na versão 2.7.

exception shutil.SameFileError

Essa exceção é levantada se a origem e o destino em **`copyfile()`** forem o mesmo arquivo.

Adicionado na versão 3.4.

shutil.copymode(*src*, *dst*, ***, *follow_symlinks=True*)

Copia os bits de permissão de *src* para *dst*. O conteúdo do arquivo, o proprietário e o grupo não são afetados. *src* e *dst* são objetos caminho ou similares ou nomes de caminhos dados como strings. Se *follow_symlinks* for falso e *src* e *dst* forem links simbólicos, **`copymode()`** tentará modificar o modo do *dst* (e não o arquivo para o qual ele aponta). Essa funcionalidade não está disponível em todas as plataformas; veja **`copystat()`** para mais informações. Se **`copymode()`** não puder modificar links simbólicos na plataforma local, e for solicitado a fazê-lo, não fará nada e retornará.

Levanta um evento de auditoria `shutil.copymode` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks*.

shutil.copystat(*src*, *dst*, ***, *follow_symlinks=True*)

Copia os bits de permissão, hora do último acesso, hora da última modificação e sinalizadores de *src* para *dst*. No Linux, **`copystat()`** também copia os “atributos estendidos” sempre que possível. O conteúdo do arquivo, o proprietário e o grupo não são afetados. *src* e *dst* são objetos caminho ou similares ou nomes de caminhos dados como strings.

Se *follow_symlinks* for falso e *src* e *dst* se referirem a links simbólicos, **`copystat()`** operará nos próprios links simbólicos, e não nos arquivos aos quais os links simbólicos se referem - lendo as informações do link simbólico *src* e gravando as informações no link simbólico *dst*.

Nota

Nem todas as plataformas oferecem a capacidade de examinar e modificar links simbólicos. O próprio Python pode dizer qual funcionalidade está disponível localmente.

- Se `os.chmod in os.supports_follow_symlinks` for `True`, **`copystat()`** pode modificar os bits de permissão de um link simbólico.
- Se `os.utime in os.supports_follow_symlinks` for `True`, **`copystat()`** pode modificar as horas da última modificação e do último acesso de um link simbólico.
- Se `os.chflags in os.supports_follow_symlinks` for `True`, **`copystat()`** pode modificar os sinalizadores de um link simbólico. (`os.chflags` não está disponível em todas as plataformas.)

Nas plataformas em que algumas ou todas essas funcionalidades não estão disponíveis, quando solicitado a modificar um link simbólico, **`copystat()`** copiará tudo o que puder. **`copystat()`** nunca retorna falha.

Por favor, veja **`os.supports_follow_symlinks`** para mais informações.

Levanta um evento de auditoria `shutil.copystat` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks* e suporte a atributos estendidos do Linux.

shutil.copy(*src*, *dst*, ***, *follow_symlinks=True*)

Copia o arquivo *src* para o arquivo ou diretório *dst*. *src* e *dst* devem ser objetos arquivo ou similares ou strings. Se *dst* especificar um diretório, o arquivo será copiado para *dst* usando o nome de arquivo base de *src*. Se *dst* especifica um arquivo que já existe, ele será substituído. Retorna o caminho para o arquivo recém-criado.

Se *follow_symlinks* for falso e *src* for um link simbólico, *dst* será criado como um link simbólico. Se *follow_symlinks* for verdadeiro e *src* for um link simbólico, *dst* será uma cópia do arquivo ao qual *src* se refere.

`copy()` copia os dados do arquivo e o modo de permissão do arquivo (consulte **`os.chmod()`**). Outros metadados, como os tempos de criação e modificação do arquivo, não são preservados. Para preservar todos os metadados do arquivo do original, use **`copy2()`**.

Levanta um evento de auditoria `shutil.copyfile` com os argumentos `src`, `dst`.

Levanta um evento de auditoria `shutil.copymode` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks*. Agora retorna o caminho para o arquivo recém-criado.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção Operações de cópia eficientes dependentes da plataforma.

shutil.copy2(*src*, *dst*, ***, *follow_symlinks=True*)

Idêntico a **copy()**, exceto que **copy2()** também tenta preservar os metadados do arquivo.

Quando *follow_symlinks* for falso e *src* for um link simbólico, **copy2()** tenta copiar todos os metadados do link simbólico *src* para o link simbólico *dst* recém-criado. No entanto, essa funcionalidade não está disponível em todas as plataformas. Nas plataformas em que algumas ou todas essas funcionalidades não estão disponíveis, **copy2()** preservará todos os metadados que puder; **copy2()** nunca levanta uma exceção porque não pode preservar os metadados do arquivo.

copy2() usa **copystat()** para copiar os metadados do arquivo. Por favor, veja **copystat()** para obter mais informações sobre o suporte da plataforma para modificar os metadados do link simbólico.

Levanta um evento de auditoria `shutil.copyfile` com os argumentos `src`, `dst`.

Levanta um evento de auditoria `shutil.copystat` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionado argumento *follow_symlinks*, tenta copiar também atributos estendidos do sistema de arquivos (atualmente apenas no Linux). Agora retorna o caminho para o arquivo recém-criado.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção Operações de cópia eficientes dependentes da plataforma.

shutil.ignore_patterns(**patterns*)

Esta função de fábrica cria uma função que pode ser usada como um chamável para o argumento *ignore* de **copytree()**, ignorando arquivos e diretórios que correspondem a um dos padrões *patterns* de estilo glob fornecidos. Veja o exemplo abaixo.

shutil.copytree(*src*, *dst*, *symlinks=False*, *ignore=None*, *copy_function=copy2*, *ignore_dangling_symlinks=False*, *dirs_exist_ok=False*)

Copia recursivamente uma árvore de diretórios inteira com raiz em *src* para um diretório chamado *dst* e retorna o diretório de destino. Todos os diretórios intermediários necessários para conter *dst* também serão criados por padrão.

Permissões e horas dos diretórios são copiados com `copystat()`, arquivos individuais são copiados usando `copy2()`.

Se *symlinks* for verdadeiro, os links simbólicos na árvore de origem são representados como links simbólicos na nova árvore e os metadados dos links originais serão copiados na medida do permitido pela plataforma; se falso ou omitido, o conteúdo e os metadados dos arquivos vinculados são copiados para a nova árvore.

Quando *symlinks* for falso, se o arquivo apontado pelo link simbólico não existir, uma exceção será adicionada na lista de erros levantada em uma exceção `Error` no final do processo de cópia. Você pode definir o sinalizador opcional *ignore_dangling_symlinks* como true se desejar silenciar esta exceção. Observe que esta opção não tem efeito em plataformas que não possuem suporte a `os.symlink()`.

Se *ignore* for fornecido, deve ser um chamável que receberá como argumento o diretório que está sendo visitado por `copytree()`, e uma lista de seu conteúdo, retornada por `os.listdir()`. Como **`copytree()`** é chamada recursivamente, o chamável *ignore* será chamado uma vez para cada diretório que é copiado. O chamável deve retornar uma sequência de nomes de diretório e arquivo em relação ao diretório atual (ou seja, um subconjunto dos itens em seu segundo argumento); esses nomes serão ignorados no processo de cópia. `ignore_patterns()` pode ser usado para criar um chamável que ignore nomes com base em padrões de estilo glob.

Se uma ou mais exceções ocorrerem, uma `Error` é levantada com uma lista dos motivos.

Se *copy_function* for fornecida, deverá ser um chamável que será usado para copiar cada arquivo. Ele será chamado com o caminho de origem e o caminho de destino como argumentos. Por padrão, `copy2()` é usada, mas qualquer função que possua suporte à mesma assinatura (como `copy()`) pode ser usada.

Se *dirs_exist_ok* for false (o padrão) e *dst* já existir, uma `FileExistsError` será levantada. Se *dirs_exist_ok* for true, a operação de cópia continuará se encontrar diretórios existentes, e os arquivos dentro da árvore *dst* serão sobrescritos pelos arquivos correspondentes da árvore *src*.

Levanta um evento de auditoria `shutil.copytree` com os argumentos `src`, `dst`.

Alterado na versão 3.2: Adicionado o argumento *copy_function* para poder fornecer uma função de cópia personalizada. Adicionado o argumento *ignore_dangling_symlinks* para silenciar erros de links simbólicos quando *symlinks* for falso.

Alterado na versão 3.3: Copia metadados quando *symlinks* for falso. Agora, retorna *dst*.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção Operações de cópia eficientes dependentes da plataforma.

Alterado na versão 3.8: Adicionado o parâmetro *dirs_exist_ok*.

shutil.rmtree(*path*, *ignore_errors=False*, *onerror=None*, *, *onexc=None*, *dir_fd=None*)

Exclui uma árvore de diretório inteira; *path* deve apontar para um diretório (mas não um link simbólico para um diretório). Se *ignore_errors* for verdadeiro, os erros resultantes de remoções com falha serão ignorados; se falso ou omitido, tais erros são tratados chamando um manipulador especificado por *onexc* ou *onerror* ou, se ambos forem omitidos, exceções são propagadas para o chamador.

Esta função tem suporte a caminhos relativos para descritores de diretório.

Nota

Em plataformas que suportam as funções baseadas em descritores de arquivo necessárias, uma versão resistente a ataques de links simbólicos de `rmtree()` é usada por padrão. Em outras plataformas, a implementação `rmtree()` é suscetível a um ataque de link simbólico: dados o tempo e as circunstâncias apropriados, os invasores podem manipular links simbólicos no sistema de arquivos para excluir arquivos que eles não seriam capazes de acessar de outra forma. Os aplicativos podem usar o atributo de função `rmtree.avoids_symlink_attacks` para determinar qual caso se aplica.

Se *onexc* for fornecido, deve ser um chamável que aceite três parâmetros: *function*, *path*, e *excinfo*.

O primeiro parâmetro, *function*, é a função que levantou a exceção; depende da plataforma e da implementação. O segundo parâmetro, *path*, será o nome do caminho passado para a *função*. O terceiro parâmetro, *excinfo*, é a exceção que foi levantada. Exceções levantadas por *onexc* não serão capturadas.

O descontinuado *onerror* é semelhante ao *onexc*, exceto que o terceiro parâmetro que ele recebe é a tupla retornada de `sys.exc_info()`.

Ver também

exemplo rmtree para um exemplo de tratamento da remoção de uma árvore de diretórios que contém arquivos somente leitura.

Levanta um evento de auditoria `shutil.rmtree` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionada uma versão resistente a ataques de link simbólico que é usada automaticamente se a plataforma suportar funções baseadas em descritor de arquivo.

Alterado na versão 3.8: No Windows, não excluirá mais o conteúdo de uma junção de diretório antes de remover a junção.

Alterado na versão 3.11: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.12: Adicionado o parâmetro *onexc*, descontinuado *onerror*.

Alterado na versão 3.13: **`rmtree()`** agora ignora exceções **`FileNotFoundError`** para todos, exceto o caminho de nível superior. Exceções diferentes de **`OSError`** e subclasses de **`OSError`** agora são sempre propagadas para o chamador.

rmtree.avoids_symlink_attacks

Indica se a plataforma e implementação atuais fornecem uma versão resistente a ataques de link simbólico de **`rmtree()`**. Atualmente, isso só é verdade para plataformas que suportam funções de acesso ao diretório baseadas em descritor de arquivo.

Adicionado na versão 3.3.

shutil.move(*src*, *dst*, *copy_function=copy2*)

Move recursivamente um arquivo ou diretório (*src*) para outro local e retorna ao destino.

Se *dst* for um diretório existente ou um symlink para um diretório, então *src* é movido para dentro desse diretório. O caminho de destino nesse diretório não deve existir ainda.

Se *dst* já existir, mas não for um diretório, ele poderá ser substituído dependendo da semântica de **`os.rename()`**.

`os.rename()` is preferably used internally when *src* and the destination are on the same filesystem. In case **`os.rename()`** fails due to **`OSError`**

(e.g. the user has write permission to the destination file but not to its parent directory), this method falls back to using *copy_function*, in which case *src* is copied to the destination using *copy_function* and then removed.

In case of symlinks, a new symlink pointing to the target of *src* will be created in or as the destination, and *src* will be removed.

Se *copy_function* for fornecido, deve ser um chamável que leva dois argumentos *src* e a destinação, e será usado para copiar *src* para a destinação se `os.rename()` não puder ser usado. Se a fonte for um diretório, `copytree()` é chamado, passando *copy_function*. A *copy_function* padrão é `copy2()`. Usar `copy()` como *copy_function* permite que a movimentação seja bem-sucedida quando não é possível copiar também os metadados, às custas de não copiar nenhum dos metadados.

Levanta um evento de auditoria `shutil.move` com os argumentos `src`, `dst`.

Alterado na versão 3.3: Adicionada manipulação de links simbólicos explícitos para sistemas de arquivos externos, adaptando-os ao comportamento do GNU **mv**. Agora retorna *dst*.

Alterado na versão 3.5: Adicionado o argumento nomeado *copy_function*.

Alterado na versão 3.8: As chamadas de sistema de cópia rápida específicas da plataforma podem ser usadas internamente para copiar o arquivo com mais eficiência. Veja a seção Operações de cópia eficientes dependentes da plataforma.

Alterado na versão 3.9: Aceita um objeto caminho ou similar para *src* e *dst*.

shutil.disk_usage(*path*)

Retorna estatísticas de uso de disco sobre o caminho fornecido como tupla nomeada com os atributos *total*, *used* e *free*, que são a quantidade

de espaço total, usado e livre, em bytes. *path* pode ser um arquivo ou diretório.

Nota

Em sistemas de arquivos Unix, *path* deve apontar para um caminho dentro de uma partição de sistema de arquivos **montada**. Nessas plataformas, o CPython não tenta recuperar informações de uso de disco de sistemas de arquivos não montados.

Adicionado na versão 3.3.

Alterado na versão 3.8: No Windows, *path* pode agora ser um arquivo ou diretório.

Disponibilidade: Unix, Windows.

shutil.chown(*path*, *user=None*, *group=None*, *, *dir_fd=None*, *follow_symlinks=True*)

Altera o proprietário *usuário* e/ou *group* do *path* fornecido.

user pode ser um nome de usuário do sistema ou um uid; o mesmo se aplica ao *group*. É necessário pelo menos um argumento.

Veja também `os.chown()`, a função subjacente.

Levanta um evento de auditoria `shutil.chown` com os argumentos `path`, `user`, `group`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Alterado na versão 3.13: Adicionados os parâmetros *dir_fd* e *follow_symlinks*.

shutil.which(*cmd*, *mode=os.F_OK | os.X_OK*, *path=None*)

Retorna o caminho para um executável que seria executado se o *cmd* fornecido fosse chamado. Se nenhum *cmd* for chamado, retorna `None`.

mode é uma máscara de permissão passada para **`os.access()`**, por padrão determinando se o arquivo existe e é executável.

path é uma "string `PATH`" que especifica os diretórios a serem procurados, delimitados por **`os.pathsep`**. Quando nenhum *path* é especificado, a variável de ambiente **`PATH`** é lida de **`os.environ`**, retornando para **`os.defpath`** se não estiver definida.

Se *cmd* contiver um componente de diretório, **`which()`** verificará apenas o caminho especificado diretamente e não pesquisará os diretórios listados em *path* ou na variável de ambiente **`PATH`** do sistema.

No Windows, o diretório atual é prefixado ao *path* se o *mode* não incluir `os.X_OK`. Quando o *mode* inclui `os.X_OK`, a API do Windows `NeedCurrentDirectoryForExePathW` será consultada para determinar se o diretório atual deve ser prefixado ao *path*. Para evitar consultar o diretório de trabalho atual para executáveis: defina a variável de ambiente `NoDefaultCurrentDirectoryInExePath`.

Também no Windows, a variável de ambiente **`PATHEXT`** é usada para resolver comandos que podem não incluir uma extensão. Por exemplo, se você chamar `shutil.which("python")`, **`which()`** pesquisará `PATHEXT` para saber que deve procurar por `python.exe` dentro dos diretórios *path*. Por exemplo, no Windows:

```
>>> shutil.which("python")
'C:\\Python33\\python.EXE'
```

Isso também se aplica quando *cmd* é um caminho que contém um componente de diretório:

```
>>> shutil.which("C:\\Python33\\python")
'C:\\Python33\\python.EXE'
```

Adicionado na versão 3.3.

Alterado na versão 3.8: O tipo **bytes** é agora aceitado. Se o tipo de *cmd* é **bytes**, o tipo resultante também é **bytes**.

Alterado na versão 3.12: No Windows, o diretório atual não é mais adicionado ao caminho de pesquisa se *mode* incluir `os.X_OK` e WinAPI `NeedCurrentDirectoryForExePathW(cmd)` for falso, caso contrário, o diretório atual será adicionado mesmo se já estiver no caminho de pesquisa; `PATHEXT` agora é usado mesmo quando *cmd* inclui um componente de diretório ou termina com uma extensão que está em `PATHEXT`; e nomes de arquivos que não têm extensão agora podem ser encontrados.

exception shutil.Error

Subclass of **OSError** collecting exceptions raised during a multi-file operation. For **copytree()**, the exception argument is a list of 3-tuples (*srcname*, *dstname*, *exception*).

Operações de cópia eficientes dependentes da plataforma

A partir do Python 3.8, todas as funções envolvendo uma cópia de arquivo (**copyfile()**, **copy()**, **copy2()**, **copytree()** e **move()**) podem usar chamadas do sistema de "cópia rápida" específicas da plataforma para copiar o arquivo de forma mais eficiente (veja bpo-33671 [https://bugs.python.org/issue?@action=redirect&bpo=33671]). "cópia rápida" significa que a operação de cópia ocorre dentro do kernel, evitando o uso de buffers de espaço de usuário em Python como em "`outfd.write(infd.read())`".

No macOS, fcopyfile [http://www.manpagez.com/man/3/copyfile/] é usado para copiar o conteúdo do arquivo (não metadados).

No Linux, **os.copy_file_range()** ou **os.sendfile()** é usado.

No Solaris, **os.sendfile()** é usado.

On Windows **shutil.copyfile()** uses a bigger default buffer size (1 MiB instead of 64 KiB) and a **memoryview()**-based variant of **shutil.copyfileobj()** is used, which still reads and writes in a loop.

`shutil.copy2()` uses the native `CopyFile2` call on Windows, which is the most efficient method, supports copy-on-write, and preserves metadata.

If the fast-copy operation fails and no data was written in the destination file then shutil will silently fall back to less efficient `copyfileobj()` function internally.

Alterado na versão 3.8.

Alterado na versão 3.14: Solaris agora usa `os.sendfile()`.

Alterado na versão 3.14: A cópia na gravação (copy-on-write) ou cópia do lado do servidor podem ser usadas internamente via `os.copy_file_range()` em sistemas de arquivos Linux suportados.

Exemplo de copytree

Um exemplo que usa o auxiliar `ignore_patterns()`:

```
from shutil import copytree, ignore_patterns

copytree(source, destination, ignore=ignore_patterns('*.pyc', 't
```

Isso irá copiar tudo, exceto os arquivos `.pyc` e arquivos ou diretórios cujo nome começa com `tmp`.

Outro exemplo que usa o argumento *ignore* para adicionar uma chamada de registro:

```
from shutil import copytree
import logging

def _logpath(path, names):
    logging.info('Working in %s', path)
    return []   # nada será ignorado

copytree(source, destination, ignore=_logpath)
```

exemplo rmtree

Este exemplo mostra como remover uma árvore de diretório no Windows onde alguns dos arquivos têm seu conjunto de bits somente leitura. Ele usa a função de retorno onexc para limpar o bit somente leitura e tentar remover novamente. Qualquer falha subsequente se propagará.

```
import os, stat
import shutil

def remove_readonly(func, path, _):
    "Clear the readonly bit and reattempt the removal"
    os.chmod(path, stat.S_IWRITE)
    func(path)

shutil.rmtree(directory, onexc=remove_readonly)
```

Operações de arquivamento

Adicionado na versão 3.2.

Alterado na versão 3.5: Adicionado suporte ao formato *xztar*.

Utilitários de alto nível para criar e ler arquivos compactados e arquivados também são fornecidos. Eles contam com os módulos **zipfile** e **tarfile**.

shutil.make_archive(*base_name*, *format*[, *root_dir*[, *base_dir*[, *verbose*[, *dry_run*[, *owner*[, *group*[, *logger*]]]]]]])

Cria um arquivo compactado (como zip ou tar) e retorna seu nome.

base_name is a string or path-like object specifying the name of the file to create, including the path, minus any format-specific extension.

format é o formato do arquivo: um de “zip” (se o módulo **zlib** estiver disponível), “tar”, “gztar” (se o módulo **zlib** estiver disponível), “bztar” (se o módulo **bz2** estiver disponível), “xztar” (se o módulo **lzma** estiver disponível) ou “zstdtar” (se o módulo **compression.zstd** estiver disponível).

root_dir is a string or path-like object specifying a directory that will be the root directory of the archive, all paths in the archive will be relative to it; for example, we typically chdir into *root_dir* before creating the archive.

base_dir is a string or path-like object specifying a directory where we start archiving from; i.e. *base_dir* will be the common prefix of all files and directories in the archive. *base_dir* must be given relative to *root_dir*. See Exemplo de arquivamento com base_dir for how to use *base_dir* and *root_dir* together.

root_dir e *base_dir* têm com padrão o diretório atual.

Se *dry_run* for verdadeiro, nenhum arquivo é criado, mas as operações que seriam executadas são registradas no *logger*.

owner e *group* são usados ao criar um arquivo tar. Por padrão, usa o proprietário e grupo atuais.

logger deve ser um objeto compatível com a **PEP 282** [https://peps.python.org/pep-0282/], geralmente uma instância de `logging.Logger`.

O argumento *verbose* não é usado e foi descontinuado.

Levanta um evento de auditoria `shutil.make_archive` com os argumentos `base_name`, `format`, `root_dir`, `base_dir`.

Nota

Esta função não é segura para thread quando arquivadores personalizados registrados com `register_archive_format()` não suportam o argumento *root_dir*. Neste caso, ele altera temporariamente o diretório de trabalho atual do processo para *root_dir* para executar o arquivamento.

Alterado na versão 3.8: O formato pax moderno (POSIX.1-2001) agora é usado em vez do formato GNU legado para arquivos criados com

`format="tar"`.

Alterado na versão 3.10.6: Esta função agora é segura para thread durante a criação de arquivos `.zip` e tar padrão.

Alterado na versão 3.15: Accepts a path-like object for *base_name*, *root_dir* and *base_dir*.

shutil.get_archive_formats()

Retorna uma lista de formatos suportados para arquivamento. Cada elemento da sequência retornada é uma tupla `(nome, descrição)`.

By default **`shutil`** provides these formats:

- *zip*: arquivo ZIP (se o módulo **`zlib`** estiver disponível).
- *tar*: Arquivo tar não compactado. Usa o formato POSIX.1-2001 pax para novos arquivos.
- *gztar*: arquivo tar compactado com gzip (se o módulo **`zlib`** estiver disponível).
- *bztar*: arquivo tar compactado com bzip2 (se o módulo **`bz2`** estiver disponível).
- *xztar*: Arquivo tar compactado com xz (se o módulo **`lzma`** estiver disponível).
- *zstdtar*: arquivo tar compactado com Zstandard (se o módulo **`compression.zstd`** estiver disponível).

Você pode registrar novos formatos ou fornecer seu próprio arquivador para quaisquer formatos existentes, usando **`register_archive_format()`**.

shutil.register_archive_format(*name*, *function*[, *extra_args*[, *description*]])

Registra um arquivador para o formato *name*.

function is the callable that will be used to create archives. The callable will receive the *base_name* of the file to create, followed by the *base_dir* (which defaults to **`os.curdir`**) to start archiving from. Further arguments are passed as keyword arguments: *owner*, *group*, *dry_run* and *logger* (as passed in **`make_archive()`**).

Se *function* tiver o atributo personalizado `function.supports_root_dir` definido como `True`, o argumento *root_dir* será passado como um argumento nomeado. Caso contrário, o diretório de trabalho atual do processo será temporariamente alterado para *root_dir* antes de chamar *function*. Nesse caso, **`make_archive()`** não é segura para thread.

Se fornecido, *extra_args* é uma sequência de pares `(nome, valor)` que serão usados como argumentos nomeados extras quando o arquivador chamável for usado.

description é usado por **`get_archive_formats()`** que retorna a lista de arquivadores. O padrão é uma string vazia.

Alterado na versão 3.12: Adicionado suporte para funções que suportam o argumento *root_dir*.

shutil.unregister_archive_format(*name*)

Remove o formato de arquivo *name* da lista de formatos suportados.

shutil.unpack_archive(*filename*[, *extract_dir*[, *format*[, *filter*]]])

Desempacota um arquivo. *filename* é o caminho completo do arquivo.

extract_dir é o nome do diretório de destino onde o arquivo é desempacotado. Se não for fornecido, o diretório de trabalho atual será usado.

format é o formato do arquivo: um de “zip”, “tar”, “gztar”, “bztar”, “xztar” e “zstdtar”. Ou qualquer outro formato registrado com **`register_unpack_format()`**. Se não for fornecido, **`unpack_archive()`** irá usar a extensão do nome do arquivo e ver se um desempacotador foi registrado para essa extensão. Caso nenhum seja encontrado, uma **`ValueError`** é levantada.

O argumento somente-nomeado *filter* é passado para a função de desempacotamento subjacente. Para arquivos zip, *filter* não é aceito. Para arquivos tar, é recomendado usar `'data'` (padrão desde o Python

3.14), a menos que esteja usando recursos específicos para tar e sistemas de arquivos do tipo UNIX. (Veja Filtros de extração para detalhes.)

Levanta um evento de auditoria `shutil.unpack_archive` com os argumentos `filename`, `extract_dir`, `format`.

Aviso

Never extract archives from untrusted sources without prior inspection. It is possible that files are created outside of the path specified in the *extract_dir* argument, for example, members that have absolute filenames or filenames with “..” components.

Desde o Python 3.14, os padrões para ambos os formatos embutidos (arquivos zip e tar) evitarão os problemas de segurança mais perigosos, mas não impedirão *todos* os comportamentos indesejados. Leia a seção Dicas para verificação adicional para detalhes específicos do tar.

Alterado na versão 3.7: Aceita um objeto caminho ou similar para *filename* e *extract_dir*.

Alterado na versão 3.12: Adicionado o argumento *filter*.

shutil.register_unpack_format(*name*, *extensions*, *function*[, *extra_args*[, *description*]])

Registra um formato de desempacotamento. *name* é o nome do formato e *extensions* é uma lista de extensões correspondentes ao formato, como `.zip` para arquivos Zip.

function é o chamável que será usado para desempacotar arquivos. O chamável receberá:

- o caminho do arquivo, como argumento posicional;
- o diretório para o qual o arquivo deve ser extraído, como um argumento posicional;

- possivelmente um argumento nomeado *filter*, se foi dado a `unpack_archive()`;
- argumentos nomeados adicionais, especificados por *extra_args* como uma sequência de tuplas `(nome, valor)`.

description pode ser fornecido para descrever o formato e será devolvido pela função `get_unpack_formats()`.

shutil.unregister_unpack_format(*name*)

Cancela o registro de um formato de desempacotamento. *name* é o nome do formato.

shutil.get_unpack_formats()

Retorna uma lista de todos os formatos registrados para desempacotamento. Cada elemento da sequência retornada é uma tupla `(name, extensions, description)`.

By default **`shutil`** provides these formats:

- *zip*: arquivo ZIP (desempacotar arquivos compactados funciona apenas se o módulo correspondente estiver disponível).
- *tar*: arquivo tar não comprimido.
- *gztar*: arquivo tar compactado com gzip (se o módulo `zlib` estiver disponível).
- *bztar*: arquivo tar compactado com bzip2 (se o módulo `bz2` estiver disponível).
- *xztar*: Arquivo tar compactado com xz (se o módulo `lzma` estiver disponível).
- *zstdtar*: arquivo tar compactado com Zstandard (se o módulo `compression.zstd` estiver disponível).

Você pode registrar novos formatos ou fornecer seu próprio desempacotador para quaisquer formatos existentes, usando `register_unpack_format()`.

Exemplo de arquivo

Neste exemplo, criamos um arquivo tar compactado com gzip contendo todos os arquivos encontrados no diretório `.ssh` do usuário:

```
>>> from shutil import make_archive
>>> import os
>>> archive_name = os.path.expanduser(os.path.join('~', 'meuarqu
>>> root_dir = os.path.expanduser(os.path.join('~', '.ssh'))
>>> make_archive(archive_name, 'gztar', root_dir)
'/Users/tarek/meuarquivo.tar.gz'
```

O arquivo resultante contém:

```
$ tar -tzvf /Users/tarek/meuarquivo.tar.gz
drwx------ tarek/staff       0 2010-02-01 16:23:40 ./
-rw-r--r-- tarek/staff     609 2008-06-09 13:26:54 ./authorized_
-rwxr-xr-x tarek/staff      65 2008-06-09 13:26:54 ./config
-rwx------ tarek/staff     668 2008-06-09 13:26:54 ./id_dsa
-rwxr-xr-x tarek/staff     609 2008-06-09 13:26:54 ./id_dsa.pub
-rw------- tarek/staff    1675 2008-06-09 13:26:54 ./id_rsa
-rw-r--r-- tarek/staff     397 2008-06-09 13:26:54 ./id_rsa.pub
-rw-r--r-- tarek/staff   37192 2010-02-06 18:23:10 ./known_hosts
```

Exemplo de arquivamento com *base_dir*

Neste exemplo, semelhante ao acima, mostramos como usar **`make_archive()`**, mas desta vez com o uso de *base_dir*. Agora temos a seguinte estrutura de diretório:

```
$ tree tmp
tmp
└── root
    └── estrutura
        ├── conteudo
            └── por_favor_adicionar.txt
        └── nao_adicionar.txt
```

No arquivo final, `please_add.txt` deve ser incluído, mas `do_not_add.txt` não deve. Portanto, usamos o seguinte:

```
>>> from shutil import make_archive
>>> import os
>>> archive_name = os.path.expanduser(os.path.join('~', 'meuarqu
>>> make_archive(
...     archive_name,
...     'tar',
...     root_dir='tmp/root',
...     base_dir='estrutura/conteudo',
... )
'/Users/tarek/meu_arquivo.tar'
```

Listar os arquivos no arquivo resultante nos dá:

```
$ python -m tarfile -l /Users/tarek/meuarquivo.tar
estrutura/conteudo/
estrutura/conteudo/por_favor_adicionar.txt
```

Consultando o tamanho do terminal de saída

shutil.get_terminal_size(*fallback=(columns, lines)*)

Obtém o tamanho da janela do terminal.

Para cada uma das duas dimensões, a variável de ambiente, `COLUMNS` e `LINES` respectivamente, é verificada. Se a variável estiver definida e o valor for um número inteiro positivo, ela será usada.

Quando `COLUMNS` ou `LINES` não está definida, que é o caso comum, o terminal conectado a `sys.__stdout__` é consultado invocando `os.get_terminal_size()`.

Se o tamanho do terminal não pode ser consultado com sucesso, ou porque o sistema não tem suporte a consultas, ou porque não estamos conectados a um terminal, o valor dado no parâmetro `fallback` é usado. O padrão de `fallback` é `(80, 24)`, que é o tamanho padrão usado por muitos emuladores de terminal.

O valor retornado é uma tupla nomeada do tipo `os.terminal_size`.

Veja também: The Single UNIX Specification, Versão 2, Other Environment Variables [https://pubs.opengroup.org/onlinepubs/7908799/xbd/envvar.html#tag_002_003].

Adicionado na versão 3.3.

Alterado na versão 3.11: Os valores `fallback` também são usados se **`os.get_terminal_size()`** retornar zeros.

Persistência de Dados

Os módulos descritos neste capítulo possuem suporte ao armazenamento de dados do Python em um formato persistente no disco. Os módulos `pickle` e `marshal` podem transformar muitos tipos de dados do Python em um fluxo de bytes e então recriar os objetos a partir dos bytes. Os vários módulos relacionados ao DBM possuem suporte a uma família de formatos de arquivo baseados em hash que armazenam um mapeamento de strings para outras strings.

A lista de módulos descritos neste capítulo é:

- `pickle` — Serialização de objetos Python
  - Relação com outros módulos Python
    - Comparação com `marshal`
    - Comparação com `json`
  - Formato de fluxo de dados
  - Interface do módulo
  - O que pode ser serializado e desserializado com pickle?
  - Serializando instâncias de classes com pickle
    - Persistência de objetos externos
    - Tabelas de despacho
    - Manipulação de objetos com estado
  - Redução personalizada para tipos, funções e outros objetos
  - Buffers fora da banda
    - API de provedor
    - API de consumidor
    - Exemplo
  - Restringindo globais
  - Desempenho
  - Exemplos
  - Interface de linha de comando
- `copyreg` — Registra funções de suporte `pickle`
  - Exemplo
- `shelve` — Python object persistence

    - Restrições
    - Exemplo
    - Exceções
- `marshal` — Serialização interna de objetos Python
- `dbm` — Interfaces para “banco de dados” Unix
    - `dbm.sqlite3` — SQLite backend for dbm
    - `dbm.gnu` — GNU database manager
    - `dbm.ndbm` — New Database Manager
    - `dbm.dumb` — Portable DBM implementation
- `sqlite3` —- interface DB-API 2.0 interface para bancos de dados SQLite
    - Tutorial
    - Referência
        - Funções do módulo
        - Constantes do módulo
        - Connection objects
        - Cursor objects
        - Row objects
        - Blob objects
        - PrepareProtocol objects
        - Exceções
        - SQLite and Python types
        - Default adapters and converters (deprecated)
        - Interface de linha de comando
    - Guias de como fazer
        - How to use placeholders to bind values in SQL queries
        - How to adapt custom Python types to SQLite values
            - How to write adaptable objects
            - How to register adapter callables
        - How to convert SQLite values to custom Python types
        - Adapter and converter recipes
        - How to use connection shortcut methods
        - How to use the connection context manager
        - How to work with SQLite URIs
        - How to create and use row factories
        - How to handle non-UTF-8 text encodings
    - Explicação

- Controle de transações
  - Controle de transações através do atributo **`autocommit`**
  - Controle de transação através do atributo **`isolation_level`**

`pickle` — Serialização de objetos Python

Código-fonte: Lib/pickle.py [https://github.com/python/cpython/tree/3.15/Lib/pickle.py]

O módulo **`pickle`** implementa protocolos binários para serializar e desserializar uma estrutura de objeto Python. *"Pickling"* é o processo pelo qual uma hierarquia de objetos Python é convertida em um fluxo de bytes, e *"unpickling"* é a operação inversa, em que um fluxo de bytes (de um arquivo binário ou objeto bytes ou similar) é convertido de volta em uma hierarquia de objetos. Pickling (e unpickling) com pickle é alternativamente conhecido como "serialização", "marshalling" [[1]] ou "flattening"; no entanto, para evitar confusão, usa-se os termos "pickling" e "unpickling". Nesta documentação traduzida, usaremos "serialização com pickle" e "desserialização com pickle", respectivamente.

Aviso

O módulo `pickle` **não é seguro**. Desserialize com pickle apenas os dados em que você confia.

É possível construir dados maliciosos em pickle que irão **executar código arbitrário durante o processo de desserialização com pickle**. Nunca desserialize com pickle os dados que possam vir de uma fonte não confiável ou que possam ter sido adulterados.

Considere assinar dados com **`hmac`** se você precisar garantir que eles não foram adulterados.

Formatos de serialização mais seguros como **`json`** podem ser mais apropriados se você estiver processando dados não confiáveis. Vejo Comparação com json.

Relação com outros módulos Python

Comparação com `marshal`

Python tem um módulo de serialização mais primitivo chamado `marshal`, mas em geral `pickle` deve ser sempre a forma preferida de serializar objetos Python. `marshal` existe principalmente para oferecer suporte a arquivos `.pyc` do Python.

O módulo `pickle` difere do `marshal` de várias maneiras significativas:

- `marshal` não pode ser usado para serializar classes definidas pelo usuário e suas instâncias. `pickle` pode salvar e restaurar instâncias de classe de forma transparente, no entanto, a definição de classe deve ser importável e estar no mesmo módulo de quando o objeto foi armazenado.
- O formato de serialização do `marshal` não tem garantia de portabilidade entre as versões do Python. Como sua principal tarefa em vida é oferecer suporte a arquivos `.pyc`, os implementadores do Python se reservam o direito de alterar o formato de serialização de maneiras não compatíveis com versões anteriores, caso haja necessidade. O formato de serialização do `pickle` tem a garantia de ser compatível com versões anteriores em todas as versões do Python, desde que um protocolo pickle compatível seja escolhido e o código de serialização e desserialização com pickle lide com as diferenças dos tipos em Python 2 e em Python 3 se seus dados estiverem cruzando essa fronteira única de alteração.

Comparação com `json`

Existem diferenças fundamentais entre os protocolos pickle e JSON (JavaScript Object Notation) [https://json.org]:

- JSON é um formato de serialização de texto (ele produz texto unicode, embora na maioria das vezes seja codificado para `utf-8`), enquanto pickle é um formato de serialização binário;
- JSON é legível por humanos, enquanto pickle não é;

- JSON é interoperável e amplamente usado fora do ecossistema Python, enquanto pickle é específico para Python;
- JSON, por padrão, só pode representar um subconjunto dos tipos embutidos do Python, e nenhuma classe personalizada; pickle pode representar um número extremamente grande de tipos Python (muitos deles automaticamente, pelo uso inteligente dos recursos de introspecção do Python; casos complexos podem ser resolvidos implementando APIs de objetos específicos);
- Ao contrário do pickle, a desserialização não confiável do JSON não cria, por si só, uma vulnerabilidade de execução de código arbitrário.

Ver também

O módulo `json`: um módulo da biblioteca padrão que permite a serialização e desserialização JSON.

Formato de fluxo de dados

O formato de dados usado pelo `pickle` é específico do Python. Isso tem a vantagem de não haver restrições impostas por padrões externos, como JSON (que não pode representar o compartilhamento de ponteiros); no entanto, isso significa que programas não Python podem não ser capazes de reconstruir objetos Python serializados com pickle.

Por padrão, o formato de dados do `pickle` usa uma representação binária relativamente compacta. Se você precisa de características de tamanho ideal, pode com eficiência comprimir dados processados com pickle.

O módulo `pickletools` contém ferramentas para analisar fluxos de dados gerados por `pickle`. O código-fonte do `pickletools` tem extensos comentários sobre códigos de operações usados por protocolos de pickle.

Existem atualmente 6 protocolos diferentes que podem ser usados para a serialização com pickle. Quanto mais alto o protocolo usado, mais recente é a versão do Python necessária para ler o pickle produzido.

- A versão 0 do protocolo é o protocolo original “legível por humanos” e é compatível com versões anteriores do Python.
- A versão 1 do protocolo é um formato binário antigo que também é compatível com versões anteriores do Python.
- A versão 2 do protocolo foi introduzida no Python 2.3. Ela fornece uma serialização com pickle muito mais eficiente de classes estilo novo. Consulte **PEP 307** [https://peps.python.org/pep-0307/] para obter informações sobre as melhorias trazidas pelo protocolo 2.
- A versão 3 do protocolo foi adicionada ao Python 3.0. Ela tem suporte explícito a objetos `bytes` e não é possível desserializar com pickle a partir do Python 2.x. Este era o protocolo padrão no Python 3.0–3.7.
- A versão 4 do protocolo foi adicionada ao Python 3.4. Ela adiciona suporte para objetos muito grandes, serialização com pickle de mais tipos de objetos e algumas otimizações de formato de dados. Isso é o protocolo padrão no Python 3.8–3.13. Consulte **PEP 3154** [https://peps.python.org/pep-3154/] para obter informações sobre as melhorias trazidas pelo protocolo 4.
- A versão 5 do protocolo foi adicionada ao Python 3.8. Ela adiciona suporte a dados fora da banda e aumento de velocidade para dados dentro da banda. É o protocolo padrão a partir do Python 3.14. Consulte **PEP 574** [https://peps.python.org/pep-0574/] para obter informações sobre as melhorias trazidas pelo protocolo 5.

Nota

A serialização é uma noção mais primitiva do que a persistência; embora o `pickle` leia e escreva objetos de arquivo, ele não lida com a questão de nomear objetos persistentes, nem a questão (ainda mais complicada) de acesso simultâneo a objetos persistentes. O módulo `pickle` pode transformar um objeto complexo em um fluxo de bytes e pode transformar o fluxo de bytes em um objeto com a mesma estrutura interna. Talvez a coisa mais óbvia a fazer com esses fluxos de bytes seja escrevê-los em um arquivo, mas também é concebível enviá-los através de uma rede ou armazená-los em um banco de dados. O módulo `shelve` fornece uma interface simples para serializar e desserializar com pickle os objetos em arquivos de banco de dados no estilo DBM.

Interface do módulo

Para serializar uma hierarquia de objeto, você simplesmente chama a função **dumps()**. Da mesma forma, para desserializar um fluxo de dados, você chama a função **loads()**. No entanto, se você quiser mais controle sobre a serialização e desserialização, pode criar um objeto **Pickler** ou **Unpickler**, respectivamente.

O módulo **pickle** fornece as seguintes constantes:

pickle.HIGHEST_PROTOCOL

Um inteiro, a versão de protocolo mais alta disponível. Este valor pode ser passado como um valor de *protocol* para as funções **dump()** e **dumps()**, bem como para o construtor de **Pickler**.

pickle.DEFAULT_PROTOCOL

Um inteiro, a versão de protocolo padrão usada para a serialização com pickle. Pode ser menor que **HIGHEST_PROTOCOL**. Atualmente, o protocolo padrão é 5, introduzido pela primeira vez no Python 3.8 e incompatível com as versões anteriores. Esta versão introduz suporte para buffers fora da banda, nos quais dados compatíveis com **PEP 3118** [https://peps.python.org/pep-3118/] podem ser transmitidos separadamente do fluxo principal do pickle.

Alterado na versão 3.0: O protocolo padrão é 3.

Alterado na versão 3.8: O protocolo padrão é 4.

Alterado na versão 3.14: O protocolo padrão é 5.

O módulo **pickle** fornece as seguintes funções para tornar o processo de serialização com pickle mais conveniente:

pickle.dump(*obj*, *file*, *protocol=None*, *, *fix_imports=True*, *buffer_callback=None*)

Escreve a representação após a serialização com pickle do objeto *obj* no objeto arquivo aberto *file*. Isso é equivalente a `Pickler(file,`

`protocol).dump(obj).`

Os argumentos *file*, *protocol*, *fix_imports* e *buffer_callback* têm o mesmo significado que no construtor de **`Pickler`**.

Alterado na versão 3.8: O argumento *buffer_callback* foi adicionado.

pickle.dumps(*obj*, *protocol=None*, *, *fix_imports=True*, *buffer_callback=None*)

Retorna a representação após a serialização com pickle do objeto *obj* como um objeto **`bytes`**, ao invés de escrevê-lo em um arquivo.

Os argumentos *protocol*, *fix_imports* e *buffer_callback* têm o mesmo significado que no construtor de **`Pickler`**.

Alterado na versão 3.8: O argumento *buffer_callback* foi adicionado.

pickle.load(*file*, *, *fix_imports=True*, *encoding='ASCII'*, *errors='strict'*, *buffers=None*)

Lê a representação serializada com pickle de um objeto a partir do objeto arquivo aberto *file* e retorna a hierarquia de objeto reconstituído especificada nele. Isso é equivalente a `Unpickler(file).load()`.

A versão do protocolo pickle é detectada automaticamente, portanto, nenhum argumento de protocolo é necessário. Bytes após a representação serializada com pickle do objeto são ignorados.

Os argumentos *file*, *fix_imports*, *encoding*, *errors*, *strict* e *buffers* têm o mesmo significado que no construtor de **`Unpickler`**.

Alterado na versão 3.8: O argumento *buffers* foi adicionado.

pickle.loads(*data*, /, *, *fix_imports=True*, *encoding='ASCII'*, *errors='strict'*, *buffers=None*)

Retorna a hierarquia de objeto reconstituído da representação serializada com pickle *data* de um objeto. *data* deve ser um objeto bytes ou similar.

A versão do protocolo pickle é detectada automaticamente, portanto, nenhum argumento de protocolo é necessário. Bytes após a representação serializada com pickle do objeto são ignorados.

Os argumentos *fix_imports*, *encoding*, *errors*, *strict* e *buffers* têm o mesmo significado que no construtor de **`Unpickler`**.

Alterado na versão 3.8: O argumento *buffers* foi adicionado.

O módulo **`pickle`** define três exceções:

exception pickle.PickleError

Classe base comum para as outras exceções de serialização com pickle. Herda de **`Exception`**.

exception pickle.PicklingError

Erro levantado quando um objeto não serializável com pickle é encontrado por **`Pickler`**. Herda de **`PickleError`**.

Consulte O que pode ser serializado e desserializado com pickle? para saber quais tipos de objetos podem ser serializados com pickle.

exception pickle.UnpicklingError

Erro levantado quando há um problema ao desserializar com pickle um objeto, como dados corrompidos ou violação de segurança. Herda de **`PickleError`**.

Observe que outras exceções também podem ser levantadas durante a desserialização com pickle, incluindo (mas não necessariamente limitado a) AttributeError, EOFError, ImportError e IndexError.

O módulo **`pickle`** exporta três classes, **`Pickler`**, **`Unpickler`** e **`PickleBuffer`**:

class pickle.Pickler(*file*, *protocol=None*, ***, *fix_imports=True*, *buffer_callback=None*)

Isso recebe um arquivo binário para escrever um fluxo de dados pickle.

O argumento opcional *protocol*, um inteiro, diz ao pickler para usar o protocolo fornecido; os protocolos suportados são de 0 a **HIGHEST_PROTOCOL**. Se não for especificado, o padrão é **DEFAULT_PROTOCOL**. Se um número negativo for especificado, **HIGHEST_PROTOCOL** é selecionado.

O argumento *file* deve ter um método write() que aceite um argumento de um único byte. Portanto, pode ser um arquivo em disco aberto para escrita binária, uma instância **io.BytesIO** ou qualquer outro objeto personalizado que atenda a esta interface.

Se *fix_imports* for verdadeiro e *protocol* for menor que 3, pickle tentará mapear os novos nomes do Python 3 para os nomes dos módulos antigos usados no Python 2, de modo que o fluxo de dados pickle seja legível com o Python 2.

Se *buffer_callback* for `None` (o padrão), as visualizações de buffer são serializadas em *file* como parte do fluxo pickle.

Se *buffer_callback* não for `None`, ele pode ser chamado qualquer número de vezes com uma visualização de buffer. Se essa chamada retornar um valor falso (tal como `None`), o buffer fornecido está fora da banda; caso contrário, o buffer é serializado dentro da banda, ou seja, dentro do fluxo pickle.

É um erro se *buffer_callback* não for `None` e *protocol* for `None` ou menor que 5.

Alterado na versão 3.8: O argumento *buffer_callback* foi adicionado.

dump(*obj*)

Escreve a representação serializada em pickle de *obj* no objeto arquivo aberto fornecido no construtor.

persistent_id(*obj*)

Não faz nada por padrão. Isso existe para que uma subclasse possa substituí-lo.

Se `persistent_id()` retornar `None`, *obj* é serializado com pickle como de costume. Qualquer outro valor faz com que `Pickler` emita o valor retornado como um ID persistente para *obj*. O significado deste ID persistente deve ser definido por `Unpickler.persistent_load()`. Observe que o valor retornado por **`persistent_id()`** não pode ter um ID persistente.

Consulte Persistência de objetos externos para detalhes e exemplos de usos.

Alterado na versão 3.13: Foi adicionada a implementação padrão desse método na implementação em C da **`Pickler`**.

dispatch_table

A tabela de despacho de um objeto pickler é um registro de *funções de redução* do tipo que pode ser declarado usando `copyreg.pickle()`. É um mapeamento cujas chaves são classes e cujos valores são funções de redução. Uma função de redução leva um único argumento da classe associada e deve estar de acordo com a mesma interface de um método `__reduce__()`.

Por padrão, um objeto pickler não terá um atributo `dispatch_table`, e em vez disso usará a tabela de despacho global gerenciada pelo módulo `copyreg`. No entanto, para personalizar a serialização com pickle de um objeto pickler específico, pode-se definir o atributo **`dispatch_table`** para um objeto do tipo dict. Alternativamente, se uma subclasse de `Pickler` tem um atributo **`dispatch_table`** então ele será usado como a tabela de despacho padrão para instâncias daquela classe.

Consulte Tabelas de despacho para exemplos de uso.

Adicionado na versão 3.3.

reducer_override(*obj*)

Redutor especial que pode ser definido em subclasses de `Pickler`. Este método tem prioridade sobre qualquer redutor em `dispatch_table`. Ele deve estar de acordo com a mesma interface

que um método `__reduce__()` e pode opcionalmente retornar `NotImplemented` como alternativa em redutores registrados em `dispatch_table` para serializar com pickle `obj`.

Para exemplo detalhado, consulte Redução personalizada para tipos, funções e outros objetos.

Adicionado na versão 3.8.

fast

Descontinuado. Ativa o modo rápido se definido como um valor verdadeiro. O modo rápido desabilita o uso de memo, portanto, agilizando o processo de serialização com pickle por não gerar códigos de operação PUT supérfluos. Ele não deve ser usado com objetos autorreferenciais, fazer o contrário fará com que `Pickler` recorra infinitamente.

Use `pickletools.optimize()` se você precisar de serializações com pickle mais compactas.

clear_memo()

Limpa o “memo” do pickler.

O memo é a estrutura de dados que lembra quais objetos o pickler já viu, de modo que objetos compartilhados ou recursivos sejam serializados por pickle por referência e não por valor. Este método é útil ao reutilizar picklers.

class pickle.Unpickler(*file*, **, fix_imports=True, encoding='ASCII', errors='strict', buffers=None*)

Recebe um arquivo binário para ler um fluxo de dados pickle.

A versão do protocolo do pickle é detectada automaticamente, portanto, nenhum argumento de protocolo é necessário.

O argumento *file* deve ter três métodos: um método read() que recebe um argumento inteiro, um método readinto() que recebe um argumento

buffer e um método readline() que não requer argumentos, como na interface `io.BufferedIOBase`. Assim, *file* pode ser um arquivo em disco aberto para leitura binária, um objeto `io.BytesIO` ou qualquer outro objeto personalizado que atenda a esta interface.

Os argumentos opcionais *fix_imports*, *encoding* e *errors* são usados para controlar o suporte de compatibilidade ao fluxo pickle gerado pelo Python 2. Se *fix_imports* for verdadeiro, pickle tentará mapear os nomes antigos do Python 2 para os novos nomes usados no Python 3. Os argumentos *encoding* e *errors* dizem ao pickle como decodificar instâncias de string de 8 bits capturadas pelo Python 2; os padrões são 'ASCII' e 'strict', respectivamente. O argumento *encoding* pode ser 'bytes' para ler essas instâncias de string de 8 bits como objetos de bytes. Usar `encoding='latin1'` é necessário para a desserialização com pickle de vetores NumPy e instâncias de `datetime`, `date` e `time` serializadas com pickle pelo Python 2.

Se *buffers* for `None` (o padrão), todos os dados necessários para desserialização devem estar contidos no fluxo pickle. Isso significa que o argumento *buffer_callback* era `None` quando um `Pickler` foi instanciado (ou quando `dump()` ou `dumps()` foi chamado).

Se *buffers* for `None`, deve ser um iterável de objetos habilitados para buffer que é consumido cada vez que o fluxo de serialização com pickle faz referência a uma visualização de buffer fora da banda. Esses buffers foram fornecidos em ordem para o *buffer_callback* de um objeto Pickler.

Alterado na versão 3.8: O argumento *buffers* foi adicionado.

load()

Lê a representação serializada com pickle de um objeto a partir do objeto arquivo aberto fornecido no construtor e retorna a hierarquia de objeto reconstituído especificada nele. Os bytes após a representação serializada com pickle do objeto são ignorados.

persistent_load(*pid*)

Levanta um **`UnpicklingError`** por padrão.

Se definido, **`persistent_load()`** deve retornar o objeto especificado pelo ID persistente *pid*. Se um ID persistente inválido for encontrado, um **`UnpicklingError`** deve ser levantado.

Consulte Persistência de objetos externos para detalhes e exemplos de usos.

Alterado na versão 3.13: Foi adicionada a implementação padrão desse método na implementação em C da **`Unpickler`**.

find_class(*module*, *name*)

Importa *module* se necessário e retorna o objeto chamado *name* dele, onde os argumentos *module* e *name* são objetos **`str`**. Observe, ao contrário do que seu nome sugere, **`find_class()`** também é usado para encontrar funções.

As subclasses podem substituir isso para obter controle sobre quais tipos de objetos e como eles podem ser carregados, reduzindo potencialmente os riscos de segurança. Confira Restringindo globais para detalhes.

Levanta um evento de auditoria `pickle.find_class` com os argumentos `module`, `name`.

class pickle.PickleBuffer(*buffer*)

Um invólucro para um buffer que representa dados serializáveis com pickle. *buffer* deve ser um objeto provedor de buffer, como um objeto bytes ou similar ou um vetor N-dimensional.

`PickleBuffer` é ele próprio um provedor de buffer, de forma que é possível passá-lo para outras APIs que esperam um objeto provedor de buffer, como **`memoryview`**.

Objetos **`PickleBuffer`** só podem ser serializados usando o protocolo pickle 5 ou superior. Eles são elegíveis para serialização fora de banda.

Adicionado na versão 3.8.

raw()

Retorna um `memoryview` da área de memória subjacente a este buffer. O objeto retornado é um memoryview unidimensional, contíguo C com formato `B` (bytes não assinados). `BufferError` é levantada se o buffer não for contíguo C nem Fortran.

release()

Libera o buffer subjacente exposto pelo objeto PickleBuffer.

O que pode ser serializado e desserializado com pickle?

Os seguintes tipos podem ser serializados com pickle:

- constantes embutidas (`None`, `True`, `False`, `Ellipsis` e `NotImplemented`);
- inteiros, números de ponto flutuante, números complexos;
- strings, bytes, bytearrays;
- tuplas, listas, conjuntos e dicionários contendo apenas objetos serializáveis com pickle;
- funções (embutidas ou definidas pelo usuário) acessíveis no nível superior de um módulo (usando `def`, não `lambda`);
- classes acessíveis no nível superior de um módulo;
- instâncias de classes cujo o resultado da chamada de `__getstate__()` seja serializável com pickle (veja a seção Serializando instâncias de classes com pickle para detalhes).

As tentativas de serializar objetos não serializáveis com pickle vão levantar a exceção `PicklingError`; quando isso acontece, um número não especificado de bytes pode já ter sido escrito no arquivo subjacente. Tentar serializar com pickle uma estrutura de dados altamente recursiva pode exceder a profundidade máxima de recursão, a `RecursionError` será levantada neste caso. Você pode aumentar este limite cuidadosamente com `sys.setrecursionlimit()`.

Observe que as funções (embutidas e definidas pelo usuário) são serializadas com pickle pelo nome qualificado, não pelo valor. [[2]] Isso significa que apenas o nome da função é serializado com pickle, junto com o nome do módulo e das classes contidas. Nem o código da função, nem qualquer um de seus atributos de função são serializados com pickle. Assim, o módulo de definição deve ser importável no ambiente de desserialização com pickle, e o módulo deve conter o objeto nomeado, caso contrário, uma exceção será levantada. [[3]]

Da mesma forma, as classes são serializadas com pickle pelo nome qualificado, portanto, aplicam-se as mesmas restrições no ambiente de desserialização com pickle. Observe que nenhum código ou dado da classe é coletado, portanto, no exemplo a seguir, o atributo de classe `attr` não é restaurado no ambiente de desserialização com pickle:

```
class Foo:
    attr = 'A class attribute'

picklestring = pickle.dumps(Foo)
```

Essas restrições são a razão pela qual as funções e classes serializáveis com pickle devem ser definidas no nível superior de um módulo.

Da mesma forma, quando as instâncias da classe são serializadas com pickle, o código e os dados de sua classe não são serializados junto com elas. Apenas os dados da instância são serializados com pickle. Isso é feito de propósito para que você possa corrigir bugs em uma classe ou adicionar métodos à classe e ainda carregar objetos que foram criados com uma versão anterior da classe. Se você planeja ter objetos de longa duração que verão muitas versões de uma classe, pode valer a pena colocar um número de versão nos objetos para que as conversões adequadas possam ser feitas pelo método `__setstate__()` da classe.

Serializando instâncias de classes com pickle

Nesta seção, descrevemos os mecanismos gerais disponíveis para você definir, personalizar e controlar como as instâncias de classe são serializadas e desserializadas com pickle.

Na maioria dos casos, nenhum código adicional é necessário para tornar as instâncias serializáveis com pickle. Por padrão, o pickle recuperará a classe e os atributos de uma instância por meio de introspecção. Quando uma instância de classe não está serializada com pickle, seu método `__init__()` geralmente *não* é invocado. O comportamento padrão primeiro cria uma instância não inicializada e, em seguida, restaura os atributos salvos. O código a seguir mostra uma implementação desse comportamento:

```
def save(obj):
    return (obj.__class__, obj.__dict__)

def restore(cls, attributes):
    obj = cls.__new__(cls)
    obj.__dict__.update(attributes)
    return obj
```

As classes podem alterar o comportamento padrão, fornecendo um ou vários métodos especiais:

object.__getnewargs_ex__()

Nos protocolos 2 e mais recentes, as classes que implementam o método `__getnewargs_ex__()` podem ditar os valores passados para o método `__new__()` após a desserialização com pickle. O método deve retornar um par `(args, kwargs)` onde *args* é uma tupla de argumentos posicionais e *kwargs* um dicionário de argumentos nomeados para construir o objeto. Esses serão passados para o método `__new__()` após a desserialização com pickle.

Você deve implementar este método se o método `__new__()` de sua classe requer argumentos somente-nomeados. Caso contrário, é recomendado para compatibilidade implementar `__getnewargs__()`.

Alterado na versão 3.6: `__getnewargs_ex__()` é agora usado em protocolos 2 e 3.

object.__getnewargs__()

Este método serve a um propósito semelhante ao de `__getnewargs_ex__()`, mas tem suporte apenas a argumentos

posicionais. Ele deve retornar uma tupla de argumentos `args` que serão passados para o método `__new__()` após a desserialização com pickle.

`__getnewargs__()` não será chamado se `__getnewargs_ex__()` estiver definido.

Alterado na versão 3.6: Antes do Python 3.6, `__getnewargs__()` era chamado em vez de `__getnewargs_ex__()` nos protocolos 2 e 3.

object.__getstate__()

Classes podem influenciar ainda mais como suas instâncias são serializadas com pickle, substituindo o método `__getstate__()`. Ele é chamado e o objeto retornado é serializado com pickle como o conteúdo da instância, em vez de um estado padrão. Existem vários casos:

- Para uma classe que não possui `__dict__` de instância e nem `__slots__` , o estado padrão é `None`.
- Para uma classe que possui `__dict__` de instância mas não `__slots__`, o estado padrão é `self.__dict__`.
- Para uma classe que possui `__dict__` de instância e `__slots__`, o estado padrão é uma tupla consistindo de dois dicionários: `self.__dict__`, e um dicionário de mapeamento de nomes de slot para valores de slot. Apenas os slots que possuem um valor são incluídos neste último.
- Para uma classe que possui `__slots__` e mas não `__dict__` de instância, o estado padrão é uma tupla cujo primeiro item é `None` e cujo segundo item é um dicionário mapeando os nomes dos slots para os valores dos slots descritos no tópico anterior.

Alterado na versão 3.11: Adicionada a implementação padrão do método `__getstate__()` na classe `object`.

object.__setstate__(*state*)

Ao desserializar com pickle, se a classe define `__setstate__()`, ela é chamada com o estado não desserializado. Nesse caso, não há nenhum requisito para que o objeto de estado seja um dicionário. Caso contrário, o estado serializado com pickle deve ser um dicionário e seus itens são atribuídos ao dicionário da nova instância.

Nota

Se `__reduce__()` retorna um estado com valor `None` na serialização com pickle, o método `__setstate__()` não será chamado quando da desserialização com pickle.

Confira a seção Manipulação de objetos com estado para mais informações sobre como usar os métodos `__getstate__()` e `__setstate__()`.

Nota

Quando da desserialização com pickle, alguns métodos, como `__getattr__()`, `__getattribute__()` ou `__setattr__()`, podem ser chamados na instância. No caso desses métodos dependerem de alguma invariante interna ser verdadeira, o tipo deve ser implementado `__new__()` para estabelecer tal invariante, pois `__init__()` não é chamado quando da desserialização com pickle em uma instância.

Como veremos, o pickle não usa diretamente os métodos descritos acima. Na verdade, esses métodos são parte do protocolo de cópia que implementa o método especial `__reduce__()`. O protocolo de cópia fornece uma interface unificada para recuperar os dados necessários para serialização com pickle e cópia de objetos. [[4]]

Apesar de poderoso, implementar `__reduce__()` diretamente em sua classe é algo propenso a erro. Por este motivo, projetistas de classe devem usar a interface de alto nível (ou seja, `__getnewargs_ex__()`, `__getstate__()` e `__setstate__()`) sempre que possível. Vamos mostrar, porém, casos em que o uso de `__reduce__()` é a única opção ou leva a uma serialização com pickle mais eficiente, ou a ambas.

object.__reduce__()

A interface está atualmente definida da seguinte maneira. O método `__reduce__()` não aceita nenhum argumento e deve retornar uma string

ou preferencialmente uma tupla (o objeto retornado é frequentemente referido como o “valor de redução”).

Se uma string é retornada, ela deve ser interpretada como o nome de uma variável global. Deve ser o nome local do objeto relativo ao seu módulo; o módulo pickle pesquisa o espaço de nomes do módulo para determinar o módulo do objeto: para um dado `obj` ser serializado com pickle, o atributo `__module__` é consultado diretamente em `obj`, cuja busca é repassada para o tipo de `obj` se o atributo de instância `__module__` não está definido. Esse comportamento é tipicamente útil para singletons.

Quando uma tupla é retornada, ela deve ter entre dois e seis itens. Os itens opcionais podem ser omitidos ou `None` pode ser fornecido como seu valor. A semântica de cada item são, em ordem:

- Um objeto chamável que será chamado para criar a versão inicial do objeto.
- Uma tupla de argumentos para o objeto chamável. Uma tupla vazia deve ser fornecida se o chamável não aceitar nenhum argumento.
- Opcionalmente, o estado do objeto, que será passado para o método **`__setstate__()`** do objeto conforme descrito anteriormente. Se o objeto não tiver tal método, o valor deve ser um dicionário e será adicionado ao atributo **`__dict__`** do objeto.
- Opcionalmente, um iterador (e não uma sequência) produzindo itens sucessivos. Esses itens serão anexados ao objeto usando `obj.append(item)` ou, em lote, usando `obj.extend(list_of_items)`. Isso é usado principalmente para subclasses de lista, mas pode ser usado por outras classes, desde que tenham os métodos **`append()`** e **`extend()`** com a assinatura apropriada. (Se **`append()`** ou **`extend()`** é usado depende de qual versão do protocolo pickle é usada, bem como o número de itens a anexar, então ambos devem estar disponíveis.)

- Opcionalmente, um iterador (não uma sequência) produzindo pares de valor-chave sucessivos. Esses itens serão armazenados no objeto usando `obj[chave]=valor`. Isso é usado principalmente para subclasses de dicionário, mas pode ser usado por outras classes, desde que implementem `__setitem__()`.

- Opcionalmente, um chamável com uma assinatura `(obj, estado)`. Este chamável permite ao usuário controlar programaticamente o comportamento de atualização de estado de um objeto específico, em vez de usar o método estático `__setstate__()` de `obj`. Se não for `None`, este chamável terá prioridade sobre o **`__setstate__()`** de `obj`.

  Adicionado na versão 3.8: O sexto item opcional de tupla, `(obj, estado)`, foi adicionado.

object.__reduce_ex__(*protocol*)

Alternativamente, um método `__reduce_ex__()` pode ser definido. A única diferença é que este método deve ter um único argumento inteiro, a versão do protocolo. Quando definido, pickle irá preferir isso ao método `__reduce__()`. Além disso, **`__reduce__()`** automaticamente se torna um sinônimo para a versão estendida. O principal uso desse método é fornecer valores de redução com compatibilidade retrocompatível para versões mais antigas do Python.

Persistência de objetos externos

For the benefit of object persistence, the **`pickle`** module supports the notion of a reference to an object outside the pickled data stream. Such objects are referenced by a persistent ID, which should be either a string of alphanumeric characters (for protocol 0) [[5]] or just an arbitrary object (for any newer protocol).

The resolution of such persistent IDs is not defined by the **`pickle`** module; it will delegate this resolution to the user-defined methods on the pickler and unpickler, `persistent_id()` and `persistent_load()` respectively.

Para serializar com pickle objetos que têm um ID externo persistente, o pickler deve ter um método `persistent_id()` personalizado que recebe um objeto como um argumento e retorna `None` ou o ID persistente para esse objeto. Quando `None` é retornado, o pickler simplesmente serializa o objeto normalmente. Quando uma string de ID persistente é retornada, o pickler serializa aquele objeto, junto com um marcador para que o unpickler o reconheça como um ID persistente.

Para desserializar com pickle objetos externos, o unpickler deve ter um método `persistent_load()` personalizado que recebe um objeto de ID persistente e retorna o objeto referenciado.

Aqui está um exemplo abrangente que apresenta como o ID persistente pode ser usado para serializar com pickle objetos externos por referência.

```
# Exemplo simples que mostra como o ID persistente pode ser usac
# serializar com pickle objetos externos por referência.

import pickle
import sqlite3
from collections import namedtuple

# Classe simples que representa um registro em nosso banco de da
MemoRecord = namedtuple("MemoRecord", "key, task")

class DBPickler(pickle.Pickler):

    def persistent_id(self, obj):
        # Em vez de conservar MemoRecord como uma instância de c
        # emitimos um ID persistente.
        if isinstance(obj, MemoRecord):
            # Aqui, nosso ID persistente é simplesmente uma tupl
            # uma tag e uma chave, que se refere a um registro e
            # no banco de dados.
            return ("MemoRecord", obj.key)
        else:
            # Se obj não tiver um ID persistente, retorna None.
            # obj precisa ser conservado como de costume.
            return None
```

```
class DBUnpickler(pickle.Unpickler):

    def __init__(self, file, connection):
        super().__init__(file)
        self.connection = connection

    def persistent_load(self, pid):
        # Este método é invocado sempre que um ID persistente é
        # Aqui, pid é a tupla retornada por DBPickler.
        cursor = self.connection.cursor()
        type_tag, key_id = pid
        if type_tag == "MemoRecord":
            # Busca o registro referenciado no banco de dados e
            cursor.execute("SELECT * FROM memos WHERE key=?", (s
            key, task = cursor.fetchone()
            return MemoRecord(key, task)
        else:
            # Sempre levanta um erro se você puder retornar o ob
            # Caso contrário, a desserialização com pickle consi
            # None é o objeto referenciado pelo ID persistente.
            raise pickle.UnpicklingError("unsupported persistent


def main():
    import io
    import pprint

    # Inicializa e popula nosso banco de dados.
    conn = sqlite3.connect(":memory:")
    cursor = conn.cursor()
    cursor.execute("CREATE TABLE memos(key INTEGER PRIMARY KEY,
    tasks = (
        'give food to fish',
        'prepare group meeting',
        'fight with a zebra',
        )
    for task in tasks:
        cursor.execute("INSERT INTO memos VALUES(NULL, ?)", (tas

    # Busca os registros a serem serializados com pickle.
    cursor.execute("SELECT * FROM memos")
    memos = [MemoRecord(key, task) for key, task in cursor]
    # Salva os registros usando nosso DBPickler personalizado.
```

```
    file = io.BytesIO()
    DBPickler(file).dump(memos)

    print("Pickled records:")
    pprint.pprint(memos)

    # Atualiza um registro, só por precaução.
    cursor.execute("UPDATE memos SET task='learn italian' WHERE

    # Carrega os registros do fluxo de dados pickle.
    file.seek(0)
    memos = DBUnpickler(file, conn).load()

    print("Unpickled records:")
    pprint.pprint(memos)


if __name__ == '__main__':
    main()
```

Tabelas de despacho

Se alguém quiser personalizar a serialização com pickle de algumas classes sem perturbar nenhum outro código que dependa da serialização, pode-se criar um pickler com uma tabela de despacho privada.

A tabela de despacho global gerenciada pelo módulo **`copyreg`** está disponível como **`copyreg.dispatch_table`**. Portanto, pode-se escolher usar uma cópia modificada de **`copyreg.dispatch_table`** como uma tabela de despacho privada.

Por exemplo

```
f = io.BytesIO()
p = pickle.Pickler(f)
p.dispatch_table = copyreg.dispatch_table.copy()
p.dispatch_table[SomeClass] = reduce_SomeClass
```

cria uma instância de **`pickle.Pickler`** com uma tabela de despacho privada que trata a classe `SomeClass` especialmente. Alternativamente, o código

```
class MyPickler(pickle.Pickler):
    dispatch_table = copyreg.dispatch_table.copy()
    dispatch_table[SomeClass] = reduce_SomeClass
f = io.BytesIO()
p = MyPickler(f)
```

faz o mesmo, mas todas as instâncias de `MyPickler` compartilharão por padrão a tabela de despacho privada. Por outro lado, o código

```
copyreg.pickle(SomeClass, reduce_SomeClass)
f = io.BytesIO()
p = pickle.Pickler(f)
```

modifica a tabela de despacho global compartilhada por todos os usuários do módulo **`copyreg`**.

Manipulação de objetos com estado

Aqui está um exemplo que mostra como modificar o comportamento de serialização com pickle de uma classe. A classe **`TextReader`** abaixo abre um arquivo texto e retorna o número da linha e o conteúdo da linha cada vez que seu método **`readline()`** é chamado. Se uma instância de **`TextReader`** for selecionada, todos os atributos *exceto* o membro do objeto arquivo são salvos. Quando a instância é removida, o arquivo é reaberto e a leitura continua a partir do último local. Os métodos **`__setstate__()`** e **`__getstate__()`** são usados para implementar este comportamento.

```
class TextReader:
    """Exibe e numera linhas em um arquivo texto."""

    def __init__(self, filename):
        self.filename = filename
        self.file = open(filename)
        self.lineno = 0

    def readline(self):
        self.lineno += 1
        line = self.file.readline()
        if not line:
            return None
```

```
        if line.endswith('\n'):
            line = line[:-1]
        return "%i: %s" % (self.lineno, line)

    def __getstate__(self):
        # Copia o estado do objeto de self.__dict__, que
        # contém todos os nossos atributos de instância.
        # Sempre use o método dict.copy() para evitar
        # modificar o estado original.
        state = self.__dict__.copy()
        # Remove as entradas não serializáveis com pickle.
        del state['file']
        return state

    def __setstate__(self, state):
        # Restaura os atributos da instância (isto é, filename e
        self.__dict__.update(state)
        # Restaura o estado do arquivo aberto anteriormente.
        # Para fazer isso, precisamos reabri-lo e ler dele até
        # a contagem de linha ser restaurada.
        file = open(self.filename)
        for _ in range(self.lineno):
            file.readline()
        # Ao final, salva o arquivo.
        self.file = file
```

Um exemplo de uso pode ser algo assim:

```
>>> reader = TextReader("hello.txt")
>>> reader.readline()
'1: Hello world!'
>>> reader.readline()
'2: I am line number two.'
>>> new_reader = pickle.loads(pickle.dumps(reader))
>>> new_reader.readline()
'3: Goodbye!'
```

Redução personalizada para tipos, funções e outros objetos

Adicionado na versão 3.8.

Às vezes, `dispatch_table` pode não ser flexível o suficiente. Em particular, podemos querer personalizar a serialização com pickle com base em outro critério que não o tipo do objeto, ou podemos personalizar a serialização com pickle de funções e classes.

Para esses casos, é possível criar uma subclasse da classe `Pickler` e implementar um método `reducer_override()`. Este método pode retornar uma tupla de redução arbitrária (veja `__reduce__()`). Ele pode, alternativamente, retornar `NotImplemented` para retornar ao comportamento tradicional.

Se `dispatch_table` e `reducer_override()` forem definidos, o método **`reducer_override()`** tem prioridade.

Nota

Por motivos de desempenho, `reducer_override()` não pode ser chamado para os seguintes objetos: `None`, `True`, `False`, e as instâncias exatas de `int`, `float`, `bytes`, `str`, `dict`, `set`, `frozenset`, `list` e `tuple`.

Aqui está um exemplo simples onde permitimos serialização com pickle e reconstrução de uma determinada classe:

```
import io
import pickle

class MyClass:
    my_attribute = 1

class MyPickler(pickle.Pickler):
    def reducer_override(self, obj):
        """Redutor personalizado para MyClass."""
        if getattr(obj, "__name__", None) == "MyClass":
            return type, (obj.__name__, obj.__bases__,
                          {'my_attribute': obj.my_attribute})
        else:
```

```
            # Para qualquer outro objeto, recorre à redução de c
            return NotImplemented

f = io.BytesIO()
p = MyPickler(f)
p.dump(MyClass)

del MyClass

unpickled_class = pickle.loads(f.getvalue())

assert isinstance(unpickled_class, type)
assert unpickled_class.__name__ == "MyClass"
assert unpickled_class.my_attribute == 1
```

Buffers fora da banda

Adicionado na versão 3.8.

In some contexts, the **pickle** module is used to transfer massive amounts of data. Therefore, it can be important to minimize the number of memory copies, to preserve performance and resource consumption. However, normal operation of the **pickle** module, as it transforms a graph-like structure of objects into a sequential stream of bytes, intrinsically involves copying data to and from the pickle stream.

Esta restrição pode ser evitada se tanto o *fornecedor* (a implementação dos tipos de objetos a serem transferidos) e o *consumidor* (a implementação do sistema de comunicações) tiverem suporte aos recursos de transferência fora de banda fornecidos pelo protocolo pickle 5 e superior.

API de provedor

Os grandes objetos de dados a serem serializados com pickle devem implementar um método **__reduce_ex__()** especializado para o protocolo 5 e superior, que retorna uma instância **PickleBuffer** (em vez de, por exemplo, um objeto **bytes**) para quaisquer dados grandes.

A `PickleBuffer` object *signals* that the underlying buffer is eligible for out-of-band data transfer. Those objects remain compatible with normal usage of the **`pickle`** module. However, consumers can also opt-in to tell **`pickle`** that they will handle those buffers by themselves.

API de consumidor

Um sistema de comunicação pode permitir o manuseio personalizado dos objetos `PickleBuffer` gerados ao serializar um grafo de objeto.

No lado emissor, é necessário passar um argumento *buffer_callback* para `Pickler` (ou para a função `dump()` ou `dumps()`), que será chamada com cada `PickleBuffer` gerado durante a serialização com pickle do grafo do objeto. Os buffers acumulados pelo *buffer_callback* não verão seus dados copiados no fluxo pickle, apenas um marcador barato será inserido.

No lado receptor, é necessário passar um argumento *buffers* para `Unpickler` (ou para a função `load()` ou **`load()`**), que é um iterável dos buffers que foram passado para *buffer_callback*. Esse iterável deve produzir buffers na mesma ordem em que foram passados para *buffer_callback*. Esses buffers fornecerão os dados esperados pelos reconstrutores dos objetos cuja serialização com pickle produziu os objetos `PickleBuffer` originais.

Entre o lado emissor e o lado receptor, o sistema de comunicações está livre para implementar seu próprio mecanismo de transferência para buffers fora de banda. As otimizações potenciais incluem o uso de memória compartilhada ou compactação dependente do tipo de dados.

Exemplo

Aqui está um exemplo trivial onde implementamos uma subclasse de `bytearray` capaz de participar de serialização com pickle de buffer fora de banda:

```
class ZeroCopyByteArray(bytearray):

    def __reduce_ex__(self, protocol):
        if protocol >= 5:
```

```
            return type(self)._reconstruct, (PickleBuffer(self),
        else:
            # PickleBuffer é proibido no pickle nos protocolos <
            return type(self)._reconstruct, (bytearray(self),)

    @classmethod
    def _reconstruct(cls, obj):
        with memoryview(obj) as m:
            # Obtém um controle sobre o objeto buffer original
            obj = m.obj
            if type(obj) is cls:
                # O objeto do buffer original é um ZeroCopyByteA
                # retorna-o como está.
                return obj
            else:
                return cls(obj)
```

O reconstrutor (o método de classe `_reconstruct`) retorna o objeto de fornecimento do buffer se ele tiver o tipo correto. Esta é uma maneira fácil de simular o comportamento de cópia zero neste exemplo de brinquedo.

Do lado consumidor, podemos serializar com pickle esses objetos da maneira usual, que quando não serializados nos dará uma cópia do objeto original:

```
b = ZeroCopyByteArray(b"abc")
data = pickle.dumps(b, protocol=5)
new_b = pickle.loads(data)
print(b == new_b)  # True
print(b is new_b)  # False: uma cópia foi feita
```

Mas se passarmos um *buffer_callback* e, em seguida, retornarmos os buffers acumulados ao desserializar, seremos capazes de recuperar o objeto original:

```
b = ZeroCopyByteArray(b"abc")
buffers = []
data = pickle.dumps(b, protocol=5, buffer_callback=buffers.apper
new_b = pickle.loads(data, buffers=buffers)
print(b == new_b)  # True
print(b is new_b)  # True: nenhuma cópia foi feita
```

Este exemplo é limitado pelo fato de que `bytearray` aloca sua própria memória: você não pode criar uma instância de `bytearray` que é apoiada pela memória de outro objeto. No entanto, tipos de dados de terceiros, como arrays de NumPy, não têm essa limitação e permitem o uso de serialização com pickle de cópia zero (ou fazer o mínimo de cópias possível) ao transferir entre processos ou sistemas distintos.

Ver também

PEP 574 [https://peps.python.org/pep-0574/] – Protocolo de Pickle 5 com buffers de dados fora da banda

Restringindo globais

Por padrão, a desserialização com pickle importará qualquer classe ou função que encontrar nos dados pickle. Para muitos aplicativos, esse comportamento é inaceitável, pois permite que o unpickler importe e invoque código arbitrário. Basta considerar o que este fluxo de dados pickle feito à mão faz quando carregado:

```
>>> import pickle
>>> pickle.loads(b"cos\nsystem\n(S'echo hello world'\ntR.")
hello world
0
```

Neste exemplo, o unpickler importa a função `os.system()` e então aplica o argumento string “echo hello world”. Embora este exemplo seja inofensivo, não é difícil imaginar um que possa danificar seu sistema.

Por esta razão, você pode querer controlar o que é desserializado com pickle personalizando `Unpickler.find_class()`. Ao contrário do que seu nome sugere, `Unpickler.find_class()` é chamado sempre que um global (ou seja, uma classe ou uma função) é solicitado. Assim, é possível proibir completamente os globais ou restringi-los a um subconjunto seguro.

Aqui está um exemplo de um unpickler que permite que apenas algumas classes seguras do módulo **builtins** sejam carregadas:

```
import builtins
import io
import pickle

safe_builtins = {
    'range',
    'complex',
    'set',
    'frozenset',
    'slice',
}

class RestrictedUnpickler(pickle.Unpickler):

    def find_class(self, module, name):
        # Só permite classes seguras de bultins.
        if module == "builtins" and name in safe_builtins:
            return getattr(builtins, name)
        # Proíbe todo o resto.
        raise pickle.UnpicklingError("global '%s.%s' is forbidde
                                     (module, name))

def restricted_loads(s):
    """Função auxiliar análoga a pickle.loads()."""
    return RestrictedUnpickler(io.BytesIO(s)).load()
```

Um exemplo de uso do nosso unpickler funcionando como esperado:

```
>>> restricted_loads(pickle.dumps([1, 2, range(15)]))
[1, 2, range(0, 15)]
>>> restricted_loads(b"cos\nsystem\n(S'echo hello world'\ntR.")
Traceback (most recent call last):
  ...
pickle.UnpicklingError: global 'os.system' is forbidden
>>> restricted_loads(b'cbuiltins\neval\n'
...                  b'(S\'getattr(__import__("os"), "system")'
...                  b'("echo hello world")\'\ntR.')
Traceback (most recent call last):
```

```
  ...
pickle.UnpicklingError: global 'builtins.eval' is forbidden
```

Como nossos exemplos mostram, você deve ter cuidado com o que permite que seja desserializado com pickle. Portanto, se a segurança é uma preocupação, você pode querer considerar alternativas como a API de marshalling em `xmlrpc.client` ou soluções de terceiros.

Desempenho

Recent versions of the pickle protocol (from protocol 2 and upwards) feature efficient binary encodings for several common features and built-in types. Also, the **`pickle`** module has a transparent optimizer written in C.

Exemplos

Para código mais simples, use as funções `dump()` e `load()`.

```
import pickle

# Uma coleção arbitrária de objetos aceitos por pickle.
data = {
    'a': [1, 2.0, 3+4j],
    'b': ("character string", b"byte string"),
    'c': {None, True, False}
}

with open('data.pickle', 'wb') as f:
    # Serializa com pickle o dicionário 'data' usando
    # o protocolo mais alto disponível.
    pickle.dump(data, f, pickle.HIGHEST_PROTOCOL)
```

O exemplo a seguir lê os dados resultantes em serializados com pickle.

```
import pickle

with open('data.pickle', 'rb') as f:
    # A versão do protocolo utilizada é detectada automaticament
```

```
    # portanto não precisamos especificá-la.
    data = pickle.load(f)
```

Interface de linha de comando

The **`pickle`** module can be invoked as a script from the command line, it will display contents of the pickle files. However, when the pickle file that you want to examine comes from an untrusted source, `-m pickletools` is a safer option because it does not execute pickle bytecode, see pickletools CLI usage.

```
python -m pickle aquivo_pickle [aquivo_pickle ...]
```

A seguinte opção é aceita:

pickle_file
: Um arquivo pickle para ler, ou `-` para indicar leitura da entrada padrão.

Ver também

Módulo `copyreg`
: Registro de construtor de interface Pickle para tipos de extensão.

Módulo `pickletools`
: Ferramentas para trabalhar e analisar dados serializados com pickle.

Módulo `shelve`
: Indexed databases of objects; uses **`pickle`**.

Módulo `copy`
: Cópia rasa e cópia profunda de objeto.

Módulo `marshal`
: Serialização de alto desempenho de tipos embutidos.

Notas de rodapé

[1]

Não confunda isso com o módulo `marshal`

[2]

É por isso que funções `lambda` não podem ser serializadas com pickle: todas as funções `lambda` compartilham o mesmo nome: `<lambda>`.

[3]

A exceção levantada provavelmente será uma `ImportError` ou uma `AttributeError`, mas poderia ser outra coisa.

[4]

O módulo `copy` usa este protocolo para operações de cópia rasa e cópia profunda.

[5]

A limitação de caracteres alfanuméricos se deve ao fato de que os IDs persistentes, no protocolo 0, serem delimitados pelo caractere de nova linha. Portanto, se qualquer tipo de caractere de nova linha ocorrer em IDs persistentes, os dados resultantes da serialização com pickle se tornarão ilegíveis.

`copyreg` — Registra funções de suporte `pickle`

Código-fonte: Lib/copyreg.py
[https://github.com/python/cpython/tree/3.15/Lib/copyreg.py]

O módulo **`copyreg`** oferece uma maneira de definir as funções usadas durante a remoção de objetos específicos. Os módulos **`pickle`** e **`copy`** usam essas funções ao selecionar/copiar esses objetos. O módulo fornece informações de configuração sobre construtores de objetos que não são classes. Esses construtores podem ser funções de fábrica ou instâncias de classes.

copyreg.constructor(*object*)

Declara *object* para ser um construtor válido. Se *object* não for chamável (e, portanto, não for válido como um construtor), levanta **`TypeError`**.

copyreg.pickle(*type*, *function*, *constructor_ob=None*)

Declara que a *function* deve ser usada como uma função de "redução" para objetos do tipo *type*. *function* deve retornar uma string ou uma tupla contendo entre dois e seis elementos. Veja **`dispatch_table`** para mais detalhes sobre a interface da *function*.

O parâmetro *constructor_ob* é um recurso herdado e agora é ignorado, mas se passado deve ser UM chamável.

Note que o atributo **`dispatch_table`** de um objeto pickler ou subclasse de **`pickle.Pickler`** também podem ser usados para declarar funções de redução.

Exemplo

O exemplo abaixo gostaria de mostrar como registrar uma função de pickle e como ela será usada:

```
>>> import copyreg, copy, pickle
>>> class C:
...     def __init__(self, a):
...         self.a = a
...
>>> def pickle_c(c):
...     print("pickling a C instance...")
...     return C, (c.a,)
...
>>> copyreg.pickle(C, pickle_c)
>>> c = C(1)
>>> d = copy.copy(c)
pickling a C instance...
>>> p = pickle.dumps(c)
pickling a C instance...
```

`shelve` — Python object persistence

Código-fonte: Lib/shelve.py [https://github.com/python/cpython/tree/3.15/Lib/shelve.py]

A “shelf” is a persistent, dictionary-like object. The difference with “dbm” databases is that the values (not the keys!) in a shelf can be essentially arbitrary Python objects — anything that the `pickle` module can handle. This includes most class instances, recursive data types, and objects containing lots of shared sub-objects. The keys are ordinary strings.

shelve.open(*filename*, *flag='c'*, *protocol=None*, *writeback=False*, ***, *serializer=None*, *deserializer=None*)

Open a persistent dictionary. The filename specified is the base filename for the underlying database. As a side-effect, an extension may be added to the filename and more than one file may be created. By default, the underlying database file is opened for reading and writing. The optional *flag* parameter has the same interpretation as the *flag* parameter of `dbm.open()`.

By default, pickles created with `pickle.DEFAULT_PROTOCOL` are used to serialize values. The version of the pickle protocol can be specified with the *protocol* parameter.

Because of Python semantics, a shelf cannot know when a mutable persistent-dictionary entry is modified. By default modified objects are written *only* when assigned to the shelf (see Exemplo). If the optional *writeback* parameter is set to `True`, all entries accessed are also cached in memory, and written back on `sync()` and `close()`; this can make it handier to mutate mutable entries in the persistent dictionary, but, if many entries are accessed, it can consume vast amounts of memory for the cache, and it can make the close operation very slow since all accessed entries are written back (there is no way to determine which accessed entries are mutable, nor which ones were actually mutated).

By default, **shelve** uses **pickle.dumps()** and **pickle.loads()** for serializing and deserializing. This can be changed by supplying *serializer* and *deserializer*, respectively.

The *serializer* argument must be a callable which takes an object `obj` and the *protocol* as inputs and returns the representation `obj` as a bytes-like object; the *protocol* value may be ignored by the serializer.

The *deserializer* argument must be a callable which takes a serialized object given as a **bytes** object and returns the corresponding object.

A **ShelveError** is raised if *serializer* is given but *deserializer* is not, or vice-versa.

Alterado na versão 3.10: **pickle.DEFAULT_PROTOCOL** is now used as the default pickle protocol.

Alterado na versão 3.11: Aceita um objeto caminho ou similar como nome de arquivo.

Alterado na versão 3.15: Accepts custom *serializer* and *deserializer* functions in place of **pickle.dumps()** and **pickle.loads()**.

Nota

Do not rely on the shelf being closed automatically; always call **close()** explicitly when you don't need it any more, or use **shelve.open()** as a context manager:

```
with shelve.open('spam') as db:
    db['eggs'] = 'eggs'
```

Aviso

Because the **shelve** module is backed by **pickle**, it is insecure to load a shelf from an untrusted source. Like with pickle, loading a shelf can execute arbitrary code.

Shelf objects support most of the methods and operations supported by dictionaries (except copying, constructors and operators `|` and `|=`). This eases the transition from dictionary based scripts to those requiring persistent storage.

Two additional methods are supported:

Shelf.sync()

Write back all entries in the cache if the shelf was opened with *writeback* set to `True`. Also empty the cache and synchronize the persistent dictionary on disk, if feasible. This is called automatically when `reorganize()` is called or the shelf is closed with `close()`.

Shelf.reorganize()

Calls `sync()` and attempts to shrink space used on disk by removing empty space resulting from deletions.

Adicionado na versão 3.15.

Shelf.close()

Synchronize and close the persistent *dict* object. Operations on a closed shelf will fail with a `ValueError`.

Ver também

Persistent dictionary recipe [https://code.activestate.com/recipes/576642-persistent-dict-with-multiple-standard-file-format/] with widely supported storage formats and having the speed of native dictionaries.

Restrições

- The choice of which database package will be used (such as `dbm.ndbm` or `dbm.gnu`) depends on which interface is available. Therefore it is not safe to open the database directly using `dbm`. The database is also (unfortunately) subject to the limitations of `dbm`, if it is used — this

means that (the pickled representation of) the objects stored in the database should be fairly small, and in rare cases key collisions may cause the database to refuse updates.

- The **shelve** module does not support *concurrent* read/write access to shelved objects. (Multiple simultaneous read accesses are safe.) When a program has a shelf open for writing, no other program should have it open for reading or writing. Unix file locking can be used to solve this, but this differs across Unix versions and requires knowledge about the database implementation used.
- On macOS `dbm.ndbm` can silently corrupt the database file on updates, which can cause hard crashes when trying to read from the database.
- `Shelf.reorganize()` may not be available for all database packages and may temporarily increase resource usage (especially disk space) when called. Additionally, it will never run automatically and instead needs to be called explicitly.

class shelve.Shelf(*dict*, *protocol=None*, *writeback=False*, *keyencoding='utf-8'*, ***, *serializer=None*, *deserializer=None*)

A subclass of `collections.abc.MutableMapping` which stores pickled values in the *dict* object.

By default, pickles created with `pickle.DEFAULT_PROTOCOL` are used to serialize values. The version of the pickle protocol can be specified with the *protocol* parameter. See the `pickle` documentation for a discussion of the pickle protocols.

If the *writeback* parameter is `True`, the object will hold a cache of all entries accessed and write them back to the *dict* at sync and close times. This allows natural operations on mutable entries, but can consume much more memory and make sync and close take a long time.

The *keyencoding* parameter is the encoding used to encode keys before they are used with the underlying dict.

The *serializer* and *deserializer* parameters have the same interpretation as in `open()`.

A **`Shelf`** object can also be used as a context manager, in which case it will be automatically closed when the **`with`** block ends.

Alterado na versão 3.2: Added the *keyencoding* parameter; previously, keys were always encoded in UTF-8.

Alterado na versão 3.4: Adicionado suporte a gerenciador de contexto.

Alterado na versão 3.10: **`pickle.DEFAULT_PROTOCOL`** is now used as the default pickle protocol.

Alterado na versão 3.15: Added the *serializer* and *deserializer* parameters.

class shelve.BsdDbShelf(*dict*, *protocol=None*, *writeback=False*, *keyencoding='utf-8'*, ***, *serializer=None*, *deserializer=None*)

A subclass of **`Shelf`** which exposes **`first()`**, **`next()`**, **`previous()`**, **`last()`** and **`set_location()`** methods. These are available in the third-party **`bsddb`** module from pybsddb [https://www.jcea.es/programacion/pybsddb.htm] but not in other database modules. The *dict* object passed to the constructor must support those methods. This is generally accomplished by calling one of **`bsddb.hashopen()`**, **`bsddb.btopen()`** or **`bsddb.rnopen()`**. The optional *protocol*, *writeback*, *keyencoding*, *serializer* and *deserializer* parameters have the same interpretation as in **`open()`**.

Alterado na versão 3.15: Added the *serializer* and *deserializer* parameters.

class shelve.DbfilenameShelf(*filename*, *flag='c'*, *protocol=None*, *writeback=False*, ***, *serializer=None*, *deserializer=None*)

A subclass of **`Shelf`** which accepts a *filename* instead of a dict-like object. The underlying file will be opened using **`dbm.open()`**. By default, the file will be created and opened for both read and write. The optional *flag* parameter has the same interpretation as for the **`open()`** function. The optional *protocol*, *writeback*, *serializer* and *deserializer* parameters have the same interpretation as in **`open()`**.

Alterado na versão 3.15: Added the *serializer* and *deserializer* parameters.

Exemplo

To summarize the interface (`key` is a string, `data` is an arbitrary object):

```
import shelve

d = shelve.open(filename)  # open -- file may get suffix added b
                           # library

d[key] = data              # store data at key (overwrites old c
                           # using an existing key)
data = d[key]              # retrieve a COPY of data at key (rai
                           # if no such key)
del d[key]                 # delete data stored at key (raises K
                           # if no such key)

flag = key in d            # true if the key exists
klist = list(d.keys())     # a list of all existing keys (slow!)

# as d was opened WITHOUT writeback=True, beware:
d['xx'] = [0, 1, 2]        # this works as expected, but...
d['xx'].append(3)          # *this doesn't!* -- d['xx'] is STILL

# having opened d without writeback=True, you need to code caref
temp = d['xx']             # extracts the copy
temp.append(5)             # mutates the copy
d['xx'] = temp             # stores the copy right back, to pers

# or, d=shelve.open(filename,writeback=True) would let you just
# d['xx'].append(5) and have it work as expected, BUT it would a
# consume more memory and make the d.close() operation slower.

d.close()                  # close it
```

Exceções

exception shelve.ShelveError

Exception raised when one of the arguments *deserializer* and *serializer* is missing in the `open()`, `Shelf`, `BsdDbShelf` and `DbfilenameShelf`.

The *deserializer* and *serializer* arguments must be given together.

Adicionado na versão 3.15.

Ver também

Módulo `dbm`

Generic interface to `dbm`-style databases.

Módulo `pickle`

Object serialization used by **`shelve`**.

`marshal` — Serialização interna de objetos Python

Este módulo contém funções que podem ler e gravar valores Python em formato binário. O formato é específico para Python, mas independente dos problemas de arquitetura da máquina (por exemplo, você pode gravar um valor Python em um arquivo em um PC, transportar o arquivo para um Mac e lê-lo de volta lá). Os detalhes do formato não são documentados propositalmente; ele pode mudar entre as versões do Python (embora raramente mude). [[1]]

This is not a general “persistence” module. For general persistence and transfer of Python objects through RPC calls, see the modules **`pickle`** and **`shelve`**. The **`marshal`** module exists mainly to support reading and writing the “pseudo-compiled” code for Python modules of `.pyc` files. Therefore, the Python maintainers reserve the right to modify the marshal format in backward incompatible ways should the need arise. The format of code objects is not compatible between Python versions, even if the version of the format is the same. De-serializing a code object in the incorrect Python version has undefined behavior. If you’re serializing and de-serializing Python objects, use the **`pickle`** module instead – the performance is comparable, version independence is guaranteed, and pickle supports a substantially wider range of objects than marshal.

Aviso

The **`marshal`** module is not intended to be secure against erroneous or maliciously constructed data. Never unmarshal data received from an untrusted or unauthenticated source.

Existem funções que leem/gravam arquivos, bem como funções que operam em objetos bytes ou similares.

Not all Python object types are supported; in general, only objects whose value is independent from a particular invocation of Python can be written and read by this module. The following types are supported:

- Numeric types: `int`, `bool`, `float`, `complex`.
- Strings (`str`) and `bytes`. Bytes-like objects like `bytearray` are marshalled as `bytes`.
- Containers: `tuple`, `list`, `dict`, `frozendict` (since `version` 6), `set`, `frozenset`, and `slice` (since `version` 5). It should be understood that these are supported only if the values contained therein are themselves supported. Recursive containers are supported since `version` 3.
- The singletons `None`, `Ellipsis` and `StopIteration`.
- `code` objects, if *allow_code* is true. See note above about version dependence.

Alterado na versão 3.4:

- Added format version 3, which supports marshalling recursive lists, sets and dictionaries.
- Added format version 4, which supports efficient representations of short strings.

Alterado na versão 3.14: Added format version 5, which allows marshalling slices.

Alterado na versão 3.15: Added format version 6, which allows marshalling `frozendict`.

O módulo define estas funções:

marshal.dump(*value*, *file*, *version=version*, /, *, *allow_code=True*)

Grava o valor no arquivo aberto. O valor deve ser um tipo compatível. O arquivo deve ser arquivo binário gravável.

Se o valor tem (ou contém um objeto que tem) um tipo não suportado, uma exceção `ValueError` é levantada – mas dados de lixo também serão gravados no arquivo. O objeto não será lido corretamente por

load(). Há suporte a objetos código somente se *allow_code* for verdadeiro.

O argumento *version* indica o formato de dados que o `dump` deve usar (veja abaixo).

Levanta um evento de auditoria `marshal.dumps` com os argumentos `value`, `version`.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

marshal.load(*file*, /, *, *allow_code=True*)

Lê um valor do arquivo aberto e retorna-o. Se nenhum valor válido for lido (por exemplo, porque os dados têm um formato de empacotamento incompatível com uma versão diferente do Python), levanta **EOFError**, **ValueError** ou **TypeError**. Há suporte a objetos código somente se *allow_code* for verdadeiro. O arquivo deve ser um arquivo binário legível.

Levanta um evento de auditoria `marshal.load` sem argumentos.

Nota

Se um objeto contendo um tipo não suportado foi empacotado com **dump()**, **load()** irá substituir `None` pelo tipo não empacotável.

Alterado na versão 3.10: Esta chamada costumava levantar um evento de auditoria `code.__new__` para cada objeto código. Agora, ele levanta um único evento `marshal.load` para toda a operação de carregamento.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

marshal.dumps(*value*, *version=version*, /, *, *allow_code=True*)

Retorna o objeto bytes que seria escrito em um arquivo por `dump(value, file)`. O valor deve ser um tipo compatível. Levanta uma exceção **ValueError** se o valor tem (ou contém um objeto que tem) um

tipo não suportado. Há suporte a objetos código somente se *allow_code* for verdadeiro.

O argumento *version* indica o formato de dados que `dumps` deve usar (veja abaixo).

Levanta um evento de auditoria `marshal.dumps` com os argumentos `value`, `version`.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

marshal.loads(*bytes*, */*, ***, *allow_code=True*)

Converte o objeto bytes ou similar em um valor. Se nenhum valor válido for encontrado, levanta **`EOFError`**, **`ValueError`** ou **`TypeError`**. Há suporte a objetos código somente se *allow_code* for verdadeiro. Bytes extras na entrada são ignorados.

Levanta um evento de auditoria `marshal.loads` com o argumento `bytes`.

Alterado na versão 3.10: Esta chamada costumava levantar um evento de auditoria `code.__new__` para cada objeto código. Agora, ele levanta um único evento `marshal.loads` para toda a operação de carregamento.

Alterado na versão 3.13: Adicionado o parâmetro *allow_code*.

Além disso, as seguintes constantes são definidas:

marshal.version

Indicates the format that the module uses. Version 0 is the historical first version; subsequent versions add new features. Generally, a new version becomes the default when it is introduced.

Versão	Available since	Novas funcionalidades
1	Python 2.4	Sharing interned strings

Versão	Available since	Novas funcionalidades
2	Python 2.5	Binary representation of floats
3	Python 3.4	Support for object instancing and recursion
4	Python 3.4	Efficient representation of short strings
5	Python 3.14	Support for `slice` objects
6	Python 3.15	Support for `frozendict` objects

Notas de rodapé

[1]

O nome deste módulo deriva de um pouco da terminologia usada pelos designers do Modula-3 (entre outros), que usam o termo “marshalling” para enviar dados em um formato independente. Estritamente falando, “to marshal” significa converter alguns dados da forma interna para a externa (em um buffer RPC, por exemplo) e “unmarshalling” para o processo reverso.

`dbm` — Interfaces para “banco de dados” Unix

Código-fonte: Lib/dbm/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/dbm/__init__.py]

`dbm` is a generic interface to variants of the DBM database:

- `dbm.sqlite3`
- `dbm.gnu`
- `dbm.ndbm`

If none of these modules are installed, the slow-but-simple implementation in module `dbm.dumb` will be used. There is a third party interface [https://www.jcea.es/programacion/pybsddb.htm] to the Oracle Berkeley DB.

> **Nota**
>
> None of the underlying modules will automatically shrink the disk space used by the database file. However, `dbm.sqlite3`, `dbm.gnu` and `dbm.dumb` provide a `reorganize()` method that can be used for this purpose.

exception dbm.error

Uma tupla contendo as exceções que podem ser levantadas por cada um dos módulos suportados, com a única exceção também chamada `dbm.error` como o primeiro item — o último é usado quando `dbm.error` é levantada.

dbm.whichdb(*filename*)

Esta função tenta adivinhar qual dos vários módulos de banco de dados simples disponíveis — `dbm.sqlite3`, `dbm.gnu`, `dbm.ndbm` ou `dbm.dumb` — deve ser usado para abrir o arquivo fornecido.

Retorna um dos seguintes valores:

- `None` se o arquivo não puder ser aberto porque está ilegível ou não existe
- a string vazia (`''`) se o formato do arquivo não puder ser adivinhado
- uma string contendo o nome do módulo necessário, tal como `'dbm.ndbm'` ou `'dbm.gnu'`

Alterado na versão 3.11: filename aceita um objeto caminho ou similar.

dbm.open(*file*, *flag='r'*, *mode=0o666*)

Abre um banco de dados e retorna o objeto banco de dados correspondendo.

Parâmetros::

- **file** (path-like object) – O arquivo de banco de dados para abrir. Se o arquivo de banco de dados j existe, a função **`whichdb()`** é usada para determina seu tipo e o módulo apropriado é usado; se ele nãc existir, o primeiro submódulo listado acima que pode ser importado é usado.
- **flag** (*str*) –
  - `'r'` (padrão): Open existing database for readin only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'`: Open database for reading and writing, creating it if it doesn't exist.
  - `'n'`: Always create a new, empty database, oper for reading and writing.
- **mode** (*int*) – The Unix file access mode of the file (default: octal `0o666`), used only when the databas has to be created.

Alterado na versão 3.11: file aceita um objeto caminho ou similar.

O objeto retornado por **`open()`** oferece suporte à funcionalidade básica de mapeamentos mutáveis; chaves e seus valores correspondentes podem ser armazenados, recuperados e excluídos, e iteração, o operador **`in`** e os métodos **`keys()`**, **`get()`**, **`setdefault()`** e **`clear()`** estão disponíveis. O método **`keys()`**

retorna uma lista em vez de um objeto de visualização. O método **`setdefault()`** requer dois argumentos.

Chaves e valores são sempre armazenados como **`bytes`**. Isso significa que quando strings são usadas, elas são implicitamente convertidas para a codificação padrão antes de serem armazenadas.

Estes objetos também oferecem suporte a serem usados em uma instrução **`with`**, que vai fechá-los automaticamente quando estiver concluída.

Alterado na versão 3.2: **`get()`** and **`setdefault()`** methods are now available for all **`dbm`** backends.

Alterado na versão 3.4: Adicionado suporte nativo para o protocolo de gerenciamento de contexto para os objetos retornados por **`open()`**.

Alterado na versão 3.8: Excluir uma chave de um banco de dados somente leitura levanta uma exceção de banco de dados específica do módulo em vez de **`KeyError`**.

Alterado na versão 3.13: **`clear()`** methods are now available for all **`dbm`** backends.

Os seguintes exemplos registram alguns hostnames e um título correspondente, e então exibe o conteúdo do banco de dados:

```
import dbm

# Abre o banco de dados, criando-o se necessário.
with dbm.open('cache', 'c') as db:

    # Registra alguns valores
    db[b'hello'] = b'there'
    db['www.python.org'] = 'Python Website'
    db['www.cnn.com'] = 'Cable News Network'

    # Note que as chaves são consideradas bytes agora.
    assert db[b'www.python.org'] == b'Python Website'
    # Observe como o valor é agora em bytes.
    assert db['www.cnn.com'] == b'Cable News Network'
```

```
    # Métodos geralmente usados da interface dict funcionam també
    print(db.get('python.org', b'not present'))

    # Armazenar uma chave não-string ou valor  vai levantar uma e
    # (provavelmente uma TypeError).
    db['www.yahoo.com'] = 4

# db é automaticamente fechado a sair da instrução with.
```

Ver também

Módulo `shelve`
Módulo persistente que armazena dados não-string.

Os submódulos individuais são descritos nas seções a seguir.

`dbm.sqlite3` — SQLite backend for dbm

Adicionado na versão 3.13.

Código-fonte: Lib/dbm/sqlite3.py
[https://github.com/python/cpython/tree/3.15/Lib/dbm/sqlite3.py]

This module uses the standard library **`sqlite3`** module to provide an SQLite backend for the **`dbm`** module. The files created by **`dbm.sqlite3`** can thus be opened by **`sqlite3`**, or any other SQLite browser, including the SQLite CLI.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

dbm.sqlite3.open(*filename*, /, *flag='r'*, *mode=0o666*)
Abre um banco de dados SQLite.

Parâmetros::

- **filename** (path-like object) – O caminho para o banco de dados a ser aberta.
- **flag** (*str*) –
  - `'r'` (padrão): Open existing database for readin only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'`: Open database for reading and writing, creating it if it doesn't exist.
  - `'n'`: Always create a new, empty database, oper for reading and writing.
- **mode** – O modo de acesso a arquivos do Unix do arquivo (padrão: octal `0o666`), usado apenas quando o banco de dados tem que ser criado.

O objeto de banco de dados retornado se comporta de forma semelhante a um mapeamento mutável, mas o método **`keys()`** retorna uma lista e o método **`setdefault()`** requer dois argumentos. Ele também oferece suporte a um gerenciador de contexto de "fechamento" por meio da palavra reservada **`with`**.

Os seguintes métodos também são fornecidos:

sqlite3.close()

Fecha o banco de dados SQLite.

sqlite3.reorganize()

If you have carried out a lot of deletions and would like to shrink the space used on disk, this method will reorganize the database; otherwise, deleted file space will be kept and reused as new (key, value) pairs are added.

Nota

While reorganizing, as much as two times the size of the original database is required in free disk space. However, be aware that this factor changes for each **`dbm`** submodule.

Adicionado na versão 3.15.

`dbm.gnu` — GNU database manager

Código-fonte: Lib/dbm/gnu.py
[https://github.com/python/cpython/tree/3.15/Lib/dbm/gnu.py]

The `dbm.gnu` module provides an interface to the GDBM library, similar to the `dbm.ndbm` module, but with additional functionality like crash tolerance.

> **Nota**
>
> The file formats created by `dbm.gnu` and `dbm.ndbm` are incompatible and can not be used interchangeably.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Disponibilidade: Unix.

exception dbm.gnu.error

Raised on `dbm.gnu`-specific errors, such as I/O errors. `KeyError` is raised for general mapping errors like specifying an incorrect key.

dbm.gnu.open_flags

Uma sequência de caracteres que o parâmetro *flag* de `open()` suporta.

dbm.gnu.open(*filename*, *flag='r'*, *mode=0o666*, /)

Abre um banco de dados GDBM e retorna um objeto `gdbm`.

Parâmetros::

- **filename** (path-like object) – O arquivo de banco de dados para abrir.
- **flag** (*str*) –
  - `'r'` (padrão): Open existing database for readin only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'`: Open database for reading and writing, creating it if it doesn't exist.
  - `'n'`: Always create a new, empty database, oper for reading and writing.

  Os seguintes caracteres adicionais podem ser acrescentados para controlar como o banco de dados é aberto:
  - `'f'`: Abre o banco de dados em modo rápido. A gravações no banco de dados não serão sincronizadas.
  - `'s'`: Modo sincronizado. Alterações no banco d dados serão gravadas imediatamente no arquivc
  - `'u'`: Não trava o banco de dados.

  Nem todos os sinalizadores são válidos para todas as versões do GDBM. Veja o membro **`open_flags`** para uma lista de caracteres de sinalizadores suportados.
- **mode** (*int*) – The Unix file access mode of the file (default: octal `0o666`), used only when the databas has to be created.

Levanta:: **error** – Se um argumento *flag* inválido for passado.

Alterado na versão 3.11: filename aceita um objeto caminho ou similar.

Objetos **`gdbm`** se comportam de forma semelhante aos mapeamentos mutáveis, mas os métodos **`items()`**, **`values()`**, **`pop()`**, **`popitem()`** e **`update()`** não são suportados, **`keys()`** retorna uma lista e o método **`setdefault()`** requer dois argumentos. Ele também oferece suporte a um gerenciador de contexto de "fechamento" por meio da palavra reservada **`with`**.

Alterado na versão 3.2: Adiciionados os métodos **`get()`** e **`setdefault()`**.

Alterado na versão 3.13: Adicionado o método **`clear()`**.

Os seguintes métodos também são fornecidos:

gdbm.close()

Fecha o banco de dados GDBM.

gdbm.firstkey()

É possível fazer um laço em cada chave no banco de dados usando este método e o método **`nextkey()`**. A travessia é ordenada pelos valores de hash internos do GDBM e não será classificada pelos valores de chave. Este método retorna a chave inicial.

gdbm.nextkey(*key*)

Retorna a chave que segue *key* na travessia. O código a seguir imprime cada chave no banco de dados `db`, sem precisar criar uma lista na memória que contenha todas elas:

```
k = db.firstkey()
while k is not None:
    print(k)
    k = db.nextkey(k)
```

gdbm.reorganize()

Se você realizou muitas exclusões e gostaria de reduzir o espaço usado pelo arquivo GDBM, esta rotina reorganizará o banco de dados. Objetos **`gdbm`** não reduzirão o tamanho de um arquivo de banco de dados, exceto usando esta reorganização; caso contrário, o espaço de arquivo excluído será mantido e reutilizado à medida que novos pares (chave, valor) forem adicionados.

Nota

While reorganizing, as much as one time the size of the original database is required in free disk space. However, be aware that this factor changes for each **`dbm`** submodule.

gdbm.sync()

Quando o banco de dados é aberto no modo rápido, esse método força a gravação de todos os dados não gravados no disco.

`dbm.ndbm` — New Database Manager

Código-fonte: Lib/dbm/ndbm.py
[https://github.com/python/cpython/tree/3.15/Lib/dbm/ndbm.py]

The `dbm.ndbm` module provides an interface to the NDBM library. This module can be used with the "classic" NDBM interface or the GDBM compatibility interface.

Nota

The file formats created by `dbm.gnu` and `dbm.ndbm` are incompatible and can not be used interchangeably.

Aviso

A biblioteca NDBM enviada como parte do macOS tem uma limitação não documentada no tamanho dos valores, o que pode resultar em arquivos de banco de dados corrompidos ao armazenar valores maiores que esse limite. Ler esses arquivos corrompidos pode resultar em uma falha grave (falha de segmentação).

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Disponibilidade: Unix.

exception dbm.ndbm.error

Raised on **dbm.ndbm**-specific errors, such as I/O errors. **KeyError** is raised for general mapping errors like specifying an incorrect key.

dbm.ndbm.library

Nome da biblioteca de implementação NDBM usada.

dbm.ndbm.open(*filename*, *flag='r'*, *mode=0o666*, */*)

Abre um banco de dados NDBM e retorna um objeto **ndbm**.

Parâmetros::

- **filename** (path-like object) – O nome base do arquivo de banco de dados (sem as extensões `.dir` ou `.pag`).
- **flag** (*str*) –
  - `'r'` (padrão): Open existing database for readin only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'`: Open database for reading and writing, creating it if it doesn't exist.
  - `'n'`: Always create a new, empty database, oper for reading and writing.
- **mode** (*int*) – The Unix file access mode of the file (default: octal `0o666`), used only when the databas has to be created.

Alterado na versão 3.11: Aceita um objeto caminho ou similar como nome de arquivo.

Objetos **ndbm** se comportam de forma semelhante aos mapeamentos mutáveis, mas os métodos **items()**, **values()**, **pop()**, **popitem()** e **update()** não são suportados, **keys()** retorna uma lista e o método **setdefault()** requer dois argumentos. Ele também oferece suporte a um gerenciador de contexto de "fechamento" por meio da palavra reservada **with**.

Alterado na versão 3.2: Adiciionados os métodos **get()** e **setdefault()**.

Alterado na versão 3.13: Adicionado o método **clear()**.

O seguinte método também é fornecido:

ndbm.close()

Fecha o banco de dados NDBM.

`dbm.dumb` — Portable DBM implementation

Código-fonte: Lib/dbm/dumb.py
[https://github.com/python/cpython/tree/3.15/Lib/dbm/dumb.py]

Nota

The `dbm.dumb` module is intended as a last resort fallback for the `dbm` module when a more robust module is not available. The `dbm.dumb` module is not written for speed and is not nearly as heavily used as the other database modules.

The `dbm.dumb` module provides a persistent `dict`-like interface which is written entirely in Python. Unlike other `dbm` backends, such as `dbm.gnu`, no external library is required.

O módulo `dbm.dumb` define o seguinte:

exception dbm.dumb.error

Raised on `dbm.dumb`-specific errors, such as I/O errors. `KeyError` is raised for general mapping errors like specifying an incorrect key.

dbm.dumb.open(*filename*, *flag='c'*, *mode=0o666*)

Abre um banco de dados `dbm.dumb`.

Parâmetros::

- **filename** – O nome base do arquivo do banco de dados (sem extensões). Um novo banco de dados cria os seguintes arquivos: - `filename.dat` - `filename.dir`
- **flag** (*str*) –
  - `'r'`: Open existing database for reading only.
  - `'w'`: Open existing database for reading and writing.
  - `'c'` (padrão): Open database for reading and writing, creating it if it doesn't exist.
  - `'n'`: Always create a new, empty database, open for reading and writing.
- **mode** (*int*) – The Unix file access mode of the file (default: octal `0o666`), used only when the database has to be created.

Aviso

É possível travar o interpretador Python ao carregar um banco de dados com uma entrada suficientemente grande/complexa devido a limitações de profundidade de pilha no compilador AST do Python.

Aviso

`dbm.dumb` does not support concurrent read/write access. (Multiple simultaneous read accesses are safe.) When a program has the database open for writing, no other program should have it open for reading or writing.

Alterado na versão 3.5: **`open()`** sempre cria um novo banco de dados quando *flag* é `'n'`.

Alterado na versão 3.8: Um banco de dados aberto somente leitura se *flag* for `'r'`. Um banco de dados não é criado se não existir se *flag* for `'r'` ou `'w'`.

Alterado na versão 3.11: filename aceita um objeto caminho ou similar.

O objeto de banco de dados retornado se comporta de forma semelhante a um mapeamento mutável, mas os métodos **`keys()`** e **`items()`** retornam uma lista e o método **`setdefault()`** requer dois argumentos. Ele também oferece suporte a um gerenciador de contexto de “fechamento” por meio da palavra reservada **`with`**.

Os seguintes métodos também são fornecidos:

dumbdbm.close()

Fecha o banco de dados.

dumbdbm.reorganize()

If you have carried out a lot of deletions and would like to shrink the space used on disk, this method will reorganize the database; otherwise, deleted file space will not be reused.

Nota

While reorganizing, no additional free disk space is required. However, be aware that this factor changes for each **`dbm`** submodule.

Adicionado na versão 3.15.

dumbdbm.sync()

Sincroniza o diretório no disco e os arquivos de dados. Este método é chamado pelo método **`shelve.Shelf.sync()`**.

`sqlite3` —- interface DB-API 2.0 interface para bancos de dados SQLite

Código-fonte: Lib/sqlite3/ [https://github.com/python/cpython/tree/3.15/Lib/sqlite3/]

SQLite é uma biblioteca C que fornece um banco de dados leve baseado em disco que não requer um processo de servidor separado e permite acessar o banco de dados usando uma variante não padrão da linguagem de consulta SQL. Algumas aplicações podem usar SQLite para armazenamento interno de dados. Também é possível prototipar uma aplicação usando SQLite e, em seguida, portar o código para um banco de dados maior, como PostgreSQL ou Oracle.

The **`sqlite3`** module was written by Gerhard Häring. It provides an SQL interface compliant with the DB-API 2.0 specification described by **PEP 249** [https://peps.python.org/pep-0249/], and requires the third-party SQLite [https://sqlite.org/] library.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Esse documento inclui quatro seções principais:

- Tutorial ensina como usar o módulo **`sqlite3`**.
- Referência descreve as classes e funções que este módulo define.
- Guias de como fazer detalha como lidar com tarefas específicas.
- Explicação fornece informações detalhadas sobre controle de transações.

Ver também

https://www.sqlite.org

A página web do SQLite; a documentação descreve a sintaxe e os tipos de dados disponíveis para o dialeto SQL suportado.

https://www.w3schools.com/sql/
Tutoriais, referências e exemplos para aprender a sintaxe SQL.

PEP 249 [https://peps.python.org/pep-0249/] - Especificação 2.0 da API de banco de dados
PEP escrita por Marc-André Lemburg.

Tutorial

Neste tutorial, você criará um banco de dados de filmes do Monty Python usando a funcionalidade básica **`sqlite3`**. Ele pressupõe uma compreensão fundamental dos conceitos de banco de dados, incluindo cursores [https://en.wikipedia.org/wiki/Cursor_(databases)] e transações [https://en.wikipedia.org/wiki/Database_transaction].

Primeiro, precisamos criar um novo banco de dados e abrir uma conexão com o banco de dados para permitir que **`sqlite3`** funcione com ele. Chame **`sqlite3.connect()`** para criar uma conexão com o banco de dados `tutorial.db` no diretório de trabalho atual, criando-o implicitamente se ele não existir:

```
import sqlite3
con = sqlite3.connect("tutorial.db")
```

O objeto **`Connection`** `con` retornado representa a conexão com o banco de dados em disco.

Para executar instruções SQL e buscar resultados de consultas SQL, precisaremos usar um cursor de banco de dados. Chame **`con.cursor()`** para criar o **`Cursor`**:

```
cur = con.cursor()
```

Agora que temos uma conexão com o banco de dados e um cursor, podemos criar uma tabela de banco de dados `movie` com colunas para título, ano de

lançamento e pontuação da revisão. Para simplificar, podemos apenas usar nomes de colunas na declaração da tabela - graças ao recurso tipagem flexível [https://www.sqlite.org/flextypegood.html] do SQLite, especificar os tipos de dados é opcional. Execute a instrução `CREATE TABLE` chamando **`cur.execute(...)`**:

```
cur.execute("CREATE TABLE movie(title, year, score)")
```

Podemos verificar se a nova tabela foi criada consultando a tabela embutida `sqlite_master` no SQLite, que agora deve conter uma entrada para a definição da tabela `movie` (veja The Schema Table [https://www.sqlite.org/schematab.html] para detalhes). Execute essa consulta chamando **`cur.execute(...)`**, atribua o resultado a `res` e chame **`res.fetchone()`** para buscar a linha resultante:

```
>>> res = cur.execute("SELECT name FROM sqlite_master")
>>> res.fetchone()
('movie',)
```

Podemos ver que a tabela foi criada, pois a consulta retorna **`tuple`** contendo o nome da tabela. Se fizermos uma consulta `sqlite_master` para uma tabela inexistente `spam`, **`res.fetchone()`** ela retornará `None`:

```
>>> res = cur.execute("SELECT name FROM sqlite_master WHERE name=
>>> res.fetchone() is None
True
```

Agora, adicione duas linhas de dados fornecidos como literais SQL executando uma instrução `INSERT`, mais uma vez chamando **`cur.execute(...)`**:

```
cur.execute("""
    INSERT INTO movie VALUES
        ('Monty Python and the Holy Grail', 1975, 8.2),
        ('And Now for Something Completely Different', 1971, 7.5)
""")
```

A instrução `INSERT` abre implicitamente uma transação, que precisa ser confirmada antes que as alterações sejam salvas no banco de dados (veja

Controle de transações para detalhes). Chame **`con.commit()`** no objeto de conexão para confirmar a transação:

```
con.commit()
```

Podemos verificar que os dados foram inseridos corretamente executando uma consulta `SELECT`. Use o já conhecido **`cur.execute(...)`** para atribuir o resultado a `res` e chame **`res.fetchall()`** para retornar todas as linhas resultantes.

```
>>> res = cur.execute("SELECT score FROM movie")
>>> res.fetchall()
[(8.2,), (7.5,)]
```

O resultado é uma **`list`** de duas **`tuple`**s, uma por linha, cada uma contendo o valor `score` dessa linha.

Agora, insira mais três linhas chamando **`cur.executemany(...)`**:

```
data = [
    ("Monty Python Live at the Hollywood Bowl", 1982, 7.9),
    ("Monty Python's The Meaning of Life", 1983, 7.5),
    ("Monty Python's Life of Brian", 1979, 8.0),
]
cur.executemany("INSERT INTO movie VALUES(?, ?, ?)", data)
con.commit()  # Lembre-sede executar a transação após o INSERT.
```

Observe que espaços reservados `?` são usados para vincular `data` à consulta. Sempre use espaços reservados em vez de formatação de string para vincular valores Python a instruções SQL, para evitar ataques de injeção de SQL [https://en.wikipedia.org/wiki/SQL_injection] (consulte How to use placeholders to bind values in SQL queries para mais detalhes).

Podemos verificar que as novas linhas foram inseridas executando uma consulta `SELECT`, desta vez iterando sobre os resultados da consulta.

```
>>> for row in cur.execute("SELECT year, title FROM movie ORDER B
...     print(row)
(1971, 'And Now for Something Completely Different')
(1975, 'Monty Python and the Holy Grail')
(1979, "Monty Python's Life of Brian")
```

```
(1982, 'Monty Python Live at the Hollywood Bowl')
(1983, "Monty Python's The Meaning of Life")
```

Cada linha é uma **tuple** de dois itens `(year, title)`, correspondendo às colunas selecionadas na consulta.

Finalmente, verifique se o banco de dados foi gravado no disco chamando **`con.close()`** para fechar a conexão existente, abrir uma nova, criar um novo cursor e, em seguida, consultar o banco de dados.

```
>>> con.close()
>>> new_con = sqlite3.connect("tutorial.db")
>>> new_cur = new_con.cursor()
>>> res = new_cur.execute("SELECT title, year FROM movie ORDER BY
>>> title, year = res.fetchone()
>>> print(f'The highest scoring Monty Python movie is {title!r},
The highest scoring Monty Python movie is 'Monty Python and the H
>>> new_con.close()
```

Você agora criou um banco de dados SQLite usando o módulo **`sqlite3`**, inseriu dados e recuperou valores dele de várias maneiras.

Ver também

- Guias de como fazer para leitura adicional:
  - How to use placeholders to bind values in SQL queries
  - How to adapt custom Python types to SQLite values
  - How to convert SQLite values to custom Python types
  - How to use the connection context manager
  - How to create and use row factories
- Explicação para obter informações detalhadas sobre o controle de transações.

Referência

Funções do módulo

sqlite3.connect(*database*, *, *timeout=5.0*, *detect_types=0*, *isolation_level='DEFERRED'*, *check_same_thread=True*, *factory=sqlite3.Connection*, *cached_statements=128*, *uri=False*, *autocommit=sqlite3.LEGACY_TRANSACTION_CONTROL*)

Abra uma conexão com um banco de dados SQLite.

Parâmetros::

- **database** (path-like object) – O caminho para o arquivo do banco de dados a ser aberto. Você pode passar `":memory:"` para criar um banco de dados SQLite que existirá apenas na memória [https://sqlite.org/inmemorydb.html], e abrir uma conexão com ele.
- **timeout** (*float*) – Quantos segundos a conexão deve aguardar antes de levantar uma exceção `OperationalError` quando uma tabela estiver bloqueada. Se outra conexão abrir uma transação para modificar uma tabela, essa tabela permanecer bloqueada até que a transação seja confirmada. O padrão é cinco segundos.
- **detect_types** (*int*) – Control whether and how data types not natively supported by SQLite are looked up to be converted to Python types, using the converters registered with `register_converter()`. Set it to any combination (using `|`, bitwise or) of `PARSE_DECLTYPES` and `PARSE_COLNAMES` to enable this. Column names take precedence over declared types if both flags are set. By default (`0`), type detection is disabled.
- **isolation_level** (*str* | *None*) – Control legacy transaction handling behaviour. See `Connection.isolation_level` and Controle de transação através do atributo isolation_level for more information. Can be `"DEFERRED"` (default), `"EXCLUSIVE"` or `"IMMEDIATE"`; or `None` to disable opening transactions implicitly. Has no effect unless `Connection.autocommit` is set to `LEGACY_TRANSACTION_CONTROL` (the default).
- **check_same_thread** (*bool*) – If `True` (default),

ProgrammingError will be raised if the database connection is used by a thread other than the one that created it. If `False`, the connection may be accessed in multiple threads; write operations may need to be serialized by the user to avoid data corruption. See **threadsafety** for more information.

- **factory** (*Connection*) – A custom subclass of **Connection** to create the connection with, if not the default **Connection** class.
- **cached_statements** (*int*) – The number of statements that **sqlite3** should internally cache fc this connection, to avoid parsing overhead. By default, 128 statements.
- **uri** (*bool*) – If set to `True`, *database* is interpreted as a URI with a file path and an optional query string. The scheme part *must* be `"file:"`, and the path can be relative or absolute. The query string allows passing parameters to SQLite, enabling various How to work with SQLite URIs.
- **autocommit** (*bool*) – Control **PEP 249** [https://peps.python.org/pep-0249/] transaction handling behaviour. See **Connection.autocommit** and Controle de transações através do atributo autocommit for more information. *autocommit* currently defaults to **LEGACY_TRANSACTION_CONTROL**. The default will change to `False` in a future Python release.

Tipo de retorno:: *Connection*

Levanta um evento de auditoria `sqlite3.connect` com o argumento `database`.

Levanta um evento de auditoria `sqlite3.connect/handle` com o argumento `connection_handle`.

Alterado na versão 3.4: Adicionado o parâmetro *uri*.

Alterado na versão 3.7: *database* can now also be a path-like object, not only a string.

Alterado na versão 3.10: Added the `sqlite3.connect/handle` auditing event.

Alterado na versão 3.12: Added the *autocommit* parameter.

Alterado na versão 3.15: All parameters except *database* are now keyword-only.

sqlite3.complete_statement(*statement*)

Return `True` if the string *statement* appears to contain one or more complete SQL statements. No syntactic verification or parsing of any kind is performed, other than checking that there are no unclosed string literals and the statement is terminated by a semicolon.

Por exemplo:

```
>>> sqlite3.complete_statement("SELECT foo FROM bar;")
True
>>> sqlite3.complete_statement("SELECT foo")
False
```

This function may be useful during command-line input to determine if the entered text seems to form a complete SQL statement, or if additional input is needed before calling **`execute()`**.

See **`runsource()`** in Lib/sqlite3/__main__.py [https://github.com/python/cpython/tree/3.15/Lib/sqlite3/__main__.py] for real-world use.

sqlite3.enable_callback_tracebacks(*flag*, /)

Enable or disable callback tracebacks. By default you will not get any tracebacks in user-defined functions, aggregates, converters, authorizer callbacks etc. If you want to debug them, you can call this function with *flag* set to `True`. Afterwards, you will get tracebacks from callbacks on **`sys.stderr`**. Use `False` to disable the feature again.

Nota

Errors in user-defined function callbacks are logged as unraisable exceptions. Use an **`unraisable_hook_handler`** for introspection of the failed callback.

sqlite3.register_adapter(*type*, *adapter*, /)

Register an *adapter* callable to adapt the Python type *type* into an SQLite type. The adapter is called with a Python object of type *type* as its sole argument, and must return a value of a type that SQLite natively understands.

sqlite3.register_converter(*typename*, *converter*, /)

Register the *converter* callable to convert SQLite objects of type *typename* into a Python object of a specific type. The converter is invoked for all SQLite values of type *typename*; it is passed a **`bytes`** object and should return an object of the desired Python type. Consult the parameter *detect_types* of **`connect()`** for information regarding how type detection works.

Note: *typename* and the name of the type in your query are matched case-insensitively.

Constantes do módulo

sqlite3.LEGACY_TRANSACTION_CONTROL

Set **`autocommit`** to this constant to select old style (pre-Python 3.12) transaction control behaviour. See Controle de transação através do atributo isolation_level for more information.

sqlite3.PARSE_DECLTYPES

Pass this flag value to the *detect_types* parameter of **`connect()`** to look up a converter function using the declared types for each column. The types are declared when the database table is created. **`sqlite3`** will look up a converter function using the first word of the declared type as the converter dictionary key. For example:

```
CREATE TABLE test(
   i integer primary key,  ! will look up a converter named "i
```

```
  p point,                  ! will look up a converter named "p
  n number(10)              ! will look up a converter named "n
)
```

This flag may be combined with **PARSE_COLNAMES** using the `|` (bitwise or) operator.

Nota

Generated fields (for example `MAX(p)`) are returned as **str**. Use **PARSE_COLNAMES** to enforce types for such queries.

sqlite3.PARSE_COLNAMES

Pass this flag value to the *detect_types* parameter of **connect()** to look up a converter function by using the type name, parsed from the query column name, as the converter dictionary key. The query column name must be wrapped in double quotes (`"`) and the type name must be wrapped in square brackets (`[]`).

```
SELECT MAX(p) as "p [point]" FROM test;  ! will look up conver
```

This flag may be combined with **PARSE_DECLTYPES** using the `|` (bitwise or) operator.

sqlite3.SQLITE_OK
sqlite3.SQLITE_DENY
sqlite3.SQLITE_IGNORE

Flags that should be returned by the *authorizer_callback* callable passed to **Connection.set_authorizer()**, to indicate whether:

- Access is allowed (**SQLITE_OK**),
- The SQL statement should be aborted with an error (**SQLITE_DENY**)
- The column should be treated as a `NULL` value (**SQLITE_IGNORE**)

sqlite3.apilevel

String constant stating the supported DB-API level. Required by the DB-API. Hard-coded to `"2.0"`.

sqlite3.paramstyle

String constant stating the type of parameter marker formatting expected by the **`sqlite3`** module. Required by the DB-API. Hard-coded to `"qmark"`.

Nota

The `named` DB-API parameter style is also supported.

sqlite3.sqlite_version

Version number of the runtime SQLite library as a **`string`**.

sqlite3.sqlite_version_info

Version number of the runtime SQLite library as a **`tuple`** of **`integers`**.

sqlite3.SQLITE_KEYWORDS

A **`tuple`** containing all SQLite keywords.

This constant is only available if Python was compiled with SQLite 3.24.0 or greater.

Adicionado na versão 3.15.

sqlite3.threadsafety

Integer constant required by the DB-API 2.0, stating the level of thread safety the **`sqlite3`** module supports. This attribute is set based on the default threading mode [https://sqlite.org/threadsafe.html] the underlying SQLite library is compiled with. The SQLite threading modes are:

1. **Single-thread**: In this mode, all mutexes are disabled and SQLite is unsafe to use in more than a single thread at once.
2. **Multi-thread**: In this mode, SQLite can be safely used by multiple threads provided that no single database connection is used simultaneously in two or more threads.
3. **Serialized**: In serialized mode, SQLite can be safely used by multiple threads with no restriction.

The mappings from SQLite threading modes to DB-API 2.0 threadsafety levels are as follows:

SQLite threading mode	threadsafety	SQLITE_THREADSAFE	DB-API 2.0 meaning
single-thread	0	0	Threads may not share the module
multi-thread	1	2	Threads may share the module, but not connections
serialized	3	1	Threads may share the module, connections and cursors

Alterado na versão 3.11: Set *threadsafety* dynamically instead of hard-coding it to `1`.

sqlite3.SQLITE_DBCONFIG_DEFENSIVE
sqlite3.SQLITE_DBCONFIG_DQS_DDL
sqlite3.SQLITE_DBCONFIG_DQS_DML
sqlite3.SQLITE_DBCONFIG_ENABLE_FKEY
sqlite3.SQLITE_DBCONFIG_ENABLE_FTS3_TOKENIZER
sqlite3.SQLITE_DBCONFIG_ENABLE_LOAD_EXTENSION
sqlite3.SQLITE_DBCONFIG_ENABLE_QPSG
sqlite3.SQLITE_DBCONFIG_ENABLE_TRIGGER
sqlite3.SQLITE_DBCONFIG_ENABLE_VIEW
sqlite3.SQLITE_DBCONFIG_LEGACY_ALTER_TABLE
sqlite3.SQLITE_DBCONFIG_LEGACY_FILE_FORMAT
sqlite3.SQLITE_DBCONFIG_NO_CKPT_ON_CLOSE
sqlite3.SQLITE_DBCONFIG_RESET_DATABASE
sqlite3.SQLITE_DBCONFIG_TRIGGER_EQP

sqlite3.SQLITE_DBCONFIG_TRUSTED_SCHEMA
sqlite3.SQLITE_DBCONFIG_WRITABLE_SCHEMA

These constants are used for the `Connection.setconfig()` and `getconfig()` methods.

The availability of these constants varies depending on the version of SQLite Python was compiled with.

Adicionado na versão 3.12.

Ver também

https://www.sqlite.org/c3ref/c_dbconfig_defensive.html
SQLite docs: Database Connection Configuration Options

Descontinuado desde a versão 3.12, removido na versão 3.14: The `version` and `version_info` constants.

Connection objects

class sqlite3.Connection

Each open SQLite database is represented by a `Connection` object, which is created using `sqlite3.connect()`. Their main purpose is creating `Cursor` objects, and Controle de transações.

Ver também

- How to use connection shortcut methods
- How to use the connection context manager

Alterado na versão 3.13: A `ResourceWarning` is emitted if `close()` is not called before a `Connection` object is deleted.

An SQLite database connection has the following attributes and methods:

cursor(*factory=Cursor*)

Create and return a `Cursor` object. The cursor method accepts a single optional parameter *factory*. If supplied, this must be a callable

returning an instance of **`Cursor`** or its subclasses.

blobopen(*table*, *column*, *rowid*, *, *, *readonly=False*, *name='main'*)

Open a **`Blob`** handle to an existing BLOB.

Parâmetros::

- **table** (*str*) – The name of the table where the blob is located.
- **column** (*str*) – The name of the column where the blob is located.
- **rowid** (*int*) – The row id where the blob is located.
- **readonly** (*bool*) – Set to `True` if the blob should be opened without write permissions. Defaults to `False`.
- **name** (*str*) – The name of the database where the blob is located. Defaults to `"main"`.

Levanta:: **OperationalError** – When trying to open a blob in a `WITHOUT ROWID` table.

Tipo de retorno:: Blob

Nota

The blob size cannot be changed using the **`Blob`** class. Use the SQL function `zeroblob` to create a blob with a fixed size.

Adicionado na versão 3.11.

commit()

Commit any pending transaction to the database. If **`autocommit`** is `True`, or there is no open transaction, this method does nothing. If **`autocommit`** is `False`, a new transaction is implicitly opened if a pending transaction was committed by this method.

rollback()

Roll back to the start of any pending transaction. If **`autocommit`** is `True`, or there is no open transaction, this method does nothing. If **`autocommit`** is `False`, a new transaction is implicitly opened if a pending transaction was rolled back by this method.

close()

Close the database connection. If `autocommit` is `False`, any pending transaction is implicitly rolled back. If `autocommit` is `True` or `LEGACY_TRANSACTION_CONTROL`, no implicit transaction control is executed. Make sure to `commit()` before closing to avoid losing pending changes.

execute(*sql*, *parameters=()*, *,*)

Create a new `Cursor` object and call `execute()` on it with the given *sql* and *parameters*. Return the new cursor object.

executemany(*sql*, *parameters*, *,*)

Create a new `Cursor` object and call `executemany()` on it with the given *sql* and *parameters*. Return the new cursor object.

executescript(*sql_script*, *,*)

Create a new `Cursor` object and call `executescript()` on it with the given *sql_script*. Return the new cursor object.

create_function(*name*, *narg*, *func*, *,*, ***, *deterministic=False*)

Create or remove a user-defined SQL function.

Parâmetros::

- **name** (*str*) – O nome da função SQL.
- **narg** (*int*) – The number of arguments the SQI function can accept. If `-1`, it may take any number of arguments.
- **func** (callback | None) – A callable that is called when the SQL function is invoked. The callable must return a type natively supported by SQLite. Set to `None` to remove an existing SQL function.
- **deterministic** (*bool*) – If `True`, the created SQ function is marked as deterministic [https://sqlite.org/deterministic.html], which allows SQLite to perform additional optimizations.

Alterado na versão 3.8: Added the *deterministic* parameter.

Alterado na versão 3.15: The first three parameters are now positional-only.

Exemplo:

```
>>> import hashlib
>>> def md5sum(t):
...     return hashlib.md5(t).hexdigest()
>>> con = sqlite3.connect(":memory:")
>>> con.create_function("md5", 1, md5sum)
>>> for row in con.execute("SELECT md5(?)", (b"foo",)):
...     print(row)
('acbd18db4cc2f85cedef654fccc4a4d8',)
>>> con.close()
```

create_aggregate(*name*, *n_arg*, *aggregate_class*, /)

Create or remove a user-defined SQL aggregate function.

Parâmetros::

- **name** (*str*) – The name of the SQL aggregate function.
- **n_arg** (*int*) – The number of arguments the SQL aggregate function can accept. If `-1`, it may take any number of arguments.
- **aggregate_class** (class | None) – Uma classe deve implementar os seguintes métodos:
  - `step()`: Add a row to the aggregate.
  - `finalize()`: Return the final result of the aggregate as a type natively supported by SQLite.

  The number of arguments that the `step()` method must accept is controlled by *n_arg*.
  Set to `None` to remove an existing SQL aggregate function.

Alterado na versão 3.15: All three parameters are now positional-only.

Exemplo:

```
class MySum:
    def __init__(self):
        self.count = 0

    def step(self, value):
        self.count += value

    def finalize(self):
        return self.count

con = sqlite3.connect(":memory:")
con.create_aggregate("mysum", 1, MySum)
cur = con.execute("CREATE TABLE test(i)")
cur.execute("INSERT INTO test(i) VALUES(1)")
cur.execute("INSERT INTO test(i) VALUES(2)")
cur.execute("SELECT mysum(i) FROM test")
print(cur.fetchone()[0])

con.close()
```

create_window_function(*name*, *num_params*, *aggregate_class*, /)

Create or remove a user-defined aggregate window function.

Parâmetros::

- **name** (*str*) – The name of the SQL aggregate window function to create or remove.
- **num_params** (*int*) – The number of argument the SQL aggregate window function can accept. If `-1`, it may take any number of arguments.
- **aggregate_class** (class | None) – Uma classe que deve implementar os seguinte métodos:
  - `step()`: Add a row to the current window.
  - `value()`: Return the current value of the aggregate.
  - `inverse()`: Remove a row from the current window.
  - `finalize()`: Return the final result of the aggregate as a type natively supported by SQLite.

  The number of arguments that the `step()` and `value()` methods must accept is controlled by *num_params*.

  Set to `None` to remove an existing SQL aggregate window function.

Levanta:: **NotSupportedError** – If used with a version of SQLite older than 3.25.0, which does not support aggregate window functions.

Adicionado na versão 3.11.

Exemplo:

```
# Example taken from https://www.sqlite.org/windowfunctions
class WindowSumInt:
    def __init__(self):
        self.count = 0

    def step(self, value):
        """Add a row to the current window."""
        self.count += value

    def value(self):
```

```
        """Return the current value of the aggregate."""
        return self.count

    def inverse(self, value):
        """Remove a row from the current window."""
        self.count -= value

    def finalize(self):
        """Return the final value of the aggregate.

        Any clean-up actions should be placed here.
        """
        return self.count


con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE test(x, y)")
values = [
    ("a", 4),
    ("b", 5),
    ("c", 3),
    ("d", 8),
    ("e", 1),
]
cur.executemany("INSERT INTO test VALUES(?, ?)", values)
con.create_window_function("sumint", 1, WindowSumInt)
cur.execute("""
    SELECT x, sumint(y) OVER (
        ORDER BY x ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING
    ) AS sum_y
    FROM test ORDER BY x
""")
print(cur.fetchall())
con.close()
```

create_collation(*name*, *callable*, /)

Create a collation named *name* using the collating function *callable*. *callable* is passed two **string** arguments, and it should return an **integer**:

- `1` if the first is ordered higher than the second
- `-1` if the first is ordered lower than the second

- `0` if they are ordered equal

The following example shows a reverse sorting collation:

```
def collate_reverse(string1, string2):
    if string1 == string2:
        return 0
    elif string1 < string2:
        return 1
    else:
        return -1

con = sqlite3.connect(":memory:")
con.create_collation("reverse", collate_reverse)

cur = con.execute("CREATE TABLE test(x)")
cur.executemany("INSERT INTO test(x) VALUES(?)", [("a",), (
cur.execute("SELECT x FROM test ORDER BY x COLLATE reverse"
for row in cur:
    print(row)
con.close()
```

Remove a collation function by setting *callable* to `None`.

Alterado na versão 3.11: The collation name can contain any Unicode character. Earlier, only ASCII characters were allowed.

interrupt()

Call this method from a different thread to abort any queries that might be executing on the connection. Aborted queries will raise an **`OperationalError`**.

set_authorizer(*authorizer_callback*, /)

Register callable *authorizer_callback* to be invoked for each attempt to access a column of a table in the database. The callback should return one of **`SQLITE_OK`**, **`SQLITE_DENY`**, or **`SQLITE_IGNORE`** to signal how access to the column should be handled by the underlying SQLite library.

The first argument to the callback signifies what kind of operation is to be authorized. The second and third argument will be arguments or `None` depending on the first argument. The 4th argument is the name of the database (“main”, “temp”, etc.) if applicable. The 5th argument is the name of the inner-most trigger or view that is responsible for the access attempt or `None` if this access attempt is directly from input SQL code.

Please consult the SQLite documentation about the possible values for the first argument and the meaning of the second and third argument depending on the first one. All necessary constants are available in the **`sqlite3`** module.

Passing `None` as *authorizer_callback* will disable the authorizer.

Alterado na versão 3.11: Added support for disabling the authorizer using `None`.

Alterado na versão 3.15: The only parameter is now positional-only.

set_progress_handler(*progress_handler*, *,* *n*)

Register callable *progress_handler* to be invoked for every *n* instructions of the SQLite virtual machine. This is useful if you want to get called from SQLite during long-running operations, for example to update a GUI.

If you want to clear any previously installed progress handler, call the method with `None` for *progress_handler*.

Returning a non-zero value from the handler function will terminate the currently executing query and cause it to raise a **`DatabaseError`** exception.

Alterado na versão 3.15: O primeiro parâmetro agora é somente-posicional.

set_trace_callback(*trace_callback*, *,*)

Register callable *trace_callback* to be invoked for each SQL statement that is actually executed by the SQLite backend.

The only argument passed to the callback is the statement (as `str`) that is being executed. The return value of the callback is ignored. Note that the backend does not only run statements passed to the `Cursor.execute()` methods. Other sources include the transaction management of the `sqlite3` module and the execution of triggers defined in the current database.

Passing `None` as *trace_callback* will disable the trace callback.

Nota

Exceptions raised in the trace callback are not propagated. As a development and debugging aid, use `enable_callback_tracebacks()` to enable printing tracebacks from exceptions raised in the trace callback.

Adicionado na versão 3.3.

Alterado na versão 3.15: O primeiro parâmetro agora é somente-posicional.

enable_load_extension(*enabled*, /)

Enable the SQLite engine to load SQLite extensions from shared libraries if *enabled* is `True`; else, disallow loading SQLite extensions. SQLite extensions can define new functions, aggregates or whole new virtual table implementations. One well-known extension is the fulltext-search extension distributed with SQLite.

Nota

The `sqlite3` module is not built with loadable extension support by default, because some platforms (notably macOS) have SQLite libraries which are compiled without this feature. To get loadable extension support, you must pass the `--enable-loadable-sqlite-extensions` option to **configure**.

Levanta um evento de auditoria `sqlite3.enable_load_extension` com os argumentos `connection`, `enabled`.

Adicionado na versão 3.2.

Alterado na versão 3.10: Added the `sqlite3.enable_load_extension` auditing event.

```
con.enable_load_extension(True)

# Load the fulltext search extension
con.execute("select load_extension('./fts3.so')")

# alternatively you can load the extension using an API cal
# con.load_extension("./fts3.so")

# disable extension loading again
con.enable_load_extension(False)

# example from SQLite wiki
con.execute("CREATE VIRTUAL TABLE recipe USING fts3(name, i
con.executescript("""
    INSERT INTO recipe (name, ingredients) VALUES('broccoli
    INSERT INTO recipe (name, ingredients) VALUES('pumpkin
    INSERT INTO recipe (name, ingredients) VALUES('broccoli
    INSERT INTO recipe (name, ingredients) VALUES('pumpkin
    """)
for row in con.execute("SELECT rowid, name, ingredients FRC
    print(row)
```

load_extension(*path*, /, *, *entrypoint=None*)

Load an SQLite extension from a shared library. Enable extension loading with `enable_load_extension()` before calling this method.

Parâmetros::

- **path** (*str*) – The path to the SQLite extension.
- **entrypoint** (*str* | *None*) – Entry point name. If `None` (the default), SQLite will come up with a entry point name of its own; see the SQLite docs Loading an Extension [https://www.sqlite.org/loadext.html#loading_an_extension] for details.

Levanta um evento de auditoria `sqlite3.load_extension` com os argumentos `connection`, `path`.

Adicionado na versão 3.2.

Alterado na versão 3.10: Added the `sqlite3.load_extension` auditing event.

Alterado na versão 3.12: Added the *entrypoint* parameter.

iterdump(*, *filter=None*)

Return an iterator to dump the database as SQL source code. Useful when saving an in-memory database for later restoration. Similar to the `.dump` command in the **sqlite3** shell.

Parâmetros::

filter (*str* | *None*) – An optional `LIKE` pattern for database objects to dump, e.g. `prefix_%`. If `None` (the default), all database objects will be included.

Exemplo:

```
# Convert file example.db to SQL dump file dump.sql
con = sqlite3.connect('example.db')
with open('dump.sql', 'w') as f:
    for line in con.iterdump():
        f.write('%s\n' % line)
con.close()
```

Ver também

How to handle non-UTF-8 text encodings

Alterado na versão 3.13: Adicionado o parâmetro *filter*.

backup(*target*, ***, *pages=-1*, *progress=None*, *name='main'*, *sleep=0.250*)

Create a backup of an SQLite database.

Works even if the database is being accessed by other clients or concurrently by the same connection.

Parâmetros::

- **target** (*Connection*) – The database connectio to save the backup to.
- **pages** (*int*) – The number of pages to copy at a time. If equal to or less than `0`, the entire database is copied in a single step. Defaults to `-1`.
- **progress** (callback | None) – If set to a callabl it is invoked with three integer arguments for every backup iteration: the *status* of the last iteration, the *remaining* number of pages still t be copied, and the *total* number of pages. Defaults to `None`.
- **name** (*str*) – The name of the database to back up. Either `"main"` (the default) for the main database, `"temp"` for the temporary database, c the name of a custom database as attached using the `ATTACH DATABASE` SQL statement.
- **sleep** (*float*) – The number of seconds to sleep between successive attempts to back up remaining pages.

Example 1, copy an existing database into another:

```
def progress(status, remaining, total):
    print(f'Copied {total-remaining} of {total} pages...')

src = sqlite3.connect('example.db')
dst = sqlite3.connect('backup.db')
with dst:
    src.backup(dst, pages=1, progress=progress)
```

```
dst.close()
src.close()
```

Example 2, copy an existing database into a transient copy:

```
src = sqlite3.connect('example.db')
dst = sqlite3.connect(':memory:')
src.backup(dst)
dst.close()
src.close()
```

Adicionado na versão 3.7.

Ver também

How to handle non-UTF-8 text encodings

getlimit(*category*, /)

Get a connection runtime limit.

Parâmetros::	**category** (*int*) – The SQLite limit category [https://www.sqlite.org/c3ref/c_limit_attached.html] to be queried.
Tipo de retorno::	int
Levanta::	**ProgrammingError** – If *category* is not recognised by the underlying SQLite library.

Example, query the maximum length of an SQL statement for **`Connection`** `con` (the default is 1000000000):

```
>>> con.getlimit(sqlite3.SQLITE_LIMIT_SQL_LENGTH)
1000000000
```

Adicionado na versão 3.11.

setlimit(*category*, *limit*, /)

Set a connection runtime limit. Attempts to increase a limit above its hard upper bound are silently truncated to the hard upper bound. Regardless of whether or not the limit was changed, the prior value of the limit is returned.

Parâmetros::
- **category** (*int*) – The SQLite limit category [https://www.sqlite.org/c3ref/c_limit_attached.html] to be set.
- **limit** (*int*) – The value of the new limit. If negative, the current limit is unchanged.

Tipo de retorno:: int

Levanta:: **ProgrammingError** – If *category* is not recognised by the underlying SQLite library.

Example, limit the number of attached databases to 1 for **`Connection`** `con` (the default limit is 10):

```
>>> con.setlimit(sqlite3.SQLITE_LIMIT_ATTACHED, 1)
10
>>> con.getlimit(sqlite3.SQLITE_LIMIT_ATTACHED)
1
```

Adicionado na versão 3.11.

getconfig(*op*, *[]*)

Query a boolean connection configuration option.

Parâmetros:: **op** (*int*) – A SQLITE_DBCONFIG code.

Tipo de retorno:: bool

Adicionado na versão 3.12.

setconfig(*op*, *enable=True*, *[]*)

Set a boolean connection configuration option.

Parâmetros::
- **op** (*int*) – A SQLITE_DBCONFIG code.
- **enable** (*bool*) – `True` if the configuration optic should be enabled (default); `False` if it should be disabled.

Adicionado na versão 3.12.

serialize(***, *name='main'*)

Serialize a database into a **`bytes`** object. For an ordinary on-disk database file, the serialization is just a copy of the disk file. For an in-

memory database or a “temp” database, the serialization is the same sequence of bytes which would be written to disk if that database were backed up to disk.

Parâmetros:: **name** (*str*) – The database name to be serialized. Defaults to `"main"`.

Tipo de retorno:: bytes

Nota

This method is only available if the underlying SQLite library has the serialize API.

Adicionado na versão 3.11.

deserialize(*data*, */*, ***, *name='main'*)

Deserialize a **`serialized`** database into a **`Connection`**. This method causes the database connection to disconnect from database *name*, and reopen *name* as an in-memory database based on the serialization contained in *data*.

Parâmetros::

- **data** (*bytes*) – A serialized database.
- **name** (*str*) – The database name to deserialize into. Defaults to `"main"`.

Levanta::

- **OperationalError** – If the database connectio is currently involved in a read transaction or a backup operation.
- **DatabaseError** – If *data* does not contain a valid SQLite database.
- **OverflowError** – If **`len(data)`** is larger than `2**63 - 1`.

Nota

This method is only available if the underlying SQLite library has the deserialize API.

Adicionado na versão 3.11.

autocommit

This attribute controls **PEP 249** [https://peps.python.org/pep-0249/]-compliant transaction behaviour. **`autocommit`** has three allowed values:

- `False`: Select **PEP 249** [https://peps.python.org/pep-0249/]-compliant transaction behaviour, implying that **`sqlite3`** ensures a transaction is always open. Use **`commit()`** and **`rollback()`** to close transactions.

  This is the recommended value of **`autocommit`**.

- `True`: Use SQLite's autocommit mode [https://www.sqlite.org/lang_transaction.html#implicit_versus_explicit_transactions]. **`commit()`** and **`rollback()`** have no effect in this mode.

- **`LEGACY_TRANSACTION_CONTROL`**: Pre-Python 3.12 (non-**PEP 249** [https://peps.python.org/pep-0249/]-compliant) transaction control. See **`isolation_level`** for more details.

  This is currently the default value of **`autocommit`**.

Changing **`autocommit`** to `False` will open a new transaction, and changing it to `True` will commit any pending transaction.

See Controle de transações através do atributo autocommit for more details.

Nota

The **`isolation_level`** attribute has no effect unless **`autocommit`** is **`LEGACY_TRANSACTION_CONTROL`**.

Adicionado na versão 3.12.

in_transaction

This read-only attribute corresponds to the low-level SQLite autocommit mode [https://www.sqlite.org/lang_transaction.html#implicit_versus_explicit_transactions].

`True` if a transaction is active (there are uncommitted changes), `False` otherwise.

Adicionado na versão 3.2.

isolation_level

Controls the legacy transaction handling mode of **`sqlite3`**. If set to `None`, transactions are never implicitly opened. If set to one of `"DEFERRED"`, `"IMMEDIATE"`, or `"EXCLUSIVE"`, corresponding to the underlying SQLite transaction behaviour [https://www.sqlite.org/lang_transaction.html#deferred_immediate_and_exclusive_transactions], implicit transaction management is performed.

If not overridden by the *isolation_level* parameter of **`connect()`**, the default is `""`, which is an alias for `"DEFERRED"`.

Nota

Using **`autocommit`** to control transaction handling is recommended over using **`isolation_level`**. **`isolation_level`** has no effect unless **`autocommit`** is set to **`LEGACY_TRANSACTION_CONTROL`** (the default).

row_factory

The initial **`row_factory`** for **`Cursor`** objects created from this connection. Assigning to this attribute does not affect the **`row_factory`** of existing cursors belonging to this connection, only new ones. Is `None` by default, meaning each row is returned as a **`tuple`**.

Consulte How to create and use row factories para mais detalhes.

Alterado na versão 3.15: Deleting the `row_factory` attribute is no longer allowed.

text_factory

A callable that accepts a **`bytes`** parameter and returns a text representation of it. The callable is invoked for SQLite values with the `TEXT` data type. By default, this attribute is set to **`str`**.

Consulte How to handle non-UTF-8 text encodings para mais detalhes.

Alterado na versão 3.15: Deleting the `text_factory` attribute is no longer allowed.

total_changes

Return the total number of database rows that have been modified, inserted, or deleted since the database connection was opened.

Cursor objects

A `Cursor` object represents a database cursor [https://en.wikipedia.org/wiki/Cursor_(databases)] which is used to execute SQL statements, and manage the context of a fetch operation. Cursors are created using **`Connection.cursor()`**, or by using any of the connection shortcut methods.

Cursor objects are iterators, meaning that if you **`execute()`** a `SELECT` query, you can simply iterate over the cursor to fetch the resulting rows:

```
for row in cur.execute("SELECT t FROM data"):
    print(row)
```

class sqlite3.Cursor

A **`Cursor`** instance has the following attributes and methods.

execute(*sql*, *parameters=()*, */*)

Execute a single SQL statement, optionally binding Python values using placeholders.

Parâmetros::

- **sql** (*str*) – A single SQL statement.
- **parameters** (**`dict`** | sequence) – Python values to bind to placeholders in *sql*. A **`dict`** if named placeholders are used. A sequence if unnamed placeholders are used. See How to use placeholders to bind values in SQL queries.

Levanta::

ProgrammingError – When *sql* contains more than one SQL statement. When named placeholders are used and *parameters* is a sequence instead of a `dict`.

If `autocommit` is `LEGACY_TRANSACTION_CONTROL`, `isolation_level` is not `None`, *sql* is an `INSERT`, `UPDATE`, `DELETE`, or `REPLACE` statement, and there is no open transaction, a transaction is implicitly opened before executing *sql*.

Alterado na versão 3.14: `ProgrammingError` is emitted if named placeholders are used and *parameters* is a sequence instead of a `dict`.

Use `executescript()` to execute multiple SQL statements.

executemany(*sql*, *parameters*, /)

For every item in *parameters*, repeatedly execute the parameterized DML SQL statement *sql*.

Uses the same implicit transaction handling as `execute()`.

Parâmetros::

- **sql** (*str*) – A single SQL DML statement.
- **parameters** (iterable) – An iterable of parameters to bind with the placeholders in *sq*
  See How to use placeholders to bind values in SQL queries.

Levanta:: **ProgrammingError** – When *sql* contains more than one SQL statement or is not a DML statement, When named placeholders are used and the items in *parameters* are sequences instead of `dict`s.

Exemplo:

```
rows = [
    ("row1",),
    ("row2",),
]
# cur is an sqlite3.Cursor object
cur.executemany("INSERT INTO data VALUES(?)", rows)
```

Nota

Any resulting rows are discarded, including DML statements with RETURNING clauses [https://www.sqlite.org/lang_returning.html].

Alterado na versão 3.14: **`ProgrammingError`** is emitted if named placeholders are used and the items in *parameters* are sequences instead of **`dict`**s.

executescript(*sql_script*, /)

Execute the SQL statements in *sql_script*. If the **`autocommit`** is **`LEGACY_TRANSACTION_CONTROL`** and there is a pending transaction, an implicit `COMMIT` statement is executed first. No other implicit transaction control is performed; any transaction control must be added to *sql_script*.

sql_script must be a **`string`**.

Exemplo:

```
# cur is an sqlite3.Cursor object
cur.executescript("""
    BEGIN;
    CREATE TABLE person(firstname, lastname, age);
    CREATE TABLE book(title, author, published);
    CREATE TABLE publisher(name, address);
    COMMIT;
""")
```

fetchone()

If **`row_factory`** is `None`, return the next row query result set as a **`tuple`**. Else, pass it to the row factory and return its result. Return `None` if no more data is available.

fetchmany(*size=cursor.arraysize*)

Return the next set of rows of a query result as a **`list`**. Return an empty list if no more rows are available.

The number of rows to fetch per call is specified by the *size* parameter. If *size* is not given, **arraysize** determines the number of rows to be fetched. If fewer than *size* rows are available, as many rows as are available are returned.

Note there are performance considerations involved with the *size* parameter. For optimal performance, it is usually best to use the arraysize attribute. If the *size* parameter is used, then it is best for it to retain the same value from one **fetchmany()** call to the next.

Alterado na versão 3.15: Negative *size* values are rejected by raising **ValueError**.

fetchall()

Return all (remaining) rows of a query result as a **list**. Return an empty list if no rows are available. Note that the **arraysize** attribute can affect the performance of this operation.

close()

Close the cursor now (rather than whenever `__del__` is called).

The cursor will be unusable from this point forward; a **ProgrammingError** exception will be raised if any operation is attempted with the cursor.

setinputsizes(*sizes*, /)

Required by the DB-API. Does nothing in **sqlite3**.

setoutputsize(*size*, *column=None*, /)

Required by the DB-API. Does nothing in **sqlite3**.

arraysize

Read/write attribute that controls the number of rows returned by **fetchmany()**. The default value is 1 which means a single row would be fetched per call.

Alterado na versão 3.15: Negative values are rejected by raising **ValueError**.

connection

Read-only attribute that provides the SQLite database **Connection** belonging to the cursor. A **Cursor** object created by calling **con.cursor()** will have a **connection** attribute that refers to *con*:

```
>>> con = sqlite3.connect(":memory:")
>>> cur = con.cursor()
>>> cur.connection == con
True
>>> con.close()
```

description

Read-only attribute that provides the column names of the last query. To remain compatible with the Python DB API, it returns a 7-tuple for each column where the last six items of each tuple are `None`.

It is set for `SELECT` statements without any matching rows as well.

lastrowid

Read-only attribute that provides the row id of the last inserted row. It is only updated after successful `INSERT` or `REPLACE` statements using the **execute()** method. For other statements, after **executemany()** or **executescript()**, or if the insertion failed, the value of `lastrowid` is left unchanged. The initial value of `lastrowid` is `None`.

Nota

Inserts into `WITHOUT ROWID` tables are not recorded.

Alterado na versão 3.6: Added support for the `REPLACE` statement.

rowcount

Read-only attribute that provides the number of modified rows for `INSERT`, `UPDATE`, `DELETE`, and `REPLACE` statements; is `-1` for other statements, including CTE queries. It is only updated by the **execute()** and **executemany()** methods, after the statement has run to completion. This means that any resulting rows must be fetched in order for **rowcount** to be updated.

row_factory

Control how a row fetched from this **`Cursor`** is represented. If `None`, a row is represented as a **`tuple`**. Can be set to the included **`sqlite3.Row`**; or a callable that accepts two arguments, a **`Cursor`** object and the **`tuple`** of row values, and returns a custom object representing an SQLite row.

Defaults to what **`Connection.row_factory`** was set to when the **`Cursor`** was created. Assigning to this attribute does not affect **`Connection.row_factory`** of the parent connection.

Consulte How to create and use row factories para mais detalhes.

Alterado na versão 3.15: Deleting the `row_factory` attribute is no longer allowed.

Row objects

class sqlite3.Row

A **`Row`** instance serves as a highly optimized **`row_factory`** for **`Connection`** objects. It supports iteration, equality testing, **`len()`**, and mapping access by column name and index.

Two **`Row`** objects compare equal if they have identical column names and values.

Consulte How to create and use row factories para mais detalhes.

keys()

Return a **`list`** of column names as **`strings`**. Immediately after a query, it is the first member of each tuple in **`Cursor.description`**.

Alterado na versão 3.5: Added support of slicing.

Blob objects

class sqlite3.Blob

Adicionado na versão 3.11.

A **`Blob`** instance is a file-like object that can read and write data in an SQLite BLOB. Call **`len(blob)`** to get the size (number of bytes) of the blob. Use indices and slices for direct access to the blob data.

Use the **`Blob`** as a context manager to ensure that the blob handle is closed after use.

```
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE test(blob_col blob)")
con.execute("INSERT INTO test(blob_col) VALUES(zeroblob(13))")

# Write to our blob, using two write operations:
with con.blobopen("test", "blob_col", 1) as blob:
    blob.write(b"hello, ")
    blob.write(b"world.")
    # Modify the first and last bytes of our blob
    blob[0] = ord("H")
    blob[-1] = ord("!")

# Read the contents of our blob
with con.blobopen("test", "blob_col", 1) as blob:
    greeting = blob.read()

print(greeting)  # outputs "b'Hello, world!'"
con.close()
```

close()

Close the blob.

The blob will be unusable from this point onward. An **`Error`** (or subclass) exception will be raised if any further operation is attempted with the blob.

read(*length=-1*, /)

Read *length* bytes of data from the blob at the current offset position. If the end of the blob is reached, the data up to EOF will be returned. When *length* is not specified, or is negative, **`read()`** will read until the end of the blob.

write(*data*, /)

Write *data* to the blob at the current offset. This function cannot change the blob length. Writing beyond the end of the blob will raise **`ValueError`**.

tell()

Return the current access position of the blob.

seek(*offset*, *origin=os.SEEK_SET*, /)

Set the current access position of the blob to *offset*. The *origin* argument defaults to **`os.SEEK_SET`** (absolute blob positioning). Other values for *origin* are **`os.SEEK_CUR`** (seek relative to the current position) and **`os.SEEK_END`** (seek relative to the blob's end).

PrepareProtocol objects

class sqlite3.PrepareProtocol

The PrepareProtocol type's single purpose is to act as a **PEP 246** [https://peps.python.org/pep-0246/] style adaption protocol for objects that can adapt themselves to native SQLite types.

Exceções

The exception hierarchy is defined by the DB-API 2.0 (**PEP 249** [https://peps.python.org/pep-0249/]).

exception sqlite3.Warning

This exception is not currently raised by the **`sqlite3`** module, but may be raised by applications using **`sqlite3`**, for example if a user-defined function truncates data while inserting. `Warning` is a subclass of **`Exception`**.

exception sqlite3.Error

The base class of the other exceptions in this module. Use this to catch all errors with one single **`except`** statement. `Error` is a subclass of **`Exception`**.

If the exception originated from within the SQLite library, the following two attributes are added to the exception:

sqlite_errorcode

The numeric error code from the SQLite API [https://sqlite.org/rescode.html]

Adicionado na versão 3.11.

sqlite_errorname

The symbolic name of the numeric error code from the SQLite API [https://sqlite.org/rescode.html]

Adicionado na versão 3.11.

exception sqlite3.InterfaceError

Exception raised for misuse of the low-level SQLite C API. In other words, if this exception is raised, it probably indicates a bug in the **`sqlite3`** module. `InterfaceError` is a subclass of **`Error`**.

exception sqlite3.DatabaseError

Exception raised for errors that are related to the database. This serves as the base exception for several types of database errors. It is only raised implicitly through the specialised subclasses. `DatabaseError` is a subclass of **`Error`**.

exception sqlite3.DataError

Exception raised for errors caused by problems with the processed data, like numeric values out of range, and strings which are too long. `DataError` is a subclass of **`DatabaseError`**.

exception sqlite3.OperationalError

Exception raised for errors that are related to the database's operation, and not necessarily under the control of the programmer. For example, the database path is not found, or a transaction could not be processed. `OperationalError` is a subclass of **`DatabaseError`**.

exception sqlite3.IntegrityError

Exception raised when the relational integrity of the database is affected, e.g. a foreign key check fails. It is a subclass of `DatabaseError`.

exception sqlite3.InternalError

Exception raised when SQLite encounters an internal error. If this is raised, it may indicate that there is a problem with the runtime SQLite library. `InternalError` is a subclass of `DatabaseError`.

exception sqlite3.ProgrammingError

Exception raised for **`sqlite3`** API programming errors, for example supplying the wrong number of bindings to a query, or trying to operate on a closed `Connection`. `ProgrammingError` is a subclass of `DatabaseError`.

exception sqlite3.NotSupportedError

Exception raised in case a method or database API is not supported by the underlying SQLite library. For example, setting *deterministic* to `True` in `create_function()`, if the underlying SQLite library does not support deterministic functions. `NotSupportedError` is a subclass of `DatabaseError`.

SQLite and Python types

SQLite natively supports the following types: `NULL`, `INTEGER`, `REAL`, `TEXT`, `BLOB`.

The following Python types can thus be sent to SQLite without any problem:

Tipo em Python	Tipo em SQLite
`None`	`NULL`
`int`	`INTEGER`

Tipo em Python	Tipo em SQLite
`float`	`REAL`
`str`	`TEXT`
`bytes`	`BLOB`

This is how SQLite types are converted to Python types by default:

Tipo em SQLite	Tipo em Python
`NULL`	`None`
`INTEGER`	`int`
`REAL`	`float`
`TEXT`	depends on `text_factory`, `str` by default
`BLOB`	`bytes`

The type system of the **`sqlite3`** module is extensible in two ways: you can store additional Python types in an SQLite database via object adapters, and you can let the **`sqlite3`** module convert SQLite types to Python types via converters.

Default adapters and converters (deprecated)

Nota

The default adapters and converters are deprecated as of Python 3.12. Instead, use the Adapter and converter recipes and tailor them to your needs.

The deprecated default adapters and converters consist of:

- An adapter for `datetime.date` objects to `strings` in ISO 8601 [https://en.wikipedia.org/wiki/ISO_8601] format.
- An adapter for `datetime.datetime` objects to strings in ISO 8601 format.
- A converter for declared "date" types to `datetime.date` objects.
- A converter for declared "timestamp" types to `datetime.datetime` objects. Fractional parts will be truncated to 6 digits (microsecond precision).

Nota

The default "timestamp" converter ignores UTC offsets in the database and always returns a naive `datetime.datetime` object. To preserve UTC offsets in timestamps, either leave converters disabled, or register an offset-aware converter with `register_converter()`.

Descontinuado desde a versão 3.12.

Interface de linha de comando

The `sqlite3` module can be invoked as a script, using the interpreter's `-m` switch, in order to provide a simple SQLite shell. The argument signature is as follows:

```
python -m sqlite3 [-h] [-v] [filename] [sql]
```

Type `.quit` or CTRL-D to exit the shell.

-h, --help
    Print CLI help.

-v, --version
    Print underlying SQLite library version.

Adicionado na versão 3.12.

Guias de como fazer

How to use placeholders to bind values in SQL queries

SQL operations usually need to use values from Python variables. However, beware of using Python's string operations to assemble queries, as they are vulnerable to SQL injection attacks [https://en.wikipedia.org/wiki/SQL_injection]. For example, an attacker can simply close the single quote and inject `OR TRUE` to select all rows:

```
>>> # Never do this -- insecure!
>>> symbol = input()
' OR TRUE; --
>>> sql = "SELECT * FROM stocks WHERE symbol = '%s'" % symbol
>>> print(sql)
SELECT * FROM stocks WHERE symbol = '' OR TRUE; --'
>>> cur.execute(sql)
```

Instead, use the DB-API's parameter substitution. To insert a variable into a query string, use a placeholder in the string, and substitute the actual values into the query by providing them as a **`tuple`** of values to the second argument of the cursor's **`execute()`** method.

An SQL statement may use one of two kinds of placeholders: question marks (qmark style) or named placeholders (named style). For the qmark style, *parameters* must be a sequence whose length must match the number of placeholders, or a **`ProgrammingError`** is raised. For the named style, *parameters* must be an instance of a **`dict`** (or a subclass), which must contain

keys for all named parameters; any extra items are ignored. Here's an example of both styles:

```
con = sqlite3.connect(":memory:")
cur = con.execute("CREATE TABLE lang(name, first_appeared)")

# This is the named style used with executemany():
data = (
    {"name": "C", "year": 1972},
    {"name": "Fortran", "year": 1957},
    {"name": "Python", "year": 1991},
    {"name": "Go", "year": 2009},
)
cur.executemany("INSERT INTO lang VALUES(:name, :year)", data)

# This is the qmark style used in a SELECT query:
params = (1972,)
cur.execute("SELECT * FROM lang WHERE first_appeared = ?", params
print(cur.fetchall())
con.close()
```

Nota

PEP 249 [https://peps.python.org/pep-0249/] numeric placeholders are *not* supported. If used, they will be interpreted as named placeholders.

How to adapt custom Python types to SQLite values

SQLite supports only a limited set of data types natively. To store custom Python types in SQLite databases, *adapt* them to one of the Python types SQLite natively understands.

There are two ways to adapt Python objects to SQLite types: letting your object adapt itself, or using an *adapter callable*. The latter will take precedence above the former. For a library that exports a custom type, it may make sense to enable that type to adapt itself. As an application developer, it may make more sense to take direct control by registering custom adapter functions.

How to write adaptable objects

Suppose we have a **`Point`** class that represents a pair of coordinates, `x` and `y`, in a Cartesian coordinate system. The coordinate pair will be stored as a text string in the database, using a semicolon to separate the coordinates. This can be implemented by adding a `__conform__(self, protocol)` method which returns the adapted value. The object passed to *protocol* will be of type **`PrepareProtocol`**.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __conform__(self, protocol):
        if protocol is sqlite3.PrepareProtocol:
            return f"{self.x};{self.y}"

con = sqlite3.connect(":memory:")
cur = con.cursor()

cur.execute("SELECT ?", (Point(4.0, -3.2),))
print(cur.fetchone()[0])
con.close()
```

How to register adapter callables

The other possibility is to create a function that converts the Python object to an SQLite-compatible type. This function can then be registered using **`register_adapter()`**.

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

def adapt_point(point):
    return f"{point.x};{point.y}"

sqlite3.register_adapter(Point, adapt_point)

con = sqlite3.connect(":memory:")
cur = con.cursor()
```

```
cur.execute("SELECT ?", (Point(1.0, 2.5),))
print(cur.fetchone()[0])
con.close()
```

How to convert SQLite values to custom Python types

Writing an adapter lets you convert *from* custom Python types *to* SQLite values. To be able to convert *from* SQLite values *to* custom Python types, we use *converters*.

Let's go back to the **`Point`** class. We stored the x and y coordinates separated via semicolons as strings in SQLite.

First, we'll define a converter function that accepts the string as a parameter and constructs a **`Point`** object from it.

> **Nota**
>
> Converter functions are **always** passed a **`bytes`** object, no matter the underlying SQLite data type.

```
def convert_point(s):
    x, y = map(float, s.split(b";"))
    return Point(x, y)
```

We now need to tell **`sqlite3`** when it should convert a given SQLite value. This is done when connecting to a database, using the *detect_types* parameter of **`connect()`**. There are three options:

- Implicit: set *detect_types* to **`PARSE_DECLTYPES`**
- Explicit: set *detect_types* to **`PARSE_COLNAMES`**
- Both: set *detect_types* to `sqlite3.PARSE_DECLTYPES | sqlite3.PARSE_COLNAMES`. Column names take precedence over declared types.

The following example illustrates the implicit and explicit approaches:

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y

    def __repr__(self):
        return f"Point({self.x}, {self.y})"

def adapt_point(point):
    return f"{point.x};{point.y}"

def convert_point(s):
    x, y = list(map(float, s.split(b";")))
    return Point(x, y)

# Register the adapter and converter
sqlite3.register_adapter(Point, adapt_point)
sqlite3.register_converter("point", convert_point)

# 1) Parse using declared types
p = Point(4.0, -3.2)
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_DECL
cur = con.execute("CREATE TABLE test(p point)")

cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute("SELECT p FROM test")
print("with declared types:", cur.fetchone()[0])
cur.close()
con.close()

# 2) Parse using column names
con = sqlite3.connect(":memory:", detect_types=sqlite3.PARSE_COLN
cur = con.execute("CREATE TABLE test(p)")

cur.execute("INSERT INTO test(p) VALUES(?)", (p,))
cur.execute('SELECT p AS "p [point]" FROM test')
print("with column names:", cur.fetchone()[0])
cur.close()
con.close()
```

Adapter and converter recipes

This section shows recipes for common adapters and converters.

```
import datetime as dt
import sqlite3

def adapt_date_iso(val):
    """Adapt datetime.date to ISO 8601 date."""
    return val.isoformat()

def adapt_datetime_iso(val):
    """Adapt datetime.datetime to timezone-naive ISO 8601 date.""
    return val.replace(tzinfo=None).isoformat()

def adapt_datetime_epoch(val):
    """Adapt datetime.datetime to Unix timestamp."""
    return int(val.timestamp())

sqlite3.register_adapter(dt.date, adapt_date_iso)
sqlite3.register_adapter(dt.datetime, adapt_datetime_iso)
sqlite3.register_adapter(dt.datetime, adapt_datetime_epoch)

def convert_date(val):
    """Convert ISO 8601 date to datetime.date object."""
    return dt.date.fromisoformat(val.decode())

def convert_datetime(val):
    """Convert ISO 8601 datetime to datetime.datetime object."""
    return dt.datetime.fromisoformat(val.decode())

def convert_timestamp(val):
    """Convert Unix epoch timestamp to datetime.datetime object."
    return dt.datetime.fromtimestamp(int(val))

sqlite3.register_converter("date", convert_date)
sqlite3.register_converter("datetime", convert_datetime)
sqlite3.register_converter("timestamp", convert_timestamp)
```

How to use connection shortcut methods

Using the `execute()`, `executemany()`, and `executescript()` methods of the `Connection` class, your code can be written more concisely because you don't have to create the (often superfluous) `Cursor` objects explicitly. Instead, the `Cursor` objects are created implicitly and these shortcut methods return

the cursor objects. This way, you can execute a `SELECT` statement and iterate over it directly using only a single call on the **`Connection`** object.

```
# Create and fill the table.
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE lang(name, first_appeared)")
data = [
    ("C++", 1985),
    ("Objective-C", 1984),
]
con.executemany("INSERT INTO lang(name, first_appeared) VALUES(?,

# Print the table contents
for row in con.execute("SELECT name, first_appeared FROM lang"):
    print(row)

print("I just deleted", con.execute("DELETE FROM lang").rowcount,

# close() is not a shortcut method and it's not called automatica
# the connection object should be closed manually
con.close()
```

How to use the connection context manager

A **`Connection`** object can be used as a context manager that automatically commits or rolls back open transactions when leaving the body of the context manager. If the body of the **`with`** statement finishes without exceptions, the transaction is committed. If this commit fails, or if the body of the `with` statement raises an uncaught exception, the transaction is rolled back. If **`autocommit`** is `False`, a new transaction is implicitly opened after committing or rolling back.

If there is no open transaction upon leaving the body of the `with` statement, or if **`autocommit`** is `True`, the context manager does nothing.

Nota

The context manager neither implicitly opens a new transaction nor closes the connection. If you need a closing context manager, consider using **`contextlib.closing()`**.

```
con = sqlite3.connect(":memory:")
con.execute("CREATE TABLE lang(id INTEGER PRIMARY KEY, name VARCH

# Successful, con.commit() is called automatically afterwards
with con:
    con.execute("INSERT INTO lang(name) VALUES(?)", ("Python",))

# con.rollback() is called after the with block finishes with an
# the exception is still raised and must be caught
try:
    with con:
        con.execute("INSERT INTO lang(name) VALUES(?)", ("Python"
except sqlite3.IntegrityError:
    print("couldn't add Python twice")

# Connection object used as context manager only commits or rollb
# so the connection object should be closed manually
con.close()
```

How to work with SQLite URIs

Some useful URI tricks include:

- Open a database in read-only mode:

```
>>> con = sqlite3.connect("file:tutorial.db?mode=ro", uri=True)
>>> con.execute("CREATE TABLE readonly(data)")
Traceback (most recent call last):
OperationalError: attempt to write a readonly database
>>> con.close()
```

- Do not implicitly create a new database file if it does not already exist; will raise `OperationalError` if unable to create a new file:

```
>>> con = sqlite3.connect("file:nosuchdb.db?mode=rw", uri=True)
Traceback (most recent call last):
OperationalError: unable to open database file
```

- Create a shared named in-memory database:

```
db = "file:mem1?mode=memory&cache=shared"
con1 = sqlite3.connect(db, uri=True)
con2 = sqlite3.connect(db, uri=True)
with con1:
    con1.execute("CREATE TABLE shared(data)")
    con1.execute("INSERT INTO shared VALUES(28)")
res = con2.execute("SELECT data FROM shared")
assert res.fetchone() == (28,)

con1.close()
con2.close()
```

More information about this feature, including a list of parameters, can be found in the SQLite URI documentation [https://www.sqlite.org/uri.html].

How to create and use row factories

By default, **`sqlite3`** represents each row as a **`tuple`**. If a **`tuple`** does not suit your needs, you can use the **`sqlite3.Row`** class or a custom **`row_factory`**.

While **`row_factory`** exists as an attribute both on the **`Cursor`** and the **`Connection`**, it is recommended to set **`Connection.row_factory`**, so all cursors created from the connection will use the same row factory.

`Row` provides indexed and case-insensitive named access to columns, with minimal memory overhead and performance impact over a **`tuple`**. To use **`Row`** as a row factory, assign it to the **`row_factory`** attribute:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = sqlite3.Row
```

Queries now return **`Row`** objects:

```
>>> res = con.execute("SELECT 'Earth' AS name, 6378 AS radius")
>>> row = res.fetchone()
>>> row.keys()
['name', 'radius']
>>> row[0]         # Access by index.
'Earth'
>>> row["name"]    # Access by name.
'Earth'
```

```
>>> row["RADIUS"]  # Column names are case-insensitive.
6378
>>> con.close()
```

Nota

The `FROM` clause can be omitted in the `SELECT` statement, as in the above example. In such cases, SQLite returns a single row with columns defined by expressions, e.g. literals, with the given aliases `expr AS alias`.

You can create a custom **`row factory`** that returns each row as a **`dict`**, with column names mapped to values:

```
def dict_factory(cursor, row):
    fields = [column[0] for column in cursor.description]
    return {key: value for key, value in zip(fields, row)}
```

Using it, queries now return a **`dict`** instead of a **`tuple`**:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = dict_factory
>>> for row in con.execute("SELECT 1 AS a, 2 AS b"):
...     print(row)
{'a': 1, 'b': 2}
>>> con.close()
```

The following row factory returns a named tuple:

```
from collections import namedtuple

def namedtuple_factory(cursor, row):
    fields = [column[0] for column in cursor.description]
    cls = namedtuple("Row", fields)
    return cls._make(row)
```

`namedtuple_factory()` can be used as follows:

```
>>> con = sqlite3.connect(":memory:")
>>> con.row_factory = namedtuple_factory
>>> cur = con.execute("SELECT 1 AS a, 2 AS b")
>>> row = cur.fetchone()
```

```
>>> row
Row(a=1, b=2)
>>> row[0]  # Indexed access.
1
>>> row.b   # Attribute access.
2
>>> con.close()
```

With some adjustments, the above recipe can be adapted to use a **dataclass**, or any other custom class, instead of a **namedtuple**.

How to handle non-UTF-8 text encodings

By default, **sqlite3** uses **str** to adapt SQLite values with the `TEXT` data type. This works well for UTF-8 encoded text, but it might fail for other encodings and invalid UTF-8. You can use a custom **text_factory** to handle such cases.

Because of SQLite's flexible typing [https://www.sqlite.org/flextypegood.html], it is not uncommon to encounter table columns with the `TEXT` data type containing non-UTF-8 encodings, or even arbitrary data. To demonstrate, let's assume we have a database with ISO-8859-2 (Latin-2) encoded text, for example a table of Czech-English dictionary entries. Assuming we now have a **Connection** instance **con** connected to this database, we can decode the Latin-2 encoded text using this **text_factory**:

```
con.text_factory = lambda data: str(data, encoding="latin2")
```

For invalid UTF-8 or arbitrary data in stored in `TEXT` table columns, you can use the following technique, borrowed from the Unicode:

```
con.text_factory = lambda data: str(data, errors="surrogateescape
```

Nota

The **sqlite3** module API does not support strings containing surrogates.

Ver também

Unicode

Explicação

Controle de transações

`sqlite3` oferece vários métodos para controlar se, quando e como as transações do banco de dados são abertas e fechadas. Controle de transações através do atributo autocommit é recomendado, enquanto Controle de transação através do atributo isolation_level mantém o comportamento pré-Python 3.12.

Controle de transações através do atributo `autocommit`

A forma recomendada de controlar o comportamento da transação é através do atributo **`Connection.autocommit`**, que deve ser preferencialmente definido usando o parâmetro *autocommit* de **`connect()`**.

É sugerido definir *autocommit* como `False`, o que implica controle de transação compatível com a **PEP 249** [https://peps.python.org/pep-0249/]. Isso significa:

- **`sqlite3`** garante que uma transação esteja sempre aberta, então **`connect()`**, **`Connection.commit()`** e **`Connection.rollback()`** abrirão implicitamente uma nova transação (imediatamente após fechando a pendência, para as duas últimas). **`sqlite3`** usa instruções `BEGIN DEFERRED` ao abrir transações.
- Transações devem ser executadas explicitamente usando **`commit()`**.
- Transações devem ser revertidas explicitamente usando **`rollback()`**.
- Uma reversão implícita é executada se o banco de dados estiver em estado **`close()`** com alterações pendentes.

Defina *autocommit* como `True` para ativar o modo autocommit [https://www.sqlite.org/lang_transaction.html#implicit_versus_explicit_transactions] do SQLite. Neste modo, **`Connection.commit()`** e **`Connection.rollback()`** não têm efeito. Observe que o modo autocommit do SQLite é distinto do atributo

`Connection.autocommit` compatível com **PEP 249** [https://peps.python.org/pep-0249/]; use `Connection.in_transaction` para consultar o modo de confirmação automática do SQLite de baixo nível.

Defina *autocommit* como `LEGACY_TRANSACTION_CONTROL` para deixar o comportamento de controle de transação para o atributo `Connection.isolation_level`. Veja Controle de transação através do atributo isolation_level para mais informações.

Controle de transação através do atributo `isolation_level`

Nota

A forma recomendada de controlar transações é através do atributo `autocommit`. Veja Controle de transações através do atributo autocommit.

Se `Connection.autocommit` estiver definido como `LEGACY_TRANSACTION_CONTROL` (o padrão), o comportamento da transação é controlado usando o atributo `Connection.isolation_level`. Caso contrário, **`isolation_level`** não tem efeito.

Se o atributo de conexão `isolation_level` não for `None`, novas transações são abertas implicitamente antes de `execute()` e `executemany()` executa instruções `INSERT`, `UPDATE`, `DELETE` ou `REPLACE`; para outras instruções, nenhuma manipulação de transação implícita é executada. Use os métodos `commit()` e `rollback()` para fazer commit e reverter respectivamente transações pendentes. Você pode escolher o comportamento subjacente de transação do SQLite [https://www.sqlite.org/lang_transaction.html#deferred_immediate_and_exclusive_transactions] – isto é, se e que tipo de instruções `BEGIN` do **`sqlite3`** são executadas implicitamente – através do atributo **`isolation_level`**.

Se `isolation_level` estiver definido como `None`, nenhuma transação será aberta implicitamente. Isso deixa a biblioteca SQLite subjacente no modo autocommit [https://www.sqlite.org/lang_transaction.html#implicit_versus_explicit_transactions], mas também permite que o usuário execute sua própria manipulação de transações usando instruções SQL explícitas. O modo de autocommit da

biblioteca SQLite subjacente pode ser consultado usando o atributo `in transaction`.

O método `executescript()` compromete implicitamente qualquer transação pendente antes da execução do script SQL fornecido, independentemente do valor de `isolation level`.

Alterado na versão 3.6: `sqlite3` costumava fazer commit de forma implícita de uma transação aberta antes das instruções DDL. Este não é mais o caso.

Alterado na versão 3.12: A forma recomendada de controlar transações agora é através do atributo `autocommit`.

Compressão de Dados e Arquivamento

Os módulos descritos neste capítulo oferecem suporte à compressão de dados com os algoritmos zlib, gzip, bzip2, lzma e zstd, e a criação de arquivos no formato ZIP e tar. Veja também Operações de arquivamento fornecido pelo módulo `shutil`.

- O pacote `compression`
- `compression.zstd` — Compactação compatível com o formato Zstandard
  - Exceções
  - Lendo e escrevendo arquivos compactados
  - Compactando e descompactando dados na memória
  - Dicionários de Zstandard
  - Controle avançado de parâmetros
  - Diversos
  - Exemplos
- `zlib` — Compactação compatível com **gzip**
- `gzip` — Suporte para arquivos **gzip**
  - Exemplos de uso
  - Interface de linha de comando
    - Opções de linha de comando
- `bz2` — Suporte para compressão **bzip2**
  - (Des)compressão de arquivos
  - (Des)compressão incremental
  - (De)compressão de uma só vez (one-shot)
  - Exemplos de uso
- `lzma` — Compressão usando o algoritmo LZMA
  - Lendo e escrevendo arquivos comprimidos
  - Comprimindo e descomprimindo dados na memória
  - Diversos
  - Especificando cadeias de filtros personalizadas
  - Constantes

    - Exemplos
- `zipfile` — Trabalha com arquivos ZIP
    - Objetos ZipFile
    - Objetos Path
    - Objetos PyZipFile
    - Objetos ZipInfo
    - Interface de linha de comando
        - Opções de linha de comando
    - Armadilhas de descompressão
        - Do próprio arquivo
        - Limitações do sistema de arquivos
        - Limitações de recursos
        - Interrupção
        - Comportamentos padrão da extração
- `tarfile` — Ler e gravar arquivos do tipo tar
    - Objetos TarFile
    - Objetos TarInfo
    - Filtros de extração
        - Default named filters
        - Filter errors
        - Dicas para verificação adicional
        - Suporte a versões mais antigas do Python
        - Stateful extraction filter example
    - Interface de Linha de Comando
        - Opções de linha de comando
    - Exemplos
        - Reading examples
        - Writing examples
    - Formatos tar suportados
    - Problemas de Unicode

O pacote `compression`

Adicionado na versão 3.14.

O pacote **`compression`** contém os módulos de compressão canônicos, que contêm interfaces para diversos algoritmos de compressão. Alguns desses módulos historicamente estiveram disponíveis como módulos separados; estes continuarão disponíveis com seus nomes originais por motivos de compatibilidade e não serão removidos sem um ciclo de descontinuação. O uso de módulos em **`compression`** é incentivado sempre que possível.

- **`compression.bz2`** – Re-exporta `bz2`
- **`compression.gzip`** – Re-exporta `gzip`
- **`compression.lzma`** – Re-exporta `lzma`
- **`compression.zlib`** – Re-exporta `zlib`
- `compression.zstd` – Invólucro para a biblioteca de compressão Zstandard

`compression.zstd` — Compactação compatível com o formato Zstandard

Adicionado na versão 3.14.

Código-fonte: Lib/compression/zstd/__init__.py [https://github.com/python/cpython/tree/3.15/Lib/compression/zstd/__init__.py]

Este módulo fornece classes e funções para compactar e descompactar dados usando o algoritmo de compactação Zstandard (ou *zstd*). O manual do zstd [https://facebook.github.io/zstd/doc/api_manual_latest.html] descreve o Zstandard como "um algoritmo de compactação rápido e sem perdas, voltado para cenários de compactação em tempo real, no nível de zlib, com taxas de compactação melhores". Também está incluída uma interface de arquivo que oferece suporte à leitura e à gravação do conteúdo de arquivos `.zst` criados pelo utilitário **zstd**, bem como de fluxos compactados zstd brutos.

O módulo **`compression.zstd`** contém:

- A função **`open()`** e a classe **`ZstdFile`** para leitura e escrita de arquivos compactados.
- As classes **`ZstdCompressor`** e **`ZstdDecompressor`** para (des)compactação incremental.
- As funções **`compress()`** e **`decompress()`** para (des)compactação de uma só vez.
- As funções **`train_dict()`** e **`finalize_dict()`** e a classe **`ZstdDict`** para treinar e gerenciar dicionários Zstandard.
- As classes **`CompressionParameter`**, **`DecompressionParameter`** e **`Strategy`** para definir parâmetros avançados de (des)compactação.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Exceções

exception compression.zstd.ZstdError

Essa exceção é levantada quando ocorre um erro durante a compactação ou descompactação ou durante a inicialização do estado (des)compactador.

Lendo e escrevendo arquivos compactados

compression.zstd.open(*file*, *, *mode='rb'*, *, *level=None*, *options=None*, *zstd_dict=None*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo compactado com Zstandard em modo binário ou texto, retornando objeto arquivo.

O argumento *file* pode ser um nome de arquivo (dado como um objeto **`str`**, **`bytes`** ou caminho ou similar), neste caso o arquivo nomeado é aberto , ou pode ser um objeto arquivo existente para leitura ou escrita.

O argumento mode pode ser `'rb'` para leitura (padrão), `'wb'` para sobrescrever, `'ab'` para anexar ou `'xb'` para criação exclusiva. Estes podem ser fornecidos como `'r'`, `'w'`, `'a'` e `'x'`, respectivamente. Você também pode abrir em modo texto com `'rt'`, `'wt'`, `'at'` e `'xt'`, respectivamente.

Durante a leitura, o argumento *options* pode ser um dicionário que fornece parâmetros avançados de descompactação; consulte **`DecompressionParameter`** para obter informações detalhadas sobre os parâmetros suportados. O argumento *zstd_dict* é uma instância de **`ZstdDict`** a ser usada durante a descompactação. Durante a leitura, se o argumento *level* não for None, uma exceção **`TypeError`** será levantada.

When writing, the *options* argument can be a dictionary providing advanced compression parameters; see `CompressionParameter` for detailed information about supported parameters. The *level* argument is the compression level to use when writing compressed data. Only one of *level* or *options* may be non-None. The *zstd_dict* argument is a `ZstdDict` instance to be used during compression.

Em modo binário, esta função equivale ao construtor `ZstdFile`: `ZstdFile(file, mode, ...)`. Nesse caso, os parâmetros *encoding*, *errors* e *newline* não devem ser fornecidos.

Em modo texto, um objeto `ZstdFile` é criado e encapsulado em uma instância `io.TextIOWrapper` com a codificação especificada, comportamento de tratamento de erros e fins de linha.

class compression.zstd.ZstdFile(*file*, */*, *mode='rb'*, ***, *level=None*, *options=None*, *zstd_dict=None*)

Abre um arquivo compactado com Zstandard no modo binário.

Um `ZstdFile` pode envolver um objeto arquivo já aberto, ou operar diretamente em um arquivo nomeado. O argumento *file* especifica o objeto arquivo a ser envolvido ou o nome do arquivo a ser aberto (como um objeto `str`, `bytes` ou objeto caminho ou similar). Se estiver envolvendo um objeto arquivo existente, o arquivo envolvido não será fechado quando o `ZstdFile` for fechado.

O argumento *mode* pode ser `'rb'` para leitura (padrão), `'wb'` para substituição, `'xb'` para criação exclusiva ou `'ab'` para anexar. Estes podem ser equivalentemente dados como `'r'`, `'w'`, `'x'` e `'a'` respectivamente.

Se *file* for um objeto arquivo (ao invés de um nome de arquivo real), um modo de `'w'` não truncará o arquivo e será equivalente a `'a'`.

Durante a leitura, o argumento *options* pode ser um dicionário que fornece parâmetros avançados de descompactação; consulte `DecompressionParameter` para obter informações detalhadas sobre os parâmetros suportados. O argumento *zstd_dict* é uma instância de

`ZstdDict` a ser usada durante a descompactação. Durante a leitura, se o argumento *level* não for None, uma exceção `TypeError` será levantada.

When writing, the *options* argument can be a dictionary providing advanced compression parameters; see `CompressionParameter` for detailed information about supported parameters. The *level* argument is the compression level to use when writing compressed data. Only one of *level* or *options* may be passed. The *zstd_dict* argument is a `ZstdDict` instance to be used during compression.

`ZstdFile` oferece suporte a todos os membros especificados por `io.BufferedIOBase`, exceto `detach()` e `truncate()`. Iteração e a instrução `with` são suportadas.

O método e os atributos a seguir também são fornecidos:

peek(*size=-1*)

Retorna dados armazenados em buffer sem avançar a posição do arquivo. Pelo menos um byte de dados será retornado, a menos que o EOF tenha sido atingido. O número exato de bytes retornados não é especificado (o argumento *size* é ignorado).

Nota

Enquanto chamar `peek()` não altera a posição do arquivo de `ZstdFile`, pode alterar a posição do objeto arquivo subjacente (por exemplo, se o `ZstdFile` foi construído passando um objeto arquivo para *file*).

mode

`'rb'` para leitura e `'wb'` para escrita.

name

O nome do arquivo Zstandard. Equivalente ao atributo `name` do objeto arquivo subjacente.

Compactando e descompactando dados na memória

compression.zstd.compress(*data*, *level=None*, *options=None*, *zstd_dict=None*)

Compacta *data* (um objeto bytes ou similar), retornando os dados compactados como um objeto `bytes`.

O argumento *level* é um inteiro que controla o nível de compactação. *level* é uma alternativa à configuração `CompressionParameter.compression_level` em *options*. Use `bounds()` em `compression_level` para obter os valores que podem ser passados para *level*. Se opções avançadas de compactação forem necessárias, o argumento *level* deve ser omitido e, no dicionário *options*, o parâmetro `CompressionParameter.compression_level` deve ser definido.

O argumento *options* é um dicionário Python que contém parâmetros de compactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação `CompressionParameter`.

O argumento *zstd_dict* é uma instância de `ZstdDict` contendo dados treinados para melhorar a eficiência da compactação. A função `train_dict()` pode ser usada para gerar um dicionário Zstandard.

compression.zstd.decompress(*data*, *zstd_dict=None*, *options=None*)

Descompacta *data* (um objeto bytes ou similar), retornando os dados descompactados como um objeto `bytes`.

O argumento *options* é um dicionário Python que contém parâmetros de descompactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação `DecompressionParameter`.

O argumento *zstd_dict* é uma instância de **ZstdDict** contendo dados treinados usados durante a compactação. Este deve ser o mesmo dicionário Zstandard usado durante a compactação.

Se *data* forem a concatenação de vários quadros compactados distintos, descompacta todos esses quadros e retorna a concatenação dos resultados.

class compression.zstd.ZstdCompressor(*level=None*, *options=None*, *zstd_dict=None*)

Cria um objeto compactação, que pode ser usado para compactar dados de forma incremental.

Para uma maneira mais conveniente de compactar um único bloco de dados, consulte a função a nível de módulo **compress()**.

O argumento *level* é um inteiro que controla o nível de compactação. *level* é uma alternativa à configuração **CompressionParameter.compression_level** em *options*. Use **bounds()** em **compression_level** para obter os valores que podem ser passados para *level*. Se opções avançadas de compactação forem necessárias, o argumento *level* deve ser omitido e, no dicionário *options*, o parâmetro **CompressionParameter.compression_level** deve ser definido.

O argumento *options* é um dicionário Python que contém parâmetros de compactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação **CompressionParameter**.

O argumento *zstd_dict* é uma instância opcional de **ZstdDict** contendo dados treinados para melhorar a eficiência da compactação. A função **train_dict()** pode ser usada para gerar um dicionário Zstandard.

compress(*data*, *mode=ZstdCompressor.CONTINUE*)

Compacte *data* (um objeto bytes ou similar), retornando um objeto **bytes** com dados compactados, se possível, ou, caso contrário, um

objeto **bytes** vazio. Alguns dos *data* podem ser armazenados internamente em buffer, para uso em chamadas posteriores a **compress()** e **flush()**. Os dados retornados devem ser concatenados com a saída de quaisquer chamadas anteriores a **compress()**.

O argumento *mode* é um atributo **ZstdCompressor**, podendo ser definido com **CONTINUE**, **FLUSH_BLOCK** ou **FLUSH_FRAME**.

Quando todos os dados forem fornecidos ao compactador, chama o método **flush()** para finalizar o processo de compactação. Se **compress()** for chamado com *mode* definido como **FLUSH_FRAME**, **flush()** não deve ser chamado, pois gravaria um novo quadro vazio.

flush(*mode=ZstdCompressor.FLUSH_FRAME*)

Conclui o processo de compactação, retornando um objeto **bytes** contendo todos os dados armazenados nos buffers internos do compressor.

O argumento *mode* é um atributo **ZstdCompressor**, podendo ser definido com **FLUSH_BLOCK** ou **FLUSH_FRAME**.

set_pledged_input_size(*size*)

Especifica a quantidade de *size* dados descompactados que será fornecida para o próximo quadro. *size* será gravado no cabeçalho do quadro do próximo quadro, a menos que **CompressionParameter.content_size_flag** seja `False` ou `0`. Um tamanho `0` significa que o quadro está vazio. Se *size* for `None`, o cabeçalho do quadro omitirá o tamanho do quadro. Quadros que incluem o tamanho dos dados descompactados requerem menos memória para descompactação, especialmente em níveis de compactação mais altos.

Se **last_mode** não for **FLUSH_FRAME**, uma **ValueError** será levantada, pois o compactador não está no início de um quadro. Se o tamanho prometido não corresponder ao tamanho real dos dados

fornecidos para `compress()`, chamadas futuras para **`compress()`** ou `flush()` poderão levantar `ZstdError` e o último bloco de dados poderá ser perdido.

Depois que `flush()` ou `compress()` são chamados com o modo `FLUSH_FRAME`, o próximo quadro não incluirá o tamanho do quadro no cabeçalho, a menos que **`set_pledged_input_size()`** seja chamado novamente.

CONTINUE

Coleta mais dados para compactação, o que pode ou não gerar saída imediata. Este modo otimiza a taxa de compactação maximizando a quantidade de dados por bloco e quadro.

FLUSH_BLOCK

Conclui e grava um bloco no fluxo de dados. Os dados retornados até o momento podem ser descompactados imediatamente. Dados anteriores ainda podem ser referenciados em blocos futuros gerados por chamadas a `compress()`, melhorando a compactação.

FLUSH_FRAME

Conclui e escreve um quadro. Dados futuros fornecidos a `compress()` serão gravados em um novo quadro e *não poderão* fazer referência a dados passados.

last_mode

O último modo passado para `compress()` ou `flush()`. O valor pode ser `CONTINUE`, `FLUSH_BLOCK` ou `FLUSH_FRAME`. O valor inicial é **`FLUSH_FRAME`**, significando que o compactador está no início de um novo quadro.

class compression.zstd.ZstdDecompressor(*zstd_dict=None*, *options=None*)

Cria um objeto descompactador, que pode ser usado para descompactar dados de forma incremental.

Para uma maneira mais conveniente de descompactar todo um fluxo compactado de uma só vez, consulte a função a nível de módulo

decompress().

O argumento *options* é um dicionário Python que contém parâmetros de descompactação avançados. As chaves e valores válidos para parâmetros de compactação estão documentados na documentação **DecompressionParameter**.

O argumento *zstd_dict* é uma instância de **ZstdDict** contendo dados treinados usados durante a compactação. Este deve ser o mesmo dicionário Zstandard usado durante a compactação.

Nota

Esta classe não lidam de forma transparente entradas contendo múltiplos quadros compactados, ao contrário da função **decompress()** e da classe **ZstdFile**. Para descompactar uma entrada com múltiplos quadros, você deve usar **decompress()**, **ZstdFile** se estiver trabalhando com um objeto arquivo ou múltiplas instâncias de **ZstdDecompressor**.

decompress(*data*, *max_length=-1*)

Descompacta dados *data* (um objeto bytes ou similar), retornando dados não compactados como bytes. Alguns dos *data* podem ser armazenados em buffer internamente, para uso em chamadas posteriores para **decompress()**. Os dados retornados devem ser concatenados com a saída de qualquer chamada anterior para **decompress()**.

If *max_length* is non-negative, the method returns at most *max_length* bytes of decompressed data. If this limit is reached and further output can be produced (or EOF is reached), the **needs_input** attribute will be set to `False`. In this case, the next call to **decompress()** may provide *data* as `b''` to obtain more of the output. The full content can thus be read like:

```
process_output(d.decompress(data, max_length))
while not d.eof and not d.needs_input:
    process_output(d.decompress(b"", max_length))
```

Se todos os dados de entrada foram descompactados e retornados (seja porque era menor que *max_length* bytes, ou porque *max_length* era negativo), o atributo `needs_input` será definido como `True`.

Attempting to decompress data after the end of a frame will raise a `EOFError`. Any data found after the end of the frame is ignored and saved in the `unused_data` attribute.

eof

`True` se o marcador de fim de fluxo foi atingido.

unused_data

Dados encontrados após o término do fluxo compactado.

Antes do final do fluxo ser alcançado, isso será `b''`.

needs_input

`False` se o método `decompress()` puder fornecer mais dados descompactados antes de exigir uma nova entrada compactada.

Dicionários de Zstandard

compression.zstd.train_dict(*samples*, *dict_size*)

Treina um dicionário Zstandard, retornando uma instância `ZstdDict`. Os dicionários Zstandard permitem uma compactação mais eficiente de dados menores, que tradicionalmente são difíceis de compactar devido à menor repetição. Se você estiver compactando vários grupos de dados semelhantes (como arquivos semelhantes), os dicionários Zstandard podem melhorar significativamente as taxas de compactação e a velocidade.

O argumento *samples* (um iterável de objetos `bytes`) é a população de amostras usada para treinar o dicionário Zstandard.

O argumento *dict_size*, um inteiro, é o tamanho máximo (em bytes) que o dicionário Zstandard deve ter. A documentação do Zstandard sugere

um máximo absoluto de no máximo 100 KB, mas o máximo pode ser menor dependendo dos dados. Dicionários maiores geralmente reduzem a compactação, mas melhoram as taxas de compactação. Dicionários menores levam a uma compactação mais rápida, mas reduzem a taxa de compactação.

compression.zstd.finalize_dict(*zstd_dict*, /, *samples*, *dict_size*, *level*)

Uma função avançada para converter um dicionário Zstandard de "conteúdo bruto" em um dicionário Zstandard comum. Dicionários de "conteúdo bruto" são uma sequência de bytes que não precisa seguir a estrutura de um dicionário Zstandard comum.

O argumento *zstd_dict* é uma instância `ZstdDict` com `dict_content` contendo o conteúdo bruto do dicionário.

O argumento *samples* (um iterável de objetos `bytes`) contém dados de amostra para gerar o dicionário Zstandard.

O argumento *dict_size*, um inteiro, é o tamanho máximo (em bytes) que o dicionário Zstandard deve ter. Consulte `train_dict()` para sugestões sobre o tamanho máximo do dicionário.

O argumento *level* (um inteiro) é o nível de compactação que se espera que seja passado aos compactadores usando este dicionário. As informações do dicionário variam para cada nível de compactação, portanto, ajustar o nível de compactação adequado pode tornar a compactação mais eficiente.

class compression.zstd.ZstdDict(*dict_content*, /, *, *is_raw=False*)

Um invólucro para dicionários Zstandard. Dicionários podem ser usados para melhorar a compactação de muitos pequenos blocos de dados. Use `train_dict()` se precisar treinar um novo dicionário a partir de dados de amostra.

O argumento *dict_content* (um objeto bytes ou similar) é a informação do dicionário já treinada.

O argumento *is_raw*, um booleano, é um parâmetro avançado que controla o significado de *dict_content*. `True` significa que *dict_content* é um dicionário de “conteúdo bruto”, sem nenhuma restrição de formato. `False` significa que *dict_content* é um dicionário Zstandard comum, criado a partir de funções Zstandard, por exemplo, **`train_dict()`** ou a CLI externa **zstd**.

Ao passar um **`ZstdDict`** para uma função, os atributos **`as_digested_dict`** e **`as_undigested_dict`** podem controlar como o dicionário é carregado, passando-os como o argumento `zstd_dict`, por exemplo, `compress(data, zstd_dict=zd.as_digested_dict)`. A digestão de um dicionário é uma operação custosa que ocorre ao carregar um dicionário Zstandard. Ao fazer várias chamadas para compactação ou descompactação, passar um dicionário digerido reduzirá a sobrecarga de carregamento do dicionário.

	Diferença para compactação	
	Dicionário resumido	**Dicionário não resumido**
Parâmetros avançados do compactador que podem ser substituídos pelos parâmetros do dicionário	`window_log`, `hash_log`, `chain_log`, `search_log`, `min_match`, `target_length`, `strategy`, `enable_long_distance_matching`, `ldm_hash_log`, `ldm_min_match`, `ldm_bucket_size_log`, `ldm_hash_rate_log` e alguns parâmetros não-públicos.	None

	Dicionário resumido	Dicionário não resumido
`ZstdDict` armazena em cache internamente o dicionário	Sim. É mais rápido carregar um dicionário resumido novamente com o mesmo nível de compactação.	Não. Se você deseja carregar um dicionário não digerido várias vezes, considere reutilizar um objeto compressor.

Se passar um **`ZstdDict`** sem nenhum atributo, um dicionário não digerido será passado por padrão ao compactar e um dicionário digerido será gerado, se necessário, e passado por padrão ao descompactar.

dict_content

O conteúdo do dicionário Zstandard, um objeto `bytes`. É o mesmo que o argumento *dict_content* no método `__init__`. Pode ser usado com outros programas, como o programa de linha de comando `zstd`.

dict_id

Identificador do dicionário Zstandard, um valor inteiro não negativo.

Um valor diferente de zero significa que o dicionário é comum, criado por funções Zstandard e seguindo o formato Zstandard.

`0` significa um dicionário de “conteúdo bruto”, livre de qualquer restrição de formato, usado para usuários avançados.

Nota

O significado de `0` para **`ZstdDict.dict_id`** é diferente do atributo `dictionary_id` da função **`get_frame_info()`**.

as_digested_dict
Carrega como um dicionário resumido.

as_undigested_dict
Carrega como um dicionário não digerido.

Controle avançado de parâmetros

class compression.zstd.CompressionParameter

Uma **`IntEnum`** contendo as chaves de parâmetros de compactação avançada que podem ser usadas ao compactar dados.

O método **`bounds()`** pode ser usado em qualquer atributo para obter os valores válidos para esse parâmetro.

Parameters are optional; any omitted parameter will have its value selected automatically.

Exemplo de obtenção da delimitação inferior e superior de **`compression_level`**:

```
lower, upper = CompressionParameter.compression_level.bounds(
```

Exemplo de configuração do atributo **`window_log`** para o tamanho máximo:

```
_lower, upper = CompressionParameter.window_log.bounds()
options = {CompressionParameter.window_log: upper}
compress(b'venezuelan beaver cheese', options=options)
```

bounds()

Retorna a tupla de limites int, `(lower, upper)`, de um parâmetro de compactação. Este método deve ser chamado no atributo cujos

limites você deseja recuperar. Por exemplo, para obter os valores válidos para **`compression_level`**, pode-se verificar o resultado de `CompressionParameter.compression_level.bounds()`.

Tanto o limite inferior quanto o superior são inclusivos.

compression_level

Um meio de alto nível para definir outros parâmetros de compactação que afetam a velocidade e a taxa de compactação de dados.

Os níveis de compactação regulares são maiores que `0`. Valores maiores que `20` são considerados compactação “ultra” e requerem mais memória do que outros níveis. Valores negativos podem ser usados para compensar uma compactação mais rápida por taxas de compactação piores.

Define o nível como zero usa **`COMPRESSION_LEVEL_DEFAULT`**.

window_log

Distância máxima de referência reversa permitida que o compactador pode usar ao compactar dados, expressa como potência de dois, `1 << window_log` bytes. Este parâmetro influencia bastante o uso de memória da compactação. Valores mais altos requerem mais memória, mas geram melhores valores de compactação.

Um valor zero faz com que o valor seja selecionado automaticamente.

hash_log

Tamanho da tabela de sondagem inicial, como uma potência de dois. O uso de memória resultante é `1 << (hash_log+2)` bytes. Tabelas maiores melhoram a taxa de compactação de estratégias <= **`dfast`** e melhoram a velocidade de compactação de estratégias > **`dfast`**.

Um valor zero faz com que o valor seja selecionado automaticamente.

chain_log

Tamanho da tabela de busca multiprobe, como uma potência de dois. O uso de memória resultante é `1 << (chain_log+2)` bytes. Tabelas maiores resultam em uma compactação melhor e mais lenta. Este parâmetro não tem efeito para a estratégia **`fast`**. Ele ainda é útil ao usar a estratégia **`dfast`**, caso em que define uma tabela de sondagem secundária.

Um valor zero faz com que o valor seja selecionado automaticamente.

search_log

Número de tentativas de busca, como uma potência de dois. Mais tentativas resultam em uma compactação melhor e mais lenta. Este parâmetro é inútil para as estratégias **`fast`** e **`dfast`**.

Um valor zero faz com que o valor seja selecionado automaticamente.

min_match

Tamanho mínimo das correspondências pesquisadas. Valores maiores aumentam a velocidade de compactação e descompactação, mas diminuem a taxa. Observe que o Zstandard ainda consegue encontrar correspondências de tamanho menor; ele apenas ajusta seu algoritmo de busca para procurar por esse tamanho e por tamanhos maiores. Para todas as estratégias < **`btopt`**, o mínimo efetivo é `4`; para todas as estratégias > **`fast`**, o máximo efetivo é `6`.

Um valor zero faz com que o valor seja selecionado automaticamente.

target_length

O impacto deste campo depende da **`Strategy`** selecionada.

Para as estratégias **`btopt`**, **`btultra`** e **`btultra2`**, o valor é o comprimento de uma correspondência considerada “boa o

suficiente” para interromper a busca. Valores maiores melhoram as taxas de compactação, mas a compactação é mais lenta.

Para a estratégia `fast`, é a distância entre as amostragens de correspondência. Valores maiores tornam a compactação mais rápida, mas com uma taxa de compactação pior.

Um valor zero faz com que o valor seja selecionado automaticamente.

strategy

Quanto maior o valor da estratégia selecionada, mais complexa será a técnica de compactação usada pelo zstd, resultando em taxas de compactação mais altas, mas em compactação mais lenta.

Ver também

`Strategy`

enable_long_distance_matching

A correspondência de longa distância pode ser usada para melhorar a compactação de entradas grandes, encontrando correspondências maiores em distâncias maiores. Ela aumenta o uso de memória e o tamanho da janela.

Os valores `True` ou `1` habilitam a correspondência a longa distância, enquanto os valores `False` ou `0` a desabilitam.

Habilitar este parâmetro aumenta o tamanho padrão de `window_log` para 128 MiB, exceto quando expressamente definido com um valor diferente. Esta configuração é habilitada por padrão se **`window_log`** >= 128 MiB e a estratégia de compactação >= `btopt` (nível de compactação 16+).

ldm_hash_log

Tamanho da tabela para correspondência de longa distância, como uma potência de dois. Valores maiores aumentam o uso de memória

e a taxa de compactação, mas diminuem a velocidade de compactação.

Um valor zero faz com que o valor seja selecionado automaticamente.

ldm_min_match

Tamanho mínimo de correspondência para correspondência de longa distância. Valores maiores ou muito pequenos podem frequentemente diminuir a taxa de compactação.

Um valor zero faz com que o valor seja selecionado automaticamente.

ldm_bucket_size_log

Tamanho do log de cada bucket na tabela hash do comparador de longa distância para resolução de colisões. Valores maiores melhoram a resolução de colisões, mas diminuem a velocidade de compactação.

Um valor zero faz com que o valor seja selecionado automaticamente.

ldm_hash_rate_log

Frequência de inserção/pesquisa de entradas na tabela hash do comparador de longa distância. Valores maiores melhoram a velocidade de compactação. Desvios muito grandes em relação ao valor padrão provavelmente resultarão em uma redução na taxa de compactação.

Um valor zero faz com que o valor seja selecionado automaticamente.

content_size_flag

Escreve o tamanho dos dados a serem compactados no cabeçalho do quadro Zstandard quando conhecido antes da compactação.

Este sinalizador só entra em vigor nos seguintes cenários:

- Chamando `compress()` para compactação única
- Fornecendo todos os dados a serem compactados no quadro em uma única chamada `ZstdCompressor.compress()`, com o modo `ZstdCompressor.FLUSH_FRAME`.
- Chamando `ZstdCompressor.set_pledged_input_size()` com a quantidade exata de dados que serão fornecidos ao compactador antes de qualquer chamada para `ZstdCompressor.compress()` para o quadro atual. **`ZstdCompressor.set_pledged_input_size()`** deve ser chamado para cada novo quadro.

Todas as outras chamadas de compactação podem não escrever as informações de tamanho no cabeçalho do quadro.

Os valores `True` ou `1` habilitam o sinalizador de tamanho de conteúdo, enquanto os valores `False` ou `0` o desabilitam.

checksum_flag

Uma soma de verificação de quatro bytes usando XXHash64 do conteúdo descompactado é estrita ao final de cada quadro. O código de descompactação do Zstandard verifica a soma de verificação. Se houver uma incompatibilidade, uma exceção `ZstdError` é levantada.

Os valores `True` ou `1` habilitam a geração da soma de integridade, enquanto os valores `False` ou `0` a desabilitam.

dict_id_flag

Ao compactar com um `ZstdDict`, o ID do dicionário é escrito no cabeçalho do quadro.

Os valores `True` ou `1` habilitam o armazenamento do ID de dicionário, enquanto os valores `False` ou `0` o desabilitam.

nb_workers

Seleciona quantas threads serão geradas para compactação em paralelo. Quando **`nb_workers`** > 0, habilita a compactação multithread, um valor `1` significa “modo multithread de uma thread”. Mais workers melhoram a velocidade, mas também

aumentam o uso de memória e reduzem ligeiramente a taxa de compactação.

Um valor zero desabilita multithreading.

job_size

Tamanho de um trabalho de compactação, em bytes. Este valor é aplicado somente quando `nb_workers` >= 1. Cada trabalho de compactação é concluído em paralelo, portanto, este valor pode impactar indiretamente o número de threads ativas.

Um valor zero faz com que o valor seja selecionado automaticamente.

overlap_log

Define a quantidade de dados recarregados de trabalhos anteriores (threads) para novos trabalhos a serem usados pela janela "Look Behind" durante a compactação. Este valor só é usado quando `nb_workers` >= 1. Os valores aceitáveis variam de 0 a 9.

- 0 significa definir dinamicamente a quantidade de sobreposição
- 1 significa nenhuma sobreposição
- 9 significa usar o tamanho total da janela do trabalho anterior

Cada incremento reduz pela metade/dobra o tamanho da sobreposição. "8" significa uma sobreposição de `window_size/2`, "7" significa uma sobreposição de `window_size/4`, etc.

class compression.zstd.DecompressionParameter

An `IntEnum` containing the advanced decompression parameter keys that can be used when decompressing data. Parameters are optional; any omitted parameter will have its value selected automatically.

O método `bounds()` pode ser usado em qualquer atributo para obter os valores válidos para esse parâmetro.

Exemplo de configuração do atributo `window_log_max` para o tamanho máximo:

```
data = compress(b'Some very long buffer of bytes...')

_lower, upper = DecompressionParameter.window_log_max.bounds(

options = {DecompressionParameter.window_log_max: upper}
decompress(data, options=options)
```

bounds()

Retorna a tupla de limites int, `(lower, upper)`, de um parâmetro de descompactação. Este método deve ser chamado no atributo cujos limites você deseja recuperar.

Tanto o limite inferior quanto o superior são inclusivos.

window_log_max

O logaritmo na base dois do tamanho máximo da janela usada durante a descompactação. Isso pode ser útil para limitar a quantidade de memória usada ao descompactar. Um tamanho máximo de janela maior resulta em uma descompactação mais rápida.

Um valor zero faz com que o valor seja selecionado automaticamente.

class compression.zstd.Strategy

Um `IntEnum` contendo estratégias de compactação. Estratégias com números mais altos correspondem a compactações mais complexas e lentas.

Nota

Os valores dos atributos de `Strategy` não são necessariamente estáveis entre as versões do zstd. Apenas a ordem dos atributos é confiável. Os atributos estão listados abaixo em ordem.

As seguintes estratégias estão disponíveis:

fast

dfast

greedy

lazy

lazy2

btlazy2

btopt

btultra

btultra2

Diversos

compression.zstd.get_frame_info(*frame_buffer*)

Recupera um objeto `FrameInfo` contendo metadados sobre um quadro Zstandard. Os quadros contêm metadados relacionados aos dados compactados que eles armazenam.

class compression.zstd.FrameInfo

Metadados relatados a um quadro Zstandard.

decompressed_size

O tamanho do conteúdo descompactado do quadro.

dictionary_id

Um número inteiro que representa o ID do dicionário Zstandard necessário para descompactar o quadro. `0` significa que o ID do dicionário não foi registrado no cabeçalho do quadro. Isso pode significar que um dicionário Zstandard não é necessário ou que o ID de um dicionário necessário não foi registrado.

compression.zstd.COMPRESSION_LEVEL_DEFAULT

O nível de compactação padrão para Zstandard é: `3`.

compression.zstd.zstd_version_info

Número da versão da biblioteca zstd em tempo de execução, representada por uma tupla de inteiros (principal, secundária, lançamento).

Exemplos

Lendo em um arquivo compactado:

```
from compression import zstd

with zstd.open("file.zst") as f:
    file_content = f.read()
```

Criando um arquivo compactado:

```
from compression import zstd

data = b"Inserir dados arquivo"
with zstd.open("file.zst", "w") as f:
    f.write(data)
```

Compactando dados em memória:

```
from compression import zstd

data_in = b"Insert Data Here"
data_out = zstd.compress(data_in)
```

Compactação incremental:

```
from compression import zstd

comp = zstd.ZstdCompressor()
out1 = comp.compress(b"Alguns dados\n")
out2 = comp.compress(b"Outro pedaço de dados\n")
out3 = comp.compress(b"Ainda mais dados\n")
```

```
out4 = comp.flush()
# Concatena todos os resultados parciais:
result = b"".join([out1, out2, out3, out4])
```

Escrevendo dados compactados para um arquivo já aberto:

```
from compression import zstd

with open("myfile", "wb") as f:
    f.write(b"Estes dados não serão compactados\n")
    with zstd.open(f, "w") as zstf:
        zstf.write(b"Isso *vai* ser compactado\n")
    f.write(b"Não compactado\n")
```

Criando um arquivo compactado usando parâmetros de compactação:

```
from compression import zstd

options = {
   zstd.CompressionParameter.checksum_flag: 1
}
with zstd.open("file.zst", "w", options=options) as f:
    f.write(b"Se importa se eu me juntar?")
```

`zlib` — Compactação compatível com gzip

Para aplicações que requerem compactação de dados, as funções neste módulo permitem compactação e descompactação, utilizando a biblioteca zlib [https://www.zlib.net].

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

As funções do zlib têm muitas opções e geralmente precisam ser usadas em uma ordem específica. Esta documentação não tenta cobrir todas as permutações; consulte o manual do zlib [https://www.zlib.net/manual.html] para obter informações oficiais.

Para leitura e escrita de arquivos `.gz`, consulte o módulo **`gzip`**.

A exceção e as funções disponíveis neste módulo são:

exception zlib.error

Exceção levantada em erros de compactação e descompactação.

zlib.adler32(*data*, *value=1*, /)

Calcula uma soma de verificação Adler-32 de *data*. (Uma soma de verificação Adler-32 é quase tão confiável quanto uma CRC32, mas pode ser calculada muito mais rapidamente.) O resultado é um número inteiro sem sinal de 32 bits. Se *value* estiver presente, ele será usado como o valor inicial da soma de verificação; caso contrário, um valor padrão de 1 é usado. A passagem de *value* permite calcular uma soma de verificação em execução através da concatenação de várias entradas. O algoritmo não é criptograficamente forte e não deve ser usado para

autenticação ou assinaturas digitais. Como o algoritmo foi projetado para uso como um algoritmo de soma de verificação, não é adequado para uso como um algoritmo de hash geral.

Alterado na versão 3.0: O resultado é sempre sem sinal.

zlib.adler32_combine(*adler1*, *adler2*, *len2*, /)

Combine two Adler-32 checksums into one.

Given the Adler-32 checksum *adler1* of a sequence `A` and the Adler-32 checksum *adler2* of a sequence `B` of length *len2*, return the Adler-32 checksum of `A` and `B` concatenated.

This function is typically useful to combine Adler-32 checksums that were concurrently computed. To compute checksums sequentially, use **`adler32()`** with the running checksum as the `value` argument.

Adicionado na versão 3.15.

zlib.compress(*data*, /, *level=Z_DEFAULT_COMPRESSION*, *wbits=MAX_WBITS*)

Compacta os bytes em *data*, retornando um objeto bytes contendo dados compactados. *level* é um inteiro de `0` a `9` ou `-1` que controla o nível de compactação; Consulte **`Z_BEST_SPEED`** (`1`), **`Z_BEST_COMPRESSION`** (`9`), **`Z_NO_COMPRESSION`** (`0`) e o padrão, **`Z_DEFAULT_COMPRESSION`** (`-1`) para obter mais informações sobre esses valores.

O argumento *wbits* controla o tamanho do buffer do histórico (ou o "tamanho da janela") usado ao compactar dados e se um cabeçalho e um trailer estão incluídos na saída. Pode levar vários intervalos de valores, padronizando para `15` (**`MAX_WBITS`**):

- +9 a +15: o logaritmo de base dois do tamanho da janela, que varia entre 512 e 32768. Valores maiores produzem melhor compactação às custas de maior uso de memória. A saída resultante incluirá um cabeçalho e uma sequência específicos para zlib.

- -9 a -15: Usa o valor absoluto de *wbits* como o logaritmo do tamanho da janela, enquanto produz um fluxo de saída bruto sem cabeçalho ou soma de verificação à direita.
- +25 a +31 = 16 + (9 a 15): Usa os 4 bits baixos do valor como logaritmo do tamanho da janela, incluindo um cabeçalho básico **gzip** e a soma de verificação à direita na saída.

Levanta uma exceção do tipo `error`, se ocorrer algum erro.

Alterado na versão 3.6: level pode agora ser usado como um palavra reservada nomeada.

Alterado na versão 3.11: O parâmetro *wbits* agora está disponível para definir janelas de bits e tipo de compactação.

zlib.compressobj(*level=Z_DEFAULT_COMPRESSION*, *method=DEFLATED*, *wbits=MAX_WBITS*, *memLevel=DEF_MEM_LEVEL*, *strategy=Z_DEFAULT_STRATEGY*[, *zdict*])

Retorna um objeto de compactação, a ser usado para compactar fluxos de dados que não cabem na memória de uma só vez.

level é um nível de compactação – um inteiro de `0` a `9` ou `-1`. Consulte `Z_BEST_SPEED` (`1`), `Z_BEST_COMPRESSION` (`9`), `Z_NO_COMPRESSION` (`0`) e o padrão, `Z_DEFAULT_COMPRESSION` (`-1`) para obter mais informações sobre esses valores.

method é o algoritmo de compactação. Atualmente, o único valor suportado é `DEFLATED`.

O parâmetro *wbits* controla o tamanho do histórico buffer (ou o "tamanho da janela do buffer") e qual o formato do cabeçalho e do trailer serão usados. Ele tem o mesmo significado que o descrito para compress().

O argumento *memLevel* controla a quantidade de memória usada para o estado de compactação interno. Os valores válidos variam de `1` a `9`.

Valores mais altos usam mais memória, mas são mais rápidos e produzem uma saída menor.

strategy é usado para ajustar o algoritmo de compactação. Os valores possíveis são `Z_DEFAULT_STRATEGY`, `Z_FILTERED`, `Z_HUFFMAN_ONLY`, `Z_RLE` e `Z_FIXED`.

zdict é um dicionário de compactação predefinido. Esta é uma sequência de bytes (como um objeto `bytes`) que contém subsequências que se espera que ocorram com frequência nos dados a serem compactados. As subsequências que se espera serem mais comuns devem aparecer no final do dicionário.

Alterado na versão 3.3: Adicionado o suporte ao parâmetro e argumento nomeado *zdict*.

zlib.crc32(*data*, *value=0*, /)

Calcula uma soma de verificação CRC (Cyclic Redundancy Check) de *data*. O resultado é um número inteiro sem sinal de 32 bits. Se *value* estiver presente, ele será usado como o valor inicial da soma de verificação; caso contrário, um valor padrão de 1 é usado. A passagem de *value* permite calcular uma soma de verificação em execução através da concatenação de várias entradas. O algoritmo não é criptograficamente forte e não deve ser usado para autenticação ou assinaturas digitais. Como o algoritmo foi projetado para uso como um algoritmo de soma de verificação, não é adequado para uso como um algoritmo de hash geral.

Alterado na versão 3.0: O resultado é sempre sem sinal.

zlib.crc32_combine(*crc1*, *crc2*, *len2*, /)

Combine two CRC-32 checksums into one.

Given the CRC-32 checksum *crc1* of a sequence `A` and the CRC-32 checksum *crc2* of a sequence `B` of length *len2*, return the CRC-32 checksum of `A` and `B` concatenated.

This function is typically useful to combine CRC-32 checksums that were concurrently computed. To compute checksums sequentially, use `crc32()` with the running checksum as the `value` argument.

Adicionado na versão 3.15.

zlib.decompress(*data*, /, *wbits=MAX_WBITS*, *bufsize=DEF_BUF_SIZE*)

Descompacta os bytes em *data*, retornando um objeto de bytes que contém os dados não compactados. O parâmetro *wbits* depende do formato de *data* e é discutido mais abaixo. Se *bufsize* for fornecido, ele será usado como o tamanho inicial do buffer de saída. Levanta a exceção `error` se ocorrer algum erro.

O parâmetro *wbits* controla o tamanho do buffer do histórico (ou "tamanho da janela") e qual formato de cabeçalho e sequência é esperado. É semelhante ao parâmetro para `compressobj()`, mas aceita mais intervalos de valores:

- +8 a +15: O logaritmo de base dois do tamanho da janela. A entrada deve incluir um cabeçalho e uma sequência de zlib.
- 0: Determina automaticamente o tamanho da janela no cabeçalho zlib. Suportado apenas desde o zlib 1.2.3.5.
- −8 a −15: Usa o valor absoluto de *wbits* como o logaritmo do tamanho da janela. A entrada deve ser um fluxo bruto sem cabeçalho ou sequência.
- +24 a +31 = 16 + (8 a 15): Usa os 4 bits baixos do valor como logaritmo do tamanho da janela. A entrada deve incluir um cabeçalho e sequência de gzip.
- +40 a +47 = 32 + (8 a 15): Usa os 4 bits baixos do valor como logaritmo do tamanho da janela e aceita automaticamente o formato zlib ou gzip.

Ao descompactar um fluxo, o tamanho da janela não deve ser menor que o tamanho originalmente usado para compactar o fluxo; o uso de um valor muito pequeno pode resultar em uma exceção `error`. O valor padrão *wbits* corresponde ao maior tamanho da janela e requer que um cabeçalho e uma sequência de zlib sejam incluídos.

bufsize é o tamanho inicial do buffer usado para armazenar dados descompactados. Se for necessário mais espaço, o tamanho do buffer será aumentado conforme necessário, para que você não precise obter esse valor exatamente correto; sintonizando, apenas algumas chamadas serão salvas em **`malloc()`**.

Alterado na versão 3.6: wbits e *bufsize* podem ser usados como argumentos nomeados.

zlib.decompressobj(*wbits=MAX_WBITS*, *zdict=b''*)

Retorna um objeto descompactado, a ser usado para descompactar fluxos de dados que não cabem na memória de uma só vez.

O parâmetro *wbits* controla o tamanho do histórico do buffer (ou o "tamanho da janela do buffer") e qual formato do cabeçalho e trailer são esperados. Ele tem o mesmo significado que o descrito para decompress().

O parâmetro *zdict* especifica uma dicionário predefinido de compactação. Se fornecido, deve ser o mesmo dicionário usado pelo compactador que produziu os dados a serem descompactados.

Nota

Se *zdict* for um objeto mutável (como um **`bytearray`**), você não deve modificar seu conteúdo entre a chamada de **`decompressobj()`** e a primeira chamada para o método de descompatação `decompress()`.

Alterado na versão 3.3: Adicionado o parâmetro *zdict*.

Um objeto do tipo Compress oferece suporte aos seguintes métodos:

Compress.compress(*data*, /)

Comprime *data*, retornando um objeto de bytes que contém dados compactados para pelo menos parte dos dados em *data*. Esses dados devem ser concatenados à saída produzida por quaisquer chamadas

anteriores ao método `compress()`. Algumas entradas podem ser mantidas em buffers internos para processamento posterior.

Compress.flush(*mode=Z_FINISH*, *[,]*)

Toda a entrada pendente é processada e um objeto de bytes contendo a saída compactada restante é retornado. O *mode* pode ser selecionado entre constantes `Z_NO_FLUSH` , `Z_PARTIAL_FLUSH`, `Z_SYNC_FLUSH`, `Z_FULL_FLUSH`, `Z_BLOCK` ou `Z_FINISH`, com o padrão sendo **`Z_FINISH`**. Exceto **`Z_FINISH`**, todas as demais constantes permitem a compactação de mais bytestrings de dados, enquanto **`Z_FINISH`** finaliza o fluxo compactado e impede a compactação de mais dados. Depois de chamar `flush()` com *mode* definido como **`Z_FINISH`**, o método `compress()` não pode ser chamado novamente; a única ação restante possível é excluir o objeto.

Compress.copy()

Retorna uma cópia do objeto de compactação. Isso pode ser usado para compactar com eficiência um conjunto de dados que compartilham um prefixo inicial comum.

Alterado na versão 3.8: As funções `copy.copy()` e `copy.deepcopy()` foram adicionadas como suporte para a compactação de objetos.

Um objeto de descompactação oferece suporte aos seguintes métodos:

Decompress.unused_data

Um objeto de bytes que contém todos os bytes após o final dos dados compactados. Ou seja, ele permanece `b""` até que o último byte que contém dados compactados esteja disponível. Se todo o bytestring contiver dados compactados, este será `b""`, um objeto de bytes vazio.

Decompress.unconsumed_tail

Um objeto de bytes que contém todos os dados que não foram consumidos pela última chamada `decompress()` porque excederam o limite dos dados não compactados no buffer. Esses dados ainda não foram vistos pela zlib, portanto, você deve alimentá-los (possivelmente

com outros dados concatenados a eles) em uma chamada subsequente para o método **`decompress()`** e, com isso, obter a saída correta.

Decompress.eof

Um booleano indicando se o fim do fluxo de dados compactados foi alcançado.

Isso permite distinguir entre um fluxo compactado formado corretamente e um fluxo incompleto ou truncado.

Adicionado na versão 3.3.

Decompress.decompress(*data*, *,* *max_length=0*)

Descompacta *data*, retornando um objeto de bytes que contém os dados não compactados correspondentes a pelo menos uma parte dos dados em *string*. Esses dados devem ser concatenados com a saída produzida por quaisquer chamadas anteriores ao método **`decompress()`** . Alguns dos dados de entrada podem ser preservados em buffers internos para processamento posterior.

If the optional parameter *max_length* is non-zero then the return value will be no longer than *max_length*. This may mean that not all of the compressed input can be processed; and unconsumed data will be stored in the attribute **`unconsumed_tail`**. This bytestring must be passed to a subsequent call to **`decompress()`** if decompression is to continue. If *max_length* is zero then the whole input is decompressed, and **`unconsumed_tail`** is empty. For example, the full content could be read like:

```
process_output(d.decompress(data, max_length))
while chunk := d.decompress(d.unconsumed_tail, max_length):
    process_output(chunk)
```

Alterado na versão 3.6: max_length pode ser usado como argumento nomeado.

Decompress.flush(*length=DEF_BUF_SIZE*, *,*)

Toda a entrada que estiver pendente é processada e um objeto de bytes contendo a saída descompactada restante é retornado. Depois de chamar `flush()`, o método `decompress()` não pode ser chamado novamente; a única ação possível é excluir o objeto.

O parâmetro opcional *comprimento* define o tamanho inicial da saída do buffer.

Decompress.copy()

Retorna uma cópia do objeto de descompactação. Isso pode ser usado para salvar o estado do descompactador no meio do fluxo de dados, a fim de acelerar as buscas aleatórias no fluxo em um ponto futuro.

Alterado na versão 3.8: As funções `copy.copy()` e `copy.deepcopy()` foram adicionadas como suporte para a objetos de descompactação.

As constantes a seguir estão disponíveis para configurar o comportamento de compactação e descompactação:

zlib.DEFLATED

O método de compactação de deflação.

zlib.MAX_WBITS

O tamanho máximo da janela, expresso como uma potência de 2. Por exemplo, se `MAX_WBITS` for `15`, isso resultará em um tamanho de janela de `32 KiB`.

zlib.DEF_MEM_LEVEL

O nível de memória padrão para objetos de compactação.

zlib.DEF_BUF_SIZE

O tamanho padrão do buffer para operações de descompactação.

zlib.Z_NO_COMPRESSION

Nível de compactação `0`; sem compactação.

Adicionado na versão 3.6.

zlib.Z_BEST_SPEED

Nível de compactação `1`; mais rápido e produz a menor compactação.

zlib.Z_BEST_COMPRESSION

Nível de compactação `9`; mais lento e produz a maior compactação.

zlib.Z_DEFAULT_COMPRESSION

Nível de compactação padrão (`-1`); um meio-termo entre velocidade e compactação. Atualmente equivalente ao nível de compactação `6`.

zlib.Z_DEFAULT_STRATEGY

Estratégia de compactação padrão, para dados normais.

zlib.Z_FILTERED

Estratégia de compactação para dados produzidos por um filtro (ou preditor).

zlib.Z_HUFFMAN_ONLY

Estratégia de compactação que força codificação Huffman apenas.

zlib.Z_RLE

Estratégia de compactação que limita distâncias de correspondência a um (codificação run-length).

Esta constante só estará disponível se o Python foi construído com zlib 1.2.0.1 ou superior.

Adicionado na versão 3.6.

zlib.Z_FIXED

Estratégia de compactação que impede o uso de códigos dinâmicos de Huffman.

Esta constante só estará disponível se o Python foi construído com zlib 1.2.2.2 ou superior.

Adicionado na versão 3.6.

zlib.Z_NO_FLUSH

Modo de saída `0`. Nenhum comportamento especial de saída.

Adicionado na versão 3.6.

zlib.Z_PARTIAL_FLUSH

Modo de saída `1`. Emite o máximo de saída possível.

zlib.Z_SYNC_FLUSH

Modo de saída `2`. Toda saída é emitida e é alinhada a um limite de bytes.

zlib.Z_FULL_FLUSH

Modo de saída `3`. Toda a saída é emitida e o estado de compactação é redefinido.

zlib.Z_FINISH

Modo de saída `4`. Toda entrada pendente é processada, nenhuma outra entrada é esperada.

zlib.Z_BLOCK

Modo de saída `5`. Um bloco de deflação é concluído e emitido.

Esta constante só estará disponível se o Python foi construído com zlib 1.2.2.2 ou superior.

Adicionado na versão 3.6.

zlib.Z_TREES

Modo de saída `6`, para operações de inflação. Instrui a inflação a retornar ao atingir o próximo limite do bloco de desinflação.

Esta constante está disponível apenas se o Python foi compilado com zlib 1.2.3.4 ou superior.

Adicionado na versão 3.6.

As informações sobre o versão da biblioteca zlib em uso estão disponíveis no seguinte constantes:

zlib.ZLIB_VERSION

Uma string com a versão da biblioteca zlib que foi usada para construir o módulo. Isso pode ser diferente da biblioteca zlib realmente usada no tempo de execução, que está disponível como `ZLIB_RUNTIME_VERSION`.

zlib.ZLIB_RUNTIME_VERSION

Uma string com a versão da biblioteca zlib atualmente utilizada pelo interpretador.

Adicionado na versão 3.3.

zlib.ZLIBNG_VERSION

A string de versão da biblioteca zlib-ng que foi usada para construir o módulo, caso o zlib-ng tenha sido usado. Quando presentes, as constantes `ZLIB_VERSION` e `ZLIB_RUNTIME_VERSION` refletem a versão da API zlib fornecida pelo zlib-ng.

Se zlib-ng não foi usado para construir o módulo, esta constante estará ausente.

Adicionado na versão 3.14.

Ver também

módulo `gzip`

Leia e escreva arquivos no formato **gzip**

https://www.zlib.net

A página inicial da biblioteca zlib.

https://www.zlib.net/manual.html

O manual da zlib explica a semântica e uso de diversas funções desta biblioteca.

Caso a (des)compactação do gzip seja um gargalo, o pacote python-isal [https://github.com/pycompression/python-isal] acelera a (des)compactação com uma API quase sempre compatível.

`gzip` — Suporte para arquivos gzip

Código-fonte: Lib/gzip.py [https://github.com/python/cpython/tree/3.15/Lib/gzip.py]

Esse módulo disponibiliza uma interface simples para comprimir e extrair arquivos como os programas GNU **gzip** e **gunzip** fazem.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

A compressão de dados é realizada pelo módulo `zlib`.

The `gzip` module provides the `GzipFile` class, as well as the `open()`, `compress()` and `decompress()` convenience functions. The `GzipFile` class reads and writes **gzip**-format files, automatically compressing or decompressing the data so that it looks like an ordinary file object.

Observe que formatos de arquivo adicionais que podem ser descompactados pelos programas **gzip** e **gunzip**, como aqueles produzidos por **compress** e **pack**, não são suportados por este módulo.

Este módulo define os seguintes itens:

gzip.open(*filename*, *mode='rb'*, *compresslevel=6*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo comprimido com gzip em modo binário ou texto, retornando objeto arquivo.

O argumento *filename* pode ser um nome de arquivo real (um objeto `str` ou `bytes`), ou um objeto arquivo existente para ler ou gravar.

O argumento *mode* pode ser qualquer um de `'r'`, `'rb'`, `'a'`, `'ab'`, `'w'`, `'wb'`, `'x'` ou `'xb'` para modo binário, ou `'rt'`, `'at'`, `'wt'` ou `'xt'` para modo texto. O padrão é `'rb'`.

O argumento *compresslevel* é um inteiro de 0 a 9, como para o construtor de **`GzipFile`**.

Para o modo binário, esta função equivale ao construtor de **`GzipFile`**: `GzipFile(filename, mode, compresslevel)`. Neste caso, os argumentos *encoding*, *errors* e *newline* não devem ser fornecidos.

Para o modo texto, um objeto **`GzipFile`** é criado e encapsulado em uma instância **`io.TextIOWrapper`** com a codificação especificada, comportamento de tratamento de erros e final(is) de linha.

Alterado na versão 3.3: Adicionado suporte para *filename* ser um objeto arquivo, suporte para modo texto e os argumentos *encoding*, *errors* e *newline*.

Alterado na versão 3.4: Adicionado suporte para os modos `'x'`, `'xb'` e `'xt'`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.15: The default compression level was reduced to 6 (down from 9). It is the default level used by most compression tools and a better tradeoff between speed and performance.

exception gzip.BadGzipFile

Uma exceção levantada para arquivos gzip inválidos. Herda de **`OSError`**. **`EOFError`** e **`zlib.error`** também podem ser levantadas para arquivos gzip inválidos.

Adicionado na versão 3.8.

class gzip.GzipFile(*filename=None*, *mode=None*, *compresslevel=6*, *fileobj=None*, *mtime=None*)

Construtor para a classe **`GzipFile`**, que simula a maioria dos métodos de um objeto arquivo, com exceção do método **`truncate()`**. Pelo menos um dos métodos *fileobj* e *filename* deve receber um valor não trivial.

A nova instância de classe é baseada em *fileobj*, que pode ser um arquivo comum, um objeto **`io.BytesIO`** ou qualquer outro objeto que simule um arquivo. O padrão é `None`, caso em que *filename* é aberto para fornecer um objeto arquivo.

Quando *fileobj* não é `None`, o argumento *filename* é usado apenas para ser incluído no cabeçalho do arquivo **gzip**, que pode incluir o nome original do arquivo descompactado. O padrão é o nome de arquivo *fileobj*, se discernível; caso contrário, o padrão é a string vazia e, neste caso, o nome original do arquivo não é incluído no cabeçalho.

O argumento *mode* pode ser qualquer um dos seguintes: `'r'`, `'rb'`, `'a'`, `'ab'`, `'w'`, `'wb'`, `'x'` ou `'xb'`, dependendo se o arquivo será lido ou escrito. O padrão é o modo de *fileobj*, se discernível; caso contrário, o padrão é `'rb'`. Em versões futuras do Python, o modo de *fileobj* não será usado. É melhor sempre especificar *mode* para escrita.

Observe que o arquivo é sempre aberto em modo binário. Para abrir um arquivo compactado em modo texto, use **`open()`** (ou envolva seu **`GzipFile`** com um **`io.TextIOWrapper`**).

O argumento *compresslevel* é um inteiro de `0` a `9` que controla o nível de compressão; `1` é o mais rápido e produz a menor compressão, e `9` é o mais lento e produz a maior compressão. `0` significa sem compressão. O padrão é `9`.

The optional *mtime* argument is the timestamp requested by gzip. The time is in Unix format, i.e., seconds since 00:00:00 UTC, January 1, 1970. Set *mtime* to `0` to generate a compressed stream that does not depend on creation time. If *mtime* is omitted or `None`, the current time is used; however, if the current time is outside the range 00:00:00 UTC, January 1, 1970 through 06:28:15 UTC, February 7, 2106, or explicitly passed *mtime* argument is outside the range `0` to `2**32-1`, then the value `0` is used instead.

Veja abaixo o atributo `mtime` que é definido durante a descompactação.

Chamar o método **`close()`** de um objeto `GzipFile` não fecha *fileobj*, pois você pode querer adicionar mais material após os dados compactados. Isso também permite que você passe um objeto `io.BytesIO` aberto para escrita como *fileobj* e recupere o buffer de memória resultante usando o método `getvalue()` do objeto **`io.BytesIO`**.

`GzipFile` suporta a interface `io.BufferedIOBase`, incluindo iteração e a instrução `with`. Apenas o método `truncate()` não é implementado.

`GzipFile` também disponibiliza os seguintes métodos e atributos:

peek(*n*)

Lê *n* bytes não compactados sem avançar a posição do arquivo. O número de bytes retornados pode ser maior ou menor que o solicitado.

Nota

Embora chamar `peek()` não altere a posição do arquivo `GzipFile`, ele pode alterar a posição do objeto arquivo subjacente (por exemplo, se o **`GzipFile`** foi construído com o parâmetro *fileobj*).

Adicionado na versão 3.2.

mode

`'rb'` para leitura e `'wb'` para escrita.

Alterado na versão 3.13: Em versões anteriores era um inteiro `1` ou `2`.

mtime

Ao descompactar, este atributo é definido como o último registro de data e hora no cabeçalho lido mais recentemente. É um inteiro que contém o número de segundos desde a era Unix (00:00:00 UTC, 1º

de janeiro de 1970). O valor inicial antes da leitura de qualquer cabeçalho é `None`.

name

O caminho para o arquivo gzip no disco, como `str` ou `bytes`. Equivalente à saída de `os.fspath()` no caminho de entrada original, sem nenhuma outra normalização, resolução ou expansão.

Alterado na versão 3.1: Foi adicionado suporte para a instrução `with`, juntamente com o argumento do construtor *mtime* e o atributo `mtime`.

Alterado na versão 3.2: Foi adicionado suporte para arquivos preenchidos com zeros e não pesquisáveis.

Alterado na versão 3.3: O método `io.BufferedIOBase.read1()` agora está implementado.

Alterado na versão 3.4: Adicionado suporte para os modos `'x'` e `'xb'`.

Alterado na versão 3.5: Adicionado suporte para escrever objetos bytes ou similares arbitrários. O método `read()` agora aceita o argumento `None`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Descontinuado desde a versão 3.9: Abrir `GzipFile` para escrita sem especificar o argumento *mode* está descontinuado.

Alterado na versão 3.12: Remove o atributo `filename` e usa o atributo `name`.

Alterado na versão 3.15: The default compression level was reduced to 6 (down from 9). It is the default level used by most compression tools and a better tradeoff between speed and performance.

gzip.compress(*data*, *compresslevel=6*, *, *mtime=0*)

Compacta *data*, retornando um objeto `bytes` contendo os dados compactados. *compresslevel* e *mtime* têm o mesmo significado que no

construtor de `GzipFile` acima, mas *mtime* assume o valor padrão 0 para uma saída reprodutível.

Adicionado na versão 3.2.

Alterado na versão 3.8: Adicionado o parâmetro *mtime* para saída reproduzível.

Alterado na versão 3.11: A velocidade é melhorada pela compactação de todos os dados de uma só vez, em vez de em fluxo contínuo. Chamadas com *mtime* definido como `0` são delegadas a `zlib.compress()` para maior velocidade. Nessa situação, a saída pode conter um valor de byte “OS” no cabeçalho gzip diferente de 255 “unknown”, conforme fornecido pela implementação subjacente do zlib.

Alterado na versão 3.13: O byte do cabeçalho gzip do sistema operacional tem a garantia de ser definido como 255 quando esta função é usada, como era o caso na versão 3.10 e anteriores.

Alterado na versão 3.14: O parâmetro *mtime* agora assume o valor padrão 0 para uma saída reprodutível. Para o comportamento anterior de usar a hora atual, passe `None` para *mtime*.

Alterado na versão 3.15: The default compression level was reduced to 6 (down from 9). It is the default level used by most compression tools and a better tradeoff between speed and performance.

gzip.decompress(*data*)

Descompacta *data*, retornando um objeto `bytes` contendo os dados descompactados. Esta função é capaz de descompactar dados de gzip de vários membros (vários blocos gzip concatenados). Quando é certo que os dados contêm apenas um membro, a função `zlib.decompress()` com *wbits* definido como 31 é mais rápida.

Adicionado na versão 3.2.

Alterado na versão 3.11: A velocidade é melhorada ao descompactar os membros de uma só vez na memória, em vez de fazê-lo de forma

contínua.

Exemplos de uso

Exemplo de como ler um arquivo comprimido:

```
import gzip
with gzip.open('/home/joe/arquivo.txt.gz', 'rb') as f:
    file_content = f.read()
```

Exemplo de como criar um arquivo comprimido com GZIP:

```
import gzip
content = b"Muito conteúdo aqui"
with gzip.open('/home/joe/arquivo.txt.gz', 'wb') as f:
    f.write(content)
```

Exemplo de como comprimir um arquivo existente com GZIP:

```
import gzip
import shutil
with open('/home/joe/arquivo.txt', 'rb') as f_in:
    with gzip.open('/home/joe/arquivo.txt.gz', 'wb') as f_out:
        shutil.copyfileobj(f_in, f_out)
```

Exemplo de como comprimir uma string binária com compressão GZIP:

```
import gzip
s_in = b"Muito conteúdo aqui"
s_out = gzip.compress(s_in)
```

Ver também

Módulo `zlib`

O módulo básico de compactação de dados necessário para dar suporte ao formato de arquivo do **gzip**.

Caso a (des)compressão do gzip seja um gargalo, o pacote python-isal [https://github.com/pycompression/python-isal] acelera a (des)compressão com uma API quase sempre compatível.

Interface de linha de comando

The **`gzip`** module provides a simple command line interface to compress or decompress files.

Once executed the **`gzip`** module keeps the input file(s).

Alterado na versão 3.8: Adiciona uma nova interface de linha de comando com um mensagem de uso. Por padrão, ao executar a CLI, o nível de compactação padrão é 6.

Opções de linha de comando

file
: Se o *arquivo* não for especificado, ler de `sys.stdin`

--fast
: Indica o método mais rápido de compressão (menor compressão)

--best
: Indica o método mais lento de compressão (melhor compressão).

-d, --decompress
: Descompacta o arquivo dado.

-h, --help
: Exibe a mensagem de ajuda.

`bz2` — Suporte para compressão bzip2

Código-fonte: Lib/bz2.py [https://github.com/python/cpython/tree/3.15/Lib/bz2.py]

Este módulo fornece uma interface abrangente para compactar e descompactar dados usando o algoritmo de compactação bzip2.

O módulo **`bz2`** contém:

- A função **`open()`** e a classe **`BZ2File`** para leitura e escrita de arquivos compactados.
- As classes **`BZ2Compressor`** e **`BZ2Decompressor`** para (des)compressão incremental.
- As funções **`compress()`** e **`decompress()`** para (des)compressão de uma só vez.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

(Des)compressão de arquivos

bz2.open(*filename*, *mode='rb'*, *compresslevel=9*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo compactado com bzip2 no modo binário ou texto, retornando um objeto arquivo.

Assim como no construtor para **`BZ2File`**, o argumento *filename* pode ser um nome de arquivo real (um objeto **`str`** ou **`bytes`**), ou um objeto arquivo existente para ler ou gravar.

O argumento *mode* pode ser qualquer um de `'r'`, `'rb'`, `'w'`, `'wb'`, `'x'`, `'xb'`, `'a'` ou `'ab'` para modo binário, ou `'rt'`, `'wt'`, `'xt'` ou `'at'` para modo texto. O padrão é `'rb'`.

O argumento *compresslevel* é um inteiro de 1 a 9, como para o construtor **`BZ2File`**.

Para o modo binário, esta função é equivalente ao construtor de **`BZ2File`**: `BZ2File(filename, mode, compresslevel=compresslevel)`. Neste caso, os argumentos *encoding*, *errors* e *newline* não devem ser fornecidos.

Para o modo texto, um objeto **`BZ2File`** é criado e envolto em uma instância **`io.TextIOWrapper`** com a codificação especificada, comportamento de tratamento de erros e final(is) de linha.

Adicionado na versão 3.3.

Alterado na versão 3.4: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

class bz2.BZ2File(*filename*, *mode='r'*, *, *compresslevel=9*)

Abre um arquivo compactado com bzip2 no modo binário.

Se *filename* for um objeto **`str`** ou **`bytes`**, abra o arquivo nomeado diretamente. Caso contrário, *filename* deve ser um objeto arquivo, que será usado para ler ou gravar os dados compactados.

O argumento *mode* pode ser `'r'` para leitura (padrão), `'w'` para substituição, `'x'` para criação exclusiva ou `'a'` para anexar. Estes podem ser equivalentemente dados como `'rb'`, `'wb'`, `'xb'` e `'ab'` respectivamente.

Se *filename* for um objeto arquivo (ao invés de um nome de arquivo real), um modo de `'w'` não truncará o arquivo e será equivalente a `'a'`.

Se *mode* for `'w'` ou `'a'`, *compresslevel* pode ser um inteiro entre `1` e `9` especificando o nível de compressão: `1` produz a menor compressão e `9` (padrão) produz a maior compactação.

Se *mode* for `'r'`, o arquivo de entrada pode ser a concatenação de vários fluxos compactados.

`BZ2File` fornece todos os membros especificados pelo **`io.BufferedIOBase`**, exceto **`detach()`** e **`truncate()`**. Iteração e a instrução **`with`** são suportadas.

`BZ2File` também fornece os seguintes métodos e atributos:

peek([*n*])

Retorna dados armazenados em buffer sem avançar a posição do arquivo. Pelo menos um byte de dados será retornado (a menos que em EOF). O número exato de bytes retornados não é especificado.

Nota

Enquanto chamar **`peek()`** não altera a posição do arquivo de **`BZ2File`**, pode alterar a posição do objeto arquivo subjacente (por exemplo, se o **`BZ2File`** foi construído passando um objeto arquivo para *filename*).

Adicionado na versão 3.3.

fileno()

Retorna o endereço descritor de arquivo do arquivo subjacente.

Adicionado na versão 3.3.

readable()

Retorna se o arquivo foi aberto para leitura.

Adicionado na versão 3.3.

seekable()

Retorna se o arquivo suporta a busca.

Adicionado na versão 3.3.

writable()

Retorna se o arquivo foi aberto para gravação.

Adicionado na versão 3.3.

read1(*size=-1*)

Lê até o tamanho *size* de bytes não compactados, tentando evitar várias leituras do fluxo subjacente. Lê até um valor buffer de dados se o tamanho for negativo.

Retorna `b''` se o arquivo tiver atingido EOF, ou seja, o fim do arquivo.

Adicionado na versão 3.3.

readinto(*b*)

Lê bytes para *b*.

Retorna o número de bytes lidos (0 para EOF).

Adicionado na versão 3.3.

mode

`'rb'` para leitura e `'wb'` para escrita.

Adicionado na versão 3.13.

name

O nome do arquivo bzip2. Equivalente ao atributo `name` do objeto arquivo subjacente.

Adicionado na versão 3.13.

Alterado na versão 3.1: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.3: Foi adicionado suporte para *filename* ser um objeto arquivo em vez de um nome de arquivo real.

O modo `'a'` (anexar) foi adicionado, juntamente com suporte para leitura de arquivos multifluxo.

Alterado na versão 3.4: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.5: O método `read()` agora aceita um argumento de `None`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.9: O parâmetro *buffering* foi removido. Foi ignorado e descontinuado desde o Python 3.0. Passe um objeto arquivo aberto para controlar como o arquivo é aberto.

O parâmetro *compresslevel* tornou-se somente-nomeado.

Alterado na versão 3.10: Esta classe não é segura para threads diante de vários leitores ou escritores simultâneos, assim como suas classes equivalentes em `gzip` e `lzma` sempre foram.

(Des)compressão incremental

class bz2.BZ2Compressor(*compresslevel=9*)

Cria um novo objeto compressor. Este objeto pode ser usado para compactar dados de forma incremental. Para compactação única, use a função `compress()`.

compresslevel, se fornecido, deve ser um inteiro entre `1` e `9`. O padrão é `9`.

compress(*data*)

Fornece dados para o objeto compressor. Retorna um pedaço de dados compactados, se possível, ou uma string de bytes vazia, caso contrário.

Quando você terminar de fornecer dados ao compactador, chame o método `flush()` para finalizar o processo de compressão.

flush()

Finaliza o processo de compactação. Retorna os dados compactados deixados em buffers internos.

O objeto compactador não pode ser usado após a chamada deste método.

class bz2.BZ2Decompressor

Cria um novo objeto descompactador. Este objeto pode ser usado para descompactar dados de forma incremental. Para compactação única, use a função `decompress()`.

Nota

Esta classe não trata de forma transparente entradas contendo múltiplos fluxos compactados, ao contrário de `decompress()` e `BZ2File`. Se você precisar descompactar uma entrada multifluxo com `BZ2Decompressor`, você deve usar um novo descompactador para cada fluxo.

decompress(*data*, *max_length=-1*)

Descompacta dados *data* (um objeto bytes ou similar), retornando dados não compactados como bytes. Alguns dos *data* podem ser armazenados em buffer internamente, para uso em chamadas posteriores para `decompress()`. Os dados retornados devem ser concatenados com a saída de qualquer chamada anterior para `decompress()`.

Se *max_length* for não negativo, retornará no máximo *max_length* bytes de dados descompactados. Se este limite for atingido e mais

saída puder ser produzida, o atributo **needs_input** será definido como `False`. Neste caso, a próxima chamada para **decompress()** pode fornecer *data* como `b''` para obter mais saída.

Se todos os dados de entrada foram descompactados e retornados (seja porque era menor que *max_length* bytes, ou porque *max_length* era negativo), o atributo **needs_input** será definido como `True` .

A tentativa de descompactar os dados após o final do fluxo ser atingido gera um **EOFError**. Quaisquer dados encontrados após o final do fluxo são ignorados e salvos no atributo **unused_data**.

Alterado na versão 3.5: Adicionado o parâmetro *max_length*.

eof

`True` se o marcador de fim de fluxo foi atingido.

Adicionado na versão 3.3.

unused_data

Dados encontrados após o término do fluxo compactado.

Se este atributo for acessado antes do final do fluxo ser alcançado, seu valor será `b''`.

needs_input

`False` se o método **decompress()** puder fornecer mais dados descompactados antes de exigir uma nova entrada descompactada.

Adicionado na versão 3.5.

(De)compressão de uma só vez (one-shot)

bz2.compress(*data*, *compresslevel=9*)

Compacta *data*, um objeto bytes ou similar.

compresslevel, se fornecido, deve ser um inteiro entre `1` e `9`. O padrão é `9`.

Para compressão incremental, use um **`BZ2Compressor`**.

bz2.decompress(*data*)

Descompacta *data*, um objeto bytes ou similar.

Se *data* for a concatenação de vários fluxos compactados, descompacta todos os fluxos.

Para descompressão incremental, use um **`BZ2Decompressor`**.

Alterado na versão 3.3: Suporte para entradas multifluxo foi adicionado.

Exemplos de uso

A seguir estão alguns exemplos de uso típico do módulo **`bz2`**.

Usando **`compress()`** e **`decompress()`** para demonstrar a compactação de ida e volta:

```
>>> import bz2
>>> data = b"""\
... Donec rhoncus quis sapien sit amet molestie. Fusce scelerisc
... nec ullamcorper. Nam rutrum pretium placerat. Aliquam vel tr
... sit amet cursus ante. In interdum laoreet mi, sit amet ultri
... pulvinar a. Nam gravida euismod magna, non varius justo tinc
... Aliquam pharetra lacus non risus vehicula rutrum. Maecenas a
... felis. Pellentesque semper nunc sit amet nibh ullamcorper, a
... dolor luctus. Curabitur lacinia mi ornare consectetur vestib
>>> c = bz2.compress(data)
>>> len(data) / len(c)  # Data compression ratio
1.513595166163142
>>> d = bz2.decompress(c)
>>> data == d  # Check equality to original object after round-t
True
```

Usando **BZ2Compressor** para compressão incremental:

```
>>> import bz2
>>> def gen_data(chunks=10, chunksize=1000):
...     """Yield incremental blocks of chunksize bytes."""
...     for _ in range(chunks):
...         yield b"z" * chunksize
...
>>> comp = bz2.BZ2Compressor()
>>> out = b""
>>> for chunk in gen_data():
...     # Provide data to the compressor object
...     out = out + comp.compress(chunk)
...
>>> # Finish the compression process.  Call this once you have
>>> # finished providing data to the compressor.
>>> out = out + comp.flush()
```

O exemplo acima usa um fluxo de dados muito “não aleatório” (um fluxo de partes `b"z"`). Dados aleatórios tendem a compactar mal, enquanto dados ordenados e repetitivos geralmente produzem uma alta taxa de compactação.

Escrevendo e lendo um arquivo compactado com bzip2 no modo binário:

```
>>> import bz2
>>> data = b"""\
... Donec rhoncus quis sapien sit amet molestie. Fusce scelerisc
... nec ullamcorper. Nam rutrum pretium placerat. Aliquam vel tr
... sit amet cursus ante. In interdum laoreet mi, sit amet ultri
... pulvinar a. Nam gravida euismod magna, non varius justo tinc
... Aliquam pharetra lacus non risus vehicula rutrum. Maecenas a
... felis. Pellentesque semper nunc sit amet nibh ullamcorper, a
... dolor luctus. Curabitur lacinia mi ornare consectetur vestib
>>> with bz2.open("myfile.bz2", "wb") as f:
...     # Write compressed data to file
...     unused = f.write(data)
...
>>> with bz2.open("myfile.bz2", "rb") as f:
...     # Decompress data from file
...     content = f.read()
...
```

```
>>> content == data  # Check equality to original object after r
True
```

`lzma` — Compressão usando o algoritmo LZMA

Adicionado na versão 3.3.

Código-fonte: Lib/lzma.py [https://github.com/python/cpython/tree/3.15/Lib/lzma.py]

Este módulo fornece classes e funções de conveniência para comprimir e descomprimir dados usando o algoritmo de compressão LZMA. Também está incluída uma interface de arquivo que oferece suporte aos formatos de arquivo `.xz` e legado `.lzma` usados pelo utilitário **xz**, bem como fluxos brutos comprimidos.

A interface fornecida por este módulo é muito semelhante à do módulo **`bz2`**. Observe que **`LZMAFile`** e **`bz2.BZ2File`** *não* são seguro para thread, portanto, se você precisar usar uma única instância **`LZMAFile`** de vários threads, é necessário protegê-la com uma trava.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

exception lzma.LZMAError

Essa exceção é levantada quando ocorre um erro durante a compressão ou descompressão ou durante a inicialização do estado compressor/descompressor.

Lendo e escrevendo arquivos comprimidos

lzma.open(*filename*, *mode='rb'*, ***, *format=None*, *check=-1*, *preset=None*, *filters=None*, *encoding=None*, *errors=None*, *newline=None*)

Abre um arquivo comprimido com LZMA no modo binário ou texto, retornando um objeto arquivo.

O argumento *filename* pode ser um nome de arquivo real (dado como um objeto `str`, `bytes` ou caminho ou similar), neste caso o arquivo nomeado é aberto , ou pode ser um objeto arquivo existente para leitura ou escrita.

O argumento *mode* pode ser qualquer um de `"r"`, `"rb"`, `"w"`, `"wb"`, `"x"`, `"xb"`, `"a"` ou `"ab"` para modo binário, ou `"rt"`, `"wt"`, `"xt"` , ou `"at"` para o modo de texto. O padrão é `"rb"`.

Ao abrir um arquivo para leitura, os argumentos *format* e *filters* têm os mesmos significados que em `LZMADecompressor`. Neste caso, os argumentos *check* e *preset* não devem ser usados.

Ao abrir um arquivo para escrita, os argumentos *format*, *check*, *preset* e *filters* têm os mesmos significados que em `LZMACompressor`.

Para o modo binário, esta função é equivalente ao construtor `LZMAFile`: `LZMAFile(filename, mode, ...)`. Nesse caso, os argumentos *encoding*, *errors* e *newline* não devem ser fornecidos.

Para o modo texto, um objeto `LZMAFile` é criado e encapsulado em uma instância `io.TextIOWrapper` com a codificação especificada, comportamento de tratamento de erros e final(is) de linha.

Alterado na versão 3.4: Adicionado suporte para os modos `"x"`, `"xb"` e `"xt"`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

class lzma.LZMAFile(*filename=None*, *mode='r'*, ***, *format=None*, *check=-1*, *preset=None*, *filters=None*)

Abre um arquivo comprimido com LZMA no modo binário.

Um `LZMAFile` pode envolver um objeto arquivo já aberto, ou operar diretamente em um arquivo nomeado. O argumento *filename* especifica

o objeto arquivo a ser encapsulado ou o nome do arquivo a ser aberto (como um objeto `str`, `bytes` ou objeto caminho ou similar). Ao agrupar um objeto arquivo existente, o arquivo agrupado não será fechado quando o `LZMAFile` for fechado.

O argumento *mode* pode ser `"r"` para leitura (padrão), `"w"` para substituição, `"x"` para criação exclusiva ou `"a"` para anexar. Estes podem ser equivalentemente dados como `"rb"`, `"wb"`, `"xb"` e `"ab"` respectivamente.

Se *filename* for um objeto arquivo (em vez de um nome de arquivo real), um modo de `"w"` não truncará o arquivo e será equivalente a `"a"`.

Ao abrir um arquivo para leitura, o arquivo de entrada pode ser a concatenação de vários fluxos comprimidos separados. Estes são decodificados de forma transparente como um único fluxo lógico.

Ao abrir um arquivo para leitura, os argumentos *format* e *filters* têm os mesmos significados que em `LZMADecompressor`. Neste caso, os argumentos *check* e *preset* não devem ser usados.

Ao abrir um arquivo para escrita, os argumentos *format*, *check*, *preset* e *filters* têm os mesmos significados que em `LZMACompressor`.

`LZMAFile` suporta todos os membros especificados por `io.BufferedIOBase`, exceto `detach()` e `truncate()`. Iteração e a instrução `with` são suportadas.

O método e os atributos a seguir também são fornecidos:

peek(*size=-1*)

Retorna dados armazenados em buffer sem avançar a posição do arquivo. Pelo menos um byte de dados será retornado, a menos que o EOF tenha sido atingido. O número exato de bytes retornados não é especificado (o argumento *size* é ignorado).

Nota

Enquanto chamar **peek()** não altera a posição do arquivo de **LZMAFile**, pode alterar a posição do objeto arquivo subjacente (por exemplo, se o **LZMAFile** foi construído passando um objeto arquivo para *nome do arquivo*).

mode

`'rb'` para leitura e `'wb'` para escrita.

Adicionado na versão 3.13.

name

O nome do arquivo lzma. Equivalente ao atributo **name** do objeto arquivo subjacente.

Adicionado na versão 3.13.

Alterado na versão 3.4: Adicionado suporte para os modos `"x"` e `"xb"`.

Alterado na versão 3.5: O método **read()** agora aceita um argumento de `None`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Comprimindo e descomprimindo dados na memória

class lzma.LZMACompressor(*format=FORMAT_XZ*, *check=-1*, *preset=None*, *filters=None*)

Cria um objeto compressão, que pode ser usado para comprimir dados de forma incremental.

Para uma maneira mais conveniente de comprimir um único bloco de dados, consulte **compress()**.

The *format* argument specifies what container format should be used. Possible values are **FORMAT_XZ** (the default), **FORMAT_ALONE** and

`FORMAT_RAW`.

The *check* argument specifies the type of integrity check to include in the compressed data. This check is used when decompressing, to ensure that the data has not been corrupted. Possible values are `CHECK_NONE`, `CHECK_CRC32`, `CHECK_CRC64` (the default for `FORMAT_XZ`) and `CHECK_SHA256`.

Se a verificação especificada não for suportada, uma exceção `LZMAError` será levantada.

As configurações de compressão podem ser especificadas como um nível de compressão predefinido (com o argumento *preset*) ou em detalhes como uma cadeia de filtros personalizada (com o argumento *filters*).

O argumento *preset* (se fornecido) deve ser um inteiro entre `0` e `9` (inclusive), opcionalmente com OR com a constante `PRESET_EXTREME`. Se nem *preset* nem *filters* forem fornecidos, o comportamento padrão é usar `PRESET_DEFAULT` (nível predefinido `6`). Predefinições mais altas produzem uma saída menor, mas tornam o processo de compressão mais lento.

Nota

Além de consumir mais CPU, a compressão com predefinições mais altas também requer muito mais memória (e produz uma saída que precisa de mais memória para descompressão). Com a predefinição `9` por exemplo, a sobrecarga para um objeto `LZMACompressor` pode chegar a 800 MiB. Por esse motivo, geralmente é melhor ficar com a predefinição padrão.

O argumento *filters* (se fornecido) deve ser um especificador de cadeia de filtros. Veja Especificando cadeias de filtros personalizadas para detalhes.

compress(*data*)

Comprime *data* (um objeto **bytes**), retornando um objeto **bytes** contendo dados comprimidos para pelo menos parte da entrada. Alguns dos *data* podem ser armazenados internamente, para uso em chamadas posteriores para **compress()** e **flush()**. Os dados retornados devem ser concatenados com a saída de quaisquer chamadas anteriores para **compress()**.

flush()

Conclui o processo de compressão, retornando um objeto **bytes** contendo todos os dados armazenados nos buffers internos do compressor.

O compressor não pode ser usado após a chamada deste método.

class lzma.LZMADecompressor(*format=FORMAT_AUTO*, *memlimit=None*, *filters=None*)

Cria um objeto descompressor, que pode ser usado para descomprimir dados de forma incremental.

Para uma maneira mais conveniente de descomprimir um fluxo comprimido inteiro de uma só vez, consulte **decompress()**.

O argumento *format* especifica o formato do contêiner que deve ser usado. O padrão é **FORMAT_AUTO**, que pode descomprimir os arquivos `.xz` e `.lzma`. Outros valores possíveis são **FORMAT_XZ**, **FORMAT_ALONE** e **FORMAT_RAW**.

O argumento *memlimit* especifica um limite (em bytes) na quantidade de memória que o descompressor pode usar. Quando esse argumento é usado, a descompressão falhará com um **LZMAError** se não for possível descomprimir a entrada dentro do limite de memória fornecido.

O argumento *filters* especifica a cadeia de filtros que foi usada para criar o fluxo que está sendo descomprimido. Este argumento é necessário se *format* for **FORMAT_RAW**, mas não deve ser usado para outros formatos. Veja Especificando cadeias de filtros personalizadas para mais informações sobre cadeias de filtros.

Nota

Esta classe não manipula de forma transparente entradas contendo múltiplos fluxos comprimidos, diferentemente de `decompress()` e `LZMAFile`. Para descomprimir uma entrada multi-fluxo com `LZMADecompressor`, você deve criar um novo descompactador para cada fluxo.

decompress(*data*, *max_length=-1*)

Descomprime dados *data* (um objeto bytes ou similar), retornando dados não comprimidos como bytes. Alguns dos *data* podem ser armazenados em buffer internamente, para uso em chamadas posteriores para `decompress()`. Os dados retornados devem ser concatenados com a saída de qualquer chamada anterior para `decompress()`.

Se *max_length* for não negativo, retornará no máximo *max_length* bytes de dados descomprimidos. Se este limite for atingido e mais saída puder ser produzida, o atributo `needs_input` será definido como `False`. Neste caso, a próxima chamada para `decompress()` pode fornecer *data* como `b''` para obter mais saída.

Se todos os dados de entrada foram descomprimidos e retornados (seja porque era menor que *max_length* bytes, ou porque *max_length* era negativo), o atributo `needs_input` será definido como `True` .

A tentativa de descomprimir os dados após o final do fluxo ser atingido gera um `EOFError`. Quaisquer dados encontrados após o final do fluxo são ignorados e salvos no atributo `unused_data`.

Alterado na versão 3.5: Adicionado o parâmetro *max_length*.

check

O ID da verificação de integridade usada pelo fluxo de entrada. Pode ser `CHECK_UNKNOWN` até que o suficiente da entrada tenha sido

decodificado para determinar qual verificação de integridade ela usa.

eof

`True` se o marcador de fim de fluxo foi atingido.

unused_data

Dados encontrados após o término do fluxo comprimido.

Antes do final do fluxo ser alcançado, isso será `b""`.

needs_input

`False` se o método **`decompress()`** puder fornecer mais dados descomprimidos antes de exigir uma nova entrada não comprimida.

Adicionado na versão 3.5.

lzma.compress(*data*, *format=FORMAT_XZ*, *check=-1*, *preset=None*, *filters=None*)

Comprime *data* (um objeto **`bytes`**), retornando os dados comprimidos como um objeto **`bytes`**.

Veja **`LZMACompressor`** acima para uma descrição dos argumentos *format*, *check*, *preset* e *filters*.

lzma.decompress(*data*, *format=FORMAT_AUTO*, *memlimit=None*, *filters=None*)

Descomprime *data* (um objeto **`bytes`**), retornando os dados não comprimidos como um objeto **`bytes`**.

Se *data* forem a concatenação de vários fluxos comprimidos distintos, descomprime todos esses fluxos e retorna a concatenação dos resultados.

Veja **`LZMADecompressor`** acima para uma descrição dos argumentos *format*, *memlimit* e *filters*.

Diversos

lzma.is_check_supported(*check*)

Retorna `True` se a verificação de integridade fornecida for suportada neste sistema.

`CHECK_NONE` e **`CHECK_CRC32`** são sempre suportados. **`CHECK_CRC64`** e **`CHECK_SHA256`** podem não estar disponíveis se você estiver usando uma versão do **liblzma** que foi compilada com um conjunto de recursos limitado.

Especificando cadeias de filtros personalizadas

Um especificador de cadeia de filtros é uma sequência de dicionários, onde cada dicionário contém o ID e as opções para um único filtro. Cada dicionário deve conter a chave `"id"` e pode conter chaves adicionais para especificar opções dependentes do filtro. Os IDs de filtro válidos são os seguintes:

- Filtro Compression:
  - **`FILTER_LZMA1`** (para ser usado com **`FORMAT_ALONE`**)
  - **`FILTER_LZMA2`** (para ser utilizado com **`FORMAT_XZ`** and **`FORMAT_RAW`**)
- Filtro delta:
  - **`FILTER_DELTA`**
- Filtros Branch-Call-Jump (BCJ):
  - **`FILTER_X86`**
  - **`FILTER_IA64`**
  - **`FILTER_ARM`**
  - **`FILTER_ARMTHUMB`**
  - **`FILTER_POWERPC`**
  - **`FILTER_SPARC`**

Uma cadeia de filtros pode consistir em até 4 filtros e não pode estar vazia. O último filtro na cadeia deve ser um filtro de compressão e quaisquer outros filtros devem ser filtros delta ou BCJ.

Os filtros de compressão suportam as seguintes opções (especificadas como entradas adicionais no dicionário que representa o filtro):

- `preset`: Uma predefinição de compressão para usar como fonte de valores padrão para opções que não são especificadas explicitamente.
- `dict_size`: Tamanho do dicionário em bytes. Deve estar entre 4 KiB e 1,5 GiB (inclusive).
- `lc`: Número de bits de contexto literal.
- `lp`: Número de bits de posição literal. A soma `lc + lp` deve ser no máximo 4.
- `pb`: Número de bits de posição; deve ser no máximo 4.
- `mode`: **`MODE_FAST`** ou **`MODE_NORMAL`**.
- `nice_len`: O que deve ser considerado um “tamanho agradável” para uma correspondência. Deve ser 273 ou menos.
- `mf`: Qual localizador de correspondências usar – **`MF_HC3`**, **`MF_HC4`**, **`MF_BT2`**, **`MF_BT3`** ou **`MF_BT4`**.
- `depth`: Profundidade máxima de pesquisa usada pelo localizador de correspondências. 0 (padrão) significa selecionar automaticamente com base em outras opções de filtro.

O filtro delta armazena as diferenças entre bytes, produzindo uma entrada mais repetitiva para o compressor em certas circunstâncias. Ele suporta uma opção, `dist`. Isso indica a distância entre bytes a ser subtraída. O padrão é 1, ou seja, pega as diferenças entre bytes adjacentes.

Os filtros BCJ são destinados a serem aplicados ao código de máquina. Eles convertem ramificações relativas, chamadas e saltos no código para usar endereçamento absoluto, com o objetivo de aumentar a redundância que pode ser explorada pelo compressor. Esses filtros suportam uma opção, `start_offset`. Isso especifica o endereço que deve ser mapeado para o início dos dados de entrada. O padrão é 0.

Constantes

The following module-level constants are provided for use as the *format*, *check*, *preset* and *filters* arguments of the classes and functions above.

Container formats:

lzma.FORMAT_XZ

The `.xz` container format.

lzma.FORMAT_ALONE

The legacy `.lzma` container format. This format is more limited than `.xz` – it does not support integrity checks or multiple filters.

lzma.FORMAT_RAW

A raw data stream, not using any container format. This format specifier does not support integrity checks, and requires that you always specify a custom filter chain (for both compression and decompression). Additionally, data compressed in this manner cannot be decompressed using **`FORMAT_AUTO`**.

lzma.FORMAT_AUTO

Used for decompression only. The container format is detected automatically, so that both `.xz` and `.lzma` files can be decompressed.

Integrity checks:

lzma.CHECK_NONE

No integrity check. This is the default (and the only acceptable value) for **`FORMAT_ALONE`** and **`FORMAT_RAW`**.

lzma.CHECK_CRC32

A 32-bit Cyclic Redundancy Check.

lzma.CHECK_CRC64

A 64-bit Cyclic Redundancy Check. This is the default for **`FORMAT_XZ`**.

lzma.CHECK_SHA256

A 256-bit Secure Hash Algorithm.

lzma.CHECK_UNKNOWN

The integrity check used by a stream could not yet be determined. This may be the value of the `LZMADecompressor.check` attribute until enough of the input has been decoded.

lzma.CHECK_ID_MAX

The largest supported integrity-check ID.

Compression presets:

lzma.PRESET_DEFAULT

The default compression preset, equivalent to preset level `6`.

lzma.PRESET_EXTREME

A flag that may be bitwise OR-ed with a preset level (`0` to `9`) to select a slower but more thorough variant of that preset.

Filter IDs and options:

lzma.FILTER_LZMA1
lzma.FILTER_LZMA2

The LZMA1 and LZMA2 compression filters. `FILTER_LZMA1` is for use with `FORMAT_ALONE`, while `FILTER_LZMA2` is for use with `FORMAT_XZ` and `FORMAT_RAW`.

lzma.FILTER_DELTA

The delta filter.

lzma.MODE_FAST
lzma.MODE_NORMAL

Compression modes that may be used as the `mode` option of a filter specifier (see Especificando cadeias de filtros personalizadas).

lzma.MF_HC3
lzma.MF_HC4
lzma.MF_BT2
lzma.MF_BT3
lzma.MF_BT4

Match finders that may be used as the `mf` option of a filter specifier (see Especificando cadeias de filtros personalizadas).

Exemplos

Lendo em um arquivo comprimido:

```
import lzma
with lzma.open("file.xz") as f:
    file_content = f.read()
```

Criando um arquivo comprimido:

```
import lzma
data = b"Insira dados aqui"
with lzma.open("file.xz", "w") as f:
    f.write(data)
```

Comprimindo dados na memória:

```
import lzma
data_in = b"Insert Data Here"
data_out = lzma.compress(data_in)
```

Compressão incremental:

```
import lzma
lzc = lzma.LZMACompressor()
out1 = lzc.compress(b"Alguns dados\n")
out2 = lzc.compress(b"Outro pedaço de dados\n")
out3 = lzc.compress(b"Ainda mais dados\n")
out4 = lzc.flush()
# Concatena todos os resultados parciais:
result = b"".join([out1, out2, out3, out4])
```

Escrevendo dados comprimidos em um arquivo já aberto:

```
import lzma
with open("file.xz", "wb") as f:
    f.write(b"Este dados serão comprimidos\n")
    with lzma.open(f, "w") as lzf:
```

```
        lzf.write(b"Isso *vai* ser comprimidos\n")
    f.write(b"Não comprimidos\n")
```

Criando um arquivo comprimido usando uma cadeia de filtros personalizada:

```
import lzma
my_filters = [
    {"id": lzma.FILTER_DELTA, "dist": 5},
    {"id": lzma.FILTER_LZMA2, "preset": 7 | lzma.PRESET_EXTREME}
]
with lzma.open("file.xz", "w", filters=my_filters) as f:
    f.write(b"blá blá blá")
```

`zipfile` — Trabalha com arquivos ZIP

Código-fonte: Lib/zipfile/ [https://github.com/python/cpython/tree/3.15/Lib/zipfile/]

O formato de arquivo ZIP é um padrão de compactação e arquivamento. Este módulo fornece ferramentas para criar, ler, escrever, adicionar, e listar um arquivo ZIP. Qualquer uso avançado deste módulo vai exigir um entendimento do formato, como definido nas Notas de Aplicação do PKZIP [https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT].

Esse módulo não trata arquivos ZIP multiparte. Ele pode manipular arquivos ZIP que usam as extensões ZIP64 (ou seja arquivos ZIP com tamanho maior do que 4 Gb). Ele suporta descriptografia de arquivos criptografados dentro do ZIP, mas atualmente não pode criar um arquivo criptografado. A descriptografia é extremamente lenta pois é implementada em Python nativo ao invés de C.

Lidar com arquivos compactados requer módulos opcionais. tal como `zlib`, `bz2`, `lzma` e `compression.zstd`. Se algumo desses estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Este módulo define os seguintes itens:

exception zipfile.BadZipFile

Este erro é levantado para arquivos ZIP corrompidos.

Adicionado na versão 3.2.

exception zipfile.BadZipfile

Alias para **BadZipFile**, para compatibilidade com versões mais antigas de Python.

Descontinuado desde a versão 3.2.

exception zipfile.LargeZipFile

Este erro é levantado quando um arquivo ZIP precisa da funcionalidade ZIP64 que não está habilitada.

class zipfile.ZipFile

A classe para ler e escrever arquivos ZIP. Veja a seção Objetos ZipFile para detalhes do construtor.

class zipfile.Path

Classe que implementa um subconjunto da interface fornecida por **pathlib.Path**, incluindo a interface completa **importlib.resources.abc.Traversable**.

Adicionado na versão 3.8.

class zipfile.PyZipFile

Classe para criar arquivos ZIP contendo bibliotecas Python.

class zipfile.ZipInfo(*filename='NoName', date_time=(1980, 1, 1, 0, 0, 0)*)

Class used to represent information about a member of an archive. Instances of this class are returned by the **getinfo()** and **infolist()** methods of **ZipFile** objects. Most users of the **zipfile** module will not need to create these, but only use those created by this module. *filename* should be the full name of the archive member, and *date_time* should be a tuple containing six fields which describe the time of the last modification to the file; the fields are described in section Objetos ZipInfo.

Alterado na versão 3.13: Um atributo público **compress_level** foi adicionado para expor o **_compresslevel** anteriormente protegido. O nome protegido mais antigo continua a funcionar como uma propriedade para compatibilidade com versões anteriores.

_for_archive(*archive*)

Resolve os atributos date_time, compression e external para padrões adequados, conforme usado por `ZipFile.writestr()`.

Retorna self para encadeamento.

Adicionado na versão 3.14.

zipfile.is_zipfile(*filename*)

Retorna `True` se *filename* é um arquivo ZIP válido baseado no seu "magic number", caso contrário retorna `False`. *filename* pode ser um arquivo ou um objeto arquivo ou similar também.

Alterado na versão 3.1: Suporte para arquivo e objetos arquivo ou similares.

zipfile.ZIP_STORED

Código numérico para um membro de um arquivo descompactado

zipfile.ZIP_DEFLATED

Código numérico para o método de compactação usual. Requer o módulo `zlib` .

zipfile.ZIP_BZIP2

Código numérico para o método de compactação BZIP2. Requer o módulo `bz2`.

Adicionado na versão 3.3.

zipfile.ZIP_LZMA

Código numérico para o método de compactação LZMA. Requer o módulo `lzma`.

Adicionado na versão 3.3.

zipfile.ZIP_ZSTANDARD

Código numérico para a compactação com Zstandard. Requer o módulo `compression.zstd`.

Nota

No APPNOTE 6.3.7, o ID do método `20` foi atribuído à compactação Zstandard. No APPNOTE 6.3.8, esse ID foi alterado para o ID do método `93` para evitar conflitos, com o ID do método `20` sendo descontinuado. Para compatibilidade, o módulo **`zipfile`** lê ambos os IDs de método, mas gravará apenas dados com o ID do método `93`.

Adicionado na versão 3.14.

Nota

A especificação do formato ZIP incluiu suporte para compactação bzip2 desde 2001, para compactação LZMA desde 2006 e para compactação Zstandard desde 2020. Porém, algumas ferramentas (incluindo versões mais antigas de Python) não suportam esses métodos de compactação, e podem recusar processar o arquivo ZIP como um todo, ou falhar em extrair arquivos individuais.

Ver também

Notas da Aplicação do PKZIP
[https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT]

Documentação do formato de arquivo ZIP feita por Phil Katz, criador do formato e dos algoritmos usados.

Site do Info-ZIP **[https://infozip.sourceforge.net/]**

Informações sobre o programas de arquivamento e desenvolvimento de bibliotecas do projeto Info-ZIP.

Objetos ZipFile

class zipfile.ZipFile(*file*, *mode='r'*, *compression=ZIP_STORED*, *allowZip64=True*, *compresslevel=None*, *, *strict_timestamps=True*, *metadata_encoding=None*)

Abre um arquivo ZIP, onde *file* pode ser um caminho para um arquivo (uma string), um objeto arquivo ou similar, ou um objeto caminho ou similar.

O parâmetro *mode* deve ser `'r'` para ler um arquivo existente, `'w'` para truncar e gravar um novo arquivo, `'a'` para adicionar a um arquivo existente, ou `'x'` exclusivamente para criar e gravar um novo arquivo. Se o *mode* é `'x'` e *file* se refere a um arquivo existente, um **`FileExistsError`** vai ser levantado. Se o *mode* é `'a'` e *file* se refere a um arquivo ZIP existente, então arquivos adicionais são adicionados ao mesmo. Se *file* não se refere a um arquivo ZIP, então um novo arquivo ZIP é adicionado ao arquivo. Isso diz respeito a adicionar um arquivo ZIP a um outro arquivo (como por exemplo `python.exe`). Se o *mode* é `'a'` e o arquivo não existe, ele será criado. Se o *mode* é `'r'` ou `'a'`, o arquivo deve ser percorrível.

compression é o método de compactação ZIP para usar ao escrever o arquivo, e deve ser **`ZIP_STORED`**, **`ZIP_DEFLATED`**, **`ZIP_BZIP2`**, **`ZIP_LZMA`** e **`ZIP_ZSTANDARD`**; valores desconhecidos devem causar **`NotImplementedError`** ser levantada. Se **`ZIP_DEFLATED`**, **`ZIP_BZIP2`**, **`ZIP_LZMA`** ou **`ZIP_ZSTANDARD`** for especificado mas o módulo correspondente (**`zlib`**, **`bz2`**, **`lzma`** e **`compression.zstd`**) não estiver disponível, é levantada uma exceção **`RuntimeError`**. O valor padrão é **`ZIP_STORED`**.

If *allowZip64* is `True` (the default) zipfile will create ZIP files that use the ZIP64 extensions when the zipfile is larger than 4 GiB. If it is `false` **`zipfile`** will raise an exception when the ZIP file would require ZIP64 extensions.

O parâmetro *compresslevel* controla o nível de compactação para usar ao gravar no arquivo ZIP. Quando usado **`ZIP_STORED`** ou **`ZIP_LZMA`** não tem efeito. Quando usado **`ZIP_DEFLATED`** inteiros de `0` a `9` são aceitos (veja **`zlib`** para mais informações). Quando usado **`ZIP_BZIP2`** inteiros

de `1` a `9` são aceitos (veja **`bz2`** para mais informações). Ao usar **`ZIP_ZSTANDARD`**, os inteiros `-131072` a `22` são comumente aceitos (consulte **`CompressionParameter.compression_level`** para obter mais informações sobre como recuperar valores válidos e seus significados).

O argumento *strict_timestamps*, quando definido como `False`, permite compactar arquivos anteriores a 1980-01-01 com o custo de definir o carimbo de data/hora para 1980-01-01. Comportamento semelhante ocorre com arquivos mais recentes que 2107-12-31, o carimbo de data/hora também é definido como o limite.

Quando o modo é `'r'`, *metadata_encoding* pode ser definido como o nome de um codec, que será usado para decodificar metadados, como os nomes dos membros e comentários ZIP.

Se o arquivo é criado com modo `'w'`, `'x'` ou `'a'` e então **`closed()`** sem adicionar nada ao arquivo, a estrutura própria para um arquivo vazio será escrita no arquivo.

ZipFile também é um gerenciador de contexto e portanto suporta a instrução **`with`**. Neste exemplo, *myzip* é fechado ao final da execução da instrução **`with`** – mesmo que ocorra uma exceção:

```
with ZipFile('spam.zip', 'w') as myzip:
    myzip.write('eggs.txt')
```

Nota

metadata_encoding é uma configuração em toda a instância para o ZipFile. Não é possível definir isso em uma base por membro.

Este atributo é uma solução alternativa para implementações legadas que produzem arquivos com nomes na codificação da localidade atual ou página de código (principalmente no Windows). De acordo com o padrão .ZIP, a codificação dos metadados pode ser especificada como página de código IBM (padrão) ou UTF-8 por meio de um sinalizador no cabeçalho do arquivo. Esse sinalizador tem precedência sobre *metadata_encoding*, que é uma extensão específica do Python.

Alterado na versão 3.2: Adicionado o uso de `ZipFile` como um gerenciador de contexto.

Alterado na versão 3.3: Adicionado suporte para compactação `bzip2` e `lzma`.

Alterado na versão 3.4: Extensões ZIP64 são habilitadas por padrão.

Alterado na versão 3.5: Adicionado suporte para escrever em streams não percorríveis. Adicionado suporte ao modo `'x'`.

Alterado na versão 3.6: Anteriormente, um simples `RuntimeError` era levantado para valores de compactação desconhecidos.

Alterado na versão 3.6.2: O parâmetro *file* aceita um objeto caminho ou similar.

Alterado na versão 3.7: Adicionado o parâmetro *compresslevel*.

Alterado na versão 3.8: O parâmetro somente-nomeado *strict_timestamps*.

Alterado na versão 3.11: Adicionado suporte para especificar a codificação do nome do membro ao ler metadados no diretório e cabeçalhos de arquivo do arquivo zip.

ZipFile.close()

Fecha o arquivo. Você deve chamar `close()` antes de sair do seu programa ou registros essenciais não serão gravados.

ZipFile.getinfo(*name*)

Retorna um objeto `ZipInfo` com informações sobre o *name* do membro do arquivo. Chamar `getinfo()` para um nome não encontrado no arquivo levanta um `KeyError`.

ZipFile.infolist()

Retorna uma lista contendo um objeto `ZipInfo` para cada membro do arquivo. Os objetos estão na mesma ordem das entradas no arquivo ZIP

em disco se um arquivo existente foi aberto.

ZipFile.namelist()

Retorna uma lista de membros do arquivo por nome.

ZipFile.open(*name*, *mode='r'*, *pwd=None*, *, *force_zip64=False*)

Acessa um membro do arquivo como um objeto binário arquivo ou similar. *name* pode ser o nome de um arquivo membro ou um objeto `ZipInfo`. O parâmetro *mode*, se informado, deve ser `'r'` (valor padrão) or `'w'`. *pwd* é a senha usada para descriptografar arquivos ZIP criptografados como um objeto `bytes`.

`open()` também é um gerenciador de contexto e, portanto, suporta a instrução `with`:

```
with ZipFile('spam.zip') as myzip:
    with myzip.open('eggs.txt') as myfile:
        print(myfile.read())
```

Com *mode* `'r'` o objeto arquivo ou similar (`ZipExtFile`) é somente leitura e fornece os seguintes métodos: `read()`, `readline()`, `readlines()`, `seek()`, `tell()`, `__iter__()`, `__next__()`. Esses objetos podem operar independentemente do ZipFile.

Com `mode='w'`, é retornado um manipulador de arquivo, que suporta o método `write()`. Quando um manipulador de arquivo modificável é aberto, tentativas de ler ou gravar outros arquivos no arquivo ZIP levanta um `ValueError`.

Em ambos os casos o objeto arquivo ou similar também possui atributos **`name`**, que é equivalente ao nome de um arquivo dentro do arquivo, e **`mode`**, que é `'rb'` ou `'wb'` dependendo do modo de entrada.

Ao gravar um arquivo, se o tamanho do arquivo não é conhecido mas pode exceder 2 GiB, passe `force_zip64=True` para assegurar que o formato do header é capaz de suportar arquivos grandes. Se o tamanho do arquivo é conhecido, construa um objeto `ZipInfo` com `file_size` informado, então use-o como parâmetro *name*.

Nota

Os métodos `open()`, `read()` e `extract()` podem receber um nome de arquivo ou um objeto `ZipInfo`. Você vai gostar disso quando tentar ler um arquivo ZIP que contém membros com nomes duplicados.

Alterado na versão 3.6: Removido suporte ao `mode='U'`. Uso de `io.TextIOWrapper` para leitura de arquivos texto compactados em modo de novas linhas universais.

Alterado na versão 3.6: `ZipFile.open()` agora pode ser usado para escrever arquivos no aquivo compactado com a opção `mode='w'`.

Alterado na versão 3.6: Chama `open()` em um ZipFile fechado levanta um `ValueError`. Anteriormente, um `RuntimeError` era levantado.

Alterado na versão 3.13: Adicionados atributos **`name`** e **`mode`** para o objeto arquivo ou similar gravável. O valor do atributo **`mode`** para o objeto arquivo ou similar legível foi alterado de `'r'` para `'rb'`.

ZipFile.extract(*member*, *path=None*, *pwd=None*)

Extrai um membro do arquivo para o diretório atual; *member* deve ser o nome completo ou um objeto `ZipInfo`. A informação do arquivo é extraída com maior precisão possível. *path* especifica um outro diretório em que deve ser gravado. *member* pode ser um nome de arquivo ou um objeto **`ZipInfo`**. *pwd* é a senha usada para criptografar arquivos como um objeto `bytes`.

Retorna o caminho normalizado criado (um diretório ou novo arquivo).

Nota

Se um nome de arquivo membro é um caminho absoluto, o drive/UNC e (contra)barras no início serão removidos, por exemplo: `///foo/bar` se torna `foo/bar` no Unix, e `C:\foo\bar` vira `foo\bar` no Windows. E todos os componentes `"..."` no nome de um arquivo membro serão removidos, por exemplo: `../../foo../../ba..r` vira `foo../ba..r`.

No Windows caracteres ilegais (`:`, `<`, `>`, `|`, `"`, `?`, and `*`) são substituídos por underscore (_).

Alterado na versão 3.6: Chama `extract()` em um ZipFile fechado levanto um `ValueError`. Anteriormente, um `RuntimeError` era levantado.

Alterado na versão 3.6.2: O parâmetro *path* aceita um objeto caminho ou similar.

ZipFile.extractall(*path=None*, *members=None*, *pwd=None*)

Extrai todos os membros de um arquivo para o diretório atual. *path* especifica um diretório diferente para gravar os arquivos extraídos. *members* é opcional e deve ser um sub-conjunto da lista retornada por `namelist()`. *pwd* é uma senha usada para criptografar arquivos como um objeto `bytes`.

Aviso

Never extract archives from untrusted sources without prior inspection. It is possible that files are created outside of *path*, for example, members that have absolute filenames or filenames with “..” components. This module attempts to prevent that. See `extract()` note.

Alterado na versão 3.6: Chama `extractall()` em um ZipFile fechado levanta um `ValueError`. Anteriormente, um `RuntimeError` era levantado.

Alterado na versão 3.6.2: O parâmetro *path* aceita um objeto caminho ou similar.

ZipFile.printdir()

Imprime a tabela de conteúdos de um arquivo para `sys.stdout`.

ZipFile.setpassword(*pwd*)

Define *pwd* (um objeto **bytes**) como senha padrão para extrair arquivos criptografados.

ZipFile.read(*name*, *pwd=None*)

Retorna os bytes do arquivo *name* no arquivo compactado. *name* é o nome do arquivo no arquivo compactado, ou um objeto **ZipInfo**. O arquivo compactado deve estar aberto para leitura ou acréscimo. *pwd* é a senha usada para arquivos criptografados como um objeto **bytes** e, se especificada, substitui a senha padrão configurada com **setpassword()**. Chamar **read()** em um ZipFile que use um método de compactação diferente de **ZIP_STORED**, **ZIP_DEFLATED**, **ZIP_BZIP2**, **ZIP_LZMA** e **ZIP_ZSTANDARD** levanta um **NotImplementedError**. Um erro também é levantado se o módulo de compactação correspondente não está disponível.

Alterado na versão 3.6: Chama **read()** em um ZipFile fechado levanta um **ValueError**. Anteriormente, um **RuntimeError** era levantado.

ZipFile.testzip()

Lê todos os arquivos no arquivo compactado e verifica seus CRC's e cabeçalhos de arquivo. Retorna o nome do primeiro arquivo corrompido, or então retorna `None`.

Alterado na versão 3.6: Chama **testzip()** em um ZipFile fechado levanta um **ValueError**. Anteriormente, um **RuntimeError** era levantado.

ZipFile.write(*filename*, *arcname=None*, *compress_type=None*, *compresslevel=None*)

Grava o arquivo chamado *filename* no arquivo compactado, dando ao arquivo compactado o nome *arcname* (por padrão, este é o mesmo de *filename*, mas sem a letra do drive e com separadores removidos do início do nome). Se informado, *compress_type* sobrescreve o valor dado ao parâmetro *compression* do construtor para a nova entrada. Da mesma forma, *compresslevel* vai sobrescrever o construtor se informado. O arquivo compactado deve ser aberto em modo `'w'`, `'x'` ou `'a'`.

Nota

O padrão de arquivo ZIP historicamente não especificava uma codificação de metadados, mas recomendava fortemente o CP437 (a codificação original do PC da IBM) para interoperabilidade. Versões recentes permitem o uso de (somente) UTF-8. Neste módulo, o UTF-8 será automaticamente usado para escrever os nomes dos membros se eles contiverem caracteres não ASCII. Não é possível escrever nomes de membros em qualquer codificação que não seja ASCII ou UTF-8.

Nota

Nomes de arquivo compactado devem ser relativos a raiz do mesmo, isto é, não devem começar com um separador de caminho.

Nota

Se `arcname` (ou `filename`, se `arcname` não for informado) contém um byte nulo, o nome do arquivo no arquivo compactado será truncado no byte nulo.

Nota

Uma barra inicial no nome do arquivo pode fazer com que o arquivo seja impossível de abrir em alguns programas zip em sistemas Windows.

Alterado na versão 3.6: Chama `write()` em um ZipFile criado com modo `'r'` ou em um ZipFile fechado levanta um `ValueError`. Anteriormente, um `RuntimeError` era levantado

ZipFile.writestr(*zinfo_or_arcname*, *data*, *compress_type=None*, *compresslevel=None*)

Grava um arquivo no arquivo compactado. O conteúdo é *data*, que pode ser uma instância de `str` ou de `bytes`; Se é uma `str`, ela é encodada como UTF-8 primeiro. *zinfo_or_arcname* é o nome que será dado ao

arquivo no arquivo compactado, ou uma instância de **ZipInfo**. Se é uma instância, pelo menos o nome do arquivo, a data, e a hora devem ser informados. Se é um nome, a data e hora recebem a data e hora atual. O arquivo compactado deve ser aberto em modo `'w'`, `'x'` ou `'a'`.

Se informado, *compress_type* sobrescreve o valor do parâmetro *compression* do construtor para a nova entrada, ou no *zinfo_or_arcname* (se é uma instância de **ZipInfo**). Da mesma forma, *compresslevel* vai sobrescrever o construtor se informado.

Nota

Quando é passada uma instância de **ZipInfo** ou o parâmetro *zinfo_or_arcname*, o método de compactação usado será aquele especificado no *compress_type* da instância de **ZipInfo**. Por padrão, o construtor da classe **ZipInfo** seta este membro para **ZIP_STORED**.

Alterado na versão 3.2: O argumento *compress_type*.

Alterado na versão 3.6: Chama **writestr()** em um ZipFile criado com modo `'r'` ou em um ZipFile fechado levanta um **ValueError**. Anteriormente, um **RuntimeError** era levantado.

Alterado na versão 3.14: Now respects the **SOURCE_DATE_EPOCH** environment variable. If set, it uses this value as the modification timestamp for the file written into the ZIP archive, instead of using the current time.

ZipFile.mkdir(*zinfo_or_directory*, *mode=511*)

Cria um diretório dentro do arquivo. Se *zinfo_or_directory* for uma string, um diretório é criado dentro do arquivo com o modo especificado no argumento *mode*. No entanto, se *zinfo_or_directory* for uma instância **ZipInfo**, o argumento *mode* é ignorado.

O arquivo deve ser aberto com o modo `'w'`, `'x'` ou `'a'`.

Adicionado na versão 3.11.

Os seguintes atributos de dados também estão disponíveis:

ZipFile.filename

Nome do arquivo ZIP.

ZipFile.debug

O nível de saída de debug para usar. Pode ser setado de `0` (valor padrão, sem nenhuma saída) a `3` (com mais saída). A informação de debug é escrita em `sys.stdout`.

ZipFile.comment

O comentário associado ao arquivo ZIP como um objeto **`bytes`**. Se atribuir um comentário a uma instância **`ZipFile`** criada com o modo `'w'`, `'x'` ou `'a'`, não deve ser maior que 65535 bytes. Comentários mais longos do que isso serão truncados.

Objetos Path

class zipfile.Path(*root*, *at=''*)

Construir um objeto Path a partir de um arquivo zip `root` (que pode ser uma instância **`ZipFile`** ou `file` adequado para passar para o construtor **`ZipFile`**).

`at` especifica a localização deste caminho dentro do arquivo zip, por exemplo, “dir/arquivo.txt”, “dir/” ou “”. O padrão é a string vazia, indicando a raiz.

Nota

The **`Path`** class does not sanitize filenames within the ZIP archive. Unlike the **`ZipFile.extract()`** and **`ZipFile.extractall()`** methods, it is the caller’s responsibility to validate or sanitize filenames to prevent path traversal vulnerabilities (for example, absolute paths or paths with “..” components). When handling untrusted archives, consider resolving filenames using **`os.path.abspath()`** and checking against the target directory with **`os.path.commonpath()`**.

Objetos Path expõem os seguintes recursos de objetos `pathlib.Path`:

Objetos Path podem ser percorridos usando o operador `/` ou `joinpath`.

Path.name

O componente final do caminho.

Path.open(*mode='r', *, pwd, **)

Invoca `ZipFile.open()` no caminho atual. Permite a abertura para leitura ou escrita, texto ou binário através dos modos suportados: “r”, “w”, “rb”, “wb”. Argumentos posicionais e argumentos nomeados são passados para `io.TextIOWrapper` quando abertos como texto e ignorados caso contrário. `pwd` é o parâmetro `pwd` para **`ZipFile.open()`**.

Alterado na versão 3.9: Adicionado suporte para modos de texto e binários para aberto. O modo padrão agora é texto.

Alterado na versão 3.11.2: O parâmetro `encoding` pode ser fornecido como um argumento posicional sem causar um `TypeError`. Como poderia em 3.9. O código que precisa ser compatível com as versões 3.10 e 3.11 não corrigidas deve passar todos os argumentos de `io.TextIOWrapper`, incluindo `encoding`, como palavras reservadas.

Path.iterdir()

Enumera os filhos do diretório atual.

Path.is_dir()

Retorna `True` se o contexto atual fizer referência a um diretório.

Path.is_file()

Retorna `True` se o contexto atual fizer referência a um arquivo.

Path.is_symlink()

Retorna `True` se o contexto atual fizer referência a um link simbólico.

Adicionado na versão 3.12.

Alterado na versão 3.13: Anteriormente, `is_symlink` retornava `False` incondicionalmente.

Path.exists()

Retorna `True` se o contexto atual fizer referência a um arquivo ou diretório no arquivo zip.

Path.suffix

A última parte separada por pontos do componente final, se houver. Isso é comumente chamado de extensão de arquivo.

Adicionado na versão 3.11: Adicionada a propriedade **`Path.suffix`**.

Path.stem

O componente final do caminho, sem seu sufixo.

Adicionado na versão 3.11: Adicionada a propriedade **`Path.stem`**.

Path.suffixes

Uma lista dos sufixos do caminho, comumente chamados de extensões de arquivo.

Adicionado na versão 3.11: Adicionada a propriedade **`Path.suffixes`**.

Path.read_text(*, **)

Leia o arquivo atual como texto Unicode. Argumentos posicionais e argumentos nomeados são passados para **`io.TextIOWrapper`** (exceto `buffer`, que está implícito no contexto).

Alterado na versão 3.11.2: O parâmetro `encoding` pode ser fornecido como um argumento posicional sem causar um **`TypeError`**. Como poderia em 3.9. O código que precisa ser compatível com as versões 3.10 e 3.11 não corrigidas deve passar todos os argumentos de **`io.TextIOWrapper`**, incluindo `encoding`, como palavras reservadas.

Path.read_bytes()

Lê o arquivo atual como bytes.

Path.joinpath(**other*)

Retorna um novo objeto Path com cada um dos *outros* argumentos unidos. Os seguintes são equivalentes:

```
>>> Path(...).joinpath('child').joinpath('grandchild')
>>> Path(...).joinpath('child', 'grandchild')
>>> Path(...) / 'child' / 'grandchild'
```

Alterado na versão 3.10: Antes de 3.10, `joinpath` não estava documentado e aceitava exatamente um parâmetro.

O projeto zipp [https://pypi.org/project/zipp/] fornece backports da funcionalidade mais recente de objeto caminho para versões mais antigas do Pythons. Use `zipp.Path` internamente de `zipfile.Path` para acesso antecipado às alterações.

Objetos PyZipFile

O construtor **`PyZipFile`** usa os mesmos parâmetros que o construtor **`ZipFile`**, e um parâmetro adicional, *otimize*.

class zipfile.PyZipFile(*file*, *mode='r'*, *compression=ZIP_STORED*, *allowZip64=True*, *optimize=-1*)

Alterado na versão 3.2: Adicionado o parâmetro *optimize*.

Alterado na versão 3.4: Extensões ZIP64 são habilitadas por padrão.

As instâncias têm um método além daqueles dos objetos **`ZipFile`**:

writepy(*pathname*, *basename=''*, *filterfunc=None*)

Pesquisa por arquivos `*.py` e adiciona o arquivo correspondente ao arquivo.

Se o parâmetro *optimize* para **`PyZipFile`** não foi fornecido ou `-1`, o arquivo correspondente é um arquivo `*.pyc`, compilando se necessário.

Se o parâmetro *Optimize* para **PyZipFile** era `0`, `1` ou `2`, apenas arquivos com esse nível de otimização (ver **compile()**) são adicionados ao o arquivo, compilando se necessário.

Se *pathname* for um arquivo, o nome do arquivo deverá terminar com `.py`, e apenas o arquivo (`*.pyc` correspondente) será adicionado no nível superior (sem informações do caminho). Se *pathname* for um arquivo que não termine com `.py`, um **RuntimeError** será levantado. Se for um diretório, e o diretório não for um diretório de pacotes, todos os arquivos `*.pyc` serão adicionados no nível superior. Se o diretório for um diretório de pacotes, todos `*.pyc` serão adicionados sob o nome do pacote como um caminho de arquivo e, se algum subdiretório for um diretório de pacotes, todos serão adicionados recursivamente na ordem de classificação.

basename destina-se apenas a uso interno.

filterfunc, se fornecido, deve ser uma função que recebe um único argumento de string. Cada caminho será passado (incluindo cada caminho de arquivo completo individual) antes de ser adicionado ao arquivo. Se *filterfunc* retornar um valor falso, o caminho não será adicionado e, se for um diretório, seu conteúdo será ignorado. Por exemplo, se nossos arquivos de teste estão todos nos diretórios `test` ou começam com a string `test_`, podemos usar um *filterfunc* para excluí-los:

```
>>> zf = PyZipFile('myprog.zip')
>>> def notests(s):
...     fn = os.path.basename(s)
...     return (not (fn == 'test' or fn.startswith('test_
...
>>> zf.writepy('myprog', filterfunc=notests)
```

O método **writepy()** faz arquivos com nomes de arquivo como este:

```
string.pyc                    # Nome de nível superior
test/__init__.pyc             # Diretório do pacote
test/testall.pyc              # Módulo test.testall
test/bogus/__init__.pyc       # Diretório do subpacote
test/bogus/myfile.pyc         # Submódulo test.bogus.myfile
```

Alterado na versão 3.4: Adicionado o parâmetro *filterfunc*.

Alterado na versão 3.6.2: O parâmetro *pathname* aceita um objeto caminho ou similar.

Alterado na versão 3.7: A recursão classifica as entradas de diretório.

Objetos ZipInfo

Instâncias da classe `ZipInfo` são retornadas pelos métodos `getinfo()` e `infolist()` dos objetos `ZipFile`. Cada objeto armazena informações sobre um único membro do arquivo ZIP.

Existe um método de classe para fazer uma instância `ZipInfo` para um arquivo de sistema de arquivos:

classmethod ZipInfo.from_file(*filename*, *arcname=None*, *, *strict_timestamps=True*)

Constrói uma instância `ZipInfo` para um arquivo no sistema de arquivos, em preparação para adicioná-lo a um arquivo zip.

filename deve ser o caminho para um arquivo ou diretório no sistema de arquivos.

Se *arcname* for especificado, ele será usado como o nome dentro do arquivo. Se *arcname* não for especificado, o nome será igual a *filename*, mas com qualquer letra de unidade e separadores de caminho removidos.

O argumento *strict_timestamps*, quando definido como `False`, permite compactar arquivos anteriores a 1980-01-01 com o custo de definir o carimbo de data/hora para 1980-01-01. Comportamento semelhante ocorre com arquivos mais recentes que 2107-12-31, o carimbo de data/hora também é definido como o limite.

Adicionado na versão 3.6.

Alterado na versão 3.6.2: O parâmetro *filename* aceita um objeto caminho ou similar.

Alterado na versão 3.8: Adicionado o parâmetro somente-nomeado *strict_timestamps*.

As instâncias têm os seguintes métodos e atributos:

ZipInfo.is_dir()

Retorna `True` se este membro do arquivo for um diretório.

Isso usa o nome da entrada: os diretórios devem sempre terminar com `/`.

Adicionado na versão 3.6.

ZipInfo.filename

Nome do arquivo no pacote.

ZipInfo.date_time

A hora e a data da última modificação no membro do arquivo. Esta é uma tupla de seis valores que representam os campos “last [modified] file time” (última hora [de modificado] do arquivo) e “last [modified] file date” (última data [de modificação] do arquivo) do diretório central do arquivo ZIP.

A tupla contém:

Índice	Valor

Índice	Valor
`0`	Ano (>= 1980)
`1`	Mês (iniciado em 1)
`2`	Dia do mês (iniciado em 1)
`3`	Horas (iniciado em 0)
`4`	Minutos (base zero)
`5`	Segundos (iniciado em 0)

Nota

O formato ZIP oferece suporte a vários campos de registro de data e hora em diferentes locais (diretório central, campos extras para sistemas NTFS/UNIX, etc.). Este atributo retorna especificamente o registro de data e hora do diretório central. O formato de registro de data e hora do diretório central em arquivos ZIP não oferece suporte a registros de data e hora anteriores a 1980. Embora alguns formatos de campos extras (como registros de data e hora do UNIX) possam representar datas anteriores, este atributo retorna apenas o registro de data e hora do diretório central.

O registro de data e hora do diretório central é interpretado como representando a hora local, em vez da hora UTC, para corresponder ao comportamento de outras ferramentas zip.

ZipInfo.compress_type

Tipo de compressão do membro do pacote.

ZipInfo.comment

Comentário para o membro individual do pacote como um objeto `bytes`.

ZipInfo.extra

Dados do campo de expansão. O PKZIP Application Note [https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT] contém alguns comentários sobre a estrutura interna dos dados contidos neste objeto `bytes`.

ZipInfo.create_system

O sistema que criou o pacote ZIP.

ZipInfo.create_version

A versão do PKZIP que criou o pacote ZIP.

ZipInfo.extract_version

A versão do PKZIP necessária para extrair o pacote.

ZipInfo.reserved

Deve ser zero

ZipInfo.flag_bits

Bits de sinalizador do ZIP.

ZipInfo.volume

Número de volume do cabeçalho do arquivo.

ZipInfo.internal_attr

Atributos internos.

ZipInfo.external_attr

Atributos de arquivo externo.

ZipInfo.header_offset

Deslocamento de byte para o cabeçalho do arquivo.

ZipInfo.CRC

CRC-32 do arquivo não comprimido.

ZipInfo.compress_size

Tamanho dos dados comprimidos.

ZipInfo.file_size
Tamanho do arquivo não comprimido.

Interface de linha de comando

The **zipfile** module provides a simple command-line interface to interact with ZIP archives.

Se você deseja criar um novo arquivo ZIP, especifique seu nome após a opção **-c** e, em seguida, liste os nomes dos arquivos que devem ser incluídos:

```
$ python -m zipfile -c monty.zip spam.txt eggs.txt
```

Passar um diretório também é aceitável:

```
$ python -m zipfile -c monty.zip life-of-brian_1979/
```

Se você deseja extrair um arquivo ZIP para o diretório especificado, use a opção **-e**:

```
$ python -m zipfile -e monty.zip target-dir/
```

Para obter uma lista dos arquivos em um arquivo ZIP, use a opção **-l**:

```
$ python -m zipfile -l monty.zip
```

Opções de linha de comando

-l <zipfile>
--list <zipfile>
Lista arquivos em um arquivo zip.

-c <zipfile> <source1> ... <sourceN>
--create <zipfile> <source1> ... <sourceN>
Cria um arquivo zip a partir dos arquivos fonte.

-e <zipfile> <output_dir>
--extract <zipfile> <output_dir>

Extrai um arquivo zip para um diretório de destino.

-t <zipfile>
--test <zipfile>

Testa se o arquivo zip é válido ou não.

--metadata-encoding <encoding>

Especifica a codificação dos nomes dos membros para `-l`, `-e` e `-t`.

Adicionado na versão 3.11.

Armadilhas de descompressão

A extração no módulo zipfile pode falhar devido a algumas armadilhas listadas abaixo.

Do próprio arquivo

A descompactação pode falhar devido a senha / soma de verificação CRC / formato ZIP incorretos ou método de compactação / descriptografia não compatível.

Limitações do sistema de arquivos

Exceder as limitações em sistemas de arquivos diferentes pode causar falha na descompactação. Como caracteres permitidos nas entradas do diretório, comprimento do nome do arquivo, comprimento do caminho, tamanho de um único arquivo e número de arquivos, etc.

Limitações de recursos

A falta de memória ou volume de disco levaria a uma falha de descompactação. Por exemplo, bombas de descompressão (também conhecidas como ZIP bomb [https://en.wikipedia.org/wiki/Zip_bomb]) aplicam-se à

biblioteca de arquivos zip que podem causar o esgotamento do volume do disco.

Interrupção

A interrupção durante a descompressão, como pressionar Control-C ou interromper o processo de descompressão pode resultar na descompressão incompleta do arquivo.

Comportamentos padrão da extração

Não saber os comportamentos de extração padrão pode causar resultados de descompressão inesperados. Por exemplo, ao extrair o mesmo arquivo duas vezes, ele sobrescreve os arquivos sem perguntar.

`tarfile` — Ler e gravar arquivos do tipo tar

Código-fonte: Lib/tarfile.py [https://github.com/python/cpython/tree/3.15/Lib/tarfile.py]

The `tarfile` module makes it possible to read and write tar archives, including those using gzip, bz2 and lzma compression. Use the `zipfile` module to read or write `.zip` files, or the higher-level functions in shutil.

Alguns fatos e números:

- lê e grava arquivos compactados com `gzip`, `bz2`, `compression.zstd` e `lzma` se os respectivos módulos estiverem disponíveis.

  Se algum desses módulos opcionais estiverem faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

- suporte de leitura/gravação para o formato POSIX.1-1988 (ustar).

- suporte de leitura/gravação para o formato GNU tar, incluindo extensões *longname* e *longlink*, suporte somente leitura para todas as variantes da extensão *sparse*, incluindo restauração de arquivos esparsos.

- suporte de leitura/gravação para o formato POSIX.1-2001 (pax).

- manipula diretórios, arquivos regulares, links físicos, links simbólicos, fifos, dispositivos de caracteres e dispositivos de bloco e é capaz de adquirir e restaurar informações de arquivo, como registro de data e hora, permissões de acesso e proprietário.

Alterado na versão 3.3: Adicionado suporte para compactação `lzma`.

Alterado na versão 3.12: Os arquivos são extraídos usando um filtro, o que torna possível limitar recursos surpreendentes/perigosos ou reconhecer que eles são esperados e que o arquivo é totalmente confiável.

Alterado na versão 3.14: Define o filtro de extração padrão como **`data`**, o que desabilita alguns recursos perigosos, como links para caminhos absolutos ou caminhos fora do destino. Anteriormente, a estratégia de filtro era equivalente a **`fully_trusted`**.

Alterado na versão 3.14: Adicionado suporte para a compactação com Zstandard usando **`compression.zstd`**.

tarfile.open(*name=None*, *mode='r'*, *fileobj=None*, *bufsize=10240*, ***kwargs*)

Retorna um objeto **`TarFile`** para o caminho *name*. Para obter informações detalhadas sobre objetos **`TarFile`** e os argumentos nomeados permitidos, consulte Objetos TarFile.

mode deve ser uma string no formato `'filemode[:compression]'`, o padrão é `'r'`. Aqui está uma lista completa de combinações de modos:

modo	ação
`'r'` ou `'r:*'`	Abre para leitura com compactação transparente (recomendado).
`'r:'`	Abre para leitura exclusivamente sem compactação.
`'r:gz'`	Abre para leitura com compactação gzip.
`'r:bz2'`	Abre para leitura com compactação bzip2.
`'r:xz'`	Abre para leitura com compactação lzma.
`'r:zst'`	Abre para leitura com compactação Zstandard.

modo	ação
`'x'` ou `'x:'`	Cria um arquivo tar exclusivamente sem compactação. Levanta uma exceção `FileExistsError` se ele já existir.
`'x:gz'`	Cria um arquivo tar com compactação gzip. Levanta uma exceção `FileExistsError` se ele já existir.
`'x:bz2'`	Cria um arquivo tar com compactação bzip2. Levanta uma exceção `FileExistsError` se ele já existir.
`'x:xz'`	Cria um arquivo tar com compactação lzma. Levanta uma exceção `FileExistsError` se ele já existir.
`'x:zst'`	Cria um arquivo tar com compactação Zstandard. Levanta uma exceção `FileExistsError` se ele já existir.
`'a'` ou `'a:'`	Aberto para acrescentar conteúdo sem compactação. O arquivo é criado se não existir.
`'w'` ou `'w:'`	Abre para gravação descompactada.
`'w:gz'`	Abre para gravação compactada com gzip.
`'w:bz2'`	Abre para gravação compactada com bzip2.
`'w:xz'`	Abre para gravação compactada com lzma.
`'w:zst'`	Abre para gravação compactada com Zstandard.

Observe que `'a:gz'`, `'a:bz2'` ou `'a:xz'` não são possíveis. Se o *mode* não for adequado para abrir um determinado arquivo (compactado) para leitura, `ReadError` será levantada. Use o *mode* `'r'` para evitar isso. Se um método de compactação não for suportado, `CompressionError` será levantada.

Se *fileobj* for especificado, ele será usado como alternativa a um objeto arquivo aberto em modo binário para *name*. Ele deve estar na posição 0.

For modes `'w:gz'`, `'x:gz'`, `'w|gz'`, `'w:bz2'`, `'x:bz2'`, `'w|bz2'`, **`tarfile.open()`** accepts the keyword argument *compresslevel* (default `6`) to specify the compression level of the file.

Para os modos `'w:xz'`, `'x:xz'` e `'w|xz'`, **`tarfile.open()`** aceita o argumento nomeado *preset* para especificar o nível de compactação do arquivo.

Para os modos `'w:zst'`, `'x:zst'` e `'w|zst'`, **`tarfile.open()`** aceita o argumento nomeado *level* para especificar o nível de compactação do arquivo. O argumento nomeado *options* também pode ser passado, fornecendo parâmetros avançados de compactação Zstandard descritos por **`CompressionParameter`**. O argumento nomeado *zstd_dict* pode ser passado para fornecer um **`ZstdDict`**, um dicionário Zstandard usado para melhorar a compactação de pequenas quantidades de dados.

For modes `'w:gz'` and `'w|gz'`, **`tarfile.open()`** accepts the keyword argument *mtime* to create a gzip archive header with that mtime. By default, the mtime is set to the time of creation of the archive. Use *mtime* `0` to generate a compressed stream that does not depend on creation time, for reproducible output.

Para fins especiais, existe um segundo formato para *mode*: `'filemode|[compression]'`. **`tarfile.open()`** retornará um objeto **`TarFile`** que processa seus dados como um fluxo de blocos. Nenhuma busca aleatória será realizada no arquivo. Se fornecido, *fileobj* pode ser qualquer objeto que tenha um método **`read()`** ou **`write()`** (dependendo do *mode*) que funcione com bytes. *bufsize* especifica o tamanho do bloco e o padrão é `20 * 512` bytes. Use esta variante em combinação com, por exemplo, `sys.stdin.buffer`, um objeto arquivo de soquete ou um dispositivo de fita. No entanto, tal objeto **`TarFile`** é limitado, pois não permite acesso aleatório; consulte Exemplos. Os modos atualmente possíveis:

Modo	Ação

Modo	Ação
`'r|*'`	Abre um *stream* de blocos tar para leitura com compactação transparente.
`'r|'`	Abre um *stream* de blocos tar não compactados para leitura.
`'r|gz'`	Abre um *stream* compactado com gzip para leitura.
`'r|bz2'`	Abre um *stream* compactado com bzip2 para leitura.
`'r|xz'`	Abre um *stream* compactado com lzma para leitura.
`'r|zst'`	Abre um *stream* compactado com Zstandard para leitura.
`'w|'`	Abre um *stream* descompactado para gravação.
`'w|gz'`	Abre um *stream* compactado com gzip para gravação.
`'w|bz2'`	Abre um *stream* compactado com bzip2 para gravação.
`'w|xz'`	Abre um *stream* compactado com lzma para gravação.
`'w|zst'`	Abre um *stream* compactado com Zstandard para gravação.

Alterado na versão 3.5: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.6: O parâmetro *name* aceita um objeto caminho ou similar.

Alterado na versão 3.12: O argumento nomeado *compresslevel* também funciona para fluxos.

Alterado na versão 3.14: O argumento nomeado *preset* também funciona para fluxos.

Alterado na versão 3.15: The default compression level was reduced to 6 (down from 9). It is the default level used by most compression tools and a better tradeoff between speed and performance.

class tarfile.TarFile

Classe para ler e escrever arquivos tar. Não use esta classe diretamente: use `tarfile.open()` em vez disso. Veja Objetos TarFile.

tarfile.is_tarfile(*name*)

Return `True` if *name* is a tar archive file, that the `tarfile` module can read. *name* may be a `str`, file, or file-like object.

Alterado na versão 3.9: Suporte para arquivo e objetos arquivo ou similares.

The `tarfile` module defines the following exceptions:

exception tarfile.TarError

Base class for all `tarfile` exceptions.

exception tarfile.ReadError

Is raised when a tar archive is opened, that either cannot be handled by the `tarfile` module or is somehow invalid.

exception tarfile.CompressionError

É levantada quando um método de compactação não é suportado ou quando os dados não podem ser decodificados corretamente.

exception tarfile.StreamError

É levantada para as limitações típicas de objetos `TarFile` do tipo fluxo.

exception tarfile.ExtractError

É levantada para erros *não fatais* ao usar `TarFile.extract()`, mas somente se `TarFile.errorlevel`== `2`.

exception tarfile.HeaderError

É levantada por `TarInfo.frombuf()` se o buffer obtido for inválido.

exception tarfile.FilterError

Classe base para membros recusados por filtros.

tarinfo

Informações sobre o membro que o filtro se recusou a extrair, como TarInfo.

exception tarfile.AbsolutePathError

Levantada para recusar a extração de um membro com um caminho absoluto.

exception tarfile.OutsideDestinationError

Levantada para recusar a extração de um membro fora do diretório de destino.

exception tarfile.SpecialFileError

Levantada para recusar a extração de um arquivo especial (por exemplo, um dispositivo ou encadeamento).

exception tarfile.AbsoluteLinkError

Levantada para recusar a extração de um link simbólico com um caminho absoluto.

exception tarfile.LinkOutsideDestinationError

Levantada para recusar a extração de um link simbólico apontando para fora do diretório de destino.

exception tarfile.LinkFallbackError

Levantada para recusar a emulação de um link (físico ou simbólico) extraindo outro membro do arquivo, quando esse membro seria rejeitado pelo local do filtro. A exceção levantada para rejeitar o membro substituto está disponível como **`BaseException.__context__`**.

Adicionado na versão 3.15.

As seguintes constantes estão disponíveis a nível de módulo:

tarfile.ENCODING

A codificação de caracteres padrão: `'utf-8'` no Windows, o valor retornado por `sys.getfilesystemencoding()`, caso contrário.

tarfile.REGTYPE
tarfile.AREGTYPE

Um arquivo normal `type`.

tarfile.LNKTYPE

Um link (dentro do arquivo tar) `type`.

tarfile.SYMTYPE

Um link simbólico `type`.

tarfile.CHRTYPE

Um dispositivo especial de caracteres `type`.

tarfile.BLKTYPE

Um dispositivo especial de bloco `type`.

tarfile.DIRTYPE

Um diretório `type`.

tarfile.FIFOTYPE

Um dispositivo especial de FIFO `type`.

tarfile.CONTTYPE

Um arquivo contíguo `type`.

tarfile.GNUTYPE_LONGNAME

Um `type` de longname de GNU tar.

tarfile.GNUTYPE_LONGLINK

Um `type` de longlink de GNU tar.

tarfile.GNUTYPE_SPARSE

Um `type` de arquivo esparso de GNU tar.

Each of the following constants defines a tar archive format that the **`tarfile`** module is able to create. See section Formatos tar suportados for details.

tarfile.USTAR_FORMAT
: formato POSIX.1-1988 (ustar).

tarfile.GNU_FORMAT
: Formato tar GNU

tarfile.PAX_FORMAT
: formato POSIX.1-2001 (pax).

tarfile.DEFAULT_FORMAT
: O formato padrão para criação de arquivos. Atualmente, é `PAX_FORMAT`.

Alterado na versão 3.8: O formato padrão para novos arquivos foi alterado de `GNU_FORMAT` para `PAX_FORMAT`.

Ver também

Módulo `zipfile`
: Documentação do módulo padrão `zipfile`.

Operações de arquivamento
: Documentação das funcionalidades de arquivamento de nível mais alto fornecidas pelo módulo padrão `shutil`.

Manual do GNU tar, Formato Básico do Tar
[https://www.gnu.org/software/tar/manual/html_node/Standard.html]
: Documentação para arquivos tar, incluindo extensões GNU tar.

Objetos TarFile

O objeto `TarFile` fornece uma interface para um arquivo tar. Um arquivo tar é uma sequência de blocos. Um membro do arquivo (um arquivo

armazenado) é composto por um bloco de cabeçalho seguido por blocos de dados. É possível armazenar um arquivo em um arquivo tar várias vezes. Cada membro do arquivo é representado por um objeto **`TarInfo`**, veja Objetos TarInfo para mais detalhes.

Um objeto **`TarFile`** pode ser usado como gerenciador de contexto em uma instrução **`with`**. Ele será fechado automaticamente quando o bloco for concluído. Observe que, em caso de exceção, um arquivo aberto para gravação não será finalizado; apenas o objeto arquivo usado internamente será fechado. Consulte a seção Exemplos para um exemplo de uso.

Adicionado na versão 3.2: Adicionado suporte para o protocolo de gerenciamento de contexto.

class tarfile.TarFile(*name=None*, *mode='r'*, *fileobj=None*, *format=DEFAULT_FORMAT*, *tarinfo=TarInfo*, *dereference=False*, *ignore_zeros=False*, *encoding=ENCODING*, *errors='surrogateescape'*, *pax_headers=None*, *debug=0*, *errorlevel=1*, *stream=False*)

Todos os argumentos a seguir são opcionais e também podem ser acessados como atributos de instância.

name é o caminho do arquivo. *name* pode ser um objeto caminho ou similar. Pode ser omitido se *fileobj* for fornecido. Nesse caso, o atributo **`name`** do objeto arquivo será usado, se existir.

O parâmetro *mode* pode ser `'r'` para ler de um arquivo existente, `'a'` para adicionar dados a um arquivo existente, `'w'` para criar um novo arquivo sobrescrevendo um existente ou `'x'` para criar um novo arquivo somente se ele ainda não existir.

Se *fileobj* for fornecido, ele será usado para leitura ou gravação de dados. Se for possível determinar, *mode* será substituído pelo modo de *fileobj*. *fileobj* será usado a partir da posição 0.

Nota

fileobj não é fechado quando **`TarFile`** é fechado.

format controla o formato de gravação do arquivo. Deve ser uma das constantes **`USTAR_FORMAT`**, **`GNU_FORMAT`** ou **`PAX_FORMAT`** definidas no nível do módulo. Durante a leitura, o formato será detectado automaticamente, mesmo que diferentes formatos estejam presentes em um único arquivo.

O argumento *tarinfo* pode ser usado para substituir a classe padrão **`TarInfo`** por uma diferente.

Se *dereference* for **`False`**, adiciona links físicos e simbólicos ao arquivo. Se for **`True`**, adiciona o conteúdo dos arquivos de destino ao arquivo. Isso não tem efeito em sistemas que não oferecem suporte a links simbólicos.

Se *ignore_zeros* for **`False`**, trata um bloco vazio como o fim do arquivo. Se for **`True`**, ignora blocos vazios (e inválidos) e tenta obter o máximo de membros possível. Isso só é útil para ler arquivos concatenados ou corrompidos.

O parâmetro *debug* pode ser configurado de `0` (nenhuma mensagem de depuração) até `3` (todas as mensagens de depuração). As mensagens são gravadas em `sys.stderr`.

errorlevel controla como os erros de extração são tratados, veja **`o atributo correspondente`**.

Os argumentos *encoding* e *errors* definem a codificação de caracteres a ser usada para leitura ou gravação do arquivo e como os erros de conversão serão tratados. As configurações padrão funcionarão para a maioria dos usuários. Consulta a seção Problemas de Unicode para obter informações detalhadas.

O argumento *pax_headers* é um dicionário opcional de strings que será adicionado como um cabeçalho global pax se *format* for **`PAX_FORMAT`**.

Se *stream* estiver definido como **`True`**, então, durante a leitura do arquivo, as informações sobre os arquivos no arquivo não serão armazenadas em cache, economizando memória.

Alterado na versão 3.2: Usa `'surrogateescape'` como padrão para o argumento *errors*.

Alterado na versão 3.5: O modo `'x'` (criação exclusiva) foi adicionado.

Alterado na versão 3.6: O parâmetro *name* aceita um objeto caminho ou similar.

Alterado na versão 3.13: Adiciona o parâmetro *stream*.

classmethod TarFile.open(...)

Construtor alternativo. A função **`tarfile.open()`** é, na verdade, um atalho para este método de classe.

TarFile.getmember(*name*)

Retorna um objeto **`TarInfo`** para o membro *name*. Se *name* não for encontrado no arquivo, uma exceção **`KeyError`** será levantada.

Nota

Se um membro aparecer mais de uma vez no arquivo, presume-se que sua última ocorrência seja a versão mais atualizada.

TarFile.getmembers()

Retorna os membros do arquivo como uma lista de objetos **`TarInfo`**. A lista tem a mesma ordem que os membros no arquivo.

TarFile.getnames()

Retorna os membros como uma lista com seus nomes. A lista está na mesma ordem que a lista retornada por **`getmembers()`**.

TarFile.list(*verbose=True*, *, *members=None*)

Envia um índice para `sys.stdout`. Se *verbose* for **`False`**, apenas os nomes dos membros serão exibidos. Se for **`True`**, a saída será semelhante à do comando **ls -l**. Se o parâmetro opcional *members* for

fornecido, ele deverá ser um subconjunto da lista retornada pelo método `getmembers()`.

Alterado na versão 3.5: Adicionado o parâmetro *members*.

TarFile.next()

Retorna o próximo membro do arquivo como um objeto `TarInfo`, quando `TarFile` é aberto para leitura. Retorna `None` se não houver mais nenhum disponível.

TarFile.extractall(*path='.', members=None, *, numeric_owner=False, filter=None*)

Extract all members from the archive to the current working directory or directory *path*. If optional *members* is given, it must be a subset of the list returned by `getmembers()`. Directory information like owner, modification time and permissions are set after all members have been extracted. This is done to work around two problems: A directory's modification time is reset each time a file is created in it. And, if a directory's permissions do not allow writing, extracting files to it will fail.

If *numeric_owner* is `True`, the uid and gid numbers from the tarfile are used to set the owner/group for the extracted files. Otherwise, the named values from the tarfile are used.

The *filter* argument specifies how `members` are modified or rejected before extraction. See Filtros de extração for details. It is recommended to set this explicitly only if specific *tar* features are required, or as `filter='data'` to support Python versions with a less secure default (3.13 and lower).

Aviso

Never extract archives from untrusted sources without prior inspection.

Since Python 3.14, the default (`data`) will prevent the most dangerous security issues. However, it will not prevent *all* unintended or insecure

behavior. Read the Filtros de extração section for details.

Alterado na versão 3.5: Adicionado o parâmetro *numeric_owner*.

Alterado na versão 3.6: O parâmetro *path* aceita um objeto caminho ou similar.

Alterado na versão 3.12: Adicionado o parâmetro *filter*.

Alterado na versão 3.14: The *filter* parameter now defaults to `'data'`.

TarFile.extract(*member*, *path=''*, *set_attrs=True*, *, *numeric_owner=False*, *filter=None*)

Extract a member from the archive to the current working directory, using its full name. Its file information is extracted as accurately as possible. *member* may be a filename or a **`TarInfo`** object. You can specify a different directory using *path*. *path* may be a path-like object. File attributes (owner, mtime, mode) are set unless *set_attrs* is false.

The *numeric_owner* and *filter* arguments are the same as for **`extractall()`**.

Nota

The **`extract()`** method does not take care of several extraction issues. In most cases you should consider using the **`extractall()`** method.

Aviso

Never extract archives from untrusted sources without prior inspection. See the warning for **`extractall()`** for details.

Alterado na versão 3.2: Added the *set_attrs* parameter.

Alterado na versão 3.5: Adicionado o parâmetro *numeric_owner*.

Alterado na versão 3.6: O parâmetro *path* aceita um objeto caminho ou similar.

Alterado na versão 3.12: Adicionado o parâmetro *filter*.

TarFile.extractfile(*member*)

Extract a member from the archive as a file object. *member* may be a filename or a `TarInfo` object. If *member* is a regular file or a link, an `io.BufferedReader` object is returned. For all other existing members, `None` is returned. If *member* does not appear in the archive, `KeyError` is raised.

Alterado na versão 3.3: Return an `io.BufferedReader` object.

Alterado na versão 3.13: The returned `io.BufferedReader` object has the `mode` attribute which is always equal to `'rb'`.

TarFile.errorlevel*: int*

Se *errorlevel* for `0`, os erros serão ignorados ao usar `TarFile.extract()` e `TarFile.extractall()`. No entanto, eles aparecem como mensagens de erro na saída de depuração quando *debug* for maior que 0. Se `1` (o padrão), todos os erros *fatais* serão levantados como exceções `OSError` ou `FilterError`. Se `2`, todos os erros *não fatais* também serão levantados como exceções `TarError`.

Some exceptions, e.g. ones caused by wrong argument types or data corruption, are always raised.

Custom extraction filters should raise `FilterError` for *fatal* errors and `ExtractError` for *non-fatal* ones.

Note that when an exception is raised, the archive may be partially extracted. It is the user's responsibility to clean up.

TarFile.extraction_filter

Adicionado na versão 3.12.

O filtro de extração usado como um padrão para o argumento *filter* de `extract()` e `extractall()`.

The attribute may be `None` or a callable. String names are not allowed for this attribute, unlike the *filter* argument to `extract()`.

If `extraction_filter` is `None` (the default), extraction methods will use the `data` filter by default.

The attribute may be set on instances or overridden in subclasses. It also is possible to set it on the `TarFile` class itself to set a global default, although, since it affects all uses of *tarfile*, it is best practice to only do so in top-level applications or `site configuration`. To set a global default this way, a filter function needs to be wrapped in `@staticmethod` to prevent injection of a `self` argument.

Alterado na versão 3.14: The default filter is set to `data`, which disallows some dangerous features such as links to absolute paths or paths outside of the destination. Previously, the default was equivalent to `fully_trusted`.

TarFile.add(*name*, *arcname=None*, *recursive=True*, *, *filter=None*)

Add the file *name* to the archive. *name* may be any type of file (directory, fifo, symbolic link, etc.). If given, *arcname* specifies an alternative name for the file in the archive. Directories are added recursively by default. This can be avoided by setting *recursive* to `False`. Recursion adds entries in sorted order. If *filter* is given, it should be a function that takes a `TarInfo` object argument and returns the changed **`TarInfo`** object. If it instead returns `None` the **`TarInfo`** object will be excluded from the archive. See Exemplos for an example.

Alterado na versão 3.2: Adicionado o parâmetro *filter*.

Alterado na versão 3.7: Recursion adds entries in sorted order.

TarFile.addfile(*tarinfo*, *fileobj=None*)

Add the `TarInfo` object *tarinfo* to the archive. If *tarinfo* represents a non zero-size regular file, the *fileobj* argument should be a binary file,

and `tarinfo.size` bytes are read from it and added to the archive. You can create **`TarInfo`** objects directly, or by using **`gettarinfo()`**.

Alterado na versão 3.13: fileobj must be given for non-zero-sized regular files.

TarFile.gettarinfo(*name=None*, *arcname=None*, *fileobj=None*)

Create a **`TarInfo`** object from the result of **`os.stat()`** or equivalent on an existing file. The file is either named by *name*, or specified as a file object *fileobj* with a file descriptor. *name* may be a path-like object. If given, *arcname* specifies an alternative name for the file in the archive, otherwise, the name is taken from *fileobj*'s **`name`** attribute, or the *name* argument. The name should be a text string.

You can modify some of the **`TarInfo`**'s attributes before you add it using **`addfile()`**. If the file object is not an ordinary file object positioned at the beginning of the file, attributes such as **`size`** may need modifying. This is the case for objects such as **`GzipFile`**. The **`name`** may also be modified, in which case *arcname* could be a dummy string.

Alterado na versão 3.6: O parâmetro *name* aceita um objeto caminho ou similar.

TarFile.close()

Close the **`TarFile`**. In write mode, two finishing zero blocks are appended to the archive.

TarFile.pax_headers*: dict*

Um dicionário contendo pares de chave-valor de cabeçalhos globais pax.

Objetos TarInfo

A **`TarInfo`** object represents one member in a **`TarFile`**. Aside from storing all required attributes of a file (like file type, size, time, permissions, owner etc.), it provides some useful methods to determine its type. It does *not* contain the file's data itself.

`TarInfo` objects are returned by `TarFile`'s methods `getmember()`, `getmembers()` and `gettarinfo()`.

Modifying the objects returned by `getmember()` or `getmembers()` will affect all subsequent operations on the archive. For cases where this is unwanted, you can use `copy.copy()` or call the `replace()` method to create a modified copy in one step.

Several attributes can be set to `None` to indicate that a piece of metadata is unused or unknown. Different `TarInfo` methods handle `None` differently:

- The `extract()` or `extractall()` methods will ignore the corresponding metadata, leaving it set to a default.
- `addfile()` falhará.
- `list()` imprimirá uma string como um espaço reservado.

class tarfile.TarInfo(*name=''*)

Cria um objeto `TarInfo`.

classmethod TarInfo.frombuf(*buf*, *encoding*, *errors*)

Cria e retorna um objeto `TarInfo` a partir do buffer de string do parâmetro *buf*.

Levanta `HeaderError` se o buffer for inválido.

classmethod TarInfo.fromtarfile(*tarfile*)

Read the next member from the `TarFile` object *tarfile* and return it as a `TarInfo` object.

TarInfo.tobuf(*format=DEFAULT_FORMAT*, *encoding=ENCODING*, *errors='surrogateescape'*)

Create a string buffer from a `TarInfo` object. For information on the arguments see the constructor of the `TarFile` class.

Alterado na versão 3.2: Usa `'surrogateescape'` como padrão para o argumento *errors*.

A `TarInfo` object has the following public data attributes:

TarInfo.name*: str*

Nome do arquivo.

TarInfo.size*: int*

Tamanho em bytes.

TarInfo.mtime*: int* | *float*

Time of last modification in seconds since the epoch, as in **`os.stat_result.st_mtime`**.

Alterado na versão 3.12: Can be set to `None` for **`extract()`** and **`extractall()`**, causing extraction to skip applying this attribute.

TarInfo.mode*: int*

Permission bits, as for **`os.chmod()`**.

Alterado na versão 3.12: Can be set to `None` for **`extract()`** and **`extractall()`**, causing extraction to skip applying this attribute.

TarInfo.type

File type. *type* is usually one of these constants: **`REGTYPE`**, **`AREGTYPE`**, **`LNKTYPE`**, **`SYMTYPE`**, **`DIRTYPE`**, **`FIFOTYPE`**, **`CONTTYPE`**, **`CHRTYPE`**, **`BLKTYPE`**, **`GNUTYPE_SPARSE`**. To determine the type of a **`TarInfo`** object more conveniently, use the `is*()` methods below.

TarInfo.linkname*: str*

Name of the target file name, which is only present in **`TarInfo`** objects of type **`LNKTYPE`** and **`SYMTYPE`**.

For symbolic links (`SYMTYPE`), the *linkname* is relative to the directory that contains the link. For hard links (`LNKTYPE`), the *linkname* is relative to the root of the archive.

TarInfo.uid*: int*

ID do usuário do usuário que originalmente armazenou este membro.

Alterado na versão 3.12: Can be set to `None` for **`extract()`** and **`extractall()`**, causing extraction to skip applying this attribute.

TarInfo.gid*:* *int*

ID do grupo do usuário que originalmente armazenou este membro.

Alterado na versão 3.12: Can be set to `None` for **`extract()`** and **`extractall()`**, causing extraction to skip applying this attribute.

TarInfo.uname*:* *str*

Nome do usuário.

Alterado na versão 3.12: Can be set to `None` for **`extract()`** and **`extractall()`**, causing extraction to skip applying this attribute.

TarInfo.gname*:* *str*

Nome do grupo.

Alterado na versão 3.12: Can be set to `None` for **`extract()`** and **`extractall()`**, causing extraction to skip applying this attribute.

TarInfo.chksum*:* *int*

Cabeçalho de soma de verificação.

TarInfo.devmajor*:* *int*

Número principal do dispositivo.

TarInfo.devminor*:* *int*

Número menor do dispositivo.

TarInfo.offset*:* *int*

O cabeçalho do tar começa aqui.

TarInfo.offset_data*:* *int*

Os dados do arquivo começam aqui.

TarInfo.sparse

Sparse member information.

TarInfo.pax_headers*: dict*

A dictionary containing key-value pairs of an associated pax extended header.

TarInfo.replace(*name=..., mtime=..., mode=..., linkname=..., uid=..., gid=..., uname=..., gname=..., deep=True*)

Adicionado na versão 3.12.

Return a *new* copy of the **`TarInfo`** object with the given attributes changed. For example, to return a `TarInfo` with the group name set to `'staff'`, use:

```
new_tarinfo = old_tarinfo.replace(gname='staff')
```

By default, a deep copy is made. If *deep* is false, the copy is shallow, i.e. `pax_headers` and any custom attributes are shared with the original `TarInfo` object.

A **`TarInfo`** object also provides some convenient query methods:

TarInfo.isfile()

Return **`True`** if the **`TarInfo`** object is a regular file.

TarInfo.isreg()

Igual a **`isfile()`**.

TarInfo.isdir()

Retorna **`True`** se for um diretório.

TarInfo.issym()

Retorna **`True`** se for um link simbólico.

TarInfo.islnk()

Retorna **`True`** se for um link físico.

TarInfo.ischr()

Retorna `True` se for um caractere.

TarInfo.isblk()

Return `True` if it is a block device.

TarInfo.isfifo()

Retorna `True` se for um FIFO.

TarInfo.isdev()

Return `True` if it is one of character device, block device or FIFO.

Filtros de extração

Adicionado na versão 3.12.

The *tar* format is designed to capture all details of a UNIX-like filesystem, which makes it very powerful. Unfortunately, the features make it easy to create tar files that have unintended – and possibly malicious – effects when extracted. For example, extracting a tar file can overwrite arbitrary files in various ways (e.g. by using absolute paths, `..` path components, or symlinks that affect later members).

In most cases, the full functionality is not needed. Therefore, *tarfile* supports extraction filters: a mechanism to limit functionality, and thus mitigate some of the security issues.

Aviso

None of the available filters blocks *all* dangerous archive features. Never extract archives from untrusted sources without prior inspection. See also Dicas para verificação adicional.

Ver também

PEP 706 **[https://peps.python.org/pep-0706/]**

Contém a motivação e lógica escolhida para este design.

The *filter* argument to `TarFile.extract()` or `extractall()` can be:

- the string `'fully_trusted'`: Honor all metadata as specified in the archive. Should be used if the user trusts the archive completely, or implements their own complex verification.

- the string `'tar'`: Honor most *tar*-specific features (i.e. features of UNIX-like filesystems), but block features that are very likely to be surprising or malicious. See `tar_filter()` for details.

- the string `'data'`: Ignore or block most features specific to UNIX-like filesystems. Intended for extracting cross-platform data archives. See `data_filter()` for details.

- `None` (default): Use `TarFile.extraction_filter`.

  If that is also `None` (the default), the `'data'` filter will be used.

  > *Alterado na versão 3.14:* The default filter is set to `data`. Previously, the default was equivalent to `fully_trusted`.

- A callable which will be called for each extracted member with a TarInfo describing the member and the destination path to where the archive is extracted (i.e. the same path is used for all members):

  ```
  filter(member: TarInfo, path: str, /) -> TarInfo | None
  ```

  The callable is called just before each member is extracted, so it can take the current state of the disk into account. It can:

  - return a `TarInfo` object which will be used instead of the metadata in the archive, or
  - return `None`, in which case the member will be skipped, or

- raise an exception to abort the operation or skip the member, depending on `errorlevel`. Note that when extraction is aborted, `extractall()` may leave the archive partially extracted. It does not attempt to clean up.

Default named filters

The pre-defined, named filters are available as functions, so they can be reused in custom filters:

tarfile.fully_trusted_filter(*member*, *path*)

Retorna *member* inalterado.

This implements the `'fully_trusted'` filter.

tarfile.tar_filter(*member*, *path*)

Implements the `'tar'` filter.

- Strip leading slashes (`/` and `os.sep`) from filenames.
- Refuse to extract files with absolute paths (in case the name is absolute even after stripping slashes, e.g. `C:/foo` on Windows). This raises `AbsolutePathError`.
- Normalize filenames (`TarInfo.name`) that contain `..` components using `os.path.normpath()`. Note that this removes internal `..` components, which may change the meaning of the name if it traverses symbolic links.
- Refuse to extract files whose absolute path (after following symlinks) would end up outside the destination. This raises `OutsideDestinationError`.
- Clear high mode bits (setuid, setgid, sticky) and group/other write bits (`S_IWGRP` | `S_IWOTH`).

Return the modified `TarInfo` member.

Alterado na versão 3.15: Filenames containing `..` components are now normalized.

tarfile.data_filter(*member*, *path*)

Implements the `'data'` filter. In addition to what `tar_filter` does:

- Normalize link targets (**`TarInfo.linkname`**) using **`os.path.normpath()`**. Note that this removes internal `..` components, which may change the meaning of the link if the path in **`TarInfo.linkname`** traverses symbolic links.

- Refuse to extract links (hard or soft) that link to absolute paths, or ones that link outside the destination.

  This raises **`AbsoluteLinkError`** or **`LinkOutsideDestinationError`**.

  Note that such files are refused even on platforms that do not support symbolic links.

- Refuse to extract device files (including pipes). This raises **`SpecialFileError`**.

- Para arquivos comuns, incluindo links físicos:

  - Set the owner read and write permissions (**`S_IRUSR`** | **`S_IWUSR`**).
  - Remove the group & other executable permission (**`S_IXGRP`** | **`S_IXOTH`**) if the owner doesn't have it (**`S_IXUSR`**).

- For other files (directories), set `mode` to `None`, so that extraction methods skip applying permission bits.

- Set user and group info (`uid`, `gid`, `uname`, `gname`) to `None`, so that extraction methods skip setting it.

Return the modified `TarInfo` member.

Note that this filter does not block *all* dangerous archive features. See Dicas para verificação adicional for details.

Alterado na versão 3.15: Link targets are now normalized.

Filter errors

When a filter refuses to extract a file, it will raise an appropriate exception, a subclass of `FilterError`. This will abort the extraction if `TarFile.errorlevel` is 1 or more. With `errorlevel=0` the error will be logged and the member will be skipped, but extraction will continue.

Dicas para verificação adicional

Even with `filter='data'`, *tarfile* is not suited for extracting untrusted files without prior inspection. Among other issues, the pre-defined filters do not prevent denial-of-service attacks. Users should do additional checks.

Aqui está uma lista incompleta de itens a serem considerados:

- Extract to a `new temporary directory` to prevent e.g. exploiting pre-existing links, and to make it easier to clean up after a failed extraction.
- Disallow symbolic links if you do not need the functionality.
- When working with untrusted data, use external (e.g. OS-level) limits on disk, memory and CPU usage.
- Check filenames against an allow-list of characters (to filter out control characters, confusables, foreign path separators, and so on).
- Check that filenames have expected extensions (discouraging files that execute when you “click on them”, or extension-less files like Windows special device names).
- Limit the number of extracted files, total size of extracted data, filename length (including symlink length), and size of individual files.
- Check for files that would be shadowed on case-insensitive filesystems.

Observe também que:

- Tar files may contain multiple versions of the same file. Later ones are expected to overwrite any earlier ones. This feature is crucial to allow updating tape archives, but can be abused maliciously.

- *tarfile* does not protect against issues with “live” data, e.g. an attacker tinkering with the destination (or source) directory while extraction (or archiving) is in progress.

Suporte a versões mais antigas do Python

Extraction filters were added to Python 3.12, but may be backported to older versions as security updates. To check whether the feature is available, use e.g. `hasattr(tarfile, 'data_filter')` rather than checking the Python version.

The following examples show how to support Python versions with and without the feature. Note that setting `extraction_filter` will affect any subsequent operations.

- Fully trusted archive:

  ```
  my_tarfile.extraction_filter = (lambda member, path: member)
  my_tarfile.extractall()
  ```

- Use the `'data'` filter if available, but revert to Python 3.11 behavior (`'fully_trusted'`) if this feature is not available:

  ```
  my_tarfile.extraction_filter = getattr(tarfile, 'data_filter
                                         (lambda member, path
  my_tarfile.extractall()
  ```

- Use o filtro `'data'`; *fail* se ele não estiver disponível:

  ```
  my_tarfile.extractall(filter=tarfile.data_filter)
  ```

  ou:

  ```
  my_tarfile.extraction_filter = tarfile.data_filter
  my_tarfile.extractall()
  ```

- Use o filtro `'data'`; *warn* se ele não estiver disponível:

```
    if hasattr(tarfile, 'data_filter'):
        my_tarfile.extractall(filter='data')
    else:
        # remove this when no longer needed
        warn_the_user('Extracting may be unsafe; consider updat
        my_tarfile.extractall()
```

Stateful extraction filter example

While *tarfile*'s extraction methods take a simple *filter* callable, custom filters may be more complex objects with an internal state. It may be useful to write these as context managers, to be used like this:

```
with StatefulFilter() as filter_func:
    tar.extractall(path, filter=filter_func)
```

Such a filter can be written as, for example:

```
class StatefulFilter:
    def __init__(self):
        self.file_count = 0

    def __enter__(self):
        return self

    def __call__(self, member, path):
        self.file_count += 1
        return member

    def __exit__(self, *exc_info):
        print(f'{self.file_count} files extracted')
```

Interface de Linha de Comando

Adicionado na versão 3.4.

The **`tarfile`** module provides a simple command-line interface to interact with tar archives.

If you want to create a new tar archive, specify its name after the `-c` option and then list the filename(s) that should be included:

```
$ python -m tarfile -c monty.tar  spam.txt eggs.txt
```

Passar um diretório também é aceito:

```
$ python -m tarfile -c monty.tar life-of-brian_1979/
```

If you want to extract a tar archive into the current directory, use the `-e` option:

```
$ python -m tarfile -e monty.tar
```

You can also extract a tar archive into a different directory by passing the directory's name:

```
$ python -m tarfile -e monty.tar  other-dir/
```

For a list of the files in a tar archive, use the `-l` option:

```
$ python -m tarfile -l monty.tar
```

Opções de linha de comando

-l <tarfile>
--list <tarfile>
    Lista os arquivos em um arquivo tar.

-c <tarfile> <source1> ... <sourceN>
--create <tarfile> <source1> ... <sourceN>
    Cria um arquivo tar a partir dos arquivos de origem.

-e <tarfile> [<output_dir>]
--extract <tarfile> [<output_dir>]
    Extract tarfile into the current directory if *output_dir* is not specified.

-t <tarfile>
--test <tarfile>

Test whether the tarfile is valid or not.

-v, --verbose
Saída detalhada.

--filter <filtername>
Specifies the *filter* for `--extract`. See Filtros de extração for details. Only string names are accepted (that is, `fully_trusted`, `tar`, and `data`).

Exemplos

Reading examples

How to extract an entire tar archive to the current working directory:

```
import tarfile
tar = tarfile.open("sample.tar.gz")
tar.extractall(filter='data')
tar.close()
```

How to extract a subset of a tar archive with `TarFile.extractall()` using a generator function instead of a list:

```
import os
import tarfile

def py_files(members):
    for tarinfo in members:
        if os.path.splitext(tarinfo.name)[1] == ".py":
            yield tarinfo

tar = tarfile.open("sample.tar.gz")
tar.extractall(members=py_files(tar))
tar.close()
```

How to read a gzip compressed tar archive and display some member information:

```
import tarfile
tar = tarfile.open("sample.tar.gz", "r:gz")
for tarinfo in tar:
    print(tarinfo.name, "is", tarinfo.size, "bytes in size and i
    if tarinfo.isreg():
        print("a regular file.")
    elif tarinfo.isdir():
        print("a directory.")
    else:
        print("something else.")
tar.close()
```

Writing examples

How to create an uncompressed tar archive from a list of filenames:

```
import tarfile
tar = tarfile.open("sample.tar", "w")
for name in ["foo", "bar", "quux"]:
    tar.add(name)
tar.close()
```

The same example using the `with` statement:

```
import tarfile
with tarfile.open("sample.tar", "w") as tar:
    for name in ["foo", "bar", "quux"]:
        tar.add(name)
```

How to create and write an archive to stdout using `sys.stdout.buffer` in the *fileobj* parameter in `TarFile.add()`:

```
import sys
import tarfile
with tarfile.open("sample.tar.gz", "w|gz", fileobj=sys.stdout.bu
    for name in ["foo", "bar", "quux"]:
        tar.add(name)
```

How to create an archive and reset the user information using the *filter* parameter in `TarFile.add()`:

```
import tarfile
def reset(tarinfo):
    tarinfo.uid = tarinfo.gid = 0
    tarinfo.uname = tarinfo.gname = "root"
    return tarinfo
tar = tarfile.open("sample.tar.gz", "w:gz")
tar.add("foo", filter=reset)
tar.close()
```

Formatos tar suportados

There are three tar formats that can be created with the **`tarfile`** module:

- The POSIX.1-1988 ustar format (**`USTAR_FORMAT`**). It supports filenames up to a length of at best 256 characters and linknames up to 100 characters. The maximum file size is 8 GiB. This is an old and limited but widely supported format.

- The GNU tar format (**`GNU_FORMAT`**). It supports long filenames and linknames, files bigger than 8 GiB and sparse files. It is the de facto standard on GNU/Linux systems. **`tarfile`** fully supports the GNU tar extensions for long names, sparse file support is read-only.

- The POSIX.1-2001 pax format (**`PAX_FORMAT`**). It is the most flexible format with virtually no limits. It supports long filenames and linknames, large files and stores pathnames in a portable way. Modern tar implementations, including GNU tar, bsdtar/libarchive and star, fully support extended *pax* features; some old or unmaintained libraries may not, but should treat *pax* archives as if they were in the universally supported *ustar* format. It is the current default format for new archives.

  It extends the existing *ustar* format with extra headers for information that cannot be stored otherwise. There are two flavours of pax headers: Extended headers only affect the subsequent file header, global headers are valid for the complete archive and affect all following files. All the data in a pax header is encoded in *UTF-8* for portability reasons.

There are some more variants of the tar format which can be read, but not created:

- The ancient V7 format. This is the first tar format from Unix Seventh Edition, storing only regular files and directories. Names must not be longer than 100 characters, there is no user/group name information. Some archives have miscalculated header checksums in case of fields with non-ASCII characters.
- The SunOS tar extended format. This format is a variant of the POSIX.1-2001 pax format, but is not compatible.

Problemas de Unicode

The tar format was originally conceived to make backups on tape drives with the main focus on preserving file system information. Nowadays tar archives are commonly used for file distribution and exchanging archives over networks. One problem of the original format (which is the basis of all other formats) is that there is no concept of supporting different character encodings. For example, an ordinary tar archive created on a *UTF-8* system cannot be read correctly on a *Latin-1* system if it contains non-*ASCII* characters. Textual metadata (like filenames, linknames, user/group names) will appear damaged. Unfortunately, there is no way to autodetect the encoding of an archive. The pax format was designed to solve this problem. It stores non-ASCII metadata using the universal character encoding *UTF-8*.

The details of character conversion in **`tarfile`** are controlled by the *encoding* and *errors* keyword arguments of the **`TarFile`** class.

encoding defines the character encoding to use for the metadata in the archive. The default value is **`sys.getfilesystemencoding()`** or `'ascii'` as a fallback. Depending on whether the archive is read or written, the metadata must be either decoded or encoded. If *encoding* is not set appropriately, this conversion may fail.

The *errors* argument defines how characters are treated that cannot be converted. Possible values are listed in section Error Handlers. The default

scheme is `'surrogateescape'` which Python also uses for its file system calls, see Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente.

For **`PAX_FORMAT`** archives (the default), *encoding* is generally not needed because all the metadata is stored using *UTF-8*. *encoding* is only used in the rare cases when binary pax headers are decoded or when strings with surrogate characters are stored.

Formatos de Arquivo

Os módulos descritos neste capítulo analisam vários formatos de arquivo diversos que não são linguagens de marcação e não estão relacionados ao e-mail.

- `csv` — Leitura e escrita de arquivos CSV
  - Conteúdo do módulo
  - Dialects and formatting parameters
  - Reader objects
  - Writer objects
  - Exemplos
- `configparser` — Analisador sintático de arquivo de configuração
  - Início rápido
  - Tipos de dados suportados
  - Valores reservas
  - Estrutura dos arquivos INI
  - Seções não nomeadas
  - Interpolação de valores
  - Acesso através do protocolo de mapeamento
  - Personalizando o comportamento do analisador sintático
  - Exemplos de APIs legadas
  - Objetos ConfigParser
  - Objetos RawConfigParser
  - Exceções
- `tomllib` — Analisa arquivos TOML
  - Exemplos
  - Tabela de conversão
  - Considerações sobre limites e interoperabilidade
- `netrc` — Arquivo de processamento netrc
  - Objetos netrc
- `plistlib` — Gera e analisa arquivos Apple `.plist`
  - Exemplos

`csv` — Leitura e escrita de arquivos CSV

Código-fonte: Lib/csv.py [https://github.com/python/cpython/tree/3.15/Lib/csv.py]

O chamado formato CSV (Comma Separated Values) é o formato mais comum de importação e exportação de planilhas e bancos de dados. O formato CSV foi usado por muitos anos antes das tentativas de descrever o formato de maneira padronizada em **RFC 4180** [https://datatracker.ietf.org/doc/html/rfc4180.html]. A falta de um padrão bem definido significa que existem diferenças sutis nos dados produzidos e consumidos por diferentes aplicativos. Essas diferenças podem tornar irritante o processamento de arquivos CSV de várias fontes. Ainda assim, enquanto os delimitadores e os caracteres de citação variam, o formato geral é semelhante o suficiente para que seja possível escrever um único módulo que possa manipular eficientemente esses dados, ocultando os detalhes da leitura e gravação dos dados do programador.

O módulo `csv` implementa classes para ler e gravar dados tabulares no formato CSV. Ele permite que os programadores digam “escreva esses dados no formato preferido pelo Excel” ou “leia os dados desse arquivo gerado pelo Excel”, sem conhecer os detalhes precisos do formato CSV usado pelo Excel. Os programadores também podem descrever os formatos CSV entendidos por outros aplicativos ou definir seus próprios formatos CSV para fins especiais.

Os objetos de `reader` e `writer` do módulo `csv` leem e escrevem sequências. Os programadores também podem ler e gravar dados no formato de dicionário usando as classes `DictReader` e `DictWriter`.

Ver também

PEP 305 [https://peps.python.org/pep-0305/] - API de arquivo CSV

A proposta de melhoria do Python que propôs essa adição ao Python.

Conteúdo do módulo

O módulo **`csv`** define as seguintes funções:

csv.reader(*csvfile*, /, *dialect='excel'*, ***fmtparams*)

Retorna um objeto leitor que irá iterar sobre as linhas no *csvfile* fornecido. Um csvfile deve ser um iterável de strings, cada uma no formato csv definido pelo leitor. Um csvfile é muito comumente um objeto similar a arquivo ou uma lista. Se *csvfile* for um objeto arquivo, ele deverá ser aberto com `newline=''`. [[1]] Pode ser fornecido um parâmetro opcional *dialect*, usado para definir um conjunto de parâmetros específicos para um dialeto CSV específico. Pode ser uma instância de uma subclasse da classe **`Dialect`** ou uma das strings retornadas pela função **`list_dialects()`**. Os outros argumentos nomeados opcionais *fmtparams* podem ser dados para substituir parâmetros de formatação individuais no dialeto atual. Para detalhes completos sobre os parâmetros de dialeto e formatação, consulte a seção Dialects and formatting parameters.

Cada linha lida no arquivo csv é retornada como uma lista de strings. Nenhuma conversão automática de tipo de dados é executada, a menos que a opção de formato **`QUOTE_NONNUMERIC`** seja especificada (nesse caso, os campos não citados são transformados em pontos flutuantes).

Um pequeno exemplo de uso:

```
>>> import csv
>>> with open('eggs.csv', newline='') as csvfile:
...     spamreader = csv.reader(csvfile, delimiter=' ', quote
...     for row in spamreader:
...         print(', '.join(row))
Spam, Spam, Spam, Spam, Spam, Baked Beans
Spam, Lovely Spam, Wonderful Spam
```

sendo que `eggs.csv` contém:

```
Spam Spam Spam Spam Spam |Baked Beans|
Spam |Lovely Spam| |Wonderful Spam|
```

csv.writer(*csvfile*, /, *dialect='excel'*, ***fmtparams*)

Retorna um objeto de escrita responsável por converter os dados de usuário em strings delimitadas no objeto arquivo ou similar. *csvfile* pode ser qualquer objeto com um método **`write()`**. Se *csvfile* for um objeto arquivo, ele deverá ser aberto com `newline=''` [[1]]. Pode ser fornecido um parâmetro opcional *dialect*, usado para definir um conjunto de parâmetros específicos para um dialeto CSV específico. Pode ser uma instância de uma subclasse da classe **`Dialect`** ou uma das strings retornadas pela função **`list_dialects()`**. Os outros argumentos nomeados opcionais *fmtparams* podem ser dados para substituir parâmetros de formatação individuais no dialeto atual. Para detalhes completos sobre os parâmetros de dialeto e formatação, consulte a seção Dialects and formatting parameters. Para tornar o mais fácil possível a interface com os módulos que implementam a API do DB, o valor **`None`** é escrito como uma string vazia. Embora isso não seja uma transformação reversível, torna mais fácil despejar valores de dados SQL NULL em arquivos CSV sem preprocessar os dados retornados de uma chamada `cursor.fetch*`. Todos os outros dados que não são de strings são codificados com **`str()`** antes de serem escritos.

Um pequeno exemplo de uso:

```
import csv
with open('eggs.csv', 'w', newline='') as csvfile:
    spamwriter = csv.writer(csvfile, delimiter=' ',
                            quotechar='|', quoting=csv.QUOTE_
    spamwriter.writerow(['Spam'] * 5 + ['Baked Beans'])
    spamwriter.writerow(['Spam', 'Lovely Spam', 'Wonderful Sp
```

o que escreve `eggs.csv` contendo:

```
Spam Spam Spam Spam Spam |Baked Beans|
Spam |Lovely Spam| |Wonderful Spam|
```

csv.register_dialect(*name*, *,, *dialect='excel'*, ***fmtparams*)

Associa *dialect* a *name*. *name* deve ser uma string. O dialeto pode ser especificado passando uma subclasse de **`Dialect`** ou por *fmtparams* argumentos nomeados, ou ambos, com argumentos nomeados substituindo os parâmetros do dialeto. Para detalhes completos sobre os parâmetros de dialeto e formatação, consulte a seção Dialects and formatting parameters.

csv.unregister_dialect(*name*)

Exclui o dialeto associado ao *name* do registro do dialeto. Um **`Error`** é levantado se *name* não for um nome de dialeto registrado.

csv.get_dialect(*name*)

Retorna o dialeto associado a *name*. Um **`Error`** é levantado se *name* não for um nome de dialeto registrado. Esta função retorna uma classe **`Dialect`** imutável.

csv.list_dialects()

Retorna os nomes de todos os dialetos registrados

csv.field_size_limit()
csv.field_size_limit(*new_limit*)

Retorna o tamanho máximo atual do campo permitido pelo analisador sintático. Se *new_limit* for fornecido, este se tornará o novo limite.

O módulo **`csv`** define as seguintes classes:

class csv.DictReader(*f*, *fieldnames=None*, *restkey=None*, *restval=None*, *dialect='excel'*, **args*, ***kwds*)

Cria um objeto que funcione como um leitor comum, mas mapeia as informações em cada linha para **`dict`** cujas chaves são fornecidas pelo parâmetro opcional *fieldnames*.

O parâmetro *fieldnames* é uma sequência. Se *fieldnames* for omitido, os valores na primeira linha do arquivo *f* serão usados como nomes de campo e serão omitidos dos resultados. Se *fieldnames* for fornecido,

eles serão usados e a primeira linha será incluída nos resultados. Independentemente de como os nomes de campo são determinados, o dicionário preserva sua ordem original.

Se uma linha tiver mais campos que nomes de campo, os dados restantes serão colocados em uma lista e armazenados com o nome do campo especificado por *restkey* (o padrão é `None`). Se uma linha que não estiver em branco tiver menos campos que nomes de campo, os valores ausentes serão preenchidos com o valor *restval* (o padrão é `None`).

Todos os outros argumentos nomeados ou opcionais são passados para a instância subjacente de **`reader`**.

Se o argumento passado para *fieldnames* for um iterador, ele será convertido para uma **`list`**.

Alterado na versão 3.6: Linhas retornadas agora são do tipo **`OrderedDict`**.

Alterado na versão 3.8: As linhas retornadas agora são do tipo **`dict`**.

Um pequeno exemplo de uso:

```
>>> import csv
>>> with open('names.csv', newline='') as csvfile:
...     reader = csv.DictReader(csvfile)
...     for row in reader:
...         print(row['first_name'], row['last_name'])
...
Eric Idle
John Cleese

>>> print(row)
{'first_name': 'John', 'last_name': 'Cleese'}
```

sendo que `names.csv` contém:

```
first_name,last_name
Eric,Idle
John,Cleese
```

class csv.DictWriter(*f*, *fieldnames*, *restval=''*, *extrasaction='raise'*, *dialect='excel'*, **args*, ***kwds*)

Cria um objeto que funcione como um de escrita comum, mas mapeia dicionários nas linhas de saída. O parâmetro *fieldnames* é uma **sequência** de chaves que identificam a ordem na qual os valores no dicionário transmitidos para o método **writerow()** são escritos no arquivo *f*. O parâmetro opcional *restval* especifica o valor a ser escrito se o dicionário estiver com falta de uma chave em *fieldnames*. Se o dicionário transmitido para o método **writerow()** contiver uma chave não encontrada em *fieldnames*, o parâmetro opcional *extrasaction* indica qual ação executar. Se estiver definido como `'raise'`, o valor padrão, a **ValueError** é levantada. Se estiver definido como `'ignore'`, valores extras no dicionário serão ignorados. Quaisquer outros argumentos nomeados ou opcionais são passados para a instância subjacente de **writer**.

Observe que, diferentemente da classe **DictReader**, o parâmetro *fieldnames* da classe **DictWriter** não é opcional.

Se o argumento passado para *fieldnames* for um iterador, ele será convertido para uma **list**.

Um pequeno exemplo de uso:

```
import csv

with open('names.csv', 'w', newline='') as csvfile:
    fieldnames = ['first_name', 'last_name']
    writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

    writer.writeheader()
    writer.writerow({'first_name': 'Baked', 'last_name': 'Bea
    writer.writerow({'first_name': 'Lovely', 'last_name': 'Sp
    writer.writerow({'first_name': 'Wonderful', 'last_name':
```

o que escreve `names.csv` contendo:

```
first_name,last_name
Baked,Beans
Lovely,Spam
Wonderful,Spam
```

class csv.Dialect

A classe **`Dialect`** é uma classe de contêiner cujos atributos contêm informações sobre como lidar com aspas duplas, espaços em branco, delimitadores, etc. Devido à falta de uma especificação CSV estrita, diferentes aplicativos produzem dados CSV sutilmente diferentes. As instâncias de **`Dialect`** definem como as instâncias **`reader`** e **`writer`** se comportam.

Todos os nomes de **`Dialect`** disponíveis são retornados por **`list_dialects()`**, e eles podem ser registrados com classes **`reader`** e **`writer`** específicas através de suas funções inicializadoras (`__init__`) como esta:

```
import csv

with open('students.csv', 'w', newline='') as csvfile:
    writer = csv.writer(csvfile, dialect='unix')
```

class csv.excel

A classe **`excel`** define as propriedades usuais de um arquivo CSV gerado pelo Excel. Ele é registrado com o nome do dialeto `'excel'`.

class csv.excel_tab

A classe **`excel_tab`** define as propriedades usuais de um arquivo delimitado por TAB gerado pelo Excel. Ela é registrado com o nome do dialeto `'excel-tab'`.

class csv.unix_dialect

A classe **`unix_dialect`** define as propriedades usuais de um arquivo CSV gerado em sistemas UNIX, ou seja, usando `'\n'` como terminador de linha e citando todos os campos. É registrado com o nome do dialeto `'unix'`.

Adicionado na versão 3.2.

class csv.Sniffer

A classe `Sniffer` é usada para deduzir o formato de um arquivo CSV.

A classe `Sniffer` fornece dois métodos:

sniff(*sample*, *delimiters=None*)

Analisa a *sample* fornecida e retorna uma subclasse `Dialect`, refletindo os parâmetros encontrados. Se o parâmetro opcional *delimiters* for fornecido, ele será interpretado como uma string contendo possíveis caracteres válidos de delimitador.

If several delimiters fit the sample equally well — for example if both `','` and `';'` split every row consistently — the delimiters listed in the `preferred` attribute are preferred, in that order, no matter how many times each of them occurs.

has_header(*sample*)

Analisa o texto de amostra (presume-se que esteja no formato CSV) e retorne `True` se a primeira linha parecer uma série de cabeçalhos de coluna. Inspecionando cada coluna, um dos dois critérios principais será considerado para estimar se a amostra contém um cabeçalho:

- da segunda até a enésima linha contém valores numéricos
- da segunda até a enésima linha contém strings em que pelo menos o comprimento de um valor difere daquele do cabeçalho putativo dessa coluna.

Vinte e uma linhas após a linha de cabeçalho são amostradas; se mais da metade das colunas + linhas atenderem aos critérios, `True` será retornado.

Nota

Este método é uma heurística aproximada e pode produzir falsos positivos e negativos.

The `Sniffer` class has the following attribute:

preferred

The list of the delimiters preferred for breaking ties, in the order of preference. It can be modified. Its initial value is `[',', '\t', ';', ' ', ':']`.

Um exemplo para uso de **`Sniffer`**:

```
with open('example.csv', newline='') as csvfile:
    dialect = csv.Sniffer().sniff(csvfile.read(1024))
    csvfile.seek(0)
    reader = csv.reader(csvfile, dialect)
    # ... processa conteúdo do arquivo CSV aqui ...
```

The **`csv`** module defines the following constants:

csv.QUOTE_ALL

Instrui objetos **`writer`** a colocar aspas em todos os campos.

csv.QUOTE_MINIMAL

Instructs **`writer`** objects to only quote those fields which contain special characters such as *delimiter*, *quotechar*, `'\r'`, `'\n'` or any of the characters in *lineterminator*. If *doublequote* is **`False`** and *escapechar* is set, the *quotechar* is escaped instead of causing the field to be quoted.

csv.QUOTE_NONNUMERIC

Instrui objetos **`writer`** a colocar aspas em todos os campos não numéricos.

Instrui objetos **`reader`** a converter todos os campos não envoltos por aspas no tipo **`float`**.

Nota

Alguns tipos numéricos, como **`bool`**, **`Fraction`** ou **`IntEnum`**, têm uma representação de string que não pode ser convertida para **`float`**. Eles não podem ser lidos nos modos **`QUOTE_NONNUMERIC`** e **`QUOTE_STRINGS`**.

csv.QUOTE_NONE

Instrui objetos `writer` a nunca colocar aspas nos campos. Quando o *delimiter*, *quotechar*, *escapechar*, `'\r'`, `'\n'` ou qualquer um dos caracteres em *lineterminator* ocorre nos dados de saída, é precedido pelo caractere *escapechar* atual. Se *escapeechar* não estiver definido, o escritor levantará `Error` se algum caractere que exija escape for encontrado. Defina *quotechar* para `None` para evitar seu escape.

Instrui objetos `reader` a não executar nenhum processamento especial de caracteres de aspas.

csv.QUOTE_NOTNULL

Instrui objetos `writer` a coloca entre aspas os campos que não são `None`. Isso é semelhante a `QUOTE_ALL`, exceto que se o valor de um campo for `None`, uma string vazia (sem aspas) é escrita.

Instrui objetos `reader` a interpretar um campo vazio (sem aspas) como `None` e a se comportar de outra forma como `QUOTE_ALL`.

Adicionado na versão 3.12.

csv.QUOTE_STRINGS

Instrui objetos `writer` a sempre coloca aspas em volta dos campos que são strings. Isso é semelhante a `QUOTE_NONNUMERIC`, exceto que se o valor de um campo for `None`, uma string vazia (sem aspas) é escrita.

Instrui objetos `reader` a interpretar uma string vazia (sem aspas) como `None` e a se comportar de outra forma como `QUOTE_NONNUMERIC`.

Adicionado na versão 3.12.

The **`csv`** module defines the following exception:

exception csv.Error

Levantada por qualquer uma das funções quando um erro é detectado.

Dialects and formatting parameters

Para facilitar a especificação do formato dos registros de entrada e saída, parâmetros específicos de formatação são agrupados em dialetos. Um dialeto é uma subclasse da classe **`Dialect`** contendo vários atributos descrevendo o formato do arquivo CSV. Ao criar objetos **`reader`** ou **`writer`**, o programador pode especificar uma string ou uma subclasse da classe **`Dialect`** como parâmetro de dialeto. Além do parâmetro *dialect*, ou em vez do parâmetro *dialect*, o programador também pode especificar parâmetros de formatação individuais, com os mesmos nomes dos atributos definidos abaixo para a classe **`Dialect`**.

Os dialetos possuem suporte aos seguintes atributos:

Dialect.delimiter

Uma string de um caractere usada para separar campos. O padrão é `','`.

Dialect.doublequote

Controla como as instâncias de *quotechar* que aparecem dentro de um campo devem estar entre aspas. Quando **`True`**, o caractere é dobrado. Quando **`False`**, o *escapechar* é usado como um prefixo para o *quotechar*. O padrão é **`True`**.

Na saída, se *doublequote* for **`False`** e não *escapeechar* for definido, **`Error`** é levantada se um *quotechar* é encontrado em um campo.

Dialect.escapechar

Uma string de caracteres de um caractere usada pelo escritor para escapar caracteres que exigem escape:

- o *delimiter*, o *quotechar*, `'\r'`, `'\n'` e qualquer um dos caracteres em *lineterminator* são escapados se *quoting* estiver definido como **`QUOTE_NONE`**;
- o *quotechar* é escapado se *doublequote* for **`False`**;
- o *escapechar* em si.

Na leitura, o *escapechar* remove qualquer significado especial do caractere seguinte. O padrão é **`None`**, o que desabilita o escape.

Alterado na versão 3.10: Previously the *escapechar* itself was not escaped, which lost it on reading.

Alterado na versão 3.11: Um *escapechar* vazio não é permitido.

Dialect.lineterminator

A string usada para terminar as linhas produzidas pelo `writer`. O padrão é `'\r\n'`.

Nota

A `reader` é codificada para reconhecer `'\r'` ou `'\n'` como fim de linha e ignora *lineterminator*. Esse comportamento pode mudar no futuro.

Dialect.quotechar

Uma string de um caractere usada para colocar entre aspas campos que contêm caracteres especiais, como o *delimiter* ou o *quotechar*, ou que contêm caracteres de nova linha (`'\r'`, `'\n'` ou qualquer um dos caracteres em *lineterminator*). O padrão é `'"'`. Pode ser definido como `None` para evitar o escape de `'"'` se *aspas* estiver definido como `QUOTE_NONE`.

Alterado na versão 3.11: Um *quotechar* vazio não é permitido.

Dialect.quoting

Controla quando as aspas devem ser geradas pelo escritor e reconhecidas pelo leitor. Ele pode assumir qualquer uma das constantes QUOTE_* constants e o padrão é `QUOTE_MINIMAL`. se *quotechar* não for `None` e, do contrário, `QUOTE_NONE`.

Dialect.skipinitialspace

Quando `True`, os espaços imediatamente após o *delimitador* são ignorados. O padrão é `False`. Ao combinar `delimiter=' '` com `skipinitialspace=True`, campos vazios sem aspas não são permitidos.

Dialect.strict

Quando `True`, levanta a exceção `Error` em uma entrada CSV ruim. O padrão é `False`.

Reader objects

Os objetos Reader (instâncias `DictReader` e objetos retornados pela função `reader()`) têm os seguintes métodos públicos:

csvreader.__next__()

Retorna a próxima linha do objeto iterável do leitor como uma lista (se o objeto foi retornado de **`leitor()`**) ou um dict (se for uma instância de `DictReader`), analisado de acordo com a `Dialect` atual. Normalmente, você deve chamar isso de `next(reader)`.

Os objetos Reader possuem os seguintes atributos públicos:

csvreader.dialect

Uma descrição somente leitura do dialeto em uso pelo analisador sintático.

csvreader.line_num

O número de linhas lidas no iterador de origem. Não é o mesmo que o número de registros retornados, pois os registros podem abranger várias linhas.

Os objetos DictReader têm o seguinte atributo público:

DictReader.fieldnames

Se não for passado como parâmetro ao criar o objeto, esse atributo será inicializado no primeiro acesso ou quando o primeiro registro for lido no arquivo.

Writer objects

Objetos `writer` (instâncias e objetos `DictWriter` retornados pela função `writer()`) possuem os seguintes métodos públicos. Uma *row* deve ser

iterável de strings ou números para objetos `writer` e um dicionário mapeando nomes de campos para strings ou números (passando-os por `str()` primeiro) para `DictWriter`. Observe que números complexos são escritos cercados por parênteses. Isso pode causar alguns problemas para outros programas que leem arquivos CSV (supondo que eles aceitem números complexos).

csvwriter.writerow(*row*, /)

Escreve o parâmetro *row* no objeto arquivo do escritor, formatado de acordo com a `Dialect` atual. Retorna o valor de retorno da chamada ao método *write* do objeto arquivo subjacente.

Alterado na versão 3.5: Adicionado suporte a iteráveis arbitrários.

csvwriter.writerows(*rows*, /)

Escreve todos os elementos em *rows* (um iterável de objetos *row*, conforme descrito acima) no objeto arquivo do escritor, formatado de acordo com o dialeto atual.

Os objetos Writer têm o seguinte atributo público:

csvwriter.dialect

Uma descrição somente leitura do dialeto em uso pelo escritor.

Os objetos DictWriter têm o seguinte método público:

DictWriter.writeheader()

Escreve uma linha com os nomes dos campos (conforme especificado no construtor) no objeto arquivo do escritor, formatado de acordo com o dialeto atual. Retorna o valor de retorno da chamada `csvwriter.writerow()` usada internamente.

Adicionado na versão 3.2.

Alterado na versão 3.8: `writeheader()` agora também retorna o valor retornado pelo método `csvwriter.writerow()` que ele usa internamente.

Exemplos

O exemplo mais simples de leitura de um arquivo CSV:

```
import csv
with open('some.csv', newline='') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

Lendo um arquivo com um formato alternativo:

```
import csv
with open('passwd', newline='') as f:
    reader = csv.reader(f, delimiter=':', quoting=csv.QUOTE_NONE
    for row in reader:
        print(row)
```

O exemplo de escrita possível mais simples possível é:

```
import csv
with open('some.csv', 'w', newline='') as f:
    writer = csv.writer(f)
    writer.writerows(someiterable)
```

Since `open()` is used to open a CSV file for reading, the file will by default be decoded into Unicode using UTF-8. To decode a file using a different encoding, use the `encoding` argument of open:

```
import csv
with open('some.csv', newline='', encoding='latin-1') as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

The same applies to writing in something other than the default encoding: specify the encoding argument when opening the output file.

Registrando um novo dialeto:

```
import csv
csv.register_dialect('unixpwd', delimiter=':', quoting=csv.QUOTE
with open('passwd', newline='') as f:
    reader = csv.reader(f, 'unixpwd')
```

Um uso um pouco mais avançado do leitor — capturando e relatando erros:

```
import csv, sys
filename = 'some.csv'
with open(filename, newline='') as f:
    reader = csv.reader(f)
    try:
        for row in reader:
            print(row)
    except csv.Error as e:
        sys.exit(f'file {filename}, line {reader.line_num}: {e}'
```

E embora o módulo não tenha suporte diretamente à análise sintática de strings, isso pode ser feito facilmente:

```
import csv
for row in csv.reader(['one,two,three']):
    print(row)
```

Notas de rodapé

[1] (1,2)

Se `newline=''` não for especificado, quebras de linha inseridas em campos entre aspas não serão interpretadas corretamente e, em plataformas que usam terminações de linha `\r\n` na gravação, um `\r` extra será adicionado. É sempre seguro especificar `newline=''`, já que o módulo csv faz seu próprio tratamento de quebra de linha (universal).

`configparser` — Analisador sintático de arquivo de configuração

Código-fonte: Lib/configparser.py
[https://github.com/python/cpython/tree/3.15/Lib/configparser.py]

Este módulo fornece a classe `ConfigParser`, que implementa uma linguagem de configuração básica, com uma estrutura semelhante à encontrada nos arquivos INI do Microsoft Windows. Você pode usar isso para escrever programas em Python que podem ser facilmente personalizados pelos usuários finais.

Nota

Esta biblioteca *não* interpreta nem escreve os prefixos de tipo de valor usados na versão estendida da sintaxe INI do Registro do Windows.

Aviso

Tenha cuidado quando estiver analisando dados de fontes não-confiáveis. Uma arquivo INI malicioso pode fazer o decodificador consumir recursos consideráveis de CPU e memória. É recomendado limitar o tamanho do dado a ser analisado.

Ver também

Módulo `tomllib`
TOML é um formato bem especificado para arquivos de configuração de aplicações. Ele foi projetado especificamente para ser uma versão

melhorada do INI.

Módulo `shlex`
Suporte para criação de minilinguagens semelhantes a shell Unix que também podem ser usadas para arquivos de configuração de aplicações.

Módulo `json`
O módulo `json` implementa um subconjunto de sintaxe JavaScript que às vezes é usado para configuração, mas não suporta comentários.

Início rápido

Vamos pegar um arquivo de configuração bem básico parecido com este:

```
[DEFAULT]
ServerAliveInterval = 45
Compression = yes
CompressionLevel = 9
ForwardX11 = yes

[forge.example]
User = hg

[topsecret.server.example]
Port = 50022
ForwardX11 = no
```

A estrutura dos arquivos INI é descrita na seção seguinte. Essencialmente, o arquivo consiste em seções, cada uma contendo chaves com valores. As classes **`configparser`** podem ler e escrever tais arquivos. Vamos começar criando o arquivo de configuração acima programaticamente.

```
>>> import configparser
>>> config = configparser.ConfigParser()
>>> config['DEFAULT'] = {'ServerAliveInterval': '45',
...                      'Compression': 'yes',
...                      'CompressionLevel': '9'}
```

```
>>> config['forge.example'] = {}
>>> config['forge.example']['User'] = 'hg'
>>> config['topsecret.server.example'] = {}
>>> topsecret = config['topsecret.server.example']
>>> topsecret['Port'] = '50022'     # mutates the parser
>>> topsecret['ForwardX11'] = 'no'  # same here
>>> config['DEFAULT']['ForwardX11'] = 'yes'
>>> with open('example.ini', 'w') as configfile:
...   config.write(configfile)
...
```

Como você pode ver, podemos tratar um analisador sintático de configuração como um dicionário. Existem diferenças, descritas posteriormente, mas o comportamento é muito próximo do que você esperaria de um dicionário.

Agora que criamos e salvamos um arquivo de configuração, vamos lê-lo e explorar os dados que ele contém.

```
>>> config = configparser.ConfigParser()
>>> config.sections()
[]
>>> config.read('example.ini')
['example.ini']
>>> config.sections()
['forge.example', 'topsecret.server.example']
>>> 'forge.example' in config
True
>>> 'python.org' in config
False
>>> config['forge.example']['User']
'hg'
>>> config['DEFAULT']['Compression']
'yes'
>>> topsecret = config['topsecret.server.example']
>>> topsecret['ForwardX11']
'no'
>>> topsecret['Port']
'50022'
>>> for key in config['forge.example']:
...     print(key)
user
```

```
compressionlevel
serveraliveinterval
compression
forwardx11
>>> config['forge.example']['ForwardX11']
'yes'
```

Como podemos ver acima, a API é bastante simples. A única mágica envolve a seção `DEFAULT` que fornece valores padrão para todas as outras seções [[1]]. Observe também que as chaves nas seções não diferenciam maiúsculas de minúsculas e são armazenadas em letras minúsculas [[1]].

É possível ler diversas configurações para um único **`ConfigParser`**, onde a configuração adicionada mais recentemente terá a maior prioridade. Para quaisquer chaves repetidas serão usados os valores da configuração mais recente, enquanto os valores das chaves anteriores serão ignorados. O exemplo a seguir lê um arquivo chamado `override.ini`, que irá substituir chaves repetidas do arquivo `example.ini`.

```
[DEFAULT]
ServerAliveInterval = -1
```

```
>>> config_override = configparser.ConfigParser()
>>> config_override['DEFAULT'] = {'ServerAliveInterval': '-1'}
>>> with open('override.ini', 'w') as configfile:
...     config_override.write(configfile)
...
>>> config_override = configparser.ConfigParser()
>>> config_override.read(['example.ini', 'override.ini'])
['example.ini', 'override.ini']
>>> print(config_override.get('DEFAULT', 'ServerAliveInterval'))
-1
```

Este comportamento é equivalente a uma chamada **`ConfigParser.read()`** com vários arquivos passados para o parâmetro *filenames*.

Tipos de dados suportados

Os analisadores sintáticos de configuração não adivinham os tipos de dados dos valores nos arquivos de configuração, sempre os armazenando internamente como strings. Isso significa que se você precisar de outros tipos de dados, deverá converter por conta própria:

```
>>> int(topsecret['Port'])
50022
>>> float(topsecret['CompressionLevel'])
9.0
```

Como essa tarefa é tão comum, os analisadores sintáticos de configuração fornecem uma variedade de métodos getter úteis para manipular com números inteiros, pontos flutuantes e booleanos. O último é o mais interessante porque simplesmente passar o valor para `bool()` não adiantaria nada já que `bool('False')` ainda é `True`. É por isso que os analisadores sintáticos de configuração também fornecem **`getboolean()`**. Este método não diferencia maiúsculas de minúsculas e reconhece valores booleanos de `'yes'`/`'no'`, `'on'`/`'off'`, `'true'` /`'false'` e `'1'`/`'0'` [[1]]. Por exemplo:

```
>>> topsecret.getboolean('ForwardX11')
False
>>> config['forge.example'].getboolean('ForwardX11')
True
>>> config.getboolean('forge.example', 'Compression')
True
```

Além de **`getboolean()`**, os analisadores sintáticos de configuração também fornecem métodos **`getint()`** e **`getfloat()`** equivalentes. Você pode registrar seus próprios conversores e personalizar os fornecidos. [[1]]

Valores reservas

Assim como acontece com um dicionário, você pode usar o método **`get()`** de uma seção para fornecer valores reservas (fallback):

```
>>> topsecret.get('Port')
'50022'
>>> topsecret.get('CompressionLevel')
'9'
```

```
>>> topsecret.get('Cipher')
>>> topsecret.get('Cipher', '3des-cbc')
'3des-cbc'
```

Observe que os valores padrão têm precedência sobre os valores substitutos. Por exemplo, em nosso exemplo a chave `'CompressionLevel'` foi especificada apenas na seção `'DEFAULT'`. Se tentarmos obtê-lo na seção `'topsecret.server.example'`, sempre obteremos o padrão, mesmo se especificarmos um substituto:

```
>>> topsecret.get('CompressionLevel', '3')
'9'
```

Mais uma coisa a ter em conta é que o método **`get()`** no nível do analisador fornece uma interface personalizada e mais complexa, mantida para compatibilidade com versões anteriores. Ao usar este método, um valor substituto pode ser fornecido através do argumento somente-nomeado `fallback`:

```
>>> config.get('forge.example', 'monster',
...            fallback='No such things as monsters')
'No such things as monsters'
```

O mesmo argumento `fallback` pode ser usado com os métodos **`getint()`**, **`getfloat()`** e **`getboolean()`**, por exemplo:

```
>>> 'BatchMode' in topsecret
False
>>> topsecret.getboolean('BatchMode', fallback=True)
True
>>> config['DEFAULT']['BatchMode'] = 'no'
>>> topsecret.getboolean('BatchMode', fallback=True)
False
```

Estrutura dos arquivos INI

Um arquivo de configuração consiste em seções, cada uma liderada por um cabeçalho `[section]`, seguido por entradas de chave/valor separadas por uma string específica (`=` ou `:` por padrão [[1]]) . Por padrão, os nomes das

seções diferenciam maiúsculas de minúsculas, mas as chaves não [[1]]. Os espaços em branco à esquerda e à direita são removidos das chaves e dos valores. Os valores podem ser omitidos se o analisador sintático estiver configurado para permitir [[1]], caso em que o delimitador chave/valor também pode ser omitido. Os valores também podem abranger várias linhas, desde que sejam indentados de forma mais profunda que a primeira linha do valor. Dependendo do modo do analisador sintático, as linhas em branco podem ser tratadas como partes de valores multilinhas ou ignoradas.

Por padrão, um nome de seção válido pode ser qualquer string que não contenha '\n'. Para alterar isso, consulte **`ConfigParser.SECTCRE`**.

O nome da primeira seção pode ser omitido se o analisador sintático estiver configurado para permitir uma seção de nível superior sem nome com `allow_unnamed_section=True`. Neste caso, as chaves/valores podem ser recuperadas por **`UNNAMED SECTION`** como em `config[UNNAMED_SECTION]`.

Os arquivos de configuração podem incluir comentários, prefixados por caracteres específicos (`#` e `;` por padrão [[1]]). Os comentários podem aparecer sozinhos em uma linha vazia, possivelmente indentados. [[1]]

Por exemplo:

```
[Valores Simples]
chave=valor
espaços nas chaves=permitidos
espaços nos valores=permitidos também
espaços em volta do delimitador = obviamente
você também pode usar : para delimitar chaves de valores

[Todos Valores São Strings]
valores como este: 1000000
ou este: 3.14159265359
são tratados como números? : não
inteiros, pontos flutuantes e booleanos são considerados: string
pode usar a API para obter valores convertidos diretamente: verd

[Valores multilinha]
chorus: Eu sou um lenhador e estou bem
    Durmo a noite toda e trabalho o dia inteiro
```

```
[Sem valores]
chave_sem_valor
valor string vazia aqui =

[Você pode usar comentários]
# como este
; ou este

# Por padrão, apenas em uma linha vazia.
# Comentários em linha podem ser prejudiciais porque eles
# impedem os usuários de usar os caracteres delimitadores
# como partes dos valores.
# Isto posto, isso pode ser personalizado.

    [Seções Podem Ser Indentadas]
        podem_ser_valores_também = True
        isso_significa_algo_especial = False
        propósito = formatação para legibilidade
        valores_multilinha = são
            tratados sem problemas desde
            que eles estejam indentados
            mais profundamente do que a
            primeira linha de um valor
        # Mencionamos que também podemos indentar comentários?
```

Seções não nomeadas

O nome da primeira seção (ou único) pode ser omitido e os valores recuperados pelo atributo **UNNAMED_SECTION**.

```
>>> config = """
... option = value
...
... [  Section 2  ]
... another = val
... """
>>> unnamed = configparser.ConfigParser(allow_unnamed_section=Tr
>>> unnamed.read_string(config)
>>> unnamed.get(configparser.UNNAMED_SECTION, 'option')
'value'
```

Interpolação de valores

Além da funcionalidade principal, `ConfigParser` oferece suporte a interpolação. Isso significa que os valores podem ser pré-processados antes de retorná-los das chamadas `get()`.

class configparser.BasicInterpolation

A implementação padrão usada por `ConfigParser`. Ele permite que os valores contenham strings de formato que se referem a outros valores na mesma seção ou valores na seção padrão especial [[1]]. Valores padrão adicionais podem ser fornecidos na inicialização.

Por exemplo:

```
[Paths]
home_dir: /Users
my_dir: %(home_dir)s/lumberjack
my_pictures: %(my_dir)s/Pictures

[Escape]
# use um %% para escapar o sinal de % (% é o único caractere
gain: 80%%
```

No exemplo acima, `ConfigParser` com *interpolation* definido como `BasicInterpolation()` resolveria `%(home_dir)s` para o valor de `home_dir` (`/Users` neste caso). `%(my_dir)s` na verdade resolveria para `/Users/lumberjack`. Todas as interpolações são feitas sob demanda, portanto as chaves usadas na cadeia de referências não precisam ser especificadas em nenhuma ordem específica no arquivo de configuração.

Com `interpolation` definido como `None`, o analisador sintático simplesmente retornaria `%(my_dir)s/Pictures` como o valor de `my_pictures` e `%(home_dir)s/lumberjack` como o valor de `my_dir`.

class configparser.ExtendedInterpolation

Um manipulador alternativo para interpolação que implementa uma sintaxe mais avançada, usada, por exemplo, em `zc.buildout`. A interpolação estendida usa `${section:option}` para denotar um valor de uma seção estrangeira. A interpolação pode abranger vários níveis. Por conveniência, se a parte `section:` for omitida, a interpolação será padronizada para a seção atual (e possivelmente para os valores padrão da seção especial).

Por exemplo, a configuração especificada acima com interpolação básica ficaria assim com interpolação estendida:

```
[Paths]
home_dir: /Users
my_dir: ${home_dir}/lumberjack
my_pictures: ${my_dir}/Pictures

[Escape]
# use a $$ para escapar o sinal de $ ($ é o único caractere c
cost: $$80
```

Valores de outras seções também podem ser obtidos:

```
[Common]
home_dir: /Users
library_dir: /Library
system_dir: /System
macports_dir: /opt/local

[Frameworks]
Python: 3.2
path: ${Common:system_dir}/Library/Frameworks/

[Arthur]
nickname: Two Sheds
last_name: Jackson
my_dir: ${Common:home_dir}/twosheds
my_pictures: ${my_dir}/Pictures
python_dir: ${Frameworks:path}/Python/Versions/${Frameworks:F
```

Acesso através do protocolo de mapeamento

Adicionado na versão 3.2.

Acesso através do protocolo de mapeamento é um nome genérico para funcionalidade que permite usar objetos personalizados como se fossem dicionários. No caso de **`configparser`**, a implementação da interface de mapeamento está usando a notação `parser['section']['option']`.

`parser['section']` em particular retorna um intermediário para os dados da seção no analisador sintático. Isso significa que os valores não são copiados, mas obtidos do analisador sintático original sob demanda. O que é ainda mais importante é que quando os valores são alterados em um intermediário de seção, eles são, na verdade, modificados no analisador sintático original.

Os objetos **`configparser`** se comportam o mais próximo possível dos dicionários reais. A interface de mapeamento é completa e segue a ABC **`MutableMapping`**. No entanto, existem algumas diferenças que devem ser levadas em consideração:

- Por padrão, todas as chaves nas seções são acessíveis sem distinção entre maiúsculas e minúsculas [[1]]. Por exemplo. `for option in parser["section"]` produz apenas nomes de chaves de opção `optionxformada`. Isso significa chaves em letras minúsculas por padrão. Ao mesmo tempo, para uma seção que contém a chave `'a'`, ambas as expressões retornam `True`:

  ```
  "a" in parser["section"]
  "A" in parser["section"]
  ```

- Todas as seções também incluem valores `DEFAULTSECT`, o que significa que `.clear()` em uma seção não pode deixá-la visivelmente vazia. Isso ocorre porque os valores padrão não podem ser excluídos da seção (porque tecnicamente eles não estão lá). Se eles forem substituídos na seção, a exclusão fará com que o valor padrão fique visível novamente. Tentar excluir um valor padrão causa um **`KeyError`**.

- `DEFAULTSECT` não pode ser removido do analisador sintático:
    - tentar excluí-lo levanta **`ValueError`**,
    - `parser.clear()` deixa-o intacto,
    - `parser.popitem()` nunca o retorna.
- `parser.get(section, option, **kwargs)` - o segundo argumento **não** é um valor substituto. Observe, entretanto, que os métodos `get()` em nível de seção são compatíveis tanto com o protocolo de mapeamento quanto com a API clássica do configparser.
- `parser.items()` é compatível com o protocolo de mapeamento (retorna uma lista de pares *section_name*, *section_proxy* incluindo o DEFAULTSECT). Entretanto, este método também pode ser invocado com argumentos: `parser.items(section, raw, vars)`. A última chamada retorna uma lista de pares *option*, *value* para uma `section` especificada, com todas as interpolações expandidas (a menos que `raw=True` seja fornecido).

O protocolo de mapeamento é implementado sobre a API legada existente para que as subclasses que substituem a interface original ainda tenham mapeamentos funcionando conforme o esperado.

Personalizando o comportamento do analisador sintático

Existem quase tantas variantes de formato INI quanto aplicações que o utilizam. **`configparser`** percorre um longo caminho para fornecer suporte para o maior conjunto sensato de estilos INI disponíveis. A funcionalidade padrão é determinada principalmente pelo histórico e é muito provável que você queira personalizar alguns dos recursos.

A maneira mais comum de alterar a forma como um analisador sintático de configuração específico funciona é usar as opções **`__init__()`**:

- *defaults*, valor padrão: `None`

Esta opção aceita um dicionário de pares chave-valor que será inicialmente colocado na seção `DEFAULT`. Isso é uma maneira elegante de oferecer suporte a arquivos de configuração concisos que não especificam valores iguais ao padrão documentado.

Dica: se você deseja especificar valores padrão para uma seção específica, use **`read_dict()`** antes de ler o arquivo real.

- *dict_type*, valor padrão: **`dict`**

  Esta opção tem um grande impacto no comportamento do protocolo de mapeamento e na aparência dos arquivos de configuração gravados. Com o dicionário padrão, cada seção é armazenada na ordem em que foram adicionadas ao analisador sintático. O mesmo vale para opções dentro das seções.

  Um tipo de dicionário alternativo pode ser usado, por exemplo, para ordenar as seções e opções ao fazer *write-back*.

  Observação: existem maneiras de adicionar um conjunto de pares de valores-chave em uma única operação. Quando você usa um dicionário regular nessas operações, a classificação das chaves será ordenada. Por exemplo:

```
>>> parser = configparser.ConfigParser()
>>> parser.read_dict({'seção1': {'chave1': 'valor1',
...                              'chave2': 'valor2',
...                              'chave3': 'valor3'},
...                   'seção2': {'chaveA': 'valorA',
...                              'chaveB': 'valorB',
...                              'chaveC': 'valorC'},
...                   'seção3': {'foo': 'x',
...                              'bar': 'y',
...                              'baz': 'z'}
... })
>>> parser.sections()
['seção1', 'seção2', 'seção3']
>>> [option for option in parser['seção3']]
['foo', 'bar', 'baz']
```

- *allow_no_value*, valor padrão: `False`

  Alguns arquivos de configuração são conhecidos por incluir configurações sem valores, mas que de outra forma estão em conformidade com a sintaxe suportada por **`configparser`**. O parâmetro *allow_no_value* para o construtor pode ser usado para indicar que tais valores devem ser aceitos:

```
>>> import configparser

>>> sample_config = """
... [mysqld]
...   user = mysql
...   pid-file = /var/run/mysqld/mysqld.pid
...   skip-external-locking
...   old_passwords = 1
...   skip-bdb
...   # não precisamos de ACID hoje
...   skip-innodb
... """
>>> config = configparser.ConfigParser(allow_no_value=True)
>>> config.read_string(sample_config)

>>> # Configurações com valores são tratados como antes:
>>> config["mysqld"]["user"]
'mysql'

>>> # Configurações sem valores fornecem None:
>>> config["mysqld"]["skip-bdb"]

>>> # Configurações que não são especificadas ainda levantar
>>> config["mysqld"]["não-existe"]
Traceback (most recent call last):
  ...
KeyError: 'não-existe'
```

- *delimiters*, valor padrão: `('=', ':')`

  Delimitadores são substrings que delimitam chaves de valores dentro de uma seção. A primeira ocorrência de uma substring delimitadora

em uma linha é considerada um delimitador. Isso significa que os valores (mas não as chaves) podem conter os delimitadores.

Veja também o argumento *space_around_delimiters* para **`ConfigParser.write()`**.

- *comment_prefixes*, valor padrão: `('#', ';')`

- *inline_comment_prefixes*, valor padrão: `None`

  Prefixos de comentários são strings que indicam o início de um comentário válido em um arquivo de configuração. *comment_prefixes* são usados apenas em linhas vazias (opcionalmente indentadas), enquanto *inline_comment_prefixes* pode ser usado após cada valor válido (por exemplo, nomes de seções, opções e linhas vazias também). Por padrão, os comentários embutidos estão desabilitados e `'#'` e `';'` são usados como prefixos para comentários de linha inteira.

  Alterado na versão 3.2: Nas versões anteriores do **`configparser`**, o comportamento correspondia a `comment_prefixes=('#',';')` e `inline_comment_prefixes=(';',)`.

  Observe que os analisadores sintáticos de configuração não oferecem suporte a escape de prefixos de comentários, portanto, usar *inline_comment_prefixes* pode impedir que os usuários especifiquem valores de opção com caracteres usados como prefixos de comentários. Em caso de dúvida, evite definir *inline_comment_prefixes*. Em qualquer circunstância, a única maneira de armazenar caracteres de prefixo de comentário no início de uma linha em valores multilinha é interpolar o prefixo, por exemplo:

```
>>> from configparser import ConfigParser, ExtendedInterpola
>>> parser = ConfigParser(interpolation=ExtendedInterpolatio
>>> # the default BasicInterpolation could be used as well
>>> parser.read_string("""
... [DEFAULT]
... hash = #
...
... [hashes]
```

```
... shebang =
...   ${hash}!/usr/bin/env python
...   ${hash} -*- coding: utf-8 -*-
...
... extensions =
...   enabled_extension
...   another_extension
...   #disabled_by_comment
...   yet_another_extension
...
... interpolation not necessary = if # is not at line start
... even in multiline values = line #1
...   line #2
...   line #3
... """)
>>> print(parser['hashes']['shebang'])

#!/usr/bin/env python
# -*- coding: utf-8 -*-
>>> print(parser['hashes']['extensions'])

enabled_extension
another_extension
yet_another_extension
>>> print(parser['hashes']['interpolation not necessary'])
if # is not at line start
>>> print(parser['hashes']['even in multiline values'])
line #1
line #2
line #3
```

- *strict*, valor padrão: `True`

  Ao definir como `True`, o analisador sintático não permitirá nenhuma seção ou opção duplicada durante a leitura de uma única fonte (usando **`read_file()`**, **`read_string()`** ou **`read_dict()`**). Recomenda-se usar analisadores sintáticos estritos em novas aplicações.

  Alterado na versão 3.2: Nas versões anteriores do **`configparser`**, o comportamento correspondia a `strict=False`.

- *empty_lines_in_values*, valor padrão: `True`

  Em analisadores sintáticos de configuração, os valores podem abranger várias linhas, desde que sejam mais indentados do que a chave que os contém. Por padrão, os analisadores sintáticos também permitem que linhas vazias façam parte de valores. Ao mesmo tempo, as chaves podem ser indentados arbitrariamente para melhorar a legibilidade. Consequentemente, quando os arquivos de configuração ficam grandes e complexos, é fácil para o usuário perder o controle da estrutura do arquivo. Tomemos por exemplo:

  ```
  [Seção]
  chave = multilinha
    valor com uma pegadinha

   esta = ainda é parte do valor multilinha de 'chave'
  ```

  Isso pode ser especialmente problemático para o usuário ver se está usando uma fonte proporcional para editar o arquivo. É por isso que quando sua aplicação não precisa de valores com linhas vazias, você deve considerar proibi-los. Isso fará com que as linhas vazias dividam as chaves sempre. No exemplo acima, seriam produzidas duas chaves, `key` e `this`.

- *default_section*, valor padrão: `configparser.DEFAULTSECT` (isto é: `"DEFAULT"`)

  A convenção de permitir uma seção especial de valores padrão para outras seções ou fins de interpolação é um conceito poderoso desta biblioteca, permitindo aos usuários criar configurações declarativas complexas. Esta seção normalmente é chamada de `"DEFAULT"`, mas pode ser personalizada para apontar para qualquer outro nome de seção válido. Alguns valores típicos incluem: `"general"` ou `"common"`. O nome fornecido é usado para reconhecer seções padrão ao ler de qualquer fonte e é usado ao gravar a configuração em um arquivo. Seu valor atual pode ser recuperado usando o atributo `parser_instance.default_section` e pode ser modificado em tempo

de execução (ou seja, para converter arquivos de um formato para outro).

- *interpolation*, valor padrão: `configparser.BasicInterpolation`

  O comportamento de interpolação pode ser personalizado fornecendo um manipulador personalizado por meio do argumento *interpolation*. `None` pode ser usado para desligar completamente a interpolação, `ExtendedInterpolation()` fornece uma variante mais avançada inspirada em `zc.buildout`. Mais sobre o assunto na seção de documentação dedicada. **`RawConfigParser`** tem um valor padrão de `None`.

- *converters*, valor padrão: não definido

  Os analisadores sintáticos de configuração fornecem getters de valor de opção que realizam conversão de tipo. Por padrão, **`getint()`**, **`getfloat()`** e **`getboolean()`** são implementados. Caso outros getters sejam desejáveis, os usuários podem defini-los em uma subclasse ou passar um dicionário onde cada chave é um nome do conversor e cada valor é um chamável que implementa a referida conversão. Por exemplo, passar `{'decimal': decimal.Decimal}` adicionaria **`getdecimal()`** no objeto analisador sintático e em todos os proxies de seção. Em outras palavras, será possível escrever `parser_instance.getdecimal('section', 'key', fallback=0)` e `parser_instance['section'].getdecimal('key', 0)`.

  Se o conversor precisar acessar o estado do analisador sintático, ele poderá ser implementado como um método em uma subclasse do analisador sintático de configuração. Se o nome deste método começar com `get`, ele estará disponível em todos os intermediários de seção, na forma compatível com dict (veja o exemplo `getdecimal()` acima).

Uma personalização mais avançada pode ser obtida substituindo os valores padrão desses atributos do analisador sintático. Os padrões são definidos nas classes, portanto podem ser substituídos por subclasses ou por atribuição de atributos.

ConfigParser.BOOLEAN_STATES

Por padrão, ao usar **`getboolean()`**, os analisadores sintáticos de configuração consideram os seguintes valores `True`: `'1'`, `'yes'`, `'true'`, `'on'` e os seguintes valores `False`: `'0'`, `'no'`, `'false'`, `'off'`. Você pode substituir isso especificando um dicionário personalizado de strings e seus resultados booleanos. Por exemplo:

```
>>> custom = configparser.ConfigParser()
>>> custom['seção1'] = {'funky': 'nope'}
>>> custom['seção1'].getboolean('funky')
Traceback (most recent call last):
...
ValueError: Not a boolean: nope
>>> custom.BOOLEAN_STATES = {'sure': True, 'nope': False}
>>> custom['seção1'].getboolean('funky')
False
```

Outros pares booleanos típicos incluem `accept`/`reject` ou `enabled`/`disabled`.

ConfigParser.optionxform(*option*)

Este método transforma nomes de opções em cada operação de leitura, obtenção ou definição. O padrão converte o nome em letras minúsculas. Isso também significa que quando um arquivo de configuração for gravado, todas as chaves estarão em letras minúsculas. Substitua esse método se for inadequado. Por exemplo:

```
>>> config = """
... [Seção1]
... Chave = Valor
...
... [Seção2]
... OutraChave = Valor
... """
>>> typical = configparser.ConfigParser()
>>> typical.read_string(config)
>>> list(typical['Seção1'].keys())
['chave']
>>> list(typical['Seção2'].keys())
['outrachave']
```

```
>>> custom = configparser.RawConfigParser()
>>> custom.optionxform = lambda option: option
>>> custom.read_string(config)
>>> list(custom['Seção1'].keys())
['Chave']
>>> list(custom['Seção2'].keys())
['OutraChave']
```

Nota

A função optionxform transforma nomes de opções em um formato canônico. Esta deve ser uma função idempotente: se o nome já estiver na forma canônica, deverá ser retornado inalterado.

ConfigParser.SECTCRE

Uma expressão regular compilada usada para analisar cabeçalhos de seção. O padrão corresponde a `[section]` para o nome `"section"`. O espaço em branco é considerado parte do nome da seção, portanto `[ larch ]` será lido como uma seção de nome `" larch "`. Substitua esse atributo se for inadequado. Por exemplo:

```
>>> import re
>>> config = """
... [Seção 1]
... opção = valor
...
... [  Seção 2  ]
... outro = valor
... """
>>> typical = configparser.ConfigParser()
>>> typical.read_string(config)
>>> typical.sections()
['Seção 1', '  Seção 2  ']
>>> custom = configparser.ConfigParser()
>>> custom.SECTCRE = re.compile(r"\[ *(?P<header>[^]]+?) *\]'
>>> custom.read_string(config)
>>> custom.sections()
['Seção 1', 'Seção 2']
```

Nota

Embora os objetos ConfigParser também usem um atributo `OPTCRE` para reconhecer linhas de opção, não é recomendado substituí-lo porque isso interferiria nas opções do construtor *allow_no_value* e *delimiters*.

Exemplos de APIs legadas

Principalmente por questões de retrocompatibilidade, **`configparser`** fornece também uma API legada com métodos `get`/`set` explícitos. Embora existam casos de uso válidos para os métodos descritos abaixo, o acesso ao protocolo de mapeamento é preferido para novos projetos. A API legada é às vezes mais avançada, de baixo nível e totalmente contraintuitiva.

Um exemplo de escrita em um arquivo de configuração:

```
import configparser

config = configparser.RawConfigParser()

# Observe que usando as funções set do RawConfigParser, você poc
# valores não string a chaves internamente, mas receberá um erro
# gravar em um arquivo ou quando obtê-lo em modo não bruta. Defi
# usando o protocolo de mapeamento ou set() do ConfigParser não
# tais atribuições ocorram.
config.add_section('Seção1')
config.set('Seção1', 'an_int', '15')
config.set('Seção1', 'a_bool', 'true')
config.set('Seção1', 'a_float', '3.1415')
config.set('Seção1', 'baz', 'divertido')
config.set('Seção1', 'bar', 'Python')
config.set('Seção1', 'foo', '%(bar)s é %(baz)s!')

# Escrevendo nosso arquivo de configuração em 'example.cfg'
with open('example.cfg', 'w') as configfile:
    config.write(configfile)
```

Um exemplo de leitura do arquivo de configuração novamente:

```
import configparser

config = configparser.RawConfigParser()
config.read('example.cfg')

# getfloat() levanta uma exceção se o valor não for um ponto flu
# getint() e getboolean() também fazem isso para seus respectivo
a_float = config.getfloat('Seção1', 'a_float')
an_int = config.getint('Seção1', 'an_int')
print(a_float + an_int)

# Note que a próxima saída não interpola '%(bar)s' ou '%(baz)s'.
# Isso ocorre porque estamos usando RawConfigParser().
if config.getboolean('Seção1', 'a_bool'):
    print(config.get('Seção1', 'foo'))
```

Para obter interpolação, use **`ConfigParser`**:

```
import configparser

cfg = configparser.ConfigParser()
cfg.read('example.cfg')

# Defina o argumento *raw* opcional de get() para True se você d
# interpolação em uma única operação de get.
print(cfg.get('Seção1', 'foo', raw=False))  # -> "Python é diver
print(cfg.get('Seção1', 'foo', raw=True))   # -> "%(bar)s é %(ba

# O argumento opcional *vars* é um dicionário com membros que vã
# precedência na interpolação.
print(cfg.get('Seção1', 'foo', vars={'bar': 'Documentação',
                                     'baz': 'maligna'}))

# The optional *fallback* argument can be used to provide a fall
print(cfg.get('Seção1', 'foo'))
      # -> "Python é divertido!"

print(cfg.get('Seção1', 'foo', fallback='Monty não é.'))
      # -> "Python é divertido!"
```

```
print(cfg.get('Seção1', 'monstro', fallback='Não existem monstro
      # -> "Não existem monstros."

# Um print(cfg.get('Seção1', 'monster')) levantaria NoOptionErro
# mas nós também podemos usar:

print(cfg.get('Seção1', 'mosntro', fallback=None))
      # -> None
```

Os valores padrão estão disponíveis em ambos os tipos de ConfigParsers. Eles são usados em interpolação se uma opção usada não estiver definida em outro lugar.

```
import configparser

# Nova instância com 'bar' e 'baz' padrão para 'A vida' e 'difíc
config = configparser.ConfigParser({'bar': 'A vida', 'baz': 'dif
config.read('example.cfg')

print(config.get('Seção1', 'foo'))     # -> "Python é divertido!
config.remove_option('Seção1', 'bar')
config.remove_option('Seção1', 'baz')
print(config.get('Seção1', 'foo'))     # -> "A vida é difícil!"
```

Objetos ConfigParser

class configparser.ConfigParser(*defaults=None*, *dict_type=dict*, *allow_no_value=False*, ***, *delimiters=('=', ':')*, *comment_prefixes=('#', ';')*, *inline_comment_prefixes=None*, *strict=True*, *empty_lines_in_values=True*, *default_section=configparser.DEFAULTSECT*, *interpolation=BasicInterpolation()*, *converters={}*, *allow_unnamed_section=False*)

O principal analisador sintático de configuração. Quando *defaults* é fornecido, ele é inicializado no dicionário de padrões intrínsecos. Quando *dict_type* for fornecido, ele será usado para criar os objetos dicionário para a lista de seções, para as opções dentro de uma seção e para os valores padrão.

Quando *delimiters* são fornecidos, eles são usados como o conjunto de substrings que dividem chaves de valores. Quando *comment_prefixes* for fornecido, ele será usado como o conjunto de substrings que prefixam comentários em linhas vazias. Os comentários podem ser indentados. Quando *inline_comment_prefixes* for fornecido, ele será usado como o conjunto de substrings que prefixam comentários em linhas não vazias.

Quando *strict* for `True` (o padrão), o analisador sintático não permitirá nenhuma seção ou opção duplicada durante a leitura de uma única fonte (arquivo, string ou dicionário), levantando **`DuplicateSectionError`** ou **`DuplicateOptionError`**. Quando *empty_lines_in_values* é `False` (padrão: `True`), cada linha vazia marca o fim de uma opção. Caso contrário, as linhas vazias internas de uma opção multilinha serão mantidas como parte do valor. Quando *allow_no_value* for `True` (padrão: `False`), opções sem valores serão aceitas; o valor mantido para estes é `None` e eles são serializados sem o delimitador final.

Quando *default_section* é fornecido, ele especifica o nome da seção especial que contém valores padrão para outras seções e propósitos de interpolação (normalmente chamada de `"DEFAULT"`). Este valor pode ser recuperado e alterado em tempo de execução usando o atributo de instância `default_section`. Isso não reavaliará um arquivo de configuração já analisado, mas será usado ao escrever configurações analisadas em um novo arquivo de configuração.

O comportamento de interpolação pode ser personalizado fornecendo um manipulador personalizado por meio do argumento *interpolation*. `None` pode ser usado para desligar completamente a interpolação, `ExtendedInterpolation()` fornece uma variante mais avançada inspirada em `zc.buildout`. Mais sobre o assunto na seção de documentação dedicada.

Todos os nomes de opções usados na interpolação serão passados através do método **`optionxform()`** assim como qualquer outra referência de nome de opção. Por exemplo, usando a implementação padrão de **`optionxform()`** (que converte nomes de opções para letras minúsculas), os valores `foo %(bar)s` e `foo %(BAR)s` são equivalentes.

Quando *converters* é fornecido, deve ser um dicionário onde cada chave representa o nome de um conversor de tipo e cada valor é um chamável implementando a conversão de string para o tipo de dados desejado. Cada conversor obtém seu próprio método **`get*()`** correspondente no objeto analisador sintático e nos intermediários de seção.

Quando *allow_unnamed_section* é `True` (padrão: `False`), o primeiro nome da seção pode ser omitido. Veja a seção "Seções não nomeadas".

É possível ler diversas configurações para um único **`ConfigParser`**, onde a configuração adicionada mais recentemente terá a maior prioridade. Para quaisquer chaves repetidas serão usados os valores da configuração mais recente, enquanto os valores das chaves anteriores serão ignorados. O exemplo a seguir lê um arquivo chamado `override.ini`, que irá substituir chaves repetidas do arquivo `example.ini`.

```
[DEFAULT]
ServerAliveInterval = -1
```

```
>>> config_override = configparser.ConfigParser()
>>> config_override['DEFAULT'] = {'ServerAliveInterval': '-1'
>>> with open('override.ini', 'w') as configfile:
...     config_override.write(configfile)
...
>>> config_override = configparser.ConfigParser()
>>> config_override.read(['example.ini', 'override.ini'])
['example.ini', 'override.ini']
>>> print(config_override.get('DEFAULT', 'ServerAliveInterval
-1
```

Alterado na versão 3.1: O padrão *dict_type* é **`collections.OrderedDict`**.

Alterado na versão 3.2: allow_no_value, *delimiters*, *comment_prefixes*, *strict*, *empty_lines_in_values*, *default_section* e *interpolation* foram adicionados.

Alterado na versão 3.5: O argumento *converters* foi adicionado.

Alterado na versão 3.7: O argumento *defaults* é lido com `read_dict()`, fornecendo um comportamento consistente em todo o analisador: chaves e valores que não são de string são convertidos implicitamente em strings.

Alterado na versão 3.8: O *dict_type* padrão é `dict`, pois agora preserva a ordem de inserção.

Alterado na versão 3.13: Levanta `MultilineContinuationError` quando *allow_no_value* for `True`, e uma chave sem valor continua com uma linha indentada.

Alterado na versão 3.13: O argumento *allow_unnamed_section* foi adicionado.

defaults()

Retorna um dicionário contendo os padrões de toda a instância.

sections()

Retorna uma lista das seções disponíveis; a *seção padrão* não está incluída na lista.

add_section(*section*)

Adiciona uma seção de nome *section* à instância. Se já existir uma seção com o nome fornecido, `DuplicateSectionError` será levantada. Se o nome da *seção padrão* for passado, `ValueError` será levantada. O nome da seção deve ser uma string; caso contrário, `TypeError` será levantada.

Alterado na versão 3.2: Nomes de seções sem string levantam `TypeError`.

has_section(*section*)

Indica se a *section* nomeada está presente na configuração. A *seção padrão* não é reconhecida.

options(*section*)

Retorna uma lista de opções disponíveis na *section* especificada.

has_option(*section*, *option*)

Se a *section* fornecida existir e contiver a *option* fornecida, retorna `True`; caso contrário, retorna `False`. Se a *section* especificada for `None` ou uma string vazia, DEFAULT será presumido.

read(*filenames*, *encoding=None*)

Tenta ler e analisar um iterável de nomes de arquivos, retornando uma lista de nomes de arquivos que foram analisados com sucesso.

Se *filenames* for uma string, um objeto `bytes` ou um objeto caminho ou similar, este parâmetro será tratado como um único nome de arquivo. Se um arquivo nomeado em *filenames* não puder ser aberto, esse arquivo será ignorado. Isso foi projetado para que você possa especificar um iterável de possíveis locais de arquivo de configuração (por exemplo, o diretório atual, o diretório inicial do usuário e algum diretório de todo o sistema) e todos os arquivos de configuração existentes no iterável serão lidos.

Se nenhum dos arquivos nomeados existir, a instância `ConfigParser` conterá um conjunto de dados vazio. Uma aplicação que requer que valores iniciais sejam carregados de um arquivo deve carregar o arquivo ou arquivos necessários usando `read_file()` antes de chamar `read()` para quaisquer arquivos opcionais:

```
import configparser, os

config = configparser.ConfigParser()
config.read_file(open('defaults.cfg'))
config.read(['site.cfg', os.path.expanduser('~/.myapp.cfg
            encoding='cp1250')
```

Alterado na versão 3.2: Adicionado o parâmetro *encoding*. Anteriormente, todos os arquivos eram lidos usando a codificação padrão para `open()`.

Alterado na versão 3.6.1: O parâmetro *filenames* aceita um objeto caminho ou similar.

Alterado na versão 3.7: O parâmetro *filenames* aceita um objeto **`bytes`**.

read_file(*f*, *source=None*)

Lê e analisa dados de configuração de *f* que devem ser iteráveis, produzindo strings Unicode (por exemplo, arquivos abertos em modo texto).

O argumento opcional *source* especifica o nome do arquivo que está sendo lido. Se não for fornecido e *f* tiver um atributo **`name`**, que é usado para *source*; o padrão é `'<???>'`.

Adicionado na versão 3.2: Substitui **`readfp()`**.

read_string(*string*, *source='<string>'*)

Analisa dados de configuração de uma string.

O argumento opcional *source* especifica um nome específico do contexto da string passada. Se não for fornecido, `'<string>'` será usado. Geralmente deve ser um caminho do sistema de arquivos ou uma URL.

Adicionado na versão 3.2.

read_dict(*dictionary*, *source='<dict>'*)

Carrega a configuração de qualquer objeto que forneça um método `items()` dict ou similar. Chaves são nomes de seções, valores são dicionários com chaves e valores que devem estar presentes na seção. Se o tipo de dicionário usado preservar a ordem, as seções e suas chaves serão adicionadas em ordem. Os valores são convertidos automaticamente em strings.

O argumento opcional *source* especifica um nome específico do contexto do dicionário passado. Se não for fornecido, `<dict>` será

usado.

Este método pode ser usado para copiar o estado entre analisadores sintáticos.

Adicionado na versão 3.2.

get(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Obtém um valor de *option* para a *section* nomeada. Se *vars* for fornecido, deverá ser um dicionário. A *option* é pesquisada em *vars* (se fornecido), *section* e em *DEFAULTSECT* nesta ordem. Se a chave não for encontrada e *fallback* for fornecido, ele será usado como um valor alternativo. `None` pode ser fornecido como um valor *fallback*.

Todas as interpolações `'%'` são expandidas nos valores de retorno, a menos que o argumento *raw* seja verdadeiro. Os valores das chaves de interpolação são consultados da mesma maneira que a opção.

Alterado na versão 3.2: Os argumentos *raw*, *vars* e *fallback* são palavras somente-nomeadas para proteger os usuários de tentarem usar o terceiro argumento como substituto *fallback* (especialmente ao usar o protocolo de mapeamento).

getint(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Um método de conveniência que força a *option* na *section* especificada ser um número inteiro. Veja **`get()`** para explicação de *raw*, *vars* e *fallback*.

getfloat(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Um método de conveniência que força a *option* na *section* especificada ser um número de ponto flutuante. Veja **`get()`** para explicação de *raw*, *vars* e *fallback*.

getboolean(*section*, *option*, *, *raw=False*, *vars=None*[, *fallback*])

Um método de conveniência que força a *option* na *section* especificada a um valor booleano. Observe que os valores aceitos para a opção são `'1'`, `'yes'`, `'true'`, e `'on'`, o que fazem com que este método retorne `True`, e `'0'`, `'no'`, `'false'`, e `'off'`, o que fazem com que ele retorne `False`. Esses valores de string são verificados sem distinção entre maiúsculas e minúsculas. Qualquer outro valor fará com que ele levante **`ValueError`**. Veja **`get()`** para explicação de *raw*, *vars* e *fallback*.

items(*raw=False*, *vars=None*)

items(*section*, *raw=False*, *vars=None*)

Quando *section* não é fornecido, retorna uma lista de pares *section_name*, *section_proxy*, incluindo DEFAULTSECT.

Caso contrário, retorna uma lista de pares *name*, *value* para as opções na *section* fornecida. Argumentos opcionais têm o mesmo significado do método **`get()`**.

Alterado na versão 3.8: Os itens presentes em *vars* não aparecem mais no resultado. O comportamento anterior misturava opções reais do analisador sintático com variáveis fornecidas para interpolação.

set(*section*, *option*, *value*)

Se a seção fornecida existir, defina a opção fornecida com o valor especificado; caso contrário, levanta **`NoSectionError`**. *option* e *value* devem ser strings; caso contrário, **`TypeError`** será levantada.

write(*fileobject*, *space_around_delimiters=True*)

Escreve uma representação da configuração no objeto arquivo especificado, que deve ser aberto em modo texto (aceitando strings). Esta representação pode ser analisada por uma futura chamada **`read()`**. Se *space_around_delimiters* for verdadeiro, os delimitadores entre chaves e valores serão envoltos por espaços.

Alterado na versão 3.14: Levanta InvalidWriteError se isso for gravar uma representação que não pode ser analisada com precisão por uma futura chamada **`read()`** deste analisador sintático.

Nota

Os comentários no arquivo de configuração original não são preservados ao escrever a configuração. O que é considerado um comentário depende dos valores fornecidos para *comment_prefix* e *inline_comment_prefix*.

remove_option(*section*, *option*)

Remove a *option* especificada da *section* especificada. Se a seção não existir, levanta **`NoSectionError`**. Se existisse a opção de ser removida, retorna **`True`**; caso contrário, retorna **`False`**.

remove_section(*section*)

Remove a *section* especificada da configuração. Se a seção de fato existiu, retorna `True`. Caso contrário, retorna `False`.

optionxform(*option*)

Transforma o nome da opção *option* conforme encontrado em um arquivo de entrada ou conforme passado pelo código do cliente no formato que deve ser usado nas estruturas internas. A implementação padrão retorna uma versão em minúsculas de *option*; subclasses podem substituir isso ou o código do cliente pode definir um atributo com esse nome nas instâncias para afetar esse comportamento.

Você não precisa criar uma subclasse do analisador sintático para usar esse método; você também pode configurá-lo em uma instância, para uma função que recebe um argumento de string e retorna uma string. Definir como `str`, por exemplo, faria com que sejam diferenciadas as letras maiúsculas das minúsculas nos nomes das opções:

```
cfgparser = ConfigParser()
cfgparser.optionxform = str
```

Observe que ao ler arquivos de configuração, os espaços em branco ao redor dos nomes das opções são removidos antes de

`optionxform()` ser chamado.

configparser.UNNAMED_SECTION

Um objeto especial que representa um nome de seção usado para referenciar a seção sem nome (veja Seções não nomeadas).

configparser.MAX_INTERPOLATION_DEPTH

A profundidade máxima para interpolação recursiva para `get()` quando o parâmetro *raw* é falso. Isso é relevante apenas quando a *interpolation* padrão é usada.

Objetos RawConfigParser

class configparser.RawConfigParser(*defaults=None, dict_type=dict, allow_no_value=False, *, delimiters=('=', ':'), comment_prefixes=('#', ';'), inline_comment_prefixes=None, strict=True, empty_lines_in_values=True, default_section=configparser.DEFAULTSECT, interpolation=BasicInterpolation(), converters={}, allow_unnamed_section=False*)

Variante legada do `ConfigParser`. Ela tem a interpolação desabilitada por padrão e permite nomes de seções não-string, nomes de opções e valores através de seus métodos inseguros `add_section` e `set`, bem como o tratamento de argumentos nomeados legados `defaults=` .

Alterado na versão 3.2: allow_no_value, *delimiters*, *comment_prefixes*, *strict*, *empty_lines_in_values*, *default_section* e *interpolation* foram adicionados.

Alterado na versão 3.5: O argumento *converters* foi adicionado.

Alterado na versão 3.8: O *dict_type* padrão é `dict`, pois agora preserva a ordem de inserção.

Alterado na versão 3.13: O argumento *allow_unnamed_section* foi adicionado.

Nota

Considere usar `ConfigParser`, que verifica os tipos de valores a serem armazenados internamente. Se você não quiser interpolação, você pode usar `ConfigParser(interpolation=None)`.

add_section(*section*)

Adiciona uma seção chamada *section* ou `UNNAMED_SECTION` à instância.

Se a seção com o nome fornecido já existir, `DuplicateSectionError` é levantada. Se o nome da *seção padrão* for passado, `ValueError` é levantada. Se `UNNAMED_SECTION` for passado e o suporte for desabilitado `UnnamedSectionDisabledError` é levantada.

O tipo da *section* não está marcado, o que permite aos usuários criar seções nomeadas sem string. Este comportamento não é compatível e pode causar erros internos.

Alterado na versão 3.14: Adicionado suporte a `UNNAMED_SECTION`.

set(*section*, *option*, *value*)

Se a seção fornecida existir, defina a opção fornecida com o valor especificado; caso contrário, levanta `NoSectionError`. Embora seja possível usar `RawConfigParser` (ou `ConfigParser` com parâmetros *raw* definidos como verdadeiro) para armazenamento *interno* de valores não-string, funcionalidade completa (incluindo interpolação e saída para arquivos) só pode ser alcançado usando valores de string.

Este método permite que os usuários atribuam valores não-string às chaves internamente. Este comportamento não é suportado e causará erros ao tentar escrever em um arquivo ou obtê-lo no modo não bruto. **Use a API do protocolo de mapeamento** que não permite que tais atribuições ocorram.

Exceções

exception configparser.Error

Classe base para todas as outras exceções do **`configparser`**.

exception configparser.NoSectionError

Exceção levantada quando uma seção especificada não é encontrada.

exception configparser.DuplicateSectionError

Exceção levantada se **`add section()`** for chamado com o nome de uma seção que já está presente ou em analisadores sintáticos estritos quando uma seção for encontrada mais de uma vez em um único arquivo de entrada, string ou dicionário.

Alterado na versão 3.2: Adicionados os atributos e parâmetros opcionais *source* e *lineno* a **`__init__()`**.

exception configparser.DuplicateOptionError

Exceção levantada por analisadores sintáticos estritos se uma única opção aparecer duas vezes durante a leitura de um único arquivo, string ou dicionário. Isso detecta erros ortográficos e erros relacionados a diferenciação de letras maiúsculas e minúsculas como, p. ex., um dicionário pode ter duas chaves representando a mesma chave de configuração que não diferencia maiúsculas de minúsculas.

exception configparser.NoOptionError

Exceção levantada quando uma opção especificada não é encontrada na seção especificada.

exception configparser.InterpolationError

Classe base para exceções levantadas quando ocorrem problemas ao executar a interpolação de strings.

exception configparser.InterpolationDepthError

Exceção levantada quando a interpolação de string não pode ser concluída porque o número de iterações excede **`MAX_INTERPOLATION_DEPTH`**. Subclasse de **`InterpolationError`**.

exception configparser.InterpolationMissingOptionError

Exceção levantada quando uma opção referenciada a partir de um valor não existe. Subclasse de `InterpolationError`.

exception configparser.InterpolationSyntaxError

Exceção levantada quando o texto fonte no qual são feitas as substituições não está em conformidade com a sintaxe exigida. Subclasse de `InterpolationError`.

exception configparser.MissingSectionHeaderError

Exceção levantada ao tentar analisar um arquivo que não possui cabeçalhos de seção.

exception configparser.ParsingError

Exceção levantada quando ocorrem erros ao tentar analisar um arquivo.

Alterado na versão 3.12: O atributo `filename` e o argumento do construtor **`__init__()`** foram removidos. Eles estiveram disponíveis usando o nome `source` desde 3.2.

exception configparser.MultilineContinuationError

Exceção levantada quando uma chave sem valor correspondente continua com uma linha indentada.

Adicionado na versão 3.13.

exception configparser.UnnamedSectionDisabledError

Exceção levantada ao tentar usar `UNNAMED SECTION` sem habilitá-lo.

Adicionado na versão 3.14.

exception configparser.InvalidWriteError

Exceção levantada quando uma tentativa de `ConfigParser.write()` não seria analisada com precisão em uma futura chamada de `ConfigParser.read()`.

Ex: escrever uma chave que comece com o padrão `ConfigParser.SECTCRE` seria interpretado como um cabeçalho de seção ao ser lido. Tentar escrever isso levantará esta exceção.

Adicionado na versão 3.14.

Notas de rodapé

[1] (1,2,3,4,5,6,7,8,9,10,11)

Os analisadores sintáticos de configuração permitem muitas personalizações. Caso você tenha interesse em alterar o comportamento descrito na referência da nota de rodapé, consulte a seção Personalizando o comportamento do analisador sintático.

`tomllib` — Analisa arquivos TOML

Código-fonte: Lib/tomllib [https://github.com/python/cpython/tree/3.15/Lib/tomllib]

Este módulo fornece uma interface para analisar TOML 1.1.0 (Tom's Obvious Minimal Language, https://toml.io [https://toml.io/pt/]). Este módulo não oferece suporte para escrever TOML.

Adicionado na versão 3.11: O módulo foi adicionado com suporte ao TOML 1.0.0.

Alterado na versão 3.15: Adicionado suporte ao TOML 1.1.0. Veja O que há de novo para detalhes.

Aviso

Tenha cuidado quando estiver analisando dados de fontes não-confiáveis. Uma string TOML maliciosa pode fazer o decodificador consumir recursos consideráveis de CPU e memória. É recomendado limitar o tamanho do dado a ser analisado.

Ver também

O pacote Tomli-W [https://pypi.org/project/tomli-w/] é um editor de TOML que pode ser usado em conjunto com este módulo, fornecendo uma API de escrita familiar aos usuários da biblioteca padrão: módulos `marshal` e `pickle`.

Ver também

O pacote TOML Kit [https://pypi.org/project/tomlkit/] é uma biblioteca TOML de preservação de estilo com capacidade de leitura e escrita. É uma substituição recomendada para este módulo para edição de arquivos TOML já existentes.

Este módulo define as seguintes funções:

tomllib.load(*fp*, */*, ***, *parse_float=float*)

Lê um arquivo TOML. O primeiro argumento deve ser um objeto arquivo binário e legível. Retorna um `dict`. Converte tipos TOML para Python usando esta tabela de conversão.

parse_float será chamado com a string de cada ponto flutuante (float) do TOML a ser decodificado. Por padrão, isso é equivalente a `float(num_str)`. Isso pode ser usado para usar outro tipo de dados ou analisador sintático para pontos flutuantes do TOML (por exemplo, `decimal.Decimal`). O chamável não deve retornar um `dict` ou um `list`, senão uma exceção `ValueError` é levantada.

Uma exceção `TOMLDecodeError` será levantada no caso de um documento TOML inválido.

tomllib.loads(*s*, */*, ***, *parse_float=float*)

Carrega TOML de um objeto `str`. Retorna um `dict`. Converte tipos TOML para Python usando esta tabela de conversão. O argumento *parse_float* tem o mesmo significado que em `load()`.

Uma exceção `TOMLDecodeError` será levantada no caso de um documento TOML inválido.

As seguintes exceções estão disponíveis:

exception tomllib.TOMLDecodeError(*msg*, *doc*, *pos*)

Subclasse de `ValueError` com os seguintes atributos adicionais:

msg

A mensagem de erro não formatada.

doc
: O documento TOML sendo analisado.

pos
: O índice de *doc* onde a análise falhou.

lineno
: A linha correspondente a *pos*.

colno
: A coluna correspondente a *pos*.

Alterado na versão 3.14: Adicionados os parâmetros *msg*, *doc* e *pos*. Adicionados os atributos `msg`, `doc`, `pos`, `lineno` e `colno`.

Descontinuado desde a versão 3.14: A passagem de argumentos posicionais de forma livre está descontinuada.

Exemplos

Analisando um arquivo TOML:

```
import tomllib

with open("pyproject.toml", "rb") as f:
    data = tomllib.load(f)
```

Analisando uma string TOML:

```
import tomllib

toml_str = """
python-version = "3.11.0"
python-implementation = "CPython"
"""

data = tomllib.loads(toml_str)
```

Tabela de conversão

TOML	Python
documento TOML	dict
string	str
inteiro	int
ponto flutuante	ponto flutuante (configurável com *parse_float*)
booleano	bool
deslocamento de data-hora	datetime.datetime (atributo de `tzinfo` definido com uma instância de `datetime.timezone`)
data-hora local	datetime.datetime (atributo de `tzinfo` definido com `None`)
data local	datetime.date
hora local	datetime.time
array	lista

TOML	Python
tabela	dict
tabela inline	dict
array de tabelas	lista de dicionários

Considerações sobre limites e interoperabilidade

`tomllib` impõe alguns limites aos documentos que consegue processar e preserva detalhes que outros analisadores de TOML têm permissão para ignorar. Ao criar arquivos TOML portáteis, utilize apenas recursos garantidos ou recomendados pelo padrão.

Os detalhes de implementação listados aqui podem mudar em versões futuras do Python.

Tabelas/dicionários

A especificação TOML não garante que os pares chave-valor em documentos e tabelas TOML estejam em qualquer ordem específica.

O **`tomllib`** carrega as entradas de dicionário na ordem em que aparecem na fonte.

Inteiros

O TOML recomenda oferecer suporte a números inteiros no intervalo `range(-2**63, 2**63)`.

O **`tomllib`** utiliza o limite do Python para a conversão de strings em inteiros (4300 dígitos por padrão).

Pontos flutuantes

O TOML recomenda oferecer suporte a, pelo menos, valores IEEE 754 binary64, o que significa que números com mais de 15 dígitos decimais significativos provavelmente serão arredondados.

tomllib usa o `float` do Python por padrão; em muitas plataformas comuns, esse é o formato binary64 recomendado. Consulte `sys.float_info` para obter detalhes.

Limite aninhado

O TOML 1.1.0 não recomenda um limite para a profundidade de aninhamento entre vetores e tabelas. (Um limite de 100 foi proposto para uma versão futura do TOML.)

Na **tomllib**, o nível de aninhamento é limitado principalmente pelo `limite de recursão` do Python. Observe que o código que chama **tomllib** pode contribuir para esse limite.

`netrc` — Arquivo de processamento netrc

Código-fonte: Lib/netrc.py [https://github.com/python/cpython/tree/3.15/Lib/netrc.py]

A classe `netrc` analisa e encapsula o formato do arquivo netrc usado pelo programa Unix **ftp** e outros clientes FTP.

class netrc.netrc([*file*])

Uma instância ou instância de subclasse de `netrc` encapsula dados de um arquivo netrc. O argumento de inicialização, se presente, especifica o arquivo a ser analisado. Se nenhum argumento for fornecido, o arquivo `.netrc` no diretório inicial do usuário – conforme determinado por `os.path.expanduser()` – será lido. Caso contrário, uma exceção `FileNotFoundError` será levantada. Os erros de análise levantam `NetrcParseError` com informações de diagnóstico, incluindo o nome do arquivo, o número da linha e o token final.

Se nenhum argumento for especificado em um sistema POSIX, a presença de senhas no arquivo `.netrc` levantará um `NetrcParseError` se a propriedade ou as permissões do arquivo forem inseguras (pertencentes a um usuário que não seja o usuário executando o processo ou acessível para leitura ou gravação por qualquer outro usuário). Isso implementa um comportamento de segurança equivalente ao do ftp e de outros programas que usam `.netrc`. Essas verificações de segurança não estão disponíveis em plataformas que não oferecem suporte a `os.getuid()`.

Alterado na versão 3.4: Adicionada a verificação de permissão POSIX.

Alterado na versão 3.7: `os.path.expanduser()` é usado para encontrar a localização do arquivo `.netrc` quando *file* não é passado como argumento.

Alterado na versão 3.10: `netrc` tenta a codificação UTF-8 antes de usar a codificação específica da localidade. A entrada no arquivo netrc não precisa mais conter todos os tokens. O valor padrão dos tokens ausentes é uma string vazia. Todos os tokens e seus valores agora podem conter caracteres arbitrários, como espaços em branco e caracteres não ASCII. Se o nome de login for anônimo, ele não acionará a verificação de segurança.

exception netrc.NetrcParseError

Exceção levantada pela classe `netrc` quando erros sintáticos são encontrados no texto fonte. As instâncias desta exceção fornecem três atributos interessantes:

msg

Explicação textual do erro.

filename

O nome do arquivo de origem.

lineno

O número da linha em que o erro foi encontrado.

Objetos netrc

Uma instância da classe `netrc` tem os seguintes métodos:

netrc.authenticators(*host*)

Retorna uma tupla de 3 elementos `(login, conta, senha)` dos autenticadores do *host*. Se o arquivo netrc não contém uma entrada para o host dado, retorna a tupla associada com a entrada padrão. Se não houver nenhum host correspondente nem uma entrada padrão estiver disponível, retorna `None`.

netrc.__repr__()

Despeja os dados da classe como uma string no formato de um arquivo netrc. (Isso descarta os comentários e pode reordenar as entradas.)

Instâncias de **`netrc`** possuem variáveis de instância públicas:

netrc.hosts

Dicionário mapeando nomes de host para tuplas `(login, conta, senha)`. A entrada *default*, se houver, é representada como um pseudo-host por esse nome.

netrc.macros

Dicionário mapeando nomes de macros para listas de strings.

`plistlib` — Gera e analisa arquivos Apple `.plist`

Código-fonte: Lib/plistlib.py [https://github.com/python/cpython/tree/3.15/Lib/plistlib.py]

Este módulo fornece uma interface para ler e escrever os arquivos “property listas (lista de propriedades) usados pela Apple, principalmente no macOS e iOS. Este módulo tem suporte a arquivos plist binários e XML.

O formato de arquivo de lista de propriedades (`.plist`) é uma serialização simples que oferece suporte a tipos básicos de objetos, como dicionários, listas, números e strings. Normalmente, o objeto de nível superior é um dicionário ou um dicionário congelado.

Para escrever e analisar um arquivo plist, use as funções `dump()` e `load()`.

Para trabalhar com dados plist em objetos bytes ou string, use `dumps()` e `loads()`.

Os valores podem ser strings, inteiros, floats, booleanos, tuplas, listas, dicionários (mas somente com chaves de string), objetos `bytes`, `bytearray` ou `datetime.datetime`.

Alterado na versão 3.4: Nova API, API antiga descontinuada. Suporte para plists de formato binário adicionado.

Alterado na versão 3.8: Adicionado suporte para leitura e escrita de tokens `UID` em plists binários, conforme usado por NSKeyedArchiver e NSKeyedUnarchiver.

Alterado na versão 3.9: API antiga removida.

Ver também

Pagina de manual de PList

[https://developer.apple.com/library/archive/documentation/Cocoa/Conceptual/PropertyLists/]

Documentação da Apple sobre o formato de arquivo.

Este módulo define as seguintes funções:

plistlib.load(*fp*, *, *fmt=None*, *dict_type=dict*, *aware_datetime=False*)

Lê um arquivo plist. *fp* deve ser um objeto arquivo binário e legível. Retorna o objeto raiz desempacotado (que geralmente é um dicionário).

fmt é o formato do arquivo e os seguintes valores são válidos:

- `None`: Detecta automaticamente o formato do arquivo
- `FMT_XML`: Formato de arquivo XML
- `FMT_BINARY`: Formato binário plist

O *dict_type* é o tipo usado para dicionários que são lidos do arquivo plist.

Quando *aware_datetime* for verdadeiro, campos com o tipo `datetime.datetime` serão criados como objeto conscientes, com **`tzinfo`** como `datetime.UTC`.

Dados XML para o formato `FMT_XML` são analisados usando o analisador sintático Expat de `xml.parsers.expat` – veja sua documentação para possíveis exceções em XML malformado. Elementos desconhecidos serão simplesmente ignorados pelo analisador de plist.

O analisador sintático levanta `InvalidFileException` quando o arquivo não pode ser analisado.

Adicionado na versão 3.4.

Alterado na versão 3.13: O parâmetro somente-nomeado *aware_datetime* foi adicionado.

plistlib.loads(*data*, *, *fmt=None*, *dict_type=dict*, *aware_datetime=False*)

Carrega um plist de um objeto bytes ou string. Veja `load()` para uma explicação dos argumentos nomeados.

Adicionado na versão 3.4.

Alterado na versão 3.13: data pode ser uma string quando *fmt* é igual a `FMT_XML`.

plistlib.dump(*value*, *fp*, *, *fmt=FMT_XML*, *sort_keys=True*, *skipkeys=False*, *aware_datetime=False*)

Escreve *value* em um arquivo plist. *fp* deve ser um objeto arquivo binário gravável.

O argumento *fmt* especifica o formato do arquivo plist e pode ser um dos seguintes valores:

- `FMT_XML`: Arquivo plist formatado em XML
- `FMT_BINARY`: Arquivo plist formatado em binário

Quando *sort_keys* é verdadeiro (o padrão), as chaves dos dicionários serão escritas ordenadas no plist, caso contrário, elas serão escritas na ordem de iteração do dicionário.

Quando *skipkeys* é falso (o padrão), a função levanta `TypeError` quando uma chave de um dicionário não é uma string, caso contrário, tais chaves são ignoradas.

Quando *aware_datetime* é verdadeiro e qualquer campo com o tipo `datetime.datetime` é definido como um objeto consciente, ele será convertido para o fuso horário UTC antes de escrevê-lo.

Uma `TypeError` será levantada se o objeto for de um tipo não suportado ou um contêiner que contém objetos de tipos não suportados.

Uma `OverflowError` será levantada para valores inteiros que não podem ser representados em arquivos plist (binários).

Adicionado na versão 3.4.

Alterado na versão 3.13: O parâmetro somente-nomeado *aware_datetime* foi adicionado.

plistlib.dumps(*value*, *, *fmt=FMT_XML*, *sort_keys=True*, *skipkeys=False*, *aware_datetime=False*)

Retorna *value* como um objeto bytes formatado em plist. Veja a documentação para `dump()` para uma explicação dos argumentos nomeados desta função.

Adicionado na versão 3.4.

As seguintes classes estão disponíveis:

class plistlib.UID(*data*)

Envolve um `int`. Isso é usado ao ler ou escrever dados codificados NSKeyedArchiver, que contém UID (veja o manual de PList).

data

Valor inteiro do UID. Deve estar no intervalo `0 <= data < 2**64`.

Adicionado na versão 3.8.

As seguintes constantes estão disponíveis:

plistlib.FMT_XML

O formato XML para arquivos plist.

Adicionado na versão 3.4.

plistlib.FMT_BINARY

O formato binário para arquivos plist

Adicionado na versão 3.4.

O módulo define as seguintes exceções:

exception plistlib.InvalidFileException

Levantada quando um arquivo não puder ser analisado.

Adicionado na versão 3.4.

Exemplos

Gerando um plist:

```
import datetime as dt
import plistlib

pl = dict(
    aString = "Doodah",
    aList = ["A", "B", 12, 32.1, [1, 2, 3]],
    aFloat = 0.1,
    anInt = 728,
    aDict = dict(
        anotherString = "<hello & hi there!>",
        aThirdString = "M\xe4ssig, Ma\xdf",
        aTrueValue = True,
        aFalseValue = False,
    ),
    someData = b"<binary gunk>",
    someMoreData = b"<lots of binary gunk>" * 10,
    aDate = dt.datetime.now()
)
print(plistlib.dumps(pl).decode())
```

Analisando um plist:

```
import plistlib

plist = b"""<plist version="1.0">
<dict>
    <key>foo</key>
    <string>bar</string>
</dict>
</plist>"""
pl = plistlib.loads(plist)
print(pl["foo"])
```

Serviços Criptográficos

Os módulos descritos neste capítulo implementam vários algoritmos de uma natureza criptográfica. Eles estão disponíveis a critério da instalação. Aqui está uma visão geral:

- `hashlib` — Resumos de mensagens e hashes seguros
  - Algoritmos de hash
  - Uso
  - Construtores
  - Atributos
  - Objetos hash
  - Resumos de comprimento variável de SHAKE
  - Hash de arquivo
  - Derivação de chave
  - BLAKE2
    - Criando objetos hash
    - Constantes
    - Exemplos
      - Simple hashing
      - Using different digest sizes
      - Keyed hashing
      - Randomized hashing
      - Personalization
      - Modo árvore
    - Credits
- `hmac` — Código de autenticação de mensagem com chave hash
- `secrets` — Gera números aleatórios seguros para gerenciar segredos
  - Números aleatórios
  - Gerando tokens
    - Quantos bytes os tokens devem usar?
  - Outras funções
  - Receitas e melhores práticas

`hashlib` — Resumos de mensagens e hashes seguros

Código-fonte: Lib/hashlib.py [https://github.com/python/cpython/tree/3.15/Lib/hashlib.py]

Este módulo implementa uma interface comum para diversos algoritmos de hash. Estão incluídos os algoritmos de hash seguros FIPS SHA224, SHA256, SHA384 e SHA512 (definidos no padrão FIPS 180-4 [https://csrc.nist.gov/news/2023/decision-to-revise-fips-180-4]), a série SHA-3 (definida no padrão FIPS 202 [https://csrc.nist.gov/pubs/fips/202/final]), bem como os algoritmos legados SHA1 (anteriormente parte do FIPS [https://csrc.nist.gov/pubs/fips/180-4/upd1/final]) e o algoritmo MD5 (definido em internet **RFC 1321** [https://datatracker.ietf.org/doc/html/rfc1321.html]).

Nota

Se você quiser as funções de hash adler32 ou crc32, elas estão disponíveis no módulo `zlib`.

Algoritmos de hash

Há um método construtor nomeado para cada tipo de *hash*. Todos retornam um objeto hash com a mesma interface simples. Por exemplo: use `sha256()` para criar um objeto hash SHA-256. Agora você pode alimentar este objeto com objetos bytes ou similares (normalmente `bytes`) usando o método `update`. A qualquer momento, você pode solicitar o *digest* da concatenação dos dados alimentados até o momento usando os métodos `digest()` ou `hexdigest()`.

Para permitir multithreading, a GIL do Python é liberada ao calcular um hash fornecido com mais de 2047 bytes de dados de uma só vez em seu

construtor ou método `.update`.

Constructors for hash algorithms that are always present in this module are `md5()`, `sha1()`, `sha224()`, `sha256()`, `sha384()`, `sha512()`, `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, `shake_128()`, `shake_256()`, `blake2b()`, and `blake2s()`. These correspond to `algorithms_guaranteed`.

Any of these may nonetheless be missing or blocked in unusual environments, such as a rare “FIPS compliant” build of Python or when OpenSSL’s “FIPS mode” is configured to exclude some algorithms from its default provider. Calling the constructor of an algorithm that is unavailable raises `ValueError`.

Additional algorithms may also be available if your Python distribution’s **`hashlib`** was linked against a build of OpenSSL that provides others. Others *are not guaranteed available* on all installations and will only be accessible by name via `new()`. See `algorithms_available`.

Aviso

Alguns algoritmos apresentam vulnerabilidades conhecidas em colisões de hash (incluindo MD5 e SHA1). Consulte Ataques a algoritmos de hash criptográficos [https://en.wikipedia.org/wiki/Cryptographic_hash_function#Attacks_on_cryptographic_hash_algorithms] e a seção hashlib-seealso no final deste documento.

Adicionado na versão 3.6: Os construtores SHA3 (Keccak) e SHAKE `sha3_224()`, `sha3_256()`, `sha3_384()`, `sha3_512()`, `shake_128()`, `shake_256()` foram adicionados. `blake2b()` e `blake2s()` foram adicionados.

Alterado na versão 3.9: Todos os construtores de hashlib aceitam um argumento somente-nomeado *usedforsecurity* com o valor padrão `True`. Um valor falso permite o uso de algoritmos de hash inseguros e bloqueados em ambientes restritos. `False` indica que o algoritmo de hash não é usado em um contexto de segurança, por exemplo, como uma função de compressão unidirecional não criptográfica.

Alterado na versão 3.9: O hashlib agora usa SHA3 e SHAKE do OpenSSL, se ele os fornecer.

Alterado na versão 3.12: Para qualquer um dos algoritmos MD5, SHA1, SHA2 ou SHA3 que o OpenSSL vinculado não fornece, recorremos a uma implementação verificada do projeto HACL* [https://github.com/hacl-star/hacl-star].

Descontinuado desde a versão 3.15, será removido na versão 3.19: The undocumented `string` keyword parameter in **`_hashlib.new()`** and hash-named constructors such as **`_md5.md5()`** is deprecated. Prefer passing the initial data as a positional argument for maximum backwards compatibility.

Uso

Para obter o resumo da string de bytes `b"Nobody inspects the spammish repetition"`:

```
>>> import hashlib
>>> m = hashlib.sha256()
>>> m.update(b"Nobody inspects")
>>> m.update(b" the spammish repetition")
>>> m.digest()
b'\x03\x1e\xdd}Ae\x15\x93\xc5\xfe\\\x00o\xa5u+7\xfd\xdf\xf7\xbcN
>>> m.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b940
```

Mais condensado:

```
>>> hashlib.sha256(b"Nobody inspects the spammish repetition").h
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b940
```

Construtores

hashlib.new(*name*, [*data*,]*, *usedforsecurity=True*)

É um construtor genérico que recebe a string *name* do algoritmo desejado como seu primeiro parâmetro. Ele também existe para permitir

acesso aos hashes listados acima, bem como a quaisquer outros algoritmos que sua biblioteca OpenSSL possa oferecer.

Usando `new()` com um nome de algoritmo:

```
>>> h = hashlib.new('sha256')
>>> h.update(b"Nobody inspects the spammish repetition")
>>> h.hexdigest()
'031edd7d41651593c5fe5c006fa5752b37fddff7bc4e843aa6af0c950f4b940
```

hashlib.md5([*data*,]*, *usedforsecurity=True*)

hashlib.sha1([*data*,]*, *usedforsecurity=True*)

hashlib.sha224([*data*,]*, *usedforsecurity=True*)

hashlib.sha256([*data*,]*, *usedforsecurity=True*)

hashlib.sha384([*data*,]*, *usedforsecurity=True*)

hashlib.sha512([*data*,]*, *usedforsecurity=True*)

hashlib.sha3_224([*data*,]*, *usedforsecurity=True*)

hashlib.sha3_256([*data*,]*, *usedforsecurity=True*)

hashlib.sha3_384([*data*,]*, *usedforsecurity=True*)

hashlib.sha3_512([*data*,]*, *usedforsecurity=True*)

Construtores nomeados como esses são mais rápidos do que passar um nome de algoritmo para `new()`.

Atributos

O hashlib fornece os seguintes atributos de módulo constantes:

hashlib.algorithms_guaranteed

Um conjunto contendo os nomes dos algoritmos de hash com suporte garantido por este módulo em todas as plataformas. Observe que “md5” está nesta lista, apesar de alguns fornecedores originais oferecerem uma estranha construção Python “compatível com FIPS” que o exclui.

Adicionado na versão 3.2.

hashlib.algorithms_available

Um conjunto contendo os nomes dos algoritmos de hash disponíveis no interpretador Python em execução. Esses nomes serão reconhecidos quando passados para `new()`. `algorithms_guaranteed` sempre será um subconjunto. O mesmo algoritmo pode aparecer várias vezes neste conjunto com nomes diferentes (graças ao OpenSSL).

Adicionado na versão 3.2.

Objetos hash

Os seguintes valores são fornecidos como atributos constantes dos objetos hash retornados pelos construtores:

hash.digest_size

O tamanho do hash resultante em bytes.

hash.block_size

O tamanho do bloco interno do algoritmo de hash em bytes.

Um objeto hash tem os seguintes atributos:

hash.name

O nome canônico deste hash, sempre em minúsculas e sempre adequado como parâmetro para `new()` para criar outro hash deste tipo.

Alterado na versão 3.4: O atributo name está presente no CPython desde o seu início, mas até o Python 3.4 não era especificado formalmente, então pode não existir em algumas plataformas.

Um objeto hash tem os seguintes métodos:

hash.update(*data*)

Atualiza o objeto hash com o objeto bytes ou similar. Chamadas repetidas são equivalentes a uma única chamada com a concatenação de todos os argumentos: `m.update(a); m.update(b)` é equivalente a `m.update(a+b)`.

hash.digest()

Retorna o resumo dos dados passados ao método **`update()`** até o momento. Este é um objeto bytes de tamanho **`digest_size`** que pode conter bytes em todo o intervalo de 0 a 255.

hash.hexdigest()

Similar a **`digest()`**, exceto que o resumo é retornado como um objeto string de comprimento duplo, contendo apenas dígitos hexadecimais. Isso pode ser usado para trocar o valor com segurança em e-mails ou outros ambientes não binários.

hash.copy()

Retorna uma cópia (“clone”) do objeto hash. Isso pode ser usado para calcular com eficiência os resumos de dados que compartilham uma substring inicial comum.

Resumos de comprimento variável de SHAKE

hashlib.shake_128([*data*,]*, *usedforsecurity=True*)

hashlib.shake_256([*data*,]*, *usedforsecurity=True*)

Os algoritmos `shake_128()` e `shake_256()` fornecem resumos de comprimento variável com length_in_bits//2 de até 128 ou 256 bits de segurança. Portanto, seus métodos de resumo exigem um comprimento. O comprimento máximo não é limitado pelo algoritmo SHAKE.

shake.digest(*length*)

Retorna o resumo dos dados passados ao método `update()` até o momento. Este é um objeto bytes de tamanho *length* que pode conter bytes em todo o intervalo de 0 a 255.

shake.hexdigest(*length*)

Similar a `digest()`, exceto que o resumo é retornado como um objeto string de comprimento duplo, contendo apenas dígitos hexadecimais. Isso pode ser usado para trocar o valor em e-mails ou outros ambientes não binários.

Exemplo de uso:

```
>>> h = hashlib.shake_256(b'Nobody inspects the spammish repetit
>>> h.hexdigest(20)
'44709d6fcb83d92a76dcb0b668c98e1b1d3dafe7'
```

Hash de arquivo

O módulo hashlib fornece uma função auxiliar para hash eficiente de um objeto arquivo ou similar.

hashlib.file_digest(*fileobj*, *digest*, /)

Retorna um objeto resumo que foi atualizado com o conteúdo do objeto arquivo.

fileobj deve ser um objeto arquivo ou similar aberto para leitura em modo binário. Ele aceita objetos arquivo da `open()` embutida, instâncias de `BytesIO`, objetos SocketIO de `socket.socket.makefile()` e similares. *fileobj* deve ser aberto em modo bloqueante, caso contrário, uma exceção `BlockingIOError` pode ser levantada.

A função pode ignorar a E/S do Python e usar o descritor de arquivo de `fileno()` diretamente. *fileobj* deve ser presumido como sendo um estado desconhecido após o retorno ou o levantamento desta função. Cabe a quem a chamou fechar *fileobj*.

digest deve ser um nome de algoritmo de hash como *str*, um construtor de hash ou um chamável que retorna um objeto de hash.

Exemplo:

```
>>> import io, hashlib, hmac
>>> with open("library/hashlib.rst", "rb") as f:
...     digest = hashlib.file_digest(f, "sha256")
...
>>> digest.hexdigest()
'...'

>>> buf = io.BytesIO(b"somedata")
>>> mac1 = hmac.HMAC(b"key", digestmod=hashlib.sha512)
>>> digest = hashlib.file_digest(buf, lambda: mac1)

>>> digest is mac1
True
>>> mac2 = hmac.HMAC(b"key", b"somedata", digestmod=hashlib.s
>>> mac1.digest() == mac2.digest()
True
```

Adicionado na versão 3.11.

Alterado na versão 3.14: Agora levanta uma `BlockingIOError` se o arquivo for aberto em modo não-bloqueante. Anteriormente, bytes nulos espúrios eram adicionados ao resumo.

Derivação de chave

Algoritmos de alongamento de chave e derivação de chave são projetados para criar hashes de senhas seguros. Algoritmos ingênuos como `sha1(password)` não são resistentes a ataques de força bruta. Uma boa

função de hashing de senhas deve ser ajustável, lenta e incluir um salt [https://pt.wikipedia.org/wiki/Sal_(criptografia)].

hashlib.pbkdf2_hmac(*hash_name*, *password*, *salt*, *iterations*, *dklen=None*)

A função fornece a função 2 de derivação de chave baseada em senha PKCS#5. Ela usa HMAC como função pseudoaleatória.

A string *hash_name* é o nome desejado do algoritmo de resumo de hash para HMAC, por exemplo, 'sha1' ou 'sha256'. *password* e *salt* são interpretados como buffers de bytes. Aplicações e bibliotecas devem limitar *password* a um comprimento razoável (por exemplo, 1024). *salt* deve ter cerca de 16 bytes ou mais de uma fonte adequada, por exemplo, `os.urandom()`.

O número de *iterations* deve ser escolhido com base no algoritmo de hash e no poder computacional. A partir de 2022, centenas de milhares de iterações do SHA-256 são sugeridas. Para entender por que e como escolher o que é melhor para sua aplicação, leia o *Appendix A.2.2* do NIST-SP-800-132 [https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf]. As respostas à pergunta sobre iterações de pbkdf2 no StackExchange [https://security.stackexchange.com/questions/3959/recommended-of-iterations-when-using-pbkdf2-sha256/] explicam em detalhes.

dklen é o comprimento da chave derivada em bytes. Se *dklen* for `None`, o tamanho do resumo do algoritmo de hash *hash_name* será usado, por exemplo, 64 para SHA-512.

```
>>> from hashlib import pbkdf2_hmac
>>> our_app_iters = 500_000  # Application specific, read abo
>>> dk = pbkdf2_hmac('sha256', b'password', b'bad salt' * 2,
>>> dk.hex()
'15530bba69924174860db778f2c6f8104d3aaf9d26241840c8c4a641c8d0
```

Função disponível somente quando Python é compilado com OpenSSL.

Adicionado na versão 3.4.

Alterado na versão 3.12: Função agora disponível apenas quando o Python é criado com OpenSSL. A implementação lenta do Python puro foi removida.

hashlib.scrypt(*password*, *, *salt*, *n*, *r*, *p*, *maxmem=0*, *dklen=64*)

A função fornece a função de derivação de chave baseada em senha scrypt, conforme definido em **RFC 7914** [https://datatracker.ietf.org/doc/html/rfc7914.html].

password e *salt* devem ser objetos bytes ou similares. Aplicações e bibliotecas devem limitar *password* a um tamanho razoável (por exemplo, 1024). *salt* deve ter cerca de 16 bytes ou mais de uma fonte adequada, por exemplo, `os.urandom()`.

n é o fator de custo de CPU/memória, *r* o tamanho do bloco, *p* o fator de paralelismo e *maxmem* limita a memória (o padrão do OpenSSL 1.1.0 é 32 MiB). *dklen* é o comprimento da chave derivada em bytes.

Adicionado na versão 3.6.

BLAKE2

BLAKE2 [https://www.blake2.net] é uma função hash criptográfica definida em **RFC 7693** [https://datatracker.ietf.org/doc/html/rfc7693.html] que vem em dois sabores:

- **BLAKE2b**, otimizado para plataformas de 64 bits e produz resumos de qualquer tamanho entre 1 e 64 bytes,
- **BLAKE2s**, otimizado para plataformas de 8 a 32 bits e produz resumos de qualquer tamanho entre 1 e 32 bytes.

BLAKE2 supports **keyed mode** (a faster and simpler replacement for HMAC [https://en.wikipedia.org/wiki/Hash-based_message_authentication_code]), **salted hashing**, **personalization**, and **tree hashing**.

Hash objects from this module follow the API of standard library's `hashlib` objects.

Criando objetos hash

Novos objetos hash são criados chamando funções construtoras:

hashlib.blake2b(*data=b'', *, digest_size=64, key=b'', salt=b'', person=b'', fanout=1, depth=1, leaf_size=0, node_offset=0, node_depth=0, inner_size=0, last_node=False, usedforsecurity=True*)

hashlib.blake2s(*data=b'', *, digest_size=32, key=b'', salt=b'', person=b'', fanout=1, depth=1, leaf_size=0, node_offset=0, node_depth=0, inner_size=0, last_node=False, usedforsecurity=True*)

Essas funções retornam os objetos hash correspondentes para calcular BLAKE2b ou BLAKE2s. Opcionalmente, elas aceitam estes parâmetros gerais:

- *data*: pedaço inicial de dados para hash, que deve ser objeto bytes ou similar. Pode ser passado apenas como argumento posicional.
- *digest_size*: tamanho do resumo de saída em bytes.
- *key*: chave para hash com chave (até 64 bytes para BLAKE2b, até 32 bytes para BLAKE2s).
- *salt*: sal para hash aleatório (até 16 bytes para BLAKE2b, até 8 bytes para BLAKE2s).
- *person*: string de personalização (até 16 bytes para BLAKE2b, até 8 bytes para BLAKE2s).

A tabela a seguir mostra os limites para parâmetros gerais (em bytes):

Hash	digest_size	len(key)	len(salt)	len(person)
BLAKE2b	64	64	16	16
BLAKE2s	32	32	8	8

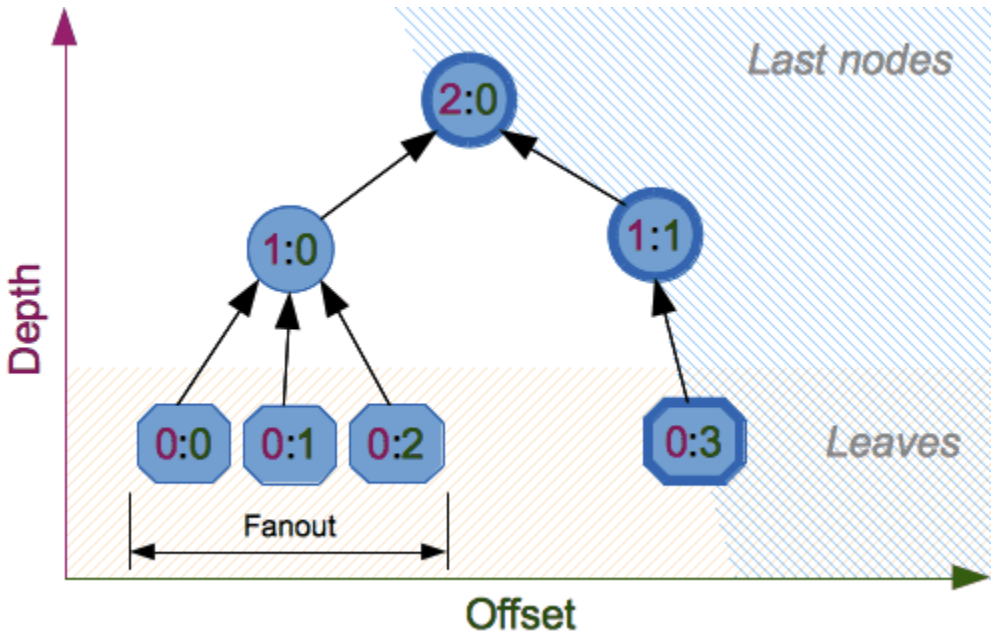
Nota

BLAKE2 specification defines constant lengths for salt and personalization parameters, however, for convenience, this implementation accepts byte strings of any size up to the specified length. If the length of the parameter is less than specified, it is padded with zeros, thus, for example, `b'salt'` and `b'salt\x00'` is the same value. (This is not the case for *key*.)

These sizes are available as module constants described below.

Constructor functions also accept the following tree hashing parameters:

- *fanout*: fanout (0 to 255, 0 if unlimited, 1 in sequential mode).
- *depth*: maximal depth of tree (1 to 255, 255 if unlimited, 1 in sequential mode).
- *leaf_size*: maximal byte length of leaf (0 to `2**32-1`, 0 if unlimited or in sequential mode).
- *node_offset*: node offset (0 to `2**64-1` for BLAKE2b, 0 to `2**48-1` for BLAKE2s, 0 for the first, leftmost, leaf, or in sequential mode).
- *node_depth*: node depth (0 to 255, 0 for leaves, or in sequential mode).
- *inner_size*: inner digest size (0 to 64 for BLAKE2b, 0 to 32 for BLAKE2s, 0 in sequential mode).
- *last_node*: boolean indicating whether the processed node is the last one (`False` for sequential mode).



See section 2.10 in [BLAKE2 specification](https://www.blake2.net/blake2_20130129.pdf) [https://www.blake2.net/blake2_20130129.pdf] for comprehensive review of tree hashing.

Constantes

blake2b.SALT_SIZE

blake2s.SALT_SIZE

Salt length (maximum length accepted by constructors).

blake2b.PERSON_SIZE

blake2s.PERSON_SIZE

Personalization string length (maximum length accepted by constructors).

blake2b.MAX_KEY_SIZE

blake2s.MAX_KEY_SIZE

Maximum key size.

blake2b.MAX_DIGEST_SIZE

blake2s.MAX_DIGEST_SIZE

Maximum digest size that the hash function can output.

Exemplos

Simple hashing

To calculate hash of some data, you should first construct a hash object by calling the appropriate constructor function (`blake2b()` or `blake2s()`), then update it with the data by calling `update()` on the object, and, finally, get the digest out of the object by calling `digest()` (or `hexdigest()` for hex-encoded string).

```
>>> from hashlib import blake2b
>>> h = blake2b()
>>> h.update(b'Hello world')
>>> h.hexdigest()
'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f3
```

As a shortcut, you can pass the first chunk of data to update directly to the constructor as the positional argument:

```
>>> from hashlib import blake2b
>>> blake2b(b'Hello world').hexdigest()
'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f3
```

You can call `hash.update()` as many times as you need to iteratively update the hash:

```
>>> from hashlib import blake2b
>>> items = [b'Hello', b' ', b'world']
>>> h = blake2b()
>>> for item in items:
...     h.update(item)
...
```

```
>>> h.hexdigest()
'6ff843ba685842aa82031d3f53c48b66326df7639a63d128974c5c14f31a0f3
```

Using different digest sizes

BLAKE2 has configurable size of digests up to 64 bytes for BLAKE2b and up to 32 bytes for BLAKE2s. For example, to replace SHA-1 with BLAKE2b without changing the size of output, we can tell BLAKE2b to produce 20-byte digests:

```
>>> from hashlib import blake2b
>>> h = blake2b(digest_size=20)
>>> h.update(b'Replacing SHA1 with the more secure function')
>>> h.hexdigest()
'd24f26cf8de66472d58d4e1b1774b4c9158b1f4c'
>>> h.digest_size
20
>>> len(h.digest())
20
```

Hash objects with different digest sizes have completely different outputs (shorter hashes are *not* prefixes of longer hashes); BLAKE2b and BLAKE2s produce different outputs even if the output length is the same:

```
>>> from hashlib import blake2b, blake2s
>>> blake2b(digest_size=10).hexdigest()
'6fa1d8fcfd719046d762'
>>> blake2b(digest_size=11).hexdigest()
'eb6ec15daf9546254f0809'
>>> blake2s(digest_size=10).hexdigest()
'1bf21a98c78a1c376ae9'
>>> blake2s(digest_size=11).hexdigest()
'567004bf96e4a25773ebf4'
```

Keyed hashing

Keyed hashing can be used for authentication as a faster and simpler replacement for Hash-based message authentication code [https://en.wikipedia.org/wiki/HMAC] (HMAC). BLAKE2 can be securely used in

prefix-MAC mode thanks to the indifferentiability property inherited from BLAKE.

This example shows how to get a (hex-encoded) 128-bit authentication code for message `b'message data'` with key `b'pseudorandom key'`:

```
>>> from hashlib import blake2b
>>> h = blake2b(key=b'pseudorandom key', digest_size=16)
>>> h.update(b'message data')
>>> h.hexdigest()
'3d363ff7401e02026f4a4687d4863ced'
```

As a practical example, a web application can symmetrically sign cookies sent to users and later verify them to make sure they weren't tampered with:

```
>>> from hashlib import blake2b
>>> from hmac import compare_digest
>>>
>>> SECRET_KEY = b'pseudorandomly generated server secret key'
>>> AUTH_SIZE = 16
>>>
>>> def sign(cookie):
...     h = blake2b(digest_size=AUTH_SIZE, key=SECRET_KEY)
...     h.update(cookie)
...     return h.hexdigest().encode('utf-8')
>>>
>>> def verify(cookie, sig):
...     good_sig = sign(cookie)
...     return compare_digest(good_sig, sig)
>>>
>>> cookie = b'user-alice'
>>> sig = sign(cookie)
>>> print("{0},{1}".format(cookie.decode('utf-8'), sig))
user-alice,b'43b3c982cf697e0c5ab22172d1ca7421'
>>> verify(cookie, sig)
True
>>> verify(b'user-bob', sig)
False
>>> verify(cookie, b'0102030405060708090a0b0c0d0e0f00')
False
```

Even though there's a native keyed hashing mode, BLAKE2 can, of course, be used in HMAC construction with **hmac** module:

```
>>> import hmac, hashlib
>>> m = hmac.new(b'secret key', digestmod=hashlib.blake2s)
>>> m.update(b'message')
>>> m.hexdigest()
'e3c8102868d28b5ff85fc35dda07329970d1a01e273c37481326fe0c861c814
```

Randomized hashing

By setting *salt* parameter users can introduce randomization to the hash function. Randomized hashing is useful for protecting against collision attacks on the hash function used in digital signatures.

> Randomized hashing is designed for situations where one party, the message preparer, generates all or part of a message to be signed by a second party, the message signer. If the message preparer is able to find cryptographic hash function collisions (i.e., two messages producing the same hash value), then they might prepare meaningful versions of the message that would produce the same hash value and digital signature, but with different results (e.g., transferring $1,000,000 to an account, rather than $10). Cryptographic hash functions have been designed with collision resistance as a major goal, but the current concentration on attacking cryptographic hash functions may result in a given cryptographic hash function providing less collision resistance than expected. Randomized hashing offers the signer additional protection by reducing the likelihood that a preparer can generate two or more messages that ultimately yield the same hash value during the digital signature generation process — even if it is practical to find collisions for the hash function. However, the use of randomized hashing may reduce the amount of security provided by a digital signature when all portions of the message are prepared by the signer.
>
> (NIST SP-800-106 "Randomized Hashing for Digital Signatures" [https://csrc.nist.gov/pubs/sp/800/106/final])

In BLAKE2 the salt is processed as a one-time input to the hash function during initialization, rather than as an input to each compression function.

Aviso

Salted hashing (or just hashing) with BLAKE2 or any other general-purpose cryptographic hash function, such as SHA-256, is not suitable for hashing passwords. See BLAKE2 FAQ [https://www.blake2.net/#qa] for more information.

```
>>> import os
>>> from hashlib import blake2b
>>> msg = b'some message'
>>> # Calculate the first hash with a random salt.
>>> salt1 = os.urandom(blake2b.SALT_SIZE)
>>> h1 = blake2b(salt=salt1)
>>> h1.update(msg)
>>> # Calculate the second hash with a different random salt.
>>> salt2 = os.urandom(blake2b.SALT_SIZE)
>>> h2 = blake2b(salt=salt2)
>>> h2.update(msg)
>>> # The digests are different.
>>> h1.digest() != h2.digest()
True
```

Personalization

Sometimes it is useful to force hash function to produce different digests for the same input for different purposes. Quoting the authors of the Skein hash function:

> We recommend that all application designers seriously consider doing this; we have seen many protocols where a hash that is computed in one part of the protocol can be used in an entirely different part because two hash computations were done on similar or related data, and the attacker can force the application to make the hash inputs the same. Personalizing each hash function used in the protocol summarily stops this type of attack.

(The Skein Hash Function Family [https://www.schneier.com/wp-content/uploads/2016/02/skein.pdf], p. 21)

BLAKE2 can be personalized by passing bytes to the *person* argument:

```
>>> from hashlib import blake2b
>>> FILES_HASH_PERSON = b'MyApp Files Hash'
>>> BLOCK_HASH_PERSON = b'MyApp Block Hash'
>>> h = blake2b(digest_size=32, person=FILES_HASH_PERSON)
>>> h.update(b'the same content')
>>> h.hexdigest()
'20d9cd024d4fb086aae819a1432dd2466de12947831b75c5a30cf2676095d3b
>>> h = blake2b(digest_size=32, person=BLOCK_HASH_PERSON)
>>> h.update(b'the same content')
>>> h.hexdigest()
'cf68fb5761b9c44e7878bfb2c4c9aea52264a80b75005e65619778de59f383a
```

Personalization together with the keyed mode can also be used to derive different keys from a single one.

```
>>> from hashlib import blake2s
>>> from base64 import b64decode, b64encode
>>> orig_key = b64decode(b'Rm5EPJai72qcK3RGBpW3vPNfZy5OZothY+kHY
>>> enc_key = blake2s(key=orig_key, person=b'kEncrypt').digest()
>>> mac_key = blake2s(key=orig_key, person=b'kMAC').digest()
>>> print(b64encode(enc_key).decode('utf-8'))
rbPb15S/Z9t+agffno5wuhB77VbRi6F9Iv2qIxU7WHw=
>>> print(b64encode(mac_key).decode('utf-8'))
G9GtHFE1YluXY1zWPlYk1e/nWfu0WSEb0KRcjhDeP/o=
```

Modo árvore

Here’s an example of hashing a minimal tree with two leaf nodes:

```
  10
 /  \
00  01
```

This example uses 64-byte internal digests, and returns the 32-byte final digest:

```
>>> from hashlib import blake2b
>>>
>>> FANOUT = 2
>>> DEPTH = 2
>>> LEAF_SIZE = 4096
>>> INNER_SIZE = 64
>>>
>>> buf = bytearray(6000)
>>>
>>> # Left leaf
... h00 = blake2b(buf[0:LEAF_SIZE], fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=0, node_depth=0, last_node=False)
>>> # Right leaf
... h01 = blake2b(buf[LEAF_SIZE:], fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=1, node_depth=0, last_node=True)
>>> # Root node
... h10 = blake2b(digest_size=32, fanout=FANOUT, depth=DEPTH,
...               leaf_size=LEAF_SIZE, inner_size=INNER_SIZE,
...               node_offset=0, node_depth=1, last_node=True)
>>> h10.update(h00.digest())
>>> h10.update(h01.digest())
>>> h10.hexdigest()
'3ad2a9b37c6070e374c7a8c508fe20ca86b6ed54e286e93a0318e95e881db5a
```

Credits

BLAKE2 [https://www.blake2.net] was designed by *Jean-Philippe Aumasson*, *Samuel Neves*, *Zooko Wilcox-O'Hearn*, and *Christian Winnerlein* based on SHA-3 [https://en.wikipedia.org/wiki/Secure_Hash_Algorithms] finalist BLAKE [https://web.archive.org/web/20200918190133/https://131002.net/blake/] created by *Jean-Philippe Aumasson*, *Luca Henzen*, *Willi Meier*, and *Raphael C.-W. Phan*.

It uses core algorithm from ChaCha [https://cr.yp.to/chacha.html] cipher designed by *Daniel J. Bernstein*.

The stdlib implementation is based on pyblake2 [https://pythonhosted.org/pyblake2/] module. It was written by *Dmitry Chestnykh* based on C implementation written by *Samuel Neves*. The documentation was copied from pyblake2 [https://pythonhosted.org/pyblake2/] and written by *Dmitry Chestnykh*.

The C code was partly rewritten for Python by *Christian Heimes*.

The following public domain dedication applies for both C hash function implementation, extension code, and this documentation:

> To the extent possible under law, the author(s) have dedicated all copyright and related and neighboring rights to this software to the public domain worldwide. This software is distributed without any warranty.
>
> You should have received a copy of the CC0 Public Domain Dedication along with this software. If not, see https://creativecommons.org/publicdomain/zero/1.0/.

The following people have helped with development or contributed their changes to the project and the public domain according to the Creative Commons Public Domain Dedication 1.0 Universal:

- *Alexandr Sokolovskiy*

Ver também

Module `hmac`
A module to generate message authentication codes using hashes.

Módulo `base64`
Another way to encode binary hashes for non-binary environments.

https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.180-4.pdf
The FIPS 180-4 publication on Secure Hash Algorithms.

https://csrc.nist.gov/pubs/fips/202/final
The FIPS 202 publication on the SHA-3 Standard.

https://www.blake2.net/
Official BLAKE2 website.

https://en.wikipedia.org/wiki/Cryptographic_hash_function
Wikipedia article with information on which algorithms have known issues and what that means regarding their use.

https://www.ietf.org/rfc/rfc8018.txt
PKCS #5: Password-Based Cryptography Specification Version 2.1

https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf
NIST Recommendation for Password-Based Key Derivation.

`hmac` — Código de autenticação de mensagem com chave hash

Código-fonte: Lib/hmac.py [https://github.com/python/cpython/tree/3.15/Lib/hmac.py]

Este módulo implementa o algoritmo HMAC conforme descrito por **RFC 2104** [https://datatracker.ietf.org/doc/html/rfc2104.html]. A interface permite o uso de qualquer função hash com um tamanho de resumo *fixo*. Em particular, funções de saída extensíveis como SHAKE-128 ou SHAKE-256 não podem ser usadas com HMAC.

hmac.new(*key*, *msg=None*, *digestmod*)

Retorna um novo objeto hmac. *key* é um objeto bytes ou bytearray que fornece a chave secreta. Se *msg* estiver presente, a chamada de método `update(msg)` é feita. *digestmod* é o nome do resumo, construtor do resumo ou módulo para o objeto HMAC usar. Pode ser qualquer nome adequado para **`hashlib.new()`**. Apesar da posição do argumento, é obrigatório.

Alterado na versão 3.4: O parâmetro *key* pode ser um objeto bytes ou bytearray. O parâmetro *msg* pode ser de qualquer tipo suportado por **`hashlib`**. O parâmetro *digestmod* pode ser o nome de um algoritmo hash.

Alterado na versão 3.8: O argumento *digestmod* agora é obrigatório. Passe-o como um argumento nomeado para evitar constrangimentos quando você não tiver uma *msg* inicial.

hmac.digest(*key*, *msg*, *digest*)

Retorna o resumo de *msg* para o segredo *key* e *digest* fornecidos. A função equivale a `HMAC(key, msg, digest).digest()`, mas usa uma implementação C otimizada ou inline, que é mais rápida para

mensagens que cabem na memória. Os parâmetros *key*, *msg* e *digest* têm o mesmo significado que em `new()`.

Detalhe da implementação do CPython: a implementação otimizada do C é usada somente quando *digest* é uma string e o nome de um algoritmo de resumo, que é suportado pelo OpenSSL.

Adicionado na versão 3.7.

class hmac.HMAC

Um objeto HMAC tem os seguintes métodos:

HMAC.update(*msg*)

Atualiza o objeto hmac com *msg*. Chamadas repetidas são equivalentes a uma única chamada com a concatenação de todos os argumentos: `m.update(a); m.update(b)` equivale a `m.update(a + b)`.

Alterado na versão 3.4: O parâmetro *msg* pode ser de qualquer tipo suportado por `hashlib`.

HMAC.digest()

Retorna o resumo dos bytes passados para o método `update()` até o momento. Este objeto bytes terá o mesmo comprimento que o *digest_size* do resumo dado ao construtor. Ele pode conter bytes não-ASCII, incluindo bytes NUL.

Aviso

Ao comparar a saída de `digest()` com um resumo fornecido externamente durante uma rotina de verificação, é recomendável usar a função `compare_digest()` em vez do operador == para reduzir a vulnerabilidade a ataques de temporização.

HMAC.hexdigest()

Como `digest()` exceto que o resumo é retornado como uma string com o dobro do comprimento contendo apenas dígitos hexadecimais. Isso

pode ser usado para trocar o valor com segurança em e-mail ou outros ambientes não binários.

Aviso

Ao comparar a saída de `hexdigest()` com um resumo fornecido externamente durante uma rotina de verificação, é recomendável usar a função `compare_digest()` em vez do operador == para reduzir a vulnerabilidade a ataques de temporização.

HMAC.copy()

Retorna uma cópia ("clone") do objeto hmac. Isso pode ser usado para calcular eficientemente os resumos de strings que compartilham uma substring inicial comum.

Um objeto hash tem os seguintes atributos:

HMAC.digest_size

O tamanho do resumo HMAC resultante em bytes.

HMAC.block_size

O tamanho do bloco interna do algoritmo de hash em bytes.

Adicionado na versão 3.4.

HMAC.name

O nome canônico deste HMAC, sempre em letras minúsculas, por exemplo, `hmac-md5`.

Adicionado na versão 3.4.

Alterado na versão 3.10: Foram removidos os atributos não documentados `HMAC.digest_cons`, `HMAC.inner` e `HMAC.outer`.

Este módulo também fornece a seguinte função auxiliar:

hmac.compare_digest(*a*, *b*)

Retorna `a == b`. Esta função usa uma abordagem projetada para evitar análise de tempo evitando comportamento de curto-circuito baseado em conteúdo, tornando-a apropriada para criptografia. *a* e *b* devem ser ambos do mesmo tipo: `str` (somente ASCII, como, por exemplo, retornado por `HMAC.hexdigest()`), ou um objeto bytes ou similar.

Nota

Se *a* e *b* tiverem comprimentos diferentes, ou se ocorrer um erro, um ataque de temporização poderia teoricamente revelar informações sobre os tipos e comprimentos de *a* e *b*, mas não seus valores.

Adicionado na versão 3.3.

Alterado na versão 3.10: A função usa `CRYPTO_memcmp()` do OpenSSL internamente quando disponível.

Ver também

Módulo `hashlib`

O módulo Python que fornece funções hash seguras.

`secrets` — Gera números aleatórios seguros para gerenciar segredos

Adicionado na versão 3.6.

Código-fonte: Lib/secrets.py [https://github.com/python/cpython/tree/3.15/Lib/secrets.py]

The `secrets` module is used for generating cryptographically strong random numbers suitable for managing data such as passwords, account authentication, security tokens, and related secrets.

In particular, `secrets` should be used in preference to the default pseudo-random number generator in the `random` module, which is designed for modelling and simulation, not security or cryptography.

Ver também

PEP 506 [https://peps.python.org/pep-0506/]

Números aleatórios

The `secrets` module provides access to the most secure source of randomness that your operating system provides.

class secrets.SystemRandom

Uma classe para gerar números aleatórios usando as fontes da mais alta qualidade fornecidas pelo sistema operacional. Veja `random.SystemRandom` para detalhes adicionais.

secrets.choice(*seq*)

Retorna um elemento escolhido aleatoriamente de uma sequência não vazia.

secrets.randbelow(*exclusive_upper_bound*)

Retorna um int aleatório no intervalo [0, *exclusive_upper_bound*).

secrets.randbits(*k*)

Retorna um int não negativo com *k* bits aleatórios.

Gerando tokens

The **`secrets`** module provides functions for generating secure tokens, suitable for applications such as password resets, hard-to-guess URLs, and similar.

secrets.token_bytes(*nbytes=None*)

Return a random byte string containing *nbytes* number of bytes.

If *nbytes* is not specified or `None`, **`DEFAULT_ENTROPY`** is used instead.

```
>>> token_bytes(16)
b'\xebr\x17D*t\xae\xd4\xe3S\xb6\xe2\xebP1\x8b'
```

secrets.token_hex(*nbytes=None*)

Return a random text string, in hexadecimal. The string has *nbytes* random bytes, each byte converted to two hex digits.

If *nbytes* is not specified or `None`, **`DEFAULT_ENTROPY`** is used instead.

```
>>> token_hex(16)
'f9bf78b9a18ce6d46a0cd2b0b86df9da'
```

secrets.token_urlsafe(*nbytes=None*)

Return a random URL-safe text string, containing *nbytes* random bytes. The text is Base64 encoded, so on average each byte results in

approximately 1.3 characters.

If *nbytes* is not specified or `None`, **`DEFAULT_ENTROPY`** is used instead.

```
>>> token_urlsafe(16)
'Drmhze6EPcv0fN_81Bj-nA'
```

Quantos bytes os tokens devem usar?

To be secure against brute-force attacks [https://en.wikipedia.org/wiki/Brute-force_attack], tokens need to have sufficient randomness. Unfortunately, what is considered sufficient will necessarily increase as computers get more powerful and able to make more guesses in a shorter period. As of 2015, it is believed that 32 bytes (256 bits) of randomness is sufficient for the typical use-case expected for the **`secrets`** module.

Para aqueles que desejam gerenciar seu próprio comprimento de token, você pode especificar explicitamente quanta aleatoriedade é usada para tokens, fornecendo um argumento **`int`** para as várias funções `token_*`. Esse argumento é considerado o número de bytes de aleatoriedade a serem usados.

Otherwise, if no argument is provided, or if the argument is `None`, the `token_*` functions use **`DEFAULT_ENTROPY`** instead.

secrets.DEFAULT_ENTROPY

Default number of bytes of randomness used by the `token_*` functions.

The exact value is subject to change at any time, including during maintenance releases.

Outras funções

secrets.compare_digest(*a*, *b*)

Return `True` if strings or bytes-like objects *a* and *b* are equal, otherwise `False`, using a “constant-time compare” to reduce the risk of timing

attacks [https://web.archive.org/web/20250815071532/https://codahale.com/a-lesson-in-timing-attacks/]. See `hmac.compare_digest()` for additional details.

Receitas e melhores práticas

This section shows recipes and best practices for using `secrets` to manage a basic level of security.

Gerar uma senha alfanumérica de oito caracteres:

```
import string
import secrets
alphabet = string.ascii_letters + string.digits
password = ''.join(secrets.choice(alphabet) for i in range(8))
```

Nota

As aplicações não devem **armazenar senhas em um formato recuperável** [https://cwe.mitre.org/data/definitions/257.html], seja em texto simples ou criptografado. Elas devem ser salgadas e transformadas em hash usando uma função hash de sentido único criptograficamente forte (irreversível).

Gerar uma senha alfanumérica de dez caracteres com pelo menos um caractere minúsculo, pelo menos um caractere maiúsculo e pelo menos três dígitos:

```
import string
import secrets
alphabet = string.ascii_letters + string.digits
while True:
    password = ''.join(secrets.choice(alphabet) for i in range(1
    if (any(c.islower() for c in password)
            and any(c.isupper() for c in password)
            and sum(c.isdigit() for c in password) >= 3):
        break
```

Gerar uma senha longa do estilo XKCD [https://xkcd.com/936/]:

```
import secrets
# Em sistemas Linux padrão, use um arquivo de dicionário conveni
# Outras plataformas podem precisar fornecer sua própria lista d
with open('/usr/share/dict/words') as f:
    words = [word.strip() for word in f]
    password = ' '.join(secrets.choice(words) for i in range(4))
```

Gerar uma URL temporária difícil de adivinhação contendo um token de segurança adequado para aplicativos de recuperação de senha:

```
import secrets
url = 'https://example.com/reset=' + secrets.token_urlsafe()
```

Serviços Genéricos do Sistema Operacional

Os módulos descritos neste capítulo fornecem interfaces aos recursos do sistema operacional e que estão disponíveis em (quase) todos os sistemas operacionais, como arquivos e um relógio. As interfaces geralmente são modeladas após as interfaces Unix ou C, mas elas também estão disponíveis na maioria dos outros sistemas. Aqui temos uma visão geral:

- `os` — Diversas interfaces de sistema operacional
  - Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente
  - Modo UTF-8 do Python
  - Parâmetros de processo
  - Criação de objetos arquivos
  - Operações dos descritores de arquivos
    - Consultando o tamanho de um terminal
    - Herança de descritores de arquivos
  - Arquivos e diretórios
    - Descritores de arquivo de temporizador
    - Atributos estendidos do Linux
  - Gerenciamento de processo
  - Interface do agendador
  - Diversas informações de sistema
  - Números aleatórios
- `io` — Ferramentas essenciais para trabalhar com fluxos
  - Visão Geral
    - E/S de texto
    - E/S binária
    - E/S bruta
  - Codificação do texto
    - Ativar EncodingWarning
  - Interface de alto nível do módulo
  - hierarquia de classe

        - I/O base classes
        - Raw file I/O
        - Buffered streams
        - E/S de texto
    - Static typing
    - Desempenho
        - E/S binária
        - E/S de texto
        - Multi-threading
        - Reentrancy
- `time` — Acesso e conversões de tempo
    - Funções
    - Constantes de ID de Relógio
    - Constantes de Fuso Horário
- `logging` — Recursos de registro de eventos para Python
    - Objetos Logger
    - Níveis de Logging
    - Objetos Handler
    - Formatter Objects
    - Filter Objects
    - LogRecord Objects
    - Atributos LogRecord
    - LoggerAdapter Objects
    - Thread Safety
    - Funções de Nível de Módulo
    - Module-Level Attributes
    - Integration with the warnings module
- `logging.config` — Configuração do módulo logging
    - Funções de configuração
    - Considerações de segurança
    - Esquema do Dicionário de Configuração
        - Detalhes do Esquema de Dicionário
        - Configuração Incremental
        - Conexões de objeto
        - Objetos definidos pelo usuário
        - Ordem de configuração de manipuladores
        - Acesso a objetos externos

        - Acesso a objetos internos
        - Resolução de importações e importadores personalizados
        - Configurando QueueHandler e QueueListener
    - Formato do arquivo de configuração
- `logging.handlers` — Manipuladores de logging
    - StreamHandler
    - FileHandler
    - NullHandler
    - WatchedFileHandler
    - BaseRotatingHandler
    - RotatingFileHandler
    - TimedRotatingFileHandler
    - SocketHandler
    - DatagramHandler
    - SysLogHandler
    - NTEventLogHandler
    - SMTPHandler
    - MemoryHandler
    - HTTPHandler
    - QueueHandler
    - QueueListener
- `platform` — Acesso aos dados de identificação da plataforma subjacente
    - Multiplataforma
    - Plataforma Windows
    - Plataforma macOS
    - Plataforma iOS
    - Plataformas Unix
    - Plataformas Linux
    - Plataforma Android
    - Uso na linha de comando
- `errno` — Símbolos padrão do sistema errno
- `ctypes` — Uma biblioteca de funções externas para Python
    - Tutorial do ctypes
        - Carregando bibliotecas de ligação dinâmica
        - Acessando funções de dlls carregadas
        - Chamando funções

    - Tipos de dados fundamentais
    - Chamando funções, continuação
    - Chamando funções variadas
    - Chamando funções com seus próprios tipos de dados personalizados
    - Especificando os tipos de argumentos necessários (protótipos de função)
    - Tipos de Retorno
    - Passando ponteiros (ou: passando parâmetros por referência)
    - Estruturas e uniões
    - Layout, alinhamento e ordem de bytes de estrutura/união
    - Campos de bit em estruturas e uniões
    - Vetores
    - Ponteiros
    - Segurança de thread (Thread safety) sem o GIL
    - Conversão de Tipos
    - Tipos Incompletos
    - Funções Callbacks
    - Acessando valores exportados de dlls
    - Surpresas
    - Tipos de dados de tamanho variável
  - Referência ctypes
    - Loading shared libraries
    - Encontrando bibliotecas compartilhadas
    - Listing loaded shared libraries
    - Foreign functions
    - Function prototypes
    - Funções utilitárias
    - Data types
    - Tipos de dados fundamentais
    - Tipos de dados estruturados
    - Arrays and pointers
    - Exceções

os — Diversas interfaces de sistema operacional

Código-fonte: Lib/os.py [https://github.com/python/cpython/tree/3.15/Lib/os.py]

Este módulo fornece uma maneira simples de usar funcionalidades que são dependentes de sistema operacional. Se você quiser ler ou escrever um arquivo, veja `open()`; se o que quer é manipular estruturas de diretórios, veja o módulo `os.path`; e se quiser ler todas as linhas, de todos os arquivos na linha de comando, veja o módulo `fileinput`. Para criar arquivos e diretórios temporários, veja o módulo `tempfile`; e, para manipulação em alto nível de arquivos e diretórios, veja o módulo `shutil`.

Notas sobre a disponibilidade dessas funções:

- O modelo dos módulos embutidos dependentes do sistema operacional no Python é tal que, desde que a mesma funcionalidade esteja disponível, a mesma interface é usada; por exemplo, a função `os.stat(path)` retorna informações estatísticas sobre *path* no mesmo formato (que é originado com a interface POSIX).
- Extensions peculiar to a particular operating system are also available through the **os** module, but using them is of course a threat to portability.
- Todas as funções que aceitam nomes de caminhos ou arquivos, aceitam objetos bytes e string, e resultam em um objeto do mesmo tipo, se um caminho ou nome de arquivo for retornado.
- Em VxWorks, os.popen, os.fork, os.execv e os.spawn*p* não são suportados.
- On WebAssembly platforms, Android and iOS, large parts of the **os** module are not available or behave differently. APIs related to processes (e.g. `fork()`, `execve()`) and resources (e.g. `nice()`) are not available. Others like `getuid()` and `getpid()` are emulated or stubs.

WebAssembly platforms also lack support for signals (e.g. `kill()`, `wait()`).

Nota

Todas as funções neste módulo levantam `OSError` (ou suas subclasses) no caso de nomes e caminhos de arquivos inválidos ou inacessíveis, ou outros argumentos que possuem o tipo correto, mas não são aceitos pelo sistema operacional.

exception os.error

Um apelido para a exceção embutida `OSError`.

os.name

O nome do módulo importado, dependente do sistema operacional. Atualmente, os seguintes nomes foram registrados: `'posix'`, `'nt'`, `'java'`.

Ver também

`sys.platform` tem uma granularidade mais fina. `os.uname()` fornece informações de versão específicas do sistema.

O módulo `platform` fornece verificações detalhadas sobre a identificação do sistema.

Nomes de arquivos, argumentos de linha de comando e variáveis de ambiente

No Python, nomes de arquivos, argumentos da linha de comando, e variáveis de ambiente são representadas usando o tipo string. Em alguns sistemas, decodificar essas strings de e para bytes é necessário antes de passá-las para o sistema operacional. Python usa o tratador de erros e

codificação do sistema de arquivos para realizar essa conversão (veja `sys.getfilesystemencoding()`).

O tratador de erros e codificação do sistema de arquivos são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Alterado na versão 3.1: Em alguns sistemas a conversão, usando a codificação do sistema de arquivos, pode falhar. Neste caso, Python usa o manipulador de erro de codificação surrogateescape, o que significa que bytes não decodificados são substituídos por um caractere Unicode U+DC*xx* na decodificação, e estes são novamente traduzidos para o byte original na codificação.

A codificação de sistema de arquivos precisa garantir que todos os bytes abaixo de 128 serão decodificados com sucesso. Se a codificação do sistemas de arquivos falhar em garantir isso, funções de API podem levantar `UnicodeError`.

Veja também a codificação da localidade.

Modo UTF-8 do Python

Adicionado na versão 3.7: Veja **PEP 540** [https://peps.python.org/pep-0540/] para mais detalhes.

Alterado na versão 3.15: Python UTF-8 mode is now enabled by default (**PEP 686** [https://peps.python.org/pep-0686/]). It may be disabled by setting `PYTHONUTF8=0` as an environment variable or by using the `-X utf8=0` command line option.

O Modo UTF-8 do Python ignora a codificação da localidade e força o uso da codificação UTF-8:

- Usa UTF-8 como a codificação do sistema de arquivos.
- `sys.getfilesystemencoding()` retorna `'utf-8'`.

- `locale.getpreferredencoding()` retorna `'utf-8'` (o argumento *do_setlocale* não causa efeito).
- `sys.stdin`, `sys.stdout` e `sys.stderr` usam UTF-8 como codificação do texto, com o tratador de erros `surrogateescape` sendo habilitado para **`sys.stdin`** e **`sys.stdout`** (**`sys.stderr`** continua a usar `backslashreplace` como faz no modo padrão com reconhecimento de localidade)
- No Unix, `os.device_encoding()` retorna `'utf-8'` invés da codificação do dispositivo.

Observe que as configurações de fluxo padrão no modo UTF-8 podem ser substituídas por `PYTHONIOENCODING` (assim como podem estar no modo com reconhecimento de localidade padrão).

Como consequência das mudanças nessas APIs de baixo nível, outras APIs de alto nível também exibem comportamentos padrões diferentes:

- Argumentos de linha de comando, variáveis de ambiente e nomes de arquivos são decodificados em texto usando a codificação UTF-8.
- `os.fsdecode()` e `os.fsencode()` usam a codificação UTF-8.
- `open()`, `io.open()` e `codecs.open()` usam a codificação UTF-8 por padrão. No entanto, elas ainda usam o tratador de erros estrito por padrão, de modo que a tentativa de abrir um arquivo binário no modo de texto provavelmente levantará uma exceção em vez de produzir dados sem sentido.

The Python UTF-8 Mode is enabled by default. It can be disabled using the `-X utf8=0` command line option or the `PYTHONUTF8=0` environment variable. The Python UTF-8 Mode can only be disabled at Python startup. Its value can be read from `sys.flags.utf8_mode`.

If the UTF-8 mode is disabled, the interpreter defaults to using the current locale settings, *unless* the current locale is identified as a legacy ASCII-based locale (as described for `PYTHONCOERCECLOCALE`), and locale coercion is either disabled or fails. In such legacy locales, the interpreter will default to enabling UTF-8 mode unless explicitly instructed not to do so.

Veja também o modo UTF-8 no Windows e o tratador de erros e codificação do sistema de arquivos.

Parâmetros de processo

Essas funções e itens de dados fornecem informações e operam no processo e usuário atuais.

os.ctermid()

Retorna o nome do arquivo correspondente ao terminal de controle do processo.

Disponibilidade: Unix, not WASI.

os.environ

Um objeto mapeamento onde as chaves e valores são strings que representam o ambiente do processo. Por exemplo, `environ['HOME']` é o nome do caminho do seu diretório pessoal (em algumas plataformas), e é equivalente a `getenv("HOME")` em C.

This mapping is captured the first time the **`os`** module is imported, typically during Python startup as part of processing `site.py`. Changes to the environment made after this time are not reflected in **`os.environ`**, except for changes made by modifying **`os.environ`** directly.

Este mapeamento pode ser usado para modificar o ambiente, além de consultá-lo. **`putenv()`** será chamado automaticamente quando o mapeamento for modificado.

No Unix, chaves e valores usam **`sys.getfilesystemencoding()`** e o tratador de erros `'surrogateescape'`. Use **`environb`** se quiser usar uma codificação diferente.

No Windows, as chaves são convertidas em maiúsculas. Isso também se aplica ao obter, definir ou excluir um item. Por exemplo, `environ['monty'] = 'python'` mapeia a chave `'MONTY'` para o valor `'python'`.

Nota

Chamar a função `putenv()` diretamente não muda `os.environ`, por isso é melhor modificar `os.environ`.

Nota

Em algumas plataformas, incluindo FreeBSD e Mac OS X, a modificação de `environ` pode causar vazamentos de memória. Consulte a documentação do sistema para `putenv()`.

You can delete items in this mapping to unset environment variables. `unsetenv()` will be called automatically when an item is deleted from `os.environ`, and when one of the `pop()` or `clear()` methods is called.

If the *clearenv(3)* *[https://manpages.debian.org/clearenv(3)]* function is available, the `clear()` method uses it and emits a single `os._clearenv` audit event. Otherwise, it emits an `os.unsetenv` event on each deleted variable.

Levanta um evento de auditoria `os.unsetenv` com o argumento `key`.

Raises an auditing event `os._clearenv` with no arguments.

Ver também

A função `os.reload_environ()`.

Alterado na versão 3.9: Atualizado para ter suporte aos operadores de mesclagem (`|`) e de atualização (`|=`) da **PEP 584** [https://peps.python.org/pep-0584/].

Alterado na versão 3.15: The `clear()` method can now emit an `os._clearenv` audit event.

os.environb

Versão bytes de **environ**: um objeto de mapeamento onde as chaves e valores são objetos **bytes** representando o ambiente do processo. **environ** e **environb** são sincronizados (modificar **environb** atualiza **environ**, e vice versa).

environb está disponível somente se **supports_bytes_environ** for `True`.

Adicionado na versão 3.2.

Alterado na versão 3.9: Atualizado para ter suporte aos operadores de mesclagem (`|`) e de atualização (`|=`) da **PEP 584** [https://peps.python.org/pep-0584/].

os.reload_environ()

Os mapeamentos **os.environ** e **os.environb** são um cache de variáveis de ambiente no momento em que o Python foi iniciado. Dessa forma, alterações no ambiente do processo atual não são refletidas se feitas fora do Python, ou por **os.putenv()** ou **os.unsetenv()**. Use **os.reload_environ()** para atualizar **os.environ** e **os.environb** com quaisquer alterações no ambiente do processo atual.

Aviso

This function is not thread-safe. Calling it while the environment is being modified in another thread is an undefined behavior. Reading from **os.environ** or **os.environb**, or calling **os.getenv()** while reloading, may return an empty result.

Adicionado na versão 3.14.

os.chdir(*path*)

os.fchdir(*fd*)

os.getcwd()

Essas funções são descritas em: Arquivos e diretórios.

os.fsencode(*filename*)

Codifica o *filename* caminho ou similar para tratador de erros e codificação do sistema de arquivos; retorna `bytes` inalterados.

`fsdecode()` é a função inversa.

Adicionado na versão 3.2.

Alterado na versão 3.6: Adicionado suporte para aceitar objetos que implementam a interface `os.PathLike`.

os.fsdecode(*filename*)

Decodifica o objeto caminho ou similar *filename* do tratador de erros e codificação do sistema de arquivos; retorna `str` inalterada.

`fsencode()` é a função inversa.

Adicionado na versão 3.2.

Alterado na versão 3.6: Adicionado suporte para aceitar objetos que implementam a interface `os.PathLike`.

os.fspath(*path*)

Retorna a representação do sistema de arquivos do caminho (argumento *path*).

Se um objeto `str` ou `bytes` é passado, é retornado inalterado. Caso contrário, o método `__fspath__()` é chamado, e seu valor é retornado, desde que seja um objeto `str` ou `bytes`. Em todos os outros casos, `TypeError` é levantada.

Adicionado na versão 3.6.

class os.PathLike

Uma classe base abstrata para objetos representando um caminho do sistema de arquivos, como, por exemplo, `pathlib.PurePath`.

Adicionado na versão 3.6.

abstractmethod __fspath__()

Retorna a representação do caminho do sistema de arquivos do objeto.

O método deverá retornar somente objetos `str` ou `bytes`, preferencialmente `str`.

os.getenv(*key*, *default=None*)

Retorna o valor da variável de ambiente *key* como uma string se existir, ou *default* se não existir. *key* é uma string. Note que uma vez que `getenv()` utiliza `os.environ`, o mapeamento de `getenv()` é também similarmente capturado na importação, e a função pode não refletir futuras mudanças no ambiente.

No Unix, chaves e valores são decodificados com a função `sys.getfilesystemencoding()` e o o manipulador de erros `'surrogateescape'`. Use `os.getenvb()` se quiser usar uma codificação diferente.

Disponibilidade: Unix, Windows.

os.getenvb(*key*, *default=None*)

Retorna o valor da variável de ambiente *key* como bytes se existir, ou *default* se não existir. *key* deve ser bytes. Note que uma vez que `getenvb()` utiliza `os.environb`, o mapeamento de `getenvb()` é também similarmente capturado na importação, e a função pode não refletir futuras mudanças no ambiente.

`getenvb()` está disponível somente se `supports_bytes_environ` for `True`.

Disponibilidade: Unix.

Adicionado na versão 3.2.

os.get_exec_path(*env=None*)

Retorna a lista de diretórios que serão buscados por um executável nomeado, semelhante a um shell, ao iniciar um processo. *env*, quando especificado, deve ser um dicionário de variáveis de ambiente para procurar o PATH. Por padrão, quando *env* é `None`, **`environ`** é usado.

Adicionado na versão 3.2.

os.getegid()

Retorna o ID do grupo efetivo do processo atual. Isso corresponde ao bit "set id" no arquivo que está sendo executado no processo atual.

Disponibilidade: Unix, not WASI.

os.geteuid()

Retorna o ID de usuário efetivo do processo atual.

Disponibilidade: Unix, not WASI.

os.getgid()

Retorna o ID do grupo real do processo atual.

Disponibilidade: Unix.

A função é uma simulação em WASI, veja Plataformas WebAssembly para mais informações.

os.getgrouplist(*user*, *group*, /)

Retorna a lista de IDs de grupo aos quais *user* pertence. Se *group* não estiver na lista, ele será incluído; normalmente, *group* é especificado como o campo de ID de grupo do registro de senha para *user*, porque esse ID de grupo será potencialmente omitido.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.getgroups()

Retorna a lista de IDs de grupos suplementares associados ao processo atual.

Disponibilidade: Unix, not WASI.

Nota

On macOS, **`getgroups()`** behavior differs somewhat from other Unix platforms. If the Python interpreter was built with a deployment target of `10.5` or earlier, **`getgroups()`** returns the list of effective group ids associated with the current user process; this list is limited to a system-defined number of entries, typically 16, and may be modified by calls to **`setgroups()`** if suitably privileged. If built with a deployment target greater than `10.5`, **`getgroups()`** returns the current group access list for the user associated with the effective user id of the process; the group access list may change over the lifetime of the process, it is not affected by calls to **`setgroups()`**, and its length is not limited to 16. The deployment target value can be obtained with **`sysconfig.get_config_var('MACOSX_DEPLOYMENT_TARGET')`**.

os.getlogin()

Retorna o nome do usuário conectado no terminal de controle do processo. Para a maioria dos propósitos, é mais útil usar **`getpass.getuser()`**, já que esse último verifica as variáveis de ambiente **`LOGNAME`** ou **`USERNAME`** para descobrir quem é o usuário, e retorna para `pwd.getpwuid(os.getuid())[0]` para obter o nome de login do ID do usuário real atual.

Disponibilidade: Unix, Windows, not WASI.

os.getpgid(*pid*)

Retorna o ID do grupo de processo com *pid*. Se *pid* for 0, o ID do grupo do processos do processo atual é retornado.

Disponibilidade: Unix, not WASI.

os.getpgrp()

Retorna o ID do grupo do processo atual.

Disponibilidade: Unix, not WASI.

os.getpid()

Retorna o ID do processo atual.

A função é uma simulação em WASI, veja Plataformas WebAssembly para mais informações.

os.getppid()

Retorna o ID do processo pai. Quando o processo pai é encerrado, no Unix, o ID retornado é o do processo init (1); no Windows, ainda é o mesmo ID, que pode já estar sendo reutilizado por outro processo.

Disponibilidade: Unix, Windows, not WASI.

Alterado na versão 3.2: Adicionado suporte para Windows.

os.getpriority(*which*, *who*)

Obtém prioridade de agendamento de programa. O valor *which* é um entre **`PRIO_PROCESS`**, **`PRIO_PGRP`** ou **`PRIO_USER`**, e *who* é interpretado em relação a *which* (um identificador de processo para **`PRIO_PROCESS`**, identificador do grupo de processos para **`PRIO_PGRP`** e um ID de usuário para **`PRIO_USER`**). Um valor zero para *who* indica (respectivamente) o processo de chamada, o grupo de processos do processo de chamada ou o ID do usuário real do processo de chamada.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.PRIO_PROCESS
os.PRIO_PGRP
os.PRIO_USER

Parâmetros para as funções **`getpriority()`** e **`setpriority()`**.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.PRIO_DARWIN_THREAD
os.PRIO_DARWIN_PROCESS
os.PRIO_DARWIN_BG
os.PRIO_DARWIN_NONUI

Parâmetros para as funções `getpriority()` e `setpriority()`.

Disponibilidade: macOS

Adicionado na versão 3.12.

os.getresuid()

Retorna uma tupla (ruid, euid, suid) indicando os IDs de usuário real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI, not macOS, not iOS.

Adicionado na versão 3.2.

os.getresgid()

Retorna uma tupla (rgid, egid, sgid) indicando os IDs de grupos real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI, not macOS, not iOS.

Adicionado na versão 3.2.

os.getuid()

Retorna o ID de usuário real do processo atual.

Disponibilidade: Unix.

A função é uma simulação em WASI, veja Plataformas WebAssembly para mais informações.

os.initgroups(*username*, *gid*, */*)

Chama o `initgroups()` do sistema para inicializar a lista de acesso ao grupo com todos os grupos dos quais o nome de usuário especificado é membro, mais o ID do grupo especificado.

Disponibilidade: Unix, not WASI, not Android.

Adicionado na versão 3.2.

os.putenv(*key*, *value*, */*)

Define a variável de ambiente denominada *key* como a string *value*. Tais mudanças no ambiente afetam os subprocessos iniciados com **os.system()**, **popen()** ou **fork()** e **execv()**.

Atribuições a itens em **os.environ** são automaticamente traduzidas em chamadas correspondentes para **putenv()**; entretanto, chamadas para **putenv()** não atualizam **os.environ**, então é preferível atribuir itens de **os.environ**. Isso também se aplica a **getenv()** e **getenvb()**, que usam respectivamente **os.environ** e **os.environb** em suas implementações.

Veja também a função **os.reload_environ()**.

Nota

Em algumas plataformas, incluindo FreeBSD e Mac OS X, a modificação de `environ` pode causar vazamentos de memória. Consulte a documentação do sistema para **putenv()**.

Levanta um evento de auditoria `os.putenv` com os argumentos `key`, `value`.

Alterado na versão 3.9: A função está agora sempre disponível.

os.setegid(*egid*, */*)

Define o ID do grupo efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.seteuid(*euid*, /)

Define o ID do usuário efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.setgid(*gid*, /)

Define o ID do grupo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.setgroups(*groups*, /)

Define a lista de IDs de grupo suplementares associados ao processo atual como *groups*. *groups* deve ser uma sequência e cada elemento deve ser um número inteiro identificando um grupo. Esta operação normalmente está disponível apenas para o superusuário.

Disponibilidade: Unix, not WASI.

Nota

No Mac OS X, o comprimento de *groups* não pode exceder o número máximo definido pelo sistema de IDs de grupo efetivos, normalmente 16. Consulte a documentação de `getgroups()` para casos em que ele pode não retornar o mesmo grupo de listas definido chamando setgroups().

os.setns(*fd*, *nstype=0*)

Reassocia a thread atual com um espaço de nomes do Linux. Veja as páginas man *setns(2)* *[https://manpages.debian.org/setns(2)]* e *namespaces(7)* *[https://manpages.debian.org/namespaces(7)]* para mais detalhes.

Se *fd* se refere a um link `/proc/pid/ns/`, `setns()` reassocia a thread de chamada com o espaço de nomes associado a esse link, e *nstype* pode ser definido como uma das constantes CLONE_NEW* para impor restrições na operação (`0` significa sem restrições).

Desde o Linux 5.8, *fd* pode se referir a um descritor de arquivo PID obtido de **`pidfd_open()`**. Neste caso, `setns()` reassocia a thread chamadora em um ou mais dos mesmos espaços de nomes que o thread referenciado por *fd*. Isso está sujeito a quaisquer restrições impostas por *nstype*, que é uma máscara de bits que combina uma ou mais das constantes CLONE_NEW*, por exemplo `setns(fd, os.CLONE_NEWUTS | os.CLONE_NEWPID)`. As associações do chamador em espaços de nomes não especificados permanecem inalteradas.

fd pode ser qualquer objeto com um método **`fileno()`** ou um descritor de arquivo bruto.

Este exemplo reassocia a thread com o espaço de nomes de rede do processo `init`:

```
fd = os.open("/proc/1/ns/net", os.O_RDONLY)
os.setns(fd, os.CLONE_NEWNET)
os.close(fd)
```

Disponibilidade: Linux >= 3.0 with glibc >= 2.14.

Adicionado na versão 3.12.

Ver também

A função **`unshare()`**.

os.setpgrp()

Executa a chamada de sistema **`setpgrp()`** ou `setpgrp(0, 0)` dependendo da versão implementada (se houver). Veja o manual do Unix para a semântica.

Disponibilidade: Unix, not WASI.

os.setpgid(*pid*, *pgrp*, /)

Executa a chamada de sistema **`setpgid()`** para definir o ID do grupo do processo com *pid* para o grupo de processos com o id *pgrp*. Veja o manual do Unix para a semântica.

Disponibilidade: Unix, not WASI.

os.setpriority(*which*, *who*, *priority*)

Define a prioridade de agendamento de programa. O valor *which* é um entre **PRIO_PROCESS**, **PRIO_PGRP** ou **PRIO_USER**, e *who* é interpretado em relação a *which* (um identificador de processo para **PRIO_PROCESS**, identificador do grupo de processos para **PRIO_PGRP** e um ID de usuário para **PRIO_USER**). Um valor zero para *who* indica (respectivamente) o processo de chamada, o grupo de processos do processo de chamada ou o ID do usuário real do processo de chamada. *priority* é um valor na faixa de -20 a 19. A prioridade padrão é 0; prioridades menores resultam em um agendamento mais favorável.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.setregid(*rgid*, *egid*, /)

Define os IDs de grupo real e efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.setresgid(*rgid*, *egid*, *sgid*, /)

Define os IDs de grupo real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI, not Android, not macOS, not iOS.

Adicionado na versão 3.2.

os.setresuid(*ruid*, *euid*, *suid*, /)

Define os IDs de usuário real, efetivo e salvo do processo atual.

Disponibilidade: Unix, not WASI, not Android, not macOS, not iOS.

Adicionado na versão 3.2.

os.setreuid(*ruid*, *euid*, /)

Define os IDs de usuário real e efetivo do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.getsid(*pid*, /)

Executa a chamada de sistema **`getsid()`**. Veja o manual do Unix para semântica.

Disponibilidade: Unix, not WASI.

os.setsid()

Executa a chamada de sistema **`setsid()`**. Veja o manual do Unix para semântica.

Disponibilidade: Unix, not WASI.

os.setuid(*uid*, /)

Define o ID de usuário do processo atual.

Disponibilidade: Unix, not WASI, not Android.

os.strerror(*code*, /)

Retorna a mensagem de erro correspondente ao código de erro em *code*. Nas plataformas em que **`strerror()`** retorna `NULL` quando recebe um número de erro desconhecido, **`ValueError`** é levantada.

os.supports_bytes_environ

`True` se o tipo de sistema operacional nativo do ambiente estiver em bytes (por exemplo, `False` no Windows).

Adicionado na versão 3.2.

os.umask(*mask*, /)

Define o umask numérico atual e retorna o umask anterior.

A função é uma simulação em WASI, veja Plataformas WebAssembly para mais informações.

os.uname()

Returns information identifying the current operating system. The return value is a `uname_result`.

On macOS, iOS and Android, this returns the *kernel* name and release (i.e., `'Darwin'` on macOS and iOS; `'Linux'` on Android). `platform.uname()` can be used to get the user-facing operating system name and release on iOS and Android.

Ver também

`sys.platform` which has finer granularity.

O módulo `platform` fornece verificações detalhadas sobre a identificação do sistema.

Disponibilidade: Unix.

Alterado na versão 3.3: Tipo de retorno foi alterado de uma tupla para um objeto tupla ou similar com atributos nomeados.

class os.uname_result

Name and information about the system returned by `os.uname()`. These attributes correspond to the members described in *uname(2)* *[https://manpages.debian.org/uname(2)]*.

For backwards compatibility, this object is also iterable, behaving like a five-tuple containing `sysname`, `nodename`, `release`, `version`, and `machine` in that order.

sysname

Operating system name.

nodename

Name of machine on network. Some systems truncate **nodename** to 8 characters or to the leading component; a better way to get the

hostname is **`socket.gethostname()`** or even `socket.gethostbyaddr(socket.gethostname())`.

release
: Operating system release.

version
: Operating system version.

machine
: Hardware identifier.

os.unsetenv(*key*, /)

Cancela (exclui) a variável de ambiente denominada *key*. Tais mudanças no ambiente afetam subprocessos iniciados com **`os.system()`**, **`popen()`** ou **`fork()`** e **`execv()`**.

A exclusão de itens em **`os.environ`** é automaticamente traduzida para uma chamada correspondente a **`unsetenv()`**; no entanto, chamadas a **`unsetenv()`** não atualizam **`os.environ`**, por isso, na verdade é preferível excluir itens de **`os.environ`**.

Veja também a função **`os.reload_environ()`**.

Levanta um evento de auditoria `os.unsetenv` com o argumento `key`.

Alterado na versão 3.9: A função está agora sempre disponível e também está disponível no Windows.

os.unshare(*flags*)

Desassocia partes do contexto de execução do processo e move-as para um espaço de nomes recém-criado. Veja a página man *unshare(2)* *[https://manpages.debian.org/unshare(2)]* para mais detalhes. O argumento *flags* é uma máscara de bits, combinando zero ou mais constantes CLONE_*, que especifica quais partes do contexto de execução devem ser descompartilhadas de suas associações existentes e movidas para um

novo espaço de nomes. Se o argumento *flags* for `0`, nenhuma alteração será feita no contexto de execução do processo de chamada.

Disponibilidade: Linux >= 2.6.16.

Adicionado na versão 3.12.

Ver também

A função `setns()`.

Sinalizações para a função `unshare()`, se a implementação oferecer suporte. Veja *unshare(2)* *[https://manpages.debian.org/unshare(2)]* no manual do Linux para seu exato efeito e disponibilidade.

os.CLONE_FILES
os.CLONE_FS
os.CLONE_NEWCGROUP
os.CLONE_NEWIPC
os.CLONE_NEWNET
os.CLONE_NEWNS
os.CLONE_NEWPID
os.CLONE_NEWTIME
os.CLONE_NEWUSER
os.CLONE_NEWUTS
os.CLONE_SIGHAND
os.CLONE_SYSVSEM
os.CLONE_THREAD
os.CLONE_VM

Criação de objetos arquivos

Estas funções criam novos objetos arquivos. (Veja também `open()` para abrir os descritores de arquivos.)

os.fdopen(*fd*, **args*, ***kwargs*)

Retorna um objeto arquivo aberto conectado ao descritor de arquivo *fd*. Este é um apelido para a função embutida `open()` e aceita os mesmos argumentos. A única diferença é que o primeiro argumento de `fdopen()` deve ser sempre um inteiro.

Operações dos descritores de arquivos

Estas funções operam em fluxos de E/S referenciados usando descritores de arquivos.

Descritores de arquivos são pequenos números inteiros correspondentes a um arquivo que foi aberto pelo processo atual. Por exemplo, a entrada padrão geralmente é o descritor de arquivo 0, a saída padrão 1 e erro padrão 2. Outros arquivos abertos por um processo serão então atribuídos 3, 4, 5, e assim por diante. O nome "descritor de arquivo" é um pouco enganoso; em plataformas UNIX, sockets e pipes também são referenciados como descritores de arquivos.

O método `fileno()` pode ser usado para obter o descritor de arquivo associado a um objeto arquivo quando solicitado. Note-se que usar o descritor de arquivo diretamente ignorará os métodos do objeto arquivo, ignorando aspectos como buffer interno de dados.

os.close(*fd*)

Fecha o descritor de arquivo *fd*.

Nota

Esta função destina-se a E/S de baixo nível e deve ser aplicada a um descritor de arquivo retornado por `os.open()` ou `pipe()`. Para fechar um "objeto arquivo" retornado pela função embutida `open()` ou por `popen()` ou `fdopen()`, use seu método `close()`.

os.closerange(*fd_low*, *fd_high*, /)

Fecha todos os descritores de arquivos de *fd_low* (inclusivo) a *fd_high* (exclusivo), ignorando erros. Equivalente a (mas muito mais rápido do

que):

```
for fd in range(fd_low, fd_high):
    try:
        os.close(fd)
    except OSError:
        pass
```

os.copy_file_range(*src*, *dst*, *count*, *offset_src=None*, *offset_dst=None*)

Copia *count* bytes do descritor de arquivo *src*, começando no deslocamento *offset_src*, para o descritor de arquivo *dst*, começando no deslocamento *offset_dst*. Se *offset_src* for `None`, *src* será lido a partir da posição atual; respectivamente para *offset_dst*.

No kernel Linux anterior a 5.3, os arquivos apontados por *src* e *dst* devem residir no mesmo sistema de arquivos, caso contrário, uma exceção **`OSError`** é levantada com **`errno`** definido como **`errno.EXDEV`**.

Essa cópia é feita sem o custo adicional de transferência de dados do kernel para o espaço do usuário e depois de volta para o kernel. Além disso, alguns sistemas de arquivos podem implementar otimizações extras, como o uso de reflinks (ou seja, dois ou mais nós-i que compartilham ponteiros para os mesmos blocos de disco copy-on-write; os sistemas de arquivos suportados incluem btrfs e XFS) e cópia do lado do servidor (no caso de NFS).

A função copia bytes entre dois descritores de arquivo. As opções de texto, como a codificação e o final da linha, são ignoradas.

O valor de retorno é a quantidade de bytes copiados. Ele pode ser inferior à quantidade solicitada.

Nota

No Linux, **`os.copy_file_range()`** não deve ser usada para copiar um intervalo de um pseudo arquivo de um sistema de arquivos especial como procfs e sysfs. Ele nunca copiará byte algum e retornará 0 como se o arquivo estivesse vazio devido a um problema conhecido do kernel do Linux.

Disponibilidade: Linux >= 4.5 with glibc >= 2.27.

Adicionado na versão 3.8.

os.device_encoding(*fd*)

Retorna uma string descrevendo a codificação do dispositivo associado a *fd* se estiver conectado a um terminal; senão retorna `None`.

No Unix, se o Modo UTF-8 do Python estiver habilitado, retorna `'UTF-8'` ao invés da codificação do dispositivo.

Alterado na versão 3.10: No Unix, a função agora implementa o Modo UTF-8 do Python.

os.dup(*fd*, /)

Retorna uma cópia do descritor de arquivo *fd*. O novo descritor de arquivo é não-herdável.

No Windows, ao duplicar um fluxo padrão (0: stdin, 1: stdout, 2: stderr), o novo descritor de arquivo é herdável.

Disponibilidade: not WASI.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-herdável.

os.dup2(*fd*, *fd2*, *inheritable=True*)

Duplica o descritor de arquivo *fd* como *fd2*, fechando o último antes disso, se necessário. Retorna *fd2*. O novo descritor de arquivo é herdável por padrão ou não-herdável se *inheritable* for `False`.

Disponibilidade: not WASI.

Alterado na versão 3.4: Adicionado o parâmetro opcional *inheritable*.

Alterado na versão 3.7: Retorna *fd2* em caso de sucesso. Anteriormente, retornava sempre `None`.

os.fchmod(*fd*, *mode*)

Altera o modo do arquivo dado por *fd* ao *mode* numérico. Veja a documentação de **`chmod()`** para valores possíveis de *mode*. A partir do Python 3.3, isto é equivalente a `os.chmod(fd, mode)`.

Levanta um evento de auditoria `os.chmod` com os argumentos `path`, `mode`, `dir_fd`.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

Alterado na versão 3.13: Adicionado suporte no Windows.

os.fchown(*fd*, *uid*, *gid*)

Altera o ID do proprietário e do grupo do arquivo dado por *fd* para o *uid* e *gid* numérico. Para deixar um dos IDs inalteradas, defina-o como -1. Veja **`chown()`**. A partir do Python 3.3, isto é equivalente a `os.chown(fd, uid, gid)`.

Levanta um evento de auditoria `os.chown` com os argumentos `path`, `uid`, `gid`, `dir_fd`.

Disponibilidade: Unix.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

os.fdatasync(*fd*)

Força a gravação do arquivo com descritor de arquivo *fd* no disco. Não força a atualização de metadados.

Disponibilidade: Unix, not macOS, not iOS.

os.fpathconf(*fd*, *name*, /)

Retorna informações de configuração de sistema relevantes para um arquivo aberto. *name* especifica o valor de configuração para recuperar; pode ser uma string que é o nome de um valor do sistema definido; estes nomes são especificados em uma série de padrões (POSIX.1, Unix 95, Unix 98 e outros). Algumas plataformas definem nomes adicionais também. Os nomes conhecidos do sistema operacional hospedeiro são dadas no dicionário `pathconf_names`. Para variáveis de configuração não incluídas neste mapeamento, também é aceito passar um número inteiro para *name*.

Se *name* for uma string e não for conhecida, uma exceção **`ValueError`** é levantada. Se um valor específico para *name* não for compatível com o sistema hospedeiro, mesmo que seja incluído no `pathconf_names`, uma exceção **`OSError`** é levantada com **`errno.EINVAL`** como número do erro.

A partir do Python 3.3, é equivalente a `os.pathconf(fd, name)`.

Disponibilidade: Unix.

os.fstat(*fd*)

Captura o estado do descritor de arquivo *fd*. Retorna um objeto **`stat_result`**.

A partir do Python 3.3, é equivalente a `os.stat(fd)`.

Ver também

A função **`stat()`**.

os.fstatvfs(*fd*, /)

Return information about the filesystem containing the file associated with file descriptor *fd* in a **`statvfs_result`**, like **`statvfs()`**. As of Python 3.3, this is equivalent to `os.statvfs(fd)`.

Disponibilidade: Unix.

os.fsync(*fd*)

Força a gravação no disco de arquivo com descritor de arquivo *fd*. No Unix, isto chama a função nativa **`fsync()`**; no Windows, a função de MS **`_commit()`**.

Se você estiver começando com um objeto arquivo *f* em buffer do Python, primeiro use `f.flush()`, e depois use `os.fsync(f.fileno())`, para garantir que todos os buffers internos associados com *f* sejam gravados no disco.

Disponibilidade: Unix, Windows.

os.ftruncate(*fd*, *length*, /)

Trunca o arquivo correspondente ao descritor de arquivo *fd*, de modo que tenha no máximo *length* bytes de tamanho. A partir do Python 3.3, isto é equivalente a `os.truncate(fd, length)`.

Levanta evento de auditoria `os.truncate` com os argumentos `fd`, `length`.

Disponibilidade: Unix, Windows.

Alterado na versão 3.5: Adicionado suporte para o Windows.

os.get_blocking(*fd*, /)

Obtém o modo de bloqueio do descritor de arquivo: `False` se o sinalizador **`O_NONBLOCK`** estiver marcado, `True` se o sinalizador estiver desmarcado.

Veja também **`set_blocking()`** e **`socket.socket.setblocking()`**.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

No Windows, esta função é limitada a encadeamentos.

Adicionado na versão 3.5.

Alterado na versão 3.12: Adicionado suporte a encadeamentos no Windows.

os.grantpt(*fd*, /)

Concede acesso ao dispositivo pseudoterminal secundário associado ao dispositivo pseudoterminal principal ao qual o descritor de arquivo *fd* se refere. O descritor de arquivo *fd* não é fechado em caso de falha.

Chama a função **`grantpt()`** da biblioteca padrão C.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

os.isatty(*fd*, /)

Retorna `True` se o descritor de arquivo *fd* estiver aberto e conectado a um dispositivo do tipo tty, senão `False`.

os.lockf(*fd*, *cmd*, *len*, /)

Aplica, testa ou remove uma trava POSIX em um descritor de arquivo aberto. *fd* é um descritor de arquivo aberto. *cmd* especifica o comando a ser usado - um dentre **`F_LOCK`**, **`F_TLOCK`**, **`F_ULOCK`** ou **`F_TEST`**. *len* especifica a seção do arquivo para travar.

Levanta um evento de auditoria `os.lockf` com os argumentos `fd`, `cmd`, `len`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.F_LOCK
os.F_TLOCK
os.F_ULOCK
os.F_TEST

Sinalizadores que especificam qual ação `lockf()` vai executar.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.login_tty(*fd*, /)

Prepara o tty do qual fd é um descritor de arquivo para uma nova sessão de login. Faz do processo de chamada um líder de sessão; torna o tty o tty de controle, o stdin, o stdout e o stderr do processo de chamada; fecha o fd.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.11.

os.lseek(*fd*, *pos*, *whence*, /)

Define a posição atual do descritor de arquivo *fd* para a posição *pos*, modificada por *whence*, e retorna a nova posição em bytes relativa ao início do arquivo. Os valores válidos de *whence* são:

- `SEEK_SET` ou `0` – define *pos* em relação ao início do arquivo
- `SEEK_CUR` ou `1` – define *pos* em relação à posição atual do arquivo
- `SEEK_END` ou `2` – define *pos* em relação ao final do arquivo
- `SEEK_HOLE` – define *pos* para o próximo local de dados, em relação a *pos*
- `SEEK_DATA` – define *pos* para o próximo buraco de dados, em relação a *pos*

Alterado na versão 3.3: Adicionado suporte para `SEEK_HOLE` e `SEEK_DATA`.

os.SEEK_SET
os.SEEK_CUR
os.SEEK_END

Parâmetros para a função `lseek()` e o método `seek()` em objetos arquivo ou similares, para onde ajustar o indicador de posição do arquivo.

SEEK_SET
Ajusta a posição do arquivo em relação ao início do arquivo.

SEEK_CUR
Ajusta a posição do arquivo em relação a sua posição atual.

SEEK_END
Ajusta a posição do arquivo em relação ao fim do arquivo.

Seus valores são 0, 1 e 2, respectivamente.

os.SEEK_HOLE
os.SEEK_DATA

Parâmetros para a função `lseek()` e o método `seek()` em objetos arquivo ou similares, para para buscar dados de arquivos e buracos em arquivos alocados de forma esparsa.

SEEK_DATA
Ajusta o deslocamento do arquivo para o próximo local contendo dados, em relação à posição de busca.

SEEK_HOLE
Ajusta o deslocamento do arquivo para o próximo local contendo um buraco, em relação à posição de busca. Um buraco é definido como uma sequência de zeros.

Nota

Essas operações só fazem sentido para sistemas de arquivos que as suportam.

Disponibilidade: Linux >= 3.1, macOS, Unix

Adicionado na versão 3.3.

os.open(*path*, *flags*, *mode=0o777*, *, *dir_fd=None*)

Abre o arquivo *path* e define vários sinalizadores de acordo com *flags* e, possivelmente, seu modo, de acordo com *mode*. Ao computar *mode*, o

valor atual de umask é primeiro mascarado. Retorna o descritor de arquivo para o arquivo recém-aberto. O novo descritor de arquivo é não-herdável.

For a description of the flag and mode values, see the C run-time documentation; flag constants (like `O_RDONLY` and `O_WRONLY`) are defined in the `os` module. In particular, on Windows adding `O_BINARY` is needed to open files in binary mode.

Esta função oferece suporte para caminhos relativos aos descritores de diretório com o parâmetro *dir_fd*.

Levanta um evento de auditoria `open` com os argumentos `path`, `mode`, `flags`.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-herdável.

Nota

This function is intended for low-level I/O. For normal usage, use the built-in function `open()`, which returns a file object with `read()` and `write()` methods. To wrap a file descriptor in a file object, use `fdopen()`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** [https://peps.python.org/pep-0475/] para entender a justificativa).

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

As seguintes constantes são opções para o parâmetro *flags* da função `open()`. Elas podem ser combinadas usando o operador bit a bit OU `|`. Algumas delas não estão disponíveis em todas as plataformas. Para obter

descrições de sua disponibilidade e uso, consulte a página de manual *open(2)* *[https://manpages.debian.org/open(2)]* para Unix ou o MSDN [https://msdn.microsoft.com/en-us/library/z0kc8e3z.aspx] para Windows.

os.O_RDONLY
os.O_WRONLY
os.O_RDWR
os.O_APPEND
os.O_CREAT
os.O_EXCL
os.O_TRUNC

As constantes acima estão disponíveis no Unix e Windows.

os.O_DSYNC
os.O_RSYNC
os.O_SYNC
os.O_NDELAY
os.O_NONBLOCK
os.O_NOCTTY
os.O_CLOEXEC

As constantes acima estão disponíveis apenas no Unix.

Alterado na versão 3.3: Adicionada a constante `O_CLOEXEC`.

os.O_BINARY
os.O_NOINHERIT
os.O_SHORT_LIVED
os.O_TEMPORARY
os.O_RANDOM
os.O_SEQUENTIAL
os.O_TEXT

As constantes acima estão disponíveis apenas no Windows.

os.O_EVTONLY
os.O_FSYNC
os.O_SYMLINK
os.O_NOFOLLOW_ANY

As constantes acima estão disponíveis apenas no macOS.

Alterado na versão 3.10: Adicionadas as constantes **`O_EVTONLY`**, **`O_FSYNC`**, **`O_SYMLINK`** e **`O_NOFOLLOW_ANY`**.

os.O_ASYNC
os.O_DIRECT
os.O_DIRECTORY
os.O_NOFOLLOW
os.O_NOATIME
os.O_PATH
os.O_TMPFILE
os.O_SHLOCK
os.O_EXLOCK

As constantes acima são extensões e não estarão presentes, se não forem definidos pela biblioteca C.

Alterado na versão 3.4: Adicionada **`O_PATH`** em sistemas que oferecem suporte. Adicionada **`O_TMPFILE`**, somente disponível no kernel Linux 3.11 ou mais recente.

os.openpty()

Abre um novo par de pseudo-terminal. Retorna um par de descritores de arquivos `(master, slave)` para o pty e o tty, respectivamente. Os novos descritores de arquivos são não-herdáveis. Para uma abordagem (ligeiramente) mais portável, use o módulo **`pty`**.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.4: Os novos descritores de arquivos agora são não-herdáveis.

os.pipe()

Cria um encadeamento (pipe). Retorna um par de descritores de arquivos `(r, w)` usáveis para leitura e escrita, respectivamente. O novo descritor de arquivo é não-herdável.

Disponibilidade: Unix, Windows.

Alterado na versão 3.4: Os novos descritores de arquivos agora são não-herdáveis.

os.pipe2(*flags*, /)

Cria um encadeamento (pipe) com *flags* definidos atomicamente. *flags* podem ser construídos por aplicação de OU junto a um ou mais destes valores: `O_NONBLOCK`, `O_CLOEXEC`. Retorna um par de descritores de arquivos `(r, w)` utilizáveis para leitura e gravação, respectivamente.

Disponibilidade: Unix, macOS >= 27.0, not WASI, not iOS.

Adicionado na versão 3.3.

os.posix_fallocate(*fd*, *offset*, *len*, /)

Garante que um espaço em disco suficiente seja alocado para o arquivo especificado por *fd* iniciando em *offset* e continuando por *len* bytes.

Disponibilidade: Unix, not macOS, not iOS.

Adicionado na versão 3.3.

os.posix_fadvise(*fd*, *offset*, *len*, *advice*, /)

Anuncia a intenção de acessar dados em um padrão específico, permitindo assim que o kernel faça otimizações. O conteúdo em *advice* se aplica à região do arquivo especificado por *fd*, iniciando em *offset* e continuando por *len* bytes. *advice* é um entre `POSIX_FADV_NORMAL`, `POSIX_FADV_SEQUENTIAL`, `POSIX_FADV_RANDOM`, `POSIX_FADV_NOREUSE`, `POSIX_FADV_WILLNEED` ou `POSIX_FADV_DONTNEED`.

Disponibilidade: Unix, not macOS, not iOS.

Adicionado na versão 3.3.

os.POSIX_FADV_NORMAL
os.POSIX_FADV_SEQUENTIAL
os.POSIX_FADV_RANDOM

os.POSIX_FADV_NOREUSE
os.POSIX_FADV_WILLNEED
os.POSIX_FADV_DONTNEED

Sinalizadores que podem ser usados em *advice* em **posix_fadvise()** que especificam o padrão de acesso que provavelmente será usado.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.pread(*fd*, *n*, *offset*, /)

Lê no máximo *n* bytes do descritor de arquivo *fd* na posição *offset*, mantendo o deslocamento do arquivo inalterado.

Retorna uma bytestring contendo os bytes lidos. Se o final do arquivo referido por *fd* for atingido, um objeto de bytes vazio será retornado.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.posix_openpt(*oflag*, /)

Abre e retorna um descritor de arquivo para um dispositivo pseudoterminal mestre.

Chama a função **posix_openpt()** da biblioteca padrão C. O argumento *oflag* é usado para definir sinalizadores de status de arquivo e modos de acesso a arquivos conforme especificado na página de manual da **posix_openpt()** do seu sistema.

O descritor de arquivo retornado é não herdável. Se o valor **O_CLOEXEC** estiver disponível no sistema, ele é adicionado a *oflag*.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

os.preadv(*fd*, *buffers*, *offset*, *flags=0*, /)

Lê de um descritor de arquivo *fd* na posição de *offset* em *buffers* mutáveis de objetos bytes ou similares, deixando o deslocamento do arquivo inalterado. Transfere os dados para cada buffer até ficar cheio e depois passa para o próximo buffer na sequência para armazenar o restante dos dados.

O argumento *flags* contém um OR bit a bit de zero ou mais dos seguintes sinalizadores:

- **`RWF_HIPRI`**
- **`RWF_NOWAIT`**
- **`RWF_DONTCACHE`**

Retorna o número total de bytes realmente lidos, que pode ser menor que a capacidade total de todos os objetos.

O sistema operacional pode definir um limite (**`sysconf()`** valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Combina a funcionalidade de **`os.readv()`** e **`os.pread()`**.

Disponibilidade: Linux >= 2.6.30, FreeBSD >= 6.0, OpenBSD >= 2.7, AIX >= 7.1.

O uso de sinalizadores requer Linux >= 4.6.

Adicionado na versão 3.7.

os.RWF_NOWAIT

Não aguarda por dados que não estão disponíveis imediatamente. Se esse sinalizador for especificado, a chamada do sistema retorna instantaneamente se for necessário ler dados do armazenamento de backup ou aguardar uma trava.

Se alguns dados foram lidos com sucesso, ele retorna o número de bytes lidos. Se nenhum bytes foi lido, ele retornará `-1` e definirá errno como **`errno.EAGAIN`**.

Disponibilidade: Linux >= 4.14.

Adicionado na versão 3.7.

os.RWF_HIPRI

Alta prioridade de leitura/gravação. Permite sistemas de arquivos baseados em blocos para usar a consulta do dispositivo, que fornece latência inferior, mas pode usar recursos adicionais.

Atualmente, no Linux, esse recurso é usável apenas em um descritor de arquivo aberto usando o sinalizador **O_DIRECT**.

Disponibilidade: Linux >= 4.6.

Adicionado na versão 3.7.

os.RWF_DONTCACHE

Use uncached buffered IO.

Disponibilidade: Linux >= 6.14

Adicionado na versão 3.15.

os.RWF_ATOMIC

Write data atomically. Requires alignment to the device's atomic write unit.

Disponibilidade: Linux >= 6.11

Adicionado na versão 3.15.

os.ptsname(*fd*, /)

Retorna o nome do dispositivo pseudoterminal secundário associado ao dispositivo pseudoterminal principal ao qual o descritor de arquivo *fd* se refere. O descritor de arquivo *fd* não é fechado em caso de falha.

Chama a função reentrante **ptsname_r()** da biblioteca padrão C se estiver disponível; caso contrário, a função **ptsname()** da biblioteca padrão C, que não é garantida como sendo segura para thread, é chamada.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

os.pwrite(*fd*, *str*, *offset*, /)

Escreve a bytestring in *str* no descritor de arquivo *fd* na posição *offset*, mantendo o deslocamento do arquivo inalterado.

Retorna o número de bytes realmente escritos.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.pwritev(*fd*, *buffers*, *offset*, *flags=0*, /)

Escreve o conteúdo de *buffers* no descritor de arquivo *fd* com um deslocamento *offset*, deixando o deslocamento do arquivo inalterado. *buffers* deve ser uma sequência de objetos bytes ou similares. Os buffers são processados em ordem de vetor. Todo o conteúdo do primeiro buffer é gravado antes de prosseguir para o segundo, e assim por diante.

O argumento *flags* contém um OR bit a bit de zero ou mais dos seguintes sinalizadores:

- **`RWF_DSYNC`**
- **`RWF_SYNC`**
- **`RWF_APPEND`**
- **`RWF_DONTCACHE`**
- **`RWF_ATOMIC`**

Retorna o número total de bytes realmente escritos.

O sistema operacional pode definir um limite (**`sysconf()`** valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Combina a funcionalidade de **`os.writev()`** e **`os.pwrite()`**.

Disponibilidade: Linux >= 2.6.30, FreeBSD >= 6.0, OpenBSD >= 2.7, AIX >= 7.1.

O uso de sinalizadores requer Linux >= 4.6.

Adicionado na versão 3.7.

os.RWF_DSYNC

Fornece um equivalente para gravação do sinalizador `O_DSYNC` da função `os.open()`. Este efeito de sinalizador se aplica apenas ao intervalo de dados escrito pela chamada de sistema.

Disponibilidade: Linux >= 4.7.

Adicionado na versão 3.7.

os.RWF_SYNC

Fornece um equivalente para gravação do sinalizador `O_SYNC` da função `os.open()`. Este efeito de sinalizador se aplica apenas ao intervalo de dados escrito pela chamada de sistema.

Disponibilidade: Linux >= 4.7.

Adicionado na versão 3.7.

os.RWF_APPEND

Fornece um equivalente para gravação do sinalizador `O_APPEND` da função `os.open()`. Esse sinalizador é significante apenas para `os.pwritev()`, e seu efeito se aplica apenas ao intervalo de dados escrito pela chamada de sistema. O argumento *offset* não afeta a operação de escrita; o dado é sempre adicionado ao fim do arquivo. Contudo, se o argumento *offset* for `-1`, o *offset* do arquivo atual é atualizado.

Disponibilidade: Linux >= 4.16.

Adicionado na versão 3.10.

os.read(*fd*, *n*, /)

Lê no máximo *n* bytes do descritor de arquivos *fd*.

Retorna uma bytestring contendo os bytes lidos. Se o final do arquivo referido por *fd* for atingido, um objeto de bytes vazio será retornado.

Nota

This function is intended for low-level I/O and must be applied to a file descriptor as returned by `os.open()` or `pipe()`. To read a "file object" returned by the built-in function `open()` or by `popen()` or `fdopen()`, or `sys.stdin`, use its `read()` or `readline()` methods.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** [https://peps.python.org/pep-0475/] para entender a justificativa).

os.readinto(*fd*, *buffer*, /)

Lê de um descritor de arquivo *fd* em um *buffer* de objeto buffer mutável.

O *buffer* deve ser mutável e bytes ou similar. Em caso de sucesso, retorna o número de bytes lidos. Menos bytes podem ser lidos do que o tamanho do buffer. A chamada de sistema subjacente será repetida quando interrompida por um sinal, a menos que o manipulador de sinais levante uma exceção. Outros erros não serão repetidos e um erro será levantado.

Retorna 0 se *fd* estiver no final do arquivo ou se o *buffer* fornecido tiver comprimento 0 (o que pode ser usado para verificar erros sem ler dados). Nunca retorna negativo.

Nota

Esta função destina-se a E/S de baixo nível e deve ser aplicada a um descritor de arquivo retornado por `os.open()` ou `os.pipe()`. Para ler um "objeto arquivo" retornado pela função embutida `open()` ou

`sys.stdin`, use suas funções-membro, por exemplo `io.BufferedIOBase.readinto()`, `io.BufferedIOBase.read()` ou `io.TextIOBase.read()`.

Adicionado na versão 3.14.

os.sendfile(*out_fd*, *in_fd*, *offset*, *count*)
os.sendfile(*out_fd*, *in_fd*, *offset*, *count*, *headers=()*, *trailers=()*, *flags=0*)

Copia *count* bytes do descritor de arquivo *in_fd* para o descritor de arquivo *out_fd* começando em *offset*. Retorna o número de bytes enviados. Quando o EOF é alcançado, retorna `0`.

A primeira notação da função está disponível por todas as plataformas que definem `sendfile()`.

No Linux, se *offset* for fornecido como `None`, os bytes são lidos da posição atual de *in_fd* e a posição de *in_fd* é atualizada.

O segundo caso pode ser usado no macOS e FreeBSD onde *headers* e *trailers* são sequências arbitrárias de buffers que são escritos antes e depois dos dados de *in_fd* ser escrito. Retorna o mesmo que no primeiro caso.

No macOS e FreeBSD, um valor de `0` para *count* especifica enviar até o fim de *in_fd* ser alcançado.

Todas as plataformas tem suporte a soquetes como descritor de arquivo *out_fd*, e algumas plataformas permitem outros tipos (por exemplo, arquivo regular, encadeamento) também.

Aplicativos de plataforma cruzada não devem usar os argumentos *headers*, *trailers* e *flags*.

Disponibilidade: Unix, not WASI.

Nota

Para um invólucro de nível mais alto de `sendfile()`, consulte `socket.socket.sendfile()`.

Adicionado na versão 3.3.

Alterado na versão 3.9: Os parâmetros *out* e *in* foram renomeados para *out_fd* e *in_fd*.

os.SF_NODISKIO
os.SF_MNOWAIT
os.SF_SYNC

Parâmetros para a função `sendfile()`, se a implementação tiver suporte a eles.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.3.

os.SF_NOCACHE

Parâmetro para a função `sendfile()`, se a implementação tiver suporte a isso. Os dados não serão armazenados em cache na memória virtual e serão liberados posteriormente.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.11.

os.set_blocking(*fd*, *blocking*, /)

Define o modo de bloqueio do descritor de arquivo especificado. Define o sinalizador `O_NONBLOCK` se blocking for `False`; do contrário, limpa o sinalizador.

Veja também `get_blocking()` e `socket.socket.setblocking()`.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

No Windows, esta função é limitada a encadeamentos.

Adicionado na versão 3.5.

Alterado na versão 3.12: Adicionado suporte a encadeamentos no Windows.

os.splice(*src*, *dst*, *count*, *offset_src=None*, *offset_dst=None*, *flags=0*)

Transfere *count* bytes do descritor de arquivo *src*, começando no deslocamento *offset_src*, para o descritor de arquivo *dst*, começando no deslocamento *offset_dst*.

O comportamento da união via splice pode ser modificado especificando um valor para *flags*. Qualquer um dos seguintes valores pode ser usado, combinados usando OU (OR) bit a bit (o operador `|`):

- Se `SPLICE_F_MOVE` for especificado, o kernel será solicitado a mover páginas em vez de copiá-las, mas as páginas ainda poderão ser copiadas se o kernel não puder movê-las do encadeamento.
- Se `SPLICE_F_NONBLOCK` for especificado, o kernel será solicitado a não bloquear em E/S. Isso torna as operações de encadeamento de emenda não bloqueantes, mas a emenda pode, ainda assim, bloquear, pois os descritores de arquivo emendados podem bloquear.
- Se `SPLICE_F_MORE` for especificado, ele indica ao kernel que mais dados chegarão em uma emenda subsequente.

Pelo menos um dos descritores de arquivo deve se referir a um encadeamento. Se *offset_src* é `None`, então *src* é lido a partir da posição atual; o mesmo para *offset_dst*. O deslocamento associado a um descritor de arquivo que se refere a um encadeamento deve ser `None`. Os arquivos apontados por *src* e *dst* devem residir no mesmo sistema de arquivos, de outra forma uma `OSError` é levantada com `errno` definido para `errno.EXDEV`.

Essa cópia é feita sem o custo adicional de transferência de dados do kernel para o espaço do usuário e, em seguida, volta para o kernel. Além disso, alguns sistemas de arquivos poderiam implementar otimizações extras. A cópia é feita como se ambos os arquivos estivessem abertos como binários.

Após a conclusão bem-sucedida, retorna o número de bytes unidos ao encadeamento ou a partir dele. Um valor de retorno de 0 significa o fim da entrada. Se *src* se refere a um encadeamento, isso significa que não havia dados para transferir, e não faria sentido bloquear porque não há escritores conectados ao fim da escrita do encadeamento.

Ver também

A página man *splice(2)* *[https://manpages.debian.org/splice(2)]*.

Disponibilidade: Linux >= 2.6.17 with glibc >= 2.5

Adicionado na versão 3.10.

os.SPLICE_F_MOVE
os.SPLICE_F_NONBLOCK
os.SPLICE_F_MORE

Adicionado na versão 3.10.

os.readv(*fd*, *buffers*, */*)

Lê de um descritor de arquivo *fd* em um número de *buffers* objetos bytes ou similares mutáveis. Transfere os dados para cada buffer até que esteja cheio e, a seguir, vai para o próximo buffer na sequência para armazenar o restante dos dados.

Retorna o número total de bytes realmente lidos, que pode ser menor que a capacidade total de todos os objetos.

O sistema operacional pode definir um limite (`sysconf()` valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.tcgetpgrp(*fd*, *[/]*)

Retorna o grupo de processos associado ao terminal fornecido por *fd* (um descritor de arquivo aberto retornado por `os.open()`).

Disponibilidade: Unix, not WASI.

os.tcsetpgrp(*fd*, *pg*, *[/]*)

Define o grupo de processos associado ao terminal fornecido por *fd* (um descritor de arquivo aberto retornado por `os.open()`) para *pg*.

Disponibilidade: Unix, not WASI.

os.ttyname(*fd*, *[/]*)

Retorna uma string que especifica o dispositivo de terminal associado ao descritor de arquivo *fd*. Se *fd* não estiver associado a um dispositivo de terminal, uma exceção é levantada.

Disponibilidade: Unix.

os.unlockpt(*fd*, *[/]*)

Destrava o dispositivo pseudoterminal secundário associado ao dispositivo pseudoterminal principal ao qual o descritor de arquivo *fd* se refere. O descritor de arquivo *fd* não é fechado em caso de falha.

Chama a função `unlockpt()` da biblioteca padrão C.

Disponibilidade: Unix, not WASI.

Adicionado na versão 3.13.

os.write(*fd*, *str*, *[/]*)

Escreve a string de bytes em *str* no descritor de arquivo *fd*.

Retorna o número de bytes realmente escritos.

Nota

This function is intended for low-level I/O and must be applied to a file descriptor as returned by `os.open()` or `pipe()`. To write a "file object" returned by the built-in function `open()` or by `popen()` or `fdopen()`, or `sys.stdout` or `sys.stderr`, use its `write()` method.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** [https://peps.python.org/pep-0475/] para entender a justificativa).

os.writev(*fd*, *buffers*, */*)

Escreve o conteúdo de *buffers* no descritor de arquivo *fd*. *buffers* deve ser uma sequência de objetos bytes ou similares. Os buffers são processados na ordem em que estão. Todo o conteúdo do primeiro buffer é gravado antes de prosseguir para o segundo, e assim por diante.

Retorna o número total de bytes realmente escritos.

O sistema operacional pode definir um limite (`sysconf()` valor `'SC_IOV_MAX'`) no número de buffers que podem ser usados.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Consultando o tamanho de um terminal

Adicionado na versão 3.3.

os.get_terminal_size(*fd=STDOUT_FILENO*, */*)

Retorna o tamanho da janela do terminal como `(columns, lines)`, tupla do tipo `terminal_size`.

O argumento opcional `fd` (padrão `STDOUT_FILENO`, ou saída padrão) especifica qual descritor de arquivo deve ser consultado.

Se o descritor de arquivo não estiver conectado a um terminal, uma exceção `OSError` é levantada.

`shutil.get_terminal_size()` é a função de alto nível que normalmente deve ser usada, `os.get_terminal_size` é a implementação de baixo nível.

Disponibilidade: Unix, Windows.

class os.terminal_size

Uma subclasse de tupla, contendo `(columns, lines)` do tamanho da janela do terminal.

columns

Largura da janela do terminal em caracteres.

lines

Altura da janela do terminal em caracteres.

Herança de descritores de arquivos

Adicionado na versão 3.4.

Um descritor de arquivo tem um sinalizador "herdável" que indica se o descritor de arquivo pode ser herdado por processos filho. A partir do Python 3.4, os descritores de arquivo criados pelo Python não são herdáveis por padrão.

On UNIX, non-inheritable file descriptors are closed in child processes at the execution of a new program, other file descriptors are inherited. Note that non-inheritable file descriptors are still *inherited* by child processes on `os.fork()`.

No Windows, identificadores não herdáveis e descritores de arquivo são fechados em processos filho, exceto para fluxos padrão (descritores de

arquivo 0, 1 e 2: stdin, stdout e stderr), que são sempre herdados. Usando as funções `spawn*`, todos os identificadores herdáveis e todos os descritores de arquivo herdáveis são herdados. Usando o módulo `subprocess`, todos os descritores de arquivo, exceto fluxos padrão, são fechados, e os manipuladores herdáveis são herdados apenas se o parâmetro *close_fds* for `False`.

Nas plataformas WebAssembly, o descritor de arquivo não pode ser modificado.

os.get_inheritable(*fd*, /)

Obtém o sinalizador "herdável" do descritor de arquivo especificado (um booleano).

os.set_inheritable(*fd*, *inheritable*, /)

Define o sinalizador "herdável" do descritor de arquivo especificado.

os.get_handle_inheritable(*handle*, /)

Obtém o sinalizador "herdável" do manipulador especificado (um booleano).

Disponibilidade: Windows.

os.set_handle_inheritable(*handle*, *inheritable*, /)

Define o sinalizador "herdável" do manipulador especificado.

Disponibilidade: Windows.

Arquivos e diretórios

Em algumas plataformas Unix, muitas dessas funções oferecem suporte para um ou mais destes recursos:

- **specifying a file descriptor:** Normally the *path* argument provided to functions in the `os` module must be a string specifying a file path. However, some functions now alternatively accept an open file

descriptor for their *path* argument. The function will then operate on the file referred to by the descriptor. For POSIX systems, Python will call the variant of the function prefixed with `f` (e.g. call `fchdir` instead of `chdir`).

Você pode verificar se *path* pode ser especificado ou não como um descritor de arquivo para uma função particular em sua plataforma usando **`os.supports_fd`**. Se esta funcionalidade não estiver disponível, usá-la levantará uma **`NotImplementedError`**.

Se a função também oferecer suporte para os argumentos *dir_fd* ou *follow_symlinks*, é um erro especificar um deles ao fornecer *path* como um descritor de arquivo.

- **caminhos relativos aos descritores de diretório:** se *dir_fd* não for `None`, deve ser um descritor de arquivo referindo-se a um diretório, e o caminho para operar deve ser relativo; o caminho será relativo a esse diretório. Se o caminho for absoluto, *dir_fd* será ignorado. Para sistemas POSIX, Python irá chamar a variante da função com um sufixo `at` e possivelmente prefixado com `f` (por exemplo, chamar `faccessat` ao invés de `access`).

  Você pode verificar se há ou não suporte para *dir_fd* em uma função particular em sua plataforma usando **`os.supports_dir_fd`**. Se não estiver disponível, usá-lo levantará uma **`NotImplementedError`**.

- **não seguir links simbólicos:** se *follow_symlinks* for `False`, e o último elemento do caminho para operar for um link simbólico, a função irá operar no próprio link simbólico ao invés do arquivo apontado pelo link. Para sistemas POSIX, Python irá chamar a variante `l...` da função.

  Você pode verificar se há ou não suporte para *follow_symlinks* em uma função particular em sua plataforma usando **`os.supports_follow_symlinks`**. Se não estiver disponível, usá-lo levantará uma **`NotImplementedError`**.

os.access(*path*, *mode*, **, dir_fd=None*, *effective_ids=False*, *follow_symlinks=True*)

Usa o uid/gid real para testar o acesso ao *path*. Observe que a maioria das operações usará o uid/gid efetivo, portanto, essa rotina pode ser usada em um ambiente suid/sgid para testar se o usuário da chamada tem o acesso especificado ao *path*. *mode* deve ser `F_OK` para testar a existência de *path*, ou pode ser o OU inclusivo de um ou mais dos `R_OK`, `W_OK`, e `X_OK` para testar as permissões. Retorna `True` se o acesso for permitido, `False` se não for. Veja a página man do Unix *access(2)* *[https://manpages.debian.org/access(2)]* para mais informações.

Esta função pode oferecer suporte a especificação de caminhos relativos aos descritores de diretório e não seguir os links simbólicos.

Se *effective_ids* for `True`, `access()` irá realizar suas verificações de acesso usando o uid/gid efetivo ao invés do uid/gid real. *effective_ids* pode não ser compatível com sua plataforma; você pode verificar se está ou não disponível usando `os.supports_effective_ids`. Se não estiver disponível, usá-lo levantará uma `NotImplementedError`.

Nota

Usar `access()` para verificar se um usuário está autorizado a, por exemplo, abrir um arquivo antes de realmente fazer isso usando `open()` cria uma brecha de segurança, porque o usuário pode explorar o curto intervalo de tempo entre a verificação e a abertura do arquivo para manipulá-lo. É preferível usar as técnicas EAFP. Por exemplo:

```
if os.access("meuarquivo", os.R_OK):
    with open("meuarquivo") as fp:
        return fp.read()
return "algum dado padrão"
```

é melhor escrito como:

```
try:
    fp = open("meuarquivo")
except PermissionError:
    return "algum dado padrão"
```

```
else:
    with fp:
        return fp.read()
```

Nota

As operações de E/S podem falhar mesmo quando `access()` indica que elas teriam sucesso, particularmente para operações em sistemas de arquivos de rede que podem ter semântica de permissões além do modelo de bits de permissão POSIX usual.

Alterado na versão 3.3: Adicionados os parâmetros *dir_fd*, *effective_ids* e *follow_symlinks*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.F_OK
os.R_OK
os.W_OK
os.X_OK

Valores a serem passados como o parâmetro *mode* de `access()` para testar a existência, legibilidade, capacidade de escrita e executabilidade de *path*, respectivamente.

os.chdir(*path*)

Altera o diretório de trabalho atual para *path*.

Esta função pode oferecer suporte a especificar um descritor de arquivo. O descritor deve fazer referência a um diretório aberto, não a um arquivo aberto.

Esta função pode levantar `OSError` e subclasses como `FileNotFoundError`, `PermissionError` e `NotADirectoryError`.

Levanta um evento de auditoria `os.chdir` com o argumento `path`.

Ver também

The `contextlib.chdir()` context manager, which changes the current working directory on entering and restores the previous one on exit.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo em algumas plataformas.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.chflags(*path*, *flags*, ***, *follow_symlinks=True*)

Define os sinalizadores de *path* para os *flags* numéricos. *flags* podem assumir uma combinação (OU bit a bit) dos seguintes valores (conforme definido no módulo `stat`):

- `stat.UF_NODUMP`
- `stat.UF_IMMUTABLE`
- `stat.UF_APPEND`
- `stat.UF_OPAQUE`
- `stat.UF_NOUNLINK`
- `stat.UF_COMPRESSED`
- `stat.UF_HIDDEN`
- `stat.SF_ARCHIVED`
- `stat.SF_IMMUTABLE`
- `stat.SF_APPEND`
- `stat.SF_NOUNLINK`
- `stat.SF_SNAPSHOT`

Esta função pode oferecer suporte a não seguir links simbólicos.

Levanta um evento de auditoria `os.chflags` com os argumentos `path`, `flags`.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.3: Adicionado o parâmetro *follow_symlinks*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.chmod(*path*, *mode*, ***, *dir_fd=None*, *follow_symlinks=True*)

Altera o modo de *path* para o *mode* numérico. *mode* pode assumir um dos seguintes valores (conforme definido no módulo `stat`) ou combinações de OU bit a bit deles:

- `stat.S_ISUID`
- `stat.S_ISGID`
- `stat.S_ENFMT`
- `stat.S_ISVTX`
- `stat.S_IREAD`
- `stat.S_IWRITE`
- `stat.S_IEXEC`
- `stat.S_IRWXU`
- `stat.S_IRUSR`
- `stat.S_IWUSR`
- `stat.S_IXUSR`
- `stat.S_IRWXG`
- `stat.S_IRGRP`
- `stat.S_IWGRP`
- `stat.S_IXGRP`
- `stat.S_IRWXO`
- `stat.S_IROTH`
- `stat.S_IWOTH`
- `stat.S_IXOTH`

Esta função pode oferecer suporte a especificar um descritor de arquivo, caminhos relativos aos descritores de diretório e não seguir os links simbólicos.

Nota

Embora o Windows ofereça suporte ao `chmod()`, você só pode definir o sinalizador de somente leitura do arquivo (através das constantes `stat.S_IWRITE` e `stat.S_IREAD` ou um valor inteiro correspondente). Todos os outros bits são ignorados. O valor padrão de *follow_symlinks* é `False` no Windows.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

Levanta um evento de auditoria `os.chmod` com os argumentos `path`, `mode`, `dir_fd`.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto e os argumentos *dir_fd* e *follow_symlinks*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.13: Adicionado suporte para um descritor de arquivo e o argumento *follow_symlinks* no Windows.

os.chown(*path*, *uid*, *gid*, **, dir_fd=None*, *follow_symlinks=True*)

Altera o proprietário e o id de grupo de *path* para *uid* e *gid* numéricos. Para deixar um dos ids inalterado, defina-o como -1.

Esta função pode oferecer suporte a especificar um descritor de arquivo, caminhos relativos aos descritores de diretório e não seguir os links simbólicos.

Consulte **`shutil.chown()`** para uma função de alto nível que aceita nomes além de ids numéricos.

Levanta um evento de auditoria `os.chown` com os argumentos `path`, `uid`, `gid`, `dir_fd`.

Disponibilidade: Unix.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto e os argumentos *dir_fd* e *follow_symlinks*.

Alterado na versão 3.6: Oferece suporte para um objeto caminho ou similar.

os.chroot(*path*)

Altera o diretório raiz do processo atual para *path*

Disponibilidade: Unix, not WASI, not Android.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.fchdir(*fd*)

Altera o diretório de trabalho atual para o diretório representado pelo descritor de arquivo *fd*. O descritor deve se referir a um diretório aberto, não a um arquivo aberto. No Python 3.3, isso é equivalente a `os.chdir(fd)`.

Levanta um evento de auditoria `os.chdir` com o argumento `path`.

Disponibilidade: Unix.

os.getcwd()

Retorna uma string representando o diretório de trabalho atual.

os.getcwdb()

Retorna uma bytestring representando o diretório de trabalho atual.

Alterado na versão 3.8: A função agora usa a codificação UTF-8 no Windows, em vez da página de código ANSI: consulte a **PEP 529** [https://peps.python.org/pep-0529/] para a justificativa. A função não está mais descontinuada no Windows.

os.lchflags(*path*, *flags*)

Define os sinalizadores de *path* para os *flags* numéricos, como **`chflags()`**, mas não segue links simbólicos. No Python 3.3, isso é equivalente a `os.chflags(path, flags, follow_symlinks=False)`.

Levanta um evento de auditoria `os.chflags` com os argumentos `path`, `flags`.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.lchmod(*path*, *mode*)

Altera o modo de *path* para o *mode* numérico. Se o caminho for um link simbólico, isso afetará o link simbólico em vez do destino. Veja a documentação de **`chmod()`** para valores possíveis de *mode*. No Python 3.3, isso é equivalente a `os.chmod(path, mode, follow_symlinks=False)`.

`lchmod()` não faz parte do POSIX, mas as implementações Unix podem tê-lo se houver suporte para alterar o modo de links simbólicos.

Levanta um evento de auditoria `os.chmod` com os argumentos `path`, `mode`, `dir_fd`.

Disponibilidade: Unix, Windows, not Linux, FreeBSD >= 1.3, NetBSD >= 1.3, not OpenBSD

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.13: Adicionado suporte no Windows.

os.lchown(*path*, *uid*, *gid*)

Altera o proprietário e o id de grupo de *path* para *uid* e *gid* numéricos. Esta função não seguirá links simbólicos. No Python 3.3, isso é equivalente a `os.chown(path, uid, gid, follow_symlinks=False)`.

Levanta um evento de auditoria `os.chown` com os argumentos `path`, `uid`, `gid`, `dir_fd`.

Disponibilidade: Unix.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.link(*src*, *dst*, ***, *src_dir_fd=None*, *dst_dir_fd=None*, *follow_symlinks=True*)

Cria um link físico apontando para *src* chamado *dst*.

Esta função oferece suporte à especificação de *src_dir_fd* e/ou *dst_dir_fd* para fornecer caminhos relativos aos descritores de diretório e não seguir links simbólicos. O valor padrão de *follow_symlinks* é `False` no Windows.

Levanta um evento de auditoria `os.link` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Disponibilidade: Unix, Windows.

Alterado na versão 3.2: Adicionado suporte ao Windows.

Alterado na versão 3.3: Adicionados os parâmetros *src_dir_fd*, *dst_dir_fd* e *follow_symlinks*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *src* e *dst*.

os.listdir(*path='.'*)

Retorna uma lista contendo os nomes das entradas no diretório fornecido por *path*. A lista está em ordem arbitrária e não inclui as entradas especiais `'.'` e `'..'` mesmo se estiverem presentes no diretório. Se um arquivo for removido ou adicionado ao diretório durante a chamada desta função, não é especificado se um nome para esse arquivo deve ser incluído.

path pode ser um objeto caminho ou similar. Se *path* for do tipo `bytes` (direta ou indiretamente por meio da interface **`PathLike`**), os nomes de arquivo retornados também serão do tipo `bytes`; em todas as outras circunstâncias, eles serão do tipo `str`.

Esta função também pode ter suporte a especificar um descritor de arquivo; o descritor de arquivo deve fazer referência a um diretório.

Levanta um evento de auditoria `os.listdir` com o argumento `path`.

Nota

Para codificar nomes de arquivos `str` para `bytes`, use **`fsencode()`**.

Ver também

A função `scandir()` retorna entradas de diretório junto com informações de atributo de arquivo, dando melhor desempenho para muitos casos de uso comuns.

Alterado na versão 3.2: O parâmetro *path* tornou-se opcional.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.15: `os.listdir(-1)` now fails with `OSError(errno.EBADF)` rather than listing the current directory.

os.listdrives()

Retorna uma lista contendo os nomes das unidades em um sistema Windows.

Um nome de unidade geralmente se parece com `'C:\\'`. Nem todo nome de unidade estará associado a um volume e alguns podem estar inacessíveis por vários motivos, incluindo permissões, conectividade de rede ou mídia ausente. Esta função não testa o acesso.

Pode levantar `OSError` se ocorrer um erro ao coletar os nomes das unidades.

Levanta um evento de auditoria `os.listdrives` sem argumentos.

Disponibilidade: Windows

Adicionado na versão 3.12.

os.listmounts(*volume*)

Retorna uma lista contendo os pontos de montagem para um volume em um sistema Windows.

volume deve ser representado como um caminho GUID, como aqueles retornados por **`os.listvolumes()`**. Os volumes podem ser montados em vários locais ou não. No último caso, a lista estará vazia. Os pontos de montagem que não estão associados a um volume não serão retornados por esta função.

Os pontos de montagem retornados por esta função serão caminhos absolutos e podem ser mais longos que o nome da unidade.

Levanta **`OSError`** se o volume não for reconhecido ou se ocorrer um erro ao coletar os caminhos.

Levanta um evento de auditoria `os.listmounts` com o argumento `volume`.

Disponibilidade: Windows

Adicionado na versão 3.12.

os.listvolumes()

Retorna uma lista contendo os volumes no sistema.

Volumes são normalmente representados como um caminho GUID que se parece com `\\?\Volume{xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx}\`. Os arquivos geralmente podem ser acessados por meio de um caminho GUID, permitindo permissões. No entanto, os usuários geralmente não estão familiarizados com eles e, portanto, o uso recomendado desta função é recuperar pontos de montagem usando **`os.listmounts()`**.

Pode levantar **`OSError`** se ocorrer um erro ao coletar os volumes.

Levanta um evento de auditoria `os.listvolumes` sem argumentos.

Disponibilidade: Windows

Adicionado na versão 3.12.

os.lstat(*path*, *, *dir_fd=None*)

Executa o equivalente a uma chamada de sistema **`lstat()`** no caminho fornecido. Semelhante a **`stat()`**, mas não segue links simbólicos. Retorna um objeto **`stat_result`**.

Em plataformas que não têm suporte a links simbólicos, este é um apelido para **`stat()`**.

No Python 3.3, isso é equivalente a `os.stat(path, dir_fd=dir_fd, follow_symlinks=False)`.

Esta função também pode ter suporte a caminhos relativos a descritores de diretório.

Ver também

A função **`stat()`**.

Alterado na versão 3.2: Adicionado suporte para links simbólicos do Windows 6.0 (Vista).

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.8: No Windows, agora abre pontos de nova análise que representam outro caminho (substitutos de nome), incluindo links simbólicos e junções de diretório. Outros tipos de pontos de nova análise são resolvidos pelo sistema operacional como para **`stat()`**.

os.mkdir(*path*, *mode=0o777*, *, *dir_fd=None*)

Cria um diretório chamado *path* com o modo numérico *mode*.

Se o diretório já existe, uma exceção **`FileExistsError`** é levantada. Se um diretório pai no caminho não existir, uma exceção **`FileNotFoundError`** será levantada.

Em alguns sistemas, *mode* é ignorado. Onde ele é usado, o valor atual do umask é primeiro mascarado. Se bits diferentes dos últimos 9 (ou seja, os últimos 3 dígitos da representação octal do *mode*) são definidos, seu significado depende da plataforma. Em algumas plataformas, eles são ignorados e você deve chamar **`chmod()`** explicitamente para defini-los.

No Windows, um modo *mode* de `0o700` é tratado especificamente para aplicar controle de acesso ao novo diretório de forma que apenas o usuário atual e os administradores tenham acesso. Outros valores de *mode* são ignorados.

Esta função também pode ter suporte a caminhos relativos a descritores de diretório.

Também é possível criar diretórios temporários; veja a função **`tempfile.mkdtemp()`** do módulo **`tempfile`**.

Levanta um evento de auditoria `os.mkdir` com os argumentos `path`, `mode`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.13: O Windows agora lida com um *mode* de `0o700`.

os.makedirs(*name*, *mode=0o777*, *exist_ok=False*, *, *parent_mode=None*)

Função de criação recursiva de diretório. Como **`mkdir()`**, mas cria todos os diretórios de nível intermediário necessários para conter o diretório folha.

O parâmetro *mode* é passado para **`mkdir()`** para criar o diretório folha; veja a descrição do mkdir() para como é interpretado. Para definir os bits de permissão de arquivo de qualquer diretório pai recém-criado, você pode definir o umask antes de invocar **`makedirs()`**. Os bits de permissão de arquivo dos diretórios pais existentes não são alterados.

Se *exist_ok* for `False` (o padrão), uma **`FileExistsError`** é levantada se o diretório alvo já existir.

If *parent_mode* is not `None`, it is used as the mode for any newly-created, intermediate-level directories. Like *mode*, it is combined with the process's umask value; see the mkdir() description. Otherwise, intermediate directories are created with the default mode, which is also subject to the umask.

Nota

`makedirs()` ficará confuso se os elementos do caminho a serem criados incluírem **`pardir`** (por exemplo, ".." em sistemas UNIX).

Esta função trata os caminhos UNC corretamente.

Levanta um evento de auditoria `os.mkdir` com os argumentos `path`, `mode`, `dir_fd`.

Alterado na versão 3.2: Adicionado o parâmetro *exist_ok*.

Alterado na versão 3.4.1: Antes do Python 3.4.1, se *exist_ok* fosse `True` e o diretório existisse, **`makedirs()`** ainda levantaria um erro se *mode* não correspondesse ao modo do diretório existente. Como esse comportamento era impossível de implementar com segurança, ele foi removido no Python 3.4.1. Consulte bpo-21082 [https://bugs.python.org/issue?@action=redirect&bpo=21082].

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.7: O argumento *mode* não afeta mais os bits de permissão de arquivo de diretórios de nível intermediário recém-criados.

Adicionado na versão 3.15: The *parent_mode* parameter. To match the behavior from Python 3.6 and earlier (where *mode* was applied to all created directories), pass `parent_mode=mode`.

os.mkfifo(*path*, *mode=0o666*, *, *dir_fd=None*)

Cria um FIFO (um encadeamento nomeado) chamado *path* com o modo numérico *mode*. O valor atual de umask é primeiro mascarado do modo.

Esta função também pode ter suporte a caminhos relativos a descritores de diretório.

FIFOs são canais que podem ser acessados como arquivos regulares. FIFOs existem até que sejam excluídos (por exemplo, com `os.unlink()`). Geralmente, os FIFOs são usados como ponto de encontro entre os processos do tipo “cliente” e “servidor”: o servidor abre o FIFO para leitura e o cliente para escrita. Observe que `mkfifo()` não abre o FIFO – apenas cria o ponto de encontro.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.mknod(*path*, *mode=0o600*, *device=0*, *, *dir_fd=None*)

Create a filesystem node (file, device special file or named pipe) named *path*. *mode* specifies both the permissions to use and the type of node to be created, being combined (bitwise OR) with one of `stat.S_IFREG`, `stat.S_IFCHR`, `stat.S_IFBLK`, and `stat.S_IFIFO`. For `stat.S_IFCHR` and `stat.S_IFBLK`, *device* defines the newly created device special file (probably using `os.makedev()`), otherwise it is ignored.

Esta função também pode ter suporte a caminhos relativos a descritores de diretório.

Disponibilidade: Unix, not WASI.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.major(*device*, /)

Extract the device major number from a raw device number (usually the `st_dev` or `st_rdev` field from `stat`).

os.minor(*device*, /)

Extract the device minor number from a raw device number (usually the `st_dev` or `st_rdev` field from `stat`).

os.makedev(*major*, *minor*, /)

Compõe um número de dispositivo bruto a partir dos números de dispositivo principais e secundários.

os.NODEV

Non-existent device.

Adicionado na versão 3.15.

os.pathconf(*path*, *name*)

Retorna informações de configuração do sistema relevantes para um arquivo nomeado. *name* especifica o valor de configuração a ser recuperado; pode ser uma string que é o nome de um valor de sistema definido; esses nomes são especificados em vários padrões (POSIX.1, Unix 95, Unix 98 e outros). Algumas plataformas também definem nomes adicionais. Os nomes conhecidos do sistema operacional do host são fornecidos no dicionário `pathconf_names`. Para variáveis de configuração não incluídas nesse mapeamento, passar um número inteiro para *name* também é aceito.

Se *name* for uma string e não for conhecida, uma exceção `ValueError` é levantada. Se um valor específico para *name* não for compatível com o sistema hospedeiro, mesmo que seja incluído no `pathconf_names`, uma exceção `OSError` é levantada com `errno.EINVAL` como número do erro.

Esta função tem suporte a especificar um descritor de arquivo.

Disponibilidade: Unix.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.pathconf_names

Nomes de mapeamento de dicionário aceitos por `pathconf()` e `fpathconf()` para os valores inteiros definidos para esses nomes pelo sistema operacional do host. Isso pode ser usado para determinar o conjunto de nomes conhecidos pelo sistema.

Disponibilidade: Unix.

os.readlink(*path*, *, *dir_fd=None*)

Retorna uma string representando o caminho para o qual o link simbólico aponta. O resultado pode ser um nome de caminho absoluto ou relativo; se for relativo, pode ser convertido para um caminho absoluto usando `os.path.join(os.path.dirname(path), result)`.

Se o *path* for um objeto string (direta ou indiretamente por meio de uma interface `PathLike`), o resultado também será um objeto string e a chamada pode levantar um UnicodeDecodeError. Se o *path* for um objeto de bytes (direto ou indireto), o resultado será um objeto de bytes.

Esta função também pode ter suporte a caminhos relativos a descritores de diretório.

Ao tentar resolver um caminho que pode conter links, use `realpath()` para lidar corretamente com a recursão e as diferenças de plataforma.

Disponibilidade: Unix, Windows.

Alterado na versão 3.2: Adicionado suporte para links simbólicos do Windows 6.0 (Vista).

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar no Unix.

Alterado na versão 3.8: Aceita um objeto caminho ou similar e um objeto bytes no Windows.

Adicionado suporte para junções de diretório e alterado para retornar o caminho de substituição (que normalmente inclui o prefixo `\\?\`) ao invés do campo opcional “print name” que era retornado anteriormente.

os.remove(*path*, *, *dir_fd=None*)

Remove (exclui) o arquivo *path*. Se *path* for um diretório, uma **`OSError`** é levantada. Use **`rmdir()`** para remover diretórios. Se o arquivo não existir, uma **`FileNotFoundError`** é levantada.

Esta função tem suporte a caminhos relativos para descritores de diretório.

No Windows, a tentativa de remover um arquivo que está em uso causa o surgimento de uma exceção; no Unix, a entrada do diretório é removida, mas o armazenamento alocado para o arquivo não é disponibilizado até que o arquivo original não esteja mais em uso.

Esta função é semanticamente idêntica a **`unlink()`**.

Levanta um evento de auditoria `os.remove` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.removedirs(*name*)

Remove os diretórios recursivamente. Funciona como **`rmdir()`**, exceto que, se o diretório folha for removido com sucesso, **`removedirs()`** tenta remover sucessivamente todos os diretórios pai mencionados em *path* até que um erro seja levantado (que é ignorado, porque geralmente significa que um diretório pai não está vazio). Por exemplo, `os.removedirs('foo/bar/baz')` primeiro removerá o diretório `'foo/bar/baz'`, e então removerá `'foo/bar'` e `'foo'` se estiverem vazios. Levanta **`OSError`** se o diretório folha não pôde ser removido com sucesso.

Levanta um evento de auditoria `os.remove` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.rename(*src*, *dst*, ***, *src_dir_fd=None*, *dst_dir_fd=None*)

Renomeia o arquivo ou diretório *src* para *dst*. Se *dst* existir, a operação falhará com uma subclasse de **`OSError`** em vários casos:

No Windows, se *dst* existe então uma exceção **`FileExistsError`** será levantada. A operação pode falar se *src* e *dst* são de sistemas de arquivos diferentes. Use **`shutil.move()`** para mover arquivos entre sistemas de arquivos diferentes.

No Unix, se *src* é um arquivo e *dst* é um diretório ou vice-versa, uma **`IsADirectoryError`** ou uma **`NotADirectoryError`** será levantada respectivamente. Se ambos forem diretórios e *dst* estiver vazio, *dst* será substituído silenciosamente. Se *dst* for um diretório não vazio, uma **`OSError`** é levantada. Se ambos forem arquivos, *dst* será substituído silenciosamente se o usuário tiver permissão. A operação pode falhar em alguns tipos de Unix se *src* e *dst* estiverem em sistemas de arquivos diferentes. Se for bem-sucedido, a renomeação será uma operação atômica (este é um requisito POSIX).

Esta função permite especificar *src_dir_fd* e/ou *dst_dir_fd* para fornecer caminhos relativos a descritores de diretório.

Se você quiser sobrescrita multiplataforma do destino, use **`replace()`**.

Levanta um evento de auditoria `os.rename` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Alterado na versão 3.3: Adicionados os parâmetros *src_dir_fd* e *dst_dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *src* e *dst*.

os.renames(*old*, *new*)

Função de renomeação de arquivo ou diretório recursiva. Funciona como `rename()`, exceto que a criação de qualquer diretório intermediário necessário para tornar o novo nome de caminho possível é tentada primeiro. Após a renomeação, os diretórios correspondentes aos segmentos de caminho mais à direita do nome antigo serão removidos usando `removedirs()`.

Nota

Esta função pode falhar com a nova estrutura de diretório criada se você não tiver as permissões necessárias para remover o arquivo ou diretório folha.

Levanta um evento de auditoria `os.rename` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *old* e *new*.

os.replace(*src*, *dst*, ***, *src_dir_fd=None*, *dst_dir_fd=None*)

Renomeia o arquivo ou diretório *src* para *dst*. Se *dst* for um diretório não vazio, `OSError` será levantada. Se *dst* existir e for um arquivo, ele será substituído silenciosamente se o usuário tiver permissão. A operação pode falhar se *src* e *dst* estiverem em sistemas de arquivos diferentes. Se for bem-sucedido, a renomeação será uma operação atômica (este é um requisito POSIX).

Esta função permite especificar *src_dir_fd* e/ou *dst_dir_fd* para fornecer caminhos relativos a descritores de diretório.

Levanta um evento de auditoria `os.rename` com os argumentos `src`, `dst`, `src_dir_fd`, `dst_dir_fd`.

Adicionado na versão 3.3.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *src* e *dst*.

os.rmdir(*path*, *, *dir_fd=None*)

Remove (exclui) o diretório *path*. Se o diretório não existe ou não está vazio, uma `FileNotFoundError` ou uma `OSError` é levantada respectivamente. Para remover árvores de diretório inteiras, pode ser usada `shutil.rmtree()`.

Esta função tem suporte a caminhos relativos para descritores de diretório.

Levanta um evento de auditoria `os.rmdir` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.scandir(*path='.'*)

Retorna um iterador de objetos `os.DirEntry` correspondentes às entradas no diretório fornecido por *path*. As entradas são produzidas em ordem arbitrária, e as entradas especiais `'.'` e `'..'` não são incluídas. Se um arquivo for removido ou adicionado ao diretório após a criação do iterador, não é especificado se uma entrada para esse arquivo deve ser incluída.

Usar `scandir()` em vez de `listdir()` pode aumentar significativamente o desempenho do código que também precisa de tipo de arquivo ou informações de atributo de arquivo, porque os objetos `os.DirEntry` expõem essas informações se o sistema operacional fornecer ao percorrer um diretório. Todos os métodos de `os.DirEntry` podem realizar uma chamada de sistema, mas `is_dir()` e `is_file()` normalmente requerem apenas uma chamada de sistema para links simbólicos; `os.DirEntry.stat()` sempre requer uma chamada de sistema no Unix, mas requer uma apenas para links simbólicos no Windows.

path pode ser um objeto caminho ou similar. Se *path* for do tipo `bytes` (direta ou indiretamente por meio da interface **`PathLike`**), o tipo dos atributos **`name`** e **`path`** de cada **`os.DirEntry`** serão `bytes`; em todas as outras circunstâncias, eles serão do tipo `str`.

Esta função também pode ter suporte a especificar um descritor de arquivo; o descritor de arquivo deve fazer referência a um diretório.

Levanta um evento de auditoria `os.scandir` com o argumento `path`.

O iterador **`scandir()`** implementa o protocolo gerenciador de contexto e tem o seguinte método:

scandir.close()

Fecha o iterador e libera os recursos alocados.

This is called automatically when the iterator is exhausted or garbage collected, or when an error happens during iterating. However it is advisable to call it explicitly or use the **`with`** statement.

Adicionado na versão 3.6.

O exemplo a seguir mostra um uso simples de **`scandir()`** para exibir todos os arquivos (excluindo diretórios) no *path* fornecido que não começa com `'.'`. A chamada `entrada.is_file()` geralmente não fará uma chamada de sistema adicional:

```
with os.scandir(caminho) as coisa:
    for entrada in coisa:
        if not entrada.name.startswith('.') and entrada.is_fi
            print(entrada.name)
```

Nota

Em sistemas baseados no Unix, **`scandir()`** usa o sistema opendir() [https://pubs.opengroup.org/onlinepubs/009695399/functions/opendir.html] e as funções readdir() [https://pubs.opengroup.org/onlinepubs/009695399/functions/readdir_r.html].

No Windows, ela usa os funções FindFirstFileW [https://msdn.microsoft.com/en-us/library/windows/desktop/aa364418(v=vs.85).aspx] e FindNextFileW [https://msdn.microsoft.com/en-us/library/windows/desktop/aa364428(v=vs.85).aspx] do Win32.

Adicionado na versão 3.5.

Alterado na versão 3.6: Adicionado suporte para o protocolo de gerenciador de contexto e o método **`close()`**. Se um iterador **`scandir()`** não estiver esgotado nem explicitamente fechado, uma **`ResourceWarning`** será emitida em seu destrutor.

A função aceita um objeto caminho ou similar.

Alterado na versão 3.7: Adicionado suporte para descritores de arquivo no Unix.

Alterado na versão 3.15: `os.scandir(-1)` now fails with `OSError(errno.EBADF)` rather than listing the current directory.

class os.DirEntry

Objeto produzido por **`scandir()`** para expor o caminho do arquivo e outros atributos de arquivo de uma entrada de diretório.

`scandir()` fornecerá o máximo possível dessas informações sem fazer chamadas de sistema adicionais. Quando uma chamada de sistema `stat()` ou `lstat()` é feita, o objeto `os.DirEntry` irá armazenar o resultado em cache.

As instâncias de `os.DirEntry` não devem ser armazenadas em estruturas de dados de longa duração; se você sabe que os metadados do arquivo foram alterados ou se passou muito tempo desde a chamada de **`scandir()`**, chame `os.stat(entry.path)` para obter informações atualizadas.

Como os métodos `os.DirEntry` podem fazer chamadas ao sistema operacional, eles também podem levantar **`OSError`**. Se você precisa de

um controle muito refinado sobre os erros, você pode pegar **`OSError`** ao chamar um dos métodos `os.DirEntry` e manipular conforme apropriado.

Para ser diretamente utilizável como um objeto caminho ou similar, `os.DirEntry` implementa a interface **`PathLike`**.

`DirEntry` objects are generic over the type of the path (**`str`** or **`bytes`**).

Atributos e métodos em uma instância de `os.DirEntry` são os seguintes:

name

O nome do arquivo base da entrada, relativo ao argumento *path* de **`scandir()`**.

O atributo **`name`** será `bytes` se o argumento *path* de **`scandir()`** for do tipo `bytes` e, caso contrário, `str`. Usa **`fsdecode()`** para decodificar nomes de arquivos de bytes.

path

The entry's path name: equivalent to `os.path.join(scandir_path, entry.name)` where *scandir_path* is the original **`scandir()`** *path* argument. Apart from the filename, the path preserves the original **`scandir()`** argument. If the **`scandir()`** *path* argument was relative, the **`path`** attribute is also relative. Changing the current working directory after creating the **`scandir()`** iterator may cause later uses of **`path`** to resolve differently. On some platforms, the constructed path may not be valid if the original **`scandir()`** argument was usable for enumeration but not for joining with the entry name. If the **`scandir()`** *path* argument was a file descriptor, the **`path`** attribute is the same as the **`name`** attribute.

O atributo **`path`** será `bytes` se o argumento *path* de **`scandir()`** for do tipo `bytes` e, caso contrário, `str`. Usa **`fsdecode()`** para decodificar nomes de arquivos de bytes.

inode()

Retorna o número de nó-i da entrada.

O resultado é armazenado em cache no objeto `os.DirEntry`. Use `os.stat(entry.path, follow_symlinks=False).st_ino` para obter informações atualizadas.

Na primeira chamada sem cache, uma chamada de sistema é necessária no Windows, mas não no Unix.

is_dir(*, *follow_symlinks=True*)

Retorna `True` se esta entrada for um diretório ou um link simbólico apontando para um diretório; retorna `False` se a entrada é ou aponta para qualquer outro tipo de arquivo, ou se ele não existe mais.

Se *follow_symlinks* for `False`, retorna `True` apenas se esta entrada for um diretório (sem seguir os links simbólicos); retorna `False` se a entrada for qualquer outro tipo de arquivo ou se ele não existe mais.

O resultado é armazenado em cache no objeto `os.DirEntry`, com um cache separado para *follow_symlinks* `True` e `False`. Chama **`os.stat()`** junto com **`stat.S_ISDIR()`** para buscar informações atualizadas.

Na primeira chamada sem cache, nenhuma chamada do sistema é necessária na maioria dos casos. Especificamente, para links não simbólicos, nem o Windows nem o Unix requerem uma chamada de sistema, exceto em certos sistemas de arquivos Unix, como sistemas de arquivos de rede, que retornam `dirent.d_type == DT_UNKNOWN`. Se a entrada for um link simbólico, uma chamada de sistema será necessária para seguir o link simbólico, a menos que *follow_symlinks* seja `False`.

Este método pode levantar **`OSError`**, tal como **`PermissionError`**, mas **`FileNotFoundError`** é capturada e não levantada.

is_file(*, *follow_symlinks=True*)

Retorna `True` se esta entrada for um arquivo ou um link simbólico apontando para um arquivo; retorna `False` se a entrada é ou aponta

para um diretório ou outra entrada que não seja de arquivo, ou se ela não existe mais.

Se *follow_symlinks* for `False`, retorna `True` apenas se esta entrada for um arquivo (sem seguir os links simbólicos); retorna `False` se a entrada for um diretório ou outra entrada que não seja um arquivo, ou se ela não existir mais.

O resultado é armazenado em cache no objeto `os.DirEntry`. Chamadas de sistema em cache feitas e exceções levantadas são conforme **`is_dir()`**.

is_symlink()

Retorna `True` se esta entrada for um link simbólico (mesmo se quebrado); retorna `False` se a entrada apontar para um diretório ou qualquer tipo de arquivo, ou se ele não existir mais.

O resultado é armazenado em cache no objeto `os.DirEntry`. Chama **`os.path.islink()`** para buscar informações atualizadas.

Na primeira chamada sem cache, nenhuma chamada do sistema é necessária na maioria dos casos. Especificamente, nem o Windows nem o Unix exigem uma chamada de sistema, exceto em certos sistemas de arquivos Unix, como sistemas de arquivos de rede, que retornam `dirent.d_type == DT_UNKNOWN`.

Este método pode levantar **`OSError`**, tal como **`PermissionError`**, mas **`FileNotFoundError`** é capturada e não levantada.

is_junction()

Retorna `True` se esta entrada for uma junção (mesmo se quebrado); retorna `False` se a entrada apontar para um diretório regular, ou qualquer tipo de arquivo, um link simbólico ou se ele não existir mais.

O resultado é armazenado em cache no objeto `os.DirEntry`. Chama **`os.path.isjunction()`** para buscar informações atualizadas.

Adicionado na versão 3.12.

stat(*, *follow_symlinks=True*)

Retorna um objeto **stat_result** para esta entrada. Este método segue links simbólicos por padrão; para estabelecer um link simbólico, adicione o argumento `follow_symlinks=False`.

No Unix, esse método sempre requer uma chamada de sistema. No Windows, ele só requer uma chamada de sistema se *follow_symlinks* for `True` e a entrada for um ponto de nova análise (por exemplo, um link simbólico ou junção de diretório).

No Windows, os atributos `st_ino`, `st_dev` e `st_nlink` da **stat_result** são sempre definidos como zero. Chame **os.stat()** para obter esses atributos.

O resultado é armazenado em cache no objeto `os.DirEntry`, com um cache separado para *follow_symlinks* `True` e `False`. Chame **os.stat()** para buscar informações atualizadas.

Note que há uma boa correspondência entre vários atributos e métodos de `os.DirEntry` e de **pathlib.Path**. Em particular, o atributo `name` tem o mesmo significado, assim como os métodos `is_dir()`, `is_file()`, `is_symlink()`, `is_junction()` e `stat()`.

Adicionado na versão 3.5.

Alterado na versão 3.6: Adicionado suporte para a interface **PathLike**. Adicionado suporte para caminhos **bytes** no Windows.

Alterado na versão 3.12: O atributo `st_ctime` do resultado de stat foi descontinuado no Windows. A hora de criação do arquivo está devidamente disponível como `st_birthtime`, e no futuro `st_ctime` pode ser alterada para retornar zero ou a hora de alteração dos metadados, se disponível.

os.stat(*path*, *, *dir_fd=None*, *follow_symlinks=True*)

Obtém o status de um arquivo ou um descritor de arquivo. Executa o equivalente a uma chamada de sistema **`stat()`** no caminho fornecido. *path* pode ser especificado como uma string ou bytes – direta ou indiretamente através da interface **`PathLike`** – ou como um descritor de arquivo aberto. Retorna um objeto **`stat_result`**.

Esta função normalmente segue links simbólicos; para obter o status do link simbólico, adicione o argumento `follow_symlinks=False`, ou use **`lstat()`**.

Esta função tem suporte a especificar um descritor de arquivo e não seguir links simbólicos.

No Windows, ao definir `follow_symlinks=False` é desativado o recurso de seguir todos os pontos de nova análise, que incluem links simbólicos e junções de diretório. Outros tipos de pontos de nova análise que não se parecem com links ou que o sistema operacional não pode seguir serão abertos diretamente. Ao seguir uma cadeia de vários links, isso pode resultar no retorno do link original em vez do não-link que impediu o percurso completo. Para obter resultados de *stat* para o caminho final neste caso, use a função **`os.path.realpath()`** para resolver o nome do caminho tanto quanto possível e chame **`lstat()`** no resultado. Isso não se aplica a links simbólicos pendentes ou pontos de junção, que levantam as exceções usuais.

Exemplo:

```
>>> import os
>>> statinfo = os.stat('algumarquivo.txt')
>>> statinfo
os.stat_result(st_mode=33188, st_ino=7876932, st_dev=23488102
st_nlink=1, st_uid=501, st_gid=501, st_size=264, st_atime=129
st_mtime=1297230027, st_ctime=1297230027)
>>> statinfo.st_size
264
```

Ver também

As funções **`fstat()`** e **`lstat()`**.

Alterado na versão 3.3: Adicionados os parâmetros *dir_fd* e *follow_symlinks*, especificando um descritor de arquivo em vez de um caminho.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.8: No Windows, todos os pontos de nova análise que podem ser resolvidos pelo sistema operacional agora são seguidos, e passar `follow_symlinks=False` desativa seguir todos os pontos de nova análise substitutos de nome. Se o sistema operacional atingir um ponto de nova análise que não é capaz de seguir, *stat* agora retorna as informações do caminho original como se `follow_symlinks=False` tivesse sido especificado em vez de levantar um erro.

class os.stat_result

Objeto cujos atributos correspondem aproximadamente aos membros da estrutura **`stat`**. É usado para o resultado de **`os.stat()`**, **`os.fstat()`** e **`os.lstat()`**.

Atributos:

st_mode

Modo de arquivo: tipo de arquivo e bits de modo de arquivo (permissões).

st_ino

Dependente da plataforma, mas se diferente de zero, identifica exclusivamente o arquivo para um determinado valor de `st_dev`. Tipicamente:

- o número do nó-i no Unix,
- o índice de arquivo [https://msdn.microsoft.com/en-us/library/aa363788] no Windows

st_dev

Identificador do dispositivo no qual este arquivo reside.

st_nlink
: Número de links físicos.

st_uid
: Identificador de usuário do proprietário do arquivo.

st_gid
: Identificador de grupo do proprietário do arquivo.

st_size
: Tamanho do arquivo em bytes, se for um arquivo normal ou um link simbólico. O tamanho de um link simbólico é o comprimento do nome do caminho que ele contém, sem um byte nulo final.

Registros de data e hora:

st_atime
: Hora do acesso mais recente expressa em segundos.

st_mtime
: Hora da modificação de conteúdo mais recente expressa em segundos.

st_ctime
: Hora da alteração de metadados mais recente expressa em segundos.

  Alterado na versão 3.12: `st_ctime` foi descontinuado no Windows. Use `st_birthtime` para o tempo de criação do arquivo. No futuro, `st_ctime` conterá a hora da alteração de metadados mais recente, como em outras plataformas.

st_atime_ns
: Hora do acesso mais recente expresso em nanossegundos como um número inteiro.

  Adicionado na versão 3.3.

st_mtime_ns

Hora da modificação de conteúdo mais recente expressa em nanossegundos como um número inteiro.

Adicionado na versão 3.3.

st_ctime_ns

Hora do alteração de metadados mais recente expresso em nanossegundos como um número inteiro.

Adicionado na versão 3.3.

Alterado na versão 3.12: `st_ctime_ns` foi descontinuado no Windows. Use `st_birthtime_ns` para o tempo de criação do arquivo. No futuro, `st_ctime` conterá a hora da alteração de metadados mais recente, como em outras plataformas.

st_birthtime

Hora de criação do arquivo expressa em segundos. Este atributo nem sempre está disponível e pode levantar **`AttributeError`**.

Alterado na versão 3.12: `st_birthtime` está agora disponível no Windows.

st_birthtime_ns

Hora de criação do arquivo expressa em nanossegundos em um inteiro. Este atributo nem sempre está disponível e pode levantar **`AttributeError`**.

Adicionado na versão 3.12.

Nota

O significado e resolução exatos dos atributos **`st_atime`**, **`st_mtime`**, **`st_ctime`** e **`st_birthtime`** dependem do sistema operacional e do sistema de arquivos. Por exemplo, em sistemas Windows que usam os sistemas de arquivos FAT32, **`st_mtime`** tem resolução de 2 segundos, e

st_atime tem resolução de apenas 1 dia. Consulte a documentação do sistema operacional para obter detalhes.

Da mesma forma, embora st_atime_ns, st_mtime_ns, st_ctime_ns e st_birthtime_ns sejam sempre expressos em nanossegundos, muitos sistemas não fornecem precisão de nanossegundos. Em sistemas que fornecem precisão de nanossegundos, o objeto de ponto flutuante usado para armazenar st_atime, st_mtime, st_ctime e st_birthtime não pode preservar tudo, e como tal será ligeiramente inexato. Se você precisa dos carimbos de data/hora exatos, sempre deve usar **st_atime_ns**, **st_mtime_ns**, **st_ctime_ns** e **st_birthtime_ns**.

Em alguns sistemas Unix (como Linux), os seguintes atributos também podem estar disponíveis:

st_blocks
: Número de blocos de 512 bytes alocados para o arquivo. Isso pode ser menor que st_size/512 quando o arquivo possuir lacunas.

st_blksize
: Tamanho de bloco “preferido” para E/S eficiente do sistema de arquivos. Gravar em um arquivo em partes menores pode causar uma leitura-modificação-reescrita ineficiente.

st_rdev
: Tipo de dispositivo, se for um dispositivo nó-i.

st_flags
: Sinalizadores definidos pelo usuário para o arquivo.

Em outros sistemas Unix (como o FreeBSD), os seguintes atributos podem estar disponíveis (mas só podem ser preenchidos se o root tentar usá-los):

st_gen
: Número de geração do arquivo.

No Solaris e derivados, os seguintes atributos também podem estar disponíveis:

st_fstype
: String que identifica exclusivamente o tipo de sistema de arquivos que contém o arquivo.

Em sistemas macOS, os seguintes atributos também podem estar disponíveis:

st_rsize
: Tamanho real do arquivo.

st_creator
: Criador do arquivo.

st_type
: Tipo de arquivo.

Em sistemas Windows, os seguintes atributos também estão disponíveis:

st_file_attributes
: Atributos de arquivos no Windows: membro `dwFileAttributes` da estrutura `BY_HANDLE_FILE_INFORMATION` retornada por **`GetFileInformationByHandle()`**. Veja as constantes **`FILE_ATTRIBUTE_* <stat.FILE_ATTRIBUTE_ARCHIVE>`** no módulo **`stat`**.

  Adicionado na versão 3.5.

st_reparse_tag
: Quando **`st_file_attributes`** tem **`FILE_ATTRIBUTE_REPARSE_POINT`** definido, este campo contém uma tag identificando o tipo do ponto de nova análise. Veja as constantes **`IO_REPARSE_TAG_*`** no módulo **`stat`**.

O módulo padrão `stat` define funções e constantes que são úteis para extrair informações de uma estrutura `stat`. (No Windows, alguns itens são preenchidos com valores fictícios.)

Para compatibilidade com versões anteriores, uma instância de `stat_result` também é acessível como uma tupla de pelo menos 10 inteiros, fornecendo os membros mais importantes (e portáveis) da estrutura `stat`, na ordem `st_mode`, `st_ino`, `st_dev`, `st_nlink`, `st_uid`, `st_gid`, `st_size`, `st_atime`, `st_mtime`, `st_ctime`. Mais itens podem ser adicionados no final por algumas implementações. Para compatibilidade com versões mais antigas do Python, acessar `stat_result` como uma tupla sempre retorna inteiros.

Alterado na versão 3.5: Windows agora retorna o arquivo de índice como `st_ino` quando disponível.

Alterado na versão 3.7: Adicionado o membro `st_fstype` ao Solaris/derivados.

Alterado na versão 3.8: Adicionado o membro `st_reparse_tag` no Windows.

Alterado na versão 3.8: On Windows, the `st_mode` member now identifies special files as `S_IFCHR`, `S_IFIFO` or `S_IFBLK` as appropriate.

Alterado na versão 3.12: No Windows, `st_ctime` foi descontinuado. Eventualmente, ele conterá a hora da última alteração de metadados, para fins de consistência com outras plataformas, mas por enquanto ainda contém a hora de criação. Use `st_birthtime` para a hora de criação.

No Windows, o `st_ino` agora pode ter até 128 bits, dependendo do sistema de arquivos. Anteriormente, não era possível ter mais de 64 bits, e identificadores de arquivo maiores seriam compactados arbitrariamente.

No Windows, `st_rdev` não retorna mais um valor. Anteriormente, ele continha o mesmo que `st_dev`, o que era incorreto.

Adicionado o membro `st_birthtime` no Windows.

os.statx(*path*, *mask*, ***, *flags=0*, *dir_fd=None*, *follow_symlinks=True*)

Get the status of a file or file descriptor by performing a **`statx()`** system call on the given path.

path is a path-like object or an open file descriptor. *mask* is a combination of the module-level `STATX_*` constants specifying the information to retrieve. *flags* is a combination of the module-level `AT_STATX_*` constants and/or `AT_NO_AUTOMOUNT`. Returns a `statx_result` object whose `stx_mask` attribute specifies the information actually retrieved (which may differ from *mask*).

This function supports specifying a file descriptor, paths relative to directory descriptors, and not following symlinks.

Ver também

The *statx(2)* *[https://manpages.debian.org/statx(2)]* man page.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28.

Adicionado na versão 3.15.

class os.statx_result

Information about a file returned by `os.statx()`.

`statx_result` has the following attributes:

stx_atime

Hora do acesso mais recente expressa em segundos.

Equal to `None` if `STATX_ATIME` is missing from `stx_mask`.

stx_atime_ns

Hora do acesso mais recente expresso em nanossegundos como um número inteiro.

Equal to `None` if `STATX_ATIME` is missing from `stx_mask`.

stx_atomic_write_segments_max

Maximum iovecs for direct I/O with torn-write protection.

Equal to `None` if `STATX_WRITE_ATOMIC` is missing from `stx_mask`.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.11.

stx_atomic_write_unit_max

Maximum size for direct I/O with torn-write protection.

Equal to `None` if `STATX_WRITE_ATOMIC` is missing from `stx_mask`.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.11.

stx_atomic_write_unit_max_opt

Maximum optimized size for direct I/O with torn-write protection.

Equal to `None` if `STATX_WRITE_ATOMIC` is missing from `stx_mask`.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.16.

stx_atomic_write_unit_min

Minimum size for direct I/O with torn-write protection.

Equal to `None` if `STATX_WRITE_ATOMIC` is missing from `stx_mask`.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.11.

stx_attributes

Bitmask of `STATX_ATTR_*` constants specifying the attributes of this file.

stx_attributes_mask

A mask indicating which bits in `stx_attributes` are supported by the VFS and the filesystem.

stx_blksize

Tamanho de bloco “preferido” para E/S eficiente do sistema de arquivos. Gravar em um arquivo em partes menores pode causar uma leitura-modificação-reescrita ineficiente.

stx_blocks

Number of 512-byte blocks allocated for file. This may be smaller than `stx_size`/512 when the file has holes.

Equal to `None` if `STATX_BLOCKS` is missing from `stx_mask`.

stx_btime

Time of file creation expressed in seconds.

Equal to `None` if `STATX_BTIME` is missing from `stx_mask`.

stx_btime_ns

Time of file creation expressed in nanoseconds as an integer.

Equal to `None` if `STATX_BTIME` is missing from `stx_mask`.

stx_ctime

Hora da alteração de metadados mais recente expressa em segundos.

Equal to `None` if `STATX_CTIME` is missing from `stx_mask`.

stx_ctime_ns

Hora do alteração de metadados mais recente expresso em nanossegundos como um número inteiro.

Equal to `None` if `STATX_CTIME` is missing from `stx_mask`.

stx_dev

Identificador do dispositivo no qual este arquivo reside.

stx_dev_major

Major number of the device on which this file resides.

stx_dev_minor

Minor number of the device on which this file resides.

stx_dio_mem_align

Direct I/O memory buffer alignment requirement.

Equal to `None` if **`STATX_DIOALIGN`** is missing from **`stx_mask`**.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.1.

stx_dio_offset_align

Direct I/O file offset alignment requirement.

Equal to `None` if **`STATX_DIOALIGN`** is missing from **`stx_mask`**.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.1.

stx_dio_read_offset_align

Direct I/O file offset alignment requirement for reads.

Equal to `None` if **`STATX_DIO_READ_ALIGN`** is missing from **`stx_mask`**.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.14.

stx_gid

Identificador de grupo do proprietário do arquivo.

Equal to `None` if **`STATX_GID`** is missing from **`stx_mask`**.

stx_ino

Número de nó-i.

Equal to `None` if `STATX_INO` is missing from `stx_mask`.

stx_mask

Bitmask of `STATX_*` constants specifying the information retrieved, which may differ from what was requested.

stx_mnt_id

Mount identifier.

Equal to `None` if `STATX_MNT_ID` is missing from `stx_mask`.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 5.8.

stx_mode

Modo de arquivo: tipo de arquivo e bits de modo de arquivo (permissões).

Equal to `None` if `STATX_TYPE | STATX_MODE` is missing from `stx_mask`.

stx_mtime

Hora da modificação de conteúdo mais recente expressa em segundos.

Equal to `None` if `STATX_MTIME` is missing from `stx_mask`.

stx_mtime_ns

Hora da modificação de conteúdo mais recente expressa em nanossegundos como um número inteiro.

Equal to `None` if `STATX_MTIME` is missing from `stx_mask`.

stx_nlink

Número de links físicos.

Equal to `None` if `STATX_NLINK` is missing from `stx_mask`.

stx_rdev
: Tipo de dispositivo, se for um dispositivo nó-i.

stx_rdev_major
: Major number of the device this file represents.

stx_rdev_minor
: Minor number of the device this file represents.

stx_size
: Tamanho do arquivo em bytes, se for um arquivo normal ou um link simbólico. O tamanho de um link simbólico é o comprimento do nome do caminho que ele contém, sem um byte nulo final.

  Equal to `None` if **`STATX_SIZE`** is missing from **`stx_mask`**.

stx_subvol
: Subvolume identifier.

  Equal to `None` if **`STATX_SUBVOL`** is missing from **`stx_mask`**.

  Disponibilidade: Linux >= 4.11 with glibc >= 2.28 and build-time kernel userspace API headers >= 6.10.

stx_uid
: Identificador de usuário do proprietário do arquivo.

  Equal to `None` if **`STATX_UID`** is missing from **`stx_mask`**.

Ver também

The *statx(2)* *[https://manpages.debian.org/statx(2)]* man page.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28.

Adicionado na versão 3.15.

os.STATX_TYPE

os.STATX_MODE
os.STATX_NLINK
os.STATX_UID
os.STATX_GID
os.STATX_ATIME
os.STATX_MTIME
os.STATX_CTIME
os.STATX_INO
os.STATX_SIZE
os.STATX_BLOCKS
os.STATX_BASIC_STATS
os.STATX_BTIME
os.STATX_MNT_ID
os.STATX_DIOALIGN
os.STATX_MNT_ID_UNIQUE
os.STATX_SUBVOL
os.STATX_WRITE_ATOMIC
os.STATX_DIO_READ_ALIGN

Bitflags for use in the *mask* parameter to `os.statx()`. Some of these flags may be available even when their corresponding members in `statx_result` are not available.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28.

Adicionado na versão 3.15.

os.AT_STATX_FORCE_SYNC

A flag for the `os.statx()` function. Requests that the kernel return up-to-date information even when doing so is expensive (for example, requiring a round trip to the server for a file on a network filesystem).

Disponibilidade: Linux >= 4.11 with glibc >= 2.28.

Adicionado na versão 3.15.

os.AT_STATX_DONT_SYNC

A flag for the `os.statx()` function. Requests that the kernel return cached information if possible.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28.

Adicionado na versão 3.15.

os.AT_STATX_SYNC_AS_STAT

A flag for the `os.statx()` function. This flag is defined as `0`, so it has no effect, but it can be used to explicitly indicate neither `AT_STATX_FORCE_SYNC` nor `AT_STATX_DONT_SYNC` is being passed. In the absence of the other two flags, the kernel will generally return information as fresh as `os.stat()` would return.

Disponibilidade: Linux >= 4.11 with glibc >= 2.28.

Adicionado na versão 3.15.

os.AT_NO_AUTOMOUNT

If the final component of a path is an automount point, operate on the automount point instead of performing the automount. On Linux, `os.stat()`, `os.fstat()` and `os.lstat()` always behave this way.

Disponibilidade: Linux.

Adicionado na versão 3.15.

os.statvfs(*path*)

Perform a *statvfs(3)* *[https://manpages.debian.org/statvfs(3)]* system call on the given path. The return value is a `statvfs_result` whose attributes describe the filesystem on the given path and correspond to the members of the `statvfs` structure.

Esta função tem suporte a especificar um descritor de arquivo.

Disponibilidade: Unix.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

class os.statvfs_result

Filesystem statistics returned by `os.statvfs()` and `os.fstatvfs()`. See *statvfs(3)* *[https://manpages.debian.org/statvfs(3)]* for more details.

f_bsize

Block size.

f_frsize

Fragment size.

f_blocks

Number of `f_frsize` sized blocks the filesystem can contain.

f_bfree

Number of free blocks.

f_bavail

Number of free blocks for unprivileged users.

f_files

Number of file entries, inodes, the filesystem can contain.

f_ffree

Number of free files entries.

f_favail

Number of free file entries for unprivileged users.

f_flag

Bit-mask of mount flags. The following flags are defined: `ST_RDONLY`, `ST_NOSUID`, `ST_NODEV`, `ST_NOEXEC`, `ST_SYNCHRONOUS`,

`ST_MANDLOCK`, `ST_WRITE`, `ST_APPEND`, `ST_IMMUTABLE`, `ST_NOATIME`, `ST_NODIRATIME`, and `ST_RELATIME`.

f_namemax
: Filesystem max filename length. OS specific limitations such as Windows MAX_PATH and those described in Linux *pathname(7)* *[https://manpages.debian.org/pathname(7)]* may exist.

f_fsid
: Filesystem ID.

Adicionado na versão 3.7.

The following flags are used in `statvfs_result.f_flag`.

os.ST_RDONLY
: Read-only filesystem.

Adicionado na versão 3.2.

os.ST_NOSUID
: Setuid/setgid bits are disabled or not supported.

Adicionado na versão 3.2.

os.ST_NODEV
: Disallow access to device special files.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_NOEXEC
: Disallow program execution.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_SYNCHRONOUS

Writes are synced at once.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_MANDLOCK

Allow mandatory locks on an FS.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_WRITE

Write on file/directory/symlink.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_APPEND

Append-only file.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_IMMUTABLE

Immutable file.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_NOATIME

Do not update access times.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_NODIRATIME

Do not update directory access times.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.ST_RELATIME

Update atime relative to mtime/ctime.

Disponibilidade: Linux.

Adicionado na versão 3.4.

os.supports_dir_fd

A **`set`** object indicating which functions in the **`os`** module accept an open file descriptor for their *dir_fd* parameter. Different platforms provide different features, and the underlying functionality Python uses to implement the *dir_fd* parameter is not available on all platforms Python supports. For consistency's sake, functions that may support *dir_fd* always allow specifying the parameter, but will throw an exception if the functionality is used when it's not locally available. (Specifying `None` for *dir_fd* is always supported on all platforms.)

Para verificar se uma função particular aceita um descritor de arquivo aberto para seu parâmetro *dir_fd*, use o operador `in` em `supports_dir_fd`. Como exemplo, esta expressão é avaliada como `True` se **`os.stat()`** aceita descritores de arquivo abertos para *dir_fd* na plataforma local:

```
os.stat in os.supports_dir_fd
```

Atualmente os parâmetros *dir_fd* funcionam apenas em plataformas Unix; nenhum deles funciona no Windows.

Adicionado na versão 3.3.

os.supports_effective_ids

Um objeto **set** que indica se **os.access()** permite especificar `True` para seu parâmetro *effective_ids* na plataforma local. (Especificar `False` para *effective_ids* é sempre válido em todas as plataformas.) Se a plataforma local oferecer suporte, a coleção conterá **os.access()**; caso contrário, ficará vazio.

Esta expressão é avaliada como `True` se **os.access()** oferecer suporte a `effective_ids=True` na plataforma local:

```
os.access in os.supports_effective_ids
```

Atualmente, *effective_ids* funciona apenas em plataformas Unix; não funciona no Windows.

Adicionado na versão 3.3.

os.supports_fd

A **set** object indicating which functions in the **os** module permit specifying their *path* parameter as an open file descriptor on the local platform. Different platforms provide different features, and the underlying functionality Python uses to accept open file descriptors as *path* arguments is not available on all platforms Python supports.

Para determinar se uma função particular permite especificar um descritor de arquivo aberto para seu parâmetro *path*, use o operador `in` em `supports_fd`. Como exemplo, esta expressão é avaliada como `True` se **os.chdir()** aceita descritores de arquivo abertos para *path* em sua plataforma local:

```
os.chdir in os.supports_fd
```

Adicionado na versão 3.3.

os.supports_follow_symlinks

A **set** object indicating which functions in the **os** module accept `False` for their *follow_symlinks* parameter on the local platform. Different

platforms provide different features, and the underlying functionality Python uses to implement *follow_symlinks* is not available on all platforms Python supports. For consistency's sake, functions that may support *follow_symlinks* always allow specifying the parameter, but will throw an exception if the functionality is used when it's not locally available. (Specifying `True` for *follow_symlinks* is always supported on all platforms.)

Para verificar se uma função em particular aceita `False` para seu parâmetro *follow_symlinks*, use o operador `in` em `supports_follow_symlinks`. Como exemplo, esta expressão é avaliada como `True` se você pode especificar `follow_symlinks=False` ao chamar **`os.stat()`** na plataforma local:

```
os.stat in os.supports_follow_symlinks
```

Adicionado na versão 3.3.

os.symlink(*src*, *dst*, *target_is_directory=False*, ***, *dir_fd=None*)

Cria um link simbólico apontando para *src* chamado *dst*.

O parâmetro *src* refere-se ao destino do link (o arquivo ou diretório para o qual o link está sendo criado), e *dst* é o nome do link que está sendo criado.

No Windows, um link simbólico representa um arquivo ou um diretório e não se transforma no destino dinamicamente. Se o alvo estiver presente, será criado um link simbólico de mesmo tipo. Caso contrário, o link simbólico será criado como um diretório se *target_is_directory* for `True` ou um link simbólico de arquivo (o padrão) caso contrário. Em plataformas não Windows, *target_is_directory* é ignorado.

Esta função tem suporte a caminhos relativos para descritores de diretório.

Nota

Em versões mais recentes do Windows 10, contas sem privilégios podem criar links simbólicos se o Modo de Desenvolvedor estiver

habilitado. Quando o modo de desenvolvedor não está disponível/ativado, o privilégio *SeCreateSymbolicLinkPrivilege* é necessário ou o processo deve ser executado como um administrador.

A exceção **`OSError`** é levantada quando a função é chamada por um usuário sem privilégios.

Levanta um evento de auditoria `os.symlink` com os argumentos `src`, `dst` e `dir_fd`.

Disponibilidade: Unix, Windows.

A função é limitada em WASI, veja Plataformas WebAssembly para mais informações.

Alterado na versão 3.2: Adicionado suporte para links simbólicos do Windows 6.0 (Vista).

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*, e agora permite *target_is_directory* em plataformas não Windows.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *src* e *dst*.

Alterado na versão 3.8: Adicionado suporte para links simbólicos não elevados no Windows com Modo de Desenvolvedor.

os.sync()

Força a escrita de tudo para o disco.

Disponibilidade: Unix.

Adicionado na versão 3.3.

os.truncate(*path*, *length*)

Trunca o arquivo correspondente ao *path*, de modo que tenha no máximo *length* bytes.

Esta função tem suporte a especificar um descritor de arquivo.

Levanta um evento de auditoria `os.truncate` com os argumentos `path` e `length`.

Disponibilidade: Unix, Windows.

Adicionado na versão 3.3.

Alterado na versão 3.5: Adicionado suporte para o Windows.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.unlink(*path*, *, *dir_fd=None*)

Remove (exclui) o arquivo *path*. Esta função é semanticamente idêntica à **`remove()`**; o nome `unlink` é seu nome Unix tradicional. Por favor, veja a documentação de **`remove()`** para mais informações.

Levanta um evento de auditoria `os.remove` com os argumentos `path`, `dir_fd`.

Alterado na versão 3.3: Adicionado o parâmetro *dir_fd*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.utime(*path*, *times=None*, *, [*ns*,]*dir_fd=None*, *follow_symlinks=True*)

Define os tempos de acesso e modificação do arquivo especificado por *path*.

`utime()` aceita dois parâmetros opcionais, *times* e *ns*. Eles especificam os horários definidos no *path* e são usados da seguinte forma:

- Se *ns* for especificado, deve ser uma tupla de 2 elementos na forma `(atime_ns, mtime_ns)` onde cada membro é um inteiro expressando nanossegundos.
- If *times* is not `None`, it must be a 2-tuple of the form `(atime, mtime)` where each member is a real number expressing seconds, rounded down to nanoseconds.

- Se *times* for `None` e *ns* não for especificado, isso é equivalente a especificar `ns=(atime_ns, mtime_ns)` onde ambos os tempos são a hora atual.

É um erro especificar tuplas para ambos *times* e *ns*.

Observe que os tempos exatos que você definiu aqui podem não ser retornados por uma chamada subsequente de **`stat()`**, dependendo da resolução com a qual seu sistema operacional registra os tempos de acesso e modificação; veja **`stat()`**. A melhor maneira de preservar os tempos exatos é usar os campos *st_atime_ns* e *st_mtime_ns* do objeto de resultado **`os.stat()`** com o parâmetro *ns* para **`utime()`**.

Esta função pode oferecer suporte a especificar um descritor de arquivo, caminhos relativos aos descritores de diretório e não seguir os links simbólicos.

Levanta um evento de auditoria `os.utime` com os argumentos `path`, `times`, `ns` e `dir_fd`.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto e os parâmetros *dir_fd*, *follow_symlinks* e *ns*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.15: Accepts any real numbers as *times*, not only integers or floats.

os.walk(*top*, *topdown=True*, *onerror=None*, *followlinks=False*)

Gera os nomes dos arquivos em uma árvore de diretório percorrendo a árvore de cima para baixo ou de baixo para cima. Para cada diretório na árvore com raiz no diretório *top* (incluindo o próprio *top*), ele produz uma tupla de 3 elementos `(dirpath, dirnames, filenames)`.

dirpath é uma string, o caminho para o diretório. *dirnames* é uma lista dos nomes dos subdiretórios em *dirpath* (incluindo links simbólicos para diretórios e excluindo `'.'` e `'..'`). *filenames* é uma lista dos nomes

dos arquivos não pertencentes ao diretório em *dirpath*. Observe que os nomes nas listas não contêm componentes de caminho. Para obter um caminho completo (que começa com *top*) para um arquivo ou diretório em *dirpath*, execute `os.path.join(dirpath, name)`. Se as listas são ou não ordenadas depende do sistema de arquivos. Se um arquivo for removido ou adicionado ao diretório *dirpath* durante a geração das listas, não é especificado se um nome para esse arquivo deve ser incluído.

Se o argumento opcional *topdown* for `True` ou não especificado, o triplo para um diretório é gerado antes dos triplos para qualquer um de seus subdiretórios (os diretórios são gerados de cima para baixo). Se *topdown* for `False`, o triplo para um diretório é gerado após os triplos para todos os seus subdiretórios (os diretórios são gerados de baixo para cima). Não importa o valor de *topdown*, a lista de subdiretórios é recuperada antes que as tuplas para o diretório e seus subdiretórios sejam geradas.

Quando *topdown* é `True`, o chamador pode modificar a lista de *dirnames* internamente (talvez usando **`del`** ou atribuição de fatia), e **`walk()`** só recursará nos subdiretórios cujos nomes permanecem em *dirnames*; isso pode ser usado para podar a busca, impor uma ordem específica de visita, ou mesmo informar à função **`walk()`** sobre os diretórios que o chamador cria ou renomeia antes de retomar **`walk()`** novamente. Modificar *dirnames* quando *topdown* for `False` não tem efeito no comportamento da caminhada, porque no modo de baixo para cima os diretórios em *dirnames* são gerados antes do próprio *dirpath* ser gerado.

Por padrão, os erros da chamada de **`scandir()`** são ignorados. Se o argumento opcional *onerror* for especificado, deve ser uma função; ela será chamado com um argumento, uma instância da exceção **`OSError`**. Ele pode relatar o erro para continuar com a caminhada ou levantar a exceção para abortar a caminhada. Observe que o nome do arquivo está disponível como o atributo `filename` do objeto de exceção.

Por padrão, a função **`walk()`** não vai seguir links simbólicos que resolvem para diretórios. Defina *followlinks* como `True` para visitar

diretórios apontados por links simbólicos, em sistemas que oferecem suporte a eles.

Nota

Esteja ciente de que definir *followlinks* para `True` pode levar a recursão infinita se um link apontar para um diretório pai de si mesmo. `walk()` não mantém registro dos diretórios que já visitou.

Nota

Se você passar um nome de caminho relativo, não mude o diretório de trabalho atual entre as continuações de `walk()`. **`walk()`** nunca muda o diretório atual, e presume que seu chamador também não.

Este exemplo exibe o número total de bytes dos arquivos não-diretório em cada diretório no diretório inicial, exceto que ele não olha em nenhum subdiretório chamado `__pycache__`:

```
import os
from os.path import join, getsize
for root, dirs, files in os.walk('python/Lib/xml'):
    print(root, "consome", end=" ")
    print(sum(getsize(join(root, name)) for name in files), e
    print("bytes em", len(files), "arquivos que não são diret
    if '__pycache__' in dirs:
        dirs.remove('__pycache__')  # não visita diretórios c
```

No próximo exemplo (implementação simples de `shutil.rmtree()`), andar na árvore de baixo para cima é essencial, `rmdir()` não permite excluir um diretório antes que o diretório esteja vazio:

```
# Exclui tudo que for acessível do diretório nomeado em "top'
# presumindo que não há links simbólicos.
# CUIDADO: Isso é perigoso! Por exemplo, se top == '/',
# ele pode excluir todos os seus arquivos de disco.
import os
for root, dirs, files in os.walk(top, topdown=False):
    for name in files:
```

```
        os.remove(os.path.join(root, name))
    for name in dirs:
        os.rmdir(os.path.join(root, name))
os.rmdir(top)
```

Levanta um evento de auditoria `os.walk` com os argumentos `top`, `topdown`, `onerror` e `followlinks`.

Alterado na versão 3.5: Esta função agora chama **`os.scandir()`** em vez de **`os.listdir()`**, tornando-a mais rápida reduzindo o número de chamadas a **`os.stat()`**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os.fwalk(*top='.', topdown=True, onerror=None, *, follow_symlinks=False, dir_fd=None*)

Se comporta exatamente como **`walk()`**, exceto que produz um tupla de 4 elementos `(dirpath, dirnames, filenames, dirfd)`, e tem suporte a `dir_fd`.

dirpath, *dirnames* e *filenames* são idênticos à saída de **`walk()`** e *dirfd* é um descritor de arquivo que faz referência ao diretório *dirpath*.

Esta função sempre tem suporte a caminhos relativos aos descritores de diretório e não seguir links simbólicos. Observe, entretanto, que, ao contrário de outras funções, o valor padrão **`fwalk()`** para *follow_symlinks* é `False`.

Nota

Uma vez que **`fwalk()`** produz descritores de arquivo, eles só são válidos até a próxima etapa de iteração, então você deve duplicá-los (por exemplo, com **`dup()`**) se quiser mantê-los por mais tempo.

Este exemplo exibe o número total de bytes dos arquivos não-diretório em cada diretório no diretório inicial, exceto que ele não olha em nenhum subdiretório chamado `__pycache__`:

```
import os
for root, dirs, files, rootfd in os.fwalk('python/Lib/xml'):
    print(root, "consome", end=" ")
    print(sum([os.stat(name, dir_fd=rootfd).st_size for name
          end=" ")
    print("bytes em", len(files), "arquivos que não são diret
    if '__pycache__' in dirs:
        dirs.remove('__pycache__')  # não visita diretórios c
```

No próximo exemplo, percorrer a árvore de baixo para cima é essencial: **rmdir()** não permite excluir um diretório antes que o diretório esteja vazio:

```
# Exclui tudo que for acessível do diretório nomeado em "top"
# presumindo que não há links simbólicos.
# CUIDADO: Isso é perigoso! Por exemplo, se top == '/',
# ele pode excluir todos os seus arquivos de disco.
import os
for root, dirs, files, rootfd in os.fwalk(top, topdown=False)
    for name in files:
        os.unlink(name, dir_fd=rootfd)
    for name in dirs:
        os.rmdir(name, dir_fd=rootfd)
```

Levanta um evento de auditoria `os.fwalk` com os argumentos `top`, `topdown`, `onerror`, `follow_symlinks` e `dir_fd`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.7: Adicionado suporte para caminhos em **bytes**.

os.memfd_create(*name*[, *flags=os.MFD_CLOEXEC*])

Cria um arquivo anônimo e retorna um descritor de arquivo que faça referência a ele. *flags* deve ser uma das constantes `os.MFD_*` disponíveis

no sistema (ou uma combinação de OU bit a bit delas). Por padrão, o novo descritor de arquivo é não herdável.

O nome fornecido em *name* é usado como um nome de arquivo e será exibido como o destino do link simbólico correspondente no diretório `/proc/self/fd/`. O nome exibido é sempre prefixado com `memfd:` e serve apenas para fins de depuração. Os nomes não afetam o comportamento do descritor de arquivo e, como tal, vários arquivos podem ter o mesmo nome sem quaisquer efeitos colaterais.

Disponibilidade: Linux >= 3.17 with glibc >= 2.27.

Adicionado na versão 3.8.

os.MFD_CLOEXEC
os.MFD_ALLOW_SEALING
os.MFD_HUGETLB
os.MFD_HUGE_SHIFT
os.MFD_HUGE_MASK
os.MFD_HUGE_64KB
os.MFD_HUGE_512KB
os.MFD_HUGE_1MB
os.MFD_HUGE_2MB
os.MFD_HUGE_8MB
os.MFD_HUGE_16MB
os.MFD_HUGE_32MB
os.MFD_HUGE_256MB
os.MFD_HUGE_512MB
os.MFD_HUGE_1GB
os.MFD_HUGE_2GB
os.MFD_HUGE_16GB

Esses sinalizadores podem ser passados para **`memfd_create()`**.

Disponibilidade: Linux >= 3.17 with glibc >= 2.27

Os sinalizadores `MFD_HUGE*` estão disponíveis somente a partir do Linux 4.14.

Adicionado na versão 3.8.

os.eventfd(*initval*[, *flags=os.EFD_CLOEXEC*])

Cria e retorna um descritor de arquivo de evento. Os descritores de arquivo implementam **read()** e **write()** brutos com um tamanho de buffer de 8, **select()**, **poll()** e similares. Veja a página man *eventfd(2)* *[https://manpages.debian.org/eventfd(2)]* para mais informações. Por padrão, o novo descritor de arquivo é não herdável.

initval é o valor inicial do contador de evento. O valor inicial deve ser um inteiro sem sinal de 32 bits. Observe que o valor inicial é limitado a um inteiro sem sinal de 32 bits ainda que o contador de evento seja um inteiro de 64 bits com um valor máximo de 2^{64}-2.

flags podem ser construídas a partir de **EFD_CLOEXEC**, **EFD_NONBLOCK** e **EFD_SEMAPHORE**.

Se **EFD_SEMAPHORE** for especificado e o contador de evento for diferente de zero, **eventfd_read()** retorna 1 e decrementa o contador em um.

Se **EFD_SEMAPHORE** não for especificado e o contador de evento for diferente de zero, **eventfd_read()** retorna o valor atual do contador de evento e zera o contador.

Se o contador de evento for zero e **EFD_NONBLOCK** não for especificado, **eventfd_read()** bloqueia.

eventfd_write() incrementa o contador de evento. A escrita bloqueia se a operação de escrita incrementar o contador para um valor maior que 2^{64}-2.

Exemplo:

```
import os

# semaphore with start value '1'
fd = os.eventfd(1, os.EFD_SEMAPHORE | os.EFD_CLOEXEC)
try:
```

```
        # acquire semaphore
        v = os.eventfd_read(fd)
        try:
            do_work()
        finally:
            # release semaphore
            os.eventfd_write(fd, v)
    finally:
        os.close(fd)
```

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.10.

os.eventfd_read(*fd*)

Lê um valor de um descritor de arquivo **`eventfd()`** e retorna um inteiro sem sinal de 64 bits. A função não verifica se *fd* é um **`eventfd()`**.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.eventfd_write(*fd*, *value*)

Adiciona *value* a um descritor de arquivo **`eventfd()`**. *value* deve ser um inteiro sem sinal de 64 bits. A função não verifica se *fd* é um **`eventfd()`**.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.EFD_CLOEXEC

Define sinalizador close-on-exec para o novo descritor de arquivo **`eventfd()`**.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.EFD_NONBLOCK

Define sinalizador de status `O_NONBLOCK` para um novo descritor de arquivo `eventfd()`.

Disponibilidade: Linux >= 2.6.27

Adicionado na versão 3.10.

os.EFD_SEMAPHORE

Fornece semântica de semáforo ou similar para leitura de um descritor de arquivo `eventfd()`. Na leitura o contador interno é decrementado em um.

Disponibilidade: Linux >= 2.6.30

Adicionado na versão 3.10.

Descritores de arquivo de temporizador

Adicionado na versão 3.13.

Essas funções fornecem suporte para a API de *descritores de arquivo de temporizador*.

os.timerfd_create(*clockid*, /, *, *flags=0*)

Cria e retorna um descritor de arquivo de temporizador (*timerfd*).

O descritor de arquivo retornado por `timerfd_create()` implementa:

- `read()`
- `select()`
- `poll()`

O método `read()` do descritor de arquivo pode ser chamado com um buffer de tamanho 8. Se o temporizador já expirou uma ou mais vezes, **`read()`** retorna o número de vezes que o descritor expirou com a ordem dos bytes no formato do hospedeiro, que pode ser convertido para um `int` com `int.from_bytes(x, byteorder=sys.byteorder)`.

`select()` e `poll()` pode ser usado para esperar até que o temporizador expire e o descritor de arquivo possa ser lido.

clockid deve ser um ID de relógio válido, conforme definido no módulo `time`:

- `time.CLOCK_REALTIME`
- `time.CLOCK_MONOTONIC`
- `time.CLOCK_BOOTTIME` (A partir de Linux 3.15 para timerfd_create)

If *clockid* is `time.CLOCK_REALTIME`, a settable system-wide real-time clock is used. If the system clock is changed, the timer setting needs to be updated. To cancel the timer when the system clock is changed, see `TFD_TIMER_CANCEL_ON_SET`.

Se *clockid* é `time.CLOCK_MONOTONIC`, é usado um relógio monotonicamente não estável. Mesmo se o relógio do sistema é alterado, a configuração do temporizador não será afetada.

If *clockid* is `time.CLOCK_BOOTTIME`, it is the same as `time.CLOCK_MONOTONIC` except it includes any time that the system is suspended.

The file descriptor's behaviour can be modified by specifying a *flags* value. Any of the following variables may be used, combined using bitwise OR (the `|` operator):

- `TFD_NONBLOCK`
- `TFD_CLOEXEC`

If `TFD_NONBLOCK` is not set as a flag, `read()` blocks until the timer expires. If it is set as a flag, **`read()`** doesn't block, but if there hasn't been an expiration since the last call to read, **`read()`** raises `OSError` with `errno` set to `errno.EAGAIN`.

`TFD_CLOEXEC` é sempre definido pelo Python automaticamente.

O descritor de arquivo deve ser fechado com `os.close()` quando não é mais necessário, caso contrário o descritor de arquivo será vazado.

Ver também

A página man *timerfd_create(2)* *[https://manpages.debian.org/timerfd_create(2)]*.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.timerfd_settime(*fd*, *, *, *flags=0*, *initial=0.0*, *interval=0.0*)

Altera um temporizador interno do descritor de arquivo de temporizador. Essa função opera no mesmo intervalo do temporizador definido com `timerfd_settime_ns()`.

fd deve ser um descritor de arquivo de temporizador válido.

The timer's behaviour can be modified by specifying a *flags* value. Any of the following variables may be used, combined using bitwise OR (the `|` operator):

- `TFD_TIMER_ABSTIME`
- `TFD_TIMER_CANCEL_ON_SET`

The timer is disabled by setting *initial* to zero (`0`). If *initial* is greater than zero, the timer is enabled. If *initial* is less than zero, it raises an `OSError` exception with `errno` set to `errno.EINVAL`.

By default the timer will fire when *initial* seconds have elapsed.

Contudo, se o sinalizador `TFD_TIMER_ABSTIME` está definido, o temporizador dispara quando o relógio do temporizador (definido por *clockid* em `timerfd_create()`) atingir *initial* segundos.

The timer's interval is set by the *interval* real number. If *interval* is zero, the timer only fires once, on the initial expiration. If *interval* is greater than zero, the timer fires every time *interval* seconds have elapsed since

the previous expiration. If *interval* is less than zero, it raises **OSError** with `errno` set to **errno.EINVAL**.

If the **TFD_TIMER_CANCEL_ON_SET** flag is set along with **TFD_TIMER_ABSTIME** and the clock for this timer is **time.CLOCK_REALTIME**, the timer is marked as cancelable if the real-time clock is changed discontinuously. Reading the descriptor is aborted with the error **errno.ECANCELED**.

Linux gerencia relógio do sistema como UTC. A transição do horário de verão é feita apenas pela alteração da compensação de horário e não causa alteração descontínua do relógio do sistema.

A alteração descontínua do relógio do sistema será causada pelos seguintes eventos:

- `settimeofday`
- `clock_settime`
- definir hora e data do sistema com o comando `date`

Retorna uma tupla com dois elementos (`next_expiration`, `interval`) do estado anterior do temporizador, antes da execução dessa função.

Ver também

timerfd_create(2) *[https://manpages.debian.org/timerfd_create(2)]*, *timerfd_settime(2)* *[https://manpages.debian.org/timerfd_settime(2)]*, *settimeofday(2)* *[https://manpages.debian.org/settimeofday(2)]*, *clock_settime(2)* *[https://manpages.debian.org/clock_settime(2)]*, e *date(1)* *[https://manpages.debian.org/date(1)]*.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.timerfd_settime_ns(*fd*, *,* ***, *flags=0*, *initial=0*, *interval=0*)

Similar a **timerfd_settime()**, mas usa nanossegundos. Essa função opera no mesmo intervalo do temporizador definido com

`timerfd_settime()`.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.timerfd_gettime(*fd*, /)

Retorna uma tupla de ponto flutuante com dois elementos (`next_expiration`, `interval`).

`next_expiration` denotes the relative time until the timer next fires, regardless of if the **`TFD_TIMER_ABSTIME`** flag is set.

`interval` indica o intervalo do cronômetro. Se for zero, o cronômetro será acionado apenas uma vez, após terem passados `next_expiration` segundos.

Ver também

timerfd_gettime(2) *[https://manpages.debian.org/timerfd_gettime(2)]*

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.timerfd_gettime_ns(*fd*, /)

Similar a **`timerfd_gettime()`**, mas retorna o tempo em nanossegundos.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.TFD_NONBLOCK

Um sinalizador para a função **`timerfd_create()`**, que define o estado do sinalizador **`O_NONBLOCK`** para o novo descritor de arquivo de temporizador. Se **`TFD_NONBLOCK`** não for definido como sinalizador, **`read()`** bloqueia.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.TFD_CLOEXEC

Um sinalizador para a função `timerfd_create()`. Se `TFD_CLOEXEC` for definido como um sinalizador, define o sinalizador close-on-exec para o novo descritor de arquivo.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.TFD_TIMER_ABSTIME

Um sinalizador para as funções `timerfd_settime()` e `timerfd_settime_ns()`. Se esse sinalizador for definido, *initial* será intepretador como um valor absoluto no relógio do temporizador (em segundos UTC ou nanossegundos da Era Unix).

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

os.TFD_TIMER_CANCEL_ON_SET

Um sinalizador para as funções `timerfd_settime()` e `timerfd_settime_ns()` juntamente com `TFD_TIMER_ABSTIME`. O cronômetro é cancelado quando a hora do relógio subjacente muda de forma descontínua.

Disponibilidade: Linux >= 2.6.27 with glibc >= 2.8

Adicionado na versão 3.13.

Atributos estendidos do Linux

Adicionado na versão 3.3.

Estas funções estão todas disponíveis apenas no Linux.

os.getxattr(*path*, *attribute*, *, *follow_symlinks=True*)

Retorna o valor do atributo estendido do sistema de arquivos *attribute* para *path*. *attribute* pode ser bytes ou str (direta ou indiretamente por meio da interface `PathLike`). Se for str, ele é codificado com a codificação do sistema de arquivos.

Esta função tem suporte a especificar um descritor de arquivo e não seguir links simbólicos.

Levanta um evento de auditoria `os.getxattr` com os argumentos `path` e `attribute`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *path* e *attribute*.

os.listxattr(*path=None*, *, *follow_symlinks=True*)

Retorna uma lista dos atributos estendidos do sistema de arquivos em *path*. Os atributos na lista são representados como strings decodificadas com a codificação do sistema de arquivos. Se *path* for `None`, `listxattr()` irá examinar o diretório atual.

Esta função tem suporte a especificar um descritor de arquivo e não seguir links simbólicos.

Levanta um evento de auditoria `os.listxattr` com o argumento `path`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.15: `os.listxattr(-1)` now fails with `OSError(errno.EBADF)` rather than listing extended attributes of the current directory.

os.removexattr(*path*, *attribute*, *, *follow_symlinks=True*)

Remove o atributo *attribute* do sistema de arquivos estendido do *path*. *attribute* deve ser bytes ou str (direta ou indiretamente através da interface `PathLike`). Se for uma string, é codificada com o tratador de erros e codificação do sistema de arquivos

Esta função tem suporte a especificar um descritor de arquivo e não seguir links simbólicos.

Levanta um evento de auditoria `os.removexattr` com os argumentos `path` e `attribute`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *path* e *attribute*.

os.setxattr(*path*, *attribute*, *value*, *flags=0*, *, *follow_symlinks=True*)

Define o atributo de sistema de arquivos estendido *attribute* em *path* como *value*. *attribute* deve ser um bytes ou str sem NULs embutidos (direta ou indiretamente por meio da interface **`PathLike`**). Se for um str, ele é codificado com tratador de erros e codificação do sistema de arquivos. *flags* podem ser **`XATTR_REPLACE`** ou **`XATTR_CREATE`**. Se **`XATTR_REPLACE`** for fornecido e o atributo não existe, `ENODATA` será levantada. Se **`XATTR_CREATE`** for fornecido e o atributo já existir, o atributo não será criado e `EEXISTS` será levantada.

Esta função tem suporte a especificar um descritor de arquivo e não seguir links simbólicos.

Nota

Um bug nas versões do kernel Linux inferiores a 2.6.39 fez com que o argumento *flags* fosse ignorado em alguns sistemas de arquivos.

Levanta um evento de auditoria `os.setxattr` com os argumentos `path`, `attribute`, `value` e `flags`.

Alterado na versão 3.6: Aceita um objeto caminho ou similar para *path* e *attribute*.

os.XATTR_SIZE_MAX

O tamanho máximo que o valor de um atributo estendido pode ter. Atualmente, são 64 KiB no Linux.

os.XATTR_CREATE

Este é um valor possível para o argumento *flags* em `setxattr()`. Indica que a operação deve criar um atributo.

os.XATTR_REPLACE

Este é um valor possível para o argumento *flags* em `setxattr()`. Indica que a operação deve substituir um atributo existente.

Gerenciamento de processo

Estas funções podem ser usadas para criar e gerenciar processos.

As várias funções `exec*` recebem uma lista de argumentos para o novo programa carregado no processo. Em cada caso, o primeiro desses argumentos é passado para o novo programa como seu próprio nome, e não como um argumento que um usuário pode ter digitado em uma linha de comando. Para o programador C, este é o `argv[0]` passado para um programa **`main()`**. Por exemplo, `os.execv('/bin/echo', ['foo', 'bar'])` exibirá apenas `bar` na saída padrão; `foo` parecerá ser ignorado.

os.abort()

Generate a `SIGABRT` signal to the current process. On Unix, the default behavior is to produce a core dump; on Windows, the process immediately returns an exit code of `3`. Be aware that calling this function will not call the Python signal handler registered for **`SIGABRT`** with `signal.signal()`.

os.add_dll_directory(*path*)

Adiciona um caminho ao caminho de pesquisa de DLL.

Este caminho de pesquisa é usado ao resolver dependências para módulos de extensão importados (o próprio módulo é resolvido por meio de `sys.path`), e também por `ctypes`.

Remove o diretório chamando **close()** no objeto retornado ou usando-o em uma instrução `with`.

Consulte a documentação da Microsoft [https://msdn.microsoft.com/44228cf2-6306-466c-8f16-f513cd3ba8b5] para obter mais informações sobre como as DLLs são carregadas.

Levanta um evento de auditoria `os.add_dll_directory` com o argumento `path`.

Disponibilidade: Windows.

Adicionado na versão 3.8: As versões anteriores do CPython resolveriam DLLs usando o comportamento padrão para o processo atual. Isso levou a inconsistências, como apenas às vezes pesquisar **`PATH`** ou o diretório de trabalho atual, e funções do sistema operacional como `AddDllDirectory` sem efeito.

No 3.8, as duas maneiras principais de carregar as DLLs agora substituem explicitamente o comportamento de todo o processo para garantir a consistência. Veja as notas de portabilidade para informações sobre atualização de bibliotecas.

os.execl(*path*, *arg0*, *arg1*, ...)
os.execle(*path*, *arg0*, *arg1*, ..., *env*)
os.execlp(*file*, *arg0*, *arg1*, ...)
os.execlpe(*file*, *arg0*, *arg1*, ..., *env*)
os.execv(*path*, *args*)
os.execve(*path*, *args*, *env*)
os.execvp(*file*, *args*)
os.execvpe(*file*, *args*, *env*)

Todas essas funções executam um novo programa, substituindo o processo atual; elas não retornam. No Unix, o novo executável é carregado no processo atual e terá a mesma identificação de processo do chamador. Os erros serão relatados como exceções de **`OSError`**.

The current process is replaced immediately. Open file objects and descriptors are not flushed, so if there may be data buffered on these

open files, you should flush them using `flush()` or `os.fsync()` before calling an `exec*` function.

As variantes “l” e “v” das funções `exec*` diferem em como os argumentos da linha de comando são passados. As variantes “l” são talvez as mais fáceis de trabalhar se o número de parâmetros for fixo quando o código for escrito; os parâmetros individuais simplesmente se tornam parâmetros adicionais para as funções **`execl*()`**. As variantes “v” são boas quando o número de parâmetros é variável, com os argumentos sendo passados em uma lista ou tupla como o parâmetro *args*. Em qualquer caso, os argumentos para o processo filho devem começar com o nome do comando que está sendo executado, mas isso não é obrigatório.

As variantes que incluem um “p” próximo ao final (`execlp()`, `execlpe()`, `execvp()` e `execvpe()`) usarão a variável de ambiente **`PATH`** para localizar o programa *file*. Quando o ambiente está sendo substituído (usando uma das variantes `exec*e`, discutidas no próximo parágrafo), o novo ambiente é usado como fonte da variável **`PATH`**. As outras variantes, **`execl()`**, `execle()`, `execv()` e `execve()`, não usarão a variável **`PATH`** para localizar o executável; *path* deve conter um caminho absoluto ou relativo apropriado. Caminhos relativos devem incluir pelo menos uma barra, mesmo no Windows, pois nomes simples não serão resolvidos.

Para `execle()`, `execlpe()`, `execve()` e `execvpe()` (observe que todos eles terminam em “e”), o parâmetro *env* deve ser um mapeamento que é usado para definir as variáveis de ambiente para o novo processo (elas são usadas no lugar do ambiente do processo atual); as funções `execl()`, `execlp()`, `execv()` e `execvp()` fazem com que o novo processo herde o ambiente do processo atual.

Para `execve()` em algumas plataformas, *path* também pode ser especificado como um descritor de arquivo aberto. Esta funcionalidade pode não ser compatível com sua plataforma; você pode verificar se está ou não disponível usando `os.supports_fd`. Se não estiver disponível, usá-lo vai levantar uma `NotImplementedError`.

Levanta um evento de auditoria `os.exec` com os argumentos `path`, `args` e `env`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

Alterado na versão 3.3: Adicionado suporte para especificar *path* como um descritor de arquivo aberto para **`execve()`**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

os._exit(*n*)

Sai do processo com status *n*, sem chamar manipuladores de limpeza, liberando buffers de entrada e saída padrões etc.

Nota

A forma padrão de sair é **`sys.exit(n)`**. **`_exit()`** normalmente só deve ser usado no processo filho após uma função **`fork()`**.

Os seguintes códigos de saída são definidos e podem ser usados com **`_exit()`**, embora não sejam obrigatórios. Eles são normalmente usados para programas de sistema escritos em Python, como um programa de entrega de comando externo de servidor de e-mail.

Nota

Alguns deles podem não estar disponíveis em todas as plataformas Unix, pois há algumas variações. Essas constantes estão definidas onde elas são definidas pela plataforma subjacente.

os.EX_OK

Código de saída que significa que não ocorreu nenhum erro. Pode ser obtido do valor definido de `EXIT_SUCCESS` em algumas plataformas. Geralmente tem um valor de zero.

Disponibilidade: Unix, Windows.

os.EX_USAGE

Código de saída que significa que o comando foi usado incorretamente, como quando o número errado de argumentos é fornecido.

Disponibilidade: Unix, not WASI.

os.EX_DATAERR

Código de saída que significa que os dados inseridos estavam incorretos.

Disponibilidade: Unix, not WASI.

os.EX_NOINPUT

Código de saída que significa que um arquivo de entrada não existe ou não pôde ser lido.

Disponibilidade: Unix, not WASI.

os.EX_NOUSER

Código de saída que significa que um usuário especificado não existe.

Disponibilidade: Unix, not WASI.

os.EX_NOHOST

Código de saída que significa que um host especificado não existe.

Disponibilidade: Unix, not WASI.

os.EX_UNAVAILABLE

Código de saída que significa que um serviço necessário está indisponível.

Disponibilidade: Unix, not WASI.

os.EX_SOFTWARE

Código de saída que significa que um erro interno do software foi detectado.

Disponibilidade: Unix, not WASI.

os.EX_OSERR

Código de saída que significa que um erro do sistema operacional foi detectado, como a incapacidade de criar um fork ou um encadeamento.

Disponibilidade: Unix, not WASI.

os.EX_OSFILE

Código de saída que significa que algum arquivo do sistema não existia, não pôde ser aberto ou teve algum outro tipo de erro.

Disponibilidade: Unix, not WASI.

os.EX_CANTCREAT

Código de saída que significa que um arquivo de saída especificado pelo usuário não pôde ser criado.

Disponibilidade: Unix, not WASI.

os.EX_IOERR

Código de saída que significa que ocorreu um erro ao fazer E/S em algum arquivo.

Disponibilidade: Unix, not WASI.

os.EX_TEMPFAIL

Código de saída que significa que ocorreu uma falha temporária. Isso indica algo que pode não ser realmente um erro, como uma conexão de rede que não pôde ser feita durante uma operação de nova tentativa.

Disponibilidade: Unix, not WASI.

os.EX_PROTOCOL

Código de saída que significa que uma troca de protocolo foi ilegal, inválida ou não compreendida.

Disponibilidade: Unix, not WASI.

os.EX_NOPERM

Código de saída que significa que não havia permissões suficientes para executar a operação (mas sem intenção de causar problemas do sistema de arquivos).

Disponibilidade: Unix, not WASI.

os.EX_CONFIG

Código de saída que significa que ocorreu algum tipo de erro de configuração.

Disponibilidade: Unix, not WASI.

os.EX_NOTFOUND

Código de saída que significa algo como “uma entrada não foi encontrada”.

Disponibilidade: Unix, not WASI.

os.fork()

Cria um processo filho. Retorna `0` no filho e o ID de processo do filho no pai. Se ocorrer um erro, uma exceção **`OSError`** é levantada.

Note que algumas plataformas incluindo FreeBSD <= 6.3 e Cygwin têm problemas conhecidos ao usar `fork()` a partir de um thread.

Levanta um evento de auditoria `os.fork` sem argumentos.

Aviso

Se você usa soquetes TLS em um aplicação chamando `fork()`, consulte alerta na documentação do módulo **`ssl`**.

Aviso

No macOS, o uso desta função é inseguro quando combinado com o uso de APIs de sistema de nível mais alto, e isso inclui o uso de **`urllib.request`**.

Alterado na versão 3.8: Chamar `fork()` em um subinterpretador não é mais possível (**`RuntimeError`** é levantada).

Alterado na versão 3.12: Se o Python for capaz de detectar que o seu processar tem várias threads, o **`os.fork()`** agora levanta uma exceção **`DeprecationWarning`**.

Optamos por apresentar isso como um alerta, quando detectável, para informar melhor aos desenvolvedores sobre um problema de design que a plataforma POSIX indica especificamente como sem suporte. Mesmo em códigos que *aparentem* funcionar, nunca foi seguro misturar threading com **`os.fork()`** em plataformas POSIX. O próprio ambiente de execução do CPython sempre fez chamadas à API que não são seguras para uso no processo filho quando existiam threads no pai (como `malloc` e `free`).

Os usuários do macOS ou usuários de implementações de libc ou malloc diferentes daquelas normalmente encontradas no glibc até o momento estão entre os que já têm maior probabilidade de sofrer com impasses (*deadlocks*) ao executar esse código.

Consulte esta discussão sobre fork ser incompatível com threads [https://discuss.python.org/t/33555] para obter detalhes técnicos sobre o motivo pelo qual estamos revelando aos desenvolvedores esse problema de longa data de compatibilidade da plataforma.

Disponibilidade: POSIX, not WASI, not Android, not iOS.

os.forkpty()

Cria um processo filho, usando um novo pseudoterminal como o terminal de controle do filho. Retorna um par de `(pid, fd)`, onde *pid* é `0` no filho, o novo ID de processo do filho no pai e *fd* é o descritor de arquivo do lado mestre do pseudoterminal. Para uma abordagem mais portátil, use o módulo **`pty`**. Se ocorrer um erro, **`OSError`** é levantada.

O descritor de arquivo *fd* retornado é não herdável.

Levanta um evento de auditoria `os.forkpty` sem argumentos.

Aviso

No macOS, o uso desta função é inseguro quando combinado com o uso de APIs de sistema de nível mais alto, e isso inclui o uso de `urllib.request`.

Alterado na versão 3.8: Chamar `forkpty()` em um subinterpretador não é mais permitido (`RuntimeError` é levantada).

Alterado na versão 3.12: Se o Python for capaz de detectar que o seu processo tem vários threads, isso agora levanta uma exceção `DeprecationWarning`. Veja a explicação mais longa em `os.fork()`.

Alterado na versão 3.15: O descritor de arquivo retornado agora for tornado não-herdável.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.kill(*pid*, *sig*, /)

Envia o sinal *sig* para o processo *pid*. Constantes dos sinais específicos disponíveis na plataforma host são definidas no módulo `signal`.

Windows: os sinais `signal.CTRL_C_EVENT` e `signal.CTRL_BREAK_EVENT` são sinais especiais que só podem ser enviados para processos de console que compartilham uma janela de console comum, por exemplo, alguns subprocessos. Qualquer outro valor para *sig* fará com que o processo seja encerrado incondicionalmente pela API TerminateProcess e o código de saída será definido como *sig*.

Veja também `signal.pthread_kill()`.

Levanta um evento de auditoria `os.kill` com os argumentos `pid` e `sig`.

Disponibilidade: Unix, Windows, not WASI, not iOS.

Alterado na versão 3.2: Adicionado suporte ao Windows.

os.killpg(*pgid*, *sig*, *[/]*)

Envia o sinal *sig* para o grupo de processos *pgid*.

Levanta um evento de auditoria `os.killpg` com os argumentos `pgid` e `sig`.

Disponibilidade: Unix, not WASI, not iOS.

os.nice(*increment*, *[/]*)

Adiciona *increment* ao nível de “nice” do processo. Retorna um novo nível de “nice”.

Disponibilidade: Unix, not WASI.

os.pidfd_open(*pid*, *flags=0*)

Retorna um descritor de arquivo referente ao processo *pid* com as *flags* definidas. Este descritor pode ser usado para gerenciar o processo sem corridas e sinais.

Veja a página man *pidfd_open(2)* *[https://manpages.debian.org/pidfd_open(2)]* para mais detalhes.

Disponibilidade: Linux >= 5.3, Android >= **`build-time`** API level 31

Adicionado na versão 3.9.

os.PIDFD_NONBLOCK

Este sinalizador indica que o descritor de arquivo será não-bloqueante. Se o processo referenciado pelo descritor de arquivo ainda não tiver terminado, então uma tentativa de aguardar no descritor de arquivo usando *waitid(2)* *[https://manpages.debian.org/waitid(2)]* retornará imediatamente o erro **`EAGAIN`** em vez de bloquear.

Disponibilidade: Linux >= 5.10

Adicionado na versão 3.12.

os.plock(*op*, /)

Trava os segmentos do programa na memória. O valor de *op* (definido em `<sys/lock.h>`) determina quais segmentos são travados.

Disponibilidade: Unix, not WASI, not macOS, not iOS.

os.popen(*cmd*, *mode='r'*, *buffering=-1*)

Abre um encadeamento, também conhecido como “pipe”, de ou para o comando *cmd*. O valor de retorno é um objeto arquivo aberto conectado ao encadeamento, que pode ser lido ou escrito dependendo se *mode* é `'r'` (padrão) ou `'w'`. O argumento *buffering* tem o mesmo significado que o argumento correspondente para a função embutida **`open()`**. O objeto arquivo retornado lê ou escreve strings de texto em vez de bytes.

O método `close` retorna **`None`** se o subprocesso foi encerrado com sucesso, ou o código de retorno do subprocesso se houve um erro. Em sistemas POSIX, se o código de retorno for positivo, ele representa o valor de retorno do processo deslocado para a esquerda em um byte. Se o código de retorno for negativo, o processo foi encerrado pelo sinal dado pelo valor negado do código de retorno. (Por exemplo, o valor de retorno pode ser `- signal.SIGKILL` se o subprocesso foi eliminado.) Em sistemas Windows, o valor de retorno contém o código de retorno inteiro com sinal do processo filho.

No Unix, **`waitstatus_to_exitcode()`** pode ser usado para converter o resultado do método `close` (status de saída) em um código de saída se não for `None`. No Windows, o resultado do método `close` é diretamente o código de saída (ou `None`).

Isso é implementado usando **`subprocess.Popen`**; consulte a documentação desta classe para maneiras mais poderosas de gerenciar e se comunicar com subprocessos.

Disponibilidade: not WASI, not Android, not iOS.

Nota

O Modo UTF-8 do Python afeta as codificações usadas para *cmd* e conteúdo de encadeamento.

`popen()` é um wrapper simples em torno de **`subprocess.Popen`**. Use **`subprocess.Popen`** ou **`subprocess.run()`** para controlar opções como codificações.

Suavemente descontinuado desde a versão 3.14: The **`subprocess`** module is recommended instead.

os.posix_spawn(*path*, *argv*, *env*, **, file_actions=None*, *setpgroup=None*, *resetids=False*, *setsid=False*, *setsigmask=()*, *setsigdef=()*, *scheduler=None*)

Envolve a API da biblioteca C **`posix_spawn()`** para uso em Python.

A maioria dos usuários deveria usar **`subprocess.run()`** em vez de **`posix_spawn()`**.

Os argumentos somente-posicional *path*, *args*, e *env* são similares a **`execve()`**. *env* pode ser `None`, caso em que o ambiente do processo atual é usado.

O parâmetro *path* é o caminho para o arquivo executável. O *path* deve conter um diretório. Use **`posix_spawnp()`** para passar um arquivo executável sem diretório.

O argumento *file_actions* pode ser uma sequência de tuplas descrevendo ações a serem tomadas em descritores de arquivo específicos no processo filho entre as etapas **`fork()`** e **`exec()`** de implementação da biblioteca C. O primeiro item em cada tupla deve ser um dos três indicadores de tipo listados abaixo, descrevendo os elementos restantes da tupla:

os.POSIX_SPAWN_OPEN

(`os.POSIX_SPAWN_OPEN`, *fd*, *path*, *flags*, *mode*)

Efetua `os.dup2(os.open(path, flags, mode), fd)`.

os.POSIX_SPAWN_CLOSE

(`os.POSIX_SPAWN_CLOSE`, *fd*)

Efetua `os.close(fd)`.

os.POSIX_SPAWN_DUP2

(`os.POSIX_SPAWN_DUP2`, *fd*, *new_fd*)

Efetua `os.dup2(fd, new_fd)`.

os.POSIX_SPAWN_CLOSEFROM

(`os.POSIX_SPAWN_CLOSEFROM`, *fd*)

Efetua `os.closerange(fd, INF)`.

Estas tuplas correspondem a chamadas de API da biblioteca C de **`posix_spawn_file_actions_addopen()`**, **`posix_spawn_file_actions_addclose()`**, **`posix_spawn_file_actions_adddup2()`** e **`posix_spawn_file_actions_addclosefrom_np()`** usadas para preparar para a chamada de **`posix_spawn()`** em si.

O argumento *setpgroup* definirá o grupo de processos do filho para o valor especificado. Se o valor especificado for 0, o ID do grupo de processo filho será igual ao ID do processo. Se o valor de *setpgroup* não for definido, o filho herdará o ID do grupo de processos do pai. Este argumento corresponde ao sinalizador **`POSIX_SPAWN_SETPGROUP`** da biblioteca C.

Se o argumento *resetids* for `True`, ele irá reconfigurar o UID e GID efetivos do filho para o UID e GID reais do processo pai. Se o argumento for `False`, então o filho retém o UID e GID efetivos do pai. Em ambos os casos, se os bits de permissão set-user-ID e set-group-ID estiverem habilitados no arquivo executável, seu efeito vai substituir a configuração do UID e GID efetivos. Este argumento corresponde ao sinalizador **`POSIX_SPAWN_RESETIDS`** da biblioteca C.

Se o argumento *setsid* for `True`, ele criará um novo ID de sessão para `posix_spawn`. *setsid* requer **`POSIX_SPAWN_SETSID`** ou **`POSIX_SPAWN_SETSID_NP`**. Caso contrário, **`NotImplementedError`** é levantada.

O argumento *setsigmask* definirá a máscara de sinal para o conjunto de sinais especificado. Se o parâmetro não for usado, o filho herda a máscara de sinal do pai. Este argumento corresponde ao sinalizador **`POSIX_SPAWN_SETSIGMASK`** da biblioteca C.

O argumento *sigdef* redefinirá a disposição de todos os sinais no conjunto especificado. Este argumento corresponde ao sinalizador **`POSIX_SPAWN_SETSIGDEF`** da biblioteca C.

O argumento *scheduler* deve ser uma tupla contendo a política do agendador (opcional) e uma instância de **`sched_param`** com os parâmetros do agendador. Um valor `None` no lugar da política do agendador indica que a política não está sendo fornecido. Este argumento é uma combinação dos sinalizadores **`POSIX_SPAWN_SETSCHEDPARAM`** e **`POSIX_SPAWN_SETSCHEDULER`** da biblioteca C.

Levanta um evento de auditoria `os.posix_spawn` com os argumentos `path`, `argv` e `env`.

Adicionado na versão 3.8.

Alterado na versão 3.13: o parâmetro *env* pode ser `None`. `os.POSIX_SPAWN_CLOSEFROM` está disponível nas plataformas onde **`posix_spawn_file_actions_addclosefrom_np()`** existe.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.posix_spawnp(*path*, *argv*, *env*, ***, *file_actions=None*, *setpgroup=None*, *resetids=False*, *setsid=False*, *setsigmask=()*, *setsigdef=()*, *scheduler=None*)

Envolve a API da biblioteca C **`posix_spawnp()`** para uso em Python.

Semelhante a **posix_spawn()** exceto que o sistema procura o arquivo *executable* na lista de diretórios especificados pela variável de ambiente **PATH** (da mesma forma que para `execvp(3)`).

Levanta um evento de auditoria `os.posix_spawn` com os argumentos `path`, `argv` e `env`.

Adicionado na versão 3.8.

Disponibilidade: POSIX, not WASI, not Android, not iOS.

Veja a documentação de **posix_spawn()**.

os.register_at_fork(*, *before=None*, *after_in_parent=None*, *after_in_child=None*)

Registra chamáveis a serem executados quando um novo processo filho é criado usando **os.fork()** ou APIs de clonagem de processos semelhantes. Os parâmetros são opcionais e somente-nomeados. Cada um especifica um ponto de chamada diferente.

- *before* é uma função chamada antes de criar um processo filho.
- *after_in_parent* é uma função chamada a partir do processo pai após criar um processo filho.
- *after_in_child* é uma função chamada a partir do processo filho.

Essas chamadas são feitas apenas se o controle deve retornar ao interpretador Python. Um lançamento típico de **subprocess** não irá acioná-los, pois o filho não entrará novamente no interpretador.

As funções registradas para execução antes de criar um processo filho são chamadas na ordem de registro reversa. As funções registradas para execução após o processo filho ser criado (no pai ou no filho) são chamadas na ordem de registro.

Note que chamadas a **fork()** feitas por código C de terceiros podem não chamar estas funções, a menos que ele explicitamente chamem **PyOS_BeforeFork()**, **PyOS_AfterFork_Parent()** e **PyOS_AfterFork_Child()**.

Não há uma forma de desfazer o registro de uma função.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.7.

os.spawnl(*mode*, *path*, ...)
os.spawnle(*mode*, *path*, ..., *env*)
os.spawnlp(*mode*, *file*, ...)
os.spawnlpe(*mode*, *file*, ..., *env*)
os.spawnv(*mode*, *path*, *args*)
os.spawnve(*mode*, *path*, *args*, *env*)
os.spawnvp(*mode*, *file*, *args*)
os.spawnvpe(*mode*, *file*, *args*, *env*)

Executa o programa *path* em um novo processo.

(Observe que o módulo **`subprocess`** fornece recursos mais poderosos para gerar novos processos e recuperar seus resultados; usar esse módulo é preferível a usar essas funções. Verifique especialmente a seção Replacing Older Functions with the subprocess Module.)

Se *mode* for **`P_NOWAIT`**, esta função retorna o id do processo do novo processo; se *mode* for **`P_WAIT`**, retorna o código de saída do processo se ele sair normalmente, ou `-signal`, onde *signal* é o sinal que matou o processo. No Windows, o id do processo será na verdade o manipulador do processo, portanto, pode ser usado com a função **`waitpid()`**.

Nota sobre VxWorks: esta função não retorna `-signal` se o novo processo é interrompido. Em vez disso, ele levanta a exceção OSError.

As variantes “l” e “v” das funções **`spawn*`** diferem em como os argumentos de linha de comando são passados. As variantes “l” são talvez as mais fáceis de trabalhar se o número de parâmetros for fixo quando o código for escrito; os parâmetros individuais simplesmente se tornam parâmetros adicionais para as funções **`spawnl*()`**. As variantes “v” são boas quando o número de parâmetros é variável, com os

argumentos sendo passados em uma lista ou tupla como o parâmetro *args*. Em ambos os casos, os argumentos para o processo filho devem começar com o nome do comando que está sendo executado.

As variantes que incluem um segundo "p" próximo ao final (**spawnlp()**, **spawnlpe()**, **spawnvp()** e **spawnvpe()**) usarão a variável de ambiente **PATH** para localizar o programa *file*. Quando o ambiente está sendo substituído (usando uma das variantes **spawn*e**, discutidas no próximo parágrafo), o novo ambiente é usado como fonte da variável **PATH**. As outras variantes, **spawnl()**, **spawnle()**, **spawnv()** e **spawnve()**, não usarão a variável **PATH** para localizar o executável; *path* deve conter um caminho absoluto ou relativo apropriado.

Para **spawnle()**, **spawnlpe()**, **spawnve()** e **spawnvpe()** (observe que todos eles terminam em "e"), o parâmetro *env* deve ser um mapeamento que é usado para definir as variáveis de ambiente para o novo processo (elas são usadas no lugar do ambiente do processo atual); as funções **spawnl()**, **spawnlp()**, **spawnv()** e **spawnvp()** fazem com que o novo processo herde o ambiente do processo atual. Note que as chaves e os valores no dicionário *env* devem ser strings; chaves ou valores inválidos farão com que a função falhe, com um valor de retorno de `127`.

Como exemplo, as seguintes chamadas a **spawnlp()** e **spawnvpe()** são equivalentes:

```
import os
os.spawnlp(os.P_WAIT, 'cp', 'cp', 'index.html', '/dev/null')

L = ['cp', 'index.html', '/dev/null']
os.spawnvpe(os.P_WAIT, 'cp', L, os.environ)
```

Levanta um evento de auditoria `os.spawn` com os argumentos `mode`, `path`, `args` e `env`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

spawnlp(), **spawnlpe()**, **spawnvp()** e **spawnvpe()** não estão disponíveis no Windows. **spawnle()** e **spawnve()** não são seguros para thread no Windows; recomendamos que você use o módulo **subprocess**.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Suavemente descontinuado desde a versão 3.14: The **subprocess** module is recommended instead.

os.P_NOWAIT
os.P_NOWAITO

Valores possíveis para o parâmetro *mode* para a família de funções **spawn***. Se qualquer um desses valores for fornecido, as funções **spawn*** retornarão assim que o novo processo for criado, com o id do processo como o valor de retorno.

Disponibilidade: Unix, Windows.

os.P_WAIT

Valor possível para o parâmetro *mode* para a família de funções **spawn***. Se for fornecido como *mode*, as funções **spawn*** não retornarão até que o novo processo seja executado até a conclusão e retornará o código de saída do processo em que a execução foi bem-sucedida, ou `-signal` se um sinal interromper o processo.

Disponibilidade: Unix, Windows.

os.P_DETACH
os.P_OVERLAY

Valores possíveis para o parâmetro *mode* para a família de funções **spawn***. Eles são menos portáveis do que os listados acima. **P_DETACH** é semelhante a **P_NOWAIT**, mas o novo processo é separado do console do processo de chamada. Se **P_OVERLAY** for usado, o processo atual será substituído; a função **spawn*** não retornará.

Disponibilidade: Windows.

os.startfile(*path*[, *operation*][, *arguments*][, *cwd*][, *show_cmd*])

Inicia um arquivo com sua aplicação associada.

Quando *operation* não é especificado, isso atua como um clique duplo no arquivo no Windows Explorer, ou como fornecer o nome do arquivo como um argumento para o comando **start** do console interativo de comandos: o arquivo é aberto com qualquer aplicação (se houver) com a extensão associada.

Quando outra *operation* é fornecida, ela deve ser um "verbo de comando" que especifica o que deve ser feito com o arquivo. Verbos comuns documentados pela Microsoft são `'open'`, `'print'` e `'edit'` (para serem usados em arquivos), bem como `'explore'` e `'find'` (para serem usados em diretórios).

Ao iniciar uma aplicação, especifique *arguments* a serem passados como uma única string. Este argumento pode não ter efeito quando esta função é usada para iniciar um documento.

O diretório de trabalho padrão é herdado, mas pode ser substituído pelo argumento *cwd*. Este deve ser um caminho absoluto. Um *path* relativo será resolvido levando em consideração este argumento.

Use *show_cmd* para substituir o estilo de janela padrão. Se isso terá algum efeito irá depender da aplicação sendo iniciada. Os valores são inteiros conforme implementado pela função Win32 **`ShellExecute()`**.

`startfile()` retorna assim que a aplicação associada é iniciada. Não há opção de aguardar o fechamento da aplicação, e nenhuma maneira de recuperar o status de saída da aplicação. O parâmetro *path* é relativo ao diretório atual ou ao *cwd*. Se você quiser usar um caminho absoluto, certifique-se que o primeiro caractere não seja uma barra (`'/'`). Use **`pathlib`** ou a função **`os.path.normpath()`** para garantir que os caminhos sejam codificados corretamente para Win32.

Para reduzir a sobrecarga de inicialização do interpretador, a função Win32 **`ShellExecute()`** não é resolvida até que esta função seja chamada pela primeira vez. Se a função não puder ser resolvida, uma exceção **`NotImplementedError`** será levantada.

Levanta um evento de auditoria `os.startfile` com os argumentos `path` e `operation`.

Levanta um evento de auditoria `os.startfile/2` com os argumentos `path`, `operation`, `arguments`, `cwd` e `show_cmd`.

Disponibilidade: Windows.

Alterado na versão 3.10: Adicionados os argumentos *arguments*, *cwd* e *show_cmd*, e o evento de auditoria `os.startfile/2`.

os.system(*command*)

Executa o comando (uma string) em um subshell. Isso é implementado chamando a função C padrão **`system()`**, e tem as mesmas limitações. Alterações em **`sys.stdin`** etc. não são refletidas no ambiente do comando executado. Se *command* gerar qualquer saída, ela será enviada ao fluxo de saída padrão do interpretador. O padrão C não especifica o significado do valor de retorno da função C, então o valor de retorno da função Python depende do sistema.

No Unix, o valor de retorno é o status de saída do processo codificado no formato especificado para **`wait()`**.

No Windows, o valor de retorno é aquele retornado pelo shell do sistema após a execução de *command*. O shell é fornecido pela variável de ambiente Windows **`COMSPEC`**: normalmente é **cmd.exe**, que retorna o status de saída da execução do comando; em sistemas que usam um shell não nativo, consulte a documentação do shell.

O módulo **`subprocess`** fornece recursos mais poderosos para gerar novos processos e recuperar seus resultados; usar esse módulo é recomendado a usar esta função. Veja a seção Replacing Older Functions with the subprocess Module na documentação do **`subprocess`** para algumas receitas úteis.

No Unix, **`waitstatus_to_exitcode()`** pode ser usada para converter o resultado (status de saída) em um código de saída. No Windows, o resultado é diretamente o código de saída.

Levanta um evento de auditoria `os.system` com o argumento `command`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

os.times()

Retorna os tempos do processo global atual. O valor de retorno é um objeto com cinco atributos:

- **`user`** - tempo do usuário
- **`system`** - tempo do sistema
- **`children_user`** - tempo do usuário de todos os processos filhos
- **`children_system`** - tempo do sistema de todos os processos filhos
- **`elapsed`** - tempo real decorrido desde um ponto fixo no passado

Para compatibilidade com versões anteriores, esse objeto também se comporta como uma tupla de 5 elementos contendo **`user`**, **`system`**, **`children_user`**, **`children_system`** e **`elapsed`** nessa ordem.

Consulte as páginas de manual *times(2)* *[https://manpages.debian.org/times(2)]* e times(3) [https://man.freebsd.org/cgi/man.cgi?time(3)] no Unix ou o GetProcessTimes MSDN [https://docs.microsoft.com/windows/win32/api/processthreadsapi/nf-processthreadsapi-getprocesstimes] no Windows. No Windows, apenas **`user`** e **`system`** são conhecidos; os outros atributos são zero.

Disponibilidade: Unix, Windows.

Alterado na versão 3.3: Tipo de retorno foi alterado de uma tupla para um objeto tupla ou similar com atributos nomeados.

os.wait()

Aguarda a conclusão de um processo filho e retorna uma tupla contendo seu pid e indicação de status de saída: um número de 16 bits, cujo byte baixo é o número do sinal que interrompeu o processo e cujo byte alto é o status de saída (se o número do sinal é zero); o bit alto do byte baixo é definido se um arquivo principal foi produzido.

Se não houver filhos que possam ser aguardados, uma exceção `ChildProcessError` é levantada.

`waitstatus_to_exitcode()` pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Ver também

As outras funções `wait*()` documentadas abaixo podem ser usadas para aguardar a conclusão de um processo filho específico e ter mais opções. `waitpid()` é o único também disponível no Windows.

os.waitid(*idtype*, *id*, *options*, /)

Aguarda a conclusão de um processo filho.

idtype pode ser `P_PID`, `P_PGID`, `P_ALL` ou (no Linux) `P_PIDFD`. A interpretação de *id* depende disso; veja suas descrições individuais.

options é uma combinação OU de sinalizadores. Pelo menos um entre `WEXITED`, `WSTOPPED` ou `WCONTINUED` é obrigatório; `WNOHANG` e `WNOWAIT` são sinalizadores opcionais adicionais.

O valor de retorno é um objeto que representa os dados contidos na estrutura `siginfo_t` com os seguintes atributos:

- `si_pid` (ID do processo)
- `si_uid` (ID de usuário real do filho)
- `si_signo` (sempre `SIGCHLD`)
- `si_status` (o status de saída ou número do sinal, dependendo de `si_code`)
- `si_code` (veja `CLD_EXITED` para possíveis valores)

Se `WNOHANG` for especificado e não houver filhos correspondentes no estado solicitado, `None` será retornado. Caso contrário, se não houver filhos correspondentes que possam ser esperados, `ChildProcessError` será levantada.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

Alterado na versão 3.13: Esta função agora também está disponível no MacOS.

os.waitpid(*pid*, *options*, /)

Os detalhes desta função diferem no Unix e no Windows.

No Unix: aguarda a conclusão de um processo filho dado pelo id de processo *pid* e retorna uma tupla contendo seu id de processo e indicação de status de saída (codificado como para `wait()`). A semântica da chamada é afetada pelo valor do inteiro *options*, que deve ser `0` para operação normal.

Se *pid* for maior que `0`, `waitpid()` solicita informações de status para aquele processo específico. Se *pid* for `0`, a solicitação é para o status de qualquer filho no grupo de processos do processo atual. Se *pid* for `-1`, a solicitação pertence a qualquer filho do processo atual. Se *pid* for menor que `-1`, o status é solicitado para qualquer processo no grupo de processos `-pid` (o valor absoluto de *pid*).

options é uma combinação OU de sinalizadores. Se contiver `WNOHANG` e não houver filhos correspondentes no estado solicitado, `(0, 0)` será retornado. Caso contrário, se não houver filhos correspondentes que possam ser esperados, `ChildProcessError` será levantada. Outras opções que podem ser usadas são `WUNTRACED` e `WCONTINUED`.

No Windows: aguarda a conclusão de um processo fornecido pelo identificador de processo *pid* e retorna uma tupla contendo *pid*, e seu status de saída deslocado 8 bits para a esquerda (o deslocamento torna o uso da função em várias plataformas mais fácil). Um *pid* menor ou igual a `0` não tem nenhum significado especial no Windows e levanta uma exceção. O valor de inteiros *options* não tem efeito. *pid* pode se referir a qualquer processo cujo id é conhecido, não necessariamente um processo filho. As funções `spawn*` chamadas com `P_NOWAIT` retornam manipuladores de processo adequados.

`waitstatus_to_exitcode()` pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

Alterado na versão 3.5: Se a chamada de sistema é interrompida e o tratador de sinal não levanta uma exceção, a função agora tenta novamente a chamada de sistema em vez de levantar uma exceção `InterruptedError` (consulte **PEP 475** [https://peps.python.org/pep-0475/] para entender a justificativa).

os.wait3(*options*)

Semelhante a `waitpid()`, exceto que nenhum argumento de id de processo é fornecido e uma tupla de 3 elementos contendo o id do processo filho, indicação de status de saída e informações de uso de recursos é retornada. Consulte `resource.getrusage()` para obter detalhes sobre as informações de uso de recursos. O argumento *option* é o mesmo fornecido para `waitpid()` e `wait4()`.

`waitstatus_to_exitcode()` pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.wait4(*pid*, *options*)

Semelhante a `waitpid()`, exceto uma tupla de 3 elementos, contendo o id do processo filho, indicação de status de saída e informações de uso de recursos é retornada. Consulte `resource.getrusage()` para obter detalhes sobre as informações de uso de recursos. Os argumentos para `wait4()` são os mesmos que aqueles fornecidos a `waitpid()`.

`waitstatus_to_exitcode()` pode ser usado para converter o status de saída em um código de saída.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.P_PID
os.P_PGID

os.P_ALL
os.P_PIDFD

Estes são os valores possíveis para *idtype* em `waitid()`. Eles afetam como *id* é interpretado:

- **`P_PID`** - espera pelo filho cujo PID é *id*.
- **`P_PGID`** - espera por qualquer filho cujo ID do grupo de progresso seja *id*.
- **`P_ALL`** - espera por qualquer filho; *id* é ignorado.
- **`P_PIDFD`** - espera pelo filho identificado pelo descritor de arquivo *id* (um descritor de arquivo de processo criado com `pidfd_open()`).

Disponibilidade: Unix, not WASI, not Android, not iOS.

Nota

`P_PIDFD` só está disponível no Linux >= 5.4.

Adicionado na versão 3.3.

Adicionado na versão 3.9: A constante **`P_PIDFD`**.

os.WCONTINUED

Este sinalizador *options* para `waitpid()`, `wait3()`, `wait4()` e `waitid()` faz com que os processos filhos sejam reportados se eles tiverem continuado de uma parada de controle de trabalho desde que foram reportados pela última vez.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WEXITED

Este sinalizador *options* para `waitid()` faz com que os processos filhos que terminaram sejam reportados.

As outras funções `wait*` sempre reportam filhos que terminaram, então esta opção não está disponível para eles.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

os.WSTOPPED

Este sinalizador *options* para **`waitid()`** faz com que os processos filhos que foram parados pela entrega de um sinal sejam reportados.

Esta opção não está disponível para as outras funções `wait*`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

os.WUNTRACED

Este sinalizador de *options* para **`waitpid()`**, **`wait3()`** e **`wait4()`** faz com que os processos filho também sejam reportados se eles foram interrompidos, mas seu estado atual não foi reportado desde que foram parados.

Esta opção não está disponível para **`waitid()`**.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WNOHANG

Este sinalizador *options* faz com que **`waitpid()`**, **`wait3()`**, **`wait4()`** e **`waitid()`** retornem imediatamente se nenhum status de processo filho estiver disponível imediatamente.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WNOWAIT

Este sinalizador *options* faz com que **`waitid()`** deixe o filho em um estado de espera, para que uma chamada **`wait*()`** posterior possa ser usada para recuperar as informações de status do filho novamente.

Esta opção não está disponível para as outras funções `wait*`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.CLD_EXITED
os.CLD_KILLED
os.CLD_DUMPED
os.CLD_TRAPPED
os.CLD_STOPPED
os.CLD_CONTINUED

Estes são os valores possíveis para **`si_code`** no resultado retornado por **`waitid()`**.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Adicionado na versão 3.3.

Alterado na versão 3.9: Adicionados os valores **`CLD_KILLED`** e **`CLD_STOPPED`**.

os.waitstatus_to_exitcode(*status*)

Converte um status de espera em um código de saída.

No Unix:

- Se o processo saiu normalmente (se `WIFEXITED(status)` for verdadeiro), retorna o status de saída do processo (retorna `WEXITSTATUS(status)`): resultado maior ou igual a 0.
- Se o processo foi encerrado por um sinal (se `WIFSIGNALED(status)` for verdadeiro), retorna `-signum` onde *signum* é o número do sinal que causou o encerramento do processo (retorna `-WTERMSIG(status)`): resultado menor que 0.
- Do contrário, levanta uma exceção **`ValueError`**.

No Windows, retorna *status* deslocado para a direita em 8 bits.

No Unix, se o processo está sendo rastreado ou se **`waitpid()`** foi chamado com a opção **`WUNTRACED`**, o chamador deve primeiro verificar se `WIFSTOPPED(status)` é verdadeiro. Esta função não deve ser chamada se `WIFSTOPPED(status)` for verdadeiro.

Ver também

As funções `WIFEXITED()`, `WEXITSTATUS()`, `WIFSIGNALED()`, `WTERMSIG()`, `WIFSTOPPED()`, `WSTOPSIG()`.

Disponibilidade: Unix, Windows, not WASI, not Android, not iOS.

Adicionado na versão 3.9.

As funções a seguir recebem um código de status do processo conforme retornado por `system()`, `wait()` ou `waitpid()` como parâmetro. Eles podem ser usados para determinar a disposição de um processo.

os.WCOREDUMP(*status*, /)

Retorna `True` se um despejo de memória *(core dump)* foi gerado para o processo; caso contrário, retorna `False`.

Esta função deve ser empregada apenas se `WIFSIGNALED()` for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WIFCONTINUED(*status*)

Retorna `True` se um filho interrompido foi retomado pela entrega de `SIGCONT` (se o processo foi continuado de uma parada de controle de trabalho); caso contrário, retorna `False`.

Veja a opção `WCONTINUED`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WIFSTOPPED(*status*)

Retorna `True` se o processo foi interrompido pela entrega de um sinal; caso contrário, retorna `False`.

`WIFSTOPPED()` só retorna `True` se a chamada de `waitpid()` foi feita usando a opção `WUNTRACED` ou quando o processo está sendo rastreado

(veja *ptrace(2)* *[https://manpages.debian.org/ptrace(2)]*).

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WIFSIGNALED(*status*)

Retorna `True` se o processo foi encerrado por um sinal; caso contrário, retorna `False`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WIFEXITED(*status*)

Retorna `True` se o processo foi encerrado normalmente, isto é, chamando `exit()` ou `_exit()`, ou retornando de `main()`; caso contrário, retorna `False`.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WEXITSTATUS(*status*)

Retorna o status de saída do processo.

Esta função deve ser empregada apenas se **`WIFEXITED()`** for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WSTOPSIG(*status*)

Retorna o sinal que causou a interrupção do processo.

Esta função deve ser empregada apenas se **`WIFSTOPPED()`** for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

os.WTERMSIG(*status*)

Retorna o número do sinal que causou o encerramento do processo.

Esta função deve ser empregada apenas se **`WIFSIGNALED()`** for verdadeira.

Disponibilidade: Unix, not WASI, not Android, not iOS.

Interface do agendador

Essas funções controlam como o tempo de CPU de um processo é alocado pelo sistema operacional. Eles estão disponíveis apenas em algumas plataformas Unix. Para informações mais detalhadas, consulte suas páginas man do Unix.

Adicionado na versão 3.3.

As políticas de agendamento a seguir serão expostas se houver suporte pelo sistema operacional.

os.SCHED_OTHER
: A política de agendamento padrão.

os.SCHED_BATCH
: Política de agendamento para processos com uso intensivo de CPU que tenta preservar a interatividade no resto do computador.

os.SCHED_DEADLINE
: Política de agendamento para tarefas com restrições de prazo.

Adicionado na versão 3.14.

os.SCHED_IDLE
: Política de agendamento para tarefas em segundo plano de prioridade extremamente baixa.

os.SCHED_NORMAL
: Apelido para `SCHED_OTHER`.

Adicionado na versão 3.14.

os.SCHED_SPORADIC
: Política de agendamento para programas de servidor esporádicos.

os.SCHED_FIFO

Uma política de agendamento Primeiro a Entrar, Primeiro a Sair (First In First Out - FIFO).

os.SCHED_RR

Uma política de agendamento round-robin.

os.SCHED_RESET_ON_FORK

Este sinalizador pode ser operado em OU com qualquer outra política de agendamento. Quando um processo com este sinalizador definido bifurca, a política de agendamento e a prioridade de seu filho são redefinidas para o padrão.

class os.sched_param(*sched_priority*)

Esta classe representa parâmetros de agendamento ajustáveis usados em `sched_setparam()`, `sched_setscheduler()` e `sched_getparam()`. É imutável.

Neste momento, há somente um único parâmetro possível:

sched_priority

A prioridade de agendamento para uma política de agendamento.

os.sched_get_priority_min(*policy*)

Obtém o valor mínimo de prioridade para *policy*. *policy* é uma das constantes de política de agendamento acima.

os.sched_get_priority_max(*policy*)

Obtém o valor máximo de prioridade para *policy*. *policy* é uma das constantes de política de agendamento acima.

os.sched_setscheduler(*pid*, *policy*, *param*, /)

Define a política de agendamento para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada. *policy* é uma das constantes de política de agendamento acima. *param* é uma instância de `sched_param`.

os.sched_getscheduler(*pid*, /)

Retorna a política de agendamento para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada. O resultado é uma das constantes de política de agendamento acima.

os.sched_setparam(*pid*, *param*, /)

Define os parâmetros de agendamento para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada. *param* é uma instância de `sched_param`.

os.sched_getparam(*pid*, /)

Retorna os parâmetros de agendamento como uma instância de `sched_param` para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada.

os.sched_rr_get_interval(*pid*, /)

Retorna o quantum round-robin em segundos para o processo com PID *pid*. Um *pid* de 0 significa o processo de chamada.

os.sched_yield()

Abandona voluntariamente a CPU. Veja *sched_yield(2)* *[https://manpages.debian.org/sched_yield(2)]* para detalhes.

os.sched_setaffinity(*pid*, *mask*, /)

Restringe o processo com PID *pid* (ou o processo atual se zero) para um conjunto de CPUs. *mask* é um iterável de inteiros que representam o conjunto de CPUs às quais o processo deve ser restrito.

os.sched_getaffinity(*pid*, /)

Retorna o conjunto de CPUs ao qual o processo com PID *pid* está restrito.

Se *pid* for zero, retorna o conjunto de CPUs às quais a thread chamadora do processo atual está restrita.

Veja também a função `process_cpu_count()`.

Diversas informações de sistema

os.confstr(*name*, /)

Retorna valores de configuração do sistema com valor de string. *name* especifica o valor de configuração a ser recuperado; pode ser uma string que é o nome de um valor de sistema definido; esses nomes são especificados em vários padrões (POSIX, Unix 95, Unix 98 e outros). Algumas plataformas também definem nomes adicionais. Os nomes conhecidos pelo sistema operacional são fornecidos como as chaves do dicionário `confstr_names`. Para variáveis de configuração não incluídas nesse mapeamento, passar um número inteiro para *name* também é aceito.

Se o valor de configuração especificado por *name* não for definido, retorna `None`.

Se *name* for uma string e não for conhecida, **`ValueError`** é levantada. Se um valor específico para *name* não for compatível com o sistema operacional do computador, mesmo que seja incluído em `confstr_names`, uma **`OSError`** é levantada com **`errno.EINVAL`** como número do erro.

Disponibilidade: Unix.

os.confstr_names

Nomes de mapeamento de dicionário aceitos por **`confstr()`** para os valores inteiros definidos para esses nomes pelo sistema operacional do computador. Isso pode ser usado para determinar o conjunto de nomes conhecidos pelo sistema.

Disponibilidade: Unix.

os.cpu_count()

Retorna o número de CPUs lógicas do **sistema**. Retorna `None` se não determinado.

A função `process_cpu_count()` pode ser usada para obter o número de CPUs lógicas utilizáveis pela thread de chamada do **processo atual**.

Adicionado na versão 3.4.

Alterado na versão 3.13: If `-X cpu_count` is given or `PYTHON_CPU_COUNT` is set, `cpu_count()` returns the override value *n*.

os.getloadavg()

Retorna o número de processos na fila de execução do sistema em média nos últimos 1, 5 e 15 minutos ou levanta `OSError` se a média de carga não foi obtida.

Disponibilidade: Unix.

os.process_cpu_count()

Obtém o número de CPUs lógicas utilizáveis pelo thread de chamada do **processo atual**. Retorna `None` se indeterminado. Pode ser menor que `cpu_count()` dependendo da afinidade da CPU.

A função `cpu_count()` pode ser usada para obter o número de CPUs lógicas no **sistema**.

If `-X cpu_count` is given or `PYTHON_CPU_COUNT` is set, `process_cpu_count()` returns the override value *n*.

Veja também a função `sched_getaffinity()`.

Adicionado na versão 3.13.

os.sysconf(*name*, /)

Retorna valores de configuração do sistema com valor inteiro. Se o valor de configuração especificado por *name* não estiver definido, `-1` é retornado. Os comentários sobre o parâmetro *name* para `confstr()` se aplicam aqui também; o dicionário que fornece informações sobre os nomes conhecidos é fornecido por `sysconf_names`.

Disponibilidade: Unix.

os.sysconf_names

Nomes de mapeamento de dicionário aceitos por `sysconf()` para os valores inteiros definidos para esses nomes pelo sistema operacional. Isso pode ser usado para determinar o conjunto de nomes conhecidos pelo sistema.

Disponibilidade: Unix.

Alterado na versão 3.11: Adiciona o nome `'SC_MINSIGSTKSZ'`.

Os dados a seguir são usados para operações de manipulação de caminhos de arquivos e diretórios. Estão definidos e disponíveis para todas as plataformas.

Operações de nível mais alto em nomes de caminho são definidos no módulo `os.path`.

os.curdir

A string constante usada pelo sistema operacional para se referir ao diretório atual. Isso é `'.'` para Windows e POSIX. Também disponível via `os.path`.

os.pardir

A string constante usada pelo sistema operacional para se referir ao diretório pai. Isso é `'..'` para Windows e POSIX. Também disponível via `os.path`.

os.sep

O caractere usado pelo sistema operacional para separar os componentes do nome do caminho. Este é `'/'` para POSIX e `'\\'` para Windows. Observe que saber disso não é suficiente para ser capaz de analisar ou concatenar nomes de caminho – use `os.path.split()` e `os.path.join()` – mas ocasionalmente é útil. Também disponível via `os.path`.

os.altsep

Um caractere alternativo usado pelo sistema operacional para separar os componentes do nome de caminho, ou `None` se apenas um caractere separador existir. Isso é definido como `'/'` em sistemas Windows onde `sep` é uma contrabarra. Também disponível via **`os.path`**.

os.extsep

O caractere que separa o nome do arquivo base da extensão; por exemplo, o `'.'` em `os.py`. Também disponível via **`os.path`**.

os.pathsep

O caractere convencionalmente usado pelo sistema operacional para separar os componentes do caminho de pesquisa (como em **`PATH`**), como `':'` para POSIX ou `';'` para Windows. Também disponível via **`os.path`**.

os.defpath

O caminho de pesquisa padrão usado por **`exec*p*`** e **`spawn*p*`** se o ambiente não tiver uma chave `'PATH'`. Também disponível via **`os.path`**.

os.linesep

A string usada para separar (ou melhor, encerrar) linhas na plataforma atual. Pode ser um único caractere, como `'\n'` para POSIX, ou múltiplos caracteres, por exemplo, `'\r\n'` para Windows. Não use *os.linesep* como terminador de linha ao escrever arquivos abertos em modo de texto (o padrão); use um único `'\n'` ao invés, em todas as plataformas.

os.devnull

O caminho do arquivo do dispositivo nulo. Por exemplo: `'/dev/null'` para POSIX, `'nul'` para Windows. Também disponível via **`os.path`**.

os.RTLD_LAZY
os.RTLD_NOW
os.RTLD_GLOBAL
os.RTLD_LOCAL
os.RTLD_NODELETE
os.RTLD_NOLOAD

os.RTLD_DEEPBIND

Sinalizadores para uso com as funções **`setdlopenflags()`** e **`getdlopenflags()`**. Veja a página man do Unix *dlopen(3)* *[https://manpages.debian.org/dlopen(3)]* para saber o que significam os diferentes sinalizadores.

Adicionado na versão 3.3.

Números aleatórios

os.getrandom(*size*, *flags=0*)

Obtém até *size* bytes aleatórios. Esta função pode retornar menos bytes que a quantia requisitada.

Esses bytes podem ser usados para propagar geradores de número aleatório no espaço do usuário ou para fins criptográficos.

`getrandom()` depende da entropia obtida de drivers de dispositivos e outras fontes de ruído ambiental. A leitura desnecessária de grandes quantidades de dados terá um impacto negativo sobre outros usuários dos dispositivos `/dev/random` e `/dev/urandom`.

O argumento *flags* é uma máscara de bits que pode conter zero ou mais dos seguintes valores operados com OU: **`os.GRND_RANDOM`** e **`GRND_NONBLOCK`**.

Veja também a página de manual do getrandom() [https://man7.org/linux/man-pages/man2/getrandom.2.html] do Linux.

Disponibilidade: Linux >= 3.17.

Adicionado na versão 3.6.

os.urandom(*size*, /)

Retorna uma sequência de bytes de tamanho *size* aleatórios próprios para uso criptográfico.

Esta função retorna bytes aleatórios de uma fonte de aleatoriedade específica do sistema operacional. Os dados retornados devem ser imprevisíveis o suficiente para aplicações criptográficos, embora sua qualidade exata dependa da implementação do sistema operacional.

No Linux, se a chamada de sistema `getrandom()` estiver disponível, ela é usada no modo bloqueante: bloqueia até que o pool de entropia urandom do sistema seja inicializado (128 bits de entropia são coletados pelo kernel). Veja a **PEP 524** [https://peps.python.org/pep-0524/] para a justificativa. No Linux, a função **`getrandom()`** pode ser usada para obter bytes aleatórios no modo não bloqueante (usando a sinalização **`GRND_NONBLOCK`**) ou para pesquisar até que o pool de entropia urandom do sistema seja inicializado.

Em um sistema semelhante ao Unix, bytes aleatórios são lidos do dispositivo `/dev/urandom`. Se o dispositivo `/dev/urandom` não estiver disponível ou não for legível, a exceção **`NotImplementedError`** é levantada.

No Windows, ainda vai usar `BCryptGenRandom()`.

Ver também

O módulo **`secrets`** fornece funções de nível mais alto. Para uma interface fácil de usar para o gerador de números aleatórios fornecido por sua plataforma, consulte **`random.SystemRandom`**.

Alterado na versão 3.5: No Linux 3.17 e mais recente, a chamada de sistema `getrandom()` agora é usada quando disponível. No OpenBSD 5.6 e mais recentes, a função C `getentropy()` agora é usada. Essas funções evitam o uso de um descritor de arquivo interno.

Alterado na versão 3.5.2: No Linux, se a chamada de sistema `getrandom()` bloqueia (o pool de entropia urandom ainda não foi inicializado), recorre à leitura `/dev/urandom`.

Alterado na versão 3.6: No Linux, `getrandom()` é usado agora no modo de bloqueio para aumentar a segurança.

Alterado na versão 3.11: No Windows, `BCryptGenRandom()` é usado em vez de `CryptGenRandom()`, o qual foi descontinuado.

os.GRND_NONBLOCK

Por padrão, ao ler de `/dev/random`, **`getrandom()`** bloqueia se nenhum byte aleatório estiver disponível, e ao ler de `/dev/urandom`, ele bloqueia se o pool de entropia ainda não foi inicializado.

Se o sinalizador **`GRND_NONBLOCK`** estiver definido, então **`getrandom()`** não bloqueia nesses casos, mas, em vez disso, levanta **`BlockingIOError`** imediatamente.

Adicionado na versão 3.6.

os.GRND_RANDOM

Se este bit é definido os bytes aleatórios são sorteados a partir de `/dev/random` ao invés de `/dev/urandom`.

Adicionado na versão 3.6.

io — Ferramentas essenciais para trabalhar com fluxos

Código-fonte: Lib/io.py [https://github.com/python/cpython/tree/3.15/Lib/io.py]

Visão Geral

O módulo **io** fornece os principais recursos do Python para lidar com vários tipos de E/S. Existem três tipos principais de E/S: *E/S de texto*, *E/S binária* e *E/S bruta*. Essas são categorias genéricas, e vários repositórios de apoio podem ser usados para cada uma delas. Um objeto concreto pertencente a qualquer uma dessas categorias é chamado de objeto arquivo. Outros termos comuns são *fluxo* e *objeto arquivo ou similar*.

Independentemente de sua categoria, cada objeto de fluxo concreto também terá vários recursos: pode ser somente leitura, somente escrita ou leitura e escrita. Também pode permitir acesso aleatório arbitrário (buscando para frente ou para trás em qualquer local) ou apenas acesso sequencial (por exemplo, no caso de um soquete ou encadeamento).

Todos os fluxos são cuidadosos com o tipo de dados que você fornece a eles. Por exemplo, atribuir um objeto **str** ao método **write()** de um fluxo binário levantará uma **TypeError**. O mesmo ocorrerá com um objeto **bytes** ao método **write()** de um fluxo de texto.

Alterado na versão 3.3: Operações que costumavam levantar **IOError** agora levantam **OSError**, já que **IOError** agora é um apelido de **OSError**.

E/S de texto

A E/S de texto espera e produz objetos **str**. Isso significa que sempre que o armazenamento de apoio for composto nativamente por bytes (como no

caso de um arquivo), a codificação e a decodificação dos dados são feitas de forma transparente, bem como a tradução opcional de caracteres de quebra de linha específicos da plataforma.

A maneira mais fácil de criar um fluxo de texto é com **`open()`**, especificando opcionalmente uma codificação:

```
f = open("myfile.txt", "r", encoding="utf-8")
```

Os fluxos de texto na memória também estão disponíveis como objetos **`StringIO`**:

```
f = io.StringIO("alguns dados em texto iniciais")
```

Nota

Ao trabalhar com um fluxo não bloqueante, esteja ciente de que operações de leitura em objetos de E/S de texto podem levantar uma **`BlockingIOError`** se o fluxo não puder executar a operação imediatamente.

A API de fluxo de texto é descrita em detalhes na documentação de **`TextIOBase`**.

E/S binária

E/S binária (também chamada de *E/S com buffer*) espera objeto bytes ou similar e produz objetos **`bytes`**. Nenhuma codificação, decodificação ou tradução de quebra de linha é realizada. Esta categoria de fluxos pode ser usada para todos os tipos de dados não textuais e também quando se deseja controle manual sobre o tratamento de dados textuais.

A maneira mais fácil de criar um fluxo binário é com **`open()`** com `'b'` na string de modo:

```
f = open("meuarquivo.jpg", "rb")
```

Os fluxos binários na memória também estão disponíveis como objetos `BytesIO`:

```
f = io.BytesIO(b"alguns dados bin\xe1rios iniciais: \x00\x01")
```

A API de fluxo binário é descrita em detalhes na documentação de `BufferedIOBase`.

Outros módulos de biblioteca podem fornecer maneiras adicionais de criar fluxos de texto ou binários. Veja `socket.socket.makefile()` como exemplo.

E/S bruta

A E/S bruta (também chamada de *E/S sem buffer*) é geralmente usada como um bloco de construção de baixo nível para fluxos binários e de texto; raramente é útil manipular diretamente um fluxo bruto a partir do código do usuário. No entanto, você pode criar um fluxo bruto abrindo um arquivo em modo binário com o buffer desabilitado:

```
f = open("meuarquivo.jpg", "rb", buffering=0)
```

A API de fluxo bruto é descrita em detalhes na documentação de `RawIOBase`.

Aviso

Raw I/O (em português, E/S bruta) é uma interface de baixo nível; geralmente, é necessário verificar os valores de retorno dos métodos e realizar novas tentativas explicitamente para garantir a conclusão de uma operação. Por exemplo, o método `write()` retorna o número de bytes gravados, o qual pode ser inferior ao número de bytes fornecidos (uma gravação parcial). Objetos de E/S de alto nível, como E/S binária e E/S de texto, implementam esse comportamento de nova tentativa.

Codificação do texto

A codificação padrão de `TextIOWrapper` e `open()` é UTF-8. Se o Modo UTF-8 Python está desabilitado. a codificação padrão é específica da localidade (`locale.getencoding()`).

Ver também

Modo UTF-8 do Python
O Modo UTF-8 do Python ignora a codificação da localidade e força o uso do UTF-8.

PEP 686 **[https://peps.python.org/pep-0686/]**
Python 3.15 tornou Modo UTF-8 do Python o padrão.

Ativar EncodingWarning

Adicionado na versão 3.10: Veja a **PEP 597** [https://peps.python.org/pep-0597/] para mais detalhes.

Para descobrir onde a codificação padrão é usada, você pode habilitar a opção de linha de comando `-X warn_default_encoding` ou definir a variável de ambiente `PYTHONWARNDEFAULTENCODING`, que emitirá um `EncodingWarning` quando a codificação padrão for usada.

Se você estiver fornecendo uma API que usa `open()` ou `TextIOWrapper` e passa `encoding=None` como parâmetro, você pode usar `text_encoding()` para que os chamadores da API emitam um `EncodingWarning` se não passarem um `encoding`. No entanto, considere usar UTF-8 por padrão (ou seja, `encoding="utf-8"`) para novas APIs.

Interface de alto nível do módulo

io.DEFAULT_BUFFER_SIZE
Um inteiro contendo o tamanho padrão do buffer usado pelas classes de E/S com buffer do módulo. `open()` usa o tamanho do bloco (blksize) do arquivo (obtido por `os.stat()`), se possível.

io.open(*file*, *mode='r'*, *buffering=-1*, *encoding=None*, *errors=None*, *newline=None*, *closefd=True*, *opener=None*)

Este é um apelido para a função embutida **`open()`**.

Esta função levanta um evento de auditoria `open` com os argumentos *path*, *mode* e *flags*. Os argumentos *mode* e *flags* podem ter sido modificados ou inferidos da chamada original.

io.open_code(*path*)

Abre o arquivo fornecido no modo `'rb'`. Esta função deve ser utilizada quando o objetivo é tratar o conteúdo como código executável.

path deve ser uma **`str`** e um caminho absoluto.

O comportamento desta função pode ser substituído por uma chamada anterior a **`PyFile_SetOpenCodeHook()`**. No entanto, presumir que *path* seja uma **`str`** e um caminho absoluto, `open_code(path)` deve sempre se comportar da mesma maneira que `open(path, 'rb')`. A substituição do comportamento destina-se a validações ou pré-processamentos adicionais do arquivo.

Adicionado na versão 3.8.

io.text_encoding(*encoding*, *stacklevel=2*, */*)

Esta é uma função auxiliar para chamáveis que utilizam **`open()`** ou **`TextIOWrapper`** e possuem um parâmetro `encoding=None`.

This function returns *encoding* if it is not `None`. Otherwise, it returns `"locale"` or `"utf-8"` depending on UTF-8 Mode.

This function emits an **`EncodingWarning`** if **`sys.flags.warn_default_encoding`** is true and *encoding* is `None`. *stacklevel* specifies where the warning is emitted. For example:

```
def read_text(path, encoding=None):
    encoding = io.text_encoding(encoding)  # stacklevel=2
    with open(path, encoding) as f:
        return f.read()
```

In this example, an **`EncodingWarning`** is emitted for the caller of `read_text()`.

Veja Codificação do texto para mais informações.

Adicionado na versão 3.10.

Alterado na versão 3.11: **`text_encoding()`** returns “utf-8” when UTF-8 mode is enabled and *encoding* is `None`.

exception io.BlockingIOError

This is a compatibility alias for the builtin **`BlockingIOError`** exception.

exception io.UnsupportedOperation

An exception inheriting **`OSError`** and **`ValueError`** that is raised when an unsupported operation is called on a stream.

Ver também

`sys`

contains the standard IO streams: **`sys.stdin`**, **`sys.stdout`**, and **`sys.stderr`**.

hierarquia de classe

The implementation of I/O streams is organized as a hierarchy of classes. First abstract base classes (ABCs), which are used to specify the various categories of streams, then concrete classes providing the standard stream implementations.

Nota

The abstract base classes also provide default implementations of some methods in order to help implementation of concrete stream classes. For example, **`BufferedIOBase`** provides unoptimized implementations of **`readinto()`** and **`readline()`**.

At the top of the I/O hierarchy is the abstract base class `IOBase`. It defines the basic interface to a stream. Note, however, that there is no separation between reading and writing to streams; implementations are allowed to raise `UnsupportedOperation` if they do not support a given operation.

The `RawIOBase` ABC extends `IOBase`. It deals with the reading and writing of bytes to a stream. `FileIO` subclasses **`RawIOBase`** to provide an interface to files in the machine's file system.

The `BufferedIOBase` ABC extends `IOBase`. It deals with buffering on a raw binary stream (`RawIOBase`). Its subclasses, `BufferedWriter`, `BufferedReader`, and `BufferedRWPair` buffer raw binary streams that are writable, readable, and both readable and writable, respectively. `BufferedRandom` provides a buffered interface to seekable streams. Another **`BufferedIOBase`** subclass, `BytesIO`, is a stream of in-memory bytes.

The `TextIOBase` ABC extends `IOBase`. It deals with streams whose bytes represent text, and handles encoding and decoding to and from strings. `TextIOWrapper`, which extends **`TextIOBase`**, is a buffered text interface to a buffered raw stream (`BufferedIOBase`). Finally, `StringIO` is an in-memory stream for text.

Argument names are not part of the specification, and only the arguments of `open()` are intended to be used as keyword arguments.

The following table summarizes the ABCs provided by the **`io`** module:

ABC	Inherits	Stub Methods	Mixin Methods and Properties

ABC	Inherits	Stub Methods	Mixin Methods and Properties
IOBase		fileno, seek, and truncate	close, closed, __enter__, __exit__, flush, isatty, __iter__, __next__, readable, readline, readlines, seekable, tell, writable, and writelines
RawIOBase	IOBase	readinto and write	Inherited IOBase methods, read, and readall
BufferedIOBase	IOBase	detach, read, read1, and write	Inherited IOBase methods, readinto, and readinto1
TextIOBase	IOBase	detach, read, readline, and write	Inherited IOBase methods, encoding, errors, and newlines

I/O base classes

class io.IOBase

The abstract base class for all I/O classes.

This class provides empty abstract implementations for many methods that derived classes can override selectively; the default implementations represent a file that cannot be read, written or seeked.

Even though **IOBase** does not declare **read()** or **write()** because their signatures will vary, implementations and clients should consider those methods part of the interface. Also, implementations may raise a **ValueError** (or **UnsupportedOperation**) when operations they do not support are called.

The basic type used for binary data read from or written to a file is **bytes**. Other bytes-like objects are accepted as method arguments too. Text I/O classes work with **str** data.

Note that calling any method (even inquiries) on a closed stream is undefined. Implementations may raise **ValueError** in this case.

IOBase (and its subclasses) supports the iterator protocol, meaning that an **IOBase** object can be iterated over yielding the lines in a stream. Lines are defined slightly differently depending on whether the stream is a binary stream (yielding bytes), or a text stream (yielding character strings). See **readline()** below.

IOBase is also a context manager and therefore supports the **with** statement. In this example, *file* is closed after the **with** statement's suite is finished—even if an exception occurs:

```
with open('spam.txt', 'w') as file:
    file.write('Spam and eggs!')
```

IOBase provides these data attributes and methods:

close()

Flush and close this stream. This method has no effect if the file is already closed. Once the file is closed, any operation on the file (e.g. reading or writing) will raise a **ValueError**.

As a convenience, it is allowed to call this method more than once; only the first call, however, will have an effect.

closed

`True` if the stream is closed.

fileno()

Return the underlying file descriptor (an integer) of the stream if it exists. An `OSError` is raised if the IO object does not use a file descriptor.

flush()

Flush the write buffers of the stream if applicable. This does nothing for read-only and non-blocking streams.

isatty()

Return `True` if the stream is interactive (i.e., connected to a terminal/tty device).

readable()

Return `True` if the stream can be read from. If `False`, **`read()`** will raise `OSError`.

readline(*size=-1*, /)

Read and return one line from the stream. If *size* is specified, at most *size* bytes will be read.

The line terminator is always `b'\n'` for binary files; for text files, the *newline* argument to `open()` can be used to select the line terminator(s) recognized.

readlines(*hint=-1*, /)

Read and return a list of lines from the stream. *hint* can be specified to control the number of lines read: no more lines will be read if the total size (in bytes/characters) of all lines so far exceeds *hint*.

hint values of `0` or less, as well as `None`, are treated as no hint.

Note that it's already possible to iterate on file objects using `for line in file: ...` without calling **`file.readlines()`**.

seek(*offset*, *whence=os.SEEK_SET*, /)

Change the stream position to the given byte *offset*, interpreted relative to the position indicated by *whence*, and return the new absolute position. Values for *whence* are:

- `os.SEEK_SET` or `0` – start of the stream (the default); *offset* should be zero or positive
- `os.SEEK_CUR` or `1` – current stream position; *offset* may be negative
- `os.SEEK_END` or `2` – end of the stream; *offset* is usually negative

Adicionado na versão 3.1: The **`SEEK_*`** constants.

Adicionado na versão 3.3: Some operating systems could support additional values, like `os.SEEK_HOLE` or `os.SEEK_DATA`. The valid values for a file could depend on it being open in text or binary mode.

seekable()

Return `True` if the stream supports random access. If `False`, `seek()`, `tell()` and `truncate()` will raise `OSError`.

tell()

Return the current stream position.

truncate(*size=None*, /)

Resize the stream to the given *size* in bytes (or the current position if *size* is not specified). The current stream position isn't changed. This resizing can extend or reduce the current file size. In case of extension, the contents of the new file area depend on the platform (on most systems, additional bytes are zero-filled). The new file size is returned.

Alterado na versão 3.5: Windows will now zero-fill files when extending.

writable()

Return `True` if the stream supports writing. If `False`, **`write()`** and **`truncate()`** will raise **`OSError`**.

writelines(*lines*, /)

Write a list of lines to the stream. Line separators are not added, so it is usual for each of the lines provided to have a line separator at the end.

__del__()

Prepare for object destruction. **`IOBase`** provides a default implementation of this method that calls the instance's **`close()`** method.

class io.RawIOBase

Base class for raw binary streams. It inherits from **`IOBase`**.

Raw binary streams typically provide low-level access to an underlying OS device or API, and do not try to encapsulate it in high-level primitives (this functionality is done at a higher-level in buffered binary streams and text streams, described later in this page).

`RawIOBase` provides these methods in addition to those from **`IOBase`**:

read(*size=-1*, /)

Read up to *size* bytes from the object and return them. As a convenience, if *size* is unspecified or -1, all bytes until EOF are returned.

Attempts to make only one system call but will retry if interrupted and the signal handler does not raise an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale). This means fewer than *size* bytes may be returned if the operating system call returns fewer than *size* bytes.

If 0 bytes are returned, and *size* was not 0, this indicates end of file. If the object is in non-blocking mode and no bytes are available, `None` is returned.

The default implementation defers to **`readall()`** and **`readinto()`**.

readall()

Read and return all the bytes from the stream until EOF, using multiple calls to the stream if necessary.

If `0` bytes are returned this indicates end of file. If the object is in non-blocking mode and the underlying **`read()`** returns `None` indicating no bytes are available, `None` is returned.

readinto(*b*, /)

Read bytes into a pre-allocated, writable bytes-like object *b*, and return the number of bytes read. For example, *b* might be a **`bytearray`**.

If `0` is returned and `len(b)` is not `0`, this indicates end of file. If the object is in non-blocking mode and no bytes are available, `None` is returned.

write(*b*, /)

Write the given bytes-like object, *b*, to the underlying raw stream, and return the number of bytes written. This can be less than the length of *b* in bytes, depending on specifics of the underlying raw stream, and especially if it is in non-blocking mode. `None` is returned if the raw stream is set not to block and no single byte could be readily written to it. The caller may release or mutate *b* after this method returns, so the implementation should only access *b* during the method call.

Aviso

This function does not ensure all bytes are written or an exception is thrown. Callers may implement that behavior by checking the return value and, if it is less than the length of *b*, looping with additional write calls until all unwritten bytes are written. High-level I/O objects like E/S binária and E/S de texto implement retry behavior.

class io.BufferedIOBase

Base class for binary streams that support some kind of buffering. It inherits from `IOBase`.

The main difference with `RawIOBase` is that methods `read()`, `readinto()` and `write()` will try (respectively) to read as much input as requested or to emit all provided data.

In addition, if the underlying raw stream is in non-blocking mode, when the system returns would block `write()` will raise `BlockingIOError` with `BlockingIOError.characters_written` and `read()` will return data read so far or `None` if no data is available.

Besides, the `read()` method does not have a default implementation that defers to `readinto()`.

A typical **`BufferedIOBase`** implementation should not inherit from a `RawIOBase` implementation, but wrap one, like `BufferedWriter` and `BufferedReader` do.

`BufferedIOBase` provides or overrides these data attributes and methods in addition to those from `IOBase`:

raw

The underlying raw stream (a `RawIOBase` instance) that **`BufferedIOBase`** deals with. This is not part of the **`BufferedIOBase`** API and may not exist on some implementations.

detach()

Separate the underlying raw stream from the buffer and return it.

After the raw stream has been detached, the buffer is in an unusable state.

Some buffers, like `BytesIO`, do not have the concept of a single raw stream to return from this method. They raise `UnsupportedOperation`.

Adicionado na versão 3.1.

read(*size=-1*, /)

Read and return up to *size* bytes. If the argument is omitted, `None`, or negative read as much as possible.

Fewer bytes may be returned than requested. An empty **`bytes`** object is returned if the stream is already at EOF. More than one read may be made and calls may be retried if specific errors are encountered, see **`os.read()`** and **PEP 475** [https://peps.python.org/pep-0475/] for more details. Less than size bytes being returned does not imply that EOF is imminent.

When reading as much as possible the default implementation will use `raw.readall` if available (which should implement **`RawIOBase.readall()`**), otherwise will read in a loop until read returns `None`, an empty **`bytes`**, or a non-retryable error. For most streams this is to EOF, but for non-blocking streams more data may become available.

Nota

When the underlying raw stream is non-blocking, implementations may either raise **`BlockingIOError`** or return `None` if no data is available. **`io`** implementations return `None`.

read1(*size=-1*, /)

Read and return up to *size* bytes, calling **`readinto()`** which may retry if **`EINTR`** is encountered per **PEP 475** [https://peps.python.org/pep-0475/]. If *size* is `-1` or not provided, the implementation will choose an arbitrary value for *size*.

Nota

When the underlying raw stream is non-blocking, implementations may either raise **`BlockingIOError`** or return `None` if no data is available. **`io`** implementations return `None`.

readinto(*b*, /)

Read bytes into a pre-allocated, writable bytes-like object *b* and return the number of bytes read. For example, *b* might be a `bytearray`.

Like `read()`, multiple reads may be issued to the underlying raw stream, unless the latter is interactive.

A `BlockingIOError` is raised if the underlying raw stream is in non blocking-mode, and has no data available at the moment.

readinto1(*b*, /)

Read bytes into a pre-allocated, writable bytes-like object *b*, using at most one call to the underlying raw stream's `read()` (or `readinto()`) method. Return the number of bytes read.

A `BlockingIOError` is raised if the underlying raw stream is in non blocking-mode, and has no data available at the moment.

Adicionado na versão 3.5.

write(*b*, /)

Write the given bytes-like object, *b*, and return the number of bytes written (always equal to the length of *b* in bytes, since if the write fails an `OSError` will be raised). Depending on the actual implementation, these bytes may be readily written to the underlying stream, or held in a buffer for performance and latency reasons.

When in non-blocking mode, a `BlockingIOError` is raised if the data needed to be written to the raw stream but it couldn't accept all the data without blocking.

The caller may release or mutate *b* after this method returns, so the implementation should only access *b* during the method call.

Raw file I/O

class io.FileIO(*name*, *mode='r'*, *closefd=True*, *opener=None*)

A raw binary stream representing an OS-level file containing bytes data. It inherits from **`RawIOBase`** and implements its low-level access design. This means **`write()`** does not guarantee all bytes are written and **`read()`** may read less bytes than requested even when more bytes may be present in the underlying file. To get “write all” and “read at least” behavior, use E/S binária.

The *name* can be one of two things:

- a character string or **`bytes`** object representing the path to the file which will be opened. In this case closefd must be `True` (the default) otherwise an error will be raised.
- an integer representing the number of an existing OS-level file descriptor to which the resulting **`FileIO`** object will give access. When the FileIO object is closed this fd will be closed as well, unless *closefd* is set to `False`.

The *mode* can be `'r'`, `'w'`, `'x'` or `'a'` for reading (default), writing, exclusive creation or appending. The file will be created if it doesn’t exist when opened for writing or appending; it will be truncated when opened for writing. **`FileExistsError`** will be raised if it already exists when opened for creating. Opening a file for creating implies writing, so this mode behaves in a similar way to `'w'`. Add a `'+'` to the mode to allow simultaneous reading and writing.

A custom opener can be used by passing a callable as *opener*. The underlying file descriptor for the file object is then obtained by calling *opener* with (*name*, *flags*). *opener* must return an open file descriptor (passing **`os.open`** as *opener* results in functionality similar to passing `None`).

O arquivo recém-criado é non-inheritable.

See the **`open()`** built-in function for examples on using the *opener* parameter.

Aviso

`FileIO` is a low-level I/O object and members, such as **`read()`** and **`write()`**, need to have their return values checked explicitly in a retry loop to implement “write all” and “read at least” behavior. High-level I/O objects E/S binária and E/S de texto implement retry behavior.

Alterado na versão 3.3: The *opener* parameter was added. The `'x'` mode was added.

Alterado na versão 3.4: O arquivo agora é não herdável.

`FileIO` provides these data attributes in addition to those from **`RawIOBase`** and **`IOBase`**:

mode
: The mode as given in the constructor.

name
: The file name. This is the file descriptor of the file when no name is given in the constructor.

Buffered streams

Buffered I/O streams provide a higher-level interface to an I/O device than raw I/O does.

class io.BytesIO(*initial_bytes=b''*)

A binary stream using an in-memory bytes buffer. It inherits from **`BufferedIOBase`**. The buffer is discarded when the **`close()`** method is called.

The optional argument *initial_bytes* is a bytes-like object that contains initial data.

Methods may be used from multiple threads without external locking in free-threaded builds.

BytesIO provides or overrides these methods in addition to those from **BufferedIOBase** and **IOBase**:

getbuffer()

Return a readable and writable view over the contents of the buffer without copying them. Also, mutating the view will transparently update the contents of the buffer:

```
>>> b = io.BytesIO(b"abcdef")
>>> view = b.getbuffer()
>>> view[2:4] = b"56"
>>> b.getvalue()
b'ab56ef'
```

Nota

As long as the view exists, the **BytesIO** object cannot be resized or closed.

Adicionado na versão 3.2.

getvalue()

Return **bytes** containing the entire contents of the buffer.

read1(*size=-1*, /)

In **BytesIO**, this is the same as **read()**.

Alterado na versão 3.7: The *size* argument is now optional.

readinto1(*b*, /)

In **BytesIO**, this is the same as **readinto()**.

Adicionado na versão 3.5.

class io.BufferedReader(*raw*, *buffer_size=DEFAULT_BUFFER_SIZE*)

A buffered binary stream providing higher-level access to a readable, non seekable **RawIOBase** raw binary stream. It inherits from

BufferedIOBase.

When reading data from this object, a larger amount of data may be requested from the underlying raw stream, and kept in an internal buffer. The buffered data can then be returned directly on subsequent reads.

The constructor creates a **BufferedReader** for the given readable *raw* stream and *buffer_size*. If *buffer_size* is omitted, **DEFAULT_BUFFER_SIZE** is used.

BufferedReader provides or overrides these methods in addition to those from **BufferedIOBase** and **IOBase**:

peek(*size=0*, /)

Return bytes from the stream without advancing the position. The number of bytes returned may be less or more than requested. If the underlying raw stream is non-blocking and the operation would block, returns empty bytes.

read(*size=-1*, /)

In **BufferedReader** this is the same as **io.BufferedIOBase.read()**

read1(*size=-1*, /)

In **BufferedReader** this is the same as **io.BufferedIOBase.read1()**

Alterado na versão 3.7: The *size* argument is now optional.

class io.BufferedWriter(*raw*, *buffer_size=DEFAULT_BUFFER_SIZE*)

A buffered binary stream providing higher-level access to a writeable, non seekable **RawIOBase** raw binary stream. It inherits from **BufferedIOBase**.

When writing to this object, data is normally placed into an internal buffer. The buffer will be written out to the underlying **RawIOBase** object under various conditions, including:

- when the buffer gets too small for all pending data;
- when `flush()` is called;
- when a `seek()` is requested (for `BufferedRandom` objects);
- when the `BufferedWriter` object is closed or destroyed.

The constructor creates a `BufferedWriter` for the given writeable *raw* stream. If the *buffer_size* is not given, it defaults to `DEFAULT_BUFFER_SIZE`.

`BufferedWriter` provides or overrides these methods in addition to those from `BufferedIOBase` and `IOBase`:

flush()

Force bytes held in the buffer into the raw stream. A `BlockingIOError` should be raised if the raw stream blocks.

write(*b*, /)

Write the bytes-like object, *b*, and return the number of bytes written. When in non-blocking mode, a `BlockingIOError` with `BlockingIOError.characters_written` set is raised if the buffer needs to be written out but the raw stream blocks.

class io.BufferedRandom(*raw*, *buffer_size=DEFAULT_BUFFER_SIZE*)

A buffered binary stream implementing `BufferedIOBase` interfaces providing higher-level access to a seekable `RawIOBase` raw binary stream.

The constructor creates a reader and writer for a seekable raw stream, given in the first argument. If the *buffer_size* is omitted it defaults to `DEFAULT_BUFFER_SIZE`.

`BufferedRandom` is capable of anything `BufferedReader` or `BufferedWriter` can do. In addition, `seek()` and `tell()` are guaranteed to be implemented.

class io.BufferedRWPair(*reader*, *writer*, *buffer_size=DEFAULT_BUFFER_SIZE*, /)

A buffered binary stream providing higher-level access to two non seekable **RawIOBase** raw binary streams—one readable, the other writeable. It inherits from **BufferedIOBase**.

reader and *writer* are **RawIOBase** objects that are readable and writeable respectively. If the *buffer_size* is omitted it defaults to **DEFAULT_BUFFER_SIZE**.

BufferedRWPair implements all of **BufferedIOBase**'s methods except for **detach()**, which raises **UnsupportedOperation**.

Aviso

BufferedRWPair does not attempt to synchronize accesses to its underlying raw streams. You should not pass it the same object as reader and writer; use **BufferedRandom** instead.

E/S de texto

class io.TextIOBase

Base class for text streams. This class provides a character and line based interface to stream I/O. It inherits from **IOBase**.

TextIOBase provides or overrides these data attributes and methods in addition to those from **IOBase**:

encoding

The name of the encoding used to decode the stream's bytes into strings, and to encode strings into bytes.

errors

The error setting of the decoder or encoder.

newlines

A string, a tuple of strings, or `None`, indicating the newlines translated so far. Depending on the implementation and the initial constructor flags, this may not be available.

buffer

The underlying binary buffer (a **BufferedIOBase** or **RawIOBase** instance) that **TextIOBase** deals with. This is not part of the **TextIOBase** API and may not exist in some implementations.

detach()

Separate the underlying binary buffer from the **TextIOBase** and return it.

After the underlying buffer has been detached, the **TextIOBase** is in an unusable state.

Some **TextIOBase** implementations, like **StringIO**, may not have the concept of an underlying buffer and calling this method will raise **UnsupportedOperation**.

Adicionado na versão 3.1.

read(*size=-1*, /)

Read and return at most *size* characters from the stream as a single **str**. If *size* is negative or `None`, reads until EOF.

readline(*size=-1*, /)

Read until newline or EOF and return a single **str**. If the stream is already at EOF, an empty string is returned.

If *size* is specified, at most *size* characters will be read.

seek(*offset*, *whence=SEEK_SET*, /)

Change the stream position to the given *offset*. Behaviour depends on the *whence* parameter. The default value for *whence* is **SEEK_SET**.

- **SEEK_SET** or `0`: seek from the start of the stream (the default); *offset* must either be a number returned by **TextIOBase.tell()**, or zero. Any other *offset* value produces undefined behaviour.

- **SEEK_CUR** or 1: “seek” to the current position; *offset* must be zero, which is a no-operation (all other values are unsupported).
- **SEEK_END** or 2: seek to the end of the stream; *offset* must be zero (all other values are unsupported).

Return the new absolute position as an opaque number.

Adicionado na versão 3.1: The **SEEK_*** constants.

tell()

Return the current stream position as an opaque number. The number does not usually represent a number of bytes in the underlying binary storage.

write(*s*, /)

Write the string *s* to the stream and return the number of characters written.

class io.TextIOWrapper(*buffer*, *encoding=None*, *errors=None*, *newline=None*, *line_buffering=False*, *write_through=False*)

A buffered text stream providing higher-level access to a **BufferedIOBase** buffered binary stream. It inherits from **TextIOBase**.

encoding gives the name of the encoding that the stream will be decoded or encoded with. This defaults to UTF-8; if UTF-8 Mode is disabled, it defaults to **locale.getencoding()**. `encoding="locale"` can be used to specify the current locale’s encoding explicitly. See Codificação do texto for more information.

errors is an optional string that specifies how encoding and decoding errors are to be handled. Pass `'strict'` to raise a **ValueError** exception if there is an encoding error (the default of `None` has the same effect), or pass `'ignore'` to ignore errors. (Note that ignoring encoding errors can lead to data loss.) `'replace'` causes a replacement marker (such as `'?'`) to be inserted where there is malformed data. `'backslashreplace'` causes malformed data to be replaced by a backslashed escape

sequence. When writing, `'xmlcharrefreplace'` (replace with the appropriate XML character reference) or `'namereplace'` (replace with `\N{...}` escape sequences) can be used. Any other error handling name that has been registered with **`codecs.register_error()`** is also valid.

newline controls how line endings are handled. It can be `None`, `''`, `'\n'`, `'\r'`, and `'\r\n'`. It works as follows:

- When reading input from the stream, if *newline* is `None`, universal newlines mode is enabled. Lines in the input can end in `'\n'`, `'\r'`, or `'\r\n'`, and these are translated into `'\n'` before being returned to the caller. If *newline* is `''`, universal newlines mode is enabled, but line endings are returned to the caller untranslated. If *newline* has any of the other legal values, input lines are only terminated by the given string, and the line ending is returned to the caller untranslated.
- Ao gravar a saída no fluxo, se *newline* for `None`, quaisquer caracteres `'\n'` gravados serão traduzidos para o separador de linhas padrão do sistema, **`os.linesep`**. Se *newline* for `''` ou `'\n'`, nenhuma tradução ocorrerá. Se *newline* for um dos outros valores legais, qualquer caractere `'\n'` escrito será traduzido para a string especificada.

If *line_buffering* is `True`, **`flush()`** is implied when a call to write contains a newline character or a carriage return.

If *write_through* is `True`, calls to **`write()`** are guaranteed not to be buffered: any data written on the **`TextIOWrapper`** object is immediately handled to its underlying binary *buffer*.

Alterado na versão 3.3: The *write_through* argument has been added.

Alterado na versão 3.3: The default *encoding* is now `locale.getpreferredencoding(False)` instead of `locale.getpreferredencoding()`. Don’t change temporary the locale encoding using **`locale.setlocale()`**, use the current locale encoding instead of the user preferred encoding.

Alterado na versão 3.10: The *encoding* argument now supports the `"locale"` dummy encoding name.

Nota

When the underlying raw stream is non-blocking, a **`BlockingIOError`** may be raised if a read operation cannot be completed immediately.

`TextIOWrapper` provides these data attributes and methods in addition to those from **`TextIOBase`** and **`IOBase`**:

line_buffering

Whether line buffering is enabled.

write_through

Whether writes are passed immediately to the underlying binary buffer.

Adicionado na versão 3.7.

reconfigure(*, *encoding=None*, *errors=None*, *newline=None*, *line_buffering=None*, *write_through=None*)

Reconfigure this text stream using new settings for *encoding*, *errors*, *newline*, *line_buffering* and *write_through*.

Parameters not specified keep current settings, except `errors='strict'` is used when *encoding* is specified but *errors* is not specified.

It is not possible to change the encoding or newline if some data has already been read from the stream. On the other hand, changing encoding after write is possible.

This method does an implicit stream flush before setting the new parameters.

Adicionado na versão 3.7.

Alterado na versão 3.11: The method supports `encoding="locale"` option.

seek(*cookie*, *whence=os.SEEK_SET*, /)

Set the stream position. Return the new stream position as an `int`.

Four operations are supported, given by the following argument combinations:

- `seek(0, SEEK_SET)`: Rewind to the start of the stream.
- `seek(cookie, SEEK_SET)`: Restore a previous position; *cookie* **must be** a number returned by `tell()`.
- `seek(0, SEEK_END)`: Fast-forward to the end of the stream.
- `seek(0, SEEK_CUR)`: Leave the current stream position unchanged.

Any other argument combinations are invalid, and may raise exceptions.

Ver também

`os.SEEK_SET`, `os.SEEK_CUR`, and `os.SEEK_END`.

tell()

Return the stream position as an opaque number. The return value of `tell()` can be given as input to `seek()`, to restore a previous stream position.

class io.StringIO(*initial_value=''*, *newline='\n'*)

A text stream using an in-memory text buffer. It inherits from `TextIOBase`.

The text buffer is discarded when the `close()` method is called.

The initial value of the buffer can be set by providing *initial_value*. If newline translation is enabled, newlines will be encoded as if by `write()`. The stream is positioned at the start of the buffer which

emulates opening an existing file in a `w+` mode, making it ready for an immediate write from the beginning or for a write that would overwrite the initial value. To emulate opening a file in an `a+` mode ready for appending, use `f.seek(0, io.SEEK_END)` to reposition the stream at the end of the buffer.

The *newline* argument works like that of **`TextIOWrapper`**, except that when writing output to the stream, if *newline* is `None`, newlines are written as `\n` on all platforms.

`StringIO` provides this method in addition to those from **`TextIOBase`** and **`IOBase`**:

getvalue()

Return a **`str`** containing the entire contents of the buffer. Newlines are decoded as if by **`read()`**, although the stream position is not changed.

Exemplo de uso:

```
import io

output = io.StringIO()
output.write('First line.\n')
print('Second line.', file=output)

# Retrieve file contents -- this will be
# 'First line.\nSecond line.\n'
contents = output.getvalue()

# Close object and discard memory buffer --
# .getvalue() will now raise an exception.
output.close()
```

class io.IncrementalNewlineDecoder

A helper codec that decodes newlines for universal newlines mode. It inherits from **`codecs.IncrementalDecoder`**.

Static typing

The following protocols can be used for annotating function and method arguments for simple stream reading or writing operations. They are decorated with `@typing.runtime_checkable`.

class io.Reader[*T*]

Generic protocol for reading from a file or other input stream. `T` will usually be `str` or `bytes`, but can be any type that is read from the stream.

Adicionado na versão 3.14.

read()

read(*size*, /)

Read data from the input stream and return it. If *size* is specified, it should be an integer, and at most *size* items (bytes/characters) will be read.

Por exemplo:

```
def read_it(reader: Reader[str]):
    data = reader.read(11)
    assert isinstance(data, str)
```

class io.Writer[*T*]

Generic protocol for writing to a file or other output stream. `T` will usually be `str` or `bytes`, but can be any type that can be written to the stream.

Adicionado na versão 3.14.

write(*data*, /)

Write *data* to the output stream and return the number of items (bytes/characters) written.

Por exemplo:

```
def write_binary(writer: Writer[bytes]):
    writer.write(b"Hello world!\n")
```

See ABCs e protocolos para trabalhar com E/S for other I/O related protocols and classes that can be used for static type checking.

Desempenho

This section discusses the performance of the provided concrete I/O implementations.

E/S binária

By reading and writing only large chunks of data even when the user asks for a single byte, buffered I/O hides any inefficiency in calling and executing the operating system's unbuffered I/O routines. The gain depends on the OS and the kind of I/O which is performed. For example, on some modern OSes such as Linux, unbuffered disk I/O can be as fast as buffered I/O. The bottom line, however, is that buffered I/O offers predictable performance regardless of the platform and the backing device. Therefore, it is almost always preferable to use buffered I/O rather than unbuffered I/O for binary data.

E/S de texto

Text I/O over a binary storage (such as a file) is significantly slower than binary I/O over the same storage, because it requires conversions between unicode and binary data using a character codec. This can become noticeable handling huge amounts of text data like large log files. Also, `tell()` and `seek()` are both quite slow due to the reconstruction algorithm used.

`StringIO`, however, is a native in-memory unicode container and will exhibit similar speed to `BytesIO`.

Multi-threading

`FileIO` objects are thread-safe to the extent that the operating system calls (such as *read(2)* *[https://manpages.debian.org/read(2)]* under Unix) they wrap are

thread-safe too.

Binary buffered objects (instances of `BufferedReader`, `BufferedWriter`, `BufferedRandom` and `BufferedRWPair`) protect their internal structures using a lock; it is therefore safe to call them from multiple threads at once.

`TextIOWrapper` objects are not thread-safe.

Reentrancy

Binary buffered objects (instances of `BufferedReader`, `BufferedWriter`, `BufferedRandom` and `BufferedRWPair`) are not reentrant. While reentrant calls will not happen in normal situations, they can arise from doing I/O in a `signal` handler. If a thread tries to re-enter a buffered object which it is already accessing, a `RuntimeError` is raised. Note this doesn't prohibit a different thread from entering the buffered object.

The above implicitly extends to text files, since the `open()` function will wrap a buffered object inside a `TextIOWrapper`. This includes standard streams and therefore affects the built-in `print()` function as well.

`time` — Acesso e conversões de tempo

Esse módulo provê várias funções relacionadas à tempo. Para funcionalidades relacionadas veja também os módulos `datetime` e `calendar`

Apesar desse módulo sempre estar disponível, nem todas as suas funções estão disponíveis em todas as plataformas. A maioria das funções definidas nesse módulo chamam funções da biblioteca da plataforma de C com mesmo nome. Pode ser útil consultar a documentação da plataforma, pois da semântica dessas funções variam a depender da plataforma.

A seguir, uma explicação de algumas terminologias e convenções.

- O *era* (*epoch*) é o ponto onde o tempo começa, o valor de retorno de `time.gmtime(0)`. É 1º de janeiro de 1970, 00:00:00 (UTC) em todas as plataformas.

- O termo *segundos desde a era* refere-se ao número total de segundos decorrido desde a era, tipicamente excluindo-se os segundos bissextos [https://en.wikipedia.org/wiki/Leap_second]. Segundos bissextos são excluídos desse total em todas as plataformas compatíveis com POSIX.

- As funções desse módulo podem não conseguir tratar de datas e horários antes do epoch ou muito distantes no futuro. O limite no futuro é determinado pela biblioteca C; para sistemas de 32 bit, geralmente é 2038.

- A função `strptime()` pode analisar anos de 2 dígitos quando é passado o código de formato `%y`. Quando anos de 2 dígitos são analisados, eles são convertidos de acordo com os padrões POSIX e

ISO C: valores 69–99 são mapeados para 1969–1999, e valores 0–68 são mapeados para 2000–2068.

- UTC é Coordinated Universal Time [https://en.wikipedia.org/wiki/Coordinated_Universal_Time] (em português, Horário Universal Coordenado) e substituiu o GMT ou Greenwich Mean Time [https://en.wikipedia.org/wiki/Greenwich_Mean_Time] (em português, Horário Médio de Greenwich) como base da cronometragem internacional. A sigla UTC não é um erro, mas está em conformidade com um esquema de nomenclatura anterior, independente de idioma, para padrões de tempo como UT0, UT1 e UT2.

- DST é Daylight Saving Time (Horário de Verão), um ajuste de fuso horário por (normalmente) uma hora durante parte do ano. As regras de Horário de Verão são mágicas (determinadas por leis locais) e podem mudar de ano a ano. A biblioteca C possui uma tabela contendo as regras locais (normalmente lidas de um arquivo de sistema por flexibilidade) e nesse contexto é a única fonte de Conhecimento Verdadeiro.

- A precisão de várias funções em tempo real podem ser menores do que o que pode estar sugerido pelas unidades nas quais seu valor ou argumento estão expressos. Por exemplo, na maioria dos sistemas Unix, o relógio “conta” apenas 50 ou 100 vezes por segundo.

- Por outro lado, a precisão de **`time()`** e **`sleep()`** é melhor do que suas equivalentes Unix: tempos são expressos como números em ponto flutuante, **`time()`** retorna o tempo mais preciso disponível (utilizando **`gettimeofday()`** do Unix, quando disponível) e **`sleep()`** irá aceitar qualquer tempo como uma fração não zero (**`select()`** do Unix é utilizada para implementar isto, quando disponível).

- O valor de tempo conforme retornado pelas **`gmtime()`**, **`localtime()`**, e **`strptime()`**, e aceito pelas **`asctime()`**, **`mktime()`** e **`strftime()`**, é uma sequência de 9 inteiros. Os valores retornados das **`gmtime()`**, **`localtime()`**, e **`strptime()`** também oferecem nomes de atributo para campos individuais.

Veja `struct_time` para a descrição desses objetos.

Alterado na versão 3.3: O tipo `struct_time` foi estendido para prover os atributos `tm_gmtoff` e `tm_zone` quando a plataforma suporta os membros `struct tm` correspondentes

Alterado na versão 3.6: Os atributos `tm_gmtoff` e `tm_zone` da classe `struct_time` estão disponíveis em todas as plataformas agora.

- Utilize as seguintes funções para converter entre representações de tempo:

De	Para	Utilize
segundos desde a era	`struct_time` em UTC	`gmtime()`
segundos desde a era	`struct_time` em tempo local	`localtime()`
`struct_time` em UTC	segundos desde a era	`calendar.timegm()`
`struct_time` em tempo local	segundos desde a era	`mktime()`

Funções

time.asctime([*time_tuple*])

Convert a tuple or `struct_time` representing a time as returned by `gmtime()` or `localtime()` to a string of the following form: `'Sun Jun 20 23:21:05 1993'`. The day field is two characters long and is space

padded if the day is a single digit, for example: `'Wed Jun  9 04:26:40 1993'`.

If *time_tuple* is not provided, the current time as returned by `localtime()` is used. Locale information is not used by `asctime()`.

Nota

Diferentemente da função em C de mesmo nome, `asctime()` não adiciona uma nova linha em seguida.

time.pthread_getcpuclockid(*thread_id*, /)

Retorna o *clk_id* do relógio de tempo de CPU específico da thread para a *thread_id* especificada.

Utilize a `threading.get_ident()` ou o atributo `ident` dos objetos `threading.Thread` para obter um valor adequado para *thread_id*.

Aviso

Passando um *thread_id* inválido ou expirado pode resultar em um comportamento indefinido, como, por exemplo, falha de segmentação.

Disponibilidade: Unix

Veja a página man para *pthread_getcpuclockid(3)* *[https://manpages.debian.org/pthread_getcpuclockid(3)]* para mais informações.

Adicionado na versão 3.7.

time.clock_getres(*clk_id*, /)

Retorna a resolução (precisão) do relógio *clk_id* especificado. Confira Constantes de ID de Relógio para uma lista de valores aceitos para *clk_id*

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.clock_gettime(*clk_id*, /) → float

Retorna o tempo to relógio *clk_id* especificado. Confira Constantes de ID de Relógio para uma lista de valores aceitos para *clk_id*.

Use `clock_gettime_ns()` para evitar perda de precisão causada pelo tipo `float`.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.clock_gettime_ns(*clk_id*, /) → int

Semelhante à `clock_gettime()`, mas retorna o tempo em nanossegundos.

Disponibilidade: Unix.

Adicionado na versão 3.7.

time.clock_settime(*clk_id*, *time*, /)

Define o tempo do relógio *clk_id* especificado. Atualmente, `CLOCK_REALTIME` é o único valor aceito para *clk_id*.

Use `clock_settime_ns()` para evitar perda de precisão causada pelo tipo `float`.

Disponibilidade: Unix, not Android, not iOS.

Adicionado na versão 3.3.

Alterado na versão 3.15: Aceita qualquer número real como *time*, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

time.clock_settime_ns(*clk_id*, *time: int*, /)

Semelhante à `clock_settime()`, mas define o tempo em nanossegundos.

Disponibilidade: Unix, not Android, not iOS.

Adicionado na versão 3.7.

time.ctime(*seconds=None*, /)

Convert a time expressed in seconds since the epoch to a string of a form: `'Sun Jun 20 23:21:05 1993'` representing local time. The day field is two characters long and is space padded if the day is a single digit, for example: `'Wed Jun  9 04:26:40 1993'`.

If *seconds* is not provided or **`None`**, the current time as returned by **`time()`** is used. `ctime(seconds)` is equivalent to `asctime(localtime(seconds))`. Locale information is not used by **`ctime()`**.

Alterado na versão 3.15: Aceita qualquer número real, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

time.get_clock_info(*name*, /)

Obtém informação do relógio específico como um objeto espaço de nomes. Nomes de relógios suportados e as funções correspondentes para ler seus valores são:

- `'monotonic'`: **`time.monotonic()`**
- `'perf_counter'`: **`time.perf_counter()`**
- `'process_time'`: **`time.process_time()`**
- `'thread_time'`: **`time.thread_time()`**
- `'time'`: **`time.time()`**

O resultado tem os seguintes atributos:

- *adjustable*: `True` se o relógio puder ser configurado para avançar ou retroceder no tempo, `False` caso contrário. Não se refere a ajustes graduais da taxa NTP.
- *implementation*: O nome da função C subjacente utilizada para obter o valor do relógio. Confira Constantes de ID de Relógio para valores possíveis.

- *monotonic*: `True` se o relógio não pode retornar a valores anteriores, backward, `False` contrário
- *resolution*: A resolução do relógio em segundos (`float`)

Adicionado na versão 3.3.

time.gmtime(*seconds=None*, /)

Convert a time expressed in seconds since the epoch to a `struct_time` in UTC in which the dst flag is always zero. If *seconds* is not provided or `None`, the current time as returned by `time()` is used. Fractions of a second are ignored. See above for a description of the **`struct_time`** object. See `calendar.timegm()` for the inverse of this function.

Alterado na versão 3.15: Aceita qualquer número real, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

time.localtime(*seconds=None*, /)

Like `gmtime()` but converts to local time. If *seconds* is not provided or `None`, the current time as returned by `time()` is used. The dst flag is set to `1` when DST applies to the given time.

`localtime()` pode levantar `OverflowError`, se o registro de data e hora estiver fora de valores suportados pelas funções **`localtime()`** or **`gmtime()`** da plataforma C, e `OSError` no caso de **`localtime()`** ou **`gmtime()`** falharem. É comum É comum que isso seja restrito a anos de 1970 a 2038.

Alterado na versão 3.15: Aceita qualquer número real, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

time.mktime(*time_tuple*, /)

Esta é a função inversa de `localtime()`. Seu argumento é a `struct_time` ou uma tupla com 9 itens (sendo o sinalizador de horário de verão necessário; utilize `-1` como sinalizador de horário de verão quando este for desconhecido) que expressa o tempo em tempo *local*, não UTC. Retorna um número em ponto flutuante, para ter compatibilidade com `time()`. Se o valor de entrada não puder ser representado como um tempo válido, ou `OverflowError` ou `ValueError`

serão levantadas (o que irá depender se o valor inválido é capturado pelo Python ou por bibliotecas C subjacentes). A data mais recente para qual um tempo pode ser gerado é dependente da plataforma.

time.monotonic() → float

Retorna o valor (em frações de segundos) de um relógio monotônico, i.e. um relógio que não pode voltar a valores anteriores. O relógio não é afetado por atualizações do relógio do sistema. O ponto de referência do valor retornado é indefinido, portanto apenas a diferença entre os resultados de duas chamadas é válida.

Relógio:

- No Windows, chama `QueryPerformanceCounter()` e `QueryPerformanceFrequency()`.
- No macOS, chama `mach_absolute_time()` e `mach_timebase_info()`.
- No HP-UX, chama `gethrtime()`.
- Chama `clock_gettime(CLOCK_HIGHRES)` se disponível.
- Do contrário, chama `clock_gettime(CLOCK_MONOTONIC)`.

Use **`monotonic_ns()`** para evitar perda de precisão causada pelo tipo **`float`**.

Adicionado na versão 3.3.

Alterado na versão 3.5: A função agora está sempre disponível e o relógio agora é o mesmo para todos os processos.

Alterado na versão 3.10: No macOS, o relógio agora é o mesmo para todos os processos.

time.monotonic_ns() → int

Semelhante à **`monotonic()`**, mas retorna tempo em nanossegundos.

Adicionado na versão 3.7.

time.perf_counter() → float

Retorna o valor (em frações de segundo) de um contador de desempenho, i.e. um relógio com a maior resolução disponível para medir uma duração curta. Inclui o tempo decorrido durante a pausa. O relógio é o mesmo para todos os processos. O ponto de referência é do valor retornado é indefinido, portanto apenas a diferença entre resultados de duas chamadas é válida.

No CPython, use o mesmo relógio que `time.monotonic()` e é um relógio monotônico, ou seja, um relógio que não pode retroceder.

Use `perf_counter_ns()` para evitar perda de precisão causada pelo tipo `float`.

Adicionado na versão 3.3.

Alterado na versão 3.10: No Windows, o relógio agora é o mesmo para todos os processos.

Alterado na versão 3.13: Usa o mesmo relógio que `time.monotonic()`.

time.perf_counter_ns() → int

Semelhante à `perf_counter()`, mas retorna o tempo em nanossegundos.

Adicionado na versão 3.7.

time.process_time() → float

Retorna o valor (em frações de segundo) da soma dos tempos do sistema e CPU de usuário do processo atual. Não inclui o tempo decorrido durante a pausa. É de todo o processo por definição. O ponto de referência do valor retornado é indefinido, então apenas a diferença dos resultados de duas chamadas é válida.

Use `process_time_ns()` para evitar perda de precisão causada pelo tipo `float`.

Adicionado na versão 3.3.

time.process_time_ns() → int

Semelhante à `process_time()`, mas retorna o tempo em nanossegundos.

Adicionado na versão 3.7.

time.sleep(*seconds*, /)

Suspende a execução da thread de chamada por um determinado número de segundos. O argumento pode ser um número não-inteiro para indicar um tempo de pausa mais preciso.

Se a pausa for interrompida por um sinal e nenhuma exceção for levantada pelo manipulador de sinal, a pausa será reiniciada com um tempo limite recalculado.

O tempo de suspensão pode ser maior do que o solicitado por um valor arbitrário, devido ao agendamento de outras atividades no sistema.

Implementação no Windows

On Windows, if *seconds* is zero, the thread relinquishes the remainder of its time slice to any other thread that is ready to run. If there are no other threads ready to run, the function returns immediately, and the thread continues execution. On Windows 10 and newer the implementation uses a high-resolution timer [https://learn.microsoft.com/windows/win32/api/synchapi/nf-synchapi-createwaitabletimerexw] which provides resolution of 100 nanoseconds. If *seconds* is zero, `Sleep(0)` is used.

Implementação no Unix

- Usa `clock_nanosleep()` se disponível (resolução: 1 nanossegundo);
- Ou usa `nanosleep()` se disponível (resolução: 1 nanossegunda);
- Ou usa `select()` (resolução: 1 microssegundo).

Nota

Para emular um “no-op”, use **`pass`** em vez de `time.sleep(0)`.

Para abrir mão voluntariamente da CPU, especifique uma política de agendamento em tempo real e use **`os.sched_yield()`** em seu lugar.

Raises an auditing event `time.sleep` with argument `seconds`.

Alterado na versão 3.5: The function now sleeps at least *seconds* even if the sleep is interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

Alterado na versão 3.11: No Unix, as funções `clock_nanosleep()` e `nanosleep()` são agora usadas se estiverem disponíveis. No Windows, um temporizador aguardável é agora usado.

Alterado na versão 3.13: Levanta um evento de auditoria.

Alterado na versão 3.15: Aceita qualquer número real, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

time.strftime(*format*[, *time_tuple*])

Convert a tuple or **`struct time`** representing a time as returned by **`gmtime()`** or **`localtime()`** to a string as specified by the *format* argument. If *time_tuple* is not provided, the current time as returned by **`localtime()`** is used. *format* must be a string. **`ValueError`** is raised if any field in *time_tuple* is outside of the allowed range.

0 é um argumento legal para qualquer posição na tupla de tempo; se é normalmente ilegal, o valor é formado a um valor correto.

As diretivas a seguir podem ser incorporadas na string *format*. Elas estão mostradas sem os campos de comprimento e especificação de precisão opcionais, e estão substituídos pelos caracteres indicados no resultado da **`strftime()`**:

Diretiva	Significado	Notas

Diretiva	Significado	Notas
`%a`	Nome abreviado do dia da semana da localidade.	
`%A`	Nome completo do dia da semana da localidade.	
`%b`	Nome abreviado do mês da localidade.	
`%B`	Nome completo do mês da localidade.	
`%c`	Representação de data e hora apropriada da localidade.	
`%d`	Dia do mês como um número decimal [01,31].	
`%f`	Microssegundos como um número decimal [000000,999999].	(1)
`%H`	Hora (relógio 24 horas) como um número decimal [00,23].	
`%I`	Hora (relógio 12 horas) como um número decimal [01,12].	
`%j`	Dia do ano como um número decimal [001,366].	
`%m`	Mês como um número decimal [01,12].	
`%M`	Minuto como um número decimal [00,59].	
`%p`	Equivalente da localidade a AM ou PM.	(2)
`%S`	Segundo como um número decimal [00,61].	(3)

Diretiva	Significado	Notas
%U	Número da semana do ano (domingo como primeiro dia da semana) como um número decimal [00,53]. Todos os dias em um ano novo que precedem o primeiro domingo são considerados como estando na semana 0.	(4)
%u	Dia da semana (segunda-feira é 1; domingo é 7) como um número decimal [1, 7].	
%w	Dia da semana como um número decimal [0(Domingo),6]	
%W	Número da semana do ano (segunda-feira como o primeiro dia da semana) como um número decimal [00,53]. Todos os dias do ano que precedem o primeiro domingo serão considerados como estando na semana 0.	(4)
%x	Representação de data apropriada de localidade.	
%X	Representação de hora apropriada da localidade.	
%y	Ano sem século como um número decimal [00,99].	
%Y	Ano com século como um número decimal.	
%z	Deslocamento de fuso horário indicando uma diferença de tempo positiva ou negativa de UTC/GMT formatado como +HHMM ou -HHMM, onde H representa os dígitos decimais de hora e M representa os dígitos decimais de minuto [-23:59, +23:59]. [[1]]	

Diretiva	Significado	Notas
`%Z`	Nome do fuso horário (nenhum caractere se nenhum fuso horário existe). Descontinuado. [[1]]	
`%G`	Ano ISO 8601 (semelhante a `%Y`, mas segue as regras para o ano calendário ISO 8601). O ano começa com a semana que contém a primeira quinta-feira do ano calendário.	
`%V`	Número da semana ISO 8601 (como um número decimal [01,53]). A primeira semana do ano é aquela que contém a primeira quinta-feira do ano. As semanas começam na segunda-feira.	
`%%`	Um caractere literal `'%'`.	

Notas:

1. A diretiva de formatação `%f` é somente aplicada para **`strptime()`**, não para **`strftime()`**. Contudo, veja também **`datetime.datetime.strptime()`** e **`datetime.datetime.strftime()`** onde a diretiva de formatação `%f` é aplicada para microssegundos.
2. Quando utilizado com a função **`strptime()`**, a diretiva `%p` apenas afeta a saída do campo se a diretiva `%I` é utilizada para analisar a hora.
3. O intervalo é realmente `0` até `61`; o valor `60` é válido em registros de data e hora representando segundos bissextos [https://en.wikipedia.org/wiki/Leap_second] e o valor `61` é suportado por razões históricas.
4. Quando utilizado co a função **`strptime()`**, `%U` e `%W` são utilizados em cálculos apenas quando o dia da semana e ano são especificados.

Veja este exemplo, um formato para datas compatível com as especificações dos padrões de e-mail **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html]. [[1]]

```
>>> from time import gmtime, strftime
>>> strftime("%a, %d %b %Y %H:%M:%S +0000", gmtime())
'Thu, 28 Jun 2001 14:17:15 +0000'
```

Diretivas adicionais podem ser suportadas por algumas plataformas, mas apenas as listadas aqui possuem significado padronizado por ANSI C. Para ver a lista completa de códigos de formato suportados na sua plataforma, consulte a documentação *strftime(3)* *[https://manpages.debian.org/strftime(3)]*.

Em algumas plataformas, um campo adicional de comprimento e especificação de precisão podem seguir imediatamente após `'%'` como uma diretiva da seguinte ordem; isto também não é portátil. O campo comprimento normalmente é 2 exceto para `%j` quando é 3.

time.strptime(*string*[, *format*])

Analisa a string representando um tempo de acordo com um formato. O valor retornado é um **`struct_time`** como retornado por **`gmtime()`** ou **`localtime()`**.

O parâmetro *format* utiliza as mesmas diretivas das utilizadas por **`strftime()`**; é definido por padrão para `"%a %b %d %H:%M:%S %Y"` que corresponde com a formatação retornada por **`ctime()`**. Se *string* não puder ser analisada de acordo com *format*, ou se possui excesso de dados após analisar, **`ValueError`** é levantado. Os valores padrão utilizados para preencher quaisquer dados faltantes quando valores mais precisos não puderem ser inferidos são `(1900, 1, 1, 0, 0, 0, 0, 1, -1)`. Ambos *string* e *format* devem ser strings.

Por exemplo:

```
>>> import time
>>> time.strptime("30 Nov 00", "%d %b %y")
time.struct_time(tm_year=2000, tm_mon=11, tm_mday=30, tm_hour
                 tm_sec=0, tm_wday=3, tm_yday=335, tm_isdst=-
```

Suporte para a diretiva `%Z` é baseado nos valores contidos em `tzname` e se `daylight` é verdade. Por causa disso, é específico de plataforma

exceto por reconhecer UTC e GMT, que são sempre conhecidos (e considerados fuso horários sem horários de verão).

Apenas as diretivas especificadas na documentação são suportadas. Como `strftime()` é implementada por plataforma, esta pode apenas as vezes oferecer mais diretivas do que as listadas aqui. Mas `strptime()` é independente de quaisquer plataformas e portanto não necessariamente suporta todas as diretivas disponíveis que não estão documentadas como suportadas.

class time.struct_time

O tipo da sequência de valor de tempo retornado por **`gmtime()`**, **`localtime()`**, e **`strptime()`**. É um objeto com uma interface named tuple: os valores podem ser acessados por um índice e por um nome de atributo. Os seguintes valores estão presentes:

Índice	Atributo	Valores
0	tm_year	(por exemplo, 1993)
1	tm_mon	intervalo [1,12]
2	tm_mday	intervalo [1,31]
3	tm_hour	intervalo [0,23]
4	tm_min	intervalo [0,59]
5	tm_sec	intervalo [0, 61]; veja Nota (2) em **`strftime()`**
6	tm_wday	intervalo [0, 6]; segunda-feira é 0
7	tm_yday	intervalo [1, 366]
8	tm_isdst	0, 1 ou -1; veja abaixo
N/D	tm_zone	abreviação do nome do fuso horário

N/D	tm_gmtoff	deslocamento a leste de UTC em segundos

Note que diferentemente da estrutura C, o valor do mês é um intervalo [1,12] e não [0,11].

Em chamadas para `mktime()`, `tm_isdst` pode ser definido como 1 quando o horário de verão estiver em efeito, e 0 quando não. Um valor de -1 indica que esta informação não é conhecida, e geralmente resultará no preenchimento do estado correto.

Quando uma tupla com comprimento incorreto é passada para uma função que espera por um `struct_time`, ou por possuir elementos do tipo errado, um `TypeError` é levantado.

time.time() → float

Retorna o tempo em segundos desde o epoch como um número em ponto flutuante. A manipulação de segundos bissextos [https://en.wikipedia.org/wiki/Leap_second] é dependente da plataforma. No Windows e na maioria dos sistemas Unix, o segundo bissexto não é contado para o tempo desde o epoch. Isso é comumente referido como epoch ou era Unix [https://en.wikipedia.org/wiki/Unix_time].

Note que mesmo o tempo sendo retornado sempre como um número em ponto flutuante, nem todos os sistemas fornecem o tempo com precisão melhor que 1 segundo. Enquanto esta função normalmente retorna valores não decrescentes, pode retornar valores menores do que os de uma chamada anterior se o relógio do sistema foi redefinido entre duas chamadas.

O número retornado por `time()` pode ser convertido a um formato de tempo mais comum (i.e. ano, mês, dia, hora etc…) em UTC por passá-lo para a função `gmtime()` ou em tempo local por passar para a função `localtime()`. Em ambos os casos, o objeto `struct_time` é retornado, por onde os componentes de data do calendário podem ser acessados ou atribuídos.

Relógio:

- No Windows, chama `GetSystemTimePreciseAsFileTime()`.
- Chama `clock_gettime(CLOCK_REALTIME)` se disponível.
- Caso contrário, chama `gettimeofday()`.

Use **`time_ns()`** para evitar perda de precisão causada pelo tipo **`float`**.

Alterado na versão 3.13: No Windows, chama `GetSystemTimePreciseAsFileTime()` em vez de `GetSystemTimeAsFileTime()`.

time.time_ns() → int

Semelhante à **`time()`**, mas retorna o tempo como um número inteiro de nanossegundos desde o epoch.

Adicionado na versão 3.7.

time.thread_time() → float

Retorna o valor (em fração de segundos) da soma dos tempos de sistema e CPU de usuário para a thread atual. Não inclui o tempo decorrido durante a pausa. É específico a thread por definição. O ponto de referência do valor retornado é indefinido, então apenas a diferença dos resultados de duas chamadas é válida.

Use **`thread_time_ns()`** para evitar perda de precisão causada pelo tipo **`float`**.

Disponibilidade: Linux, Unix, Windows.

Sistemas Unix possuem suporte para `CLOCK_THREAD_CPUTIME_ID`.

Adicionado na versão 3.7.

time.thread_time_ns() → int

Semelhante à **`thread_time()`**, mas retorna o tempo em nanossegundos.

Adicionado na versão 3.7.

time.tzset()

Redefine as regras de conversão utilizadas pelas rotinas da biblioteca. A variável de ambiente **TZ** especifica como isto é feito. Também irá redefinir as variáveis `tzname` (da variável de ambiente **TZ**), `timezone` (segundos sem horário de verão a oeste de UTC), `altzone` (segundos com horário de verão a oeste de UTC) e `daylight` (para 0 se este fuso horário não possui nenhuma regra de horário de verão, ou diferente de zero se há um tempo, no presente, passado ou futuro quando o horário de verão se aplica).

Disponibilidade: Unix.

Nota

Embora em vários casos, alterar a variável de sistema **TZ** pode afetar a saída de funções como **`localtime()`** sem chamar **`tzset()`**, este comportamento não deve ser confiado.

A variável de sistema **TZ** não deve conter espaços em branco.

O formato padrão da variável de sistema **TZ** é (espaços foram adicionados por motivos de clareza):

```
std offset [dst [offset [,start[/time], end[/time]]]]
```

Onde os componentes são:

`std` e `dst`

Três ou mais alfanuméricos fornecendo a abreviação do fuso horário. Estes serão propagados para time.tzname

`offset`

O deslocamento tem a forma: `± hh[:mm[:ss]]`. Isso indica que o valor adicionado adicionou o horário local para chegar a UTC. Se precedido por um '-', o fuso horário está a leste do Meridiano Primário; do contrário, está a oeste. Se nenhum deslocamento segue o horário de verão, o tempo no verão é presumido como estando uma hora a frente do horário padrão.

`start[/time], end[/time]`

Indica quando mudar e voltar do Horário de Verão. O formato das datas de início e fim é um dos seguintes:

`J`*n*

I dia juliano *n* (1 <= *n* <= 365). Os dias bissextos não são contados, então, em todos os anos, 28 de fevereiro é o dia 59 e 1 de março é o dia 60.

n

O dia juliano baseado em zero (0 <= *n* <= 365). Dias bissextos são contados, e é possível fazer referência a 29 de fevereiro.

`M`*m*`.`*n*`.`*d*

O *d*-ésimo dia (0 <= *d* <= 6) da semana *n* do mês *m* do ano (1 <= *n* <= 5, 1 <= *m* <= 12, onde semana 5 significa "o último dia *d* no mês *m*" que pode ocorrer tanto na quarta como quinta semana). Semana 1 é a primeira semana na qual o *d*-ésimo dia ocorre. Dia zero é o domingo.

`time` tem o mesmo formato que `offset`, exceto que nenhum sinal no início é permitido ('-' ou '+'). O padrão, se o tempo não é dado, é 02:00:00.

```
>>> os.environ['TZ'] = 'EST+05EDT,M4.1.0,M10.5.0'
>>> time.tzset()
>>> time.strftime('%X %x %Z')
'02:07:36 05/08/03 EDT'
>>> os.environ['TZ'] = 'AEST-10AEDT-11,M10.5.0,M3.5.0'
>>> time.tzset()
>>> time.strftime('%X %x %Z')
'16:08:12 05/08/03 AEST'
```

Em muitos sistemas Unix (incluindo *BSD, Linux, Solaris, e Darwin), é mais conveniente utilizar o banco de dados de informação de fuso do sistema (*tzfile(5)* *[https://manpages.debian.org/tzfile(5)]*) para especificar as regras de fuso horário. Para fazer isso, defina a variável de sistema **`TZ`** ao path do arquivo de dados requerido de fuso horários, relativo à raiz do banco de dados de fuso horário 'zoneinfo' do sistema, geralmente

encontrado em `/usr/share/zoneinfo`. Por exemplo, `'US/Eastern'`, `'Australia/Melbourne'`, `'Egypt'` ou `'Europe/Amsterdam'`.

```
>>> os.environ['TZ'] = 'US/Eastern'
>>> time.tzset()
>>> time.tzname
('EST', 'EDT')
>>> os.environ['TZ'] = 'Egypt'
>>> time.tzset()
>>> time.tzname
('EET', 'EEST')
```

Constantes de ID de Relógio

Essas constantes são utilizadas como parâmetros para **`clock_getres()`** e **`clock_gettime()`**.

time.CLOCK_BOOTTIME

Idêntica a **`CLOCK_MONOTONIC`**, exceto por também incluir qualquer tempo que o sistema está suspenso.

Isto permite que aplicações recebam um relógio monotônico consciente suspenso sem precisar lidar com as complicações de **`CLOCK_REALTIME`**, que pode conter descontinuidades se o tempo é alterado utilizando `settimeofday()` ou algo semelhante.

Disponibilidade: Linux >= 2.6.39.

Adicionado na versão 3.7.

time.CLOCK_HIGHRES

O Solaris OS possui um timer `CLOCK_HIGHRES` que tenta utilizar recursos otimizados do hardware, e pode fornecer resolução perto de nanossegundos. `CLOCK_HIGHRES` é o relógio nanoajustável de alta resolução.

Disponibilidade: Solaris.

Adicionado na versão 3.3.

time.CLOCK_MONOTONIC

Relógio que não pode ser definido e representa um tempo monotônico desde um ponto de início não especificado.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.CLOCK_MONOTONIC_RAW

Semelhante à `CLOCK_MONOTONIC`, mas fornece acesso a um tempo bruto baseado em hardware que não está sujeito a ajustes NTP.

Disponibilidade: Linux >= 2.6.28, macOS >= 10.12.

Adicionado na versão 3.3.

time.CLOCK_MONOTONIC_RAW_APPROX

Semelhante a `CLOCK_MONOTONIC_RAW`, mas lê um armazenado pelo sistema no gerenciador de contexto e, portanto, tem menor precisão.

Disponibilidade: macOS >= 10.12.

Adicionado na versão 3.13.

time.CLOCK_PROCESS_CPUTIME_ID

Timer de alta resolução por processo no CPU.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.CLOCK_PROF

Timer de alta resolução por processo no CPU.

Disponibilidade: FreeBSD, NetBSD >= 7, OpenBSD.

Adicionado na versão 3.7.

time.CLOCK_TAI

Tempo Atômico Internacional [https://www.nist.gov/pml/time-and-frequency-division/how-utcnist-related-coordinated-universal-time-utc-international]

O sistema deve ter uma tabela de segundos bissextos para que possa fornecer a resposta correta. Softwares PTP ou NTP podem manter uma tabela de segundos bissextos.

Disponibilidade: Linux.

Adicionado na versão 3.9.

time.CLOCK_THREAD_CPUTIME_ID

Relógio de tempo de CPU específico a thread.

Disponibilidade: Unix.

Adicionado na versão 3.3.

time.CLOCK_UPTIME

Tempo cujo valor absoluto é o tempo que o sistema está sendo executado e não suspenso, fornecendo medidas de tempo de atividade precisas, tanto em valor absoluto quanto intervalo.

Disponibilidade: FreeBSD, OpenBSD >= 5.5.

Adicionado na versão 3.7.

time.CLOCK_UPTIME_RAW

Relógio que incrementa de forma monotônica, contando o tempo desde um ponto arbitrário, não afetado pela frequência ou ajustes de tempo e não incrementado enquanto o sistema está dormindo.

Disponibilidade: macOS >= 10.12.

Adicionado na versão 3.8.

time.CLOCK_UPTIME_RAW_APPROX

Semelhante a **`CLOCK_UPTIME_RAW`**, mas o valor é armazenado pelo sistema no gerenciador de contexto e, portanto, tem menor precisão.

Disponibilidade: macOS >= 10.12.

Adicionado na versão 3.13.

A constante a seguir é o único parâmetro que pode ser enviado para **`clock_settime()`**.

time.CLOCK_REALTIME

Relógio em tempo real. Configurar este relógio requer privilégios apropriados. O relógio é o mesmo para todos os processos.

Disponibilidade: Unix.

Adicionado na versão 3.3.

Constantes de Fuso Horário

time.altzone

O deslocamento do fuso horário DST local, em segundos a oeste de UTC, se algum for fornecido. É negativo se o fuso horário DST local está a leste de UTC (como na Europa Ocidental, incluindo o Reino Unido). Somente utilize se `daylight` for diferente de zero. Veja a nota abaixo.

time.daylight

Diferente de zero se um fuso horário DST é definido. Veja nota abaixo.

time.timezone

O deslocamento para o fuso horário local (não DST), em segundos a oeste de UTC (negativo na maior parte da Europa Ocidental, positivo nos Estados Unidos e Brasil, zero no Reino Unido). Ver nota abaixo.

time.tzname

A tupla de duas strings: A primeira é o nome do fuso horário local não DST, a segunda é o nome do fuso horário local DST. Se nenhum fuso horário DST for definido, a segunda string é usada. Veja nota abaixo.

Nota

Para as constantes de Fuso Horário acima (`altzone`, `daylight`, `timezone`, e `tzname`), o valor é determinado pelas regras de fuso horário em efeito no módulo de carregamento de tempo ou a última vez que `tzset()` é chamada e pode estar incorreto para tempos no passado. É recomendado utilizar os resultados `tm_gmtoff` e `tm_zone` da `localtime()` para obter informação de fuso horário.

Ver também

Módulo `datetime`
Mais interfaces orientada a objetos para datas e tempos.

Módulo `locale`
Serviços de internacionalização. A configuração de localidade afeta a interpretação de muitos especificadores de formato em `strftime()` e `strptime()`.

Módulo `calendar`
Funções gerais relacionadas a calendários. `timegm()` é a função inversa de `gmtime()` deste módulo.

Notas de rodapé

[1] (1,2,3)

A utilização de `%Z` está descontinuada, mas o escape `%z` que expande para um deslocamento hora/minuto preferido não é suportado por todas as bibliotecas ANSI C. Além disso, a leitura do padrão original **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] de 1982 mostra que este pede um ano

com dois dígitos (%y em vez de %Y), mas a prática consolidou a utilização de anos com 4 dígitos mesmo antes dos anos 2000. Após isso, o **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] tornou-se obsoleto e o ano de 4 dígitos foi primeiro recomendado por **RFC 1123** [https://datatracker.ietf.org/doc/html/rfc1123.html] e depois obrigatório por **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html], com **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] continuando este requisito.

`logging` — Recursos de registro de eventos para Python

Código-fonte: Lib/logging/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/logging/__init__.py]

Este módulo define funções e classes que implementam um registro flexível de eventos de sistema para aplicações e bibliotecas.

O principal benefício de ter a API de registro de eventos a partir de um módulo da biblioteca padrão é que todos os módulos Python podem participar no registro de eventos, de forma que sua aplicação pode incluir suas próprias mensagens, integradas com mensagens de módulos de terceiros.

> **Important**
>
> Esta página contêm informação de referência da API. Para tutorial e discussão de tópicos mais avançados, consulte
>
> - Tutorial básico
> - Tutorial avançado
> - Livro de receitas de logging

Aqui está um exemplo simples de uso idiomático:

```
# myapp.py
import logging
import mylib
logger = logging.getLogger(__name__)

def main():
    logging.basicConfig(filename='myapp.log', level=logging.INFO)
    logger.info('Started')
    mylib.do_something()
    logger.info('Finished')

if __name__ == '__main__':
    main()
```

```
# mylib.py
import logging
logger = logging.getLogger(__name__)

def do_something():
    logger.info('Doing something')
```

Se você executar *myapp.py*, deverá ver isso em *myapp.log*:

```
INFO:__main__:Started
INFO:mylib:Doing something
INFO:__main__:Finished
```

O ponto chave desse modo de uso idiomático é que a maior parte do código está simplesmente criando um registrador de nível de módulo com `getLogger(__name__)` e usando esse registrador para fazer qualquer registro necessário. Isso é conciso e, ao mesmo tempo, permite o controle refinado do código subjacente, se necessário. As mensagens registradas pelo registrador de nível de módulo são encaminhadas para tratadores de registradores em nível de módulo mais alto, até o registrador de nível mais alto conhecido como registrador raiz; essa abordagem é conhecida como registro hierárquico.

Para que o registro de eventos seja útil, ele precisa ser configurado: definir os níveis e os destinos de cada registrador, possivelmente alterando a forma como módulos específico fazem seus registros, geralmente com base em argumentos de linha de comando ou configuração da aplicação. Na maioria dos casos, como o descrito acima, somente o registrador raiz precisa ser configurado dessa forma, já que todos os registradores de nível inferior no nível módulo acabam encaminhando suas mensagens para seus tratadores. **`basicConfig()`** oferece uma maneira rápida de configurar o registrador raiz que lida com muitos casos de uso.

O módulo provê várias funcionalidades e flexibilidade. Se você não está familiarizado com registro de eventos, a melhor maneira para se ter uma noção sobre é ver os tutoriais (**veja os links acima e à direita**).

As classes básicas definidas pelo módulo, juntamente com seus atributos e métodos, estão listadas nas seções abaixo.

- Loggers expõem a interface que o código da aplicação usa diretamente.
- Handlers enviam os registros do evento (criados por loggers) aos destinos apropriados.
- Filters fornecem uma facilidade granular para determinar quais registros de eventos enviar à saída.
- Formatters especificam o layout dos registros de eventos na saída final.

Objetos Logger

Registradores têm os atributos e métodos a seguir. Observem que registradores *NUNCA* devem ser instanciados diretamente, mas sempre através da função `logging.getLogger(name)`. Múltiplas chamadas à função **`getLogger()`** com o mesmo nome sempre retornará uma referência para o mesmo objeto Logger.

The `name` is potentially a period-separated hierarchical value, like `foo.bar.baz` (though it could also be just plain `foo`, for example). Loggers that are further down in the hierarchical list are children of loggers higher up in the list. For example, given a logger with a name of `foo`, loggers with names of `foo.bar`, `foo.bar.baz`, and `foo.bam` are all descendants of `foo`. In addition, all loggers are descendants of the root logger. The logger name hierarchy is analogous to the Python package hierarchy, and identical to it if you organise your loggers on a per-module basis using the recommended construction `logging.getLogger(__name__)`. That's because in a module, `__name__` is the module's name in the Python package namespace.

class logging.Logger

name

This is the logger's name, and is the value that was passed to **`getLogger()`** to obtain the logger.

Nota

Este atributo deve ser tratado como somente leitura.

level

The threshold of this logger, as set by the **`setLevel()`** method.

Nota

Do not set this attribute directly - always use `setLevel()`, which has checks for the level passed to it.

parent

The parent logger of this logger. It may change based on later instantiation of loggers which are higher up in the namespace hierarchy.

Nota

Este valor deve ser tratado como somente leitura.

propagate

Se este atributo for verdadeiro, os eventos registrados nesse registrador serão transmitidos aos tratadores dos registradores de nível mais alto (ancestrais), além de quaisquer tratadores anexados a esse registrador. As mensagens são passadas diretamente para os tratadores dos registradores ancestrais - nem o nível nem os filtros dos registradores ancestrais em questão são considerados.

Se o valor for falso, as mensagens de registro não serão passadas para os tratadores de registradores ancestrais.

Explicando com um exemplo: se o atributo propagate do registrador chamado `A.B.C` for avaliado como verdadeiro, qualquer evento registrado em `A.B.C` pela chamada de métodos como `logging.getLogger('A.B.C').error(...)` será [sujeito à passagem das configurações de filtro e nível do registrador] passado por sua vez para qualquer tratador ligado aos registradores nomeados `A.B`, `A` e ao registrador raiz, após ser passado para qualquer tratador ligado ao `A.B.C`. Se qualquer registrador na cadeia `A.B.C`, `A.B`, `A` tem o atributo `propagate` definido como falso, então esse será o último registrador cujos tratadores terão os eventos oferecidos para serem tratados e a propagação terminará naquele ponto.

O construtor atribui este valor como `True`.

Nota

Se você configurar um tratador para o registrador *e* um ou mais dos ancestrais, pode acontecer que a mesma mensagem seja registrada várias vezes. Em geral, você não precisa configurar saídas para mais que um registrador - se você configurar apenas para o registrador principal da hierarquia, então todos os eventos dos registradores descendentes serão visualizados ali, fornecido pela configuração de propagação, cujo padrão é `True`. Um cenário comum é configurar as saídas somente no registrador raiz, e deixar a propagação tomar conta do resto.

handlers

The list of handlers directly attached to this logger instance.

Nota

This attribute should be treated as read-only; it is normally changed via the **`addHandler()`** and **`removeHandler()`** methods, which use locks to ensure thread-safe operation.

disabled

This attribute disables handling of any events. It is set to `False` in the initializer, and only changed by logging configuration code.

Nota

Este atributo deve ser tratado como somente leitura.

setLevel(*level*)

Ajuste o limite para este logger para *level*. Mensagens de registro de eventos que forem menos severas que este *level* serão ignoradas; mensagens que tenham nível de severidade igual ou maior que *level* serão emitidas para os destinos de saída configurados para o logger, a

menos que o nível da saída tenha sido configurado para uma severidade ainda maior.

Quando um logger é criado, o nível é definido como **NOTSET** (que faz com que todas as mensagens sejam processadas quando o logger for o logger raiz, ou delegação para o pai quando o logger não for um logger raiz). Observe que o logger raiz é criado com nível **WARNING**.

O termo “delegação ao pai” significa que o logger possui um nível de NOTSET, e a sua cadeia de loggers ancestrais será percorrida até que um ancestral com o nível diferente de NOTSET seja encontrado ou até a raiz ser alcançada.

Se um ancestral for achado com um nível diferente de NOTSET, então o nível daquele ancestral será tratado como o nível efetivo do logger que começou a busca por ancestrais, e é usado para determinar como um registro de evento será manipulado.

Se a raiz é alcançada e o seu nível é NOTSET, então todas as mensagens serão processadas. Caso contrário, o nível da raiz será usada como o nível efetivo.

Veja Níveis de Logging para uma lista de níveis.

Alterado na versão 3.2: O parâmetro *level* agora é aceito como uma string representando o nível tal qual ‘INFO’, como uma alternativa as constantes inteiras tal qual **INFO**. Note, entretanto, que os níveis são guardados internamente como inteiros e os métodos como, por exemplo, **getEffectiveLevel()** e **isEnabledFor()** retornarão/esperarão que sejam passados inteiros.

isEnabledFor(*level*)

Indica se a mensagem com gravidade *level* seria processada por esse logger. Esse método checa primeiro o nível à nível de módulo definido por `logging.disable(level)` e então o nível efetivo do logger como determinado por **getEffectiveLevel()**.

getEffectiveLevel()

Indica o nível efeito para esse logger. Se um valor diferente de **NOTSET** foi definido usando **setLevel()**, ele é retornado. Caso contrário, a hierarquia é percorrida em direção a raiz até um valor diferente de **NOTSET** ser encontrado e então esse valor é retornado. O valor retornado é um inteiro, tipicamento um entre **logging.DEBUG**, **logging.INFO** etc

getChild(*suffix*)

Retorna um logger que é descendente deste logger, como determinado pelo sufixo. Portanto, `logging.getLogger('abc').getChild('def.ghi')` retornaria o mesmo logger que seria retornado por `logging.getLogger('abc.def.ghi')`. Esse é um método de conveniência, útil quando o logger pai é nomeado usando, por exemplo `__name__` ao invés de uma string literal.

Adicionado na versão 3.2.

getChildren()

Returns a set of loggers which are immediate children of this logger. So for example `logging.getLogger().getChildren()` might return a set containing loggers named `foo` and `bar`, but a logger named `foo.bar` wouldn't be included in the set. Likewise, `logging.getLogger('foo').getChildren()` might return a set including a logger named `foo.bar`, but it wouldn't include one named `foo.bar.baz`.

Adicionado na versão 3.12.

debug(*msg*, **args*, ***kwargs*)

Logs a message with level **DEBUG** on this logger. The *msg* is the message format string, and the *args* are the arguments which are merged into *msg* using the string formatting operator. (Note that this means that you can use keywords in the format string, together with a single dictionary argument.) No % formatting operation is performed on *msg* when no *args* are supplied.

Existem quatro argumentos nomeados em *kwargs* que serão inspecionados: *exc_info*, *stack_info*, *stacklevel* e *extra*.

If *exc_info* does not evaluate as false, it causes exception information to be added to the logging message. If an exception tuple (in the format returned by `sys.exc_info()`) or an exception instance is provided, it is used; otherwise, `sys.exc_info()` is called to get the exception information.

The second optional keyword argument is *stack_info*, which defaults to `False`. If true, stack information is added to the logging message, including the actual logging call. Note that this is not the same stack information as that displayed through specifying *exc_info*: The former is stack frames from the bottom of the stack up to the logging call in the current thread, whereas the latter is information about stack frames which have been unwound, following an exception, while searching for exception handlers.

You can specify *stack_info* independently of *exc_info*, e.g. to just show how you got to a certain point in your code, even when no exceptions were raised. The stack frames are printed following a header line which says:

```
Stack (most recent call last):
```

Isso imita o `Traceback (most recent call last):` que é usado ao exibir quadros de exceção.

The third optional keyword argument is *stacklevel*, which defaults to `1`. If greater than 1, the corresponding number of stack frames are skipped when computing the line number and function name set in the `LogRecord` created for the logging event. This can be used in logging helpers so that the function name, filename and line number recorded are not the information for the helper function/method, but rather its caller. The name of this parameter mirrors the equivalent one in the `warnings` module.

The fourth keyword argument is *extra* which can be used to pass a dictionary which is used to populate the `__dict__` of the `LogRecord`

created for the logging event with user-defined attributes. These custom attributes can then be used as you like. For example, they could be incorporated into logged messages. For example:

```
FORMAT = '%(asctime)s %(clientip)-15s %(user)-8s %(message)
logging.basicConfig(format=FORMAT)
d = {'clientip': '192.168.0.1', 'user': 'fbloggs'}
logger = logging.getLogger('tcpserver')
logger.warning('Protocol problem: %s', 'connection reset',
```

imprimiria algo como

```
2006-02-08 22:20:02,165 192.168.0.1 fbloggs  Protocol probl
```

The keys in the dictionary passed in *extra* should not clash with the keys used by the logging system. (See the section on Atributos LogRecord for more information on which keys are used by the logging system.)

If you choose to use these attributes in logged messages, you need to exercise some care. In the above example, for instance, the `Formatter` has been set up with a format string which expects ‘clientip’ and ‘user’ in the attribute dictionary of the `LogRecord`. If these are missing, the message will not be logged because a string formatting exception will occur. So in this case, you always need to pass the *extra* dictionary with these keys.

While this might be annoying, this feature is intended for use in specialized circumstances, such as multi-threaded servers where the same code executes in many contexts, and interesting conditions which arise are dependent on this context (such as remote client IP address and authenticated user name, in the above example). In such circumstances, it is likely that specialized `Formatter`s would be used with particular `Handler`s.

If no handler is attached to this logger (or any of its ancestors, taking into account the relevant `Logger.propagate` attributes), the message will be sent to the handler set on `lastResort`.

Alterado na versão 3.2: O parâmetro *stack_info* foi adicionado.

Alterado na versão 3.5: The *exc_info* parameter can now accept exception instances.

Alterado na versão 3.8: O parâmetro *stacklevel* foi adicionado.

info(*msg*, **args*, ***kwargs*)

Logs a message with level `INFO` on this logger. The arguments are interpreted as for `debug()`.

warning(*msg*, **args*, ***kwargs*)

Logs a message with level `WARNING` on this logger. The arguments are interpreted as for `debug()`.

Nota

There is an obsolete method `warn` which is functionally identical to `warning`. As `warn` is deprecated, please do not use it - use `warning` instead.

error(*msg*, **args*, ***kwargs*)

Logs a message with level `ERROR` on this logger. The arguments are interpreted as for `debug()`.

critical(*msg*, **args*, ***kwargs*)

Logs a message with level `CRITICAL` on this logger. The arguments are interpreted as for `debug()`.

log(*level*, *msg*, **args*, ***kwargs*)

Logs a message with integer level *level* on this logger. The other arguments are interpreted as for `debug()`.

exception(*msg*, **args*, ***kwargs*)

Logs a message with level `ERROR` on this logger. The arguments are interpreted as for `debug()`. Exception info is added to the logging

message. This method should only be called from an exception handler.

addFilter(*filter*)

Adds the specified filter *filter* to this logger.

removeFilter(*filter*)

Removes the specified filter *filter* from this logger.

filter(*record*)

Apply this logger's filters to the record and return `True` if the record is to be processed. The filters are consulted in turn, until one of them returns a false value. If none of them return a false value, the record will be processed (passed to handlers). If one returns a false value, no further processing of the record occurs.

addHandler(*hdlr*)

Adiciona o tratador especificado por *hdlr* deste logger.

removeHandler(*hdlr*)

Remove o tratador especificado por *hdlr* deste logger.

findCaller(*stack_info=False, stacklevel=1*)

Finds the caller's source filename and line number. Returns the filename, line number, function name and stack information as a 4-element tuple. The stack information is returned as `None` unless *stack_info* is `True`.

The *stacklevel* parameter is passed from code calling the **`debug()`** and other APIs. If greater than 1, the excess is used to skip stack frames before determining the values to be returned. This will generally be useful when calling logging APIs from helper/wrapper code, so that the information in the event log refers not to the helper/wrapper code, but to the code that calls it.

handle(*record*)

Handles a record by passing it to all handlers associated with this logger and its ancestors (until a false value of *propagate* is found). This method is used for unpickled records received from a socket, as well as those created locally. Logger-level filtering is applied using `filter()`.

makeRecord(*name*, *level*, *fn*, *lno*, *msg*, *args*, *exc_info*, *func=None*, *extra=None*, *sinfo=None*)

This is a factory method which can be overridden in subclasses to create specialized `LogRecord` instances.

hasHandlers()

Checks to see if this logger has any handlers configured. This is done by looking for handlers in this logger and its parents in the logger hierarchy. Returns `True` if a handler was found, else `False`. The method stops searching up the hierarchy whenever a logger with the 'propagate' attribute set to false is found - that will be the last logger which is checked for the existence of handlers.

Adicionado na versão 3.2.

Alterado na versão 3.7: Loggers can now be pickled and unpickled.

Níveis de Logging

Os valores numéricos dos níveis de logging estão listados na tabela abaixo. Eles são principalmente de interesse se você quiser definir seus próprios níveis, e precisa deles para definir seus valores específicos relativos aos níveis predefinidos. Se você define um nível com o mesmo valor numérico, ele sobrescreve o valor predefinido; o nome predefinido é perdido.

Nível	Valor numérico	What it means / When to use it

Nível	Valor numérico	What it means / When to use it
logging.NOTSET	0	When set on a logger, indicates that ancestor loggers are to be consulted to determine the effective level. If that still resolves to `NOTSET`, then all events are logged. When set on a handler, all events are handled.
logging.DEBUG	10	Detailed information, typically only of interest to a developer trying to diagnose a problem.
logging.INFO	20	Confirmação de que as coisas estão funcionando como esperado.
logging.WARNING	30	An indication that something unexpected happened, or that a problem might occur in the near future (e.g. ‘disk space low’). The software is still working as expected.
logging.ERROR	40	Por conta de um problema mais grave, o software não conseguiu executar alguma função.
logging.CRITICAL	50	Um erro grave, indicando que o programa pode não conseguir continuar rodando.

Objetos Handler

Handlers have the following attributes and methods. Note that **`Handler`** is never instantiated directly; this class acts as a base for more useful subclasses. However, the **`__init__()`** method in subclasses needs to call **`Handler.__init__()`**.

class logging.Handler

__init__(*level=NOTSET*)

Initializes the **`Handler`** instance by setting its level, setting the list of filters to the empty list and creating a lock (using **`createLock()`**) for serializing access to an I/O mechanism.

createLock()

Initializes a thread lock which can be used to serialize access to underlying I/O functionality which may not be threadsafe.

acquire()

Acquires the thread lock created with **`createLock()`**.

release()

Releases the thread lock acquired with **`acquire()`**.

setLevel(*level*)

Sets the threshold for this handler to *level*. Logging messages which are less severe than *level* will be ignored. When a handler is created, the level is set to **`NOTSET`** (which causes all messages to be processed).

Veja Níveis de Logging para uma lista de níveis.

Alterado na versão 3.2: The *level* parameter now accepts a string representation of the level such as 'INFO' as an alternative to the integer constants such as **`INFO`**.

setFormatter(*fmt*)

Sets the formatter for this handler to *fmt*. The *fmt* argument must be a **`Formatter`** instance or `None`.

addFilter(*filter*)

Adds the specified filter *filter* to this handler.

removeFilter(*filter*)

Removes the specified filter *filter* from this handler.

filter(*record*)

Apply this handler's filters to the record and return `True` if the record is to be processed. The filters are consulted in turn, until one of them returns a false value. If none of them return a false value, the record will be emitted. If one returns a false value, the handler will not emit the record.

flush()

Ensure all logging output has been flushed. This version does nothing and is intended to be implemented by subclasses.

close()

Tidy up any resources used by the handler. This version does no output but removes the handler from an internal map of handlers, which is used for handler lookup by name.

Subclasses should ensure that this gets called from overridden **`close()`** methods.

handle(*record*)

Conditionally emits the specified logging record, depending on filters which may have been added to the handler. Wraps the actual emission of the record with acquisition/release of the I/O thread lock.

handleError(*record*)

This method should be called from handlers when an exception is encountered during an **`emit()`** call. If the module-level attribute **`raiseExceptions`** is `False`, exceptions get silently ignored. This is what is mostly wanted for a logging system - most users will not care about errors in the logging system, they are more interested in application errors. You could, however, replace this with a custom handler if you wish. The specified record is the one which was being

processed when the exception occurred. (The default value of **`raiseExceptions`** is `True`, as that is more useful during development).

format(*record*)

Do formatting for a record - if a formatter is set, use it. Otherwise, use the default formatter for the module.

emit(*record*)

Do whatever it takes to actually log the specified logging record. This version is intended to be implemented by subclasses and so raises a **`NotImplementedError`**.

Aviso

This method is called after a handler-level lock is acquired, which is released after this method returns. When you override this method, note that you should be careful when calling anything that invokes other parts of the logging API which might do locking, because that might result in a deadlock. Specifically:

- Logging configuration APIs acquire the module-level lock, and then individual handler-level locks as those handlers are configured.
- Many logging APIs lock the module-level lock. If such an API is called from this method, it could cause a deadlock if a configuration call is made on another thread, because that thread will try to acquire the module-level lock *before* the handler-level lock, whereas this thread tries to acquire the module-level lock *after* the handler-level lock (because in this method, the handler-level lock has already been acquired).

For a list of handlers included as standard, see **`logging.handlers`**.

Formatter Objects

class logging.Formatter(*fmt=None*, *datefmt=None*, *style='%'*, *validate=True*, ***, *defaults=None*)

Responsible for converting a **LogRecord** to an output string to be interpreted by a human or external system.

Parâmetros::

- **fmt** (*str*) – A format string in the given *style* for th logged output as a whole. The possible mapping keys are drawn from the **LogRecord** object's Atributos LogRecord. If not specified, `'%(message)s'` is used, which is just the logged message.
- **datefmt** (*str*) – A format string for the date/time portion of the logged output. If not specified, the default described in **formatTime()** is used.
- **style** (*str*) – Can be one of `'%'`, `'{'` or `'$'` and determines how the format string will be merged with its data: using one of Formatação de string no estilo printf (`%`), **str.format()** (`{`) or **string.Template** (`$`). This only applies to *fmt* (e.g `'%(message)s'` versus `'{message}'`), not to the actual log messages passed to the logging methods However, there are other ways to use `{`- and `$`-formatting for log messages.
- **validate** (*bool*) – If `True` (the default), incorrect or mismatched *fmt* and *style* will raise a **ValueError**; for example, `logging.Formatter('%(asctime)s %(message)s', style='{')`.
- **defaults** (*dict[str, Any]*) – A dictionary with default values to use in custom fields. For example `logging.Formatter('%(ip)s %(message)s', defaults={"ip": None})`

Alterado na versão 3.2: Adicionado o parâmetro *style*.

Alterado na versão 3.8: Adicionado o parâmetro *validate*.

Alterado na versão 3.10: Adicionado o parâmetro *defaults*.

format(*record*)

The record's attribute dictionary is used as the operand to a string formatting operation. Returns the resulting string. Before formatting the dictionary, a couple of preparatory steps are carried out. The *message* attribute of the record is computed using *msg* % *args*. If the formatting string contains `'(asctime)'`, `formatTime()` is called to format the event time. If there is exception information, it is formatted using `formatException()` and appended to the message. Note that the formatted exception information is cached in attribute *exc_text*. This is useful because the exception information can be pickled and sent across the wire, but you should be careful if you have more than one **`Formatter`** subclass which customizes the formatting of exception information. In this case, you will have to clear the cached value (by setting the *exc_text* attribute to `None`) after a formatter has done its formatting, so that the next formatter to handle the event doesn't use the cached value, but recalculates it afresh.

If stack information is available, it's appended after the exception information, using `formatStack()` to transform it if necessary.

formatTime(*record*, *datefmt=None*)

This method should be called from `format()` by a formatter which wants to make use of a formatted time. This method can be overridden in formatters to provide for any specific requirement, but the basic behavior is as follows: if *datefmt* (a string) is specified, it is used with `time.strftime()` to format the creation time of the record. Otherwise, the format '%Y-%m-%d %H:%M:%S,uuu' is used, where the uuu part is a millisecond value and the other letters are as per the **`time.strftime()`** documentation. An example time in this format is `2003-01-23 00:29:50,411`. The resulting string is returned.

This function uses a user-configurable function to convert the creation time to a tuple. By default, `time.localtime()` is used; to change this for a particular formatter instance, set the `converter` attribute to a function with the same signature as **`time.localtime()`** or `time.gmtime()`. To change it for all formatters, for example if you want all logging times to be shown in GMT, set the `converter` attribute in the `Formatter` class.

Alterado na versão 3.3: Previously, the default format was hard-coded as in this example: `2010-09-06 22:38:15,292` where the part before the comma is handled by a strptime format string (`'%Y-%m-%d %H:%M:%S'`), and the part after the comma is a millisecond value. Because strptime does not have a format placeholder for milliseconds, the millisecond value is appended using another format string, `'%s,%03d'` — and both of these format strings have been hardcoded into this method. With the change, these strings are defined as class-level attributes which can be overridden at the instance level when desired. The names of the attributes are `default_time_format` (for the strptime format string) and `default_msec_format` (for appending the millisecond value).

Alterado na versão 3.9: The `default_msec_format` can be `None`.

formatException(*exc_info*)

Formats the specified exception information (a standard exception tuple as returned by `sys.exc_info()`) as a string. This default implementation just uses `traceback.print_exception()`. The resulting string is returned.

formatStack(*stack_info*)

Formats the specified stack information (a string as returned by `traceback.print_stack()`, but with the last newline removed) as a string. This default implementation just returns the input value.

class logging.BufferingFormatter(*linefmt=None*)

A base formatter class suitable for subclassing when you want to format a number of records. You can pass a `Formatter` instance which you want to use to format each line (that corresponds to a single record). If not specified, the default formatter (which just outputs the event message) is used as the line formatter.

formatHeader(*records*)

Return a header for a list of *records*. The base implementation just returns the empty string. You will need to override this method if you

want specific behaviour, e.g. to show the count of records, a title or a separator line.

formatFooter(*records*)

Return a footer for a list of *records*. The base implementation just returns the empty string. You will need to override this method if you want specific behaviour, e.g. to show the count of records or a separator line.

format(*records*)

Return formatted text for a list of *records*. The base implementation just returns the empty string if there are no records; otherwise, it returns the concatenation of the header, each record formatted with the line formatter, and the footer.

Filter Objects

`Filters` can be used by `Handlers` and `Loggers` for more sophisticated filtering than is provided by levels. The base filter class only allows events which are below a certain point in the logger hierarchy. For example, a filter initialized with ‘A.B’ will allow events logged by loggers ‘A.B’, ‘A.B.C’, ‘A.B.C.D’, ‘A.B.D’ etc. but not ‘A.BB’, ‘B.A.B’ etc. If initialized with the empty string, all events are passed.

class logging.Filter(*name=''*)

Returns an instance of the **`Filter`** class. If *name* is specified, it names a logger which, together with its children, will have its events allowed through the filter. If *name* is the empty string, allows every event.

filter(*record*)

Is the specified record to be logged? Returns false for no, true for yes. Filters can either modify log records in-place or return a completely different record instance which will replace the original log record in any future processing of the event.

Note that filters attached to handlers are consulted before an event is emitted by the handler, whereas filters attached to loggers are consulted whenever an

event is logged (using **debug()**, **info()**, etc.), before sending an event to handlers. This means that events which have been generated by descendant loggers will not be filtered by a logger's filter setting, unless the filter has also been applied to those descendant loggers.

You don't actually need to subclass `Filter`: you can pass any instance which has a `filter` method with the same semantics.

Alterado na versão 3.2: You don't need to create specialized `Filter` classes, or use other classes with a `filter` method: you can use a function (or other callable) as a filter. The filtering logic will check to see if the filter object has a `filter` attribute: if it does, it's assumed to be a `Filter` and its **filter()** method is called. Otherwise, it's assumed to be a callable and called with the record as the single parameter. The returned value should conform to that returned by **filter()**.

Alterado na versão 3.12: You can now return a **LogRecord** instance from filters to replace the log record rather than modifying it in place. This allows filters attached to a **Handler** to modify the log record before it is emitted, without having side effects on other handlers.

Although filters are used primarily to filter records based on more sophisticated criteria than levels, they get to see every record which is processed by the handler or logger they're attached to: this can be useful if you want to do things like counting how many records were processed by a particular logger or handler, or adding, changing or removing attributes in the **LogRecord** being processed. Obviously changing the LogRecord needs to be done with some care, but it does allow the injection of contextual information into logs (see Using Filters to impart contextual information).

LogRecord Objects

LogRecord instances are created automatically by the **Logger** every time something is logged, and can be created manually via **makeLogRecord()** (for example, from a pickled event received over the wire).

class logging.LogRecord(*name*, *level*, *pathname*, *lineno*, *msg*, *args*, *exc_info*, *func=None*, *sinfo=None*)

Contains all the information pertinent to the event being logged.

The primary information is passed in *msg* and *args*, which are combined using `msg % args` to create the **`message`** attribute of the record.

Parâmetros::

- **name** (*str*) – The name of the logger used to log the event represented by this **`LogRecord`**. Note that the logger name in the **`LogRecord`** will always have this value, even though it may be emitted by a handler attached to a different (ancestor) logger.
- **level** (*int*) – The numeric level of the logging event (such as `10` for `DEBUG`, `20` for `INFO`, etc). Note that this is converted to *two* attributes of the LogRecord: **`levelno`** for the numeric value and **`levelname`** for the corresponding level name.
- **pathname** (*str*) – The full string path of the source file where the logging call was made.
- **lineno** (*int*) – The line number in the source file where the logging call was made.
- **msg** (*Any*) – The event description message, which can be a %-format string with placeholders for variable data, or an arbitrary object (see Usando objetos arbitrários como mensagens).
- **args** (*tuple* | *dict[str, Any]*) – Variable data to merge into the *msg* argument to obtain the event description.
- **exc_info** (*tuple[type[BaseException], BaseException, types.TracebackType]* | *None*) – An exception tuple with the current exception information, as returned by **`sys.exc_info()`**, or `None` if no exception information is available.
- **func** (*str* | *None*) – The name of the function or method from which the logging call was invoked.
- **sinfo** (*str* | *None*) – A text string representing stack information from the base of the stack in the current thread, up to the logging call.

getMessage()

Returns the message for this **`LogRecord`** instance after merging any user-supplied arguments with the message. If the user-supplied message argument to the logging call is not a string, **`str()`** is called on it to convert it to a string. This allows use of user-defined classes as messages, whose `__str__` method can return the actual format string to be used.

Alterado na versão 3.2: The creation of a **`LogRecord`** has been made more configurable by providing a factory which is used to create the record. The factory can be set using **`getLogRecordFactory()`** and **`setLogRecordFactory()`** (see this for the factory's signature).

This functionality can be used to inject your own values into a **`LogRecord`** at creation time. You can use the following pattern:

```
old_factory = logging.getLogRecordFactory()

def record_factory(*args, **kwargs):
    record = old_factory(*args, **kwargs)
    record.custom_attribute = 0xdecafbad
    return record

logging.setLogRecordFactory(record_factory)
```

With this pattern, multiple factories could be chained, and as long as they don't overwrite each other's attributes or unintentionally overwrite the standard attributes listed above, there should be no surprises.

Atributos LogRecord

The LogRecord has a number of attributes, most of which are derived from the parameters to the constructor. (Note that the names do not always correspond exactly between the LogRecord constructor parameters and the LogRecord attributes.) These attributes can be used to merge data from the record into the format string. The following table lists (in alphabetical order) the attribute names, their meanings and the corresponding placeholder in a %-style format string.

If you are using {}-formatting (`str.format()`), you can use `{attrname}` as the placeholder in the format string. If you are using $-formatting (`string.Template`), use the form `${attrname}`. In both cases, of course, replace `attrname` with the actual attribute name you want to use.

In the case of {}-formatting, you can specify formatting flags by placing them after the attribute name, separated from it with a colon. For example: a placeholder of `{msecs:03.0f}` would format a millisecond value of `4` as `004`. Refer to the `str.format()` documentation for full details on the options available to you.

Attribute name	Formatação	Descrição
args	You shouldn't need to format this yourself.	The tuple of arguments merged into `msg` to produce `message`, or a dict whose values are used for the merge (when there is only one argument, and it is a dictionary).
asctime	`%(asctime)s`	Human-readable time when the `LogRecord` was created. By default this is of the form '2003-07-08 16:49:45,896' (the numbers after the comma are millisecond portion of the time).
created	`%(created)f`	Time when the `LogRecord` was created (as returned by `time.time_ns()` / 1e9).

Attribute name	Formatação	Descrição
exc_info	You shouldn’t need to format this yourself.	Exception tuple (à la `sys.exc_info`) or, if no exception has occurred, `None`.
exc_text	You shouldn’t need to format this yourself.	Exception information formatted as a string. This is set when `Formatter.format()` is invoked, or `None` if no exception has occurred.
filename	`%(filename)s`	Filename portion of `pathname`.
funcName	`%(funcName)s`	Name of function containing the logging call.
levelname	`%(levelname)s`	Text logging level for the message (`'DEBUG'`, `'INFO'`, `'WARNING'`, `'ERROR'`, `'CRITICAL'`).
levelno	`%(levelno)s`	Numeric logging level for the message (`DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`).
lineno	`%(lineno)d`	Source line number where the logging call was issued (if available).

Attribute name	Formatação	Descrição
message	`%(message)s`	The logged message, computed as `msg % args`. This is set when `Formatter.format()` is invoked.
módulo	`%(module)s`	Module (name portion of `filename`).
msecs	`%(msecs)d`	Millisecond portion of the time when the `LogRecord` was created.
msg	You shouldn't need to format this yourself.	The format string passed in the original logging call. Merged with `args` to produce `message`, or an arbitrary object (see Usando objetos arbitrários como mensagens).
nome	`%(name)s`	Name of the logger used to log the call.
pathname	`%(pathname)s`	Full pathname of the source file where the logging call was issued (if available).
processo	`%(process)d`	Process ID (if available).

Attribute name	Formatação	Descrição
processName	`%(processName)s`	Process name (if available).
relativeCreated	`%(relativeCreated)d`	Time in milliseconds when the LogRecord was created, relative to the time the logging module was loaded.
stack_info	You shouldn't need to format this yourself.	Stack frame information (where available) from the bottom of the stack in the current thread, up to and including the stack frame of the logging call which resulted in the creation of this record.
thread	`%(thread)d`	Thread ID (if available).
threadName	`%(threadName)s`	Thread name (if available).
taskName	`%(taskName)s`	`asyncio.Task` name (if available).

Alterado na versão 3.1: processName was added.

Alterado na versão 3.12: taskName was added.

LoggerAdapter Objects

`LoggerAdapter` instances are used to conveniently pass contextual information into logging calls. For a usage example, see the section on

adding contextual information to your logging output.

class logging.LoggerAdapter(*logger*, *extra=None*, *merge_extra=False*)

Returns an instance of **`LoggerAdapter`** initialized with an underlying **`Logger`** instance, an optional dict-like object (*extra*), and an optional boolean (*merge_extra*) indicating whether or not the *extra* argument of individual log calls should be merged with the **`LoggerAdapter`** extra. The default behavior is to ignore the *extra* argument of individual log calls and only use the one of the **`LoggerAdapter`** instance

process(*msg*, *kwargs*)

Modifies the message and/or keyword arguments passed to a logging call in order to insert contextual information. This implementation takes the object passed as *extra* to the constructor and adds it to *kwargs* using key 'extra'. The return value is a (*msg*, *kwargs*) tuple which has the (possibly modified) versions of the arguments passed in.

manager

Delegates to the underlying **`manager`** on *logger*.

_log

Delegates to the underlying **`_log()`** method on *logger*.

In addition to the above, **`LoggerAdapter`** supports the following methods of **`Logger`**: **`debug()`**, **`info()`**, **`warning()`**, **`error()`**, **`exception()`**, **`critical()`**, **`log()`**, **`isEnabledFor()`**, **`getEffectiveLevel()`**, **`setLevel()`** and **`hasHandlers()`**. These methods have the same signatures as their counterparts in **`Logger`**, so you can use the two types of instances interchangeably.

Alterado na versão 3.2: The **`isEnabledFor()`**, **`getEffectiveLevel()`**, **`setLevel()`** and **`hasHandlers()`** methods were added to **`LoggerAdapter`**. These methods delegate to the underlying logger.

Alterado na versão 3.6: Attribute **`manager`** and method **`_log()`** were added, which delegate to the underlying logger and allow adapters to be nested.

Alterado na versão 3.10: The *extra* argument is now optional.

Alterado na versão 3.13: The *merge_extra* parameter was added.

Thread Safety

The logging module is intended to be thread-safe without any special work needing to be done by its clients. It achieves this through using threading locks; there is one lock to serialize access to the module's shared data, and each handler also creates a lock to serialize access to its underlying I/O.

If you are implementing asynchronous signal handlers using the `signal` module, you may not be able to use logging from within such handlers. This is because lock implementations in the `threading` module are not always re-entrant, and so cannot be invoked from such signal handlers.

Funções de Nível de Módulo

In addition to the classes described above, there are a number of module-level functions.

logging.getLogger(*name=None*)

Return a logger with the specified name or, if name is `None`, return the root logger of the hierarchy. If specified, the name is typically a dot-separated hierarchical name like *'a'*, *'a.b'* or *'a.b.c.d'*. Choice of these names is entirely up to the developer who is using logging, though it is recommended that `__name__` be used unless you have a specific reason for not doing that, as mentioned in Objetos Logger.

All calls to this function with a given name return the same logger instance. This means that logger instances never need to be passed between different parts of an application.

logging.getLoggerClass()

Return either the standard `Logger` class, or the last class passed to `setLoggerClass()`. This function may be called from within a new class

definition, to ensure that installing a customized **`Logger`** class will not undo customizations already applied by other code. For example:

```
class MyLogger(logging.getLoggerClass()):
    # ... override behaviour here
```

logging.getLogRecordFactory()

Return a callable which is used to create a **`LogRecord`**.

Adicionado na versão 3.2: This function has been provided, along with **`setLogRecordFactory()`**, to allow developers more control over how the **`LogRecord`** representing a logging event is constructed.

See **`setLogRecordFactory()`** for more information about the how the factory is called.

logging.debug(*msg*, **args*, ***kwargs*)

This is a convenience function that calls **`Logger.debug()`**, on the root logger. The handling of the arguments is in every way identical to what is described in that method.

The only difference is that if the root logger has no handlers, then **`basicConfig()`** is called, prior to calling `debug` on the root logger.

For very short scripts or quick demonstrations of `logging` facilities, `debug` and the other module-level functions may be convenient. However, most programs will want to carefully and explicitly control the logging configuration, and should therefore prefer creating a module-level logger and calling **`Logger.debug()`** (or other level-specific methods) on it, as described at the beginning of this documentation.

logging.info(*msg*, **args*, ***kwargs*)

Logs a message with level **`INFO`** on the root logger. The arguments and behavior are otherwise the same as for **`debug()`**.

logging.warning(*msg*, **args*, ***kwargs*)

Logs a message with level **`WARNING`** on the root logger. The arguments and behavior are otherwise the same as for **`debug()`**.

Nota

There is an obsolete function `warn` which is functionally identical to `warning`. As `warn` is deprecated, please do not use it - use `warning` instead.

logging.error(*msg*, **args*, ***kwargs*)

Logs a message with level **`ERROR`** on the root logger. The arguments and behavior are otherwise the same as for **`debug()`**.

logging.critical(*msg*, **args*, ***kwargs*)

Logs a message with level **`CRITICAL`** on the root logger. The arguments and behavior are otherwise the same as for **`debug()`**.

logging.exception(*msg*, **args*, ***kwargs*)

Logs a message with level **`ERROR`** on the root logger. The arguments and behavior are otherwise the same as for **`debug()`**. Exception info is added to the logging message. This function should only be called from an exception handler.

logging.log(*level*, *msg*, **args*, ***kwargs*)

Logs a message with level *level* on the root logger. The arguments and behavior are otherwise the same as for **`debug()`**.

logging.disable(*level=CRITICAL*)

Provides an overriding level *level* for all loggers which takes precedence over the logger's own level. When the need arises to temporarily throttle logging output down across the whole application, this function can be useful. Its effect is to disable all logging calls of severity *level* and below, so that if you call it with a value of INFO, then all INFO and DEBUG events would be discarded, whereas those of severity WARNING and above would be processed according to the logger's effective level. If `logging.disable(logging.NOTSET)` is called, it effectively removes this overriding level, so that logging output again depends on the effective levels of individual loggers.

Note that if you have defined any custom logging level higher than `CRITICAL` (this is not recommended), you won't be able to rely on the default value for the *level* parameter, but will have to explicitly supply a suitable value.

Alterado na versão 3.7: The *level* parameter was defaulted to level `CRITICAL`. See bpo-28524 [https://bugs.python.org/issue?@action=redirect&bpo=28524] for more information about this change.

logging.addLevelName(*level*, *levelName*)

Associates level *level* with text *levelName* in an internal dictionary, which is used to map numeric levels to a textual representation, for example when a **`Formatter`** formats a message. This function can also be used to define your own levels. The only constraints are that all levels used must be registered using this function, levels should be positive integers and they should increase in increasing order of severity.

Nota

If you are thinking of defining your own levels, please see the section on Níveis personalizados.

logging.getLevelNamesMapping()

Returns a mapping from level names to their corresponding logging levels. For example, the string "CRITICAL" maps to **`CRITICAL`**. The returned mapping is copied from an internal mapping on each call to this function.

Adicionado na versão 3.11.

logging.getLevelName(*level*)

Returns the textual or numeric representation of logging level *level*.

If *level* is one of the predefined levels **`CRITICAL`**, **`ERROR`**, **`WARNING`**, **`INFO`** or **`DEBUG`** then you get the corresponding string. If you have associated levels with names using **`addLevelName()`** then the name you have associated with *level* is returned. If a numeric value corresponding to one of the

defined levels is passed in, the corresponding string representation is returned.

The *level* parameter also accepts a string representation of the level such as 'INFO'. In such cases, this functions returns the corresponding numeric value of the level.

If no matching numeric or string value is passed in, the string 'Level %s' % level is returned.

Nota

Levels are internally integers (as they need to be compared in the logging logic). This function is used to convert between an integer level and the level name displayed in the formatted log output by means of the `%(levelname)s` format specifier (see Atributos LogRecord), and vice versa.

Alterado na versão 3.4: In Python versions earlier than 3.4, this function could also be passed a text level, and would return the corresponding numeric value of the level. This undocumented behaviour was considered a mistake, and was removed in Python 3.4, but reinstated in 3.4.2 in order to retain backward compatibility.

logging.getHandlerByName(*name*)

Returns a handler with the specified *name*, or `None` if there is no handler with that name.

Adicionado na versão 3.12.

logging.getHandlerNames()

Returns an immutable set of all known handler names.

Adicionado na versão 3.12.

logging.makeLogRecord(*attrdict*)

Creates and returns a new **`LogRecord`** instance whose attributes are defined by *attrdict*. This function is useful for taking a pickled **`LogRecord`**

attribute dictionary, sent over a socket, and reconstituting it as a **`LogRecord`** instance at the receiving end.

logging.basicConfig(**_kwargs_)

Does basic configuration for the logging system by either creating a **`StreamHandler`** with a default **`Formatter`** or using the given *formatter* instance, and adding it to the root logger. The functions **`debug()`**, **`info()`**, **`warning()`**, **`error()`** and **`critical()`** will call **`basicConfig()`** automatically if no handlers are defined for the root logger.

This function does nothing if the root logger already has handlers configured, unless the keyword argument *force* is set to `True`.

Nota

This function should be called from the main thread before other threads are started. In versions of Python prior to 2.7.1 and 3.2, if this function is called from multiple threads, it is possible (in rare circumstances) that a handler will be added to the root logger more than once, leading to unexpected results such as messages being duplicated in the log.

The following keyword arguments are supported.

Formatação	Descrição
filename	Specifies that a **`FileHandler`** be created, using the specified filename, rather than a **`StreamHandler`**.
filemode	If *filename* is specified, open the file in this mode. Defaults to `'a'`.
format	Use the specified format string for the handler. Defaults to attributes `levelname`, `name` and `message` separated by colons.

Formatação	Descrição
datefmt	Use the specified date/time format, as accepted by `time.strftime()`.
style	If *format* is specified, use this style for the format string. One of `'%'`, `'{'` or `'$'` for printf-style, `str.format()` or `string.Template` respectively. Defaults to `'%'`.
level	Set the root logger level to the specified level.
stream	Use the specified stream to initialize the `StreamHandler`. Note that this argument is incompatible with *filename* - if both are present, a `ValueError` is raised.
handlers	If specified, this should be an iterable of already created handlers to add to the root logger. Any handlers which don't already have a formatter set will be assigned the default formatter created in this function. Note that this argument is incompatible with *filename* or *stream* - if both are present, a `ValueError` is raised.
force	If this keyword argument is specified as true, any existing handlers attached to the root logger are removed and closed, before carrying out the configuration as specified by the other arguments.
encoding	If this keyword argument is specified along with *filename*, its value is used when the `FileHandler` is created, and thus used when opening the output file.

Formatação	Descrição
errors	If this keyword argument is specified along with *filename*, its value is used when the `FileHandler` is created, and thus used when opening the output file. If not specified, the value 'backslashreplace' is used. Note that if `None` is specified, it will be passed as such to `open()`, which means that it will be treated the same as passing 'errors'.
formatter	If specified, set this formatter instance (see Formatter Objects) for all involved handlers. If not specified, the default is to create and use an instance of `logging.Formatter` based on arguments *format*, *datefmt* and *style*. When *formatter* is specified together with any of the three arguments *format*, *datefmt* and *style*, a `ValueError` is raised to signal that these arguments would lose meaning otherwise.

Alterado na versão 3.2: O argumento *style* foi adicionado.

Alterado na versão 3.3: The *handlers* argument was added. Additional checks were added to catch situations where incompatible arguments are specified (e.g. *handlers* together with *stream* or *filename*, or *stream* together with *filename*).

Alterado na versão 3.8: O argumento *force* foi adicionado.

Alterado na versão 3.9: Os argumentos *encoding* e *errors* foram adicionados.

Alterado na versão 3.15: The *formatter* argument was added.

logging.shutdown()

Informs the logging system to perform an orderly shutdown by flushing and closing all handlers. This should be called at application exit and no further use of the logging system should be made after this call.

When the logging module is imported, it registers this function as an exit handler (see `atexit`), so normally there's no need to do that manually.

logging.setLoggerClass(*klass*)

Tells the logging system to use the class *klass* when instantiating a logger. The class should define `__init__()` such that only a name argument is required, and the `__init__()` should call `Logger.__init__()`. This function is typically called before any loggers are instantiated by applications which need to use custom logger behavior. After this call, as at any other time, do not instantiate loggers directly using the subclass: continue to use the `logging.getLogger()` API to get your loggers.

logging.setLogRecordFactory(*factory*)

Set a callable which is used to create a `LogRecord`.

Parâmetros::	**factory** – The factory callable to be used to instantiate a log record.

Adicionado na versão 3.2: This function has been provided, along with `getLogRecordFactory()`, to allow developers more control over how the `LogRecord` representing a logging event is constructed.

The factory has the following signature:

```
factory(name, level, fn, lno, msg, args, exc_info, func=None,
sinfo=None, **kwargs)
```

nome::	The logger name.
level::	The logging level (numeric).
fn::	The full pathname of the file where the logging call was made.
lno::	The line number in the file where the logging call was made.
msg::	The logging message.
args::	The arguments for the logging message.
exc_info::	An exception tuple, or `None`.
func::	The name of the function or method which invoked the logging call.

sinfo::	A stack traceback such as is provided by `traceback.print_stack()`, showing the call hierarchy.
kwargs::	Additional keyword arguments.

Module-Level Attributes

logging.lastResort

A “handler of last resort” is available through this attribute. This is a `StreamHandler` writing to `sys.stderr` with a level of `WARNING`, and is used to handle logging events in the absence of any logging configuration. The end result is to just print the message to `sys.stderr`. This replaces the earlier error message saying that “no handlers could be found for logger XYZ”. If you need the earlier behaviour for some reason, `lastResort` can be set to `None`.

Adicionado na versão 3.2.

logging.raiseExceptions

Used to see if exceptions during handling should be propagated.

Padrão: `True`.

If **`raiseExceptions`** is `False`, exceptions get silently ignored. This is what is mostly wanted for a logging system - most users will not care about errors in the logging system, they are more interested in application errors.

Integration with the warnings module

The `captureWarnings()` function can be used to integrate **`logging`** with the `warnings` module.

logging.captureWarnings(*capture*)

This function is used to turn the capture of warnings by logging on and off.

If *capture* is `True`, warnings issued by the `warnings` module will be redirected to the logging system. Specifically, a warning will be formatted using `warnings.formatwarning()` and the resulting string logged to a logger named `'py.warnings'` with a severity of `WARNING`.

If *capture* is `False`, the redirection of warnings to the logging system will stop, and warnings will be redirected to their original destinations (i.e. those in effect before `captureWarnings(True)` was called).

Ver também

Módulo `logging.config`
API de configuração para o módulo logging.

Módulo `logging.handlers`
Tratadores úteis incluídos no módulo logging.

PEP 282 [https://peps.python.org/pep-0282/] - A Logging System
The proposal which described this feature for inclusion in the Python standard library.

Original Python logging package [https://old.red-dove.com/python_logging.html]
This is the original source for the **`logging`** package. The version of the package available from this site is suitable for use with Python 1.5.2, 2.1.x and 2.2.x, which do not include the **`logging`** package in the standard library.

`logging.config` — Configuração do módulo logging

Código-fonte: Lib/logging/config.py
[https://github.com/python/cpython/tree/3.15/Lib/logging/config.py]

Esta seção descreve a API para configuração do módulo logging.

> **Important**
>
> Esta página contém apenas informações de referência. Para tutoriais, por favor consulte
>
> - Tutorial básico
> - Tutorial avançado
> - Livro de receitas do logging

Funções de configuração

As funções a seguir configuram o módulo logging. Elas estão localizadas no módulo **`logging.config`**. Seu uso é opcional — você pode configurar o módulo logging usando essas funções ou fazendo chamadas para a API principal (definido no próprio **`logging`**) e definindo manipuladores que são declarados em **`logging`** ou em **`logging.handlers`**.

logging.config.dictConfig(*config*)

Obtém a configuração de logging de um dicionário. Os conteúdos desse dicionário estão descritos abaixo em Esquema do Dicionário de Configuração.

Se um erro for encontrado durante a configuração, esta função levantará uma exceção **`ValueError`**, **`TypeError`**, **`AttributeError`** ou **`ImportError`** com uma mensagem adequadamente descritiva. A seguir está uma lista (possivelmente incompleta) de condições que levantarão um erro:

- Um `level` que não seja uma string ou que seja uma string que não corresponda a um nível de logging atual
- Um valor `propagate` que não seja um booleano.

- Um id que não tenha um destino correspondente.
- Um id de manipulador inexistente encontrado durante uma chamada incremental.
- Um nome de logger inválido.
- Incapacidade de resolver para um objeto interno ou externo.

A análise é realizada pela classe **`DictConfigurator`**, cujo construtor recebe o dicionário usado para a configuração e possui um método **`configure()`**. O módulo **`logging.config`** tem um atributo chamável **`dictConfigClass`** que é inicialmente definido como **`DictConfigurator`**. Você pode substituir o valor de **`dictConfigClass`** por uma implementação adequada de sua autoria.

A função **`dictConfig()`** chama **`dictConfigClass`** passando o dicionário especificado e, em seguida, chama o método **`configure()`** no objeto devolvido para colocar a configuração em efeito:

```
def dictConfig(config):
    dictConfigClass(config).configure()
```

Por exemplo, uma subclasse de **`DictConfigurator`** poderia chamar `DictConfigurator.__init__()` no seu próprio **`__init__()`**, em seguida, configurar prefixos personalizados que seriam utilizáveis na chamada **`configure()`** subsequente. **`dictConfigClass`** seria ligado a esta nova subclasse, e então **`dictConfig()`** poderia ser chamado exatamente como no estado padrão, sem personalização.

Adicionado na versão 3.2.

logging.config.fileConfig(*fname*, *defaults=None*, *disable_existing_loggers=True*, *encoding=None*)

Lê a configuração de logging de um arquivo no formato **`configparser`**. O formato do arquivo deve ser como descrito em Formato do arquivo de configuração. Esta função pode ser chamada várias vezes a partir de uma aplicação, permitindo que um usuário final selecione entre várias configurações pré-prontas (se o desenvolvedor fornecer um mecanismo para apresentar as escolhas e carregar a configuração escolhida).

A função levantará uma exceção `FileNotFoundError` se o arquivo não existir e `RuntimeError` se o arquivo for inválido ou vazio.

Parâmetros::

- **fname** – Um nome de arquivo, ou um objeto arquivo, ou uma instância derivada de `RawConfigParser`. Se uma instância derivada de `RawConfigParser` for passada, ela é usada como está. Caso contrário, uma instância de `ConfigParser` é criada, e a configuração é lida por ela a partir do objeto passado em `fname`. Se esse objeto tiver um método `readline()`, ele é presumido como um objeto arquivo ou similar e lido usando `read_file()`; caso contrário, é presumido como um nome de arquivo e passado para `read()`.
- **defaults** – Os defaults a serem passados para o `ConfigParser` podem ser especificados nesse argumento.
- **disable_existing_loggers** – Se especificado como `False`, os loggers que existem quando esta chamada é feita são deixados habilitados. O padrã é `True` porque isso habilita o comportamento antigo de forma retrocompatível. Este comportamento é para desabilitar quaisquer loggers não-raiz existentes, a menos que eles ou seus ancestrais sejam explicitamente nomeados na configuração de logging.
- **encoding** – A codificação usada para abrir o arquivo quando fname é um nome de arquivo.

Alterado na versão 3.4: Uma instância de uma subclasse de `RawConfigParser` agora é aceita como um valor para `fname`. Isso facilita:

- Uso de um arquivo de configuração onde a configuração de logging é apenas parte da configuração geral da aplicação.
- Uso de uma configuração lida de um arquivo, e então modificada pela aplicação que a usa (por exemplo, baseada em parâmetros de linha de comando ou outros aspectos do

ambiente de tempo de execução) antes de ser passada para `fileConfig`.

Alterado na versão 3.10: Adicionado o parâmetro encoding.

Alterado na versão 3.12: Uma exceção será levantada se o arquivo fornecido não existir ou for inválido ou for vazio.

logging.config.listen(*port=DEFAULT_LOGGING_CONFIG_PORT*, *verify=None*)

Inicia um servidor soquete na porta especificada e escuta por novas configurações. Se nenhuma porta for especificada, o padrão do módulo **`DEFAULT_LOGGING_CONFIG_PORT`** é usado. As configurações de logging serão enviadas como um arquivo adequado para processamento por **`dictConfig()`** ou por **`fileConfig()`**. Retorna uma instância de **`Thread`** na qual você pode chamar **`start()`** para iniciar o servidor, e à qual você pode **`join()`** quando apropriado. Para interromper o servidor, chame **`stopListening()`**.

O argumento `verify`, se especificado, deve ser um chamável que verifique se os bytes recebidos através do soquete são válidos e devem ser processados. Isso pode ser feito criptografando e/ou assinando o que é enviado através do soquete, de modo que o chamável `verify` possa realizar a verificação de assinatura e/ou a descriptografia. O chamável `verify` é chamado com um único argumento – os bytes recebidos através do soquete – e deve retornar os bytes a serem processados, ou `None` para indicar que os bytes devem ser descartados. Os bytes devolvidos podem ser os mesmos que os bytes passados (por exemplo, quando apenas a verificação é feita), ou podem ser completamente diferentes (talvez se a descriptografia for realizada).

Para enviar uma configuração para o soquete, leia o arquivo de configuração e o envie para o soquete como uma sequência de bytes precedida por uma string de comprimento de quatro bytes empacotada em binário usando `struct.pack('>L', n)`.

Nota

Como partes da configuração são passadas por **eval()**, o uso dessa função pode expor seus usuários a um risco de segurança. Embora a função apenas se vincule a um soquete em `localhost` e, portanto, não aceite conexões de máquinas remotas, existem cenários em que código não confiável pode ser executado sob a conta do processo que chama **listen()**. Especificamente, se o processo que chama **listen()** estiver sendo executado em uma máquina multiusuário onde os usuários não confiam uns nos outros, um usuário malicioso pode conseguir executar essencialmente código arbitrário no processo de um usuário vítima, simplesmente conectando-se ao soquete **listen()** da vítima e enviando uma configuração que execute qualquer código que o atacante queira que seja executado no processo da vítima. Isso é especialmente fácil de fazer se a porta padrão for usada, mas não é difícil mesmo se uma porta diferente for utilizada. Para evitar o risco de isso acontecer, use o argumento `verify` de **listen()** para prevenir que configurações não reconhecidas sejam aplicadas.

Alterado na versão 3.4: O argumento `verify` foi adicionado.

Nota

Se você deseja enviar configurações para o listener que não desabilitem os loggers existentes, você precisará usar um formato JSON para a configuração, que usará **dictConfig()** para configuração. Este método permite que você especifique `disable_existing_loggers` como `False` na configuração que você envia.

logging.config.stopListening()

Interrompe o servidor de escuta que foi criado com uma chamada para **listen()**. Essa função é tipicamente chamada antes de chamar **join()** no valor de retorno de **listen()**.

Considerações de segurança

A funcionalidade de configuração de logging tenta oferecer conveniência, e em parte isso é feito ao oferecer a capacidade de converter texto presente em

arquivos de configuração em objetos Python usados na configuração de logging — por exemplo, conforme descrito em Objetos definidos pelo usuário. No entanto, esses mesmos mecanismos (importar chamáveis de módulos definidos pelo usuário e chamá-los com parâmetros da configuração) poderiam ser usados para invocar qualquer código que você desejar, e por essa razão você deve tratar arquivos de configuração de fontes não confiáveis com extrema cautela e certificar-se de que nada de ruim pode acontecer se você carregá-los, antes de realmente carregá-los.

Esquema do Dicionário de Configuração

Descrever uma configuração de logging requer listar os vários objetos a serem criados e as conexões entre eles; por exemplo, você pode criar um manipulador nomeado ‘console’ e, em seguida, dizer que o logger nomeado ‘startup’ enviará suas mensagens para o manipulador ‘console’. Estes objetos não se limitam àqueles fornecidos pelo módulo `logging`, pois você pode escrever sua própria classe de formatador ou de manipulador. Os parâmetros para essas classes também podem precisar incluir objetos externos, como `sys.stderr`. A sintaxe para descrever esses objetos e conexões é definida abaixo em Conexões de objeto.

Detalhes do Esquema de Dicionário

O dicionário passado para `dictConfig()` deve conter as chaves a seguir:

- version - deve ser definido como um valor inteiro representando a versão do esquema. O único valor válido atualmente é 1, mas ter esta chave permite que o esquema evolua, preservando ainda a retrocompatibilidade.

Todas as outras chaves são opcionais, mas se estiverem presentes, serão interpretadas conforme descrito abaixo. Em todos os casos abaixo onde um ‘dicionário de configuração’ é mencionado, ele será verificado quanto à chave especial `'()'` para consultar se uma instanciação personalizada é necessária. Se for o caso, o mecanismo descrito em Objetos definidos pelo usuário abaixo é usado para criar uma instância; caso contrário, o contexto é usado para determinar o que instanciar.

- formatters - o valor correspondente será um dicionário em que cada chave é um id de formatador e cada valor é um dicionário que descreve como configurar a instância correspondente de `Formatter`.

  O dicionário de configuração é buscado pelas chaves opcionais a seguir que correspondem aos argumentos passados para criar um objeto `Formatter`:

  - `format`
  - `datefmt`
  - `style`
  - `validate` (desde a versão >=3.8)
  - `defaults` (desde versão >=3.12)

  Uma chave opcional `class` indica o nome da classe do formatador (como um módulo pontilhado e nome da classe). Os argumentos de instanciação são os mesmos que para `Formatter` portanto, esta chave é mais útil para instanciar uma subclasse personalizada de :`Formatter`. Por exemplo, a classe alternativa pode apresentar tracebacks de exceção em um formato expandido ou condensado. Se o seu formatter exigir chaves de configuração diferentes ou extras, você deve usar Objetos definidos pelo usuário.

- filters - o valor correspondente será um dicionário no qual cada chave é um id de filter e cada valor é um dicionário que descreve como configurar a instância Filter correspondente.

  O dicionário de configuração é pesquisado pela chave `name` (que assume como padrão a string vazia) e esta é usada para construir uma instância de `logging.Filter`.

- handlers - o valor correspondente será um dicionário no qual cada chave é um id de manipulador e cada valor é um dicionário que descreve como configurar a instância de Manipulador correspondente.

  O dicionário de configuração é pesquisado para as chaves a seguir:

  - `class` (obrigatório). Esse é o nome qualificado completo da classe do handler.

- `level` (opcional). O nível do manipulador.
- `formatter` (opcional). O id do formatador para esse manipulador.
- `filters` (opcional). Uma lista de ids dos filtros para esse manipulador.

  Alterado na versão 3.11: `filters` podem receber instâncias de filtro além de ids.

Todas as *outras* chaves são transmitidas como argumento nomeado para o construtor do manipulador. Por exemplo, dado o trecho:

```
handlers:
  console:
    class : logging.StreamHandler
    formatter: brief
    level   : INFO
    filters: [allow_foo]
    stream  : ext://sys.stdout
  file:
    class : logging.handlers.RotatingFileHandler
    formatter: precise
    filename: logconfig.log
    maxBytes: 1024
    backupCount: 3
```

o manipulador com o id `console` é instanciado como um `logging.StreamHandler`, usando `sys.stdout` como o fluxo subjacente. O manipulador com o id `file` é instanciado como um `logging.handlers.RotatingFileHandler` com os argumentos nomeados `filename='logconfig.log', maxBytes=1024, backupCount=3`.

- *loggers* - o valor correspondente será um dicionário no qual cada chave é o nome de um registradores e cada valor é um dicionário que descreve como configurar a instância de Registrador correspondente.

  O dicionário de configuração é pesquisado para as chaves a seguir:

  - `level` (opcional). O nível do logger.

- `propagate` (opcional). A configuração de propagação do logger.
- `filters` (opcional). Uma lista de ids dos filtros para esse logger.

  Alterado na versão 3.11: `filters` podem receber instâncias de filtro além de ids.

- `handlers` (opcional). Uma lista de ids dos manipuladores para esse logger.

Os loggers especificados serão configurados de acordo com o nível, a propagação, os filtros e os manipuladores especificados.

- *root* - essa será a configuração para o logger raiz. O processamento da configuração será igual ao de qualquer logger, exceto pelo fato de que a configuração `propagate` não será aplicável.

- *incremental* - se a configuração deve ser interpretada como incremental à configuração existente. O padrão deste valor é `False`, o que significa que a configuração especificada substitui a configuração existente com a mesma semântica usada pela configuração da API existente **`fileConfig()`**.

  Se o valor especificado for `True`, a configuração será processada conforme descrito na seção Configuração Incremental.

- *disable_existing_loggers* - se quaisquer loggers não raiz existentes devem ser desabilitados. Essa configuração reflete o parâmetro de mesmo nome em **`fileConfig()`**. Se ausente, o padrão deste parâmetro é `True`. Este valor é ignorado se *incremental* for `True`.

Configuração Incremental

É difícil oferecer total flexibilidade para a configuração incremental. Por exemplo, como objetos, assim como filtros e formatadores, são anônimos, depois que uma configuração é definida, não é possível fazer referência a esse objeto anônimo ao ampliar uma configuração.

Além disso, não há um argumento convincente para alterar arbitrariamente o grafo de objetos de loggers, manipuladores, filtros e formatadores em tempo de execução, depois que uma configuração é definida; a verbosidade de loggers e manipuladores pode ser controlada simplesmente ajustando os níveis (e, no caso de loggers, os sinalizadores de propagação). Alterar o grafo de objetos arbitrariamente de forma segura é problemático em um ambiente multithread; embora não seja impossível, os benefícios não compensam a complexidade que isso acrescentaria à implementação.

Assim, quando a chave `incremental` de um dicionário de configuração estiver presente e for `True`, o sistema ignorará completamente quaisquer entradas `formatters` e `filters`, e processará apenas as configurações de `level` nas entradas `handlers`, e as configurações de `level` e `propagate` nas entradas `loggers` e `root`.

Usar um valor no dicionário de configuração permite que configurações sejam enviadas pela rede como dicionários pickled para um listener soquete. Assim, a verbosidade de logging de uma aplicação de longa execução pode ser alterada ao longo do tempo sem a necessidade de parar e reiniciar a aplicação.

Conexões de objeto

O esquema descreve um conjunto de objetos de logging — loggers, manipuladores, formatadores, filtros — que são conectados uns aos outros em um grafo de objetos. Assim, o esquema precisa representar conexões entre os objetos. Por exemplo, suponha que, após configurado, um determinado logger tenha anexado a si um determinado manipulador. Para os propósitos desta discussão, podemos dizer que o logger representa a origem e o manipulador representa o destino de uma conexão entre os dois. É claro que, nos objetos configurados, isso é representado pelo logger mantendo uma referência ao manipulador. No dicionário de configuração, isso é feito atribuindo a cada objeto de destino um id que o identifica de forma inequívoca e, em seguida, usando esse id na configuração do objeto de origem para indicar que existe uma conexão entre o objeto de origem e o objeto de destino com aquele id.

Então, por exemplo, considere o seguinte trecho de YAML:

```
formatters:
  brief:
    # configuration for formatter with id 'brief' goes here
  precise:
    # configuration for formatter with id 'precise' goes here
handlers:
  h1: #This is an id
   # configuration of handler with id 'h1' goes here
   formatter: brief
  h2: #This is another id
   # configuration of handler with id 'h2' goes here
   formatter: precise
loggers:
  foo.bar.baz:
    # other configuration for logger 'foo.bar.baz'
    handlers: [h1, h2]
```

(Observação: o YAML é usado aqui porque é um pouco mais legível do que o formulário de origem Python equivalente para o dicionário.)

Os ids para loggers são os nomes dos loggers que seriam usados programaticamente para obter uma referência a esses loggers, por exemplo, `foo.bar.baz`. Os ids para Formatadores e Filtros podem ser qualquer valor de string (como `brief`, `precise` acima) e são transitórios, no sentido de que só têm significado durante o processamento do dicionário de configuração e são usados para determinar conexões entre objetos, não sendo persistidos em nenhum lugar após a conclusão da chamada de configuração.

O trecho acima indica que o logger nomeado `foo.bar.baz` deve ter dois manipuladores anexados a ele, que são descritos pelos IDs de manipulador `h1` e `h2`. O formatador para `h1` é o descrito pelo id `brief`, e o formatador para `h2` é o descrito pelo id `precise`.

Objetos definidos pelo usuário

O esquema oferece suporte a objetos definidos pelo usuário para manipuladores, filtros e formatadores. (Os loggers não precisam ter tipos diferentes para instâncias diferentes, portanto este esquema de configuração não provê suporte às classes de loggers definidas pelo usuário.)

Objetos a serem configurados são descritos por dicionários que detalham sua configuração. Em alguns casos, o sistema de logging consegue inferir pelo contexto como um objeto deve ser instanciado, mas, quando se trata de um objeto definido pelo usuário, o sistema não saberá como fazer isso. Para oferecer total flexibilidade na instanciação desses objetos definidos pelo usuário, é necessário fornecer uma factory — um chamável que recebe um dicionário de configuração e retorna o objeto instanciado. Isso é sinalizado por um caminho de importação absoluto para a factory, disponibilizado sob a chave especial `'()'`. Aqui está um exemplo concreto:

```
formatters:
  brief:
    format: '%(message)s'
  default:
    format: '%(asctime)s %(levelname)-8s %(name)-15s %(message)s'
    datefmt: '%Y-%m-%d %H:%M:%S'
  custom:
      (): my.package.customFormatterFactory
      bar: baz
      spam: 99.9
      answer: 42
```

O trecho YAML acima define três formatadores. O primeiro, com id `brief`, é uma instância padrão de **`logging.Formatter`** com a string de formato especificado. O segundo, com id `default`, tem um formato mais longo e também define explicitamente o formato de tempo, e resultará em um **`logging.Formatter`** inicializado com essas duas strings de formatação. Em código-fonte Python, os formatadores `brief` e `default` têm subdicionários de configuração:

```
{
  'format' : '%(message)s'
}
```

e:

```
{
  'format' : '%(asctime)s %(levelname)-8s %(name)-15s %(message)s
  'datefmt' : '%Y-%m-%d %H:%M:%S'
}
```

respectivamente; e como esses dicionários não contêm a chave especial `'()'`, a instanciação é inferida pelo contexto. Como resultado, são criadas instâncias padrão de **`logging.Formatter`**. O subdicionário de configuração do terceiro formatador, com o id `custom`, é:

```
{
  '()' : 'my.package.customFormatterFactory',
  'bar' : 'baz',
  'spam' : 99.9,
  'answer' : 42
}
```

e isso contém a chave especial `'()'`, o que significa que se deseja uma instanciação definida pelo usuário. Nesse caso, a função factory especificada será usada. Se for um chamável real, ele será usado diretamente — caso contrário, se você especificar uma string (como no exemplo), o chamável real será localizado usando os mecanismos normais de importação. O callable será chamado com os itens restantes no subdicionário de configuração como argumentos nomeados. No exemplo acima, assume-se que o formatter com id `custom` seja retornado pela chamada:

```
my.package.customFormatterFactory(bar='baz', spam=99.9, answer=42
```

Aviso

Os valores de chaves como `bar`, `spam` e `answer` no exemplo acima não devem ser dicionários de configuração nem referências como `cfg://foo`, ou `ext://bar` porque eles não serão processados pelo maquinário de configuração — serão simplesmente passados para a função callable como estão.

A chave `'()'` foi usada como chave especial porque não é um nome válido de parâmetro nomeado e, portanto, não conflita com os nomes dos argumentos nomeados usados na chamada. O `'()'` também serve como um mnemônico de que o valor correspondente é um chamável.

Alterado na versão 3.11: O membro `filters` de `handlers` e `loggers` pode receber instâncias de filtro, além de ids.

Você também pode especificar uma chave especial `'.'` cujo valor é um mapeamento de nomes de atributos para valores. Se essa chave for encontrada, os atributos especificados serão definidos no objeto definido pelo usuário antes que ele seja retornado. Assim, com a seguinte configuração:

```
{
  '()' : 'my.package.customFormatterFactory',
  'bar' : 'baz',
  'spam' : 99.9,
  'answer' : 42,
  '.' : {
    'foo': 'bar',
    'baz': 'bozz'
  }
}
```

o formatador retornado terá o atributo `foo` definido como `'bar'` e o atributo `baz` definido como `'bozz'`.

Aviso

Os valores de atributos como `foo` e `baz` no exemplo acima não devem ser dicionários de configuração nem referências como `cfg://foo` ou `ext://bar` porque eles não serão processados pelo maquinário de configuração, mas definidos como valores de atributo exatamente como foram fornecidos.

Ordem de configuração de manipuladores

Os manipuladores são configurados em ordem alfabética de suas chaves, e um manipulador configurado substitui o dicionário de configuração no (cópia de trabalho do) dicionário `handlers` no esquema. Se você usar uma construção como `cfg://handlers.foo`, então inicialmente `handlers['foo']` aponta para o dicionário de configuração do manipulador nomeado `foo` e, posteriormente (quando esse manipulador tiver sido configurado), passa a apontar para a instância configurada do manipulador. Assim, `cfg://handlers.foo` pode ser resolvido tanto como um dicionário quanto como uma instância de manipulador. Em geral, é prudente nomear os

manipuladores de modo que os manipuladores dependentes sejam configurados depois de quaisquer manipuladores dos quais dependam; isso permite que algo como `cfg://handlers.foo` seja usado na configuração de um manipulador que depende do handler `foo`. Se esse manipulador dependente fosse nomeado `bar`, surgiriam problemas, pois a configuração de `bar` seria tentada antes da de `foo`, e `foo` ainda não teria sido configurado. No entanto, se o manipulador dependente fosse nomeado `foobar`, ele seria configurado após `foo`, com o resultado de que `cfg://handlers.foo` seria resolvido para o manipulador `foo` configurado, e não para o seu dicionário de configuração.

Acesso a objetos externos

Há momentos em que uma configuração precisa se referir a objetos externos à configuração, por exemplo `sys.stderr`. Se o dicionário de configuração for construído usando código Python, isso é simples, mas surge um problema quando a configuração é fornecida por meio de um arquivo texto (por exemplo, JSON, YAML). Em um arquivo texto, não há uma forma padrão de distinguir `sys.stderr` da string literal `'sys.stderr'`. Para facilitar essa distinção, o sistema de configuração procura por determinados prefixos especiais em valores de string e os trata de forma especial. Por exemplo, se a string literal `'ext://sys.stderr'` for fornecida como um valor na configuração, então o prefixo `ext://` será removido e o restante do valor será processado usando os mecanismos normais de importação.

O tratamento desses prefixos é feito de forma análoga ao tratamento de protocolos: existe um mecanismo genérico para procurar prefixos que correspondam à expressão regular ^(?P<prefix>[a-z]+)://(?P<suffix>.*), de modo que, se o `prefix` for reconhecido, o `suffix` é processado de uma forma dependente do prefixo e o resultado desse processamento substitui o valor da string. Se o prefixo não for reconhecido, então o valor da string é mantido como está.

Acesso a objetos internos

Além de objetos externos, às vezes também é necessário fazer referência a objetos na configuração. Isso será feito de forma implícita pelo sistema de configuração para coisas que ele conhece. Por exemplo, o valor em forma de

string `'DEBUG'` para um `level` em um logger ou manipulador será automaticamente convertido para o valor `logging.DEBUG`, e as entradas `handlers`, `filters` e `formatter` receberão um identificador de objeto e o resolverão para o objeto de destino apropriado.

No entanto, é necessário um mecanismo mais genérico para objetos definidos pelo usuário que não são conhecidos pelo módulo **`logging`**. Por exemplo, considere **`logging.handlers.MemoryHandler`**, que recebe um argumento `target` que é outro manipulador para o qual delegar. Como o sistema já conhece essa classe, então, na configuração, o `target` fornecido só precisa ser o id do objeto do manipulador de destino relevante, e o sistema resolverá esse id para o manipulador correspondente. Se, porém, um usuário definir um `my.package.MyHandler` que tenha um manipulador `alternate`, o sistema de configuração não saberia que `alternate` se refere a um manipulador. Para atender a esse caso, um sistema genérico de resolução permite que o usuário especifique:

```
handlers:
  file:
    # configuration of file handler goes here

  custom:
    (): my.package.MyHandler
    alternate: cfg://handlers.file
```

A cadeia literal `'cfg://handlers.file'` será resolvida de maneira análoga às strings com o prefixo `ext://` mas procurando na própria configuração em vez do espaço de nomes de importação. O mecanismo permite acesso por ponto ou por índice, de forma semelhante ao que é fornecido por `str.format`. Assim, dado o seguinte trecho:

```
handlers:
  email:
    class: logging.handlers.SMTPHandler
    mailhost: localhost
    fromaddr: my_app@domain.tld
    toaddrs:
      - support_team@domain.tld
      - dev_team@domain.tld
    subject: Houston, we have a problem.
```

Na configuração, a string `'cfg://handlers'` seria resolvida para o dicionário com a chave `handlers`, a string `'cfg://handlers.email` seria resolvida para o dicionário com a chave `email` dentro do dicionário `handlers`, e assim por diante. A string `'cfg://handlers.email.toaddrs[1]` seria resolvida para `'dev_team@domain.tld'` e a string `'cfg://handlers.email.toaddrs[0]'` seria resolvida para o valor `'support_team@domain.tld'`. O valor de `subject` poderia ser acessado usando `'cfg://handlers.email.subject'` ou, de forma equivalente, `'cfg://handlers.email[subject]'`. Esta última forma só precisa ser utilizada se a chave contiver espaços ou caracteres não alfanuméricos. Observe que os caracteres `[` e `]` não são permitidos nas chaves. Se um valor de índice consistir apenas em dígitos decimais, o acesso será tentado usando o valor inteiro correspondente, recorrendo ao valor de string caso necessário.

Dada uma string `cfg://handlers.myhandler.mykey.123`, ela será resolvida como `config_dict['handlers']['myhandler']['mykey']['123']`. Se a string for especificada como `cfg://handlers.myhandler.mykey[123]`, o sistema tentará obter o valor de `config_dict['handlers']['myhandler']['mykey'][123]` e, caso isso falhe, recorrerá a `config_dict['handlers']['myhandler']['mykey']['123']`.

Resolução de importações e importadores personalizados

Por padrão, a resolução de importação utiliza a função embutida **`__import__()`** para realizar as importações. Caso deseje substituir esse comportamento por seu próprio mecanismo de importação, você pode alterar o atributo **`importer`** da classe **`DictConfigurator`** ou de sua superclasse, **`BaseConfigurator`**. No entanto, é preciso ter cuidado devido à forma como funções são acessadas a partir de classes por meio de descritores. Se você estiver utilizando um objeto chamável do Python para realizar as importações e quiser defini-lo em nível de classe, em vez de em nível de instância, deverá envolvê-lo com **`staticmethod()`**. Por exemplo:

```
from importlib import import_module
from logging.config import BaseConfigurator

BaseConfigurator.importer = staticmethod(import_module)
```

Você não precisa envolver com `staticmethod()` se estiver definindo o chamável de importação em uma *instância* do configurador.

Configurando QueueHandler e QueueListener

Se você deseja configurar um `QueueHandler` — lembrando que ele é normalmente utilizado em conjunto com um `QueueListener` —, pode configurar ambos simultaneamente. Após a configuração, a instância do `QueueListener` estará disponível como o atributo `listener` do manipulador criado; este, por sua vez, poderá ser acessado utilizando `getHandlerByName()` e passando o nome atribuído ao `QueueHandler` na sua configuração. O esquema de dicionário para configurar o par é apresentado no exemplo de trecho YAML abaixo.

```
handlers:
  qhand:
    class: logging.handlers.QueueHandler
    queue: my.module.queue_factory
    listener: my.package.CustomListener
    handlers:
      - hand_name_1
      - hand_name_2
      ...
```

As chaves `queue` e `listener` são opcionais.

Se a chave `queue` estiver presente, o valor correspondente pode ser um dos seguintes:

- Um objeto que implementa a API pública `Queue.put_nowait` e `Queue.get`. Por exemplo, pode ser uma instância real de `queue.Queue` ou uma subclasse dela, ou um proxy obtido por meio de `multiprocessing.managers.SyncManager.Queue()`.

  É claro que isso só é possível se você estiver construindo ou modificando o dicionário de configuração no código.

- Uma string que é resolvida para um chamável que, quando chamado sem argumentos, retorna a instância de fila a ser usada. Esse chamável

pode ser uma subclasse de **queue.Queue** ou uma função que retorna uma instância de fila adequada, como `my.module.queue_factory()`.

- Um dicionário com a chave `'()'` que é construído da forma usual discutida em Objetos definidos pelo usuário O resultado dessa construção deve ser uma instância de **queue.Queue**.

Se a chave `queue` estiver ausente, uma instância padrão e não limitada de **queue.Queue** será criada e usada.

Se a chave `listener` estiver presente, o valor correspondente pode ser um dos seguintes:

- Uma subclasse de **logging.handlers.QueueListener**. Isso, naturalmente, só é possível se você estiver construindo ou modificando o dicionário de configuração em código.
- Uma string que é resolvida para uma classe que seja uma subclasse de `QueueListener` como `'my.package.CustomListener'`.
- Um dicionário com a chave `'()'`, construído da maneira usual conforme discutido em Objetos definidos pelo usuário O resultado dessa construção deve ser um chamável com a mesma assinatura do inicializador de `QueueListener`.

Se a chave `listener` estiver ausente, será usada **logging.handlers.QueueListener**.

Os valores sob a chave `handlers` são os nomes de outros manipuladores na configuração (não mostrados no trecho acima) que serão passados ao queue listener.

Quaisquer classes personalizadas de manipulador de filas e de listener precisarão ser definidas com as mesmas assinaturas de inicialização que **QueueHandler** e **QueueListener**.

Adicionado na versão 3.12.

Formato do arquivo de configuração

O formato do arquivo de configuração reconhecido por **fileConfig()** baseia-se na funcionalidade do módulo **configparser**. O arquivo deve conter seções chamadas `[loggers]`, `[handlers]` e `[formatters]`, que identificam pelo nome as entidades de cada tipo definidas no arquivo. Para cada uma dessas entidades, existe uma seção separada que especifica como a entidade está configurada. Assim, para um logger chamado `log01` na seção `[loggers]`, os detalhes de configuração relevantes são mantidos em uma seção `[logger_log01]`. Da mesma forma, um handler chamado `hand01` na seção `[handlers]` terá sua configuração mantida em uma seção chamada `[handler_hand01]`, enquanto um formatador chamado `form01` na seção `[formatters]` terá sua configuração especificada em uma seção chamada `[formatter_form01]`. A configuração do logger raiz deve ser especificada em uma seção chamada `[logger_root]`.

Nota

A API **fileConfig()** é mais antiga que a API **dictConfig()** e não oferece funcionalidade para cobrir certos aspectos do registro de logs (*logging*). Por exemplo, não é possível configurar objetos **Filter** — que permitem filtrar mensagens além dos simples níveis numéricos (inteiros) — utilizando a **fileConfig()**. Se você precisar de instâncias de **Filter** em sua configuração de log, deverá utilizar a **dictConfig()**. Vale ressaltar que futuros aprimoramentos nas funcionalidades de configuração serão incorporados à **dictConfig()**; portanto, considere migrar para essa API mais recente quando for conveniente.

Exemplos dessas seções no arquivo são fornecidos abaixo.

```
[loggers]
keys=root,log02,log03,log04,log05,log06,log07

[handlers]
keys=hand01,hand02,hand03,hand04,hand05,hand06,hand07,hand08,hand

[formatters]
keys=form01,form02,form03,form04,form05,form06,form07,form08,form
```

O logger raiz deve especificar um nível e uma lista de manipuladores. Um exemplo de seção de logger raiz é dado abaixo.

```
[logger_root]
level=NOTSET
handlers=hand01
```

A entrada `level` pode ser `DEBUG, INFO, WARNING, ERROR, CRITICAL` ou `NOTSET`. Apenas para o logger raiz, `NOTSET` significa que todas as mensagens serão registradas. Os valores de nível são avaliados no contexto do espaço de nomes do pacote `logging`.

A entrada `handlers` é uma lista separada por vírgulas com os nomes dos manipuladores, que devem aparecer na seção `[handlers]`. Esses nomes precisam aparecer na seção `[handlers]` e ter seções correspondentes no arquivo de configuração.

Para loggers que não sejam o logger raiz, são necessárias algumas informações adicionais. Isso é ilustrado pelo exemplo a seguir.

```
[logger_parser]
level=DEBUG
handlers=hand01
propagate=1
qualname=compiler.parser
```

As entradas `level` e `handlers` são interpretadas da mesma forma que o logger raiz, exceto que, se o nível de um logger não raiz for especificado como `NOTSET`, o sistema consulta os loggers situados acima na hierarquia para determinar o nível efetivo do logger. O item `propagate` é definido como 1 para indicar que as mensagens devem ser propagadas para os manipuladores acima na hierarquia de loggers a partir deste logger, ou 0 para indicar que as mensagens não são propagadas para manipuladores acima na hierarquia. O item `qualname` é o nome de canal hierárquico do logger, ou seja, o nome usado pela aplicação para obter o logger.

As seções que especificam a configuração de manipuladores são exemplificadas pelo seguinte.

```
[handler_hand01]
class=StreamHandler
level=NOTSET
formatter=form01
args=(sys.stdout,)
```

A entrada `class` indica a classe do manipulador (conforme determinada por **`eval()`** no espaço de nomes do pacote `logging`). O `level` é interpretado da mesma forma que para loggers, e `NOTSET` é entendido como “registrar tudo”.

A entrada `formatter` indica o nome da chave do formatador para esse manipulador. Se estiver em branco, um formatador padrão (`logging._defaultFormatter`) será usado. Se um nome for especificado, ele deve aparecer na seção `[formatters]` e ter uma seção correspondente no arquivo de configuração.

A entrada `args`, quando avaliada no contexto do espaço de nomes do pacote `logging`, é a lista de argumentos para o construtor da classe de manipulador (*handler*). Consulte os construtores dos manipuladores relevantes, ou os exemplos abaixo, para ver como as entradas típicas são construídas. Se não for fornecida, o padrão é `()`.

A entrada opcional `kwargs`, quando avaliada no contexto do espaço de nomes do pacote `logging`, é o dicionário de argumentos nomeados para o construtor da classe de manipulador (*handler*). Se não for fornecida, o padrão é `{}`.

```
[handler_hand02]
class=FileHandler
level=DEBUG
formatter=form02
args=('python.log', 'w')

[handler_hand03]
class=handlers.SocketHandler
level=INFO
formatter=form03
args=('localhost', handlers.DEFAULT_TCP_LOGGING_PORT)

[handler_hand04]
class=handlers.DatagramHandler
level=WARN
```

```
formatter=form04
args=('localhost', handlers.DEFAULT_UDP_LOGGING_PORT)

[handler_hand05]
class=handlers.SysLogHandler
level=ERROR
formatter=form05
args=(('localhost', handlers.SYSLOG_UDP_PORT), handlers.SysLogHan

[handler_hand06]
class=handlers.NTEventLogHandler
level=CRITICAL
formatter=form06
args=('Python Application', '', 'Application')

[handler_hand07]
class=handlers.SMTPHandler
level=WARN
formatter=form07
args=('localhost', 'from@abc', ['user1@abc', 'user2@xyz'], 'Logge
kwargs={'timeout': 10.0}

[handler_hand08]
class=handlers.MemoryHandler
level=NOTSET
formatter=form08
target=
args=(10, ERROR)

[handler_hand09]
class=handlers.HTTPHandler
level=NOTSET
formatter=form09
args=('localhost:9022', '/log', 'GET')
kwargs={'secure': True}
```

As seções que especificam a configuração do formatador são caracterizadas pelo seguinte.

```
[formatter_form01]
format=F1 %(asctime)s %(levelname)s %(message)s %(customfield)s
datefmt=
style=%
```

```
validate=True
defaults={'customfield': 'defaultvalue'}
class=logging.Formatter
```

Os argumentos para a configuração do formatador são os mesmos que as chaves no esquema do dicionário da seção de formatadores.

A entrada `defaults`, quando avaliada no contexto do espaço de nomes do pacote `logging`, é um dicionário de valores padrão para campos de formatação personalizados. Se não for fornecida, o padrão é `None`.

Nota

Devido ao uso de **`eval()`** conforme descrito acima, há riscos de segurança potenciais decorrentes do uso de **`listen()`** para enviar e receber configurações por meio de soquetes. Os riscos se limitam a situações em que vários usuários sem confiança mútua executam código na mesma máquina; consulte a documentação de **`listen()`** para mais informações.

Ver também

Módulo `logging`
Referência da API para o módulo de logging.

Módulo `logging.handlers`
Manipuladores úteis incluídos no módulo logging.

`logging.handlers` — Manipuladores de logging

Código-fonte: Lib/logging/handlers.py
[https://github.com/python/cpython/tree/3.15/Lib/logging/handlers.py]

Os seguintes tratadores úteis são fornecidos neste pacote. Observe que três dos tratadores (`StreamHandler`, `FileHandler` e `NullHandler`) são, na verdade, definidos no próprio módulo `logging`, mas foram documentados aqui junto com os outros tratadores.

Important

Esta página contém apenas informações de referência. Para tutoriais, por favor consulte

- Tutorial básico
- Tutorial avançado
- Livro de receitas de logging

StreamHandler

A classe `StreamHandler` localizada no pacote principal `logging`, envia a saída de log para fluxos como *sys.stdout*, *sys.stderr* ou qualquer objeto arquivo ou similar (ou, mais precisamente, qualquer objeto que provê os métodos `write()` e `flush()`).

class logging.StreamHandler(*stream=None*)

Retorna uma nova instância da classe `StreamHandler` Se *stream* for especificado, a instância irá utilizá-lo para a saída de log; caso contrário, *sys.stderr* será usado.

emit(*record*)

Se um formatador for especificado, ele será usado para formatar o registro. Em seguida, o registro é escrito no fluxo, seguido por `terminator` Se houver informações de exceção, elas são formatadas usando `traceback.print_exception()` e anexadas ao fluxo.

flush()

Esvazia o fluxo chamando seu método **flush()**. Observe que o método **close()** é herdado de **Handler** e, portanto, não produz nenhuma saída, de modo que uma chamada explícita a **flush()** pode ser necessária em alguns casos.

setStream(*stream*)

Define o fluxo da instância para o valor especificado, caso seja diferente. O fluxo antigo é esvaziado antes que o novo fluxo seja definido.

Parâmetros::	**stream** – O fluxo que o manipulador deve usar.
Retorna::	o fluxo antigo, se o fluxo tiver sido alterado, ou `None`, se não tiver sido.

Adicionado na versão 3.7.

terminator

String usada como terminador ao gravar um registro formatado em um fluxo. O valor padrão é `'\n'`.

Se você não quiser uma terminação com nova linha, pode definir o atributo `terminator` da instância do manipulador como uma string vazia.

Nas versões anteriores, o terminador era codificado diretamente como `'\n'`.

Adicionado na versão 3.2.

FileHandler

A classe **FileHandler**, localizada no pacote principal **logging**, envia a saída de logging para um arquivo em disco. Ela herda a funcionalidade de saída da classe **StreamHandler**.

class logging.FileHandler(*filename*, *mode='a'*, *encoding=None*, *delay=False*, *errors=None*)

Retorna uma nova instância da classe `FileHandler`. O arquivo especificado é aberto e usado como o fluxo para logging. Se *mode* não for especificado, `'a'` é usado. Se *encoding* não for `None`, ele é usado para abrir o arquivo com essa codificação. Se *delay* for verdadeiro, então a abertura do arquivo é adiada até a primeira chamada de `emit()` Por padrão, o arquivo cresce indefinidamente. Se *errors* for especificado, ele é usado para determinar como os erros de codificação serão tratados.

Alterado na versão 3.6: Além de valores do tipo string, objetos `Path` também são aceitos para o argumento *filename*.

Alterado na versão 3.9: O parâmetro *errors* foi adicionado.

close()

Fecha o arquivo.

emit(*record*)

Envia o registro para o arquivo.

Observe que, se o arquivo tiver sido fechado devido ao encerramento do sistema de logging na saída e o modo do arquivo for 'w', o registro não será emitido (veja bpo-42378 [https://bugs.python.org/issue?@action=redirect&bpo=42378]).

NullHandler

Adicionado na versão 3.1.

A classe `NullHandler`, localizada no pacote principal `logging`, não realiza qualquer formatação ou saída. Ela é essencialmente um manipulador 'no-op', destinado ao uso por desenvolvedores de bibliotecas.

class logging.NullHandler

Retorna uma nova instância da classe `NullHandler`.

emit(*record*)

Esse método não faz nada.

handle(*record*)

Esse método não faz nada.

createLock()

Este método retorna `None` para a trava, pois não há E/S subjacente cujo acesso precise ser serializado.

Consulte Configurando logging para uma biblioteca para obter mais informações sobre como usar **`NullHandler`**.

WatchedFileHandler

The **`WatchedFileHandler`** class, located in the **`logging.handlers`** module, is a **`FileHandler`** which watches the file it is logging to. If the file changes, it is closed and reopened using the file name.

Uma alteração no arquivo pode ocorrer devido ao modo de usar de programas como *newsyslog* e *logrotate*, que realizam a rotação de arquivos de log. Este manipulador, destinado ao uso em Unix/Linux, monitora o arquivo para verificar se ele foi alterado desde a última chamada a emit. (Considera-se que um arquivo foi alterado se o seu dispositivo ou nó-i tiverem mudado.) Se o arquivo tiver sido alterado, o fluxo do arquivo antigo é fechado, e o arquivo é aberto novamente para obter um novo fluxo.

Este manipulador não é apropriado para uso no Windows, porque, nesse sistema, arquivos de log abertos não podem ser movidos ou renomeados — o logging abre os arquivos com travas exclusivas — e, portanto, não há necessidade desse tipo de manipulador. Além disso, *ST_INO* não é suportado no Windows; **`stat()`** sempre retorna zero para esse valor.

class logging.handlers.WatchedFileHandler(*filename*, *mode='a'*, *encoding=None*, *delay=False*, *errors=None*)

Retorna uma nova instância da classe **`WatchedFileHandler`**. O arquivo especificado é aberto e usado como o fluxo para registro de logs. Se *mode* não for especificado, `'a'` será usado. Se encoding não for `None`, ele será usado para abrir o arquivo com essa codificação. Se *delay* for verdadeiro, a abertura do arquivo é adiada até a primeira chamada a **`emit()`**. Por

padrão, o arquivo cresce indefinidamente. Se *errors* for fornecido, ele determina como erros de codificação são tratados.

Alterado na versão 3.6: Além de valores do tipo string, objetos `Path` também são aceitos para o argumento *filename*.

Alterado na versão 3.9: O parâmetro *errors* foi adicionado.

reopenIfNeeded()

Verifica se o arquivo foi alterado. Se tiver sido, o fluxo existente é descarregado e fechado, e o arquivo é aberto novamente, normalmente como um passo preliminar antes de enviar o registro para o arquivo.

Adicionado na versão 3.6.

emit(*record*)

Emite o registro para o arquivo, mas antes chama `reopenIfNeeded()` para reabrir o arquivo caso ele tenha sido alterado.

BaseRotatingHandler

The `BaseRotatingHandler` class, located in the `logging.handlers` module, is the base class for the rotating file handlers, `RotatingFileHandler` and `TimedRotatingFileHandler`. You should not need to instantiate this class, but it has attributes and methods you may need to override.

class logging.handlers.BaseRotatingHandler(*filename*, *mode*, *encoding=None*, *delay=False*, *errors=None*)

Os parâmetros são os mesmos de `FileHandler`. Os atributos são:

namer

Se esse atributo for definido como um chamável, o método `rotation_filename()` delega a ele. Os parâmetros passados ao chamável são os mesmos passados para `rotation_filename()`.

Nota

A função namer é chamada muitas vezes durante a rotação, portanto deve ser a mais simples e rápida possível. Ela também deve retornar sempre o mesmo resultado para uma dada entrada; caso contrário, o comportamento da rotação pode não funcionar como esperado.

Também vale observar que é preciso ter cuidado ao usar um namer para preservar certos atributo no nome do arquivo que são usados durante a rotação. Por exemplo, `RotatingFileHandler` espera ter um conjunto de arquivos de log cujos nomes contenham inteiros sucessivos, para que a rotação funcione como esperado, e `TimedRotatingFileHandler` remova arquivos de log antigos (com base no parâmetro `backupCount` passado ao inicializador do manipulador) determinando quais são os arquivos mais antigos a serem excluídos. Para que isso funcione, os nomes dos arquivos devem ser ordenáveis usando a porção de data/hora do nome, e um namer precisa respeitar isso. (Se for desejado um namer que não respeite esse esquema, ele deverá ser usado em uma subclasse de **`TimedRotatingFileHandler`** que substitua o método `getFilesToDelete()` para se adequar ao esquema de nomeação personalizado.)

Adicionado na versão 3.3.

rotator

Se este atributo estiver definido como um chamável, o método `rotate()` delega a esse chamável. Os parâmetros passados ao chamável são os mesmos passados para **`rotate()`**.

Adicionado na versão 3.3.

rotation_filename(*default_name*)

Modifica o nome do arquivo de log durante a rotação.

Isso é fornecido para que um nome de arquivo personalizado possa ser providenciado.

A implementação padrão chama o atributo namer do manipulador, se ele for chamável, passando o nome padrão para ele. Se o atributo não

for chamável (o padrão é `None`), o nome é devolvido sem alterações.

Parâmetros:: **default_name** – O nome padrão para o arquivo de log.

Adicionado na versão 3.3.

rotate(*source*, *dest*)

Quando ocorre a rotação, rotaciona o log atual.

A implementação padrão chama o atributo rotator do manipulador, se ele for chamável, passando os argumentos source e dest para ele. Se o atributo não for chamável (o padrão é `None`), o arquivo de origem é simplesmente renomeado para o destino.

Parâmetros::

- **source** – O nome do arquivo de origem. Normalmente, este é o nome base do arquivo, por exemplo, 'test.log'.
- **dest** – O nome do arquivo de destino. Normalmente, é para onde o arquivo de origem é rotacionado, por exemplo, 'test.log.1'.

Adicionado na versão 3.3.

O motivo da existência desses atributos é evitar que você precise criar subclasses — é possível usar os mesmos chamáveis para instâncias de **`RotatingFileHandler`** e **`TimedRotatingFileHandler`**. Se o chamável namer ou rotator levantar uma exceção, ela será tratada da mesma forma que qualquer outra exceção durante uma chamada a **`emit()`**, isto é, por meio do método **`handleError()`** do manipulador.

Se precisar fazer alterações mais significativas no processamento de rotação, você pode substituir os métodos.

Para um exemplo, consulte Using a rotator and namer to customize log rotation processing.

RotatingFileHandler

The **RotatingFileHandler** class, located in the **logging.handlers** module, supports rotation of disk log files.

class logging.handlers.RotatingFileHandler(*filename*, *mode='a'*, *maxBytes=0*, *backupCount=0*, *encoding=None*, *delay=False*, *errors=None*)

Retorna uma nova instância da classe **RotatingFileHandler**. O arquivo especificado é aberto e usado como o stream para logging. Se *mode* não for especificado, `'a'` é usado. Se *encoding* não for `None`, ele é usado para abrir o arquivo com essa codificação. Se *delay* for verdadeiro, a abertura do arquivo é adiada até a primeira chamada a **emit()**. Por padrão, o arquivo cresce indefinidamente. Se *errors* for fornecido, ele é usado para determinar como erros de codificação são tratados.

Você pode usar os valores *maxBytes* e *backupCount* para permitir que o arquivo faça *rollover* em um tamanho predeterminado. Quando o tamanho está prestes a ser excedido, o arquivo é fechado e um novo arquivo é aberto silenciosamente para saída. A rotação ocorre sempre que o arquivo de log atual está próximo de *maxBytes* em comprimento; mas, se *maxBytes* ou *backupCount* for zero, a rotação nunca ocorre, portanto, em geral, você deve definir *backupCount* como pelo menos 1 e usar um *maxBytes* diferente de zero. Quando *backupCount* é diferente de zero, o sistema salva arquivos de log antigos acrescentando as extensões ‘.1’, ‘.2’ etc. ao nome do arquivo. Por exemplo, com um *backupCount* de 5 e um nome de arquivo base `app.log`, você terá `app.log`, `app.log.1`, `app.log.2`, até `app.log.5`. O arquivo no qual se escreve é sempre `app.log`. Quando esse arquivo é preenchido, ele é fechado e renomeado para `app.log.1` e, se existirem os arquivos `app.log.1`, `app.log.2` etc., eles são renomeados para `app.log.2`, `app.log.3` etc., respectivamente.

Alterado na versão 3.6: Além de valores do tipo string, objetos **Path** também são aceitos para o argumento *filename*.

Alterado na versão 3.9: O parâmetro *errors* foi adicionado.

doRollover()

Realiza uma rotação, conforme descrito acima.

emit(*record*)

Grava o registro no arquivo, tratando a rotação conforme descrito anteriormente.

shouldRollover(*record*)

Verifica se o registro fornecido faria o arquivo exceder o limite de tamanho configurado.

TimedRotatingFileHandler

The `TimedRotatingFileHandler` class, located in the **`logging.handlers`** module, supports rotation of disk log files at certain timed intervals.

class logging.handlers.TimedRotatingFileHandler(*filename*, *when='h'*, *interval=1*, *backupCount=0*, *encoding=None*, *delay=False*, *utc=False*, *atTime=None*, *errors=None*)

Retorna uma nova instância da classe `TimedRotatingFileHandler`. O arquivo especificado é aberto e usado como fluxo para o logging. Durante a rotação, o sufixo do nome do arquivo também é definido. A rotação ocorre com base no produto de *when* e *interval*.

Você pode usar o *when* para especificar o tipo de *intervalo*. A lista de possíveis valores está abaixo. Observe que eles não diferenciam maiúsculas de minúsculas.

Valor	**Tipo de intervalo**	**Se/como *atTime* é usado**
`'S'`	Segundos	Ignorado
`'M'`	Minutos	Ignorado
`'H'`	Horas	Ignorado
`'D'`	Dias	Ignorado

Valor	Tipo de intervalo	Se/como *atTime* é usado
`'W0'-'W6'`	Dia da semana (0=Segunda)	Usado para calcular o tempo inicial de rotação
`'midnight'`	Faz a rotação à meia-noite, se *atTime* não for especificado; caso contrário, no horário *atTime*.	Usado para calcular o tempo inicial de rotação

Ao usar rotação baseada em dia da semana, especifique ‘W0’ para segunda-feira, ‘W1’ para terça-feira, e assim por diante até ‘W6’ para domingo. Nesse caso, o valor passado para *interval* não é utilizado.

O sistema salvará arquivos de log antigos acrescentando extensões ao nome do arquivo. As extensões são baseadas em data e hora, usando o formato strftime `%Y-%m-%d_%H-%M-%S` ou uma porção inicial dele, dependendo do intervalo de rollover.

Quando a próximo rotação é calculada pela primeira vez (no momento em que o manipulador é criado), o tempo da última modificação de um arquivo de log existente — ou, caso contrário, o tempo atual — é usado para determinar quando a próxima rotação ocorrerá.

Se o argumento *utc* for verdadeiro, serão usados horários em Horário Universal Coordenado (UTC); caso contrário, será usado o horário local.

Se *backupCount* for diferente de zero, no máximo *backupCount* arquivos serão mantidos, e, se mais arquivos fossem criados quando ocorrer a rotação, o mais antigo será excluído. A lógica de exclusão usa o interval para determinar quais arquivos devem ser removidos; assim, alterar o intervalo pode deixar arquivos antigos remanescentes.

Se *delay* for verdadeiro, a abertura do arquivo será adiada até a primeira chamada para `emit()`.

Se *atTime* não for `None`, ele deve ser uma instância de `datetime.time` que especifica a hora do dia em que ocorre a rotação, nos casos em que a rotação está configurada para acontecer “à meia-noite” ou “em um determinado dia da semana”. Observe que, nesses casos, o valor de *atTime* é efetivamente usado para calcular a rotação inicial, e as rotações subsequentes são calculadas por meio do cálculo normal do intervalo.

Se *erros* for especificado, ele será usado para determinar como erros de codificação são tratados.

Nota

O cálculo do tempo da rotação inicial é feito quando o manipulador é inicializado. O cálculo dos tempos de rotações subsequentes é feito apenas quando ocorre uma rotação, e a rotação ocorre somente quando há emissão de saída. Se isso não for levado em consideração, pode causar alguma confusão. Por exemplo, se for definido um intervalo de “a cada minuto”, isso não significa que você sempre verá arquivos de log com horários (no nome do arquivo) separados por um minuto; se, durante a execução da aplicação, a saída de log for gerada com maior frequência do que uma vez por minuto, então você pode esperar observar arquivos de log com horários separados por um minuto. Se, por outro lado, as mensagens de log forem emitidas apenas uma vez a cada cinco minutos (por exemplo), então haverá lacunas nos horários dos arquivos correspondentes aos minutos em que não houve saída (e, portanto, nenhuma rotação).

Alterado na versão 3.4: parâmetro *atTime* foi adicionado.

Alterado na versão 3.6: Além de valores do tipo string, objetos `Path` também são aceitos para o argumento *filename*.

Alterado na versão 3.9: O parâmetro *errors* foi adicionado.

doRollover()

Realiza uma rotação, conforme descrito acima.

emit(*record*)

Emite o registro para o arquivo, com a preparação para a rotação, conforme descrito acima.

getFilesToDelete()

Retorna uma lista de nomes de arquivos que devem ser excluídos como parte da rotação. Esses são os caminhos absolutos dos arquivos de log de backup mais antigos gravados pelo manipulador.

shouldRollover(*record*)

Verifica se passou tempo suficiente para que a rotação ocorra e, se for o caso, calcula o próximo horário da rotação.

SocketHandler

The `SocketHandler` class, located in the **`logging.handlers`** module, sends logging output to a network socket. The base class uses a TCP socket.

class logging.handlers.SocketHandler(*host*, *port*)

Retorna uma nova instância da classe `SocketHandler`, destinada a se comunicar com uma máquina remota cujo endereço é fornecido por *host* e *port*.

Alterado na versão 3.4: Se `port` for especificado como `None`, um Soquete de domínio Unix será criado usando o valor em `host` - caso contrário, um soquete TCP será criado.

close()

Fecha o soquete.

emit()

Serializa com pickle o dicionário de atributos do registro e o grava no soquete em formato binário. Se houver um erro com o soquete, o pacote é descartado silenciosamente. Se a conexão tiver sido perdida anteriormente, ela é restabelecida. Para desserializar com pickle o registro no lado receptor em um `LogRecord`, use a função `makeLogRecord()`.

handleError()

Trata um erro ocorrido durante `emit()`. A causa mais provável é a perda da conexão. Fecha o soquete para que seja possível tentar novamente no próximo evento.

makeSocket()

Este é um método de fábrica que permite que subclasses definam o tipo exato de soquete que desejam. A implementação padrão cria um soquete TCP (`socket.SOCK_STREAM`).

makePickle(*record*)

Serializa com pickle o dicionário de atributos do registro em formato binário com um prefixo de comprimento e o retorna pronto para transmissão através do socket. Os detalhes dessa operação são equivalentes a:

```
data = pickle.dumps(record_attr_dict, 1)
datalen = struct.pack('>L', len(data))
return datalen + data
```

Observe que os pickles não são completamente seguros. Se você estiver preocupado com a segurança, pode ser necessário substituir este método para implementar um mecanismo mais seguro. Por exemplo, você pode assinar os pickles usando HMAC e depois verificá-los no lado receptor, ou alternativamente pode desativar a desserialização com pickle de objetos globais no lado receptor.

send(*packet*)

Envia para o soquete uma string de bytes serializada com pickle denominada *packet*. O formato da string de bytes enviada é o descrito na documentação de `makePickle()`.

Este função permite envios parciais, que podem ocorrer quando a rede está ocupada.

createSocket()

Tenta criar um soquete; em caso de falha, utiliza um algoritmo de back-off exponencial. Na falha inicial, o manipulador descarta a

mensagem que estava tentando enviar. Quando mensagens subsequentes são tratadas pela mesma instância, ela não tentará se reconectar até que algum tempo tenha passado. Os parâmetros padrão são tais que o atraso inicial é de um segundo e, se após esse atraso a conexão ainda não puder ser estabelecida, o manipulador dobrará o atraso a cada tentativa, até um máximo de 30 segundos.

Este comportamento é controlado pelo seguintes atributos do manipulador:

- `retryStart` (atraso inicial, padrão de 1,0 segundo).
- `retryFactor` (multiplicador, com padrão de 2,0).
- `retryMax` (atraso máximo, padrão de 30,0 segundos).

Isso significa que, se o ouvinte remoto iniciar após o manipulador já ter sido utilizado, mensagens podem ser perdidas (pois o manipulador nem sequer tentará estabelecer uma conexão até que o atraso tenha decorrido, apenas descartando silenciosamente as mensagens durante esse período de atraso).

DatagramHandler

The **`DatagramHandler`** class, located in the **`logging.handlers`** module, inherits from **`SocketHandler`** to support sending logging messages over UDP sockets.

class logging.handlers.DatagramHandler(*host*, *port*)

Retorna uma nova instância da classe **`DatagramHandler`**, destinada a se comunicar com uma máquina remota cujo endereço é fornecido por *host* e *port*.

Nota

Como o UDP não é um protocolo de fluxo contínuo, não há uma conexão persistente entre uma instância deste manipulador e host. Por esse motivo, ao usar um soquete de rede, pode ser necessário realizar uma consulta DNS a cada vez que um evento é registrado, o que pode introduzir alguma latência no sistema. Se isso for um problema para

você, pode realizar a consulta manualmente e inicializar este manipulador usando o endereço IP obtido, em vez do nome do host.

Alterado na versão 3.4: Se `port` for especificado como `None`, um soquete de domínio Unix é criado usando o valor de `host`; caso contrário, é criado um soquete UDP.

emit()

Serializa com pickle o dicionário de atributos do registro e o grava no soquete em formato binário. Se houver um erro com o soquete, o pacote é descartado silenciosamente. Para desserializar com pickle o registro no lado receptor em um **`LogRecord`**, use a função **`makeLogRecord()`**.

makeSocket()

A método de fábrica de **`SocketHandler`** aqui é substituído para criar um soquete UDP (**`socket.SOCK_DGRAM`**).

send(*s*)

Envia uma sequência de bytes serializada com pickle para um soquete. O formato da sequência de bytes enviada é conforme descrito na documentação de **`SocketHandler.makePickle()`**.

SysLogHandler

The **`SysLogHandler`** class, located in the **`logging.handlers`** module, supports sending logging messages to a remote or local Unix syslog.

class logging.handlers.SysLogHandler(*address=('localhost', SYSLOG_UDP_PORT), facility=LOG_USER, socktype=socket.SOCK_DGRAM, timeout=None*)

Returns a new instance of the **`SysLogHandler`** class intended to communicate with a remote Unix machine whose address is given by *address* in the form of a `(host, port)` tuple. If *address* is not specified, `('localhost', 514)` is used. The address is used to open a socket. An alternative to providing a `(host, port)` tuple is providing an address as a

string, for example ‘/dev/log’. In this case, a Unix domain socket is used to send the message to the syslog. If *facility* is not specified, **LOG_USER** is used. The type of socket opened depends on the *socktype* argument, which defaults to **socket.SOCK_DGRAM** and thus opens a UDP socket. To open a TCP socket (for use with the newer syslog daemons such as rsyslog), specify a value of **socket.SOCK_STREAM**. If *timeout* is specified, it sets a timeout (in seconds) for the socket operations. This can help prevent the program from hanging indefinitely if the syslog server is unreachable. By default, *timeout* is `None`, meaning no timeout is applied.

Observe que, se o seu servidor não estiver escutando na porta UDP 514, o **SysLogHandler** pode parecer não funcionar. Nesse caso, verifique qual endereço deve ser usado para um soquete de domínio — isso depende do sistema. Por exemplo, no Linux geralmente é ‘/dev/log’, enquanto no OS/X é ‘/var/run/syslog’. Será necessário verificar a sua plataforma e usar o endereço apropriado (talvez seja preciso fazer essa verificação em tempo de execução se a aplicação precisar rodar em várias plataformas). No Windows, praticamente é necessário usar a opção UDP.

Nota

No macOS 12.x (Monterey), a Apple alterou o comportamento do daemon syslog — ele não escuta mais em um soquete de domínio. Portanto, não é possível esperar que **SysLogHandler** funcione nesse sistema.

Consulte gh-91070 [https://github.com/python/cpython/issues/91070] para obter mais informações.

Alterado na versão 3.2: socktype foi adicionado.

Alterado na versão 3.14: timeout foi adicionado.

close()

Fecha o soquete para o host remoto.

createSocket()

Tenta criar um soquete e, se ele não for um soquete de datagrama, conectá-lo à outra extremidade. Este método é chamado durante a inicialização do manipulador, mas não é considerado um erro se a outra extremidade não estiver escutando nesse momento — o método será chamado novamente ao emitir um evento, caso não haja um soquete nesse ponto.

Adicionado na versão 3.11.

emit(*record*)

O registro é formatado e, em seguida, enviado ao servidor syslog. Se houver informações de exceção, elas *não* são enviadas ao servidor.

Alterado na versão 3.2.1: (Veja: bpo-12168 [https://bugs.python.org/issue?@action=redirect&bpo=12168].) Em versões anteriores, a mensagem enviada aos daemons syslog era sempre terminada com um byte NUL, porque versões antigas desses daemons esperavam uma mensagem terminada em NUL — embora isso não conste na especificação relevante (**RFC 5424** [https://datatracker.ietf.org/doc/html/rfc5424.html]). Versões mais recentes desses daemons não esperam o byte NUL, mas o removem caso esteja presente, e daemons ainda mais recentes (que aderem mais de perto à RFC 5424) repassam o byte NUL como parte da mensagem.

Para facilitar o tratamento de mensagens do syslog diante de todos esses comportamentos distintos dos daemons, a adição do byte NUL tornou-se configurável por meio de um atributo em nível de classe, `append_nul` O valor padrão é `True` (preservando o comportamento existente), mas pode ser definido como `False` em uma instância de `SysLogHandler` para que essa instância não acrescente o terminador NUL.

Alterado na versão 3.3: (Consulte: bpo-12419 [https://bugs.python.org/issue?@action=redirect&bpo=12419]) Em versões anteriores, não havia uma instalação para um prefixo “ident” ou “tag” para identificar a origem da mensagem. Isso agora pode ser especificado usando um atributo de nível de classe, que por padrão é `""` para preservar o comportamento existente, mas pode ser substituído em uma instância de `SysLogHandler` para que essa instância adicione o ident a cada

mensagem manipulada. Observe que o ident fornecido deve ser texto, não bytes, e é adicionado à mensagem exatamente como está.

encodePriority(*facility*, *priority*)

Codifica a instalação e a prioridade em um inteiro. Você pode passar strings ou inteiros — se strings forem passadas, dicionários de mapeamento internos são usados para convertê-las em inteiros.

Os valores simbólicos `LOG_` são definidos em **`SysLogHandler`** e espelham os valores definidos no arquivo de cabeçalho `sys/syslog.h`.

Prioridades

Nome (string)	Valor simbólico
`alert`	LOG_ALERT
`crit` ou `critical`	LOG_CRIT
`debug`	LOG_DEBUG
`emerg` or `panic`	LOG_EMERG
`err` ou `error`	LOG_ERR
`info`	LOG_INFO
`notice`	LOG_NOTICE
`warn` or `warning`	LOG_WARNING

Facilities

Nome (string)	Valor simbólico
`auth`	LOG_AUTH

Nome (string)	Valor simbólico
`authpriv`	LOG_AUTHPRIV
`cron`	LOG_CRON
`daemon`	LOG_DAEMON
`ftp`	LOG_FTP
`kern`	LOG_KERN
`lpr`	LOG_LPR
`mail`	LOG_MAIL
`news`	LOG_NEWS
`syslog`	LOG_SYSLOG
`user`	LOG_USER
`uucp`	LOG_UUCP
`local0`	LOG_LOCAL0
`local1`	LOG_LOCAL1
`local2`	LOG_LOCAL2
`local3`	LOG_LOCAL3
`local4`	LOG_LOCAL4
`local5`	LOG_LOCAL5
`local6`	LOG_LOCAL6

Nome (string)	Valor simbólico
`local7`	LOG_LOCAL7

mapPriority(*levelname*)

Maps a logging level name to a syslog priority name. You may need to override this if you are using custom levels, or if the default algorithm is not suitable for your needs. The default algorithm maps `DEBUG`, `INFO`, `WARNING`, `ERROR` and `CRITICAL` to the equivalent syslog names, and all other level names to 'warning'.

NTEventLogHandler

The **`NTEventLogHandler`** class, located in the **`logging.handlers`** module, supports sending logging messages to a local Windows NT, Windows 2000 or Windows XP event log. Before you can use it, you need Mark Hammond's Win32 extensions for Python installed.

class logging.handlers.NTEventLogHandler(*appname*, *dllname=None*, *logtype='Application'*)

Returns a new instance of the **`NTEventLogHandler`** class. The *appname* is used to define the application name as it appears in the event log. An appropriate registry entry is created using this name. The *dllname* should give the fully qualified pathname of a .dll or .exe which contains message definitions to hold in the log (if not specified, `'win32service.pyd'` is used - this is installed with the Win32 extensions and contains some basic placeholder message definitions. Note that use of these placeholders will make your event logs big, as the entire message source is held in the log. If you want slimmer logs, you have to pass in the name of your own .dll or .exe which contains the message definitions you want to use in the event log). The *logtype* is one of `'Application'`, `'System'` or `'Security'`, and defaults to `'Application'`.

close()

At this point, you can remove the application name from the registry as a source of event log entries. However, if you do this, you will not

be able to see the events as you intended in the Event Log Viewer - it needs to be able to access the registry to get the .dll name. The current version does not do this.

emit(*record*)

Determines the message ID, event category and event type, and then logs the message in the NT event log.

getEventCategory(*record*)

Returns the event category for the record. Override this if you want to specify your own categories. This version returns 0.

getEventType(*record*)

Returns the event type for the record. Override this if you want to specify your own types. This version does a mapping using the handler's typemap attribute, which is set up in **`__init__()`** to a dictionary which contains mappings for **`DEBUG`**, **`INFO`**, **`WARNING`**, **`ERROR`** and **`CRITICAL`**. If you are using your own levels, you will either need to override this method or place a suitable dictionary in the handler's *typemap* attribute.

getMessageID(*record*)

Returns the message ID for the record. If you are using your own messages, you could do this by having the *msg* passed to the logger being an ID rather than a format string. Then, in here, you could use a dictionary lookup to get the message ID. This version returns 1, which is the base message ID in `win32service.pyd`.

SMTPHandler

The **`SMTPHandler`** class, located in the **`logging.handlers`** module, supports sending logging messages to an email address via SMTP.

class logging.handlers.SMTPHandler(*mailhost*, *fromaddr*, *toaddrs*, *subject*, *credentials=None*, *secure=None*, *timeout=1.0*)

Returns a new instance of the **`SMTPHandler`** class. The instance is initialized with the from and to addresses and subject line of the email. The *toaddrs* should be a list of strings. To specify a non-standard SMTP port, use the (host, port) tuple format for the *mailhost* argument. If you use a string, the standard SMTP port is used. If your SMTP server requires authentication, you can specify a (username, password) tuple for the *credentials* argument.

To specify the use of a secure protocol (TLS), pass in a tuple to the *secure* argument. This will only be used when authentication credentials are supplied. The tuple should be either an empty tuple, or a single-value tuple with the name of a keyfile, or a 2-value tuple with the names of the keyfile and certificate file. (This tuple is passed to the **`smtplib.SMTP.starttls()`** method.)

A timeout can be specified for communication with the SMTP server using the *timeout* argument.

Alterado na versão 3.3: Added the *timeout* parameter.

emit(*record*)

Formats the record and sends it to the specified addressees.

getSubject(*record*)

If you want to specify a subject line which is record-dependent, override this method.

MemoryHandler

The **`MemoryHandler`** class, located in the **`logging.handlers`** module, supports buffering of logging records in memory, periodically flushing them to a *target* handler. Flushing occurs whenever the buffer is full, or when an event of a certain severity or greater is seen.

`MemoryHandler` is a subclass of the more general **`BufferingHandler`**, which is an abstract class. This buffers logging records in memory. Whenever each record is added to the buffer, a check is made by calling **`shouldFlush()`** to

see if the buffer should be flushed. If it should, then **`flush()`** is expected to do the flushing.

class logging.handlers.BufferingHandler(*capacity*)

Initializes the handler with a buffer of the specified capacity. Here, *capacity* means the number of logging records buffered.

emit(*record*)

Append the record to the buffer. If **`shouldFlush()`** returns true, call **`flush()`** to process the buffer.

flush()

For a **`BufferingHandler`** instance, flushing means that it sets the buffer to an empty list. This method can be overwritten to implement more useful flushing behavior.

shouldFlush(*record*)

Return `True` if the buffer is up to capacity. This method can be overridden to implement custom flushing strategies.

class logging.handlers.MemoryHandler(*capacity*, *flushLevel=ERROR*, *target=None*, *flushOnClose=True*)

Returns a new instance of the **`MemoryHandler`** class. The instance is initialized with a buffer size of *capacity* (number of records buffered). If *flushLevel* is not specified, **`ERROR`** is used. If no *target* is specified, the target will need to be set using **`setTarget()`** before this handler does anything useful. If *flushOnClose* is specified as `False`, then the buffer is *not* flushed when the handler is closed. If not specified or specified as `True`, the previous behaviour of flushing the buffer will occur when the handler is closed.

Alterado na versão 3.6: The *flushOnClose* parameter was added.

close()

Calls **`flush()`**, sets the target to `None` and clears the buffer.

flush()

For a **MemoryHandler** instance, flushing means just sending the buffered records to the target, if there is one. The buffer is also cleared when buffered records are sent to the target. Override if you want different behavior.

setTarget(*target*)

Sets the target handler for this handler.

shouldFlush(*record*)

Checks for buffer full or a record at the *flushLevel* or higher.

HTTPHandler

The **HTTPHandler** class, located in the **logging.handlers** module, supports sending logging messages to a web server, using either `GET` or `POST` semantics.

class logging.handlers.HTTPHandler(*host*, *url*, *method='GET'*, *secure=False*, *credentials=None*, *context=None*)

Returns a new instance of the **HTTPHandler** class. The *host* can be of the form `host:port`, should you need to use a specific port number. If no *method* is specified, `GET` is used. If *secure* is true, a HTTPS connection will be used. The *context* parameter may be set to a **ssl.SSLContext** instance to configure the SSL settings used for the HTTPS connection. If *credentials* is specified, it should be a 2-tuple consisting of userid and password, which will be placed in a HTTP 'Authorization' header using Basic authentication. If you specify credentials, you should also specify secure=True so that your userid and password are not passed in cleartext across the wire.

Alterado na versão 3.5: The *context* parameter was added.

mapLogRecord(*record*)

Provides a dictionary, based on `record`, which is to be URL-encoded and sent to the web server. The default implementation just returns `record.__dict__`. This method can be overridden if e.g. only a subset

of `LogRecord` is to be sent to the web server, or if more specific customization of what's sent to the server is required.

emit(*record*)

Sends the record to the web server as a URL-encoded dictionary. The `mapLogRecord()` method is used to convert the record to the dictionary to be sent.

Nota

Since preparing a record for sending it to a web server is not the same as a generic formatting operation, using `setFormatter()` to specify a `Formatter` for a `HTTPHandler` has no effect. Instead of calling `format()`, this handler calls `mapLogRecord()` and then `urllib.parse.urlencode()` to encode the dictionary in a form suitable for sending to a web server.

QueueHandler

Adicionado na versão 3.2.

The `QueueHandler` class, located in the `logging.handlers` module, supports sending logging messages to a queue, such as those implemented in the `queue` or `multiprocessing` modules.

Along with the `QueueListener` class, `QueueHandler` can be used to let handlers do their work on a separate thread from the one which does the logging. This is important in web applications and also other service applications where threads servicing clients need to respond as quickly as possible, while any potentially slow operations (such as sending an email via `SMTPHandler`) are done on a separate thread.

class logging.handlers.QueueHandler(*queue*)

Returns a new instance of the `QueueHandler` class. The instance is initialized with the queue to send messages to. The *queue* can be any queue-like object; it's used as-is by the `enqueue()` method, which needs to know how to send messages to it. The queue is not *required* to have the

task tracking API, which means that you can use `SimpleQueue` instances for *queue*.

Nota

If you are using `multiprocessing`, you should avoid using `SimpleQueue` and instead use `multiprocessing.Queue`.

Aviso

The `multiprocessing` module uses an internal logger created and accessed via `get_logger()`. `multiprocessing.Queue` will log `DEBUG` level messages upon items being queued. If those log messages are processed by a **`QueueHandler`** using the same **`multiprocessing.Queue`** instance, it will cause a deadlock or infinite recursion.

emit(*record*)

Enqueues the result of preparing the LogRecord. Should an exception occur (e.g. because a bounded queue has filled up), the `handleError()` method is called to handle the error. This can result in the record silently being dropped (if `logging.raiseExceptions` is `False`) or a message printed to `sys.stderr` (if **`logging.raiseExceptions`** is `True`).

prepare(*record*)

Prepares a record for queuing. The object returned by this method is enqueued.

The base implementation formats the record to merge the message, arguments, exception and stack information, if present. It also removes unpickleable items from the record in-place. Specifically, it overwrites the record's **`msg`** and **`message`** attributes with the merged message (obtained by calling the handler's `format()` method), and sets the **`args`**, **`exc_info`** and **`exc_text`** attributes to `None`.

You might want to override this method if you want to convert the record to a dict or JSON string, or send a modified copy of the record while leaving the original intact.

Nota

The base implementation formats the message with arguments, sets the `message` and `msg` attributes to the formatted message and sets the `args` and `exc_text` attributes to `None` to allow pickling and to prevent further attempts at formatting. This means that a handler on the **`QueueListener`** side won't have the information to do custom formatting, e.g. of exceptions. You may wish to subclass `QueueHandler` and override this method to e.g. avoid setting `exc_text` to `None`. Note that the `message` / `msg` / `args` changes are related to ensuring the record is pickleable, and you might or might not be able to avoid doing that depending on whether your `args` are pickleable. (Note that you may have to consider not only your own code but also code in any libraries that you use.)

enqueue(*record*)

Enqueues the record on the queue using `put_nowait()`; you may want to override this if you want to use blocking behaviour, or a timeout, or a customized queue implementation.

listener

When created via configuration using **`dictConfig()`**, this attribute will contain a **`QueueListener`** instance for use with this handler. Otherwise, it will be `None`.

Adicionado na versão 3.12.

QueueListener

Adicionado na versão 3.2.

The **`QueueListener`** class, located in the **`logging.handlers`** module, supports receiving logging messages from a queue, such as those implemented in the **`queue`** or **`multiprocessing`** modules. The messages are received from a queue in an internal thread and passed, on the same thread, to one or more handlers for processing. While **`QueueListener`** is not itself a

handler, it is documented here because it works hand-in-hand with **QueueHandler**.

Along with the **QueueHandler** class, **QueueListener** can be used to let handlers do their work on a separate thread from the one which does the logging. This is important in web applications and also other service applications where threads servicing clients need to respond as quickly as possible, while any potentially slow operations (such as sending an email via **SMTPHandler**) are done on a separate thread.

class logging.handlers.QueueListener(*queue*, **handlers*, *respect_handler_level=False*)

Returns a new instance of the **QueueListener** class. The instance is initialized with the queue to send messages to and a list of handlers which will handle entries placed on the queue. The queue can be any queue-like object; it's passed as-is to the **dequeue()** method, which needs to know how to get messages from it. The queue is not *required* to have the task tracking API (though it's used if available), which means that you can use **SimpleQueue** instances for *queue*.

Nota

If you are using **multiprocessing**, you should avoid using **SimpleQueue** and instead use **multiprocessing.Queue**.

If `respect_handler_level` is `True`, a handler's level is respected (compared with the level for the message) when deciding whether to pass messages to that handler; otherwise, the behaviour is as in previous Python versions - to always pass each message to each handler.

Alterado na versão 3.5: The `respect_handler_level` argument was added.

Alterado na versão 3.14: **QueueListener** can now be used as a context manager via **with**. When entering the context, the listener is started. When exiting the context, the listener is stopped. **__enter__()** returns the **QueueListener** object.

dequeue(*block*)

Dequeues a record and return it, optionally blocking.

The base implementation uses `get()`. You may want to override this method if you want to use timeouts or work with custom queue implementations.

prepare(*record*)

Prepare a record for handling.

This implementation just returns the passed-in record. You may want to override this method if you need to do any custom marshalling or manipulation of the record before passing it to the handlers.

handle(*record*)

Handle a record.

This just loops through the handlers offering them the record to handle. The actual object passed to the handlers is that which is returned from **`prepare()`**.

start()

Starts the listener.

This starts up a background thread to monitor the queue for LogRecords to process.

Alterado na versão 3.14: Raises **`RuntimeError`** if called and the listener is already running.

stop()

Stops the listener.

This asks the thread to terminate, and then waits for it to do so. Note that if you don’t call this before your application exits, there may be some records still left on the queue, which won’t be processed.

enqueue_sentinel()

Writes a sentinel to the queue to tell the listener to quit. This implementation uses `put_nowait()`. You may want to override this method if you want to use timeouts or work with custom queue implementations.

Adicionado na versão 3.3.

Ver também

Módulo `logging`
Referência da API para o módulo de logging.

Módulo `logging.config`
API de configuração para o módulo logging.

`platform` — Acesso aos dados de identificação da plataforma subjacente

Código-fonte: Lib/platform.py
[https://github.com/python/cpython/tree/3.15/Lib/platform.py]

Nota

Plataformas específicas listadas em ordem alfabética, com Linux incluído na seção Unix.

Multiplataforma

platform.architecture(*executable=sys.executable*, *bits=''*, *linkage=''*)

Consulta o executável fornecido (o padrão é o binário do interpretador Python) para obter várias informações de arquitetura.

Retorna uma tupla `(bits, linkage)` que contém informações sobre a arquitetura de bits e o formato de linkage usado para o executável. Ambos os valores são retornados como strings.

Valores que não podem ser determinados são retornados como fornecidos pelas predefinições de parâmetros. Se bits for fornecido como `''`, o `sizeof(pointer)` (ou `sizeof(long)` na versão Python < 1.5.2) é usado como indicador para o tamanho de ponteiro suportado.

A função depende do comando `file` do sistema para fazer o trabalho real. Isso está disponível na maioria, se não em todas as plataformas Unix e algumas plataformas não Unix e somente se o executável

apontar para o interpretador Python. Padrões razoáveis são usados quando as necessidades acima não são atendidas.

Nota

No macOS (e talvez em outras plataformas), arquivos executáveis podem ser arquivos universais contendo diversas arquiteturas.

Para obter a “característica de 64 bits” do interpretador atual, é mais confiável consultar o atributo `sys.maxsize`:

```
is_64bits = sys.maxsize > 2**32
```

platform.machine()

Retorna o tipo de máquina, por exemplo, `'AMD64'`. Uma string vazia é retornada se o valor não puder ser determinado.

A saída depende da plataforma e pode diferir nas convenções de nomenclatura e uso de maiúsculas e minúsculas.

platform.node()

Retorna o nome de rede do computador (pode não ser totalmente qualificado!). Uma string vazia é retornada se o valor não puder ser determinado.

platform.platform(*aliased=False*, *terse=False*)

Retorna uma única string identificando a plataforma subjacente com o máximo de informações úteis possível.

A saída é pretendida para ser *legível por humanos* em vez de analisável por máquinas. Pode parecer diferente em diferentes plataformas e isso é intencional.

Se *aliased* for true, a função usará apelidos para várias plataformas que relatam nomes de sistemas que diferem de seus nomes comuns, por exemplo, SunOS será relatado como Solaris. A função `system_alias()` é usada para implementar isso.

Definir *terse* como true faz com que a função retorne apenas o mínimo absoluto de informações necessárias para identificar a plataforma.

Alterado na versão 3.8: No macOS, a função agora usa `mac_ver()`, se retornar uma string de versão não vazia, para obter a versão do macOS em vez da versão do darwin.

platform.processor()

Retorna o nome (real) do processador, por exemplo `'amdk6'`.

Uma string vazia é retornada se o valor não puder ser determinado. Note que muitas plataformas não fornecem essas informações ou simplesmente retornam o mesmo valor que para `machine()`. O NetBSD faz isso.

platform.python_build()

Retorna uma tupla `(buildno, builddate)` informando o número e a data da construção do Python como strings.

platform.python_compiler()

Retorna uma string identificando o compilador usado para compilar Python.

platform.python_branch()

Retorna uma string identificando a branch SCM da implementação do Python.

platform.python_implementation()

Retorna uma string identificando a implementação do Python. Os valores de retorno possíveis são: ‘CPython’, ‘IronPython’, ‘Jython’, ‘PyPy’.

platform.python_revision()

Retorna uma string que identifica a revisão do SCM da implementação Python.

platform.python_version()

Retorna a versão do Python como string `'major.minor.patchlevel'`.

Observe que, diferentemente do `sys.version` do Python, o valor retornado sempre incluirá o patchlevel (o padrão é 0).

platform.python_version_tuple()

Retorna a versão do Python como tupla `(major, minor, patchlevel)` de strings.

Observe que, diferentemente do `sys.version` do Python, o valor retornado sempre incluirá o patchlevel (o padrão é `'0'`).

platform.release()

Retorna a versão do sistema, por exemplo, `'2.2.0'` ou `'NT'`. Uma string vazia é retornada se o valor não puder ser determinado.

No iOS e Android, esta é a versão do SO voltada para o usuário. Para obter a versão do kernel Darwin ou Linux, use **`os.uname()`**.

platform.system()

Retorna o nome do sistema/SO, como `'Linux'`, `'Darwin'`, `'Java'`, `'Windows'`. Uma string vazia é retornada se o valor não puder ser determinado.

No iOS e Android, isso retorna o nome do SO voltado para o usuário (por exemplo, `'iOS'`, `'iPadOS'` ou `'Android'`). Para obter o nome do kernel (`'Darwin'` ou `'Linux'`), use **`os.uname()`**.

platform.system_alias(*system*, *release*, *version*)

Retorna `(sistema, release, versão)` com apelido para nomes comuns de marketing usados para alguns sistemas. Ele também faz alguma reordenação das informações em alguns casos em que, de outra forma, causaria confusão.

platform.version()

Retorna a versão de lançamento do sistema, por exemplo, `'#3 on degas'`. Uma string vazia é retornada se o valor não puder ser determinado.

platform.uname()

Interface para uname razoavelmente portátil. Retorna um **`namedtuple()`** contendo seis atributos: **`system`**, **`node`**, **`release`**, **`version`**, **`machine`** e **`processor`**.

`processor` é resolvido tardiamente, sob demanda.

Nota: os dois primeiros nomes de atributos diferem dos nomes apresentados por **`os.uname()`**, onde são chamados **`sysname`** e **`nodename`**.

As entradas que não podem ser determinadas são definidas como `''`.

Alterado na versão 3.3: O resultado mudou de uma tupla para **`namedtuple()`**.

Alterado na versão 3.9: **`processor`** é resolvido tarde em vez de imediatamente.

platform.invalidate_caches()

Limpa o cache interno de informações, como **`uname()`**. Isso normalmente é útil quando o **`node()`** da plataforma é alterado por um processo externo e é necessário recuperar o valor atualizado.

Adicionado na versão 3.14.

Plataforma Windows

platform.win32_ver(*release=''*, *version=''*, *csd=''*, *ptype=''*)

Obtém informações adicionais sobre a versão do Registro do Windows e retorna uma tupla `(release, version, csd, ptype)` referente à versão do SO, número da versão, nível do CSD (service pack) e tipo de sistema operacional (processador múltiplo/único). Os valores que não

podem ser determinados são definidos para os padrões fornecidos como parâmetros (que são todos padrão para uma string vazia).

Como dica: *ptype* é `'Uniprocessor Free'` em máquinas NT de processador único e `'Multiprocessor Free'` em máquinas de multiprocessador. O `'Free'` se refere à versão do sistema operacional que está livre de código de depuração. Ele também pode declarar `'Checked'`, o que significa que a versão do sistema operacional usa código de depuração, ou seja, código que verifica argumentos, intervalos, etc.

platform.win32_edition()

Retorna uma string representando a edição atual do Windows, ou `None` se o valor não puder ser determinado. Os valores possíveis incluem, mas não estão limitados a `'Enterprise'`, `'IoTUAP'`, `'ServerStandard'` e `'nanoserver'`.

Adicionado na versão 3.8.

platform.win32_is_iot()

Retorna `True` se a edição do Windows retornada por **`win32_edition()`** for reconhecida como uma edição IoT.

Adicionado na versão 3.8.

Plataforma macOS

platform.mac_ver(*release='', versioninfo=('', '', ''), machine=''*)

Obtém informações sobre a versão do macOS e retorna-as como tupla `(release, versioninfo, machine)`, sendo *versioninfo* uma tupla `(version, dev_stage, non_release_version)`.

Entradas que não podem ser determinadas são definidas como `''`. Todas as entradas de tupla são strings.

Plataforma iOS

platform.ios_ver(*system=''*, *release=''*, *model=''*, *is_simulator=False*)

Obtém informações sobre a versão do iOS e retorna-as como `namedtuple()` com os seguintes atributos:

- `system` é o nome do sistema operacional; `'iOS'` ou `'iPadOS'`.
- `release` é o número da versão do iOS como uma string (por exemplo, `'17.2'`).
- `model` é o identificador do modelo do dispositivo; será uma string como `'iPhone13,2'` para um dispositivo físico ou `'iPhone'` em um simulador.
- `is_simulator` é um booleano que descreve se o aplicativo está sendo executado em um simulador ou em um dispositivo físico.

As entradas que não podem ser determinadas são definidas para os padrões fornecidos como parâmetros.

Plataformas Unix

platform.libc_ver(*executable=sys.executable*, *lib=''*, *version=''*, *chunksize=16384*)

Tenta determinar a versão libc contra a qual o executável do arquivo (padrão para o interpretador Python) está vinculado. Retorna uma tupla de strings `(lib, version)` que tem como padrão os parâmetros fornecidos caso a pesquisa falhe.

Observe que esta função tem conhecimento profundo de como diferentes versões da libc adicionam símbolos ao executável e provavelmente só pode ser usada para executáveis compilados usando **gcc**.

O arquivo é lido e escaneado em blocos de *chunksize* bytes.

Plataformas Linux

platform.freedesktop_os_release()

Obtém a identificação do sistema operacional do arquivo `os-release` e retorna-a como um dict. O arquivo `os-release` é um padrão freedesktop.org [https://www.freedesktop.org/software/systemd/man/os-release.html] e está disponível na maioria das distribuições Linux. Uma exceção notável é o Android e as distribuições baseadas em Android.

Levanta **`OSError`** ou subclasse quando nem `/etc/os-release` nem `/usr/lib/os-release` podem ser lidos.

Em caso de sucesso, a função retorna um dicionário onde chaves e valores são strings. Valores têm seus caracteres especiais como `"` e `$` sem aspas. Os campos `NAME`, `ID` e `PRETTY_NAME` são sempre definidos de acordo com o padrão. Todos os outros campos são opcionais. Os fornecedores podem incluir campos adicionais.

Note que campos como `NAME`, `VERSION` e `VARIANT` são strings adequadas para apresentação aos usuários. Os programas devem usar campos como `ID`, `ID_LIKE`, `VERSION_ID` ou `VARIANT_ID` para identificar distribuições Linux.

Exemplo:

```
def get_like_distro():
    info = platform.freedesktop_os_release()
    ids = [info["ID"]]
    if "ID_LIKE" in info:
        # ids são separados por espaço e ordenado por precedé
        ids.extend(info["ID_LIKE"].split())
    return ids
```

Adicionado na versão 3.10.

Plataforma Android

platform.android_ver(*release='', api_level=0, manufacturer='', model='', device='', is_emulator=False*)

Obtém informações do dispositivo Android. Retorna um `namedtuple()` com os seguintes atributos. Valores que não podem ser determinados são definidos para os padrões fornecidos como parâmetros.

- `release` - Versão do Android, como uma string (por exemplo, `"14"`).
- `api_level` - Nível de API do dispositivo em execução, como um inteiro (por exemplo, `34` para Android 14). Para obter o nível de API contra o qual o Python foi criado, consulte `sys.getandroidapilevel()`.
- `manufacturer` - Nome do fabricante [https://developer.android.com/reference/android/os/Build#MANUFACTURER].
- `model` - Nome do modelo [https://developer.android.com/reference/android/os/Build#MODEL] – normalmente o nome de marketing ou o número do modelo.
- `device` - Nome do dispositivo [https://developer.android.com/reference/android/os/Build#DEVICE] – normalmente o número do modelo ou um codinome.
- `is_emulator` - `True` se o dispositivo for um emulador; `False` se for um dispositivo físico.

O Google mantém uma lista de nomes de modelos e dispositivos conhecidos [https://storage.googleapis.com/play_public/supported_devices.html].

Adicionado na versão 3.13.

Uso na linha de comando

`platform` também pode ser invocado diretamente usando a opção `-m` do interpretador:

```
python -m platform [--terse] [--nonaliased] [{nonaliased,terse}
```

As seguintes opções são aceitas:

--terse

Exibe informações concisas sobre a plataforma. Isso equivale a chamar `platform.platform()` com o argumento *terse* definido como `True`.

--nonaliased

Exibe informações da plataforma sem alias de nome do sistema/SO. Isso equivale a chamar `platform.platform()` com o argumento *aliased* definido como `True`.

Você também pode passar um ou mais argumentos posicionais (`terse`, `nonaliased`) para controlar explicitamente o formato de saída. Eles se comportam de forma semelhante às suas opções correspondentes.

`errno` — Símbolos padrão do sistema errno

Este módulo disponibiliza símbolos de sistema padrão `errno`. O valor de cada símbolo é o valor inteiro correspondente. Os nomes e descrições são emprestados de `linux/include/errno.h`, que deve ser tudo inclusivo.

errno.errorcode

Dicionário que fornece um mapeamento do valor errno para o nome da string no sistema subjacente. Por exemplo, `errno.errorcode[errno.EPERM]` mapeia para `'EPERM'`.

Para traduzir um código de erro numérico em uma mensagem de erro, use **`os.strerror()`**.

Da lista a seguir, símbolos que não são usados na plataforma atual não são definidos pelo módulo. A lista específica de símbolos definidos está disponível como `errno.errorcode.keys()`. Os símbolos disponíveis podem incluir:

errno.EPERM

Operação não permitida. Este erro é mapeado para a exceção **`PermissionError`**.

errno.ENOENT

Arquivo ou diretório inexistente. Este erro é mapeado para a exceção **`FileNotFoundError`**.

errno.ESRCH

Processo inexistente. Este erro é mapeado para a exceção **`ProcessLookupError`**.

errno.EINTR

Chamada de sistema interrompida. Este erro é mapeado para a exceção `InterruptedError`.

errno.EIO
Erro de E/S

errno.ENXIO
Endereço ou dispositivo inexistente

errno.E2BIG
Lista de argumentos muito longa

errno.ENOEXEC
Erro no formato exec

errno.EBADF
Descritor de arquivo inválido

errno.ECHILD
Não há processos filhos. Este erro é mapeado para a exceção `ChildProcessError`.

errno.EAGAIN
Tente novamente. Este erro é mapeado para a exceção `BlockingIOError`.

errno.ENOMEM
Memória insuficiente

errno.EACCES
Permissão negada. Este erro é mapeado para a exceção `PermissionError`.

errno.EFAULT
Endereço inválido

errno.ENOTBLK

Dispositivo de bloco requerido

errno.EBUSY
Dispositivo ou recurso ocupado

errno.EEXIST
Arquivo existe. Este erro é mapeado para a exceção `FileExistsError`.

errno.EXDEV
Link entre dispositivos inválido

errno.ENODEV
Dispositivo inexistente

errno.ENOTDIR
Não é um diretório. Este erro é mapeado para a exceção `NotADirectoryError`.

errno.EISDIR
É um diretório. Este erro é mapeado para a exceção `IsADirectoryError`.

errno.EINVAL
Argumento inválido

errno.ENFILE
Estouro de tabela de arquivos

errno.EMFILE
Muitos arquivos abertos

errno.ENOTTY
Not a typewriter

errno.ETXTBSY
Arquivo texto ocupado

errno.EFBIG
Arquivo muito grande

errno.ENOSPC
Não há espaço disponível no dispositivo

errno.ESPIPE
Procura ilegal

errno.EROFS
Sistema de arquivos de somente leitura

errno.EMLINK
Muitos links

errno.EPIPE
Pipe quebrado. Este erro é mapeado para a exceção `BrokenPipeError`.

errno.EDOM
Argumento matemático fora do domínio da função

errno.ERANGE
Resultado matemático não representável

errno.EDEADLK
Ocorreria um impasse (deadlock) de recursos

errno.ENAMETOOLONG
Nome de arquivo muito longo

errno.ENOLCK
Nenhuma trava de registro disponível

errno.ENOSYS
Função não implementada

errno.ENOTEMPTY

Diretório não vazio

errno.ELOOP
Foram encontrados muitos links simbólicos

errno.EWOULDBLOCK
Operation causaria bloqueio. Este erro é mapeado para a exceção `BlockingIOError`.

errno.ENOMSG
Nenhuma mensagem do tipo desejado

errno.EIDRM
Identificador removido

errno.ECHRNG
Número do canal fora do intervalo

errno.EL2NSYNC
Não sincronizado nível 2

errno.EL3HLT
Parada de sistema nível 3

errno.EL3RST
Reinicialização nível 3

errno.ELNRNG
Número de link fora da faixa

errno.EUNATCH
Driver de protocolo não anexado

errno.ENOCSI
Não há estrutura CSI disponível

errno.EL2HLT

Parada de sistema nível 2

errno.EBADE
Troca inválida

errno.EBADR
Descritor de requisição inválido

errno.EXFULL
Troca completa

errno.ENOANO
Sem anode

errno.EBADRQC
Código de requisição inválido

errno.EBADSLT
Slot inválido

errno.EDEADLOCK
Erro de impasse em travamento de arquivo

errno.EBFONT
Formato inválido do arquivo de fonte

errno.ENOSTR
Dispositivo não é um stream

errno.ENODATA
Não há dados disponíveis

errno.ETIME
Temporizador expirado

errno.ENOSR
Sem recursos de streams

errno.ENONET
A máquina não está na rede

errno.ENOPKG
Pacote não instalado

errno.EREMOTE
O objeto é remoto

errno.ENOLINK
A conexão foi rompida

errno.EADV
Erro de anúncio

errno.ESRMNT
Erro Srmount

errno.ECOMM
Erro de comunicação ao enviar

errno.EPROTO
Erro de Protocolo

errno.EMULTIHOP
Tentativa de hops múltiplos

errno.EDOTDOT
Erro específico de RFS

errno.EBADMSG
Não é uma mensagem de dados

errno.EOVERFLOW
Valor muito grande para o tipo de dados definido

errno.ENOTUNIQ

O nome não é único na rede

errno.EBADFD
Descritor de arquivo em mal estado

errno.EREMCHG
Endereço remoto mudou

errno.ELIBACC
Não é possível acessar uma biblioteca compartilhada necessária

errno.ELIBBAD
Acessando uma biblioteca compartilhado corrompida

errno.ELIBSCN
Seção .lib corrompida em a.out

errno.ELIBMAX
Tentando ligar em muitas bibliotecas compartilhadas

errno.ELIBEXEC
Não é possível executar uma biblioteca compartilhada diretamente

errno.EILSEQ
Sequência de bytes ilegal

errno.ERESTART
Chamada de sistema interrompida deve ser reiniciada

errno.ESTRPIPE
Erro de fluxos de pipe

errno.EUSERS
Muitos usuários

errno.ENOTSOCK
Operação socket em um arquivo não-socket

errno.EDESTADDRREQ
Endereço de destino necessário

errno.EMSGSIZE
Mensagem muito longa

errno.EPROTOTYPE
Tipo errado de protocolo para socket

errno.ENOPROTOOPT
Protocolo não disponível

errno.EPROTONOSUPPORT
Protocolo sem suporte

errno.ESOCKTNOSUPPORT
Tipo socket sem suporte

errno.EOPNOTSUPP
Operação sem suporte na extremidade do transporte

errno.ENOTSUP
Operação sem suporte

Adicionado na versão 3.2.

errno.EPFNOSUPPORT
Família de protocolo sem suporte

errno.EAFNOSUPPORT
Família de endereços sem suporte pelo protocolo

errno.EADDRINUSE
Endereço já em uso

errno.EADDRNOTAVAIL
Não é possível acessar o endereço requisitado

errno.ENETDOWN
A rede não responde

errno.ENETUNREACH
A rede está fora de alcance

errno.ENETRESET
A rede desconectou-se ao reiniciar

errno.ECONNABORTED
O software causou a interrupção da conexão. Este erro é mapeado para a exceção `ConnectionAbortedError`.

errno.ECONNRESET
Conexão fechada pela outra ponta. Este erro é mapeado para a exceção `ConnectionResetError`.

errno.ENOBUFS
Não há espaço de buffer disponível

errno.EISCONN
A extremidade do transporte já está conectada

errno.ENOTCONN
A extremidade do transporte não está conectada

errno.ESHUTDOWN
Cannot send after transport endpoint shutdown. This error is mapped to the exception `BrokenPipeError`.

errno.ETOOMANYREFS
Too many references: cannot splice

errno.ETIMEDOUT
Connection timed out. This error is mapped to the exception `TimeoutError`.

errno.ECONNREFUSED

Connection refused. This error is mapped to the exception `ConnectionRefusedError`.

errno.EHOSTDOWN

Host is down

errno.EHOSTUNREACH

No route to host

errno.EHWPOISON

Memory page has hardware error.

Adicionado na versão 3.14.

errno.EALREADY

Operation already in progress. This error is mapped to the exception `BlockingIOError`.

errno.EINPROGRESS

Operation now in progress. This error is mapped to the exception `BlockingIOError`.

errno.ESTALE

Stale NFS file handle

errno.EUCLEAN

Structure needs cleaning

errno.ENOTNAM

Not a XENIX named type file

errno.ENAVAIL

No XENIX semaphores available

errno.EISNAM

É um arquivo de tipo nomeado

errno.EREMOTEIO
Erro de E/S remoto

errno.EDQUOT
Quota exceeded

errno.EQFULL
Interface output queue is full

Adicionado na versão 3.11.

errno.ENOMEDIUM
No medium found

errno.EMEDIUMTYPE
Wrong medium type

errno.ENOKEY
Required key not available

errno.EKEYEXPIRED
Key has expired

errno.EKEYREVOKED
Key has been revoked

errno.EKEYREJECTED
Key was rejected by service

errno.ERFKILL
Operation not possible due to RF-kill

errno.ELOCKUNMAPPED
Locked lock was unmapped

errno.ENOTACTIVE
Facility is not active

errno.EAUTH
Authentication error

Adicionado na versão 3.2.

errno.EBADARCH
Bad CPU type in executable

Adicionado na versão 3.2.

errno.EBADEXEC
Bad executable (or shared library)

Adicionado na versão 3.2.

errno.EBADMACHO
Malformed Mach-o file

Adicionado na versão 3.2.

errno.EDEVERR
Device error

Adicionado na versão 3.2.

errno.EFTYPE
Inappropriate file type or format

Adicionado na versão 3.2.

errno.ENEEDAUTH
Need authenticator

Adicionado na versão 3.2.

errno.ENOATTR
Attribute not found

Adicionado na versão 3.2.

errno.ENOPOLICY
Policy not found

Adicionado na versão 3.2.

errno.EPROCLIM
Too many processes

Adicionado na versão 3.2.

errno.EPROCUNAVAIL
Bad procedure for program

Adicionado na versão 3.2.

errno.EPROGMISMATCH
Program version wrong

Adicionado na versão 3.2.

errno.EPROGUNAVAIL
RPC prog. not avail

Adicionado na versão 3.2.

errno.EPWROFF
Device power is off

Adicionado na versão 3.2.

errno.EBADRPC
RPC struct is bad

Adicionado na versão 3.2.

errno.ERPCMISMATCH

RPC version wrong

Adicionado na versão 3.2.

errno.ESHLIBVERS
Shared library version mismatch

Adicionado na versão 3.2.

errno.ENOTCAPABLE
Capabilities insufficient. This error is mapped to the exception `PermissionError`.

Disponibilidade: WASI, FreeBSD

Adicionado na versão 3.11.1.

errno.ECANCELED
Operation canceled

Adicionado na versão 3.2.

errno.EOWNERDEAD
Owner died

Adicionado na versão 3.2.

errno.ENOTRECOVERABLE
State not recoverable

Adicionado na versão 3.2.

`ctypes` — Uma biblioteca de funções externas para Python

Código-fonte: Lib/ctypes [https://github.com/python/cpython/tree/3.15/Lib/ctypes]

`ctypes` é uma biblioteca de funções externas para Python. Ela fornece tipos de dados compatíveis com C e permite chamar funções em DLLs ou bibliotecas compartilhadas. Ela pode ser usada para agrupar essas bibliotecas em Python puro.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Aviso

`ctypes` provides low-level access to native libraries and the process's memory, bypassing Python's safety mechanisms and allowing execution of arbitrary native code. Incorrect use can corrupt data and objects, reveal sensitive information, cause crashes, or otherwise compromise the running process.

Tutorial do ctypes

Nota: Alguns exemplos de código fazem referência ao tipo ctypes **`c_int`**. Em plataformas em que `sizeof(long) == sizeof(int)` é um apelido para **`c_long`**. Então, você não deve ficar confuso se **`c_long`** for impresso se você esperaria **`c_int`** — eles são, na verdade, o mesmo tipo.

Carregando bibliotecas de ligação dinâmica

ctypes exports the **cdll**, and on Windows **windll** and **oledll** objects, for loading dynamic link libraries.

You load libraries by accessing them as attributes of these objects. **cdll** loads libraries which export functions using the standard `cdecl` calling convention, while **windll** libraries call functions using the `stdcall` calling convention. **oledll** also uses the `stdcall` calling convention, and assumes the functions return a Windows **HRESULT** error code. The error code is used to automatically raise an **OSError** exception when the function call fails.

Alterado na versão 3.3: Erros do Windows costumavam levantar **WindowsError**, que agora é um apelido de **OSError**.

Veja alguns exemplos para Windows. Note que `msvcrt` é a biblioteca C padrão da MS que contém a maioria das funções C padrão e usa a convenção de chamada `cdecl`:

```
>>> from ctypes import *
>>> print(windll.kernel32)
<WinDLL 'kernel32', handle ... at ...>
>>> print(cdll.msvcrt)
<CDLL 'msvcrt', handle ... at ...>
>>> libc = cdll.msvcrt
>>>
```

O Windows acrescenta automaticamente o sufixo de arquivo `.dll` usual.

Nota

Acessar a biblioteca padrão C por meio de `cdll.msvcrt` usará uma versão desatualizada da biblioteca, que pode ser incompatível com a usada pelo Python. Onde possível, use a funcionalidade nativa do Python; caso contrário, importe e use o módulo `msvcrt`.

Other systems require the filename *including* the extension to load a library, so attribute access can not be used to load libraries. Either the **LoadLibrary()** method of the dll loaders should be used, or you should load the library by creating an instance of **CDLL** by calling the constructor.

For example, on Linux:

```
>>> cdll.LoadLibrary("libc.so.6")
<CDLL 'libc.so.6', handle ... at ...>
>>> libc = CDLL("libc.so.6")
>>> libc
<CDLL 'libc.so.6', handle ... at ...>
>>>
```

On macOS:

```
>>> cdll.LoadLibrary("libc.dylib")
<CDLL 'libc.dylib', handle ... at ...>
>>> libc = CDLL("libc.dylib")
>>> libc
<CDLL 'libc.dylib', handle ... at ...>
```

Acessando funções de dlls carregadas

Funções são acessadas como atributos de objetos dll:

```
>>> libc.printf
<_FuncPtr object at 0x...>
>>> print(windll.kernel32.GetModuleHandleA)
<_FuncPtr object at 0x...>
>>> print(windll.kernel32.MyOwnFunction)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "ctypes.py", line 239, in __getattr__
    func = _StdcallFuncPtr(name, self)
AttributeError: function 'MyOwnFunction' not found
>>>
```

Observe que DLLs de sistema do Win32, como `kernel32` e `user32`, frequentemente exportam versões ANSI e UNICODE de uma função. A versão UNICODE é exportada com um `W` anexado ao nome, enquanto a versão ANSI é exportada com um `A` anexado ao nome. A função `GetModuleHandle` do Win32, que retorna um *identificador de módulo* para um determinado nome de módulo, tem o seguinte protótipo em C, e uma

macro é usada para expor um deles como `GetModuleHandle`, dependendo se UNICODE está definido ou não:

```
/* versão ANSI */
HMODULE GetModuleHandleA(LPCSTR lpModuleName);
/* versão UNICODE */
HMODULE GetModuleHandleW(LPCWSTR lpModuleName);
```

windll não tenta selecionar um deles magicamente, você deve acessar a versão necessária especificando `GetModuleHandleA` ou `GetModuleHandleW` explicitamente e então chamá-lo com objetos bytes ou string, respectivamente.

Às vezes, DLLs exportam funções com nomes que não são identificadores Python válidos, como `"??2@YAPAXI@Z"`. Nesse caso, você precisa usar **`getattr()`** para recuperar a função:

```
>>> getattr(cdll.msvcrt, "??2@YAPAXI@Z")
<_FuncPtr object at 0x...>
>>>
```

No Windows, algumas dlls exportam funções não por nome, mas por ordinal. Essas funções podem ser acessadas indexando o objeto dll com o número ordinal:

```
>>> cdll.kernel32[1]
<_FuncPtr object at 0x...>
>>> cdll.kernel32[0]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "ctypes.py", line 310, in __getitem__
    func = _StdcallFuncPtr(name, self)
AttributeError: function ordinal 0 not found
>>>
```

Chamando funções

Você pode chamar essas funções como qualquer outro chamável do Python. Este exemplo usa a função `rand()`, que não aceita argumentos e retorna um inteiro pseudoaleatório:

```
>>> print(libc.rand())
1804289383
```

No Windows, você pode chamar a função `GetModuleHandleA()`, que retorna um identificador de módulo win32 (passando `None` como único argumento para chamá-lo com um ponteiro `NULL`):

```
>>> print(hex(windll.kernel32.GetModuleHandleA(None)))
0x1d000000
>>>
```

`ValueError` é gerado quando você chama uma função `stdcall` com a convenção de chamada `cdecl`, ou vice-versa:

```
>>> cdll.kernel32.GetModuleHandleA(None)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Procedure probably called with not enough arguments
>>>

>>> windll.msvcrt.printf(b"spam")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: Procedure probably called with too many arguments (4
>>>
```

Para descobrir a convenção de chamada correta, você precisa consultar o arquivo de cabeçalho C ou a documentação da função que deseja chamar.

On Windows, **`ctypes`** uses win32 structured exception handling to prevent crashes from general protection faults when functions are called with invalid argument values:

```
>>> windll.kernel32.GetModuleHandleA(32)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
OSError: exception: access violation reading 0x00000020
>>>
```

The `faulthandler` module can help debug crashes, such as segmentation faults produced by erroneous C library calls.

`None`, inteiros, objetos bytes e strings (unicode) são os únicos objetos nativos do Python que podem ser usados diretamente como parâmetros nessas chamadas de função. `None` é passado como um ponteiro `NULL` em C, objetos bytes e strings são passados como ponteiros para o bloco de memória que contém seus dados (char* ou wchar_t*). Inteiros em Python são passados como o tipo padrão da plataforma em C int, e seu valor é mascarado para se encaixar no tipo em C.

Before we move on calling functions with other parameter types, we have to learn more about **`ctypes`** data types.

Tipos de dados fundamentais

`ctypes` define vários tipos de dados primitivos compatíveis com C:

Tipo ctypes	Tipo em C	Tipo em Python	`_type_`
`c_bool`	_Bool	`bool`	`'?'`
`c_char`	char	1-character `bytes`	`'c'`
`c_wchar`	**`wchar_t`**	1-character `str`	`'u'`
`c_byte`	char	`int`	`'b'`
`c_ubyte`	unsigned char	`int`	`'B'`

Tipo ctypes	Tipo em C	Tipo em Python	`_type_`
`c_short`	short	`int`	`'h'`
`c_ushort`	unsigned short	`int`	`'H'`
`c_int`	int	`int`	`'i'` *
`c_int8`	**`int8_t`**	`int`	*
`c_int16`	**`int16_t`**	`int`	*
`c_int32`	**`int32_t`**	`int`	*
`c_int64`	**`int64_t`**	`int`	*
`c_uint`	unsigned int	`int`	`'I'` *
`c_uint8`	**`uint8_t`**	`int`	*
`c_uint16`	**`uint16_t`**	`int`	*
`c_uint32`	**`uint32_t`**	`int`	*

Tipo ctypes	Tipo em C	Tipo em Python	_type_
c_uint64	uint64_t	int	*
c_long	long	int	'l'
c_ulong	unsigned long	int	'L'
c_longlong	long long	int	'q' *
c_ulonglong	unsigned long long	int	'Q' *
c_size_t	size_t	int	*
c_ssize_t	Py_ssize_t	int	*
c_time_t	time_t	int	*
c_float	float	float	'f'
c_double	double	float	'd'
c_longdouble	long double	float	'g' *

Tipo ctypes	Tipo em C	Tipo em Python	_type_
c_char_p	char* (finalizado com NUL)	bytes or None	'z'
c_wchar_p	wchar_t* (finalizado com NUL)	str or None	'Z'
c_void_p	void*	int or None	'P'
py_object	PyObject*	object	'O'
VARIANT_BOOL	short int	bool	'v'

Além disso, se a aritmética complexa compatível com IEC 60559 (Anexo G) for suportada em C e `libffi`, os seguintes tipos complexos estarão disponíveis:

Tipo ctypes	Tipo em C	Tipo em Python	_type_
c_float_complex	float complex	complex	'Zf'
c_double_complex	double complex	complex	'Zd'
c_longdouble_complex	long double complex	complex	'Zg'

Alterado na versão 3.15: The `_type_` types `F`, `D` and `G` have been replaced with `Zf`, `Zd` and `Zg`.

Todos esses tipos podem ser criados chamando-os com um inicializador opcional do tipo e valor corretos:

```
>>> c_int()
c_long(0)
>>> c_wchar_p("Olá, mundo")
c_wchar_p(139878537078816)
>>> c_ushort(-3)
c_ushort(65533)
>>>
```

The constructors for numeric types will convert input using `__bool__()`, `__index__()` (for `int`), `__float__()` or `__complex__()`. This means `c_bool` accepts any object with a truth value:

```
>>> empty_list = []
>>> c_bool(empty_list)
c_bool(False)
```

Como esses tipos são mutáveis, seus valores também podem ser alterados posteriormente:

```
>>> i = c_int(42)
>>> print(i)
c_long(42)
>>> print(i.value)
42
>>> i.value = -99
>>> print(i.value)
-99
>>>
```

Atribuir um novo valor a instâncias dos tipos de ponteiro `c_char_p`, `c_wchar_p` e `c_void_p` altera o *local de memória* para o qual eles apontam, *não o conteúdo* do bloco de memória (claro que não, porque objetos string do Python são imutáveis):

```
>>> s = "Olá, mundo"
>>> c_s = c_wchar_p(s)
>>> print(c_s)
c_wchar_p(139878542513936)
>>> print(c_s.value)
Olá, mundo
>>> c_s.value = "Opa, beleza?"
>>> print(c_s)              # o local da memória foi alterado
c_wchar_p(139878536944240)
>>> print(c_s.value)
Opa, beleza?
>>> print(s)                # primeiro objeto está inalterado
Olá, mundo
>>>
```

No entanto, tome cuidado para não passá-los para funções que esperam ponteiros para memória mutável. Se precisar de blocos de memória mutáveis, o ctypes possui uma função **`create_string_buffer()`** que os cria de várias maneiras. O conteúdo do bloco de memória atual pode ser acessado (ou alterado) com a propriedade `raw`; se quiser acessá-lo como uma string terminada em NUL, use a propriedade `value`:

```
>>> from ctypes import *
>>> p = create_string_buffer(3)            # cria um buffer de 3
>>> print(sizeof(p), repr(p.raw))
3 b'\x00\x00\x00'
>>> p = create_string_buffer(b"Opa")       # cria um buffer cont
>>> print(sizeof(p), repr(p.raw))
4 b'Opa\x00'
>>> print(repr(p.value))
b'Opa'
>>> p = create_string_buffer(b"Oi", 10)    # cria um buffer de 1
>>> print(sizeof(p), repr(p.raw))
10 b'Oi\x00\x00\x00\x00\x00\x00\x00\x00'
>>> p.value = b"Oi"
>>> print(sizeof(p), repr(p.raw))
10 b'Oi\x00\x00\x00\x00\x00\x00\x00\x00'
>>>
```

A função **`create_string_buffer()`** substitui a antiga função **`c_buffer()`** (que ainda está disponível como um apelido). Para criar um bloco de

memória mutável contendo caracteres Unicode do tipo C **`wchar_t`**, use a função **`create_unicode_buffer()`**.

Chamando funções, continuação

Observe que printf imprime no canal de saída padrão real, *não* em **`sys.stdout`**, então esses exemplos só funcionarão no prompt do console, não de dentro do *IDLE* ou *PythonWin*:

```
>>> printf = libc.printf
>>> printf(b"Hello, %s\n", b"World!")
Hello, World!
14
>>> printf(b"Hello, %S\n", "World!")
Hello, World!
14
>>> printf(b"%d bottles of beer\n", 42)
42 bottles of beer
19
>>> printf(b"%f bottles of beer\n", 42.5)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ctypes.ArgumentError: argument 2: TypeError: Don't know how to c
>>>
```

Como mencionado anteriormente, todos os tipos Python, exceto inteiros, strings e objetos bytes, devem ser encapsulados no seu tipo **`ctypes`** correspondente, para que possam ser convertidos para o tipo de dados C necessário:

```
>>> printf(b"An int %d, a double %f\n", 1234, c_double(3.14))
An int 1234, a double 3.140000
31
>>>
```

Chamando funções variadas

Em muitas plataformas, chamar funções variádicas por meio do ctypes é exatamente o mesmo que chamar funções com um número fixo de

parâmetros. Em algumas plataformas, em particular no ARM64 para plataformas Apple, a convenção de chamada para funções variádicas difere da usada para funções regulares.

Nessas plataformas é necessário especificar o atributo **argtypes** para os argumentos de funções regulares (não variádicas):

```
libc.printf.argtypes = [ctypes.c_char_p]
```

Já que especificar o atributo não impede a portabilidade, recomenda-se sempre especificar o **argtypes** para todas as funções variádicas.

Chamando funções com seus próprios tipos de dados personalizados

Você também pode personalizar a conversão de argumentos **ctypes** para permitir que instâncias de suas próprias classes sejam usadas como argumentos de função. **ctypes** procura o atributo **_as_parameter_** e o usa como argumento de função. O atributo deve ser um inteiro, string, bytes, uma instância **ctypes** ou um objeto com um atributo **_as_parameter_**:

```
>>> class Bottles:
...     def __init__(self, number):
...         self._as_parameter_ = number
...
>>> bottles = Bottles(42)
>>> printf(b"%d bottles of beer\n", bottles)
42 bottles of beer
19
>>>
```

If you don’t want to store the instance’s data in the **_as_parameter_** instance variable, you could define a **@property** which makes the attribute available on request.

Especificando os tipos de argumentos necessários (protótipos de função)

É possível especificar os tipos de argumentos necessários de funções exportadas de DLLs definindo o atributo **argtypes**.

argtypes deve ser uma sequência de tipos de dados C (a função **printf()** provavelmente não é um bom exemplo nesse caso, pois ela aceita um número variável e diferentes tipos de parâmetros dependendo da string de formato; por outro lado, ela é bastante útil para experimentar esse recurso):

```
>>> printf.argtypes = [c_char_p, c_char_p, c_int, c_double]
>>> printf(b"String '%s', Int %d, Double %f\n", b"Hi", 10, 2.2)
String 'Hi', Int 10, Double 2.200000
37
>>>
```

Especificar um formato protege contra tipos de argumentos incompatíveis (assim como um protótipo para uma função em C), e tenta converter os argumentos para tipos válidos:

```
>>> printf(b"%d %d %d", 1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ctypes.ArgumentError: argument 2: TypeError: 'int' object cannot
>>> printf(b"%s %d %f\n", b"X", 2, 3)
X 2 3.000000
13
>>>
```

If you have defined your own classes which you pass to function calls, you have to implement a **from_param()** class method for them to be able to use them in the **argtypes** sequence. The **from_param()** class method receives the Python object passed to the function call, it should do a typecheck or whatever is needed to make sure this object is acceptable, and then return the object itself, its **_as_parameter_** attribute, or whatever you want to pass as the C function argument in this case. Again, the result should be an integer, string, bytes, a **ctypes** instance, or an object with an **_as_parameter_** attribute.

Tipos de Retorno

Por padrão, as funções são presumidas como retornando o tipo C int. Outros retornos podem ser especificados definindo o atributo **restype** do objeto função.

O protótipo C de **time()** é `time_t time(time_t *)`. Porque **time_t** pode ser de um tipo diferente do tipo de retorno padrão int, você deve especificar o atributo **restype**.

```
>>> libc.time.restype = c_time_t
```

Os tipos de argumentos podem ser especificados usando **argtypes**:

```
>>> libc.time.argtypes = (POINTER(c_time_t),)
```

Para chamar uma função com um ponteiro `NULL` como primeiro argumento, use `None`:

```
>>> print(libc.time(None))
1150640792
```

Aqui está um exemplo mais avançado, ele usa a função **strchr()**, que espera um ponteiro de string e um char, e retorna um ponteiro para uma string:

```
>>> strchr = libc.strchr
>>> strchr(b"abcdef", ord("d"))
8059983
>>> strchr.restype = c_char_p    # c_char_p is a pointer to a st
>>> strchr(b"abcdef", ord("d"))
b'def'
>>> print(strchr(b"abcdef", ord("x")))
None
>>>
```

If you want to avoid the **ord("x")** calls above, you can set the **argtypes** attribute, and the second argument will be converted from a single character Python bytes object into a C char:

```
>>> strchr.restype = c_char_p
>>> strchr.argtypes = [c_char_p, c_char]
```

```
>>> strchr(b"abcdef", b"d")
b'def'
>>> strchr(b"abcdef", b"def")
Traceback (most recent call last):
ctypes.ArgumentError: argument 2: TypeError: one character bytes
>>> print(strchr(b"abcdef", b"x"))
None
>>> strchr(b"abcdef", b"d")
b'def'
>>>
```

Você também pode usar um objeto Python chamável (uma função ou uma classe, por exemplo) como o atributo **`restype`**, se a função externa retornar um inteiro. O objeto chamável será chamado com o *inteiro* que a função C retorna, e o resultado desta chamada será usado como o resultado da sua chamada de função. Isso é útil para verificar valores de retorno de erro e levantar uma exceção automaticamente:

```
>>> GetModuleHandle = windll.kernel32.GetModuleHandleA
>>> def ValidHandle(value):
...     if value == 0:
...         raise WinError()
...     return value
...
>>>
>>> GetModuleHandle.restype = ValidHandle
>>> GetModuleHandle(None)
486539264
>>> GetModuleHandle("something silly")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 3, in ValidHandle
OSError: [Errno 126] The specified module could not be found.
>>>
```

`WinError` é uma função que chamará a API `FormatMessage()` do Windows para obter a representação em string de um código de erro, e *retorna* uma exceção. `WinError` aceita um parâmetro de código de erro opcional, se nenhum for usado, ela chama **`GetLastError()`** para recuperá-lo.

Por favor, repare que um mecanismo de checagem de erro muito mais poderoso está disponível através do atributo **errcheck**; consulte o manual de referência para mais detalhes.

Passando ponteiros (ou: passando parâmetros por referência)

Às vezes, uma função da API C espera um *ponteiro* para um tipo de dado como parâmetro, provavelmente para escrever no local correspondente, ou se os dados forem muito grandes para serem passados por valor. Isso também é conhecido como *passar parâmetros por referência*.

ctypes exports the **byref()** function which is used to pass parameters by reference. The same effect can be achieved with the **pointer()** function, although **pointer()** does a lot more work since it constructs a real pointer object, so it is faster to use **byref()** if you don't need the pointer object in Python itself:

```
>>> i = c_int()
>>> f = c_float()
>>> s = create_string_buffer(b'\000' * 32)
>>> print(i.value, f.value, repr(s.value))
0 0.0 b''
>>> libc.sscanf(b"1 3.14 Hello", b"%d %f %s",
...             byref(i), byref(f), s)
3
>>> print(i.value, f.value, repr(s.value))
1 3.1400001049 b'Hello'
>>>
```

Estruturas e uniões

Structures and unions must derive from the **Structure** and **Union** base classes which are defined in the **ctypes** module. Each subclass must define a **_fields_** attribute. **_fields_** must be a list of *2-tuples*, containing a *field name* and a *field type*.

The field type must be a **ctypes** type like **c_int**, or any other derived **ctypes** type: structure, union, array, pointer.

Aqui está um exemplo simples de uma estrutura POINT, que contém dois inteiros nomeados *x* e *y*, e também mostra como inicializar uma estrutura no construtor:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = [("x", c_int),
...                 ("y", c_int)]
...
>>> point = POINT(10, 20)
>>> print(point.x, point.y)
10 20
>>> point = POINT(y=5)
>>> print(point.x, point.y)
0 5
>>> POINT(1, 2, 3)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: too many initializers
>>>
```

Você pode, no entanto, construir estruturas muito mais complicadas. Uma estrutura pode conter outras estruturas, usando uma estrutura como um tipo de campo.

Aqui está uma estrutura RECT que contém dois POINTs nomeados *upperleft* e *lowerright*:

```
>>> class RECT(Structure):
...     _fields_ = [("upperleft", POINT),
...                 ("lowerright", POINT)]
...
>>> rc = RECT(point)
>>> print(rc.upperleft.x, rc.upperleft.y)
0 5
>>> print(rc.lowerright.x, rc.lowerright.y)
0 0
>>>
```

Estruturas aninhadas também podem ser inicializadas no construtor de várias maneiras:

```
>>> r = RECT(POINT(1, 2), POINT(3, 4))
>>> r = RECT((1, 2), (3, 4))
```

Os descritores de campo podem ser obtidos a partir da *classe*; são úteis para depuração, pois podem fornecer informações úteis. Veja `CField`:

```
>>> POINT.x
<ctypes.CField 'x' type=c_int, ofs=0, size=4>
>>> POINT.y
<ctypes.CField 'y' type=c_int, ofs=4, size=4>
>>>
```

Aviso

`ctypes` does not support passing unions or structures with bit-fields to functions by value. While this may work on 32-bit x86, it's not guaranteed by the library to work in the general case. Unions and structures with bit-fields should always be passed to functions by pointer.

Layout, alinhamento e ordem de bytes de estrutura/união

Por padrão, os campos de Structure e Union são dispostos da mesma forma que o compilador do C o faz. É possível substituir este comportamento inteiramente especificando um atributo de classe `_layout_` na definição da subclasse; veja a documentação do atributo para detalhes.

É possível especificar o alinhamento máximo para os campos e/ou para a própria estrutura definindo os atributos de classe `_pack_` e/ou `_align_` respectivamente. Veja a documentação do atributo para mais detalhes.

`ctypes` uses the native byte order for Structures and Unions. To build structures with non-native byte order, you can use one of the `BigEndianStructure`, `LittleEndianStructure`, `BigEndianUnion`, and `LittleEndianUnion` base classes. These classes cannot contain pointer fields.

Campos de bit em estruturas e uniões

É possível criar estruturas e uniões contendo campos de bits. Campos de bits só são possíveis para campos de inteiros, a largura do bit é especificada como o terceiro item nas tuplas `_fields_`:

```
>>> class Int(Structure):
...     _fields_ = [("first_16", c_int, 16),
...                 ("second_16", c_int, 16)]
...
>>> print(Int.first_16)
<ctypes.CField 'first_16' type=c_int, ofs=0, bit_size=16, bit_of
>>> print(Int.second_16)
<ctypes.CField 'second_16' type=c_int, ofs=0, bit_size=16, bit_c
```

É importante notar que a alocação e o layout de campos de bits na memória não são definidos como um padrão C; sua implementação é específica do compilador. Por padrão, o Python tentará corresponder ao comportamento de um compilador “nativo” para a plataforma atual. Veja o atributo `_layout_` para detalhes sobre o comportamento padrão e como alterá-lo.

Vetores

Vetores são sequências que contêm um número fixo de instâncias do mesmo tipo.

A maneira recomendada de criar tipos vetor é multiplicando um tipo de dado por um inteiro positivo:

```
TenPointsArrayType = POINT * 10
```

Aqui está um exemplo de um tipo de dado um tanto artificial, uma estrutura contendo 4 POINTs entre outras coisas:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = ("x", c_int), ("y", c_int)
...
>>> class MyStruct(Structure):
...     _fields_ = [("a", c_int),
...                 ("b", c_float),
```

```
...                 ("point_array", POINT * 4)]
>>>
>>> print(len(MyStruct().point_array))
4
>>>
```

Instâncias são criadas da maneira usual, chamando a classe:

```
arr = TenPointsArrayType()
for pt in arr:
    print(pt.x, pt.y)
```

O código acima exibe uma série de linhas `0 0`, pois o conteúdo do vetor é inicializado com zeros.

Inicializadores do tipo correto também podem ser especificados:

```
>>> from ctypes import *
>>> TenIntegers = c_int * 10
>>> ii = TenIntegers(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
>>> print(ii)
<c_long_Array_10 object at 0x...>
>>> for i in ii: print(i, end=" ")
...
1 2 3 4 5 6 7 8 9 10
>>>
```

Ponteiros

Pointer instances are created by calling the **`pointer()`** function on a **`ctypes`** type:

```
>>> from ctypes import *
>>> i = c_int(42)
>>> pi = pointer(i)
>>>
```

Instâncias de ponteiros têm um atributo **`contents`** que retorna o objeto para qual o ponteiro aponta, o objeto `i` acima:

```
>>> pi.contents
c_long(42)
>>>
```

Note that **`ctypes`** does not have OOR (original object return), it constructs a new, equivalent object each time you retrieve an attribute:

```
>>> pi.contents is i
False
>>> pi.contents is pi.contents
False
>>>
```

Atribuir outra instância **`c_int`** ao atributo do conteúdo do ponteiro faria com que o ponteiro apontasse para o local de memória onde ela está armazenada:

```
>>> i = c_int(99)
>>> pi.contents = i
>>> pi.contents
c_long(99)
>>>
```

Instâncias de ponteiro também podem ser indexadas com inteiros:

```
>>> pi[0]
99
>>>
```

Atribuir a um índice inteiro altera o valor apontado:

```
>>> print(i)
c_long(99)
>>> pi[0] = 22
>>> print(i)
c_long(22)
>>>
```

Também é possível usar índices diferentes de 0, mas você deve saber o que está fazendo, assim como em C: Você pode acessar ou alterar locais arbitrários da memória. Geralmente, você só usa este recurso se receber um

ponteiro de uma função C, e você *sabe* que o ponteiro na verdade aponta para um vetor em vez de um único item.

Aviso

Because pointer objects support subscription, they implicitly support iteration. Unless doing this in a controlled manner, such as by manually calling `next()` on a `pointer()` iterator, this will typically lead to infinite loops or crashes, because ctypes has no way of knowing when to stop iteration. In other words, a `pointer` iterator will infinitely yield arbitrary memory.

Behind the scenes, the `pointer()` function does more than simply create pointer instances, it has to create pointer *types* first. This is done with the `POINTER()` function, which accepts any **`ctypes`** type, and returns a new type:

```
>>> PI = POINTER(c_int)
>>> PI
<class 'ctypes.LP_c_long'>
>>> PI(42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: expected c_long instead of int
>>> PI(c_int(42))
<ctypes.LP_c_long object at 0x...>
>>>
```

Chamar o tipo ponteiro sem um argumento cria um ponteiro `NULL`. Ponteiros `NULL` possuem o valor booleano `False`:

```
>>> null_ptr = POINTER(c_int)()
>>> print(bool(null_ptr))
False
>>>
```

`ctypes` checks for `NULL` when dereferencing pointers (but dereferencing invalid non-`NULL` pointers would crash Python):

```
>>> null_ptr[0]
Traceback (most recent call last):
    ....
ValueError: NULL pointer access
>>>

>>> null_ptr[0] = 1234
Traceback (most recent call last):
    ....
ValueError: NULL pointer access
>>>
```

Segurança de thread (Thread safety) sem o GIL

From Python 3.13 onward, the GIL can be disabled on the free-threaded build. In ctypes, reads and writes to a single object concurrently is safe, but not across multiple objects:

```
>>> number = c_int(42)
>>> pointer_a = pointer(number)
>>> pointer_b = pointer(number)
```

No exemplo acima, é seguro apenas para um objeto ler e escrever no endereço simultaneamente se o GIL estiver desabilitado. Portanto, `pointer_a` pode ser compartilhado e escrito entre múltiplas threads, mas apenas se `pointer_b` não estiver também tentando fazer o mesmo. Se isso for um problema, considere usar um **`threading.Lock`** para sincronizar o acesso à memória:

```
>>> import threading
>>> lock = threading.Lock()
>>> # Thread 1
>>> with lock:
...     pointer_a.contents = 24
>>> # Thread 2
>>> with lock:
...     pointer_b.contents = 42
```

Conversão de Tipos

Normalmente, o ctypes faz uma verificação de tipos estrita. Isso significa que, se você tiver `POINTER(c_int)` na lista **argtypes** de uma função ou como o tipo de um campo membro em uma definição de estrutura, apenas instâncias de exatamente o mesmo tipo são aceitas. Existem algumas exceções a esta regra, onde o ctypes aceita outros objetos. Por exemplo, você pode passar instâncias de vetor compatíveis em vez de tipos de ponteiro. Assim, para `POINTER(c_int)`, o ctypes aceita um vetor de c_int:

```
>>> class Bar(Structure):
...     _fields_ = [("count", c_int), ("values", POINTER(c_int))
...
>>> bar = Bar()
>>> bar.values = (c_int * 3)(1, 2, 3)
>>> bar.count = 3
>>> for i in range(bar.count):
...     print(bar.values[i])
...
1
2
3
>>>
```

Além disso, se um argumento de função for explicitamente declarado como um tipo ponteiro (como `POINTER(c_int)`) em **argtypes**, um objeto do tipo apontado (`c_int` neste caso) pode ser passado para a função. O ctypes aplicará a conversão **byref()** necessária neste caso automaticamente.

Para definir um campo do tipo PONTEIRO como `NULL`, você pode atribuir `None`:

```
>>> bar.values = None
>>>
```

Sometimes you have instances of incompatible types. In C, you can cast one type into another type. **ctypes** provides a **cast()** function which can be used in the same way. The `Bar` structure defined above accepts `POINTER(c_int)` pointers or **c_int** arrays for its `values` field, but not instances of other types:

```
>>> bar.values = (c_byte * 4)()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: incompatible types, c_byte_Array_4 instance instead c
>>>
```

Para esses casos, a função `cast()` é útil.

A função `cast()` pode ser usada para converter uma instância ctypes em um ponteiro para um diferente tipo de dado ctypes. **`cast()`** recebe dois parâmetros, um objeto ctypes que é ou pode ser convertido para um ponteiro de algum tipo, e um tipo ponteiro ctypes. Ela retorna uma instância do segundo argumento, que referencia o mesmo bloco de memória que o primeiro argumento:

```
>>> a = (c_byte * 4)()
>>> cast(a, POINTER(c_int))
<ctypes.LP_c_long object at ...>
>>>
```

Então, `cast()` pode ser usada para atribuir ao campo `values` da estrutura `Bar`:

```
>>> bar = Bar()
>>> bar.values = cast((c_byte * 4)(), POINTER(c_int))
>>> print(bar.values[0])
0
>>>
```

Tipos Incompletos

Tipos Incompletos são estruturas, uniões ou vetores, cujos membros ainda não foram especificados. Em C, eles são especificados por declarações antecipadas, que são definidas posteriormente:

```
struct cell; /* forward declaration */

struct cell {
    char *name;
```

```
    struct cell *next;
};
```

A tradução direta para código ctypes seria esta, mas não funciona:

```
>>> class cell(Structure):
...     _fields_ = [("name", c_char_p),
...                 ("next", POINTER(cell))]
...
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in cell
NameError: name 'cell' is not defined
>>>
```

because the new `class cell` is not available in the class statement itself. In **`ctypes`**, we can define the `cell` class and set the `_fields_` attribute later, after the class statement:

```
>>> from ctypes import *
>>> class cell(Structure):
...     pass
...
>>> cell._fields_ = [("name", c_char_p),
...                  ("next", POINTER(cell))]
>>>
```

Vamos tentar. Criamos duas instâncias de `cell`, e deixamos que elas apontem uma para a outra, e finalmente seguimos a cadeia de ponteiros algumas vezes:

```
>>> c1 = cell()
>>> c1.name = b"foo"
>>> c2 = cell()
>>> c2.name = b"bar"
>>> c1.next = pointer(c2)
>>> c2.next = pointer(c1)
>>> p = c1
>>> for i in range(8):
...     print(p.name, end=" ")
...     p = p.next[0]
...
```

```
foo bar foo bar foo bar foo bar
>>>
```

Funções Callbacks

`ctypes` allows creating C callable function pointers from Python callables. These are sometimes called *callback functions*.

Primeiro, você deve criar uma classe para função de retorno. A classe sabe a convenção de chamada , o tipo de retorno, e o número e tipos de argumentos que essa função irá receber.

A função de fábrica **`CFUNCTYPE()`** cria tipos para funções de retorno usando a convenção de chamada `cdecl`. No Windows, a função de fábrica **`WINFUNCTYPE()`** cria tipos para funções de retorno usando a convenção de chamada `stdcall`.

Ambas estas funções de fábrica são chamadas com o tipo de resultado como primeiro argumento, e os tipos de argumento esperados da função de retorno como os argumentos restantes.

I will present an example here which uses the standard C library's **`qsort()`** function, that is used to sort items with the help of a callback function. **`qsort()`** will be used to sort an array of integers:

```
>>> IntArray5 = c_int * 5
>>> ia = IntArray5(5, 1, 7, 33, 99)
>>> qsort = libc.qsort
>>> qsort.restype = None
>>>
```

A **`qsort()`** deve ser chamada com um ponteiro para os dados a serem ordenados, o número de itens no vetor de dados, o tamanho de um item, e um ponteiro para a função de comparação, a função de retorno (callback). A função de retorno será então chamada com dois ponteiros para itens, e ela deve retornar um inteiro negativo se o primeiro item for menor que o segundo, um zero se eles forem iguais, e um inteiro positivo caso contrário.

Então, nossa função de retorno (callback function) recebe ponteiros para inteiros, e deve retornar um inteiro. Primeiro criamos o `type` para a função de retorno:

```
>>> CMPFUNC = CFUNCTYPE(c_int, POINTER(c_int), POINTER(c_int))
>>>
```

Para começar, aqui está uma função de retorno (callback) simples que mostra os valores que lhe são passados:

```
>>> def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return 0
...
>>> cmp_func = CMPFUNC(py_cmp_func)
>>>
```

O resultado:

```
>>> qsort(ia, len(ia), sizeof(c_int), cmp_func)
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 5 7
py_cmp_func 1 7
>>>
```

Agora podemos realmente comparar os dois itens e retornar um resultado útil:

```
>>> def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return a[0] - b[0]
...
>>>
>>> qsort(ia, len(ia), sizeof(c_int), CMPFUNC(py_cmp_func))
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 1 7
py_cmp_func 5 7
>>>
```

Como podemos verificar facilmente, nosso vetor está ordenado agora:

```
>>> for i in ia: print(i, end=" ")
...
1 5 7 33 99
>>>
```

As fábricas de funções podem ser usadas como fábricas de decoradores, então também podemos escrever:

```
>>> @CFUNCTYPE(c_int, POINTER(c_int), POINTER(c_int))
... def py_cmp_func(a, b):
...     print("py_cmp_func", a[0], b[0])
...     return a[0] - b[0]
...
>>> qsort(ia, len(ia), sizeof(c_int), py_cmp_func)
py_cmp_func 5 1
py_cmp_func 33 99
py_cmp_func 7 33
py_cmp_func 1 7
py_cmp_func 5 7
>>>
```

Nota

Make sure you keep references to `CFUNCTYPE()` objects as long as they are used from C code. `ctypes` doesn't, and if you don't, they may be garbage collected, crashing your program when a callback is made.

Além disso, note que se a função de retorno for chamada em uma thread criada fora do controle do Python (por exemplo, pelo código externo que chama a função de retorno), o ctypes cria uma nova thread Python "dummy" (fictícia) em cada invocação. Este comportamento é correto para a maioria dos propósitos, mas significa que os valores armazenados com `threading.local` *não* sobreviverão entre diferentes funções de retorno, mesmo quando essas chamadas são feitas a partir da mesma thread C.

Acessando valores exportados de dlls

Algumas bibliotecas compartilhadas não exportam apenas funções, elas também exportam variáveis. Um exemplo na própria biblioteca Python é a `Py_Version`, o número da versão de tempo de execução do Python codificado em um único inteiro constante.

`ctypes` can access values like this with the `in_dll()` class methods of the type. *pythonapi* is a predefined symbol giving access to the Python C api:

```
>>> version = ctypes.c_int.in_dll(ctypes.pythonapi, "Py_Version'
>>> print(hex(version.value))
0x30c00a0
```

Um exemplo estendido que também demonstra o uso de ponteiros acessa o ponteiro `PyImport_FrozenModules` exportado pelo Python.

Citando a documentação para esse valor:

> Este ponteiro é inicializado para apontar para um vetor de registros de `_frozen`, terminado por um cujos membros são todos `NULL` ou zero. Quando um módulo congelado é importado, ele é pesquisado nesta tabela. O código de terceiros pode fazer truques com isso para fornecer uma coleção criada dinamicamente de módulos congelados.

So manipulating this pointer could even prove useful. To restrict the example size, we show only how this table can be read with **`ctypes`**:

```
>>> from ctypes import *
>>>
>>> class struct_frozen(Structure):
...     _fields_ = [("name", c_char_p),
...                 ("code", POINTER(c_ubyte)),
...                 ("size", c_int),
...                 ("get_code", POINTER(c_ubyte)),  # Function
...                ]
...
>>>
```

Nós definimos tipo de dado `_frozen`, para que possamos obter o ponteiro para a tabela:

```
>>> FrozenTable = POINTER(struct_frozen)
>>> table = FrozenTable.in_dll(pythonapi, "_PyImport_FrozenBoots
>>>
```

Como `table` é um `pointer` para o vetor de registros `struct_frozen`, nós podemos iterar sobre ele, mas só temos que nos certificar de que nosso loop termine, porque ponteiros não têm tamanho. Cedo ou tarde, ele provavelmente travaria com uma violação de acesso ou algo assim, então é melhor sair do loop quando atingirmos a entrada `NULL`:

```
>>> for item in table:
...     if item.name is None:
...         break
...     print(item.name.decode("ascii"), item.size)
...
_frozen_importlib 31764
_frozen_importlib_external 41499
zipimport 12345
>>>
```

O fato de que o Python padrão tem um módulo congelado e um pacote congelado (indicado pelo membro `size` negativo) não é bem conhecido, é usado apenas para testes. Experimente com `import __hello__` por exemplo.

Surpresas

There are some edges in **`ctypes`** where you might expect something other than what actually happens.

Considere o exemplo a seguir:

```
>>> from ctypes import *
>>> class POINT(Structure):
...     _fields_ = ("x", c_int), ("y", c_int)
...
```

```
>>> class RECT(Structure):
...     _fields_ = ("a", POINT), ("b", POINT)
...
>>> p1 = POINT(1, 2)
>>> p2 = POINT(3, 4)
>>> rc = RECT(p1, p2)
>>> print(rc.a.x, rc.a.y, rc.b.x, rc.b.y)
1 2 3 4
>>> # now swap the two points
>>> rc.a, rc.b = rc.b, rc.a
>>> print(rc.a.x, rc.a.y, rc.b.x, rc.b.y)
3 4 3 4
>>>
```

Hm. Nós com certeza esperávamos que a última instrução exibisse `3 4 1 2`. O que aconteceu? Aqui estão os passos da linha `rc.a, rc.b = rc.b, rc.a` acima:

```
>>> temp0, temp1 = rc.b, rc.a
>>> rc.a = temp0
>>> rc.b = temp1
>>>
```

Observe que `temp0` e `temp1` ainda são objetos que utilizam o buffer interno do objeto `rc` acima. Portanto, executar `rc.a = temp0` copia o conteúdo do buffer de `temp0` para o buffer de `rc`. Isso, por sua vez, altera o conteúdo de `temp1`. Assim, a última atribuição `rc.b = temp1` não produz o efeito esperado.

Tenha em mente que obter sub-objetos de estruturas, uniões e vetores não copia o sub-objeto; em vez disso, recupera um objeto de encapsulamento (wrapper) que acessa o buffer subjacente do objeto raiz.

Outro exemplo que pode se comportar de maneira diferente do que alguém poderia esperar é o seguinte:

```
>>> s = c_char_p()
>>> s.value = b"abc def ghi"
>>> s.value
b'abc def ghi'
>>> s.value is s.value
```

```
False
>>>
```

Nota

Objetos instanciados a partir de `c_char_p` só podem ter seu valor definido para bytes ou inteiros.

Por que está imprimindo `False`? Instâncias de ctypes são objetos que contêm um bloco de memória, além de alguns descriptors que acessam o conteúdo dessa memória. Armazenar um objeto Python no bloco de memória não armazena o próprio objeto; em vez disso, armazena-se o seu atributo `contents`. Acessar o contents novamente constrói um novo objeto Python a cada vez!

Tipos de dados de tamanho variável

`ctypes` provides some support for variable-sized arrays and structures.

A função `resize()` pode ser usada para redimensionar o buffer de memória de um objeto ctypes existente. A função recebe o objeto como primeiro argumento, e o tamanho solicitado em bytes como o segundo argumento. O bloco de memória não pode ser tornado menor do que o bloco de memória natural especificado pelo tipo do objeto, um `ValueError` é levantado se isso for tentado:

```
>>> short_array = (c_short * 4)()
>>> print(sizeof(short_array))
8
>>> resize(short_array, 4)
Traceback (most recent call last):
    ...
ValueError: minimum size is 8
>>> resize(short_array, 32)
>>> sizeof(short_array)
32
>>> sizeof(type(short_array))
```

```
8
>>>
```

Isso é bom e funcional, mas como alguém acessaria os elementos contidos neste vetor? Já que o tipo ainda sabe apenas sobre 4 elementos, obtemos erros ao acessar outros elementos:

```
>>> short_array[:]
[0, 0, 0, 0]
>>> short_array[7]
Traceback (most recent call last):
    ...
IndexError: invalid index
>>>
```

Another way to use variable-sized data types with **`ctypes`** is to use the dynamic nature of Python, and (re-)define the data type after the required size is already known, on a case by case basis.

Referência ctypes

Loading shared libraries

There are several ways to load shared libraries into the Python process. One way is to instantiate **`CDLL`** or one of its subclasses:

class ctypes.CDLL(*name*, *mode=DEFAULT_MODE*, *handle=None*, *use_errno=False*, *use_last_error=False*, *winmode=None*)

Represents a loaded shared library.

Functions in this library use the standard C calling convention, and are assumed to return int. The Python global interpreter lock is released before calling any function exported by these libraries, and reacquired afterwards. For different function behavior, use a subclass: **`OleDLL`**, **`WinDLL`**, or **`PyDLL`**.

If you have an existing **`handle`** to an already loaded shared library, it can be passed as the *handle* argument to wrap the opened library in a new

CDLL object. In this case, *name* is only used to set the **_name** attribute, but it may be adjusted and/or validated.

If *handle* is None, the underlying platform's *dlopen(3)* *[https://manpages.debian.org/dlopen(3)]* or LoadLibraryExW [https://learn.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibraryexw] function is used to load the library into the process, and to get a handle to it.

name is the pathname of the shared library to open. If *name* does not contain a path separator, the library is found in a platform-specific way.

On Windows, the .DLL suffix may be missing. (For details, see LoadLibraryExW [https://learn.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibraryexw] documentation.) Other platform-specific prefixes and suffixes (for example, lib, .so, .dylib, or version numbers) must be present in *name*; they are not added automatically. See Encontrando bibliotecas compartilhadas for more information.

On non-Windows systems, *name* can be None. In this case, **dlopen()** is called with NULL, which opens the main program as a "library". (Some systems do the same is *name* is empty; None/NULL is more portable.)

Detalhes da implementação do CPython:

Since CPython is linked to libc, a None *name* is often used to access the C standard library:

```
>>> printf = ctypes.CDLL(None).printf
>>> printf.argtypes = [ctypes.c_char_p]
>>> printf(b"hello\n")
hello
6
```

To access the Python C API, prefer **ctypes.pythonapi** which works across platforms.

O parâmetro *mode* pode ser utilizado para especificar como a biblioteca é carregada. Para detalhes, consulte a página de manual *dlopen(3)*

[https://manpages.debian.org/dlopen(3)]. No Windows, *mode* é ignorado. Em sistemas posix, RTLD_NOW é sempre adicionado e, não é configurável.

The *use_errno* parameter, when set to true, enables a ctypes mechanism that allows accessing the system **`errno`** error number in a safe way. **`ctypes`** maintains a thread-local copy of the system's **`errno`** variable; if you call foreign functions created with `use_errno=True` then the **`errno`** value before the function call is swapped with the ctypes private copy, the same happens immediately after the function call.

The function **`ctypes.get_errno()`** returns the value of the ctypes private copy, and the function **`ctypes.set_errno()`** changes the ctypes private copy to a new value and returns the former value.

The *use_last_error* parameter, when set to true, enables the same mechanism for the Windows error code which is managed by the **`GetLastError()`** and **`SetLastError()`** Windows API functions; **`ctypes.get_last_error()`** and **`ctypes.set_last_error()`** are used to request and change the ctypes private copy of the windows error code.

The *winmode* parameter is used on Windows to specify how the library is loaded (since *mode* is ignored). It takes any value that is valid for the Win32 API LoadLibraryExW [https://learn.microsoft.com/en-us/windows/win32/api/libloaderapi/nf-libloaderapi-loadlibraryexw] flags parameter. When omitted, the default is to use the flags that result in the most secure DLL load, which avoids issues such as DLL hijacking. Passing the full path to the DLL is the safest way to ensure the correct library and dependencies are loaded.

On Windows creating a **`CDLL`** instance may fail even if the DLL name exists. When a dependent DLL of the loaded DLL is not found, a **`OSError`** error is raised with the message *"[WinError 126] The specified module could not be found"*. This error message does not contain the name of the missing DLL because the Windows API does not return this information making this error hard to diagnose. To resolve this error and determine which DLL is not found, you need to find the list of

dependent DLLs and determine which one is not found using Windows debugging and tracing tools.

Ver também

Microsoft DUMPBIN tool [https://learn.microsoft.com/en-us/cpp/build/reference/dumpbin-reference?view=msvc-170] – A tool to find DLL dependents.

Alterado na versão 3.8: Added *winmode* parameter.

Alterado na versão 3.12: The *name* parameter can now be a path-like object.

Instances of this class have no public methods. Functions exported by the shared library can be accessed as attributes or by index. Please note that accessing the function through an attribute caches the result and therefore accessing it repeatedly returns the same object each time. On the other hand, accessing it through an index returns a new object each time:

```
>>> from ctypes import CDLL
>>> libc = CDLL("libc.so.6")  # On Linux
>>> libc.time == libc.time
True
>>> libc['time'] == libc['time']
False
```

The following public attributes are available. Their name starts with an underscore to not clash with exported function names:

_handle
: The system handle used to access the library.

_name
: The name of the library passed in the constructor.

class ctypes.OleDLL

See **CDLL**, the superclass, for common information.

Functions in this library use the `stdcall` calling convention, and are assumed to return the windows specific **`HRESULT`** code. **`HRESULT`** values contain information specifying whether the function call failed or succeeded, together with additional error code. If the return value signals a failure, an **`OSError`** is automatically raised.

Disponibilidade: Windows

Alterado na versão 3.3: Costumava-se levantar **`WindowsError`**, que agora é um apelido de **`OSError`**.

class ctypes.WinDLL

See **`CDLL`**, the superclass, for common information.

Functions in these libraries use the `stdcall` calling convention, and are assumed to return int by default.

Disponibilidade: Windows

class ctypes.PyDLL

See **`CDLL`**, the superclass, for common information.

When functions in this library are called, the Python GIL is *not* released during the function call, and after the function execution the Python error flag is checked. If the error flag is set, a Python exception is raised.

Thus, this is only useful to call Python C API functions directly.

ctypes.RTLD_GLOBAL

Flag to use as *mode* parameter. On platforms where this flag is not available, it is defined as the integer zero.

ctypes.RTLD_LOCAL

Flag to use as *mode* parameter. On platforms where this is not available, it is the same as *RTLD_GLOBAL*.

ctypes.DEFAULT_MODE

The default mode which is used to load shared libraries. On OSX 10.3, this is *RTLD_GLOBAL*, otherwise it is the same as *RTLD_LOCAL*.

Shared libraries can also be loaded by using one of the prefabricated objects, which are instances of the **`LibraryLoader`** class, either by calling the **`LoadLibrary()`** method, or by retrieving the library as attribute of the loader instance.

class ctypes.LibraryLoader(*dlltype*)

Class which loads shared libraries. *dlltype* should be one of the **`CDLL`**, **`PyDLL`**, **`WinDLL`**, or **`OleDLL`** types.

`__getattr__()` has special behavior: It allows loading a shared library by accessing it as attribute of a library loader instance. The result is cached, so repeated attribute accesses return the same library each time.

LoadLibrary(*name*)

Load a shared library into the process and return it. This method always returns a new instance of the library.

These prefabricated library loaders are available:

ctypes.cdll

Creates **`CDLL`** instances.

ctypes.windll

Creates **`WinDLL`** instances.

Disponibilidade: Windows

ctypes.oledll

Creates **`OleDLL`** instances.

Disponibilidade: Windows

ctypes.pydll

Creates **`PyDLL`** instances.

ctypes.pythonapi

An instance of **PyDLL** that exposes Python C API functions as attributes. Note that all these functions are assumed to return C int, which is of course not always the truth, so you have to assign the correct **restype** attribute to use these functions.

Nota

If the Python interpreter is statically linked, this may be `None`.

Carregar uma biblioteca através de qualquer um desses objetos levanta um evento de auditoria `ctypes.dlopen` com o argumento string `name`, o nome usado para carregar a biblioteca.

Accessing a function on a loaded library raises an auditing event `ctypes.dlsym` with arguments `library` (the library object) and `name` (the symbol's name as a string or integer).

In cases when only the library handle is available rather than the object, accessing a function raises an auditing event `ctypes.dlsym/handle` with arguments `handle` (the raw library handle) and `name`.

Encontrando bibliotecas compartilhadas

When programming in a compiled language, shared libraries are accessed when compiling/linking a program, and when the program is run. The programmer specifies a short name; the C compiler, linker, and runtime dynamic library loader then interact in system-specific ways to find the filename of the library to load.

While the mapping from short names to filenames is not consistently exposed by platforms, the **ctypes.util** module provides a function, **find_library()**, that attempts to match it. However, as backwards compatibility concerns make it difficult to adjust its behavior for new platforms and configurations, the function is soft deprecated.

If wrapping a shared library with **ctypes**, consider determining the shared library name at development time, and hardcoding it into the wrapper module instead of using **find_library()** to locate the library at runtime. Also consider adding a configuration option or environment variable to let users select a library to use, and then perhaps use **find_library()** as a default or fallback.

ctypes.util.find_library(*name*)

Try to find a library and return a pathname.

name is the "short" library name without any prefix like `lib`, suffix like `.so`, `.dylib` or version number. (This is the form used for the posix linker option **-l**.) The result is in a format suitable for passing to **CDLL**.

If no library can be found, return `None`.

The exact functionality is system dependent, and is *not guaranteed* to match the behavior of the compiler, linker, and loader used for (or by) Python. It is recommended to only use this function as a default or fallback,

Suavemente descontinuado desde a versão 3.15: This function is kept for use in cases where it works, but not expected to be updated for additional platforms and configurations.

On Linux, **find_library()** tries to run external programs (`/sbin/ldconfig`, `gcc`, `objdump` and `ld`) to find the library file. If the output of these programs does not correspond to the dynamic linker used by Python, the result of this function may be misleading.

Alterado na versão 3.6: On Linux, the value of the environment variable `LD_LIBRARY_PATH` is used when searching for libraries, if a library cannot be found by any other means.

Veja alguns exemplos:

```
>>> from ctypes.util import find_library
>>> find_library("m")
```

```
'libm.so.6'
>>> find_library("c")
'libc.so.6'
>>> find_library("bz2")
'libbz2.so.1.0'
>>>
```

On macOS and Android, **`find_library()`** uses the system's standard naming schemes and paths to locate the library, and returns a full pathname if successful:

```
>>> from ctypes.util import find_library
>>> find_library("c")
'/usr/lib/libc.dylib'
>>> find_library("m")
'/usr/lib/libm.dylib'
>>> find_library("bz2")
'/usr/lib/libbz2.dylib'
>>> find_library("AGL")
'/System/Library/Frameworks/AGL.framework/AGL'
>>>
```

On Windows, **`find_library()`** searches along the system search path, and returns the full pathname, but since there is no predefined naming scheme a call like `find_library("c")` will fail and return `None`.

ctypes.util.find_msvcrt()

Returns the filename of the VC runtime library used by Python, and by the extension modules.

If the name of the library cannot be determined, `None` is returned. Notably, this will happen for recent versions of the VC runtime library, which are not directly loadable.

If you need to free memory, for example, allocated by an extension module with a call to the `free(void *)`, it is important that you use the function in the same library that allocated the memory.

Disponibilidade: Windows

Listing loaded shared libraries

When writing code that relies on code loaded from shared libraries, it can be useful to know which shared libraries have already been loaded into the current process.

The **ctypes.util** module provides the **dllist()** function, which calls the different APIs provided by the various platforms to help determine which shared libraries have already been loaded into the current process.

The exact output of this function will be system dependent. On most platforms, the first entry of this list represents the current process itself, which may be an empty string. For example, on glibc-based Linux, the return may look like:

```
>>> from ctypes.util import dllist
>>> dllist()
['', 'linux-vdso.so.1', '/lib/x86_64-linux-gnu/libm.so.6', '/lib
```

Foreign functions

As explained in the previous section, foreign functions can be accessed as attributes of loaded shared libraries. The function objects created in this way by default accept any number of arguments, accept any ctypes data instances as arguments, and return the default result type specified by the library loader.

They are instances of a private local class **_FuncPtr** (not exposed in **ctypes**) which inherits from the private **_CFuncPtr** class:

```
>>> import ctypes
>>> lib = ctypes.CDLL(None)
>>> issubclass(lib._FuncPtr, ctypes._CFuncPtr)
True
>>> lib._FuncPtr is ctypes._CFuncPtr
False
```

class ctypes._CFuncPtr

Base class for C callable foreign functions.

Instances of foreign functions are also C compatible data types; they represent C function pointers.

This behavior can be customized by assigning to special attributes of the foreign function object.

restype

Assign a ctypes type to specify the result type of the foreign function. Use `None` for void, a function not returning anything.

It is possible to assign a callable Python object that is not a ctypes type, in this case the function is assumed to return a C int, and the callable will be called with this integer, allowing further processing or error checking. Using this is deprecated, for more flexible post processing or error checking use a ctypes data type as **`restype`** and assign a callable to the **`errcheck`** attribute.

argtypes

Assign a tuple of ctypes types to specify the argument types that the function accepts. Functions using the `stdcall` calling convention can only be called with the same number of arguments as the length of this tuple; functions using the C calling convention accept additional, unspecified arguments as well.

When a foreign function is called, each actual argument is passed to the **`from_param()`** class method of the items in the **`argtypes`** tuple, this method allows adapting the actual argument to an object that the foreign function accepts. For example, a **`c_char_p`** item in the **`argtypes`** tuple will convert a string passed as argument into a bytes object using ctypes conversion rules.

New: It is now possible to put items in argtypes which are not ctypes types, but each item must have a **`from_param()`** method which returns a value usable as argument (integer, string, ctypes instance). This allows defining adapters that can adapt custom objects as function parameters.

errcheck

Assign a Python function or another callable to this attribute. The callable will be called with three or more arguments:

callable(*result*, *func*, *arguments*)

result is what the foreign function returns, as specified by the **`restype`** attribute.

func is the foreign function object itself, this allows reusing the same callable object to check or post process the results of several functions.

arguments is a tuple containing the parameters originally passed to the function call, this allows specializing the behavior on the arguments used.

The object that this function returns will be returned from the foreign function call, but it can also check the result value and raise an exception if the foreign function call failed.

On Windows, when a foreign function call raises a system exception (for example, due to an access violation), it will be captured and replaced with a suitable Python exception. Further, an auditing event `ctypes.set_exception` with argument `code` will be raised, allowing an audit hook to replace the exception with its own.

Some ways to invoke foreign function calls as well as some of the functions in this module may raise an auditing event `ctypes.call_function` with arguments `function pointer` and `arguments`.

Function prototypes

Foreign functions can also be created by instantiating function prototypes. Function prototypes are similar to function prototypes in C; they describe a function (return type, argument types, calling convention) without defining an implementation. The factory functions must be called with the desired result type and the argument types of the function, and can be used as

decorator factories, and as such, be applied to functions through the `@wrapper` syntax. See Funções Callbacks for examples.

ctypes.CFUNCTYPE(*restype*, **argtypes*, *use_errno=False*, *use_last_error=False*)

The returned function prototype creates functions that use the standard C calling convention. The function will release the GIL during the call. If *use_errno* is set to true, the ctypes private copy of the system **`errno`** variable is exchanged with the real **`errno`** value before and after the call; *use_last_error* does the same for the Windows error code.

ctypes.WINFUNCTYPE(*restype*, **argtypes*, *use_errno=False*, *use_last_error=False*)

The returned function prototype creates functions that use the `stdcall` calling convention. The function will release the GIL during the call. *use_errno* and *use_last_error* have the same meaning as above.

Disponibilidade: Windows

ctypes.PYFUNCTYPE(*restype*, **argtypes*)

The returned function prototype creates functions that use the Python calling convention. The function will *not* release the GIL during the call.

Function prototypes created by these factory functions can be instantiated in different ways, depending on the type and number of the parameters in the call:

prototype(*address*)

Returns a foreign function at the specified address which must be an integer.

prototype(*callable*)

Create a C callable function (a callback function) from a Python *callable*.

prototype(*func_spec*[, *paramflags*])

Returns a foreign function exported by a shared library. *func_spec* must be a 2-tuple `(name_or_ordinal, library)`. The first item is the name of the exported function as string, or the ordinal of the exported function as small integer. The second item is the shared library instance.

prototype(*vtbl_index*, *name*[, *paramflags*[, *iid*]])

Returns a foreign function that will call a COM method. *vtbl_index* is the index into the virtual function table, a small non-negative integer. *name* is name of the COM method. *iid* is an optional pointer to the interface identifier which is used in extended error reporting.

If *iid* is not specified, an **`OSError`** is raised if the COM method call fails. If *iid* is specified, a **`COMError`** is raised instead.

COM methods use a special calling convention: They require a pointer to the COM interface as first argument, in addition to those parameters that are specified in the **`argtypes`** tuple.

Disponibilidade: Windows

The optional *paramflags* parameter creates foreign function wrappers with much more functionality than the features described above.

paramflags must be a tuple of the same length as **`argtypes`**.

Each item in this tuple contains further information about a parameter, it must be a tuple containing one, two, or three items.

The first item is an integer containing a combination of direction flags for the parameter:

1

Specifies an input parameter to the function.

2

Output parameter. The foreign function fills in a value.

4

Input parameter which defaults to the integer zero.

The optional second item is the parameter name as string. If this is specified, the foreign function can be called with named parameters.

The optional third item is the default value for this parameter.

The following example demonstrates how to wrap the Windows `MessageBoxW` function so that it supports default parameters and named arguments. The C declaration from the windows header file is this:

```
WINUSERAPI int WINAPI
MessageBoxW(
    HWND hWnd,
    LPCWSTR lpText,
    LPCWSTR lpCaption,
    UINT uType);
```

Here is the wrapping with **`ctypes`**:

```
>>> from ctypes import c_int, WINFUNCTYPE, windll
>>> from ctypes.wintypes import HWND, LPCWSTR, UINT
>>> prototype = WINFUNCTYPE(c_int, HWND, LPCWSTR, LPCWSTR, UINT)
>>> paramflags = (1, "hwnd", 0), (1, "text", "Hi"), (1, "captior
>>> MessageBox = prototype(("MessageBoxW", windll.user32), param
```

The `MessageBox` foreign function can now be called in these ways:

```
>>> MessageBox()
>>> MessageBox(text="Spam, spam, spam")
>>> MessageBox(flags=2, text="foo bar")
```

A second example demonstrates output parameters. The win32 `GetWindowRect` function retrieves the dimensions of a specified window by copying them into `RECT` structure that the caller has to supply. Here is the C declaration:

```
WINUSERAPI BOOL WINAPI
GetWindowRect(
```

```
     HWND hWnd,
     LPRECT lpRect);
```

Here is the wrapping with **`ctypes`**:

```
>>> from ctypes import POINTER, WINFUNCTYPE, windll, WinError
>>> from ctypes.wintypes import BOOL, HWND, RECT
>>> prototype = WINFUNCTYPE(BOOL, HWND, POINTER(RECT))
>>> paramflags = (1, "hwnd"), (2, "lprect")
>>> GetWindowRect = prototype(("GetWindowRect", windll.user32),
>>>
```

Functions with output parameters will automatically return the output parameter value if there is a single one, or a tuple containing the output parameter values when there are more than one, so the GetWindowRect function now returns a RECT instance, when called.

Output parameters can be combined with the **`errcheck`** protocol to do further output processing and error checking. The win32 `GetWindowRect` api function returns a `BOOL` to signal success or failure, so this function could do the error checking, and raises an exception when the api call failed:

```
>>> def errcheck(result, func, args):
...     if not result:
...         raise WinError()
...     return args
...
>>> GetWindowRect.errcheck = errcheck
>>>
```

If the **`errcheck`** function returns the argument tuple it receives unchanged, **`ctypes`** continues the normal processing it does on the output parameters. If you want to return a tuple of window coordinates instead of a `RECT` instance, you can retrieve the fields in the function and return them instead, the normal processing will no longer take place:

```
>>> def errcheck(result, func, args):
...     if not result:
...         raise WinError()
```

```
...     rc = args[1]
...     return rc.left, rc.top, rc.bottom, rc.right
...
>>> GetWindowRect.errcheck = errcheck
>>>
```

Funções utilitárias

ctypes.addressof(*obj*)

Returns the address of the memory buffer as integer. *obj* must be an instance of a ctypes type.

Levanta um evento de auditoria `ctypes.addressof` com o argumento `obj`.

ctypes.alignment(*obj_or_type*)

Returns the alignment requirements of a ctypes type. *obj_or_type* must be a ctypes type or instance.

ctypes.byref(*obj*[, *offset*])

Returns a light-weight pointer to *obj*, which must be an instance of a ctypes type. *offset* defaults to zero, and must be an integer that will be added to the internal pointer value.

`byref(obj, offset)` corresponds to this C code:

```
(((char *)&obj) + offset)
```

The returned object can only be used as a foreign function call parameter. It behaves similar to `pointer(obj)`, but the construction is a lot faster.

ctypes.CopyComPointer(*src*, *dst*)

Copia um ponteiro COM de *src* para *dst* e devolve o valor **`HRESULT`** específico para Windows.

Se *src* não for `NULL`, o seu método `AddRef` é chamado, incrementando a contagem de referências.

Em contraste, a contagem de referências de *dst* não será decrementada antes de atribuir o novo valor. A menos que *dst* seja `NULL`, o chamador é responsável por decrementar a contagem de referências, chamando o método `Release` de *dst* quando necessário.

Disponibilidade: Windows

Adicionado na versão 3.14.

ctypes.cast(*obj*, *type*)

Esta função é semelhante ao operador cast em C. Retorna uma nova instância de *type* que aponta para o mesmo bloco de memória que *obj*. *type* deve ser um tipo de ponteiro e *obj* deve ser um objeto que pode ser interpretado como um ponteiro.

ctypes.create_string_buffer(*init*, *size=None*)

ctypes.create_string_buffer(*size*)

This function creates a mutable character buffer. The returned object is a ctypes array of **`c_char`**.

If *size* is given (and not `None`), it must be an **`int`**. It specifies the size of the returned array.

If the *init* argument is given, it must be **`bytes`**. It is used to initialize the array items. Bytes not initialized this way are set to zero (NUL).

If *size* is not given (or if it is `None`), the buffer is made one element larger than *init*, effectively adding a NUL terminator.

If both arguments are given, *size* must not be less than `len(init)`.

Aviso

If *size* is equal to `len(init)`, a NUL terminator is not added. Do not treat such a buffer as a C string.

Por exemplo:

```
>>> bytes(create_string_buffer(2))
b'\x00\x00'
>>> bytes(create_string_buffer(b'ab'))
b'ab\x00'
>>> bytes(create_string_buffer(b'ab', 2))
b'ab'
>>> bytes(create_string_buffer(b'ab', 4))
b'ab\x00\x00'
>>> bytes(create_string_buffer(b'abcdef', 2))
Traceback (most recent call last):
   ...
ValueError: byte string too long
```

Levanta um evento de auditoria `ctypes.create_string_buffer` com os argumentos `init`, `size`.

ctypes.create_unicode_buffer(*init*, *size=None*)
ctypes.create_unicode_buffer(*size*)

This function creates a mutable unicode character buffer. The returned object is a ctypes array of **`c_wchar`**.

The function takes the same arguments as **`create_string_buffer()`** except *init* must be a string and *size* counts **`c_wchar`**.

Levanta um evento de auditoria `ctypes.create_unicode_buffer` com os argumentos `init`, `size`.

ctypes.DllCanUnloadNow()

This function is a hook which allows implementing in-process COM servers with ctypes. It is called from the DllCanUnloadNow function that the _ctypes extension dll exports.

Disponibilidade: Windows

ctypes.DllGetClassObject()

This function is a hook which allows implementing in-process COM servers with ctypes. It is called from the DllGetClassObject function that the `_ctypes` extension dll exports.

Disponibilidade: Windows

ctypes.util.dllist()

Tenta fornecer uma lista de caminhos das bibliotecas partilhadas carregadas no processo atual. Estes caminhos não são normalizados nem processados de forma alguma. A função pode levantar **`OSError`** se as APIs da plataforma subjacente falharem. A funcionalidade exata depende do sistema.

Na maioria das plataformas, o primeiro elemento da lista corresponde ao arquivo executável atual. Pode ser uma string vazia.

Disponibilidade: Windows, macOS, iOS, glibc, BSD libc, musl

Adicionado na versão 3.14.

ctypes.FormatError([*code*])

Returns a textual description of the error code *code*. If no error code is specified, the last error code is used by calling the Windows API function **`GetLastError()`**.

Disponibilidade: Windows

ctypes.GetLastError()

Returns the last error code set by Windows in the calling thread. This function calls the Windows `GetLastError()` function directly, it does not return the ctypes-private copy of the error code.

Disponibilidade: Windows

ctypes.get_errno()

Returns the current value of the ctypes-private copy of the system **`errno`** variable in the calling thread.

Levanta um evento de auditoria `ctypes.get_errno` sem argumentos.

ctypes.get_last_error()

Returns the current value of the ctypes-private copy of the system **`LastError`** variable in the calling thread.

Disponibilidade: Windows

Levanta um evento de auditoria `ctypes.get_last_error` sem argumentos.

ctypes.memmove(*dst*, *src*, *count*)

Same as the standard C memmove library function: copies *count* bytes from *src* to *dst*. *dst* and *src* must be integers or ctypes instances that can be converted to pointers.

ctypes.memset(*dst*, *c*, *count*)

Same as the standard C memset library function: fills the memory block at address *dst* with *count* bytes of value *c*. *dst* must be an integer specifying an address, or a ctypes instance.

ctypes.POINTER(*type*, /)

Cria ou retorna um tipo de ponteiro ctypes. Os tipos de ponteiro são guardados em cache e reutilizados internamente, assim, chamar esta função repetidamente é eficiente. *type* deve ser um tipo ctypes.

O tipo de ponteiro resultante é armazenado em cache no atributo `__pointer_type__` de *type*. É possível definir este atributo antes da primeira chamada a `POINTER` para definir um tipo de ponteiro personalizado. No entanto, fazer isto é desencorajado: criar manualmente um tipo de ponteiro adequado é difícil sem depender de detalhes de implementação que podem mudar em versões futuras do Python.

ctypes.pointer(*obj*, /)

Cria uma nova instância de ponteiro, apontando para *obj*. O objeto devolvido é do tipo `POINTER(type(obj))`.

Nota: se você pretende apenas passar um ponteiro para um objeto em uma chamada de função externa, deve usar `byref(obj)`, que é muito mais rápido.

ctypes.resize(*obj*, *size*)

Esta função redimensiona o buffer de memória interno de *obj*, que deve ser uma instância de um tipo ctypes. Não é possível reduzir o buffer para menos do que o tamanho nativo do tipo do objeto, conforme dado por `sizeof(type(obj))`, mas é possível aumentá-lo.

ctypes.set_errno(*value*)

Define o valor atual da cópia privada do ctypes da variável do sistema **`errno`** na thread de chamada para *value* e devolve o valor anterior.

Levanta um evento de auditoria `ctypes.set_errno` com o argumento `errno`.

ctypes.set_last_error(*value*)

Define o valor atual da cópia privada do ctypes da variável do sistema **`LastError`** na thread de chamada para *value* e devolve o valor anterior.

Disponibilidade: Windows

Levanta um evento de auditoria `ctypes.set_last_error` com o argumento `error`.

ctypes.sizeof(*obj_or_type*)

Retornar o tamanho em bytes de um tipo ctypes ou buffer de memória de instância. Faz o mesmo que o operador C `sizeof`.

ctypes.string_at(*ptr*, *size=-1*)

Retorna a string de bytes em *void *ptr*. Se *size* for especificado, é usado como tamanho; caso contrário, assume-se que a string é terminada por zero.

Levanta um evento de auditoria `ctypes.string_at` com os argumentos `ptr` e `size`.

ctypes.WinError(*code=None*, *descr=None*)

Creates an instance of **OSError**. If *code* is not specified, **GetLastError()** is called to determine the error code. If *descr* is not specified, **FormatError()** is called to get a textual description of the error.

Disponibilidade: Windows

Alterado na versão 3.3: An instance of **WindowsError** used to be created, which is now an alias of **OSError**.

ctypes.wstring_at(*ptr*, *size=-1*)

Return the wide-character string at *void *ptr*. If *size* is specified, it is used as the number of characters of the string, otherwise the string is assumed to be zero-terminated.

Levanta um evento de auditoria `ctypes.wstring_at` com os argumentos `ptr`, `size`.

ctypes.memoryview_at(*ptr*, *size*, *readonly=False*)

Retorna um objeto **memoryview** de comprimento *size* que referencia a memória a partir de *void *ptr*.

Se *readonly* for verdadeiro, o objeto **memoryview** devolvido não pode ser usado para modificar a memória subjacente. (Alterações feitas por outros meios ainda serão refletidas no objeto devolvido).

Esta função é semelhante a **string_at()**, com a diferença fundamental de não fazer uma cópia da memória especificada. É uma alternativa semanticamente equivalente (mas mais eficiente) a `memoryview((c_byte * size).from_address(ptr))`. (Embora **from_address()** só aceite inteiros, *ptr* também pode ser fornecido como um objeto **ctypes.POINTER** ou um objeto **byref()**.)

Levanta um evento de auditoria `ctypes.memoryview_at` com os argumentos `address`, `size` e `readonly`.

Adicionado na versão 3.14.

Data types

class ctypes._CData

This non-public class is the common base class of all ctypes data types. Among other things, all ctypes type instances contain a memory block that hold C compatible data; the address of the memory block is returned by the `addressof()` helper function. Another instance variable is exposed as `_objects`; this contains other Python objects that need to be kept alive in case the memory block contains pointers.

Common methods of ctypes data types, these are all class methods (to be exact, they are methods of the metaclass):

from_buffer(*source*[, *offset*])

This method returns a ctypes instance that shares the buffer of the *source* object. The *source* object must support the writeable buffer interface. The optional *offset* parameter specifies an offset into the source buffer in bytes; the default is zero. If the source buffer is not large enough a `ValueError` is raised.

Levanta um evento de auditoria `ctypes.cdata/buffer` com os argumentos `pointer`, `size`, `offset`.

from_buffer_copy(*source*[, *offset*])

This method creates a ctypes instance, copying the buffer from the *source* object buffer which must be readable. The optional *offset* parameter specifies an offset into the source buffer in bytes; the default is zero. If the source buffer is not large enough a `ValueError` is raised.

Levanta um evento de auditoria `ctypes.cdata/buffer` com os argumentos `pointer`, `size`, `offset`.

from_address(*address*)

This method returns a ctypes type instance using the memory specified by *address* which must be an integer.

Este método, e outros que indiretamente chamam este método, levantam um evento de auditoria `ctypes.cdata` com o argumento `address`.

from_param(*obj*)

This method adapts *obj* to a ctypes type. It is called with the actual object used in a foreign function call when the type is present in the foreign function's **`argtypes`** tuple; it must return an object that can be used as a function call parameter.

All ctypes data types have a default implementation of this classmethod that normally returns *obj* if that is an instance of the type. Some types accept other objects as well.

in_dll(*library*, *name*)

This method returns a ctypes type instance exported by a shared library. *name* is the name of the symbol that exports the data, *library* is the loaded shared library.

Common class variables of ctypes data types:

__pointer_type__

O tipo de ponteiro que foi criado chamando **`POINTER()`** para o tipo de dados ctypes correspondente. Se um tipo de ponteiro ainda não foi criado, o atributo está ausente.

Adicionado na versão 3.14.

Variáveis de instância comuns dos tipos de dados ctypes:

_b_base_

Sometimes ctypes data instances do not own the memory block they contain, instead they share part of the memory block of a base

object. The `_b_base_` read-only member is the root ctypes object that owns the memory block.

_b_needsfree_

This read-only variable is true when the ctypes data instance has allocated the memory block itself, false otherwise.

_objects

This member is either `None` or a dictionary containing Python objects that need to be kept alive so that the memory block contents is kept valid. This object is only exposed for debugging; never modify the contents of this dictionary.

Tipos de dados fundamentais

class ctypes._SimpleCData

This non-public class is the base class of all fundamental ctypes data types. It is mentioned here because it contains the common attributes of the fundamental ctypes data types. **`_SimpleCData`** is a subclass of **`_CData`**, so it inherits their methods and attributes. ctypes data types that are not and do not contain pointers can now be pickled.

Instances have a single attribute:

value

This attribute contains the actual value of the instance. For integer and pointer types, it is an integer, for character types, it is a single character bytes object or string, for character pointer types it is a Python bytes object or string.

When the `value` attribute is retrieved from a ctypes instance, usually a new object is returned each time. **`ctypes`** does *not* implement original object return, always a new object is constructed. The same is true for all other ctypes object instances.

Each subclass has a class attribute:

type

Class attribute that contains an internal type code, as a single-character string. See Tipos de dados fundamentais for a summary.

Types marked * in the summary may be (or always are) aliases of a different **_SimpleCData** subclass, and will not necessarily use the listed type code. For example, if the platform's long, long long and time_t C types are the same, then **c_long**, **c_longlong** and **c_time_t** all refer to a single class, **c_long**, whose **_type_** code is `'l'`. The `'L'` code will be unused.

Ver também

The **array** and struct modules, as well as third-party modules like numpy [https://numpy.org/doc/stable/reference/arrays.interface.html#object.__array_interface__], use similar – but slightly different – type codes.

Fundamental data types, when returned as foreign function call results, or, for example, by retrieving structure field members or array items, are transparently converted to native Python types. In other words, if a foreign function has a **restype** of **c_char_p**, you will always receive a Python bytes object, *not* a **c_char_p** instance.

Subclasses of fundamental data types do *not* inherit this behavior. So, if a foreign functions **restype** is a subclass of **c_void_p**, you will receive an instance of this subclass from the function call. Of course, you can get the value of the pointer by accessing the `value` attribute.

These are the fundamental ctypes data types:

class ctypes.c_byte

Representa o tipo de dados C signed char e interpreta o valor como um inteiro pequeno. O construtor aceita um inicializador inteiro opcional; não há verificação de overflow.

class ctypes.c_char

Representa o tipo de dados C char e interpreta o valor como um único caractere. O construtor aceita um inicializador de string opcional; o comprimento da string deve ser exatamente um caractere.

class ctypes.c_char_p

Representa o tipo de dados C char* quando aponta para uma string terminada em zero. Para um ponteiro de caractere geral que também pode apontar para dados binários, deve-se usar `POINTER(c_char)`. O construtor aceita um endereço inteiro ou um objeto bytes.

class ctypes.c_double

Representa o tipo de dados C double. O construtor aceita um inicializador opcional de tipo float.

class ctypes.c_longdouble

Representa o tipo de dados C long double. O construtor aceita um inicializador opcional de tipo float. Em plataformas onde `sizeof(long double) == sizeof(double)`, é um alias para **`c_double`**.

class ctypes.c_float

Representa o tipo de dados C float. O construtor aceita um inicializador opcional de tipo float.

class ctypes.c_double_complex

Representa o tipo de dados C double complex, se disponível. O construtor aceita um inicializador opcional **`complex`**.

Adicionado na versão 3.14.

class ctypes.c_float_complex

Representa o tipo de dados C float complex, se disponível. O construtor aceita um inicializador opcional **`complex`**.

Adicionado na versão 3.14.

class ctypes.c_longdouble_complex

Representa o tipo de dados C long double complex, se disponível. O construtor aceita um inicializador opcional `complex`.

Adicionado na versão 3.14.

class ctypes.c_int

Represents the C signed int datatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where `sizeof(int) == sizeof(long)` it is an alias to `c_long`.

class ctypes.c_int8

Represents the C 8-bit signed int datatype. It is an alias for `c_byte`.

class ctypes.c_int16

Represents the C 16-bit signed int datatype. Usually an alias for `c_short`.

class ctypes.c_int32

Represents the C 32-bit signed int datatype. Usually an alias for `c_int`.

class ctypes.c_int64

Represents the C 64-bit signed int datatype. Usually an alias for `c_longlong`.

class ctypes.c_long

Represents the C signed long datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.c_longlong

Represents the C signed long long datatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where `sizeof(long long) == sizeof(long)` it is an alias to `c_long`.

class ctypes.c_short

Represents the C signed short datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.c_size_t

Represents the C **size_t** datatype. Usually an alias for another unsigned integer type.

class ctypes.c_ssize_t

Represents the **Py_ssize_t** datatype. This is a signed version of **size_t**; that is, the POSIX **ssize_t** type. Usually an alias for another integer type.

Adicionado na versão 3.2.

class ctypes.c_time_t

Represents the C **time_t** datatype. Usually an alias for another integer type.

Adicionado na versão 3.12.

class ctypes.c_ubyte

Represents the C unsigned char datatype, it interprets the value as small integer. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.c_uint

Represents the C unsigned int datatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where `sizeof(int) == sizeof(long)` it is an alias for **c_ulong**.

class ctypes.c_uint8

Represents the C 8-bit unsigned int datatype. It is an alias for **c_ubyte**.

class ctypes.c_uint16

Represents the C 16-bit unsigned int datatype. Usually an alias for **c_ushort**.

class ctypes.c_uint32

Represents the C 32-bit unsigned int datatype. Usually an alias for **c_uint**.

class ctypes.c_uint64

Represents the C 64-bit unsigned int datatype. Usually an alias for **c_ulonglong**.

class ctypes.c_ulong

Represents the C unsigned long datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.c_ulonglong

Represents the C unsigned long long datatype. The constructor accepts an optional integer initializer; no overflow checking is done. On platforms where `sizeof(long long) == sizeof(long)` it is an alias to **c_long**.

class ctypes.c_ushort

Represents the C unsigned short datatype. The constructor accepts an optional integer initializer; no overflow checking is done.

class ctypes.c_void_p

Represents the C void* type. The value is represented as integer. The constructor accepts an optional integer initializer.

class ctypes.c_wchar

Represents the C **wchar_t** datatype, and interprets the value as a single character unicode string. The constructor accepts an optional string initializer, the length of the string must be exactly one character.

class ctypes.c_wchar_p

Represents the C wchar_t* datatype, which must be a pointer to a zero-terminated wide character string. The constructor accepts an integer address, or a string.

class ctypes.c_bool

Represent the C bool datatype (more accurately, _Bool from C99). Its value can be `True` or `False`, and the constructor accepts any object that has a truth value.

class ctypes.HRESULT

Represents a **HRESULT** value, which contains success or error information for a function or method call.

Disponibilidade: Windows

class ctypes.py_object

Representa o tipo de dados C PyObject*. Chamar isto sem um argumento cria um ponteiro `NULL` PyObject*.

Alterado na versão 3.14: **py_object** é agora um tipo genérico.

The **ctypes.wintypes** module provides quite some other Windows specific data types, for example **HWND**, **WPARAM**, **VARIANT_BOOL** or **DWORD**. Some useful structures like **MSG** or **RECT** are also defined.

Tipos de dados estruturados

class ctypes.Union(**args*, ***kw*)

Classe base abstrata para uniões em ordem de bytes nativa.

Uniões partilham atributos e comportamentos comuns com estruturas; veja a documentação da **Structure** para mais detalhes.

class ctypes.BigEndianUnion(**args*, ***kw*)

Classe base abstrata para uniões em ordem de bytes *big endian*.

Adicionado na versão 3.11.

class ctypes.LittleEndianUnion(**args*, ***kw*)

Classe base abstrata para uniões em ordem de bytes *little endian*.

Adicionado na versão 3.11.

class ctypes.BigEndianStructure(**args*, ***kw*)

Classe base abstrata para estruturas em ordem de bytes *big endian*.

class ctypes.LittleEndianStructure(**args*, ***kw*)

Classe base abstrata para estruturas em ordem de bytes *little endian*.

Estruturas e uniões com ordem de bytes não nativa não podem conter campos de tipo ponteiro ou quaisquer outros tipos de dados que contenham campos de tipo ponteiro.

class ctypes.Structure(**args*, ***kw*)

Classe base abstrata para estruturas em ordem de bytes *nativa*.

Concrete structure and union types must be created by subclassing one of these types, and at least define a `_fields_` class variable. **`ctypes`** will create descriptors which allow reading and writing the fields by direct attribute accesses. These are the

fields

Uma sequência que define os campos da estrutura. Os itens devem ser tuplas com 2 ou 3 elementos. O primeiro item é o nome do campo; o segundo item especifica o tipo do campo, que pode ser qualquer tipo de dados ctypes.

Para campos de tipo inteiro como `c_int`, pode ser fornecido um terceiro item opcional. Deve ser um inteiro positivo pequeno que define a largura em bits do campo.

Os nomes dos campos devem ser únicos em uma estrutura ou união. Isto não é verificado, apenas um campo pode ser acessado quando os nomes são repetidos.

É possível definir a variável de classe `_fields_` *depois* da instrução de classe que define a subclasse de Structure, o que permite criar tipos de dados que referenciam diretamente ou indiretamente a si mesmos:

```
class List(Structure):
    pass
List._fields_ = [("pnext", POINTER(List)),
```

```
                    ...
                  ]
```

A variável de classe **_fields_** só pode ser definida uma vez. Atribuições posteriores levantarão um **AttributeError**.

Além disso, a variável de classe **_fields_** deve ser definida antes que o tipo de estrutura ou união seja usado pela primeira vez: uma instância ou subclasse é criada, **sizeof()** é chamada sobre ela, etc. Atribuições posteriores a **_fields_** levantarão um **AttributeError**. Se **_fields_** não tiver sido definida antes de tal uso, a estrutura ou união não terá campos próprios, como se **_fields_** estivesse vazia.

As sub-subclasses de tipos de estrutura herdam os campos da classe base, bem como os **_fields_** definidos na sub-subclasse, se existirem.

pack

An optional small integer that allows overriding the alignment of structure fields in the instance.

This is only implemented for the MSVC-compatible memory layout (see **_layout_**).

Setting **_pack_** to 0 is the same as not setting it at all. Otherwise, the value must be a positive power of two. The effect is equivalent to `#pragma pack(N)` in C, except **ctypes** may allow larger *n* than what the compiler accepts.

pack must already be defined when **_fields_** is assigned, otherwise it will have no effect.

Descontinuado desde a versão 3.14, será removido na versão 3.19: For historical reasons, if **_pack_** is non-zero, the MSVC-compatible layout will be used by default. On non-Windows platforms, this default is deprecated and is slated to become an error in Python 3.19. If it is intended, set **_layout_** to `'ms'` explicitly.

align

An optional small integer that allows increasing the alignment of the structure when being packed or unpacked to/from memory.

The value must not be negative. The effect is equivalent to `__attribute__((aligned(N)))` on GCC or `#pragma align(N)` on MSVC, except **`ctypes`** may allow values that the compiler would reject.

`_align_` can only *increase* a structure's alignment requirements. Setting it to 0 or 1 has no effect.

Using values that are not powers of two is discouraged and may lead to surprising behavior.

`_align_` must already be defined when **`_fields_`** is assigned, otherwise it will have no effect.

Adicionado na versão 3.13.

layout

Uma string opcional que nomeia o layout da estrutura/união. Atualmente, ele pode ser definido como:

- `"ms"`: o layout usado pelo compilador Microsoft (MSVC). No GCC e no Clang, este layout pode ser selecionado com `__attribute__((ms_struct))`.
- `"gcc-sysv"`: o layout usado pelo GCC com o modelo de dados System V ou "SysV-like", como usado no Linux e no macOS. Com este layout, **`_pack_`** não deve estar definido ou deve ser igual a zero.

Se não for definido explicitamente, `ctypes` usará um valor predefinido que corresponde às convenções da plataforma. Este valor predefinido pode mudar em futuras versões do Python (por exemplo, quando uma nova plataforma ganha suporte oficial ou quando é identificada uma diferença entre plataformas semelhantes). Atualmente, o valor predefinido será:

- No Windows: `"ms"`

- Quando **_pack_** está especificado: `"ms"`. (Isto está descontinuado; veja a documentação de **_pack_**.)
- Caso contrário: `"gcc-sysv"`

layout já deve estar definido quando **_fields_** é atribuído, caso contrário, não terá efeito.

Adicionado na versão 3.14.

anonymous

An optional sequence that lists the names of unnamed (anonymous) fields. **_anonymous_** must be already defined when **_fields_** is assigned, otherwise it will have no effect.

The fields listed in this variable must be structure or union type fields. **ctypes** will create descriptors in the structure type that allows accessing the nested fields directly, without the need to create the structure or union field.

Here is an example type (Windows):

```
class _U(Union):
    _fields_ = [("lptdesc", POINTER(TYPEDESC)),
                ("lpadesc", POINTER(ARRAYDESC)),
                ("hreftype", HREFTYPE)]

class TYPEDESC(Structure):
    _anonymous_ = ("u",)
    _fields_ = [("u", _U),
                ("vt", VARTYPE)]
```

The `TYPEDESC` structure describes a COM data type, the `vt` field specifies which one of the union fields is valid. Since the `u` field is defined as anonymous field, it is now possible to access the members directly off the TYPEDESC instance. `td.lptdesc` and `td.u.lptdesc` are equivalent, but the former is faster since it does not need to create a temporary union instance:

```
td = TYPEDESC()
td.vt = VT_PTR
```

```
td.lptdesc = POINTER(some_type)
td.u.lptdesc = POINTER(some_type)
```

It is possible to define sub-subclasses of structures, they inherit the fields of the base class. If the subclass definition has a separate `_fields_` variable, the fields specified in this are appended to the fields of the base class.

Structure and union constructors accept both positional and keyword arguments. Positional arguments are used to initialize member fields in the same order as they are appear in `_fields_`. Keyword arguments in the constructor are interpreted as attribute assignments, so they will initialize **_fields_** with the same name, or create new attributes for names not present in **_fields_**.

class ctypes.CField(**args*, ***kw*)

Descritor para campos de uma `Structure` ou `Union`. Por exemplo:

```
>>> class Color(Structure):
...     _fields_ = (
...         ('red', c_uint8),
...         ('green', c_uint8),
...         ('blue', c_uint8),
...         ('intense', c_bool, 1),
...         ('blinking', c_bool, 1),
...    )
...
>>> Color.red
<ctypes.CField 'red' type=c_ubyte, ofs=0, size=1>
>>> Color.green.type
<class 'ctypes.c_ubyte'>
>>> Color.blue.byte_offset
2
>>> Color.intense
<ctypes.CField 'intense' type=c_bool, ofs=3, bit_size=1, bit_
>>> Color.blinking.bit_offset
1
```

Todos os atributos são somente leitura.

Os objetos **`CField`** são criados por meio de **`_fields_`**; não instancie a classe diretamente.

Adicionado na versão 3.14: Anteriormente, os descritores tinham apenas os atributos `offset` e `size` e uma representação de string legível; a classe **`CField`** não estava disponível diretamente.

name

Nome do campo, como string.

type

Tipo do campo, como uma classe ctypes.

offset
byte_offset

Deslocamento do campo, em bytes.

Para campos de bits, este é o deslocamento da unidade de armazenamento alinhada por byte subjacente; veja **`bit_offset`**.

byte_size

Tamanho do campo, em bytes.

Para campos de bits, este é o tamanho da unidade de armazenamento subjacente. Normalmente, tem o mesmo tamanho que o tipo do campo de bits.

size

Para campos não-bit, equivalente a **`byte_size`**.

Para campos de bits, isto contém um valor compactado de bits compatível com versões anteriores que combina **`bit_size`** e **`bit_offset`**. Prefira usar os atributos explícitos em vez disso.

is_bitfield

Verdadeiro se este for um campo de bits.

bit_offset

bit_size

A localização de um campo de bits dentro da sua unidade de armazenamento, ou seja, dentro dos **`byte_size`** bytes de memória a partir de **`byte_offset`**.

Para obter o valor do campo, leia a unidade de armazenamento como um inteiro, desloque para a esquerda por **`bit_offset`** e acesse os **`bit_size`** bits menos significativos.

Para campos não-bit, **`bit_offset`** é zero e **`bit_size`** é igual a `byte_size * 8`.

is_anonymous

Verdadeiro se este campo for anônimo, ou seja, contém subcampos aninhados que devem ser fundidos em uma estrutura ou união contêiner.

Arrays and pointers

class ctypes.Array(**args*)

Abstract base class for arrays.

The recommended way to create concrete array types is by multiplying any **`ctypes`** data type with a non-negative integer. Alternatively, you can subclass this type and define **`_length_`** and **`_type_`** class variables. Array elements can be read and written using standard subscript and slice accesses; for slice reads, the resulting object is *not* itself an **`Array`**.

Arrays are generic over the type of their elements.

length

A positive integer specifying the number of elements in the array. Out-of-range subscripts result in an **`IndexError`**. Will be returned by **`len()`**.

type

Specifies the type of each element in the array.

Array subclass constructors accept positional arguments, used to initialize the elements in order.

ctypes.ARRAY(*type*, *length*)

Create an array. Equivalent to `type * length`, where *type* is a **`ctypes`** data type and *length* an integer.

Suavemente descontinuado desde a versão 3.14: In favor of multiplication.

class ctypes._Pointer

Private, abstract base class for pointers.

Concrete pointer types are created by calling **`POINTER()`** with the type that will be pointed to; this is done automatically by **`pointer()`**.

If a pointer points to an array, its elements can be read and written using standard subscript and slice accesses. Pointer objects have no size, so **`len()`** will raise **`TypeError`**. Negative subscripts will read from the memory *before* the pointer (as in C), and out-of-range subscripts will probably crash with an access violation (if you're lucky).

type

Specifies the type pointed to.

contents

Returns the object to which to pointer points. Assigning to this attribute changes the pointer to point to the assigned object.

Exceções

exception ctypes.ArgumentError

Esta exceção é levantada quando uma chamada de função externa não consegue converter um dos argumentos passados.

exception ctypes.COMError(*hresult*, *text*, *details*)

Esta exceção é levantada quando uma chamada de método COM falha.

hresult

O valor inteiro que representa o código de erro.

text

A mensagem de erro.

details

A tupla de 5 itens `(descr, source, helpfile, helpcontext, progid)`.

descr é a descrição textual. *source* é o `ProgID` dependente da linguagem para a classe ou aplicação que levantou o erro. *helpfile* é o caminho do arquivo de ajuda. *helpcontext* é o identificador de contexto de ajuda. *progid* é o `ProgID` da interface que definiu o erro.

Disponibilidade: Windows

Adicionado na versão 3.14.

Bibliotecas de interface de linha de comando

Os módulos descritos neste capítulo ajudam a implementar as interfaces de linha de comando e de terminal para aplicações.

Veja aqui uma visão geral:

- **`argparse`** — Analisador sintático para opções de linha de comando, argumentos e subcomandos
- **`optparse`** — Analisador sintático para opções de linha de comando
- **`getpass`** — Entrada de senha portátil
- **`fileinput`** — Itera sobre linhas de múltiplos fluxos de entrada
- **`curses`** — Gerenciador de terminal para visualizadores de células de caracteres.
- **`curses.textpad`** — Text input widget for curses programs
- **`curses.ascii`** — Utilitários para caracteres ASCII
- **`curses.panel`** — Uma extensão de pilha de painéis para o curses
- **`cmd`** — Suporte para interpretadores de comando orientado a linhas

`argparse` — Analisador sintático para opções de linha de comando, argumentos e subcomandos

Adicionado na versão 3.2.

Código-fonte: Lib/argparse.py
[https://github.com/python/cpython/tree/3.15/Lib/argparse.py]

> **Nota**
>
> Embora o **`argparse`** seja o módulo de biblioteca padrão recomendado para implementar aplicações de linha de comando básicas, autores com requisitos mais rigorosos sobre como exatamente suas aplicações de linha de comando se comportam podem descobrir que ele não fornece o nível de controle necessário. Consulte Escolhendo uma biblioteca de análise de linha de comando para alternativas a considerar quando `argparse` não oferece suporte a comportamentos que a aplicação requer (como desabilitar completamente o suporte a opções intercaladas e argumentos posicionais, ou aceitar valores de parâmetros de opção que começam com `-` mesmo quando correspondem a outra opção definida).

O módulo **`argparse`** torna fácil a escrita de interfaces de linha de comando amigáveis. O programa define quais argumentos são necessários e **`argparse`** descobrirá como analisá-lo e interpretá-los a partir do **`sys.argv`**. O módulo **`argparse`** também gera automaticamente o texto ajuda e mensagens de uso. O módulo também vai emitir erros quando

> **Tutorial**
>
> Esta página contém informações da API de Referência. Para uma introdução mais prática para o parser de linha de comando

o usuário prover argumentos inválidos para o programa.

Python, acesse o tutorial do argparse.

O suporte do módulo **argparse** para interfaces de linha de comando é construído em torno de uma instância de **argparse.ArgumentParser**. É um contêiner para especificações de argumentos e possui opções que se aplicam ao analisador sintático como um todo:

```
parser = argparse.ArgumentParser(
                     prog='NomePrograma',
                     description='O que o programa faz',
                     epilog='Texto na parte inferior da memsagem
```

O método **ArgumentParser.add_argument()** anexa especificações de argumentos individuais ao analisador. Ele oferece suporte a argumentos posicionais, opções que aceitam valores e sinalizadores de ligar/desligar:

```
parser.add_argument('filename')           # argumento posicional
parser.add_argument('-c', '--count')      # a opção recebe um va
parser.add_argument('-v', '--verbose',
                     action='store_true')  # sinalizador de ligar
```

O método **ArgumentParser.parse_args()** executa o analisador e coloca os dados extraídos em um objeto **argparse.Namespace**:

```
args = parser.parse_args()
print(args.filename, args.count, args.verbose)
```

Nota

Se você estiver procurando um guia sobre como atualizar um código **optparse** para **argparse**, veja Atualizando código optparse.

Objetos ArgumentParser

class argparse.ArgumentParser(*prog=None, usage=None, description=None, epilog=None, parents=[], formatter_class=argparse.HelpFormatter, prefix_chars='-', fromfile_prefix_chars=None, argument_default=None, conflict_handler='error', add_help=True, allow_abbrev=True, exit_on_error=True, *, suggest_on_error=True, color=True*)

Cria um novo objeto **`ArgumentParser`**. Todos os parâmetros devem ser passados como argumentos nomeados. Cada parâmetro tem sua própria descrição mais detalhada abaixo, mas em resumo eles são:

- prog - O nome do programa (padrão: gerado a partir de atributos de módulo `__main__` e de `sys.argv[0]`)
- usage - A string que descreve o uso do programa (padrão: gerado a partir de argumentos adicionados ao analisador sintático)
- description - Texto para exibir antes da ajuda dos argumentos (por padrão, nenhum texto)
- epilog - Texto para exibir após da ajuda dos argumentos (por padrão, nenhum texto)
- parents - Uma lista de objetos **`ArgumentParser`** cujos argumentos também devem ser incluídos
- formatter_class - Uma classe para personalizar a saída de ajuda
- prefix_chars - O conjunto de caracteres que prefixam argumentos opcionais (padrão: “-“)
- fromfile_prefix_chars - O conjunto de caracteres que prefixam os arquivos dos quais os argumentos adicionais devem ser lidos (padrão: `None`)
- argument_default - O valor padrão global para argumentos (padrão: `None`)
- conflict_handler - A estratégia para resolver opcionais conflitantes (geralmente desnecessário)
- add_help - Adiciona uma opção `-h/--help` para o analisador sintático (padrão: `True`)
- allow_abbrev - Permite que opções longas sejam abreviadas se a abreviação não for ambígua. (padrão: `True`)
- exit_on_error - Determina se **`ArgumentParser`** sai ou não com informações de erro quando ocorre um erro. (padrão: `True`)

- suggest_on_error - Habilita sugestões para escolhas de argumentos e nomes de subanalisadores digitados incorretamente (padrão: `True`)
- color - Permite saída colorida (padrão: `True`)

Alterado na versão 3.5: O parâmetro *allow_abbrev* foi adicionado.

Alterado na versão 3.8: Em versões anteriores, *allow_abbrev* também desabilitava o agrupamento de sinalizadores curtos, como `-vv` para significar `-v -v`.

Alterado na versão 3.9: O parâmetro *exit_on_error* foi adicionado.

Alterado na versão 3.14: Os parâmetros *suggest_on_error* e *color* foram adicionados.

Alterado na versão 3.15: suggest_on_error teve seu padrão alterado para `True`.

As seções a seguir descrevem como cada um deles é usado.

prog

Por padrão, **`ArgumentParser`** calcula o nome do programa a ser exibido nas mensagens de ajuda dependendo da maneira como o interpretador Python foi executado:

- O **`nome_base`** de `sys.argv[0]` se um arquivo foi passado como argumento.
- O nome do interpretador Python seguido por `sys.argv[0]` se um diretório ou um arquivo zip foi passado como argumento.
- O nome do interpretador Python seguido por `-m` seguido pelo nome do módulo ou pacote se a opção **`-m`** foi usada.

Este padrão é quase sempre desejável porque fará com que as mensagens de ajuda correspondam à string que foi usada para invocar o programa na linha de comando. No entanto, para alterar esse comportamento padrão, outro valor pode ser fornecido usando o argumento `prog=` para **`ArgumentParser`**:

```
>>> parser = argparse.ArgumentParser(prog='meuprograma')
>>> parser.print_help()
usage: meuprograma [-h]

options:
 -h, --help  show this help message and exit
```

Observe que o nome do programa, seja determinado a partir de `sys.argv[0]`, a partir de atributos de módulo `__main__` ou do argumento `prog=`, está disponível para mensagens de ajuda usando o especificador de formato `%(prog)s`.

```
>>> parser = argparse.ArgumentParser(prog='meuprograma')>>> pars
```

Alterado na versão 3.14: O valor padrão `prog` agora reflete como `__main__` foi realmente executado, em vez de ser sempre `os.path.basename(sys.argv[0])`.

usage

Por padrão, **`ArgumentParser`** calcula a mensagem de uso a partir dos argumentos que contém. A mensagem padrão pode ser substituída pelo argumento nomeado `usage=`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', usage='%(prog)
>>> parser.add_argument('--foo', nargs='?', help='foo help')
>>> parser.add_argument('bar', nargs='+', help='bar help')
>>> parser.print_help()
usage: PROG [options]

positional arguments:
 bar           bar help

options:
 -h, --help   show this help message and exit
 --foo [FOO]  foo help
```

O especificador de formato `%(prog)s` está disponível para preencher o nome do programa em suas mensagens de uso.

Quando uma mensagem de uso personalizada é especificada para o analisador sintático principal, você também pode considerar passar o argumento `prog` para **`add_subparsers()`** ou os argumentos `prog` e `usage` para **`add_parser()`**, para garantir prefixos de comando e informações de uso consistentes entre os subanalisadores.

description

A maioria das chamadas para o construtor **`ArgumentParser`** usará o argumento nomeado `description=`. Este argumento fornece uma breve descrição do que o programa faz e como funciona. Nas mensagens de ajuda, a descrição é exibida entre a string de uso da linha de comando e as mensagens de ajuda para os vários argumentos.

Por padrão, a descrição terá sua linha quebrada para que se encaixe no espaço fornecido. Para alterar esse comportamento, consulte o argumento formatter_class.

epilog

Alguns programas gostam de exibir uma descrição adicional do programa após a descrição dos argumentos. Esse texto pode ser especificado usando o argumento `epilog=` para **`ArgumentParser`**:

```
>>> parser = argparse.ArgumentParser(
...     description='Um foo que faz bars',
...     epilog="E é assim que você faria foo de bar")
>>> parser.print_help()
usage: argparse.py [-h]

Um foo que faz bars

options:
 -h, --help  show this help message and exit

E é assim que você faria foo de bar
```

Tal como acontece com o argumento description, o texto de `epilog=` tem sua quebra de linha habilitada por padrão, mas este comportamento pode

ser ajustado com o argumento formatter_class para **ArgumentParser**.

parents

Às vezes, vários analisadores sintáticos compartilham um conjunto comum de argumentos. Ao invés de repetir as definições desses argumentos, um único analisador com todos os argumentos compartilhados e passado para o argumento `parents=` para **ArgumentParser** pode ser usado. O argumento `parents=` pega uma lista de objetos **ArgumentParser**, coleta todas as ações posicionais e opcionais deles, e adiciona essas ações ao objeto **ArgumentParser** sendo construído:

```
>>> parent_parser = argparse.ArgumentParser(add_help=False)>>> p
```

Observe que a maioria dos analisadores sintáticos pais especificará `add_help=False`. Caso contrário, o **ArgumentParser** verá duas opções `-h/--help` (uma no pai e outra no filho) e levantará um erro.

> **Nota**
>
> Você deve inicializar totalmente os analisadores sintáticos antes de passá-los via `parents=`. Se você alterar os analisadores pais após o analisador filho, essas mudanças não serão refletidas no filho.

formatter_class

Objetos **ArgumentParser** permitem que a formação do texto de ajuda seja personalizada por meio da especificação de uma classe de formatação alternativa. Atualmente, há quatro dessas classes:

class argparse.RawDescriptionHelpFormatter
class argparse.RawTextHelpFormatter
class argparse.ArgumentDefaultsHelpFormatter
class argparse.MetavarTypeHelpFormatter

RawDescriptionHelpFormatter e **RawTextHelpFormatter** dão mais controle sobre como as descrições textuais são exibidas. Por padrão, objetos **ArgumentParser** quebram em linha os textos description e epilog nas mensagens de ajuda da linha de comando:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     description='''esta descrição
...         foi indentada de forma estranha,
...             mas está tudo bem''',
...     epilog='''
...             da mesma forma para este epílogo, cujos espaços
...         ser apagados e cujas palavras serão quebradas
...         em algumas linhas''')
>>> parser.print_help()
usage: PROG [-h]

esta descrição foi indentada de forma estranha, mas está tudo be

options:
 -h, --help  show this help message and exit

da mesma forma para este epílogo, cujos espaços vão ser apagados
cujas palavras serão quebradas em algumas linhas
```

Passar **RawDescriptionHelpFormatter** como `formatter_class=` indica que description e epilog já estão formatados corretamente e não devem ter suas linhas quebradas:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.RawDescriptionHelpFormatter,
...     description=textwrap.dedent('''\
...         Por favor, não bagunce este texto!
...         --------------------------------
...             Eu indentei o texto
...             exatamente na forma
...             que quero que fique
...         '''))
>>> parser.print_help()
usage: PROG [-h]
```

```
Por favor, não bagunce este texto!
--------------------------------
   Eu indentei o texto
   exatamente na forma
   que quero que fique

options:
 -h, --help  show this help message and exit
```

RawTextHelpFormatter mantém espaços em branco para todos os tipos de texto de ajuda, incluindo descrições de argumentos. No entanto, várias novas linhas são substituídas por uma. Se você deseja preservar várias linhas em branco, adicione espaços entre as novas linhas.

ArgumentDefaultsHelpFormatter adiciona automaticamente informações sobre os valores padrão para cada uma das mensagens de ajuda do argumento:

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
...     formatter_class=argparse.ArgumentDefaultsHelpFormatter)
>>> parser.add_argument('--foo', type=int, default=42, help='FOO
>>> parser.add_argument('bar', nargs='*', default=[1, 2, 3], hel
>>> parser.print_help()
usage: PROG [-h] [--foo FOO] [bar ...]

positional arguments:
 bar         BAR! (default: [1, 2, 3])

options:
 -h, --help  show this help message and exit
 --foo FOO   FOO! (default: 42)
```

MetavarTypeHelpFormatter usa o nome de argumento type para cada argumento como o nome de exibição para seus valores (em vez de usar o dest como o formatador regular faz):

```
>>> parser = argparse.ArgumentParser(
...     prog='PROG',
```

```
...     formatter_class=argparse.MetavarTypeHelpFormatter)
>>> parser.add_argument('--foo', type=int)
>>> parser.add_argument('bar', type=float)
>>> parser.print_help()
usage: PROG [-h] [--foo int] float

positional arguments:
  float

options:
  -h, --help  show this help message and exit
  --foo int
```

prefix_chars

A maioria das opções de linha de comando usará `-` como prefixo, por exemplo, `-f/--foo`. Analisadores sintáticos que precisam ter suporte a caracteres de prefixo diferentes ou adicionais, por exemplo, para opções como `+f` ou `/foo`, podem especificá-las usando o argumento `prefix_chars=` para o construtor **`ArgumentParser`**:

```
>>> parser = argparse.ArgumentParser(prog='PROG', prefix_chars='
>>> parser.add_argument('+f')
>>> parser.add_argument('++bar')
>>> parser.parse_args('+f X ++bar Y'.split())
Namespace(bar='Y', f='X')
```

O argumento `prefix_chars=` é padronizado como `'-'`. Fornecer um conjunto de caracteres que não inclua `-` fará com que as opções `-f/--foo` não sejam permitidas.

fromfile_prefix_chars

Às vezes ao lidar com uma lista de argumentos particularmente longa, pode fazer sentido manter a lista de argumentos em um arquivo em vez de digitá-la na linha de comando. Se o argumento `fromfile_prefix_chars=` for dado ao construtor **`ArgumentParser`**, então os argumentos que começam com qualquer um dos caracteres especificados serão tratados como arquivos e serão substituídos pelos argumentos que eles contêm. Por exemplo:

```
>>> with open('args.txt', 'w', encoding=sys.getfilesystemencodir
...     fp.write('-f\nbar')
...
>>> parser = argparse.ArgumentParser(fromfile_prefix_chars='@')
>>> parser.add_argument('-f')
>>> parser.parse_args(['-f', 'foo', '@args.txt'])
Namespace(f='bar')
```

Por padrão, os argumentos lidos de um arquivo devem ser um por linha (mas veja também `convert_arg_line_to_args()`) e são tratados como se estivessem no mesmo lugar que o argumento original que faz referência ao arquivo na linha de comando. Portanto, no exemplo acima, a expressão `['-f', 'foo', '@args.txt']` é considerada equivalente à expressão `['-f', 'foo', '-f', 'bar']`.

Nota

Cada linha é tratada como um único argumento; portanto, uma linha vazia é lida como uma string vazia (`''`).

`ArgumentParser` usa tratador de erros e codificação do sistema de arquivos para ler o arquivo que contém argumentos.

O argumento `fromfile_prefix_chars=` é padronizado como `None`, significando que os argumentos nunca serão tratados como referências de arquivo.

Alterado na versão 3.12: `ArgumentParser` alterou a codificação e os erros para ler arquivos de argumentos do padrão (por exemplo, `locale.getpreferredencoding(False)` e `"strict"`) para tratador de erros e codificação do sistema de arquivos. O arquivo de argumentos deve ser codificado em UTF-8 em vez de página de código ANSI no Windows.

argument_default

Geralmente, os padrões dos argumentos são especificados passando um padrão para `add_argument()` ou chamando os métodos `set_defaults()`

com um conjunto específico de pares nome-valor. Às vezes, no entanto, pode ser útil especificar um único padrão para argumentos, válido em todo o analisador. Isso pode ser feito passando o argumento nomeado `argument_default=` para **`ArgumentParser`**. Por exemplo, para suprimir globalmente a criação de atributos em chamadas **`parse_args()`**, fornecemos `argument_default=SUPPRESS`:

```
>>> parser = argparse.ArgumentParser(argument_default=argparse.S
>>> parser.add_argument('--foo')
>>> parser.add_argument('bar', nargs='?')
>>> parser.parse_args(['--foo', '1', 'BAR'])
Namespace(bar='BAR', foo='1')
>>> parser.parse_args([])
Namespace()
```

allow_abbrev

Normalmente, quando você passa uma lista de argumentos para o método **`parse_args()`** de um **`ArgumentParser`**, ele reconhece as abreviações de opções longas.

Este recurso pode ser desabilitado configurando `allow_abbrev` para `False`:

```
>>> parser = argparse.ArgumentParser(prog='PROG', allow_abbrev=F
>>> parser.add_argument('--foobar', action='store_true')
>>> parser.add_argument('--foonley', action='store_false')
>>> parser.parse_args(['--foon'])
usage: PROG [-h] [--foobar] [--foonley]
PROG: error: unrecognized arguments: --foon
```

Adicionado na versão 3.5.

conflict_handler

Objetos **`ArgumentParser`** não permitem duas ações com a mesma string de opções. Por padrão, objetos **`ArgumentParser`** levantam uma exceção se for feita uma tentativa de criar um argumento com uma string de opção que já esteja em uso:

```
>>> parser = argparse.ArgumentParser(prog='PROG')>>> parser.add_
```

Às vezes (por exemplo, ao usar os parents) pode ser útil simplesmente substituir quaisquer argumentos mais antigos com a mesma string de opções. Para obter este comportamento, o valor `'resolve'` pode ser fornecido ao argumento `conflict_handler=` de **`ArgumentParser`**:

```
>>> parser = argparse.ArgumentParser(prog='PROG', conflict_handl
```

Observe que os objetos **`ArgumentParser`** só removem uma ação se todas as suas strings de opção forem substituídas. Assim, no exemplo acima, a antiga ação `-f/--foo` é mantida como `-f`, pois apenas a string de opção `--foo` foi substituída.

add_help

Por padrão, os objetos **`ArgumentParser`** adicionam uma opção que simplesmente exibe a mensagem de ajuda do analisador sintático. Se `-h` ou `--help` for fornecido na linha de comando, a ajuda do **`ArgumentParser`** será exibida.

Às vezes, pode ser útil desabilitar o acréscimo desta opção de ajuda. Isto pode ser feito passando `False` como o argumento `add_help=` para a classe **`ArgumentParser`**:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=False
>>> parser.add_argument('--foo', help='ajuda de foo')
>>> parser.print_help()
usage: PROG [--foo FOO]

options:
 --foo FOO  ajuda de foo
```

A opção de ajuda é normalmente `-h/--help`. A exceção a isso é se o `prefix_chars=` for especificado e não incluir `-`, neste caso `-h` e `--help` não são opções válidas. Neste caso, o primeiro caractere em `prefix_chars` é usado para prefixar as opções de ajuda:

```
>>> parser = argparse.ArgumentParser(prog='PROG', prefix_chars='
>>> parser.print_help()
usage: PROG [+h]

options:
  +h, ++help  show this help message and exit
```

exit_on_error

Normalmente, quando você passa uma lista de argumentos inválidos para o método `parse_args()` de uma instância de `ArgumentParser`, ele vai exibir uma *message* para `sys.stderr` e vai sair com o código de status de 2.

Se o usuário quiser detectar erros manualmente, o recurso pode ser habilitado configurando `exit_on_error` para `False`:

```
>>> parser = argparse.ArgumentParser(exit_on_error=False)
>>> parser.add_argument('--integers', type=int)
_StoreAction(option_strings=['--integers'], dest='integers', nar
>>> try:
...     parser.parse_args('--integers a'.split())
... except argparse.ArgumentError:
...     print('Capturando um argumentError')
...
Capturando um argumentError
```

Adicionado na versão 3.9.

suggest_on_error

Por padrão, quando um usuário fornece uma opção de argumento inválida ou um nome de subparser inválido, a `ArgumentParser` encerra a execução exibindo informações de erro e sugestões para argumentos digitados incorretamente. A mensagem de erro listará as opções de argumento permitidas (se especificadas) ou os nomes de subparsers, juntamente com uma sugestão do tipo “talvez você quisesse dizer” caso seja encontrada uma correspondência próxima. Observe que isso se aplica apenas a argumentos cujas opções especificadas sejam strings:

```
>>> parser = argparse.ArgumentParser(suggest_on_error=True)
>>> parser.add_argument('--action', choices=['debug', 'dryrun'])
>>> parser.parse_args(['--action', 'debugg'])
usage: tester.py [-h] [--action {debug,dryrun}]
tester.py: error: argument --action: invalid choice: 'debugg', m
```

Você pode desativar as sugestões definindo `suggest_on_error` como `False`.

Adicionado na versão 3.14.

Alterado na versão 3.15: O valor padrão de `suggest_on_error` foi alterado de `False` para `True`.

color

Por padrão, a mensagem de ajuda é impressa em cores usando sequências de escape ANSI [https://en.wikipedia.org/wiki/ANSI_escape_code]. Se desejar mensagens de ajuda em texto simples, você pode desabilitar esta no seu ambiente local ou no próprio analisador de argumentos, definindo `color` como `False`:

```
>>> parser = argparse.ArgumentParser(description='Process some i
...                                  color=False)
>>> parser.add_argument('--action', choices=['sum', 'max'])
>>> parser.add_argument('integers', metavar='N', type=int, nargs
...                     help='an integer for the accumulator')
>>> parser.parse_args(['--help'])
```

Note que, quando `color=True`, a saída colorida depende tanto das variáveis de ambiente quanto das capacidades do terminal. No entanto, se `color=False`, a saída colorida é sempre desativada, mesmo que variáveis de ambiente como `FORCE_COLOR` estejam definidas.

Adicionado na versão 3.14.

Para destacar código em linha na sua descrição, no epílogo ou no texto de `help` de um argumento, você pode usar crases simples ou duplas:

```
>>> parser = argparse.ArgumentParser(
...     formatter_class=argparse.RawDescriptionHelpFormatter,
...     description='Execute ``python -m myapp`` para iniciar.',
...     epilog='''Exemplos:
...   `python -m myapp --verbose`
...   ``python -m myapp --config settings.json``
... ''')
>>> parser.add_argument('--foo', help='define o valor de `foo`')
```

Quando as cores estão ativadas, o texto entre crases é exibido em uma cor distinta para destacar os exemplos. Quando as cores estão desativadas, as crases são preservadas como estão, o que permite a leitura em texto simples.

Adicionado na versão 3.15.

O método add_argument()

ArgumentParser.add_argument(*name or flags...*, *[, *action*][, *nargs*][, *const*][, *default*][, *type*][, *choices*][, *required*][, *help*][, *metavar*][, *dest*][, *deprecated*])

Define como um único argumento de linha de comando deve ser analisado. Cada parâmetro tem sua própria descrição mais detalhada abaixo, mas resumidamente são eles:

- name ou flags - Um nome ou uma lista de strings de opções, por exemplo. `'foo'` ou `'-f', '--foo'`.
- action - O tipo básico de ação a ser executada quando esse argumento é encontrado na linha de comando.
- nargs - O número de argumentos de linha de comando que devem ser consumidos.
- const - Um valor constante exigido por algumas seleções action e nargs.
- default - O valor produzido se o argumento estiver ausente da linha de comando e se estiver ausente do objeto espaço de nomes.

- type - O tipo para o qual o argumento de linha de comando deve ser convertido.
- choices - Uma sequência de valores permitidos para o argumento.
- required - Se a opção de linha de comando pode ser omitida ou não (somente opcionais).
- help - Uma breve descrição do que o argumento faz.
- metavar - Um nome para o argumento nas mensagens de uso.
- dest - O nome do atributo a ser adicionado ao objeto retornado por **`parse_args()`**.
- deprecated - Se o uso do argumento foi descontinuado ou não.

O método retorna um objeto **`Action`** que representa o argumento.

As seções a seguir descrevem como cada um deles é usado.

name ou flags

O método **`add_argument()`** deve saber se um argumento opcional, como `-f` ou `--foo`, ou um argumento posicional, como uma lista de nomes de arquivos, é esperado. Os primeiros argumentos passados para **`add_argument()`** devem, portanto, ser uma série de sinalizadores ou um único nome de argumento.

Por exemplo, um argumento opcional poderia ser criado como:

```
>>> parser.add_argument('-f', '--foo')
```

enquanto um argumento posicional pode ser criado como:

```
>>> parser.add_argument('bar')
```

Quando **`parse_args()`** é chamado, argumentos opcionais serão identificados pelo prefixo `-`, e os argumentos restantes serão considerados posicionais:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-f', '--foo')
>>> parser.add_argument('bar')
>>> parser.parse_args(['BAR'])
```

```
Namespace(bar='BAR', foo=None)
>>> parser.parse_args(['BAR', '--foo', 'FOO'])
Namespace(bar='BAR', foo='FOO')
>>> parser.parse_args(['--foo', 'FOO'])
usage: PROG [-h] [-f FOO] bar
PROG: error: the following arguments are required: bar
```

Por padrão, **`argparse`** lida automaticamente com a nomenclatura interna e os nomes de exibição dos argumentos, simplificando o processo sem exigir configuração adicional. Dessa forma, você não precisa especificar os parâmetros dest e metavar. Para argumentos opcionais, o parâmetro dest assume por padrão o nome do argumento com sublinhados _ substituindo os hífens `-`. O parâmetro metavar assume por padrão o nome em letras maiúsculas. Por exemplo:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo-bar')
>>> parser.parse_args(['--foo-bar', 'FOO-BAR'])
Namespace(foo_bar='FOO-BAR')
>>> parser.print_help()
usage:  [-h] [--foo-bar FOO-BAR]

optional arguments:
 -h, --help  show this help message and exit
 --foo-bar FOO-BAR
```

action

Objetos **`ArgumentParser`** associam argumentos de linha de comando com ações. Essas ações podem fazer praticamente qualquer coisa com os argumentos de linha de comando associados a elas, embora a maioria das ações simplesmente adicione um atributo ao objeto retornado por **`parse_args()`**. O argumento nomeado `action` especifica como os argumentos da linha de comando devem ser tratados. As ações fornecidas são:

- `'store'` - Isso apenas armazena o valor do argumento. Esta é a ação padrão.

- `'store_const'` - Isso armazena o valor especificado pelo argumento nomeado const; observe que o argumento nomeado const tem como padrão `None`. A ação `'store_const'` é mais comumente usada com argumentos opcionais que especificam algum tipo de sinalizador. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_const', const
>>> parser.parse_args(['--foo'])
Namespace(foo=42)
```

- `'store_true'` e `'store_false'` - Estes são casos especiais de `'store_const'` que armazenam respectivamente os valores `True` e `False` e com valores padrão de `False` e `True`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_true')
>>> parser.add_argument('--bar', action='store_false')
>>> parser.add_argument('--baz', action='store_false')
>>> parser.parse_args('--foo --bar'.split())
Namespace(foo=True, bar=False, baz=True)
```

- `'append'` - Isso anexa cada valor de argumento a uma lista. É útil permitir para permitir que uma opção seja especificada várias vezes. Se o valor padrão for uma lista não-vazia, o valor analisado vai iniciar com os elementos padrão da lista e quaisquer valores da linha de comando serão anexados após os valores padrão. Exemplo de uso:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='append', default=[
>>> parser.parse_args('--foo 1 --foo 2'.split())
Namespace(foo=['0', '1', '2'])
```

- `'append_const'` - Isso anexa o valor especificado pelo argumento nomeado const a uma lista; observe que o argumento nomeado const tem como padrão `None`. A ação `'append_const'` é normalmente útil quando vários argumentos precisam armazenar constantes na mesma lista. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--str', dest='types', action='appe
>>> parser.add_argument('--int', dest='types', action='appe
>>> parser.parse_args('--str --int'.split())
Namespace(types=[<class 'str'>, <class 'int'>])
```

- `'extend'` - Isso anexa cada item de um argumento multivalorado a uma lista. A ação `'extend'` é normalmente usada com o valor do argumento nomeado nargs `'+'` ou `'*'`. Observe que quando nargs é `None` (o padrão) ou `'?'`, cada caractere da string do argumento será anexado à lista. Exemplo de uso:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument("--foo", action="extend", nargs="+",
>>> parser.parse_args(["--foo", "f1", "--foo", "f2", "f3", '
Namespace(foo=['f1', 'f2', 'f3', 'f4'])
```

Adicionado na versão 3.8.

- `'count'` - Isso conta o número de vezes que um argumento ocorre. Por exemplo, isso é útil para aumentar os níveis de verbosidade:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--verbose', '-v', action='count', d
>>> parser.parse_args(['-vvv'])
Namespace(verbose=3)
```

A menos que seja definido explicitamente, o valor padrão será `None`. Se o valor padrão for um número diferente de zero, a contagem começa a partir desse número, em vez de zero.

- `'help'` - Isso imprime uma mensagem de ajuda completa para todas as opções no analisador sintático atual e sai. Por padrão, uma ação de ajuda é adicionada automaticamente ao analisador sintático. Veja **`ArgumentParser`** para detalhes de como a saída é criada.

- `'version'` - Isso espera um argumento nomeado `version=` na chamada **`add_argument()`** e imprime informações de versão e sai

quando invocado:

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--version', action='version', vers
>>> parser.parse_args(['--version'])
PROG 2.0
```

Você também pode especificar uma ação arbitrária passando uma subclasse **`Action`** (por exemplo, **`BooleanOptionalAction`**) ou outro objeto que implemente a mesma interface. Somente ações que consomem argumentos de linha de comando (por exemplo, `'store'`, `'append'`, `'extend'` ou ações personalizadas com `nargs` diferentes de zero) podem ser usadas com argumentos posicionais.

A maneira recomendada de criar uma ação personalizada é estender **`Action`**, substituindo o método **`__call__()`** e opcionalmente os métodos **`__init__()`** e **`format_usage()`**. Você também pode registrar ações personalizadas usando o método **`register()`** e referenciá-las pelo nome registrado.

Um exemplo de uma ação personalizada:

```
>>> class FooAction(argparse.Action):
...     def __init__(self, option_strings, dest, nargs=None, **k
...         if nargs is not None:
...             raise ValueError("nargs not allowed")
...         super().__init__(option_strings, dest, **kwargs)
...     def __call__(self, parser, namespace, values, option_str
...         print('%r %r %r' % (namespace, values, option_string
...         setattr(namespace, self.dest, values)
...
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action=FooAction)
>>> parser.add_argument('bar', action=FooAction)
>>> args = parser.parse_args('1 --foo 2'.split())
Namespace(bar=None, foo=None) '1' None
Namespace(bar='1', foo=None) '2' '--foo'
>>> args
Namespace(bar='1', foo='2')
```

Para mais detalhes, veja `Action`.

nargs

Os objetos `ArgumentParser` geralmente associam um único argumento de linha de comando a uma única ação a ser executada. O argumento nomeado `nargs` associa um número diferente de argumentos de linha de comando com uma única ação. Veja também Especificando argumentos ambíguos. Os valores suportados são:

- `N` (um inteiro). Os argumentos `N` da linha de comando serão reunidos em uma lista. Por exemplo:

  ```
  >>> parser = argparse.ArgumentParser()
  >>> parser.add_argument('--foo', nargs=2)
  >>> parser.add_argument('bar', nargs=1)
  >>> parser.parse_args('c --foo a b'.split())
  Namespace(bar=['c'], foo=['a', 'b'])
  ```

  Observe que `nargs=1` produz uma lista de um item. Isso é diferente do padrão, em que o item é produzido sozinho.

- `'?'`. Um argumento será consumido da linha de comando, se possível, e produzido como um único item. Se nenhum argumento de linha de comando estiver presente, o valor de default será produzido. Observe que, para argumentos opcionais, há um caso adicional - a string de opções está presente, mas não é seguida por um argumento de linha de comando. Neste caso o valor de const será produzido. Alguns exemplos para ilustrar isso:

  ```
  >>> parser = argparse.ArgumentParser()
  >>> parser.add_argument('--foo', nargs='?', const='c', defau
  >>> parser.add_argument('bar', nargs='?', default='d')
  >>> parser.parse_args(['XX', '--foo', 'YY'])
  Namespace(bar='XX', foo='YY')
  >>> parser.parse_args(['XX', '--foo'])
  Namespace(bar='XX', foo='c')
  ```

```
>>> parser.parse_args([])
Namespace(bar='d', foo='d')
```

Um dos usos mais comuns de `nargs='?'` é permitir arquivos de entrada e saída opcionais:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('infile', nargs='?')
>>> parser.add_argument('outfile', nargs='?')
>>> parser.parse_args(['input.txt', 'output.txt'])
Namespace(infile='input.txt', outfile='output.txt')
>>> parser.parse_args(['input.txt'])
Namespace(infile='input.txt', outfile=None)
>>> parser.parse_args([])
Namespace(infile=None, outfile=None)
```

- `'*'`. Todos os argumentos de linha de comando presentes são reunidos em uma lista. Note que geralmente não faz muito sentido ter mais de um argumento posicional com `nargs='*'`, mas vários argumentos opcionais com `nargs='*'` são possíveis. Por exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', nargs='*')
>>> parser.add_argument('--bar', nargs='*')
>>> parser.add_argument('baz', nargs='*')
>>> parser.parse_args('a b --foo x y --bar 1 2'.split())
Namespace(bar=['1', '2'], baz=['a', 'b'], foo=['x', 'y'])
```

- `'+'`. Assim como `'*'`, todos os argumentos de linha de comando presentes são reunidos em uma lista. Além disso, uma mensagem de erro será gerada se não houver pelo menos um argumento de linha de comando presente. Por exemplo:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('foo', nargs='+')
>>> parser.parse_args(['a', 'b'])
Namespace(foo=['a', 'b'])
>>> parser.parse_args([])
usage: PROG [-h] foo [foo ...]
PROG: error: the following arguments are required: foo
```

Se o argumento nomeado `nargs` não for fornecido, o número de argumentos consumidos é determinado por action. Geralmente, isso significa que um único argumento de linha de comando será usado e um único item (não uma lista) será produzido. Ações que não fazem uso de argumentos da linha de comando (por exemplo, `'store_const'`) definem `nargs=0`.

const

O argumento `const` de **`add_argument()`** é usado para manter valores constantes que não são lidos da linha de comando, mas são necessários para as várias ações **`ArgumentParser`**. Os dois usos mais comuns são:

- Quando **`add_argument()`** é chamado com `action='store_const'` ou `action='append_const'`. Essas ações adicionam o valor `const` a um dos atributos do objeto retornado por **`parse_args()`**. Consulte a descrição da action para obter exemplos. Se `const` não for fornecido **`add_argument()`**, será recebido um valor padrão de `None`.
- Quando **`add_argument()`** é chamado com strings de opções (como `-f` ou `--foo`) e `nargs='?'`. Isso cria um argumento opcional que pode ser seguido por zero ou um argumento de linha de comando. Ao analisar a linha de comando, se a string de opções for encontrada sem nenhum argumento de linha de comando seguindo, o valor de `const` será usado. Veja a descrição de nargs para exemplos.

Alterado na versão 3.11: `const=None` por padrão, incluindo quando `action='append_const'` ou `action='store_const'`.

default

Todos os argumentos opcionais e alguns argumentos posicionais podem ser omitidos na linha de comando. O argumento nomeado `default` de **`add_argument()`**, cujo valor padrão é `None`, especifica qual valor deve ser usado se o argumento de linha de comando não estiver presente. Para argumentos opcionais, o valor `default` é usado quando a string de opção não estava presente na linha de comando:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=42)
>>> parser.parse_args(['--foo', '2'])
Namespace(foo='2')
>>> parser.parse_args([])
Namespace(foo=42)
```

Se o espaço de nomes de destino já tiver um atributo definido, a ação *default* não o substituirá:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=42)
>>> parser.parse_args([], namespace=argparse.Namespace(foo=101))
Namespace(foo=101)
```

Se o valor `default` for uma string, o analisador analisa o valor como se fosse um argumento de linha de comando. Em particular, o analisador aplica qualquer argumento de conversão type, se fornecido, antes de definir o atributo no valor de retorno **`Namespace`**. Caso contrário, o analisador usa o valor como está:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--length', default='10', type=int)
>>> parser.add_argument('--width', default=10.5, type=int)
>>> parser.parse_args()
Namespace(length=10, width=10.5)
```

Para argumentos posicionais com nargs igual a `?` ou `*`, o valor `default` é usado quando nenhum argumento de linha de comando estava presente:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('foo', nargs='?', default=42)
>>> parser.parse_args(['a'])
Namespace(foo='a')
>>> parser.parse_args([])
Namespace(foo=42)
```

Como `nargs='*'` agrupa quaisquer valores fornecidos em uma lista, um argumento posicional ausente produz em uma lista vazia (`[]`). Apenas um

default com valor diferente de `None` substitui esse comportamento (portanto, `default=None` ainda resulta em `[]`).

Para argumentos obrigatórios, isto é, marcados com required, o valor `default` é ignorado. Por exemplo, isso se aplica a argumentos posicionais com valores de nargs diferentes de `?` ou `*`, ou argumentos opcionais marcados como `required=True`.

Fornecer `default=argparse.SUPPRESS` faz com que nenhum atributo seja adicionado se o argumento da linha de comando não estiver presente:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default=argparse.SUPPRESS)
>>> parser.parse_args([])
Namespace()
>>> parser.parse_args(['--foo', '1'])
Namespace(foo='1')
```

type

Por padrão, o analisador sintático lê os argumentos de linha de comando como strings simples. No entanto, muitas vezes a string da linha de comando deve ser interpretada como outro tipo, como **`float`** ou **`int`**. O argumento nomeado `type` para **`add_argument()`** permite que qualquer verificação de tipo e conversões de tipo necessárias sejam realizadas.

Se o argumento nomeado type for usado com default, o conversor de tipo só será aplicado se o padrão for uma string.

O argumento para `type` pode ser um chamável que aceite uma única string ou o nome de um tipo registrado (veja **`register()`**). Se a função levantar **`ArgumentTypeError`**, **`TypeError`** ou **`ValueError`**, a exceção será capturada e uma mensagem de erro bem formatada será exibida. Nenhum outro tipo de exceção é tratado.

Tipos e funções embutidas comuns podem ser usados como conversores de tipo:

```
import argparse
import pathlib

parser = argparse.ArgumentParser()
parser.add_argument('count', type=int)
parser.add_argument('distance', type=float)
parser.add_argument('street', type=ascii)
parser.add_argument('code_point', type=ord)
parser.add_argument('datapath', type=pathlib.Path)
```

Funções definidas pelo usuário também podem ser usadas:

```
>>> def hyphenated(string):
...     return '-'.join([word[:4] for word in string.casefold().
...
>>> parser = argparse.ArgumentParser()
>>> _ = parser.add_argument('short_title', type=hyphenated)
>>> parser.parse_args(['"Um conto de duas cidades"'])
Namespace(short_title='"um-cont-de-duas-cida')
```

A função `bool()` não é recomendada como conversor de tipo. Tudo o que ele faz é converter strings vazias em `False` e strings não vazias em `True`. Geralmente não é isso que se deseja:

```
>>> parser = argparse.ArgumentParser()
>>> _ = parser.add_argument('--verbose', type=bool)
>>> parser.parse_args(['--verbose', 'False'])
Namespace(verbose=True)
```

Consulte `BooleanOptionalAction` ou `action='store_true'` para alternativas comuns.

Em geral, o argumento nomeado `type` é uma conveniência a ser usada apenas para conversões simples que geram apenas uma das três exceções suportadas. Qualquer coisa com tratamento de erros ou gerenciamento de recursos mais interessante deve ser feita posteriormente, após a análise dos argumentos.

Por exemplo, conversões JSON ou YAML têm casos de erros complexos que exigem relatórios melhores do que os fornecidos pelo argumento

nomeado `type`. Um **`JSONDecodeError`** não seria bem formatado e uma exceção **`FileNotFoundError`** não seria tratada.

Mesmo **`FileType`** tem suas limitações no uso com o argumento nomeado `type`. Se um argumento usar **`FileType`** e um argumento subsequente falhar, um erro será relatado, mas o arquivo não será fechado automaticamente. Neste caso, seria melhor esperar até que o analisador tenha sido executado e então usar a instrução **`with`** para gerenciar os arquivos.

Para verificadores de tipo que simplesmente verificam um conjunto fixo de valores, considere usar o argumento nomeado choices.

choices

Alguns argumentos de linha de comando devem ser selecionados em um conjunto restrito de valores. Eles podem ser tratados passando um objeto sequência como o argumento nomeado *choices* para **`add_argument()`**. Quando a linha de comando for analisada, os valores dos argumentos serão verificados e uma mensagem de erro será exibida se o argumento não for um dos valores aceitáveis:

```
>>> parser = argparse.ArgumentParser(prog='game.py')
>>> parser.add_argument('move', choices=['pedra', 'papel', 'tesc
>>> parser.parse_args(['padra'])
Namespace(move='padr')
>>> parser.parse_args(['fogo'])
usage: game.py [-h] {pedra,papel,tesoura}
game.py: error: argument move: invalid choice: 'fogo' (choose fr
'papel', 'tesoura')
```

Qualquer sequência pode ser passada como o valor *choices*, portanto objetos **`list`**, objetos **`tuple`** e sequências personalizadas são todos suportados.

O uso de **`enum.Enum`** não é recomendado porque é difícil controlar sua aparência no uso, na ajuda e nas mensagens de erro.

Observe que *choices* são verificadas após a realização de quaisquer conversões de type, portanto, os objetos em *choices* devem corresponder ao type especificado. Isso pode fazer com que *choices* pareçam estranhos no uso, na ajuda ou em mensagens de erro.

Para manter *choices* fácel de usar, considere um invólucro de tipo personalizado que converta e formate valores, ou omita type e lide com a conversão no código da sua aplicação.

As opções formatadas substituem o *metavar* padrão que normalmente é derivado de *dest*. Geralmente é isso que você deseja porque o usuário nunca vê o parâmetro *dest*. Se esta exibição não for desejável (talvez porque haja muitas opções), basta especificar um metavar explícito.

required

Em geral, o módulo **`argparse`** presume que sinalizadores como `-f` e `--bar` indicam argumentos *opcionais*, que sempre podem ser omitidos na linha de comando. Para tornar uma opção obrigatória, `True` pode ser especificado para o argumento nomeado `required=` para **`add_argument()`**:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', required=True)
>>> parser.parse_args(['--foo', 'BAR'])
Namespace(foo='BAR')
>>> parser.parse_args([])
usage: [-h] --foo FOO
: error: the following arguments are required: --foo
```

Como mostra o exemplo, se uma opção estiver marcada como `required`, **`parse_args()`** reportará um erro se essa opção não estiver presente na linha de comando.

Nota

As opções obrigatórias são geralmente consideradas inadequadas porque os usuários esperam que as *opções* sejam *opcionais* e, portanto, devem ser evitadas quando possível.

help

O valor `help` é uma string contendo uma breve descrição do argumento. Quando um usuário solicita ajuda (geralmente usando `-h` ou `--help` na linha de comando), estas descrições de `help` serão exibidas com cada argumento.

As strings `help` podem incluir vários especificadores de formato para evitar a repetição de coisas como o nome do programa ou o argumento default. Os especificadores disponíveis incluem o nome do programa, `%(prog)s` e a maioria dos argumentos nomeados para **`add_argument()`**, por exemplo. `%(default)s`, `%(type)s`, etc.:

```
>>> parser = argparse.ArgumentParser(prog='frobble')
>>> parser.add_argument('bar', nargs='?', type=int, default=42,
...                     help='the bar to %(prog)s (default: %(de
>>> parser.print_help()
usage: frobble [-h] [bar]

positional arguments:
 bar     the bar to frobble (default: 42)

options:
 -h, --help  show this help message and exit
```

Como a string de ajuda oferece suporte à formatação com %, se você quiser que um literal `%` apareça na string de ajuda, você deve escapá-lo como `%%`.

`argparse` oferece suporte a silenciar a entrada de ajuda para certas opções, definindo o valor `help` como `argparse.SUPPRESS`:

```
>>> parser = argparse.ArgumentParser(prog='frobble')
>>> parser.add_argument('--foo', help=argparse.SUPPRESS)
>>> parser.print_help()
usage: frobble [-h]

options:
  -h, --help  show this help message and exit
```

metavar

Quando **`ArgumentParser`** gera mensagens de ajuda, ele precisa de alguma forma de se referir a cada argumento esperado. Por padrão, os objetos **`ArgumentParser`** usam o valor dest como o “nome” de cada objeto. Por padrão, para ações de argumentos posicionais, o valor dest é usado diretamente, e para ações de argumentos opcionais, o valor dest é maiúsculo. Portanto, um único argumento posicional com `dest='bar'` será referido como `bar`. Um único argumento opcional `--foo` que deve ser seguido por um único argumento de linha de comando será referido como `FOO`. Um exemplo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.add_argument('bar')
>>> parser.parse_args('X --foo Y'.split())
Namespace(bar='X', foo='Y')
>>> parser.print_help()
usage:  [-h] [--foo FOO] bar

positional arguments:
 bar

options:
 -h, --help  show this help message and exit
 --foo FOO
```

Um nome alternativo pode ser especificado com `metavar`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', metavar='YYY')
>>> parser.add_argument('bar', metavar='XXX')
>>> parser.parse_args('X --foo Y'.split())
Namespace(bar='X', foo='Y')
>>> parser.print_help()
usage:  [-h] [--foo YYY] XXX

positional arguments:
 XXX

options:
```

```
 -h, --help  show this help message and exit
 --foo YYY
```

Observe que `metavar` apenas altera o nome *exibido* - o nome do atributo no objeto **`parse_args()`** ainda é determinado pelo valor dest.

Valores diferentes de `nargs` podem fazer com que o metavar seja usado múltiplas vezes. Fornecer uma tupla para `metavar` especifica uma exibição diferente para cada um dos argumentos:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x', nargs=2)
>>> parser.add_argument('--foo', nargs=2, metavar=('bar', 'baz'))
>>> parser.print_help()
usage: PROG [-h] [-x X X] [--foo bar baz]

options:
 -h, --help     show this help message and exit
 -x X X
 --foo bar baz
```

dest

A maioria das ações **`ArgumentParser`** adiciona algum valor como um atributo do objeto retornado por **`parse_args()`**. O nome deste atributo é determinado pelo argumento nomeado `dest` de **`add_argument()`**. Para ações de argumento posicional, `dest` é normalmente fornecido como o primeiro argumento para **`add_argument()`**:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('bar')
>>> parser.parse_args(['XXX'])
Namespace(bar='XXX')
```

For optional argument actions, the value of `dest` is normally inferred from the option strings. **`ArgumentParser`** generates the value of `dest` by taking the first double-dash long option string and stripping away the initial `-` characters. If no double-dash long option strings were supplied, `dest` will be derived from the first single-dash long option string by stripping the

initial `-` character. If no long option strings were supplied, `dest` will be derived from the first short option string by stripping the initial `-` character. Any internal `-` characters will be converted to _ characters to make sure the string is a valid attribute name. The examples below illustrate this behavior:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('-f', '--foo-bar', '--foo')
>>> parser.add_argument('-q', '-quz')
>>> parser.add_argument('-x', '-y')
>>> parser.parse_args('-f 1 -q 2 -x 3'.split())
Namespace(foo_bar='1', quz='2', x='3')
>>> parser.parse_args('--foo 1 -quz 2 -y 3'.split())
Namespace(foo_bar='1', quz='2', x='2')
```

`dest` permite que um nome de atributo personalizado seja fornecido:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', dest='bar')
>>> parser.parse_args('--foo XXX'.split())
Namespace(bar='XXX')
```

Multiple arguments may share the same `dest`. By default, the value from the last such argument given on the command line wins. Use `action='append'` to collect values from all of them into a list instead. For conflicting *option strings* rather than `dest` names, see conflict_handler.

Alterado na versão 3.15: Single-dash long option now takes precedence over short options.

deprecated

Durante a vida de um projeto, alguns argumentos podem precisar ser removidos da linha de comando. Antes de removê-los, você deve informar aos seus usuários que os argumentos estão descontinuados e serão removidos. O argumento nomeado `deprecated` de **`add_argument()`**, cujo padrão é `False`, especifica se o argumento está descontinuado e será removido no futuro. Para argumentos, se `deprecated` for `True`, então um aviso será enviado para **`sys.stderr`** (saída de erro) quando o argumento for usado:

```
>>> import argparse
>>> parser = argparse.ArgumentParser(prog='snake.py')
>>> parser.add_argument('--legs', default=0, type=int, deprecate
>>> parser.parse_args([])
Namespace(legs=0)
>>> parser.parse_args(['--legs', '4'])
snake.py: warning: option '--legs' is deprecated
Namespace(legs=4)
```

Adicionado na versão 3.13.

Classes de ação

As classes **`Action`** implementam a API de Action, um chamável que retorna um chamável que processa argumentos da linha de comando. Qualquer objeto que siga esta API pode ser passado como parâmetro `action` para **`add_argument()`**.

class argparse.Action(*option_strings*, *dest*, *nargs=None*, *const=None*, *default=None*, *type=None*, *choices=None*, *required=False*, *help=None*, *metavar=None*)

Objetos **`Action`** são usados por um **`ArgumentParser`** para representar as informações necessárias para analisar um único argumento de uma ou mais strings da linha de comando. A classe **`Action`** deve aceitar os dois argumentos posicionais mais quaisquer argumentos nomeados passados para **`ArgumentParser.add_argument()`**, exceto o próprio `action`.

Instâncias de **`Action`** (ou valor de retorno de qualquer chamável para o parâmetro `action`) devem ter atributos **`dest`**, **`option_strings`**, **`default`**, **`type`**, **`required`**, **`help`**, etc. definidos. A maneira mais fácil de garantir que esses atributos sejam definidos é chamar **`Action.__init__()`**.

__call__(*parser*, *namespace*, *values*, *option_string=None*)

As instâncias de **`Action`** devem ser chamáveis, portanto, as subclasses devem substituir o método **`__call__()`**, que deve aceitar quatro parâmetros:

- *parser* - O objeto **`ArgumentParser`** que contém esta ação.
- *namespace* - O objeto **`Namespace`** que será retornado por **`parse_args()`**. A maioria das ações adicionam um atributo a este objeto usando **`setattr()`**.
- *values* - Os argumentos de linha de comando associados, com qualquer conversões de tipo aplicadas. Conversões de tipo são especificadas com o argumento nomeado type para **`add_argument()`**.
- *option_string* - A string da opção que foi usada para invocar esta ação. O argumento `option_string` é opcional e estará ausente se a ação estiver associada a um argumento posicional.

O método **`__call__()`** pode executar ações arbitrárias, mas normalmente definirá atributos em `namespace` com base em `dest` e `values`.

format_usage()

As subclasses de **`Action`** podem definir um método **`format_usage()`** que não recebe argumento e retorna uma string que será usada ao exibir a mensagem de uso do programa. Se tal método não for fornecido, um padrão sensato será usado.

class argparse.BooleanOptionalAction

Uma subclasse de **`Action`** para manipular sinalizadores booleanos com opções positivas e negativas. Adicionar um único argumento, como `--foo`, cria automaticamente as opções `--foo` e `--no-foo`, armazenando `True` e `False` respectivamente:

```
>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action=argparse.BooleanOptio
>>> parser.parse_args(['--no-foo'])
Namespace(foo=False)
```

Single-dash long options are also supported. For example, negative option `-nofoo` is automatically added for positive option `-foo`. But no additional options are added for short options such as `-f`.

Adicionado na versão 3.9.

Alterado na versão 3.15: Added support for single-dash options.

Added support for alternate prefix_chars.

O método parse_args()

ArgumentParser.parse_args(*args=None*, *namespace=None*)

Converte strings de argumento em objetos e os atribui como atributos do espaço de nomes. Retorna o espaço de nomes preenchido.

Chamadas anteriores para `add_argument()` determinam exatamente quais objetos são criados e como eles são atribuídos. Veja a documentação de **`add_argument()`** para detalhes.

- args - Lista de strings para analisar. O padrão é obtido de `sys.argv`.
- namespace - Um objeto para receber os atributos. O padrão é um novo objeto `Namespace` vazio.

Sintaxe de valores da opção

O método `parse_args()` provê várias maneiras de especificar o valor de uma opção (se ele pegar uma). No caso mais simples, a opção e seu valor são passados como dois argumentos separados:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x')
>>> parser.add_argument('--foo')
>>> parser.parse_args(['-x', 'X'])
Namespace(foo=None, x='X')
>>> parser.parse_args(['--foo', 'FOO'])
Namespace(foo='FOO', x=None)
```

Para opções longas (opções com nomes maiores que um único caractere), a opção e o valor também podem ser passados como um único argumento de linha de comando, usando = para separá-los:

```
>>> parser.parse_args(['--foo=FOO'])
Namespace(foo='FOO', x=None)
```

Para opções curtas (opções com apenas um caractere), a opção e seu valor podem ser concatenados:

```
>>> parser.parse_args(['-xX'])
Namespace(foo=None, x='X')
```

Várias opções curtas podem ser unidas, usando apenas um único prefixo `-`, desde que apenas a última opção (ou nenhuma delas) exija um valor:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x', action='store_true')
>>> parser.add_argument('-y', action='store_true')
>>> parser.add_argument('-z')
>>> parser.parse_args(['-xyzZ'])
Namespace(x=True, y=True, z='Z')
```

Argumentos inválidos

Ao analisar a linha de comando, `parse_args()` verifica uma variedade de erros, incluindo opções ambíguas, tipos inválidos, opções inválidas, número incorreto de argumentos posicionais, etc. Quando encontra tal erro, ele sai e imprime o erro junto com uma mensagem de uso:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('--foo', type=int)
>>> parser.add_argument('bar', nargs='?')

>>> # invalid type
>>> parser.parse_args(['--foo', 'spam'])
usage: PROG [-h] [--foo FOO] [bar]
PROG: error: argument --foo: invalid int value: 'spam'

>>> # invalid option
>>> parser.parse_args(['--bar'])
usage: PROG [-h] [--foo FOO] [bar]
PROG: error: unrecognized arguments: --bar

>>> # wrong number of arguments
```

```
>>> parser.parse_args(['spam', 'badger'])
usage: PROG [-h] [--foo FOO] [bar]
PROG: error: unrecognized arguments: badger
```

Argumentos contendo -

O método `parse_args()` tenta mostrar erros sempre que o usuário claramente cometeu um erro, mas algumas situações são inerentemente ambíguas. Por exemplo, o argumento de linha de comando `-1` pode ser uma tentativa de especificar uma opção ou uma tentativa de fornecer um argumento posicional. O método **`parse_args()`** é cauteloso aqui: argumentos posicionais só podem começar com `-` se eles se parecerem com números negativos e não houver opções no analisador sintático que se pareçam com números negativos:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-x')
>>> parser.add_argument('foo', nargs='?')

>>> # no negative number options, so -1 is a positional argument
>>> parser.parse_args(['-x', '-1'])
Namespace(foo=None, x='-1')

>>> # no negative number options, so -1 and -5 are positional ar
>>> parser.parse_args(['-x', '-1', '-5'])
Namespace(foo='-5', x='-1')

>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-1', dest='one')
>>> parser.add_argument('foo', nargs='?')

>>> # negative number options present, so -1 is an option
>>> parser.parse_args(['-1', 'X'])
Namespace(foo=None, one='X')

>>> # negative number options present, so -2 is an option
>>> parser.parse_args(['-2'])
usage: PROG [-h] [-1 ONE] [foo]
PROG: error: unrecognized arguments: -2

>>> # negative number options present, so both -1s are options
```

```
>>> parser.parse_args(['-1', '-1'])
usage: PROG [-h] [-1 ONE] [foo]
PROG: error: argument -1: expected one argument
```

Se você tiver argumentos posicionais que devem começar com `-` e não se parecem com números negativos, você pode inserir o pseudoargumento `'--'` que informa **`parse_args()`** que tudo depois disso é um argumento posicional:

```
>>> parser.parse_args(['--', '-f'])
Namespace(foo='-f', one=None)
```

Veja também o tutorial do argparse sobre argumentos ambíguos para mais detalhes.

Alterado na versão 3.14: Negative-number matching was expanded to include numbers in scientific notation (`-2.5e-6`), numbers containing underscores (`-1_234.5`), and complex numbers (`-1.2e-3j`).

Abreviações de argumento (correspondência de prefixo)

O método **`parse_args()`** por padrão permite que opções longas sejam abreviadas para um prefixo, se a abreviação não for ambígua (o prefixo corresponde a uma opção única):

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-bacon')
>>> parser.add_argument('-badger')
>>> parser.parse_args('-bac MMM'.split())
Namespace(bacon='MMM', badger=None)
>>> parser.parse_args('-bad WOOD'.split())
Namespace(bacon=None, badger='WOOD')
>>> parser.parse_args('-ba BA'.split())
usage: PROG [-h] [-bacon BACON] [-badger BADGER]
PROG: error: ambiguous option: -ba could match -badger, -bacon
```

Um erro é produzido para argumentos que podem produzir mais de uma opção. Este recurso pode ser desabilitado definindo allow_abbrev como `False`.

Além do `sys.argv`

Às vezes, pode ser útil ter uma instância de `ArgumentParser` analisando argumentos diferentes daqueles de `sys.argv`. Isso pode ser feito passando uma lista de strings para `parse_args()`. Isso é útil para testar no prompt interativo:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument(
...     'integers', metavar='int', type=int, choices=range(10),
...     nargs='+', help='an integer in the range 0..9')
>>> parser.add_argument(
...     '--sum', dest='accumulate', action='store_const', const=
...     default=max, help='sum the integers (default: find the m
>>> parser.parse_args(['1', '2', '3', '4'])
Namespace(accumulate=<built-in function max>, integers=[1, 2, 3,
>>> parser.parse_args(['1', '2', '3', '4', '--sum'])
Namespace(accumulate=<built-in function sum>, integers=[1, 2, 3,
```

O objeto Namespace

class argparse.Namespace

Classe simples usada por padrão por `parse_args()` para criar um objeto contendo atributos e retorná-lo.

Esta classe é deliberadamente simples, apenas uma subclasse `object` com uma representação de string legível. Se você preferir ter uma visão dos atributos do tipo dict, você pode usar o idioma padrão do Python, `vars()`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> args = parser.parse_args(['--foo', 'BAR'])
>>> vars(args)
{'foo': 'BAR'}
```

Também pode ser útil ter um `ArgumentParser` atribuindo atributos a um objeto já existente, em vez de um novo objeto `Namespace`. Isso pode ser obtido especificando o argumento nomeado `namespace=`:

```
>>> class C:
...     pass
...
>>> c = C()
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.parse_args(args=['--foo', 'BAR'], namespace=c)
>>> c.foo
'BAR'
```

Outros utilitários

Subcomandos

ArgumentParser.add_subparsers(*[, *title*][, *description*][, *prog*][, *parser_class*][, *action*][, *dest*][, *required*][, *help*][, *metavar*])

Muitos programas dividem sua funcionalidade em vários subcomandos, por exemplo, o programa `svn` pode invocar subcomandos como `svn checkout`, `svn update` e `svn commit`. Dividir a funcionalidade dessa forma pode ser uma ideia particularmente boa quando um programa executa várias funções diferentes que exigem diferentes tipos de argumentos de linha de comando. **`ArgumentParser`** oferece suporte à criação de tais subcomandos por meio do método **`add_subparsers()`**. O método **`add_subparsers()`** é normalmente chamado sem argumentos e retorna um objeto de ação especial. Este objeto tem um único método, **`add_parser()`**, que recebe um nome de comando e quaisquer argumentos do construtor **`ArgumentParser`** e retorna um objeto **`ArgumentParser`** que pode ser modificado normalmente.

Descrição de parâmetros:

- *title* - título para o grupo de subanalisadores na saída de ajuda; por padrão “subcomandos” se a descrição for fornecida, caso contrário, usa o título para argumentos posicionais
- *description* - descrição para o grupo de subanalisadores na saída de ajuda, por padrão `None`

- *prog* - informações de uso que serão exibidas com a ajuda do subcomando, por padrão o nome do programa e quaisquer argumentos posicionais antes do argumento do subanalisador
- *parser_class* - classe que será usada para criar instâncias de subanalisadores, por padrão a classe do analisador atual (por exemplo, `ArgumentParser`)
- action - o tipo básico de ação a ser executada quando esse argumento é encontrado na linha de comando
- dest - nome do atributo sob o qual o nome do subcomando será armazenado; por padrão `None` e nenhum valor é armazenado
- required - Se um subcomando deve ou não ser fornecido, por padrão `False` (adicionado em 3.7)
- help - ajuda para o grupo de subanalisadores na saída de ajuda, por padrão `None`
- metavar - string que apresenta os subcomandos disponíveis na ajuda; por padrão é `None` e apresenta os subcomandos no formato {cmd1, cmd2, ..}

Alguns exemplos de uso:

```
>>> # cria o analisador de nível superior>>> parser = argpars
```

Note that the object returned by `parse_args()` will only contain attributes for the main parser and the subparser that was selected by the command line (and not any other subparsers). So in the example above, when the `a` command is specified, only the `foo` and `bar` attributes are present, and when the `b` command is specified, only the `foo` and `baz` attributes are present.

If a subparser defines an argument with the same `dest` as the parent parser, the two share a single namespace attribute, so the parent's value won't be retained. Users should give them distinct `dest` values to keep both.

Similarmente, quando uma mensagem de ajuda é solicitada de um subanalisador, apenas a ajuda para aquele parser em particular será exibida. A mensagem de ajuda não incluirá mensagens do analisador sintático pai nem do analisador sintático irmão. (Uma mensagem de

ajuda para cada comando do subanalisador pode, entretanto, ser fornecida passando o argumento `help=` para **`add_parser()`**, como acima.)

```
>>> parser.parse_args(['--help'])usage: PROG [-h] [--foo] {a,
```

The **`add_subparsers()`** method also supports `title` and `description` keyword arguments. When either is present, the subparser's commands will appear in their own group in the help output. For example:

```
>>> parser = argparse.ArgumentParser()>>> subparsers = parser
```

One particularly effective way of handling subcommands is to combine the use of the **`add_subparsers()`** method with calls to **`set_defaults()`** so that each subparser knows which Python function it should execute. For example:

```
>>> # funções do subanalisador functions>>> def foo(args):...
```

This way, you can let **`parse_args()`** do the job of calling the appropriate function after argument parsing is complete. Associating functions with actions like this is typically the easiest way to handle the different actions for each of your subparsers. However, if it is necessary to check the name of the subparser that was invoked, the `dest` keyword argument to the **`add_subparsers()`** call will work:

```
>>> parser = argparse.ArgumentParser()>>> subparsers = parser
```

Alterado na versão 3.7: Novo parâmetro somente-nomeado *required*.

Alterado na versão 3.14: O *prog* do subanalisador não é mais afetado por uma mensagem de uso personalizada no analisador principal.

_SubParsersAction.add_parser(*name*, *, *help=None*, *aliases=None*, *deprecated=False*, ***kwargs*)

Create and return a new **`ArgumentParser`** object for the subcommand *name*.

The *name* argument is the name of the sub-command.

The *help* argument provides a short description for this sub-command.

The *aliases* argument allows providing alternative names for this sub-command. For example:

```
>>> parser = argparse.ArgumentParser()>>> subparsers = parser
```

The *deprecated* argument, if `True`, marks the sub-command as deprecated and will issue a warning when used. For example:

```
>>> parser = argparse.ArgumentParser(prog='chicken.py')
>>> subparsers = parser.add_subparsers()
>>> fly = subparsers.add_parser('fly', deprecated=True)
>>> args = parser.parse_args(['fly'])
chicken.py: warning: command 'fly' is deprecated
Namespace()
```

All other keyword arguments are passed directly to the **`ArgumentParser`** constructor.

Adicionado na versão 3.13: Added the *deprecated* parameter.

Objetos FileType

class argparse.FileType(*mode='r'*, *bufsize=-1*, *encoding=None*, *errors=None*)

A fábrica **`FileType`** cria objetos que podem ser passados para o argumento de tipo de **`ArgumentParser.add_argument()`**. Argumentos que têm objetos **`FileType`** como seu tipo abrirão argumentos de linha de comando como arquivos com os modos solicitados, tamanhos de buffer, codificações e tratamento de erros (veja a função **`open()`** para mais detalhes):

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--raw', type=argparse.FileType('wb',
>>> parser.add_argument('out', type=argparse.FileType('w', er
```

```
>>> parser.parse_args(['--raw', 'raw.dat', 'file.txt'])
Namespace(out=<_io.TextIOWrapper name='file.txt' mode='w' enc
```

Objetos FileType entendem o pseudoargumento `'-'` e o convertem automaticamente em `sys.stdin` para objetos `FileType` legíveis e `sys.stdout` para objetos `FileType` graváveis:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('infile', type=argparse.FileType('r')
>>> parser.parse_args(['-'])
Namespace(infile=<_io.TextIOWrapper name='<stdin>' encoding='
```

Nota

Se um argumento usar *FileType* e um argumento subsequente falhar, um erro será relatado, mas o arquivo não será fechado automaticamente. Isso também pode danificar os arquivos de saída. Neste caso, seria melhor esperar até que o analisador sintático tenha sido executado e então usar a instrução `with` para gerenciar os arquivos.

Alterado na versão 3.4: Adicionados os parâmetros *encoding* e *errors*.

Descontinuado desde a versão 3.14.

Grupos de argumentos

ArgumentParser.add_argument_group(*title=None*, *description=None*, *[, *argument_default*][, *conflict_handler*])

Por padrão, `ArgumentParser` agrupa os argumentos de linha de comando em “argumentos posicionais” e “opções” ao exibir mensagens de ajuda. Quando há um melhor agrupamento conceitual de argumentos do que este padrão, grupos apropriados podem ser criados usando o método **`add_argument_group()`**:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=Fa
>>> group = parser.add_argument_group('grupo')
```

```
>>> group.add_argument('--foo', help='Ajuda de foo')
>>> group.add_argument('bar', help='Ajuda de bar')
>>> parser.print_help()
usage: PROG [--foo FOO] bar

grupo:
  bar    Ajuda de bar
  --foo FOO  Ajuda de foo
```

O método **add_argument_group()** retorna um objeto de grupo de argumentos que tem o método **add_argument()** como um **ArgumentParser** regular. Quando um argumento é adicionado ao grupo, o analisador o trata como um argumento normal, mas o exibe em um grupo separado nas mensagens de ajuda. O método **add_argument_group()** aceita argumentos *title* e *description* que podem ser usados para personalizar esta exibição:

```
>>> parser = argparse.ArgumentParser(prog='PROG', add_help=Fa
>>> group1 = parser.add_argument_group('grupo1', 'Descrição d
>>> group1.add_argument('foo', help='Ajuda de foo')
>>> group2 = parser.add_argument_group('grupo2', 'Descrição d
>>> group2.add_argument('--bar', help='Ajuda de bar')
>>> parser.print_help()
usage: PROG [--bar BAR] foo

grupo1:
  Descrição de grupo1

  foo    Ajuda de foo

grupo2:
  Descrição de grupo2

  --bar BAR  Ajuda de bar
```

Os parâmetros opcionais somente-nomeados argument_default e conflict_handler permitem um controle mais refinado do comportamento do grupo de argumentos. Esses parâmetros têm o mesmo significado que no construtor **ArgumentParser**, mas se aplicam

especificamente ao grupo de argumentos em vez do analisador sintático inteiro.

Observe que quaisquer argumentos que não estejam nos grupos definidos pelo usuário serão encaminhados às seções usuais de “argumentos posicionais” e “argumentos opcionais”.

Em cada grupo, os argumentos podem ser exibidos em ordem diferente da em que foram adicionados.

Descontinuado desde a versão 3.11, removido na versão 3.14: Chamar **`add_argument_group()`** em um grupo de argumentos agora levanta uma exceção. Esse aninhamento nunca foi suportado, frequentemente não funcionava corretamente e era exposto involuntariamente por meio de herança.

Descontinuado desde a versão 3.14: Passar prefix_chars para **`add_argument_group()`** agora está descontinuado.

Exclusão mútua

ArgumentParser.add_mutually_exclusive_group(*required=False*)

Cria um grupo mutuamente exclusivo. **`argparse`** garantirá que apenas um dos argumentos no grupo mutuamente exclusivo esteja presente na linha de comando:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> group = parser.add_mutually_exclusive_group()
>>> group.add_argument('--foo', action='store_true')
>>> group.add_argument('--bar', action='store_false')
>>> parser.parse_args(['--foo'])
Namespace(bar=True, foo=True)
>>> parser.parse_args(['--bar'])
Namespace(bar=False, foo=False)
>>> parser.parse_args(['--foo', '--bar'])
usage: PROG [-h] [--foo | --bar]
PROG: error: argument --bar: not allowed with argument --foo
```

O método `add_mutually_exclusive_group()` também aceita um argumento *obrigatório*, para indicar que pelo menos um dos argumentos mutuamente exclusivos é necessário:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> group = parser.add_mutually_exclusive_group(required=True
>>> group.add_argument('--foo', action='store_true')
>>> group.add_argument('--bar', action='store_false')
>>> parser.parse_args([])
usage: PROG [-h] (--foo | --bar)
PROG: error: one of the arguments --foo --bar is required
```

Observe que atualmente grupos de argumentos mutuamente exclusivos não oferecem suporte aos argumentos *title* e *description* de `add_argument_group()`. No entanto, um grupo mutuamente exclusivo pode ser adicionado a um grupo de argumentos que tenha um título e uma descrição. Por exemplo:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> group = parser.add_argument_group('Título do grupo', 'Des
>>> exclusive_group = group.add_mutually_exclusive_group(requ
>>> exclusive_group.add_argument('--foo', help='ajuda de foo'
>>> exclusive_group.add_argument('--bar', help='ajuda de bar'
>>> parser.print_help()
usage: PROG [-h] (--foo FOO | --bar BAR)

options:
  -h, --help  show this help message and exit

Título do grupo:
  Descrição do grupo

  --foo FOO   foo help
  --bar BAR   bar help
```

Descontinuado desde a versão 3.11, removido na versão 3.14: Chamar `add_argument_group()` ou `add_mutually_exclusive_group()` em um grupo mutuamente exclusivo agora levanta uma exceção. Esse aninhamento nunca foi suportado, frequentemente não funcionava corretamente e era exposto involuntariamente por meio de herança.

Padrões do analisador sintático

ArgumentParser.set_defaults(**kwargs)

Na maioria das vezes, os atributos do objeto retornado por `parse_args()` serão totalmente determinados pela inspeção dos argumentos da linha de comando e das ações dos argumentos. `set_defaults()` permite que alguns atributos adicionais que são determinados sem qualquer inspeção da linha de comando sejam adicionados:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('foo', type=int)
>>> parser.set_defaults(bar=42, baz='badger')
>>> parser.parse_args(['736'])
Namespace(bar=42, baz='badger', foo=736)
```

Observe que os valores padrão podem ser definidos tanto no nível do analisador sintático usando `set_defaults()` quanto no nível do argumento usando `add_argument()`. Se ambos forem chamados para o mesmo argumento, o último valor padrão definido para o argumento será usado.

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default='bar')
>>> parser.set_defaults(foo='spam')
>>> parser.parse_args([])
Namespace(foo='spam')
```

Padrões de nível de analisador podem ser particularmente úteis ao trabalhar com vários analisadores. Veja o método `add_subparsers()` para um exemplo desse tipo.

ArgumentParser.get_default(*dest*)

Obtém o valor padrão para um atributo de espaço de nomes, conforme definido por `add_argument()` ou por `set_defaults()`:

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', default='badger')
```

```
>>> parser.get_default('foo')
'badger'
```

Imprimindo a ajuda

Na maioria das aplicações típicas, `parse_args()` cuidará da formatação e da impressão de quaisquer mensagens de uso ou erro. No entanto, vários métodos de formatação estão disponíveis:

ArgumentParser.print_usage(*file=None*)

Imprime uma breve descrição de como o `ArgumentParser` deve ser invocado na linha de comando. Se *file* for `None`, `sys.stdout` será presumido.

ArgumentParser.print_help(*file=None*)

Imprime uma mensagem de ajuda, incluindo o uso do programa e informações sobre os argumentos registrados com o `ArgumentParser`. Se *file* for `None`, `sys.stdout` será presumido.

Também há variantes desses métodos que simplesmente retornam uma string em vez de imprimi-la:

ArgumentParser.format_usage()

Retorna uma string contendo uma breve descrição de como o `ArgumentParser` deve ser invocado na linha de comando.

ArgumentParser.format_help()

Retorna uma string contendo uma mensagem de ajuda, incluindo o uso do programa e informações sobre os argumentos registrados com o `ArgumentParser`.

Análise parcial

ArgumentParser.parse_known_args(*args=None*, *namespace=None*)

Às vezes, um script precisa manipular apenas um conjunto específico de argumentos de linha de comando, deixando quaisquer argumentos não

reconhecidos para outro script ou programa. Nesses casos, o método `parse_known_args()` pode ser útil.

Este método funciona de forma semelhante a `parse_args()`, mas não levanta erro para argumentos extras não reconhecidos. Em vez disso, ele analisa os argumentos conhecidos e retorna uma tupla de dois itens que contém o espaço de nomes preenchido e a lista de quaisquer argumentos não reconhecidos.

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo', action='store_true')
>>> parser.add_argument('bar')
>>> parser.parse_known_args(['--foo', '--badger', 'BAR', 'spa
(Namespace(bar='BAR', foo=True), ['--badger', 'spam'])
```

Aviso

As regras de correspondência de prefixo se aplicam a `parse_known_args()`. O analisador pode usar uma opção mesmo que seja apenas um prefixo de uma de suas opções conhecidas, em vez de deixá-la na lista de argumentos restantes.

Personalizando a análise de arquivos

ArgumentParser.convert_arg_line_to_args(*arg_line*)

Argumentos lidos de um arquivo (veja o argumento nomeado *fromfile_prefix_chars* para o construtor `ArgumentParser`) são lidos com um argumento por linha. `convert_arg_line_to_args()` pode ser substituído para uma leitura mais sofisticada.

Este método recebe um único argumento *arg_line* que é uma string lida do arquivo de argumentos. Ele retorna uma lista de argumentos analisados dessa string. O método é chamado uma vez por linha lida do arquivo de argumentos, em ordem.

Uma substituição útil desse método é aquela que trata cada palavra separada por espaços como um argumento. O exemplo a seguir demonstra como fazer isso:

```
class MyArgumentParser(argparse.ArgumentParser):
    def convert_arg_line_to_args(self, arg_line):
        return arg_line.split()
```

Note that with this override an argument can no longer contain spaces, since each space-separated word becomes a separate argument.

Métodos de saída

ArgumentParser.exit(*status=0*, *message=None*)

Este método encerra o programa, saindo com o *status* especificado e, se fornecido, imprime uma *message* para `sys.stderr` antes disso. O usuário pode sobrescrever este método para lidar com essas etapas de forma diferente:

```
class ErrorCatchingArgumentParser(argparse.ArgumentParser):
    def exit(self, status=0, message=None):
        if status:
            raise Exception(f'Exiting because of an error: {m
        exit(status)
```

ArgumentParser.error(*message*)

Este método imprime uma mensagem de uso, incluindo a *message*, em `sys.stderr` e encerra o programa com um código de status 2.

Análise misturada

ArgumentParser.parse_intermixed_args(*args=None*, *namespace=None*)

ArgumentParser.parse_known_intermixed_args(*args=None*, *namespace=None*)

Vários comandos Unix permitem que o usuário misture argumentos opcionais com argumentos posicionais. Os métodos

`parse_intermixed_args()` e `parse_known_intermixed_args()` oferecem suporte a esse estilo de análise.

Esses analisadores não oferecem suporte a todos os recursos do **`argparse`** e vão levantar exceções se recursos não suportados forem usados. Em particular, subanalisadores e grupos mutuamente exclusivos que incluem tanto opcionais quanto posicionais não são suportados.

O exemplo a seguir mostra a diferença entre `parse_known_args()` e `parse_intermixed_args()`: o primeiro retorna `['2', '3']` como argumentos não analisados, enquanto o último coleta todos os posicionais em `rest`.

```
>>> parser = argparse.ArgumentParser()
>>> parser.add_argument('--foo')
>>> parser.add_argument('cmd')
>>> parser.add_argument('rest', nargs='*', type=int)
>>> parser.parse_known_args('doit 1 --foo bar 2 3'.split())
(Namespace(cmd='doit', foo='bar', rest=[1]), ['2', '3'])
>>> parser.parse_intermixed_args('doit 1 --foo bar 2 3'.split
Namespace(cmd='doit', foo='bar', rest=[1, 2, 3])
```

`parse_known_intermixed_args()` retorna uma tupla de dois itens contendo o espaço de nomes preenchido e a lista de strings de argumentos restantes. `parse_intermixed_args()` levanta um erro se houver strings de argumentos não analisadas restantes.

Adicionado na versão 3.7.

Registrando ações e tipos personalizados

ArgumentParser.register(*registry_name*, *value*, *object*)

Às vezes, é desejável usar uma string personalizada em mensagens de erro para fornecer uma saída mais amigável ao usuário. Nesses casos, **`register()`** pode ser usado para registrar ações ou tipos personalizados com um analisador sintático e permitir que você faça referência ao tipo pelo nome registrado em vez do nome do chamável.

O método **`register()`** aceita três argumentos - um *registry_name*, especificando o registro interno onde o objeto será armazenado (por exemplo, `action`, `type`), *value*, que é a chave sob a qual o objeto será registrado, e object, o chamável a ser registrado.

O exemplo a seguir mostra como registrar um tipo personalizado com um analisador:

```
>>> import argparse
>>> parser = argparse.ArgumentParser()
>>> parser.register('type', 'hexadecimal integer', lambda s:
>>> parser.add_argument('--foo', type='hexadecimal integer')
_StoreAction(option_strings=['--foo'], dest='foo', nargs=None
>>> parser.parse_args(['--foo', '0xFA'])
Namespace(foo=250)
>>> parser.parse_args(['--foo', '1.2'])
usage: PROG [-h] [--foo FOO]
PROG: error: argument --foo: invalid 'hexadecimal integer' va
```

Exceções

exception argparse.ArgumentError

Um erro ao criar ou usar um argumento (opcional ou posicional).

O valor da string dessa exceção é a mensagem, complementada com informações sobre o argumento que a causou.

exception argparse.ArgumentTypeError

Levantada quando algo dá errado ao converter uma string de linha de comando em um tipo.

Guias e tutoriais

- Tutorial de Argparse
- Migrando código **`optparse`** para **`argparse`**

Tutorial de Argparse

Este tutorial pretende ser uma introdução gentil ao **argparse** — o módulo recomendado na biblioteca padrão do Python para fazer a análise de linha de comando.

Nota

A biblioteca padrão inclui duas outras bibliotecas diretamente relacionadas ao processamento de parâmetros de linha de comando: o módulo de nível inferior **optparse** (que pode exigir mais código para configurar uma determinada aplicação, mas também permite que uma aplicação solicite comportamentos que `argparse` não suporta), e o nível muito baixo **getopt** (que serve especificamente como um equivalente à família de funções **`getopt()`** disponível para programadores C). Embora nenhum desses módulos seja abordado diretamente neste guia, muitos dos conceitos principais em `argparse` se originaram primeiro em `optparse`, então alguns aspectos deste tutorial também serão relevantes para usuários de `optparse`.

Conceitos

Demonstraremos o tipo de funcionalidade que vamos explorar neste tutorial introdutório fazendo uso do comando **ls**:

```
$ ls
cpython  devguide  prog.py  pypy  rm-unused-function.patch
$ ls pypy
ctypes_configure  demo  dotviewer  include  lib_pypy  lib-pythor
$ ls -l
total 20
drwxr-xr-x 19 wena wena 4096 Feb 18 18:51 cpython
drwxr-xr-x  4 wena wena 4096 Feb  8 12:04 devguide
-rwxr-xr-x  1 wena wena  535 Feb 19 00:05 prog.py
```

```
drwxr-xr-x 14 wena wena 4096 Feb  7 00:59 pypy
-rw-r--r--  1 wena wena  741 Feb 18 01:01 rm-unused-function.pat
$ ls --help
Uso: ls [OPÇÃO]... [ARQUIVO]...
Lista informações sobre os ARQUIVOs (no diretório atual por padr
Lista as entradas em ordem alfabética se não for usada nenhuma c
nem --sort..
...
```

Alguns conceitos que podemos aprender a partir destes quatro comandos:

- O comando **ls** é útil quando usado sem nenhuma opção. Por padrão, ele mostra o conteúdo do diretório atual.
- Se quisermos além do que ele fornece por padrão, contamos um pouco mais. Neste caso, queremos que ele exiba um diretório diferente, `pypy`. O que fizemos foi especificar o que é conhecido como argumento posicional. Ele é chamado assim porque o programa deve saber o que fazer com o valor, apenas com base em onde ele aparece na linha de comando. Este conceito é mais relevante para um comando como **cp**, cujo uso mais básico é `cp SRC DEST`. A primeira posição é *o que você deseja copiar* e a segunda posição é *para onde você deseja copiar*.
- Agora, digamos que queremos mudar o comportamento do programa. Em nosso exemplo, exibimos mais informações para cada arquivo em vez de apenas mostrar os nomes dos arquivos. O `-l` nesse caso é conhecido como um argumento opcional.
- Esse é um trecho do texto de ajuda. É muito útil que possas encontrar um programa que nunca usastes antes e poder descobrir como o mesmo funciona simplesmente lendo o seu texto de ajuda.

O básico

Comecemos com um exemplo muito simples que irá fazer (quase) nada:

```
import argparse
parser = argparse.ArgumentParser()
parser.parse_args()
```

A seguir, temos o resultado da execução do código:

```
$ python prog.py
$ python prog.py --help
usage: prog.py [-h]

options:
  -h, --help  show this help message and exit
$ python prog.py --verbose
usage: prog.py [-h]
prog.py: error: unrecognized arguments: --verbose
$ python prog.py foo
usage: prog.py [-h]
prog.py: error: unrecognized arguments: foo
```

Eis aqui o que está acontecendo:

- Executar o script sem qualquer opção resultará que nada será exibido em stdout. Isso não é útil.
- O segundo começa a exibir as utilidades do módulo **`argparse`**. Não fizemos quase nada, mas já recebemos uma boa mensagem de ajuda.
- A opção `--help`, que também pode ser encurtada para `-h`, é a única opção que obtemos livremente (ou seja, não é necessário determiná-la). Determinar qualquer outra coisa resulta num erro. Mas mesmo assim, recebemos uma mensagem de uso bastante útil, também de graça.

Apresentando os argumentos posicionais

Um exemplo:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("echo")
args = parser.parse_args()
print(args.echo)
```

E executando o código:

```
$ python prog.py
usage: prog.py [-h] echo
prog.py: error: the following arguments are required: echo
```

```
$ python prog.py --help
usage: prog.py [-h] echo

positional arguments:
  echo

options:
  -h, --help  show this help message and exit
$ python prog.py foo
foo
```

Aqui está o que acontecerá:

- Nós adicionamos o método `add_argument()`, cujo o mesmo usamos para especificar quais opções de linha de comando o programa está disposto a aceitar. Neste caso, eu o nomeei `echo` para que ele esteja de acordo com sua função.
- Chamar o nosso programa neste momento, requer a especificação de uma opção.
- O método `parse_args()` realmente retorna alguns dados das opções especificadas, neste caso, `echo`.
- A variável é uma forma de "mágica" que `argparse` executa de brinde (ou seja, não é necessário especificar em qual variável esse valor é armazenado). Você também notará que seu nome corresponde ao argumento string dado ao método, `echo`.

Observe, no entanto, que, embora a tela de ajuda pareça boa e tudo, atualmente não é tão útil quanto poderia ser. Por exemplo, vemos que temos `echo` como um argumento posicional, mas não sabemos o que ele faz, além de adivinhar ou ler o código-fonte. Então, vamos torná-lo um pouco mais útil:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("echo", help="echo the string you use here")
args = parser.parse_args()
print(args.echo)
```

E, iremos obter:

```
$ python prog.py -h
usage: prog.py [-h] echo

positional arguments:
  echo        echo the string you use here

options:
  -h, --help  show this help message and exit
```

Agora, que tal fazer algo ainda mais útil:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", help="display a square of a given
args = parser.parse_args()
print(args.square**2)
```

A seguir, temos o resultado da execução do código:

```
$ python prog.py 4
Traceback (most recent call last):
  File "prog.py", line 5, in <module>
    print(args.square**2)
TypeError: unsupported operand type(s) for ** or pow(): 'str' ar
```

Isso não correu tão bem. Isso porque **argparse** trata as opções que damos a ele como strings, a menos que digamos o contrário. Então, vamos dizer ao **argparse** para tratar essa entrada como um inteiro:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", help="display a square of a given
                    type=int)
args = parser.parse_args()
print(args.square**2)
```

A seguir, temos o resultado da execução do código:

```
$ python prog.py 4
16
```

```
$ python prog.py four
usage: prog.py [-h] square
prog.py: error: argument square: invalid int value: 'four'
```

Correu bem. O programa agora até fecha com ajuda de entrada ilegal ruim antes de prosseguir.

Apresentando os argumentos opcionais

Até agora, jogamos com argumentos posicionais. Vamos dar uma olhada em como adicionar opcionais:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--verbosity", help="increase output verbosi
args = parser.parse_args()
if args.verbosity:
    print("verbosity turned on")
```

E a saída:

```
$ python prog.py --verbosity 1
verbosity turned on
$ python prog.py
$ python prog.py --help
usage: prog.py [-h] [--verbosity VERBOSITY]

options:
  -h, --help            show this help message and exit
  --verbosity VERBOSITY
                        increase output verbosity
$ python prog.py --verbosity
usage: prog.py [-h] [--verbosity VERBOSITY]
prog.py: error: argument --verbosity: expected one argument
```

Eis aqui o que está acontecendo:

- O programa é escrito de forma a exibir algo quando `--verbosity` é especificado e não exibir nada quando não for.

- Para mostrar que a opção é realmente opcional, não há erro ao executar o programa sem ela. Observe que, por padrão, se um argumento opcional não for usado, a variável relevante, neste caso `args.verbosity`, recebe `None` como valor, razão pela qual falha no teste de verdade da instrução `if`.
- A mensagem de ajuda é um pouco diferente.
- Ao usar a opção `--verbosity`, deve-se também especificar algum valor, qualquer valor.

O exemplo acima aceita valores inteiros arbitrários para `--verbosity`, mas para nosso programa simples, apenas dois valores são realmente úteis, `True` ou `False`. Vamos modificar o código de acordo:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("--verbose", help="increase output verbosity
                    action="store_true")
args = parser.parse_args()
if args.verbose:
    print("verbosity turned on")
```

E a saída:

```
$ python prog.py --verbose
verbosity turned on
$ python prog.py --verbose 1
usage: prog.py [-h] [--verbose]
prog.py: error: unrecognized arguments: 1
$ python prog.py --help
usage: prog.py [-h] [--verbose]

options:
  -h, --help  show this help message and exit
  --verbose   increase output verbosity
```

Eis aqui o que está acontecendo:

- A opção agora é mais um sinalizador do que algo que requer um valor. Até mudamos o nome da opção para corresponder a essa ideia. Observe que agora especificamos uma nova palavra reservada, `action`,

e damos a ela o valor `"store_true"`. Isso significa que, se a opção for especificada, atribui o valor `True` para `args.verbose`. Não especificá-la implica em `False`.
- Ele reclama quando você especifica um valor, no verdadeiro espírito do que os sinalizadores realmente são.
- Observe o texto de ajuda diferente.

Opções curtas

Se você estiver familiarizado com o uso da linha de comando, notará que ainda não toquei no tópico das versões curtas das opções. É bem simples:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("-v", "--verbose", help="increase output ver
                    action="store_true")
args = parser.parse_args()
if args.verbose:
    print("verbosity turned on")
```

E aqui vai:

```
$ python prog.py -v
verbosity turned on
$ python prog.py --help
usage: prog.py [-h] [-v]

options:
  -h, --help     show this help message and exit
  -v, --verbose  increase output verbosity
```

Observe que a nova habilidade também é refletida no texto de ajuda.

Combinando argumentos posicionais e opcionais

Nosso programa continua crescendo em complexidade:

```
import argparse
parser = argparse.ArgumentParser()
```

```
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbose", action="store_true",
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbose:
    print(f"the square of {args.square} equals {answer}")
else:
    print(answer)
```

E agora a saída:

```
$ python prog.py
usage: prog.py [-h] [-v] square
prog.py: error: the following arguments are required: square
$ python prog.py 4
16
$ python prog.py 4 --verbose
the square of 4 equals 16
$ python prog.py --verbose 4
the square of 4 equals 16
```

- Trouxemos de volta um argumento posicional, daí a reclamação.
- Observe que a ordem não importa.

Que tal devolvermos a este nosso programa a capacidade de ter vários valores de verbosidade e realmente usá-los:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", type=int,
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity == 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity == 1:
    print(f"{args.square}^2 == {answer}")
```

```
else:
    print(answer)
```

E a saída:

```
$ python prog.py 4
16
$ python prog.py 4 -v
usage: prog.py [-h] [-v VERBOSITY] square
prog.py: error: argument -v/--verbosity: expected one argument
$ python prog.py 4 -v 1
4^2 == 16
$ python prog.py 4 -v 2
the square of 4 equals 16
$ python prog.py 4 -v 3
16
```

Todos eles parecem bons, exceto o último, que expõe um bug em nosso programa. Vamos corrigi-lo restringindo os valores que a opção `--verbosity` pode aceitar:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", type=int, choices=[0, 1
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity == 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity == 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

E a saída:

```
$ python prog.py 4 -v 3
usage: prog.py [-h] [-v {0,1,2}] square
prog.py: error: argument -v/--verbosity: invalid choice: 3 (choc
```

```
$ python prog.py 4 -h
usage: prog.py [-h] [-v {0,1,2}] square

positional arguments:
  square                display a square of a given number

options:
  -h, --help            show this help message and exit
  -v, --verbosity {0,1,2}
                        increase output verbosity
```

Observe que a alteração também reflete tanto na mensagem de erro quanto na string de ajuda.

Agora, vamos usar uma abordagem diferente de brincar com a verbosidade, que é bastante comum. Ele também corresponde à maneira como o executável do CPython trata seu próprio argumento de verbosidade (verifique a saída de `python --help`):

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display the square of a given number")
parser.add_argument("-v", "--verbosity", action="count",
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
if args.verbosity == 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity == 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

Introduzimos outra ação, “contar”, para contar o número de ocorrências de opções específicas.

```
$ python prog.py 4
16
$ python prog.py 4 -v
```

```
4^2 == 16
$ python prog.py 4 -vv
the square of 4 equals 16
$ python prog.py 4 --verbosity --verbosity
the square of 4 equals 16
$ python prog.py 4 -v 1
usage: prog.py [-h] [-v] square
prog.py: error: unrecognized arguments: 1
$ python prog.py 4 -h
usage: prog.py [-h] [-v] square

positional arguments:
  square           display a square of a given number

options:
  -h, --help       show this help message and exit
  -v, --verbosity  increase output verbosity
$ python prog.py 4 -vvv
16
```

- Sim, agora é mais um sinalizador (semelhante a `action="store_true"`) na versão anterior do nosso script. Isso deve explicar a reclamação.
- Ele também se comporta de maneira semelhante à ação “store_true”.
- Agora aqui está uma demonstração do que a ação “contar” oferece. Você provavelmente já viu esse tipo de uso antes.
- E se você não especificar o sinalizador `-v`, esse sinalizador será considerado como tendo valor `None`.
- Como deve ser esperado, especificando a forma longa do sinalizador, devemos obter a mesma saída.
- Infelizmente, nossa saída de ajuda não é muito informativa sobre a nova habilidade que nosso script adquiriu, mas isso sempre pode ser corrigido melhorando a documentação de nosso script (por exemplo, através do argumento nomeado `help`).
- Essa última saída expõe um bug em nosso programa.

Vamos corrigir:

```
import argparse
parser = argparse.ArgumentParser()
```

```
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", action="count",
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2

# bugfix: replace == with >=
if args.verbosity >= 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity >= 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

E isso aqui é o mesmo retorna:

```
$ python prog.py 4 -vvv
the square of 4 equals 16
$ python prog.py 4 -vvvv
the square of 4 equals 16
$ python prog.py 4
Traceback (most recent call last):
  File "prog.py", line 11, in <module>
    if args.verbosity >= 2:
TypeError: '>=' not supported between instances of 'NoneType' ar
```

- A primeira saída correu bem e corrige o bug que tínhamos antes. Ou seja, queremos que qualquer valor >= 2 seja o mais detalhado possível.
- A terceira saída não está tão boa.

Vamos corrigir esse bug:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("square", type=int,
                    help="display a square of a given number")
parser.add_argument("-v", "--verbosity", action="count", default
                    help="increase output verbosity")
args = parser.parse_args()
answer = args.square**2
```

```
if args.verbosity >= 2:
    print(f"the square of {args.square} equals {answer}")
elif args.verbosity >= 1:
    print(f"{args.square}^2 == {answer}")
else:
    print(answer)
```

Acabamos de introduzir outra palavra reservada, `default`. Nós o configuramos como `0` para torná-lo comparável aos outros valores int. Lembre-se que por padrão, se um argumento opcional não for especificado, ele obtém o valor `None`, e isso não pode ser comparado a um valor int (daí a exceção **`TypeError`**).

E:

```
$ python prog.py 4
16
```

Você pode ir muito longe apenas com o que aprendemos até agora, e nós apenas arranhamos a superfície. O módulo **`argparse`** é muito poderoso, e vamos explorar um pouco mais antes de terminar este tutorial.

Avançando um pouco mais

E se quiséssemos expandir nosso pequeno programa, ampliando seu potencial:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
parser.add_argument("-v", "--verbosity", action="count", default
args = parser.parse_args()
answer = args.x**args.y
if args.verbosity >= 2:
    print(f"{args.x} to the power {args.y} equals {answer}")
elif args.verbosity >= 1:
    print(f"{args.x}^{args.y} == {answer}")
```

```
else:
    print(answer)
```

Saída:

```
$ python prog.py
usage: prog.py [-h] [-v] x y
prog.py: error: the following arguments are required: x, y
$ python prog.py -h
usage: prog.py [-h] [-v] x y

positional arguments:
  x                 the base
  y                 the exponent

options:
  -h, --help        show this help message and exit
  -v, --verbosity
$ python prog.py 4 2 -v
4^2 == 16
```

Observe que até agora estamos usando o nível de verbosidade para *alterar* o texto que é exibido. O exemplo a seguir usa o nível de verbosidade para exibir *mais* texto:

```
import argparse
parser = argparse.ArgumentParser()
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
parser.add_argument("-v", "--verbosity", action="count", default
args = parser.parse_args()
answer = args.x**args.y
if args.verbosity >= 2:
    print(f"Running '{__file__}'")
if args.verbosity >= 1:
    print(f"{args.x}^{args.y} == ", end="")
print(answer)
```

Saída:

```
$ python prog.py 4 2
16
$ python prog.py 4 2 -v
4^2 == 16
$ python prog.py 4 2 -vv
Running 'prog.py'
4^2 == 16
```

Especificando argumentos ambíguos

Quando há ambiguidade em decidir se um argumento é posicional ou para um argumento, `--` pode ser usado para dizer `parse_args()` que tudo depois disso é um argumento posicional:

```
>>> parser = argparse.ArgumentParser(prog='PROG')
>>> parser.add_argument('-n', nargs='+')
>>> parser.add_argument('args', nargs='*')

>>> # ambiguous, so parse_args assumes it's an option
>>> parser.parse_args(['-f'])
usage: PROG [-h] [-n N [N ...]] [args ...]
PROG: error: unrecognized arguments: -f

>>> parser.parse_args(['--', '-f'])
Namespace(args=['-f'], n=None)

>>> # ambiguous, so the -n option greedily accepts arguments
>>> parser.parse_args(['-n', '1', '2', '3'])
Namespace(args=[], n=['1', '2', '3'])

>>> parser.parse_args(['-n', '1', '--', '2', '3'])
Namespace(args=['2', '3'], n=['1'])
```

Opções conflitantes

Até agora, trabalhamos com dois métodos de uma instância `argparse.ArgumentParser`. Vamos apresentar um terceiro, `add_mutually_exclusive_group()`. Ele nos permite especificar opções que entram em conflito umas com as outras. Vamos também alterar o resto do

programa para que a nova funcionalidade faça mais sentido: vamos introduzir a opção `--quiet`, que será o oposto da opção `--verbose`:

```
import argparse

parser = argparse.ArgumentParser()
group = parser.add_mutually_exclusive_group()
group.add_argument("-v", "--verbose", action="store_true")
group.add_argument("-q", "--quiet", action="store_true")
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
args = parser.parse_args()
answer = args.x**args.y

if args.quiet:
    print(answer)
elif args.verbose:
    print(f"{args.x} to the power {args.y} equals {answer}")
else:
    print(f"{args.x}^{args.y} == {answer}")
```

Nosso programa agora está mais simples e perdemos algumas funcionalidades para demonstração. De qualquer forma, aqui está a saída:

```
$ python prog.py 4 2
4^2 == 16
$ python prog.py 4 2 -q
16
$ python prog.py 4 2 -v
4 to the power 2 equals 16
$ python prog.py 4 2 -vq
usage: prog.py [-h] [-v | -q] x y
prog.py: error: argument -q/--quiet: not allowed with argument -
$ python prog.py 4 2 -v --quiet
usage: prog.py [-h] [-v | -q] x y
prog.py: error: argument -q/--quiet: not allowed with argument -
```

Isso deve ser fácil de seguir. Eu adicionei essa última saída para que você possa ver o tipo de flexibilidade que obtém, ou seja, misturar opções de formato longo com formatos curtos.

Antes de concluirmos, você provavelmente quer dizer aos seus usuários o propósito principal do seu programa, caso eles não saibam:

```
import argparse

parser = argparse.ArgumentParser(description="calculate X to the
group = parser.add_mutually_exclusive_group()
group.add_argument("-v", "--verbose", action="store_true")
group.add_argument("-q", "--quiet", action="store_true")
parser.add_argument("x", type=int, help="the base")
parser.add_argument("y", type=int, help="the exponent")
args = parser.parse_args()
answer = args.x**args.y

if args.quiet:
    print(answer)
elif args.verbose:
    print(f"{args.x} to the power {args.y} equals {answer}")
else:
    print(f"{args.x}^{args.y} == {answer}")
```

Observe essa pequena diferença no texto de uso. Observe o `[-v | -q]`, que nos diz que podemos usar `-v` ou `-q`, mas não ambos ao mesmo tempo:

```
$ python prog.py --help
usage: prog.py [-h] [-v | -q] x y

calculate X to the power of Y

positional arguments:
  x              the base
  y              the exponent

options:
  -h, --help     show this help message and exit
  -v, --verbose
  -q, --quiet
```

Como traduzir a saída do argparse

A saída do módulo **argparse**, como seu texto de ajuda e mensagens de erro, são todos traduzidos usando o módulo **gettext**. Isso permite que os aplicativos localizem facilmente as mensagens produzidas por **argparse**. Veja também Internacionalizando seus programas e módulos.

Por exemplo, nesta saída **argparse**:

```
$ python prog.py --help
usage: prog.py [-h] [-v | -q] x y

calculate X to the power of Y

positional arguments:
  x              the base
  y              the exponent

options:
  -h, --help     show this help message and exit
  -v, --verbose
  -q, --quiet
```

As strings `usage:`, `positional arguments:`, `options:` e `show this help message and exit` são todos traduzíveis.

Para traduzir essas strings, elas devem primeiro ser extraídas em um arquivo `.po`. Por exemplo, usando Babel [https://babel.pocoo.org/], execute este comando:

```
$ pybabel extract -o messages.po /usr/lib/python3.12/argparse.py
```

Este comando extrairá todas as strings traduzíveis do módulo **argparse** e as enviará para um arquivo chamado `messages.po`. Este comando presume que sua instalação do Python está em `/usr/lib`.

Você pode descobrir a localização do módulo **argparse** em seu sistema usando este script:

```
import argparse
print(argparse.__file__)
```

Uma vez que as mensagens no arquivo `.po` sejam traduzidas e as traduções instaladas usando `gettext`, `argparse` será capaz de exibir as mensagens traduzidas.

Para traduzir suas próprias strings na saída `argparse`, use `gettext`.

Conversores de tipo personalizados

O módulo `argparse` permite que você especifique conversores de tipo personalizados para seus argumentos de linha de comando. Isso permite modificar a entrada de usuário antes de armazená-la na `argparse.Namespace`. Isso pode ser útil quando você precisa pré-processar a entrada antes dela ser usada em seu programa.

Ao usar um conversor de tipo personalizado, você pode usar qualquer chamável que recebe uma única string como argumento (o valor do argumento) e retorna o valor convertido. Porém, se você precisa tratar de cenários mais complexos, você pode usar uma classe de ação personalizada com o parâmetro **action**.

Por exemplo, digamos que vcoê deseja tratar de argumentos com diferentes prefixos e processá-los adequadamente:

```
import argparse

parser = argparse.ArgumentParser(prefix_chars='-+')

parser.add_argument('-a', metavar='<value>', action='append',
                    type=lambda x: ('-', x))
parser.add_argument('+a', metavar='<value>', action='append',
                    type=lambda x: ('+', x))

args = parser.parse_args()
print(args)
```

Saída:

```
$ python prog.py -a valor1 +a valor2
Namespace(a=[('-', 'valor1'), ('+', 'valor2')])
```

Neste exemplos, nós:

- Criamos um analisador sintático com caracteres de prefixo personalizado usando parâmetro `prefix_chars`.
- Definimos dois argumentos, `-a` e `+a`, que usaram o parâmetro `type` para criar conversores de tipo personalizados para armazenar o valor em uma tupla com o prefixo.

Sem os conversores de tipo personalizados, os argumentos tratariam `-a` e `+a` como o mesmo argumento, o que não era o desejado. Ao usar conversores de tipo personalizados, conseguimos diferenciar entre dois argumentos.

Conclusão

O módulo **`argparse`** oferece muito mais do que o mostrado aqui. Sua documentação é bastante detalhada e completo, e cheia de exemplos. Tendo passado por este tutorial, você deve digeri-la facilmente sem se sentir sobrecarregado.

Migrando código `optparse` para `argparse`

O módulo **`argparse`** oferece vários recursos de nível mais alto não fornecidos nativamente pelo módulo **`optparse`**, incluindo:

- Tratando argumentos posicionais.
- Prover suporte a subcomandos.
- Permitir prefixos alternativos de opções como `+` e `/`.
- Manipular argumentos de estilo “zero ou mais” e “um ou mais”.
- Produzir mensagens de uso mais informativas.
- Fornecer uma interface muito mais simples para `type` e `action` personalizados.

Originalmente, o módulo **`argparse`** tentou manter a compatibilidade com **`optparse`**. No entanto, as diferenças fundamentais de design entre suportar o processamento de opções de linha de comando declarativas (enquanto deixa o processamento de argumentos posicionais para o código da aplicação) e suportar opções nomeadas e argumentos posicionais na interface declarativa significam que a API divergiu daquela de `optparse` ao longo do tempo.

Conforme descrito em Escolhendo uma biblioteca de análise de linha de comando, as aplicações que atualmente usam **`optparse`** e estão satisfeitos com a forma como ele funciona podem continuar usando `optparse`.

Os desenvolvedores de aplicações que estão pensando em migrar também devem revisar a lista de diferenças comportamentais intrínsecas descritas nessa seção antes de decidir se a migração é desejável ou não.

Para aplicações que optam por migrar de **`optparse`** para **`argparse`**, as seguintes sugestões devem ser úteis:

- Substituir todas as chamadas de `optparse.OptionParser.add_option()` por chamadas de `ArgumentParser.add_argument()`.
- Substituir `(options, args) = parser.parse_args()` por `args = parser.parse_args()` e adicionar chamadas adicionais a `ArgumentParser.add_argument()` para os argumentos posicionais. Tenha em mente que o que anteriormente era chamado de `options`, agora no contexto do `argparse` é chamado de `args`.
- Substituir `optparse.OptionParser.disable_interspersed_args()` usando `parse_intermixed_args()` em vez de `parse_args()`.
- Substituir ações de função de retorno e argumentos nomeados `callback_*` por argumentos `type` ou `action`.
- Substituir nomes de strings para argumentos nomeados `type` pelos objetos de tipo correspondentes (por exemplo, int, float, complex, etc).
- Substituir `optparse.Values` por `Namespace` e `optparse.OptionError` e `optparse.OptionValueError` por `ArgumentError`.
- Substituir strings com argumentos implícitos tal como `%default` ou `%prog` pela sintaxe padrão do Python para usar dicionários para formatar strings, ou seja, `%(default)s` e `%(prog)s`.
- Substituir o argumento `version` do construtor do OptionParser por uma chamada a `parser.add_argument('--version', action='version', version='<the version>')`.

`optparse` — Analisador sintático para opções de linha de comando

Código-fonte: Lib/optparse.py
[https://github.com/python/cpython/tree/3.15/Lib/optparse.py]

Escolhendo uma biblioteca de análise de linha de comando

A biblioteca padrão inclui três bibliotecas de análise de argumentos:

- `getopt`: a module that closely mirrors the procedural C `getopt` API.
- **`optparse`**: a declarative replacement for `getopt` that provides equivalent functionality without requiring each application to implement its own procedural option parsing logic.
- `argparse`: a more opinionated alternative to `optparse` that provides more functionality by default, at the expense of reduced application flexibility in controlling exactly how arguments are processed.

Na ausência de restrições de design de análise de argumentos mais específicas, `argparse` é a escolha recomendada para implementar aplicações de linha de comando, pois oferece o mais alto nível de funcionalidade básica com o mínimo de código em nível de aplicação.

`getopt` é mantido quase inteiramente por razões de retrocompatibilidade. No entanto, ele também serve como um caso de uso de nicho como uma ferramenta para prototipagem e teste de manipulação de argumentos de linha de comando em aplicações C baseadas em `getopt`.

`optparse` should be considered as an alternative to `argparse` in the following cases:

- an application is already using **`optparse`** and doesn't want to risk the subtle behavioural changes that may arise when migrating to **`argparse`**
- a aplicação requer controle adicional sobre a maneira como as opções e os parâmetros posicionais são intercalados na linha de comando (incluindo a capacidade de desabilitar completamente o recurso de intercalação)
- a aplicação requer controle adicional sobre a análise incremental de elementos de linha de comando (embora `argparse` suporte isso, a maneira exata como ele funciona na prática é indesejável para alguns casos de uso)
- a aplicação requer controle adicional sobre o tratamento de opções que aceitam valores de parâmetros que podem começar com `-` (como opções delegadas a serem passadas para subprocessos invocados)
- a aplicação requer algum outro comportamento de processamento de parâmetros de linha de comando que `argparse` não suporta, mas que pode ser implementado em termos da interface de nível inferior oferecida por `optparse`

These considerations also mean that **`optparse`** is likely to provide a better foundation for library authors writing third party command line argument processing libraries.

Como exemplo concreto, considere as duas configurações de análise de argumentos de linha de comando a seguir, a primeira usando `optparse` e a segunda usando `argparse`:

```
import optparse

if __name__ == '__main__':
    parser = optparse.OptionParser()
    parser.add_option('-o', '--output')
    parser.add_option('-v', dest='verbose', action='store_true')
    opts, args = parser.parse_args()
    process(args, output=opts.output, verbose=opts.verbose)
```

```
import argparse

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
```

```
parser.add_argument('-o', '--output')
parser.add_argument('-v', dest='verbose', action='store_true
parser.add_argument('rest', nargs='*')
args = parser.parse_args()
process(args.rest, output=args.output, verbose=args.verbose)
```

A diferença mais óbvia é que na versão `optparse`, os argumentos não opcionais são processados separadamente pelo aplicativo após o processamento da opção ser concluído. Na versão `argparse`, os argumentos posicionais são declarados e processados da mesma forma que as opções nomeadas.

No entanto, a versão `argparse` também manipulará algumas combinações de parâmetros de forma diferente da forma como a versão `optparse` as manipularia. Por exemplo (entre outras diferenças):

- fornecer `-o -v` retorna `output="-v"` e `verbose=False` ao usar `optparse`, mas um erro de uso com `argparse` (reclamando que nenhum valor foi fornecido para `-o/--output`, já que `-v` é interpretado como significando o sinalizador de verbosidade)
- da mesma forma, fornecer `-o --` retorna `output="--"` e `args=()` ao usar `optparse`, mas um erro de uso com `argparse` (também reclamando que nenhum valor foi fornecido para `-o/--output`, já que `--` é interpretado como o término do processamento da opção e o tratamento de todos os valores restantes como argumentos posicionais)
- fornecer `-o=foo` retorna `output="=foo"` ao usar `optparse`, mas retorna `output="foo"` com `argparse` (já que `=` é um caso especial como um separador alternativo para valores de parâmetros de opção)

Se esses comportamentos diferentes na versão `argparse` são considerados desejáveis ou um problema dependerá do caso de uso específico da aplicação de linha de comando.

Ver também

click [https://pypi.org/project/click/] é uma biblioteca de processamento de argumentos de terceiros (originalmente baseada em `optparse`), que

permite que aplicações de linha de comando sejam desenvolvidas como um conjunto de funções de implementação de comando decoradas.

Outras bibliotecas de terceiros, como typer [https://pypi.org/project/typer/] ou msgspec-click [https://pypi.org/project/msgspec-click/], permitem que interfaces de linha de comando sejam especificadas de maneiras que se integrem de forma mais eficaz com a verificação estática de anotações de tipo Python.

Introdução

`optparse` is a more convenient, flexible, and powerful library for parsing command-line options than the minimalist **`getopt`** module. **`optparse`** uses a more declarative style of command-line parsing: you create an instance of **`OptionParser`**, populate it with options, and parse the command line. **`optparse`** allows users to specify options in the conventional GNU/POSIX syntax, and additionally generates usage and help messages for you.

Here's an example of using **`optparse`** in a simple script:

```
from optparse import OptionParser
...
parser = OptionParser()
parser.add_option("-f", "--file", dest="filename",
                  help="write report to FILE", metavar="FILE")
parser.add_option("-q", "--quiet",
                  action="store_false", dest="verbose", default=
                  help="don't print status messages to stdout")

(options, args) = parser.parse_args()
```

Com essas poucas linhas de código, os usuários do seu script agora podem fazer "coisas comuns" na linha de comando, por exemplo:

```
<seuscript> --file=outfile -q
```

As it parses the command line, **`optparse`** sets attributes of the `options` object returned by **`parse_args()`** based on user-supplied command-line

values. When **parse_args()** returns from parsing this command line, options.filename will be "outfile" and options.verbose will be False. **optparse** supports both long and short options, allows short options to be merged together, and allows options to be associated with their arguments in a variety of ways. Thus, the following command lines are all equivalent to the above example:

```
<seuscript> -f outfile --quiet
<seuscript> --quiet --file outfile
<seuscript> -q -foutfile
<seuscript> -qfoutfile
```

Além disso, os usuários podem executar um dos seguintes:

```
<seuscript> -h
<seuscript> --help
```

and **optparse** will print out a brief summary of your script's options:

```
Usage: <seuscript> [options]

Options:
  -h, --help            show this help message and exit
  -f FILE, --file=FILE  write report to FILE
  -q, --quiet           don't print status messages to stdout
```

onde o valor de *seuscript* é determinado em tempo de execução (normalmente de sys.argv[0]).

Contexto

optparse was explicitly designed to encourage the creation of programs with straightforward command-line interfaces that follow the conventions established by the **getopt()** family of functions available to C developers. To that end, it supports only the most common command-line syntax and semantics conventionally used under Unix. If you are unfamiliar with these conventions, reading this section will allow you to acquaint yourself with them.

Terminologia

argumento

uma string inserida na linha de comando e passada pelo shell para `execl()` ou `execv()`. Em Python, argumentos são elementos de `sys.argv[1:]` (`sys.argv[0]` é o nome do programa que está sendo executado). Shells Unix também usam o termo “word”.

Ocasionalmente, é desejável substituir uma lista de argumentos diferente de `sys.argv[1:]`, então você deve ler “argumento” como “um elemento de `sys.argv[1:]`, ou de alguma outra lista fornecida como um substituto para `sys.argv[1:]`”.

opção

an argument used to supply extra information to guide or customize the execution of a program. There are many different syntaxes for options; the traditional Unix syntax is a hyphen (“-”) followed by a single letter, e.g. `-x` or `-F`. Also, traditional Unix syntax allows multiple options to be merged into a single argument, e.g. `-x -F` is equivalent to `-xF`. The GNU project introduced `--` followed by a series of hyphen-separated words, e.g. `--file` or `--dry-run`. These are the only two option syntaxes provided by **`optparse`**.

Algumas outras sintaxes de opções que o mundo viu incluem:

- um hífen seguido por algumas letras, por exemplo `-pf` (isso *não* é o mesmo que várias opções mescladas em um único argumento)
- um hífen seguido por uma palavra completa, por exemplo `-file` (isso é tecnicamente equivalente à sintaxe anterior, mas eles geralmente não são vistos no mesmo programa)
- um sinal de mais seguido por uma única letra, ou algumas letras, ou uma palavra, por exemplo `+f`, `+rgb`
- uma barra seguida de uma letra, ou algumas letras, ou uma palavra, por exemplo `/f`, `/file`

These option syntaxes are not supported by **`optparse`**, and they never will be. This is deliberate: the first three are non-standard on any

environment, and the last only makes sense if you're exclusively targeting Windows or certain legacy platforms (e.g. VMS, MS-DOS).

argumento de opção

an argument that follows an option, is closely associated with that option, and is consumed from the argument list when that option is. With **`optparse`**, option arguments may either be in a separate argument from their option:

```
-f foo
--file foo
```

ou incluídos no mesmo argumento:

```
-ffoo
--file=foo
```

Typically, a given option either takes an argument or it doesn't. Lots of people want an "optional option arguments" feature, meaning that some options will take an argument if they see it, and won't if they don't. This is somewhat controversial, because it makes parsing ambiguous: if `-a` takes an optional argument and `-b` is another option entirely, how do we interpret `-ab`? Because of this ambiguity, **`optparse`** does not support this feature.

argumento posicional

algo que sobrou na lista de argumentos depois que as opções foram analisadas, ou seja, depois que as opções e seus argumentos foram analisados e removidos da lista de argumentos.

opção obrigatória

an option that must be supplied on the command-line; note that the phrase "required option" is self-contradictory in English. **`optparse`** doesn't prevent you from implementing required options, but doesn't give you much help at it either.

Por exemplo, considere esta linha de comando hipotética:

```
prog -v --report report.txt foo bar
```

`-v` e `--report` são ambas opções. Supondo que `--report` receba um argumento, `report.txt` é um argumento de opção. `foo` e `bar` são argumentos posicionais.

Para que servem as opções?

As opções são usadas para fornecer informações extras para ajustar ou personalizar a execução de um programa. Caso não tenha ficado claro, as opções geralmente são *opcionais*. Um programa deve ser capaz de executar muito bem sem nenhuma opção. (Escolha um programa aleatório dos conjuntos de ferramentas Unix ou GNU. Ele pode ser executado sem nenhuma opção e ainda fazer sentido? As principais exceções são `find`, `tar` e `dd` — todos eles são mutantes excêntricos que foram corretamente criticados por sua sintaxe não padrão e interfaces confusas.)

Muitas pessoas querem que seus programas tenham “opções obrigatórias”. Pense nisso. Se é obrigatório, então *não é opcional*! Se há uma informação que seu programa absolutamente requer para ser executado com sucesso, é para isso que servem os argumentos posicionais.

Como um exemplo de bom design de interface de linha de comando, considere o humilde utilitário `cp`, para copiar arquivos. Não faz muito sentido tentar copiar arquivos sem fornecer um destino e pelo menos uma fonte. Portanto, `cp` falha se você executá-lo sem argumentos. No entanto, ele tem uma sintaxe flexível e útil que não requer nenhuma opção:

```
cp ORIGEM DESTINO
cp ORIGEM ... DIRETÓRIO
```

Você pode ir bem longe só com isso. A maioria das implementações de `cp` fornece um monte de opções para ajustar exatamente como os arquivos são copiados: você pode preservar o modo e o tempo de modificação, evitar seguir links simbólicos, perguntar antes de destruir arquivos existentes, etc. Mas nada disso distrai da missão principal de `cp`, que é copiar um arquivo para outro, ou vários arquivos para outro diretório.

Para que servem os argumentos posicionais?

Argumentos posicionais são para aquelas informações que seu programa absolutamente e positivamente requer para ser executado.

Uma boa interface de usuário deve ter o mínimo de requisitos absolutos possível. Se seu programa requer 17 informações distintas para ser executado com sucesso, não importa muito *como* você obtém essas informações do usuário — a maioria das pessoas desistirá e irá embora antes de executar o programa com sucesso. Isso se aplica se a interface de usuário for uma linha de comando, um arquivo de configuração ou uma GUI: se você fizer tantas exigências aos seus usuários, a maioria deles simplesmente desistirá.

Em resumo, tente minimizar a quantidade de informações que os usuários são absolutamente obrigados a fornecer — use padrões sensatos sempre que possível. Claro, você também quer tornar seus programas razoavelmente flexíveis. É para isso que servem as opções. Novamente, não importa se são entradas em um arquivo de configuração, widgets no diálogo “Preferências” de uma GUI ou opções de linha de comando — quanto mais opções você implementar, mais flexível será seu programa e mais complicada se tornará sua implementação. Muita flexibilidade também tem desvantagens, é claro; muitas opções podem sobrecarregar os usuários e tornar seu código muito mais difícil de manter.

Tutorial

While **`optparse`** is quite flexible and powerful, it’s also straightforward to use in most cases. This section covers the code patterns that are common to any **`optparse`**-based program.

Primeiro, você precisa importar a classe OptionParser; então, no início do programa principal, crie uma instância de OptionParser:

```
from optparse import OptionParser
...
parser = OptionParser()
```

Então você pode começar a definir opções. A sintaxe básica é:

```
parser.add_option(opt_str, ...,
                  attr=value, ...)
```

Each option has one or more option strings, such as `-f` or `--file`, and several option attributes that tell **`optparse`** what to expect and what to do when it encounters that option on the command line.

Normalmente, cada opção terá uma sequência de opções curta e uma sequência de opções longa, por exemplo:

```
parser.add_option("-f", "--file", ...)
```

Você tem a liberdade de definir quantas strings de opções curtas e longas quiser (incluindo zero), desde que haja pelo menos uma string de opção no geral.

The option strings passed to **`OptionParser.add_option()`** are effectively labels for the option defined by that call. For brevity, we will frequently refer to *encountering an option* on the command line; in reality, **`optparse`** encounters *option strings* and looks up options from them.

Once all of your options are defined, instruct **`optparse`** to parse your program's command line:

```
(options, args) = parser.parse_args()
```

(Se desejar, você pode passar uma lista de argumentos personalizada para **`parse_args()`**, mas isso raramente é necessário: por padrão, ele usa `sys.argv[1:]`.)

`parse_args()` retorna dois valores:

- `options`, um objeto que contém valores para todas as suas opções — por exemplo, se `--file` receber um único argumento de string, então `options.file` será o nome do arquivo fornecido pelo usuário, ou `None` se o usuário não forneceu essa opção
- `args`, a lista de argumentos posicionais restantes após a análise de opções

Esta seção do tutorial abrange apenas os quatro atributos de opção mais importantes: **`action`**, **`type`**, **`dest`** (destino) e **`help`**. Destes, **`action`** é o mais fundamental.

Compreendendo as ações de opções

Actions tell **`optparse`** what to do when it encounters an option on the command line. There is a fixed set of actions hard-coded into **`optparse`**; adding new actions is an advanced topic covered in section Extending optparse. Most actions tell **`optparse`** to store a value in some variable—for example, take a string from the command line and store it in an attribute of `options`.

If you don't specify an option action, **`optparse`** defaults to `store`.

A ação store

The most common option action is `store`, which tells **`optparse`** to take the next argument (or the remainder of the current argument), ensure that it is of the correct type, and store it to your chosen destination.

Por exemplo:

```
parser.add_option("-f", "--file",
                  action="store", type="string", dest="filename'
```

Now let's make up a fake command line and ask **`optparse`** to parse it:

```
args = ["-f", "foo.txt"]
(options, args) = parser.parse_args(args)
```

When **`optparse`** sees the option string `-f`, it consumes the next argument, `foo.txt`, and stores it in `options.filename`. So, after this call to **`parse_args()`**, `options.filename` is `"foo.txt"`.

Some other option types supported by **`optparse`** are `int` and `float`. Here's an option that expects an integer argument:

```
parser.add_option("-n", type="int", dest="num")
```

Note que esta opção não tem uma string de opção longa, o que é perfeitamente aceitável. Além disso, não há nenhuma ação explícita, já que o padrão é `store`.

Vamos analisar outra linha de comando falsa. Desta vez, vamos colocar o argumento de opção bem perto da opção: já que `-n42` (um argumento) equivale a `-n 42` (dois argumentos), o código

```
(options, args) = parser.parse_args(["-n42"])
print(options.num)
```

vai exibir `42`.

If you don't specify a type, **`optparse`** assumes `string`. Combined with the fact that the default action is `store`, that means our first example can be a lot shorter:

```
parser.add_option("-f", "--file", dest="filename")
```

If you don't supply a destination, **`optparse`** figures out a sensible default from the option strings: if the first long option string is `--foo-bar`, then the default destination is `foo_bar`. If there are no long option strings, **`optparse`** looks at the first short option string: the default destination for `-f` is `f`.

`optparse` also includes the built-in `complex` type. Adding types is covered in section Extending optparse.

Manipulando opções (sinalizadores) booleanas

Flag options—set a variable to true or false when a particular option is seen—are quite common. **`optparse`** supports them with two separate actions, `store_true` and `store_false`. For example, you might have a `verbose` flag that is turned on with `-v` and off with `-q`:

```
parser.add_option("-v", action="store_true", dest="verbose")
parser.add_option("-q", action="store_false", dest="verbose")
```

Aqui temos duas opções diferentes com o mesmo destino, o que é perfeitamente OK. (Isso significa apenas que você tem que ter um pouco de cuidado ao definir valores padrão — veja abaixo.)

When **`optparse`** encounters `-v` on the command line, it sets `options.verbose` to `True`; when it encounters `-q`, `options.verbose` is set to `False`.

Outras ações

Some other actions supported by **`optparse`** are:

`"store_const"`
: armazena um valor constante, predefinido via **`Option.const`**

`"append"`
: anexa o argumento desta opção a uma lista

`"count"`
: incrementa um contador em um

`"callback"`
: chama uma função especificada

Elas são abordadas na seção Guia de referência e na seção Funções de retorno de opção.

Valores padrão

All of the above examples involve setting some variable (the “destination”) when certain command-line options are seen. What happens if those options are never seen? Since we didn’t supply any defaults, they are all set to `None`. This is usually fine, but sometimes you want more control. **`optparse`** lets you supply a default value for each destination, which is assigned before the command line is parsed.

First, consider the verbose/quiet example. If we want **`optparse`** to set `verbose` to `True` unless `-q` is seen, then we can do this:

```
parser.add_option("-v", action="store_true", dest="verbose", def
parser.add_option("-q", action="store_false", dest="verbose")
```

Como os valores padrão se aplicam ao *destino* e não a qualquer opção específica, e essas duas opções têm o mesmo destino, isso é exatamente equivalente:

```
parser.add_option("-v", action="store_true", dest="verbose")
parser.add_option("-q", action="store_false", dest="verbose", de
```

Considere isso:

```
parser.add_option("-v", action="store_true", dest="verbose", def
parser.add_option("-q", action="store_false", dest="verbose", de
```

Novamente, o valor padrão para `verbose` será `True`: o último valor padrão fornecido para qualquer destino específico é o que conta.

Uma maneira mais clara de especificar valores padrão é o método **`set_defaults()`** do OptionParser, que você pode chamar a qualquer momento antes de chamar **`parse_args()`**:

```
parser.set_defaults(verbose=True)
parser.add_option(...)
(options, args) = parser.parse_args()
```

Como antes, o último valor especificado para um destino de opção dado é o que conta. Para maior clareza, tente usar um método ou outro de configuração de valores padrão, não ambos.

Gerando texto de ajuda

`optparse`'s ability to generate help and usage text automatically is useful for creating user-friendly command-line interfaces. All you have to do is supply a **`help`** value for each option, and optionally a short usage message for your whole program. Here's an OptionParser populated with user-friendly (documented) options:

```
usage = "usage: %prog [options] arg1 arg2"
parser = OptionParser(usage=usage)
parser.add_option("-v", "--verbose",
                  action="store_true", dest="verbose", default=1
                  help="make lots of noise [default]")
parser.add_option("-q", "--quiet",
                  action="store_false", dest="verbose",
                  help="be vewwy quiet (I'm hunting wabbits)")
parser.add_option("-f", "--filename",
                  metavar="FILE", help="write output to FILE")
parser.add_option("-m", "--mode",
                  default="intermediate",
                  help="interaction mode: novice, intermediate,
                       "or expert [default: %default]")
```

If **`optparse`** encounters either `-h` or `--help` on the command-line, or if you just call **`parser.print_help()`**, it prints the following to standard output:

```
Usage: <yourscript> [options] arg1 arg2

Options:
  -h, --help            show this help message and exit
  -v, --verbose         make lots of noise [default]
  -q, --quiet           be vewwy quiet (I'm hunting wabbits)
  -f FILE, --filename=FILE
                        write output to FILE
  -m MODE, --mode=MODE  interaction mode: novice, intermediate,
                        expert [default: intermediate]
```

(If the help output is triggered by a help option, **`optparse`** exits after printing the help text.)

There’s a lot going on here to help **`optparse`** generate the best possible help message:

- o script define sua própria mensagem de uso:

  ```
  usage = "usage: %prog [options] arg1 arg2"
  ```

optparse expands `%prog` in the usage string to the name of the current program, i.e. `os.path.basename(sys.argv[0])`. The expanded string is then printed before the detailed option help.

If you don't supply a usage string, **optparse** uses a bland but sensible default: `"Usage: %prog [options]"`, which is fine if your script doesn't take any positional arguments.

- every option defines a help string, and doesn't worry about line-wrapping—**optparse** takes care of wrapping lines and making the help output look good.

- opções que assumem um valor indicam esse fato em sua mensagem de ajuda gerada automaticamente, por exemplo, para a opção "mode":

  ```
  -m MODE, --mode=MODE
  ```

  Here, "MODE" is called the meta-variable: it stands for the argument that the user is expected to supply to `-m`/`--mode`. By default, **optparse** converts the destination variable name to uppercase and uses that for the meta-variable. Sometimes, that's not what you want—for example, the `--filename` option explicitly sets `metavar="FILE"`, resulting in this automatically generated option description:

  ```
  -f FILE, --filename=FILE
  ```

  Isso é importante para mais do que apenas economizar espaço: o texto de ajuda escrito manualmente usa a metavariável `FILE` para dar uma pista ao usuário de que há uma conexão entre a sintaxe semiformal `-f FILE` e a descrição semântica informal como, por exemplo, "write output to FILE". Essa é uma maneira simples, mas eficaz, de tornar seu texto de ajuda muito mais claro e útil para usuários finais.

- options that have a default value can include `%default` in the help string—**optparse** will replace it with `str()` of the option's default value. If an option has no default value (or the default value is `None`), `%default` expands to `none`.

Agrupando opções

Ao lidar com muitas opções, é conveniente agrupar essas opções para melhor saída de ajuda. Um `OptionParser` pode conter vários grupos de opções, cada um dos quais pode conter várias opções.

Um grupo de opções é obtido usando a classe `OptionGroup`:

class optparse.OptionGroup(*parser*, *title*, *description=None*)

onde

- *parser* é a instância `OptionParser` na qual o grupo será inserido
- *title* é o título do grupo
- *description*, opcional, é uma descrição longa do grupo

`OptionGroup` herda de **`OptionContainer`** (como `OptionParser`) e, portanto, o método **`add_option()`** pode ser usado para adicionar uma opção ao grupo.

Depois que todas as opções são declaradas, usando o método `OptionParser` **`add_option_group()`** o grupo é adicionado ao analisador definido anteriormente.

Continuando com o analisador definido na seção anterior, adicionar um `OptionGroup` a um analisador é fácil:

```
group = OptionGroup(parser, "Dangerous Options",
                    "Caution: use these options at your own risk
                    "It is believed that some of them bite.")
group.add_option("-g", action="store_true", help="Group option.'
parser.add_option_group(group)
```

Isso resultaria na seguinte saída de texto de ajuda:

```
Usage: <yourscript> [options] arg1 arg2

Options:
  -h, --help            show this help message and exit
  -v, --verbose         make lots of noise [default]
  -q, --quiet           be vewwy quiet (I'm hunting wabbits)
```

```
  -f FILE, --filename=FILE
                        write output to FILE
  -m MODE, --mode=MODE  interaction mode: novice, intermediate,
                        expert [default: intermediate]

  Dangerous Options:
    Caution: use these options at your own risk.  It is believec
    of them bite.

    -g                  Group option.
```

Um exemplo um pouco mais completo pode envolver o uso de mais de um grupo: ainda estendendo o exemplo anterior:

```
group = OptionGroup(parser, "Dangerous Options",
                    "Caution: use these options at your own risk
                    "It is believed that some of them bite.")
group.add_option("-g", action="store_true", help="Group option.'
parser.add_option_group(group)

group = OptionGroup(parser, "Debug Options")
group.add_option("-d", "--debug", action="store_true",
                 help="Print debug information")
group.add_option("-s", "--sql", action="store_true",
                 help="Print all SQL statements executed")
group.add_option("-e", action="store_true", help="Print every ac
parser.add_option_group(group)
```

que resulta na seguinte saída:

```
Usage: <yourscript> [options] arg1 arg2

Options:
  -h, --help            show this help message and exit
  -v, --verbose         make lots of noise [default]
  -q, --quiet           be vewwy quiet (I'm hunting wabbits)
  -f FILE, --filename=FILE
                        write output to FILE
  -m MODE, --mode=MODE  interaction mode: novice, intermediate,
                        [default: intermediate]
```

```
  Dangerous Options:
    Caution: use these options at your own risk.  It is believed
    of them bite.

    -g                  Group option.

  Debug Options:
    -d, --debug         Print debug information
    -s, --sql           Print all SQL statements executed
    -e                  Print every action done
```

Outro método interessante, especialmente ao trabalhar programaticamente com grupos de opções, é:

OptionParser.get_option_group(*opt_str*)

Retorna o objeto **`OptionGroup`** ao qual a string de opção curta ou longa *opt_str* (por exemplo, `'-o'` ou `'--option'`) pertence. Se não houver tal **`OptionGroup`**, retorna `None`.

Exibindo uma string de versão

Similar to the brief usage string, **`optparse`** can also print a version string for your program. You have to supply the string as the `version` argument to OptionParser:

```
parser = OptionParser(usage="%prog [-f] [-q]", version="%prog 1.
```

`%prog` is expanded just like it is in `usage`. Apart from that, `version` can contain anything you like. When you supply it, **`optparse`** automatically adds a `--version` option to your parser. If it encounters this option on the command line, it expands your `version` string (by replacing `%prog`), prints it to stdout, and exits.

Por exemplo, se o seu script for chamado `/usr/bin/foo`:

```
$ /usr/bin/foo --version
foo 1.0
```

Os dois métodos a seguir podem ser usados para exibir e obter a string `version`:

OptionParser.print_version(*file=None*)

Exibe a mensagem de versão para o programa atual (`self.version`) em *file* (stdout padrão). Assim como em **`print_usage()`**, qualquer ocorrência de `%prog` em `self.version` é substituída pelo nome do programa atual. Não faz nada se `self.version` estiver vazio ou indefinido.

OptionParser.get_version()

O mesmo que **`print_version()`**, mas retorna a string da versão em vez de exibi-la.

How `optparse` handles errors

There are two broad classes of errors that **`optparse`** has to worry about: programmer errors and user errors. Programmer errors are usually erroneous calls to **`OptionParser.add_option()`**, e.g. invalid option strings, unknown option attributes, missing option attributes, etc. These are dealt with in the usual way: raise an exception (either **`optparse.OptionError`** or **`TypeError`**) and let the program crash.

Handling user errors is much more important, since they are guaranteed to happen no matter how stable your code is. **`optparse`** can automatically detect some user errors, such as bad option arguments (passing `-n 4x` where `-n` takes an integer argument), missing arguments (`-n` at the end of the command line, where `-n` takes an argument of any type). Also, you can call **`OptionParser.error()`** to signal an application-defined error condition:

```
(options, args) = parser.parse_args()
...
if options.a and options.b:
    parser.error("options -a and -b are mutually exclusive")
```

In either case, **`optparse`** handles the error the same way: it prints the program's usage message and an error message to standard error and exits with error status 2.

Considere o primeiro exemplo acima, onde o usuário passa `4x` para uma opção que aceita um inteiro:

```
$ /usr/bin/foo -n 4x
Usage: foo [options]

foo: error: option -n: invalid integer value: '4x'
```

Ou, quando o usuário não passa nenhum valor:

```
$ /usr/bin/foo -n
Usage: foo [options]

foo: error: -n option requires an argument
```

`optparse`-generated error messages take care always to mention the option involved in the error; be sure to do the same when calling **`OptionParser.error()`** from your application code.

If **`optparse`**'s default error-handling behaviour does not suit your needs, you'll need to subclass OptionParser and override its **`exit()`** and/or **`error()`** methods.

Juntando tudo

Here's what **`optparse`**-based scripts usually look like:

```
from optparse import OptionParser
...
def main():
    usage = "usage: %prog [options] arg"
    parser = OptionParser(usage)
    parser.add_option("-f", "--file", dest="filename",
                      help="read data from FILENAME")
    parser.add_option("-v", "--verbose",
                      action="store_true", dest="verbose")
    parser.add_option("-q", "--quiet",
                      action="store_false", dest="verbose")
    ...
    (options, args) = parser.parse_args()
    if len(args) != 1:
```

```
        parser.error("incorrect number of arguments")
    if options.verbose:
        print("reading %s..." % options.filename)
    ...

if __name__ == "__main__":
    main()
```

Guia de referência

Criando o analisador sintático

The first step in using **`optparse`** is to create an OptionParser instance.

class optparse.OptionParser(...)

O construtor do OptionParser não tem argumentos obrigatórios, mas vários argumentos nomeados opcionais. Você deve sempre passá-los como argumentos nomeados, ou seja, não confie na ordem em que os argumentos são declarados.

`usage` (padrão: `"%prog [options]"`)

The usage summary to print when your program is run incorrectly or with a help option. When **`optparse`** prints the usage string, it expands `%prog` to `os.path.basename(sys.argv[0])` (or to `prog` if you passed that keyword argument). To suppress a usage message, pass the special value **`optparse.SUPPRESS_USAGE`**.

`option_list` (padrão: `[]`)

Uma lista de objetos Option para preencher o analisador sintático. As opções em `option_list` são adicionadas após quaisquer opções em `standard_option_list` (um atributo de classe que pode ser definido por subclasses de OptionParser), mas antes de quaisquer opções de versão ou ajuda. Descontinuado; use **`add_option()`** após criar o analisador sintático.

`option_class` (padrão: optparse.Option)

Classe para usar ao adicionar opções ao analisador sintático em **`add_option()`**.

`version` (padrão: `None`)
: A version string to print when the user supplies a version option. If you supply a true value for `version`, **`optparse`** automatically adds a version option with the single option string `--version`. The substring `%prog` is expanded the same as for `usage`.

`conflict_handler` (padrão: `"error"`)
: Especifica o que fazer quando opções com strings de opções conflitantes são adicionadas ao analisador sintático; consulte a seção Conflitos entre opções.

`description` (padrão: `None`)
: A paragraph of text giving a brief overview of your program. **`optparse`** reformats this paragraph to fit the current terminal width and prints it when the user requests help (after `usage`, but before the list of options).

`formatter` (padrão: um novo **`IndentedHelpFormatter`**)
: An instance of optparse.HelpFormatter that will be used for printing help text. **`optparse`** provides two concrete classes for this purpose: IndentedHelpFormatter and TitledHelpFormatter.

`add_help_option` (padrão: `True`)
: If true, **`optparse`** will add a help option (with option strings `-h` and `--help`) to the parser.

`prog`
: A string a ser usada ao expandir `%prog` em `usage` e `version` em vez de `os.path.basename(sys.argv[0])`.

`epilog` (padrão: `None`)
: Um parágrafo de texto de ajuda para exibir após a ajuda da opção.

Preenchendo o analisador sintático

Há várias maneiras de preencher o analisador sintático com opções. A maneira preferida é usando **`OptionParser.add_option()`**, como mostrado

na seção Tutorial. **`add_option()`** pode ser chamado de uma das duas maneiras:

- passar uma instância de Option (conforme retornado por **`make_option()`**)
- passar qualquer combinação de argumentos nomeados e posicionais que sejam aceitáveis para **`make_option()`** (ou seja, para o construtor de Option), e ela criará a instância de Option para você

A outra alternativa é passar uma lista de instâncias de Option pré-construídas para o construtor de OptionParser, como em:

```
option_list = [
    make_option("-f", "--filename",
                action="store", type="string", dest="filename"),
    make_option("-q", "--quiet",
                action="store_false", dest="verbose"),
    ]
parser = OptionParser(option_list=option_list)
```

(**`make_option()`** is a factory function for creating Option instances; currently it is an alias for the Option constructor. A future version of **`optparse`** may split Option into several classes, and **`make_option()`** will pick the right class to instantiate. Do not instantiate Option directly.)

Definindo as opções

Cada instância de Option representa um conjunto de strings de opções de linha de comando sinônimas, por exemplo, `-f` e `--file`. Você pode especificar qualquer número de strings de opções curtas ou longas, mas deve especificar pelo menos uma string de opção geral.

A maneira canônica de criar uma instância de **`Option`** é com o método **`add_option()`** de **`OptionParser`**.

OptionParser.add_option(*option*)
OptionParser.add_option(**opt_str*, *attr=value*, ...)

Para definir uma opção com apenas uma sequência de opções curta:

```
parser.add_option("-f", attr=value, ...)
```

E para definir uma opção com apenas uma longa sequência de opções:

```
parser.add_option("--foo", attr=value, ...)
```

The keyword arguments define attributes of the new Option object. The most important option attribute is **`action`**, and it largely determines which other attributes are relevant or required. If you pass irrelevant option attributes, or fail to pass required ones, **`optparse`** raises an **`OptionError`** exception explaining your mistake.

An option's *action* determines what **`optparse`** does when it encounters this option on the command-line. The standard option actions hard-coded into **`optparse`** are:

`"store"`
: armazena o argumento desta opção (padrão)

`"store_const"`
: armazena um valor constante, predefinido via **`Option.const`**

`"store_true"`
: armazena `True`

`"store_false"`
: armazena `False`

`"append"`
: anexa o argumento desta opção a uma lista

`"append_const"`
: anexar um valor constante a uma lista, predefinido via **`Option.const`**

`"count"`
: incrementa um contador em um

`"callback"`

chama uma função especificada

`"help"`
exibe uma mensagem de uso incluindo todas as opções e a documentação para elas

(Se você não fornecer uma ação, o padrão é `"store"`. Para esta ação, você também pode fornecer os atributos de opção **`type`** e **`dest`**; consulte Ações de opção padrão.)

As you can see, most actions involve storing or updating a value somewhere. **`optparse`** always creates a special object for this, conventionally called `options`, which is an instance of **`optparse.Values`**.

class optparse.Values
Um objeto que contém nomes e valores de argumentos analisados como atributos. Normalmente criado chamando ao chamar **`OptionParser.parse_args()`**, e pode ser substituído por uma subclasse personalizada passada para o argumento *values* de **`OptionParser.parse_args()`** (conforme descrito em Análise de argumentos).

Argumentos de opção (e vários outros valores) são armazenados como atributos deste objeto, de acordo com o atributo de opção **`dest`** (destino).

Por exemplo, quando você chama

```
parser.parse_args()
```

one of the first things **`optparse`** does is create the `options` object:

```
options = Values()
```

Se uma das opções neste analisador sintático for definida com

```
parser.add_option("-f", "--file", action="store", type="string",
```

e a linha de comando que está sendo analisada inclui qualquer um dos seguintes:

```
-ffoo
-f foo
--file=foo
--file foo
```

then **optparse**, on seeing this option, will do the equivalent of

```
options.filename = "foo"
```

Os atributos de opção **type** e **dest** são quase tão importantes quanto **action**, mas **action** é o único que faz sentido para *todas* as opções.

Atributos da classe Option

class optparse.Option

Um único argumento de linha de comando, com vários atributos passados como nomeados para o construtor. Normalmente criado com **OptionParser.add_option()** em vez de diretamente, e pode ser substituído por uma classe personalizada por meio do argumento *option_class* para **OptionParser**.

The following option attributes may be passed as keyword arguments to **OptionParser.add_option()**. If you pass an option attribute that is not relevant to a particular option, or fail to pass a required option attribute, **optparse** raises **OptionError**.

Option.action

(padrão: `"store"`)

Determines **optparse**'s behaviour when this option is seen on the command line; the available options are documented here.

Option.type

(padrão: `"string"`)

O tipo de argumento esperado por esta opção (por exemplo, `"string"` ou `"int"`); os tipos de opções disponíveis estão documentados aqui.

Option.dest

(padrão: derivado de strings de opção)

If the option's action implies writing or modifying a value somewhere, this tells **`optparse`** where to write it: **`dest`** names an attribute of the `options` object that **`optparse`** builds as it parses the command line.

Option.default

O valor para usar para o destino desta opção se a opção não for vista na linha de comando. Veja também **`OptionParser.set_defaults()`**.

Option.nargs

(padrão: 1)

How many arguments of type **`type`** should be consumed when this option is seen. If > 1, **`optparse`** will store a tuple of values to **`dest`**.

Option.const

Para ações que armazenam um valor constante, o valor constante a ser armazenado.

Option.choices

Para opções do tipo `"choice"`, a lista de strings que o usuário pode escolher.

Option.callback

Para opções com ação `"callback"`, o chamável para chamar quando esta opção for vista. Veja a seção Funções de retorno de opção para detalhes sobre os argumentos passados para o chamável.

Option.callback_args
Option.callback_kwargs

Argumentos nomeados e posicionais adicionais para passar para `callback` após os quatro argumentos de retorno de chamada padrão.

Option.help

Texto de ajuda para exibir para esta opção ao listar todas as opções disponíveis após o usuário fornecer uma opção **help** (como `--help`). Se nenhum texto de ajuda for fornecido, a opção será listada sem texto de ajuda. Para ocultar esta opção, use o valor especial **optparse.SUPPRESS_HELP**.

Option.metavar

(padrão: derivado de strings de opção)

Substituto para o(s) argumento(s) de opção a ser(em) usado(s) ao exibir texto de ajuda. Veja a seção Tutorial para um exemplo.

Ações de opção padrão

The various option actions all have slightly different requirements and effects. Most actions have several relevant option attributes which you may specify to guide **optparse**'s behaviour; a few have required attributes, which you must specify for any option using that action.

- `"store"` [relevante: **type**, **dest**, **nargs**, **choices**]

  A opção deve ser seguida por um argumento, que é convertido em um valor de acordo com **type** e armazenado em **dest**. Se **nargs** > 1, vários argumentos serão consumidos da linha de comando; todos serão convertidos de acordo com **type** e armazenados em **dest** como uma tupla. Veja a seção Tipos de opção padrão.

  Se **choices** for fornecido (uma lista ou tupla de strings), o tipo padrão será `"choice"`.

  Se **type** não for fornecido, o padrão será `"string"`.

  If **dest** is not supplied, **optparse** derives a destination from the first long option string (e.g., `--foo-bar` implies `foo_bar`). If there are no long option strings, **optparse** derives a destination from the first short option string (e.g., `-f` implies `f`).

Exemplo:

```
parser.add_option("-f")
parser.add_option("-p", type="float", nargs=3, dest="point")
```

Como ele analisa a linha de comando

```
-f foo.txt -p 1 -3.5 4 -fbar.txt
```

`optparse` will set

```
options.f = "foo.txt"
options.point = (1.0, -3.5, 4.0)
options.f = "bar.txt"
```

- `"store_const"` [obrigatório: **`const`**; relevante: **`dest`**]

  O valor de **`const`** é armazenado em **`dest`**.

  Exemplo:

```
parser.add_option("-q", "--quiet",
                  action="store_const", const=0, dest="verbo
parser.add_option("-v", "--verbose",
                  action="store_const", const=1, dest="verbo
parser.add_option("--noisy",
                  action="store_const", const=2, dest="verbo
```

  If `--noisy` is seen, **`optparse`** will set

```
options.verbose = 2
```

- `"store_true"` [relevante: **`dest`**]

  Um caso especial de `"store_const"` que armazena `True` em **`dest`**.

- `"store_false"` [relevante: **`dest`**]

  Como `"store_true"`, mas armazenad `False`.

Exemplo:

```
parser.add_option("--clobber", action="store_true", dest="c
parser.add_option("--no-clobber", action="store_false", dest
```

- "append" [relevante: **type**, **dest**, **nargs**, **choices**]

  The option must be followed by an argument, which is appended to the list in **dest**. If no default value for **dest** is supplied, an empty list is automatically created when **optparse** first encounters this option on the command-line. If **nargs** > 1, multiple arguments are consumed, and a tuple of length **nargs** is appended to **dest**.

  Os padrões para **type** e **dest** são os mesmos da ação "store".

  Exemplo:

```
parser.add_option("-t", "--tracks", action="append", type="
```

  If -t3 is seen on the command-line, **optparse** does the equivalent of:

```
options.tracks = []
options.tracks.append(int("3"))
```

  Se, um pouco mais tarde, --tracks=4 for encontrado, ele faz:

```
options.tracks.append(int("4"))
```

  A ação append chama o método append no valor atual da opção. Isso significa que qualquer valor padrão especificado deve ter um método append. Isso também significa que se o valor padrão não for vazio, os elementos padrão estarão presentes no valor analisado para a opção, com quaisquer valores da linha de comando anexados após esses valores padrão:

```
>>> parser.add_option("--files", action="append", default=[
>>> opts, args = parser.parse_args(['--files', 'overrides.my
>>> opts.files
['~/.mypkg/defaults', 'overrides.mypkg']
```

- "append_const" [obrigatório: **const**; relevante: **dest**]

  Como "store_const", mas o valor de **const** é anexado a **dest**; assim como em "append", **dest** assume como padrão None, e uma lista vazia é criada automaticamente na primeira vez que a opção é encontrada.

- "count" [relevante: **dest**]

  Incrementa o inteiro armazenado em **dest**. Se nenhum valor padrão for fornecido, **dest** é definido como zero antes de ser incrementado pela primeira vez.

  Exemplo:

  ```
  parser.add_option("-v", action="count", dest="verbosity")
  ```

  The first time -v is seen on the command line, **optparse** does the equivalent of:

  ```
  options.verbosity = 0
  options.verbosity += 1
  ```

  Cada ocorrência subsequente de -v resulta em

  ```
  options.verbosity += 1
  ```

- "callback" [obrigatório: **callback**; relevante: **type**, **nargs**, **callback_args**, **callback_kwargs**]

  Chama a função especificada por **callback**, que é chamada como

  ```
  func(option, opt_str, value, parser, *args, **kwargs)
  ```

  Veja a seção Funções de retorno de opção para mais detalhes.

- "help"

Exibe uma mensagem de ajuda completa para todas as opções no analisador sintático de opções atual. A mensagem de ajuda é construída a partir da string `usage` passada para o construtor do OptionParser e da string **`help`** passada para cada opção.

Se nenhuma string de **`help`** for fornecida para uma opção, ela ainda será listada na mensagem de ajuda. Para omitir uma opção completamente, use o valor especial **`optparse.SUPPRESS_HELP`**.

`optparse` automatically adds a **`help`** option to all OptionParsers, so you do not normally need to create one.

Exemplo:

```
from optparse import OptionParser, SUPPRESS_HELP

# geralmente, uma opção de ajuda é adicionada
# automaticamente, mas que podem ser suprimida
# usando o argumento add_help_option
parser = OptionParser(add_help_option=False)

parser.add_option("-h", "--help", action="help")
parser.add_option("-v", action="store_true", dest="verbose",
                  help="Be moderately verbose")
parser.add_option("--file", dest="filename",
                  help="Input file to read data from")
parser.add_option("--secret", help=SUPPRESS_HELP)
```

If **`optparse`** sees either `-h` or `--help` on the command line, it will print something like the following help message to stdout (assuming `sys.argv[0]` is `"foo.py"`):

```
Usage: foo.py [options]

Options:
  -h, --help        Show this help message and exit
  -v                Be moderately verbose
  --file=FILENAME   Input file to read data from
```

After printing the help message, **optparse** terminates your process with `sys.exit(0)`.

- `"version"`

  Prints the version number supplied to the OptionParser to stdout and exits. The version number is actually formatted and printed by the `print_version()` method of OptionParser. Generally only relevant if the `version` argument is supplied to the OptionParser constructor. As with **help** options, you will rarely create `version` options, since **optparse** automatically adds them when needed.

Tipos de opção padrão

optparse has five built-in option types: `"string"`, `"int"`, `"choice"`, `"float"` and `"complex"`. If you need to add new option types, see section Extending optparse.

Os argumentos para opções de string não são verificados ou convertidos de forma alguma: o texto na linha de comando é armazenado no destino (ou passado para a função de retorno) como está.

Argumentos inteiros (tipo `"int"`) são analisados da seguinte forma:

- se o número começar com `0x`, ele será analisado como um número hexadecimal
- se o número começar com `0`, ele será analisado como um número octal
- se o número começar com `0b`, ele será analisado como um número binário
- caso contrário, o número é analisado como um número decimal

The conversion is done by calling **int()** with the appropriate base (2, 8, 10, or 16). If this fails, so will **optparse**, although with a more useful error message.

Os argumentos de opção `"float"` e `"complex"` são convertidos diretamente com **float()** e **complex()**, com tratamento de erros semelhante.

As opções `"choice"` são um subtipo das opções `"string"`. O atributo da opção **`choices`** (uma sequência de strings) define o conjunto de argumentos de opção permitidos. **`optparse.check_choice()`** compara os argumentos de opção fornecidos pelo usuário com esta lista mestre e levanta **`OptionValueError`** se uma string inválida for fornecida.

Análise de argumentos

O objetivo de criar e preencher uma instância de OptionParser é chamar seu método **`parse_args()`**.

OptionParser.parse_args(*args=None*, *values=None*)

Analisa as opções de linha de comando encontradas em *args*.

Os parâmetros de entrada são

`args`
: a lista de argumentos para processar (padrão: `sys.argv[1:]`)

`values`
: um objeto **`Values`** para armazenar argumentos de opção (padrão: uma nova instância de **`Values`**) – se você fornecer um objeto existente, os padrões de opção não serão inicializados nele

e o valor de retorno é um par `(options, args)` sendo

`options`
: the same object that was passed in as *values*, or the `optparse.Values` instance created by **`optparse`**

`args`
: os argumentos posicionais restantes depois que todas as opções foram processadas

O uso mais comum é não fornecer nenhum argumento nomeado. Se você fornecer `values`, ele será modificado com chamadas repetidas de **`setattr()`** (aproximadamente uma para cada argumento de opção armazenado em um destino de opção) e retornado por **`parse_args()`**.

Se **parse_args()** encontrar algum erro na lista de argumentos, ele chama o método **error()** da instância de OptionParser com uma mensagem de erro apropriada para o usuário final. Isso, em última análise, encerra seu processo com um status de saída 2 (o status de saída tradicional do Unix para erros de linha de comando).

Consultando e manipulando seu analisador sintático de opções

O comportamento padrão do analisador sintático de opções pode ser ligeiramente personalizado, e você também pode dar uma olhada no seu analisador de opções e ver o que há lá. A classe OptionParser fornece vários métodos para ajudar você:

OptionParser.disable_interspersed_args()

Set parsing to stop on the first non-option. For example, if `-a` and `-b` are both simple options that take no arguments, **optparse** normally accepts this syntax:

```
prog -a arg1 -b arg2
```

e trata-a como equivalente a

```
prog -a -b arg1 arg2
```

Para desabilitar esse recurso, chame **disable_interspersed_args()**. Isso restaura a sintaxe Unix tradicional, onde a análise de opções para com o primeiro argumento não-opcional.

Use isto se você tiver um processador de comando que executa outro comando que tem opções próprias e você quer ter certeza de que essas opções não fiquem confusas. Por exemplo, cada comando pode ter um conjunto diferente de opções.

OptionParser.enable_interspersed_args()

Define a análise para não parar na primeira não opção, permitindo intercalar alternadores com argumentos de comando. Esse é o comportamento padrão.

OptionParser.get_option(*opt_str*)

Retorna a instância de Option com a string de opção *opt_str*, ou `None` se nenhuma opção tiver essa string de opção.

OptionParser.has_option(*opt_str*)

Retorna `True` se o objeto OptionParser tiver uma opção com string de opção *opt_str* (por exemplo, `-q` ou `--verbose`).

OptionParser.remove_option(*opt_str*)

Se o **`OptionParser`** tiver uma opção correspondente a *opt_str*, essa opção será removida. Se essa opção fornecer quaisquer outras strings de opção, todas essas strings de opção se tornarão inválidas. Se *opt_str* não ocorrer em nenhuma opção pertencente a este **`OptionParser`**, levanta **`ValueError`**.

Conflitos entre opções

Se você não tomar cuidado, é fácil definir opções com strings de opções conflitantes:

```
parser.add_option("-n", "--dry-run", ...)
...
parser.add_option("-n", "--noisy", ...)
```

(Isto é particularmente verdadeiro se você definiu sua própria subclasse de OptionParser com algumas opções padrão.)

Every time you add an option, **`optparse`** checks for conflicts with existing options. If it finds any, it invokes the current conflict-handling mechanism. You can set the conflict-handling mechanism either in the constructor:

```
parser = OptionParser(..., conflict_handler=handler)
```

ou com uma chamada separada:

```
parser.set_conflict_handler(handler)
```

Os manipuladores de conflitos disponíveis são:

`"error"` (padrão)
: presume que os conflitos de opções são um erro de programação e levanta **`OptionConflictError`**

`"resolve"`
: resolve conflitos de opções de forma inteligente (veja abaixo)

Como exemplo, vamos definir um **`OptionParser`** que resolve conflitos de forma inteligente e adiciona opções conflitantes a ele:

```
parser = OptionParser(conflict_handler="resolve")
parser.add_option("-n", "--dry-run", ..., help="do no harm")
parser.add_option("-n", "--noisy", ..., help="be noisy")
```

At this point, **`optparse`** detects that a previously added option is already using the `-n` option string. Since `conflict_handler` is `"resolve"`, it resolves the situation by removing `-n` from the earlier option's list of option strings. Now `--dry-run` is the only way for the user to activate that option. If the user asks for help, the help message will reflect that:

```
Options:
  --dry-run     do no harm
  ...
  -n, --noisy   be noisy
```

It's possible to whittle away the option strings for a previously added option until there are none left, and the user has no way of invoking that option from the command-line. In that case, **`optparse`** removes that option completely, so it doesn't show up in help text or anywhere else. Carrying on with our existing OptionParser:

```
parser.add_option("--dry-run", ..., help="new dry-run option")
```

At this point, the original `-n`/`--dry-run` option is no longer accessible, so **`optparse`** removes it, leaving this help text:

```
Options:
  ...
  -n, --noisy   be noisy
  --dry-run     new dry-run option
```

Limpeza

As instâncias de OptionParser têm várias referências cíclicas. Isso não deve ser um problema para o coletor de lixo do Python, mas você pode desejar quebrar as referências cíclicas explicitamente chamando **`destroy()`** no seu OptionParser quando terminar de usá-lo. Isso é particularmente útil em aplicações de longa execução, onde gráficos de objetos grandes podem ser acessados do seu OptionParser.

Outros métodos

OptionParser provê vários outros métodos públicos:

OptionParser.set_usage(*usage*)

Defina a string de uso de acordo com as regras descritas acima para o argumento nomeado do construtor `usage`. Passar `None` define a string de uso padrão; use **`optparse.SUPPRESS_USAGE`** para suprimir uma mensagem de uso.

OptionParser.print_usage(*file=None*)

Exibe a mensagem de uso para o programa atual (`self.usage`) em *file* (stdout padrão). Qualquer ocorrência da string `%prog` em `self.usage` é substituída pelo nome do programa atual. Não faz nada se `self.usage` estiver vazio ou não definido.

OptionParser.get_usage()

O mesmo que **`print_usage()`**, mas retorna a string de uso em vez de exibi-la.

OptionParser.set_defaults(*dest=value*, ...)

Define valores padrão para vários destinos de opção de uma vez. Usar **`set_defaults()`** é a maneira preferida de definir valores padrão para opções, já que várias opções podem compartilhar o mesmo destino. Por exemplo, se várias opções de “modo” definirem o mesmo destino, qualquer uma delas pode definir o padrão, e a última vence:

```
parser.add_option("--advanced", action="store_const",
                  dest="mode", const="advanced",
                  default="novice")    # substituída abaixo
parser.add_option("--novice", action="store_const",
                  dest="mode", const="novice",
                  default="advanced")  # substitui a configur
```

Para evitar essa confusão, use `set_defaults()`:

```
parser.set_defaults(mode="advanced")
parser.add_option("--advanced", action="store_const",
                  dest="mode", const="advanced")
parser.add_option("--novice", action="store_const",
                  dest="mode", const="novice")
```

Funções de retorno de opção

When **`optparse`**'s built-in actions and types aren't quite enough for your needs, you have two choices: extend **`optparse`** or define a callback option. Extending **`optparse`** is more general, but overkill for a lot of simple cases. Quite often a simple callback is all you need.

Há duas etapas para definir uma opção de retorno de chamada:

- definir a opção em si usando a ação `"callback"`
- escrever o função de retorno; esta é uma função (ou método) que recebe pelo menos quatro argumentos, conforme descrito abaixo

Definindo uma opção de retorno de chamada

Como sempre, a maneira mais fácil de definir uma opção de retorno de chamada é usando o método `OptionParser.add_option()`. Além de `action`, o único atributo de opção que você deve especificar é `callback`, a função a ser chamada:

```
parser.add_option("-c", action="callback", callback=my_callback)
```

`callback` is a function (or other callable object), so you must have already defined `my_callback()` when you create this callback option. In this simple case, **`optparse`** doesn't even know if `-c` takes any arguments, which usually means that the option takes no arguments—the mere presence of `-c` on the command-line is all it needs to know. In some circumstances, though, you might want your callback to consume an arbitrary number of command-line arguments. This is where writing callbacks gets tricky; it's covered later in this section.

`optparse` always passes four particular arguments to your callback, and it will only pass additional arguments if you specify them via **`callback_args`** and **`callback_kwargs`**. Thus, the minimal callback function signature is:

```
def my_callback(option, opt, value, parser):
```

Os quatro argumentos para um retorno de chamada são descritos abaixo.

Há vários outros atributos de opção que você pode fornecer ao definir uma opção de retorno de chamada:

`type`
: has its usual meaning: as with the `"store"` or `"append"` actions, it instructs **`optparse`** to consume one argument and convert it to **`type`**. Rather than storing the converted value(s) anywhere, though, **`optparse`** passes it to your callback function.

`nargs`
: also has its usual meaning: if it is supplied and > 1, **`optparse`** will consume **`nargs`** arguments, each of which must be convertible to **`type`**. It then passes a tuple of converted values to your callback.

`callback_args`
: uma tupla de argumentos posicionais extras para passar para a função de retorno

`callback_kwargs`
: um dicionário de argumentos nomeados extras para passar para a função de retorno

Como os retornos de chamada são chamados

Todos os retornos de chamada são chamados da seguinte forma:

```
func(option, opt_str, value, parser, *args, **kwargs)
```

onde

`option`
: é a instância de Option que está chamando o retorno de chamada

`opt_str`
: é a string de opção vista na linha de comando que está acionando o retorno de chamada. (Se uma opção longa abreviada foi usada, `opt_str` será a string de opção canônica completa — por exemplo, se o usuário colocar `--foo` na linha de comando como uma abreviação para `--foobar`, então `opt_str` será `"--foobar"`.)

`value`
: is the argument to this option seen on the command-line. **`optparse`** will only expect an argument if **`type`** is set; the type of `value` will be the type implied by the option's type. If **`type`** for this option is `None` (no argument expected), then `value` will be `None`. If **`nargs`** > 1, `value` will be a tuple of values of the appropriate type.

`parser`
: é a instância OptionParser que controla tudo, útil principalmente porque você pode acessar alguns outros dados interessantes por meio de seus atributos de instância:

  `parser.largs`
  : a lista atual de argumentos restantes, ou seja, argumentos que foram consumidos, mas não são opções nem argumentos de opção. Sinta-se à vontade para modificar `parser.largs`, por exemplo, adicionando mais argumentos a ele. (Esta lista se tornará `args`, o segundo valor de retorno de **`parse_args()`**.)

  `parser.rargs`

a lista atual de argumentos restantes, ou seja, com `opt_str` e `value` (se aplicável) removidos, e apenas os argumentos que os seguem ainda lá. Sinta-se à vontade para modificar `parser.rargs`, por exemplo, consumindo mais argumentos.

`parser.values`
: the object where option values are by default stored (an instance of optparse.OptionValues). This lets callbacks use the same mechanism as the rest of **`optparse`** for storing option values; you don't need to mess around with globals or closures. You can also access or modify the value(s) of any options already encountered on the command-line.

`args`
: é uma tupla de argumentos posicionais arbitrários fornecidos por meio do atributo de opção **`callback_args`**.

`kwargs`
: é um dicionário de argumentos nomeados arbitrários fornecidos via **`callback_kwargs`**.

Levantando erros em uma função de retorno

The callback function should raise **`OptionValueError`** if there are any problems with the option or its argument(s). **`optparse`** catches this and terminates the program, printing the error message you supply to stderr. Your message should be clear, concise, accurate, and mention the option at fault. Otherwise, the user will have a hard time figuring out what they did wrong.

Exemplo de função de retorno 1: retorno de chamada trivial

Aqui está um exemplo de uma opção de retorno de chamada que não aceita argumentos e simplesmente registra que a opção foi vista:

```
def record_foo_seen(option, opt_str, value, parser):
    parser.values.saw_foo = True
```

```
parser.add_option("--foo", action="callback", callback=record_fo
```

Claro, você pode fazer isso com a ação `"store_true"`.

Exemplo de função de retorno 2: verificar a ordem das opções

Aqui está um exemplo um pouco mais interessante: registra o fato de que `-a` é visto, mas explode se ele vier depois de `-b` na linha de comando.

```
def check_order(option, opt_str, value, parser):
    if parser.values.b:
        raise OptionValueError("can't use -a after -b")
    parser.values.a = 1
...
parser.add_option("-a", action="callback", callback=check_order)
parser.add_option("-b", action="store_true", dest="b")
```

Exemplo 3 de função de retorno: verificar a ordem das opções (generalizada)

Se você quiser reutilizar essa função de retorno para várias opções semelhantes (definir um sinalizador, mas explodir se `-b` já tiver sido visto), será preciso um pouco de trabalho: a mensagem de erro e o sinalizador que ele define devem ser generalizados.

```
def check_order(option, opt_str, value, parser):
    if parser.values.b:
        raise OptionValueError("can't use %s after -b" % opt_str
    setattr(parser.values, option.dest, 1)
...
parser.add_option("-a", action="callback", callback=check_order,
parser.add_option("-b", action="store_true", dest="b")
parser.add_option("-c", action="callback", callback=check_order,
```

Exemplo de função de retorno 4: verificar uma condição arbitrária

Claro, você pode colocar qualquer condição ali—você não está limitado a verificar os valores de opções já definidas. Por exemplo, se você tem opções que não devem ser chamadas quando a lua está cheia, tudo o que você tem a fazer é isto:

```
def check_moon(option, opt_str, value, parser):
    if is_moon_full():
        raise OptionValueError("%s option invalid when moon is f
                               % opt_str)
    setattr(parser.values, option.dest, 1)
...
parser.add_option("--foo",
                  action="callback", callback=check_moon, dest='
```

(A definição de `is_moon_full()` é deixada como um exercício para o leitor.)

Exemplo de função de retorno 5: argumentos fixos

As coisas ficam um pouco mais interessantes quando você define opções de retorno de chamada que aceitam um número fixo de argumentos. Especificar que uma opção de retorno de chamada aceita argumentos é semelhante a definir uma opção `"store"` ou `"append"`: se você definir **`type`**, então a opção aceita um argumento que deve ser conversível para esse tipo; se você definir ainda **`nargs`**, então a opção aceita argumentos **`nargs`**.

Aqui está um exemplo que apenas emula a ação padrão `"store"`:

```
def store_value(option, opt_str, value, parser):
    setattr(parser.values, option.dest, value)
...
parser.add_option("--foo",
                  action="callback", callback=store_value,
                  type="int", nargs=3, dest="foo")
```

Note that **`optparse`** takes care of consuming 3 arguments and converting them to integers for you; all you have to do is store them. (Or whatever; obviously you don’t need a callback for this example.)

Exemplo de função de retorno 6: argumentos variáveis

Things get hairy when you want an option to take a variable number of arguments. For this case, you must write a callback, as **`optparse`** doesn't provide any built-in capabilities for it. And you have to deal with certain intricacies of conventional Unix command-line parsing that **`optparse`** normally handles for you. In particular, callbacks should implement the conventional rules for bare `--` and `-` arguments:

- `--` ou `-` podem ser argumentos de opção
- `--` apenas (se não for o argumento para alguma opção): interrompe o processamento da linha de comando e descarta o `--`
- `-` apenas (se não for o argumento para alguma opção): interrompe o processamento da linha de comando, mas mantém o `-` (anexa-o a `parser.largs`)

If you want an option that takes a variable number of arguments, there are several subtle, tricky issues to worry about. The exact implementation you choose will be based on which trade-offs you're willing to make for your application (which is why **`optparse`** doesn't support this sort of thing directly).

No entanto, aqui vai uma tentativa de função de retorno para uma opção com argumentos variáveis:

```
def vararg_callback(option, opt_str, value, parser):
    assert value is None
    value = []

    def floatable(str):
        try:
            float(str)
            return True
        except ValueError:
            return False

    for arg in parser.rargs:
        # para em --foo como opções
        if arg[:2] == "--" and len(arg) > 2:
```

```
            break
        # para em -a, mas não em -3 ou -3.0
        if arg[:1] == "-" and len(arg) > 1 and not floatable(arg
            break
        value.append(arg)

    del parser.rargs[:len(value)]
    setattr(parser.values, option.dest, value)

...
parser.add_option("-c", "--callback", dest="vararg_attr",
                  action="callback", callback=vararg_callback)
```

Extending `optparse`

Since the two major controlling factors in how **`optparse`** interprets command-line options are the action and type of each option, the most likely direction of extension is to add new actions and new types.

Adicionando novos tipos

To add new types, you need to define your own subclass of **`optparse`**'s **`Option`** class. This class has a couple of attributes that define **`optparse`**'s types: **`TYPES`** and **`TYPE_CHECKER`**.

Option.TYPES

Uma tupla de nomes de tipos; na sua subclasse, basta definir uma nova tupla **`TYPES`** que se baseia na tupla padrão.

Option.TYPE_CHECKER

Um dicionário que mapeia nomes de tipos para funções de verificação de tipos. Uma função de verificação de tipos tem a seguinte assinatura:

```
def check_mytype(option, opt, value)
```

sendo `option` uma instância **`Option`**, `opt` é uma string de opção (por exemplo, `-f`), e `value` é a string da linha de comando que deve ser verificada e convertida para o tipo desejado. `check_mytype()` deve

retornar um objeto do tipo hipotético `mytype`. O valor retornado por uma função de verificação de tipo acabará na instância OptionValues retornada por **`OptionParser.parse_args()`**, ou será passado para uma função de retorno como o parâmetro `value`.

Sua função de verificação de tipo deve levantar **`OptionValueError`** se encontrar algum problema. **`OptionValueError`** recebe um único argumento de string, que é passado como está para o método **`error()`** do **`OptionParser`**, que por sua vez adiciona o nome do programa e a string `"error:"` e exibe tudo no stderr antes de encerrar o processo.

Here's a silly example that demonstrates adding a `"complex"` option type to parse Python-style complex numbers on the command line. (This is even sillier than it used to be, because **`optparse`** 1.3 added built-in support for complex numbers, but never mind.)

Primeiro, as importações necessárias:

```
from copy import copy
from optparse import Option, OptionValueError
```

Você precisa definir seu verificador de tipo primeiro, pois ele será referenciado mais tarde (no atributo de classe **`TYPE_CHECKER`** da sua subclasse de Option):

```
def check_complex(option, opt, value):
    try:
        return complex(value)
    except ValueError:
        raise OptionValueError(
            "option %s: invalid complex value: %r" % (opt, value
```

Finalmente, a subclasse de Option:

```
class MyOption (Option):
    TYPES = Option.TYPES + ("complex",)
    TYPE_CHECKER = copy(Option.TYPE_CHECKER)
    TYPE_CHECKER["complex"] = check_complex
```

(If we didn't make a **copy()** of **Option.TYPE_CHECKER**, we would end up modifying the **TYPE_CHECKER** attribute of **optparse**'s Option class. This being Python, nothing stops you from doing that except good manners and common sense.)

That's it! Now you can write a script that uses the new option type just like any other **optparse**-based script, except you have to instruct your OptionParser to use MyOption instead of Option:

```
parser = OptionParser(option_class=MyOption)
parser.add_option("-c", type="complex")
```

Como alternativa, você pode criar sua própria lista de opções e passá-la para o OptionParser; se você não usar **add_option()** da maneira acima, não precisa informar ao OptionParser qual classe de opção usar:

```
option_list = [MyOption("-c", action="store", type="complex", de
parser = OptionParser(option_list=option_list)
```

Adicionando novas ações

Adding new actions is a bit trickier, because you have to understand that **optparse** has a couple of classifications for actions:

Ações "store"
: actions that result in **optparse** storing a value to an attribute of the current OptionValues instance; these options require a **dest** attribute to be supplied to the Option constructor.

Ações "typed"
: Ações que pegam um valor da linha de comando e esperam que ele seja de um certo tipo; ou melhor, uma string que pode ser convertida para um certo tipo. Essas opções requerem um atributo **type** para o construtor Option.

Esses são conjuntos sobrepostos: algumas ações "store" padrão são `"store"`, `"store_const"`, `"append"` e `"count"`, enquanto as ações "typed" padrão são `"store"`, `"append"` e `"callback"`.

Ao adicionar uma ação, você precisa categorizá-la listando-a em pelo menos um dos seguintes atributos de classe de Option (todos são listas de strings):

Option.ACTIONS
: Todas as ações devem ser listadas em ACTIONS.

Option.STORE_ACTIONS
: As ações “store” também são listadas aqui.

Option.TYPED_ACTIONS
: As ações “typed” também são listadas aqui.

Option.ALWAYS_TYPED_ACTIONS
: Actions that always take a type (i.e. whose options always take a value) are additionally listed here. The only effect of this is that **`optparse`** assigns the default type, `"string"`, to options with no explicit type whose action is listed in **`ALWAYS_TYPED_ACTIONS`**.

Para realmente implementar sua nova ação, você deve substituir o método **`take_action()`** de Option e adicionar um caso que reconheça sua ação.

Por exemplo, vamos adicionar uma ação `"extend"`. Isso é semelhante à ação padrão `"append"`, mas em vez de pegar um único valor da linha de comando e anexá-lo a uma lista existente, `"extend"` pegará vários valores em uma única string delimitada por vírgulas e estenderá uma lista existente com eles. Ou seja, se `--names` for uma opção `"extend"` do tipo `"string"`, a linha de comando

```
--names=foo,bar --names blah --names ding,dong
```

resultaria em uma lista

```
["foo", "bar", "blah", "ding", "dong"]
```

Novamente, nós definimos uma subclasse de Option:

```
class MyOption(Option):
```

```
ACTIONS = Option.ACTIONS + ("extend",)
STORE_ACTIONS = Option.STORE_ACTIONS + ("extend",)
TYPED_ACTIONS = Option.TYPED_ACTIONS + ("extend",)
ALWAYS_TYPED_ACTIONS = Option.ALWAYS_TYPED_ACTIONS + ("exter

def take_action(self, action, dest, opt, value, values, pars
    if action == "extend":
        lvalue = value.split(",")
        values.ensure_value(dest, []).extend(lvalue)
    else:
        Option.take_action(
            self, action, dest, opt, value, values, parser)
```

Características de nota:

- `"extend"` espera um valor na linha de comando e armazena esse valor em algum lugar, então ele vai em **`STORE_ACTIONS`** e **`TYPED_ACTIONS`**.

- to ensure that **`optparse`** assigns the default type of `"string"` to `"extend"` actions, we put the `"extend"` action in **`ALWAYS_TYPED_ACTIONS`** as well.

- **`MyOption.take_action()`** implements just this one new action, and passes control back to **`Option.take_action()`** for the standard **`optparse`** actions.

- `values` é uma instância da classe optparse_parser.Values, que fornece o método muito útil **`ensure_value()`**. **`ensure_value()`** é essencialmente **`getattr()`** com uma válvula de segurança; é chamado como

  ```
  values.ensure_value(attr, value)
  ```

  Se o atributo `attr` de `values` não existir ou for `None`, então ensure_value() primeiro o define como `value` e então retorna `value`. Isso é muito útil para ações como `"extend"`, `"append"` e `"count"`, todas as quais acumulam dados em uma variável e esperam que essa variável seja de um certo tipo (uma lista para as duas primeiras, um inteiro para a última). Usar **`ensure_value()`** significa que os scripts

que usam sua ação não precisam se preocupar em definir um valor padrão para os destinos de opção em questão; eles podem simplesmente deixar o padrão como `None` e **`ensure_value()`** cuidará de acertar quando for necessário.

Exceções

exception optparse.OptionError

Levantada se uma instância de **`Option`** for criada com argumentos inválidos ou inconsistentes.

exception optparse.OptionConflictError

Levantada se opções conflitantes forem adicionadas a um instância de **`OptionParser`**.

exception optparse.OptionValueError

Levantada se um valor de opção inválido for encontrado na linha de comando.

exception optparse.BadOptionError

Levantada se uma opção inválida for passada na linha de comando.

exception optparse.AmbiguousOptionError

Levantada se uma opção ambígua for passada na linha de comando.

`getpass` — Entrada de senha portátil

Código-fonte: Lib/getpass.py
[https://github.com/python/cpython/tree/3.15/Lib/getpass.py]

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

O módulo **`getpass`** fornece duas funções:

getpass.getpass(*prompt='Password: ', stream=None, *, echo_char=None*)

Solicita uma senha do usuário sem emiti-la. O usuário é solicitado usando a string *prompt*, cujo padrão é `'Password: '`. No Unix, o prompt é escrito no objeto arquivo ou similar *stream* usando o tratador de erros de substituição, se necessário. O *stream* padrão para o terminal de controle (`/dev/tty`) ou se não estiver disponível para `sys.stderr` (este argumento é ignorado no Windows).

O argumento *echo_char* controla como a entrada do usuário é exibida durante a digitação. Se *echo_char* for `None` (padrão), a entrada permanece oculta. Caso contrário, *echo_char* deve ser um único caractere ASCII imprimível e cada caractere digitado será substituído por ela. Por exemplo, `echo_char='*'` exibirá asteriscos em vez da entrada real.

Se uma entrada sem exibição em tela não estiver disponível, getpass() recorre a exibir uma mensagem de aviso para *stream* e lê de `sys.stdin` e levantar de um **`GetPassWarning`**.

Nota

Se você chamar getpass de dentro do IDLE, a entrada pode ser feita no terminal de onde você iniciou o IDLE, e não na própria janela ociosa.

Nota

On Unix systems, when *echo_char* is set, the terminal will be configured to operate in *noncanonical mode* *[https://manpages.debian.org/termios(3)#Canonical_and_noncanonical_mode]*. Common terminal control characters are supported:

- `Ctrl+A` - Move cursor to beginning of line
- `Ctrl+E` - Move cursor to end of line
- `Ctrl+K` - Kill (delete) from cursor to end of line
- `Ctrl+U` - Kill (delete) entire line
- `Ctrl+W` - Erase previous word
- `Ctrl+V` - Insert next character literally (quote)
- `Backspace`/`DEL` - Delete character before cursor

These shortcuts work by reading the terminal's configured control character mappings from termios settings.

Alterado na versão 3.14: Adicionado o parâmetro *echo_char* para feedback do teclado.

Alterado na versão 3.15: When using non-empty *echo_char* on Unix, keyboard shortcuts (including cursor movement and line editing) are now properly handled using the terminal's control character configuration.

exception getpass.GetPassWarning

A subclasse `UserWarning` é levantada quando a entrada de senha pode acabar sendo exibida na tela.

getpass.getuser()

Retorna o "nome de login" do usuário.

Esta função verifica as variáveis de ambiente **`LOGNAME`**, **`USER`**, **`LNAME`** e **`USERNAME`**, nesta ordem, e retorna o valor da primeiro que estiver definida como uma string não vazia. Se nenhuma estiver definida, o nome de login do banco de dados de senhas é retornado em sistemas que oferecem suporte ao módulo **`pwd`**, caso contrário, uma exceção **`OSError`** é levantada.

Em geral, esta função deve ter preferência sobre **`os.getlogin()`**.

Alterado na versão 3.13: Anteriormente, várias exceções além de apenas **`OSError`** eram levantadas.

`fileinput` — Itera sobre linhas de múltiplos fluxos de entrada

Código-fonte: Lib/fileinput.py
[https://github.com/python/cpython/tree/3.15/Lib/fileinput.py]

Este módulo implementa uma classe auxiliar e funções para escrever rapidamente um laço sobre uma entrada padrão ou uma lista de arquivos. Se você quiser apenas ler ou escrever um arquivo veja `open()`.

O uso típico é:

```
import fileinput
for line in fileinput.input(encoding="utf-8"):
    process(line)
```

Isto itera sobre as linhas de todos os arquivos listados em `sys.argv[1:]`, padronizando `sys.stdin` se a lista estiver vazia. Se o nome de um arquivo for `'-'`, ele também será substituído por `sys.stdin` e os argumentos opcionais *mode* e *openhook* serão ignorados. Para especificar uma lista alternativa de nomes de arquivos, passe-a como primeiro argumento para `input()`. Um único nome de arquivo também é permitido.

Todos os arquivos são abertos em modo texto por padrão, mas você pode substituir isso especificando o parâmetro *mode* na chamada para `input()` ou `FileInput`. Se ocorrer um erro de E/S durante a abertura ou leitura de um arquivo, `OSError` será levantada.

Alterado na versão 3.3: `IOError` costumava ser levantada; agora é um apelido de `OSError`.

Se `sys.stdin` for usado mais de uma vez, o segundo e posterior uso não retornará nenhuma linha, exceto talvez para uso interativo, ou se tiver sido explicitamente redefinido (por exemplo, usando `sys.stdin.seek(0)`).

Arquivos vazios são abertos e fechados imediatamente; a única vez que sua presença na lista de nomes de arquivos é perceptível é quando o último arquivo aberto está vazio.

As linhas são retornadas com novas linhas intactas, o que significa que a última linha de um arquivo pode não ter nenhuma.

Você pode controlar como os arquivos são abertos fornecendo um gancho de abertura através do parâmetro *openhook* para `fileinput.input()` ou `FileInput()`. O gancho deve ser uma função que recebe dois argumentos, *filename* e *mode*, e retorna um objeto arquivo ou similar aberto de acordo. Se *encoding* e/ou *errors* forem especificados, eles serão passados para o gancho como argumentos nomeados adicionais. Este módulo fornece um `hook_compressed()` para oferece suporte a arquivos compactados.

A seguinte função é a interface principal deste módulo:

fileinput.input(*files=None*, *inplace=False*, *backup=''*, ***, *mode='r'*, *openhook=None*, *encoding=None*, *errors=None*)

Cria uma instância da classe `FileInput`. A instância será usada como estado global para as funções deste módulo e também será retornada para uso durante a iteração. Os parâmetros desta função serão passados para o construtor da classe `FileInput`.

A instância `FileInput` pode ser usada como um gerenciador de contexto na instrução `with`. Neste exemplo, *input* é fechado após a saída da instrução `with`, mesmo se ocorrer uma exceção:

```
with fileinput.input(files=('spam.txt', 'eggs.txt'), encoding
    for line in f:
        process(line)
```

Alterado na versão 3.2: Pode ser usado como gerenciador de contexto.

Alterado na versão 3.8: Os parâmetros nomeados *mode* e *openhook* agora são somente-nomeados.

Alterado na versão 3.10: Os parâmetros somente-nomeados *encoding* e *errors* foram adicionados.

As funções a seguir usam o estado global criado por `fileinput.input()`; se não houver estado ativo, `RuntimeError` será levantada.

fileinput.filename()

Retorna o nome do arquivo que está sendo lido no momento. Antes da primeira linha ser lida, retorna `None`.

fileinput.fileno()

Retorna o número inteiro de "descritor de arquivo" para o arquivo atual. Quando nenhum arquivo é aberto (antes da primeira linha e entre arquivos), retorna `-1`.

fileinput.lineno()

Retorna o número cumulativo da linha que acabou de ser lida. Antes da primeira linha ser lida, retorna `0`. Após a leitura da última linha do último arquivo, retorna o número da linha dessa linha.

fileinput.filelineno()

Retorna o número da linha no arquivo atual. Antes da primeira linha ser lida, retorna `0`. Após a leitura da última linha do último arquivo, retorna o número da linha dessa linha no arquivo.

fileinput.isfirstline()

Retorna `True` se a linha que acabou de ler for a primeira linha do seu arquivo, caso contrário retorna `False`.

fileinput.isstdin()

Retorna `True` se a última linha foi lida em `sys.stdin`, caso contrário retorna `False`.

fileinput.nextfile()

Fecha o arquivo atual para que a próxima iteração leia a primeira linha do próximo arquivo (se houver); as linhas não lidas do arquivo não

contarão para a contagem cumulativa de linhas. O nome do arquivo não é alterado até que a primeira linha do próximo arquivo seja lida. Antes da leitura da primeira linha, esta função não tem efeito; ele não pode ser usado para pular o primeiro arquivo. Após a leitura da última linha do último arquivo, esta função não terá efeito.

fileinput.close()

Fecha a sequência.

A classe que implementa o comportamento de sequência fornecido pelo módulo também está disponível para subclasses:

class fileinput.FileInput(*files=None*, *inplace=False*, *backup=''*, ***, *mode='r'*, *openhook=None*, *encoding=None*, *errors=None*)

A classe **FileInput** é a implementação; seus métodos **filename()**, **fileno()**, **lineno()**, **filelineno()**, **isfirstline()**, **isstdin()**, **nextfile()** e **close()** correspondem às funções de mesmo nome no módulo. Além disso, é iterável e possui um método **readline()** que retorna a próxima linha de entrada. A sequência deve ser acessada em ordem estritamente sequencial; acesso aleatório e **readline()** não podem ser misturados.

Com *mode* você pode especificar qual modo de arquivo será passado para **open()**. Deve ser um entre `'r'` e `'rb'`.

O *openhook*, quando fornecido, deve ser uma função que recebe dois argumentos, *filename* e *mode*, e retorna um objeto arquivo ou similar aberto de acordo. Você não pode usar *inplace* e *openhook* juntos.

Você pode especificar *encoding* e *errors* que são passados para **open()** ou *openhook*.

Uma instância **FileInput** pode ser usada como um gerenciador de contexto na instrução **with**. Neste exemplo, *input* é fechado após a saída da instrução **with**, mesmo se ocorrer uma exceção:

```
with FileInput(files=('spam.txt', 'eggs.txt')) as input:
    process(input)
```

Alterado na versão 3.2: Pode ser usado como gerenciador de contexto.

Alterado na versão 3.8: Os parâmetros nomeados *mode* e *openhook* agora são somente-nomeados.

Alterado na versão 3.10: Os parâmetros somente-nomeados *encoding* e *errors* foram adicionados.

Alterado na versão 3.11: Os modos `'rU'` e `'U'` e o método **`__getitem__()`** foram removidos.

Filtragem local opcional: se o argumento nomeado `inplace=True` for passado para **`fileinput.input()`** ou para o construtor **`FileInput`**, o arquivo é movido para um arquivo de backup e a saída padrão é direcionada para o arquivo de entrada (se já existir um arquivo com o mesmo nome do arquivo de backup, ele será substituído silenciosamente). Isso torna possível escrever um filtro que reescreva seu arquivo de entrada internamente. Se o parâmetro *backup* for fornecido (normalmente como `backup='.<some extension>'`), ele especifica a extensão do arquivo de backup, e o arquivo de backup permanece disponível; por padrão, a extensão é `'.bak'` e é excluída quando o arquivo de saída é fechado. A filtragem local é desativada quando a entrada padrão é lida.

Os dois ganchos de abertura a seguir são fornecidos por este módulo:

fileinput.hook_compressed(*filename*, *mode*, ***, *encoding=None*, *errors=None*)

Abre de forma transparente arquivos compactados com gzip e bzip2 (reconhecidos pelas extensões `'.gz'` e `'.bz2'`) usando os módulos **`gzip`** e **`bz2`**. Se a extensão do nome do arquivo não for `'.gz'` ou `'.bz2'`, o arquivo é aberto normalmente (ou seja, usando **`open()`** sem qualquer descompactação).

Os valores *encoding* e *errors* são passados para **`io.TextIOWrapper`** para arquivos compactados e abertos para arquivos normais.

Exemplo de uso: `fi = fileinput.FileInput(openhook=fileinput.hook_compressed,`

`encoding="utf-8")`

Alterado na versão 3.10: Os parâmetros somente-nomeados *encoding* e *errors* foram adicionados.

fileinput.hook_encoded(*encoding*, *errors=None*)

Retorna um gancho que abre cada arquivo com **`open()`**, usando a *encoding* e *errors* fornecidas para ler o arquivo.

Exemplo de uso: `fi = fileinput.FileInput(openhook=fileinput.hook_encoded("utf-8", "surrogateescape"))`

Alterado na versão 3.6: Adicionado o parâmetro opcional *errors*.

Descontinuado desde a versão 3.10: Esta função foi descontinuado já que **`fileinput.input()`** e **`FileInput`** agora possuem parâmetros *encoding* e *errors*.

`curses` — Gerenciador de terminal para visualizadores de células de caracteres.

Código-fonte: Lib/curses [https://github.com/python/cpython/tree/3.15/Lib/curses]

The `curses` module provides an interface to the curses library, the de-facto standard for portable advanced terminal handling.

While curses is most widely used in the Unix environment, versions are available for Windows, DOS, and possibly other systems as well. This extension module is designed to match the API of ncurses, an open-source curses library hosted on Linux and the BSD variants of Unix.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Disponibilidade: Unix.

Nota

Whenever the documentation mentions a *character* it can be specified as an integer, a one-character Unicode string or a one-byte byte string. An integer is the code of a single encoded byte, optionally combined with attributes and a color pair, as returned by `window.inch()`.

Whenever the documentation mentions a *character string* it can be specified as a Unicode string or a byte string.

Nota

Whether curses may be used from several threads depends on the underlying library and how it was built. In many implementations, including the default build of ncurses, the screen state is shared and not thread-safe; since the blocking and refresh methods (such as `getch()` and `refresh()`) release the GIL, unsynchronized use from several threads can then crash the interpreter. Serialize the calls.

Ver também

Módulo `curses.ascii`
: Utilidades para trabalhar com caracteres ASCII, independentemente de suas configurações locais.

Módulo `curses.panel`
: Uma extensão de painel stackeada que adiciona profundidade às janelas do curses.

Módulo `curses.textpad`
: Widget de texto editável para suporte ao curses **Emacs**-like bindings.

Programação em Curses com Python
: Tutorial material on using curses with Python, by Andrew Kuchling and Eric Raymond.

Funções

O módulo `curses` define a seguinte exceção:

exception curses.error

Exceção lançada quando uma função da biblioteca curses retorna um erro.

Nota

Sempre que os argumentos *x* ou *y* de uma função ou método forem opcionais, o valor predefinido é a posição atual do cursor. Sempre que *attr* for opcional, o valor predefinido é `A_NORMAL`.

O módulo `curses` define as seguintes funções:

curses.assume_default_colors(*fg*, *bg*, /)

Permite o uso de valores predefinidos de cores em terminais que oferecem suporte a esta funcionalidade. Use isto para prover suporte à transparência na sua aplicação.

- Atribui as cores predefinidas do terminal (primeiro plano/fundo) ao número de cor `-1`. Assim, `init_pair(x, COLOR_RED, -1)` inicializará o par *x* com fundo predefinido em vermelho e `init_pair(x, -1, COLOR_BLUE)` inicializará o par *x* como primeiro plano predefinido em azul.
- Altera a definição do par de cores `0` para `(fg, bg)`.

This is an ncurses extension.

Adicionado na versão 3.14.

curses.baudrate()

Return the output speed of the terminal in bits per second. On software terminal emulators it will have a fixed high value. Included for historical reasons; in former times, it was used to write output loops for time delays and occasionally to change interfaces depending on the line speed.

curses.beep()

Emit a short attention sound.

curses.can_change_color()

Return `True` or `False`, depending on whether the programmer can change the colors displayed by the terminal.

curses.cbreak()

Enter cbreak mode. In cbreak mode (sometimes called “rare” mode) normal tty line buffering is turned off and characters are available to be read one by one. However, unlike raw mode, special characters (interrupt, quit, suspend, and flow control) retain their effects on the tty driver and calling program. Calling first **raw()** then **cbreak()** leaves the terminal in cbreak mode.

curses.color_content(*color_number*)

Return the intensity of the red, green, and blue (RGB) components in the color *color_number*, which must be between `0` and `COLORS - 1`. Return a 3-tuple, containing the R,G,B values for the given color, which will be between `0` (no component) and `1000` (maximum amount of component). Raise an exception if the color is not supported.

curses.color_pair(*pair_number*)

Return the attribute value for displaying text in the specified color pair. Only the first 256 color pairs are supported. This attribute value can be combined with **A_STANDOUT**, **A_REVERSE**, and the other **A_*** attributes. **pair_number()** is the counterpart to this function.

curses.curs_set(*visibility*)

Set the cursor state. *visibility* can be set to `0`, `1`, or `2`, for invisible, normal, or very visible. If the terminal supports the visibility requested, return the previous cursor state; otherwise raise an exception. On many terminals, the “visible” mode is an underline cursor and the “very visible” mode is a block cursor.

curses.def_prog_mode()

Save the current terminal mode as the “program” mode, the mode when the running program is using curses. (Its counterpart is the “shell”

mode, for when the program is not in curses.) Subsequent calls to `reset_prog_mode()` will restore this mode.

curses.def_shell_mode()

Save the current terminal mode as the “shell” mode, the mode when the running program is not using curses. (Its counterpart is the “program” mode, when the program is using curses capabilities.) Subsequent calls to `reset_shell_mode()` will restore this mode.

curses.delay_output(*ms*)

Insert an *ms* millisecond pause in output.

curses.doupdate()

Update the physical screen. The curses library keeps two data structures, one representing the current physical screen contents and a virtual screen representing the desired next state. The **`doupdate()`** function updates the physical screen to match the virtual screen.

The virtual screen may be updated by a `noutrefresh()` call after write operations such as `addstr()` have been performed on a window. The normal `refresh()` call is simply **`noutrefresh()`** followed by **`doupdate()`**; if you have to update multiple windows, you can speed performance and perhaps reduce screen flicker by issuing **`noutrefresh()`** calls on all windows, followed by a single **`doupdate()`**.

curses.echo()

Enter echo mode. In echo mode, each character input is echoed to the screen as it is entered.

curses.endwin()

De-initialize the library, and return terminal to normal status.

curses.erasechar()

Return the user’s current erase character as a one-byte bytes object. Under Unix operating systems this is a property of the controlling tty of the curses program, and is not set by the curses library itself.

curses.filter()

The **filter()** routine, if used, must be called before **initscr()** is called. The effect is that, during the initialization, **LINES** is set to 1; the capabilities clear, cup, cud, cud1, cuu1, cuu, vpa are disabled; and the home string is set to the value of cr. The effect is that the cursor is confined to the current line, and so are screen updates. This may be used for enabling character-at-a-time line editing without touching the rest of the screen.

curses.flash()

Flash the screen. That is, change it to reverse-video and then change it back in a short interval. Some people prefer such as 'visible bell' to the audible attention signal produced by **beep()**.

curses.flushinp()

Flush all input buffers. This throws away any typeahead that has been typed by the user and has not yet been processed by the program.

curses.getmouse()

After **getch()** returns **KEY_MOUSE** to signal a mouse event, this method should be called to retrieve the queued mouse event, represented as a 5-tuple (id, x, y, z, bstate). *id* is an ID value used to distinguish multiple devices, and *x*, *y*, *z* are the event's coordinates. (*z* is currently unused.) *bstate* is an integer value whose bits will be set to indicate the type of event, and will be the bitwise OR of one or more of the following constants, where *n* is the button number from 1 to 5: **BUTTONn_PRESSED**, **BUTTONn_RELEASED**, **BUTTONn_CLICKED**, **BUTTONn_DOUBLE_CLICKED**, **BUTTONn_TRIPLE_CLICKED**, **BUTTON_SHIFT**, **BUTTON_CTRL**, **BUTTON_ALT**.

Alterado na versão 3.10: The BUTTON5_* constants are now exposed if they are provided by the underlying curses library.

curses.getsyx()

Return the current coordinates of the virtual screen cursor as a tuple (y, x). If **leaveok** is currently True, then return (-1, -1).

curses.getwin(*file*)

Read window-related data stored in the file by an earlier `window.putwin()` call. The routine then creates and initializes a new window using that data, returning the new window object. The *file* argument must be a file object opened for reading in binary mode.

curses.has_colors()

Return `True` if the terminal can display colors; otherwise, return `False`.

curses.has_extended_color_support()

Return `True` if the module supports extended colors; otherwise, return `False`. Extended color support allows more than 256 color pairs for terminals that support more than 16 colors (for example, xterm-256color).

Extended color support requires ncurses version 6.1 or later.

Adicionado na versão 3.10.

curses.has_ic()

Return `True` if the terminal has insert- and delete-character capabilities. This function is included for historical reasons only, as all modern software terminal emulators have such capabilities.

curses.has_il()

Return `True` if the terminal has insert- and delete-line capabilities, or can simulate them using scrolling regions. This function is included for historical reasons only, as all modern software terminal emulators have such capabilities.

curses.has_key(*ch*)

Take a key value *ch*, and return `True` if the current terminal type recognizes a key with that value.

curses.halfdelay(*tenths*)

Used for half-delay mode, which is similar to cbreak mode in that characters typed by the user are immediately available to the program. However, after blocking for *tenths* tenths of seconds, raise an exception if nothing has been typed. The value of *tenths* must be a number between `1` and `255`. Use **`nocbreak()`** to leave half-delay mode.

curses.init_color(*color_number*, *r*, *g*, *b*)

Change the definition of a color, taking the number of the color to be changed followed by three RGB values (for the amounts of red, green, and blue components). The value of *color_number* must be between `0` and `COLORS - 1`. Each of *r*, *g*, *b*, must be a value between `0` and `1000`. When **`init_color()`** is used, all occurrences of that color on the screen immediately change to the new definition. This function is a no-op on most terminals; it is active only if **`can_change_color()`** returns `True`.

curses.init_pair(*pair_number*, *fg*, *bg*)

Change the definition of a color-pair. It takes three arguments: the number of the color-pair to be changed, the foreground color number, and the background color number. The value of *pair_number* must be between `1` and `COLOR_PAIRS - 1` (the `0` color pair can only be changed by **`use_default_colors()`** and **`assume_default_colors()`**). The value of *fg* and *bg* arguments must be between `0` and `COLORS - 1`, or, after calling **`use_default_colors()`** or **`assume_default_colors()`**, `-1`. If the color-pair was previously initialized, the screen is refreshed and all occurrences of that color-pair are changed to the new definition.

curses.initscr()

Initialize the library. Return a window object which represents the whole screen.

See **`setupterm()`** for a caveat about calling it before this function.

Nota

If there is an error opening the terminal, the underlying curses library may cause the interpreter to exit.

curses.intrflush(*flag*)

If *flag* is `True`, pressing an interrupt key (interrupt, break, or quit) will flush all output in the terminal driver queue. If *flag* is `False`, no flushing is done.

curses.is_term_resized(*nlines*, *ncols*)

Return `True` if **`resize_term()`** would modify the window structure, `False` otherwise.

curses.isendwin()

Return `True` if **`endwin()`** has been called (that is, the curses library has been deinitialized).

curses.keyname(*k*)

Return the name of the key numbered *k* as a bytes object. The name of a key generating printable ASCII character is the key's character. The name of a control-key combination is a two-byte bytes object consisting of a caret (`b'^'`) followed by the corresponding printable ASCII character. The name of an alt-key combination (128–255) is a bytes object consisting of the prefix `b'M-'` followed by the name of the corresponding ASCII character.

Raise a **`ValueError`** if *k* is negative.

curses.killchar()

Return the user's current line kill character as a one-byte bytes object. Under Unix operating systems this is a property of the controlling tty of the curses program, and is not set by the curses library itself.

curses.longname()

Return a bytes object containing the terminfo long name field describing the current terminal. The maximum length of a verbose description is 128 characters. It is defined only after the call to **`initscr()`**.

curses.meta(*flag*)

If *flag* is `True`, allow 8-bit characters to be input. If *flag* is `False`, allow only 7-bit chars.

curses.mouseinterval(*interval*)

Set the maximum time in milliseconds that can elapse between press and release events in order for them to be recognized as a click, and return the previous interval value. The default value is 166 milliseconds, or one sixth of a second. Use a negative *interval* to obtain the interval value without changing it.

curses.mousemask(*mousemask*)

Set the mouse events to be reported, and return a tuple `(availmask, oldmask)`. *availmask* indicates which of the specified mouse events can be reported; on complete failure it returns `0`. *oldmask* is the previous value of the mouse event mask. If this function is never called, no mouse events are ever reported.

curses.napms(*ms*)

Sleep for *ms* milliseconds.

curses.newpad(*nlines*, *ncols*)

Create and return a pointer to a new pad data structure with the given number of lines and columns. Return a pad as a window object.

A pad is like a window, except that it is not restricted by the screen size, and is not necessarily associated with a particular part of the screen. Pads can be used when a large window is needed, and only a part of the window will be on the screen at one time. Automatic refreshes of pads (such as from scrolling or echoing of input) do not occur. The **`refresh()`** and **`noutrefresh()`** methods of a pad require 6 arguments to specify the part of the pad to be displayed and the location on the screen to be used for the display. The arguments are *pminrow*, *pmincol*, *sminrow*, *smincol*, *smaxrow*, *smaxcol*; the *p* arguments refer to the upper-left corner of the pad region to be displayed and the *s* arguments define a clipping box on the screen within which the pad region is to be displayed.

curses.newwin(*nlines*, *ncols*)
curses.newwin(*nlines*, *ncols*, *begin_y*, *begin_x*)

Return a new window, whose left-upper corner is at `(begin_y, begin_x)`, and whose height/width is *nlines*/*ncols*.

By default, the window will extend from the specified position to the lower right corner of the screen.

curses.nl(*flag=True*)

Enter newline mode. This mode translates the return key into newline on input, and translates newline into return and line-feed on output. Newline mode is initially on.

If *flag* is `False`, the effect is the same as calling **`nonl()`**.

curses.nocbreak()

Leave cbreak mode. Return to normal "cooked" mode with line buffering.

curses.noecho()

Leave echo mode. Echoing of input characters is turned off.

curses.nonl()

Leave newline mode. Disable translation of return into newline on input, and disable low-level translation of newline into newline/return on output (but this does not change the behavior of `addch('\n')`, which always does the equivalent of return and line feed on the virtual screen). With translation off, curses can sometimes speed up vertical motion a little; also, it will be able to detect the return key on input.

curses.noqiflush()

When the **`noqiflush()`** routine is used, normal flush of input and output queues associated with the `INTR`, `QUIT` and `SUSP` characters will not be done. You may want to call **`noqiflush()`** in a signal handler if you want output to continue as though the interrupt had not occurred, after the handler exits.

curses.noraw()

Leave raw mode. Return to normal “cooked” mode with line buffering.

curses.pair_content(*pair_number*)

Return a tuple `(fg, bg)` containing the colors for the requested color pair. The value of *pair_number* must be between `0` and `COLOR_PAIRS - 1`.

curses.pair_number(*attr*)

Return the number of the color-pair set by the attribute value *attr*. **`color_pair()`** is the counterpart to this function.

curses.putp(*str*)

Equivalent to `tputs(str, 1, putchar)`; emit the value of a specified terminfo capability, a bytes object, for the current terminal. Note that the output of **`putp()`** always goes to standard output.

`setupterm()` (or **`initscr()`**) must be called first.

curses.qiflush([*flag*])

If *flag* is `False`, the effect is the same as calling **`noqiflush()`**. If *flag* is `True`, or no argument is provided, the queues will be flushed when these control characters are read.

curses.raw()

Enter raw mode. In raw mode, normal line buffering and processing of interrupt, quit, suspend, and flow control keys are turned off; characters are presented to curses input functions one by one.

curses.reset_prog_mode()

Restore the terminal to “program” mode, as previously saved by **`def_prog_mode()`**.

curses.reset_shell_mode()

Restore the terminal to “shell” mode, as previously saved by `def_shell_mode()`.

curses.resetty()

Restore the state of the terminal modes to what it was at the last call to `savetty()`.

curses.resize_term(*nlines*, *ncols*)

Backend function used by `resizeterm()`, performing most of the work; when resizing the windows, **`resize_term()`** blank-fills the areas that are extended. The calling application should fill in these areas with appropriate data. The **`resize_term()`** function attempts to resize all windows. However, due to the calling convention of pads, it is not possible to resize these without additional interaction with the application.

curses.resizeterm(*nlines*, *ncols*)

Resize the standard and current windows to the specified dimensions, and adjusts other bookkeeping data used by the curses library that record the window dimensions (in particular the SIGWINCH handler).

curses.savetty()

Save the current state of the terminal modes in a buffer, usable by `resetty()`.

curses.get_escdelay()

Retrieves the value set by `set_escdelay()`.

Adicionado na versão 3.9.

curses.set_escdelay(*ms*)

Sets the number of milliseconds to wait after reading an escape character, to distinguish between an individual escape character entered on the keyboard from escape sequences sent by cursor and function keys.

Adicionado na versão 3.9.

curses.get_tabsize()

Retrieves the value set by **set_tabsize()**.

Adicionado na versão 3.9.

curses.set_tabsize(*size*)

Sets the number of columns used by the curses library when converting a tab character to spaces as it adds the tab to a window.

Adicionado na versão 3.9.

curses.setsyx(*y*, *x*)

Set the virtual screen cursor to *y*, *x*. If *y* and *x* are both `-1`, then **leaveok** is set `True`.

curses.setupterm(*term=None*, *fd=-1*)

Initialize the terminal. *term* is a string giving the terminal name, or `None`; if omitted or `None`, the value of the **TERM** environment variable will be used. *fd* is the file descriptor to which any initialization sequences will be sent; if not supplied or `-1`, the file descriptor for `sys.stdout` will be used.

Raise a **curses.error** if the terminal could not be found or its terminfo database entry could not be read. If the terminal has already been initialized, this function has no effect.

Nota

Calling **initscr()** after **setupterm()** leaks the terminal that **setupterm()** allocated: the curses library keeps only a single current terminal and does not free the previously allocated one.

curses.start_color()

Must be called if the programmer wants to use colors, and before any other color manipulation routine is called. It is good practice to call this routine right after `initscr()`.

`start_color()` initializes eight basic colors (black, red, green, yellow, blue, magenta, cyan, and white), and two global variables in the `curses` module, `COLORS` and `COLOR_PAIRS`, containing the maximum number of colors and color-pairs the terminal can support. It also restores the colors on the terminal to the values they had when the terminal was just turned on.

curses.termattrs()

Return a logical OR of all video attributes supported by the terminal. This information is useful when a curses program needs complete control over the appearance of the screen.

curses.termname()

Return the value of the environment variable `TERM`, as a bytes object, truncated to 14 characters.

curses.tigetflag(*capname*)

Return the value of the Boolean capability corresponding to the terminfo capability name *capname* as an integer. Return the value `-1` if *capname* is not a Boolean capability, or `0` if it is canceled or absent from the terminal description.

`setupterm()` (or `initscr()`) must be called first.

curses.tigetnum(*capname*)

Return the value of the numeric capability corresponding to the terminfo capability name *capname* as an integer. Return the value `-2` if *capname* is not a numeric capability, or `-1` if it is canceled or absent from the terminal description.

`setupterm()` (or `initscr()`) must be called first.

curses.tigetstr(*capname*)

Return the value of the string capability corresponding to the terminfo capability name *capname* as a bytes object. Return `None` if *capname* is not a terminfo “string capability”, or is canceled or absent from the terminal description.

`setupterm()` (or `initscr()`) must be called first.

curses.tparm(*str*[, ...])

Instantiate the bytes object *str* with the supplied parameters, where *str* should be a parameterized byte string obtained from the terminfo database. For example, `tparm(tigetstr("cup"), 5, 3)` could result in `b'\033[6;4H'`, the exact result depending on terminal type. Up to nine integer parameters may be supplied.

`setupterm()` (or `initscr()`) must be called first.

curses.typeahead(*fd*)

Specify that the file descriptor *fd* be used for typeahead checking. If *fd* is `-1`, then no typeahead checking is done.

The curses library does “line-breakout optimization” by looking for typeahead periodically while updating the screen. If input is found, and it is coming from a tty, the current update is postponed until refresh or doupdate is called again, allowing faster response to commands typed in advance. This function allows specifying a different file descriptor for typeahead checking.

curses.unctrl(*ch*)

Return a bytes object which is a printable representation of the character *ch*; any attributes and color pair are ignored. Control characters are represented as a caret followed by the character, for example as `b'^C'`. Printing characters are left as they are.

curses.ungetch(*ch*)

Push *ch* so the next `getch()` will return it.

ch may be an integer (a key code or the code of an encoded byte), a byte, or a string of length 1 which encodes to a single byte.

Nota

Only one *ch* can be pushed before **`getch()`** is called.

curses.update_lines_cols()

Update the **`LINES`** and **`COLS`** module variables. Useful for detecting manual screen resize.

Adicionado na versão 3.5.

curses.unget_wch(*ch*)

Push *ch* so the next **`get_wch()`** will return it.

ch may be an integer (a character code, not a key code) or a string of length 1.

Nota

Only one *ch* can be pushed before **`get_wch()`** is called.

Adicionado na versão 3.3.

curses.ungetmouse(*id*, *x*, *y*, *z*, *bstate*)

Push a **`KEY_MOUSE`** event onto the input queue, associating the given state data with it.

curses.use_env(*flag*)

If used, this function should be called before **`initscr()`** or newterm are called. When *flag* is `False`, the values of lines and columns specified in the terminfo database will be used, even if environment variables **`LINES`** and **`COLUMNS`** (used by default) are set, or if curses is running in a window (in which case default behavior would be to use the window size if **`LINES`** and **`COLUMNS`** are not set).

curses.use_default_colors()

Equivalente a `assume_default_colors(-1, -1)`.

curses.wrapper(*func*, /, **args*, ***kwargs*)

Inicializa curses e chama outro objeto chamável, *func*, que deve ser o resto da sua aplicação que usa curses. Se a aplicação levantar uma exceção, esta função restaurará o terminal para um estado coerente antes de levantá-la novamente e gerar um traceback. O objeto chamável *func* recebe então a janela principal 'stdscr' como seu primeiro argumento, seguido de quaisquer outros argumentos passados para **`wrapper()`**. Antes de chamar *func*, **`wrapper()`** ativa o modo cbreak, desativa o eco, ativa o teclado do terminal e inicializa as cores, se o terminal tiver suporte a cores. Na saída (seja normal ou por exceção), restaura o modo cozinhado, ativa o eco e desativa o teclado do terminal.

Window objects

class curses.window

Window objects, as returned by **`initscr()`** and **`newwin()`** above, have the following methods and attributes:

window.addch(*ch*[, *attr*])

window.addch(*y*, *x*, *ch*[, *attr*])

Paint character *ch* at `(y, x)` with attributes *attr*, overwriting any character previously painted at that location. By default, the character position and attributes are the current settings for the window object.

Nota

Writing outside the window, subwindow, or pad raises a **`curses.error`**. Attempting to write to the lower-right corner of a window, subwindow, or pad will cause an exception to be raised after the character is printed.

window.addnstr(*str*, *n*[, *attr*])

window.addnstr(*y*, *x*, *str*, *n*[, *attr*])

Paint at most *n* characters of the character string *str* at `(y, x)` with attributes *attr*, overwriting anything previously on the display.

window.addstr(*str*[, *attr*])

window.addstr(*y*, *x*, *str*[, *attr*])

Paint the character string *str* at `(y, x)` with attributes *attr*, overwriting anything previously on the display.

Nota

- Writing outside the window, subwindow, or pad raises `curses.error`. Attempting to write to the lower-right corner of a window, subwindow, or pad will cause an exception to be raised after the string is printed.
- A bug in ncurses, the backend for this Python module, could cause segfaults when resizing windows. This was fixed in ncurses-6.1-20190511. If you are stuck with an earlier ncurses, you can avoid triggering it by not calling **`addstr()`** with a *str* that has embedded newlines; instead, call **`addstr()`** separately for each line.

window.attroff(*attr*)

Remove attribute *attr* from the “background” set applied to all writes to the current window.

window.attron(*attr*)

Add attribute *attr* to the “background” set applied to all writes to the current window.

window.attrset(*attr*)

Set the “background” set of attributes to *attr*. This set is initially `0` (no attributes).

window.bkgd([*ch*[, *attr*]])

Set the background property of the window to the character *ch*, with attributes *attr*. The change is then applied to every character position in that window:

- The attribute of every character in the window is changed to the new background attribute.
- Wherever the former background character appears, it is changed to the new background character.

window.bkgdset([*ch*[, *attr*]])

Set the window's background. A window's background consists of a character and any combination of attributes. The attribute part of the background is combined (OR'ed) with all non-blank characters that are written into the window. Both the character and attribute parts of the background are combined with the blank characters. The background becomes a property of the character and moves with the character through any scrolling and insert/delete line/character operations.

window.border([*ls*[, *rs*[, *ts*[, *bs*[, *tl*[, *tr*[, *bl*[, *br*]]]]]]]]])

Draw a border around the edges of the window. Each parameter specifies the character to use for a specific part of the border; see the table below for more details.

Nota

A `0` value for any parameter will cause the default character to be used for that parameter. Keyword parameters can *not* be used. The defaults are listed in this table:

Parameter	Descrição	Valor padrão
ls	Left side	`ACS_VLINE`
rs	Right side	`ACS_VLINE`

Parameter	Descrição	Valor padrão
ts	Top	`ACS_HLINE`
bs	Bottom	`ACS_HLINE`
tl	Upper-left corner	`ACS_ULCORNER`
tr	Upper-right corner	`ACS_URCORNER`
bl	Bottom-left corner	`ACS_LLCORNER`
br	Bottom-right corner	`ACS_LRCORNER`

window.box([*vertch*, *horch*])

Similar to `border()`, but both *ls* and *rs* are *vertch* and both *ts* and *bs* are *horch*. The default corner characters are always used by this function.

window.chgat(*attr*)
window.chgat(*num*, *attr*)
window.chgat(*y*, *x*, *attr*)
window.chgat(*y*, *x*, *num*, *attr*)

Set the attributes of *num* characters at the current cursor position, or at position `(y, x)` if supplied. If *num* is not given or is `-1`, the attribute will be set on all the characters to the end of the line. This function moves cursor to position `(y, x)` if supplied. The changed line will be touched using the `touchline()` method so that the contents will be redisplayed by the next window refresh.

window.clear()

Like `erase()`, but also cause the whole window to be repainted upon next call to `refresh()`.

window.clearok(*flag*)

If *flag* is `True`, the next call to **`refresh()`** will clear the window completely.

window.clrtobot()

Erase from cursor to the end of the window: all lines below the cursor are deleted, and then the equivalent of **`clrtoeol()`** is performed.

window.clrtoeol()

Erase from cursor to the end of the line.

window.cursyncup()

Update the current cursor position of all the ancestors of the window to reflect the current cursor position of the window.

window.delch([*y*, *x*])

Delete the character under the cursor, or at `(y, x)` if specified. All characters to the right on the same line are shifted one position left.

window.deleteln()

Delete the line under the cursor. All following lines are moved up by one line.

window.derwin(*begin_y*, *begin_x*)

window.derwin(*nlines*, *ncols*, *begin_y*, *begin_x*)

An abbreviation for "derive window", **`derwin()`** is the same as calling **`subwin()`**, except that *begin_y* and *begin_x* are relative to the origin of the window, rather than relative to the entire screen. Return a window object for the derived window.

window.echochar(*ch*[, *attr*])

Add character *ch* with attribute *attr*, and immediately call **`refresh()`** on the window.

window.enclose(*y*, *x*)

Test whether the given pair of screen-relative character-cell coordinates are enclosed by the given window, returning `True` or `False`. It is useful for determining what subset of the screen windows enclose the location of a mouse event.

Alterado na versão 3.10: Previously it returned `1` or `0` instead of `True` or `False`.

window.encoding

Encoding used to encode the string arguments of the methods and to decode their results on a build without wide-character support. The encoding attribute is inherited from the parent window when a subwindow is created, for example with **`window.subwin()`**. By default, current locale encoding is used (see **`locale.getencoding()`**).

Adicionado na versão 3.3.

window.erase()

Clear the window.

window.getbegyx()

Return a tuple `(y, x)` of coordinates of upper-left corner.

window.getbkgd()

Return the given window's current background character/attribute pair. Its components can be extracted like those of **`inch()`**.

window.getch([*y*, *x*])

Read a key press, after moving the cursor to *y*, *x* if specified, and return it as an integer. The window is refreshed first if it is not a pad and was modified since the last refresh. Wait until a key is pressed, or return `-1` if the read is non-blocking or times out (see **`nodelay()`** and **`timeout()`**).

An ordinary key is returned as the code of a single byte of its encoding in the current locale, so a character encoded with several bytes takes several calls. For example, in a UTF-8 locale `'é'` is read as `195`, then `169`. Use **`get_wch()`** to read it as a single character.

In keypad mode (see `keypad()`) function keys and other special keys are returned as one of the KEY_* constants, which cannot be mistaken for an ordinary key. Otherwise, or if their escape sequence does not arrive in time (see `notimeout()` and `set_escdelay()`), their bytes are returned one at a time.

In echo mode (see `echo()`) the key is added to the window as by `addch()`; special keys are not echoed.

window.get_wch([*y*, *x*])

Read a key press, after moving the cursor to *y*, *x* if specified, and return it as a one-character `str`. The window is refreshed first if it is not a pad and was modified since the last refresh. Wait until a key is pressed, or raise `error` if the read is non-blocking or times out (see `nodelay()` and `timeout()`).

In keypad mode (see `keypad()`) function keys and other special keys are returned as one of the KEY_* constants, an integer. Otherwise, or if their escape sequence does not arrive in time (see `notimeout()` and `set_escdelay()`), their characters are returned one at a time.

In echo mode (see `echo()`) the key is added to the window as by `addch()`; special keys are not echoed.

Adicionado na versão 3.3.

window.getkey([*y*, *x*])

Read a key press as `getch()` does, but return it as a `str`: an ordinary key as a one-character string, the byte decoded as Latin-1, and a special key as its name, such as `'KEY_UP'` (see `keyname()`). Raise `error` instead of returning `-1` if there is no input.

window.getmaxyx()

Return a tuple `(y, x)` of the height and width of the window.

window.getparyx()

Return the beginning coordinates of this window relative to its parent window as a tuple `(y, x)`. Return `(-1, -1)` if this window has no parent.

window.getstr()
window.getstr(*n*)
window.getstr(*y*, *x*)
window.getstr(*y*, *x*, *n*)

Read a line of input from the user, with primitive line editing capacity, after moving the cursor to *y*, *x* if specified. Return it as a bytes object, in the encoding of the current locale and without the terminating newline. At most *n* bytes are read; *n* defaults to and cannot exceed 2047.

Alterado na versão 3.14: The maximum value for *n* was increased from 1023 to 2047.

window.getyx()

Return a tuple `(y, x)` of current cursor position relative to the window's upper-left corner.

window.hline(*ch*, *n*[, *attr*])
window.hline(*y*, *x*, *ch*, *n*[, *attr*])

Display a horizontal line starting at `(y, x)` with length *n* consisting of the character *ch* with attributes *attr*. The line stops at the right edge of the window if fewer than *n* cells are available.

window.idcok(*flag*)

If *flag* is `False`, curses no longer considers using the hardware insert/delete character feature of the terminal; if *flag* is `True`, use of character insertion and deletion is enabled. When curses is first initialized, use of character insert/delete is enabled by default.

window.idlok(*flag*)

If *flag* is `True`, **`curses`** will try to use hardware line editing facilities. Otherwise, curses will not use them.

window.immedok(*flag*)

If *flag* is `True`, any change in the window image automatically causes the window to be refreshed; you no longer have to call **`refresh()`** yourself. However, it may degrade performance considerably, due to repeated calls to wrefresh. This option is disabled by default.

window.inch([*y*, *x*])

Return the character at the given position in the window. The bottom 8 bits are the character proper and the upper bits are the attributes; extract them with the **`A_CHARTEXT`** and **`A_ATTRIBUTES`** bit-masks, and the color pair with **`pair_number()`**. The character byte is the locale-encoded byte of the cell's character, consistent with **`instr()`**. On a wide-character build, a character that does not fit in a single byte in the current locale has a character byte of `0`; use **`instr()`** to read such characters.

window.insch(*ch*[, *attr*])

window.insch(*y*, *x*, *ch*[, *attr*])

Insert character *ch* with attributes *attr* before the character under the cursor, or at `(y, x)` if specified. All characters to the right of the cursor are shifted one position right, with the rightmost character on the line being lost. The cursor position does not change.

window.insdelln(*nlines*)

Insert *nlines* lines into the specified window above the current line. The *nlines* bottom lines are lost. For negative *nlines*, delete *nlines* lines starting with the one under the cursor, and move the remaining lines up. The bottom *nlines* lines are cleared. The current cursor position remains the same.

window.insertln()

Insert a blank line under the cursor. All following lines are moved down by one line.

window.insnstr(*str*, *n*[, *attr*])

window.insnstr(*y*, *x*, *str*, *n*[, *attr*])

Insert a character string (as many characters as will fit on the line) before the character under the cursor, up to *n* characters. If *n* is zero or negative, the entire string is inserted. All characters to the right of the cursor are shifted right, with the rightmost characters on the line being lost. The cursor position does not change (after moving to *y*, *x*, if specified).

window.insstr(*str*[, *attr*])

window.insstr(*y*, *x*, *str*[, *attr*])

Insert a character string (as many characters as will fit on the line) before the character under the cursor. All characters to the right of the cursor are shifted right, with the rightmost characters on the line being lost. The cursor position does not change (after moving to *y*, *x*, if specified).

window.instr([*n*])

window.instr(*y*, *x*[, *n*])

Read the text of the window from the current cursor position, or from *y*, *x* if specified, to the end of the line, and return it as a bytes object, in the encoding of the current locale. Attributes and color pairs are stripped. At most *n* bytes are read; *n* defaults to and cannot exceed 2047.

Alterado na versão 3.14: The maximum value for *n* was increased from 1023 to 2047.

window.is_linetouched(*line*)

Return `True` if the specified line was modified since the last call to `refresh()`; otherwise return `False`. Raise a `curses.error` exception if *line* is not valid for the given window.

window.is_wintouched()

Return `True` if the specified window was modified since the last call to `refresh()`; otherwise return `False`.

window.keypad(*flag*)

If *flag* is `True`, escape sequences generated by some keys (keypad, function keys) will be interpreted by **`curses`**. If *flag* is `False`, escape sequences will be left as is in the input stream. Keypad mode is disabled by default, but **`wrapper()`** enables it for the main window.

window.leaveok(*flag*)

If *flag* is `True`, cursor is left where it is on update, instead of being at "cursor position." This reduces cursor movement where possible.

If *flag* is `False`, cursor will always be at "cursor position" after an update.

window.move(*new_y*, *new_x*)

Move cursor to `(new_y, new_x)`.

window.mvderwin(*y*, *x*)

Move the window inside its parent window. The screen-relative parameters of the window are not changed. This routine is used to display different parts of the parent window at the same physical position on the screen.

window.mvwin(*new_y*, *new_x*)

Move the window so its upper-left corner is at `(new_y, new_x)`.

Moving the window so that any part of it would be off the screen is an error: the window is not moved and **`curses.error`** is raised.

window.nodelay(*flag*)

If *flag* is `True`, **`getch()`** will be non-blocking.

window.notimeout(*flag*)

If *flag* is `True`, escape sequences will not be timed out.

If *flag* is `False`, after a few milliseconds, an escape sequence will not be interpreted, and will be left in the input stream as is.

window.noutrefresh()
window.noutrefresh(*pminrow*, *pmincol*, *sminrow*, *smincol*, *smaxrow*, *smaxcol*)

Mark for refresh but wait. This function updates the data structure representing the desired state of the window, but does not force an update of the physical screen. To accomplish that, call **doupdate()**.

The 6 arguments can only be specified, and are then required, when the window is a pad created with **newpad()**; they have the same meaning as for **refresh()**.

window.overlay(*destwin*[, *sminrow*, *smincol*, *dminrow*, *dmincol*, *dmaxrow*, *dmaxcol*])

Overlay the window on top of *destwin*. The windows need not be the same size, only the overlapping region is copied. This copy is non-destructive, which means that the current background character does not overwrite the old contents of *destwin*.

To get fine-grained control over the copied region, the second form of **overlay()** can be used. *sminrow* and *smincol* are the upper-left coordinates of the source window, and the other variables mark a rectangle in the destination window.

window.overwrite(*destwin*[, *sminrow*, *smincol*, *dminrow*, *dmincol*, *dmaxrow*, *dmaxcol*])

Overwrite the window on top of *destwin*. The windows need not be the same size, in which case only the overlapping region is copied. This copy is destructive, which means that the current background character overwrites the old contents of *destwin*.

To get fine-grained control over the copied region, the second form of **overwrite()** can be used. *sminrow* and *smincol* are the upper-left coordinates of the source window, the other variables mark a rectangle in the destination window.

window.putwin(*file*)

Write all data associated with the window into the provided file object. This information can be later retrieved using the `getwin()` function.

window.redrawln(*beg*, *num*)

Indicate that the *num* screen lines, starting at line *beg*, are corrupted and should be completely redrawn on the next `refresh()` call.

window.redrawwin()

Touch the entire window, causing it to be completely redrawn on the next `refresh()` call.

window.refresh([*pminrow*, *pmincol*, *sminrow*, *smincol*, *smaxrow*, *smaxcol*])

Update the display immediately (sync actual screen with previous drawing/deleting methods).

The 6 arguments can only be specified, and are then required, when the window is a pad created with `newpad()`. The additional parameters are needed to indicate what part of the pad and screen are involved. *pminrow* and *pmincol* specify the upper-left corner of the rectangle to be displayed in the pad. *sminrow*, *smincol*, *smaxrow*, and *smaxcol* specify the edges of the rectangle to be displayed on the screen. The lower-right corner of the rectangle to be displayed in the pad is calculated from the screen coordinates, since the rectangles must be the same size. Both rectangles must be entirely contained within their respective structures. Negative values of *pminrow*, *pmincol*, *sminrow*, or *smincol* are treated as if they were zero.

window.resize(*nlines*, *ncols*)

Reallocate storage for a curses window to adjust its dimensions to the specified values. If either dimension is larger than the current values, the window's data is filled with blanks that have the current background rendition (as set by `bkgdset()`) merged into them.

window.scroll([*lines=1*])

Scroll the screen or scrolling region. Scroll upward by *lines* lines if *lines* is positive, or downward if it is negative. Scrolling has no effect unless it has been enabled for the window with **`scrollok()`**.

window.scrollok(*flag*)

Control what happens when the cursor of a window is moved off the edge of the window or scrolling region, either as a result of a newline action on the bottom line, or typing the last character of the last line. If *flag* is `False`, the cursor is left on the bottom line. If *flag* is `True`, the window is scrolled up one line. Note that in order to get the physical scrolling effect on the terminal, it is also necessary to call **`idlok()`**.

window.setscrreg(*top*, *bottom*)

Set the scrolling region from line *top* to line *bottom*. All scrolling actions will take place in this region.

window.standend()

Turn off the standout attribute. On some terminals this has the side effect of turning off all attributes.

window.standout()

Turn on attribute *A_STANDOUT*.

window.subpad(*begin_y*, *begin_x*)

window.subpad(*nlines*, *ncols*, *begin_y*, *begin_x*)

Return a sub-pad, whose upper-left corner is at `(begin_y, begin_x)`, and whose width/height is *ncols*/*nlines*. The coordinates are relative to the parent pad (unlike **`subwin()`**, which uses screen coordinates). This method is only available for pads created with **`newpad()`**.

window.subwin(*begin_y*, *begin_x*)

window.subwin(*nlines*, *ncols*, *begin_y*, *begin_x*)

Return a sub-window, whose upper-left corner is at the screen-relative coordinates `(begin_y, begin_x)`, and whose width/height is *ncols*/*nlines*.

By default, the sub-window will extend from the specified position to the lower right corner of the window.

window.syncdown()

Touch each location in the window that has been touched in any of its ancestor windows. This routine is called by **`refresh()`**, so it should almost never be necessary to call it manually.

window.syncok(*flag*)

If *flag* is `True`, then **`syncup()`** is called automatically whenever there is a change in the window.

window.syncup()

Touch all locations in ancestors of the window that have been changed in the window.

window.timeout(*delay*)

Set blocking or non-blocking read behavior for the window. If *delay* is negative, blocking read is used (which will wait indefinitely for input). If *delay* is zero, then non-blocking read is used, and **`getch()`** will return `-1` if no input is waiting. If *delay* is positive, then **`getch()`** will block for *delay* milliseconds, and return `-1` if there is still no input at the end of that time.

window.touchline(*start*, *count*[, *changed*])

Pretend *count* lines have been changed, starting with line *start*. If *changed* is supplied, it specifies whether the affected lines are marked as having been changed (*changed*=`True`) or unchanged (*changed*=`False`).

window.touchwin()

Pretend the whole window has been changed, for purposes of drawing optimizations.

window.untouchwin()

Mark all lines in the window as unchanged since the last call to **refresh()**.

window.vline(*ch*, *n*[, *attr*])

window.vline(*y*, *x*, *ch*, *n*[, *attr*])

Display a vertical line starting at `(y, x)` with length *n* consisting of the character *ch* with attributes *attr*.

Constantes

The **curses** module defines the following data members:

curses.ERR

Some curses routines that return an integer, such as **getch()**, return **ERR** upon failure.

curses.OK

Some curses routines that return an integer, such as **napms()**, return **OK** upon success.

curses.version

A bytes object representing the current version of the module.

curses.ncurses_version

A named tuple containing the three components of the ncurses library version: *major*, *minor*, and *patch*. All values are integers. The components can also be accessed by name, so `curses.ncurses_version[0]` is equivalent to `curses.ncurses_version.major` and so on.

Availability: if the ncurses library is used.

Adicionado na versão 3.8.

curses.COLORS

The maximum number of colors the terminal can support. It is defined only after the call to `start_color()`.

curses.COLOR_PAIRS

The maximum number of color pairs the terminal can support. It is defined only after the call to `start_color()`.

curses.COLS

The width of the screen, that is, the number of columns. It is defined only after the call to `initscr()`. Updated by `update_lines_cols()`, `resizeterm()` and `resize_term()`.

curses.LINES

The height of the screen, that is, the number of lines. It is defined only after the call to `initscr()`. Updated by `update_lines_cols()`, `resizeterm()` and `resize_term()`.

Some constants are available to specify character cell attributes. The exact constants available are system dependent.

Atributo	**Significado**
curses.A_ALTCHARSET	Alternate character set mode
curses.A_BLINK	Blink mode
curses.A_BOLD	Bold mode
curses.A_DIM	Dim mode
curses.A_INVIS	Invisible or blank mode

Atributo	Significado
curses.A_ITALIC	Italic mode
curses.A_NORMAL	Normal attribute
curses.A_PROTECT	Protected mode
curses.A_REVERSE	Reverse background and foreground colors
curses.A_STANDOUT	Standout mode
curses.A_UNDERLINE	Underline mode
curses.A_HORIZONTAL	Horizontal highlight
curses.A_LEFT	Left highlight
curses.A_LOW	Low highlight
curses.A_RIGHT	Right highlight
curses.A_TOP	Top highlight
curses.A_VERTICAL	Vertical highlight

Adicionado na versão 3.7: `A_ITALIC` was added.

Several constants are available to extract corresponding attributes returned by some methods.

Bit-mask	Significado
curses.A_ATTRIBUTES	Bit-mask to extract attributes
curses.A_CHARTEXT	Bit-mask to extract a character
curses.A_COLOR	Bit-mask to extract color-pair field information

Keys are referred to by integer constants with names starting with `KEY_`. The exact keycaps available are system dependent.

Key constant	Chave
curses.KEY_MIN	Minimum key value
curses.KEY_BREAK	Break key (unreliable)
curses.KEY_DOWN	Down-arrow
curses.KEY_UP	Up-arrow
curses.KEY_LEFT	Left-arrow

Key constant	Chave
curses.KEY_RIGHT	Right-arrow
curses.KEY_HOME	Home key (upward+left arrow)
curses.KEY_BACKSPACE	Backspace (unreliable)
curses.KEY_F0	Function keys. Up to 64 function keys are supported.
curses.KEY_Fn	Value of function key *n*
curses.KEY_DL	Delete line
curses.KEY_IL	Insert line
curses.KEY_DC	Delete character
curses.KEY_IC	Insert char or enter insert mode
curses.KEY_EIC	Exit insert char mode
curses.KEY_CLEAR	Clear screen

Key constant	Chave
curses.KEY_EOS	Clear to end of screen
curses.KEY_EOL	Clear to end of line
curses.KEY_SF	Scroll 1 line forward
curses.KEY_SR	Scroll 1 line backward (reverse)
curses.KEY_NPAGE	Next page
curses.KEY_PPAGE	Previous page
curses.KEY_STAB	Set tab
curses.KEY_CTAB	Clear tab
curses.KEY_CATAB	Clear all tabs
curses.KEY_ENTER	Enter or send (unreliable)
curses.KEY_SRESET	Soft (partial) reset (unreliable)
curses.KEY_RESET	Reset or hard reset (unreliable)

Key constant	Chave
curses.KEY_PRINT	Print
curses.KEY_LL	Home down or bottom (lower left)
curses.KEY_A1	Upper left of keypad
curses.KEY_A3	Upper right of keypad
curses.KEY_B2	Center of keypad
curses.KEY_C1	Lower left of keypad
curses.KEY_C3	Lower right of keypad
curses.KEY_BTAB	Back tab
curses.KEY_BEG	Beg (beginning)
curses.KEY_CANCEL	Cancelar
curses.KEY_CLOSE	Close
curses.KEY_COMMAND	Cmd (command)

Key constant	Chave
curses.KEY_COPY	Copy
curses.KEY_CREATE	Create
curses.KEY_END	End
curses.KEY_EXIT	Exit
curses.KEY_FIND	Find
curses.KEY_HELP	Help
curses.KEY_MARK	Mark
curses.KEY_MESSAGE	Message
curses.KEY_MOVE	Move
curses.KEY_NEXT	Next
curses.KEY_OPEN	Open
curses.KEY_OPTIONS	Opções

Key constant	Chave
curses.KEY_PREVIOUS	Prev (previous)
curses.KEY_REDO	Redo
curses.KEY_REFERENCE	Ref (reference)
curses.KEY_REFRESH	Refresh
curses.KEY_REPLACE	Replace
curses.KEY_RESTART	Restart
curses.KEY_RESUME	Resume
curses.KEY_SAVE	Salvar
curses.KEY_SBEG	Shifted Beg (beginning)
curses.KEY_SCANCEL	Shifted Cancel
curses.KEY_SCOMMAND	Shifted Command
curses.KEY_SCOPY	Shifted Copy

Key constant	Chave
curses.KEY_SCREATE	Shifted Create
curses.KEY_SDC	Shifted Delete char
curses.KEY_SDL	Shifted Delete line
curses.KEY_SELECT	Select
curses.KEY_SEND	Shifted End
curses.KEY_SEOL	Shifted Clear line
curses.KEY_SEXIT	Shifted Exit
curses.KEY_SFIND	Shifted Find
curses.KEY_SHELP	Shifted Help
curses.KEY_SHOME	Shifted Home
curses.KEY_SIC	Shifted Input
curses.KEY_SLEFT	Shifted Left arrow

Key constant	Chave
curses.KEY_SMESSAGE	Shifted Message
curses.KEY_SMOVE	Shifted Move
curses.KEY_SNEXT	Shifted Next
curses.KEY_SOPTIONS	Shifted Options
curses.KEY_SPREVIOUS	Shifted Prev
curses.KEY_SPRINT	Shifted Print
curses.KEY_SREDO	Shifted Redo
curses.KEY_SREPLACE	Shifted Replace
curses.KEY_SRIGHT	Shifted Right arrow
curses.KEY_SRSUME	Resumo alterado
curses.KEY_SSAVE	Shifted Save
curses.KEY_SSUSPEND	Shifted Suspend

Key constant	Chave
curses.KEY_SUNDO	Shifted Undo
curses.KEY_SUSPEND	Suspend
curses.KEY_UNDO	Desfazer
curses.KEY_MOUSE	Mouse event has occurred
curses.KEY_RESIZE	Terminal resize event
curses.KEY_MAX	Maximum key value

On VT100s and their software emulations, such as X terminal emulators, there are normally at least four function keys (`KEY_F1`, `KEY_F2`, `KEY_F3`, `KEY_F4`) available, and the arrow keys mapped to `KEY_UP`, `KEY_DOWN`, `KEY_LEFT` and `KEY_RIGHT` in the obvious way. If your machine has a PC keyboard, it is safe to expect arrow keys and twelve function keys (older PC keyboards may have only ten function keys); also, the following keypad mappings are standard:

Keycap	Constante
`Insert`	KEY_IC
`Delete`	KEY_DC

Keycap	Constante
`Home`	KEY_HOME
`End`	KEY_END
`Page Up`	KEY_PPAGE
`Page Down`	KEY_NPAGE

The following table lists characters from the alternate character set. These are inherited from the VT100 terminal, and will generally be available on software emulations such as X terminals. When there is no graphic available, curses falls back on a crude printable ASCII approximation.

Nota

These are available only after `initscr()` has been called.

Código ACS	Significado
curses.ACS_BBSS	alternate name for upper-right corner
curses.ACS_BLOCK	solid square block
curses.ACS_BOARD	board of squares

Código ACS	Significado
curses.ACS_BSBS	alternate name for horizontal line
curses.ACS_BSSB	alternate name for upper-left corner
curses.ACS_BSSS	alternate name for top tee
curses.ACS_BTEE	bottom tee
curses.ACS_BULLET	bullet
curses.ACS_CKBOARD	checker board (stipple)
curses.ACS_DARROW	arrow pointing down
curses.ACS_DEGREE	degree symbol
curses.ACS_DIAMOND	diamond
curses.ACS_GEQUAL	greater-than-or-equal-to
curses.ACS_HLINE	horizontal line
curses.ACS_LANTERN	lantern symbol

Código ACS	Significado
curses.ACS_LARROW	left arrow
curses.ACS_LEQUAL	less-than-or-equal-to
curses.ACS_LLCORNER	lower-left corner
curses.ACS_LRCORNER	lower-right corner
curses.ACS_LTEE	left tee
curses.ACS_NEQUAL	not-equal sign
curses.ACS_PI	letter pi
curses.ACS_PLMINUS	plus-or-minus sign
curses.ACS_PLUS	big plus sign
curses.ACS_RARROW	right arrow
curses.ACS_RTEE	right tee
curses.ACS_S1	scan line 1

Código ACS	Significado
curses.ACS_S3	scan line 3
curses.ACS_S7	scan line 7
curses.ACS_S9	scan line 9
curses.ACS_SBBS	alternate name for lower-right corner
curses.ACS_SBSB	alternate name for vertical line
curses.ACS_SBSS	alternate name for right tee
curses.ACS_SSBB	alternate name for lower-left corner
curses.ACS_SSBS	alternate name for bottom tee
curses.ACS_SSSB	alternate name for left tee
curses.ACS_SSSS	alternate name for crossover or big plus
curses.ACS_STERLING	pound sterling
curses.ACS_TTEE	top tee

Código ACS	Significado
curses.ACS_UARROW	up arrow
curses.ACS_ULCORNER	upper-left corner
curses.ACS_URCORNER	upper-right corner
curses.ACS_VLINE	vertical line

The following table lists mouse button constants used by `getmouse()`:

Mouse button constant	Significado
curses.BUTTONn_PRESSED	Mouse button *n* pressed
curses.BUTTONn_RELEASED	Mouse button *n* released
curses.BUTTONn_CLICKED	Mouse button *n* clicked
curses.BUTTONn_DOUBLE_CLICKED	Mouse button *n* double clicked
curses.BUTTONn_TRIPLE_CLICKED	Mouse button *n* triple clicked

Mouse button constant	Significado
curses.BUTTON_SHIFT	Shift was down during button state change
curses.BUTTON_CTRL	Control was down during button state change
curses.BUTTON_ALT	Alt was down during button state change

Alterado na versão 3.10: The `BUTTON5_*` constants are now exposed if they are provided by the underlying curses library.

The following table lists the predefined colors:

Constante	Color
curses.COLOR_BLACK	Black
curses.COLOR_BLUE	Blue
curses.COLOR_CYAN	Cyan (light greenish blue)
curses.COLOR_GREEN	Green
curses.COLOR_MAGENTA	Magenta (purplish red)

Constante	Color
curses.COLOR_RED	Red
curses.COLOR_WHITE	White
curses.COLOR_YELLOW	Yellow

`curses.textpad` — Text input widget for curses programs

The **`curses.textpad`** module provides a **`Textbox`** class that handles elementary text editing in a curses window, supporting a set of keybindings resembling those of Emacs (thus, also of Netscape Navigator, BBedit 6.x, FrameMaker, and many other programs). The module also provides a rectangle-drawing function useful for framing text boxes or for other purposes.

The module **`curses.textpad`** defines the following function:

curses.textpad.rectangle(*win*, *uly*, *ulx*, *lry*, *lrx*)

Draw a rectangle. The first argument must be a window object; the remaining arguments are coordinates relative to that window. The second and third arguments are the y and x coordinates of the upper-left corner of the rectangle to be drawn; the fourth and fifth arguments are the y and x coordinates of the lower-right corner. The rectangle will be drawn using VT100/IBM PC forms characters on terminals that make this possible (including xterm and most other software terminal emulators). Otherwise it will be drawn with ASCII dashes, vertical bars, and plus signs.

Textbox objects

You can instantiate a **`Textbox`** object as follows:

class curses.textpad.Textbox(*win*, *insert_mode=False*)

Return a textbox widget object. The *win* argument should be a curses window object in which the textbox is to be contained. If *insert_mode* is true, the textbox inserts typed characters, shifting existing text to the right, rather than overwriting it. The edit cursor of the textbox is initially located at the upper-left corner of the containing window, with coordinates `(0, 0)`. The instance's **`stripspaces`** flag is initially on.

`Textbox` objects have the following methods:

edit(*validate=None*)

This is the entry point you will normally use. It accepts editing keystrokes until one of the termination keystrokes is entered. If *validate* is supplied, it must be a function. It will be called for each keystroke entered with the keystroke as a parameter; command dispatch is done on the result. If it returns a false value, the keystroke is ignored. This method returns the window contents as a string; whether blanks in the window are included is affected by the **`stripspaces`** attribute.

do_command(*ch*)

Process a single command keystroke. Returns `1` to continue editing, or `0` if a termination keystroke was processed. Here are the supported special keystrokes:

Keystroke	Ação
`Control-A`	Go to left edge of window.
`Control-B`	Cursor left, wrapping to previous line if appropriate.

Keystroke	Ação
`Control-D`	Delete character under cursor.
`Control-E`	Go to right edge (stripspaces off) or end of line (stripspaces on).
`Control-F`	Cursor right, wrapping to next line when appropriate.
`Control-G`	Terminate, returning the window contents.
`Control-H`	Delete character backward.
`Control-J`	Terminate if the window is 1 line, otherwise move to the start of the next line.
`Control-K`	If line is blank, delete it, otherwise clear to end of line.
`Control-L`	Refresh screen.
`Control-N`	Cursor down; move down one line.
`Control-O`	Insert a blank line at cursor location.
`Control-P`	Cursor up; move up one line.

Move operations do nothing if the cursor is at an edge where the movement is not possible. The following synonyms are supported where possible:

Constante	Keystroke
`KEY_LEFT`	`Control-B`
`KEY_RIGHT`	`Control-F`

Constante	Keystroke
KEY_UP	Control-P
KEY_DOWN	Control-N
KEY_BACKSPACE	Control-h

All other keystrokes are treated as a command to insert the given character and move right (with line wrapping).

gather()

Return the window contents as a string; whether blanks in the window are included is affected by the `stripspaces` member.

stripspaces

This attribute is a flag which controls the interpretation of blanks in the window. When it is on, trailing blanks on each line are ignored; any cursor motion that would land the cursor on a trailing blank goes to the end of that line instead, and trailing blanks are stripped when the window contents are gathered.

`curses.ascii` — Utilitários para caracteres ASCII

Código-fonte: Lib/curses/ascii.py
[https://github.com/python/cpython/tree/3.15/Lib/curses/ascii.py]

The `curses.ascii` module supplies name constants for ASCII characters and functions to test membership in various ASCII character classes. The constants supplied are names for control characters as follows:

Nome	Significado
curses.ascii.NUL	
curses.ascii.SOH	Início do cabeçalho, interrupção do console
curses.ascii.STX	Início de texto
curses.ascii.ETX	Fim de texto
curses.ascii.EOT	Fim de transmissão
curses.ascii.ENQ	Consulta, segue com controle de fluxo `ACK`
curses.ascii.ACK	Confirmação

Nome	Significado
curses.ascii.BEL	Campainha
curses.ascii.BS	Backspace
curses.ascii.TAB	Tabulação
curses.ascii.HT	Apelido para `TAB`: “Tabulação horizontal”
curses.ascii.LF	Alimentação de linha ou *line feed*
curses.ascii.NL	Apelido para `LF`: “Nova linha”
curses.ascii.VT	Tabulação vertical
curses.ascii.FF	Alimentação de formulário ou *form feed*
curses.ascii.CR	Retorno de carro ou *carriage return*
curses.ascii.SO	Deslocamento para fora ou *shift-out*, inicia um conjunto de caracteres alternativo
curses.ascii.SI	Deslocamento para dentro ou *shift-in*, retoma o conjunto de caracteres padrão

Nome	Significado
curses.ascii.DLE	Escape de conexão ou *data-link escape*
curses.ascii.DC1	XON, para controle de fluxo
curses.ascii.DC2	Controle de dispositivo 2, control de fluxo em modo bloco
curses.ascii.DC3	XOFF, para controle de fluxo
curses.ascii.DC4	Controle de dispositivo 4
curses.ascii.NAK	Confirmação negativa
curses.ascii.SYN	Estado ocioso síncrono
curses.ascii.ETB	Bloco de fim de transmissão
curses.ascii.CAN	Cancelar
curses.ascii.EM	Fim de mídia
curses.ascii.SUB	Substituir

Nome	Significado
curses.ascii.ESC	Escapar
curses.ascii.FS	Separador de arquivos
curses.ascii.GS	Separador de grupos
curses.ascii.RS	Separador de registros, terminador de modo bloco
curses.ascii.US	Separador de unidades
curses.ascii.SP	Espaço
curses.ascii.DEL	Excluir

Note que muitos deles têm pouca significância prática no uso moderno. Os mnemônicos derivam de convenções de teleimpressoras que antecedem os computadores digitais.

O módulo fornece as seguintes funções, baseadas nas da biblioteca C padrão:

curses.ascii.isalnum(*c*)

Verifica se há um caractere alfanumérico ASCII; equivale a `isalpha(c) or isdigit(c)`.

curses.ascii.isalpha(*c*)

Verifica se há um caractere alfabético ASCII; equivale a `isupper(c) or islower(c)`.

curses.ascii.isascii(*c*)

Verifica se há um valor de caractere que se encaixa no conjunto ASCII de 7 bits.

curses.ascii.isblank(*c*)

Checks for an ASCII blank character; space or horizontal tab.

curses.ascii.iscntrl(*c*)

Verifica se há um caractere de controle ASCII (no intervalo de 0x00 a 0x1f ou 0x7f).

curses.ascii.isdigit(*c*)

Verifica se há um dígito decimal ASCII, `'0'` a `'9'`. Isso equivale a `c in string.digits`.

curses.ascii.isgraph(*c*)

Checks for any ASCII printable character except space.

curses.ascii.islower(*c*)

Verifica se há um caractere ASCII minúsculo.

curses.ascii.isprint(*c*)

Verifica se há algum caractere ASCII imprimível, incluindo espaço.

curses.ascii.ispunct(*c*)

Checks for any ASCII printable character which is not a space or an alphanumeric character.

curses.ascii.isspace(*c*)

Verifica caracteres de espaço em branco ASCII; espaço, quebra de linha, retorno de carro, quebra de página, tabulação horizontal, tabulação vertical.

curses.ascii.isupper(*c*)

Verifica se há uma letra maiúscula ASCII.

curses.ascii.isxdigit(*c*)

Verifica se há um dígito hexadecimal ASCII. Isso equivale a `c in string.hexdigits`.

curses.ascii.isctrl(*c*)

Checks for an ASCII control character (ordinal values 0 to 31). Unlike **`iscntrl()`**, this does not include the delete character (0x7f).

curses.ascii.ismeta(*c*)

Verifica se há um caractere não ASCII (valores ordinais 0x80 e acima).

Essas funções aceitam números inteiros ou strings de caracteres únicos; quando o argumento é uma string, ele é primeiro convertido usando a função embutida **`ord()`**.

Observe que todas essas funções verificam valores de bits ordinais derivados do caractere da string que você passa; elas não sabem nada sobre a codificação de caracteres da máquina host.

As duas funções a seguir aceitam uma string de caractere único ou um valor de byte inteiro; elas retornam um valor do mesmo tipo.

curses.ascii.ascii(*c*)

Retorna o valor ASCII correspondente aos 7 bits mais baixos de *c*.

curses.ascii.ctrl(*c*)

Retorna o caractere de controle correspondente ao caractere fornecido (é feito um E bit a bit com 0x1f sobre o valor do bit do caractere).

curses.ascii.alt(*c*)

Retorna o caractere de 8 vits correspondente ao caractere ASCII fornecido (é feito um OU bit a bit com 0x80 sobre o valor do bit do caractere).

A função a seguir aceita uma string de um único caractere ou um valor inteiro; ela retorna uma string.

curses.ascii.unctrl(*c*)

Retorna uma representação de string do caractere ASCII *c*. Se *c* for imprimível, essa string é o próprio caractere. Se o caractere for um caractere de controle (0x00–0x1f), a string consiste em um acento circunflexo (`'^'`) seguido pela letra maiúscula correspondente. Se o caractere for de exclusão ASCII (0x7f), a string é `'^?'`. Se o caractere tiver seu metabit (0x80) definido, o metabit é removido, as regras precedentes são aplicadas e `'!'` é adicionado ao resultado.

curses.ascii.controlnames

Um vetor de strings de 33 elementos que contém os mnemônicos ASCII para os trinta e dois caracteres de controle ASCII de 0 (NUL) a 0x1f (US), em ordem, mais o mnemônico `SP` para o caractere de espaço.

`curses.panel` — Uma extensão de pilha de painéis para o curses

Painéis são janelas com o recurso adicional de profundidade, então eles podem ser empilhados uns sobre os outros, e somente as partes visíveis de cada janela serão exibidas. Painéis podem ser adicionados, movidos para cima ou para baixo na pilha, e removidos.

Funções

The module **`curses.panel`** defines the following exception:

exception curses.panel.error

Exception raised when a curses panel library function returns an error.

O módulo **`curses.panel`** define as seguintes funções:

curses.panel.bottom_panel()

Retorna o painel inferior da pilha de painéis.

curses.panel.new_panel(*win*)

Returns a panel object, associating it with the given window *win* and placing the new panel on top of the panel stack. Be aware that you need to keep the returned panel object referenced explicitly. If you don't, the panel object is garbage collected and removed from the panel stack.

curses.panel.top_panel()

Retorna o painel superior da pilha de painéis.

curses.panel.update_panels()

Atualiza a tela virtual após alterações na pilha de painéis. Isso não chama `curses.doupdate()`, então você terá que fazer isso sozinho.

Panel objects

class curses.panel.panel

Panel objects, as returned by `new_panel()` above, are windows with a stacking order. There's always a window associated with a panel which determines the content, while the panel methods are responsible for the window's depth in the panel stack.

Objetos painel possuem os seguintes métodos:

panel.above()

Retorna o painel acima do painel atual.

panel.below()

Retorna o painel abaixo do painel atual.

panel.bottom()

Insere o painel para o fundo da pilha.

panel.hidden()

Retorna `True` se o painel estiver oculto (não visível), `False` caso contrário.

panel.hide()

Oculta o painel. Isso não exclui o objeto, apenas torna a janela invisível na tela.

panel.move(*y*, *x*)

Move o painel para as coordenadas `(y, x)` da tela.

panel.replace(*win*)

Altera a janela associada ao painel para a janela *win*.

panel.set_userptr(*obj*)

Define o ponteiro do usuário do painel para *obj*. Isso é usado para associar um pedaço arbitrário de dados ao painel, e pode ser qualquer objeto Python.

panel.show()

Display the panel (which might have been hidden), placing it on top of the panel stack.

panel.top()

Insere o painel para o topo da pilha.

panel.userptr()

Retorna o ponteiro do usuário para o painel. Pode ser qualquer objeto Python.

panel.window()

Retorna o objeto de janela associado ao painel.

`cmd` — Suporte para interpretadores de comando orientado a linhas

Código-fonte: Lib/cmd.py [https://github.com/python/cpython/tree/3.15/Lib/cmd.py]

A classe **`Cmd`** fornece um framework simples para escrever interpretadores de comando orientados a linhas. Eles são frequentemente úteis para melhorar testes, ferramentas administrativas e protótipos que mais tarde serão encapsulados em uma interface mais sofisticada.

class cmd.Cmd(*completekey='tab', stdin=None, stdout=None*)

Uma instância de **`Cmd`** ou instância de sua subclasse é um framework de interpretador orientado a linhas. Não há um bom motivo para instanciar **`Cmd`** em si; em vez disso, ele é útil como uma superclasse de uma classe de interpretador que você mesmo define para herdar os métodos de **`Cmd`** e encapsular métodos de ação.

O argumento opcional *completekey* é o nome **`readline`** de uma chave de conclusão; o padrão é `Tab`. Se *completekey* não for **`None`** e **`readline`** estiver disponível, a conclusão do comando será feita automaticamente.

O padrão, `'tab'`, é tratado especialmente, de modo que se refere à tecla `Tab` em cada **`readline.backend`**. Especificamente, se **`readline.backend`** for `editline`, `Cmd` usará `'^I'` em vez de `'tab'`. Observe que outros valores não são tratados dessa forma e podem funcionar apenas com um backend específico.

Os argumentos opcionais *stdin* e *stdout* especificam os objetos arquivo de entrada e saída que a instância Cmd ou instância de sua subclasse usará para entrada e saída. Se não forem especificados, eles serão padronizados para **`sys.stdin`** e **`sys.stdout`**.

Se você quiser que um determinado *stdin* seja usado, certifique-se de definir o atributo `use_rawinput` da instância como `False`, caso contrário, *stdin* será ignorado.

Alterado na versão 3.13: `completekey='tab'` é substituído por `'^I'` para `editline`.

Objetos Cmd

Uma instância `Cmd` tem os seguintes métodos:

Cmd.cmdloop(*intro=None*)

Emite um prompt repetidamente, aceite a entrada, analise um prefixo inicial da entrada recebida e despache para métodos de ação, passando a eles o restante da linha como argumento.

O argumento opcional é um banner ou uma string de introdução a ser emitida antes do primeiro prompt (isso substitui o atributo de classe `intro`).

Se o módulo `readline` for carregado, a entrada vai herdar automaticamente a edição de lista de histórico semelhante a **bash** (por exemplo, `Control-P` retorna ao último comando, `Control-N` avança para o próximo, `Control-F` move o cursor para a direita de forma não destrutiva, `Control-B` move o cursor para a esquerda de forma não destrutiva, etc.).

Um fim de arquivo na entrada é retornado como a string `'EOF'`.

Uma instância de interpretador reconhecerá um comando chamado `foo` se e somente se ele tiver um método **`do_foo()`**. Como um caso especial, uma linha começando com o caractere `'?'` é despachada para o método `do_help()`. Como outro caso especial, uma linha começando com o caractere `'!'` é despachada para o método **`do_shell()`** (se tal método for definido).

Este método retornará quando o método **postcmd()** retornar um valor true. O argumento *stop* para **postcmd()** é o valor de retorno do método **do_*()** correspondente do comando.

Se o autocompletamento estiver habilitado, os comandos serão completados automaticamente, e o autocompletamento dos argumentos dos comandos será feito chamando **complete_foo()** com os argumentos *text*, *line*, *begidx* e *endidx*. *text* é o prefixo da string que estamos tentando corresponder: todas as correspondências retornadas devem começar com ele. *line* é a linha de entrada atual com os espaços em branco iniciais removidos, *begidx* e *endidx* são os índices inicial e final do texto de prefixo, que podem ser usados para fornecer diferentes autocompletamentos dependendo da posição em que o argumento está.

Cmd.do_help(*arg*)

Todas as subclasses de **Cmd** herdam um **do_help()** predefinido. Este método, chamado com um argumento `'bar'`, invoca o método correspondente **help_bar()**, e se este não estiver presente, imprime a docstring de **do_bar()**, se disponível. Sem argumento, **do_help()** lista todos os tópicos de ajuda disponíveis (isto é, todos os comandos com métodos **help_*()** correspondentes ou comandos que têm docstrings), e também lista quaisquer comandos não documentados.

Cmd.onecmd(*str*)

Interpreta o argumento como se tivesse sido digitado em resposta ao prompt. Isso pode ser substituído, mas normalmente não precisa ser; veja os métodos **precmd()** e **postcmd()** para ganchos de execução úteis. O valor de retorno é um sinalizador que indica se a interpretação de comandos pelo interpretador deve parar. Se houver um método **do_*()** para o comando *str*, o valor de retorno desse método é retornado, caso contrário, o valor de retorno do método **default()** é retornado.

Cmd.emptyline()

Método chamado quando uma linha vazia é inserida em resposta ao prompt. Se esse método não for substituído, ele repete o último comando não vazio inserido.

Cmd.default(*line*)

Método chamado em uma linha de entrada quando o prefixo do comando não é reconhecido. Se esse método não for substituído, ele imprime uma mensagem de erro e retorna.

Cmd.completedefault(*text*, *line*, *begidx*, *endidx*)

Método chamado para completar uma linha de entrada quando nenhum método **`complete_*()`** especificado pelo comando está disponível. Por padrão, ele retorna uma lista vazia.

Cmd.columnize(*list*, *displaywidth=80*)

Método chamado para exibir uma lista de strings como um conjunto compacto de colunas. Cada coluna tem apenas a largura necessária. As colunas são separadas por dois espaços para legibilidade.

Cmd.precmd(*line*)

Método gancho executado logo antes da linha de comando *line* ser interpretada, mas depois que o prompt de entrada é gerado e emitido. Este método é um stub em **`Cmd`**; ele existe para ser substituído por subclasses. O valor de retorno é usado como o comando que será executado pelo método **`onecmd()`**; a implementação **`precmd()`** pode reescrever o comando ou simplesmente retornar *line* inalterado.

Cmd.postcmd(*stop*, *line*)

Método gancho executado logo após o término do despacho de um comando. Este método é um stub em **`Cmd`**; ele existe para ser substituído por subclasses. *line* é a linha de comando que foi executada, e *stop* é um sinalizador que indica se a execução será encerrada após a chamada para **`postcmd()`**; este será o valor de retorno do método **`onecmd()`**. O valor de retorno deste método será usado como o novo valor para o sinalizador interno que corresponde a *stop*; retornar false fará com que a interpretação continue.

Cmd.preloop()

Método gancho executado uma vez quando **`cmdloop()`** é chamado. Este método é um stub em **`Cmd`**; ele existe para ser substituído por subclasses.

Cmd.postloop()

Método gancho executado uma vez quando `cmdloop()` está para ser retornado. Este método é um stub em `Cmd`; ele existe para ser substituído por subclasses.

Instâncias das subclasses `Cmd` têm algumas variáveis de instância públicas:

Cmd.prompt

O prompt emitido para solicitar informações.

Cmd.identchars

A sequência de caracteres aceita para o prefixo do comando.

Cmd.lastcmd

O último prefixo de comando não vazio visto.

Cmd.cmdqueue

Uma lista de linhas de entrada enfileiradas. A lista cmdqueue é verificada em `cmdloop()` quando uma nova entrada é necessária; se não estiver vazia, seus elementos serão processados em ordem, como se tivessem sido inseridos no prompt.

Cmd.intro

Uma string para emitir como uma introdução ou banner. Pode ser substituída dando ao método `cmdloop()` um argumento.

Cmd.doc_header

O cabeçalho a ser emitido se a saída de ajuda tiver uma seção para comandos documentados.

Cmd.misc_header

O cabeçalho a ser emitido se a saída de ajuda tiver uma seção para tópicos de ajuda diversos (ou seja, há métodos `help_*()` sem métodos `do_*()` correspondentes).

Cmd.undoc_header

O cabeçalho a ser emitido se a saída de ajuda tiver uma seção para comandos não documentados (ou seja, houver métodos **`do_*()`** sem métodos **`help_*()`** correspondentes).

Cmd.ruler

O caractere usado para desenhar linhas separadoras sob os cabeçalhos de mensagem de ajuda. Se estiver vazio, nenhuma linha de régua será desenhada. O padrão é `'='`.

Cmd.use_rawinput

Um sinalizador, com padrão true. Se true, **`cmdloop()`** usa **`input()`** para exibir um prompt e ler o próximo comando; se false, **`sys.stdout.write()`** e **`sys.stdin.readline()`** são usados. (Isso significa que ao importar **`readline`**, em sistemas que o suportam, o interpretador suportará automaticamente edição de linha no estilo **Emacs** e pressionamentos de teclas de histórico de comando.)

Exemplo do Cmd

O módulo **`cmd`** é útil principalmente para criar shells personalizados que permitem ao usuário trabalhar com um programa interativamente.

Esta seção apresenta um exemplo simples de como construir um shell em torno de alguns dos comandos do módulo **`turtle`**.

Comandos básicos do turtle, como **`forward()`**, são adicionados a uma subclasse de **`Cmd`** com o método chamado **`do_forward()`**. O argumento é convertido em um número e despachado para o módulo turtle. A docstring é usada no utilitário de ajuda fornecido pelo shell.

O exemplo também inclui um recurso básico de gravação e reprodução implementado com o método **`precmd()`** que é responsável por converter a entrada para minúsculas e gravar os comandos em um arquivo. O método **`do_playback()`** lê o arquivo e adiciona os comandos gravados ao **`cmdqueue`** para reprodução imediata:

```
import cmd, sys
from turtle import *

class TurtleShell(cmd.Cmd):
    intro = 'Welcome to the turtle shell.   Type help or ? to li
    prompt = '(turtle) '
    file = None

    # ----- comandos básicos do turtle -----
    def do_forward(self, arg):
        'Move the turtle forward by the specified distance:  FOR
        forward(*parse(arg))
    def do_right(self, arg):
        'Turn turtle right by given number of degrees:  RIGHT 20
        right(*parse(arg))
    def do_left(self, arg):
        'Turn turtle left by given number of degrees:  LEFT 90'
        left(*parse(arg))
    def do_goto(self, arg):
        'Move turtle to an absolute position with changing orier
        goto(*parse(arg))
    def do_home(self, arg):
        'Return turtle to the home position:  HOME'
        home()
    def do_circle(self, arg):
        'Draw circle with given radius an options extent and ste
        circle(*parse(arg))
    def do_position(self, arg):
        'Print the current turtle position:  POSITION'
        print('Current position is %d %d\n' % position())
    def do_heading(self, arg):
        'Print the current turtle heading in degrees:  HEADING'
        print('Current heading is %d\n' % (heading(),))
    def do_color(self, arg):
        'Set the color:  COLOR BLUE'
        color(arg.lower())
    def do_undo(self, arg):
        'Undo (repeatedly) the last turtle action(s):  UNDO'
    def do_reset(self, arg):
        'Clear the screen and return turtle to center:  RESET'
        reset()
    def do_bye(self, arg):
        'Stop recording, close the turtle window, and exit:  BYE
```

```
        print('Thank you for using Turtle')
        self.close()
        bye()
        return True

    # ----- registra e reproduz -----
    def do_record(self, arg):
        'Save future commands to filename:  RECORD rose.cmd'
        self.file = open(arg, 'w')
    def do_playback(self, arg):
        'Playback commands from a file:  PLAYBACK rose.cmd'
        self.close()
        with open(arg) as f:
            self.cmdqueue.extend(f.read().splitlines())
    def precmd(self, line):
        line = line.lower()
        if self.file and 'playback' not in line:
            print(line, file=self.file)
        return line
    def close(self):
        if self.file:
            self.file.close()
            self.file = None

def parse(arg):
    'Convert a series of zero or more numbers to an argument tup
    return tuple(map(int, arg.split()))

if __name__ == '__main__':
    TurtleShell().cmdloop()
```

Aqui está uma sessão de exemplo com o shell do turtle mostrando as funções de ajuda, usando linhas em branco para repetir comandos e o recurso simples de gravação e reprodução:

```
Welcome to the turtle shell.   Type help or ? to list commands.

(turtle) ?

Documented commands (type help <topic>):
========================================
bye     color    goto     home  playback  record  right
```

```
circle  forward  heading  left  position  reset   undo

(turtle) help forward
Move the turtle forward by the specified distance:  FORWARD 10
(turtle) record spiral.cmd
(turtle) position
Current position is 0 0

(turtle) heading
Current heading is 0

(turtle) reset
(turtle) circle 20
(turtle) right 30
(turtle) circle 40
(turtle) right 30
(turtle) circle 60
(turtle) right 30
(turtle) circle 80
(turtle) right 30
(turtle) circle 100
(turtle) right 30
(turtle) circle 120
(turtle) right 30
(turtle) circle 120
(turtle) heading
Current heading is 180

(turtle) forward 100
(turtle)
(turtle) right 90
(turtle) forward 100
(turtle)
(turtle) right 90
(turtle) forward 400
(turtle) right 90
(turtle) forward 500
(turtle) right 90
(turtle) forward 400
(turtle) right 90
(turtle) forward 300
(turtle) playback spiral.cmd
Current position is 0 0
```

```
Current heading is 0

Current heading is 180

(turtle) bye
Thank you for using Turtle
```

Execução Concorrente

Os módulos descritos neste capítulo fornecem suporte a execução simultânea de código. A escolha apropriada da ferramenta dependerá da tarefa a ser executada (CPU bound ou IO bound) e do estilo de desenvolvimento preferencial (multitarefa cooperativa orientada a eventos versus multitarefa preemptiva). Eis uma visão geral:

- `threading` — Paralelismo baseado em threads
  - Introdução
  - GIL and performance considerations
  - Referência
    - Thread-local data
    - Thread objects
    - Lock objects
    - RLock objects
    - Condition objects
    - Semaphore objects
    - `Semaphore` example
    - Event objects
    - Timer objects
    - Barrier objects
  - Using locks, conditions, and semaphores in the `with` statement
  - Iterator synchronization
- `multiprocessing` — Paralelismo baseado em processo
  - Introdução
    - A classe `Process`
    - Contextos e métodos de inicialização
    - Trocando objetos entre processos
    - Sincronização entre processos
    - Compartilhando estado entre processos
    - Usando um pool de workers
  - Referência
    - Global start method
    - `Process` e exceções

        - Encadeamentos e filas
        - Diversos
        - Objetos de conexão
        - Primitivas de sincronização
        - Objetos `ctypes` compartilhados
            - The `multiprocessing.sharedctypes` module
        - Gerenciadores
            - Customized managers
            - Using a remote manager
        - Objetos proxies
            - Limpeza
        - Process Pools
        - Listeners and Clients
            - Formatos de Endereços
        - Authentication keys
        - Gerando logs
        - The `multiprocessing.dummy` module
    - Programming guidelines
        - All start methods
        - The *spawn* and *forkserver* start methods
    - Exemplos
- `multiprocessing.shared_memory` — Memória compartilhada para acesso direto entre processos
- O pacote `concurrent`
- `concurrent.futures` — Launching parallel tasks
    - Executor Objects
    - ThreadPoolExecutor
        - Exemplo de ThreadPoolExecutor
    - InterpreterPoolExecutor
    - ProcessPoolExecutor
        - ProcessPoolExecutor Example
    - Future Objects
    - Module Functions
    - Exception classes
- `concurrent.interpreters` — Múltiplos interpretadores no mesmo processo
    - Detalhes chaves

- Introdução
  - Multiple Interpreters and Isolation
  - Running in an Interpreter
  - Concurrency and Parallelism
  - Communication Between Interpreters
  - “Sharing” Objects
- Referência
  - Objetos interpretador
  - Exceções
  - Communicating Between Interpreters
- Uso básico

- `subprocess` — Gerenciamento de subprocessos
  - Using the `subprocess` Module
    - Argumentos usados frequentemente
    - Construtor Popen
    - Exceções
  - Considerações de Segurança
  - Objetos Popen
  - Windows Popen Helpers
    - Constantes do Windows
  - API de alto nível mais antiga
  - Replacing Older Functions with the `subprocess` Module
    - Replacing **/bin/sh** shell command substitution
    - Replacing shell pipeline
    - Substituindo `os.system()`
    - Replacing the `os.spawn` family
    - Replacing `os.popen()`
  - Legacy Shell Invocation Functions
  - Notas
    - Timeout Behavior
    - Converter uma sequência de argumentos em uma string no Windows
    - Disable use of `posix_spawn()`
- `sched` — Agendador de eventos
  - Objetos Scheduler
- `queue` — Uma classe de fila sincronizada
  - Objetos Queue

        - Waiting for task completion
        - Terminating queues
    - Objetos SimpleQueue
- `contextvars` — Variáveis de contexto
    - Variáveis de contexto
    - Gerenciamento de contexto manual
    - Suporte a asyncio

A seguir, os módulos de suporte para alguns dos serviços acima:

- `_thread` — API de segmentação de baixo nível

`threading` — Paralelismo baseado em threads

Código-fonte: Lib/threading.py
[https://github.com/python/cpython/tree/3.15/Lib/threading.py]

Este módulo constrói interfaces de alto nível para threading usando o módulo `_thread`, de mais baixo nível.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Introdução

O módulo `threading` fornece uma maneira de executar múltiplas threads [https://en.wikipedia.org/wiki/Thread_(computing)] (unidades menores de um processo) simultaneamente dentro de um único processo. Ele permite a criação e o gerenciamento de threads, possibilitando a execução de tarefas em paralelo, compartilhando espaço de memória. Threads são particularmente úteis quando tarefas são limitadas por E/S, como operações de arquivo ou solicitações de rede, onde grande parte do tempo é gasto aguardando recursos externos.

Um caso de uso típico para `threading` inclui o gerenciamento de um conjunto de threads de trabalho que podem processar múltiplas tarefas simultaneamente. Aqui está um exemplo básico de criação e inicialização de threads usando `Thread`:

```
import threading
import time
```

```
def crawl(link, delay=3):
    print(f"crawl started for {link}")
    time.sleep(delay)  # Blocking I/O (simulating a network requ
    print(f"crawl ended for {link}")

links = [
    "https://python.org",
    "https://docs.python.org",
    "https://peps.python.org",
]

# Start threads for each link
threads = []
for link in links:
    # Using `args` to pass positional arguments and `kwargs` for
    t = threading.Thread(target=crawl, args=(link,), kwargs={"de
    threads.append(t)

# Start each thread
for t in threads:
    t.start()

# Wait for all threads to finish
for t in threads:
    t.join()
```

Alterado na versão 3.7: Este módulo costumava ser opcional, agora está sempre disponível.

Ver também

`concurrent.futures.ThreadPoolExecutor` offers a higher level interface to push tasks to a background thread without blocking execution of the calling thread, while still being able to retrieve their results when needed.

`queue` provides a thread-safe interface for exchanging data between running threads.

`asyncio` offers an alternative approach to achieving task level concurrency without requiring the use of multiple operating system

threads.

Nota

In the Python 2.x series, this module contained `camelCase` names for some methods and functions. These are deprecated as of Python 3.10, but they are still supported for compatibility with Python 2.5 and lower.

Detalhes da implementação do CPython: In CPython, due to the Global Interpreter Lock, only one thread can execute Python code at once (even though certain performance-oriented libraries might overcome this limitation). If you want your application to make better use of the computational resources of multi-core machines, you are advised to use `multiprocessing` or `concurrent.futures.ProcessPoolExecutor`. However, threading is still an appropriate model if you want to run multiple I/O-bound tasks simultaneously.

GIL and performance considerations

Unlike the `multiprocessing` module, which uses separate processes to bypass the global interpreter lock (GIL), the threading module operates within a single process, meaning that all threads share the same memory space. However, the GIL limits the performance gains of threading when it comes to CPU-bound tasks, as only one thread can execute Python bytecode at a time. Despite this, threads remain a useful tool for achieving concurrency in many scenarios.

As of Python 3.13, free-threaded builds can disable the GIL, enabling true parallel execution of threads, but this feature is not available by default (see **PEP 703** [https://peps.python.org/pep-0703/]).

Referência

Este módulo define as seguintes funções:

threading.active_count()

Retorna o número de objetos `Thread` atualmente ativos. A quantidade retornada é igual ao tamanho da lista retornada por `enumerate()`.

The function `activeCount` is a deprecated alias for this function.

threading.current_thread()

Return the current `Thread` object, corresponding to the caller's thread of control. If the caller's thread of control was not created through the **`threading`** module, a dummy thread object with limited functionality is returned.

The function `currentThread` is a deprecated alias for this function.

threading.excepthook(*args*, /)

Handle uncaught exception raised by `Thread.run()`.

O argumento *args* tem os seguintes atributos:

- *exc_type*: Tipo de exceção..
- *exc_value*: Valor da exceção, pode ser `None`.
- *exc_traceback*: Pilha de execução da exceção, pode ser `None`.
- *thread*: Thread que levantou a exceção, pode ser `None`.

Se *exc_type* é `SystemExit`, a exceção é silenciosamente ignorada. Caso contrário, a exceção é exibida em `sys.stderr`.

Se esta função levantar uma exceção, `sys.excepthook()` é chamada para tratá-la.

`threading.excepthook()` pode ser substituída para controlar como exceções não capturadas levantadas por `Thread.run()` são tratadas.

Storing *exc_value* using a custom hook can create a reference cycle. It should be cleared explicitly to break the reference cycle when the exception is no longer needed.

Storing *thread* using a custom hook can resurrect it if it is set to an object which is being finalized. Avoid storing *thread* after the custom hook completes to avoid resurrecting objects.

Ver também

`sys.excepthook()` trata exceções não capturadas.

Adicionado na versão 3.8.

threading.__excepthook__

Holds the original value of `threading.excepthook()`. It is saved so that the original value can be restored in case they happen to get replaced with broken or alternative objects.

Adicionado na versão 3.10.

threading.get_ident()

Retorna o 'identificador de thread' do thread atual. Este é um número inteiro diferente de zero. Seu valor não tem significado direto; pretende-se que seja um cookie mágico para ser usado, por exemplo, para indexar um dicionário de dados específicos do thread. identificadores de thread podem ser reciclados quando um thread sai e outro é criado.

Adicionado na versão 3.3.

threading.get_native_id()

Retorna a ID de thread integral nativa da thread atual atribuída pelo kernel. Este é um número inteiro não negativo. Seu valor pode ser usado para identificar exclusivamente essa thread específica em todo o sistema (até que a thread termine, após o que o valor poderá ser reciclado pelo sistema operacional).

Disponibilidade: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD, GNU/kFreeBSD, Solaris.

Adicionado na versão 3.8.

Alterado na versão 3.13: Adicionado suporte a GNU/kFreeBSD.

Alterado na versão 3.15: Adicionado suporte para Solaris.

threading.enumerate()

Return a list of all `Thread` objects currently active. The list includes daemonic threads and dummy thread objects created by `current_thread()`. It excludes terminated threads and threads that have not yet been started. However, the main thread is always part of the result, even when terminated.

threading.main_thread()

Return the main `Thread` object. In normal conditions, the main thread is the thread from which the Python interpreter was started.

Adicionado na versão 3.4.

threading.settrace(*func*)

Set a trace function for all threads started from the `threading` module. The *func* will be passed to `sys.settrace()` for each thread, before its `run()` method is called.

threading.settrace_all_threads(*func*)

Set a trace function for all threads started from the `threading` module and all Python threads that are currently executing.

The *func* will be passed to `sys.settrace()` for each thread, before its `run()` method is called.

Adicionado na versão 3.12.

threading.gettrace()

Obtém a função trace conforme definido por `settrace()`.

Adicionado na versão 3.10.

threading.setprofile(*func*)

Set a profile function for all threads started from the **threading** module. The *func* will be passed to **sys.setprofile()** for each thread, before its **run()** method is called.

threading.setprofile_all_threads(*func*)

Set a profile function for all threads started from the **threading** module and all Python threads that are currently executing.

The *func* will be passed to **sys.setprofile()** for each thread, before its **run()** method is called.

Adicionado na versão 3.12.

threading.getprofile()

Obtém a função do criador de perfil conforme definido por **setprofile()**.

Adicionado na versão 3.10.

threading.stack_size([*size*])

Retorna o tamanho da pilha de threads usado ao criar novos threads. O argumento opcional *size* especifica o tamanho da pilha a ser usado para threads criados posteriormente e deve ser 0 (usar plataforma ou padrão configurado) ou um valor inteiro positivo de pelo menos 32.768 (32 KiB). Se *size* não for especificado, 0 será usado. Se a alteração do tamanho da pilha de threads não for suportada, uma **RuntimeError** será levantada. Se o tamanho da pilha especificado for inválido, uma **ValueError** será levantada e o tamanho da pilha não será modificado. Atualmente, 0 KiB é o valor mínimo de tamanho de pilha suportado para garantir espaço suficiente para o próprio interpretador. Observe que algumas plataformas podem ter restrições específicas sobre valores para o tamanho da pilha, como exigir um tamanho mínimo de pilha > 32 KiB ou exigir alocação em múltiplos do tamanho da página de memória do sistema – a documentação da plataforma deve ser consultada para obter mais informações (4 páginas KiB são comuns; usar múltiplos de 4096 para o tamanho da pilha é a abordagem sugerida na ausência de informações mais específicas).

Disponibilidade: Windows, pthreads.

Plataformas Unix com suporte a threads POSIX.

This module also defines the following constant:

threading.TIMEOUT_MAX

The maximum value allowed for the *timeout* parameter of blocking functions (`Lock.acquire()`, `RLock.acquire()`, `Condition.wait()`, etc.). Specifying a timeout greater than this value will raise an `OverflowError`.

Adicionado na versão 3.2.

This module defines a number of classes, which are detailed in the sections below.

The design of this module is loosely based on Java's threading model. However, where Java makes locks and condition variables basic behavior of every object, they are separate objects in Python. Python's `Thread` class supports a subset of the behavior of Java's Thread class; currently, there are no priorities, no thread groups, and threads cannot be destroyed, stopped, suspended, resumed, or interrupted. The static methods of Java's Thread class, when implemented, are mapped to module-level functions.

All of the methods described below are executed atomically.

Thread-local data

Thread-local data is data whose values are thread specific. If you have data that you want to be local to a thread, create a `local` object and use its attributes:

```
>>> mydata = local()
>>> mydata.number = 42
>>> mydata.number
42
```

You can also access the `local`-object's dictionary:

```
>>> mydata.__dict__
{'number': 42}
>>> mydata.__dict__.setdefault('widgets', [])
[]
>>> mydata.widgets
[]
```

If we access the data in a different thread:

```
>>> log = []
>>> def f():
...     items = sorted(mydata.__dict__.items())
...     log.append(items)
...     mydata.number = 11
...     log.append(mydata.number)

>>> import threading
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
>>> log
[[], 11]
```

we get different data. Furthermore, changes made in the other thread don't affect data seen in this thread:

```
>>> mydata.number
42
```

Of course, values you get from a **local** object, including their **__dict__** attribute, are for whatever thread was current at the time the attribute was read. For that reason, you generally don't want to save these values across threads, as they apply only to the thread they came from.

You can create custom **local** objects by subclassing the **local** class:

```
>>> class MyLocal(local):
...     number = 2
...     def __init__(self, /, **kw):
...         self.__dict__.update(kw)
...     def squared(self):
...         return self.number ** 2
```

This can be useful to support default values, methods and initialization. Note that if you define an `__init__()` method, it will be called each time the `local` object is used in a separate thread. This is necessary to initialize each thread's dictionary.

Now if we create a `local` object:

```
>>> mydata = MyLocal(color='red')
```

we have a default number:

```
>>> mydata.number
2
```

an initial color:

```
>>> mydata.color
'red'
>>> del mydata.color
```

And a method that operates on the data:

```
>>> mydata.squared()
4
```

As before, we can access the data in a separate thread:

```
>>> log = []
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
>>> log
[[('color', 'red')], 11]
```

without affecting this thread's data:

```
>>> mydata.number
2
>>> mydata.color
Traceback (most recent call last):
```

```
...
AttributeError: 'MyLocal' object has no attribute 'color'
```

Note that subclasses can define __slots__, but they are not thread local. They are shared across threads:

```
>>> class MyLocal(local):
...     __slots__ = 'number'

>>> mydata = MyLocal()
>>> mydata.number = 42
>>> mydata.color = 'red'
```

So, the separate thread:

```
>>> thread = threading.Thread(target=f)
>>> thread.start()
>>> thread.join()
```

affects what we see:

```
>>> mydata.number
11
```

class threading.local

A class that represents thread-local data.

Thread objects

The **Thread** class represents an activity that is run in a separate thread of control. There are two ways to specify the activity: by passing a callable object to the constructor, or by overriding the **run()** method in a subclass. No other methods (except for the constructor) should be overridden in a subclass. In other words, *only* override the `__init__()` and **run()** methods of this class.

Once a thread object is created, its activity must be started by calling the thread's **start()** method. This invokes the **run()** method in a separate thread of control.

Once the thread's activity is started, the thread is considered 'alive'. It stops being alive when its `run()` method terminates – either normally, or by raising an unhandled exception. The `is_alive()` method tests whether the thread is alive.

Other threads can call a thread's `join()` method. This blocks the calling thread until the thread whose **`join()`** method is called is terminated.

A thread has a name. The name can be passed to the constructor, and read or changed through the `name` attribute.

If the `run()` method raises an exception, `threading.excepthook()` is called to handle it. By default, **`threading.excepthook()`** ignores silently `SystemExit`.

A thread can be flagged as a "daemon thread". The significance of this flag is that the entire Python program exits when only daemon threads are left. The initial value is inherited from the creating thread. The flag can be set through the `daemon` property or the *daemon* constructor argument.

Nota

Daemon threads are abruptly stopped at shutdown. Their resources (such as open files, database transactions, etc.) may not be released properly. If you want your threads to stop gracefully, make them non-daemonic and use a suitable signalling mechanism such as an `Event`.

There is a "main thread" object; this corresponds to the initial thread of control in the Python program. It is not a daemon thread.

There is the possibility that "dummy thread objects" are created. These are thread objects corresponding to "alien threads", which are threads of control started outside the threading module, such as directly from C code. Dummy thread objects have limited functionality; they are always considered alive and daemonic, and cannot be joined. They are never deleted, since it is impossible to detect the termination of alien threads.

class threading.Thread(*group=None*, *target=None*, *name=None*, *args=()*, *kwargs={}*, *, *daemon=None*, *context=None*)

This constructor should always be called with keyword arguments. Arguments are:

group must be `None` as it is reserved for future extension when a **`ThreadGroup`** class is implemented.

target is the callable object to be invoked by the **`run()`** method. Defaults to `None`, meaning nothing is called.

name is the thread name. By default, a unique name is constructed of the form "Thread-*N*" where *N* is a small decimal number, or "Thread-*N* (target)" where "target" is `target.__name__` if the *target* argument is specified.

args is a list or tuple of arguments for the target invocation. Defaults to `()`.

kwargs is a dictionary of keyword arguments for the target invocation. Defaults to `{}`.

If not `None`, *daemon* explicitly sets whether the thread is daemonic. If `None` (the default), the daemonic property is inherited from the current thread.

context is the **`Context`** value to use when starting the thread. The default value is `None` which indicates that the **`sys.flags.thread_inherit_context`** flag controls the behaviour. If the flag is true, threads will start with a copy of the context of the caller of **`start()`**. If false, they will start with an empty context. To explicitly start with an empty context, pass a new instance of **`Context()`**. To explicitly start with a copy of the current context, pass the value from **`copy_context()`**. The flag defaults true on free-threaded builds and false otherwise.

If the subclass overrides the constructor, it must make sure to invoke the base class constructor (`Thread.__init__()`) before doing anything else

to the thread.

Alterado na versão 3.3: Adicionado o parâmetro *daemon*.

Alterado na versão 3.10: Use the *target* name if *name* argument is omitted.

Alterado na versão 3.14: Adicionado o parâmetro *context*.

start()

Start the thread's activity.

It must be called at most once per thread object. It arranges for the object's `run()` method to be invoked in a separate thread of control.

This method will raise a `RuntimeError` if called more than once on the same thread object.

If supported, set the operating system thread name to `threading.Thread.name`. The name can be truncated depending on the operating system thread name limits.

Alterado na versão 3.14: Set the operating system thread name.

run()

Method representing the thread's activity.

You may override this method in a subclass. The standard `run()` method invokes the callable object passed to the object's constructor as the *target* argument, if any, with positional and keyword arguments taken from the *args* and *kwargs* arguments, respectively.

Using list or tuple as the *args* argument which passed to the `Thread` could achieve the same effect.

Exemplo:

```
>>> from threading import Thread
>>> t = Thread(target=print, args=[1])
>>> t.run()
1
>>> t = Thread(target=print, args=(1,))
>>> t.run()
1
```

join(*timeout=None*)

Wait until the thread terminates. This blocks the calling thread until the thread whose **`join()`** method is called terminates – either normally or through an unhandled exception – or until the optional timeout occurs.

When the *timeout* argument is present and not `None`, it should be a real number specifying a timeout for the operation in seconds (or fractions thereof). As **`join()`** always returns `None`, you must call **`is_alive()`** after **`join()`** to decide whether a timeout happened – if the thread is still alive, the **`join()`** call timed out.

When the *timeout* argument is not present or `None`, the operation will block until the thread terminates.

A thread can be joined many times.

`join()` raises a **`RuntimeError`** if an attempt is made to join the current thread as that would cause a deadlock. It is also an error to **`join()`** a thread before it has been started and attempts to do so raise the same exception.

If an attempt is made to join a running daemonic thread in late stages of Python finalization **`join()`** raises a **`PythonFinalizationError`**.

Alterado na versão 3.14: May raise **`PythonFinalizationError`**.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

name

A string used for identification purposes only. It has no semantics. Multiple threads may be given the same name. The initial name is set by the constructor.

On some platforms, the thread name is set at the operating system level when the thread starts, so that it is visible in task managers. This name may be truncated to fit in a system-specific limit (for example, 15 bytes on Linux or 63 bytes on macOS).

Changes to *name* are only reflected at the OS level when the currently running thread is renamed. (Setting the *name* attribute of a different thread only updates the Python Thread object.)

getName()

setName()

Deprecated getter/setter API for `name`; use it directly as a property instead.

Descontinuado desde a versão 3.10.

ident

The ‘thread identifier’ of this thread or `None` if the thread has not been started. This is a nonzero integer. See the `get_ident()` function. Thread identifiers may be recycled when a thread exits and another thread is created. The identifier is available even after the thread has exited.

native_id

The Thread ID (`TID`) of this thread, as assigned by the OS (kernel). This is a non-negative integer, or `None` if the thread has not been started. See the `get_native_id()` function. This value may be used to uniquely identify this particular thread system-wide (until the thread terminates, after which the value may be recycled by the OS).

Nota

Similar to Process IDs, Thread IDs are only valid (guaranteed unique system-wide) from the time the thread is created until the thread has been terminated.

Disponibilidade: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD.

Adicionado na versão 3.8.

is_alive()

Return whether the thread is alive.

This method returns `True` just before the **`run()`** method starts until just after the **`run()`** method terminates. The module function **`enumerate()`** returns a list of all alive threads.

daemon

A boolean value indicating whether this thread is a daemon thread (`True`) or not (`False`). This must be set before **`start()`** is called, otherwise **`RuntimeError`** is raised. Its initial value is inherited from the creating thread; the main thread is not a daemon thread and therefore all threads created in the main thread default to **`daemon`** = `False`.

The entire Python program exits when no alive non-daemon threads are left.

isDaemon()

setDaemon()

Deprecated getter/setter API for **`daemon`**; use it directly as a property instead.

Descontinuado desde a versão 3.10.

Lock objects

A primitive lock is a synchronization primitive that is not owned by a particular thread when locked. In Python, it is currently the lowest level synchronization primitive available, implemented directly by the `_thread` extension module.

A primitive lock is in one of two states, “locked” or “unlocked”. It is created in the unlocked state. It has two basic methods, `acquire()` and `release()`. When the state is unlocked, **`acquire()`** changes the state to locked and returns immediately. When the state is locked, **`acquire()`** blocks until a call to **`release()`** in another thread changes it to unlocked, then the **`acquire()`** call resets it to locked and returns. The **`release()`** method should only be called in the locked state; it changes the state to unlocked and returns immediately. If an attempt is made to release an unlocked lock, a `RuntimeError` will be raised.

Locks also support the context management protocol.

When more than one thread is blocked in `acquire()` waiting for the state to turn to unlocked, only one thread proceeds when a `release()` call resets the state to unlocked; which one of the waiting threads proceeds is not defined, and may vary across implementations.

All methods are executed atomically.

class threading.Lock

The class implementing primitive lock objects. Once a thread has acquired a lock, subsequent attempts to acquire it block, until it is released; any thread may release it.

Alterado na versão 3.13: `Lock` is now a class. In earlier Pythons, `Lock` was a factory function which returned an instance of the underlying private lock type.

acquire(*blocking=True*, *timeout=-1*)

Adquire uma trava, bloqueante ou não.

When invoked with the *blocking* argument set to `True` (the default), block until the lock is unlocked, then set it to locked and return

`True`.

When invoked with the *blocking* argument set to `False`, do not block. If a call with *blocking* set to `True` would block, return `False` immediately; otherwise, set the lock to locked and return `True`.

When invoked with the *timeout* argument set to a positive value, block for at most the number of seconds specified by *timeout* and as long as the lock cannot be acquired. A *timeout* argument of `-1` specifies an unbounded wait. It is forbidden to specify a *timeout* when *blocking* is `False`.

The return value is `True` if the lock is acquired successfully, `False` if not (for example if the *timeout* expired).

Alterado na versão 3.2: O parâmetro *timeout* é novo.

Alterado na versão 3.2: Lock acquisition can now be interrupted by signals on POSIX if the underlying threading implementation supports it.

Alterado na versão 3.14: Lock acquisition can now be interrupted by signals on Windows.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

release()

Release a lock. This can be called from any thread, not only the thread which has acquired the lock.

When the lock is locked, reset it to unlocked, and return. If any other threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed.

When invoked on an unlocked lock, a `RuntimeError` is raised.

There is no return value.

locked()

Return `True` if the lock is acquired.

RLock objects

A reentrant lock is a synchronization primitive that may be acquired multiple times by the same thread. Internally, it uses the concepts of "owning thread" and "recursion level" in addition to the locked/unlocked state used by primitive locks. In the locked state, some thread owns the lock; in the unlocked state, no thread owns it.

Threads call a lock's **`acquire()`** method to lock it, and its **`release()`** method to unlock it.

Nota

Reentrant locks support the context management protocol, so it is recommended to use **`with`** instead of manually calling **`acquire()`** and **`release()`** to handle acquiring and releasing the lock for a block of code.

RLock's **`acquire()`**/**`release()`** call pairs may be nested, unlike Lock's **`acquire()`**/**`release()`**. Only the final **`release()`** (the **`release()`** of the outermost pair) resets the lock to an unlocked state and allows another thread blocked in **`acquire()`** to proceed.

`acquire()`/**`release()`** must be used in pairs: each acquire must have a release in the thread that has acquired the lock. Failing to call release as many times the lock has been acquired can lead to deadlock.

class threading.RLock

This class implements reentrant lock objects. A reentrant lock must be released by the thread that acquired it. Once a thread has acquired a reentrant lock, the same thread may acquire it again without blocking; the thread must release it once for each time it has acquired it.

Note that `RLock` is actually a factory function which returns an instance of the most efficient version of the concrete RLock class that is supported by the platform.

acquire(*blocking=True*, *timeout=-1*)

Adquire uma trava, bloqueante ou não.

Ver também

Using RLock as a context manager

Recommended over manual **`acquire()`** and **`release()`** calls whenever practical.

When invoked with the *blocking* argument set to `True` (the default):

- If no thread owns the lock, acquire the lock and return immediately.
- If another thread owns the lock, block until we are able to acquire lock, or *timeout*, if set to a positive value.
- If the same thread owns the lock, acquire the lock again, and return immediately. This is the difference between **`Lock`** and **`RLock`**; **`Lock`** handles this case the same as the previous, blocking until the lock can be acquired.

When invoked with the *blocking* argument set to `False`:

- If no thread owns the lock, acquire the lock and return immediately.
- If another thread owns the lock, return immediately.
- If the same thread owns the lock, acquire the lock again and return immediately.

In all cases, if the thread was able to acquire the lock, return `True`. If the thread was unable to acquire the lock (i.e. if not blocking or the timeout was reached) return `False`.

If called multiple times, failing to call **`release()`** as many times may lead to deadlock. Consider using **`RLock`** as a context manager

rather than calling acquire/release directly.

Alterado na versão 3.2: O parâmetro *timeout* é novo.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

release()

Release a lock, decrementing the recursion level. If after the decrement it is zero, reset the lock to unlocked (not owned by any thread), and if any other threads are blocked waiting for the lock to become unlocked, allow exactly one of them to proceed. If after the decrement the recursion level is still nonzero, the lock remains locked and owned by the calling thread.

Only call this method when the calling thread owns the lock. A `RuntimeError` is raised if this method is called when the lock is not acquired.

There is no return value.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

Condition objects

A condition variable is always associated with some kind of lock; this can be passed in or one will be created by default. Passing one in is useful when several condition variables must share the same lock. The lock is part of the condition object: you don't have to track it separately.

A condition variable obeys the context management protocol: using the `with` statement acquires the associated lock for the duration of the enclosed block. The `acquire()` and `release()` methods also call the corresponding methods of the associated lock.

Other methods must be called with the associated lock held. The **wait()** method releases the lock, and then blocks until another thread awakens it by calling **notify()** or **notify_all()**. Once awakened, **wait()** re-acquires the lock and returns. It is also possible to specify a timeout.

The **notify()** method wakes up one of the threads waiting for the condition variable, if any are waiting. The **notify_all()** method wakes up all threads waiting for the condition variable.

Note: the **notify()** and **notify_all()** methods don't release the lock; this means that the thread or threads awakened will not return from their **wait()** call immediately, but only when the thread that called **notify()** or **notify_all()** finally relinquishes ownership of the lock.

The typical programming style using condition variables uses the lock to synchronize access to some shared state; threads that are interested in a particular change of state call **wait()** repeatedly until they see the desired state, while threads that modify the state call **notify()** or **notify_all()** when they change the state in such a way that it could possibly be a desired state for one of the waiters. For example, the following code is a generic producer-consumer situation with unlimited buffer capacity:

```
# Consume one item
with cv:
    while not an_item_is_available():
        cv.wait()
    get_an_available_item()

# Produce one item
with cv:
    make_an_item_available()
    cv.notify()
```

The `while` loop checking for the application's condition is necessary because **wait()** can return after an arbitrary long time, and the condition which prompted the **notify()** call may no longer hold true. This is inherent to multi-threaded programming. The **wait_for()** method can be used to automate the condition checking, and eases the computation of timeouts:

```
# Consume an item
with cv:
    cv.wait_for(an_item_is_available)
    get_an_available_item()
```

To choose between **notify()** and **notify_all()**, consider whether one state change can be interesting for only one or several waiting threads. E.g. in a typical producer-consumer situation, adding one item to the buffer only needs to wake up one consumer thread.

class threading.Condition(*lock=None*)

This class implements condition variable objects. A condition variable allows one or more threads to wait until they are notified by another thread.

If the *lock* argument is given and not `None`, it must be a **Lock** or **RLock** object, and it is used as the underlying lock. Otherwise, a new **RLock** object is created and used as the underlying lock.

Alterado na versão 3.3: changed from a factory function to a class.

acquire(**args*)

Acquire the underlying lock. This method calls the corresponding method on the underlying lock; the return value is whatever that method returns.

release()

Release the underlying lock. This method calls the corresponding method on the underlying lock; there is no return value.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

wait(*timeout=None*)

Wait until notified or until a timeout occurs. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised.

This method releases the underlying lock, and then blocks until it is awakened by a `notify()` or `notify_all()` call for the same condition variable in another thread, or until the optional timeout occurs. Once awakened or timed out, it re-acquires the lock and returns.

When the *timeout* argument is present and not `None`, it should be a real number specifying a timeout for the operation in seconds (or fractions thereof).

When the underlying lock is an `RLock`, it is not released using its `release()` method, since this may not actually unlock the lock when it was acquired multiple times recursively. Instead, an internal interface of the `RLock` class is used, which really unlocks it even when it has been recursively acquired several times. Another internal interface is then used to restore the recursion level when the lock is reacquired.

O valor de retorno é `True` exceto se um dado *timout* expire, no caso em que é `False`.

Alterado na versão 3.2: Previously, the method always returned `None`.

wait_for(*predicate*, *timeout=None*)

Wait until a condition evaluates to true. *predicate* should be a callable which result will be interpreted as a boolean value. A *timeout* may be provided giving the maximum time to wait.

This utility method may call `wait()` repeatedly until the predicate is satisfied, or until a timeout occurs. The return value is the last return value of the predicate and will evaluate to `False` if the method timed out.

Ignoring the timeout feature, calling this method is roughly equivalent to writing:

```
while not predicate():
    cv.wait()
```

Therefore, the same rules apply as with `wait()`: The lock must be held when called and is re-acquired on return. The predicate is evaluated with the lock held.

Adicionado na versão 3.2.

notify(*n=1*)

By default, wake up one thread waiting on this condition, if any. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised.

This method wakes up at most *n* of the threads waiting for the condition variable; it is a no-op if no threads are waiting.

The current implementation wakes up exactly *n* threads, if at least *n* threads are waiting. However, it's not safe to rely on this behavior. A future, optimized implementation may occasionally wake up more than *n* threads.

Note: an awakened thread does not actually return from its `wait()` call until it can reacquire the lock. Since **`notify()`** does not release the lock, its caller should.

notify_all()

Wake up all threads waiting on this condition. This method acts like `notify()`, but wakes up all waiting threads instead of one. If the calling thread has not acquired the lock when this method is called, a `RuntimeError` is raised.

The method `notifyAll` is a deprecated alias for this method.

Semaphore objects

This is one of the oldest synchronization primitives in the history of computer science, invented by the early Dutch computer scientist Edsger W. Dijkstra (he used the names `P()` and `V()` instead of **`acquire()`** and **`release()`**).

A semaphore manages an internal counter which is decremented by each **`acquire()`** call and incremented by each **`release()`** call. The counter can never go below zero; when **`acquire()`** finds that it is zero, it blocks, waiting until some other thread calls **`release()`**.

Semaphores also support the context management protocol.

class threading.Semaphore(*value=1*)

This class implements semaphore objects. A semaphore manages an atomic counter representing the number of **`release()`** calls minus the number of **`acquire()`** calls, plus an initial value. The **`acquire()`** method blocks if necessary until it can return without making the counter negative. If not given, *value* defaults to 1.

The optional argument gives the initial *value* for the internal counter; it defaults to `1`. If the *value* given is less than 0, **`ValueError`** is raised.

Alterado na versão 3.3: changed from a factory function to a class.

acquire(*blocking=True*, *timeout=None*)

Acquire a semaphore.

When invoked without arguments:

- If the internal counter is larger than zero on entry, decrement it by one and return `True` immediately.
- If the internal counter is zero on entry, block until awoken by a call to **`release()`**. Once awoken (and the counter is greater than 0), decrement the counter by 1 and return `True`. Exactly one thread will be awoken by each call to **`release()`**. The order in which threads are awoken should not be relied on.

When invoked with *blocking* set to `False`, do not block. If a call without an argument would block, return `False` immediately; otherwise, do the same thing as when called without arguments, and return `True`.

When invoked with a *timeout* other than `None`, it will block for at most *timeout* seconds. If acquire does not complete successfully in that interval, return `False`. Return `True` otherwise.

Alterado na versão 3.2: O parâmetro *timeout* é novo.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

release(*n=1*)

Release a semaphore, incrementing the internal counter by *n*. When it was zero on entry and other threads are waiting for it to become larger than zero again, wake up *n* of those threads.

Alterado na versão 3.9: Added the *n* parameter to release multiple waiting threads at once.

class threading.BoundedSemaphore(*value=1*)

Class implementing bounded semaphore objects. A bounded semaphore checks to make sure its current value doesn't exceed its initial value. If it does, `ValueError` is raised. In most situations semaphores are used to guard resources with limited capacity. If the semaphore is released too many times it's a sign of a bug. If not given, *value* defaults to 1.

Alterado na versão 3.3: changed from a factory function to a class.

`Semaphore` example

Semaphores are often used to guard resources with limited capacity, for example, a database server. In any situation where the size of the resource is fixed, you should use a bounded semaphore. Before spawning any worker threads, your main thread would initialize the semaphore:

```
maxconnections = 5
# ...
pool_sema = BoundedSemaphore(value=maxconnections)
```

Once spawned, worker threads call the semaphore's acquire and release methods when they need to connect to the server:

```
with pool_sema:
    conn = connectdb()
    try:
        # ... use connection ...
    finally:
        conn.close()
```

The use of a bounded semaphore reduces the chance that a programming error which causes the semaphore to be released more than it's acquired will go undetected.

Event objects

This is one of the simplest mechanisms for communication between threads: one thread signals an event and other threads wait for it.

An event object manages an internal flag that can be set to true with the `set()` method and reset to false with the `clear()` method. The `wait()` method blocks until the flag is true.

class threading.Event

Class implementing event objects. An event manages a flag that can be set to true with the `set()` method and reset to false with the `clear()` method. The `wait()` method blocks until the flag is true. The flag is initially false.

Alterado na versão 3.3: changed from a factory function to a class.

is_set()

Return `True` if and only if the internal flag is true.

The method `isSet` is a deprecated alias for this method.

set()

Set the internal flag to true. All threads waiting for it to become true are awakened. Threads that call `wait()` once the flag is true will not block at all.

clear()

Reset the internal flag to false. Subsequently, threads calling `wait()` will block until `set()` is called to set the internal flag to true again.

wait(*timeout=None*)

Block as long as the internal flag is false and the timeout, if given, has not expired. The return value represents the reason that this blocking method returned; `True` if returning because the internal flag is set to true, or `False` if a timeout is given and the internal flag did not become true within the given wait time.

When the timeout argument is present and not `None`, it should be a real number specifying a timeout for the operation in seconds, or fractions thereof.

Alterado na versão 3.1: Previously, the method always returned `None`.

Timer objects

This class represents an action that should be run only after a certain amount of time has passed — a timer. `Timer` is a subclass of `Thread` and as such also functions as an example of creating custom threads.

Timers are started, as with threads, by calling their `Timer.start` method. The timer can be stopped (before its action has begun) by calling the `cancel()` method. The interval the timer will wait before executing its action may not be exactly the same as the interval specified by the user.

Por exemplo:

```
def hello():
    print("hello, world")

t = Timer(30.0, hello)
t.start()  # after 30 seconds, "hello, world" will be printed
```

class threading.Timer(*interval*, *function*, *args=None*, *kwargs=None*)

Create a timer that will run *function* with arguments *args* and keyword arguments *kwargs*, after *interval* seconds have passed. If *args* is `None` (the default) then an empty list will be used. If *kwargs* is `None` (the default) then an empty dict will be used.

Alterado na versão 3.3: changed from a factory function to a class.

cancel()

Stop the timer, and cancel the execution of the timer's action. This will only work if the timer is still in its waiting stage.

Barrier objects

Adicionado na versão 3.2.

This class provides a simple synchronization primitive for use by a fixed number of threads that need to wait for each other. Each of the threads tries to pass the barrier by calling the **`wait()`** method and will block until all of the threads have made their **`wait()`** calls. At this point, the threads are released simultaneously.

The barrier can be reused any number of times for the same number of threads.

As an example, here is a simple way to synchronize a client and server thread:

```
b = Barrier(2, timeout=5)

def server():
    start_server()
    b.wait()
```

```
    while True:
        connection = accept_connection()
        process_server_connection(connection)

def client():
    b.wait()
    while True:
        connection = make_connection()
        process_client_connection(connection)
```

class threading.Barrier(*parties*, *action=None*, *timeout=None*)

Create a barrier object for *parties* number of threads. An *action*, when provided, is a callable to be called by one of the threads when they are released. *timeout* is the default timeout value if none is specified for the `wait()` method.

wait(*timeout=None*)

Pass the barrier. When all the threads party to the barrier have called this function, they are all released simultaneously. If a *timeout* is provided, it is used in preference to any that was supplied to the class constructor.

The return value is an integer in the range 0 to *parties* – 1, different for each thread. This can be used to select a thread to do some special housekeeping, e.g.:

```
i = barrier.wait()
if i == 0:
    # Only one thread needs to print this
    print("passed the barrier")
```

If an *action* was provided to the constructor, one of the threads will have called it prior to being released. Should this call raise an error, the barrier is put into the broken state.

If the call times out, the barrier is put into the broken state.

This method may raise a `BrokenBarrierError` exception if the barrier is broken or reset while a thread is waiting.

reset()

Return the barrier to the default, empty state. Any threads waiting on it will receive the `BrokenBarrierError` exception.

Note that using this function may require some external synchronization if there are other threads whose state is unknown. If a barrier is broken it may be better to just leave it and create a new one.

abort()

Put the barrier into a broken state. This causes any active or future calls to `wait()` to fail with the `BrokenBarrierError`. Use this for example if one of the threads needs to abort, to avoid deadlocking the application.

It may be preferable to simply create the barrier with a sensible *timeout* value to automatically guard against one of the threads going awry.

parties

The number of threads required to pass the barrier.

n_waiting

The number of threads currently waiting in the barrier.

broken

A boolean that is `True` if the barrier is in the broken state.

exception threading.BrokenBarrierError

This exception, a subclass of `RuntimeError`, is raised when the `Barrier` object is reset or broken.

Using locks, conditions, and semaphores in the `with` statement

All of the objects provided by this module that have `acquire` and `release` methods can be used as context managers for a **`with`** statement. The `acquire` method will be called when the block is entered, and `release` will be called when the block is exited. Hence, the following snippet:

```
with some_lock:
    # do something...
```

é equivalente a:

```
some_lock.acquire()
try:
    # do something...
finally:
    some_lock.release()
```

Currently, **`Lock`**, **`RLock`**, **`Condition`**, **`Semaphore`**, and **`BoundedSemaphore`** objects may be used as **`with`** statement context managers.

Iterator synchronization

By default, Python iterators do not support concurrent access. Most iterators make no guarantees when accessed simultaneously from multiple threads. Generator iterators, for example, raise **`ValueError`** if one of their iterator methods is called while the generator is already executing. The tools in this section allow reliable concurrency support to be added to ordinary iterators and iterator-producing callables.

The **`serialize_iterator`** wrapper lets multiple threads share a single iterator and take turns consuming from it. While one thread is running `__next__()`, the others block until the iterator becomes available. Each value produced by the underlying iterator is delivered to exactly one caller.

The **`concurrent_tee()`** function lets multiple threads each receive the full stream of values from one underlying iterator. It creates independent iterators that all draw from the same source. Values are buffered until consumed by all of the derived iterators.

class threading.serialize_iterator(*iterable*)

Return an iterator wrapper that serializes concurrent calls to `__next__()` using a lock.

If the wrapped iterator also defines `send()`, `throw()`, or `close()`, those calls are serialized as well.

This makes it possible to share a single iterator, including a generator iterator, between multiple threads. A lock ensures that calls are handled one at a time. No values are duplicated or skipped by the wrapper itself. Each item from the underlying iterator is given to exactly one caller.

This wrapper does not copy or buffer values. Threads that call `next()` while another thread is already advancing the iterator will block until the active call completes.

Exemplo:

```
import threading

def squares(n):
    for x in range(n):
        yield x * x

def consume(name, iterable):
    for item in iterable:
        print(name, item)

source = threading.serialize_iterator(squares(5))

t1 = threading.Thread(target=consume, args=("left", source))
t2 = threading.Thread(target=consume, args=("right", source))
t1.start()
t2.start()
t1.join()
t2.join()
```

In this example, each number is printed exactly once, but the work is shared between the two threads.

Adicionado na versão 3.15.

threading.synchronized_iterator(*func*)

Wrap an iterator-producing callable so that each iterator it returns is automatically passed through **`serialize_iterator`**.

This is especially useful as a decorator for generator functions, allowing their generator-iterators to be consumed from multiple threads.

Exemplo:

```
import threading

@threading.synchronized_iterator
def squares(n):
    for x in range(n):
        yield x * x

def consume(name, iterable):
    for item in iterable:
        print(name, item)

source = squares(5)

t1 = threading.Thread(target=consume, args=("left", source))
t2 = threading.Thread(target=consume, args=("right", source))
t1.start()
t2.start()
t1.join()
t2.join()
```

The returned wrapper preserves the metadata of *func*, such as its name and wrapped function reference.

Adicionado na versão 3.15.

threading.concurrent_tee(*iterable*, *n=2*)

Return *n* independent iterators from a single input *iterable*, with guaranteed behavior when the derived iterators are consumed

concurrently.

This function is similar to `itertools.tee()`, but is intended for cases where the source iterator may feed consumers running in different threads. Each returned iterator yields every value from the underlying iterable, in the same order.

Internally, values are buffered until every derived iterator has consumed them.

The returned iterators share the same underlying synchronization lock. Each individual derived iterator is intended to be consumed by one thread at a time. If a single derived iterator must itself be shared by multiple threads, wrap it with `serialize_iterator`.

If *n* is `0`, return an empty tuple. If *n* is negative, raise `ValueError`.

Exemplo:

```
import threading

def squares(n):
    for x in range(n):
        yield x * x

def consume(name, iterable):
    for item in iterable:
        print(name, item)

source = squares(5)
left, right = threading.concurrent_tee(source)

t1 = threading.Thread(target=consume, args=("left", left))
t2 = threading.Thread(target=consume, args=("right", right))
t1.start()
t2.start()
t1.join()
t2.join()
```

In this example, both consumer threads see the full sequence of squares from a single generator expression.

Adicionado na versão 3.15.

`multiprocessing` — Paralelismo baseado em processo

Código-fonte: Lib/multiprocessing/
[https://github.com/python/cpython/tree/3.15/Lib/multiprocessing/]

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Introdução

`multiprocessing` is a package that supports spawning processes using an API similar to the **`threading`** module. The **`multiprocessing`** package offers both local and remote concurrency, effectively side-stepping the Global Interpreter Lock by using subprocesses instead of threads. Due to this, the **`multiprocessing`** module allows the programmer to fully leverage multiple processors on a given machine. It runs on both POSIX and Windows.

The **`multiprocessing`** module also introduces the **`Pool`** object which offers a convenient means of parallelizing the execution of a function across multiple input values, distributing the input data across processes (data parallelism). The following example demonstrates the common practice of defining such functions in a module so that child processes can successfully import that module. This basic example of data parallelism using **`Pool`**,

```
from multiprocessing import Pool

def f(x):
    return x*x
```

```
if __name__ == '__main__':
    with Pool(5) as p:
        print(p.map(f, [1, 2, 3]))
```

vai exibir na saída padrão

```
[1, 4, 9]
```

The **multiprocessing** module also introduces APIs which do not have analogs in the **threading** module, like the ability to **terminate**, **interrupt** or **kill** a running process.

Ver também

concurrent.futures.ProcessPoolExecutor oferece uma interface de nível mais alto para enviar tarefas para um processo em segundo plano sem bloquear a execução do processo de chamada. Comparado ao uso direto da interface **Pool**, a API **concurrent.futures** permite mais prontamente que o envio de trabalho para o pool de processos subjacente seja separado da espera pelos resultados.

A classe Process

In **multiprocessing**, processes are spawned by creating a **Process** object and then calling its **start()** method. **Process** follows the API of **threading.Thread**. A trivial example of a multiprocess program is

```
from multiprocessing import Process

def f(name):
    print('hello', name)

if __name__ == '__main__':
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

Para mostrar os IDs de processo individuais envolvidos, aqui está um exemplo expandido:

```
from multiprocessing import Process
import os

def info(title):
    print(title)
    print('module name:', __name__)
    print('parent process:', os.getppid())
    print('process id:', os.getpid())

def f(name):
    info('function f')
    print('hello', name)

if __name__ == '__main__':
    info('main line')
    p = Process(target=f, args=('bob',))
    p.start()
    p.join()
```

Para uma explicação do porquê a parte `if __name__ == '__main__'` é necessária, veja Programming guidelines.

The arguments to **`Process`** usually need to be picklable so they can be passed to the child process. If you tried typing the above example directly into a REPL it could lead to an **`AttributeError`** in the child process trying to locate the *f* function in the `__main__` module.

Contextos e métodos de inicialização

Depending on the platform, **`multiprocessing`** supports three ways to start a process. These *start methods* are

spawn

O processo pai inicia um novo processo do interpretador Python. O processo filho herdará apenas os recursos necessários para executar o método **`run()`** do objeto do processo. Em particular, descritores de arquivo e identificadores desnecessários do processo pai não

serão herdados. Iniciar um processo usando esse método é bem lento comparado a usar *fork* ou *forkserver*.

Disponível em plataformas POSIX e Windows. O padrão no Windows e macOS.

fork

O processo pai usa `os.fork()` para criar um fork do interpretador Python. O processo filho, quando começa, é efetivamente idêntico ao processo pai. Todos os recursos do pai são herdados pelo processo filho. Observe que criar um fork com segurança de um processo multithread é problemático.

Disponível em sistemas POSIX.

Alterado na versão 3.14: Esse não é mais o método de inicialização padrão em nenhuma plataforma. O código que requer *fork* deve especificar explicitamente isso por meio de `get_context()` ou `set_start_method()`.

Alterado na versão 3.12: Se o Python for capaz de detectar que seu processo tem várias threads, a função `os.fork()` que esse método de inicialização chama internamente levantará `DeprecationWarning`. Use um método de inicialização diferente. Veja a documentação de **`os.fork()`** para mais explicações.

forkserver

Quando o programa é inicializado e seleciona o método *forkserver*, um processo de servidor é gerado. A partir disso, sempre que um novo processo é necessário, o processo pai conecta-se ao servidor e solicita que um novo processo seja feito um fork. O processo fork do servidor é de thread única, a menos que bibliotecas do sistema ou importações pré-carregadas gerem threads como um efeito colateral; neste sentido, geralmente é seguro usar `os.fork()`. Nenhum recurso desnecessário é herdado.

Disponível em plataformas POSIX que suportem a passagem de arquivo descritor para encadeamentos Unix, como Linux. O padrão

nessas plataformas.

Alterado na versão 3.14: Isso se tornou o método de inicialização padrão nas plataformas POSIX.

Alterado na versão 3.4: spawn adicionado em todas as plataformas POSIX, e *forkserver* adicionado para algumas plataformas POSIX. Processos filhos não herdam mais todos os identificadores herdáveis dos pais no Windows.

Alterado na versão 3.8: No macOS, o método de inicialização *spawn* agora é o padrão. O método *fork* deve ser considerado inseguro, pois pode levar a travamentos do subprocesso, uma vez que as bibliotecas do sistema macOS podem iniciar threads. Consulte bpo-33725 [https://bugs.python.org/issue?@action=redirect&bpo=33725].

Alterado na versão 3.14: Nas plataformas POSIX, o método de inicialização padrão foi alterado de *fork* para *forkserver* para manter o desempenho, mas evitar incompatibilidades comuns de processos multithreads. Consulte gh-84559 [https://github.com/python/cpython/issues/84559].

No POSIX, usar os métodos de inicialização *spawn* ou *forkserver* também iniciará um processo *rastreador de recursos* que rastreia os recursos de sistema nomeados não vinculados (como semáforos nomeados ou objetos **`SharedMemory`**) criados por processos do programa. Quando todos os processos tiverem saído, o rastreador de recursos desvincula qualquer objeto rastreado restante. Normalmente, não deve haver nenhum, mas se um processo foi morto por um sinal, pode haver alguns recursos “vazados”. (Nem os semáforos vazados nem os segmentos de memória compartilhada serão desvinculados automaticamente até a próxima reinicialização do sistema. Isso é problemático para ambos os objetos porque o sistema permite apenas um número limitado de semáforos nomeados, e os segmentos de memória compartilhada ocupam algum espaço na memória principal.)

Para selecionar um método de inicialização, você usa **`set_start_method()`** na cláusula `if __name__ == '__main__'` do módulo principal. Por exemplo:

```
import multiprocessing as mp

def foo(q):
    q.put('hello')

if __name__ == '__main__':
    mp.set_start_method('spawn')
    q = mp.Queue()
    p = mp.Process(target=foo, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

set_start_method() não deve ser usada mais de uma vez no programa.

Alternativamente, você pode usar **get_context()** para obter um objeto de contexto. Objetos de contexto têm a mesma API que o módulo multiprocessing e permitem que se usem vários métodos de inicialização no mesmo programa.

```
import multiprocessing as mp

def foo(q):
    q.put('hello')

if __name__ == '__main__':
    ctx = mp.get_context('spawn')
    q = ctx.Queue()
    p = ctx.Process(target=foo, args=(q,))
    p.start()
    print(q.get())
    p.join()
```

Note que objetos relacionados a um contexto podem não ser compatíveis com processos para um contexto diferente. Em particular, travas criadas usando o contexto *fork* não podem ser passados para processos iniciados usando os métodos de inicialização *spawn* ou *forkserver*.

Libraries using **multiprocessing** or **ProcessPoolExecutor** should be designed to allow their users to provide their own multiprocessing context. Using a specific context of your own within a library can lead to

incompatibilities with the rest of the library user's application. Always document if your library requires a specific start method.

Aviso

Os métodos de inicialização `'spawn'` e `'forkserver'` geralmente não podem ser usadas com executáveis "congelados" (por exemplo, binários produzidos por pacotes como **PyInstaller** e **cx_Freeze**) em sistemas POSIX. O método de inicialização `'fork'` pode funcionar se o código não usar threads.

Trocando objetos entre processos

`multiprocessing` supports two types of communication channel between processes:

Filas

A classe **`Queue`** é quase um clone de **`queue.Queue`**. Por exemplo:

```
from multiprocessing import Process, Queue

def f(q):
    q.put([42, None, 'hello'])

if __name__ == '__main__':
    q = Queue()
    p = Process(target=f, args=(q,))
    p.start()
    print(q.get())    # prints "[42, None, 'hello']"
    p.join()
```

Queues are thread and process safe. Any object put into a **`multiprocessing`** queue will be serialized.

Encadeamentos

A função **Pipe()** retorna um par de objetos de conexão conectados por um encadeamento que por padrão é duplex (bidirecional). Por exemplo:

```
from multiprocessing import Process, Pipe

def f(conn):
    conn.send([42, None, 'hello'])
    conn.close()

if __name__ == '__main__':
    parent_conn, child_conn = Pipe()
    p = Process(target=f, args=(child_conn,))
    p.start()
    print(parent_conn.recv())   # exibe "[42, None, 'hello']
    p.join()
```

Os dois objetos de conexão retornados por **Pipe()** representam as duas extremidades do encadeamento. Cada objeto de conexão tem os métodos **send()** e **recv()** (entre outros). Observe que os dados em um encadeamento podem ser corrompidos se dois processos (ou threads) tentarem ler ou gravar na *mesma* extremidade do encadeamento ao mesmo tempo. Claro que não há risco de corrupção de processos usando extremidades diferentes do encadeamento ao mesmo tempo.

O método **send()** serializa o objeto e **recv()** recria o objeto.

Sincronização entre processos

multiprocessing contains equivalents of all the synchronization primitives from **threading**. For instance one can use a lock to ensure that only one process prints to standard output at a time:

```
from multiprocessing import Process, Lock

def f(l, i):
    l.acquire()
    try:
        print('hello world', i)
```

```
    finally:
        l.release()

if __name__ == '__main__':
    lock = Lock()

    for num in range(10):
        Process(target=f, args=(lock, num)).start()
```

Sem utilizar a saída da trava dos diferentes processos, é possível que tudo fique confuso.

Compartilhando estado entre processos

Conforme mencionado acima, ao fazer programação concorrente, geralmente é melhor evitar o uso de estado compartilhado, tanto quanto possível. Isso é particularmente verdadeiro ao utilizar múltiplos processos.

However, if you really do need to use some shared data then **`multiprocessing`** provides a couple of ways of doing so.

Memória compartilhada

Os dados podem ser armazenados em um mapa de memória compartilhado utilizando **`Value`** ou **`Array`**. Por exemplo, o código a seguir

```
from multiprocessing import Process, Value, Array

def f(n, a):
    n.value = 3.1415927
    for i in range(len(a)):
        a[i] = -a[i]

if __name__ == '__main__':
    num = Value('d', 0.0)
    arr = Array('i', range(10))

    p = Process(target=f, args=(num, arr))
    p.start()
```

```
    p.join()

    print(num.value)
    print(arr[:])
```

vai exibir

```
3.1415927
[0, -1, -2, -3, -4, -5, -6, -7, -8, -9]
```

Os argumentos `'d'` e `'i'` usados ao criar `num` e `arr` são typecodes do tipo usado pelo módulo **`array`**: `'d'` indica um ponto flutuante de precisão dupla e `'i'` indica um inteiro com sinal. Esses objetos compartilhados serão seguros para thread e processo.

Para mais flexibilidade no uso de memória compartilhada, pode-se utilizar o módulo **`multiprocessing.sharedctypes`**, que suporta a criação de objetos ctypes arbitrários alocados da memória compartilhada.

Processo de servidor

Um objeto gerenciador retornado por **`Manager()`** controla um processo de servidor que contém objetos Python e permite que outros processos os manipulem usando proxies.

A manager returned by **`Manager()`** will support types **`list`**, **`dict`**, **`set`**, **`Namespace`**, **`Lock`**, **`RLock`**, **`Semaphore`**, **`BoundedSemaphore`**, **`Condition`**, **`Event`**, **`Barrier`**, **`Queue`**, **`Value`** and **`Array`**. For example,

```
from multiprocessing import Process, Manager

def f(d, l, s):
    d[1] = '1'
    d['2'] = 2
    d[0.25] = None
    l.reverse()
    s.add('a')
    s.add('b')
```

```
if __name__ == '__main__':
    with Manager() as manager:
        d = manager.dict()
        l = manager.list(range(10))
        s = manager.set()

        p = Process(target=f, args=(d, l, s))
        p.start()
        p.join()

        print(d)
        print(l)
        print(s)
```

vai exibir

```
{0.25: None, 1: '1', '2': 2}
[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
{'a', 'b'}
```

Os gerenciadores de processos de servidor são mais flexíveis do que usar objetos de memória compartilhada porque podem ser feitos para oferecer suporte a tipos de objetos arbitrários. Além disso, um único gerenciador pode ser compartilhado por processos em diferentes computadores em uma rede. Eles são, no entanto, mais lentos do que usar memória compartilhada.

Usando um pool de workers

A classe `Pool` representa um pool de processos de worker. Ela tem métodos que permitem que tarefas sejam descarregadas para os processos de worker de algumas maneiras diferentes.

Por exemplo:

```
from multiprocessing import Pool, TimeoutError
import time
import os

def f(x):
```

```
    return x*x

if __name__ == '__main__':
    # inicia 4 processos de trabalhador
    with Pool(processes=4) as pool:

        # exibe "[0, 1, 4,..., 81]"
        print(pool.map(f, range(10)))

        # exibe mesmo números em ordem arbitrária
        for i in pool.imap_unordered(f, range(10)):
            print(i)

        # calcula "f(20)" assincronamente
        res = pool.apply_async(f, (20,))      # executa em *aper
        print(res.get(timeout=1))             # exibe "400"

        # calcula "os.getpid()" assincronamente
        res = pool.apply_async(os.getpid, ()) # executa em *aper
        print(res.get(timeout=1))             # exibe o PID daqu

        # iniciando vários cálculos de forma assíncrona *pod* us
        multiple_results = [pool.apply_async(os.getpid, ()) for
        print([res.get(timeout=1) for res in multiple_results])

        # faz um único worker dormir por 10 segundos
        res = pool.apply_async(time.sleep, (10,))
        try:
            print(res.get(timeout=1))
        except TimeoutError:
            print("We lacked patience and got a multiprocessing.

        print("For the moment, the pool remains available for mo

    # saindo o bloco 'with' parou o pool
    print("Now the pool is closed and no longer available")
```

Observe que os métodos de um pool só devem ser usados pelo processo que o criou.

Nota

A funcionalidade dentro deste pacote requer que o módulo `__main__` seja importável pelos filhos. Isso é abordado em Programming guidelines, mas vale a pena apontar aqui. Isso significa que alguns exemplos, como os exemplos **`multiprocessing.pool.Pool`** não funcionarão no interpretador interativo. Por exemplo:

```
>>> from multiprocessing import Pool
>>> p = Pool(5)
>>> def f(x):
...     return x*x
...
>>> with p:
...     p.map(f, [1,2,3])
Process PoolWorker-1:
Process PoolWorker-2:
Process PoolWorker-3:
Traceback (most recent call last):
Traceback (most recent call last):
Traceback (most recent call last):
AttributeError: Can't get attribute 'f' on <module '__main__'
AttributeError: Can't get attribute 'f' on <module '__main__'
AttributeError: Can't get attribute 'f' on <module '__main__'
```

(Se você tentar isso, na verdade, serão gerados três tracebacks completos intercalados de forma semi-aleatória, e então você pode ter que interromper o processo pai de alguma forma.)

Referência

The **`multiprocessing`** package mostly replicates the API of the **`threading`** module.

Global start method

Python supports several ways to create and initialize a process. The global start method sets the default mechanism for creating a process.

Several multiprocessing functions and methods that may also instantiate certain objects will implicitly set the global start method to the system's default, if it hasn't been set already. The global start method can only be set once. If you need to change the start method from the system default, you must proactively set the global start method before calling functions or methods, or creating these objects.

`Process` e exceções

class multiprocessing.Process(*group=None*, *target=None*, *name=None*, *args=()*, *kwargs={}*, *, *daemon=None*)

Objetos processo representam atividades que são executadas em um processo separado. A classe `Process` possui equivalentes de todos os métodos de `threading.Thread`.

O construtor deve sempre ser chamado com argumentos nomeados. *group* deve sempre ser `None`; ele existe somente para compatibilidade com `threading.Thread`. *target* é o objeto chamável a ser invocado pelo método `run()`. O padrão é `None`, o que significa que nada é chamado. *name* é o nome do processo (veja `name` para mais detalhes). *args* é a tupla de argumento para a invocação de destino. *kwargs* é um dicionário de argumentos nomeados para a invocação de destino. Se fornecido, o argumento somente-nomeados *daemon* define o sinalizador do processo `daemon` como `True` ou `False`. Se `None` (o padrão), este sinalizador será herdado do processo de criação.

Por padrão, nenhum argumento é passado para *target*. O argumento *args*, que tem como padrão `()`, pode ser usado para especificar uma lista ou tupla de argumentos a serem passados para *target*.

If a subclass overrides the constructor, it must make sure it invokes the base class constructor (`super().__init__()`) before doing anything else to the process.

Nota

In general, all arguments to **`Process`** must be picklable. This is frequently observed when trying to create a **`Process`** or use a **`concurrent.futures.ProcessPoolExecutor`** from a REPL with a locally defined *target* function.

Passing a callable object defined in the current REPL session causes the child process to die via an uncaught **`AttributeError`** exception when starting as *target* must have been defined within an importable module in order to be loaded during unpickling.

Example of this uncatchable error from the child:

```
>>> import multiprocessing as mp
>>> def knigit():
...     print("Ni!")
...
>>> process = mp.Process(target=knigit)
>>> process.start()
>>> Traceback (most recent call last):
  File ".../multiprocessing/spawn.py", line ..., in spawn_m
  File ".../multiprocessing/spawn.py", line ..., in _main
AttributeError: module '__main__' has no attribute 'knigit'
>>> process
<SpawnProcess name='SpawnProcess-1' pid=379473 parent=37870
```

See The spawn and forkserver start methods. While this restriction is not true if using the `"fork"` start method, as of Python `3.14` that is no longer the default on any platform. See Contextos e métodos de inicialização. See also gh-132898 [https://github.com/python/cpython/issues/132898].

Alterado na versão 3.3: Adicionado o parâmetro *daemon*.

run()

Método que representa a atividade do processo.

Você pode substituir esse método em uma subclasse. O método padrão **`run()`** invoca o objeto chamável passado ao construtor do

objeto como o argumento alvo, se houver, com argumentos nomeados e sequenciais retirados dos argumentos *args* e *kwargs*, respectivamente.

Usar uma lista ou tupla como argumento *args* passado para **`Process`** obtém o mesmo efeito.

Exemplo:

```
>>> from multiprocessing import Process
>>> p = Process(target=print, args=[1])
>>> p.run()
1
>>> p = Process(target=print, args=(1,))
>>> p.run()
1
```

start()

Inicia a atividade do processo.

Isso deve ser chamado no máximo uma vez por objeto processo. Ele organiza para que o método **`run()`** do objeto seja invocado em um processo separado.

join([*timeout*])

Se o argumento opcional *timeout* for `None` (o padrão), o método bloqueia até que o processo cujo método **`join()`** é chamado termine. Se *timeout* for um número positivo, ele bloqueia no máximo *timeout* segundos. Observe que o método retorna `None` se seu processo terminar ou se o método tiver tempo limite. Verifique o **`exitcode`** do processo para determinar se ele terminou.

Um processo pode ser usar “join” muitas vezes.

Um processo não pode se unir porque isso causaria um impasse. É um erro tentar se unir a um processo antes que ele tenha sido iniciado.

name

O nome do processo. O nome é uma string usada apenas para fins de identificação. Não tem semântica. Vários processos podem receber o mesmo nome.

O nome inicial é definido pelo construtor. Se nenhum nome explícito for fornecido ao construtor, um nome do formato ‘Processo-N_1:N_2:…:N_k’ é construído, onde cada N_k é o N-ésimo filho de seu pai.

is_alive()

Retorna se o processo está ativo.

Em termos gerais, um objeto processo está ativo desde o momento em que o método `start()` retorna até o término do processo filho.

daemon

O sinalizador daemon do processo, um valor Booleano. Isso deve ser definido antes de `start()` ser chamado.

O valor inicial é herdado do processo de criação.

Quando um processo sai, ele tenta encerrar todos os seus processos filhos daemônicos.

Note que um processo daemônico não tem permissão para criar processos filhos. Caso contrário, um processo daemônico deixaria seus filhos órfãos se ele fosse encerrado quando seu processo pai saísse. Além disso, esses **não** são daemons ou serviços Unix, eles são processos normais que serão encerrados (e em vez de usar “join”) se processos não daemônicos tiverem saído.

Além da API `threading.Thread`, os objetos `Process` também oferecem suporte aos seguintes atributos e métodos:

pid

Retorna o ID do processo. Antes do processo ser gerado, este será `None`.

exitcode

O código de saída da criança. Este será `None` se o processo ainda não tiver terminado.

Se o método **`run()`** da criança retornar normalmente, o código de saída será 0. Se ele terminar via **`sys.exit()`** com um argumento inteiro *N*, o código de saída será *N*.

Se a criança for encerrada devido a uma exceção não capturada em **`run()`**, o código de saída será 1. Se ela for encerrada pelo sinal *N*, o código de saída será o valor negativo -*N*.

authkey

A chave de autenticação do processo (uma string de bytes).

When **`multiprocessing`** is initialized the main process is assigned a random string using **`os.urandom()`**.

Quando um objeto **`Process`** é criado, ele herda a chave de autenticação do seu processo pai, embora isso possa ser alterado definindo **`authkey`** para outra sequência de bytes.

Veja Authentication keys.

sentinel

Um identificador numérico de um objeto do sistema que ficará "pronto" quando o processo terminar.

Você pode usar esse valor se quiser esperar por vários eventos ao mesmo tempo usando **`multiprocessing.connection.wait()`**. Caso contrário, chamar **`join()`** é mais simples.

No Windows, este é um identificador de sistema operacional utilizável com a família de chamadas de API `WaitForSingleObject`

e `WaitForMultipleObjects`. No POSIX, este é um descritor de arquivo utilizável com primitivos do módulo **`select`**.

Adicionado na versão 3.3.

interrupt()

Encerra o processo. Funciona em POSIX usando o sinal **`SIGINT`**. O comportamento no Windows é indefinido.

Por padrão, isso encerra o processo filho levantando **`KeyboardInterrupt`**. Esse comportamento pode ser alterado com a configuração do respectivo sinal manipulador no processo filho **`signal.signal()`** para **`SIGINT`**.

Observação: se o processo filho capturar e descartar **`KeyboardInterrupt`**, o processo não será encerrado.

Observação: o comportamento padrão também definirá **`exitcode`** para `1` como se um exceção não-capturada fosse levantada no filho processo. Para ter um **`exitcode`** diferente, você pode simplesmente capturar **`KeyboardInterrupt`** e chamar `exit(your_code)`.

Adicionado na versão 3.14.

terminate()

Encerra o processo. No POSIX isso é feito usando o sinal **`SIGTERM`**; no Windows é usado **`TerminateProcess()`**. Note que os manipuladores de saída e cláusulas finally, etc., não serão executados.

Observe que os processos descendentes do processo *não* serão encerrados — eles simplesmente ficarão órfãos.

Aviso

Se esse método for usado quando o processo associado estiver usando um encadeamento ou fila, então o encadeamento ou fila é passível de ser corrompido e pode se tornar inutilizável por outro

processo. Similarmente, se o processo adquiriu um trava ou semáforo etc., então encerrá-lo é passível de causar impasse em outros processos.

kill()

O mesmo que **terminate()**, mas usando o sinal `SIGKILL` no POSIX.

Adicionado na versão 3.7.

close()

Fecha o objeto **Process**, liberando todos os recursos associados a ele. **ValueError** é levantado se o processo subjacente ainda estiver em execução. Uma vez que **close()** retorne com sucesso, a maioria dos outros métodos e atributos do objeto **Process** levantará **ValueError**.

Adicionado na versão 3.7.

Observe que os métodos **start()**, **join()**, **is_alive()**, **terminate()** e **exitcode** devem ser chamados somente pelo processo que criou o objeto processo.

Exemplo de uso de alguns dos métodos de **Process**:

```
>>> import multiprocessing, time, signal
>>> mp_context = multiprocessing.get_context('spawn')
>>> p = mp_context.Process(target=time.sleep, args=(1000,))
>>> print(p, p.is_alive())
<...Process ... initial> False
>>> p.start()
>>> print(p, p.is_alive())
<...Process ... started> True
>>> p.terminate()
>>> time.sleep(0.1)
>>> print(p, p.is_alive())
<...Process ... stopped exitcode=-SIGTERM> False
>>> p.exitcode == -signal.SIGTERM
True
```

exception multiprocessing.ProcessError

The base class of all **`multiprocessing`** exceptions.

exception multiprocessing.BufferTooShort

Exceção levantada por **`Connection.recv_bytes_into()`** quando o objeto buffer fornecido é muito pequeno para a mensagem lida.

Se `e` for uma instância de **`BufferTooShort`**, então `e.args[0]` retornará a mensagem como uma string de bytes.

exception multiprocessing.AuthenticationError

Levantada quando há um erro de autenticação.

exception multiprocessing.TimeoutError

Levantada por métodos com um tempo limite quando o tempo limite expira.

Encadeamentos e filas

Ao usar vários processos, geralmente é usada a passagem de mensagens para comunicação entre processos e evita-se ter que usar quaisquer primitivas de sincronização, como travas.

Para passar mensagens, pode-se usar **`Pipe()`** (para uma conexão entre dois processos) ou uma fila (que permite múltiplos produtores e consumidores).

Os tipos **`Queue`**, **`SimpleQueue`** e **`JoinableQueue`** são filas FIFO multiprodutoras e multiconsumidoras modeladas na classe **`queue.Queue`** da biblioteca padrão. Elas diferem porque **`Queue`** não tem os métodos **`task_done()`** e **`join()`** introduzidos na classe **`queue.Queue`** do Python 2.5.

Se você usar **`JoinableQueue`**, então você **deve** chamar **`JoinableQueue.task_done()`** para cada tarefa removida da fila, caso contrário, o semáforo usado para contar o número de tarefas não concluídas pode eventualmente transbordar, levantando uma exceção.

One difference from other Python queue implementations, is that **multiprocessing** queues serializes all objects that are put into them using **pickle**. The object returned by the get method is a re-created object that does not share memory with the original object.

Observe que também é possível criar uma fila compartilhada usando um objeto gerenciador — veja Gerenciadores.

Nota

multiprocessing uses the usual **queue.Empty** and **queue.Full** exceptions to signal a timeout. They are not available in the **multiprocessing** namespace so you need to import them from **queue**.

Nota

Quando um objeto é colocado em uma fila, o objeto é serializado com pickle e uma thread em segundo plano depois descarrega os dados serializados com pickle para um encadeamento subjacente. Isso tem algumas consequências que são um pouco surpreendentes, mas não devem causar nenhuma dificuldade prática – se elas realmente o incomodam, então você pode usar uma fila criada com um gerenciador.

1. Depois de colocar um objeto em uma fila vazia, pode haver um atraso infinitesimal antes que o método **empty()** da fila retorne **False** e **get_nowait()** possa retornar sem levantar **queue.Empty**.
2. Se vários processos estiverem enfileirando objetos, é possível que os objetos sejam recebidos na outra extremidade fora de ordem. No entanto, objetos enfileirados pelo mesmo processo sempre estarão na ordem esperada em relação uns aos outros.

Aviso

Se um processo for morto usando **Process.terminate()** ou **os.kill()** enquanto estiver tentando usar uma **Queue**, os dados na fila provavelmente

serão corrompidos. Isso pode fazer com que qualquer outro processo obtenha uma exceção quando tentar usar a fila mais tarde.

Aviso

Conforme mencionado acima, se um processo filho tiver colocado itens em uma fila (e não tiver usado `JoinableQueue.cancel_join_thread`), esse processo não será encerrado até que todos os itens armazenados em buffer tenham sido liberados para o encadeamento.

Isso significa que se você tentar juntar esse processo, poderá obter um impasse, a menos que tenha certeza de que todos os itens que foram colocados na fila foram consumidos. Da mesma forma, se o processo filho não for daemônico, o processo pai pode travar na saída quando tentar juntar todos os seus filhos não daemônicos.

Note que uma fila criada usando um gerenciador não tem esse problema. Veja Programming guidelines.

Para um exemplo do uso de filas para comunicação entre processos, veja Exemplos.

multiprocessing.Pipe(*duplex=True*)

Retorna um par `(conn1, conn2)` de objetos `Connection` representando as extremidades de um encadeamento.

Se *duplex* for `True` (o padrão), então o encadeamento é bidirecional. Se *duplex* for `False`, então o encadeamento é unidirecional: `conn1` pode ser usado somente para receber mensagens e `conn2` pode ser usado somente para enviar mensagens.

O método **`send()`** serializa o objeto usando `pickle` e **`recv()`** recria o objeto.

class multiprocessing.Queue([*maxsize*])

Retorna uma fila compartilhada de processo implementada usando um encadeamento e algumas travas/semáforos. Quando um processo coloca um item na fila pela primeira vez, uma thread alimentadora é iniciada, a qual transfere objetos de um buffer para o encadeamento.

Instantiating this class may set the global start method. See Global start method for more details.

As exceções usuais `queue.Empty` e `queue.Full` do módulo `queue` da biblioteca padrão são levantadas para sinalizar tempos limite.

`Queue` implements all the methods of `queue.Queue` except for `task_done()`, `join()`, and `shutdown()`.

qsize()

Retorna o tamanho aproximado da fila. Devido à semântica de multithreading/multiprocessamento, esse número não é confiável.

Observe que isso pode levantar `NotImplementedError` em plataformas como macOS, onde `sem_getvalue()` não está implementado.

empty()

Retorna `True` se a fila estiver vazia, `False` caso contrário. Devido à semântica de multithreading/multiprocessamento, isso não é confiável.

Pode levantar uma exceção `OSError` em filas fechadas. (não garantido)

full()

Retorna `True` se a fila estiver cheia, `False` caso contrário. Devido à semântica de multithreading/multiprocessamento, isso não é confiável.

put(*obj*[, *block*[, *timeout*]])

Coloca *obj* na fila. Se o argumento opcional *block* for `True` (o padrão) e *timeout* for `None` (o padrão), bloqueia se necessário até que um slot livre esteja disponível. Se *timeout* for um número positivo, ele bloqueia no máximo *timeout* segundos e levanta a exceção **`queue.Full`** se nenhum slot livre estiver disponível dentro desse tempo. Caso contrário (*block* é `False`), coloca um item na fila se um slot livre estiver imediatamente disponível, senão levanta a exceção **`queue.Full`** (*timeout* é ignorado nesse caso).

Alterado na versão 3.8: Se a fila for fechada, **`ValueError`** será levantada em vez de **`AssertionError`**.

put_nowait(*obj*)

Equivalente a `put(obj, False)`.

get([*block*[, *timeout*]])

Remove e retorna um item da fila. Se os argumentos opcionais *block* forem `True` (o padrão) e *timeout* forem `None` (o padrão), bloqueia se necessário até que um item esteja disponível. Se *timeout* for um número positivo, ele bloqueia no máximo *timeout* segundos e levantada a exceção **`queue.Empty`** se nenhum item estiver disponível dentro desse tempo. Caso contrário (block for `False`), retorna um item se um estiver imediatamente disponível, senão levantada a exceção **`queue.Empty`** (*timeout* é ignorado nesse caso).

Alterado na versão 3.8: Se a fila for fechada, **`ValueError`** será levantada em vez de **`OSError`**.

get_nowait()

Equivalente a `get(False)`.

`multiprocessing.Queue` tem alguns métodos adicionais não encontrados em **`queue.Queue`**. Esses métodos geralmente são desnecessários para a maioria dos códigos:

close()

Fecha a fila: libera recursos internos.

A queue must not be used anymore after it is closed. For example, `get()`, `put()` and `empty()` methods must no longer be called.

The background thread will quit once it has flushed all buffered data to the pipe. This is called automatically when the queue is garbage collected.

join_thread()

Junta a thread de segundo plano. Isso só pode ser usado depois que `close()` for chamado. Isso bloqueia até que a thread de segundo plano saia, garantindo que todos os dados no buffer tenham sido liberados para o encadeamento.

Por padrão, se um processo não for o criador da fila, ao sair, ele tentará se juntar ao thread de segundo plano da fila. O processo pode chamar `cancel_join_thread()` para fazer `join_thread()` não fazer nada.

cancel_join_thread()

Impede que `join_thread()` bloqueie. Em particular, isso impede que o thread de segundo plano seja unido automaticamente quando o processo sai – veja **`join_thread()`**.

Um nome melhor para esse método pode ser `allow_exit_without_flush()`. É provável que ele faça com que dados enfileirados sejam perdidos, e você quase certamente não precisará usá-lo. Ele está realmente lá somente se você precisar que o processo atual saia imediatamente sem esperar para liberar dados enfileirados para o encadeamento subjacente, e você não se importa com dados perdidos.

Nota

A funcionalidade desta classe requer uma implementação de semáforo compartilhado funcional no sistema operacional host. Sem uma, a funcionalidade nesta classe será desabilitada, e as tentativas de instanciar uma **`Queue`** resultarão em um **`ImportError`**. Veja bpo-3770 [https://bugs.python.org/issue?@action=redirect&bpo=3770] para informações

adicionais. O mesmo vale para qualquer um dos tipos de fila especializados listados abaixo.

class multiprocessing.SimpleQueue

É um tipo `Queue` simplificado, muito próximo de um `Pipe` travado.

Instantiating this class may set the global start method. See Global start method for more details.

close()

Fecha a fila: libera recursos internos.

Uma fila não deve mais ser usada depois de fechada. Por exemplo, os métodos `get()`, `put()` e `empty()` não devem mais ser chamados.

Adicionado na versão 3.9.

empty()

Retorna `True` se a fila estiver vazia, `False` caso contrário.

Sempre levanta `OSError` se a SimpleQueue estiver fechada.

get()

Remove e retorna um item da fila.

put(*item*)

Coloca *item* na fila.

class multiprocessing.JoinableQueue([*maxsize*])

`JoinableQueue`, uma subclasse de `Queue`, é uma fila que também possui os métodos `task_done()` e `join()`.

Instantiating this class may set the global start method. See Global start method for more details.

task_done()

Indica que a tarefa anteriormente enfileirado está concluída. Para cada `get()` usado para buscar uma tarefa, uma chamada subsequente para `task_done()` avisa à fila, que o processamento na tarefa está concluído.

Se um `join()` estiver causando bloqueio no momento, ele irá continuar quando todos os itens tiverem sido processados (significando que uma chamada `task_done()` foi recebida para cada item que foi chamado o método `put()` para colocar na fila).

Levanta `ValueError` se for chamado mais vezes do que o número de itens colocados na fila.

join()

Bloqueia até que todos os itens na fila tenham sido obtidos e processados.

A contagem de tarefas inacabadas aumenta sempre que um item é adicionado à fila. A contagem diminui sempre que uma consumidora chama `task_done()` para indicar que o item foi recuperado e todo o trabalho nele foi concluído. Quando a contagem de tarefas inacabadas chega a zero, `join()` desbloqueia.

Diversos

multiprocessing.active_children()

Retorna a lista de todos os filhos ativos do processo atual.

Chamar isso tem o efeito colateral de “juntar” quaisquer processos que já tenham sido concluídos.

multiprocessing.cpu_count()

Retorna o número de CPUs no sistema.

Este número não equivale ao número de CPUs que o processo atual pode usar. O número de CPUs utilizáveis pode ser obtido com

`os.process_cpu_count()` (ou `len(os.sched_getaffinity(0))`).

Quando o número de CPUs não pode ser determinado, uma `NotImplementedError` é levantada.

Ver também

`os.cpu_count()` `os.process_cpu_count()`

Alterado na versão 3.13: O valor de retorno também pode ser substituído usando o sinalizador `-X cpu_count` ou `PYTHON_CPU_COUNT`, pois este é apenas um invólucro em torno das APIs de contagem de CPU do `os`.

multiprocessing.current_process()

Retorna o objeto `Process` correspondente ao processo atual.

Um análogo de `threading.current_thread()`.

multiprocessing.parent_process()

Retorna o objeto `Process` correspondente ao processo pai do `current_process()`. Para o processo principal, `parent_process` será `None`.

Adicionado na versão 3.8.

multiprocessing.freeze_support()

Add support for when a program which uses **`multiprocessing`** has been frozen to produce an executable. (Has been tested with **py2exe**, **PyInstaller** and **cx_Freeze**.)

É preciso chamar esta função logo após a linha `if __name__ == '__main__'` do módulo principal. Por exemplo:

```
from multiprocessing import Process, freeze_support

def f():
    print('hello world!')
```

```
if __name__ == '__main__':
    freeze_support()
    Process(target=f).start()
```

Se a linha `freeze_support()` for omitida, tentar executar o executável congelado levantará **`RuntimeError`**.

Calling `freeze_support()` has no effect when the start method is not *spawn*. In addition, if the module is being run normally by the Python interpreter (the program has not been frozen), then `freeze_support()` has no effect.

multiprocessing.get_all_start_methods()

Retorna uma lista dos métodos de inicialização suportados, o primeiro dos quais é o padrão. Os métodos possíveis são `'fork'`, `'spawn'` e `'forkserver'`. Nem todas as plataformas suportam todos os métodos. Consulte Contextos e métodos de inicialização.

Adicionado na versão 3.4.

multiprocessing.get_context(*method=None*)

Return a context object which has the same attributes as the **`multiprocessing`** module.

If *method* is `None` then the default context is returned. Note that if the global start method has not been set, this will set it to the system default See Global start method for more details. Otherwise *method* should be `'fork'`, `'spawn'`, `'forkserver'`. **`ValueError`** is raised if the specified start method is not available. See Contextos e métodos de inicialização.

Adicionado na versão 3.4.

multiprocessing.get_start_method(*allow_none=False*)

Retorna o nome do método de inicialização usado para iniciar processos.

If the global start method is not set and *allow_none* is `False`, the global start method is set to the default, and its name is returned. See Global start method for more details.

O valor de retorno pode ser `'fork'`, `'spawn'`, `'forkserver'` ou `None`. Veja Contextos e métodos de inicialização.

Adicionado na versão 3.4.

Alterado na versão 3.8: No macOS, o método de inicialização *spawn* agora é o padrão. O método de inicialização *fork* deve ser considerado inseguro, pois pode levar a travamentos do subprocesso. Consulte bpo-33725 [https://bugs.python.org/issue?@action=redirect&bpo=33725].

multiprocessing.set_executable(*executable*)

Set the path of the Python interpreter to use when starting a child process. (By default **`sys.executable`** is used). Embedders will probably need to do something like

```
set_executable(os.path.join(sys.exec_prefix, 'pythonw.exe'))
```

antes que eles possam criar processos filho.

Alterado na versão 3.4: Agora com suporte no POSIX quando o método de inicialização `'spawn'` é usado.

Alterado na versão 3.11: Aceita um objeto caminho ou similar.

multiprocessing.set_forkserver_preload(*module_names*, *, *on_error='ignore'*)

Set a list of module names for the forkserver main process to attempt to import so that their already imported state is inherited by forked processes. This can be used as a performance enhancement to avoid repeated work in every process.

Para que isso aconteça, trabalho deve ser chamado antes que o forkserver processar tenha sido iniciado (antes de criar um **`Pool`** ou iniciar um **`Process`**).

The *on_error* parameter controls how `ImportError` exceptions during module preloading are handled: `"ignore"` (default) silently ignores failures, `"warn"` causes the forkserver subprocess to emit an `ImportWarning` to stderr, and `"fail"` causes the forkserver subprocess to exit with the exception traceback on stderr, making subsequent process creation fail with `EOFError` or `ConnectionError`.

Só é significativo ao usar o método de inicialização `'forkserver'`. Consulte Contextos e métodos de inicialização.

Adicionado na versão 3.4.

Alterado na versão 3.15: Added the *on_error* parameter.

multiprocessing.set_start_method(*method*, *force=False*)

Define o método que deve ser usado para iniciar processos filhos. O argumento *method* pode ser `'fork'`, `'spawn'` ou `'forkserver'`. Levanta `RuntimeError` se o método de inicialização já tiver sido definido e *force* não for `True`. Se *method* for `None` e *force* for `True`, o método de inicialização será definido como `None`. Se *method* for `None` e *force* for `False`, o contexto será definido como o contexto padrão.

Observe que isso deve ser chamado no máximo uma vez e deve ser protegido dentro da cláusula `if __name__ == '__main__'` do módulo principal.

Veja Contextos e métodos de inicialização.

Adicionado na versão 3.4.

Nota

`multiprocessing` contains no analogues of `threading.active_count()`, `threading.enumerate()`, `threading.settrace()`, `threading.setprofile()`, `threading.Timer`, or `threading.local`.

Objetos de conexão

Objetos de conexão permitem o envio e recebimento de strings e objetos que podem ser serializados com pickle. Eles podem ser pensados como sockets conectados orientados a mensagens.

Objetos de conexão geralmente são criados usando `Pipe` – veja também Listeners and Clients.

class multiprocessing.connection.Connection

send(*obj*)

Envia um objeto para a outra extremidade da conexão que deve ser lido usando `recv()`.

O objeto deve poder ser serializado com pickle. Serializações muito grandes com pickles (aproximadamente 32 MiB+, embora isso dependa do sistema operacional) podem levantar uma exceção `ValueError`.

recv()

Retorna um objeto enviado a partir da outra extremidade da conexão usando `send()`. Bloqueia até que haja algo para receber. Levanta `EOFError` se não houver mais nada para receber e a outra extremidade tenha sido fechada.

fileno()

Retorna o descritor de arquivo ou identificador usado pela conexão.

close()

Fecha a conexão.

Isso é chamado automaticamente quando a conexão é coletada como lixo.

poll([*timeout*])

Retorna se há algum dado disponível para leitura.

Se *timeout* não for especificado, ele retornará imediatamente. Se *timeout* for um número, isso especificará o tempo máximo em segundos para bloquear. Se *timeout* for `None`, um tempo limite infinito será usado.

Observe que vários objetos de conexão podem ser pesquisados ao mesmo tempo usando `multiprocessing.connection.wait()`.

send_bytes(*buf*[, *offset*[, *size*]])

Enviar dados de bytes de um objeto bytes ou similar como uma mensagem completa.

If *offset* is given then data is read from that position in *buf*. If *size* is given then that many bytes will be read from *buf*. Very large buffers (approximately 32 MiB+, though it depends on the OS) may raise a `ValueError` exception

recv_bytes([*maxlength*])

Retorna uma mensagem completa de dados como bytes enviados a partir da outra extremidade da conexão como uma string. Bloqueia até que haja algo para receber. Levanta `EOFError` se não houver mais nada para receber e a outra extremidade tenha sido fechada.

Se *maxlength* for especificado e a mensagem for maior que *maxlength*, `OSError` será levantada e a conexão não será mais legível.

Alterado na versão 3.3: Esta função costumava levantada `IOError`, que agora é um apelido de `OSError`.

recv_bytes_into(*buf*[, *offset*])

Read into *buf* a complete message of byte data sent from the other end of the connection and return the number of bytes in the message. Blocks until there is something to receive. Raises `EOFError` if there is nothing left to receive and the other end was closed.

buf must be a writable bytes-like object. If *offset* is given then the message will be written into the buffer from that position. Offset must be a non-negative integer less than the length of *buf* (in bytes).

Se o buffer for muito curto, uma exceção **`BufferTooShort`** será levantada e a mensagem completa estará disponível como `e.args[0]`, onde `e` é a instância da exceção.

Alterado na versão 3.3: Os próprios objetos de conexão agora podem ser transferidos entre processos usando **`Connection.send()`** e **`Connection.recv()`**.

Objetos de conexão agora também oferecem suporte ao protocolo de gerenciamento de contexto — veja Tipos de Gerenciador de Contexto. **`__enter__()`** retorna o objeto de conexão, e **`__exit__()`** chama **`close()`**.

Por exemplo:

```
>>> from multiprocessing import Pipe
>>> a, b = Pipe()
>>> a.send([1, 'hello', None])
>>> b.recv()
[1, 'hello', None]
>>> b.send_bytes(b'thank you')
>>> a.recv_bytes()
b'thank you'
>>> import array
>>> arr1 = array.array('i', range(5))
>>> arr2 = array.array('i', [0] * 10)
>>> a.send_bytes(arr1)
>>> count = b.recv_bytes_into(arr2)
>>> assert count == len(arr1) * arr1.itemsize
>>> arr2
array('i', [0, 1, 2, 3, 4, 0, 0, 0, 0, 0])
```

Aviso

O método **`Connection.recv()`** deserializa com picke automaticamente os dados recebidos, o que pode ser um risco à segurança, a menos que você

possa confiar no processo que enviou a mensagem.

Portanto, a menos que o objeto de conexão tenha sido produzido usando `Pipe()`, você deve usar apenas os métodos `recv()` e `send()` após executar algum tipo de autenticação. Veja Authentication keys.

Aviso

Se um processo for encerrado enquanto estiver tentando ler ou escrever em um encadeamento, os dados no encadeamento provavelmente serão corrompidos, porque pode se tornar impossível ter certeza de onde estão os limites do encadeamento da mensagem.

Primitivas de sincronização

Geralmente, primitivas de sincronização não são tão necessárias em um programa multiprocesso quanto em um programa multithread. Veja a documentação do módulo `threading`.

Observe que também é possível criar primitivas de sincronização usando um objeto gerenciador — veja Gerenciadores.

class multiprocessing.Barrier(*parties*[, *action*[, *timeout*]])

Um objeto de barreira: um clone de `threading.Barrier`.

Instantiating this class may set the global start method. See Global start method for more details.

Adicionado na versão 3.3.

class multiprocessing.BoundedSemaphore([*value*])

Um objeto semáforo delimitado: um análogo próximo de `threading.BoundedSemaphore`.

Instantiating this class may set the global start method. See Global start method for more details.

Existe uma única diferença em relação ao seu análogo próximo: o primeiro argumento do método `acquire` é chamado *block*, como é consistente com **`Lock.acquire()`**.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

Nota

No macOS, isso é indistinguível de **`Semaphore`** porque `sem_getvalue()` não está implementado nessa plataforma.

class multiprocessing.Condition([*lock*])

Uma variável de condição: um apelido para **`threading.Condition`**.

If *lock* is specified then it should be a **`Lock`** or **`RLock`** object from **`multiprocessing`**.

Instantiating this class may set the global start method. See Global start method for more details.

Alterado na versão 3.3: O método **`wait_for()`** foi adicionado.

class multiprocessing.Event

Um clone de **`threading.Event`**.

Instantiating this class may set the global start method. See Global start method for more details.

class multiprocessing.Lock

Um objeto de trava não recursivo: um análogo próximo de **`threading.Lock`**. Uma vez que um processo ou thread tenha adquirido

um trava, tentativas subsequentes de adquiri-la de qualquer processo ou thread bloquearão até que ele seja liberada; qualquer processo ou thread pode liberá-la. Os conceitos e comportamentos de **`threading.Lock`** conforme se aplica a threads são replicados aqui em **`multiprocessing.Lock`** conforme se aplica a processos ou threads, exceto conforme observado.

Observe que **`Lock`** é na verdade uma função de fábrica que retorna uma instância de `multiprocessing.synchronize.Lock` inicializada com um contexto padrão.

Instantiating this class may set the global start method. See Global start method for more details.

`Lock` oferece suporte ao protocolo gerenciador de contexto e, portanto, pode ser usado em instruções **`with`**.

acquire(*block=True*, *timeout=None*)

Adquire uma trava, bloqueante ou não.

Com o argumento *block* definido como `True` (o padrão), a chamada do método bloqueará até que a trava esteja em um estado destravado, então o definirá como travada e retornará `True`. Observe que o nome deste primeiro argumento difere daquele em **`threading.Lock.acquire()`**.

Com o argumento *block* definido como `False`, a chamada do método não bloqueia. Se a trava estiver atualmente em um estado travado, retorna `False`; caso contrário, defina a trava para um estado travado e retorna `True`.

Quando invocado com um valor positivo de ponto flutuante para *timeout*, bloqueie por no máximo o número de segundos especificado por *timeout* enquanto a trava não puder ser adquirida. Invocações com um valor negativo para *timeout* são equivalentes a um *timeout* de zero. Invocações com um valor *timeout* de `None` (o padrão) definem o período de tempo limite como infinito. Observe que o tratamento de valores negativos ou `None` para *timeout* difere

do comportamento implementado em `threading.Lock.acquire()`. O argumento *timeout* não tem implicações práticas se o argumento *block* for definido como `False` e, portanto, for ignorado. Retorna `True` se a trava tiver sido adquirida ou `False` se o período de tempo limite tiver decorrido.

release()

Libera uma trava. Isso pode ser chamado de qualquer processo ou thread, não apenas do processo ou thread que originalmente adquiriu a trava.

O comportamento é o mesmo de `threading.Lock.release()`, exceto que quando invocado em uma trava desatravada, uma `ValueError` é levantada.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

class multiprocessing.RLock

Um objeto de trava recursiva: um análogo próximo de `threading.RLock`. Uma trava recursiva deve ser liberada pelo processo ou thread que o adquiriu. Uma vez que um processo ou thread tenha adquirido uma trava recursiva, o mesmo processo ou thread pode adquiri-la novamente sem trava; esse processo ou thread deve liberá-la uma vez para cada vez que ela foi adquirida.

Observe que `RLock` é na verdade uma função de fábrica que retorna uma instância de `multiprocessing.synchronize.RLock` inicializada com um contexto padrão.

Instantiating this class may set the global start method. See Global start method for more details.

`RLock` oferece suporte ao protocolo gerenciador de contexto e, portanto, pode ser usado em instruções `with`.

acquire(*block=True*, *timeout=None*)

Adquire uma trava, bloqueante ou não.

Quando invocado com o argumento *block* definido como `True`, bloqueia até que a trava esteja em um estado destravado (não pertencente a nenhum processo ou thread), a menos que a trava já seja de propriedade do processo ou thread atual. O processo ou thread atual então assume a propriedade da trava (se ainda não tiver propriedade) e o nível de recursão dentro de incrementos por um da trava, resultando em um valor de retorno de `True`. Observe que há várias diferenças no comportamento deste primeiro argumento em comparação com a implementação de **`threading.RLock.acquire()`**, começando pelo nome do argumento em si.

Quando invocado com o argumento *block* definido como `False`, não bloqueie. Se a trava já tiver sido adquirida (e, portanto, for de propriedade) por outro processo ou thread, o processo ou thread atual não assume a propriedade e o nível de recursão dentro da trava não é alterada, resultando em um valor de retorno de `False`. Se a trava estiver em um estado destravado, o processo ou thread atual assume a propriedade e o nível de recursão é incrementado, resultando em um valor de retorno de `True`.

O uso e os comportamentos do argumento *timeout* são os mesmos que em **`Lock.acquire()`**. Observe que alguns desses comportamentos de *timeout* diferem dos comportamentos implementados em **`threading.RLock.acquire()`**.

release()

Libera uma trava, decrementando o nível de recursão. Se após o decremento o nível de recursão for zero, redefine a trava para destravada (não pertencente a nenhum processo ou thread) e se quaisquer outros processos ou threads estiverem bloqueados esperando a trava ser destravada, permita que exatamente um deles prossiga. Se após o decremento o nível de recursão ainda for

diferente de zero, o trava permanece travada e pertencente ao processo ou thread de chamada.

Somente chame esse método quando o processo ou thread de chamada for proprietária da trava. Uma `AssertionError` é levantada se esse método for chamado por um processo ou thread diferente do proprietário ou se a trava estiver em um estado destravado (sem proprietário). Observe que o tipo de exceção levantada nessa situação difere do comportamento implementado em `threading.RLock.release()`.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

class multiprocessing.Semaphore([*value*])

Um objeto semáforo: um análogo próximo de `threading.Semaphore`.

Instantiating this class may set the global start method. See Global start method for more details.

Existe uma única diferença em relação ao seu análogo próximo: o primeiro argumento do método `acquire` é chamado *block*, como é consistente com `Lock.acquire()`.

get_value()

Return the current value of semaphore.

Observe que isso pode levantar `NotImplementedError` em plataformas como macOS, onde `sem_getvalue()` não está implementado.

locked()

Return a boolean indicating whether this object is locked right now.

Adicionado na versão 3.14.

Nota

No macOS, não há suporte a `sem_timedwait`, então chamar `acquire()` com um tempo limite emulará o comportamento dessa função usando um laço de suspensão.

Nota

Algumas das funcionalidades deste pacote exibem uma implementação de semáforo compartilhado funcional no sistema operacional host. Sem uma, o módulo **`multiprocessing.synchronize`** será desabilitado e as tentativas de importação dele resultarão em um **`ImportError`**. Veja bpo-3770 [https://bugs.python.org/issue?@action=redirect&bpo=3770] para informações adicionais.

Objetos `ctypes` compartilhados

É possível criar objetos compartilhados usando memória compartilhada que pode ser herdada por processos filhos.

multiprocessing.Value(*typecode_or_type*, **args*, *lock=True*)

Retorna um objeto **`ctypes`** alocado da memória compartilhada. Por padrão, o valor de retorno é, na verdade, um invólucro sincronizado para o objeto. O objeto em si pode ser acessado por meio do atributo *value* de um **`Value`**.

typecode_or_type determina o tipo do objeto retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo **`array`**. **args* é passado para o construtor para o tipo.

Se *lock* for `True` (o padrão), então um novo objeto de trava recursiva é criado para sincronizar o acesso ao valor. Se *lock* for um objeto **`Lock`** ou **`RLock`**, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido

automaticamente por uma trava, então ele não será necessariamente “seguro para processo”.

Operações como `+=` que envolvem uma leitura e escrita não são atômicas. Então se, por exemplo, você quiser incrementar atomicamente um valor compartilhado, não é suficiente apenas fazer

```
counter.value += 1
```

Supondo que a trava associada seja recursiva (o que é por padrão), você pode fazer

```
with counter.get_lock():
    counter.value += 1
```

Observe que *lock* é um argumento somente-nomeado.

multiprocessing.Array(*typecode_or_type*, *size_or_initializer*, *, *lock=True*)

Retorna um vetor de ctypes alocado da memória compartilhada. Por padrão, o valor de retorno é, na verdade, um invólucro sincronizado para o vetor.

typecode_or_type determines the type of the elements of the returned array: it is either a ctypes type or a one character typecode of the kind used by the **`array`** module with the exception of `'w'`, which is not supported. In addition, the `'c'` typecode is an alias for **`ctypes.c_char`**. If *size_or_initializer* is an integer, then it determines the length of the array, and the array will be initially zeroed. Otherwise, *size_or_initializer* is a sequence which is used to initialize the array and whose length determines the length of the array.

Se *lock* for `True` (o padrão), então um novo objeto de trava é criado para sincronizar o acesso ao valor. Se *lock* for um objeto **`Lock`** ou **`RLock`**, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente “seguro para processo”.

Observe que *lock* é um argumento somente-nomeado.

Note that an array of `ctypes.c_char` has *value* and *raw* attributes which can both be used to store and retrieve byte strings. While *raw* allows interaction with a `bytes` object the full size of the array, reading *value* will terminate after a null byte, like most programming languages handle strings.

The `multiprocessing.sharedctypes` module

The `multiprocessing.sharedctypes` module provides functions for allocating `ctypes` objects from shared memory which can be inherited by child processes.

Nota

Embora seja possível armazenar um ponteiro na memória compartilhada, lembre-se de que isso se referirá a um local no espaço de endereço de um processo específico. No entanto, é bem provável que o ponteiro seja inválido no contexto de um segundo processo e tentar desreferenciar o ponteiro do segundo processo pode causar um travamento.

multiprocessing.sharedctypes.RawArray(*typecode_or_type*, *size_or_initializer*)

Retorna um vetor de ctypes alocado da memória compartilhada.

typecode_or_type determina o tipo dos elementos do vetor retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo `array`. Se *size_or_initializer* for um inteiro, então ele determina o comprimento do vetor, e o vetor será inicialmente zerado. Caso contrário, *size_or_initializer* é uma sequência que é usada para inicializar o vetor e cujo comprimento determina o comprimento do vetor.

Observe que definir e obter um elemento é potencialmente não atômico – use `Array()` para garantir que o acesso seja sincronizado automaticamente usando uma trava.

multiprocessing.sharedctypes.RawValue(*typecode_or_type*, **args*)

Retorna um objeto ctypes alocado da memória compartilhada.

typecode_or_type determina o tipo do objeto retornado: é um tipo ctypes ou um typecode de um caractere do tipo usado pelo módulo **array**. **args* é passado para o construtor para o tipo.

Observe que definir e obter o valor é potencialmente não atômico – use **Value()** para garantir que o acesso seja sincronizado automaticamente usando uma trava.

Observe que um vetor de **ctypes.c_char** tem atributos `value` e `raw` que permitem usá-la para armazenar e recuperar strings – veja a documentação de **ctypes**.

multiprocessing.sharedctypes.Array(*typecode_or_type*, *size_or_initializer*, *, *lock=True*, *ctx=None*)

O mesmo que **RawArray()**, exceto que dependendo do valor de *lock*, um invólucro de sincronização seguro para processo pode ser retornado em vez de um vetor de ctypes brutos.

Se *lock* for `True` (o padrão), então um novo objeto de trava é criado para sincronizar o acesso ao valor. Se *lock* for um objeto **Lock** ou **RLock**, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente "seguro para processo".

ctx is a context object, or `None` (use the current context). If `None`, calling this may set the global start method. See Global start method for more details.

Note that *lock* and *ctx* are keyword-only parameters.

multiprocessing.sharedctypes.Value(*typecode_or_type*, **args*, *lock=True*, *ctx=None*)

O mesmo que **`RawValue()`**, exceto que dependendo do valor de *lock*, um invólucro de sincronização seguro para processo pode ser retornado em vez de um objeto ctypes brutos.

Se *lock* for `True` (o padrão), então um novo objeto de trava é criado para sincronizar o acesso ao valor. Se *lock* for um objeto **`Lock`** ou **`RLock`**, então ele será usado para sincronizar o acesso ao valor. Se *lock* for `False`, então o acesso ao objeto retornado não será protegido automaticamente por uma trava, então ele não será necessariamente "seguro para processo".

ctx is a context object, or `None` (use the current context). If `None`, calling this may set the global start method. See Global start method for more details.

Note that *lock* and *ctx* are keyword-only parameters.

multiprocessing.sharedctypes.copy(*obj*)

Retorna um objeto ctypes alocado da memória compartilhada que é uma cópia do objeto ctypes *obj*.

multiprocessing.sharedctypes.synchronized(*obj*, *lock=None*, *ctx=None*)

Retorna um objeto invólucro seguro para o processo para um objeto ctypes que usa *lock* para sincronizar o acesso. Se *lock* for `None` (o padrão), então um objeto **`multiprocessing.RLock`** é criado automaticamente.

ctx is a context object, or `None` (use the current context). If `None`, calling this may set the global start method. See Global start method for more details.

A synchronized wrapper will have two methods in addition to those of the object it wraps: **`get_obj()`** returns the wrapped object and **`get_lock()`** returns the lock object used for synchronization.

Note that accessing the ctypes object through the wrapper can be a lot slower than accessing the raw ctypes object.

Alterado na versão 3.5: Synchronized objects support the context manager protocol.

The table below compares the syntax for creating shared ctypes objects from shared memory with the normal ctypes syntax. (In the table `MyStruct` is some subclass of `ctypes.Structure`.)

ctypes	sharedctypes using type	sharedctypes using typecode
c_double(2.4)	RawValue(c_double, 2.4)	RawValue(‘d’, 2.4)
MyStruct(4, 6)	RawValue(MyStruct, 4, 6)	
(c_short * 7)()	RawArray(c_short, 7)	RawArray(‘h’, 7)
(c_int * 3)(9, 2, 8)	RawArray(c_int, (9, 2, 8))	RawArray(‘i’, (9, 2, 8))

Below is an example where a number of ctypes objects are modified by a child process:

```
from multiprocessing import Process, Lock
from multiprocessing.sharedctypes import Value, Array
from ctypes import Structure, c_double

class Point(Structure):
    _fields_ = [('x', c_double), ('y', c_double)]

def modify(n, x, s, A):
    n.value **= 2
    x.value **= 2
```

```
    s.value = s.value.upper()
    for a in A:
        a.x **= 2
        a.y **= 2

if __name__ == '__main__':
    lock = Lock()

    n = Value('i', 7)
    x = Value(c_double, 1.0/3.0, lock=False)
    s = Array('c', b'hello world', lock=lock)
    A = Array(Point, [(1.875,-6.25), (-5.75,2.0), (2.375,9.5)],

    p = Process(target=modify, args=(n, x, s, A))
    p.start()
    p.join()

    print(n.value)
    print(x.value)
    print(s.value)
    print([(a.x, a.y) for a in A])
```

The results printed are

```
49
0.1111111111111111
HELLO WORLD
[(3.515625, 39.0625), (33.0625, 4.0), (5.640625, 90.25)]
```

Gerenciadores

Managers provide a way to create data which can be shared between different processes, including sharing over a network between processes running on different machines. A manager object controls a server process which manages *shared objects*. Other processes can access the shared objects by using proxies.

multiprocessing.Manager()

Retorna um objeto `SyncManager` inicializado que pode ser usado para compartilhar objetos entre processos. O objeto gerenciador retornado

corresponde a um processo filho gerado e tem métodos que criarão objetos compartilhados e retornarão proxies correspondentes.

Manager processes will be shutdown as soon as they are garbage collected or their parent process exits. The manager classes are defined in the **multiprocessing.managers** module:

class multiprocessing.managers.BaseManager(*address=None*, *authkey=None*, *serializer='pickle'*, *ctx=None*, *, *shutdown_timeout=1.0*)

Criando um objeto BaseManager.

Once created one should call **start()** or `get_server().serve_forever()` to ensure that the manager object refers to a started manager process.

address is the address on which the manager process listens for new connections. If *address* is `None` then an arbitrary one is chosen.

authkey is the authentication key which will be used to check the validity of incoming connections to the server process. If *authkey* is `None` then `current_process().authkey` is used. Otherwise *authkey* is used and it must be a byte string.

serializer must be `'pickle'` (use **pickle** serialization) or `'xmlrpclib'` (use **xmlrpc.client** serialization).

ctx is a context object, or `None` (use the current context). If `None`, calling this may set the global start method. See Global start method for more details.

shutdown_timeout is a timeout in seconds used to wait until the process used by the manager completes in the **shutdown()** method. If the shutdown times out, the process is terminated. If terminating the process also times out, the process is killed.

Alterado na versão 3.11: Added the *shutdown_timeout* parameter.

start([*initializer*[, *initargs*]])

Start a subprocess to start the manager. If *initializer* is not `None` then the subprocess will call `initializer(*initargs)` when it starts.

get_server()

Returns a **`Server`** object which represents the actual server under the control of the Manager. The **`Server`** object supports the **`serve_forever()`** method:

```
>>> from multiprocessing.managers import BaseManager
>>> manager = BaseManager(address=('', 50000), authkey=b'a
>>> server = manager.get_server()
>>> server.serve_forever()
```

`Server` additionally has an **`address`** attribute.

connect()

Connect a local manager object to a remote manager process:

```
>>> from multiprocessing.managers import BaseManager
>>> m = BaseManager(address=('127.0.0.1', 50000), authkey=
>>> m.connect()
```

shutdown()

Stop the process used by the manager. This is only available if **`start()`** has been used to start the server process.

This can be called multiple times.

register(*typeid*[, *callable*[, *proxytype*[, *exposed*[, *method_to_typeid*[, *create_method*]]]]])

A classmethod which can be used for registering a type or callable with the manager class.

typeid is a “type identifier” which is used to identify a particular type of shared object. This must be a string.

callable is a callable used for creating objects for this type identifier. If a manager instance will be connected to the server using the **connect()** method, or if the *create_method* argument is `False` then this can be left as `None`.

proxytype is a subclass of **BaseProxy** which is used to create proxies for shared objects with this *typeid*. If `None` then a proxy class is created automatically.

exposed is used to specify a sequence of method names which proxies for this typeid should be allowed to access using **BaseProxy.__callmethod()**. (If *exposed* is `None` then **proxytype._exposed_** is used instead if it exists.) In the case where no exposed list is specified, all "public methods" of the shared object will be accessible. (Here a "public method" means any attribute which has a **__call__()** method and whose name does not begin with `'_'`.)

method_to_typeid is a mapping used to specify the return type of those exposed methods which should return a proxy. It maps method names to typeid strings. (If *method_to_typeid* is `None` then **proxytype._method_to_typeid_** is used instead if it exists.) If a method's name is not a key of this mapping or if the mapping is `None` then the object returned by the method will be copied by value.

create_method determines whether a method should be created with name *typeid* which can be used to tell the server process to create a new shared object and return a proxy for it. By default it is `True`.

BaseManager instances also have one read-only property:

address

The address used by the manager.

Alterado na versão 3.3: Manager objects support the context management protocol – see Tipos de Gerenciador de Contexto. **__enter__()** starts the server process (if it has not already started) and then returns the manager object. **__exit__()** calls **shutdown()**.

In previous versions `__enter__()` did not start the manager's server process if it was not already started.

class multiprocessing.managers.SyncManager

A subclass of `BaseManager` which can be used for the synchronization of processes. Objects of this type are returned by `multiprocessing.Manager()`.

Its methods create and return Objetos proxies for a number of commonly used data types to be synchronized across processes. This notably includes shared lists and dictionaries.

Barrier(*parties*[, *action*[, *timeout*]])

Create a shared `threading.Barrier` object and return a proxy for it.

Adicionado na versão 3.3.

BoundedSemaphore([*value*])

Create a shared `threading.BoundedSemaphore` object and return a proxy for it.

Condition([*lock*])

Create a shared `threading.Condition` object and return a proxy for it.

If *lock* is supplied then it should be a proxy for a `threading.Lock` or `threading.RLock` object.

Alterado na versão 3.3: O método `wait_for()` foi adicionado.

Event()

Create a shared `threading.Event` object and return a proxy for it.

Lock()

Create a shared `threading.Lock` object and return a proxy for it.

Namespace()

Create a shared `Namespace` object and return a proxy for it.

Queue([*maxsize*])

Create a shared `queue.Queue` object and return a proxy for it.

RLock()

Create a shared `threading.RLock` object and return a proxy for it.

Semaphore([*value*])

Create a shared `threading.Semaphore` object and return a proxy for it.

Array(*typecode*, *sequence*)

Create an array and return a proxy for it.

Value(*typecode*, *value*)

Create an object with a writable `value` attribute and return a proxy for it.

dict()
dict(*mapping*)
dict(*sequence*)

Create a shared `dict` object and return a proxy for it.

list()
list(*sequence*)

Create a shared `list` object and return a proxy for it.

set()
set(*sequence*)
set(*mapping*)

Cria um objeto `set` compartilhado e retorna um proxy para ele.

Adicionado na versão 3.14: Foi adicionado suporte para `set`.

Alterado na versão 3.6: Objetos compartilhados podem ser aninhados. Por exemplo, em um objeto contêiner compartilhado, como uma lista compartilhada, é possível conter outros objeto compartilhados que serão gerenciados e sincronizados pelo `SyncManager`.

class multiprocessing.managers.Namespace

A type that can register with `SyncManager`.

A namespace object has no public methods, but does have writable attributes. Its representation shows the values of its attributes.

However, when using a proxy for a namespace object, an attribute beginning with `'_'` will be an attribute of the proxy and not an attribute of the referent:

```
>>> mp_context = multiprocessing.get_context('spawn')
>>> manager = mp_context.Manager()
>>> Global = manager.Namespace()
>>> Global.x = 10
>>> Global.y = 'hello'
>>> Global._z = 12.3    # this is an attribute of the proxy
>>> print(Global)
Namespace(x=10, y='hello')
```

Customized managers

To create one's own manager, one creates a subclass of `BaseManager` and uses the `register()` classmethod to register new types or callables with the manager class. For example:

```
from multiprocessing.managers import BaseManager

class MathsClass:
    def add(self, x, y):
        return x + y
    def mul(self, x, y):
        return x * y
```

```
class MyManager(BaseManager):
    pass

MyManager.register('Maths', MathsClass)

if __name__ == '__main__':
    with MyManager() as manager:
        maths = manager.Maths()
        print(maths.add(4, 3))         # prints 7
        print(maths.mul(7, 8))         # prints 56
```

Using a remote manager

It is possible to run a manager server on one machine and have clients use it from other machines (assuming that the firewalls involved allow it).

Running the following commands creates a server for a single shared queue which remote clients can access:

```
>>> from multiprocessing.managers import BaseManager
>>> from queue import Queue
>>> queue = Queue()
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue', callable=lambda:queue)
>>> m = QueueManager(address=('', 50000), authkey=b'abracadabra'
>>> s = m.get_server()
>>> s.serve_forever()
```

One client can access the server as follows:

```
>>> from multiprocessing.managers import BaseManager
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue')
>>> m = QueueManager(address=('foo.bar.org', 50000), authkey=b'a
>>> m.connect()
>>> queue = m.get_queue()
>>> queue.put('hello')
```

Another client can also use it:

```
>>> from multiprocessing.managers import BaseManager
>>> class QueueManager(BaseManager): pass
>>> QueueManager.register('get_queue')
>>> m = QueueManager(address=('foo.bar.org', 50000), authkey=b'a
>>> m.connect()
>>> queue = m.get_queue()
>>> queue.get()
'hello'
```

Local processes can also access that queue, using the code from above on the client to access it remotely:

```
>>> from multiprocessing import Process, Queue
>>> from multiprocessing.managers import BaseManager
>>> class Worker(Process):
...     def __init__(self, q):
...         self.q = q
...         super().__init__()
...     def run(self):
...         self.q.put('local hello')
...
>>> queue = Queue()
>>> w = Worker(queue)
>>> w.start()
>>> class QueueManager(BaseManager): pass
...
>>> QueueManager.register('get_queue', callable=lambda: queue)
>>> m = QueueManager(address=('', 50000), authkey=b'abracadabra'
>>> s = m.get_server()
>>> s.serve_forever()
```

Objetos proxies

Um proxy é um objeto que *refere-se* a um objeto compartilhado que reside (presumivelmente) em um processo diferente. Dizemos que o objeto compartilhado é o *referente* do proxy. Vários objetos proxies podem ter o mesmo referente.

Um proxy objeto tem métodos que invocam métodos correspondes de seu referente (embora nem todo método do referente esteja necessariamente disponível por meio do proxy). Dessa forma, um proxy pode ser usado da mesma forma que seu referente:

```
>>> mp_context = multiprocessing.get_context('spawn')
>>> manager = mp_context.Manager()
>>> l = manager.list([i*i for i in range(10)])
>>> print(l)
[0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
>>> print(repr(l))
<ListProxy object, typeid 'list' at 0x...>
>>> l[4]
16
>>> l[2:5]
[4, 9, 16]
```

Observe que, ao aplicar **`str()`** a um proxy, o retorno representará o referente, enquanto que ao aplicar o **`repr()`**, o retorno representará o proxy.

Um aspecto importante dos objetos proxies é o fato de que eles podem ser serializados com pickle, portanto podem ser transferidos entre processos. Dessa forma, um referente pode conter Objetos proxies. Isso permite o aninhamento dessas listas e dicionários gerenciados e outros Objetos proxies:

```
>>> a = manager.list()
>>> b = manager.list()
>>> a.append(b)         # referente de a agora contém referente
>>> print(a, b)
[<ListProxy object, typeid 'list' at ...>] []
>>> b.append('hello')
>>> print(a[0], b)
['hello'] ['hello']
```

Da mesma forma, os proxies de dicionários e listas podem ser aninhados um dentro do outro:

```
>>> l_outer = manager.list([ manager.dict() for i in range(2) ])
>>> d_first_inner = l_outer[0]
```

```
>>> d_first_inner['a'] = 1
>>> d_first_inner['b'] = 2
>>> l_outer[1]['c'] = 3
>>> l_outer[1]['z'] = 26
>>> print(l_outer[0])
{'a': 1, 'b': 2}
>>> print(l_outer[1])
{'c': 3, 'z': 26}
```

Se objetos padrões (não-proxy) de **`list`** ou **`dict`** estiverem contidos em um referente, as modificações nesses mutáveis não serão propagadas pelo gerente porque o proxy não tem como saber quando os valores contidos nele são modificados. No entanto, o armazenamento de um valor em um proxy contêiner (que aciona um `__setitem__` no objeto proxy) se propaga pelo gerenciador e, portanto, para modificar efetivamente esse item, é possível reatribuir o valor modificado ao proxy contêiner:

```
# cria um proxy para uma lista e adicionar um objeto mutável (um
lproxy = manager.list()
lproxy.append({})
# agora modifica o dicionário
d = lproxy[0]
d['a'] = 1
d['b'] = 2
# neste ponto, as mudanças em d não estão ainda sincronizadas, m
# atualizando o dicionário, o proxy é notificado da mudança
lproxy[0] = d
```

Essa abordagem talvez seja menos conveniente do que usar Objetos proxies aninhados para a maioria dos casos de uso, mas também demonstra um nível de controle sobre a sincronização.

Nota

The proxy types in **`multiprocessing`** do nothing to support comparisons by value. So, for instance, we have:

```
>>> manager.list([1,2,3]) == [1,2,3]
False
```

Em vez disso, deve-se usar uma cópia do referente ao fazer comparações.

class multiprocessing.managers.BaseProxy

Proxy objects are instances of subclasses of **`BaseProxy`**.

_callmethod(*methodname*[, *args*[, *kwds*]])

Call and return the result of a method of the proxy's referent.

If `proxy` is a proxy whose referent is `obj` then the expression

```
proxy._callmethod(methodname, args, kwds)
```

will evaluate the expression

```
getattr(obj, methodname)(*args, **kwds)
```

in the manager's process.

The returned value will be a copy of the result of the call or a proxy to a new shared object – see documentation for the *method_to_typeid* argument of **`BaseManager.register()`**.

If an exception is raised by the call, then is re-raised by **`_callmethod()`**. If some other exception is raised in the manager's process then this is converted into a **`RemoteError`** exception and is raised by **`_callmethod()`**.

Note in particular that an exception will be raised if *methodname* has not been *exposed*.

An example of the usage of **`_callmethod()`**:

```
>>> l = manager.list(range(10))
>>> l._callmethod('__len__')
10
>>> l._callmethod('__getitem__', (slice(2, 7),)) # equiva
[2, 3, 4, 5, 6]
>>> l._callmethod('__getitem__', (20,))          # equiva
```

```
Traceback (most recent call last):
...
IndexError: list index out of range
```

_getvalue()

Return a copy of the referent.

If the referent is unpicklable then this will raise an exception.

__repr__()

Return a representation of the proxy object.

__str__()

Return the representation of the referent.

Limpeza

A proxy object uses a weakref callback so that when it gets garbage collected it deregisters itself from the manager which owns its referent.

A shared object gets deleted from the manager process when there are no longer any proxies referring to it.

Process Pools

One can create a pool of processes which will carry out tasks submitted to it with the `Pool` class.

class multiprocessing.pool.Pool([*processes*[, *initializer*[, *initargs*[, *maxtasksperchild*[, *context*]]]]])

A process pool object which controls a pool of worker processes to which jobs can be submitted. It supports asynchronous results with timeouts and callbacks and has a parallel map implementation.

processes is the number of worker processes to use. If *processes* is `None` then the number returned by `os.process_cpu_count()` is used.

If *initializer* is not `None` then each worker process will call `initializer(*initargs)` when it starts.

maxtasksperchild is the number of tasks a worker process can complete before it will exit and be replaced with a fresh worker process, to enable unused resources to be freed. The default *maxtasksperchild* is `None`, which means worker processes will live as long as the pool.

context can be used to specify the context used for starting the worker processes. Usually a pool is created using the function **`multiprocessing.Pool()`** or the **`Pool()`** method of a context object. In both cases *context* is set appropriately. If `None`, calling this function will have the side effect of setting the current global start method if it has not been set already. See the **`get_context()`** function.

Note that the methods of the pool object should only be called by the process which created the pool.

Aviso

`multiprocessing.pool` objects have internal resources that need to be properly managed (like any other resource) by using the pool as a context manager or by calling **`close()`** and **`terminate()`** manually. Failure to do this can lead to the process hanging on finalization.

Note that it is **not correct** to rely on the garbage collector to destroy the pool as CPython does not assure that the finalizer of the pool will be called (see **`object.__del__()`** for more information).

Alterado na versão 3.2: Added the *maxtasksperchild* parameter.

Alterado na versão 3.4: Adicionado o parâmetro *context*.

Alterado na versão 3.13: processes uses **`os.process_cpu_count()`** by default, instead of **`os.cpu_count()`**.

Nota

Worker processes within a **`Pool`** typically live for the complete duration of the Pool's work queue. A frequent pattern found in other systems (such as Apache, mod_wsgi, etc) to free resources held by workers is to allow a worker within a pool to complete only a set amount of work before exiting, being cleaned up and a new process spawned to replace the old one. The *maxtasksperchild* argument to the **`Pool`** exposes this ability to the end user.

apply(*func*[, *args*[, *kwds*]])

Call *func* with arguments *args* and keyword arguments *kwds*. It blocks until the result is ready. Given this blocks, **`apply_async()`** is better suited for performing work in parallel. Additionally, *func* is only executed in one of the workers of the pool.

apply_async(*func*[, *args*[, *kwds*[, *callback*[, *error_callback*]]]])

A variant of the **`apply()`** method which returns a **`AsyncResult`** object.

If *callback* is specified then it should be a callable which accepts a single argument. When the result becomes ready *callback* is applied to it, that is unless the call failed, in which case the *error_callback* is applied instead.

If *error_callback* is specified then it should be a callable which accepts a single argument. If the target function fails, then the *error_callback* is called with the exception instance.

Callbacks should complete immediately since otherwise the thread which handles the results will get blocked.

map(*func*, *iterable*[, *chunksize*])

A parallel equivalent of the **`map()`** built-in function (it supports only one *iterable* argument though, for multiple iterables see **`starmap()`**). It blocks until the result is ready.

This method chops the iterable into a number of chunks which it submits to the process pool as separate tasks. The (approximate) size of these chunks can be specified by setting *chunksize* to a positive integer.

Note that it may cause high memory usage for very long iterables. Consider using `imap()` or `imap_unordered()` with explicit *chunksize* option for better efficiency.

map_async(*func*, *iterable*[, *chunksize*[, *callback*[, *error_callback*]]])

A variant of the `map()` method which returns a `AsyncResult` object.

If *callback* is specified then it should be a callable which accepts a single argument. When the result becomes ready *callback* is applied to it, that is unless the call failed, in which case the *error_callback* is applied instead.

If *error_callback* is specified then it should be a callable which accepts a single argument. If the target function fails, then the *error_callback* is called with the exception instance.

Callbacks should complete immediately since otherwise the thread which handles the results will get blocked.

imap(*func*, *iterable*[, *chunksize*])

An iterator-based version of `map()`.

The *chunksize* argument is the same as the one used by the `map()` method. For very long iterables using a large value for *chunksize* can make the job complete **much** faster than using the default value of `1`.

Also if *chunksize* is `1` then the **`next()`** method of the iterator returned by the **`imap()`** method has an optional *timeout* parameter: `next(timeout)` will raise `multiprocessing.TimeoutError` if the result cannot be returned within *timeout* seconds.

imap_unordered(*func*, *iterable*[, *chunksize*])

The same as `imap()` except that the ordering of the results from the returned iterator should be considered arbitrary. (Only when there is only one worker process is the order guaranteed to be “correct”.)

starmap(*func*, *iterable*[, *chunksize*])

Like `map()` except that the elements of the *iterable* are expected to be iterables that are unpacked as arguments.

Hence an *iterable* of `[(1,2), (3, 4)]` results in `[func(1,2), func(3,4)]`.

Adicionado na versão 3.3.

starmap_async(*func*, *iterable*[, *chunksize*[, *callback*[, *error_callback*]]])

A combination of `starmap()` and `map_async()` that iterates over *iterable* of iterables and calls *func* with the iterables unpacked. Returns a result object.

Adicionado na versão 3.3.

close()

Prevents any more tasks from being submitted to the pool. Once all the tasks have been completed the worker processes will exit.

terminate()

Stops the worker processes immediately without completing outstanding work. When the pool object is garbage collected **`terminate()`** will be called immediately.

join()

Wait for the worker processes to exit. One must call `close()` or `terminate()` before using **`join()`**.

Alterado na versão 3.3: Pool objects now support the context management protocol – see Tipos de Gerenciador de Contexto. `__enter__()` returns the pool object, and `__exit__()` calls `terminate()`.

class multiprocessing.pool.AsyncResult

The class of the result returned by `Pool.apply_async()` and `Pool.map_async()`.

get([*timeout*])

Return the result when it arrives. If *timeout* is not `None` and the result does not arrive within *timeout* seconds then `multiprocessing.TimeoutError` is raised. If the remote call raised an exception then that exception will be reraised by `get()`.

wait([*timeout*])

Wait until the result is available or until *timeout* seconds pass.

ready()

Return whether the call has completed.

successful()

Return whether the call completed without raising an exception. Will raise `ValueError` if the result is not ready.

Alterado na versão 3.7: If the result is not ready, `ValueError` is raised instead of `AssertionError`.

The following example demonstrates the use of a pool:

```
from multiprocessing import Pool
import time

def f(x):
    return x*x

if __name__ == '__main__':
```

```
with Pool(processes=4) as pool:         # start 4 worker pro
    result = pool.apply_async(f, (10,)) # evaluate "f(10)" a
    print(result.get(timeout=1))        # prints "100" unles

    print(pool.map(f, range(10)))       # prints "[0, 1, 4,.

    it = pool.imap(f, range(10))
    print(next(it))                     # prints "0"
    print(next(it))                     # prints "1"
    print(it.next(timeout=1))           # prints "4" unless

    result = pool.apply_async(time.sleep, (10,))
    print(result.get(timeout=1))        # raises multiproces
```

Listeners and Clients

Usually message passing between processes is done using queues or by using `Connection` objects returned by `Pipe()`.

However, the **`multiprocessing.connection`** module allows some extra flexibility. It basically gives a high level message oriented API for dealing with sockets or Windows named pipes. It also has support for *digest authentication* using the `hmac` module, and for polling multiple connections at the same time.

multiprocessing.connection.deliver_challenge(*connection*, *authkey*)

Send a randomly generated message to the other end of the connection and wait for a reply.

If the reply matches the digest of the message using *authkey* as the key then a welcome message is sent to the other end of the connection. Otherwise `AuthenticationError` is raised.

multiprocessing.connection.answer_challenge(*connection*, *authkey*)

Receive a message, calculate the digest of the message using *authkey* as the key, and then send the digest back.

If a welcome message is not received, then **`AuthenticationError`** is raised.

multiprocessing.connection.Client(*address*[, *family*[, *authkey*]])

Attempt to set up a connection to the listener which is using address *address*, returning a **`Connection`**.

The type of the connection is determined by *family* argument, but this can generally be omitted since it can usually be inferred from the format of *address*. (See Formatos de Endereços)

If *authkey* is given and not `None`, it should be a byte string and will be used as the secret key for an HMAC-based authentication challenge. No authentication is done if *authkey* is `None`. **`AuthenticationError`** is raised if authentication fails. See Authentication keys.

class multiprocessing.connection.Listener([*address*[, *family*[, *backlog*[, *authkey*]]]])

A wrapper for a bound socket or Windows named pipe which is 'listening' for connections.

address is the address to be used by the bound socket or named pipe of the listener object.

Nota

If an address of '0.0.0.0' is used, the address will not be a connectable end point on Windows. If you require a connectable end-point, you should use '127.0.0.1'.

family is the type of socket (or named pipe) to use. This can be one of the strings `'AF_INET'` (for a TCP socket), `'AF_UNIX'` (for a Unix domain socket) or `'AF_PIPE'` (for a Windows named pipe). Of these only the first is guaranteed to be available. If *family* is `None` then the family is inferred from the format of *address*. If *address* is also `None` then a default is chosen. This default is the family which is assumed to

be the fastest available. See Formatos de Endereços. Note that if *family* is `'AF_UNIX'` and address is `None` then the socket will be created in a private temporary directory created using **`tempfile.mkstemp()`**.

If the listener object uses a socket then *backlog* (1 by default) is passed to the **`listen()`** method of the socket once it has been bound.

If *authkey* is given and not `None`, it should be a byte string and will be used as the secret key for an HMAC-based authentication challenge. No authentication is done if *authkey* is `None`. **`AuthenticationError`** is raised if authentication fails. See Authentication keys.

accept()

Accept a connection on the bound socket or named pipe of the listener object and return a **`Connection`** object. If authentication is attempted and fails, then **`AuthenticationError`** is raised.

close()

Close the bound socket or named pipe of the listener object. This is called automatically when the listener is garbage collected. However it is advisable to call it explicitly.

Listener objects have the following read-only properties:

address

The address which is being used by the Listener object.

last_accepted

The address from which the last accepted connection came. If this is unavailable then it is `None`.

Alterado na versão 3.3: Listener objects now support the context management protocol – see Tipos de Gerenciador de Contexto. **`__enter__()`** returns the listener object, and **`__exit__()`** calls **`close()`**.

multiprocessing.connection.wait(*object_list*, *timeout=None*)

Wait till an object in *object_list* is ready. Returns the list of those objects in *object_list* which are ready. If *timeout* is a float then the call blocks for at most that many seconds. If *timeout* is `None` then it will block for an unlimited period. A negative timeout is equivalent to a zero timeout.

For both POSIX and Windows, an object can appear in *object_list* if it is

- a readable **`Connection`** object;
- a connected and readable **`socket.socket`** object; or
- the **`sentinel`** attribute of a **`Process`** object.

A connection or socket object is ready when there is data available to be read from it, or the other end has been closed.

POSIX: `wait(object_list, timeout)` almost equivalent `select.select(object_list, [], [], timeout)`. The difference is that, if **`select.select()`** is interrupted by a signal, it can raise **`OSError`** with an error number of `EINTR`, whereas **`wait()`** will not.

Windows: An item in *object_list* must either be an integer handle which is waitable (according to the definition used by the documentation of the Win32 function `WaitForMultipleObjects()`) or it can be an object with a **`fileno()`** method which returns a socket handle or pipe handle. (Note that pipe handles and socket handles are **not** waitable handles.)

Adicionado na versão 3.3.

Examples

The following server code creates a listener which uses `'secret password'` as an authentication key. It then waits for a connection and sends some data to the client:

```
from multiprocessing.connection import Listener
from array import array

address = ('localhost', 6000)     # family is deduced to be 'AF_

with Listener(address, authkey=b'secret password') as listener:
    with listener.accept() as conn:
```

```
        print('connection accepted from', listener.last_acceptec

        conn.send([2.25, None, 'junk', float])

        conn.send_bytes(b'hello')

        conn.send_bytes(array('i', [42, 1729]))
```

The following code connects to the server and receives some data from the server:

```
from multiprocessing.connection import Client
from array import array

address = ('localhost', 6000)

with Client(address, authkey=b'secret password') as conn:
    print(conn.recv())                  # => [2.25, None, 'junk'

    print(conn.recv_bytes())            # => 'hello'

    arr = array('i', [0, 0, 0, 0, 0])
    print(conn.recv_bytes_into(arr))    # => 8
    print(arr)                          # => array('i', [42, 172
```

The following code uses `wait()` to wait for messages from multiple processes at once:

```
from multiprocessing import Process, Pipe, current_process
from multiprocessing.connection import wait

def foo(w):
    for i in range(10):
        w.send((i, current_process().name))
    w.close()

if __name__ == '__main__':
    readers = []

    for i in range(4):
        r, w = Pipe(duplex=False)
```

```
        readers.append(r)
        p = Process(target=foo, args=(w,))
        p.start()
        # We close the writable end of the pipe now to be sure t
        # p is the only process which owns a handle for it.  Thi
        # ensures that when p closes its handle for the writable
        # wait() will promptly report the readable end as being
        w.close()

    while readers:
        for r in wait(readers):
            try:
                msg = r.recv()
            except EOFError:
                readers.remove(r)
            else:
                print(msg)
```

Formatos de Endereços

- Um endereço `'AF_INET'` é uma tupla na forma de `(hostname, port)` sendo *hostname* uma string e *port* um inteiro.
- An `'AF_UNIX'` address is a string representing a filename on the filesystem.
- An `'AF_PIPE'` address is a string of the form `r'\\.\pipe\PipeName'`. To use `Client()` to connect to a named pipe on a remote computer called *ServerName* one should use an address of the form `r'\\ServerName\pipe\PipeName'` instead.

Note that any string beginning with two backslashes is assumed by default to be an `'AF_PIPE'` address rather than an `'AF_UNIX'` address.

Authentication keys

When one uses `Connection.recv`, the data received is automatically unpickled. Unfortunately unpickling data from an untrusted source is a security risk. Therefore `Listener` and `Client()` use the `hmac` module to provide digest authentication.

An authentication key is a byte string which can be thought of as a password: once a connection is established both ends will demand proof that the other knows the authentication key. (Demonstrating that both ends are using the same key does **not** involve sending the key over the connection.)

If authentication is requested but no authentication key is specified then the return value of `current_process().authkey` is used (see `Process`). This value will be automatically inherited by any `Process` object that the current process creates. This means that (by default) all processes of a multi-process program will share a single authentication key which can be used when setting up connections between themselves.

Suitable authentication keys can also be generated by using `os.urandom()`.

This authentication protects `Listener` and `Client()` connections, which are reachable by address. It is not applied to the anonymous pipes created by `Pipe()` or used internally by `Queue`. `multiprocessing` treats all local processes running as the same user as trusted; on most operating systems such processes can access each other's pipe file descriptors regardless. Applications that require isolation between processes of the same user must arrange it at the operating-system level – for example, by running workers under a different user account or in a sandbox.

Gerando logs

Some support for logging is available. Note, however, that the `logging` package does not use process shared locks so it is possible (depending on the handler type) for messages from different processes to get mixed up.

multiprocessing.get_logger()

Returns the logger used by `multiprocessing`. If necessary, a new one will be created.

When first created the logger has level `logging.NOTSET` and no default handler. Messages sent to this logger will not by default propagate to the root logger.

Note that on Windows child processes will only inherit the level of the parent process's logger – any other customization of the logger will not be inherited.

multiprocessing.log_to_stderr(*level=None*)

This function performs a call to **get_logger()** but in addition to returning the logger created by get_logger, it adds a handler which sends output to **sys.stderr** using format `'[%(levelname)s/%(processName)s] %(message)s'`. You can modify `levelname` of the logger by passing a `level` argument.

Below is an example session with logging turned on:

```
>>> import multiprocessing, logging
>>> logger = multiprocessing.log_to_stderr()
>>> logger.setLevel(logging.INFO)
>>> logger.warning('doomed')
[WARNING/MainProcess] doomed
>>> m = multiprocessing.Manager()
[INFO/SyncManager-...] child process calling self.run()
[INFO/SyncManager-...] created temp directory /.../pymp-...
[INFO/SyncManager-...] manager serving at '/.../listener-...'
>>> del m
[INFO/MainProcess] sending shutdown message to manager
[INFO/SyncManager-...] manager exiting with exitcode 0
```

For a full table of logging levels, see the **logging** module.

The `multiprocessing.dummy` module

`multiprocessing.dummy` replicates the API of **`multiprocessing`** but is no more than a wrapper around the **threading** module.

In particular, the `Pool` function provided by **`multiprocessing.dummy`** returns an instance of **ThreadPool**, which is a subclass of **Pool** that supports all the same method calls but uses a pool of worker threads rather than worker processes.

class multiprocessing.pool.ThreadPool([*processes*[, *initializer*[, *initargs*]]])

A thread pool object which controls a pool of worker threads to which jobs can be submitted. **`ThreadPool`** instances are fully interface compatible with **`Pool`** instances, and their resources must also be properly managed, either by using the pool as a context manager or by calling **`close()`** and **`terminate()`** manually.

processes is the number of worker threads to use. If *processes* is `None` then the number returned by **`os.process_cpu_count()`** is used.

If *initializer* is not `None` then each worker process will call `initializer(*initargs)` when it starts.

Unlike **`Pool`**, *maxtasksperchild* and *context* cannot be provided.

Nota

A **`ThreadPool`** shares the same interface as **`Pool`**, which is designed around a pool of processes and predates the introduction of the **`concurrent.futures`** module. As such, it inherits some operations that don't make sense for a pool backed by threads, and it has its own type for representing the status of asynchronous jobs, **`AsyncResult`**, that is not understood by any other libraries.

Users should generally prefer to use **`concurrent.futures.ThreadPoolExecutor`**, which has a simpler interface that was designed around threads from the start, and which returns **`concurrent.futures.Future`** instances that are compatible with many other libraries, including **`asyncio`**.

Programming guidelines

There are certain guidelines and idioms which should be adhered to when using **`multiprocessing`**.

All start methods

The following applies to all start methods.

Avoid shared state

As far as possible one should try to avoid shifting large amounts of data between processes.

It is probably best to stick to using queues or pipes for communication between processes rather than using the lower level synchronization primitives.

Picklability

Ensure that the arguments to the methods of proxies are picklable.

Thread safety of proxies

Do not use a proxy object from more than one thread unless you protect it with a lock.

(There is never a problem with different processes using the *same* proxy.)

Joining zombie processes

On POSIX when a process finishes but has not been joined it becomes a zombie. There should never be very many because each time a new process starts (or `active_children()` is called) all completed processes which have not yet been joined will be joined. Also calling a finished process's `Process.is_alive` will join the process. Even so it is probably good practice to explicitly join all the processes that you start.

Better to inherit than pickle/unpickle

When using the *spawn* or *forkserver* start methods many types from **multiprocessing** need to be picklable so that child processes can use them. However, one should generally avoid sending shared objects to other processes using pipes or queues. Instead you should arrange the program so that a process which needs access to a shared resource created elsewhere can inherit it from an ancestor process.

Avoid terminating processes

Using the **Process.terminate** method to stop a process is liable to cause any shared resources (such as locks, semaphores, pipes and queues) currently being used by the process to become broken or unavailable to other processes.

Therefore it is probably best to only consider using **Process.terminate** on processes which never use any shared resources.

Joining processes that use queues

Bear in mind that a process that has put items in a queue will wait before terminating until all the buffered items are fed by the “feeder” thread to the underlying pipe. (The child process can call the **Queue.cancel_join_thread** method of the queue to avoid this behaviour.)

This means that whenever you use a queue you need to make sure that all items which have been put on the queue will eventually be removed before the process is joined. Otherwise you cannot be sure that processes which have put items on the queue will terminate. Remember also that non-daemonic processes will be joined automatically.

An example which will deadlock is the following:

```
from multiprocessing import Process, Queue

def f(q):
    q.put('X' * 1000000)
```

```
if __name__ == '__main__':
    queue = Queue()
    p = Process(target=f, args=(queue,))
    p.start()
    p.join()                    # this deadlocks
    obj = queue.get()
```

A fix here would be to swap the last two lines (or simply remove the `p.join()` line).

Explicitly pass resources to child processes

On POSIX using the *fork* start method, a child process can make use of a shared resource created in a parent process using a global resource. However, it is better to pass the object as an argument to the constructor for the child process.

Apart from making the code (potentially) compatible with Windows and the other start methods this also ensures that as long as the child process is still alive the object will not be garbage collected in the parent process. This might be important if some resource is freed when the object is garbage collected in the parent process.

So for instance

```
from multiprocessing import Process, Lock

def f():
    ... do something using "lock" ...

if __name__ == '__main__':
    lock = Lock()
    for i in range(10):
        Process(target=f).start()
```

should be rewritten as

```
from multiprocessing import Process, Lock

def f(l):
```

```
    ... do something using "l" ...

if __name__ == '__main__':
    lock = Lock()
    for i in range(10):
        Process(target=f, args=(lock,)).start()
```

Beware of replacing **sys.stdin** with a “file like object”

multiprocessing originally unconditionally called:

```
os.close(sys.stdin.fileno())
```

in the **multiprocessing.Process._bootstrap()** method — this resulted in issues with processes-in-processes. This has been changed to:

```
sys.stdin.close()
sys.stdin = open(os.open(os.devnull, os.O_RDONLY), closefd=F
```

Which solves the fundamental issue of processes colliding with each other resulting in a bad file descriptor error, but introduces a potential danger to applications which replace **sys.stdin()** with a “file-like object” with output buffering. This danger is that if multiple processes call **close()** on this file-like object, it could result in the same data being flushed to the object multiple times, resulting in corruption.

If you write a file-like object and implement your own caching, you can make it fork-safe by storing the pid whenever you append to the cache, and discarding the cache when the pid changes. For example:

```
@property
def cache(self):
    pid = os.getpid()
    if pid != self._pid:
        self._pid = pid
        self._cache = []
    return self._cache
```

For more information, see bpo-5155 [https://bugs.python.org/issue?@action=redirect&bpo=5155], bpo-5313 [https://bugs.python.org/issue?@action=redirect&bpo=5313] and bpo-5331 [https://bugs.python.org/issue?@action=redirect&bpo=5331]

The *spawn* and *forkserver* start methods

There are a few extra restrictions which don't apply to the *fork* start method.

More picklability

Ensure that all arguments to **`Process`** are picklable. Also, if you subclass `Process.__init__`, you must make sure that instances will be picklable when the **`Process.start`** method is called.

Global variables

Bear in mind that if code run in a child process tries to access a global variable, then the value it sees (if any) may not be the same as the value in the parent process at the time that **`Process.start`** was called.

However, global variables which are just module level constants cause no problems.

Safe importing of main module

Make sure that the main module can be safely imported by a new Python interpreter without causing unintended side effects (such as starting a new process).

For example, using the *spawn* or *forkserver* start method running the following module would fail with a **`RuntimeError`**:

```
from multiprocessing import Process

def foo():
    print('hello')
```

```
p = Process(target=foo)
p.start()
```

Instead one should protect the “entry point” of the program by using `if __name__ == '__main__':` as follows:

```
from multiprocessing import Process, freeze_support, set_sta

def foo():
    print('hello')

if __name__ == '__main__':
    freeze_support()
    set_start_method('spawn')
    p = Process(target=foo)
    p.start()
```

(The `freeze_support()` line can be omitted if the program will be run normally instead of frozen.)

This allows the newly spawned Python interpreter to safely import the module and then run the module’s `foo()` function.

Similar restrictions apply if a pool or manager is created in the main module.

Exemplos

Demonstration of how to create and use customized managers and proxies:

```
from multiprocessing import freeze_support
from multiprocessing.managers import BaseManager, BaseProxy
import operator

##

class Foo:
    def f(self):
        print('you called Foo.f()')
    def g(self):
```

```
        print('you called Foo.g()')
    def _h(self):
        print('you called Foo._h()')

# A simple generator function
def baz():
    for i in range(10):
        yield i*i

# Proxy type for generator objects
class GeneratorProxy(BaseProxy):
    _exposed_ = ['__next__']
    def __iter__(self):
        return self
    def __next__(self):
        return self._callmethod('__next__')

# Function to return the operator module
def get_operator_module():
    return operator

##

class MyManager(BaseManager):
    pass

# register the Foo class; make `f()` and `g()` accessible via pr
MyManager.register('Foo1', Foo)

# register the Foo class; make `g()` and `_h()` accessible via p
MyManager.register('Foo2', Foo, exposed=('g', '_h'))

# register the generator function baz; use `GeneratorProxy` to m
MyManager.register('baz', baz, proxytype=GeneratorProxy)

# register get_operator_module(); make public functions accessib
MyManager.register('operator', get_operator_module)

##

def test():
    manager = MyManager()
    manager.start()
```

```
    print('-' * 20)

    f1 = manager.Foo1()
    f1.f()
    f1.g()
    assert not hasattr(f1, '_h')
    assert sorted(f1._exposed_) == sorted(['f', 'g'])

    print('-' * 20)

    f2 = manager.Foo2()
    f2.g()
    f2._h()
    assert not hasattr(f2, 'f')
    assert sorted(f2._exposed_) == sorted(['g', '_h'])

    print('-' * 20)

    it = manager.baz()
    for i in it:
        print('<%d>' % i, end=' ')
    print()

    print('-' * 20)

    op = manager.operator()
    print('op.add(23, 45) =', op.add(23, 45))
    print('op.pow(2, 94) =', op.pow(2, 94))
    print('op._exposed_ =', op._exposed_)

##

if __name__ == '__main__':
    freeze_support()
    test()
```

Using **Pool**:

```
import multiprocessing
import time
import random
```

```
import sys

#
# Functions used by test code
#

def calculate(func, args):
    result = func(*args)
    return '%s says that %s%s = %s' % (
        multiprocessing.current_process().name,
        func.__name__, args, result
        )

def calculatestar(args):
    return calculate(*args)

def mul(a, b):
    time.sleep(0.5 * random.random())
    return a * b

def plus(a, b):
    time.sleep(0.5 * random.random())
    return a + b

def f(x):
    return 1.0 / (x - 5.0)

def pow3(x):
    return x ** 3

def noop(x):
    pass

#
# Test code
#

def test():
    PROCESSES = 4
    print('Creating pool with %d processes\n' % PROCESSES)

    with multiprocessing.Pool(PROCESSES) as pool:
        #
```

```
# Tests
#

TASKS = [(mul, (i, 7)) for i in range(10)] + \
        [(plus, (i, 8)) for i in range(10)]

results = [pool.apply_async(calculate, t) for t in TASKS
imap_it = pool.imap(calculatestar, TASKS)
imap_unordered_it = pool.imap_unordered(calculatestar, T

print('Ordered results using pool.apply_async():')
for r in results:
    print('\t', r.get())
print()

print('Ordered results using pool.imap():')
for x in imap_it:
    print('\t', x)
print()

print('Unordered results using pool.imap_unordered():')
for x in imap_unordered_it:
    print('\t', x)
print()

print('Ordered results using pool.map() --- will block t
for x in pool.map(calculatestar, TASKS):
    print('\t', x)
print()

#
# Test error handling
#

print('Testing error handling:')

try:
    print(pool.apply(f, (5,)))
except ZeroDivisionError:
    print('\tGot ZeroDivisionError as expected from pool
else:
    raise AssertionError('expected ZeroDivisionError')
```

```
try:
    print(pool.map(f, list(range(10))))
except ZeroDivisionError:
    print('\tGot ZeroDivisionError as expected from pool
else:
    raise AssertionError('expected ZeroDivisionError')

try:
    print(list(pool.imap(f, list(range(10)))))
except ZeroDivisionError:
    print('\tGot ZeroDivisionError as expected from list
else:
    raise AssertionError('expected ZeroDivisionError')

it = pool.imap(f, list(range(10)))
for i in range(10):
    try:
        x = next(it)
    except ZeroDivisionError:
        if i == 5:
            pass
    except StopIteration:
        break
    else:
        if i == 5:
            raise AssertionError('expected ZeroDivisionE

assert i == 9
print('\tGot ZeroDivisionError as expected from IMapIter
print()

#
# Testing timeouts
#

print('Testing ApplyResult.get() with timeout:', end=' '
res = pool.apply_async(calculate, TASKS[0])
while 1:
    sys.stdout.flush()
    try:
        sys.stdout.write('\n\t%s' % res.get(0.02))
        break
    except multiprocessing.TimeoutError:
```

```
                sys.stdout.write('.')
        print()
        print()

        print('Testing IMapIterator.next() with timeout:', end='
        it = pool.imap(calculatestar, TASKS)
        while 1:
            sys.stdout.flush()
            try:
                sys.stdout.write('\n\t%s' % it.next(0.02))
            except StopIteration:
                break
            except multiprocessing.TimeoutError:
                sys.stdout.write('.')
        print()
        print()


if __name__ == '__main__':
    multiprocessing.freeze_support()
    test()
```

An example showing how to use queues to feed tasks to a collection of worker processes and collect the results:

```
import time
import random

from multiprocessing import Process, Queue, current_process, fre

#
# Function run by worker processes
#

def worker(input, output):
    for func, args in iter(input.get, 'STOP'):
        result = calculate(func, args)
        output.put(result)

#
# Function used to calculate result
#
```

```
def calculate(func, args):
    result = func(*args)
    return '%s says that %s%s = %s' % \
        (current_process().name, func.__name__, args, result)

#
# Functions referenced by tasks
#

def mul(a, b):
    time.sleep(0.5*random.random())
    return a * b

def plus(a, b):
    time.sleep(0.5*random.random())
    return a + b

#
#
#

def test():
    NUMBER_OF_PROCESSES = 4
    TASKS1 = [(mul, (i, 7)) for i in range(20)]
    TASKS2 = [(plus, (i, 8)) for i in range(10)]

    # Create queues
    task_queue = Queue()
    done_queue = Queue()

    # Submit tasks
    for task in TASKS1:
        task_queue.put(task)

    # Start worker processes
    for i in range(NUMBER_OF_PROCESSES):
        Process(target=worker, args=(task_queue, done_queue)).st

    # Get and print results
    print('Unordered results:')
    for i in range(len(TASKS1)):
        print('\t', done_queue.get())
```

```
    # Add more tasks using `put()`
    for task in TASKS2:
        task_queue.put(task)

    # Get and print some more results
    for i in range(len(TASKS2)):
        print('\t', done_queue.get())

    # Tell child processes to stop
    for i in range(NUMBER_OF_PROCESSES):
        task_queue.put('STOP')


if __name__ == '__main__':
    freeze_support()
    test()
```

`multiprocessing.shared_memory` — Memória compartilhada para acesso direto entre processos

Código-fonte: Lib/multiprocessing/shared_memory.py
[https://github.com/python/cpython/tree/3.15/Lib/multiprocessing/shared_memory.py]

Adicionado na versão 3.8.

Este módulo fornece uma classe, `SharedMemory`, para a alocação e gerenciamento da memória compartilhada a ser acessada por um ou mais processos em uma máquina multicore ou de multiprocessamento simétrico (SMP). Para ajudar com o gerenciamento do ciclo de vida da memória compartilhada especialmente entre processos distintos, uma subclasse de `BaseManager`, `SharedMemoryManager`, também é fornecida no módulo `multiprocessing.managers`.

Neste módulo, memória compartilhada refere-se a blocos de memória compartilhada no "estilo POSIX" (embora não seja necessariamente implementado explicitamente como tal) e não se refere a "memória compartilhada distribuída". Este estilo de memória compartilhada permite que processos distintos potencialmente leiam e escrevam em uma região comum (ou compartilhada) de memória volátil. Os processos são convencionalmente limitados a ter acesso somente ao próprio espaço de memória de processo mas a memória compartilhada permite o compartilhamento de dados entre processos, evitando a necessidade de enviar mensagens entre processos contendo estes dados. Compartilhar dados diretamente via memória pode fornecer ganhos de desempenho significativos comparado ao compartilhamento de dados via disco ou soquete ou outras comunicações que requerem a serialização/desserialização e cópia dos dados.

class multiprocessing.shared_memory.SharedMemory(*name=None*, *create=False*, *size=0*, *, *track=True*)

Cria uma instância da classe **`SharedMemory`** para criar um novo bloco de memória compartilhada ou anexar a um bloco de memória compartilhada existente. A cada bloco de memória compartilhada é atribuído um nome único. Desta forma, um processo pode criar um bloco de memória compartilhada com um nome particular e um processo diferente pode ser anexado a esse mesmo bloco de memória compartilhada usando este mesmo nome.

Como um recurso para compartilhar dados entre processos, os blocos de memória compartilhada podem sobreviver ao processo original que os criou. Quando um processo não precisa mais acessar um bloco de memória compartilhada que ainda pode ser necessário para outros processos, o método **`close()`** deve ser chamado. Quando um bloco de memória compartilhada não é mais necessário para nenhum processo, o método **`unlink()`** deve ser chamado para garantir a limpeza apropriada.

Parâmetros::

- **name** (*str* | *None*) – O nome único para a memória compartilhada requisitada, especificado como uma string. Ao criar um novo bloco de memória compartilhada, se `None` (o padrão) é fornecido para o nome, um novo nome será gerado.
- **create** (*bool*) – Controla quando um novo bloco de memória compartilhada é criado (`True`) ou um bloco de memória compartilhada existente é anexado (`False`).
- **size** (*int*) – O número de bytes requeridos ao criar um novo bloco de memória compartilhada. Como algumas plataformas optam por alocar pedaços de memória com base no tamanho da página de memória da própria plataforma, o tamanho exato do bloco de memória compartilhada pode ser maior ou igual ao tamanho requerido. Ao anexar a um bloco de memória compartilhada existente, o parâmetro *size* é ignorado.
- **track** (*bool*) – Quando `True`, registra o bloco de memória compartilhada com um processo de

rastreador de recursos em plataformas onde o sistema operacional não faz isso automaticamente O rastreador de recursos garante a limpeza adequada da memória compartilhada, mesmo se todos os outros processos com acesso à memória saírem sem fazer isso. Os processos Python criado a partir de um ancestral comum usando os recurso de `multiprocessing` compartilham um único processo de rastreador de recursos, e o tempo de vida dos segmentos de memória compartilhada é manipulado automaticamente entre esses processos. Os processos Python criados de qualquer outra forma receberão seu próprio rastreador de recursos ao acessar a memória compartilhada com *track* habilitado. Isso fará com que a memória compartilhada seja excluída pelo rastreador de recursos do primeiro processo que terminar. Para evitar esse problema, os usuários de `subprocess` ou processos Python autônomos devem definir *track* como `False` quando já houver outro processo em vigor que faça a contabilidade. *track* é ignorado no Windows, que tem seu próprio rastreamento e exclui automaticamente a memória compartilhada quando todos os identificadores par ele foram fechados.

Alterado na versão 3.13: Adicionado o parâmetro *track*.

close()

Fecha o identificador/descritor de arquivo para a memória compartilhada desta instância. `close()` deve ser chamado quando o acesso ao bloco de memória compartilhada desta instância não for mais necessário. Dependendo do sistema operacional, a memória subjacente pode ou não ser liberada, mesmo que todos os identificadores para ela tenham sido fechados. Para garantir uma limpeza adequada, use o método `unlink()`.

unlink()

Exclui o bloco de memória compartilhada subjacente. Isso deve ser chamado apenas uma vez por bloco de memória compartilhada, independentemente do número de handles para ele, mesmo em outros processos. **unlink()** e **close()** podem ser chamados em qualquer ordem, mas tentar acessar dados dentro de um bloco de memória compartilhada após **unlink()** pode resultar em erros de acesso à memória, dependendo da plataforma.

Este método não tem efeito no Windows, onde a única maneira de excluir um bloco de memória compartilhada é fechar todos os identificadores.

buf

Uma visualização de memória do conteúdo do bloco de memória compartilhada.

name

Acesso somente leitura ao nome único do bloco de memória compartilhada.

size

Acesso somente leitura ao tamanho em bytes do bloco de memória compartilhada.

O exemplo a seguir demonstra um uso baixo nível de instâncias de **SharedMemory**:

```
>>> from multiprocessing import shared_memory
>>> shm_a = shared_memory.SharedMemory(create=True, size=10)
>>> type(shm_a.buf)
<class 'memoryview'>
>>> buffer = shm_a.buf
>>> len(buffer)
10
>>> buffer[:4] = bytearray([22, 33, 44, 55])  # Modifica vários d
>>> buffer[4] = 100                           # Modifica um único
>>> # Anexa a um bloco de memória compartilhada existente
>>> shm_b = shared_memory.SharedMemory(shm_a.name)
>>> import array
>>> array.array('b', shm_b.buf[:5])  # Copia os dados para um nov
```

```
array('b', [22, 33, 44, 55, 100])
>>> shm_b.buf[:5] = b'howdy'  # Modifica via shm_b usando bytes
>>> bytes(shm_a.buf[:5])      # Acessa via shm_a
b'howdy'
>>> shm_b.close()   # Fecha cada instância de SharedMemory
>>> shm_a.close()
>>> shm_a.unlink()  # Chama unlink uma vez para liberar a memória
```

O exemplo a seguir demonstra um uso prático da classe **`SharedMemory`** com arrays do NumPy [https://numpy.org/], acessando o mesmo **`numpy.ndarray`** de dois consoles Python distintos.

```
>>> # No primeiro console interativo do Python
>>> import numpy as np
>>> a = np.array([1, 1, 2, 3, 5, 8])  # Inicia com um array de Nu
>>> from multiprocessing import shared_memory
>>> shm = shared_memory.SharedMemory(create=True, size=a.nbytes)
>>> # Agora cria um array de NumPy suportado por memória comparti
>>> b = np.ndarray(a.shape, dtype=a.dtype, buffer=shm.buf)
>>> b[:] = a[:]  # Copia os dados originais para memória comparti
>>> b
array([1, 1, 2, 3, 5, 8])
>>> type(b)
<class 'numpy.ndarray'>
>>> type(a)
<class 'numpy.ndarray'>
>>> shm.name  # Não especificamos um nome, então um foi escolhido
'psm_21467_46075'

>>> # No mesmo shell ou em um novo console do Python na mesma máq
>>> import numpy as np
>>> from multiprocessing import shared_memory
>>> # Anexa ao bloco de memória compartilhada existente
>>> existing_shm = shared_memory.SharedMemory(name='psm_21467_460
>>> # Observe que a.shape é (6,) e a.dtype é np.int64 neste exemp
>>> c = np.ndarray((6,), dtype=np.int64, buffer=existing_shm.buf)
>>> c
array([1, 1, 2, 3, 5, 8])
>>> c[-1] = 888
>>> c
array([  1,   1,   2,   3,   5, 888])
```

```
>>> # De volta ao primeiro console interativo do Python, b reflet
>>> b
array([  1,   1,   2,   3,   5, 888])

>>> # Limpeza de dentro do segundo console Python
>>> del c  # Desnecessário; apenas enfatizar a array não é mais u
>>> existing_shm.close()

>>> # Limpeza de dentro do primeiro console Python
>>> del b  # Desnecessário; apenas enfatizar a array não é mais u
>>> shm.close()
>>> shm.unlink()  # Libera o bloco de memória compartilhada no fi
```

class multiprocessing.managers.SharedMemoryManager([*address*[, *authkey*]])

Uma subclasse de **multiprocessing.managers.BaseManager** que pode ser usada para o gerenciamento dos blocos de memória compartilhada entre processos.

Uma chamada ao método **start()** em uma instância de **SharedMemoryManager** faz com que um novo processo seja iniciado. A única finalidade desse novo processo é gerenciar o ciclo de vida de todos os blocos de memória criados através dele. Para acionar a liberação de todos os blocos de memória gerenciados por este processo, invoque o método **shutdown()** na instância. Isso aciona uma chamada de **unlink()** em todos os objetos **SharedMemory** gerenciados por esse processo e então para o processo em si. Ao criar instâncias de **SharedMemory** através de um **SharedMemoryManager**, evitamos a necessidade de rastrear e acionar manualmente a liberação dos recursos de memória compartilhada.

Esta classe fornece métodos para criar e retornar instâncias de **SharedMemory** e para criar um objeto lista ou similar (**ShareableList**) apoiado por memória compartilhada.

Consulte **BaseManager** para obter uma descrição dos argumentos de entrada opcionais herdados *address* e *authkey* e como eles podem ser usados para conectar-se a um serviço **SharedMemoryManager** existente de outros processos.

SharedMemory(*size*)

Cria e retorna um novo objeto **SharedMemory** com o *size* especificado em bytes.

ShareableList(*sequence*)

Cria e retorna um novo objeto **ShareableList**, inicializado pelos valores da entrada *sequence*.

O exemplo a seguir demonstra os mecanismos básicos de um **SharedMemoryManager**:

```
>>> from multiprocessing.managers import SharedMemoryManager
>>> smm = SharedMemoryManager()
>>> smm.start()  # Inicia o procsso que gerencia os blocos de mem
>>> sl = smm.ShareableList(range(4))
>>> sl
ShareableList([0, 1, 2, 3], name='psm_6572_7512')
>>> raw_shm = smm.SharedMemory(size=128)
>>> another_sl = smm.ShareableList('alpha')
>>> another_sl
ShareableList(['a', 'l', 'p', 'h', 'a'], name='psm_6572_12221')
>>> smm.shutdown()  # Chama unlink() em sl, raw_shm, e another_sl
```

O exemplo a seguir retrata um padrão potencialmente mais conveniente para usar objetos **SharedMemoryManager** através da instrução **with** para garantir que todos os blocos de memória compartilhada são liberados depois que não são mais necessários.

```
>>> with SharedMemoryManager() as smm:
...     sl = smm.ShareableList(range(2000))
...     # Divide o trabalho entre dois processos, armazenando res
...     p1 = Process(target=do_work, args=(sl, 0, 1000))
...     p2 = Process(target=do_work, args=(sl, 1000, 2000))
...     p1.start()
...     p2.start()  # Um multiprocessing.Pool pode ser mais efici
...     p1.join()
...     p2.join()   # Espera todo trabalho ser concluído nos dois
...     total_result = sum(sl)  # Consolida os resutlados parciai
```

Ao usar um **SharedMemoryManager** em uma instrução **with**, os blocos de memória compartilhada criados utilizando este gerenciador são todos liberados quando o bloco de código com a instrução **with** termina sua execução.

class multiprocessing.shared_memory.ShareableList(*sequence=None*, *, *name=None*)

Fornece um objeto mutável do tipo lista onde todos os valores armazenados dentro são armazenados em um bloco de memória compartilhada. Isso restringe valores armazenáveis aos seguintes tipos de dados embutidos:

- **int** (com sinal, 64-bit)
- **float**
- **bool**
- **str** (menos de 10M bytes cada quando codificado como UTF-8)
- **bytes** (menos de 10M bytes cada)
- `None`

Ele também difere notavelmente do tipo embutido **list**, pois essas listas não podem alterar seu comprimento geral (ou seja, sem **append()**, **insert()**, etc.) e não oferecem suporte à criação dinâmica de novas instâncias **ShareableList** por meio de fatiamento.

sequence é usado para preencher um **ShareableList** com valores. Defina como `None` para anexar a uma **ShareableList** já existente pelo seu nome único de memória compartilhada.

name é um nome único para a memória compartilhada requerida, como descrito na definição de **SharedMemory**. Ao anexar a uma **ShareableList** já existente, deve-se especificar o nome único do bloco de memória compartilhada e definir *sequence* como `None`.

Nota

Existe um problema conhecido para os valores **bytes** e **str**. Se eles terminarem com bytes ou caracteres nulos `\x00`, eles podem ser *silenciosamente removidos* ao buscá-los pelo índice da **ShareableList**. Esse comportamento `.rstrip(b'\x00')` é considerado um bug e pode

desaparecer no futuro. Veja gh-106939 [https://github.com/python/cpython/issues/106939].

Para aplicações onde a remoção de nulos finais é um problema, contorne-o sempre anexando incondicionalmente um byte extra diferente de 0 ao final de tais valores ao armazená-los e removendo-os incondicionalmente ao buscá-los:

```
>>> from multiprocessing import shared_memory
>>> nul_bug_demo = shared_memory.ShareableList(['?\x00', b'\x0
>>> nul_bug_demo[0]
'?'
>>> nul_bug_demo[1]
b'\x03\x02\x01'
>>> nul_bug_demo.shm.unlink()
>>> padded = shared_memory.ShareableList(['?\x00\x07', b'\x03\
>>> padded[0][:-1]
'?\x00'
>>> padded[1][:-1]
b'\x03\x02\x01\x00\x00\x00'
>>> padded.shm.unlink()
```

count(*value*)

Retorna o número de ocorrências de *value*.

index(*value*)

Retorna a primeira posição do índice de *value*. Levanta `ValueError` se *value* não estiver presente.

format

Atributo somente leitura contendo o formato de empacotamento `struct` usado por todos os valores armazenados atualmente.

shm

A instância de `SharedMemory` onde os valores são armazenados.

O exemplo a seguir demonstra o uso básico de uma instância de `ShareableList`:

```
>>> from multiprocessing import shared_memory
>>> a = shared_memory.ShareableList(['howdy', b'HoWdY', -273.154,
>>> [ type(entry) for entry in a ]
[<class 'str'>, <class 'bytes'>, <class 'float'>, <class 'int'>,
>>> a[2]
-273.154
>>> a[2] = -78.5
>>> a[2]
-78.5
>>> a[2] = 'dry ice'  # Changing data types is supported as well
>>> a[2]
'dry ice'
>>> a[2] = 'larger than previously allocated storage space'
Traceback (most recent call last):
  ...
ValueError: exceeds available storage for existing str
>>> a[2]
'dry ice'
>>> len(a)
7
>>> a.index(42)
6
>>> a.count(b'howdy')
0
>>> a.count(b'HoWdY')
1
>>> a.shm.close()
>>> a.shm.unlink()
>>> del a  # Use of a ShareableList after call to unlink() is uns
```

O exemplo a seguir retrata como um, dois ou mais processos podem acessar a mesma **`ShareableList`** fornecendo o nome do bloco de memória compartilhada por trás dela:

```
>>> b = shared_memory.ShareableList(range(5))         # In a firs
>>> c = shared_memory.ShareableList(name=b.shm.name)  # In a seco
>>> c
ShareableList([0, 1, 2, 3, 4], name='...')
>>> c[-1] = -999
>>> b[-1]
-999
>>> b.shm.close()
```

```
>>> c.shm.close()
>>> c.shm.unlink()
```

Os exemplos a seguir demonstram que os objetos **ShareableList** (e o subjacente **SharedMemory**) podem ser serializados e desserializados com pickle, se preciso. Note, ainda será o mesmo objeto compartilhado. Isto acontece pois o objeto desserializado tem o mesmo nome único e é somente anexado ao objeto existente com o mesmo nome (se o objeto ainda existe):

```
>>> import pickle
>>> from multiprocessing import shared_memory
>>> sl = shared_memory.ShareableList(range(10))
>>> list(sl)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> deserialized_sl = pickle.loads(pickle.dumps(sl))
>>> list(deserialized_sl)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

>>> sl[0] = -1
>>> deserialized_sl[1] = -2
>>> list(sl)
[-1, -2, 2, 3, 4, 5, 6, 7, 8, 9]
>>> list(deserialized_sl)
[-1, -2, 2, 3, 4, 5, 6, 7, 8, 9]

>>> sl.shm.close()
>>> sl.shm.unlink()
```

O pacote `concurrent`

Este pacote contém os seguintes módulos:

- `concurrent.futures` – Iniciando tarefas em paralelo
- `concurrent.interpreters` — Múltiplos interpretadores no mesmo processo

`concurrent.futures` — Launching parallel tasks

Adicionado na versão 3.2.

Source code: Lib/concurrent/futures/thread.py [https://github.com/python/cpython/tree/3.15/Lib/concurrent/futures/thread.py], Lib/concurrent/futures/process.py [https://github.com/python/cpython/tree/3.15/Lib/concurrent/futures/process.py], and Lib/concurrent/futures/interpreter.py [https://github.com/python/cpython/tree/3.15/Lib/concurrent/futures/interpreter.py]

The **`concurrent.futures`** module provides a high-level interface for asynchronously executing callables.

The asynchronous execution can be performed with threads, using **`ThreadPoolExecutor`** or **`InterpreterPoolExecutor`**, or separate processes, using **`ProcessPoolExecutor`**. Each implements the same interface, which is defined by the abstract **`Executor`** class.

`concurrent.futures.Future` must not be confused with **`asyncio.Future`**, which is designed for use with **`asyncio`** tasks and coroutines. See the asyncio's Future documentation for a detailed comparison of the two.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Executor Objects

class concurrent.futures.Executor

An abstract class that provides methods to execute calls asynchronously. It should not be used directly, but through its concrete subclasses.

submit(*fn*, /, **args*, ***kwargs*)

Schedules the callable, *fn*, to be executed as `fn(*args, **kwargs)` and returns a **`Future`** object representing the execution of the callable.

```
with ThreadPoolExecutor(max_workers=1) as executor:
    future = executor.submit(pow, 323, 1235)
    print(future.result())
```

map(*fn*, **iterables*, *timeout=None*, *chunksize=1*, *buffersize=None*)

Similar to **`map(fn, *iterables)`** except:

- The *iterables* are collected immediately rather than lazily, unless a *buffersize* is specified to limit the number of submitted tasks whose results have not yet been yielded. If the buffer is full, iteration over the *iterables* pauses until a result is yielded from the buffer.
- *fn* is executed asynchronously and several calls to *fn* may be made concurrently.

The returned iterator raises a **`TimeoutError`** if **`__next__()`** is called and the result isn't available after *timeout* seconds from the original call to **`Executor.map()`**. *timeout* can be an int or a float. If *timeout* is not specified or `None`, there is no limit to the wait time.

If a *fn* call raises an exception, then that exception will be raised when its value is retrieved from the iterator.

When using **`ProcessPoolExecutor`**, this method chops *iterables* into a number of chunks which it submits to the pool as separate tasks. The (approximate) size of these chunks can be specified by setting *chunksize* to a positive integer. For very long iterables, using a large value for *chunksize* can significantly improve performance compared to the default size of 1. With **`ThreadPoolExecutor`** and **`InterpreterPoolExecutor`**, *chunksize* has no effect.

Alterado na versão 3.5: Added the *chunksize* parameter.

Alterado na versão 3.14: Added the *buffersize* parameter.

shutdown(*wait=True*, *, *cancel_futures=False*)

Signal the executor that it should free any resources that it is using when the currently pending futures are done executing. Calls to **`Executor.submit()`** and **`Executor.map()`** made after shutdown will raise **`RuntimeError`**.

If *wait* is `True` then this method will not return until all the pending futures are done executing and the resources associated with the executor have been freed. If *wait* is `False` then this method will return immediately and the resources associated with the executor will be freed when all pending futures are done executing. Regardless of the value of *wait*, the entire Python program will not exit until all pending futures are done executing.

If *cancel_futures* is `True`, this method will cancel all pending futures that the executor has not started running. Any futures that are completed or running won't be cancelled, regardless of the value of *cancel_futures*.

If both *cancel_futures* and *wait* are `True`, all futures that the executor has started running will be completed prior to this method returning. The remaining futures are cancelled.

You can avoid having to call this method explicitly if you use the executor as a context manager via the **`with`** statement, which will shutdown the **`Executor`** (waiting as if **`Executor.shutdown()`** were called with *wait* set to `True`):

```
import shutil
with ThreadPoolExecutor(max_workers=4) as e:
    e.submit(shutil.copy, 'src1.txt', 'dest1.txt')
    e.submit(shutil.copy, 'src2.txt', 'dest2.txt')
    e.submit(shutil.copy, 'src3.txt', 'dest3.txt')
    e.submit(shutil.copy, 'src4.txt', 'dest4.txt')
```

Alterado na versão 3.9: Adicionado *cancel_futures*.

ThreadPoolExecutor

ThreadPoolExecutor is an **Executor** subclass that uses a pool of threads to execute calls asynchronously.

Deadlocks can occur when the callable associated with a **Future** waits on the results of another **Future**. For example:

```
import time
def wait_on_b():
    time.sleep(5)
    print(b.result())  # b will never complete because it is wai
    return 5

def wait_on_a():
    time.sleep(5)
    print(a.result())  # a will never complete because it is wai
    return 6


executor = ThreadPoolExecutor(max_workers=2)
a = executor.submit(wait_on_b)
b = executor.submit(wait_on_a)
```

And:

```
def wait_on_future():
    f = executor.submit(pow, 5, 2)
    # This will never complete because there is only one worker
    # it is executing this function.
    print(f.result())

executor = ThreadPoolExecutor(max_workers=1)
future = executor.submit(wait_on_future)
# Note: calling future.result() would also cause a deadlock beca
# the single worker thread is already waiting for wait_on_future
```

class concurrent.futures.ThreadPoolExecutor(*max_workers=None*, *thread_name_prefix=''*, *initializer=None*, *initargs=()*)

An **`Executor`** subclass that uses a pool of at most *max_workers* threads to execute calls asynchronously.

All threads enqueued to `ThreadPoolExecutor` will be joined before the interpreter can exit. Note that the exit handler which does this is executed *before* any exit handlers added using `atexit`. This means exceptions in the main thread must be caught and handled in order to signal threads to exit gracefully. For this reason, it is recommended that `ThreadPoolExecutor` not be used for long-running tasks.

initializer is an optional callable that is called at the start of each worker thread; *initargs* is a tuple of arguments passed to the initializer. Should *initializer* raise an exception, all currently pending jobs will raise a **`BrokenThreadPool`**, as well as any attempt to submit more jobs to the pool.

Alterado na versão 3.5: If *max_workers* is `None` or not given, it will default to the number of processors on the machine, multiplied by `5`, assuming that **`ThreadPoolExecutor`** is often used to overlap I/O instead of CPU work and the number of workers should be higher than the number of workers for **`ProcessPoolExecutor`**.

Alterado na versão 3.6: Added the *thread_name_prefix* parameter to allow users to control the **`threading.Thread`** names for worker threads created by the pool for easier debugging.

Alterado na versão 3.7: Added the *initializer* and *initargs* arguments.

Alterado na versão 3.8: Default value of *max_workers* is changed to `min(32, os.cpu_count() + 4)`. This default value preserves at least 5 workers for I/O bound tasks. It utilizes at most 32 CPU cores for CPU bound tasks which release the GIL. And it avoids using very large resources implicitly on many-core machines.

ThreadPoolExecutor now reuses idle worker threads before starting *max_workers* worker threads too.

Alterado na versão 3.13: Default value of *max_workers* is changed to `min(32, (os.process_cpu_count() or 1) + 4)`.

Exemplo de ThreadPoolExecutor

```
import concurrent.futures
import urllib.request

URLS = ['http://www.foxnews.com/',
        'http://www.cnn.com/',
        'http://europe.wsj.com/',
        'http://www.bbc.co.uk/',
        'http://nonexistent-subdomain.python.org/']

# Retrieve a single page and report the URL and contents
def load_url(url, timeout):
    with urllib.request.urlopen(url, timeout=timeout) as conn:
        return conn.read()

# We can use a with statement to ensure threads are cleaned up p
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as exe
    # Start the load operations and mark each future with its UR
    future_to_url = {executor.submit(load_url, url, 60): url for
    for future in concurrent.futures.as_completed(future_to_url)
        url = future_to_url[future]
        try:
            data = future.result()
        except Exception as exc:
            print('%r generated an exception: %s' % (url, exc))
        else:
            print('%r page is %d bytes' % (url, len(data)))
```

InterpreterPoolExecutor

Adicionado na versão 3.14.

The **`InterpreterPoolExecutor`** class uses a pool of interpreters to execute calls asynchronously. It is a **`ThreadPoolExecutor`** subclass, which means

each worker is running in its own thread. The difference here is that each worker has its own interpreter, and runs each task using that interpreter.

The biggest benefit to using interpreters instead of only threads is true multi-core parallelism. Each interpreter has its own Global Interpreter Lock, so code running in one interpreter can run on one CPU core, while code in another interpreter runs unblocked on a different core.

The tradeoff is that writing concurrent code for use with multiple interpreters can take extra effort. However, this is because it forces you to be deliberate about how and when interpreters interact, and to be explicit about what data is shared between interpreters. This results in several benefits that help balance the extra effort, including true multi-core parallelism, For example, code written this way can make it easier to reason about concurrency. Another major benefit is that you don't have to deal with several of the big pain points of using threads, like race conditions.

Each worker's interpreter is isolated from all the other interpreters. "Isolated" means each interpreter has its own runtime state and operates completely independently. For example, if you redirect `sys.stdout` in one interpreter, it will not be automatically redirected to any other interpreter. If you import a module in one interpreter, it is not automatically imported in any other. You would need to import the module separately in interpreter where you need it. In fact, each module imported in an interpreter is a completely separate object from the same module in a different interpreter, including `sys`, `builtins`, and even `__main__`.

Isolation means a mutable object, or other data, cannot be used by more than one interpreter at the same time. That effectively means interpreters cannot actually share such objects or data. Instead, each interpreter must have its own copy, and you will have to synchronize any changes between the copies manually. Immutable objects and data, like the builtin singletons, strings, and tuples of immutable objects, don't have these limitations.

Communicating and synchronizing between interpreters is most effectively done using dedicated tools, like those proposed in **PEP 734** [https://peps.python.org/pep-0734/]. One less efficient alternative is to serialize with `pickle` and then send the bytes over a shared `socket` or `pipe`.

class concurrent.futures.InterpreterPoolExecutor(*max_workers=None*, *thread_name_prefix=''*, *initializer=None*, *initargs=()*)

Uma subclasse de **ThreadPoolExecutor** que executa chamadas de forma assíncrona usando um pool de no máximo *max_workers* threads. Cada thread executa tarefas em seu próprio interpretador. Os interpretadores dos trabalhadores estão isolados uns dos outros, o que significa que cada um tem o seu próprio estado de execução e não pode partilhar quaisquer objetos mutáveis nem outros dados. Cada interpretador tem a sua própria trava global, o que significa que o código executado por este executor tem paralelismo multinúcleo verdadeiro.

Os argumentos opcionais *initializer* e *initargs* têm o mesmo significado que para **ThreadPoolExecutor**: o inicializador é executado quando cada trabalhador é criado, embora neste caso seja executado no interpretador do próprio trabalhador. O executor serializa *initializer* e *initargs* com **pickle** ao enviá-los ao interpretador do trabalhador.

Nota

O executor pode substituir exceções não capturadas de *initializer* por **ExecutionFailed**.

Outras advertências da classe pai **ThreadPoolExecutor** aplicam-se aqui.

submit() e **map()** funcionam normalmente, exceto que o trabalhador serializa o objeto chamável e os argumentos usando **pickle** ao enviá-los para o seu interpretador. O trabalhador também serializa o valor de retorno ao enviá-lo de volta.

Quando a tarefa atual de um trabalhador levanta uma exceção não capturada, ele tenta sempre preservar a exceção como está. Se isso for bem-sucedido, também define o `__cause__` para uma instância correspondente de **ExecutionFailed**, que contém um resumo da exceção original. No caso incomum em que o trabalhador não consegue preservar a instância original, preserva diretamente a instância correspondente de **ExecutionFailed**.

ProcessPoolExecutor

The **`ProcessPoolExecutor`** class is an **`Executor`** subclass that uses a pool of processes to execute calls asynchronously. **`ProcessPoolExecutor`** uses the **`multiprocessing`** module, which allows it to side-step the Global Interpreter Lock but also means that only picklable objects can be executed and returned.

The `__main__` module must be importable by worker subprocesses. This means that **`ProcessPoolExecutor`** will not work in the interactive interpreter.

Calling **`Executor`** or **`Future`** methods from a callable submitted to a **`ProcessPoolExecutor`** will result in deadlock.

Note that the restrictions on functions and arguments needing to picklable as per **`multiprocessing.Process`** apply when using **`submit()`** and **`map()`** on a **`ProcessPoolExecutor`**. A function defined in a REPL or a lambda should not be expected to work.

class concurrent.futures.ProcessPoolExecutor(*max_workers=None*, *mp_context=None*, *initializer=None*, *initargs=()*, *max_tasks_per_child=None*)

An **`Executor`** subclass that executes calls asynchronously using a pool of at most *max_workers* processes. If *max_workers* is `None` or not given, it will default to **`os.process_cpu_count()`**. If *max_workers* is less than or equal to `0`, then a **`ValueError`** will be raised. On Windows, *max_workers* must be less than or equal to `61`. If it is not then **`ValueError`** will be raised. If *max_workers* is `None`, then the default chosen will be at most `61`, even if more processors are available. *mp_context* can be a **`multiprocessing`** context or `None`. It will be used to launch the workers. If *mp_context* is `None` or not given, the default **`multiprocessing`** context is used. See Contextos e métodos de inicialização.

initializer is an optional callable that is called at the start of each worker process; *initargs* is a tuple of arguments passed to the initializer. Should

initializer raise an exception, all currently pending jobs will raise a `BrokenProcessPool`, as well as any attempt to submit more jobs to the pool.

max_tasks_per_child is an optional argument that specifies the maximum number of tasks a single process can execute before it will exit and be replaced with a fresh worker process. By default *max_tasks_per_child* is `None` which means worker processes will live as long as the pool. When a max is specified, the “spawn” multiprocessing start method will be used by default in absence of a *mp_context* parameter. This feature is incompatible with the “fork” start method.

Alterado na versão 3.3: When one of the worker processes terminates abruptly, a `BrokenProcessPool` error is now raised. Previously, behaviour was undefined but operations on the executor or its futures would often freeze or deadlock.

Alterado na versão 3.7: The *mp_context* argument was added to allow users to control the start_method for worker processes created by the pool.

Added the *initializer* and *initargs* arguments.

Alterado na versão 3.11: The *max_tasks_per_child* argument was added to allow users to control the lifetime of workers in the pool.

Alterado na versão 3.12: On POSIX systems, if your application has multiple threads and the `multiprocessing` context uses the `"fork"` start method: The `os.fork()` function called internally to spawn workers may raise a `DeprecationWarning`. Pass a *mp_context* configured to use a different start method. See the `os.fork()` documentation for further explanation.

Alterado na versão 3.13: max_workers uses `os.process_cpu_count()` by default, instead of `os.cpu_count()`.

Alterado na versão 3.14: The default process start method (see Contextos e métodos de inicialização) changed away from *fork*. If you

require the *fork* start method for **`ProcessPoolExecutor`** you must explicitly pass `mp_context=multiprocessing.get_context("fork")`.

Alterado na versão 3.15: Fixed a deadlock (gh-115634 [https://github.com/python/cpython/issues/115634]) where the executor could hang after a worker process exited upon reaching its *max_tasks_per_child* limit while tasks remained queued.

terminate_workers()

Attempt to terminate all living worker processes immediately by calling **`Process.terminate`** on each of them. Internally, it will also call **`Executor.shutdown()`** to ensure that all other resources associated with the executor are freed.

After calling this method the caller should no longer submit tasks to the executor.

Adicionado na versão 3.14.

kill_workers()

Attempt to kill all living worker processes immediately by calling **`Process.kill`** on each of them. Internally, it will also call **`Executor.shutdown()`** to ensure that all other resources associated with the executor are freed.

After calling this method the caller should no longer submit tasks to the executor.

Adicionado na versão 3.14.

ProcessPoolExecutor Example

```
import concurrent.futures
import math

PRIMES = [
    112272535095293,
    112582705942171,
```

```
    112272535095293,
    115280095190773,
    115797848077099,
    1099726899285419]

def is_prime(n):
    if n < 2:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    sqrt_n = int(math.floor(math.sqrt(n)))
    for i in range(3, sqrt_n + 1, 2):
        if n % i == 0:
            return False
    return True

def main():
    with concurrent.futures.ProcessPoolExecutor() as executor:
        for number, prime in zip(PRIMES, executor.map(is_prime,
            print('%d is prime: %s' % (number, prime))

if __name__ == '__main__':
    main()
```

Future Objects

The **`Future`** class encapsulates the asynchronous execution of a callable. **`Future`** instances are created by **`Executor.submit()`**.

class concurrent.futures.Future

Encapsulates the asynchronous execution of a callable. **`Future`** instances are created by **`Executor.submit()`** and should not be created directly except for testing.

cancel()

Attempt to cancel the call. If the call is currently being executed or finished running and cannot be cancelled then the method will return `False`, otherwise the call will be cancelled and the method will return `True`.

cancelled()

Return `True` if the call was successfully cancelled.

running()

Return `True` if the call is currently being executed and cannot be cancelled.

done()

Return `True` if the call was successfully cancelled or finished running.

result(*timeout=None*)

Return the value returned by the call. If the call hasn't yet completed then this method will wait up to *timeout* seconds. If the call hasn't completed in *timeout* seconds, then a **`TimeoutError`** will be raised. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

If the future is cancelled before completing then **`CancelledError`** will be raised.

If the call raised an exception, this method will raise the same exception.

exception(*timeout=None*)

Return the exception raised by the call. If the call hasn't yet completed then this method will wait up to *timeout* seconds. If the call hasn't completed in *timeout* seconds, then a **`TimeoutError`** will be raised. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

If the future is cancelled before completing then **`CancelledError`** will be raised.

If the call completed without raising, `None` is returned.

add_done_callback(*fn*)

Attaches the callable *fn* to the future. *fn* will be called, with the future as its only argument, when the future is cancelled or finishes running.

Added callables are called in the order that they were added and are always called in a thread belonging to the process that added them. If the callable raises an **`Exception`** subclass, it will be logged and ignored. If the callable raises a **`BaseException`** subclass, the behavior is undefined.

If the future has already completed or been cancelled, *fn* will be called immediately.

The following **`Future`** methods are meant for use in unit tests and **`Executor`** implementations.

set_running_or_notify_cancel()

This method should only be called by **`Executor`** implementations before executing the work associated with the **`Future`** and by unit tests.

If the method returns `False` then the **`Future`** was cancelled, i.e. **`Future.cancel()`** was called and returned `True`. Any threads waiting on the **`Future`** completing (i.e. through **`as_completed()`** or **`wait()`**) will be woken up.

If the method returns `True` then the **`Future`** was not cancelled and has been put in the running state, i.e. calls to **`Future.running()`** will return `True`.

This method can only be called once and cannot be called after **`Future.set_result()`** or **`Future.set_exception()`** have been

called.

set_result(*result*)

Sets the result of the work associated with the **`Future`** to *result*.

This method should only be used by **`Executor`** implementations and unit tests.

Alterado na versão 3.8: This method raises **`concurrent.futures.InvalidStateError`** if the **`Future`** is already done.

set_exception(*exception*)

Sets the result of the work associated with the **`Future`** to the **`Exception`** *exception*.

This method should only be used by **`Executor`** implementations and unit tests.

Alterado na versão 3.8: This method raises **`concurrent.futures.InvalidStateError`** if the **`Future`** is already done.

Module Functions

concurrent.futures.wait(*fs*, *timeout=None*, *return_when=ALL_COMPLETED*)

Wait for the **`Future`** instances (possibly created by different **`Executor`** instances) given by *fs* to complete. Duplicate futures given to *fs* are removed and will be returned only once. Returns a named 2-tuple of sets. The first set, named `done`, contains the futures that completed (finished or cancelled futures) before the wait completed. The second set, named `not_done`, contains the futures that did not complete (pending or running futures).

timeout can be used to control the maximum number of seconds to wait before returning. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

return_when indica quando esta função deve retornar. Ele deve ser uma das seguintes constantes:

Constante	Descrição
concurrent.futures.FIRST_COMPLETED	A função irá retornar quando qualquer futuro terminar ou for cancelado.
concurrent.futures.FIRST_EXCEPTION	The function will return when any future finishes by raising an exception. If no future raises an exception then it is equivalent to `ALL_COMPLETED`.
concurrent.futures.ALL_COMPLETED	A função irá retornar quando todos os futuros encerrarem ou forem cancelados.

concurrent.futures.as_completed(*fs*, *timeout=None*)

Returns an iterator over the `Future` instances (possibly created by different `Executor` instances) given by *fs* that yields futures as they complete (finished or cancelled futures). Any futures given by *fs* that are duplicated will be returned once. Any futures that completed before **`as_completed()`** is called will be yielded first. The returned iterator raises a `TimeoutError` if `__next__()` is called and the result isn't available after *timeout* seconds from the original call to **`as_completed()`**. *timeout* can be an int or float. If *timeout* is not specified or `None`, there is no limit to the wait time.

Ver também

PEP 3148 [https://peps.python.org/pep-3148/] – futures - execute computations asynchronously

The proposal which described this feature for inclusion in the Python standard library.

Exception classes

exception concurrent.futures.CancelledError

Raised when a future is cancelled.

exception concurrent.futures.TimeoutError

A deprecated alias of `TimeoutError`, raised when a future operation exceeds the given timeout.

Alterado na versão 3.11: Esta classe foi feita como um apelido de `TimeoutError`.

exception concurrent.futures.BrokenExecutor

Derived from `RuntimeError`, this exception class is raised when an executor is broken for some reason, and cannot be used to submit or execute new tasks.

Adicionado na versão 3.7.

exception concurrent.futures.InvalidStateError

Raised when an operation is performed on a future that is not allowed in the current state.

Adicionado na versão 3.8.

exception concurrent.futures.thread.BrokenThreadPool

Derived from `BrokenExecutor`, this exception class is raised when one of the workers of a `ThreadPoolExecutor` has failed initializing.

Adicionado na versão 3.7.

exception concurrent.futures.interpreter.BrokenInterpreterPool

Derived from `BrokenThreadPool`, this exception class is raised when one of the workers of a `InterpreterPoolExecutor` has failed initializing.

Adicionado na versão 3.14.

exception concurrent.futures.process.BrokenProcessPool

Derived from `BrokenExecutor` (formerly `RuntimeError`), this exception class is raised when one of the workers of a `ProcessPoolExecutor` has terminated in a non-clean fashion (for example, if it was killed from the outside).

Adicionado na versão 3.3.

`concurrent.interpreters` — Múltiplos interpretadores no mesmo processo

Adicionado na versão 3.14.

Source code: Lib/concurrent/interpreters
[https://github.com/python/cpython/tree/3.15/Lib/concurrent/interpreters]

O módulo **`concurrent.interpreters`** constrói interfaces de nível mais alto sobre o módulo de mais baixo nível **`_interpreters`**.

O objetivo principal do módulo é fornecer uma API básica para gerenciar interpretadores (também conhecidos como "subinterpretadores") e executar operações neles. A execução envolve principalmente a mudança para um interpretador (na thread atual) e a chamada de uma função nesse contexto de execução.

For concurrency, interpreters themselves (and this module) don't provide much more than isolation, which on its own isn't useful. Actual concurrency is available separately through **`threads`** – see below.

Ver também

`InterpreterPoolExecutor`
Combina threads com interpretadores em uma interface familiar.

Isolando módulos de extensão
Como atualizar um módulo de extensão para suporte a vários interpretadores.

PEP 554 [https://peps.python.org/pep-0554/]

PEP 734 [https://peps.python.org/pep-0734/]

PEP 684 [https://peps.python.org/pep-0684/]

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Detalhes chaves

Antes de prosseguirmos, há um pequeno número de detalhes que devem ser lembrados sobre o uso de múltiplos interpretadores:

- isolado, por padrão
- nenhuma thread implícita
- nem todos os pacotes PyPI oferecem suporte ao uso em múltiplos interpretadores ainda

Introdução

An “interpreter” is effectively the execution context of the Python runtime. It contains all of the state the runtime needs to execute a program. This includes things like the import state and builtins. (Each thread, even if there’s only the main thread, has some extra runtime state, in addition to the current interpreter, related to the current exception and the bytecode eval loop.)

The concept and functionality of the interpreter have been a part of Python since version 2.2, but the feature was only available through the C-API and not well known, and the isolation was relatively incomplete until version 3.12.

Multiple Interpreters and Isolation

A Python implementation may support using multiple interpreters in the same process. CPython has this support. Each interpreter is effectively isolated from the others (with a limited number of carefully managed process-global exceptions to the rule).

That isolation is primarily useful as a strong separation between distinct logical components of a program, where you want to have careful control of how those components interact.

Nota

Interpreters in the same process can technically never be strictly isolated from one another since there are few restrictions on memory access within the same process. The Python runtime makes a best effort at isolation but extension modules may easily violate that. Therefore, do not use multiple interpreters in security-sensitive situations, where they shouldn't have access to each other's data.

Running in an Interpreter

Running in a different interpreter involves switching to it in the current thread and then calling some function. The runtime will execute the function using the current interpreter's state. The **`concurrent.interpreters`** module provides a basic API for creating and managing interpreters, as well as the switch-and-call operation.

No other threads are automatically started for the operation. There is a helper for that though. There is another dedicated helper for calling the builtin **`exec()`** in an interpreter.

When **`exec()`** (or **`eval()`**) are called in an interpreter, they run using the interpreter's **`__main__`** module as the "globals" namespace. The same is true for functions that aren't associated with any module. This is the same as how scripts invoked from the command-line run in the **`__main__`** module.

Concurrency and Parallelism

As noted earlier, interpreters do not provide any concurrency on their own. They strictly represent the isolated execution context the runtime will use *in the current thread*. That isolation makes them similar to processes, but they still enjoy in-process efficiency, like threads.

All that said, interpreters do naturally support certain flavors of concurrency. There's a powerful side effect of that isolation. It enables a different approach to concurrency than you can take with async or threads. It's a similar concurrency model to CSP or the actor model, a model which is relatively easy to reason about.

You can take advantage of that concurrency model in a single thread, switching back and forth between interpreters, Stackless-style. However, this model is more useful when you combine interpreters with multiple threads. This mostly involves starting a new thread, where you switch to another interpreter and run what you want there.

Each actual thread in Python, even if you're only running in the main thread, has its own *current* execution context. Multiple threads can use the same interpreter or different ones.

At a high level, you can think of the combination of threads and interpreters as threads with opt-in sharing.

As a significant bonus, interpreters are sufficiently isolated that they do not share the GIL, which means combining threads with multiple interpreters enables full multi-core parallelism. (This has been the case since Python 3.12.)

Communication Between Interpreters

In practice, multiple interpreters are useful only if we have a way to communicate between them. This usually involves some form of message passing, but can even mean sharing data in some carefully managed way.

With this in mind, the **`concurrent.interpreters`** module provides a **`queue.Queue`** implementation, available through **`create_queue()`**.

“Sharing” Objects

Any data actually shared between interpreters loses the thread-safety provided by the GIL. There are various options for dealing with this in extension modules. However, from Python code the lack of thread-safety means objects can’t actually be shared, with a few exceptions. Instead, a copy must be created, which means mutable objects won’t stay in sync.

By default, most objects are copied with `pickle` when they are passed to another interpreter. Nearly all of the immutable builtin objects are either directly shared or copied efficiently. For example:

- `None`
- `bool` (`True` and `False`)
- `bytes`
- `str`
- `int`
- `float`
- `tuple` (of similarly supported objects)

There are a small number of Python types that actually share mutable data between interpreters:

- `memoryview`
- `Queue`

Referência

Este módulo define as seguintes funções:

concurrent.interpreters.list_all()

Retorna `list` de objetos `Interpreter`, um para cada interpretador existente.

concurrent.interpreters.get_current()

Retorna um objeto `Interpreter` para o interpretador em execução no momento.

concurrent.interpreters.get_main()

Return an **`Interpreter`** object for the main interpreter. This is the interpreter the runtime created to run the REPL or the script given at the command-line. It is usually the only one.

concurrent.interpreters.create()

Inicializa um novo interpretador Python (ocioso) e retorna um objeto **`Interpreter`** para ele.

concurrent.interpreters.create_queue()

Initialize a new cross-interpreter queue and return a **`Queue`** object for it.

Objetos interpretador

class concurrent.interpreters.Interpreter(*id*)

Um único interpretador no processo atual.

Geralmente, **`Interpreter`** não deve ser chamado diretamente. Em vez disso, use **`create()`** ou uma das outras funções do módulo.

id

(somente leitura)

The underlying interpreter's ID.

whence

(somente leitura)

Uma string descrevendo de onde o interpretador veio.

is_running()

Retorna `True` se o interpretador estiver executando código em seu módulo **`__main__`** e `False` caso contrário.

close()

Finaliza e destrói o interpretador.

prepare_main(*ns=None*, ***kwargs*)

Bind objects in the interpreter's **`__main__`** module.

Some objects are actually shared and some are copied efficiently, but most are copied via **`pickle`**. See "Sharing" Objects.

exec(*code*, /, *dedent=True*)

Executa o código-fonte fornecido no interpretador (na thread atual).

call(*callable*, /, **args*, ***kwargs*)

Return the result of running the given function in the interpreter (in the current thread).

call_in_thread(*callable*, /, **args*, ***kwargs*)

Executa a função fornecida no interpretador (em uma nova thread).

Exceções

exception concurrent.interpreters.InterpreterError

Esta exceção, uma subclasse de **`Exception`**, é levantada quando ocorre um erro relacionado ao interpretador.

exception concurrent.interpreters.InterpreterNotFoundError

Esta exceção, uma subclasse de **`InterpreterError`**, é levantada quando o interpretador de destino não existe mais.

exception concurrent.interpreters.ExecutionFailed

Esta exceção, uma subclasse de **`InterpreterError`**, é levantada quando o código em execução levanta uma exceção não capturada.

excinfo

Um snapshot básico da exceção levantada no outro interpretador.

exception concurrent.interpreters.NotShareableError

Esta exceção, uma subclasse de **`TypeError`**, é levantada quando um objeto não pode ser enviado para outro interpretador.

Communicating Between Interpreters

class concurrent.interpreters.Queue(*id*)

A wrapper around a low-level, cross-interpreter queue, which implements the **`queue.Queue`** interface. The underlying queue can only be created through **`create_queue()`**.

Some objects are actually shared and some are copied efficiently, but most are copied via **`pickle`**. See "Sharing" Objects.

id

(somente leitura)

The queue's ID.

exception concurrent.interpreters.QueueEmptyError

This exception, a subclass of **`queue.Empty`**, is raised from **`Queue.get()`** and **`Queue.get_nowait()`** when the queue is empty.

exception concurrent.interpreters.QueueFullError

This exception, a subclass of **`queue.Full`**, is raised from **`Queue.put()`** and **`Queue.put_nowait()`** when the queue is full.

Uso básico

Criando um interpretador e executando código nele:

```
from concurrent import interpreters

interp = interpreters.create()

# Executa na thread atual do SO.

interp.exec('print("spam!")')

interp.exec("""if True:
    print('spam!')
    """)
```

```
from textwrap import dedent
interp.exec(dedent("""
    print('spam!')
    """))

def run(arg):
    return arg

res = interp.call(run, 'spam!')
print(res)

def run():
    print('spam!')

interp.call(run)

# Executa em uma nova thread do SO.

t = interp.call_in_thread(run)
t.join()
```

`subprocess` — Gerenciamento de subprocessos

Código-fonte: Lib/subprocess.py
[https://github.com/python/cpython/tree/3.15/Lib/subprocess.py]

The `subprocess` module allows you to spawn new processes, connect to their input/output/error pipes, and obtain their return codes. This module intends to replace several older modules and functions:

```
os.system
os.spawn*
```

Information about how the `subprocess` module can be used to replace these modules and functions can be found in the following sections.

Ver também

PEP 324 [https://peps.python.org/pep-0324/] – PEP propondo o módulo subprocess

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Using the `subprocess` Module

A abordagem recomendada para invocar subprocessos é usar a função `run()` para todos os casos de uso que ela pode manipular. Para casos de uso mais avançados, a interface subjacente `Popen` pode ser usada diretamente.

subprocess.run(*args*, *, *stdin=None*, *input=None*, *stdout=None*, *stderr=None*, *capture_output=False*, *shell=False*, *cwd=None*, *timeout=None*, *check=False*, *encoding=None*, *errors=None*, *text=None*, *env=None*, *universal_newlines=None*, ***other_popen_kwargs*)

Executa o comando descrito por *args*. Aguarda a conclusão do comando e retorna uma instância de **`CompletedProcess`**.

Os argumentos mostrados acima são meramente os mais comuns, descritos abaixo em Argumentos usados frequentemente (daí o uso de notação somente-nomeado na assinatura abreviada). A assinatura da função completa é basicamente a mesma do construtor **`Popen`** - a maioria dos argumentos para esta função são passados para aquela interface. (*timeout*, *input*, *check* e *capture_output* não são.)

Se *capture_output* for verdadeiro, *stdout* e *stderr* serão capturados. Quando usado, o objeto interno **`Popen`** é automaticamente criado com *stdout* and *stderr* definidos como **`PIPE`**. Os argumentos *stdout * e *stderr* não podem ser fornecidos ao mesmo tempo que *capture_output*. Se você deseja capturar e combinar ambos os fluxos em um, defina *stdout* para **`PIPE`** e *stderr* para **`STDOUT`**, ao invés de usar *capture_output*.

Um *timeout* pode ser especificado em segundos, ele é passado internamente para **`Popen.communicate()`**. Se o tempo limite expirar, o processo filho será eliminado e aguardado. A exceção **`TimeoutExpired`** será levantada novamente após o término do processo filho. A criação inicial do processo em si não pode ser interrompida em muitas APIs de plataforma, portanto, não há garantia de que você verá uma exceção de tempo limite até, pelo menos, o tempo que a criação do processo demorar.

O argumento *input* é passado para **`Popen.communicate()`** e, portanto, para o stdin do subprocesso. Se usado, deve ser uma sequência de bytes ou uma string se *encoding* ou *errors* forem especificados ou *text* for verdadeiro. Quando usado, o objeto internal **`Popen`** é automaticamente criado com **`PIPE`**, e o argumento *stdin* também não pode ser usado.

Se *check* for verdadeiro, e o processo sair com um código de saída diferente de zero, uma exceção **`CalledProcessError`** será levantada. Os atributos dessa exceção contêm os argumentos, o código de saída e stdout e stderr se eles foram capturados.

Se *encoding* ou *errors* forem especificados, ou *text* for verdadeiro, os objetos arquivo para stdin, stdout e stderr são abertos em modo de texto usando a *encoding* e *errors* especificados ou o **`io.TextIOWrapper`** padrão. O argumento *universal_newlines* é equivalente a *text* e é fornecido para compatibilidade com versões anteriores. Por padrão, os objetos arquivo são abertos no modo binário.

Se *env* não for `None`, deve ser um mapeamento que defina as variáveis de ambiente para o novo processo; elas são usadas em vez do comportamento padrão de herdar o ambiente do processo atual. É passado diretamente para **`Popen`**. Este mapeamento pode ser str para str em qualquer plataforma ou bytes para bytes em plataformas POSIX como **`os.environ`** ou **`os.environb`**.

Exemplos:

```
>>> subprocess.run(["ls", "-l"])  # não captura a saída
CompletedProcess(args=['ls', '-l'], returncode=0)

>>> subprocess.run("exit 1", shell=True, check=True)
Traceback (most recent call last):
  ...
subprocess.CalledProcessError: Command 'exit 1' returned non-

>>> subprocess.run(["ls", "-l", "/dev/null"], capture_output=
CompletedProcess(args=['ls', '-l', '/dev/null'], returncode=0
stdout=b'crw-rw-rw- 1 root root 1, 3 Jan 23 16:23 /dev/null\r
```

Adicionado na versão 3.5.

Alterado na versão 3.6: Adicionados os parâmetros *encoding* e *errors*.

Alterado na versão 3.7: Adicionado o parâmetro *text*, como um apelido mais compreensível que *universal_newlines*. Adicionado o parâmetro

capture_output.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

class subprocess.CompletedProcess

O valor de retorno de **`run()`**, representando um processo que foi concluído.

args

Os argumentos usados para iniciar o processo. Pode ser uma lista ou uma string.

returncode

Status de saída do processo filho. Normalmente, um status de saída 0 indica que foi executado com êxito.

Um valor negativo `-N` indica que o filho foi terminado pelo sinal `N` (POSIX apenas).

stdout

Stdout capturado do processo filho. Uma sequência de bytes ou uma string se **`run()`** foi chamada com uma codificação, erros ou text=True. `None` se stdout não foi capturado.

Se você executou o processo com `stderr=subprocess.STDOUT`, stdout e stderr serão combinados neste atributo, e **`stderr`** será `None`.

stderr

Stderr capturado do processo filho. Uma sequência de bytes ou uma string se **`run()`** foi chamada com uma codificação, erros ou text=True. `None` se stderr não foi capturado.

check_returncode()

Se `returncode` é diferente de zero, levantar um `CalledProcessError`.

Adicionado na versão 3.5.

subprocess.DEVNULL

Valor especial que pode ser usado como o argumento *stdin*, *stdout* ou *stderr* para `Popen` e indica que o arquivo especial `os.devnull` será usado.

Adicionado na versão 3.3.

subprocess.PIPE

Valor especial que pode ser usado como o argumento *stdin*, *stdout* ou *stderr* para `Popen` e indica que um encadeamento para o fluxo padrão deve ser aberto. Mais útil com `Popen.communicate()`.

subprocess.STDOUT

Valor especial que pode ser usado como o argumento *stderr* para `Popen` e indica que o erro padrão deve ir para o mesmo manipulador que a saída padrão.

exception subprocess.SubprocessError

Classe base para todas as outras exceções deste módulo.

Adicionado na versão 3.3.

exception subprocess.TimeoutExpired

Subclasse de `SubprocessError`, levantada quando um tempo limite expira enquanto espera por um processo filho.

cmd

Comando que foi usado para gerar o processo filho.

timeout

Tempo limite em segundos.

output

Saída do processo filho se ele foi capturado por `run()` ou `check_output()`. Caso contrário, `None`. Isto é sempre `bytes` quando qualquer saída foi capturada independente da configuração `text=True`. Pode permanecer `None` em vez de `b''` quando nenhuma saída foi observada.

stdout

Apelido para a saída, para simetria com `stderr`.

stderr

Saída de stderr do processo filho se ele foi capturado por `run()`. Caso contrário, `None`. Isto é sempre `bytes` quando a saída de stderr foi capturada independente da configuração `text=True`. Pode permanecer `None` em vez de `b''` quando nenhuma saída de stderr foi observada.

Adicionado na versão 3.3.

Alterado na versão 3.5: Adicionados os atributos *stdout* e *stderr*

exception subprocess.CalledProcessError

Subclasse de `SubprocessError`, levantada quando um processo executado por `check_call()`, `check_output()` ou `run()` (com `check=True`) retorna um status de saída diferente de zero.

returncode

Exit status of the child process, an integer. If the process exited due to a signal, this will be the negative signal number.

cmd

Comando que foi usado para gerar o processo filho.

output

Saída do processo filho se ele foi capturado por `run()` ou `check_output()`. Caso contrário, `None`.

stdout

Apelido para a saída, para simetria com **stderr**.

stderr

Saída stderr do processo filho se ele foi capturado por **run()**. Caso contrário, None.

Alterado na versão 3.5: Adicionados os atributos *stdout* e *stderr*

Argumentos usados frequentemente

Para suportar uma ampla variedade de casos de uso, o construtor **Popen** (e as funções de conveniência) aceita muitos argumentos opcionais. Para a maioria dos casos de uso típicos, muitos desses argumentos podem ser seguramente deixados em seus valores padrão. Os argumentos mais comumente necessários são:

args é necessário para todas as chamadas e deve ser uma string ou uma sequência de argumentos do programa. Fornecer uma sequência de argumentos geralmente é preferível, pois permite que o módulo cuide de qualquer escape e aspas em argumentos que forem necessários (por exemplo, para permitir espaços em nomes de arquivo). Se for passada uma única string, *shell* deve ser **True** (veja abaixo) ou então a string deve apenas nomear o programa a ser executado sem especificar nenhum argumento.

stdin, *stdout* e *stderr* especificam a entrada padrão, a saída padrão e a saída de erro do programa executado, respectivamente. Os valores válidos são None, **PIPE**, **DEVNULL**, um descritor de arquivo existente (um número inteiro positivo) e um objeto arquivo existente com um descritor de arquivo válido. Com as configurações padrão de None, nenhum redirecionamento ocorrerá. **PIPE** indica que um novo canal para o filho deve ser criado. **DEVNULL** indica que o arquivo especial **os.devnull** será usado. Além disso, *stderr* pode ser **STDOUT**, o que indica que os dados de stderr do processo filho devem ser capturados no mesmo identificador de arquivo que para *stdout*.

Se *encoding* ou *errors* forem especificados, ou *text* (também conhecido como *universal_newlines*) for verdadeiro, os objetos arquivo *stdin*, *stdout* e *stderr* serão abertos em modo de texto usando a *encoding* e *errors* especificados na chamada ou os valores padrão para `io.TextIOWrapper`.

Para *stdin*, os caracteres de fim de linha `'\n'` na entrada serão convertidos para o separador de linha padrão `os.linesep`. Para *stdout* e *stderr*, todas as terminações de linha na saída serão convertidas para `'\n'`. Para obter mais informações, consulte a documentação da classe `io.TextIOWrapper` quando o argumento *newline* para seu construtor é `None`.

Se o modo de texto não for usado, *stdin*, *stdout* e *stderr* serão abertos como fluxos binários. Nenhuma codificação ou conversão de final de linha é executada.

Alterado na versão 3.6: Adicionados os parâmetros *encoding* e *errors*.

Alterado na versão 3.7: Adicionado o parâmetro *text* como um apelido para *universal_newlines*.

Nota

O atributo newlines dos objetos arquivo `Popen.stdin`, `Popen.stdout` e `Popen.stderr` não são atualizados pelo método `Popen.communicate()`.

Se *shell* for `True`, o comando especificado será executado através do shell. Isso pode ser útil se você estiver usando Python principalmente para o fluxo de controle aprimorado que ele oferece sobre a maioria dos shells do sistema e ainda deseja acesso conveniente a outros recursos do shell, como canais de shell, caracteres curinga de nome de arquivo, expansão de variável de ambiente e expansão de ~ para o diretório inicial de um usuário. No entanto, observe que o próprio Python oferece implementações de muitos recursos semelhantes a shell

(em particular, **glob**, **fnmatch**, **os.walk()**, **os.path.expandvars()**, **os.path.expanduser()** e **shutil**).

Alterado na versão 3.3: Quando *universal_newlines* é `True`, a classe usa a codificação **locale.getpreferredencoding(False)** em vez de `locale.getpreferredencoding()`. Veja a classe **io.TextIOWrapper** para mais informações sobre esta alteração.

Nota

Leia a seção Security Considerations antes de usar `shell=True`.

Essas opções, junto com todas as outras opções, são descritas em mais detalhes na documentação do construtor **Popen**.

Construtor Popen

A criação e gerenciamento do processo subjacente neste módulo é manipulado pela classe **Popen**. Ela oferece muita flexibilidade para que os desenvolvedores sejam capazes de lidar com os casos menos comuns não cobertos pelas funções de conveniência.

class subprocess.Popen(*args*, *bufsize=-1*, *executable=None*, *stdin=None*, *stdout=None*, *stderr=None*, *preexec_fn=None*, *close_fds=True*, *shell=False*, *cwd=None*, *env=None*, *universal_newlines=None*, *startupinfo=None*, *creationflags=0*, *restore_signals=True*, *start_new_session=False*, *pass_fds=()*, ***, *group=None*, *extra_groups=None*, *user=None*, *umask=-1*, *encoding=None*, *errors=None*, *text=None*, *pipesize=-1*, *process_group=None*)

Executa um programa filho em um novo processo. No POSIX, a classe usa o comportamento de **os.execvpe()** para executar o programa filho. No Windows, a classe usa a função Windows `CreateProcess()`. Os argumentos para **Popen** são os seguintes.

args deve ser uma sequência de argumentos do programa ou então uma única string ou objeto caminho ou similar. Por padrão, o programa a ser

executado é o primeiro item em *args* se *args* for uma sequência. Se *args* for uma string, a interpretação depende da plataforma e é descrita abaixo. Consulte os argumentos *shell* e *executable* para obter diferenças adicionais em relação ao comportamento padrão. Salvo indicação em contrário, é recomendado passar *args* como uma sequência.

Aviso

Para confiabilidade máxima, use um caminho totalmente qualificado para o executável. Para procurar um nome não qualificado no **`PATH`**, use `shutil.which()`. Em todas as plataformas, passar `sys.executable` é a maneira recomendada de relançar o interpretador Python atual, e usar o formato de linha de comando `-m` para iniciar um módulo instalado.

Resolver o caminho do executável (ou o primeiro item de *args*) depende da plataforma. Para POSIX, veja `os.execvpe()`, e note que ao resolver ou procurar o caminho do executável, *cwd* substitui o diretório de trabalho atual e *env* pode substituir a variável de ambiente `PATH`. Para Windows, veja a documentação dos parâmetros `lpApplicationName` e `lpCommandLine` do WinAPI `CreateProcess`, e note que ao resolver ou procurar o caminho do executável com `shell=False`, *cwd* não substitui o diretório de trabalho atual e *env* não pode substituir a variável de ambiente `PATH`. Usar um caminho completo evita todas essas variações.

Um exemplo de passagem de alguns argumentos para um programa externo como uma sequência é:

```
Popen(["/usr/bin/git", "commit", "-m", "Fixes a bug."])
```

No POSIX, se *args* for uma string, a string é interpretada como o nome ou caminho do programa a ser executado. No entanto, isso só pode ser feito se não forem passados argumentos para o programa.

Nota

Pode não ser óbvio como quebrar um comando shell em uma sequência de argumentos, especialmente em casos complexos. **`shlex.split()`** pode ilustrar como determinar a tokenização correta para *args*:

```
>>> import shlex, subprocess
>>> command_line = input()
/bin/vikings -input eggs.txt -output "spam spam.txt" -cmd "
>>> args = shlex.split(command_line)
>>> print(args)
['/bin/vikings', '-input', 'eggs.txt', '-output', 'spam spa
>>> p = subprocess.Popen(args) # Successo!
```

Observe em particular que as opções (como *-input*) e argumentos (como *eggs.txt*) que são separados por espaços em branco no shell vão em elementos de lista separados, enquanto os argumentos que precisam de aspas ou escape de contrabarra quando usados no shell (como nomes de arquivos contendo espaços ou o comando *echo* mostrado acima) são um único elemento de lista.

No Windows, se *args* for uma sequência, será convertido em uma string da maneira descrita em Converter uma sequência de argumentos em uma string no Windows. Isso ocorre porque o `CreateProcess()` subjacente opera em strings.

Alterado na versão 3.6: O parâmetro *args* aceita um objeto caminho ou similar se *shell* é `False` e uma sequência contendo objetos caminho ou similares no POSIX.

Alterado na versão 3.8: O parâmetro *args* aceita um objeto caminho ou similar se *shell* for `False` e uma sequência contendo bytes e objetos caminho ou similares no Windows.

O argumento *shell* (que por padrão é `False`) especifica se deve usar o *shell* como o programa a ser executado. Se *shell* for `True`, é recomendado passar *args* como uma string em vez de uma sequência.

Em POSIX com `shell=True`, o *shell* padrão é `/bin/sh`. Se *args* for uma string, a *string* especifica o comando a ser executado através do *shell*. Isso significa que a *string* deve ser formatada exatamente como seria ao ser digitada no prompt do *shell*. Isso inclui, por exemplo, citar ou escapar com contrabarra nomes de arquivos que contenham espaços. Se *args* for uma sequência, o primeiro item especifica a *string* do comando, e quaisquer itens adicionais serão tratados como argumentos adicionais para o próprio *shell*. Ou seja, **`Popen`** faz o equivalente a:

```
Popen(['/bin/sh', '-c', args[0], args[1], ...])
```

No Windows com `shell=True`, a variável de ambiente **`COMSPEC`** especifica o shell padrão. A única vez que você precisa especificar `shell=True` no Windows é quando o comando que você deseja executar é construído no shell (por exemplo, **dir** ou **copy**). Você não precisa de `shell=True` para executar um arquivo em lote ou executável baseado em console.

Nota

Leia a seção Security Considerations antes de usar `shell=True`.

bufsize será fornecido como o argumento correspondente à função **`open()`** ao criar os objetos arquivo de encadeamento stdin/stdout/stderr:

- `0` significa sem buffer (leitura e gravação são uma chamada de sistema e podem retornar curto)
- `1` significa buffer de linha (utilizável somente se `text=True` ou `universal_newlines=True`)
- qualquer outro valor positivo significa usar um buffer de aproximadamente esse tamanho
- bufsize negativo (o padrão) significa que o padrão do sistema io.DEFAULT_BUFFER_SIZE será usado.

Alterado na versão 3.3.1: bufsize agora assume o padrão -1 para habilitar o buffer por padrão para corresponder ao comportamento que a maioria dos códigos espera. Em versões anteriores ao Python 3.2.4 e 3.3.1, ele assumia incorretamente o padrão `0`, que não tinha buffer e

permitia leituras curtas. Isso não foi intencional e não correspondeu ao comportamento do Python 2, como a maioria dos códigos esperava.

O argumento *executable* especifica um programa de substituição para executar. Ele é muito raramente necessário. Quando `shell=False`, *executable* substitui o programa a ser executado especificado por *args*. No entanto, o *args* original ainda é passado para o programa. A maioria dos programas trata o programa especificado por *args* como o nome do comando, que pode então ser diferente do programa realmente executado. No POSIX, o nome *args* se torna o nome de exibição para o executável em utilitários como **ps**. Se `shell=True`, no POSIX o argumento *executable* especifica um shell de substituição para o padrão `/bin/sh`.

Alterado na versão 3.6: O parâmetro *executable* aceita um objeto caminho ou similar no POSIX.

Alterado na versão 3.8: O parâmetro *executable* aceita um objeto caminho ou similar bytes no Windows.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

stdin, *stdout* e *stderr* especificam a entrada padrão, a saída padrão e a saída de erro do programa executado, respectivamente. Os valores válidos são `None`, **`PIPE`**, **`DEVNULL`**, um descritor de arquivo existente (um número inteiro positivo) e um objeto arquivo existente com um descritor de arquivo válido. Com as configurações padrão de `None`, nenhum redirecionamento ocorrerá. **`PIPE`** indica que um novo canal para o filho deve ser criado. **`DEVNULL`** indica que o arquivo especial **`os.devnull`** será usado. Além disso, *stderr* pode ser **`STDOUT`**, o que indica que os dados de stderr da aplicação devem ser capturados no mesmo identificador de arquivo que para *stdout*.

Se *preexec_fn* for definido como um objeto chamável, esse objeto será chamado no processo filho pouco antes de o filho ser executado. (Somente POSIX)

Aviso

O parâmetro *preexec_fn* NÃO É SEGURO para uso na presença de threads em sua aplicação. O processo filho pode travar antes de exec ser chamado.

Nota

Se você precisar modificar o ambiente para o filho, use o parâmetro *env* em vez de fazer isso em um *preexec_fn*. Os parâmetros *start_new_session* e *process_group* devem tomar o lugar do código usando *preexec_fn* para chamar `os.setsid()` ou `os.setpgid()` no filho.

Alterado na versão 3.8: O parâmetro *preexec_fn* não é mais suportado em subinterpretadores. O uso do parâmetro em um subinterpretador levanta `RuntimeError`. A nova restrição pode afetar aplicações que são implantadas em mod_wsgi, uWSGI e outros ambientes incorporados.

If *close_fds* is true, all file descriptors except `0`, `1` and `2` will be closed before the child process is executed. Otherwise when *close_fds* is false, file descriptors obey their inheritable flag as described in Herança de descritores de arquivos.

On Windows, if *close_fds* is true then no handles will be inherited by the child process unless explicitly passed in the `handle_list` element of `STARTUPINFO.lpAttributeList`, or by standard handle redirection.

Alterado na versão 3.2: The default for *close_fds* was changed from `False` to what is described above.

Alterado na versão 3.7: On Windows the default for *close_fds* was changed from `False` to `True` when redirecting the standard handles. It's

now possible to set *close_fds* to **`True`** when redirecting the standard handles.

pass_fds is an optional sequence of file descriptors to keep open between the parent and child. Providing any *pass_fds* forces *close_fds* to be **`True`**. (POSIX only)

Alterado na versão 3.2: O parâmetro *pass_fds* foi adicionado.

If *cwd* is not `None`, the function changes the working directory to *cwd* before executing the child. *cwd* can be a string, bytes or path-like object. On POSIX, the function looks for *executable* (or for the first item in *args*) relative to *cwd* if the executable path is a relative path.

Alterado na versão 3.6: cwd parameter accepts a path-like object on POSIX.

Alterado na versão 3.7: cwd parameter accepts a path-like object on Windows.

Alterado na versão 3.8: cwd parameter accepts a bytes object on Windows.

If *restore_signals* is true (the default) all signals that Python has set to SIG_IGN are restored to SIG_DFL in the child process before the exec. Currently this includes the SIGPIPE, SIGXFZ and SIGXFSZ signals. (POSIX only)

Alterado na versão 3.2: restore_signals foi adicionado.

If *start_new_session* is true the `setsid()` system call will be made in the child process prior to the execution of the subprocess.

Disponibilidade: POSIX

Alterado na versão 3.2: start_new_session foi adicionado.

If *process_group* is a non-negative integer, the `setpgid(0, value)` system call will be made in the child process prior to the execution of

the subprocess.

Disponibilidade: POSIX

Alterado na versão 3.11: process_group was added.

If *group* is not `None`, the setregid() system call will be made in the child process prior to the execution of the subprocess. If the provided value is a string, it will be looked up via **`grp.getgrnam()`** and the value in `gr_gid` will be used. If the value is an integer, it will be passed verbatim. (POSIX only)

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *extra_groups* is not `None`, the setgroups() system call will be made in the child process prior to the execution of the subprocess. Strings provided in *extra_groups* will be looked up via **`grp.getgrnam()`** and the values in `gr_gid` will be used. Integer values will be passed verbatim. (POSIX only)

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *user* is not `None`, the setreuid() system call will be made in the child process prior to the execution of the subprocess. If the provided value is a string, it will be looked up via **`pwd.getpwnam()`** and the value in `pw_uid` will be used. If the value is an integer, it will be passed verbatim. (POSIX only)

Nota

Specifying *user* will not drop existing supplementary group memberships! The caller must also pass `extra_groups=()` to reduce the group membership of the child process for security purposes.

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *umask* is not negative, the umask() system call will be made in the child process prior to the execution of the subprocess.

Disponibilidade: POSIX

Adicionado na versão 3.9.

If *env* is not `None`, it must be a mapping that defines the environment variables for the new process; these are used instead of the default behavior of inheriting the current process' environment. This mapping can be str to str on any platform or bytes to bytes on POSIX platforms much like **`os.environ`** or **`os.environb`**.

Nota

If specified, *env* must provide any variables required for the program to execute. On Windows, in order to run a side-by-side assembly [https://en.wikipedia.org/wiki/Side-by-Side_Assembly] the specified *env* **must** include a valid `%SystemRoot%`.

If *encoding* or *errors* are specified, or *text* is true, the file objects *stdin*, *stdout* and *stderr* are opened in text mode with the specified *encoding* and *errors*, as described above in Argumentos usados frequentemente. The *universal_newlines* argument is equivalent to *text* and is provided for backwards compatibility. By default, file objects are opened in binary mode.

Adicionado na versão 3.6: encoding e *errors* foram adicionados.

Adicionado na versão 3.7: text foi adicionado como um atalho mais legível para *universal_newlines*.

If given, *startupinfo* will be a **`STARTUPINFO`** object, which is passed to the underlying `CreateProcess` function.

If given, *creationflags*, can be one or more of the following flags:

- `CREATE_NEW_CONSOLE`
- `CREATE_NEW_PROCESS_GROUP`
- `ABOVE_NORMAL_PRIORITY_CLASS`
- `BELOW_NORMAL_PRIORITY_CLASS`
- `HIGH_PRIORITY_CLASS`
- `IDLE_PRIORITY_CLASS`
- `NORMAL_PRIORITY_CLASS`
- `REALTIME_PRIORITY_CLASS`
- `CREATE_NO_WINDOW`
- `DETACHED_PROCESS`
- `CREATE_DEFAULT_ERROR_MODE`
- `CREATE_BREAKAWAY_FROM_JOB`

pipesize can be used to change the size of the pipe when `PIPE` is used for *stdin*, *stdout* or *stderr*. The size of the pipe is only changed on platforms that support this (only Linux at this time of writing). Other platforms will ignore this parameter.

Alterado na versão 3.10: Added the *pipesize* parameter.

Popen objects are supported as context managers via the `with` statement: on exit, standard file descriptors are closed, and the process is waited for.

```
with Popen(["ifconfig"], stdout=PIPE) as proc:
    log.write(proc.stdout.read())
```

Popen and the other functions in this module that use it raise an auditing event `subprocess.Popen` with arguments `executable`, `args`, `cwd`, and `env`. The value for `args` may be a single string or a list of strings, depending on platform.

Alterado na versão 3.2: Adicionado suporte a gerenciador de contexto.

Alterado na versão 3.6: O destruidor Popen agora emite um aviso `ResourceWarning` se o processo filho ainda estiver em execução.

Alterado na versão 3.8: Popen can use `os.posix_spawn()` in some cases for better performance. On Windows Subsystem for Linux and QEMU User Emulation, Popen constructor using **`os.posix_spawn()`** no longer raise an exception on errors like missing program, but the child process fails with a non-zero `returncode`.

Exceções

Exceções levantadas no processo filho, antes que o novo programa comece a ser executado, serão levantadas novamente no pai.

The most common exception raised is `OSError`. This occurs, for example, when trying to execute a non-existent file. Applications should prepare for **`OSError`** exceptions. Note that, when `shell=True`, **`OSError`** will be raised by the child only if the selected shell itself was not found. To determine if the shell failed to find the requested application, it is necessary to check the return code or output from the subprocess.

A exceção `ValueError` será levantada se `Popen` for chamado com argumentos inválidos.

`check_call()` e `check_output()` levantarão `CalledProcessError` se o processo chamado retornar um código de retorno diferente de zero.

All of the functions and methods that accept a *timeout* parameter, such as `run()` and `Popen.communicate()` will raise `TimeoutExpired` if the timeout expires before the process exits.

Todas as exceções definidas neste módulo herdam de `SubprocessError`.

Adicionado na versão 3.3: A classe base `SubprocessError` foi adicionada.

Considerações de Segurança

Unlike some other popen functions, this library will not implicitly choose to call a system shell. This means that all characters, including shell metacharacters, can safely be passed to child processes. If the shell is

invoked explicitly, via `shell=True`, it is the application's responsibility to ensure that all whitespace and metacharacters are quoted appropriately to avoid shell injection [https://en.wikipedia.org/wiki/Shell_injection#Shell_injection] vulnerabilities. On some platforms, it is possible to use **`shlex.quote()`** for this escaping.

On Windows, batch files (`*.bat` or `*.cmd`) may be launched by the operating system in a system shell regardless of the arguments passed to this library. This could result in arguments being parsed according to shell rules, but without any escaping added by Python. If you are intentionally launching a batch file with arguments from untrusted sources, consider passing `shell=True` to allow Python to escape special characters. See gh-114539 [https://github.com/python/cpython/issues/114539] for additional discussion.

Objetos Popen

Instâncias da classe **`Popen`** têm os seguintes métodos:

Popen.poll()

Check if child process has terminated. Set and return **`returncode`** attribute. Otherwise, returns `None`.

Popen.wait(*timeout=None*)

Wait for child process to terminate. Set and return **`returncode`** attribute.

If the process does not terminate after *timeout* seconds, raise a **`TimeoutExpired`** exception. It is safe to catch this exception and retry the wait.

Nota

This will deadlock when using `stdout=PIPE` or `stderr=PIPE` and the child process generates enough output to a pipe such that it blocks waiting for the OS pipe buffer to accept more data. Use **`Popen.communicate()`** when using pipes to avoid that.

Nota

When `timeout` is not `None` and the platform supports it, an efficient event-driven mechanism is used to wait for process termination:

- Linux >= 5.3 uses `os.pidfd_open()` + `select.poll()`
- macOS and other BSD variants use `select.kqueue()` + `KQ_FILTER_PROC` + `KQ_NOTE_EXIT`
- Windows uses `WaitForSingleObject`

If none of these mechanisms are available, the function falls back to a busy loop (non-blocking call and short sleeps).

Nota

Use the `asyncio` module for an asynchronous wait: see `asyncio.create_subprocess_exec`.

Alterado na versão 3.3: timeout foi adicionado.

Alterado na versão 3.15: if *timeout* is not `None`, use efficient event-driven implementation on Linux >= 5.3 and macOS / BSD.

Popen.communicate(*input=None*, *timeout=None*)

Interact with process: Send data to stdin. Read data from stdout and stderr, until end-of-file is reached. Wait for process to terminate and set the `returncode` attribute. The optional *input* argument should be data to be sent to the child process, or `None`, if no data should be sent to the child. If streams were opened in text mode, *input* must be a string. Otherwise, it must be bytes.

`communicate()` returns a tuple `(stdout_data, stderr_data)`. The data will be strings if streams were opened in text mode; otherwise, bytes.

Note that if you want to send data to the process's stdin, you need to create the Popen object with `stdin=PIPE`. Similarly, to get anything other than `None` in the result tuple, you need to give `stdout=PIPE` and/or `stderr=PIPE` too.

If the process does not terminate after *timeout* seconds, a **`TimeoutExpired`** exception will be raised. Catching this exception and retrying communication will not lose any output. Supplying *input* to a subsequent post-timeout **`communicate()`** call is in undefined behavior and may become an error in the future.

The child process is not killed if the timeout expires, so in order to cleanup properly a well-behaved application should kill the child process and finish communication:

```
proc = subprocess.Popen(...)
try:
    outs, errs = proc.communicate(timeout=15)
except TimeoutExpired:
    proc.kill()
    outs, errs = proc.communicate()
```

After a call to **`communicate()`** raises **`TimeoutExpired`**, do not call **`wait()`**. Use an additional **`communicate()`** call to finish handling pipes and populate the **`returncode`** attribute.

Nota

The data read is buffered in memory, so do not use this method if the data size is large or unlimited.

Alterado na versão 3.3: timeout foi adicionado.

Popen.send_signal(*signal*)

Envia o sinal *signal* para o filho.

Não faz nada se o processo for concluído.

Nota

On Windows, SIGTERM is an alias for **`terminate()`**. CTRL_C_EVENT and CTRL_BREAK_EVENT can be sent to processes started with a *creationflags* parameter which includes `CREATE_NEW_PROCESS_GROUP`.

Popen.terminate()

Stop the child. On POSIX OSs the method sends **SIGTERM** to the child. On Windows the Win32 API function **TerminateProcess()** is called to stop the child.

Popen.kill()

Kills the child. On POSIX OSs the function sends SIGKILL to the child. On Windows **kill()** is an alias for **terminate()**.

The following attributes are also set by the class for you to access. Reassigning them to new values is unsupported:

Popen.args

O argumento *args* conforme foi passado para **Popen** - uma sequência de argumentos do programa ou então uma única string.

Adicionado na versão 3.3.

Popen.stdin

If the *stdin* argument was **PIPE**, this attribute is a writeable stream object as returned by **open()**. If the *encoding* or *errors* arguments were specified or the *text* or *universal_newlines* argument was `True`, the stream is a text stream, otherwise it is a byte stream. If the *stdin* argument was not **PIPE**, this attribute is `None`.

Popen.stdout

If the *stdout* argument was **PIPE**, this attribute is a readable stream object as returned by **open()**. Reading from the stream provides output from the child process. If the *encoding* or *errors* arguments were specified or the *text* or *universal_newlines* argument was `True`, the stream is a text stream, otherwise it is a byte stream. If the *stdout* argument was not **PIPE**, this attribute is `None`.

Popen.stderr

If the *stderr* argument was **PIPE**, this attribute is a readable stream object as returned by **open()**. Reading from the stream provides error output from the child process. If the *encoding* or *errors* arguments were specified or the *text* or *universal_newlines* argument was `True`, the stream is a text stream, otherwise it is a byte stream. If the *stderr* argument was not **PIPE**, this attribute is `None`.

Aviso

Use **communicate()** rather than **.stdin.write**, **.stdout.read** or **.stderr.read** to avoid deadlocks due to any of the other OS pipe buffers filling up and blocking the child process.

Popen.pid

O ID de processo do processo filho.

Observe que se você definir o argumento *shell* como `True`, este é o ID do processo do shell gerado.

Popen.returncode

The child return code. Initially `None`, **returncode** is set by a call to the **poll()**, **wait()**, or **communicate()** methods if they detect that the process has terminated.

A `None` value indicates that the process hadn't yet terminated at the time of the last method call.

Um valor negativo `-N` indica que o filho foi terminado pelo sinal `N` (POSIX apenas).

When `shell=True`, the return code reflects the exit status of the shell itself (e.g. `/bin/sh`), which may map signals to codes such as `128+N`. See the documentation of the shell (for example, the Bash manual's Exit Status) for details.

Windows Popen Helpers

The **STARTUPINFO** class and following constants are only available on Windows.

class subprocess.STARTUPINFO(*, *dwFlags=0*, *hStdInput=None*, *hStdOutput=None*, *hStdError=None*, *wShowWindow=0*, *lpAttributeList=None*)

Partial support of the Windows STARTUPINFO [https://msdn.microsoft.com/en-us/library/ms686331(v=vs.85).aspx] structure is used for **Popen** creation. The following attributes can be set by passing them as keyword-only arguments.

Alterado na versão 3.7: Keyword-only argument support was added.

dwFlags

A bit field that determines whether certain **STARTUPINFO** attributes are used when the process creates a window.

```
si = subprocess.STARTUPINFO()
si.dwFlags = subprocess.STARTF_USESTDHANDLES | subprocess
```

hStdInput

If **dwFlags** specifies **STARTF_USESTDHANDLES**, this attribute is the standard input handle for the process. If **STARTF_USESTDHANDLES** is not specified, the default for standard input is the keyboard buffer.

hStdOutput

If **dwFlags** specifies **STARTF_USESTDHANDLES**, this attribute is the standard output handle for the process. Otherwise, this attribute is ignored and the default for standard output is the console window's buffer.

hStdError

If **dwFlags** specifies **STARTF_USESTDHANDLES**, this attribute is the standard error handle for the process. Otherwise, this attribute is ignored and the default for standard error is the console window's buffer.

wShowWindow

If **`dwFlags`** specifies **`STARTF_USESHOWWINDOW`**, this attribute can be any of the values that can be specified in the `nCmdShow` parameter for the ShowWindow [https://msdn.microsoft.com/en-us/library/ms633548(v=vs.85).aspx] function, except for `SW_SHOWDEFAULT`. Otherwise, this attribute is ignored.

`SW_HIDE` is provided for this attribute. It is used when **`Popen`** is called with `shell=True`.

lpAttributeList

A dictionary of additional attributes for process creation as given in `STARTUPINFOEX`, see UpdateProcThreadAttribute [https://msdn.microsoft.com/en-us/library/windows/desktop/ms686880(v=vs.85).aspx].

Atributos suportados:

handle_list

Sequence of handles that will be inherited. *close_fds* must be true if non-empty.

The handles must be temporarily made inheritable by **`os.set_handle_inheritable()`** when passed to the **`Popen`** constructor, else **`OSError`** will be raised with Windows error `ERROR_INVALID_PARAMETER` (87).

Aviso

In a multithreaded process, use caution to avoid leaking handles that are marked inheritable when combining this feature with concurrent calls to other process creation functions that inherit all handles such as **`os.system()`**. This also applies to standard handle redirection, which temporarily creates inheritable handles.

Adicionado na versão 3.7.

Constantes do Windows

The **`subprocess`** module exposes the following constants.

subprocess.STD_INPUT_HANDLE

The standard input device. Initially, this is the console input buffer, `CONIN$`.

subprocess.STD_OUTPUT_HANDLE

The standard output device. Initially, this is the active console screen buffer, `CONOUT$`.

subprocess.STD_ERROR_HANDLE

The standard error device. Initially, this is the active console screen buffer, `CONOUT$`.

subprocess.SW_HIDE

Oculta a janela. Outra janela será ativada.

subprocess.STARTF_USESTDHANDLES

Specifies that the **`STARTUPINFO.hStdInput`**, **`STARTUPINFO.hStdOutput`**, and **`STARTUPINFO.hStdError`** attributes contain additional information.

subprocess.STARTF_USESHOWWINDOW

Specifies that the **`STARTUPINFO.wShowWindow`** attribute contains additional information.

subprocess.STARTF_FORCEONFEEDBACK

A **`STARTUPINFO.dwFlags`** parameter to specify that the *Working in Background* mouse cursor will be displayed while a process is launching. This is the default behavior for GUI processes.

Adicionado na versão 3.13.

subprocess.STARTF_FORCEOFFFEEDBACK

A **`STARTUPINFO.dwFlags`** parameter to specify that the mouse cursor will not be changed when launching a process.

Adicionado na versão 3.13.

subprocess.CREATE_NEW_CONSOLE

The new process has a new console, instead of inheriting its parent's console (the default).

subprocess.CREATE_NEW_PROCESS_GROUP

A **Popen** `creationflags` parameter to specify that a new process group will be created. This flag is necessary for using **os.kill()** on the subprocess.

This flag is ignored if **CREATE_NEW_CONSOLE** is specified.

subprocess.ABOVE_NORMAL_PRIORITY_CLASS

A **Popen** `creationflags` parameter to specify that a new process will have an above average priority.

Adicionado na versão 3.7.

subprocess.BELOW_NORMAL_PRIORITY_CLASS

A **Popen** `creationflags` parameter to specify that a new process will have a below average priority.

Adicionado na versão 3.7.

subprocess.HIGH_PRIORITY_CLASS

A **Popen** `creationflags` parameter to specify that a new process will have a high priority.

Adicionado na versão 3.7.

subprocess.IDLE_PRIORITY_CLASS

A **Popen** `creationflags` parameter to specify that a new process will have an idle (lowest) priority.

Adicionado na versão 3.7.

subprocess.NORMAL_PRIORITY_CLASS

A **Popen** `creationflags` parameter to specify that a new process will have a normal priority. (default)

Adicionado na versão 3.7.

subprocess.REALTIME_PRIORITY_CLASS

A **Popen** `creationflags` parameter to specify that a new process will have realtime priority. You should almost never use REALTIME_PRIORITY_CLASS, because this interrupts system threads that manage mouse input, keyboard input, and background disk flushing. This class can be appropriate for applications that “talk” directly to hardware or that perform brief tasks that should have limited interruptions.

Adicionado na versão 3.7.

subprocess.CREATE_NO_WINDOW

A **Popen** `creationflags` parameter to specify that a new process will not create a window.

Adicionado na versão 3.7.

subprocess.DETACHED_PROCESS

A **Popen** `creationflags` parameter to specify that a new process will not inherit its parent’s console. This value cannot be used with CREATE_NEW_CONSOLE.

Adicionado na versão 3.7.

subprocess.CREATE_DEFAULT_ERROR_MODE

A **Popen** `creationflags` parameter to specify that a new process does not inherit the error mode of the calling process. Instead, the new process gets the default error mode. This feature is particularly useful for multithreaded shell applications that run with hard errors disabled.

Adicionado na versão 3.7.

subprocess.CREATE_BREAKAWAY_FROM_JOB

A **Popen** `creationflags` parameter to specify that a new process is not associated with the job.

Adicionado na versão 3.7.

API de alto nível mais antiga

Antes do Python 3.5, essas três funções constituíam a API de alto nível para subprocesso. Agora você pode usar **run()** em muitos casos, mas muitos códigos existentes chamam essas funções.

subprocess.call(*args*, ***, *stdin=None*, *stdout=None*, *stderr=None*, *shell=False*, *cwd=None*, *timeout=None*, ***other_popen_kwargs*)

Run the command described by *args*. Wait for command to complete, then return the **returncode** attribute.

Code needing to capture stdout or stderr should use **run()** instead:

```
run(...).returncode
```

To suppress stdout or stderr, supply a value of **DEVNULL**.

The arguments shown above are merely some common ones. The full function signature is the same as that of the **Popen** constructor - this function passes all supplied arguments other than *timeout* directly through to that interface.

Nota

Do not use `stdout=PIPE` or `stderr=PIPE` with this function. The child process will block if it generates enough output to a pipe to fill up the OS pipe buffer as the pipes are not being read from.

Alterado na versão 3.3: timeout foi adicionado.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos

por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

subprocess.check_call(*args*, ***, *stdin=None*, *stdout=None*, *stderr=None*, *shell=False*, *cwd=None*, *timeout=None*, ***other_popen_kwargs*)

Run command with arguments. Wait for command to complete. If the return code was zero then return, otherwise raise **`CalledProcessError`**. The **`CalledProcessError`** object will have the return code in the **`returncode`** attribute. If **`check_call()`** was unable to start the process it will propagate the exception that was raised.

Code needing to capture stdout or stderr should use **`run()`** instead:

```
run(..., check=True)
```

To suppress stdout or stderr, supply a value of **`DEVNULL`**.

The arguments shown above are merely some common ones. The full function signature is the same as that of the **`Popen`** constructor - this function passes all supplied arguments other than *timeout* directly through to that interface.

Nota

Do not use `stdout=PIPE` or `stderr=PIPE` with this function. The child process will block if it generates enough output to a pipe to fill up the OS pipe buffer as the pipes are not being read from.

Alterado na versão 3.3: timeout foi adicionado.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

subprocess.check_output(*args*, *, *stdin=None*, *stderr=None*, *shell=False*, *cwd=None*, *encoding=None*, *errors=None*, *universal_newlines=None*, *timeout=None*, *text=None*, ***other_popen_kwargs*)

Executa o comando com argumentos e retorna sua saída.

If the return code was non-zero it raises a **`CalledProcessError`**. The **`CalledProcessError`** object will have the return code in the **`returncode`** attribute and any output in the **`output`** attribute.

Isso equivale a:

```
run(..., check=True, stdout=PIPE).stdout
```

The arguments shown above are merely some common ones. The full function signature is largely the same as that of **`run()`** - most arguments are passed directly through to that interface. One API deviation from **`run()`** behavior exists: passing `input=None` will behave the same as `input=b''` (or `input=''`, depending on other arguments) rather than using the parent's standard input file handle.

By default, this function will return the data as encoded bytes. The actual encoding of the output data may depend on the command being invoked, so the decoding to text will often need to be handled at the application level.

This behaviour may be overridden by setting *text*, *encoding*, *errors*, or *universal_newlines* to `True` as described in Argumentos usados frequentemente and **`run()`**.

To also capture standard error in the result, use `stderr=subprocess.STDOUT`:

```
>>> subprocess.check_output(
...     "ls non_existent_file; exit 0",
...     stderr=subprocess.STDOUT,
...     shell=True)
'ls: non_existent_file: No such file or directory\n'
```

Adicionado na versão 3.1.

Alterado na versão 3.3: timeout foi adicionado.

Alterado na versão 3.4: Support for the *input* keyword argument was added.

Alterado na versão 3.6: encoding and *errors* were added. See `run()` for details.

Adicionado na versão 3.7: text foi adicionado como um atalho mais legível para *universal_newlines*.

Alterado na versão 3.12: Ordem de pesquisa do shell do Windows alterada para `shell=True`. O diretório atual e `%PATH%` são substituídos por `%COMSPEC%` e `%SystemRoot%\System32\cmd.exe`. Como resultado, colocar um programa malicioso chamado `cmd.exe` em um diretório atual não funciona mais.

Replacing Older Functions with the `subprocess` Module

In this section, “a becomes b” means that b can be used as a replacement for a.

Nota

All “a” functions in this section fail (more or less) silently if the executed program cannot be found; the “b” replacements raise `OSError` instead.

In addition, the replacements using `check_output()` will fail with a `CalledProcessError` if the requested operation produces a non-zero return code. The output is still available as the `output` attribute of the raised exception.

In the following examples, we assume that the relevant functions have already been imported from the `subprocess` module.

Replacing /bin/sh shell command substitution

```
output=$(mycmd myarg)
```

torna-se:

```
output = check_output(["mycmd", "myarg"])
```

Replacing shell pipeline

```
output=$(dmesg | grep hda)
```

torna-se:

```
p1 = Popen(["dmesg"], stdout=PIPE)
p2 = Popen(["grep", "hda"], stdin=p1.stdout, stdout=PIPE)
p1.stdout.close()  # Allow p1 to receive a SIGPIPE if p2 exits.
output = p2.communicate()[0]
```

The `p1.stdout.close()` call after starting the p2 is important in order for p1 to receive a SIGPIPE if p2 exits before p1.

Alternatively, for trusted input, the shell's own pipeline support may still be used directly:

```
output=$(dmesg | grep hda)
```

torna-se:

```
output = check_output("dmesg | grep hda", shell=True)
```

Substituindo `os.system()`

```
sts = os.system("mycmd" + " myarg")
# becomes
retcode = call("mycmd" + " myarg", shell=True)
```

Notas:

- Calling the program through the shell is usually not required.

- The `call()` return value is encoded differently to that of `os.system()`.
- The `os.system()` function ignores SIGINT and SIGQUIT signals while the command is running, but the caller must do this separately when using the **`subprocess`** module.

Um exemplo mais realista ficaria assim:

```
try:
    retcode = call("mycmd" + " myarg", shell=True)
    if retcode < 0:
        print("Child was terminated by signal", -retcode, file=s
    else:
        print("Child returned", retcode, file=sys.stderr)
except OSError as e:
    print("Execution failed:", e, file=sys.stderr)
```

Replacing the `os.spawn` family

Exemplo P_NOWAIT:

```
pid = os.spawnlp(os.P_NOWAIT, "/bin/mycmd", "mycmd", "myarg")
==>
pid = Popen(["/bin/mycmd", "myarg"]).pid
```

Exemplo P_WAIT:

```
retcode = os.spawnlp(os.P_WAIT, "/bin/mycmd", "mycmd", "myarg")
==>
retcode = call(["/bin/mycmd", "myarg"])
```

Exemplo de vetor:

```
os.spawnvp(os.P_NOWAIT, path, args)
==>
Popen([path] + args[1:])
```

Exemplo de ambiente:

```
os.spawnlpe(os.P_NOWAIT, "/bin/mycmd", "mycmd", "myarg", env)
==>
```

```
Popen(["/bin/mycmd", "myarg"], env={"PATH": "/usr/bin"})
```

Replacing os.popen()

Return code handling translates as follows:

```
pipe = os.popen(cmd, 'w')
...
rc = pipe.close()
if rc is not None and rc >> 8:
    print("There were some errors")
==>
process = Popen(cmd, stdin=PIPE)
...
process.stdin.close()
if process.wait() != 0:
    print("There were some errors")
```

Legacy Shell Invocation Functions

This module also provides the following legacy functions from the 2.x `commands` module. These operations implicitly invoke the system shell and none of the guarantees described above regarding security and exception handling consistency are valid for these functions.

subprocess.getstatusoutput(*cmd*, *, *encoding=None*, *errors=None*)

Return `(exitcode, output)` of executing *cmd* in a shell.

Execute the string *cmd* in a shell with `check_output()` and return a 2-tuple `(exitcode, output)`. *encoding* and *errors* are used to decode output; see the notes on Argumentos usados frequentemente for more details.

A trailing newline is stripped from the output. The exit code for the command can be interpreted as the return code of subprocess. Example:

```
>>> subprocess.getstatusoutput('ls /bin/ls')
(0, '/bin/ls')
>>> subprocess.getstatusoutput('cat /bin/junk')
```

```
(1, 'cat: /bin/junk: No such file or directory')
>>> subprocess.getstatusoutput('/bin/junk')
(127, 'sh: /bin/junk: not found')
>>> subprocess.getstatusoutput('/bin/kill $$')
(-15, '')
```

Disponibilidade: Unix, Windows.

Alterado na versão 3.3.4: Suporte ao Windows foi adicionado.

The function now returns (exitcode, output) instead of (status, output) as it did in Python 3.3.3 and earlier. exitcode has the same value as **`returncode`**.

Alterado na versão 3.11: Adicionados os parâmetros *encoding* e *errors*.

subprocess.getoutput(*cmd*, ***, *encoding=None*, *errors=None*)

Return output (stdout and stderr) of executing *cmd* in a shell.

Like **`getstatusoutput()`**, except the exit code is ignored and the return value is a string containing the command's output. Example:

```
>>> subprocess.getoutput('ls /bin/ls')
'/bin/ls'
```

Disponibilidade: Unix, Windows.

Alterado na versão 3.3.4: Suporte para Windows adicionado.

Alterado na versão 3.11: Adicionados os parâmetros *encoding* e *errors*.

Notas

Timeout Behavior

When using the `timeout` parameter in functions like **`run()`**, **`Popen.wait()`**, or **`Popen.communicate()`**, users should be aware of the following behaviors:

1. **Process Creation Delay**: The initial process creation itself cannot be interrupted on many platform APIs. This means that even when specifying a timeout, you are not guaranteed to see a timeout exception until at least after however long process creation takes.
2. **Extremely Small Timeout Values**: Setting very small timeout values (such as a few milliseconds) may result in almost immediate `TimeoutExpired` exceptions because process creation and system scheduling inherently require time.

Converter uma sequência de argumentos em uma string no Windows

On Windows, an *args* sequence is converted to a string that can be parsed using the following rules (which correspond to the rules used by the MS C runtime):

1. Arguments are delimited by white space, which is either a space or a tab.
2. A string surrounded by double quotation marks is interpreted as a single argument, regardless of white space contained within. A quoted string can be embedded in an argument.
3. A double quotation mark preceded by a backslash is interpreted as a literal double quotation mark.
4. Backslashes are interpreted literally, unless they immediately precede a double quotation mark.
5. If backslashes immediately precede a double quotation mark, every pair of backslashes is interpreted as a literal backslash. If the number of backslashes is odd, the last backslash escapes the next double quotation mark as described in rule 3.

Ver também

`shlex`
Module which provides function to parse and escape command lines.

Disable use of `posix_spawn()`

On Linux, **subprocess** defaults to using the `vfork()` system call internally when it is safe to do so rather than `fork()`. This greatly improves performance.

```
subprocess._USE_POSIX_SPAWN = False  # See CPython issue gh-NNNN
```

It is safe to set this to false on any Python version. It will have no effect on older or newer versions where unsupported. Do not assume the attribute is available to read. Despite the name, a true value does not indicate the corresponding function will be used, only that it may be.

Please file issues any time you have to use these private knobs with a way to reproduce the issue you were seeing. Link to that issue from a comment in your code.

Adicionado na versão 3.8: `_USE_POSIX_SPAWN`

`sched` — Agendador de eventos

Código-fonte: Lib/sched.py [https://github.com/python/cpython/tree/3.15/Lib/sched.py]

O módulo **`sched`** define uma classe que implementa um agendador de eventos de uso geral:

class sched.scheduler(*timefunc=time.monotonic*, *delayfunc=time.sleep*)

A classe **`scheduler`** define uma interface genérica para agendar eventos. Ela precisa de duas funções para lidar com o "mundo externo" — *timefunc* deve ser chamável sem argumentos e retornar um número (o "tempo", em quaisquer unidades). A função *delayfunc* deve ser chamável com um argumento, compatível com a saída de *timefunc*, e deve atrasar a mesma quantidade de unidades de tempo. *delayfunc* também será chamada com o argumento `0` após a execução de cada evento para permitir que outras threads executem em aplicações multithread.

Alterado na versão 3.3: Os parâmetros *timefunc* e *delayfunc* são opcionais.

Alterado na versão 3.3: A classe **`scheduler`** pode ser usada com segurança em ambientes multithread.

Exemplo:

```
>>> import sched, time
>>> s = sched.scheduler(time.time, time.sleep)
>>> def print_time(a='default'):
...     print("From print_time", time.time(), a)
...
>>> def print_some_times():
...     print(time.time())
...     s.enter(10, 1, print_time)
...     s.enter(5, 2, print_time, argument=('positional',))
```

```
...     # apesar de ter prioridade mais alta, 'keyword' é execut
...     s.enter(5, 1, print_time, kwargs={'a': 'keyword'})
...     s.enterabs(1_650_000_000, 10, print_time, argument=("fir
...     s.enterabs(1_650_000_000, 5, print_time, argument=("secc
...     s.run()
...     print(time.time())
...
>>> print_some_times()
1652342830.3640375
From print_time 1652342830.3642538 second enterabs
From print_time 1652342830.3643398 first enterabs
From print_time 1652342835.3694863 positional
From print_time 1652342835.3696074 keyword
From print_time 1652342840.369612 default
1652342840.3697174
```

Objetos Scheduler

As instâncias **`scheduler`** têm os seguintes métodos e atributos:

scheduler.enterabs(*time*, *priority*, *action*, *argument=()*, *kwargs={}*)

Agenda um novo evento. O argumento *time* deve ser um tipo numérico compatível com o valor de retorno da função *timefunc* passada ao construtor. Eventos agendados para o mesmo *time* serão executados na ordem de sua prioridade *priority*. Um número menor representa uma prioridade maior.

Executa o evento significa executar `action(*argument, **kwargs)`. *argument* é uma sequência que contém os argumentos posicionais para *action*. *kwargs* é um dicionário que contém os argumentos nomeados para *action*.

O valor de retorno é um evento que pode ser usado para cancelamento posterior do evento (veja **`cancel()`**).

Alterado na versão 3.3: o parâmetro *argument* é opcional.

Alterado na versão 3.3: o parâmetro *kwargs* foi adicionado.

scheduler.enter(*delay*, *priority*, *action*, *argument=()*, *kwargs={}*)

Agenda um evento para atrasar *delay* mais unidades de tempo. Além do tempo relativo, os outros argumentos, o efeito e o valor de retorno são os mesmos de `enterabs()`.

Alterado na versão 3.3: o parâmetro *argument* é opcional.

Alterado na versão 3.3: o parâmetro *kwargs* foi adicionado.

scheduler.cancel(*event*)

Remove o evento da fila. Se *event* não for um evento atualmente na fila, este método levantará uma `ValueError`.

scheduler.empty()

Retorna `True` se a fila de eventos estiver vazia.

scheduler.run(*blocking=True*)

Executa todos os eventos agendados. Este método aguardará (usando a função *delayfunc* passada ao construtor) o próximo evento, depois o executará e assim por diante até que não haja mais eventos agendados.

Se *blocking* for falso, executa imediatamente todos os eventos na fila que tenham um valor de tempo menor ou igual ao valor atual de *timefunc* (se houver) e retorna a diferença entre o valor atual de *timefunc* e o valor de tempo do próximo evento agendado na fila de eventos do agendador. Se a fila estiver vazia, retorna `None`.

Tanto *action* quanto *delayfunc* podem levantar uma exceção. Em ambos os casos, o agendador manterá um estado consistente e propagará a exceção. Se uma exceção for gerada por *action*, o evento não será tentado em chamadas futuras para `run()`.

Se uma sequência de eventos demorar mais para ser executada do que o tempo disponível antes do próximo evento, o agendador simplesmente ficará para trás. Nenhum evento será descartado; o código de chamada é responsável por cancelar eventos que não são mais pertinentes.

Alterado na versão 3.3: O argumento *blocking* foi adicionado.

scheduler.queue

Atributo somente leitura que retorna uma lista de eventos futuros na ordem em que serão executados. Cada evento é exibido como uma tupla nomeada com os seguintes campos: time, priority, action, argument, kwargs.

queue — Uma classe de fila sincronizada

Código-fonte: Lib/queue.py [https://github.com/python/cpython/tree/3.15/Lib/queue.py]

O módulo **queue** implementa filas multi-produtor e multi-consumidor. É especialmente útil em programação com threads quando a informação deve ser trocada de forma segura entre múltiplas threads. A classe **Queue** neste módulo implementa toda a semântica de trava necessária.

O módulo implementa três tipos de fila, que diferem apenas na ordem em que os itens são recuperados. Em uma fila FIFO, as primeiras tarefas adicionadas são as primeiras a ser recuperadas. Em uma fila LIFO, o item adicionado mais recentemente é o primeiro a ser recuperado (funcionando como uma pilha). Em uma fila de prioridade, os itens são mantidos ordenados (utilizando o módulo **heapq**) e o item de menor valor é recuperado primeiro.

Internamente, esses três tipos de filas utilizam travas para bloquear temporariamente threads concorrentes; no entanto, não foram projetados para lidar com a reentrância dentro de uma mesma thread.

Além disso, o módulo implementa um tipo “simples” de fila FIFO, **SimpleQueue**, cuja implementação específica oferece garantias adicionais em troca de uma funcionalidade mais reduzida.

The **queue** module defines the following classes and exceptions:

class queue.Queue(*maxsize=0*)

Constructor for a FIFO queue. *maxsize* is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue

items are consumed. If *maxsize* is less than or equal to zero, the queue size is infinite.

class queue.LifoQueue(*maxsize=0*)

Constructor for a LIFO queue. *maxsize* is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If *maxsize* is less than or equal to zero, the queue size is infinite.

class queue.PriorityQueue(*maxsize=0*)

Constructor for a priority queue. *maxsize* is an integer that sets the upperbound limit on the number of items that can be placed in the queue. Insertion will block once this size has been reached, until queue items are consumed. If *maxsize* is less than or equal to zero, the queue size is infinite.

The lowest valued entries are retrieved first (the lowest valued entry is the one that would be returned by `min(entries)`). A typical pattern for entries is a tuple in the form: `(priority_number, data)`.

If the *data* elements are not comparable, the data can be wrapped in a class that ignores the data item and only compares the priority number:

```
from dataclasses import dataclass, field
from typing import Any

@dataclass(order=True)
class PrioritizedItem:
    priority: int
    item: Any=field(compare=False)
```

class queue.SimpleQueue

Constructor for an unbounded FIFO queue. Simple queues lack advanced functionality such as task tracking.

Simple queues are generic over the type of their items.

Adicionado na versão 3.7.

exception queue.Empty

Exception raised when non-blocking `get()` (or `get_nowait()`) is called on a `Queue` object which is empty.

exception queue.Full

Exception raised when non-blocking `put()` (or `put_nowait()`) is called on a `Queue` object which is full.

exception queue.ShutDown

Exception raised when `put()` or `get()` is called on a `Queue` object which has been shut down.

Adicionado na versão 3.13.

Objetos Queue

Queue objects (`Queue`, `LifoQueue`, or `PriorityQueue`) provide the public methods described below.

Queue.qsize()

Return the approximate size of the queue. Note, qsize() > 0 doesn't guarantee that a subsequent get() will not block, nor will qsize() < maxsize guarantee that put() will not block.

Queue.empty()

Return `True` if the queue is empty, `False` otherwise. If empty() returns `True` it doesn't guarantee that a subsequent call to put() will not block. Similarly, if empty() returns `False` it doesn't guarantee that a subsequent call to get() will not block.

Queue.full()

Return `True` if the queue is full, `False` otherwise. If full() returns `True` it doesn't guarantee that a subsequent call to get() will not block.

Similarly, if full() returns `False` it doesn't guarantee that a subsequent call to put() will not block.

Queue.put(*item*, *block=True*, *timeout=None*)

Put *item* into the queue. If optional args *block* is true and *timeout* is `None` (the default), block if necessary until a free slot is available. If *timeout* is a positive number, it blocks at most *timeout* seconds and raises the **`Full`** exception if no free slot was available within that time. Otherwise (*block* is false), put an item on the queue if a free slot is immediately available, else raise the **`Full`** exception (*timeout* is ignored in that case).

Raises **`ShutDown`** if the queue has been shut down.

Queue.put_nowait(*item*)

Equivalent to `put(item, block=False)`.

Queue.get(*block=True*, *timeout=None*)

Remove and return an item from the queue. If optional args *block* is true and *timeout* is `None` (the default), block if necessary until an item is available. If *timeout* is a positive number, it blocks at most *timeout* seconds and raises the **`Empty`** exception if no item was available within that time. Otherwise (*block* is false), return an item if one is immediately available, else raise the **`Empty`** exception (*timeout* is ignored in that case).

Prior to 3.0 on POSIX systems, and for all versions on Windows, if *block* is true and *timeout* is `None`, this operation goes into an uninterruptible wait on an underlying lock. This means that no exceptions can occur, and in particular a SIGINT will not trigger a **`KeyboardInterrupt`**.

Raises **`ShutDown`** if the queue has been shut down and is empty, or if the queue has been shut down immediately.

Queue.get_nowait()

Equivalente a `get(False)`.

Two methods are offered to support tracking whether enqueued tasks have been fully processed by daemon consumer threads.

Queue.task_done()

Indicate that a formerly enqueued task is complete. Used by queue consumer threads. For each `get()` used to fetch a task, a subsequent call to `task_done()` tells the queue that the processing on the task is complete.

If a `join()` is currently blocking, it will resume when all items have been processed (meaning that a `task_done()` call was received for every item that had been `put()` into the queue).

Levanta `ValueError` se for chamado mais vezes do que o número de itens colocados na fila.

Queue.join()

Blocks until all items in the queue have been gotten and processed.

The count of unfinished tasks goes up whenever an item is added to the queue. The count goes down whenever a consumer thread calls `task_done()` to indicate that the item was retrieved and all work on it is complete. When the count of unfinished tasks drops to zero, **`join()`** unblocks.

Waiting for task completion

Example of how to wait for enqueued tasks to be completed:

```
import threading
import queue

q = queue.Queue()

def worker():
    while True:
        item = q.get()
        print(f'Working on {item}')
```

```
        print(f'Finished {item}')
        q.task_done()

# Turn-on the worker thread.
threading.Thread(target=worker, daemon=True).start()

# Send thirty task requests to the worker.
for item in range(30):
    q.put(item)

# Block until all tasks are done.
q.join()
print('All work completed')
```

Terminating queues

When no longer needed, **Queue** objects can be wound down until empty or terminated immediately with a hard shutdown.

Queue.shutdown(*immediate=False*)

Coloca uma instância de **Queue** em modo de desligamento.

The queue can no longer grow. Future calls to **put()** raise **ShutDown**. Currently blocked callers of **put()** will be unblocked and will raise **ShutDown** in the formerly blocked thread.

Se *immediate* for falso (o padrão), a fila pode ser encerrada normalmente com chamadas **get()** para extrair tarefas que já foram carregadas.

E se **task_done()** for chamado para cada tarefa restante, um **join()** pendente será desbloqueado normalmente.

Once the queue is empty, future calls to **get()** will raise **ShutDown**.

If *immediate* is true, the queue is terminated immediately. The queue is drained to be completely empty and the count of unfinished tasks is reduced by the number of tasks drained. If unfinished tasks is zero,

callers of `join()` are unblocked. Also, blocked callers of `get()` are unblocked and will raise `ShutDown` because the queue is empty.

Tenha cuidado ao usar `join()` com a opção *immediate* definida como verdadeiro. Isso desbloqueia a junção mesmo quando nenhuma tarefa foi executada, violando a invariante usual para a junção de uma fila.

Adicionado na versão 3.13.

Objetos SimpleQueue

`SimpleQueue` objects provide the public methods described below.

SimpleQueue.qsize()

Return the approximate size of the queue. Note, qsize() > 0 doesn't guarantee that a subsequent get() will not block.

SimpleQueue.empty()

Return `True` if the queue is empty, `False` otherwise. If empty() returns `False` it doesn't guarantee that a subsequent call to get() will not block.

SimpleQueue.put(*item*, *block=True*, *timeout=None*)

Put *item* into the queue. The method never blocks and always succeeds (except for potential low-level errors such as failure to allocate memory). The optional args *block* and *timeout* are ignored and only provided for compatibility with `Queue.put()`.

Detalhes da implementação do CPython: This method has a C implementation which is reentrant. That is, a `put()` or `get()` call can be interrupted by another `put()` call in the same thread without deadlocking or corrupting internal state inside the queue. This makes it appropriate for use in destructors such as `__del__` methods or `weakref` callbacks.

SimpleQueue.put_nowait(*item*)

Equivalent to `put(item, block=False)`, provided for compatibility with **`Queue.put_nowait()`**.

SimpleQueue.get(*block=True*, *timeout=None*)

Remove and return an item from the queue. If optional args *block* is true and *timeout* is `None` (the default), block if necessary until an item is available. If *timeout* is a positive number, it blocks at most *timeout* seconds and raises the **`Empty`** exception if no item was available within that time. Otherwise (*block* is false), return an item if one is immediately available, else raise the **`Empty`** exception (*timeout* is ignored in that case).

SimpleQueue.get_nowait()

Equivalente a `get(False)`.

Ver também

Class `multiprocessing.Queue`

A queue class for use in a multi-processing (rather than multi-threading) context.

`collections.deque` is an alternative implementation of unbounded queues with fast atomic **`append()`** and **`popleft()`** operations that do not require locking and also support indexing.

`contextvars` — Variáveis de contexto

Este módulo fornece APIs para gerenciar, armazenar e acessar o estado local do contexto. A classe `ContextVar` é usada para declarar e trabalhar com *Variáveis de Contexto*. A função `copy_context()` e a classe `Context` devem ser usadas para gerenciar o contexto atual em frameworks assíncronos.

Os gerenciadores de contexto que possuem estado devem usar Variáveis de Contexto ao invés de `threading.local()` para evitar que seu estado vaze para outro código inesperadamente, quando usado em código concorrente.

Veja também a **PEP 567** [https://peps.python.org/pep-0567/] para detalhes adicionais.

Adicionado na versão 3.7.

Variáveis de contexto

class contextvars.ContextVar(*name*[, *, *default*])

Esta classe é usada para declarar uma nova variável de contexto, como, por exemplo:

```
var: ContextVar[int] = ContextVar('var', default=42)
```

O parâmetro obrigatório *name* é usado para fins de introspecção e depuração.

O parâmetro somente-nomeado opcional *default* é retornado por `ContextVar.get()` quando nenhum valor para a variável é encontrado no contexto atual.

Importante: Variáveis de Contexto devem ser criadas no nível do módulo superior e nunca em fechamentos. Os objetos `Context` contêm referências fortes a variáveis de contexto que evitam que as variáveis de contexto sejam coletadas como lixo corretamente.

`ContextVar`s are generic over the type of their contained value.

name

O nome da variável. Esta é uma propriedade somente leitura.

Adicionado na versão 3.7.1.

get([*default*])

Retorna um valor para a variável de contexto para o contexto atual.

Se não houver valor para a variável no contexto atual, o método vai:

- retornar o valor do argumento *default* do método, se fornecido; ou
- retornar o valor padrão para a variável de contexto, se ela foi criada com uma; ou
- levantar uma `LookupError`.

set(*value*)

Chame para definir um novo valor para a variável de contexto no contexto atual.

O argumento *value* obrigatório é o novo valor para a variável de contexto.

Retorna um objeto `Token` que pode ser usado para restaurar a variável ao seu valor anterior através do método `ContextVar.reset()`.

For convenience, the token object can be used as a context manager to avoid calling `ContextVar.reset()` manually:

```
var = ContextVar('var', default='default value')with var.s
```

It is a shorthand for:

```
var = ContextVar('var', default='default value')

token = var.set('new value')
try:
    assert var.get() == 'new value'
finally:
    var.reset(token)

assert var.get() == 'default value'
```

Adicionado na versão 3.14: Added support for using tokens as context managers.

reset(*token*)

Redefine a variável de contexto para o valor que tinha antes de `ContextVar.set()`. que criou o *token*, ser usado.

Por exemplo:

```
var = ContextVar('var')

token = var.set('novo valor')
# código que usa 'var'; var.get() retorna 'novo valor'.
var.reset(token)

# Após uma chamada de redefinição, var não tem mais valor,
# então var.get() levantaria uma exceção LookupError.
```

The same *token* cannot be used twice.

class contextvars.Token

Token objects are returned by the `ContextVar.set()` method. They can be passed to the `ContextVar.reset()` method to revert the value of the variable to what it was before the corresponding *set*. A single token cannot reset a context variable more than once.

Tokens support the context manager protocol to automatically reset context variables. See `ContextVar.set()`.

Tokens are generic over the same type as the `ContextVar` which created them.

Adicionado na versão 3.14: Adicionado suporte para uso como gerenciador de contexto.

var

Uma propriedade somente leitura. Aponta para o objeto `ContextVar` que criou o token.

old_value

Uma propriedade somente leitura. Definida com o valor que a variável tinha antes da chamada do método `ContextVar.set()` que criou o token. Aponta para `Token.MISSING` se a variável não foi definida antes da chamada.

MISSING

Um objeto marcador usado por `Token.old_value`.

Gerenciamento de contexto manual

contextvars.copy_context()

Retorna uma cópia do objeto `Context` atual.

O trecho a seguir obtém uma cópia do contexto atual e imprime todas as variáveis e seus valores que estão definidos nele:

```
ctx: Context = copy_context()
print(list(ctx.items()))
```

A função tem uma complexidade $O(1)$, ou seja, funciona igualmente rápida para contextos com algumas variáveis de contexto e para contextos que têm muitas delas.

class contextvars.Context

Um mapeamento de `ContextVars` para seus valores.

`Context()` cria um contexto vazio sem valores nele. Para obter uma cópia do contexto atual, use a função `copy_context()`.

Cada thread tem sua própria pilha efetiva de objetos **`Context`**. O contexto atual é o objeto **`Context`** no topo da pilha da thread atual. Todos os objetos **`Context`** nas pilhas são considerados como *inseridos*.

Inserir um contexto, o que pode ser feito chamando seu método `run()`, torna o contexto o contexto atual, colocando-o no topo da pilha de contexto da thread atual.

Sair do contexto atual, o que pode ser feito retornando do retorno de chamada passado para o método `run()`, restaura o contexto atual para o que era antes de o contexto ser inserido, retirando o contexto do topo da pilha de contextos.

Como cada thread tem sua própria pilha de contexto, os objetos `ContextVar` se comportam de maneira semelhante a `threading.local()` quando valores são atribuídos em threads diferentes.

Tentar entrar em um contexto já inserido, incluindo contextos inseridos em outras threads, levanta uma exceção `RuntimeError`.

Após sair de um contexto, ele pode ser acessado novamente (de qualquer thread).

Quaisquer alterações nos valores de `ContextVar` por meio do método `ContextVar.set()` são registradas no contexto atual. O método `ContextVar.get()` retorna o valor associado ao contexto atual. Sair de um contexto efetivamente reverte quaisquer alterações feitas nas variáveis de contexto enquanto o contexto foi inserido (se necessário, os valores podem ser restaurados ao inserir novamente o contexto).

Context implementa a interface `collections.abc.Mapping`.

run(*callable*, **args*, ***kwargs*)

Entra no Context, executa `callable(*args, **kwargs)` e sai do Context. Retorna o valor de retorno de *callable* ou propaga uma exceção, se ocorrer uma.

Exemplo:

```
import contextvars

var = contextvars.ContextVar('var')
var.set('spam')
print(var.get())  # 'spam'

ctx = contextvars.copy_context()

def main():
    # 'var' foi definida para 'spam' antes de
    # chamar 'copy_context()' e 'ctx.run(main)', então:
    print(var.get())  # 'spam'
    print(ctx[var])  # 'spam'

    var.set('ham')

    # Agora, após definir 'var' para 'ham':
    print(var.get())  # 'ham'
    print(ctx[var])  # 'ham'

# Qualquer alteração que a função 'main' feitas a 'var'
# serão contidos em 'ctx'.
ctx.run(main)

# A função 'main()' era executada no contexto 'ctx',
# então alterações a 'var' são contidas nele:
print(ctx[var])  # 'ham'

# No entanto, fora de 'ctx', 'var' ainda está definida par
print(var.get())  # 'spam'
```

copy()

Retorna uma cópia rasa do objeto contexto.

var in context

Retorna `True` se *context* tem uma variável para *var* definida; do contrário, retorna `False`.

context[var]

Retorna o valor da variável **`ContextVar`** *var*. Se a variável não for definida no objeto contexto, uma **`KeyError`** é levantada.

get(*var*[, *default*])

Retorna o valor para *var* se *var* tiver o valor no objeto contexto. Caso contrário, retorna *default*. Se *default* não for fornecido, retorna `None`.

iter(context)

Retorna um iterador sobre as variáveis armazenadas no objeto contexto.

len(proxy)

Retorna o número das variáveis definidas no objeto contexto.

keys()

Retorna uma lista de todas as variáveis no objeto contexto.

values()

Retorna uma lista dos valores de todas as variáveis no objeto contexto.

items()

Retorna uma lista de tuplas de 2 elementos contendo todas as variáveis e seus valores no objeto contexto.

Suporte a asyncio

Variáveis de contexto encontram suporte nativo em **`asyncio`** e estão prontas para serem usadas sem qualquer configuração extra. Por exemplo, aqui está

um servidor simples de eco, que usa uma variável de contexto para disponibilizar o endereço de um cliente remoto na Task que lida com esse cliente:

```
import asyncio
import contextvars

client_addr_var = contextvars.ContextVar('client_addr')

def render_goodbye():
    # The address of the currently handled client can be accesse
    # without passing it explicitly to this function.

    client_addr = client_addr_var.get()
    return f'Good bye, client @ {client_addr}\r\n'.encode()

async def handle_request(reader, writer):
    addr = writer.transport.get_extra_info('socket').getpeernam
    client_addr_var.set(addr)

    # In any code that we call, it is now possible to get the
    # client's address by calling 'client_addr_var.get()'.

    while True:
        line = await reader.readline()
        print(line)
        if not line.strip():
            break

    writer.write(b'HTTP/1.1 200 OK\r\n')  # status line
    writer.write(b'\r\n')  # headers
    writer.write(render_goodbye())  # body
    writer.close()

async def main():
    srv = await asyncio.start_server(
        handle_request, '127.0.0.1', 8081)

    async with srv:
        await srv.serve_forever()

asyncio.run(main())
```

```
# To test it you can use telnet or curl:
#     telnet 127.0.0.1 8081
#     curl 127.0.0.1:8081
```

`_thread`— API de segmentação de baixo nível

Este módulo fornece primitivos de baixo nível para trabalhar com vários threads (também chamados *processos leves* ou *tarefas*) — vários threads de controle compartilhando seu espaço de dados global. Para sincronização, travas simples (também chamadas de *mutexes*, *exclusão mútua* ou *semáforos binários*) são fornecidas. O módulo `threading` fornece uma API de segmentação mais fácil de usar e de nível mais alto, construída sobre este módulo.

Alterado na versão 3.7: Este módulo costumava ser opcional, agora está sempre disponível.

Este módulo define as seguintes constantes e funções:

exception _thread.error

Gerado em erros específicos de segmento.

Alterado na versão 3.3: Agora este é um sinônimo da exceção embutida `RuntimeError`.

_thread.start_new_thread(*function*, *args*[, *kwargs*])

Começa uma nova thread e retorna seu identificador. A thread executa a função *function* com a lista de argumentos *args* (que deve ser uma tupla). O argumento opcional *kwargs* especifica um dicionário de argumentos nomeados.

Quando a função retorna, a thread termina silenciosamente.

Quando a função termina com uma exceção não processada, `sys.unraisablehook()` é chamada para lidar com a exceção. O atributo *object* do argumento do hook é *function*. Por padrão, um stack trace

(situação da pilha de execução) é exibido e, em seguida, a thread termina (mas outras threads continuam a ser executadas).

Quando a função gera uma exceção `SystemExit`, ela é ignorada.

Levanta um evento de auditoria `_thread.start_new_thread` com os argumentos `function`, `args`, `kwargs`.

Alterado na versão 3.8: `sys.unraisablehook()` agora é usada para tratar exceções não tratadas.

_thread.interrupt_main(*signum=signal.SIGINT*, /)

Simula o efeito de um sinal chegando na thread principal. Uma thread pode usar esta função para interromper a thread principal, embora não haja garantia de que a interrupção ocorrerá imediatamente.

Se fornecido, *signum* é o número do sinal a ser simulado. Se *signum* não for fornecido, `signal.SIGINT` será simulado.

Se o sinal fornecido não for tratado pelo Python (o sinal foi definido como `signal.SIG_DFL` ou `signal.SIG_IGN`), esta função faz nada.

Alterado na versão 3.10: O argumento *signum* é adicionado para personalizar o sinal de número.

Nota

Isso não emite o sinal correspondente, mas agenda uma chamada para o tratador associado (se existir). Se você quer realmente emitir o sinal, use `signal.raise_signal()`.

_thread.exit()

Levanta a exceção `SystemExit`. Quando não for detectada, a thread terminará silenciosamente.

_thread.allocate_lock()

Retorna um novo objeto de trava. Métodos de trava são descritos abaixo. A trava é desativada inicialmente.

_thread.get_ident()

Retorna o ‘identificador de thread’ da thread atual. Este é um número inteiro diferente de zero. Seu valor não tem significado direto; pretende-se que seja um cookie mágico para ser usado, por exemplo, para indexar um dicionário de dados específicos do thread. Identificadores de thread podem ser reciclados quando uma thread termina e outra é criada.

_thread.get_native_id()

Retorna a ID de thread integral nativa da thread atual atribuída pelo kernel. Este é um número inteiro não negativo. Seu valor pode ser usado para identificar exclusivamente essa thread específica em todo o sistema (até que a thread termine, após o que o valor poderá ser reciclado pelo sistema operacional).

Disponibilidade: Windows, FreeBSD, Linux, macOS, OpenBSD, NetBSD, AIX, DragonFlyBSD, GNU/kFreeBSD, Solaris.

Adicionado na versão 3.8.

Alterado na versão 3.13: Adicionado suporte a GNU/kFreeBSD.

Alterado na versão 3.15: Adicionado suporte para Solaris.

_thread.stack_size([*size*])

Retorna o tamanho da pilha de threads usado ao criar novas threads. O argumento opcional *size* especifica o tamanho da pilha a ser usado para threads criadas posteriormente e deve ser 0 (usar plataforma ou padrão configurado) ou um valor inteiro positivo de pelo menos 32.768 (32 KiB). Se *size* não for especificado, 0 será usado. Se a alteração do tamanho da pilha de threads não for permitida, uma `RuntimeError` será levantada. Se o tamanho da pilha especificado for inválido, um `ValueError` será levantado e o tamanho da pilha não será modificado. Atualmente, 0 KiB é o valor mínimo de tamanho de pilha suportado para garantir espaço suficiente para o próprio interpretador. Observe que

algumas plataformas podem ter restrições específicas sobre valores para o tamanho da pilha, como exigir um tamanho mínimo de pilha > 32 KiB ou exigir alocação em múltiplos do tamanho da página de memória do sistema – a documentação da plataforma deve ser consultada para obter mais informações (4 páginas KiB são comuns; usar múltiplos de 4096 para o tamanho da pilha é a abordagem sugerida na ausência de informações mais específicas).

Disponibilidade: Windows, pthreads.

Plataformas Unix com suporte a threads POSIX.

_thread.TIMEOUT_MAX

O valor máximo permitido para o parâmetro *timeout* de `Lock.acquire`. A especificação de um tempo limite maior que esse valor vai levantar um `OverflowError`.

Adicionado na versão 3.2.

class _thread.LockType

Este é o tipo de objetos de trava.

Os objetos de trava têm os seguintes métodos:

acquire(*blocking=True*, *timeout=-1*)

Sem nenhum argumento opcional, esse método adquire a trava incondicionalmente, se necessário, aguardando até que seja liberada por outra thread (apenas uma thread por vez pode adquirir uma trava — esse é o motivo da sua existência).

Se o argumento inteiro *blocking* estiver presente, a ação dependerá do seu valor: se for falso, a trava será adquirida apenas se puder ser adquirida imediatamente sem aguardar, enquanto se for verdadeiro, a trava será adquirida incondicionalmente, conforme acima.

Se o argumento de ponto flutuante *timeout* estiver presente e positivo, ele especificará o tempo máximo de espera em segundos antes de retornar. Um argumento negativo *timeout* especifica uma

espera ilimitada. Você não pode especificar um *timeout* se *blocking* for falso.

O valor de retorno é `True` se a trava for adquirida com sucesso, se não `False`.

Alterado na versão 3.2: O parâmetro *timeout* é novo.

Alterado na versão 3.2: As aquisições de trava agora podem ser interrompidas por sinais no POSIX.

Alterado na versão 3.14: As aquisições de trava agora podem ser interrompidas por sinais no Windows.

release()

Libera a trava. A trava deve ter sido adquirido anteriormente, mas não necessariamente pela mesma thread.

locked()

Retorna o status da trava: `True` se tiver sido adquirida por alguma thread, `False` se não for o caso.

Além desses métodos, os objetos de trava também podem ser usados através da instrução `with`, por exemplo:

```
import _thread

uma_trava = _thread.allocate_lock()

with uma_trava:
    print("uma_trava está travada enquanto isto executa")
```

Ressalvas:

- Interrupções sempre vão para a thread principal (a exceção `KeyboardInterrupt` será recebida por essa thread).
- Chamar `sys.exit()` ou levantar a exceção `SystemExit` é o equivalente a chamar `_thread.exit()`.

- Quando a thread principal se encerra, o fato de outras threads sobreviverem depende do sistema. Na maioria dos sistemas, elas são eliminadas sem executar cláusulas `try` … `finally` ou destruidores de objetos.

Comunicação em Rede e Interprocesso

Os módulos descritos neste capítulo fornecem mecanismos para a comunicação em rede e entre processos.

Alguns módulos funcionam apenas para dois processos que estão na mesma máquina como, por exemplo, `signal` e `mmap`. Outros módulos possuem suporte a protocolos de rede que dois ou mais processos podem usar para se comunicar entre máquinas.

A lista de módulos descritos neste capítulo é:

- `asyncio` — E/S assíncrona
- `socket` — Interface de rede de baixo nível
- `ssl` — Invólucro de TLS/SSL para objetos socket
- `select` — Waiting for I/O completion
- `selectors` — High-level I/O multiplexing
- `signal` — Define manipuladores para eventos assíncronos
- `mmap` — Suporte a arquivos mapeados na memória

`asyncio` — E/S assíncrona

---

asyncio é uma biblioteca para escrever código **simultâneo** usando a sintaxe **async/await**.

O asyncio é usado como uma base para várias estruturas assíncronas do Python que fornecem rede e servidores web de alto desempenho, bibliotecas de conexão de banco de dados, filas de tarefas distribuídas etc.

Olá Mundo!

```
import asyncio

async def main():
    print('Olá ...')
    await asyncio.sleep(1)
    print('... Mundo!')

asyncio.run(main())
```

asyncio geralmente serve perfeitamente para código de rede **estruturado** de alto nível e vinculado a E/S.

Ver também

Uma visão geral conceitual de asyncio
Explicação dos fundamentos do asyncio.

asyncio fornece um conjunto de APIs de **alto nível** para:

- executar corrotinas do Python simultaneamente e ter controle total sobre sua execução;
- realizar IPC e E/S de rede;
- controlar subprocessos;
- distribuir tarefas por meio de filas;
- sincronizar código simultâneo;

For **introspection**, asyncio provides APIs and tools for:

- inspecting the async call graph of tasks and futures;

- inspecting tasks in another running Python process with command-line tools;

Além disso, há APIs de **baixo nível** para *desenvolvedores de biblioteca e framework* para:

- criar e gerenciar laços de eventos, que fornecem APIs assíncronas para rede, execução de subprocessos, tratamento de sinais de sistemas operacionais etc;
- implementar protocolos eficientes usando transportes;
- fazer uma ponte sobre bibliotecas baseadas em chamadas e codificar com a sintaxe de async/await.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

REPL de asyncio

Você pode experimentar um contexto atual `asyncio` no REPL:

```
$ python -m asyncio
asyncio REPL ...
Use "await" directly instead of "asyncio.run()".
Type "help", "copyright", "credits" or "license" for more inform
>>> import asyncio
>>> await asyncio.sleep(10, result='hello')
'hello'
```

Este REPL oferece compatibilidade limitada com a variável de ambiente **`PYTHON_BASIC_REPL`**. Recomenda-se o uso do REPL padrão para obter todas as funcionalidades e os recursos mais recentes.

Levanta um evento de auditoria `cpython.run_stdin` sem argumentos.

Alterado na versão 3.12.5: (também 3.11.10, 3.10.15, 3.9.20 e 3.8.20) Emite eventos de auditoria.

Alterado na versão 3.13: Usa PyrePL, se possível, nesse caso `PYTHONSTARTUP` também é executado. Emite eventos de auditoria.

Referência

APIs de alto nível

- Runners
- Coroutines and tasks
- Streams
- Synchronization Primitives
- Subprocessos
- Filas
- Exceções

Introspection APIs

- Call graph introspection
- Command-line introspection tools

APIs de baixo nível

- Laço de eventos
- Futuros
- Transports e Protocols
- Políticas
- Suporte a plataformas
- Extensão

Guias e tutoriais

- Índice da API de alto nível
- Índice de APIs de baixo nível
- Desenvolvendo com asyncio
- asyncio and free-threaded Python

Nota

O código-fonte para o asyncio pode ser encontrado em Lib/asyncio/ [https://github.com/python/cpython/tree/3.15/Lib/asyncio/].

Runners

Source code: Lib/asyncio/runners.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/runners.py]

This section outlines high-level asyncio primitives to run asyncio code.

They are built on top of an event loop with the aim to simplify async code usage for common wide-spread scenarios.

- Executando um programa asyncio
- Runner context manager
- Handling Keyboard Interruption

Executando um programa asyncio

asyncio.run(*coro*, *, *debug=None*, *loop_factory=None*)

Execute *coro* in an asyncio event loop and return the result.

The argument can be any awaitable object.

This function runs the awaitable, taking care of managing the asyncio event loop, *finalizing asynchronous generators*, and closing the executor.

Esta função não pode ser chamada quando outro laço de eventos asyncio está executando na mesma thread.

If *debug* is `True`, the event loop will be run in debug mode. `False` disables debug mode explicitly. `None` is used to respect the global Modo de Depuração settings.

If *loop_factory* is not `None`, it is used to create a new event loop; otherwise `asyncio.new_event_loop()` is used. The loop is closed at the end. This function should be used as a main entry point for asyncio programs, and should ideally only be called once. It is recommended to use *loop_factory* to configure the event loop instead of policies. Passing `asyncio.EventLoop` allows running asyncio without the policy system.

The executor is given a timeout duration of 5 minutes to shutdown. If the executor hasn't finished within that duration, a warning is emitted and the executor is closed.

Exemplo:

```
async def main():
    await asyncio.sleep(1)
    print('hello')

asyncio.run(main())
```

Adicionado na versão 3.7.

Alterado na versão 3.9: Atualizado para usar `loop.shutdown_default_executor()`.

Alterado na versão 3.10: debug is `None` by default to respect the global debug mode settings.

Alterado na versão 3.12: Added *loop_factory* parameter.

Alterado na versão 3.14: coro can be any awaitable object.

Nota

The **`asyncio`** policy system is deprecated and will be removed in Python 3.16; from there on, an explicit *loop_factory* is needed to configure the event loop.

Runner context manager

class asyncio.Runner(*, *debug=None*, *loop_factory=None*)

A context manager that simplifies *multiple* async function calls in the same context.

Sometimes several top-level async functions should be called in the same event loop and `contextvars.Context`.

If *debug* is `True`, the event loop will be run in debug mode. `False` disables debug mode explicitly. `None` is used to respect the global Modo de Depuração settings.

loop_factory could be used for overriding the loop creation. It is the responsibility of the *loop_factory* to set the created loop as the current one. By default `asyncio.new_event_loop()` is used and set as current event loop with `asyncio.set_event_loop()` if *loop_factory* is `None`.

Basically, `asyncio.run()` example can be rewritten with the runner usage:

```
async def main():
    await asyncio.sleep(1)
    print('hello')

with asyncio.Runner() as runner:
    runner.run(main())
```

Adicionado na versão 3.11.

run(*coro*, *, *context=None*)

Execute *coro* in the embedded event loop.

The argument can be any awaitable object.

If the argument is a coroutine, it is wrapped in a Task.

An optional keyword-only *context* argument allows specifying a custom `contextvars.Context` for the code to run in. The runner's default context is used if context is `None`.

Returns the awaitable's result or raises an exception.

Esta função não pode ser chamada quando outro laço de eventos asyncio está executando na mesma thread.

Alterado na versão 3.14: coro can be any awaitable object.

close()

Close the runner.

Finalize asynchronous generators, shutdown default executor, close the event loop and release embedded `contextvars.Context`.

get_loop()

Return the event loop associated with the runner instance.

Nota

`Runner` uses the lazy initialization strategy, its constructor doesn't initialize underlying low-level structures.

Embedded *loop* and *context* are created at the `with` body entering or the first call of `run()` or `get_loop()`.

Handling Keyboard Interruption

Adicionado na versão 3.11.

When `signal.SIGINT` is raised by `Ctrl-C`, `KeyboardInterrupt` exception is raised in the main thread by default. However this doesn't work with `asyncio` because it can interrupt asyncio internals and can hang the program from exiting.

To mitigate this issue, `asyncio` handles `signal.SIGINT` as follows:

1. `asyncio.Runner.run()` installs a custom `signal.SIGINT` handler before any user code is executed and removes it when exiting from the

function.

2. The `Runner` creates the main task for the passed coroutine for its execution.
3. When `signal.SIGINT` is raised by `Ctrl-C`, the custom signal handler cancels the main task by calling `asyncio.Task.cancel()` which raises `asyncio.CancelledError` inside the main task. This causes the Python stack to unwind, `try/except` and `try/finally` blocks can be used for resource cleanup. After the main task is cancelled, `asyncio.Runner.run()` raises `KeyboardInterrupt`.
4. A user could write a tight loop which cannot be interrupted by `asyncio.Task.cancel()`, in which case the second following `Ctrl-C` immediately raises the `KeyboardInterrupt` without cancelling the main task.

Coroutines and tasks

Esta seção descreve APIs assíncronas de alto nível para trabalhar com corrotinas e tarefas.

- Corrotinas
- Aguardáveis
- Creating tasks
- Task cancellation
- Task groups
- Dormindo
- Running tasks concurrently
- Eager task factory
- Shielding from cancellation
- Tempo limite
- Waiting primitives
- Running in threads
- Scheduling from other threads
- Introspecção
- Task object

Corrotinas

Código-fonte: Lib/asyncio/coroutines.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/coroutines.py]

Corrotinas declaradas com a sintaxe async/await é a forma preferida de escrever aplicações assíncronas. Por exemplo, o seguinte trecho de código imprime “hello”, espera 1 segundo, e então imprime “world”:

```
>>> import asyncio
```

```
>>> async def main():
...     print('hello')
...     await asyncio.sleep(1)
...     print('world')

>>> asyncio.run(main())
hello
world
```

Perceba que simplesmente chamar uma corrotina não irá agendá-la para ser executada:

```
>>> main()
<coroutine object main at 0x1053bb7c8>
```

To actually run a coroutine, asyncio provides the following mechanisms:

- A função **`asyncio.run()`** para executar a função “main()” do ponto de entrada no nível mais alto (veja o exemplo acima.)

- Aguardando uma corrotina. O seguinte trecho de código exibirá “hello” após esperar por 1 segundo e, em seguida, exibirá “world” após esperar por *outros* 2 segundos:

  ```
  import asyncio
  import time

  async def say_after(delay, what):
      await asyncio.sleep(delay)
      print(what)

  async def main():
      print(f"started at {time.strftime('%X')}")

      await say_after(1, 'hello')
      await say_after(2, 'world')

      print(f"finished at {time.strftime('%X')}")

  asyncio.run(main())
  ```

Resultado esperado:

```
started at 17:13:52
hello
world
finished at 17:13:55
```

- A função **`asyncio.create_task()`** para executar corrotinas concorrentemente como **`Tasks`** asyncio.

  Vamos modificar o exemplo acima e executar duas corrotinas `say_after` *concorrentemente*:

```
async def main():
    task1 = asyncio.create_task(
        say_after(1, 'hello'))

    task2 = asyncio.create_task(
        say_after(2, 'world'))

    print(f"started at {time.strftime('%X')}")

    # Wait until both tasks are completed (should take
    # around 2 seconds.)
    await task1
    await task2

    print(f"finished at {time.strftime('%X')}")
```

  Perceba que a saída esperada agora mostra que o trecho de código é executado 1 segundo mais rápido do que antes:

```
started at 17:14:32
hello
world
finished at 17:14:34
```

- The **`asyncio.TaskGroup`** class provides a more modern alternative to **`create_task()`**. Using this API, the last example becomes:

```
    async def main():
        async with asyncio.TaskGroup() as tg:
            task1 = tg.create_task(
                say_after(1, 'hello'))

            task2 = tg.create_task(
                say_after(2, 'world'))

            print(f"started at {time.strftime('%X')}")

        # The await is implicit when the context manager exits.

        print(f"finished at {time.strftime('%X')}")
```

The timing and output should be the same as for the previous version.

Adicionado na versão 3.11: `asyncio.TaskGroup`.

Aguardáveis

Dizemos que um objeto é um objeto **aguardável** se ele pode ser usado em uma expressão `await`. Muitas APIs asyncio são projetadas para aceitar aguardáveis.

Existem três tipos principais de objetos *aguardáveis*: **corrotinas**, **Tarefas**, e **Futuros**.

Corrotinas

Corrotinas Python são *aguardáveis* e portanto podem ser aguardadas a partir de outras corrotinas:

```
import asyncio

async def nested():
    return 42

async def main():
    # Nothing happens if we just call "nested()".
    # A coroutine object is created but not awaited,
```

```
    # so it *won't run at all*.
    nested()  # will raise a "RuntimeWarning".

    # Let's do it differently now and await it:
    print(await nested())  # will print "42".

asyncio.run(main())
```

Importante

Nesta documentação, o termo “corrotina” pode ser usado para dois conceitos intimamente relacionados:

- uma *função de corrotina*: uma função `async def`;
- um *objeto de corrotina*: um objeto retornado ao chamar uma *função de corrotina*.

Tarefas

Tarefas são usadas para agendar corrotinas *concorrentemente*.

Quando uma corrotina é envolta em uma *tarefa* com funções como `asyncio.create_task()`, a corrotina é automaticamente agendada para executar em breve:

```
import asyncio

async def nested():
    return 42

async def main():
    # Schedule nested() to run soon concurrently
    # with "main()".
    task = asyncio.create_task(nested())

    # "task" can now be used to cancel "nested()", or
    # can simply be awaited to wait until it is complete:
    await task
```

```
asyncio.run(main())
```

Futuros

Um `Future` é um objeto aguardável especial de **baixo nível** que representa um **resultado eventual** de uma operação assíncrona.

Quando um objeto Future é *aguardado* isso significa que a corrotina irá esperar até que o Future seja resolvido em algum outro local.

Objetos Future em asyncio são necessários para permitir que código baseado em função de retorno seja utilizado com async/await.

Normalmente **não existe necessidade** em criar objetos Future no nível de código da aplicação.

Objetos Future, algumas vezes expostos por bibliotecas e algumas APIs asyncio, podem ser aguardados:

```
async def main():
    await function_that_returns_a_future_object()

    # this is also valid:
    await asyncio.gather(
        function_that_returns_a_future_object(),
        some_python_coroutine()
    )
```

Um bom exemplo de uma função de baixo nível que retorna um objeto Future é `loop.run_in_executor()`.

Creating tasks

Source code: Lib/asyncio/tasks.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/tasks.py]

---

asyncio.create_task(*coro*, *, *name=None*, *context=None*, *eager_start=None*, ***kwargs*)

Envolve a corrotina *coro* em uma `Task` e agenda sua execução. Retorna o objeto Task.

A assinatura completa da função é, em grande parte, a mesma que a do construtor (ou fábrica) de `Task` — todos os argumentos nomeados desta função são passados para essa interface.

Um argumento opcional somente-nomeado *context* permite especificar um `contextvars.Context` personalizado para a *coro* ser executada. Uma cópia do contexto atual é criada quando nenhum *context* for fornecido.

Um argumento opcional somente-nomeado *eager_start* permite especificar se a tarefa deve ser executada imediatamente durante a chamada para create_task ou ser agendada para mais tarde. Se *eager_start* não for fornecido, o modo definido por `loop.set_task_factory()` será utilizado.

A tarefa é executada no laço e retornada por `get_running_loop()`, `RuntimeError` é levantado se não existir um laço em execução na thread atual.

Nota

`asyncio.TaskGroup.create_task()` é uma nova alternativa que aproveita a concorrência estruturada e permite aguardar um grupo de tarefas relacionadas, com fortes garantias de segurança.

Importante

Mantenha uma referência para o resultado dessa função, evitando que uma tarefa desapareça durante a execução. O laço de eventos mantém apenas referências fracas para as tarefas. Uma tarefa que não é referenciada por nada mais pode ser removida pelo coletor de lixo a qualquer momento, antes mesmo da função ser finalizada. Para tarefas de segundo plano “atire-e-esqueça”, junte-as em uma coleção:

```
background_tasks = set()for i in range(10):    task = async
```

Note that this approach never awaits the tasks, so if a task fails, its exception is never retrieved and asyncio logs a “Task exception was never retrieved” message when the task is garbage collected. To avoid this, use `asyncio.TaskGroup` which keeps a strong reference to each task, awaits them and propagates their exceptions:

```
async with asyncio.TaskGroup() as tg:
    for i in range(10):
        tg.create_task(some_coro(param=i))
```

Adicionado na versão 3.7.

Alterado na versão 3.8: Adicionado o parâmetro *name*.

Alterado na versão 3.11: Adicionado o parâmetro *context*.

Alterado na versão 3.14: Adicionado o parâmetro *eager_start*, passando todos os *kwargs*.

Task cancellation

Tasks can easily and safely be cancelled. When a task is cancelled, `asyncio.CancelledError` will be raised in the task at the next opportunity.

It is recommended that coroutines use `try/finally` blocks to robustly perform clean-up logic. In case `asyncio.CancelledError` is explicitly caught, it should generally be propagated when clean-up is complete. **`asyncio.CancelledError`** directly subclasses `BaseException` so most code will not need to be aware of it.

The asyncio components that enable structured concurrency, like `asyncio.TaskGroup` and `asyncio.timeout()`, are implemented using cancellation internally and might misbehave if a coroutine swallows `asyncio.CancelledError`. Similarly, user code should not generally call `uncancel`. However, in cases when suppressing **`asyncio.CancelledError`**

is truly desired, it is necessary to also call `uncancel()` to completely remove the cancellation state.

Task groups

Grupos de tarefas combinam uma API de criação de tarefas com uma forma conveniente e confiável de aguardar que todas as tarefas do grupo terminem.

class asyncio.TaskGroup

Um gerenciador de contexto assíncrono que contém um grupo de tarefas. As tarefas podem ser adicionadas ao grupo usando `create_task()`. Todas as tarefas são aguardadas quando o gerenciador de contexto sai.

Adicionado na versão 3.11.

create_task(*coro*, **, name=None, context=None, eager_start=None, **kwargs*)

Create a task in this task group. The signature matches that of `asyncio.create_task()`. If the task group is inactive (e.g. not yet entered, already finished, or in the process of shutting down), we will close the given `coro` and raise `RuntimeError`.

Alterado na versão 3.13: Fecha a corrotina dada se o grupo de tarefas não estiver ativo.

Alterado na versão 3.14: Passa todos os *kwargs* para `loop.create_task()`

cancel()

Cancel the task group. This is a non-exceptional, early exit of the task group's lifetime – useful once the group's goal has been met or its services no longer needed.

`cancel()` will be called on any tasks in the group that aren't yet done, as well as the parent (body) of the group. The task group

context manager will exit *without* **asyncio.CancelledError** being raised.

If **cancel()** is called before entering the task group, the group will be cancelled upon entry. This is useful for patterns where one piece of code passes an unused **asyncio.TaskGroup** instance to another in order to have the ability to cancel anything run within the group.

cancel() is idempotent and may be called after the task group has already exited.

Some ways to use **cancel()**:

- call it from the task group body based on some condition or event
- pass the task group instance to child tasks via **create_task()**, allowing a child task to conditionally cancel the entire group
- pass the task group instance or bound **cancel()** method to some other task *before* opening the task group, allowing remote cancellation

Adicionado na versão 3.15.

Exemplo:

```
async def main():    async with asyncio.TaskGroup() as tg:
```

The `async with` statement will wait for all tasks in the group to finish. While waiting, new tasks may still be added to the group (for example, by passing `tg` into one of the coroutines and calling `tg.create_task()` in that coroutine). There is also opportunity to request termination of the entire task group with `tg.cancel()`, based on some condition. Once the last task has finished and the `async with` block is exited, no new tasks may be added to the group.

Na primeira vez que qualquer uma das tarefas do grupo falha com uma exceção diferente de **asyncio.CancelledError**, as tarefas restantes do grupo são canceladas. Não podem ser adicionadas mais tarefas ao grupo.

Neste ponto, se o corpo da instrução `async with` ainda estiver ativo (ou seja, `__aexit__()` ainda não foi chamado), a tarefa que contém diretamente a instrução `async with` também é cancelada. A **`asyncio.CancelledError`** resultante irá interromper um `await`, mas não se propagará para fora da instrução `async with`.

Uma vez que todas as tarefas tenham terminado, se alguma tarefa falhou com uma exceção diferente de `asyncio.CancelledError`, essas exceções são combinadas em uma `ExceptionGroup` ou `BaseExceptionGroup` (conforme apropriado; veja a sua documentação), que é então levantada.

Duas exceções base são tratadas de forma especial: se alguma tarefa falhar com `KeyboardInterrupt` ou `SystemExit`, o grupo de tarefas ainda cancela as tarefas restantes e aguarda por elas, mas depois a **`KeyboardInterrupt`** ou **`SystemExit`** inicial é levantada novamente em vez de `ExceptionGroup` ou `BaseExceptionGroup`.

If the body of the `async with` statement exits with an exception (so `__aexit__()` is called with an exception set), this is treated the same as if one of the tasks failed: the remaining tasks are cancelled and then waited for, and non-cancellation exceptions are grouped into an exception group and raised. The exception passed into **`__aexit__()`**, unless it is `asyncio.CancelledError`, is also included in the exception group. The same special case is made for `KeyboardInterrupt` and `SystemExit` as in the previous paragraph. There is an additional special case made only for the body of the `async with`: if it raises `GeneratorExit` and none of the other tasks raise exceptions that would be reported, then the **`GeneratorExit`** is reraised.

Os grupos de tarefas tomam cuidado para não confundir o cancelamento interno, usado para “acordar” o seu `__aexit__()`, com pedidos de cancelamento para a tarefa na qual estão a ser executados, feitos por outras partes. Em particular, quando um grupo de tarefas está sintaticamente aninhado noutro, e ambos experimentam uma exceção numa das suas tarefas filhas simultaneamente, o grupo de tarefas interno processará as suas exceções, e depois o grupo de tarefas externo receberá outro cancelamento e processará as suas próprias exceções.

No caso de um grupo de tarefas ser cancelado externamente e também precisar levantar uma `ExceptionGroup`, chamará o método `cancel()` da tarefa pai. Isto garante que um `asyncio.CancelledError` será levantada no próximo `await`, para que o cancelamento não se perca.

Os grupos de tarefas preservam a contagem de cancelamentos reportada por `asyncio.Task.cancelling()`.

Alterado na versão 3.13: Melhoria no tratamento de cancelamentos internos e externos simultâneos e na preservação correta das contagens de cancelamentos.

Alterado na versão 3.15: Addition of the special case for `GeneratorExit`.

Dormindo

async asyncio.sleep(*delay*, *result=None*)

Bloqueia por *delay* segundos.

Se *result* é fornecido, é retornado para o autor da chamada quando a corrotina termina.

`sleep()` sempre suspende a tarefa atual, permitindo que outras tarefas sejam executadas.

Configurando o delay para 0 fornece um caminho otimizado para permitir que outras tarefas executem. Isto pode ser usado por funções de longa execução para evitar que bloqueiem o laço de eventos por toda a duração da chamada da função.

Exemplo de uma corrotina exibindo a data atual a cada segundo durante 5 segundos:

```
import asyncio
import datetime as dt

async def display_date():
    loop = asyncio.get_running_loop()
```

```
    end_time = loop.time() + 5.0
    while True:
        print(dt.datetime.now())
        if (loop.time() + 1.0) >= end_time:
            break
        await asyncio.sleep(1)

asyncio.run(display_date())
```

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.13: Raises `ValueError` if *delay* is `nan`.

Running tasks concurrently

awaitable asyncio.gather(**aws*, *return_exceptions=False*)

Executa objetos aguardáveis na sequência *aws* de forma *concorrente*.

Se qualquer aguardável em *aws* é uma corrotina, ele é automaticamente agendado como uma Tarefa.

Se todos os aguardáveis forem concluídos com sucesso, o resultado é uma lista agregada de valores retornados. A ordem dos valores resultantes corresponde a ordem dos aguardáveis em *aws*.

Se *return_exceptions* for `False` (valor padrão), a primeira exceção levantada é imediatamente propagada para a tarefa que espera em `gather()`. Outros aguardáveis na sequência *aws* **não serão cancelados** e irão continuar a executar.

Se *return_exceptions* for `True`, exceções são tratadas da mesma forma que resultados com sucesso, e agregadas na lista de resultados.

Se `gather()` for *cancelado*, todos os aguardáveis que foram submetidos (que não foram concluídos ainda) também são *cancelados*.

Se qualquer Tarefa ou Futuro da sequência *aws* for *cancelado*, ele é tratado como se tivesse levantado `CancelledError` – a chamada para `gather()` **não** é cancelada neste caso. Isso existe para prevenir que o

cancelamento de uma Tarefa/Futuro submetida ocasione outras Tarefas/Futuros a serem cancelados.

Nota

A new alternative to create and run tasks concurrently and wait for their completion is `asyncio.TaskGroup`. *TaskGroup* provides stronger safety guarantees than *gather* for scheduling a nesting of subtasks: if a task (or a subtask, a task scheduled by a task) raises an exception, *TaskGroup* will, while *gather* will not, cancel the remaining scheduled tasks.

Exemplo:

```
import asyncio

async def factorial(name, number):
    f = 1
    for i in range(2, number + 1):
        print(f"Task {name}: Compute factorial({number}), cu
        await asyncio.sleep(1)
        f *= i
    print(f"Task {name}: factorial({number}) = {f}")
    return f

async def main():
    # Schedule three calls *concurrently*:
    L = await asyncio.gather(
        factorial("A", 2),
        factorial("B", 3),
        factorial("C", 4),
    )
    print(L)

asyncio.run(main())

# Expected output:
#
#     Task A: Compute factorial(2), currently i=2...
#     Task B: Compute factorial(3), currently i=2...
#     Task C: Compute factorial(4), currently i=2...
```

```
#     Task A: factorial(2) = 2
#     Task B: Compute factorial(3), currently i=3...
#     Task C: Compute factorial(4), currently i=3...
#     Task B: factorial(3) = 6
#     Task C: Compute factorial(4), currently i=4...
#     Task C: factorial(4) = 24
#     [2, 6, 24]
```

Nota

If *return_exceptions* is false, cancelling gather() after it has been marked done won't cancel any submitted awaitables. For instance, gather can be marked done after propagating an exception to the caller, therefore, calling `gather.cancel()` after catching an exception (raised by one of the awaitables) from gather won't cancel any other awaitables.

Alterado na versão 3.7: Se *gather* por si mesmo for cancelado, o cancelamento é propagado independente de *return_exceptions*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se nenhum argumento posicional for fornecido, ou nem todos os argumentos posicionais são objetos similar a Futuro, e não existe nenhum laço de eventos em execução.

Eager task factory

asyncio.eager_task_factory(*loop*, *coro*, ***, *name=None*, *context=None*)

A task factory for eager task execution.

When using this factory (via `loop.set_task_factory(asyncio.eager_task_factory)`), coroutines begin execution synchronously during `Task` construction. Tasks are only scheduled on the event loop if they block. This can be a performance

improvement as the overhead of loop scheduling is avoided for coroutines that complete synchronously.

A common example where this is beneficial is coroutines which employ caching or memoization to avoid actual I/O when possible.

Nota

Immediate execution of the coroutine is a semantic change. If the coroutine returns or raises, the task is never scheduled to the event loop. If the coroutine execution blocks, the task is scheduled to the event loop. This change may introduce behavior changes to existing applications. For example, the application's task execution order is likely to change.

Adicionado na versão 3.12.

asyncio.create_eager_task_factory(*custom_task_constructor*)

Create an eager task factory, similar to `eager_task_factory()`, using the provided *custom_task_constructor* when creating a new task instead of the default `Task`.

custom_task_constructor must be a *callable* with the signature matching the signature of `Task.__init__`. The callable must return a `asyncio.Task`-compatible object.

This function returns a *callable* intended to be used as a task factory of an event loop via `loop.set_task_factory(factory)`).

Adicionado na versão 3.12.

Shielding from cancellation

awaitable asyncio.shield(*aw*)

Protege um objeto aguardável de ser `cancelado`.

Se *aw* é uma corrotina, ela é automaticamente agendada como uma Tarefa.

A instrução:

```
task = asyncio.create_task(something())
res = await shield(task)
```

é equivalente a:

```
res = await something()
```

exceto que se a corrotina contendo-a for cancelada, a Tarefa executando em `something()` não é cancelada. Do ponto de vista de `something()`, o cancelamento não aconteceu. Apesar do autor da chamada ainda estar cancelado, então a expressão "await" ainda levanta um **`CancelledError`**.

Se `something()` é cancelada por outros meios (isto é, dentro ou a partir de si mesma) isso também iria cancelar `shield()`.

Se for desejado ignorar completamente os cancelamentos (não recomendado) a função `shield()` deve ser combinada com uma cláusula try/except, conforme abaixo:

```
task = asyncio.create_task(something())
try:
    res = await shield(task)
except CancelledError:
    res = None
```

Importante

Mantenha uma referência para as tarefas passadas para essa função função, evitando assim que uma tarefa desapareça durante a execução. O laço de eventos mantém apenas referências fracas para as tarefas. Uma tarefa que não é referenciada por nada mais pode ser removida pelo coletor de lixo a qualquer momento, antes mesmo da função ser finalizada.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se *aw* não é um objeto similar a Futuro, e não existe nenhum laço de eventos em execução.

Tempo limite

asyncio.timeout(*delay*)

Return an asynchronous context manager that can be used to limit the amount of time spent waiting on something.

delay can either be `None`, or a float/int number of seconds to wait. If *delay* is `None`, no time limit will be applied; this can be useful if the delay is unknown when the context manager is created.

In either case, the context manager can be rescheduled after creation using **`Timeout.reschedule()`**.

Exemplo:

```
async def main():
    async with asyncio.timeout(10):
        await long_running_task()
```

If `long_running_task` takes more than 10 seconds to complete, the context manager will cancel the current task and handle the resulting **`asyncio.CancelledError`** internally, transforming it into a **`TimeoutError`** which can be caught and handled.

Nota

The **`asyncio.timeout()`** context manager is what transforms the **`asyncio.CancelledError`** into a **`TimeoutError`**, which means the **`TimeoutError`** can only be caught *outside* of the context manager.

Example of catching **`TimeoutError`**:

```
async def main():
    try:
        async with asyncio.timeout(10):
            await long_running_task()
    except TimeoutError:
        print("The long operation timed out, but we've handle

    print("This statement will run regardless.")
```

The context manager produced by **`asyncio.timeout()`** can be rescheduled to a different deadline and inspected.

class asyncio.Timeout(*when*)

An asynchronous context manager for cancelling overdue coroutines.

Prefer using **`asyncio.timeout()`** or **`asyncio.timeout_at()`** rather than instantiating **`Timeout`** directly.

`when` should be an absolute time at which the context should time out, as measured by the event loop's clock:

- If `when` is `None`, the timeout will never trigger.
- If `when < loop.time()`, the timeout will trigger on the next iteration of the event loop.

when() → float | None

Return the current deadline, or `None` if the current deadline is not set.

reschedule(*when: float | None*)

Reschedule the timeout.

expired() → bool

Return whether the context manager has exceeded its deadline (expired).

Exemplo:

```
async def main():
    try:
        # We do not know the timeout when starting, so we pas
        async with asyncio.timeout(None) as cm:
            # We know the timeout now, so we reschedule it.
            new_deadline = get_running_loop().time() + 10
            cm.reschedule(new_deadline)

            await long_running_task()
    except TimeoutError:
        pass

    if cm.expired():
        print("Looks like we haven't finished on time.")
```

Timeout context managers can be safely nested.

Adicionado na versão 3.11.

asyncio.timeout_at(*when*)

Similar to `asyncio.timeout()`, except *when* is the absolute time to stop waiting, or `None`.

Exemplo:

```
async def main():
    loop = get_running_loop()
    deadline = loop.time() + 20
    try:
        async with asyncio.timeout_at(deadline):
            await long_running_task()
    except TimeoutError:
        print("The long operation timed out, but we've handle

    print("This statement will run regardless.")
```

Adicionado na versão 3.11.

async asyncio.wait_for(*aw*, *timeout*)

Espera o aguardável *aw* concluir sem ultrapassar o tempo limite “timeout”.

timeout pode ser `None`, ou um ponto flutuante, ou um número inteiro de segundos para aguardar. Se *timeout* é `None`, aguarda até o future encerrar.

If a timeout occurs, it cancels *aw* and raises **`TimeoutError`**.

To prevent *aw* from being cancelled, wrap it in **`shield()`**.

A função irá aguardar até o future ser realmente cancelado, então o tempo total de espera pode exceder o tempo limite *timeout*. Se uma exceção ocorrer durante o cancelamento, ela será propagada.

Se ele for cancelado, o future *aw* também é cancelado.

Exemplo:

```
async def eternity():
    # Sleep for one hour
    await asyncio.sleep(3600)
    print('yay!')

async def main():
    # Wait for at most 1 second
    try:
        await asyncio.wait_for(eternity(), timeout=1.0)
    except TimeoutError:
        print('timeout!')

asyncio.run(main())

# Expected output:
#
#     timeout!
```

Alterado na versão 3.7: When *aw* is cancelled due to a timeout, `wait_for` waits for *aw* to be cancelled. Previously, it raised **`TimeoutError`** immediately.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Raises `TimeoutError` instead of `asyncio.TimeoutError`.

Alterado na versão 3.12: Implemented using `asyncio.timeout()`, a coroutine passed as *aw* is no longer wrapped in a `Task` when *timeout* is positive.

Waiting primitives

async asyncio.wait(*aws*, *, *timeout=None*, *return_when=ALL_COMPLETED*)

Run `Future` and `Task` instances in the *aws* iterable concurrently and block until the condition specified by *return_when*.

O iterável *aws* não deve ser vazio.

Retorna dois conjuntos de Tarefas/Futuros: `(done, pending)`.

Uso:

```
done, pending = await asyncio.wait(aws)
```

timeout (um ponto flutuante ou inteiro), se especificado, pode ser usado para controlar o número máximo de segundos para aguardar antes de retornar.

Note that this function does not raise `TimeoutError`. Futures or Tasks that aren't done when the timeout occurs are simply returned in the second set.

return_when indica quando esta função deve retornar. Ele deve ser uma das seguintes constantes:

Constante	Descrição

Constante	Descrição
asyncio.FIRST_COMPLETED	A função irá retornar quando qualquer futuro terminar ou for cancelado.
asyncio.FIRST_EXCEPTION	The function will return when any future finishes by raising an exception. If no future raises an exception then it is equivalent to `ALL_COMPLETED`.
asyncio.ALL_COMPLETED	A função irá retornar quando todos os futuros encerrarem ou forem cancelados.

Diferente de `wait_for()`, `wait()` não cancela os futuros quando um tempo limite é atingido.

If `wait()` is cancelled, the futures in *aws* are not cancelled and continue to run.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Passing coroutine objects to `wait()` directly is forbidden.

Alterado na versão 3.12: Added support for generators yielding tasks.

asyncio.as_completed(*aws*, *, *timeout=None*)

Run awaitable objects in the *aws* iterable concurrently. The returned object can be iterated to obtain the results of the awaitables as they finish.

The object returned by `as_completed()` can be iterated as an asynchronous iterator or a plain iterator. When asynchronous iteration is used, the originally-supplied awaitables are yielded if they are tasks or

futures. This makes it easy to correlate previously-scheduled tasks with their results. Example:

```
ipv4_connect = create_task(open_connection("127.0.0.1", 80))
ipv6_connect = create_task(open_connection("::1", 80))
tasks = [ipv4_connect, ipv6_connect]

async for earliest_connect in as_completed(tasks):
    # earliest_connect is done. The result can be obtained by
    # awaiting it or calling earliest_connect.result()
    reader, writer = await earliest_connect

    if earliest_connect is ipv6_connect:
        print("IPv6 connection established.")
    else:
        print("IPv4 connection established.")
```

During asynchronous iteration, implicitly-created tasks will be yielded for supplied awaitables that aren't tasks or futures.

When used as a plain iterator, each iteration yields a new coroutine that returns the result or raises the exception of the next completed awaitable. This pattern is compatible with Python versions older than 3.13:

```
ipv4_connect = create_task(open_connection("127.0.0.1", 80))
ipv6_connect = create_task(open_connection("::1", 80))
tasks = [ipv4_connect, ipv6_connect]

for next_connect in as_completed(tasks):
    # next_connect is not one of the original task objects. I
    # awaited to obtain the result value or raise the excepti
    # awaitable that finishes next.
    reader, writer = await next_connect
```

A `TimeoutError` is raised if the timeout occurs before all awaitables are done. This is raised by the `async for` loop during asynchronous iteration or by the coroutines yielded during plain iteration.

`as_completed()` does not cancel the tasks running the supplied awaitables: if a timeout occurs or the iteration is cancelled, the remaining tasks continue to run.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se nem todos os objetos aguardáveis no iterável *aws* forem objetos similar a Futuro, e não existe nenhum laço de eventos em execução.

Alterado na versão 3.12: Added support for generators yielding tasks.

Alterado na versão 3.13: The result can now be used as either an asynchronous iterator or as a plain iterator (previously it was only a plain iterator).

Running in threads

async asyncio.to_thread(*func*, /, **args*, ***kwargs*)

Executa a função *func* assincronamente em uma thread separada.

Quaisquer *args e **kwargs fornecidos para esta função são diretamente passados para *func*. Além disso, o `contextvars.Context` atual é propagado, permitindo que variáveis de contexto da thread do laço de eventos sejam acessadas na thread separada.

Retorna uma corrotina que pode ser aguardada para obter o resultado eventual de *func*.

This coroutine function is primarily intended to be used for executing IO-bound functions/methods that would otherwise block the event loop if they were run in the main thread. For example:

```
def blocking_io():
    print(f"start blocking_io at {time.strftime('%X')}")
    # Note that time.sleep() can be replaced with any blockir
    # IO-bound operation, such as file operations.
```

```
    time.sleep(1)
    print(f"blocking_io complete at {time.strftime('%X')}")

async def main():
    print(f"started main at {time.strftime('%X')}")

    await asyncio.gather(
        asyncio.to_thread(blocking_io),
        asyncio.sleep(1))

    print(f"finished main at {time.strftime('%X')}")


asyncio.run(main())

# Expected output:
#
# started main at 19:50:53
# start blocking_io at 19:50:53
# blocking_io complete at 19:50:54
# finished main at 19:50:54
```

Chamar diretamente `blocking_io()` em qualquer corrotina iria bloquear o laço de eventos durante a sua duração, resultando em 1 segundo adicional no tempo de execução. Ao invés disso, ao utilizar `asyncio.to_thread()`, nós podemos executá-la em uma thread separada sem bloquear o laço de eventos.

Nota

Devido à GIL, `asyncio.to_thread()` pode tipicamente ser usado apenas para fazer funções vinculadas a IO não-bloqueantes. Entretanto, para módulos de extensão que liberam o GIL ou implementações alternativas do Python que não tem um, `asyncio.to_thread()` também pode ser usado para funções vinculadas à CPU.

Adicionado na versão 3.9.

Scheduling from other threads

asyncio.run_coroutine_threadsafe(*coro*, *loop*)

Envia uma corrotina para o laço de eventos fornecido. Seguro para thread.

Retorna um `concurrent.futures.Future` para aguardar pelo resultado de outra thread do sistema operacional.

Esta função destina-se a ser chamada partir de uma thread diferente do sistema operacional, da qual o laço de eventos está executando. Exemplo:

```
def in_thread(loop: asyncio.AbstractEventLoop) -> None:
    # Run some blocking IO
    pathlib.Path("example.txt").write_text("hello world", enc

    # Create a coroutine
    coro = asyncio.sleep(1, result=3)

    # Submit the coroutine to a given loop
    future = asyncio.run_coroutine_threadsafe(coro, loop)

    # Wait for the result with an optional timeout argument
    assert future.result(timeout=2) == 3

async def amain() -> None:
    # Get the running loop
    loop = asyncio.get_running_loop()

    # Run something in a thread
    await asyncio.to_thread(in_thread, loop)
```

It’s also possible to run the other way around. Example:

```
@contextlib.contextmanager
def loop_in_thread() -> Generator[asyncio.AbstractEventLoop]:
    loop_fut = concurrent.futures.Future[asyncio.AbstractEver
    stop_event = asyncio.Event()
```

```
    async def main() -> None:
        loop_fut.set_result(asyncio.get_running_loop())
        await stop_event.wait()

    with concurrent.futures.ThreadPoolExecutor(1) as tpe:
        complete_fut = tpe.submit(asyncio.run, main())
        for fut in concurrent.futures.as_completed((loop_fut,
            if fut is loop_fut:
                loop = loop_fut.result()
                try:
                    yield loop
                finally:
                    loop.call_soon_threadsafe(stop_event.set)
            else:
                fut.result()

# Create a loop in another thread
with loop_in_thread() as loop:
    # Create a coroutine
    coro = asyncio.sleep(1, result=3)

    # Submit the coroutine to a given loop
    future = asyncio.run_coroutine_threadsafe(coro, loop)

    # Wait for the result with an optional timeout argument
    assert future.result(timeout=2) == 3
```

Se uma exceção for levantada na corrotina, o Futuro retornado será notificado. Isso também pode ser usado para cancelar a tarefa no laço de eventos:

```
try:
    result = future.result(timeout)
except TimeoutError:
    print('The coroutine took too long, cancelling the task..
    future.cancel()
except Exception as exc:
    print(f'The coroutine raised an exception: {exc!r}')
else:
    print(f'The coroutine returned: {result!r}')
```

Veja a seção concorrência e multithreading da documentação.

Ao contrário de outras funções asyncio, esta função requer que o argumento *loop* seja passado explicitamente.

Adicionado na versão 3.5.1.

Introspecção

asyncio.current_task(*loop=None*)

Retorna a instância `Task` atualmente em execução, ou `None` se nenhuma tarefa estiver executando.

Se *loop* for `None`, então `get_running_loop()` é usado para obter o laço atual.

Adicionado na versão 3.7.

asyncio.all_tasks(*loop=None*)

Retorna um conjunto de objetos `Task` ainda não concluídos a serem executados pelo laço.

Se *loop* for `None`, então `get_running_loop()` é usado para obter o laço atual.

Adicionado na versão 3.7.

asyncio.iscoroutine(*obj*)

Return `True` if *obj* is a coroutine object.

Adicionado na versão 3.4.

Task object

class asyncio.Task(*coro*, ***, *loop=None*, *name=None*, *context=None*, *eager_start=False*)

Um objeto `similar a Futuro` que executa uma corrotina Python. Não é seguro para thread.

Tarefas são usadas para executar corrotinas em laços de eventos. Se uma corrotina espera por um Futuro, a Tarefa suspende a execução da corrotina e aguarda a conclusão do Futuro. Quando o Futuro é *concluído*, a execução da corrotina contida é retomada.

Laços de eventos usam agendamento cooperativo: um ciclo de evento executa uma Task de cada vez. Enquanto uma Task aguarda a conclusão de um Future, o laço de eventos executa outras Tasks, funções de retorno, ou executa operações de IO.

Use a função de alto nível `asyncio.create_task()` para criar Tasks, ou as funções de baixo nível `loop.create_task()` ou `ensure_future()`. Instanciação manual de Tasks é desencorajada.

To cancel a running Task use the `cancel()` method. Calling it will cause the Task to throw a `CancelledError` exception into the wrapped coroutine. If a coroutine is awaiting on a future-like object during cancellation, the awaited object will be cancelled.

`cancelled()` pode ser usado para verificar se a Task foi cancelada. O método retorna `True` se a corrotina envolvente não suprimiu a exceção `CancelledError` e, na verdade, foi cancelada.

`asyncio.Task` herda de `Future` todas as suas APIs exceto `Future.set_result()` e `Future.set_exception()`.

An optional keyword-only *context* argument allows specifying a custom `contextvars.Context` for the *coro* to run in. If no *context* is provided, the Task copies the current context and later runs its coroutine in the copied context.

Um argumento opcional somente-nomeado *eager_start* permite iniciar imediatamente a execução da `asyncio.Task` no momento da criação da tarefa. Se definido como `True` e o laço de eventos estiver em execução, a tarefa começará a executar a corrotina imediatamente, até à primeira

vez que a corrotina bloquear. Se a corrotina devolver ou levantar uma exceção sem bloquear, a tarefa será concluída imediatamente e não será agendada para o laço de eventos.

Tasks are generic over the return type of their wrapped coroutines.

Alterado na versão 3.7: Adicionado suporte ao módulo `contextvars`.

Alterado na versão 3.8: Adicionado o parâmetro *name*.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se *loop* não for especificado e não existir nenhum laço de eventos em execução.

Alterado na versão 3.11: Adicionado o parâmetro *context*.

Alterado na versão 3.12: Adicionado o parâmetro *eager_start*.

done()

Retorna `True` se a Tarefa estiver *concluída*.

Uma Tarefa está *concluída* quando a corrotina contida retornou um valor, ou levantou uma exceção, ou a Tarefa foi cancelada.

result()

Retorna o resultado da Tarefa.

Se a Tarefa estiver *concluída*, o resultado da corrotina contida é retornado (ou se a corrotina levantou uma exceção, essa exceção é re-levantada.)

Se a Tarefa foi *cancelada*, este método levanta uma exceção `CancelledError`.

If the Task's result isn't yet available, this method raises an `InvalidStateError` exception.

exception()

Retorna a exceção de uma Tarefa.

Se a corrotina contida levantou uma exceção, essa exceção é retornada. Se a corrotina contida retornou normalmente, este método retorna `None`.

Se a Tarefa foi *cancelada*, este método levanta uma exceção `CancelledError`.

Se a Tarefa não estiver *concluída* ainda, este método levanta uma exceção `InvalidStateError`.

add_done_callback(*callback*, *, *context=None*)

Adiciona uma função de retorno para ser executada quando a Tarefa estiver *concluída*.

Este método deve ser usado apenas em código de baixo nível baseado em funções de retorno.

Veja a documentação para `Future.add_done_callback()` para mais detalhes.

remove_done_callback(*callback*)

Remove *callback* da lista de funções de retorno.

Este método deve ser usado apenas em código de baixo nível baseado em funções de retorno.

Veja a documentação do método `Future.remove_done_callback()` para mais detalhes.

get_stack(*, *limit=None*)

Retorna a lista de frames da pilha para esta Tarefa.

Se a corrotina contida não estiver concluída, isto retorna a pilha onde ela foi suspensa. Se a corrotina foi concluída com sucesso ou cancelada, isto retorna uma lista vazia. Se a corrotina foi

interrompida por uma exceção, isso retorna a lista de frames do traceback (situação da pilha de execução).

Os quadros são sempre ordenados dos mais antigos para os mais recentes.

Apenas um frame da pilha é retornado para uma corrotina suspensa.

O argumento opcional *limit* define o número máximo de frames a serem retornados; por padrão, todos os frames disponíveis são retornados. O ordenamento da lista retornada é diferente dependendo se uma pilha ou um traceback (situação da pilha de execução) é retornado: os frames mais recentes de uma pilha são retornados, mas os frames mais antigos de um traceback são retornados. (Isso combina com o comportamento do módulo traceback.)

print_stack(*, *limit=None*, *file=None*)

Exibe a pilha ou a situação da pilha de execução desta Task.

Isto produz uma saída similar à do módulo traceback para frames recuperados por `get_stack()`.

O argumento *limit* é passado diretamente para `get_stack()`.

O argumento *file* é um fluxo de I/O para o qual a saída é escrita; por padrão, a saída é escrita para `sys.stdout`.

get_coro()

Retorna o objeto corrotina contido pela `Task`.

Nota

Isto retornará `None` para Tasks que já tenham sido concluídas ansiosamente. Consulte fábrica de tarefas ansiosas.

Adicionado na versão 3.8.

Alterado na versão 3.12: A execução de tarefas recentemente adicionadas, ansiosas, pode resultar em `None`.

get_context()

Retorna o objeto `contextvars.Context` associado à tarefa.

Adicionado na versão 3.12.

get_name()

Retorna o nome da Tarefa.

Se nenhum nome foi explicitamente designado para a Tarefa, a implementação padrão asyncio da classe Task gera um nome padrão durante a instanciação.

Adicionado na versão 3.8.

set_name(*value*)

Define o nome da Tarefa.

O argumento *value* pode ser qualquer objeto, que é então convertido em uma string.

Na implementação padrão da Tarefa, o nome será visível na `repr()` de saída de um objeto task.

Adicionado na versão 3.8.

cancel(*msg=None*)

Solicita o cancelamento da Tarefa.

If the Task is already *done* or *cancelled*, return `False`, otherwise, return `True`.

O método prepara para que uma exceção `CancelledError` seja lançada na corrotina contida no próximo ciclo do laço de eventos.

The coroutine then has a chance to clean up or even deny the request by suppressing the exception with a `try` … … `except` `CancelledError` … `finally` block. Therefore, unlike `Future.cancel()`, **`Task.cancel()`** does not guarantee that the Task will be cancelled, although suppressing cancellation completely is not common and is actively discouraged. Should the coroutine nevertheless decide to suppress the cancellation, it needs to call `Task.uncancel()` in addition to catching the exception.

If the Task being cancelled is currently awaiting on a future-like object, that awaited object will also be cancelled. This cancellation propagates down the entire chain of awaited objects.

Alterado na versão 3.9: Adicionado o parâmetro *msg*.

Alterado na versão 3.11: O parâmetro `msg` é propagado da tarefa cancelada para o seu aguardador.

O seguinte exemplo ilustra como corrotinas podem interceptar o cancelamento de requisições:

```
async def cancel_me():    print('cancel_me(): antes de do
```

cancelled()

Retorna `True` se a Tarefa for *cancelada*.

A Tarefa é *cancelada* quando o cancelamento foi requisitado com `cancel()` e a corrotina contida propagou a exceção `CancelledError` gerada nela.

uncancel()

Decrement the count of cancellation requests to this Task.

Retorna o número restante de pedidos de cancelamento.

Note que, uma vez concluída a execução de uma tarefa cancelada, chamadas adicionais para `uncancel()` são ineficazes.

Adicionado na versão 3.11.

This method is used by asyncio's internals and isn't expected to be used by end-user code. In particular, if a Task gets successfully uncancelled, this allows for elements of structured concurrency like Task groups and **`asyncio.timeout()`** to continue running, isolating cancellation to the respective structured block. For example:

```
async def make_request_with_timeout():    try:        asyn
```

Enquanto o bloco com `make_request()` e `make_another_request()` possa ser cancelado devido ao timeout, `unrelated_code()` deve continuar a executar mesmo em caso de timeout. Isto é implementado com **`uncancel()`**. Os gerenciadores de contexto **`TaskGroup`** usam **`uncancel()`** de forma semelhante.

Se o código do usuário final, por alguma razão, estiver suprimindo o cancelamento ao capturar **`CancelledError`**, precisa chamar este método para remover o estado de cancelamento.

Quando este método decrementa a contagem de cancelamentos para zero, verifica se uma chamada anterior a **`cancel()`** tinha preparado para que **`CancelledError`** fosse lançado na tarefa. Se ainda não tiver sido lançado, esse preparo será revogado (redefinindo o sinalizador interna `_must_cancel`).

Alterado na versão 3.13: Alterado para revogar os pedidos de cancelamento pendentes ao atingir zero.

cancelling()

Return the number of pending cancellation requests to this Task, i.e., the number of calls to **`cancel()`** less the number of **`uncancel()`** calls.

Note that if this number is greater than zero but the Task is still executing, **`cancelled()`** will still return `False`. This is because this number can be lowered by calling **`uncancel()`**, which can lead to

the task not being cancelled after all if the cancellation requests go down to zero.

Este método é usado pelos internos do asyncio e não é esperado que seja usado por código do usuário final. Consulte `uncancel()` para mais detalhes.

Adicionado na versão 3.11.

Streams

Código-fonte: Lib/asyncio/streams.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/streams.py]

Streams são conexões de rede de alto-nível assincronias/espera-pronta. Streams permitem envios e recebimentos de dados sem usar retornos de chamadas ou protocolos de baixo nível.

Aqui está um exemplo de um cliente TCP realizando eco, escrito usando streams asyncio:

```
import asyncio

async def tcp_echo_client(message):
    reader, writer = await asyncio.open_connection(
        '127.0.0.1', 8888)

    print(f'Send: {message!r}')
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(100)
    print(f'Received: {data.decode()!r}')

    print('Close the connection')
    writer.close()
    await writer.wait_closed()

asyncio.run(tcp_echo_client('Hello World!'))
```

Veja também a seção Exemplos abaixo.

Funções Stream

As seguintes funções asyncio de alto nível podem ser usadas para criar e trabalhar com streams:

async asyncio.open_connection(*host=None*, *port=None*, ***, *limit=65536*, *ssl=None*, *family=0*, *proto=0*, *flags=0*, *sock=None*, *local_addr=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *happy_eyeballs_delay=None*, *interleave=None*)

Estabelece uma conexão de rede e retorna um par de objetos `(reader, writer)`.

Os objetos *reader* e *writer* retornados são instâncias das classes **`StreamReader`** e **`StreamWriter`**.

limit determina o tamanho limite do buffer usado pela instância **`StreamReader`** retornada. Por padrão, *limit* é definido em 64 KiB.

O resto dos argumentos é passado diretamente para **`loop.create_connection()`**.

Nota

The *sock* argument transfers ownership of the socket to the **`StreamWriter`** created. To close the socket, call its **`close()`** method.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*.

Alterado na versão 3.8: Adicionados os parâmetros *happy_eyeballs_delay* e *interleave*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

async asyncio.start_server(*client_connected_cb*, *host=None*, *port=None*, ***, *limit=65536*, *family=socket.AF_UNSPEC*, *flags=socket.AI_PASSIVE*,

sock=None, *backlog=100*, *ssl=None*, *reuse_address=None*, *reuse_port=None*, *keep_alive=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*)

Inicia um soquete no servidor.

A função de retorno *client_connected_cb* é chamada sempre que uma nova conexão de um cliente é estabelecida. Ela recebe um par `(reader, writer)` como dois argumentos, instâncias das classes **`StreamReader`** e **`StreamWriter`**.

client_connected_cb pode ser simplesmente algo chamável ou uma função de corrotina; se ele for uma função de corrotina, ele será automaticamente agendado como uma **`Task`**.

limit determina o tamanho limite do buffer usado pela instância **`StreamReader`** retornada. Por padrão, *limit* é definido em 64 KiB.

O resto dos argumentos são passados diretamente para **`loop.create_server()`**.

Nota

The *sock* argument transfers ownership of the socket to the server created. To close the socket, call the server's **`close()`** method.

Alterado na versão 3.7: Added the *ssl_handshake_timeout* and *start_serving* parameters.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.13: Added the *keep_alive* parameter.

Soquetes Unix

async asyncio.open_unix_connection(*path=None*, *, *limit=65536*, *ssl=None*, *sock=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Estabelece uma conexão de soquete Unix e retorna um par com `(reader, writer)`.

Similar a `open_connection()`, mas opera em soquetes Unix.

Veja também a documentação do método `loop.create_unix_connection()`.

Nota

The *sock* argument transfers ownership of the socket to the `StreamWriter` created. To close the socket, call its `close()` method.

Disponibilidade: Unix.

Alterado na versão 3.7: Added the *ssl_handshake_timeout* parameter. The *path* parameter can now be a path-like object

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

async asyncio.start_unix_server(*client_connected_cb*, *path=None*, *, *limit=65536*, *sock=None*, *backlog=100*, *ssl=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*, *cleanup_socket=True*)

Inicia um servidor com soquete Unix.

Similar a `start_server()`, mas funciona com soquetes Unix.

If *cleanup_socket* is true then the Unix socket will automatically be removed from the filesystem when the server is closed, unless the socket has been replaced after the server has been created.

Veja também a documentação do método `loop.create_unix_server()`.

Nota

The *sock* argument transfers ownership of the socket to the server created. To close the socket, call the server's `close()` method.

Disponibilidade: Unix.

Alterado na versão 3.7: Added the *ssl_handshake_timeout* and *start_serving* parameters. The *path* parameter can now be a path-like object.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.13: Added the *cleanup_socket* parameter.

StreamReader

class asyncio.StreamReader

Represents a reader object that provides APIs to read data from the IO stream. As an asynchronous iterable, the object supports the `async for` statement.

Não é recomendado instanciar objetos *StreamReader* diretamente; use `open_connection()` e `start_server()` ao invés disso.

feed_eof()

Acknowledge the EOF.

async read(*n=-1*)

Read up to *n* bytes from the stream.

If *n* is not provided or set to `-1`, read until EOF, then return all read `bytes`. If EOF was received and the internal buffer is empty, return

an empty `bytes` object.

If *n* is `0`, return an empty `bytes` object immediately.

If *n* is positive, return at most *n* available `bytes` as soon as at least 1 byte is available in the internal buffer. If EOF is received before any byte is read, return an empty `bytes` object.

async readline()

Lê uma linha, onde “line” é uma sequência de bytes encerrando com `\n`.

Se EOF é recebido e `\n` não foi encontrado, o método retorna os dados parcialmente lidos.

Se EOF for recebido e o buffer interno estiver vazio, retorna um objeto `bytes` vazio.

async readexactly(*n*)

Lê exatamente *n* bytes.

Levanta um **`IncompleteReadError`** se EOF é atingido antes que *n* sejam lidos. Use o atributo **`IncompleteReadError.partial`** para obter os dados parcialmente lidos.

async readuntil(*separator=b'\n'*)

Lê dados a partir do stream até que *separator* seja encontrado.

Ao ter sucesso, os dados e o separador serão removidos do buffer interno (consumido). Dados retornados irão incluir o separador no final.

Se a quantidade de dados lidos excede o limite configurado para o stream, uma exceção **`LimitOverrunError`** é levantada, e os dados são deixados no buffer interno e podem ser lidos novamente.

Se EOF for atingido antes que o separador completo seja encontrado, uma exceção **`IncompleteReadError`** é levantada, e o

buffer interno é redefinido. O atributo `IncompleteReadError.partial` pode conter uma parte do separador.

The *separator* may also be a tuple of separators. In this case the return value will be the shortest possible that has any separator as the suffix. For the purposes of `LimitOverrunError`, the shortest possible separator is considered to be the one that matched.

Adicionado na versão 3.5.2.

Alterado na versão 3.13: The *separator* parameter may now be a `tuple` of separators.

at_eof()

Retorna `True` se o buffer estiver vazio e `feed_eof()` foi chamado.

StreamWriter

class asyncio.StreamWriter

Representa um objeto de escrita que fornece APIs para escrever dados para o stream de IO.

Não é recomendado instanciar objetos *StreamWriter* diretamente; use `open_connection()` e `start_server()` ao invés.

write(*data*)

O método tenta escrever *data* para o soquete subjacente imediatamente. Se isso falhar, data é enfileirado em um buffer interno de escrita, até que possa ser enviado.

The *data* buffer should be a bytes, bytearray, or C-contiguous one-dimensional memoryview object.

O método deve ser usado juntamente com o método `drain()`:

```
stream.write(data)
await stream.drain()
```

writelines(*data*)

O método escreve imediatamente a lista (ou qualquer iterável) de bytes para o soquete subjacente. Se isso falhar, os dados são enfileirados em um buffer de escrita interno até que possam ser enviados.

O método deve ser usado juntamente com o método `drain()`:

```
stream.writelines(lines)
await stream.drain()
```

close()

O método fecha o stream e o soquete subjacente.

The method should be used, though not mandatory, along with the `wait_closed()` method:

```
stream.close()
await stream.wait_closed()
```

can_write_eof()

Retorna `True` se o transporte subjacente suporta o método `write_eof()`, `False` caso contrário.

write_eof()

Fecha o extremo de escrita do stream após os dados no buffer de escrita terem sido descarregados.

transport

Retorna o transporte asyncio subjacente.

get_extra_info(*name*, *default=None*)

Acessa informações de transporte opcionais; veja `BaseTransport.get_extra_info()` para detalhes.

async drain()

Aguarda até que seja apropriado continuar escrevendo no stream. Exemplo:

```
writer.write(data)
await writer.drain()
```

Este é um método de controle de fluxo que interage com o buffer de entrada e saída de escrita subjacente. Quando o tamanho do buffer atinge a marca d'água alta, *drain()* bloqueia até que o tamanho do buffer seja drenado para a marca d'água baixa, e a escrita possa continuar. Quando não existe nada que cause uma espera, o método **`drain()`** retorna imediatamente.

Nota

When the write buffer is below the high watermark, **`drain()`** returns immediately without yielding to the event loop. As a result, code which repeatedly calls `write()` followed by `await drain()` may prevent other tasks from running. To prevent blocking behavior, yield to the event loop explicitly with `await asyncio.sleep(0)` (see **`asyncio.sleep()`**).

async start_tls(*sslcontext*, *, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Upgrade an existing stream-based connection to TLS.

Parâmetros:

- *sslcontext*: uma instância configurada de **`SSLContext`**.
- *server_hostname*: define ou substitui o nome do host no qual o servidor alvo do certificado será comparado.
- *ssl_handshake_timeout* is the time in seconds to wait for the TLS handshake to complete before aborting the connection. `60.0` seconds if `None` (default).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection.

`30.0` seconds if `None` (default).

Adicionado na versão 3.11.

Alterado na versão 3.12: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.15: Raises a `ValueError` if `sslcontext.check_hostname` is `True` and `server_hostname` is not supplied.

is_closing()

Retorna `True` se o stream estiver fechado ou em processo de ser fechado.

Adicionado na versão 3.7.

async wait_closed()

Aguarda até que o stream seja fechado.

Should be called after `close()` to wait until the underlying connection is closed, ensuring that all data has been flushed before e.g. exiting the program.

Adicionado na versão 3.7.

Exemplos

Cliente para eco TCP usando streams

Cliente de eco TCP usando a função `asyncio.open_connection()`:

```
import asyncio

async def tcp_echo_client(message):
    reader, writer = await asyncio.open_connection(
        '127.0.0.1', 8888)
```

```
    print(f'Send: {message!r}')
    writer.write(message.encode())
    await writer.drain()

    data = await reader.read(100)
    print(f'Received: {data.decode()!r}')

    print('Close the connection')
    writer.close()
    await writer.wait_closed()

asyncio.run(tcp_echo_client('Hello World!'))
```

Ver também

O exemplo de protocolo do cliente para eco TCP usa o método de baixo nível **`loop.create_connection()`**.

Servidor eco TCP usando streams

Servidor eco TCP usando a função **`asyncio.start_server()`**:

```
import asyncio

async def handle_echo(reader, writer):
    data = await reader.read(100)
    message = data.decode()
    addr = writer.get_extra_info('peername')

    print(f"Received {message!r} from {addr!r}")

    print(f"Send: {message!r}")
    writer.write(data)
    await writer.drain()

    print("Close the connection")
    writer.close()
    await writer.wait_closed()

async def main():
```

```
    server = await asyncio.start_server(
        handle_echo, '127.0.0.1', 8888)

    addrs = ', '.join(str(sock.getsockname()) for sock in server
    print(f'Serving on {addrs}')

    async with server:
        await server.serve_forever()

asyncio.run(main())
```

Ver também

O exemplo de protocolo eco de servidor TCP utiliza o método `loop.create_server()`.

Obtém headers HTTP

Exemplo simples consultando cabeçalhos HTTP da URL passada na linha de comando:

```
import asyncio
import urllib.parse
import sys

async def print_http_headers(url):
    url = urllib.parse.urlsplit(url)
    if url.scheme == 'https':
        reader, writer = await asyncio.open_connection(
            url.hostname, 443, ssl=True)
    else:
        reader, writer = await asyncio.open_connection(
            url.hostname, 80)

    query = (
        f"HEAD {url.path or '/'} HTTP/1.0\r\n"
        f"Host: {url.hostname}\r\n"
        f"\r\n"
    )
```

```
    writer.write(query.encode('latin-1'))
    while True:
        line = await reader.readline()
        if not line:
            break

        line = line.decode('latin1').rstrip()
        if line:
            print(f'HTTP header> {line}')

    # Ignore the body, close the socket
    writer.close()
    await writer.wait_closed()

url = sys.argv[1]
asyncio.run(print_http_headers(url))
```

Uso:

```
python example.py http://example.com/path/page.html
```

ou com HTTPS:

```
python example.py https://example.com/path/page.html
```

Registra um soquete aberto para aguardar por dados usando streams

Corrotina aguardando até que um soquete receba dados usando a função `open_connection()`:

```
import asyncio
import socket

async def wait_for_data():
    # Get a reference to the current event loop because
    # we want to access low-level APIs.
    loop = asyncio.get_running_loop()

    # Create a pair of connected sockets.
```

```
    rsock, wsock = socket.socketpair()

    # Register the open socket to wait for data.
    reader, writer = await asyncio.open_connection(sock=rsock)

    # Simulate the reception of data from the network
    loop.call_soon(wsock.send, 'abc'.encode())

    # Wait for data
    data = await reader.read(100)

    # Got data, we are done: close the socket
    print("Received:", data.decode())
    writer.close()
    await writer.wait_closed()

    # Close the second socket
    wsock.close()

asyncio.run(wait_for_data())
```

Ver também

O exemplo de registro de um soquete aberto para aguardar por dados usando um protocolo utiliza um protocolo de baixo nível e o método **`loop.create_connection()`**.

O exemplo para monitorar um descritor de arquivo para leitura de eventos utiliza o método de baixo nível **`loop.add_reader()`** para monitorar um descritor de arquivo.

Synchronization Primitives

Código-fonte: Lib/asyncio/locks.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/locks.py]

---

asyncio synchronization primitives are designed to be similar to those of the `threading` module with two important caveats:

- asyncio primitives are not thread-safe, therefore they should not be used for OS thread synchronization (use `threading` for that);
- methods of these synchronization primitives do not accept the *timeout* argument; use the `asyncio.wait_for()` function to perform operations with timeouts.

asyncio has the following basic synchronization primitives:

- `Lock`
- `Event`
- `Condition`
- `Semaphore`
- `BoundedSemaphore`
- `Barrier`

Lock

class asyncio.Lock

Implements a mutex lock for asyncio tasks. Not thread-safe.

An asyncio lock can be used to guarantee exclusive access to a shared resource.

The preferred way to use a Lock is an `async with` statement:

```
lock = asyncio.Lock()

# ... later
async with lock:
    # access shared state
```

which is equivalent to:

```
lock = asyncio.Lock()

# ... later
await lock.acquire()
try:
    # access shared state
finally:
    lock.release()
```

Alterado na versão 3.10: Removido o parâmetro *loop*.

async acquire()

Acquire the lock.

This method waits until the lock is *unlocked*, sets it to *locked* and returns `True`.

When more than one coroutine is blocked in **`acquire()`** waiting for the lock to be unlocked, only one coroutine eventually proceeds.

Acquiring a lock is *fair*: the coroutine that proceeds will be the first coroutine that started waiting on the lock.

release()

Release the lock.

When the lock is *locked*, reset it to *unlocked* and return.

If the lock is *unlocked*, a **`RuntimeError`** is raised.

locked()

Return `True` if the lock is *locked*.

Evento

class asyncio.Event

An event object. Not thread-safe.

An asyncio event can be used to notify multiple asyncio tasks that some event has happened.

An Event object manages an internal flag that can be set to *true* with the `set()` method and reset to *false* with the `clear()` method. The `wait()` method blocks until the flag is set to *true*. The flag is set to *false* initially.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Exemplo:

```
async def waiter(event):
    print('waiting for it ...')
    await event.wait()
    print('... got it!')

async def main():
    # Create an Event object.
    event = asyncio.Event()

    # Spawn a Task to wait until 'event' is set.
    waiter_task = asyncio.create_task(waiter(event))

    # Sleep for 1 second and set the event.
    await asyncio.sleep(1)
    event.set()

    # Wait until the waiter task is finished.
    await waiter_task

asyncio.run(main())
```

async wait()

Wait until the event is set.

If the event is set, return `True` immediately. Otherwise block until another task calls `set()`.

set()

Set the event.

All tasks waiting for event to be set will be immediately awakened.

clear()

Clear (unset) the event.

Subsequent tasks awaiting on `wait()` will now block until the `set()` method is called again.

is_set()

Return `True` if the event is set.

Condição

class asyncio.Condition(*lock=None*)

A Condition object. Not thread-safe.

An asyncio condition primitive can be used by a task to wait for some event to happen and then get exclusive access to a shared resource.

In essence, a Condition object combines the functionality of an `Event` and a `Lock`. It is possible to have multiple Condition objects share one Lock, which allows coordinating exclusive access to a shared resource between different tasks interested in particular states of that shared resource.

The optional *lock* argument must be a `Lock` object or `None`. In the latter case a new Lock object is created automatically.

Alterado na versão 3.10: Removido o parâmetro *loop*.

The preferred way to use a Condition is an `async with` statement:

```
cond = asyncio.Condition()

# ... later
async with cond:
    await cond.wait()
```

which is equivalent to:

```
cond = asyncio.Condition()

# ... later
await cond.acquire()
try:
    await cond.wait()
finally:
    cond.release()
```

async acquire()

Acquire the underlying lock.

This method waits until the underlying lock is *unlocked*, sets it to *locked* and returns `True`.

notify(*n=1*)

Wake up *n* tasks (1 by default) waiting on this condition. If fewer than *n* tasks are waiting they are all awakened.

The lock must be acquired before this method is called and released shortly after. If called with an *unlocked* lock a **`RuntimeError`** error is raised.

locked()

Return `True` if the underlying lock is acquired.

notify_all()

Wake up all tasks waiting on this condition.

This method acts like **`notify()`**, but wakes up all waiting tasks.

The lock must be acquired before this method is called and released shortly after. If called with an *unlocked* lock a `RuntimeError` error is raised.

release()

Release the underlying lock.

When invoked on an unlocked lock, a `RuntimeError` is raised.

async wait()

Wait until notified.

If the calling task has not acquired the lock when this method is called, a `RuntimeError` is raised.

This method releases the underlying lock, and then blocks until it is awakened by a `notify()` or `notify_all()` call. Once awakened, the Condition re-acquires its lock and this method returns `True`.

Note that a task *may* return from this call spuriously, which is why the caller should always re-check the state and be prepared to **`wait()`** again. For this reason, you may prefer to use `wait_for()` instead.

async wait_for(*predicate*)

Wait until a predicate becomes *true*.

The predicate must be a callable which result will be interpreted as a boolean value. The method will repeatedly `wait()` until the predicate evaluates to *true*. The final value is the return value.

Semaphore

class asyncio.Semaphore(*value=1*)

A Semaphore object. Not thread-safe.

A semaphore manages an internal counter which is decremented by each **acquire()** call and incremented by each **release()** call. The counter can never go below zero; when **acquire()** finds that it is zero, it blocks, waiting until some task calls **release()**.

The optional *value* argument gives the initial value for the internal counter (`1` by default). If the given value is less than `0` a **ValueError** is raised.

Alterado na versão 3.10: Removido o parâmetro *loop*.

The preferred way to use a Semaphore is an **async with** statement:

```
sem = asyncio.Semaphore(10)

# ... later
async with sem:
    # work with shared resource
```

which is equivalent to:

```
sem = asyncio.Semaphore(10)

# ... later
await sem.acquire()
try:
    # work with shared resource
finally:
    sem.release()
```

async acquire()

Acquire a semaphore.

If the internal counter is greater than zero, decrement it by one and return `True` immediately. If it is zero, wait until a **release()** is called and return `True`.

locked()

Returns `True` if semaphore can not be acquired immediately.

release()

Release a semaphore, incrementing the internal counter by one. Can wake up a task waiting to acquire the semaphore.

Unlike **BoundedSemaphore**, **Semaphore** allows making more `release()` calls than `acquire()` calls.

BoundedSemaphore

class asyncio.BoundedSemaphore(*value=1*)

A bounded semaphore object. Not thread-safe.

Bounded Semaphore is a version of **Semaphore** that raises a **ValueError** in **release()** if it increases the internal counter above the initial *value*.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Barrier

class asyncio.Barrier(*parties*)

A barrier object. Not thread-safe.

A barrier is a simple synchronization primitive that allows to block until *parties* number of tasks are waiting on it. Tasks can wait on the **wait()** method and would be blocked until the specified number of tasks end up waiting on **wait()**. At that point all of the waiting tasks would unblock simultaneously.

async with can be used as an alternative to awaiting on **wait()**.

The barrier can be reused any number of times.

Exemplo:

```
async def example_barrier():
   # barrier with 3 parties
   b = asyncio.Barrier(3)
```

```
    # create 2 new waiting tasks
    asyncio.create_task(b.wait())
    asyncio.create_task(b.wait())

    await asyncio.sleep(0)
    print(b)

    # The third .wait() call passes the barrier
    await b.wait()
    print(b)
    print("barrier passed")

    await asyncio.sleep(0)
    print(b)

asyncio.run(example_barrier())
```

Result of this example is:

```
<asyncio.locks.Barrier object at 0x... [filling, waiters:2/3]
<asyncio.locks.Barrier object at 0x... [draining, waiters:0/3
barrier passed
<asyncio.locks.Barrier object at 0x... [filling, waiters:0/3]
```

Adicionado na versão 3.11.

async wait()

Pass the barrier. When all the tasks party to the barrier have called this function, they are all unblocked simultaneously.

When a waiting or blocked task in the barrier is cancelled, this task exits the barrier which stays in the same state. If the state of the barrier is “filling”, the number of waiting task decreases by 1.

The return value is an integer in the range of 0 to `parties-1`, different for each task. This can be used to select a task to do some special housekeeping, e.g.:

```
...
async with barrier as position:
   if position == 0:
      # Only one task prints this
      print('End of *draining phase*')
```

This method may raise a **BrokenBarrierError** exception if the barrier is broken or reset while a task is waiting. It could raise a **CancelledError** if a task is cancelled.

async reset()

Return the barrier to the default, empty state. Any tasks waiting on it will receive the **BrokenBarrierError** exception.

If a barrier is broken it may be better to just leave it and create a new one.

async abort()

Put the barrier into a broken state. This causes any active or future calls to **wait()** to fail with the **BrokenBarrierError**. Use this for example if one of the tasks needs to abort, to avoid infinite waiting tasks.

parties

The number of tasks required to pass the barrier.

n_waiting

The number of tasks currently waiting in the barrier while filling.

broken

A boolean that is `True` if the barrier is in the broken state.

exception asyncio.BrokenBarrierError

This exception, a subclass of **RuntimeError**, is raised when the **Barrier** object is reset or broken.

Alterado na versão 3.9: Acquiring a lock using `await lock` or `yield from lock` and/or **`with`** statement (`with await lock`, `with (yield from lock)`) was removed. Use `async with lock` instead.

Subprocessos

Código-fonte: Lib/asyncio/subprocess.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/subprocess.py], Lib/asyncio/base_subprocess.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/base_subprocess.py]

Esta seção descreve APIs async/await asyncio de alto nível para criar e gerenciar subprocessos.

Aqui está um exemplo de como asyncio pode executar um comando shell e obter o seu resultado:

```
import asyncio

async def run(cmd):
    proc = await asyncio.create_subprocess_shell(
        cmd,
        stdout=asyncio.subprocess.PIPE,
        stderr=asyncio.subprocess.PIPE)

    stdout, stderr = await proc.communicate()

    print(f'[{cmd!r} exited with {proc.returncode}]')
    if stdout:
        print(f'[stdout]\n{stdout.decode()}')
    if stderr:
        print(f'[stderr]\n{stderr.decode()}')

asyncio.run(run('ls /zzz'))
```

irá exibir:

```
['ls /zzz' exited with 1]
[stderr]
ls: /zzz: No such file or directory
```

Devido ao fato que todas as funções de subprocessos asyncio são assíncronas e asyncio fornece muitas ferramentas para trabalhar com tais funções, é fácil executar e monitorar múltiplos subprocessos em paralelo. É na verdade trivial modificar o exemplo acima para executar diversos comandos simultaneamente:

```
async def main():
    await asyncio.gather(
        run('ls /zzz'),
        run('sleep 1; echo "hello"'))

asyncio.run(main())
```

Veja também a subseção Exemplos.

Criando subprocessos

async asyncio.create_subprocess_exec(*program*, **args*, *stdin=None*, *stdout=None*, *stderr=None*, *limit=65536*, ***kwds*)

Cria um subprocesso.

O argumento *limit* define o limite do buffer para os invólucros `StreamReader` para `stdout` e `stderr` (se `subprocess.PIPE` for passado para os argumentos *stdout* e *stderr*).

Retorna uma instância de `Process`.

Veja a documentação de `loop.subprocess_exec()` para outros parâmetros.

Se o objeto do processo for coletado pelo coletor de lixo enquanto o processo ainda estiver em execução, o processo filho será finalizado.

Alterado na versão 3.10: Removido o parâmetro *loop*.

async asyncio.create_subprocess_shell(*cmd*, *stdin=None*, *stdout=None*, *stderr=None*, *limit=65536*, ***kwds*)

Executa o comando *cmd* no shell.

O argumento *limit* define o limite do buffer para os invólucros `StreamReader` para `stdout` e `stderr` (se `subprocess.PIPE` for passado para os argumentos *stdout* e *stderr*).

Retorna uma instância de `Process`.

Veja a documentação de `loop.subprocess_shell()` para outros parâmetros.

Se o objeto do processo for coletado pelo coletor de lixo enquanto o processo ainda estiver em execução, o processo filho será finalizado.

Importante

É responsabilidade da aplicação garantir que todos os espaços em branco e caracteres especiais tenham aspas apropriadamente para evitar vulnerabilidades de injeção de shell [https://en.wikipedia.org/wiki/Shell_injection#Shell_injection]. A função `shlex.quote()` pode ser usada para escapar espaços em branco e caracteres especiais de shell apropriadamente em strings que serão usadas para construir comandos shell.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Nota

Subprocessos estão disponíveis para Windows se uma `ProactorEventLoop` for usada. Veja Suporte para subprocesso para Windows para detalhes.

Ver também

asyncio também tem as seguintes APIs *de baixo nível* para trabalhar com subprocessos: `loop.subprocess_exec()`, `loop.subprocess_shell()`,

`loop.connect_read_pipe()`, `loop.connect_write_pipe()`, assim como os Transportes de Subprocesso e Protocolos de Subprocesso.

Constantes

asyncio.subprocess.PIPE

Pode ser passado para os parâmetros *stdin*, *stdout* ou *stderr*.

Se *PIPE* for passado para o argumento *stdin*, o atributo `Process.stdin` irá apontar para uma instância `StreamWriter`.

Se *PIPE* for passado para os argumentos *stdout* ou *stderr*, os atributos `Process.stdout` e `Process.stderr` irão apontar para instâncias `StreamReader`.

asyncio.subprocess.STDOUT

Valor especial que pode ser usado como o argumento *stderr* e indica que a saída de erro padrão deve ser redirecionada para a saída padrão.

asyncio.subprocess.DEVNULL

Valor especial que pode ser usado como argumento *stdin*, *stdout* ou *stderr* para funções de criação de processos. Ele indica que o arquivo especial `os.devnull` será usado para o fluxo de subprocesso correspondente.

Interagindo com subprocessos

Ambas as funções `create_subprocess_exec()` e `create_subprocess_shell()` retornam instâncias da classe *Process*. *Process* é um invólucro de alto nível que permite a comunicação com subprocessos e observar eles serem completados.

class asyncio.subprocess.Process

Um objeto que envolve processos do sistema operacional criados pelas funções `create_subprocess_exec()` e `create_subprocess_shell()`.

Esta classe é projetada para ter uma API similar a classe **`subprocess.Popen`**, mas existem algumas diferenças notáveis:

- ao contrário de Popen, instâncias de Process não têm um equivalente ao método **`poll()`**;
- os métodos **`communicate()`** e **`wait()`** não têm um parâmetro *timeout*: utilize a função **`wait_for()`**;
- o método **`Process.wait()`** é assíncrono, enquanto que o método **`subprocess.Popen.wait()`** é implementado como um laço bloqueante para indicar que está ocupado;
- o parâmetro *universal_newlines* não é suportado.

Esta classe não é segura para thread.

Veja também a seção Subprocesso e Threads.

async wait()

Aguarda o processo filho encerrar.

Define e retorna o atributo **`returncode`**.

Nota

Este método pode entrar em deadlock ao usar `stdout=PIPE` ou `stderr=PIPE` e o processo filho gera tantas saídas que ele bloqueia a espera pelo encadeamento de buffer do sistema operacional para aceitar mais dados. Use o método **`communicate()`** ao usar encadeamentos para evitar essa condição.

async communicate(*input=None*)

Interage com processo:

1. envia dados para *stdin* (se *input* for diferente de `None`);
2. fecha *stdin*;
3. lê dados a partir de *stdout* e *stderr*, até que EOF (fim do arquivo) seja atingido;
4. aguarda o processo encerrar.

O argumento opcional *input* é a informação (objeto **bytes**) que será enviada para o processo filho.

Retorna uma tupla `(stdout_data, stderr_data)`.

Se qualquer exceção **BrokenPipeError** ou **ConnectionResetError** for levantada ao escrever *input* em *stdin*, a exceção é ignorada. Esta condição ocorre quando o processo encerra antes de todos os dados serem escritos em *stdin*.

Se for desejado enviar dados para o *stdin* do processo, o mesmo precisa ser criado com `stdin=PIPE`. De forma similar, para obter qualquer coisa além de `None` na tupla resultante, o processo precisa ser criado com os argumentos `stdout=PIPE` e/ou `stderr=PIPE`.

Perceba que, os dados lidos são armazenados em um buffer na memória, então não use este método se o tamanho dos dados é grande ou ilimitado.

Alterado na versão 3.12: stdin é fechado quando `input=None` é também.

send_signal(*signal*)

Envia o sinal *signal* para o processo filho.

Nota

No Windows, **SIGTERM** é um apelido para **terminate()**. `CTRL_C_EVENT` e `CTRL_BREAK_EVENT` podem ser enviados para processos iniciados com um parâmetro *creationflags*, o qual inclui `CREATE_NEW_PROCESS_GROUP`.

terminate()

Interrompe o processo filho.

Em sistemas POSIX este método envia **SIGTERM** para o processo filho.

No Windows a função **`TerminateProcess()`** da API Win32 é chamada para interromper o processo filho.

kill()

Mata o processo filho.

Em sistemas POSIX este método envia **`SIGKILL`** para o processo filho.

No Windows, este método é um atalho para **`terminate()`**.

stdin

Fluxo de entrada padrão (**`StreamWriter`**) ou `None` se o processo foi criado com `stdin=None`.

stdout

Fluxo de saída padrão (**`StreamReader`**) ou `None` se o processo foi criado com `stdout=None`.

stderr

Erro de fluxo padrão (**`StreamReader`**) ou `None` se o processo foi criado com `stderr=None`.

Aviso

Use o método **`communicate()`** ao invés de **`process.stdin.write()`**, **`await process.stdout.read()`** ou **`await process.stderr.read()`**. Isso evita deadlocks devido a fluxos pausando a leitura ou escrita, e bloqueando o processo filho.

pid

Número de identificação do processo (PID).

Perceba que para processos criados pela função **`create_subprocess_shell()`**, este atributo é o PID do console gerado.

returncode

Retorna o código do processo quando o mesmo terminar.

Um valor `None` indica que o processo ainda não terminou.

For processes created with `create_subprocess_exec()`, a negative value `-N` indicates that the child was terminated by signal `N` (POSIX only).

For processes created with `create_subprocess_shell()`, the return code reflects the exit status of the shell itself (e.g. `/bin/sh`), which may map signals to codes such as `128+N`. See the documentation of the shell (for example, the Bash manual's Exit Status) for details.

Subprocesso e Threads

Laço de eventos padrão do asyncio suporta a execução de subprocessos a partir de diferentes threads por padrão.

No Windows, subprocessos são fornecidos pela classe `ProactorEventLoop` apenas (por padrão), a classe `SelectorEventLoop` não tem suporte a subprocesso.

Perceba que implementações alternativas do laço de eventos podem ter limitações próprias; por favor, verifique a sua documentação.

Ver também

A seção Concorrência e multithreading em asyncio.

Exemplos

Um exemplo de uso da classe `Process` para controlar um subprocesso e a classe `StreamReader` para ler a partir da sua saída padrão.

O subprocesso é criado pela função `create_subprocess_exec()`:

```
import asyncio
import sys

async def get_date():
    code = 'import datetime as dt; print(dt.datetime.now())'

    # Create the subprocess; redirect the standard output
    # into a pipe.
    proc = await asyncio.create_subprocess_exec(
        sys.executable, '-c', code,
        stdout=asyncio.subprocess.PIPE)

    # Read one line of output.
    data = await proc.stdout.readline()
    line = data.decode('ascii').rstrip()

    # Wait for the subprocess exit.
    await proc.wait()
    return line

date = asyncio.run(get_date())
print(f"Current date: {date}")
```

Veja também o mesmo exemplo escrito usando APIs de baixo nível.

Filas

Código-fonte: Lib/asyncio/queues.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/queues.py]

Filas asyncio são projetadas para serem similar a classes do módulo `queue`. Apesar de filas asyncio não serem seguras para thread, elas são projetadas para serem usadas especificamente em código async/await.

Perceba que métodos de filas asyncio não possuem um parâmetro *timeout*; use a função `asyncio.wait_for()` para realizar operações de fila com um tempo limite timeout.

Veja também a seção Exemplos abaixo.

Queue

class asyncio.Queue(*maxsize=0*)

Uma fila onde o primeiro a entrar, é o primeiro a sair (FIFO - First In First Out).

Se *maxsize* for menor que ou igual a zero, o tamanho da fila é infinito. Se ele for um inteiro maior que `0`, então `await put()` bloqueia quando a fila atingir *maxsize* até que um item seja removido por `get()`.

Ao contrário da biblioteca padrão de threading `queue`, o tamanho da fila é sempre conhecido e pode ser obtido através da chamada do método `qsize()`.

Alterado na versão 3.10: Removido o parâmetro *loop*.

Esta classe não é segura para thread.

maxsize

Número de itens permitidos na fila.

empty()

Retorna `True` se a fila estiver vazia, `False` caso contrário.

full()

Retorna `True` se existem **`maxsize`** itens na fila.

Se a fila foi inicializada com `maxsize=0` (o padrão), então **`full()`** nunca retorna `True`.

async get()

Remove e retorna um item da fila. Se a fila estiver vazia, aguarda até que um item esteja disponível.

Levanta **`QueueShutDown`** se a fila foi encerrada e está vazia, ou se a fila foi encerrada imediatamente.

get_nowait()

Retorna um item se houver um imediatamente disponível, caso contrário levanta **`QueueEmpty`**.

Levanta **`QueueShutDown`** se a fila foi encerrada e está vazia.

async join()

Bloqueia até que todos os itens na fila tenham sido recebidos e processados.

A contagem de tarefas inacabadas aumenta sempre que um item é adicionado à fila. A contagem diminui sempre que uma corrotina consumidora chama **`task_done()`** para indicar que o item foi recuperado e todo o trabalho nele foi concluído. Quando a contagem de tarefas inacabadas chega a zero, **`join()`** desbloqueia.

async put(*item*)

Coloca um item na fila. Se a fila estiver cheia, aguarda até que uma posição livre esteja disponível antes de adicionar o item.

Levanta `QueueShutDown` se a fila foi encerrada.

put_nowait(*item*)

Coloca um item na fila sem bloqueá-la.

Se nenhuma posição livre estiver imediatamente disponível, levanta `QueueFull`.

Levanta `QueueShutDown` se a fila foi encerrada.

qsize()

Retorna o número de itens na fila.

shutdown(*immediate=False*)

Coloca uma instância de `Queue` em modo de desligamento.

A fila não pode mais crescer. Chamadas futuras a `put()` levantam `QueueShutDown`. Chamadores de **`put()`** atualmente bloqueados serão desbloqueados e levantarão **`QueueShutDown`** na tarefa que estava aguardando anteriormente.

Se *immediate* for falso (o padrão), a fila pode ser encerrada normalmente com chamadas `get()` para extrair tarefas que já foram carregadas.

E se `task_done()` for chamado para cada tarefa restante, um `join()` pendente será desbloqueado normalmente.

Assim que a fila estiver vazia, chamadas futuras para `get()` vão levantar `QueueShutDown`.

Se *immediate* for verdadeiro, a fila é encerrada imediatamente. A fila é esvaziada completamente e a contagem de tarefas não concluídas é reduzida pelo número de tarefas concluídas. Se o número de tarefas não concluídas for zero, as chamadas de `join()` são desbloqueadas. Além disso, as chamadas bloqueadas de `get()`

são desbloqueadas e levantarão uma exceção `QueueShutDown` porque a fila está vazia.

Tenha cuidado ao usar `join()` com a opção *immediate* definida como verdadeiro. Isso desbloqueia a junção mesmo quando nenhuma tarefa foi executada, violando a invariante usual para a junção de uma fila.

Adicionado na versão 3.13.

task_done()

Indica que o item de trabalho anteriormente enfileirado está concluído.

Usada por consumidores de fila. Para cada `get()` usado para buscar um item de trabalho, uma chamada subsequente para `task_done()` avisa à fila, que o processamento no item de trabalho está concluído.

Se um `join()` estiver sendo bloqueado no momento, ele irá continuar quando todos os itens tiverem sido processados (significando que uma chamada `task_done()` foi recebida para cada item que foi chamado o método `put()` para colocar na fila).

Levanta `ValueError` se chamada mais vezes do que a quantidade de itens existentes na fila.

Fila de prioridade

class asyncio.PriorityQueue

Uma variante de `Queue`; recupera entradas em ordem de prioridade (mais baixas primeiro).

Entradas são tipicamente tuplas no formato `(priority_number, data)`.

Filas LIFO (último a entrar, primeiro a sair)

class asyncio.LifoQueue

Uma variante de `Queue` que recupera as entradas adicionadas mais recentemente primeiro (último a entrar, primeiro a sair).

Exceções

exception asyncio.QueueEmpty

Esta exceção é levantada quando o método `get_nowait()` é chamado em uma fila vazia.

exception asyncio.QueueFull

Exceção levantada quando o método `put_nowait()` é chamado em uma fila que atingiu seu *maxsize*.

exception asyncio.QueueShutDown

Exceção levantada quando `put()`, `put_nowait()`, `get()` ou `get_nowait()` é chamado em uma fila que foi desligada.

Adicionado na versão 3.13.

Exemplos

Filas podem ser usadas para distribuir cargas de trabalho entre diversas tarefas concorrentes:

```
import asyncio
import random
import time


async def worker(nome, fila):
    while True:
        # Retira um "item de trabalho" da fila.
        dormir_por = await queue.get()

        # Dorme por "dormir_por" segundos.
        await asyncio.sleep(dormir_por)

        # Notifica a fila de que o "item de trabalho" foi proces
```

```
        fila.task_done()

        print(f'{nome} dormiu por {dormir_por:.2f} segundos')


async def main():
    # Cria uma fila para armazenar nossa "carga de trabalho".
    fila = asyncio.Queue()

    # Gera tempos aleatórios e os insere na fila.
    tempo_total_de_sono = 0
    for _ in range(20):
        dormir_por = random.uniform(0.05, 1.0)
        tempo_total_de_sono += dormir_por
        fila.put_nowait(dormir_por)

    # Cria três tarefas "worker" para processarem a fila concorr
    tarefas = []
    for i in range(3):
        tarefa = asyncio.create_task(worker(f'worker-{i}', fila)
        tarefas.append(tarefa)

    # Espera até a fila ser completamente processada.
    comecou_em = time.monotonic()
    await fila.join()
    tempo_dormido = time.monotonic() - comecou_em

    # Cancela nossas tarefas.
    for tarefa in tarefas:
        tarefa.cancel()
    # Espera até todas as tarefas serem canceladas.
    await asyncio.gather(*tarefas, return_exceptions=True)

    print('====')
    print(f'3 trabalhadoras dormiram em paralelo por {tempo_dorm
    print(f'Tempo total de sono esperado: {tempo_total_de_sono:.


asyncio.run(main())
```

Exceções

Código-fonte: Lib/asyncio/exceptions.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/exceptions.py]

---

exception asyncio.TimeoutError

Um apelido descontinuado de `TimeoutError`, levantado quando a operação excedeu o prazo determinado.

Alterado na versão 3.11: Esta classe foi feita como um apelido de `TimeoutError`.

exception asyncio.CancelledError

A operação foi cancelada.

Esta exceção pode ser capturada para executar operações personalizadas quando as tarefas assíncronas são canceladas. Em quase todas as situações, a exceção deve ser levantada novamente.

Alterado na versão 3.8: `CancelledError` é agora uma subclasse de `BaseException` em vez de `Exception`.

exception asyncio.InvalidStateError

Estado interno inválido de `Task` ou `Future`.

Pode ser levantada em situações como definir um valor de resultado para um objeto *Future* que já tem um valor de resultado definido.

exception asyncio.SendfileNotAvailableError

A *syscall* “sendfile” não está disponível para o soquete ou tipo de arquivo fornecido.

Uma subclasse de `RuntimeError`.

exception asyncio.IncompleteReadError

A operação de leitura solicitada não foi totalmente concluída.

Levantada pelas APIs de fluxo de asyncio.

Esta exceção é uma subclasse de `EOFError`.

expected

O número total (`int`) de bytes esperados.

partial

Uma string de `bytes` lida antes do final do fluxo ser alcançado.

exception asyncio.LimitOverrunError

Atingiu o limite de tamanho do buffer ao procurar um separador.

Levantada pelas APIs de fluxo de asyncio.

consumed

O número total de bytes a serem consumidos.

Call graph introspection

Código-fonte: Lib/asyncio/grafo.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/grafo.py]

O asyncio disponibiliza utilitários poderosos de introspeção do grafo de chamadas em tempo de execução, que permitem traçar o grafo de chamadas completo de uma *corrotina* ou *tarefa* em execução, ou de um *futuro* suspenso. Estes utilitários e a respectiva maquinaria subjacente podem ser utilizados em um programa Python ou por perfis externos e depuradores.

Ver também

Command-line introspection tools
Command-line tools for inspecting tasks in another running Python process.

Adicionado na versão 3.14.

asyncio.print_call_graph(*future=None*, /, *, *file=None*, *depth=1*, *limit=None*)

Exibe o grafo de chamadas assíncronas para a tarefa atual ou para a `Task` ou `Future` fornecida.

Esta função exibe as entradas a partir do quadro superior até ao ponto de invocação.

A função recebe um argumento opcional *future*. Se não for fornecido, será utilizada a tarefa em execução atual.

Se a função for chamada na *tarefa atual*, o argumento opcional somente-nomeado *depth* pode ser usado para ignorar o número especificado de quadros a partir do topo da pilha.

Se o argumento opcional somente-nomeado *limit* for fornecido, cada pilha de chamadas no gráfico resultante será truncada para incluir no máximo `abs(limit)` entradas. Se *limit* for positivo, as entradas exibidas serão as mais próximas do ponto de invocação. Se *limit* for negativo, as entradas mais altas são exibidas. Se *limit* for omitido ou `None`, todas as entradas são apresentadas. Se *limit* for `0`, a pilha de chamadas não é exibida, apenas as informações "aguardadas por" são exibidas.

Se *file* for omitido ou for `None`, a saída da função será encaminhada para **`sys.stdout`**.

Exemplo:

O seguinte código Python:

```
import asyncioasync def test():    asyncio.print_call_graph(
```

irá exibir:

```
* Task(name='test', id=0x1039f0fe0)+ Call stack:|   File 't2.
```

asyncio.format_call_graph(*future=None*, /, *, *depth=1*, *limit=None*)

Semelhante a **`print_call_graph()`**, mas retorna uma string. Se *future* for `None` e não houver nenhuma tarefa atual, a função retorna uma string vazia.

asyncio.capture_call_graph(*future=None*, /, *, *depth=1*, *limit=None*)

Captura o grafo de chamadas assíncronas para a tarefa atual ou para a **`Task`** ou **`Future`** fornecida.

A função recebe um argumento opcional *future*. Se não for fornecido, será utilizada a tarefa em execução atual. Se não existir nenhuma tarefa atual, a função retorna `None`.

Se a função for chamada na *tarefa atual*, o argumento opcional somente-nomeado *depth* pode ser usado para ignorar o número especificado de quadros a partir do topo da pilha.

Retorna um objeto da classe de dados `FutureCallGraph`:

- `FutureCallGraph(future, call_stack, awaited_by)`

  Onde *future* é uma referência a um **`Future`** ou a uma **`Task`** (ou às suas subclasses).

  `call_stack` é uma tupla de objetos `FrameCallGraphEntry`.

  `awaited_by` é uma tupla de objetos `FutureCallGraph`.

- `FrameCallGraphEntry(frame)`

  Onde *frame* é um objeto de quadro de uma função Python regular na pilha de chamadas.

Funções utilitárias de baixo nível

Para efetuar a introspeção de um grafo de chamadas assíncronas, o asyncio requer a cooperação de estruturas de controle de fluxo, como **`shield()`** ou **`TaskGroup`**. Sempre que um objeto **`Future`** intermediário com APIs de baixo nível, como **`Future.add_done_callback()`**, estiver envolvido, as duas funções seguintes devem ser utilizadas para informar o asyncio sobre a forma como esses objetos *future* intermédios estão ligados às tarefas que envolvem ou controlam.

asyncio.future_add_to_awaited_by(*future*, *waiter*, /)

Registra que *future* está a ser aguardado por *waiter*.

Tanto *future* quanto *waiter* devem ser instâncias de **`Future`** ou **`Task`** ou de suas subclasses; caso contrário, a chamada não terá efeito.

Uma chamada a `future_add_to_awaited_by()` deve ser seguida por uma chamada eventual à função **`future_discard_from_awaited_by()`** com os mesmos argumentos.

asyncio.future_discard_from_awaited_by(*future*, *waiter*, /)

Registra que *future* não está mais sendo aguardado por *waiter*.

Tanto *future* quanto *waiter* devem ser instâncias de `Future` ou `Task` ou de suas subclasses; caso contrário, a chamada não terá efeito.

Command-line introspection tools

Source code: Lib/asyncio/tools.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/tools.py]

The **`asyncio`** module can be invoked as a script via `python -m asyncio` to inspect the task graph of another running Python process without modifying it or restarting it. The **`asyncio.tools`** submodule implements this interface.

The following commands inspect the process identified by `PID`:

```
$ python -m asyncio pstree [--retries N] PID
$ python -m asyncio ps [--retries N] PID
```

The commands read the target process state without executing any code in it. They are only available on supported platforms and may require permission to inspect another process. See the permission requirements for details.

Ver também

Call graph introspection
Programmatic APIs for inspecting the async call graph of a task or future in the current process.

The command examples below use this program, which creates a task hierarchy suitable for inspection and prints its process ID:

example.py

```
import asyncio
import os
```

```
async def play(track):
    await asyncio.sleep(3600)
    print(f"□ Finished: {track}")

async def album(name, tracks):
    async with asyncio.TaskGroup() as tg:
        for track in tracks:
            tg.create_task(play(track), name=track)

async def main():
    print(f"PID: {os.getpid()}")
    async with asyncio.TaskGroup() as tg:
        tg.create_task(
            album("Sundowning", ["TNDNBTG", "Levitate"]),
            name="Sundowning",
        )
        tg.create_task(
            album("TMBTE", ["DYWTYLM", "Aqua Regia"]),
            name="TMBTE",
        )

asyncio.run(main())
```

Run the program in one terminal and leave it running:

```
$ python example.py
PID: 12345
```

Then pass the printed process ID to the commands from another terminal. Thread IDs, task IDs, file paths, and line numbers vary between runs and source layouts.

Adicionado na versão 3.14.

Opções de linha de comando

pstree PID

Display task and coroutine relationships as a tree. Each task is shown with its full coroutine stack, nested under the task (if any) that is awaiting it. This subcommand is useful for quickly identifying which

branch of a task hierarchy is blocked and where in its coroutine stack execution has paused:

```
$ python -m asyncio pstree 12345
└── (T) Task-1
    └── main example.py:12
        └── TaskGroup.__aexit__ Lib/asyncio/taskgroups.py:75
            └── TaskGroup._aexit Lib/asyncio/taskgroups.py:1
                ├── (T) Sundowning
                │   └── album example.py:7
                │       └── TaskGroup.__aexit__ Lib/asyncio/
                │           └── TaskGroup._aexit Lib/asynci
                │               ├── (T) TNDNBTG
                │               │   └── play example.py:4
                │               │       └── sleep Lib/asynci
                │               └── (T) Levitate
                │                   └── play example.py:4
                │                       └── sleep Lib/asynci
                │
                └── (T) TMBTE
                    └── album example.py:7
                        └── TaskGroup.__aexit__ Lib/asyncio/
                            └── TaskGroup._aexit Lib/asynci
                                ├── (T) DYWTYLM
                                │   └── play example.py:4
                                │       └── sleep Lib/asynci
                                └── (T) Aqua Regia
                                    └── play example.py:4
                                        └── sleep Lib/asynci
```

If the await graph contains a cycle, `pstree` reports an error instead of printing a tree. A cycle in the await graph is unusual and typically indicates a programming error:

```
$ python -m asyncio pstree 12345
ERROR: await-graph contains cycles - cannot print a tree!

cycle: Task-2 → Task-3 → Task-2
```

ps PID

Display a flat table of all pending tasks in the process *PID*. Each row shows the event-loop thread ID, task ID and name, coroutine stack, and

the awaiting task's stack, name, and ID, if any.

This subcommand prints all tasks regardless of whether the await graph contains cycles:

```
$ python -m asyncio ps 12345
tid        task id              task name            coroutir
-------------------------------------------------------------
18445801   0x10a456060          Task-1               TaskGrou
18445801   0x10a439f60          Sundowning           TaskGrou
18445801   0x10a439d70          TMBTE                TaskGrou
18445801   0x10a2a3a80          TNDNBTG              sleep ->
18445801   0x10a2a38a0          Levitate             sleep ->
18445801   0x10a2d7150          DYWTYLM              sleep ->
18445801   0x10a6bdaa0          Aqua Regia           sleep ->
```

--retries N

Retry failed attempts to inspect the target process up to *N* times. This can help when the target process changes while its state is being read.

Adicionado na versão 3.15.

Laço de eventos

Código-fonte: Lib/asyncio/events.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/events.py], Lib/asyncio/base_events.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/base_events.py]

Prefácio

O laço de eventos é o núcleo de toda aplicação asyncio. Laços de eventos executam tarefas e funções de retorno assíncronas, realizam operações de entrada e saída e executam subprocessos.

Os desenvolvedores de aplicação normalmente devem usar as funções asyncio de alto nível, como `asyncio.run()`, e devem raramente precisar fazer referência ao objeto de loop ou chamar seus métodos. Esta seção destina-se principalmente a autores de código de baixo nível, bibliotecas e frameworks, que precisam de um controle mais preciso sobre o comportamento do laço de evento.

Obtendo o laço de eventos

As seguintes funções baixo nível podem ser usadas para obter, definir, ou criar um laço de eventos:

asyncio.get_running_loop()

Retorna o laço de eventos em execução na thread atual do sistema operacional.

Levanta uma `RuntimeError` se não houver nenhum laço de eventos em execução.

Esta função só pode ser chamada a partir de uma corrotina ou de um retorno de chamada.

Adicionado na versão 3.7.

asyncio.get_event_loop()

Obtém o laço de eventos atual.

Quando chamada de uma corrotina ou função de retorno (por exemplo, agendada com call_soon ou API semelhante), esta função sempre retornará o laço de eventos em execução.

Se não houver nenhum laço de eventos em execução definido, a função retornará o resultado da chamada `get_event_loop_policy().get_event_loop()`.

Devido ao fato desta função ter um comportamento particularmente complexo (especialmente quando políticas de laço de eventos customizadas estão sendo usadas), usar a função `get_running_loop()` é preferido ao invés de `get_event_loop()` em corrotinas e funções de retorno.

Como observado acima, considere usar também a função de alto nível `asyncio.run()` ao invés de usar funções de baixo nível para manualmente criar e fechar um laço de eventos.

Alterado na versão 3.14: Levanta uma `RuntimeError` se não houver nenhum laço de eventos atual.

Nota

The `asyncio` policy system is deprecated and will be removed in Python 3.16; from there on, this function will return the current running event loop if present else it will return the loop set by `set_event_loop()`.

asyncio.set_event_loop(*loop*)

Define *loop* como o laço de eventos atual para a thread atual do sistema operacional.

asyncio.new_event_loop()

Cria e retorna um novo objeto de laço de eventos.

Perceba que o comportamento das funções `get_event_loop()`, `set_event_loop()`, e `new_event_loop()` podem ser alteradas definindo uma política de laço de eventos customizada.

Conteúdo

Esta página de documentação contém as seguintes seções:

- A seção Métodos do laço de eventos é a documentação de referência das APIs de laço de eventos;
- A seção Tratadores de função de retorno documenta as instâncias das classes `Handle` e `TimerHandle`, que são retornadas por métodos de agendamento tais como `loop.call_soon()` e `loop.call_later()`;
- A seção Objetos Server documenta tipos retornados a partir de métodos de laço de eventos, como `loop.create_server()`;
- A seção Implementações do Laço de Eventos documenta as classes `SelectorEventLoop` e `ProactorEventLoop`;
- A seção Exemplos demonstra como trabalhar com algumas APIs do laço de eventos APIs.

Event loop methods

Laços de eventos possuem APIs de **baixo nível** para as seguintes situações:

- Executar e interromper o laço
- Agendando funções de retorno
- Agendando funções de retorno atrasadas
- Creating futures and tasks
- Abrindo conexões de rede

- Criando servidores de rede
- Transferindo arquivos
- TLS upgrade
- Observando descritores de arquivo
- Trabalhando com objetos soquete diretamente
- DNS
- Trabalhando com encadeamentos
- Sinais Unix
- Executando código em conjuntos de threads ou processos
- Error handling API
- Habilitando o modo de debug
- Running subprocesses

Executar e interromper o laço

loop.run_until_complete(*future*)

Executar até que o *future* (uma instância da classe `Future`) seja completada.

Se o argumento é um objeto corrotina, ele é implicitamente agendado para executar como uma `asyncio.Task`.

Retorna o resultado do Future ou levanta sua exceção.

loop.run_forever()

Executa o laço de eventos até que `stop()` seja chamado.

If `stop()` is called before `run_forever()` is called, the loop will poll the I/O selector once with a timeout of zero, run all callbacks scheduled in response to I/O events (and those that were already scheduled), and then exit.

Se `stop()` for chamado enquanto `run_forever()` estiver executando, o laço irá executar o lote atual de funções de retorno e então sair. Perceba que novas funções de retorno agendadas por funções de retorno não serão executadas neste caso; ao invés disso, elas serão executadas na

próxima vez que **`run_forever()`** ou **`run_until_complete()`** forem chamadas.

loop.stop()

Para o laço de eventos.

loop.is_running()

Retorna `True` se o laço de eventos estiver em execução.

loop.is_closed()

Retorna `True` se o laço de eventos foi fechado.

loop.close()

Fecha o laço de eventos.

O laço não deve estar em execução quando esta função for chamada. Qualquer função de retorno pendente será descartada.

Este método limpa todas as filas e desliga o executor, mas não aguarda pelo encerramento do executor.

Este método é idempotente e irreversível. Nenhum outro método deve ser chamado depois que o laço de eventos esteja fechado.

async loop.shutdown_asyncgens()

Schedule all currently open asynchronous generator objects to close with an **`aclose()`** call. After calling this method, the event loop will issue a warning if a new asynchronous generator is iterated. This should be used to reliably finalize all scheduled asynchronous generators.

Perceba que não é necessário chamar esta função quando **`asyncio.run()`** for usado.

Exemplo:

```
try:
    loop.run_forever()
```

```
finally:
    loop.run_until_complete(loop.shutdown_asyncgens())
    loop.close()
```

Adicionado na versão 3.6.

async loop.shutdown_default_executor(*timeout=None*)

Agenda o encerramento do executor padrão e aguarda ele se juntar a todas as threads no `ThreadPoolExecutor`. Uma vez que este método tenha sido chamado, usar o executor padrão com `loop.run_in_executor()` levantará um `RuntimeError`.

O parâmetro *timeout* especifica a quantidade de tempo (em `float` segundos) que o executor terá para terminar a junção. Com o padrão `None`, o executor tem permissão para um período de tempo ilimitado.

Se o *timeout* for atingido, uma exceção `RuntimeWarning` será emitida e o executor padrão será finalizado sem esperar que suas threads terminem a junção.

Nota

Do not call this method when using `asyncio.run()`, as the latter handles default executor shutdown automatically.

Adicionado na versão 3.9.

Alterado na versão 3.12: Added the *timeout* parameter.

Agendando funções de retorno

loop.call_soon(*callback*, **args*, *context=None*)

Agenda a *função de retorno* callback para ser chamada com argumentos *args* na próxima iteração do laço de eventos.

Return an instance of `asyncio.Handle`, which can be used later to cancel the callback.

Funções de retorno são chamadas na ordem em que elas foram registradas. Cada função de retorno será chamada exatamente uma vez.

The optional keyword-only *context* argument specifies a custom `contextvars.Context` for the *callback* to run in. Callbacks use the current context when no *context* is provided.

Unlike `call_soon_threadsafe()`, this method is not thread-safe.

loop.call_soon_threadsafe(*callback*, **args*, *context=None*)

A thread-safe variant of `call_soon()`. When scheduling callbacks from another thread, this function *must* be used, since `call_soon()` is not thread-safe.

This function is safe to be called from a reentrant context or signal handler, however, it is not safe or fruitful to use the returned handle in such contexts.

Levanta `RuntimeError` se chamado em um laço que já foi fechado. Isto pode acontecer em uma thread secundária quando a aplicação principal estiver desligando.

Veja a seção concorrência e multithreading da documentação.

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** [https://peps.python.org/pep-0567/] para mais detalhes.

Nota

Maior parte das funções de agendamento `asyncio` não permite passar argumentos nomeados. Para fazer isso, use `functools.partial()`:

```
# will schedule "print("Hello", flush=True)"
loop.call_soon(
    functools.partial(print, "Hello", flush=True))
```

Usar objetos parciais é usualmente mais conveniente que usar lambdas, pois asyncio pode renderizar objetos parciais melhor durante debug e mensagens de erro.

Agendando funções de retorno atrasadas

Laço de eventos fornece mecanismos para agendar funções de retorno para serem chamadas em algum ponto no futuro. Laço de eventos usa relógios monotônico para acompanhar o tempo.

loop.call_later(*delay*, *callback*, **args*, *context=None*)

Agenda *callback* para ser chamada após o *delay* número de segundos fornecido (pode ser um inteiro ou um ponto flutuante).

Uma instância de `asyncio.TimerHandle` é retornada, a qual pode ser usada para cancelar a função de retorno.

callback será chamada exatamente uma vez. Se duas funções de retorno são agendadas para exatamente o mesmo tempo, a ordem na qual elas são chamadas é indefinida.

The optional positional *args* will be passed to the callback when it is called. Use `functools.partial()` to pass keyword arguments to *callback*.

Um argumento opcional somente-nomeado *context* permite especificar um `contextvars.Context` customizado para executar na *função de retorno*. O contexto atual é usado quando nenhum *context* é fornecido.

Nota

For performance, callbacks scheduled with `loop.call_later()` may run up to one clock-resolution early (see `time.get_clock_info('monotonic').resolution`).

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** [https://peps.python.org/pep-0567/] para mais detalhes.

Alterado na versão 3.8: No Python 3.7 e anterior, com a implementação padrão do laço de eventos, o *delay* não poderia exceder um dia. Isto foi ajustado no Python 3.8.

loop.call_at(*when*, *callback*, **args*, *context=None*)

Agenda *callback* para ser chamada no timestamp absoluto fornecido *when* (um inteiro ou um ponto flutuante), usando o mesmo horário de referência que `loop.time()`.

O comportamento deste método é o mesmo que `call_later()`.

Uma instância de `asyncio.TimerHandle` é retornada, a qual pode ser usada para cancelar a função de retorno.

Nota

For performance, callbacks scheduled with **`loop.call_at()`** may run up to one clock-resolution early (see `time.get_clock_info('monotonic').resolution`).

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** [https://peps.python.org/pep-0567/] para mais detalhes.

Alterado na versão 3.8: No Python 3.7 e anterior, com a implementação padrão do laço de eventos, a diferença entre *when* e o horário atual não poderia exceder um dia. Isto foi ajustado no Python 3.8.

loop.time()

Retorna o horário atual, como um valor `float`, de acordo com o relógio monotônico interno do laço de eventos.

Nota

Alterado na versão 3.8: No Python 3.7 e anterior, tempos limites (*delay* relativo ou *when* absoluto) não poderiam exceder um dia. Isto foi ajustado no Python 3.8.

Ver também

A função `asyncio.sleep()`.

Creating futures and tasks

loop.create_future()

Cria um objeto `asyncio.Future` anexado ao laço de eventos.

Este é o modo preferido para criar Futures em asyncio. Isto permite que laços de eventos de terceiros forneçam implementações alternativas do objeto Future (com melhor desempenho ou instrumentação).

Adicionado na versão 3.5.2.

loop.create_task(*coro*, *, *name=None*, *context=None*, *eager_start=None*, ***kwargs*)

Agenda a execução da corrotina *coro*. Retorna um objeto `Task`.

Laços de eventos de terceiros podem usar suas próprias subclasses de `Task` para interoperabilidade. Neste caso, o tipo do resultado é uma subclasse de `Task`.

A assinatura completa da função é, em grande parte, a mesma que a do construtor (ou fábrica) de `Task` — todos os argumentos nomeados desta função são passados para essa interface.

Se o argumento *name* for fornecido e não for `None`, ele é definido como o nome da tarefa, usando `Task.set_name()`.

Um argumento opcional somente-nomeado *context* permite especificar um `contextvars.Context` personalizado para a *coro* ser executada. Uma cópia do contexto atual é criada quando nenhum *context* for fornecido.

Um argumento opcional somente-nomeado *eager_start* permite especificar se a tarefa deve ser executada imediatamente durante a chamada para create_task ou ser agendada para mais tarde. Se *eager_start* não for fornecido, o modo definido por `loop.set_task_factory()` será utilizado.

Alterado na versão 3.8: Adicionado o parâmetro *name*.

Alterado na versão 3.11: Adicionado o parâmetro *context*.

Alterado na versão 3.13.3: Adicionado `kwargs`, que passa parâmetros extra arbitrários, incluindo `name` e `context`.

Alterado na versão 3.13.4: Reverteu a alteração que passava *name* e *context* (se for None), enquanto ainda passa outros argumentos somente-nomeados arbitrários (para evitar quebrar a compatibilidade retroativa com a versão 3.13.3).

Alterado na versão 3.14: Todos os *kwargs* são agora passados. O parâmetro *eager_start* funciona com fábricas de tarefas imediatas.

loop.set_task_factory(*factory*)

Define a fábrica da tarefa que será usada por `loop.create_task()`.

Se *factory* for `None`, será usada a fábrica de tarefas padrão. Caso contrário, *factory* deve ser um *chamável* com a assinatura `(loop, coro, **kwargs)`, em que *loop* é uma referência ao ciclo de eventos ativo e *coro* é um objeto corrotina. O chamável deve passar todos os *kwargs* e devolver um objeto compatível com `asyncio.Task`.

Alterado na versão 3.13.3: Exigido que todos os *kwargs* sejam passados para `asyncio.Task`.

Alterado na versão 3.13.4: name não é mais passado para fábricas de tarefas. *context* não é mais passado para fábricas de tarefas se for `None`.

Alterado na versão 3.14: name e *context* agora são passados incondicionalmente para as fábricas de tarefas novamente.

loop.get_task_factory()

Retorna uma fábrica de tarefa ou `None` se a fábrica padrão estiver em uso.

Abrindo conexões de rede

async loop.create_connection(*protocol_factory*, *host=None*, *port=None*, *, *ssl=None*, *family=0*, *proto=0*, *flags=0*, *sock=None*, *local_addr=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *happy_eyeballs_delay=None*, *interleave=None*, *all_errors=False*)

Abre uma conexão de transporte para streaming, para um endereço fornecido, especificado por *host* e *port*.

A família de sockets pode ser **`AF_INET`** ou **`AF_INET6`** dependendo do *host* (ou do argumento *family*, se fornecido).

The socket type will be **`SOCK_STREAM`**.

protocol_factory deve ser um chamável que retorne uma implementação do protocolo asyncio.

Este método tentará estabelecer a conexão em segundo plano. Quando tiver sucesso, ele retorna um par `(transport, protocol)`.

A sinopse cronológica da operação subjacente é conforme abaixo:

1. A conexão é estabelecida e um transporte é criado para ela.
2. *protocol_factory* é chamada sem argumentos e é esperada que retorne uma instância de protocolo.

3. A instância de protocolo é acoplada com o transporte, através da chamada do seu método `connection_made()`.
4. Uma tupla `(transport, protocol)` é retornada ao ter sucesso.

O transporte criado é um stream bi-direcional dependente de implementação.

Outros argumentos:

- *ssl*: se fornecido e não for falso, um transporte SSL/TLS é criado (por padrão um transporte TCP simples é criado). Se *ssl* for um objeto `ssl.SSLContext`, este contexto é usado para criar o transporte; se *ssl* for `True`, um contexto padrão retornado de `ssl.create_default_context()` é usado.

  Ver também

  Considerações de segurança sobre SSL/TLS

- *server_hostname* define ou substitui o hostname que o certificado do servidor de destino será pareado contra. Deve ser passado apenas se *ssl* não for `None`. Por padrão o valor do argumento *host* é usado. Se *host* for vazio, não existe valor padrão e você deve passar um valor para *server_hostname*. Se *server_hostname* for uma string vazia, o pareamento de hostname é desabilitado (o que é um risco de segurança sério, permitindo ataques potenciais man-in-the-middle).

- *family*, *proto*, *flags* são os endereços familiares, protocolos e sinalizadores opcionais a serem passados por getaddrinfo() para resolução do *host*. Se fornecidos, eles devem ser todos inteiros e constantes correspondentes do módulo `socket`.

- *happy_eyeballs_delay*, se fornecido, habilita Happy Eyeballs para esta conexão. Ele deve ser um número de ponto flutuante representando o tempo em segundos para aguardar uma tentativa de conexão encerrar, antes de começar a próxima tentativa em paralelo. Este é o “Atraso na tentativa de conexão” conforme definido na **RFC 8305** [https://datatracker.ietf.org/doc/html/rfc8305.html]. Um

valor padrão sensível recomendado pela RFC é `0.25` (250 millissegundos).

- *interleave* controla o reordenamento de endereços quando um nome de servidor resolve para múltiplos endereços IP. Se `0` ou não especificado, nenhum reordenamento é feito, e endereços são tentados na ordem retornada por **`getaddrinfo()`**. Se um inteiro positivo for especificado, os endereços são intercalados por um endereço familiar, e o inteiro fornecido é interpretado como "Contagem da família do primeiro endereço" conforme definido na **RFC 8305** [https://datatracker.ietf.org/doc/html/rfc8305.html]. O padrão é `0` se *happy_eyeballs_delay* não for especificado, e `1` se ele for.

- *sock*, se fornecido, deve ser um objeto **`socket.socket`** já existente, já conectado, para ser usado por transporte. Se *sock* é fornecido, *host*, *port*, *family*, *proto*, *flags*, *happy_eyeballs_delay*, *interleave* e *local_addr* não devem ser especificados.

  Nota

  The *sock* argument transfers ownership of the socket to the transport created. To close the socket, call the transport's **`close()`** method.

- *local_addr*, se fornecido, é uma tupla de `(local_host, local_port)` usada para se ligar ao soquete localmente. O *local_host* e a *local_port* são procurados usando `getaddrinfo()`, de forma similar para *host* e *port*.

- *ssl_handshake_timeout* é (para uma conexão TLS) o tempo em segundos para aguardar pelo encerramento do aperto de mão TLS, antes de abortar a conexão. `60.0` segundos se for `None` (valor padrão).

- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).

- *all_errors* determines what exceptions are raised when a connection cannot be created. By default, only a single `Exception` is raised: the first exception if there is only one or all errors have same message, or a single `OSError` with the error messages combined. When `all_errors` is `True`, an `ExceptionGroup` will be raised containing all exceptions (even if there is only one).

Alterado na versão 3.5: Adicionado suporte para SSL/TLS na **`ProactorEventLoop`**.

Alterado na versão 3.6: The socket option socket.TCP_NODELAY is set by default for all TCP connections.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*.

Alterado na versão 3.8: Adicionados os parâmetros *happy_eyeballs_delay* e *interleave*.

Happy Eyeballs Algorithm: Success with Dual-Stack Hosts. When a server's IPv4 path and protocol are working, but the server's IPv6 path and protocol are not working, a dual-stack client application experiences significant connection delay compared to an IPv4-only client. This is undesirable because it causes the dual-stack client to have a worse user experience. This document specifies requirements for algorithms that reduce this user-visible delay and provides an algorithm.

For more information: https://datatracker.ietf.org/doc/html/rfc6555

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.12: all_errors was added.

Alterado na versão 3.15: Raises a `ValueError` if `ssl.check_hostname` is `True` and `server_hostname` is not supplied.

Ver também

A função `open_connection()` é uma API alternativa de alto nível. Ela retorna um par de (`StreamReader`, `StreamWriter`) que pode ser usado diretamente em código async/await.

async loop.create_datagram_endpoint(*protocol_factory*, *local_addr=None*, *remote_addr=None*, *, *family=0*, *proto=0*, *flags=0*, *reuse_port=None*, *allow_broadcast=None*, *sock=None*)

Cria uma conexão de datagrama.

The socket family can be either `AF_INET`, `AF_INET6`, or `AF_UNIX`, depending on *host* (or the *family* argument, if provided).

The socket type will be `SOCK_DGRAM`.

protocol_factory deve ser algo chamável, retornando uma implementação de protocolo.

Uma tupla de `(transport, protocol)` é retornada em caso de sucesso.

Outros argumentos:

- *local_addr*, se fornecido, é uma tupla de `(local_host, local_port)` usada para ligar o soquete localmente. O *local_host* e a *local_port* são procurados usando `getaddrinfo()`.

  Nota

  On Windows, when using the proactor event loop with `local_addr=None`, an `OSError` with `errno.WSAEINVAL` will be raised when running it.

- *remote_addr*, se fornecido, é uma tupla de `(remote_host, remote_port)` usada para conectar o soquete a um endereço remoto. O *remote_host* e a *remote_port* são procurados usando `getaddrinfo()`.

- *family*, *proto*, *flags* são os endereços familiares, protocolo e flags opcionais a serem passados para `getaddrinfo()` para resolução do *host*. Se fornecido, esses devem ser todos inteiros do módulo de constantes `socket` correspondente.

- *reuse_port* tells the kernel to allow this endpoint to be bound to the same port as other existing endpoints are bound to, so long as they all set this flag when being created. This option is not supported on Windows and some Unixes. If the socket.SO_REUSEPORT constant is not defined then this capability is unsupported.

- *allow_broadcast* avisa o kernel para permitir que este endpoint envie mensagens para o endereço de broadcast.

- *sock* pode opcionalmente ser especificado em ordem para usar um objeto `socket.socket` pre-existente, já conectado, para ser usado pelo transporte. Se especificado, *local_addr* e *remote_addr* devem ser omitidos (devem ser `None`).

  Nota

  The *sock* argument transfers ownership of the socket to the transport created. To close the socket, call the transport's `close()` method.

Veja protocolo UDP eco cliente e protocolo UDP eco servidor para exemplos.

Alterado na versão 3.4.4: The *family*, *proto*, *flags*, *reuse_address*, *reuse_port*, *allow_broadcast*, and *sock* parameters were added.

Alterado na versão 3.8: Adicionado suporte para Windows.

Alterado na versão 3.8.1: The *reuse_address* parameter is no longer supported, as using socket.SO_REUSEADDR poses a significant security concern for UDP. Explicitly passing `reuse_address=True` will raise an exception.

Quando múltiplos processos com diferentes UIDs atribuem soquetes para um endereço de soquete UDP idêntico com `SO_REUSEADDR`, pacotes recebidos podem ser distribuídos aleatoriamente entre os soquetes.

For supported platforms, *reuse_port* can be used as a replacement for similar functionality. With *reuse_port*, socket.SO_REUSEPORT is used instead, which specifically prevents processes with differing UIDs from assigning sockets to the same socket address.

Alterado na versão 3.11: The *reuse_address* parameter, disabled since Python 3.8.1, 3.7.6 and 3.6.10, has been entirely removed.

async loop.create_unix_connection(*protocol_factory*, *path=None*, *, *ssl=None*, *sock=None*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Cria uma conexão Unix.

The socket family will be **`AF_UNIX`**; socket type will be **`SOCK_STREAM`**.

Uma tupla de `(transport, protocol)` é retornada em caso de sucesso.

path é o nome de um soquete de domínio Unix e é obrigatório, a não ser que um parâmetro *sock* seja especificado. Soquetes Unix abstratos, **`str`**, **`bytes`**, e caminhos **`Path`** são suportados.

Veja a documentação do método **`loop.create_connection()`** para informações a respeito de argumentos para este método.

Disponibilidade: Unix.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*. O parâmetro *path* agora pode ser um path-like object.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Criando servidores de rede

async loop.create_server(*protocol_factory*, *host=None*, *port=None*, *, *family=socket.AF_UNSPEC*, *flags=socket.AI_PASSIVE*, *sock=None*, *backlog=100*, *ssl=None*, *reuse_address=None*, *reuse_port=None*, *keep_alive=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*)

Create a TCP server (socket type `SOCK_STREAM`) listening on *port* of the *host* address.

Retorna um objeto `Server`.

Argumentos:

- *protocol_factory* deve ser algo chamável, retornando uma implementação de protocolo.
- O parâmetro *host* pode ser definido para diversos tipos, o qual determina onde o servidor deve escutar:
  - Se *host* for uma string, o servidor TCP está vinculado a apenas uma interface de rede, especificada por *host*.
  - Se *host* é uma sequência de strings, o servidor TCP está vinculado a todas as interfaces de rede especificadas pela sequência.
  - Se *host* é uma string vazia ou `None`, todas as interfaces são presumidas e uma lista de múltiplos soquetes será retornada (muito provavelmente um para IPv4 e outro para IPv6).
- O parâmetro *port* pode ser definido para especificar qual porta o servidor deve escutar. Se `0` ou `None` (o padrão), uma porta aleatória disponível será selecionada (note que se *host* resolve para múltiplas interfaces de rede, uma porta aleatória diferente será selecionada para cada interface).
- *family* can be set to either `socket.AF_INET` or `AF_INET6` to force the socket to use IPv4 or IPv6. If not set, the *family* will be determined from host name (defaults to `AF_UNSPEC`).
- *flags* é uma máscara de bits para `getaddrinfo()`.

- *sock* pode opcionalmente ser especificado para usar um objeto soquete pré-existente. Se especificado, *host* e *port* não devem ser especificados.

  Nota

  The *sock* argument transfers ownership of the socket to the server created. To close the socket, call the server's `close()` method.

- *backlog* é o número máximo de conexões enfileiradas passadas para `listen()` (padrão é 100).

- *ssl* pode ser definido para uma instância de `SSLContext` para habilitar TLS sobre as conexões aceitas.

- *reuse_address* diz ao kernel para reusar um soquete local em estado `TIME_WAIT`, sem aguardar pela expiração natural do seu tempo limite. Se não especificado, será automaticamente definida como `True` no Unix.

- *reuse_port* diz ao kernel para permitir que este endpoint seja vinculado a mesma porta que outros endpoints existentes estão vinculados, contanto que todos eles definam este sinalizador quando forem criados. Esta opção não é suportada no Windows.

- *keep_alive* set to `True` keeps connections active by enabling the periodic transmission of messages.

Alterado na versão 3.13: Added the *keep_alive* parameter.

- *ssl_handshake_timeout* é (para um servidor TLS) o tempo em segundos para aguardar pelo aperto de mão TLS ser concluído, antes de abortar a conexão. `60.0` segundos se `None` (valor padrão).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).
- Definir *start_serving* para `True` (o valor padrão) faz o servidor criado começar a aceitar conexões imediatamente. Quando

definido para `False`, o usuário deve aguardar com `Server.start_serving()` ou `Server.serve_forever()` para fazer o servidor começar a aceitar conexões.

Alterado na versão 3.5: Adicionado suporte para SSL/TLS na `ProactorEventLoop`.

Alterado na versão 3.5.1: O parâmetro *host* pode ser uma sequência de strings.

Alterado na versão 3.6: Added *ssl_handshake_timeout* and *start_serving* parameters. The socket option socket.TCP_NODELAY is set by default for all TCP connections.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Ver também

A função `start_server()` é uma API alternativa de alto nível que retorna um par de `StreamReader` e `StreamWriter` que pode ser usado em um código async/await.

async loop.create_unix_server(*protocol_factory*, *path=None*, ***, *sock=None*, *backlog=100*, *ssl=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*, *start_serving=True*, *cleanup_socket=True*)

Similar to `loop.create_server()` but works with the `AF_UNIX` socket family.

path é o nome de um soquete de domínio Unix, e é obrigatório, a não ser que um argumento *sock* seja fornecido. Soquetes Unix abstratos, `str`, `bytes`, e caminhos `Path` são suportados.

If *cleanup_socket* is true then the Unix socket will automatically be removed from the filesystem when the server is closed, unless the socket has been replaced after the server has been created.

Veja a documentação do método `loop.create_server()` para informações sobre argumentos para este método.

Disponibilidade: Unix.

Alterado na versão 3.7: Adicionados os parâmetros *ssl_handshake_timeout* e *start_serving*. O parâmetros *path* agora pode ser um objeto da classe `Path`.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Alterado na versão 3.13: Added the *cleanup_socket* parameter.

async loop.connect_accepted_socket(*protocol_factory*, *sock*, *, *ssl=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Envolve uma conexão já aceita em um par transporte/protocolo.

Este método pode ser usado por servidores que aceitam conexões fora do asyncio, mas que usam asyncio para manipulá-las.

Parâmetros:

- *protocol_factory* deve ser algo chamável, retornando uma implementação de protocolo.
- *sock* é um objeto soquete pré-existente retornado a partir de `socket.accept`.

  Nota

  The *sock* argument transfers ownership of the socket to the transport created. To close the socket, call the transport's `close()` method.

- *ssl* pode ser definido para um `SSLContext` para habilitar SSL sobre as conexões aceitas.
- *ssl_handshake_timeout* é (para uma conexão SSL) o tempo em segundos para aguardar pelo aperto de mão SSL ser concluído, antes de abortar a conexão. `60.0` segundos se `None` (valor padrão).

- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0` seconds if `None` (default).

Retorna um par `(transport, protocol)`.

Adicionado na versão 3.5.3.

Alterado na versão 3.7: Adicionado o parâmetro *ssl_handshake_timeout*.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Transferindo arquivos

async loop.sendfile(*transport*, *file*, *offset=0*, *count=None*, *, *fallback=True*)

Envia um *file* sobre um *transport*. Retorna o número total de bytes enviados.

O método usa `os.sendfile()` de alto desempenho, se disponível.

file deve ser um objeto arquivo regular aberto em modo binário.

offset indica a partir de onde deve iniciar a leitura do arquivo. Se especificado, *count* é o número total de bytes para transmitir, ao contrário de transmitir o arquivo até que EOF seja atingido. A posição do arquivo é sempre atualizada, mesmo quando este método levanta um erro, e `file.tell()` pode ser usado para obter o número atual de bytes enviados.

fallback definido para `True` faz o asyncio manualmente ler e enviar o arquivo quando a plataforma não suporta a chamada de sistema sendfile (por exemplo Windows ou soquete SSL no Unix).

Levanta `SendfileNotAvailableError` se o sistema não suporta a chamada de sistema *sendfile* e *fallback* é `False`.

Adicionado na versão 3.7.

TLS upgrade

async loop.start_tls(*transport*, *protocol*, *sslcontext*, *, *server_side=False*, *server_hostname=None*, *ssl_handshake_timeout=None*, *ssl_shutdown_timeout=None*)

Atualiza um conexão baseada em transporte existente para TLS.

Create a TLS coder/decoder instance and insert it between the *transport* and the *protocol*. The coder/decoder implements both *transport*-facing protocol and *protocol*-facing transport.

Return the created two-interface instance. After *await*, the *protocol* must stop using the original *transport* and communicate with the returned object only because the coder caches *protocol*-side data and sporadically exchanges extra TLS session packets with *transport*.

In some situations (e.g. when the passed transport is already closing) this may return `None`.

Parâmetros:

- instâncias de *transport* e *protocol*, que métodos como `create_server()` e `create_connection()` retornam.
- *sslcontext*: uma instância configurada de `SSLContext`.
- *server_side* informe `True` quando uma conexão no lado do servidor estiver sendo atualizada (como a que é criada por `create_server()`).
- *server_hostname*: define ou substitui o nome do host no qual o servidor alvo do certificado será comparado.
- *ssl_handshake_timeout* é (para uma conexão TLS) o tempo em segundos para aguardar pelo encerramento do aperto de mão TLS, antes de abortar a conexão. `60.0` segundos se for `None` (valor padrão).
- *ssl_shutdown_timeout* is the time in seconds to wait for the SSL shutdown to complete before aborting the connection. `30.0`

seconds if `None` (default).

Adicionado na versão 3.7.

Alterado na versão 3.11: Added the *ssl_shutdown_timeout* parameter.

Observando descritores de arquivo

loop.add_reader(*fd*, *callback*, **args*)

Começa a monitorar o descritor de arquivo *fd* para disponibilidade de leitura e invoca a *callback* com os argumentos especificados assim que *fd* esteja disponível para leitura.

Any preexisting callback registered for *fd* is cancelled and replaced by *callback*.

loop.remove_reader(*fd*)

Stop monitoring the *fd* file descriptor for read availability. Returns `True` if *fd* was previously being monitored for reads.

loop.add_writer(*fd*, *callback*, **args*)

Start monitoring the *fd* file descriptor for write availability and invoke *callback* with the specified arguments *args* once *fd* is available for writing.

Any preexisting callback registered for *fd* is cancelled and replaced by *callback*.

Use **`functools.partial()`** para passar argumentos nomeados para a *callback*.

loop.remove_writer(*fd*)

Stop monitoring the *fd* file descriptor for write availability. Returns `True` if *fd* was previously being monitored for writes.

Veja também a seção de Suporte a Plataformas para algumas limitações desses métodos.

Trabalhando com objetos soquete diretamente

Em geral, implementações de protocolo que usam APIs baseadas em transporte, tais como `loop.create_connection()` e `loop.create_server()` são mais rápidas que implementações que trabalham com soquetes diretamente. Entretanto, existem alguns casos de uso quando o desempenho não é crítica, e trabalhar com objetos `socket` diretamente é mais conveniente.

async loop.sock_recv(*sock*, *nbytes*)

Recebe até *nbytes* do *sock*. Versão assíncrona de `socket.recv()`.

Retorna os dados recebidos como um objeto de bytes.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.7: Apesar deste método sempre ter sido documentado como um método de corrotina, versões anteriores ao Python 3.7 retornavam um `Future`. Desde o Python 3.7 este é um método `async def`.

async loop.sock_recv_into(*sock*, *buf*)

Dados recebidos do *sock* no buffer *buf*. Modelado baseado no método bloqueante `socket.recv_into()`.

Retorna o número de bytes escritos no buffer.

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.7.

async loop.sock_recvfrom(*sock*, *bufsize*)

Receive a datagram of up to *bufsize* from *sock*. Asynchronous version of `socket.recvfrom()`.

Return a tuple of (received data, remote address).

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.11.

async loop.sock_recvfrom_into(*sock*, *buf*, *nbytes=0*)

Receive a datagram of up to *nbytes* from *sock* into *buf*. Asynchronous version of `socket.recvfrom_into()`.

Return a tuple of (number of bytes received, remote address).

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.11.

async loop.sock_sendall(*sock*, *data*)

Envia *data* para o soquete *sock*. Versão assíncrona de `socket.sendall()`.

Este método continua a enviar para o soquete até que todos os dados em *data* tenham sido enviados ou um erro ocorra. `None` é retornado em caso de sucesso. Ao ocorrer um erro, uma exceção é levantada. Adicionalmente, não existe nenhuma forma de determinar quantos dados, se algum, foram processados com sucesso pelo destinatário na conexão.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.7: Apesar deste método sempre ter sido documentado como um método de corrotina, antes do Python 3.7 ele retornava um `Future`. Desde o Python 3.7, este é um método `async def`.

async loop.sock_sendto(*sock*, *data*, *address*)

Send a datagram from *sock* to *address*. Asynchronous version of `socket.sendto()`.

Return the number of bytes sent.

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.11.

async loop.sock_connect(*sock*, *address*)

Conecta o *sock* em um endereço *address* remoto.

Versão assíncrona de `socket.connect()`.

sock deve ser um soquete não bloqueante.

With `SelectorEventLoop`, *address* does not need to be resolved: for `AF_INET` and `AF_INET6` sockets, `sock_connect` first checks whether *address* is already resolved by calling `socket.inet_pton()`, and uses `loop.getaddrinfo()` to resolve it if it is not.

`ProactorEventLoop`, the default event loop on Windows, does not resolve *address*. The host must already be a numeric IP address; passing a host name raises `OSError`. Resolve the address with `loop.getaddrinfo()` first, or use `loop.create_connection()`, which resolves the address on every platform.

Alterado na versão 3.5.2: With `SelectorEventLoop`, `address` no longer needs to be resolved.

Ver também

`loop.create_connection()` e `asyncio.open_connection()`.

async loop.sock_accept(*sock*)

Aceita uma conexão. Modelado baseado no método bloqueante `socket.accept()`.

O soquete deve estar vinculado a um endereço e escutando por conexões. O valor de retorno é um par `(conn, address)` onde *conn* é um *novo* objeto de soquete usável para enviar e receber dados na

conexão, e *address* é o endereço vinculado ao soquete no outro extremo da conexão.

sock deve ser um soquete não bloqueante.

Alterado na versão 3.7: Apesar deste método sempre ter sido documentado como um método de corrotina, antes do Python 3.7 ele retornava um `Future`. Desde o Python 3.7, este é um método `async def`.

Ver também

`loop.create_server()` e `start_server()`.

async loop.sock_sendfile(*sock*, *file*, *offset=0*, *count=None*, *, *fallback=True*)

Envia um arquivo usando `os.sendfile` de alto desempenho se possível. Retorna o número total de bytes enviados.

Versão assíncrona de `socket.sendfile()`.

sock deve ser um `socket` `socket.SOCK_STREAM` não bloqueante.

file deve ser um objeto arquivo regular aberto em modo binário.

offset indica a partir de onde deve iniciar a leitura do arquivo. Se especificado, *count* é o número total de bytes para transmitir, ao contrário de transmitir o arquivo até que EOF seja atingido. A posição do arquivo é sempre atualizada, mesmo quando este método levanta um erro, e `file.tell()` pode ser usado para obter o número atual de bytes enviados.

fallback, quando definido para `True`, faz asyncio ler e enviar manualmente o arquivo, quando a plataforma não suporta a chamada de sistema sendfile (por exemplo Windows ou soquete SSL no Unix).

Levanta `SendfileNotAvailableError` se o sistema não suporta chamadas de sistema *sendfile* e *fallback* é `False`.

sock deve ser um soquete não bloqueante.

Adicionado na versão 3.7.

DNS

async loop.getaddrinfo(*host*, *port*, ***, *family=0*, *type=0*, *proto=0*, *flags=0*)

Versão assíncrona de `socket.getaddrinfo()`.

async loop.getnameinfo(*sockaddr*, *flags=0*)

Versão assíncrona de `socket.getnameinfo()`.

Nota

Both *getaddrinfo* and *getnameinfo* internally utilize their synchronous versions through the loop's default thread pool executor. When this executor is saturated, these methods may experience delays, which higher-level networking libraries may report as increased timeouts. To mitigate this, consider using a custom executor for other user tasks, or setting a default executor with a larger number of workers.

Alterado na versão 3.7: Ambos os métodos *getaddrinfo* e *getnameinfo* sempre foram documentados para retornar uma corrotina, mas antes do Python 3.7 eles estavam, na verdade, retornando objetos `asyncio.Future`. A partir do Python 3.7, ambos os métodos são corrotinas.

Trabalhando com encadeamentos

async loop.connect_read_pipe(*protocol_factory*, *pipe*)

Registra o extremo da leitura de um *pipe* no laço de eventos.

protocol_factory deve ser um chamável que retorne uma implementação do protocolo asyncio.

pipe is a file-like object. See Supported pipe objects for the objects supported as *pipe*.

Retorna um par `(transport, protocol)`, onde *transport* suporta a interface **`ReadTransport`** e *protocol* é um objeto instanciado pelo *protocol_factory*.

Com o **`SelectorEventLoop`** do laço de eventos, o *pipe* é definido para modo não bloqueante.

async loop.connect_write_pipe(*protocol_factory*, *pipe*)

Registra o extremo de escrita do *pipe* no laço de eventos.

protocol_factory deve ser um chamável que retorne uma implementação do protocolo asyncio.

pipe is a file-like object. See Supported pipe objects for the objects supported as *pipe*.

Retorna um part `(transport, protocol)`, onde *transport* suporta a interface **`WriteTransport`** e *protocol* é um objeto instanciado pelo *protocol_factory*.

Com o **`SelectorEventLoop`** do laço de eventos, o *pipe* é definido para modo não bloqueante.

Supported pipe objects

These methods only work with objects the operating system can poll for readiness or perform overlapped I/O on. Regular files on disk are **not** supported on any platform. There is no asynchronous file I/O in asyncio; use **`loop.run_in_executor()`** to read and write regular files without blocking the event loop.

On Unix, with **`SelectorEventLoop`**, *pipe* must wrap one of the following:

- a pipe, such as an end of an `os.pipe()` pair or a FIFO created with `os.mkfifo()`;
- a socket;
- a character device, such as a terminal.

On Windows, where only `ProactorEventLoop` implements these methods, *pipe* must wrap a handle opened for overlapped I/O (that is, created with the `FILE_FLAG_OVERLAPPED` flag), since the handle has to be associated with an I/O completion port. Handles that were not opened for overlapped I/O are rejected. In particular, the standard streams (`sys.stdin`, `sys.stdout` and `sys.stderr`), console handles, and the pipes created by `os.pipe()` are **not** opened for overlapped I/O and therefore cannot be used with these methods.

Nota

`SelectorEventLoop` não suporta os métodos acima no Windows. Use `ProactorEventLoop` ao invés para Windows.

Ver também

Os métodos `loop.subprocess_exec()` e `loop.subprocess_shell()`.

Sinais Unix

loop.add_signal_handler(*signum*, *callback*, **args*)

Set *callback* as the handler for the *signum* signal, passing *args* as positional arguments.

A função de retorno será invocada pelo *loop*, juntamente com outras funções de retorno enfileiradas e corrotinas executáveis daquele laço de eventos. Ao contrário de tratadores de sinal registrados usando `signal.signal()`, uma função de retorno registrada com esta função tem autorização para interagir com o laço de eventos.

Levanta `ValueError` se o número do sinal é inválido ou impossível de capturar. Levanta `RuntimeError` se existe um problema definindo o tratador.

Use `functools.partial()` para passar argumentos nomeados para a *callback*.

Assim como `signal.signal()`, esta função deve ser invocada na thread principal.

loop.remove_signal_handler(*sig*)

Remove o tratador para o sinal *sig*.

Retorna `True` se o tratador de sinal foi removido, ou `False` se nenhum tratador foi definido para o sinal fornecido.

Disponibilidade: Unix.

Ver também

O módulo `signal`.

Executando código em conjuntos de threads ou processos

awaitable loop.run_in_executor(*executor*, *func*, **args*)

Arrange for *func* to be called in the specified executor passing *args* as positional arguments.

The *executor* argument should be an `concurrent.futures.Executor` instance. The default executor is used if *executor* is `None`. The default executor can be set by `loop.set_default_executor()`, otherwise, a `concurrent.futures.ThreadPoolExecutor` will be lazy-initialized and used by `run_in_executor()` if needed.

Exemplo:

```
import asyncio
import concurrent.futures

def blocking_io():
    # File operations (such as logging) can block the
    # event loop: run them in a thread pool.
    with open('/dev/urandom', 'rb') as f:
        return f.read(100)

def cpu_bound():
    # CPU-bound operations will block the event loop:
    # in general it is preferable to run them in a
    # process pool.
    return sum(i * i for i in range(10 ** 7))

async def main():
    loop = asyncio.get_running_loop()

    ## Options:

    # 1. Run in the default loop's executor:
    result = await loop.run_in_executor(
        None, blocking_io)
    print('default thread pool', result)

    # 2. Run in a custom thread pool:
    with concurrent.futures.ThreadPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool, blocking_io)
        print('custom thread pool', result)

    # 3. Run in a custom process pool:
    with concurrent.futures.ProcessPoolExecutor() as pool:
        result = await loop.run_in_executor(
            pool, cpu_bound)
        print('custom process pool', result)

    # 4. Run in a custom interpreter pool:
    with concurrent.futures.InterpreterPoolExecutor() as pool
        result = await loop.run_in_executor(
            pool, cpu_bound)
        print('custom interpreter pool', result)
```

```
if __name__ == '__main__':
    asyncio.run(main())
```

Note that the entry point guard (`if __name__ == '__main__'`) is required for option 3 due to the peculiarities of **`multiprocessing`**, which is used by **`ProcessPoolExecutor`**. See Safe importing of main module.

Este método retorna um objeto **`asyncio.Future`**.

Use **`functools.partial()`** para passar argumentos nomeados para *func*.

Alterado na versão 3.5.3: **`loop.run_in_executor()`** não configura mais o atributo `max_workers` do executor do conjunto de thread que ele cria, ao invés disso ele deixa para o executor do conjunto de thread (**`ThreadPoolExecutor`**) para setar o valor padrão.

loop.set_default_executor(*executor*)

Set *executor* as the default executor used by **`run_in_executor()`**. *executor* must be an instance of **`ThreadPoolExecutor`**, which includes **`InterpreterPoolExecutor`**.

Alterado na versão 3.11: *executor* must be an instance of **`ThreadPoolExecutor`**.

Error handling API

Permite customizar como exceções são tratadas no laço de eventos.

loop.set_exception_handler(*handler*)

Define *handler* como o novo tratador de exceções do laço de eventos.

Se *handler* for `None`, o tratador de exceções padrão será definido. Caso contrário, *handler* deve ser um chamável com a assinatura combinando `(loop, context)`, onde `loop` é a referência para o laço de eventos ativo, e `context` é um objeto `dict` contendo os detalhes da exceção

(veja a documentação `call_exception_handler()` para detalhes a respeito do contexto).

If the handler is called on behalf of a `Task` or `Handle`, it is run in the `contextvars.Context` of that task or callback handle.

Alterado na versão 3.12: The handler may be called in the `Context` of the task or handle where the exception originated.

loop.get_exception_handler()

Retorna o tratador de exceção atual, ou `None` se nenhum tratador de exceção customizado foi definido.

Adicionado na versão 3.5.2.

loop.default_exception_handler(*context*)

Tratador de exceção padrão.

Isso é chamado quando uma exceção ocorre e nenhum tratador de exceção foi definido. Isso pode ser chamado por um tratador de exceção customizado que quer passar adiante para o comportamento do tratador padrão.

parâmetro *context* tem o mesmo significado que em `call_exception_handler()`.

loop.call_exception_handler(*context*)

Chama o tratador de exceção do laço de eventos atual.

context é um objeto `dict` contendo as seguintes chaves (novas chaves podem ser introduzidas em versões futuras do Python):

- ‘message’: Mensagem de erro;
- ‘exception’ (opcional): Objeto Exception;
- ‘future’ (opcional): instância de `asyncio.Future`;
- ‘task’ (opcional): instância de `asyncio.Task`;
- ‘handle’ (opcional): instância de `asyncio.Handle`;

- ‘protocol’ (opcional): instância de Protocol;
- ‘transport’ (opcional): instância de Transport;
- ‘socket’ (opcional): instância de `socket.socket`;
- ‘source_traceback’ (optional): Traceback of the source;
- ‘handle_traceback’ (optional): Traceback of the handle;
- ‘asyncgen’ (opcional): Gerador assíncrono que causou a exceção.

Nota

This method should not be overloaded in subclassed event loops. For custom exception handling, use the `set_exception_handler()` method.

Habilitando o modo de debug

loop.get_debug()

Obtém o modo de debug (`bool`) do laço de eventos.

O valor padrão é `True` se a variável de ambiente `PYTHONASYNCIODEBUG` estiver definida para uma string não vazia, `False` caso contrário.

loop.set_debug(*enabled: bool*)

Define o modo de debug do laço de eventos.

Alterado na versão 3.7: O novo Modo de Desenvolvimento do Python agora também pode ser usado para habilitar o modo de debug.

loop.slow_callback_duration

This attribute can be used to set the minimum execution duration in seconds that is considered “slow”. When debug mode is enabled, “slow” callbacks are logged.

Default value is 100 milliseconds.

Ver também

O modo de debug de asyncio.

Running subprocesses

Métodos descritos nestas sub-seções são de baixo nível. Em código async/await regular, considere usar as funções convenientes de alto nível `asyncio.create_subprocess_shell()` e `asyncio.create_subprocess_exec()` ao invés.

Nota

No Windows, o laço de eventos padrão `ProactorEventLoop` oferece suporte pra subprocessos, enquanto `SelectorEventLoop`, não. Veja Suporte para subprocessos no Windows para detalhes.

async loop.subprocess_exec(*protocol_factory*, **args*, *stdin=subprocess.PIPE*, *stdout=subprocess.PIPE*, *stderr=subprocess.PIPE*, ***kwargs*)

Cria um subprocesso a partir de um ou mais argumentos de string especificados por *args*.

args deve ser uma lista de strings representada por:

- `str`;
- ou `bytes`, encodados na codificação do sistema de arquivos.

A primeira string especifica o programa executável, e as strings remanescentes especificam os argumentos. Juntas, argumentos em string formam o `argv` do programa.

Isto é similar a classe `subprocess.Popen` da biblioteca padrão ser chamada com `shell=False` e a lista de strings ser passada como o primeiro argumento; entretanto, onde `Popen` recebe apenas um argumento no qual é uma lista de strings, *subprocess_exec* recebe múltiplos argumentos string.

O *protocol_factory* deve ser um chamável que retorne uma subclasse da classe `asyncio.SubprocessProtocol`.

Outros parâmetros:

- *stdin* pode ser qualquer um destes:
    - a file-like object
    - an existing file descriptor (a positive integer), for example those created with `os.pipe()`
    - a constante `subprocess.PIPE` (padrão), a qual criará um novo encadeamento e conectar a ele,
    - o valor `None`, o qual fará o subprocesso herdar o descritor de arquivo deste processo
    - a constante `subprocess.DEVNULL`, a qual indica que o arquivo especial `os.devnull` será usado
- *stdout* pode ser qualquer um destes:
    - a file-like object
    - a constante `subprocess.PIPE` (padrão), a qual criará um novo encadeamento e conectar a ele,
    - o valor `None`, o qual fará o subprocesso herdar o descritor de arquivo deste processo
    - a constante `subprocess.DEVNULL`, a qual indica que o arquivo especial `os.devnull` será usado
- *stderr* pode ser qualquer um destes:
    - a file-like object
    - a constante `subprocess.PIPE` (padrão), a qual criará um novo encadeamento e conectar a ele,
    - o valor `None`, o qual fará o subprocesso herdar o descritor de arquivo deste processo
    - a constante `subprocess.DEVNULL`, a qual indica que o arquivo especial `os.devnull` será usado
    - a constante `subprocess.STDOUT`, a qual irá conectar o stream de erro padrão ao stream de saída padrão do processo

- Todos os outros argumentos nomeados são passados para `subprocess.Popen` sem interpretação, exceto *bufsize*, *universal_newlines*, *shell*, *text*, *encoding* e *errors*, os quais não devem ser especificados de forma alguma.

  A API de subprocesso `asyncio` não suporta decodificar os streams como texto. `bytes.decode()` pode ser usado para converter os bytes retornados do stream para texto.

If a file-like object passed as *stdin*, *stdout* or *stderr* represents a pipe, then the other side of this pipe should be registered with `connect_write_pipe()` or `connect_read_pipe()` for use with the event loop.

Veja o construtor da classe `subprocess.Popen` para documentação sobre outros argumentos.

Retorna um par `(transport, protocol)`, onde *transport* conforma com a classe base `asyncio.SubprocessTransport` e *protocol* é um objeto instanciado pelo *protocol_factory*.

If the transport is closed or is garbage collected, the child process is killed if it is still running.

async loop.subprocess_shell(*protocol_factory*, *cmd*, **, stdin=subprocess.PIPE, stdout=subprocess.PIPE, stderr=subprocess.PIPE, **kwargs*)

Cria um subprocesso a partir do *cmd*, o qual pode ser um `str` ou uma string de `bytes` codificada na codificação do sistema de arquivos, usando a sintaxe “shell” da plataforma.

Isto é similar a classe `subprocess.Popen` da biblioteca padrão sendo chamada com `shell=True`.

O argumento *protocol_factory* deve ser um chamável que retorna uma subclasse da classe `SubprocessProtocol`.

Veja `subprocess_exec()` para mais detalhes sobre os argumentos remanescentes.

Retorna um par `(transport, protocol)`, onde *transport* conforma com a classe base `SubprocessTransport` e *protocol* é um objeto instanciado pelo *protocol_factory*.

If the transport is closed or is garbage collected, the child process is killed if it is still running.

Nota

É responsabilidade da aplicação garantir que todos os espaços em branco e caracteres especiais sejam tratados apropriadamente para evitar vulnerabilidades de injeção shell [https://en.wikipedia.org/wiki/Shell_injection#Shell_injection] . A função `shlex.quote()` pode ser usada para escapar espaços em branco e caracteres especiais apropriadamente em strings que serão usadas para construir comandos shell.

Callback handles

class asyncio.Handle

Um objeto invólucro de função de retorno retornado por `loop.call_soon()`, `loop.call_soon_threadsafe()`.

get_context()

Return the `contextvars.Context` object associated with the handle.

Adicionado na versão 3.12.

cancel()

Cancela a função de retorno. Se a função de retorno já tiver sido cancelada ou executada, este método não tem efeito.

cancelled()

Retorna `True` se a função de retorno foi cancelada.

Adicionado na versão 3.7.

class asyncio.TimerHandle

Um objeto invólucro de função de retorno retornado por `loop.call_later()`, e `loop.call_at()`.

Esta classe é uma subclasse de `Handle`.

when()

Retorna o tempo de uma função de retorno agendada como `float` segundos.

O tempo é um timestamp absoluto, usando a mesma referência de tempo que `loop.time()`.

Adicionado na versão 3.7.

Server objects

Objetos Server são criados pelas funções `loop.create_server()`, `loop.create_unix_server()`, `start_server()`, e `start_unix_server()`.

Do not instantiate the `Server` class directly.

class asyncio.Server

Objetos *Server* são gerenciadores de contexto assíncronos. Quando usados em uma instrução `async with`, é garantido que o objeto Server está fechado e não está aceitando novas conexões quando a instrução `async with` estiver completa:

```
srv = await loop.create_server(...)

async with srv:
    # algum código
```

```
# Neste ponto, srv é fechado e não mais aceita novas conexões
```

Alterado na versão 3.7: Objeto Server é um gerenciador de contexto assíncrono desde o Python 3.7.

Alterado na versão 3.11: This class was exposed publicly as `asyncio.Server` in Python 3.9.11, 3.10.3 and 3.11.

close()

Para de servir: fecha soquetes que estavam ouvindo e define o atributo **`sockets`** para `None`.

Os soquetes que representam conexões de clientes existentes que estão chegando são deixados em aberto.

The server is closed asynchronously; use the **`wait_closed()`** coroutine to wait until the server is closed (and no more connections are active).

close_clients()

Close all existing incoming client connections.

Calls **`close()`** on all associated transports.

`close()` should be called before **`close_clients()`** when closing the server to avoid races with new clients connecting.

Adicionado na versão 3.13.

abort_clients()

Close all existing incoming client connections immediately, without waiting for pending operations to complete.

Calls **`abort()`** on all associated transports.

`close()` should be called before **`abort_clients()`** when closing the server to avoid races with new clients connecting.

Adicionado na versão 3.13.

get_loop()

Retorna o laço de eventos associado com o objeto server.

Adicionado na versão 3.7.

async start_serving()

Começa a aceitar conexões.

This method is idempotent, so it can be called when the server is already serving.

O parâmetro somente-nomeado *start_serving* para `loop.create_server()` e `asyncio.start_server()` permite criar um objeto Server que não está aceitando conexões inicialmente. Neste caso `Server.start_serving()`, ou `Server.serve_forever()` podem ser usados para fazer o Server começar a aceitar conexões.

Adicionado na versão 3.7.

async serve_forever()

Começa a aceitar conexões até que a corrotina seja cancelada. Cancelamento da task `serve_forever` causa o fechamento do servidor.

Este método pode ser chamado se o servidor já estiver aceitando conexões. Apenas uma task `serve_forever` pode existir para cada objeto *Server*.

Exemplo:

```
async def client_connected(reader, writer):
    # Communicate with the client with
    # reader/writer streams.  For example:
    await reader.readline()

async def main(host, port):
```

```
        srv = await asyncio.start_server(
            client_connected, host, port)
        await srv.serve_forever()

    asyncio.run(main('127.0.0.1', 0))
```

Adicionado na versão 3.7.

is_serving()

Retorna `True` se o servidor estiver aceitando novas conexões.

Adicionado na versão 3.7.

async wait_closed()

Wait until the **`close()`** method completes and all active connections have finished.

Alterado na versão 3.12: `wait_closed()` now waits until the server is closed and all active connections have finished. Previously, it returned immediately if the server was already closed, even if connections were still active.

sockets

List of socket-like objects, `asyncio.trsock.TransportSocket`, which the server is listening on.

Alterado na versão 3.7: Antes do Python 3.7 `Server.sockets` era usado para retornar uma lista interna de soquetes do server diretamente. No uma cópia dessa lista é retornada.

Event loop implementations

asyncio vem com duas implementações de laço de eventos diferente: **`SelectorEventLoop`** e **`ProactorEventLoop`**.

By default asyncio is configured to use **`EventLoop`**.

class asyncio.SelectorEventLoop

A subclass of **`AbstractEventLoop`** based on the **`selectors`** module.

Usa o *seletor* mais eficiente disponível para a plataforma fornecida. Também é possível configurar manualmente a implementação exata do seletor a ser utilizada:

```
import asyncio
import selectors

async def main():
    ...

loop_factory = lambda: asyncio.SelectorEventLoop(selectors.Se
asyncio.run(main(), loop_factory=loop_factory)
```

Disponibilidade: Unix, Windows.

class asyncio.ProactorEventLoop

A subclass of **`AbstractEventLoop`** for Windows that uses “I/O Completion Ports” (IOCP).

Disponibilidade: Windows.

Ver também

MSDN documentation on I/O Completion Ports [https://learn.microsoft.com/windows/win32/fileio/i-o-completion-ports].

class asyncio.EventLoop

An alias to the most efficient available subclass of **`AbstractEventLoop`** for the given platform.

It is an alias to **`SelectorEventLoop`** on Unix and **`ProactorEventLoop`** on Windows.

Adicionado na versão 3.13.

class asyncio.AbstractEventLoop

Classe base abstrata para laços de eventos compatíveis com asyncio.

The Event loop methods section lists all methods that an alternative implementation of `AbstractEventLoop` should have defined.

Exemplos

Perceba que todos os exemplos nesta seção **propositalmente** mostram como usar as APIs de baixo nível do laço de eventos, tais como `loop.run_forever()` e `loop.call_soon()`. Aplicações asyncio modernas raramente precisam ser escritas desta forma; considere usar as funções de alto nível como `asyncio.run()`.

Hello World com call_soon()

Um exemplo usando o método `loop.call_soon()` para agendar uma função de retorno. A função de retorno exibe `"Hello World"` e então para o laço de eventos:

```
import asyncio

def hello_world(loop):
    """A callback to print 'Hello World' and stop the event loop
    print('Hello World')
    loop.stop()

loop = asyncio.new_event_loop()

# Schedule a call to hello_world()
loop.call_soon(hello_world, loop)

# Blocking call interrupted by loop.stop()
try:
    loop.run_forever()
finally:
    loop.close()
```

Ver também

Um exemplo similar a Hello World criado com uma corrotina e a função `run()`.

Exibe a data atual com call_later()

Um exemplo de uma função de retorno mostrando a data atual a cada segundo. A função de retorno usa o método `loop.call_later()` para reagendar a si mesma depois de 5 segundos, e então para o laço de eventos:

```
import asyncio
import datetime as dt

def display_date(end_time, loop):
    print(dt.datetime.now())
    if (loop.time() + 1.0) < end_time:
        loop.call_later(1, display_date, end_time, loop)
    else:
        loop.stop()

loop = asyncio.new_event_loop()

# Schedule the first call to display_date()
end_time = loop.time() + 5.0
loop.call_soon(display_date, end_time, loop)

# Blocking call interrupted by loop.stop()
try:
    loop.run_forever()
finally:
    loop.close()
```

Ver também

Um exemplo similar a data atual criado com uma corrotina e a função `run()`.

Observa um descritor de arquivo por eventos de leitura

Aguarda até que um descritor de arquivo tenha recebido alguns dados usando o método `loop.add_reader()` e então fecha o laço de eventos:

```
import asyncio
from socket import socketpair

# Create a pair of connected file descriptors
rsock, wsock = socketpair()

loop = asyncio.new_event_loop()

def reader():
    data = rsock.recv(100)
    print("Received:", data.decode())

    # We are done: unregister the file descriptor
    loop.remove_reader(rsock)

    # Stop the event loop
    loop.stop()

# Register the file descriptor for read event
loop.add_reader(rsock, reader)

# Simulate the reception of data from the network
loop.call_soon(wsock.send, 'abc'.encode())

try:
    # Run the event loop
    loop.run_forever()
finally:
    # We are done. Close sockets and the event loop.
    rsock.close()
    wsock.close()
    loop.close()
```

Ver também

- Um exemplo similar usando transportes, protocolos, e o método `loop.create_connection()`.
- Outro exemplo similar usando a função de alto nível `asyncio.open_connection()` e streams.

Define tratadores de sinais para SIGINT e SIGTERM

(This `signal` example only works on Unix.)

Register handlers for signals `SIGINT` and `SIGTERM` using the `loop.add_signal_handler()` method:

```
import asyncio
import functools
import os
import signal

def ask_exit(signame, loop):
    print("got signal %s: exit" % signame)
    loop.stop()

async def main():
    loop = asyncio.get_running_loop()

    for signame in {'SIGINT', 'SIGTERM'}:
        loop.add_signal_handler(
            getattr(signal, signame),
            functools.partial(ask_exit, signame, loop))

    await asyncio.sleep(3600)

print("Event loop running for 1 hour, press Ctrl+C to interrupt.
print(f"pid {os.getpid()}: send SIGINT or SIGTERM to exit.")

asyncio.run(main())
```

Futuros

Código-fonte: Lib/asyncio/futures.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/futures.py], Lib/asyncio/base_futures.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/base_futures.py]

Objetos *Future* são usados para fazer a ponte entre **código baseado em funções de retorno (callback) de baixo nível** e código async/await de alto nível.

Funções Future

asyncio.isfuture(*obj*)

Return `True` if *obj* is either of:

- an instance of `asyncio.Future`,
- an instance of `asyncio.Task`,
- a Future-like object with a `_asyncio_future_blocking` attribute.

Adicionado na versão 3.5.

asyncio.ensure_future(*obj*, *, *loop=None*)

Retorna:

- *obj* argument as is, if *obj* is a `Future`, a `Task`, or a Future-like object (`isfuture()` is used for the test.)
- a `Task` object wrapping *obj*, if *obj* is a coroutine (`iscoroutine()` is used for the test); in this case the coroutine will be scheduled by `ensure_future()`.
- a `Task` object that would await on *obj*, if *obj* is an awaitable (`inspect.isawaitable()` is used for the test.)

If *obj* is neither of the above a `TypeError` is raised.

Importante

Save a reference to the result of this function, to avoid a task disappearing mid-execution.

See also the `create_task()` function which is the preferred way for creating new tasks or use `asyncio.TaskGroup` which keeps reference to the task internally.

Alterado na versão 3.5.1: The function accepts any awaitable object.

Descontinuado desde a versão 3.10: Deprecation warning is emitted if *obj* is not a Future-like object and *loop* is not specified and there is no running event loop.

asyncio.wrap_future(*future*, *, *loop=None*)

Wrap a `concurrent.futures.Future` object in a `asyncio.Future` object.

Descontinuado desde a versão 3.10: Deprecation warning is emitted if *future* is not a Future-like object and *loop* is not specified and there is no running event loop.

Objeto Future

class asyncio.Future(*, *loop=None*)

Um Future representa um resultado eventual de uma operação assíncrona. Não é seguro para thread.

Future é um objeto aguardável. Corrotinas podem aguardar o objeto Future até que ele tenha um resultado ou um conjunto de exceções, ou até que seja cancelado. Um Future pode ser aguardado várias vezes e o resultado é sempre o mesmo.

Normalmente, objetos Future são usados para permitir que o código baseado em funções de retorno de baixo nível (por exemplo, em

protocolos implementados usando transportes asyncio) interopere com o código async/await de alto nível.

A regra geral é nunca expor o objeto Future em API voltada para o usuário, e a maneira recomendada de criar um objeto Future objeto chamar `loop.create_future()`. Dessa forma, as implementações de laços de eventos alternativos podem injetar suas próprias versões otimizadas de um objeto Future.

Futures are generic over the type of their results.

Alterado na versão 3.7: Adicionado suporte ao módulo `contextvars`.

Descontinuado desde a versão 3.10: Aviso de descontinuidade é emitido se *loop* não é especificado e não existe nenhum laço de eventos em execução.

result()

Return the result of the Future.

If the Future is *done* and has a result set by the `set_result()` method, the result value is returned.

If the Future is *done* and has an exception set by the `set_exception()` method, this method raises the exception.

Se o futuro foi *cancelled*, este método levanta uma exceção `CancelledError`.

If the Future's result isn't yet available, this method raises an `InvalidStateError` exception.

set_result(*result*)

Mark the Future as *done* and set its result.

Raises an `InvalidStateError` error if the Future is already *done*.

set_exception(*exception*)

Mark the Future as *done* and set an exception.

Raises an `InvalidStateError` error if the Future is already *done*.

done()

Return `True` if the Future is *done*.

A Future is *done* if it was *cancelled* or if it has a result or an exception set with `set_result()` or `set_exception()` calls.

cancelled()

Return `True` if the Future was *cancelled*.

The method is usually used to check if a Future is not *cancelled* before setting a result or an exception for it:

```
if not fut.cancelled():
    fut.set_result(42)
```

add_done_callback(*callback*, *, *context=None*)

Add a callback to be run when the Future is *done*.

The *callback* is called with the Future object as its only argument.

If the Future is already *done* when this method is called, the callback is scheduled with `loop.call_soon()`.

Um argumento opcional somente-nomeado *context* permite especificar um `contextvars.Context` customizado para executar na *função de retorno*. O contexto atual é usado quando nenhum *context* é fornecido.

`functools.partial()` can be used to pass parameters to the callback, e.g.:

```
# Call 'print("Future:", fut)' when "fut" is done.
fut.add_done_callback(
    functools.partial(print, "Future:"))
```

Alterado na versão 3.7: O parâmetro somente-nomeado *context* foi adicionado. Veja **PEP 567** [https://peps.python.org/pep-0567/] para mais detalhes.

remove_done_callback(*callback*)

Remove *callback* da lista de funções de retorno.

Returns the number of callbacks removed, which is typically 1, unless a callback was added more than once.

cancel(*msg=None*)

Cancel the Future and schedule callbacks.

If the Future is already *done* or *cancelled*, return `False`. Otherwise, change the Future's state to *cancelled*, schedule the callbacks, and return `True`.

The optional string argument *msg* is passed as the argument to the `CancelledError` exception raised when a cancelled Future is awaited.

Alterado na versão 3.9: Adicionado o parâmetro *msg*.

exception()

Return the exception that was set on this Future.

The exception (or `None` if no exception was set) is returned only if the Future is *done*.

Se o futuro foi *cancelled*, este método levanta uma exceção `CancelledError`.

If the Future isn't *done* yet, this method raises an `InvalidStateError` exception.

get_loop()

Return the event loop the Future object is bound to.

Adicionado na versão 3.7.

This example creates a Future object, creates and schedules an asynchronous Task to set result for the Future, and waits until the Future has a result:

```
async def set_after(fut, delay, value):
    # Sleep for *delay* seconds.
    await asyncio.sleep(delay)

    # Set *value* as a result of *fut* Future.
    fut.set_result(value)

async def main():
    # Get the current event loop.
    loop = asyncio.get_running_loop()

    # Create a new Future object.
    fut = loop.create_future()

    # Run "set_after()" coroutine in a parallel Task.
    # We are using the low-level "loop.create_task()" API here b
    # we already have a reference to the event loop at hand.
    # Otherwise we could have just used "asyncio.create_task()".
    loop.create_task(
        set_after(fut, 1, '... world'))

    print('hello ...')

    # Wait until *fut* has a result (1 second) and print it.
    print(await fut)

asyncio.run(main())
```

Importante

The Future object was designed to mimic `concurrent.futures.Future`. Key differences include:

- unlike asyncio Futures, `concurrent.futures.Future` instances cannot be awaited.
- `asyncio.Future.result()` and `asyncio.Future.exception()` do not accept the *timeout* argument.
- `asyncio.Future.result()` and `asyncio.Future.exception()` raise an `InvalidStateError` exception when the Future is not *done*.
- Callbacks registered with `asyncio.Future.add_done_callback()` are not called immediately. They are scheduled with `loop.call_soon()` instead.
- asyncio Future is not compatible with the `concurrent.futures.wait()` and `concurrent.futures.as_completed()` functions.
- `asyncio.Future.cancel()` accepts an optional `msg` argument, but `concurrent.futures.Future.cancel()` does not.

Transports e Protocols

Prefácio

Transports and Protocols are used by the **low-level** event loop APIs such as `loop.create_connection()`. They use callback-based programming style and enable high-performance implementations of network or IPC protocols (e.g. HTTP).

Essentially, transports and protocols should only be used in libraries and frameworks and never in high-level asyncio applications.

This documentation page covers both Transports and Protocols.

Introdução

At the highest level, the transport is concerned with *how* bytes are transmitted, while the protocol determines *which* bytes to transmit (and to some extent when).

A different way of saying the same thing: a transport is an abstraction for a socket (or similar I/O endpoint) while a protocol is an abstraction for an application, from the transport's point of view.

Yet another view is the transport and protocol interfaces together define an abstract interface for using network I/O and interprocess I/O.

There is always a 1:1 relationship between transport and protocol objects: the protocol calls transport methods to send data, while the transport calls protocol methods to pass it data that has been received.

Most of connection oriented event loop methods (such as `loop.create_connection()`) usually accept a *protocol_factory* argument used to create a *Protocol* object for an accepted connection, represented by

a *Transport* object. Such methods usually return a tuple of `(transport, protocol)`.

Conteúdo

Esta página de documentação contém as seguintes seções:

- The Transports section documents asyncio **`BaseTransport`**, **`ReadTransport`**, **`WriteTransport`**, **`Transport`**, **`DatagramTransport`**, and **`SubprocessTransport`** classes.
- The Protocols section documents asyncio **`BaseProtocol`**, **`Protocol`**, **`BufferedProtocol`**, **`DatagramProtocol`**, and **`SubprocessProtocol`** classes.
- The Examples section showcases how to work with transports, protocols, and low-level event loop APIs.

Transportes

Source code: Lib/asyncio/transports.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/transports.py]

Transports are classes provided by **`asyncio`** in order to abstract various kinds of communication channels.

Transport objects are always instantiated by an asyncio event loop.

asyncio implements transports for TCP, UDP, SSL, and subprocess pipes. The methods available on a transport depend on the transport's kind.

The transport classes are not thread safe.

Transports Hierarchy

class asyncio.BaseTransport

Base class for all transports. Contains methods that all asyncio transports share.

class asyncio.WriteTransport(*BaseTransport*)

A base transport for write-only connections.

Instances of the *WriteTransport* class are returned from the `loop.connect_write_pipe()` event loop method and are also used by subprocess-related methods like `loop.subprocess_exec()`.

class asyncio.ReadTransport(*BaseTransport*)

A base transport for read-only connections.

Instances of the *ReadTransport* class are returned from the `loop.connect_read_pipe()` event loop method and are also used by subprocess-related methods like `loop.subprocess_exec()`.

class asyncio.Transport(*WriteTransport*, *ReadTransport*)

Interface representing a bidirectional transport, such as a TCP connection.

The user does not instantiate a transport directly; they call a utility function, passing it a protocol factory and other information necessary to create the transport and protocol.

Instances of the *Transport* class are returned from or used by event loop methods like `loop.create_connection()`, `loop.create_unix_connection()`, `loop.create_server()`, `loop.sendfile()`, etc.

class asyncio.DatagramTransport(*BaseTransport*)

A transport for datagram (UDP) connections.

Instances of the *DatagramTransport* class are returned from the `loop.create_datagram_endpoint()` event loop method.

class asyncio.SubprocessTransport(*BaseTransport*)

An abstraction to represent a connection between a parent and its child OS process.

Instances of the *SubprocessTransport* class are returned from event loop methods `loop.subprocess_shell()` and `loop.subprocess_exec()`.

Base Transport

BaseTransport.close()

Fecha o transporte.

If the transport has a buffer for outgoing data, buffered data will be flushed asynchronously. No more data will be received. After all buffered data is flushed, the protocol's `protocol.connection_lost()` method will be called with `None` as its argument. The transport should not be used once it is closed.

BaseTransport.is_closing()

Retorna `True` se o transporte estiver fechando ou estiver fechado.

BaseTransport.get_extra_info(*name*, *default=None*)

Return information about the transport or underlying resources it uses.

name is a string representing the piece of transport-specific information to get.

default is the value to return if the information is not available, or if the transport does not support querying it with the given third-party event loop implementation or on the current platform.

For example, the following code attempts to get the underlying socket object of the transport:

```
sock = transport.get_extra_info('socket')
if sock is not None:
    print(sock.getsockopt(...))
```

Categories of information that can be queried on some transports:

- socket:

    - `'peername'`: the remote address to which the socket is connected, result of `socket.socket.getpeername()` (`None` on error)
    - `'socket'`: `socket.socket` instance
    - `'sockname'`: the socket's own address, result of `socket.socket.getsockname()`
  - SSL socket:
    - `'compression'`: the compression algorithm being used as a string, or `None` if the connection isn't compressed; result of `ssl.SSLSocket.compression()`
    - `'cipher'`: a three-value tuple containing the name of the cipher being used, the version of the SSL protocol that defines its use, and the number of secret bits being used; result of `ssl.SSLSocket.cipher()`
    - `'peercert'`: peer certificate; result of `ssl.SSLSocket.getpeercert()`
    - `'sslcontext'`: `ssl.SSLContext` instance
    - `'ssl_object'`: `ssl.SSLObject` or `ssl.SSLSocket` instance
  - pipe:
    - `'pipe'`: pipe object
  - subprocess:
    - `'subprocess'`: `subprocess.Popen` instance

BaseTransport.set_protocol(*protocol*)

Define um novo protocolo.

Switching protocol should only be done when both protocols are documented to support the switch.

BaseTransport.get_protocol()

Retorna o protocolo atual.

Read-only Transports

ReadTransport.is_reading()

Return `True` if the transport is receiving new data.

Adicionado na versão 3.7.

ReadTransport.pause_reading()

Pause the receiving end of the transport. No data will be passed to the protocol's `protocol.data_received()` method until `resume_reading()` is called.

Alterado na versão 3.7: The method is idempotent, i.e. it can be called when the transport is already paused or closed.

ReadTransport.resume_reading()

Resume the receiving end. The protocol's `protocol.data_received()` method will be called once again if some data is available for reading.

Alterado na versão 3.7: The method is idempotent, i.e. it can be called when the transport is already reading.

Write-only Transports

WriteTransport.abort()

Close the transport immediately, without waiting for pending operations to complete. Buffered data will be lost. No more data will be received. The protocol's `protocol.connection_lost()` method will eventually be called with `None` as its argument.

WriteTransport.can_write_eof()

Return `True` if the transport supports `write_eof()`, `False` if not.

WriteTransport.get_write_buffer_size()

Return the current size of the output buffer used by the transport.

WriteTransport.get_write_buffer_limits()

Get the *high* and *low* watermarks for write flow control. Return a tuple `(low, high)` where *low* and *high* are positive number of bytes.

Use `set_write_buffer_limits()` to set the limits.

Adicionado na versão 3.4.2.

WriteTransport.set_write_buffer_limits(*high=None*, *low=None*)

Set the *high* and *low* watermarks for write flow control.

These two values (measured in number of bytes) control when the protocol's `protocol.pause_writing()` and `protocol.resume_writing()` methods are called. If specified, the low watermark must be less than or equal to the high watermark. Neither *high* nor *low* can be negative.

`pause_writing()` is called when the buffer size becomes greater than or equal to the *high* value. If writing has been paused, `resume_writing()` is called when the buffer size becomes less than or equal to the *low* value.

The defaults are implementation-specific. If only the high watermark is given, the low watermark defaults to an implementation-specific value less than or equal to the high watermark. Setting *high* to zero forces *low* to zero as well, and causes `pause_writing()` to be called whenever the buffer becomes non-empty. Setting *low* to zero causes `resume_writing()` to be called only once the buffer is empty. Use of zero for either limit is generally sub-optimal as it reduces opportunities for doing I/O and computation concurrently.

Use `get_write_buffer_limits()` to get the limits.

WriteTransport.write(*data*)

Write some *data* bytes to the transport.

This method does not block; it buffers the data and arranges for it to be sent out asynchronously.

WriteTransport.writelines(*list_of_data*)

Write a list (or any iterable) of data bytes to the transport. This is functionally equivalent to calling `write()` on each element yielded by the iterable, but may be implemented more efficiently.

WriteTransport.write_eof()

Close the write end of the transport after flushing all buffered data. Data may still be received.

This method can raise `NotImplementedError` if the transport (e.g. SSL) doesn't support half-closed connections.

Transportes de datagrama

DatagramTransport.sendto(*data*, *addr=None*)

Send the *data* bytes to the remote peer given by *addr* (a transport-dependent target address). If *addr* is `None`, the data is sent to the target address given on transport creation.

This method does not block; it buffers the data and arranges for it to be sent out asynchronously.

Alterado na versão 3.13: This method can be called with an empty bytes object to send a zero-length datagram. The buffer size calculation used for flow control is also updated to account for the datagram header.

DatagramTransport.abort()

Close the transport immediately, without waiting for pending operations to complete. Buffered data will be lost. No more data will be received. The protocol's `protocol.connection_lost()` method will eventually be called with `None` as its argument.

Transportes de Subprocesso

SubprocessTransport.get_pid()

Return the subprocess process id as an integer.

SubprocessTransport.get_pipe_transport(*fd*)

Return the transport for the communication pipe corresponding to the integer file descriptor *fd*:

- `0`: writable streaming transport of the standard input (*stdin*), or `None` if the subprocess was not created with `stdin=PIPE`
- `1`: readable streaming transport of the standard output (*stdout*), or `None` if the subprocess was not created with `stdout=PIPE`
- `2`: readable streaming transport of the standard error (*stderr*), or `None` if the subprocess was not created with `stderr=PIPE`
- other *fd*: `None`

SubprocessTransport.get_returncode()

Return the subprocess return code as an integer or `None` if it hasn't returned, which is similar to the `subprocess.Popen.returncode` attribute.

SubprocessTransport.kill()

Mata o subprocesso.

On POSIX systems, the function sends SIGKILL to the subprocess. On Windows, this method is an alias for `terminate()`.

See also `subprocess.Popen.kill()`.

SubprocessTransport.send_signal(*signal*)

Send the *signal* number to the subprocess, as in `subprocess.Popen.send_signal()`.

SubprocessTransport.terminate()

Interrompe o subprocesso.

On POSIX systems, this method sends `SIGTERM` to the subprocess. On Windows, the Windows API function **`TerminateProcess()`** is called to stop the subprocess.

See also `subprocess.Popen.terminate()`.

SubprocessTransport.close()

Kill the subprocess by calling the `kill()` method.

If the subprocess hasn't returned yet, and close transports of *stdin*, *stdout*, and *stderr* pipes.

Protocolos

Source code: Lib/asyncio/protocols.py
[https://github.com/python/cpython/tree/3.15/Lib/asyncio/protocols.py]

---

asyncio provides a set of abstract base classes that should be used to implement network protocols. Those classes are meant to be used together with transports.

Subclasses of abstract base protocol classes may implement some or all methods. All these methods are callbacks: they are called by transports on certain events, for example when some data is received. A base protocol method should be called by the corresponding transport.

Protocolos de Base

class asyncio.BaseProtocol

Base protocol with methods that all protocols share.

class asyncio.Protocol(*BaseProtocol*)

The base class for implementing streaming protocols (TCP, Unix sockets, etc).

class asyncio.BufferedProtocol(*BaseProtocol*)

A base class for implementing streaming protocols with manual control of the receive buffer.

class asyncio.DatagramProtocol(*BaseProtocol*)

The base class for implementing datagram (UDP) protocols.

class asyncio.SubprocessProtocol(*BaseProtocol*)

The base class for implementing protocols communicating with child processes (unidirectional pipes).

Base Protocol

All asyncio protocols can implement Base Protocol callbacks.

Connection Callbacks

Connection callbacks are called on all protocols, exactly once per a successful connection. All other protocol callbacks can only be called between those two methods.

BaseProtocol.connection_made(*transport*)

Chamado quando uma conexão é estabelecida.

The *transport* argument is the transport representing the connection. The protocol is responsible for storing the reference to its transport.

BaseProtocol.connection_lost(*exc*)

Chamado quanto a conexão é perdida ou fechada.

The argument is either an exception object or `None`. The latter means a regular EOF is received, or the connection was aborted or closed by this side of the connection.

Flow Control Callbacks

Flow control callbacks can be called by transports to pause or resume writing performed by the protocol.

See the documentation of the `set_write_buffer_limits()` method for more details.

BaseProtocol.pause_writing()

Called when the transport's buffer goes over the high watermark.

BaseProtocol.resume_writing()

Called when the transport's buffer drains below the low watermark.

If the buffer size equals the high watermark, `pause_writing()` is not called: the buffer size must go strictly over.

Conversely, `resume_writing()` is called when the buffer size is equal or lower than the low watermark. These end conditions are important to ensure that things go as expected when either mark is zero.

Streaming Protocols

Event methods, such as `loop.create_server()`, `loop.create_unix_server()`, `loop.create_connection()`, `loop.create_unix_connection()`, `loop.connect_accepted_socket()`, `loop.connect_read_pipe()`, and `loop.connect_write_pipe()` accept factories that return streaming protocols.

Protocol.data_received(*data*)

Called when some data is received. *data* is a non-empty bytes object containing the incoming data.

Whether the data is buffered, chunked or reassembled depends on the transport. In general, you shouldn't rely on specific semantics and instead make your parsing generic and flexible. However, data is always received in the correct order.

The method can be called an arbitrary number of times while a connection is open.

However, `protocol.eof_received()` is called at most once. Once `eof_received()` is called, `data_received()` is not called anymore.

Protocol.eof_received()

Called when the other end signals it won't send any more data (for example by calling `transport.write_eof()`, if the other end also uses asyncio).

This method may return a false value (including `None`), in which case the transport will close itself. Conversely, if this method returns a true value, the protocol used determines whether to close the transport. Since the default implementation returns `None`, it implicitly closes the connection.

Some transports, including SSL, don't support half-closed connections, in which case returning true from this method will result in the connection being closed.

State machine:

```
start -> connection_made
    [-> data_received]*
    [-> eof_received]?
-> connection_lost -> end
```

Protocolos de Streaming Bufferizados

Adicionado na versão 3.7.

Buffered Protocols can be used with any event loop method that supports Streaming Protocols.

`BufferedProtocol` implementations allow explicit manual allocation and control of the receive buffer. Event loops can then use the buffer provided by the protocol to avoid unnecessary data copies. This can result in noticeable performance improvement for protocols that receive big amounts of data. Sophisticated protocol implementations can significantly reduce the number of buffer allocations.

The following callbacks are called on **`BufferedProtocol`** instances:

BufferedProtocol.get_buffer(*sizehint*)

Chamada para alocar um novo buffer para recebimento.

sizehint is the recommended minimum size for the returned buffer. It is acceptable to return smaller or larger buffers than what *sizehint*

suggests. When set to -1, the buffer size can be arbitrary. It is an error to return a buffer with a zero size.

`get_buffer()` must return an object implementing the buffer protocol.

BufferedProtocol.buffer_updated(*nbytes*)

Chamado quando o buffer foi atualizado com os dados recebidos.

nbytes is the total number of bytes that were written to the buffer.

BufferedProtocol.eof_received()

See the documentation of the **`protocol.eof_received()`** method.

`get_buffer()` can be called an arbitrary number of times during a connection. However, **`protocol.eof_received()`** is called at most once and, if called, **`get_buffer()`** and **`buffer_updated()`** won't be called after it.

State machine:

```
start -> connection_made
    [-> get_buffer
        [-> buffer_updated]?
    ]*
    [-> eof_received]?
-> connection_lost -> end
```

Protocolos de Datagramas

Datagram Protocol instances should be constructed by protocol factories passed to the **`loop.create_datagram_endpoint()`** method.

DatagramProtocol.datagram_received(*data*, *addr*)

Called when a datagram is received. *data* is a bytes object containing the incoming data. *addr* is the address of the peer sending the data; the exact format depends on the transport.

DatagramProtocol.error_received(*exc*)

Called when a previous send or receive operation raises an `OSError`. *exc* is the `OSError` instance.

This method is called in rare conditions, when the transport (e.g. UDP) detects that a datagram could not be delivered to its recipient. In many conditions though, undeliverable datagrams will be silently dropped.

Nota

On BSD systems (macOS, FreeBSD, etc.) flow control is not supported for datagram protocols, because there is no reliable way to detect send failures caused by writing too many packets.

The socket always appears 'ready' and excess packets are dropped. An `OSError` with `errno` set to `errno.ENOBUFS` may or may not be raised; if it is raised, it will be reported to `DatagramProtocol.error_received()` but otherwise ignored.

Protocolos de Subprocesso

Subprocess Protocol instances should be constructed by protocol factories passed to the `loop.subprocess_exec()` and `loop.subprocess_shell()` methods.

SubprocessProtocol.pipe_data_received(*fd*, *data*)

Called when the child process writes data into its stdout or stderr pipe.

fd is the integer file descriptor of the pipe.

data is a non-empty bytes object containing the received data.

SubprocessProtocol.pipe_connection_lost(*fd*, *exc*)

Chamado quando um dos encadeamentos comunicando com o processo filho é fechado.

fd is the integer file descriptor that was closed.

SubprocessProtocol.process_exited()

Chamado quando o processo filho encerrou.

It can be called before `pipe_data_received()` and `pipe_connection_lost()` methods.

Exemplos

TCP Echo Server

Create a TCP echo server using the `loop.create_server()` method, send back received data, and close the connection:

```
import asyncio


class EchoServerProtocol(asyncio.Protocol):
    def connection_made(self, transport):
        peername = transport.get_extra_info('peername')
        print('Connection from {}'.format(peername))
        self.transport = transport

    def data_received(self, data):
        message = data.decode()
        print('Data received: {!r}'.format(message))

        print('Send: {!r}'.format(message))
        self.transport.write(data)

        print('Close the client socket')
        self.transport.close()


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    server = await loop.create_server(
        EchoServerProtocol,
```

```
        '127.0.0.1', 8888)

    async with server:
        await server.serve_forever()


asyncio.run(main())
```

Ver também

The TCP echo server using streams example uses the high-level **`asyncio.start_server()`** function.

TCP Echo Client

A TCP echo client using the **`loop.create_connection()`** method, sends data, and waits until the connection is closed:

```
import asyncio


class EchoClientProtocol(asyncio.Protocol):
    def __init__(self, message, on_con_lost):
        self.message = message
        self.on_con_lost = on_con_lost

    def connection_made(self, transport):
        transport.write(self.message.encode())
        print('Data sent: {!r}'.format(self.message))

    def data_received(self, data):
        print('Data received: {!r}'.format(data.decode()))

    def connection_lost(self, exc):
        print('The server closed the connection')
        self.on_con_lost.set_result(True)


async def main():
    # Get a reference to the event loop as we plan to use
```

```
    # low-level APIs.
    loop = asyncio.get_running_loop()

    on_con_lost = loop.create_future()
    message = 'Hello World!'

    transport, protocol = await loop.create_connection(
        lambda: EchoClientProtocol(message, on_con_lost),
        '127.0.0.1', 8888)

    # Wait until the protocol signals that the connection
    # is lost and close the transport.
    try:
        await on_con_lost
    finally:
        transport.close()


asyncio.run(main())
```

Ver também

The TCP echo client using streams example uses the high-level **`asyncio.open_connection()`** function.

UDP Echo Server

A UDP echo server, using the **`loop.create_datagram_endpoint()`** method, sends back received data:

```
import asyncio


class EchoServerProtocol:
    def connection_made(self, transport):
        self.transport = transport

    def datagram_received(self, data, addr):
        message = data.decode()
        print('Received %r from %s' % (message, addr))
```

```
        print('Send %r to %s' % (message, addr))
        self.transport.sendto(data, addr)


async def main():
    print("Starting UDP server")

    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    # One protocol instance will be created to serve all
    # client requests.
    transport, protocol = await loop.create_datagram_endpoint(
        EchoServerProtocol,
        local_addr=('127.0.0.1', 9999))

    try:
        await asyncio.sleep(3600)  # Serve for 1 hour.
    finally:
        transport.close()


asyncio.run(main())
```

UDP Echo Client

A UDP echo client, using the **`loop.create_datagram_endpoint()`** method, sends data and closes the transport when it receives the answer:

```
import asyncio


class EchoClientProtocol:
    def __init__(self, message, on_con_lost):
        self.message = message
        self.on_con_lost = on_con_lost
        self.transport = None

    def connection_made(self, transport):
        self.transport = transport
        print('Send:', self.message)
```

```
        self.transport.sendto(self.message.encode())

    def datagram_received(self, data, addr):
        print("Received:", data.decode())

        print("Close the socket")
        self.transport.close()

    def error_received(self, exc):
        print('Error received:', exc)

    def connection_lost(self, exc):
        print("Connection closed")
        self.on_con_lost.set_result(True)


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    on_con_lost = loop.create_future()
    message = "Hello World!"

    transport, protocol = await loop.create_datagram_endpoint(
        lambda: EchoClientProtocol(message, on_con_lost),
        remote_addr=('127.0.0.1', 9999))

    try:
        await on_con_lost
    finally:
        transport.close()


asyncio.run(main())
```

Connecting Existing Sockets

Wait until a socket receives data using the `loop.create_connection()` method with a protocol:

```
import asyncio
import socket


class MyProtocol(asyncio.Protocol):

    def __init__(self, on_con_lost):
        self.transport = None
        self.on_con_lost = on_con_lost

    def connection_made(self, transport):
        self.transport = transport

    def data_received(self, data):
        print("Received:", data.decode())

        # We are done: close the transport;
        # connection_lost() will be called automatically.
        self.transport.close()

    def connection_lost(self, exc):
        # The socket has been closed
        self.on_con_lost.set_result(True)


async def main():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()
    on_con_lost = loop.create_future()

    # Create a pair of connected sockets
    rsock, wsock = socket.socketpair()

    # Register the socket to wait for data.
    transport, protocol = await loop.create_connection(
        lambda: MyProtocol(on_con_lost), sock=rsock)

    # Simulate the reception of data from the network.
    loop.call_soon(wsock.send, 'abc'.encode())

    try:
        await protocol.on_con_lost
```

```
    finally:
        transport.close()
        wsock.close()

asyncio.run(main())
```

Ver também

The watch a file descriptor for read events example uses the low-level **`loop.add_reader()`** method to register an FD.

The register an open socket to wait for data using streams example uses high-level streams created by the **`open_connection()`** function in a coroutine.

loop.subprocess_exec() and SubprocessProtocol

An example of a subprocess protocol used to get the output of a subprocess and to wait for the subprocess exit.

The subprocess is created by the **`loop.subprocess_exec()`** method:

```
import asyncio
import sys

class DateProtocol(asyncio.SubprocessProtocol):
    def __init__(self, exit_future):
        self.exit_future = exit_future
        self.output = bytearray()
        self.pipe_closed = False
        self.exited = False

    def pipe_connection_lost(self, fd, exc):
        self.pipe_closed = True
        self.check_for_exit()

    def pipe_data_received(self, fd, data):
        self.output.extend(data)
```

```
    def process_exited(self):
        self.exited = True
        # process_exited() method can be called before
        # pipe_connection_lost() method: wait until both methods
        # called.
        self.check_for_exit()

    def check_for_exit(self):
        if self.pipe_closed and self.exited:
            self.exit_future.set_result(True)

async def get_date():
    # Get a reference to the event loop as we plan to use
    # low-level APIs.
    loop = asyncio.get_running_loop()

    code = 'import datetime as dt; print(dt.datetime.now())'
    exit_future = asyncio.Future(loop=loop)

    # Create the subprocess controlled by DateProtocol;
    # redirect the standard output into a pipe.
    transport, protocol = await loop.subprocess_exec(
        lambda: DateProtocol(exit_future),
        sys.executable, '-c', code,
        stdin=None, stderr=None)

    # Wait for the subprocess exit using the process_exited()
    # method of the protocol.
    await exit_future

    # Close the stdout pipe.
    transport.close()

    # Read the output which was collected by the
    # pipe_data_received() method of the protocol.
    data = bytes(protocol.output)
    return data.decode('ascii').rstrip()

date = asyncio.run(get_date())
print(f"Current date: {date}")
```

See also the same example written using high-level APIs.

Políticas

Aviso

Policies are deprecated and will be removed in Python 3.16. Users are encouraged to use the `asyncio.run()` function or the `asyncio.Runner` with *loop_factory* to use the desired loop implementation.

An event loop policy is a global object used to get and set the current event loop, as well as create new event loops. The default policy can be replaced with built-in alternatives to use different event loop implementations, or substituted by a custom policy that can override these behaviors.

The policy object gets and sets a separate event loop per *context*. This is per-thread by default, though custom policies could define *context* differently.

Custom event loop policies can control the behavior of `get_event_loop()`, `set_event_loop()`, and `new_event_loop()`.

Policy objects should implement the APIs defined in the `AbstractEventLoopPolicy` abstract base class.

Getting and Setting the Policy

The following functions can be used to get and set the policy for the current process:

asyncio.get_event_loop_policy()

Retorna a política de todo o processo atual.

Descontinuado desde a versão 3.14: The `get_event_loop_policy()` function is deprecated and will be removed in Python 3.16.

asyncio.set_event_loop_policy(*policy*)

Set the current process-wide policy to *policy*.

If *policy* is set to `None`, the default policy is restored.

Descontinuado desde a versão 3.14: The **`set_event_loop_policy()`** function is deprecated and will be removed in Python 3.16.

Policy Objects

The abstract event loop policy base class is defined as follows:

class asyncio.AbstractEventLoopPolicy

An abstract base class for asyncio policies.

get_event_loop()

Obtém o laço de eventos para o contexto atual.

Return an event loop object implementing the **`AbstractEventLoop`** interface.

This method should never return `None`.

Alterado na versão 3.6.

set_event_loop(*loop*)

Define o laço de eventos do contexto atual como *loop*.

new_event_loop()

Cria e retorna um novo objeto de laço de eventos.

This method should never return `None`.

Descontinuado desde a versão 3.14: The **`AbstractEventLoopPolicy`** class is deprecated and will be removed in Python 3.16.

asyncio ships with the following built-in policies:

class asyncio.DefaultEventLoopPolicy

The default asyncio policy. Uses `SelectorEventLoop` on Unix and `ProactorEventLoop` on Windows.

There is no need to install the default policy manually. asyncio is configured to use the default policy automatically.

Alterado na versão 3.8: On Windows, `ProactorEventLoop` is now used by default.

Alterado na versão 3.14: The `get_event_loop()` method of the default asyncio policy now raises a `RuntimeError` if there is no set event loop.

Descontinuado desde a versão 3.14: The `DefaultEventLoopPolicy` class is deprecated and will be removed in Python 3.16.

class asyncio.WindowsSelectorEventLoopPolicy

An alternative event loop policy that uses the `SelectorEventLoop` event loop implementation.

Disponibilidade: Windows.

Descontinuado desde a versão 3.14: The `WindowsSelectorEventLoopPolicy` class is deprecated and will be removed in Python 3.16.

class asyncio.WindowsProactorEventLoopPolicy

An alternative event loop policy that uses the `ProactorEventLoop` event loop implementation.

Disponibilidade: Windows.

Descontinuado desde a versão 3.14: The `WindowsProactorEventLoopPolicy` class is deprecated and will be removed in Python 3.16.

Custom Policies

To implement a new event loop policy, it is recommended to subclass **`DefaultEventLoopPolicy`** and override the methods for which custom behavior is wanted, e.g.:

```
class MyEventLoopPolicy(asyncio.DefaultEventLoopPolicy):

    def get_event_loop(self):
        """Get the event loop.

        This may be None or an instance of EventLoop.
        """
        loop = super().get_event_loop()
        # Do something with loop ...
        return loop

asyncio.set_event_loop_policy(MyEventLoopPolicy())
```

Suporte a plataformas

The `asyncio` module is designed to be portable, but some platforms have subtle differences and limitations due to the platforms' underlying architecture and capabilities.

Todas as plataformas

- `loop.add_reader()` and `loop.add_writer()` cannot be used to monitor file I/O.
- `loop.connect_read_pipe()` and `loop.connect_write_pipe()` cannot be used with regular files. See Supported pipe objects for the objects that are accepted on each platform.

Windows

Source code: Lib/asyncio/proactor_events.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/proactor_events.py], Lib/asyncio/windows_events.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/windows_events.py], Lib/asyncio/windows_utils.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/windows_utils.py]

Alterado na versão 3.8: On Windows, `ProactorEventLoop` is now the default event loop.

All event loops on Windows do not support the following methods:

- `loop.create_unix_connection()` and `loop.create_unix_server()` are not supported. The `socket.AF_UNIX` socket family is specific to Unix.
- `loop.add_signal_handler()` and `loop.remove_signal_handler()` are not supported.

`SelectorEventLoop` has the following limitations:

- `SelectSelector` is used to wait on socket events: it supports sockets and is limited to 512 sockets.
- `loop.add_reader()` and `loop.add_writer()` only accept socket handles (e.g. pipe file descriptors are not supported).
- Pipes are not supported, so the `loop.connect_read_pipe()` and `loop.connect_write_pipe()` methods are not implemented.
- Subprocesses are not supported, i.e. `loop.subprocess_exec()` and `loop.subprocess_shell()` methods are not implemented.

`ProactorEventLoop` has the following limitations:

- The `loop.add_reader()` and `loop.add_writer()` methods are not supported.
- `loop.connect_read_pipe()` and `loop.connect_write_pipe()` only accept a handle opened for overlapped I/O. See Supported pipe objects for which objects are supported.

The resolution of the monotonic clock on Windows is usually around 15.6 milliseconds. The best resolution is 0.5 milliseconds. The resolution depends on the hardware (availability of HPET [https://en.wikipedia.org/wiki/High_Precision_Event_Timer]) and on the Windows configuration.

Suporte para subprocesso no Windows

On Windows, the default event loop `ProactorEventLoop` supports subprocesses, whereas `SelectorEventLoop` does not.

macOS

Modern macOS versions are fully supported.

macOS <= 10.8

On macOS 10.6, 10.7 and 10.8, the default event loop uses `selectors.KqueueSelector`, which does not support character devices on these versions. The `SelectorEventLoop` can be manually configured to use `SelectSelector` or `PollSelector` to support character devices on these older versions of macOS. Example:

```
import asyncio
import selectors

selector = selectors.SelectSelector()
loop = asyncio.SelectorEventLoop(selector)
asyncio.set_event_loop(loop)
```

Extensão

A direção principal para estender o módulo **asyncio** é escrever classes de *laços de eventos* personalizados. O Asyncio tem funções auxiliares que podem ser usadas para simplificar essa tarefa.

Nota

Terceiros devem reutilizar o código assíncrono existente com cuidado. Uma versão nova de Python pode quebrar a compatibilidade com versões anteriores da parte *interna* da API.

Escrevendo um laço de eventos personalizado

asyncio.AbstractEventLoop declara muitos métodos. Implementar todos eles do zero é um trabalho tedioso.

Um laço de eventos pode herdar várias implementações de métodos da classe **asyncio.BaseEventLoop**.

Por sua vez, o sucessor deve implementar um conjunto de métodos *privados* declarados, porém não implementados, em **asyncio.BaseEventLoop**.

Por exemplo, `loop.create_connection()``verifica os argumentos, resolve endereços DNS, e chama a função ``loop._make_socket_transport()`, que deve ser implementada pela classe herdada. O método``_make_socket_transport()`` não está documentado, e é considerado parte de uma API *interna*.

Construtores privados de Future e Task

As classes `asyncio.Future` e `asyncio.Task` nunca deverão ser criadas diretamente. Por favor, as substitua pelas *factories* correspondentes: `loop.create_future()` e `loop.create_task()` ou `asyncio.create_task()`.

Porém, *laços de eventos* de terceiros podem *reutilizar* as implementações embutidas de Future e Task, obtendo um código complexo e altamente otimizado.

Por isso, segue a listagem de construtores *privados*:

Future.__init__(*, *loop=None*)

Cria uma instância de Future embutida.

loop é uma instância opcional do laço de eventos.

Task.__init__(*coro*, *, *loop=None*, *name=None*, *context=None*)

Cria uma instância embutida de Task.

loop é uma instância opcional do laço de eventos. Os outros argumentos estão descritos em `loop.create_task()`.

Alterado na versão 3.11: o argumento *context* foi adicionado

Suporte ao ciclo de vida de tarefas

Implementações de Task de terceiros devem chamar as seguintes funções para manter uma tarefa visível para `asyncio.all_tasks()` e `asyncio.current_task()`:

asyncio._register_task(*task*)

Registra uma nova *task* como gerenciada pelo *asyncio*.

Chame esta função no construtor de uma tarefa.

asyncio._unregister_task(*task*)

Cancela o registro de *task* nas estruturas internas do *asyncio*.

Chame esta função quando a tarefa estiver terminando.

asyncio._enter_task(*loop*, *task*)

Troca a tarefa atual para o argumento *task*.

Chame esta função logo antes de executar uma porção da *corrotina* embutida (`coroutine.send()` ou `coroutine.throw()`).

asyncio._leave_task(*loop*, *task*)

Troca a tarefa atual de *task* para `None`.

Chame esta função logo após executar `coroutine.send()` ou `coroutine.throw()`.

Índice da API de alto nível

Esta página lista todas as APIs asyncio de alto nível habilitadas por async/await.

Tarefas

Utilitários para executar programas asyncio, criar Tarefas, e esperar por múltiplas coisas com tempos limites.

`run()`	Cria um laço de eventos, roda uma corrotina, fecha o laço.
`Runner`	Um gerenciador de contexto que simplifica várias chamadas assíncronas de funções.
`Task`	Objeto Task.
`TaskGroup`	Um gerenciador de contexto que mantém um grupo de tarefas. Oferece uma maneira conveniente e confiável de aguardar a conclusão de todas as tarefas do grupo.
`create_task()`	Inicia uma Task asyncio e a retorna.
`current_task()`	Retorna para a Tarefa atual.

`all_tasks()`	Retorna todas as tarefas que ainda não foram concluídas em um laço de eventos.
`await sleep()`	Dorme for um número de segundos.
`await gather()`	Agenda e espera por coisas concorrentemente.
`await wait_for()`	Executa com um tempo limite.
`await shield()`	Protege contra cancelamento.
`await wait()`	Monitora para conclusão.
`timeout()`	Executa com um tempo limite. Útil nos casos em que o `wait_for` não é adequado.
`to_thread()`	Executa uma função assincronamente em uma thread separada.
`run_coroutine_threadsafe()`	Agenda uma corrotina a partir de outra thread do sistema operacional.
`for in as_completed()`	Monitora a conclusão com um loop `for`.

Exemplos

- Usando asyncio.gather() para executar coisas em paralelo.
- Usando asyncio.wait_for() para forçar um tempo limite de execução.
- Cancelamento.
- Usando asyncio.sleep().
- Veja também a página principal de documentação sobre Tarefas.

Filas

Filas devem ser usadas para distribuir trabalho entre múltiplas Tarefas asyncio, implementar pools de conexão, e padrões pub/sub.

`Queue`	Uma fila FIFO - Primeiro que entra, é o primeiro que sai.
`PriorityQueue`	Uma fila de prioridade.
`LifoQueue`	Uma fila LIFO - Último que entra, é o primeiro que sai.

Exemplos

- Usando asyncio.Queue para distribuir cargas de trabalho entre diversas Tasks.
- Veja também a Página de documentação da classe Queue.

Subprocessos

Utilitários para iniciar subprocessos e executar comandos no console.

`await create_subprocess_exec()`	Cria um subprocesso.

`await` `create_subprocess_shell()`	Executa um comando no console.

Exemplos

- Executando um comando no console.
- Veja também a documentação de subprocessos de APIs.

Streams

APIs de alto nível para trabalhar com entrada e saída de rede.

`await` `open_connection()`	Estabelece uma conexão TCP.
`await` `open_unix_connection()`	Estabelece uma conexão com soquete Unix.
`await` `start_server()`	Inicia um servidor TCP.
`await` `start_unix_server()`	Inicia um servidor com soquete Unix.
`StreamReader`	Objeto async/await de alto nível para receber dados de rede.
`StreamWriter`	Objeto async/await de alto nível para enviar dados pela rede.

Exemplos

- Exemplo de cliente TCP.
- Veja também a documentação das APIs de streams.

Sincronização

Primitivas de sincronização similares a threads, que podem ser usadas em tarefas.

`Lock`	Uma trava mutex.
`Event`	Um objeto de evento.
`Condition`	Um objeto de condição.
`Semaphore`	Um semáforo.
`BoundedSemaphore`	Um semáforo limitado.
`Barrier`	Um objeto barreira.

Exemplos

- Usando asyncio.Event.
- Usando asyncio.Barrier.
- Veja também a documentação das primitivas de sincronização de asyncio.

Exceções

`asyncio.CancelledError`	Levantado quanto a Tarefa é cancelada. Veja também `Task.cancel()`.
`asyncio.BrokenBarrierError`	Levantado quando um Barreira é quebrada. veja também `Barrier.wait()` .

Exemplos

- Manipulando CancelledError para executar código no cancelamento de uma requisição.
- Veja também a lista completa de exceções específicas de asyncio.

Índice de APIs de baixo nível

Esta página lista todas as APIs de baixo nível do asyncio.

Obtendo o laço de eventos

`asyncio.get_running_loop()`	A função **preferida** para obter o laço de eventos em execução.
`asyncio.get_event_loop()`	Obtém uma instância do laço de eventos (em execução ou atual por meio da política atual).
`asyncio.set_event_loop()`	Define o laço de eventos como atual através da política atual.
`asyncio.new_event_loop()`	Cria um novo laço de eventos.

Exemplos

- Usando asyncio.get_running_loop().

Métodos do laço de eventos

Veja também a seção principal da documentação sobre os Event loop methods.

Ciclo de vida

`loop.run_until_complete()`	Executa um Future/Task/aguardável até que esteja completo.
`loop.run_forever()`	Executa o laço de eventos para sempre.
`loop.stop()`	Para o laço de eventos.
`loop.close()`	Fecha o laço de eventos.
`loop.is_running()`	Retorna `True` se o laço de eventos estiver rodando.
`loop.is_closed()`	Retorna `True` se o laço de eventos estiver fechado.
`await loop.shutdown_asyncgens()`	Fecha geradores assíncronos.

Depuração

`loop.set_debug()`	Habilita ou desabilita o modo de debug.
`loop.get_debug()`	Obtém o modo de debug atual.

Agendando funções de retorno (callbacks)

`loop.call_soon()`	Invoca uma função de retorno brevemente.
`loop.call_soon_threadsafe()`	Uma variante segura para thread de `loop.call_soon()`.
`loop.call_later()`	Invoca uma função de retorno *após* o tempo especificado.
`loop.call_at()`	Invoca uma função de retorno *no* instante especificado.

Grupo de Thread/Interpretador/Processo

`await loop.run_in_executor()`	Executa uma função vinculada à CPU ou outra que seja bloqueante em um executor `concurrent.futures`.
`loop.set_default_executor()`	Define o executor padrão para `loop.run_in_executor()`.

Tasks e Futures

`loop.create_future()`	Cria um objeto `Future`.
`loop.create_task()`	Agenda corrotina como uma `Task`.

`loop.set_task_factory()`	Define uma factory usada por `loop.create_task()` para criar `Tasks`.
`loop.get_task_factory()`	Obtém o factory `loop.create_task()` usado para criar `Tasks`.

DNS

`await loop.getaddrinfo()`	Versão assíncrona de `socket.getaddrinfo()`.
`await loop.getnameinfo()`	Versão assíncrona de `socket.getnameinfo()`.

Rede e IPC

`await loop.create_connection()`	Abre uma conexão TCP.
`await loop.create_server()`	Cria um servidor TCP.
`await loop.create_unix_connection()`	Abre uma conexão soquete Unix.
`await loop.create_unix_server()`	Cria um servidor soquete Unix.
`await`	Envolve um `socket` em um par

`loop.connect_accepted_socket()`	`(transport, protocol)`.
`await loop.create_datagram_endpoint()`	Abre uma conexão por datagrama (UDP).
`await loop.sendfile()`	Envia um arquivo por meio de um transporte.
`await loop.start_tls()`	Atualiza uma conexão existente para TLS.
`await loop.connect_read_pipe()`	Envolve a leitura final de um encadeamento em um par `(transport, protocol)`.
`await loop.connect_write_pipe()`	Envolve a escrita final de um encadeamento em um par `(transport, protocol)`.

Soquetes

`await loop.sock_recv()`	Recebe dados do `socket`.
`await loop.sock_recv_into()`	Recebe dados do `socket` em um buffer.
`await loop.sock_recvfrom()`	Recebe um datagrama do `socket`.

`await` `loop.sock_recvfrom_into()`	Recebe um datagrama do `socket` em um buffer.
`await` `loop.sock_sendall()`	Envia dados para o `socket`.
`await` `loop.sock_sendto()`	Envia um datagrama por meio de `socket` para o endereço dado.
`await` `loop.sock_connect()`	Conecta ao `socket`.
`await` `loop.sock_accept()`	Aceita uma conexão do `socket`.
`await` `loop.sock_sendfile()`	Envia um arquivo usando o `socket`.
`loop.add_reader()`	Começa a observar um descritor de arquivo, aguardando por disponibilidade de leitura.
`loop.remove_reader()`	Interrompe o monitoramento de um descritor de arquivo, que aguarda disponibilidade de leitura.
`loop.add_writer()`	Começa a observar um descritor de arquivo, aguardando por disponibilidade para escrita.
`loop.remove_writer()`	Interrompe o monitoramento de um descritor de arquivo, que aguarda disponibilidade para escrita.

Sinais Unix

`loop.add_signal_handler()`	Adiciona um tratador para um `signal`.
`loop.remove_signal_handler()`	Remove um tratador para um `signal`.

Subprocessos

`loop.subprocess_exec()`	Inicia um subprocesso.
`loop.subprocess_shell()`	Inicia um subprocesso a partir de um comando shell.

Tratamento de erros

`loop.call_exception_handler()`	Chama o tratamento de exceção.
`loop.set_exception_handler()`	Define um novo tratador de exceção.
`loop.get_exception_handler()`	Obtém o tratador de exceção atual.
`loop.default_exception_handler()`	A implementação padrão do

	tratador de exceção.

Exemplos

- Usando asyncio.new_event_loop() e loop.run_forever().
- Usando loop.call_later().
- Usando `loop.create_connection()` para implementar um cliente-eco.
- Usando `loop.create_connection()` para conectar a um soquete.
- Usando add_reader() para monitorar um descritor de arquivo para eventos de leitura.
- Usando loop.add_signal_handler().
- Usando loop.subprocess_exec().

Transportes

Todos os transportes implementam os seguintes métodos:

`transport.close()`	Fecha o transporte.
`transport.is_closing()`	Retorna `True` se o transporte estiver fechando ou estiver fechado.
`transport.get_extra_info()`	Solicita informação a respeito do transporte.
`transport.set_protocol()`	Define um novo protocolo.
`transport.get_protocol()`	Retorna o protocolo atual.

Transportes que podem receber dados (TCP e conexões Unix, encadeamentos, etc). Retornado a partir de métodos como `loop.create_connection()`, `loop.create_unix_connection()`, `loop.connect_read_pipe()`, etc:

Realiza leitura de Transportes

`transport.is_reading()`	Retorna `True` se o transporte estiver recebendo.
`transport.pause_reading()`	Pausa o recebimento.
`transport.resume_reading()`	Continua o recebimento.

Transportes que podem enviar dados (TCP e conexões Unix, encadeamentos, etc). Retornado a partir de métodos como `loop.create_connection()`, `loop.create_unix_connection()`, `loop.connect_write_pipe()`, etc:

Realiza escrita de Transportes

`transport.write()`	Escreve dados para o transporte.
`transport.writelines()`	Escreve buffers para o transporte.
`transport.can_write_eof()`	Retorna `True` se o transporte suporta o envio de EOF.

`transport.write_eof()`	Fecha e envia EOF após descarregar dados que estavam no buffer.
`transport.abort()`	Fecha o transporte imediatamente.
`transport.get_write_buffer_size()`	Retorna o tamanho atual do buffer de saída.
`transport.get_write_buffer_limits()`	Retorna marcas d'água alta e baixa para controle do fluxo de escrita.
`transport.set_write_buffer_limits()`	Define novas marcas d'água alta e baixa para controle do fluxo de escrita.

Transporte retornado por `loop.create_datagram_endpoint()`:

Transportes de datagrama

`transport.sendto()`	Envia dados para o par remoto.
`transport.abort()`	Fecha o transporte imediatamente.

Abstração de transporte de baixo nível sobre subprocessos. Retornado por `loop.subprocess_exec()` e `loop.subprocess_shell()`:

Transportes de Subprocesso

`transport.get_pid()`	Retorna o process id do subprocesso.
`transport.get_pipe_transport()`	Retorna o transporte para o encadeamento de comunicação requisitada (*stdin*, *stdout*, ou *stderr*).
`transport.get_returncode()`	Retorna o código de retorno do subprocesso.
`transport.kill()`	Mata o subprocesso.
`transport.send_signal()`	Envia um sinal para o subprocesso.
`transport.terminate()`	Interrompe o subprocesso.
`transport.close()`	Mata o subprocesso e fecha todos os encadeamentos.

Protocolos

Classes de protocolos podem implementar os seguintes **métodos de função de retorno**:

`callback connection_made()`	Chamado quando uma conexão é estabelecida.
`callback connection_lost()`	Chamado quanto a conexão é

	perdida ou fechada.
`callback` **`pause_writing()`**	Chamado quando o buffer de transporte ultrapassa a marca de nível alto d'água.
`callback` **`resume_writing()`**	Chamado quando o buffer de transporte drena abaixo da marca de nível baixo d'agua.

Protocolos de Streaming (TCP, Soquetes Unix, Encadeamentos)

`callback` **`data_received()`**	Chamado quando algum dado é recebido.
`callback` **`eof_received()`**	Chamado quando um EOF é recebido.

Protocolos de Streaming Bufferizados

`callback` **`get_buffer()`**	Chamada para alocar um novo buffer para recebimento.
`callback` **`buffer_updated()`**	Chamado quando o buffer foi atualizado com os dados recebidos.
`callback` **`eof_received()`**	Chamado quando um EOF é recebido.

Protocolos de Datagramas

`callback` **`datagram_received()`**	Chamado quando um datagrama é recebido.
`callback` **`error_received()`**	Chamado quando uma operação de envio ou recebimento anterior levanta um **`OSError`**.

Protocolos de Subprocesso

`callback` **`pipe_data_received()`**	Chamado quando o processo filho escreve dados no seu encadeamento *stdout* ou *stderr*.
`callback` **`pipe_connection_lost()`**	Chamado quando um dos encadeamentos comunicando com o processo filho é fechado.
`callback` **`process_exited()`**	Chamado quando o processo filho sai. Isso pode ser chamado antes dos métodos **`pipe_data_received()`** e **`pipe_connection_lost()`**.

Políticas de laço de eventos

Política é um mecanismo de baixo nível para alterar o comportamento de funções, similar a **`asyncio.get_event_loop()`**. Veja também a seção principal de políticas para mais detalhes.

Acessando Políticas

`asyncio.get_event_loop_policy()`	Retorna a política de todo o processo atual.
`asyncio.set_event_loop_policy()`	Define uma nova política para todo o processo.
`AbstractEventLoopPolicy`	Classe base para objetos de política.

Desenvolvendo com asyncio

A programação assíncrona é diferente da programação “sequencial” clássica.

Esta página lista erros e armadilhas comuns e explica como evitá-los.

Modo de Depuração

Por padrão, o asyncio é executado em modo de produção. Para facilitar o desenvolvimento, o asyncio fornece um modo de depuração.

Há várias maneiras de habilitar o modo de depuração do asyncio:

- Definindo a variável de ambiente `PYTHONASYNCIODEBUG` como `1`.
- Usando o Modo de Desenvolvimento do Python.
- Passando `debug=True` para `asyncio.run()`.
- Chamando `loop.set_debug()`.

Além de habilitar o modo de depuração, considere também:

- setting the log level of the asyncio logger to `logging.DEBUG`, for example the following snippet of code can be run at startup of the application:

  ```
  logging.basicConfig(level=logging.DEBUG)
  ```

- configurando o módulo `warnings` para exibir avisos `ResourceWarning`. Uma forma de fazer isso é usando a opção de linha de comando: `-W default`.

Quando o modo de depuração está habilitado:

- Muitas APIs do asyncio que não são thread-safe (como os métodos `loop.call_soon()` e `loop.call_at()`) levantam uma exceção se

forem chamadas a partir de uma thread incorreta.
- O tempo de execução de um seletor de E/S é registrado se demorar muito para executar a operação E/S.
- Funções de retorno demorando mais do que 100 milissegundos são registradas. O atributo `loop.slow_callback_duration` pode ser usado para definir a duração de execução mínima em segundos para se considerada “devagar”.

Concorrência e Múltiplas Threads

Um laço de eventos é executado em uma thread (geralmente a thread principal) e executa todos as funções de retorno e Tasks nessa mesma thread. Enquanto uma Task está em execução no laço de eventos, nenhuma outra Task pode ser executada na mesma thread. Quando uma Task executa uma expressão `await`, a Task em execução é suspensa, e o laço de eventos passa a executar a próxima Task.

Para agendar uma callback de outra thread do SO, o método `loop.call_soon_threadsafe()` deve ser usado. Exemplo:

```
loop.call_soon_threadsafe(callback, *args)
```

Quase todos os objetos do asyncio não são seguros para threads, o que normalmente não é um problema, a menos que exista código que funcione com eles fora de uma Task ou de um callback. Se houver a necessidade de esse tipo de código chamar uma API de baixo nível do asyncio, o método `loop.call_soon_threadsafe()` deve ser usado, por exemplo:

```
loop.call_soon_threadsafe(fut.cancel)
```

Para agendar um objeto corrotina a partir de uma thread diferente do sistema operacional, a função `run_coroutine_threadsafe()` deve ser usada. Ela retorna um `concurrent.futures.Future`. para acessar o resultado:

```
async def coro_func():
     return await asyncio.sleep(1, 42)
```

```
# Later in another OS thread:

future = asyncio.run_coroutine_threadsafe(coro_func(), loop)
# Wait for the result:
result = future.result()
```

Para manipular sinais, o laço de eventos deve ser executado na thread principal.

O método `loop.run_in_executor()` pode ser usado com um `concurrent.futures.ThreadPoolExecutor` ou `InterpreterPoolExecutor` para executar o código de bloqueio em uma thread de sistema operacional diferente, sem bloquear a thread de sistema operacional na qual o laço de eventos é executado.

There is currently no way to schedule coroutines or callbacks directly from a different process (such as one started with `multiprocessing`). The Event loop methods section lists APIs that can read from pipes and watch file descriptors without blocking the event loop. In addition, asyncio's Subprocess APIs provide a way to start a process and communicate with it from the event loop. Lastly, the aforementioned `loop.run_in_executor()` method can also be used with a `concurrent.futures.ProcessPoolExecutor` to execute code in a different process.

Executando código bloqueante

O código bloqueante (vinculada à CPU) não deve ser chamado diretamente. Por exemplo, se uma função executa um cálculo pesado de CPU durante 1 segundo, todas as Tasks concorrentes do asyncio e as operações de E/S serão atrasadas por 1 segundo.

Um executor pode ser usado para executar uma tarefa em uma thread diferente, incluindo em um interpretador diferente, ou até mesmo em um processo diferente, para evitar bloquear a thread do sistema operacional onde o laço de eventos está sendo executado. Consulte o método `loop.run_in_executor()` para mais detalhes.

Gerando logs

asyncio usa o módulo **`logging`** e todo registro é feito via o registrador `"asyncio"`.

O nível padrão de log é **`logging.INFO`**, que pode ser facilmente ajustado:

```
logging.getLogger("asyncio").setLevel(logging.WARNING)
```

Logging de rede pode bloquear o laço de eventos. Recomenda-se usar uma thread separada para lidar com os logs ou usar E/S não bloqueante. Por exemplo, veja Dealing with handlers that block.

Detectar corrotinas nunca aguardadas

Quando uma função de corrotina é chamada, mas não é aguardada (por exemplo, `coro()` em vez de `await coro()`) ou a corrotina não é agendada com **`asyncio.create_task()`**, o asyncio emitirá um **`RuntimeWarning`**:

```
import asyncio

async def test():
    print("never scheduled")

async def main():
    test()

asyncio.run(main())
```

Saída:

```
test.py:7: RuntimeWarning: coroutine 'test' was never awaited
  test()
```

Saída no modo de depuração:

```
test.py:7: RuntimeWarning: coroutine 'test' was never awaited
Coroutine created at (most recent call last)
  File "../t.py", line 9, in <module>
```

```
    asyncio.run(main(), debug=True)

  < .. >

  File "../t.py", line 7, in main
    test()
  test()
```

A correção usual é aguardar a corrotina ou chamar a função `asyncio.create_task()`:

```
async def main():
    await test()
```

Detectar exceções nunca recuperadas

Se um `Future.set_exception()` é chamado, mas o objeto Future nunca é aguardado, a exceção nunca será propagada para o código do usuário. Nesse caso, o asyncio emitirá uma mensagem de log quando o objeto Future for coletado pela coleta de lixo.

Exemplo de uma exceção não tratada:

```
import asyncio

async def bug():
    raise Exception("not consumed")

async def main():
    asyncio.create_task(bug())

asyncio.run(main())
```

Saída:

```
Task exception was never retrieved
future: <Task finished coro=<bug() done, defined at test.py:3>
  exception=Exception('not consumed')>

Traceback (most recent call last):
```

```
  File "test.py", line 4, in bug
    raise Exception("not consumed")
Exception: not consumed
```

Enable the debug mode to get the traceback where the task was created:

```
asyncio.run(main(), debug=True)
```

Saída no modo de depuração:

```
Task exception was never retrieved
future: <Task finished coro=<bug() done, defined at test.py:3>
    exception=Exception('not consumed') created at asyncio/tasks

source_traceback: Object created at (most recent call last):
  File "../t.py", line 9, in <module>
    asyncio.run(main(), debug=True)

< .. >

Traceback (most recent call last):
  File "../t.py", line 4, in bug
    raise Exception("not consumed")
Exception: not consumed
```

Asynchronous generators best practices

Writing correct and efficient asyncio code requires awareness of certain pitfalls. This section outlines essential best practices that can save you hours of debugging.

Close asynchronous generators explicitly

It is recommended to manually close the asynchronous generator. If a generator exits early - for example, due to an exception raised in the body of an `async for` loop - its asynchronous cleanup code may run in an unexpected context. This can occur after the tasks it depends on have completed, or during the event loop shutdown when the async-generator's garbage collection hook is called.

To avoid this, explicitly close the generator by calling its **aclose()** method, or use the **contextlib.aclosing()** context manager:

```
import asyncio
import contextlib

async def gen():
  yield 1
  yield 2

async def func():
  async with contextlib.aclosing(gen()) as g:
    async for x in g:
      break  # Don't iterate until the end

asyncio.run(func())
```

As noted above, the cleanup code for these asynchronous generators is deferred. The following example demonstrates that the finalization of an asynchronous generator can occur in an unexpected order:

```
import asyncio
work_done = False

async def cursor():
    try:
        yield 1
    finally:
        assert work_done

async def rows():
    global work_done
    try:
        yield 2
    finally:
        await asyncio.sleep(0.1) # imitate some async work
        work_done = True


async def main():
    async for c in cursor():
        async for r in rows():
```

```
            break
        break

asyncio.run(main())
```

For this example, we get the following output:

```
unhandled exception during asyncio.run() shutdown
task: <Task finished name='Task-3' coro=<<async_generator_athrow
Traceback (most recent call last):
  File "example.py", line 6, in cursor
    yield 1
asyncio.exceptions.CancelledError

During handling of the above exception, another exception occurr

Traceback (most recent call last):
  File "example.py", line 8, in cursor
    assert work_done
           ^^^^^^^^^
AssertionError
```

The `cursor()` asynchronous generator was finalized before the `rows` generator - an unexpected behavior.

The example can be fixed by explicitly closing the `cursor` and `rows` async-generators:

```
async def main():
    async with contextlib.aclosing(cursor()) as cursor_gen:
        async for c in cursor_gen:
            async with contextlib.aclosing(rows()) as rows_gen:
                async for r in rows_gen:
                    break
            break
```

Create asynchronous generators only when the event loop is running

It is recommended to create asynchronous generators only after the event loop has been created.

To ensure that asynchronous generators close reliably, the event loop uses the `sys.set_asyncgen_hooks()` function to register callback functions. These callbacks update the list of running asynchronous generators to keep it in a consistent state.

When the `loop.shutdown_asyncgens()` function is called, the running generators are stopped gracefully and the list is cleared.

The asynchronous generator invokes the corresponding system hook during its first iteration. At the same time, the generator records that the hook has been called and does not call it again.

Therefore, if iteration begins before the event loop is created, the event loop will not be able to add the generator to its list of active generators because the hooks are set after the generator attempts to call them. Consequently, the event loop will not be able to terminate the generator if necessary.

Considere o exemplo a seguir:

```
import asyncio

async def agenfn():
    try:
        yield 10
    finally:
        await asyncio.sleep(0)


with asyncio.Runner() as runner:
    agen = agenfn()
    print(runner.run(anext(agen)))
    del agen
```

Saída:

```
10
Exception ignored while closing generator <async_generator objec
```

```
Traceback (most recent call last):
  File "example.py", line 13, in <module>
    del agen
        ^^^^
RuntimeError: async generator ignored GeneratorExit
```

This example can be fixed as follows:

```
import asyncio

async def agenfn():
    try:
        yield 10
    finally:
        await asyncio.sleep(0)

async def main():
    agen = agenfn()
    print(await anext(agen))
    del agen

asyncio.run(main())
```

Avoid concurrent iteration and closure of the same generator

Async generators may be reentered while another `__anext__()` / `athrow()` / `aclose()` call is in progress. This may lead to an inconsistent state of the async generator and can cause errors.

Let's consider the following example:

```
import asyncio

async def consumer():
    for idx in range(100):
        await asyncio.sleep(0)
        message = yield idx
        print('received', message)

async def amain():
    agenerator = consumer()
```

```
    await agenerator.asend(None)

    fa = asyncio.create_task(agenerator.asend('A'))
    fb = asyncio.create_task(agenerator.asend('B'))
    await fa
    await fb

asyncio.run(amain())
```

Saída:

```
received A
Traceback (most recent call last):
  File "test.py", line 38, in <module>
    asyncio.run(amain())
    ~~~~~~~~~~~^^^^^^^^^
  File "Lib/asyncio/runners.py", line 204, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "Lib/asyncio/runners.py", line 127, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "Lib/asyncio/base_events.py", line 719, in run_until_comp
    return future.result()
           ~~~~~~~~~~~~~^^
  File "test.py", line 36, in amain
    await fb
RuntimeError: anext(): asynchronous generator is already running
```

Therefore, it is recommended to avoid using asynchronous generators in parallel tasks or across multiple event loops.

asyncio and free-threaded Python

asyncio uses an event loop as a scheduler to enable highly efficient concurrency by switching between tasks to allow non-blocking I/O operations. This results in better performance for I/O-bound use cases. It also allows off-loading CPU-bound work to a thread or process pool, but that is still limited by the global interpreter lock in CPython.

However, in free-threaded Python, the GIL is disabled and Python can run true multi-threaded code. This means that asyncio can now take advantage of multiple CPU cores without the limitations imposed by the GIL.

Since Python 3.14, asyncio has first-class support for free-threaded Python, and the implementation of asyncio is safe to use in a multi-threaded environment.

A single event loop on one core can handle many connections concurrently, but the Python code that runs to handle each one still executes serially. Once requests involve a non-trivial amount of per-request computation, that handling becomes the bottleneck, and a single core can no longer keep up. Combining asyncio with threads is most useful here: by running an event loop per thread, the handling of different requests can run in parallel across multiple CPU cores. It is also useful when you need to run blocking or CPU-bound code from an asyncio application.

Ver também

Scaling asyncio on Free-Threaded Python [https://labs.quansight.org/blog/scaling-asyncio-on-free-threaded-python], a blog post by Kumar Aditya which explains the internal changes that make asyncio safe and efficient under free-threaded Python, together with benchmarks of the resulting improvements.

Thread safety considerations

While asyncio is designed to be thread-safe in a free-threaded Python environment, there are still some considerations to keep in mind when using asyncio with threads:

1. **Event loop**: Each thread should have its own event loop which should not be shared across threads. This ensures that the event loop can manage its own tasks and callbacks without interference from other threads.
2. **Task management**: Tasks and futures created in one thread should not be awaited or manipulated from another thread.
3. **Thread-safe APIs**: When interacting with asyncio from multiple threads, it's important to use thread-safe APIs provided by asyncio, such as `asyncio.run_coroutine_threadsafe()` for submitting coroutines to an event loop from another thread. If you need to call a callback from a different thread, you can use `loop.call_soon_threadsafe()` to schedule it safely.
4. **Synchronization**: The synchronization primitives provided by asyncio (like `asyncio.Lock` and `asyncio.Event`) are not designed to be used across threads. If you need to synchronize between threads, you should use the synchronization primitives from the `threading` module instead.

Using asyncio with threads

asyncio supports running one event loop per thread, which allows you to take advantage of multiple CPU cores in a free-threaded Python environment. Each thread can run its own event loop, and tasks can be scheduled on those loops independently.

Here's an example of how to use asyncio with threads:

```
import asyncio
import threading

async def worker(name: str) -> None:
```

```
    print(f"Worker {name} starting")
    await asyncio.sleep(1)
    print(f"Worker {name} done")

def run_loop(name: str) -> None:
    asyncio.run(worker(name))

threads = [
    threading.Thread(target=run_loop, args=(f"T{i}",))
    for i in range(4)
]
for t in threads:
    t.start()
for t in threads:
    t.join()
```

In this example, each thread creates its own event loop with `asyncio.run()` and runs a coroutine on it. The threads execute concurrently, and in a free-threaded build they can run on separate CPU cores in parallel.

Producer/consumer across threads

When a regular (non-asyncio) thread needs to hand work to an asyncio event loop running in another thread, use a thread-safe primitive such as `queue.Queue` rather than `asyncio.Queue`, which is only safe within a single event loop.:

```
import asyncio
import queue
import threading

def producer(q: queue.Queue[int]) -> None:
    for i in range(5):
        print(f"Producing {i}")
        q.put(i)
    q.shutdown()

async def consumer(q: queue.Queue[int]) -> None:
    while True:
        try:
            item = q.get_nowait()
```

```
        except queue.Empty:
            await asyncio.sleep(0.1)
            continue
        except queue.ShutDown:
            break
        print(f"Consumed {item}")
        await asyncio.sleep(item)

q: queue.Queue[int] = queue.Queue()
consumer_thread = threading.Thread(
    target=lambda: asyncio.run(consumer(q))
)
consumer_thread.start()
producer(q)
consumer_thread.join()
```

The producer runs on the main thread while the consumer runs inside an event loop on its own thread, yet they communicate safely through `queue.Queue`. When the queue is empty the consumer sleeps briefly and tries again. When the producer is done it calls **`shutdown()`**, which causes subsequent **`get_nowait()`** calls to raise **`queue.ShutDown`** so the consumer can exit cleanly.

`socket` — Interface de rede de baixo nível

Código-fonte: Lib/socket.py [https://github.com/python/cpython/tree/3.15/Lib/socket.py]

Este módulo provê acesso à interface de *soquete* do BSD. Está disponível em todos os sistemas modernos Unix, Windows, MacOS e provavelmente outras plataformas.

Nota

Algum comportamento pode depender da plataforma, pois as chamadas são feitas para as APIs de soquete do sistema operacional.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

A interface Python é uma transliteração direta da chamada de sistema e interface de biblioteca de soquetes do Unix para o estilo orientado a objetos do Python: a função `socket()` retorna um *objeto socket* cujos métodos implementam as diversas chamadas de sistema de soquetes. Os tipos de parâmetro são consideravelmente de nível mais alto que os da interface em C: assim como as operações `read()` e `write()` em arquivos Python, a alocação de buffer em operações de recebimento é automática, e o comprimento do buffer é implícito em operações de envio.

Ver também

Módulo `socketserver`
Classes que simplificam a escrita de servidores de rede.

Módulo `ssl`

Um invólucro de TLS/SSL para objetos socket.

Famílias de soquete

Dependendo do sistema e das opções de construção, várias famílias de soquetes são suportadas por este módulo.

O formato de endereço requerido por um objeto socket em particular é selecionado automaticamente com base na família de endereços especificada quando o objeto socket foi criado. Endereços de socket são representados da seguinte forma:

- O endereço de um socket `AF_UNIX` vinculado a um nó do sistema de arquivos é representado como uma string, usando a codificação do sistema de arquivos e o tratador de erros `'surrogateescape'` (veja **PEP 383** [https://peps.python.org/pep-0383/]). Um endereço no espaço de nomes abstrato do Linux é retornado como um objeto bytes ou similar com um byte nulo inicial; observe que os sockets neste espaço de nomes podem se comunicar com sockets normais do sistema de arquivos, então programas destinados a rodar no Linux podem precisar lidar com ambos os tipos de endereço. Uma string ou um objeto bytes ou similar pode ser usado para qualquer tipo de endereço ao passá-lo como um argumento.

  Alterado na versão 3.3: Previously, `AF_UNIX` socket paths were assumed to use UTF-8 encoding.

  Alterado na versão 3.5: Writable bytes-like object is now accepted.

- A pair `(host, port)` is used for the `AF_INET` address family, where *host* is a string representing either a hostname in internet domain notation like `'daring.cwi.nl'` or an IPv4 address like `'100.50.200.5'`, and *port* is an integer.

  - For IPv4 addresses, two special forms are accepted instead of a host address: `''` represents **`INADDR_ANY`**, which is used to bind to all interfaces, and the string `'<broadcast>'` represents **`INADDR_BROADCAST`**. This behavior is not compatible with IPv6, therefore, you may want to avoid these if you intend to support IPv6 with your Python programs.

- For **`AF_INET6`** address family, a four-tuple `(host, port, flowinfo, scope_id)` is used, where *flowinfo* and *scope_id* represent the `sin6_flowinfo` and `sin6_scope_id` members in **`struct sockaddr_in6`** in C. For **`socket`** module methods, *flowinfo* and *scope_id* can be omitted just for backward compatibility. Note, however, omission of *scope_id* can cause problems in manipulating scoped IPv6 addresses.

  Alterado na versão 3.7: For multicast addresses (with *scope_id* meaningful) *address* may not contain `%scope_id` (or `zone id`) part. This information is superfluous and may be safely omitted (recommended).

- **`AF_NETLINK`** sockets are represented as pairs `(pid, groups)`.

- Linux-only support for TIPC is available using the **`AF_TIPC`** address family. TIPC is an open, non-IP based networked protocol designed for use in clustered computer environments. Addresses are represented by a tuple, and the fields depend on the address type. The general tuple form is `(addr_type, v1, v2, v3 [, scope])`, where:

  - *addr_type* is one of **`TIPC_ADDR_NAMESEQ`**, **`TIPC_ADDR_NAME`**, or **`TIPC_ADDR_ID`**.

  - *scope* is one of **`TIPC_ZONE_SCOPE`**, **`TIPC_CLUSTER_SCOPE`**, and **`TIPC_NODE_SCOPE`**.

  - If *addr_type* is **`TIPC_ADDR_NAME`**, then *v1* is the server type, *v2* is the port identifier, and *v3* should be 0.

If *addr_type* is **TIPC_ADDR_NAMESEQ**, then *v1* is the server type, *v2* is the lower port number, and *v3* is the upper port number.

If *addr_type* is **TIPC_ADDR_ID**, then *v1* is the node, *v2* is the reference, and *v3* should be set to 0.

- A tuple `(interface, )` is used for the **AF_CAN** address family, where *interface* is a string representing a network interface name like `'can0'`. The network interface name `''` can be used to receive packets from all network interfaces of this family.
  - **CAN_ISOTP** protocol requires a tuple `(interface, rx_addr, tx_addr)` where both additional parameters are unsigned long integer that represent a CAN identifier (standard or extended).
  - **CAN_J1939** protocol requires a tuple `(interface, name, pgn, addr)` where additional parameters are 64-bit unsigned integer representing the ECU name, a 32-bit unsigned integer representing the Parameter Group Number (PGN), and an 8-bit integer representing the address.
- A string or a tuple `(id, unit)` is used for the **SYSPROTO_CONTROL** protocol of the **PF_SYSTEM** family. The string is the name of a kernel control using a dynamically assigned ID. The tuple can be used if ID and unit number of the kernel control are known or if a registered ID is used.

  Adicionado na versão 3.3.
- **AF_BLUETOOTH** supports the following protocols and address formats:
  - **BTPROTO_L2CAP** accepts a tuple `(bdaddr, psm[, cid[, bdaddr_type]])` where:
    - `bdaddr` is a string specifying the Bluetooth address.
    - `psm` is an integer specifying the Protocol/Service Multiplexer.
    - `cid` is an optional integer specifying the Channel Identifier. If not given, defaults to zero.

        - `bdaddr_type` is an optional integer specifying the address type; one of **`BDADDR_BREDR`** (default), **`BDADDR_LE_PUBLIC`**, **`BDADDR_LE_RANDOM`**.

      Alterado na versão 3.14: Added `cid` and `bdaddr_type` fields.

    - **`BTPROTO_RFCOMM`** accepts `(bdaddr, channel)` where `bdaddr` is the Bluetooth address as a string and `channel` is an integer.

    - **`BTPROTO_HCI`** accepts a format that depends on your OS.

        - On Linux it accepts an integer `device_id` or a tuple `(device_id, [channel])` where `device_id` specifies the number of the Bluetooth device, and `channel` is an optional integer specifying the HCI channel (**`HCI_CHANNEL_RAW`** by default).
        - On FreeBSD, NetBSD and DragonFly BSD it accepts `bdaddr` where `bdaddr` is the Bluetooth address as a string.

      Alterado na versão 3.2: NetBSD and DragonFlyBSD support added.

      Alterado na versão 3.13.3: FreeBSD support added.

      Alterado na versão 3.14: Added `channel` field. `device_id` not packed in a tuple is now accepted.

    - **`BTPROTO_SCO`** accepts `bdaddr` where `bdaddr` is the Bluetooth address as a string or a **`bytes`** object. (ex. `'12:23:34:45:56:67'` or `b'12:23:34:45:56:67'`)

      Alterado na versão 3.14: FreeBSD support added.

- **`AF_ALG`** is a Linux-only socket based interface to Kernel cryptography. An algorithm socket is configured with a tuple of two to four elements `(type, name [, feat [, mask]])`, where:

    - *type* is the algorithm type as string, e.g. `aead`, `hash`, `skcipher` or `rng`.

- *name* is the algorithm name and operation mode as string, e.g. `sha256`, `hmac(sha256)`, `cbc(aes)` or `drbg_nopr_ctr_aes256`.
- *feat* and *mask* are unsigned 32bit integers.

Disponibilidade: Linux >= 2.6.38.

Some algorithm types require more recent Kernels.

Adicionado na versão 3.6.

- **`AF_VSOCK`** allows communication between virtual machines and their hosts. The sockets are represented as a `(CID, port)` tuple where the context ID or CID and port are integers.

  Disponibilidade: Linux >= 3.9

  See *vsock(7)* *[https://manpages.debian.org/vsock(7)]*

  Adicionado na versão 3.7.

- **`AF_PACKET`** is a low-level interface directly to network devices. The addresses are represented by the tuple `(ifname, proto[, pkttype[, hatype[, addr]]])` where:

  - *ifname* - String specifying the device name.
  - *proto* - The Ethernet protocol number. May be **`ETH_P_ALL`** to capture all protocols, one of the ETHERTYPE_* constants or any other Ethernet protocol number.
  - *pkttype* - Optional integer specifying the packet type:
    - `PACKET_HOST` (the default) - Packet addressed to the local host.
    - `PACKET_BROADCAST` - Physical-layer broadcast packet.
    - `PACKET_MULTICAST` - Packet sent to a physical-layer multicast address.
    - `PACKET_OTHERHOST` - Packet to some other host that has been caught by a device driver in promiscuous mode.
    - `PACKET_OUTGOING` - Packet originating from the local host that is looped back to a packet socket.

- *hatype* - Optional integer specifying the ARP hardware address type.
- *addr* - Optional bytes-like object specifying the hardware physical address, whose interpretation depends on the device.

Disponibilidade: Linux >= 2.2.

- **`AF_QIPCRTR`** is a Linux-only socket based interface for communicating with services running on co-processors in Qualcomm platforms. The address family is represented as a `(node, port)` tuple where the *node* and *port* are non-negative integers.

  Disponibilidade: Linux >= 4.7.

  Adicionado na versão 3.8.

- **`IPPROTO_UDPLITE`** is a variant of UDP which allows you to specify what portion of a packet is covered with the checksum. It adds two socket options that you can change. `self.setsockopt(IPPROTO_UDPLITE, UDPLITE_SEND_CSCOV, length)` will change what portion of outgoing packets are covered by the checksum and `self.setsockopt(IPPROTO_UDPLITE, UDPLITE_RECV_CSCOV, length)` will filter out packets which cover too little of their data. In both cases `length` should be in `range(8, 2**16, 8)`.

  Such a socket should be constructed with `socket(AF_INET, SOCK_DGRAM, IPPROTO_UDPLITE)` for IPv4 or `socket(AF_INET6, SOCK_DGRAM, IPPROTO_UDPLITE)` for IPv6.

  Disponibilidade: Linux >= 2.6.20, FreeBSD >= 10.1

  Adicionado na versão 3.9.

- **`AF_HYPERV`** is a Windows-only socket based interface for communicating with Hyper-V hosts and guests. The address family is represented as a `(vm_id, service_id)` tuple where the `vm_id` and `service_id` are UUID strings.

The `vm_id` is the virtual machine identifier or a set of known VMID values if the target is not a specific virtual machine. Known VMID constants defined on `socket` are:

- `HV_GUID_ZERO`
- `HV_GUID_BROADCAST`
- `HV_GUID_WILDCARD` - Used to bind on itself and accept connections from all partitions.
- `HV_GUID_CHILDREN` - Used to bind on itself and accept connection from child partitions.
- `HV_GUID_LOOPBACK` - Used as a target to itself.
- `HV_GUID_PARENT` - When used as a bind accepts connection from the parent partition. When used as an address target it will connect to the parent partition.

The `service_id` is the service identifier of the registered service.

Adicionado na versão 3.12.

If you use a hostname in the *host* portion of IPv4/v6 socket address, the program may show a nondeterministic behavior, as Python uses the first address returned from the DNS resolution. The socket address will be resolved differently into an actual IPv4/v6 address, depending on the results from DNS resolution and/or the host configuration. For deterministic behavior use a numeric address in *host* portion.

All errors raise exceptions. The normal exceptions for invalid argument types and out-of-memory conditions can be raised. Errors related to socket or address semantics raise **`OSError`** or one of its subclasses.

Non-blocking mode is supported through **`setblocking()`**. A generalization of this based on timeouts is supported through **`settimeout()`**.

Conteúdo do módulo

The module **`socket`** exports the following elements.

Exceções

exception socket.error

Um apelido descontinuado de **`OSError`**.

Alterado na versão 3.3: Seguindo a **PEP 3151** [https://peps.python.org/pep-3151/], esta classe foi transformada em um apelido de **`OSError`**.

exception socket.herror

A subclass of **`OSError`**, this exception is raised for address-related errors, i.e. for functions that use *h_errno* in the POSIX C API, including **`gethostbyname_ex()`** and **`gethostbyaddr()`**. The accompanying value is a pair `(h_errno, string)` representing an error returned by a library call. *h_errno* is a numeric value, while *string* represents the description of *h_errno*, as returned by the **`hstrerror()`** C function.

Alterado na versão 3.3: This class was made a subclass of **`OSError`**.

exception socket.gaierror

A subclass of **`OSError`**, this exception is raised for address-related errors by **`getaddrinfo()`** and **`getnameinfo()`**. The accompanying value is a pair `(error, string)` representing an error returned by a library call. *string* represents the description of *error*, as returned by the **`gai_strerror()`** C function. The numeric *error* value will match one of the **`EAI_*`** constants defined in this module.

Alterado na versão 3.3: This class was made a subclass of **`OSError`**.

exception socket.timeout

A deprecated alias of **`TimeoutError`**.

A subclass of **`OSError`**, this exception is raised when a timeout occurs on a socket which has had timeouts enabled via a prior call to **`settimeout()`** (or implicitly through **`setdefaulttimeout()`**). The accompanying value is a string whose value is currently always “timed out”.

Alterado na versão 3.3: This class was made a subclass of `OSError`.

Alterado na versão 3.10: Esta classe foi feita como um apelido de `TimeoutError`.

Constantes

The AF_* and SOCK_* constants are now **`AddressFamily`** and **`SocketKind`** `IntEnum` collections.

Adicionado na versão 3.4.

socket.AF_UNIX
socket.AF_INET
socket.AF_INET6

These constants represent the address (and protocol) families, used for the first argument to `socket()`. If the `AF_UNIX` constant is not defined then this protocol is unsupported. More constants may be available depending on the system.

socket.AF_UNSPEC

`AF_UNSPEC` means that `getaddrinfo()` should return socket addresses for any address family (either IPv4, IPv6, or any other) that can be used.

socket.SOCK_STREAM
socket.SOCK_DGRAM
socket.SOCK_RAW
socket.SOCK_RDM
socket.SOCK_SEQPACKET

These constants represent the socket types, used for the second argument to `socket()`. More constants may be available depending on the system. (Only `SOCK_STREAM` and `SOCK_DGRAM` appear to be generally useful.)

socket.SOCK_CLOEXEC
socket.SOCK_NONBLOCK

These two constants, if defined, can be combined with the socket types and allow you to set some flags atomically (thus avoiding possible race conditions and the need for separate calls).

Ver também

Secure File Descriptor Handling [https://udrepper.livejournal.com/20407.html] for a more thorough explanation.

Disponibilidade: Linux >= 2.6.27.

Adicionado na versão 3.2.

SO_*
socket.SOMAXCONN
MSG_*
SOL_*
SCM_*
IPPROTO_*
IPPORT_*
INADDR_*
IP_*
IPV6_*
EAI_*
AI_*
NI_*
TCP_*

Many constants of these forms, documented in the Unix documentation on sockets and/or the IP protocol, are also defined in the socket module. They are generally used in arguments to the `setsockopt()` and `getsockopt()` methods of socket objects. In most cases, only those symbols that are defined in the Unix header files are defined; for a few symbols, default values are provided.

Alterado na versão 3.6: `SO_DOMAIN`, `SO_PROTOCOL`, `SO_PEERSEC`, `SO_PASSSEC`, `TCP_USER_TIMEOUT`, `TCP_CONGESTION` were added.

Alterado na versão 3.6.5: Added support for `TCP_FASTOPEN`, `TCP_KEEPCNT` on Windows platforms when available.

Alterado na versão 3.7: `TCP_NOTSENT_LOWAT` was added.

Added support for `TCP_KEEPIDLE`, `TCP_KEEPINTVL` on Windows platforms when available.

Alterado na versão 3.10: `IP_RECVTOS` was added. Added `TCP_KEEPALIVE`. On MacOS this constant can be used in the same way that `TCP_KEEPIDLE` is used on Linux.

Alterado na versão 3.11: Added `TCP_CONNECTION_INFO`. On MacOS this constant can be used in the same way that `TCP_INFO` is used on Linux and BSD.

Alterado na versão 3.12: Added `SO_RTABLE` and `SO_USER_COOKIE`. On OpenBSD and FreeBSD respectively those constants can be used in the same way that `SO_MARK` is used on Linux. Also added missing TCP socket options from Linux: `TCP_MD5SIG`, `TCP_THIN_LINEAR_TIMEOUTS`, `TCP_THIN_DUPACK`, `TCP_REPAIR`, `TCP_REPAIR_QUEUE`, `TCP_QUEUE_SEQ`, `TCP_REPAIR_OPTIONS`, `TCP_TIMESTAMP`, `TCP_CC_INFO`, `TCP_SAVE_SYN`, `TCP_SAVED_SYN`, `TCP_REPAIR_WINDOW`, `TCP_FASTOPEN_CONNECT`, `TCP_ULP`, `TCP_MD5SIG_EXT`, `TCP_FASTOPEN_KEY`, `TCP_FASTOPEN_NO_COOKIE`, `TCP_ZEROCOPY_RECEIVE`, `TCP_INQ`, `TCP_TX_DELAY`. Added `IP_PKTINFO`, `IP_UNBLOCK_SOURCE`, `IP_BLOCK_SOURCE`, `IP_ADD_SOURCE_MEMBERSHIP`, `IP_DROP_SOURCE_MEMBERSHIP`.

Alterado na versão 3.13: Added `SO_BINDTOIFINDEX`. On Linux this constant can be used in the same way that `SO_BINDTODEVICE` is used, but with the index of a network interface instead of its name.

Alterado na versão 3.14: Added missing `IP_FREEBIND`, `IP_RECVERR`, `IPV6_RECVERR`, `IP_RECVTTL`, and `IP_RECVORIGDSTADDR` on Linux.

Alterado na versão 3.14: Added support for `TCP_QUICKACK` on Windows platforms when available.

Alterado na versão 3.15: `IPV6_HDRINCL` was added. Added support for `SO_PASSRIGHTS` on Linux platforms when available.

socket.AF_CAN
socket.PF_CAN
SOL_CAN_*
CAN_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.6.25, NetBSD >= 8.

Adicionado na versão 3.3.

Alterado na versão 3.11: NetBSD support was added.

Alterado na versão 3.14: Restored missing `CAN_RAW_ERR_FILTER` on Linux.

socket.CAN_BCM
CAN_BCM_*

CAN_BCM, in the CAN protocol family, is the broadcast manager (BCM) protocol. Broadcast manager constants, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.6.25.

Nota

The **`CAN_BCM_CAN_FD_FRAME`** flag is only available on Linux >= 4.8.

Adicionado na versão 3.4.

socket.CAN_RAW_FD_FRAMES

Enables CAN FD support in a CAN_RAW socket. This is disabled by default. This allows your application to send both CAN and CAN FD frames; however, you must accept both CAN and CAN FD frames when reading from the socket.

This constant is documented in the Linux documentation.

Disponibilidade: Linux >= 3.6.

Adicionado na versão 3.5.

socket.CAN_RAW_JOIN_FILTERS

Joins the applied CAN filters such that only CAN frames that match all given CAN filters are passed to user space.

This constant is documented in the Linux documentation.

Disponibilidade: Linux >= 4.1.

Adicionado na versão 3.9.

socket.CAN_ISOTP

CAN_ISOTP, in the CAN protocol family, is the ISO-TP (ISO 15765-2) protocol. ISO-TP constants, documented in the Linux documentation.

Disponibilidade: Linux >= 2.6.25.

Adicionado na versão 3.7.

socket.CAN_J1939

CAN_J1939, in the CAN protocol family, is the SAE J1939 protocol. J1939 constants, documented in the Linux documentation.

Disponibilidade: Linux >= 5.4.

Adicionado na versão 3.9.

socket.AF_DIVERT
socket.PF_DIVERT

These two constants, documented in the FreeBSD divert(4) manual page, are also defined in the socket module.

Disponibilidade: FreeBSD >= 14.0.

Adicionado na versão 3.12.

socket.AF_PACKET
socket.PF_PACKET
PACKET_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.2.

socket.ETH_P_ALL

ETH_P_ALL can be used in the **socket** constructor as *proto* for the **AF_PACKET** family in order to capture every packet, regardless of protocol.

For more information, see the *packet(7)* *[https://manpages.debian.org/packet(7)]* manpage.

Disponibilidade: Linux.

Adicionado na versão 3.12.

socket.AF_RDS
socket.PF_RDS
socket.SOL_RDS
RDS_*

Many constants of these forms, documented in the Linux documentation, are also defined in the socket module.

Disponibilidade: Linux >= 2.6.30.

Adicionado na versão 3.3.

socket.SIO_RCVALL
socket.SIO_KEEPALIVE_VALS
socket.SIO_LOOPBACK_FAST_PATH
RCVALL_*

Constants for Windows' WSAIoctl(). The constants are used as arguments to the `ioctl()` method of socket objects.

Alterado na versão 3.6: `SIO_LOOPBACK_FAST_PATH` was added.

TIPC_*

TIPC related constants, matching the ones exported by the C socket API. See the TIPC documentation for more information.

socket.AF_ALG
socket.SOL_ALG
ALG_*

Constants for Linux Kernel cryptography.

Disponibilidade: Linux >= 2.6.38.

Adicionado na versão 3.6.

socket.AF_VSOCK
socket.IOCTL_VM_SOCKETS_GET_LOCAL_CID
VMADDR*
SO_VM*

Constants for Linux host/guest communication.

Disponibilidade: Linux >= 4.8.

Adicionado na versão 3.7.

socket.AF_LINK

Disponibilidade: BSD, macOS.

Adicionado na versão 3.4.

socket.has_ipv6

This constant contains a boolean value which indicates if IPv6 is supported on this platform.

socket.AF_BLUETOOTH

socket.BTPROTO_L2CAP
socket.BTPROTO_RFCOMM
socket.BTPROTO_HCI
socket.BTPROTO_SCO

Integer constants for use with Bluetooth addresses.

socket.BDADDR_ANY
socket.BDADDR_LOCAL

These are string constants containing Bluetooth addresses with special meanings. For example, `BDADDR_ANY` can be used to indicate any address when specifying the binding socket with `BTPROTO_RFCOMM`.

socket.BDADDR_BREDR
socket.BDADDR_LE_PUBLIC
socket.BDADDR_LE_RANDOM

These constants describe the Bluetooth address type when binding or connecting a `BTPROTO_L2CAP` socket.

Disponibilidade: Linux, FreeBSD

Adicionado na versão 3.14.

socket.SOL_RFCOMM
socket.SOL_L2CAP
socket.SOL_HCI
socket.SOL_SCO
socket.SOL_BLUETOOTH

Used in the level argument to the `setsockopt()` and `getsockopt()` methods of Bluetooth socket objects.

`SOL_BLUETOOTH` is only available on Linux. Other constants are available if the corresponding protocol is supported.

SO_L2CAP_*
socket.L2CAP_LM
L2CAP_LM_*
SO_RFCOMM_*

RFCOMM_LM_*
SO_SCO_*
SO_BTH_*
BT_*

Used in the option name and value argument to the **setsockopt()** and **getsockopt()** methods of Bluetooth socket objects.

BT_* and **L2CAP_LM** are only available on Linux. **SO_BTH_*** are only available on Windows. Other constants may be available on Linux and various BSD platforms.

Adicionado na versão 3.14.

socket.HCI_FILTER
socket.HCI_TIME_STAMP
socket.HCI_DATA_DIR
socket.SO_HCI_EVT_FILTER
socket.SO_HCI_PKT_FILTER

Option names for use with **BTPROTO_HCI**. Availability and format of the option values depend on platform.

Alterado na versão 3.14: Added **SO_HCI_EVT_FILTER** and **SO_HCI_PKT_FILTER** on NetBSD and DragonFly BSD. Added **HCI_DATA_DIR** on FreeBSD, NetBSD and DragonFly BSD.

socket.HCI_DEV_NONE

The `device_id` value used to create an HCI socket that isn't specific to a single Bluetooth adapter.

Disponibilidade: Linux

Adicionado na versão 3.14.

socket.HCI_CHANNEL_RAW
socket.HCI_CHANNEL_USER
socket.HCI_CHANNEL_MONITOR
socket.HCI_CHANNEL_CONTROL
socket.HCI_CHANNEL_LOGGING

Possible values for `channel` field in the **`BTPROTO_HCI`** address.

Disponibilidade: Linux

Adicionado na versão 3.14.

socket.AF_QIPCRTR

Constant for Qualcomm's IPC router protocol, used to communicate with service providing remote processors.

Disponibilidade: Linux >= 4.7.

socket.SCM_CREDS2
socket.LOCAL_CREDS
socket.LOCAL_CREDS_PERSISTENT

LOCAL_CREDS and LOCAL_CREDS_PERSISTENT can be used with SOCK_DGRAM, SOCK_STREAM sockets, equivalent to Linux/DragonFlyBSD SO_PASSCRED, while LOCAL_CREDS sends the credentials at first read, LOCAL_CREDS_PERSISTENT sends for each read, SCM_CREDS2 must be then used for the latter for the message type.

Adicionado na versão 3.11.

Disponibilidade: FreeBSD.

socket.SO_INCOMING_CPU

Constant to optimize CPU locality, to be used in conjunction with **`SO_REUSEPORT`**.

Adicionado na versão 3.11.

Disponibilidade: Linux >= 3.9

socket.SO_REUSEPORT_LB

Constant to enable duplicate address and port bindings with load balancing.

Adicionado na versão 3.14.

Disponibilidade: FreeBSD >= 12.0

socket.AF_HYPERV
socket.HV_PROTOCOL_RAW
socket.HVSOCKET_CONNECT_TIMEOUT
socket.HVSOCKET_CONNECT_TIMEOUT_MAX
socket.HVSOCKET_CONNECTED_SUSPEND
socket.HVSOCKET_ADDRESS_FLAG_PASSTHRU
socket.HV_GUID_ZERO
socket.HV_GUID_WILDCARD
socket.HV_GUID_BROADCAST
socket.HV_GUID_CHILDREN
socket.HV_GUID_LOOPBACK
socket.HV_GUID_PARENT

Constants for Windows Hyper-V sockets for host/guest communications.

Disponibilidade: Windows.

Adicionado na versão 3.12.

socket.ETHERTYPE_ARP
socket.ETHERTYPE_IP
socket.ETHERTYPE_IPV6
socket.ETHERTYPE_VLAN

IEEE 802.3 protocol number [https://www.iana.org/assignments/ieee-802-numbers/ieee-802-numbers.txt]. constants.

Disponibilidade: Linux, FreeBSD, macOS.

Adicionado na versão 3.12.

socket.SHUT_RD
socket.SHUT_WR
socket.SHUT_RDWR

These constants are used by the `shutdown()` method of socket objects.

Disponibilidade: not WASI.

Funções

Criação de sockets

The following functions all create socket objects.

The `socket` class constructor creates a new socket directly; see Socket Objects for its parameters and full description.

socket.socketpair([*family*[, *type*[, *proto*]]])

Build a pair of connected socket objects using the given address family, socket type, and protocol number. Address family, socket type, and protocol number are as for the `socket()` function. The default family is `AF_UNIX` if defined on the platform; otherwise, the default is `AF_INET`.

The newly created sockets are non-inheritable.

Alterado na versão 3.2: The returned socket objects now support the whole socket API, rather than a subset.

Alterado na versão 3.4: The returned sockets are now non-inheritable.

Alterado na versão 3.5: Windows support added.

socket.create_connection(*address*, *timeout=GLOBAL_DEFAULT*, *source_address=None*, *, *all_errors=False*)

Connect to a TCP service listening on the internet *address* (a 2-tuple `(host, port)`), and return the socket object. This is a higher-level function than `socket.connect()`: if *host* is a non-numeric hostname, it will try to resolve it for both `AF_INET` and `AF_INET6`, and then try to connect to all possible addresses in turn until a connection succeeds. This makes it easy to write clients that are compatible to both IPv4 and IPv6.

Passing the optional *timeout* parameter will set the timeout on the socket instance before attempting to connect. If no *timeout* is supplied, the global default timeout setting returned by `getdefaulttimeout()` is used.

If supplied, *source_address* must be a 2-tuple `(host, port)` for the socket to bind to as its source address before connecting. If host or port are ‘’ or 0 respectively the OS default behavior will be used.

When a connection cannot be created, an exception is raised. By default, it is the exception from the last address in the list. If *all_errors* is `True`, it is an `ExceptionGroup` containing the errors of all attempts.

Alterado na versão 3.2: source_address foi adicionado.

Alterado na versão 3.11: all_errors was added.

socket.create_server(*address*, *, *family=AF_INET*, *backlog=None*, *reuse_port=False*, *dualstack_ipv6=False*)

Convenience function which creates a TCP socket bound to *address* (a 2-tuple `(host, port)`) and returns the socket object.

family should be either `AF_INET` or `AF_INET6`. *backlog* is the queue size passed to `socket.listen()`; if not specified , a default reasonable value is chosen. *reuse_port* dictates whether to set the **SO_REUSEPORT** socket option.

If *dualstack_ipv6* is true, *family* is `AF_INET6` and the platform supports it the socket will be able to accept both IPv4 and IPv6 connections, else it will raise `ValueError`. Most POSIX platforms and Windows are supposed to support this functionality. When this functionality is enabled the address returned by `socket.getpeername()` when an IPv4 connection occurs will be an IPv6 address represented as an IPv4-mapped IPv6 address. If *dualstack_ipv6* is false it will explicitly disable this functionality on platforms that enable it by default (e.g. Linux). This parameter can be used in conjunction with `has_dualstack_ipv6()`:

```
import socket

addr = ("", 8080)  # all interfaces, port 8080
if socket.has_dualstack_ipv6():
    s = socket.create_server(addr, family=socket.AF_INET6, du
else:
    s = socket.create_server(addr)
```

Nota

On POSIX platforms the **`SO_REUSEADDR`** socket option is set in order to immediately reuse previous sockets which were bound on the same *address* and remained in TIME_WAIT state.

Adicionado na versão 3.8.

socket.has_dualstack_ipv6()

Return `True` if the platform supports creating a TCP socket which can handle both IPv4 and IPv6 connections.

Adicionado na versão 3.8.

socket.fromfd(*fd*, *family*, *type*, *proto=0*)

Duplicate the file descriptor *fd* (an integer as returned by a file object's **`fileno()`** method) and build a socket object from the result. Address family, socket type and protocol number are as for the **`socket()`** function. The file descriptor should refer to a socket, but this is not checked — subsequent operations on the object may fail if the file descriptor is invalid. This function is rarely needed, but can be used to get or set socket options on a socket passed to a program as standard input or output (such as a server started by the Unix inet daemon). The socket is assumed to be in blocking mode.

The newly created socket is non-inheritable.

Alterado na versão 3.4: The returned socket is now non-inheritable.

socket.fromshare(*data*)

Instantiate a socket from data obtained from the **socket.share()** method. The socket is assumed to be in blocking mode.

Disponibilidade: Windows.

Adicionado na versão 3.3.

Outras funções

The **socket** module also offers various network-related services:

socket.close(*fd*)

Close a socket file descriptor. This is like **os.close()**, but for sockets. On some platforms (most notably Windows) **os.close()** does not work for socket file descriptors.

Adicionado na versão 3.7.

socket.getaddrinfo(*host*, *port*, *family=AF_UNSPEC*, *type=0*, *proto=0*, *flags=0*)

This function wraps the C function `getaddrinfo` of the underlying system.

Translate the *host*/*port* argument into a sequence of 5-tuples that contain all the necessary arguments for creating a socket connected to that service. *host* is a domain name, a string representation of an IPv4/v6 address or `None`. *port* is a string service name such as `'http'`, a numeric port number or `None`. By passing `None` as the value of *host* and *port*, you can pass `NULL` to the underlying C API.

The *family*, *type* and *proto* arguments can be optionally specified in order to provide options and limit the list of addresses returned. Pass their default values (**AF_UNSPEC**, 0, and 0, respectively) to not limit the results. See the note below for details.

The *flags* argument can be one or several of the `AI_*` constants, and will influence how results are computed and returned. For example,

AI_NUMERICHOST will disable domain name resolution and will raise an error if *host* is a domain name.

The function returns a list of 5-tuples with the following structure:

```
(family, type, proto, canonname, sockaddr)
```

In these tuples, *family*, *type*, *proto* are all integers and are meant to be passed to the **socket()** function. *canonname* will be a string representing the canonical name of the *host* if **AI_CANONNAME** is part of the *flags* argument; else *canonname* will be empty. *sockaddr* is a tuple describing a socket address, whose format depends on the returned *family* and flags Python was compiled with, and is meant to be passed to the **socket.connect()** method.

sockaddr can be one of the following:

- a `(address, port)` 2-tuple for **AF_INET**
- a `(address, port, flowinfo, scope_id)` 4-tuple for **AF_INET6** if Python was compiled with `--enable-ipv6` (the default)
- a 2-tuple containing raw data for **AF_INET6** if Python was compiled with `--disable-ipv6`

Nota

If you intend to use results from **getaddrinfo()** to create a socket (rather than, for example, retrieve *canonname*), consider limiting the results by *type* (e.g. **SOCK_STREAM** or **SOCK_DGRAM**) and/or *proto* (e.g. **IPPROTO_TCP** or **IPPROTO_UDP**) that your application can handle.

The behavior with default values of *family*, *type*, *proto* and *flags* is system-specific.

Many systems (for example, most Linux configurations) will return a sorted list of all matching addresses. These addresses should generally be tried in order until a connection succeeds (possibly tried in parallel, for example, using a Happy Eyeballs [https://en.wikipedia.org/wiki/Happy_Eyeballs] algorithm). In these cases,

limiting the *type* and/or *proto* can help eliminate unsuccessful or unusable connection attempts.

Some systems will, however, only return a single address. (For example, this was reported on Solaris and AIX configurations.) On these systems, limiting the *type* and/or *proto* helps ensure that this address is usable.

Levanta um evento de auditoria `socket.getaddrinfo` com os argumentos `host`, `port`, `family`, `type`, `protocol`.

The following example fetches address information for a hypothetical TCP connection to `example.org` on port 80 (results may differ on your system if IPv6 isn't enabled):

```
>>> socket.getaddrinfo("example.org", 80, proto=socket.IPPROT
[(socket.AF_INET6, socket.SOCK_STREAM,
 6, '', ('2606:2800:220:1:248:1893:25c8:1946', 80, 0, 0)),
 (socket.AF_INET, socket.SOCK_STREAM,
 6, '', ('93.184.216.34', 80))]
```

Alterado na versão 3.2: parameters can now be passed using keyword arguments.

Alterado na versão 3.7: for IPv6 multicast addresses, string representing an address will not contain `%scope_id` part.

socket.getfqdn([*name*])

Return a fully qualified domain name for *name*. If *name* is omitted or empty, it is interpreted as the local host. To find the fully qualified name, the hostname returned by **`gethostbyaddr()`** is checked, followed by aliases for the host, if available. The first name which includes a period is selected. In case no fully qualified domain name is available and *name* was provided, it is returned unchanged. If *name* was empty or equal to `'0.0.0.0'`, the hostname from **`gethostname()`** is returned.

socket.gethostbyname(*hostname*)

Translate a host name to IPv4 address format. The IPv4 address is returned as a string, such as `'100.50.200.5'`. If the host name is an IPv4 address itself it is returned unchanged. See **`gethostbyname_ex()`** for a more complete interface. **`gethostbyname()`** does not support IPv6 name resolution, and **`getaddrinfo()`** should be used instead for IPv4/v6 dual stack support.

Levanta um evento de auditoria `socket.gethostbyname` com o argumento `hostname`.

Disponibilidade: not WASI.

socket.gethostbyname_ex(*hostname*)

Translate a host name to IPv4 address format, extended interface. Return a 3-tuple `(hostname, aliaslist, ipaddrlist)` where *hostname* is the host's primary host name, *aliaslist* is a (possibly empty) list of alternative host names for the same address, and *ipaddrlist* is a list of IPv4 addresses for the same interface on the same host (often but not always a single address). **`gethostbyname_ex()`** does not support IPv6 name resolution, and **`getaddrinfo()`** should be used instead for IPv4/v6 dual stack support.

Levanta um evento de auditoria `socket.gethostbyname` com o argumento `hostname`.

Disponibilidade: not WASI.

socket.gethostname()

Return a string containing the hostname of the machine where the Python interpreter is currently executing.

Levanta um evento de auditoria `socket.gethostname` sem argumentos.

Note: **`gethostname()`** doesn't always return the fully qualified domain name; use **`getfqdn()`** for that.

Disponibilidade: not WASI.

socket.gethostbyaddr(*ip_address*)

Return a 3-tuple `(hostname, aliaslist, ipaddrlist)` where *hostname* is the primary host name responding to the given *ip_address*, *aliaslist* is a (possibly empty) list of alternative host names for the same address, and *ipaddrlist* is a list of IPv4/v6 addresses for the same interface on the same host (most likely containing only a single address). To find the fully qualified domain name, use the function **`getfqdn()`**. **`gethostbyaddr()`** supports both IPv4 and IPv6.

Levanta um evento de auditoria `socket.gethostbyaddr` com o argumento `ip_address`.

Disponibilidade: not WASI.

socket.getnameinfo(*sockaddr*, *flags*)

Translate a socket address *sockaddr* into a 2-tuple `(host, port)`. Depending on the settings of *flags*, the result can contain a fully qualified domain name or numeric address representation in *host*. Similarly, *port* can contain a string port name or a numeric port number.

For IPv6 addresses, `%scope_id` is appended to the host part if *sockaddr* contains meaningful *scope_id*. Usually this happens for multicast addresses.

For more information about *flags* you can consult *getnameinfo(3)* *[https://manpages.debian.org/getnameinfo(3)]*.

Levanta um evento de auditoria `socket.getnameinfo` com o argumento `sockaddr`.

Disponibilidade: not WASI.

socket.getprotobyname(*protocolname*)

Translate an internet protocol name (for example, `'icmp'`) to a constant suitable for passing as the (optional) third argument to the **`socket()`** function. This is usually only needed for sockets opened in “raw” mode

(`SOCK_RAW`); for the normal socket modes, the correct protocol is chosen automatically if the protocol is omitted or zero.

Disponibilidade: not WASI.

socket.getservbyname(*servicename*[, *protocolname*])

Translate an internet service name and protocol name to a port number for that service. The optional protocol name, if given, should be `'tcp'` or `'udp'`, otherwise any protocol will match.

Levanta um evento de auditoria `socket.getservbyname` com os argumentos `servicename`, `protocolname`.

Disponibilidade: not WASI.

socket.getservbyport(*port*[, *protocolname*])

Translate an internet port number and protocol name to a service name for that service. The optional protocol name, if given, should be `'tcp'` or `'udp'`, otherwise any protocol will match.

Levanta um evento de auditoria `socket.getservbyport` com os argumentos `port` e `protocolname`.

Disponibilidade: not WASI.

socket.ntohl(*x*)

Convert 32-bit positive integers from network to host byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 4-byte swap operation.

socket.ntohs(*x*)

Convert 16-bit positive integers from network to host byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 2-byte swap operation.

Alterado na versão 3.10: Raises `OverflowError` if *x* does not fit in a 16-bit unsigned integer.

socket.htonl(*x*)

Convert 32-bit positive integers from host to network byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 4-byte swap operation.

socket.htons(*x*)

Convert 16-bit positive integers from host to network byte order. On machines where the host byte order is the same as network byte order, this is a no-op; otherwise, it performs a 2-byte swap operation.

Alterado na versão 3.10: Raises **`OverflowError`** if *x* does not fit in a 16-bit unsigned integer.

socket.inet_aton(*ip_string*)

Convert an IPv4 address from dotted-quad string format (for example, '123.45.67.89') to 32-bit packed binary format, as a bytes object four characters in length. This is useful when conversing with a program that uses the standard C library and needs objects of type **`in_addr`**, which is the C type for the 32-bit packed binary this function returns.

`inet_aton()` also accepts strings with less than three dots; see the Unix manual page *inet(3)* *[https://manpages.debian.org/inet(3)]* for details.

If the IPv4 address string passed to this function is invalid, **`OSError`** will be raised. Note that exactly what is valid depends on the underlying C implementation of **`inet_aton()`**.

`inet_aton()` does not support IPv6, and **`inet_pton()`** should be used instead for IPv4/v6 dual stack support.

socket.inet_ntoa(*packed_ip*)

Convert a 32-bit packed IPv4 address (a bytes-like object four bytes in length) to its standard dotted-quad string representation (for example, '123.45.67.89'). This is useful when conversing with a program that uses the standard C library and needs objects of type **`in_addr`**, which is the C type for the 32-bit packed binary data this function takes as an argument.

If the byte sequence passed to this function is not exactly 4 bytes in length, **OSError** will be raised. **inet_ntoa()** does not support IPv6, and **inet_ntop()** should be used instead for IPv4/v6 dual stack support.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

socket.inet_pton(*address_family*, *ip_string*)

Convert an IP address from its family-specific string format to a packed, binary format. **inet_pton()** is useful when a library or network protocol calls for an object of type **in_addr** (similar to **inet_aton()**) or **in6_addr**.

Supported values for *address_family* are currently **AF_INET** and **AF_INET6**. If the IP address string *ip_string* is invalid, **OSError** will be raised. Note that exactly what is valid depends on both the value of *address_family* and the underlying implementation of **inet_pton()**.

Disponibilidade: Unix, Windows.

Alterado na versão 3.4: Suporte para Windows adicionado.

socket.inet_ntop(*address_family*, *packed_ip*)

Convert a packed IP address (a bytes-like object of some number of bytes) to its standard, family-specific string representation (for example, `'7.10.0.5'` or `'5aef:2b::8'`). **inet_ntop()** is useful when a library or network protocol returns an object of type **in_addr** (similar to **inet_ntoa()**) or **in6_addr**.

Supported values for *address_family* are currently **AF_INET** and **AF_INET6**. If the bytes object *packed_ip* is not the correct length for the specified address family, **ValueError** will be raised. **OSError** is raised for errors from the call to **inet_ntop()**.

Disponibilidade: Unix, Windows.

Alterado na versão 3.4: Suporte para Windows adicionado.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

socket.CMSG_LEN(*length*)

Return the total length, without trailing padding, of an ancillary data item with associated data of the given *length*. This value can often be used as the buffer size for `recvmsg()` to receive a single item of ancillary data, but **RFC 3542** [https://datatracker.ietf.org/doc/html/rfc3542.html] requires portable applications to use `CMSG_SPACE()` and thus include space for padding, even when the item will be the last in the buffer. Raises `OverflowError` if *length* is outside the permissible range of values.

Disponibilidade: Unix, not WASI.

Most Unix platforms.

Adicionado na versão 3.3.

socket.CMSG_SPACE(*length*)

Return the buffer size needed for `recvmsg()` to receive an ancillary data item with associated data of the given *length*, along with any trailing padding. The buffer space needed to receive multiple items is the sum of the **`CMSG_SPACE()`** values for their associated data lengths. Raises `OverflowError` if *length* is outside the permissible range of values.

Note that some systems might support ancillary data without providing this function. Also note that setting the buffer size using the results of this function may not precisely limit the amount of ancillary data that can be received, since additional data may be able to fit into the padding area.

Disponibilidade: Unix, not WASI.

most Unix platforms.

Adicionado na versão 3.3.

socket.getdefaulttimeout()

Return the default timeout in seconds (float) for new socket objects. A value of `None` indicates that new socket objects have no timeout. When the socket module is first imported, the default is `None`.

socket.setdefaulttimeout(*timeout*)

Set the default timeout in seconds (real number) for new socket objects. When the socket module is first imported, the default is `None`. See **`settimeout()`** for possible values and their respective meanings.

Alterado na versão 3.15: Aceita qualquer número real, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

socket.sethostname(*name*)

Set the machine's hostname to *name*. This will raise an **`OSError`** if you don't have enough rights.

Levanta um evento de auditoria `socket.sethostname` com o argumento `name`.

Disponibilidade: Unix, not Android.

Adicionado na versão 3.3.

socket.if_nameindex()

Return a list of network interface information (index int, name string) tuples. **`OSError`** if the system call fails.

Disponibilidade: Unix, Windows, not WASI.

Adicionado na versão 3.3.

Alterado na versão 3.8: Suporte ao Windows foi adicionado.

Nota

On Windows network interfaces have different names in different contexts (all names are examples):

- UUID: `{FB605B73-AAC2-49A6-9A2F-25416AEA0573}`
- name: `ethernet_32770`
- friendly name: `vEthernet (nat)`
- description: `Hyper-V Virtual Ethernet Adapter`

This function returns names of the second form from the list, `ethernet_32770` in this example case.

socket.if_nametoindex(*if_name*)

Return a network interface index number corresponding to an interface name. **`OSError`** if no interface with the given name exists.

Disponibilidade: Unix, Windows, not WASI.

Adicionado na versão 3.3.

Alterado na versão 3.8: Suporte ao Windows foi adicionado.

Ver também

"Interface name" is a name as documented in **`if_nameindex()`**.

socket.if_indextoname(*if_index*)

Return a network interface name corresponding to an interface index number. **`OSError`** if no interface with the given index exists.

Disponibilidade: Unix, Windows, not WASI.

Adicionado na versão 3.3.

Alterado na versão 3.8: Suporte ao Windows foi adicionado.

Ver também

"Interface name" is a name as documented in **`if_nameindex()`**.

socket.send_fds(*sock*, *buffers*, *fds*[, *flags*[, *address*]])

Send the list of file descriptors *fds* over an **AF_UNIX** socket *sock*. The *fds* parameter is a sequence of file descriptors. Consult **sendmsg()** for the documentation of these parameters.

Disponibilidade: Unix, not WASI.

Unix platforms supporting **sendmsg()** and **SCM_RIGHTS** mechanism.

Adicionado na versão 3.9.

socket.recv_fds(*sock*, *bufsize*, *maxfds*[, *flags*])

Receive up to *maxfds* file descriptors from an **AF_UNIX** socket *sock*. Return `(msg, list(fds), flags, addr)`. Consult **recvmsg()** for the documentation of these parameters.

Disponibilidade: Unix, not WASI.

Unix platforms supporting **recvmsg()** and **SCM_RIGHTS** mechanism.

Adicionado na versão 3.9.

Nota

Any truncated integers at the end of the list of file descriptors.

Socket Objects

class socket.socket(*family=AF_INET*, *type=SOCK_STREAM*, *proto=0*, *fileno=None*)

Create a new socket using the given address family, socket type and protocol number. The address family should be **AF_INET** (the default), **AF_INET6**, **AF_UNIX**, **AF_CAN**, **AF_PACKET**, or **AF_RDS**. The socket type should be **SOCK_STREAM** (the default), **SOCK_DGRAM**, **SOCK_RAW** or perhaps one of the other `SOCK_` constants. The protocol number is usually zero

and may be omitted or in the case where the address family is **AF_CAN** the protocol should be one of **CAN_RAW**, **CAN_BCM**, **CAN_ISOTP** or **CAN_J1939**.

If *fileno* is specified, the values for *family*, *type*, and *proto* are auto-detected from the specified file descriptor. Auto-detection can be overruled by calling the function with explicit *family*, *type*, or *proto* arguments. This only affects how Python represents e.g. the return value of **socket.getpeername()** but not the actual OS resource. Unlike **socket.fromfd()**, *fileno* will return the same socket and not a duplicate. This may help close a detached socket using **close()**.

The newly created socket is non-inheritable.

Levanta um evento de auditoria `socket.__new__` com os argumentos `self`, `family`, `type`, `protocol`.

Alterado na versão 3.3: The AF_CAN family was added. The AF_RDS family was added.

Alterado na versão 3.4: The CAN_BCM protocol was added.

Alterado na versão 3.4: The returned socket is now non-inheritable.

Alterado na versão 3.7: The CAN_ISOTP protocol was added.

Alterado na versão 3.7: When **SOCK_NONBLOCK** or **SOCK_CLOEXEC** bit flags are applied to *type* they are cleared, and **socket.type** will not reflect them. They are still passed to the underlying system `socket()` call. Therefore,

```
sock = socket.socket(
    socket.AF_INET,
    socket.SOCK_STREAM | socket.SOCK_NONBLOCK)
```

will still create a non-blocking socket on OSes that support `SOCK_NONBLOCK`, but `sock.type` will be set to `socket.SOCK_STREAM`.

Alterado na versão 3.9: The CAN_J1939 protocol was added.

Alterado na versão 3.10: The IPPROTO_MPTCP protocol was added.

Socket objects have the following methods. Except for **`makefile()`**, these correspond to Unix system calls applicable to sockets.

Alterado na versão 3.2: Support for the context manager protocol was added. Exiting the context manager is equivalent to calling **`close()`**.

accept()

Accept a connection. The socket must be bound to an address and listening for connections. The return value is a pair `(conn, address)` where *conn* is a *new* socket object usable to send and receive data on the connection, and *address* is the address bound to the socket on the other end of the connection.

The newly created socket is non-inheritable.

Alterado na versão 3.4: The socket is now non-inheritable.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an **`InterruptedError`** exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

bind(*address*)

Bind the socket to *address*. The socket must not already be bound. The format of *address* depends on the address family — see Famílias de soquete.

Levanta um evento de auditoria `socket.bind` com os argumentos `self`, `address`.

Disponibilidade: not WASI.

close()

Mark the socket closed. The underlying system resource (e.g. a file descriptor) is also closed when all file objects from **`makefile()`** are

closed. Once that happens, all future operations on the socket object will fail. The remote end will receive no more data (after queued data is flushed).

Sockets are automatically closed when they are garbage-collected, but it is recommended to **`close()`** them explicitly, or to use a **`with`** statement around them.

Alterado na versão 3.6: **`OSError`** is now raised if an error occurs when the underlying **`close()`** call is made.

Nota

`close()` releases the resource associated with a connection but does not necessarily close the connection immediately. If you want to close the connection in a timely fashion, call **`shutdown()`** before **`close()`**.

connect(*address*)

Connect to a remote socket at *address*. The format of *address* depends on the address family — see Famílias de soquete.

If the connection is interrupted by a signal, the method waits until the connection completes, or raises a **`TimeoutError`** on timeout, if the signal handler doesn't raise an exception and the socket is blocking or has a timeout. For non-blocking sockets, the method raises an **`InterruptedError`** exception if the connection is interrupted by a signal (or the exception raised by the signal handler).

Levanta um evento de auditoria `socket.connect` com os argumentos `self`, `address`.

Alterado na versão 3.5: The method now waits until the connection completes instead of raising an **`InterruptedError`** exception if the connection is interrupted by a signal, the signal handler doesn't raise

an exception and the socket is blocking or has a timeout (see the **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

Disponibilidade: not WASI.

connect_ex(*address*)

Like `connect(address)`, but return an error indicator instead of raising an exception for errors returned by the C-level **`connect()`** call (other problems, such as "host not found," can still raise exceptions). The error indicator is `0` if the operation succeeded, otherwise the value of the **`errno`** variable. This is useful to support, for example, asynchronous connects.

Levanta um evento de auditoria `socket.connect` com os argumentos `self`, `address`.

Disponibilidade: not WASI.

detach()

Put the socket object into closed state without actually closing the underlying file descriptor. The file descriptor is returned, and can be reused for other purposes.

Adicionado na versão 3.2.

dup()

Duplicate the socket.

The newly created socket is non-inheritable.

Alterado na versão 3.4: The socket is now non-inheritable.

Disponibilidade: not WASI.

fileno()

Return the socket's file descriptor (a small integer), or -1 on failure. This is useful with **`select.select()`**.

Under Windows the small integer returned by this method cannot be used where a file descriptor can be used (such as `os.fdopen()`). Unix does not have this limitation.

get_inheritable()

Get the inheritable flag of the socket's file descriptor or socket's handle: `True` if the socket can be inherited in child processes, `False` if it cannot.

Adicionado na versão 3.4.

getpeername()

Return the remote address to which the socket is connected. This is useful to find out the port number of a remote IPv4/v6 socket, for instance. The format of the address returned depends on the address family — see Famílias de soquete. On some systems this function is not supported.

getsockname()

Return the socket's own address. This is useful to find out the port number of an IPv4/v6 socket, for instance. The format of the address returned depends on the address family — see Famílias de soquete.

getsockopt(*level*, *optname*[, *buflen*])

Return the value of the given socket option (see the Unix man page *getsockopt(2)* *[https://manpages.debian.org/getsockopt(2)]*). The needed symbolic constants (SO_* etc.) are defined in this module. If *buflen* is absent, an integer option is assumed and its integer value is returned by the function. If *buflen* is present, it specifies the maximum length of the buffer used to receive the option in, and this buffer is returned as a bytes object. It is up to the caller to decode the contents of the buffer (see the optional built-in module `struct` for a way to decode C structures encoded as byte strings).

Disponibilidade: not WASI.

getblocking()

Return `True` if socket is in blocking mode, `False` if in non-blocking.

This is equivalent to checking `socket.gettimeout() != 0`.

Adicionado na versão 3.7.

gettimeout()

Return the timeout in seconds (float) associated with socket operations, or `None` if no timeout is set. This reflects the last call to **`setblocking()`** or **`settimeout()`**.

ioctl(*control*, *option*)

The **`ioctl()`** method is a limited interface to the WSAIoctl system interface. Please refer to the Win32 documentation [https://msdn.microsoft.com/en-us/library/ms741621%28VS.85%29.aspx] for more information.

On other platforms, the generic **`fcntl.fcntl()`** and **`fcntl.ioctl()`** functions may be used; they accept a socket object as their first argument.

Currently only the following control codes are supported: `SIO_RCVALL`, `SIO_KEEPALIVE_VALS`, and `SIO_LOOPBACK_FAST_PATH`.

Disponibilidade: Windows

Alterado na versão 3.6: `SIO_LOOPBACK_FAST_PATH` was added.

listen([*backlog*])

Enable a server to accept connections. If *backlog* is specified, it must be at least 0 (if it is lower, it is set to 0); it specifies the number of unaccepted connections that the system will allow before refusing new connections. If not specified, a default reasonable value is chosen.

Disponibilidade: not WASI.

Alterado na versão 3.5: The *backlog* parameter is now optional.

makefile(*mode='r', buffering=None, *, encoding=None, errors=None, newline=None*)

Return a file object associated with the socket. The exact returned type depends on the arguments given to **`makefile()`**. These arguments are interpreted the same way as by the built-in **`open()`** function, except the only supported *mode* values are `'r'` (default), `'w'`, `'b'`, or a combination of those.

The socket must be in blocking mode; it can have a timeout, but the file object's internal buffer may end up in an inconsistent state if a timeout occurs.

Closing the file object returned by **`makefile()`** won't close the original socket unless all other file objects have been closed and **`close()`** has been called on the socket object.

Nota

On Windows, the file-like object created by **`makefile()`** cannot be used where a file object with a file descriptor is expected, such as the stream arguments of **`subprocess.Popen()`**.

recv(*bufsize*[, *flags*])

Receive data from the socket. The return value is a bytes object representing the data received. The maximum amount of data to be received at once is specified by *bufsize*. A returned empty bytes object indicates that the client has disconnected. See the Unix manual page *recv(2)* *[https://manpages.debian.org/recv(2)]* for the meaning of the optional argument *flags*; it defaults to zero.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an **`InterruptedError`** exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

recvfrom(*bufsize*[, *flags*])

Receive data from the socket. The return value is a pair `(bytes, address)` where *bytes* is a bytes object representing the data received and *address* is the address of the socket sending the data. See the Unix manual page *recv(2)* *[https://manpages.debian.org/recv(2)]* for the meaning of the optional argument *flags*; it defaults to zero. The format of *address* depends on the address family — see Famílias de soquete.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an **`InterruptedError`** exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

Alterado na versão 3.7: For multicast IPv6 address, first item of *address* does not contain `%scope_id` part anymore. In order to get full IPv6 address use **`getnameinfo()`**.

recvmsg(*bufsize*[, *ancbufsize*[, *flags*]])

Receive normal data (up to *bufsize* bytes) and ancillary data from the socket. The *ancbufsize* argument sets the size in bytes of the internal buffer used to receive the ancillary data; it defaults to 0, meaning that no ancillary data will be received. Appropriate buffer sizes for ancillary data can be calculated using **`CMSG_SPACE()`** or **`CMSG_LEN()`**, and items which do not fit into the buffer might be truncated or discarded. The *flags* argument defaults to 0 and has the same meaning as for **`recv()`**.

The return value is a 4-tuple: `(data, ancdata, msg_flags, address)`. The *data* item is a **`bytes`** object holding the non-ancillary data received. The *ancdata* item is a list of zero or more tuples `(cmsg_level, cmsg_type, cmsg_data)` representing the ancillary data (control messages) received: *cmsg_level* and *cmsg_type* are integers specifying the protocol level and protocol-specific type respectively, and *cmsg_data* is a **`bytes`** object holding the associated data. The *msg_flags* item is the bitwise OR of various flags

indicating conditions on the received message; see your system documentation for details. If the receiving socket is unconnected, *address* is the address of the sending socket, if available; otherwise, its value is unspecified.

On some systems, **sendmsg()** and **recvmsg()** can be used to pass file descriptors between processes over an **AF_UNIX** socket. When this facility is used (it is often restricted to **SOCK_STREAM** sockets), **recvmsg()** will return, in its ancillary data, items of the form `(socket.SOL_SOCKET, socket.SCM_RIGHTS, fds)`, where *fds* is a **bytes** object representing the new file descriptors as a binary array of the native C int type. If **recvmsg()** raises an exception after the system call returns, it will first attempt to close any file descriptors received via this mechanism.

Some systems do not indicate the truncated length of ancillary data items which have been only partially received. If an item appears to extend beyond the end of the buffer, **recvmsg()** will issue a **RuntimeWarning**, and will return the part of it which is inside the buffer provided it has not been truncated before the start of its associated data.

On systems which support the **SCM_RIGHTS** mechanism, the following function will receive up to *maxfds* file descriptors, returning the message data and a list containing the descriptors (while ignoring unexpected conditions such as unrelated control messages being received). See also **sendmsg()**.

```
import socket, array

def recv_fds(sock, msglen, maxfds):
    fds = array.array("i")   # Array of ints
    msg, ancdata, flags, addr = sock.recvmsg(msglen, socke
    for cmsg_level, cmsg_type, cmsg_data in ancdata:
        if cmsg_level == socket.SOL_SOCKET and cmsg_type =
            # Append data, ignoring any truncated integers
            fds.frombytes(cmsg_data[:len(cmsg_data) - (len
    return msg, list(fds)
```

Disponibilidade: Unix.

Most Unix platforms.

Adicionado na versão 3.3.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an **`InterruptedError`** exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

recvmsg_into(*buffers*[, *ancbufsize*[, *flags*]])

Receive normal data and ancillary data from the socket, behaving as **`recvmsg()`** would, but scatter the non-ancillary data into a series of buffers instead of returning a new bytes object. The *buffers* argument must be an iterable of objects that export writable buffers (e.g. **`bytearray`** objects); these will be filled with successive chunks of the non-ancillary data until it has all been written or there are no more buffers. The operating system may set a limit (**`sysconf()`** value `SC_IOV_MAX`) on the number of buffers that can be used. The *ancbufsize* and *flags* arguments have the same meaning as for **`recvmsg()`**.

The return value is a 4-tuple: `(nbytes, ancdata, msg_flags, address)`, where *nbytes* is the total number of bytes of non-ancillary data written into the buffers, and *ancdata*, *msg_flags* and *address* are the same as for **`recvmsg()`**.

Exemplo:

```
>>> import socket
>>> s1, s2 = socket.socketpair()
>>> b1 = bytearray(b'----')
>>> b2 = bytearray(b'0123456789')
>>> b3 = bytearray(b'--------------')
>>> s1.send(b'Mary had a little lamb')
22
>>> s2.recvmsg_into([b1, memoryview(b2)[2:9], b3])
(22, [], 0, None)
```

```
>>> [b1, b2, b3]
[bytearray(b'Mary'), bytearray(b'01 had a 9'), bytearray(b
```

Disponibilidade: Unix.

Most Unix platforms.

Adicionado na versão 3.3.

recvfrom_into(*buffer*[, *nbytes*[, *flags*]])

Receive data from the socket, writing it into *buffer* instead of creating a new bytestring. The return value is a pair `(nbytes, address)` where *nbytes* is the number of bytes received and *address* is the address of the socket sending the data. See the Unix manual page *recv(2)* *[https://manpages.debian.org/recv(2)]* for the meaning of the optional argument *flags*; it defaults to zero. The format of *address* depends on the address family — see Famílias de soquete.

recv_into(*buffer*[, *nbytes*[, *flags*]])

Receive up to *nbytes* bytes from the socket, storing the data into a buffer rather than creating a new bytestring. If *nbytes* is not specified (or 0), receive up to the size available in the given buffer. Returns the number of bytes received. See the Unix manual page *recv(2)* *[https://manpages.debian.org/recv(2)]* for the meaning of the optional argument *flags*; it defaults to zero.

send(*bytes*[, *flags*])

Send data to the socket. The socket must be connected to a remote socket. The optional *flags* argument has the same meaning as for **`recv()`**. Returns the number of bytes sent. Applications are responsible for checking that all data has been sent; if only some of the data was transmitted, the application needs to attempt delivery of the remaining data. For further information on this topic, consult the Programação de Soquetes.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

sendall(*bytes*[, *flags*])

Send data to the socket. The socket must be connected to a remote socket. The optional *flags* argument has the same meaning as for `recv()`. Unlike `send()`, this method continues to send data from *bytes* until either all data has been sent or an error occurs. `None` is returned on success. On error, an exception is raised, and there is no way to determine how much data, if any, was successfully sent.

Alterado na versão 3.5: The socket timeout is no longer reset each time data is sent successfully. The socket timeout is now the maximum total duration to send all data.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

sendto(*bytes*, *address*)

sendto(*bytes*, *flags*, *address*)

Send data to the socket. The socket should not be connected to a remote socket, since the destination socket is specified by *address*. The optional *flags* argument has the same meaning as for `recv()`. Return the number of bytes sent. The format of *address* depends on the address family — see Famílias de soquete.

Levanta um evento de auditoria `socket.sendto` com os argumentos `self`, `address`.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

sendmsg(*buffers*[, *ancdata*[, *flags*[, *address*]]])

Send normal and ancillary data to the socket, gathering the non-ancillary data from a series of buffers and concatenating it into a single message. The *buffers* argument specifies the non-ancillary data as an iterable of bytes-like objects (e.g. `bytes` objects); the operating system may set a limit (`sysconf()` value `SC_IOV_MAX`) on the number of buffers that can be used. The *ancdata* argument specifies the ancillary data (control messages) as an iterable of zero or more tuples `(cmsg_level, cmsg_type, cmsg_data)`, where *cmsg_level* and *cmsg_type* are integers specifying the protocol level and protocol-specific type respectively, and *cmsg_data* is a bytes-like object holding the associated data. Note that some systems (in particular, systems without `CMSG_SPACE()`) might support sending only one control message per call. The *flags* argument defaults to 0 and has the same meaning as for `send()`. If *address* is supplied and not `None`, it sets a destination address for the message. The return value is the number of bytes of non-ancillary data sent.

The following function sends the list of file descriptors *fds* over an `AF_UNIX` socket, on systems which support the **SCM_RIGHTS** mechanism. See also `recvmsg()`.

```
import socket, array

def send_fds(sock, msg, fds):
    return sock.sendmsg([msg], [(socket.SOL_SOCKET, socke
```

Disponibilidade: Unix, not WASI.

Most Unix platforms.

Levanta um evento de auditoria `socket.sendmsg` com os argumentos `self`, `address`.

Adicionado na versão 3.3.

Alterado na versão 3.5: If the system call is interrupted and the signal handler does not raise an exception, the method now retries the system call instead of raising an `InterruptedError` exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale).

sendmsg_afalg([*msg*,]*, *op*[, *iv*[, *assoclen*[, *flags*]]])

Specialized version of `sendmsg()` for `AF_ALG` socket. Set mode, IV, AEAD associated data length and flags for `AF_ALG` socket.

Disponibilidade: Linux >= 2.6.38.

Adicionado na versão 3.6.

sendfile(*file*, *offset=0*, *count=None*)

Send a file until EOF is reached by using high-performance `os.sendfile` and return the total number of bytes which were sent. *file* must be a regular file object opened in binary mode. If `os.sendfile` is not available (e.g. Windows) or *file* is not a regular file `send()` will be used instead. *offset* tells from where to start reading the file. If specified, *count* is the total number of bytes to transmit as opposed to sending the file until EOF is reached. File position is updated on return or also in case of error in which case `file.tell()` can be used to figure out the number of bytes which were sent. The socket must be of `SOCK_STREAM` type. Non-blocking sockets are not supported.

Adicionado na versão 3.5.

set_inheritable(*inheritable*)

Set the inheritable flag of the socket's file descriptor or socket's handle.

Adicionado na versão 3.4.

setblocking(*flag*)

Set blocking or non-blocking mode of the socket: if *flag* is false, the socket is set to non-blocking, else to blocking mode.

This method is a shorthand for certain **settimeout()** calls:

- `sock.setblocking(True)` is equivalent to `sock.settimeout(None)`
- `sock.setblocking(False)` is equivalent to `sock.settimeout(0.0)`

Alterado na versão 3.7: The method no longer applies **SOCK_NONBLOCK** flag on **socket.type**.

settimeout(*value*)

Set a timeout on blocking socket operations. The *value* argument can be a nonnegative real number expressing seconds, or `None`. If a non-zero value is given, subsequent socket operations will raise a **timeout** exception if the timeout period *value* has elapsed before the operation has completed. If zero is given, the socket is put in non-blocking mode. If `None` is given, the socket is put in blocking mode.

For further information, please consult the notes on socket timeouts.

Alterado na versão 3.7: The method no longer toggles **SOCK_NONBLOCK** flag on **socket.type**.

Alterado na versão 3.15: Aceita qualquer número real, e não apenas inteiros (*integer*) ou números de vírgula flutuante (*float*).

setsockopt(*level*, *optname*, *value: int* | *Buffer*)

setsockopt(*level*, *optname*, *None*, *optlen: int*)

Set the value of the given socket option (see the Unix manual page *setsockopt(2)* *[https://manpages.debian.org/setsockopt(2)]*). The needed symbolic constants are defined in this module (SO_* etc. <socket-unix-constants>). The value can be an integer, `None` or a bytes-like object representing a buffer. In the latter case it is up to the caller to ensure that the bytestring contains the proper bits (see the optional built-in module **struct** for a way to encode C structures as bytestrings). When *value* is set to `None`, *optlen* argument is required. It's equivalent to calling **setsockopt()** C function with `optval=NULL` and `optlen=optlen`.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

Alterado na versão 3.6: setsockopt(level, optname, None, optlen: int) form added.

Disponibilidade: not WASI.

shutdown(*how*)

Shut down one or both halves of the connection. If *how* is **`SHUT_RD`**, further receives are disallowed. If *how* is **`SHUT_WR`**, further sends are disallowed. If *how* is **`SHUT_RDWR`**, further sends and receives are disallowed.

Disponibilidade: not WASI.

share(*process_id*)

Duplicate a socket and prepare it for sharing with a target process. The target process must be provided with *process_id*. The resulting bytes object can then be passed to the target process using some form of interprocess communication and the socket can be recreated there using **`fromshare()`**. Once this method has been called, it is safe to close the socket since the operating system has already duplicated it for the target process.

Disponibilidade: Windows.

Adicionado na versão 3.3.

Note that there are no methods **`read()`** or **`write()`**; use **`recv()`** and **`send()`** without *flags* argument instead.

Socket objects also have these (read-only) attributes that correspond to the values given to the **`socket`** constructor.

family

The socket family.

type

The socket type.

proto
The socket protocol.

class socket.SocketType
The base class of the **socket** type, re-exported from **_socket**. An instance check such as `isinstance(socket(...), SocketType)` is true, but `SocketType` is not the same as `type(socket(...))`, which is **socket** itself.

Notas sobre tempo limite de soquete

Um objeto soquete pode estar em um dos três modos: bloqueio, não-bloqueio ou tempo limite. Por padrão, os soquetes sempre são criados no modo de bloqueio, mas isso pode ser alterado chamando **setdefaulttimeout()**.

- In *blocking mode*, operations block until complete or the system returns an error (such as connection timed out).
- In *non-blocking mode*, operations fail (with an error that is unfortunately system-dependent) if they cannot be completed immediately: functions from the **select** module can be used to know when and whether a socket is available for reading or writing.
- In *timeout mode*, operations fail if they cannot be completed within the timeout specified for the socket (they raise a **timeout** exception) or if the system returns an error.

Nota

At the operating system level, sockets in *timeout mode* are internally set in non-blocking mode. Also, the blocking and timeout modes are shared between file descriptors and socket objects that refer to the same network endpoint. This implementation detail can have visible consequences if e.g. you decide to use the **fileno()** of a socket.

Timeouts and the `connect` method

The `connect()` operation is also subject to the timeout setting, and in general it is recommended to call `settimeout()` before calling `connect()` or pass a timeout parameter to `create_connection()`. However, the system network stack may also return a connection timeout error of its own regardless of any Python socket timeout setting.

Timeouts and the `accept` method

If `getdefaulttimeout()` is not `None`, sockets returned by the `accept()` method inherit that timeout. Otherwise, the behaviour depends on settings of the listening socket:

- if the listening socket is in *blocking mode* or in *timeout mode*, the socket returned by `accept()` is in *blocking mode*;
- if the listening socket is in *non-blocking mode*, whether the socket returned by `accept()` is in blocking or non-blocking mode is operating system-dependent. If you want to ensure cross-platform behaviour, it is recommended you manually override this setting.

Exemplo

Here are four minimal example programs using the TCP/IP protocol: a server that echoes all data that it receives back (servicing only one client), and a client using it. Note that a server must perform the sequence `socket()`, `bind()`, `listen()`, `accept()` (possibly repeating the `accept()` to service more than one client), while a client only needs the sequence `socket()`, `connect()`. Also note that the server does not `sendall()`/`recv()` on the socket it is listening on but on the new socket returned by `accept()`.

The first two examples support IPv4 only.

```
# Echo server program
import socket

HOST = ''                 # Symbolic name meaning all available
```

```
PORT = 50007              # Arbitrary non-privileged port
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen(1)
    conn, addr = s.accept()
    with conn:
        print('Connected by', addr)
        while True:
            data = conn.recv(1024)
            if not data: break
            conn.sendall(data)
```

```
# Echo client program
import socket

HOST = 'daring.cwi.nl'    # The remote host
PORT = 50007              # The same port as used by the server
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b'Hello, world')
    data = s.recv(1024)
print('Received', repr(data))
```

The next two examples are identical to the above two, but support both IPv4 and IPv6. The server side will listen to the first address family available (it should listen to both instead). On most of IPv6-ready systems, IPv6 will take precedence and the server may not accept IPv4 traffic. The client side will try to connect to all the addresses returned as a result of the name resolution, and sends traffic to the first one connected successfully.

```
# Echo server program
import socket
import sys

HOST = None               # Symbolic name meaning all available
PORT = 50007              # Arbitrary non-privileged port
s = None
for res in socket.getaddrinfo(HOST, PORT, socket.AF_UNSPEC,
                              socket.SOCK_STREAM, 0, socket.AI_P
    af, socktype, proto, canonname, sa = res
    try:
```

```
        s = socket.socket(af, socktype, proto)
    except OSError as msg:
        s = None
        continue
    try:
        s.bind(sa)
        s.listen(1)
    except OSError as msg:
        s.close()
        s = None
        continue
    break
if s is None:
    print('could not open socket')
    sys.exit(1)
conn, addr = s.accept()
with conn:
    print('Connected by', addr)
    while True:
        data = conn.recv(1024)
        if not data: break
        conn.send(data)
```

```
# Echo client program
import socket
import sys

HOST = 'daring.cwi.nl'    # The remote host
PORT = 50007              # The same port as used by the server
s = None
for res in socket.getaddrinfo(HOST, PORT, socket.AF_UNSPEC, sock
    af, socktype, proto, canonname, sa = res
    try:
        s = socket.socket(af, socktype, proto)
    except OSError as msg:
        s = None
        continue
    try:
        s.connect(sa)
    except OSError as msg:
        s.close()
        s = None
```

```
        continue
    break
if s is None:
    print('could not open socket')
    sys.exit(1)
with s:
    s.sendall(b'Hello, world')
    data = s.recv(1024)
print('Received', repr(data))
```

The next example shows how to write a very simple network sniffer with raw sockets on Windows. The example requires administrator privileges to modify the interface:

```
import socket

# the public network interface
HOST = socket.gethostbyname(socket.gethostname())

# create a raw socket and bind it to the public interface
s = socket.socket(socket.AF_INET, socket.SOCK_RAW, socket.IPPROT
s.bind((HOST, 0))

# Include IP headers
s.setsockopt(socket.IPPROTO_IP, socket.IP_HDRINCL, 1)

# receive all packets
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_ON)

# receive a packet
print(s.recvfrom(65565))

# disabled promiscuous mode
s.ioctl(socket.SIO_RCVALL, socket.RCVALL_OFF)
```

The next example shows how to use the socket interface to communicate to a CAN network using the raw socket protocol. To use CAN with the broadcast manager protocol instead, open a socket with:

```
socket.socket(socket.AF_CAN, socket.SOCK_DGRAM, socket.CAN_BCM)
```

After binding (**CAN_RAW**) or connecting (**CAN_BCM**) the socket, you can use the **socket.send()** and **socket.recv()** operations (and their counterparts) on the socket object as usual.

This last example might require special privileges:

```
import socket
import struct


# CAN frame packing/unpacking (see 'struct can_frame' in <linux/

can_frame_fmt = "=IB3x8s"
can_frame_size = struct.calcsize(can_frame_fmt)

def build_can_frame(can_id, data):
    can_dlc = len(data)
    data = data.ljust(8, b'\x00')
    return struct.pack(can_frame_fmt, can_id, can_dlc, data)

def dissect_can_frame(frame):
    can_id, can_dlc, data = struct.unpack(can_frame_fmt, frame)
    return (can_id, can_dlc, data[:can_dlc])


# create a raw socket and bind it to the 'vcan0' interface
s = socket.socket(socket.AF_CAN, socket.SOCK_RAW, socket.CAN_RAW
s.bind(('vcan0',))

while True:
    cf, addr = s.recvfrom(can_frame_size)

    print('Received: can_id=%x, can_dlc=%x, data=%s' % dissect_c

    try:
        s.send(cf)
    except OSError:
        print('Error sending CAN frame')

    try:
        s.send(build_can_frame(0x01, b'\x01\x02\x03'))
```

```
    except OSError:
        print('Error sending CAN frame')
```

Running an example several times with too small delay between executions, could lead to this error:

```
OSError: [Errno 98] Address already in use
```

This is because the previous execution has left the socket in a `TIME_WAIT` state, and can't be immediately reused.

There is a **`socket`** flag to set, in order to prevent this, **`socket.SO_REUSEADDR`**:

```
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
s.bind((HOST, PORT))
```

the **`SO_REUSEADDR`** flag tells the kernel to reuse a local socket in `TIME_WAIT` state, without waiting for its natural timeout to expire.

Ver também

Para uma introdução à programação de socket (em C), veja os seguintes artigos:

- *An Introductory 4.3BSD Interprocess Communication Tutorial*, by Stuart Sechrest
- *An Advanced 4.3BSD Interprocess Communication Tutorial*, by Samuel J. Leffler et al,

both in the UNIX Programmer's Manual, Supplementary Documents 1 (sections PS1:7 and PS1:8). The platform-specific reference material for the various socket-related system calls are also a valuable source of information on the details of socket semantics. For Unix, refer to the manual pages; for Windows, see the WinSock (or Winsock 2) specification. For IPv6-ready APIs, readers may want to refer to **RFC**

3493 [https://datatracker.ietf.org/doc/html/rfc3493.html] titled Basic Socket Interface Extensions for IPv6.

`ssl` — Invólucro de TLS/SSL para objetos socket

Código-fonte: Lib/ssl.py [https://github.com/python/cpython/tree/3.15/Lib/ssl.py]

This module provides access to Transport Layer Security (often known as "Secure Sockets Layer") encryption and peer authentication facilities for network sockets, both client-side and server-side. This module uses the OpenSSL library.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Nota

Alguns comportamentos podem depender da plataforma, já que chamadas são feitas para as APIs de socket do sistema operacional. A versão instalada do OpenSSL também pode causar variações no comportamento. Por exemplo, o TLSv1.3 vem com a versão 1.1.1 do OpenSSL.

Aviso

Não use este módulo sem ler o Considerações de segurança. Fazer isso pode levar a uma falsa sensação de segurança, pois as configurações padrão do módulo ssl não são necessariamente apropriadas para sua aplicação.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Esta seção documenta os objetos e funções no módulo `ssl`; para obter mais informações gerais sobre TLS, SSL e certificados, o leitor pode consultar os documentos na seção “Consulte também” na parte inferior.

Este módulo fornece uma classe, **`ssl.SSLSocket`**, que é derivada do tipo **`socket.socket`**, e fornece um wrapper semelhante a um socket que também criptografa e descriptografa os dados que passam pelo socket com SSL. Ele suporta métodos adicionais como **`getpeercert()`**, que recupera o certificado do outro lado da conexão, **`cipher()`**, que recupera a cifra sendo usada para a conexão segura ou **`get_verified_chain()`**, **`get_unverified_chain()`** que recupera a cadeia de certificados.

Para aplicações mais sofisticadas, a classe **`ssl.SSLContext`** ajuda a gerenciar configurações e certificados, que podem ser herdados por soquetes SSL criados por meio do método **`SSLContext.wrap_socket()`**.

Alterado na versão 3.5.3: Atualizado para oferecer suporte à vinculação com OpenSSL 1.1.0

Alterado na versão 3.6: OpenSSL 0.9.8, 1.0.0 e 1.0.1 estão descontinuado e não são mais suportados. No futuro, o módulo ssl exigirá pelo menos OpenSSL 1.0.2 ou 1.1.0.

Alterado na versão 3.10: **PEP 644** [https://peps.python.org/pep-0644/] foi implementado. O módulo ssl requer OpenSSL 1.1.1 ou mais recente.

O uso de constantes e funções descontinuados resulta em avisos de descontinuação.

Funções, constantes e exceções.

Criação de socket

Instâncias de `SSLSocket` devem ser criadas usando o método `SSLContext.wrap_socket()`. A função auxiliar `create_default_context()` retorna um novo contexto com configurações padrão seguras.

Exemplo de soquete de cliente com contexto padrão e pilha dupla IPv4/IPv6:

```
import socket
import ssl

hostname = 'www.python.org'
context = ssl.create_default_context()

with socket.create_connection((hostname, 443)) as sock:
    with context.wrap_socket(sock, server_hostname=hostname) as
        print(ssock.version())
```

Exemplo de soquete de cliente com contexto personalizado e IPv4:

```
hostname = 'www.python.org'
# PROTOCOL_TLS_CLIENT requer hostname e cadeia de certificados
context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
context.load_verify_locations('path/to/cabundle.pem')

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as soc
    with context.wrap_socket(sock, server_hostname=hostname) as
        print(ssock.version())
```

Exemplo de soquete de servidor escutando no localhost IPv4:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain('/caminho/para/cadeia-certicado.pem', '/

with socket.socket(socket.AF_INET, socket.SOCK_STREAM, 0) as soc
    sock.bind(('127.0.0.1', 8443))
    sock.listen(5)
    with context.wrap_socket(sock, server_side=True) as ssock:
        conn, addr = ssock.accept()
        ...
```

Criação de contexto

A convenience function helps create `SSLContext` objects for common purposes.

ssl.create_default_context(*purpose=Purpose.SERVER_AUTH*, *, *cafile=None*, *capath=None*, *cadata=None*)

Return a new `SSLContext` object with default settings for the given *purpose*. The settings are chosen by the `ssl` module, and usually represent a higher security level than when calling the `SSLContext` constructor directly.

cafile, *capath*, *cadata* represent optional CA certificates to trust for certificate verification, as in `SSLContext.load_verify_locations()`. If all three are `None`, this function can choose to trust the system's default CA certificates instead.

The settings are: `PROTOCOL_TLS_CLIENT` or `PROTOCOL_TLS_SERVER`, `OP_NO_SSLv2`, and `OP_NO_SSLv3` with high encryption cipher suites without RC4 and without unauthenticated cipher suites. Passing `SERVER_AUTH` as *purpose* sets `verify_mode` to `CERT_REQUIRED` and either loads CA certificates (when at least one of *cafile*, *capath* or *cadata* is given) or uses `SSLContext.load_default_certs()` to load default CA certificates.

When `keylog_filename` is supported and the environment variable `SSLKEYLOGFILE` is set, `create_default_context()` enables key logging.

The default settings for this context include `VERIFY_X509_PARTIAL_CHAIN` and `VERIFY_X509_STRICT`. These make the underlying OpenSSL implementation behave more like a conforming implementation of **RFC 5280** [https://datatracker.ietf.org/doc/html/rfc5280.html], in exchange for a small amount of incompatibility with older X.509 certificates.

Nota

The protocol, options, cipher and other settings may change to more restrictive values anytime without prior deprecation. The values represent a fair balance between compatibility and security.

If your application needs specific settings, you should create a `SSLContext` and apply the settings yourself.

Nota

If you find that when certain older clients or servers attempt to connect with a `SSLContext` created by this function that they get an error stating "Protocol or cipher suite mismatch", it may be that they only support SSL3.0 which this function excludes using the `OP_NO_SSLv3`. SSL3.0 is widely considered to be completely broken [https://en.wikipedia.org/wiki/POODLE]. If you still wish to continue to use this function but still allow SSL 3.0 connections you can re-enable them using:

```
ctx = ssl.create_default_context(Purpose.CLIENT_AUTH)
ctx.options &= ~ssl.OP_NO_SSLv3
```

Nota

This context enables `VERIFY_X509_STRICT` by default, which may reject pre-**RFC 5280** [https://datatracker.ietf.org/doc/html/rfc5280.html] or malformed certificates that the underlying OpenSSL implementation otherwise would accept. While disabling this is not recommended, you can do so using:

```
ctx = ssl.create_default_context()
ctx.verify_flags &= ~ssl.VERIFY_X509_STRICT
```

Adicionado na versão 3.4.

Alterado na versão 3.4.4: RC4 was dropped from the default cipher string.

Alterado na versão 3.6: ChaCha20/Poly1305 was added to the default cipher string.

3DES was dropped from the default cipher string.

Alterado na versão 3.8: Support for key logging to **SSLKEYLOGFILE** was added.

Alterado na versão 3.10: The context now uses **PROTOCOL_TLS_CLIENT** or **PROTOCOL_TLS_SERVER** protocol instead of generic **PROTOCOL_TLS**.

Alterado na versão 3.13: The context now uses **VERIFY_X509_PARTIAL_CHAIN** and **VERIFY_X509_STRICT** in its default verify flags.

Signature algorithms

ssl.get_sigalgs()

Return a list of available TLS signature algorithm names used by servers to complete the TLS handshake or clients requesting certificate-based authentication. For example:

```
>>> ssl.get_sigalgs()
['ecdsa_secp256r1_sha256', 'ecdsa_secp384r1_sha384', ...]
```

These names can be used when building string values to pass to the **SSLContext.set_client_sigalgs()** and **SSLContext.set_server_sigalgs()** methods.

Adicionado na versão 3.15.

Exceções

exception ssl.SSLError

Raised to signal an error from the underlying SSL implementation (currently provided by the OpenSSL library). This signifies some problem in the higher-level encryption and authentication layer that's superimposed on the underlying network connection. This error is a

subtype of **`OSError`**. The error code and message of **`SSLError`** instances are provided by the OpenSSL library.

Alterado na versão 3.3: **`SSLError`** used to be a subtype of **`socket.error`**.

library

A string mnemonic designating the OpenSSL submodule in which the error occurred, such as `SSL`, `PEM` or `X509`. The range of possible values depends on the OpenSSL version.

Adicionado na versão 3.3.

reason

A string mnemonic designating the reason this error occurred, for example `CERTIFICATE_VERIFY_FAILED`. The range of possible values depends on the OpenSSL version.

Adicionado na versão 3.3.

exception ssl.SSLZeroReturnError

A subclass of **`SSLError`** raised when trying to read or write and the SSL connection has been closed cleanly. Note that this doesn't mean that the underlying transport (read TCP) has been closed.

Adicionado na versão 3.3.

exception ssl.SSLWantReadError

A subclass of **`SSLError`** raised by a non-blocking SSL socket when trying to read or write data, but more data needs to be received on the underlying TCP transport before the request can be fulfilled.

Adicionado na versão 3.3.

exception ssl.SSLWantWriteError

A subclass of **`SSLError`** raised by a non-blocking SSL socket when trying to read or write data, but more data needs to be sent on the underlying TCP transport before the request can be fulfilled.

Adicionado na versão 3.3.

exception ssl.SSLSyscallError

A subclass of `SSLError` raised when a system error was encountered while trying to fulfill an operation on a SSL socket. Unfortunately, there is no easy way to inspect the original errno number.

Adicionado na versão 3.3.

exception ssl.SSLEOFError

A subclass of `SSLError` raised when the SSL connection has been terminated abruptly. Generally, you shouldn't try to reuse the underlying transport when this error is encountered.

Adicionado na versão 3.3.

exception ssl.SSLCertVerificationError

A subclass of `SSLError` raised when certificate validation has failed.

Adicionado na versão 3.7.

verify_code

A numeric error number that denotes the verification error.

verify_message

A human readable string of the verification error.

exception ssl.CertificateError

An alias for `SSLCertVerificationError`.

Alterado na versão 3.7: The exception is now an alias for `SSLCertVerificationError`.

Random generation

ssl.RAND_bytes(*num*, *[,]*)

Return *num* cryptographically strong pseudo-random bytes. Raises an `SSLError` if the PRNG has not been seeded with enough data or if the operation is not supported by the current RAND method. `RAND_status()` can be used to check the status of the PRNG and `RAND_add()` can be used to seed the PRNG.

For almost all applications `os.urandom()` is preferable.

Read the Wikipedia article, Cryptographically secure pseudorandom number generator (CSPRNG) [https://en.wikipedia.org/wiki/Cryptographically_secure_pseudorandom_number_generator], to get the requirements of a cryptographically strong generator.

Adicionado na versão 3.3.

ssl.RAND_status()

Return `True` if the SSL pseudo-random number generator has been seeded with 'enough' randomness, and `False` otherwise. Use `ssl.RAND_add()` to increase the randomness of the pseudo-random number generator.

ssl.RAND_add(*bytes*, *entropy*, /)

Mix the given *bytes* into the SSL pseudo-random number generator. The parameter *entropy* (a float) is a lower bound on the entropy contained in string (so you can always use `0.0`). See **RFC 1750** [https://datatracker.ietf.org/doc/html/rfc1750.html] for more information on sources of entropy.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

Manipulação de certificados

ssl.cert_time_to_seconds(*cert_time*)

Return the time in seconds since the epoch, given the `cert_time` string representing the "notBefore" or "notAfter" date from a certificate in `"%b %d %H:%M:%S %Y %Z"` strptime format (C locale).

Here's an example:

```
>>> import ssl
>>> import datetime as dt
>>> timestamp = ssl.cert_time_to_seconds("Jan  5 09:34:43 20
>>> timestamp
1515144883
>>> print(dt.datetime.fromtimestamp(timestamp, dt.UTC))
2018-01-05 09:34:43+00:00
```

"notBefore" or "notAfter" dates must use GMT (**RFC 5280** [https://datatracker.ietf.org/doc/html/rfc5280.html]).

Alterado na versão 3.5: Interpret the input time as a time in UTC as specified by 'GMT' timezone in the input string. Local timezone was used previously. Return an integer (no fractions of a second in the input format)

ssl.get_server_certificate(*addr*, *ssl_version=PROTOCOL_TLS_CLIENT*, *ca_certs=None*[, *timeout*])

Given the address `addr` of an SSL-protected server, as a (*hostname*, *port-number*) pair, fetches the server's certificate, and returns it as a PEM-encoded string. If `ssl_version` is specified, uses that version of the SSL protocol to attempt to connect to the server. If *ca_certs* is specified, it should be a file containing a list of root certificates, the same format as used for the *cafile* parameter in **`SSLContext.load_verify_locations()`**. The call will attempt to validate the server certificate against that set of root certificates, and will fail if the validation attempt fails. A timeout can be specified with the `timeout` parameter.

Alterado na versão 3.3: This function is now IPv6-compatible.

Alterado na versão 3.5: The default *ssl_version* is changed from **`PROTOCOL_SSLv3`** to **`PROTOCOL_TLS`** for maximum compatibility with modern servers.

Alterado na versão 3.10: The *timeout* parameter was added.

ssl.DER_cert_to_PEM_cert(*der_cert_bytes*)

Given a certificate as a DER-encoded blob of bytes, returns a PEM-encoded string version of the same certificate.

ssl.PEM_cert_to_DER_cert(*pem_cert_string*)

Given a certificate as an ASCII PEM string, returns a DER-encoded sequence of bytes for that same certificate.

ssl.get_default_verify_paths()

Returns a named tuple with paths to OpenSSL's default cafile and capath. The paths are the same as used by **`SSLContext.set_default_verify_paths()`**. The return value is a named tuple `DefaultVerifyPaths`:

- **`cafile`** - resolved path to cafile or `None` if the file doesn't exist,
- **`capath`** - resolved path to capath or `None` if the directory doesn't exist,
- **`openssl_cafile_env`** - OpenSSL's environment key that points to a cafile,
- **`openssl_cafile`** - hard coded path to a cafile,
- **`openssl_capath_env`** - OpenSSL's environment key that points to a capath,
- **`openssl_capath`** - hard coded path to a capath directory

Adicionado na versão 3.4.

ssl.enum_certificates(*store_name*)

Retrieve certificates from Windows' system cert store. *store_name* may be one of `CA`, `ROOT` or `MY`. Windows may provide additional cert stores, too.

The function returns a list of (cert_bytes, encoding_type, trust) tuples. The encoding_type specifies the encoding of cert_bytes. It is either **`x509_asn`** for X.509 ASN.1 data or **`pkcs_7_asn`** for PKCS#7 ASN.1 data. Trust specifies the purpose of the certificate as a set of OIDS or exactly `True` if the certificate is trustworthy for all purposes.

Exemplo:

```
>>> ssl.enum_certificates("CA")
[(b'data...', 'x509_asn', {'1.3.6.1.5.5.7.3.1', '1.3.6.1.5.5.
 (b'data...', 'x509_asn', True)]
```

Disponibilidade: Windows.

Adicionado na versão 3.4.

ssl.enum_crls(*store_name*)

Retrieve CRLs from Windows' system cert store. *store_name* may be one of `CA`, `ROOT` or `MY`. Windows may provide additional cert stores, too.

The function returns a list of (cert_bytes, encoding_type, trust) tuples. The encoding_type specifies the encoding of cert_bytes. It is either **`x509_asn`** for X.509 ASN.1 data or **`pkcs_7_asn`** for PKCS#7 ASN.1 data.

Disponibilidade: Windows.

Adicionado na versão 3.4.

Constantes

All constants are now **`enum.IntEnum`** or **`enum.IntFlag`** collections.

Adicionado na versão 3.6.

ssl.CERT_NONE

Possible value for **`SSLContext.verify_mode`**. Except for **`PROTOCOL_TLS_CLIENT`**, it is the default mode. With client-side sockets, just about any cert is accepted. Validation errors, such as untrusted or expired cert, are ignored and do not abort the TLS/SSL handshake.

In server mode, no certificate is requested from the client, so the client does not send any for client cert authentication.

See the discussion of Considerações de segurança below.

ssl.CERT_OPTIONAL

Possible value for `SSLContext.verify_mode`. In client mode, `CERT_OPTIONAL` has the same meaning as `CERT_REQUIRED`. It is recommended to use `CERT_REQUIRED` for client-side sockets instead.

In server mode, a client certificate request is sent to the client. The client may either ignore the request or send a certificate in order perform TLS client cert authentication. If the client chooses to send a certificate, it is verified. Any verification error immediately aborts the TLS handshake.

Use of this setting requires a valid set of CA certificates to be passed to `SSLContext.load_verify_locations()`.

ssl.CERT_REQUIRED

Possible value for `SSLContext.verify_mode`. In this mode, certificates are required from the other side of the socket connection; an `SSLError` will be raised if no certificate is provided, or if its validation fails. This mode is **not** sufficient to verify a certificate in client mode as it does not match hostnames. `check_hostname` must be enabled as well to verify the authenticity of a cert. `PROTOCOL_TLS_CLIENT` uses `CERT_REQUIRED` and enables `check_hostname` by default.

With server socket, this mode provides mandatory TLS client cert authentication. A client certificate request is sent to the client and the client must provide a valid and trusted certificate.

Use of this setting requires a valid set of CA certificates to be passed to `SSLContext.load_verify_locations()`.

class ssl.VerifyMode

`enum.IntEnum` collection of CERT_* constants.

Adicionado na versão 3.6.

ssl.VERIFY_DEFAULT

Possible value for `SSLContext.verify_flags`. In this mode, certificate revocation lists (CRLs) are not checked. By default OpenSSL does neither require nor verify CRLs.

Adicionado na versão 3.4.

ssl.VERIFY_CRL_CHECK_LEAF

Possible value for `SSLContext.verify_flags`. In this mode, only the peer cert is checked but none of the intermediate CA certificates. The mode requires a valid CRL that is signed by the peer cert's issuer (its direct ancestor CA). If no proper CRL has been loaded with `SSLContext.load_verify_locations`, validation will fail.

Adicionado na versão 3.4.

ssl.VERIFY_CRL_CHECK_CHAIN

Possible value for `SSLContext.verify_flags`. In this mode, CRLs of all certificates in the peer cert chain are checked.

Adicionado na versão 3.4.

ssl.VERIFY_X509_STRICT

Possible value for `SSLContext.verify_flags` to disable workarounds for broken X.509 certificates.

Adicionado na versão 3.4.

ssl.VERIFY_ALLOW_PROXY_CERTS

Possible value for `SSLContext.verify_flags` to enables proxy certificate verification.

Adicionado na versão 3.10.

ssl.VERIFY_X509_TRUSTED_FIRST

Possible value for `SSLContext.verify_flags`. It instructs OpenSSL to prefer trusted certificates when building the trust chain to validate a certificate. This flag is enabled by default.

Adicionado na versão 3.4.4.

ssl.VERIFY_X509_PARTIAL_CHAIN

Possible value for `SSLContext.verify_flags`. It instructs OpenSSL to accept intermediate CAs in the trust store to be treated as trust-anchors, in the same way as the self-signed root CA certificates. This makes it possible to trust certificates issued by an intermediate CA without having to trust its ancestor root CA.

Adicionado na versão 3.10.

class ssl.VerifyFlags

`enum.IntFlag` collection of VERIFY_* constants.

Adicionado na versão 3.6.

ssl.PROTOCOL_TLS

Selects the highest protocol version that both the client and server support. Despite the name, this option can select both “SSL” and “TLS” protocols.

Adicionado na versão 3.6.

Descontinuado desde a versão 3.10: TLS clients and servers require different default settings for secure communication. The generic TLS protocol constant is deprecated in favor of `PROTOCOL_TLS_CLIENT` and `PROTOCOL_TLS_SERVER`.

ssl.PROTOCOL_TLS_CLIENT

Auto-negotiate the highest protocol version that both the client and server support, and configure the context client-side connections. The protocol enables `CERT_REQUIRED` and `check_hostname` by default.

Adicionado na versão 3.6.

ssl.PROTOCOL_TLS_SERVER

Auto-negotiate the highest protocol version that both the client and server support, and configure the context server-side connections.

Adicionado na versão 3.6.

ssl.PROTOCOL_SSLv23

Alias for **PROTOCOL_TLS**.

Descontinuado desde a versão 3.6: Use **PROTOCOL_TLS** instead.

ssl.PROTOCOL_SSLv3

Selects SSL version 3 as the channel encryption protocol.

This protocol is not available if OpenSSL is compiled with the `no-ssl3` option.

Aviso

SSL version 3 is insecure. Its use is highly discouraged.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols. Use the default protocol **PROTOCOL_TLS_SERVER** or **PROTOCOL_TLS_CLIENT** with **SSLContext.minimum_version** and **SSLContext.maximum_version** instead.

ssl.PROTOCOL_TLSv1

Selects TLS version 1.0 as the channel encryption protocol.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols.

ssl.PROTOCOL_TLSv1_1

Selects TLS version 1.1 as the channel encryption protocol. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols.

ssl.PROTOCOL_TLSv1_2

Selects TLS version 1.2 as the channel encryption protocol. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.6: OpenSSL has deprecated all version specific protocols.

ssl.OP_ALL

Enables workarounds for various bugs present in other SSL implementations. This option is set by default. It does not necessarily set the same flags as OpenSSL's `SSL_OP_ALL` constant.

Adicionado na versão 3.2.

ssl.OP_NO_SSLv2

Prevents an SSLv2 connection. This option is only applicable in conjunction with **`PROTOCOL_TLS`**. It prevents the peers from choosing SSLv2 as the protocol version.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.6: SSLv2 is deprecated

ssl.OP_NO_SSLv3

Prevents an SSLv3 connection. This option is only applicable in conjunction with **`PROTOCOL_TLS`**. It prevents the peers from choosing SSLv3 as the protocol version.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.6: SSLv3 is deprecated

ssl.OP_NO_TLSv1

Prevents a TLSv1 connection. This option is only applicable in conjunction with **`PROTOCOL_TLS`**. It prevents the peers from choosing TLSv1 as the protocol version.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0, use the new `SSLContext.minimum_version` and `SSLContext.maximum_version` instead.

ssl.OP_NO_TLSv1_1

Prevents a TLSv1.1 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1.1 as the protocol version. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0.

ssl.OP_NO_TLSv1_2

Prevents a TLSv1.2 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1.2 as the protocol version. Available only with openssl version 1.0.1+.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0.

ssl.OP_NO_TLSv1_3

Prevents a TLSv1.3 connection. This option is only applicable in conjunction with `PROTOCOL_TLS`. It prevents the peers from choosing TLSv1.3 as the protocol version. TLS 1.3 is available with OpenSSL 1.1.1 or later. When Python has been compiled against an older version of OpenSSL, the flag defaults to *0*.

Adicionado na versão 3.6.3.

Descontinuado desde a versão 3.7: The option is deprecated since OpenSSL 1.1.0. It was added to 2.7.15 and 3.6.3 for backwards compatibility with OpenSSL 1.0.2.

ssl.OP_NO_RENEGOTIATION

Disable all renegotiation in TLSv1.2 and earlier. Do not send HelloRequest messages, and ignore renegotiation requests via ClientHello.

This option is only available with OpenSSL 1.1.0h and later.

Adicionado na versão 3.7.

ssl.OP_CIPHER_SERVER_PREFERENCE

Use the server's cipher ordering preference, rather than the client's. This option has no effect on client sockets and SSLv2 server sockets.

Adicionado na versão 3.3.

ssl.OP_SINGLE_DH_USE

Prevents reuse of the same DH key for distinct SSL sessions. This improves forward secrecy but requires more computational resources. This option only applies to server sockets.

Adicionado na versão 3.3.

ssl.OP_SINGLE_ECDH_USE

Prevents reuse of the same ECDH key for distinct SSL sessions. This improves forward secrecy but requires more computational resources. This option only applies to server sockets.

Adicionado na versão 3.3.

ssl.OP_ENABLE_MIDDLEBOX_COMPAT

Send dummy Change Cipher Spec (CCS) messages in TLS 1.3 handshake to make a TLS 1.3 connection look more like a TLS 1.2 connection.

This option is only available with OpenSSL 1.1.1 and later.

Adicionado na versão 3.8.

ssl.OP_NO_COMPRESSION

Disable compression on the SSL channel. This is useful if the application protocol supports its own compression scheme.

Adicionado na versão 3.3.

class ssl.Options

`enum.IntFlag` collection of OP_* constants.

ssl.OP_NO_TICKET

Prevent client side from requesting a session ticket.

Adicionado na versão 3.6.

ssl.OP_IGNORE_UNEXPECTED_EOF

Ignore unexpected shutdown of TLS connections.

This option is only available with OpenSSL 3.0.0 and later.

Adicionado na versão 3.10.

ssl.OP_ENABLE_KTLS

Enable the use of the kernel TLS. To benefit from the feature, OpenSSL must have been compiled with support for it, and the negotiated cipher suites and extensions must be supported by it (a list of supported ones may vary by platform and kernel version).

Note that with enabled kernel TLS some cryptographic operations are performed by the kernel directly and not via any available OpenSSL Providers. This might be undesirable if, for example, the application requires all cryptographic operations to be performed by the FIPS provider.

This option is only available with OpenSSL 3.0.0 and later.

Adicionado na versão 3.12.

ssl.OP_LEGACY_SERVER_CONNECT

Allow legacy insecure renegotiation between OpenSSL and unpatched servers only.

Adicionado na versão 3.12.

ssl.HAS_ALPN

Whether the OpenSSL library has built-in support for the *Application-Layer Protocol Negotiation* TLS extension as described in **RFC 7301** [https://datatracker.ietf.org/doc/html/rfc7301.html].

Adicionado na versão 3.5.

ssl.HAS_NEVER_CHECK_COMMON_NAME

Whether the OpenSSL library has built-in support not checking subject common name and `SSLContext.hostname_checks_common_name` is writeable.

Adicionado na versão 3.7.

ssl.HAS_ECDH

Whether the OpenSSL library has built-in support for the Elliptic Curve-based Diffie-Hellman key exchange. This should be true unless the feature was explicitly disabled by the distributor.

Adicionado na versão 3.3.

ssl.HAS_SNI

Whether the OpenSSL library has built-in support for the *Server Name Indication* extension (as defined in **RFC 6066** [https://datatracker.ietf.org/doc/html/rfc6066.html]).

Adicionado na versão 3.2.

ssl.HAS_NPN

Whether the OpenSSL library has built-in support for the *Next Protocol Negotiation* as described in the Application Layer Protocol Negotiation [https://en.wikipedia.org/wiki/Application-Layer_Protocol_Negotiation]. When true, you can use the `SSLContext.set_npn_protocols()` method to advertise which protocols you want to support.

Adicionado na versão 3.3.

ssl.HAS_SSLv2

Whether the OpenSSL library has built-in support for the SSL 2.0 protocol.

Adicionado na versão 3.7.

ssl.HAS_SSLv3

Whether the OpenSSL library has built-in support for the SSL 3.0 protocol.

Adicionado na versão 3.7.

ssl.HAS_TLSv1

Whether the OpenSSL library has built-in support for the TLS 1.0 protocol.

Adicionado na versão 3.7.

ssl.HAS_TLSv1_1

Whether the OpenSSL library has built-in support for the TLS 1.1 protocol.

Adicionado na versão 3.7.

ssl.HAS_TLSv1_2

Whether the OpenSSL library has built-in support for the TLS 1.2 protocol.

Adicionado na versão 3.7.

ssl.HAS_TLSv1_3

Whether the OpenSSL library has built-in support for the TLS 1.3 protocol.

Adicionado na versão 3.7.

ssl.HAS_PSK

Whether the OpenSSL library has built-in support for TLS-PSK.

Adicionado na versão 3.13.

ssl.HAS_PSK_TLS13

Whether the OpenSSL library has built-in support for External PSKs in TLS 1.3 as described in **RFC 9258** [https://datatracker.ietf.org/doc/html/rfc9258.html].

Adicionado na versão 3.15.

ssl.HAS_PHA

Whether the OpenSSL library has built-in support for TLS-PHA.

Adicionado na versão 3.14.

ssl.CHANNEL_BINDING_TYPES

List of supported TLS channel binding types. Strings in this list can be used as arguments to `SSLSocket.get_channel_binding()`.

Adicionado na versão 3.3.

ssl.OPENSSL_VERSION

The version string of the OpenSSL library loaded by the interpreter:

```
>>> ssl.OPENSSL_VERSION
'OpenSSL 1.0.2k  26 Jan 2017'
```

Adicionado na versão 3.2.

ssl.OPENSSL_VERSION_INFO

A tuple of five integers representing version information about the OpenSSL library:

```
>>> ssl.OPENSSL_VERSION_INFO
(1, 0, 2, 11, 15)
```

Adicionado na versão 3.2.

ssl.OPENSSL_VERSION_NUMBER

The raw version number of the OpenSSL library, as a single integer:

```
>>> ssl.OPENSSL_VERSION_NUMBER
268443839
>>> hex(ssl.OPENSSL_VERSION_NUMBER)
'0x100020bf'
```

Adicionado na versão 3.2.

ssl.ALERT_DESCRIPTION_HANDSHAKE_FAILURE
ssl.ALERT_DESCRIPTION_INTERNAL_ERROR
ALERT_DESCRIPTION_*

Alert Descriptions from **RFC 5246** [https://datatracker.ietf.org/doc/html/rfc5246.html] and others. The IANA TLS Alert Registry [https://www.iana.org/assignments/tls-parameters/tls-parameters.xml#tls-parameters-6] contains this list and references to the RFCs where their meaning is defined.

Used as the return value of the callback function in `SSLContext.set_servername_callback()`.

Adicionado na versão 3.4.

class ssl.AlertDescription

`enum.IntEnum` collection of ALERT_DESCRIPTION_* constants.

Adicionado na versão 3.6.

Purpose.SERVER_AUTH

Option for `create_default_context()` and `SSLContext.load_default_certs()`. This value indicates that the context may be used to authenticate web servers (therefore, it will be used to create client-side sockets).

Adicionado na versão 3.4.

Purpose.CLIENT_AUTH

Option for `create_default_context()` and `SSLContext.load_default_certs()`. This value indicates that the context may be used to authenticate web clients (therefore, it will be used to create server-side sockets).

Adicionado na versão 3.4.

class ssl.SSLErrorNumber

`enum.IntEnum` collection of SSL_ERROR_* constants.

Adicionado na versão 3.6.

class ssl.TLSVersion

`enum.IntEnum` collection of SSL and TLS versions for `SSLContext.maximum_version` and `SSLContext.minimum_version`.

Adicionado na versão 3.7.

TLSVersion.MINIMUM_SUPPORTED

TLSVersion.MAXIMUM_SUPPORTED

The minimum or maximum supported SSL or TLS version. These are magic constants. Their values don't reflect the lowest and highest available TLS/SSL versions.

TLSVersion.SSLv3

TLSVersion.TLSv1

TLSVersion.TLSv1_1

TLSVersion.TLSv1_2

TLSVersion.TLSv1_3

SSL 3.0 to TLS 1.3.

Descontinuado desde a versão 3.10: All `TLSVersion` members except `TLSVersion.TLSv1_2` and `TLSVersion.TLSv1_3` are deprecated.

SSL sockets

class ssl.SSLSocket(*socket.socket*)

SSL sockets provide the following methods of Socket Objects:

- `accept()`
- `bind()`
- `close()`
- `connect()`
- `detach()`
- `fileno()`
- `getpeername()`, `getsockname()`
- `getsockopt()`, `setsockopt()`
- `gettimeout()`, `settimeout()`, `setblocking()`
- `listen()`
- `makefile()`
- `recv()`, `recv_into()` (but passing a non-zero `flags` argument is not allowed)
- `send()`, `sendall()` (with the same limitation)
- `sendfile()` (it may be high-performant only when the kernel TLS is enabled by setting `OP_ENABLE_KTLS` or when a socket is plain-text, else `send()` will be used)
- `shutdown()`

However, since the SSL (and TLS) protocol has its own framing atop of TCP, the SSL sockets abstraction can, in certain respects, diverge from the specification of normal, OS-level sockets. See especially the notes on non-blocking sockets.

Instances of **SSLSocket** must be created using the **SSLContext.wrap_socket()** method.

Alterado na versão 3.5: The **sendfile()** method was added.

Alterado na versão 3.5: The **shutdown()** does not reset the socket timeout each time bytes are received or sent. The socket timeout is now the maximum total duration of the shutdown.

Descontinuado desde a versão 3.6: It is deprecated to create a **SSLSocket** instance directly, use **SSLContext.wrap_socket()** to wrap a socket.

Alterado na versão 3.7: **SSLSocket** instances must be created with **wrap_socket()**. In earlier versions, it was possible to create instances directly. This was never documented or officially supported.

Alterado na versão 3.10: Python now uses `SSL_read_ex` and `SSL_write_ex` internally. The functions support reading and writing of data larger than 2 GB. Writing zero-length data no longer fails with a protocol violation error.

Alterado na versão 3.15: Python now uses `SSL_sendfile` internally when possible. The function sends a file more efficiently because it performs TLS encryption in the kernel to avoid additional context switches.

SSL sockets also have the following additional methods and attributes:

SSLSocket.read(*len=1024*, *buffer=None*)

Read up to *len* bytes of data from the SSL socket and return the result as a `bytes` instance. If *buffer* is specified, then read into the buffer instead, and return the number of bytes read.

Raise **SSLWantReadError** or **SSLWantWriteError** if the socket is non-blocking and the read would block.

As at any time a re-negotiation is possible, a call to **`read()`** can also cause write operations.

Alterado na versão 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration to read up to *len* bytes.

Descontinuado desde a versão 3.6: Use **`recv()`** instead of **`read()`**.

SSLSocket.write(*data*)

Write *data* to the SSL socket and return the number of bytes written. The *data* argument must be an object supporting the buffer interface.

Raise **`SSLWantReadError`** or **`SSLWantWriteError`** if the socket is non-blocking and the write would block.

As at any time a re-negotiation is possible, a call to **`write()`** can also cause read operations.

Alterado na versão 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration to write *data*.

Descontinuado desde a versão 3.6: Use **`send()`** instead of **`write()`**.

Nota

The **`read()`** and **`write()`** methods are the low-level methods that read and write unencrypted, application-level data and decrypt/encrypt it to encrypted, wire-level data. These methods require an active SSL connection, i.e. the handshake was completed and **`SSLSocket.unwrap()`** was not called.

Normally you should use the socket API methods like **`recv()`** and **`send()`** instead of these methods.

SSLSocket.do_handshake(*block=False*)

Perform the SSL setup handshake.

If *block* is true and the timeout obtained by **gettimeout()** is zero, the socket is set in blocking mode until the handshake is performed.

Alterado na versão 3.4: The handshake method also performs **match_hostname()** when the **check_hostname** attribute of the socket's **context** is true.

Alterado na versão 3.5: The socket timeout is no longer reset each time bytes are received or sent. The socket timeout is now the maximum total duration of the handshake.

Alterado na versão 3.7: Hostname or IP address is matched by OpenSSL during handshake. The function **match_hostname()** is no longer used. In case OpenSSL refuses a hostname or IP address, the handshake is aborted early and a TLS alert message is sent to the peer.

SSLSocket.getpeercert(*binary_form=False*)

If there is no certificate for the peer on the other end of the connection, return `None`. If the SSL handshake hasn't been done yet, raise **ValueError**.

If the `binary_form` parameter is **False**, and a certificate was received from the peer, this method returns a **dict** instance. If the certificate was not validated, the dict is empty. If the certificate was validated, it returns a dict with several keys, amongst them `subject` (the principal for which the certificate was issued) and `issuer` (the principal issuing the certificate). If a certificate contains an instance of the *Subject Alternative Name* extension (see **RFC 3280** [https://datatracker.ietf.org/doc/html/rfc3280.html]), there will also be a `subjectAltName` key in the dictionary.

The `subject` and `issuer` fields are tuples containing the sequence of relative distinguished names (RDNs) given in the certificate's data structure for the respective fields, and each RDN is a sequence of name-value pairs. Here is a real-world example:

```
{'issuer': ((('countryName', 'IL'),),
            (('organizationName', 'StartCom Ltd.'),),
            (('organizationalUnitName',
              'Secure Digital Certificate Signing'),),
            (('commonName',
              'StartCom Class 2 Primary Intermediate Server (
 'notAfter': 'Nov 22 08:15:19 2013 GMT',
 'notBefore': 'Nov 21 03:09:52 2011 GMT',
 'serialNumber': '95F0',
 'subject': ((('description', '571208-SLe257oHY9fVQ07Z'),),
             (('countryName', 'US'),),
             (('stateOrProvinceName', 'California'),),
             (('localityName', 'San Francisco'),),
             (('organizationName', 'Electronic Frontier Found
             (('commonName', '*.eff.org'),),
             (('emailAddress', 'hostmaster@eff.org'),)),
 'subjectAltName': (('DNS', '*.eff.org'), ('DNS', 'eff.org'))
 'version': 3}
```

If the `binary_form` parameter is **`True`**, and a certificate was provided, this method returns the DER-encoded form of the entire certificate as a sequence of bytes, or **`None`** if the peer did not provide a certificate. Whether the peer provides a certificate depends on the SSL socket's role:

- for a client SSL socket, the server will always provide a certificate, regardless of whether validation was required;
- for a server SSL socket, the client will only provide a certificate when requested by the server; therefore **`getpeercert()`** will return **`None`** if you used **`CERT_NONE`** (rather than **`CERT_OPTIONAL`** or **`CERT_REQUIRED`**).

See also **`SSLContext.check_hostname`**.

Alterado na versão 3.2: The returned dictionary includes additional items such as `issuer` and `notBefore`.

Alterado na versão 3.4: **`ValueError`** is raised when the handshake isn't done. The returned dictionary includes additional X509v3 extension items such as `crlDistributionPoints`, `caIssuers` and `OCSP` URIs.

Alterado na versão 3.9: IPv6 address strings no longer have a trailing new line.

SSLSocket.get_verified_chain()

Returns verified certificate chain provided by the other end of the SSL channel as a list of DER-encoded bytes. If certificate verification was disabled method acts the same as **`get_unverified_chain()`**.

Adicionado na versão 3.13.

SSLSocket.get_unverified_chain()

Returns raw certificate chain provided by the other end of the SSL channel as a list of DER-encoded bytes.

Adicionado na versão 3.13.

SSLSocket.cipher()

Returns a three-value tuple containing the name of the cipher being used, the version of the SSL protocol that defines its use, and the number of secret bits being used. If no connection has been established, returns `None`.

SSLSocket.shared_ciphers()

Return the list of ciphers available in both the client and server. Each entry of the returned list is a three-value tuple containing the name of the cipher, the version of the SSL protocol that defines its use, and the number of secret bits the cipher uses. **`shared_ciphers()`** returns `None` if no connection has been established or the socket is a client socket.

Adicionado na versão 3.5.

SSLSocket.group()

Return the group used for doing key agreement on this connection. If no connection has been established, returns `None`.

Adicionado na versão 3.15.

SSLSocket.client_sigalg()

Return the signature algorithm used for performing certificate-based client authentication on this connection, or `None` if no connection has been established or client authentication didn't occur.

Adicionado na versão 3.15.

SSLSocket.server_sigalg()

Return the signature algorithm used by the server to complete the TLS handshake on this connection, or `None` if no connection has been established or the cipher suite has no signature.

Adicionado na versão 3.15.

SSLSocket.compression()

Return the compression algorithm being used as a string, or `None` if the connection isn't compressed.

If the higher-level protocol supports its own compression mechanism, you can use **`OP_NO_COMPRESSION`** to disable SSL-level compression.

Adicionado na versão 3.3.

SSLSocket.get_channel_binding(*cb_type='tls-unique'*)

Get channel binding data for current connection, as a bytes object. Returns `None` if not connected or the handshake has not been completed.

The *cb_type* parameter allow selection of the desired channel binding type. Valid channel binding types are listed in the **`CHANNEL_BINDING_TYPES`** list. Currently only the 'tls-unique' channel binding, defined by **RFC 5929** [https://datatracker.ietf.org/doc/html/rfc5929.html], is supported. **`ValueError`** will be raised if an unsupported channel binding type is requested.

Adicionado na versão 3.3.

SSLSocket.selected_alpn_protocol()

Return the protocol that was selected during the TLS handshake. If `SSLContext.set_alpn_protocols()` was not called, if the other party does not support ALPN, if this socket does not support any of the client's proposed protocols, or if the handshake has not happened yet, `None` is returned.

Adicionado na versão 3.5.

SSLSocket.selected_npn_protocol()

Return the higher-level protocol that was selected during the TLS/SSL handshake. If `SSLContext.set_npn_protocols()` was not called, or if the other party does not support NPN, or if the handshake has not yet happened, this will return `None`.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.10: NPN has been superseded by ALPN

SSLSocket.unwrap()

Performs the SSL shutdown handshake, which removes the TLS layer from the underlying socket, and returns the underlying socket object. This can be used to go from encrypted operation over a connection to unencrypted. The returned socket should always be used for further communication with the other side of the connection, rather than the original socket.

SSLSocket.verify_client_post_handshake()

Requests post-handshake authentication (PHA) from a TLS 1.3 client. PHA can only be initiated for a TLS 1.3 connection from a server-side socket, after the initial TLS handshake and with PHA enabled on both sides, see `SSLContext.post_handshake_auth`.

The method does not perform a cert exchange immediately. The server-side sends a CertificateRequest during the next write event and expects the client to respond with a certificate on the next read event.

If any precondition isn't met (e.g. not TLS 1.3, PHA not enabled), an `SSLError` is raised.

Nota

Only available with OpenSSL 1.1.1 and TLS 1.3 enabled. Without TLS 1.3 support, the method raises `NotImplementedError`.

Adicionado na versão 3.8.

SSLSocket.version()

Return the actual SSL protocol version negotiated by the connection as a string, or `None` if no secure connection is established. As of this writing, possible return values include `"SSLv2"`, `"SSLv3"`, `"TLSv1"`, `"TLSv1.1"` and `"TLSv1.2"`. Recent OpenSSL versions may define more return values.

Adicionado na versão 3.5.

SSLSocket.pending()

Returns the number of already decrypted bytes available for read, pending on the connection.

SSLSocket.context

The `SSLContext` object this SSL socket is tied to.

Adicionado na versão 3.2.

SSLSocket.server_side

A boolean which is `True` for server-side sockets and `False` for client-side sockets.

Adicionado na versão 3.2.

SSLSocket.server_hostname

Hostname of the server: `str` type, or `None` for server-side socket or if the hostname was not specified in the constructor.

Adicionado na versão 3.2.

Alterado na versão 3.7: The attribute is now always ASCII text. When `server_hostname` is an internationalized domain name (IDN), this attribute now stores the A-label form (`"xn--pythn-mua.org"`), rather than the U-label form (`"pythön.org"`).

SSLSocket.session

The **`SSLSession`** for this SSL connection. The session is available for client and server side sockets after the TLS handshake has been performed. For client sockets the session can be set before **`do_handshake()`** has been called to reuse a session.

Adicionado na versão 3.6.

SSLSocket.session_reused

Adicionado na versão 3.6.

SSL contexts

Adicionado na versão 3.2.

An SSL context holds various data longer-lived than single SSL connections, such as SSL configuration options, certificate(s) and private key(s). It also manages a cache of SSL sessions for server-side sockets, in order to speed up repeated connections from the same clients.

class ssl.SSLContext(*protocol=None*)

Create a new SSL context. You may pass *protocol* which must be one of the `PROTOCOL_*` constants defined in this module. The parameter specifies which version of the SSL protocol to use. Typically, the server chooses a particular protocol version, and the client must adapt to the server's choice. Most of the versions are not interoperable with the other versions. If not specified, the default is **`PROTOCOL_TLS`**; it provides the most compatibility with other versions.

Here's a table showing which versions in a client (down the side) can connect to which versions in a server (along the top):

client / **server**	**SSLv2**	**SSLv3**	**TLS** [[3]]	**TLSv1**	**TLSv1.1**	**TLSv1.2**
SSLv2	sim	não	no [[1]]	não	não	não
SSLv3	não	sim	no [[2]]	não	não	não
TLS (*SSLv23*) [[3]]	no [[1]]	no [[2]]	sim	sim	sim	sim
TLSv1	não	não	sim	sim	não	não
TLSv1.1	não	não	sim	não	sim	não
TLSv1.2	não	não	sim	não	não	sim

Notas de rodapé

[1] (1,2)
`SSLContext` disables SSLv2 with `OP_NO_SSLv2` by default.

[2] (1,2)
`SSLContext` disables SSLv3 with `OP_NO_SSLv3` by default.

[3] (1,2)
TLS 1.3 protocol will be available with `PROTOCOL_TLS` in OpenSSL >= 1.1.1. There is no dedicated PROTOCOL constant for just TLS 1.3.

Ver também

`create_default_context()` lets the `ssl` module choose security settings for a given purpose.

Alterado na versão 3.6: The context is created with secure default values. The options `OP_NO_COMPRESSION`, `OP_CIPHER_SERVER_PREFERENCE`, `OP_SINGLE_DH_USE`, `OP_SINGLE_ECDH_USE`, `OP_NO_SSLv2`, and `OP_NO_SSLv3` (except for `PROTOCOL_SSLv3`) are set by default. The initial cipher suite list contains only `HIGH` ciphers, no `NULL` ciphers and no `MD5` ciphers.

Descontinuado desde a versão 3.10: `SSLContext` without protocol argument is deprecated. The context class will either require `PROTOCOL_TLS_CLIENT` or `PROTOCOL_TLS_SERVER` protocol in the future.

Alterado na versão 3.10: The default cipher suites now include only secure AES and ChaCha20 ciphers with forward secrecy and security level 2. RSA and DH keys with less than 2048 bits and ECC keys with less than 224 bits are prohibited. `PROTOCOL_TLS`, `PROTOCOL_TLS_CLIENT`, and `PROTOCOL_TLS_SERVER` use TLS 1.2 as minimum TLS version.

Nota

`SSLContext` only supports limited mutation once it has been used by a connection. Adding new certificates to the internal trust store is allowed, but changing ciphers, verification settings, or mTLS certificates may result in surprising behavior.

Nota

`SSLContext` is designed to be shared and used by multiple connections. Thus, it is thread-safe as long as it is not reconfigured after being used by a connection.

`SSLContext` objects have the following methods and attributes:

SSLContext.cert_store_stats()

Get statistics about quantities of loaded X.509 certificates, count of X.509 certificates flagged as CA certificates and certificate revocation

lists as dictionary.

Example for a context with one CA cert and one other cert:

```
>>> context.cert_store_stats()
{'crl': 0, 'x509_ca': 1, 'x509': 2}
```

Adicionado na versão 3.4.

SSLContext.load_cert_chain(*certfile*, *keyfile=None*, *password=None*)

Load a private key and the corresponding certificate. The *certfile* string must be the path to a single file in PEM format containing the certificate as well as any number of CA certificates needed to establish the certificate's authenticity. The *keyfile* string, if present, must point to a file containing the private key. Otherwise the private key will be taken from *certfile* as well. See the discussion of Certificados for more information on how the certificate is stored in the *certfile*.

The *password* argument may be a function to call to get the password for decrypting the private key. It will only be called if the private key is encrypted and a password is necessary. It will be called with no arguments, and it should return a string, bytes, or bytearray. If the return value is a string it will be encoded as UTF-8 before using it to decrypt the key. Alternatively a string, bytes, or bytearray value may be supplied directly as the *password* argument. It will be ignored if the private key is not encrypted and no password is needed.

If the *password* argument is not specified and a password is required, OpenSSL's built-in password prompting mechanism will be used to interactively prompt the user for a password.

An `SSLError` is raised if the private key doesn't match with the certificate.

Alterado na versão 3.3: New optional argument *password*.

SSLContext.load_default_certs(*purpose=Purpose.SERVER_AUTH*)

Load a set of default “certification authority” (CA) certificates from default locations. On Windows it loads CA certs from the `CA` and `ROOT` system stores. On all systems it calls `SSLContext.set_default_verify_paths()`. In the future the method may load CA certificates from other locations, too.

The *purpose* flag specifies what kind of CA certificates are loaded. The default settings `Purpose.SERVER_AUTH` loads certificates, that are flagged and trusted for TLS web server authentication (client side sockets). `Purpose.CLIENT_AUTH` loads CA certificates for client certificate verification on the server side.

Adicionado na versão 3.4.

SSLContext.load_verify_locations(*cafile=None*, *capath=None*, *cadata=None*)

Load a set of “certification authority” (CA) certificates used to validate other peers’ certificates when `verify_mode` is other than `CERT_NONE`. At least one of *cafile* or *capath* must be specified.

This method can also load certification revocation lists (CRLs) in PEM or DER format. In order to make use of CRLs, `SSLContext.verify_flags` must be configured properly.

The *cafile* string, if present, is the path to a file of concatenated CA certificates in PEM format. See the discussion of Certificados for more information about how to arrange the certificates in this file.

The *capath* string, if present, is the path to a directory containing several CA certificates in PEM format, following an OpenSSL specific layout [https://docs.openssl.org/master/man3/SSL_CTX_load_verify_locations/].

The *cadata* object, if present, is either an ASCII string of one or more PEM-encoded certificates or a bytes-like object of DER-encoded certificates. Like with *capath* extra lines around PEM-encoded certificates are ignored but at least one certificate must be present.

Alterado na versão 3.4: New optional argument *cadata*

SSLContext.get_ca_certs(*binary_form=False*)

Get a list of loaded “certification authority” (CA) certificates. If the `binary_form` parameter is `False` each list entry is a dict like the output of `SSLSocket.getpeercert()`. Otherwise the method returns a list of DER-encoded certificates. The returned list does not contain certificates from *capath* unless a certificate was requested and loaded by a SSL connection.

Nota

Certificates in a capath directory aren’t loaded unless they have been used at least once.

Adicionado na versão 3.4.

SSLContext.get_ciphers()

Get a list of enabled ciphers. The list is in order of cipher priority. See `SSLContext.set_ciphers()`.

Exemplo:

```
>>> ctx = ssl.SSLContext(ssl.PROTOCOL_SSLv23)
>>> ctx.set_ciphers('ECDHE+AESGCM:!ECDSA')
>>> ctx.get_ciphers()
[{'aead': True,
  'alg_bits': 256,
  'auth': 'auth-rsa',
  'description': 'ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2 Kx=ECDH
                 'Enc=AESGCM(256) Mac=AEAD',
  'digest': None,
  'id': 50380848,
  'kea': 'kx-ecdhe',
  'name': 'ECDHE-RSA-AES256-GCM-SHA384',
  'protocol': 'TLSv1.2',
  'strength_bits': 256,
  'symmetric': 'aes-256-gcm'},
 {'aead': True,
  'alg_bits': 128,
  'auth': 'auth-rsa',
```

```
  'description': 'ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 Kx=ECDH
                 'Enc=AESGCM(128) Mac=AEAD',
  'digest': None,
  'id': 50380847,
  'kea': 'kx-ecdhe',
  'name': 'ECDHE-RSA-AES128-GCM-SHA256',
  'protocol': 'TLSv1.2',
  'strength_bits': 128,
  'symmetric': 'aes-128-gcm'}]
```

Adicionado na versão 3.6.

SSLContext.get_groups(*, *include_aliases=False*)

Get a list of groups implemented for key agreement, taking into account the current TLS **minimum_version** and **maximum_version** values. For example:

```
>>> ctx = ssl.create_default_context()
>>> ctx.minimum_version = ssl.TLSVersion.TLSv1_3
>>> ctx.maximum_version = ssl.TLSVersion.TLSv1_3
>>> ctx.get_groups()
['secp256r1', 'secp384r1', 'secp521r1', 'x25519', 'x448', ...
```

By default, this method returns only the preferred IANA names for the available groups. However, if the `include_aliases` parameter is set to **True** this method will also return any associated aliases such as the ECDH curve names supported in older versions of OpenSSL.

Adicionado na versão 3.15.

SSLContext.set_default_verify_paths()

Load a set of default “certification authority” (CA) certificates from a filesystem path defined when building the OpenSSL library. Unfortunately, there’s no easy way to know whether this method succeeds: no error is returned if no certificates are to be found. When the OpenSSL library is provided as part of the operating system, though, it is likely to be configured properly.

SSLContext.set_ciphers(*ciphers*, /)

Set the allowed ciphers for sockets created with this context when connecting using TLS 1.2 and earlier. The *ciphers* argument should be a string in the OpenSSL cipher list format [https://docs.openssl.org/master/man1/ciphers/]. To set allowed TLS 1.3 ciphers, use `SSLContext.set_ciphersuites()`.

If no cipher can be selected (because compile-time options or other configuration forbids use of all the specified ciphers), an `SSLError` will be raised.

Nota

When connected, the `SSLSocket.cipher()` method of SSL sockets will return details about the negotiated cipher.

SSLContext.set_ciphersuites(*ciphersuites*, /)

Set the allowed ciphers for sockets created with this context when connecting using TLS 1.3. The *ciphersuites* argument should be a colon-separate string of TLS 1.3 cipher names. If no cipher can be selected (because compile-time options or other configuration forbids use of all the specified ciphers), an `SSLError` will be raised.

Nota

When connected, the `SSLSocket.cipher()` method of SSL sockets will return details about the negotiated cipher.

Adicionado na versão 3.15.

SSLContext.set_groups(*groups*, /)

Set the groups allowed for key agreement for sockets created with this context. It should be a string in the OpenSSL group list format [https://docs.openssl.org/master/man3/SSL_CTX_set1_groups_list/].

Nota

When connected, the `SSLSocket.group()` method of SSL sockets will return the group used for key agreement on that connection.

Adicionado na versão 3.15.

SSLContext.set_client_sigalgs(*sigalgs*, /)

Set the signature algorithms allowed for certificate-based client authentication. It should be a string in the OpenSSL client sigalgs list format [https://docs.openssl.org/master/man3/SSL_CTX_set1_client_sigalgs_list/].

Nota

When connected, the `SSLSocket.client_sigalg()` method of SSL sockets will return the signature algorithm used for performing certificate-based client authentication on that connection.

Adicionado na versão 3.15.

SSLContext.set_server_sigalgs(*sigalgs*, /)

Set the signature algorithms allowed for the server to complete the TLS handshake. It should be a string in the OpenSSL sigalgs list format [https://docs.openssl.org/master/man3/SSL_CTX_set1_sigalgs_list/].

Nota

When connected, the `SSLSocket.server_sigalg()` method of SSL sockets will return the signature algorithm used by the server to complete the TLS handshake on that connection.

Adicionado na versão 3.15.

SSLContext.set_alpn_protocols(*alpn_protocols*)

Specify which protocols the socket should advertise during the SSL/TLS handshake. It should be a list of ASCII strings, like `['http/1.1', 'spdy/2']`, ordered by preference. The selection of a protocol will happen during the handshake, and will play out according

to **RFC 7301** [https://datatracker.ietf.org/doc/html/rfc7301.html]. After a successful handshake, the **`SSLSocket.selected_alpn_protocol()`** method will return the agreed-upon protocol.

This method will raise **`NotImplementedError`** if **`HAS_ALPN`** is `False`.

Adicionado na versão 3.5.

SSLContext.set_npn_protocols(*npn_protocols*)

Specify which protocols the socket should advertise during the SSL/TLS handshake. It should be a list of strings, like `['http/1.1', 'spdy/2']`, ordered by preference. The selection of a protocol will happen during the handshake, and will play out according to the Application Layer Protocol Negotiation [https://en.wikipedia.org/wiki/Application-Layer_Protocol_Negotiation]. After a successful handshake, the **`SSLSocket.selected_npn_protocol()`** method will return the agreed-upon protocol.

This method will raise **`NotImplementedError`** if **`HAS_NPN`** is `False`.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.10: NPN has been superseded by ALPN

SSLContext.sni_callback

Register a callback function that will be called after the TLS Client Hello handshake message has been received by the SSL/TLS server when the TLS client specifies a server name indication. The server name indication mechanism is specified in **RFC 6066** [https://datatracker.ietf.org/doc/html/rfc6066.html] section 3 - Server Name Indication.

Only one callback can be set per `SSLContext`. If *sni_callback* is set to `None` then the callback is disabled. Calling this function a subsequent time will disable the previously registered callback.

The callback function will be called with three arguments; the first being the `ssl.SSLSocket`, the second is a string that represents the server name that the client is intending to communicate (or `None` if the TLS Client Hello does not contain a server name) and the third argument is the original `SSLContext`. The server name argument is text. For internationalized domain name, the server name is an IDN A-label (`"xn--pythn-mua.org"`).

A typical use of this callback is to change the `ssl.SSLSocket`'s `SSLSocket.context` attribute to a new object of type `SSLContext` representing a certificate chain that matches the server name.

If the callback assigns a new context to `SSLSocket.context`, any further ClientHello message on the same connection (for example after a TLS 1.3 HelloRetryRequest) is dispatched to the new context's *sni_callback*, if it has one; the original callback is not called again for that connection.

Due to the early negotiation phase of the TLS connection, only limited methods and attributes are usable like `SSLSocket.selected_alpn_protocol()` and `SSLSocket.context`. The `SSLSocket.getpeercert()`, `SSLSocket.get_verified_chain()`, `SSLSocket.get_unverified_chain()` `SSLSocket.cipher()` and `SSLSocket.compression()` methods require that the TLS connection has progressed beyond the TLS Client Hello and therefore will not return meaningful values nor can they be called safely.

The *sni_callback* function must return `None` to allow the TLS negotiation to continue. If a TLS failure is required, a constant `ALERT_DESCRIPTION_*` can be returned. Other return values will result in a TLS fatal error with **`ALERT_DESCRIPTION_INTERNAL_ERROR`**.

If an exception is raised from the *sni_callback* function the TLS connection will terminate with a fatal TLS alert message `ALERT_DESCRIPTION_HANDSHAKE_FAILURE`.

This method will raise `NotImplementedError` if the OpenSSL library had OPENSSL_NO_TLSEXT defined when it was built.

Adicionado na versão 3.7.

Alterado na versão 3.15: After the callback assigns a new `SSLSocket.context`, later ClientHello messages on the connection are dispatched to the new context's *sni_callback*.

SSLContext.set_servername_callback(*server_name_callback*)

This is a legacy API retained for backwards compatibility. When possible, you should use `sni_callback` instead. The given *server_name_callback* is similar to *sni_callback*, except that when the server hostname is an IDN-encoded internationalized domain name, the *server_name_callback* receives a decoded U-label (`"pythön.org"`).

If there is a decoding error on the server name, the TLS connection will terminate with an `ALERT_DESCRIPTION_INTERNAL_ERROR` fatal TLS alert message to the client.

Adicionado na versão 3.4.

SSLContext.load_dh_params(*dhfile*, */*)

Load the key generation parameters for Diffie-Hellman (DH) key exchange. Using DH key exchange improves forward secrecy at the expense of computational resources (both on the server and on the client). The *dhfile* parameter should be the path to a file containing DH parameters in PEM format.

This setting doesn't apply to client sockets. You can also use the `OP_SINGLE_DH_USE` option to further improve security.

Adicionado na versão 3.3.

SSLContext.set_ecdh_curve(*curve_name*, */*)

Set the curve name for Elliptic Curve-based Diffie-Hellman (ECDH) key exchange. ECDH is significantly faster than regular DH while arguably as secure. The *curve_name* parameter should be a string describing a well-known elliptic curve, for example `prime256v1` for a widely supported curve.

This setting doesn't apply to client sockets. You can also use the `OP_SINGLE_ECDH_USE` option to further improve security.

This method is not available if `HAS_ECDH` is `False`.

Adicionado na versão 3.3.

Ver também

SSL/TLS & Perfect Forward Secrecy
[https://vincent.bernat.ch/en/blog/2011-ssl-perfect-forward-secrecy]
Vincent Bernat.

SSLContext.wrap_socket(*sock*, *server_side=False*, *do_handshake_on_connect=True*, *suppress_ragged_eofs=True*, *server_hostname=None*, *session=None*)

Wrap an existing Python socket *sock* and return an instance of `SSLContext.sslsocket_class` (default `SSLSocket`). The returned SSL socket is tied to the context, its settings and certificates. *sock* must be a `SOCK_STREAM` socket; other socket types are unsupported.

The parameter `server_side` is a boolean which identifies whether server-side or client-side behavior is desired from this socket.

For client-side sockets, the context construction is lazy; if the underlying socket isn't connected yet, the context construction will be performed after `connect()` is called on the socket. For server-side sockets, if the socket has no remote peer, it is assumed to be a listening socket, and the server-side SSL wrapping is automatically performed on client connections accepted via the `accept()` method. The method may raise `SSLError`.

On client connections, the optional parameter *server_hostname* specifies the hostname of the service which we are connecting to. This allows a single server to host multiple SSL-based services with distinct certificates, quite similarly to HTTP virtual hosts. Specifying *server_hostname* will raise a `ValueError` if *server_side* is true.

The parameter `do_handshake_on_connect` specifies whether to do the SSL handshake automatically after doing a **`socket.connect()`**, or whether the application program will call it explicitly, by invoking the **`SSLSocket.do_handshake()`** method. Calling **`SSLSocket.do_handshake()`** explicitly gives the program control over the blocking behavior of the socket I/O involved in the handshake.

The parameter `suppress_ragged_eofs` specifies how the **`SSLSocket.recv()`** method should signal unexpected EOF from the other end of the connection. If specified as **`True`** (the default), it returns a normal EOF (an empty bytes object) in response to unexpected EOF errors raised from the underlying socket; if **`False`**, it will raise the exceptions back to the caller.

session, see **`session`**.

To wrap an **`SSLSocket`** in another **`SSLSocket`**, use **`SSLContext.wrap_bio()`**.

Alterado na versão 3.5: Always allow a server_hostname to be passed, even if OpenSSL does not have SNI.

Alterado na versão 3.6: o argumento *session* foi adicionado.

Alterado na versão 3.7: The method returns an instance of **`SSLContext.sslsocket_class`** instead of hard-coded **`SSLSocket`**.

SSLContext.sslsocket_class

The return type of **`SSLContext.wrap_socket()`**, defaults to **`SSLSocket`**. The attribute can be assigned to on instances of **`SSLContext`** in order to return a custom subclass of **`SSLSocket`**.

Adicionado na versão 3.7.

SSLContext.wrap_bio(*incoming*, *outgoing*, *server_side=False*, *server_hostname=None*, *session=None*)

Wrap the BIO objects *incoming* and *outgoing* and return an instance of **`SSLContext.sslobject_class`** (default **`SSLObject`**). The SSL routines

will read input data from the incoming BIO and write data to the outgoing BIO.

The *server_side*, *server_hostname* and *session* parameters have the same meaning as in `SSLContext.wrap_socket()`, and are validated in the same way: in particular a `ValueError` is raised when `check_hostname` is enabled but no *server_hostname* is given, since there would be no name to match the peer's certificate against.

Alterado na versão 3.6: o argumento *session* foi adicionado.

Alterado na versão 3.7: The method returns an instance of `SSLContext.sslobject_class` instead of hard-coded `SSLObject`.

Alterado na versão 3.15: The *server_side*, *server_hostname* and *session* parameters are now validated as `SSLContext.wrap_socket()` validates them. Previously a context with `check_hostname` enabled and no *server_hostname* was accepted, and verified the certificate chain but never the peer's identity.

SSLContext.sslobject_class

The return type of `SSLContext.wrap_bio()`, defaults to `SSLObject`. The attribute can be overridden on instance of class in order to return a custom subclass of **`SSLObject`**.

Adicionado na versão 3.7.

SSLContext.session_stats()

Get statistics about the SSL sessions created or managed by this context. A dictionary is returned which maps the names of each piece of information [https://docs.openssl.org/1.1.1/man3/SSL_CTX_sess_number/] to their numeric values. For example, here is the total number of hits and misses in the session cache since the context was created:

```
>>> stats = context.session_stats()
>>> stats['hits'], stats['misses']
(0, 0)
```

SSLContext.check_hostname

Whether to match the peer cert's hostname in `SSLSocket.do_handshake()`. The context's `verify_mode` must be set to `CERT_OPTIONAL` or `CERT_REQUIRED`, and you must pass *server_hostname* to `wrap_socket()` in order to match the hostname. Enabling hostname checking automatically sets **`verify_mode`** from `CERT_NONE` to **`CERT_REQUIRED`**. It cannot be set back to **`CERT_NONE`** as long as hostname checking is enabled. The `PROTOCOL_TLS_CLIENT` protocol enables hostname checking by default. With other protocols, hostname checking must be enabled explicitly.

Exemplo:

```
import socket, ssl

context = ssl.SSLContext(ssl.PROTOCOL_TLSv1_2)
context.verify_mode = ssl.CERT_REQUIRED
context.check_hostname = True
context.load_default_certs()

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
ssl_sock = context.wrap_socket(s, server_hostname='www.verisi
ssl_sock.connect(('www.verisign.com', 443))
```

Adicionado na versão 3.4.

Alterado na versão 3.7: `verify_mode` is now automatically changed to `CERT_REQUIRED` when hostname checking is enabled and **`verify_mode`** is `CERT_NONE`. Previously the same operation would have failed with a `ValueError`.

SSLContext.keylog_filename

Write TLS keys to a keylog file, whenever key material is generated or received. The keylog file is designed for debugging purposes only. The file format is specified by NSS and used by many traffic analyzers such as Wireshark. The log file is opened in append-only mode. Writes are synchronized between threads, but not between processes.

Adicionado na versão 3.8.

SSLContext.maximum_version

A `TLSVersion` enum member representing the highest supported TLS version. The value defaults to `TLSVersion.MAXIMUM_SUPPORTED`. The attribute is read-only for protocols other than `PROTOCOL_TLS`, `PROTOCOL_TLS_CLIENT`, and `PROTOCOL_TLS_SERVER`.

The attributes **`maximum_version`**, `minimum_version` and `SSLContext.options` all affect the supported SSL and TLS versions of the context. The implementation does not prevent invalid combinations. For example a context with `OP_NO_TLSv1_2` in **`options`** and **`maximum_version`** set to `TLSVersion.TLSv1_2` will not be able to establish a TLS 1.2 connection.

Adicionado na versão 3.7.

SSLContext.minimum_version

Like `SSLContext.maximum_version` except it is the lowest supported version or `TLSVersion.MINIMUM_SUPPORTED`.

Adicionado na versão 3.7.

SSLContext.num_tickets

Control the number of TLS 1.3 session tickets of a `PROTOCOL_TLS_SERVER` context. The setting has no impact on TLS 1.0 to 1.2 connections.

Adicionado na versão 3.8.

SSLContext.options

An integer representing the set of SSL options enabled on this context. The default value is `OP_ALL`, but you can specify other options such as `OP_NO_SSLv2` by ORing them together.

Alterado na versão 3.6: **`SSLContext.options`** returns `Options` flags:

```
>>> ssl.create_default_context().options
<Options.OP_ALL|OP_NO_SSLv3|OP_NO_SSLv2|OP_NO_COMPRESSION: 2
```

Descontinuado desde a versão 3.7: All `OP_NO_SSL*` and `OP_NO_TLS*` options have been deprecated since Python 3.7. Use **`SSLContext.minimum_version`** and **`SSLContext.maximum_version`** instead.

SSLContext.post_handshake_auth

Enable TLS 1.3 post-handshake client authentication. Post-handshake auth is disabled by default and a server can only request a TLS client certificate during the initial handshake. When enabled, a server may request a TLS client certificate at any time after the handshake.

When enabled on client-side sockets, the client signals the server that it supports post-handshake authentication.

When enabled on server-side sockets, **`SSLContext.verify_mode`** must be set to **`CERT_OPTIONAL`** or **`CERT_REQUIRED`**, too. The actual client cert exchange is delayed until **`SSLSocket.verify_client_post_handshake()`** is called and some I/O is performed.

Adicionado na versão 3.8.

SSLContext.protocol

The protocol version chosen when constructing the context. This attribute is read-only.

SSLContext.hostname_checks_common_name

Whether **`check_hostname`** falls back to verify the cert's subject common name in the absence of a subject alternative name extension (default: true).

Adicionado na versão 3.7.

Alterado na versão 3.10: The flag had no effect with OpenSSL before version 1.1.1l. Python 3.8.9, 3.9.3, and 3.10 include workarounds for

previous versions.

SSLContext.security_level

An integer representing the security level [https://docs.openssl.org/master/man3/SSL_CTX_get_security_level/] for the context. This attribute is read-only.

Adicionado na versão 3.10.

SSLContext.verify_flags

The flags for certificate verification operations. You can set flags like **VERIFY_CRL_CHECK_LEAF** by ORing them together. By default OpenSSL does neither require nor verify certificate revocation lists (CRLs).

Adicionado na versão 3.4.

Alterado na versão 3.6: **SSLContext.verify_flags** returns **VerifyFlags** flags:

```
>>> ssl.create_default_context().verify_flags
<VerifyFlags.VERIFY_X509_TRUSTED_FIRST: 32768>
```

SSLContext.verify_mode

Whether to try to verify other peers' certificates and how to behave if verification fails. This attribute must be one of **CERT_NONE**, **CERT_OPTIONAL** or **CERT_REQUIRED**.

Alterado na versão 3.6: **SSLContext.verify_mode** returns **VerifyMode** enum:

```
>>> ssl.create_default_context().verify_mode
<VerifyMode.CERT_REQUIRED: 2>
```

SSLContext.set_psk_client_callback(*callback*)

Enables TLS-PSK (pre-shared key) authentication on a client-side connection.

In general, certificate based authentication should be preferred over this method.

The parameter `callback` is a callable object with the signature: `def callback(hint: str | None) -> tuple[str | None, bytes]`. The `hint` parameter is an optional identity hint sent by the server. The return value is a tuple in the form (client-identity, psk). Client-identity is an optional string which may be used by the server to select a corresponding PSK for the client. The string must be less than or equal to `256` octets when UTF-8 encoded. PSK is a bytes-like object representing the pre-shared key. Return a zero length PSK to reject the connection.

Setting `callback` to **`None`** removes any existing callback.

Nota

When using TLS 1.3:

- the `hint` parameter is always **`None`**.
- client-identity must be a non-empty string.

Exemplo de uso:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
context.check_hostname = False
context.verify_mode = ssl.CERT_NONE
context.maximum_version = ssl.TLSVersion.TLSv1_2
context.set_ciphers('PSK')

# A simple lambda:
psk = bytes.fromhex('c0ffee')
context.set_psk_client_callback(lambda hint: (None, psk))

# A table using the hint from the server:
psk_table = { 'ServerId_1': bytes.fromhex('c0ffee'),
              'ServerId_2': bytes.fromhex('facade')
}
def callback(hint):
```

```
    return 'ClientId_1', psk_table.get(hint, b'')
context.set_psk_client_callback(callback)
```

This method will raise **NotImplementedError** if **HAS_PSK** is `False`.

Adicionado na versão 3.13.

SSLContext.set_psk_server_callback(*callback*, *identity_hint=None*)

Enables TLS-PSK (pre-shared key) authentication on a server-side connection.

In general, certificate based authentication should be preferred over this method.

The parameter `callback` is a callable object with the signature: `def callback(identity: str | None) -> bytes`. The `identity` parameter is an optional identity sent by the client which can be used to select a corresponding PSK. The return value is a bytes-like object representing the pre-shared key. Return a zero length PSK to reject the connection.

Setting `callback` to **None** removes any existing callback.

The parameter `identity_hint` is an optional identity hint string sent to the client. The string must be less than or equal to `256` octets when UTF-8 encoded.

Nota

When using TLS 1.3 the `identity_hint` parameter is not sent to the client.

Exemplo de uso:

```
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.maximum_version = ssl.TLSVersion.TLSv1_2
context.set_ciphers('PSK')

# A simple lambda:
```

```
psk = bytes.fromhex('c0ffee')
context.set_psk_server_callback(lambda identity: psk)

# A table using the identity of the client:
psk_table = { 'ClientId_1': bytes.fromhex('c0ffee'),
              'ClientId_2': bytes.fromhex('facade')
}
def callback(identity):
    return psk_table.get(identity, b'')
context.set_psk_server_callback(callback, 'ServerId_1')
```

This method will raise **`NotImplementedError`** if **`HAS_PSK`** is `False`.

Adicionado na versão 3.13.

Certificados

Certificates in general are part of a public-key / private-key system. In this system, each *principal*, (which may be a machine, or a person, or an organization) is assigned a unique two-part encryption key. One part of the key is public, and is called the *public key*; the other part is kept secret, and is called the *private key*. The two parts are related, in that if you encrypt a message with one of the parts, you can decrypt it with the other part, and **only** with the other part.

A certificate contains information about two principals. It contains the name of a *subject*, and the subject's public key. It also contains a statement by a second principal, the *issuer*, that the subject is who they claim to be, and that this is indeed the subject's public key. The issuer's statement is signed with the issuer's private key, which only the issuer knows. However, anyone can verify the issuer's statement by finding the issuer's public key, decrypting the statement with it, and comparing it to the other information in the certificate. The certificate also contains information about the time period over which it is valid. This is expressed as two fields, called "notBefore" and "notAfter".

In the Python use of certificates, a client or server can use a certificate to prove who they are. The other side of a network connection can also be required to produce a certificate, and that certificate can be validated to the

satisfaction of the client or server that requires such validation. The connection attempt can be set to raise an exception if the validation fails. Validation is done automatically, by the underlying OpenSSL framework; the application need not concern itself with its mechanics. But the application does usually need to provide sets of certificates to allow this process to take place.

Python uses files to contain certificates. They should be formatted as "PEM" (see **RFC 1422** [https://datatracker.ietf.org/doc/html/rfc1422.html]), which is a base-64 encoded form wrapped with a header line and a footer line:

```
-----BEGIN CERTIFICATE-----
... (certificate in base64 PEM encoding) ...
-----END CERTIFICATE-----
```

Certificate chains

The Python files which contain certificates can contain a sequence of certificates, sometimes called a *certificate chain*. This chain should start with the specific certificate for the principal who "is" the client or server, and then the certificate for the issuer of that certificate, and then the certificate for the issuer of *that* certificate, and so on up the chain till you get to a certificate which is *self-signed*, that is, a certificate which has the same subject and issuer, sometimes called a *root certificate*. The certificates should just be concatenated together in the certificate file. For example, suppose we had a three certificate chain, from our server certificate to the certificate of the certification authority that signed our server certificate, to the root certificate of the agency which issued the certification authority's certificate:

```
-----BEGIN CERTIFICATE-----
... (certificate for your server)...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (the certificate for the CA)...
-----END CERTIFICATE-----
-----BEGIN CERTIFICATE-----
... (the root certificate for the CA's issuer)...
-----END CERTIFICATE-----
```

CA certificates

If you are going to require validation of the other side of the connection's certificate, you need to provide a "CA certs" file, filled with the certificate chains for each issuer you are willing to trust. Again, this file just contains these chains concatenated together. For validation, Python will use the first chain it finds in the file which matches. The platform's certificates file can be used by calling `SSLContext.load_default_certs()`, this is done automatically with `create_default_context()`.

Combined key and certificate

Often the private key is stored in the same file as the certificate; in this case, only the `certfile` parameter to `SSLContext.load_cert_chain()` needs to be passed. If the private key is stored with the certificate, it should come before the first certificate in the certificate chain:

```
-----BEGIN RSA PRIVATE KEY-----
... (private key in base64 encoding) ...
-----END RSA PRIVATE KEY-----
-----BEGIN CERTIFICATE-----
... (certificate in base64 PEM encoding) ...
-----END CERTIFICATE-----
```

Self-signed certificates

If you are going to create a server that provides SSL-encrypted connection services, you will need to acquire a certificate for that service. There are many ways of acquiring appropriate certificates, such as buying one from a certification authority. Another common practice is to generate a self-signed certificate. The simplest way to do this is with the OpenSSL package, using something like the following:

```
% openssl req -new -x509 -days 365 -nodes -out cert.pem -keyout
Generating a 1024 bit RSA private key
.......++++++
.............................++++++
writing new private key to 'cert.pem'
```

```
-----
You are about to be asked to enter information that will be incc
into your certificate request.
What you are about to enter is what is called a Distinguished Na
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:US
State or Province Name (full name) [Some-State]:MyState
Locality Name (eg, city) []:Some City
Organization Name (eg, company) [Internet Widgits Pty Ltd]:My Or
Organizational Unit Name (eg, section) []:My Group
Common Name (eg, YOUR name) []:myserver.mygroup.myorganization.c
Email Address []:ops@myserver.mygroup.myorganization.com
%
```

The disadvantage of a self-signed certificate is that it is its own root certificate, and no one else will have it in their cache of known (and trusted) root certificates.

Exemplos

Testing for SSL support

To test for the presence of SSL support in a Python installation, user code should use the following idiom:

```
try:
    import ssl
except ImportError:
    pass
else:
    ...  # do something that requires SSL support
```

Client-side operation

This example creates a SSL context with the recommended security settings for client sockets, including automatic certificate verification:

```
>>> context = ssl.create_default_context()
```

If you prefer to tune security settings yourself, you might create a context from scratch (but beware that you might not get the settings right):

```
>>> context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
>>> context.load_verify_locations("/etc/ssl/certs/ca-bundle.crt'
```

(this snippet assumes your operating system places a bundle of all CA certificates in `/etc/ssl/certs/ca-bundle.crt`; if not, you'll get an error and have to adjust the location)

The **`PROTOCOL_TLS_CLIENT`** protocol configures the context for cert validation and hostname verification. **`verify_mode`** is set to **`CERT_REQUIRED`** and **`check_hostname`** is set to `True`. All other protocols create SSL contexts with insecure defaults.

When you use the context to connect to a server, **`CERT_REQUIRED`** and **`check_hostname`** validate the server certificate: it ensures that the server certificate was signed with one of the CA certificates, checks the signature for correctness, and verifies other properties like validity and identity of the hostname:

```
>>> conn = context.wrap_socket(socket.socket(socket.AF_INET),
...                            server_hostname="www.python.org")
>>> conn.connect(("www.python.org", 443))
```

You may then fetch the certificate:

```
>>> cert = conn.getpeercert()
```

Visual inspection shows that the certificate does identify the desired service (that is, the HTTPS host `www.python.org`):

```
>>> pprint.pprint(cert)
{'OCSP': ('http://ocsp.digicert.com',),
 'caIssuers': ('http://cacerts.digicert.com/DigiCertSHA2Extendec
 'crlDistributionPoints': ('http://crl3.digicert.com/sha2-ev-ser
                           'http://crl4.digicert.com/sha2-ev-ser
```

```
 'issuer': ((('countryName', 'US'),),
            (('organizationName', 'DigiCert Inc'),),
            (('organizationalUnitName', 'www.digicert.com'),),
            (('commonName', 'DigiCert SHA2 Extended Validation S
 'notAfter': 'Sep  9 12:00:00 2016 GMT',
 'notBefore': 'Sep  5 00:00:00 2014 GMT',
 'serialNumber': '01BB6F00122B177F36CAB49CEA8B6B26',
 'subject': ((('businessCategory', 'Private Organization'),),
             (('1.3.6.1.4.1.311.60.2.1.3', 'US'),),
             (('1.3.6.1.4.1.311.60.2.1.2', 'Delaware'),),
             (('serialNumber', '3359300'),),
             (('streetAddress', '16 Allen Rd'),),
             (('postalCode', '03894-4801'),),
             (('countryName', 'US'),),
             (('stateOrProvinceName', 'NH'),),
             (('localityName', 'Wolfeboro'),),
             (('organizationName', 'Python Software Foundation')
             (('commonName', 'www.python.org'),)),
 'subjectAltName': (('DNS', 'www.python.org'),
                    ('DNS', 'python.org'),
                    ('DNS', 'pypi.org'),
                    ('DNS', 'docs.python.org'),
                    ('DNS', 'testpypi.org'),
                    ('DNS', 'bugs.python.org'),
                    ('DNS', 'wiki.python.org'),
                    ('DNS', 'hg.python.org'),
                    ('DNS', 'mail.python.org'),
                    ('DNS', 'packaging.python.org'),
                    ('DNS', 'pythonhosted.org'),
                    ('DNS', 'www.pythonhosted.org'),
                    ('DNS', 'test.pythonhosted.org'),
                    ('DNS', 'us.pycon.org'),
                    ('DNS', 'id.python.org')),
 'version': 3}
```

Now the SSL channel is established and the certificate verified, you can proceed to talk with the server:

```
>>> conn.sendall(b"HEAD / HTTP/1.0\r\nHost: linuxfr.org\r\n\r\n'
>>> pprint.pprint(conn.recv(1024).split(b"\r\n"))
[b'HTTP/1.1 200 OK',
 b'Date: Sat, 18 Oct 2014 18:27:20 GMT',
```

```
 b'Server: nginx',
 b'Content-Type: text/html; charset=utf-8',
 b'X-Frame-Options: SAMEORIGIN',
 b'Content-Length: 45679',
 b'Accept-Ranges: bytes',
 b'Via: 1.1 varnish',
 b'Age: 2188',
 b'X-Served-By: cache-lcy1134-LCY',
 b'X-Cache: HIT',
 b'X-Cache-Hits: 11',
 b'Vary: Cookie',
 b'Strict-Transport-Security: max-age=63072000; includeSubDomair
 b'Connection: close',
 b'',
 b'']
```

See the discussion of Considerações de segurança below.

Server-side operation

For server operation, typically you'll need to have a server certificate, and private key, each in a file. You'll first create a context holding the key and the certificate, so that clients can check your authenticity. Then you'll open a socket, bind it to a port, call **`listen()`** on it, and start waiting for clients to connect:

```
import socket, ssl

context = ssl.create_default_context(ssl.Purpose.CLIENT_AUTH)
context.load_cert_chain(certfile="mycertfile", keyfile="mykeyfil

bindsocket = socket.socket()
bindsocket.bind(('myaddr.example.com', 10023))
bindsocket.listen(5)
```

When a client connects, you'll call **`accept()`** on the socket to get the new socket from the other end, and use the context's **`SSLContext.wrap_socket()`** method to create a server-side SSL socket for the connection:

```
while True:
    newsocket, fromaddr = bindsocket.accept()
    connstream = context.wrap_socket(newsocket, server_side=True
    try:
        deal_with_client(connstream)
    finally:
        connstream.shutdown(socket.SHUT_RDWR)
        connstream.close()
```

Then you'll read data from the `connstream` and do something with it till you are finished with the client (or the client is finished with you):

```
def deal_with_client(connstream):
    data = connstream.recv(1024)
    # empty data means the client is finished with us
    while data:
        if not do_something(connstream, data):
            # we'll assume do_something returns False
            # when we're finished with client
            break
        data = connstream.recv(1024)
    # finished with client
```

And go back to listening for new client connections (of course, a real server would probably handle each client connection in a separate thread, or put the sockets in non-blocking mode and use an event loop).

Notes on non-blocking sockets

SSL sockets behave slightly different than regular sockets in non-blocking mode. When working with non-blocking sockets, there are thus several things you need to be aware of:

- Most **`SSLSocket`** methods will raise either **`SSLWantWriteError`** or **`SSLWantReadError`** instead of **`BlockingIOError`** if an I/O operation would block. **`SSLWantReadError`** will be raised if a read operation on the underlying socket is necessary, and **`SSLWantWriteError`** for a write operation on the underlying socket. Note that attempts to *write* to an SSL socket may require *reading* from the underlying socket first, and

attempts to *read* from the SSL socket may require a prior *write* to the underlying socket.

Alterado na versão 3.5: In earlier Python versions, the **`SSLSocket.send()`** method returned zero instead of raising **`SSLWantWriteError`** or **`SSLWantReadError`**.

- Calling **`select()`** tells you that the OS-level socket can be read from (or written to), but it does not imply that there is sufficient data at the upper SSL layer. For example, only part of an SSL frame might have arrived. Therefore, you must be ready to handle **`SSLSocket.recv()`** and **`SSLSocket.send()`** failures, and retry after another call to **`select()`**.

- Conversely, since the SSL layer has its own framing, a SSL socket may still have data available for reading without **`select()`** being aware of it. Therefore, you should first call **`SSLSocket.recv()`** to drain any potentially available data, and then only block on a **`select()`** call if still necessary.

  (of course, similar provisions apply when using other primitives such as **`poll()`**, or those in the **`selectors`** module)

- The SSL handshake itself will be non-blocking: the **`SSLSocket.do_handshake()`** method has to be retried until it returns successfully. Here is a synopsis using **`select()`** to wait for the socket's readiness:

```
while True:
    try:
        sock.do_handshake()
        break
    except ssl.SSLWantReadError:
        select.select([sock], [], [])
    except ssl.SSLWantWriteError:
        select.select([], [sock], [])
```

Ver também

The `asyncio` module supports non-blocking SSL sockets and provides a higher level Streams API. It polls for events using the `selectors` module and handles `SSLWantWriteError`, `SSLWantReadError` and `BlockingIOError` exceptions. It runs the SSL handshake asynchronously as well.

Memory BIO support

Adicionado na versão 3.5.

Ever since the SSL module was introduced in Python 2.6, the `SSLSocket` class has provided two related but distinct areas of functionality:

- SSL protocol handling
- Network IO

The network IO API is identical to that provided by `socket.socket`, from which `SSLSocket` also inherits. This allows an SSL socket to be used as a drop-in replacement for a regular socket, making it very easy to add SSL support to an existing application.

Combining SSL protocol handling and network IO usually works well, but there are some cases where it doesn't. An example is async IO frameworks that want to use a different IO multiplexing model than the "select/poll on a file descriptor" (readiness based) model that is assumed by `socket.socket` and by the internal OpenSSL socket IO routines. This is mostly relevant for platforms like Windows where this model is not efficient. For this purpose, a reduced scope variant of `SSLSocket` called `SSLObject` is provided.

class ssl.SSLObject

A reduced-scope variant of `SSLSocket` representing an SSL protocol instance that does not contain any network IO methods. This class is typically used by framework authors that want to implement asynchronous IO for SSL through memory buffers.

This class implements an interface on top of a low-level SSL object as implemented by OpenSSL. This object captures the state of an SSL connection but does not provide any network IO itself. IO needs to be performed through separate “BIO” objects which are OpenSSL’s IO abstraction layer.

This class has no public constructor. An **`SSLObject`** instance must be created using the `wrap_bio()` method. This method will create the **`SSLObject`** instance and bind it to a pair of BIOs. The *incoming* BIO is used to pass data from Python to the SSL protocol instance, while the *outgoing* BIO is used to pass data the other way around.

The following methods are available:

- `context`
- `server_side`
- `server_hostname`
- `session`
- `session_reused`
- `read()`
- `write()`
- `getpeercert()`
- `get_verified_chain()`
- `get_unverified_chain()`
- `selected_alpn_protocol()`
- `selected_npn_protocol()`
- `cipher()`
- `shared_ciphers()`
- `compression()`
- `pending()`
- `do_handshake()`
- `verify_client_post_handshake()`
- `unwrap()`
- `get_channel_binding()`
- `version()`

When compared to `SSLSocket`, this object lacks the following features:

- Any form of network IO; `recv()` and `send()` read and write only to the underlying **`MemoryBIO`** buffers.
- There is no *do_handshake_on_connect* machinery. You must always manually call **`do_handshake()`** to start the handshake.
- There is no handling of *suppress_ragged_eofs*. All end-of-file conditions that are in violation of the protocol are reported via the **`SSLEOFError`** exception.
- The method **`unwrap()`** call does not return anything, unlike for an SSL socket where it returns the underlying socket.
- The *server_name_callback* callback passed to **`SSLContext.set_servername_callback()`** will get an **`SSLObject`** instance instead of a **`SSLSocket`** instance as its first parameter.

Some notes related to the use of **`SSLObject`**:

- All IO on an **`SSLObject`** is non-blocking. This means that for example **`read()`** will raise an **`SSLWantReadError`** if it needs more data than the incoming BIO has available.

Alterado na versão 3.7: **`SSLObject`** instances must be created with **`wrap_bio()`**. In earlier versions, it was possible to create instances directly. This was never documented or officially supported.

An SSLObject communicates with the outside world using memory buffers. The class **`MemoryBIO`** provides a memory buffer that can be used for this purpose. It wraps an OpenSSL memory BIO (Basic IO) object:

class ssl.MemoryBIO

A memory buffer that can be used to pass data between Python and an SSL protocol instance.

pending

Return the number of bytes currently in the memory buffer.

eof

A boolean indicating whether the memory BIO is current at the end-of-file position.

read(*n=-1*, *[/]*)

Read up to *n* bytes from the memory buffer. If *n* is not specified or negative, all bytes are returned.

write(*buf*, *[/]*)

Write the bytes from *buf* to the memory BIO. The *buf* argument must be an object supporting the buffer protocol.

The return value is the number of bytes written, which is always equal to the length of *buf*.

write_eof()

Write an EOF marker to the memory BIO. After this method has been called, it is illegal to call `write()`. The attribute `eof` will become true after all data currently in the buffer has been read.

SSL session

Adicionado na versão 3.6.

class ssl.SSLSession

Session object used by `session`.

id

time

timeout

ticket_lifetime_hint

has_ticket

Considerações de segurança

Best defaults

For **client use**, if you don't have any special requirements for your security policy, it is highly recommended that you use the `create_default_context()` function to create your SSL context. It will load the system's trusted CA certificates, enable certificate validation and hostname checking, and try to choose reasonably secure protocol and cipher settings.

For example, here is how you would use the `smtplib.SMTP` class to create a trusted, secure connection to a SMTP server:

```
>>> import ssl, smtplib
>>> smtp = smtplib.SMTP("mail.python.org", port=587)
>>> context = ssl.create_default_context()
>>> smtp.starttls(context=context)
(220, b'2.0.0 Ready to start TLS')
```

If a client certificate is needed for the connection, it can be added with `SSLContext.load_cert_chain()`.

By contrast, if you create the SSL context by calling the `SSLContext` constructor yourself, it will not have certificate validation nor hostname checking enabled by default. If you do so, please read the paragraphs below to achieve a good security level.

Manual settings

Verifying certificates

When calling the `SSLContext` constructor directly, `CERT_NONE` is the default. Since it does not authenticate the other peer, it can be insecure, especially in client mode where most of the time you would like to ensure the authenticity of the server you're talking to. Therefore, when in client mode, it is highly recommended to use `CERT_REQUIRED`. However, it is in itself not sufficient; you also have to check that the server certificate, which can be obtained by calling `SSLSocket.getpeercert()`, matches the desired service. For many protocols and applications, the service can be identified by the hostname. This common check is automatically performed when `SSLContext.check_hostname` is enabled.

Alterado na versão 3.7: Hostname matchings is now performed by OpenSSL. Python no longer uses **`match_hostname()`**.

In server mode, if you want to authenticate your clients using the SSL layer (rather than using a higher-level authentication mechanism), you'll also have to specify **`CERT_REQUIRED`** and similarly check the client certificate.

Protocol versions

SSL versions 2 and 3 are considered insecure and are therefore dangerous to use. If you want maximum compatibility between clients and servers, it is recommended to use **`PROTOCOL_TLS_CLIENT`** or **`PROTOCOL_TLS_SERVER`** as the protocol version. SSLv2 and SSLv3 are disabled by default.

```
>>> client_context = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
>>> client_context.minimum_version = ssl.TLSVersion.TLSv1_2
>>> client_context.maximum_version = ssl.TLSVersion.TLSv1_3
```

The SSL client context created above will only allow TLSv1.2 and TLSv1.3 (if supported by your system) connections to a server. **`PROTOCOL_TLS_CLIENT`** implies certificate validation and hostname checks by default. You have to load certificates into the context.

Cipher selection

If you have advanced security requirements, fine-tuning of the ciphers enabled when negotiating a SSL session is possible through the **`SSLContext.set_ciphers()`** method. Starting from Python 3.2.3, the ssl module disables certain weak ciphers by default, but you may want to further restrict the cipher choice. Be sure to read OpenSSL's documentation about the cipher list format [https://docs.openssl.org/1.1.1/man1/ciphers/#cipher-list-format]. If you want to check which ciphers are enabled by a given cipher list, use **`SSLContext.get_ciphers()`** or the `openssl ciphers` command on your system.

Multi-processing

If using this module as part of a multi-processed application (using, for example the `multiprocessing` or `concurrent.futures` modules), be aware that OpenSSL's internal random number generator does not properly handle forked processes. Applications must change the PRNG state of the parent process if they use any SSL feature with `os.fork()`. Any successful call of `RAND_add()` or `RAND_bytes()` is sufficient.

TLS 1.3

Adicionado na versão 3.7.

The TLS 1.3 protocol behaves slightly differently than previous version of TLS/SSL. Some new TLS 1.3 features are not yet available.

- TLS 1.3 uses a disjunct set of cipher suites. All AES-GCM and ChaCha20 cipher suites are enabled by default. To restrict which TLS 1.3 ciphers are allowed, the `SSLContext.set_ciphersuites()` method should be called instead of `SSLContext.set_ciphers()`, which only affects ciphers in older TLS versions. The `SSLContext.get_ciphers()` method returns information about ciphers for both TLS 1.3 and earlier versions and the method `SSLSocket.cipher()` returns information about the negotiated cipher for both TLS 1.3 and earlier versions once a connection is established.
- Session tickets are no longer sent as part of the initial handshake and are handled differently. `SSLSocket.session` and `SSLSession` are not compatible with TLS 1.3.
- Client-side certificates are also no longer verified during the initial handshake. A server can request a certificate at any time. Clients process certificate requests while they send or receive application data from the server.
- TLS 1.3 features like early data, deferred TLS client cert request, and rekeying are not supported yet.

Ver também

Class `socket.socket`

Documentation of underlying `socket` class

SSL/TLS Strong Encryption: An Introduction
[https://httpd.apache.org/docs/trunk/en/ssl/ssl_intro.html]
Intro from the Apache HTTP Server documentation

RFC 1422: Privacy Enhancement for Internet Electronic Mail: Part II: Certificate-Based Key Management
[https://datatracker.ietf.org/doc/html/rfc1422.html]
Steve Kent

RFC 4086: Randomness Requirements for Security
[https://datatracker.ietf.org/doc/html/rfc4086.html]
Donald E. Eastlake, Jeffrey I. Schiller, Steve Crocker

RFC 5280: Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile
[https://datatracker.ietf.org/doc/html/rfc5280.html]
David Cooper et al.

RFC 5246: The Transport Layer Security (TLS) Protocol Version 1.2
[https://datatracker.ietf.org/doc/html/rfc5246.html]
Tim Dierks and Eric Rescorla.

RFC 6066: Transport Layer Security (TLS) Extensions
[https://datatracker.ietf.org/doc/html/rfc6066.html]
Donald E. Eastlake

IANA TLS: Transport Layer Security (TLS) Parameters
[https://www.iana.org/assignments/tls-parameters/tls-parameters.xml]
IANA

RFC 7525: Recommendations for Secure Use of Transport Layer Security (TLS) and Datagram Transport Layer Security (DTLS)
[https://datatracker.ietf.org/doc/html/rfc7525.html]
IETF

Mozilla's Server Side TLS recommendations
[https://wiki.mozilla.org/Security/Server_Side_TLS]
Mozilla

`select` — Waiting for I/O completion

Este módulo fornece acesso às funções **`select()`** e **`poll()`** disponíveis na maioria dos sistemas operacionais, **`devpoll()`** disponível no Solaris e derivados, **`epoll()`** disponível no Linux 2.5+ e **`kqueue()`** disponível na maioria dos BSD. Observe que no Windows, ele funciona apenas para sockets; em outros sistemas operacionais, ele também funciona para outros tipos de arquivo (em particular, no Unix, ele funciona em pipes). Ele não pode ser usado em arquivos regulares para determinar se um arquivo cresceu desde a última leitura.

Nota

The **`selectors`** module allows high-level and efficient I/O multiplexing, built upon the **`select`** module primitives. Users are encouraged to use the **`selectors`** module instead, unless they want precise control over the OS-level primitives used.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

O módulo define o seguinte:

exception select.error

Um apelido descontinuado de **`OSError`**.

Alterado na versão 3.3: Seguindo a **PEP 3151** [https://peps.python.org/pep-3151/], esta classe foi transformada em um apelido de **`OSError`**.

select.devpoll()

Returns a `/dev/poll` polling object; see section /dev/poll polling objects below for the methods supported by devpoll objects.

Objetos **`devpoll()`** são vinculados ao número de descritores de arquivo permitidos no momento da instanciação. Se seu programa reduzir esse valor, **`devpoll()`** falhará. Se seu programa aumentar esse valor, **`devpoll()`** pode retornar uma lista incompleta de descritores de arquivo ativos.

O novo descritor de arquivo é não herdável.

Adicionado na versão 3.3.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-hereditário.

Disponibilidade: Solaris.

select.epoll(*sizehint=-1*, *flags=0*)

Return an edge polling object, which can be used as Edge or Level Triggered interface for I/O events.

sizehint informs epoll about the expected number of events to be registered. It must be positive, or `-1` to use the default. It is only used on older systems where *epoll_create1(2)* *[https://manpages.debian.org/epoll_create1(2)]* is not available; otherwise it has no effect (though its value is still checked).

flags is deprecated and completely ignored. However, when supplied, its value must be `0` or `select.EPOLL_CLOEXEC`, otherwise `OSError` is raised.

See the Edge and level trigger polling (epoll) objects section below for the methods supported by epolling objects.

`epoll` objects support the context management protocol: when used in a **`with`** statement, the new file descriptor is automatically closed at the end of the block.

O novo descritor de arquivo é não herdável.

Alterado na versão 3.3: Added the *flags* parameter.

Alterado na versão 3.4: Support for the `with` statement was added. The new file descriptor is now non-inheritable.

Descontinuado desde a versão 3.4: The *flags* parameter. `select.EPOLL_CLOEXEC` is used by default now. Use `os.set_inheritable()` to make the file descriptor inheritable.

Alterado na versão 3.15: When CPython is built, this function may be disabled using `--disable-epoll`.

Disponibilidade: Linux >= 2.5.44.

select.poll()

Returns a polling object, which supports registering and unregistering file descriptors, and then polling them for I/O events; see section Polling objects below for the methods supported by polling objects.

Disponibilidade: Unix.

select.kqueue()

Returns a kernel queue object; see section Kqueue objects below for the methods supported by kqueue objects.

O novo descritor de arquivo é não herdável.

Alterado na versão 3.4: O novo descritor de arquivo agora é não-hereditário.

Disponibilidade: BSD, macOS.

select.kevent(*ident*, *filter=KQ_FILTER_READ*, *flags=KQ_EV_ADD*, *fflags=0*, *data=0*, *udata=0*)

Returns a kernel event object; see section Kevent objects below for the methods supported by kevent objects.

Disponibilidade: BSD, macOS.

select.select(*rlist*, *wlist*, *xlist*, *timeout=None*)

This is a straightforward interface to the Unix **`select()`** system call. The first three arguments are iterables of 'waitable objects': either integers representing file descriptors or objects with a parameterless method named **`fileno()`** returning such an integer:

- *rlist*: wait until ready for reading
- *wlist*: wait until ready for writing
- *xlist*: wait for an "exceptional condition" (see the manual page for what your system considers such a condition)

Empty iterables are allowed, but acceptance of three empty iterables is platform-dependent. (It is known to work on Unix but not on Windows.) The optional *timeout* argument specifies a time-out in seconds; it may be a non-integer to specify fractions of seconds. When the *timeout* argument is omitted or `None`, the function blocks until at least one file descriptor is ready. A time-out value of zero specifies a poll and never blocks.

The return value is a triple of lists of objects that are ready: subsets of the first three arguments. When the time-out is reached without a file descriptor becoming ready, three empty lists are returned.

Among the acceptable object types in the iterables are Python file objects (e.g. `sys.stdin`, or objects returned by **`open()`** or **`os.popen()`**), socket objects returned by **`socket.socket()`**. You may also define a *wrapper* class yourself, as long as it has an appropriate **`fileno()`** method (that really returns a file descriptor, not just a random integer).

Nota

File objects on Windows are not acceptable, but sockets are. On Windows, the underlying **`select()`** function is provided by the WinSock library, and does not handle file descriptors that don't originate from WinSock.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale), instead of raising `InterruptedError`.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

select.PIPE_BUF

The minimum number of bytes which can be written without blocking to a pipe when the pipe has been reported as ready for writing by `select()`, `poll()` or another interface in this module. This doesn't apply to other kinds of file-like objects such as sockets.

This value is guaranteed by POSIX to be at least 512.

Disponibilidade: Unix

Adicionado na versão 3.2.

`/dev/poll` polling objects

Solaris and derivatives have `/dev/poll`. While `select()` is *O*(*highest file descriptor*) and `poll()` is *O*(*number of file descriptors*), `/dev/poll` is *O*(*active file descriptors*).

`/dev/poll` behaviour is very close to the standard `poll()` object.

devpoll.close()

Close the file descriptor of the polling object.

Adicionado na versão 3.4.

devpoll.closed

`True` if the polling object is closed.

Adicionado na versão 3.4.

devpoll.fileno()

Return the file descriptor number of the polling object.

Adicionado na versão 3.4.

devpoll.register(*fd*[, *eventmask*])

Register a file descriptor with the polling object. Future calls to the `poll()` method will then check whether the file descriptor has any pending I/O events. *fd* can be either an integer, or an object with a `fileno()` method that returns an integer. File objects implement `fileno()`, so they can also be used as the argument.

eventmask is an optional bitmask describing the type of events you want to check for. The constants are the same as with `poll()` object. The default value is a combination of the constants `POLLIN`, `POLLPRI`, and `POLLOUT`.

Aviso

Registering a file descriptor that's already registered is not an error, but the result is undefined. The appropriate action is to unregister or modify it first. This is an important difference compared with `poll()`.

devpoll.modify(*fd*[, *eventmask*])

This method does an `unregister()` followed by a `register()`. It is (a bit) more efficient than doing the same explicitly.

devpoll.unregister(*fd*)

Remove a file descriptor being tracked by a polling object. Just like the `register()` method, *fd* can be an integer or an object with a `fileno()` method that returns an integer.

Attempting to remove a file descriptor that was never registered is safely ignored.

devpoll.poll([*timeout*])

Polls the set of registered file descriptors, and returns a possibly empty list containing `(fd, event)` 2-tuples for the descriptors that have events or errors to report. *fd* is the file descriptor, and *event* is a bitmask with bits set for the reported events for that descriptor — **`POLLIN`** for waiting input, **`POLLOUT`** to indicate that the descriptor can be written to, and so forth. An empty list indicates that the call timed out and no file descriptors had any events to report. If *timeout* is given, it specifies the length of time in milliseconds which the system will wait for events before returning. If *timeout* is omitted, -1, or **`None`**, the call will block until there is an event for this poll object.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale), instead of raising **`InterruptedError`**.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

Edge and level trigger polling (epoll) objects

https://linux.die.net/man/4/epoll

The *eventmask* is a bit mask using the following constants:

Constante	Significado
`EPOLLIN`	Available for read.
`EPOLLOUT`	Available for write.

Constante	Significado
`EPOLLPRI`	Urgent data for read.
`EPOLLERR`	Error condition happened on the associated fd.
`EPOLLHUP`	Hang up happened on the associated fd.
`EPOLLET`	Set Edge Trigger behavior, the default is Level Trigger behavior.
`EPOLLONESHOT`	Set one-shot behavior. After one event is pulled out, the fd is internally disabled.
`EPOLLEXCLUSIVE`	Wake only one epoll object when the associated fd has an event. The default (if this flag is not set) is to wake all epoll objects polling on an fd.
`EPOLLRDHUP`	Stream socket peer closed connection or shut down writing half of connection.
`EPOLLRDNORM`	Equivalent to **`EPOLLIN`**
`EPOLLRDBAND`	Priority data band can be read.
`EPOLLWRNORM`	Equivalent to **`EPOLLOUT`**.

Constante	Significado
`EPOLLWRBAND`	Priority data may be written.
`EPOLLMSG`	Ignorado.
`EPOLLWAKEUP`	Prevents sleep during event waiting.

Adicionado na versão 3.6: **`EPOLLEXCLUSIVE`** was added. It's only supported by Linux Kernel 4.5 or later.

Adicionado na versão 3.14: **`EPOLLWAKEUP`** was added. It's only supported by Linux Kernel 3.5 or later.

epoll.close()

Close the control file descriptor of the epoll object.

epoll.closed

`True` if the epoll object is closed.

epoll.fileno()

Return the file descriptor number of the control fd.

epoll.fromfd(*fd*)

Create an epoll object from a given file descriptor.

epoll.register(*fd*[, *eventmask*])

Register a file descriptor *fd* with the epoll object.

epoll.modify(*fd*, *eventmask*)

Modify a registered file descriptor *fd*.

epoll.unregister(*fd*)

Remove a registered file descriptor from the epoll object.

Alterado na versão 3.9: The method no longer ignores the `EBADF` error.

epoll.poll(*timeout=None*, *maxevents=-1*)

Wait for events. If *timeout* is given, it specifies the length of time in seconds (may be non-integer) which the system will wait for events before returning.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale), instead of raising `InterruptedError`.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

Polling objects

The `poll()` system call, supported on most Unix systems, provides better scalability for network servers that service many, many clients at the same time. `poll()` scales better because the system call only requires listing the file descriptors of interest, while `select()` builds a bitmap, turns on bits for the fds of interest, and then afterward the whole bitmap has to be linearly scanned again. `select()` is *O*(*highest file descriptor*), while `poll()` is *O*(*number of file descriptors*).

poll.register(*fd*[, *eventmask*])

Register a file descriptor with the polling object. Future calls to the `poll()` method will then check whether the file descriptor has any pending I/O events. *fd* can be either an integer, or an object with a `fileno()` method that returns an integer. File objects implement `fileno()`, so they can also be used as the argument.

eventmask is an optional bitmask describing the type of events you want to check for, and can be a combination of the constants **POLLIN**, **POLLPRI**, and **POLLOUT**, described in the table below. If not specified, the default value used will check for all 3 types of events.

Constante	Significado
POLLIN	There is data to read.
POLLPRI	There is urgent data to read.
POLLOUT	Ready for output: writing will not block.
POLLERR	Error condition of some sort.
POLLHUP	Hung up.
POLLRDHUP	Stream socket peer closed connection, or shut down writing half of connection.
POLLNVAL	Invalid request: descriptor not open.

Registering a file descriptor that's already registered is not an error, and has the same effect as registering the descriptor exactly once.

poll.modify(*fd*, *eventmask*)

Modifies an already registered fd. This has the same effect as `register(fd, eventmask)`. Attempting to modify a file descriptor that was never registered causes an **OSError** exception with errno **ENOENT** to be raised.

poll.unregister(*fd*)

Remove a file descriptor being tracked by a polling object. Just like the **register()** method, *fd* can be an integer or an object with a **fileno()** method that returns an integer.

Attempting to remove a file descriptor that was never registered causes a **KeyError** exception to be raised.

poll.poll([*timeout*])

Polls the set of registered file descriptors, and returns a possibly empty list containing `(fd, event)` 2-tuples for the descriptors that have events or errors to report. *fd* is the file descriptor, and *event* is a bitmask with bits set for the reported events for that descriptor — **POLLIN** for waiting input, **POLLOUT** to indicate that the descriptor can be written to, and so forth. An empty list indicates that the call timed out and no file descriptors had any events to report. If *timeout* is given, it specifies the length of time in milliseconds which the system will wait for events before returning. If *timeout* is omitted, negative, or **None**, the call will block until there is an event for this poll object.

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale), instead of raising **InterruptedError**.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float. If `ppoll()` function is available, *timeout* has a resolution of `1` ns (`1e-6` ms) instead of `1` ms.

Kqueue objects

kqueue.close()

Close the control file descriptor of the kqueue object.

kqueue.closed

`True` if the kqueue object is closed.

kqueue.fileno()

Return the file descriptor number of the control fd.

kqueue.fromfd(*fd*)

Create a kqueue object from a given file descriptor.

kqueue.control(*changelist*, *max_events*[, *timeout*]) → eventlist

Low level interface to kevent

- changelist must be an iterable of kevent objects or `None`
- max_events must be 0 or a positive integer
- timeout in seconds (non-integers are possible); the default is `None`, to wait forever

Alterado na versão 3.5: The function is now retried with a recomputed timeout when interrupted by a signal, except if the signal handler raises an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale), instead of raising `InterruptedError`.

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

Kevent objects

https://man.freebsd.org/cgi/man.cgi?query=kqueue&sektion=2

kevent.ident

Value used to identify the event. The interpretation depends on the filter but it's usually the file descriptor. In the constructor ident can either be an int or an object with a `fileno()` method. kevent stores the integer internally.

kevent.filter

Name of the kernel filter.

Constante	Significado
`KQ_FILTER_READ`	Takes a descriptor and returns whenever there is data available to read.

Constante	Significado
`KQ_FILTER_WRITE`	Takes a descriptor and returns whenever there is data available to write.
`KQ_FILTER_AIO`	AIO requests.
`KQ_FILTER_VNODE`	Returns when one or more of the requested events watched in *fflag* occurs.
`KQ_FILTER_PROC`	Watch for events on a process ID.
`KQ_FILTER_NETDEV`	Watch for events on a network device (not available on macOS).
`KQ_FILTER_SIGNAL`	Returns whenever the watched signal is delivered to the process.
`KQ_FILTER_TIMER`	Establishes an arbitrary timer.

kevent.flags

Filter action.

Constante	Significado
`KQ_EV_ADD`	Adds or modifies an event.
`KQ_EV_DELETE`	Removes an event from the queue.
`KQ_EV_ENABLE`	Permits control() to return the event.
`KQ_EV_DISABLE`	Disables event.
`KQ_EV_ONESHOT`	Removes event after first occurrence.

Constante	Significado
KQ_EV_CLEAR	Reset the state after an event is retrieved.
KQ_EV_SYSFLAGS	Internal event.
KQ_EV_FLAG1	Internal event.
KQ_EV_EOF	Filter-specific EOF condition.
KQ_EV_ERROR	See return values.

kevent.fflags

Filter-specific flags.

KQ_FILTER_READ and **KQ_FILTER_WRITE** filter flags:

Constante	Significado
KQ_NOTE_LOWAT	Low water mark of a socket buffer.

KQ_FILTER_VNODE filter flags:

Constante	Significado
KQ_NOTE_DELETE	*unlink()* was called.
KQ_NOTE_WRITE	A write occurred.
KQ_NOTE_EXTEND	The file was extended.
KQ_NOTE_ATTRIB	An attribute was changed.
KQ_NOTE_LINK	The link count has changed.

Constante	Significado
`KQ_NOTE_RENAME`	The file was renamed.
`KQ_NOTE_REVOKE`	Access to the file was revoked.

`KQ_FILTER_PROC` filter flags:

Constante	Significado
`KQ_NOTE_EXIT`	The process has exited.
`KQ_NOTE_FORK`	The process has called *fork()*.
`KQ_NOTE_EXEC`	The process has executed a new process.
`KQ_NOTE_PCTRLMASK`	Internal filter flag.
`KQ_NOTE_PDATAMASK`	Internal filter flag.
`KQ_NOTE_TRACK`	Follow a process across *fork()*.
`KQ_NOTE_CHILD`	Returned on the child process for *NOTE_TRACK*.
`KQ_NOTE_TRACKERR`	Unable to attach to a child.

`KQ_FILTER_NETDEV` filter flags (not available on macOS):

Constante	Significado
`KQ_NOTE_LINKUP`	Link is up.
`KQ_NOTE_LINKDOWN`	Link is down.

Constante	Significado
`KQ_NOTE_LINKINV`	Link state is invalid.

kevent.data
: Filter-specific data.

kevent.udata
: User-defined value.

`selectors` — High-level I/O multiplexing

Adicionado na versão 3.4.

Código-fonte: Lib/selectors.py
[https://github.com/python/cpython/tree/3.15/Lib/selectors.py]

Introdução

This module allows high-level and efficient I/O multiplexing, built upon the `select` module primitives. Users are encouraged to use this module instead, unless they want precise control over the OS-level primitives used.

It defines a `BaseSelector` abstract base class, along with several concrete implementations (`KqueueSelector`, `EpollSelector`…), that can be used to wait for I/O readiness notification on multiple file objects. In the following, "file object" refers to any object with a `fileno()` method, or a raw file descriptor. See file object.

`DefaultSelector` is an alias to the most efficient implementation available on the current platform: this should be the default choice for most users.

Nota

The type of file objects supported depends on the platform: on Windows, sockets are supported, but not pipes, whereas on Unix, both are supported (some other types may be supported as well, such as fifos or special file devices).

Ver também

select

Low-level I/O multiplexing module.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Classes

Classes hierarchy:

```
BaseSelector
+-- SelectSelector
+-- PollSelector
+-- EpollSelector
+-- DevpollSelector
+-- KqueueSelector
```

In the following, *events* is a bitwise mask indicating which I/O events should be waited for on a given file object. It can be a combination of the module's constants below:

Constante	Significado
selectors.EVENT_READ	Disponível para leitura
selectors.EVENT_WRITE	Disponível para escrita

class selectors.SelectorKey

A **SelectorKey** is a **namedtuple** used to associate a file object to its underlying file descriptor, selected event mask and attached data. It is returned by several **BaseSelector** methods.

fileobj

File object registered.

fd

O descritor de arquivo subjacente.

events

Events that must be waited for on this file object.

data

Optional opaque data associated to this file object: for example, this could be used to store a per-client session ID.

class selectors.BaseSelector

A **`BaseSelector`** is used to wait for I/O event readiness on multiple file objects. It supports file stream registration, unregistration, and a method to wait for I/O events on those streams, with an optional timeout. It's an abstract base class, so cannot be instantiated. Use **`DefaultSelector`** instead, or one of **`SelectSelector`**, **`KqueueSelector`** etc. if you want to specifically use an implementation, and your platform supports it. **`BaseSelector`** and its concrete implementations support the context manager protocol.

abstractmethod register(*fileobj*, *events*, *data=None*)

Registra um objeto arquivo para seleção, monitorando-o para eventos de I/O

fileobj is the file object to monitor. It may either be an integer file descriptor or an object with a `fileno()` method. *events* is a bitwise mask of events to monitor. *data* is an opaque object.

This returns a new **`SelectorKey`** instance, or raises a **`ValueError`** in case of invalid event mask or file descriptor, or **`KeyError`** if the file object is already registered.

abstractmethod unregister(*fileobj*)

Unregister a file object from selection, removing it from monitoring. A file object shall be unregistered prior to being closed.

fileobj must be a file object previously registered.

This returns the associated **`SelectorKey`** instance, or raises a **`KeyError`** if *fileobj* is not registered. It will raise **`ValueError`** if *fileobj* is invalid (e.g. it has no `fileno()` method or its `fileno()` method has an invalid return value).

modify(*fileobj*, *events*, *data=None*)

Change a registered file object's monitored events or attached data.

This is equivalent to `BaseSelector.unregister(fileobj)` followed by `BaseSelector.register(fileobj, events, data)`, except that it can be implemented more efficiently.

This returns a new **`SelectorKey`** instance, or raises a **`ValueError`** in case of invalid event mask or file descriptor, or **`KeyError`** if the file object is not registered.

abstractmethod select(*timeout=None*)

Wait until some registered file objects become ready, or the timeout expires.

If `timeout > 0`, this specifies the maximum wait time, in seconds. If `timeout <= 0`, the call won't block, and will report the currently ready file objects. If *timeout* is `None`, the call will block until a monitored file object becomes ready.

This returns a list of `(key, events)` tuples, one for each ready file object.

key is the **`SelectorKey`** instance corresponding to a ready file object. *events* is a bitmask of events ready on this file object.

Nota

This method can return before any file object becomes ready or the timeout has elapsed if the current process receives a signal: in this case, an empty list will be returned.

Alterado na versão 3.5: The selector is now retried with a recomputed timeout when interrupted by a signal if the signal handler did not raise an exception (see **PEP 475** [https://peps.python.org/pep-0475/] for the rationale), instead of returning an empty list of events before the timeout.

close()

Close the selector.

This must be called to make sure that any underlying resource is freed. The selector shall not be used once it has been closed.

get_key(*fileobj*)

Return the key associated with a registered file object.

This returns the `SelectorKey` instance associated to this file object, or raises `KeyError` if the file object is not registered.

abstractmethod get_map()

Return a mapping of file objects to selector keys.

This returns a `Mapping` instance mapping registered file objects to their associated `SelectorKey` instance.

class selectors.DefaultSelector

The default selector class, using the most efficient implementation available on the current platform. This should be the default choice for most users.

class selectors.SelectSelector

`select.select()`-based selector.

class selectors.PollSelector

`select.poll()`-based selector.

class selectors.EpollSelector

`select.epoll()`-based selector.

fileno()

This returns the file descriptor used by the underlying `select.epoll()` object.

class selectors.DevpollSelector

`select.devpoll()`-based selector.

fileno()

This returns the file descriptor used by the underlying `select.devpoll()` object.

Adicionado na versão 3.5.

class selectors.KqueueSelector

`select.kqueue()`-based selector.

fileno()

This returns the file descriptor used by the underlying `select.kqueue()` object.

Exemplos

Here is a simple echo server implementation:

```
import selectors
import socket

sel = selectors.DefaultSelector()

def accept(sock, mask):
    conn, addr = sock.accept()  # Should be ready
```

```
    print('accepted', conn, 'from', addr)
    conn.setblocking(False)
    sel.register(conn, selectors.EVENT_READ, read)

def read(conn, mask):
    data = conn.recv(1000)  # Should be ready
    if data:
        print('echoing', repr(data), 'to', conn)
        conn.send(data)  # Hope it won't block
    else:
        print('closing', conn)
        sel.unregister(conn)
        conn.close()

sock = socket.socket()
sock.bind(('localhost', 1234))
sock.listen(100)
sock.setblocking(False)
sel.register(sock, selectors.EVENT_READ, accept)

while True:
    events = sel.select()
    for key, mask in events:
        callback = key.data
        callback(key.fileobj, mask)
```

`signal` — Define manipuladores para eventos assíncronos

Código-fonte: Lib/signal.py [https://github.com/python/cpython/tree/3.15/Lib/signal.py]

Este módulo fornece mecanismos para usar manipuladores de sinal em Python.

Regras gerais

A função `signal.signal()` permite definir manipuladores personalizados a serem executados quando um sinal é recebido. Um pequeno número de manipuladores padrão são instalados: `SIGPIPE` é ignorado (então erros de gravação em encadeamentos e sockets podem ser relatados como exceções comuns do Python) e `SIGINT` é traduzido em uma exceção `KeyboardInterrupt` se o processo pai não a tiver alterado.

Um manipulador para um sinal específico, uma vez definido, permanece instalado até ser explicitamente redefinido (o Python emula a interface de estilo BSD independentemente da implementação subjacente), com exceção do manipulador para `SIGCHLD`, que segue a implementação subjacente.

Em plataformas WebAssembly, os sinais são emulados e, portanto, se comportam de forma diferente. Várias funções e sinais não estão disponíveis nessas plataformas.

Execução de manipuladores de sinais Python

A Python signal handler does not get executed inside the low-level (C) signal handler. Instead, the low-level signal handler sets a flag which tells the virtual machine to execute the corresponding Python signal handler at a

later point (for example, at the next bytecode instruction). This has consequences:

- Faz pouco sentido capturar erros síncronos como `SIGFPE` ou `SIGSEGV` que são causados por uma operação inválida no código C. O Python retornará do manipulador de sinais para o código C, o que provavelmente levantará o mesmo sinal novamente, fazendo com que o Python aparentemente trave. Do Python 3.3 em diante, você pode usar o módulo `faulthandler` para relatar erros síncronos.
- Um cálculo de longa duração implementado puramente em C (como correspondência de expressão regular em um grande corpo de texto) pode ser executado ininterruptamente por um período de tempo arbitrário, independentemente de quaisquer sinais recebidos. Os manipuladores de sinal do Python serão chamados quando o cálculo terminar.
- Se o manipulador levantar uma exceção, ela será levantada "do nada" na thread principal. Veja a nota abaixo para uma discussão.

Sinais e threads

Os manipuladores de sinais Python são sempre executados na thread principal do Python do interpretador principal, mesmo se o sinal foi recebido em outra thread. Isso significa que os sinais não podem ser usados como um meio de comunicação entre threads. Você pode usar as primitivas de sincronização do módulo `threading` em vez disso.

Além disso, somente a thread principal do interpretador principal tem permissão para definir um novo manipulador de sinal.

Aviso

Synchronization primitives such as `threading.Lock` should not be used within signal handlers. Doing so can lead to unexpected deadlocks.

Conteúdo do módulo

Alterado na versão 3.5: As constantes relacionadas a sinal (SIG*), manipulador (`SIG_DFL`, `SIG_IGN`) e sigmask (`SIG_BLOCK`, `SIG_UNBLOCK`, `SIG_SETMASK`) listadas abaixo foram transformadas em `enums` (`Signals`, `Handlers` e `Sigmasks` respectivamente). As funções `getsignal()`, `pthread_sigmask()`, `sigpending()` e `sigwait()` retornam **`enums`** legíveis por humanos como objetos **`Signals`**.

O módulo de sinal define três enumerações:

class signal.Signals

Coleção de `enum.IntEnum` de constantes SIG* e constantes CTRL_*.

Adicionado na versão 3.5.

class signal.Handlers

`enum.IntEnum` collection of the constants `SIG_DFL` and `SIG_IGN`.

Adicionado na versão 3.5.

class signal.Sigmasks

`enum.IntEnum` collection of the constants `SIG_BLOCK`, `SIG_UNBLOCK` and `SIG_SETMASK`.

Disponibilidade: Unix.

Veja a página man *sigprocmask(2)* *[https://manpages.debian.org/sigprocmask(2)]* e *pthread_sigmask(3)* *[https://manpages.debian.org/pthread_sigmask(3)]* para mais informações.

Adicionado na versão 3.5.

The variables defined in the **`signal`** module are:

signal.SIG_DFL

Esta é uma das duas opções de manipulação de sinal padrão; ela simplesmente executará a função padrão para o sinal. Por exemplo, na maioria dos sistemas, a ação padrão para `SIGQUIT` é despejar o núcleo e sair, enquanto a ação padrão para `SIGCHLD` é simplesmente ignorá-lo.

signal.SIG_IGN

Este é outro manipulador de sinal padrão, que simplesmente ignorará o sinal fornecido.

signal.SIGABRT

Sinal de abortar de *abort(3)* *[https://manpages.debian.org/abort(3)]*.

signal.SIGALRM

Sinal do temporizador de *alarm(2)* *[https://manpages.debian.org/alarm(2)]*.

Disponibilidade: Unix.

signal.SIGBREAK

Interrupção do teclado (CTRL + BREAK).

Disponibilidade: Windows.

signal.SIGBUS

Erro de barramento (acesso incorreto à memória).

Disponibilidade: Unix.

signal.SIGCHLD

Processo filho interrompido ou encerrado.

Disponibilidade: Unix.

signal.SIGCLD

Apelido para **`SIGCHLD`**.

Disponibilidade: not macOS.

signal.SIGCONT

Continua o processo se ele estiver parado no momento

Disponibilidade: Unix.

signal.SIGFPE

Exceção de ponto flutuante. Por exemplo, divisão por zero.

Ver também

`ZeroDivisionError` é levatanda quando o segundo argumento de uma operação de divisão ou módulo é zero.

signal.SIGHUP

Travamento detectado no terminal de controle ou morte do processo de controle.

Disponibilidade: Unix.

signal.SIGILL

Instrução ilegal.

signal.SIGINT

Interrupção do teclado (CTRL + C).

A ação padrão é levantar `KeyboardInterrupt`.

signal.SIGKILL

Sinal para encerrar forçadamente.

Ele não pode ser capturado, bloqueado ou ignorado.

Disponibilidade: Unix.

signal.SIGPIPE

Encadeamento quebrado: escreve no encadeamento sem leitores.

A ação padrão é ignorar o sinal.

Disponibilidade: Unix.

signal.SIGPROF

Temporizador de perfilamento expirado.

Disponibilidade: Unix.

signal.SIGQUIT
Sinal de saída do terminal.

Disponibilidade: Unix.

signal.SIGSEGV
Falha de segmentação: referência de memória inválida.

signal.SIGSTOP
Para a execução (não pode ser capturado ou ignorado).

Disponibilidade: Unix.

signal.SIGSTKFLT
Falha de pilha no coprocessador. O kernel Linux não levanta esse sinal: ele só pode ser emitido no espaço do usuário.

Disponibilidade: Linux.

Em arquiteturas onde o sinal está disponível. Veja a página man *signal(7)* *[https://manpages.debian.org/signal(7)]* para mais informações.

Adicionado na versão 3.11.

signal.SIGTERM
Sinal de encerramento.

signal.SIGUSR1
Sinal definido pelo usuário 1.

Disponibilidade: Unix.

signal.SIGUSR2
Sinal definido pelo usuário 2.

Disponibilidade: Unix.

signal.SIGVTALRM

Temporizador virtual expirou.

Disponibilidade: Unix.

signal.SIGWINCH

Sinal de redimensionamento do Windows.

Disponibilidade: Unix.

signal.SIGXCPU

Limite de tempo da CPU excedido.

Disponibilidade: Unix.

SIG*

Todos os números de sinal são definidos simbolicamente. Por exemplo, o sinal de desligamento é definido como **`signal.SIGHUP`**; os nomes das variáveis são idênticos aos usados em programas C, como encontrados em `<signal.h>`. A página man do Unix para '`signal`' lista os sinais existentes (em alguns sistemas, é *signal(2)* *[https://manpages.debian.org/signal(2)]*, em outros, a lista está em *signal(7)* *[https://manpages.debian.org/signal(7)]*). Observe que nem todos os sistemas definem o mesmo conjunto de nomes de sinais; apenas os nomes definidos pelo sistema são definidos por este módulo.

signal.CTRL_C_EVENT

O sinal correspondente ao evento de pressionamento de tecla `Ctrl+C`. Este sinal só pode ser usado com **`os.kill()`**.

Disponibilidade: Windows.

Adicionado na versão 3.2.

signal.CTRL_BREAK_EVENT

O sinal correspondente ao evento de pressionamento de tecla `Ctrl+Break`. Este sinal só pode ser usado com **`os.kill()`**.

Disponibilidade: Windows.

Adicionado na versão 3.2.

signal.NSIG

Um a mais que o número do sinal mais alto. Use `valid_signals()` para obter números de sinais válidos.

signal.ITIMER_REAL

Diminui o intervalo do temporizador em tempo real e entrega `SIGALRM` ao expirar.

signal.ITIMER_VIRTUAL

Diminui o intervalo do temporizador somente quando o processo está em execução e entrega SIGVTALRM após a expiração.

signal.ITIMER_PROF

Diminui o temporizador de intervalo tanto quando o processo é executado quanto quando o sistema está executando em nome do processo. Juntamente com o ITIMER_VIRTUAL, este temporizador é geralmente usado para criar um perfil do tempo gasto pelo aplicativo nos espaços do usuário e do kernel. O SIGPROF é fornecido após a expiração.

signal.SIG_BLOCK

Um possível valor para o parâmetro *how* para `pthread_sigmask()` indicando que os sinais devem ser bloqueados.

Adicionado na versão 3.3.

signal.SIG_UNBLOCK

Um possível valor para o parâmetro *how* para `pthread_sigmask()` indicando que os sinais devem ser desbloqueados.

Adicionado na versão 3.3.

signal.SIG_SETMASK

Um possível valor para o parâmetro *how* para `pthread_sigmask()` indicando que a máscara de sinal deve ser substituída.

Adicionado na versão 3.3.

The `signal` module defines one exception:

exception signal.ItimerError

Levantada para sinalizar um erro da implementação subjacente de `setitimer()` ou `getitimer()`. Espere este erro se um temporizador de intervalo inválido ou um tempo negativo for passado para `setitimer()`. Este erro é um subtipo de `OSError`.

Adicionado na versão 3.3: Este erro costumava ser um subtipo de `IOError`, que agora é um apelido de `OSError`.

The `signal` module defines the following functions:

signal.alarm(*time*)

Se *time* for diferente de zero, esta função solicita que um sinal `SIGALRM` seja enviado ao processo em *time* segundos. Qualquer alarme previamente agendado será cancelado (apenas um alarme pode ser agendado por vez). O valor retornado será o número de segundos antes de qualquer alarme previamente configurado ter sido emitido. Se *time* for zero, nenhum alarme será agendado e qualquer alarme agendado será cancelado. Se o valor retornado for zero, nenhum alarme será agendado no momento.

Disponibilidade: Unix.

Veja a página man *alarm(2)* *[https://manpages.debian.org/alarm(2)]* para mais informações.

signal.getsignal(*signalnum*)

Retorna o manipulador de sinal atual para o sinal *signalnum*. O valor retornado pode ser um objeto Python invocável ou um dos valores especiais `signal.SIG_IGN`, `signal.SIG_DFL` ou `None`. Aqui, `signal.SIG_IGN` significa que o sinal foi ignorado anteriormente,

signal.SIG_DFL significa que a maneira padrão de manipular o sinal estava em uso anteriormente e `None` significa que o manipulador de sinal anterior não foi instalado a partir do Python.

signal.strsignal(*signalnum*)

Retorna a descrição do sinal *signalnum*, como “Interrupt” para **SIGINT**. Retorna **None** se *signalnum* não tiver descrição. Levanta **ValueError** se *signalnum* for inválido.

Adicionado na versão 3.8.

signal.valid_signals()

Retorna o conjunto de números de sinais válidos nesta plataforma. Pode ser menor que `range(1, NSIG)` se alguns sinais forem reservados pelo sistema para uso interno.

Adicionado na versão 3.8.

signal.pause()

Faz o processo hibernar até que um sinal seja recebido; o manipulador apropriado será então chamado. Não retorna nada.

Disponibilidade: Unix.

Veja a página man *signal(2)* *[https://manpages.debian.org/signal(2)]* para mais informações.

veja também **sigwait()**, **sigwaitinfo()**, **sigtimedwait()** e **sigpending()**.

signal.raise_signal(*signum*)

Envia um sinal para o processo de chamada. Não retorna nada.

Adicionado na versão 3.8.

signal.pidfd_send_signal(*pidfd*, *sig*, *siginfo=None*, *flags=0*)

Envia o sinal *sig* para o processo referenciado pelo descritor de arquivo *pidfd*. Atualmente, o Python não suporta o parâmetro *siginfo*; ele deve ser `None`. O argumento *flags* é fornecido para futuras extensões; nenhum valor de sinalizador está definido no momento.

Veja a página man *pidfd_send_signal(2)* *[https://manpages.debian.org/pidfd_send_signal(2)]* para mais informações.

Disponibilidade: Linux >= 5.1, Android >= **`build-time`** API level 31

Adicionado na versão 3.9.

signal.pthread_kill(*thread_id*, *signalnum*)

Envia o sinal *signalnum* para a thread *thread_id*, outra thread no mesmo processo que a chamadora. A thread alvo pode estar executando qualquer código (Python ou não). No entanto, se a thread alvo estiver executando o interpretador Python, os manipuladores de sinal Python serão executados pela thread principal do interpretador principal. Portanto, o único objetivo de enviar um sinal para uma thread Python específica seria forçar uma chamada de sistema em execução a falhar com **`InterruptedError`**.

Utilize a **`threading.get_ident()`** ou o atributo **`ident`** dos objetos **`threading.Thread`** para obter um valor adequado para *thread_id*.

Se *signalnum* for 0, nenhum sinal será enviado, mas a verificação de erros ainda será realizada; isso pode ser usado para verificar se a thread de destino ainda está em execução.

Levanta um evento de auditoria `signal.pthread_kill` com os argumentos `thread_id`, `signalnum`.

Disponibilidade: Unix.

Veja a página man *pthread_kill(3)* *[https://manpages.debian.org/pthread_kill(3)]* para mais informações.

Veja também **`os.kill()`**.

Adicionado na versão 3.3.

signal.pthread_sigmask(*how*, *mask*)

Busca e/ou altera a máscara de sinal da thread chamadora. A máscara de sinal é o conjunto de sinais cuja entrega está atualmente bloqueada para o chamador. Retorna a máscara de sinal antiga como um conjunto de sinais.

O comportamento da chamada depende do valor de *how*, como segue.

- **`SIG_BLOCK`**: O conjunto de sinais bloqueados é a união do conjunto atual e do argumento *mask*.
- **`SIG_UNBLOCK`**: Os sinais em *mask* são removidos do conjunto atual de sinais bloqueados. É permitido tentar desbloquear um sinal que não esteja bloqueado.
- **`SIG_SETMASK`**: O conjunto de sinais bloqueados é definido como o argumento *mask*.

mask é um conjunto de números de sinais (ex.: {**`signal.SIGINT`**, **`signal.SIGTERM`**}). Use **`valid_signals()`** para uma máscara completa, incluindo todos os sinais.

Por exemplo, `signal.pthread_sigmask(signal.SIG_BLOCK, [])` lê a máscara de sinal da thread de chamada.

`SIGKILL` e **`SIGSTOP`** não podem ser bloqueados.

Disponibilidade: Unix.

Veja a página man *sigprocmask(2)* *[https://manpages.debian.org/sigprocmask(2)]* e *pthread_sigmask(3)* *[https://manpages.debian.org/pthread_sigmask(3)]* para mais informações.

Veja também **`pause()`**, **`sigpending()`** e **`sigwait()`**.

Adicionado na versão 3.3.

signal.setitimer(*which*, *seconds*, *interval=0*)

Sets given interval timer (one of `signal.ITIMER_REAL`, `signal.ITIMER_VIRTUAL` or `signal.ITIMER_PROF`) specified by *which* to fire after *seconds* (rounded up to microseconds, different from `alarm()`) and after that every *interval* seconds (if *interval* is non-zero). The interval timer specified by *which* can be cleared by setting *seconds* to zero.

Quando um temporizador de intervalo dispara, um sinal é enviado ao processo. O sinal enviado depende do temporizador utilizado; `signal.ITIMER_REAL` enviará `SIGALRM`, `signal.ITIMER_VIRTUAL` enviará `SIGVTALRM` e `signal.ITIMER_PROF` enviará `SIGPROF`.

The old values are returned as a two-tuple of floats: (`delay`, `interval`).

Tentar passar um intervalo de tempo inválido causará uma `ItimerError`.

Disponibilidade: Unix.

Alterado na versão 3.15: Accepts any real numbers as *seconds* and *interval*, not only integers or floats.

signal.getitimer(*which*)

Retorna o valor atual de um intervalo de tempo especificado por *which*.

Disponibilidade: Unix.

signal.set_wakeup_fd(*fd*, *, *warn_on_full_buffer=True*)

Define o descritor de arquivo de ativação como *fd*. Quando um sinal para o qual seu programa registrou um manipulador de sinais é recebido, o número do sinal é escrito como um único byte no fd. Se você não registrou um manipulador de sinais para os sinais de seu interesse, nada será escrito no fd de ativação. Isso pode ser usado por uma biblioteca para ativar uma chamada de poll ou seleção, permitindo que o sinal seja totalmente processado.

O antigo fd de ativação é retornado (ou -1 se a ativação do descritor de arquivo não estava habilitada). Se *fd* for -1, a ativação do descritor de

arquivo está desabilitada. Se não for -1, *fd* deve ser não bloqueante. Cabe à biblioteca remover quaisquer bytes de *fd* antes de chamar poll ou select novamente.

Quando threads estão habilitadas, esta função só pode ser chamada da thread principal do interpretador principal; tentar chamá-la de outras threads levantará uma exceção **`ValueError`**.

Há duas maneiras comuns de usar esta função. Em ambas as abordagens, você usa o fd para despertar quando um sinal chega, mas elas diferem na forma como determinam *which* sinal ou sinais chegaram.

Na primeira abordagem, lemos os dados do buffer do fd, e os valores dos bytes fornecem os números dos sinais. Isso é simples, mas em casos raros pode apresentar um problema: geralmente, o fd terá um espaço de buffer limitado e, se muitos sinais chegarem muito rápido, o buffer pode ficar cheio e alguns sinais podem ser perdidos. Se você usar essa abordagem, defina `warn_on_full_buffer=True`, o que pelo menos fará com que um aviso seja impresso no stderr quando os sinais forem perdidos.

Na segunda abordagem, usamos o fd de ativação *apenas* para ativações e ignoramos os valores de bytes reais. Nesse caso, tudo o que importa é se o buffer do fd está vazio ou não; um buffer cheio não indica nenhum problema. Se você usar essa abordagem, defina `warn_on_full_buffer=False` para que seus usuários não sejam confundidos por mensagens de aviso falsas.

Alterado na versão 3.5: No Windows, a função agora também suporta manipuladores de socket.

Alterado na versão 3.7: Adiciona o parâmetro `warn_on_full_buffer`.

signal.siginterrupt(*signalnum*, *flag*)

Altera o comportamento de reinicialização de chamadas de sistema: se *flag* for **`False`**, as chamadas de sistema serão reiniciadas quando

interrompidas pelo sinal *signalnum*; caso contrário, as chamadas de sistema serão interrompidas. Não retorna nada.

Disponibilidade: Unix.

Veja a página man *siginterrupt(3)* *[https://manpages.debian.org/siginterrupt(3)]* para mais informações.

Observe que a instalação de um manipulador de sinal com **`signal()`** redefinirá o comportamento de reinicialização para interrompível ao chamar implicitamente **`siginterrupt()`** com *flag* contendo valor verdadeiro para o sinal fornecido.

signal.signal(*signalnum*, *handler*)

Define o manipulador do sinal *signalnum* para a função *handler*. *handler* pode ser um objeto Python invocável que recebe dois argumentos (veja abaixo), ou um dos valores especiais **`signal.SIG_IGN`** ou **`signal.SIG_DFL`**. O manipulador de sinal anterior será retornado (veja a descrição de **`getsignal()`** acima). (Consulte a página man do Unix *signal(2)* *[https://manpages.debian.org/signal(2)]* para mais informações.)

Quando threads estão habilitadas, esta função só pode ser chamada da thread principal do interpretador principal; tentar chamá-la de outras threads levantará uma exceção **`ValueError`**.

O *handler* é chamado com dois argumentos: o número do sinal e o quadro de pilha atual (`None` ou um objeto quadro; para uma descrição dos objetos quadro, consulte a descrição na hierarquia de tipo ou consulte as descrições de atributo no módulo **`inspect`**).

No Windows, **`signal()`** só pode ser chamado com **`SIGABRT`**, **`SIGFPE`**, **`SIGILL`**, **`SIGINT`**, **`SIGSEGV`**, **`SIGTERM`** ou **`SIGBREAK`**. Uma exceção **`ValueError`** será levantada em qualquer outro caso. Observe que nem todos os sistemas definem o mesmo conjunto de nomes de sinais; uma exceção **`AttributeError`** será levantada se um nome de sinal não for definido como constante de nível de módulo `SIG*`.

signal.sigpending()

Examine o conjunto de sinais pendentes para entrega ao thread de chamada (ou seja, os sinais que foram gerados enquanto bloqueados). Retorne o conjunto de sinais pendentes.

Disponibilidade: Unix.

Veja a página man *sigpending(2)* *[https://manpages.debian.org/sigpending(2)]* para mais informações.

Veja também `pause()`, `pthread_sigmask()` e `sigwait()`.

Adicionado na versão 3.3.

signal.sigwait(*sigset*)

Suspende a execução do thread de chamada até a entrega de um dos sinais especificados no conjunto de sinais *sigset*. A função aceita o sinal (remove-o da lista de sinais pendentes) e retorna o número do sinal.

Disponibilidade: Unix.

Veja a página man *sigwait(3)* *[https://manpages.debian.org/sigwait(3)]* para mais informações.

Veja também `pause()`, `pthread_sigmask()`, `sigpending()`, `sigwaitinfo()` e `sigtimedwait()`.

Adicionado na versão 3.3.

signal.sigwaitinfo(*sigset*)

Suspende a execução da thread chamadora até a entrega de um dos sinais especificados no conjunto de sinais *sigset*. A função aceita o sinal e o remove da lista de sinais pendentes. Se um dos sinais em *sigset* já estiver pendente para a thread chamadora, a função retornará imediatamente com informações sobre esse sinal. O manipulador de sinais não é chamado para o sinal entregue. A função levanta `InterruptedError` se for interrompida por um sinal que não esteja em *sigset*.

O valor de retorno é um objeto que representa os dados contidos na estrutura de `siginfo_t`, a saber: `si_signo`, `si_code`, `si_errno`, `si_pid`, `si_uid`, `si_status`, `si_band`.

Disponibilidade: Unix.

Veja a página man *sigwaitinfo(2)* *[https://manpages.debian.org/sigwaitinfo(2)]* para mais informações.

Veja também **`pause()`**, **`sigwait()`** e **`sigtimedwait()`**.

Adicionado na versão 3.3.

Alterado na versão 3.5: A função agora é tentada novamente se interrompida por um sinal que não esteja em *sigset* e o manipulador de sinal não levanta uma exceção (veja **PEP 475** [https://peps.python.org/pep-0475/] para a justificativa).

signal.sigtimedwait(*sigset*, *timeout*)

Semelhante a **`sigwaitinfo()`**, mas recebe um argumento *timeout* adicional que especifica um tempo limite. Se *timeout* for especificado como `0`, uma consulta será realizada. Retorna **`None`** se ocorrer um tempo limite.

Disponibilidade: Unix.

Veja a página man *sigtimedwait(2)* *[https://manpages.debian.org/sigtimedwait(2)]* para mais informações.

Veja também **`pause()`**, **`sigwait()`** e **`sigwaitinfo()`**.

Adicionado na versão 3.3.

Alterado na versão 3.5: A função agora é tentada novamente com o *timeout* recalculado se interrompida por um sinal que não esteja em *sigset* e o manipulador de sinal não levanta uma exceção (veja **PEP 475** [https://peps.python.org/pep-0475/] para a justificativa).

Alterado na versão 3.15: Accepts any real number as *timeout*, not only integer or float.

Exemplos

Aqui está um programa de exemplo mínimo. Ele usa a função **`alarm()`** para limitar o tempo gasto esperando para abrir um arquivo; isso é útil se o arquivo for para um dispositivo serial que pode não estar ligado, o que normalmente faria com que o **`os.open()`** travasse indefinidamente. A solução é definir um alarme de 5 segundos antes de abrir o arquivo; se a operação demorar muito, o sinal de alarme será enviado e o manipulador levantará uma exceção.

```
import signal, os

def handler(signum, frame):
    signame = signal.Signals(signum).name
    print(f'Signal handler called with signal {signame} ({signum
    raise OSError("Couldn't open device!")

# Define o manipulador de sinal e um alarme de 5 segundos
signal.signal(signal.SIGALRM, handler)
signal.alarm(5)

# Este open() pode travar indefinidamente
fd = os.open('/dev/ttyS0', os.O_RDWR)

signal.alarm(0)          # Desabilita o alarme
```

Nota sobre SIGPIPE

Canalizar a saída do seu programa para ferramentas como *head(1)* *[https://manpages.debian.org/head(1)]* fará com que um sinal **`SIGPIPE`** seja enviado ao seu processo quando o receptor da saída padrão fechar antes do tempo. Isso resulta em uma exceção como `BrokenPipeError: [Errno 32] Broken pipe`. Para lidar com esse caso, envolva seu ponto de entrada para capturar essa exceção da seguinte maneira:

```
import os
import sys

def main():
    try:
        # simula uma saída grande (seu código substitui este loc
        for x in range(10000):
            print("y")
        # limpa a saída aqui para forçar o acionamento do SIGPIF
        # enquanto estiver dentro deste bloco try.
        sys.stdout.flush()
    except BrokenPipeError:
        # Python libera fluxos padrão na saída; redireciona a sa
        # para devnull para evitar outro BrokenPipeError no desl
        devnull = os.open(os.devnull, os.O_WRONLY)
        os.dup2(devnull, sys.stdout.fileno())
        sys.exit(1)  # Python exits with error code 1 on EPIPE

if __name__ == '__main__':
    main()
```

Não defina a disposição de **`SIGPIPE`** como **`SIG_DFL`** para evitar **`BrokenPipeError`**. Isso faria com que seu programa encerrasse inesperadamente sempre que qualquer conexão de soquete fosse interrompida enquanto o programa ainda estivesse escrevendo nele.

Nota sobre manipuladores de sinais e exceções

Se um manipulador de sinais levantar uma exceção, a exceção será propagada para a thread principal e poderá ser levantada após qualquer instrução bytecode. Mais notavelmente, uma **`KeyboardInterrupt`** pode aparecer a qualquer momento durante a execução. A maioria dos códigos Python, incluindo a biblioteca padrão, não pode ser robusta contra isso, e, portanto, uma **`KeyboardInterrupt`** (ou qualquer outra exceção resultante de um manipulador de sinais) pode, em raras ocasiões, colocar o programa em um estado inesperado.

Para ilustrar esse problema, considere o seguinte código:

```
class SpamContext:
    def __init__(self):
        self.lock = threading.Lock()

    def __enter__(self):
        # Se ocorrer KeyboardInterrupt aqui, está tudo bem
        self.lock.acquire()
        # Se KeyboardInterrupt ocorrer aqui, __exit__ não será c
        ...
        # KeyboardInterrupt pode ocorrer logo antes do retorno c

    def __exit__(self, exc_type, exc_val, exc_tb):
        ...
        self.lock.release()
```

Para muitos programas, especialmente aqueles que simplesmente desejam encerrar em **KeyboardInterrupt**, isso não é um problema, mas aplicações complexos ou que exigem alta confiabilidade devem evitar levantar exceções de manipuladores de sinal. Eles também devem evitar capturar **KeyboardInterrupt** como forma de encerrar o programa sem problemas. Em vez disso, devem instalar seu próprio manipulador **SIGINT**. Abaixo está um exemplo de um servidor HTTP que evita **KeyboardInterrupt**:

```
import signal
import socket
from selectors import DefaultSelector, EVENT_READ
from http.server import HTTPServer, SimpleHTTPRequestHandler

interrupt_read, interrupt_write = socket.socketpair()

def handler(signum, frame):
    print('Signal handler called with signal', signum)
    interrupt_write.send(b'\0')
signal.signal(signal.SIGINT, handler)

def serve_forever(httpd):
    sel = DefaultSelector()
    sel.register(interrupt_read, EVENT_READ)
    sel.register(httpd, EVENT_READ)

    while True:
```

```
        for key, _ in sel.select():
            if key.fileobj == interrupt_read:
                interrupt_read.recv(1)
                return
            if key.fileobj == httpd:
                httpd.handle_request()

print("Serving on port 8000")
httpd = HTTPServer(('', 8000), SimpleHTTPRequestHandler)
serve_forever(httpd)
print("Shutdown...")
```

`mmap` — Suporte a arquivos mapeados na memória

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Objetos arquivo mapeados na memória se comportam como **`bytearray`** e como objetos arquivo. Você pode usar objetos mmap na maioria dos lugares onde **`bytearray`** é esperado; por exemplo, você pode usar o módulo **`re`** para pesquisar um arquivo mapeado na memória. Você também pode alterar um único byte executando `obj[index] = 97` ou alterar uma subsequência atribuindo a uma fatia: `obj[i1:i2] = b'...'`. Você também pode ler e gravar dados começando na posição atual do arquivo e **`seek()`** através do arquivo para diferentes posições.

Um arquivo mapeado na memória é criado pelo construtor **`mmap`**, que é diferente no Unix e no Windows. Nos dois casos, você deve fornecer um descritor de arquivo para um arquivo aberto para atualização. Se você deseja mapear um objeto arquivo Python existente, use o método **`fileno()`** para obter o valor correto para o parâmetro *fileno*. Caso contrário, você pode abrir o arquivo usando a função **`os.open()`**, que retorna um descritor de arquivo diretamente (o arquivo ainda precisa ser fechado quando terminar).

Nota

Se você deseja criar um mapeamento de memória para um arquivo gravável e armazenado em buffer, deve usar **`flush()`** no arquivo primeiro. Isso é necessário para garantir que as modificações locais nos buffers estejam realmente disponíveis para o mapeamento.

Para as versões Unix e Windows do construtor, *access* pode ser especificado como um parâmetro opcional de palavra-chave. *access* aceita um dos quatro valores: **`ACCESS_READ`**, **`ACCESS_WRITE`** ou **`ACCESS_COPY`** para especificar memória somente leitura, gravação ou cópia na gravação, respectivamente **`ACCESS_DEFAULT`** para adiar para *prot*. *access* pode ser usado no Unix e no Windows. Se *access* não for especificado, o mmap do Windows retornará um mapeamento de gravação. Os valores iniciais da memória para todos os três tipos de acesso são obtidos do arquivo especificado. A atribuição a um mapa de memória **`ACCESS_READ`** gera uma exceção **`TypeError`**. A atribuição a um mapa de memória **`ACCESS_WRITE`** afeta a memória e o arquivo subjacente. A atribuição a um mapa de memória **`ACCESS_COPY`** afeta a memória, mas não atualiza o arquivo subjacente.

Alterado na versão 3.7: Adicionada a constante **`ACCESS_DEFAULT`**.

Para mapear a memória anônima, -1 deve ser passado como o fileno junto com o comprimento.

class mmap.mmap(*fileno*, *length*, *tagname=None*, *access=ACCESS_DEFAULT*, *offset=0*, ***, *trackfd=True*)

(Windows version) Maps *length* bytes from the file specified by the file descriptor *fileno*, and creates a mmap object. If *length* is larger than the current size of the file, the file is extended to contain *length* bytes. If *length* is `0`, the maximum length of the map is the current size of the file, except that if the file is empty Windows raises an exception (you cannot create an empty mapping on Windows).

tagname, se especificado e não `None`, é uma string que fornece um nome de tag para o mapeamento. O Windows permite que você tenha muitos mapeamentos diferentes no mesmo arquivo. Se você especificar o nome de uma marca existente, essa marca será aberta; caso contrário, uma nova marca com esse nome será criada. Se este parâmetro for omitido ou `None`, o mapeamento será criado sem um nome. Evitar o uso do parâmetro *tagname* ajudará a manter seu código portátil entre o Unix e o Windows.

offset pode ser especificado como um deslocamento inteiro não negativo. As referências mmap serão relativas ao deslocamento do início do arquivo. *offset* tem como padrão 0. *offset* deve ser um múltiplo de **ALLOCATIONGRANULARITY**.

If *trackfd* is `False`, the file handle corresponding to *fileno* will not be duplicated, and the resulting **mmap** object will not be associated with the map's underlying file. This means that the **size()** and **resize()** methods will fail. This mode is useful to limit the number of open file handles. The original file can be renamed (but not deleted) after closing *fileno*.

Alterado na versão 3.15: The *trackfd* parameter was added.

Levanta um evento de auditoria `mmap.__new__` com os argumentos `fileno`, `length`, `access`, `offset`.

class mmap.mmap(*fileno*, *length*, *flags=MAP_SHARED*, *prot=PROT_WRITE | PROT_READ*, *access=ACCESS_DEFAULT*, *offset=0*, *, *trackfd=True*)

(Versão Unix) Mapeia *length* bytes do arquivo especificado pelo descritor de arquivo *fileno* e retorna um objeto mmap. Se *length* for `0`, o comprimento máximo do mapa será o tamanho atual do arquivo quando **mmap** for chamado.

flags especifica a natureza do mapeamento. **MAP_PRIVATE** cria um mapeamento privado de cópia na gravação, de modo que alterações no conteúdo do objeto mmap serão privadas para este processo, e **MAP_SHARED** cria um mapeamento compartilhado com todos os outros processos que mapeiam as mesmas áreas do arquivo. O valor padrão é **MAP_SHARED**. Alguns sistemas têm sinalizadores adicionais possíveis, com a lista completa especificada em constantes MAP_*.

prot, se especificado, fornece a proteção de memória desejada; os dois valores mais úteis são **PROT_READ** e **PROT_WRITE**, para especificar que as páginas podem ser lidas ou gravadas. *prot* tem como padrão **PROT_READ | PROT_WRITE**.

access pode ser especificado em vez de *flags* e *prot* como um parâmetro nomeado opcional. É um erro especificar *flags*, *prot* e *access*. Consulte a descrição de *access* acima para obter informações sobre como usar este parâmetro.

offset pode ser especificado como um deslocamento inteiro não negativo. As referências mmap serão relativas ao deslocamento do início do arquivo. *offset* tem como padrão 0. *offset* deve ser um múltiplo de **`ALLOCATIONGRANULARITY`**, que é igual a **`PAGESIZE`** em sistemas Unix.

Se *trackfd* for `False`, o descritor de arquivo especificado por *fileno* não será duplicado e o objeto **`mmap`** resultante não será associado ao arquivo subjacente do mapa. Isso significa que os métodos **`size()`** e **`resize()`** falharão. Este modo é útil para limitar o número de descritores de arquivo abertos.

Para garantir a validade do mapeamento de memória criado, o arquivo especificado pelo descritor *fileno* é sincronizado internamente de forma automática com o armazenamento de suporte físico no macOS.

Alterado na versão 3.13: The *trackfd* parameter was added.

This example shows a simple way of using **`mmap`**:

```
import mmap

# write a simple example file
with open("hello.txt", "wb") as f:
    f.write(b"Hello Python!\n")

with open("hello.txt", "r+b") as f:
    # memory-map the file, size 0 means whole file
    mm = mmap.mmap(f.fileno(), 0)
    # read content via standard file methods
    print(mm.readline())  # prints b"Hello Python!\n"
    # read content via slice notation
    print(mm[:5])  # prints b"Hello"
    # update content using slice notation;
    # note that new content must have same size
    mm[6:] = b" world!\n"
```

```
    # ... and read again using standard file methods
    mm.seek(0)
    print(mm.readline())  # prints b"Hello  world!\n"
    # close the map
    mm.close()
```

mmap can also be used as a context manager in a **with** statement:

```
import mmap

with mmap.mmap(-1, 13) as mm:
    mm.write(b"Hello world!")
```

Adicionado na versão 3.2: Suporte a gerenciador de contexto.

The next example demonstrates how to create an anonymous map and exchange data between the parent and child processes:

```
import mmap
import os

mm = mmap.mmap(-1, 13)
mm.write(b"Hello world!")

pid = os.fork()

if pid == 0:  # In a child process
    mm.seek(0)
    print(mm.readline())

    mm.close()
```

Levanta um evento de auditoria `mmap.__new__` com os argumentos `fileno`, `length`, `access`, `offset`.

Memory-mapped file objects support the following methods:

close()

Closes the mmap. Subsequent calls to other methods of the object will result in a ValueError exception being raised. This will not close the open file.

closed

`True` if the file is closed.

Adicionado na versão 3.2.

find(*sub*[, *start*[, *end*]])

Returns the lowest index in the object where the subsequence *sub* is found, such that *sub* is contained in the range [*start*, *end*]. Optional arguments *start* and *end* are interpreted as in slice notation. Returns `-1` on failure.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

flush([*offset*, [*size*,]]*, *flags=MS_SYNC*)

Flushes changes made to the in-memory copy of a file back to disk. Without use of this call there is no guarantee that changes are written back before the object is destroyed. If *offset* and *size* are specified, only changes to the given range of bytes will be flushed to disk; otherwise, the whole extent of the mapping is flushed. *offset* must be a multiple of the **`PAGESIZE`** or **`ALLOCATIONGRANULARITY`**.

The *flags* parameter specifies the synchronization behavior. *flags* must be one of the MS_* constants available on the system.

On Windows, the *flags* parameter is ignored.

`None` is returned to indicate success. An exception is raised when the call failed.

Alterado na versão 3.8: Previously, a nonzero value was returned on success; zero was returned on error under Windows. A zero value was returned on success; an exception was raised on error under Unix.

Alterado na versão 3.15: Allow specifying *offset* without *size*. Previously, both *offset* and *size* parameters were required together.

Now *offset* can be specified alone, and the flush operation will extend from *offset* to the end of the mmap.

Alterado na versão 3.15: Added *flags* parameter to control synchronization behavior.

madvise(*option*[, *start*[, *length*]])

Send advice *option* to the kernel about the memory region beginning at *start* and extending *length* bytes. *option* must be one of the MADV_* constants available on the system. If *start* and *length* are omitted, the entire mapping is spanned. On some systems (including Linux), *start* must be a multiple of the **PAGESIZE**.

Availability: Systems with the `madvise()` system call.

Adicionado na versão 3.8.

move(*dest*, *src*, *count*)

Copy the *count* bytes starting at offset *src* to the destination index *dest*. If the mmap was created with **ACCESS_READ**, then calls to move will raise a **TypeError** exception.

read([*n*])

Return a **bytes** containing up to *n* bytes starting from the current file position. If the argument is omitted, `None` or negative, return all bytes from the current file position to the end of the mapping. The file position is updated to point after the bytes that were returned.

Alterado na versão 3.3: Argument can be omitted or `None`.

read_byte()

Returns a byte at the current file position as an integer, and advances the file position by 1.

readline()

Returns a single line, starting at the current file position and up to the next newline. The file position is updated to point after the bytes that were returned.

resize(*newsize*)

Resizes the map and the underlying file, if any.

Resizing a map created with *access* of **`ACCESS_READ`** or **`ACCESS_COPY`**, will raise a **`TypeError`** exception. Resizing a map created with *trackfd* set to `False`, will raise a **`ValueError`** exception.

On Windows: Resizing the map will raise an **`OSError`** if there are other maps against the same named file. Resizing an anonymous map (ie against the pagefile) will silently create a new map with the original data copied over up to the length of the new size.

Availability: Windows and systems with the `mremap()` system call.

Alterado na versão 3.11: Correctly fails if attempting to resize when another map is held Allows resize against an anonymous map on Windows

rfind(*sub*[, *start*[, *end*]])

Returns the highest index in the object where the subsequence *sub* is found, such that *sub* is contained in the range [*start*, *end*]. Optional arguments *start* and *end* are interpreted as in slice notation. Returns `-1` on failure.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

seek(*pos*[, *whence*])

Set the file's current position. *whence* argument is optional and defaults to `os.SEEK_SET` or `0` (absolute file positioning); other values are `os.SEEK_CUR` or `1` (seek relative to the current position) and `os.SEEK_END` or `2` (seek relative to the file's end).

Alterado na versão 3.13: Return the new absolute position instead of `None`.

seekable()

Return whether the file supports seeking, and the return value is always `True`.

Adicionado na versão 3.13.

set_name(*name*, /)

Annotate the memory mapping with the given *name* for easier identification in `/proc/<pid>/maps` if the kernel supports the feature and **`-X dev`** is passed to Python or if Python is built in debug mode. The length of *name* must not exceed 67 bytes including the `'\0'` terminator.

Disponibilidade: Linux >= 5.17 (kernel built with `CONFIG_ANON_VMA_NAME` option)

Adicionado na versão 3.15.

size()

Return the length of the file, which can be larger than the size of the memory-mapped area. For an anonymous mapping, return its size.

Alterado na versão 3.15: Anonymous mappings are now supported on Unix.

tell()

Returns the current position of the file pointer.

write(*bytes*)

Write the bytes in *bytes* into memory at the current position of the file pointer and return the number of bytes written (never less than `len(bytes)`, since if the write fails, a **`ValueError`** will be raised). The file position is updated to point after the bytes that were written.

If the mmap was created with **ACCESS_READ**, then writing to it will raise a **TypeError** exception.

Alterado na versão 3.5: Writable bytes-like object is now accepted.

Alterado na versão 3.6: The number of bytes written is now returned.

write_byte(*byte*)

Write the integer *byte* into memory at the current position of the file pointer; the file position is advanced by `1`. If the mmap was created with **ACCESS_READ**, then writing to it will raise a **TypeError** exception.

Constantes MADV_*

mmap.MADV_NORMAL
mmap.MADV_RANDOM
mmap.MADV_SEQUENTIAL
mmap.MADV_WILLNEED
mmap.MADV_DONTNEED
mmap.MADV_REMOVE
mmap.MADV_DONTFORK
mmap.MADV_DOFORK
mmap.MADV_HWPOISON
mmap.MADV_MERGEABLE
mmap.MADV_UNMERGEABLE
mmap.MADV_SOFT_OFFLINE
mmap.MADV_HUGEPAGE
mmap.MADV_NOHUGEPAGE
mmap.MADV_DONTDUMP
mmap.MADV_DODUMP
mmap.MADV_FREE
mmap.MADV_NOSYNC
mmap.MADV_AUTOSYNC
mmap.MADV_NOCORE
mmap.MADV_CORE

mmap.MADV_PROTECT
mmap.MADV_FREE_REUSABLE
mmap.MADV_FREE_REUSE

These options can be passed to `mmap.madvise()`. Not every option will be present on every system.

Availability: Systems with the madvise() system call.

Adicionado na versão 3.8.

Constantes MAP_*

mmap.MAP_SHARED
mmap.MAP_PRIVATE
mmap.MAP_32BIT
mmap.MAP_ALIGNED_SUPER
mmap.MAP_ANON
mmap.MAP_ANONYMOUS
mmap.MAP_CONCEAL
mmap.MAP_DENYWRITE
mmap.MAP_EXECUTABLE
mmap.MAP_HASSEMAPHORE
mmap.MAP_JIT
mmap.MAP_NOCACHE
mmap.MAP_NOEXTEND
mmap.MAP_NORESERVE
mmap.MAP_POPULATE
mmap.MAP_RESILIENT_CODESIGN
mmap.MAP_RESILIENT_MEDIA
mmap.MAP_STACK
mmap.MAP_TPRO
mmap.MAP_TRANSLATED_ALLOW_EXECUTE
mmap.MAP_UNIX03

These are the various flags that can be passed to `mmap.mmap()`. **`MAP_ALIGNED_SUPER`** is only available at FreeBSD and **`MAP_CONCEAL`** is only available at OpenBSD. Note that some options might not be present on some systems.

Alterado na versão 3.10: Added **`MAP_POPULATE`** constant.

Adicionado na versão 3.11: Added **`MAP_STACK`** constant.

Adicionado na versão 3.12: Added **`MAP_ALIGNED_SUPER`** and **`MAP_CONCEAL`** constants.

Adicionado na versão 3.13: Added **`MAP_32BIT`**, **`MAP_HASSEMAPHORE`**, **`MAP_JIT`**, **`MAP_NOCACHE`**, **`MAP_NOEXTEND`**, **`MAP_NORESERVE`**, **`MAP_RESILIENT_CODESIGN`**, **`MAP_RESILIENT_MEDIA`**, **`MAP_TPRO`**, **`MAP_TRANSLATED_ALLOW_EXECUTE`**, and **`MAP_UNIX03`** constants.

MS_* Constants

mmap.MS_SYNC
mmap.MS_ASYNC
mmap.MS_INVALIDATE

These flags control the synchronization behavior for **`mmap.flush()`**:

- **`MS_SYNC`** - Synchronous flush: writes are scheduled and the call blocks until they are physically written to storage.
- **`MS_ASYNC`** - Asynchronous flush: writes are scheduled but the call returns immediately without waiting for completion.
- **`MS_INVALIDATE`** - Invalidate cached data: invalidates other mappings of the same file so they can see the changes.

Adicionado na versão 3.15.

Manuseio de Dados na Internet

Este capítulo descreve módulos que suportam a manipulação de formatos de dados comumente usados na Internet.

- `email` — Um e-mail e um pacote MIME manipulável
  - `email.message`: Representing an email message
  - `email.parser`: Parsing email messages
    - API do FeedParser
    - Parser API
    - Additional notes
  - `email.generator`: Geração de documentos MIME
  - `email.policy`: Policy Objects
  - `email.errors`: Classes de Exceção e Defeito.
  - `email.headerregistry`: Custom Header Objects
  - `email.contentmanager`: Gerenciando conteúdo MIME
    - Content Manager Instances
  - `email`: Exemplos
  - `email.message.Message`: Representing an email message using the `compat32` API
  - `email.mimeemail.mime`: Criando e-mail e objetos MIME do zero
  - `email.header`: Internationalized headers
  - `email.charset`: Representando conjuntos de caracteres
  - `email.encoders`: Codificadores
  - `email.utils`: Utilitários diversos
  - `email.iterators`: Iteradores
- `json` — Codificador e decodificador JSON
  - Uso básico
  - Codificadores e decodificadores
  - Exceções
  - Conformidade e interoperabilidade entre padrões
    - Codificações de caracteres
    - Valores numéricos infinitos e NaN
    - Nomes repetidos dentro de um objeto
    - Valores não objeto e não vetor de nível superior

        - Limitações de implementação
    - Interface de linha de comando
        - Opções de linha de comando
- `mailbox` — Manipule caixas de entradas em vários formatos
    - Objetos `Mailbox`
        - `Maildir` objects
        - `mbox` objects
        - `MH` objects
        - `Babyl` objects
        - `MMDF` objects
    - `Message` objects
        - `MaildirMessage` objects
        - `mboxMessage` objects
        - `MHMessage` objects
        - `BabylMessage` objects
        - `MMDFMessage` objects
    - Exceções
    - Exemplos
- `mimetypes` — Mapeia nomes de arquivos para tipos MIME
    - Objetos MimeTypes
    - Uso na linha de comando
    - Exemplos na linha de comando
- `base64` — Base16, Base32, Base64, Base85 Data Encodings
    - RFC 4648 Encodings
    - Base85 Encodings
    - Legacy Interface
    - Considerações de Segurança
- `binascii` — Converte entre binário e ASCII
- `quopri` — Codifica e decodifica dados MIME imprimidos entre aspas

`email` — Um e-mail e um pacote MIME manipulável

Código-fonte: Lib/email/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/email/__init__.py]

The `email` package is a library for managing email messages. It is specifically *not* designed to do any sending of email messages to SMTP (**RFC 2821** [https://datatracker.ietf.org/doc/html/rfc2821.html]), NNTP, or other servers; those are functions of modules such as `smtplib`. The `email` package attempts to be as RFC-compliant as possible, supporting **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] and **RFC 6532** [https://datatracker.ietf.org/doc/html/rfc6532.html], as well as such MIME-related RFCs as **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html], **RFC 2046** [https://datatracker.ietf.org/doc/html/rfc2046.html], **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html], **RFC 2183** [https://datatracker.ietf.org/doc/html/rfc2183.html], and **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html].

No geral a estrutura do pacote de e-mail pode ser dividida em três componentes principais, mais um quarto componente que controla o comportamento dos outros componentes.

O componente central do pacote é um “modelo de objeto” que representa mensagens de e-mail. Uma aplicação interage com o pacote principalmente através da interface do modelo de objeto definida no submódulo `message`. A aplicação pode usar essa API para fazer perguntas sobre um e-mail existente, construir um novo e-mail ou adicionar ou remover subcomponentes de e-mail que usam a mesma interface de modelo de objeto. Ou seja, seguindo a natureza das mensagens de e-mail e seus subcomponentes MIME, o modelo de objeto de e-mail é uma estrutura em árvore de objetos que fornecem a API `EmailMessage`.

Os outros dois componentes principais do pacote são `parser` e `generator`. O analisador sintático pega a versão serializada de uma mensagem de e-mail (um fluxo de bytes) e a converte em uma árvore de objetos `EmailMessage`. O gerador pega um **`EmailMessage`** e o transforma novamente em um fluxo de bytes serializado. (O analisador sintático e o gerador também lidam com fluxos de caracteres de texto, mas esse uso é desencorajado, pois é muito fácil terminar com mensagens que não são válidas de uma maneira ou de outra.)

O componente de controle é o módulo `policy`. Cada `EmailMessage`, cada `generator` e cada `parser` tem um objeto associado **`policy`** que controla seu comportamento. Normalmente, uma aplicação precisa especificar a política apenas quando uma **`EmailMessage`** é criada, instanciando diretamente uma **`EmailMessage`** para criar um novo e-mail ou analisando um fluxo de entrada usando um **`parser`**. Mas a política pode ser alterada quando a mensagem é serializada usando um **`generator`**. Isso permite, por exemplo, analisar uma mensagem de e-mail genérica do disco, mas serializá-la usando as configurações SMTP padrão ao enviá-la para um servidor de e-mail.

The email package does its best to hide the details of the various governing RFCs from the application. Conceptually the application should be able to treat the email message as a structured tree of Unicode text and binary attachments, without having to worry about how these are represented when serialized. In practice, however, it is often necessary to be aware of at least some of the rules governing MIME messages and their structure, specifically the names and nature of the MIME “content types” and how they identify multipart documents. For the most part this knowledge should only be required for more complex applications, and even then it should only be the high level structure in question, and not the details of how those structures are represented. Since MIME content types are used widely in modern internet software (not just email), this will be a familiar concept to many programmers.

The following sections describe the functionality of the **`email`** package. We start with the `message` object model, which is the primary interface an application will use, and follow that with the `parser` and `generator`

components. Then we cover the `policy` controls, which completes the treatment of the main components of the library.

As próximas três seções cobrem as exceções que o pacote pode apresentar e os defeitos (não conformidade com as RFCs) que o `parser` pode detectar. A seguir, abordamos os subcomponentes `headerregistry` e os subcomponentes `contentmanager`, que fornecem ferramentas para manipulação mais detalhada de cabeçalhos e cargas úteis, respectivamente. Ambos os componentes contêm recursos relevantes para consumir e produzir mensagens não triviais, mas também documentam suas APIs de extensibilidade, que serão de interesse para aplicações avançadas.

A seguir, é apresentado um conjunto de exemplos de uso das partes fundamentais das APIs abordadas nas seções anteriores.

The foregoing represent the modern (Unicode friendly) API of the email package. The remaining sections, starting with the `Message` class, cover the legacy `compat32` API that deals much more directly with the details of how email messages are represented. The **`compat32`** API does *not* hide the details of the RFCs from the application, but for applications that need to operate at that level, they can be useful tools. This documentation is also relevant for applications that are still using the `compat32` API for backward compatibility reasons.

Alterado na versão 3.6: Documentos reorganizados e reescritos para promover a nova API `EmailMessage`/`EmailPolicy`.

Contents of the **`email`** package documentation:

- `email.message`: Representing an email message
- `email.parser`: Parsing email messages
  - API do FeedParser
  - Parser API
  - Additional notes
- `email.generator`: Geração de documentos MIME
- `email.policy`: Policy Objects
- `email.errors`: Classes de Exceção e Defeito.
- `email.headerregistry`: Custom Header Objects

- `email.contentmanager`: Gerenciando conteúdo MIME
  - Content Manager Instances
- `email`: Exemplos

API legada

- `email.message.Message`: Representing an email message using the `compat32` API
- `email.mimeemail.mime`: Criando e-mail e objetos MIME do zero
- `email.header`: Internationalized headers
- `email.charset`: Representando conjuntos de caracteres
- `email.encoders`: Codificadores
- `email.utils`: Utilitários diversos
- `email.iterators`: Iteradores

Ver também

Módulo `smtplib`
Cliente SMTP (Protocolo Simples de Envio de E-mail)

Módulo `poplib`
Cliente POP (Post Office Protocol)

Módulo `imaplib`
Cliente IMAP (Internet Message Access Protocol)

Módulo `mailbox`
Ferramentas para criar, ler, e gerenciar coleções de mensagem em disco usando vários formatos padrão.

`email.message`: Representing an email message

Código-fonte: Lib/email/message.py
[https://github.com/python/cpython/tree/3.15/Lib/email/message.py]

Adicionado na versão 3.6: [[1]]

The central class in the `email` package is the `EmailMessage` class, imported from the `email.message` module. It is the base class for the `email` object model. `EmailMessage` provides the core functionality for setting and querying header fields, for accessing message bodies, and for creating or modifying structured messages.

An email message consists of *headers* and a *payload* (which is also referred to as the *content*). Headers are **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] or **RFC 6532** [https://datatracker.ietf.org/doc/html/rfc6532.html] style field names and values, where the field name and value are separated by a colon. The colon is not part of either the field name or the field value. The payload may be a simple text message, or a binary object, or a structured sequence of sub-messages each with their own set of headers and their own payload. The latter type of payload is indicated by the message having a MIME type such as *multipart/** or *message/rfc822*.

The conceptual model provided by an `EmailMessage` object is that of an ordered dictionary of headers coupled with a *payload* that represents the **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] body of the message, which might be a list of sub-`EmailMessage` objects. In addition to the normal dictionary methods for accessing the header names and values, there are methods for accessing specialized information from the headers (for example the MIME content type), for operating on the payload, for

generating a serialized version of the message, and for recursively walking over the object tree.

The **EmailMessage** dictionary-like interface is indexed by the header names, which must be ASCII values. The values of the dictionary are strings with some extra methods. Headers are stored and returned in case-preserving form, but field names are matched case-insensitively. The keys are ordered, but unlike a real dict, there can be duplicates. Additional methods are provided for working with headers that have duplicate keys.

The *payload* is either a string or bytes object, in the case of simple message objects, or a list of **EmailMessage** objects, for MIME container documents such as *multipart/** and *message/rfc822* message objects.

class email.message.EmailMessage(*policy=default*)

If *policy* is specified use the rules it specifies to update and serialize the representation of the message. If *policy* is not set, use the **default** policy, which follows the rules of the email RFCs except for line endings (instead of the RFC mandated `\r\n`, it uses the Python standard `\n` line endings). For more information see the **policy** documentation. [[2]]

as_string(*unixfrom=False*, *maxheaderlen=None*, *policy=None*)

Return the entire message flattened as a string. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to `False`. For backward compatibility with the base **Message** class *maxheaderlen* is accepted, but defaults to `None`, which means that by default the line length is controlled by the **max_line_length** of the policy. The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the **Generator**.

Flattening the message may trigger changes to the **EmailMessage** if defaults need to be filled in to complete the transformation to a

string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not be the most useful way to serialize messages in your application, especially if you are dealing with multiple messages. See **`email.generator.Generator`** for a more flexible API for serializing messages. Note also that this method is restricted to producing messages serialized as "7 bit clean" when **`utf8`** is `False`, which is the default.

Alterado na versão 3.6: the default behavior when *maxheaderlen* is not specified was changed from defaulting to 0 to defaulting to the value of *max_line_length* from the policy.

__str__()

Equivalent to `as_string(policy=self.policy.clone(utf8=True))`. Allows `str(msg)` to produce a string containing the serialized message in a readable format.

Alterado na versão 3.4: the method was changed to use `utf8=True`, thus producing an **RFC 6531** [https://datatracker.ietf.org/doc/html/rfc6531.html]-like message representation, instead of being a direct alias for **`as_string()`**.

as_bytes(*unixfrom=False*, *policy=None*)

Return the entire message flattened as a bytes object. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to `False`. The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the **`BytesGenerator`**.

Flattening the message may trigger changes to the **`EmailMessage`** if defaults need to be filled in to complete the transformation to a

string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not be the most useful way to serialize messages in your application, especially if you are dealing with multiple messages. See **email.generator.BytesGenerator** for a more flexible API for serializing messages.

__bytes__()

Equivalent to **as_bytes()**. Allows `bytes(msg)` to produce a bytes object containing the serialized message.

is_multipart()

Return `True` if the message's payload is a list of sub-**EmailMessage** objects, otherwise return `False`. When **is_multipart()** returns `False`, the payload should be a string object (which might be a CTE encoded binary payload). Note that **is_multipart()** returning `True` does not necessarily mean that "msg.get_content_maintype() == 'multipart'" will return the `True`. For example, `is_multipart` will return `True` when the **EmailMessage** is of type `message/rfc822`.

set_unixfrom(*unixfrom*)

Set the message's envelope header to *unixfrom*, which should be a string. (See **mboxMessage** for a brief description of this header.)

get_unixfrom()

Return the message's envelope header. Defaults to `None` if the envelope header was never set.

The following methods implement the mapping-like interface for accessing the message's headers. Note that there are some semantic differences between these methods and a normal mapping (i.e. dictionary) interface. For example, in a dictionary there are no duplicate keys, but here there may be duplicate message headers. Also, in dictionaries there is no guaranteed order to the keys returned by **keys()**, but in an **EmailMessage** object, headers are always returned in the order

they appeared in the original message, or in which they were added to the message later. Any header deleted and then re-added is always appended to the end of the header list.

These semantic differences are intentional and are biased toward convenience in the most common use cases.

Note that in all cases, any envelope header present in the message is not included in the mapping interface.

__len__()

Return the total number of headers, including duplicates.

__contains__(*name*)

Return `True` if the message object has a field named *name*. Matching is done without regard to case and *name* does not include the trailing colon. Used for the `in` operator. For example:

```
if 'message-id' in myMessage:
   print('Message-ID:', myMessage['message-id'])
```

__getitem__(*name*)

Return the value of the named header field. *name* does not include the colon field separator. If the header is missing, `None` is returned; a **`KeyError`** is never raised.

Note that if the named field appears more than once in the message's headers, exactly which of those field values will be returned is undefined. Use the **`get_all()`** method to get the values of all the extant headers named *name*.

Using the standard (non-`compat32`) policies, the returned value is an instance of a subclass of **`email.headerregistry.BaseHeader`**.

__setitem__(*name*, *val*)

Add a header to the message with field name *name* and value *val*. The field is appended to the end of the message's existing headers.

Note that this does *not* overwrite or delete any existing header with the same name. If you want to ensure that the new header is the only one present in the message with field name *name*, delete the field first, e.g.:

```
del msg['subject']
msg['subject'] = 'Python roolz!'
```

If the `policy` defines certain headers to be unique (as the standard policies do), this method may raise a `ValueError` when an attempt is made to assign a value to such a header when one already exists. This behavior is intentional for consistency's sake, but do not depend on it as we may choose to make such assignments do an automatic deletion of the existing header in the future.

__delitem__(*name*)

Delete all occurrences of the field with name *name* from the message's headers. No exception is raised if the named field isn't present in the headers.

keys()

Return a list of all the message's header field names.

values()

Return a list of all the message's field values.

items()

Return a list of 2-tuples containing all the message's field headers and values.

get(*name*, *failobj=None*)

Return the value of the named header field. This is identical to `__getitem__()` except that optional *failobj* is returned if the named header is missing (*failobj* defaults to `None`).

Here are some additional useful header related methods:

get_all(*name*, *failobj=None*)

Return a list of all the values for the field named *name*. If there are no such named headers in the message, *failobj* is returned (defaults to `None`).

add_header(*_name*, *_value*, ***_params*)

Extended header setting. This method is similar to `__setitem__()` except that additional header parameters can be provided as keyword arguments. *_name* is the header field to add and *_value* is the *primary* value for the header.

For each item in the keyword argument dictionary *_params*, the key is taken as the parameter name, with underscores converted to dashes (since dashes are illegal in Python identifiers). Normally, the parameter will be added as `key="value"` unless the value is `None`, in which case only the key will be added.

If the value contains non-ASCII characters, the charset and language may be explicitly controlled by specifying the value as a three tuple in the format `(CHARSET, LANGUAGE, VALUE)`, where `CHARSET` is a string naming the charset to be used to encode the value, `LANGUAGE` can usually be set to `None` or the empty string (see **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] for other possibilities), and `VALUE` is the string value containing non-ASCII code points. If a three tuple is not passed and the value contains non-ASCII characters, it is automatically encoded in **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] format using a `CHARSET` of `utf-8` and a `LANGUAGE` of `None`.

Aqui está um exemplo:

```
msg.add_header('Content-Disposition', 'attachment', filena
```

This will add a header that looks like

```
Content-Disposition: attachment; filename="bud.gif"
```

An example of the extended interface with non-ASCII characters:

```
msg.add_header('Content-Disposition', 'attachment',
               filename=('iso-8859-1', '', 'Fußballer.ppt
```

replace_header(*_name*, *_value*)

Replace a header. Replace the first header found in the message that matches *_name*, retaining header order and field name case of the original header. If no matching header is found, raise a `KeyError`.

get_content_type()

Return the message's content type, coerced to lower case of the form *maintype/subtype*. If there is no *Content-Type* header in the message return the value returned by `get_default_type()`. If the *Content-Type* header is invalid, return `text/plain`.

(According to **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html], messages always have a default type, **`get_content_type()`** will always return a value. **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] defines a message's default type to be *text/plain* unless it appears inside a *multipart/digest* container, in which case it would be *message/rfc822*. If the *Content-Type* header has an invalid type specification, **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] mandates that the default type be *text/plain*.)

get_content_maintype()

Return the message's main content type. This is the *maintype* part of the string returned by `get_content_type()`.

get_content_subtype()

Return the message's sub-content type. This is the *subtype* part of the string returned by `get_content_type()`.

get_default_type()

Return the default content type. Most messages have a default content type of *text/plain*, except for messages that are subparts of *multipart/digest* containers. Such subparts have a default content type of *message/rfc822*.

set_default_type(*ctype*)

Set the default content type. *ctype* should either be *text/plain* or *message/rfc822*, although this is not enforced. The default content type is not stored in the *Content-Type* header, so it only affects the return value of the `get_content_type` methods when no *Content-Type* header is present in the message.

set_param(*param*, *value*, *header='Content-Type'*, *requote=True*, *charset=None*, *language=''*, *replace=False*)

Set a parameter in the *Content-Type* header. If the parameter already exists in the header, replace its value with *value*. When *header* is `Content-Type` (the default) and the header does not yet exist in the message, add it, set its value to *text/plain*, and append the new parameter value. Optional *header* specifies an alternative header to *Content-Type*.

If the value contains non-ASCII characters, the charset and language may be explicitly specified using the optional *charset* and *language* parameters. Optional *language* specifies the **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] language, defaulting to the empty string. Both *charset* and *language* should be strings. The default is to use the `utf8` *charset* and `None` for the *language*.

If *replace* is `False` (the default) the header is moved to the end of the list of headers. If *replace* is `True`, the header will be updated in place.

Uso do parâmetro *requote* com objetos `EmailMessage` está descontinuado.

Note that existing parameter values of headers may be accessed through the `params` attribute of the header value (for example,

`msg['Content-Type'].params['charset']`).

Alterado na versão 3.4: Palavra-chave `replace` foi adicionada.

del_param(*param*, *header='content-type'*, *requote=True*)

Remove the given parameter completely from the *Content-Type* header. The header will be re-written in place without the parameter or its value. Optional *header* specifies an alternative to *Content-Type*.

Uso do parâmetro *requote* com objetos **`EmailMessage`** está descontinuado.

get_filename(*failobj=None*)

Return the value of the `filename` parameter of the *Content-Disposition* header of the message. If the header does not have a `filename` parameter, this method falls back to looking for the `name` parameter on the *Content-Type* header. If neither is found, or the header is missing, then *failobj* is returned. The returned string will always be unquoted as per **`email.utils.unquote()`**.

get_boundary(*failobj=None*)

Return the value of the `boundary` parameter of the *Content-Type* header of the message, or *failobj* if either the header is missing, or has no `boundary` parameter. The returned string will always be unquoted as per **`email.utils.unquote()`**.

set_boundary(*boundary*)

Set the `boundary` parameter of the *Content-Type* header to *boundary*. **`set_boundary()`** will always quote *boundary* if necessary. A **`HeaderParseError`** is raised if the message object has no *Content-Type* header.

Note that using this method is subtly different from deleting the old *Content-Type* header and adding a new one with the new boundary via **`add_header()`**, because **`set_boundary()`** preserves the order of the *Content-Type* header in the list of headers.

get_content_charset(*failobj=None*)

Return the `charset` parameter of the *Content-Type* header, coerced to lower case. If there is no *Content-Type* header, or if that header has no `charset` parameter, *failobj* is returned.

get_charsets(*failobj=None*)

Return a list containing the character set names in the message. If the message is a *multipart*, then the list will contain one element for each subpart in the payload, otherwise, it will be a list of length 1.

Each item in the list will be a string which is the value of the `charset` parameter in the *Content-Type* header for the represented subpart. If the subpart has no *Content-Type* header, no `charset` parameter, or is not of the *text* main MIME type, then that item in the returned list will be *failobj*.

is_attachment()

Return `True` if there is a *Content-Disposition* header and its (case insensitive) value is `attachment`, `False` otherwise.

Alterado na versão 3.4.2: is_attachment is now a method instead of a property, for consistency with **`is_multipart()`**.

get_content_disposition()

Return the lowercased value (without parameters) of the message's *Content-Disposition* header if it has one, or `None`. The possible values for this method are *inline*, *attachment* or `None` if the message follows **RFC 2183** [https://datatracker.ietf.org/doc/html/rfc2183.html].

Adicionado na versão 3.5.

The following methods relate to interrogating and manipulating the content (payload) of the message.

walk()

The **`walk()`** method is an all-purpose generator which can be used to iterate over all the parts and subparts of a message object tree, in

depth-first traversal order. You will typically use **`walk()`** as the iterator in a `for` loop; each iteration returns the next subpart.

Here's an example that prints the MIME type of every part of a multipart message structure:

```
>>> for part in msg.walk():
...     print(part.get_content_type())
multipart/report
text/plain
message/delivery-status
text/plain
text/plain
message/rfc822
text/plain
```

`walk` iterates over the subparts of any part where **`is_multipart()`** returns `True`, even though `msg.get_content_maintype() == 'multipart'` may return `False`. We can see this in our example by making use of the `_structure` debug helper function:

```
>>> from email.iterators import _structure
>>> for part in msg.walk():
...     print(part.get_content_maintype() == 'multipart',
...           part.is_multipart())
True True
False False
False True
False False
False False
False True
False False
>>> _structure(msg)
multipart/report
    text/plain
    message/delivery-status
        text/plain
        text/plain
    message/rfc822
        text/plain
```

Here the `message` parts are not `multiparts`, but they do contain subparts. `is_multipart()` returns `True` and `walk` descends into the subparts.

get_body(*preferencelist=('related', 'html', 'plain')*)

Return the MIME part that is the best candidate to be the “body” of the message.

preferencelist must be a sequence of strings from the set `related`, `html`, and `plain`, and indicates the order of preference for the content type of the part returned.

Start looking for candidate matches with the object on which the `get_body` method is called.

If `related` is not included in *preferencelist*, consider the root part (or subpart of the root part) of any related encountered as a candidate if the (sub-)part matches a preference.

When encountering a `multipart/related`, check the `start` parameter and if a part with a matching *Content-ID* is found, consider only it when looking for candidate matches. Otherwise consider only the first (default root) part of the `multipart/related`.

If a part has a *Content-Disposition* header, only consider the part a candidate match if the value of the header is `inline`.

If none of the candidates matches any of the preferences in *preferencelist*, return `None`.

Notes: (1) For most applications the only *preferencelist* combinations that really make sense are `('plain',)`, `('html', 'plain')`, and the default `('related', 'html', 'plain')`. (2) Because matching starts with the object on which `get_body` is called, calling `get_body` on a `multipart/related` will return the object itself unless *preferencelist* has a non-default value. (3) Messages (or message parts) that do not specify a *Content-Type* or whose *Content-Type* header is invalid will be treated as if they are of

type `text/plain`, which may occasionally cause `get_body` to return unexpected results.

iter_attachments()

Return an iterator over all of the immediate sub-parts of the message that are not candidate "body" parts. That is, skip the first occurrence of each of `text/plain`, `text/html`, `multipart/related`, or `multipart/alternative` (unless they are explicitly marked as attachments via *Content-Disposition: attachment*), and return all remaining parts. When applied directly to a `multipart/related`, return an iterator over the all the related parts except the root part (ie: the part pointed to by the `start` parameter, or the first part if there is no `start` parameter or the `start` parameter doesn't match the *Content-ID* of any of the parts). When applied directly to a `multipart/alternative` or a non-`multipart`, return an empty iterator.

iter_parts()

Return an iterator over all of the immediate sub-parts of the message, which will be empty for a non-`multipart`. (See also **`walk()`**.)

get_content(**args*, *content_manager=None*, ***kw*)

Call the **`get_content()`** method of the *content_manager*, passing self as the message object, and passing along any other arguments or keywords as additional arguments. If *content_manager* is not specified, use the `content_manager` specified by the current **`policy`**.

set_content(**args*, *content_manager=None*, ***kw*)

Call the **`set_content()`** method of the *content_manager*, passing self as the message object, and passing along any other arguments or keywords as additional arguments. If *content_manager* is not specified, use the `content_manager` specified by the current **`policy`**.

make_related(*boundary=None*)

Convert a non-`multipart` message into a `multipart/related` message, moving any existing *Content-* headers and payload into a (new) first part of the `multipart`. If *boundary* is specified, use it as the boundary string in the multipart, otherwise leave the boundary to be automatically created when it is needed (for example, when the message is serialized).

make_alternative(*boundary=None*)

Convert a non-`multipart` or a `multipart/related` into a `multipart/alternative`, moving any existing *Content-* headers and payload into a (new) first part of the `multipart`. If *boundary* is specified, use it as the boundary string in the multipart, otherwise leave the boundary to be automatically created when it is needed (for example, when the message is serialized).

make_mixed(*boundary=None*)

Convert a non-`multipart`, a `multipart/related`, or a `multipart-alternative` into a `multipart/mixed`, moving any existing *Content-* headers and payload into a (new) first part of the `multipart`. If *boundary* is specified, use it as the boundary string in the multipart, otherwise leave the boundary to be automatically created when it is needed (for example, when the message is serialized).

add_related(**args*, *content_manager=None*, ***kw*)

If the message is a `multipart/related`, create a new message object, pass all of the arguments to its **`set_content()`** method, and **`attach()`** it to the `multipart`. If the message is a non-`multipart`, call **`make_related()`** and then proceed as above. If the message is any other type of `multipart`, raise a **`TypeError`**. If *content_manager* is not specified, use the `content_manager` specified by the current **`policy`**. If the added part has no *Content-Disposition* header, add one with the value `inline`.

add_alternative(**args*, *content_manager=None*, ***kw*)

If the message is a `multipart/alternative`, create a new message object, pass all of the arguments to its **`set_content()`** method, and **`attach()`** it to the `multipart`. If the message is a non-`multipart` or `multipart/related`, call **`make_alternative()`** and then proceed as above. If the message is any other type of `multipart`, raise a **`TypeError`**. If *content_manager* is not specified, use the `content_manager` specified by the current **`policy`**.

add_attachment(**args*, *content_manager=None*, ***kw*)

If the message is a `multipart/mixed`, create a new message object, pass all of the arguments to its **`set_content()`** method, and **`attach()`** it to the `multipart`. If the message is a non-`multipart`, `multipart/related`, or `multipart/alternative`, call **`make_mixed()`** and then proceed as above. If *content_manager* is not specified, use the `content_manager` specified by the current **`policy`**. If the added part has no *Content-Disposition* header, add one with the value `attachment`. This method can be used both for explicit attachments (*Content-Disposition: attachment*) and `inline` attachments (*Content-Disposition: inline*), by passing appropriate options to the `content_manager`.

clear()

Remove the payload and all of the headers.

clear_content()

Remove the payload and all of the *!Content-* headers, leaving all other headers intact and in their original order.

`EmailMessage` objects have the following instance attributes:

preamble

The format of a MIME document allows for some text between the blank line following the headers, and the first multipart boundary string. Normally, this text is never visible in a MIME-aware mail reader because it falls outside the standard MIME armor. However, when viewing the raw text of the message, or when viewing the message in a non-MIME aware reader, this text can become visible.

The *preamble* attribute contains this leading extra-armor text for MIME documents. When the `Parser` discovers some text after the headers but before the first boundary string, it assigns this text to the message's *preamble* attribute. When the `Generator` is writing out the plain text representation of a MIME message, and it finds the message has a *preamble* attribute, it will write this text in the area between the headers and the first boundary. See `email.parser` and `email.generator` for details.

Note that if the message object has no preamble, the *preamble* attribute will be `None`.

epilogue

The *epilogue* attribute acts the same way as the *preamble* attribute, except that it contains text that appears between the last boundary and the end of the message. As with the `preamble`, if there is no epilog text this attribute will be `None`.

defects

The *defects* attribute contains a list of all the problems found when parsing this message. See `email.errors` for a detailed description of the possible parsing defects.

class email.message.MIMEPart(*policy=default*)

This class represents a subpart of a MIME message. It is identical to `EmailMessage`, except that no *MIME-Version* headers are added when `set_content()` is called, since sub-parts do not need their own *MIME-Version* headers.

Notas de rodapé

[1]

Originally added in 3.4 as a provisional module. Docs for legacy message class moved to email.message.Message: Representing an email message using the compat32 API.

[2]

The **`EmailMessage`** class requires a policy that provides a `content_manager` attribute for content management methods like `set_content()` and `get_content()` to work. The legacy **`compat32`** policy does not support these methods and should not be used with **`EmailMessage`**.

`email.parser`: Parsing email messages

Código-fonte: Lib/email/parser.py
[https://github.com/python/cpython/tree/3.15/Lib/email/parser.py]

Message object structures can be created in one of two ways: they can be created from whole cloth by creating an `EmailMessage` object, adding headers using the dictionary interface, and adding payload(s) using `set_content()` and related methods, or they can be created by parsing a serialized representation of the email message.

The `email` package provides a standard parser that understands most email document structures, including MIME documents. You can pass the parser a bytes, string or file object, and the parser will return to you the root `EmailMessage` instance of the object structure. For simple, non-MIME messages the payload of this root object will likely be a string containing the text of the message. For MIME messages, the root object will return `True` from its `is_multipart()` method, and the subparts can be accessed via the payload manipulation methods, such as `get_body()`, `iter_parts()`, and `walk()`.

There are actually two parser interfaces available for use, the `Parser` API and the incremental `FeedParser` API. The `Parser` API is most useful if you have the entire text of the message in memory, or if the entire message lives in a file on the file system. `FeedParser` is more appropriate when you are reading the message from a stream which might block waiting for more input (such as reading an email message from a socket). The `FeedParser` can consume and parse the message incrementally, and only returns the root object when you close the parser.

Note that the parser can be extended in limited ways, and of course you can implement your own parser completely from scratch. All of the logic that

connects the email package's bundled parser and the EmailMessage class is embodied in the Policy class, so a custom parser can create message object trees any way it finds necessary by implementing custom versions of the appropriate Policy methods.

API do FeedParser

The BytesFeedParser, imported from the email.feedparser module, provides an API that is conducive to incremental parsing of email messages, such as would be necessary when reading the text of an email message from a source that can block (such as a socket). The BytesFeedParser can of course be used to parse an email message fully contained in a bytes-like object, string, or file, but the BytesParser API may be more convenient for such use cases. The semantics and results of the two parser APIs are identical.

The BytesFeedParser's API is simple; you create an instance, feed it a bunch of bytes until there's no more to feed it, then close the parser to retrieve the root message object. The BytesFeedParser is extremely accurate when parsing standards-compliant messages, and it does a very good job of parsing non-compliant messages, providing information about how a message was deemed broken. It will populate a message object's defects attribute with a list of any problems it found in a message. See the email.errors module for the list of defects that it can find.

Here is the API for the BytesFeedParser:

class email.parser.BytesFeedParser(*_factory=None*, *, *policy=policy.compat32*)

Create a BytesFeedParser instance. Optional *_factory* is a no-argument callable; if not specified use the message_factory from the *policy*. Call *_factory* whenever a new message object is needed.

If *policy* is specified use the rules it specifies to update the representation of the message. If *policy* is not set, use the compat32 policy, which maintains backward compatibility with the Python 3.2 version of the email package and provides Message as the default

factory. All other policies provide `EmailMessage` as the default *_factory*. For more information on what else *policy* controls, see the `policy` documentation.

Note: **The policy keyword should always be specified**; The default will change to `email.policy.default` in a future version of Python.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Alterado na versão 3.6: _factory defaults to the policy `message_factory`.

feed(*data*)

Feed the parser some more data. *data* should be a bytes-like object containing one or more lines. The lines can be partial and the parser will stitch such partial lines together properly. The lines can have any of the three common line endings: carriage return, newline, or carriage return and newline (they can even be mixed).

close()

Complete the parsing of all previously fed data and return the root message object. It is undefined what happens if `feed()` is called after this method has been called.

class email.parser.FeedParser(*_factory=None*, *, *policy=policy.compat32*)

Works like `BytesFeedParser` except that the input to the `feed()` method must be a string. This is of limited utility, since the only way for such a message to be valid is for it to contain only ASCII text or, if `utf8` is `True`, no binary attachments.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Parser API

The **BytesParser** class, imported from the **email.parser** module, provides an API that can be used to parse a message when the complete contents of the message are available in a bytes-like object or file. The **email.parser** module also provides **Parser** for parsing strings, and header-only parsers, **BytesHeaderParser** and **HeaderParser**, which can be used if you're only interested in the headers of the message. **BytesHeaderParser** and **HeaderParser** can be much faster in these situations, since they do not attempt to parse the message body, instead setting the payload to the raw body.

class email.parser.BytesParser(*_class=None*, *, *policy=policy.compat32*)

Create a **BytesParser** instance. The *_class* and *policy* arguments have the same meaning and semantics as the *_factory* and *policy* arguments of **BytesFeedParser**.

Note: **The policy keyword should always be specified**; The default will change to **email.policy.default** in a future version of Python.

Alterado na versão 3.3: Removed the *strict* argument that was deprecated in 2.4. Added the *policy* keyword.

Alterado na versão 3.6: _class defaults to the policy `message_factory`.

parse(*fp*, *headersonly=False*)

Read all the data from the binary file-like object *fp*, parse the resulting bytes, and return the message object. *fp* must support both the **readline()** and the **read()** methods.

The bytes contained in *fp* must be formatted as a block of **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] (or, if **utf8** is `True`, **RFC 6532** [https://datatracker.ietf.org/doc/html/rfc6532.html]) style headers and header continuation lines, optionally preceded by an envelope header. The header block is terminated either by the end of the data or by a blank line. Following the header block is the body of the message (which may contain MIME-encoded subparts, including subparts with a *Content-Transfer-Encoding* of `8bit`).

Optional *headersonly* is a flag specifying whether to stop parsing after reading the headers or not. The default is `False`, meaning it parses the entire contents of the file.

parsebytes(*bytes*, *headersonly=False*)

Similar to the **`parse()`** method, except it takes a bytes-like object instead of a file-like object. Calling this method on a bytes-like object is equivalent to wrapping *bytes* in a **`BytesIO`** instance first and calling **`parse()`**.

Optional *headersonly* is as with the **`parse()`** method.

Adicionado na versão 3.2.

class email.parser.BytesHeaderParser(*_class=None*, *, *policy=policy.compat32*)

Exactly like **`BytesParser`**, except that *headersonly* defaults to `True`.

Adicionado na versão 3.3.

class email.parser.Parser(*_class=None*, *, *policy=policy.compat32*)

This class is parallel to **`BytesParser`**, but handles string input.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

Alterado na versão 3.6: _class defaults to the policy `message_factory`.

parse(*fp*, *headersonly=False*)

Read all the data from the text-mode file-like object *fp*, parse the resulting text, and return the root message object. *fp* must support both the **`readline()`** and the **`read()`** methods on file-like objects.

Other than the text mode requirement, this method operates like **`BytesParser.parse()`**.

parsestr(*text*, *headersonly=False*)

Similar to the **parse()** method, except it takes a string object instead of a file-like object. Calling this method on a string is equivalent to wrapping *text* in a **StringIO** instance first and calling **parse()**.

Optional *headersonly* is as with the **parse()** method.

class email.parser.HeaderParser(*_class=None*, *, *policy=policy.compat32*)

Exactly like **Parser**, except that *headersonly* defaults to `True`.

Since creating a message object structure from a string or a file object is such a common task, four functions are provided as a convenience. They are available in the top-level **email** package namespace.

email.message_from_bytes(*s*, *_class=None*, *, *policy=policy.compat32*)

Return a message object structure from a bytes-like object. This is equivalent to `BytesParser().parsebytes(s)`. Optional *_class* and *policy* are interpreted as with the **BytesParser** class constructor.

Adicionado na versão 3.2.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

email.message_from_binary_file(*fp*, *_class=None*, *, *policy=policy.compat32*)

Return a message object structure tree from an open binary file object. This is equivalent to `BytesParser().parse(fp)`. *_class* and *policy* are interpreted as with the **BytesParser** class constructor.

Adicionado na versão 3.2.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

email.message_from_string(*s*, *_class=None*, *, *policy=policy.compat32*)

Return a message object structure from a string. This is equivalent to `Parser().parsestr(s)`. *_class* and *policy* are interpreted as with the

`Parser` class constructor.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

email.message_from_file(*fp*, *_class=None*, *, *policy=policy.compat32*)

Return a message object structure tree from an open file object. This is equivalent to `Parser().parse(fp)`. *_class* and *policy* are interpreted as with the `Parser` class constructor.

Alterado na versão 3.3: Removed the *strict* argument. Added the *policy* keyword.

Alterado na versão 3.6: *_class* defaults to the policy `message_factory`.

Here's an example of how you might use `message_from_bytes()` at an interactive Python prompt:

```
>>> import email
>>> msg = email.message_from_bytes(myBytes)
```

Additional notes

Here are some notes on the parsing semantics:

- Most non-*multipart* type messages are parsed as a single message object with a string payload. These objects will return `False` for `is_multipart()`, and `iter_parts()` will yield an empty list.
- All *multipart* type messages will be parsed as a container message object with a list of sub-message objects for their payload. The outer container message will return `True` for `is_multipart()`, and `iter_parts()` will yield a list of subparts.
- Most messages with a content type of *message/** (such as *message/delivery-status* and *message/rfc822*) will also be parsed as container object containing a list payload of length 1. Their `is_multipart()` method will return `True`. The single element yielded by `iter_parts()` will be a sub-message object.

- Some non-standards-compliant messages may not be internally consistent about their *multipart*-edness. Such messages may have a *Content-Type* header of type *multipart*, but their `is_multipart()` method may return `False`. If such messages were parsed with the `FeedParser`, they will have an instance of the `MultipartInvariantViolationDefect` class in their *defects* attribute list. See `email.errors` for details.

`email.generator`: Geração de documentos MIME

Código-fonte: Lib/email/generator.py
[https://github.com/python/cpython/tree/3.15/Lib/email/generator.py]

Uma das tarefas mais comuns é gerar a versão plana (serializada) da mensagem de e-mail representada por uma estrutura de objeto de mensagem. Você precisará fazer isso se quiser enviar sua mensagem via `smtplib.SMTP.sendmail()` ou exibir a mensagem no console. A tarefa das classes geradoras é pegar uma estrutura de objeto de mensagem e produzir uma representação serializada.

Assim como no módulo `email.parser`, você não está limitado à funcionalidade do gerador integrado; você mesmo pode criar um do zero. No entanto, o gerador integrado sabe como gerar a maioria dos e-mails em conformidade com os padrões, deve lidar perfeitamente com mensagens de e-mail MIME e não MIME e foi projetado para que as operações de análise e geração orientadas a bytes sejam inversas, presumindo que a mesma `policy` não transformadora seja usada para ambas. Ou seja, analisar o fluxo de bytes serializado por meio da classe `BytesParser` e, em seguida, regenerar o fluxo de bytes serializado usando `BytesGenerator` deve produzir uma saída idêntica à entrada [[1]]. (Por outro lado, usar o gerador em um `EmailMessage` construído pelo programa pode resultar em alterações no objeto `EmailMessage` à medida que os padrões são preenchidos.)

A classe `Generator` pode ser usada para simplificar uma mensagem em uma representação serializada de texto (em oposição a binária), mas como o Unicode não pode representar dados binários diretamente, a mensagem é necessariamente transformada em algo que contém apenas caracteres ASCII, usando as técnicas de codificação de transferência de conteúdo RFC de e-mail padrão para codificar mensagens de e-mail para transporte em canais que não são “limpos de 8 bits”.

Para acomodar o processamento reproduzível de mensagens assinadas por SMIME, `Generator` desabilita a dobragem de cabeçalho para partes de mensagens do tipo `multipart/signed` e todas as subpartes.

class email.generator.BytesGenerator(*outfp*, *mangle_from_=None*, *maxheaderlen=None*, *, *policy=None*)

Retorna um objeto `BytesGenerator` que gravará qualquer mensagem fornecida ao método `flatten()`, ou qualquer texto codificado surrogateescape fornecido ao método `write()`, no objeto arquivo ou similar *outfp*. *outfp* deve oferecer suporte a um método `write` que aceite dados binários.

Se o *mangle_from_* opcional for `True`, coloca um caractere `>` na frente de qualquer linha no corpo que comece com a string exata `"From "`, ou seja, `From` seguido por um espaço no início de uma linha. *mangle_from_* assume por padrão o valor da configuração `mangle_from_` da *policy* (que é `True` para a política `compat32` e `False` para todas as outras). *mangle_from_* deve ser usado quando as mensagens são armazenadas no formato mbox do Unix (consulte `mailbox` e WHY THE CONTENT-LENGTH FORMAT IS BAD [https://www.jwz.org/doc/content-length.html]).

Se *maxheaderlen* não for `None`, redobra quaisquer linhas de cabeçalho maiores que *maxheaderlen* ou, se for `0`, não redobra nenhum cabeçalho. Se *manheaderlen* for `None` (o padrão), redobra os cabeçalhos e outras linhas de mensagem de acordo com as configurações da *policy*.

Se *policy* for especificado, usa essa política para controlar a geração de mensagens. Se *policy* for `None` (o padrão), usa a política associada ao objeto `Message` ou `EmailMessage` passado para `flatten` para controlar a geração de mensagens. Consulte `email.policy` para obter detalhes sobre o que *policy* controla.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Alterado na versão 3.6: O comportamento padrão dos parâmetros *mangle_from_* e *maxheaderlen* é seguir a política.

flatten(*msg*, *unixfrom=False*, *linesep=None*)

Exibe a representação textual da estrutura do objeto de mensagem com raiz em *msg* no arquivo de saída especificado quando a instância **`BytesGenerator`** foi criada.

Se a opção **`policy`** **`cte_type`** for `8bit` (o padrão), copia todos os cabeçalhos da mensagem original analisada que não tenham sido modificados para a saída, com todos os bytes com o bit mais alto definido reproduzidos como no original, e preserva a *Content-Transfer-Encoding* não ASCII de quaisquer partes do corpo que os contenham. Se `cte_type` for `7bit`, converte os bytes com o bit mais alto definido, conforme necessário, usando uma *Content-Transfer-Encoding* compatível com ASCII. Ou seja, transforma partes com *Content-Transfer-Encoding* não ASCII (*Content-Transfer-Encoding: 8bit*) em um *Content-Transfer-Encoding* compatível com ASCII e codifica bytes não ASCII inválidos para RFC em cabeçalhos usando o conjunto de caracteres MIME `unknown-8bit`, tornando-os compatíveis com RFC.

Se *unixfrom* for `True`, exibe o delimitador de cabeçalho de envelope usado pelo formato de caixa de correio Unix (consulte **`mailbox`**) antes do primeiro dos cabeçalhos **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] do objeto de mensagem raiz. Se o objeto raiz não tiver cabeçalho de envelope, crie um padrão. O padrão é `False`. Observe que, para subpartes, nenhum cabeçalho de envelope é exibido.

Se *linesep* não for `None`, usa-o como caractere separador entre todas as linhas da mensagem simplificada. Se *linesep* for `None` (o padrão), usa o valor especificado na *policy*.

clone(*fp*)

Retorna um clone independente desta instância **`BytesGenerator`** com exatamente as mesmas configurações de opção e *fp* como o novo *outfp*.

write(*s*)

Codifica *s* usando o codec `ASCII` e o tratador de erros `surrogateescape` e passa-o para o método *write* do *outfp* passado ao construtor **`BytesGenerator`**.

Para facilitar, **`EmailMessage`** fornece os métodos **`as_bytes()`** e `bytes(aMessage)` (também conhecidos como **`__bytes__()`**), que simplificam a geração de uma representação binária serializada de um objeto de mensagem. Para mais detalhes, consulte **`email.message`**.

Como strings não podem representar dados binários, a classe **`Generator`** deve converter quaisquer dados binários em qualquer mensagem que ela nivele para um formato compatível com ASCII, convertendo-os para um *Content-Transfer_Encoding* compatível com ASCII. Usando a terminologia dos RFCs de e-mail, você pode pensar nisso como **`Generator`** serializando para um fluxo de E/S que não é "limpo em 8 bits". Em outras palavras, a maioria das aplicações usará **`BytesGenerator`**, e não **`Generator`**.

class email.generator.Generator(*outfp*, *mangle_from_=None*, *maxheaderlen=None*, *, *policy=None*)

Retorna um objeto **`Generator`** que gravará qualquer mensagem fornecida ao método **`flatten()`**, ou qualquer texto fornecido ao método **`write()`**, no objeto arquivo ou similar *outfp*. *outfp* deve oferecer suporte a um método `write` que aceite dados string.

Se o *mangle_from_* opcional for `True`, coloca um caractere `>` na frente de qualquer linha no corpo que comece com a string exata `"From "`, ou seja, `From` seguido por um espaço no início de uma linha. *mangle_from_* assume por padrão o valor da configuração **`mangle_from_`** da *policy* (que é `True` para a política **`compat32`** e `False` para todas as outras). *mangle_from_* deve ser usado quando as mensagens são armazenadas no formato mbox do Unix (consulte **`mailbox`** e WHY THE CONTENT-LENGTH FORMAT IS BAD [https://www.jwz.org/doc/content-length.html]).

Se *maxheaderlen* não for `None`, redobra quaisquer linhas de cabeçalho maiores que *maxheaderlen* ou, se for `0`, não redobra nenhum cabeçalho. Se *manheaderlen* for `None` (o padrão), redobra os cabeçalhos e outras linhas de mensagem de acordo com as configurações da *policy*.

Se *policy* for especificado, usa essa política para controlar a geração de mensagens. Se *policy* for `None` (o padrão), usa a política associada ao objeto **`Message`** ou **`EmailMessage`** passado para `flatten` para controlar a geração de mensagens. Consulte **`email.policy`** para obter detalhes sobre o que *policy* controla.

Alterado na versão 3.3: Adicionada o argumento nomeado *policy*.

Alterado na versão 3.6: O comportamento padrão dos parâmetros *mangle_from_* e *maxheaderlen* é seguir a política.

flatten(*msg*, *unixfrom=False*, *linesep=None*)

Exibe a representação textual da estrutura do objeto de mensagem com raiz em *msg* no arquivo de saída especificado quando a instância **`Generator`** foi criada.

Se a opção **`policy`** **`cte_type`** for `8bit`, gera a mensagem como se a opção estivesse definida como `7bit`. (Isso é necessário porque strings não podem representar bytes não ASCII.) Converte quaisquer bytes com o bit mais alto definido, conforme necessário, usando uma *Content-Transfer-Encoding* compatível com ASCII. Ou seja, transforma partes com *Content-Transfer-Encoding* não ASCII (*Content-Transfer-Encoding: 8bit*) em um *Content-Transfer-Encoding* compatível com ASCII e codifica bytes não ASCII inválidos para RFC em cabeçalhos usando o conjunto de caracteres MIME `unknown-8bit`, tornando-os compatíveis com RFC.

Se *unixfrom* for `True`, exibe o delimitador de cabeçalho de envelope usado pelo formato de caixa de correio Unix (consulte **`mailbox`**) antes do primeiro dos cabeçalhos **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] do objeto de mensagem raiz. Se o objeto raiz não tiver cabeçalho de envelope, crie um padrão. O padrão é `False`. Observe que, para subpartes, nenhum cabeçalho de envelope é exibido.

Se *linesep* não for `None`, usa-o como caractere separador entre todas as linhas da mensagem simplificada. Se *linesep* for `None` (o padrão), usa o valor especificado na *policy*.

Alterado na versão 3.2: Adicionado suporte para recodificação de corpos de mensagens `8bit` e o argumento *linesep*.

clone(*fp*)

Retorna um clone independente desta instância **`Generator`** com exatamente as mesmas opções e *fp* como o novo *outfp*.

write(*s*)

Escreve *s* no método *write* do *outfp* passado ao construtor de **`Generator`**. Isso fornece API arquivo ou similar suficiente para que instâncias de **`Generator`** sejam usadas na função **`print()`**.

Para facilitar, **`EmailMessage`** fornece os métodos **`as_string()`** e `str(aMessage)` (também conhecidos como **`__str__()`**), que simplificam a geração de uma representação de string formatada de um objeto de mensagem. Para mais detalhes, consulte **`email.message`**.

O módulo **`email.generator`** também fornece uma classe derivada, **`DecodedGenerator`**, que é como a classe base **`Generator`**, exceto que as partes não-*text* não são serializadas, mas são representadas no fluxo de saída por uma string derivada de um modelo preenchido com informações sobre a parte.

class email.generator.DecodedGenerator(*outfp*, *mangle_from_=None*, *maxheaderlen=None*, *fmt=None*, ***, *policy=None*)

Age como **`Generator`**, exceto que para qualquer subparte da mensagem passada para **`Generator.flatten()`**, se a subparte for do tipo principal *text*, exibe a carga decodificada da subparte e, se o tipo principal não for *text*, em vez de exibi-la, preenche a string *fmt* usando informações da parte e exibe a string preenchida resultante.

Para preencher *fmt*, executa `fmt % part_info`, onde `part_info` é um dicionário composto pelas seguintes chaves e valores:

- `type` – Tipo MIME completo da parte não-*text*
- `maintype` – Tipo MIME principal da parte não-*text*
- `subtype` – Tipo sub-MIME da parte não-*text*

- `filename` – Nome de arquivo da parte não-*text*
- `description` – Descrição associada à parte não-*text*
- `encoding` – Codificação de transferência de conteúdo da parte não-*text*

Se *fmt* for `None`, usa o seguinte *fmt* padrão:

> "[Non-text (%(type)s) part of message omitted, filename %(filename)s]"

Os opcionais *_mangle_from_* e *maxheaderlen* são como os da classe base **`Generator`**.

Notas de rodapé

[1]

Esta instrução presume que você usa a configuração apropriada para `unixfrom` e que não haja configurações **`email.policy`** que exijam ajustes automáticos (por exemplo, **`refold_source`** deve ser `none`, que *não* é o padrão). Também não é 100% verdadeiro, pois, se a mensagem não estiver em conformidade com os padrões RFC, ocasionalmente informações sobre o texto original exato são perdidas durante a recuperação de erros de análise. O objetivo é corrigir esses últimos casos extremos sempre que possível.

`email.policy`: Policy Objects

Adicionado na versão 3.3.

Código-fonte: Lib/email/policy.py
[https://github.com/python/cpython/tree/3.15/Lib/email/policy.py]

The `email` package's prime focus is the handling of email messages as described by the various email and MIME RFCs. However, the general format of email messages (a block of header fields each consisting of a name followed by a colon followed by a value, the whole block followed by a blank line and an arbitrary 'body'), is a format that has found utility outside of the realm of email. Some of these uses conform fairly closely to the main email RFCs, some do not. Even when working with email, there are times when it is desirable to break strict compliance with the RFCs, such as generating emails that interoperate with email servers that do not themselves follow the standards, or that implement extensions you want to use in ways that violate the standards.

Policy objects give the email package the flexibility to handle all these disparate use cases.

A `Policy` object encapsulates a set of attributes and methods that control the behavior of various components of the email package during use. `Policy` instances can be passed to various classes and methods in the email package to alter the default behavior. The settable values and their defaults are described below.

There is a default policy used by all classes in the email package. For all of the `parser` classes and the related convenience functions, and for the `Message` class, this is the `Compat32` policy, via its corresponding pre-defined instance `compat32`. This policy provides for complete backward compatibility (in some cases, including bug compatibility) with the pre-Python3.3 version of the email package.

This default value for the *policy* keyword to **EmailMessage** is the **EmailPolicy** policy, via its pre-defined instance **default**.

When a **Message** or **EmailMessage** object is created, it acquires a policy. If the message is created by a **parser**, a policy passed to the parser will be the policy used by the message it creates. If the message is created by the program, then the policy can be specified when it is created. When a message is passed to a **generator**, the generator uses the policy from the message by default, but you can also pass a specific policy to the generator that will override the one stored on the message object.

The default value for the *policy* keyword for the **email.parser** classes and the parser convenience functions **will be changing** in a future version of Python. Therefore you should **always specify explicitly which policy you want to use** when calling any of the classes and functions described in the **parser** module.

The first part of this documentation covers the features of **Policy**, an abstract base class that defines the features that are common to all policy objects, including **compat32**. This includes certain hook methods that are called internally by the email package, which a custom policy could override to obtain different behavior. The second part describes the concrete classes **EmailPolicy** and **Compat32**, which implement the hooks that provide the standard behavior and the backward compatible behavior and features, respectively.

Policy instances are immutable, but they can be cloned, accepting the same keyword arguments as the class constructor and returning a new **Policy** instance that is a copy of the original but with the specified attributes values changed.

As an example, the following code could be used to read an email message from a file on disk and pass it to the system `sendmail` program on a Unix system:

```
>>> from email import message_from_binary_file
>>> from email.generator import BytesGenerator
>>> from email import policy
```

```
>>> from subprocess import Popen, PIPE
>>> with open('mymsg.txt', 'rb') as f:
...     msg = message_from_binary_file(f, policy=policy.default)
...
>>> p = Popen(['sendmail', msg['To'].addresses[0]], stdin=PIPE)
>>> g = BytesGenerator(p.stdin, policy=msg.policy.clone(linesep=
>>> g.flatten(msg)
>>> p.stdin.close()
>>> rc = p.wait()
```

Here we are telling `BytesGenerator` to use the RFC correct line separator characters when creating the binary string to feed into `sendmail's stdin`, where the default policy would use `\n` line separators.

Some email package methods accept a *policy* keyword argument, allowing the policy to be overridden for that method. For example, the following code uses the `as_bytes()` method of the *msg* object from the previous example and writes the message to a file using the native line separators for the platform on which it is running:

```
>>> import os
>>> with open('converted.txt', 'wb') as f:
...     f.write(msg.as_bytes(policy=msg.policy.clone(linesep=os.
17
```

Policy objects can also be combined using the addition operator, producing a policy object whose settings are a combination of the non-default values of the summed objects:

```
>>> compat_SMTP = policy.compat32.clone(linesep='\r\n')
>>> compat_strict = policy.compat32.clone(raise_on_defect=True)
>>> compat_strict_SMTP = compat_SMTP + compat_strict
```

This operation is not commutative; that is, the order in which the objects are added matters. To illustrate:

```
>>> policy100 = policy.compat32.clone(max_line_length=100)
>>> policy80 = policy.compat32.clone(max_line_length=80)
>>> apolicy = policy100 + policy80
```

```
>>> apolicy.max_line_length
80
>>> apolicy = policy80 + policy100
>>> apolicy.max_line_length
100
```

class email.policy.Policy(**kw*)

This is the abstract base class for all policy classes. It provides default implementations for a couple of trivial methods, as well as the implementation of the immutability property, the `clone()` method, and the constructor semantics.

The constructor of a policy class can be passed various keyword arguments. The arguments that may be specified are any non-method properties on this class, plus any additional non-method properties on the concrete class. A value specified in the constructor will override the default value for the corresponding attribute.

This class defines the following properties, and thus values for the following may be passed in the constructor of any policy class:

max_line_length

The maximum length of any line in the serialized output, not counting the end of line character(s). Default is 78, per **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html]. A value of `0` or `None` indicates that no line wrapping should be done at all.

linesep

The string to be used to terminate lines in serialized output. The default is `\n` because that's the internal end-of-line discipline used by Python, though `\r\n` is required by the RFCs.

cte_type

Controls the type of Content Transfer Encodings that may be or are required to be used. The possible values are:

`7bit` all data must be "7 bit clean" (ASCII-only). This means that where necessary data will be encoded using either quoted-

	printable or base64 encoding.
`8bit`	data is not constrained to be 7 bit clean. Data in headers is still required to be ASCII-only and so will be encoded (see **`fold_binary()`** and **`utf8`** below for exceptions), but body parts may use the `8bit` CTE.

A `cte_type` value of `8bit` only works with `BytesGenerator`, not `Generator`, because strings cannot contain binary data. If a `Generator` is operating under a policy that specifies `cte_type=8bit`, it will act as if `cte_type` is `7bit`.

raise_on_defect

If **`True`**, any defects encountered will be raised as errors. If **`False`** (the default), defects will be passed to the **`register_defect()`** method.

mangle_from_

If **`True`**, lines starting with *"From "* in the body are escaped by putting a `>` in front of them. This parameter is used when the message is being serialized by a generator. Default: **`False`**.

Adicionado na versão 3.5.

message_factory

A factory function for constructing a new empty message object. Used by the parser when building messages. Defaults to `None`, in which case **`Message`** is used.

Adicionado na versão 3.6.

verify_generated_headers

If `True` (the default), the generator will raise **`HeaderWriteError`** instead of writing a header that is improperly folded or delimited, such that it would be parsed as multiple headers or joined with adjacent data. Such headers can be generated by custom header classes or bugs in the `email` module.

As it's a security feature, this defaults to `True` even in the **`Compat32`** policy. For backwards compatible, but unsafe, behavior, it must be set to `False` explicitly.

Adicionado na versão 3.13.

The following **`Policy`** method is intended to be called by code using the email library to create policy instances with custom settings:

clone(**kw)

Return a new **`Policy`** instance whose attributes have the same values as the current instance, except where those attributes are given new values by the keyword arguments.

The remaining **`Policy`** methods are called by the email package code, and are not intended to be called by an application using the email package. A custom policy must implement all of these methods.

handle_defect(*obj*, *defect*)

Handle a *defect* found on *obj*. When the email package calls this method, *defect* will always be a subclass of **`MessageDefect`**.

The default implementation checks the **`raise_on_defect`** flag. If it is `True`, *defect* is raised as an exception. If it is `False` (the default), *obj* and *defect* are passed to **`register_defect()`**.

register_defect(*obj*, *defect*)

Register a *defect* on *obj*. In the email package, *defect* will always be a subclass of **`MessageDefect`**.

The default implementation calls the `append` method of the `defects` attribute of *obj*. When the email package calls **`handle_defect`**, *obj* will normally have a `defects` attribute that has an `append` method. Custom object types used with the email package (for example, custom `Message` objects) should also provide such an attribute, otherwise defects in parsed messages will raise unexpected errors.

header_max_count(*name*)

Return the maximum allowed number of headers named *name*.

Called when a header is added to an `EmailMessage` or `Message` object. If the returned value is not `0` or `None`, and there are already a number of headers with the name *name* greater than or equal to the value returned, a `ValueError` is raised.

Because the default behavior of `Message.__setitem__` is to append the value to the list of headers, it is easy to create duplicate headers without realizing it. This method allows certain headers to be limited in the number of instances of that header that may be added to a `Message` programmatically. (The limit is not observed by the parser, which will faithfully produce as many headers as exist in the message being parsed.)

The default implementation returns `None` for all header names.

header_source_parse(*sourcelines*)

The email package calls this method with a list of strings, each string ending with the line separation characters found in the source being parsed. The first line includes the field header name and separator. All whitespace in the source is preserved. The method should return the `(name, value)` tuple that is to be stored in the `Message` to represent the parsed header.

If an implementation wishes to retain compatibility with the existing email package policies, *name* should be the case preserved name (all characters up to the '`:`' separator), while *value* should be the unfolded value (all line separator characters removed, but whitespace kept intact), stripped of leading whitespace.

sourcelines may contain surrogateescaped binary data.

There is no default implementation

header_store_parse(*name*, *value*)

The email package calls this method with the name and value provided by the application program when the application program is modifying a `Message` programmatically (as opposed to a `Message` created by a parser). The method should return the `(name, value)` tuple that is to be stored in the `Message` to represent the header.

If an implementation wishes to retain compatibility with the existing email package policies, the *name* and *value* should be strings or string subclasses that do not change the content of the passed in arguments.

There is no default implementation

header_fetch_parse(*name*, *value*)

The email package calls this method with the *name* and *value* currently stored in the `Message` when that header is requested by the application program, and whatever the method returns is what is passed back to the application as the value of the header being retrieved. Note that there may be more than one header with the same name stored in the `Message`; the method is passed the specific name and value of the header destined to be returned to the application.

value may contain surrogateescaped binary data. There should be no surrogateescaped binary data in the value returned by the method.

There is no default implementation

fold(*name*, *value*)

The email package calls this method with the *name* and *value* currently stored in the `Message` for a given header. The method should return a string that represents that header "folded" correctly (according to the policy settings) by composing the *name* with the *value* and inserting **`linesep`** characters at the appropriate places. See **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] for a discussion of the rules for folding email headers.

value may contain surrogateescaped binary data. There should be no surrogateescaped binary data in the string returned by the method.

fold_binary(*name*, *value*)

The same as `fold()`, except that the returned value should be a bytes object rather than a string.

value may contain surrogateescaped binary data. These could be converted back into binary data in the returned bytes object.

class email.policy.EmailPolicy(**kw)

This concrete `Policy` provides behavior that is intended to be fully compliant with the current email RFCs. These include (but are not limited to) **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html], **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html], and the current MIME RFCs.

This policy adds new header parsing and folding algorithms. Instead of simple strings, headers are `str` subclasses with attributes that depend on the type of the field. The parsing and folding algorithm fully implement **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] and **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html].

The default value for the `message_factory` attribute is `EmailMessage`.

In addition to the settable attributes listed above that apply to all policies, this policy adds the following additional attributes:

Adicionado na versão 3.6: [[1]]

utf8

If `False`, follow **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html], supporting non-ASCII characters in headers by encoding them as **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] “encoded words”. If `True`, follow **RFC 6532** [https://datatracker.ietf.org/doc/html/rfc6532.html] and use `utf-8` encoding for headers. Messages formatted in this way may be passed to SMTP servers that support the `SMTPUTF8` extension (**RFC 6531** [https://datatracker.ietf.org/doc/html/rfc6531.html]).

When `False`, the generator will raise **`HeaderWriteError`** if any header includes non-ASCII characters in a context where **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] does not permit encoded words. This particularly applies to mailboxes (“addr-spec”) with non-ASCII characters, which can be created via **`Address`**. To use a mailbox with a non-ASCII domain name with `utf8=False`, first encode the domain using the third-party idna [https://pypi.org/project/idna/] or uts46 [https://pypi.org/project/uts46/] module or with **`encodings.idna`**. It is not possible to use a non-ASCII username (“local-part”) in a mailbox when `utf8=False`.

Alterado na versão 3.15: Can trigger the raising of **`HeaderWriteError`**. (Earlier versions incorrectly applied **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] in certain contexts, mostly notably in addr-specs.)

refold_source

If the value for a header in the `Message` object originated from a **`parser`** (as opposed to being set by a program), this attribute indicates whether or not a generator should refold that value when transforming the message back into serialized form. The possible values are:

`none`	all source values use original folding
`long`	source values that have any line that is longer than `max_line_length` will be refolded
`all`	todos os valores são redobrados.

O padrão é `long`.

header_factory

A callable that takes two arguments, `name` and `value`, where `name` is a header field name and `value` is an unfolded header field value, and returns a string subclass that represents that header. A default `header_factory` (see **`headerregistry`**) is provided that supports

custom parsing for the various address and date **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] header field types, and the major MIME header field stypes. Support for additional custom parsing will be added in the future.

content_manager

An object with at least two methods: get_content and set_content. When the `get_content()` or `set_content()` method of an `EmailMessage` object is called, it calls the corresponding method of this object, passing it the message object as its first argument, and any arguments or keywords that were passed to it as additional arguments. By default `content_manager` is set to `raw_data_manager`.

Adicionado na versão 3.4.

The class provides the following concrete implementations of the abstract methods of `Policy`:

header_max_count(*name*)

Returns the value of the `max_count` attribute of the specialized class used to represent the header with the given name.

header_source_parse(*sourcelines*)

The name is parsed as everything up to the '`:`' and returned unmodified. The value is determined by stripping leading whitespace off the remainder of the first line, joining all subsequent lines together, and stripping any trailing carriage return or linefeed characters.

header_store_parse(*name*, *value*)

The name is returned unchanged. If the input value has a `name` attribute and it matches *name* ignoring case, the value is returned unchanged. Otherwise the *name* and *value* are passed to `header_factory`, and the resulting header object is returned as the value. In this case a `ValueError` is raised if the input value contains CR or LF characters.

header_fetch_parse(*name*, *value*)

If the value has a `name` attribute, it is returned to unmodified. Otherwise the *name*, and the *value* with any CR or LF characters removed, are passed to the `header_factory`, and the resulting header object is returned. Any surrogateescaped bytes get turned into the Unicode unknown-character glyph.

fold(*name*, *value*)

Header folding is controlled by the **`refold_source`** policy setting. A value is considered to be a 'source value' if and only if it does not have a `name` attribute (having a `name` attribute means it is a header object of some sort). If a source value needs to be refolded according to the policy, it is converted into a header object by passing the *name* and the *value* with any CR and LF characters removed to the `header_factory`. Folding of a header object is done by calling its `fold` method with the current policy.

Source values are split into lines using **`splitlines()`**. If the value is not to be refolded, the lines are rejoined using the `linesep` from the policy and returned. The exception is lines containing non-ascii binary data. In that case the value is refolded regardless of the `refold_source` setting, which causes the binary data to be CTE encoded using the `unknown-8bit` charset.

fold_binary(*name*, *value*)

The same as **`fold()`** if **`cte_type`** is `7bit`, except that the returned value is bytes.

If **`cte_type`** is `8bit`, non-ASCII binary data is converted back into bytes. Headers with binary data are not refolded, regardless of the `refold_header` setting, since there is no way to know whether the binary data consists of single byte characters or multibyte characters.

The following instances of **`EmailPolicy`** provide defaults suitable for specific application domains. Note that in the future the behavior of these

instances (in particular the `HTTP` instance) may be adjusted to conform even more closely to the RFCs relevant to their domains.

email.policy.default

An instance of `EmailPolicy` with all defaults unchanged. This policy uses the standard Python `\n` line endings rather than the RFC-correct `\r\n`.

email.policy.SMTP

Suitable for serializing messages in conformance with the email RFCs. Like `default`, but with `linesep` set to `\r\n`, which is RFC compliant.

email.policy.SMTPUTF8

The same as `SMTP` except that **`utf8`** is `True`. Useful for serializing messages to a message store without using encoded words in the headers. Should only be used for SMTP transmission if the sender or recipient addresses have non-ASCII characters (the **`smtplib.SMTP.send_message()`** method handles this automatically).

email.policy.HTTP

Suitable for serializing headers with for use in HTTP traffic. Like `SMTP` except that `max_line_length` is set to `None` (unlimited).

email.policy.strict

Convenience instance. The same as `default` except that `raise_on_defect` is set to `True`. This allows any policy to be made strict by writing:

```
somepolicy + policy.strict
```

With all of these **`EmailPolicies`**, the effective API of the email package is changed from the Python 3.2 API in the following ways:

- Setting a header on a **`Message`** results in that header being parsed and a header object created.
- Fetching a header value from a **`Message`** results in that header being parsed and a header object created and returned.

- Any header object, or any header that is refolded due to the policy settings, is folded using an algorithm that fully implements the RFC folding algorithms, including knowing where encoded words are required and allowed.

From the application view, this means that any header obtained through the `EmailMessage` is a header object with extra attributes, whose string value is the fully decoded value of the header. Likewise, a header may be assigned a new value, or a new header created, using a string, and the policy will take care of converting the string into the correct RFC encoded form.

The header objects and their attributes are described in `headerregistry`.

class email.policy.Compat32(**kw*)

This concrete `Policy` is the backward compatibility policy. It replicates the behavior of the email package in Python 3.2. The `policy` module also defines an instance of this class, `compat32`, that is used as the default policy. Thus the default behavior of the email package is to maintain compatibility with Python 3.2.

The following attributes have values that are different from the `Policy` default:

mangle_from_

O padrão é `True`.

The class provides the following concrete implementations of the abstract methods of `Policy`:

header_source_parse(*sourcelines*)

The name is parsed as everything up to the '`:`' and returned unmodified. The value is determined by stripping leading whitespace off the remainder of the first line, joining all subsequent lines together, and stripping any trailing carriage return or linefeed characters.

header_store_parse(*name*, *value*)

The name and value are returned unmodified.

header_fetch_parse(*name*, *value*)

If the value contains binary data, it is converted into a **`Header`** object using the `unknown-8bit` charset. Otherwise it is returned unmodified.

fold(*name*, *value*)

Headers are folded using the **`Header`** folding algorithm, which preserves existing line breaks in the value, and wraps each resulting line to the `max_line_length`. Non-ASCII binary data are CTE encoded using the `unknown-8bit` charset.

fold_binary(*name*, *value*)

Headers are folded using the **`Header`** folding algorithm, which preserves existing line breaks in the value, and wraps each resulting line to the `max_line_length`. If `cte_type` is `7bit`, non-ascii binary data is CTE encoded using the `unknown-8bit` charset. Otherwise the original source header is used, with its existing line breaks and any (RFC invalid) binary data it may contain.

email.policy.compat32

An instance of **`Compat32`**, providing backward compatibility with the behavior of the email package in Python 3.2.

Nota

The **`compat32`** policy should not be used as a policy for **`EmailMessage`** objects, and should only be used to serialize messages that were created using the **`compat32`** policy.

Notas de rodapé

[1]

Originally added in 3.3 as a provisional feature.

`email.errors`: Classes de Exceção e Defeito.

Código-fonte: Lib/email/errors.py
[https://github.com/python/cpython/tree/3.15/Lib/email/errors.py]

The following exception classes are defined in the `email.errors` module:

exception email.errors.MessageError

Essa é a classe base para todas as exceções que o pacote `email` pode levantar. Ela é derivada da classe padrão `Exception` e não define métodos adicionais.

exception email.errors.MessageParseError

Essa é a classe base para exceções levantadas pela classe `Parser`. Ela é derivada do `MessageError`. Essa classe pode ser usada internamente pelo analisador sintático usado pelo `headerregistry`.

exception email.errors.HeaderParseError

Raised under some error conditions when parsing the **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] headers of a message, this class is derived from `MessageParseError`. The `set_boundary()` method will raise this error if the content type is unknown when the method is called. `Header` may raise this error for certain base64 decoding errors, and when an attempt is made to create a header that appears to contain an embedded header (that is, there is what is supposed to be a continuation line that has no leading whitespace and looks like a header).

exception email.errors.BoundaryError

Descontinuado e não mais usado.

exception email.errors.MultipartConversionError

Raised if the `attach()` method is called on an instance of a class derived from `MIMENonMultipart` (e.g. `MIMEImage`). `MultipartConversionError` multiply inherits from `MessageError` and the built-in `TypeError`.

exception email.errors.HeaderWriteError

Raised when an error occurs when the `generator` outputs headers.

exception email.errors.MessageDefect

This is the base class for all defects found when parsing email messages. It is derived from `ValueError`.

exception email.errors.HeaderDefect

This is the base class for all defects found when parsing email headers. It is derived from `MessageDefect`.

Here is the list of the defects that the `FeedParser` can find while parsing messages. Note that the defects are added to the message where the problem was found, so for example, if a message nested inside a *multipart/alternative* had a malformed header, that nested message object would have a defect, but the containing messages would not.

All defect classes are subclassed from `email.errors.MessageDefect`.

exception email.errors.NoBoundaryInMultipartDefect

A message claimed to be a multipart, but had no *boundary* parameter.

exception email.errors.StartBoundaryNotFoundDefect

The start boundary claimed in the *Content-Type* header was never found.

exception email.errors.CloseBoundaryNotFoundDefect

A start boundary was found, but no corresponding close boundary was ever found.

Adicionado na versão 3.3.

exception email.errors.FirstHeaderLineIsContinuationDefect

The message had a continuation line as its first header line.

exception email.errors.MisplacedEnvelopeHeaderDefect

A “Unix From” header was found in the middle of a header block.

exception email.errors.MissingHeaderBodySeparatorDefect

A line was found while parsing headers that had no leading white space but contained no ‘:’. Parsing continues assuming that the line represents the first line of the body.

Adicionado na versão 3.3.

exception email.errors.MalformedHeaderDefect

A header was found that was missing a colon, or was otherwise malformed.

Descontinuado desde a versão 3.3: This defect has not been used for several Python versions.

exception email.errors.MultipartInvariantViolationDefect

A message claimed to be a *multipart*, but no subparts were found. Note that when a message has this defect, its `is_multipart()` method may return `False` even though its content type claims to be *multipart*.

exception email.errors.InvalidBase64PaddingDefect

When decoding a block of base64 encoded bytes, the padding was not correct. Enough padding is added to perform the decode, but the resulting decoded bytes may be invalid.

exception email.errors.InvalidBase64CharactersDefect

When decoding a block of base64 encoded bytes, characters outside the base64 alphabet were encountered. The characters are ignored, but the resulting decoded bytes may be invalid.

exception email.errors.InvalidBase64LengthDefect

When decoding a block of base64 encoded bytes, the number of non-padding base64 characters was invalid (1 more than a multiple of 4).

The encoded block was kept as-is.

exception email.errors.InvalidDateDefect

When decoding an invalid or unparsable date field. The original value is kept as-is.

email.headerregistry: Custom Header Objects

Código-fonte: Lib/email/headerregistry.py
[https://github.com/python/cpython/tree/3.15/Lib/email/headerregistry.py]

Adicionado na versão 3.6: [[1]]

Headers are represented by customized subclasses of `str`. The particular class used to represent a given header is determined by the `header_factory` of the `policy` in effect when the headers are created. This section documents the particular `header_factory` implemented by the email package for handling **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] compliant email messages, which not only provides customized header objects for various header types, but also provides an extension mechanism for applications to add their own custom header types.

When using any of the policy objects derived from `EmailPolicy`, all headers are produced by `HeaderRegistry` and have `BaseHeader` as their last base class. Each header class has an additional base class that is determined by the type of the header. For example, many headers have the class `UnstructuredHeader` as their other base class. The specialized second class for a header is determined by the name of the header, using a lookup table stored in the `HeaderRegistry`. All of this is managed transparently for the typical application program, but interfaces are provided for modifying the default behavior for use by more complex applications.

The sections below first document the header base classes and their attributes, followed by the API for modifying the behavior of `HeaderRegistry`, and finally the support classes used to represent the data parsed from structured headers.

class email.headerregistry.BaseHeader(*name*, *value*)

name and *value* are passed to `BaseHeader` from the **`header_factory`** call. The string value of any header object is the *value* fully decoded to a string.

This base class defines the following read-only properties:

name

The name of the header (the portion of the field before the ':'). This is exactly the value passed in the **`header_factory`** call for *name*; that is, case is preserved.

defects

A tuple of **`HeaderDefect`** instances reporting any RFC compliance problems found during parsing. The email package tries to be complete about detecting compliance issues. See the **`errors`** module for a discussion of the types of defects that may be reported.

max_count

The maximum number of headers of this type that can have the same `name`. A value of `None` means unlimited. The `BaseHeader` value for this attribute is `None`; it is expected that specialized header classes will override this value as needed.

`BaseHeader` also provides the following method, which is called by the email library code and should not in general be called by application programs:

fold(*, *policy*)

Return a string containing **`linesep`** characters as required to correctly fold the header according to *policy*. A **`cte_type`** of `8bit` will be treated as if it were `7bit`, since headers may not contain arbitrary binary data. If **`utf8`** is `False`, non-ASCII data will be **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] encoded.

`BaseHeader` by itself cannot be used to create a header object. It defines a protocol that each specialized header cooperates with in order to produce the header object. Specifically, `BaseHeader` requires that the

specialized class provide a **classmethod()** named `parse`. This method is called as follows:

```
parse(string, kwds)
```

`kwds` is a dictionary containing one pre-initialized key, `defects`. `defects` is an empty list. The parse method should append any detected defects to this list. On return, the `kwds` dictionary *must* contain values for at least the keys `decoded`, `defects` and `parse_tree`. `decoded` should be the string value for the header (that is, the header value fully decoded to a string). `parse_tree` is set to the parse tree obtained from parsing the header. The parse method should assume that *string* may contain content-transfer-encoded parts, but should correctly handle all valid Unicode characters as well so that it can parse un-encoded header values.

`BaseHeader`'s `__new__` then creates the header instance, and calls its `init` method. The specialized class only needs to provide an `init` method if it wishes to set additional attributes beyond those provided by `BaseHeader` itself. Such an `init` method should look like this:

```
def init(self, /, *args, **kw):
    self._myattr = kw.pop('myattr')
    super().init(*args, **kw)
```

That is, anything extra that the specialized class puts in to the `kwds` dictionary should be removed and handled, and the remaining contents of `kw` (and `args`) passed to the `BaseHeader` `init` method.

class email.headerregistry.UnstructuredHeader

An "unstructured" header is the default type of header in **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html]. Any header that does not have a specified syntax is treated as unstructured. The classic example of an unstructured header is the *Subject* header.

In **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html], an unstructured header is a run of arbitrary text in the ASCII character set. **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html], however, has an **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] compatible mechanism for

encoding non-ASCII text as ASCII characters within a header value. When a *value* containing encoded words is passed to the constructor, the `UnstructuredHeader` parser converts such encoded words into a string, following the **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] rules for unstructured text. The parser uses heuristics to attempt to decode certain non-compliant encoded words. Defects are registered in such cases, as well as defects for issues such as invalid characters within the encoded words or the non-encoded text.

This header type provides no additional attributes.

class email.headerregistry.DateHeader

RFC 5322 [https://datatracker.ietf.org/doc/html/rfc5322.html] specifies a very specific format for dates within email headers. The `DateHeader` parser recognizes that date format, as well as recognizing a number of variant forms that are sometimes found "in the wild".

This header type provides the following additional attributes:

datetime

If the header value can be recognized as a valid date of one form or another, this attribute will contain a **`datetime`** instance representing that date. If the timezone of the input date is specified as `-0000` (indicating it is in UTC but contains no information about the source timezone), then **`datetime`** will be a naive **`datetime`**. If a specific timezone offset is found (including `+0000`), then **`datetime`** will contain an aware `datetime` that uses **`datetime.timezone`** to record the timezone offset.

The `decoded` value of the header is determined by formatting the `datetime` according to the **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] rules; that is, it is set to:

```
email.utils.format_datetime(self.datetime)
```

When creating a `DateHeader`, *value* may be **`datetime`** instance. This means, for example, that the following code is valid and does what one would expect:

```
msg['Date'] = datetime(2011, 7, 15, 21)
```

Because this is a naive `datetime` it will be interpreted as a UTC timestamp, and the resulting value will have a timezone of `-0000`. Much more useful is to use the **`localtime()`** function from the **`utils`** module:

```
msg['Date'] = utils.localtime()
```

This example sets the date header to the current time and date using the current timezone offset.

class email.headerregistry.AddressHeader

Address headers are one of the most complex structured header types. The `AddressHeader` class provides a generic interface to any address header.

This header type provides the following additional attributes:

groups

A tuple of **`Group`** objects encoding the addresses and groups found in the header value. Addresses that are not part of a group are represented in this list as single-address `Groups` whose **`display_name`** is `None`.

addresses

A tuple of **`Address`** objects encoding all of the individual addresses from the header value. If the header value contains any groups, the individual addresses from the group are included in the list at the point where the group occurs in the value (that is, the list of addresses is “flattened” into a one dimensional list).

The `decoded` value of the header will have all encoded words decoded to a string. **`idna`** encoded domain names are also decoded to a string. The `decoded` value is set by joining the **`str`** value of the elements of the `groups` attribute with `', '`.

A list of **`Address`** and **`Group`** objects in any combination may be used to set the value of an address header. `Group` objects whose `display_name`

is `None` will be interpreted as single addresses, which allows an address list to be copied with groups intact by using the list obtained from the `groups` attribute of the source header.

class email.headerregistry.SingleAddressHeader

Uma subclasse de **`AddressHeader`** que adiciona um atributo adicional:

address

The single address encoded by the header value. If the header value actually contains more than one address (which would be a violation of the RFC under the default **`policy`**), accessing this attribute will result in a **`ValueError`**.

Many of the above classes also have a `Unique` variant (for example, `UniqueUnstructuredHeader`). The only difference is that in the `Unique` variant, **`max_count`** is set to 1.

class email.headerregistry.MIMEVersionHeader

There is really only one valid value for the *MIME-Version* header, and that is `1.0`. For future proofing, this header class supports other valid version numbers. If a version number has a valid value per **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html], then the header object will have non-`None` values for the following attributes:

version

The version number as a string, with any whitespace and/or comments removed.

major

The major version number as an integer

minor

The minor version number as an integer

class email.headerregistry.ParameterizedMIMEHeader

MIME headers all start with the prefix ‘Content-’. Each specific header has a certain value, described under the class for that header. Some can

also take a list of supplemental parameters, which have a common format. This class serves as a base for all the MIME headers that take parameters.

params

A dictionary mapping parameter names to parameter values.

Alterado na versão 3.15: It is now a `frozendict` instead of a `types.MappingProxyType`.

class email.headerregistry.ContentTypeHeader

A `ParameterizedMIMEHeader` class that handles the *Content-Type* header.

content_type

The content type string, in the form `maintype/subtype`.

maintype

subtype

class email.headerregistry.ContentDispositionHeader

Uma classe `ParameterizedMIMEHeader` que lida com o cabeçalho *Content-Disposition*.

content_disposition

`inline` and `attachment` are the only valid values in common use.

class email.headerregistry.ContentTransferEncodingHeader

Handles the *Content-Transfer-Encoding* header.

cte

Valid values are `7bit`, `8bit`, `base64`, and `quoted-printable`. See **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] for more information.

class email.headerregistry.HeaderRegistry(*base_class=BaseHeader*, *default_class=UnstructuredHeader*, *use_default_map=True*)

This is the factory used by `EmailPolicy` by default. `HeaderRegistry` builds the class used to create a header instance dynamically, using *base_class* and a specialized class retrieved from a registry that it holds. When a given header name does not appear in the registry, the class specified by *default_class* is used as the specialized class. When *use_default_map* is `True` (the default), the standard mapping of header names to classes is copied in to the registry during initialization. *base_class* is always the last class in the generated class's `__bases__` list.

The default mappings are:

subject::	UniqueUnstructuredHeader
date::	UniqueDateHeader
resent-date::	DateHeader
orig-date::	UniqueDateHeader
sender::	UniqueSingleAddressHeader
resent-sender::	SingleAddressHeader
to::	UniqueAddressHeader
resent-to::	AddressHeader
cc::	UniqueAddressHeader
resent-cc::	AddressHeader
bcc::	UniqueAddressHeader
resent-bcc::	AddressHeader
from::	UniqueAddressHeader
resent-from::	AddressHeader
reply-to::	UniqueAddressHeader
mime-version::	MIMEVersionHeader
content-type::	ContentTypeHeader
content-disposition::	ContentDispositionHeader
content-transfer-encoding::	ContentTransferEncodingHeader
message-id::	MessageIDHeader

`HeaderRegistry` has the following methods:

map_to_type(*self*, *name*, *cls*)

name is the name of the header to be mapped. It will be converted to lower case in the registry. *cls* is the specialized class to be used, along with *base_class*, to create the class used to instantiate headers that match *name*.

__getitem__(*name*)

Construct and return a class to handle creating a *name* header.

__call__(*name*, *value*)

Retrieves the specialized header associated with *name* from the registry (using *default_class* if *name* does not appear in the registry) and composes it with *base_class* to produce a class, calls the constructed class's constructor, passing it the same argument list, and finally returns the class instance created thereby.

The following classes are the classes used to represent data parsed from structured headers and can, in general, be used by an application program to construct structured values to assign to specific headers.

class email.headerregistry.Address(*display_name=''*, *username=''*, *domain=''*, *addr_spec=None*)

The class used to represent an email address. The general form of an address is:

```
[display_name] <username@domain>
```

ou:

```
username@domain
```

where each part must conform to specific syntax rules spelled out in **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html].

As a convenience *addr_spec* can be specified instead of *username* and *domain*, in which case *username* and *domain* will be parsed from the

addr_spec. An *addr_spec* must be a properly RFC quoted string; if it is not `Address` will raise an error. Unicode characters are allowed and will be property encoded when serialized. However, per the RFCs, Unicode is *not* allowed in the username portion of the address.

display_name

The display name portion of the address, if any, with all quoting removed. If the address does not have a display name, this attribute will be an empty string.

username

The `username` portion of the address, with all quoting removed.

domain

The `domain` portion of the address.

addr_spec

The `username@domain` portion of the address, correctly quoted for use as a bare address (the second form shown above). This attribute is not mutable.

__str__()

The `str` value of the object is the address quoted according to **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] rules, but with no Content Transfer Encoding of any non-ASCII characters.

To support SMTP (**RFC 5321** [https://datatracker.ietf.org/doc/html/rfc5321.html]), `Address` handles one special case: if `username` and `domain` are both the empty string (or `None`), then the string value of the `Address` is `<>`.

class email.headerregistry.Group(*display_name=None*, *addresses=None*)

The class used to represent an address group. The general form of an address group is:

```
display_name: [address-list];
```

As a convenience for processing lists of addresses that consist of a mixture of groups and single addresses, a `Group` may also be used to represent single addresses that are not part of a group by setting *display_name* to `None` and providing a list of the single address as *addresses*.

display_name

The `display_name` of the group. If it is `None` and there is exactly one `Address` in `addresses`, then the `Group` represents a single address that is not in a group.

addresses

A possibly empty tuple of **`Address`** objects representing the addresses in the group.

__str__()

The `str` value of a `Group` is formatted according to **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html], but with no Content Transfer Encoding of any non-ASCII characters. If `display_name` is none and there is a single `Address` in the `addresses` list, the `str` value will be the same as the `str` of that single `Address`.

Notas de rodapé

[1]

Originally added in 3.3 as a provisional module

`email.contentmanager`: Gerenciando conteúdo MIME

Código-fonte: Lib/email/contentmanager.py
[https://github.com/python/cpython/tree/3.15/Lib/email/contentmanager.py]

Adicionado na versão 3.6: [[1]]

class email.contentmanager.ContentManager

Classe base para gerenciadores de conteúdo. Fornece os mecanismos de registro padrão para registrar conversores entre conteúdo MIME e outras representações, bem como os métodos de despacho `get_content` e `set_content`.

get_content(*msg*, **args*, ***kw*)

Procura uma função manipuladora baseada no `mimetype` de *msg* (veja o próximo parágrafo), chama-a, passando por todos os argumentos, e retorna o resultado da chamada. A expectativa é que a função manipuladora extraia o payload de *msg* e retorne um objeto que codifique informações sobre os dados extraídos.

Para encontrar o manipulador, procura as seguintes chaves no registro, parando na primeira encontrada:

- a string que representa o tipo MIME completo (`maintype/subtype`)
- the string representing the `maintype`
- the empty string

If none of these keys produce a handler, raise a `KeyError` for the full MIME type.

set_content(*msg*, *obj*, **args*, ***kw*)

If the `maintype` is `multipart`, raise a **`TypeError`**; otherwise look up a handler function based on the type of *obj* (see next paragraph), call **`clear_content()`** on the *msg*, and call the handler function, passing through all arguments. The expectation is that the handler will transform and store *obj* into *msg*, possibly making other changes to *msg* as well, such as adding various MIME headers to encode information needed to interpret the stored data.

To find the handler, obtain the type of *obj* (`typ = type(obj)`), and look for the following keys in the registry, stopping with the first one found:

- the type itself (`typ`)
- the type's fully qualified name (`typ.__module__ + '.' + typ.__qualname__`).
- the type's **`qualname`** (`typ.__qualname__`)
- the type's **`name`** (`typ.__name__`).

If none of the above match, repeat all of the checks above for each of the types in the MRO (**`typ.__mro__`**). Finally, if no other key yields a handler, check for a handler for the key `None`. If there is no handler for `None`, raise a **`KeyError`** for the fully qualified name of the type.

Also add a *MIME-Version* header if one is not present (see also **`MIMEPart`**).

add_get_handler(*key*, *handler*)

Record the function *handler* as the handler for *key*. For the possible values of *key*, see **`get_content()`**.

add_set_handler(*typekey*, *handler*)

Record *handler* as the function to call when an object of a type matching *typekey* is passed to **`set_content()`**. For the possible values of *typekey*, see **`set_content()`**.

Content Manager Instances

Currently the email package provides only one concrete content manager, `raw_data_manager`, although more may be added in the future. **`raw_data_manager`** is the `content_manager` provided by `EmailPolicy` and its derivatives.

email.contentmanager.raw_data_manager

This content manager provides only a minimum interface beyond that provided by `Message` itself: it deals only with text, raw bytes, and **`Message`** objects. Nevertheless, it provides significant advantages compared to the base API: `get_content` on a text part will return a string without the application needing to manually decode it, `set_content` provides a rich set of options for controlling the headers added to a part and controlling the content transfer encoding, and it enables the use of the various `add_` methods, thereby simplifying the creation of multipart messages.

email.contentmanager.get_content(*msg*, *errors='replace'*)

Return the payload of the part as either a string (for `text` parts), an `EmailMessage` object (for `message/rfc822` parts), or a `bytes` object (for all other non-multipart types). Raise a `KeyError` if called on a `multipart`. If the part is a `text` part and *errors* is specified, use it as the error handler when decoding the payload to a string. The default error handler is `replace`.

email.contentmanager.set_content(*msg*, *<'str'>*, *subtype="plain"*, *charset='utf-8'*, *cte=None*, *disposition=None*, *filename=None*, *cid=None*, *params=None*, *headers=None*)

email.contentmanager.set_content(*msg*, *<'bytes'>*, *maintype*, *subtype*, *cte="base64"*, *disposition=None*, *filename=None*, *cid=None*, *params=None*, *headers=None*)

email.contentmanager.set_content(*msg*, *<'EmailMessage'>*, *cte=None*, *disposition=None*, *filename=None*, *cid=None*, *params=None*, *headers=None*)

Adicione headers e payload à *msg*:

Add a *Content-Type* header with a `maintype/subtype` value.

- For `str`, set the MIME `maintype` to `text`, and set the subtype to *subtype* if it is specified, or `plain` if it is not.
- For `bytes`, use the specified *maintype* and *subtype*, or raise a **`TypeError`** if they are not specified.
- For **`EmailMessage`** objects, set the maintype to `message`, and set the subtype to *subtype* if it is specified or `rfc822` if it is not. If *subtype* is `partial`, raise an error (`bytes` objects must be used to construct `message/partial` parts).

If *charset* is provided (which is valid only for `str`), encode the string to bytes using the specified character set. The default is `utf-8`. If the specified *charset* is a known alias for a standard MIME charset name, use the standard charset instead.

If *cte* is set, encode the payload using the specified content transfer encoding, and set the *Content-Transfer-Encoding* header to that value. Possible values for *cte* are `quoted-printable`, `base64`, `7bit`, `8bit`, and `binary`. If the input cannot be encoded in the specified encoding (for example, specifying a *cte* of `7bit` for an input that contains non-ASCII values), raise a **`ValueError`**.

- For `str` objects, if *cte* is not set use heuristics to determine the most compact encoding. Prior to encoding, **`str.splitlines()`** is used to normalize all line boundaries, ensuring that each line of the payload is terminated by the current policy's **`linesep`** property (even if the original string did not end with one).
- For `bytes` objects, *cte* is taken to be base64 if not set, and the aforementioned newline translation is not performed.
- For **`EmailMessage`**, per **RFC 2046** [https://datatracker.ietf.org/doc/html/rfc2046.html], raise an error if a *cte* of `quoted-printable` or `base64` is requested for *subtype* `rfc822`, and for any *cte* other than `7bit` for *subtype* `external-body`. For `message/rfc822`, use `8bit` if *cte* is not specified. For all other values of *subtype*, use `7bit`.

Nota

A *cte* of `binary` does not actually work correctly yet. The `EmailMessage` object as modified by `set_content` is correct, but

`BytesGenerator` does not serialize it correctly.

If *disposition* is set, use it as the value of the *Content-Disposition* header. If not specified, and *filename* is specified, add the header with the value `attachment`. If *disposition* is not specified and *filename* is also not specified, do not add the header. The only valid values for *disposition* are `attachment` and `inline`.

If *filename* is specified, use it as the value of the `filename` parameter of the *Content-Disposition* header.

If *cid* is specified, add a *Content-ID* header with *cid* as its value.

If *params* is specified, iterate its `items` method and use the resulting `(key, value)` pairs to set additional parameters on the *Content-Type* header.

If *headers* is specified and is a list of strings of the form `headername: headervalue` or a list of `header` objects (distinguished from strings by having a `name` attribute), add the headers to *msg*.

Notas de rodapé

[1]

Originally added in 3.4 as a provisional module

email: Exemplos

Aqui estão alguns exemplos de como usar o pacote email para ler, escrever e enviar mensagens de e-mail simples, bem como mensagens MIME mais complexas.

Primeiro, vamos ver como criar e enviar uma mensagem de texto simples (tanto o conteúdo do texto quanto os endereços podem conter caracteres Unicode):

```
# Importa smtplib para a função de envio
import smtplib

# Importa os módulos de email que precisamos
from email.message import EmailMessage

# Abre o arquivo de texto simples cujo nome está em arquivo text
with open(textfile) as fp:
    # Create a text/plain message
    msg = EmailMessage()
    msg.set_content(fp.read())

# me == o endereço de e-mail do remetente
# you == o endereço de e-mail do destinatário
msg['Subject'] = f'The contents of {textfile}'
msg['From'] = me
msg['To'] = you

# Envia a mensagem via nosso próprio servidor SMTP.
s = smtplib.SMTP('localhost')
s.send_message(msg)
s.quit()
```

A análise dos cabeçalhos RFC 822 [https://datatracker.ietf.org/doc/html/rfc822.html] pode ser feita facilmente usando as classes do módulo parser:

```
# Importa os módulos de email que precisamos
#from email.parser import BytesParser
```

```
from email.parser import Parser
from email.policy import default

# Se os cabeçalhos de e-mal estão em um arquivo, descomente essa
# with open(messagefile, 'rb') as fp:
#     headers = BytesParser(policy=default).parse(fp)

#  Ou para análise de cabeçalhos em uma string (essa é uma opera
headers = Parser(policy=default).parsestr(
        'From: Foo Bar <user@example.com>\n'
        'To: <someone_else@example.com>\n'
        'Subject: Test message\n'
        '\n'
        'Body would go here\n')

#  Agora os itens de cabeçalho podem ser acessadas como um dicic
print('To: {}'.format(headers['to']))
print('From: {}'.format(headers['from']))
print('Subject: {}'.format(headers['subject']))

# Você também pode acessar as partes dos endereços:
print('Recipient username: {}'.format(headers['to'].addresses[0]
print('Sender name: {}'.format(headers['from'].addresses[0].disp
```

Aqui está um exemplo de como enviar uma mensagem MIME contendo várias fotos de família que podem estar em um diretório:

```
# Importa smtplib para a função de envio.
import smtplib

# Aqui estão os módulos de pacotes de e-mail que precisamos.
from email.message import EmailMessage

# Cria a contêiner da mensagem de e-mail.
msg = EmailMessage()
msg['Subject'] = 'Our family reunion'
# me == o endereço de e-mail do remetente
# family = a lista dos endereços de e-mail de todos os destinatá
msg['From'] = me
msg['To'] = ', '.join(family)
msg.preamble = 'You will not see this in a MIME-aware mail reade
```

```
# Abre os arquivos no modo binário. Você também pode omitir
# o subtipo se você quiser que MIMEImage adivinhe.
for file in pngfiles:
    with open(file, 'rb') as fp:
        img_data = fp.read()
    msg.add_attachment(img_data, maintype='image',
                               subtype='png')

# Envia o e-mail por meio de nosso próprio servidor SMTP.
with smtplib.SMTP('localhost') as s:
    s.send_message(msg)
```

Aqui está um exemplo de como enviar todo o conteúdo de um diretório como uma mensagem de e-mail: [[1]]

```
#!/usr/bin/env python3

"""Envia o conteúdo de um diretório como uma mensagem MIME."""

import os
import smtplib
# Para adivinhar o tipo MIME com base na extensão do nome do arc
import mimetypes

from argparse import ArgumentParser

from email.message import EmailMessage
from email.policy import SMTP


def main():
    parser = ArgumentParser(description="""\
Envia o conteúdo de um diretório como uma mensagem MIME.
A menos que a opção -o seja informada, o e-mail será enviado por
meio de encaminhamento para o seu servidor SMTP local, que então
realiza o processo de entrega normal. Sua máquina local deve est
executando um servidor SMTP.
""")
    parser.add_argument('-d', '--directory',
                        help="""Mail the contents of the specifi
                        otherwise use the current directory.  Or
                        files in the directory are sent, and we
```

```
                    subdirectories.""")
parser.add_argument('-o', '--output',
                    metavar='FILE',
                    help="""Print the composed message to FI
                    sending the message to the SMTP server.'
parser.add_argument('-s', '--sender', required=True,
                    help='The value of the From: header (rec
parser.add_argument('-r', '--recipient', required=True,
                    action='append', metavar='RECIPIENT',
                    default=[], dest='recipients',
                    help='A To: header value (at least one r
args = parser.parse_args()
directory = args.directory
if not directory:
    directory = '.'
# Cria a mensagem
msg = EmailMessage()
msg['Subject'] = f'Contents of directory {os.path.abspath(di
msg['To'] = ', '.join(args.recipients)
msg['From'] = args.sender
msg.preamble = 'You will not see this in a MIME-aware mail r

for filename in os.listdir(directory):
    path = os.path.join(directory, filename)
    if not os.path.isfile(path):
        continue
    # Adivinhe o tipo de conteúdo com base na extensão do ar
    # A codificação será ignorada, embora devamos verificar
    # simples como arquivos compactados ou compactados com g
    ctype, encoding = mimetypes.guess_file_type(path)
    if ctype is None or encoding is not None:
        # Não foi possível fazer nenhuma suposição, ou o arc
        # codificado (compactado), então use um tipo genéric
        # conjunto de bits.
        ctype = 'application/octet-stream'
    maintype, subtype = ctype.split('/', 1)
    with open(path, 'rb') as fp:
        msg.add_attachment(fp.read(),
                           maintype=maintype,
                           subtype=subtype,
                           filename=filename)
# Agora envia ou armazena a mensagem
if args.output:
```

```
        with open(args.output, 'wb') as fp:
            fp.write(msg.as_bytes(policy=SMTP))
    else:
        with smtplib.SMTP('localhost') as s:
            s.send_message(msg)


if __name__ == '__main__':
    main()
```

Aqui está um exemplo de como desempacotar uma mensagem MIME, como a acima, para um diretório de arquivos:

```
#!/usr/bin/env python3

"""Desempacota uma mensagem MIME em um diretório de arquivos."""

import os
import email
import mimetypes

from email.policy import default

from argparse import ArgumentParser


def main():
    parser = ArgumentParser(description="""\
Desempacota uma mensagem MIME em um diretório de arquivos.
""")
    parser.add_argument('-d', '--directory', required=True,
                        help="""Unpack the MIME message into the
                        directory, which will be created if it d
                        exist.""")
    parser.add_argument('msgfile')
    args = parser.parse_args()

    with open(args.msgfile, 'rb') as fp:
        msg = email.message_from_binary_file(fp, policy=default)

    try:
        os.mkdir(args.directory)
```

```
    except FileExistsError:
        pass

    counter = 1
    for part in msg.walk():
        # multipart/* são apenas contêineres
        if part.get_content_maintype() == 'multipart':
            continue
        # Os aplicativos realmente devem higienizar o nome de ar
        # fornecido para que uma mensagem de e-mail não possa se
        # usada para substituir arquivos importantes
        filename = part.get_filename()
        if not filename:
            ext = mimetypes.guess_extension(part.get_content_typ
            if not ext:
                # Usa uma extensão genérica de conjunto de bits
                ext = '.bin'
            filename = f'part-{counter:03d}{ext}'
        counter += 1
        with open(os.path.join(args.directory, filename), 'wb')
            fp.write(part.get_payload(decode=True))


if __name__ == '__main__':
    main()
```

Aqui está um exemplo de como criar uma mensagem HTML com uma versão alternativa em texto simples. Para tornar as coisas um pouco mais interessantes, incluímos uma imagem relacionada na parte html e salvamos uma cópia do que vamos enviar para o disco, assim como enviamos.

```
#!/usr/bin/env python3

import smtplib

from email.message import EmailMessage
from email.headerregistry import Address
from email.utils import make_msgid

# Cria a mensagem texto base.
msg = EmailMessage()
```

```
msg['Subject'] = "Pourquoi pas des asperges pour ce midi ?"
msg['From'] = Address("Pepé Le Pew", "pepe", "example.com")
msg['To'] = (Address("Penelope Pussycat", "penelope", "example.c
             Address("Fabrette Pussycat", "fabrette", "example.c
msg.set_content("""\
Salut!

Cette recette [1] sera sûrement un très bon repas.

[1] http://www.yummly.com/recipe/Roasted-Asparagus-Epicurious-20

--Pepé
""")

# Adiciona a versão HTML. Isso converte a mensagem em um contêir
# multiparte/alternativo, com a mensagem de texto original como
# parte e a nova mensagem HTML como a segunda parte.
asparagus_cid = make_msgid()
msg.add_alternative("""\
<html>
  <head></head>
  <body>
    <p>Salut!</p>
    <p>Cette
        <a href="http://www.yummly.com/recipe/Roasted-Asparagus-
            recette
        </a> sera sûrement un très bon repas.
    </p>
    <img src="cid:{asparagus_cid}">
  </body>
</html>
""".format(asparagus_cid=asparagus_cid[1:-1]), subtype='html')
# observe que precisamos remover o <> do msgid para usar no html

# Agora adiciona a imagem relacionada à parte html.
with open("roasted-asparagus.jpg", 'rb') as img:
    msg.get_payload()[1].add_related(img.read(), 'image', 'jpeg'
                                     cid=asparagus_cid)

# Agora adiciona a imagem relacionada à parte html.
with open('outgoing.msg', 'wb') as f:
    f.write(bytes(msg))
```

```
# Envia a mensagem via servidor SMTP local.
with smtplib.SMTP('localhost') as s:
    s.send_message(msg)
```

Se nos fosse enviada a mensagem do último exemplo, aqui está uma maneira de processá-la:

```
import os
import sys
import tempfile
import mimetypes
import webbrowser

# Importa os módulos de e-mail que precisaremos
from email import policy
from email.parser import BytesParser


def magic_html_parser(html_text, partfiles):
    """Retorna HTML com segurança e link para partfiles.

    Reescreve os atributos href="cid:...." para apontar aos arqu
    Embora não seja trivial, isso deve ser possível usando html.
    """
    raise NotImplementedError("Add the magic needed")


# Em um programa real, você pegaria o nome de arquivo a partir d
with open('outgoing.msg', 'rb') as fp:
    msg = BytesParser(policy=policy.default).parse(fp)

# Agora os itens de cabeçalho podem ser acessados como um dicior
# qualquer caractere não-ASCII será convertido para unicode:
print('To:', msg['to'])
print('From:', msg['from'])
print('Subject:', msg['subject'])

# Se quisermos imprimir uma prévia do conteúdo da mensagem, pode
# extrair o conteúdo menos formatado e imprimir as três primeira
# É claro que, se a mensagem não tiver uma parte de texto simple
# imprimir as três primeiras linhas de HTML provavelmente será i
# mas este é apenas um exemplo conceitual.
```

```
simplest = msg.get_body(preferencelist=('plain', 'html'))
print()
print(''.join(simplest.get_content().splitlines(keepends=True)[:

ans = input("View full message?")
if ans.lower()[0] == 'n':
    sys.exit()

# Podemos extrair a alternativa richest para exibi-la:
richest = msg.get_body()
partfiles = {}
if richest['content-type'].maintype == 'text':
    if richest['content-type'].subtype == 'plain':
        for line in richest.get_content().splitlines():
            print(line)
        sys.exit()
    elif richest['content-type'].subtype == 'html':
        body = richest
    else:
        print("Don't know how to display {}".format(richest.get_
        sys.exit()
elif richest['content-type'].content_type == 'multipart/related'
    body = richest.get_body(preferencelist=('html'))
    for part in richest.iter_attachments():
        fn = part.get_filename()
        if fn:
            extension = os.path.splitext(part.get_filename())[1]
        else:
            extension = mimetypes.guess_extension(part.get_conte
        with tempfile.NamedTemporaryFile(suffix=extension, delet
            f.write(part.get_content())
            # remove novamente o <> para ir da forma email de ci
            partfiles[part['content-id'][1:-1]] = f.name
else:
    print("Don't know how to display {}".format(richest.get_cont
    sys.exit()
with tempfile.NamedTemporaryFile(mode='w', delete=False) as f:
    f.write(magic_html_parser(body.get_content(), partfiles))
webbrowser.open(f.name)
os.remove(f.name)
for fn in partfiles.values():
    os.remove(fn)
```

```
# É claro que há muitas mensagens de e-mail que podem quebrar
# esse simples programa, mas ele lidará com as mais comuns.
```

Até o prompt, a saída do comando acima é:

```
To: Penelope Pussycat <penelope@example.com>, Fabrette Pussycat
From: Pepé Le Pew <pepe@example.com>
Subject: Pourquoi pas des asperges pour ce midi ?

Salut!

Cette recette [1] sera sûrement un très bon repas.
```

Notas de rodapé

[1]

Obrigado a Matthew Dixon Cowles pela inspiração original e pelos exemplos.

email.message.Message: Representing an email message using the compat32 API

The `Message` class is very similar to the `EmailMessage` class, without the methods added by that class, and with the default behavior of certain other methods being slightly different. We also document here some methods that, while supported by the `EmailMessage` class, are not recommended unless you are dealing with legacy code.

The philosophy and structure of the two classes is otherwise the same.

This document describes the behavior under the default (for `Message`) policy `Compat32`. If you are going to use another policy, you should be using the `EmailMessage` class instead.

An email message consists of *headers* and a *payload*. Headers must be **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] style names and values, where the field name and value are separated by a colon. The colon is not part of either the field name or the field value. The payload may be a simple text message, or a binary object, or a structured sequence of sub-messages each with their own set of headers and their own payload. The latter type of payload is indicated by the message having a MIME type such as *multipart/** or *message/rfc822*.

The conceptual model provided by a `Message` object is that of an ordered dictionary of headers with additional methods for accessing both specialized information from the headers, for accessing the payload, for generating a serialized version of the message, and for recursively walking over the object tree. Note that duplicate headers are supported but special methods must be used to access them.

The **Message** pseudo-dictionary is indexed by the header names, which must be ASCII values. The values of the dictionary are strings that are supposed to contain only ASCII characters; there is some special handling for non-ASCII input, but it doesn't always produce the correct results. Headers are stored and returned in case-preserving form, but field names are matched case-insensitively. There may also be a single envelope header, also known as the *Unix-From* header or the `From_` header. The *payload* is either a string or bytes, in the case of simple message objects, or a list of **Message** objects, for MIME container documents (e.g. *multipart/** and *message/rfc822*).

Here are the methods of the **Message** class:

class email.message.Message(*policy=compat32*)

If *policy* is specified (it must be an instance of a **policy** class) use the rules it specifies to update and serialize the representation of the message. If *policy* is not set, use the **compat32** policy, which maintains backward compatibility with the Python 3.2 version of the email package. For more information see the **policy** documentation.

Alterado na versão 3.3: The *policy* keyword argument was added.

as_string(*unixfrom=False, maxheaderlen=0, policy=None*)

Return the entire message flattened as a string. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to `False`. For backward compatibility reasons, *maxheaderlen* defaults to `0`, so if you want a different value you must override it explicitly (the value specified for *max_line_length* in the policy will be ignored by this method). The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the `Generator`.

Flattening the message may trigger changes to the **Message** if defaults need to be filled in to complete the transformation to a

string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not always format the message the way you want. For example, by default it does not do the mangling of lines that begin with `From` that is required by the Unix mbox format. For more flexibility, instantiate a **`Generator`** instance and use its **`flatten()`** method directly. For example:

```
from io import StringIO
from email.generator import Generator
fp = StringIO()
g = Generator(fp, mangle_from_=True, maxheaderlen=60)
g.flatten(msg)
text = fp.getvalue()
```

If the message object contains binary data that is not encoded according to RFC standards, the non-compliant data will be replaced by Unicode "unknown character" code points. (See also **`as_bytes()`** and **`BytesGenerator`**.)

Alterado na versão 3.4: the *policy* keyword argument was added.

__str__()

Equivalent to **`as_string()`**. Allows `str(msg)` to produce a string containing the formatted message.

as_bytes(*unixfrom=False*, *policy=None*)

Return the entire message flattened as a bytes object. When optional *unixfrom* is true, the envelope header is included in the returned string. *unixfrom* defaults to `False`. The *policy* argument may be used to override the default policy obtained from the message instance. This can be used to control some of the formatting produced by the method, since the specified *policy* will be passed to the `BytesGenerator`.

Flattening the message may trigger changes to the **Message** if defaults need to be filled in to complete the transformation to a string (for example, MIME boundaries may be generated or modified).

Note that this method is provided as a convenience and may not always format the message the way you want. For example, by default it does not do the mangling of lines that begin with `From` that is required by the Unix mbox format. For more flexibility, instantiate a **BytesGenerator** instance and use its **flatten()** method directly. For example:

```
from io import BytesIO
from email.generator import BytesGenerator
fp = BytesIO()
g = BytesGenerator(fp, mangle_from_=True, maxheaderlen=60
g.flatten(msg)
text = fp.getvalue()
```

Adicionado na versão 3.4.

__bytes__()

Equivalent to **as_bytes()**. Allows `bytes(msg)` to produce a bytes object containing the formatted message.

Adicionado na versão 3.4.

is_multipart()

Return `True` if the message's payload is a list of sub-**Message** objects, otherwise return `False`. When **is_multipart()** returns `False`, the payload should be a string object (which might be a CTE encoded binary payload). (Note that **is_multipart()** returning `True` does not necessarily mean that "msg.get_content_maintype() == 'multipart'" will return the `True`. For example, `is_multipart` will return `True` when the **Message** is of type `message/rfc822`.)

set_unixfrom(*unixfrom*)

Set the message's envelope header to *unixfrom*, which should be a string.

get_unixfrom()

Return the message's envelope header. Defaults to `None` if the envelope header was never set.

attach(*payload*)

Add the given *payload* to the current payload, which must be `None` or a list of **`Message`** objects before the call. After the call, the payload will always be a list of **`Message`** objects. If you want to set the payload to a scalar object (e.g. a string), use **`set_payload()`** instead.

This is a legacy method. On the **`EmailMessage`** class its functionality is replaced by **`set_content()`** and the related `make` and `add` methods.

get_payload(*i=None*, *decode=False*)

Return the current payload, which will be a list of **`Message`** objects when **`is_multipart()`** is `True`, or a string when **`is_multipart()`** is `False`. If the payload is a list and you mutate the list object, you modify the message's payload in place.

With optional argument *i*, **`get_payload()`** will return the *i*-th element of the payload, counting from zero, if **`is_multipart()`** is `True`. An **`IndexError`** will be raised if *i* is less than 0 or greater than or equal to the number of items in the payload. If the payload is a string (i.e. **`is_multipart()`** is `False`) and *i* is given, a **`TypeError`** is raised.

Optional *decode* is a flag indicating whether the payload should be decoded or not, according to the *Content-Transfer-Encoding* header. When `True` and the message is not a multipart, the payload will be decoded if this header's value is `quoted-printable` or `base64`. If some other encoding is used, or *Content-Transfer-Encoding* header is missing, the payload is returned as-is (undecoded). In all cases the

returned value is binary data. If the message is a multipart and the *decode* flag is `True`, then `None` is returned. If the payload is base64 and it was not perfectly formed (missing padding, characters outside the base64 alphabet), then an appropriate defect will be added to the message's defect property (**`InvalidBase64PaddingDefect`** or **`InvalidBase64CharactersDefect`**, respectively).

When *decode* is `False` (the default) the body is returned as a string without decoding the *Content-Transfer-Encoding*. However, for a *Content-Transfer-Encoding* of 8bit, an attempt is made to decode the original bytes using the `charset` specified by the *Content-Type* header, using the `replace` error handler. If no `charset` is specified, or if the `charset` given is not recognized by the email package, the body is decoded using the default ASCII charset.

This is a legacy method. On the **`EmailMessage`** class its functionality is replaced by **`get_content()`** and **`iter_parts()`**.

set_payload(*payload*, *charset=None*)

Set the entire message object's payload to *payload*. It is the client's responsibility to ensure the payload invariants. Optional *charset* sets the message's default character set; see **`set_charset()`** for details.

This is a legacy method. On the **`EmailMessage`** class its functionality is replaced by **`set_content()`**.

set_charset(*charset*)

Set the character set of the payload to *charset*, which can either be a **`Charset`** instance (see **`email.charset`**), a string naming a character set, or `None`. If it is a string, it will be converted to a **`Charset`** instance. If *charset* is `None`, the `charset` parameter will be removed from the *Content-Type* header (the message will not be otherwise modified). Anything else will generate a **`TypeError`**.

If there is no existing *MIME-Version* header one will be added. If there is no existing *Content-Type* header, one will be added with a value of *text/plain*. Whether the *Content-Type* header already exists

or not, its `charset` parameter will be set to *charset.output_charset*. If *charset.input_charset* and *charset.output_charset* differ, the payload will be re-encoded to the *output_charset*. If there is no existing *Content-Transfer-Encoding* header, then the payload will be transfer-encoded, if needed, using the specified **`Charset`**, and a header with the appropriate value will be added. If a *Content-Transfer-Encoding* header already exists, the payload is assumed to already be correctly encoded using that *Content-Transfer-Encoding* and is not modified.

This is a legacy method. On the **`EmailMessage`** class its functionality is replaced by the *charset* parameter of the **`email.message.EmailMessage.set_content()`** method.

get_charset()

Return the **`Charset`** instance associated with the message's payload.

This is a legacy method. On the **`EmailMessage`** class it always returns `None`.

The following methods implement a mapping-like interface for accessing the message's **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html] headers. Note that there are some semantic differences between these methods and a normal mapping (i.e. dictionary) interface. For example, in a dictionary there are no duplicate keys, but here there may be duplicate message headers. Also, in dictionaries there is no guaranteed order to the keys returned by **`keys()`**, but in a **`Message`** object, headers are always returned in the order they appeared in the original message, or were added to the message later. Any header deleted and then re-added are always appended to the end of the header list.

These semantic differences are intentional and are biased toward maximal convenience.

Note that in all cases, any envelope header present in the message is not included in the mapping interface.

In a model generated from bytes, any header values that (in contravention of the RFCs) contain non-ASCII bytes will, when retrieved through this interface, be represented as `Header` objects with a charset of `unknown-8bit`.

__len__()

Return the total number of headers, including duplicates.

__contains__(*name*)

Return `True` if the message object has a field named *name*. Matching is done case-insensitively and *name* should not include the trailing colon. Used for the `in` operator, e.g.:

```
if 'message-id' in myMessage:
   print('Message-ID:', myMessage['message-id'])
```

__getitem__(*name*)

Return the value of the named header field. *name* should not include the colon field separator. If the header is missing, `None` is returned; a `KeyError` is never raised.

Note that if the named field appears more than once in the message's headers, exactly which of those field values will be returned is undefined. Use the `get_all()` method to get the values of all the extant named headers.

__setitem__(*name*, *val*)

Add a header to the message with field name *name* and value *val*. The field is appended to the end of the message's existing fields.

Note that this does *not* overwrite or delete any existing header with the same name. If you want to ensure that the new header is the only one present in the message with field name *name*, delete the field first, e.g.:

```
del msg['subject']
msg['subject'] = 'Python roolz!'
```

__delitem__(*name*)

Delete all occurrences of the field with name *name* from the message's headers. No exception is raised if the named field isn't present in the headers.

keys()

Return a list of all the message's header field names.

values()

Return a list of all the message's field values.

items()

Return a list of 2-tuples containing all the message's field headers and values.

get(*name*, *failobj=None*)

Return the value of the named header field. This is identical to `__getitem__()` except that optional *failobj* is returned if the named header is missing (defaults to `None`).

Here are some additional useful methods:

get_all(*name*, *failobj=None*)

Return a list of all the values for the field named *name*. If there are no such named headers in the message, *failobj* is returned (defaults to `None`).

add_header(*_name*, *_value*, ***_params*)

Extended header setting. This method is similar to `__setitem__()` except that additional header parameters can be provided as keyword arguments. *_name* is the header field to add and *_value* is the *primary* value for the header.

For each item in the keyword argument dictionary *_params*, the key is taken as the parameter name, with underscores converted to dashes (since dashes are illegal in Python identifiers). Normally, the

parameter will be added as `key="value"` unless the value is `None`, in which case only the key will be added. If the value contains non-ASCII characters, it can be specified as a three tuple in the format `(CHARSET, LANGUAGE, VALUE)`, where `CHARSET` is a string naming the charset to be used to encode the value, `LANGUAGE` can usually be set to `None` or the empty string (see **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] for other possibilities), and `VALUE` is the string value containing non-ASCII code points. If a three tuple is not passed and the value contains non-ASCII characters, it is automatically encoded in **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] format using a `CHARSET` of `utf-8` and a `LANGUAGE` of `None`.

Aqui está um exemplo:

```
msg.add_header('Content-Disposition', 'attachment', filena
```

This will add a header that looks like

```
Content-Disposition: attachment; filename="bud.gif"
```

Um exemplo com caracteres não-ASCII:

```
msg.add_header('Content-Disposition', 'attachment',
               filename=('iso-8859-1', '', 'Fußballer.ppt
```

Que produz

```
Content-Disposition: attachment; filename*="iso-8859-1''Fu
```

replace_header(*_name*, *_value*)

Replace a header. Replace the first header found in the message that matches *_name*, retaining header order and field name case. If no matching header was found, a `KeyError` is raised.

get_content_type()

Return the message's content type. The returned string is coerced to lower case of the form *maintype/subtype*. If there was no *Content-Type* header in the message the default type as given by `get_default_type()` will be returned. Since according to **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html], messages always have a default type, `get_content_type()` will always return a value.

RFC 2045 [https://datatracker.ietf.org/doc/html/rfc2045.html] defines a message's default type to be *text/plain* unless it appears inside a *multipart/digest* container, in which case it would be *message/rfc822*. If the *Content-Type* header has an invalid type specification, **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] mandates that the default type be *text/plain*.

get_content_maintype()

Return the message's main content type. This is the *maintype* part of the string returned by `get_content_type()`.

get_content_subtype()

Return the message's sub-content type. This is the *subtype* part of the string returned by `get_content_type()`.

get_default_type()

Return the default content type. Most messages have a default content type of *text/plain*, except for messages that are subparts of *multipart/digest* containers. Such subparts have a default content type of *message/rfc822*.

set_default_type(*ctype*)

Set the default content type. *ctype* should either be *text/plain* or *message/rfc822*, although this is not enforced. The default content type is not stored in the *Content-Type* header.

get_params(*failobj=None*, *header='content-type'*, *unquote=True*)

Return the message's *Content-Type* parameters, as a list. The elements of the returned list are 2-tuples of key/value pairs, as split

on the `'='` sign. The left hand side of the `'='` is the key, while the right hand side is the value. If there is no `'='` sign in the parameter the value is the empty string, otherwise the value is as described in `get_param()` and is unquoted if optional *unquote* is `True` (the default).

Optional *failobj* is the object to return if there is no *Content-Type* header. Optional *header* is the header to search instead of *Content-Type*.

This is a legacy method. On the `EmailMessage` class its functionality is replaced by the *params* property of the individual header objects returned by the header access methods.

get_param(*param*, *failobj=None*, *header='content-type'*, *unquote=True*)

Return the value of the *Content-Type* header's parameter *param* as a string. If the message has no *Content-Type* header or if there is no such parameter, then *failobj* is returned (defaults to `None`).

Optional *header* if given, specifies the message header to use instead of *Content-Type*.

Parameter keys are always compared case insensitively. The return value can either be a string, or a 3-tuple if the parameter was **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] encoded. When it's a 3-tuple, the elements of the value are of the form `(CHARSET, LANGUAGE, VALUE)`. Note that both `CHARSET` and `LANGUAGE` can be `None`, in which case you should consider `VALUE` to be encoded in the `us-ascii` charset. You can usually ignore `LANGUAGE`.

If your application doesn't care whether the parameter was encoded as in **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html], you can collapse the parameter value by calling `email.utils.collapse_rfc2231_value()`, passing in the return value from **`get_param()`**. This will return a suitably decoded Unicode string when the value is a tuple, or the original string unquoted if it isn't. For example:

```
rawparam = msg.get_param('foo')
param = email.utils.collapse_rfc2231_value(rawparam)
```

In any case, the parameter value (either the returned string, or the `VALUE` item in the 3-tuple) is always unquoted, unless *unquote* is set to `False`.

This is a legacy method. On the **`EmailMessage`** class its functionality is replaced by the *params* property of the individual header objects returned by the header access methods.

set_param(*param*, *value*, *header='Content-Type'*, *requote=True*, *charset=None*, *language=''*, *replace=False*)

Set a parameter in the *Content-Type* header. If the parameter already exists in the header, its value will be replaced with *value*. If the *Content-Type* header as not yet been defined for this message, it will be set to *text/plain* and the new parameter value will be appended as per **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html].

Optional *header* specifies an alternative header to *Content-Type*, and all parameters will be quoted as necessary unless optional *requote* is `False` (the default is `True`).

If optional *charset* is specified, the parameter will be encoded according to **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html]. Optional *language* specifies the RFC 2231 language, defaulting to the empty string. Both *charset* and *language* should be strings.

If *replace* is `False` (the default) the header is moved to the end of the list of headers. If *replace* is `True`, the header will be updated in place.

Alterado na versão 3.4: Palavra-chave `replace` foi adicionada.

del_param(*param*, *header='content-type'*, *requote=True*)

Remove the given parameter completely from the *Content-Type* header. The header will be re-written in place without the parameter or its value. All values will be quoted as necessary unless *requote* is

`False` (the default is `True`). Optional *header* specifies an alternative to *Content-Type*.

set_type(*type*, *header='Content-Type'*, *requote=True*)

Set the main type and subtype for the *Content-Type* header. *type* must be a string in the form *maintype/subtype*, otherwise a **`ValueError`** is raised.

This method replaces the *Content-Type* header, keeping all the parameters in place. If *requote* is `False`, this leaves the existing header's quoting as is, otherwise the parameters will be quoted (the default).

An alternative header can be specified in the *header* argument. When the *Content-Type* header is set a *MIME-Version* header is also added.

This is a legacy method. On the **`EmailMessage`** class its functionality is replaced by the `make_` and `add_` methods.

get_filename(*failobj=None*)

Return the value of the `filename` parameter of the *Content-Disposition* header of the message. If the header does not have a `filename` parameter, this method falls back to looking for the `name` parameter on the *Content-Type* header. If neither is found, or the header is missing, then *failobj* is returned. The returned string will always be unquoted as per **`email.utils.unquote()`**.

get_boundary(*failobj=None*)

Return the value of the `boundary` parameter of the *Content-Type* header of the message, or *failobj* if either the header is missing, or has no `boundary` parameter. The returned string will always be unquoted as per **`email.utils.unquote()`**.

set_boundary(*boundary*)

Set the `boundary` parameter of the *Content-Type* header to *boundary*. **`set_boundary()`** will always quote *boundary* if

necessary. A `HeaderParseError` is raised if the message object has no *Content-Type* header.

Note that using this method is subtly different than deleting the old *Content-Type* header and adding a new one with the new boundary via `add_header()`, because `set_boundary()` preserves the order of the *Content-Type* header in the list of headers. However, it does *not* preserve any continuation lines which may have been present in the original *Content-Type* header.

get_content_charset(*failobj=None*)

Return the `charset` parameter of the *Content-Type* header, coerced to lower case. If there is no *Content-Type* header, or if that header has no `charset` parameter, *failobj* is returned.

Note that this method differs from `get_charset()` which returns the `Charset` instance for the default encoding of the message body.

get_charsets(*failobj=None*)

Return a list containing the character set names in the message. If the message is a *multipart*, then the list will contain one element for each subpart in the payload, otherwise, it will be a list of length 1.

Each item in the list will be a string which is the value of the `charset` parameter in the *Content-Type* header for the represented subpart. However, if the subpart has no *Content-Type* header, no `charset` parameter, or is not of the *text* main MIME type, then that item in the returned list will be *failobj*.

get_content_disposition()

Return the lowercased value (without parameters) of the message's *Content-Disposition* header if it has one, or `None`. The possible values for this method are *inline*, *attachment* or `None` if the message follows **RFC 2183** [https://datatracker.ietf.org/doc/html/rfc2183.html].

Adicionado na versão 3.5.

walk()

The **`walk()`** method is an all-purpose generator which can be used to iterate over all the parts and subparts of a message object tree, in depth-first traversal order. You will typically use **`walk()`** as the iterator in a `for` loop; each iteration returns the next subpart.

Here's an example that prints the MIME type of every part of a multipart message structure:

```
>>> for part in msg.walk():
...     print(part.get_content_type())
multipart/report
text/plain
message/delivery-status
text/plain
text/plain
message/rfc822
text/plain
```

`walk` iterates over the subparts of any part where **`is_multipart()`** returns `True`, even though `msg.get_content_maintype() == 'multipart'` may return `False`. We can see this in our example by making use of the `_structure` debug helper function:

```
>>> for part in msg.walk():
...     print(part.get_content_maintype() == 'multipart',
...           part.is_multipart())
True True
False False
False True
False False
False False
False True
False False
>>> _structure(msg)
multipart/report
    text/plain
    message/delivery-status
        text/plain
        text/plain
```

```
message/rfc822
    text/plain
```

Here the `message` parts are not `multiparts`, but they do contain subparts. `is_multipart()` returns `True` and `walk` descends into the subparts.

`Message` objects can also optionally contain two instance attributes, which can be used when generating the plain text of a MIME message.

preamble

The format of a MIME document allows for some text between the blank line following the headers, and the first multipart boundary string. Normally, this text is never visible in a MIME-aware mail reader because it falls outside the standard MIME armor. However, when viewing the raw text of the message, or when viewing the message in a non-MIME aware reader, this text can become visible.

The *preamble* attribute contains this leading extra-armor text for MIME documents. When the **`Parser`** discovers some text after the headers but before the first boundary string, it assigns this text to the message's *preamble* attribute. When the **`Generator`** is writing out the plain text representation of a MIME message, and it finds the message has a *preamble* attribute, it will write this text in the area between the headers and the first boundary. See **`email.parser`** and **`email.generator`** for details.

Note that if the message object has no preamble, the *preamble* attribute will be `None`.

epilogue

The *epilogue* attribute acts the same way as the *preamble* attribute, except that it contains text that appears between the last boundary and the end of the message.

You do not need to set the epilogue to the empty string in order for the **`Generator`** to print a newline at the end of the file.

defects

The *defects* attribute contains a list of all the problems found when parsing this message. See `email.errors` for a detailed description of the possible parsing defects.

`email.mimeemail.mime`: Criando e-mail e objetos MIME do zero

Código-fonte: Lib/email/mime/
[https://github.com/python/cpython/tree/3.15/Lib/email/mime/]

Este módulo faz parte da API de e-mail legada (`Compat32`). Sua funcionalidade é parcialmente substituída por **`contentmanager`** na nova API, mas em certos aplicativos essas classes ainda podem ser úteis, mesmo em código não legado.

Normalmente, você obtém uma estrutura de objeto de mensagem passando um arquivo ou algum texto para um analisador sintático, que analisa o texto e retorna o objeto de mensagem raiz. No entanto, você também pode criar uma estrutura de mensagem completa do zero, ou até objetos individuais de **`Message`** manualmente. De fato, você também pode pegar uma estrutura existente e adicionar novos objetos **`Message`**, movê-los, etc. Isso cria uma interface muito conveniente para fatiar e cortar dados de mensagens MIME.

Você pode criar uma nova estrutura de objeto criando instâncias de **`Message`**, adicionando anexos e todos os cabeçalhos apropriados manualmente. Porém, para mensagens MIME, o pacote **`email`** fornece algumas subclasses convenientes para facilitar as coisas.

Arquivo estão as classes:

class email.mime.base.MIMEBase(*_maintype*, *_subtype*, *, *policy=compat32*, ***_params*)

Módulo: **`email.mime.base`**

Esta é a classe base para todas as subclasses específicas de MIME de **`Message`**. Normalmente você não criará instâncias especificamente de **`MIMEBase`**, embora possa. A **`MIMEBase`** é fornecida principalmente como

uma classe base conveniente para subclasses mais específicas para MIME.

_maintype é o tipo principal de *Content-Type* (ex., *text* ou *image*) e *_subtype* é o tipo principal de *Content-Type* (ex., *plain* ou *gif*). *_params* é um dicionário de parâmetros chave/valor e é passado diretamente para `Message.add_header`.

Se *policy* for especificado, (o padrão é a política `compat32`) será passado para `Message`.

A classe `MIMEBase` sempre adiciona um cabeçalho *Content-Type* (com base em *_maintype*, *_subtype* e *_params*) e um cabeçalho *MIME-Version* (sempre definido como `1.0`).

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.nonmultipart.MIMENonMultipart

Módulo: `email.mime.nonmultipart`

Uma subclasse de `MIMEBase`, esta é uma classe base intermediária para mensagens MIME que não são *multipart*. O principal objetivo desta classe é impedir o uso do método `attach()`, que só faz sentido para mensagens *multipart*. Se **`attach()`** for chamado, uma exceção `MultipartConversionError` será levantada.

class email.mime.multipart.MIMEMultipart(*_subtype='mixed', boundary=None, _subparts=None, *, policy=compat32, **_params*)

Módulo: `email.mime.multipart`

Uma subclasse de `MIMEBase`, esta é uma classe base intermediária para mensagens MIME que são *multipart*. O *_subtype* opcional é padronizado como *mixed*, mas pode ser usado para especificar o subtipo da mensagem. Um cabeçalho *Content-Type* de *multipart/_subtype* será adicionado ao objeto da mensagem. Um cabeçalho *MIME-Version* também será adicionado.

O *boundary* opcional é a string de limites de várias partes. Quando `None` (o padrão), o limite é calculado quando necessário (por exemplo, quando a mensagem é serializada).

_subparts é uma sequência de subpartes iniciais para a carga. Deve ser possível converter essa sequência em uma lista. Você sempre pode anexar novas subpartes à mensagem usando o método `Message.attach`.

O argumento opcional *policy* tem como padrão `compat32`.

Parâmetros adicionais para o cabeçalho *Content-Type* são retirados dos argumentos nomeados ou passados para o argumento *_params*, que é um dicionário.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.application.MIMEApplication(*_data*, *_subtype='octet-stream'*, *_encoder=email.encoders.encode_base64*, **, policy=compat32*, ***_params*)

Módulo: `email.mime.application`

Uma subclasse de `MIMENonMultipart`, a classe `MIMEApplication` é usada para representar objetos de mensagem MIME do tipo principal *application*. *_data* contém os bytes para os dados brutos da aplicação. O *_subtype* opcional especifica o subtipo MIME e o padrão é *octet-stream*.

O *_encoder* opcional é um chamável (isto é, função) que executará a codificação real dos dados para transporte. Esse chamável requer um argumento, que é a instância `MIMEApplication`. Ele deve usar `get_payload()` e `set_payload()` para alterar a carga útil para o formulário codificado. Também deve adicionar *Content-Transfer-Encoding* ou outros cabeçalhos ao objeto de mensagem, conforme necessário. A codificação padrão é base64. Veja o módulo `email.encoders` para obter uma lista dos codificadores embutidos.

O argumento opcional *policy* tem como padrão `compat32`.

_params são passados diretamente para o construtor da classe base.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.audio.MIMEAudio(*_audiodata*, *_subtype=None*, *_encoder=email.encoders.encode_base64*, *, *policy=compat32*, ***_params*)

Módulo: `email.mime.audio`

Uma subclasse de `MIMENonMultipart`, a classe `MIMEAudio` é usada para criar objetos de mensagem MIME do tipo principal *audio*. *_audiodata* contém os bytes para os dados brutos de áudio. Se esses dados puderem ser decodificados como au, wav, aiff ou aifc, então o subtipo será automaticamente incluído no cabeçalho *Content-Type*. Caso contrário, você pode especificar explicitamente o subtipo de áudio por meio do argumento *_subtype*. Se o tipo menor não pôde ser adivinhado e *_subtype* não foi fornecido, então um exceção `TypeError` é levantada.

O *_encoder* opcional é um chamável (ou seja, função) que executará a codificação real dos dados de áudio para transporte. Esse chamável requer um argumento, que é a instância `MIMEAudio`. Ele deve usar `get_payload()` e `set_payload()` para alterar a carga útil para a forma codificada. Também deve adicionar *Content-Transfer-Encoding* ou outros cabeçalhos ao objeto de mensagem, conforme necessário. A codificação padrão é base64. Veja o módulo `email.encoders` para obter uma lista dos codificadores embutidos.

O argumento opcional *policy* tem como padrão `compat32`.

_params são passados diretamente para o construtor da classe base.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.image.MIMEImage(*_imagedata*, *_subtype=None*, *_encoder=email.encoders.encode_base64*, *, *policy=compat32*,

**_*params*)

Módulo: `email.mime.image`

Uma subclasse de `MIMENonMultipart`, a classe `MIMEImage` é usada para criar objetos de mensagem MIME do tipo principal *image*. *_imagedata* contém os bytes para os dados de imagem bruta. Se esse tipo de dados puder ser detectado (jpeg, png, gif, tiff, rgb, pbm, pgm, ppm, rast, xbm, bmp, webp e exr tentados), então o subtipo será automaticamente incluído no cabeçalho *Content-Type*. Caso contrário, você pode especificar explicitamente o subtipo de imagem por meio do argumento *_subtype*. Se o tipo secundário não puder ser adivinhado e *_subtype* não for fornecido, então `TypeError` será levantada.

O *_encoder* opcional é um chamável (ou seja, função) que executará a codificação real dos dados de imagem para transporte. Esse chamável requer um argumento, que é a instância `MIMEImage`. Ele deve usar `get_payload()` e `set_payload()` para alterar a carga útil para a forma codificada. Também deve adicionar *Content-Transfer-Encoding* ou outros cabeçalhos ao objeto de mensagem, conforme necessário. A codificação padrão é base64. Veja o módulo `email.encoders` para obter uma lista dos codificadores embutidos.

O argumento opcional *policy* tem como padrão `compat32`.

_params são passados diretamente para o construtor `MIMEBase`.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.message.MIMEMessage(*_msg*, *_subtype='rfc822'*, *, *policy=compat32*)

Módulo: `email.mime.message`

Uma subclasse de `MIMENonMultipart`, a classe `MIMEMessage` é usada para criar objetos MIME do tipo principal *message*. *_msg* é usado como carga útil e deve ser uma instância da classe `Message` (ou uma subclasse dela), caso contrário, uma `TypeError` é levantada.

O opcional *_subtype* define o subtipo da mensagem; o padrão é *rfc822*.

O argumento opcional *policy* tem como padrão `compat32`.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

class email.mime.text.MIMEText(*_text*, *_subtype='plain'*, *_charset=None*, **, policy=compat32*)

Módulo: `email.mime.text`

Uma subclasse de `MIMENonMultipart`, a classe `MIMEText` é usada para criar objetos MIME do tipo principal *text*. *_text* é a string para a carga útil. *_subtype* é o tipo secundário e o padrão é *plain*. *_charset* é o conjunto de caracteres do texto e é passado como um argumento para o construtor `MIMENonMultipart`; o padrão é `us-ascii` se a string contiver apenas pontos de código `ascii` e `utf-8` caso contrário. O parâmetro *_charset* aceita uma string ou uma instância `Charset`.

A menos que o argumento *_charset* seja explicitamente definido como `None`, o objeto MIMEText criado terá um cabeçalho *Content-Type* com um parâmetro `charset` e um cabeçalho *Content-Transfer-Encoding*. Isso significa que uma chamada `set_payload` subsequente não resultará em uma carga útil codificada, mesmo se um charset for passado no comando `set_payload`. Você pode “redefinir” esse comportamento excluindo o cabeçalho `Content-Transfer-Encoding`, após o qual uma chamada `set_payload` codificará automaticamente a nova carga útil (e adicionará um novo cabeçalho *Content-Transfer-Encoding*).

O argumento opcional *policy* tem como padrão `compat32`.

Alterado na versão 3.5: *_charset* também aceita instâncias de `Charset`.

Alterado na versão 3.6: Adicionado o parâmetro somente-nomeado *policy*.

`email.header`: Internationalized headers

Código-fonte: Lib/email/header.py
[https://github.com/python/cpython/tree/3.15/Lib/email/header.py]

This module is part of the legacy (`Compat32`) email API. In the current API encoding and decoding of headers is handled transparently by the dictionary-like API of the **`EmailMessage`** class. In addition to uses in legacy code, this module can be useful in applications that need to completely control the character sets used when encoding headers.

O texto restante nesta seção é a documentação original do módulo.

RFC 2822 [https://datatracker.ietf.org/doc/html/rfc2822.html] is the base standard that describes the format of email messages. It derives from the older **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] standard which came into widespread use at a time when most email was composed of ASCII characters only. **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html] is a specification written assuming email contains only 7-bit ASCII characters.

Of course, as email has been deployed worldwide, it has become internationalized, such that language specific character sets can now be used in email messages. The base standard still requires email messages to be transferred using only 7-bit ASCII characters, so a slew of RFCs have been written describing how to encode email containing non-ASCII characters into **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]-compliant format. These RFCs include **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html], **RFC 2046** [https://datatracker.ietf.org/doc/html/rfc2046.html], **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html], and **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html]. The **`email`** package supports these standards in its **`email.header`** and **`email.charset`** modules.

If you want to include non-ASCII characters in your email headers, say in the *Subject* or *To* fields, you should use the **Header** class and assign the field in the **Message** object to an instance of **Header** instead of using a string for the header value. Import the **Header** class from the **email.header** module. For example:

```
>>> from email.message import Message
>>> from email.header import Header
>>> msg = Message()
>>> h = Header('p\xf6stal', 'iso-8859-1')
>>> msg['Subject'] = h
>>> msg.as_string()
'Subject: =?iso-8859-1?q?p=F6stal?=\n\n'
```

Notice here how we wanted the *Subject* field to contain a non-ASCII character? We did this by creating a **Header** instance and passing in the character set to use when encoding it. When the subsequent **Message** instance was flattened, the *Subject* field was properly **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] encoded. MIME-aware mail readers would show this header using the embedded ISO-8859-1 character.

Here is the **Header** class description:

class email.header.Header(*s=None*, *charset=None*, *maxlinelen=None*, *header_name=None*, *continuation_ws=' '*, *errors='strict'*)

Create a MIME-compliant header that can contain strings in different character sets.

Optional *s* is the initial header value. If `None` (the default), the initial header value is not set. You can later append to the header with **append()** method calls. *s* may be an instance of **bytes** or **str**, but see the **append()** documentation for semantics.

Optional *charset* serves two purposes: it has the same meaning as the *charset* argument to the **append()** method. It also sets the default character set for all subsequent **append()** calls that omit the *charset* argument. If *charset* is not provided in the constructor (the default), the

`us-ascii` character set is used both as *s*'s initial charset and as the default for subsequent **`append()`** calls.

The maximum line length can be specified explicitly via *maxlinelen*. For splitting the first line to a shorter value (to account for the field header which isn't included in *s*, e.g. *Subject*) pass in the name of the field in *header_name*. The default *maxlinelen* is 78, and the default value for *header_name* is `None`, meaning it is not taken into account for the first line of a long, split header.

Optional *continuation_ws* must be **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]-compliant folding whitespace, and is usually either a space or a hard tab character. This character will be prepended to continuation lines. *continuation_ws* defaults to a single space character.

Optional *errors* is passed straight through to the **`append()`** method.

append(*s*, *charset=None*, *errors='strict'*)

Append the string *s* to the MIME header.

Optional *charset*, if given, should be a **`Charset`** instance (see **`email.charset`**) or the name of a character set, which will be converted to a **`Charset`** instance. A value of `None` (the default) means that the *charset* given in the constructor is used.

s may be an instance of **`bytes`** or **`str`**. If it is an instance of **`bytes`**, then *charset* is the encoding of that byte string, and a **`UnicodeError`** will be raised if the string cannot be decoded with that character set.

If *s* is an instance of **`str`**, then *charset* is a hint specifying the character set of the characters in the string.

In either case, when producing an **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]-compliant header using **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] rules, the string will be encoded using the output codec of the charset. If the string cannot be encoded using the output codec, a UnicodeError will be raised.

Optional *errors* is passed as the errors argument to the decode call if *s* is a byte string.

encode(*splitchars=';, \t', maxlinelen=None, linesep='\n'*)

Encode a message header into an RFC-compliant format, possibly wrapping long lines and encapsulating non-ASCII parts in base64 or quoted-printable encodings.

Optional *splitchars* is a string containing characters which should be given extra weight by the splitting algorithm during normal header wrapping. This is in very rough support of **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]'s 'higher level syntactic breaks': split points preceded by a splitchar are preferred during line splitting, with the characters preferred in the order in which they appear in the string. Space and tab may be included in the string to indicate whether preference should be given to one over the other as a split point when other split chars do not appear in the line being split. Splitchars does not affect **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] encoded lines.

maxlinelen, if given, overrides the instance's value for the maximum line length.

linesep specifies the characters used to separate the lines of the folded header. It defaults to the most useful value for Python application code (`\n`), but `\r\n` can be specified in order to produce headers with RFC-compliant line separators.

Alterado na versão 3.2: Adicionado o argumento *linesep*.

The **`Header`** class also provides a number of methods to support standard operators and built-in functions.

__str__()

Returns an approximation of the **`Header`** as a string, using an unlimited line length. All pieces are decoded using the specified encoding and joined together appropriately. Any pieces with a

charset of `'unknown-8bit'` are decoded as ASCII using the `'replace'` error handler.

Alterado na versão 3.2: Added handling for the `'unknown-8bit'` charset.

__eq__(*other*)

This method allows you to compare two **`Header`** instances for equality.

__ne__(*other*)

This method allows you to compare two **`Header`** instances for inequality.

The **`email.header`** module also provides the following convenient functions.

email.header.decode_header(*header*)

Decode a message header value without converting the character set. The header value is in *header*.

For historical reasons, this function may return either:

1. A list of pairs containing each of the decoded parts of the header, `(decoded_bytes, charset)`, where *decoded_bytes* is always an instance of **`bytes`**, and *charset* is either:
    - A lower case string containing the name of the character set specified.
    - `None` for non-encoded parts of the header.
2. A list of length 1 containing a pair `(string, None)`, where *string* is always an instance of **`str`**.

An **`email.errors.HeaderParseError`** may be raised when certain decoding errors occur (e.g. a base64 decoding exception).

Here are examples:

```
>>> from email.header import decode_header
>>> decode_header('=?iso-8859-1?q?p=F6stal?=')
[(b'p\xf6stal', 'iso-8859-1')]
>>> decode_header('unencoded_string')
[('unencoded_string', None)]
>>> decode_header('bar =?utf-8?B?ZsOzbw==?=')
[(b'bar ', None), (b'f\xc3\xb3o', 'utf-8')]
```

Nota

This function exists for backwards compatibility only. For new code, we recommend using `email.headerregistry.HeaderRegistry`.

email.header.make_header(*decoded_seq*, *maxlinelen=None*, *header_name=None*, *continuation_ws=''*)

Create a `Header` instance from a sequence of pairs as returned by `decode_header()`.

`decode_header()` takes a header value string and returns a sequence of pairs of the format `(decoded_string, charset)` where *charset* is the name of the character set.

This function takes one of those sequence of pairs and returns a `Header` instance. Optional *maxlinelen*, *header_name*, and *continuation_ws* are as in the `Header` constructor.

Nota

This function exists for backwards compatibility only, and is not recommended for use in new code.

`email.charset`: Representando conjuntos de caracteres

Código-fonte: Lib/email/charset.py
[https://github.com/python/cpython/tree/3.15/Lib/email/charset.py]

Este módulo faz parte da API de e-mail legada (`Compat32`). Na nova API, apenas a tabela de apelidos é usada.

O texto restante nesta seção é a documentação original do módulo.

Este módulo fornece uma classe **`Charset`** para representar conjuntos de caracteres e conversões de conjuntos de caracteres em mensagens de e-mail, bem como um registro de conjuntos de caracteres e vários métodos práticos para manipular esse registro. Instâncias de **`Charset`** são usadas em vários outros módulos do pacote **`email`**.

Importa esta classe do módulo **`email.charset`**.

class email.charset.Charset(*input_charset=DEFAULT_CHARSET*)

Mapeia conjuntos de caracteres para suas propriedades de e-mail.

Esta classe fornece informações sobre os requisitos impostos ao e-mail para um conjunto de caracteres específico. Também fornece rotinas práticas para conversão entre conjuntos de caracteres, considerando a disponibilidade dos codecs aplicáveis. Dado um conjunto de caracteres, ela fará o possível para fornecer informações sobre como usá-lo em uma mensagem de e-mail de forma compatível com RFC.

Certos conjuntos de caracteres devem ser codificados com quoted-printable ou base64 quando usados em cabeçalhos ou corpos de e-mail. Certos conjuntos de caracteres devem ser convertidos diretamente e não são permitidos em e-mails.

O *input_charset* opcional é descrito abaixo; ele é sempre convertido para letras minúsculas. Após ser normalizado por apelido, ele também é usado como uma consulta no registro de conjuntos de caracteres para descobrir a codificação do cabeçalho, a codificação do corpo e o codec de conversão de saída a serem usados para o conjunto de caracteres. Por exemplo, se *input_charset* for `iso-8859-1`, os cabeçalhos e corpos serão codificados usando quoted-printable e nenhum codec de conversão de saída será necessário. Se *input_charset* for `euc-jp`, os cabeçalhos serão codificados em base64, os corpos não serão codificados, mas o texto de saída será convertido do conjunto de caracteres `euc-jp` para o conjunto de caracteres `iso-2022-jp`.

Instâncias **`Charset`** têm os seguintes atributos de dados:

input_charset
: O conjunto de caracteres inicial especificado. Apelidos comuns são convertidos em seus nomes de e-mail *oficiais* (por exemplo, `latin_1` é convertido para `iso-8859-1`). O padrão é `us-ascii` de 7 bits.

header_encoding
: Se o conjunto de caracteres precisar ser codificado antes de ser usado em um cabeçalho de e-mail, este atributo será definido como `charset.QP` (para quoted-printable), `charset.BASE64` (para codificação base64) ou `charset.SHORTEST` para a codificação mais curta entre QP ou BASE64. Caso contrário, será `None`.

body_encoding
: O mesmo que *header_encoding*, mas descreve a codificação do corpo da mensagem de e-mail, que pode ser diferente da codificação do cabeçalho. `charset.SHORTEST` não é permitido para *body_encoding*.

output_charset
: Alguns conjuntos de caracteres precisam ser convertidos antes de serem usados em cabeçalhos ou corpos de e-mail. Se *input_charset*

for um deles, este atributo conterá o nome do conjunto de caracteres para o qual a saída será convertida. Caso contrário, será `None`.

input_codec

O nome do codec Python usado para converter o *input_charset* para Unicode. Se nenhum codec de conversão for necessário, este atributo será `None`.

output_codec

O nome do codec Python usado para converter Unicode para o *output_charset*. Se nenhum codec de conversão for necessário, este atributo terá o mesmo valor que o *input_codec*.

Instâncias **`Charset`** também têm os seguintes métodos:

get_body_encoding()

Retorna a codificação de transferência de conteúdo usada para codificação do corpo.

Esta é a string `quoted-printable` ou `base64`, dependendo da codificação usada, ou é uma função, caso em que você deve chamar a função com um único argumento, sendo o objeto Message codificado. A função deve então definir o próprio cabeçalho *Content-Transfer-Encoding* com o valor apropriado.

Retorna a string `quoted-printable` se *body_encoding* for `QP`, retorna a string `base64` se *body_encoding* for `BASE64` e retorna a string `7bit` caso contrário.

get_output_charset()

Retorna o conjunto de caracteres de saída.

Este é o atributo *output_charset* se não for `None`, caso contrário, é *input_charset*.

header_encode(*string*)

Codifica o cabeçalho com a string *string*.

O tipo de codificação (base64 ou quoted-printable) será baseado no atributo *header_encoding*.

header_encode_lines(*string*, *maxlengths*)

Codifica um cabeçalho de uma *string* convertendo-a primeiro em bytes.

Isso é semelhante a `header_encode()`, exceto que a string é ajustada aos comprimentos máximos de linha, conforme fornecido pelo argumento *maxlengths*, que deve ser um iterador: cada elemento retornado deste iterador fornecerá o próximo comprimento máximo de linha.

body_encode(*string*)

Codifica o corpo com a string *string*.

O tipo de codificação (base64 ou quoted-printable) será baseado no atributo *body_encoding*.

A classe `Charset` também fornece vários métodos para dar suporte a operações padrão e funções embutidas.

__str__()

Retorna *input_charset* como uma string convertida para minúsculas. `__repr__()` é um apelido para `__str__()`.

__eq__(*other*)

Este método permite que você compare duas instâncias `Charset` para verificar a igualdade.

__ne__(*other*)

Este método permite que você compare duas instâncias `Charset` para verificar a desigualdade.

O módulo `email.charset` também fornece as seguintes funções para adicionar novas entradas ao conjunto global de caracteres, alias e registros de codec.

email.charset.add_charset(*charset*, *header_enc=None*, *body_enc=None*, *output_charset=None*)

Adiciona propriedades de caracteres ao registro global.

charset é o conjunto de caracteres de entrada e deve ser o nome canônico de um conjunto de caracteres.

header_enc e *body_enc* opcionais são `charset.QP` para quoted-printable, `charset.BASE64` para codificação base64, `charset.SHORTEST` para a codificação mais curta entre quoted-printable ou base64, ou `None` para nenhuma codificação. `SHORTEST` é válido apenas para *header_enc*. O padrão é `None` para nenhuma codificação.

output_charset opcional é o conjunto de caracteres que a saída deve conter. As conversões prosseguirão do conjunto de caracteres de entrada para Unicode e para o conjunto de caracteres de saída quando o método **`Charset.convert()`** for chamado. O padrão é gerar a saída no mesmo conjunto de caracteres da entrada.

Tanto *input_charset* quanto *output_charset* devem ter entradas de codec Unicode no mapeamento de conjunto de caracteres para codec do módulo; use **`add_codec()`** para adicionar codecs que o módulo não conhece. Consulte a documentação do módulo **`codecs`** para obter mais informações.

O registro do conjunto de caracteres global é mantido no dicionário global do módulo `CHARSETS`.

email.charset.add_alias(*alias*, *canonical*)

Adicione um apelido para o conjunto de caracteres. *alias* é o nome do alias, por exemplo, `latin-1`. *canonical* é o nome canônico do conjunto de caracteres, por exemplo, `iso-8859-1`.

O registro global de apelido de conjunto de caracteres é mantido no dicionário global do módulo `ALIASES`.

email.charset.add_codec(*charset*, *codecname*)

Adiciona um codec que mapeia caracteres no conjunto de caracteres fornecido para e a partir do Unicode.

charset é o nome canônico de um conjunto de caracteres. *codecname* é o nome de um codec Python, conforme apropriado para o segundo argumento do método `encode()` do `str`.

`email.encoders`: Codificadores

Código-fonte: Lib/email/encoders.py
[https://github.com/python/cpython/tree/3.15/Lib/email/encoders.py]

Este módulo faz parte da API legada de e-mail (`Compat32`). Na nova API, a funcionalidade é fornecida pelo parâmetro *cte* do método `set_content()`.

Este módulo está descontinuado no Python 3. As funções fornecidas aqui não devem ser chamadas explicitamente, pois a classe `MIMEText` define o tipo de conteúdo e o cabeçalho CTE usando os valores *_subtype* e *_charset* passados durante a instanciação dessa classe.

O texto restante nesta seção é a documentação original do módulo.

Ao criar objetos `Message` do zero, você frequentemente precisa codificar as cargas úteis para transporte por meio de servidores de e-mail compatíveis. Isso é especialmente verdadeiro para mensagens do tipo *image/** e *text/** contendo dados binários.

O pacote `email` fornece alguns codificadores convenientes em seu módulo `encoders`. Esses codificadores são realmente usados pelos construtores de classe `MIMEAudio` e `MIMEImage` para fornecer codificações padrão. Todas as funções de codificador recebem exatamente um argumento, o objeto de mensagem a ser codificado. Eles geralmente extraem a carga útil, codificam-na e redefinem a carga útil para esse valor recém-codificado. Eles também devem definir o cabeçalho *Content-Transfer-Encoding* conforme apropriado.

Observe que essas funções não são significativas para uma mensagem multiparte. Elas devem ser aplicadas a subpartes individuais, e levantarão um `TypeError` se for passada uma mensagem cujo tipo seja multiparte.

Aqui estão as funções de codificação fornecidas:

email.encoders.encode_quopri(*msg*)

Codifica a carga útil em formato quoted-printable e define o cabeçalho *Content-Transfer-Encoding* como `quoted-printable` [[1]]. Esta é uma boa codificação para usar quando a maior parte da sua carga útil é de dados imprimíveis normais, mas contém alguns caracteres não imprimíveis.

email.encoders.encode_base64(*msg*)

Codifica a carga útil em formato base64 e define o cabeçalho *Content-Transfer-Encoding* para `base64`. Esta é uma boa codificação para usar quando a maior parte da sua carga útil é de dados não imprimíveis, pois é um formato mais compacto do que quoted-printable. A desvantagem da codificação base64 é que ela torna o texto não legível por humanos.

email.encoders.encode_7or8bit(*msg*)

Na verdade, isso não modifica a carga útil da mensagem, mas define o cabeçalho *Content-Transfer-Encoding* para `7bit` ou `8bit`, conforme apropriado, com base nos dados da carga útil.

email.encoders.encode_noop(*msg*)

Isso não faz nada; nem mesmo define o cabeçalho *Content-Transfer-Encoding*.

Notas de rodapé

[1]

Observe que a codificação com **`encode_quopri()`** também codifica todos os caracteres de tabulação e espaço nos dados.

`email.utils`: Utilitários diversos

Código-fonte: Lib/email/utils.py
[https://github.com/python/cpython/tree/3.15/Lib/email/utils.py]

Existem alguns utilitários úteis fornecidos no **`email.utils`** module:

email.utils.localtime(*dt=None*)

Retorna a hora local como um objeto datetime consciente. Se chamado sem argumentos, retorna a hora atual. Caso contrário, o argumento *dt* deve ser uma instância **`datetime`** e é convertido para o fuso horário local de acordo com o banco de dados de fuso horário do sistema. Se *dt* for ingênuo (ou seja, `dt.tzinfo` for `None`), será presumido como estando no horário local.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.12, removido na versão 3.14: O parâmetro *isdst*.

email.utils.make_msgid(*idstring=None*, *domain=None*)

Retorna uma string adequada para um cabeçalho *Message-ID* compatível com **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]. O *idstring* opcional, se fornecido, é uma string usada para fortalecer a exclusividade do ID da mensagem. O *domain* Opcional, se dado, fornece a porção do msgid após o '@'. O padrão é o nome do host local. Normalmente, não é necessário substituir esse padrão, mas pode ser útil em alguns casos, como um sistema distribuído de construção que usa um nome de domínio consistente em vários hosts.

Alterado na versão 3.2: Adicionada a palavra-chave *domain*.

As funções restantes fazem parte da API de e-mail herdada (`Compat32`). Não há necessidade de usá-las diretamente com a nova API, pois a análise e

a formatação fornecidas são feitas automaticamente pelo mecanismo de análise de cabeçalhos da nova API.

email.utils.quote(*str*)

Devolve uma nova string com barras invertidas em *str* substituídas por duas barras invertidas e aspas duplas substituídas por aspas duplas invertidas.

email.utils.unquote(*str*)

Retorna uma nova string que é uma versão sem aspas de *str*. Se *str* terminar e começar com aspas duplas, elas serão removidas. Da mesma forma, se *str* termina e começa com colchetes angulares, eles são removidos.

email.utils.parseaddr(*address*, *, *strict=True*)

Analisa o endereço – que deve ser o valor de algum campo contendo endereço, como *To* ou *Cc* – em suas partes constituinte *realname* e *email address*. Retorna uma tupla daquela informação, a menos que a análise falhe, caso em que uma tupla de 2 de `('', '')` é retornada.

Se *strict* for verdadeiro, usa um analisador sintático estrito que rejeite entradas malformadas.

Alterado na versão 3.13: Adiciona o parâmetro opcional *strict* e rejeita entradas malformadas por padrão.

email.utils.formataddr(*pair*, *charset='utf-8'*)

O inverso de **`parseaddr()`**, isto leva uma tupla de 2 do forma `(realname, email_address)` e retorna o valor de string adequado para um cabeçalho *To* ou *Cc* . Se o primeiro elemento de *pair* for falso, o segundo elemento será retornado sem modificações.

O *charset* opcional é o conjunto de caracteres que será usado na codificação **RFC 2047** [https://datatracker.ietf.org/doc/html/rfc2047.html] do `realname` se o `realname` contiver caracteres não-ASCII. Pode ser uma instância de **`str`** ou a **`Charset`**. O padrão é `utf-8`.

Alterado na versão 3.3: Adicionada a opção *charset*.

email.utils.getaddresses(*fieldvalues*, *, *strict=True*)

Este método retorna uma lista de tuplas 2 do formulário retornado por `parseaddr()`. *fieldvalues* é uma sequência de valores do campo de cabeçalho que pode ser retornada por `Message.get_all`.

Se *strict* for verdadeiro, usa um analisador sintático estrito que rejeite entradas malformadas.

Aqui está um exemplo simples que recebe todos os destinatários de uma mensagem:

```
from email.utils import getaddresses

tos = msg.get_all('to', [])
ccs = msg.get_all('cc', [])
resent_tos = msg.get_all('resent-to', [])
resent_ccs = msg.get_all('resent-cc', [])
all_recipients = getaddresses(tos + ccs + resent_tos + resent
```

Alterado na versão 3.13: Adiciona o parâmetro opcional *strict* e rejeita entradas malformadas por padrão.

email.utils.parsedate(*date*)

Tenta analisar uma data de acordo com as regras em **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]. no entanto, alguns mailers não seguem esse formato conforme especificado, portanto `parsedate()` tenta adivinhar corretamente em tais casos. *date* é uma string contendo uma data **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html], como `"Mon, 20 Nov 1995 19:12:08 -0500"`. Se conseguir analisar a data, **`parsedate()`** retorna uma 9-tupla que pode ser passada diretamente para `time.mktime()`; caso contrário, `None` será retornado. Observe que os índices 6, 7 e 8 da tupla de resultados não são utilizáveis.

email.utils.parsedate_tz(*date*)

Executa a mesma função que `parsedate()`, mas retorna `None` ou uma tupla de 10; os primeiros 9 elementos formam uma tupla que pode ser

passada diretamente para `time.mktime()`, e o décimo é o deslocamento do fuso horário da data do UTC (que é o termo oficial para o horário de Greenwich) [[1]]. Se a string de entrada não tem fuso horário, o último elemento da tupla retornado é `0`, que representa UTC. Observe que os índices 6, 7 e 8 da tupla de resultado não podem ser usados.

email.utils.parsedate_to_datetime(*date*)

O inverso de `format_datetime()`. Desempenha a mesma função que `parsedate()`, mas em caso de sucesso retorna um `datetime`; caso contrário, `ValueError` é levantada se *date* contiver um valor inválido, como uma hora maior que 23 ou uma diferença de fuso horário não entre -24 e 24 horas. Se a data de entrada tem um fuso horário de `-0000`, o `datetime` será um `datetime` ingênuo, e se a data estiver em conformidade com os RFCs representará um horário em UTC, mas sem indicação do fuso horário de origem real da mensagem de onde vem a data. Se a data de entrada tiver qualquer outro deslocamento de fuso horário válido, o `datetime` será um `datetime` consciente com o correspondente a `timezone` `tzinfo`.

Adicionado na versão 3.3.

email.utils.mktime_tz(*tuple*)

Transforma uma tupla de 10 conforme retornado por `parsedate_tz()` em um timestamp UTC (segundos desde a Era Unix). Se o item de fuso horário na tupla for `None`, considera a hora local.

email.utils.formatdate(*timeval=None*, *localtime=False*, *usegmt=False*)

Retorna uma string de data conforme **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html]. Por exemplo:

```
Fri, 09 Nov 2001 01:08:47 -0000
```

O *timeval* opcional, se fornecido, é um valor de tempo de ponto flutuante, conforme aceito por `time.gmtime()` e `time.localtime()`, caso contrário, o tempo atual é usado.

Há um sinalizador opcional *localtime*, que, quando é `True`, interpreta *timeval* e retorna uma data relativa ao fuso horário local em vez do UTC, levando em consideração o horário de verão. O padrão é `False`, o que significa que o UTC é usado.

O *usegmt* opcional é um sinalizador que quando `True`, produz uma string de data com o fuso horário como uma string ascii `GMT`, ao invés de um numérico `-0000`. Isso é necessário para alguns protocolos (como HTTP). Isso se aplica apenas quando *localtime* for `False`. O padrão é `False`.

email.utils.format_datetime(*dt*, *usegmt=False*)

Como `formatdate`, mas a entrada é uma instância de **`datetime`**. Se for uma data e hora ingênua, será considerado "UTC sem informações sobre o fuso horário de origem" e o convencional `-0000` será usado para o fuso horário. Se for um `datetime` ciente, então o deslocamento de fuso horário numérico é usado. Se for um fuso horário ciente com deslocamento zero, então *usegmt* pode ser definido como `True`, caso em que a string `GMT` é usada em vez do deslocamento numérico do fuso horário. Isso fornece uma maneira de gerar cabeçalhos de data HTTP em conformidade com os padrões.

Adicionado na versão 3.3.

email.utils.decode_rfc2231(*s*)

Decodifica a string *s* de acordo com **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html].

email.utils.encode_rfc2231(*s*, *charset=None*, *language=None*)

Codifica a string *s* de acordo com **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html]. *charset* e *language* opcionais, se fornecido, são o nome do conjunto de caracteres e o nome do idioma a ser usado. Se nenhum deles for fornecido, *s* é retornado como está. Se *charset* for fornecido, mas *language* não, a string será codificada usando a string vazia para *language*.

email.utils.collapse_rfc2231_value(*value*, *errors='replace'*, *fallback_charset='us-ascii'*)

Quando um parâmetro de cabeçalho é codificado no formato **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html], `Message.get_param` pode retornar uma tupla de 3 elementos contendo o conjunto de caracteres, idioma e valor. `collapse_rfc2231_value()` transforma isso em uma string. *errors* opcionais são passados para o argumento *errors* do método `encode()` de `str`; o padrão é `'replace'`. *fallback_charset* opcional especifica o conjunto de caracteres a ser usado se aquele no cabeçalho **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html] não for conhecido pelo Python; o padrão é `'us-ascii'`.

Por conveniência, se *value* passado para `collapse_rfc2231_value()` não for uma tupla, deve ser uma string e é retornado sem aspas.

email.utils.decode_params(*params*)

Decodifica a lista de parâmetros de acordo com **RFC 2231** [https://datatracker.ietf.org/doc/html/rfc2231.html]. *params* é uma sequência de 2 tuplas contendo elementos do formulário `(content-type, string-value)`.

Notas de rodapé

[1]

Observa que o sinal do deslocamento de fuso horário é o oposto do sinal da variável `time.timezone` para o mesmo fuso horário; a última variável segue o padrão POSIX enquanto este módulo segue **RFC 2822** [https://datatracker.ietf.org/doc/html/rfc2822.html].

`email.iterators`: Iteradores

Código-fonte: Lib/email/iterators.py
[https://github.com/python/cpython/tree/3.15/Lib/email/iterators.py]

A iteração sobre uma árvore de objetos de mensagem é bastante fácil com o método `Message.walk`. O módulo `email.iterators` fornece algumas iterações úteis de nível mais alto sobre as árvores de objetos de mensagens.

email.iterators.body_line_iterator(*msg*, *decode=False*)

Isso itera sobre todas as cargas úteis em todas as subpartes de *msg*, retornando as cargas úteis das strings de linhas por linha. Ele pula todos os cabeçalhos da subparte e pula qualquer subparte com uma carga útil que não seja uma string Python. Isso é um pouco equivalente à leitura da representação de texto simples da mensagem de um arquivo usando `readline()`, pulando todos os cabeçalhos intermediários.

decode opcional é passado por meio do `Message.get_payload`.

email.iterators.typed_subpart_iterator(*msg*, *maintype='text'*, *subtype=None*)

Isso itera sobre todas as subpartes de *msg*, retornando apenas as subpartes que correspondem ao tipo MIME especificado por *maintype* e *subtype*.

Observe que *subtipo* é opcional; se omitido, a correspondência de tipo MIME da subparte é feita apenas com o tipo principal. *maintype* também é opcional; o padrão é *text*.

Assim, por padrão `typed_subpart_iterator()` retorna cada subparte que possui um tipo MIME de *text/**.

A seguinte função foi adicionada como uma ferramenta de depuração útil. *Não* deve ser considerado parte da interface pública suportada para o pacote.

email.iterators._structure(*msg*, *fp=None*, *level=0*, *include_default=False*)

Imprime uma representação recuada dos tipos de conteúdo da estrutura do objeto de mensagem. Por exemplo:

```
>>> msg = email.message_from_file(algum_arquivo)
>>> _structure(msg)
multipart/mixed
    text/plain
    text/plain
    multipart/digest
        message/rfc822
            text/plain
        message/rfc822
            text/plain
        message/rfc822
            text/plain
        message/rfc822
            text/plain
        message/rfc822
            text/plain
    text/plain
```

O *fp* opcional é um objeto arquivo ou similar para o qual deve-se imprimir a saída. Deve ser adequado para a função Python `print()`. *level* usado internamente. *include_default*, se verdadeiro, também imprime o tipo padrão.

`json` — Codificador e decodificador JSON

Código-fonte: Lib/json/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/json/__init__.py]

JSON (JavaScript Object Notation) [https://json.org], especificado pelo **RFC 7159** [https://datatracker.ietf.org/doc/html/rfc7159.html] (que tornou o **RFC 4627** [https://datatracker.ietf.org/doc/html/rfc4627.html] obsoleta) e pelo ECMA-404 [https://ecma-international.org/publications-and-standards/standards/ecma-404/], é um formato leve de troca de dados inspirado pela sintaxe de objeto JavaScript [https://en.wikipedia.org/wiki/JavaScript] (embora não seja um subconjunto estrito de JavaScript [[1]]).

Nota

O termo “objeto” no contexto do processamento JSON em Python pode ser ambíguo. Todos os valores em Python são objetos. Em JSON, um objeto se refere a qualquer dado entre chaves, semelhante a um dicionário Python.

Aviso

Tenha cuidado quando estiver analisando dados JSON de fontes não-confiáveis. Uma string JSON maliciosa pode fazer o decodificador consumir recursos consideráveis de CPU e memória. É recomendado limitar o tamanho do dado a ser analisado.

Este módulo expõe uma API familiar para pessoas usuárias dos módulos `marshal` e `pickle` da biblioteca padrão.

Codificação de hierarquias básicas de objetos Python:

```
>>> import json
>>> json.dumps(['foo', {'bar': ('baz', None, 1.0, 2)}])
'["foo", {"bar": ["baz", null, 1.0, 2]}]'
>>> print(json.dumps("\"foo\bar"))
"\"foo\bar"
>>> print(json.dumps('\u1234'))
"\u1234"
>>> print(json.dumps('\\'))
"\\"
>>> print(json.dumps({"c": 0, "b": 0, "a": 0}, sort_keys=True))
{"a": 0, "b": 0, "c": 0}
>>> from io import StringIO
>>> io = StringIO()
>>> json.dump(['streaming API'], io)
>>> io.getvalue()
'["streaming API"]'
```

Codificação compacta:

```
>>> import json
>>> json.dumps([1, 2, 3, {'4': 5, '6': 7}], separators=(',', ':')
'[1,2,3,{"4":5,"6":7}]'
```

Saída bonita:

```
>>> import json
>>> print(json.dumps({'6': 7, '4': 5}, sort_keys=True, indent=4))
{
    "4": 5,
    "6": 7
}
```

Personalizando em codificação de objeto JSON:

```
>>> import json
>>> def custom_json(obj):
...     if isinstance(obj, complex):
...         return {'__complex__': True, 'real': obj.real, 'imag'
...     raise TypeError(f'Cannot serialize object of {type(obj)}'
...
>>> json.dumps(1 + 2j, default=custom_json)
'{"__complex__": true, "real": 1.0, "imag": 2.0}'
```

Decodificando JSON:

```
>>> import json
>>> json.loads('["foo", {"bar":["baz", null, 1.0, 2]}]')
['foo', {'bar': ['baz', None, 1.0, 2]}]
>>> json.loads('"\\"foo\\bar"')
'"foo\x08ar'
>>> from io import StringIO
>>> io = StringIO('["streaming API"]')
>>> json.load(io)
['streaming API']
```

Personalizando em decodificação de objeto JSON:

```
>>> import json
>>> def as_complex(dct):
...     if '__complex__' in dct:
...         return complex(dct['real'], dct['imag'])
...     return dct
...
>>> json.loads('{"__complex__": true, "real": 1, "imag": 2}',
...     object_hook=as_complex)
(1+2j)
>>> import decimal
>>> json.loads('1.1', parse_float=decimal.Decimal)
Decimal('1.1')
```

Estendendo **JSONEncoder**:

```
>>> import json
>>> class ComplexEncoder(json.JSONEncoder):
...     def default(self, obj):
...         if isinstance(obj, complex):
...             return [obj.real, obj.imag]
...         # Let the base class default method raise the TypeErr
...         return super().default(obj)
...
>>> json.dumps(2 + 1j, cls=ComplexEncoder)
'[2.0, 1.0]'
>>> ComplexEncoder().encode(2 + 1j)
'[2.0, 1.0]'
>>> list(ComplexEncoder().iterencode(2 + 1j))
['[2.0', ', 1.0', ']']
```

Usando **`json`** para validar a partir do console e exibir formatado:

```
$ echo '{"json":"obj"}' | python -m json
{
    "json": "obj"
}
$ echo '{1.2:3.4}' | python -m json
Expecting property name enclosed in double quotes: line 1 column
```

Veja Interface de linha de comando para a documentação detalhada.

Nota

JSON é um subconjunto do YAML [https://yaml.org/] 1.2. O JSON gerado pelas definições padrões desse módulo (particularmente, o valor padrão dos *separadores*) é também um subconjunto do YAML 1.0 e 1.1. Esse módulo pode, portanto, também ser usado como serializador YAML.

Nota

O codificador e o decodificador do módulo preservam a ordem de entrada e saída por padrão. A ordem só é perdida se os contêineres subjacentes estão desordenados.

Uso básico

json.dump(*obj*, *fp*, ***, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *cls=None*, *indent=None*, *separators=None*, *default=None*, *sort_keys=False*, ***kw*)

Serializa um *obj* como um stream JSON formatado para *fp* (um objeto arquivo ou similar com suporte a `.write()`) usando essa tabela de conversão de Python para JSON.

Nota

Diferente de **pickle** e **marshal**, JSON não é um protocolo com datagrama, assim tentar serializar múltiplos objetos com chamadas repetidas para **dump()** usando o mesmo *fp* resultará em um arquivo JSON inválido.

Parâmetros::

- **obj** (*object*) – O objeto Python a ser serializado.
- **fp** (file-like object) – O objeto arquivo ou similar *obj* para o qual será serializado. O módulo **json** sempre produz objetos **str**, e não objetos **bytes**. Dessa forma, `fp.write()` precisa ter suporte para entradas **str**.
- **skipkeys** (*bool*) – Se `True`, as chaves que não forem de um tipo básico (**str**, **int**, **float**, **bool**, `None`) serão ignoradas ao invés de levantar uma exceção **TypeError**. Padrão `False`.
- **ensure_ascii** (*bool*) – Se definido como `True` (o padrão), a saída terá todos os caracteres não ASCI e não imprimíveis recebidos escapados. Se definido como `False`, todos os caracteres serão exibidos como estão, exceto os caracteres que devem ser escapados: aspas, barra invertida e os caracteres de controle U+0000 a U+001F.
- **check_circular** (*bool*) – Se `False`, a verificação d referência circular for de tipos contêineres é ignorada e uma referência circular resultará em uma exceção **RecursionError** (ou pior). Padrão `True`.
- **allow_nan** (*bool*) – Se `False`, serialização de valores **float** fora de intervalo (`nan`, `inf`, `-inf`) va resultar em um **ValueError** em estrita conformidade com a especificação. Se `True` (o padrão), seus equivalentes JavaScript (`NaN`, `Infinity`, `-Infinity`) serão usados.
- **cls** (a **JSONEncoder** subclass) – Se definido, um codificador JSON personalizado com o método **default()** substituído, para serializar em tipos de dados personalizados. Se `None` (o padrão), **JSONEncoder** é usado.

JSONEncoder é usado.

- **indent** (*int* | *str* | *None*) – Se for um inteiro positivo ou string, os elementos de array JSON e membros de objeto serão formatados de forma legível com esse nível de indentação. Um inteiro positivo indenta essa quantidade de espaços por nível; uma string (como `"\t"`) é usada para indentar cada nível. Se for zero, negativo ou `""` (a string vazia), apenas quebras de linha são inserida Se for `None` (o padrão), nenhuma quebra de linha é inserida.
- **separators** (*tuple* | *None*) – Uma tupla dupla: `(item_separator, key_separator)`. Se `None` (o padrão), *separators* assume como padrão `(', ', ': ')` se *indent* for `None`, e `(',', ': ')` caso contrário. Para o JSON mais compacto, especifiqu `(',', ':')` para eliminar espaços em branco.
- **default** (callable | None) – Uma função que é chamada para objetos que não podem ser serializados de outra forma. Ela deve retornar uma versão JSON codificável do objeto ou gerar um **`TypeError`**. Se `None` (o padrão), **`TypeError`** é levantada.
- **sort_keys** (*bool*) – Se `True`, os dicionários serão emitidos classificados por chave. Padrão `False`.

Nota

Chaves nos pares chave/valor de JSON são sempre do tipo **`str`**. Quando um dicionário é convertido para JSON, todas as chaves são convertidas para strings. Como resultado disso, se um dicionário é convertido para JSON e depois de volta para um dicionário, o dicionário pode não ser igual ao original. Isto é, `loads(dumps(x)) != x` se `x` tem chaves não strings. *sort_keys* ordena as chaves antes de elas serem convertidas em strings; portanto, chaves numéricas são ordenadas pelo valor, e não pela sua representação como string.

Alterado na versão 3.2: Permite strings para *indent*, além de inteiros.

Alterado na versão 3.4: Usa `(',', ': ')` como padrão se *indent* não for `None`.

Alterado na versão 3.6: Todos os parâmetros opcionais agora são somente-nomeados.

json.dumps(*obj*, *, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *cls=None*, *indent=None*, *separators=None*, *default=None*, *sort_keys=False*, **kw)

Serializa *obj* para uma **`str`** com formato JSON usando essa tabela de conversão. Os argumentos têm o mesmo significado que na função **`dump()`**.

json.load(*fp*, *, *cls=None*, *object_hook=None*, *parse_float=None*, *parse_int=None*, *parse_constant=None*, *object_pairs_hook=None*, *array_hook=None*, **kw)

Desserializa *fp* para um objeto Python usando a tabela de conversão JSON para Python.

Parâmetros::

- **fp** (file-like object) – Um arquivo texto ou arquivo binário com suporte a `.read()` contendo o documento JSON a ser desserializado.
- **cls** (a **`JSONDecoder`** subclass) – Se definido, um decodificador JSON personalizado. Argumentos nomeados adicionais para **`load()`** serão passados para o construtor de *cls*. Se `None` (o padrão), **`JSONDecoder`** é usado.
- **object_hook** (callable | None) – Se definido, uma função que é chamada com o resultado de qualque literal de objeto JSON decodificado (um **`dict`**). O valor de retorno desta função será usado em vez d **`dict`**. Este recurso pode ser usado para implementar decodificadores personalizados, por exemplo dica de classes JSON-RPC [https://www.jsonrpc.org]. O padrão é `None`.
- **object_pairs_hook** (callable | None) – Se definid uma função que é chamada com o resultado de qualquer literal de objeto JSON decodificado com

uma lista ordenada de pares. O valor de retorno desta função será usado em vez de **`dict`**. Este recurso pode ser usado para implementar decodificadores personalizados. Se *object_hook* também for definido, *object_pairs_hook* terá prioridade. O padrão é `None`.

- **array_hook** (callable | None) – Se definido, uma função que é chamada com o resultado de qualque literal de vetor JSON decodificado como uma lista Python. O valor de retorno desta função será usad em vez de **`list`**. Este recurso pode ser usado para implementar decodificadores personalizados. O padrão é `None`.
- **parse_float** (callable | None) – Se definido, uma função que é chamada com a string de cada ponto flutuante JSON a ser decodificada. Se `None` (o padrão), é equivalente a `float(num_str)`. Isso pode ser usado para analisar pontos flutuantes JSON em tipos de dados personalizados, por exemplo **`decimal.Decimal`**.
- **parse_int** (callable | None) – Se definido, uma função que é chamada com a string de cada int JSON a ser decodificado. Se `None` (o padrão), é equivalente a `int(num_str)`. Isso pode ser usado para analisar inteiros JSON em tipos de dados personalizados, por exemplo **`float`**.
- **parse_constant** (callable | None) – Se definido, uma função que é chamada com uma das seguinte strings: `'-Infinity'`, `'Infinity'` ou `'NaN'`. Isso pode ser usado para levantar uma exceção se números JSON inválidos forem encontrados. Padrão `None`.

Levanta::

- **JSONDecodeError** – Quando os dados a serem desserializados não forem um documento JSON válido.
- **UnicodeDecodeError** – Quando os dados que estão sendo desserializados não contêm dados codificados em UTF-8, UTF-16 ou UTF-32.

Alterado na versão 3.1:

- Adicionado o parâmetro opcional *object_pairs_hook*.
- *parse_constant* não é mais chamada para ‘null’, ‘true’, ‘false’.

Alterado na versão 3.6:

- Todos os parâmetros opcionais agora são somente-nomeados.
- *fp* agora pode ser um arquivo binário. A entrada deve estar codificada como UTF-8, UTF-16 ou UTF-32.

Alterado na versão 3.11: O *parse_int* padrão para `int()` agora limita o tamanho máximo da string de inteiro via limitação de comprimento de string na conversão para inteiro do interpretador para ajudar a evitar ataques por negação de serviço.

Alterado na versão 3.15: Adicionado o parâmetro opcional *array_hook*.

json.loads(*s*, **, cls=None, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, object_pairs_hook=None, array_hook=None, **kw*)

Idêntico a `load()`, mas em vez de um objeto arquivo ou similar, desserializa *s* (uma instância de `str`, `bytes` ou `bytearray` contendo um documento JSON) para um objeto Python usando essa tabela de conversão.

Alterado na versão 3.6: s agora pode ser um do tipo `bytes` ou `bytearray`. A entrada deve estar codificado como UTF-8, UTF-16 ou UTF-32.

Alterado na versão 3.9: O argumento nomeado *encoding* foi removido.

Codificadores e decodificadores

class json.JSONDecoder(**, object_hook=None, parse_float=None, parse_int=None, parse_constant=None, strict=True, object_pairs_hook=None, array_hook=None*)

Decodificador JSON simples.

Executa as seguintes traduções na decodificação por padrão:

JSON	Python
objeto	dict
array	lista
string	str
número (inteiro)	int
número (real)	ponto flutuante
true	True
false	False
null	None

Ele também entende `NaN`, `Infinity` e `-Infinity` como seus valores `float` correspondentes, que estão fora da especificação JSON.

object_hook é uma função opcional que será chamada com o resultado de cada objeto JSON decodificado e seu valor de retorno será usado no lugar do dado **`dict`**. Isso pode ser usado para fornecer desserializações personalizadas (por exemplo, para oferecer suporte a dicas de classe JSON-RPC [https://www.jsonrpc.org])

object_pairs_hook é uma função opcional que será chamada com o resultado de todo objeto JSON decodificado com uma lista ordenada de pares. O valor de retorno de *object_pairs_hook* será usado no lugar do **`dict`**. Este recurso pode ser usado para implementar decodificadores personalizados. Se *object_pairs_hook* também for definido, o *object_pairs_hook* terá prioridade.

Alterado na versão 3.1: Adicionado suporte para *object_pairs_hook*.

array_hook é uma função opcional que será chamada com o resultado de todo vetor JSON decodificado como uma lista. O valor de retorno de *array_hook* será usado em vez de `list`. Este recurso pode ser usado para implementar decodificadores personalizados.

Alterado na versão 3.15: Adicionado suporte a *array_hook*.

parse_float é uma função opcional que será chamada com a string de cada ponto flutuante JSON para ser decodificado. Por padrão, equivale a `float(num_str)`. Pode ser usado para qualquer outro tipo de dado ou analisador de pontos flutuante JSON (por exemplo, `decimal.Decimal`).

parse_int é uma função opcional que será chamada com a string de cada inteiro JSON para ser decodificado. Por padrão, equivale a `int(num_str)`. Pode ser usado para qualquer outro tipo de dado ou analisador de inteiros JSON (por exemplo, `float`).

parse_constant é uma função opcional que será chamada para cada um das seguintes strings: `'-Infinity'`, `'Infinity'`, `'NaN'`. Isso pode ser usado para levantar uma exceção se forem encontrados números JSON inválidos.

Se *strict* for falso (`True` é o padrão), os caracteres de controle serão permitidos dentro das strings. Os caracteres de controle neste contexto são aqueles com códigos de caracteres no intervalo 0–31, incluindo `'\t'` (tab), `'\n'`, `'\r'` e `'\0'`.

Se os dados a serem desserializados não forem um documento JSON válido, será levantada uma exceção `JSONDecodeError`.

Alterado na versão 3.6: Todos os parâmetros agora são somente-nomeado.

decode(*s*)

Retorna a representação Python de *s* (uma instância `str` contendo um documento JSON).

Uma exceção `JSONDecodeError` será levantada se o documento JSON fornecido não for válido.

raw_decode(*s*)

Decodifica um documento JSON a partir de *s* (uma `str` iniciando com um documento JSON) e retornando uma tupla de 2 elementos, a representação Python e o índice em *s* onde o documento finaliza.

Isso pode ser usado para decodificar um documento JSON a partir de uma string que possa ter dados extras ao final.

class json.JSONEncoder(*, *skipkeys=False*, *ensure_ascii=True*, *check_circular=True*, *allow_nan=True*, *sort_keys=False*, *indent=None*, *separators=None*, *default=None*)

Codificador JSON extensível para estruturas de dados Python.

Por padrão, possui suporte para os seguintes objetos e tipos:

Python	**JSON**
dict, frozendict	objeto
list, tuple	array
str	string
int, float e Enums derivados de int e float	número
True	true
False	false
None	null

Alterado na versão 3.4: Adicionado suporte para classes Enum derivadas de int e float.

Alterado na versão 3.15: Adicionado suporte para `frozendict`.

Para estender isso para reconhecer outros objetos, crie uma subclasse e implemente o método `default()` com outro método que retorne um objeto serializável para `o` se possível, caso contrário deveria chamar a implementação da superclasse (para levantar `TypeError`).

Se *skipkeys* é falso (o padrão), então sluma `TypeError` será levantada ao tentar codificar as chaves que não são `str`, `int`, `float`, `bool` ou `None`. Se *skipkeys* é verdadeiro, esses itens são simplesmente pulados.

Se *ensure_ascii* for verdadeiro (o padrão), a saída terá todos os caracteres não ASCII e não imprimíveis recebidos escapados. Se *ensure_ascii* for falso, todos os caracteres serão exibidos como estão, exceto os caracteres que devem ser escapados: aspas, barra invertida e os caracteres de controle U+0000 a U+001F.

Se *check_circular* é verdadeiro (o padrão), então listas, dicionários e objetos codificados personalizados serão verificados por referências circulares durante a codificação para prevenir uma recursão infinita (que iria causar uma `RecursionError`). Caso contrário, nenhuma verificação será feita.

Se *allow_nan* for verdadeiro (o padrão), então `NaN`, `Infinity`, e `-Infinity` serão codificados como tal. Esse comportamento não é compatível com a especificação do JSON, mas é consistente com a maioria dos codificadores e decodificadores baseados em JavaScript. Caso contrário, será levantada uma `ValueError` para tais pontos flutuantes.

Se *sort_keys* for verdadeiro (padrão: `False`), então a saída dos dicionários serão ordenados pela chave; isto é útil para testes de regressão para certificar-se que as serializações de JSON possam ser comparadas com uma base no dia a dia.

Se *indent* for um inteiro não negativo ou uma string, então elementos de um vetor JSON e membros de objetos terão uma saída formatada com este nível de indentação. Um nível de indentação 0, negativo ou `""` apenas colocará novas linhas. `None` (o padrão) seleciona a representação mais compacta. Usando um inteiro positivo a indentação terá alguns

espaços por nível. Se *indent* for uma string (como `"\t"`), essa string será usada para indentar cada nível.

Alterado na versão 3.2: Permite strings para *indent*, além de inteiros.

Se especificado, *separators* deve ser uma tupla `(item_separator, key_separator)`. O padrão é `(', ', ': ')` se *indent* for `None` e `(',', ': ')` caso contrário. Para pegar representação JSON mais compacta, você deve especificar `(',', ':')` para eliminar espaços em branco.

Alterado na versão 3.4: Usa `(',', ': ')` como padrão se *indent* não for `None`.

Se especificado, *default* deve ser uma função para ser chamada para objetos que não podem ser serializados de outra forma. Deve retornar uma versão codificável JSON do objeto ou levantar uma exceção **`TypeError`**. Se não for especificada, **`TypeError`** é levantada.

Alterado na versão 3.6: Todos os parâmetros agora são somente-nomeado.

default(*o*)

Implemente este método em uma subclasse que retorna um objeto serializável para *o* ou que chame a implementação base (para levantar uma **`TypeError`**).

Por exemplo, para suporte a iteradores arbitrários, você poderia implementar **`default()`** dessa forma:

```
def default(self, o):
   try:
       iterable = iter(o)
   except TypeError:
       pass
   else:
       return list(iterable)
   # Faz o método padrão de classe base levantar a TypeErro
   return super().default(o)
```

encode(*o*)

Retorna uma string representando um JSON a partir da estrutura de dados Python, *o*. Por exemplo:

```
>>> json.JSONEncoder().encode({"foo": ["bar", "baz"]})
'{"foo": ["bar", "baz"]}'
```

iterencode(*o*)

Codifica o objeto dado, *o*, e produz cada representação em string assim que disponível. Por exemplo:

```
for chunk in json.JSONEncoder().iterencode(bigobject):
    mysocket.write(chunk)
```

Exceções

exception json.JSONDecodeError(*msg*, *doc*, *pos*)

Subclasse de `ValueError` com os seguintes atributos adicionais:

msg

A mensagem de erro não formatada.

doc

O documento JSON sendo analisado.

pos

O índice inicial de *doc* em que a análise falhou.

lineno

A linha correspondente a *pos*.

colno

A coluna correspondente a *pos*.

Adicionado na versão 3.5.

Conformidade e interoperabilidade entre padrões

O formato JSON é especificado pela **RFC 7159** [https://datatracker.ietf.org/doc/html/rfc7159.html] e por ECMA-404 [https://ecma-international.org/publications-and-standards/standards/ecma-404/]. Esta seção detalha o nível de conformidade deste módulo com a RFC. Para simplificar, as subclasses `JSONEncoder` e `JSONDecoder`, e outros parâmetros além daqueles explicitamente mencionados, não são considerados.

Este módulo não está em conformidade com a RFC de forma estrita, implementando algumas extensões que são JavaScript válidas, mas não JSON válido. Em particular:

- Os valores de números infinitos e NaN são aceitos e produzidos;
- Nomes repetidos em um objeto são aceitos e apenas o valor do último par nome-valor é usado.

Uma vez que a RFC permite que os analisadores compatíveis com a RFC aceitem textos de entrada que não sejam compatíveis com a RFC, o desserializador deste módulo é tecnicamente compatível com a RFC nas configurações padrão.

Codificações de caracteres

A RFC requer que JSON seja representado usando UTF-8, UTF-16 ou UTF-32, com UTF-8 sendo o padrão recomendado para interoperabilidade máxima.

Conforme permitido, embora não exigido, pela RFC, o serializador deste módulo define *ensure_ascii=True* por padrão, escapando a saída para que as strings resultantes contenham apenas caracteres ASCII imprimíveis.

Além do parâmetro *ensure_ascii*, este módulo é definido estritamente em termos de conversão entre objetos Python e `strings Unicode` e, portanto, não aborda diretamente o problema de codificação de caracteres.

A RFC proíbe adicionar uma marca de ordem de byte (do inglês *byte order mark* - BOM) ao início de um texto JSON, e o serializador deste módulo não adiciona um BOM à sua saída. A RFC permite, mas não exige, que os desserializadores JSON ignorem um BOM inicial em sua entrada. O

desserializador deste módulo levanta uma `ValueError` quando um BOM inicial está presente.

A RFC não proíbe explicitamente as strings JSON que contêm sequências de bytes que não correspondem a caracteres Unicode válidos (por exemplo, substitutos UTF-16 não emparelhados), mas observa que podem causar problemas de interoperabilidade. Por padrão, este módulo aceita e produz (quando presente no original `str`) pontos de código para tais sequências.

Valores numéricos infinitos e NaN

A RFC não permite a representação de valores infinitos ou numéricos NaN. Apesar disso, por padrão, este módulo aceita e produz `Infinity`, `-Infinity` e `NaN` como se fossem valores literais de número JSON válidos:

```
>>> # Nenhuma dessas chamadas levanta uma exceção, mas os resulta
>>> json.dumps(float('-inf'))
'-Infinity'
>>> json.dumps(float('nan'))
'NaN'
>>> # Same when deserializing
>>> json.loads('-Infinity')
-inf
>>> json.loads('NaN')
nan
```

No serializador, o parâmetro *allow_nan* pode ser usado para alterar esse comportamento. No desserializador, o parâmetro *parse_constant* pode ser usado para alterar esse comportamento.

Nomes repetidos dentro de um objeto

A RFC especifica que os nomes em um objeto JSON devem ser exclusivos, mas não determina como os nomes repetidos em objetos JSON devem ser tratados. Por padrão, este módulo não levanta uma exceção; em vez disso, ele ignora tudo, exceto o último par nome-valor para um determinado nome:

```
>>> weird_json = '{"x": 1, "x": 2, "x": 3}'
>>> json.loads(weird_json)
```

```
{'x': 3}
```

O parâmetro *object_pairs_hook* pode ser usado para alterar este comportamento.

Valores não objeto e não vetor de nível superior

A versão antiga de JSON especificada pela obsoleta **RFC 4627** [https://datatracker.ietf.org/doc/html/rfc4627.html] exige que o valor de nível superior do texto JSON deve ser do tipo object ou array (`dict` ou `list` Python), e não poderia ser dos tipos null, boolean, number ou string. **RFC 7159** [https://datatracker.ietf.org/doc/html/rfc7159.html] removeu essa restrição, e esse módulo não tem nenhuma implementação que faça essa restrição, seja em seus serializadores, sejam nos desserializadores.

Independentemente, para máxima interoperabilidade, você pode querer aderir voluntariamente à restrição.

Limitações de implementação

Algumas implementações de desserializadores JSON podem definir limites em:

- o tamanho de textos JSON aceitos
- o nível máximo de aninhamento de objetos e vetores JSON
- o intervalo e a precisão de números JSON
- o conteúdo e o tamanho máximo de strings JSON

Esse módulo não impõe nenhum limite além daqueles já colocados pelas estruturas de dados Python ou pelo interpretador Python em si.

Quando serializando para JSON, tenha cuidado com qualquer limitação nas aplicações que irão consumir seu JSON. Em particular, é comum para números JSON serem desserializados com números de precisão dupla definida em IEEE 754 e, portanto, sujeito a limitações de precisão e de intervalo da representação. Isso é especialmente relevante quando serializando valores Python `int` de magnitude extremamente grande, ou

quando serializando instâncias de tipos numéricos “exóticos” como **decimal.Decimal**.

Interface de linha de comando

Código-fonte: Lib/json/tool.py
[https://github.com/python/cpython/tree/3.15/Lib/json/tool.py]

O módulo **json** pode ser invocado como um script via `python -m json` para validar e fazer uma impressão bem formatada de objetos JSON. O submódulo **json.tool** implementa essa interface.

Se os argumentos opcionais `infile` e `outfile` não forem especificados, **sys.stdin** e **sys.stdout** serão usados respectivamente:

```
$ echo '{"json": "obj"}' | python -m json
{
    "json": "obj"
}
$ echo '{1.2:3.4}' | python -m json
Expecting property name enclosed in double quotes: line 1 column
```

Alterado na versão 3.5: A saída agora está na mesma ordem da entrada. Use a opção **--sort-keys** para ordenar a saída de dicionários alfabeticamente pela chave.

Alterado na versão 3.14: O módulo **json** agora pode ser executado diretamente como `python -m json`. Para retrocompatibilidade, a invocação da CLI como `python -m json.tool` continua sendo aceita.

Opções de linha de comando

infile

O arquivo JSON para ser validado ou saída embelezada:

```
$ python -m json mp_films.json
[
```

```
    {
        "title": "And Now for Something Completely Different",
        "year": 1971
    },
    {
        "title": "Monty Python and the Holy Grail",
        "year": 1975
    }
]
```

Se *infile* não é especificado, lê de `sys.stdin`.

outfile

Escreve a saída de *infile* para o *outfile* dado. Caso contrário, escreve em `sys.stdout`.

--sort-keys

Ordena a saída de dicionários alfabeticamente pela chave.

Adicionado na versão 3.5.

--no-ensure-ascii

Desabilita escape de caracteres não-ascii, veja `json.dumps()` para mais informações.

Adicionado na versão 3.9.

--json-lines

Analisa cada linha da entrada como um objeto JSON separado.

Adicionado na versão 3.8.

--indent, --tab, --no-indent, --compact

Opções mutualmente exclusivas para controle de espaços em branco.

Adicionado na versão 3.9.

-h, --help

Exibe a mensagem de ajuda.

Notas de rodapé

[1]

Como apresentado na errata para RFC 7159 [https://www.rfc-editor.org/errata_search.php?rfc=7159], JSON permite os caracteres literais U+2028 (SEPARADOR DE LINHA) e U+2029 (SEPARADOR DE PARÁGRAFO) em strings, enquanto que JavaScript (ECMAScript Edition 5.1) não.

`mailbox` — Manipule caixas de entradas em vários formatos

Código-fonte: Lib/mailbox.py
[https://github.com/python/cpython/tree/3.15/Lib/mailbox.py]

Este módulo define duas classes, `Mailbox` e `Message`, para acessar e manipular caixas de correio em disco e as mensagens que elas contêm. `Mailbox` oferece um mapeamento semelhante a um dicionário de chaves para mensagens. `Message` estende a classe `Message` do módulo `email.message` com estado e comportamento específicos para cada formato. Os formatos de caixa de correio suportados são Maildir, mbox, MH, Babyl e MMDF.

Ver também

Módulo `email`
Representa e manipula mensagens.

Objetos `Mailbox`

class mailbox.Mailbox

Uma caixa de correio, que pode ser inspecionada e modificada.

A classe `Mailbox` define uma interface e não deve ser instanciada. Em vez disso, subclasses específicas de cada formato devem herdar de `Mailbox` e seu código deve instanciar uma subclasse específica.

A interface `Mailbox` é semelhante a um dicionário, com pequenas chaves correspondentes às mensagens. As chaves são emitidas pela instância de `Mailbox` com a qual serão usadas e só têm significado para

essa instância de **`Mailbox`**. Uma chave continua a identificar uma mensagem mesmo que a mensagem correspondente seja modificada, por exemplo, ao ser substituída por outra mensagem.

As mensagens podem ser adicionadas a uma instância de **`Mailbox`** usando o método semelhante a um conjunto **`add()`** e removidas usando uma instrução `del` ou os métodos semelhantes a conjuntos **`remove()`** e **`discard()`**.

A semântica da interface **`Mailbox`** difere da semântica de dicionário em alguns aspectos importantes. Cada vez que uma mensagem é solicitada, uma nova representação (normalmente uma instância de **`Message`**) é gerada com base no estado atual da caixa de correio. Da mesma forma, quando uma mensagem é adicionada a uma instância de **`Mailbox`**, o conteúdo da representação da mensagem fornecida é copiado. Em nenhum dos casos a instância de **`Mailbox`** mantém uma referência à representação da mensagem.

O iterador padrão de **`Mailbox`** itera sobre representações de mensagens, e não sobre chaves como o iterador padrão de **`dicionário`**. Além disso, a modificação de uma caixa de correio durante a iteração é segura e bem definida. Mensagens adicionadas à caixa de correio após a criação de um iterador não serão vistas pelo iterador. Mensagens removidas da caixa de correio antes que o iterador as produza serão ignoradas silenciosamente, embora o uso de uma chave de um iterador possa resultar em uma exceção **`KeyError`** se a mensagem correspondente for posteriormente removida.

Aviso

Be very cautious when modifying mailboxes that might be simultaneously changed by some other process. The safest mailbox format to use for such tasks is **`Maildir`**; try to avoid using single-file formats such as **`mbox`** for concurrent writing. If you're modifying a mailbox, you *must* lock it by calling the **`lock()`** and **`unlock()`** methods *before* reading any messages in the file or making any changes by adding or deleting a message. Failing to lock the mailbox runs the risk of losing messages or corrupting the entire mailbox.

The **`Mailbox`** class supports the **`with`** statement. When used as a context manager, **`Mailbox`** calls **`lock()`** when the context is entered, returns the mailbox object as the context object, and at context end calls **`close()`**, thereby releasing the lock.

Alterado na versão 3.15: Suporte para a instrução **`with`** foi adicionado.

`Mailbox` instances have the following methods:

add(*message*)

Add *message* to the mailbox and return the key that has been assigned to it.

Parameter *message* may be a **`Message`** instance, an **`email.message.Message`** instance, a string, a byte string, or a file-like object (which should be open in binary mode). If *message* is an instance of the appropriate format-specific **`Message`** subclass (e.g., if it's an **`mboxMessage`** instance and this is an **`mbox`** instance), its format-specific information is used. Otherwise, reasonable defaults for format-specific information are used.

Alterado na versão 3.2: Support for binary input was added.

remove(*key*)
__delitem__(*key*)
discard(*key*)

Delete the message corresponding to *key* from the mailbox.

If no such message exists, a **`KeyError`** exception is raised if the method was called as **`remove()`** or **`__delitem__()`** but no exception is raised if the method was called as **`discard()`**. The behavior of **`discard()`** may be preferred if the underlying mailbox format supports concurrent modification by other processes.

__setitem__(*key*, *message*)

Replace the message corresponding to *key* with *message*. Raise a `KeyError` exception if no message already corresponds to *key*.

As with `add()`, parameter *message* may be a `Message` instance, an `email.message.Message` instance, a string, a byte string, or a file-like object (which should be open in binary mode). If *message* is an instance of the appropriate format-specific `Message` subclass (e.g., if it's an `mboxMessage` instance and this is an `mbox` instance), its format-specific information is used. Otherwise, the format-specific information of the message that currently corresponds to *key* is left unchanged.

iterkeys()

Return an iterator over all keys

keys()

The same as `iterkeys()`, except that a `list` is returned rather than an iterator

itervalues()

__iter__()

Return an iterator over representations of all messages. The messages are represented as instances of the appropriate format-specific `Message` subclass unless a custom message factory was specified when the `Mailbox` instance was initialized.

Nota

The behavior of `__iter__()` is unlike that of dictionaries, which iterate over keys.

values()

The same as `itervalues()`, except that a `list` is returned rather than an iterator

iteritems()

Return an iterator over (*key*, *message*) pairs, where *key* is a key and *message* is a message representation. The messages are represented as instances of the appropriate format-specific **`Message`** subclass unless a custom message factory was specified when the **`Mailbox`** instance was initialized.

items()

The same as **`iteritems()`**, except that a **`list`** of pairs is returned rather than an iterator of pairs.

get(*key*, *default=None*)

__getitem__(*key*)

Return a representation of the message corresponding to *key*. If no such message exists, *default* is returned if the method was called as **`get()`** and a **`KeyError`** exception is raised if the method was called as **`__getitem__()`**. The message is represented as an instance of the appropriate format-specific **`Message`** subclass unless a custom message factory was specified when the **`Mailbox`** instance was initialized.

get_message(*key*)

Return a representation of the message corresponding to *key* as an instance of the appropriate format-specific **`Message`** subclass, or raise a **`KeyError`** exception if no such message exists.

get_bytes(*key*)

Return a byte representation of the message corresponding to *key*, or raise a **`KeyError`** exception if no such message exists.

Adicionado na versão 3.2.

get_string(*key*)

Return a string representation of the message corresponding to *key*, or raise a **`KeyError`** exception if no such message exists. The message is processed through **`email.message.Message`** to convert it to a 7bit clean representation.

get_file(*key*)

Return a file-like representation of the message corresponding to *key*, or raise a **`KeyError`** exception if no such message exists. The file-like object behaves as if open in binary mode. This file should be closed once it is no longer needed.

Alterado na versão 3.2: The file object really is a binary file; previously it was incorrectly returned in text mode. Also, the file-like object now supports the context manager protocol: you can use a **`with`** statement to automatically close it.

Nota

Unlike other representations of messages, file-like representations are not necessarily independent of the **`Mailbox`** instance that created them or of the underlying mailbox. More specific documentation is provided by each subclass.

__contains__(*key*)

Return `True` if *key* corresponds to a message, `False` otherwise.

__len__()

Return a count of messages in the mailbox.

clear()

Delete all messages from the mailbox.

pop(*key*, *default=None*)

Return a representation of the message corresponding to *key* and delete the message. If no such message exists, return *default*. The message is represented as an instance of the appropriate format-specific **`Message`** subclass unless a custom message factory was specified when the **`Mailbox`** instance was initialized.

popitem()

Return an arbitrary (*key*, *message*) pair, where *key* is a key and *message* is a message representation, and delete the corresponding message. If the mailbox is empty, raise a **KeyError** exception. The message is represented as an instance of the appropriate format-specific **Message** subclass unless a custom message factory was specified when the **Mailbox** instance was initialized.

update(*arg*)

Parameter *arg* should be a *key*-to-*message* mapping or an iterable of (*key*, *message*) pairs. Updates the mailbox so that, for each given *key* and *message*, the message corresponding to *key* is set to *message* as if by using **__setitem__()**. As with **__setitem__()**, each *key* must already correspond to a message in the mailbox or else a **KeyError** exception will be raised, so in general it is incorrect for *arg* to be a **Mailbox** instance.

Nota

Unlike with dictionaries, keyword arguments are not supported.

flush()

Write any pending changes to the filesystem. For some **Mailbox** subclasses, changes are always written immediately and **flush()** does nothing, but you should still make a habit of calling this method.

lock()

Acquire an exclusive advisory lock on the mailbox so that other processes know not to modify it. An **ExternalClashError** is raised if the lock is not available. The particular locking mechanisms used depend upon the mailbox format. You should *always* lock the mailbox before making any modifications to its contents.

unlock()

Release the lock on the mailbox, if any.

close()

Flush the mailbox, unlock it if necessary, and close any open files. For some **`Mailbox`** subclasses, this method does nothing.

`Maildir` objects

class mailbox.Maildir(*dirname*, *factory=None*, *create=True*)

A subclass of **`Mailbox`** for mailboxes in Maildir format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, **`MaildirMessage`** is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

If *create* is `True` and the *dirname* path exists, it will be treated as an existing maildir without attempting to verify its directory layout.

It is for historical reasons that *dirname* is named as such rather than *path*.

Maildir is a directory-based mailbox format invented for the qmail mail transfer agent and now widely supported by other programs. Messages in a Maildir mailbox are stored in separate files within a common directory structure. This design allows Maildir mailboxes to be accessed and modified by multiple unrelated programs without data corruption, so file locking is unnecessary.

Maildir mailboxes contain three subdirectories, namely: `tmp`, `new`, and `cur`. Messages are created momentarily in the `tmp` subdirectory and then moved to the `new` subdirectory to finalize delivery. A mail user agent may subsequently move the message to the `cur` subdirectory and store information about the state of the message in a special “info” section appended to its file name.

Folders of the style introduced by the Courier mail transfer agent are also supported. Any subdirectory of the main mailbox is considered a folder if `'.'` is the first character in its name. Folder names are

represented by **Maildir** without the leading `'.'`. Each folder is itself a Maildir mailbox but should not contain other folders. Instead, a logical nesting is indicated using `'.'` to delimit levels, e.g., “Archived.2005.07”.

colon

The Maildir specification requires the use of a colon (`':'`) in certain message file names. However, some operating systems do not permit this character in file names, If you wish to use a Maildir-like format on such an operating system, you should specify another character to use instead. The exclamation point (`'!'`) is a popular choice. For example:

```
import mailbox
mailbox.Maildir.colon = '!'
```

The **colon** attribute may also be set on a per-instance basis.

Alterado na versão 3.13: **Maildir** now ignores files with a leading dot.

Maildir instances have all of the methods of **Mailbox** in addition to the following:

list_folders()

Return a list of the names of all folders.

get_folder(*folder*)

Return a **Maildir** instance representing the folder whose name is *folder*. A **NoSuchMailboxError** exception is raised if the folder does not exist.

add_folder(*folder*)

Create a folder whose name is *folder* and return a **Maildir** instance representing it.

remove_folder(*folder*)

Delete the folder whose name is *folder*. If the folder contains any messages, a `NotEmptyError` exception will be raised and the folder will not be deleted.

clean()

Delete temporary files from the mailbox that have not been accessed in the last 36 hours. The Maildir specification says that mail-reading programs should do this occasionally.

get_flags(*key*)

Return as a string the flags that are set on the message corresponding to *key*. This is the same as `get_message(key).get_flags()` but much faster, because it does not open the message file. Use this method when iterating over the keys to determine which messages are interesting to get.

If you do have a `MaildirMessage` object, use its `get_flags()` method instead, because changes made by the message's `set_flags()`, `add_flag()` and `remove_flag()` methods are not reflected here until the mailbox's `__setitem__()` method is called.

Adicionado na versão 3.13.

set_flags(*key*, *flags*)

On the message corresponding to *key*, set the flags specified by *flags* and unset all others. Calling `some_mailbox.set_flags(key, flags)` is similar to

```
one_message = some_mailbox.get_message(key)
one_message.set_flags(flags)
some_mailbox[key] = one_message
```

but faster, because it does not open the message file.

If you do have a `MaildirMessage` object, use its `set_flags()` method instead, because changes made with this mailbox method will not be visible to the message object's method, `get_flags()`.

Adicionado na versão 3.13.

add_flag(*key*, *flag*)

On the message corresponding to *key*, set the flags specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character.

Considerations for using this method versus the message object's `add_flag()` method are similar to those for `set_flags()`; see the discussion there.

Adicionado na versão 3.13.

remove_flag(*key*, *flag*)

On the message corresponding to *key*, unset the flags specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* may be a string of more than one character.

Considerations for using this method versus the message object's `remove_flag()` method are similar to those for `set_flags()`; see the discussion there.

Adicionado na versão 3.13.

get_info(*key*)

Return a string containing the info for the message corresponding to *key*. This is the same as `get_message(key).get_info()` but much faster, because it does not open the message file. Use this method when iterating over the keys to determine which messages are interesting to get.

If you do have a `MaildirMessage` object, use its `get_info()` method instead, because changes made by the message's `set_info()` method are not reflected here until the mailbox's `__setitem__()` method is called.

Adicionado na versão 3.13.

set_info(*key*, *info*)

Set the info of the message corresponding to *key* to *info*. Calling `some_mailbox.set_info(key, flags)` is similar to

```
one_message = some_mailbox.get_message(key)
one_message.set_info(info)
some_mailbox[key] = one_message
```

but faster, because it does not open the message file.

If you do have a **`MaildirMessage`** object, use its **`set_info()`** method instead, because changes made with this mailbox method will not be visible to the message object's method, **`get_info()`**.

Adicionado na versão 3.13.

Some **`Mailbox`** methods implemented by **`Maildir`** deserve special remarks:

add(*message*)

__setitem__(*key*, *message*)

update(*arg*)

> **Aviso**
>
> These methods generate unique file names based upon the current process ID. When using multiple threads, undetected name clashes may occur and cause corruption of the mailbox unless threads are coordinated to avoid using these methods to manipulate the same mailbox simultaneously.

flush()

All changes to Maildir mailboxes are immediately applied, so this method does nothing.

lock()

unlock()

Maildir mailboxes do not support (or require) locking, so these methods do nothing.

close()

`Maildir` instances do not keep any open files and the underlying mailboxes do not support locking, so this method does nothing.

get_file(*key*)

Depending upon the host platform, it may not be possible to modify or remove the underlying message while the returned file remains open.

Ver também

maildir man page from Courier **[https://www.courier-mta.org/maildir.html]**
A specification of the format. Describes a common extension for supporting folders.

Using maildir format **[https://cr.yp.to/proto/maildir.html]**
Notes on Maildir by its inventor. Includes an updated name-creation scheme and details on “info” semantics.

`mbox` objects

class mailbox.mbox(*path*, *factory=None*, *create=True*)

A subclass of **`Mailbox`** for mailboxes in mbox format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, **`mboxMessage`** is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

The mbox format is the classic format for storing mail on Unix systems. All messages in an mbox mailbox are stored in a single file with the

beginning of each message indicated by a line whose first five characters are “From “.

Several variations of the mbox format exist to address perceived shortcomings in the original. In the interest of compatibility, **`mbox`** implements the original format, which is sometimes referred to as *mboxo*. This means that the *Content-Length* header, if present, is ignored and that any occurrences of “From “ at the beginning of a line in a message body are transformed to “>From “ when storing the message, although occurrences of “>From “ are not transformed to “From “ when reading the message.

Some **`Mailbox`** methods implemented by **`mbox`** deserve special remarks:

get_bytes(*key*, *from_=False*)

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If *from_* is False, the first line of the file is dropped.

get_file(*key*, *from_=False*)

Using the file after calling **`flush()`** or **`close()`** on the **`mbox`** instance may yield unpredictable results or raise an exception.

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If *from_* is False, the first line of the file is dropped.

get_string(*key*, *from_=False*)

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If *from_* is False, the first line of the file is dropped.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the **`flock()`** and **`lockf()`** system calls.

Ver também

mbox man page from tin **[http://www.tin.org/bin/man.cgi?section=5&topic=mbox]**
A specification of the format, with details on locking.

Configuring Netscape Mail on Unix: Why The Content-Length Format is Bad **[https://www.jwz.org/doc/content-length.html]**
An argument for using the original mbox format rather than a variation.

"mbox" is a family of several mutually incompatible mailbox formats
[https://www.loc.gov/preservation/digital/formats/fdd/fdd000383.shtml]
A history of mbox variations.

`MH` objects

class mailbox.MH(*path*, *factory=None*, *create=True*)

A subclass of `Mailbox` for mailboxes in MH format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, `MHMessage` is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

MH is a directory-based mailbox format invented for the MH Message Handling System, a mail user agent. Each message in an MH mailbox resides in its own file. An MH mailbox may contain other MH mailboxes (called *folders*) in addition to messages. Folders may be nested indefinitely. MH mailboxes also support *sequences*, which are named lists used to logically group messages without moving them to sub-folders. Sequences are defined in a file called `.mh_sequences` in each folder.

The `MH` class manipulates MH mailboxes, but it does not attempt to emulate all of **mh**'s behaviors. In particular, it does not modify and is

not affected by the `context` or `.mh_profile` files that are used by **mh** to store its state and configuration.

`MH` instances have all of the methods of **`Mailbox`** in addition to the following:

Alterado na versão 3.13: Supported folders that don't contain a `.mh_sequences` file.

list_folders()

Return a list of the names of all folders.

get_folder(*folder*)

Return an **`MH`** instance representing the folder whose name is *folder*. A **`NoSuchMailboxError`** exception is raised if the folder does not exist.

add_folder(*folder*)

Create a folder whose name is *folder* and return an **`MH`** instance representing it.

remove_folder(*folder*)

Delete the folder whose name is *folder*. If the folder contains any messages, a **`NotEmptyError`** exception will be raised and the folder will not be deleted.

get_sequences()

Return a dictionary of sequence names mapped to key lists. If there are no sequences, the empty dictionary is returned.

set_sequences(*sequences*)

Re-define the sequences that exist in the mailbox based upon *sequences*, a dictionary of names mapped to key lists, like returned by **`get_sequences()`**.

pack()

Rename messages in the mailbox as necessary to eliminate gaps in numbering. Entries in the sequences list are updated correspondingly.

Nota

Already-issued keys are invalidated by this operation and should not be subsequently used.

Some `Mailbox` methods implemented by `MH` deserve special remarks:

remove(*key*)
__delitem__(*key*)
discard(*key*)

These methods immediately delete the message. The MH convention of marking a message for deletion by prepending a comma to its name is not used.

lock()
unlock()

Three locking mechanisms are used—dot locking and, if available, the `flock()` and `lockf()` system calls. For MH mailboxes, locking the mailbox means locking the `.mh_sequences` file and, only for the duration of any operations that affect them, locking individual message files.

get_file(*key*)

Depending upon the host platform, it may not be possible to remove the underlying message while the returned file remains open.

flush()

All changes to MH mailboxes are immediately applied, so this method does nothing.

close()

`MH` instances do not keep any open files, so this method is equivalent to `unlock()`.

Ver também

nmh - Message Handling System **[https://www.nongnu.org/nmh/]**
Home page of **nmh**, an updated version of the original **mh**.

MH & nmh: Email for Users & Programmers **[https://rand-mh.sourceforge.io/book/]**
A GPL-licensed book on **mh** and **nmh**, with some information on the mailbox format.

`Babyl` objects

class mailbox.Babyl(*path*, *factory=None*, *create=True*)

A subclass of `Mailbox` for mailboxes in Babyl format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, `BabylMessage` is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

Babyl is a single-file mailbox format used by the Rmail mail user agent included with Emacs. The beginning of a message is indicated by a line containing the two characters Control-Underscore (`'\037'`) and Control-L (`'\014'`). The end of a message is indicated by the start of the next message or, in the case of the last message, a line containing a Control-Underscore (`'\037'`) character.

Messages in a Babyl mailbox have two sets of headers, original headers and so-called visible headers. Visible headers are typically a subset of the original headers that have been reformatted or abridged to be more attractive. Each message in a Babyl mailbox also has an accompanying list of *labels*, or short strings that record extra information about the

message, and a list of all user-defined labels found in the mailbox is kept in the Babyl options section.

`Babyl` instances have all of the methods of **`Mailbox`** in addition to the following:

get_labels()

Return a list of the names of all user-defined labels used in the mailbox.

> **Nota**
>
> The actual messages are inspected to determine which labels exist in the mailbox rather than consulting the list of labels in the Babyl options section, but the Babyl section is updated whenever the mailbox is modified.

Some **`Mailbox`** methods implemented by **`Babyl`** deserve special remarks:

get_file(*key*)

In Babyl mailboxes, the headers of a message are not stored contiguously with the body of the message. To generate a file-like representation, the headers and body are copied together into an **`io.BytesIO`** instance, which has an API identical to that of a file. As a result, the file-like object is truly independent of the underlying mailbox but does not save memory compared to a string representation.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the **`flock()`** and **`lockf()`** system calls.

Ver também

Format of Version 5 Babyl Files **[https://quimby.gnus.org/notes/BABYL]**

A specification of the Babyl format.

Reading Mail with Rmail
[https://www.gnu.org/software/emacs/manual/html_node/emacs/Rmail.html]

The Rmail manual, with some information on Babyl semantics.

`MMDF` objects

class mailbox.MMDF(*path*, *factory=None*, *create=True*)

A subclass of `Mailbox` for mailboxes in MMDF format. Parameter *factory* is a callable object that accepts a file-like message representation (which behaves as if opened in binary mode) and returns a custom representation. If *factory* is `None`, `MMDFMessage` is used as the default message representation. If *create* is `True`, the mailbox is created if it does not exist.

MMDF is a single-file mailbox format invented for the Multichannel Memorandum Distribution Facility, a mail transfer agent. Each message is in the same form as an mbox message but is bracketed before and after by lines containing four Control-A (`'\001'`) characters. As with the mbox format, the beginning of each message is indicated by a line whose first five characters are “From “, but additional occurrences of “From “ are not transformed to “>From “ when storing messages because the extra message separator lines prevent mistaking such occurrences for the starts of subsequent messages.

Some `Mailbox` methods implemented by `MMDF` deserve special remarks:

get_bytes(*key*, *from_=False*)

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If *from_* is False, the first line of the file is dropped.

get_file(*key*, *from_=False*)

Using the file after calling `flush()` or `close()` on the `MMDF` instance may yield unpredictable results or raise an exception.

Note: This method has an extra parameter (*from_*) compared with other classes. The first line of an mbox file entry is the Unix “From “ line. If *from_* is False, the first line of the file is dropped.

lock()

unlock()

Three locking mechanisms are used—dot locking and, if available, the **`flock()`** and **`lockf()`** system calls.

Ver também

mmdf man page from tin **[http://www.tin.org/bin/man.cgi?section=5&topic=mmdf]**
A specification of MMDF format from the documentation of tin, a newsreader.

MMDF **[https://en.wikipedia.org/wiki/MMDF]**
A Wikipedia article describing the Multichannel Memorandum Distribution Facility.

`Message` objects

class mailbox.Message(*message=None*)

A subclass of the **`email.message`** module’s **`Message`**. Subclasses of **`mailbox.Message`** add mailbox-format-specific state and behavior.

If *message* is omitted, the new instance is created in a default, empty state. If *message* is an **`email.message.Message`** instance, its contents are copied; furthermore, any format-specific information is converted insofar as possible if *message* is a **`Message`** instance. If *message* is a string, a byte string, or a file, it should contain an **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html]-compliant message, which is read and parsed. Files should be open in binary mode, but text mode files are accepted for backward compatibility.

The format-specific state and behaviors offered by subclasses vary, but in general it is only the properties that are not specific to a particular mailbox that are supported (although presumably the properties are specific to a particular mailbox format). For example, file offsets for single-file mailbox formats and file names for directory-based mailbox formats are not retained, because they are only applicable to the original mailbox. But state such as whether a message has been read by the user or marked as important is retained, because it applies to the message itself.

There is no requirement that **`Message`** instances be used to represent messages retrieved using **`Mailbox`** instances. In some situations, the time and memory required to generate **`Message`** representations might not be acceptable. For such situations, **`Mailbox`** instances also offer string and file-like representations, and a custom message factory may be specified when a **`Mailbox`** instance is initialized.

`MaildirMessage` objects

class mailbox.MaildirMessage(*message=None*)

A message with Maildir-specific behaviors. Parameter *message* has the same meaning as with the **`Message`** constructor.

Typically, a mail user agent application moves all of the messages in the `new` subdirectory to the `cur` subdirectory after the first time the user opens and closes the mailbox, recording that the messages are old whether or not they've actually been read. Each message in `cur` has an "info" section added to its file name to store information about its state. (Some mail readers may also add an "info" section to messages in `new`.) The "info" section may take one of two forms: it may contain "2," followed by a list of standardized flags (e.g., "2,FR") or it may contain "1," followed by so-called experimental information. Standard flags for Maildir messages are as follows:

Sinalizador	Significado	Explicação

Sinalizador	Significado	Explicação
D	Draft	Under composition
F	Flagged	Marked as important
P	Passed	Forwarded, resent, or bounced
R	Replied	Replied to
S	Seen	Read
T	Trashed	Marked for subsequent deletion

`MaildirMessage` instances offer the following methods:

get_subdir()

Return either “new” (if the message should be stored in the `new` subdirectory) or “cur” (if the message should be stored in the `cur` subdirectory).

Nota

A message is typically moved from `new` to `cur` after its mailbox has been accessed, whether or not the message has been read. A message `msg` has been read if `"S" in msg.get_flags()` is `True`.

set_subdir(*subdir*)

Set the subdirectory the message should be stored in. Parameter *subdir* must be either “new” or “cur”.

get_flags()

Return a string specifying the flags that are currently set. If the message complies with the standard Maildir format, the result is the concatenation in alphabetical order of zero or one occurrence of each of `'D'`, `'F'`, `'P'`, `'R'`, `'S'`, and `'T'`. The empty string is

returned if no flags are set or if “info” contains experimental semantics.

set_flags(*flags*)

Set the flags specified by *flags* and unset all others.

add_flag(*flag*)

Set the flag(s) specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character. The current “info” is overwritten whether or not it contains experimental information rather than flags.

remove_flag(*flag*)

Unset the flag(s) specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* may be a string of more than one character. If “info” contains experimental information rather than flags, the current “info” is not modified.

get_date()

Return the delivery date of the message as a floating-point number representing seconds since the epoch.

set_date(*date*)

Set the delivery date of the message to *date*, a floating-point number representing seconds since the epoch.

get_info()

Return a string containing the “info” for a message. This is useful for accessing and modifying “info” that is experimental (i.e., not a list of flags).

set_info(*info*)

Set “info” to *info*, which should be a string.

When a **`MaildirMessage`** instance is created based upon an **`mboxMessage`** or **`MMDFMessage`** instance, the *Status* and *X-Status* headers are omitted and the

following conversions take place:

Resulting state	`mboxMessage` **or** `MMDFMessage` **state**
“cur” subdirectory	O flag
F flag	F flag
R flag	A flag
S flag	R flag
T flag	D flag

When a **`MaildirMessage`** instance is created based upon an `MHMessage` instance, the following conversions take place:

Resulting state	`MHMessage` **state**
“cur” subdirectory	“unseen” sequence
“cur” subdirectory and S flag	no “unseen” sequence
F flag	“flagged” sequence
R flag	“replied” sequence

When a **`MaildirMessage`** instance is created based upon a **`BabylMessage`** instance, the following conversions take place:

Resulting state	`BabylMessage` state
“cur” subdirectory	“unseen” label
“cur” subdirectory and S flag	no “unseen” label
P flag	“forwarded” or “resent” label
R flag	“answered” label
T flag	“deleted” label

`mboxMessage` objects

class mailbox.mboxMessage(*message=None*)

A message with mbox-specific behaviors. Parameter *message* has the same meaning as with the **`Message`** constructor.

Messages in an mbox mailbox are stored together in a single file. The sender’s envelope address and the time of delivery are typically stored in a line beginning with “From “ that is used to indicate the start of a message, though there is considerable variation in the exact format of this data among mbox implementations. Flags that indicate the state of the message, such as whether it has been read or marked as important, are typically stored in *Status* and *X-Status* headers.

Conventional flags for mbox messages are as follows:

Sinalizador	Significado	Explicação
R	Read	Read
O	Antigo	Previously detected by MUA
D	Deleted	Marked for subsequent deletion
F	Flagged	Marked as important
A	Answered	Replied to

The “R” and “O” flags are stored in the *Status* header, and the “D”, “F”, and “A” flags are stored in the *X-Status* header. The flags and headers typically appear in the order mentioned.

`mboxMessage` instances offer the following methods:

get_from()

Return a string representing the “From “ line that marks the start of the message in an mbox mailbox. The leading “From “ and the trailing newline are excluded.

set_from(*from_*, *time_=None*)

Set the “From “ line to *from_*, which should be specified without a leading “From “ or trailing newline. For convenience, *time_* may be specified and will be formatted appropriately and appended to *from_*. If *time_* is specified, it should be a `time.struct_time` instance, a tuple suitable for passing to `time.strftime()`, or `True` (to use `time.gmtime()`).

get_flags()

Return a string specifying the flags that are currently set. If the message complies with the conventional format, the result is the concatenation in the following order of zero or one occurrence of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

set_flags(*flags*)

Set the flags specified by *flags* and unset all others. Parameter *flags* should be the concatenation in any order of zero or more occurrences of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

add_flag(*flag*)

Set the flag(s) specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character.

remove_flag(*flag*)

Unset the flag(s) specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* may be a string of more than one character.

When an **`mboxMessage`** instance is created based upon a **`MaildirMessage`** instance, a “From “ line is generated based upon the **`MaildirMessage`** instance’s delivery date, and the following conversions take place:

Resulting state	**`MaildirMessage` state**
R flag	S flag
O flag	“cur” subdirectory
D flag	T flag
F flag	F flag
A flag	R flag

When an **`mboxMessage`** instance is created based upon an **`MHMessage`** instance, the following conversions take place:

Resulting state	**`MHMessage` state**
R flag and O flag	no “unseen” sequence
O flag	“unseen” sequence
F flag	“flagged” sequence
A flag	“replied” sequence

When an **`mboxMessage`** instance is created based upon a **`BabylMessage`** instance, the following conversions take place:

Resulting state	**`BabylMessage` state**
R flag and O flag	no “unseen” label
O flag	“unseen” label
D flag	“deleted” label
A flag	“answered” label

When a **mboxMessage** instance is created based upon an **MMDFMessage** instance, the “From “ line is copied and all flags directly correspond:

Resulting state	MMDFMessage state
R flag	R flag
O flag	O flag
D flag	D flag
F flag	F flag
A flag	A flag

MHMessage objects

class mailbox.MHMessage(*message=None*)

A message with MH-specific behaviors. Parameter *message* has the same meaning as with the **Message** constructor.

MH messages do not support marks or flags in the traditional sense, but they do support sequences, which are logical groupings of arbitrary messages. Some mail reading programs (although not the standard **mh** and **nmh**) use sequences in much the same way flags are used with other formats, as follows:

Sequence	Explicação
unseen	Not read, but previously detected by MUA

Sequence	Explicação
replied	Replied to
flagged	Marked as important

`MHMessage` instances offer the following methods:

get_sequences()
: Return a list of the names of sequences that include this message.

set_sequences(*sequences*)
: Set the list of sequences that include this message.

add_sequence(*sequence*)
: Add *sequence* to the list of sequences that include this message.

remove_sequence(*sequence*)
: Remove *sequence* from the list of sequences that include this message.

When an **`MHMessage`** instance is created based upon a **`MaildirMessage`** instance, the following conversions take place:

Resulting state	`MaildirMessage` state
“unseen” sequence	no S flag
“replied” sequence	R flag
“flagged” sequence	F flag

When an **MHMessage** instance is created based upon an mboxMessage or MMDFMessage instance, the *Status* and *X-Status* headers are omitted and the following conversions take place:

Resulting state	mboxMessage or MMDFMessage state
“unseen” sequence	no R flag
“replied” sequence	A flag
“flagged” sequence	F flag

When an **MHMessage** instance is created based upon a BabylMessage instance, the following conversions take place:

Resulting state	BabylMessage state
“unseen” sequence	“unseen” label
“replied” sequence	“answered” label

BabylMessage objects

class mailbox.BabylMessage(*message=None*)

A message with Babyl-specific behaviors. Parameter *message* has the same meaning as with the Message constructor.

Certain message labels, called *attributes*, are defined by convention to have special meanings. The attributes are as follows:

Label	Explicação
unseen	Not read, but previously detected by MUA
deleted	Marked for subsequent deletion
filed	Copied to another file or mailbox
answered	Replied to
forwarded	Forwarded
edited	Modified by the user
resent	Resent

By default, Rmail displays only visible headers. The `BabylMessage` class, though, uses the original headers because they are more complete. Visible headers may be accessed explicitly if desired.

`BabylMessage` instances offer the following methods:

get_labels()
: Return a list of labels on the message.

set_labels(*labels*)
: Set the list of labels on the message to *labels*.

add_label(*label*)
: Add *label* to the list of labels on the message.

remove_label(*label*)
: Remove *label* from the list of labels on the message.

get_visible()

Return a `Message` instance whose headers are the message's visible headers and whose body is empty.

set_visible(*visible*)

Set the message's visible headers to be the same as the headers in *message*. Parameter *visible* should be a `Message` instance, an `email.message.Message` instance, a string, or a file-like object (which should be open in text mode).

update_visible()

When a **`BabylMessage`** instance's original headers are modified, the visible headers are not automatically modified to correspond. This method updates the visible headers as follows: each visible header with a corresponding original header is set to the value of the original header, each visible header without a corresponding original header is removed, and any of *Date*, *From*, *Reply-To*, *To*, *CC*, and *Subject* that are present in the original headers but not the visible headers are added to the visible headers.

When a **`BabylMessage`** instance is created based upon a `MaildirMessage` instance, the following conversions take place:

Resulting state	`MaildirMessage` **state**
"unseen" label	no S flag
"deleted" label	T flag
"answered" label	R flag
"forwarded" label	P flag

When a **`BabylMessage`** instance is created based upon an **`mboxMessage`** or **`MMDFMessage`** instance, the *Status* and *X-Status* headers are omitted and the following conversions take place:

Resulting state	**`mboxMessage`** **or** **`MMDFMessage`** **state**
“unseen” label	no R flag
“deleted” label	D flag
“answered” label	A flag

When a **`BabylMessage`** instance is created based upon an **`MHMessage`** instance, the following conversions take place:

Resulting state	**`MHMessage`** **state**
“unseen” label	“unseen” sequence
“answered” label	“replied” sequence

`MMDFMessage` objects

class mailbox.MMDFMessage(*message=None*)

A message with MMDF-specific behaviors. Parameter *message* has the same meaning as with the **`Message`** constructor.

As with message in an mbox mailbox, MMDF messages are stored with the sender’s address and the delivery date in an initial line beginning

with “From “. Likewise, flags that indicate the state of the message are typically stored in *Status* and *X-Status* headers.

Conventional flags for MMDF messages are identical to those of mbox message and are as follows:

Sinalizador	**Significado**	**Explicação**
R	Read	Read
O	Antigo	Previously detected by MUA
D	Deleted	Marked for subsequent deletion
F	Flagged	Marked as important
A	Answered	Replied to

The “R” and “O” flags are stored in the *Status* header, and the “D”, “F”, and “A” flags are stored in the *X-Status* header. The flags and headers typically appear in the order mentioned.

`MMDFMessage` instances offer the following methods, which are identical to those offered by `mboxMessage`:

get_from()

Return a string representing the “From “ line that marks the start of the message in an mbox mailbox. The leading “From “ and the trailing newline are excluded.

set_from(*from_*, *time_=None*)

Set the “From “ line to *from_*, which should be specified without a leading “From “ or trailing newline. For convenience, *time_* may be specified and will be formatted appropriately and appended to *from_*. If *time_* is specified, it should be a `time.struct_time` instance, a tuple suitable for passing to `time.strftime()`, or `True` (to use `time.gmtime()`).

get_flags()

Return a string specifying the flags that are currently set. If the message complies with the conventional format, the result is the concatenation in the following order of zero or one occurrence of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

set_flags(*flags*)

Set the flags specified by *flags* and unset all others. Parameter *flags* should be the concatenation in any order of zero or more occurrences of each of `'R'`, `'O'`, `'D'`, `'F'`, and `'A'`.

add_flag(*flag*)

Set the flag(s) specified by *flag* without changing other flags. To add more than one flag at a time, *flag* may be a string of more than one character.

remove_flag(*flag*)

Unset the flag(s) specified by *flag* without changing other flags. To remove more than one flag at a time, *flag* may be a string of more than one character.

When an **`MMDFMessage`** instance is created based upon a **`MaildirMessage`** instance, a “From “ line is generated based upon the **`MaildirMessage`** instance’s delivery date, and the following conversions take place:

Resulting state	**`MaildirMessage` state**
R flag	S flag
O flag	“cur” subdirectory
D flag	T flag

Resulting state	`MaildirMessage` state
F flag	F flag
A flag	R flag

When an **`MMDFMessage`** instance is created based upon an `MHMessage` instance, the following conversions take place:

Resulting state	`MHMessage` state
R flag and O flag	no “unseen” sequence
O flag	“unseen” sequence
F flag	“flagged” sequence
A flag	“replied” sequence

When an **`MMDFMessage`** instance is created based upon a `BabylMessage` instance, the following conversions take place:

Resulting state	`BabylMessage` state
R flag and O flag	no “unseen” label

Resulting state	`BabylMessage` state
O flag	“unseen” label
D flag	“deleted” label
A flag	“answered” label

When an **`MMDFMessage`** instance is created based upon an `mboxMessage` instance, the “From “ line is copied and all flags directly correspond:

Resulting state	`mboxMessage` state
R flag	R flag
O flag	O flag
D flag	D flag
F flag	F flag
A flag	A flag

Exceções

The following exception classes are defined in the **`mailbox`** module:

exception mailbox.Error

The base class for all other module-specific exceptions.

exception mailbox.NoSuchMailboxError

Raised when a mailbox is expected but is not found, such as when instantiating a **`Mailbox`** subclass with a path that does not exist (and with the *create* parameter set to `False`), or when opening a folder that does not exist.

exception mailbox.NotEmptyError

Raised when a mailbox is not empty but is expected to be, such as when deleting a folder that contains messages.

exception mailbox.ExternalClashError

Raised when some mailbox-related condition beyond the control of the program causes it to be unable to proceed, such as when failing to acquire a lock that another program already holds, or when a uniquely generated file name already exists.

exception mailbox.FormatError

Raised when the data in a file cannot be parsed, such as when an **`MH`** instance attempts to read a corrupted `.mh_sequences` file.

Exemplos

A simple example of printing the subjects of all messages in a mailbox that seem interesting:

```
import mailbox
for message in mailbox.mbox('~/mbox'):
    subject = message['subject']       # Could possibly be None.
    if subject and 'python' in subject.lower():
        print(subject)
```

To copy all mail from a Babyl mailbox to an MH mailbox, converting all of the format-specific information that can be converted:

```
import mailbox
destination = mailbox.MH('~/Mail')
destination.lock()
for message in mailbox.Babyl('~/RMAIL'):
    destination.add(mailbox.MHMessage(message))
destination.flush()
destination.unlock()
```

This example sorts mail from several mailing lists into different mailboxes, being careful to avoid mail corruption due to concurrent modification by other programs, mail loss due to interruption of the program, or premature termination due to malformed messages in the mailbox:

```
import mailbox
import email.errors

list_names = ('python-list', 'python-dev', 'python-bugs')

boxes = {name: mailbox.mbox('~/email/%s' % name) for name in lis
inbox = mailbox.Maildir('~/Maildir', factory=None)

for key in inbox.iterkeys():
    try:
        message = inbox[key]
    except email.errors.MessageParseError:
        continue                # The message is malformed. Just

    for name in list_names:
        list_id = message['list-id']
        if list_id and name in list_id:
            # Get mailbox to use
            box = boxes[name]

            # Write copy to disk before removing original.
            # If there's a crash, you might duplicate a message,
            # that's better than losing a message completely.
            box.lock()
            box.add(message)
            box.flush()
            box.unlock()

            # Remove original message
```

```
            inbox.lock()
            inbox.discard(key)
            inbox.flush()
            inbox.unlock()
            break               # Found destination, so stop loc

for box in boxes.itervalues():
    box.close()
```

`mimetypes` — Mapeia nomes de arquivos para tipos MIME

Código-fonte: Lib/mimetypes.py
[https://github.com/python/cpython/tree/3.15/Lib/mimetypes.py]

O módulo **`mimetypes`** converte entre um nome de arquivo ou URL e o tipo MIME associado à extensão do arquivo. As conversões são fornecidas do nome do arquivo para o tipo MIME e da extensão do tipo MIME para o nome do arquivo; codificações não são suportadas para a última conversão.

O módulo fornece uma classe e várias funções convenientes. As funções são a interface normal para este módulo, mas algumas aplicações também podem estar interessadas na classe.

As funções descritas abaixo fornecem a interface principal para este módulo. Se o módulo não foi inicializado, eles chamarão **`init()`** se confiarem nas informações configuradas por **`init()`**.

mimetypes.guess_type(*url*, *strict=True*)

Adivinha o tipo de arquivo com base em seu nome de arquivo, caminho ou URL, fornecido por *url*. A URL pode ser uma string ou um objeto caminho ou similar.

O valor de retorno é uma tupla `(type, encoding)` onde o *tipo* é `None` se o tipo não puder ser ser adivinhado (sufixo ausente ou desconhecido) ou uma string no formato `'type/subtype'`, utilizável para um cabeçalho MIME *content-type*.

encoding é `None` para nenhuma codificação ou o nome do programa usado para codificar (por exemplo **compress** ou **gzip**). A codificação é adequada para uso como cabeçalho *Content-Encoding* , **não** como cabeçalho *Content-Transfer-Encoding*. Os mapeamentos são orientados

por tabela. Os sufixos de codificação diferenciam maiúsculas de minúsculas; os sufixos de tipo são testados primeiro considerando maiúsculas e minúsculas e depois sem considerar.

O argumento opcional *strict* é um sinalizador que especifica se a lista de tipos MIME conhecidos é limitada apenas aos tipos oficiais registrados na IANA [https://www.iana.org/assignments/media-types/media-types.xhtml]. No entanto, o comportamento deste módulo também depende do sistema operacional subjacente. Somente tipos de arquivo reconhecidos pelo sistema operacional ou registrados explicitamente no banco de dados interno do Python podem ser identificados. Quando *strict* é `True` (o padrão), apenas os tipos IANA são suportados; quando *strict* é `False`, alguns tipos MIME adicionais não padronizados, mas geralmente usados, também são reconhecidos.

Alterado na versão 3.8: Adicionado suporte para que a *url* seja um objeto caminho ou similar.

Suavemente descontinuado desde a versão 3.13: Passar um caminho de arquivo em vez de URL. Use **`guess_file_type()`** para isso.

mimetypes.guess_file_type(*path*, ***, *strict=True*)

Adivinha o tipo de um arquivo com base em seu caminho, dado por *path*. Semelhante à função **`guess_type()`**, mas aceita um caminho em vez de URL. O caminho pode ser uma string, um objeto bytes ou um objeto caminho ou similar.

Adicionado na versão 3.13.

mimetypes.guess_all_extensions(*type*, *strict=True*)

Adivinha as extensões para um arquivo com base em seu tipo MIME, fornecido por *type*. O valor de retorno é uma lista de strings que fornecem todas as extensões possíveis de nome de arquivo, incluindo o ponto (`'.'`). Não é garantido que as extensões tenham sido associadas a qualquer fluxo de dados específico, mas seriam mapeadas para o tipo MIME *type* por **`guess_type()`** e **`guess_file_type()`**.

O argumento opcional *strict* tem o mesmo significado que usado com a função `guess_type()`.

mimetypes.guess_extension(*type*, *strict=True*)

Adivinha a extensão para um arquivo com base em seu tipo MIME, fornecido por *type*. O valor de retorno é uma string que fornece uma extensão de nome de arquivo, incluindo o ponto (`'.'`). Não é garantido que a extensão tenha sido associada a qualquer fluxo de dados específico, mas seria mapeada para o tipo MIME *type* por `guess_type()` e `guess_file_type()`. Se nenhuma extensão puder ser adivinhada para *type*, `None` é retornado.

O argumento opcional *strict* tem o mesmo significado que usado com a função `guess_type()`.

Algumas funções e dados adicionais estão disponíveis para controlar o comportamento do módulo.

mimetypes.init(*files=None*)

Inicializa as estruturas de dados internas. Se fornecido, *files* deverá ser uma sequência de nomes de arquivos que devem ser usados para aumentar o mapa de tipos padrão. Se for omitido, os nomes de arquivos a serem usados serão retirados de `knownfiles`; no Windows, as configurações atuais do registro são carregadas. Cada arquivo nomeado em *files* ou `knownfiles` tem precedência sobre os definidos anteriormente. É permitido chamar `init()` repetidamente.

Especificar uma lista vazia para *files* impedirá que os padrões do sistema sejam carregados: apenas os valores bem conhecidos estarão presentes em uma lista interna.

Se *files* for `None`, a estrutura de dados interna será completamente reconstruída com seu valor inicial padrão. Essa é uma operação estável e produzirá os mesmos resultados quando chamada várias vezes.

Alterado na versão 3.2: Anteriormente, as configurações do registro do Windows eram ignoradas.

mimetypes.read_mime_types(*file*)

Carrega o mapa de tipos fornecido no arquivo chamado *file*, se existir. *file* deve ser uma string que especifica o nome do arquivo a ser lido. O mapa de tipos é retornado como um dicionário que faz o mapeamento de extensões de arquivo, incluindo o ponto inicial (`'.'`), para strings no formato `'tipo/subtipo'`. Se o arquivo não existir ou não puder ser lido, `None` é retornado.

mimetypes.add_type(*type*, *ext*, *strict=True*)

Adiciona um mapeamento do tipo MIME *type* para a extensão *ext*. Quando a extensão já for conhecida, o novo tipo substituirá o antigo. Quando o tipo já for conhecido, a extensão será adicionada à lista de extensões conhecidas.

Quando *strict* for `True` (o padrão), o mapeamento será adicionado aos tipos MIME oficiais, caso contrário, aos não-padrão.

mimetypes.inited

Sinalizador indicando se as estruturas de dados globais foram inicializadas ou não. Isso é definido para `True` por **`init()`**.

mimetypes.knownfiles

Lista de nomes de arquivos de mapas de tipos comumente instalados. Esses arquivos são normalmente nomeado como `mime.types` e são instalados em diferentes locais por diferentes pacote.

mimetypes.suffix_map

Dicionário mapeando sufixos para sufixos. Isso é usado para permitir o reconhecimento de arquivos codificados para os quais a codificação e o tipo são indicados pela mesma extensão. Por exemplo, em um arquivo codificado, a extensão `.tgz` é mapeada para `.tar.gz` para permitir que a codificação e o tipo sejam reconhecidos separadamente.

mimetypes.encodings_map

Dicionário mapeando extensões de nome de arquivo para os tipos de codificação.

mimetypes.types_map

Dicionário mapeando extensões de nome do arquivo para tipos MIME.

mimetypes.common_types

Dicionário mapeando extensões de nome de arquivo para tipos MIME não padronizados, mas comumente encontrados.

Um exemplo de uso do módulo:

```
>>> import mimetypes
>>> mimetypes.init()
>>> mimetypes.knownfiles
['/etc/mime.types', '/etc/httpd/mime.types', ... ]
>>> mimetypes.suffix_map['.tgz']
'.tar.gz'
>>> mimetypes.encodings_map['.gz']
'gzip'
>>> mimetypes.types_map['.tgz']
'application/x-tar-gz'
```

Objetos MimeTypes

A classe **`MimeTypes`** pode ser útil para aplicações que possam querer mais de um banco de dados de tipos MIME; ela fornece uma interface semelhante à do módulo **`mimetypes`**.

class mimetypes.MimeTypes(*filenames=(), strict=True*)

Esta classe representa um banco de dados de tipos MIME. Por padrão, ela fornece acesso ao mesmo banco de dados do restante deste módulo. O banco de dados inicial é criado a partir de tabelas de tipos MIME embutis no Python. Ele pode ser estendido ao carregar arquivos adicionais no formato `mime.types` no banco de dados, usando os métodos **`read()`** ou **`readfp()`**. Os dicionários de mapeamento também podem ser limpos antes de carregar dados adicionais, caso os dados padrões não sejam desejados.

O parâmetro opcional *filenames* pode ser usado para carregar arquivos adicionais “em cima” do banco de dados padrão.

suffix_map

Dicionário mapeando sufixo para sufixo. Isso é usado para permitir o reconhecimento de arquivos codificados para os quais a codificação e o tipo são indicados pela mesma extensão. Por exemplo, a extensão `.tgz` é mapeada para `.tar.gz` para permitir que a codificação e o tipo sejam reconhecidos separadamente. Isso é inicializado com alguns valores predefinidos.

encodings_map

Dicionário que mapeia extensões de arquivo para tipos de codificação. Ele é inicializado com alguns valores predefinidos.

types_map

Tupla contendo dois dicionários, mapeando extensões de nomes de arquivo para tipos MIME: o primeiro dicionário é para tipos não padrão e o segundo, para tipos padrão. Eles são inicializados com alguns valores predefinidos e informações de tipo MIME carregadas a partir dos arquivos especificados pelo argumento *filenames*.

types_map_inv

Tupla contendo dois dicionários, mapeando tipos MIME para uma lista de extensões de nomes de arquivo: o primeiro dicionário é para tipos não padrão e o segundo, para tipos padrão. Eles são inicializados com alguns valores predefinidos e informações de tipo MIME carregadas a partir dos arquivos especificados pelo argumento *filenames*.

guess_extension(*type*, *strict=True*)

Similar à função **`guess_extension()`**, usando as tabelas armazenadas como parte do objeto.

guess_type(*url*, *strict=True*)

Similar à função **`guess_type()`**, usando as tabelas armazenadas como parte do objeto.

guess_file_type(*path*, *, *strict=True*)

Similar à função `guess_file_type()`, usando as tabelas armazenadas como parte do objeto.

Adicionado na versão 3.13.

guess_all_extensions(*type*, *strict=True*)

Similar à função `guess_all_extensions()`, usando as tabelas armazenadas como parte do objeto.

read(*filename*, *strict=True*)

Carrega as informações do tipo MIME de um arquivo chamado *filename*. Usa `readfp()` para analisar o arquivo.

Se *strict* for `True`, as informações serão adicionadas à lista de tipos padrão, caso contrário, à lista de tipos não padrão.

readfp(*fp*, *strict=True*)

Carrega as informações do tipo MIME de um arquivo aberto *fp*. O arquivo precisa estar no formato padrão dos arquivos `mime.types`.

Se *strict* for `True`, as informações serão adicionadas à lista de tipos padrão, caso contrário, à lista de tipos não padrão.

read_windows_registry(*strict=True*)

Carrega as informações do tipo MIME a partir do registro do Windows.

Disponibilidade: Windows.

Se *strict* for `True`, as informações serão adicionadas à lista de tipos padrão, caso contrário, à lista de tipos não padrão.

Adicionado na versão 3.2.

add_type(*type*, *ext*, *strict=True*)

Adiciona um mapeamento do tipo MIME *type* para a extensão *ext*. As extensões válidas começam com um '.' ou são vazias. Quando a

extensão já for conhecida, o novo tipo substituirá o antigo. Quando o tipo já for conhecido, a extensão será adicionada à lista de extensões conhecidas.

Quando *strict* for `True` (o padrão), o mapeamento será adicionado aos tipos MIME oficiais, caso contrário, aos não-padrão.

Descontinuado desde a versão 3.14, será removido na versão 3.16: Extensões inválidas e sem ponto levantarão uma exceção **`ValueError`** em Python 3.16.

Uso na linha de comando

O módulo **`mimetypes`** pode ser invocado como um script na linha de comando:

```
python -m mimetypes [-h] [-e] [-l] type [type ...]
```

As seguintes opções são aceitas:

-h
--help
: Mostra a mensagem de ajuda e sai.

-e
--extension
: Adivinha a extensão em vez do tipo.

-l
--lenient
: Adicionalmente, pesquisa alguns tipos comuns, mas não padronizados.

Por padrão, o script converte os tipos MIME em extensões de arquivo. Entretanto, se `--extension` for especificado, ele converterá a extensão do arquivo em tipos MIME.

Para cada entrada `type`, o script grava uma linha no fluxo de saída padrão. Se ocorrer um tipo desconhecido, ele gravará uma mensagem de erro no fluxo padrão de saída e sairá com o código de retorno `1` .

Exemplos na linha de comando

Aqui estão alguns exemplos de uso típico da interface de linha de comando do **`mimetypes`**:

```
$ # obtém um tipo MIME a partir de um nome de arquivo
$ python -m mimetypes filename.png
type: image/png encoding: None

$ # obtém um tipo MIME a partir de uma URL
$ python -m mimetypes https://example.com/filename.txt
type: text/plain enconding: None

$ # obtém um tipo MIME complexo
$ python -m mimetypes filename.tar.gz
type: aplicação/x-tar codificação: gzip

$ # obtém um tipo MIME para uma extensão de arquivo rara
$ python -m mimetypes filename.pict
error: unknown extension of filename.pict

$ # agora procura no banco de dados estendido embutido no Pythor
$ python -m mimetypes --lenient filename.pict
type: image/pict encoding: None

$ # obtém uma extensão de arquivo a partir de tipo MIME
$ python -m mimetypes --extension text/javascript
.js

$ # obtém uma extensão de arquivo  a partir de um tipo MIME rarc
$ python -m mimetypes --extension text/xul
error: unknown type text/xul

$ # agora procura novamente no banco de dados estendido
$ python -m mimetypes --extension --lenient text/xul
.xul
```

```
$ # tenta alimentar uma extensão de arquivo desconhecida
$ python -m mimetypes filename.sh filename.nc filename.xxx filer
type: application/x-sh encoding: None
type: application/x-netcdf encoding: None
error: media type unknown for filename.xxx
type: text/plain encoding: None

$ # tenta alimentar um tipo MIME desconhecido
$ python -m mimetypes --extension audio/aac audio/opus audio/fut
.aac
.opus
error: unknown type audio/future
```

`base64` — Base16, Base32, Base64, Base85 Data Encodings

Código-fonte: Lib/base64.py [https://github.com/python/cpython/tree/3.15/Lib/base64.py]

This module provides functions for encoding binary data to printable ASCII characters and decoding such encodings back to binary data. This includes the encodings specified in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] (Base64, Base32 and Base16), the Base85 encoding specified in PDF 2.0 [https://pdfa.org/resource/iso-32000-2/], and non-standard variants of Base85 used elsewhere.

Existem duas interfaces fornecidas por este módulo. A interface moderna oferece suporte a codificar objetos bytes ou similares para `bytes` ASCII, e decodificar objetos bytes ou similares ou strings contendo ASCII para `bytes`. Ambos os alfabetos de base 64 definidos em **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] (normal e seguro para URL e sistema de arquivos) são suportados.

The legacy interface does not support decoding from strings, but it does provide functions for encoding and decoding to and from file objects. It only supports the Base64 standard alphabet, and it adds newlines every 76 characters as per **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html]. Note that if you are looking for **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] support you probably want to be looking at the `email` package instead.

Alterado na versão 3.3: Strings Unicode exclusivamente ASCII agora são aceitas pelas funções de decodificação da interface moderna.

Alterado na versão 3.4: Quaisquer objetos bytes ou similares agora são aceitos por todas as funções de codificação e decodificação neste módulo. Adicionado suporte a ASCII85/Base85.

RFC 4648 Encodings

The **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] encodings are suitable for encoding binary data so that it can be safely sent by email, used as parts of URLs, or included as part of an HTTP POST request.

base64.b64encode(*s*, *altchars=None*, ***, *padded=True*, *wrapcol=0*)

Codifica o objeto bytes ou similar *s* usando Base64 e retorna o **`bytes`** codificado.

Optional *altchars* must be a bytes-like object of length 2 which specifies an alternative alphabet for the `+` and `/` characters. This allows an application to e.g. generate URL or filesystem safe Base64 strings. The default is `None`, for which the standard Base64 alphabet is used.

If *padded* is true (default), pad the encoded data with the '=' character to a size multiple of 4. If *padded* is false, do not add the pad characters.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not insert any newlines.

Alterado na versão 3.15: Added the *padded* and *wrapcol* parameters.

base64.b64decode(*s*, *altchars=None*, *validate=False*, ***, *padded=True*, *canonical=False*)
base64.b64decode(*s*, *altchars=None*, *validate=True*, ***, *ignorechars*, *padded=True*, *canonical=False*)

Decodifica o objeto bytes ou similar ou string ASCII *s* codificada em Base64 e retorna o **`bytes`** decodificado.

Optional *altchars* must be a bytes-like object or ASCII string of length 2 which specifies the alternative alphabet used instead of the `+` and `/` characters.

If *padded* is true, the last group of 4 base 64 alphabet characters must be padded with the '=' character. If *padded* is false, padding is neither

required nor recognized: the '=' character is not treated as padding but as a non-alphabet character, which means it is silently discarded when *validate* is false, or causes an **`Error`** when *validate* is true unless b'=' is included in *ignorechars*.

A **`binascii.Error`** exception is raised if *s* is incorrectly padded.

If *ignorechars* is specified, it should be a bytes-like object containing characters to ignore from the input when *validate* is true. If *ignorechars* contains the pad character `'='`, the pad characters presented before the end of the encoded data and the excess pad characters will be ignored. The default value of *validate* is `True` if *ignorechars* is specified, `False` otherwise.

If *validate* is false, characters that are neither in the normal base-64 alphabet nor (if *ignorechars* is not specified) the alternative alphabet are discarded prior to the padding check, but the `+` and `/` characters keep their meaning if they are not in *altchars* (they will be discarded in future Python versions).

If *validate* is true, these non-alphabet characters in the input result in a **`binascii.Error`**.

If *canonical* is true, non-zero padding bits are rejected. See **`binascii.a2b_base64()`** for details.

For more information about the strict base64 check, see **`binascii.a2b_base64()`**

Alterado na versão 3.15: Added the *canonical*, *ignorechars*, and *padded* parameters.

Descontinuado desde a versão 3.15: Accepting the `+` and `/` characters with an alternative alphabet is now deprecated.

base64.standard_b64encode(*s*)

Encode bytes-like object *s* using the standard Base64 alphabet and return the encoded **`bytes`**.

base64.standard_b64decode(*s*)

Decode bytes-like object or ASCII string *s* using the standard Base64 alphabet and return the decoded **`bytes`**.

base64.urlsafe_b64encode(*s*, *, *padded=True*)

Encode bytes-like object *s* using the URL- and filesystem-safe alphabet, which substitutes `-` instead of `+` and _ instead of `/` in the standard Base64 alphabet, and return the encoded **`bytes`**. The result can still contain `=` if *padded* is true (default).

Alterado na versão 3.15: Added the *padded* parameter.

base64.urlsafe_b64decode(*s*, *, *padded=False*)

Decode bytes-like object or ASCII string *s* using the URL- and filesystem-safe alphabet, which substitutes `-` instead of `+` and _ instead of `/` in the standard Base64 alphabet, and return the decoded **`bytes`**.

Alterado na versão 3.15: Added the *padded* parameter. Padding of input is no longer required by default.

Descontinuado desde a versão 3.15: Accepting the `+` and `/` characters is now deprecated.

base64.b32encode(*s*, *, *padded=True*, *wrapcol=0*)

Encode the bytes-like object *s* using Base32 and return the encoded **`bytes`**.

If *padded* is true (default), pad the encoded data with the '=' character to a size multiple of 8. If *padded* is false, do not add the pad characters.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not add any newlines.

Alterado na versão 3.15: Added the *padded* and *wrapcol* parameters.

base64.b32decode(*s*, *casefold=False*, *map01=None*, *, *padded=True*, *ignorechars=b''*, *canonical=False*)

Decode the Base32 encoded bytes-like object or ASCII string *s* and return the decoded `bytes`.

casefold opcional é uma flag especificando se um alfabeto minúsculo é aceitável como entrada. Por razões de segurança, o padrão é `False`.

RFC 4648 [https://datatracker.ietf.org/doc/html/rfc4648.html] allows for optional mapping of the digit 0 (zero) to the letter O (oh), and for optional mapping of the digit 1 (one) to either the letter I (eye) or letter L (el). The optional argument *map01* when not `None`, specifies which letter the digit 1 should be mapped to (when *map01* is not `None`, the digit 0 is always mapped to the letter O). For security purposes the default is `None`, so that 0 and 1 are not allowed in the input.

If *padded* is true, the last group of 8 base 32 alphabet characters must be padded with the '=' character. If *padded* is false, padding is neither required nor recognized: the '=' character is not treated as padding but as a non-alphabet character, which means it raises an `Error` unless b'=' is included in *ignorechars*.

ignorechars should be a bytes-like object containing characters to ignore from the input.

If *canonical* is true, non-zero padding bits are rejected. See `binascii.a2b_base32()` for details.

A `binascii.Error` is raised if *s* is incorrectly padded or if there are non-alphabet characters present in the input.

Alterado na versão 3.15: Added the *canonical*, *ignorechars*, and *padded* parameters.

base64.b32hexencode(*s*, *, *padded=True*, *wrapcol=0*)

Similar to `b32encode()` but uses the Extended Hex Alphabet, as defined in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].

Adicionado na versão 3.10.

Alterado na versão 3.15: Added the *padded* and *wrapcol* parameters.

base64.b32hexdecode(*s*, *casefold=False*, *, *padded=True*, *ignorechars=b''*, *canonical=False*)

Similar to **`b32decode()`** but uses the Extended Hex Alphabet, as defined in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].

This version does not allow the digit 0 (zero) to the letter O (oh) and digit 1 (one) to either the letter I (eye) or letter L (el) mappings, all these characters are included in the Extended Hex Alphabet and are not interchangeable.

Adicionado na versão 3.10.

Alterado na versão 3.15: Added the *canonical*, *ignorechars*, and *padded* parameters.

base64.b16encode(*s*, *, *wrapcol=0*)

Encode the bytes-like object *s* using Base16 and return the encoded **`bytes`**.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not add any newlines.

Alterado na versão 3.15: Added the *wrapcol* parameter.

base64.b16decode(*s*, *casefold=False*, *, *ignorechars=b''*)

Decode the Base16 encoded bytes-like object or ASCII string *s* and return the decoded **`bytes`**.

casefold opcional é uma flag especificando se um alfabeto minúsculo é aceitável como entrada. Por razões de segurança, o padrão é `False`.

ignorechars should be a bytes-like object containing characters to ignore from the input.

A `binascii.Error` is raised if *s* is incorrectly padded or if there are non-alphabet characters present in the input.

Alterado na versão 3.15: Added the *ignorechars* parameter.

Base85 Encodings

Base85 encoding is a family of algorithms which represent four bytes using five ASCII characters. Originally implemented in the Unix `btoa(1)` utility, a version of it was later adopted by Adobe in the PostScript language and is standardized in PDF 2.0 (ISO 32000-2). This version, in both its `btoa` and PDF variants, is implemented by `a85encode()`.

A separate version, using a different output character set, was defined as an April Fool's joke in **RFC 1924** [https://datatracker.ietf.org/doc/html/rfc1924.html] but is now used by Git and other software. This version is implemented by `b85encode()`.

Finally, a third version, using yet another output character set designed for safe inclusion in programming language strings, is defined by ZeroMQ and implemented here by `z85encode()`.

The functions present in this module differ in how they handle the following:

- Whether to include and expect enclosing <~ and ~> markers.
- Whether to fold the input into multiple lines.
- The set of ASCII characters used for encoding.
- Compact encodings of sequences of spaces and null bytes.
- The encoding of zero-padding bytes applied to the input.

Refer to the documentation of the individual functions for more information.

base64.a85encode(*b*, *, *foldspaces=False*, *wrapcol=0*, *pad=False*, *adobe=False*)

Encode the bytes-like object *b* using Ascii85 and return the encoded **bytes**.

foldspaces is an optional flag that uses the special short sequence ‘y’ instead of 4 consecutive spaces (ASCII 0x20) as supported by ‘btoa’. This feature is not supported by the standard encoding used in PDF.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not insert any newlines.

pad controls whether zero-padding applied to the end of the input is fully retained in the output encoding, as done by `btoa`, producing an exact multiple of 5 bytes of output. This is not part of the standard encoding used in PDF, as it does not preserve the length of the data.

adobe controls whether the encoded byte sequence is framed with <~ and ~>, as in a PostScript base-85 string literal. Note that while ASCII85Decode streams in PDF documents *must* be terminated with ~>, they *must not* use a leading <~.

Adicionado na versão 3.4.

base64.a85decode(*b*, *, *foldspaces=False*, *adobe=False*, *ignorechars=b'\t\n\r\x0b'*, *canonical=False*)

Decode the Ascii85 encoded bytes-like object or ASCII string *b* and return the decoded **bytes**.

foldspaces is a flag that specifies whether the ‘y’ short sequence should be accepted as shorthand for 4 consecutive spaces (ASCII 0x20). This feature is not supported by the standard Ascii85 encoding used in PDF and PostScript.

adobe controls whether the <~ and ~> markers are present. While the leading <~ is not required, the input must end with ~>, or a **ValueError**

is raised.

ignorechars should be a bytes-like object containing characters to ignore from the input. This should only contain whitespace characters, and by default contains all whitespace characters in ASCII.

If *canonical* is true, non-canonical encodings are rejected. See `binascii.a2b_ascii85()` for details.

Adicionado na versão 3.4.

Alterado na versão 3.15: Added the *canonical* parameter. Single-character final groups are now always rejected as encoding violations.

base64.b85encode(*b*, *pad=False*, *, *wrapcol=0*)

Encode the bytes-like object *b* using base85 (as used in e.g. git-style binary diffs) and return the encoded `bytes`.

The input is padded with `b'\0'` so its length is a multiple of 4 bytes before encoding. If *pad* is true, all the resulting characters are retained in the output, which will always be a multiple of 5 bytes, and thus the length of the data may not be preserved on decoding.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not add any newlines.

Adicionado na versão 3.4.

Alterado na versão 3.15: Added the *wrapcol* parameter.

base64.b85decode(*b*, *, *ignorechars=b''*, *canonical=False*)

Decode the base85-encoded bytes-like object or ASCII string *b* and return the decoded `bytes`.

ignorechars should be a bytes-like object containing characters to ignore from the input.

If *canonical* is true, non-canonical encodings are rejected. See **binascii.a2b_base85()** for details.

Adicionado na versão 3.4.

Alterado na versão 3.15: Added the *canonical* and *ignorechars* parameters. Single-character final groups are now always rejected as encoding violations.

base64.z85encode(*s*, *pad=False*, ***, *wrapcol=0*)

Encode the bytes-like object *s* using Z85 (as used in ZeroMQ) and return the encoded **bytes**.

The input is padded with `b'\0'` so its length is a multiple of 4 bytes before encoding. If *pad* is true, all the resulting characters are retained in the output, which will always be a multiple of 5 bytes, as required by the ZeroMQ standard.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not add any newlines.

Adicionado na versão 3.13.

Alterado na versão 3.15: The *pad* parameter was added.

Alterado na versão 3.15: Added the *wrapcol* parameter.

base64.z85decode(*s*, ***, *ignorechars=b''*, *canonical=False*)

Decode the Z85-encoded bytes-like object or ASCII string *s* and return the decoded **bytes**.

ignorechars should be a bytes-like object containing characters to ignore from the input.

If *canonical* is true, non-canonical encodings are rejected. See **binascii.a2b_base85()** for details.

Adicionado na versão 3.13.

Alterado na versão 3.15: Added the *canonical* and *ignorechars* parameters. Single-character final groups are now always rejected as encoding violations.

Legacy Interface

base64.decode(*input*, *output*)

Decode the contents of the binary *input* file and write the resulting binary data to the *output* file. *input* and *output* must be file objects. *input* will be read until `input.readline()` returns an empty bytes object.

base64.decodebytes(*s*)

Decode the bytes-like object *s*, which must contain one or more lines of base64 encoded data, and return the decoded **`bytes`**.

Adicionado na versão 3.1.

base64.encode(*input*, *output*)

Encode the contents of the binary *input* file and write the resulting base64 encoded data to the *output* file. *input* and *output* must be file objects. *input* will be read until `input.read()` returns an empty bytes object. **`encode()`** inserts a newline character (`b'\n'`) after every 76 bytes of the output, as well as ensuring that the output always ends with a newline, as per **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] (MIME).

base64.encodebytes(*s*)

Encode the bytes-like object *s*, which can contain arbitrary binary data, and return **`bytes`** containing the base64-encoded data, with newlines (`b'\n'`) inserted after every 76 bytes of output, and ensuring that there is a trailing newline, as per **RFC 2045** [https://datatracker.ietf.org/doc/html/rfc2045.html] (MIME).

Adicionado na versão 3.1.

Um exemplo de uso do módulo:

```
>>> import base64
>>> encoded = base64.b64encode(b'data to be encoded')
>>> encoded
b'ZGF0YSB0byBiZSBlbmNvZGVk'
>>> data = base64.b64decode(encoded)
>>> data
b'data to be encoded'
```

Considerações de Segurança

A new security considerations section was added to **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] (section 12); it's recommended to review the security section for any code deployed to production.

Ver também

Módulo `binascii`

Módulo de suporte contendo conversões ASCII para binário e binário para ASCII.

RFC 1521 [https://datatracker.ietf.org/doc/html/rfc1521.html] - MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies

Section 5.2, "Base64 Content-Transfer-Encoding," provides the definition of the base64 encoding.

ISO 32000-2 Portable document format - Part 2: PDF 2.0 [https://pdfa.org/resource/iso-32000-2/]

Section 7.4.3, "ASCII85Decode Filter," provides the definition of the Ascii85 encoding used in PDF and PostScript, including the output character set and the details of data length preservation using zero-padding and partial output groups.

ZeroMQ RFC 32/Z85 [https://rfc.zeromq.org/spec/32/]

The "Formal Specification" section provides the character set used in Z85.

`binascii` — Converte entre binário e ASCII

The **`binascii`** module contains a number of methods to convert between binary and various ASCII-encoded binary representations. Normally, you will not use these functions directly but use wrapper modules like **`base64`** instead. The **`binascii`** module contains low-level functions written in C for greater speed that are used by the higher-level modules.

Nota

Funções `a2b_*` aceitam strings Unicode contendo apenas caracteres ASCII. Outras funções aceitam apenas objetos bytes ou similares (como **`bytes`**, **`bytearray`** e outros objetos que suportam o protocolo buffer).

Alterado na versão 3.3: Strings unicode exclusivamente ASCII agora são aceitas pelas funções `a2b_*`.

The **`binascii`** module defines the following functions:

binascii.a2b_uu(*string*)

Converte uma única linha de dados uuencoded de volta para binário e retorna os dados binários. As linhas normalmente contêm 45 bytes (binários), exceto a última linha. Os dados da linha podem ser seguidos por espaços em branco.

binascii.b2a_uu(*data*, *, *backtick=False*)

Converte dados binários para uma linha de caracteres ASCII, o valor de retorno é a linha convertida, incluindo um caractere de nova linha. O comprimento de *data* deve ser no máximo 45. Se *backtick* for true, zeros são representados por `'`'` em vez de espaços.

Alterado na versão 3.7: Adicionado o parâmetro *backtick*.

binascii.a2b_base64(*string*, /, *, *padded=True*, *alphabet=BASE64_ALPHABET*, *strict_mode=False*, *canonical=False*)
binascii.a2b_base64(*string*, /, *, *ignorechars*, *padded=True*, *alphabet=BASE64_ALPHABET*, *strict_mode=True*, *canonical=False*)

Converte um bloco de dados base64 de volta para binário e retorna os dados binários. Mais de uma linha pode ser passada por vez.

Optional *alphabet* must be a `bytes` object of length 64 which specifies an alternative alphabet.

If *padded* is true, the last group of 4 base 64 alphabet characters must be padded with the '=' character. If *padded* is false, padding is neither required nor recognized: the '=' character is not treated as padding but as a non-alphabet character, which means it is silently discarded when *strict_mode* is false, or causes an `Error` when *strict_mode* is true unless b'=' is included in *ignorechars*.

If *ignorechars* is specified, it should be a bytes-like object containing characters to ignore from the input when *strict_mode* is true. If *ignorechars* contains the pad character `'='`, the pad characters presented before the end of the encoded data and the excess pad characters will be ignored. The default value of *strict_mode* is `True` if *ignorechars* is specified, `False` otherwise.

Se *strict_mode* for true, somente dados base64 válidos serão convertidos. Dados base64 inválidos levantarão `binascii.Error`.

base64 válido:

- Conforms to **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].
- Contém apenas caracteres do alfabeto base64.
- Não contém dados excedentes após o preenchimento (incluindo excesso de preenchimento, novas linhas, etc.).
- Não começa com um preenchimento.

If *canonical* is true, non-zero padding bits in the last group are rejected with `binascii.Error`, enforcing canonical encoding as defined in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] section 3.5. This check is independent of *strict_mode*.

Alterado na versão 3.11: Adicionado o parâmetro *strict_mode*.

Alterado na versão 3.15: Added the *alphabet*, *canonical*, *ignorechars*, and *padded* parameters.

binascii.b2a_base64(*data*, ***, *padded=True*, *alphabet=BASE64_ALPHABET*, *wrapcol=0*, *newline=True*)

Convert binary data to a line(s) of ASCII characters in base64 coding, as specified in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].

If *padded* is true (default), pad the encoded data with the '=' character to a size multiple of 4. If *padded* is false, do not add the pad characters.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not insert any newlines.

If *newline* is true (default), a newline character will be added at the end of the output.

Alterado na versão 3.6: Adicionado o parâmetro *newline*.

Alterado na versão 3.15: Added the *alphabet*, *padded* and *wrapcol* parameters.

binascii.a2b_ascii85(*string*, */*, ***, *foldspaces=False*, *adobe=False*, *ignorechars=b''*, *canonical=False*)

Convert Ascii85 data back to binary and return the binary data.

Valid Ascii85 data contains characters from the Ascii85 alphabet in groups of five (except for the final group, which may have from two to five characters). Each group encodes 32 bits of binary data in the range

from `0` to `2 ** 32 - 1`, inclusive. The special character `z` is accepted as a short form of the group `!!!!!`, which encodes four consecutive null bytes. A single-character final group is always rejected as an encoding violation.

foldspaces is a flag that specifies whether the ‘y’ short sequence should be accepted as shorthand for 4 consecutive spaces (ASCII 0x20). This feature is not supported by the “standard” Ascii85 encoding.

adobe controls whether the encoded byte sequence is framed with `<~` and `~>`, as in a PostScript base-85 string literal. If *adobe* is true, a leading `<~` is optionally accepted, while a trailing `~>` is *required*, and **`binascii.Error`** is raised if it is not found.

ignorechars should be a bytes-like object containing characters to ignore from the input. This should only contain whitespace characters.

If *canonical* is true, non-canonical encodings are rejected with **`binascii.Error`**. Here “canonical” means the encoding that **`b2a_ascii85()`** would produce: the `z` abbreviation must be used for all-zero groups (rather than `!!!!!`), and partial final groups must use the same padding digits as the encoder.

Invalid Ascii85 data will raise **`binascii.Error`**.

Adicionado na versão 3.15.

binascii.b2a_ascii85(*data*, */*, ***, *foldspaces=False*, *wrapcol=0*, *pad=False*, *adobe=False*)

Convert binary data to a formatted sequence of ASCII characters in Ascii85 coding. The return value is the converted data.

foldspaces is an optional flag that uses the special short sequence ‘y’ instead of 4 consecutive spaces (ASCII 0x20) as supported by ‘btoa’. This feature is not supported by the “standard” Ascii85 encoding.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not insert any

newlines.

If *pad* is true, the zero-padding applied to the end of the input is fully retained in the output encoding, as done by `btoa`, producing an exact multiple of 5 bytes of output. This is not part of the standard encoding used in PDF, as it does not preserve the length of the data.

adobe controls whether the encoded byte sequence is framed with `<~` and `~>`, as in a PostScript base-85 string literal. Note that while ASCII85Decode streams in PDF documents *must* be terminated with `~>`, they *must not* use a leading `<~`.

Adicionado na versão 3.15.

binascii.a2b_base85(*string*, */*, ***, *alphabet=BASE85_ALPHABET*, *ignorechars=b''*, *canonical=False*)

Convert Base85 data back to binary and return the binary data. More than one line may be passed at a time.

Valid Base85 data contains characters from the Base85 alphabet in groups of five (except for the final group, which may have from two to five characters). Each group encodes 32 bits of binary data in the range from `0` to `2 ** 32 - 1`, inclusive. A single-character final group is always rejected as an encoding violation.

Optional *alphabet* must be a **`bytes`** object of length 85 which specifies an alternative alphabet.

ignorechars should be a bytes-like object containing characters to ignore from the input.

If *canonical* is true, non-canonical encodings are rejected with **`binascii.Error`**. Here “canonical” means the encoding that **`b2a_base85()`** would produce: partial final groups must use the same padding digits as the encoder.

Invalid Base85 data will raise **`binascii.Error`**.

Adicionado na versão 3.15.

binascii.b2a_base85(*data*, */*, ***, *alphabet=BASE85_ALPHABET*, *wrapcol=0*, *pad=False*)

Convert binary data to a line of ASCII characters in Base85 coding. The return value is the converted line.

Optional *alphabet* must be a bytes-like object of length 85 which specifies an alternative alphabet.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not insert any newlines.

If *pad* is true, the zero-padding applied to the end of the input is retained in the output, which will always be a multiple of 5 bytes, and thus the length of the data may not be preserved on decoding.

Adicionado na versão 3.15.

binascii.a2b_base32(*string*, */*, ***, *padded=True*, *alphabet=BASE32_ALPHABET*, *ignorechars=b''*, *canonical=False*)

Convert base32 data back to binary and return the binary data.

Valid base32 data contains characters from the base32 alphabet specified in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] in groups of eight (if necessary, the final group is padded to eight characters with `=`). Each group encodes 40 bits of binary data in the range from `0` to `2 ** 40 - 1`, inclusive.

Nota

This function does not map lowercase characters (which are invalid in standard base32) to their uppercase counterparts, nor does it contextually map `0` to `O` and `1` to `I`/`L` as **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] allows.

Optional *alphabet* must be a `bytes` object of length 32 which specifies an alternative alphabet.

If *padded* is true, the last group of 8 base 32 alphabet characters must be padded with the '=' character. If *padded* is false, the '=' character is treated as other non-alphabet characters (depending on the value of *ignorechars*).

ignorechars should be a bytes-like object containing characters to ignore from the input. If *ignorechars* contains the pad character `'='`, the pad characters presented before the end of the encoded data and the excess pad characters will be ignored.

If *canonical* is true, non-zero padding bits in the last group are rejected with `binascii.Error`, enforcing canonical encoding as defined in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html] section 3.5.

Invalid base32 data will raise `binascii.Error`.

Adicionado na versão 3.15.

binascii.b2a_base32(*data*, *, *, *padded=True*, *alphabet=BASE32_ALPHABET*, *wrapcol=0*)

Convert binary data to a line of ASCII characters in base32 coding, as specified in **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html]. The return value is the converted line.

Optional *alphabet* must be a bytes-like object of length 32 which specifies an alternative alphabet.

If *padded* is true (default), pad the encoded data with the '=' character to a size multiple of 8. If *padded* is false, do not add the pad characters.

If *wrapcol* is non-zero, insert a newline (`b'\n'`) character after at most every *wrapcol* characters. If *wrapcol* is zero (default), do not insert any newlines.

Adicionado na versão 3.15.

binascii.a2b_qp(*data*, *header=False*)

Convert a block of quoted-printable data back to binary and return the binary data. More than one line may be passed at a time. If the optional argument *header* is present and true, underscores will be decoded as spaces.

binascii.b2a_qp(*data*, *quotetabs=False*, *istext=True*, *header=False*)

Convert binary data to a line(s) of ASCII characters in quoted-printable encoding. The return value is the converted line(s). If the optional argument *quotetabs* is present and true, all tabs and spaces will be encoded. If the optional argument *istext* is present and true, newlines are not encoded but trailing whitespace will be encoded. If the optional argument *header* is present and true, spaces will be encoded as underscores per **RFC 1522** [https://datatracker.ietf.org/doc/html/rfc1522.html]. If the optional argument *header* is present and false, newline characters will be encoded as well; otherwise linefeed conversion might corrupt the binary data stream.

binascii.crc_hqx(*data*, *value*)

Compute a 16-bit CRC value of *data*, starting with *value* as the initial CRC, and return the result. This uses the CRC-CCITT polynomial $x^{16} + x^{12} + x^5 + 1$, often represented as 0x1021. This CRC is used in the binhex4 format.

binascii.crc32(*data*[, *value*])

Compute CRC-32, the unsigned 32-bit checksum of *data*, starting with an initial CRC of *value*. The default initial CRC is zero. The algorithm is consistent with the ZIP file checksum. Since the algorithm is designed for use as a checksum algorithm, it is not suitable for use as a general hash algorithm. Use as follows:

```
print(binascii.crc32(b"hello world"))
# Or, in two pieces:
crc = binascii.crc32(b"hello")
crc = binascii.crc32(b" world", crc)
print('crc32 = {:#010x}'.format(crc))
```

Alterado na versão 3.0: O resultado é sempre sem sinal.

binascii.b2a_hex(*data*[, *sep*[, *bytes_per_sep=1*]])

binascii.hexlify(*data*[, *sep*[, *bytes_per_sep=1*]])

Return the hexadecimal representation of the binary *data*. Every byte of *data* is converted into the corresponding 2-digit hex representation. The returned bytes object is therefore twice as long as the length of *data*.

Similar functionality (but returning a text string) is also conveniently accessible using the **bytes.hex()** method.

If *sep* is specified, it must be a single character str or bytes object. It will be inserted in the output after every *bytes_per_sep* input bytes. Separator placement is counted from the right end of the output by default, if you wish to count from the left, supply a negative *bytes_per_sep* value.

```
>>> import binascii
>>> binascii.b2a_hex(b'\xb9\x01\xef')
b'b901ef'
>>> binascii.hexlify(b'\xb9\x01\xef', '-')
b'b9-01-ef'
>>> binascii.b2a_hex(b'\xb9\x01\xef', b'_', 2)
b'b9_01ef'
>>> binascii.b2a_hex(b'\xb9\x01\xef', b' ', -2)
b'b901 ef'
```

Alterado na versão 3.8: The *sep* and *bytes_per_sep* parameters were added.

binascii.a2b_hex(*hexstr*, *, *ignorechars=b''*)

binascii.unhexlify(*hexstr*, *, *ignorechars=b''*)

Return the binary data represented by the hexadecimal string *hexstr*. This function is the inverse of **b2a_hex()**. *hexstr* must contain an even number of hexadecimal digits (which can be upper or lower case), otherwise an **Error** exception is raised.

ignorechars should be a bytes-like object containing characters to ignore from the input.

Similar functionality (but more liberal towards whitespace) is also accessible using the `bytes.fromhex()` class method.

Alterado na versão 3.15: Added the *ignorechars* parameter.

exception binascii.Error

Exception raised on errors. These are usually programming errors.

exception binascii.Incomplete

Exception raised on incomplete data. These are usually not programming errors, but may be handled by reading a little more data and trying again.

binascii.BASE64_ALPHABET

The Base 64 alphabet according to **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].

Adicionado na versão 3.15.

binascii.URLSAFE_BASE64_ALPHABET

The "URL and filename safe" Base 64 alphabet according to **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].

Adicionado na versão 3.15.

binascii.UU_ALPHABET

The uuencoding alphabet.

Adicionado na versão 3.15.

binascii.CRYPT_ALPHABET

The Base 64 alphabet used in the *crypt(3)* *[https://manpages.debian.org/crypt(3)]* routine and in the GEDCOM format.

Adicionado na versão 3.15.

binascii.BINHEX_ALPHABET

The Base 64 alphabet used in BinHex 4 (HQX) within the classic Mac OS.

Adicionado na versão 3.15.

binascii.BASE85_ALPHABET

The Base85 alphabet.

Adicionado na versão 3.15.

binascii.ASCII85_ALPHABET

The Ascii85 alphabet.

Adicionado na versão 3.15.

binascii.Z85_ALPHABET

The Z85 [https://rfc.zeromq.org/spec/32/] alphabet.

Adicionado na versão 3.15.

binascii.BASE32_ALPHABET

The Base 32 alphabet according to **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html].

Adicionado na versão 3.15.

binascii.BASE32HEX_ALPHABET

The “Extended Hex” Base 32 alphabet according to **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html]. Data encoded with this alphabet maintains its sort order during bitwise comparisons.

Adicionado na versão 3.15.

Ver também

Módulo `base64`

Support for RFC compliant base64-style encoding in base 16, 32, 64, and 85.

Módulo `quopri`

Support for quoted-printable encoding used in MIME email messages.

`quopri` — Codifica e decodifica dados MIME imprimidos entre aspas

Código-fonte: Lib/quopri.py [https://github.com/python/cpython/tree/3.15/Lib/quopri.py]

Este módulo realiza codificação e decodificação de transporte imprimida entre aspas, como definido em **RFC 1521** [https://datatracker.ietf.org/doc/html/rfc1521.html]: “MIME (Multipurpose Internet Mail Extensions) Part One: Mechanisms for Specifying and Describing the Format of Internet Message Bodies”. A codificação imprimida entre aspas é projetada para dados em que há relativamente poucos caracteres não imprimíveis; o esquema de codificação base64 disponível através do módulo `base64` é mais compacto se existirem muitos desses caracteres, como no envio de um arquivo gráfico.

quopri.decode(*input*, *output*, *header=False*)

Decodifica o conteúdo do arquivo *input* e escreve os dados binários decodificados resultantes no arquivo *output*. *input* e *output* devem ser objetos arquivos binários. Se o argumento opcional *header* estiver presente e for true, o sublinhado será decodificado como espaço. Isso é usado para decodificar cabeçalhos codificados em “Q”, conforme descrito em **RFC 1522** [https://datatracker.ietf.org/doc/html/rfc1522.html]: “MIME (Multipurpose Internet Mail Extensions) Part Two: Message Header Extensions for Non-ASCII Text”.

quopri.encode(*input*, *output*, *quotetabs*, *header=False*)

Codifica o conteúdo do arquivo *input* e grava os dados imprimíveis entre aspas resultantes no arquivo *output*. *input* e *output* devem ser objetos arquivos binários. *quotetabs*, um sinalizador não opcional que controla a codificação de espaços e tabulações incorporados; quando

true, codifica esses espaços em branco incorporados e, quando false, os deixa sem codificação. Observe que os espaços e tabulações que aparecem no final das linhas são sempre codificados, conforme **RFC 1521** [https://datatracker.ietf.org/doc/html/rfc1521.html]. *header* é um sinalizador que controla se os espaços são codificados como sublinhados, conforme **RFC 1522** [https://datatracker.ietf.org/doc/html/rfc1522.html].

quopri.decodestring(*s*, *header=False*)

Como **`decode()`**, exceto pelo fato de aceitar uma fonte **`bytes`** e retornar o correspondente decodificado **`bytes`**.

quopri.encodestring(*s*, *quotetabs=False*, *header=False*)

Como **`encode()`**, exceto pelo fato de aceitar uma fonte **`bytes`** e retornar o **`bytes`** codificado correspondente. Por padrão, envia um valor `False` para o parâmetro *quotetabs* da função **`encode()`**.

Ver também

Módulo `base64`

Codifica e decodifica dados de base64 MIME

Ferramentas de Processamento de Markup Estruturado

O Python suporta uma variedade de módulos para trabalhar com vários formatos de marcação de dados estruturados. Isso inclui módulos para trabalhar com o Standard Generalized Markup Language (SGML) e o Hypertext Markup Language (HTML) e várias interfaces para trabalhar com o XML (Extensible Markup Language).

- `html` — Suporte HTML (HyperText Markup Language)
- `html.parser` — Analisador simples de HTML e XHTML
  - Exemplo de aplicação para análise de HTML
  - Métodos `HTMLParser`
  - Exemplos
- `html.entities` — Definições de entidades gerais de HTML
- Módulos de Processamento de XML
  - Segurança no XML
- `xml.etree.ElementTree` — A API XML ElementTree
  - Tutorial
    - Árvore e elementos XML
    - Analisando XML
    - A API de pull para análise sem bloqueio
    - Encontrando elementos interessantes
    - Modificando um arquivo XML
    - Construindo documentos XML
    - Analisando XML com espaços de nomes
  - Suporte a XPath
    - Exemplo
    - Supported XPath syntax
  - Referência
    - Funções
  - Suporte a XInclude
    - Exemplo
  - Referência

        - Funções
        - Element Objects
        - ElementTree Objects
        - QName Objects
        - TreeBuilder Objects
        - Objetos XMLParser
        - XMLPullParser Objects
        - Exceções
- `xml.dom` — The Document Object Model API
    - Conteúdo do módulo
    - Objects in the DOM
        - DOMImplementation Objects
        - Objetos Node
        - Objetos NodeList
        - DocumentType Objects
        - Document Objects
        - Element Objects
        - Attr Objects
        - NamedNodeMap Objects
        - DocumentFragment Objects
        - CharacterData Objects
        - Comment Objects
        - Text and CDATASection Objects
        - Objetos ProcessingInstruction
        - Entity Objects
        - Notation Objects
        - Exceções
    - Conformance
        - Type Mapping
        - Accessor Methods
- `xml.dom.minidom` — Minimal DOM implementation
    - Objetos DOM
    - DOM Example
    - minidom e o padrão DOM
- `xml.dom.pulldom` — Suporte para construir árvores parciais de DOM
    - Objetos DOMEventStream
- `xml.sax` — Support for SAX2 parsers

    - SAXException Objects
- `xml.sax.handler` — Classes base para manipuladores de SAX
    - ContentHandler Objects
    - DTDHandler Objects
    - EntityResolver Objects
    - ErrorHandler Objects
    - LexicalHandler Objects
- `xml.sax.saxutils` — Utilitários de SAX
- `xml.sax.xmlreader` — Interface for XML parsers
    - XMLReader Objects
    - IncrementalParser Objects
    - Locator Objects
    - InputSource Objects
    - The `Attributes` Interface
    - The `AttributesNS` Interface
- `xml.parsers.expat` — Fast XML parsing using Expat
    - Objetos XMLParser
    - Exceções ExpatError
    - Exemplo
    - Content Model Descriptions
    - Constantes de erro de expatriados

`html` — Suporte HTML (HyperText Markup Language)

Código-fonte: Lib/html/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/html/__init__.py]

Este módulo define utilitários para manipular HTML.

html.escape(*s*, *quote=True*)

Converte os caracteres `&`, `<` e `>` na string *s* para sequências seguras em HTML. Use se necessitar exibir texto que possa conter estes caracteres no HTML. Se o flag opcional *quote* é verdadeiro (padrão), os caracteres (`"`) e (`'`) também são convertidos; isso auxilia na inclusão de valores delimitados por aspas num atributo HTML, como em `<a href="...">`. Se *quote* for definido para falso, os caracteres (`"`) e (`'`) não são traduzidos.

Adicionado na versão 3.2.

html.unescape(*s*)

Converte todas as referências de caracteres numéricos e nomeados (ex. `>`, `>`, `>`) na string *s* para caracteres Unicode correspondentes. Essa função usa as regras definidas pelo padrão HTML 5 para referências de caracteres, sejam válidas ou inválidas, e a **`lista de referência de caracteres nomeados do HTML 5`**.

Adicionado na versão 3.4.

Sub módulos no pacote `html` são:

- **`html.parser`** – analisador HTML/XHTML com modo de análise branda

- `html.entities` – definições das entidade HTML

`html.parser` — Analisador simples de HTML e XHTML

Código-fonte: Lib/html/parser.py
[https://github.com/python/cpython/tree/3.15/Lib/html/parser.py]

Este módulo define uma classe **`HTMLParser`** que serve como base para analisar arquivos de texto formatados em HTML (HyperText Markup Language) e XHTML.

class html.parser.HTMLParser(*, *convert_charrefs=True*, *scripting=False*)

Cria uma instância de analisador sintático capaz de analisar marcação inválida.

Se *convert_charrefs* for verdadeiro (o padrão), todas as referências de caracteres (exceto as de elementos como `script` e `style`) serão convertidas automaticamente para os caracteres Unicode correspondentes.

Se *scripting* for falso (o padrão), o conteúdo do elemento `noscript` será analisado normalmente; se for verdadeiro, será retornado como está, sem ser analisado.

Uma instância de **`HTMLParser`** recebe dados HTML e chama métodos de tratamento quando encontra tags de abertura, tags de fechamento, texto, comentários e outros elementos de marcação. O usuário deve criar uma subclasse de **`HTMLParser`** e substituir seus métodos para implementar o comportamento desejado.

Este analisador sintático não verifica se as tags de fechamento correspondem às tags de abertura nem chama o manipulador de tags de fechamento para elementos que são fechados implicitamente pelo fechamento de um elemento externo.

Alterado na versão 3.4: Adicionado o argumento nomeado *convert_charrefs*.

Alterado na versão 3.5: O valor padrão para o argumento *convert_charrefs* agora é `True`.

Alterado na versão 3.14.1: Adicionado o parâmetro *scripting*.

Exemplo de aplicação para análise de HTML

Como exemplo básico, abaixo está um analisador HTML simples que usa a classe **`HTMLParser`** para exibir as tags de abertura, tags de fechamento e dados à medida que são encontrados:

```
from html.parser import HTMLParser

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Encontrada a tag de abertura  :", tag)

    def handle_endtag(self, tag):
        print("Encontrada a tag de fechamento:", tag)

    def handle_data(self, data):
        print("Encontrados alguns dados      :", data)

parser = MyHTMLParser()
parser.feed('<html><head><title>Teste</title></head>'
            '<body><h1>Me analise!</h1></body></html>')
```

A saída então será:

```
Encontrada a tag de abertura  : html
Encontrada a tag de abertura  : head
Encontrada a tag de abertura  : title
Encontrados alguns dados      : Teste
Encontrada a tag de fechamento: title
Encontrada a tag de fechamento: head
Encontrada a tag de abertura  : body
Encontrada a tag de abertura  : h1
```

```
Encontrados alguns dados      : Me analise!
Encontrada a tag de fechamento: h1
Encontrada a tag de fechamento: body
Encontrada a tag de fechamento: html
```

Métodos `HTMLParser`

Instâncias de `HTMLParser` têm os seguintes métodos:

HTMLParser.feed(*data*)

Fornece algum texto ao analisador. Ele será processado na medida em que consistir em elementos completos; dados incompletos serão armazenados em buffer até que mais dados sejam fornecidos ou que o método `close()` seja chamado. Os dados devem ser do tipo `str`.

HTMLParser.close()

Força o processamento de todos os dados em buffer como se fossem seguidos por uma marca de fim de arquivo. Este método pode ser redefinido por uma classe derivada para definir processamento adicional no final da entrada, mas a versão redefinida deve sempre chamar o método `close()` da classe base `HTMLParser`.

HTMLParser.reset()

Reinicializa a instância. Perde todos os dados não processados. Este método é chamado implicitamente no momento da instanciação.

HTMLParser.getpos()

Retorna o número da linha atual e o deslocamento.

HTMLParser.get_starttag_text()

Retorna o texto da tag de abertura aberta mais recentemente. Normalmente, isso não é necessário para processamento estruturado, mas pode ser útil ao lidar com HTML “como implantado” ou para regenerar a entrada com alterações mínimas (o espaço em branco entre os atributos pode ser preservado, etc.).

Os seguintes métodos são chamados quando elementos de dados ou de marcação são encontrados e devem ser sobrescritos em uma subclasse. As implementações da classe base não fazem nada (exceto por **`handle_startendtag()`**):

HTMLParser.handle_starttag(*tag*, *attrs*)

Este método é chamado para manipular a tag de abertura de um elemento (por exemplo, `<div id="main">`).

The *tag* argument is the name of the tag converted to lower case. The *attrs* argument is a list of `(name, value)` pairs containing the attributes found inside the tag's `<>` brackets. The *name* will be translated to lower case, and quotes in the *value* have been removed, and character and entity references have been replaced. For empty attributes, *value* is `None`.

Por exemplo, para a tag `<A HREF="https://www.cwi.nl/">`, este método seria chamado como `handle_starttag('a', [('href', 'https://www.cwi.nl/')])`.

Todas as referências de entidade de **`html.entities`** são substituídas nos valores dos atributos.

HTMLParser.handle_endtag(*tag*)

Este método é chamado para manipular a tag de fechamento de um elemento (por exemplo, `</div>`).

O argumento *tag* é o nome da etiqueta convertido para letras minúsculas.

HTMLParser.handle_startendtag(*tag*, *attrs*)

Similar a **`handle_starttag()`**, mas chamado quando o analisador sintático encontra uma tag vazia no estilo XHTML (`<img ... />`). Este método pode ser substituído por subclasses que requerem essa informação lexical específica; a implementação padrão simplesmente chama **`handle_starttag()`** e **`handle_endtag()`**.

HTMLParser.handle_data(*data*)

Este método é chamado para processar dados arbitrários (por exemplo, nós de texto e o conteúdo de elementos como `script` e `style`).

HTMLParser.handle_entityref(*name*)

Este método é chamado para processar uma referência de caractere nomeada no formato `&name;` (por exemplo, `>`), onde *name* é uma referência de entidade genérica (por exemplo, `'gt'`). Este método só é chamado se *convert_charrefs* for falso.

HTMLParser.handle_charref(*name*)

Este método é chamado para processar referências de caracteres numéricos decimais e hexadecimais no formato `&#NNN;` e `&#xNNN;`. Por exemplo, o equivalente decimal para `>` é `>`, enquanto o hexadecimal é `>`; neste caso, o método receberá `'62'` ou `'x3E'`. Este método só é chamado se *convert_charrefs* for falso.

HTMLParser.handle_comment(*data*)

Este método é chamado quando um comentário é encontrado (por exemplo, `<!--comentário-->`).

Por exemplo, o comentário `<!-- comentário -->` fará com que este método seja chamado com o argumento `' comentário '`.

O conteúdo dos comentários condicionais do Internet Explorer (condcoms) também será enviado para este método, portanto, para `<!--[if IE 9]>conteúdo específico do IE9<![endif]-->`, este método receberá `'[if IE 9]>conteúdo específico do IE9<![endif]'`.

HTMLParser.handle_decl(*decl*)

Este método é chamado para lidar com uma declaração de doctype HTML (por exemplo, `<!DOCTYPE html>`).

O parâmetro *decl* será todo o conteúdo da declaração dentro da marcação `<!...>` (por exemplo, `'DOCTYPE html'`).

HTMLParser.handle_pi(*data*)

Método chamado quando uma instrução de processamento é encontrada. O parâmetro *data* conterá a instrução de processamento completa. Por exemplo, para a instrução de processamento `<?proc color='red'>`, este método seria chamado como `handle_pi("proc color='red'")`. Ele foi projetado para ser substituído por uma classe derivada; a implementação da classe base não faz nada.

Nota

A classe **`HTMLParser`** usa as regras sintáticas SGML para processar instruções. Uma instrução de processamento XHTML que usa o caractere `'?'` no final fará com que o `'?'` seja incluído em *data*.

HTMLParser.unknown_decl(*data*)

Este método é chamado quando uma declaração não reconhecida é lida pelo analisador sintático.

O parâmetro *data* será todo o conteúdo da declaração dentro da marcação `<![...]>`. Às vezes, é útil que ele seja sobrescrito por uma classe derivada. A implementação da classe base não faz nada.

Exemplos

A classe a seguir implementa um analisador sintático que será usado para ilustrar mais exemplos:

```
from html.parser import HTMLParser
from html.entities import name2codepoint

class MyHTMLParser(HTMLParser):
    def handle_starttag(self, tag, attrs):
        print("Tag inicial:", tag)
        for attr in attrs:
            print("     attr:", attr)

    def handle_endtag(self, tag):
```

```
        print("Tag final  :", tag)

    def handle_data(self, data):
        print("Dados      :", data)

    def handle_comment(self, data):
        print("Comentário :", data)

    def handle_entityref(self, name):
        c = chr(name2codepoint[name])
        print("Ent nomeada:", c)

    def handle_charref(self, name):
        if name.startswith('x'):
            c = chr(int(name[1:], 16))
        else:
            c = chr(int(name))
        print("Num ent    :", c)

    def handle_decl(self, data):
        print("Decl       :", data)

parser = MyHTMLParser()
```

Analisando um doctype:

```
>>> parser.feed('<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//E
...             '"http://www.w3.org/TR/html4/strict.dtd">')
Decl       : DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "ht
```

Analisando um elemento com alguns atributos e um título:

```
>>> parser.feed('<img src="python-logo.png" alt="The Python logo
Tag inicial: img
     attr: ('src', 'python-logo.png')
     attr: ('alt', 'The Python logo')
>>>
>>> parser.feed('<h1>Python</h1>')
Tag inicial: h1
Dados      : Python
Tag final  : h1
```

O conteúdo de elementos como `script` e `style` é retornado como está, sem mais análises:

```
>>> parser.feed('<style type="text/css">#python { color: green }
Tag inicial: style
     attr: ('type', 'text/css')
Dados      : #python { color: green }
Tag final  : style

>>> parser.feed('<script type="text/javascript">'
...             'alert("<strong>hello! &#9786;</strong>");</scri
Tag inicial: script
     attr: ('type', 'text/javascript')
Dados      : alert("<strong>hello! &#9786;</strong>");
Tag final  : script
```

Attribute names are converted to lowercase, quotes from attribute values removed, and `None` is returned as *value* for empty attributes (such as `checked`):

```
>>> parser.feed("<input TYPE='checkbox' checked required='' disa
Start tag: input
     attr: ('type', 'checkbox')
     attr: ('checked', None)
     attr: ('required', '')
     attr: ('disabled', 'disabled')
```

Analisando comentários:

```
>>> parser.feed('<!-- a comment -->'
...             '<!--[if IE 9]>IE-specific content<![endif]-->')
Comentário : a comment
Comentário : [if IE 9]>IE-specific content<![endif]
```

Analisando referências de caracteres nomeadas e numéricas e convertendo-as para o caractere correto (nota: essas 3 referências são todas equivalentes a `'>'`):

```
>>> parser = MyHTMLParser()
>>> parser.feed('&gt;&#62;&#x3E;')
Data     : >>>

>>> parser = MyHTMLParser(convert_charrefs=False)
>>> parser.feed('&gt;&#62;&#x3E;')
Named ent: >
Num ent  : >
Num ent  : >
```

Alimentar o `feed()` com blocos incompletos funciona, mas `handle_data()` pode ser chamado mais de uma vez se *convert_charrefs* for falso:

```
>>> for chunk in ['<sp', 'an>buff', 'ered', ' text</s', 'pan>']:
...     parser.feed(chunk)
...
Start tag: span
Data     : buff
Data     : ered
Data     :  text
End tag  : span
```

Analisando HTML inválido (por exemplo, atributos sem aspas) também funciona:

```
>>> parser.feed('<p><a class=link href=#main>tag soup</p ></a>')
Tag inicial: p
Tag inicial: a
     attr: ('class', 'link')
     attr: ('href', '#main')
Dados      : tag soup
Tag final  : p
Tag final  : a
```

`html.entities` — Definições de entidades gerais de HTML

Código-fonte: Lib/html/entities.py
[https://github.com/python/cpython/tree/3.15/Lib/html/entities.py]

Esse módulo define quatro dicionários, `html5`, `name2codepoint`, `codepoint2name` e `entitydefs`.

html.entities.html5

Um dicionário que mapeia referências de caracteres nomeados em HTML5 [[1]] para os caracteres Unicode equivalentes, por exemplo, `html5['gt;'] == '>'`. Note que o caractere de ponto e vírgula final está incluído no nome (por exemplo, `'gt;'`), entretanto alguns dos nomes são aceitos pelo padrão mesmo sem o ponto e vírgula: neste caso o nome está presente com e sem o `';'`. Veja também `html.unescape()`.

Adicionado na versão 3.3.

html.entities.entitydefs

Um dicionário que mapeia as definições de entidade XHTML 1.0 para seu texto substituto em ISO Latin-1.

html.entities.name2codepoint

Um dicionário que mapeia nomes de entidades HTML4 para os pontos de código Unicode.

html.entities.codepoint2name

Um dicionário que mapeia pontos de código Unicode para nomes de entidades HTML4.

Notas de rodapé

[1]

Veja https://html.spec.whatwg.org/multipage/named-characters.html#named-character-references

Módulos de Processamento de XML

Código-fonte: Lib/xml/ [https://github.com/python/cpython/tree/3.15/Lib/xml/]

As interfaces do Python para processar XML estão agrupadas no pacote `xml`.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte Segurança no XML.

É importante observar que os módulos no pacote **`xml`** exigem que está disponível pelo menos um analisador sintático XML compatível com SAX. O analisador sintático Expat está incluído no Python, então o módulo **`xml.parsers.expat`** estará sempre disponível.

A documentação para os pacotes **`xml.dom`** e **`xml.sax`** são a definição das ligações Python para as interfaces DOM e SAX.

Os submódulos de manipulação XML são:

- **`xml.etree.ElementTree`**: a API de ElementTree, um processador XML simples e leve

- **`xml.dom`**: a definição da API de DOM
- **`xml.dom.minidom`**: uma implementação mínima do DOM
- suporte para construir árvores parciais de DOM no **`xml.dom.pulldom`**

- **`xml.sax`**: Classe base SAX2 e funções de conveniência
- **`xml.parsers.expat`**: a ligação do analisador sintático Expat

This module also defines utility functions.

xml.is_valid_name(*name*)

Return `True` if the string is a valid element or attribute name, `False` otherwise.

Almost all characters are permitted in names, except control characters and those which either are or reasonably could be used as delimiters. Characters like “:”, “-”, “.”, “_”, and “·” are permitted, but “<”, “/”, “!”, “?”, and “=” are forbidden. The name cannot start with a digit or a character like “-”, “.”, and “·”.

Adicionado na versão 3.15.

xml.is_valid_text(*data*)

Return `True` if the string is a sequence of legal XML 1.0 characters, `False` otherwise.

Almost all characters are permitted in XML 1.0 documents, except C0 control characters (excluding TAB, CR and LF), surrogate characters and special Unicode characters U+FFFE and U+FFFF.

Adicionado na versão 3.15.

Segurança no XML

Um atacante pode abusar dos recursos XML para realizar ataques de negação de serviço, acessar arquivos locais, gerar conexões de rede com outras máquinas ou contornar firewalls. quando XML controlado pelo atacante está sendo analisado, no Python ou de qualquer outra forma.

Os analisadores XML embutidos do Python dependem da biblioteca libexpat [https://github.com/libexpat/libexpat], comumente chamada Expat, para análise de XML.

Por padrão, Expat não acessa arquivos locais nem cria conexões de rede.

Versões do Expat anteriores à 2.7.2 podem ser vulneráveis às vulnerabilidades “billion laughs”, “quadratic blowup” e “large tokens”, ou ao uso desproporcional de memória dinâmica. O Python inclui uma cópia do Expat, e o uso da versão incluída ou de uma versão do sistema depende de como o interpretador Python **foi configurado** no seu ambiente. O Python pode ser vulnerável se usar versões antigas do Expat. Verifique **`pyexpat.EXPAT_VERSION`**.

`xmlrpc` é **vulnerável** ao ataque “decompression bomb”.

billion laughs / exponential entity expansion

O ataque Billion Laughs [https://en.wikipedia.org/wiki/Billion_laughs] (bilhões de risadas, em uma tradução livre) - também conhecido como “exponential entity expansion” (expansão exponencial de entidades, em uma tradução livre) - usa vários níveis de entidades aninhadas. Cada entidade se refere a outra entidade várias vezes, e a definição final da entidade contém uma pequena string. A expansão exponencial resulta em vários gigabytes de texto e consome muita memória e tempo de CPU.

quadratic blowup entity expansion

Um ataque “quadratic blowup” (explosão quadrática, em uma tradução livre) português) é semelhante a um ataque Billion Laughs [https://en.wikipedia.org/wiki/Billion_laughs]; ele abusa da expansão de entidades também. Em vez de entidades aninhadas, ele repete uma grande entidade com alguns milhares de caracteres repetidamente. O ataque não é tão eficiente quanto o caso exponencial, mas evita acionar contramedidas do analisador sintático que proíbem entidades profundamente aninhadas.

decompression bomb

“Bombas de descompressão” em uma tradução livre. Também conhecidas como ZIP bomb [https://en.wikipedia.org/wiki/Zip_bomb], se aplicam a todas as bibliotecas XML que podem analisar fluxos XML comprimidos, como fluxos de HTTP compactados com gzip ou arquivos comprimidos com LZMA. Para um atacante, isso pode reduzir a quantidade de dados transmitidos em três magnitudes ou mais.

large tokens

Expat precisa analisar novamente os símbolos não finalizados; sem a proteção introduzida no Expat 2.6.0, isso pode levar a um tempo de execução quadrático que pode ser usado para causar negação de serviço na aplicação ao analisar XML. O problema é conhecido como **CVE 2023-52425** [https://www.cve.org/CVERecord?id=CVE-2023-52425].

`xml.etree.ElementTree` — A API XML ElementTree

Código-fonte: Lib/xml/etree/ElementTree.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/etree/ElementTree.py]

The `xml.etree.ElementTree` module implements a simple and efficient API for parsing and creating XML data.

Alterado na versão 3.3: Este módulo usará uma implementação rápida sempre que disponível.

Descontinuado desde a versão 3.3: The `xml.etree.cElementTree` alias of this module is deprecated.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte Segurança no XML.

Tutorial

This is a short tutorial for using `xml.etree.ElementTree` (`ET` in short). The goal is to demonstrate some of the building blocks and basic concepts of the module.

Árvore e elementos XML

XML é um formato de dados estritamente hierárquico, e a maneira mais natural de representá-lo é como uma árvore. `ET` possui duas classes para esse propósito - `ElementTree` representa todo o documento XML como

uma árvore e **Element** representa um único nó desta árvore. Interações com o documento inteiro (ler e escrever de/para arquivos) são frequentemente feitos em nível de **ElementTree**. Interações com um único elemento XML e seus subelementos são feitos a nível de **Element** .

Analisando XML

Usaremos o documento XML fictício `country_data.xml` como dados de amostra para esta seção:

```
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank>1</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank>4</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank>68</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>
```

Nós podemos importar esses dados lendo de um arquivo:

```
import xml.etree.ElementTree as ET
tree = ET.parse('country_data.xml')
root = tree.getroot()
```

Ou diretamente de uma string:

```
root = ET.fromstring(country_data_as_string)
```

`fromstring()` obtém o XML de uma string e armazena em um `Element`, que será o elemento raiz dessa árvore. Outras funções de análise sintática podem criar um `ElementTree`. Cheque a documentação para se certificar sobre qual dado será retornado.

Assim como um `Element`, `root` tem uma tag e um dicionário de atributos:

```
>>> root.tag
'data'
>>> root.attrib
{}
```

Ele também tem nós filhos sobre os quais nós podemos iterar:

```
>>> for child in root:
...     print(child.tag, child.attrib)
...
country {'name': 'Liechtenstein'}
country {'name': 'Singapore'}
country {'name': 'Panama'}
```

Nós filhos são os mais próximos, e nós podemos acessar nós específicos por índices:

```
>>> root[0][1].text
'2008'
```

Nota

Nem todos os elementos da entrada XML acabarão como elementos da árvore analisada. Atualmente, este módulo ignora quaisquer comentários XML, instruções de processamento e declarações de tipo de documento na entrada. No entanto, árvores construídas usando a API deste módulo, em vez de serem analisadas a partir de texto XML, podem conter comentários e instruções de processamento; eles serão incluídos ao gerar a saída XML. Uma declaração de tipo de documento pode ser acessada passando uma instância personalizada `TreeBuilder` para o construtor `XMLParser`.

A API de pull para análise sem bloqueio

A maioria das funções de análise fornecidas por este módulo exigem que todo o documento seja lido de uma só vez antes de retornar qualquer resultado. É possível usar um `XMLParser` e alimentar dados nele de forma incremental, mas é uma API de push que chama métodos em um destino da função de retorno, o que é de muito baixo nível e inconveniente para a maioria das necessidades. Às vezes, o que o usuário realmente deseja é ser capaz de analisar XML de forma incremental, sem operações bloqueantes, enquanto desfruta da conveniência de objetos `Element` totalmente construídos.

A ferramenta mais poderosa para fazer isso é `XMLPullParser`. Ela não requer uma leitura bloqueante para obter os dados XML e, em vez disso, é alimentada com dados de forma incremental com chamadas de `XMLPullParser.feed()`. Para obter os elementos XML analisados, chame `XMLPullParser.read_events()`. Aqui está um exemplo:

```
>>> parser = ET.XMLPullParser(['start', 'end'])
>>> parser.feed('<mytag>sometext')
>>> list(parser.read_events())
[('start', <Element 'mytag' at 0x7fa66db2be58>)]
>>> parser.feed(' more text</mytag>')
>>> for event, elem in parser.read_events():
...     print(event)
...     print(elem.tag, 'text=', elem.text)
...
end
mytag text= sometext more text
```

O caso de uso óbvio são aplicações que operam sem bloqueio, onde os dados XML são recebidos de um soquete ou lidos de forma incremental de algum dispositivo de armazenamento. Nesses casos, leituras bloqueantes são inaceitáveis.

Because it’s so flexible, `XMLPullParser` can be inconvenient to use for simpler use-cases. If you don’t mind your application blocking on reading

XML data but would still like to have incremental parsing capabilities, take a look at `iterparse()`.

Note that both parsers build the tree incrementally: it is not freed incrementally, so every parsed element is kept until the whole document is read. To keep the memory usage low, get rid of the data which is not needed any more.

If the processed elements are large, it is enough to clear them. This works wherever they are in the tree, but the emptied elements are left in it:

```
for event, elem in ET.iterparse(source):
    if elem.tag == 'record':
        process(elem)
        elem.clear()
```

If an element has a large number of children, remove the processed children from it:

```
for event, elem in ET.iterparse(source, events=('start', 'end')):
    if event == 'start' and elem.tag == 'parent':
        parent = elem
    elif event == 'end' and elem.tag == 'child':
        process(elem)
        parent.remove(elem)
```

These examples are not universal, they only give an idea for two common cases. If you do not need a tree at all, parse with `XMLParser` and a custom target instead; it is not built then, and nothing has to be removed.

Onde o feedback *imediato* através de eventos é desejado, chamar o método `XMLPullParser.flush()` pode ajudar a reduzir o atraso; certifique-se de estudar as notas de segurança relacionadas.

Encontrando elementos interessantes

`Element` possui alguns métodos úteis que ajudam a iterar recursivamente sobre toda a subárvore abaixo dele (seus filhos, seus filhos e assim por diante). Por exemplo, `Element.iter()`:

```
>>> for neighbor in root.iter('neighbor'):
...     print(neighbor.attrib)
...
{'name': 'Austria', 'direction': 'E'}
{'name': 'Switzerland', 'direction': 'W'}
{'name': 'Malaysia', 'direction': 'N'}
{'name': 'Costa Rica', 'direction': 'W'}
{'name': 'Colombia', 'direction': 'E'}
```

`Element.findall()` encontra apenas elementos com uma tag que são filhos diretos do elemento atual. `Element.find()` encontra o *primeiro* filho com uma tag específica, e `Element.text` acessa o conteúdo de texto do elemento. `Element.get()` acessa os atributos do elemento:

```
>>> for country in root.findall('country'):
...     rank = country.find('rank').text
...     name = country.get('name')
...     print(name, rank)
...
Liechtenstein 1
Singapore 4
Panama 68
```

Uma especificação mais sofisticada de quais elementos procurar é possível usando XPath.

Modificando um arquivo XML

`ElementTree` fornece uma maneira simples de construir documentos XML e escrevê-los em arquivos. O método `ElementTree.write()` serve para esse propósito.

Uma vez criado, um objeto `Element` pode ser manipulado alterando diretamente seus campos (como `Element.text`), adicionando e modificando atributos (método `Element.set()`), bem como como adicionar novos filhos (por exemplo, com `Element.append()`).

Digamos que queremos adicionar um à classificação de cada país e adicionar um atributo `updated` ao elemento de classificação:

```
>>> for rank in root.iter('rank'):
...     new_rank = int(rank.text) + 1
...     rank.text = str(new_rank)
...     rank.set('updated', 'yes')
...
>>> tree.write('output.xml')
```

Nosso XML agora se parece com isto:

```
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank updated="yes">2</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank updated="yes">5</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
    <country name="Panama">
        <rank updated="yes">69</rank>
        <year>2011</year>
        <gdppc>13600</gdppc>
        <neighbor name="Costa Rica" direction="W"/>
        <neighbor name="Colombia" direction="E"/>
    </country>
</data>
```

Podemos remover elementos usando `Element.remove()`. Digamos que queremos remover todos os países com classificação superior a 50:

```
>>> for country in root.findall('country'):
...     # using root.findall() to avoid removal during traversal
...     rank = int(country.find('rank').text)
...     if rank > 50:
...         root.remove(country)
```

```
...
>>> tree.write('output.xml')
```

Observe que a modificação simultânea durante a iteração pode levar a problemas, assim como ao iterar e modificar listas ou dicionários do Python. Portanto, o exemplo primeiro coleta todos os elementos correspondentes com `root.findall()`, e só então itera sobre a lista de correspondências.

Nosso XML agora se parece com isto:

```
<?xml version="1.0"?>
<data>
    <country name="Liechtenstein">
        <rank updated="yes">2</rank>
        <year>2008</year>
        <gdppc>141100</gdppc>
        <neighbor name="Austria" direction="E"/>
        <neighbor name="Switzerland" direction="W"/>
    </country>
    <country name="Singapore">
        <rank updated="yes">5</rank>
        <year>2011</year>
        <gdppc>59900</gdppc>
        <neighbor name="Malaysia" direction="N"/>
    </country>
</data>
```

Construindo documentos XML

A função `SubElement()` também fornece uma maneira conveniente de criar novos subelementos para um determinado elemento:

```
>>> a = ET.Element('a')
>>> b = ET.SubElement(a, 'b')
>>> c = ET.SubElement(a, 'c')
>>> d = ET.SubElement(c, 'd')
>>> ET.dump(a)
<a><b /><c><d /></c></a>
```

Analisando XML com espaços de nomes

Se a entrada XML tiver espaços de nomes [https://en.wikipedia.org/wiki/XML_namespace], tags e atributos com prefixos no formato `prefixo:algumatag` serão expandidos para `{uri}algumatag` onde *prefixo* é substituído pelo *URI* completo. Além disso, se houver um espaço de nomes padrão [https://www.w3.org/TR/xml-names/#defaulting], esse URI completo será anexado a todas as tags não prefixadas.

Aqui está um exemplo XML que incorpora dois espaços de nomes, um com o prefixo “fictional” e outro servindo como espaço de nomes padrão:

```
<?xml version="1.0"?>
<actors xmlns:fictional="http://characters.example.com"
        xmlns="http://people.example.com">
    <actor>
        <name>John Cleese</name>
        <fictional:character>Lancelot</fictional:character>
        <fictional:character>Archie Leach</fictional:character>
    </actor>
    <actor>
        <name>Eric Idle</name>
        <fictional:character>Sir Robin</fictional:character>
        <fictional:character>Gunther</fictional:character>
        <fictional:character>Commander Clement</fictional:charac
    </actor>
</actors>
```

Uma maneira de pesquisar e explorar este exemplo XML é adicionar manualmente o URI a cada tag ou atributo no xpath de um **`find()`** ou **`findall()`**:

```
root = fromstring(xml_text)
for actor in root.findall('{http://people.example.com}actor'):
    name = actor.find('{http://people.example.com}name')
    print(name.text)
    for char in actor.findall('{http://characters.example.com}ch
        print(' |-->', char.text)
```

A melhor maneira de pesquisar o exemplo XML com espaço de nomes é criar um dicionário com seus próprios prefixos e usá-los nas funções de pesquisa:

```
ns = {'real_person': 'http://people.example.com',
      'role': 'http://characters.example.com'}

for actor in root.findall('real_person:actor', ns):
    name = actor.find('real_person:name', ns)
    print(name.text)
    for char in actor.findall('role:character', ns):
        print(' |-->', char.text)
```

Essas duas abordagens resultam no seguinte:

```
John Cleese
 |--> Lancelot
 |--> Archie Leach
Eric Idle
 |--> Sir Robin
 |--> Gunther
 |--> Commander Clement
```

Suporte a XPath

Este módulo fornece suporte limitado para expressões XPath [https://www.w3.org/TR/xpath] para localizar elementos em uma árvore. O objetivo é oferecer suporte a um pequeno subconjunto da sintaxe abreviada; um mecanismo XPath completo está fora do escopo do módulo.

Exemplo

Aqui está um exemplo que demonstra alguns dos recursos XPath do módulo. Estaremos usando o documento XML `countrydata` da seção Analisando XML:

```
import xml.etree.ElementTree as ET

root = ET.fromstring(countrydata)
```

```
# Top-level elements
root.findall(".")

# All 'neighbor' grand-children of 'country' children of the top
# elements
root.findall("./country/neighbor")

# Nodes with name='Singapore' that have a 'year' child
root.findall(".//year/..[@name='Singapore']")

# 'year' nodes that are children of nodes with name='Singapore'
root.findall(".//*[@name='Singapore']/year")

# All 'neighbor' nodes that are the second child of their parent
root.findall(".//neighbor[2]")
```

Para XML com espaços de nomes, use a notação qualificada usual `{espaço-de-nomes}tag`:

```
# All dublin-core "title" tags in the document
root.findall(".//{http://purl.org/dc/elements/1.1/}title")
```

Supported XPath syntax

Sintaxe	Significado

Sintaxe	Significado
`tag`	Selects all child elements with the given tag. For example, `spam` selects all child elements named `spam`, and `spam/egg` selects all grandchildren named `egg` in all children named `spam`. `{namespace}*` selects all tags in the given namespace, `{*}spam` selects tags named `spam` in any (or no) namespace, and `{}*` only selects tags that are not in a namespace. *Alterado na versão 3.8:* Support for star-wildcards was added.
`*`	Selects all child elements, including comments and processing instructions. For example, `*/egg` selects all grandchildren named `egg`.
`.`	Selects the current node. This is mostly useful at the beginning of the path, to indicate that it’s a relative path.
`//`	Selects all subelements, on all levels beneath the current element. For example, `.//egg` selects all `egg` elements in the entire tree.
`..`	Selects the parent element. Returns `None` if the path attempts to reach the ancestors of the start element (the element `find` was called on).

Sintaxe	Significado
`[@attrib]`	Selects all elements that have the given attribute.
`[@attrib='value']`	Selects all elements for which the given attribute has the given value. The value cannot contain quotes.
`[@attrib!='value']`	Selects all elements for which the given attribute does not have the given value. The value cannot contain quotes. *Adicionado na versão 3.10.*
`[tag]`	Selects all elements that have a child named `tag`. Only immediate children are supported.
`[.='text']`	Selects all elements whose complete text content, including descendants, equals the given `text`. *Adicionado na versão 3.7.*
`[.!='text']`	Selects all elements whose complete text content, including descendants, does not equal the given `text`. *Adicionado na versão 3.10.*

Sintaxe	Significado
`[tag='text']`	Selects all elements that have a child named `tag` whose complete text content, including descendants, equals the given `text`.
`[tag!='text']`	Selects all elements that have a child named `tag` whose complete text content, including descendants, does not equal the given `text`. *Adicionado na versão 3.10.*
`[position]`	Selects all elements that are located at the given position. The position can be either an integer (1 is the first position), the expression `last()` (for the last position), or a position relative to the last position (e.g. `last()-1`).

Predicates (expressions within square brackets) must be preceded by a tag name, an asterisk, or another predicate. `position` predicates must be preceded by a tag name.

Referência

Funções

xml.etree.ElementTree.canonicalize(*xml_data=None*, *, *out=None*, *from_file=None*, ***options*)

C14N 2.0 [https://www.w3.org/TR/xml-c14n2/] transformation function.

Canonicalization is a way to normalise XML output in a way that allows byte-by-byte comparisons and digital signatures. It reduces the

freedom that XML serializers have and instead generates a more constrained XML representation. The main restrictions regard the placement of namespace declarations, the ordering of attributes, and ignorable whitespace.

This function takes an XML data string (*xml_data*) or a file path or file-like object (*from_file*) as input, converts it to the canonical form, and writes it out using the *out* file(-like) object, if provided, or returns it as a text string if not. The output file receives text, not bytes. It should therefore be opened in text mode with `utf-8` encoding.

Typical uses:

```
xml_data = "<root>...</root>"
print(canonicalize(xml_data))

with open("c14n_output.xml", mode='w', encoding='utf-8') as o
    canonicalize(xml_data, out=out_file)

with open("c14n_output.xml", mode='w', encoding='utf-8') as o
    canonicalize(from_file="inputfile.xml", out=out_file)
```

As *options* de configuração são as seguintes:

- *with_comments*: set to true to include comments (default: false)
- *strip_text*: set to true to strip whitespace before and after text content
  (default: false)
- *rewrite_prefixes*: set to true to replace namespace prefixes by “n{number}”
  (default: false)
- *qname_aware_tags*: a set of qname aware tag names in which prefixes
  should be replaced in text content (default: empty)
- *qname_aware_attrs*: a set of qname aware attribute names in which prefixes

should be replaced in text content (default: empty)

- *exclude_attrs*: a set of attribute names that should not be serialised
- *exclude_tags*: a set of tag names that should not be serialised

In the option list above, “a set” refers to any collection or iterable of strings, no ordering is expected.

Adicionado na versão 3.8.

xml.etree.ElementTree.Comment(*text=None*)

Comment element factory. This factory function creates a special element that will be serialized as an XML comment by the standard serializer. *text* is a string containing the comment string. Returns an element instance representing a comment.

Note that `XMLParser` skips over comments in the input instead of creating comment objects for them. An `ElementTree` will only contain comment nodes if they have been inserted into to the tree using one of the `Element` methods.

xml.etree.ElementTree.dump(*elem*)

Writes an element tree or element structure to sys.stdout. This function should be used for debugging only.

The exact output format is implementation dependent. In this version, it’s written as an ordinary XML file.

elem is an element tree or an individual element.

Alterado na versão 3.8: The `dump()` function now preserves the attribute order specified by the user.

xml.etree.ElementTree.fromstring(*text*, *parser=None*)

Parses an XML section from a string constant. Same as `XML()`. *text* is a string containing XML data. *parser* is an optional parser instance. If not given, the standard `XMLParser` parser is used. Returns an `Element` instance.

xml.etree.ElementTree.fromstringlist(*sequence*, *parser=None*)

Parses an XML document from a sequence of string fragments. *sequence* is a list or other sequence containing XML data fragments. *parser* is an optional parser instance. If not given, the standard **`XMLParser`** parser is used. Returns an **`Element`** instance.

Adicionado na versão 3.2.

xml.etree.ElementTree.indent(*tree*, *space=' '*, *level=0*)

Appends whitespace to the subtree to indent the tree visually. This can be used to generate pretty-printed XML output. *tree* can be an Element or ElementTree. *space* is the whitespace string that will be inserted for each indentation level, two space characters by default. For indenting partial subtrees inside of an already indented tree, pass the initial indentation level as *level*.

Adicionado na versão 3.9.

xml.etree.ElementTree.iselement(*element*)

Check if an object appears to be a valid element object. *element* is an element instance. Return `True` if this is an element object.

xml.etree.ElementTree.iterparse(*source*, *events=None*, *parser=None*)

Parses an XML section into an element tree incrementally, and reports what's going on to the user. *source* is a filename or file object containing XML data. *events* is a sequence of events to report back. The supported events are the strings `"start"`, `"end"`, `"comment"`, `"pi"`, `"start-ns"` and `"end-ns"` (the "ns" events are used to get detailed namespace information). If *events* is omitted, only `"end"` events are reported. *parser* is an optional parser instance. If not given, the standard **`XMLParser`** parser is used. *parser* must be an instance of **`XMLParser`** or its subclass and can only use the default **`TreeBuilder`** as a target. Returns an iterator providing `(event, elem)` pairs; it has a `root` attribute that references the root element of the resulting XML tree once *source* is fully read. The iterator has the **`close()`** method that closes the internal file object if *source* is a filename.

Note that while **`iterparse()`** builds the tree incrementally, it issues blocking reads on *source* (or the file it names). As such, it's unsuitable for applications where blocking reads can't be made. For fully non-blocking parsing, see **`XMLPullParser`**.

The tree is only built incrementally, it is not freed incrementally: every parsed element is kept until the whole document is read. See A API de pull para análise sem bloqueio for how to keep the memory usage low.

Nota

`iterparse()` only guarantees that it has seen the ">" character of a starting tag when it emits a "start" event, so the attributes are defined, but the contents of the text and tail attributes are undefined at that point. The same applies to the element children; they may or may not be present.

If you need a fully populated element, look for "end" events instead.

Descontinuado desde a versão 3.4: O argumento *parser*.

Alterado na versão 3.8: Os eventos `comment` e `pi` foram adicionados.

Alterado na versão 3.13: Added the **`close()`** method.

Alterado na versão 3.15: A **`ResourceWarning`** is now emitted if the iterator opened a file and is not explicitly closed.

xml.etree.ElementTree.parse(*source*, *parser=None*)

Parses an XML section into an element tree. *source* is a filename or file object containing XML data. *parser* is an optional parser instance. If not given, the standard **`XMLParser`** parser is used. Returns an **`ElementTree`** instance.

xml.etree.ElementTree.ProcessingInstruction(*target*, *text=None*)

PI element factory. This factory function creates a special element that will be serialized as an XML processing instruction. *target* is a string

containing the PI target. *text* is a string containing the PI contents, if given. Returns an element instance, representing a processing instruction.

Note that `XMLParser` skips over processing instructions in the input instead of creating PI objects for them. An `ElementTree` will only contain processing instruction nodes if they have been inserted into to the tree using one of the `Element` methods.

xml.etree.ElementTree.register_namespace(*prefix*, *uri*)

Registers a namespace prefix. The registry is global, and any existing mapping for either the given prefix or the namespace URI will be removed. *prefix* is a namespace prefix. *uri* is a namespace uri. Tags and attributes in this namespace will be serialized with the given prefix, if at all possible.

Adicionado na versão 3.2.

xml.etree.ElementTree.SubElement(*parent*, *tag*, *, *attrib={}*, ***extra*)

Subelement factory. This function creates an element instance, and appends it to an existing element.

parent is the parent element. *tag* is the subelement name. *attrib* is an optional dictionary, containing element attributes. *extra* contains additional attributes, given as keyword arguments. Returns an element instance.

Alterado na versão 3.15: attrib can now be a `frozendict`.

Alterado na versão 3.15: parent and *tag* are now positional-only parameters.

xml.etree.ElementTree.tostring(*element*, *encoding='us-ascii'*, *method='xml'*, *, *xml_declaration=None*, *default_namespace=None*, *short_empty_elements=True*)

Generates a string representation of an XML element, including all subelements. *element* is an `Element` instance. *encoding* [[1]] is the

output encoding (default is US-ASCII). Use `encoding="unicode"` to generate a Unicode string (otherwise, a bytestring is generated). *method* is either `"xml"`, `"html"` or `"text"` (default is `"xml"`). *xml_declaration*, *default_namespace* and *short_empty_elements* has the same meaning as in **`ElementTree.write()`**. Returns an (optionally) encoded string containing the XML data.

Alterado na versão 3.4: Foi adicionado o parâmetro *short_empty_elements*.

Alterado na versão 3.8: Added the *xml_declaration* and *default_namespace* parameters.

Alterado na versão 3.8: The **`tostring()`** function now preserves the attribute order specified by the user.

xml.etree.ElementTree.tostringlist(*element*, *encoding='us-ascii'*, *method='xml'*, ***, *xml_declaration=None*, *default_namespace=None*, *short_empty_elements=True*)

Generates a string representation of an XML element, including all subelements. *element* is an **`Element`** instance. *encoding* [[1]] is the output encoding (default is US-ASCII). Use `encoding="unicode"` to generate a Unicode string (otherwise, a bytestring is generated). *method* is either `"xml"`, `"html"` or `"text"` (default is `"xml"`). *xml_declaration*, *default_namespace* and *short_empty_elements* has the same meaning as in **`ElementTree.write()`**. Returns a list of (optionally) encoded strings containing the XML data. It does not guarantee any specific sequence, except that `b"".join(tostringlist(element)) == tostring(element)`.

Adicionado na versão 3.2.

Alterado na versão 3.4: Foi adicionado o parâmetro *short_empty_elements*.

Alterado na versão 3.8: Added the *xml_declaration* and *default_namespace* parameters.

Alterado na versão 3.8: The **`tostringlist()`** function now preserves the attribute order specified by the user.

xml.etree.ElementTree.XML(*text*, *parser=None*)

Parses an XML section from a string constant. This function can be used to embed “XML literals” in Python code. *text* is a string containing XML data. *parser* is an optional parser instance. If not given, the standard **`XMLParser`** parser is used. Returns an **`Element`** instance.

xml.etree.ElementTree.XMLID(*text*, *parser=None*)

Parses an XML section from a string constant, and also returns a dictionary which maps from element id:s to elements. *text* is a string containing XML data. *parser* is an optional parser instance. If not given, the standard **`XMLParser`** parser is used. Returns a tuple containing an **`Element`** instance and a dictionary.

Suporte a XInclude

This module provides limited support for XInclude directives [https://www.w3.org/TR/xinclude/], via the **`xml.etree.ElementInclude`** helper module. This module can be used to insert subtrees and text strings into element trees, based on information in the tree.

Exemplo

Here’s an example that demonstrates use of the XInclude module. To include an XML document in the current document, use the `{http://www.w3.org/2001/XInclude}include` element and set the **parse** attribute to `"xml"`, and use the **href** attribute to specify the document to include.

```
<?xml version="1.0"?>
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <xi:include href="source.xml" parse="xml" />
</document>
```

By default, the **href** attribute is treated as a file name. You can use custom loaders to override this behaviour. Also note that the standard helper does not support XPointer syntax.

To process this file, load it as usual, and pass the root element to the **`xml.etree.ElementTree`** module:

```
from xml.etree import ElementTree, ElementInclude

tree = ElementTree.parse("document.xml")
root = tree.getroot()

ElementInclude.include(root)
```

The ElementInclude module replaces the `{http://www.w3.org/2001/XInclude}include` element with the root element from the **source.xml** document. The result might look something like this:

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  <para>This is a paragraph.</para>
</document>
```

If the **parse** attribute is omitted, it defaults to “xml”. The href attribute is required.

To include a text document, use the `{http://www.w3.org/2001/XInclude}include` element, and set the **parse** attribute to “text”:

```
<?xml version="1.0"?>
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  Copyright (c) <xi:include href="year.txt" parse="text" />.
</document>
```

The result might look something like:

```
<document xmlns:xi="http://www.w3.org/2001/XInclude">
  Copyright (c) 2003.
</document>
```

Referência

Funções

xml.etree.ElementInclude.default_loader(*href*, *parse*, *encoding=None*)

Default loader. This default loader reads an included resource from disk. *href* is a URL. *parse* is for parse mode either “xml” or “text”. *encoding* is an optional text encoding. If not given, encoding is `utf-8`. Returns the expanded resource. If the parse mode is `"xml"`, this is an **`Element`** instance. If the parse mode is `"text"`, this is a string. If the loader fails, it can return `None` or raise an exception.

xml.etree.ElementInclude.include(*elem*, *loader=None*, *base_url=None*, *max_depth=6*)

This function expands XInclude directives in-place in tree pointed by *elem*. *elem* is either the root **`Element`** or an **`ElementTree`** instance to find such element. *loader* is an optional resource loader. If omitted, it defaults to **`default_loader()`**. If given, it should be a callable that implements the same interface as **`default_loader()`**. *base_url* is base URL of the original file, to resolve relative include file references. *max_depth* is the maximum number of recursive inclusions. Limited to reduce the risk of malicious content explosion. Pass `None` to disable the limitation.

Alterado na versão 3.9: Added the *base_url* and *max_depth* parameters.

Element Objects

class xml.etree.ElementTree.Element(*tag*, *, *attrib={}*, ***extra*)

Element class. This class defines the Element interface, and provides a reference implementation of this interface.

tag is the element name. *attrib* is an optional dictionary, containing element attributes. *extra* contains additional attributes, given as keyword arguments.

The element name and the attribute names and values are strings or `QName` instances, and the text and the tail are strings or `None`. The element name can also be `Comment()` or `ProcessingInstruction()`, which are used for special elements. If it is `None`, the element itself is not serialized: only its text and its children are written, and its attributes are ignored. This can be used for a fragment which contains several elements. With `method="html"` the attribute value can also be `None`, which produces an empty attribute (such as `checked`). Other objects can be stored in the tree, but they cannot be serialized.

Alterado na versão 3.15: attrib can now be a `frozendict`.

Alterado na versão 3.15: tag is now a positional-only parameter.

tag

A string identifying what kind of data this element represents (the element type, in other words).

text
tail

These attributes can be used to hold additional data associated with the element. Their values are usually strings but may be any application-specific object. If the element is created from an XML file, the *text* attribute holds either the text between the element's start tag and its first child or end tag, or `None`, and the *tail* attribute holds either the text between the element's end tag and the next tag, or `None`. For the XML data

```
<a><b>1<c>2<d/>3</c></b>4</a>
```

the *a* element has `None` for both *text* and *tail* attributes, the *b* element has *text* `"1"` and *tail* `"4"`, the *c* element has *text* `"2"` and *tail* `None`, and the *d* element has *text* `None` and *tail* `"3"`.

To collect the inner text of an element, see `itertext()`, for example `"".join(element.itertext())`.

Applications may store arbitrary objects in these attributes.

attrib

A dictionary containing the element's attributes. Note that while the *attrib* value is always a real mutable Python dictionary, an ElementTree implementation may choose to use another internal representation, and create the dictionary only if someone asks for it. To take advantage of such implementations, use the dictionary methods below whenever possible.

The following dictionary-like methods work on the element attributes.

clear()

Resets an element. This function removes all subelements, clears all attributes, and sets the text and tail attributes to `None`.

get(*key*, *default=None*)

Gets the element attribute named *key*.

Returns the attribute value, or *default* if the attribute was not found.

items()

Returns the element attributes as (name, value) pairs.

keys()

Returns the element attribute names.

set(*key*, *value*)

Set the attribute *key* on the element to *value*.

Os métodos a seguir funcionam nos filhos do elemento (subelementos).

append(*subelement*)

Adds the element *subelement* to the end of this element's internal list of subelements. Raises `TypeError` if *subelement* is not an `Element`.

extend(*subelements*)

Appends *subelements* from an iterable of elements. Raises **`TypeError`** if a subelement is not an **`Element`**.

Adicionado na versão 3.2.

find(*match*, *namespaces=None*)

Finds the first subelement matching *match*. *match* may be a tag name or a path. Returns an element instance or `None`. *namespaces* is an optional mapping from namespace prefix to full name. Pass `''` as prefix to move all unprefixed tag names in the expression into the given namespace.

findall(*match*, *namespaces=None*)

Finds all matching subelements, by tag name or path. Returns a list containing all matching elements in document order. *namespaces* is an optional mapping from namespace prefix to full name. Pass `''` as prefix to move all unprefixed tag names in the expression into the given namespace.

findtext(*match*, *default=None*, *namespaces=None*)

Finds text for the first subelement matching *match*. *match* may be a tag name or a path. Returns the text content of the first matching element, or *default* if no element was found. Note that if the matching element has no text content an empty string is returned. *namespaces* is an optional mapping from namespace prefix to full name. Pass `''` as prefix to move all unprefixed tag names in the expression into the given namespace.

insert(*index*, *subelement*)

Inserts *subelement* at the given position in this element. Raises **`TypeError`** if *subelement* is not an **`Element`**.

iter(*tag=None*)

Creates a tree iterator with the current element as the root. The iterator iterates over this element and all elements below it, in document (depth first) order. If *tag* is not `None` or `'*'`, only elements

whose tag equals *tag* are returned from the iterator. If the tree structure is modified during iteration, the result is undefined.

Adicionado na versão 3.2.

iterfind(*match*, *namespaces=None*)

Finds all matching subelements, by tag name or path. Returns an iterable yielding all matching elements in document order. *namespaces* is an optional mapping from namespace prefix to full name.

Adicionado na versão 3.2.

itertext()

Creates a text iterator. The iterator loops over this element and all subelements, in document order, and returns all inner text.

Adicionado na versão 3.2.

makeelement(*tag*, *attrib*)

Creates a new element object of the same type as this element. Do not call this method, use the `SubElement()` factory function instead.

remove(*subelement*)

Removes *subelement* from the element. Unlike the find* methods this method compares elements based on the instance identity, not on tag value or contents.

`Element` objects also support the following sequence type methods for working with subelements: `__delitem__()`, `__getitem__()`, `__setitem__()`, `__len__()`.

Caution: Elements with no subelements will test as `False`. In a future release of Python, all elements will test as `True` regardless of whether subelements exist. Instead, prefer explicit `len(elem)` or `elem is not None` tests.:

```
element = root.find('foo')

if not element:  # careful!
    print("element not found, or element has no subelements")

if element is None:
    print("element not found")
```

Alterado na versão 3.12: Testing the truth value of an Element emits **`DeprecationWarning`**.

Prior to Python 3.8, the serialisation order of the XML attributes of elements was artificially made predictable by sorting the attributes by their name. Based on the now guaranteed ordering of dicts, this arbitrary reordering was removed in Python 3.8 to preserve the order in which attributes were originally parsed or created by user code.

In general, user code should try not to depend on a specific ordering of attributes, given that the XML Information Set [https://www.w3.org/TR/xml-infoset/] explicitly excludes the attribute order from conveying information. Code should be prepared to deal with any ordering on input. In cases where deterministic XML output is required, e.g. for cryptographic signing or test data sets, canonical serialisation is available with the **`canonicalize()`** function.

In cases where canonical output is not applicable but a specific attribute order is still desirable on output, code should aim for creating the attributes directly in the desired order, to avoid perceptual mismatches for readers of the code. In cases where this is difficult to achieve, a recipe like the following can be applied prior to serialisation to enforce an order independently from the Element creation:

```
def reorder_attributes(root):
    for el in root.iter():
        attrib = el.attrib
        if len(attrib) > 1:
            # adjust attribute order, e.g. by sorting
            attribs = sorted(attrib.items())
```

```
attrib.clear()
attrib.update(attribs)
```

ElementTree Objects

class xml.etree.ElementTree.ElementTree(*element=None*, *file=None*)

ElementTree wrapper class. This class represents an entire element hierarchy, and adds some extra support for serialization to and from standard XML.

element is the root element. The tree is initialized with the contents of the XML *file* if given.

_setroot(*element*)

Replaces the root element for this tree. This discards the current contents of the tree, and replaces it with the given element. Use with care. *element* is an element instance.

find(*match*, *namespaces=None*)

Same as `Element.find()`, starting at the root of the tree.

findall(*match*, *namespaces=None*)

Same as `Element.findall()`, starting at the root of the tree.

findtext(*match*, *default=None*, *namespaces=None*)

Same as `Element.findtext()`, starting at the root of the tree.

getroot()

Returns the root element for this tree.

iter(*tag=None*)

Creates and returns a tree iterator for the root element. The iterator loops over all elements in this tree, in section order. *tag* is the tag to look for (default is to return all elements).

iterfind(*match*, *namespaces=None*)

Same as **`Element.iterfind()`**, starting at the root of the tree.

Adicionado na versão 3.2.

parse(*source*, *parser=None*)

Loads an external XML section into this element tree. *source* is a file name or file object. *parser* is an optional parser instance. If not given, the standard **`XMLParser`** parser is used. Returns the section root element.

write(*file*, *encoding='us-ascii'*, *xml_declaration=None*, *default_namespace=None*, *method='xml'*, *, *short_empty_elements=True*)

Writes the element tree to a file, as XML. *file* is a file name, or a file object opened for writing. *encoding* [[1]] is the output encoding (default is US-ASCII). *xml_declaration* controls if an XML declaration should be added to the file. Use `False` for never, `True` for always, `None` for only if not US-ASCII or UTF-8 or Unicode (default is `None`). *default_namespace* sets the default XML namespace (for “xmlns”). *method* is either `"xml"`, `"html"` or `"text"` (default is `"xml"`). The keyword-only *short_empty_elements* parameter controls the formatting of elements that contain no content. If `True` (the default), they are emitted as a single self-closed tag, otherwise they are emitted as a pair of start/end tags.

The output is either a string (**`str`**) or binary (**`bytes`**). This is controlled by the *encoding* argument. If *encoding* is `"unicode"`, the output is a string; otherwise, it’s binary. Note that this may conflict with the type of *file* if it’s an open file object; make sure you do not try to write a string to a binary stream and vice versa.

Alterado na versão 3.4: Foi adicionado o parâmetro *short_empty_elements*.

Alterado na versão 3.8: The **`write()`** method now preserves the attribute order specified by the user.

This is the XML file that is going to be manipulated:

```
<html>
    <head>
        <title>Example page</title>
    </head>
    <body>
        <p>Moved to <a href="http://example.org/">example.org</a
        or <a href="http://example.com/">example.com</a>.</p>
    </body>
</html>
```

Example of changing the attribute “target” of every link in first paragraph:

```
>>> from xml.etree.ElementTree import ElementTree
>>> tree = ElementTree()
>>> tree.parse("index.xhtml")
<Element 'html' at 0xb77e6fac>
>>> p = tree.find("body/p")     # Finds first occurrence of tag
>>> p
<Element 'p' at 0xb77ec26c>
>>> links = list(p.iter("a"))   # Returns list of all links
>>> links
[<Element 'a' at 0xb77ec2ac>, <Element 'a' at 0xb77ec1cc>]
>>> for i in links:             # Iterates through all found lin
...     i.attrib["target"] = "blank"
...
>>> tree.write("output.xhtml")
```

QName Objects

class xml.etree.ElementTree.QName(*text_or_uri*, *tag=None*)

QName wrapper. This can be used to wrap a QName attribute value, in order to get proper namespace handling on output. *text_or_uri* is a string containing the QName value, in the form {uri}local, or, if the tag argument is given, the URI part of a QName. If *tag* is given, the first argument is interpreted as a URI, and this argument is interpreted as a local name. **`QName`** instances are opaque.

TreeBuilder Objects

class xml.etree.ElementTree.TreeBuilder(*element_factory=None*, *, *comment_factory=None*, *pi_factory=None*, *insert_comments=False*, *insert_pis=False*)

Generic element structure builder. This builder converts a sequence of start, data, end, comment and pi method calls to a well-formed element structure. You can use this class to build an element structure using a custom XML parser, or a parser for some other XML-like format.

element_factory, when given, must be a callable accepting two positional arguments: a tag and a dict of attributes. It is expected to return a new element instance.

The *comment_factory* and *pi_factory* functions, when given, should behave like the `Comment()` and `ProcessingInstruction()` functions to create comments and processing instructions. When not given, the default factories will be used. When *insert_comments* and/or *insert_pis* is true, comments/pis will be inserted into the tree if they appear within the root element (but not outside of it).

close()

Flushes the builder buffers, and returns the toplevel document element. Returns an `Element` instance.

data(*data*)

Adds text to the current element. *data* is a string.

end(*tag*)

Closes the current element. *tag* is the element name. Returns the closed element.

start(*tag*, *attrs*)

Opens a new element. *tag* is the element name. *attrs* is a dictionary containing element attributes. Returns the opened element.

comment(*text*)

Creates a comment with the given *text*. If `insert_comments` is true, this will also add it to the tree.

Adicionado na versão 3.8.

pi(*target*, *text*)

Creates a process instruction with the given *target* name and *text*. If `insert_pis` is true, this will also add it to the tree.

Adicionado na versão 3.8.

In addition, a custom **`TreeBuilder`** object can provide the following methods:

doctype(*name*, *pubid*, *system*)

Handles a doctype declaration. *name* is the doctype name. *pubid* is the public identifier. *system* is the system identifier. This method does not exist on the default **`TreeBuilder`** class.

Adicionado na versão 3.2.

start_ns(*prefix*, *uri*)

Is called whenever the parser encounters a new namespace declaration, before the `start()` callback for the opening element that defines it. *prefix* is `''` for the default namespace and the declared namespace prefix name otherwise. *uri* is the namespace URI.

Adicionado na versão 3.8.

end_ns(*prefix*)

Is called after the `end()` callback of an element that declared a namespace prefix mapping, with the name of the *prefix* that went out of scope.

Adicionado na versão 3.8.

class xml.etree.ElementTree.C14NWriterTarget(*write*, *, *with_comments=False*, *strip_text=False*, *rewrite_prefixes=False*, *qname_aware_tags=None*, *qname_aware_attrs=None*, *exclude_attrs=None*, *exclude_tags=None*)

A C14N 2.0 [https://www.w3.org/TR/xml-c14n2/] writer. Arguments are the same as for the `canonicalize()` function. This class does not build a tree but translates the callback events directly into a serialised form using the *write* function.

Adicionado na versão 3.8.

Objetos XMLParser

class xml.etree.ElementTree.XMLParser(*, *target=None*, *encoding=None*)

This class is the low-level building block of the module. It uses `xml.parsers.expat` for efficient, event-based parsing of XML. It can be fed XML data incrementally with the `feed()` method, and parsing events are translated to a push API - by invoking callbacks on the *target* object. If *target* is omitted, the standard `TreeBuilder` is used. If *encoding* [[1]] is given, the value overrides the encoding specified in the XML file.

Alterado na versão 3.8: Parameters are now keyword-only. The *html* argument is no longer supported.

close()

Finishes feeding data to the parser. Returns the result of calling the `close()` method of the *target* passed during construction; by default, this is the toplevel document element.

feed(*data*)

Feeds data to the parser. *data* is a string or encoded data (`bytes` or a bytes-like object).

flush()

Triggers parsing of any previously fed unparsed data, which can be used to ensure more immediate feedback, in particular with Expat >=2.6.0. The implementation of **`flush()`** temporarily disables reparse deferral with Expat (if currently enabled) and triggers a reparse. Disabling reparse deferral has security consequences; please see **`xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()`** for details.

`flush()` has been backported to some prior releases of CPython as a security fix. Check for availability using **`hasattr()`** if used in code running across a variety of Python versions.

Adicionado na versão 3.13.

`XMLParser.feed()` calls *target*'s `start(tag, attrs_dict)` method for each opening tag, its `end(tag)` method for each closing tag, and data is processed by method `data(data)`. For further supported callback methods, see the **`TreeBuilder`** class. **`XMLParser.close()`** calls *target*'s method `close()`. **`XMLParser`** can be used not only for building a tree structure. This is an example of counting the maximum depth of an XML file:

```
>>> from xml.etree.ElementTree import XMLParser
>>> class MaxDepth:                     # The target object c
...     maxDepth = 0
...     depth = 0
...     def start(self, tag, attrib):   # Called for each ope
...         self.depth += 1
...         if self.depth > self.maxDepth:
...             self.maxDepth = self.depth
...     def end(self, tag):             # Called for each clo
...         self.depth -= 1
...     def data(self, data):
...         pass            # We do not need to do anything w
...     def close(self):    # Called when all data has been p
...         return self.maxDepth
...
>>> target = MaxDepth()
>>> parser = XMLParser(target=target)
```

```
>>> exampleXml = """
... <a>
...   <b>
...   </b>
...   <b>
...     <c>
...       <d>
...       </d>
...     </c>
...   </b>
... </a>"""
>>> parser.feed(exampleXml)
>>> parser.close()
4
```

XMLPullParser Objects

class xml.etree.ElementTree.XMLPullParser(*events=None*)

A pull parser suitable for non-blocking applications. Its input-side API is similar to that of **`XMLParser`**, but instead of pushing calls to a callback target, **`XMLPullParser`** collects an internal list of parsing events and lets the user read from it. *events* is a sequence of events to report back. The supported events are the strings `"start"`, `"end"`, `"comment"`, `"pi"`, `"start-ns"` and `"end-ns"` (the “ns” events are used to get detailed namespace information). If *events* is omitted, only `"end"` events are reported.

feed(*data*)

Feed the given data to the parser. *data* is a string or encoded data (**`bytes`** or a bytes-like object).

flush()

Triggers parsing of any previously fed unparsed data, which can be used to ensure more immediate feedback, in particular with Expat >=2.6.0. The implementation of **`flush()`** temporarily disables reparse deferral with Expat (if currently enabled) and triggers a reparse. Disabling reparse deferral has security consequences;

please see `xml.parsers.expat.xmlparser.SetReparseDeferralEnabled()` for details.

`flush()` has been backported to some prior releases of CPython as a security fix. Check for availability using `hasattr()` if used in code running across a variety of Python versions.

Adicionado na versão 3.13.

close()

Signal the parser that the data stream is terminated. Unlike `XMLParser.close()`, this method always returns `None`. Any events not yet retrieved when the parser is closed can still be read with `read_events()`.

read_events()

Return an iterator over the events which have been encountered in the data fed to the parser. The iterator yields `(event, elem)` pairs, where *event* is a string representing the type of event (e.g. `"end"`) and *elem* is the encountered `Element` object, or other context value as follows.

- `start`, `end`: O Element atual.
- `comment`, `pi`: the current comment / processing instruction
- `start-ns`: a tuple `(prefix, uri)` naming the declared namespace mapping.
- `end-ns`: `None` (this may change in a future version)

Events provided in a previous call to **`read_events()`** will not be yielded again. Events are consumed from the internal queue only when they are retrieved from the iterator, so multiple readers iterating in parallel over iterators obtained from **`read_events()`** will have unpredictable results.

Nota

`XMLPullParser` only guarantees that it has seen the “>” character of a starting tag when it emits a “start” event, so the attributes are defined, but the contents of the text and tail attributes are undefined at that point. The same applies to the element children; they may or may not be present.

If you need a fully populated element, look for “end” events instead.

Adicionado na versão 3.4.

Alterado na versão 3.8: Os eventos `comment` e `pi` foram adicionados.

Exceções

class xml.etree.ElementTree.ParseError

XML parse error, raised by the various parsing methods in this module when parsing fails. The string representation of an instance of this exception will contain a user-friendly error message. In addition, it will have the following attributes available:

code

A numeric error code from the expat parser. See the documentation of **`xml.parsers.expat`** for the list of error codes and their meanings.

position

A tuple of *line*, *column* numbers, specifying where the error occurred.

Notas de rodapé

[1] (1,2,3,4)

The encoding string included in XML output should conform to the appropriate standards. For example, “UTF-8” is valid, but “UTF8” is not. See https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-

EncodingDecl and https://www.iana.org/assignments/character-sets/character-sets.xhtml.

`xml.dom` — The Document Object Model API

Código-fonte: Lib/xml/dom/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/dom/__init__.py]

The Document Object Model, or “DOM,” is a cross-language API from the World Wide Web Consortium (W3C) for accessing and modifying XML documents. A DOM implementation presents an XML document as a tree structure, or allows client code to build such a structure from scratch. It then gives access to the structure through a set of objects which provided well-known interfaces.

The DOM is extremely useful for random-access applications. SAX only allows you a view of one bit of the document at a time. If you are looking at one SAX element, you have no access to another. If you are looking at a text node, you have no access to a containing element. When you write a SAX application, you need to keep track of your program’s position in the document somewhere in your own code. SAX does not do it for you. Also, if you need to look ahead in the XML document, you are just out of luck.

Some applications are simply impossible in an event driven model with no access to a tree. Of course you could build some sort of tree yourself in SAX events, but the DOM allows you to avoid writing that code. The DOM is a standard tree representation for XML data.

The Document Object Model is being defined by the W3C in stages, or “levels” in their terminology. The Python mapping of the API is substantially based on the DOM Level 2 recommendation.

DOM applications typically start by parsing some XML into a DOM. How this is accomplished is not covered at all by DOM Level 1, and Level 2 provides only limited improvements: There is a `DOMImplementation` object class which provides access to `Document` creation methods, but no way to access an XML reader/parser/Document builder in an implementation-independent way. There is also no well-defined way to access these methods without an existing `Document` object. In Python, each DOM implementation will provide a function `getDOMImplementation()`. DOM Level 3 adds a Load/Store specification, which

defines an interface to the reader, but this is not yet available in the Python standard library.

Once you have a DOM document object, you can access the parts of your XML document through its properties and methods. These properties are defined in the DOM specification; this portion of the reference manual describes the interpretation of the specification in Python.

The specification provided by the W3C defines the DOM API for Java, ECMAScript, and OMG IDL. The Python mapping defined here is based in large part on the IDL version of the specification, but strict compliance is not required (though implementations are free to support the strict mapping from IDL). See section Conformance for a detailed discussion of mapping requirements.

Ver também

Document Object Model (DOM) Level 2 Specification
[https://www.w3.org/TR/2000/REC-DOM-Level-2-Core-20001113/]
The W3C recommendation upon which the Python DOM API is based.

Document Object Model (DOM) Level 1 Specification
[https://www.w3.org/TR/REC-DOM-Level-1/]
The W3C recommendation for the DOM supported by `xml.dom.minidom`.

Python Language Mapping Specification **[https://www.omg.org/spec/PYTH/1.2/PDF]**
This specifies the mapping from OMG IDL to Python.

Conteúdo do módulo

The `xml.dom` contains the following functions:

xml.dom.registerDOMImplementation(*name*, *factory*)
Register the *factory* function with the name *name*. The factory function should return an object which implements the `DOMImplementation` interface. The factory function can return the same object every time, or a new one for each call, as appropriate for the specific implementation (e.g. if that implementation supports some customization).

xml.dom.getDOMImplementation(*name=None*, *features=()*)

Return a suitable DOM implementation. The *name* is either well-known, the module name of a DOM implementation, or `None`. If it is not `None`, imports the corresponding module and returns a **`DOMImplementation`** object if the import succeeds. If no name is given, and if the environment variable **`PYTHON_DOM`** is set, this variable is used to find the implementation. The only well-known name in the standard library is `'minidom'`, for **`xml.dom.minidom`**.

If name is not given, this examines the available implementations to find one with the required feature set. If no implementation can be found, raise an **`ImportError`**. The features list must be a sequence of `(feature, version)` pairs which are passed to the **`hasFeature()`** method on available **`DOMImplementation`** objects.

Some convenience constants are also provided:

xml.dom.EMPTY_NAMESPACE

The value used to indicate that no namespace is associated with a node in the DOM. This is typically found as the **`namespaceURI`** of a node, or used as the *namespaceURI* parameter to a namespaces-specific method.

xml.dom.XML_NAMESPACE

The namespace URI associated with the reserved prefix `xml`, as defined by Namespaces in XML [https://www.w3.org/TR/REC-xml-names/] (section 4).

xml.dom.XMLNS_NAMESPACE

The namespace URI for namespace declarations, as defined by Document Object Model (DOM) Level 2 Core Specification [https://www.w3.org/TR/DOM-Level-2-Core/core.html] (section 1.1.8).

xml.dom.XHTML_NAMESPACE

The URI of the XHTML namespace as defined by XHTML 1.0: The Extensible HyperText Markup Language [https://www.w3.org/TR/xhtml1/] (section 3.1.1).

In addition, **`xml.dom`** contains a base **`Node`** class and the DOM exception classes. The **`Node`** class provided by this module does not implement any of the methods or attributes defined by the DOM specification; concrete DOM implementations must provide those. The **`Node`** class provided as part of this module does provide the constants used for the **`nodeType`** attribute on concrete **`Node`** objects; they are located within the class rather than at the module level to conform with the DOM specifications.

Objects in the DOM

The definitive documentation for the DOM is the DOM specification from the W3C.

The names documented in this section are DOM interfaces. With the exception of `Node` and the exception classes, they are not provided by the `xml.dom` module itself, but by concrete DOM implementations, such as `xml.dom.minidom`.

Note that DOM attributes may also be manipulated as nodes instead of as simple strings. It is fairly rare that you must do this, however, so this usage is not yet documented.

Interface	Seção	Propósito
`DOMImplementation`	DOMImplementation Objects	Interface to the underlying implementation.
`Node`	Objetos Node	Base interface for most objects in a document.
`NodeList`	Objetos NodeList	Interface for a sequence of nodes.
`DocumentType`	DocumentType Objects	Information about the declarations needed to process a document.
`Document`	Document Objects	Object which represents an entire document.
`Element`	Element Objects	Element nodes in the document hierarchy.

Interface	Seção	Propósito
`Attr`	Attr Objects	Attribute value nodes on element nodes.
`Comment`	Comment Objects	Representation of comments in the source document.
`Text`	Text and CDATASection Objects	Nodes containing textual content from the document.
`ProcessingInstruction`	Objetos ProcessingInstruction	Processing instruction representation.

An additional section describes the exceptions defined for working with the DOM in Python.

DOMImplementation Objects

The `DOMImplementation` interface provides a way for applications to determine the availability of particular features in the DOM they are using. DOM Level 2 added the ability to create new `Document` and `DocumentType` objects using the `DOMImplementation` as well.

DOMImplementation.hasFeature(*feature*, *version*)

Return `True` if the feature identified by the pair of strings *feature* and *version* is implemented.

DOMImplementation.createDocument(*namespaceUri*, *qualifiedName*, *doctype*)

Return a new `Document` object (the root of the DOM), with a child `Element` object having the given *namespaceUri* and *qualifiedName*. The *doctype* must

be a **DocumentType** object created by **createDocumentType()**, or None. In the Python DOM API, the first two arguments can also be None in order to indicate that no **Element** child is to be created.

DOMImplementation.createDocumentType(*qualifiedName*, *publicId*, *systemId*)

Return a new **DocumentType** object that encapsulates the given *qualifiedName*, *publicId*, and *systemId* strings, representing the information contained in an XML document type declaration.

Objetos Node

All of the components of an XML document are subclasses of **Node**.

Only nodes of the following types can have children, and only children of the listed types:

Document

at most one **Element**, at most one **DocumentType**, **ProcessingInstruction** and **Comment**

DocumentFragment and **Element**

Element, **Text**, **CDATASection**, **ProcessingInstruction** and **Comment**

Attr

Text

Nodes of other types cannot have children. Inserting a child of a not allowed type raises **HierarchyRequestErr**.

Node.nodeType

An integer representing the node type. Symbolic constants for the types are on the **Node** object. This is a read-only attribute.

Node.ELEMENT_NODE
Node.ATTRIBUTE_NODE
Node.TEXT_NODE
Node.CDATA_SECTION_NODE
Node.ENTITY_REFERENCE_NODE
Node.ENTITY_NODE
Node.PROCESSING_INSTRUCTION_NODE
Node.COMMENT_NODE

Node.DOCUMENT_NODE
Node.DOCUMENT_TYPE_NODE
Node.DOCUMENT_FRAGMENT_NODE
Node.NOTATION_NODE

Integer constants for the possible values of the `nodeType` attribute.

Node.parentNode

The parent of the current node, or `None` for the document node. The value is always a `Node` object or `None`. For `Element` nodes, this will be the parent element, except for the root element, in which case it will be the `Document` object. For `Attr` nodes, this is always `None`. This is a read-only attribute.

Node.attributes

A `NamedNodeMap` of attribute objects. Only elements have actual values for this; others provide `None` for this attribute. This is a read-only attribute.

Node.previousSibling

The node that immediately precedes this one with the same parent. For instance the element with an end-tag that comes just before the *self* element's start-tag. Of course, XML documents are made up of more than just elements so the previous sibling could be text, a comment, or something else. If this node is the first child of the parent, this attribute will be `None`. This is a read-only attribute.

Node.nextSibling

The node that immediately follows this one with the same parent. See also `previousSibling`. If this is the last child of the parent, this attribute will be `None`. This is a read-only attribute.

Node.childNodes

A `NodeList` of the children of this node. If the node has no children, the list is empty. This is a read-only attribute.

Node.firstChild

The first child of the node, if there are any, or `None`. This is a read-only attribute.

Node.lastChild

The last child of the node, if there are any, or `None`. This is a read-only attribute.

Node.localName

The part of the `tagName` following the colon if there is one, else the entire `tagName`. The value is a string.

Node.prefix

The part of the `tagName` preceding the colon if there is one, else the empty string. The value is a string, or `None`.

Node.namespaceURI

The namespace associated with the element name. This will be a string or `None`. This is a read-only attribute.

Node.ownerDocument

The `Document` object to which this node belongs, or `None` for a document itself. This is a read-only attribute.

Node.isSupported(*feature*, *version*)

Return whether the DOM implementation supports a particular *feature*, as `DOMImplementation.hasFeature()` does.

Node.setUserData(*key*, *data*, *handler*)

Associate *data* with *key* on this node and return the data previously associated with *key*, or `None`. If *data* is `None`, the association is removed. *handler* is called when the node is cloned, imported, renamed or deleted; pass `None` if no notification is needed.

Node.getUserData(*key*)

Return the data associated with *key* on this node by `setUserData()`, or `None`.

Node.nodeName

The name of this node, depending on its type; see the table below. You can always get the information you would get here from another property such as the `tagName` property for elements or the `name` property for attributes. This is a read-only attribute.

Node.nodeValue

The value of this node, depending on its type; see the table below. The value is a string or `None`.

The values of `nodeName` and `nodeValue` for each node type are:

Node type	nodeName	nodeValue
Attr	name	value
CDATASection	'#cdata-section'	the content
Comment	'#comment'	the content
Document	'#document'	None
DocumentFragment	'#document-fragment'	None
DocumentType	name	None
Element	tagName	None
Entity	the name of the entity	None
Notation	the name of the notation	None
ProcessingInstruction	target	data
Text	'#text'	the content

Node.hasAttributes()

Return `True` if the node has any attributes.

Node.hasChildNodes()

Return `True` if the node has any child nodes.

Node.isSameNode(*other*)

Return `True` if *other* refers to the same node as this node. This is especially useful for DOM implementations which use any sort of proxy architecture (because more than one object can refer to the same node).

Nota

This is based on a proposed DOM Level 3 API which is still in the "working draft" stage, but this particular interface appears uncontroversial. Changes from the W3C will not necessarily affect this method in the Python DOM interface (though any new W3C API for this would also be supported).

Node.appendChild(*newChild*)

Add a new child node to this node at the end of the list of children, returning *newChild*. If the node was already in the tree, it is removed first.

Node.insertBefore(*newChild*, *refChild*)

Insert a new child node before an existing child. It must be the case that *refChild* is a child of this node; if not, `NotFoundErr` is raised. *newChild* is returned. If *refChild* is `None`, it inserts *newChild* at the end of the children's list.

Node.removeChild(*oldChild*)

Remove a child node. *oldChild* must be a child of this node; if not, `NotFoundErr` is raised. *oldChild* is returned on success. If *oldChild* will not be used further, its `unlink()` method should be called.

Node.replaceChild(*newChild*, *oldChild*)

Replace an existing node with a new node. It must be the case that *oldChild* is a child of this node; if not, `NotFoundErr` is raised.

Node.normalize()

Join adjacent text nodes so that all stretches of text are stored as single `Text` instances. This simplifies processing text from a DOM tree for many applications.

Node.cloneNode(*deep*)

Clone this node. Setting *deep* means to clone all child nodes as well. This returns the clone.

Objetos NodeList

A `NodeList` represents a sequence of nodes. These objects are used in two ways in the DOM Core recommendation: an `Element` object provides one as its list of child nodes, and the `getElementsByTagName()` and `getElementsByTagNameNS()` methods of `Node` return objects with this interface to represent query results.

`NodeList` does *not* inherit from `Node`.

The DOM Level 2 recommendation defines one method and one attribute for these objects:

NodeList.item(*i*)
: Return the *i*'th item from the sequence, or `None` if *i* is out of range. Negative indices are not supported.

NodeList.length
: The number of nodes in the sequence.

In addition, the Python DOM interface requires that some additional support is provided to allow `NodeList` objects to be used as Python sequences. All **`NodeList`** implementations must include support for `__len__()` and `__getitem__()`; this allows iteration over the **`NodeList`** in `for` statements and proper support for the `len()` built-in function.

If a DOM implementation supports modification of the document, the `NodeList` implementation must also support the `__setitem__()` and `__delitem__()` methods.

DocumentType Objects

Information about the notations and entities declared by a document (including the external subset if the parser uses it and can provide the information) is available from a `DocumentType` object. The **`DocumentType`** for a document is available from the `Document` object's `doctype` attribute; if there is no `DOCTYPE` declaration for the document, the document's **`doctype`** attribute will be set to `None` instead of an instance of this interface.

`DocumentType` is a specialization of `Node`, and adds the following attributes:

DocumentType.publicId

The public identifier for the external subset of the document type definition, or `None` if the `DOCTYPE` declaration does not specify it.

DocumentType.systemId

The system identifier, a URI, for the external subset of the document type definition, or `None` if the `DOCTYPE` declaration does not specify it.

DocumentType.internalSubset

A string giving the complete internal subset from the document. This does not include the brackets which enclose the subset. If the document has no internal subset, this should be `None`.

DocumentType.name

The name of the root element as given in the `DOCTYPE` declaration, if present.

DocumentType.entities

This is a **`NamedNodeMap`** of **`Entity`** nodes giving the definitions of external entities. For entity names defined more than once, only the first definition is provided (others are ignored as required by the XML recommendation). This may be `None` if the information is not provided by the parser, or if no entities are defined.

DocumentType.notations

This is a **`NamedNodeMap`** of **`Notation`** nodes giving the definitions of notations. For notation names defined more than once, only the first definition is provided (others are ignored as required by the XML recommendation). This may be `None` if the information is not provided by the parser, or if no notations are defined.

Document Objects

A **`Document`** represents an entire XML document, including its constituent elements, attributes, processing instructions, comments etc. Remember that it inherits properties from **`Node`**.

Document.documentElement

The one and only root element of the document.

Document.doctype

The **`DocumentType`** node of the document, or `None`. This is a read-only attribute.

Document.implementation

The `DOMImplementation` object which created this document. This is a read-only attribute.

Document.strictErrorChecking

Whether error checking is enforced.

Document.documentURI

The location of the document, or `None` if it is unknown.

Document.createDocumentFragment()

Create and return an empty `DocumentFragment` node.

Document.createCDATASection(*data*)

Create and return a `CDATASection` node containing *data*.

Document.importNode(*importedNode*, *deep*)

Return a copy of *importedNode* which belongs to this document. The original node is not removed from its document. If *deep* is true, the descendants of the node are copied too.

Document.createElement(*tagName*)

Create and return a new element node. The element is not inserted into the document when it is created. You need to explicitly insert it with one of the other methods such as `insertBefore()` or `appendChild()`.

Document.createElementNS(*namespaceURI*, *tagName*)

Create and return a new element with a namespace. The *tagName* may have a prefix. The element is not inserted into the document when it is created. You need to explicitly insert it with one of the other methods such as `insertBefore()` or `appendChild()`.

Document.createTextNode(*data*)

Create and return a text node containing the data passed as a parameter. As with the other creation methods, this one does not insert the node into the tree.

Document.createComment(*data*)

Create and return a comment node containing the data passed as a parameter. As with the other creation methods, this one does not insert the node into the

tree.

Document.createProcessingInstruction(*target*, *data*)

Create and return a processing instruction node containing the *target* and *data* passed as parameters. As with the other creation methods, this one does not insert the node into the tree.

Document.createAttribute(*name*)

Create and return an attribute node. This method does not associate the attribute node with any particular element. You must use **`setAttributeNode()`** on the appropriate **`Element`** object to use the newly created attribute instance.

Document.createAttributeNS(*namespaceURI*, *qualifiedName*)

Create and return an attribute node with a namespace. The *tagName* may have a prefix. This method does not associate the attribute node with any particular element. You must use **`setAttributeNode()`** on the appropriate **`Element`** object to use the newly created attribute instance.

Document.getElementById(*id*)

Return the element with the given ID, or `None`. Only attributes declared as being of type ID in the DTD or by **`Element.setIdAttribute()`** are searched.

Document.getElementsByTagName(*tagName*)

Search for all descendants (direct children, children's children, etc.) with a particular element type name.

Document.getElementsByTagNameNS(*namespaceURI*, *localName*)

Search for all descendants (direct children, children's children, etc.) with a particular namespace URI and localname. The localname is the part of the namespace after the prefix.

Document.renameNode(*n*, *namespaceURI*, *name*)

Rename the element or attribute node *n* and return it. *namespaceURI* is the new namespace URI, or **`EMPTY_NAMESPACE`** if the node does not belong to a namespace. *name* is the new qualified name.

Raise **`WrongDocumentErr`** if *n* was created by another document, and **`NotSupportedErr`** if it is neither an element nor an attribute.

Element Objects

`Element` is a subclass of `Node`, so inherits all the attributes of that class.

Element.tagName

The element type name. In a namespace-using document it may have colons in it. The value is a string.

Element.setIdAttribute(*name*)

Declare that the attribute *name* is of type ID, so that the element is found by `Document.getElementById()`. Raise `NotFoundErr` if the element has no such attribute.

Element.setIdAttributeNS(*namespaceURI*, *localName*)

The same as `setIdAttribute()`, but for an attribute specified by its namespace URI and local name.

Element.setIdAttributeNode(*idAttr*)

The same as `setIdAttribute()`, but for an already retrieved attribute node.

Element.getElementsByTagName(*tagName*)

Same as equivalent method in the `Document` class.

Element.getElementsByTagNameNS(*namespaceURI*, *localName*)

Same as equivalent method in the `Document` class.

Element.hasAttribute(*name*)

Return `True` if the element has an attribute named by *name*.

Element.hasAttributeNS(*namespaceURI*, *localName*)

Return `True` if the element has an attribute named by *namespaceURI* and *localName*.

Element.getAttribute(*name*)

Return the value of the attribute named by *name* as a string. If no such attribute exists, an empty string is returned, as if the attribute had no value.

Element.getAttributeNode(*attrname*)

Return the `Attr` node for the attribute named by *attrname*.

Element.getAttributeNS(*namespaceURI*, *localName*)

Return the value of the attribute named by *namespaceURI* and *localName* as a string. If no such attribute exists, an empty string is returned, as if the attribute had no value.

Element.getAttributeNodeNS(*namespaceURI*, *localName*)

Return an attribute value as a node, given a *namespaceURI* and *localName*.

Element.removeAttribute(*name*)

Remove an attribute by name.

Element.removeAttributeNode(*oldAttr*)

Remove and return *oldAttr* from the attribute list, if present. If *oldAttr* is not present, `NotFoundErr` is raised.

Element.removeAttributeNS(*namespaceURI*, *localName*)

Remove an attribute by name. Note that it uses a localName, not a qname.

Element.setAttribute(*name*, *value*)

Set an attribute value from a string.

Element.setAttributeNode(*newAttr*)

Add a new attribute node to the element, replacing an existing attribute if necessary if the `name` attribute matches. If a replacement occurs, the old attribute node will be returned. If *newAttr* is already in use, `InuseAttributeErr` will be raised.

Element.setAttributeNodeNS(*newAttr*)

Add a new attribute node to the element, replacing an existing attribute if necessary if the `namespaceURI` and `localName` attributes match. If a replacement occurs, the old attribute node will be returned. If *newAttr* is already in use, `InuseAttributeErr` will be raised.

Element.setAttributeNS(*namespaceURI*, *qname*, *value*)

Set an attribute value from a string, given a *namespaceURI* and a *qname*. Note that a qname is the whole attribute name. This is different than above.

Attr Objects

`Attr` inherits from `Node`, so inherits all its attributes.

Attribute nodes are not part of the document tree. They are contained in the `attributes` map of an element, not in its children, and their `parentNode`, `previousSibling` and `nextSibling` are always `None`.

Attr.name
: The attribute name. In a namespace-using document it may include a colon.

Attr.localName
: The part of the name following the colon if there is one, else the entire name. This is a read-only attribute.

Attr.prefix
: The part of the name preceding the colon if there is one, else the empty string.

Attr.isId
: Whether this attribute is of type ID, either because it is declared as such in the DTD or because `Element.setIdAttribute()` was used. This is a read-only attribute.

Attr.ownerElement
: The `Element` node to which this attribute belongs, or `None` if it is not used. This is a read-only attribute.

Attr.specified
: Whether the value of the attribute was explicitly set in the document, as opposed to being defaulted from the DTD. This is a read-only attribute.

Attr.value
: The text value of the attribute. This is a synonym for the `nodeValue` attribute.

NamedNodeMap Objects

`NamedNodeMap` does *not* inherit from `Node`.

NamedNodeMap.length
: The length of the attribute list.

NamedNodeMap.item(*index*)

Return an attribute with a particular index. The order you get the attributes in is arbitrary but will be consistent for the life of a DOM. Each item is an attribute node. Get its value with the `value` attribute.

NamedNodeMap.getNamedItem(*name*)

Return the node with the given `name`, or `None` if there is no such node.

NamedNodeMap.getNamedItemNS(*namespaceURI*, *localName*)

Return the node with the given namespace URI and local name, or `None` if there is no such node.

NamedNodeMap.setNamedItem(*node*)

Add *node* to the map, using its `name` as the key. Return the node which it replaces, or `None` if it replaces no node.

NamedNodeMap.setNamedItemNS(*node*)

Add *node* to the map, using its namespace URI and local name as the key. Return the node which it replaces, or `None` if it replaces no node.

NamedNodeMap.removeNamedItem(*name*)

Remove and return the node with the given `name`. Raise `NotFoundErr` if there is no such node.

NamedNodeMap.removeNamedItemNS(*namespaceURI*, *localName*)

Remove and return the node with the given namespace URI and local name. Raise `NotFoundErr` if there is no such node.

You can also use the standardized **`getAttribute*()`** family of methods on the `Element` objects.

DocumentFragment Objects

`DocumentFragment` is a lightweight container of nodes. It is a subclass of `Node`. When it is inserted into the document tree, its children are inserted instead of it, and it becomes empty.

CharacterData Objects

`CharacterData` represents text-like data in the XML document. It is a subclass of `Node`, and the base class of `Text`, `CDATASection` and `Comment`. Such nodes cannot have child nodes.

CharacterData.data
: The content of the node as a string.

CharacterData.length
: The number of characters in `data`. This is a read-only attribute.

CharacterData.substringData(*offset*, *count*)
: Return the substring of `data` of *count* characters starting at *offset*.

CharacterData.appendData(*arg*)
: Append the string *arg* to `data`.

CharacterData.insertData(*offset*, *arg*)
: Insert the string *arg* into `data` at *offset*.

CharacterData.deleteData(*offset*, *count*)
: Remove *count* characters from `data` starting at *offset*.

CharacterData.replaceData(*offset*, *count*, *arg*)
: Replace *count* characters of `data` starting at *offset* with the string *arg*.

Comment Objects

`Comment` represents a comment in the XML document. It is a subclass of `CharacterData`.

Comment.data
: The content of the comment as a string. The attribute contains all characters between the leading `<!--` and trailing `-->`, but does not include them.

Text and CDATASection Objects

The `Text` interface represents text in the XML document. If the parser and DOM implementation support the DOM's XML extension, portions of the text enclosed

in CDATA marked sections are stored in `CDATASection` objects. These two interfaces are identical, but provide different values for the `nodeType` attribute.

`Text` extends the `CharacterData` interface, and `CDATASection` extends `Text`.

Text.data
: The content of the text node as a string.

Text.wholeText
: The text of all `Text` nodes logically adjacent to this node, concatenated in document order. This is a read-only attribute.

Text.replaceWholeText(*content*)
: Replace the text of all `Text` nodes logically adjacent to this node with *content*, removing the other nodes. Return this node, or `None` if *content* is empty.

Text.splitText(*offset*)
: Split this node into two nodes at *offset*, keeping the first part in this node and returning a new sibling node with the rest.

Nota

The use of a `CDATASection` node does not indicate that the node represents a complete CDATA marked section, only that the content of the node was part of a CDATA section. A single CDATA section may be represented by more than one node in the document tree. There is no way to determine whether two adjacent `CDATASection` nodes represent different CDATA marked sections.

Objetos ProcessingInstruction

Represents a processing instruction in the XML document; this inherits from the `Node` interface and cannot have child nodes.

ProcessingInstruction.target
: The content of the processing instruction up to the first whitespace character. This is a read-only attribute.

ProcessingInstruction.data

The content of the processing instruction following the first whitespace character.

Entity Objects

`Entity` represents a parsed or unparsed entity declared in the DTD. It is a subclass of `Node`. Entity nodes are contained in `DocumentType.entities` and cannot be inserted into the document tree. The name of the entity is its `nodeName`.

Entity.publicId
: The public identifier of the entity, or `None` if it is not specified. This is a read-only attribute.

Entity.systemId
: The system identifier of the entity, or `None` if it is not specified. This is a read-only attribute.

Entity.notationName
: The name of the notation for an unparsed entity, or `None` for a parsed entity. This is a read-only attribute.

Notation Objects

`Notation` represents a notation declared in the DTD. It is a subclass of `Node` and cannot have child nodes. Notation nodes are contained in `DocumentType.notations` and cannot be inserted into the document tree. The name of the notation is its `nodeName`.

Notation.publicId
: The public identifier of the notation, or `None` if it is not specified. This is a read-only attribute.

Notation.systemId
: The system identifier of the notation, or `None` if it is not specified. This is a read-only attribute.

Exceções

The DOM Level 2 recommendation defines a single exception, `DOMException`, and a number of constants that allow applications to determine what sort of error

occurred. **`DOMException`** instances carry a **`code`** attribute that provides the appropriate value for the specific exception.

The Python DOM interface provides the constants, but also expands the set of exceptions so that a specific exception exists for each of the exception codes defined by the DOM. The implementations must raise the appropriate specific exception, each of which carries the appropriate value for the **`code`** attribute.

exception xml.dom.DOMException

Base exception class used for all specific DOM exceptions. This exception class cannot be directly instantiated.

exception xml.dom.DomstringSizeErr

Raised when a specified range of text does not fit into a string. This is not known to be used in the Python DOM implementations, but may be received from DOM implementations not written in Python.

exception xml.dom.HierarchyRequestErr

Raised when an attempt is made to insert a node where the node type is not allowed.

exception xml.dom.IndexSizeErr

Raised when an index or size parameter to a method is negative or exceeds the allowed values.

exception xml.dom.InuseAttributeErr

Raised when an attempt is made to insert an **`Attr`** node that is already present elsewhere in the document.

exception xml.dom.InvalidAccessErr

Raised if a parameter or an operation is not supported on the underlying object.

exception xml.dom.InvalidCharacterErr

This exception is raised when a string parameter contains a character that is not permitted in the context it's being used in by the XML 1.0 recommendation. For example, attempting to create an **`Element`** node with a space in the element type name will cause this error to be raised.

exception xml.dom.InvalidModificationErr

Raised when an attempt is made to modify the type of a node.

exception xml.dom.InvalidStateErr

Raised when an attempt is made to use an object that is not defined or is no longer usable.

exception xml.dom.NamespaceErr

If an attempt is made to change any object in a way that is not permitted with regard to the Namespaces in XML [https://www.w3.org/TR/REC-xml-names/] recommendation, this exception is raised.

exception xml.dom.NotFoundErr

Exception when a node does not exist in the referenced context. For example, `NamedNodeMap.removeNamedItem()` will raise this if the node passed in does not exist in the map.

exception xml.dom.NotSupportedErr

Raised when the implementation does not support the requested type of object or operation.

exception xml.dom.NoDataAllowedErr

This is raised if data is specified for a node which does not support data.

exception xml.dom.NoModificationAllowedErr

Raised on attempts to modify an object where modifications are not allowed (such as for read-only nodes).

exception xml.dom.SyntaxErr

Raised when an invalid or illegal string is specified.

exception xml.dom.ValidationErr

Raised when an operation would make the document invalid with respect to partial validity. This is not known to be used in the Python DOM implementations, but may be received from DOM implementations not written in Python.

exception xml.dom.WrongDocumentErr

Raised when a node is inserted in a different document than it currently belongs to, and the implementation does not support migrating the node from one document to the other.

The exception codes defined in the DOM recommendation map to the exceptions described above according to this table:

Constante	Exception
xml.dom.DOMSTRING_SIZE_ERR	DomstringSizeErr
xml.dom.HIERARCHY_REQUEST_ERR	HierarchyRequestErr
xml.dom.INDEX_SIZE_ERR	IndexSizeErr
xml.dom.INUSE_ATTRIBUTE_ERR	InuseAttributeErr
xml.dom.INVALID_ACCESS_ERR	InvalidAccessErr
xml.dom.INVALID_CHARACTER_ERR	InvalidCharacterErr
xml.dom.INVALID_MODIFICATION_ERR	InvalidModificationErr
xml.dom.INVALID_STATE_ERR	InvalidStateErr
xml.dom.NAMESPACE_ERR	NamespaceErr
xml.dom.NOT_FOUND_ERR	NotFoundErr
xml.dom.NOT_SUPPORTED_ERR	NotSupportedErr
xml.dom.NO_DATA_ALLOWED_ERR	NoDataAllowedErr
xml.dom.NO_MODIFICATION_ALLOWED_ERR	NoModificationAllowedErr

Constante	Exception
xml.dom.SYNTAX_ERR	`SyntaxErr`
xml.dom.VALIDATION_ERR	`ValidationErr`
xml.dom.WRONG_DOCUMENT_ERR	`WrongDocumentErr`

Conformance

This section describes the conformance requirements and relationships between the Python DOM API, the W3C DOM recommendations, and the OMG IDL mapping for Python.

Type Mapping

The IDL types used in the DOM specification are mapped to Python types according to the following table.

Tipo IDL	Tipo em Python
`boolean`	`bool` or `int`
`int`	`int`
`long int`	`int`
`unsigned int`	`int`

Tipo IDL	Tipo em Python
`DOMString`	`str` or `bytes`
`null`	`None`

Accessor Methods

The mapping from OMG IDL to Python defines accessor functions for IDL `attribute` declarations in much the way the Java mapping does. Mapping the IDL declarations

```
readonly attribute string someValue;
         attribute string anotherValue;
```

yields three accessor functions: a “get” method for **`someValue`** (**`_get_someValue()`**), and “get” and “set” methods for **`anotherValue`** (**`_get_anotherValue()`** and **`_set_anotherValue()`**). The mapping, in particular, does not require that the IDL attributes are accessible as normal Python attributes: `object.someValue` is *not* required to work, and may raise an **`AttributeError`**.

The Python DOM API, however, *does* require that normal attribute access work. This means that the typical surrogates generated by Python IDL compilers are not likely to work, and wrapper objects may be needed on the client if the DOM objects are accessed via CORBA. While this does require some additional consideration for CORBA DOM clients, the implementers with experience using DOM over CORBA from Python do not consider this a problem. Attributes that are declared `readonly` may not restrict write access in all DOM implementations.

In the Python DOM API, accessor functions are not required. If provided, they should take the form defined by the Python IDL mapping, but these methods are considered unnecessary since the attributes are accessible directly from Python. “Set” accessors should never be provided for `readonly` attributes.

The IDL definitions do not fully embody the requirements of the W3C DOM API, such as the notion of certain objects, such as the return value of **`getElementsByTagName()`**, being “live”. The Python DOM API does not require implementations to enforce such requirements.

`xml.dom.minidom` — Minimal DOM implementation

Código-fonte: Lib/xml/dom/minidom.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/dom/minidom.py]

`xml.dom.minidom` is a minimal implementation of the Document Object Model interface, with an API similar to that in other languages. It is intended to be simpler than the full DOM and also significantly smaller. Users who are not already proficient with the DOM should consider using the **`xml.etree.ElementTree`** module for their XML processing instead.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte Segurança no XML.

DOM applications typically start by parsing some XML into a DOM. With **`xml.dom.minidom`**, this is done through the parse functions:

```
from xml.dom.minidom import parse, parseString

dom1 = parse('c:\\temp\\mydata.xml')  # parse an XML file by nam

datasource = open('c:\\temp\\mydata.xml')
dom2 = parse(datasource)  # parse an open file

dom3 = parseString('<myxml>Some data<empty/> some more data</myx
```

The **`parse()`** function can take either a filename or an open file object.

xml.dom.minidom.parse(*filename_or_file*, *parser=None*, *bufsize=None*)

Return a **Document** from the given input. *filename_or_file* may be either a file name, or a file-like object. *parser*, if given, must be a SAX2 parser object. This function will change the document handler of the parser and activate namespace support; other parser configuration (like setting an entity resolver) must have been done in advance.

Se você tiver XML em uma string, poderá usar a função **parseString()**:

xml.dom.minidom.parseString(*string*, *parser=None*)

Return a **Document** that represents the *string*. This method creates an **io.StringIO** object for the string and passes that on to **parse()**.

Both functions return a **Document** object representing the content of the document.

What the **parse()** and **parseString()** functions do is connect an XML parser with a "DOM builder" that can accept parse events from any SAX parser and convert them into a DOM tree. The names of the functions are perhaps misleading, but are easy to grasp when learning the interfaces. The parsing of the document will be completed before these functions return; it's simply that these functions do not provide a parser implementation themselves.

You can also create a **Document** by calling a method on a "DOM Implementation" object. You can get this object either by calling the **getDOMImplementation()** function in the **xml.dom** package or the **xml.dom.minidom** module. Once you have a **Document**, you can add child nodes to it to populate the DOM:

```
from xml.dom.minidom import getDOMImplementation

impl = getDOMImplementation()

newdoc = impl.createDocument(None, "some_tag", None)
top_element = newdoc.documentElement
text = newdoc.createTextNode('Some textual content.')
top_element.appendChild(text)
```

Once you have a DOM document object, you can access the parts of your XML document through its properties and methods. These properties are defined in the DOM specification. The main property of the document object is the **`documentElement`** property. It gives you the main element in the XML document: the one that holds all others. Here is an example program:

```
dom3 = parseString("<myxml>Some data</myxml>")
assert dom3.documentElement.tagName == "myxml"
```

When you are finished with a DOM tree, you may optionally call the **`unlink()`** method to encourage early cleanup of the now-unneeded objects. **`unlink()`** is an **`xml.dom.minidom`**-specific extension to the DOM API that renders the node and its descendants essentially useless. Otherwise, Python's garbage collector will eventually take care of the objects in the tree.

Ver também

Document Object Model (DOM) Level 1 Specification
[https://www.w3.org/TR/REC-DOM-Level-1/]

The W3C recommendation for the DOM supported by **`xml.dom.minidom`**.

Objetos DOM

The definition of the DOM API for Python is given as part of the **`xml.dom`** module documentation. This section lists the differences between the API and **`xml.dom.minidom`**.

Node.unlink()

Break internal references within the DOM so that it will be garbage collected on versions of Python without cyclic GC. Even when cyclic GC is available, using this can make large amounts of memory available sooner, so calling this on DOM objects as soon as they are no longer needed is good practice. This only needs to be called on the **`Document`**

object, but may be called on child nodes to discard children of that node.

You can avoid calling this method explicitly by using the **`with`** statement. The following code will automatically unlink *dom* when the **`with`** block is exited:

```
with xml.dom.minidom.parse(datasource) as dom:
    ... # Work with dom.
```

Node.writexml(*writer*, *indent=''*, *addindent=''*, *newl=''*, *encoding=None*, *standalone=None*)

Write XML to the writer object. The writer receives texts but not bytes as input, it should have a **`write()`** method which matches that of the file object interface. The *indent* parameter is the indentation of the current node. The *addindent* parameter is the incremental indentation to use for subnodes of the current one. The *newl* parameter specifies the string to use to terminate newlines.

For the **`Document`** node, an additional keyword argument *encoding* can be used to specify the encoding field of the XML header.

Similarly, explicitly stating the *standalone* argument causes the standalone document declarations to be added to the prologue of the XML document. If the value is set to `True`, `standalone="yes"` is added, otherwise it is set to `"no"`. Not stating the argument will omit the declaration from the document.

Alterado na versão 3.8: The **`writexml()`** method now preserves the attribute order specified by the user.

Alterado na versão 3.9: The *standalone* parameter was added.

Node.toxml(*encoding=None*, *standalone=None*)

Return a string or byte string containing the XML represented by the DOM node.

With an explicit *encoding* [[1]] argument, the result is a byte string in the specified encoding. With no *encoding* argument, the result is a Unicode string, and the XML declaration in the resulting string does not specify an encoding. Encoding this string in an encoding other than UTF-8 is likely incorrect, since UTF-8 is the default encoding of XML.

The *standalone* argument behaves exactly as in **writexml()**.

Alterado na versão 3.8: The **toxml()** method now preserves the attribute order specified by the user.

Alterado na versão 3.9: The *standalone* parameter was added.

Node.toprettyxml(*indent='\t', newl='\n', encoding=None, standalone=None*)

Return a pretty-printed version of the document. *indent* specifies the indentation string and defaults to a tabulator; *newl* specifies the string emitted at the end of each line and defaults to `\n`.

The *encoding* argument behaves like the corresponding argument of **toxml()**.

The *standalone* argument behaves exactly as in **writexml()**.

Alterado na versão 3.8: The **toprettyxml()** method now preserves the attribute order specified by the user.

Alterado na versão 3.9: The *standalone* parameter was added.

DOM Example

This example program is a fairly realistic example of a simple program. In this particular case, we do not take much advantage of the flexibility of the DOM.

```
import xml.dom.minidom

document = """\
```

```
<slideshow>
<title>Demo slideshow</title>
<slide><title>Slide title</title>
<point>This is a demo</point>
<point>Of a program for processing slides</point>
</slide>

<slide><title>Another demo slide</title>
<point>It is important</point>
<point>To have more than</point>
<point>one slide</point>
</slide>
</slideshow>
"""

dom = xml.dom.minidom.parseString(document)

def getText(nodelist):
    rc = []
    for node in nodelist:
        if node.nodeType == node.TEXT_NODE:
            rc.append(node.data)
    return ''.join(rc)

def handleSlideshow(slideshow):
    print("<html>")
    handleSlideshowTitle(slideshow.getElementsByTagName("title")
    slides = slideshow.getElementsByTagName("slide")
    handleToc(slides)
    handleSlides(slides)
    print("</html>")

def handleSlides(slides):
    for slide in slides:
        handleSlide(slide)

def handleSlide(slide):
    handleSlideTitle(slide.getElementsByTagName("title")[0])
    handlePoints(slide.getElementsByTagName("point"))

def handleSlideshowTitle(title):
    print(f"<title>{getText(title.childNodes)}</title>")
```

```
def handleSlideTitle(title):
    print(f"<h2>{getText(title.childNodes)}</h2>")

def handlePoints(points):
    print("<ul>")
    for point in points:
        handlePoint(point)
    print("</ul>")

def handlePoint(point):
    print(f"<li>{getText(point.childNodes)}</li>")

def handleToc(slides):
    for slide in slides:
        title = slide.getElementsByTagName("title")[0]
        print(f"<p>{getText(title.childNodes)}</p>")

handleSlideshow(dom)
```

minidom e o padrão DOM

The **`xml.dom.minidom`** module is essentially a DOM 1.0-compatible DOM with some DOM 2 features (primarily namespace features).

Usage of the DOM interface in Python is straight-forward. The following mapping rules apply:

- Interfaces are accessed through instance objects. Applications should not instantiate the classes themselves; they should use the creator functions available on the **`Document`** object. Derived interfaces support all operations (and attributes) from the base interfaces, plus any new operations.

- Operations are used as methods. Since the DOM uses only **`in`** parameters, the arguments are passed in normal order (from left to right). There are no optional arguments. `void` operations return `None`.

- IDL attributes map to instance attributes. For compatibility with the OMG IDL language mapping for Python, an attribute `foo` can also be

accessed through accessor methods **`_get_foo()`** and **`_set_foo()`**. `readonly` attributes must not be changed; this is not enforced at runtime.

- The types `short int`, `unsigned int`, `unsigned long long`, and `boolean` all map to Python integer objects.

- The type `DOMString` maps to Python strings. **`xml.dom.minidom`** supports either bytes or strings, but will normally produce strings. Values of type `DOMString` may also be `None` where allowed to have the IDL `null` value by the DOM specification from the W3C.

- `const` declarations map to variables in their respective scope (e.g. `xml.dom.minidom.Node.PROCESSING_INSTRUCTION_NODE`); they must not be changed.

- `DOMException` is currently not supported in **`xml.dom.minidom`**. Instead, **`xml.dom.minidom`** uses standard Python exceptions such as **`TypeError`** and **`AttributeError`**.

- Each of the **`NodeList`** and **`NamedNodeMap`** interfaces has two implementations, which provide additional methods and operations.

  `childNodes` is a subclass of **`list`**, or, for nodes which cannot have children, a subclass of **`tuple`**. It supports iteration, concatenation, indexing and slicing.

  `attributes` supports `len()`, the **`in`** operator, subscription by a name or by a `(namespaceURI, localName)` tuple, assignment and deletion, and the methods **`get()`**, **`keys()`**, **`keysNS()`**, **`values()`**, **`items()`** and **`itemsNS()`**. **`entities`** and **`notations`** are read-only and support only `len()` and subscription by a name.

- **`strictErrorChecking`** and **`specified`** are always `False`.

- **`removeAttribute()`** and **`removeAttributeNS()`** raise **`NotFoundErr`** if there is no matching attribute, while the DOM specifies that this has no effect.

The following interfaces have no implementation in **`xml.dom.minidom`**:

- **`DOMTimeStamp`**
- **`EntityReference`**

Most of these reflect information in the XML document that is not of general utility to most DOM users.

Notas de rodapé

[1]

The encoding name included in the XML output should conform to the appropriate standards. For example, “UTF-8” is valid, but “UTF8” is not valid in an XML document’s declaration, even though Python accepts it as an encoding name. See https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl and https://www.iana.org/assignments/character-sets/character-sets.xhtml.

`xml.dom.pulldom` — Suporte para construir árvores parciais de DOM

Código-fonte: Lib/xml/dom/pulldom.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/dom/pulldom.py]

O módulo **`xml.dom.pulldom`** fornece um “analisador sintático de pull” que também pode ser solicitado a produzir fragmentos do documento acessíveis via DOM, quando necessário. O conceito básico envolve extrair “eventos” de um fluxo de XML recebido e processá-los. Ao contrário do SAX, que também emprega um modelo de processamento orientado a eventos juntamente com retornos de chamada, o usuário de um analisador sintático pull é responsável por extrair explicitamente os eventos do fluxo, repetindo esses eventos até que o processamento seja concluído ou ocorra uma condição de erro.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte Segurança no XML.

Alterado na versão 3.7.1: O analisador sintático de SAX não processa mais entidades externas gerais por padrão para aumentar a segurança. Para habilitar o processamento de entidades externas, passe uma instância personalizada do analisador sintático em:

```
from xml.dom.pulldom import parse
from xml.sax import make_parser
from xml.sax.handler import feature_external_ges

parser = make_parser()
parser.setFeature(feature_external_ges, True)
parse(filename, parser=parser)
```

Exemplo:

```
from xml.dom import pulldom

doc = pulldom.parse('sales_items.xml')
for event, node in doc:
    if event == pulldom.START_ELEMENT and node.tagName == 'item'
        if int(node.getAttribute('price')) > 50:
            doc.expandNode(node)
            print(node.toxml())
```

`event` é uma das seguintes constantes, e `node` é o nó ao qual o evento se refere. Os nós implementam as interfaces `xml.dom`; eles são criados pela implementação do DOM fornecida a `PullDOM`, que é `xml.dom.minidom` por padrão.

xml.dom.pulldom.START_DOCUMENT
xml.dom.pulldom.END_DOCUMENT

O início e o fim do documento. *node* é o `Document`.

xml.dom.pulldom.START_ELEMENT
xml.dom.pulldom.END_ELEMENT

A tag de início e a tag de fim de um elemento. *node* é o `Element`.

xml.dom.pulldom.CHARACTERS

Dados de caractere. *node* é o nó `Text`.

xml.dom.pulldom.IGNORABLE_WHITESPACE

Espaço em branco no conteúdo do elemento, conforme declarado na DTD. *node* é o nó `Text`.

xml.dom.pulldom.COMMENT

Um comentário. *node* é o nó `Comment`.

xml.dom.pulldom.PROCESSING_INSTRUCTION

Uma instrução de processamento. *node* é o nó `ProcessingInstruction`.

Como o documento é tratado como um fluxo “plano” de eventos, a “árvore” do documento é implicitamente percorrida e os elementos desejados são encontrados independentemente de sua profundidade na árvore. Em outras palavras, não é necessário considerar questões hierárquicas, como a busca recursiva dos nós do documento, embora, se o contexto dos elementos fosse importante, seria necessário manter algum estado relacionado ao contexto (ou seja, lembrar onde se está no documento em um determinado ponto) ou utilizar o método `DOMEventStream.expandNode()` e alternar para o processamento relacionado ao DOM.

class xml.dom.pulldom.PullDOM(*documentFactory=None*)

Subclasse de `xml.sax.handler.ContentHandler` que converte eventos SAX em eventos do *pull parser*. Os nós são criados, mas não são adicionados à árvore, a menos que `expandNode()` seja chamado. *documentFactory*, se fornecido, é uma implementação DOM usada para criar o documento; por padrão, utiliza-se a implementação de `xml.dom.minidom`.

class xml.dom.pulldom.SAX2DOM(*documentFactory=None*)

Subclasse de `PullDOM` que também adiciona cada nó criado à árvore, de modo que o documento completo seja construído.

xml.dom.pulldom.parse(*stream_or_string*, *parser=None*, *bufsize=None*)

Retorna um `DOMEventStream` da entrada fornecida. *stream_or_string* pode ser um nome de arquivo ou um objeto arquivo ou similar. *parser*, se fornecido, deve ser um objeto `XMLReader`. Esta função alterará o manipulador de documentos do analisador e ativará o suporte a espaços de nomes; outras configurações do analisador sintático (como a definição de um resolvedor de entidades) devem ter sido feitas previamente.

Se você tiver XML em uma string, poderá usar a função `parseString()`:

xml.dom.pulldom.parseString(*string*, *parser=None*)

Retorna um `DOMEventStream` que representa a *string*. A *string* deve ser uma instância de `str`; para analisar bytes, passe um objeto arquivo binário para `parse()`.

xml.dom.pulldom.default_bufsize

Valor padrão para o parâmetro *bufsize* para `parse()`.

O valor desta variável pode ser alterado antes de chamar `parse()` e o novo valor entrará em vigor.

Objetos DOMEventStream

class xml.dom.pulldom.DOMEventStream(*stream*, *parser*, *bufsize*)

Produz os eventos para os dados lidos do objeto arquivo *stream* pelo *parser* `XMLReader`. Os dados são lidos em blocos de *bufsize* bytes — ou caracteres, no caso de um fluxo de texto — por vez.

Alterado na versão 3.11: O suporte para o método `__getitem__()` foi removido

getEvent()

Retorna a próxima tupla `(event, node)` ou `None` ao final do documento. Consulte acima as informações sobre os eventos e os nós correspondentes. O nó atual não contém informações sobre seus filhos, a menos que `expandNode()` seja chamado.

expandNode(*node*)

Expande todos os filhos de *node* para *node*. Exemplo:

```
from xml.dom import pulldom

xml = '<html><title>Foo</title> <p>Algum texto <div>e mai
doc = pulldom.parseString(xml)
for event, node in doc:
    if event == pulldom.START_ELEMENT and node.tagName ==
        # A seguinte instrução só exibe '<p/>'
        print(node.toxml())
        doc.expandNode(node)
        # A seguinte instrução exibe o nó com todos os se
        print(node.toxml())
```

reset()

Descarta os eventos que ainda não foram lidos e prepare o objeto para análise de um novo documento.

clear()

Libera o analisador e o documento. O fluxo não é fechado, e o objeto não pode mais ser utilizado.

`xml.sax` — Support for SAX2 parsers

Código-fonte: Lib/xml/sax/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/sax/__init__.py]

The `xml.sax` package provides a number of modules which implement the Simple API for XML (SAX) interface for Python. The package itself provides the SAX exceptions and the convenience functions which will be most used by users of the SAX API.

> **Nota**
>
> Se você precisar analisar dados não confiáveis ou não autenticados, consulte Segurança no XML.

Alterado na versão 3.7.1: The SAX parser no longer processes general external entities by default to increase security. Before, the parser created network connections to fetch remote files or loaded local files from the file system for DTD and entities. The feature can be enabled again with method `setFeature()` on the parser object and argument `feature_external_ges`.

The convenience functions and data are:

xml.sax.make_parser(*parser_list=()*)

Create and return a SAX `XMLReader` object. The first parser found will be used. If *parser_list* is provided, it must be an iterable of strings which name modules that have a function named `create_parser()`. Modules listed in *parser_list* will be used before modules in the default list of parsers.

Alterado na versão 3.8: The *parser_list* argument can be any iterable, not just a list.

xml.sax.parse(*filename_or_stream*, *handler*, *errorHandler=handler.ErrorHandler()*)

Create a SAX parser and use it to parse a document. The document, passed in as *filename_or_stream*, can be a system identifier (a string identifying the input source – typically a file name or a URL), a path-like object, or a file object. A system identifier which does not refer to an existing file is opened with `urllib.request.urlopen()`. The *handler* parameter needs to be a SAX `ContentHandler` instance. If *errorHandler* is given, it must be a SAX `ErrorHandler` instance; if omitted, `SAXParseException` will be raised on all errors. There is no return value; all work must be done by the *handler* passed in.

xml.sax.parseString(*string*, *handler*, *errorHandler=handler.ErrorHandler()*)

Similar to `parse()`, but parses from a buffer *string* received as a parameter. *string* must be a `str` instance or a bytes-like object.

Alterado na versão 3.5: Added support of `str` instances.

xml.sax.default_parser_list

The list of the names of modules which are tried by `make_parser()` after the modules named in its *parser_list* argument. It contains `'xml.sax.expatreader'`, or, if the **PY_SAX_PARSER** environment variable is set and the environment is not ignored, the comma-separated list of module names taken from it.

A typical SAX application uses three kinds of objects: readers, handlers and input sources. “Reader” in this context is another term for parser, i.e. some piece of code that reads the bytes or characters from the input source, and produces a sequence of events. The events then get distributed to the handler objects, i.e. the reader invokes a method on the handler. A SAX application must therefore obtain a reader object, create or open the input sources, create the handlers, and connect these objects all together. As the final step of preparation, the reader is called to parse the input. During

parsing, methods on the handler objects are called based on structural and syntactic events from the input data.

For these objects, only the interfaces are relevant; they are normally not instantiated by the application itself. Since Python does not have an explicit notion of interface, they are formally introduced as classes, but applications may use implementations which do not inherit from the provided classes. The **InputSource**, **Locator**, **Attributes**, **AttributesNS**, and **XMLReader** interfaces are defined in the module **xml.sax.xmlreader**. The handler interfaces are defined in **xml.sax.handler**. For convenience, **InputSource** (which is often instantiated directly) and the handler classes are also available from **xml.sax**. These interfaces are described below.

In addition to these classes, **xml.sax** provides the following exception classes.

exception xml.sax.SAXException(*msg*, *exception=None*)

Encapsulate an XML error or warning. This class can contain basic error or warning information from either the XML parser or the application: it can be subclassed to provide additional functionality or to add localization. Note that although the handlers defined in the **ErrorHandler** interface receive instances of this exception, it is not required to actually raise the exception — it is also useful as a container for information.

When instantiated, *msg* should be a human-readable description of the error. The optional *exception* parameter, if given, should be `None` or an exception that was caught by the parsing code and is being passed along as information.

This is the base class for the other SAX exception classes.

exception xml.sax.SAXParseException(*msg*, *exception*, *locator*)

Subclass of **SAXException** raised on parse errors. Instances of this class are passed to the methods of the SAX **ErrorHandler** interface to provide information about the parse error. This class supports the SAX **Locator** interface as well as the **SAXException** interface.

exception xml.sax.SAXNotRecognizedException(*msg*, *exception=None*)

Subclass of `SAXException` raised when a SAX `XMLReader` is confronted with an unrecognized feature or property. SAX applications and extensions may use this class for similar purposes.

exception xml.sax.SAXNotSupportedException(*msg*, *exception=None*)

Subclass of `SAXException` raised when a SAX `XMLReader` is asked to enable a feature that is not supported, or to set a property to a value that the implementation does not support. SAX applications and extensions may use this class for similar purposes.

exception xml.sax.SAXReaderNotAvailable(*msg*, *exception=None*)

Subclass of `SAXNotSupportedException` raised when no parser is available. A parser module raises it when it is imported or during parsing if the parser it provides cannot be used, and `make_parser()` raises it if no module from the tried ones provides a usable parser.

Ver também

SAX: The Simple API for XML **[http://www.saxproject.org/]**

This site is the focal point for the definition of the SAX API. It provides a Java implementation and online documentation. Links to implementations and historical information are also available.

Module `xml.sax.handler`

Definitions of the interfaces for application-provided objects.

Module `xml.sax.saxutils`

Convenience functions for use in SAX applications.

Module `xml.sax.xmlreader`

Definitions of the interfaces for parser-provided objects.

SAXException Objects

The `SAXException` exception class supports the following methods:

SAXException.getMessage()

Return a human-readable message describing the error condition.

SAXException.getException()

Return an encapsulated exception object, or `None`.

`xml.sax.handler` — Classes base para manipuladores de SAX

Código-fonte: Lib/xml/sax/handler.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/sax/handler.py]

The SAX API defines five kinds of handlers: content handlers, DTD handlers, error handlers, entity resolvers and lexical handlers. Applications normally only need to implement those interfaces whose events they are interested in; they can implement the interfaces in a single object or in multiple objects. Handler implementations should inherit from the base classes provided in the module **`xml.sax.handler`**, so that all methods get default implementations.

class xml.sax.handler.ContentHandler

This is the main callback interface in SAX, and the one most important to applications. The order of events in this interface mirrors the order of the information in the document.

class xml.sax.handler.DTDHandler

Manipular eventos DTD.

This interface specifies only those DTD events required for basic parsing (unparsed entities and attributes).

class xml.sax.handler.EntityResolver

Basic interface for resolving entities. If you create an object implementing this interface, then register the object with your Parser, the parser will call the method in your object to resolve all external entities.

class xml.sax.handler.ErrorHandler

Interface used by the parser to present error and warning messages to the application. The methods of this object control whether errors are immediately converted to exceptions or are handled in some other way.

class xml.sax.handler.LexicalHandler

Interface used by the parser to represent low frequency events which may not be of interest to many applications.

In addition to these classes, **`xml.sax.handler`** provides symbolic constants for the feature and property names.

xml.sax.handler.feature_namespaces

valor: `"http://xml.org/sax/features/namespaces"`
true: Executa o processamento do espaço de nomes.
false: Optionally do not perform Namespace processing (implies namespace-prefixes; default).
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_namespace_prefixes

valor: `"http://xml.org/sax/features/namespace-prefixes"`
true: Report the original prefixed names and attributes used for Namespace declarations.
false: Do not report attributes used for Namespace declarations, and optionally do not report original prefixed names (default).
access: (parsing) read-only; (not parsing) read/write

The parser based on **`xml.parsers.expat`** does not support this feature.

xml.sax.handler.feature_string_interning

valor: `"http://xml.org/sax/features/string-interning"`
true: All element names, prefixes, attribute names, Namespace URIs, and local names are interned in a dictionary (see **`property_interning_dict`**).
false: Names are not necessarily interned, although they may be (default).

access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_validation

valor: `"http://xml.org/sax/features/validation"`
true: Report all validation errors (implies external-general-entities and external-parameter-entities).
false: Do not report validation errors.
access: (parsing) read-only; (not parsing) read/write

The parser based on **`xml.parsers.expat`** does not support this feature, because Expat is a non-validating parser.

xml.sax.handler.feature_external_ges

Aviso

Enabling opens a vulnerability to external entity attacks [https://en.wikipedia.org/wiki/XML_external_entity_attack] if the parser is used with user-provided XML content. Please reflect on your threat model [https://en.wikipedia.org/wiki/Threat_model] before enabling this feature.

valor: `"http://xml.org/sax/features/external-general-entities"`
true: Include all external general (text) entities.
false: Do not include external general entities.
access: (parsing) read-only; (not parsing) read/write

xml.sax.handler.feature_external_pes

valor: `"http://xml.org/sax/features/external-parameter-entities"`
true: Include all external parameter entities, including the external DTD subset.
false: Do not include any external parameter entities, even the external DTD subset.
access: (parsing) read-only; (not parsing) read/write

The parser based on **`xml.parsers.expat`** does not support this feature.

xml.sax.handler.all_features

Lista de todos os recursos.

xml.sax.handler.property_lexical_handler

valor: `"http://xml.org/sax/properties/lexical-handler"`
data type: **`LexicalHandler`**
descrição: Um tratador de extensão opcional para eventos lexicais como comentários.
acesso: leitura/gravação

xml.sax.handler.property_declaration_handler

valor: `"http://xml.org/sax/properties/declaration-handler"`
data type: an object implementing the SAX2 `DeclHandler` interface
description: An optional extension handler for DTD-related events other than notations and unparsed entities.
acesso: leitura/gravação

No parser in the standard library supports this property, and the standard library provides no such handler.

xml.sax.handler.property_dom_node

valor: `"http://xml.org/sax/properties/dom-node"`
data type: **`xml.dom.Node`**
description: When parsing, the current DOM node being visited if this is a DOM iterator; when not parsing, the root DOM node for iteration.
access: (parsing) read-only; (not parsing) read/write

No parser in the standard library supports this property.

xml.sax.handler.property_xml_string

valor: `"http://xml.org/sax/properties/xml-string"`
data type: Bytes
description: The literal string of characters that was the source for the current event.

access: read-only, and only during a handler callback

xml.sax.handler.property_encoding

value: `"http://www.python.org/sax/properties/encoding"`
data type: String
description: The name of the encoding to assume for input data.
acesso: leitura/gravação

No parser in the standard library supports this property.

xml.sax.handler.property_interning_dict

value: `"http://www.python.org/sax/properties/interning-dict"`
data type: Dictionary
description: The dictionary used to intern names, or `None` if names are not interned. Setting it enables interning, as does the **`feature_string_interning`** feature.
acesso: leitura/gravação

xml.sax.handler.all_properties
List of all known property names.

ContentHandler Objects

Users are expected to subclass **`ContentHandler`** to support their application. The following methods are called by the parser on the appropriate events in the input document:

ContentHandler.setDocumentLocator(*locator*)
Called by the parser to give the application a locator for locating the origin of document events.

SAX parsers are strongly encouraged (though not absolutely required) to supply a locator: if it does so, it must supply the locator to the application by invoking this method before invoking any of the other methods in the DocumentHandler interface.

The locator allows the application to determine the end position of any document-related event, even if the parser is not reporting an error. Typically, the application will use this information for reporting its own errors (such as character content that does not match an application's business rules). The information returned by the locator is probably not sufficient for use with a search engine.

Note that the locator will return correct information only during the invocation of the events in this interface. The application should not attempt to use it at any other time.

ContentHandler.startDocument()

Receive notification of the beginning of a document.

The SAX parser will invoke this method only once, before any other methods in this interface or in DTDHandler (except for **`setDocumentLocator()`**).

ContentHandler.endDocument()

Receive notification of the end of a document.

The SAX parser will invoke this method only once, and it will be the last method invoked during the parse. The parser shall not invoke this method until it has either abandoned parsing (because of an unrecoverable error) or reached the end of input.

ContentHandler.startPrefixMapping(*prefix*, *uri*)

Begin the scope of a prefix-URI Namespace mapping.

The information from this event is not necessary for normal Namespace processing: the SAX XML reader will automatically replace prefixes for element and attribute names when the `feature_namespaces` feature is enabled (the default).

There are cases, however, when applications need to use prefixes in character data or in attribute values, where they cannot safely be expanded automatically; the **`startPrefixMapping()`** and

`endPrefixMapping()` events supply the information to the application to expand prefixes in those contexts itself, if necessary.

Note that **`startPrefixMapping()`** and `endPrefixMapping()` events are not guaranteed to be properly nested relative to each-other: all **`startPrefixMapping()`** events will occur before the corresponding `startElement()` event, and all **`endPrefixMapping()`** events will occur after the corresponding `endElement()` event, but their order is not guaranteed.

ContentHandler.endPrefixMapping(*prefix*)

End the scope of a prefix-URI mapping.

See `startPrefixMapping()` for details. This event will always occur after the corresponding `endElement()` event, but the order of **`endPrefixMapping()`** events is not otherwise guaranteed.

ContentHandler.startElement(*name*, *attrs*)

Signals the start of an element in non-namespace mode.

The *name* parameter contains the raw XML 1.0 name of the element type as a string and the *attrs* parameter holds an object of the Attributes interface containing the attributes of the element. The object passed as *attrs* may be re-used by the parser; holding on to a reference to it is not a reliable way to keep a copy of the attributes. To keep a copy of the attributes, use the `copy()` method of the *attrs* object.

ContentHandler.endElement(*name*)

Signals the end of an element in non-namespace mode.

The *name* parameter contains the name of the element type, just as with the `startElement()` event.

ContentHandler.startElementNS(*name*, *qname*, *attrs*)

Signals the start of an element in namespace mode.

The *name* parameter contains the name of the element type as a `(uri, localname)` tuple, the *qname* parameter contains the raw XML 1.0 name used in the source document, and the *attrs* parameter holds an instance of the AttributesNS interface containing the attributes of the element. If no namespace is associated with the element, the *uri* component of *name* will be `None`. The object passed as *attrs* may be re-used by the parser; holding on to a reference to it is not a reliable way to keep a copy of the attributes. To keep a copy of the attributes, use the **`copy()`** method of the *attrs* object.

Parsers may set the *qname* parameter to `None`, unless the `feature_namespace_prefixes` feature is activated.

ContentHandler.endElementNS(*name*, *qname*)

Signals the end of an element in namespace mode.

O parâmetro *name* contém o nome do tipo de elemento, assim como no método **`startElementNS()`**, da mesma forma o parâmetro *qname*.

ContentHandler.characters(*content*)

Receive notification of character data.

The Parser will call this method to report each chunk of character data. SAX parsers may return all contiguous character data in a single chunk, or they may split it into several chunks; however, all of the characters in any single event must come from the same external entity so that the Locator provides useful information.

content may be a string or bytes instance; the `expat` reader module always produces strings.

Nota

The earlier SAX 1 interface provided by the Python XML Special Interest Group used a more Java-like interface for this method. Since most parsers used from Python did not take advantage of the older interface, the simpler signature was chosen to replace it. To convert

old code to the new interface, use *content* instead of slicing content with the old *offset* and *length* parameters.

ContentHandler.ignorableWhitespace(*whitespace*)

Receive notification of ignorable whitespace in element content.

Validating Parsers must use this method to report each chunk of ignorable whitespace (see the W3C XML 1.0 recommendation, section 2.10): non-validating parsers may also use this method if they are capable of parsing and using content models.

SAX parsers may return all contiguous whitespace in a single chunk, or they may split it into several chunks; however, all of the characters in any single event must come from the same external entity, so that the Locator provides useful information.

ContentHandler.processingInstruction(*target*, *data*)

Receive notification of a processing instruction.

The Parser will invoke this method once for each processing instruction found: note that processing instructions may occur before or after the main document element.

A SAX parser should never report an XML declaration (XML 1.0, section 2.8) or a text declaration (XML 1.0, section 4.3.1) using this method.

ContentHandler.skippedEntity(*name*)

Receive notification of a skipped entity.

The Parser will invoke this method once for each entity skipped. Non-validating processors may skip entities if they have not seen the declarations (because, for example, the entity was declared in an external DTD subset). All processors may skip external entities, depending on the values of the `feature_external_ges` and the `feature_external_pes` properties.

DTDHandler Objects

As instâncias `DTDHandler` fornecem os seguintes métodos:

DTDHandler.notationDecl(*name*, *publicId*, *systemId*)

Handle a notation declaration event.

DTDHandler.unparsedEntityDecl(*name*, *publicId*, *systemId*, *ndata*)

Handle an unparsed entity declaration event.

EntityResolver Objects

EntityResolver.resolveEntity(*publicId*, *systemId*)

Resolve the system identifier of an entity and return either the system identifier to read from as a string, or an InputSource to read from. The default implementation returns *systemId*.

ErrorHandler Objects

Objects with this interface are used to receive error and warning information from the `XMLReader`. If you create an object that implements this interface, then register the object with your `XMLReader`, the parser will call the methods in your object to report all warnings and errors. There are three levels of errors available: warnings, (possibly) recoverable errors, and unrecoverable errors. All methods take a `SAXParseException` as the only parameter. Errors and warnings may be converted to an exception by raising the passed-in exception object.

ErrorHandler.error(*exception*)

Called when the parser encounters a recoverable error. If this method does not raise an exception, parsing may continue, but further document information should not be expected by the application. Allowing the parser to continue may allow additional errors to be discovered in the input document.

ErrorHandler.fatalError(*exception*)

Called when the parser encounters an error it cannot recover from; parsing is expected to terminate when this method returns.

ErrorHandler.warning(*exception*)

Called when the parser presents minor warning information to the application. Parsing is expected to continue when this method returns, and document information will continue to be passed to the application. Raising an exception in this method will cause parsing to end.

LexicalHandler Objects

Optional SAX2 handler for lexical events.

This handler is used to obtain lexical information about an XML document. Lexical information includes information describing the document encoding used and XML comments embedded in the document, as well as section boundaries for the DTD and for any CDATA sections. The lexical handlers are used in the same manner as content handlers.

Set the LexicalHandler of an XMLReader by using the setProperty method with the property identifier `'http://xml.org/sax/properties/lexical-handler'`.

LexicalHandler.comment(*content*)

Reports a comment anywhere in the document (including the DTD and outside the document element).

LexicalHandler.startDTD(*name*, *public_id*, *system_id*)

Reports the start of the DTD declarations if the document has an associated DTD.

LexicalHandler.endDTD()

Reports the end of DTD declaration.

LexicalHandler.startCDATA()

Reports the start of a CDATA marked section.

The contents of the CDATA marked section will be reported through the characters handler.

LexicalHandler.endCDATA()

Reports the end of a CDATA marked section.

`xml.sax.saxutils` — Utilitários de SAX

Código-fonte: Lib/xml/sax/saxutils.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/sax/saxutils.py]

O módulo **`xml.sax.saxutils`** contém uma série de classes e funções que são comumente úteis ao criar aplicações SAX, seja em uso direto ou como classes base.

xml.sax.saxutils.escape(*data*, *entities={}*)

Escapa `'&'`, `'<'` e `'>'` em uma string de dados.

Você pode escapar outras strings de dados passando um dicionário como o parâmetro opcional *entities*. As chaves e os valores devem ser strings; cada chave será substituída pelo seu valor correspondente. Os caracteres `'&'`, `'<'` e `'>'` são sempre escapados, mesmo que *entities* seja fornecido.

Nota

Esta função deve ser usada apenas para escapar caracteres que não podem ser usados diretamente em XML. Não a utilize como uma função geral de tradução de strings.

xml.sax.saxutils.unescape(*data*, *entities={}*)

Remove escape de `'&'`, `'<'` e `'>'` em uma string de dados.

Você pode remover escape de outras strings de dados passando um dicionário como o parâmetro opcional *entities*. As chaves e os valores devem ser strings; cada chave será substituída pelo seu valor

correspondente. `'&'`, `'<'` e `'>'` têm sempre o escape removido, mesmo que *entities* seja fornecido.

xml.sax.saxutils.quoteattr(*data*, *entities={}*)

Semelhante a **escape()**, mas também prepara *data* para ser usado como um valor de atributo. O valor de retorno é uma versão entre aspas de *data* com quaisquer substituições adicionais necessárias. **quoteattr()** selecionará um caractere de aspas com base no conteúdo de *data*, tentando evitar a codificação de quaisquer caracteres de aspas na string. Se já houver caracteres de aspas simples e duplas em *data*, os caracteres de aspas duplas serão codificados e *data* será colocado entre aspas duplas. A string resultante pode ser usada diretamente como um valor de atributo:

```
>>> print("<element attr=%s>" % quoteattr("ab ' cd \" ef"))
<element attr="ab ' cd &quot; ef">
```

Esta função é útil ao gerar valores de atributos para HTML ou qualquer SGML usando a sintaxe concreta de referência.

class xml.sax.saxutils.XMLGenerator(*out=None*, *encoding='iso-8859-1'*, *short_empty_elements=False*)

Esta classe implementa a interface **ContentHandler** escrevendo eventos SAX de volta em um documento XML. Em outras palavras, usar um **XMLGenerator** como manipulador de conteúdo reproduzirá o documento original que está sendo analisado. *out* deve ser um objeto arquivo ou similar, cujo padrão é *sys.stdout*. *encoding* é a codificação do fluxo de saída, cujo padrão é `'iso-8859-1'`. *short_empty_elements* controla a formatação de elementos sem conteúdo: se `False` (o padrão), eles são emitidos como um par de tags de início/fim; se definido como `True`, eles são emitidos como uma única tag autofechada.

Alterado na versão 3.2: Foi adicionado o parâmetro *short_empty_elements*.

class xml.sax.saxutils.XMLFilterBase(*base*)

Esta classe foi projetada para ficar entre um **XMLReader** e os manipuladores de eventos da aplicação cliente. Por padrão, ela não faz nada além de passar requisições para o leitor e eventos para os manipuladores sem modificações, mas subclasses podem substituir métodos específicos para modificar o fluxo de eventos ou as requisições de configuração à medida que passam.

getParent()

Return the parent reader, or `None` if it is not set.

setParent(*parent*)

Set the parent reader, which the events are read from.

xml.sax.saxutils.prepare_input_source(*source*, *base=''*)

Esta função recebe uma fonte de entrada e uma URL base opcional e retorna um objeto **InputSource** totalmente resolvido, pronto para leitura. A fonte de entrada pode ser fornecida como uma string, um objeto arquivo ou similar ou um objeto **InputSource**; os analisadores usarão esta função para implementar o argumento polimórfico *source* em seu método **parse()**.

`xml.sax.xmlreader` — Interface for XML parsers

Código-fonte: Lib/xml/sax/xmlreader.py
[https://github.com/python/cpython/tree/3.15/Lib/xml/sax/xmlreader.py]

SAX parsers implement the `XMLReader` interface. They are implemented in a Python module, which must provide a function `create_parser()`. This function is invoked by `xml.sax.make_parser()` with no arguments to create a new parser object.

class xml.sax.xmlreader.XMLReader

Base class which can be inherited by SAX parsers.

class xml.sax.xmlreader.IncrementalParser

In some cases, it is desirable not to parse an input source at once, but to feed chunks of the document as they get available. Note that the reader will normally not read the entire file, but read it in chunks as well; still `parse()` won't return until the entire document is processed. So these interfaces should be used if the blocking behaviour of `parse()` is not desirable.

When the parser is instantiated it is ready to begin accepting data from the feed method immediately. After parsing has been finished with a call to close the reset method must be called to make the parser ready to accept new data, either from feed or using the parse method.

Note that these methods must *not* be called during parsing, that is, after parse has been called and before it returns.

By default, the class also implements the parse method of the XMLReader interface using the feed, close and reset methods of the IncrementalParser interface as a convenience to SAX 2.0 driver writers.

class xml.sax.xmlreader.Locator

Interface for associating a SAX event with a document location. A locator object will return valid results only during calls to DocumentHandler methods; at any other time, the results are unpredictable. If information is not available, methods may return `None`.

class xml.sax.xmlreader.InputSource(*system_id=None*)

Encapsulation of the information needed by the **`XMLReader`** to read entities.

This class may include information about the public identifier, system identifier, byte stream (possibly with character encoding information) and/or the character stream of an entity.

Applications will create objects of this class for use in the **`XMLReader.parse()`** method and for returning from EntityResolver.resolveEntity.

An **`InputSource`** belongs to the application, the **`XMLReader`** is not allowed to modify **`InputSource`** objects passed to it from the application, although it may make copies and modify those.

class xml.sax.xmlreader.AttributesImpl(*attrs*)

This is an implementation of the **`Attributes`** interface (see section The Attributes Interface). This is a dictionary-like object which represents the element attributes in a **`startElement()`** call. In addition to the most useful dictionary operations, it supports a number of other methods as described by the interface. Objects of this class should be instantiated by readers; *attrs* must be a dictionary-like object containing a mapping from attribute names to attribute values.

class xml.sax.xmlreader.AttributesNSImpl(*attrs*, *qnames*)

Namespace-aware variant of **`AttributesImpl`**, which will be passed to **`startElementNS()`**. It is derived from **`AttributesImpl`**, but understands attribute names as two-tuples of *namespaceURI* and *localname*. In addition, it provides a number of methods expecting qualified names as

they appear in the original document. This class implements the **`AttributesNS`** interface (see section The AttributesNS Interface).

XMLReader Objects

The **`XMLReader`** interface supports the following methods:

XMLReader.parse(*source*)

Process an input source, producing SAX events. The *source* object can be a system identifier (a string identifying the input source – typically a file name or a URL), a **`pathlib.Path`** or path-like object, or an **`InputSource`** object. When **`parse()`** returns, the input is completely processed, and the parser object can be discarded or reset.

Alterado na versão 3.5: Added support of character streams.

Alterado na versão 3.8: Added support of path-like objects.

XMLReader.getContentHandler()

Return the current **`ContentHandler`**.

XMLReader.setContentHandler(*handler*)

Set the current **`ContentHandler`**. If no **`ContentHandler`** is set, content events will be discarded.

XMLReader.getDTDHandler()

Return the current **`DTDHandler`**.

XMLReader.setDTDHandler(*handler*)

Set the current **`DTDHandler`**. If no **`DTDHandler`** is set, DTD events will be discarded.

XMLReader.getEntityResolver()

Return the current **`EntityResolver`**.

XMLReader.setEntityResolver(*handler*)

Set the current **`EntityResolver`**. If no **`EntityResolver`** is set, attempts to resolve an external entity will result in opening the system identifier for the entity, and fail if it is not available.

XMLReader.getErrorHandler()

Return the current **`ErrorHandler`**.

XMLReader.setErrorHandler(*handler*)

Set the current error handler. If no **`ErrorHandler`** is set, errors will be raised as exceptions, and warnings will be printed.

XMLReader.setLocale(*locale*)

Allow an application to set the locale for errors and warnings.

SAX parsers are not required to provide localization for errors and warnings; if they cannot support the requested locale, however, they must raise a SAX exception. Applications may request a locale change in the middle of a parse.

XMLReader.getFeature(*featurename*)

Return the current setting for feature *featurename*. If the feature is not recognized, **`SAXNotRecognizedException`** is raised. The well-known featurenames are listed in the module **`xml.sax.handler`**.

XMLReader.setFeature(*featurename*, *value*)

Set the *featurename* to *value*. If the feature is not recognized, **`SAXNotRecognizedException`** is raised. If the feature or its setting is not supported by the parser, *SAXNotSupportedException* is raised.

XMLReader.getProperty(*propertyname*)

Return the current setting for property *propertyname*. If the property is not recognized, a **`SAXNotRecognizedException`** is raised. The well-known propertynames are listed in the module **`xml.sax.handler`**.

XMLReader.setProperty(*propertyname*, *value*)

Set the *propertyname* to *value*. If the property is not recognized, **`SAXNotRecognizedException`** is raised. If the property or its setting is not supported by the parser, *SAXNotSupportedException* is raised.

IncrementalParser Objects

Instances of **`IncrementalParser`** offer the following additional methods:

IncrementalParser.feed(*data*)

Process a chunk of *data*.

IncrementalParser.close()

Assume the end of the document. That will check well-formedness conditions that can be checked only at the end, invoke handlers, and may clean up resources allocated during parsing.

IncrementalParser.prepareParser(*source*)

Prepare the parser for parsing *source*, an **`InputSource`** instance. It is called by **`parse()`** before feeding the data. The parser implementation must override this method; the default implementation raises **`NotImplementedError`**.

IncrementalParser.reset()

This method is called after close has been called to reset the parser so that it is ready to parse new documents. The results of calling parse or feed after close without calling reset are undefined.

Locator Objects

Instances of **`Locator`** provide these methods:

Locator.getColumnNumber()

Return the column number where the current event begins.

Locator.getLineNumber()

Return the line number where the current event begins.

Locator.getPublicId()
Return the public identifier for the current event.

Locator.getSystemId()
Return the system identifier for the current event.

InputSource Objects

InputSource.setPublicId(*id*)
Sets the public identifier of this `InputSource`.

InputSource.getPublicId()
Returns the public identifier of this `InputSource`.

InputSource.setSystemId(*id*)
Sets the system identifier of this `InputSource`.

InputSource.getSystemId()
Returns the system identifier of this `InputSource`.

InputSource.setEncoding(*encoding*)
Sets the character encoding of this `InputSource`.

The encoding must be a string acceptable for an XML encoding declaration (see section 4.3.3 of the XML recommendation).

The encoding attribute of the `InputSource` is ignored if the `InputSource` also contains a character stream.

InputSource.getEncoding()
Obtém a codificação de caracteres deste InputSource.

InputSource.setByteStream(*bytefile*)

Set the byte stream (a binary file) for this input source.

The SAX parser will ignore this if there is also a character stream specified, but it will use a byte stream in preference to opening a URI connection itself.

If the application knows the character encoding of the byte stream, it should set it with the setEncoding method.

InputSource.getByteStream()

Get the byte stream for this input source.

The getEncoding method will return the character encoding for this byte stream, or `None` if unknown.

InputSource.setCharacterStream(*charfile*)

Set the character stream (a text file) for this input source.

If there is a character stream specified, the SAX parser will ignore any byte stream and will not attempt to open a URI connection to the system identifier.

InputSource.getCharacterStream()

Get the character stream for this input source.

The `Attributes` Interface

`Attributes` objects implement a portion of the mapping protocol, including the methods **`copy()`**, **`get()`**, **`__contains__()`**, **`items()`**, **`keys()`**, and **`values()`**. The following methods are also provided:

Attributes.getLength()

Return the number of attributes.

Attributes.getNames()

Return the names of the attributes.

Attributes.getType(*name*)

Returns the type of the attribute *name*, which is normally `'CDATA'`.

Attributes.getValue(*name*)

Return the value of attribute *name*.

The `AttributesNS` Interface

This interface is a subtype of the **`Attributes`** interface (see section The Attributes Interface). All methods supported by that interface are also available on **`AttributesNS`** objects.

The following methods are also available:

AttributesNS.getValueByQName(*name*)

Return the value for a qualified name.

AttributesNS.getNameByQName(*name*)

Return the `(namespace, localname)` pair for a qualified *name*.

AttributesNS.getQNameByName(*name*)

Return the qualified name for a `(namespace, localname)` pair.

AttributesNS.getQNames()

Return the qualified names of all attributes.

`xml.parsers.expat` — Fast XML parsing using Expat

Nota

Se você precisar analisar dados não confiáveis ou não autenticados, consulte Segurança no XML.

The **`xml.parsers.expat`** module is a Python interface to the Expat non-validating XML parser. The module provides a single extension type, **`xmlparser`**, that represents the current state of an XML parser. After an **`xmlparser`** object has been created, various attributes of the object can be set to handler functions. When an XML document is then fed to the parser, the handler functions are called for the character data and markup in the XML document.

Este módulo usa o módulo **`pyexpat`** para fornecer acesso ao analisador sintático Expat. O uso direto do módulo **`pyexpat`** foi descontinuado.

This module provides the following exception, type object and data items:

exception xml.parsers.expat.ExpatError

A exceção levantada quando o Expat relata um erro. Veja a seção Exceções ExpatError para mais informações sobre como interpretar erros do Expat.

exception xml.parsers.expat.error

Apelido para **`ExpatError`**.

xml.parsers.expat.XMLParserType

O tipo dos valores de retorno da função **`ParserCreate()`**.

xml.parsers.expat.EXPAT_VERSION

The version string of the Expat library loaded by the interpreter, like `'expat_2.8.4'`.

xml.parsers.expat.version_info

The version of the Expat library loaded by the interpreter, as a tuple of three integers: major, minor and micro version.

xml.parsers.expat.features

The list of the features with which the loaded Expat library was compiled, as `(name, value)` pairs. The value is only meaningful for features which have one, like `'XML_CONTEXT_BYTES'` or the default protection limits `'XML_BLAP_ACT_THRES'` and `'XML_AT_MAX_AMP'`; for other features, like `'XML_DTD'` and `'XML_NS'`, the value is `0` and only the presence of the name is significant.

The **`xml.parsers.expat`** module contains two functions:

xml.parsers.expat.ErrorString(*errno*)

Retorna uma string explicativa para um determinado número de erro *errno*.

xml.parsers.expat.ParserCreate(*encoding=None*, *namespace_separator=None*, *intern=None*)

Cria e retorna um novo objeto **`xmlparser`**. *encoding*, se especificado, deve ser uma string que nomeia a codificação usada pelos dados XML. Expat não provê tantas codificações quanto Python e seu repertório de codificações não pode ser estendido; provê UTF-8, UTF-16, ISO-8859-1 (Latin1) e ASCII. Se *encoding* [[1]] for fornecido, ele substituirá a codificação implícita ou explícita do documento.

Parsers created through **`ParserCreate()`** are called “root” parsers, in the sense that they do not have any parent parser attached. Non-root parsers are created by **`parser.ExternalEntityParserCreate`**.

Opcionalmente, o Expat pode fazer o processamento de espaço de nomes XML para você, habilitado ao fornecer um valor para

namespace_separator. O valor deve ser uma string de um caractere; uma exceção **`ValueError`** será levantada se a string tiver um comprimento ilegal (`None` é considerado o mesmo que omissão). Quando o processamento de espaço de nomes estiver ativado, os nomes de tipos de elementos e nomes de atributos que pertencem a um espaço de nomes serão expandidos. O nome do elemento passado para os manipuladores de elemento **`StartElementHandler`** e **`EndElementHandler`** será a concatenação do URI do espaço de nomes, o caractere separador do espaço de nomes e a parte local do nome. Se o separador do espaço de nomes for um byte zero (`chr(0)`), então o URI do espaço de nomes e a parte local serão concatenados sem qualquer separador.

Por exemplo, se *namespace_separator* for definido como um caractere de espaço (`' '`) e o seguinte documento for analisado:

```
<?xml version="1.0"?>
<root xmlns    = "http://default-namespace.org/"
      xmlns:py = "http://www.python.org/ns/">
  <py:elem1 />
  <elem2 xmlns="" />
</root>
```

`StartElementHandler` receberá as seguintes strings para cada elemento:

```
http://default-namespace.org/ root
http://www.python.org/ns/ elem1
elem2
```

intern, if given, must be a dictionary. It is used to intern the names of elements and attributes, and is available as the **`intern`** attribute. By default a new empty dictionary is created for every parser.

Devido a limitações na biblioteca `Expat` usada por **`pyexpat`**, a instância **`xmlparser`** retornada só pode ser usada para analisar um único documento XML. Chame `ParserCreate` para cada documento para fornecer instâncias exclusivas do analisador sintático.

Ver também

Expat, analisador sintático de XML **[http://www.libexpat.org/]**
Site do projeto Expat

Objetos XMLParser

`xmlparser` objects have the following methods:

xmlparser.Parse(*data*[, *isfinal*])

Parses the contents of *data*, calling the appropriate handler functions to process the parsed data. *data* can be a bytes-like object or a string. If it is a string, the encoding declaration in the XML data is ignored, and the data is parsed as already decoded text. *isfinal* must be true on the final call to this method; it allows the parsing of a single file in fragments, not the submission of multiple files. *data* can be empty at any time.

xmlparser.ParseFile(*file*)

Parse XML data reading from the object *file*. *file* only needs to provide the `read(nbytes)` method, which returns bytes, and an empty bytes object when there's no more data. Text files are not supported; use **`Parse()`** for data which is already decoded.

xmlparser.SetBase(*base*)

Sets the base to be used for resolving relative URIs in system identifiers in declarations. Resolving relative identifiers is left to the application: this value will be passed through as the *base* argument to the **`ExternalEntityRefHandler()`**, **`NotationDeclHandler()`**, and **`UnparsedEntityDeclHandler()`** functions.

xmlparser.GetBase()

Returns a string containing the base set by a previous call to **`SetBase()`**, or `None` if **`SetBase()`** hasn't been called.

xmlparser.GetInputContext()

Returns the input data which generated the current event as a **bytes** object. The data is in the encoding of the entity which contains the text. It extends to the end of the currently buffered input, therefore it can contain also the data of the following events, and if the event was generated by a large amount of text, not all of it may be available. When called while an event handler is not active, the return value is `None`.

xmlparser.ExternalEntityParserCreate(*context*[, *encoding*])

Create a "child" parser which can be used to parse an external parsed entity referred to by content parsed by the parent parser. The *context* parameter should be the string passed to the **ExternalEntityRefHandler()** handler function, described below. The child parser is created with the **ordered_attributes** and **specified_attributes** set to the values of this parser.

xmlparser.SetParamEntityParsing(*flag*)

Control parsing of parameter entities (including the external DTD subset). Possible *flag* values are **XML_PARAM_ENTITY_PARSING_NEVER**, **XML_PARAM_ENTITY_PARSING_UNLESS_STANDALONE** and **XML_PARAM_ENTITY_PARSING_ALWAYS**. Return true if setting the flag was successful.

xmlparser.UseForeignDTD([*flag*])

Calling this with a true value for *flag* (the default) will cause Expat to call the **ExternalEntityRefHandler** with **None** for all arguments to allow an alternate DTD to be loaded. If the document does not contain a document type declaration, the **ExternalEntityRefHandler** will still be called, but the **StartDoctypeDeclHandler** and **EndDoctypeDeclHandler** will not be called.

Passing a false value for *flag* will cancel a previous call that passed a true value, but otherwise has no effect.

This method can only be called before the **Parse()** or **ParseFile()** methods are called; calling it after either of those have been called causes **ExpatError** to be raised with the **code** attribute set to

`errors.codes[errors.XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING]`.

xmlparser.SetReparseDeferralEnabled(*enabled*)

Aviso

Calling `SetReparseDeferralEnabled(False)` has security implications, as detailed below; please make sure to understand these consequences prior to using the `SetReparseDeferralEnabled` method.

Expat 2.6.0 introduced a security mechanism called “reparse deferral” where instead of causing denial of service through quadratic runtime from reparsing large tokens, reparsing of unfinished tokens is now delayed by default until a sufficient amount of input is reached. Due to this delay, registered handlers may — depending of the sizing of input chunks pushed to Expat — no longer be called right after pushing new input to the parser. Where immediate feedback and taking over responsibility of protecting against denial of service from large tokens are both wanted, calling `SetReparseDeferralEnabled(False)` disables reparse deferral for the current Expat parser instance, temporarily or altogether. Calling `SetReparseDeferralEnabled(True)` allows re-enabling reparse deferral.

`SetReparseDeferralEnabled()` has been backported to some prior releases of CPython as a security fix. Check for availability using **`hasattr()`** if used in code running across a variety of Python versions.

Adicionado na versão 3.13.

xmlparser.GetReparseDeferralEnabled()

Returns whether reparse deferral is currently enabled for the given Expat parser instance.

Adicionado na versão 3.13.

`xmlparser` objects have the following methods to tune protections against some common XML vulnerabilities.

xmlparser.SetBillionLaughsAttackProtectionActivationThreshold(*threshold*, *[/]*)

Sets the number of output bytes needed to activate protection against billion laughs [https://en.wikipedia.org/wiki/Billion_laughs_attack] attacks.

The number of output bytes includes amplification from entity expansion and reading DTD files.

Parser objects usually have a protection activation threshold of 8 MiB, but the actual default value depends on the underlying Expat library.

An **`ExpatError`** is raised if this method is called on a non-root parser. The corresponding **`lineno`** and **`offset`** should not be used as they may have no special meaning.

`SetBillionLaughsAttackProtectionActivationThreshold()` has been backported to some prior releases of CPython as a security fix. Check for availability using **`hasattr()`** if used in code running across a variety of Python versions.

Nota

Activation thresholds below 4 MiB are known to break support for DITA 1.3 payload and are hence not recommended.

Adicionado na versão 3.15.

xmlparser.SetBillionLaughsAttackProtectionMaximumAmplification(*max_factor*, *[/]*)

Sets the maximum tolerated amplification factor for protection against billion laughs [https://en.wikipedia.org/wiki/Billion_laughs_attack] attacks.

The amplification factor is calculated as `(direct + indirect) / direct` while parsing, where `direct` is the number of bytes read from

the primary document in parsing and `indirect` is the number of bytes added by expanding entities and reading of external DTD files.

The *max_factor* value must be a non-NaN **`float`** value greater than or equal to 1.0. Peak amplifications of factor 15,000 for the entire payload and of factor 30,000 in the middle of parsing have been observed with small benign files in practice. In particular, the activation threshold should be carefully chosen to avoid false positives.

Parser objects usually have a maximum amplification factor of 100, but the actual default value depends on the underlying Expat library.

An **`ExpatError`** is raised if this method is called on a non-root parser or if *max_factor* is outside the valid range. The corresponding **`lineno`** and **`offset`** should not be used as they may have no special meaning.

`SetBillionLaughsAttackProtectionMaximumAmplification()` has been backported to some prior releases of CPython as a security fix. Check for availability using **`hasattr()`** if used in code running across a variety of Python versions.

Nota

The maximum amplification factor is only considered if the threshold that can be adjusted by **`SetBillionLaughsAttackProtectionActivationThreshold()`** is exceeded.

Adicionado na versão 3.15.

xmlparser.SetAllocTrackerActivationThreshold(*threshold*, /)

Sets the number of allocated bytes of dynamic memory needed to activate protection against disproportionate use of RAM.

Parser objects usually have an allocation activation threshold of 64 MiB, but the actual default value depends on the underlying Expat library.

An **`ExpatError`** is raised if this method is called on a non-root parser. The corresponding **`lineno`** and **`offset`** should not be used as they may have no special meaning.

`SetAllocTrackerActivationThreshold()` has been backported to some prior releases of CPython as a security fix. Check for availability using **`hasattr()`** if used in code running across a variety of Python versions.

Adicionado na versão 3.15.

xmlparser.SetAllocTrackerMaximumAmplification(*max_factor*, /)

Sets the maximum amplification factor between direct input and bytes of dynamic memory allocated.

The amplification factor is calculated as `allocated / direct` while parsing, where `direct` is the number of bytes read from the primary document in parsing and `allocated` is the number of bytes of dynamic memory allocated in the parser hierarchy.

The *max_factor* value must be a non-NaN **`float`** value greater than or equal to 1.0. Amplification factors greater than 100.0 can be observed near the start of parsing even with benign files in practice. In particular, the activation threshold should be carefully chosen to avoid false positives.

Parser objects usually have a maximum amplification factor of 100, but the actual default value depends on the underlying Expat library.

An **`ExpatError`** is raised if this method is called on a non-root parser or if *max_factor* is outside the valid range. The corresponding **`lineno`** and **`offset`** should not be used as they may have no special meaning.

`SetAllocTrackerMaximumAmplification()` has been backported to some prior releases of CPython as a security fix. Check for availability using **`hasattr()`** if used in code running across a variety of Python versions.

Nota

The maximum amplification factor is only considered if the threshold that can be adjusted by `SetAllocTrackerActivationThreshold()` is exceeded.

Adicionado na versão 3.15.

`xmlparser` objects have the following attributes:

xmlparser.buffer_size

The size of the buffer used when `buffer_text` is true. A new buffer size can be set by assigning a new integer value to this attribute. When the size is changed, the buffer will be flushed.

xmlparser.buffer_text

Setting this to true causes the `xmlparser` object to buffer textual content returned by Expat to avoid multiple calls to the `CharacterDataHandler()` callback whenever possible. This can improve performance substantially since Expat normally breaks character data into chunks at every line ending. This attribute is false by default, and may be changed at any time. Note that when it is false, data that does not contain newlines may be chunked too.

xmlparser.buffer_used

If `buffer_text` is enabled, the number of bytes stored in the buffer. These bytes represent UTF-8 encoded text. This attribute has no meaningful interpretation when `buffer_text` is false.

xmlparser.ordered_attributes

Setting this attribute to a non-zero integer causes the attributes to be reported as a list rather than a dictionary. The attributes are presented in the order found in the document text. For each attribute, two list entries are presented: the attribute name and the attribute value. (Older versions of this module also used this format.) By default, this attribute is false; it may be changed at any time.

xmlparser.specified_attributes

If set to a non-zero integer, the parser will report only those attributes which were specified in the document instance and not those which were derived from attribute declarations. Applications which set this need to be especially careful to use what additional information is available from the declarations as needed to comply with the standards for the behavior of XML processors. By default, this attribute is false; it may be changed at any time.

xmlparser.intern

The dictionary used to intern the names of elements and attributes. It is either the dictionary passed as the *intern* argument of **`ParserCreate()`**, or a new dictionary created for this parser.

xmlparser.namespace_prefixes

If set to a true value, and namespace processing is enabled, the namespace prefix is reported as the third part of the expanded name, separated by the namespace separator. Names which have no prefix are not affected. By default, this attribute is false; it may be changed at any time.

The following attributes contain values relating to the most recent error encountered by an **`xmlparser`** object, and will only have correct values once a call to **`Parse()`** or **`ParseFile()`** has raised an **`xml.parsers.expat.ExpatError`** exception.

xmlparser.ErrorByteIndex

Byte index at which an error occurred.

xmlparser.ErrorCode

Numeric code specifying the problem. This value can be passed to the **`ErrorString()`** function, or compared to one of the constants defined in the `errors` object.

xmlparser.ErrorColumnNumber

Column number at which an error occurred.

xmlparser.ErrorLineNumber

Line number at which an error occurred.

The following attributes contain values relating to the current parse location in an **`xmlparser`** object. During a callback reporting a parse event they indicate the location of the first of the sequence of characters that generated the event. When called outside of a callback, the position indicated will be just past the last parse event (regardless of whether there was an associated callback).

xmlparser.CurrentByteIndex

Current byte index in the parser input.

xmlparser.CurrentColumnNumber

Current column number in the parser input.

xmlparser.CurrentLineNumber

Current line number in the parser input.

Here is the list of handlers that can be set. To set a handler on an **`xmlparser`** object *o*, use `o.handlername = func`. *handlername* must be taken from the following list, and *func* must be a callable object accepting the correct number of arguments. The arguments are all strings, unless otherwise stated.

xmlparser.XmlDeclHandler(*version*, *encoding*, *standalone*)

Called when the XML declaration is parsed. The XML declaration is the (optional) declaration of the applicable version of the XML recommendation, the encoding of the document text, and an optional "standalone" declaration. *version* and *encoding* will be strings, and *standalone* will be `1` if the document is declared standalone, `0` if it is declared not to be standalone, or `-1` if the standalone clause was omitted.

xmlparser.StartDoctypeDeclHandler(*doctypeName*, *systemId*, *publicId*, *has_internal_subset*)

Called when Expat begins parsing the document type declaration (`<!DOCTYPE ...`). The *doctypeName* is provided exactly as presented. The *systemId* and *publicId* parameters give the system and public identifiers if specified, or `None` if omitted. *has_internal_subset* will be true if the document contains an internal document declaration subset.

xmlparser.EndDoctypeDeclHandler()

Called when Expat is done parsing the document type declaration.

xmlparser.ElementDeclHandler(*name*, *model*)

Called once for each element type declaration. *name* is the name of the element type, and *model* is a representation of the content model.

xmlparser.AttlistDeclHandler(*elname*, *attname*, *type*, *default*, *required*)

Called for each declared attribute for an element type. If an attribute list declaration declares three attributes, this handler is called three times, once for each attribute. *elname* is the name of the element to which the declaration applies and *attname* is the name of the attribute declared. The The attribute type is a string passed as *type*: `'CDATA'`, `'ID'`, `'IDREF'`, `'IDREFS'`, `'ENTITY'`, `'ENTITIES'`, `'NMTOKEN'` or `'NMTOKENS'`, an enumeration like `'(x|y)'`, or a notation list like `'NOTATION(n1|n2)'`. *default* gives the default value for the attribute used when the attribute is not specified by the document instance, or `None` if there is no default value (`#IMPLIED` values). If the attribute is required to be given in the document instance, *required* will be true.

xmlparser.StartElementHandler(*name*, *attributes*)

Called for the start of every element. *name* is a string containing the element name, and *attributes* is the element attributes. If **`ordered_attributes`** is true, this is a list (see **`ordered_attributes`** for a full description). Otherwise it's a dictionary mapping names to values.

xmlparser.EndElementHandler(*name*)

Called for the end of every element.

xmlparser.ProcessingInstructionHandler(*target*, *data*)

Called for every processing instruction.

xmlparser.CharacterDataHandler(*data*)

Called for character data. This will be called for normal character data, CDATA marked content, and ignorable whitespace. Applications which must distinguish these cases can use the **`StartCdataSectionHandler`**, **`EndCdataSectionHandler`**, and **`ElementDeclHandler`** callbacks to collect the required information. Note that the character data may be chunked even if it is short and so you may receive more than one call to **`CharacterDataHandler()`**. Set the **`buffer_text`** instance attribute to `True` to avoid that.

xmlparser.UnparsedEntityDeclHandler(*entityName*, *base*, *systemId*, *publicId*, *notationName*)

Called for unparsed (NDATA) entity declarations. If this handler is not set, such declarations are reported by **`EntityDeclHandler`**, which is preferred for new code. (The underlying function in the Expat library has been declared obsolete.)

xmlparser.EntityDeclHandler(*entityName*, *is_parameter_entity*, *value*, *base*, *systemId*, *publicId*, *notationName*)

Called for all entity declarations. For parameter and internal entities, *value* will be a string giving the declared contents of the entity; this will be `None` for external entities. The *notationName* parameter will be `None` for parsed entities, and the name of the notation for unparsed entities. *is_parameter_entity* will be true if the entity is a parameter entity or false for general entities (most applications only need to be concerned with general entities).

xmlparser.NotationDeclHandler(*notationName*, *base*, *systemId*, *publicId*)

Called for notation declarations. *notationName*, *base*, and *systemId*, and *publicId* are strings if given. If the public identifier is omitted, *publicId* will be `None`.

xmlparser.StartNamespaceDeclHandler(*prefix*, *uri*)

Called when an element contains a namespace declaration. Namespace declarations are processed before the `StartElementHandler` is called for the element on which declarations are placed.

xmlparser.EndNamespaceDeclHandler(*prefix*)

Called when the closing tag is reached for an element that contained a namespace declaration. This is called once for each namespace declaration on the element in the reverse of the order for which the `StartNamespaceDeclHandler` was called to indicate the start of each namespace declaration's scope. Calls to this handler are made after the corresponding `EndElementHandler` for the end of the element.

xmlparser.CommentHandler(*data*)

Called for comments. *data* is the text of the comment, excluding the leading `'<!--'` and trailing `'-->'`.

xmlparser.StartCdataSectionHandler()

Called at the start of a CDATA section. This and `EndCdataSectionHandler` are needed to be able to identify the syntactical start and end for CDATA sections.

xmlparser.EndCdataSectionHandler()

Called at the end of a CDATA section.

xmlparser.DefaultHandler(*data*)

Called for any characters in the XML document for which no applicable handler has been specified. This means characters that are part of a construct which could be reported, but for which no handler has been supplied.

xmlparser.DefaultHandlerExpand(*data*)

This is the same as the `DefaultHandler`, but doesn't inhibit expansion of internal entities. The entity reference will not be passed to the default handler.

xmlparser.NotStandaloneHandler()

Called if the XML document hasn't been declared as being a standalone document. This happens when there is an external subset or a reference to a parameter entity, but the XML declaration does not set standalone to `yes` in an XML declaration. If this handler returns `0`, then the parser will raise an **`XML_ERROR_NOT_STANDALONE`** error. If this handler is not set, no exception is raised by the parser for this condition.

xmlparser.ExternalEntityRefHandler(*context*, *base*, *systemId*, *publicId*)

Aviso

Implementing a handler that accesses local files and/or the network may create a vulnerability to external entity attacks [https://en.wikipedia.org/wiki/XML_external_entity_attack] if **`xmlparser`** is used with user-provided XML content. Please reflect on your threat model [https://en.wikipedia.org/wiki/Threat_model] before implementing this handler.

Called for references to external entities. *base* is the current base, as set by a previous call to **`SetBase()`**. The public and system identifiers, *systemId* and *publicId*, are strings if given; if the public identifier is not given, *publicId* will be `None`. The *context* value is opaque and should only be used as described below.

For external entities to be parsed, this handler must be implemented. It is responsible for creating the sub-parser using `ExternalEntityParserCreate(context)`, initializing it with the appropriate callbacks, and parsing the entity. This handler should return an integer; if it returns `0`, the parser will raise an **`XML_ERROR_EXTERNAL_ENTITY_HANDLING`** error, otherwise parsing will continue.

If this handler is not provided, external entities are reported by the **`DefaultHandler`** callback, if provided.

xmlparser.SkippedEntityHandler(*entityName*, *is_parameter_entity*)

Called for entity references which are not expanded, because the parser did not read the declaration of the entity. This happens when the external DTD subset or an external parameter entity is not parsed.

is_parameter_entity is true for a parameter entity and false for a general entity.

Exceções ExpatError

`ExpatError` exceptions have a number of interesting attributes:

ExpatError.code

Expat's internal error number for the specific error. The `errors.messages` dictionary maps these error numbers to Expat's error messages. For example:

```
from xml.parsers.expat import ParserCreate, ExpatError, error

p = ParserCreate()
try:
    p.Parse(some_xml_document)
except ExpatError as err:
    print("Error:", errors.messages[err.code])
```

The `errors` module also provides error message constants and a dictionary `codes` mapping these messages back to the error codes, see below.

ExpatError.lineno

Line number on which the error was detected. The first line is numbered `1`.

ExpatError.offset

Character offset into the line where the error occurred. The first column is numbered `0`.

Exemplo

The following program defines three handlers that just print out their arguments.

```
import xml.parsers.expat

# 3 handler functions
def start_element(name, attrs):
    print('Start element:', name, attrs)
def end_element(name):
    print('End element:', name)
def char_data(data):
    print('Character data:', repr(data))

p = xml.parsers.expat.ParserCreate()

p.StartElementHandler = start_element
p.EndElementHandler = end_element
p.CharacterDataHandler = char_data

p.Parse("""<?xml version="1.0"?>
<parent id="top">><child1 name="paul">Text goes here</child1>
<child2 name="fred">More text</child2>
</parent>""", 1)
```

The output from this program is:

```
Start element: parent {'id': 'top'}
Start element: child1 {'name': 'paul'}
Character data: 'Text goes here'
End element: child1
Character data: '\n'
Start element: child2 {'name': 'fred'}
Character data: 'More text'
End element: child2
Character data: '\n'
End element: parent
```

Content Model Descriptions

Content models are described using nested tuples. Each tuple contains four values: the type, the quantifier, the name, and a tuple of children. Children are simply additional content model descriptions.

The values of the first two fields are constants defined in the **`xml.parsers.expat.model`** module. These constants can be collected in two groups: the model type group and the quantifier group.

The constants in the model type group are:

xml.parsers.expat.model.XML_CTYPE_ANY
: The element named by the model name was declared to have a content model of `ANY`.

xml.parsers.expat.model.XML_CTYPE_CHOICE
: The named element allows a choice from a number of options; this is used for content models such as `(A | B | C)`.

xml.parsers.expat.model.XML_CTYPE_EMPTY
: Elementos declarados como `EMPTY` têm este tipo de modelo.

xml.parsers.expat.model.XML_CTYPE_MIXED
: The named element allows character data, optionally interspersed with the named children; this is used for content models such as `(#PCDATA)` and `(#PCDATA | A | B)*`.

xml.parsers.expat.model.XML_CTYPE_NAME
: The model names a single element, as for `A`.

xml.parsers.expat.model.XML_CTYPE_SEQ
: Models which represent a series of models which follow one after the other are indicated with this model type. This is used for models such as `(A, B, C)`.

The constants in the quantifier group are:

xml.parsers.expat.model.XML_CQUANT_NONE
: No modifier is given, so it can appear exactly once, as for `A`.

xml.parsers.expat.model.XML_CQUANT_OPT
: The model is optional: it can appear once or not at all, as for `A?`.

xml.parsers.expat.model.XML_CQUANT_PLUS
The model must occur one or more times (like `A+`).

xml.parsers.expat.model.XML_CQUANT_REP
The model must occur zero or more times, as for `A*`.

Constantes de erro de expatriados

The following constants are provided in the **`xml.parsers.expat.errors`** module. These constants are useful in interpreting some of the attributes of the **`ExpatError`** exception objects raised when an error has occurred. Since for backwards compatibility reasons, the constants' value is the error *message* and not the numeric error *code*, you do this by comparing its **`code`** attribute with `errors.codes[errors.XML_ERROR_`*`CONSTANT_NAME`*`]`.

The `errors` module has the following attributes:

xml.parsers.expat.errors.codes
A dictionary mapping string descriptions to their error codes.

Adicionado na versão 3.2.

xml.parsers.expat.errors.messages
A dictionary mapping numeric error codes to their string descriptions.

Adicionado na versão 3.2.

xml.parsers.expat.errors.XML_ERROR_ASYNC_ENTITY

xml.parsers.expat.errors.XML_ERROR_ATTRIBUTE_EXTERNAL_ENTITY_REF
An entity reference in an attribute value referred to an external entity instead of an internal entity.

xml.parsers.expat.errors.XML_ERROR_BAD_CHAR_REF
A character reference referred to a character which is illegal in XML (for example, character `0`, or '`&#0;`').

xml.parsers.expat.errors.XML_ERROR_BINARY_ENTITY_REF

An entity reference referred to an entity which was declared with a notation, so cannot be parsed.

xml.parsers.expat.errors.XML_ERROR_DUPLICATE_ATTRIBUTE

An attribute was used more than once in a start tag.

xml.parsers.expat.errors.XML_ERROR_INCORRECT_ENCODING

xml.parsers.expat.errors.XML_ERROR_INVALID_TOKEN

Raised when an input byte could not properly be assigned to a character; for example, a NUL byte (value `0`) in a UTF-8 input stream.

xml.parsers.expat.errors.XML_ERROR_JUNK_AFTER_DOC_ELEMENT

Something other than whitespace occurred after the document element.

xml.parsers.expat.errors.XML_ERROR_MISPLACED_XML_PI

An XML declaration was found somewhere other than the start of the input data.

xml.parsers.expat.errors.XML_ERROR_NO_ELEMENTS

The document contains no elements (XML requires all documents to contain exactly one top-level element).

xml.parsers.expat.errors.XML_ERROR_NO_MEMORY

Expat was not able to allocate memory internally.

xml.parsers.expat.errors.XML_ERROR_PARAM_ENTITY_REF

A parameter entity reference was found where it was not allowed.

xml.parsers.expat.errors.XML_ERROR_PARTIAL_CHAR

An incomplete character was found in the input.

xml.parsers.expat.errors.XML_ERROR_RECURSIVE_ENTITY_REF

An entity reference contained another reference to the same entity; possibly via a different name, and possibly indirectly.

xml.parsers.expat.errors.XML_ERROR_SYNTAX

Some unspecified syntax error was encountered.

xml.parsers.expat.errors.XML_ERROR_TAG_MISMATCH

An end tag did not match the innermost open start tag.

xml.parsers.expat.errors.XML_ERROR_UNCLOSED_TOKEN

Some token (such as a start tag) was not closed before the end of the stream or the next token was encountered.

xml.parsers.expat.errors.XML_ERROR_UNDEFINED_ENTITY

A reference was made to an entity which was not defined.

xml.parsers.expat.errors.XML_ERROR_UNKNOWN_ENCODING

The document encoding is not supported by Expat.

xml.parsers.expat.errors.XML_ERROR_UNCLOSED_CDATA_SECTION

A CDATA marked section was not closed.

xml.parsers.expat.errors.XML_ERROR_EXTERNAL_ENTITY_HANDLING

xml.parsers.expat.errors.XML_ERROR_NOT_STANDALONE

The parser determined that the document was not “standalone” though it declared itself to be in the XML declaration, and the **`NotStandaloneHandler`** was set and returned `0`.

xml.parsers.expat.errors.XML_ERROR_UNEXPECTED_STATE

xml.parsers.expat.errors.XML_ERROR_ENTITY_DECLARED_IN_PE

xml.parsers.expat.errors.XML_ERROR_FEATURE_REQUIRES_XML_DTD

An operation was requested that requires DTD support to be compiled in, but Expat was configured without DTD support. This should never be reported by a standard build of the **`xml.parsers.expat`** module.

xml.parsers.expat.errors.XML_ERROR_CANT_CHANGE_FEATURE_ONCE_PARSING

A behavioral change was requested after parsing started that can only be changed before parsing has started. This is (currently) only raised by `UseForeignDTD()`.

xml.parsers.expat.errors.XML_ERROR_UNBOUND_PREFIX

An undeclared prefix was found when namespace processing was enabled.

xml.parsers.expat.errors.XML_ERROR_UNDECLARING_PREFIX

The document attempted to remove the namespace declaration associated with a prefix.

xml.parsers.expat.errors.XML_ERROR_INCOMPLETE_PE

A parameter entity contained incomplete markup.

xml.parsers.expat.errors.XML_ERROR_XML_DECL

There was an error parsing the XML declaration.

xml.parsers.expat.errors.XML_ERROR_TEXT_DECL

There was an error parsing a text declaration in an external entity.

xml.parsers.expat.errors.XML_ERROR_PUBLICID

Characters were found in the public id that are not allowed.

xml.parsers.expat.errors.XML_ERROR_SUSPENDED

The requested operation was made on a suspended parser, but isn’t allowed. This includes attempts to provide additional input or to stop the parser.

xml.parsers.expat.errors.XML_ERROR_NOT_SUSPENDED

An attempt to resume the parser was made when the parser had not been suspended.

xml.parsers.expat.errors.XML_ERROR_ABORTED

This should not be reported to Python applications.

xml.parsers.expat.errors.XML_ERROR_FINISHED

The requested operation was made on a parser which was finished parsing input, but isn't allowed. This includes attempts to provide additional input or to stop the parser.

xml.parsers.expat.errors.XML_ERROR_SUSPEND_PE

xml.parsers.expat.errors.XML_ERROR_RESERVED_PREFIX_XML

An attempt was made to undeclare reserved namespace prefix `xml` or to bind it to another namespace URI.

xml.parsers.expat.errors.XML_ERROR_RESERVED_PREFIX_XMLNS

An attempt was made to declare or undeclare reserved namespace prefix `xmlns`.

xml.parsers.expat.errors.XML_ERROR_RESERVED_NAMESPACE_URI

An attempt was made to bind the URI of one the reserved namespace prefixes `xml` and `xmlns` to another namespace prefix.

xml.parsers.expat.errors.XML_ERROR_INVALID_ARGUMENT

This should not be reported to Python applications.

xml.parsers.expat.errors.XML_ERROR_NO_BUFFER

This should not be reported to Python applications.

xml.parsers.expat.errors.XML_ERROR_AMPLIFICATION_LIMIT_BREACH

The limit on input amplification factor (from DTD and entities) has been breached.

xml.parsers.expat.errors.XML_ERROR_NOT_STARTED

The parser was tried to be stopped or suspended before it started.

Adicionado na versão 3.14.

Notas de rodapé

[1]

The encoding string included in XML output should conform to the appropriate standards. For example, “UTF-8” is valid, but “UTF8” is not. See https://www.w3.org/TR/2006/REC-xml11-20060816/#NT-EncodingDecl and https://www.iana.org/assignments/character-sets/character-sets.xhtml.

Protocolos de Internet e Suporte

Os módulos descritos neste capítulo implementam protocolos de internet e suporte para tecnologia relacionada. Todos eles são implementados em Python. A maioria desses módulos requer a presença do módulo dependente do sistema `socket`, que atualmente é suportado na maioria das plataformas populares. Segue uma visão geral:

- `webbrowser` — Controlador de navegador web conveniente
  - Interface de linha de comando
  - Objetos controladores de navegador
- `wsgiref` — Implementação de referência e utilitários WSGI
  - `wsgiref.util` – WSGI environment utilities
  - `wsgiref.headers` – WSGI response header tools
  - `wsgiref.simple_server` – a simple WSGI HTTP server
  - `wsgiref.validate` — WSGI conformance checker
  - `wsgiref.handlers` – server/gateway base classes
  - `wsgiref.types` – WSGI types for static type checking
  - Exemplos
- `urllib` — Módulos de manipulação de URL
- `urllib.request` — Biblioteca extensível para abrir URLs
  - Objeto Request
  - OpenerDirector Objects
  - BaseHandler Objects
  - HTTPRedirectHandler Objects
  - HTTPCookieProcessor Objects
  - ProxyHandler Objects
  - HTTPPasswordMgr Objects
  - HTTPPasswordMgrWithPriorAuth Objects
  - AbstractBasicAuthHandler Objects
  - HTTPBasicAuthHandler Objects
  - ProxyBasicAuthHandler Objects
  - AbstractDigestAuthHandler Objects
  - HTTPDigestAuthHandler Objects
  - ProxyDigestAuthHandler Objects

    - HTTPHandler Objects
    - Objetos HTTPSHandler
    - FileHandler Objects
    - DataHandler Objects
    - FTPHandler Objects
    - CacheFTPHandler Objects
    - Objetos UnknownHandler
    - HTTPErrorProcessor Objects
    - Exemplos
    - Legacy interface
    - `urllib.request` Restrictions
- `urllib.response` — Response classes used by urllib
- `urllib.parse` — Analisa URLs para componentes
    - Análise de URL
    - Segurança ao analisar URLs
    - Analisando bytes codificados em ASCII
    - Structured Parse Results
    - URL Quoting
- `urllib.error` — Classes de exceção levantadas por urllib.request
- `urllib.robotparser` — Analisador sintático de robots.txt
- `http` — Módulos HTTP
    - Códigos de estado HTTP
    - Categoria de estado HTTP
    - Métodos HTTP
- `http.client` — HTTP protocol client
    - Objetos de HTTPConnection
    - Objetos HTTPResponse
    - Exemplos
    - HTTPMessage Objects
- `ftplib` — FTP protocol client
    - Referência
        - FTP objects
        - FTP_TLS objects
        - Module variables
- `poplib` — Cliente de protocolo POP3
    - Objetos POP3
    - Exemplo POP3

- `imaplib` — IMAP4 protocol client
  - Objetos IMAP4
  - Exemplo IMAP4
- `smtplib` — Cliente de protocolo SMTP
  - Objetos SMTP
  - Exemplo SMTP
- `uuid` — Objetos UUID conforme **RFC 9562**
  - Uso na linha de comando
  - Exemplo
  - Exemplo de linha de comando
- `socketserver` — Um framework para servidores de rede
  - Notas sobre criação do servidor
  - Objetos Server
  - Objetos manipulador de requisições
  - Exemplos
    - Exemplo de `socketserver.TCPServer`
    - Exemplo `socketserver.UDPServer`
    - Mixins assíncronos
- `http.server` — HTTP servers
  - Interface de linha de comando
  - Considerações de segurança
- `http.cookies` — HTTP state management
  - Objetos Cookie
  - Objetos Morsel
  - Exemplo
- `http.cookiejar` — Cookie handling for HTTP clients
  - CookieJar and FileCookieJar objects
  - FileCookieJar subclasses and co-operation with web browsers
  - CookiePolicy objects
  - DefaultCookiePolicy objects
  - Cookie objects
  - Exemplos
- `xmlrpc` — Módulos de servidor e cliente XMLRPC
- `xmlrpc.client` — Cliente XML-RPC
  - Objetos ServerProxy
  - Objetos DateTime
  - Objetos Binários

    - Objetos Fault
    - Objetos ProtocolError
    - Objetos MultiCall
    - Convenience Functions
    - Exemplo de uso do cliente
    - Example of Client and Server Usage
- `xmlrpc.server` — Servidores XML-RPC básicos
    - SimpleXMLRPCServer objects
        - SimpleXMLRPCServer example
    - CGIXMLRPCRequestHandler
    - Documentando servidor XMLRPC
    - DocXMLRPCServer objects
    - DocCGIXMLRPCRequestHandler
- `ipaddress` — Biblioteca de manipulação de IPv4/IPv6
    - Funções de fábrica de conveniência
    - Endereços IP
        - Objetos de endereço
        - Conversão para strings e inteiros
        - Operadores
            - Operadores de comparação
            - Operadores aritméticos
    - Definições de rede IP
        - Prefixo, máscara de rede e máscara de host
        - Objetos de rede
        - Operadores
            - Operadores lógicos
            - Iteração
            - Redes como contêineres de endereços
    - Objetos de interface
        - Operadores
            - Operadores lógicos
    - Outras funções de nível de módulo
    - Exceções personalizadas

`webbrowser` — Controlador de navegador web conveniente

Código-fonte: Lib/webbrowser.py
[https://github.com/python/cpython/tree/3.15/Lib/webbrowser.py]

The **`webbrowser`** module provides a high-level interface to allow displaying web-based documents to users. Under most circumstances, simply calling the **`open()`** function from this module will do the right thing.

No Unix, navegadores gráficos são preferidos no X11, mas navegadores em modo texto serão usados se navegadores gráficos não estiverem disponíveis ou um display X11 não estiver disponível. Se navegadores em modo texto forem usados, o processo de chamada será bloqueado até que o usuário saia do navegador.

Se a variável de ambiente **`BROWSER`** existir, ela será interpretada como a lista separada por **`os.pathsep`** de navegadores a serem tentados antes dos padrões da plataforma. Quando o valor de uma parte da lista contém a string `%s`, então ela é interpretada como uma linha de comando literal do navegador a ser usada com o argumento URL substituído por `%s`; se o valor for uma única palavra que se refere a um dos navegadores já registrados, este navegador será adicionado à frente da lista de pesquisa; se a parte não contiver `%s`, ela será simplesmente interpretada como o nome do navegador a ser iniciado. [[1]]

Alterado na versão 3.14: A variável **`BROWSER`** agora também pode ser usada para reordenar a lista de padrões da plataforma. Isso é particularmente útil no macOS, onde os padrões da plataforma não se referem às ferramentas de linha de comando em **`PATH`**.

Para plataformas não-Unix, ou quando um navegador remoto estiver disponível no Unix, o processo de controle não esperará que o usuário

termine de usar o navegador, mas permitirá que o navegador remoto mantenha suas próprias janelas na tela. Se os navegadores remotos não estiverem disponíveis no Unix, o processo de controle iniciará um novo navegador e aguardará.

On iOS, the **BROWSER** environment variable, as well as any arguments controlling autoraise, browser preference, and new tab/window creation will be ignored. Web pages will *always* be opened in the user's preferred browser, in a new tab, with the browser being brought to the foreground. The use of the **webbrowser** module on iOS requires the **ctypes** module. If **ctypes** isn't available, calls to **open()** will fail.

Interface de linha de comando

O script **webbrowser** pode ser usado como uma interface de linha de comando para o módulo. Ele aceita uma URL como argumento. Ele aceita os seguintes parâmetros opcionais:

-n, --new-window
    Abre a URL em uma nova janela do navegador, se possível.

-t, --new-tab
    Abre a URL em uma nova aba do navegador.

As opções são, naturalmente, mutuamente exclusivas. Exemplo de uso:

```
python -m webbrowser -t "https://www.python.org"
```

Disponibilidade: not WASI, not Android.

A seguinte exceção é definida:

exception webbrowser.Error
    Exceção levantada quando ocorre um erro de controle do navegador.

As seguintes funções estão definidas:

webbrowser.open(*url*, *new=0*, *autoraise=True*)

Exibe *url* usando o navegador padrão. Se *new* for 0, o *url* será aberto na mesma janela do navegador, se possível. Se *new* for 1, uma nova janela do navegador será aberta, se possível. Se *new* for 2, uma nova página do navegador (“aba” ou “guia”) será aberta, se possível. Se *autoraise* for `True`, a janela será levantada, se possível (observe que, em muitos gerenciadores de janelas, isso ocorrerá independentemente da configuração dessa variável).

Retorna `True` se um navegador foi iniciado com sucesso, `False` caso contrário.

Note que em algumas plataformas, tentar abrir um nome de arquivo usando esta função pode funcionar e iniciar o programa associado do sistema operacional. No entanto, isso não é suportado nem portátil.

Levanta um evento de auditoria `webbrowser.open` com o argumento `url`.

webbrowser.open_new(*url*)

Abre *url* em uma nova janela do navegador padrão, se possível; caso contrário, abre *url* na única janela do navegador.

Retorna `True` se um navegador foi iniciado com sucesso, `False` caso contrário.

webbrowser.open_new_tab(*url*)

Abre *url* em uma nova página (“aba” ou “guia”) do navegador padrão, se possível, caso contrário, equivalente a **`open_new()`**.

Retorna `True` se um navegador foi iniciado com sucesso, `False` caso contrário.

webbrowser.get(*using=None*)

Retorna um objeto controlador para o tipo de navegador *using*. Se *using* for `None`, retorna um controlador para um navegador padrão apropriado para o ambiente do chamador.

webbrowser.register(*name*, *constructor*, *instance=None*, *, *preferred=False*)

Registra o tipo de navegador *name*. Uma vez que um tipo de navegador é registrado, a função **get()** pode retornar um controlador para esse tipo de navegador. Se *instance* não for fornecido, ou for `None`, *constructor* será chamado sem parâmetros para criar uma instância quando necessário. Se *instance* for fornecido, *constructor* nunca será chamado, e pode ser `None`.

Definir *preferred* como `True` torna este navegador um resultado preferencial para uma chamada **get()** sem argumento. Caso contrário, este ponto de entrada só é útil se você planeja definir a variável **BROWSER** ou chamar **get()** com um argumento não vazio que corresponda ao nome de um manipulador que você declarar.

Alterado na versão 3.7: Parâmetro somente-nomeado *preferred* foi adicionado.

Vários tipos de navegadores são predefinidos. Esta tabela fornece os nomes de tipos que podem ser passados para a função **get()** e as instanciações correspondentes para as classes do controlador, todas definidas neste módulo.

Nome do tipo	Nome da classe	Notas
`'mozilla'`	`Mozilla('mozilla')`	
`'firefox'`	`Mozilla('mozilla')`	
`'epiphany'`	`Epiphany('epiphany')`	
`'kfmclient'`	`Konqueror()`	(1)

Nome do tipo	Nome da classe	Notas
`'konqueror'`	`Konqueror()`	(1)
`'kfm'`	`Konqueror()`	(1)
`'opera'`	`Opera()`	
`'links'`	`GenericBrowser('links')`	
`'elinks'`	`Elinks('elinks')`	
`'lynx'`	`GenericBrowser('lynx')`	
`'w3m'`	`GenericBrowser('w3m')`	
`'windows-default'`	`WindowsDefault`	(2)
`'macos'`	`MacOS('default')`	(3)
`'safari'`	`MacOS('safari')`	(3)
`'chrome'`	`MacOS('google chrome')`	(3)
`'firefox'`	`MacOS('firefox')`	(3)

Nome do tipo	Nome da classe	Notas
`'google-chrome'`	`Chrome('google-chrome')`	
`'chromium'`	`Chromium('chromium')`	
`'chromium-browser'`	`Chromium('chromium-browser')`	
`'iosbrowser'`	`IOSBrowser`	(4)

Notas:

1. “Konqueror” é o gerenciador de arquivos para o ambiente de desktop KDE para Unix, e só faz sentido usar se estiver no KDE. Alguma maneira de detectar o KDE de forma confiável seria legal; a variável **`KDEDIR`** não é suficiente. Note também que o nome “kfm” é usado mesmo ao usar o comando **konqueror** com o KDE 2 — a implementação seleciona a melhor estratégia para executar o Konqueror.
2. Somente em plataformas Windows.
3. Somente no macOS.
4. Somente no iOS.

Adicionado na versão 3.2: Uma nova classe **`MacOSXOSAScript`** foi adicionada e é usada no Mac em vez da classe anterior **`MacOSX`**. Isso adiciona suporte para abrir navegadores que não estão definidos como padrão do sistema operacional no momento.

Adicionado na versão 3.3: Foi adicionado suporte para Chrome/Chromium.

Alterado na versão 3.12: O suporte para vários navegadores obsoletos foi removido. Os navegadores removidos incluem Grail, Mosaic, Netscape,

Galeon, Skipstone, Iceape e Firefox versões 35 e abaixo.

Alterado na versão 3.13: Foi adicionado suporte para iOS.

Adicionado na versão 3.15: **`MacOS`** has been added as a replacement for **`MacOSXOSAScript`**, opening browsers via **/usr/bin/open** instead of **osascript**.

Descontinuado desde a versão 3.15, será removido na versão 3.17: **`MacOSXOSAScript`** is deprecated in favour of **`MacOS`**. Using **/usr/bin/open** instead of **osascript** is a security and usability improvement: **osascript** may be blocked on managed systems due to its abuse potential as a general-purpose scripting interpreter.

Aqui estão alguns exemplos simples:

```
url = 'https://docs.python.org/'

# Abre a URL em uma nova aba, se uma janela do navegador já esti
webbrowser.open_new_tab(url)

# Abre a URL em uma nova janela, aumentando a janela se possível
webbrowser.open_new(url)
```

Objetos controladores de navegador

Os controladores de navegador fornecem o atributo **`name`** e os três métodos a seguir que são paralelos às funções de conveniência em nível de módulo:

controller.name

Nome dependente do sistema para o navegador.

controller.open(*url*, *new=0*, *autoraise=True*)

Exibe *url* usando o navegador manipulado por este controlador. Se *new* for 1, uma nova janela do navegador será aberta, se possível. Se *new* for 2, uma nova página do navegador (“aba” ou “guia”) será aberta, se possível.

controller.open_new(*url*)

Abre *url* em uma nova janela do navegador manipulado por este controlador, se possível, caso contrário, abra *url* na única janela do navegador. Apelido para `open_new()`.

controller.open_new_tab(*url*)

Abre *url* em uma nova página (“aba” ou “guia”) do navegador manipulado por este controlador, se possível, caso contrário, equivalente a `open_new()`.

Notas de rodapé

[1]

Executáveis nomeados aqui sem um caminho completo serão pesquisados nos diretórios fornecidos na variável de ambiente `PATH`.

`wsgiref` — Implementação de referência e utilitários WSGI

Código-fonte: Lib/wsgiref [https://github.com/python/cpython/tree/3.15/Lib/wsgiref]

Aviso

`wsgiref` is a reference implementation and is not recommended for production. The module only implements basic security checks.

A Interface de Gateway para Servidor Web (em inglês, Web Server Gateway Interface - WSGI) é uma interface padrão localizada entre software de servidores web e aplicações web escritas em Python. Ter um interface padrão torna mais fácil a utilização de uma aplicação que suporta WSGI com inúmeros diferentes servidores web.

Apenas autores de servidores web e frameworks de programação necessitam saber todos os detalhes e especificidades do design WSGI. Você não precisa entender todos os detalhes do WSGI para apenas instalar uma aplicação WSGI ou para escrever uma aplicação web usando um framework existente.

`wsgiref` is a reference implementation of the WSGI specification that can be used to add WSGI support to a web server or framework. It provides utilities for manipulating WSGI environment variables and response headers, base classes for implementing WSGI servers, a demo HTTP server that serves WSGI applications, types for static type checking, and a validation tool that checks WSGI servers and applications for conformance to the WSGI specification (**PEP 3333** [https://peps.python.org/pep-3333/]).

Veja wsgi.readthedocs.io [https://wsgi.readthedocs.io/] para mais informações sobre WSGI, além de links para tutoriais e outros recursos.

`wsgiref.util` – WSGI environment utilities

Este módulo fornece uma variedade de funções de utilidade para trabalhar com ambientes WSGI. Um ambiente WSGI é um dicionário contendo variáveis de solicitação HTTP, conforme descrito na **PEP 3333** [https://peps.python.org/pep-3333/]. Todas as funções que recebem um parâmetro *environ* esperam que um dicionário compatível com WSGI seja fornecido; consulte a **PEP 3333** [https://peps.python.org/pep-3333/] para uma especificação detalhada e `WSGIEnvironment` para um apelido de tipo que pode ser usado em anotações de tipos.

wsgiref.util.guess_scheme(*environ*)

Retorna uma sugestão sobre `wsgi.url_scheme` ser “http” ou “https” buscando por uma variável de ambiente `HTTPS` dentro do dicionário *environ*. O valor de retorno é uma string.

Esta função é útil ao criar um gateway que encapsula CGI ou um protocolo semelhante a CGI, como FastCGI. Normalmente, servidores que fornecem tais protocolos incluirão uma variável `HTTPS` com um valor de “1”, “yes” ou “on” quando uma solicitação é recebida via SSL. Então, esta função retorna “https” se tal valor for encontrado, e “http” caso contrário.

wsgiref.util.request_uri(*environ*, *include_query=True*)

Retorna o URI de solicitação completo, opcionalmente incluindo a string de consulta, usando o algoritmo encontrado na seção “URL Reconstruction” da **PEP 3333** [https://peps.python.org/pep-3333/]. Se *include_query* for falso, a string de consulta não será incluída no URI resultante.

wsgiref.util.application_uri(*environ*)

Semelhante a `request_uri()`, exceto que as variáveis `PATH_INFO` e `QUERY_STRING` são ignoradas. O resultado é o URI base do objeto da aplicação endereçado pela solicitação.

wsgiref.util.shift_path_info(*environ*)

Desloca um único nome de `PATH_INFO` para `SCRIPT_NAME` e retorna o nome. O dicionário *environ* é *modificado* no local; use uma cópia se precisar manter o `PATH_INFO` ou `SCRIPT_NAME` original intacto.

Se não houver segmentos de caminho restantes em `PATH_INFO`, `None` será retornado.

Normalmente, essa rotina é usada para processar cada porção de um caminho de URI de solicitação, por exemplo, para tratar o caminho como uma série de chaves de dicionário. Essa rotina modifica o ambiente passado para torná-lo adequado para invocar outra aplicação WSGI localizada no URI de destino. Por exemplo, se houver uma aplicação WSGI em `/foo`, e o caminho de URI de solicitação for `/foo/bar/baz`, e a aplicação WSGI em `/foo` chamar **`shift_path_info()`**, ele receberá a string "bar", e o ambiente será atualizado para ser adequado para passar para uma aplicação WSGI em `/foo/bar`. Ou seja, `SCRIPT_NAME` mudará de `/foo` para `/foo/bar`, e `PATH_INFO` mudará de `/bar/baz` para `/baz`.

Quando `PATH_INFO` é apenas um "/", esta rotina retorna uma string vazia e acrescenta uma barra final a `SCRIPT_NAME`, embora segmentos de caminho vazios sejam normalmente ignorados, e `SCRIPT_NAME` normalmente não termine em uma barra. Este é um comportamento intencional, para garantir que uma aplicação possa diferenciar URIs terminando em `/x` daqueles terminando em `/x/` ao usar esta rotina para fazer travessia de objetos.

wsgiref.util.setup_testing_defaults(*environ*)

Atualiza *environ* com padrões triviais para fins de teste.

Esta rotina adiciona vários parâmetros necessários para WSGI, incluindo `HTTP_HOST`, `SERVER_NAME`, `SERVER_PORT`, `REQUEST_METHOD`, `SCRIPT_NAME`, `PATH_INFO` e todas as variáveis `wsgi.*` definidas pela **PEP 3333** [https://peps.python.org/pep-3333/]. Ela fornece apenas valores padrão e não substitui nenhuma configuração existente para essas variáveis.

Esta rotina tem como objetivo facilitar que testes unitários de servidores e aplicações WSGI configurem ambientes fictícios. Ela NÃO deve ser usada por servidores ou aplicações WSGI reais, pois os dados são falsos!

Exemplo de uso (veja também **demo_app()** para outro exemplo):

```
from wsgiref.util import setup_testing_defaults
from wsgiref.simple_server import make_server

# Uma aplicação WSGI relativamente simples. Vai exibir o dici
# do ambiente após ser atualizada por setup_testing_defaults
def simple_app(environ, start_response):
    setup_testing_defaults(environ)

    status = '200 OK'
    headers = [('Content-type', 'text/plain; charset=utf-8')]

    start_response(status, headers)

    ret = [("%s: %s\n" % (key, value)).encode("utf-8")
           for key, value in environ.items()]
    return ret

with make_server('', 8000, simple_app) as httpd:
    print("Serving on port 8000...")
    httpd.serve_forever()
```

In addition to the environment functions above, the **wsgiref.util** module also provides these miscellaneous utilities:

wsgiref.util.is_hop_by_hop(*header_name*)

Retorna `True` se ‘header_name’ for um cabeçalho HTTP/1.1 “Hop-by-Hop”, conforme definido por **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html].

class wsgiref.util.FileWrapper(*filelike*, *blksize=8192*)

Uma implementação concreta do protocolo **wsgiref.types.FileWrapper** usado para converter um objeto arquivo

ou similar em um iterador. Os objetos resultantes são iteráveis. Conforme o objeto é iterado, o parâmetro opcional *blksize* será repetidamente passado para o método **`read()`** do objeto *filelike* para obter bytestrings para produzir. Quando **`read()`** retorna uma string de bytes vazia, a iteração é encerrada e não pode ser retomada.

Se *filelike* tiver um método **`close()`**, o objeto retornado também terá um método **`close()`** e invocará o método **`close()`** do objeto *filelike* quando chamado.

Exemplo de uso:

```
from io import StringIO
from wsgiref.util import FileWrapper

# Estamos usando um StringIO-buffer como objeto arquivo ou si
filelike = StringIO("This is an example file-like object"*10)
wrapper = FileWrapper(filelike, blksize=5)

for chunk in wrapper:
    print(chunk)
```

Alterado na versão 3.11: O suporte para o método **`__getitem__()`** foi removido

`wsgiref.headers` – WSGI response header tools

Este módulo fornece uma única classe, **`Headers`**, para manipulação conveniente de cabeçalhos de resposta WSGI usando uma interface semelhante a mapeamento.

class wsgiref.headers.Headers([*headers*])

Cria um objeto mapeamento envolvendo *headers*, que deve ser uma lista de tuplas de nome/valor de cabeçalho, conforme descrito na **PEP 3333** [https://peps.python.org/pep-3333/]. O valor padrão de *headers* é uma lista vazia.

Os objetos **Headers** oferecem suporte a operações de mapeamento típicas, incluindo **__getitem__()**, **get()**, **__setitem__()**, **setdefault()**, **__delitem__()** e **__contains__()**. Para cada um desses métodos, a chave é o nome do cabeçalho (tratado sem distinção entre maiúsculas e minúsculas) e o valor é o primeiro valor associado a esse nome de cabeçalho. Definir um cabeçalho exclui quaisquer valores existentes para esse cabeçalho e, em seguida, adiciona um novo valor ao final da lista de cabeçalhos encapsulados. A ordem existente dos cabeçalhos geralmente é mantida, com novos cabeçalhos adicionados ao final da lista encapsulada.

Diferentemente de um dicionário, objetos **Headers** não levantam um erro quando você tenta obter ou excluir uma chave que não está na lista de cabeçalhos encapsulados. Obter um cabeçalho inexistente retorna apenas `None`, e excluir um cabeçalho inexistente não faz nada.

Os objetos **Headers** também oferecem suporte aos métodos **keys()**, **values()** e **items()**. As listas retornadas por **keys()** e **items()** podem incluir a mesma chave mais de uma vez se houver um cabeçalho multivalorado. O `len()` de um objeto **Headers** é o mesmo que o comprimento de seus **items()**, que é o mesmo que o comprimento da lista de cabeçalhos encapsulados. Na verdade, o método **items()** apenas retorna uma cópia da lista de cabeçalhos encapsulados.

Chamar `bytes()` em um objeto **Headers** retorna uma bytestring formatada adequada para transmissão como cabeçalhos de resposta HTTP. Cada cabeçalho é colocado em uma linha com seu valor, separado por dois pontos e um espaço. Cada linha é terminada por um retorno de carro e uma quebra de linha, e a bytestring é terminada com uma linha em branco.

Além de sua interface de mapeamento e recursos de formatação, os objetos **Headers** também têm os seguintes métodos para consultar e adicionar cabeçalhos multivalorados e para adicionar cabeçalhos com parâmetros MIME:

get_all(*name*)

Retorna uma lista de todos os valores para o cabeçalho nomeado.

A lista retornada será classificada na ordem em que apareceu na lista de cabeçalho original ou foi adicionada a esta instância, e pode conter duplicatas. Quaisquer campos excluídos e reinseridos são sempre anexados à lista de cabeçalho. Se não houver campos com o nome fornecido, retorna uma lista vazia.

add_header(*name*, *value*, ***_params*)

Adiciona um cabeçalho (possivelmente multivalorado), com parâmetros MIME opcionais especificados por meio de argumentos nomeados.

name é o campo de cabeçalho a ser adicionado. Argumentos nomeados podem ser usados para definir parâmetros MIME para o campo de cabeçalho. Cada parâmetro deve ser uma string ou `None`. Sublinhados em nomes de parâmetros são convertidos em traços, já que traços são ilegais em identificadores Python, mas muitos nomes de parâmetros MIME incluem traços. Se o valor do parâmetro for uma string, ele será adicionado aos parâmetros de valor do cabeçalho no formato `name="value"`. Se for `None`, somente o nome do parâmetro será adicionado. (Isso é usado para parâmetros MIME sem um valor.) Exemplo de uso:

```
h.add_header('content-disposition', 'attachment', filename
```

O exemplo acima adicionará um cabeçalho parecido com este:

```
Content-Disposition: attachment; filename="bud.gif"
```

Alterado na versão 3.5: o parâmetro *headers* é opcional.

`wsgiref.simple_server` – a simple WSGI HTTP server

Este módulo implementa um servidor HTTP simples (com base em `http.server`) que serve aplicações WSGI. Cada instância de servidor serve

uma única aplicação WSGI em um host e porta fornecidos. Se você quiser servir várias aplicações em um único host e porta, você deve criar uma aplicação WSGI que analise `PATH_INFO` para selecionar qual aplicação invocar para cada solicitação. (Por exemplo, usando a função **`shift_path_info()`** de **`wsgiref.util`**.)

wsgiref.simple_server.make_server(*host*, *port*, *app*, *server_class=WSGIServer*, *handler_class=WSGIRequestHandler*)

Cria um novo servidor WSGI escutando em *host* e *port*, aceitando conexões para *app*. O valor de retorno é uma instância da *server_class* fornecida e processará solicitações usando a *handler_class* especificada. *app* deve ser um objeto de aplicação WSGI, conforme definido pela **PEP 3333** [https://peps.python.org/pep-3333/].

Exemplo de uso:

```
from wsgiref.simple_server import make_server, demo_app

with make_server('', 8000, demo_app) as httpd:
    print("Serving HTTP on port 8000...")

    # Responde a solicitações até o processo ser encerrado
    httpd.serve_forever()

    # Alternativa: serve uma solicitação e então sai
    httpd.handle_request()
```

wsgiref.simple_server.demo_app(*environ*, *start_response*)

This function is a small but complete WSGI application that returns a text page containing the message “Hello world!” and a list of the key/value pairs provided in the *environ* parameter. It’s useful for verifying that a WSGI server (such as **`wsgiref.simple_server`**) is able to run a simple WSGI application correctly.

O chamável *start_response* deve seguir o protocolo **`StartResponse`**.

class wsgiref.simple_server.WSGIServer(*server_address*, *RequestHandlerClass*)

Cria uma instância `WSGIServer`. *server_address* deve ser uma tupla `(host,port)` e *RequestHandlerClass* deve ser a subclasse de `http.server.BaseHTTPRequestHandler` que será usada para processar solicitações.

Normalmente você não precisa chamar esse construtor, pois a função `make_server()` pode cuidar de todos os detalhes para você.

`WSGIServer` é uma subclasse de `http.server.HTTPServer`, então todos os seus métodos (como **`serve_forever()`** e **`handle_request()`**) estão disponíveis. **`WSGIServer`** também fornece estes métodos específicos do WSGI:

set_app(*application*)

Define o chamável *application* como a aplicação WSGI que receberá solicitações.

get_app()

Retorna o chamável da aplicação definido atualmente.

Normalmente, no entanto, você não precisa usar esses métodos adicionais, pois `set_app()` é normalmente chamado por `make_server()`, e `get_app()` existe principalmente para o benefício de instâncias do manipulador de solicitações.

class wsgiref.simple_server.WSGIRequestHandler(*request*, *client_address*, *server*)

Cria um manipulador HTTP para *request* fornecido (isto é, um soquete), *client_address* (uma tupla `(host,port)`) e *server* (instância `WSGIServer`).

Você não precisa criar instâncias desta classe diretamente; elas são criadas automaticamente conforme necessário pelos objetos `WSGIServer`. Você pode, no entanto, estender esta classe e fornecê-la como uma *handler_class* para a função `make_server()`. Alguns métodos possivelmente relevantes para substituir em subclasses:

get_environ()

Retorna um dicionário **WSGIEnvironment** para uma solicitação. A implementação padrão copia o conteúdo do atributo de dicionário **base_environ** do objeto **WSGIServer** e então adiciona vários cabeçalhos derivados da solicitação HTTP. Cada chamada para esse método deve retornar um novo dicionário contendo todas as variáveis de ambiente CGI relevantes conforme especificado na **PEP 3333** [https://peps.python.org/pep-3333/].

get_stderr()

Retorna o objeto que deve ser usado como o fluxo `wsgi.errors`. A implementação padrão retorna apenas `sys.stderr`.

handle()

Processa a solicitação HTTP. A implementação padrão cria uma instância manipuladora usando uma classe **wsgiref.handlers** para implementar a interface da aplicação WSGI.

wsgiref.validate — WSGI conformance checker

When creating new WSGI application objects, frameworks, servers, or middleware, it can be useful to validate the new code's conformance using **wsgiref.validate**. This module provides a function that creates WSGI application objects that validate communications between a WSGI server or gateway and a WSGI application object, to check both sides for protocol conformance.

Note que este utilitário não garante a conformidade completa com **PEP 3333** [https://peps.python.org/pep-3333/]; uma ausência de erros deste módulo não significa necessariamente que os erros não existam. No entanto, se este módulo produzir um erro, então é virtualmente certo que o servidor ou a aplicação não está 100% em conformidade.

Este módulo é baseado no módulo **paste.lint** da biblioteca "Python Paste" de Ian Bicking.

wsgiref.validate.validator(*application*)

Encapsula *application* e retorna um novo objeto de aplicação WSGI. A aplicação retornada encaminhará todas as solicitações para a *application* original e verificará se tanto a *application* quanto o servidor que o invoca estão em conformidade com a especificação WSGI e com **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html].

Qualquer falta de conformidade detectada resulta em uma `AssertionError` sendo levantado; note, entretanto, que a forma como esses erros são manipulados depende do servidor. Por exemplo, `wsgiref.simple_server` e outros servidores baseados em `wsgiref.handlers` (que não substituem os métodos de manipulação de erros para fazer outra coisa) simplesmente emitirão uma mensagem de que ocorreu um erro e despejarão o traceback para `sys.stderr` ou algum outro fluxo de erro.

Este invólucro também pode gerar saída usando o módulo `warnings` para indicar comportamentos que são questionáveis, mas que podem não ser realmente proibidos por **PEP 3333** [https://peps.python.org/pep-3333/]. A menos que sejam suprimidos usando opções de linha de comando do Python ou a API **`warnings`**, quaisquer avisos serão gravados em `sys.stderr` (*não* `wsgi.errors`, a menos que sejam o mesmo objeto).

Exemplo de uso:

```
from wsgiref.validate import validator
from wsgiref.simple_server import make_server

# Nosso objeto chamável que intencionalmente não está em
# conformidade com o padrão, então o validador irá quebrar
def simple_app(environ, start_response):
    status = '200 OK'  # HTTP Status
    headers = [('Content-type', 'text/plain')]  # HTTP Header
    start_response(status, headers)

    # Isso vai quebrar porque precisamos retornar uma lista,
    # e o validador vai nos informar
    return b"Hello World"
```

```
# Este é a aplicação encapsulada em um validador
validator_app = validator(simple_app)

with make_server('', 8000, validator_app) as httpd:
    print("Listening on port 8000....")
    httpd.serve_forever()
```

`wsgiref.handlers` – server/gateway base classes

Este módulo fornece classes de manipulador base para implementar servidores e gateways WSGI. Essas classes base lidam com a maior parte do trabalho de comunicação com uma aplicação WSGI, desde que recebam um ambiente semelhante ao CGI, junto com fluxos de entrada, saída e erro.

class wsgiref.handlers.CGIHandler

Invocação baseada em CGI via `sys.stdin`, `sys.stdout`, `sys.stderr` e `os.environ`. Isso é útil quando você tem uma aplicação WSGI e quer executá-lo como um script CGI. Basta invocar `CGIHandler().run(app)`, onde `app` é o objeto da aplicação WSGI que você deseja invocar.

Esta classe é uma subclasse de **`BaseCGIHandler`** que define `wsgi.run_once` como true, `wsgi.multithread` como false e `wsgi.multiprocess` como true, e sempre usa **`sys`** e **`os`** para obter os fluxos CGI e o ambiente necessários.

class wsgiref.handlers.IISCGIHandler

Uma alternativa especializada para **`CGIHandler`**, para uso ao implantar no servidor web IIS da Microsoft, sem ter definido a opção de configuração allowPathInfo (IIS>=7) ou metabase allowPathInfoForScriptMappings (IIS<7).

Por padrão, o IIS fornece um `PATH_INFO` que duplica o `SCRIPT_NAME` na frente, causando problemas para aplicações WSGI que desejam implementar roteamento. Este manipulador remove qualquer caminho duplicado.

O IIS pode ser configurado para passar o `PATH_INFO` correto, mas isso causa outro bug onde `PATH_TRANSLATED` está errado. Felizmente, essa variável raramente é usada e não é garantida pelo WSGI. No IIS<7, no entanto, a configuração só pode ser feita em um nível de vhost, afetando todos os outros mapeamentos de script, muitos dos quais quebram quando expostos ao bug `PATH_TRANSLATED`. Por esse motivo, o IIS<7 quase nunca é implantado com a correção (mesmo o IIS7 raramente a usa porque ainda não há uma UI para ela).

Não há como o código CGI dizer se a opção foi definida, então uma classe de manipulador separada é fornecida. Ela é usada da mesma forma que **`CGIHandler`**, ou seja, chamando `IISCGIHandler().run(app)`, onde `app` é o objeto de aplicação WSGI que você deseja invocar.

Adicionado na versão 3.2.

class wsgiref.handlers.BaseCGIHandler(*stdin*, *stdout*, *stderr*, *environ*, *multithread=True*, *multiprocess=False*)

Similar to **`CGIHandler`**, but instead of using the **`sys`** and **`os`** modules, the CGI environment and I/O streams are specified explicitly. The *multithread* and *multiprocess* values are used to set the `wsgi.multithread` and `wsgi.multiprocess` flags for any applications run by the handler instance.

This class is a subclass of **`SimpleHandler`** intended for use with software other than HTTP "origin servers". If you are writing a gateway protocol implementation (such as CGI, FastCGI, SCGI, etc.) that uses a `Status:` header to send an HTTP status, you probably want to subclass this instead of **`SimpleHandler`**.

class wsgiref.handlers.SimpleHandler(*stdin*, *stdout*, *stderr*, *environ*, *multithread=True*, *multiprocess=False*)

Similar to **`BaseCGIHandler`**, but designed for use with HTTP origin servers. If you are writing an HTTP server implementation, you will probably want to subclass this instead of **`BaseCGIHandler`**.

This class is a subclass of `BaseHandler`. It overrides the `__init__()`, `get_stdin()`, `get_stderr()`, `add_cgi_vars()`, `_write()`, and `_flush()` methods to support explicitly setting the environment and streams via the constructor. The supplied environment and streams are stored in the `stdin`, `stdout`, `stderr`, and `environ` attributes.

The `write()` method of *stdout* should write each chunk in full, like `io.BufferedIOBase`.

class wsgiref.handlers.BaseHandler

This is an abstract base class for running WSGI applications. Each instance will handle a single HTTP request, although in principle you could create a subclass that was reusable for multiple requests.

`BaseHandler` instances have only one method intended for external use:

run(*app*)

Run the specified WSGI application, *app*.

All of the other `BaseHandler` methods are invoked by this method in the process of running the application, and thus exist primarily to allow customizing the process.

The following methods MUST be overridden in a subclass:

_write(*data*)

Buffer the bytes *data* for transmission to the client. It's okay if this method actually transmits the data; `BaseHandler` just separates write and flush operations for greater efficiency when the underlying system actually has such a distinction.

_flush()

Force buffered data to be transmitted to the client. It's okay if this method is a no-op (i.e., if `_write()` actually sends the data).

get_stdin()

Return an object compatible with **InputStream** suitable for use as the `wsgi.input` of the request currently being processed.

get_stderr()

Return an object compatible with **ErrorStream** suitable for use as the `wsgi.errors` of the request currently being processed.

add_cgi_vars()

Insert CGI variables for the current request into the **environ** attribute.

Here are some other methods and attributes you may wish to override. This list is only a summary, however, and does not include every method that can be overridden. You should consult the docstrings and source code for additional information before attempting to create a customized **BaseHandler** subclass.

Attributes and methods for customizing the WSGI environment:

wsgi_multithread

The value to be used for the `wsgi.multithread` environment variable. It defaults to true in **BaseHandler**, but may have a different default (or be set by the constructor) in the other subclasses.

wsgi_multiprocess

The value to be used for the `wsgi.multiprocess` environment variable. It defaults to true in **BaseHandler**, but may have a different default (or be set by the constructor) in the other subclasses.

wsgi_run_once

The value to be used for the `wsgi.run_once` environment variable. It defaults to false in **BaseHandler**, but **CGIHandler** sets it to true by default.

os_environ

The default environment variables to be included in every request's WSGI environment. By default, this is a copy of `os.environ` at the

time that **wsgiref.handlers** was imported, but subclasses can either create their own at the class or instance level. Note that the dictionary should be considered read-only, since the default value is shared between multiple classes and instances.

server_software

If the origin_server attribute is set, this attribute's value is used to set the default SERVER_SOFTWARE WSGI environment variable, and also to set a default Server: header in HTTP responses. It is ignored for handlers (such as BaseCGIHandler and CGIHandler) that are not HTTP origin servers.

Alterado na versão 3.3: O termo "Python" é substituído por termos específicos de implementação, como "CPython", "Jython" etc.

get_scheme()

Return the URL scheme being used for the current request. The default implementation uses the **guess_scheme()** function from wsgiref.util to guess whether the scheme should be "http" or "https", based on the current request's **environ** variables.

setup_environ()

Set the **environ** attribute to a fully populated WSGI environment. The default implementation uses all of the above methods and attributes, plus the get_stdin(), get_stderr(), and add_cgi_vars() methods and the wsgi_file_wrapper attribute. It also inserts a SERVER_SOFTWARE key if not present, as long as the origin_server attribute is a true value and the server_software attribute is set.

Methods and attributes for customizing exception handling:

log_exception(*exc_info*)

Log the *exc_info* tuple in the server log. *exc_info* is a (type, value, traceback) tuple. The default implementation simply writes the traceback to the request's wsgi.errors stream and flushes it. Subclasses can override this method to change the format or

retarget the output, mail the traceback to an administrator, or whatever other action may be deemed suitable.

traceback_limit

The maximum number of frames to include in tracebacks output by the default **`log_exception()`** method. If `None`, all frames are included.

error_output(*environ*, *start_response*)

This method is a WSGI application to generate an error page for the user. It is only invoked if an error occurs before headers are sent to the client.

This method can access the current error using `sys.exception()`, and should pass that information to *start_response* when calling it (as described in the "Error Handling" section of **PEP 3333** [https://peps.python.org/pep-3333/]). In particular, the *start_response* callable should follow the **`StartResponse`** protocol.

The default implementation just uses the **`error_status`**, **`error_headers`**, and **`error_body`** attributes to generate an output page. Subclasses can override this to produce more dynamic error output.

Note, however, that it's not recommended from a security perspective to spit out diagnostics to any old user; ideally, you should have to do something special to enable diagnostic output, which is why the default implementation doesn't include any.

error_status

The HTTP status used for error responses. This should be a status string as defined in **PEP 3333** [https://peps.python.org/pep-3333/]; it defaults to a 500 code and message.

error_headers

The HTTP headers used for error responses. This should be a list of WSGI response headers (`(name, value)` tuples), as described in

PEP 3333 [https://peps.python.org/pep-3333/]. The default list just sets the content type to `text/plain`.

error_body

The error response body. This should be an HTTP response body bytestring. It defaults to the plain text, "A server error occurred. Please contact the administrator."

Methods and attributes for **PEP 3333** [https://peps.python.org/pep-3333/]'s "Optional Platform-Specific File Handling" feature:

wsgi_file_wrapper

A `wsgi.file_wrapper` factory, compatible with **`wsgiref.types.FileWrapper`**, or `None`. The default value of this attribute is the **`wsgiref.util.FileWrapper`** class.

sendfile()

Override to implement platform-specific file transmission. This method is called only if the application's return value is an instance of the class specified by the **`wsgi_file_wrapper`** attribute. It should return a true value if it was able to successfully transmit the file, so that the default transmission code will not be executed. The default implementation of this method just returns a false value.

Miscellaneous methods and attributes:

origin_server

This attribute should be set to a true value if the handler's **`_write()`** and **`_flush()`** are being used to communicate directly to the client, rather than via a CGI-like gateway protocol that wants the HTTP status in a special `Status:` header.

This attribute's default value is true in **`BaseHandler`**, but false in **`BaseCGIHandler`** and **`CGIHandler`**.

http_version

If `origin_server` is true, this string attribute is used to set the HTTP version of the response set to the client. It defaults to `"1.0"`.

wsgiref.handlers.read_environ()

Transcode CGI variables from `os.environ` to **PEP 3333** [https://peps.python.org/pep-3333/] "bytes in unicode" strings, returning a new dictionary. This function is used by `CGIHandler` and `IISCGIHandler` in place of directly using `os.environ`, which is not necessarily WSGI-compliant on all platforms and web servers using Python 3 – specifically, ones where the OS's actual environment is Unicode (i.e. Windows), or ones where the environment is bytes, but the system encoding used by Python to decode it is anything other than ISO-8859-1 (e.g. Unix systems using UTF-8).

If you are implementing a CGI-based handler of your own, you probably want to use this routine instead of just copying values out of `os.environ` directly.

Adicionado na versão 3.2.

`wsgiref.types` – WSGI types for static type checking

This module provides various types for static type checking as described in **PEP 3333** [https://peps.python.org/pep-3333/].

Adicionado na versão 3.11.

class wsgiref.types.StartResponse

A `typing.Protocol` describing **start_response()** [https://peps.python.org/pep-3333/#the-start-response-callable] callables (**PEP 3333** [https://peps.python.org/pep-3333/]).

wsgiref.types.WSGIEnvironment

A type alias describing a WSGI environment dictionary.

wsgiref.types.WSGIApplication

A type alias describing a WSGI application callable.

class wsgiref.types.InputStream

A `typing.Protocol` describing a **WSGI Input Stream** [https://peps.python.org/pep-3333/#input-and-error-streams].

class wsgiref.types.ErrorStream

A `typing.Protocol` describing a **WSGI Error Stream** [https://peps.python.org/pep-3333/#input-and-error-streams].

class wsgiref.types.FileWrapper

A `typing.Protocol` describing a **file wrapper** [https://peps.python.org/pep-3333/#optional-platform-specific-file-handling]. See `wsgiref.util.FileWrapper` for a concrete implementation of this protocol.

Exemplos

This is a working "Hello World" WSGI application, where the *start_response* callable should follow the `StartResponse` protocol:

```
"""
Every WSGI application must have an application object - a calla
object that accepts two arguments. For that purpose, we're going
use a function (note that you're not limited to a function, you
use a class for example). The first argument passed to the funct
is a dictionary containing CGI-style environment variables and t
second variable is the callable object.
"""
from wsgiref.simple_server import make_server


def hello_world_app(environ, start_response):
    status = "200 OK"  # HTTP Status
    headers = [("Content-type", "text/plain; charset=utf-8")]  #
    start_response(status, headers)

    # The returned object is going to be printed
    return [b"Hello World"]
```

```
with make_server("", 8000, hello_world_app) as httpd:
    print("Serving on port 8000...")

    # Serve until process is killed
    httpd.serve_forever()
```

Example of a WSGI application serving the current directory, accept optional directory and port number (default: 8000) on the command line:

```
"""
Small wsgiref based web server. Takes a path to serve from and a
optional port number (defaults to 8000), then tries to serve fil
MIME types are guessed from the file names, 404 errors are raise
if the file is not found.
"""
import mimetypes
import os
import sys
from wsgiref import simple_server, util


def app(environ, respond):
    # Get the file name and MIME type
    fn = os.path.join(path, environ["PATH_INFO"][1:])
    if "." not in fn.split(os.path.sep)[-1]:
        fn = os.path.join(fn, "index.html")
    mime_type = mimetypes.guess_file_type(fn)[0]

    # Return 200 OK if file exists, otherwise 404 Not Found
    if os.path.exists(fn):
        respond("200 OK", [("Content-Type", mime_type)])
        return util.FileWrapper(open(fn, "rb"))
    else:
        respond("404 Not Found", [("Content-Type", "text/plain")
        return [b"not found"]


if __name__ == "__main__":
    # Get the path and port from command-line arguments
    path = sys.argv[1] if len(sys.argv) > 1 else os.getcwd()
    port = int(sys.argv[2]) if len(sys.argv) > 2 else 8000
```

```
    # Make and start the server until control-c
    httpd = simple_server.make_server("", port, app)
    print(f"Serving {path} on port {port}, control-C to stop")
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print("Shutting down.")
        httpd.server_close()
```

`urllib` — Módulos de manipulação de URL

Código-fonte: Lib/urllib/ [https://github.com/python/cpython/tree/3.15/Lib/urllib/]

`urllib` é um pacote que coleciona vários módulos para trabalhar com URLs:

- **`urllib.request`** para abrir e ler URLs
- **`urllib.error`** contendo as exceções levantadas por **`urllib.request`**
- **`urllib.parse`** para analisar URLs
- **`urllib.robotparser`** para analisar arquivos `robots.txt`

`urllib.request` — Biblioteca extensível para abrir URLs

Código-fonte: Lib/urllib/request.py
[https://github.com/python/cpython/tree/3.15/Lib/urllib/request.py]

The **`urllib.request`** module defines functions and classes which help in opening URLs (mostly HTTP) in a complex world — basic and digest authentication, redirections, cookies and more.

Ver também

O pacote Requests [https://requests.readthedocs.io/en/master/] é recomendado para uma interface alto nível de cliente HTTP.

Aviso

No macOS, não é seguro usar este módulo em programas que utilizam **`os.fork()`** porque a implementação **`getproxies()`** para macOS utiliza uma API de sistema de nível mais alto. Defina a variável de ambiente `no_proxy` como `*` para evitar esse problema (por exemplo, `os.environ["no_proxy"] = "*"`).

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

The **`urllib.request`** module defines the following functions:

urllib.request.urlopen(*url*, *data=None*, [*timeout*,]*, *context=None*)

Abre *url*, que pode ser uma string contendo uma URL válida e devidamente codificada, ou um objeto `Request`.

data deve ser um objeto que especifique dados adicionais a serem enviados ao servidor ou `None`, se nenhum dado for necessário. Veja `Request` para detalhes.

O módulo urllib.request usa HTTP/1.1 e inclui o cabeçalho `Connection:close` em suas solicitações HTTP.

O parâmetro opcional *timeout* especifica um tempo limite em segundos para bloquear operações como a tentativa de conexão (se não for especificado, a configuração de tempo limite padrão global será usada). Na verdade, isso só funciona para conexões HTTP, HTTPS e FTP.

Se *context* for especificado, deve ser uma instância de `ssl.SSLContext` descrevendo as várias opções SSL. Veja `HTTPSConnection` para mais detalhes.

Esta função sempre retorna um objeto que pode funcionar como um gerenciador de contexto e tem as propriedades *url*, *headers* e *status*. Veja `urllib.response.addinfourl` para mais detalhes sobre essas propriedades.

Para URLs HTTP e HTTPS, esta função retorna um objeto `http.client.HTTPResponse` ligeiramente modificado. Além dos três novos métodos acima, o atributo msg contém as mesmas informações que o atributo `reason` — a frase de razão retornada pelo servidor — em vez dos cabeçalhos de resposta como é especificado na documentação para `HTTPResponse`.

Para URLs de FTP, arquivo e dados, esta função retorna um objeto `urllib.response.addinfourl`.

Levanta `URLError` quando ocorrer erros de protocolo.

Observe que `None` pode ser retornado se nenhum manipulador lidar com a solicitação (embora a `OpenerDirector` global instalada padrão use

UnknownHandler para garantir que isso nunca aconteça).

Além disso, se as configurações de proxy forem detectadas (por exemplo, quando uma variável de ambiente `*_proxy` como **`http_proxy`** é definida), **ProxyHandler** é instalado por padrão e garante que as solicitações sejam manipuladas pelo proxy.

A função legada `urllib.urlopen` do Python 2.6 e anteriores foi descontinuada; **urllib.request.urlopen()** corresponde à antiga `urllib2.urlopen`. O tratamento de proxy, que foi feito passando um parâmetro de dicionário para `urllib.urlopen`, pode ser obtido usando objetos **ProxyHandler**.

O abridor padrão levanta um evento de auditoria `urllib.Request` com os argumentos `fullurl`, `data`, `headers`, `method` obtidos do objeto de requisição.

Alterado na versão 3.2: cafile e *capath* foram adicionados.

Hosts virtuais HTTPS agora são suportados, se possível (ou seja, se **ssl.HAS_SNI** for verdadeiro).

data pode ser um objeto iterável.

Alterado na versão 3.3: cadefault foi adicionado.

Alterado na versão 3.4.3: context foi adicionado.

Alterado na versão 3.10: A conexão HTTPS agora envia uma extensão ALPN com o indicador de protocolo `http/1.1` quando nenhum *context* é fornecido. *context* personalizado deve definir protocolos ALPN com **set_alpn_protocols()**.

Alterado na versão 3.13: Remove os parâmetros *cafile*, *capath* e *cadefault*: use o parâmetro *context* em seu lugar.

urllib.request.install_opener(*opener*)

Instala uma instância de **OpenerDirector** como o abridor global padrão. A instalação de um abridor só é necessária se você quiser que o urlopen o utilize; caso contrário, basta chamar **OpenerDirector.open()** em vez de **urlopen()**. O código não verifica se há um **OpenerDirector** real, e qualquer classe com a interface apropriada funcionará.

urllib.request.build_opener([*handler*, ...])

Return an **OpenerDirector** instance, which chains the handlers in the order given. *handler*s can be either instances of **BaseHandler**, or subclasses of **BaseHandler** (in which case it must be possible to call the constructor without any parameters). Instances of the following classes will be in front of the *handler*s, unless the *handler*s contain them, instances of them or subclasses of them: **ProxyHandler** (if proxy settings are detected), **UnknownHandler**, **HTTPHandler**, **HTTPDefaultErrorHandler**, **HTTPRedirectHandler**, **FTPHandler**, **FileHandler**, **HTTPErrorProcessor**.

If the Python installation has SSL support (i.e., if the **ssl** module can be imported), **HTTPSHandler** will also be added.

A **BaseHandler** subclass may also change its **handler_order** attribute to modify its position in the handlers list.

urllib.request.pathname2url(*path*, *, *add_scheme=False*)

Convert the given local path to a `file:` URL. This function uses **quote()** function to encode the path.

If *add_scheme* is false (the default), the return value omits the `file:` scheme prefix. Set *add_scheme* to true to return a complete URL.

This example shows the function being used on Windows:

```
>>> from urllib.request import pathname2url
>>> path = 'C:\\Program Files'
>>> pathname2url(path, add_scheme=True)
'file:///C:/Program%20Files'
```

Alterado na versão 3.14: Windows drive letters are no longer converted to uppercase, and `:` characters not following a drive letter no longer cause an `OSError` exception to be raised on Windows.

Alterado na versão 3.14: Paths beginning with a slash are converted to URLs with authority sections. For example, the path `/etc/hosts` is converted to the URL `///etc/hosts`.

Alterado na versão 3.14: The *add_scheme* parameter was added.

urllib.request.url2pathname(*url*, ***, *require_scheme=False*, *resolve_host=False*)

Convert the given `file:` URL to a local path. This function uses `unquote()` to decode the URL.

If *require_scheme* is false (the default), the given value should omit a `file:` scheme prefix. If *require_scheme* is set to true, the given value should include the prefix; a `URLError` is raised if it doesn't.

The URL authority is discarded if it is empty, `localhost`, or the local hostname. Otherwise, if *resolve_host* is set to true, the authority is resolved using `socket.gethostbyname()` and discarded if it matches a local IP address (as per **RFC 8089 §3** [https://datatracker.ietf.org/doc/html/rfc8089.html#section-3]). If the authority is still unhandled, then on Windows a UNC path is returned, and on other platforms a `URLError` is raised.

This example shows the function being used on Windows:

```
>>> from urllib.request import url2pathname
>>> url = 'file:///C:/Program%20Files'
>>> url2pathname(url, require_scheme=True)
'C:\\Program Files'
```

Alterado na versão 3.14: Windows drive letters are no longer converted to uppercase, and `:` characters not following a drive letter no longer cause an `OSError` exception to be raised on Windows.

Alterado na versão 3.14: The URL authority is discarded if it matches the local hostname. Otherwise, if the authority isn't empty or `localhost`, then on Windows a UNC path is returned (as before), and on other platforms a `URLError` is raised.

Alterado na versão 3.14: The URL query and fragment components are discarded if present.

Alterado na versão 3.14: The *require_scheme* and *resolve_host* parameters were added.

urllib.request.getproxies()

This helper function returns a dictionary of scheme to proxy server URL mappings. It scans the environment for variables named `<scheme>_proxy`, in a case insensitive approach, for all operating systems first, and when it cannot find it, looks for proxy information from System Configuration for macOS and Windows Systems Registry for Windows. If both lowercase and uppercase environment variables exist (and disagree), lowercase is preferred.

Nota

If the environment variable `REQUEST_METHOD` is set, which usually indicates your script is running in a CGI environment, the environment variable `HTTP_PROXY` (uppercase `_PROXY`) will be ignored. This is because that variable can be injected by a client using the "Proxy:" HTTP header. If you need to use an HTTP proxy in a CGI environment, either use `ProxyHandler` explicitly, or make sure the variable name is in lowercase (or at least the `_proxy` suffix).

As seguintes classes são disponibilizadas:

class urllib.request.Request(*url*, *data=None*, *headers={}*, *origin_req_host=None*, *unverifiable=False*, *method=None*)

This class is an abstraction of a URL request.

url should be a string containing a valid, properly encoded URL.

data must be an object specifying additional data to send to the server, or `None` if no such data is needed. Currently HTTP requests are the only ones that use *data*. The supported object types include bytes, file-like objects, and iterables of bytes-like objects. If no `Content-Length` nor `Transfer-Encoding` header field has been provided, **`HTTPHandler`** will set these headers according to the type of *data*. `Content-Length` will be used to send bytes objects, while `Transfer-Encoding: chunked` as specified in **RFC 7230** [https://datatracker.ietf.org/doc/html/rfc7230.html], Section 3.3.1 will be used to send files and other iterables.

For an HTTP POST request method, *data* should be a buffer in the standard *application/x-www-form-urlencoded* format. The **`urllib.parse.urlencode()`** function takes a mapping or sequence of 2-tuples and returns an ASCII string in this format. It should be encoded to bytes before being used as the *data* parameter.

headers should be a dictionary, and will be treated as if **`add_header()`** was called with each key and value as arguments. This is often used to "spoof" the `User-Agent` header value, which is used by a browser to identify itself – some HTTP servers only allow requests coming from common browsers as opposed to scripts. For example, Mozilla Firefox may identify itself as `"Mozilla/5.0 (X11; U; Linux i686) Gecko/20071127 Firefox/2.0.0.11"`, while **`urllib`**'s default user agent string is `"Python-urllib/2.6"` (on Python 2.6). All header keys are sent in camel case.

An appropriate `Content-Type` header should be included if the *data* argument is present. If this header has not been provided and *data* is not `None`, `Content-Type: application/x-www-form-urlencoded` will be added as a default.

The next two arguments are only of interest for correct handling of third-party HTTP cookies:

origin_req_host should be the request-host of the origin transaction, as defined by **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html]. It defaults to `http.cookiejar.request_host(self)`. This is the host name or IP address of the original request that was initiated by the user. For

example, if the request is for an image in an HTML document, this should be the request-host of the request for the page containing the image.

unverifiable should indicate whether the request is unverifiable, as defined by **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html]. It defaults to `False`. An unverifiable request is one whose URL the user did not have the option to approve. For example, if the request is for an image in an HTML document, and the user had no option to approve the automatic fetching of the image, this should be true.

method should be a string that indicates the HTTP request method that will be used (e.g. `'HEAD'`). If provided, its value is stored in the **`method`** attribute and is used by **`get_method()`**. The default is `'GET'` if *data* is `None` or `'POST'` otherwise. Subclasses may indicate a different default method by setting the **`method`** attribute in the class itself.

Nota

The request will not work as expected if the data object is unable to deliver its content more than once (e.g. a file or an iterable that can produce the content only once) and the request is retried for HTTP redirects or authentication. The *data* is sent to the HTTP server right away after the headers. There is no support for a 100-continue expectation in the library.

Alterado na versão 3.3: **`Request.method`** argument is added to the Request class.

Alterado na versão 3.4: Default **`Request.method`** may be indicated at the class level.

Alterado na versão 3.6: Do not raise an error if the `Content-Length` has not been provided and *data* is neither `None` nor a bytes object. Fall back to use chunked transfer encoding instead.

class urllib.request.OpenerDirector

The **`OpenerDirector`** class opens URLs via **`BaseHandler`**s chained together. It manages the chaining of handlers, and recovery from errors.

class urllib.request.BaseHandler

This is the base class for all registered handlers — and handles only the simple mechanics of registration.

class urllib.request.HTTPDefaultErrorHandler

A class which defines a default handler for HTTP error responses; all responses are turned into **`HTTPError`** exceptions.

class urllib.request.HTTPRedirectHandler

Uma classe para lidar com redirecionamentos.

class urllib.request.HTTPCookieProcessor(*cookiejar=None*)

Classe para lidar com Cookies HTTP.

class urllib.request.ProxyHandler(*proxies=None*)

Cause requests to go through a proxy. If *proxies* is given, it must be a dictionary mapping protocol names to URLs of proxies. The default is to read the list of proxies from the environment variables `<protocol>_proxy`. If no proxy environment variables are set, then in a Windows environment proxy settings are obtained from the registry's Internet Settings section, and in a macOS environment proxy information is retrieved from the System Configuration Framework.

To disable autodetected proxy pass an empty dictionary.

The **`no_proxy`** environment variable can be used to specify hosts which shouldn't be reached via proxy; if set, it should be a comma-separated list of hostname suffixes, optionally with `:port` appended, for example `cern.ch,ncsa.uiuc.edu,some.host:8080`.

Nota

`HTTP_PROXY` será ignorado se uma variável `REQUEST_METHOD` estiver definida; veja a documentação em **`getproxies()`**.

class urllib.request.HTTPPasswordMgr

Keep a database of `(realm, uri) -> (user, password)` mappings.

class urllib.request.HTTPPasswordMgrWithDefaultRealm

Keep a database of `(realm, uri) -> (user, password)` mappings. A realm of `None` is considered a catch-all realm, which is searched if no other realm fits.

class urllib.request.HTTPPasswordMgrWithPriorAuth

A variant of **`HTTPPasswordMgrWithDefaultRealm`** that also has a database of `uri -> is_authenticated` mappings. Can be used by a BasicAuth handler to determine when to send authentication credentials immediately instead of waiting for a `401` response first.

Adicionado na versão 3.5.

class urllib.request.AbstractBasicAuthHandler(*password_mgr=None*)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. *password_mgr*, if given, should be something that is compatible with **`HTTPPasswordMgr`**; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. If *passwd_mgr* also provides `is_authenticated` and `update_authenticated` methods (see HTTPPasswordMgrWithPriorAuth Objects), then the handler will use the `is_authenticated` result for a given URI to determine whether or not to send authentication credentials with the request. If `is_authenticated` returns `True` for the URI, credentials are sent. If `is_authenticated` is `False`, credentials are not sent, and then if a `401` response is received the request is re-sent with the authentication credentials. If authentication succeeds, `update_authenticated` is called to set `is_authenticated` `True` for the URI, so that subsequent requests to the URI or any of its super-URIs will automatically include the authentication credentials.

Adicionado na versão 3.5: Added `is_authenticated` support.

class urllib.request.HTTPBasicAuthHandler(*password_mgr=None*)

Handle authentication with the remote host. *password_mgr*, if given, should be something that is compatible with **HTTPPasswordMgr**; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. HTTPBasicAuthHandler will raise a **ValueError** when presented with a wrong Authentication scheme.

class urllib.request.ProxyBasicAuthHandler(*password_mgr=None*)

Handle authentication with the proxy. *password_mgr*, if given, should be something that is compatible with **HTTPPasswordMgr**; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

class urllib.request.AbstractDigestAuthHandler(*password_mgr=None*)

This is a mixin class that helps with HTTP authentication, both to the remote host and to a proxy. *password_mgr*, if given, should be something that is compatible with **HTTPPasswordMgr**; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

Alterado na versão 3.14: Added support for HTTP digest authentication algorithm `SHA-256`.

class urllib.request.HTTPDigestAuthHandler(*password_mgr=None*)

Handle authentication with the remote host. *password_mgr*, if given, should be something that is compatible with **HTTPPasswordMgr**; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported. When both Digest Authentication Handler and Basic Authentication Handler are both added, Digest Authentication is always tried first. If the Digest Authentication returns a 40x response again, it is sent to Basic Authentication handler to Handle. This Handler method will raise a **ValueError** when presented with an authentication scheme other than Digest or Basic.

Alterado na versão 3.3: Raise **ValueError** on unsupported Authentication Scheme.

class urllib.request.ProxyDigestAuthHandler(*password_mgr=None*)

Handle authentication with the proxy. *password_mgr*, if given, should be something that is compatible with `HTTPPasswordMgr`; refer to section HTTPPasswordMgr Objects for information on the interface that must be supported.

class urllib.request.HTTPHandler

A class to handle opening of HTTP URLs.

class urllib.request.HTTPSHandler(*debuglevel=0, context=None, check_hostname=None*)

A class to handle opening of HTTPS URLs. *context* and *check_hostname* have the same meaning as in `http.client.HTTPSConnection`.

Alterado na versão 3.2: context and *check_hostname* were added.

class urllib.request.FileHandler

Abre arquivos locais.

class urllib.request.DataHandler

Abre dados das URLs.

Adicionado na versão 3.4.

class urllib.request.FTPHandler

Abre URLs de FTP.

class urllib.request.CacheFTPHandler

Open FTP URLs, keeping a cache of open FTP connections to minimize delays.

class urllib.request.UnknownHandler

A catch-all class to handle unknown URLs.

class urllib.request.HTTPErrorProcessor

Process HTTP error responses.

Objeto Request

The following methods describe **`Request`**'s public interface, and so all may be overridden in subclasses. It also defines several public attributes that can be used by clients to inspect the parsed request.

Request.full_url

The original URL passed to the constructor.

Alterado na versão 3.4.

Request.full_url is a property with setter, getter and a deleter. Getting **`full_url`** returns the original request URL with the fragment, if it was present.

Request.type

The URI scheme.

Request.host

The URI authority, typically a host, but may also contain a port separated by a colon.

Request.origin_req_host

The original host for the request, without port.

Request.selector

The URI path. If the **`Request`** uses a proxy, then selector will be the full URL that is passed to the proxy.

Request.data

The entity body for the request, or `None` if not specified.

Alterado na versão 3.4: Changing value of **`Request.data`** now deletes "Content-Length" header if it was previously set or calculated.

Request.unverifiable

boolean, indicates whether the request is unverifiable as defined by **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html].

Request.method

The HTTP request method to use. By default its value is `None`, which means that `get_method()` will do its normal computation of the method to be used. Its value can be set (thus overriding the default computation in `get_method()`) either by providing a default value by setting it at the class level in a `Request` subclass, or by passing a value in to the `Request` constructor via the *method* argument.

Adicionado na versão 3.3.

Alterado na versão 3.4: A default value can now be set in subclasses; previously it could only be set via the constructor argument.

Request.get_method()

Return a string indicating the HTTP request method. If `Request.method` is not `None`, return its value, otherwise return `'GET'` if `Request.data` is `None`, or `'POST'` if it's not. This is only meaningful for HTTP requests.

Alterado na versão 3.3: get_method now looks at the value of `Request.method`.

Request.add_header(*key*, *val*)

Add another header to the request. Headers are currently ignored by all handlers except HTTP handlers, where they are added to the list of headers sent to the server. Note that there cannot be more than one header with the same name, and later calls will overwrite previous calls in case the *key* collides. Currently, this is no loss of HTTP functionality, since all headers which have meaning when used more than once have a (header-specific) way of gaining the same functionality using only one header. Note that headers added using this method are also added to redirected requests.

Request.add_unredirected_header(*key*, *header*)

Add a header that will not be added to a redirected request.

Request.has_header(*header*)

Return whether the instance has the named header (checks both regular and unredirected).

Request.remove_header(*header*)

Remove o cabeçalho nomeado da instância de solicitação (tanto de cabeçalhos regulares como de cabeçalhos não-redirecionados).

Adicionado na versão 3.4.

Request.get_full_url()

Return the URL given in the constructor.

Alterado na versão 3.4.

Returns `Request.full_url`

Request.set_proxy(*host*, *type*)

Prepare the request by connecting to a proxy server. The *host* and *type* will replace those of the instance, and the instance's selector will be the original URL given in the constructor.

Request.get_header(*header_name*, *default=None*)

Return the value of the given header. If the header is not present, return the default value.

Request.header_items()

Return a list of tuples (header_name, header_value) of the Request headers.

Alterado na versão 3.4: The request methods add_data, has_data, get_data, get_type, get_host, get_selector, get_origin_req_host and is_unverifiable that were deprecated since 3.3 have been removed.

OpenerDirector Objects

`OpenerDirector` instances have the following methods:

OpenerDirector.add_handler(*handler*)

handler should be an instance of `BaseHandler`. The following methods are searched, and added to the possible chains (note that HTTP errors are a special case). Note that, in the following, *protocol* should be replaced with the actual protocol to handle, for example **`http_response()`** would be the HTTP protocol response handler. Also *type* should be replaced with the actual HTTP code, for example **`http_error_404()`** would handle HTTP 404 errors.

- **`<protocol>_open()`** — signal that the handler knows how to open *protocol* URLs.

  Veja `BaseHandler.<protocol>_open()` para maiores informações.

- **`http_error_<type>()`** — signal that the handler knows how to handle HTTP errors with HTTP error code *type*.

  Veja `BaseHandler.http_error_<nnn>()` para maiores informações.

- **`<protocol>_error()`** — signal that the handler knows how to handle errors from (non-`http`) *protocol*.

- **`<protocol>_request()`** — signal that the handler knows how to pre-process *protocol* requests.

  Veja `BaseHandler.<protocol>_request()` para maiores informações.

- **`<protocol>_response()`** — signal that the handler knows how to post-process *protocol* responses.

  Veja `BaseHandler.<protocol>_response()` para maiores informações.

OpenerDirector.open(*url*, *data=None*[, *timeout*])

Open the given *url* (which can be a request object or a string), optionally passing the given *data*. Arguments, return values and exceptions raised are the same as those of `urlopen()` (which simply calls the `open()` method on the currently installed global `OpenerDirector`). The optional *timeout* parameter specifies a timeout in seconds for blocking operations like the connection attempt (if not specified, the global default timeout setting will be used). The timeout feature actually works only for HTTP, HTTPS and FTP connections.

OpenerDirector.error(*proto*, **args*)

Handle an error of the given protocol. This will call the registered error handlers for the given protocol with the given arguments (which are protocol specific). The HTTP protocol is a special case which uses the HTTP response code to determine the specific error handler; refer to the `http_error_<type>()` methods of the handler classes.

Return values and exceptions raised are the same as those of `urlopen()`.

OpenerDirector objects open URLs in three stages:

The order in which these methods are called within each stage is determined by sorting the handler instances.

1. Every handler with a method named like `<protocol>_request()` has that method called to pre-process the request.

2. Handlers with a method named like `<protocol>_open()` are called to handle the request. This stage ends when a handler either returns a non-`None` value (ie. a response), or raises an exception (usually `URLError`). Exceptions are allowed to propagate.

   In fact, the above algorithm is first tried for methods named `default_open()`. If all such methods return `None`, the algorithm is repeated for methods named like `<protocol>_open()`. If all such methods return `None`, the algorithm is repeated for methods named `unknown_open()`.

Note that the implementation of these methods may involve calls of the parent `OpenerDirector` instance's `open()` and `error()` methods.

3. Every handler with a method named like `<protocol>_response()` has that method called to post-process the response.

BaseHandler Objects

`BaseHandler` objects provide a couple of methods that are directly useful, and others that are meant to be used by derived classes. These are intended for direct use:

BaseHandler.add_parent(*director*)

Add a director as parent.

BaseHandler.close()

Remove any parents.

The following attribute and methods should only be used by classes derived from `BaseHandler`.

Nota

The convention has been adopted that subclasses defining `<protocol>_request()` or `<protocol>_response()` methods are named `*Processor`; all others are named `*Handler`.

BaseHandler.parent

A valid `OpenerDirector`, which can be used to open using a different protocol, or handle errors.

BaseHandler.default_open(*req*)

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to catch all URLs.

This method, if implemented, will be called by the parent `OpenerDirector`. It should return a file-like object as described in the return value of the `open()` method of `OpenerDirector`, or `None`. It should raise `URLError`, unless a truly exceptional thing happens (for example, `MemoryError` should not be mapped to `URLError`).

This method will be called before any protocol-specific open method.

BaseHandler.<protocol>_open(req)

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to handle URLs with the given protocol.

This method, if defined, will be called by the parent `OpenerDirector`. Return values should be the same as for `default_open()`.

BaseHandler.unknown_open(*req*)

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to catch all URLs with no specific registered handler to open it.

This method, if implemented, will be called by the `parent OpenerDirector`. Return values should be the same as for `default_open()`.

BaseHandler.http_error_default(*req*, *fp*, *code*, *msg*, *hdrs*)

This method is *not* defined in `BaseHandler`, but subclasses should override it if they intend to provide a catch-all for otherwise unhandled HTTP errors. It will be called automatically by the `OpenerDirector` getting the error, and should not normally be called in other circumstances.

`OpenerDirector` will call this method with five positional arguments:

1. a `Request` object,
2. a file-like object with the HTTP error body,
3. the three-digit code of the error, as a string,
4. the user-visible explanation of the code, as a string, and

5. the headers of the error, as a mapping object.

Return values and exceptions raised should be the same as those of `urlopen()`.

BaseHandler.http_error_<nnn>(req, fp, code, msg, hdrs)

nnn should be a three-digit HTTP error code. This method is also not defined in `BaseHandler`, but will be called, if it exists, on an instance of a subclass, when an HTTP error with code *nnn* occurs.

Subclasses should override this method to handle specific HTTP errors.

Arguments, return values and exceptions raised should be the same as for `http_error_default()`.

BaseHandler.<protocol>_request(req)

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to pre-process requests of the given protocol.

This method, if defined, will be called by the parent `OpenerDirector`. *req* will be a `Request` object. The return value should be a `Request` object.

BaseHandler.<protocol>_response(req, response)

This method is *not* defined in `BaseHandler`, but subclasses should define it if they want to post-process responses of the given protocol.

This method, if defined, will be called by the parent `OpenerDirector`. *req* will be a `Request` object. *response* will be an object implementing the same interface as the return value of `urlopen()`. The return value should implement the same interface as the return value of `urlopen()`.

HTTPRedirectHandler Objects

Nota

Some HTTP redirections require action from this module's client code. If this is the case, `HTTPError` is raised. See **RFC 2616**

[https://datatracker.ietf.org/doc/html/rfc2616.html] for details of the precise meanings of the various redirection codes.

An **`HTTPError`** exception raised as a security consideration if the HTTPRedirectHandler is presented with a redirected URL which is not an HTTP, HTTPS or FTP URL.

HTTPRedirectHandler.redirect_request(*req*, *fp*, *code*, *msg*, *hdrs*, *newurl*)

Return a **`Request`** or `None` in response to a redirect. This is called by the default implementations of the **`http_error_30*()`** methods when a redirection is received from the server. If a redirection should take place, return a new **`Request`** to allow **`http_error_30*()`** to perform the redirect to *newurl*. Otherwise, raise **`HTTPError`** if no other handler should try to handle this URL, or return `None` if you can't but another handler might.

Nota

The default implementation of this method does not strictly follow **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html], which says that 301 and 302 responses to `POST` requests must not be automatically redirected without confirmation by the user. In reality, browsers do allow automatic redirection of these responses, changing the POST to a `GET`, and the default implementation reproduces this behavior.

HTTPRedirectHandler.http_error_301(*req*, *fp*, *code*, *msg*, *hdrs*)

Redirect to the `Location:` or `URI:` URL. This method is called by the parent **`OpenerDirector`** when getting an HTTP 'moved permanently' response.

HTTPRedirectHandler.http_error_302(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as **`http_error_301()`**, but called for the 'found' response.

HTTPRedirectHandler.http_error_303(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as **`http_error_301()`**, but called for the 'see other' response.

HTTPRedirectHandler.http_error_307(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as `http_error_301()`, but called for the 'temporary redirect' response. It does not allow changing the request method from `POST` to `GET`.

HTTPRedirectHandler.http_error_308(*req*, *fp*, *code*, *msg*, *hdrs*)

The same as `http_error_301()`, but called for the 'permanent redirect' response. It does not allow changing the request method from `POST` to `GET`.

Adicionado na versão 3.11.

HTTPCookieProcessor Objects

`HTTPCookieProcessor` instances have one attribute:

HTTPCookieProcessor.cookiejar

The `http.cookiejar.CookieJar` in which cookies are stored.

ProxyHandler Objects

ProxyHandler.<protocol>_open(request)

The `ProxyHandler` will have a method **`<protocol>_open()`** for every *protocol* which has a proxy in the *proxies* dictionary given in the constructor. The method will modify requests to go through the proxy, by calling `request.set_proxy()`, and call the next handler in the chain to actually execute the protocol.

HTTPPasswordMgr Objects

These methods are available on `HTTPPasswordMgr` and `HTTPPasswordMgrWithDefaultRealm` objects.

HTTPPasswordMgr.add_password(*realm*, *uri*, *user*, *passwd*)

uri can be either a single URI, or a sequence of URIs. *realm*, *user* and *passwd* must be strings. This causes `(user, passwd)` to be used as authentication tokens when authentication for *realm* and a super-URI of any of the given URIs is given. If a URI includes a scheme, its credentials only match authentication URIs with the same scheme or no scheme. A URI without a scheme matches authentication URIs with any scheme.

Alterado na versão 3.15: Authentication credentials for URIs with a scheme are now scoped by that scheme.

HTTPPasswordMgr.find_user_password(*realm*, *authuri*)

Get user/password for given realm and URI, if any. This method will return `(None, None)` if there is no matching user/password.

For **`HTTPPasswordMgrWithDefaultRealm`** objects, the realm `None` will be searched if the given *realm* has no matching user/password.

HTTPPasswordMgrWithPriorAuth Objects

This password manager extends **`HTTPPasswordMgrWithDefaultRealm`** to support tracking URIs for which authentication credentials should always be sent.

HTTPPasswordMgrWithPriorAuth.add_password(*realm*, *uri*, *user*, *passwd*, *is_authenticated=False*)

realm, *uri*, *user*, *passwd* are as for **`HTTPPasswordMgr.add_password()`**. *is_authenticated* sets the initial value of the `is_authenticated` flag for the given URI or list of URIs. If *is_authenticated* is specified as `True`, *realm* is ignored.

HTTPPasswordMgrWithPriorAuth.find_user_password(*realm*, *authuri*)

Same as for **`HTTPPasswordMgrWithDefaultRealm`** objects

HTTPPasswordMgrWithPriorAuth.update_authenticated(*self*, *uri*, *is_authenticated=False*)

Update the `is_authenticated` flag for the given *uri* or list of URIs.

HTTPPasswordMgrWithPriorAuth.is_authenticated(*self*, *authuri*)

Returns the current state of the `is_authenticated` flag for the given URI.

AbstractBasicAuthHandler Objects

AbstractBasicAuthHandler.http_error_auth_reqed(*authreq*, *host*, *req*, *headers*)

Handle an authentication request by getting a user/password pair, and re-trying the request. *authreq* should be the name of the header where the information about the realm is included in the request, *host* specifies the URL and path to authenticate for, *req* should be the (failed) **`Request`** object, and *headers* should be the error headers.

headers must be a mapping-like object with case-insensitive lookup that implements the `get_all()` method, such as **`email.message.Message`** or **`wsgiref.headers.Headers`**.

host is either an authority (e.g. `"python.org"`) or a URL containing an authority component (e.g. `"https://python.org/"`). In either case, the authority must not contain a userinfo component (so, `"python.org"` and `"python.org:80"` are fine, `"joe:password@python.org"` is not).

HTTPBasicAuthHandler Objects

HTTPBasicAuthHandler.http_error_401(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

ProxyBasicAuthHandler Objects

ProxyBasicAuthHandler.http_error_407(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

AbstractDigestAuthHandler Objects

AbstractDigestAuthHandler.http_error_auth_reqed(*authreq*, *host*, *req*, *headers*)

authreq should be the name of the header where the information about the realm is included in the request, *host* should be the host to authenticate to, *req* should be the (failed) `Request` object, and *headers* should be the error headers.

headers must be a mapping-like object with case-insensitive lookup, such as `email.message.Message` or `wsgiref.headers.Headers`.

HTTPDigestAuthHandler Objects

HTTPDigestAuthHandler.http_error_401(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

ProxyDigestAuthHandler Objects

ProxyDigestAuthHandler.http_error_407(*req*, *fp*, *code*, *msg*, *hdrs*)

Retry the request with authentication information, if available.

HTTPHandler Objects

HTTPHandler.http_open(*req*)

Send an HTTP request, which can be either GET or POST, depending on `req.data`.

Objetos HTTPSHandler

HTTPSHandler.https_open(*req*)

Send an HTTPS request, which can be either GET or POST, depending on `req.data`.

FileHandler Objects

FileHandler.file_open(*req*)

Open the file locally, if there is no host name, or the host name is `'localhost'`.

Alterado na versão 3.2: This method is applicable only for local hostnames. When a remote hostname is given, a `URLError` is raised.

DataHandler Objects

DataHandler.data_open(*req*)

Read a data URL. This kind of URL contains the content encoded in the URL itself. The data URL syntax is specified in **RFC 2397** [https://datatracker.ietf.org/doc/html/rfc2397.html]. This implementation ignores white spaces in base64 encoded data URLs so the URL may be wrapped in whatever source file it comes from. But even though some browsers don't mind about a missing padding at the end of a base64 encoded data URL, this implementation will raise a `ValueError` in that case.

FTPHandler Objects

FTPHandler.ftp_open(*req*)

Open the FTP file indicated by *req*. The login is always done with empty username and password.

CacheFTPHandler Objects

`CacheFTPHandler` objects are `FTPHandler` objects with the following additional methods:

CacheFTPHandler.setTimeout(*t*)

Set timeout of connections to *t* seconds.

CacheFTPHandler.setMaxConns(*m*)

Set maximum number of cached connections to *m*.

Objetos UnknownHandler

UnknownHandler.unknown_open()

Raise a **URLError** exception.

HTTPErrorProcessor Objects

HTTPErrorProcessor.http_response(*request*, *response*)

Process HTTP error responses.

For 200 error codes, the response object is returned immediately.

For non-200 error codes, this simply passes the job on to the **http_error_<type>()** handler methods, via **OpenerDirector.error()**. Eventually, **HTTPDefaultErrorHandler** will raise an **HTTPError** if no other handler handles the error.

HTTPErrorProcessor.https_response(*request*, *response*)

Process HTTPS error responses.

The behavior is same as **http_response()**.

Exemplos

In addition to the examples below, more examples are given in Como buscar recursos da Internet usando o pacote urllib.

This example gets the python.org main page and displays the first 300 bytes of it:

```
>>> import urllib.request
>>> with urllib.request.urlopen('https://www.python.org/') as f:
...     # The response may be compressed (for example, 'gzip').
```

```
...     print(f.headers.get('Content-Encoding'))
...     data = f.read()
...     if f.headers.get('Content-Encoding') == 'gzip':
...         import gzip
...         data = gzip.decompress(data)
...     print(data[:300].decode('utf-8', errors='replace'))
```

Note that urlopen returns a bytes object. This is because there is no way for urlopen to automatically determine the encoding of the byte stream it receives from the HTTP server. In general, a program will decode the returned bytes object to string once it determines or guesses the appropriate encoding.

The following HTML spec document, https://html.spec.whatwg.org/#charset, lists the various ways in which an HTML or an XML document could have specified its encoding information.

For additional information, see the W3C document: https://www.w3.org/International/questions/qa-html-encoding-declarations.

As the python.org website uses *utf-8* encoding as specified in its meta tag, we will use the same for decoding the bytes object:

```
>>> with urllib.request.urlopen('https://www.python.org/') as f:
...     # Check for compression and decode appropriately.
...     enc = f.headers.get('Content-Encoding')
...     data = f.read()
...     if enc == 'gzip':
...         import gzip
...         data = gzip.decompress(data)
...     print(data[:100].decode('utf-8', errors='replace'))
...
```

It is also possible to achieve the same result without using the context manager approach:

```
>>> import urllib.request
>>> f = urllib.request.urlopen('https://www.python.org/')
```

```
>>> try:
...     enc = f.headers.get('Content-Encoding')
...     data = f.read()
...     if enc == 'gzip':
...         import gzip
...         data = gzip.decompress(data)
...     print(data[:100].decode('utf-8', errors='replace'))
... finally:
...     f.close()
```

In the following example, we are sending a data-stream to the stdin of a CGI and reading the data it returns to us. Note that this example will only work when the Python installation supports SSL.

```
>>> import urllib.request
>>> req = urllib.request.Request(url='https://localhost/cgi-bin/
...                              data=b'This data is passed to stdin of
>>> with urllib.request.urlopen(req) as f:
...     print(f.read().decode('utf-8'))
...
Got Data: "This data is passed to stdin of the CGI"
```

The code for the sample CGI used in the above example is:

```
#!/usr/bin/env python
import sys
data = sys.stdin.read()
print('Content-type: text/plain\n\nGot Data: "%s"' % data)
```

Here is an example of doing a `PUT` request using **`Request`**:

```
import urllib.request
DATA = b'some data'
req = urllib.request.Request(url='http://localhost:8080', data=D
with urllib.request.urlopen(req) as f:
    pass
print(f.status)
print(f.reason)
```

Use of Basic HTTP Authentication:

```
import urllib.request
# Create an OpenerDirector with support for Basic HTTP Authentic
auth_handler = urllib.request.HTTPBasicAuthHandler()
auth_handler.add_password(realm='PDQ Application',
                          uri='https://mahler:8092/site-updates.
                          user='klem',
                          passwd='kadidd!ehopper')
opener = urllib.request.build_opener(auth_handler)
# ...and install it globally so it can be used with urlopen.
urllib.request.install_opener(opener)
with urllib.request.urlopen('http://www.example.com/login.html')
    print(f.read().decode('utf-8'))
```

`build_opener()` provides many handlers by default, including a `ProxyHandler`. By default, **`ProxyHandler`** uses the environment variables named `<scheme>_proxy`, where `<scheme>` is the URL scheme involved. For example, the **`http_proxy`** environment variable is read to obtain the HTTP proxy's URL.

This example replaces the default `ProxyHandler` with one that uses programmatically supplied proxy URLs, and adds proxy authorization support with `ProxyBasicAuthHandler`.

```
proxy_handler = urllib.request.ProxyHandler({'http': 'http://www
proxy_auth_handler = urllib.request.ProxyBasicAuthHandler()
proxy_auth_handler.add_password('realm', 'host', 'username', 'pa

opener = urllib.request.build_opener(proxy_handler, proxy_auth_h
# This time, rather than install the OpenerDirector, we use it d
with opener.open('http://www.example.com/login.html') as f:
   print(f.read().decode('utf-8'))
```

Adicionando cabeçalhos HTTP:

Use the *headers* argument to the `Request` constructor, or:

```
import urllib.request
req = urllib.request.Request('http://www.example.com/')
req.add_header('Referer', 'https://www.python.org/')
# Customize the default User-Agent header value:
```

```
req.add_header('User-Agent', 'urllib-example/0.1 (Contact: . . .
with urllib.request.urlopen(req) as f:
    print(f.read().decode('utf-8'))
```

`OpenerDirector` automatically adds a *User-Agent* header to every `Request`. To change this:

```
import urllib.request
opener = urllib.request.build_opener()
opener.addheaders = [('User-agent', 'Mozilla/5.0')]
with opener.open('http://www.example.com/') as f:
   print(f.read().decode('utf-8'))
```

Also, remember that a few standard headers (*Content-Length*, *Content-Type* and *Host*) are added when the `Request` is passed to `urlopen()` (or `OpenerDirector.open()`).

Here is an example session that uses the `GET` method to retrieve a URL containing parameters:

```
>>> import urllib.request
>>> import urllib.parse
>>> params = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacc
>>> url = "https://www.python.org/?%s" % params
>>> with urllib.request.urlopen(url) as f:
...     print(f.read().decode('utf-8'))
...
```

The following example uses the `POST` method instead. Note that params output from urlencode is encoded to bytes before it is sent to urlopen as data:

```
>>> import urllib.request
>>> import urllib.parse
>>> data = urllib.parse.urlencode({'spam': 1, 'eggs': 2, 'bacon'
>>> data = data.encode('ascii')
>>> with urllib.request.urlopen("https://httpbin.org/post", data
...     print(f.read().decode('utf-8'))
...
```

The following example uses an explicitly specified HTTP proxy, overriding environment settings:

```
>>> import urllib.request
>>> proxies = {'http': 'http://proxy.example.com:8080/'}
>>> opener = urllib.request.build_opener(urllib.request.ProxyHar
>>> with opener.open("https://www.python.org") as f:
...     f.read().decode('utf-8')
...
```

The following example uses no proxies at all, overriding environment settings:

```
>>> import urllib.request
>>> opener = urllib.request.build_opener(urllib.request.ProxyHar
>>> with opener.open("https://www.python.org/") as f:
...     f.read().decode('utf-8')
...
```

Legacy interface

The following functions and classes are ported from the Python 2 module `urllib` (as opposed to `urllib2`). They might become deprecated at some point in the future.

urllib.request.urlretrieve(*url*, *filename=None*, *reporthook=None*, *data=None*)

Copy a network object denoted by a URL to a local file. If the URL points to a local file, the object will not be copied unless filename is supplied. Return a tuple `(filename, headers)` where *filename* is the local file name under which the object can be found, and *headers* is whatever the **`info()`** method of the object returned by **`urlopen()`** returned (for a remote object). Exceptions are the same as for **`urlopen()`**.

The second argument, if present, specifies the file location to copy to (if absent, the location will be a tempfile with a generated name). The third

argument, if present, is a callable that will be called once on establishment of the network connection and once after each block read thereafter. The callable will be passed three arguments; a count of blocks transferred so far, a block size in bytes, and the total size of the file. The third argument may be `-1` on older FTP servers which do not return a file size in response to a retrieval request.

The following example illustrates the most common usage scenario:

```
>>> import urllib.request
>>> local_filename, headers = urllib.request.urlretrieve('htt
>>> html = open(local_filename)
>>> html.close()
```

If the *url* uses the `http:` scheme identifier, the optional *data* argument may be given to specify a `POST` request (normally the request type is `GET`). The *data* argument must be a bytes object in standard *application/x-www-form-urlencoded* format; see the `urllib.parse.urlencode()` function.

`urlretrieve()` will raise `ContentTooShortError` when it detects that the amount of data available was less than the expected amount (which is the size reported by a *Content-Length* header). This can occur, for example, when the download is interrupted.

The *Content-Length* is treated as a lower bound: if there's more data to read, urlretrieve reads more data, but if less data is available, it raises the exception.

You can still retrieve the downloaded data in this case, it is stored in the **`content`** attribute of the exception instance.

If no *Content-Length* header was supplied, urlretrieve can not check the size of the data it has downloaded, and just returns it. In this case you just have to assume that the download was successful.

urllib.request.urlcleanup()

Cleans up temporary files that may have been left behind by previous calls to `urlretrieve()`. It also resets the default global opener installed by `install_opener()`.

`urllib.request` Restrictions

- Currently, only the following protocols are supported: HTTP (versions 0.9 and 1.0), FTP, local files, and data URLs.

  Alterado na versão 3.4: Added support for data URLs.

- The caching feature of `urlretrieve()` has been disabled until someone finds the time to hack proper processing of Expiration time headers.

- There should be a function to query whether a particular URL is in the cache.

- For backward compatibility, if a URL appears to point to a local file but the file can't be opened, the URL is re-interpreted using the FTP protocol. This can sometimes cause confusing error messages.

- The `urlopen()` and `urlretrieve()` functions can cause arbitrarily long delays while waiting for a network connection to be set up. This means that it is difficult to build an interactive web client using these functions without using threads.

- The data returned by `urlopen()` or `urlretrieve()` is the raw data returned by the server. This may be binary data (such as an image), plain text or (for example) HTML. The HTTP protocol provides type information in the reply header, which can be inspected by looking at the *Content-Type* header. If the returned data is HTML, you can use the module `html.parser` to parse it.

- The code handling the FTP protocol cannot differentiate between a file and a directory. This can lead to unexpected behavior when attempting to read a URL that points to a file that is not accessible. If the URL

ends in a `/`, it is assumed to refer to a directory and will be handled accordingly. But if an attempt to read a file leads to a 550 error (meaning the URL cannot be found or is not accessible, often for permission reasons), then the path is treated as a directory in order to handle the case when a directory is specified by a URL but the trailing `/` has been left off. This can cause misleading results when you try to fetch a file whose read permissions make it inaccessible; the FTP code will try to read it, fail with a 550 error, and then perform a directory listing for the unreadable file. If fine-grained control is needed, consider using the **`ftplib`** module.

`urllib.response` — Response classes used by urllib

The **`urllib.response`** module defines functions and classes which define a minimal file-like interface, including `read()` and `readline()`. Functions defined by this module are used internally by the **`urllib.request`** module. The typical response object is a **`urllib.response.addinfourl`** instance:

class urllib.response.addinfourl

url

URL of the resource retrieved, commonly used to determine if a redirect was followed.

headers

Returns the headers of the response in the form of an **`EmailMessage`** instance.

status

Adicionado na versão 3.9.

Status code returned by server.

geturl()

Descontinuado desde a versão 3.9: Deprecated in favor of `url`.

info()

Descontinuado desde a versão 3.9: Deprecated in favor of `headers`.

code

Descontinuado desde a versão 3.9: Deprecated in favor of `status`.

getcode()

Descontinuado desde a versão 3.9: Deprecated in favor of `status`.

`urllib.parse` — Analisa URLs para componentes

Código-fonte: Lib/urllib/parse.py
[https://github.com/python/cpython/tree/3.15/Lib/urllib/parse.py]

Este módulo define uma interface padrão para quebrar strings de Uniform Resource Locator (URL) em componentes (esquema de endereçamento, local de rede, caminho etc.), para combinar os componentes de volta em uma string de URL e para converter uma “URL relativo” em uma URL absoluta dado uma “URL base”.

O módulo foi projetado para corresponder ao RFC da internet sobre Localizadores de Recursos Uniformes Relativos, ou URL relativa. Ele tem suporte aos seguintes esquemas de URL: `file`, `ftp`, `gopher`, `hdl`, `http`, `https`, `imap`, `itms-services`, `mailto`, `mms`, `news`, `nntp`, `prospero`, `rsync`, `rtsp`, `rtsps`,``rtspu``, `sftp`, `shttp`, `sip`, `sips`, `snews`, `svn`, `svn+ssh`, `telnet`, `wais`, `ws`, `wss`.

A inclusão do esquema de URL `itms-services` pode impedir que uma aplicação passe pelo processo de revisão da App Store da Apple para as App Stores do macOS e iOS. O tratamento do esquema `itms-services` é sempre removido no iOS; no macOS, ele *pode* ser removido se o CPython tiver sido criado com a opção **`--with-app-store-compliance`**.

The **`urllib.parse`** module defines functions that fall into two broad categories: URL parsing and URL quoting. These are covered in detail in the following sections.

As funções deste módulo usam o termo descontinuado `netloc` (ou `net_loc`), que foi introduzido no **RFC 1808** [https://datatracker.ietf.org/doc/html/rfc1808.html]. No entanto, este termo foi descontinuado pelo **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], que

introduziu o termo `authority` como seu substituto. O uso de `netloc` continua para compatibilidade com versões anteriores.

Análise de URL

As funções de análise de URL se concentram na divisão de uma string de URL em seus componentes ou na combinação de componentes de URL em uma string de URL.

urllib.parse.urlsplit(*urlstring*, *scheme=None*, *allow_fragments=True*, *, *missing_as_none=False*)

Parse a URL into five components, returning a 5-item named tuple **`SplitResult`** or **`SplitResultBytes`**. This corresponds to the general structure of a URL: `scheme://netloc/path?query#fragment`. Each tuple item is a string, possibly empty, or `None` if *missing_as_none* is true. Not defined component are represented an empty string (by default) or `None` if *missing_as_none* is true. The delimiters as shown above are not part of the result, except for a leading slash in the *path* component, which is retained if present.

Additionally, the netloc property is broken down into these additional attributes added to the returned object: username, password, hostname, and port.

Percent-encoded sequences are not decoded.

Por exemplo:

```
>>> from urllib.parse import urlsplit
>>> urlsplit("scheme://netloc/path?query#fragment")
SplitResult(scheme='scheme', netloc='netloc', path='/path',
            query='query', fragment='fragment')
>>> o = urlsplit("http://docs.python.org:80/3/library/urllib.
...              "highlight=params#url-parsing")
>>> o
SplitResult(scheme='http', netloc='docs.python.org:80',
            path='/3/library/urllib.parse.html',
            query='highlight=params', fragment='url-parsing')
```

```
>>> o.scheme
'http'
>>> o.netloc
'docs.python.org:80'
>>> o.hostname
'docs.python.org'
>>> o.port
80
>>> o._replace(fragment="").geturl()
'http://docs.python.org:80/3/library/urllib.parse.html?highli
>>> urlsplit("http://docs.python.org?")
SplitResult(scheme='http', netloc='docs.python.org', path='',
            query='', fragment='')
>>> urlsplit("http://docs.python.org?", missing_as_none=True)
SplitResult(scheme='http', netloc='docs.python.org', path='',
            query='', fragment=None)
```

Following the syntax specifications in **RFC 1808** [https://datatracker.ietf.org/doc/html/rfc1808.html], **`urlsplit()`** recognizes a netloc only if it is properly introduced by '//'. Otherwise the input is presumed to be a relative URL and thus to start with a path component.

```
>>> from urllib.parse import urlsplit
>>> urlsplit('//www.cwi.nl:80/%7Eguido/Python.html')
SplitResult(scheme='', netloc='www.cwi.nl:80', path='/%7Eguid
            query='', fragment='')
>>> urlsplit('www.cwi.nl/%7Eguido/Python.html')
SplitResult(scheme='', netloc='', path='www.cwi.nl/%7Eguido/P
            query='', fragment='')
>>> urlsplit('help/Python.html')
SplitResult(scheme='', netloc='', path='help/Python.html',
            query='', fragment='')
>>> urlsplit('help/Python.html', missing_as_none=True)
SplitResult(scheme=None, netloc=None, path='help/Python.html'
            query=None, fragment=None)
```

The *scheme* argument gives the default addressing scheme, to be used only if the URL does not specify one. It should be the same type (text or bytes) as *urlstring* or `None`, except that the `''` is always allowed, and is automatically converted to `b''` if appropriate.

If the *allow_fragments* argument is false, fragment identifiers are not recognized. Instead, they are parsed as part of the path or query component, and **`fragment`** is set to `None` or the empty string (depending on the value of *missing_as_none*) in the return value.

O valor de retorno é uma tupla nomeada, o que significa que seus itens podem ser acessados por índice ou como atributos nomeados, que são:

Atributo	Índice	Valor	Valor, se não presente
`scheme`	0	Especificador do esquema da URL	*scheme* parameter or empty string [[1]]
`netloc`	1	Parte da localização na rede	`None` or empty string [[1]]
`path`	2	Caminho hierárquico	string vazia
`query`	3	Componente da consulta	`None` or empty string [[1]]
`fragment`	4	Identificador do fragmento	`None` or empty string [[1]]
`username`		Nome do usuário	`None`
`password`		Senha	`None`
`hostname`		Nome de máquina (em minúsculo)	`None`
`port`		Número da porta como inteiro, se presente	`None`

[1] (1,2,3,4)
Depending on the value of the *missing_as_none* argument.

Ler o atributo **`port`** irá levantar uma **`ValueError`** se uma porta inválida for especificada no URL. Veja a seção Structured Parse Results para mais informações sobre o objeto de resultado.

Colchetes sem correspondência no atributo **`netloc`** levantará uma **`ValueError`**.

Caracteres no atributo **`netloc`** que se decompõem sob a normalização NFKC (como usado pela codificação IDNA) em qualquer um dos `/`, `?`, `#`, `@` ou `:` vai levantar uma **`ValueError`**. Se a URL for decomposta antes da análise, nenhum erro será levantado.

Following some of the WHATWG spec [https://url.spec.whatwg.org/#concept-basic-url-parser] that updates **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], leading C0 control and space characters are stripped from the URL. `\n`, `\r` and tab `\t` characters are removed from the URL at any position.

As is the case with all named tuples, the subclass has a few additional methods and attributes that are particularly useful. One such method is **`_replace()`**. The **`_replace()`** method will return a new **`SplitResult`** object replacing specified fields with new values.

```
>>> from urllib.parse import urlsplit
>>> u = urlsplit('//www.cwi.nl:80/%7Eguido/Python.html')
>>> u
SplitResult(scheme='', netloc='www.cwi.nl:80', path='/%7Eguic
            query='', fragment='')
>>> u._replace(scheme='http')
SplitResult(scheme='http', netloc='www.cwi.nl:80', path='/%7E
            query='', fragment='')
```

Aviso

`urlsplit()` não executa validação. Veja Segurança ao analisar URLs para detalhes.

Alterado na versão 3.2: Adicionados recursos de análise de URL IPv6.

Alterado na versão 3.3: O fragmento agora é analisado para todos os esquemas de URL (a menos que *allow_fragments* seja falso), de acordo com a **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html]. Anteriormente, existia uma lista de permitidos de esquemas que suportam fragmentos.

Alterado na versão 3.6: Out-of-range port numbers now raise `ValueError`, instead of returning `None`.

Alterado na versão 3.8: Os caracteres que afetam a análise de netloc sob normalização NFKC agora levantarão `ValueError`.

Alterado na versão 3.10: Caracteres de nova linha e tabulação ASCII são removidos do URL.

Alterado na versão 3.12: Os caracteres iniciais de espaço e de controle WHATWG C0 são removidos do URL.

Alterado na versão 3.15: Added the *missing_as_none* parameter.

urllib.parse.parse_qs(*qs*, *keep_blank_values=False*, *strict_parsing=False*, *encoding='utf-8'*, *errors='replace'*, *max_num_fields=None*, *separator='&'*)

Analisa uma string de consulta fornecida como um argumento de string (dados do tipo *application/x-www-form-urlencoded*). Os dados são retornados como um dicionário. As chaves de dicionário são os nomes de variáveis de consulta exclusivos e os valores são listas de valores para cada nome.

O argumento opcional *keep_blank_values* é um sinalizador que indica se os valores em branco em consultas codificadas por porcentagem devem ser tratados como strings em branco. Um valor verdadeiro indica que os espaços em branco devem ser mantidos como strings em branco. O valor falso padrão indica que os valores em branco devem ser ignorados e tratados como se não tivessem sido incluídos.

O argumento opcional *strict_parsing* é um sinalizador que indica o que fazer com os erros de análise. Se falso (padrão), os erros são ignorados silenciosamente. Se verdadeiro, os erros levantam uma exceção `ValueError`.

Os parâmetros opcionais *encoding* e *errors* especificam como decodificar sequências codificadas em porcentagem em caracteres Unicode, conforme aceito pelo método `bytes.decode()`.

O argumento opcional *max_num_fields* é o número máximo de campos a serem lidos. Se definido, então levanta um `ValueError` se houver mais de *max_num_fields* campos lidos.

O argumento opcional *separador* é o símbolo a ser usado para separar os argumentos da consulta. O padrão é `&`.

Use a função `urllib.parse.urlencode()` (com o parâmetro `doseq` definido como `True`) para converter esses dicionários em strings de consulta.

Alterado na versão 3.2: Adicionado os parâmetros *encoding* e *errors*.

Alterado na versão 3.8: Adicionado o parâmetro *max_num_fields*.

Alterado na versão 3.10: Adicionado parâmetro *separator* com o valor padrão de `&`. Versões do Python anteriores ao Python 3.10 permitiam o uso de `;` e `&` como separador de parâmetro de consulta. Isso foi alterado para permitir apenas uma única chave separadora, com `&` como o separador padrão.

Descontinuado desde a versão 3.14: Accepting objects with false values (like `0` and `[]`) except empty strings and byte-like objects and `None` is now deprecated.

urllib.parse.parse_qsl(*qs*, *keep_blank_values=False*, *strict_parsing=False*, *encoding='utf-8'*, *errors='replace'*, *max_num_fields=None*, *separator='&'*)

Analisa uma string de consulta fornecida como um argumento de string (dados do tipo *application/x-www-form-urlencoded*). Os dados são retornados como uma lista de pares de nome e valor.

O argumento opcional *keep_blank_values* é um sinalizador que indica se os valores em branco em consultas codificadas por porcentagem devem ser tratados como strings em branco. Um valor verdadeiro indica

que os espaços em branco devem ser mantidos como strings em branco. O valor falso padrão indica que os valores em branco devem ser ignorados e tratados como se não tivessem sido incluídos.

O argumento opcional *strict_parsing* é um sinalizador que indica o que fazer com os erros de análise. Se falso (padrão), os erros são ignorados silenciosamente. Se verdadeiro, os erros levantam uma exceção `ValueError`.

Os parâmetros opcionais *encoding* e *errors* especificam como decodificar sequências codificadas em porcentagem em caracteres Unicode, conforme aceito pelo método `bytes.decode()`.

O argumento opcional *max_num_fields* é o número máximo de campos a serem lidos. Se definido, então levanta um `ValueError` se houver mais de *max_num_fields* campos lidos.

O argumento opcional *separador* é o símbolo a ser usado para separar os argumentos da consulta. O padrão é `&`.

Use a função `urllib.parse.urlencode()` para converter essas listas pares de strings de consulta.

Alterado na versão 3.2: Adicionado os parâmetros *encoding* e *errors*.

Alterado na versão 3.8: Adicionado o parâmetro *max_num_fields*.

Alterado na versão 3.10: Adicionado parâmetro *separator* com o valor padrão de `&`. Versões do Python anteriores ao Python 3.10 permitiam o uso de `;` e `&` como separador de parâmetro de consulta. Isso foi alterado para permitir apenas uma única chave separadora, com `&` como o separador padrão.

urllib.parse.urlunsplit(*parts*)
urllib.parse.urlunsplit(*parts*, *, *keep_empty*)

Construct a URL from a tuple as returned by `urlsplit()`. The *parts* argument can be any five-item iterable.

This may result in a slightly different, but equivalent URL, if the URL that was parsed originally had unnecessary delimiters (for example, a `?` with an empty query; the RFC states that these are equivalent).

If *keep_empty* is true, empty strings are kept in the result (for example, a `?` for an empty query), only `None` components are omitted. This allows rebuilding a URL that was parsed with option `missing_as_none=True`. By default, *keep_empty* is true if *parts* is the result of the **`urlsplit()`** call with `missing_as_none=True`.

Alterado na versão 3.15: Added the *keep_empty* parameter.

urllib.parse.urlparse(*urlstring*, *scheme=None*, *allow_fragments=True*, *, *missing_as_none=False*)

This is similar to **`urlsplit()`**, but additionally splits the *path* component on *path* and *params*. This function returns a 6-item named tuple **`ParseResult`** or **`ParseResultBytes`**. Its items are the same as for the **`urlsplit()`** result, except that *params* is inserted at index 3, between *path* and *query*.

This function is based on obsoleted **RFC 1738** [https://datatracker.ietf.org/doc/html/rfc1738.html] and **RFC 1808** [https://datatracker.ietf.org/doc/html/rfc1808.html], which listed *params* as the main URL component. The more recent URL syntax allows parameters to be applied to each segment of the *path* portion of the URL (see **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html]). **`urlsplit()`** should generally be used instead of **`urlparse()`**. A separate function is needed to separate the path segments and parameters.

urllib.parse.urlunparse(*parts*)

urllib.parse.urlunparse(*parts*, *, *keep_empty*)

Combine the elements of a tuple as returned by **`urlparse()`** into a complete URL as a string. The *parts* argument can be any six-item iterable.

This may result in a slightly different, but equivalent URL, if the URL that was parsed originally had unnecessary delimiters (for example, a `?`

with an empty query; the RFC states that these are equivalent).

If *keep_empty* is true, empty strings are kept in the result (for example, a `?` for an empty query), only `None` components are omitted. This allows rebuilding a URL that was parsed with option `missing_as_none=True`. By default, *keep_empty* is true if *parts* is the result of the **`urlparse()`** call with `missing_as_none=True`.

Alterado na versão 3.15: Added the *keep_empty* parameter.

urllib.parse.urljoin(*base*, *url*, *allow_fragments=True*)

Constrói uma URL completa ("absoluta") combinando uma "URL base" (*base*) com outra URL (*url*). Informalmente, isso usa componentes da URL base, em particular o esquema de endereçamento, o local da rede e (parte do) caminho, para fornecer componentes ausentes na URL relativa. Por exemplo:

```
>>> from urllib.parse import urljoin
>>> urljoin('http://www.cwi.nl/%7Eguido/Python.html', 'FAQ.ht
'http://www.cwi.nl/%7Eguido/FAQ.html'
```

The *allow_fragments* argument has the same meaning and default as for **`urlsplit()`**.

Nota

Se *url* for uma URL absoluta (ou seja, começa com `//` ou `scheme://`), o nome do host e/ou esquema da *url* estarão presentes no resultado. Por exemplo:

```
>>> urljoin('http://www.cwi.nl/%7Eguido/Python.html',
...         '//www.python.org/%7Eguido')
'http://www.python.org/%7Eguido'
```

Se você não quiser esse comportamento, pré-processe a *url* com **`urlsplit()`** e **`urlunsplit()`**, removendo possíveis partes *scheme* e *netloc*.

Aviso

Como uma URL absoluta pode ser passada como o parâmetro `url`, geralmente **não é seguro** usar `urljoin` com uma `url` controlada pelo invasor. Por exemplo, em `urljoin("https://website.com/users/", username)`, se `username` puder conter uma URL absoluta, o resultado de `urljoin` será a URL absoluta.

Alterado na versão 3.5: Comportamento atualizado para corresponder à semântica definida no **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html].

urllib.parse.urldefrag(*url*, *, *missing_as_none=False*)

If *url* contains a fragment identifier, return a modified version of *url* with no fragment identifier, and the fragment identifier as a separate string. If there is no fragment identifier in *url*, return *url* unmodified and an empty string (by default) or `None` if *missing_as_none* is true.

O valor de retorno é uma tupla nomeada, seus itens podem ser acessados por índice ou como atributos nomeados:

Atributo	Índice	Valor	Valor, se não presente
`url`	0	URL com nenhum fragmento	string vazia
`fragment`	1	Identificador do fragmento	`None` or empty string [[3]]

[3]
Depending on the value of the *missing_as_none* argument.

Veja a seção Structured Parse Results para obter mais informações sobre o objeto de resultado.

Alterado na versão 3.2: O resultado é um objeto estruturado em vez de uma simples tupla de 2 elementos.

Alterado na versão 3.15: Added the *missing_as_none* parameter.

urllib.parse.unwrap(*url*)

Extrai a URL de uma URL encapsulada (ou seja, uma string formatada como `<URL:esquema://host/caminho>`, `<esquema://host/caminho>`, `URL:esquema://host/caminho` ou `esquema://host/caminho`). Se *url* não for uma URL encapsulada, ela será retornada sem alterações.

Segurança ao analisar URLs

As APIs **`urlsplit()`** e **`urlparse()`** não realizam **validação** de entradas. Elas podem não levantar erros em entradas que outras aplicações consideram inválidas. Elas também podem ter sucesso em algumas entradas que podem não ser consideradas URLs em outros lugares. Seu propósito é para funcionalidade prática em vez de pureza.

Instead of raising an exception on unusual input, they may instead return some component parts as empty strings or `None` (depending on the value of the *missing_as_none* argument). Or components may contain more than perhaps they should.

Recomendamos que os usuários dessas APIs, onde os valores podem ser usados em qualquer lugar com implicações de segurança, codifiquem defensivamente. Faça alguma verificação dentro do seu código antes de confiar em uma parte do componente retornada. Esse `esquema` faz sentido? Esse `caminho` sensato? Há algo estranho sobre esse `hostname`? etc.

O que constitui uma URL não é universalmente bem definido. Diferentes aplicações têm diferentes necessidades e restrições desejadas. Por exemplo, a especificação WHATWG [https://url.spec.whatwg.org/#concept-basic-url-parser] descreve o que os clientes web voltados para o usuário, como um navegador web, exigem. Por outro lado, **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html] é mais geral. Essas funções incorporam alguns aspectos de ambos, mas não podem ser reivindicadas como compatíveis com nenhum deles. As APIs e o código de usuário existente com expectativas sobre comportamentos específicos são anteriores

a ambos os padrões, o que nos leva a ser muito cautelosos ao fazer alterações no comportamento da API.

Analisando bytes codificados em ASCII

As funções de análise de URL foram originalmente projetadas para operar somente em strings de caracteres. Na prática, é útil ser capaz de manipular URLs corretamente citadas e codificadas como sequências de bytes ASCII. Consequentemente, todas as funções de análise de URL neste módulo operam em objetos **bytes** e **bytearray** além de objetos **str**.

Se dados **str** forem passados, o resultado também conterá apenas dados **str**. Se dados **bytes** ou **bytearray** forem passados, o resultado conterá apenas dados **bytes**.

Tentar misturar dados **str** com **bytes** ou **bytearray** em uma única chamada de função resultará em uma **TypeError** sendo levantada, enquanto tentar passar valores de bytes não-ASCII acionará **UnicodeDecodeError**.

Para prover uma conversão mais fácil de objetos de resultado entre **str** e **bytes**, todos os valores de retorno das funções de análise de URL fornecem um método **encode()** (quando o resultado contém dados **str**) ou um método **decode()** (quando o resultado contém dados **bytes**). As assinaturas desses métodos correspondem às dos métodos **str** e **bytes** correspondentes (exceto que a codificação padrão é `'ascii'` em vez de `'utf-8'`). Cada um produz um valor de um tipo correspondente que contém dados **bytes** (para métodos **encode()**) ou dados **str** (para métodos **decode()**).

Os aplicações que precisam operar em URLs possivelmente envolta em aspas de forma incorreta, e que podem conter dados não ASCII, precisarão fazer sua própria decodificação de bytes para caracteres antes de invocar os métodos de análise de URL.

O comportamento descrito nesta seção se aplica somente às funções de análise de URL. As funções de aplicação de aspas em URL usam suas próprias regras ao produzir ou consumir sequências de bytes, conforme detalhado na documentação das funções de citação de URL individuais.

Alterado na versão 3.2: As funções de análise de URL agora aceitam sequências de bytes codificadas em ASCII

Structured Parse Results

The result objects from the `urlsplit()`, `urlparse()` and `urldefrag()` functions are subclasses of the `tuple` type. These subclasses add the attributes listed in the documentation for those functions, the encoding and decoding support described in the previous section, as well as an additional method:

urllib.parse.SplitResult.geturl()

Return the re-combined version of the original URL as a string. This may differ from the original URL in that the scheme may be normalized to lower case and empty components may be dropped. Specifically, empty parameters, queries, and fragment identifiers will be removed unless the URL was parsed with `missing_as_none=True`.

For `urldefrag()` results, only empty fragment identifiers will be removed. For `urlsplit()` and `urlparse()` results, all noted changes will be made to the URL returned by this method.

The result of this method remains unchanged if passed back through the original parsing function:

```
>>> from urllib.parse import urlsplit
>>> url = 'HTTP://www.Python.org/doc/#'
>>> r1 = urlsplit(url)
>>> r1.geturl()
'http://www.Python.org/doc/'
>>> r2 = urlsplit(r1.geturl())
>>> r2.geturl()
'http://www.Python.org/doc/'
>>> r3 = urlsplit(url, missing_as_none=True)
>>> r3.geturl()
'http://www.Python.org/doc/#'
```

The following classes provide the implementations of the structured parse results when operating on `str` objects:

class urllib.parse.DefragResult(*url*, *fragment*)

Concrete class for `urldefrag()` results containing `str` data. The `encode()` method returns a `DefragResultBytes` instance.

Adicionado na versão 3.2.

class urllib.parse.ParseResult(*scheme*, *netloc*, *path*, *params*, *query*, *fragment*)

Concrete class for `urlparse()` results containing `str` data. The `encode()` method returns a `ParseResultBytes` instance.

class urllib.parse.SplitResult(*scheme*, *netloc*, *path*, *query*, *fragment*)

Concrete class for `urlsplit()` results containing `str` data. The `encode()` method returns a `SplitResultBytes` instance.

The following classes provide the implementations of the parse results when operating on `bytes` or `bytearray` objects:

class urllib.parse.DefragResultBytes(*url*, *fragment*)

Concrete class for `urldefrag()` results containing `bytes` data. The `decode()` method returns a `DefragResult` instance.

Adicionado na versão 3.2.

class urllib.parse.ParseResultBytes(*scheme*, *netloc*, *path*, *params*, *query*, *fragment*)

Concrete class for `urlparse()` results containing `bytes` data. The `decode()` method returns a `ParseResult` instance.

Adicionado na versão 3.2.

class urllib.parse.SplitResultBytes(*scheme*, *netloc*, *path*, *query*, *fragment*)

Concrete class for `urlsplit()` results containing `bytes` data. The `decode()` method returns a `SplitResult` instance.

Adicionado na versão 3.2.

URL Quoting

The URL quoting functions focus on taking program data and making it safe for use as URL components by quoting special characters and appropriately encoding non-ASCII text. They also support reversing these operations to recreate the original data from the contents of a URL component if that task isn't already covered by the URL parsing functions above.

urllib.parse.quote(*string*, *safe='/'*, *encoding=None*, *errors=None*)

Replace special characters in *string* using the `%xx` escape. Letters, digits, and the characters `'_.-~'` are never quoted. By default, this function is intended for quoting the path section of a URL. The optional *safe* parameter specifies additional ASCII characters that should not be quoted — its default value is `'/'`.

string may be either a `str` or a `bytes` object.

Alterado na versão 3.7: Moved from **RFC 2396** [https://datatracker.ietf.org/doc/html/rfc2396.html] to **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html] for quoting URL strings. "~" is now included in the set of unreserved characters.

The optional *encoding* and *errors* parameters specify how to deal with non-ASCII characters, as accepted by the `str.encode()` method. Although these parameters default to `None` in the function signature, when processing `str` inputs, *encoding* effectively defaults to `'utf-8'` and *errors* to `'strict'`, meaning unsupported characters raise a `UnicodeEncodeError`. *encoding* and *errors* must not be supplied if *string* is a `bytes`, or a `TypeError` is raised.

Note that `quote(string, safe, encoding, errors)` is equivalent to `quote_from_bytes(string.encode(encoding, errors), safe)`.

Example: `quote('/El Niño/')` yields `'/El%20Ni%C3%B1o/'`.

urllib.parse.quote_plus(*string*, *safe=''*, *encoding=None*, *errors=None*)

Like **`quote()`**, but also replace spaces with plus signs, as required for quoting HTML form values when building up a query string to go into a URL. Plus signs in the original string are escaped unless they are included in *safe*. It also does not have *safe* default to `'/'`.

Example: `quote_plus('/El Niño/')` yields `'%2FEl+Ni%C3%B1o%2F'`.

urllib.parse.quote_from_bytes(*bytes*, *safe='/'*)

Like **`quote()`**, but accepts a **`bytes`** object rather than a **`str`**, and does not perform string-to-bytes encoding.

Example: `quote_from_bytes(b'a&\xef')` yields `'a%26%EF'`.

urllib.parse.unquote(*string*, *encoding='utf-8'*, *errors='replace'*)

Replace `%xx` escapes with their single-character equivalent. The optional *encoding* and *errors* parameters specify how to decode percent-encoded sequences into Unicode characters, as accepted by the **`bytes.decode()`** method.

string may be either a **`str`** or a **`bytes`** object.

encoding defaults to `'utf-8'`. *errors* defaults to `'replace'`, meaning invalid sequences are replaced by a placeholder character.

Example: `unquote('/El%20Ni%C3%B1o/')` yields `'/El Niño/'`.

Alterado na versão 3.9: string parameter supports bytes and str objects (previously only str).

urllib.parse.unquote_plus(*string*, *encoding='utf-8'*, *errors='replace'*)

Like **unquote()**, but also replace plus signs with spaces, as required for unquoting HTML form values.

string must be a **str**.

Example: `unquote_plus('/El+Ni%C3%B1o/')` yields `'/El Niño/'`.

urllib.parse.unquote_to_bytes(*string*)

Replace `%xx` escapes with their single-octet equivalent, and return a **bytes** object.

string may be either a **str** or a **bytes** object.

If it is a **str**, unescaped non-ASCII characters in *string* are encoded into UTF-8 bytes.

Example: `unquote_to_bytes('a%26%EF')` yields `b'a&\xef'`.

urllib.parse.urlencode(*query*, *doseq=False*, *safe=''*, *encoding=None*, *errors=None*, *quote_via=quote_plus*)

Convert a mapping object or a sequence of two-element tuples, which may contain **str** or **bytes** objects, to a percent-encoded ASCII text string. If the resultant string is to be used as a *data* for POST operation with the **urlopen()** function, then it should be encoded to bytes, otherwise it would result in a **TypeError**.

The resulting string is a series of `key=value` pairs separated by `'&'` characters, where both *key* and *value* are quoted using the *quote_via* function. By default, **quote_plus()** is used to quote the values, which means spaces are quoted as a `'+'` character and '/' characters are encoded as `%2F`, which follows the standard for GET requests (`application/x-www-form-urlencoded`). An alternate function that can be passed as *quote_via* is **quote()**, which will encode spaces as `%20` and not encode '/' characters. For maximum control of what is quoted, use `quote` and specify a value for *safe*.

When a sequence of two-element tuples is used as the *query* argument, the first element of each tuple is a key and the second is a value. The value element in itself can be a sequence and in that case, if the optional parameter *doseq* evaluates to `True`, individual `key=value` pairs separated by `'&'` are generated for each element of the value sequence for the key. The order of parameters in the encoded string will match the order of parameter tuples in the sequence.

The *safe*, *encoding*, and *errors* parameters are passed down to *quote_via* (the *encoding* and *errors* parameters are only passed when a query element is a **`str`**).

To reverse this encoding process, **`parse_qs()`** and **`parse_qsl()`** are provided in this module to parse query strings into Python data structures.

Refer to urllib examples to find out how the **`urllib.parse.urlencode()`** method can be used for generating the query string of a URL or data for a POST request.

Alterado na versão 3.2: query supports bytes and string objects.

Alterado na versão 3.5: Added the *quote_via* parameter.

Descontinuado desde a versão 3.14: Accepting objects with false values (like `0` and `[]`) except empty strings and byte-like objects and `None` is now deprecated.

Ver também

WHATWG [https://url.spec.whatwg.org/] - URL Living standard
Working Group for the URL Standard that defines URLs, domains, IP addresses, the application/x-www-form-urlencoded format, and their API.

RFC 3986 [https://datatracker.ietf.org/doc/html/rfc3986.html] - Uniform Resource Identifiers

This is the current standard (STD66). Any changes to urllib.parse module should conform to this. Certain deviations could be observed, which are mostly for backward compatibility purposes and for certain de-facto parsing requirements as commonly observed in major browsers.

RFC 2732 [https://datatracker.ietf.org/doc/html/rfc2732.html] - Format for Literal IPv6 Addresses in URL's.

This specifies the parsing requirements of IPv6 URLs.

RFC 2396 [https://datatracker.ietf.org/doc/html/rfc2396.html] - Uniform Resource Identifiers (URI): Generic Syntax

Document describing the generic syntactic requirements for both Uniform Resource Names (URNs) and Uniform Resource Locators (URLs).

RFC 2368 [https://datatracker.ietf.org/doc/html/rfc2368.html] - O esquema de URL mailto.

Parsing requirements for mailto URL schemes.

RFC 1808 [https://datatracker.ietf.org/doc/html/rfc1808.html] - Relative Uniform Resource Locators

This Request For Comments includes the rules for joining an absolute and a relative URL, including a fair number of "Abnormal Examples" which govern the treatment of border cases.

RFC 1738 [https://datatracker.ietf.org/doc/html/rfc1738.html] - Uniform Resource Locators (URL)

This specifies the formal syntax and semantics of absolute URLs.

`urllib.error` — Classes de exceção levantadas por urllib.request

Código-fonte: Lib/urllib/error.py
[https://github.com/python/cpython/tree/3.15/Lib/urllib/error.py]

O módulo **`urllib.error`** define as classes de exceção para exceções levantadas por **`urllib.request`**. A classe de exceção base é **`URLError`**.

As seguintes exceções são levantadas por **`urllib.error`** conforme apropriado:

exception urllib.error.URLError

Os manipuladores levantam essa exceção (ou exceções derivadas) quando encontram um problema. É uma subclasse de **`OSError`**.

reason

O motivo desse erro. Pode ser uma string de mensagem ou outra instância de exceção.

Alterado na versão 3.3: **`URLError`** costumava ser um subtipo de **`IOError`**, que agora é um apelido de **`OSError`**.

exception urllib.error.HTTPError(*url*, *code*, *msg*, *hdrs*, *fp*)

Embora seja uma exceção (uma subclasse de **`URLError`**), uma **`HTTPError`** também pode funcionar como um valor de retorno não excepcional do tipo arquivo (a mesma coisa que **`urlopen()`** retorna). Isso é útil ao lidar com erros de HTTP exóticos, como solicitações de autenticação.

url

Contém a solicitação URL. Um apelido para *nome do arquivo* atributo.

code

Um código de status HTTP conforme definido em **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html]. Este valor numérico corresponde a um valor encontrado no dicionário de códigos conforme encontrado em `http.server.BaseHTTPRequestHandler.responses`.

reason

Geralmente é uma string explicando o motivo desse erro. Um apelido para o atributo *msg*.

headers

Os cabeçalhos de resposta HTTP para a solicitação HTTP que causou a `HTTPError`. Um apelido para o atributo *hdrs*.

Adicionado na versão 3.4.

fp

Um objeto arquivo ou similar no qual o corpo do HTTP erro pode ser lido.

exception urllib.error.ContentTooShortError(*msg*, *content*)

Esta exceção é levantada quando a função `urlretrieve()` detecta que a quantidade de dados baixados é menor que a quantidade esperada (fornecida pelo cabeçalho *Content-Length*).

content

Os dados baixados (e supostamente truncados).

`urllib.robotparser` — Analisador sintático de robots.txt

Código-fonte: Lib/urllib/robotparser.py
[https://github.com/python/cpython/tree/3.15/Lib/urllib/robotparser.py]

Este módulo fornece uma única classe, **`RobotFileParser`**, que responde a perguntas sobre se um agente de usuário específico pode ou não buscar uma URL no site que publicou o arquivo `robots.txt`. Para mais detalhes sobre a estrutura dos arquivos `robots.txt`, consulte **RFC 9309**
[https://datatracker.ietf.org/doc/html/rfc9309.html].

class urllib.robotparser.RobotFileParser(*url=''*)

Esta classe fornece métodos para ler, analisar e responder perguntas sobre o arquivo `robots.txt` em *url*.

set_url(*url*)

Define a URL referente a um arquivo `robots.txt`.

read()

Lê o URL `robots.txt` e o alimenta para o analisador sintático.

parse(*lines*)

Analisa o argumento de linhas.

can_fetch(*useragent*, *url*)

Retorna `True` se o *useragent* tiver permissão para buscar a *url* de acordo com as regras contidas no arquivo `robots.txt` analisado.

mtime()

Retorna o horário em que o arquivo `robots.txt` foi buscado pela última vez. Isso é útil para web spiders de longa duração que

precisam verificar novos arquivos `robots.txt` periodicamente.

modified()

Define a hora em que o arquivo `robots.txt` foi buscado pela última vez para a hora atual.

crawl_delay(*useragent*)

Retorna o valor do parâmetro `Crawl-delay` de `robots.txt` para o *useragent* em questão. Se não houver tal parâmetro ou se ele não se aplicar ao *useragent* especificado ou se a entrada `robots.txt` para este parâmetro tiver sintaxe inválida, retorna `None`.

Adicionado na versão 3.6.

request_rate(*useragent*)

Retorna o conteúdo do parâmetro `Request-rate` de `robots.txt` como uma tupla nomeada `RequestRate(requests, seconds)`. Se não houver tal parâmetro ou se ele não se aplicar ao *useragent* especificado ou se a entrada `robots.txt` para este parâmetro tiver sintaxe inválida, retorna `None`.

Adicionado na versão 3.6.

site_maps()

Retorna o conteúdo do parâmetro `Sitemap` de `robots.txt` no formato de uma **`list()`**. Se não houver tal parâmetro ou a entrada `robots.txt` para este parâmetro tiver sintaxe inválida, retorna `None`.

Adicionado na versão 3.8.

O exemplo a seguir demonstra o uso básico da classe **`RobotFileParser`**:

```
>>> import urllib.robotparser
>>> rp = urllib.robotparser.RobotFileParser()
>>> rp.set_url("http://www.pythontest.net/robots.txt")
>>> rp.read()
>>> rrate = rp.request_rate("*")
>>> rrate.requests
```

```
1
>>> rrate.seconds
1
>>> rp.crawl_delay("*")
6
>>> rp.can_fetch("*", "http://www.pythontest.net/")
True
>>> rp.can_fetch("*", "http://www.pythontest.net/no-robots-here/
False
```

`http` — Módulos HTTP

Código-fonte: Lib/http/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/http/__init__.py]

`http` é um pacote que agrupa vários módulos para trabalhar com o Protocolo de Transferência de Hipertexto:

- **`http.client`** é um cliente de protocolo HTTP de baixo nível; para abertura de URL de alto nível, use **`urllib.request`**
- **`http.server`** contém classes básicas de servidor HTTP baseadas em **`socketserver`**
- **`http.cookies`** tem utilitários para implementar gerenciamento de estado com cookies
- **`http.cookiejar`** provê persistência de cookies

O módulo **`http`** também define as seguintes enumerações que podem ajudar você a trabalhar com os códigos relacionados ao HTTP:

class http.HTTPStatus

Adicionado na versão 3.5.

Subclasse de **`enum.IntEnum`** que define um conjunto de códigos de estado HTTP, frases de razão e descrições longas em inglês.

Uso:

```
>>> from http import HTTPStatus
>>> HTTPStatus.OK
HTTPStatus.OK
>>> HTTPStatus.OK == 200
True
>>> HTTPStatus.OK.value
200
>>> HTTPStatus.OK.phrase
'OK'
```

```
>>> HTTPStatus.OK.description
'Request fulfilled, document follows'
>>> list(HTTPStatus)
[HTTPStatus.CONTINUE, HTTPStatus.SWITCHING_PROTOCOLS, ...]
```

Códigos de estado HTTP

Os [códigos de estado registrados no IANA](https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml) [https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml] suportados e disponíveis em `http.HTTPStatus` são:

Código	Nome da enumeração	Detalhes (em inglês)
100	`CONTINUE`	HTTP Semantics **RFC 9110**, Seção 15.2.1
101	`SWITCHING_PROTOCOLS`	HTTP Semantics **RFC 9110**, Seção 15.2.2
102	`PROCESSING`	WebDAV **RFC 2518**, Seção 10.1
103	`EARLY_HINTS`	An HTTP Status Code for Indicating Hints **RFC 8297**
200	`OK`	HTTP Semantics **RFC 9110**, Seção 15.3.1

Código	Nome da enumeração	Detalhes (em inglês)
201	CREATED	HTTP Semantics **RFC 9110**, Seção 15.3.2
202	ACCEPTED	HTTP Semantics **RFC 9110**, Seção 15.3.3
203	NON_AUTHORITATIVE_INFORMATION	HTTP Semantics **RFC 9110**, Seção 15.3.4
204	NO_CONTENT	HTTP Semantics **RFC 9110**, Seção 15.3.5
205	RESET_CONTENT	HTTP Semantics **RFC 9110**, Seção 15.3.6
206	PARTIAL_CONTENT	HTTP Semantics **RFC 9110**, Seção 15.3.7
207	MULTI_STATUS	WebDAV **RFC 4918**, Seção 11.1
208	ALREADY_REPORTED	WebDAV Binding Extensions **RFC 5842**, Seção 7.1 (Experimental)

Código	Nome da enumeração	Detalhes (em inglês)
226	IM_USED	Delta Encoding in HTTP **RFC 3229**, Seção 10.4.1
300	MULTIPLE_CHOICES	HTTP Semantics **RFC 9110**, Seção 15.4.1
301	MOVED_PERMANENTLY	HTTP Semantics **RFC 9110**, Seção 15.4.2
302	FOUND	HTTP Semantics **RFC 9110**, Seção 15.4.3
303	SEE_OTHER	HTTP Semantics **RFC 9110**, Seção 15.4.4
304	NOT_MODIFIED	HTTP Semantics **RFC 9110**, Seção 15.4.5
305	USE_PROXY	HTTP Semantics **RFC 9110**, Seção 15.4.6
307	TEMPORARY_REDIRECT	HTTP Semantics **RFC 9110**, Seção 15.4.8

Código	Nome da enumeração	Detalhes (em inglês)
308	PERMANENT_REDIRECT	HTTP Semantics **RFC 9110**, Seção 15.4.9
400	BAD_REQUEST	HTTP Semantics **RFC 9110**, Seção 15.5.1
401	UNAUTHORIZED	HTTP Semantics **RFC 9110**, Seção 15.5.2
402	PAYMENT_REQUIRED	HTTP Semantics **RFC 9110**, Seção 15.5.3
403	FORBIDDEN	HTTP Semantics **RFC 9110**, Seção 15.5.4
404	NOT_FOUND	HTTP Semantics **RFC 9110**, Seção 15.5.5
405	METHOD_NOT_ALLOWED	HTTP Semantics **RFC 9110**, Seção 15.5.6
406	NOT_ACCEPTABLE	HTTP Semantics **RFC 9110**, Seção 15.5.7

Código	Nome da enumeração	Detalhes (em inglês)
407	PROXY_AUTHENTICATION_REQUIRED	HTTP Semantics **RFC 9110**, Seção 15.5.8
408	REQUEST_TIMEOUT	HTTP Semantics **RFC 9110**, Seção 15.5.9
409	CONFLICT	HTTP Semantics **RFC 9110**, Seção 15.5.10
410	GONE	HTTP Semantics **RFC 9110**, Seção 15.5.11
411	LENGTH_REQUIRED	HTTP Semantics **RFC 9110**, Seção 15.5.12
412	PRECONDITION_FAILED	HTTP Semantics **RFC 9110**, Seção 15.5.13
413	CONTENT_TOO_LARGE	HTTP Semantics **RFC 9110**, Seção 15.5.14
414	URI_TOO_LONG	HTTP Semantics **RFC 9110**, Seção 15.5.15

Código	Nome da enumeração	Detalhes (em inglês)
415	`UNSUPPORTED_MEDIA_TYPE`	HTTP Semantics **RFC 9110**, Seção 15.5.16
416	`RANGE_NOT_SATISFIABLE`	HTTP Semantics **RFC 9110**, Seção 15.5.17
417	`EXPECTATION_FAILED`	HTTP Semantics **RFC 9110**, Seção 15.5.18
418	`IM_A_TEAPOT`	HTCPCP/1.0 **RFC 2324**, Seção 2.3.2
421	`MISDIRECTED_REQUEST`	HTTP Semantics **RFC 9110**, Seção 15.5.20
422	`UNPROCESSABLE_CONTENT`	HTTP Semantics **RFC 9110**, Seção 15.5.21
423	`LOCKED`	WebDAV **RFC 4918**, Seção 11.3
424	`FAILED_DEPENDENCY`	WebDAV **RFC 4918**, Seção 11.4

Código	Nome da enumeração	Detalhes (em inglês)
425	TOO_EARLY	Using Early Data in HTTP **RFC 8470**
426	UPGRADE_REQUIRED	HTTP Semantics **RFC 9110**, Seção 15.5.22
428	PRECONDITION_REQUIRED	Additional HTTP Status Codes **RFC 6585**
429	TOO_MANY_REQUESTS	Additional HTTP Status Codes **RFC 6585**
431	REQUEST_HEADER_FIELDS_TOO_LARGE	Additional HTTP Status Codes **RFC 6585**
451	UNAVAILABLE_FOR_LEGAL_REASONS	An HTTP Status Code to Report Legal Obstacles **RFC 7725**
500	INTERNAL_SERVER_ERROR	HTTP Semantics **RFC 9110**, Seção 15.6.1
501	NOT_IMPLEMENTED	HTTP Semantics **RFC 9110**, Seção 15.6.2

Código	Nome da enumeração	Detalhes (em inglês)
502	BAD_GATEWAY	HTTP Semantics **RFC 9110**, Seção 15.6.3
503	SERVICE_UNAVAILABLE	HTTP Semantics **RFC 9110**, Seção 15.6.4
504	GATEWAY_TIMEOUT	HTTP Semantics **RFC 9110**, Seção 15.6.5
505	HTTP_VERSION_NOT_SUPPORTED	HTTP Semantics **RFC 9110**, Seção 15.6.6
506	VARIANT_ALSO_NEGOTIATES	Transparent Content Negotiation in HTTP **RFC 2295**, Seção 8.1 (Experimental)
507	INSUFFICIENT_STORAGE	WebDAV **RFC 4918**, Seção 11.5
508	LOOP_DETECTED	WebDAV Binding Extensions **RFC 5842**, Seção 7.2 (Experimental)

Código	Nome da enumeração	Detalhes (em inglês)
510	NOT_EXTENDED	An HTTP Extension Framework **RFC 2774**, Seção 7 (Experimental)
511	NETWORK_AUTHENTICATION_REQUIRED	Additional HTTP Status Codes **RFC 6585**, Seção 6

Para preservar a retrocompatibilidade, os valores de enumerações também estão presentes no módulo `http.client` na forma de constantes. O nome da enumeração é igual ao da constante (i.e., `http.HTTPStatus.OK` também está disponível como `http.client.OK`).

Alterado na versão 3.7: Adicionado código de estado `421 MISDIRECTED_REQUEST`.

Adicionado na versão 3.8: Adicionado código de estado `451 UNAVAILABLE_FOR_LEGAL_REASONS`.

Adicionado na versão 3.9: Adicionado os códigos de estado `103 EARLY_HINTS`, `418 IM_A_TEAPOT` e `425 TOO_EARLY`.

Alterado na versão 3.13: Implementada a nomenclatura do RFC9110 para constantes de estado. Os nomes antigos das constantes foram mantidos para retrocompatibilidade: `413 REQUEST_ENTITY_TOO_LARGE`, `414 REQUEST_URI_TOO_LONG`, `416 REQUESTED_RANGE_NOT_SATISFIABLE` e `422 UNPROCESSABLE_ENTITY`.

Categoria de estado HTTP

Adicionado na versão 3.12.

Os valores das enumerações possuem várias propriedades que indicam a categoria do estado HTTP.

Propriedade	Indica que	Detalhes (em inglês)
`is_informational`	`100 <= estado <= 199`	HTTP Semantics **RFC 9110**, Seção 15
`is_success`	`200 <= estado <= 299`	HTTP Semantics **RFC 9110**, Seção 15
`is_redirection`	`300 <= estado <= 399`	HTTP Semantics **RFC 9110**, Seção 15
`is_client_error`	`400 <= estado <= 499`	HTTP Semantics **RFC 9110**, Seção 15
`is_server_error`	`500 <= estado <= 599`	HTTP Semantics **RFC 9110**, Seção 15

Uso:

```
>>> from http import HTTPStatus
>>> HTTPStatus.OK.is_success
True
>>> HTTPStatus.OK.is_client_error
False
```

class http.HTTPMethod

Adicionado na versão 3.11.

Subclasse de `enum.StrEnum` que define um conjunto de métodos HTTP e suas respectivas descrições em inglês.

Uso:

```
>>> from http import HTTPMethod
>>>
>>> HTTPMethod.GET
<HTTPMethod.GET>
>>> HTTPMethod.GET == 'GET'
True
>>> HTTPMethod.GET.value
'GET'
>>> HTTPMethod.GET.description
'Retrieve the target.'
>>> list(HTTPMethod)
[<HTTPMethod.CONNECT>,
 <HTTPMethod.DELETE>,
 <HTTPMethod.GET>,
 <HTTPMethod.HEAD>,
 <HTTPMethod.OPTIONS>,
 <HTTPMethod.PATCH>,
 <HTTPMethod.POST>,
 <HTTPMethod.PUT>,
 <HTTPMethod.TRACE>]
```

Métodos HTTP

Os métodos registrados no IANA [https://www.iana.org/assignments/http-methods/http-methods.xhtml] suportados e disponíveis em `http.HTTPMethod` são:

Método	Nome da enumeração	Detalhes (em inglês)
`GET`	`GET`	HTTP Semantics **RFC 9110**, Seção 9.3.1

Método	Nome da enumeração	Detalhes (em inglês)
`HEAD`	`HEAD`	HTTP Semantics **RFC 9110**, Seção 9.3.2
`POST`	`POST`	HTTP Semantics **RFC 9110**, Seção 9.3.3
`PUT`	`PUT`	HTTP Semantics **RFC 9110**, Seção 9.3.4
`DELETE`	`DELETE`	HTTP Semantics **RFC 9110**, Seção 9.3.5
`CONNECT`	`CONNECT`	HTTP Semantics **RFC 9110**, Seção 9.3.6
`OPTIONS`	`OPTIONS`	HTTP Semantics **RFC 9110**, Seção 9.3.7
`TRACE`	`TRACE`	HTTP Semantics **RFC 9110**, Seção 9.3.8
`PATCH`	`PATCH`	HTTP/1.1 **RFC 5789**

`http.client` — HTTP protocol client

Código-fonte: Lib/http/client.py
[https://github.com/python/cpython/tree/3.15/Lib/http/client.py]

This module defines classes that implement the client side of the HTTP and HTTPS protocols. It is normally not used directly — the module `urllib.request` uses it to handle URLs that use HTTP and HTTPS.

> **Ver também**
>
> The Requests package [https://requests.readthedocs.io/en/latest/] is recommended for a higher-level HTTP client interface.

> **Nota**
>
> Suporte HTTPS está disponível somente se Python foi compilado com suporte SSL (através do módulo `ssl`).

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

O módulo fornece as seguintes classes:

class http.client.HTTPConnection(*host*, *port=None*, [*timeout*,]*source_address=None*, *blocksize=8192*, *max_response_headers=None*)

An **`HTTPConnection`** instance represents one transaction with an HTTP server. It should be instantiated by passing it a host and optional port number. If no port number is passed, the port is extracted from the host string if it has the form `host:port`, else the default HTTP port (80) is used. If the optional *timeout* parameter is given, blocking operations (like connection attempts) will timeout after that many seconds (if it is not given, the global default timeout setting is used). The optional *source_address* parameter may be a tuple of a (host, port) to use as the source address the HTTP connection is made from. The optional *blocksize* parameter sets the buffer size in bytes for sending a file-like message body. The optional *max_response_headers* parameter sets the maximum number of allowed response headers to help prevent denial-of-service attacks, otherwise the default value (100) is used.

Por exemplo, todas as seguintes chamadas criam instâncias que conectam ao servidor com o mesmo host e porta:

```
>>> h1 = http.client.HTTPConnection('www.python.org')
>>> h2 = http.client.HTTPConnection('www.python.org:80')
>>> h3 = http.client.HTTPConnection('www.python.org', 80)
>>> h4 = http.client.HTTPConnection('www.python.org', 80, tim
```

Alterado na versão 3.2: source_address foi adicionado.

Alterado na versão 3.4: The *strict* parameter was removed. HTTP 0.9-style “Simple Responses” are no longer supported.

Alterado na versão 3.7: O argumento *blocksize* foi adicionado.

Alterado na versão 3.15: max_response_headers parameter was added.

class http.client.HTTPSConnection(*host*, *port=None*, *, [*timeout*,]*source_address=None*, *context=None*, *blocksize=8192*, *max_response_headers=None*)

Uma subclasse de **`HTTPConnection`** que utiliza SSL para comunicação com servidores seguros. A porta padrão é `443`. Se *context* for

especificado, ele deve ser uma instância de **ssl.SSLContext** descrevendo as várias opções do SSL.

Por favor leia Considerações de segurança para mais informações sobre as melhores práticas.

Alterado na versão 3.2: source_address, *context* e *check_hostname* foram adicionados.

Alterado na versão 3.2: This class now supports HTTPS virtual hosts if possible (that is, if **ssl.HAS_SNI** is true).

Alterado na versão 3.4: O argumento *strict* foi removido. “Respostas Simples” HTTP com o estilo 0.9 não são mais suportadas.

Alterado na versão 3.4.3: This class now performs all the necessary certificate and hostname checks by default. To revert to the previous, unverified, behavior **ssl._create_unverified_context()** can be passed to the *context* parameter.

Alterado na versão 3.8: Esta classe agora habilita TLS 1.3 **ssl.SSLContext.post_handshake_auth** para o padrão *context* ou quanto *cert_file* é fornecido com um *context* personalizado.

Alterado na versão 3.10: This class now sends an ALPN extension with protocol indicator `http/1.1` when no *context* is given. Custom *context* should set ALPN protocols with **set_alpn_protocols()**.

Alterado na versão 3.12: The deprecated *key_file*, *cert_file* and *check_hostname* parameters have been removed.

Alterado na versão 3.15: max_response_headers parameter was added.

class http.client.HTTPResponse(*sock*, *debuglevel=0*, *method=None*, *url=None*)

Classe em que instâncias são retornadas mediante de conexão bem-sucedida. Não é instanciável diretamente pelo usuário.

Alterado na versão 3.4: O argumento *strict* foi removido. “Respostas Simples” HTTP com o estilo 0.9 não são mais suportadas.

Este módulo fornece a seguinte função:

http.client.parse_headers(*fp*)

Parse the headers from a file pointer *fp* representing a HTTP request/response. The file has to be a `BufferedIOBase` reader (i.e. not text) and must provide a valid **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] style header.

Esta função retorna uma instância de `http.client.HTTPMessage` que armazena os campos do cabeçalho, mas não o payload (o mesmo que `HTTPResponse.msg` e `http.server.BaseHTTPRequestHandler.headers`). Depois de retornar, o ponteiro de arquivo *fp* está pronto para ler o corpo da requisição HTTP.

Nota

`parse_headers()` não analisa a linha inicial de uma mensagem HTTP; ele apenas analisa as linhas de `Name: value`. O arquivo deve estar pronto para ler essas linhas de campo, então a primeira linha já deve ter sido consumida antes de chamar a função.

As seguintes exceções são levantadas conforme apropriado:

exception http.client.HTTPException

A classe base das outras exceções neste módulo. É uma subclasse de `Exception`.

exception http.client.NotConnected

Uma subclasse de `HTTPException`.

exception http.client.InvalidURL

Uma subclasse de `HTTPException`, levantada se uma porta é fornecida e esta é não-numérica ou vazia.

exception http.client.UnknownProtocol

Uma subclasse de `HTTPException`.

exception http.client.UnknownTransferEncoding

Uma subclasse de `HTTPException`.

exception http.client.UnimplementedFileMode

Uma subclasse de `HTTPException`.

exception http.client.IncompleteRead

Uma subclasse de `HTTPException`.

exception http.client.ImproperConnectionState

Uma subclasse de `HTTPException`.

exception http.client.CannotSendRequest

Uma subclasse de `ImproperConnectionState`.

exception http.client.CannotSendHeader

Uma subclasse de `ImproperConnectionState`.

exception http.client.ResponseNotReady

Uma subclasse de `ImproperConnectionState`.

exception http.client.BadStatusLine

Uma subclasse de `HTTPException`. Levantada se um servidor responde com um código de status HTTP que não é entendido.

exception http.client.LineTooLong

Uma subclasse de `HTTPException`. Levantada se uma linha excessivamente longa é recebida, do servidor, no protocolo HTTP.

exception http.client.RemoteDisconnected

Uma subclasse de `ConnectionResetError` e `BadStatusLine`. Levantada por `HTTPConnection.getresponse()` quando a tentativa de ler a resposta resulta na não leitura dos dados pela conexão, indicando que o fim remoto fechou a conexão.

Adicionado na versão 3.5: Anteriormente, a exceção `BadStatusLine('')` foi levantada.

As constantes definidas neste módulo são:

http.client.HTTP_PORT
: A porta padrão para o protocolo HTTP (sempre `80`).

http.client.HTTPS_PORT
: A porta padrão para o protocolo HTTPS (sempre `443`).

http.client.responses
: Este dicionário mapeia os códigos de status HTTP 1.1 para os nomes em W3C.

  Exemplo: `http.client.responses[http.client.NOT_FOUND]` é `'Not Found'`.

Ver Códigos de estado HTTP para uma lista de códigos de status HTTP que estão disponíveis neste módulo como constantes.

Objetos de HTTPConnection

Instâncias `HTTPConnection` contêm os seguintes métodos:

HTTPConnection.request(*method*, *url*, *body=None*, *headers={}*, *, *encode_chunked=False*)
: This will send a request to the server using the HTTP request method *method* and the request URI *url*. The provided *url* must be an absolute path to conform with **RFC 2616 §5.1.2** [https://datatracker.ietf.org/doc/html/rfc2616.html#section-5.1.2] (unless connecting to an HTTP proxy server or using the `OPTIONS` or `CONNECT` methods).

  If *body* is specified, the specified data is sent after the headers are finished. It may be a `str`, a bytes-like object, an open file object, or an iterable of `bytes`. If *body* is a string, it is encoded as ISO-8859-1, the default for HTTP. If it is a bytes-like object, the bytes are sent as is. If it

is a file object, the contents of the file is sent; this file object should support at least the `read()` method. If the file object is an instance of **`io.TextIOBase`**, the data returned by the `read()` method will be encoded as ISO-8859-1, otherwise the data returned by `read()` is sent as is. If *body* is an iterable, the elements of the iterable are sent as is until the iterable is exhausted.

The *headers* argument should be a mapping of extra HTTP headers to send with the request. A **Host header** [https://datatracker.ietf.org/doc/html/rfc2616.html#section-14.23] must be provided to conform with **RFC 2616 §5.1.2** [https://datatracker.ietf.org/doc/html/rfc2616.html#section-5.1.2] (unless connecting to an HTTP proxy server or using the `OPTIONS` or `CONNECT` methods).

If *headers* contains neither Content-Length nor Transfer-Encoding, but there is a request body, one of those header fields will be added automatically. If *body* is `None`, the Content-Length header is set to `0` for methods that expect a body (`PUT`, `POST`, and `PATCH`). If *body* is a string or a bytes-like object that is not also a file, the Content-Length header is set to its length. Any other type of *body* (files and iterables in general) will be chunk-encoded, and the Transfer-Encoding header will automatically be set instead of Content-Length.

The *encode_chunked* argument is only relevant if Transfer-Encoding is specified in *headers*. If *encode_chunked* is `False`, the HTTPConnection object assumes that all encoding is handled by the calling code. If it is `True`, the body will be chunk-encoded.

For example, to perform a `GET` request to `https://docs.python.org/3/`:

```
>>> import http.client
>>> host = "docs.python.org"
>>> conn = http.client.HTTPSConnection(host)
>>> conn.request("GET", "/3/", headers={"Host": host})
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
200 OK
```

Nota

Chunked transfer encoding has been added to the HTTP protocol version 1.1. Unless the HTTP server is known to handle HTTP 1.1, the caller must either specify the Content-Length, or must pass a `str` or bytes-like object that is not also a file as the body representation.

Nota

Note that you must have read the whole response or call `close()` if `getresponse()` raised an non-`ConnectionError` exception before you can send a new request to the server.

Alterado na versão 3.2: body pode agora ser um iterável.

Alterado na versão 3.6: If neither Content-Length nor Transfer-Encoding are set in *headers*, file and iterable *body* objects are now chunk-encoded. The *encode_chunked* argument was added. No attempt is made to determine the Content-Length for file objects.

HTTPConnection.getresponse()

Should be called after a request is sent to get the response from the server. Returns an `HTTPResponse` instance.

Alterado na versão 3.5: Se uma `ConnectionError` ou subclasse for levantada, o objeto `HTTPConnection` estará pronto para se reconectar quando uma nova solicitação for enviada.

Note that this does not apply to `OSError`s raised by the underlying socket. Instead the caller is responsible to call `close()` on the existing connection.

HTTPConnection.set_debuglevel(*level*)

Set the debugging level. The default debug level is `0`, meaning no debugging output is printed. Any value greater than `0` will cause all currently defined debug output to be printed to stdout. The `debuglevel` is passed to any new `HTTPResponse` objects that are created.

Adicionado na versão 3.1.

HTTPConnection.set_tunnel(*host*, *port=None*, *headers=None*)

Set the host and the port for HTTP Connect Tunnelling. This allows running the connection through a proxy server.

The *host* and *port* arguments specify the endpoint of the tunneled connection (i.e. the address included in the CONNECT request, *not* the address of the proxy server).

The *headers* argument should be a mapping of extra HTTP headers to send with the CONNECT request.

As HTTP/1.1 is used for HTTP CONNECT tunnelling request, as per the RFC [https://datatracker.ietf.org/doc/html/rfc7231#section-4.3.6], a HTTP `Host:` header must be provided, matching the authority-form of the request target provided as the destination for the CONNECT request. If a HTTP `Host:` header is not provided via the headers argument, one is generated and transmitted automatically.

For example, to tunnel through a HTTPS proxy server running locally on port 8080, we would pass the address of the proxy to the **`HTTPSConnection`** constructor, and the address of the host that we eventually want to reach to the **`set_tunnel()`** method:

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("localhost", 8080)
>>> conn.set_tunnel("www.python.org")
>>> conn.request("HEAD","/index.html")
```

Adicionado na versão 3.2.

Alterado na versão 3.12: HTTP CONNECT tunnelling requests use protocol HTTP/1.1, upgraded from protocol HTTP/1.0. `Host:` HTTP headers are mandatory for HTTP/1.1, so one will be automatically generated and transmitted if not provided in the headers argument.

HTTPConnection.get_proxy_response_headers()

Returns a dictionary with the headers of the response received from the proxy server to the CONNECT request.

If the CONNECT request was not sent, the method returns `None`.

Adicionado na versão 3.12.

HTTPConnection.connect()

Connect to the server specified when the object was created. By default, this is called automatically when making a request if the client does not already have a connection.

Levanta um evento de auditoria `http.client.connect` com os argumentos `self`, `host`, `port`.

HTTPConnection.close()

Close the connection to the server.

HTTPConnection.blocksize

Buffer size in bytes for sending a file-like message body.

Adicionado na versão 3.7.

HTTPConnection.max_response_headers

The maximum number of allowed response headers to help prevent denial-of-service attacks. By default, the maximum number of allowed headers is set to 100. The same limit applies to the trailer section of a chunked response.

Adicionado na versão 3.15.

As an alternative to using the **`request()`** method described above, you can also send your request step by step, by using the four functions below.

HTTPConnection.putrequest(*method*, *url*, *skip_host=False*, *skip_accept_encoding=False*)

This should be the first call after the connection to the server has been made. It sends a line to the server consisting of the *method* string, the *url* string, and the HTTP version (`HTTP/1.1`). To disable automatic sending of `Host:` or `Accept-Encoding:` headers (for example to accept additional content encodings), specify *skip_host* or *skip_accept_encoding* with non-False values.

HTTPConnection.putheader(*header*, *argument*[, ...])

Send an **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html]-style header to the server. It sends a line to the server consisting of the header, a colon and a space, and the first argument. If more arguments are given, continuation lines are sent, each consisting of a tab and an argument.

HTTPConnection.endheaders(*message_body=None*, *, *encode_chunked=False*)

Send a blank line to the server, signalling the end of the headers. The optional *message_body* argument can be used to pass a message body associated with the request.

If *encode_chunked* is `True`, the result of each iteration of *message_body* will be chunk-encoded as specified in **RFC 7230** [https://datatracker.ietf.org/doc/html/rfc7230.html], Section 3.3.1. How the data is encoded is dependent on the type of *message_body*. If *message_body* implements the buffer interface the encoding will result in a single chunk. If *message_body* is a `collections.abc.Iterable`, each iteration of *message_body* will result in a chunk. If *message_body* is a file object, each call to `.read()` will result in a chunk. The method automatically signals the end of the chunk-encoded data immediately after *message_body*.

Nota

Due to the chunked encoding specification, empty chunks yielded by an iterator body will be ignored by the chunk-encoder. This is to avoid premature termination of the read of the request by the target server due to malformed encoding.

Alterado na versão 3.6: Added chunked encoding support and the *encode_chunked* parameter.

HTTPConnection.send(*data*)

Send data to the server. This should be used directly only after the `endheaders()` method has been called and before `getresponse()` is called.

Levanta um evento de auditoria `http.client.send` com os argumentos `self`, `data`.

Objetos HTTPResponse

An `HTTPResponse` instance wraps the HTTP response from the server. It provides access to the request headers and the entity body. The response is an iterable object and can be used in a with statement.

Alterado na versão 3.5: The `io.BufferedIOBase` interface is now implemented and all of its reader operations are supported.

HTTPResponse.read([*amt*])

Reads and returns the response body, or up to the next *amt* bytes.

HTTPResponse.readinto(*b*)

Reads up to the next len(b) bytes of the response body into the buffer *b*. Returns the number of bytes read.

Adicionado na versão 3.3.

HTTPResponse.getheader(*name*, *default=None*)

Return the value of the header *name*, or *default* if there is no header matching *name*. If there is more than one header with the name *name*, return all of the values joined by ‘, ‘. If *default* is any iterable other than a single string, its elements are similarly returned joined by commas.

HTTPResponse.getheaders()

Return a list of (header, value) tuples.

HTTPResponse.fileno()

Return the `fileno` of the underlying socket.

HTTPResponse.msg

A `http.client.HTTPMessage` instance containing the response headers. **`http.client.HTTPMessage`** is a subclass of `email.message.Message`.

HTTPResponse.version

HTTP protocol version used by server. 10 for HTTP/1.0, 11 for HTTP/1.1.

HTTPResponse.url

URL of the resource retrieved, commonly used to determine if a redirect was followed.

HTTPResponse.headers

Headers of the response in the form of an `email.message.EmailMessage` instance.

HTTPResponse.status

Status code returned by server.

HTTPResponse.reason

Reason phrase returned by server.

HTTPResponse.debuglevel

A debugging hook. If `debuglevel` is greater than zero, messages will be printed to stdout as the response is read and parsed.

HTTPResponse.closed

Is `True` if the stream is closed.

HTTPResponse.geturl()

Descontinuado desde a versão 3.9: Deprecated in favor of `url`.

HTTPResponse.info()

Descontinuado desde a versão 3.9: Deprecated in favor of **headers**.

HTTPResponse.getcode()

Descontinuado desde a versão 3.9: Deprecated in favor of **status**.

Exemplos

Here is an example session that uses the `GET` method:

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("www.python.org")
>>> conn.request("GET", "/")
>>> r1 = conn.getresponse()
>>> print(r1.status, r1.reason)
200 OK
>>> data1 = r1.read()  # This will return entire content.
>>> # The following example demonstrates reading data in chunks.
>>> conn.request("GET", "/")
>>> r1 = conn.getresponse()
>>> while chunk := r1.read(200):
...     print(repr(chunk))
b'<!doctype html>\n<!--[if"...
...
>>> # Example of an invalid request
>>> conn = http.client.HTTPSConnection("docs.python.org")
>>> conn.request("GET", "/parrot.spam")
>>> r2 = conn.getresponse()
>>> print(r2.status, r2.reason)
404 Not Found
>>> data2 = r2.read()
>>> conn.close()
```

Here is an example session that uses the `HEAD` method. Note that the `HEAD` method never returns any data.

```
>>> import http.client
>>> conn = http.client.HTTPSConnection("www.python.org")
>>> conn.request("HEAD", "/")
```

```
>>> res = conn.getresponse()
>>> print(res.status, res.reason)
200 OK
>>> data = res.read()
>>> print(len(data))
0
>>> data == b''
True
```

Here is an example session that uses the `POST` method:

```
>>> import http.client, urllib.parse
>>> params = urllib.parse.urlencode({'@number': 12524, '@type':
>>> headers = {"Content-type": "application/x-www-form-urlencode
...            "Accept": "text/plain"}
>>> conn = http.client.HTTPConnection("bugs.python.org")
>>> conn.request("POST", "", params, headers)
>>> response = conn.getresponse()
>>> print(response.status, response.reason)
302 Found
>>> data = response.read()
>>> data
b'Redirecting to <a href="https://bugs.python.org/issue12524">ht
>>> conn.close()
```

Client side HTTP `PUT` requests are very similar to `POST` requests. The difference lies only on the server side where HTTP servers will allow resources to be created via `PUT` requests. It should be noted that custom HTTP methods are also handled in **`urllib.request.Request`** by setting the appropriate method attribute. Here is an example session that uses the `PUT` method:

```
>>> # This creates an HTTP request
>>> # with the content of BODY as the enclosed representation
>>> # for the resource http://localhost:8080/file
...
>>> import http.client
>>> BODY = "***filecontents***"
>>> conn = http.client.HTTPConnection("localhost", 8080)
>>> conn.request("PUT", "/file", BODY)
>>> response = conn.getresponse()
```

```
>>> print(response.status, response.reason)
200, OK
```

HTTPMessage Objects

class http.client.HTTPMessage(*email.message.Message*)

An **`http.client.HTTPMessage`** instance holds the headers from an HTTP response. It is implemented using the **`email.message.Message`** class.

`ftplib` — FTP protocol client

Código-fonte: Lib/ftplib.py [https://github.com/python/cpython/tree/3.15/Lib/ftplib.py]

This module defines the class `FTP` and a few related items. The `FTP` class implements the client side of the FTP protocol. You can use this to write Python programs that perform a variety of automated FTP jobs, such as mirroring other FTP servers. It is also used by the module `urllib.request` to handle URLs that use FTP. For more information on FTP (File Transfer Protocol), see internet **RFC 959** [https://datatracker.ietf.org/doc/html/rfc959.html].

The default encoding is UTF-8, following **RFC 2640** [https://datatracker.ietf.org/doc/html/rfc2640.html].

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Here's a sample session using the `ftplib` module:

```
>>> from ftplib import FTP
>>> ftp = FTP('ftp.us.debian.org')  # connect to host, default po
>>> ftp.login()                     # user anonymous, passwd anon
'230 Login successful.'
>>> ftp.cwd('debian')               # change into "debian" direct
'250 Directory successfully changed.'
>>> ftp.retrlines('LIST')           # list directory contents
-rw-rw-r--    1 1176     1176         1063 Jun 15 10:18 README
...
drwxr-sr-x    5 1176     1176         4096 Dec 19  2000 pool
drwxr-sr-x    4 1176     1176         4096 Nov 17  2008 project
drwxr-xr-x    3 1176     1176         4096 Oct 10  2012 tools
'226 Directory send OK.'
>>> with open('README', 'wb') as fp:
>>>     ftp.retrbinary('RETR README', fp.write)
'226 Transfer complete.'
```

```
>>> ftp.quit()
'221 Goodbye.'
```

Referência

FTP objects

class ftplib.FTP(*host=''*, *user=''*, *passwd=''*, *acct=''*, *timeout=None*, *source_address=None*, ***, *encoding='utf-8'*)

Return a new instance of the **FTP** class.

Parâmetros::

- **host** (*str*) – The hostname to connect to. If given, `connect(host)` is implicitly called by the constructor.
- **user** (*str*) – The username to log in with (default: `'anonymous'`). If given, `login(host, passwd, acct)` is implicitly called by the constructor.
- **passwd** (*str*) – The password to use when logging in. If not given, and if *passwd* is the empty string or `"-"`, a password will be automatically generated
- **acct** (*str*) – Account information to be used for the `ACCT` FTP command. Few systems implement this. See RFC-959 [https://datatracker.ietf.org/doc/html/rfc959.html] for more details.
- **timeout** (*float* | *None*) – A timeout in seconds for blocking operations like **`connect()`** (default: the global default timeout setting).
- **source_address** (*tuple* | *None*) – A 2-tuple `(host, port)` for the socket to bind to as its source address before connecting.
- **encoding** (*str*) – The encoding for directories and filenames (default: `'utf-8'`).

The **FTP** class supports the **`with`** statement, e.g.:

```
>>> from ftplib import FTP
>>> with FTP("ftp1.at.proftpd.org") as ftp:
```

```
...     ftp.login()
...     ftp.dir()
...
'230 Anonymous login ok, restrictions apply.'
dr-xr-xr-x   9 ftp      ftp           154 May  6 10:43 .
dr-xr-xr-x   9 ftp      ftp           154 May  6 10:43 ..
dr-xr-xr-x   5 ftp      ftp          4096 May  6 10:43 CentOS
dr-xr-xr-x   3 ftp      ftp            18 Jul 10  2008 Fedora
>>>
```

Alterado na versão 3.2: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.3: source_address parameter was added.

Alterado na versão 3.9: If the *timeout* parameter is set to be zero, it will raise a `ValueError` to prevent the creation of a non-blocking socket. The *encoding* parameter was added, and the default was changed from Latin-1 to UTF-8 to follow **RFC 2640** [https://datatracker.ietf.org/doc/html/rfc2640.html].

Several `FTP` methods are available in two flavors: one for handling text files and another for binary files. The methods are named for the command which is used followed by `lines` for the text version or `binary` for the binary version.

`FTP` instances have the following methods:

set_debuglevel(*level*)

Set the instance's debugging level as an `int`. This controls the amount of debugging output printed. The debug levels are:

- `0` (default): No debug output.
- `1`: Produce a moderate amount of debug output, generally a single line per request.
- `2` or higher: Produce the maximum amount of debugging output, logging each line sent and received on the control connection.

connect(*host=''*, *port=0*, *timeout=None*, *source_address=None*)

Connect to the given host and port. This function should be called only once for each instance; it should not be called if a *host* argument

was given when the **FTP** instance was created. All other **FTP** methods can only be called after a connection has successfully been made.

Parâmetros::

- **host** (*str*) – The host to connect to.
- **port** (*int*) – The TCP port to connect to (default: `21`, as specified by the FTP protocol specification). It is rarely needed to specify a different port number.
- **timeout** (*float* | *None*) – A timeout in seconds for the connection attempt (default: the global default timeout setting).
- **source_address** (*tuple* | *None*) – A 2-tuple `(host, port)` for the socket to bind to as its source address before connecting.

Levanta um evento de auditoria `ftplib.connect` com os argumentos `self`, `host`, `port`.

Alterado na versão 3.3: source_address parameter was added.

getwelcome()

Return the welcome message sent by the server in reply to the initial connection. (This message sometimes contains disclaimers or help information that may be relevant to the user.)

login(*user='anonymous'*, *passwd=''*, *acct=''*)

Log on to the connected FTP server. This function should be called only once for each instance, after a connection has been established; it should not be called if the *host* and *user* arguments were given when the **FTP** instance was created. Most FTP commands are only allowed after the client has logged in.

Parâmetros::

- **user** (*str*) – The username to log in with (default: `'anonymous'`).
- **passwd** (*str*) – The password to use when logging in. If not given, and if *passwd* is the empty string or `"-"`, a password will be automatically generated.
- **acct** (*str*) – Account information to be used for the `ACCT` FTP command. Few systems implement this. See RFC-959 [https://datatracker.ietf.org/doc/html/rfc959.html] for more details.

abort()

Abort a file transfer that is in progress. Using this does not always work, but it's worth a try.

sendcmd(*cmd*)

Send a simple command string to the server and return the response string.

Levanta um evento de auditoria `ftplib.sendcmd` com os argumentos `self`, `cmd`.

voidcmd(*cmd*)

Send a simple command string to the server and handle the response. Return the response string if the response code corresponds to success (codes in the range 200–299). Raise **`error_reply`** otherwise.

Levanta um evento de auditoria `ftplib.sendcmd` com os argumentos `self`, `cmd`.

retrbinary(*cmd*, *callback*, *blocksize=8192*, *rest=None*)

Retrieve a file in binary transfer mode.

Parâmetros::

- **cmd** (*str*) – An appropriate `RETR` command: `"RETR filename"`.
- **callback** (callable) – A single parameter callable that is called for each block of data received, with its single argument being the data as `bytes`.
- **blocksize** (*int*) – The maximum chunk size to read on the low-level `socket` object created to do the actual transfer. This also corresponds to the largest size of data that will be passed to *callback*. Defaults to `8192`.
- **rest** (*int*) – A `REST` command to be sent to the server. See the documentation for the *rest* parameter of the `transfercmd()` method.

retrlines(*cmd*, *callback=None*)

Retrieve a file or directory listing in the encoding specified by the *encoding* parameter at initialization. *cmd* should be an appropriate `RETR` command (see `retrbinary()`) or a command such as `LIST` or `NLST` (usually just the string `'LIST'`). `LIST` retrieves a list of files and information about those files. `NLST` retrieves a list of file names. The *callback* function is called for each line with a string argument containing the line with the trailing CRLF stripped. The default *callback* prints the line to `sys.stdout`.

set_pasv(*val*)

Enable “passive” mode if *val* is true, otherwise disable passive mode. Passive mode is on by default.

storbinary(*cmd*, *fp*, *blocksize=8192*, *callback=None*, *rest=None*)

Store a file in binary transfer mode.

Parâmetros::

- **cmd** (*str*) – An appropriate `STOR` command: `"STOR filename"`.
- **fp** (file object) – A file object (opened in binar mode) which is read until EOF, using its **`read()`** method in blocks of size *blocksize* to provide the data to be stored.
- **blocksize** (*int*) – The read block size. Defaults to `8192`.
- **callback** (callable) – A single parameter callable that is called for each block of data sent, with its single argument being the data as **`bytes`**.
- **rest** (*int*) – A `REST` command to be sent to the server. See the documentation for the *rest* parameter of the **`transfercmd()`** method.

Alterado na versão 3.2: The *rest* parameter was added.

storlines(*cmd*, *fp*, *callback=None*)

Store a file in line mode. *cmd* should be an appropriate `STOR` command (see **`storbinary()`**). Lines are read until EOF from the file object *fp* (opened in binary mode) using its **`readline()`** method to provide the data to be stored. *callback* is an optional single parameter callable that is called on each line after it is sent.

transfercmd(*cmd*, *rest=None*)

Initiate a transfer over the data connection. If the transfer is active, send an `EPRT` or `PORT` command and the transfer command specified by *cmd*, and accept the connection. If the server is passive, send an `EPSV` or `PASV` command, connect to it, and start the transfer command. Either way, return the socket for the connection.

If optional *rest* is given, a `REST` command is sent to the server, passing *rest* as an argument. *rest* is usually a byte offset into the requested file, telling the server to restart sending the file's bytes at the requested offset, skipping over the initial bytes. Note however that the **`transfercmd()`** method converts *rest* to a string with the *encoding* parameter specified at initialization, but no check is performed on the

string's contents. If the server does not recognize the `REST` command, an **`error_reply`** exception will be raised. If this happens, simply call **`transfercmd()`** without a *rest* argument.

ntransfercmd(*cmd*, *rest=None*)

Like **`transfercmd()`**, but returns a tuple of the data connection and the expected size of the data. If the expected size could not be computed, `None` will be returned as the expected size. *cmd* and *rest* means the same thing as in **`transfercmd()`**.

mlsd(*path=''*, *facts=[]*)

List a directory in a standardized format by using `MLSD` command (**RFC 3659** [https://datatracker.ietf.org/doc/html/rfc3659.html]). If *path* is omitted the current directory is assumed. *facts* is a list of strings representing the type of information desired (e.g. `["type", "size", "perm"]`). Return a generator object yielding a tuple of two elements for every file found in path. First element is the file name, the second one is a dictionary containing facts about the file name. Content of this dictionary might be limited by the *facts* argument but server is not guaranteed to return all requested facts.

Adicionado na versão 3.3.

nlst(*argument*[, ...])

Return a list of file names as returned by the `NLST` command. The optional *argument* is a directory to list (default is the current server directory). Multiple arguments can be used to pass non-standard options to the `NLST` command.

Nota

If your server supports the command, **`mlsd()`** offers a better API.

dir(*argument*[, ...])

Produce a directory listing as returned by the `LIST` command, printing it to standard output. The optional *argument* is a directory to list

(default is the current server directory). Multiple arguments can be used to pass non-standard options to the `LIST` command. If the last argument is a function, it is used as a *callback* function as for `retrlines()`; the default prints to `sys.stdout`. This method returns `None`.

Nota

If your server supports the command, `mlsd()` offers a better API.

rename(*fromname*, *toname*)

Rename file *fromname* on the server to *toname*.

delete(*filename*)

Remove the file named *filename* from the server. If successful, returns the text of the response, otherwise raises `error_perm` on permission errors or `error_reply` on other errors.

cwd(*pathname*)

Set the current directory on the server.

mkd(*pathname*)

Create a new directory on the server.

pwd()

Return the pathname of the current directory on the server.

rmd(*dirname*)

Remove the directory named *dirname* on the server.

size(*filename*)

Request the size of the file named *filename* on the server. On success, the size of the file is returned as an integer, otherwise `None` is returned. Note that the `SIZE` command is not standardized, but is supported by many common server implementations.

quit()

Send a `QUIT` command to the server and close the connection. This is the “polite” way to close a connection, but it may raise an exception if the server responds with an error to the `QUIT` command. This implies a call to the **`close()`** method which renders the **`FTP`** instance useless for subsequent calls (see below).

close()

Close the connection unilaterally. This should not be applied to an already closed connection such as after a successful call to **`quit()`**. After this call the **`FTP`** instance should not be used any more (after a call to **`close()`** or **`quit()`** you cannot reopen the connection by issuing another **`login()`** method).

FTP_TLS objects

class ftplib.FTP_TLS(*host=''*, *user=''*, *passwd=''*, *acct=''*, ***, *context=None*, *timeout=None*, *source_address=None*, *encoding='utf-8'*)

An **`FTP`** subclass which adds TLS support to FTP as described in **RFC 4217** [https://datatracker.ietf.org/doc/html/rfc4217.html]. Connect to port 21 implicitly securing the FTP control connection before authenticating.

Nota

The user must explicitly secure the data connection by calling the **`prot_p()`** method.

Parâmetros::

- **host** (*str*) – The hostname to connect to. If given, `connect(host)` is implicitly called by the constructor.
- **user** (*str*) – The username to log in with (default: `'anonymous'`). If given, `login(host, passwd, acct)` is implicitly called by the constructor.
- **passwd** (*str*) – The password to use when logging in. If not given, and if *passwd* is the empty string or `"-"`, a password will be automatically generated
- **acct** (*str*) – Account information to be used for the `ACCT` FTP command. Few systems implement this. See RFC-959 [https://datatracker.ietf.org/doc/html/rfc959.html] for more details.
- **context** (`ssl.SSLContext`) – An SSL context object which allows bundling SSL configuration options, certificates and private keys into a single, potentially long-lived, structure. Please read Considerações de segurança for best practices.
- **timeout** (*float* | *None*) – A timeout in seconds for blocking operations like `connect()` (default: the global default timeout setting).
- **source_address** (*tuple* | *None*) – A 2-tuple `(host, port)` for the socket to bind to as its source address before connecting.
- **encoding** (*str*) – The encoding for directories and filenames (default: `'utf-8'`).

Adicionado na versão 3.2.

Alterado na versão 3.3: Added the *source_address* parameter.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com `ssl.SSLContext.check_hostname` e *Indicação de nome de servidor* (veja `ssl.HAS_SNI`).

Alterado na versão 3.9: If the *timeout* parameter is set to be zero, it will raise a `ValueError` to prevent the creation of a non-blocking socket. The *encoding* parameter was added, and the default was changed from Latin-1 to UTF-8 to follow **RFC 2640** [https://datatracker.ietf.org/doc/html/rfc2640.html].

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

Here's a sample session using the **FTP_TLS** class:

```
>>> ftps = FTP_TLS('ftp.pureftpd.org')
>>> ftps.login()
'230 Anonymous user logged in'
>>> ftps.prot_p()
'200 Data protection level set to "private"'
>>> ftps.nlst()
['6jack', 'OpenBSD', 'antilink', 'blogbench', 'bsdcam', 'clock
```

FTP_TLS class inherits from **FTP**, defining these additional methods and attributes:

auth()

Set up a secure control connection by using TLS.

Alterado na versão 3.4: The method now supports hostname check with **ssl.SSLContext.check_hostname** and *Server Name Indication* (see **ssl.HAS_SNI**).

ccc()

Revert control channel back to plaintext. This can be useful to take advantage of firewalls that know how to handle NAT with non-secure FTP without opening fixed ports.

Adicionado na versão 3.3.

prot_p()

Set up secure data connection by using TLS.

prot_c()

Set up clear text data connection.

Module variables

exception ftplib.error_reply

Exception raised when an unexpected reply is received from the server.

exception ftplib.error_temp

Exception raised when an error code signifying a temporary error (response codes in the range 400–499) is received.

exception ftplib.error_perm

Exception raised when an error code signifying a permanent error (response codes in the range 500–599) is received.

exception ftplib.error_proto

Exception raised when a reply is received from the server that does not fit the response specifications of the File Transfer Protocol, i.e. begin with a digit in the range 1–5.

ftplib.all_errors

The set of all exceptions (as a tuple) that methods of `FTP` instances may raise as a result of problems with the FTP connection (as opposed to programming errors made by the caller). This set includes the four exceptions listed above as well as `OSError` and `EOFError`.

Ver também

Módulo `netrc`

Parser for the `.netrc` file format. The file `.netrc` is typically used by FTP clients to load user authentication information before prompting the user.

`poplib` — Cliente de protocolo POP3

Source code: Lib/poplib.py [https://github.com/python/cpython/tree/3.15/Lib/poplib.py]

Este módulo define uma classe, **`POP3`**, que encapsula uma conexão com um servidor POP3 e implementa o protocolo conforme definido em **RFC 1939** [https://datatracker.ietf.org/doc/html/rfc1939.html]. A classe **`POP3`** oferece suporte aos conjuntos de comandos mínimos e opcionais de **RFC 1939** [https://datatracker.ietf.org/doc/html/rfc1939.html]. A classe **`POP3`** também oferece suporte ao comando `STLS` introduzido em **RFC 2595** [https://datatracker.ietf.org/doc/html/rfc2595.html] para permitir a comunicação criptografada em uma conexão já estabelecida.

Além disso, este módulo fornece uma classe **`POP3_SSL`**, que fornece suporte para conexão a servidores POP3 que usam SSL como uma camada de protocolo subjacente.

Observe que o POP3, embora amplamente suportado, está obsoleto. A qualidade de implementação dos servidores POP3 varia muito, e muitos são bastante ruins. Se o seu servidor de e-mail oferece suporte a IMAP, seria melhor usar a classe **`imaplib.IMAP4`**, pois os servidores IMAP tendem a ter uma implementação melhor.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

The **`poplib`** module provides two classes:

class poplib.POP3(*host*, *port=POP3_PORT*[, *timeout*])

Esta classe implementa o protocolo POP3 real. A conexão é criada quando a instância é inicializada. Se *port* for omitido, a porta POP3 padrão (110) será usada. O parâmetro opcional *timeout* especifica um tempo limite em segundos para a tentativa de conexão (se não for especificado, a configuração de tempo limite padrão global será usada).

Levanta um evento de auditoria `ftplib.connect` com os argumentos `self`, `host`, `port`.

Todos os comandos levantarão um evento de auditoria `poplib.putline` com os argumentos `self` e `line`, onde `line` são os bytes prestes a serem enviados ao host remoto.

Alterado na versão 3.9: Se o parâmetro *timeout* for definido como zero, será levantada **`ValueError`** para impedir a criação de um soquete não bloqueante.

class poplib.POP3_SSL(*host*, *port=POP3_SSL_PORT*, *, *timeout=None*, *context=None*)

Esta é uma subclasse de **`POP3`** que se conecta ao servidor por meio de um soquete criptografado SSL. Se *port* não for especificado, 995, a porta padrão POP3 sobre SSL será usada. *timeout* funciona como no construtor **`POP3`**. *context* é um objeto opcional **`ssl.SSLContext`** que permite agrupar opções de configuração SSL, certificados e chaves privadas em uma única estrutura (potencialmente duradoura). Consulte Considerações de segurança para obter as melhores práticas.

Levanta um evento de auditoria `ftplib.connect` com os argumentos `self`, `host`, `port`.

Todos os comandos levantarão um evento de auditoria `poplib.putline` com os argumentos `self` e `line`, onde `line` são os bytes prestes a serem enviados ao host remoto.

Alterado na versão 3.2: Parâmetro *context* adicionado.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com **`ssl.SSLContext.check_hostname`** e *Indicação de*

nome de servidor (veja `ssl.HAS_SNI`).

Alterado na versão 3.9: Se o parâmetro *timeout* for definido como zero, será levantada `ValueError` para impedir a criação de um soquete não bloqueante.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

One exception is defined as an attribute of the **`poplib`** module:

exception poplib.error_proto

Exceção levantada em caso de erros deste módulo (erros do módulo `socket` não são detectados). O motivo da exceção é passado ao construtor como uma string.

Ver também

Módulo `imaplib`

The standard Python IMAP module.

Frequently Asked Questions About Fetchmail
[http://www.catb.org/~esr/fetchmail/fetchmail-FAQ.html]

The FAQ for the **fetchmail** POP/IMAP client collects information on POP3 server variations and RFC noncompliance that may be useful if you need to write an application based on the POP protocol.

Objetos POP3

All POP3 commands are represented by methods of the same name, in lowercase; most return the response text sent by the server.

A `POP3` instance has the following methods:

POP3.set_debuglevel(*level*)

Set the instance's debugging level. This controls the amount of debugging output printed. The default, `0`, produces no debugging output. A value of `1` produces a moderate amount of debugging output, generally a single line per request. A value of `2` or higher produces the maximum amount of debugging output, logging each line sent and received on the control connection.

POP3.getwelcome()

Returns the greeting string sent by the POP3 server.

POP3.capa()

Query the server's capabilities as specified in **RFC 2449** [https://datatracker.ietf.org/doc/html/rfc2449.html]. Returns a dictionary in the form `{'name': ['param'...]}`.

Adicionado na versão 3.4.

POP3.user(*username*)

Send user command, response should indicate that a password is required.

POP3.pass_(*password*)

Send password, response includes message count and mailbox size. Note: the mailbox on the server is locked until `quit()` is called.

POP3.apop(*user*, *secret*)

Use the more secure APOP authentication to log into the POP3 server.

POP3.rpop(*user*)

Use RPOP authentication (similar to UNIX r-commands) to log into POP3 server.

POP3.stat()

Get mailbox status. The result is a tuple of 2 integers: `(message count, mailbox size)`.

POP3.list([*which*])

Request message list, result is in the form `(response, ['mesg_num octets', ...], octets)`. If *which* is set, it is the message to list.

POP3.retr(*which*)

Retrieve whole message number *which*, and set its seen flag. Result is in form `(response, ['line', ...], octets)`.

POP3.dele(*which*)

Flag message number *which* for deletion. On most servers deletions are not actually performed until QUIT (the major exception is Eudora QPOP, which deliberately violates the RFCs by doing pending deletes on any disconnect).

POP3.rset()

Remove any deletion marks for the mailbox.

POP3.noop()

Do nothing. Might be used as a keep-alive.

POP3.quit()

Signoff: commit changes, unlock mailbox, drop connection.

POP3.top(*which*, *howmuch*)

Retrieves the message header plus *howmuch* lines of the message after the header of message number *which*. Result is in form `(response, ['line', ...], octets)`.

The POP3 TOP command this method uses, unlike the RETR command, doesn't set the message's seen flag; unfortunately, TOP is poorly specified in the RFCs and is frequently broken in off-brand servers. Test this method by hand against the POP3 servers you will use before trusting it.

POP3.uidl(*which=None*)

Return message digest (unique id) list. If *which* is specified, result contains the unique id for that message in the form `'response mesgnum uid`, otherwise result is list `(response, ['mesgnum uid', ...], octets)`.

POP3.utf8()

Try to switch to UTF-8 mode. Returns the server response if successful, raises **`error_proto`** if not. Specified in **RFC 6856** [https://datatracker.ietf.org/doc/html/rfc6856.html].

Adicionado na versão 3.5.

POP3.stls(*context=None*)

Start a TLS session on the active connection as specified in **RFC 2595** [https://datatracker.ietf.org/doc/html/rfc2595.html]. This is only allowed before user authentication

context parameter is a **`ssl.SSLContext`** object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read Considerações de segurança for best practices.

This method supports hostname checking via **`ssl.SSLContext.check_hostname`** and *Server Name Indication* (see **`ssl.HAS_SNI`**).

Adicionado na versão 3.4.

Instances of **`POP3_SSL`** have no additional methods. The interface of this subclass is identical to its parent.

Exemplo POP3

Here is a minimal example (without error checking) that opens a mailbox and retrieves and prints all messages:

```
import getpass, poplib

M = poplib.POP3('localhost')
M.user(getpass.getuser())
M.pass_(getpass.getpass())
numMessages = len(M.list()[1])
for i in range(numMessages):
    for j in M.retr(i+1)[1]:
        print(j)
```

At the end of the module, there is a test section that contains a more extensive example of usage.

`imaplib` — IMAP4 protocol client

Código-fonte: Lib/imaplib.py
[https://github.com/python/cpython/tree/3.15/Lib/imaplib.py]

This module defines three classes, `IMAP4`, `IMAP4_SSL` and `IMAP4_stream`, which encapsulate a connection to an IMAP4 server and implement a large subset of the IMAP4rev1 client protocol as defined in **RFC 3501** [https://datatracker.ietf.org/doc/html/rfc3501.html]. It is backward compatible with IMAP4 (**RFC 1730** [https://datatracker.ietf.org/doc/html/rfc1730.html]) servers, but note that the `STATUS` command is not supported in IMAP4.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Three classes are provided by the **`imaplib`** module, `IMAP4` is the base class:

class imaplib.IMAP4(*host=''*, *port=IMAP4_PORT*, *timeout=None*)

This class implements the actual IMAP4 protocol. The connection is created and protocol version (IMAP4 or IMAP4rev1) is determined when the instance is initialized. If *host* is not specified, `''` (the local host) is used. If *port* is omitted, the standard IMAP4 port (143) is used. The optional *timeout* parameter specifies a timeout in seconds for the connection attempt. If timeout is not given or is `None`, the global default socket timeout is used.

The **`IMAP4`** class supports the `with` statement. When used like this, the IMAP4 `LOGOUT` command is issued automatically when the **`with`** statement exits. E.g.:

```
>>> from imaplib import IMAP4
>>> with IMAP4("domain.org") as M:
...     M.noop()
```

```
...
('OK', [b'Nothing Accomplished. d25if65hy903weo.87'])
```

Alterado na versão 3.5: Suporte para a instrução `with` foi adicionado.

Alterado na versão 3.9: O parâmetro opcional *timeout* foi adicionado.

Three exceptions are defined as attributes of the `IMAP4` class:

exception IMAP4.error

Exception raised on any errors. The reason for the exception is passed to the constructor as a string.

exception IMAP4.abort

IMAP4 server errors cause this exception to be raised. This is a sub-class of `IMAP4.error`. Note that closing the instance and instantiating a new one will usually allow recovery from this exception.

exception IMAP4.readonly

This exception is raised when a writable mailbox has its status changed by the server. This is a sub-class of `IMAP4.error`. Some other client now has write permission, and the mailbox will need to be re-opened to re-obtain write permission.

There's also a subclass for secure connections:

class imaplib.IMAP4_SSL(*host=''*, *port=IMAP4_SSL_PORT*, ***, *ssl_context=None*, *timeout=None*)

This is a subclass derived from `IMAP4` that connects over an SSL encrypted socket (to use this class you need a socket module that was compiled with SSL support). If *host* is not specified, `''` (the local host) is used. If *port* is omitted, the standard IMAP4-over-SSL port (993) is used. *ssl_context* is a `ssl.SSLContext` object which allows bundling SSL configuration options, certificates and private keys into a single (potentially long-lived) structure. Please read Considerações de segurança for best practices.

Nota

With the default *ssl_context*, the connection is encrypted but the server certificate and hostname are not verified. To verify them, pass a context created by `ssl.create_default_context()`.

The optional *timeout* parameter specifies a timeout in seconds for the connection attempt. If timeout is not given or is `None`, the global default socket timeout is used.

Alterado na versão 3.3: ssl_context parameter was added.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com `ssl.SSLContext.check_hostname` e *Indicação de nome de servidor* (veja `ssl.HAS_SNI`).

Alterado na versão 3.9: O parâmetro opcional *timeout* foi adicionado.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

The second subclass allows for connections created by a child process:

class imaplib.IMAP4_stream(*command*)

This is a subclass derived from `IMAP4` that connects to the `stdin/stdout` file descriptors created by passing *command* to `subprocess.Popen()`.

The following utility functions are defined:

imaplib.Internaldate2tuple(*resp*)

Parse a bytes-like object containing an IMAP4 `INTERNALDATE` response and return the corresponding local time. The return value is a `time.struct_time` tuple or `None` if the input has wrong format.

imaplib.Int2AP(*num*)

Converts an integer into a bytes representation using characters from the set [`A` .. `P`].

imaplib.ParseFlags(*resp*)

Converts a bytes-like object containing an IMAP4 `FLAGS` response to a tuple of individual flags as `bytes`. The return value is an empty tuple if the input has wrong format.

imaplib.Time2Internaldate(*date_time*)

Convert *date_time* to an IMAP4 `INTERNALDATE` representation. The return value is a string in the form: `"DD-Mmm-YYYY HH:MM:SS +HHMM"` (including double-quotes). The *date_time* argument can be a number (int or float) representing seconds since epoch (as returned by `time.time()`), a 9-tuple representing local time an instance of `time.struct_time` (as returned by `time.localtime()`), an aware instance of `datetime.datetime`, or a double-quoted string. In the last case, it is assumed to already be in the correct format.

Note that IMAP4 message numbers change as the mailbox changes; in particular, after an `EXPUNGE` command performs deletions the remaining messages are renumbered. So it is highly advisable to use UIDs instead, with the UID command.

At the end of the module, there is a test section that contains a more extensive example of usage.

Ver também

Documents describing the protocol, sources for servers implementing it, by the University of Washington's IMAP Information Center can all be found at (**Source Code**) https://github.com/uw-imap/imap (**Not Maintained**).

Objetos IMAP4

All IMAP4rev1 commands are represented by methods of the same name, either uppercase or lowercase.

All arguments to commands are converted to strings, except for `AUTHENTICATE`, and the last argument to `APPEND` which is passed as an IMAP4 literal. If necessary (the string contains IMAP4 protocol-sensitive characters and isn't enclosed with either parentheses or double quotes) each string is quoted. However, the *password* argument to the `LOGIN` command is always quoted. If you want to avoid having an argument string quoted (eg: the *flags* argument to `STORE`) then enclose the string in parentheses (eg: `r'(\Deleted)'`). In general, pass arguments unquoted and let the module quote them as needed. An argument that is already enclosed in double quotes is left unchanged, so that code which quotes arguments itself keeps working.

Most commands return a tuple: `(type, [data, ...])` where *type* is usually `'OK'` or `'NO'`, and *data* is either the text from the command response, or mandated results from the command. Each *data* is either a `bytes`, or a tuple. If a tuple, then the first part is the header of the response, and the second part contains the data (ie: 'literal' value).

The *message_set* options to commands below is a string specifying one or more messages to be acted upon. It may be a simple message number (`'1'`), a range of message numbers (`'2:4'`), or a group of non-contiguous ranges separated by commas (`'1:3,6:9'`). A range can contain an asterisk to indicate an infinite upper bound (`'3:*'`).

An **`IMAP4`** instance has the following methods:

IMAP4.append(*mailbox*, *flags*, *date_time*, *message*)

Append *message* to named mailbox.

flags may be `None` or a string of IMAP flag tokens. Multiple flags are separated by spaces, for example `r'\Seen \Answered'`. If *flags* is not already enclosed in parentheses, parentheses are added automatically.

IMAP4.authenticate(*mechanism*, *authobject*)

Authenticate command — requires response processing.

mechanism specifies which authentication mechanism is to be used - it should appear in the instance variable `capabilities` in the form `AUTH=mechanism`.

authobject must be a callable object:

```
data = authobject(response)
```

It will be called to process server continuation responses; the *response* argument it is passed will be `bytes`. It should return `bytes` *data* that will be base64 encoded and sent to the server. It should return `None` if the client abort response `*` should be sent instead.

Alterado na versão 3.5: string usernames and passwords are now encoded to `utf-8` instead of being limited to ASCII.

IMAP4.check()

Checkpoint mailbox on server.

IMAP4.close()

Close currently selected mailbox. Deleted messages are removed from writable mailbox. This is the recommended command before `LOGOUT`.

IMAP4.copy(*message_set*, *new_mailbox*)

Copy *message_set* messages onto end of *new_mailbox*.

IMAP4.create(*mailbox*)

Create new mailbox named *mailbox*.

IMAP4.delete(*mailbox*)

Delete old mailbox named *mailbox*.

IMAP4.deleteacl(*mailbox*, *who*)

Delete the ACLs (remove any rights) set for who on mailbox.

IMAP4.enable(*capability*)

Enable *capability* (see **RFC 5161** [https://datatracker.ietf.org/doc/html/rfc5161.html]). Most capabilities do not need to be enabled. Currently only the `UTF8=ACCEPT` capability is supported (see **RFC 6855** [https://datatracker.ietf.org/doc/html/rfc6855.html]).

Adicionado na versão 3.5: The **`enable()`** method itself, and **RFC 6855** [https://datatracker.ietf.org/doc/html/rfc6855.html] support.

IMAP4.expunge()

Permanently remove deleted items from selected mailbox. Generates an `EXPUNGE` response for each deleted message. Returned data contains a list of `EXPUNGE` message numbers in order received.

IMAP4.fetch(*message_set*, *message_parts*)

Fetch (parts of) messages. *message_parts* should be a string of message part names enclosed within parentheses, eg: `"(UID BODY[TEXT])"`. Returned data are tuples of message part envelope and data.

IMAP4.getacl(*mailbox*)

Get the `ACL`s for *mailbox*. The method is non-standard, but is supported by the `Cyrus` server.

IMAP4.getannotation(*mailbox*, *entry*, *attribute*)

Retrieve the specified `ANNOTATION`s for *mailbox*. The method is non-standard, but is supported by the `Cyrus` server.

IMAP4.getquota(*root*)

Get the `quota` *root*'s resource usage and limits. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.getquotaroot(*mailbox*)

Get the list of `quota roots` for the named *mailbox*. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.idle(*duration=None*)

Return an **`Idler`**: an iterable context manager implementing the IMAP4 `IDLE` command as defined in **RFC 2177** [https://datatracker.ietf.org/doc/html/rfc2177.html].

The returned object sends the `IDLE` command when activated by the **`with`** statement, produces IMAP untagged responses via the iterator protocol, and sends `DONE` upon context exit.

All untagged responses that arrive after sending the `IDLE` command (including any that arrive before the server acknowledges the command) will be available via iteration. Any leftover responses (those not iterated in the **`with`** context) can be retrieved in the usual way after `IDLE` ends, using **`IMAP4.response()`**.

Responses are represented as `(type, [data, ...])` tuples, as described in IMAP4 Objects.

The *duration* argument sets a maximum duration (in seconds) to keep idling, after which any ongoing iteration will stop. It can be an **`int`** or **`float`**, or `None` for no time limit. Callers wishing to avoid inactivity timeouts on servers that impose them should keep this at most 29 minutes (1740 seconds). Requires a socket connection; *duration* must be `None` on **`IMAP4_stream`** connections.

```
>>> with M.idle(duration=29 * 60) as idler:
...     for typ, data in idler:
...         print(typ, data)
...
EXISTS [b'1']
RECENT [b'1']
```

Idler.burst(*interval=0.1*)

Yield a burst of responses no more than *interval* seconds apart (expressed as an **`int`** or **`float`**).

This generator is an alternative to iterating one response at a time, intended to aid in efficient batch processing. It retrieves the next response along with any immediately available subsequent

responses. (For example, a rapid series of `EXPUNGE` responses after a bulk delete.)

Requires a socket connection; does not work on **`IMAP4 stream`** connections.

```
>>> with M.idle() as idler:
...     # get a response and any others following by < 0.:
...     batch = list(idler.burst())
...     print(f'processing {len(batch)} responses...')
...     print(batch)
...
processing 3 responses...
[('EXPUNGE', [b'2']), ('EXPUNGE', [b'1']), ('RECENT', [b'(
```

Dica

The `IDLE` context's maximum duration, as passed to **`IMAP4.idle()`**, is respected when waiting for the first response in a burst. Therefore, an expired **`Idler`** will cause this generator to return immediately without producing anything. Callers should consider this if using it in a loop.

Nota

The iterator returned by **`IMAP4.idle()`** is usable only within a **`with`** statement. Before or after that context, unsolicited responses are collected internally whenever a command finishes, and can be retrieved with **`IMAP4.response()`**.

Nota

The **`Idler`** class name and structure are internal interfaces, subject to change. Calling code can rely on its context management, iteration, and public method to remain stable, but should not subclass, instantiate, compare, or otherwise directly reference the class.

Adicionado na versão 3.14.

IMAP4.list(*directory=''*, *pattern='*'*)

List mailbox names in *directory* matching *pattern*. *directory* defaults to the top-level mail folder, and *pattern* defaults to match anything. Returned data contains a list of `LIST` responses.

IMAP4.login(*user*, *password*)

Identify the client using a plaintext password. The *password* will be quoted.

IMAP4.login_cram_md5(*user*, *password*)

Force use of `CRAM-MD5` authentication when identifying the client to protect the password. Will only work if the server `CAPABILITY` response includes the phrase `AUTH=CRAM-MD5`.

Alterado na versão 3.15: An `IMAP4.error` is raised if MD5 support is not available.

IMAP4.logout()

Shutdown connection to server. Returns server `BYE` response.

Alterado na versão 3.8: The method no longer ignores silently arbitrary exceptions.

IMAP4.lsub(*directory=''*, *pattern='*'*)

List subscribed mailbox names in directory matching pattern. *directory* defaults to the top level directory and *pattern* defaults to match any mailbox. Returned data are tuples of message part envelope and data.

IMAP4.myrights(*mailbox*)

Show my ACLs for a mailbox (i.e. the rights that I have on mailbox).

IMAP4.namespace()

Returns IMAP namespaces as defined in **RFC 2342** [https://datatracker.ietf.org/doc/html/rfc2342.html].

IMAP4.noop()

Envia `NOOP` para o servidor.

IMAP4.open(*host*, *port*, *timeout=None*)

Opens socket to *port* at *host*. The optional *timeout* parameter specifies a timeout in seconds for the connection attempt. If timeout is not given or is `None`, the global default socket timeout is used. Also note that if the *timeout* parameter is set to be zero, it will raise a **`ValueError`** to reject creating a non-blocking socket. This method is implicitly called by the **`IMAP4`** constructor. The connection objects established by this method will be used in the **`IMAP4.read()`**, **`IMAP4.readline()`**, **`IMAP4.send()`**, and **`IMAP4.shutdown()`** methods. You may override this method.

Levanta um evento de auditoria `imaplib.open` com os argumentos `self`, `host`, `port`.

Alterado na versão 3.9: The *timeout* parameter was added.

IMAP4.partial(*message_num*, *message_part*, *start*, *length*)

Fetch truncated part of a message. Returned data is a tuple of message part envelope and data.

IMAP4.proxyauth(*user*)

Assume authentication as *user*. Allows an authorised administrator to proxy into any user's mailbox.

IMAP4.read(*size*)

Reads *size* bytes from the remote server. You may override this method.

IMAP4.readline()

Reads one line from the remote server. You may override this method.

IMAP4.recent()

Prompt server for an update. Returned data is `None` if no new messages, else value of `RECENT` response.

IMAP4.rename(*oldmailbox*, *newmailbox*)

Rename mailbox named *oldmailbox* to *newmailbox*.

IMAP4.response(*code*)

Return data for response *code* if received, or `None`. Returns the given code, instead of the usual type.

IMAP4.search(*charset*, *criterion*[, ...])

Search mailbox for matching messages. *charset* may be `None`, in which case no `CHARSET` will be specified in the request to the server. The IMAP protocol requires that at least one criterion be specified; an exception will be raised when the server returns an error. *charset* must be `None` if the `UTF8=ACCEPT` capability was enabled using the **`enable()`** command.

Exemplo:

```
# M is a connected IMAP4 instance...
typ, msgnums = M.search(None, 'FROM', '"LDJ"')

# or:
typ, msgnums = M.search(None, '(FROM "LDJ")')
```

IMAP4.select(*mailbox='INBOX'*, *readonly=False*)

Select a mailbox. Returned data is the count of messages in *mailbox* (`EXISTS` response). The default *mailbox* is `'INBOX'`. If the *readonly* flag is set, modifications to the mailbox are not allowed.

IMAP4.send(*data*)

Sends `data` to the remote server. You may override this method.

Levanta um evento de auditoria `imaplib.send` com os argumentos `self`, `data`.

IMAP4.setacl(*mailbox*, *who*, *what*)

Set an `ACL` for *mailbox*. The method is non-standard, but is supported by the `Cyrus` server.

IMAP4.setannotation(*mailbox*, *entry*, *attribute*[, ...])

Set `ANNOTATIONS` for *mailbox*. The method is non-standard, but is supported by the `Cyrus` server.

IMAP4.setquota(*root*, *limits*)

Set the `quota` *root*'s resource *limits*. This method is part of the IMAP4 QUOTA extension defined in rfc2087.

IMAP4.shutdown()

Close connection established in `open`. This method is implicitly called by **`IMAP4.logout()`**. You may override this method.

IMAP4.socket()

Returns socket instance used to connect to server.

IMAP4.sort(*sort_criteria*, *charset*, *search_criterion*[, ...])

The `sort` command is a variant of `search` with sorting semantics for the results. Returned data contains a space separated list of matching message numbers.

Sort has two arguments before the *search_criterion* argument(s); a parenthesized list of *sort_criteria*, and the searching *charset*. Note that unlike `search`, the searching *charset* argument is mandatory. There is also a `uid sort` command which corresponds to `sort` the way that `uid search` corresponds to `search`. The `sort` command first searches the mailbox for messages that match the given searching criteria using the charset argument for the interpretation of strings in the searching criteria. It then returns the numbers of matching messages.

This is an `IMAP4rev1` extension command.

IMAP4.starttls(*ssl_context=None*)

Send a `STARTTLS` command. The *ssl_context* argument is optional and should be a **`ssl.SSLContext`** object. This will enable encryption on the IMAP connection. Please read Considerações de segurança for best practices.

Nota

With the default *ssl_context*, the connection is encrypted but the server certificate and hostname are not verified. To verify them, pass a context created by `ssl.create_default_context()`.

Adicionado na versão 3.2.

Alterado na versão 3.4: The method now supports hostname check with `ssl.SSLContext.check_hostname` and *Server Name Indication* (see `ssl.HAS_SNI`).

IMAP4.status(*mailbox*, *names*)

Request named status conditions for *mailbox*.

IMAP4.store(*message_set*, *command*, *flag_list*)

Alters flag dispositions for messages in mailbox. *command* is specified by section 6.4.6 of **RFC 3501** [https://datatracker.ietf.org/doc/html/rfc3501.html] as being one of “FLAGS”, “+FLAGS”, or “-FLAGS”, optionally with a suffix of “.SILENT”.

For example, to set the delete flag on all messages:

```
typ, data = M.search(None, 'ALL')
for num in data[0].split():
   M.store(num, '+FLAGS', '\\Deleted')
M.expunge()
```

Nota

Creating flags containing ‘]’ (for example: “[test]”) violates **RFC 3501** [https://datatracker.ietf.org/doc/html/rfc3501.html] (the IMAP protocol). However, imaplib has historically allowed creation of such flags, and popular IMAP servers, such as Gmail, accept and produce such flags. There are non-Python programs which also create such flags. Although it is an RFC violation and IMAP clients and servers are supposed to be strict, imaplib still continues to allow such flags to be created for backward compatibility reasons, and as of Python 3.6,

handles them if they are sent from the server, since this improves real-world compatibility.

IMAP4.subscribe(*mailbox*)

Subscribe to new mailbox.

IMAP4.thread(*threading_algorithm*, *charset*, *search_criterion*[, ...])

The `thread` command is a variant of `search` with threading semantics for the results. Returned data contains a space separated list of thread members.

Thread members consist of zero or more messages numbers, delimited by spaces, indicating successive parent and child.

Thread has two arguments before the *search_criterion* argument(s); a *threading_algorithm*, and the searching *charset*. Note that unlike `search`, the searching *charset* argument is mandatory. There is also a `uid thread` command which corresponds to `thread` the way that `uid search` corresponds to `search`. The `thread` command first searches the mailbox for messages that match the given searching criteria using the *charset* argument for the interpretation of strings in the searching criteria. It then returns the matching messages threaded according to the specified threading algorithm.

This is an `IMAP4rev1` extension command.

IMAP4.uid(*command*, *arg*[, ...])

Execute command args with messages identified by UID, rather than message number. Returns response appropriate to command. At least one argument must be supplied; if none are provided, the server will return an error and an exception will be raised.

IMAP4.unsubscribe(*mailbox*)

Unsubscribe from old mailbox.

IMAP4.unselect()

`imaplib.IMAP4.unselect()` frees server's resources associated with the selected mailbox and returns the server to the authenticated state. This command performs the same actions as **`imaplib.IMAP4.close()`**, except that no messages are permanently removed from the currently selected mailbox.

Adicionado na versão 3.9.

IMAP4.xatom(*name*[, ...])

Allow simple extension commands notified by server in `CAPABILITY` response.

The following attributes are defined on instances of **`IMAP4`**:

IMAP4.capabilities

A tuple of the capabilities advertised by the server, in upper case.

It is set when the connection is established, and refreshed after a successful **`login()`**, **`authenticate()`** or **`starttls()`**, because the server can advertise different capabilities in different connection states.

Alterado na versão 3.14.7: Refreshed after **`login()`** and **`authenticate()`**.

IMAP4.PROTOCOL_VERSION

The most recent supported protocol in the `CAPABILITY` response from the server.

IMAP4.debug

Integer value to control debugging output. The initialize value is taken from the module variable `Debug`. Values greater than three trace each command.

IMAP4.utf8_enabled

Boolean value that is normally `False`, but is set to `True` if an **`enable()`** command is successfully issued for the `UTF8=ACCEPT` capability.

Adicionado na versão 3.5.

property IMAP4.file

Internal `BufferedReader` associated with the underlying socket. This property is documented for legacy purposes but not part of the public interface. The caller is responsible to ensure that the current file is closed before changing it.

Descontinuado desde a versão 3.15, será removido na versão 3.19.

Exemplo IMAP4

Here is a minimal example (without error checking) that opens a mailbox and retrieves and prints all messages:

```
import getpass, imaplib

M = imaplib.IMAP4(host='example.org')
M.login(getpass.getuser(), getpass.getpass())
M.select()
typ, data = M.search(None, 'ALL')
for num in data[0].split():
    typ, data = M.fetch(num, '(RFC822)')
    print('Message %s\n%s\n' % (num, data[0][1]))
M.close()
M.logout()
```

Nota

A `FETCH` response may contain additional or unsolicited data (see **RFC 3501** [https://datatracker.ietf.org/doc/html/rfc3501.html], section 7.4.2), so production code should inspect the whole response rather than rely on `data[0][1]`.

`smtplib` — Cliente de protocolo SMTP

Código-fonte: Lib/smtplib.py
[https://github.com/python/cpython/tree/3.15/Lib/smtplib.py]

The **`smtplib`** module defines an SMTP client session object that can be used to send mail to any internet machine with an SMTP or ESMTP listener daemon. For details of SMTP and ESMTP operation, consult **RFC 821** [https://datatracker.ietf.org/doc/html/rfc821.html] (Simple Mail Transfer Protocol) and **RFC 1869** [https://datatracker.ietf.org/doc/html/rfc1869.html] (SMTP Service Extensions).

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

class smtplib.SMTP(*host=''*, *port=0*, *local_hostname=None*, [*timeout*,]*source_address=None*)

An **`SMTP`** instance encapsulates an SMTP connection. It has methods that support a full repertoire of SMTP and ESMTP operations.

If the host parameter is set to a truthy value, **`SMTP.connect()`** is called with host and port automatically when the object is created; otherwise, **`connect()`** must be called manually.

If specified, *local_hostname* is used as the FQDN of the local host in the HELO/EHLO command. Otherwise, the local hostname is found using **`socket.getfqdn()`**. If the **`connect()`** call returns anything other than a success code, an **`SMTPConnectError`** is raised. The optional *timeout* parameter specifies a timeout in seconds for blocking operations

like the connection attempt (if not specified, the global default timeout setting will be used). If the timeout expires, **`TimeoutError`** is raised. The optional *source_address* parameter allows binding to some specific source address in a machine with multiple network interfaces, and/or to some specific source TCP port. It takes a 2-tuple `(host, port)`, for the socket to bind to as its source address before connecting. If omitted (or if *host* or *port* are `''` and/or `0` respectively) the OS default behavior will be used.

For normal use, you should only require the initialization/connect, **`sendmail()`**, and **`SMTP.quit()`** methods. An example is included below.

The **`SMTP`** class supports the **`with`** statement. When used like this, the SMTP `QUIT` command is issued automatically when the **`with`** statement exits. E.g.:

```
>>> from smtplib import SMTP
>>> with SMTP("domain.org") as smtp:
...     smtp.noop()
...
(250, b'Ok')
>>>
```

All commands will raise an auditing event `smtplib.SMTP.send` with arguments `self` and `data`, where `data` is the bytes about to be sent to the remote host.

default_port

The default port used for SMTP connections (25).

Alterado na versão 3.3: Suporte para a instrução **`with`** foi adicionado.

Alterado na versão 3.3: source_address argument was added.

Adicionado na versão 3.5: A extensão SMTPUTF8 (**RFC 6531** [https://datatracker.ietf.org/doc/html/rfc6531.html]) é suportada agora.

Alterado na versão 3.9: Se o parâmetro *timeout* for definido como zero, será levantada **`ValueError`** para impedir a criação de um soquete não

bloqueante.

class smtplib.SMTP_SSL(*host='', port=0, local_hostname=None, *, [timeout,]context=None, source_address=None*)

An **SMTP_SSL** instance behaves exactly the same as instances of **SMTP**. **SMTP_SSL** should be used for situations where SSL is required from the beginning of the connection and using **SMTP.starttls()** is not appropriate.

If the host parameter is set to a truthy value, **SMTP.connect()** is called with host and port automatically when the object is created; otherwise, **SMTP.connect()** must be called manually.

The optional arguments *local_hostname*, *timeout* and *source_address* have the same meaning as they do in the **SMTP** class. *context*, also optional, can contain a **SSLContext** and allows configuring various aspects of the secure connection. Please read Considerações de segurança for best practices.

default_port

The default port used for SMTP-over-SSL connections (465).

Alterado na versão 3.3: context foi adicionado.

Alterado na versão 3.3: O argumento *source_address* foi adicionado.

Alterado na versão 3.4: A classe agora oferece suporte a verificação de nome de host com **ssl.SSLContext.check_hostname** e *Indicação de nome de servidor* (veja **ssl.HAS_SNI**).

Alterado na versão 3.9: If the *timeout* parameter is set to be zero, it will raise a **ValueError** to prevent the creation of a non-blocking socket

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

class smtplib.LMTP(*host=''*, *port=LMTP_PORT*, *local_hostname=None*, *source_address=None*[, *timeout*])

The LMTP protocol, which is very similar to ESMTP, is heavily based on the standard SMTP client. It's common to use Unix sockets for LMTP, so our `connect()` method must support that as well as a regular host:port server. The optional arguments *local_hostname* and *source_address* have the same meaning as they do in the `SMTP` class. To specify a Unix socket, you must use an absolute path for *host*, starting with a '/'.

Authentication is supported, using the regular SMTP mechanism. When using a Unix socket, LMTP generally don't support or require any authentication, but your mileage might vary.

Alterado na versão 3.9: O parâmetro opcional *timeout* foi adicionado.

A nice selection of exceptions is defined as well:

exception smtplib.SMTPException

Subclass of `OSError` that is the base exception class for all the other exceptions provided by this module.

Alterado na versão 3.4: SMTPException became subclass of `OSError`

exception smtplib.SMTPServerDisconnected

This exception is raised when the server unexpectedly disconnects, or when an attempt is made to use the `SMTP` instance before connecting it to a server.

exception smtplib.SMTPResponseException

Base class for all exceptions that include an SMTP error code. These exceptions are generated in some instances when the SMTP server returns an error code.

smtp_code

O código do erro.

smtp_error

A mensagem de erro.

exception smtplib.SMTPSenderRefused

Sender address refused. In addition to the attributes set by on all `SMTPResponseException` exceptions, this sets ‘sender’ to the string that the SMTP server refused.

exception smtplib.SMTPRecipientsRefused

All recipient addresses refused.

recipients

A dictionary of exactly the same sort as returned by `SMTP.sendmail()` containing the errors for each recipient.

exception smtplib.SMTPDataError

O servidor SMTP recusou aceitar o dados da mensagem.

exception smtplib.SMTPConnectError

Ocorreu um erro ao estabelecer uma conexão com o servidor.

exception smtplib.SMTPHeloError

O servidor recusou sua mensagem `HELO`.

exception smtplib.SMTPNotSupportedError

The command or option attempted is not supported by the server.

Adicionado na versão 3.5.

exception smtplib.SMTPAuthenticationError

SMTP authentication went wrong. Most probably the server didn’t accept the username/password combination provided.

Ver também

RFC 821 [https://datatracker.ietf.org/doc/html/rfc821.html] - Simple Mail Transfer Protocol

Protocol definition for SMTP. This document covers the model, operating procedure, and protocol details for SMTP.

RFC 1869 [https://datatracker.ietf.org/doc/html/rfc1869.html] - SMTP Service Extensions

Definition of the ESMTP extensions for SMTP. This describes a framework for extending SMTP with new commands, supporting dynamic discovery of the commands provided by the server, and defines a few additional commands.

Objetos SMTP

An `SMTP` instance has the following methods:

SMTP.set_debuglevel(*level*)

Set the debug output level. A value of 1 or `True` for *level* results in debug messages for connection and for all messages sent to and received from the server. A value of 2 for *level* results in these messages being timestamped.

Alterado na versão 3.5: Adicionado o nível de depuração 2.

SMTP.docmd(*cmd*, *args=''*)

Send a command *cmd* to the server. The optional argument *args* is simply concatenated to the command, separated by a space.

This returns a 2-tuple composed of a numeric response code and the actual response line (multiline responses are joined into one long line.)

In normal operation it should not be necessary to call this method explicitly. It is used to implement other methods and may be useful for testing private extensions.

If the connection to the server is lost while waiting for the reply, `SMTPServerDisconnected` will be raised.

SMTP.connect(*host='localhost', port=0*)

Connect to a host on a given port. The defaults are to connect to the local host at the standard SMTP port (25). If the hostname ends with a colon (`':'`) followed by a number, that suffix will be stripped off and the number interpreted as the port number to use. This method is automatically invoked by the constructor if a host is specified during instantiation. Returns a 2-tuple of the response code and message sent by the server in its connection response.

If port is not changed from its default value of 0, the value of the **`default_port`** attribute is used.

Levanta um evento de auditoria `smtplib.connect` com os argumentos `self`, `host`, `port`.

SMTP.helo(*name=''*)

Identify yourself to the SMTP server using `HELO`. The hostname argument defaults to the fully qualified domain name of the local host. The message returned by the server is stored as the **`helo_resp`** attribute of the object.

In normal operation it should not be necessary to call this method explicitly. It will be implicitly called by the **`sendmail()`** when necessary.

SMTP.ehlo(*name=''*)

Identify yourself to an ESMTP server using `EHLO`. The hostname argument defaults to the fully qualified domain name of the local host. Examine the response for ESMTP option and store them for use by **`has_extn()`**. Also sets several informational attributes: the message returned by the server is stored as the **`ehlo_resp`** attribute, **`does_esmtp`** is set to `True` or `False` depending on whether the server supports ESMTP, and **`esmtp_features`** will be a dictionary containing the names of the SMTP service extensions this server supports, and their parameters (if any).

Unless you wish to use `has_extn()` before sending mail, it should not be necessary to call this method explicitly. It will be implicitly called by `sendmail()` when necessary.

SMTP.ehlo_or_helo_if_needed()

This method calls `ehlo()` and/or `helo()` if there has been no previous `EHLO` or `HELO` command this session. It tries ESMTP `EHLO` first.

`SMTPHeloError`

The server didn't reply properly to the `HELO` greeting.

SMTP.has_extn(*name*)

Return `True` if *name* is in the set of SMTP service extensions returned by the server, `False` otherwise. Case is ignored.

SMTP.verify(*address*)

Check the validity of an address on this server using SMTP `VRFY`. Returns a tuple consisting of code 250 and a full **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] address (including human name) if the user address is valid. Otherwise returns an SMTP error code of 400 or greater and an error string.

Nota

Muitos servidores desabilitam o SMTP `VRFY` para despistar spammers.

SMTP.login(*user*, *password*, *, *initial_response_ok=True*)

Log in on an SMTP server that requires authentication. The arguments are the username and the password to authenticate with. If there has been no previous `EHLO` or `HELO` command this session, this method tries ESMTP `EHLO` first. This method will return normally if the authentication was successful, or may raise the following exceptions:

`SMTPHeloError`

The server didn't reply properly to the `HELO` greeting.

SMTPAuthenticationError

O servidor não aceitou a combinação de usuário/senha.

SMTPNotSupportedError

The AUTH command is not supported by the server.

SMTPException

No suitable authentication method was found.

Each of the authentication methods supported by **smtplib** are tried in turn if they are advertised as supported by the server. See **auth()** for a list of supported authentication methods. *initial_response_ok* is passed through to **auth()**.

Optional keyword argument *initial_response_ok* specifies whether, for authentication methods that support it, an "initial response" as specified in **RFC 4954** [https://datatracker.ietf.org/doc/html/rfc4954.html] can be sent along with the AUTH command, rather than requiring a challenge/response.

Alterado na versão 3.5: **SMTPNotSupportedError** may be raised, and the *initial_response_ok* parameter was added.

SMTP.auth(*mechanism*, *authobject*, *, *initial_response_ok=True*)

Issue an SMTP AUTH command for the specified authentication *mechanism*, and handle the challenge response via *authobject*.

mechanism specifies which authentication mechanism is to be used as argument to the AUTH command; the valid values are those listed in the auth element of **esmtp_features**.

authobject must be a callable object taking an optional single argument:

```
data = authobject(challenge=None)
```

If optional keyword argument *initial_response_ok* is true, authobject() will be called first with no argument. It can return the **RFC 4954** [https://datatracker.ietf.org/doc/html/rfc4954.html] "initial response" ASCII str which will be encoded and sent with the AUTH command as below. If the

`authobject()` does not support an initial response (e.g. because it requires a challenge), it should return `None` when called with `challenge=None`. If *initial_response_ok* is false, then `authobject()` will not be called first with `None`.

If the initial response check returns `None`, or if *initial_response_ok* is false, `authobject()` will be called to process the server's challenge response; the *challenge* argument it is passed will be a `bytes`. It should return ASCII `str` *data* that will be base64 encoded and sent to the server.

The `SMTP` class provides `authobjects` for the `CRAM-MD5`, `PLAIN`, and `LOGIN` mechanisms; they are named `SMTP.auth_cram_md5`, `SMTP.auth_plain`, and `SMTP.auth_login` respectively. They all require that the `user` and `password` properties of the `SMTP` instance are set to appropriate values.

User code does not normally need to call `auth` directly, but can instead call the **`login()`** method, which will try each of the above mechanisms in turn, in the order listed. `auth` is exposed to facilitate the implementation of authentication methods not (or not yet) supported directly by **`smtplib`**.

Adicionado na versão 3.5.

SMTP.starttls(*, *context=None*)

Put the SMTP connection in TLS (Transport Layer Security) mode. All SMTP commands that follow will be encrypted. You should then call **`ehlo()`** again.

If *keyfile* and *certfile* are provided, they are used to create an **`ssl.SSLContext`**.

Optional *context* parameter is an **`ssl.SSLContext`** object; This is an alternative to using a keyfile and a certfile and if specified both *keyfile* and *certfile* should be `None`.

If there has been no previous `EHLO` or `HELO` command this session, this method tries ESMTP `EHLO` first.

Alterado na versão 3.12: Os parâmetros depreciados *keyfile* e *certfile* foram removidos.

`SMTPHeloError`
The server didn't reply properly to the `HELO` greeting.

`SMTPNotSupportedError`
O servidor não da suporte a extensão STARTTLS.

`RuntimeError`
SSL/TLS support is not available to your Python interpreter.

Alterado na versão 3.3: context foi adicionado.

Alterado na versão 3.4: The method now supports hostname check with **`ssl.SSLContext.check_hostname`** and *Server Name Indicator* (see **`HAS_SNI`**).

Alterado na versão 3.5: The error raised for lack of STARTTLS support is now the **`SMTPNotSupportedError`** subclass instead of the base **`SMTPException`**.

SMTP.sendmail(*from_addr*, *to_addrs*, *msg*, *mail_options=()*, *rcpt_options=()*)

Send mail. The required arguments are an **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] from-address string, a list of **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] to-address strings (a bare string will be treated as a list with 1 address), and a message string. The caller may pass a list of ESMTP options (such as `"8bitmime"`) to be used in `MAIL FROM` commands as *mail_options*. ESMTP options (such as `DSN` commands) that should be used with all `RCPT` commands can be passed as *rcpt_options*. Each option should be passed as a string containing the full text of the option, including any potential key (for instance, `"NOTIFY=SUCCESS,FAILURE"`). (If you need to use different

ESMTP options to different recipients you have to use the low-level methods such as **`mail()`**, **`rcpt()`** and **`data()`** to send the message.)

> **Nota**
>
> The *from_addr* and *to_addrs* parameters are used to construct the message envelope used by the transport agents. `sendmail` does not modify the message headers in any way.

msg may be a string containing characters in the ASCII range, or a byte string. A string is encoded to bytes using the ascii codec, and lone `\r` and `\n` characters are converted to `\r\n` characters. A byte string is not modified.

If there has been no previous `EHLO` or `HELO` command this session, this method tries ESMTP `EHLO` first. If the server does ESMTP, message size and each of the specified options will be passed to it (if the option is in the feature set the server advertises). If `EHLO` fails, `HELO` will be tried and ESMTP options suppressed.

This method will return normally if the mail is accepted for at least one recipient. Otherwise it will raise an exception. That is, if this method does not raise an exception, then someone should get your mail. If this method does not raise an exception, it returns a dictionary, with one entry for each recipient that was refused. Each entry contains a tuple of the SMTP error code and the accompanying error message sent by the server.

If `SMTPUTF8` is included in *mail_options*, and the server supports it, *from_addr* and *to_addrs* may contain non-ASCII characters.

Esse método deve levantar as seguintes exceções:

`SMTPRecipientsRefused`
: All recipients were refused. Nobody got the mail.

`SMTPHeloError`
: The server didn’t reply properly to the `HELO` greeting.

SMTPSenderRefused

O servidor não aceita o *from_addr*.

SMTPDataError

The server replied with an unexpected error code (other than a refusal of a recipient).

SMTPNotSupportedError

`SMTPUTF8` was given in the *mail_options* but is not supported by the server.

Unless otherwise noted, the connection will be open even after an exception is raised.

Alterado na versão 3.2: msg may be a byte string.

Alterado na versão 3.5: `SMTPUTF8` support added, and **SMTPNotSupportedError** may be raised if `SMTPUTF8` is specified but the server does not support it.

SMTP.send_message(*msg*, *from_addr=None*, *to_addrs=None*, *mail_options=()*, *rcpt_options=()*)

This is a convenience method for calling **sendmail()** with the message represented by an **email.message.Message** object. The arguments have the same meaning as for **sendmail()**, except that *msg* is a `Message` object.

If *from_addr* is `None` or *to_addrs* is `None`, `send_message` fills those arguments with addresses extracted from the headers of *msg* as specified in **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html]: *from_addr* is set to the *Sender* field if it is present, and otherwise to the *From* field. *to_addrs* combines the values (if any) of the *To*, *Cc*, and *Bcc* fields from *msg*. If exactly one set of *Resent-** headers appear in the message, the regular headers are ignored and the *Resent-** headers are used instead. If the message contains more than one set of *Resent-** headers, a **ValueError** is raised, since there is no way to unambiguously detect the most recent set of *Resent-* headers.

`send_message` serializes *msg* using **`BytesGenerator`** with `\r\n` as the *linesep*, and calls **`sendmail()`** to transmit the resulting message. Regardless of the values of *from_addr* and *to_addrs*, `send_message` does not transmit any *Bcc* or *Resent-Bcc* headers that may appear in *msg*. If any of the addresses in *from_addr* and *to_addrs* contain non-ASCII characters and the server does not advertise `SMTPUTF8` support, an **`SMTPNotSupportedError`** is raised. Otherwise the `Message` is serialized with a clone of its **`policy`** with the **`utf8`** attribute set to `True`, and `SMTPUTF8` and `BODY=8BITMIME` are added to *mail_options*.

Adicionado na versão 3.2.

Adicionado na versão 3.5: Support for internationalized addresses (`SMTPUTF8`).

SMTP.quit()

Terminate the SMTP session and close the connection. Return the result of the SMTP `QUIT` command.

Low-level methods corresponding to the standard SMTP/ESMTP commands `HELP`, `RSET`, `NOOP`, `MAIL`, `RCPT`, and `DATA` are also supported. Normally these do not need to be called directly, so they are not documented here. For details, consult the module code.

Additionally, an SMTP instance has the following attributes:

SMTP.helo_resp

The response to the `HELO` command, see **`helo()`**.

SMTP.ehlo_resp

The response to the `EHLO` command, see **`ehlo()`**.

SMTP.does_esmtp

A boolean value indicating whether the server supports ESMTP, see **`ehlo()`**.

SMTP.esmtp_features

A dictionary of the names of SMTP service extensions supported by the server, see `ehlo()`.

Exemplo SMTP

This example prompts the user for addresses needed in the message envelope (‘To’ and ‘From’ addresses), and the message to be delivered. Note that the headers to be included with the message must be included in the message as entered; this example doesn’t do any processing of the **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] headers. In particular, the ‘To’ and ‘From’ addresses must be included in the message headers explicitly:

```
import smtplib

def prompt(title):
    return input(title).strip()

from_addr = prompt("From: ")
to_addrs  = prompt("To: ").split()
print("Enter message, end with ^D (Unix) or ^Z (Windows):")

# Add the From: and To: headers at the start!
lines = [f"From: {from_addr}", f"To: {', '.join(to_addrs)}", ""]
while True:
    try:
        line = input()
    except EOFError:
        break
    else:
        lines.append(line)

msg = "\r\n".join(lines)
print("Message length is", len(msg))

server = smtplib.SMTP("localhost")
server.set_debuglevel(1)
server.sendmail(from_addr, to_addrs, msg)
server.quit()
```

Nota

In general, you will want to use the `email` package's features to construct an email message, which you can then send via `send_message()`; see email: Exemplos.

`uuid` — Objetos UUID conforme RFC 9562 [https://datatracker.ietf.org/doc/html/rfc9562.html]

Código-fonte: Lib/uuid.py [https://github.com/python/cpython/tree/3.15/Lib/uuid.py]

Este módulo fornece objetos `UUID` imutáveis (a classe `UUID`) e funções para gerar UUIDs correspondentes a uma versão específica de UUID, conforme especificado em **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html] (que substitui **RFC 4122** [https://datatracker.ietf.org/doc/html/rfc4122.html]), por exemplo, `uuid1()` para a versão 1 de UUID, `uuid3()` para a versão 3 de UUID e assim por diante. Observe que a versão 2 de UUID foi omitida deliberadamente, pois está fora do escopo da RFC.

Se tudo o que você quer é um ID único, você provavelmente deve chamar `uuid1()` ou `uuid4()`. Note que `uuid1()` pode comprometer a privacidade, pois cria um UUID contendo o endereço de rede do computador. `uuid4()` cria um UUID aleatório.

Dependendo do suporte da plataforma subjacente, `uuid1()` pode ou não retornar um UUID "seguro". Um UUID seguro é aquele que é gerado usando métodos de sincronização que garantem que dois processos não possam obter o mesmo UUID. Todas as instâncias de `UUID` têm um atributo `is_safe` que retransmite qualquer informação sobre a segurança do UUID, usando esta enumeração:

class uuid.SafeUUID

Adicionado na versão 3.7.

safe

O UUID foi gerado pela plataforma de forma segura para multiprocessamento.

unsafe

O UUID não foi gerado de forma segura para multiprocessamento.

unknown

A plataforma não fornece informações sobre se o UUID foi gerado com segurança ou não.

class uuid.UUID(*hex=None*, *bytes=None*, *bytes_le=None*, *fields=None*, *int=None*, *version=None*, ***, *is_safe=SafeUUID.unknown*)

Create a UUID from either a string of 32 hexadecimal digits, a 16-byte `bytes` object in big-endian order as the *bytes* argument, a 16-byte `bytes` object in little-endian order as the *bytes_le* argument, a tuple of six integers (32-bit *time_low*, 16-bit *time_mid*, 16-bit *time_hi_version*, 8-bit *clock_seq_hi_variant*, 8-bit *clock_seq_low*, 48-bit *node*) as the *fields* argument, or a single 128-bit integer as the *int* argument. When a string of hex digits is given, curly braces, hyphens, and a URN prefix are all optional. For example, these expressions all yield the same UUID:

```
UUID('{12345678-1234-5678-1234-567812345678}')
UUID('12345678123456781234567812345678')
UUID('urn:uuid:12345678-1234-5678-1234-567812345678')
UUID(bytes=b'\x12\x34\x56\x78'*4)
UUID(bytes_le=b'\x78\x56\x34\x12\x34\x12\x78\x56' +
              b'\x12\x34\x56\x78\x12\x34\x56\x78')
UUID(fields=(0x12345678, 0x1234, 0x5678, 0x12, 0x34, 0x567812
UUID(int=0x12345678123456781234567812345678)
```

Exatamente um de *hex*, *bytes*, *bytes_le*, *fields* ou *int* deve ser fornecido. O argumento *version* é opcional; se fornecido, o UUID resultante terá sua variante e número de versão definidos de acordo com **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html], substituindo bits no *hex*, *bytes*, *bytes_le*, *fields* ou *int* fornecido.

A comparação de objetos UUID é feita por meio da comparação de seus atributos `UUID.int`. A comparação com um objeto não UUID levanta

uma `TypeError`.

`str(uuid)` retorna uma string no formato `12345678-1234-5678-1234-567812345678` onde os 32 dígitos hexadecimais representam o UUID.

As instâncias `UUID` têm estes atributos de somente leitura:

UUID.bytes

The UUID as a 16-byte `bytes` object (containing the six integer fields in big-endian byte order).

UUID.bytes_le

The UUID as a 16-byte `bytes` object (with *time_low*, *time_mid*, and *time_hi_version* in little-endian byte order).

UUID.fields

Uma tupla dos seis campos de inteiros do UUID, que também estão disponíveis como seis atributos individuais e dois atributos derivados:

Campo	Significado
UUID.time_low	Os primeiros 32 bits do UUID. Relevante apenas para a versão 1.
UUID.time_mid	Os próximos 16 bits do UUID. Relevante apenas para a versão 1.
UUID.time_hi_version	Os próximos 16 bits do UUID. Relevante apenas para a versão 1.
UUID.clock_seq_hi_variant	Os próximos 8 bits do UUID. Relevante apenas para as versões 1 e 6.

UUID.clock_seq_low	Os próximos 8 bits do UUID. Relevante apenas para as versões 1 e 6.
UUID.node	Os últimos 48 bits do UUID. Relevante apenas para a versão 1.
UUID.time	O registro de data e hora de 60 bits como uma contagem de intervalos de 100 nanossegundos desde a época gregoriana (1582-10-15 00:00:00) para as versões 1 e 6, ou o registro de data e hora de 48 bits em milissegundos desde a época Unix (1970-01-01 00:00:00) para a versão 7.
UUID.clock_seq	Número de sequência de 14 bits. Relevante apenas para as versões 1 e 6.

UUID.hex

O UUID como uma sequência hexadecimal de 32 caracteres minúsculos.

UUID.int

O UUID como um inteiro de 128 bits.

UUID.urn

O UUID como um URN conforme especificado no **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html].

UUID.variant

A variante UUID, que determina o layout interno do UUID. Esta será uma das constantes `RESERVED_NCS`, `RFC_4122`, `RESERVED_MICROSOFT` ou `RESERVED_FUTURE`.

UUID.version

O número da versão do UUID (1 a 8, significativo somente quando a variante é `RFC 4122`).

Alterado na versão 3.14: Adicionadas versões 6, 7 e 8 do UUID.

UUID.is_safe

Uma enumeração de `SafeUUID` que indica se a plataforma gerou o UUID de forma segura para multiprocessamento.

Adicionado na versão 3.7.

The `uuid` module defines the following functions:

uuid.getnode()

Obtém o endereço de hardware como um inteiro positivo de 48 bits. Na primeira vez que isso for executado, ele pode iniciar um programa separado, o que pode ser bem lento. Se todas as tentativas de obter o endereço de hardware falharem, escolhemos um número aleatório de 48 bits com o bit multicast (bit menos significativo do primeiro octeto) definido como 1, conforme recomendado no **RFC 4122** [https://datatracker.ietf.org/doc/html/rfc4122.html]. "Endereço de hardware" significa o endereço MAC de uma interface de rede. Em uma máquina com várias interfaces de rede, endereços MAC administrados universalmente (ou seja, onde o segundo bit menos significativo do primeiro octeto é *não definido*) serão preferidos em relação a endereços MAC administrados localmente, mas sem nenhuma outra garantia de ordenação.

Alterado na versão 3.7: Endereços MAC administrados universalmente são preferidos em vez de endereços MAC administrados localmente, pois os primeiros têm garantia de serem globalmente exclusivos, enquanto os últimos não.

uuid.uuid1(*node=None, clock_seq=None*)

Gera um UUID a partir de um ID de host, número de sequência e hora atual de acordo com **RFC 9562, §5.1** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.1].

Quando *node* não é especificado, `getnode()` é usado para obter o endereço de hardware como um inteiro positivo de 48 bits. Quando um número de sequência *clock_seq* não é especificado, um inteiro positivo pseudoaleatório de 14 bits é gerado.

Se *node* ou *clock_seq* excederem a contagem de bits esperada, apenas seus bits menos significativos serão mantidos.

uuid.uuid3(*namespace*, *name*)

Gera um UUID com base no hash MD5 de um identificador de espaço de nomes (que é um UUID) e um nome (que é um objeto `bytes` ou uma string que será codificada usando UTF-8) de acordo com **RFC 9562, §5.3** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.3].

uuid.uuid4()

Gera um UUID aleatório em um método criptograficamente seguro de acordo com **RFC 9562, §5.4** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.4].

uuid.uuid5(*namespace*, *name*)

Gera um UUID com base no hash SHA-1 de um identificador de espaço de nomes (que é um UUID) e um nome (que é um objeto `bytes` ou uma string que será codificada usando UTF-8) de acordo com **RFC 9562, §5.5** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.5].

uuid.uuid6(*node=None*, *clock_seq=None*)

Gera um UUID a partir de um número de sequência e hora atual de acordo com **RFC 9562, §5.6** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.6].

Esta é uma alternativa ao `uuid1()` para melhorar a localidade do banco de dados.

Quando *node* não é especificado, `getnode()` é usado para obter o endereço de hardware como um inteiro positivo de 48 bits. Quando um número de sequência *clock_seq* não é especificado, um inteiro positivo pseudoaleatório de 14 bits é gerado.

Se *node* ou *clock_seq* excederem a contagem de bits esperada, apenas seus bits menos significativos serão mantidos.

Adicionado na versão 3.14.

uuid.uuid7()

Gera um UUID baseado em tempo de acordo com **RFC 9562, §5.7** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.7].

Para portabilidade entre plataformas sem precisão de submilissegundos, os UUIDs produzidos por esta função incorporam um registro de data e hora de 48 bits e usam um contador de 42 bits para garantir a monotonicidade dentro de um milissegundo.

Adicionado na versão 3.14.

uuid.uuid8(*a=None*, *b=None*, *c=None*)

Gera um UUID pseudoaleatório de acordo com **RFC 9562, §5.8** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.8].

Quando especificados, espera-se que os parâmetros *a*, *b* e *c* sejam inteiros positivos de 48, 12 e 62 bits, respectivamente. Se excederem a contagem de bits esperada, apenas os bits menos significativos serão mantidos; argumentos não especificados são substituídos por um inteiro pseudoaleatório de tamanho apropriado.

Por padrão, *a*, *b* e *c* não são gerados por um gerador de números pseudoaleatórios criptograficamente seguro (CSPRNG). Use `uuid4()` quando um UUID precisar ser usado em um contexto de segurança sensível.

Adicionado na versão 3.14.

The `uuid` module defines the following namespace identifiers for use with `uuid3()` or `uuid5()`.

uuid.NAMESPACE_DNS

Quando esse espaço de nomes é especificado, a string *name* é um nome de domínio totalmente qualificado.

uuid.NAMESPACE_URL

Quando esse espaço de nomes é especificado, a string *name* é uma URL.

uuid.NAMESPACE_OID

Quando esse espaço de nomes é especificado, a string *name* é um OID ISO.

uuid.NAMESPACE_X500

Quando esse espaço de nomes é especificado, a string *name* é um DN X.500 em DER ou um formato de saída de texto.

The **`uuid`** module defines the following constants for the possible values of the **`variant`** attribute:

uuid.RESERVED_NCS

Reservado para compatibilidade com NCS.

uuid.RFC_4122

Especifica o layout do UUID fornecido em **RFC 4122** [https://datatracker.ietf.org/doc/html/rfc4122.html]. Esta constante é mantida para retrocompatibilidade, mesmo que **RFC 4122** [https://datatracker.ietf.org/doc/html/rfc4122.html] tenha sido substituído por **RFC 9562** [https://datatracker.ietf.org/doc/html/rfc9562.html].

uuid.RESERVED_MICROSOFT

Reservado para compatibilidade com Microsoft.

uuid.RESERVED_FUTURE

Reservado para definição futura.

The **`uuid`** module defines the special Nil and Max UUID values:

uuid.NIL

Uma forma especial de UUID que é especificada para ter todos os 128 bits definidos como zero de acordo com **RFC 9562, §5.9** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.9].

Adicionado na versão 3.14.

uuid.MAX

Uma forma especial de UUID que é especificada para ter todos os 128 bits definidos como um, de acordo com **RFC 9562, §5.10** [https://datatracker.ietf.org/doc/html/rfc9562.html#section-5.10].

Adicionado na versão 3.14.

Ver também

RFC 9562 [https://datatracker.ietf.org/doc/html/rfc9562.html] - Um espaço de nomes de URN de identificador universalmente exclusivo (UUID)

Esta especificação define um espaço de nomes de Uniform Resource Name para UUIDs, o formato interno de UUIDs e métodos de geração de UUIDs.

Uso na linha de comando

Adicionado na versão 3.12.

The **`uuid`** module can be executed as a script from the command line.

```
python -m uuid [-h] [-u {uuid1,uuid3,uuid4,uuid5,uuid6,uuid7,uui
```

As seguintes opções são aceitas:

-h, --help

Mostra a mensagem de ajuda e sai.

-u <uuid>
--uuid <uuid>

Especifica o nome da função a ser usada para gerar o uuid. Por padrão, **uuid4()** é usado.

Alterado na versão 3.14: Permite gerar versões 6, 7 e 8 do UUID.

-n <namespace>
--namespace <namespace>

O espaço de nomes é um `UUID`, ou `@ns` onde `ns` é um UUID predefinido bem conhecido endereçado pelo nome do espaço de nomes. Como `@dns`, `@url`, `@oid` e `@x500`. Necessário apenas para funções **uuid3()** / **uuid5()**.

-N <name>
--name <name>

O nome usado como parte da geração do uuid. Requerido apenas para funções **uuid3()** / **uuid5()**.

-C <num>
--count <num>

Gera *num* novos UUIDs.

Adicionado na versão 3.14.

Exemplo

Here are some examples of typical usage of the **uuid** module:

```
>>> import uuid

>>> # make a UUID based on the host ID and current time
>>> uuid.uuid1()
UUID('a8098c1a-f86e-11da-bd1a-00112444be1e')

>>> # make a UUID using an MD5 hash of a namespace UUID and a na
>>> uuid.uuid3(uuid.NAMESPACE_DNS, 'python.org')
UUID('6fa459ea-ee8a-3ca4-894e-db77e160355e')

>>> # make a random UUID
>>> uuid.uuid4()
UUID('16fd2706-8baf-433b-82eb-8c7fada847da')
```

```
>>> # make a UUID using a SHA-1 hash of a namespace UUID and a r
>>> uuid.uuid5(uuid.NAMESPACE_DNS, 'python.org')
UUID('886313e1-3b8a-5372-9b90-0c9aee199e5d')

>>> # make a UUID from a string of hex digits (braces and hypher
>>> x = uuid.UUID('{00010203-0405-0607-0809-0a0b0c0d0e0f}')

>>> # convert a UUID to a string of hex digits in standard form
>>> str(x)
'00010203-0405-0607-0809-0a0b0c0d0e0f'

>>> # get the raw 16 bytes of the UUID
>>> x.bytes
b'\x00\x01\x02\x03\x04\x05\x06\x07\x08\t\n\x0b\x0c\r\x0e\x0f'

>>> # make a UUID from a 16-byte bytes object
>>> uuid.UUID(bytes=x.bytes)
UUID('00010203-0405-0607-0809-0a0b0c0d0e0f')

>>> # get the Nil UUID
>>> uuid.NIL
UUID('00000000-0000-0000-0000-000000000000')

>>> # get the Max UUID
>>> uuid.MAX
UUID('ffffffff-ffff-ffff-ffff-ffffffffffff')

>>> # same as UUIDv1 but with fields reordered to improve DB loc
>>> uuid.uuid6()
UUID('1f0799c0-98b9-62db-92c6-a0d365b91053')

>>> # get UUIDv7 creation (local) time as a timestamp in millise
>>> u = uuid.uuid7()
>>> u.time
1743936859822

>>> # get UUIDv7 creation (local) time as a datetime object
>>> import datetime as dt
>>> dt.datetime.fromtimestamp(u.time / 1000)
datetime.datetime(...)

>>> # make a UUID with custom blocks
```

```
>>> uuid.uuid8(0x12345678, 0x9abcdef0, 0x11223344)
UUID('00001234-5678-8ef0-8000-000011223344')
```

Exemplo de linha de comando

Here are some examples of typical usage of the **`uuid`** command-line interface:

```
# gera um UUID aleatório - por padrão uuid4() é usado
$ python -m uuid

# gera um UUID usando uuid1()
$ python -m uuid -u uuid1

# gera um UUID usando uuid5
$ python -m uuid -u uuid5 -n @url -N example.com

# gera 42 UUIDs aleatórios
$ python -m uuid -C 42
```

`socketserver` — Um framework para servidores de rede

Código-fonte: Lib/socketserver.py
[https://github.com/python/cpython/tree/3.15/Lib/socketserver.py]

O módulo **`socketserver`** simplifica a tarefa de escrever servidores de rede.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Existem quatro classes básicas de servidores concretos:

class socketserver.TCPServer(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

Isso usa o protocolo TCP da internet, que fornece fluxos contínuos de dados entre o cliente e o servidor. Se *bind_and_activate* for true, o construtor tenta automaticamente invocar **`server_bind()`** e **`server_activate()`**. Os outros parâmetros são passados para a classe base **`BaseServer`**.

Alterado na versão 3.15: O tamanho padrão de filas agora é `socket.SOMAXCONN` para **`socketserver.TCPServer`**.

class socketserver.UDPServer(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

Isso usa datagramas, que são pacotes discretos de informações que podem chegar fora de ordem ou ser perdidos durante o trânsito. Os parâmetros são os mesmos de **`TCPServer`**.

class socketserver.UnixStreamServer(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

class socketserver.UnixDatagramServer(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*)

Essas classes usadas com menos frequência são semelhantes às classes TCP e UDP, mas usam soquetes de domínio Unix; elas não estão disponíveis em plataformas não Unix. Os parâmetros são os mesmos de `TCPServer`.

Essas quatro classes processam solicitações *synchronously*; cada solicitação deve ser concluída antes que a próxima solicitação possa ser iniciada. Isso não é adequado se cada solicitação demorar muito para ser concluída, porque exige muita computação ou porque retorna muitos dados que o cliente demora a processar. A solução é criar um processo ou thread separado para lidar com cada solicitação; as classes misturadas `ForkingMixIn` e `ThreadingMixIn` podem ser usadas para oferecer suporte ao comportamento assíncrono.

Criar um servidor requer várias etapas. Primeiro, você deve criar uma classe de manipulador de solicitação estendendo a classe `BaseRequestHandler` e substituindo seu método `handle()`; esse método processará as solicitações recebidas. Segundo, você deve instanciar uma das classes de servidor, passando a ela o endereço do servidor e a classe de manipulador de solicitação. É recomendável usar o servidor em uma instrução `with`. Em seguida, chame o método `handle_request()` ou `serve_forever()` do objeto do servidor para processar uma ou muitas solicitações. Finalmente, chame `server_close()` para fechar o soquete (a menos que você tenha usado uma instrução `with`).

Ao herdar de `ThreadingMixIn` para comportamento de conexão em threads, você deve declarar explicitamente como quer que suas threads se comportem em um desligamento abrupto. A classe `ThreadingMixIn` define um atributo *daemon_threads*, que indica se o servidor deve ou não esperar pelo término da thread. Você deve definir o sinalizador explicitamente se quiser que as threads se comportem de forma autônoma; o padrão é `False`, o que significa que o Python não sairá até que todas as threads criadas por `ThreadingMixIn` tenham sido encerradas.

As classes de servidor têm os mesmos métodos e atributos externos, não importa qual protocolo de rede eles usam.

Notas sobre criação do servidor

Existem cinco classes em um diagrama de herança, quatro das quais representam servidores síncronos de quatro tipos:

```
+------------+
| BaseServer |
+------------+
      |
      v
+-----------+        +------------------+
| TCPServer |------->| UnixStreamServer |
+-----------+        +------------------+
      |
      v
+-----------+        +--------------------+
| UDPServer |------->| UnixDatagramServer |
+-----------+        +--------------------+
```

Observe que `UnixDatagramServer` deriva de `UDPServer`, não de `UnixStreamServer` — a única diferença entre um IP e um servidor Unix é a família de endereços.

class socketserver.ForkingMixIn
class socketserver.ThreadingMixIn

Versões de fork e threads de cada tipo de servidor podem ser criadas usando essas classes mix-in. Por exemplo, `ThreadingUDPServer` é criado da seguinte forma:

```
class ThreadingUDPServer(ThreadingMixIn, UDPServer):
    pass
```

A classe mix-in vem primeiro, pois ela substitui um método definido em `UDPServer`. Definir os vários atributos também altera o comportamento do mecanismo de servidor subjacente.

`ForkingMixIn` e as classes Forking mencionadas abaixo estão disponíveis apenas em plataformas POSIX que oferecem suporte a `fork()`.

block_on_close

`ForkingMixIn.server_close` aguarda até que todos os processos filho sejam concluídos, exceto se o atributo `block_on_close` for `False`.

`ThreadingMixIn.server_close` aguarda até que todos as threads não daemon sejam concluídos, exceto se o atributo `block_on_close` for `False`.

max_children

Especifica quantos processos filho existirão para processar solicitações simultaneamente para `ForkingMixIn`. Se o limite for atingido, novas solicitações aguardarão até que um processo filho seja concluído.

daemon_threads

Para `ThreadingMixIn`, use threads como daemon definindo `ThreadingMixIn.daemon_threads` como `True` para não esperar até que as threads sejam concluídas.

Alterado na versão 3.7: `ForkingMixIn.server_close` e **`ThreadingMixIn.server_close`** agora esperam até que todos os processos filhos e threads não daemon sejam concluídos. Adicione um novo atributo de classe `ForkingMixIn.block_on_close` para optar pelo comportamento pré-3.7.

class socketserver.ForkingTCPServer
class socketserver.ForkingUDPServer
class socketserver.ThreadingTCPServer
class socketserver.ThreadingUDPServer
class socketserver.ForkingUnixStreamServer
class socketserver.ForkingUnixDatagramServer
class socketserver.ThreadingUnixStreamServer

class socketserver.ThreadingUnixDatagramServer

Essas classes são predefinidas usando as classes mix-in.

Adicionado na versão 3.12: As classes `ForkingUnixStreamServer` e `ForkingUnixDatagramServer` foram adicionadas.

Para implementar um serviço, você deve derivar uma classe de **`BaseRequestHandler`** e redefinir seu método **`handle()`**. Você pode então executar várias versões do serviço combinando uma das classes de servidor com sua classe de manipulador de solicitação. A classe de manipulador de solicitação deve ser diferente para serviços de datagrama ou fluxo. Isso pode ser ocultado usando as subclasses de manipulador **`StreamRequestHandler`** ou **`DatagramRequestHandler`**.

Claro, você ainda tem que usar sua cabeça! Por exemplo, não faz sentido usar um servidor de Forking se o serviço contém estado na memória que pode ser modificado por diferentes solicitações, já que as modificações no processo filho nunca atingiriam o estado inicial mantido no processo pai e passado para cada filho. Neste caso, você pode usar um servidor de Threading, mas provavelmente terá que usar travas para proteger a integridade dos dados compartilhados.

Por outro lado, se você estiver construindo um servidor HTTP onde todos os dados são armazenados externamente (por exemplo, no sistema de arquivos), uma classe síncrona essencialmente tornará o serviço “surdo” enquanto uma solicitação estiver sendo manipulada – o que pode ser por um tempo muito longo se um cliente for lento para receber todos os dados que solicitou. Aqui, um servidor de Threading ou Forking é apropriado.

Em alguns casos, pode ser apropriado processar parte de uma solicitação de forma síncrona, mas terminar o processamento em um filho bifurcado dependendo dos dados da solicitação. Isso pode ser implementado usando um servidor síncrono e fazendo uma bifurcação explícita no método da classe do manipulador de solicitações **`handle()`**.

Outra abordagem para lidar com múltiplas requisições simultâneas em um ambiente que não oferece suporte a threads nem **`fork()`** (ou onde estes são muito caros ou inapropriados para o serviço) é manter uma tabela explícita

de requisições parcialmente finalizadas e usar `selectors` para decidir em qual requisição trabalhar em seguida (ou se deve lidar com uma nova requisição recebida). Isto é particularmente importante para serviços de fluxo onde cada cliente pode potencialmente ficar conectado por um longo tempo (se threads ou subprocessos não puderem ser usados).

Objetos Server

class socketserver.BaseServer(*server_address*, *RequestHandlerClass*)

Esta é a superclasse de todos os objetos Server no módulo. Ela define a interface, dada abaixo, mas não implementa a maioria dos métodos, o que é feito em subclasses. Os dois parâmetros são armazenados nos respectivos atributos `server_address` e `RequestHandlerClass`.

fileno()

Retorna um descritor de arquivo inteiro para o soquete no qual o servidor está escutando. Esta função é mais comumente passada para `selectors`, para permitir o monitoramento de múltiplos servidores no mesmo processo.

handle_request()

Processa uma única solicitação. Esta função chama os seguintes métodos em ordem: `get_request()`, `verify_request()` e `process_request()`. Se o método `handle()` fornecido pelo usuário da classe manipuladora levantar uma exceção, o método `handle_error()` do servidor será chamado. Se nenhuma solicitação for recebida dentro de `timeout` segundos, `handle_timeout()` será chamado e `handle_request()` retornará.

serve_forever(*poll_interval=0.5*)

Manipula solicitações até uma solicitação explícita `shutdown()`. Sonda para desligamento a cada *poll_interval* segundos. Ignora o atributo `timeout`. Ele também chama `service_actions()`, que pode ser usado por uma subclasse ou mixin para fornecer ações específicas para um determinado serviço. Por exemplo, a classe

`ForkingMixIn` usa **`service_actions()`** para limpar processos filhos zumbis.

Alterado na versão 3.3: Adicionada chamada `service_actions` ao método `serve_forever`.

service_actions()

Isso é chamado no laço `serve_forever()`. Este método pode ser substituído por subclasses ou classes mixin para executar ações específicas para um determinado serviço, como ações de limpeza.

Adicionado na versão 3.3.

shutdown()

Diz ao laço `serve_forever()` para parar e esperar até que isso aconteça. `shutdown()` deve ser chamado enquanto **`serve_forever()`** estiver em execução em um thread diferente, caso contrário, ocorrerá um impasse.

server_close()

Limpa o servidor. Pode ser substituído.

address_family

A família de protocolos à qual o soquete do servidor pertence. Exemplos comuns são `socket.AF_INET`, `socket.AF_INET6` e `socket.AF_UNIX`. Subclasse as classes de servidor TCP ou UDP neste módulo com o atributo de classe `address_family = AF_INET6` definido se você quiser classes de servidor IPv6.

RequestHandlerClass

A classe de manipulador de solicitações fornecida pelo usuário; uma instância dessa classe é criada para cada solicitação.

server_address

O endereço no qual o servidor está escutando. O formato dos endereços varia dependendo da família do protocolo; veja a documentação do módulo `socket` para detalhes. Para protocolos de

internet, esta é uma tupla contendo uma string dando o endereço e um número de porta inteiro: `('127.0.0.1', 80)`, por exemplo.

socket

O objeto soquete no qual o servidor ouve para solicitações recebidas.

As classes de servidor oferecem suporte às seguintes variáveis de classe:

allow_reuse_address

Se o servidor permitirá a reutilização de um endereço. O padrão é `False`, e pode ser definido em subclasses para alterar a política.

request_queue_size

O tamanho da fila de requisições. Se levar muito tempo para processar uma única requisição, todas as requisições que chegarem enquanto o servidor estiver ocupado serão colocadas em uma fila, até `request_queue_size` requisições. Quando a fila estiver cheia, outras requisições de clientes receberão um erro “Conexão negada”. O valor padrão é geralmente 5, mas isso pode ser substituído por subclasses.

socket_type

O tipo de soquete usado pelo servidor; `socket.SOCK_STREAM` e `socket.SOCK_DGRAM` são dois valores comuns.

timeout

Duração do tempo limite, medida em segundos, ou `None` se nenhum tempo limite for desejado. Se `handle_request()` não receber nenhuma requisição de entrada dentro do período de tempo limite, o método `handle_timeout()` será chamado.

Existem vários métodos de servidor que podem ser substituídos por subclasses de classes de servidor base, como `TCPServer`; esses métodos não são úteis para usuários externos do objeto de servidor.

finish_request(*request*, *client_address*)

Na verdade, processa a solicitação instanciando `RequestHandlerClass` e chamando seu método `handle()`.

get_request()

Deve aceitar uma solicitação do soquete e retornar uma tupla de 2 elementos contendo o *novo* objeto de soquete a ser usado para se comunicar com o cliente e o endereço do cliente.

handle_error(*request*, *client_address*)

Esta função é chamada se o método `handle()` de uma instância `RequestHandlerClass` levanta uma exceção. A ação padrão é exibir o traceback para erro padrão e continuar manipulando solicitações futuras.

Alterado na versão 3.6: Agora é chamada apenas para exceções derivadas da classe `Exception`.

handle_timeout()

Esta função é chamada quando o atributo `timeout` foi definido para um valor diferente de `None` e o período de timeout passou sem que nenhuma solicitação tenha sido recebida. A ação padrão para servidores de bifurcação é coletar o status de quaisquer processos filhos que tenham saído, enquanto em servidores de threading esse método não faz nada.

process_request(*request*, *client_address*)

Chama `finish_request()` para criar uma instância de `RequestHandlerClass`. Se desejado, esta função pode criar um novo processo ou thread para manipular a solicitação; as classes `ForkingMixIn` e `ThreadingMixIn` fazem isso.

server_activate()

Chamado pelo construtor do servidor para ativar o servidor. O comportamento padrão para um servidor TCP apenas invoca `listen()` no soquete do servidor. Pode ser substituído.

server_bind()

Chamada pelo construtor do servidor para vincular o soquete ao endereço desejado. Pode ser substituída.

verify_request(*request*, *client_address*)

Deve retornar um valor booleano; se o valor for `True`, a solicitação será processada, e se for `False`, a solicitação será negada. Esta função pode ser substituída para implementar controles de acesso para um servidor. A implementação padrão sempre retorna `True`.

Alterado na versão 3.6: Foi adicionado suporte para o protocolo gerenciador de contexto. Sair do gerenciador de contexto é equivalente a chamar `server_close()`.

Objetos manipulador de requisições

class socketserver.BaseRequestHandler

Esta é a superclasse de todos os objetos manipulador de requisições. Ela define a interface, dada abaixo. Uma subclasse concreta do manipulador de requisições deve definir um novo método `handle()` e pode substituir qualquer um dos outros métodos. Uma nova instância da subclasse é criada para cada requisição.

setup()

Chamada antes do método `handle()` para executar quaisquer ações de inicialização necessárias. A implementação padrão não faz nada.

handle()

Esta função deve fazer todo o trabalho necessário para atender a uma solicitação. A implementação padrão não faz nada. Vários atributos de instância estão disponíveis para ela; a solicitação está disponível como `request`; o endereço do cliente como `client_address`; e a instância do servidor como `server`, caso precise de acesso a informações por servidor.

O tipo de `request` é diferente para serviços de datagrama ou fluxo. Para serviços de fluxo, `request` é um objeto soquete; para serviços de datagrama, `request` é um par de string e soquete.

finish()

Chamada após o método `handle()` para executar quaisquer ações de limpeza necessárias. A implementação padrão não faz nada. Se `setup()` levanta uma exceção, esta função não será chamada.

request

O *novo* objeto `socket.socket` a ser usado para se comunicar com o cliente.

client_address

Endereço do cliente retornado por `BaseServer.get_request()`.

server

Objeto `BaseServer` usado para manipular a solicitação.

class socketserver.StreamRequestHandler
class socketserver.DatagramRequestHandler

Essas subclasses `BaseRequestHandler` substituem os métodos `setup()` e `finish()` e fornecem os atributos `rfile` e `wfile`.

rfile

Um objeto arquivo do qual recebe a solicitação é lido. Oferece suporte à interface legível `io.BufferedIOBase`.

wfile

Um objeto arquivo no qual a resposta é escrita. Oferece suporte à interface gravável `io.BufferedIOBase`

Alterado na versão 3.6: `wfile` também oferece suporte à interface gravável `io.BufferedIOBase`.

Exemplos

Exemplo de `socketserver.TCPServer`

Este é o lado do servidor:

```
import socketserver

class MyTCPHandler(socketserver.BaseRequestHandler):
    """
    A classe de manipulador de solicitação para nosso servidor.

    Ela é instanciada uma vez por conexão com o servidor e deve
    substituir o método handle() para implementar a comunicação
    com o cliente.
    """

    def handle(self):
        # self.request é o soquete TCP conectado ao cliente
        pieces = [b'']
        total = 0
        while b'\n' not in pieces[-1] and total < 10_000:
            pieces.append(self.request.recv(2000))
            total += len(pieces[-1])
        self.data = b''.join(pieces)
        print(f"Received from {self.client_address[0]}:")
        print(self.data.decode("utf-8"))
        # só envia de volta os mesmos dados, mas em caixa alta
        self.request.sendall(self.data.upper())
        # após retornarmos, o soquete será fechado.

if __name__ == "__main__":
    HOST, PORT = "localhost", 9999

    # Cria o servidor, ligando-o ao localhost na porta 9999
    with socketserver.TCPServer((HOST, PORT), MyTCPHandler) as s
        # Ativa o servidor; isso vai mantê-lo em execução até vo
        # interromper o programa com Ctrl-C
        server.serve_forever()
```

Uma classe de manipulador de solicitações alternativa que faz uso de fluxos (objetos arquivo ou similares que simplificam a comunicação ao fornecer a interface de arquivo padrão):

```
class MyTCPHandler(socketserver.StreamRequestHandler):

    def handle(self):
        # self.rfile é um objeto arquivo ou similar criado pelo
        # Agora podemos usar, por exemplo, readline() em vez de
        # brutas a recv().
        # Nos limitamos a 10000 bytes para evitar abuso pelo rem
        self.data = self.rfile.readline(10000).rstrip()
        print(f"{self.client_address[0]} wrote:")
        print(self.data.decode("utf-8"))
        # Da mesma forma, self.wfile é um objeto arquivo ou simi
        # escrever de volta para o cliente
        self.wfile.write(self.data.upper())
```

A diferença é que a chamada `readline()` no segundo manipulador chamará `recv()` várias vezes até encontrar um caractere de nova linha, enquanto o primeiro manipulador teve que usar um laço `recv()` para acumular dados até uma nova linha. Se tivesse usado apenas um único `recv()` sem o laço, ele teria retornado apenas o que foi recebido até agora do cliente. O TCP é baseado em fluxo: os dados chegam na ordem em que foram enviados, mas não há correlação entre as chamadas `send()` ou `sendall()` do cliente e o número de chamadas `recv()` no servidor necessárias para recebê-los.

Este é o lado do cliente:

```
import socket
import sys

HOST, PORT = "localhost", 9999
data = " ".join(sys.argv[1:])

# Cria um soquete (SOCK_STREAM significa um soquete TCP)
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as sock:
    # Conecta a um servidor e envia dados
    sock.connect((HOST, PORT))
    sock.sendall(bytes(data, "utf-8"))
    sock.sendall(b"\n")

    # Recebe dados do servidor e desliga
    received = str(sock.recv(1024), "utf-8")
```

```
print("Sent:    ", data)
print("Received:", received)
```

A saída do exemplo deve se parecer com algo como isso:

Servidor:

```
$ python TCPServer.py
127.0.0.1 wrote:
b'hello world with TCP'
127.0.0.1 wrote:
b'python is nice'
```

Cliente:

```
$ python TCPClient.py hello world with TCP
Sent:     hello world with TCP
Received: HELLO WORLD WITH TCP
$ python TCPClient.py python is nice
Sent:     python is nice
Received: PYTHON IS NICE
```

Exemplo `socketserver.UDPServer`

Este é o lado do servidor:

```
import socketserver

class MyUDPHandler(socketserver.BaseRequestHandler):
    """
    Esta classe funciona de forma semelhante à classe do manipul
    exceto que self.request consiste em um par de dados e soquet
    e, como não há conexão, o endereço do cliente deve ser forne
    explicitamente ao enviar dados de volta via sendto().
    """

    def handle(self):
        data = self.request[0].strip()
        socket = self.request[1]
        print(f"{self.client_address[0]} wrote:")
        print(data)
        socket.sendto(data.upper(), self.client_address)
```

```
if __name__ == "__main__":
    HOST, PORT = "localhost", 9999
    with socketserver.UDPServer((HOST, PORT), MyUDPHandler) as s
        server.serve_forever()
```

Este é o lado do cliente:

```
import socket
import sys

HOST, PORT = "localhost", 9999
data = " ".join(sys.argv[1:])

# SOCK_DGRAM é o tipo de soquete para usar para soquetes UDP
sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)

# Como você pode ver, não há uma chamada connect(); UDP não tem
# Em vez disso, dados são enviados diretamente para o destinatár
sock.sendto(bytes(data + "\n", "utf-8"), (HOST, PORT))
received = str(sock.recv(1024), "utf-8")

print("Sent:    ", data)
print("Received:", received)
```

A saída do exemplo deve ser exatamente igual à do exemplo do servidor TCP.

Mixins assíncronos

Para construir manipuladores assíncronos, use as classes `ThreadingMixIn` e `ForkingMixIn`.

Um exemplo para a classe `ThreadingMixIn`:

```
import socket
import threading
import socketserver

class ThreadedTCPRequestHandler(socketserver.BaseRequestHandler)
```

```
    def handle(self):
        data = str(self.request.recv(1024), 'ascii')
        cur_thread = threading.current_thread()
        response = bytes("{}: {}".format(cur_thread.name, data),
        self.request.sendall(response)

class ThreadedTCPServer(socketserver.ThreadingMixIn, socketserve
    pass

def client(ip, port, message):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as so
        sock.connect((ip, port))
        sock.sendall(bytes(message, 'ascii'))
        response = str(sock.recv(1024), 'ascii')
        print("Received: {}".format(response))

if __name__ == "__main__":
    # Porta 0 significa selecionar uma porta não usada arbitrari
    HOST, PORT = "localhost", 0

    server = ThreadedTCPServer((HOST, PORT), ThreadedTCPRequestH
    with server:
        ip, port = server.server_address

        # Inicia uma thread com o servidor -- essa thread vai er
        # iniciar mais um thread para cada requisição
        server_thread = threading.Thread(target=server.serve_for
        # Sai da thread do servidor quando a thread principal é
        server_thread.daemon = True
        server_thread.start()
        print("Server loop running in thread:", server_thread.na

        client(ip, port, "Hello World 1")
        client(ip, port, "Hello World 2")
        client(ip, port, "Hello World 3")

        server.shutdown()
```

A saída do exemplo deve se parecer com algo como isso:

```
$ python ThreadedTCPServer.py
Server loop running in thread: Thread-1
Received: Thread-2: Hello World 1
Received: Thread-3: Hello World 2
Received: Thread-4: Hello World 3
```

A classe **`ForkingMixIn`** é usada da mesma forma, exceto que o servidor gerará um novo processo para cada solicitação. Disponível apenas em plataformas POSIX que oferecem suporte a **`fork()`**.

`http.server` — HTTP servers

Código-fonte: Lib/http/server.py
[https://github.com/python/cpython/tree/3.15/Lib/http/server.py]

This module defines classes for implementing HTTP servers.

Aviso

`http.server` is not recommended for production. It only implements basic security checks.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

One class, **`HTTPServer`**, is a **`socketserver.TCPServer`** subclass. It creates and listens at the HTTP socket, dispatching the requests to a handler. Code to create and run the server looks like this:

```
def run(server_class=HTTPServer, handler_class=BaseHTTPRequestHa
    server_address = ('', 8000)
    httpd = server_class(server_address, handler_class)
    httpd.serve_forever()
```

class http.server.HTTPServer(*server_address*, *RequestHandlerClass*)

This class builds on the **`TCPServer`** class by storing the server address as instance variables named **`server_name`** and **`server_port`**. The server is accessible by the handler, typically through the handler's **`server`** instance variable.

server_name

The HTTP server's fully qualified domain name.

server_port

The HTTP server's port number obtained from *server_address*.

class http.server.ThreadingHTTPServer(*server_address*, *RequestHandlerClass*)

This class is identical to HTTPServer but uses threads to handle requests by using the `ThreadingMixIn`. This is useful to handle web browsers pre-opening sockets, on which `HTTPServer` would wait indefinitely.

Adicionado na versão 3.7.

class http.server.HTTPSServer(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*, *, *certfile*, *keyfile=None*, *password=None*, *alpn_protocols=None*)

Subclass of `HTTPServer` with a wrapped socket using the `ssl` module. If the `ssl` module is not available, instantiating a `HTTPSServer` object fails with a `RuntimeError`.

The *certfile* argument is the path to the SSL certificate chain file, and the *keyfile* is the path to the file containing the private key.

A *password* can be specified for files protected and wrapped with PKCS#8, but beware that this could possibly expose hardcoded passwords in clear.

Ver também

See `ssl.SSLContext.load_cert_chain()` for additional information on the accepted values for *certfile*, *keyfile* and *password*.

When specified, the *alpn_protocols* argument must be a sequence of strings specifying the "Application-Layer Protocol Negotiation" (ALPN) protocols supported by the server. ALPN allows the server and

the client to negotiate the application protocol during the TLS handshake.

By default, it is set to `["http/1.1"]`, meaning the server supports HTTP/1.1.

Adicionado na versão 3.14.

class http.server.ThreadingHTTPSServer(*server_address*, *RequestHandlerClass*, *bind_and_activate=True*, *, *certfile*, *keyfile=None*, *password=None*, *alpn_protocols=None*)

This class is identical to **`HTTPSServer`** but uses threads to handle requests by inheriting from **`ThreadingMixIn`**. This is analogous to **`ThreadingHTTPServer`** only using **`HTTPSServer`**.

Adicionado na versão 3.14.

The **`HTTPServer`**, **`ThreadingHTTPServer`**, **`HTTPSServer`** and **`ThreadingHTTPSServer`** must be given a *RequestHandlerClass* on instantiation, of which this module provides three different variants:

class http.server.BaseHTTPRequestHandler(*request*, *client_address*, *server*)

This class is used to handle the HTTP requests that arrive at the server. By itself, it cannot respond to any actual HTTP requests; it must be subclassed to handle each request method (for example, `'GET'` or `'POST'`). **`BaseHTTPRequestHandler`** provides a number of class and instance variables, and methods for use by subclasses.

The handler will parse the request and the headers, then call a method specific to the request type. The method name is constructed from the request. For example, for the request method `SPAM`, the **`do_SPAM()`** method will be called with no arguments. All of the relevant information is stored in instance variables of the handler. Subclasses should not need to override or extend the **`__init__()`** method.

`BaseHTTPRequestHandler` tem os seguintes atributos de instância:

client_address

Contains a tuple of the form `(host, port)` referring to the client's address.

server

Contains the server instance.

close_connection

Boolean that should be set before `handle_one_request()` returns, indicating if another request may be expected, or if the connection should be shut down.

requestline

Contains the string representation of the HTTP request line. The terminating CRLF is stripped. This attribute should be set by `handle_one_request()`. If no valid request line was processed, it should be set to the empty string.

command

Contains the command (request type). For example, `'GET'`.

path

Contains the request path. If the query component of the URL is present, then `path` includes the query. Using the terminology of **RFC 3986** [https://datatracker.ietf.org/doc/html/rfc3986.html], `path` here includes `hier-part` and the `query`.

request_version

Contains the version string from the request. For example, `'HTTP/1.0'`.

headers

Holds an instance of the class specified by the `MessageClass` class variable. This instance parses and manages the headers in the HTTP request. The `parse_headers()` function from `http.client` is used to parse the headers and it requires that the HTTP request provide a valid **RFC 5322** [https://datatracker.ietf.org/doc/html/rfc5322.html] style header.

rfile

An `io.BufferedIOBase` input stream, ready to read from the start of the optional input data.

wfile

Contains the output stream for writing a response back to the client. Proper adherence to the HTTP protocol must be used when writing to this stream in order to achieve successful interoperation with HTTP clients.

Alterado na versão 3.6: This is an `io.BufferedIOBase` stream.

`BaseHTTPRequestHandler` tem os seguintes atributos:

server_version

Specifies the server software version. You may want to override this. The format is multiple whitespace-separated strings, where each string is of the form name[/version]. For example, `'BaseHTTP/0.2'`.

sys_version

Contains the Python system version, in a form usable by the `version_string` method and the `server_version` class variable. For example, `'Python/1.4'`.

error_message_format

Specifies a format string that should be used by `send_error()` method for building an error response to the client. The string is filled by default with variables from `responses` based on the status code passed to **`send_error()`**.

error_content_type

Specifies the Content-Type HTTP header of error responses sent to the client. The default value is `'text/html'`.

protocol_version

Specifies the HTTP version to which the server is conformant. It is sent in responses to let the client know the server's communication capabilities for future requests. If set to `'HTTP/1.1'`, the server will permit HTTP persistent connections; however, your server *must* then include an accurate `Content-Length` header (using **`send_header()`**) in all of its responses to clients. For backwards compatibility, the setting defaults to `'HTTP/1.0'`.

MessageClass

Specifies an **`email.message.Message`**-like class to parse HTTP headers. Typically, this is not overridden, and it defaults to **`http.client.HTTPMessage`**.

responses

This attribute contains a mapping of error code integers to two-element tuples containing a short and long message. For example, `{code: (shortmessage, longmessage)}`. The *shortmessage* is usually used as the *message* key in an error response, and *longmessage* as the *explain* key. It is used by **`send_response_only()`** and **`send_error()`** methods.

Uma instância de **`BaseHTTPRequestHandler`** tem os seguintes métodos:

handle()

Calls **`handle_one_request()`** once (or, if persistent connections are enabled, multiple times) to handle incoming HTTP requests. You should never need to override it; instead, implement appropriate **`do_*()`** methods.

handle_one_request()

This method will parse and dispatch the request to the appropriate **`do_*()`** method. You should never need to override it.

handle_expect_100()

When an HTTP/1.1 conformant server receives an `Expect: 100-continue` request header it responds with a `100 Continue` followed by `200 OK` headers. This method can be overridden to raise an error

if the server does not want the client to continue. For example, the server can choose to send `417 Expectation Failed` as a response header and `return False`.

Adicionado na versão 3.2.

send_error(*code*, *message=None*, *explain=None*)

Sends and logs a complete error reply to the client. The numeric *code* specifies the HTTP error code, with *message* as an optional, short, human readable description of the error. The *explain* argument can be used to provide more detailed information about the error; it will be formatted using the **`error_message_format`** attribute and emitted, after a complete set of headers, as the response body. The **`responses`** attribute holds the default values for *message* and *explain* that will be used if no value is provided; for unknown codes the default value for both is the string `???`. The body will be empty if the method is HEAD or the response code is one of the following: `1`*xx*, `204 No Content`, `205 Reset Content`, `304 Not Modified`.

Alterado na versão 3.4: The error response includes a Content-Length header. Added the *explain* argument.

send_response(*code*, *message=None*)

Adds a response header to the headers buffer and logs the accepted request. The HTTP response line is written to the internal buffer, followed by *Server* and *Date* headers. The values for these two headers are picked up from the **`version_string()`** and **`date_time_string()`** methods, respectively. If the server does not intend to send any other headers using the **`send_header()`** method, then **`send_response()`** should be followed by an **`end_headers()`** call.

Alterado na versão 3.3: Headers are stored to an internal buffer and **`end_headers()`** needs to be called explicitly.

send_header(*keyword*, *value*)

Adds the HTTP header to an internal buffer which will be written to the output stream when either **`end_headers()`** or **`flush_headers()`** is invoked. *keyword* should specify the header keyword, with *value* specifying its value. Note that, after the send_header calls are done, **`end_headers()`** MUST BE called in order to complete the operation.

This method does not reject input containing CRLF sequences.

Alterado na versão 3.2: Headers are stored in an internal buffer.

send_response_only(*code*, *message=None*)

Sends the response header only, used for the purposes when `100 Continue` response is sent by the server to the client. The headers are not buffered and sent directly the output stream. If the *message* is not specified, the HTTP message corresponding the response *code* is sent.

This method does not reject *message* containing CRLF sequences.

Adicionado na versão 3.2.

end_headers()

Adds a blank line (indicating the end of the HTTP headers in the response) to the headers buffer and calls **`flush_headers()`**.

Alterado na versão 3.2: The buffered headers are written to the output stream.

flush_headers()

Finally send the headers to the output stream and flush the internal headers buffer.

Adicionado na versão 3.3.

log_request(*code='-', size='-'*)

Logs an accepted (successful) request. *code* should specify the numeric HTTP code associated with the response. If a size of the

response is available, then it should be passed as the *size* parameter.

log_error(...)

Logs an error when a request cannot be fulfilled. By default, it passes the message to `log_message()`, so it takes the same arguments (*format* and additional values).

log_message(*format*, ...)

Logs an arbitrary message to `sys.stderr`. This is typically overridden to create custom error logging mechanisms. The *format* argument is a standard printf-style format string, where the additional arguments to **`log_message()`** are applied as inputs to the formatting. The client IP address and current date and time are prefixed to every message logged.

version_string()

Returns the server software's version string. This is a combination of the `server_version` and `sys_version` attributes.

date_time_string(*timestamp=None*)

Returns the date and time given by *timestamp* (which must be `None` or in the format returned by `time.time()`), formatted for a message header. If *timestamp* is omitted, it uses the current date and time.

The result looks like `'Sun, 06 Nov 1994 08:49:37 GMT'`.

log_date_time_string()

Returns the current date and time, formatted for logging.

address_string()

Retorna o endereço do cliente.

Alterado na versão 3.3: Previously, a name lookup was performed. To avoid name resolution delays, it now always returns the IP address.

class http.server.SimpleHTTPRequestHandler(*request*, *client_address*, *server*, *, *directory=None*, *extra_response_headers=None*)

This class serves files from the directory *directory* and below, or the current directory if *directory* is not provided, directly mapping the directory structure to HTTP requests.

Alterado na versão 3.7: Added the *directory* parameter.

Alterado na versão 3.9: The *directory* parameter accepts a path-like object.

Alterado na versão 3.15: Added *extra_response_headers* parameter.

Muito do trabalho, como analisar o pedido, é feito pela classe base **`BaseHTTPRequestHandler`**. Esta classe implementa as funções **`do_GET()`** e **`do_HEAD()`**.

The following are defined as class-level attributes of **`SimpleHTTPRequestHandler`**:

server_version

This will be `"SimpleHTTP/" + __version__`, where `__version__` is defined at the module level.

default_content_type

Specifies the Content-Type header value sent when the MIME type cannot be guessed from the file extension of the requested URL. By default, it is set to `'application/octet-stream'`.

Adicionado na versão 3.15.

index_pages

Specifies the filenames that are treated as directory index pages.

Defaults to `("index.html", "index.htm")`.

Adicionado na versão 3.12.

extensions_map

A dictionary mapping suffixes into MIME types, contains custom overrides for the default system mappings. The mapping is used case-insensitively, and so should contain only lower-cased keys.

Alterado na versão 3.9: This dictionary is no longer filled with the default system mappings, but only contains overrides.

extra_response_headers

A sequence of `(name, value)` pairs containing user-defined extra HTTP response headers to add to each successful HTTP status 200 response. These headers are not included in other status code responses.

Headers that the server sends automatically such as `Content-Type` will not be overwritten by **`extra_response_headers`**.

The **`SimpleHTTPRequestHandler`** class defines the following methods:

do_HEAD()

This method serves the `'HEAD'` request type: it sends the headers it would send for the equivalent `GET` request. See the **`do_GET()`** method for a more complete explanation of the possible headers.

do_GET()

The request is mapped to a local file by interpreting the request as a path relative to the current working directory.

If the request was mapped to a directory, the directory is checked for an index page as specified by **`index_pages`**. If found, the file's contents are returned; otherwise a directory listing is generated by calling the **`list_directory()`** method. This method uses **`os.listdir()`** to scan the directory, and returns a `404` error response if the **`listdir()`** fails.

If the request was mapped to a file, it is opened. Any **`OSError`** exception in opening the requested file is mapped to a `404`, `'File`

`not found'` error. If there was an `'If-Modified-Since'` header in the request, and the file was not modified after this time, a `304`, `'Not Modified'` response is sent. Otherwise, the content type is guessed by calling the **`guess_type()`** method, which in turn uses the *extensions_map* variable, and the file contents are returned.

A `'Content-type:'` header with the guessed content type is output, followed by a `'Content-Length:'` header with the file's size and a `'Last-Modified:'` header with the file's modification time.

The instance attribute **`extra_response_headers`** is a sequence of `(name, value)` pairs containing user-defined extra response headers.

Then follows a blank line signifying the end of the headers, and then the contents of the file are output.

For example usage, see the implementation of the `test` function in Lib/http/server.py [https://github.com/python/cpython/tree/3.15/Lib/http/server.py].

Alterado na versão 3.7: Support of the `'If-Modified-Since'` header.

list_directory(*path*)

Helper to list the contents of *path* when no index page is present.

This returns either a file-like object (which must be closed by the caller) or `None` to indicate an error, in which case the caller has nothing further to do. In either case, the headers are sent.

guess_type(*path*)

Guess the type of the file at the given *path*.

This returns a string of the form `type/subtype`, usable for a MIME Content-type header.

The default implementation looks the file's extension up in **`extensions_map`**, falling back to **`mimetypes.guess_file_type()`**

and then to `default_content_type`.

Alterado na versão 3.13: Add `mimetypes.guess_file_type()` as a fallback.

The `SimpleHTTPRequestHandler` class can be used to create a very basic webserver serving files relative to the current directory as follows:

```
import http.server
import socketserver

PORT = 8000

Handler = http.server.SimpleHTTPRequestHandler

with socketserver.TCPServer(("", PORT), Handler) as httpd:
    print("serving at port", PORT)
    httpd.serve_forever()
```

`SimpleHTTPRequestHandler` can also be subclassed to enhance behavior, such as using different index file names by overriding the class attribute `index_pages`.

Interface de linha de comando

`http.server` can also be invoked directly using the `-m` switch of the interpreter. The following example illustrates how to serve files relative to the current directory:

```
python -m http.server [OPTIONS] [port]
```

As seguintes opções são aceitas:

port

The server listens to port 8000 by default. The default can be overridden by passing the desired port number as an argument:

```
python -m http.server 9000
```

-b, --bind <address>

Specifies a specific address to which it should bind. Both IPv4 and IPv6 addresses are supported. By default, the server binds itself to all interfaces. For example, the following command causes the server to bind to localhost only:

```
python -m http.server --bind 127.0.0.1
```

Adicionado na versão 3.4.

Alterado na versão 3.8: Support IPv6 in the `--bind` option.

-d, --directory <dir>

Specifies a directory to which it should serve the files. By default, the server uses the current directory. For example, the following command uses a specific directory:

```
python -m http.server --directory /tmp/
```

Adicionado na versão 3.7.

-p, --protocol <version>

Specifies the HTTP version to which the server is conformant. By default, the server is conformant to HTTP/1.0. For example, the following command runs an HTTP/1.1 conformant server:

```
python -m http.server --protocol HTTP/1.1
```

Adicionado na versão 3.11.

--content-type <content_type>

Specifies the default Content-Type HTTP header used when the MIME type cannot be guessed from the URL's file extension. By default, the server uses `'application/octet-stream'`:

```
python -m http.server --content-type text/html
```

Adicionado na versão 3.15.

--tls-cert

Specifies a TLS certificate chain for HTTPS connections:

```
python -m http.server --tls-cert fullchain.pem
```

Adicionado na versão 3.14.

--tls-key

Specifies a private key file for HTTPS connections.

This option requires `--tls-cert` to be specified.

Adicionado na versão 3.14.

--tls-password-file

Specifies the password file for password-protected private keys:

```
python -m http.server \
       --tls-cert cert.pem \
       --tls-key key.pem \
       --tls-password-file password.txt
```

This option requires `--tls-cert` to be specified.

Adicionado na versão 3.14.

-H, --header <header> <value>

Specify an additional HTTP Response Header to send on successful HTTP 200 responses. Can be used multiple times to send additional custom response headers. Headers that are sent automatically by the server (for instance Content-Type) will not be overwritten by the server.

Adicionado na versão 3.15.

Considerações de segurança

`SimpleHTTPRequestHandler` will follow symbolic links when handling requests which makes it possible for files outside of the specified directory

to be served.

Methods `BaseHTTPRequestHandler.send_header()` and `BaseHTTPRequestHandler.send_response_only()` assume sanitized input and do not perform input validation such as checking for the presence of CRLF sequences. Untrusted input may result in HTTP header injection attacks.

Earlier versions of Python did not scrub control characters from the log messages emitted to stderr from `python -m http.server` or the default `BaseHTTPRequestHandler` `.log_message` implementation. This could allow remote clients connecting to your server to send nefarious control codes to your terminal.

Alterado na versão 3.12: Control characters are scrubbed in stderr logs.

`http.cookies` — HTTP state management

Código-fonte: Lib/http/cookies.py
[https://github.com/python/cpython/tree/3.15/Lib/http/cookies.py]

The **`http.cookies`** module defines classes for abstracting the concept of cookies, an HTTP state management mechanism. It supports both simple string-only cookies, and provides an abstraction for having any serializable data-type as cookie value.

The module formerly strictly applied the parsing rules described in the **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] and **RFC 2068** [https://datatracker.ietf.org/doc/html/rfc2068.html] specifications. It has since been discovered that MSIE 3.0x didn't follow the character rules outlined in those specs; many current-day browsers and servers have also relaxed parsing rules when it comes to cookie handling. As a result, this module now uses parsing rules that are a bit less strict than they once were.

The character set, **`string.ascii_letters`**, **`string.digits`** and `!#$%&'*+-.^_`|~:` denote the set of valid characters allowed by this module in a cookie name (as **`key`**).

Alterado na versão 3.3: Allowed ':' as a valid cookie name character.

> **Nota**
>
> On encountering an invalid cookie, **`CookieError`** is raised, so if your cookie data comes from a browser you should always prepare for invalid data and catch **`CookieError`** on parsing.

exception http.cookies.CookieError

Exception failing because of **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] invalidity: incorrect attributes, incorrect *Set-Cookie* header, etc.

class http.cookies.BaseCookie([*input*])

This class is a dictionary-like object whose keys are strings and whose values are `Morsel` instances. Note that upon setting a key to a value, the value is first converted to a `Morsel` containing the key and the value.

If *input* is given, it is passed to the `load()` method.

class http.cookies.SimpleCookie([*input*])

This class derives from `BaseCookie` and overrides `value_decode()` and `value_encode()`. `SimpleCookie` supports strings as cookie values. When setting the value, `SimpleCookie` calls the builtin `str()` to convert the value to a string. Values received from HTTP are kept as strings.

Ver também

Módulo `http.cookiejar`

HTTP cookie handling for web *clients*. The `http.cookiejar` and `http.cookies` modules do not depend on each other.

RFC 2109 [https://datatracker.ietf.org/doc/html/rfc2109.html] - HTTP State Management Mechanism

This is the state management specification implemented by this module.

Objetos Cookie

BaseCookie.value_decode(*val*)

Return a tuple `(real_value, coded_value)` from a string representation. `real_value` can be any type. This method does no decoding in `BaseCookie` — it exists so it can be overridden.

BaseCookie.value_encode(*val*)

Return a tuple `(real_value, coded_value)`. *val* can be any type, but `coded_value` will always be converted to a string. This method does no encoding in **`BaseCookie`** — it exists so it can be overridden.

In general, it should be the case that **`value_encode()`** and **`value_decode()`** are inverses on the range of *value_decode*.

BaseCookie.output(*attrs=None*, *header='Set-Cookie:'*, *sep='\r\n'*)

Return a string representation suitable to be sent as HTTP headers. *attrs* and *header* are sent to each **`Morsel`**'s **`output()`** method. *sep* is used to join the headers together, and is by default the combination `'\r\n'` (CRLF).

BaseCookie.js_output(*attrs=None*)

Return an embeddable JavaScript snippet, which, if run on a browser which supports JavaScript, will act the same as if the HTTP headers was sent.

The meaning for *attrs* is the same as in **`output()`**.

Descontinuado desde a versão 3.15, será removido na versão 3.19: This method generates a JavaScript snippet to set cookies in the browser, which is no longer considered a standard or recommended approach. Use **`output()`** instead to generate HTTP headers.

BaseCookie.load(*rawdata*)

If *rawdata* is a string, parse it as an `HTTP_COOKIE` and add the values found there as **`Morsel`**s. If it is a dictionary, it is equivalent to:

```
for k, v in rawdata.items():
    cookie[k] = v
```

Objetos Morsel

class http.cookies.Morsel

Abstract a key/value pair, which has some **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] attributes.

Morsels are dictionary-like objects, whose set of keys is constant — the valid **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] attributes, which are:

expires
path
comment
domain
max-age
secure
version
httponly
samesite
partitioned

The attribute `httponly` specifies that the cookie is only transferred in HTTP requests, and is not accessible through JavaScript. This is intended to mitigate some forms of cross-site scripting.

The attribute `samesite` controls when the browser sends the cookie with cross-site requests. This helps to mitigate CSRF attacks. Valid values are “Strict” (only sent with same-site requests), “Lax” (sent with same-site requests and top-level navigations), and “None” (sent with same-site and cross-site requests). When using “None”, the “secure” attribute must also be set, as required by modern browsers.

The attribute `partitioned` indicates to user agents that these cross-site cookies *should* only be available in the same top-level context that the cookie was first set in. For this to be accepted by the user agent, you **must** also set `Secure`.

In addition, it is recommended to use the `__Host` prefix when setting partitioned cookies to make them bound to the hostname and not the registrable domain. Read CHIPS (Cookies Having Independent

Partitioned State) [https://github.com/privacycg/CHIPS/blob/main/README.md] for full details and examples.

The keys are case-insensitive and their default value is `''`.

Alterado na versão 3.5: **`__eq__()`** now takes **`key`** and **`value`** into account.

Alterado na versão 3.7: Attributes **`key`**, **`value`** and **`coded_value`** are read-only. Use **`set()`** for setting them.

Alterado na versão 3.8: Added support for the **`samesite`** attribute.

Alterado na versão 3.14: Added support for the **`partitioned`** attribute.

Morsel.value

O valor do cookie.

Morsel.coded_value

The encoded value of the cookie — this is what should be sent.

Morsel.key

O nome do cookie.

Morsel.set(*key*, *value*, *coded_value*)

Set the *key*, *value* and *coded_value* attributes.

Morsel.isReservedKey(*K*)

Whether *K* is a member of the set of keys of a **`Morsel`**.

Morsel.output(*attrs=None*, *header='Set-Cookie:'*)

Return a string representation of the Morsel, suitable to be sent as an HTTP header. By default, all the attributes are included, unless *attrs* is given, in which case it should be a list of attributes to use. *header* is by default `"Set-Cookie:"`.

Morsel.js_output(*attrs=None*)

Return an embeddable JavaScript snippet, which, if run on a browser which supports JavaScript, will act the same as if the HTTP header was sent.

The meaning for *attrs* is the same as in `output()`.

Descontinuado desde a versão 3.15, será removido na versão 3.19: This method generates a JavaScript snippet to set cookies in the browser, which is no longer considered a standard or recommended approach. Use `output()` instead to generate HTTP headers.

Morsel.OutputString(*attrs=None*)

Return a string representing the Morsel, without any surrounding HTTP or JavaScript.

The meaning for *attrs* is the same as in `output()`.

Morsel.update(*values*)

Update the values in the Morsel dictionary with the values in the dictionary *values*. Raise an error if any of the keys in the *values* dict is not a valid **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] attribute.

Alterado na versão 3.5: an error is raised for invalid keys.

Morsel.copy(*value*)

Return a shallow copy of the Morsel object.

Alterado na versão 3.5: return a Morsel object instead of a dict.

Morsel.setdefault(*key*, *value=None*)

Raise an error if key is not a valid **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] attribute, otherwise behave the same as `dict.setdefault()`.

Exemplo

The following example demonstrates how to use the **`http.cookies`** module.

```
>>> from http import cookies
>>> C = cookies.SimpleCookie()
>>> C["fig"] = "newton"
>>> C["sugar"] = "wafer"
>>> print(C) # generate HTTP headers
Set-Cookie: fig=newton
Set-Cookie: sugar=wafer
>>> print(C.output()) # same thing
Set-Cookie: fig=newton
Set-Cookie: sugar=wafer
>>> C = cookies.SimpleCookie()
>>> C["rocky"] = "road"
>>> C["rocky"]["path"] = "/cookie"
>>> print(C.output(header="Cookie:"))
Cookie: rocky=road; Path=/cookie
>>> print(C.output(attrs=[], header="Cookie:"))
Cookie: rocky=road
>>> C = cookies.SimpleCookie()
>>> C.load("chips=ahoy; vienna=finger") # load from a string (HT
>>> print(C)
Set-Cookie: chips=ahoy
Set-Cookie: vienna=finger
>>> C = cookies.SimpleCookie()
>>> C.load('keebler="E=everybody; L=\\"Loves\\"; fudge=;";')
>>> print(C)
Set-Cookie: keebler="E=everybody; L=\"Loves\"; fudge=;"
>>> C = cookies.SimpleCookie()
>>> C["oreo"] = "doublestuff"
>>> C["oreo"]["path"] = "/"
>>> print(C)
Set-Cookie: oreo=doublestuff; Path=/
>>> C = cookies.SimpleCookie()
>>> C["twix"] = "none for you"
>>> C["twix"].value
'none for you'
>>> C = cookies.SimpleCookie()
>>> C["number"] = 7 # equivalent to C["number"] = str(7)
>>> C["string"] = "seven"
>>> C["number"].value
'7'
>>> C["string"].value
'seven'
>>> print(C)
```

```
Set-Cookie: number=7
Set-Cookie: string=seven
```

`http.cookiejar` — Cookie handling for HTTP clients

Código-fonte: Lib/http/cookiejar.py
[https://github.com/python/cpython/tree/3.15/Lib/http/cookiejar.py]

The **`http.cookiejar`** module defines classes for automatic handling of HTTP cookies. It is useful for accessing websites that require small pieces of data – *cookies* – to be set on the client machine by an HTTP response from a web server, and then returned to the server in later HTTP requests.

Both the regular Netscape cookie protocol and the protocol defined by **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] are handled. RFC 2965 handling is switched off by default. **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] cookies are parsed as Netscape cookies and subsequently treated either as Netscape or RFC 2965 cookies according to the 'policy' in effect. Note that the great majority of cookies on the internet are Netscape cookies. **`http.cookiejar`** attempts to follow the de-facto Netscape cookie protocol (which differs substantially from that set out in the original Netscape specification), including taking note of the `max-age` and `port` cookie-attributes introduced with RFC 2965.

Nota

The various named parameters found in *Set-Cookie* and *Set-Cookie2* headers (for example, `domain` and `expires`) are conventionally referred to as *attributes*. To distinguish them from Python attributes, the documentation for this module uses the term *cookie-attribute* instead.

The module defines the following exception:

exception http.cookiejar.LoadError

Instances of `FileCookieJar` raise this exception on failure to load cookies from a file. `LoadError` is a subclass of `OSError`.

Alterado na versão 3.3: `LoadError` used to be a subtype of `IOError`, which is now an alias of `OSError`.

As seguintes classes são disponibilizadas:

class http.cookiejar.CookieJar(*policy=None*)

policy is an object implementing the `CookiePolicy` interface.

The `CookieJar` class stores HTTP cookies. It extracts cookies from HTTP requests, and returns them in HTTP responses. `CookieJar` instances automatically expire contained cookies when necessary. Subclasses are also responsible for storing and retrieving cookies from a file or database.

class http.cookiejar.FileCookieJar(*filename=None*, *delayload=None*, *policy=None*)

policy is an object implementing the `CookiePolicy` interface. For the other arguments, see the documentation for the corresponding attributes.

A `CookieJar` which can load cookies from, and perhaps save cookies to, a file on disk. Cookies are **NOT** loaded from the named file until either the `load()` or `revert()` method is called. Subclasses of this class are documented in section FileCookieJar subclasses and co-operation with web browsers.

This should not be initialized directly – use its subclasses below instead.

Alterado na versão 3.8: The filename parameter supports a path-like object.

class http.cookiejar.CookiePolicy

This class is responsible for deciding whether each cookie should be accepted from / returned to the server.

class http.cookiejar.DefaultCookiePolicy(*blocked_domains=None*, *allowed_domains=None*, *netscape=True*, *rfc2965=False*, *rfc2109_as_netscape=None*, *hide_cookie2=False*, *strict_domain=False*, *strict_rfc2965_unverifiable=True*, *strict_ns_unverifiable=False*, *strict_ns_domain=DefaultCookiePolicy.DomainLiberal*, *strict_ns_set_initial_dollar=False*, *strict_ns_set_path=False*, *secure_protocols=('https', 'wss')*)

Constructor arguments should be passed as keyword arguments only. *blocked_domains* is a sequence of domain names that we never accept cookies from, nor return cookies to. *allowed_domains* if not `None`, this is a sequence of the only domains for which we accept and return cookies. *secure_protocols* is a sequence of protocols for which secure cookies can be added to. By default *https* and *wss* (secure websocket) are considered secure protocols. For all other arguments, see the documentation for **`CookiePolicy`** and **`DefaultCookiePolicy`** objects.

`DefaultCookiePolicy` implements the standard accept / reject rules for Netscape and **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] cookies. By default, **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] cookies (that is, cookies received in a *Set-Cookie* header with a version cookie-attribute of 1) are treated according to the RFC 2965 rules. However, if RFC 2965 handling is turned off or **`rfc2109_as_netscape`** is `True`, RFC 2109 cookies are 'downgraded' by the **`CookieJar`** instance to Netscape cookies, by setting the **`version`** attribute of the **`Cookie`** instance to 0. **`DefaultCookiePolicy`** also provides some parameters to allow some fine-tuning of policy.

class http.cookiejar.Cookie

This class represents Netscape, **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] and **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] cookies. It is not expected that users of **`http.cookiejar`** construct their own **`Cookie`** instances. Instead, if necessary, call **`make_cookies()`** on a **`CookieJar`** instance.

Ver também

Módulo `urllib.request`

URL opening with automatic cookie handling.

Module `http.cookies`

HTTP cookie classes, principally useful for server-side code. The **`http.cookiejar`** and `http.cookies` modules do not depend on each other.

https://curl.se/rfc/cookie_spec.html

The specification of the original Netscape cookie protocol. Though this is still the dominant protocol, the 'Netscape cookie protocol' implemented by all the major browsers (and **`http.cookiejar`**) only bears a passing resemblance to the one sketched out in `cookie_spec.html`.

RFC 2109 [https://datatracker.ietf.org/doc/html/rfc2109.html] - HTTP State Management Mechanism

Obsoleted by **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html]. Uses *Set-Cookie* with version=1.

RFC 2965 [https://datatracker.ietf.org/doc/html/rfc2965.html] - HTTP State Management Mechanism

The Netscape protocol with the bugs fixed. Uses *Set-Cookie2* in place of *Set-Cookie*. Not widely used.

https://kristol.org/cookie/errata.html

Unfinished errata to **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html].

RFC 2964 [https://datatracker.ietf.org/doc/html/rfc2964.html] - Use of HTTP State Management

CookieJar and FileCookieJar objects

`CookieJar` objects support the iterator protocol for iterating over contained `Cookie` objects.

`CookieJar` has the following methods:

CookieJar.add_cookie_header(*request*)

Add correct *Cookie* header to *request*.

If policy allows (that is, the `rfc2965` and `hide_cookie2` attributes of the `CookieJar`'s `CookiePolicy` instance are true and false respectively), the *Cookie2* header is also added when appropriate.

The *request* object (usually a `urllib.request.Request` instance) must support the methods `get_full_url()`, `has_header()`, `get_header()`, `header_items()`, `add_unredirected_header()` and the attributes `host`, `type`, `unverifiable` and `origin_req_host` as documented by `urllib.request`.

Alterado na versão 3.3: request object needs `origin_req_host` attribute. Dependency on a deprecated method **`get_origin_req_host()`** has been removed.

CookieJar.extract_cookies(*response*, *request*)

Extract cookies from HTTP *response* and store them in the `CookieJar`, where allowed by policy.

The `CookieJar` will look for allowable *Set-Cookie* and *Set-Cookie2* headers in the *response* argument, and store cookies as appropriate (subject to the `CookiePolicy.set_ok()` method's approval).

The *response* object (usually the result of a call to `urllib.request.urlopen()`, or similar) should support an `info()` method, which returns an `email.message.Message` instance.

The *request* object (usually a `urllib.request.Request` instance) must support the method `get_full_url()` and the attributes `host`, `unverifiable` and `origin_req_host`, as documented by `urllib.request`. The request is used to set default values for cookie-attributes as well as for checking that the cookie is allowed to be set.

Alterado na versão 3.3: request object needs `origin_req_host` attribute. Dependency on a deprecated method **`get_origin_req_host()`** has been removed.

CookieJar.set_policy(*policy*)

Set the `CookiePolicy` instance to be used.

CookieJar.make_cookies(*response*, *request*)

Return sequence of `Cookie` objects extracted from *response* object.

See the documentation for `extract_cookies()` for the interfaces required of the *response* and *request* arguments.

CookieJar.set_cookie_if_ok(*cookie*, *request*)

Set a `Cookie` if policy says it's OK to do so.

CookieJar.set_cookie(*cookie*)

Set a `Cookie`, without checking with policy to see whether or not it should be set.

CookieJar.clear([*domain*[, *path*[, *name*]]])

Clear some cookies.

If invoked without arguments, clear all cookies. If given a single argument, only cookies belonging to that *domain* will be removed. If given two arguments, cookies belonging to the specified *domain* and URL *path* are removed. If given three arguments, then the cookie with the specified *domain*, *path* and *name* is removed.

Raises `KeyError` if no matching cookie exists.

CookieJar.clear_session_cookies()

Descarta os cookies de sessão.

Discards all contained cookies that have a true `discard` attribute (usually because they had either no `max-age` or `expires` cookie-attribute, or an explicit `discard` cookie-attribute). For interactive browsers, the end of a session usually corresponds to closing the browser window.

Note that the `save()` method won't save session cookies anyway, unless you ask otherwise by passing a true *ignore_discard* argument.

`FileCookieJar` implements the following additional methods:

FileCookieJar.save(*filename=None*, *ignore_discard=False*, *ignore_expires=False*)

Save cookies to a file.

This base class raises `NotImplementedError`. Subclasses may leave this method unimplemented.

filename is the name of file in which to save cookies. If *filename* is not specified, `self.filename` is used (whose default is the value passed to the constructor, if any); if `self.filename` is `None`, `ValueError` is raised.

ignore_discard: save even cookies set to be discarded. *ignore_expires*: save even cookies that have expired

The file is overwritten if it already exists, thus wiping all the cookies it contains. Saved cookies can be restored later using the `load()` or `revert()` methods.

FileCookieJar.load(*filename=None*, *ignore_discard=False*, *ignore_expires=False*)

Load cookies from a file.

Old cookies are kept unless overwritten by newly loaded ones.

Arguments are as for `save()`.

The named file must be in the format understood by the class, or `LoadError` will be raised. Also, `OSError` may be raised, for example if the file does not exist.

Alterado na versão 3.3: `IOError` costumava ser levantado, agora ele é um codinome para `OSError`.

FileCookieJar.revert(*filename=None*, *ignore_discard=False*, *ignore_expires=False*)

Clear all cookies and reload cookies from a saved file.

`revert()` can raise the same exceptions as **`load()`**. If there is a failure, the object's state will not be altered.

`FileCookieJar` instances have the following public attributes:

FileCookieJar.filename

Filename of default file in which to keep cookies. This attribute may be assigned to.

FileCookieJar.delayload

If true, load cookies lazily from disk. This attribute should not be assigned to. This is only a hint, since this only affects performance, not behaviour (unless the cookies on disk are changing). A **`CookieJar`** object may ignore it. None of the **`FileCookieJar`** classes included in the standard library lazily loads cookies.

FileCookieJar subclasses and co-operation with web browsers

The following **`CookieJar`** subclasses are provided for reading and writing.

class http.cookiejar.MozillaCookieJar(*filename=None*, *delayload=None*, *policy=None*)

A **`FileCookieJar`** that can load from and save cookies to disk in the Mozilla `cookies.txt` file format (which is also used by curl and the Lynx and Netscape browsers).

Nota

This loses information about **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] cookies, and also about newer or non-standard cookie-attributes such as `port`.

Aviso

Back up your cookies before saving if you have cookies whose loss / corruption would be inconvenient (there are some subtleties which may lead to slight changes in the file over a load / save round-trip).

Also note that cookies saved while Mozilla is running will get clobbered by Mozilla.

class http.cookiejar.LWPCookieJar(*filename=None*, *delayload=None*, *policy=None*)

A **`FileCookieJar`** that can load from and save cookies to disk in format compatible with the libwww-perl library's `Set-Cookie3` file format. This is convenient if you want to store cookies in a human-readable file.

Alterado na versão 3.8: The filename parameter supports a path-like object.

CookiePolicy objects

Objects implementing the **`CookiePolicy`** interface have the following methods:

CookiePolicy.set_ok(*cookie*, *request*)

Return boolean value indicating whether cookie should be accepted from server.

cookie is a **`Cookie`** instance. *request* is an object implementing the interface defined by the documentation for **`CookieJar.extract_cookies()`**.

CookiePolicy.return_ok(*cookie*, *request*)

Return boolean value indicating whether cookie should be returned to server.

cookie is a **Cookie** instance. *request* is an object implementing the interface defined by the documentation for **CookieJar.add_cookie_header()**.

CookiePolicy.domain_return_ok(*domain*, *request*)

Return `False` if cookies should not be returned, given cookie domain.

This method is an optimization. It removes the need for checking every cookie with a particular domain (which might involve reading many files). Returning true from **domain_return_ok()** and **path_return_ok()** leaves all the work to **return_ok()**.

If **domain_return_ok()** returns true for the cookie domain, **path_return_ok()** is called for the cookie path. Otherwise, **path_return_ok()** and **return_ok()** are never called for that cookie domain. If **path_return_ok()** returns true, **return_ok()** is called with the **Cookie** object itself for a full check. Otherwise, **return_ok()** is never called for that cookie path.

Note that **domain_return_ok()** is called for every *cookie* domain, not just for the *request* domain. For example, the function might be called with both `".example.com"` and `"www.example.com"` if the request domain is `"www.example.com"`. The same goes for **path_return_ok()**.

The *request* argument is as documented for **return_ok()**.

CookiePolicy.path_return_ok(*path*, *request*)

Return `False` if cookies should not be returned, given cookie path.

See the documentation for **domain_return_ok()**.

In addition to implementing the methods above, implementations of the **CookiePolicy** interface must also supply the following attributes, indicating which protocols should be used, and how. All of these attributes may be assigned to.

CookiePolicy.netscape

Implement Netscape protocol.

CookiePolicy.rfc2965

Implement **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] protocol.

CookiePolicy.hide_cookie2

Don't add *Cookie2* header to requests (the presence of this header indicates to the server that we understand **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] cookies).

The most useful way to define a `CookiePolicy` class is by subclassing from `DefaultCookiePolicy` and overriding some or all of the methods above. `CookiePolicy` itself may be used as a 'null policy' to allow setting and receiving any and all cookies (this is unlikely to be useful).

DefaultCookiePolicy objects

Implements the standard rules for accepting and returning cookies.

Both **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] and Netscape cookies are covered. RFC 2965 handling is switched off by default.

The easiest way to provide your own policy is to override this class and call its methods in your overridden implementations before adding your own additional checks:

```
import http.cookiejar
class MyCookiePolicy(http.cookiejar.DefaultCookiePolicy):
    def set_ok(self, cookie, request):
        if not http.cookiejar.DefaultCookiePolicy.set_ok(self, c
            return False
        if i_dont_want_to_store_this_cookie(cookie):
            return False
        return True
```

In addition to the features required to implement the `CookiePolicy` interface, this class allows you to block and allow domains from setting and receiving cookies. There are also some strictness switches that allow you to

tighten up the rather loose Netscape protocol rules a little bit (at the cost of blocking some benign cookies).

A domain blocklist and allowlist is provided (both off by default). Only domains not in the blocklist and present in the allowlist (if the allowlist is active) participate in cookie setting and returning. Use the *blocked_domains* constructor argument, and `blocked_domains()` and `set_blocked_domains()` methods (and the corresponding argument and methods for *allowed_domains*). If you set an allowlist, you can turn it off again by setting it to `None`.

Domains in block or allow lists that do not start with a dot must equal the cookie domain to be matched. For example, `"example.com"` matches a blocklist entry of `"example.com"`, but `"www.example.com"` does not. Domains that do start with a dot are matched by more specific domains too. For example, both `"www.example.com"` and `"www.coyote.example.com"` match `".example.com"` (but `"example.com"` itself does not). IP addresses are an exception, and must match exactly. For example, if blocked_domains contains `"192.168.1.2"` and `".168.1.2"`, 192.168.1.2 is blocked, but 193.168.1.2 is not.

`DefaultCookiePolicy` implements the following additional methods:

DefaultCookiePolicy.blocked_domains()

Return the sequence of blocked domains (as a tuple).

DefaultCookiePolicy.set_blocked_domains(*blocked_domains*)

Set the sequence of blocked domains.

DefaultCookiePolicy.is_blocked(*domain*)

Return `True` if *domain* is on the blocklist for setting or receiving cookies.

DefaultCookiePolicy.allowed_domains()

Return `None`, or the sequence of allowed domains (as a tuple).

DefaultCookiePolicy.set_allowed_domains(*allowed_domains*)

Set the sequence of allowed domains, or `None`.

DefaultCookiePolicy.is_not_allowed(*domain*)

Return `True` if *domain* is not on the allowlist for setting or receiving cookies.

`DefaultCookiePolicy` instances have the following attributes, which are all initialised from the constructor arguments of the same name, and which may all be assigned to.

DefaultCookiePolicy.rfc2109_as_netscape

If true, request that the **`CookieJar`** instance downgrade **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] cookies (that is, cookies received in a *Set-Cookie* header with a version cookie-attribute of 1) to Netscape cookies by setting the version attribute of the **`Cookie`** instance to 0. The default value is `None`, in which case RFC 2109 cookies are downgraded if and only if **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] handling is turned off. Therefore, RFC 2109 cookies are downgraded by default.

General strictness switches:

DefaultCookiePolicy.strict_domain

Don't allow sites to set two-component domains with country-code top-level domains like `.co.uk`, `.gov.uk`, `.co.nz`.etc. This is far from perfect and isn't guaranteed to work!

RFC 2965 [https://datatracker.ietf.org/doc/html/rfc2965.html] protocol strictness switches:

DefaultCookiePolicy.strict_rfc2965_unverifiable

Follow **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] rules on unverifiable transactions (usually, an unverifiable transaction is one resulting from a redirect or a request for an image hosted on another site). If this is false, cookies are *never* blocked on the basis of verifiability

Netscape protocol strictness switches:

DefaultCookiePolicy.strict_ns_unverifiable

Apply **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] rules on unverifiable transactions even to Netscape cookies.

DefaultCookiePolicy.strict_ns_domain

Flags indicating how strict to be with domain-matching rules for Netscape cookies. See below for acceptable values.

DefaultCookiePolicy.strict_ns_set_initial_dollar

Ignore cookies in Set-Cookie: headers that have names starting with `'$'`.

DefaultCookiePolicy.strict_ns_set_path

Don't allow setting cookies whose path doesn't path-match request URI.

`strict_ns_domain` is a collection of flags. Its value is constructed by or-ing together (for example, `DomainStrictNoDots|DomainStrictNonDomain` means both flags are set).

DefaultCookiePolicy.DomainStrictNoDots

When setting cookies, the 'host prefix' must not contain a dot (for example, `www.foo.bar.com` can't set a cookie for `.bar.com`, because `www.foo` contains a dot).

DefaultCookiePolicy.DomainStrictNonDomain

Cookies that did not explicitly specify a `domain` cookie-attribute can only be returned to a domain equal to the domain that set the cookie (for example, `spam.example.com` won't be returned cookies from `example.com` that had no `domain` cookie-attribute).

DefaultCookiePolicy.DomainRFC2965Match

When setting cookies, require a full **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] domain-match.

The following attributes are provided for convenience, and are the most useful combinations of the above flags:

DefaultCookiePolicy.DomainLiberal

Equivalent to 0 (that is, all of the above Netscape domain strictness flags switched off).

DefaultCookiePolicy.DomainStrict

Equivalente a `DomainStrictNoDots|DomainStrictNonDomain`.

Cookie objects

`Cookie` instances have Python attributes roughly corresponding to the standard cookie-attributes specified in the various cookie standards. The correspondence is not one-to-one, because there are complicated rules for assigning default values, because the `max-age` and `expires` cookie-attributes contain equivalent information, and because **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] cookies may be 'downgraded' by **`http.cookiejar`** from version 1 to version 0 (Netscape) cookies.

Assignment to these attributes should not be necessary other than in rare circumstances in a **`CookiePolicy`** method. The class does not enforce internal consistency, so you should know what you're doing if you do that.

Cookie.version

Integer or `None`. Netscape cookies have **`version`** 0. **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] and **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] cookies have a `version` cookie-attribute of 1. However, note that **`http.cookiejar`** may 'downgrade' RFC 2109 cookies to Netscape cookies, in which case **`version`** is 0.

Cookie.name

Cookie name (a string).

Cookie.value

Cookie value (a string), or `None`.

Cookie.port

String representing a port or a set of ports (for example, '80', or '80,8080'), or `None`.

Cookie.domain

Cookie domain (a string).

Cookie.path

Cookie path (a string, for example, `'/acme/rocket_launchers'`).

Cookie.secure

`True` if cookie should only be returned over a secure connection.

Cookie.expires

Integer expiry date in seconds since epoch, or `None`. See also the **`is_expired()`** method.

Cookie.discard

`True` if this is a session cookie.

Cookie.comment

String comment from the server explaining the function of this cookie, or `None`.

Cookie.comment_url

URL linking to a comment from the server explaining the function of this cookie, or `None`.

Cookie.rfc2109

`True` if this cookie was received as an **RFC 2109** [https://datatracker.ietf.org/doc/html/rfc2109.html] cookie (that is, the cookie arrived in a *Set-Cookie* header, and the value of the Version cookie-attribute in that header was 1). This attribute is provided because **`http.cookiejar`** may 'downgrade' RFC 2109 cookies to Netscape cookies, in which case **`version`** is 0.

Cookie.port_specified

`True` if a port or set of ports was explicitly specified by the server (in the *Set-Cookie* / *Set-Cookie2* header).

Cookie.domain_specified

`True` if a domain was explicitly specified by the server.

Cookie.domain_initial_dot

`True` if the domain explicitly specified by the server began with a dot (`'.'`).

Cookies may have additional non-standard cookie-attributes. These may be accessed using the following methods:

Cookie.has_nonstandard_attr(*name*)

Return `True` if cookie has the named cookie-attribute.

Cookie.get_nonstandard_attr(*name*, *default=None*)

If cookie has the named cookie-attribute, return its value. Otherwise, return *default*.

Cookie.set_nonstandard_attr(*name*, *value*)

Set the value of the named cookie-attribute.

The **`Cookie`** class also defines the following method:

Cookie.is_expired(*now=None*)

`True` if cookie has passed the time at which the server requested it should expire. If *now* is given (in seconds since the epoch), return whether the cookie has expired at the specified time.

Exemplos

The first example shows the most common usage of **`http.cookiejar`**:

```
import http.cookiejar, urllib.request
cj = http.cookiejar.CookieJar()
opener = urllib.request.build_opener(urllib.request.HTTPCookiePr
r = opener.open("http://example.com/")
```

This example illustrates how to open a URL using your Netscape, Mozilla, or Lynx cookies (assumes Unix/Netscape convention for location of the cookies file):

```
import os, http.cookiejar, urllib.request
cj = http.cookiejar.MozillaCookieJar()
cj.load(os.path.join(os.path.expanduser("~"), ".netscape", "cook
opener = urllib.request.build_opener(urllib.request.HTTPCookiePr
r = opener.open("http://example.com/")
```

The next example illustrates the use of **`DefaultCookiePolicy`**. Turn on **RFC 2965** [https://datatracker.ietf.org/doc/html/rfc2965.html] cookies, be more strict about domains when setting and returning Netscape cookies, and block some domains from setting cookies or having them returned:

```
import urllib.request
from http.cookiejar import CookieJar, DefaultCookiePolicy
policy = DefaultCookiePolicy(
    rfc2965=True, strict_ns_domain=Policy.DomainStrict,
    blocked_domains=["ads.net", ".ads.net"])
cj = CookieJar(policy)
opener = urllib.request.build_opener(urllib.request.HTTPCookiePr
r = opener.open("http://example.com/")
```

`xmlrpc` — Módulos de servidor e cliente XMLRPC

XML-RPC é um método de chamada de procedimento remoto que usa XML passado via HTTP como um transporte. Com ele, um cliente pode chamar métodos com parâmetros em um servidor remoto (o servidor é nomeado por um URI) e receber dados estruturados.

`xmlrpc` é um pacote que coleta módulos de servidor e de cliente que implementam o XML-RPC. Os módulos são:

- `xmlrpc.client`
- `xmlrpc.server`

`xmlrpc.client` — Cliente XML-RPC

Código-fonte: Lib/xmlrpc/client.py
[https://github.com/python/cpython/tree/3.15/Lib/xmlrpc/client.py]

XML-RPC é um método de chamada remota de métodos que usa XML usando HTTP(S) como transporte. Com ele, um cliente pode chamar métodos com parâmetros em um servidor remoto (o servidor é nomeado por um URI) e receber de volta dados estruturados. Este módulo oferece suporte à escrita de código de clientes XML-RPC; ele lida com todos os detalhes da tradução entre Python objetos e XML.

> **Aviso**
>
> The `xmlrpc.client` module is not secure against maliciously constructed data. If you need to parse untrusted or unauthenticated data, see Segurança no XML.

Alterado na versão 3.5: For HTTPS URIs, `xmlrpc.client` now performs all the necessary certificate and hostname checks by default.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

class xmlrpc.client.ServerProxy(*uri*, *transport=None*, *encoding=None*, *verbose=False*, *allow_none=False*, *use_datetime=False*, *use_builtin_types=False*, *, *headers=()*, *context=None*)

Uma instância de **`ServerProxy`** é um objeto que gerencia a comunicação com um servidor XML-RPC remoto. O primeiro argumento obrigatório é uma URI (Uniform Resource Indicator - Indicador de Recurso Uniforme) e normalmente vai ser a URL do servidor. O segundo parâmetro, opcional, é uma instância de um factory de transporte; por padrão é uma instância de **`SafeTransport`** para URLs https e uma instância de **`Transport`** caso contrário. O terceiro parâmetro, opcional, é o encoding, por padrão sendo UTF-8. O quarto argumento, opcional, é a flag de debug.

Os seguinte parâmetros coordenam o uso da instância de proxy retornada. Se *alow_none* for verdadeiro, a constante `None` do Python será traduzida para XML; o comportamento padrão é que `None` levante uma **`TypeError`**, Isso é uma extensão comum da especificação do XML-RPC, mas não é suportado por todos os clientes e servidores, veja http://ontosys.com/xml-rpc/extensions.php [https://web.archive.org/web/20130120074804/http://ontosys.com/xml-rpc/extensions.php] para uma descrição. A flag *use_builtin_types* pode ser usada para que valores de data e hora sejam representados como objetos **`datetime.datetime`** e dados binários representados com objetos **`bytes`**; essa flag é False por padrão. Objetos **`datetime.datetime`**, **`bytes`** e **`bytearray`** podem ser usados nas chamadas. O parâmetro *headers* é uma sequência opcional de headers a serem enviados em cada requisição, representados por uma sequência de tuplas de dois valores representando o nome do header e seu valor (como `[('Header-Name', 'value')]`). Se uma URL HTTPS é fornecida, *context* pode ser **`ssl.SSLContext`** e configura as definições SSL da conexão HTTPS subjacente. A flag *use_datetime* é obsoleta e é similar a *use_builtin_types* mas se aplica somente a valores de data e hora.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

Alterado na versão 3.8: O parâmetro *headers* foi adicionado.

Both the HTTP and HTTPS transports support the URL syntax extension for HTTP Basic Authentication: `http://user:pass@host:port/path`. The `user:pass` portion will be base64-encoded as an HTTP ‘Authorization’ header, and sent to the

remote server as part of the connection process when invoking an XML-RPC method. You only need to use this if the remote server requires a Basic Authentication user and password.

A instância retornada é um objeto proxy com métodos que podem ser usados para invocar a chamada RPC correspondendo no servidor remoto. Se o servidor remoto suportar a API de instrospecção, o proxy também pode ser usado para perguntar ao servidor remoto pelos métodos que ele suporta (descoberta de serviço) e recuperar outros meta-dados associados com o servidor.

Os tipos que são conformáveis (por exemplo, que podem ser convertidos para XML) incluem os seguintes (e exceto onde indicado, eles não são convertidos como o mesmo tipo Python):

Tipo XML-RPC	**Tipo em Python**
`boolean`	**`bool`**
`int`, `i1`, `i2`, `i4`, `i8` ou `biginteger`	**`int`** no intervalo de -2147483648 a 2147483647. Os valores recebem a tag `<int>`.
`double` ou `float`	**`float`**. Os valores recebem a tag `<double>`.
`string`	**`str`**
`array`	**`list`** ou **`tuple`** contendo elementos conformáveis. As matrizes são retornadas como **`lists`**.
`struct`	**`dict`**. Chaves devem ser strings, valores podem ser qualquer tipo conformáveis. Objetos de classes definidas pelo usuário pode ser usadas; apenas o atributo **`__dict__`** delas é transmitido.
`dateTime.iso8601`	**`DateTime`** ou **`datetime.datetime`**. O tipo retornado depende de volumes de sinalizadores *use_builtin_types* e *use_datetime*.

Tipo XML-RPC	Tipo em Python
`base64`	**`Binary`**, **`bytes`** ou **`bytearray`**. O tipo retornado depende do valor da flag *use_builtin_types*.
`nil`	A constante `None`. A passagem é permitida somente se *allow_none* for verdadeiro.
`bigdecimal`	**`decimal.Decimal`**. Somente tipo retornado.

Esta é a lista completa de tipos suportados por XML-RPC. Chamadas de método podem também levantar uma instância de **`Fault`**, usado para que o servidor XML-RPC indique erros, ou **`ProtocolError`** para indicar erros na camada HTTP/HTTPS. Tnato **`Fault`** quanto **`ProtocolError`** derivam da classe base **`Error`**. Observe que o módulo de cliente xmlrpc atualmente não converte instância de subclasses dos tipos built-in.

Ao passar strings, os caracteres especiais para XML, como `<`, `>` e `&`, serão automaticamente escapados. No entanto, é responsabilidade do chamador garantir que o string esteja livre de caracteres que não são permitidos em XML, como os caracteres de controle com valores ASCII entre 0 e 31 (exceto, é claro, tabulação, nova linha e retorno de carro); se isso não for feito, resultará em uma solicitação XML-RPC que não é um XML bem formado. Se você precisar passar bytes arbitrários via XML-RPC, use as classes **`bytes`** ou **`bytearray`** ou a classe wrapper **`Binary`** descrito abaixo.

`Server` foi mantido como um apelido para **`ServerProxy`** para compatibilidade retroativa. Código novo deve usar **`ServerProxy`**.

Alterado na versão 3.5: Argumento *context* adicionado.

Alterado na versão 3.6: Adicionado suporte para tags com prefixos (por exemplo `ex:nil`) Adicionado suporte para desconversão de tipos adicionados usados pela implementação do Apache XML-RPC para numéricos: `i1`, `i2`, `i8`, `biginteger`, `float` e `bigdecimal`. Veja https://ws.apache.org/xmlrpc/types.html para uma descrição.

Ver também

XML-RPC HOWTO **[https://tldp.org/HOWTO/XML-RPC-HOWTO/index.html]**
Uma boa descrição das operações XML-RPC e software cliente em vários idiomas. Contém praticamente tudo o que um desenvolvedor de clientes XML-RPC precisa saber.

XML-RPC Introspection **[https://xmlrpc-c.sourceforge.io/introspection.html]**
Describe a extensão do protocolo XML-RPC para instrospecção.

XML-RPC Specification **[http://xmlrpc.scripting.com/spec.html]**
A especificação oficial.

Objetos ServerProxy

A `ServerProxy` instance has a method corresponding to each remote procedure call accepted by the XML-RPC server. Calling the method performs an RPC, dispatched by both name and argument signature (e.g. the same method name can be overloaded with multiple argument signatures). The RPC finishes either by returning data in a conformant type or by raising a `Fault` or `ProtocolError` exception indicating an error.

Servers that support the XML introspection API support some common methods grouped under the reserved `system` attribute:

ServerProxy.system.listMethods()
This method returns a list of strings, one for each (non-system) method supported by the XML-RPC server.

ServerProxy.system.methodSignature(*name*)
This method takes one parameter, the name of a method implemented by the XML-RPC server. It returns an array of possible signatures for this method. A signature is an array of types. The first of these types is the return type of the method, the rest are parameters.

Because multiple signatures (ie. overloading) is permitted, this method returns a list of signatures rather than a singleton.

Signatures themselves are restricted to the top level parameters expected by a method. For instance if a method expects one array of structs as a parameter, and it returns a string, its signature is simply “string, array”. If it expects three integers and returns a string, its signature is “string, int, int, int”.

If no signature is defined for the method, a non-array value is returned. In Python this means that the type of the returned value will be something other than list.

ServerProxy.system.methodHelp(*name*)

This method takes one parameter, the name of a method implemented by the XML-RPC server. It returns a documentation string describing the use of that method. If no such string is available, an empty string is returned. The documentation string may contain HTML markup.

Alterado na versão 3.5: Instances of **`ServerProxy`** support the context manager protocol for closing the underlying transport.

A working example follows. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

def is_even(n):
    return n % 2 == 0

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(is_even, "is_even")
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

with xmlrpc.client.ServerProxy("http://localhost:8000/") as prox
```

```
    print("3 is even: %s" % str(proxy.is_even(3)))
    print("100 is even: %s" % str(proxy.is_even(100)))
```

Objetos DateTime

class xmlrpc.client.DateTime

This class may be initialized with seconds since the epoch, a time tuple, an ISO 8601 time/date string, or a **datetime.datetime** instance. It has the following methods, supported mainly for internal use by the marshalling/unmarshalling code:

decode(*string*)

Accept a string as the instance's new time value.

encode(*out*)

Write the XML-RPC encoding of this **DateTime** item to the *out* stream object.

It also supports certain of Python's built-in operators through **rich comparison** and **__repr__()** methods.

A working example follows. The server code:

```
import datetime as dt
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client

def today():
    today = dt.datetime.today()
    return xmlrpc.client.DateTime(today)

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(today, "today")
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client
import datetime as dt

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")

today = proxy.today()
# convert the ISO 8601 string to a datetime object
converted = dt.datetime.strptime(today.value, "%Y%m%dT%H:%M:%S")
print(f"Today: {converted.strftime('%d.%m.%Y, %H:%M')}")
```

Objetos Binários

class xmlrpc.client.Binary

This class may be initialized from bytes data (which may include NULs). The primary access to the content of a **`Binary`** object is provided by an attribute:

data

The binary data encapsulated by the **`Binary`** instance. The data is provided as a **`bytes`** object.

`Binary` objects have the following methods, supported mainly for internal use by the marshalling/unmarshalling code:

decode(*bytes*)

Accept a base64 **`bytes`** object and decode it as the instance's new data.

encode(*out*)

Write the XML-RPC base 64 encoding of this binary item to the *out* stream object.

The encoded data will have newlines every 76 characters as per **RFC 2045 section 6.8** [https://datatracker.ietf.org/doc/html/rfc2045.html#section-6.8], which was the de facto standard base64 specification when the XML-RPC spec was written.

It also supports certain of Python's built-in operators through **__eq__()** and **__ne__()** methods.

Example usage of the binary objects. We're going to transfer an image over XMLRPC:

```
from xmlrpc.server import SimpleXMLRPCServer
import xmlrpc.client

def python_logo():
    with open("python_logo.jpg", "rb") as handle:
        return xmlrpc.client.Binary(handle.read())

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(python_logo, 'python_logo')

server.serve_forever()
```

The client gets the image and saves it to a file:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
with open("fetched_python_logo.jpg", "wb") as handle:
    handle.write(proxy.python_logo().data)
```

Objetos Fault

class xmlrpc.client.Fault

A **Fault** object encapsulates the content of an XML-RPC fault tag. Fault objects have the following attributes:

faultCode

An int indicating the fault type.

faultString

A string containing a diagnostic message associated with the fault.

In the following example we're going to intentionally cause a **`Fault`** by returning a complex type object. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

# A marshalling error is going to occur because we're returning
# complex number
def add(x, y):
    return x+y+0j

server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_function(add, 'add')

server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
try:
    proxy.add(2, 5)
except xmlrpc.client.Fault as err:
    print("A fault occurred")
    print("Fault code: %d" % err.faultCode)
    print("Fault string: %s" % err.faultString)
```

Objetos ProtocolError

class xmlrpc.client.ProtocolError

A **`ProtocolError`** object describes a protocol error in the underlying transport layer (such as a 404 'not found' error if the server named by the URI does not exist). It has the following attributes:

url

The URI or URL that triggered the error.

errcode

O código do erro.

errmsg
The error message or diagnostic string.

headers
A dict containing the headers of the HTTP/HTTPS request that triggered the error.

In the following example we’re going to intentionally cause a **`ProtocolError`** by providing an invalid URI:

```
import xmlrpc.client

# create a ServerProxy with a URI that doesn't respond to XMLRPC
proxy = xmlrpc.client.ServerProxy("http://google.com/")

try:
    proxy.some_method()
except xmlrpc.client.ProtocolError as err:
    print("A protocol error occurred")
    print("URL: %s" % err.url)
    print("HTTP/HTTPS headers: %s" % err.headers)
    print("Error code: %d" % err.errcode)
    print("Error message: %s" % err.errmsg)
```

Objetos MultiCall

The **`MultiCall`** object provides a way to encapsulate multiple calls to a remote server into a single request [[1]].

class xmlrpc.client.MultiCall(*server*)
Create an object used to boxcar method calls. *server* is the eventual target of the call. Calls can be made to the result object, but they will immediately return `None`, and only store the call name and arguments in the **`MultiCall`** object. Calling the object itself causes all stored calls to be transmitted as a single `system.multicall` request. The result of this

call is a generator; iterating over this generator yields the individual results.

A usage example of this class follows. The server code:

```
from xmlrpc.server import SimpleXMLRPCServer

def add(x, y):
    return x + y

def subtract(x, y):
    return x - y

def multiply(x, y):
    return x * y

def divide(x, y):
    return x // y

# A simple server with simple arithmetic functions
server = SimpleXMLRPCServer(("localhost", 8000))
print("Listening on port 8000...")
server.register_multicall_functions()
server.register_function(add, 'add')
server.register_function(subtract, 'subtract')
server.register_function(multiply, 'multiply')
server.register_function(divide, 'divide')
server.serve_forever()
```

The client code for the preceding server:

```
import xmlrpc.client

proxy = xmlrpc.client.ServerProxy("http://localhost:8000/")
multicall = xmlrpc.client.MultiCall(proxy)
multicall.add(7, 3)
multicall.subtract(7, 3)
multicall.multiply(7, 3)
multicall.divide(7, 3)
result = multicall()

print("7+3=%d, 7-3=%d, 7*3=%d, 7//3=%d" % tuple(result))
```

Convenience Functions

xmlrpc.client.dumps(*params*, *methodname=None*, *methodresponse=None*, *encoding=None*, *allow_none=False*)

Convert *params* into an XML-RPC request, or into a response if *methodresponse* is true. *params* can be either a tuple of arguments or an instance of the **`Fault`** exception class. If *methodresponse* is true, only a single value can be returned, meaning that *params* must be of length 1. *encoding*, if supplied, is the encoding to use in the generated XML; the default is UTF-8. Python's **`None`** value cannot be used in standard XML-RPC; to allow using it via an extension, provide a true value for *allow_none*.

xmlrpc.client.loads(*data*, *use_datetime=False*, *use_builtin_types=False*)

Convert an XML-RPC request or response into Python objects, a `(params, methodname)`. *params* is a tuple of argument; *methodname* is a string, or `None` if no method name is present in the packet. If the XML-RPC packet represents a fault condition, this function will raise a **`Fault`** exception. The *use_builtin_types* flag can be used to cause date/time values to be presented as **`datetime.datetime`** objects and binary data to be presented as **`bytes`** objects; this flag is false by default.

The obsolete *use_datetime* flag is similar to *use_builtin_types* but it applies only to date/time values.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

Exemplo de uso do cliente

```
# simple test program (from the XML-RPC specification)
from xmlrpc.client import ServerProxy, Error

# server = ServerProxy("http://localhost:8000") # local server
with ServerProxy("http://betty.userland.com") as proxy:

    print(proxy)
```

```
    try:
        print(proxy.examples.getStateName(41))
    except Error as v:
        print("ERROR", v)
```

To access an XML-RPC server through a HTTP proxy, you need to define a custom transport. The following example shows how:

```
import http.client
import xmlrpc.client

class ProxiedTransport(xmlrpc.client.Transport):

    def set_proxy(self, host, port=None, headers=None):
        self.proxy = host, port
        self.proxy_headers = headers

    def make_connection(self, host):
        connection = http.client.HTTPConnection(*self.proxy)
        connection.set_tunnel(host, headers=self.proxy_headers)
        self._connection = host, connection
        return connection

transport = ProxiedTransport()
transport.set_proxy('proxy-server', 8080)
server = xmlrpc.client.ServerProxy('http://betty.userland.com',
print(server.examples.getStateName(41))
```

Example of Client and Server Usage

Veja SimpleXMLRPCServer example.

Notas de rodapé

[1]

This approach has been first presented in a discussion on xmlrpc.com [https://web.archive.org/web/20060624230303/http://www.xmlrpc.com/discuss/msgReader$1208?mode=topic].

`xmlrpc.server` — Servidores XML-RPC básicos

Código-fonte: Lib/xmlrpc/server.py
[https://github.com/python/cpython/tree/3.15/Lib/xmlrpc/server.py]

The `xmlrpc.server` module provides a basic server framework for XML-RPC servers written in Python. Servers can either be free standing, using `SimpleXMLRPCServer`, or embedded in a CGI environment, using `CGIXMLRPCRequestHandler`.

Aviso

The `xmlrpc.server` module is not secure against maliciously constructed data. If you need to parse untrusted or unauthenticated data, see Segurança no XML.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

class xmlrpc.server.SimpleXMLRPCServer(*addr*, *requestHandler=SimpleXMLRPCRequestHandler*, *logRequests=True*, *allow_none=False*, *encoding=None*, *bind_and_activate=True*, *use_builtin_types=False*)

Cria uma nova instância do servidor. Esta classe fornece métodos para registro de funções que podem ser chamadas pelo protocolo XML-RPC. O parâmetro *requestHandler* deve ser uma fábrica para instâncias do tratador de solicitações; o padrão é `SimpleXMLRPCRequestHandler`. Os parâmetros *addr* e *requestHandler* são passados para o construtor

`socketserver.TCPServer`. Se *logRequests* for true (o padrão), as solicitações serão registradas; definir esse parâmetro como false desativará os registros. Os parâmetros *allow_none* e *encoding* são transmitidos para `xmlrpc.client` e controlam as respostas XML-RPC que serão retornadas do servidor. O parâmetro *bind_and_activate* controla se **`server_bind()`** e **`server_activate()`** são chamados imediatamente pelo construtor; o padrão é true. A configuração como false permite que o código manipule a variável de classe *allow_reuse_address* antes que o endereço seja vinculado. O parâmetro *use_builtin_types* é passado para a função `loads()` e controla quais tipos são processados quando valores de data/hora ou dados binários são recebidos; o padrão é false.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

class xmlrpc.server.CGIXMLRPCRequestHandler(*allow_none=False*, *encoding=None*, *use_builtin_types=False*)

Cria uma nova instância para manipular solicitações XML-RPC em um ambiente CGI. Os parâmetros *allow_none* e *encoding* são transmitidos para `xmlrpc.client` e controlam as respostas XML-RPC que serão retornadas do servidor. O parâmetro *use_builtin_types* é passado para a função `loads()` e controla quais tipos são processados quando valores de data/hora ou dados binários são recebidos; o padrão é false.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

class xmlrpc.server.SimpleXMLRPCRequestHandler

Cria uma nova instância do manipulador de solicitações. Este manipulador de solicitação possui suporte a solicitações `POST` e modifica o registro para que o parâmetro *logRequests* para o construtor de `SimpleXMLRPCServer` seja respeitado.

SimpleXMLRPCServer objects

A classe `SimpleXMLRPCServer` é baseada em `socketserver.TCPServer` e fornece um meio de criar servidores XML-RPC simples e independentes.

SimpleXMLRPCServer.register_function(*function=None*, *name=None*)

Registra uma função que possa responder às solicitações XML-RPC. Se *name* for fornecido, será o nome do método associado a *function*, caso contrário, `function.__name__` será usado. *name* é uma string e pode conter caracteres ilegais para identificadores Python, incluindo o caractere de ponto.

Este método também pode ser usado como um decorador. Quando usado como decorador, *name* só pode ser fornecido como argumento nomeado para registrar *function* em *name*. Se nenhum *name* for fornecido, `function.__name__` será usado.

Alterado na versão 3.7: `register_function()` pode ser usado como um decorador.

SimpleXMLRPCServer.register_instance(*instance*, *allow_dotted_names=False*)

Registra um objeto que é usado para expor nomes de métodos que não foram registrados usando `register_function()`. Se *instance* contiver um método **`_dispatch()`**, ele será chamado com o nome do método solicitado e os parâmetros da solicitação. Sua API é `def _dispatch(self, method, params)` (observe que *params* não representa uma lista de argumentos variáveis). Se ele chama uma função subjacente para executar sua tarefa, essa função é chamada como `func(*params)`, expandindo a lista de parâmetros. O valor de retorno de **`_dispatch()`** é retornado ao cliente como resultado. Se *instance* não possui o método **`_dispatch()`**, é procurado por um atributo correspondente ao nome do método solicitado.

Se o argumento opcional *allow_dotted_names* for true e a instância não tiver o método **`_dispatch()`**, e se o nome do método solicitado contiver pontos, cada componente do nome do método será pesquisado individualmente, com o efeito de que um simples pesquisa hierárquica é realizada. O valor encontrado nessa pesquisa é chamado com os parâmetros da solicitação e o valor retornado é passado de volta ao cliente.

Aviso

A ativação da opção *allow_dotted_names* permite que os invasores acessem as variáveis globais do seu módulo e podem permitir que os invasores executem códigos arbitrários em sua máquina. Use esta opção apenas em uma rede fechada e segura.

SimpleXMLRPCServer.register_introspection_functions()

Registra as funções de introspecção XML-RPC `system.listMethods`, `system.methodHelp` e `system.methodSignature`.

SimpleXMLRPCServer.register_multicall_functions()

Registra a função de multichamada XML-RPC system.multicall.

SimpleXMLRPCRequestHandler.rpc_paths

Um valor de atributo que deve ser uma tupla listando partes do caminho válidas da URL para receber solicitações XML-RPC. Solicitações postadas em outros caminhos resultarão em um erro HTTP 404 “página inexistente”. Se esta tupla estiver vazia, todos os caminhos serão considerados válidos. O valor padrão é `('/', '/RPC2')`.

SimpleXMLRPCServer example

Código do servidor:

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

# Restringe a um caminho em particular.
class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)

# Cria o servidor
with SimpleXMLRPCServer(('localhost', 8000),
                        requestHandler=RequestHandler) as server
    server.register_introspection_functions()

    # Registra uma função pow(); isso vai usar o valor de
```

```
    # pow.__name__ como o nome, que é apenas 'pow'.
    server.register_function(pow)

    # Registra uma função sob um nome diferente
    def adder_function(x, y):
        return x + y
    server.register_function(adder_function, 'add')

    # Registra uma instância; todos os métodos da instância
    # são publicados como métodos XML-RPC (neste caso,
    # apenas 'mul').
    class MyFuncs:
        def mul(self, x, y):
            return x * y

    server.register_instance(MyFuncs())

    # Executa o laço principal do servidor
    server.serve_forever()
```

O código do cliente a seguir chamará os métodos disponibilizados pelo servidor anterior:

```
import xmlrpc.client

s = xmlrpc.client.ServerProxy('http://localhost:8000')
print(s.pow(2,3))  # Returns 2**3 = 8
print(s.add(2,3))  # Returns 5
print(s.mul(5,2))  # Returns 5*2 = 10

# Exibe uma lista dos métodos disponíveis
print(s.system.listMethods())
```

`register_function()` também pode ser usado como um decorador. O exemplo anterior do servidor pode registrar funções com um decorador:

```
from xmlrpc.server import SimpleXMLRPCServer
from xmlrpc.server import SimpleXMLRPCRequestHandler

class RequestHandler(SimpleXMLRPCRequestHandler):
    rpc_paths = ('/RPC2',)
```

```
with SimpleXMLRPCServer(('localhost', 8000),
                        requestHandler=RequestHandler) as server
    server.register_introspection_functions()

    # Registra a função pow(); isso vai usar o valor de
    # pow.__name__ como o nome, que é apenas 'pow'.
    server.register_function(pow)

    # Registra uma função sob um nome diferente, usando
    # register_function como um decorador. *name* só pode
    # ser dado como um argumento nomeado.
    @server.register_function(name='add')
    def adder_function(x, y):
        return x + y

    # Registra uma função sob function.__name__.
    @server.register_function
    def mul(x, y):
        return x * y

    server.serve_forever()
```

O exemplo a seguir, incluído no módulo `Lib/xmlrpc/server.py`, mostra um servidor que permite nomes com pontos e registra uma função de várias chamadas.

Aviso

Enabling the *allow_dotted_names* option allows intruders to access your module's global variables and may allow intruders to execute arbitrary code on your machine. Only use this example within a secure, closed network.

```
import datetime as dt

class ExampleService:
    def getData(self):
        return '42'
```

```
    class currentTime:
        @staticmethod
        def getCurrentTime():
            return dt.datetime.now()

with SimpleXMLRPCServer(("localhost", 8000)) as server:
    server.register_function(pow)
    server.register_function(lambda x,y: x+y, 'add')
    server.register_instance(ExampleService(), allow_dotted_name
    server.register_multicall_functions()
    print('Serving XML-RPC on localhost port 8000')
    try:
        server.serve_forever()
    except KeyboardInterrupt:
        print("\nKeyboard interrupt received, exiting.")
        sys.exit(0)
```

Esta demonstração ExampleService pode ser chamada na linha de comando:

```
python -m xmlrpc.server
```

O cliente que interage com o servidor acima está incluído em `Lib/xmlrpc/client.py`:

```
server = ServerProxy("http://localhost:8000")

try:
    print(server.currentTime.getCurrentTime())
except Error as v:
    print("ERROR", v)

multi = MultiCall(server)
multi.getData()
multi.pow(2,9)
multi.add(1,2)
try:
    for response in multi():
        print(response)
```

```
except Error as v:
    print("ERROR", v)
```

Este cliente que interage com o servidor XMLRPC de demonstração pode ser chamado como:

```
python -m xmlrpc.client
```

CGIXMLRPCRequestHandler

A classe `CGIXMLRPCRequestHandler` pode ser usada para manipular solicitações XML-RPC enviadas para scripts CGI Python.

CGIXMLRPCRequestHandler.register_function(*function=None*, *name=None*)

Registra uma função que possa responder às solicitações XML-RPC. Se *name* for fornecido, será o nome do método associado a *function*, caso contrário, `function.__name__` será usado. *name* é uma string e pode conter caracteres ilegais para identificadores Python, incluindo o caractere de ponto.

Este método também pode ser usado como um decorador. Quando usado como decorador, *name* só pode ser fornecido como argumento nomeado para registrar *function* em *name*. Se nenhum *name* for fornecido, `function.__name__` será usado.

Alterado na versão 3.7: `register_function()` pode ser usado como um decorador.

CGIXMLRPCRequestHandler.register_instance(*instance*)

Registra um objeto que é usado para expor nomes de métodos que não foram registrados usando `register_function()`. Se a instância contiver um método **`_dispatch()`**, ela será chamada com o nome do método solicitado e os parâmetros da solicitação; o valor retornado é retornado ao cliente como resultado. Se a instância não tiver um método **`_dispatch()`**, será procurado um atributo correspondente ao nome do método solicitado; se o nome do método solicitado contiver pontos,

cada componente do nome do método será pesquisado individualmente, com o efeito de que uma pesquisa hierárquica simples é executada. O valor encontrado nessa pesquisa é chamado com os parâmetros da solicitação e o valor retornado é passado de volta ao cliente.

CGIXMLRPCRequestHandler.register_introspection_functions()

Registra as funções de introspecção XML-RPC `system.listMethods`, `system.methodHelp` e `system.methodSignature`.

CGIXMLRPCRequestHandler.register_multicall_functions()

Registra a função de multichamada XML-RPC `system.multicall`.

CGIXMLRPCRequestHandler.handle_request(*request_text=None*)

Manipula uma solicitação XML-RPC. Se *request_text* for fornecido, devem ser os dados POST fornecidos pelo servidor HTTP, caso contrário, o conteúdo do stdin será usado.

Exemplo:

```
class MyFuncs:
    def mul(self, x, y):
        return x * y


handler = CGIXMLRPCRequestHandler()
handler.register_function(pow)
handler.register_function(lambda x,y: x+y, 'add')
handler.register_introspection_functions()
handler.register_instance(MyFuncs())
handler.handle_request()
```

Documentando servidor XMLRPC

Essas classes estendem as classes acima para servir a documentação HTML em resposta a solicitações HTTP GET. Os servidores podem ser independentes, usando **`DocXMLRPCServer`** ou incorporados em um ambiente CGI, usando **`DocCGIXMLRPCRequestHandler`**.

class xmlrpc.server.DocXMLRPCServer(*addr*, *requestHandler=DocXMLRPCRequestHandler*, *logRequests=True*, *allow_none=False*, *encoding=None*, *bind_and_activate=True*, *use_builtin_types=True*)

Cria uma nova instância do servidor. Todos os parâmetros têm o mesmo significado que para `SimpleXMLRPCServer`; *requestHandler* assume como padrão `DocXMLRPCRequestHandler`.

Alterado na versão 3.3: O sinalizador *use_builtin_types* foi adicionado.

class xmlrpc.server.DocCGIXMLRPCRequestHandler

Cria uma nova instância para manipular solicitações XML-RPC em um ambiente CGI.

class xmlrpc.server.DocXMLRPCRequestHandler

Cria uma nova instância do manipulador de solicitações. Este manipulador de solicitações possui suporte a solicitações POST de XML-RPC, documenta solicitações GET e modifica o registro para que o parâmetro *logRequests* no parâmetro `DocXMLRPCServer` seja respeitado.

DocXMLRPCServer objects

A classe `DocXMLRPCServer` é derivada de `SimpleXMLRPCServer` e fornece um meio de criar servidores XML-RPC autodocumentados e independentes. Solicitações HTTP POST são tratadas como chamadas de método XML-RPC. As solicitações HTTP GET são tratadas gerando documentação HTML no estilo pydoc. Isso permite que um servidor forneça sua própria documentação baseada na Web.

DocXMLRPCServer.set_server_title(*server_title*)

Define o título usado na documentação HTML gerada. Este título será usado dentro do elemento “title” do HTML.

DocXMLRPCServer.set_server_name(*server_name*)

Define o nome usado na documentação HTML gerada. Este nome aparecerá na parte superior da documentação gerada dentro de um elemento “h1”.

DocXMLRPCServer.set_server_documentation(*server_documentation*)

Define a descrição usada na documentação HTML gerada. Esta descrição aparecerá na documentação como um parágrafo, abaixo do nome do servidor.

DocCGIXMLRPCRequestHandler

A classe **`DocCGIXMLRPCRequestHandler`** é derivada de **`CGIXMLRPCRequestHandler`** e fornece um meio de criar scripts CGI XML-RPC autodocumentados. Solicitações HTTP POST são tratadas como chamadas de método XML-RPC. As solicitações HTTP GET são tratadas gerando documentação HTML no estilo pydoc. Isso permite que um servidor forneça sua própria documentação baseada na web.

DocCGIXMLRPCRequestHandler.set_server_title(*server_title*)

Define o título usado na documentação HTML gerada. Este título será usado dentro do elemento “title” do HTML.

DocCGIXMLRPCRequestHandler.set_server_name(*server_name*)

Define o nome usado na documentação HTML gerada. Este nome aparecerá na parte superior da documentação gerada dentro de um elemento “h1”.

DocCGIXMLRPCRequestHandler.set_server_documentation(*server_documentation*)

Define a descrição usada na documentação HTML gerada. Esta descrição aparecerá na documentação como um parágrafo, abaixo do nome do servidor.

`ipaddress` — Biblioteca de manipulação de IPv4/IPv6

Código-fonte: Lib/ipaddress.py
[https://github.com/python/cpython/tree/3.15/Lib/ipaddress.py]

`ipaddress` fornece recursos para criar, manipular e operar em endereços e redes IPv4 e IPv6.

As funções e classes neste módulo facilitam o tratamento de várias tarefas relacionadas a endereços IP, incluindo verificar se dois hosts estão ou não na mesma sub-rede, iterar sobre todos os hosts em uma sub-rede específica, verificar se uma string representa ou não um valor válido. Endereço IP ou definição de rede e assim por diante.

Esta é a referência completa da API do módulo – para uma visão geral e introdução, consulte Uma introdução ao módulo ipaddress.

Adicionado na versão 3.3.

Funções de fábrica de conveniência

O módulo **`ipaddress`** fornece funções de fábrica para criar endereços IP, redes e interfaces de forma conveniente:

ipaddress.ip_address(*address*)

Retorna um objeto **`IPv4Address`** ou **`IPv6Address`** dependendo do endereço IP passado como argumento. Podem ser fornecidos endereços IPv4 ou IPv6; números inteiros menores que `2**32` serão considerados IPv4 por padrão. Uma exceção **`ValueError`** é levantada se *address* não representar um endereço IPv4 ou IPv6 válido.

```
>>> ipaddress.ip_address('192.168.0.1')
IPv4Address('192.168.0.1')
>>> ipaddress.ip_address('2001:db8::')
IPv6Address('2001:db8::')
```

ipaddress.ip_network(*address*, *strict=True*)

Retorna um objeto **`IPv4Network`** ou **`IPv6Network`** dependendo do endereço IP passado como argumento. *address* é uma string ou número inteiro que representa a rede IP. Podem ser fornecidas redes IPv4 ou IPv6; números inteiros menores que `2**32` serão considerados IPv4 por padrão. *strict* é passado para o construtor **`IPv4Network`** ou **`IPv6Network`**. Uma exceção **`ValueError`** é levantada se *address* não representar um endereço IPv4 ou IPv6 válido, ou se a rede tiver bits de host configurados.

```
>>> ipaddress.ip_network('192.168.0.0/28')
IPv4Network('192.168.0.0/28')
```

ipaddress.ip_interface(*address*)

Retorna um objeto **`IPv4Interface`** ou **`IPv6Interface`** dependendo do endereço IP passado como argumento. *address* é uma string ou um inteiro representando o endereço IP. Podem ser fornecidos endereços IPv4 ou IPv6; números inteiros menores que `2**32` serão considerados IPv4 por padrão. Uma exceção **`ValueError`** é levantada se *address* não representar um endereço IPv4 ou IPv6 válido.

Uma desvantagem dessas funções de conveniência é que a necessidade de lidar com os formatos IPv4 e IPv6 significa que as mensagens de erro fornecem informações mínimas sobre o erro exato, pois as funções não sabem se o formato IPv4 ou IPv6 foi pretendido. Relatórios de erros mais detalhados podem ser obtidos chamando diretamente os construtores de classe específicos da versão apropriada.

Endereços IP

Objetos de endereço

Os objetos **IPv4Address** e **IPv6Address** compartilham muitos atributos comuns. Alguns atributos que são significativos apenas para endereços IPv6 também são implementados por objetos **IPv4Address**, para facilitar a escrita de código que lide corretamente com ambas as versões de IP. Os objetos de endereço são hasheáveis, portanto podem ser usados como chaves em dicionários.

class ipaddress.IPv4Address(*address*)

Constrói um endereço IPv4. Uma exceção **AddressValueError** é levantada se *address* não for um endereço IPv4 válido.

O seguinte constitui um endereço IPv4 válido:

1. Uma string em notação decimal por ponto, consistindo de quatro inteiros decimais em um intervalo inclusivo 0–255 separado por pontos (e.g. `192.168.0.1`). Cada inteiro representa um octeto (byte) no endereço. Zeros à esquerda não são tolerados para evitar confusão com notação octal.
2. Um inteiro que cabe em 32 bits.
3. Um inteiro compactado em um objeto **bytes** de comprimento 4 (octeto mais significativo primeiro).

```
>>> ipaddress.IPv4Address('192.168.0.1')
IPv4Address('192.168.0.1')
>>> ipaddress.IPv4Address(3232235521)
IPv4Address('192.168.0.1')
>>> ipaddress.IPv4Address(b'\xC0\xA8\x00\x01')
IPv4Address('192.168.0.1')
```

Alterado na versão 3.8: Zeros à esquerda são tolerados, mesmo em casos ambíguos que se parecem com notação octal.

Alterado na versão 3.9.5: Zeros à esquerda não são mais tolerados e são tratados como um erro. As strings de endereço IPv4 agora são analisadas de forma tão estrita quanto glibc **inet_pton()**.

version

O número de versão apropriado: `4` para IPv4, `6` para IPv6.

Alterado na versão 3.14: Disponibilizado na classe.

max_prefixlen

O número total de bits na representação de endereço para esta versão: `32` para IPv4, `128` para IPv6.

O prefixo define o número de bits iniciais em um endereço que são comparados para determinar se um endereço faz ou não parte de uma rede.

Alterado na versão 3.14: Disponibilizado na classe.

compressed

exploded

A representação de string em notação decimal pontilhada. Zeros à esquerda nunca são incluídos na representação.

Como o IPv4 não define uma notação abreviada para endereços com octetos definidos como zero, esses dois atributos são sempre iguais a `str(addr)` para endereços IPv4. A exposição desses atributos facilita a gravação de código de exibição que pode tratar endereços IPv4 e IPv6.

packed

A representação binária deste endereço – um objeto **`bytes`** de comprimento apropriado (primeiro o octeto mais significativo). São 4 bytes para IPv4 e 16 bytes para IPv6.

reverse_pointer

O nome do registro PTR de DNS reverso para o endereço IP, por exemplo:

```
>>> ipaddress.ip_address("127.0.0.1").reverse_pointer
'1.0.0.127.in-addr.arpa'
>>> ipaddress.ip_address("2001:db8::1").reverse_pointer
'1.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.0.8.b.d.0.1
```

Este é o nome que pode ser usado para realizar uma pesquisa PTR, não o nome do host resolvido em si.

Adicionado na versão 3.5.

is_multicast

`True` se o endereço estiver reservado para uso multicast. Consulte **RFC 3171** [https://datatracker.ietf.org/doc/html/rfc3171.html] (para IPv4) ou **RFC 2373** [https://datatracker.ietf.org/doc/html/rfc2373.html] (para IPv6).

is_private

`True` se o endereço for definido como não acessível globalmente por iana-ipv4-special-registry [https://www.iana.org/assignments/iana-ipv4-special-registry/iana-ipv4-special-registry.xhtml] (para IPv4) ou iana-ipv6-special-registry [https://www.iana.org/assignments/iana-ipv6-special-registry/iana-ipv6-special-registry.xhtml] (para IPv6) com as seguintes exceções:

- `is_private` é `False` para o espaço de endereço compartilhado (`100.64.0.0/10`)
- Para endereços IPv6 mapeados em IPv4, o valor `is_private` é determinado pela semântica dos endereços IPv4 subjacentes e a seguinte condição é válida (consulte **`IPv6Address.ipv4_mapped`**):

  ```
  address.is_private == address.ipv4_mapped.is_private
  ```

`is_private` tem valor oposto a **`is_global`**, exceto para o espaço de endereço compartilhado (intervalo `100.64.0.0/10`) onde ambos são `False`.

Alterado na versão 3.13: Corrigidos alguns falsos positivos e falsos negativos.

- `192.0.0.0/24` é considerado privado com exceção de `192.0.0.9/32` e `192.0.0.10/32` (anteriormente: apenas o sub-intervalo `192.0.0.0/29` foi considerado privado).
- `64:ff9b:1::/48` é considerado privado.

- `2002::/16` é considerado privado.
- Existem exceções em `2001::/23` (de outra forma considerado privado): `2001:1::1/128`, `2001:1::2/128`, `2001:3::/32`, `2001:4:112::/48`, `2001:20::/28`, `2001:30::/28`. As exceções não são consideradas privadas.

is_global

`True` se o endereço for definido como não acessível globalmente por iana-ipv4-special-registry [https://www.iana.org/assignments/iana-ipv4-special-registry/iana-ipv4-special-registry.xhtml] (para IPv4) ou iana-ipv6-special-registry [https://www.iana.org/assignments/iana-ipv6-special-registry/iana-ipv6-special-registry.xhtml] (para IPv6) com a seguinte exceção:

Para endereços IPv6 mapeados em IPv4, o valor `is_private` é determinado pela semântica dos endereços IPv4 subjacentes e a seguinte condição é válida (consulte **`IPv6Address.ipv4_mapped`**):

```
address.is_global == address.ipv4_mapped.is_global
```

`is_global` tem valor oposto a **`is_private`**, exceto para o espaço de endereço compartilhado (intervalo `100.64.0.0/10`) onde ambos são `False`.

Adicionado na versão 3.4.

Alterado na versão 3.13: Corrigidos alguns falsos positivos e falsos negativos, veja **`is_private`** para detalhes.

is_unspecified

`True` se o endereço não estiver especificado. Consulte **RFC 5735** [https://datatracker.ietf.org/doc/html/rfc5735.html] (para IPv4) ou **RFC 2373** [https://datatracker.ietf.org/doc/html/rfc2373.html] (para IPv6).

is_reserved

`True` se o endereço for indicado como reservado pela IETF. Para IPv4, este é apenas `240.0.0.0/4`, o bloco de endereços `Reserved`. Para IPv6, este é todos os endereços alocados

[https://www.iana.org/assignments/ipv6-address-space/ipv6-address-space.xhtml] como `Reserved by IETF` para uso futuro.

Nota

Para IPv4, `is_reserved` não está relacionado ao valor do bloco de endereço da coluna `Reserved-by-Protocol` em iana-ipv4-special-registry [https://www.iana.org/assignments/iana-ipv4-special-registry/iana-ipv4-special-registry.xhtml].

Cuidado

Para IPv6, `fec0::/10` um antigo prefixo de endereço de escopo Site-Local está atualmente excluído dessa lista (veja **`is_site_local`** & **RFC 3879** [https://datatracker.ietf.org/doc/html/rfc3879.html]).

is_loopback

`True` se este for um endereço de loopack. Consulte **RFC 3330** [https://datatracker.ietf.org/doc/html/rfc3330.html] (para IPv4) ou **RFC 2373** [https://datatracker.ietf.org/doc/html/rfc2373.html] (para IPv6).

is_link_local

`True` se o endereço está reservado para uso de link local. Veja: **RFC 3927** [https://datatracker.ietf.org/doc/html/rfc3927.html].

ipv6_mapped

Objeto **`IPv6Address`** representando o endereço IPv6 mapeado para IPv6. Veja **RFC 4291** [https://datatracker.ietf.org/doc/html/rfc4291.html].

Adicionado na versão 3.13.

IPv4Address.__format__(*fmt*)

Retorna a representação string do endereço de IP, controlado por uma string de formato explícito. *fmt* pode ser um dos seguintes: `'s'`, a opção padrão, equivalente à **`str()`**, `'b'` para uma string binária preenchida com zeros, `'X'` ou `'x'` para uma representação hexadecimal maiúscula

ou minúscula, ou `'n'`, que equivale a `'b'` para endereços IPv4 e `'x'` para IPv6. Para representações binárias e hexadecimais, o especificador de forma `'#'` e a opção de agrupamento `'_'` estão disponíveis. `__format__` é utilizado por `format`, `str.format` e f-strings.

```
>>> format(ipaddress.IPv4Address('192.168.0.1'))
'192.168.0.1'
>>> '{:#b}'.format(ipaddress.IPv4Address('192.168.0.1'))
'0b11000000101010000000000000000001'
>>> f'{ipaddress.IPv6Address("2001:db8::1000"):s}'
'2001:db8::1000'
>>> format(ipaddress.IPv6Address('2001:db8::1000'), '_X')
'2001_0DB8_0000_0000_0000_0000_0000_1000'
>>> '{:#_n}'.format(ipaddress.IPv6Address('2001:db8::1000'))
'0x2001_0db8_0000_0000_0000_0000_0000_1000'
```

Adicionado na versão 3.9.

class ipaddress.IPv6Address(*address*)

Constrói um endereço IPv6. Uma exceção **`AddressValueError`** é levantada se *address* não for um endereço IPv6 válido.

O seguinte constitui um endereço IPv6 válido:

1. Uma string constituída de oito grupos de quatro dígitos hexadecimais, cada grupo representando 16 bits. Os grupos são separados por dois pontos. Isto descreve uma notação *explodida* (longa); A string também pode ser *compactada* (notação curta) por vários meios. Ver **RFC 4291** [https://datatracker.ietf.org/doc/html/rfc4291.html] para detalhes. Por exemplo, `"0000:0000:0000:0000:0000:0abc:0007:0def"` pode ser compactada para `"::abc:7:def"`.

   Opcionalmente, a string pode ter um ID de escopo de zona, expressado por um sufixo `%scope_id`. Se presente, o ID de escopo deve ser não vazio, e pode não conter `%`. Ver **RFC 4007** [https://datatracker.ietf.org/doc/html/rfc4007.html] para detalhes. Por exemplo, `fe80::1234%1` pode identificar o endereço `fe80::1234` no primeiro link do nó.

2. Um inteiro que cabe em 128 bits.

3. Um inteiro compactado em um objeto **bytes** de comprimento 16, big-endian.

```
>>> ipaddress.IPv6Address('2001:db8::1000')
IPv6Address('2001:db8::1000')
>>> ipaddress.IPv6Address('ff02::5678%1')
IPv6Address('ff02::5678%1')
```

compressed

A forma curta da representação do endereço, com zeros à esquerda em grupos omitidos e a sequência mais longa de grupos consistida inteiramente por zeros colapsada em um grupo vazio único.

Este também é o valor retornado por `str(addr)` para endereços IPv6.

exploded

A forma longa da representação do endereço, com todos zeros à esquerda em grupos consistindo inteiramente de zeros incluídos.

Para os seguintes atributos e métodos, veja a documentação correspondente para a classe **IPv4Address**:

packed

reverse_pointer

version

max_prefixlen

is_multicast

is_private

is_global

Adicionado na versão 3.4.

is_unspecified

is_reserved

is_loopback

is_link_local

is_site_local

`True` se o endereço estiver reservado para uso local do site. Observe que o espaço de endereço local do site foi descontinuado pelo **RFC 3879** [https://datatracker.ietf.org/doc/html/rfc3879.html]. Use **`is_private`** para testar se este endereço está no espaço de endereços locais exclusivos conforme definido pelo **RFC 4193** [https://datatracker.ietf.org/doc/html/rfc4193.html].

ipv4_mapped

Para endereços que aparentam ser endereços mapeados para IPv4 no intervalo `::FFFF:0:0/96` conforme definido pela **RFC 4291** [https://datatracker.ietf.org/doc/html/rfc4291.html], esta propriedade relata o endereço IPv4 embutido. Para qualquer outro endereço, esta propriedade será `None`.

scope_id

Para endereços com escopo definido pelo **RFC 4007** [https://datatracker.ietf.org/doc/html/rfc4007.html], esta propriedade identifica a zona específica do escopo do endereço à qual o endereço pertence, como uma string. Quando nenhuma zona de escopo for especificada, esta propriedade será `None`.

sixtofour

Para endereços que parecem ser endereços 6to4 (começando com `2002::/16`), como definido pelo **RFC 3056** [https://datatracker.ietf.org/doc/html/rfc3056.html], esta propriedade reportará o endereço IPv4 incorporado. Para qualquer outro endereço, esta propriedade será `None`.

teredo

Para endereços que parecem ser endereços Teredo (começando com `2001::/32`), como definido pelo **RFC 4380** [https://datatracker.ietf.org/doc/html/rfc4380.html], esta propriedade reportará o par de endereços `(server, client)` incorporado. Para qualquer outro endereço, esta propriedade será `None`.

IPv6Address.__format__(*fmt*)

Consulte a documentação do método correspondente em **`IPv4Address`**.

Adicionado na versão 3.9.

Conversão para strings e inteiros

Para interoperar com interfaces de rede, como o módulo de soquete, os endereços devem ser convertidos em strings ou inteiros. Isso é tratado usando as funções internas **`str()`** e **`int()`**:

```
>>> str(ipaddress.IPv4Address('192.168.0.1'))
'192.168.0.1'
>>> int(ipaddress.IPv4Address('192.168.0.1'))
3232235521
>>> str(ipaddress.IPv6Address('::1'))
'::1'
>>> int(ipaddress.IPv6Address('::1'))
1
```

Observe que os endereços com escopo IPv6 são convertidos em números inteiros sem ID de zona de escopo.

Operadores

Os objetos de endereço têm suporte a alguns operadores. Salvo indicação em contrário, os operadores só podem ser aplicados entre objetos compatíveis (ou seja, IPv4 com IPv4, IPv6 com IPv6).

Operadores de comparação

Os objetos de endereço podem ser comparados com o conjunto usual de operadores de comparação. Os mesmos endereços IPv6 com IDs de zona de escopo diferentes não são iguais. Alguns exemplos:

```
>>> IPv4Address('127.0.0.2') > IPv4Address('127.0.0.1')
True
>>> IPv4Address('127.0.0.2') == IPv4Address('127.0.0.1')
False
>>> IPv4Address('127.0.0.2') != IPv4Address('127.0.0.1')
True
>>> IPv6Address('fe80::1234') == IPv6Address('fe80::1234%1')
False
>>> IPv6Address('fe80::1234%1') != IPv6Address('fe80::1234%2')
True
```

Operadores aritméticos

Inteiros podem ser adicionados ou subtraídos de objetos de endereço. Alguns exemplos:

```
>>> IPv4Address('127.0.0.2') + 3
IPv4Address('127.0.0.5')
>>> IPv4Address('127.0.0.2') - 3
IPv4Address('126.255.255.255')
>>> IPv4Address('255.255.255.255') + 1
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ipaddress.AddressValueError: 4294967296 (>= 2**32) is not permit
```

Definições de rede IP

Os objetos `IPv4Network` e `IPv6Network` fornecem um mecanismo para definir e inspecionar definições de rede IP. Uma definição de rede consiste em uma *máscara* e um *endereço de rede* e, como tal, define um intervalo de endereços IP que é igual ao endereço de rede quando mascarado (E binário) com a máscara. Por exemplo, uma definição de rede com a máscara `255.255.255.0` e o endereço de rede `192.168.1.0` consiste em endereços IP no intervalo inclusivo `192.168.1.0` a `192.168.1.255`.

Prefixo, máscara de rede e máscara de host

Existem várias maneiras equivalentes de especificar máscaras de rede IP. Um *prefixo* `/<nbits>` é uma notação que denota quantos bits de ordem superior estão definidos na máscara de rede. Uma máscara de rede, ou *netmask*, é um endereço IP com um certo número de bits de alta ordem definidos. Assim o prefixo `/24` é equivalente à máscara de rede `255.255.255.0` em IPv4, ou `ffff:ff00::` em IPv6. Além disso, uma máscara de host, ou *host mask*, é o inverso lógico de uma máscara de rede e às vezes é usada (por exemplo, nas listas de controle de acesso da Cisco) para denotar uma máscara de rede. A máscara de host equivalente a `/24` em IPv4 é `0.0.0.255`.

Objetos de rede

Todos os atributos implementados por objetos de endereço também são implementados por objetos de rede. Além disso, os objetos de rede implementam atributos adicionais. Todos estes são comuns entre **`IPv4Network`** e **`IPv6Network`**, portanto, para evitar duplicação, eles são documentados apenas para **`IPv4Network`**. Objetos de rede são hasheáveis, portanto podem ser usados como chaves em dicionários.

class ipaddress.IPv4Network(*address*, *strict=True*)

Constrói uma definição de rede IPv4. *address* pode ser um dos seguintes:

1. Uma string que consiste em um endereço IP e uma máscara opcional, separados por uma barra (`/`). O endereço IP é o endereço de rede, e a máscara pode ser um único número, o que significa que é um *prefixo*, ou uma representação de string de um endereço IPv4. Se for o último, a máscara será interpretada como uma *máscara de rede* se começar com um campo diferente de zero, ou como uma *máscara de host* se começar com um campo zero, com a única exceção de uma máscara totalmente zero que é tratada como uma *máscara de rede*. Se nenhuma máscara for fornecida, será considerada `/32`.

Por exemplo, as seguintes especificações de *address* são equivalentes: `192.168.1.0/24`, `192.168.1.0/255.255.255.0` e `192.168.1.0/0.0.0.255`.

2. Um número inteiro que cabe em 32 bits. Isto é equivalente a uma rede de endereço único, com o endereço de rede sendo *address* e a máscara sendo `/32`.

3. Um inteiro compactado em um objeto **`bytes`** de comprimento 4, big-endian. A interpretação é semelhante a um *address* no formato inteiro.

4. Uma tupla de dois elementos de uma descrição de endereço e uma máscara de rede, onde a descrição do endereço é uma string, um número inteiro de 32 bits, um número inteiro compactado de 4 bytes ou um objeto **`IPv4Address`** existente; e a máscara de rede é um número inteiro que representa o comprimento do prefixo (por exemplo, `24`) ou uma string que representa a máscara do prefixo (por exemplo, `255.255.255.0`).

Uma exceção **`AddressValueError`** é levantada se *address* não for um endereço IPv4 válido. Uma exceção **`NetmaskValueError`** é levantada se a máscara não for válida para um endereço IPv4.

Se *strict* for `True` e os bits do host estiverem definidos no endereço fornecido, então **`ValueError`** será levantada. Caso contrário, os bits do host serão mascarados para determinar o endereço de rede apropriado.

Salvo indicação em contrário, todos os métodos de rede que aceitam outros objetos rede/endereço irão levantar **`TypeError`** se a versão IP do argumento for incompatível com `self`.

Alterado na versão 3.5: Adicionado o formulário de tupla de 2 elementos para o parâmetro do construtor *address*.

version

max_prefixlen

Consulte a documentação do atributo correspondente em `IPv4Address`.

is_multicast

is_private

is_unspecified

is_reserved

is_loopback

is_link_local

Esses atributos terão valor verdadeiro para a rede como um todo se forem verdadeiros tanto para o endereço de rede quanto para o endereço de broadcast.

network_address

O endereço de rede da rede. O endereço de rede e o comprimento do prefixo juntos definem exclusivamente uma rede.

broadcast_address

O endereço de broadcast da rede. Os pacotes enviados para o endereço de broadcast devem ser recebidos por todos os hosts da rede.

hostmask

A máscara do host, como um objeto `IPv4Address`.

netmask

A máscara de rede, como um objeto `IPv4Address`.

with_prefixlen

compressed

exploded

Uma representação de string da rede, com a máscara em notação de prefixo.

`with_prefixlen` e `compressed` são sempre iguais a `str(network)`. `exploded` usa a forma explodida do endereço de rede.

with_netmask

Uma representação em string da rede, com a máscara na notação de máscara de rede.

with_hostmask

Uma representação de string da rede, com a máscara na notação de máscara de host.

num_addresses

O número total de endereços na rede.

prefixlen

Comprimento do prefixo de rede, em bits.

hosts()

Retorna um iterador sobre os hosts utilizáveis na rede. Os hosts utilizáveis são todos os endereços IP que pertencem à rede, exceto o próprio endereço de rede e o endereço de transmissão da rede. Para redes com comprimento de máscara 31, o endereço de rede e o endereço de transmissão de rede também são incluídos no resultado. Redes com máscara 32 retornarão uma lista contendo o endereço de host único.

```
>>> list(ip_network('192.0.2.0/29').hosts())
[IPv4Address('192.0.2.1'), IPv4Address('192.0.2.2'),
 IPv4Address('192.0.2.3'), IPv4Address('192.0.2.4'),
 IPv4Address('192.0.2.5'), IPv4Address('192.0.2.6')]
>>> list(ip_network('192.0.2.0/31').hosts())
[IPv4Address('192.0.2.0'), IPv4Address('192.0.2.1')]
>>> list(ip_network('192.0.2.1/32').hosts())
[IPv4Address('192.0.2.1')]
```

overlaps(*other*)

`True` se esta rede estiver parcial ou totalmente contida em *other* ou *other* estiver totalmente contida nesta rede.

address_exclude(*network*)

Calcula as definições de rede resultantes da remoção da *network* fornecida desta. Retorna um iterador de objetos de rede. Levanta `ValueError` se *network* não estiver completamente contida nesta rede.

```
>>> n1 = ip_network('192.0.2.0/28')
>>> n2 = ip_network('192.0.2.1/32')
>>> list(n1.address_exclude(n2))
[IPv4Network('192.0.2.8/29'), IPv4Network('192.0.2.4/30'),
 IPv4Network('192.0.2.2/31'), IPv4Network('192.0.2.0/32')]
```

subnets(*prefixlen_diff=1*, *new_prefix=None*)

As sub-redes que se unem para criar a definição de rede atual, dependendo dos valores dos argumentos. *prefixlen_diff* é o valor pelo qual o comprimento do nosso prefixo deve ser aumentado. *new_prefix* é o novo prefixo desejado das sub-redes; deve ser maior que nosso prefixo. Um, e apenas um, de *prefixlen_diff* e *new_prefix* deve ser definido. Retorna um iterador de objetos de rede.

```
>>> list(ip_network('192.0.2.0/24').subnets())
[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/25
>>> list(ip_network('192.0.2.0/24').subnets(prefixlen_dif
[IPv4Network('192.0.2.0/26'), IPv4Network('192.0.2.64/26'
 IPv4Network('192.0.2.128/26'), IPv4Network('192.0.2.192/
>>> list(ip_network('192.0.2.0/24').subnets(new_prefix=26
[IPv4Network('192.0.2.0/26'), IPv4Network('192.0.2.64/26'
 IPv4Network('192.0.2.128/26'), IPv4Network('192.0.2.192/
>>> list(ip_network('192.0.2.0/24').subnets(new_prefix=23
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
    raise ValueError('new prefix must be longer')
ValueError: new prefix must be longer
>>> list(ip_network('192.0.2.0/24').subnets(new_prefix=25
[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/25
```

supernet(*prefixlen_diff=1*, *new_prefix=None*)

As super-redes contendo esta definição de rede, dependendo dos valores dos argumentos. *prefixlen_diff* é o valor pelo qual o comprimento do nosso prefixo deve ser diminuída. *new_prefix* é o novo prefixo desejado da super-rede; deve ser maior que nosso prefixo. Um, e apenas um, de *prefixlen_diff* e *new_prefix* deve ser definido. Retorna um único objeto rede.

```
>>> ip_network('192.0.2.0/24').supernet()
IPv4Network('192.0.2.0/23')
>>> ip_network('192.0.2.0/24').supernet(prefixlen_diff=2)
IPv4Network('192.0.0.0/22')
>>> ip_network('192.0.2.0/24').supernet(new_prefix=20)
IPv4Network('192.0.0.0/20')
```

subnet_of(*other*)

Retorna `True` se esta rede é uma sub-rede de *other*.

```
>>> a = ip_network('192.168.1.0/24')
>>> b = ip_network('192.168.1.128/30')
>>> b.subnet_of(a)
True
```

Adicionado na versão 3.7.

supernet_of(*other*)

Retorna `True` se esta rede é uma super-rede de *other*.

```
>>> a = ip_network('192.168.1.0/24')
>>> b = ip_network('192.168.1.128/30')
>>> a.supernet_of(b)
True
```

Adicionado na versão 3.7.

compare_networks(*other*)

Compara esta rede com *other*. Nesta comparação são considerados apenas os endereços de rede; bits de host não são. Retorna `-1`, `0` ou `1`.

```
>>> ip_network('192.0.2.1/32').compare_networks(ip_networl
-1
>>> ip_network('192.0.2.1/32').compare_networks(ip_networl
1
>>> ip_network('192.0.2.1/32').compare_networks(ip_networl
0
```

Descontinuado desde a versão 3.7: Ele usa o mesmo algoritmo de ordenação e comparação que “<”, “==” e “>”

class ipaddress.IPv6Network(*address*, *strict=True*)

Constrói uma definição de rede IPv6. *address* pode ser um dos seguintes:

1. Uma string que consiste em um endereço IP e um comprimento de prefixo opcional, separados por uma barra (`/`). O endereço IP é o endereço de rede e o comprimento do prefixo deve ser um único número, o *prefixo*. Se nenhum comprimento de prefixo for fornecido, será considerado `/128`.

   Observe que as máscaras de rede expandidas atualmente não são suportadas. Isso significa que `2001:db00::0/24` é um argumento válido enquanto `2001:db00::0/ffff:ff00::` não é.

2. Um número inteiro que cabe em 128 bits. Isto é equivalente a uma rede de endereço único, com o endereço de rede sendo *address* e a máscara sendo `/128`.

3. Um inteiro compactado em um objeto **`bytes`** de comprimento 16, big-endian. A interpretação é semelhante a um *address* no formato inteiro.

4. Uma tupla de dois elementos de uma descrição de endereço e uma máscara de rede, onde a descrição do endereço é uma string, um número inteiro de 128 bits, um número inteiro compactado de 16 bytes ou um objeto **`IPv6Address`** existente; e a máscara de rede é um número inteiro que representa o comprimento do prefixo.

Uma exceção `AddressValueError` é levantada se *address* não for um endereço IPv6 válido. Uma exceção `NetmaskValueError` é levantada se a máscara não for válida para um endereço IPv6.

Se *strict* for `True` e os bits do host estiverem definidos no endereço fornecido, então `ValueError` será levantada. Caso contrário, os bits do host serão mascarados para determinar o endereço de rede apropriado.

Alterado na versão 3.5: Adicionado o formulário de tupla de 2 elementos para o parâmetro do construtor *address*.

version

max_prefixlen

is_multicast

is_private

is_unspecified

is_reserved

is_loopback

is_link_local

network_address

broadcast_address

hostmask

netmask

with_prefixlen

compressed

exploded

with_netmask

with_hostmask

num_addresses

prefixlen

hosts()

Retorna um iterador sobre os hosts utilizáveis na rede. Os hosts utilizáveis são todos os endereços IP que pertencem à rede, exceto o próprio endereço de anycast do roteador da sub-rede. Para redes com comprimento de máscara 127, o endereço anycast do roteador da sub-rede de rede é também incluído no resultado. Redes com máscara 128 retornarão uma lista contendo o endereço de host único.

overlaps(*other*)

address_exclude(*network*)

subnets(*prefixlen_diff=1*, *new_prefix=None*)

supernet(*prefixlen_diff=1*, *new_prefix=None*)

subnet_of(*other*)

supernet_of(*other*)

compare_networks(*other*)

Consulte a documentação do atributo correspondente em `IPv4Network`.

is_site_local

Este atributo terá valor verdadeiro para a rede como um todo se for verdadeiro tanto para o endereço de rede quanto para o endereço de broadcast.

Operadores

Os objetos de rede têm suporte a alguns operadores. Salvo indicação em contrário, os operadores só podem ser aplicados entre objetos compatíveis (ou seja, IPv4 com IPv4, IPv6 com IPv6).

Operadores lógicos

Os objetos de rede podem ser comparados com o conjunto usual de operadores lógicos. Os objetos de rede são ordenados primeiro por endereço de rede e depois por máscara de rede.

Iteração

Os objetos de rede podem ser iterados para listar todos os endereços pertencentes à rede. Para iteração, *todos* os hosts são retornados, incluindo os hosts inutilizáveis (para hosts utilizáveis, use o método `hosts()`). Um exemplo:

```
>>> for addr in IPv4Network('192.0.2.0/28'):
...     addr
...
IPv4Address('192.0.2.0')
IPv4Address('192.0.2.1')
IPv4Address('192.0.2.2')
IPv4Address('192.0.2.3')
IPv4Address('192.0.2.4')
IPv4Address('192.0.2.5')
IPv4Address('192.0.2.6')
IPv4Address('192.0.2.7')
IPv4Address('192.0.2.8')
IPv4Address('192.0.2.9')
IPv4Address('192.0.2.10')
IPv4Address('192.0.2.11')
IPv4Address('192.0.2.12')
IPv4Address('192.0.2.13')
IPv4Address('192.0.2.14')
IPv4Address('192.0.2.15')
```

Redes como contêineres de endereços

Os objetos de rede podem atuar como contêineres de endereços. Alguns exemplos:

```
>>> IPv4Network('192.0.2.0/28')[0]
IPv4Address('192.0.2.0')
>>> IPv4Network('192.0.2.0/28')[15]
IPv4Address('192.0.2.15')
>>> IPv4Address('192.0.2.6') in IPv4Network('192.0.2.0/28')
True
>>> IPv4Address('192.0.3.6') in IPv4Network('192.0.2.0/28')
False
```

Objetos de interface

Os objetos de interface são hasheáveis, portanto podem ser usados como chaves em dicionários.

class ipaddress.IPv4Interface(*address*)

Constrói uma interface IPv4. O significado de *address* é o mesmo do construtor de **`IPv4Network`**, exceto que endereços de host arbitrários são sempre aceitos.

`IPv4Interface` é uma subclasse de **`IPv4Address`**, portanto herda todos os atributos dessa classe. Além disso, os seguintes atributos estão disponíveis:

ip

O endereço (**`IPv4Address`**) sem informações de rede.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.ip
IPv4Address('192.0.2.5')
```

network

A rede (**`IPv4Network`**) à qual esta interface pertence.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.network
IPv4Network('192.0.2.0/24')
```

with_prefixlen

Uma representação de string da interface com a máscara em notação de prefixo.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.with_prefixlen
'192.0.2.5/24'
```

with_netmask

Uma representação de string da interface com a rede como uma máscara de rede.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.with_netmask
'192.0.2.5/255.255.255.0'
```

with_hostmask

Uma representação de string da interface com a rede como uma máscara de host.

```
>>> interface = IPv4Interface('192.0.2.5/24')
>>> interface.with_hostmask
'192.0.2.5/0.0.0.255'
```

class ipaddress.IPv6Interface(*address*)

Constrói uma interface IPv6. O significado de *address* é o mesmo do construtor de `IPv6Network`, exceto que endereços de host arbitrários são sempre aceitos.

`IPv6Interface` é uma subclasse de `IPv6Address`, portanto herda todos os atributos dessa classe. Além disso, os seguintes atributos estão disponíveis:

ip

network

with_prefixlen

with_netmask

with_hostmask

Consulte a documentação do atributo correspondente em `IPv4Interface`.

Operadores

Os objetos de interface têm suporte a alguns operadores. Salvo indicação em contrário, os operadores só podem ser aplicados entre objetos compatíveis (ou seja, IPv4 com IPv4, IPv6 com IPv6).

Operadores lógicos

Os objetos de interface podem ser comparados com o conjunto usual de operadores lógicos.

Para comparação de igualdade (`==` e `!=`), tanto o endereço IP quanto a rede devem ser iguais para que os objetos sejam iguais. Uma interface não será igual a nenhum endereço ou objeto de rede.

Para ordenação (<, >, etc) as regras são diferentes. Objetos de interface e de endereço com a mesma versão IP podem ser comparados e os objetos de endereço sempre serão ordenados antes dos objetos de interface. Dois objetos de interface são primeiro comparados por suas redes e, se forem iguais, então por seus endereços IP.

Outras funções de nível de módulo

O módulo também fornece as seguintes funções de nível de módulo:

ipaddress.v4_int_to_packed(*address*)

Representa um endereço como 4 bytes compactados em ordem de rede (big-endian). *address* é uma representação inteira de um endereço IP IPv4. Uma exceção `ValueError` é levantada se o número inteiro for negativo ou muito grande para ser um endereço IP IPv4.

```
>>> ipaddress.ip_address(3221225985)
IPv4Address('192.0.2.1')
>>> ipaddress.v4_int_to_packed(3221225985)
b'\xc0\x00\x02\x01'
```

ipaddress.v6_int_to_packed(*address*)

Representa um endereço como 16 bytes compactados em ordem de rede (big-endian). *address* é uma representação inteira de um endereço IP IPv6. Uma exceção **ValueError** é levantada se o número inteiro for negativo ou muito grande para ser um endereço IP IPv6.

ipaddress.summarize_address_range(*first*, *last*)

Retorna um iterador do intervalo de rede resumido, considerando o primeiro e o último endereço IP. *first* é o primeiro **IPv4Address** ou **IPv6Address** no intervalo e *last* é o último **IPv4Address** ou **IPv6Address** no intervalo. Uma exceção **TypeError** é levantada se *first* ou *last* não forem endereços IP ou não forem da mesma versão. Uma exceção **ValueError** é levantada se *last* não for maior que *first* ou se a versão do *first* endereço não for 4 ou 6.

```
>>> [ipaddr for ipaddr in ipaddress.summarize_address_range(
...     ipaddress.IPv4Address('192.0.2.0'),
...     ipaddress.IPv4Address('192.0.2.130'))]
[IPv4Network('192.0.2.0/25'), IPv4Network('192.0.2.128/31'),
```

ipaddress.collapse_addresses(*addresses*)

Retorna um iterador dos objetos **IPv4Network** ou **IPv6Network** recolhidos. *addresses* é um iterável de objetos **IPv4Network** ou **IPv6Network**. Uma exceção **TypeError** é levantada se *addresses* contiver objetos de versão mista.

```
>>> [ipaddr for ipaddr in
... ipaddress.collapse_addresses([ipaddress.IPv4Network('192.
... ipaddress.IPv4Network('192.0.2.128/25')])]
[IPv4Network('192.0.2.0/24')]
```

ipaddress.get_mixed_type_key(*obj*)

Retorna uma chave adequada para ordenação entre redes e endereços. Os objetos de endereço e de rede não são ordenáveis por padrão; eles são fundamentalmente diferentes, então a expressão:

```
IPv4Address('192.0.2.0') <= IPv4Network('192.0.2.0/24')
```

não faz sentido. Porém, há alguns momentos em que você pode desejar que **`ipaddress`** os ordene de qualquer maneira. Se precisar fazer isso, você pode usar esta função como argumento *key* para **`sorted()`**.

obj é um objeto de rede ou de endereço.

Exceções personalizadas

Para oferecer suporte a relatórios de erros mais específicos de construtores de classe, o módulo define as seguintes exceções:

exception ipaddress.AddressValueError(*ValueError*)

Qualquer erro de valor relacionado ao endereço.

exception ipaddress.NetmaskValueError(*ValueError*)

Qualquer erro de valor relacionado à máscara de rede.

Serviços Multimídia

Os módulos descritos neste capítulo implementam vários algoritmos ou interfaces que são bastante úteis em aplicações multimídia. Eles estão disponíveis a critério da instalação. Aqui está uma visão geral:

- `wave` — Read and write WAV files
  - Objetos Wave_read
  - Objetos Wave_write
- `colorsys` — Conversões entre sistemas de cores

`wave` — Read and write WAV files

Código-fonte: Lib/wave.py [https://github.com/python/cpython/tree/3.15/Lib/wave.py]

The **`wave`** module provides a convenient interface to the Waveform Audio "WAVE" (or "WAV") file format.

The module supports uncompressed PCM and IEEE floating-point WAV formats.

Alterado na versão 3.12: Support for `WAVE_FORMAT_EXTENSIBLE` headers was added, provided that the extended format is `KSDATAFORMAT_SUBTYPE_PCM`.

Alterado na versão 3.15: Support for reading and writing `WAVE_FORMAT_IEEE_FLOAT` files was added.

The **`wave`** module defines the following function and exception:

wave.open(*file*, *mode=None*)

If *file* is a string, a path-like object or a bytes-like object open the file by that name, otherwise treat it as a file-like object. *mode* can be:

`'rb'`
Modo somente para leitura.

`'wb'`
Modo somente para escrita.

Note that it does not allow read/write WAV files.

A *mode* of `'rb'` returns a **`Wave_read`** object, while a *mode* of `'wb'` returns a **`Wave_write`** object. If *mode* is omitted and a file-like object is passed as *file*, `file.mode` is used as the default value for *mode*.

If you pass in a file-like object, the wave object will not close it when its `close()` method is called; it is the caller's responsibility to close the file object.

The **`open()`** function may be used in a **`with`** statement. When the **`with`** block completes, the **`Wave_read.close()`** or **`Wave_write.close()`** method is called.

Alterado na versão 3.4: Added support for unseekable files.

Alterado na versão 3.15: Added support for path-like objects and bytes-like objects.

exception wave.Error

An error raised when something is impossible because it violates the WAV specification or hits an implementation deficiency.

wave.WAVE_FORMAT_PCM

Format code for uncompressed PCM audio.

wave.WAVE_FORMAT_IEEE_FLOAT

Format code for IEEE floating-point audio.

wave.WAVE_FORMAT_EXTENSIBLE

Format code for WAVE extensible headers.

Objetos Wave_read

class wave.Wave_read

Read a WAV file.

Wave_read objects, as returned by **`open()`**, have the following methods:

close()

Close the stream if it was opened by **`wave`**, and make the instance unusable. This is called automatically on object collection.

getnchannels()

Returns number of audio channels (`1` for mono, `2` for stereo).

getsampwidth()

Retorna a largura da amostra em bytes.

getframerate()

Retorna a frequência de amostragem.

getnframes()

Retorna o número de quadros de áudio.

getformat()

Returns the frame format code.

This is one of **`WAVE_FORMAT_PCM`**, **`WAVE_FORMAT_IEEE_FLOAT`**, or **`WAVE_FORMAT_EXTENSIBLE`**.

Adicionado na versão 3.15.

getcomptype()

Returns compression type (`'NONE'` is the only supported type).

getcompname()

Human-readable version of **`getcomptype()`**. Usually `'not compressed'` parallels `'NONE'`.

getparams()

Returns a **`namedtuple()`** `(nchannels, sampwidth, framerate, nframes, comptype, compname)`, equivalent to output of the `get*()` methods.

readframes(*n*)

Reads and returns at most *n* frames of audio, as a **`bytes`** object.

rewind()

Volta o ponteiro do arquivo para o início do fluxo de áudio.

Os dois métodos a seguir definem um termo “posição” que é compatível entre eles e é dependente da implementação.

setpos(*pos*)
Define o ponteiro de arquivo para a posição especificada.

tell()
Retorna a posição atual do ponteiro de arquivo.

Objetos Wave_write

class wave.Wave_write
Escreve um arquivo WAV.

Objetos Wave_write, como retornados por `open()`.

For seekable output streams, the `wave` header will automatically be updated to reflect the number of frames actually written. For unseekable streams, the *nframes* value must be accurate when the first frame data is written. An accurate *nframes* value can be achieved either by calling `setnframes()` or `setparams()` with the number of frames that will be written before `close()` is called and then using `writeframesraw()` to write the frame data, or by calling `writeframes()` with all of the frame data to be written. In the latter case **`writeframes()`** will calculate the number of frames in the data and set *nframes* accordingly before writing the frame data.

Alterado na versão 3.4: Added support for unseekable files.

Wave_write objects have the following methods:

close()
Make sure *nframes* is correct, and close the file if it was opened by **`wave`**. This method is called upon object collection. It will raise an

exception if the output stream is not seekable and *nframes* does not match the number of frames actually written.

setnchannels(*n*)

Define o número de canais.

getnchannels()

Return the number of channels.

setsampwidth(*n*)

Set the sample width to *n* bytes.

For `WAVE_FORMAT_IEEE_FLOAT`, only 4-byte (32-bit) and 8-byte (64-bit) sample widths are supported.

getsampwidth()

Return the sample width in bytes.

setframerate(*n*)

Set the frame rate to *n*.

Alterado na versão 3.2: A non-integral input to this method is rounded to the nearest integer.

getframerate()

Return the frame rate.

setnframes(*n*)

Set the number of frames to *n*. This will be changed later if the number of frames actually written is different (this update attempt will raise an error if the output stream is not seekable).

getnframes()

Return the number of audio frames written so far.

setcomptype(*type*, *name*)

Set the compression type and description. At the moment, only compression type `NONE` is supported, meaning no compression.

getcomptype()

Return the compression type (`'NONE'`).

getcompname()

Return the human-readable compression type name.

setformat(*format*)

Set the frame format code.

Supported values are **`WAVE_FORMAT_PCM`** and **`WAVE_FORMAT_IEEE_FLOAT`**.

When setting **`WAVE_FORMAT_IEEE_FLOAT`**, the sample width must be 4 or 8 bytes.

getformat()

Return the current frame format code.

Adicionado na versão 3.15.

setparams(*tuple*)

The *tuple* should be `(nchannels, sampwidth, framerate, nframes, comptype, compname, format)`, with values valid for the `set*()` methods. Sets all parameters.

For backwards compatibility, a 6-item tuple without *format* is also accepted and defaults to **`WAVE_FORMAT_PCM`**.

For `format=WAVE_FORMAT_IEEE_FLOAT`, *sampwidth* must be 4 or 8.

getparams()

Return a **`namedtuple()`** `(nchannels, sampwidth, framerate, nframes, comptype, compname)` containing the current output parameters.

tell()

Return current position in the file, with the same disclaimer for the `Wave_read.tell()` and `Wave_read.setpos()` methods.

writeframesraw(*data*)

Escreve quadros de áudio, sem corrigir *nframes*.

Alterado na versão 3.4: Todo objeto bytes ou similar agora é aceito.

writeframes(*data*)

Write audio frames and make sure *nframes* is correct. It will raise an error if the output stream is not seekable and the total number of frames that have been written after *data* has been written does not match the previously set value for *nframes*.

Alterado na versão 3.4: Todo objeto bytes ou similar agora é aceito.

Note that it is invalid to set any parameters after calling **`writeframes()`** or `writeframesraw()`, and any attempt to do so will raise `wave.Error`.

For `WAVE_FORMAT_IEEE_FLOAT` output, a `fact` chunk is written as required by the WAVE specification for non-PCM formats.

`colorsys` — Conversões entre sistemas de cores

Código-fonte: Lib/colorsys.py
[https://github.com/python/cpython/tree/3.15/Lib/colorsys.py]

O módulo **`colorsys`** define conversões bidirecionais de valores de cores entre cores expressas no espaço de cores RGB (Red Green Blue) usado em monitores de computador e três outros sistemas de coordenadas: YIQ, HLS (Hue Lightness Saturation) e HSV (Hue Saturation Value). As coordenadas em todos esses espaços de cores são valores de ponto flutuante. No espaço YIQ, a coordenada Y está entre 0 e 1, mas as coordenadas I e Q podem ser positivas ou negativas. Em todos os outros espaços, as coordenadas estão todas entre 0 e 1.

Ver também

Mais informações sobre espaços de cores podem ser encontradas em https://www.poynton.ca/pdf/ColourFAQ.pdf e https://www.cambridgeincolour.com/tutorials/color-spaces.htm.

O módulo **`colorsys`** define as seguintes funções:

colorsys.rgb_to_yiq(*r*, *g*, *b*)
Converte a cor de coordenadas RGB para coordenadas YIQ.

colorsys.yiq_to_rgb(*y*, *i*, *q*)
Converte a cor de coordenadas YIQ para coordenadas RGB.

colorsys.rgb_to_hls(*r*, *g*, *b*)
Converte a cor de coordenadas RGB para coordenadas HLS.

colorsys.hls_to_rgb(*h*, *l*, *s*)

Converte a cor de coordenadas HLS para coordenadas RGB.

colorsys.rgb_to_hsv(*r*, *g*, *b*)

Converte a cor de coordenadas RGB para coordenadas HSV.

colorsys.hsv_to_rgb(*h*, *s*, *v*)

Converte a cor de coordenadas HSV para coordenadas RGB.

Exemplo:

```
>>> import colorsys
>>> colorsys.rgb_to_hsv(0.2, 0.4, 0.4)
(0.5, 0.5, 0.4)
>>> colorsys.hsv_to_rgb(0.5, 0.5, 0.4)
(0.2, 0.4, 0.4)
```

Internacionalização

Os módulos descritos neste capítulo ajudam você a criar um software que é independente de idioma e localidade, fornecendo mecanismos para selecionar um idioma a ser usado em mensagens de programa ou adaptando a saída para corresponder às convenções locais.

A lista de módulos descritos neste capítulo é:

- `gettext` — Serviços de internacionalização multilíngues
  - API do GNU **gettext**
  - API baseada em classe
    - A classe `NullTranslations`
    - A classe `GNUTranslations`
    - Suporte a catálogo de mensagens do Solaris
    - O construtor Catalog
  - Internacionalizando seus programas e módulos
    - Localizando seu módulo
    - Localizando sua aplicação
    - Alterando os idiomas durante o uso
    - Traduções adiadas
  - Reconhecimentos
- `locale` — Serviços de internacionalização
  - Histórico, detalhes, dicas, dicas e advertências
  - Nomes de localidades
  - Para escritores de extensão e programas que incorporam Python
  - Acesso a catálogos de mensagens

`gettext` — Serviços de internacionalização multilíngues

Código-fonte: Lib/gettext.py [https://github.com/python/cpython/tree/3.15/Lib/gettext.py]

O módulo **`gettext`** fornece serviços de internacionalização (I18N) e localização (L10N) para seus módulos e aplicativos Python. Ele suporta a API do catálogo de mensagens GNU **gettext** e uma API baseada em classes de nível mais alto que podem ser mais apropriadas para arquivos Python. A interface descrita abaixo permite gravar o módulo e as mensagens do aplicativo em um idioma natural e fornecer um catálogo de mensagens traduzidas para execução em diferentes idiomas naturais.

Algumas dicas sobre localização de seus módulos e aplicativos Python também são fornecidas.

API do GNU gettext

O módulo **`gettext`** define a API a seguir, que é muito semelhante à API do GNU **gettext**. Se você usar esta API, você afetará a tradução de todo o seu aplicativo globalmente. Geralmente, é isso que você deseja se o seu aplicativo for monolíngue, com a escolha do idioma dependente da localidade do seu usuário. Se você estiver localizando um módulo Python, ou se seu aplicativo precisar alternar idiomas rapidamente, provavelmente desejará usar a API baseada em classe.

gettext.bindtextdomain(*domain*, *localedir=None*)

Liga o *domain* ao diretório de localidade *localedir*. Mais concretamente, **`gettext`** procurará arquivos binários `.mo` para o domínio especificado usando o caminho (no Unix): *`localedir`*`/`*`language`*`/LC_MESSAGES/`*`domain`*`.mo`, sendo *language*

pesquisado nas variáveis de ambiente **LANGUAGE**, **LC_ALL**, **LC_MESSAGES** e **LANG** respectivamente.

Se *localedir* for omitido ou `None`, a ligação atual para *domain* será retornada. [[1]]

gettext.textdomain(*domain=None*)

Altera ou consulta o domínio global atual. Se *domain* for `None`, o domínio global atual será retornado; caso contrário, o domínio global será definido como *domain*, o qual será retornado.

gettext.gettext(*message*, /)

Retorna a tradução localizada de *message*, com base no diretório global atual de domínio, idioma e localidade. Essa função geralmente é apelidada como **_()** no espaço de nomes local (veja exemplos abaixo).

gettext.dgettext(*domain*, *message*, /)

Semelhante a **gettext()**, mas procura a mensagem no *domain* especificado.

gettext.ngettext(*singular*, *plural*, *n*, /)

Semelhante a **gettext()**, mas considera formas plurais. Se uma tradução for encontrada, aplica a fórmula do plural a *n* e retorne a mensagem resultante (alguns idiomas têm mais de duas formas no plural). Se nenhuma tradução for encontrada, retorna *singular* se *n* for 1; retorna *plural* caso contrário.

A fórmula de Plural é retirada do cabeçalho do catálogo. É uma expressão C ou Python que possui uma variável livre *n*; a expressão é avaliada para o índice do plural no catálogo. Veja a documentação do gettext GNU [https://www.gnu.org/software/gettext/manual/gettext.html] para obter a sintaxe precisa a ser usada em arquivos `.po` e as fórmulas para um variedade de idiomas.

gettext.dngettext(*domain*, *singular*, *plural*, *n*, /)

Semelhante a **ngettext()**, mas procura a mensagem no *domain* especificado.

gettext.pgettext(*context*, *message*, /)

gettext.dpgettext(*domain*, *context*, *message*, /)

gettext.npgettext(*context*, *singular*, *plural*, *n*, /)

gettext.dnpgettext(*domain*, *context*, *singular*, *plural*, *n*, /)

Semelhante às funções correspondentes sem o `p` no prefixo (ou seja, **gettext()**, **dgettext()**, **ngettext()**, **dngettext()**), mas a tradução é restrita ao *context* de mensagem fornecido.

Adicionado na versão 3.8.

Note que GNU **gettext** também define um método **dcgettext()**, mas isso não foi considerado útil e, portanto, atualmente não está implementado.

Aqui está um exemplo de uso típico para esta API:

```
import gettext
gettext.bindtextdomain('minhaaplicacao', '/caminho/para/diretóri
gettext.textdomain('minhaaplicacao')
_ = gettext.gettext
# ...
print(_('Esta é uma string traduzível.'))
```

API baseada em classe

A API baseada em classe do módulo **gettext** oferece mais flexibilidade e maior conveniência do que a API do GNU **gettext**. É a maneira recomendada de localizar suas aplicações e módulos Python. **gettext** define uma classe **GNUTranslations** que implementa a análise de arquivos no formato GNU `.mo` e possui métodos para retornar strings. Instâncias dessa classe também podem se instalar no espaço de nomes embutido como a função **_()**.

gettext.find(*domain*, *localedir=None*, *languages=None*, *all=False*)

Esta função implementa o algoritmo de busca de arquivos `.mo` padrão. É necessário um *domain*, idêntico ao que **`textdomain()`** leva. *localedir* opcional é como em **`bindtextdomain()`**. *languages* opcional é uma lista de strings, em que cada string é um código de idioma.

Se *localedir* não for fornecido, o diretório local do sistema padrão será usado. [[2]] Se *languages* não for fornecido, as seguintes variáveis de ambiente serão pesquisadas: **`LANGUAGE`**, **`LC_ALL`**, **`LC_MESSAGES`** e **`LANG`**. O primeiro retornando um valor não vazio é usado para a variável *languages*. As variáveis de ambiente devem conter uma lista de idiomas separada por dois pontos, que será dividida nos dois pontos para produzir a lista esperada de strings de código de idioma.

`find()` expande e normaliza os idiomas e itera através deles, procurando por um arquivo existente construído com esses componentes:

`localedir/language/``LC_MESSAGES/`*`domain`*`.mo`

O primeiro nome de arquivo existente é retornado por **`find()`**. Se nenhum desses arquivos for encontrado, será retornado `None`. Se *all* for fornecido, ele retornará uma lista de todos os nomes de arquivos, na ordem em que aparecem na lista de idiomas ou nas variáveis de ambiente.

gettext.translation(*domain*, *localedir=None*, *languages=None*, *class_=None*, *fallback=False*)

Retorna uma instância de `*Translations` com base nos *domain*, *localedir* e *languages*, que são passados primeiro para **`find()`** para obter uma lista dos caminhos de arquivos `.mo` associados. Instâncias com nomes de arquivo idênticos `.mo` são armazenados em cache. A classe atual instanciada é *class_* se fornecida, caso contrário **`GNUTranslations`**. O construtor da classe deve usar um único argumento objeto arquivo.

Se vários arquivos forem encontrados, os arquivos posteriores serão usados como fallbacks para os anteriores. Para permitir a configuração do fallback, `copy.copy()` é usado para clonar cada objeto de conversão do cache; os dados reais da instância ainda são compartilhados com o cache.

Se nenhum arquivo `.mo` for encontrado, essa função levanta `OSError` se *fallback* for falso (que é o padrão) e retorna uma instância `NullTranslations` se *fallback* for verdadeiro.

Alterado na versão 3.3: `IOError` costumava ser levantada, agora ele é um apelido para `OSError`.

Alterado na versão 3.11: O parâmetro *codeset* foi removido.

gettext.install(*domain*, *localedir=None*, *, *names=None*)

Isso instala a função `_()` no espaço de nomes interno do Python, com base em *domain* e *localedir* que são passados para a função `translation()`.

Para o parâmetro *names*, por favor, veja a descrição do método `install()` do objeto de tradução.

Como visto abaixo, você normalmente marca as strings candidatas à tradução em sua aplicação, envolvendo-as em uma chamada para a função `_()`, assim:

```
print(_('Esta string será traduzida.'))
```

Por conveniência, você deseja que a função `_()` seja instalada no espaço de nomes interno do Python, para que seja facilmente acessível em todos os módulos do sua aplicação.

Alterado na versão 3.11: names é agora um parâmetro somente-nomeado.

A classe `NullTranslations`

As classes de tradução são o que realmente implementa a tradução de strings de mensagens do arquivo-fonte original para strings de mensagens traduzidas. A classe base usada por todas as classes de tradução é **`NullTranslations`**; isso fornece a interface básica que você pode usar para escrever suas próprias classes de tradução especializadas. Aqui estão os métodos de **`NullTranslations`**:

class gettext.NullTranslations(*fp=None*)

Recebe um objeto arquivo opcional *fp*, que é ignorado pela classe base. Inicializa as variáveis de instância "protegidas" *_info* e *_charset*, que são definidas por classes derivadas, bem como *_fallback*, que é definido através de **`add_fallback()`**. Ele então chama `self._parse(fp)` se *fp* não for `None`.

_parse(*fp*)

No-op na classe base, esse método pega o objeto arquivo *fp* e lê os dados do arquivo, inicializando seu catálogo de mensagens. Se você tiver um formato de arquivo de catálogo de mensagens não suportado, substitua esse método para analisar seu formato.

add_fallback(*fallback*)

Adiciona *fallback* como o objeto reserva para o objeto de tradução atual. Um objeto de tradução deve consultar o fallback se não puder fornecer uma tradução para uma determinada mensagem.

gettext(*message*, *...*)

Se um fallback tiver sido definido, encaminha **`gettext()`** para o fallback. Caso contrário, retorna *message*. Substituído em classes derivadas.

ngettext(*singular*, *plural*, *n*, *...*)

Se um fallback tiver sido definido, encaminha **`ngettext()`** para o fallback. Caso contrário, retorna *singular* se *n* for 1; do contrário, retorna *plural*. Substituído em classes derivadas.

pgettext(*context*, *message*, *...*)

Se um fallback tiver sido definido, encaminha `pgettext()` para o fallback. Caso contrário, retorna a mensagem traduzida. Substituído em classes derivadas.

Adicionado na versão 3.8.

npgettext(*context*, *singular*, *plural*, *n*, /)

Se um fallback tiver sido definido, encaminha `npgettext()` para o fallback. Caso contrário, retorna a mensagem traduzida. Substituído em classes derivadas.

Adicionado na versão 3.8.

info()

Retorna um dicionário que contém os metadados encontrados no arquivo de catálogo de mensagens.

charset()

Retorna a codificação do arquivo de catálogo de mensagens.

install(*names=None*)

Este método instala `gettext()` no espaço de nomes embutido, ligando-o a _.

Se o parâmetro *names* for fornecido, deve ser uma sequência contendo os nomes das funções que você deseja instalar no espaço de nomes embutidos, além de `_()`. Há suporte aos nomes `'gettext'`, `'ngettext'`, `'pgettext'` e `'npgettext'`

Observe que esta é apenas uma maneira, embora a maneira mais conveniente, de disponibilizar a função `_()` para sua aplicação. Como afeta a aplicação inteira globalmente, e especificamente o espaço de nomes embutido, os módulos localizados nunca devem instalar `_()`. Em vez disso, eles devem usar este código para disponibilizar `_()` para seu módulo:

```
import gettext
t = gettext.translation('meumódulo', ...)
_ = t.gettext
```

Isso coloca **_()** apenas no espaço de nomes global do módulo e, portanto, afeta apenas as chamadas dentro deste módulo.

Alterado na versão 3.8: Adicionado `'pgettext'` e `'npgettext'`.

A classe `GNUTranslations`

O módulo **gettext** fornece uma classe adicional derivada de `NullTranslations`: `GNUTranslations`. Esta classe substitui **_parse()** para permitir a leitura de arquivos `.mo` do formato GNU **gettext** nos formatos big-endian e little-endian.

`GNUTranslations` analisa metadados opcionais do catálogo de tradução. É uma convenção com o GNU **gettext** incluir metadados como tradução para a string vazia. Esses metadados estão nos pares `key: value` no estilo **RFC 822** [https://datatracker.ietf.org/doc/html/rfc822.html] e devem conter a chave `Project-Id-Version`. Se a chave `Content-Type` for encontrada, a propriedade `charset` será usada para inicializar a variável de instância **_charset** "protegida", com o padrão `None` se não for encontrada. Se a codificação de "charset" for especificada, todos os IDs e strings de mensagens lidos no catálogo serão convertidos em Unicode usando essa codificação, caso contrário, o ASCII será presumido.

Como os IDs de mensagens também são lidos como strings Unicode, todos os métodos `*gettext()` presumem os IDs de mensagens como sendo strings Unicode, não como strings de bytes.

Todo o conjunto de pares chave/valor é colocado em um dicionário e definido como a variável de instância **_info** "protegida".

Se o número mágico do arquivo `.mo` for inválido, o número principal da versão é inesperado ou se ocorrerem outros problemas durante a leitura do arquivo, instanciando uma classe `GNUTranslations` pode levantar `OSError`.

class gettext.GNUTranslations

Os seguintes métodos são substituídos a partir da implementação da classe base:

gettext(*message*, /)

Procura o ID da *message* no catálogo e retorna a string de mensagens correspondente, como uma string Unicode. Se não houver entrada no catálogo para o ID da *message* e um fallback tiver sido definido, a pesquisa será encaminhada para o método `gettext()` do fallback. Caso contrário, o ID da *message* é retornado.

ngettext(*singular*, *plural*, *n*, /)

Faz uma pesquisa de plural-forms de um ID de mensagem. *singular* é usado como o ID da mensagem para fins de pesquisa no catálogo, enquanto *n* é usado para determinar qual forma plural usar. A string de mensagens retornada é uma string Unicode.

Se o ID da mensagem não for encontrado no catálogo e um fallback for especificado, a solicitação será encaminhada para o método do fallback `ngettext()`. Caso contrário, quando *n* for 1, *singular* será retornado e *plural* será retornado em todos os outros casos.

Aqui está um exemplo:

```
n = len(os.listdir('.'))
cat = GNUTranslations(somefile)
message = cat.ngettext(
    'There is %(num)d file in this directory',
    'There are %(num)d files in this directory',
    n) % {'num': n}
```

pgettext(*context*, *message*, /)

Procura o ID do *context* e da *message* no catálogo e retorna a string de mensagens correspondente, como uma string Unicode. Se não houver entrada no catálogo para o ID do *context* e da *message*, e um fallback tiver sido definido, a pesquisa será encaminhada para o

método `pgettext()` do fallback. Caso contrário, o ID da *message* é retornado.

Adicionado na versão 3.8.

npgettext(*context*, *singular*, *plural*, *n*, /)

Faz uma pesquisa de plural-forms de um ID de mensagem. *singular* é usado como o ID da mensagem para fins de pesquisa no catálogo, enquanto *n* é usado para determinar qual forma plural usar.

Se o ID da mensagem para *context* não for encontrado no catálogo e um fallback for especificado, a solicitação será encaminhada para o método `npgettext()` do fallback. Caso contrário, quando *n* for 1, *singular* será retornado e *plural* será retornado em todos os outros casos.

Adicionado na versão 3.8.

Suporte a catálogo de mensagens do Solaris

O sistema operacional Solaris define seu próprio formato de arquivo binário `.mo`, mas como nenhuma documentação pode ser encontrada nesse formato, ela não é suportada no momento.

O construtor Catalog

O GNOME usa uma versão do módulo **`gettext`** de James Henstridge, mas esta versão tem uma API um pouco diferente. Seu uso documentado foi:

```
import gettext
cat = gettext.Catalog(domain, localedir)
_ = cat.gettext
print(_('hello world'))
```

Para compatibilidade com este módulo mais antigo, a função **`Catalog()`** é um apelido para a função `translation()` descrita acima.

Uma diferença entre este módulo e o de Henstridge: seus objetos de catálogo suportavam o acesso por meio de uma API de mapeamento, mas isso parece não ser utilizado e, portanto, não é atualmente suportado.

Internacionalizando seus programas e módulos

Internationalization (I18N), ou internacionalização (I17O) em português, refere-se à operação pela qual um programa é informado sobre vários idiomas. Localization (L10N), ou localização em português, refere-se à adaptação do seu programa, uma vez internacionalizado, aos hábitos culturais e de idioma local. Para fornecer mensagens multilíngues para seus programas Python, você precisa executar as seguintes etapas:

1. preparar seu programa ou módulo especialmente marcando strings traduzíveis
2. executar um conjunto de ferramentas nos arquivos marcados para gerar catálogos de mensagens não tratadas
3. criar traduções específicas do idioma dos catálogos de mensagens
4. usar o módulo **`gettext`** para que as strings das mensagens sejam traduzidas corretamente

Para preparar seu código para I18N, você precisa examinar todas as strings em seus arquivos. Qualquer string que precise ser traduzida deve ser marcada envolvendo-a em `_('...')` — isto é, uma chamada para a função `_`. Por exemplo:

```
filename = 'meulog.txt'
message = _('escrevendo uma mensagem de log')
with open(filename, 'w') as fp:
    fp.write(message)
```

Neste exemplo, a string `'escrevendo uma mensagem de log'` está marcada como um candidato para tradução, enquanto as strings `'meulog.txt'` e `'w'` não estão.

Existem algumas ferramentas para extrair as strings destinadas à tradução. O GNU **gettext** original tem suporte apenas ao código-fonte C ou C++, mas sua versão estendida **xgettext** varre o código escrito em várias linguagens,

incluindo Python, para encontrar strings marcadas como traduzíveis. Babel [https://babel.pocoo.org/] é uma biblioteca de internacionalização do Python que inclui um script `pybabel` para extrair e compilar catálogos de mensagens. O programa de François Pinard chamado **xpot** faz um trabalho semelhante e está disponível como parte de seu pacote po-utils [https://github.com/pinard/po-utils].

(O Python também inclui versões em Python puro desses programas, chamadas **pygettext.py** e **msgfmt.py**; algumas distribuições do Python as instalam para você. O **pygettext.py** é semelhante ao **xgettext**, mas apenas entende o código-fonte do Python e não consegue lidar com outras linguagens de programação como C ou C++. O **pygettext.py** possui suporte a uma interface de linha de comando semelhante à do **xgettext**; para detalhes sobre seu uso, execute `pygettext.py --help`. O **msgfmt.py** é compatível com binários com GNU **msgfmt**. Com esses dois programas, você pode não precisar do pacote GNU **gettext** para internacionalizar suas aplicações Python.)

xgettext, **pygettext** e ferramentas similares geram `.po` que são catálogos de mensagens. Eles são arquivos legíveis por humanos estruturados que contêm todas as strings marcadas no código-fonte, junto com um espaço reservado para as versões traduzidas dessas strings.

Cópias destes arquivos `.po` são entregues aos tradutores humanos individuais que escrevem traduções para todos os idiomas naturais suportados. Eles enviam de volta as versões completas específicas do idioma como um arquivo `<nome-do-idioma>.po` que é compilado em um arquivo de catálogo binário legível por máquina `.mo` usando o programa **msgfmt**. Os arquivos `.mo` são usados pelo módulo **`gettext`** para o processamento de tradução real em tempo de execução.

Como você usa o módulo **`gettext`** no seu código depende se você está internacionalizando um único módulo ou sua aplicação inteira. As próximas duas seções discutirão cada caso.

Localizando seu módulo

Se você estiver localizando seu módulo, tome cuidado para não fazer alterações globais, por exemplo para o espaço de nomes embutidos. Você não deve usar a API GNU **gettext**, mas a API baseada em classe.

Digamos que seu módulo seja chamado “spam” e as várias traduções do idioma natural do arquivo `.mo` residam em `/usr/share/locale` no formato GNU **gettext**. Aqui está o que você colocaria sobre o seu módulo:

```
import gettext
t = gettext.translation('spam', '/usr/share/locale')
_ = t.gettext
```

Localizando sua aplicação

Se você estiver localizando sua aplicação, poderá instalar a função **`_()`** globalmente no espaço de nomes embutidos, geralmente no arquivo principal do driver do sua aplicação. Isso permitirá que todos os arquivos específicos de sua aplicação usem `_('...')` sem precisar instalá-la explicitamente em cada arquivo.

No caso simples, você precisa adicionar apenas o seguinte código ao arquivo do driver principal da sua aplicação:

```
import gettext
gettext.install('minhaaplicacao')
```

Se você precisar definir o diretório da localidade, poderá passá-lo para a função **`install()`**:

```
import gettext
gettext.install('minhaaplicacao', '/usr/share/locale')
```

Alterando os idiomas durante o uso

Se o seu programa precisar oferecer suporte a vários idiomas ao mesmo tempo, convém criar várias instâncias de tradução e alternar entre elas explicitamente, assim:

```
import gettext

lang1 = gettext.translation('minhaaplicacao', languages=['en'])
lang2 = gettext.translation('minhaaplicacao', languages=['fr'])
lang3 = gettext.translation('minhaaplicacao', languages=['de'])

# começa usando o idioma 1
lang1.install()

# ... o tempo passa, o usuário seleciona o idioma 2
lang2.install()

# ... mais tempo passa, o usuário seleciona o idioma 3
lang3.install()
```

Traduções adiadas

Na maioria das situações de codificação, as strings são traduzidas onde são codificadas. Ocasionalmente, no entanto, é necessário marcar strings para tradução, mas adiar a tradução real até mais tarde. Um exemplo clássico é:

```
animals = ['mollusk',
           'albatross',
           'rat',
           'penguin',
           'python', ]
# ...
for a in animals:
    print(a)
```

Aqui, você deseja marcar as strings na lista `animals` como traduzíveis, mas na verdade não deseja traduzi-las até que sejam impressas.

Aqui está uma maneira de lidar com esta situação:

```
def _(message): return message

animals = [_('mollusk'),
           _('albatross'),
           _('rat'),
           _('penguin'),
```

```
           _('python'), ]

del _

# ...
for a in animals:
    print(_(a))
```

Isso funciona porque a definição fictícia de **`_()`** simplesmente retorna a string inalterada. E essa definição fictícia vai substituir temporariamente qualquer definição de **`_()`** no espaço de nomes embutido (até o comando **`del`**). Tome cuidado, se você tiver uma definição anterior de **`_()`** no espaço de nomes local.

Observe que o segundo uso de **`_()`** não identificará "a" como traduzível para o programa **gettext**, porque o parâmetro não é uma string literal.

Outra maneira de lidar com isso é com o seguinte exemplo:

```
def N_(message): return message

animals = [N_('mollusk'),
           N_('albatross'),
           N_('rat'),
           N_('penguin'),
           N_('python'), ]

# ...
for a in animals:
    print(_(a))
```

Nesse caso, você está marcando strings traduzíveis com a função **`N_()`**, que não entra em conflito com nenhuma definição de **`_()`**. No entanto, você precisará ensinar seu programa de extração de mensagens a procurar strings traduzíveis marcadas com **`N_()`**. **xgettext**, **pygettext**, `pybabel extract` e **xpot** possuem suporte a isso através do uso da opção de linha de comando **`-k`**. A escolha de **`N_()`** aqui é totalmente arbitrária; poderia facilmente ter sido **`MarkThisStringForTranslation()`**.

Reconhecimentos

As seguintes pessoas contribuíram com código, feedback, sugestões de design, implementações anteriores e experiência valiosa para a criação deste módulo:

- Peter Funk
- James Henstridge
- Juan David Ibáñez Palomar
- Marc-André Lemburg
- Martin von Löwis
- François Pinard
- Barry Warsaw
- Gustavo Niemeyer

Notas de rodapé

[1]

O diretório de localidade padrão depende do sistema; por exemplo, no Red Hat Linux é `/usr/share/locale`, mas no Solaris é `/usr/lib/locale`. O módulo **`gettext`** não tenta dar suporte a esses padrões dependentes do sistema; em vez disso, seu padrão é *`sys.base_prefix`*`/share/locale` (consulte **`sys.base_prefix`**). Por esse motivo, é sempre melhor chamar **`bindtextdomain()`** com um caminho absoluto explícito no início da sua aplicação.

[2]

Consulte a nota de rodapé para a **`bindtextdomain()`** acima.

`locale` — Serviços de internacionalização

Código-fonte: Lib/locale.py [https://github.com/python/cpython/tree/3.15/Lib/locale.py]

O módulo **`locale`** abre o acesso ao banco de dados de localidade POSIX e funcionalidade. O mecanismo de localidade POSIX permite que os programadores lidem com certas questões culturais em uma aplicação, sem exigir que o programador conheça todas as especificidades de cada país onde o software é executado.

O módulo **`locale`** é implementado em cima do módulo **`_locale`**, que por sua vez usa uma implementação a localidade ANSI C se disponível.

O módulo **`locale`** define a seguinte exceção e funções:

exception locale.Error

Exceção levantada quando a localidade passada para **`setlocale()`** não é reconhecida.

locale.setlocale(*category*, *locale=None*)

Se *locale* for fornecido e não `None`, **`setlocale()`** modifica a configuração de locale para a *category*. As categorias disponíveis estão listadas na descrição dos dados abaixo. *locale* pode ser uma string ou um par, código de idioma e codificação. Uma string vazia especifica as configurações padrão do usuário. Se a modificação da localidade falhar, a exceção **`Error`** é levantada. Se for bem-sucedido, a nova configuração de localidade será retornada.

Se *locale* for um par, ele será convertido em um nome de localidade usando o mecanismo de apelidamento de localidade. O código de idioma tem o mesmo formato de nome da localidade, mas sem codificação. O código de idioma e a codificação podem ser `None`.

Se *locale* for omitido ou `None`, a configuração atual para *category* é retornada.

Exemplo:

```
>>> import locale
>>> loc = locale.setlocale(locale.LC_ALL)  # obtém localidade
# usa localidade de alemão; nome e disponibilidade pode varia
>>> locale.setlocale(locale.LC_ALL, 'de_DE.UTF-8')
>>> locale.strcoll('f\xe4n', 'foo')  # compara uma string cor
>>> locale.setlocale(locale.LC_ALL, '')   # usa a localidade
>>> locale.setlocale(locale.LC_ALL, 'C')  # usa localidade pa
>>> locale.setlocale(locale.LC_ALL, loc)  # restaura localida
```

`setlocale()` não é seguro para thread na maioria dos sistemas. As aplicações normalmente começam com uma chamada de:

```
import locale
locale.setlocale(locale.LC_ALL, '')
```

Isso define a localidade de todas as categorias para a configuração padrão do usuário (normalmente especificada na variável de ambiente **`LANG`**). Se a localidade não for alterada depois disso, o uso de multithreading não deve causar problemas.

Alterado na versão 3.15: Oferece suporte a códigos de idiomas com modificadores `@`.

locale.localeconv()

Retorna o banco de dados das convenções locais como um dicionário. Este dicionário possui as seguintes strings como chaves:

Categoria	Chave	Significado
`LC_NUMERIC`	`'decimal_point'`	Caractere de ponto decimal.

Categoria	Chave	Significado
	`'grouping'`	Sequência de números especificando quais posições relativas os `'thousands_sep'` são esperados. Se a sequência for terminada com **`CHAR_MAX`**, nenhum agrupamento adicional é realizado. Se a sequência termina com um `0`, o tamanho do último grupo é usado repetidamente.
	`'thousands_sep'`	Caractere usado entre grupos.
`LC_MONETARY`	`'int_curr_symbol'`	Símbolo internacional de moeda.
	`'currency_symbol'`	Símbolo local de moeda.

Categoria	Chave	Significado
	`'p_cs_precedes/n_cs_precedes'`	Se o símbolo da moeda precede o valor (para valores positivos ou negativos).
	`'p_sep_by_space/n_sep_by_space'`	Se o símbolo monetário está separado do valor por um espaço (para valores positivos ou negativos).
	`'mon_decimal_point'`	Ponto decimal usado para valores monetários.
	`'frac_digits'`	Número de dígitos fracionários usados na formatação local de valores monetários.

Categoria	Chave	Significado
	`'int_frac_digits'`	Número de dígitos fracionários usados na formatação internacional de valores monetários.
	`'mon_thousands_sep'`	Separador de grupo usado para valores monetários.
	`'mon_grouping'`	Equivalente a `'grouping'`, usado para valores monetários.
	`'positive_sign'`	Símbolo usado para anotar um valor monetário positivo.
	`'negative_sign'`	Símbolo usado para anotar um valor monetário negativo.

Categoria	Chave	Significado
	`'p_sign_posn/n_sign_posn'`	A posição do sinal (para valores positivos resp. negativos), veja abaixo.

Todos os valores numéricos podem ser definidos como **`CHAR_MAX`** para indicar que não há valor especificado nesta localidade.

Os valores possíveis para `'p_sign_posn'` e `'n_sign_posn'` são dados abaixo.

Valor	Explicação
`0`	A moeda e o valor estão entre parênteses.
`1`	O sinal deve preceder o valor e o símbolo da moeda.
`2`	O sinal deve seguir o valor e o símbolo da moeda.
`3`	O sinal deve preceder imediatamente o valor.
`4`	O sinal deve seguir imediatamente o valor.
`CHAR_MAX`	Nada é especificado nesta localidade.

A função define temporariamente a localidade de `LC_CTYPE` para a localidade de `LC_NUMERIC` ou a localidade de `LC_MONETARY` se as localidades forem diferentes e as strings numéricas ou monetárias não forem ASCII. Esta mudança temporária afeta outras threads.

Alterado na versão 3.7: A função agora define temporariamente a localidade de `LC_CTYPE` para a localidade de `LC_NUMERIC` em alguns casos.

locale.nl_langinfo(*option*)

Retorna algumas informações específicas da localidade em uma string. Esta função não está disponível em todos os sistemas e o conjunto de opções possíveis também pode variar entre as plataformas. Os valores de argumento possíveis são números, para os quais constantes simbólicas estão disponíveis no módulo da localidade.

A função `nl_langinfo()` aceita uma das seguintes chaves. A maioria das descrições são tiradas da descrição correspondente na biblioteca GNU C.

locale.CODESET

Obtém uma string com o nome da codificação de caracteres usada na localidade selecionado.

locale.D_T_FMT

Obtém uma string que pode ser usada como uma string de formato para `time.strftime()` para representar a data e a hora de uma maneira específica da localidade.

locale.D_FMT

Obtém uma string que pode ser usada como uma string de formato para `time.strftime()` para representar uma data de uma maneira específica da localidade.

locale.T_FMT

Obtém uma string que pode ser usada como uma string de formato para `time.strftime()` para representar uma hora de uma maneira específica da localidade.

locale.T_FMT_AMPM

Obtém uma string de formato para `time.strftime()` para representar a hora no formato am/pm.

locale.DAY_1
locale.DAY_2
locale.DAY_3

locale.DAY_4
locale.DAY_5
locale.DAY_6
locale.DAY_7

Obtém o nome do enésimo dia da semana.

Nota

Isso segue a convenção dos EUA de `DAY_1` ser no domingo, não a convenção internacional (ISO 8601) que segunda-feira é o primeiro dia da semana.

locale.ABDAY_1
locale.ABDAY_2
locale.ABDAY_3
locale.ABDAY_4
locale.ABDAY_5
locale.ABDAY_6
locale.ABDAY_7

Obtém o nome abreviado do enésimo dia da semana.

locale.MON_1
locale.MON_2
locale.MON_3
locale.MON_4
locale.MON_5
locale.MON_6
locale.MON_7
locale.MON_8
locale.MON_9
locale.MON_10
locale.MON_11
locale.MON_12

Obtém o nome do enésimo dia do mês.

locale.ABMON_1
locale.ABMON_2

locale.ABMON_3
locale.ABMON_4
locale.ABMON_5
locale.ABMON_6
locale.ABMON_7
locale.ABMON_8
locale.ABMON_9
locale.ABMON_10
locale.ABMON_11
locale.ABMON_12

Obtém o nome abreviado do enésimo dia do mês.

locale.RADIXCHAR

Obtém o caractere separador decimal (ponto decimal, vírgula decimal etc.).

locale.THOUSEP

Obtém o caractere separador para milhares (grupos de três dígitos).

locale.YESEXPR

Obtém uma expressão regular que pode ser usada com a função regex para reconhecer uma resposta positiva a uma pergunta sim/não.

locale.NOEXPR

Obtém uma expressão regular que pode ser usada com a função `regex(3)` para reconhecer uma resposta negativa a uma pergunta sim/não.

Nota

As expressões regulares para **`YESEXPR`** e **`NOEXPR`** usam uma sintaxe adequada para a função `regex` da biblioteca C, que pode ser diferente da sintaxe usada em **`re`**.

locale.CRNCYSTR

Obtém o símbolo da moeda, precedido por “-” se o símbolo deve aparecer antes do valor, “+” se o símbolo deve aparecer após o valor ou “.” se o símbolo deve substituir o caractere separador decimal.

locale.ERA

Obtém uma string que descreva como os anos são contados e exibidos para cada era em um localidade.

A maioria das localidades não define esse valor. Um exemplo de localidade que define esse valor é o japonês. No Japão, a representação tradicional de datas inclui o nome da época correspondente ao reinado do então imperador.

Normalmente, não deve ser necessário usar esse valor diretamente. Especificar o modificador `E` em suas strings de formato faz com que a função `time.strftime()` use essas informações. O formato da string retornada é especificado em *The Open Group Base Specifications Issue 8*, parágrafo 7.3.5.2 LC_TIME C-Language Access [https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap07.html#tag_07_03_05_02].

locale.ERA_D_T_FMT

Obtém uma string de formato para `time.strftime()` para representar a data e a hora de uma forma baseada na era específica da localidade.

locale.ERA_D_FMT

Obtém uma string de formato para `time.strftime()` para representar uma data em uma forma baseada em era específica da localidade.

locale.ERA_T_FMT

Obtém uma string de formato para `time.strftime()` para representar uma hora em uma forma baseada em era específica da localidade.

locale.ALT_DIGITS

Obtém uma string consistindo de até 100 símbolos separados por ponto e vírgula usados para representar os valores de 0 a 99 de uma forma específica de localidade. Na maioria das localidades, esta é uma string vazia.

A função define temporariamente a localidade `LC_CTYPE` para a localidade da categoria que determina o valor solicitado (`LC_TIME`, `LC_NUMERIC`, `LC_MONETARY` ou `LC_MESSAGES`) se as localidades forem diferentes e a string resultante não for ASCII. Essa alteração temporária afeta outras threads.

Alterado na versão 3.14: A função agora define temporariamente a localidade de `LC_CTYPE` em alguns casos.

locale.getdefaultlocale([*envvars*])

Tenta determinar as configurações de localidade padrão e as retorna como uma tupla na forma `(language code, encoding)`.

De acordo com POSIX, um programa que não chamou `setlocale(LC_ALL, '')` executa usando a localidade portátil `'C'`. Chamar `setlocale(LC_ALL, '')` permite que ele use a localidade padrão conforme definido pela variável **`LANG`**. Como não queremos interferir com a configuração de localidade atual, emulamos o comportamento da maneira descrita acima.

Para manter a compatibilidade com outras plataformas, não apenas a variável **`LANG`** é testada, mas uma lista de variáveis fornecida como parâmetro envvars. Será utilizado o primeiro encontrado a ser definido. *envvars* padroniza para o caminho de pesquisa usado no GNU gettext; deve sempre conter o nome da variável `'LANG'`. O caminho de pesquisa do GNU gettext contém `'LC_ALL'`, `'LC_CTYPE'`, `'LANG'` e `'LANGUAGE'`, nesta ordem.

O código de idioma tem o mesmo formato de nome de localidade, mas sem codificação e modificador `@`. O código de idioma e a codificação podem ser `None` se seus valores não puderem ser determinados. O locale “C” é representado como `(None, None)`.

locale.getlocale(*category=LC_CTYPE*)

Retorna a configuração atual para a categoria de localidade fornecida como uma tupla contendo o código de idioma e a codificação. *category* pode ser um dos valores **LC_***, exceto **LC_ALL**. O padrão é **LC_CTYPE**.

O código de idioma tem o mesmo formato de nome de localidade, mas sem codificação. O código de idioma e a codificação podem ser `None` se seus valores não puderem ser determinados. A localidade "C" é representada como `(None, None)`.

Alterado na versão 3.15: Modificadores `@` não são mais removidos silenciosamente, mas incluídos no código do idioma.

locale.getpreferredencoding(*do_setlocale=True*)

Retorna a codificação da localidade usada para dados de texto, de acordo com as preferências do usuário. As preferências do usuário são expressas de maneira diferente em sistemas diferentes e podem não estar disponíveis programaticamente em alguns sistemas, portanto, essa função retorna apenas uma estimativa.

Em alguns sistemas, é necessário invocar **setlocale()** para obter as preferências do usuário, portanto, esta função não é segura para thread. Se invocar setlocale não for necessário ou desejado, *do_setlocale* deve ser definido como `False`.

No Android ou se o Modo UTF-8 do Python é retornado, sempre retorna `'utf-8'`, a codificação da localidade e o argumento *do_setlocale* são ignorados.

A pré-inicialização do Python configura a localidade LC_CTYPE. Veja também tratador de erros e codificação do sistema de arquivos.

Alterado na versão 3.7: A função agora sempre retorna `"utf-8"` no Android ou se o Modo UTF-8 do Python estiver habilitado.

locale.getencoding()

Obtém a atual codificação da localidade:

- No Android e no VxWorks, retorna `"utf-8"`.
- No Unix, retorna a codificação da localidade **`LC_CTYPE`** atual. Retorna `"utf-8"` se `nl_langinfo(CODESET)` retornar uma string vazia: por exemplo, se a localidade LC_CTYPE atual não for compatível.
- No Windows, retorna a página de código ANSI.

A pré-inicialização do Python configura a localidade LC_CTYPE. Veja também tratador de erros e codificação do sistema de arquivos.

Este função é semelhante a **`getpreferredencoding(False)`**, exceto pelo fato de que esta função ignora o Modo UTF-8 do Python.

Adicionado na versão 3.11.

locale.normalize(*localename*)

Retorna um código de localidade normalizado para o nome de localidade fornecido. O código de localidade retornado é formatado para uso com **`setlocale()`**. Se a normalização falhar, o nome original será retornado inalterado.

Se a codificação fornecida não for conhecida, o padrão da função é a codificação padrão para o código da localidade, assim como **`setlocale()`**.

locale.strcoll(*string1*, *string2*)

Compara duas strings de acordo com a configuração atual **`LC_COLLATE`**. Como qualquer outra função de comparação, retorna um valor negativo ou positivo, ou `0`, dependendo se *string1* agrupa antes ou depois de *string2* ou é igual a ele.

locale.strxfrm(*string*)

Transforma uma string em uma que pode ser usada em comparações com reconhecimento de localidade. Por exemplo, `strxfrm(s1) < strxfrm(s2)` é equivalente a `strcoll(s1, s2) < 0`. Esta função pode ser usada quando a mesma string é comparada repetidamente, por exemplo, ao agrupar uma sequência de strings.

locale.format_string(*format*, *val*, *grouping=False*, *monetary=False*)

Formata um número *val* de acordo com a configuração atual do `LC_NUMERIC`. O formato segue as convenções do operador `%`. Para valores de ponto flutuante, o ponto decimal é modificado, se apropriado. Se *grouping* for `True`, também levará em conta o agrupamento.

Se *monetary* for verdadeiro, a conversão usa o separador de milhares monetários e strings de agrupamento.

Processa especificadores de formatação como em `format % val`, mas leva as configurações de localidade atuais em consideração.

Alterado na versão 3.7: O parâmetro nomeado *monetary* foi adicionado.

locale.currency(*val*, *symbol=True*, *grouping=False*, *international=False*)

Formata um número *val* de acordo com as configurações atuais de `LC_MONETARY`.

A string retornada inclui o símbolo da moeda se *symbol* for verdadeiro, que é o padrão. Se *grouping* for `True` (o que não é o padrão), o agrupamento é feito com o valor. Se *international* for `True` (o que não é o padrão), o símbolo da moeda internacional será usado.

Nota

Esta função não funcionará com a localidade ‘C’, então você deve definir uma localidade via `setlocale()` primeiro.

locale.str(*float*)

Formata um número de ponto flutuante usando o mesmo formato da função embutida `str(float)`, mas leva o ponto decimal em consideração.

locale.delocalize(*string*)

Converte uma string em uma string numérica normalizada, seguindo as configurações de `LC_NUMERIC`.

Adicionado na versão 3.5.

locale.localize(*string*, *grouping=False*, *monetary=False*)

Converte uma string numérica normalizada em uma string formatada, seguindo as configurações de `LC_NUMERIC`.

Adicionado na versão 3.10.

locale.atof(*string*, *func=float*)

Converte uma string para um número, de acordo com o valor da constante `LC_NUMERIC`, chamando *func* com o resultado da chamada à `delocalize()` em *string*.

locale.atoi(*string*)

Converte uma string em um número inteiro, seguindo as convenções de `LC_NUMERIC`.

locale.LC_CTYPE

Categoria da localidade para as funções de tipo de caracteres. Mais importante ainda, essa categoria define a codificação do texto, ou seja, como os bytes são interpretado como pontos de código Unicode. Consulte **PEP 538** [https://peps.python.org/pep-0538/] e **PEP 540** [https://peps.python.org/pep-0540/] para saber como essa variável pode ser automaticamente coagida para `C.UTF-8` para evitar problemas criados por configurações inválidas em contêineres ou configurações incompatíveis passadas por conexões remotas com SSH.

O Python não usa internamente funções de transformação de caracteres dependente de localidade do `ctype.h`. Em vez disso, `pyctype.h` fornece equivalentes independentes de localidade como `Py_TOLOWER`.

locale.LC_COLLATE

Categoria da localidade para classificação de strings. As funções `strcoll()` e `strxfrm()` do módulo **`locale`** são afetadas.

locale.LC_TIME

Categoria da localidade para a formatação de hora. A função `time.strftime()` segue essas convenções.

locale.LC_MONETARY

Categoria da localidade para formatação de valores monetários. As opções disponíveis estão disponíveis na função `localeconv()`.

locale.LC_MESSAGES

Categoria da localidade para exibição de mensagens. Python atualmente não oferece suporte a mensagens com reconhecimento de localidade específicas da aplicação. Mensagens exibidas pelo sistema operacional, como aquelas retornadas por `os.strerror()` podem ser afetadas por esta categoria.

Esse valor pode não estar disponível em sistemas operacionais que não estejam em conformidade com o padrão POSIX, principalmente o Windows.

locale.LC_NUMERIC

Categoria da localidade para formatação de números. As funções `format_string()`, `atoi()`, `atof()` e `str()` do módulo **`locale`** são afetadas por essa categoria. Todas as outras operações de formatação numérica não são afetadas.

locale.LC_ALL

Combinação de todas as configurações da localidade. Se este sinalizador for usado quando a localidade for alterada, a configuração da localidade para todas as categorias será tentada. Se isso falhar para qualquer categoria, nenhuma categoria é alterada. Quando a localidade é recuperada usando este sinalizador, uma string indicando a configuração para todas as categorias é retornada. Esta string pode ser usada posteriormente para restaurar as configurações.

locale.CHAR_MAX

Esta é uma constante simbólica usada para diferentes valores retornados por `localeconv()`.

Histórico, detalhes, dicas, dicas e advertências

O padrão C define a localidade como uma propriedade de todo o programa que pode ser relativamente cara para alterar. Além disso, algumas implementações são interrompidas de forma que mudanças frequentes de localidade podem causar despejos de memória. Isso torna a localidade um tanto dolorosa de usar corretamente.

Inicialmente, quando um programa é iniciado, a localidade é a localidade `C`, não importa qual a localidade preferida do usuário. Há uma exceção: a categoria **`LC_CTYPE`** é alterada na inicialização para definir a codificação de localidade atual para a codificação de localidade preferida do usuário. O programa deve dizer explicitamente que deseja as configurações de localidade preferidas do usuário para outras categorias chamando `setlocale(LC_ALL, '')`.

Geralmente é uma má ideia chamar **`setlocale()`** em alguma rotina de biblioteca, já que como efeito colateral afeta todo o programa. Salvar e restaurar é quase tão ruim: é caro e afeta outras threads que são executadas antes de as configurações serem restauradas.

Se, ao codificar um módulo para uso geral, você precisa de uma versão independente da localidade de uma operação que é afetada pela localidade (como certos formatos usados com **`time.strftime()`**), você terá que encontrar uma maneira de faça isso sem usar a rotina de biblioteca padrão. Melhor ainda é se convencer de que não há problema em usar as configurações da localidade. Apenas como último recurso, você deve documentar que seu módulo não é compatível com configurações de localidade não-`C`.

A única maneira de realizar operações numéricas de acordo com a localidade é usar as funções especiais definidas por este módulo: **`atof()`**, **`atoi()`**, **`format_string()`**, **`str()`**.

Não há como realizar conversões de maiúsculas e minúsculas e classificações de caracteres de acordo com a localidade. Para strings de texto (Unicode), isso é feito de acordo com o valor do caractere apenas,

enquanto para strings de byte, as conversões e classificações são feitas de acordo com o valor ASCII do byte e bytes cujo bit alto está definido (ou seja, bytes não ASCII) nunca são convertidos ou considerados parte de uma classe de caracteres, como letras ou espaços em branco.

Nomes de localidades

O formato do nome de localidade depende da plataforma, e o conjunto de locais suportados pode depender da configuração do sistema.

Em plataformas Posix, isso geralmente tem o formato [[1]]:

```
language ["_" territory] ["." charset] ["@" modifier]
```

onde *language* é um código de idioma de duas ou três letras da ISO 639 [https://www.iso.org/iso-639-language-code], *territory* é um código de país ou região de duas letras da ISO 3166 [https://www.iso.org/iso-3166-country-codes.html], *charset* é uma codificação de localidade e *modifier* é um nome de script, uma subtag de idioma, um identificador de ordem de classificação ou outro modificador de localidade (por exemplo, “latin”, “valencia”, “stroke” e “euro”).

No Windows, vários formatos são suportados. [[2]] [[3]] Um subconjunto de tags IETF BCP 47 [https://www.rfc-editor.org/info/bcp47]:

```
language ["-" script] ["-" territory] ["." charset]
language ["-" script] "-" territory "-" modifier
```

onde *language* e *territory* têm o mesmo significado que em Posix, *script* é um código de script de quatro letras da ISO 15924 [https://www.unicode.org/iso15924/] e *modifier* é uma subtag de idioma, um identificador de ordem de classificação ou modificador personalizado (por exemplo, “valencia”, “stroke” ou “x-python”). São aceitos os separadores hífen (`'-'`) e sublinhado (`'_'`). Somente a codificação UTF-8 é permitida para tags BCP 47.

O Windows também oferece suporte a nomes de localidade no formato:

```
language ["_" territory] ["." charset]
```

onde *language* e *territory* são nomes completos, como “English” e “United States”, e *charset* é um número de página de código (por exemplo, “1252”) ou UTF-8. Só é aceito o separador sublinhado neste formato.

A localidade “C” é aceita em todas as plataformas.

[1]

IEEE Std 1003.1-2024; 8.2 Internationalization Variables [https://pubs.opengroup.org/onlinepubs/9799919799/basedefs/V1_chap08.html#tag_08_02]

[2]

Nomes de localidade, idiomas e strings de país/região do UCRT [https://learn.microsoft.com/pt-br/cpp/c-runtime-library/locale-names-languages-and-country-region-strings]

[3]

Nomes de localidade [https://learn.microsoft.com/pt-br/windows/win32/intl/locale-names]

Para escritores de extensão e programas que incorporam Python

Módulos de extensão nunca devem chamar **`setlocale()`**, exceto para descobrir qual é a localidade atual. Mas uma vez que o valor de retorno só pode ser usado portavelmente para restaurá-lo, isso não é muito útil (exceto talvez para descobrir se a localidade é ou não `C`).

Quando o código Python usa o módulo **`locale`** para alterar a localidade, isso também afeta a aplicação de incorporação. Se a aplicação de incorporação não quiser que isso aconteça, ele deve remover o módulo de extensão **`_locale`** (que faz todo o trabalho) da tabela de módulos embutidos no arquivo `config.c` e certificar-se de que o módulo **`_locale`** não está acessível como uma biblioteca compartilhada.

Acesso a catálogos de mensagens

locale.gettext(*msg*)

locale.dgettext(*domain*, *msg*)

locale.dcgettext(*domain*, *msg*, *category*)

locale.textdomain(*domain*)

locale.bindtextdomain(*domain*, *dir*)

locale.bind_textdomain_codeset(*domain*, *codeset*)

O módulo locale expõe a interface gettext da biblioteca C em sistemas que fornecem essa interface. Consiste nas funções **`gettext()`**, **`dgettext()`**, **`dcgettext()`**, **`textdomain()`**, **`bindtextdomain()`** e **`bind_textdomain_codeset()`**. Elas são semelhantes às mesmas funções no módulo **`gettext`**, mas usam o formato binário da biblioteca C para catálogos de mensagens e os algoritmos de pesquisa da biblioteca C para localizar catálogos de mensagens.

As aplicações Python normalmente não precisam invocar essas funções e devem usar **`gettext`** em seu lugar. Uma exceção conhecida a esta regra são os aplicativos que se vinculam a bibliotecas C adicionais que invocam internamente `gettext` ou `dcgettext`. Para essas aplicações, pode ser necessário vincular o domínio de texto, para que as bibliotecas possam localizar adequadamente seus catálogos de mensagens.

Interfaces gráficas de usuário com Tk

Tk/Tcl tem sido parte integrante do Python. Ele fornece um kit de ferramentas de janela robusto e independente de plataforma, que está disponível para programadores Python usando o pacote `tkinter` e sua extensão, o módulo `tkinter.ttk`.

O pacote `tkinter` é uma fina camada orientada a objetos no topo do Tcl/Tk. Para usar `tkinter`, você não precisa escrever o código Tcl, mas precisará consultar a documentação do Tk e, ocasionalmente, a documentação do Tcl. `tkinter` é um conjunto de envólucros que implementam os widgets Tk como classes Python.

As principais virtudes do `tkinter` são que ele é rápido, e que geralmente vem junto com o Python. Embora sua documentação padrão seja fraca, um bom material está disponível, que inclui: referências, tutoriais, um livro e outros. `tkinter` também é famoso por ter uma aparência desatualizada, que foi amplamente melhorada no Tk 8.5. No entanto, existem muitas outras bibliotecas GUI nas quais você pode estar interessado. A wiki do Python lista várias frameworks e ferramentas GUI [https://wiki.python.org/moin/GuiProgramming] alternativas.

- `tkinter` — Interface Python para Tcl/Tk
  - Arquitetura
  - Tkinter modules
  - Tkinter life preserver
    - A Hello World program
    - Important Tk concepts
    - Understanding how Tkinter wraps Tcl/Tk
    - Como é que eu…? Que opção faz…?
    - Navigating the Tcl/Tk reference manual
  - Modelo de threading
  - Handy reference

        - Setting options
        - Geometry management
        - Coupling widget variables
        - The window manager
        - Tk option data types
        - Bindings and events
        - The index parameter
        - Imagens
    - Referência
        - Base and mixin classes
        - Toplevel widgets
        - Widget classes
        - Variable classes
        - Image classes
        - Other classes
        - Module-level functions
        - File handlers
        - Constantes
- `tkinter.colorchooser` — Diálogo de escolha de cor
- `tkinter.font` — Invólucro de fontes Tkinter
- Tkinter dialogs
    - `tkinter.simpledialog` — Standard Tkinter input dialogs
    - `tkinter.filedialog` — File selection dialogs
        - Native load/save dialogs
    - `tkinter.commondialog` — Dialog window templates
    - `tkinter.dialog` — Classic Tk dialog boxes
- `tkinter.messagebox` — Prompts de mensagem do Tkinter
- `tkinter.scrolledtext` — Widget de texto com *scrollbar*
- `tkinter.dnd` — Suporte para arrastar e soltar
- `tkinter.ttk` — Widgets temáticos do Tk
    - Usando Ttk
    - Ttk widgets
    - Widget
        - Standard options
        - Scrollable widget options
        - Label options
        - Compatibility options

        - Widget states
        - ttk.Widget
    - Combobox
        - Opções
        - Eventos virtuais
        - ttk.Combobox
    - Spinbox
        - Opções
        - Eventos virtuais
        - ttk.Spinbox
    - Notebook
        - Opções
        - Tab options
        - Tab identifiers
        - Eventos virtuais
        - ttk.Notebook
    - Progressbar
        - Opções
        - ttk.Progressbar
    - Separator
        - Opções
    - Sizegrip
        - Platform-specific notes
        - Bugs
    - Treeview
        - Opções
        - Item options
        - Tag options
        - Column identifiers
        - Eventos virtuais
        - ttk.Treeview
    - Ttk styling
        - Layouts
    - Additional widgets
- IDLE — Editor e console Python
    - Menus
        - Menu File (Shell e Editor)

        - Menu Edit (Shell e Editor)
        - Menu Format (somente na janela do Editor)
        - Menu Run (somente na janela do Editor)
        - Menu Shell (somente na janela do Shell)
        - Menu Debug (somente na janela do Shell)
        - Menu Options (Shell e Editor)
        - Menu Window (Shell e Editor)
        - Menu Help (Shell e Editor)
        - Menus de contexto
    - Edição e navegação
        - Janelas do Editor
        - Teclas de atalho
        - Indentação automática
        - Busca e substituição
        - Autocomplementos
        - Dicas de chamada
        - Formatação de bloco
        - Contexto de código
        - Janela Shell
        - Text colors
    - Startup and Code Execution
        - Uso na linha de comando
        - Startup failure
        - Running user code
        - User output in Shell
        - Developing tkinter applications
        - Running without a subprocess
    - Ajuda e preferências
        - Help sources
        - Configurando preferências
        - IDLE on macOS
        - Extensions
    - idlelib — implementation of IDLE application
- `turtle` — Gráficos tartaruga
    - Tutorial
        - Iniciando um ambiente da tartaruga
        - Desenho básico

        - Controle da Caneta
        - A posição da tartaruga
    - Criação de padrões algorítmicos
- Referência
    - Métodos e funções Turtle
        - Movimento e desenho
        - Fala o estado de Turtle
        - Configurações de medida
        - Controle da Caneta
            - Estado do Desenho
            - Controle da Cor
            - Preenchimento
            - Mais sobre o Controle do Desenho
        - Estado da tartaruga
            - Visibilidade
            - Aparência
        - Eventos Utilizados
        - Métodos Especiais da Tartaruga
        - Formas compostas
    - Métodos e funções de janela
        - Controle da Janela
        - Controle da animação
        - Usando os eventos de tela
        - Métodos de entrada
        - Configurações e métodos especiais
        - Métodos apenas de janela
    - Classes Públicas
    - Exceções
- Guias de como fazer
    - Inicie e encerre o preenchimento automaticamente
    - Use o espaço de nomes do módulo `turtle`
    - Use o módulo gráfico de tartaruga dentro de um script com código específico
    - Use o módulo gráficos da tartaruga orientado a objetos
    - Como usar a Ajuda
    - Tradução de docstrings em diferentes idiomas
    - Como configurar Screen and Turtles

- Explicação
- `turtledemo` — Scripts de Demonstração

`tkinter` — Interface Python para Tcl/Tk

Código-fonte: Lib/tkinter/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/__init__.py]

The **`tkinter`** package ("Tk interface") is the standard Python interface to the Tcl/Tk GUI toolkit. Both Tk and **`tkinter`** are available on most Unix platforms, including macOS, as well as on Windows systems.

Running `python -m tkinter` from the command line should open a window demonstrating a simple Tk interface, letting you know that **`tkinter`** is properly installed on your system, and also showing what version of Tcl/Tk is installed, so you can read the Tcl/Tk documentation specific to that version.

Tkinter supports a range of Tcl/Tk versions, built either with or without thread support. Tcl/Tk 8.5.12 is the minimum supported version; the official Python binary release bundles Tcl/Tk 9.0. See the source code for the **`_tkinter`** module for more information about supported versions.

Alterado na versão 3.11: Support for Tcl/Tk versions older than 8.5.12 was removed.

O Tkinter não é um invólucro fino, mas adiciona uma boa quantidade de sua própria lógica para tornar a experiência mais pythônica. Esta documentação se concentrará nessas adições e mudanças, e consulte a documentação oficial do Tcl/Tk para detalhes que não foram alterados.

Nota

Tcl/Tk 8.5 (2007) introduced a modern set of themed user interface components along with a new API to use them (see **`tkinter.ttk`**). Both

old and new APIs are still available. Most documentation you will find online still uses the old API and can be woefully outdated.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Ver também

- **TkDocs** **[https://tkdocs.com/]**
  Um extenso tutorial sobre como criar interfaces de usuário com o Tkinter. Explica conceitos chave, e ilustra as abordagens recomendadas usando a API moderna.
- **Referência do Tkinter 8.5: uma GUI para Python** **[https://www.tkdocs.com/shipman/']**
  Documentação de referência para Tkinter 8.5 detalhando classes, métodos e opções disponíveis.

Recursos Tcl/Tk:

- **Tk commands** **[https://www.tcl-lang.org/man/tcl9.0/TkCmd/index.html]**
  Referência abrangente para cada um dos comandos Tcl/Tk subjacentes usados pelo Tkinter.
- **Tcl/Tk Website** **[https://www.tcl.tk]**
  Documentação adicional e links para o desenvolvimento do core do Tcl/Tk.

Livros:

- **Modern Tkinter for Busy Python Developers** **[https://tkdocs.com/book.html]**
  Por Mark Roseman. (ISBN 978-1999149567)
- **Python GUI programming with Tkinter** **[https://www.packtpub.com/en-br/product/python-gui-programming-with-tkinter-9781788835886]**
  por Alan D. Moore. (ISBN 978-1788835886)
- **Programming Python** **[https://learning-python.com/about-pp4e.html]**

Por Mark Lutz; tem uma excelente cobertura sobre o Tkinter. (ISBN 978-0596158101)

- **Tcl and the Tk Toolkit (2nd edition)**
  [https://www.amazon.com/exec/obidos/ASIN/032133633X]
  Por John Ousterhout, criador do Tcl/Tk, e Ken Jones; não cobre o Tkinter. (ISBN 978-0321336330)

Arquitetura

Tcl/Tk não é uma biblioteca única mas consiste em alguns módulos distintos, cada um com sua própria funcionalidade e sua própria documentação oficial. As versões binárias do Python também incluem um módulo adicional junto com ele.

Tcl
: Tcl é uma linguagem de programação interpretada dinâmica, assim como Python. Embora possa ser usado sozinho como uma linguagem de programação de propósito geral, é mais comumente embutido em aplicativos C como um mecanismo de script ou uma interface para o kit de ferramentas Tk. A biblioteca Tcl tem uma interface C para criar e gerenciar uma ou mais instâncias de um interpretador Tcl, executar comandos e scripts Tcl nessas instâncias e adicionar comandos personalizados implementados em Tcl ou C. Cada interpretador tem uma fila de eventos, e há instalações para enviar eventos e processá-los. Ao contrário do Python, o modelo de execução do Tcl é projetado em torno da multitarefa cooperativa, e o Tkinter faz a ponte entre essa diferença (veja Modelo de threading para detalhes).

Tk
: Tk é um pacote Tcl [https://wiki.tcl-lang.org/37432] implementado em C que adiciona comandos personalizados para criar e manipular widgets GUI. Cada objeto `Tk` incorpora sua própria instância do interpretador Tcl com Tk carregado nele. Os widgets do Tk são muito personalizáveis, embora ao custo de uma aparência desatualizada. Tk usa a fila de eventos do Tcl para gerar e processar eventos da GUI.

Ttk

Themed Tk (Ttk) é uma família mais recente de widgets Tk que fornecem uma aparência muito melhor em diferentes plataformas do que muitos dos widgets Tk clássicos. O Ttk é distribuído como parte do Tk, começando com o Tk versão 8.5. As ligações Python são fornecidas em um módulo separado, `tkinter.ttk`.

Internally, Tk and Ttk use facilities of the underlying operating system, that is, Xlib on Unix/X11, Cocoa on macOS, GDI on Windows.

When your Python application uses a class in Tkinter, for example, to create a widget, the `tkinter` module first assembles a Tcl/Tk command string. It passes that Tcl command string to an internal `_tkinter` binary module, which then calls the Tcl interpreter to evaluate it. The Tcl interpreter will then call into the Tk and/or Ttk packages, which will in turn make calls to Xlib, Cocoa, or GDI.

Tkinter modules

Support for Tkinter is spread across several modules. Most applications will need the main `tkinter` module, as well as the `tkinter.ttk` module, which provides the modern themed widget set and API:

```
from tkinter import *
from tkinter import ttk
```

Os módulos que fornecem suporte ao Tk incluem:

`tkinter`

Módulo principal Tkinter.

`tkinter.colorchooser`

Caixa de diálogo para permitir que o usuário escolha uma cor.

`tkinter.commondialog`

Classe base para os diálogos definidos nos outros módulos listados aqui.

`tkinter.filedialog`

Diálogos comuns para permitir ao usuário especificar um arquivo para abrir ou salvar.

tkinter.font
Utilitários para ajudar a trabalhar com fontes.

tkinter.messagebox
Acesso às caixas de diálogo padrão do Tk.

tkinter.scrolledtext
Componente de texto com uma barra de rolagem vertical integrada.

tkinter.simpledialog
Diálogos básicos e funções de conveniência.

tkinter.ttk
Themed widget set introduced in Tk 8.5, providing modern alternatives for many of the classic widgets in the main **tkinter** module.

Módulos adicionais:

_tkinter
A binary module that contains the low-level interface to Tcl/Tk. It is automatically imported by the main **tkinter** module, and should never be used directly by application programmers. It is usually a shared library (or DLL), but might in some cases be statically linked with the Python interpreter.

idlelib
Python's Integrated Development and Learning Environment (IDLE). Based on **tkinter**.

tkinter.constants
Symbolic constants that can be used in place of strings when passing various parameters to Tkinter calls. Automatically imported by the main **tkinter** module.

tkinter.dnd

(experimental) Drag-and-drop support for **`tkinter`**. This will become deprecated when it is replaced with the Tk DND.

`turtle`
Gráficos tartaruga em uma janela Tk.

Tkinter life preserver

Esta seção não foi projetada para ser um tutorial exaustivo sobre Tk ou Tkinter. Para isso, consulte um dos recursos externos mencionados anteriormente. Em vez disso, esta seção fornece uma orientação muito rápida sobre como é uma aplicação Tkinter, identifica os conceitos fundamentais do Tk e explica como o wrapper do Tkinter é estruturado.

O restante desta seção ajudará a identificar as classes, métodos e opções que você precisará em uma aplicação Tkinter e onde encontrar documentação mais detalhada sobre isso, inclusive no guia de referência oficial do Tcl/Tk.

A Hello World program

Começaremos a explorar o Tkinter através de uma simples aplicação “Hello World”. Esta não é a menor aplicação que poderíamos escrever, mas tem o suficiente para ilustrar alguns conceitos-chave que você precisa saber.

```
from tkinter import *
from tkinter import ttk
root = Tk()
frm = ttk.Frame(root, padding=10)
frm.grid()
ttk.Label(frm, text="Hello World!").grid(column=0, row=0)
ttk.Button(frm, text="Quit", command=root.destroy).grid(column=1
root.mainloop()
```

Após as importações, a próxima linha será criar a instância da classe **`Tk`** que inicia Tk e cria o interpretador associado a Tcl. Ele também cria uma janela de nível superior, conhecida como janela raiz, que serve como a janela principal da aplicação.

A linha a seguir cria um frame widget , que neste caso conterá um rótulo e um botão que criaremos a seguir. O frame se encaixa dentro da janela raiz.

The next line creates a label widget holding a static text string. The `grid()` method is used to specify the relative layout (position) of the label within its containing frame widget, similar to how tables in HTML work.

A button widget is then created, and placed to the right of the label. When pressed, it will call the `destroy()` method of the root window.

Por fim, o método **`destroy()`** coloca tudo no display, e responde à entrada do usuário até que o programa termine.

Important Tk concepts

Mesmo com este programa simples, os conceitos-chave do Tk podem ser mostrados:

widgets
: A interface de usuário do Tkinter é composta de *widgets* individuais. Cada widget é representado como um objeto Python, instanciado de classes como `ttk.Frame`, `ttk.Label`, e `ttk.Button`.

hierarquia dos widgets
: Os widgets são organizados em uma *hierarquia*. O rótulo e botão estavam contidos em um frame, que por sua vez estava contido na janela raiz. Quando criamos um widget *filho*, seu widget *pai* é passado como primeiro argumento para o construtor do widget.

opções de configuração
: Widgets possuem *opções de configuração*, que modificam sua aparência e comportamento, como o texto a ser exibido em um rótulo ou botão. Diferentes classes de widgets terão diferentes conjuntos de opções.

gerenciamento de geometria
: Os widgets não são automaticamente adicionados à interface de usuário quando eles são criados. Um *gerenciador de geometria* como `grid` controla onde na interface de usuário eles são colocados.

loop de eventos

O Tkinter reage à entrada do usuário, muda de seu programa, e até mesmo atualiza a tela somente ao executar ativamente um *loop de eventos*. Se o seu programa não estiver executando o loop de eventos, sua interface de usuário não será atualizada.

Understanding how Tkinter wraps Tcl/Tk

Quando a sua aplicação utiliza as classes e métodos do Tkinter, internamente o Tkinter está montando strings representando comandos Tcl/Tk e executando esses comandos no interpretador Tcl anexado à instância `Tk` de sua aplicação.

Seja tentando navegar na documentação de referência, tentando encontrar o método ou opção certa, adaptando algum código existente ou depurando seu aplicativo Tkinter, há momentos em que será útil entender como são os comandos Tcl/Tk subjacentes.

Para ilustrar, aqui está o equivalente Tcl/Tk da parte principal do script Tkinter acima.

```
ttk::frame .frm -padding 10
grid .frm
grid [ttk::label .frm.lbl -text "Hello World!"] -column 0 -row 0
grid [ttk::button .frm.btn -text "Quit" -command "destroy ."] -c
```

A sintaxe do Tcl é semelhante a muitas linguagens de shell, onde a primeira palavra é o comando a ser executado, seguido de argumentos para esse comando separados por espaços. Sem entrar em muitos detalhes, observe o seguinte:

- Os comandos usados para criar widgets (como `ttk::frame`) correspondem a classes de widgets no Tkinter.
- As opções de widget Tcl (como `-text`) correspondem a argumentos nomeados no Tkinter.
- Widgets são referenciados por um *pathname* em Tcl (like `.frm.btn`), enquanto o Tkinter não utiliza nomes, mas referências a objetos.

- O lugar de um widget na hierarquia widget é codificado em seu pathname (hierárquico), que utiliza um `.` (ponto) como um separador de caminho. O pathname para a janela raiz é apenas `.` (ponto). No Tkinter, a hierarquia é definida não pelo pathname, mas pela especificação do widget pai ao criar cada widget filho.
- Operações que são implementadas como *comandos* separados em Tcl (como `grid` ou `destroy`) são representadas como *métodos* em objetos de widget Tkinter. Como você verá em breve, em outras ocasiões o Tcl usa o que parecem ser chamadas de método em objetos widget, que espelham mais de perto o que é usado no Tkinter.

Como é que eu…? Que opção faz…?

Se você não tem certeza de como fazer algo no Tkinter e não consegue encontrá-lo imediatamente no tutorial ou na documentação de referência que está usando, existem algumas estratégias que podem ser úteis.

Primeiro, lembre-se de que os detalhes de como os widgets individuais funcionam podem variar entre diferentes versões tanto do Tkinter quanto do Tcl/Tk. Se você estiver procurando por documentação, certifique-se de que corresponda às versões do Python e Tcl/Tk instaladas em seu sistema.

Ao procurar como usar uma API, é útil saber o nome exato da classe, opção ou método que você está usando. A introspecção, seja em um console Python interativo ou com a `print()`, pode ajudar a identificar o que você precisa.

To find out what configuration options are available on any widget, call its `configure()` method, which returns a dictionary containing a variety of information about each object, including its default and current values. Use `keys()` to get just the names of each option.

```
btn = ttk.Button(frm, ...)
print(btn.configure().keys())
```

Como a maioria dos widgets tem muitas opções de configuração em comum, pode ser útil descobrir quais são específicas para uma determinada

classe de widget. Comparar a lista de opções com a de um widget mais simples, como um frame, é uma maneira de fazer isso.

```
print(set(btn.configure().keys()) - set(frm.configure().keys()))
```

Da mesma forma, você pode encontrar os métodos disponíveis para um objeto widget usando a função padrão **`dir()`**. Se você tentar, verá que existem mais de 200 métodos comuns de widget, então identificar aqueles específicos para uma classe de widget é útil.

```
print(dir(btn))
print(set(dir(btn)) - set(dir(frm)))
```

Navigating the Tcl/Tk reference manual

As noted, the official Tk commands [https://www.tcl-lang.org/man/tcl9.0/TkCmd/index.html] reference manual (man pages) is often the most accurate description of what specific operations on widgets do. Even when you know the name of the option or method that you need, you may still have a few places to look.

While all operations in Tkinter are implemented as method calls on widget objects, you've seen that many Tcl/Tk operations appear as commands that take a widget pathname as its first parameter, followed by optional parameters, for example

```
destroy .
grid .frm.btn -column 0 -row 0
```

Outros, no entanto, parecem mais com métodos chamados em um objeto widget (na verdade, quando você cria um widget em Tcl/Tk, ele cria um comando Tcl com o nome do caminho do widget, sendo o primeiro parâmetro desse comando o nome de um método a ser chamado).

```
.frm.btn invoke
.frm.lbl configure -text "Goodbye"
```

In the official Tcl/Tk reference documentation, you'll find most operations that look like method calls on the man page for a specific widget (for example, you'll find the `invoke()` method on the ttk::button [https://www.tcl-lang.org/man/tcl9.0/TkCmd/ttk_button.html] man page), while functions that take a widget as a parameter often have their own man page (for example, grid [https://www.tcl-lang.org/man/tcl9.0/TkCmd/grid.html]).

You'll find many common options and methods in the options [https://www.tcl-lang.org/man/tcl9.0/TkCmd/options.html] or ttk::widget [https://www.tcl-lang.org/man/tcl9.0/TkCmd/ttk_widget.html] man pages, while others are found in the man page for a specific widget class.

You'll also find that many Tkinter methods have compound names, for example, `winfo x()`, `winfo height()`, `winfo viewable()`. You'd find documentation for all of these in the winfo [https://www.tcl-lang.org/man/tcl9.0/TkCmd/winfo.html] man page.

Nota

De forma bem confusa, também existem métodos em todos os widgets do Tkinter que na verdade não operam no widget, mas operam em um escopo global, independente de qualquer widget. Exemplos são métodos para acessar a área de transferência ou o sinal do sistema. (Eles são implementados como métodos na classe base `Widget` da qual todos os widgets do Tkinter herdam).

Modelo de threading

Python and Tcl/Tk have very different threading models, which `tkinter` tries to bridge. If you use threads, you may need to be aware of this.

Um interpretador Python pode ter muitas threads associados a ele. Em Tcl, várias threads podem ser criadas, mas cada thread tem uma instância de interpretador Tcl separada associada a ele. Threads também podem criar mais de uma instância do interpretador, embora cada instância do interpretador possa ser usada apenas pela thread que a criou.

Each `Tk` object created by `tkinter` contains a Tcl interpreter. It also keeps track of which thread created that interpreter. Calls to `tkinter` can be made from any Python thread. Internally, if a call comes from a thread other than the one that created the `Tk` object, an event is posted to the interpreter's event queue, and when executed, the result is returned to the calling Python thread.

Tcl/Tk applications are normally event-driven, meaning that after initialization, the interpreter runs an event loop (that is, `Tk.mainloop`) and responds to events. Because it is single-threaded, event handlers must respond quickly, otherwise they will block other events from being processed. To avoid this, any long-running computations should not run in an event handler, but are either broken into smaller pieces using timers, or run in another thread. This is different from many GUI toolkits where the GUI runs in a completely separate thread from all application code including event handlers.

If the Tcl interpreter is not running the event loop and processing events, any `tkinter` calls made from threads other than the one running the Tcl interpreter will fail.

Existem vários casos especiais:

- Tcl/Tk libraries built without thread support are now rare: Tcl/Tk 9.0 (the bundled version) is always thread-aware, so this case only arises with some older 8.x builds. When the library is not thread-aware, `tkinter` calls the library from the originating Python thread, even if this is different than the thread that created the Tcl interpreter. A global lock ensures only one call occurs at a time.
- While `tkinter` allows you to create more than one instance of a `Tk` object (with its own interpreter), all interpreters that are part of the same thread share a common event queue, which gets ugly fast. In practice, don't create more than one instance of `Tk` at a time. Otherwise, it's best to create them in separate threads and ensure you're running a thread-aware Tcl/Tk build.
- Bloquear manipuladores de eventos não é a única maneira de evitar que o interpretador Tcl entre novamente no laço de eventos. É ainda possível executar vários laços de eventos aninhados ou abandonar

totalmente o laço de eventos. Se você estiver fazendo algo complicado quando se trata de eventos ou threads, esteja ciente dessas possibilidades.

- There are a few select **`tkinter`** functions that presently work only when called from the thread that created the Tcl interpreter.

Handy reference

Setting options

As opções controlam coisas como a cor e a largura da borda de um widget. As opções podem ser definidas de três maneiras:

No momento da criação do objeto, usando argumentos nomeados

```
fred = Button(self, fg="red", bg="blue")
```

Após a criação do objeto, tratando o nome da opção como um índice de dicionário

```
fred["fg"] = "red"
fred["bg"] = "blue"
```

Use o método config() para atualizar vários attrs posteriores à criação do objeto

```
fred.config(fg="red", bg="blue")
```

Nota

The `fg` and `bg` options used here, and other options that control a widget's appearance, belong to the classic **`tkinter`** widgets. The themed `tkinter.ttk` widgets recommended in the introduction do not accept them; style a themed widget through the `ttk.Style` class instead. The three ways of setting an option shown above apply to both widget sets.

Para uma explicação completa de uma dada opção e seu comportamento, veja as páginas man do Tk para o widget em questão.

Note que as páginas man listam “OPÇÕES PADRÃO” e “OPÇÕES DE WIDGET ESPECÍFICO” para cada widget. A primeira é uma lista de opções que são comuns a vários widgets, e a segunda são opções que são idiossincráticas ao widget em particular. As Opções Padrão estão documentadas na página man *options(3)* *[https://manpages.debian.org/options(3)]*.

Nenhuma distinção entre as opções padrão e específicas de widget são feitas neste documento. Algumas opções não se aplicam a alguns tipos de widgets. A resposta de um dado widget a uma opção particular depende da classe do widget; botões possuem a opção `command`, etiquetas não.

The options supported by a given widget are listed in that widget’s man page, or can be queried at runtime by calling the **`config()`** method without arguments, or by calling the **`keys()`** method on that widget. The return value of these calls is a dictionary whose key is the name of the option as a string (for example, `'relief'`) and whose values are 5-tuples.

Some options, like `bg`, are synonyms for common options with long names (`bg` is shorthand for “background”).

Índice	Significado	Exemplo
0	nome da opção	`'relief'`
1	nome da opção para buscas de banco de dados	`'relief'`
2	classe de opção para busca de banco de dados	`'Relief'`
3	valor padrão	`'raised'`
4	valor atual	`'groove'`

Exemplo:

```
>>> print(fred.config())
{'relief': ('relief', 'relief', 'Relief', 'raised', 'groove')}
```

É claro que o dicionário exibido irá incluir todas as opções disponíveis e seus valores. Isso foi apenas um exemplo.

Geometry management

Creating a widget does not display it. A widget appears only after it has been handed to a *geometry manager*, which works out its size and position inside its container and keeps the layout up to date as the container is resized or its content changes. Forgetting to call a geometry manager is a common early mistake: the widget is created, but nothing shows up.

Tk provides three geometry managers. Each is inherited by every widget, so any widget can be managed by any of them (but see the warning below about the incompatibility of grid and pack). The choice depends on the kind of layout you want.

`grid`

Arranges widgets in a two-dimensional table of rows and columns. It is the most flexible manager and the one to reach for by default: layouts that would otherwise need several nested frames can often be expressed as a single grid, and rows and columns can be told how to absorb extra space.

```
ttk.Label(frm, text="Name:").grid(column=0, row=0, sticky="w'
ttk.Entry(frm).grid(column=1, row=0)
ttk.Button(frm, text="OK").grid(column=1, row=1, sticky="e")
```

`pack`

Stacks widgets against one side of their container – `"top"` (the default), `"bottom"`, `"left"` or `"right"` – and can make them fill or expand into the space that is left. It is convenient for simple arrangements, such as a single row or column of widgets or a content area framed by a toolbar and a status bar.

```
toolbar.pack(side="top", fill="x")
status.pack(side="bottom", fill="x")
body.pack(side="left", expand=True, fill="both")
```

place

Positions each widget at an explicit spot, given either as absolute screen distances or as a fraction of the container's size. It offers the most control but the least automatic behavior, and is used the least; it suits special cases such as overlapping widgets or precise custom layouts.

```
background.place(x=0, y=0, relwidth=1.0, relheight=1.0)
badge.place(relx=1.0, rely=0.0, anchor="ne")
```

Layouts are built up by nesting: grid or pack widgets, including frames, inside a frame or toplevel. Toplevels are managed by the OS window manager. Classic and themed **tkinter.ttk** widgets can be managed interchangeably.

Aviso

Do not apply **pack()** and **grid()** to two widgets that share the same container. The two managers negotiate sizes in incompatible ways, and the application can hang as they repeatedly resize the container against each other. To combine them, keep each manager's widgets in a separate frame.

The full set of options accepted by each manager, with their values and defaults, is documented under **Grid.grid_configure()**, **Pack.pack_configure()** and **Place.place_configure()**; see also the *grid(3tk)* *[https://manpages.debian.org/grid(3tk)]*, *pack(3tk)* *[https://manpages.debian.org/pack(3tk)]* and *place(3tk)* *[https://manpages.debian.org/place(3tk)]* man pages.

Coupling widget variables

Some widgets can tie their current value directly to a program variable, so that the two stay in sync. Options such as `variable`, `textvariable`, `value`,

`onvalue` and `offvalue` set up this connection: when the user changes the widget the variable is updated, and when the variable is set the widget redraws to match.

A widget can be linked only to a **`Variable`** object, not to an ordinary Python variable. This is not a limitation of **`tkinter`** but a consequence of how the two languages differ: the link relies on Tcl being notified every time the value changes, and Python offers no way to react when a plain variable is reassigned. A **`Variable`** sidesteps this by keeping its value inside the Tcl interpreter and exposing it through explicit **`get()`** and **`set()`** methods.

Ready-made subclasses cover the common types: **`StringVar`**, **`IntVar`**, **`DoubleVar`** and **`BooleanVar`**. Pass one as a widget's `textvariable` (or `variable`) option, then read and update it with **`get()`** and **`set()`**; the widget tracks it with no further work on your part.

Keep a reference to the variable for as long as the widget uses it – for example by storing it as an attribute. A **`Variable`** that is garbage collected removes its underlying Tcl variable, breaking the connection to the widget (see **`Variable`**).

Por exemplo:

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()

# Create the application variable and give it an initial value.
contents = tk.StringVar(value="this is a variable")

# Tell the entry widget to track the variable.
entry = ttk.Entry(root, textvariable=contents)
entry.pack()

# Print the current value whenever the user presses Return.
def print_contents(event):
    print("The current entry content is:", contents.get())

entry.bind("<Return>", print_contents)
```

```
# Setting the variable from the program updates the entry throug
# same link.
def clear():
    contents.set("")

ttk.Button(root, text="Clear", command=clear).pack()

root.mainloop()
```

The window manager

The *window manager* is the part of the desktop responsible for the title bar, border and controls drawn around each top-level window, and for such things as its title, position, size and icon. Tk gives access to these through the `Wm` mixin, which is inherited by the `Tk` root window and by every `Toplevel`. You therefore call the window-manager methods directly on a top-level window. Each has a short name and an equivalent `wm_`-prefixed name, for example `title()` and `wm_title()`.

These methods act on the top-level window whether its content is built from the classic widgets or the themed `tkinter.ttk` widgets. To reach the top-level window containing an arbitrary widget, call its `winfo_toplevel()` method.

Por exemplo:

```
import tkinter as tk
from tkinter import ttk

root = tk.Tk()
root.title("My Application")
root.geometry("640x480")
root.minsize(320, 240)

ttk.Label(root, text="Hello").pack(padx=20, pady=20)

root.mainloop()
```

See `Wm` for the full set of window-manager methods.

Tk option data types

Many widget options documented in the reference accept values of a small number of common types, described here.

anchor

Valores legais são os pontos cardeais: `"n"`, `"ne"`, `"e"`, `"se"`, `"s"`, `"sw"`, `"w"`, `"nw"`, e também `"center"`.

bitmap

There are ten built-in, named bitmaps: `'error'`, `'gray12'`, `'gray25'`, `'gray50'`, `'gray75'`, `'hourglass'`, `'info'`, `'questhead'`, `'question'`, `'warning'`. To specify an X bitmap filename, give the full path to the file, preceded with an `@`, as in `"@/usr/contrib/bitmap/gumby.bit"`.

booleano

Você pode passar os inteiros 0 ou 1 ou as strings `"yes"` ou `"no"`.

função de retorno

Esta é uma função Python que não aceita argumentos. Por exemplo:

```
def print_it():
    print("hi there")
fred["command"] = print_it
```

cor

Colors can be given as the names of X colors in the rgb.txt file, or as strings representing RGB values in 4 bit: `"#RGB"`, 8 bit: `"#RRGGBB"`, 12 bit: `"#RRRGGGBBB"`, or 16 bit: `"#RRRRGGGGBBBB"` ranges, where R,G,B here represent any legal hex digit. See the *colors(3tk)* *[https://manpages.debian.org/colors(3tk)]* man page for the list of named colors.

cursor

The name of the mouse cursor to display while the pointer is over the widget. Tk provides a portable set of cursor names available on all platforms (for example `"arrow"`, `"watch"`, `"cross"`, or `"hand2"`); the

standard X cursor names from `cursorfont.h` may also be used, without the `XC_` prefix (so `XC_hand2` becomes `"hand2"`). The full list of names, including the platform-specific ones, is given in the *cursors(3tk)* *[https://manpages.debian.org/cursors(3tk)]* manual page. You can also specify a bitmap and mask file of your own. On Windows a cursor file (`.cur` or `.ani`) may be used directly, giving its path preceded with an `@`, as in `"@C:/cursors/bart.ani"`.

distance

As distâncias da tela podem ser especificadas em pixels ou distâncias absolutas. Pixels são dados como números e distâncias absolutas como strings, com os caracteres finais denotando unidades: `c` para centímetros, `i` para polegadas, `m` para milímetros, `p` para os pontos da impressora. Por exemplo, 3,5 polegadas é expresso como `"3.5i"`.

font

Tk uses a font description such as `{courier 10 bold}`; in **`tkinter`** this is most naturally passed as a tuple of `(family, size, *styles)` (or as the equivalent string `"Courier 10 bold"`). Font sizes with positive numbers are measured in points; sizes with negative numbers are measured in pixels.

geometria

Esta é uma string no formato `widthxheight`, onde largura e altura são medidas em pixels para a maioria dos widgets (em caracteres para widgets que exibem texto). Por exemplo: `fred["geometry"] = "200x100"`.

justify

Valores aceitos são as strings: `"left"`, `"center"` e `"right"`.

region

Esta é uma string com quatro elementos delimitados por espaço, cada um dos quais a uma distância legal (veja acima). Por exemplo: `"2 3 4 5"` e `"3i 2i 4.5i 2i"` e `"3c 2c 4c 10.43c"` são todas regiões legais.

relief

Determines what the border style of a widget will be. Legal values are: `"raised"`, `"sunken"`, `"flat"`, `"groove"`, `"ridge"`, and `"solid"`.

scrollcommand
: Este é quase sempre o método **`set()`** de algum widget da barra de rolagem, mas pode ser qualquer método de widget que receba um único argumento.

wrap
: Deve ser um de: `"none"`, `"char"` ou `"word"`.

Bindings and events

O método de ligação do comando widget permite que você observe certos eventos e tenha um acionamento de função de retorno de chamada quando esse tipo de evento ocorrer. A forma do método de ligação é:

```
def bind(self, sequence, func, add=''):
```

sendo que:

sequence
: is a string that denotes the target kind of event. Physical events use the `<modifier-modifier-type-detail>` form (for example `"<Enter>"` or `"<Control-Button-1>"`); application-defined virtual events use double angle brackets, as in `"<<Paste>>"`. (See the *bind(3tk)* *[https://manpages.debian.org/bind(3tk)]* man page for details.)

func
: é uma função Python, tendo um argumento, a ser invocada quando o evento ocorre. Uma instância de evento será passada como argumento. (As funções implantadas dessa forma são comumente conhecidas como *funções de retorno*.)

add
: é opcional, tanto `''` ou `'+'`. Passar uma string vazia indica que essa ligação substitui todas as outras ligações às quais esse evento está

associado. Passar um `'+'` significa que esta função deve ser adicionada à lista de funções ligadas a este tipo de evento.

Por exemplo:

```
def turn_red(self, event):
    event.widget["activeforeground"] = "red"

self.button.bind("<Enter>", self.turn_red)
```

Observe como o campo widget do evento está sendo acessado na função de retorno `turn_red()`. Este campo contém o widget que capturou o evento X. A tabela a seguir lista os outros campos de evento que você pode acessar e como eles são denotados no Tk, o que pode ser útil ao se referir às páginas man do Tk.

Tk	Campo de Eventos do Tkinter	Tk	Campo de Eventos do Tkinter
%f	focus	%A	char
%h	height	%E	send_event
%k	keycode	%K	keysym
%s	state	%N	keysym_num
%t	time	%T	tipo
%w	width	%W	widget

Tk	Campo de Eventos do Tkinter	Tk	Campo de Eventos do Tkinter
%x	x	%X	x_root
%y	y	%Y	y_root
%#	serial	%b	num
%d	detail	%D	delta

The `add` parameter above only affects the bindings you make yourself. Every widget also inherits *class bindings* that implement its standard behavior – for example a **`Text`** widget binds `Control-t` to transpose two characters. These are described in the bindings section of the widget's Tk man page (such as *text(3tk)* *[https://manpages.debian.org/text(3tk)]* or *entry(3tk)* *[https://manpages.debian.org/entry(3tk)]*).

Class bindings are processed separately from your own, so binding an event yourself does not replace the default; both run. To suppress an unwanted default binding, bind the event on the widget and return the string `"break"` from your callback.

The index parameter

Vários widgets requerem que parâmetros de "indx" sejam passados. Eles são usados para apontar para um local específico em um widget Text, ou para caracteres específicos em um widget Entry, ou para itens de menu específicos em um widget Menu.

Índices de widget de entrada (índice, índice de visualização, etc.)

Entry widgets have methods and options that refer to character positions in the text being displayed. Anytime an index is needed, you may pass in:

- an integer which refers to the numeric position of a character, counted from the beginning of the text, starting with 0;
- the string `"anchor"`, which refers to the anchor point of the selection, set with the widget's selection methods;
- the string `"end"`, which refers to the position just after the last character;
- the string `"insert"`, which refers to the character just after the insertion cursor;
- the strings `"sel.first"` and `"sel.last"`, which refer to the first character in the selection and the position just after the last (it is an error to use these if there is no selection);
- a string consisting of `@` followed by an integer, as in `"@6"`, where the integer is interpreted as an x pixel coordinate in the entry's coordinate system, selecting the character spanning that point.

Índice de widget de texto

A notação de índice do widget de texto é muito rica e é melhor detalhada nas páginas do manual Tk.

Índices de menu (menu.invoke(), menu.entryconfig(), etc.)

Algumas opções e métodos de menus manipulam entradas de menu específicas. Sempre que um índice de menu é necessário para uma opção ou parâmetro, você pode passar:

- um número inteiro que se refere à posição numérica da entrada no widget, contada do topo, começando com 0;
- a string `"active"`, que se refere à posição do menu que está atualmente sob o cursor;
- a string `"last"` que se refere ao último item do menu;
- a string consisting of `@` followed by an integer, as in `"@6"`, where the integer is interpreted as a y pixel coordinate in the menu's coordinate system;
- a string `"none"`, que indica nenhuma entrada de menu, mais frequentemente usada com menu.activate() para desativar todas as

entradas e, finalmente,
- uma string de texto cujo padrão corresponde ao rótulo da entrada do menu, conforme varrido da parte superior do menu para a parte inferior. Observe que este tipo de índice é considerado após todos os outros, o que significa que as correspondências para itens de menu rotulados como `last`, `active` ou `none` podem ser interpretadas como os literais acima, em vez disso.

Imagens

Imagens de diferentes formatos podem ser criadas por meio da subclasse correspondente de **`tkinter.Image`**:

- **`BitmapImage`** para imagens no formato XBM.
- **`PhotoImage`** para imagens nos formatos PGM, PPM, GIF e PNG. O suporte ao último foi adicionado a partir do Tk 8.6.

Qualquer tipo de imagem é criado através da opção `file` ou `data` (outras opções também estão disponíveis).

Alterado na versão 3.13: Adicionado o método **`copy_replace()`** à classe **`PhotoImage`** para copiar uma região de uma imagem para outra imagem, possivelmente com ampliação e subamostragem de pixels. Adiciona parâmetro *from_coords* para os métodos **`copy()`**, **`zoom()`** e **`subsample()`** da classe **`PhotoImage`**. Adiciona parâmetros *zoom* e *subsample* para o método **`copy()`** da classe **`PhotoImage`**.

The image object can then be used wherever an `image` option is supported by some widget (for example, labels, buttons, menus). In these cases, Tk will not keep a reference to the image. When the last Python reference to the image object is deleted, the image data is deleted as well, and Tk will display an empty box wherever the image was used.

Ver também

O pacote Pillow [https://python-pillow.org/] adiciona suporte para formatos como BMP, JPEG, TIFF e WebP, entre outros.

Referência

This section documents the classes, methods, functions and constants of the **`tkinter`** module. Most of them wrap Tcl/Tk commands; consult the official Tcl/Tk manual pages for the full list of widget options and further details.

exception tkinter.TclError

The exception raised when a call into the Tcl interpreter fails, for example when a widget is given an unknown option or an invalid value.

Base and mixin classes

class tkinter.Misc

The **`Misc`** class is a mix-in inherited by **`Tk`** and, through **`BaseWidget`**, by every widget. It provides the large set of methods common to all Tk objects: querying window information, managing event bindings and the event loop, controlling the keyboard focus and pointer grabs, accessing the selection, clipboard and option database, and assorted utility and introspection services. Because they are inherited, these methods are available on every widget and on the **`Tk`** application object, and are documented here once rather than repeated for each widget.

cget(*key*)

Return the current value of the configuration option named *key* for this widget, as a string. The expression `widget[key]` is equivalent and may be used instead.

configure(*cnf=None*, ***kw*)

Query or modify the configuration options of the widget. With no arguments, return a dictionary mapping every available option name to a tuple describing it (its name, X resource name, X resource class, default value and current value). If a single option name is given as a string, return the tuple for just that option. If one or more keyword arguments are given, or a dictionary is passed as *cnf*, set each named

option to the corresponding value; the expression `widget[key] = value` sets a single option in the same way.

`config()` is an alias of **`configure()`**.

keys()

Return a list of the names of all configuration options of this widget.

getboolean(*s*)

Interpret the string *s* as a Tcl boolean and return the corresponding **`bool`**. Tcl accepts values such as `'1'`, `'0'`, `'yes'`, `'no'`, `'true'` and `'false'`. Raise **`ValueError`** if *s* is not a valid boolean.

getdouble(*s*)

Interpret the string *s* as a Tcl floating-point number and return it as a **`float`**. Raise **`ValueError`** if *s* is not a valid number.

Adicionado na versão 3.5.

getint(*s*)

Interpret the string *s* as a Tcl integer and return it as an **`int`**. Raise **`ValueError`** if *s* is not a valid integer.

getvar(*name*)

Return the value of the Tcl global variable named *name*.

setvar(*name*, *value*)

Set the Tcl global variable named *name* to *value*.

The **`getvar()`** and **`setvar()`** methods give direct access to Tcl variables. In most code you will instead use a **`Variable`** subclass such as **`StringVar`** or **`IntVar`**, which wraps a Tcl variable and converts its value to and from a Python type.

register(*func*, *subst=None*, *needcleanup=1*)

Register the Python callable *func* as a Tcl command and return the name of the new command as a string. Whenever Tcl invokes that command, *func* is called; if *subst* is given, it is applied to the command's arguments first. This is the mechanism used internally to turn Python callbacks into the command names passed to Tk options such as *command*. Unless *needcleanup* is false, the command is deleted automatically when the widget is destroyed.

Alterado na versão 3.13: The arguments passed to *func* are no longer converted to strings.

deletecommand(*name*)

Delete the Tcl command named *name*, such as one previously returned by `register()`.

nametowidget(*name*)

Return the widget instance corresponding to the Tk pathname *name*.

send(*interp*, *cmd*, **args*)

Send the Tcl command *cmd*, with the given *args*, to the Tcl interpreter registered under the name *interp*, and return its result. This is not available on all platforms.

destroy()

Destroy this widget and all of its descendant widgets, and delete the Tcl commands associated with them.

tkraise(*aboveThis=None*)

Raise this widget in the stacking order so that it is drawn on top of its siblings. If *aboveThis* is given, the widget is moved to be just above it in the stacking order instead.

`lift()` is an alias of `tkraise()`.

lower(*belowThis=None*)

Lower this widget in the stacking order so that it is drawn beneath its siblings. If *belowThis* is given, the widget is moved to be just below it in the stacking order instead.

tkraise()/**lift()** and **lower()** are overridden by the **Canvas** widget, where they restack canvas items instead.

image_names()

Return the names of all images that currently exist in the Tcl interpreter.

This is overridden by the **Text** widget, where **image_names()** returns the names of its embedded images instead.

image_types()

Return the available image types, such as `'photo'` and `'bitmap'`.

grid_anchor(*anchor=None*)

Set the anchor that controls where the grid is placed inside this container when the container is larger than the grid and no row or column has a non-zero weight. *anchor* is one of the usual anchor strings, such as `'nw'` (the default) or `'center'`. Called with no argument, this method has no effect.

anchor() is an alias of **grid_anchor()**.

Adicionado na versão 3.3.

grid_bbox(*column=None*, *row=None*, *col2=None*, *row2=None*)

Return the bounding box, in pixels, of a region of the grid laid out in this container, as a 4-tuple `(xoffset, yoffset, width, height)`. With no arguments the bounding box of the whole grid is returned. If *column* and *row* are given, the box spans from the cell at row and column 0 to that cell; if *col2* and *row2* are also given, it spans from the cell (*column*, *row*) to the cell (*col2*, *row2*).

`bbox()` is an alias of **`grid_bbox()`**, except on `Canvas`, `Listbox`, `Spinbox`, `Text`, `ttk.Entry` and `ttk.Treeview`, which provide their own **`bbox()`** method.

grid_columnconfigure(*index*, *cnf={}*, ***kw*)

Query or set the properties of the column (or columns) *index* of the grid managed by this container. *index* may be a column number; when setting options it may also be a list of column numbers, the string `'all'` to affect every column, or a child widget whose occupied columns are affected. The supported options are:

minsize

The column's minimum size, in pixels.

weight

An integer setting how much of any extra space is apportioned to the column. A weight of `0` keeps the column at its requested size, and a column of weight two grows twice as fast as a column of weight one.

uniform

The name of a uniform group. Columns sharing a non-empty group name are kept in sizes that are strictly proportional to their weights.

pad

Extra space, in pixels, added to the largest widget in the column when computing the column's size.

With a single option name, return that option's value; with no options, return a dictionary of all of them.

`columnconfigure()` is an alias of **`grid_columnconfigure()`**.

grid_rowconfigure(*index*, *cnf={}*, ***kw*)

Query or set the properties of the row (or rows) *index* of the grid managed by this container. *index* is interpreted as for `grid_columnconfigure()`, and the supported options (*minsize*,

weight, *uniform* and *pad*) are the same, applied to a row instead of a column.

`rowconfigure()` is an alias of **`grid_rowconfigure()`**.

grid_location(*x*, *y*)

Return the `(column, row)` of the grid cell that contains the pixel at position (*x*, *y*), given in pixels relative to this container. For locations above or to the left of the grid, `-1` is returned for the corresponding coordinate.

grid_propagate()

grid_propagate(*flag*)

Enable or disable geometry propagation for this container when it manages its children with the grid geometry manager. When *flag* is true, the container resizes itself to fit the requested sizes of its children; when it is false, its size is left under your control. Called with no argument, return the current setting as a boolean.

grid_size()

Return the size of the grid managed by this container as a `(columns, rows)` tuple.

`size()` is an alias of **`grid_size()`**, except on the `Listbox` widget, which provides its own **`size()`** method.

grid_slaves(*row=None*, *column=None*)

Return a list of the child widgets managed in this container's grid, most recently managed first. If *row* or *column* is given, only the children in that row or column are returned.

grid_content(*row=None*, *column=None*)

Same as `grid_slaves()`: return the child widgets managed in this container's grid, optionally restricted to a given *row* or *column*.

Adicionado na versão 3.15.

pack_propagate()

pack_propagate(*flag*)

Enable or disable geometry propagation for this container when it manages its children with the pack geometry manager. When *flag* is true, the container resizes itself to fit the requested sizes of its children; when it is false, its size is left under your control. Called with no argument, return the current setting as a boolean.

`propagate()` is an alias of **`pack_propagate()`**.

pack_slaves()

Return a list of the child widgets managed by this container with the pack geometry manager, in packing order.

`slaves()` is an alias of **`pack_slaves()`**.

pack_content()

Same as `pack_slaves()`: return the child widgets managed by this container with the pack geometry manager, in packing order.

`content()` is an alias of **`pack_content()`**.

Adicionado na versão 3.15.

place_slaves()

Return a list of the child widgets managed by this container with the place geometry manager.

place_content()

Same as `place_slaves()`: return the child widgets managed by this container with the place geometry manager.

Adicionado na versão 3.15.

The methods with the `bind` and `unbind` prefixes associate event patterns with callbacks and remove those associations.

bind(*sequence=None*, *func=None*, *add=None*)

Bind the event pattern *sequence* on this widget to the callable *func*.

sequence is an event pattern, such as `'<Button-1>'` (a mouse click) or `'<KeyPress-a>'`, optionally a concatenation of several such patterns that must occur shortly after one another. When the event occurs, *func* is called with an **`Event`** instance describing it as its only argument; if *func* returns the string `'break'`, no further bindings for the event are invoked.

If *add* is true, *func* is added to any functions already bound to *sequence*; otherwise it replaces them. The binding applies only to this widget.

`bind()` returns a string identifier (a *funcid*) that can later be passed to **`unbind()`** to remove the binding without leaking the associated Tcl command.

If *func* is omitted, return the binding currently associated with *sequence*; if *sequence* is also omitted, return a list of all the sequences for which bindings exist on this widget.

bind_class(*className*, *sequence=None*, *func=None*, *add=None*)

Like **`bind()`**, but bind *func* to the binding tag *className* rather than to a single widget, so that the binding applies to every widget having that tag. *className* is usually the name of a widget class, such as `'Button'`, in which case the binding affects all widgets of that class. The set of binding tags for a widget can be inspected and changed with **`bindtags()`**.

The remaining arguments and the return value are as for **`bind()`**.

bind_all(*sequence=None*, *func=None*, *add=None*)

Like **`bind()`**, but bind *func* to the special binding tag `'all'`, so that the binding applies to every widget in the application.

The remaining arguments and the return value are as for **`bind()`**.

unbind(*sequence*, *funcid=None*)

Remove bindings for the event pattern *sequence* on this widget.

If *funcid* is given, only the function identified by it (a value returned from a previous call to **`bind()`**) is removed, and its associated Tcl command is deleted. Otherwise all bindings for *sequence* are destroyed, leaving it unbound.

Alterado na versão 3.13: If *funcid* is given, only that callback is unbound; other callbacks bound to *sequence* are kept.

unbind_class(*className*, *sequence*)

Remove all bindings for the event pattern *sequence* from the binding tag *className*. See **`bind class()`**.

unbind_all(*sequence*)

Remove all bindings for the event pattern *sequence* from the special binding tag `'all'`. See **`bind all()`**.

bindtags(*tagList=None*)

If *tagList* is omitted, return a tuple of the binding tags associated with this widget. When an event occurs in a widget, it is applied to each of the widget's binding tags in order, and for each tag the most specific matching binding is executed. By default a widget has four binding tags: its own pathname, its widget class, the pathname of its nearest toplevel ancestor, and `'all'`, in that order.

If *tagList* is given, it must be a sequence of strings; the widget's binding tags are set to its elements, which determines the order in which bindings are evaluated.

The methods with the `event_` prefix define virtual events and generate events programmatically.

event_add(*virtual*, **sequences*)

Associate the virtual event *virtual*, whose name has the form `'<<Paste>>'`, with each of the physical event patterns given by

sequences, so that the virtual event triggers whenever any of them occurs. If *virtual* is already defined, the new sequences are added to its existing ones.

event_delete(*virtual*, **sequences*)

Remove each of *sequences* from those associated with the virtual event *virtual*. Sequences that are not currently associated with *virtual* are ignored. If no *sequences* are given, all physical event sequences are removed, so that *virtual* no longer triggers.

event_generate(*sequence*, ***kw*)

Generate the event *sequence* on this widget and arrange for it to be processed just as if it had come from the window system. *sequence* must be a single event pattern, such as `'<Button-1>'` or `'<<Paste>>'`, not a concatenation of several. Keyword arguments specify additional fields of the event, for example *x* and *y* for the pointer position, or *when* to control when the event is processed; refer to the Tk `event` manual page for the full list.

event_info(*virtual=None*)

If *virtual* is omitted, return a tuple of all the virtual events that are currently defined. If *virtual* is given, return a tuple of the physical event sequences currently associated with it, or an empty tuple if it is not defined.

The methods with the `after` prefix schedule callbacks to run after a delay or when the application is idle.

after(*ms*, *func=None*, **args*, ***kw*)

Schedule the callable *func* to be called after *ms* milliseconds, with *args* and *kw* passed to it as positional and keyword arguments. Return an identifier that can be passed to **`after cancel()`** to cancel the call.

If *func* is omitted, sleep for *ms* milliseconds instead, processing no events during that time, and return `None`.

Alterado na versão 3.10: *func* can now be any callable object, not only a function.

Alterado na versão 3.14: Keyword arguments are now passed to *func*.

after_cancel(*id*)

Cancel a callback previously scheduled with `after()` or `after_idle()`. *id* must be an identifier returned by one of those methods; passing a value that is not such an identifier raises `ValueError`. If the callback has already run or been cancelled, this has no effect.

Alterado na versão 3.7: Passing `None` (or any false value) as *id* now raises `ValueError`.

after_idle(*func*, **args*, ***kw*)

Schedule the callable *func* to be called, with *args* and *kw* passed to it, when the Tk main loop next becomes idle, that is, when it has no other events to process. Return an identifier that can be passed to `after_cancel()` to cancel the call.

Alterado na versão 3.14: Keyword arguments are now passed to *func*.

after_info(*id=None*)

If *id* is omitted, return a tuple of the identifiers of all callbacks currently scheduled with `after()` and `after_idle()` for this interpreter.

If *id* is given, it must identify a callback that has not yet run or been cancelled, and the return value is a tuple `(script, type)`, where *script* refers to the function to be called and *type* is either `'idle'` or `'timer'`. A `TclError` is raised if *id* does not exist.

Adicionado na versão 3.13.

mainloop(*n=0*)

Enter the Tk event loop, which processes events until all windows are destroyed. This is normally called once, on the root window, to run the application.

quit()

Quit the Tcl interpreter, causing `mainloop()` to return.

update()

Enter the event loop until all pending events, including idle callbacks, have been processed. This brings the display up to date and handles any events that are already queued, then returns.

update_idletasks()

Enter the event loop until all pending idle callbacks have been called. This updates the display of windows, for example after geometry changes, but does not process events caused by the user.

wait_variable(*name*)

Wait until the Tcl variable *name* is modified, continuing to process events in the meantime so that the application stays responsive. *name* is usually a `Variable` instance, such as an `IntVar` or `StringVar`.

`waitvar()` is an alias of **`wait_variable()`**.

wait_window(*window=None*)

Wait until *window* is destroyed, continuing to process events in the meantime. If *window* is omitted, this widget is used. This is typically used to wait for the user to finish interacting with a dialog box.

wait_visibility(*window=None*)

Wait until the visibility state of *window* changes, for example when it first appears on the screen, continuing to process events in the meantime. If *window* is omitted, this widget is used. This is

typically used to wait for a newly created window to become visible before acting on it.

The methods with the `focus_` prefix manage the keyboard focus.

focus()

Direct the keyboard input focus for this widget's display to this widget. If the application does not currently have the input focus on this widget's display, the widget is remembered as the focus window for its top level, and the focus will be redirected to it the next time the window manager gives the focus to the top level. **`focus()`** is an alias of **`focus_set()`**, except on the **`Canvas`** and **`ttk.Treeview`** widgets, which provide their own **`focus()`** method.

focus_force()

Direct the keyboard input focus to this widget even if the application does not currently have the input focus for the widget's display. This method should be used sparingly, if at all; normally an application should wait for the window manager to give it the focus rather than claiming it.

focus_get()

Return the widget that currently has the keyboard focus in the application, or `None` if no widget in the application has the focus. Use **`focus_displayof()`** to work correctly with several displays.

focus_displayof()

Return the widget that currently has the keyboard focus on the display where this widget is located, or `None` if no widget in the application has the focus on that display.

focus_lastfor()

Return the most recent widget to have had the keyboard focus among all the widgets in the same top level as this widget; this is the widget that will receive the focus the next time the window manager gives the focus to the top level. If no widget in that top level has

ever had the focus, or if the most recent focus widget has been deleted, the top level itself is returned.

tk_focusFollowsMouse()

Reconfigure Tk to use an implicit focus model in which the focus is set to a widget whenever the mouse pointer enters it. This cannot easily be disabled once enabled.

tk_focusNext()

Return the next widget after this one in the keyboard traversal order, or `None` if there is none. The traversal order goes first to the next child, then recursively to the children of that child, and then to the next sibling higher in the stacking order. A widget is skipped if its `takefocus` option is set to `0`. This method is used in the default bindings for the `Tab` key.

tk_focusPrev()

Return the previous widget before this one in the keyboard traversal order, or `None` if there is none. See **`tk focusNext()`** for how the order is defined. This method is used in the default bindings for the `Shift-Tab` key.

The methods with the `grab_` prefix set and query the input grab, which directs all input events to a single widget.

grab_set()

Set a local grab on this widget. A grab confines pointer events to this widget and its descendants: while the pointer is outside the widget's subtree, button presses and releases and pointer motion are reported to the grab widget, and windows outside the subtree become insensitive until the grab is released. A local grab affects only the grabbing application. Any grab previously set by this application on the widget's display is automatically released. Setting a grab is the usual way to make a dialog modal: while the grab is in effect the user cannot interact with the other windows of the application.

grab_set_global()

Set a global grab on this widget. A global grab is like the local grab set by **grab_set()**, but it locks out all other applications on the screen, so that only this widget's subtree is sensitive to pointer events, and it also grabs the keyboard. Use with caution: it is easy to render a display unusable with a global grab, since other applications stop receiving events until it is released.

grab_release()

Release the grab on this widget if there is one; otherwise do nothing.

grab_current()

Return the widget that currently holds the grab in this application for this widget's display, or `None` if there is no such widget.

grab_status()

Return `None` if no grab is currently set on this widget, `"local"` if a local grab is set, or `"global"` if a global grab is set.

The methods with the `selection_` prefix retrieve and manage the X selection.

selection_clear(**kw)

Clear the X selection, so that no window owns it anymore. The selection to clear is given by the keyword argument *selection*, an atom name such as `'PRIMARY'` or `'CLIPBOARD'`; it defaults to `PRIMARY`. The *displayof* keyword argument names a widget that determines the display on which to operate, and defaults to this widget.

This is overridden by the **Entry**, **Listbox** and **Spinbox** widgets, where **selection_clear()** clears the widget's own selection instead.

selection_get(**kw)

Return the contents of the current X selection. The keyword argument *selection* names the selection and defaults to `PRIMARY`. The keyword argument *type* specifies the form in which the data is to be returned (the desired conversion target), an atom name such as `'STRING'` or `'FILE_NAME'`; it defaults to `STRING`, except on X11, where `UTF8_STRING` is tried first and `STRING` is used as a fallback. The *displayof* keyword argument names a widget that determines the display from which to retrieve the selection, and defaults to this widget.

selection_handle(*command*, ***kw*)

Register *command* as a handler to supply the X selection owned by this widget when another application requests it. When the selection is retrieved, *command* is called with two arguments, the starting character offset and the maximum number of characters to return, and must return at most that many characters of the selection starting at that offset; for very long selections it is called repeatedly with increasing offsets. The keyword argument *selection* names the selection (default `PRIMARY`) and the keyword argument *type* gives the form of the selection that the handler supplies (such as `'STRING'` or `'FILE_NAME'`, default `STRING`).

selection_own(***kw*)

Make this widget the owner of the X selection on its display. The previous owner, if any, is notified that it has lost the selection. The keyword argument *selection* names the selection and defaults to `PRIMARY`.

selection_own_get(***kw*)

Return the widget in this application that owns the X selection on the display containing this widget, or `None` if no widget in this application owns the selection. The keyword argument *selection* names the selection and defaults to `PRIMARY`. The *displayof* keyword argument names a widget that determines the display to query, and defaults to this widget.

The methods with the `clipboard_` prefix manage the clipboard.

clipboard_append(*string*, ***kw*)

Append *string* to the Tk clipboard and claim ownership of the clipboard on this widget's display. Before appending, the clipboard should be emptied with **`clipboard clear()`**; all appends should be completed before returning to the event loop so that the clipboard is updated atomically. The keyword argument *type* specifies the form of the data, an atom name such as `'STRING'` or `'FILE_NAME'` (default `STRING`), and the keyword argument *format* specifies the representation used to transmit it (default `STRING`). The *displayof* keyword argument names a widget that determines the target display, and defaults to this widget. The contents can be retrieved with **`clipboard get()`** or **`selection get()`**.

clipboard_clear(***kw*)

Claim ownership of the clipboard on this widget's display and remove any previous contents. The *displayof* keyword argument names a widget that determines the target display, and defaults to this widget.

clipboard_get(***kw*)

Retrieve data from the clipboard on this widget's display. The keyword argument *type* specifies the form in which the data is to be returned, an atom name such as `'STRING'` or `'FILE_NAME'`; it defaults to `STRING`, except on X11, where `UTF8_STRING` is tried first and `STRING` is used as a fallback. The *displayof* keyword argument names a widget that determines the display, and defaults to the root window of the application. This is equivalent to `selection_get(selection='CLIPBOARD')`.

The methods with the `option_` prefix query and modify the Tk option database.

option_add(*pattern*, *value*, *priority=None*)

Add an option to the Tk option database that associates *value* with *pattern*. *pattern* consists of names and/or classes separated by asterisks or dots, in the usual X format. *priority* is an integer

between 0 and 100, or one of the symbolic names `'widgetDefault'` (20), `'startupFile'` (40), `'userDefault'` (60), or `'interactive'` (80); it defaults to `interactive`.

option_clear()

Clear the Tk option database. Default options from the **`RESOURCE_MANAGER`** property or the `.Xdefaults` file are reloaded automatically the next time an option is added to or removed from the database.

option_get(*name*, *className*)

Return the value of the option matching this widget under *name* and *className* from the Tk option database, or an empty string if there is no matching entry. When several entries match, the one with the highest priority is returned, and among entries of equal priority the most recently added one.

option_readfile(*fileName*, *priority=None*)

Read the file named *fileName*, which should have the standard format for an X resource database such as `.Xdefaults`, and add all the options it specifies to the Tk option database. *priority* is interpreted as for **`option add()`** and defaults to `interactive`.

bell(*displayof=0*)

Ring the bell on the display for this widget, using the display's current bell-related settings, and reset the screen saver for the screen. If *displayof* is given as a widget, the bell is rung on that widget's display instead.

tk_setPalette(*background*, /)

tk_setPalette(**args*, ***kw*)

Set a new color scheme for all Tk widget elements. Existing widgets are updated and the option database is changed so that future widgets use the new colors. A single color argument is taken as the normal background color, from which a complete palette is computed. Alternatively, the arguments may be given as keyword

name/*value* pairs naming individual options in the option database. The recognized option names are `activeBackground`, `activeForeground`, `background`, `disabledForeground`, `foreground`, `highlightBackground`, `highlightColor`, `insertBackground`, `selectColor`, `selectBackground`, `selectForeground`, and `troughColor`; reasonable defaults are computed for any that are not specified.

tk_bisque()

Restore the application's colors to the light brown (bisque) color scheme used in Tk 3.6 and earlier versions. Provided for backward compatibility.

tk_strictMotif(*boolean=None*)

Query or set whether Tk's look and feel should strictly adhere to Motif. A true *boolean* value enables strict Motif compliance (for example, no color change when the mouse passes over a slider). Return the resulting setting.

The methods with the `busy_` prefix manage the busy state of a window, which shows a busy cursor and ignores user input.

tk_busy_hold(***kw*)

Make this widget appear busy. A transparent window is placed in front of the widget, so that it and all of its descendants in the widget hierarchy are blocked from pointer events and display a busy cursor. Normally **`update()`** should be called immediately afterwards to ensure that the hold operation is in effect before the application starts its processing.

The only supported configuration option is *cursor*, the cursor to be displayed while the widget is busy; it may have any of the values accepted by **`configure()`**.

`busy_hold()`, **`busy()`** and **`tk_busy()`** are aliases of **`tk_busy_hold()`**.

Adicionado na versão 3.13.

tk_busy_configure(*cnf=None*, ***kw*)

Query or modify the configuration options of the busy window. The widget must have been previously made busy by **tk_busy_hold()**. With no arguments, return a dictionary describing all of the available options; if *cnf* is the name of an option, return a tuple describing that one option. Otherwise set the given options to the given values. Options may have any of the values accepted by **tk_busy_hold()**.

The option database is referenced through the widget name or class. For example, if a **Frame** widget named `frame` is to be made busy, the busy cursor can be specified for it by either of the calls:

```
w.option_add('*frame.busyCursor', 'gumby')
w.option_add('*Frame.BusyCursor', 'gumby')
```

busy_configure(), **busy_config()** and **tk_busy_config()** are aliases of **tk_busy_configure()**.

Adicionado na versão 3.13.

tk_busy_cget(*option*)

Return the current value of the busy configuration *option*. The widget must have been previously made busy by **tk_busy_hold()**, and *option* may have any of the values accepted by that method.

busy_cget() is an alias of **tk_busy_cget()**.

Adicionado na versão 3.13.

tk_busy_forget()

Make this widget no longer busy, releasing the resources (including the transparent window) allocated when it was made busy. User events will again be received by the widget. These resources are also released when the widget is destroyed.

`busy_forget()` is an alias of **`tk_busy_forget()`**.

Adicionado na versão 3.13.

tk_busy_status()

Return `True` if the widget is currently busy, `False` otherwise.

`busy_status()` is an alias of **`tk_busy_status()`**.

Adicionado na versão 3.13.

tk_busy_current(*pattern=None*)

Return a list of widgets that are currently busy. If *pattern* is given, only busy widgets whose path names match the pattern are returned.

`busy_current()` is an alias of **`tk_busy_current()`**.

Adicionado na versão 3.13.

The methods with the `winfo_` prefix retrieve information about windows managed by Tk.

winfo_atom(*name*, *displayof=0*)

Return the integer identifier for the atom whose name is *name*, creating a new atom if none exists. If *displayof* is given, the atom is looked up on the display of that window; otherwise it is looked up on the display of the application's main window.

winfo_atomname(*id*, *displayof=0*)

Return the textual name for the atom whose integer identifier is *id*. This is the inverse of `winfo_atom()`. If *displayof* is given, the identifier is looked up on the display of that window; otherwise it is looked up on the display of the application's main window.

winfo_cells()

Return the number of cells in the colormap for the widget.

winfo_children()

Return a list containing the widgets that are children of the widget, in stacking order from lowest to highest. Toplevel windows are returned as children of their logical parents.

winfo_class()

Return the class name of the widget.

winfo_colormapfull()

Return `True` if the colormap for the widget is known to be full, `False` otherwise.

winfo_containing(*rootX*, *rootY*, *displayof=0*)

Return the widget containing the point given by *rootX* and *rootY*, or `None` if no window in this application contains the point. The coordinates are in screen units in the coordinate system of the root window. If *displayof* is given, the coordinates refer to the screen containing that window; otherwise they refer to the screen of the application's main window.

winfo_depth()

Return the depth of the widget, that is, the number of bits per pixel.

winfo_exists()

Return true if the widget exists, false otherwise.

winfo_fpixels(*number*)

Return a floating-point value giving the number of pixels in the widget corresponding to the screen distance *number* (for example, `"2.0c"` or `"1i"`). The result may be fractional; for a rounded integer value use **`winfo_pixels()`**.

winfo_geometry()

Return the geometry of the widget, in the form `widthxheight+x+y`. All dimensions are in pixels. An offset can be negative; see **`geometry()`**.

winfo_height()

Return the height of the widget in pixels. When a window is first created its height is 1 pixel; it is eventually changed by a geometry manager. See also `winfo reqheight()`.

winfo_id()

Return a low-level platform-specific identifier for the widget. On Unix this is the X window identifier, and on Windows it is the window handle.

winfo_interps(*displayof=0*)

Return a tuple of the names of all Tcl interpreters currently registered for a particular display. If *displayof* is given, the return value refers to the display of that window; otherwise it refers to the display of the application's main window.

winfo_ismapped()

Return true if the widget is currently mapped, false otherwise.

winfo_manager()

Return the name of the geometry manager currently responsible for the widget, or an empty string if it is not managed by any geometry manager.

winfo_name()

Return the widget's name within its parent, as opposed to its full path name.

winfo_parent()

Return the path name of the widget's parent, or an empty string if the widget is the main window of the application.

winfo_pathname(*id*, *displayof=0*)

Return the path name of the window whose identifier is *id*. If *displayof* is given, the identifier is looked up on the display of that

window; otherwise it is looked up on the display of the application's main window.

winfo_pixels(*number*)

Return the number of pixels in the widget corresponding to the screen distance *number* (for example, `"2.0c"` or `"1i"`). The result is rounded to the nearest integer; for a fractional result use `winfo_fpixels()`.

winfo_pointerx()

Return the pointer's *x* coordinate, in pixels, relative to the screen's root window (or virtual root, if one is in use). Return `-1` if the pointer is not on the same screen as the widget.

winfo_pointerxy()

Return the pointer's coordinates as an `(x, y)` tuple, in pixels, relative to the screen's root window (or virtual root, if one is in use). Both coordinates are `-1` if the pointer is not on the same screen as the widget.

winfo_pointery()

Return the pointer's *y* coordinate, in pixels, relative to the screen's root window (or virtual root, if one is in use). Return `-1` if the pointer is not on the same screen as the widget.

winfo_reqheight()

Return the widget's requested height in pixels. This is the value used by the widget's geometry manager to compute its geometry.

winfo_reqwidth()

Return the widget's requested width in pixels. This is the value used by the widget's geometry manager to compute its geometry.

winfo_rgb(*color*)

Return an `(r, g, b)` tuple of the red, green, and blue intensities, in the range 0 to 65535, that correspond to *color* in the widget. *color*

may be specified in any of the forms acceptable for a color option.

winfo_rootx()

Return the *x* coordinate, in the root window of the screen, of the upper-left corner of the widget's border (or of the widget itself if it has no border).

winfo_rooty()

Return the *y* coordinate, in the root window of the screen, of the upper-left corner of the widget's border (or of the widget itself if it has no border).

winfo_screen()

Return the name of the screen associated with the widget, in the form `displayName.screenIndex`.

winfo_screencells()

Return the number of cells in the default colormap for the widget's screen.

winfo_screendepth()

Return the depth of the root window of the widget's screen, that is, the number of bits per pixel.

winfo_screenheight()

Return the height of the widget's screen in pixels.

winfo_screenmmheight()

Return the height of the widget's screen in millimeters.

winfo_screenmmwidth()

Return the width of the widget's screen in millimeters.

winfo_screenvisual()

Return the default visual class for the widget's screen, one of `"directcolor"`, `"grayscale"`, `"pseudocolor"`, `"staticcolor"`,

`"staticgray"`, or `"truecolor"`.

winfo_screenwidth()

Return the width of the widget's screen in pixels.

winfo_server()

Return a string containing information about the server for the widget's display. The exact format of this string may vary from platform to platform.

winfo_toplevel()

Return the top-of-hierarchy window containing the widget. In standard Tk this is always a **`Toplevel`** widget.

winfo_viewable()

Return true if the widget and all of its ancestors up through the nearest toplevel window are mapped, false otherwise.

winfo_visual()

Return the visual class for the widget, one of `"directcolor"`, `"grayscale"`, `"pseudocolor"`, `"staticcolor"`, `"staticgray"`, or `"truecolor"`.

winfo_visualid()

Return the X identifier for the visual for the widget.

winfo_visualsavailable(*includeids=False*)

Return a list describing the visuals available for the widget's screen. Each item consists of a visual class (see **`winfo visual()`**) followed by an integer depth. If *includeids* is true, the X identifier for the visual is also included.

winfo_vrootheight()

Return the height of the virtual root window associated with the widget if there is one; otherwise return the height of the widget's screen.

winfo_vrootwidth()

Return the width of the virtual root window associated with the widget if there is one; otherwise return the width of the widget's screen.

winfo_vrootx()

Return the *x* offset of the virtual root window associated with the widget, relative to the root window of its screen. This is normally zero or negative, and is `0` if there is no virtual root window.

winfo_vrooty()

Return the *y* offset of the virtual root window associated with the widget, relative to the root window of its screen. This is normally zero or negative, and is `0` if there is no virtual root window.

winfo_width()

Return the width of the widget in pixels. When a window is first created its width is 1 pixel; it is eventually changed by a geometry manager. See also `winfo_reqwidth()`.

winfo_x()

Return the *x* coordinate, in the widget's parent, of the upper-left corner of the widget's border (or of the widget itself if it has no border).

winfo_y()

Return the *y* coordinate, in the widget's parent, of the upper-left corner of the widget's border (or of the widget itself if it has no border).

info_patchlevel()

Return the Tcl/Tk patch level as a named tuple with the same five fields as `sys.version_info`: *major*, *minor*, *micro*, *releaselevel* and *serial*. *releaselevel* is `'alpha'`, `'beta'` or `'final'`. Converting it to a string gives the version in the usual Tcl/Tk notation, for example `'9.0.3'` for a final release or `'9.1b2'` for a pre-release.

Adicionado na versão 3.11.

class tkinter.Wm

The **Wm** mixin provides access to the window manager, allowing an application to control such things as the title, geometry and icon of a top-level window, the way it is resized, and how it responds to window manager protocols. It is mixed into **Tk** and **Toplevel**, so its methods are available on every top-level window. Each method has two equivalent spellings: a short name and a `wm_`-prefixed name (for example, **title()** and **wm_title()**). See also The window manager.

aspect(*minNumer=None*, *minDenom=None*, *maxNumer=None*, *maxDenom=None*)

Constrain the aspect ratio (the ratio of width to height) of the window. If all four arguments are given, the window manager keeps the ratio between `minNumer/minDenom` and `maxNumer/maxDenom`; passing empty strings removes any existing restriction. With no arguments, return a tuple of the four current values, or `None` if no aspect restriction is in effect. **wm_aspect()** is an alias of **aspect()**.

attributes(**args*, *return_python_dict=False*, ***kwargs*)

Query or set platform-specific attributes of the window. With no arguments, return the platform-specific flags and their values; pass *return_python_dict* as true to get them as a dictionary. A single option name such as `'alpha'` returns the value of that option, and options are set using keyword arguments (`alpha=0.5`).

The available attributes differ by platform. All platforms support:

alpha

The window's opacity, from `0.0` (fully transparent) to `1.0` (opaque). Where transparency is unsupported the value stays at `1.0`.

appearance

Whether the window is rendered in dark mode on Windows and macOS: `'auto'`, `'light'` or `'dark'` (this has no effect on X11).

fullscreen
: Whether the window takes up the entire screen and has no borders.

topmost
: Whether the window is displayed above all other windows.

Windows additionally supports:

disabled
: Whether the window is in a disabled state.

toolwindow
: Whether the window uses the tool window style.

transparentcolor
: The color that is made fully transparent, or an empty string for none.

macOS additionally supports:

class
: Whether the underlying Aqua window is an `nswindow` or an `nspanel`; this can only be set before the window is created.

modified
: The modification state shown by the window's close button and proxy icon.

notify
: Whether the application's dock icon bounces to request attention.

stylemask
: The style mask of the underlying Aqua window, given as a list of bit names such as `titled` or `resizable`.

tabbingid
: The identifier of the tab group that the window belongs to.

tabbingmode
: Whether the window may be opened as a tab: `'auto'`, `'preferred'` or `'disallowed'`.

titlepath
: The path of the file represented by the window's proxy icon.

transparent
: Whether the content area is transparent and the window shadow is turned off.

X11 additionally supports:

"type"
: The window type, or a list of types in order of preference, that the window manager should use to interpret the window, such as `'dialog'` or `'splash'`.

zoomed
: Whether the window is maximized.

Nota

Tk 8.6 added the *type* attribute, and Tk 9.0 added the *appearance*, *class*, *stylemask*, *tabbingid* and *tabbingmode* attributes.

On X11 changes are applied asynchronously, so a queried value may not yet reflect the most recent request. `wm attributes()` is an alias of **`attributes()`**.

Alterado na versão 3.13: A single attribute may now be queried by name without the leading `-`, and attributes may be set using keyword arguments. The *return_python_dict* parameter was added.

Descontinuado desde a versão 3.13: Setting an attribute by passing the option name (with a leading `-`) and its value as two positional arguments, as in `w.attributes('-alpha', 0.5)`, is deprecated; use keyword arguments instead.

client(*name=None*)

Store *name*, which should be the name of the host on which the application is running, in the window's `WM_CLIENT_MACHINE` property for use by the window or session manager. An empty string deletes the property. With no argument, return the last name set, or an empty string. `wm client()` is an alias of **`client()`**.

colormapwindows(**wlist*)

Manipulate the `WM_COLORMAP_WINDOWS` property, which tells the window manager about windows that have private colormaps. If *wlist* is given, overwrite the property with those windows (their order is a priority order for installing colormaps). With no arguments, return the list of windows currently named in the property. `wm colormapwindows()` is an alias of **`colormapwindows()`**.

command(*value=None*)

Store *value* in the window's `WM_COMMAND` property for use by the window or session manager; it should be a list giving the words of the command used to invoke the application. An empty string deletes the property. With no argument, return the last value set, or an empty string. `wm command()` is an alias of **`command()`**.

deiconify()

Display the window in normal (non-iconified) form by mapping it. If the window has never been mapped, this ensures it appears de-iconified when it is first mapped. On Windows the window is also raised and given the focus. `wm deiconify()` is an alias of **`deiconify()`**.

focusmodel(*model=None*)

Set or query the focus model for the window. *model* is either `'active'` (the window claims the input focus for itself or its descendants, even when the focus is in another application) or `'passive'` (the window relies on the window manager to give it the focus). With no argument, return the current model. The default is

`'passive'`, which is what the **`focus()`** command assumes. `wm_focusmodel()` is an alias of **`focusmodel()`**.

forget(*window*)

Unmap *window* from the screen so that it is no longer managed by the window manager. A `Toplevel` is then treated like a `Frame`, although its `-menu` configuration is remembered and the menu reappears if the widget is managed again. `wm_forget()` is an alias of **`forget()`**.

Not to be confused with `Pack.forget()`.

Adicionado na versão 3.3.

frame()

Return the platform-specific window identifier for the outermost decorative frame containing the window, if the window manager has reparented it into such a frame; otherwise return the identifier of the window itself. `wm_frame()` is an alias of **`frame()`**.

geometry(*newGeometry=None*)

Set or query the geometry of the window. *newGeometry* has the form `=widthxheight+x+y`, where any of `=`, `widthxheight` and the `+x+y` position may be omitted. *width* and *height* are in pixels (or grid units for a gridded window); a position preceded by `+` is measured from the left or top edge of the screen and one preceded by `-` from the right or bottom edge. An offset can be negative, as in `'200x100+-9+-8'`, when the window edge is positioned beyond the corresponding screen edge. An empty string cancels any user-specified geometry, letting the window revert to its natural size. With no argument, return the current geometry as a string of the form `'200x200+10+10'`. `wm_geometry()` is an alias of **`geometry()`**.

grid(*baseWidth=None*, *baseHeight=None*, *widthInc=None*, *heightInc=None*)

Manage the window as a gridded window and define the relationship between grid units and pixels. *baseWidth* and *baseHeight* are the numbers of grid units for the window's internally requested size, and *widthInc* and *heightInc* are the pixel sizes of a horizontal and vertical grid unit. Empty strings turn off gridded management. With no arguments, return a tuple of the four current values, or `None` if the window is not gridded. `wm_grid()` is an alias of **`grid()`**.

Not to be confused with the grid geometry manager `Grid.grid()`.

group(*pathName=None*)

Set or query the leader of a group of related windows. *pathName* gives the path name of the group leader; the window manager may, for example, unmap all windows in the group when the leader is iconified. An empty string removes the window from any group. With no argument, return the path name of the current group leader, or an empty string. `wm_group()` is an alias of **`group()`**.

iconbitmap(*bitmap=None*, *default=None*)

Set or query the bitmap used by the window manager for the window's icon. *bitmap* names a bitmap in one of the standard forms accepted by Tk; an empty string cancels the current icon bitmap. With no argument, return the name of the current icon bitmap, or an empty string. On Windows the *default* argument names an icon (for example an `.ico` file) applied to all top-level windows that have no icon of their own. `wm_iconbitmap()` is an alias of **`iconbitmap()`**.

iconify()

Iconify the window. If the window has not yet been mapped for the first time, arrange for it to appear in the iconified state when it is eventually mapped. `wm_iconify()` is an alias of **`iconify()`**.

iconmask(*bitmap=None*)

Set or query the bitmap used as a mask for the icon (see `iconbitmap()`). Where the mask is zero no icon is displayed; where it is one, the corresponding bits of the icon bitmap are shown. An

empty string cancels the current mask. With no argument, return the name of the current icon mask, or an empty string. `wm_iconmask()` is an alias of **`iconmask()`**.

iconname(*newName=None*)

Set or query the name displayed by the window manager inside the window's icon. With no argument, return the current icon name, or an empty string if none has been set (in which case the window manager normally displays the window's title). `wm_iconname()` is an alias of **`iconname()`**.

iconphoto(*default=False, *images*)

Set the titlebar icon for the window from one or more **`PhotoImage`** objects given in *images*. Several images of different sizes (for example 16x16 and 32x32) may be supplied so that the window manager can choose an appropriate one. The image data is taken as a snapshot at the time of the call; later changes to the images are not reflected. If *default* is true, the icon is also applied to all top-level windows created in the future. On macOS only the first image is used. `wm_iconphoto()` is an alias of **`iconphoto()`**.

Adicionado na versão 3.3.

iconposition(*x=None, y=None*)

Set or query a hint to the window manager about where the window's icon should be positioned. Empty strings cancel an existing hint. With no arguments, return a tuple of the two current values, or `None` if no hint is in effect. `wm_iconposition()` is an alias of **`iconposition()`**.

iconwindow(*pathName=None*)

Set or query the window used as the icon for the window. When the window is iconified, *pathName* is mapped to serve as its icon and unmapped again when it is de-iconified. An empty string cancels the association. With no argument, return the path name of the current icon window, or an empty string. Not all window managers support

icon windows, and the concept is meaningless on non-X11 platforms. `wm_iconwindow()` is an alias of **`iconwindow()`**.

manage(*widget*)

Make *widget* a stand-alone top-level window, decorated by the window manager with a title bar and so on. Only `Frame`, `LabelFrame` and `Toplevel` widgets may be used (the `tkinter.ttk` versions are **not** accepted); passing any other widget type raises an error. `wm_manage()` is an alias of **`manage()`**.

Adicionado na versão 3.3.

maxsize(*width=None*, *height=None*)

Set or query the maximum permissible dimensions of the window, in pixels (or grid units for a gridded window). The window manager restricts the window to be no larger than *width* and *height*. With no arguments, return a tuple of the current maximum width and height. The maximum size defaults to the size of the screen. `wm_maxsize()` is an alias of **`maxsize()`**.

minsize(*width=None*, *height=None*)

Set or query the minimum permissible dimensions of the window, in pixels (or grid units for a gridded window). The window manager restricts the window to be no smaller than *width* and *height*. With no arguments, return a tuple of the current minimum width and height. The minimum size defaults to one pixel in each dimension. `wm_minsize()` is an alias of **`minsize()`**.

overrideredirect(*boolean=None*)

Set or query the override-redirect flag for the window. When this flag is set, the window is ignored by the window manager: it is not reparented into a decorative frame and the user cannot manipulate it through the usual window manager controls. With no argument, return a boolean indicating whether the flag is set, or `None` if it has not been set. The flag is reliably honored only when the window is first mapped or remapped from the withdrawn state. `wm_overrideredirect()` is an alias of **`overrideredirect()`**.

positionfrom(*who=None*)

Set or query the source of the window's current position. *who* is either `'program'` or `'user'` and indicates whether the position was requested by the program or by the user; an empty string cancels the current source. With no argument, return the current source, or an empty string if none has been set. Tk automatically sets the source to `'user'` when **`geometry()`** is called, unless it has been set explicitly to `'program'`. **`wm_positionfrom()`** is an alias of **`positionfrom()`**.

protocol(*name=None*, *func=None*)

Register *func* as the handler for the window manager protocol *name*, an atom such as `'WM_DELETE_WINDOW'`, `'WM_SAVE_YOURSELF'` or `'WM_TAKE_FOCUS'`; *func* is then called whenever the window manager sends a message of that protocol. Tk installs a default `WM_DELETE_WINDOW` handler that destroys the window, which this method can replace. If *func* is an empty string, the handler is removed. With only *name*, return the name of its registered handler command, or an empty string if none is set (the default `WM_DELETE_WINDOW` handler is not reported); with no arguments, return a tuple of the protocols that currently have handlers. **`wm_protocol()`** is an alias of **`protocol()`**.

resizable(*width=None*, *height=None*)

Control whether the user may interactively resize the window. *width* and *height* are boolean values that determine whether the window's width and height may be changed. With no arguments, return a tuple of two `0`/`1` values indicating whether each dimension is currently resizable. By default a window is resizable in both dimensions. **`wm_resizable()`** is an alias of **`resizable()`**.

sizefrom(*who=None*)

Set or query the source of the window's current size. *who* is either `'program'` or `'user'` and indicates whether the size was requested by the program or by the user; an empty string cancels the current source. With no argument, return the current source, or an empty

string if none has been set. `wm_sizefrom()` is an alias of **`sizefrom()`**.

state(*newstate=None*)

Set or query the state of the window. With no argument, return the current state: one of `'normal'`, `'iconic'`, `'withdrawn'`, `'icon'` or, on Windows and macOS only, `'zoomed'`. `'iconic'` refers to a window that has been iconified, while `'icon'` refers to a window serving as the icon for another window (see `iconwindow()`); the `'icon'` state cannot be set. `wm_state()` is an alias of **`state()`**.

Not to be confused with `ttk.Widget.state`.

title(*string=None*)

Set or query the title for the window, which the window manager should display in the window's title bar. With no argument, return the current title. The title defaults to the window's name. `wm_title()` is an alias of **`title()`**.

transient(*master=None*)

Mark the window as a transient window (such as a pull-down menu or dialog) working on behalf of *master*, the path name of another top-level window. An empty string clears the transient status. With no argument, return the path name of the current master, or an empty string. A transient window mirrors state changes in its master and may be decorated differently by the window manager; it is an error to make a window a transient of itself. `wm_transient()` is an alias of **`transient()`**.

withdraw()

Withdraw the window from the screen, unmapping it and causing the window manager to forget about it. If the window has never been mapped, it is instead mapped in the withdrawn state. It is sometimes necessary to withdraw a window and then re-map it (for example with `deiconify()`) to make some window managers notice changes to window attributes. `wm_withdraw()` is an alias of **`withdraw()`**.

class tkinter.Pack

Geometry manager that arranges widgets by packing them against the sides of their container. The **`Pack`** mix-in is inherited by all widgets (through **`Widget`**) and provides the methods for managing a widget with the *pack* geometry manager. See also Geometry management.

Nota

`Pack`, **`Place`** and **`Grid`** all define the short method names **`forget()`**, **`info()`**, **`slaves()`**, **`content()`** and **`propagate()`**. On a widget the bare names resolve to the *pack* manager's versions, since **`Pack`** and **`Misc`** precede **`Place`** and **`Grid`** in the method resolution order, whatever manager actually manages the widget; and **`configure()`**/**`config()`** configure the widget's options, not its geometry. Use the explicit `pack_*`, `grid_*` and `place_*` methods (and `pack`, `grid`, `place` for geometry configuration) to act on a specific geometry manager.

pack_configure(*cnf={}, **kw*)

pack(*cnf={}, **kw*)

Pack the widget inside its container, positioning it relative to the siblings already packed there. The supported options are:

side

Which side of the container to pack the widget against: `'top'` (the default), `'bottom'`, `'left'` or `'right'`.

fill

Whether to stretch the widget to fill its parcel: `'none'` (the default), `'x'`, `'y'` or `'both'`.

expand

Whether the widget should expand to consume any extra space in its container (a boolean, default false).

anchor

Where to position the widget in its parcel when the parcel is larger than the widget: an anchor such as `'n'` or `'sw'` (default

`'center'`).

ipadx, *ipady*
: Internal padding added on the left and right (*ipadx*) or top and bottom (*ipady*) of the widget, as a screen distance (default `0`).

padx, *pady*
: External padding left on the left and right (*padx*) or top and bottom (*pady*) of the widget, as a screen distance or a pair of two distances for the two sides (default `0`).

after
: Pack the widget after the given widget in the packing order, using the same container.

before
: Pack the widget before the given widget in the packing order, using the same container.

*in_*
: The container in which to pack the widget; it defaults to the parent widget.

`pack()`, **`configure()`** and **`config()`** are aliases of **`pack_configure()`**.

pack_forget()
: Unmap the widget and remove it from the packing order, forgetting its packing options. It can be packed again later with **`pack_configure()`**. **`forget()`** is an alias of **`pack_forget()`**, except on **`PanedWindow`**, **`ttk.Notebook`** and **`ttk.PanedWindow`**, which provide their own **`forget()`** method.

Not to be confused with **`Wm.forget()`**.

pack_info()
: Return a dictionary of the widget's current packing options. **`info()`** is an alias of **`pack_info()`**.

pack_propagate()

pack_propagate(*flag*)

Same as `Misc.pack_propagate()`, treating this widget as a container: enable or disable geometry propagation. `propagate()` is an alias of **`pack_propagate()`**.

pack_slaves()

Same as `Misc.pack_slaves()`: return the list of widgets packed in this widget. `slaves()` is an alias of **`pack_slaves()`**.

pack_content()

Same as `Misc.pack_content()`. `content()` is an alias of **`pack_content()`**.

Adicionado na versão 3.15.

class tkinter.Place

Geometry manager that places widgets at explicit positions and sizes within their container. The **`Place`** mix-in is inherited by all widgets (through `Widget`). See also Geometry management.

place_configure(*cnf={}, **kw*)

place(*cnf={}, **kw*)

Place the widget inside its container at an absolute or relative position. The supported options are:

x, y

The absolute horizontal and vertical position of the widget's anchor point, as a screen distance (default `0`).

relx, rely

The horizontal and vertical position of the widget's anchor point as a fraction of the container's width and height, where `0.0` is the left or top edge and `1.0` is the right or bottom edge. If both the absolute and the relative option are given, their values are summed.

anchor
Which point of the widget is placed at the given position: an anchor such as `'n'` or `'se'` (default `'nw'`).

width, *height*
The absolute width and height of the widget, as a screen distance. By default the widget's requested size is used.

relwidth, *relheight*
The width and height of the widget as a fraction of the container's width and height. If both the absolute and the relative option are given, their values are summed.

bordermode
How the container's border affects placement: `'inside'` (the default) measures the area inside the border, `'outside'` measures the area including the border, and `'ignore'` uses the official X area.

*in_*
The container relative to which the widget is placed; it must be the widget's parent or a descendant of the parent, and defaults to the parent.

`place()`, **`configure()`** and **`config()`** are aliases of **`place_configure()`**.

place_forget()
Unmap the widget and remove it from the placement, forgetting its place options.

place_info()
Return a dictionary of the widget's current place options.

place_slaves()
Same as **`Misc.place_slaves()`**: return the list of widgets placed in this widget.

place_content()

Same as **`Misc.place_content()`**.

Adicionado na versão 3.15.

class tkinter.Grid

Geometry manager that arranges widgets in a two-dimensional grid of rows and columns within their container. The **`Grid`** mix-in is inherited by all widgets (through **`Widget`**). See also Geometry management.

grid_configure(*cnf={}, **kw*)

grid(*cnf={}, **kw*)

Position the widget in a cell of its container's grid.

Not to be confused with **`Wm.grid()`**.

The supported options are:

row, column

The row and column of the cell to place the widget in, counting from `0`. *column* defaults to the column after the previous widget placed in the same **`grid_configure()`** call (or `0`), and *row* defaults to the next empty row.

rowspan, columnspan

The number of rows and columns the widget should span (default `1`).

sticky

How to position or stretch the widget when its cell is larger than the widget: a string containing zero or more of the characters `'n'`, `'s'`, `'e'` and `'w'`, naming the cell sides the widget sticks to. Specifying both `'n'` and `'s'` (or `'e'` and `'w'`) stretches the widget to fill the height (or width) of the cell. The default is `''`, which centers the widget at its requested size.

ipadx, ipady

Internal padding added on the left and right (*ipadx*) or top and bottom (*ipady*) of the widget, as a screen distance (default `0`).

padx, *pady*
External padding left on the left and right (*padx*) or top and bottom (*pady*) of the widget, as a screen distance or a pair of two distances for the two sides (default `0`).

*in_*
The container in whose grid to place the widget; it defaults to the parent widget.

`grid()`, **`configure()`** and **`config()`** are aliases of **`grid_configure()`**.

grid_forget()
Unmap the widget and remove it from the grid, forgetting its grid options.

grid_remove()
Unmap the widget and remove it from the grid, but remember its grid options so that it is restored to the same cell if it is gridded again.

grid_info()
Return a dictionary of the widget's current grid options.

grid_bbox(*column=None*, *row=None*, *col2=None*, *row2=None*)
Same as **`Misc.grid_bbox()`**. **`bbox()`** is an alias of **`grid_bbox()`**, except on **`Canvas`**, **`Listbox`**, **`Spinbox`**, **`Text`**, **`ttk.Entry`** and **`ttk.Treeview`**, which provide their own **`bbox()`** method.

grid_columnconfigure(*index*, *cnf={}*, ***kw*)
Same as **`Misc.grid_columnconfigure()`**: query or set the options (such as *weight*, *minsize*, *pad* and *uniform*) of a grid column. **`columnconfigure()`** is an alias of **`grid_columnconfigure()`**.

grid_rowconfigure(*index*, *cnf={}*, ***kw*)

Same as `Misc.grid_rowconfigure()`: query or set the options of a grid row. `rowconfigure()` is an alias of **`grid_rowconfigure()`**.

grid_location(*x*, *y*)

Same as `Misc.grid_location()`: return the `(column, row)` of the cell that covers the pixel at *x*, *y*. `location()` is an alias of **`grid_location()`**.

grid_size()

Same as `Misc.grid_size()`: return a `(columns, rows)` tuple giving the size of the grid. `size()` is an alias of **`grid_size()`**, except on the `Listbox` widget, which provides its own **`size()`** method.

grid_propagate()

grid_propagate(*flag*)

Same as `Misc.grid_propagate()`.

grid_slaves(*row=None*, *column=None*)

Same as `Misc.grid_slaves()`: return the widgets managed in the grid, optionally restricted to a *row* and/or *column*.

grid_content(*row=None*, *column=None*)

Same as `Misc.grid_content()`.

Adicionado na versão 3.15.

class tkinter.XView

Mix-in providing the horizontal-scrolling interface shared by widgets such as `Entry`, `Canvas`, `Listbox`, `Text` and `Spinbox`. A widget's `xview()` method is registered as the *command* of a horizontal `Scrollbar`.

xview(**args*)

Query or change the horizontal position of the view. With no arguments, return a tuple `(first, last)` of two fractions between 0 and 1 giving the portion of the document that is currently visible.

Otherwise the arguments are passed to the Tk `xview` widget command and are usually generated by a scrollbar; **`xview_moveto()`** and **`xview_scroll()`** provide a more convenient interface.

xview_moveto(*fraction*)

Adjust the view so that *fraction* of the total width of the document is off-screen to the left. *fraction* is a number between 0 and 1.

xview_scroll(*number*, *what*)

Shift the view left or right by *number* units. *what* is either `'units'` or `'pages'`; a negative *number* scrolls left and a positive one scrolls right.

class tkinter.YView

Mix-in providing the vertical-scrolling interface shared by widgets such as **`Canvas`**, **`Listbox`** and **`Text`**. A widget's **`yview()`** method is registered as the *command* of a vertical **`Scrollbar`**.

yview(**args*)

Query or change the vertical position of the view. With no arguments, return a tuple `(first, last)` of two fractions between 0 and 1 giving the portion of the document that is currently visible. Otherwise the arguments are passed to the Tk `yview` widget command, usually generated by a scrollbar; **`yview_moveto()`** and **`yview_scroll()`** provide a more convenient interface.

yview_moveto(*fraction*)

Adjust the view so that *fraction* of the total height of the document is off-screen above the top. *fraction* is a number between 0 and 1.

yview_scroll(*number*, *what*)

Shift the view up or down by *number* units. *what* is either `'units'` or `'pages'`; a negative *number* scrolls up and a positive one scrolls down.

class tkinter.BaseWidget(*master*, *widgetName*, *cnf={}*, *kw={}*, *extra=()*)

Internal base class for all widgets. It inherits from `Misc` and adds the machinery that creates the underlying Tk widget; application code normally uses `Widget` or a concrete widget class rather than instantiating **`BaseWidget`** directly.

destroy()

Destroy this widget and all of its children, removing the corresponding Tk widgets and deleting the associated Tcl commands.

class tkinter.Widget(*master*, *widgetName*, *cnf={}*, *kw={}*, *extra=()*)

Internal base class for the standard widgets. It combines `BaseWidget` with the geometry-manager mix-ins `Pack`, `Place` and `Grid`, so that every widget can be managed by any of the three geometry managers. The concrete widget classes (`Button`, `Label`, and so on) derive from **`Widget`**.

Toplevel widgets

class tkinter.Tk(*screenName=None*, *baseName=None*, *className='Tk'*, *useTk=True*, *sync=False*, *use=None*)

Construct a toplevel Tk widget, which is usually the main window of an application, and initialize a Tcl interpreter for this widget. Each instance has its own associated Tcl interpreter. Inherits from `Misc` and `Wm`.

To create a Tcl interpreter without initializing the Tk subsystem, use the `Tcl()` factory function instead.

A classe `Tk` normalmente é instanciada usando todos os valores padrão. No entanto, os seguintes argumentos nomeados são reconhecidos atualmente:

screenName

Quando fornecido (como uma string), define a variável de ambiente **`DISPLAY`**. (Somente X11)

baseName

Nome do arquivo de perfil. Por padrão, *baseName* é derivado do nome do programa (`sys.argv[0]`).

className

Nome da classe widget. Usado como um arquivo de perfil e também como o nome com o qual Tcl é invocado (*argv0* em *interp*).

useTk

Se `True`, inicialize o subsistema Tk. A função `tkinter.Tcl()` define isso como `False`.

sync

Se `True`, execute todos os comandos do servidor X de forma síncrona, para que os erros sejam relatados imediatamente. Pode ser usado para depuração. (Somente X11)

use

Specifies the *id* of the window in which to embed the application, instead of it being created as an independent toplevel window. *id* must be specified in the same way as the value for the -use option for toplevel widgets (that is, it has a form like that returned by `winfo_id()`).

Observe que em algumas plataformas isso só funcionará corretamente se *id* se referir a um frame Tk ou nível superior que tenha sua opção -container habilitada.

`Tk` lê e interpreta os arquivos de perfil, chamados `.`*`className`*`.tcl` e `.`*`baseName`*`.tcl`, por meio do interpretador Tcl e chama `exec()` no conteúdo de `.`*`className`*`.py` e `.`*`baseName`*`.py`. O caminho para os arquivos de perfil encontra-se na variável de ambiente **`HOME`** ou, se não estiver definida, então `os.curdir`.

Nota

On Windows, creating a Tcl interpreter (by instantiating **`Tk`** or calling `Tcl()`) sets the **`HOME`** environment variable for the process, if it is not

already set, to `%HOMEDRIVE%%HOMEPATH%` (or **`USERPROFILE`**, or `c:\`). This is done by Tcl and can affect other code that reads **`HOME`**.

tk

Objeto da aplicação Tk criado ao instanciar **`Tk`**. Isso fornece acesso ao interpretador Tcl. Cada widget anexado à mesma instância de **`Tk`** tem o mesmo valor para o atributo **`tk`**.

master

The widget object that contains this widget. For **`Tk`**, the **`master`** is **`None`** because it is the main window. The terms *master* and *parent* are similar and sometimes used interchangeably as argument names; however, calling **`winfo_parent()`** returns a string of the widget name whereas **`master`** returns the object. *parent*/*child* reflects the tree-like relationship while *master* (or *container*)/*content* reflects the container structure.

children

Descendentes diretos deste widget como um **`dict`** com os nomes do widget filho como a chave e os objetos de instância filho como o valor.

destroy()

Destroy this and all descendant widgets and, for the main window, end the connection to the underlying Tcl interpreter.

loadtk()

Finish loading and initializing the Tk subsystem. This is needed only when the interpreter was created without Tk (for example through **`Tcl()`**); it is called automatically when *useTk* is true.

readprofile(*baseName*, *className*)

Read and source the user's profile files `.`*`className`*`.tcl` and `.`*`baseName`*`.tcl` into the Tcl interpreter, and execute the corresponding `.`*`className`*`.py` and `.`*`baseName`*`.py` files. This is

called during initialization; see the description of the constructor above.

report_callback_exception(*exc*, *val*, *tb*)

Report a callback exception. This is called when an exception propagates out of a Tkinter callback; *exc*, *val* and *tb* are the exception type, value and traceback as returned by `sys.exc_info()`. The default implementation prints a traceback to `sys.stderr`. It can be overridden to customize error handling, for example to display the traceback in a dialog.

class tkinter.Toplevel(*master=None*, *cnf={}*, ***kw*)

A **`Toplevel`** widget is a top-level window, similar to a `Frame` except that its X parent is the root window of a screen rather than its logical parent. Its primary purpose is to serve as a container for dialog boxes and other collections of widgets; its only visible features are its background and an optional 3-D border. Notable options include *menu*, which installs a `Menu` as the window's menubar. Inherits from `BaseWidget` and `Wm`, so a toplevel is managed by the window manager. Refer to the Tk `toplevel` manual page for the full list of options.

Widget classes

class tkinter.Button(*master=None*, *cnf={}*, ***kw*)

A **`Button`** widget displays a textual string, bitmap or image and invokes a command when the user presses it (by clicking mouse button 1 over the button or, when the button has focus, by pressing the space key). Inherits from `Widget`. In addition to the standard widget options, a button accepts the options documented in the Tk `button` manual page, such as *command* (the callback invoked when the button is pressed), *textvariable*, *state* and *default*.

invoke()

Invoke the command associated with the button, if there is one, and return its result, or an empty string if no command is associated with the button. This is ignored if the button's state is `disabled`.

flash()

Flash the button by redisplaying it several times, alternating between the active and normal colors. At the end of the flash the button is left in the same normal or active state as when the method was called. This is ignored if the button's state is `disabled`.

class tkinter.Canvas(*master=None*, *cnf={}*, ***kw*)

A **`Canvas`** widget implements structured graphics. It displays any number of *items*, such as arcs, lines, ovals, polygons, rectangles, text, bitmaps, images and embedded windows, which may be drawn, moved, re-colored and bound to events. Inherits from **`Widget`**, **`XView`** and **`YView`**, so the view can be scrolled horizontally and vertically with **`xview()`** and **`yview()`**. Refer to the Tk `canvas` manual page for the full list of widget and item options.

Each item has a unique integer *id*, assigned when it is created, and zero or more string *tags*. A tag is an arbitrary string that does not have the form of an integer; the same tag may be shared by many items, which makes tags convenient for grouping items. The special tag `'all'` matches every item in the canvas, and `'current'` matches the topmost item under the mouse pointer. Most methods take a *tagOrId* argument that may be an integer id naming a single item, or a tag naming zero or more items; as described in the Tk `canvas` manual page, a tag may also be a logical expression of tags combined with the operators `&&`, `||`, `^`, `!` and parentheses. When a method that operates on a single item is given a *tagOrId* matching several items, it normally uses the lowest matching item in the display list.

The items are kept in a *display list* that determines drawing order: items later in the list are drawn on top of earlier ones. A newly created item is placed at the top of the list; the order can be changed with **`tag_raise()`** and **`tag_lower()`**.

create_arc(**args*, ***kw*)

create_bitmap(**args*, ***kw*)

create_image(**args*, ***kw*)

create_line(**args*, ***kw*)
create_oval(**args*, ***kw*)
create_polygon(**args*, ***kw*)
create_rectangle(**args*, ***kw*)
create_text(**args*, ***kw*)
create_window(**args*, ***kw*)

Create a new item of the corresponding type and return its integer id. Each method is called as `create_TYPE(coord..., **options)`: the leading positional arguments give the coordinates that define the item (as separate numbers, as a single sequence of numbers, or as coordinate pairs), and the keyword arguments set item-specific options. Coordinates and screen distances may be given as numbers (interpreted as pixels) or as strings with a unit suffix (`'m'`, `'c'`, `'i'` or `'p'` for millimetres, centimetres, inches or printer's points), but are always stored and returned in pixels.

The item types are: `arc` (an arc-shaped region that is a section of an oval, defined by two diagonally opposite corners `x1, y1, x2, y2` of the enclosing rectangle); `bitmap` (a two-color bitmap positioned at a point `x, y`); `image` (a Tk image positioned at a point `x, y`); `line` (a line or curve through the points `x1, y1, ..., xn, yn`); `oval` (a circle or ellipse inscribed in the rectangle `x1, y1, x2, y2`); `polygon` (a closed polygon through the points `x1, y1, ..., xn, yn`); `rectangle` (a rectangle with corners `x1, y1, x2, y2`); `text` (a string of text positioned at a point `x, y`); and `window` (a child widget embedded in the canvas at a point `x, y`, specified with the *window* option).

Most item types accept a common set of *standard item options*, plus a few options specific to each type. Option names are passed as keyword arguments, without the leading hyphen.

The standard item options are:

fill

The color used to fill the item's interior, or to draw a *line* item or the characters of a *text* item. An empty string (the default for all types except *line* and *text*) leaves the item unfilled.

outline
The color used to draw the item's outline. An empty string draws no outline.

width
The width of the outline, defaulting to `1.0`. Has no effect if *outline* is empty.

dash
A dash pattern for the outline, given either as a sequence of segment lengths in pixels or as a string of the characters `'.'`, `','`, `'-'`, `'_'` and space. An empty pattern (the default) draws a solid outline.

dashoffset
The starting offset in pixels into the *dash* pattern. Ignored if there is no *dash* pattern.

stipple
A bitmap used as a stipple pattern when filling the item. Only well supported on X11.

outlinestipple
A bitmap used as a stipple pattern when drawing the outline. Has no effect if *outline* is empty.

offset, *outlineoffset*
The offset of the fill and outline stipple patterns, given as `'x,y'` or as a side such as `'n'`, `'se'` or `'center'`. Stipple offsets are only supported on X11.

state
Overrides the canvas state for this item; one of `'normal'`, `'disabled'` or `'hidden'`.

tags
: A single tag or a sequence of tags to associate with the item, replacing any existing tags.

Many of these options have *active…* and *disabled…* variants (such as *activefill*, *disabledfill*, *activewidth*, *disableddash*, *activeoutline*, *disabledstipple*) that override the base option when the item is the active item (under the mouse pointer) or is in the disabled state.

The following item types support additional options.

For `arc` items:

start
: The start of the arc's angular range, in degrees measured counter-clockwise from the 3-o'clock position.

extent
: The size of the angular range, in degrees counter-clockwise from *start*.

style
: How the arc is drawn: `'pieslice'` (the default), `'chord'` or `'arc'`.

For `line` items:

arrow
: Where to draw arrowheads: `'none'` (the default), `'first'`, `'last'` or `'both'`.

arrowshape
: A sequence of three distances describing the shape of the arrowheads.

capstyle
: How line ends are drawn: `'butt'` (the default), `'projecting'` or `'round'`.

joinstyle
: How line vertices are drawn: `'round'` (the default), `'bevel'` or `'miter'`.

smooth
: The smoothing method: a false value (the default) for no smoothing, or `'true'`/`'bezier'` or `'raw'` to draw the line as a curve.

splinesteps
: The number of line segments approximating each spline when *smooth* is enabled.

For `polygon` items:

joinstyle, *smooth*, *splinesteps*
: As for `line` items, applied to the polygon's outline.

For `text` items:

text
: The string to display; newline characters start new lines.

font
: The font used for the text.

justify
: How lines are justified: `'left'` (the default), `'right'` or `'center'`.

anchor
: How the text is positioned relative to its point, defaulting to `'center'`.

width
: The maximum line length; if non-zero, lines are wrapped at spaces.

angle

How many degrees to rotate the text counter-clockwise about its positioning point, from `0.0` to `360.0` (default `0.0`).

underline
: The index of a character to underline, or `-1` for none.

For `bitmap` items:

bitmap
: The bitmap to display.

anchor
: How the bitmap is positioned relative to its point.

background, *foreground*
: The colors used for the bitmap's `0` and `1` pixels; an empty *background* makes the `0` pixels transparent. Both have *active…* and *disabled…* variants, and *bitmap* has *activebitmap* and *disabledbitmap* variants.

For `image` items:

image
: The Tk image to display, previously created with the image protocols.

anchor
: How the image is positioned relative to its point.

Both options have *active…* and *disabled…* variants (*activeimage*, *disabledimage*) used in the active and disabled states.

For `window` items:

window
: The widget to embed; it must be a child of the canvas or of one of its ancestors, and may not be a top-level window.

anchor

How the window is positioned relative to its point.

width, *height*
The size to assign to the window; if zero (the default), the window is given its requested size.

`oval` and `rectangle` items have no type-specific options; they use only the standard item options.

Nota

Tk 8.6 added the *angle* option and Tk 9.0 added the *underline* option for `text` items.

coords(*tagOrId*)
coords(*tagOrId*, *coordList*, ?)
coords(*tagOrId*, ?, **coordList*)

Query or modify the coordinates of an item. With only *tagOrId*, return a list of the floating-point coordinates of the item given by *tagOrId* (the first matching item if it matches several). Given new coordinates, replace the coordinates of that item with them; like the `create_*` methods, the coordinates may be given as separate numbers, as a single sequence, or as coordinate pairs. The returned coordinates are always in pixels, regardless of the units used to specify them; for rectangles, ovals and arcs they are ordered left, top, right, bottom.

Alterado na versão 3.12: The arguments are now flattened: the coordinates may be given as separate arguments, as a single sequence, or grouped in pairs, like the `create_*` methods.

move(*tagOrId*, *xAmount*, *yAmount*, ?)

Move each of the items given by *tagOrId* in the canvas coordinate space by adding *xAmount* to every x-coordinate and *yAmount* to every y-coordinate of the item.

moveto(*tagOrId*, *x=''*, *y=''*)

Move the items given by *tagOrId* so that the first coordinate pair (the upper-left corner of the bounding box) of the lowest matching item is at position (*x*, *y*). *x* or *y* may be an empty string, in which case the corresponding coordinate is unchanged. All matching items keep their positions relative to each other.

Adicionado na versão 3.8.

scale(*tagOrId*, *xOrigin*, *yOrigin*, *xScale*, *yScale*, ?)

Rescale the coordinates of all items given by *tagOrId* in canvas coordinate space. Each x-coordinate is adjusted so that its distance from *xOrigin* changes by a factor of *xScale*, and each y-coordinate so that its distance from *yOrigin* changes by a factor of *yScale* (a factor of `1.0` leaves the coordinate unchanged).

delete(**tagOrIds*)

Delete each of the items given by the *tagOrIds* arguments.

dchars(*tagOrId*, *first*, ?)
dchars(*tagOrId*, *first*, *last*, ?)

Delete from each of the items given by *tagOrId* the characters (for text items) or coordinates (for line and polygon items) in the range from *first* to *last* inclusive; *last* defaults to *first*. Items that do not support indexing ignore this operation.

insert(*tagOrId*, *beforeThis*, *string*, ?)

Insert *string* into each of the items given by *tagOrId* just before the character or coordinate whose index is *beforeThis*. For line and polygon items *string* must be a valid sequence of coordinates.

itemcget(*tagOrId*, *option*)

Return the current value of the configuration option *option* for the item given by *tagOrId* (the lowest matching item if it matches several). This is like **`cget()`** but applies to an individual item.

itemconfigure(*tagOrId*, *cnf=None*, ***kw*)

Query or modify the configuration options of the items given by *tagOrId*. This mirrors `configure()`, except that it applies to individual items rather than to the canvas as a whole. With no options, it returns a dictionary describing the current options of the first matching item; otherwise it sets the given options on every matching item. The legal options are those accepted by the corresponding `create_*` method. `itemconfig()` is an alias of **`itemconfigure()`**.

type(*tagOrId*)

Return the type of the item given by *tagOrId* (the first matching item if it matches several), such as `'rectangle'` or `'text'`, or `None` if *tagOrId* does not match any item.

gettags(*tagOrId*, */*)

Return a tuple of the tags associated with the item given by *tagOrId* (the first matching item in display-list order if it matches several). Return an empty tuple if no item matches or the item has no tags.

dtag(*tagOrId*, */*)

dtag(*tagOrId*, *tagToDelete*, */*)

Remove the tag *tagToDelete* (which defaults to *tagOrId*) from each of the items given by *tagOrId*. Items that do not have that tag are unaffected.

addtag(*newtag*, *searchSpec*, */*, **args*)

Add the tag *newtag* to each item selected by the search specification *searchSpec* (and any further *args*). *searchSpec* is one of `'above'`, `'all'`, `'below'`, `'closest'`, `'enclosed'`, `'overlapping'` or `'withtag'`; the `addtag_*` methods below are convenient wrappers that supply each of these forms.

addtag_above(*newtag*, *tagOrId*)

Add the tag *newtag* to the item just above (after) *tagOrId* in the display list.

addtag_all(*newtag*)

Add the tag *newtag* to all items in the canvas.

addtag_below(*newtag*, *tagOrId*)

Add the tag *newtag* to the item just below (before) *tagOrId* in the display list.

addtag_closest(*newtag*, *x*, *y*, *halo=None*, *start=None*)

Add the tag *newtag* to the item closest to the point (*x*, *y*). If *halo* is given, any item within that distance of the point is treated as overlapping it. If *start* is given (a tag or id), select the topmost closest item that lies below *start* in the display list, which can be used to step through all the closest items.

addtag_enclosed(*newtag*, *x1*, *y1*, *x2*, *y2*)

Add the tag *newtag* to every item completely enclosed within the rectangle (*x1*, *y1*, *x2*, *y2*), where *x1* <= *x2* and *y1* <= *y2*.

addtag_overlapping(*newtag*, *x1*, *y1*, *x2*, *y2*)

Add the tag *newtag* to every item that overlaps or is enclosed within the rectangle (*x1*, *y1*, *x2*, *y2*), where *x1* <= *x2* and *y1* <= *y2*.

addtag_withtag(*newtag*, *tagOrId*)

Add the tag *newtag* to every item given by *tagOrId*.

find(*searchSpec*, /, **args*)

Return a tuple of the ids of all items selected by the search specification *searchSpec* (and any further *args*), in stacking order with the lowest item first. The search specification has any of the forms accepted by **`addtag()`**. The `find_*` methods below are more convenient wrappers around it.

find_above(*tagOrId*)

Return a tuple containing the id of the item just above *tagOrId* in the display list.

find_all()

Return a tuple of the ids of all items in the canvas, in stacking order.

find_below(*tagOrId*)

Return a tuple containing the id of the item just below *tagOrId* in the display list.

find_closest(*x*, *y*, *halo=None*, *start=None*)

Return a tuple containing the id of the item closest to the point (*x*, *y*). *halo* and *start* are interpreted as for `addtag_closest()`.

find_enclosed(*x1*, *y1*, *x2*, *y2*)

Return a tuple of the ids of all items completely enclosed within the rectangle (*x1*, *y1*, *x2*, *y2*).

find_overlapping(*x1*, *y1*, *x2*, *y2*)

Return a tuple of the ids of all items that overlap or are enclosed within the rectangle (*x1*, *y1*, *x2*, *y2*).

find_withtag(*tagOrId*)

Return a tuple of the ids of all items given by *tagOrId*.

tag_raise(*tagOrId*, *aboveThis=None*, /)

Move all items given by *tagOrId* to a new position in the display list just above the item given by *aboveThis*, or to the top of the display list if *aboveThis* is omitted. When several items are moved their relative order is preserved. This has no effect on embedded window items, whose stacking order is controlled by `Misc.tkraise()` and `Misc.lower()` instead. `lift()` and `tkraise()` are aliases of **`tag_raise()`**.

tag_lower(*tagOrId*, *belowThis=None*, /)

Move all items given by *tagOrId* to a new position in the display list just below the item given by *belowThis*, or to the bottom of the display list if *belowThis* is omitted. When several items are moved

their relative order is preserved. This has no effect on embedded window items. `lower()` is an alias of **`tag_lower()`**.

Nota

On a **`Canvas`**, `tkraise()`/`lift()` and `lower()` restack canvas items, shadowing the inherited `Misc.tkraise()`/`Misc.lift()` and `Misc.lower()` methods that restack the widget itself, which are therefore not available.

tag_bind(*tagOrId*, *sequence=None*, *func=None*, *add=None*)

Bind the callback *func* to the event *sequence* for all items given by *tagOrId*, so that *func* is invoked whenever that event occurs for one of the items. This is like `Widget.bind` but operates on canvas items rather than on whole widgets; only mouse, keyboard and virtual events may be bound. Mouse events are directed to the current item and keyboard events to the focus item (see `focus()`). If *add* is true the new binding is added to any existing bindings for the same sequence, rather than replacing them. Return the identifier of the bound function, which can be passed to `tag_unbind()`.

tag_unbind(*tagOrId*, *sequence*, *funcid=None*)

Remove for all items given by *tagOrId* the binding for the event *sequence*. If *funcid* is given, only that callback (as returned by `tag_bind()`) is unbound and deregistered.

Alterado na versão 3.13: If *funcid* is given, only that callback is unbound.

bbox(*tagOrId*, /, **tagOrIds*)

Return a 4-tuple `(x1, y1, x2, y2)` giving an approximate bounding box, in pixels, that encloses all the items given by *tagOrId* and any further *tagOrIds*. The result may overestimate the true bounding box by a few pixels. Return `None` if no item matches or the matching items have nothing to display.

This shadows the inherited **`Misc.bbox()`**; use **`grid bbox()`** for the grid bounding box.

canvasx(*screenx*, *gridspacing=None*)

Given a window x-coordinate *screenx*, return the canvas x-coordinate displayed at that location. If *gridspacing* is given, the result is rounded to the nearest multiple of *gridspacing* units.

canvasy(*screeny*, *gridspacing=None*)

Given a window y-coordinate *screeny*, return the canvas y-coordinate displayed at that location. If *gridspacing* is given, the result is rounded to the nearest multiple of *gridspacing* units.

focus()

focus(*tagOrId*, */*)

With *tagOrId*, set the keyboard focus for the canvas to the first item given by *tagOrId* that supports the insertion cursor; the focus is left unchanged if no such item exists. If *tagOrId* is an empty string, reset the focus so that no item has it. With no argument, return the id of the item that currently has the focus, or an empty string if none does. An item only displays the insertion cursor when both it is the focus item and its canvas has the input focus.

This shadows the inherited **`Misc.focus()`**; use **`focus set()`** to focus the widget itself.

icursor(*tagOrId*, *index*, */*)

Set the insertion cursor of the items given by *tagOrId* to just before the character given by *index*. Items that do not support an insertion cursor are unaffected. The cursor is only displayed when the item has the focus, but its position may be set at any time.

index(*tagOrId*, *index*, */*)

Return as an integer the numerical index within *tagOrId* corresponding to *index*, which is a textual description of a position (for text items an index into the characters, for line and polygon

items an index into the coordinates). If *tagOrId* matches several items, the first one that supports indexing is used.

select_adjust(*tagOrId*, *index*)

Adjust the end of the selection in *tagOrId* nearest to *index* so that it is at *index*, and make the other end the anchor point for future `select_to()` calls. If the selection is not currently in *tagOrId*, this behaves like **`select_to()`**.

select_clear()

Clear the selection if it is in this canvas; otherwise do nothing.

select_from(*tagOrId*, *index*)

Set the selection anchor point to just before the character given by *index* in the item given by *tagOrId*. This does not change the selection itself; it sets the fixed end for future `select_to()` calls.

select_item()

Return the id of the item that holds the selection, or `None` if the selection is not in this canvas. Unlike `find()` and the `find_*` methods, this returns the id as a string rather than an integer.

select_to(*tagOrId*, *index*)

Set the selection to the characters of *tagOrId* between the selection anchor point and *index*, inclusive of *index*. The anchor point is the one set by the most recent `select_adjust()` or `select_from()` call.

scan_mark(*x*, *y*)

Record *x*, *y* and the current view, for use with later `scan_dragto()` calls. This is typically bound to a mouse button press in the widget.

scan_dragto(*x*, *y*, *gain=10*)

Scroll the canvas by *gain* times the difference between *x*, *y* and the coordinates passed to the last `scan_mark()` call. This is typically bound to mouse motion events in the widget, producing the effect of dragging the canvas at high speed through its window.

postscript(*cnf={}, **kw*)

Generate a PostScript (Encapsulated PostScript, version 3.0) representation of part or all of the canvas. If the *file* or *channel* option is given, the PostScript is written there and an empty string is returned; otherwise it is returned as a string. By default only the area currently visible in the window is generated, so it is usually necessary either to call **`update()`** first or to use the *width* and *height* options. Supported options include *colormap*, *colormode*, *file*, *fontmap*, *height*, *pageanchor*, *pageheight*, *pagewidth*, *pagex*, *pagey*, *rotate*, *width*, *x* and *y*.

class tkinter.Checkbutton(*master=None, cnf={}, **kw*)

A **`Checkbutton`** widget displays a textual string, bitmap or image together with a square indicator, and toggles a boolean selection when pressed. It has all the behavior of a simple button and, in addition, can be selected: when selected the indicator is drawn with a check mark and the associated variable is set to the `onvalue`, and when deselected the indicator is drawn empty and the variable is set to the `offvalue`. Inherits from **`Widget`**. In addition to the standard widget options, a checkbutton accepts the options documented in the Tk `checkbutton` manual page, such as *variable*, *onvalue*, *offvalue* and *command*.

invoke()

Do just what would happen if the user pressed the checkbutton with the mouse: toggle the selection state of the button and invoke the associated command, if there is one. Return the result of the command, or an empty string if no command is associated with the checkbutton. This is ignored if the checkbutton's state is `disabled`.

select()

Select the checkbutton and set the associated variable to its `onvalue`.

deselect()

Deselect the checkbutton and set the associated variable to its `offvalue`.

toggle()

Toggle the selection state of the button, redisplaying it and modifying its associated variable to reflect the new state.

flash()

Flash the checkbutton by redisplaying it several times, alternating between the active and normal colors. At the end of the flash the checkbutton is left in the same normal or active state as when the method was called. This is ignored if the checkbutton's state is `disabled`.

class tkinter.Entry(*master=None*, *cnf={}*, ***kw*)

An **`Entry`** widget displays a single line of text and lets the user edit it. Inherits from **`Widget`** and **`XView`**; since entries can hold strings too long to fit in the window, they support horizontal scrolling through **`xview()`**.

In addition to the standard widget options, an entry accepts the options documented in the Tk `entry` manual page. Notable ones are *textvariable* (the name of a variable kept in sync with the entry's contents), *show* (if set, each character is displayed as the given character rather than its true value, useful for password entry), *validate* and *validatecommand* (which together let a callback accept or reject edits), and *state* (one of `'normal'`, `'disabled'` or `'readonly'`).

Many of the methods below take an *index* argument that selects a character in the entry's string. As described in the Tk `entry` manual page, *index* may be a number (counting from 0), `'insert'` (the character just after the insertion cursor), `'end'` (just after the last character), `'anchor'` (the selection anchor point), `'sel.first'` and `'sel.last'` (the ends of the selection), or `@x` (the character covering pixel x-coordinate *x* in the window). Out-of-range indices are rounded to the nearest legal value.

delete(*first*, *last=None*)

Delete the characters from index *first* up to but not including index *last*. If *last* is omitted, only the single character at *first* is deleted.

get()

Return the entry's current string.

insert(*index*, *string*)

Insert *string* just before the character given by *index*.

icursor(*index*)

Arrange for the insertion cursor to be displayed just before the character given by *index*.

index(*index*)

Return the numerical index corresponding to *index*.

selection_adjust(*index*)

Locate the end of the selection nearest to the character given by *index*, and adjust that end to be at *index* (including but not going beyond it); the other end becomes the anchor point for future `selection_to()` calls. If there is no selection in the entry, a new one is created between *index* and the most recent anchor point, inclusive. `select_adjust()` is an alias of **`selection_adjust()`**.

selection_clear()

Clear the selection if it is currently in this widget. If the selection is not in this widget the method has no effect. `select_clear()` is an alias of **`selection_clear()`**.

Nota

This shadows the inherited `Misc.selection_clear()`, which clears the X selection; that method is not available on an **`Entry`**.

selection_from(*index*)

Set the selection anchor point to just before the character given by *index*, without changing the selection. `select_from()` is an alias of **`selection_from()`**.

selection_present()

Return `True` if there are characters selected in the entry, `False` otherwise. `select_present()` is an alias of **`selection_present()`**.

selection_range(*start*, *end*)

Set the selection to include the characters starting with the one indexed by *start* and ending with the one just before *end*. If *end* refers to the same character as *start* or an earlier one, the selection is cleared. `select_range()` is an alias of **`selection_range()`**.

selection_to(*index*)

Set the selection between the anchor point and *index*: if *index* is before the anchor point, the selection runs from *index* up to but not including the anchor; if *index* is after it, from the anchor up to but not including *index*; if they coincide, nothing happens. The anchor point is the one set by the most recent `selection_from()` or `selection_adjust()` call. If there is no selection in the entry, a new one is created using the most recent anchor point. `select_to()` is an alias of **`selection_to()`**.

scan_mark(*x*)

Record *x* and the current view in the entry window, for use with later `scan_dragto()` calls. Typically associated with a mouse button press in the widget.

scan_dragto(*x*)

Compute the difference between *x* and the *x* given to the last `scan_mark()` call, and adjust the view left or right by 10 times that difference. Typically associated with mouse motion events, to produce the effect of dragging the entry at high speed through the window.

class tkinter.Frame(*master=None*, *cnf={}*, ***kw*)

A **`Frame`** widget is a simple container. Its primary purpose is to act as a spacer or container for complex window layouts; its only features are its background and an optional 3-D border to make the frame appear raised

or sunken. Inherits from **Widget**. Refer to the Tk `frame` manual page for the full list of options.

class tkinter.Label(*master=None, cnf={}, **kw*)

A **`Label`** widget displays a non-interactive textual string, bitmap or image. The displayed text is set with the *text* option or linked to a variable through *textvariable*, and an image can be shown using the *image* option. Text must all be in a single font but may occupy multiple lines, and one character may be underlined with the *underline* option. Inherits from **Widget**. Refer to the Tk `label` manual page for the full list of options.

class tkinter.LabelFrame(*master=None, cnf={}, **kw*)

A **`LabelFrame`** widget is a container that has the features of a **`Frame`** plus the ability to display a label. The label text is set with the *text* option and positioned with *labelanchor*, or an arbitrary widget may be used as the label by giving it as the *labelwidget* option. Inherits from **Widget**. Refer to the Tk `labelframe` manual page for the full list of options.

class tkinter.Listbox(*master=None, cnf={}, **kw*)

A **`Listbox`** widget displays a list of single-line text items, one per line, of which the user can select one or more. The way the selection behaves is governed by the *selectmode* option, which is one of `browse` (the default; at most one item, which may be dragged with the mouse), `single` (at most one item), `multiple` (any number of items, toggled individually), or `extended` (any number of items, including discontiguous ranges, selected by clicking and dragging). Inherits from **Widget**, **XView** and **YView**, so the view can be scrolled horizontally and vertically with **xview()** and **yview()**. Refer to the Tk `listbox` manual page for the full list of options.

Many of the methods take an *index* argument identifying a particular item. As described in the Tk `listbox` manual page, *index* may be a numeric index (counting from 0 at the top), `'active'` (the item with the location cursor, set with **activate()**), `'anchor'` (the selection anchor, set with **selection_anchor()**), `'end'` (the last item, or for **index()** and **insert()** the position just after it), or `@x,y` (the item covering pixel

coordinates *x*, *y* in the listbox window). Arguments named *first* and *last* are indices of the same forms.

insert(*index*, **elements*)

Insert the given *elements* as new items just before the item given by *index*. If *index* is `'end'`, the new items are appended to the end of the list.

delete(*first*, *last=None*)

Delete the items in the range from *first* to *last* inclusive. If *last* is omitted, it defaults to *first*, so that a single item is deleted.

get(*first*, *last=None*)

If *last* is omitted, return the contents of the item given by *first*, or an empty string if *first* refers to a non-existent item. If *last* is given, return a tuple of all the items in the range from *first* to *last* inclusive.

size()

Return the total number of items in the listbox.

This shadows the inherited **`Misc.size()`**; use **`grid size()`** for the grid size.

index(*index*)

Return the integer index value corresponding to *index*, or `None` if *index* is out of range. If *index* is `'end'`, the result is a count of the number of items in the listbox (not the index of the last item).

bbox(*index*)

Return a tuple `(x, y, width, height)` describing the bounding box, in pixels relative to the widget, of the text of the item given by *index*. Return `None` if no part of that item is visible on the screen, or if *index* refers to a non-existent item; if the item is only partly visible, the result still gives the full area of the item, including the parts that are not visible.

This shadows the inherited **`Misc.bbox()`**; use **`grid bbox()`** for the grid bounding box.

nearest(*y*)

Given a y-coordinate within the listbox window, return the index of the visible item nearest to that y-coordinate.

see(*index*)

Adjust the view so that the item given by *index* is visible. If the item is already visible the method has no effect; if it is near an edge of the window the listbox scrolls just enough to bring it into view at that edge, otherwise the listbox scrolls to center the item.

activate(*index*)

Set the active item to the one given by *index*. If *index* is outside the range of items, the closest item is activated instead. The active item is drawn as specified by the *activestyle* option when the widget has the input focus, and its index may be retrieved with the `'active'` index.

curselection()

Return a tuple containing the numerical indices of all of the items that are currently selected, or an empty tuple if no items are selected.

selection_anchor(*index*)

Set the selection anchor to the item given by *index*. If *index* refers to a non-existent item, the closest item is used. The selection anchor is the end of the selection that is fixed while dragging out a selection with the mouse, and may afterwards be referred to with the `'anchor'` index. **`select anchor()`** is an alias of **`selection_anchor()`**.

selection_clear(*first*, *last=None*)

Deselect any of the items in the range from *first* to *last* inclusive that are selected. The selection state of items outside this range is not

changed. `select_clear()` is an alias of **`selection_clear()`**.

Nota

This shadows the inherited `Misc.selection_clear()`, which clears the X selection; that method is not available on a **`Listbox`**.

selection_includes(*index*)

Return `True` if the item given by *index* is currently selected, `False` otherwise. `select_includes()` is an alias of **`selection_includes()`**.

selection_set(*first*, *last=None*)

Select all of the items in the range from *first* to *last* inclusive, without affecting the selection state of items outside that range. `select_set()` is an alias of **`selection_set()`**.

itemcget(*index*, *option*)

Return the current value of the configuration option *option* for the item given by *index*.

itemconfigure(*index*, *cnf=None*, ***kw*)

Query or modify the configuration options of the item given by *index*. This mirrors `configure()`, except that it applies to an individual item rather than to the listbox as a whole. With no options, it returns a dictionary describing the current options of the item; otherwise it sets the given options. The supported item options are *background*, *foreground*, *selectbackground* and *selectforeground*. `itemconfig()` is an alias of **`itemconfigure()`**.

scan_mark(*x*, *y*)

Record *x*, *y* and the current view, for use with later `scan_dragto()` calls. This is typically bound to a mouse button press in the widget.

scan_dragto(*x*, *y*)

Scroll the listbox by 10 times the difference between *x*, *y* and the coordinates passed to the last **scan_mark()** call. This is typically bound to mouse motion events in the widget, producing the effect of dragging the list at high speed through the window.

class tkinter.Menu(*master=None*, *cnf={}*, ***kw*)

A **Menu** widget displays a column of entries, each of which may be a command, a checkbutton, a radiobutton, a cascade (which posts an associated submenu) or a separator. Menus are used as the menubar of a toplevel window, as pulldown menus posted from a cascade entry or menubutton, and as popup menus. Inherits from **Widget**.

Many of the entry methods take an *index* argument that selects which entry to operate on. As described in the Tk `menu` manual page, *index* may be a numeric index (counting from 0 at the top), `'active'` (the currently active entry), `'end'` or `'last'` (the bottommost entry), `'none'` (no entry at all, written `{}` in Tcl), `@y` (the entry covering pixel y-coordinate *y* in the menu window), or a pattern matched against the labels of the entries from the top down.

add(*itemType*, *cnf={}*, ***kw*)

Add a new entry to the bottom of the menu. *itemType* is one of `'command'`, `'cascade'`, `'checkbutton'`, `'radiobutton'` or `'separator'` and determines the type of the new entry; the remaining options configure it. The **add_command()**, **add_cascade()**, **add_checkbutton()**, **add_radiobutton()** and **add_separator()** convenience methods call this method with the corresponding *itemType*.

The entry is configured by the following options, although not every option applies to every entry type (a separator accepts none of them):

label

The text to display in the entry.

“command”

The function to call when the entry is invoked (command, checkbutton and radiobutton entries).

accelerator
A string displayed at the right of the entry to advertise an accelerator keystroke; it does not itself create the binding.

underline
The index of a character in the label to underline for keyboard traversal.

state
One of `'normal'`, `'active'` or `'disabled'`.

image
An image to display instead of, or together with, the text label.

compound
Where to show the image relative to the text: `'none'` (the default), `'text'`, `'image'`, `'top'`, `'bottom'`, `'left'` or `'right'`.

bitmap
A bitmap to display instead of the text label.

font
The font to use for the text.

background, *foreground*
The entry's background and foreground colors in its normal state (ignored on macOS).

activebackground, *activeforeground*
The background and foreground colors used when the entry is active (ignored on macOS).

columnbreak
If true, the entry starts a new column instead of being placed below the previous entry.

hidemargin
If true, the standard margin around the entry is omitted, which is useful when a menu is used as a palette.

menu
The submenu posted by a cascade entry; it must be a child of this menu.

variable
The variable associated with a checkbutton or radiobutton entry.

onvalue, *offvalue*
The values stored in *variable* when a checkbutton entry is selected or cleared.

value
The value stored in *variable* when a radiobutton entry is selected.

indicatoron
Whether to display the indicator of a checkbutton or radiobutton entry.

selectcolor
The color of the indicator of a checkbutton or radiobutton entry when it is selected.

selectimage
The image displayed when a checkbutton or radiobutton entry is selected and *image* is also given.

add_cascade(*cnf={}*, ***kw*)
Add a new cascade entry to the bottom of the menu. A cascade entry has an associated submenu, given by its *menu* option, which must be a child of this menu; posting the entry posts the submenu next to it.

add_checkbutton(*cnf={}*, ***kw*)

Add a new checkbutton entry to the bottom of the menu. When invoked, a checkbutton entry toggles between its *onvalue* and *offvalue*, storing the result in its associated *variable*, and displays an indicator showing whether it is selected.

add_command(*cnf={}, **kw*)

Add a new command entry to the bottom of the menu. A command entry behaves much like a button: when it is invoked, the callback given by its *command* option is called.

add_radiobutton(*cnf={}, **kw*)

Add a new radiobutton entry to the bottom of the menu. Radiobutton entries sharing the same *variable* form a group of which only one may be selected at a time; selecting an entry stores its *value* in the variable.

add_separator(*cnf={}, **kw*)

Add a separator to the bottom of the menu. A separator is displayed as a horizontal dividing line and cannot be activated or invoked.

insert(*index, itemType, cnf={}, **kw*)

Same as **`add()`**, except that the new entry is inserted just before the entry given by *index* instead of being appended to the end of the menu. *itemType* is one of `'command'`, `'cascade'`, `'checkbutton'`, `'radiobutton'` or `'separator'`. The **`insert_command()`**, **`insert_cascade()`**, **`insert_checkbutton()`**, **`insert_radiobutton()`** and **`insert_separator()`** convenience methods call this method with the corresponding *itemType*.

insert_cascade(*index, cnf={}, **kw*)

Insert a new cascade entry before the entry given by *index* (see **`add_cascade()`**).

insert_checkbutton(*index, cnf={}, **kw*)

Insert a new checkbutton entry before the entry given by *index* (see **`add_checkbutton()`**).

insert_command(*index*, *cnf={}*, ***kw*)

Insert a new command entry before the entry given by *index* (see **add_command()**).

insert_radiobutton(*index*, *cnf={}*, ***kw*)

Insert a new radiobutton entry before the entry given by *index* (see **add_radiobutton()**).

insert_separator(*index*, *cnf={}*, ***kw*)

Insert a separator before the entry given by *index* (see **add_separator()**).

delete(*index1*, *index2=None*)

Delete all of the menu entries between *index1* and *index2* inclusive. If *index2* is omitted, it defaults to *index1*, so that a single entry is deleted. Attempts to delete a tear-off entry are ignored; remove it by changing the *tearoff* option instead.

entrycget(*index*, *option*)

Return the current value of the configuration option *option* for the entry given by *index*.

entryconfigure(*index*, *cnf=None*, ***kw*)

Query or modify the configuration options of the entry given by *index*. This mirrors **configure()**, except that it applies to an individual entry rather than to the menu as a whole. With no options, it returns a dictionary describing the current options of the entry; otherwise it sets the given options. The supported options are those accepted by **add()** for the entry's type. **entryconfig()** is an alias of **entryconfigure()**.

index(*index*)

Return the numerical index corresponding to *index*, or `None` if *index* selects no entry.

type(*index*)

Return the type of the entry given by *index*: one of `'command'`, `'cascade'`, `'checkbutton'`, `'radiobutton'`, `'separator'` or `'tearoff'` (for the tear-off entry).

activate(*index*)

Make the entry given by *index* the active entry, redisplaying it with its active colors, and deactivate any previously active entry. If *index* selects no entry, or the selected entry is disabled, the menu ends up with no active entry.

invoke(*index*)

Invoke the action of the entry given by *index*, as if it had been clicked. Nothing happens if the entry is disabled. If the entry has a *command* associated with it, the result of that command is returned; otherwise the result is an empty string.

post(*x*, *y*)

Display the menu on the screen at the root-window coordinates *x* and *y*, adjusting them if necessary so that the whole menu is visible. If the *postcommand* option has been specified, it is evaluated before the menu is posted.

tk_popup(*x*, *y*, *entry=''*)

Post the menu as a popup at the root-window coordinates *x* and *y*. If *entry* is given, the menu is positioned so that this entry is displayed under the pointer.

unpost()

Unmap the menu so that it is no longer displayed, also unposting any posted lower-level cascaded submenu. This has no effect on Windows and macOS, which manage the unposting of menus themselves.

xposition(*index*)

Return the x-coordinate, within the menu window, of the leftmost pixel of the entry given by *index*.

Adicionado na versão 3.3.

yposition(*index*)

Return the y-coordinate, within the menu window, of the topmost pixel of the entry given by *index*.

class tkinter.Menubutton(*master=None*, *cnf={}*, ***kw*)

A **`Menubutton`** widget displays a textual string, bitmap or image and posts an associated **`Menu`**, given by its *menu* option, when the user presses it. Like a **`Label`** it can show *text*, a *textvariable*, or an *image*, and the *direction* option controls where the menu appears relative to the button. Inherits from **`Widget`**. Refer to the Tk `menubutton` manual page for the full list of options.

class tkinter.Message(*master=None*, *cnf={}*, ***kw*)

A **`Message`** widget displays a non-interactive textual string, given by the *text* option or linked to a variable through *textvariable*. Unlike a **`Label`**, it breaks the string into multiple lines in order to produce a given aspect ratio, choosing line breaks at word boundaries, and it can justify the text left, centered or right. Inherits from **`Widget`**. Refer to the Tk `message` manual page for the full list of options.

class tkinter.OptionMenu(*master*, *variable*, *value*, **values*, ***kwargs*)

A helper subclass of **`Menubutton`** that displays a pop-up menu of mutually exclusive choices. *variable* is a **`Variable`** kept in sync with the selection, *value* is the initial choice, and *values* are the remaining menu entries. The keyword argument *command* may be given a callback that is invoked with the selected value, and the keyword argument *name* sets the Tk widget name.

destroy()

Destroy the widget, also cleaning up the associated pop-up menu.

Alterado na versão 3.14: Added support for the *name* keyword argument.

class tkinter.PanedWindow(*master=None*, *cnf={}*, ***kw*)

A **`PanedWindow`** is a geometry-manager widget that arranges any number of child *panes* in a row (when *orient* is `'horizontal'`) or a column (when *orient* is `'vertical'`). Each pane holds one widget, and each pair of adjacent panes is separated by a movable *sash* that the user can drag with the mouse to resize the widgets on either side of it. Inherits from **`Widget`**.

The *orient* option selects the layout direction, *sashwidth* sets the width of each sash and *sashrelief* its relief. When *showhandle* is true a small handle is drawn on each sash that the user can grab to drag it. Refer to the Tk `panedwindow` manual page for the full list of options.

add(*child*, ***kw*)

Add *child* to the panedwindow as a new pane, placed after any existing panes. The keyword arguments specify per-pane management options for *child*; they may be any of the options accepted by **`paneconfigure()`**.

remove(*child*)

Remove the pane containing *child* from the panedwindow. All geometry management options for *child* are forgotten. **`forget()`** is an alias of **`remove()`**. This shadows the inherited geometry-manager **`forget()`**; use **`pack forget()`**, **`grid forget()`** or **`place forget()`** to remove the widget itself from its manager.

panes()

Return a tuple of the widgets managed by the panedwindow, one per pane, in order.

panecget(*child*, *option*)

Return the current value of the management option *option* for the pane containing *child*. *option* may be any value allowed by **`paneconfigure()`**.

paneconfigure(*tagOrId*, *cnf=None*, ***kw*)

Query or modify the management options of the pane containing the widget *tagOrId*. With no options, it returns a dictionary describing all of the available options for the pane; given a single option name as a string, it returns a description of that one option; otherwise it sets the given options. The supported options include *after* and *before* (insert the pane after or before another managed window), *height* and *width* (the outer dimensions of the window, including any border), *minsize* (the minimum size in the paned dimension), *padx* and *pady* (extra space to leave on each side of the window), *sticky* (position or stretch the window within an oversized pane, using a string of the characters `n`, `s`, `e` and `w`), *hide* (hide the pane while keeping it in the list of panes) and *stretch* (how extra space is allocated to the pane: one of `'always'`, `'first'`, `'last'`, `'middle'` or `'never'`). **`paneconfig()`** is an alias of **`paneconfigure()`**.

identify(*x*, *y*)

Identify the panedwindow component underneath the point given by *x* and *y*, in window coordinates. If the point is over a sash or a sash handle, the result is a two-element tuple containing the index of the sash or handle and a word indicating whether it is over a sash or a handle, such as `(0, 'sash')` or `(2, 'handle')`. If the point is over any other part of the panedwindow, the result is an empty string.

sash(**args*)

Query or change the position of the sashes in the panedwindow. This is a thin wrapper around the Tk `sash` subcommand; the convenience methods **`sash_coord()`**, **`sash_mark()`** and **`sash_place()`** should normally be used instead.

sash_coord(*index*)

Return the current x and y coordinate pair for the sash given by *index*, which must be an integer between 0 and one less than the number of panes in the panedwindow. The coordinates returned are those of the top left corner of the region containing the sash.

sash_mark(*index*)

Record the current mouse position for the sash given by *index*, for use together with later sash-drag operations to move the sash.

sash_place(*index*, *x*, *y*)

Place the sash given by *index* at the coordinates *x* and *y*.

proxy(**args*)

Query or change the position of the sash proxy, the "ghost" sash shown while a sash is being dragged with non-opaque resizing. This is a thin wrapper around the Tk `proxy` subcommand; the convenience methods **`proxy_coord()`**, **`proxy_forget()`** and **`proxy_place()`** should normally be used instead.

proxy_coord()

Return a tuple containing the x and y coordinates of the most recent proxy location.

proxy_forget()

Remove the proxy from the display.

proxy_place(*x*, *y*)

Place the proxy at the coordinates *x* and *y*.

class tkinter.Radiobutton(*master=None*, *cnf={}*, ***kw*)

A **`Radiobutton`** widget displays a textual string, bitmap or image together with a diamond or circular indicator, and selects one choice out of several. It has all the behavior of a simple button and, in addition, can be selected: typically several radiobuttons share a single *variable*, and selecting one sets that variable to the radiobutton's *value*; each radiobutton also monitors the variable and automatically selects or deselects itself when the variable changes. Inherits from **`Widget`**. In addition to the standard widget options, a radiobutton accepts the options documented in the Tk `radiobutton` manual page, such as *variable*, *value* and *command*.

invoke()

Do just what would happen if the user pressed the radiobutton with the mouse: select the button and invoke the associated command, if there is one. Return the result of the command, or an empty string if no command is associated with the radiobutton. This is ignored if the radiobutton's state is `disabled`.

select()

Select the radiobutton and set the associated variable to the value corresponding to this widget.

deselect()

Deselect the radiobutton and set the associated variable to an empty string. If this radiobutton was not currently selected, this has no effect.

flash()

Flash the radiobutton by redisplaying it several times, alternating between the active and normal colors. At the end of the flash the radiobutton is left in the same normal or active state as when the method was called. This is ignored if the radiobutton's state is `disabled`.

class tkinter.Scale(*master=None*, *cnf={}*, ***kw*)

A **`Scale`** widget lets the user select a numerical value by moving a slider along a trough. It can be oriented vertically or horizontally and can optionally display a label and the current value. Inherits from **`Widget`**.

In addition to the standard widget options, a scale accepts the options documented in the Tk `scale` manual page, such as *from_*, *to*, *resolution*, *orient*, *tickinterval*, *variable* and *command*. As elsewhere in **`tkinter`**, the leading `-` of the Tk option name is dropped; *from* is spelled `from_` because **`from`** is a Python keyword.

With a non-integer *resolution*, see numeric values and the locale.

get()

Return the current value of the scale. The result is an integer if the scale's *resolution* yields whole numbers, and a float otherwise.

set(*value*)

Set the scale to *value*, moving the slider accordingly. This has no effect if the scale is disabled.

coords(*value=None*)

Return a tuple `(x, y)` giving the pixel coordinates, relative to the widget, of the point on the centerline of the trough that corresponds to *value*. If *value* is omitted, the scale's current value is used.

identify(*x*, *y*)

Return a string describing the part of the scale at the pixel coordinates *x*, *y*: `'slider'`, `'trough1'` (the part of the trough above or to the left of the slider), `'trough2'` (below or to the right of the slider), or an empty string if the point is not over any of these elements.

class tkinter.Scrollbar(*master=None*, *cnf={}*, ***kw*)

A **`Scrollbar`** widget displays a slider and two arrows that let the user scroll an associated widget, such as a **`Listbox`**, **`Text`**, **`Canvas`** or **`Entry`**. It is connected to the scrolled widget by setting that widget's *xscrollcommand* or *yscrollcommand* option to the scrollbar's **`set()`** method, and the scrollbar's *command* option to the scrolled widget's **`xview()`** or **`yview()`** method. Inherits from **`Widget`**.

get()

Return the current scrollbar settings as a tuple `(first, last)` of two fractions between 0 and 1, describing the portion of the document that is currently visible, as last passed to **`set()`**.

set(*first*, *last*)

Set the scrollbar. *first* and *last* are fractions between 0 and 1 giving the positions of the start and end of the visible portion of the associated document. This method is normally registered as the

scrolled widget's *xscrollcommand* or *yscrollcommand* and called by that widget.

activate(*index=None*)

Mark the element *index* (one of `'arrow1'`, `'slider'` or `'arrow2'`) as active, displaying it according to the *activebackground* and *activerelief* options. If *index* is omitted, return the name of the currently active element, or `None` if no element is active.

Alterado na versão 3.5: The *index* argument is now optional.

delta(*deltax*, *deltay*)

Return a float indicating the fractional change in the scrollbar setting that corresponds to moving the slider by *deltax* pixels horizontally (for horizontal scrollbars) or *deltay* pixels vertically (for vertical scrollbars).

fraction(*x*, *y*)

Return a float between 0 and 1 indicating where the point at pixel coordinates *x*, *y* lies in the trough: 0 corresponds to the top or left of the trough and 1 to the bottom or right.

identify(*x*, *y*)

Return the name of the element under the pixel coordinates *x*, *y* (such as `'arrow1'`), or an empty string if the point does not lie in any element of the scrollbar.

class tkinter.Spinbox(*master=None*, *cnf={}*, ***kw*)

A **`Spinbox`** widget is an `Entry`-like widget with a pair of up/down arrow buttons that let the user step through a range of values in addition to editing the value directly. The set of values may be a numeric range given by the *from_*, *to* and *increment* options, or an explicit list of strings given by the *values* option (which takes precedence over the range). Each time an arrow is invoked the *command* callback, if any, is called; the *wrap* option controls whether stepping past either end of the range wraps around to the other end; the *format* option specifies how

numeric values are formatted; and the *validate* option enables validation of the entered text. Inherits from **`Widget`** and **`XView`**.

With a non-integer *increment*, see numeric values and the locale.

Many of the methods take an *index* argument identifying a character in the spinbox's string. As described in the Tk `spinbox` manual page, *index* may be a numeric index (counting from 0), `'anchor'` (the selection anchor point), `'end'` (just after the last character), `'insert'` (the character just after the insertion cursor), `'sel.first'` or `'sel.last'` (the ends of the selection), or `@x` (the character covering pixel x-coordinate *x* in the window).

get()

Return the spinbox's string.

insert(*index*, *s*)

Insert the characters of the string *s* just before the character given by *index*.

delete(*first*, *last=None*)

Delete one or more characters of the spinbox. *first* is the index of the first character to delete, and *last* is the index of the character just after the last one to delete. If *last* is omitted, a single character at *first* is deleted.

icursor(*index*)

Arrange for the insertion cursor to be displayed just before the character given by *index*.

index(*index*)

Return the numerical index corresponding to *index*, as a string.

bbox(*index*)

Return a tuple of four integers `(x, y, width, height)` describing the bounding box of the character given by *index*. *x* and *y* are the

pixel coordinates of the upper-left corner of the character relative to the widget, and *width* and *height* are its size in pixels. The bounding box may refer to a region outside the visible area of the window.

This shadows the inherited **`Misc.bbox()`**; use **`grid bbox()`** for the grid bounding box.

identify(*x*, *y*)

Return the name of the window element at the pixel coordinates *x*, *y*: one of `'buttondown'`, `'buttonup'`, `'entry'` or `'none'`.

invoke(*element*)

Invoke the spin button given by *element*, either `'buttonup'` or `'buttondown'`, triggering the action associated with it.

scan(**args*)

A thin wrapper around the Tk `scan` widget subcommand, used to implement fast dragging of the view: `scan('mark', x)` records *x* and the current view, and `scan('dragto', x)` adjusts the view relative to that mark. The **`scan mark()`** and **`scan dragto()`** methods wrap the two forms.

scan_mark(*x*)

Record *x* and the current view in the spinbox window, for use with a later **`scan dragto()`** call. This is typically associated with a mouse button press in the widget.

scan_dragto(*x*)

Adjust the view by 10 times the difference between *x* and the *x* passed to the last **`scan mark()`** call. This is typically associated with mouse motion events, producing the effect of dragging the spinbox at high speed through the window.

selection(**args*)

A thin wrapper around the Tk `selection` widget subcommand, used to adjust the selection within the spinbox. It has several forms

depending on the first argument, such as `selection('adjust', index)`, `selection('clear')`, `selection('element', ?elem?)`, `selection('from', index)`, `selection('present')`, `selection('range', start, end)` and `selection('to', index)`. The **`selection adjust()`**, **`selection clear()`**, **`selection element()`**, **`selection from()`**, **`selection present()`**, **`selection range()`** and **`selection to()`** methods wrap these forms.

selection_adjust(*index*)

Locate the end of the selection nearest to the character given by *index* and adjust that end of the selection to be at *index* (including but not going beyond *index*). The other end becomes the anchor point for future **`selection to()`** calls. If the selection is not currently in the spinbox, a new selection is created to include the characters between *index* and the most recent anchor point, inclusive.

selection_clear()

Clear the selection if it is currently in this widget. If the selection is not in this widget, the method has no effect.

Nota

This shadows the inherited **`Misc.selection clear()`**, which clears the X selection; that method is not available on a **`Spinbox`**.

selection_element(*element=None*)

Set or get the currently selected element. If *element* (one of `'buttonup'`, `'buttondown'` or `'none'`) is given, that spin button is selected and displayed depressed; otherwise the name of the currently selected element is returned.

selection_from(*index*)

Set the selection anchor point to just before the character given by *index*, without changing the selection itself.

Adicionado na versão 3.8.

selection_present()

Return `True` if there are characters selected in the spinbox, `False` otherwise.

Adicionado na versão 3.8.

selection_range(*start*, *end*)

Set the selection to include the characters starting with the one indexed by *start* and ending with the one just before *end*. If *end* refers to the same character as *start* or an earlier one, the selection is cleared.

Adicionado na versão 3.8.

selection_to(*index*)

Set the selection between *index* and the anchor point. If *index* is before the anchor point, the selection runs from *index* up to but not including the anchor point; if it is after, the selection runs from the anchor point up to but not including *index*; if it is the same, nothing happens. The anchor point is the one set by the most recent `selection_from()` or `selection_adjust()` call. If the selection is not in this widget, a new selection is created using the most recent anchor point.

Adicionado na versão 3.8.

class tkinter.Text(*master=None*, *cnf={}*, ***kw*)

A **`Text`** widget displays and edits multi-line text. Portions of the text may be styled with **tags**, particular positions may be annotated with floating **marks**, and arbitrary images and other widgets may be embedded in the text. The widget also provides an unlimited undo/redo mechanism and supports peer widgets that share the same underlying data. Inherits from `Widget`, `XView` and `YView`, so the view can be scrolled horizontally and vertically with `xview()` and `yview()`. Refer to the Tk `text` manual page for the full list of options.

Most of the methods take one or more *index* arguments that identify a position within the text. As described in the Tk `text` manual page, an index is a string consisting of a base, optionally followed by one or more modifiers. The base may be `'line.char'` (line *line*, character *char*, where lines are counted from 1 and characters within a line from 0; `'line.end'` refers to the newline ending the line), `'end'` (the position just after the last newline), the name of a mark, `'tag.first'` or `'tag.last'` (the first character tagged with *tag*, or the position just after the last such character), the name of an embedded image or window, or `@x,y` (the character covering pixel coordinates *x*, *y* in the widget). A modifier such as `'+5 chars'`, `'-3 lines'`, `'linestart'`, `'lineend'`, `'wordstart'` or `'wordend'` adjusts the index relative to its base; several modifiers may be combined and are applied from left to right, for example `'insert wordstart - 1 c'`.

insert(*index*, *chars*, **args*)

Insert the string *chars* just before the character at *index* (if *index* is `'end'`, just before the final newline). By default the new text inherits any tags present on both sides of the insertion point. If *args* is given, it consists of alternating *tagList*, *chars* values: the preceding *chars* receives exactly the tags listed (a tag list may be a single tag name or a sequence of names), overriding the surrounding tags.

delete(*index1*, *index2=None*)

Delete the range of characters from *index1* up to but not including *index2*. If *index2* is omitted, the single character at *index1* is deleted. The widget always keeps a newline as its last character, so a deletion that would remove it is adjusted accordingly.

replace(*index1*, *index2*, *chars*, **args*)

Replace the range of characters from *index1* up to but not including *index2* with *chars*. This is equivalent to a **`delete()`** followed by an **`insert()`** at *index1*; *args* is interpreted as for **`insert()`**.

Adicionado na versão 3.3.

get(*index1*, *index2=None*)

Return the text from *index1* up to but not including *index2* as a string. If *index2* is omitted, return the single character at *index1*. Embedded images and windows are omitted from the result.

index(*index*)

Return the position corresponding to *index* in the canonical `'line.char'` form.

compare(*index1*, *op*, *index2*)

Compare the positions of *index1* and *index2* using the relational operator *op*, which must be one of `'<'`, `'<='`, `'=='`, `'>='`, `'>'` or `'!='`, and return the boolean result.

count(*index1*, *index2*, **options*, *return_ints=False*)

Count the number of items of the requested kinds between *index1* and *index2*; the count is negative if *index1* is after *index2*. Each of *options* names a kind of item to count: `'chars'`, `'displaychars'`, `'displayindices'`, `'displaylines'`, `'indices'`, `'lines'`, `'xpixels'` or `'ypixels'` (the default, used when no option is given, is `'indices'`). The pseudo-option `'update'` forces any out-of-date layout information to be recalculated before the following options are evaluated. When *return_ints* is true and a single counting option is given, return a plain integer; otherwise return a tuple with one integer per counting option (or `None` if the result is empty).

Adicionado na versão 3.3.

Alterado na versão 3.13: Added the *return_ints* parameter.

see(*index*)

Adjust the view so that the character given by *index* is visible. If it is already visible the method has no effect; if it is a short distance out of view the widget scrolls just enough to bring it to the nearest edge, otherwise it scrolls to center *index* in the window.

bbox(*index*)

Return a tuple `(x, y, width, height)` giving the bounding box, in pixels, of the visible part of the character at *index*, or `None` if that character is not visible on the screen.

This shadows the inherited **`Misc.bbox()`**; use **`grid bbox()`** for the grid bounding box.

dlineinfo(*index*)

Return a tuple `(x, y, width, height, baseline)` describing the display line that contains *index*: the first four values give the bounding box of the line in pixels and *baseline* gives the offset of the baseline measured down from the top of the area. Return `None` if that display line is not visible on the screen.

mark_set(*markName*, *index*)

Set the mark named *markName* to the position just before the character at *index*, creating the mark if it does not already exist. A mark created this way has right gravity by default.

mark_unset(**markNames*)

Remove each of the marks named in *markNames*. The special `insert` and `current` marks may not be removed.

mark_names()

Return a tuple of the names of all marks currently set in the widget.

mark_gravity(*markName*, *direction=None*)

If *direction* is omitted, return the gravity of mark *markName*, either `'left'` or `'right'`. Otherwise set its gravity to *direction*. The gravity determines on which side of the mark text inserted at the mark's position appears: a mark with right gravity (the default) stays to the right of such text.

mark_next(*index*)

Return the name of the first mark at or after *index*, or `None` if there is none. When *index* is the name of a mark, the search starts just after

that mark.

mark_previous(*index*)

Return the name of the last mark at or before *index*, or `None` if there is none. When *index* is the name of a mark, the search starts just before that mark.

tag_add(*tagName*, *index1*, **args*)

Add the tag *tagName* to the range of characters from *index1* up to but not including the next index in *args*. Further pairs of indices may follow in *args* to tag additional ranges; a trailing single index tags just the character at that index.

tag_remove(*tagName*, *index1*, *index2=None*)

Remove the tag *tagName* from the characters from *index1* up to but not including *index2* (or from the single character at *index1* if *index2* is omitted). The tag itself continues to exist even if no characters carry it.

tag_delete(**tagNames*)

Delete each of the tags named in *tagNames*, removing them from all characters and discarding their options and bindings.

tag_configure(*tagName*, *cnf=None*, ***kw*)

Query or modify the configuration options of the tag *tagName*. This mirrors `configure()`, except that it applies to a tag rather than to the widget as a whole: with no options it returns a dictionary describing the current options, otherwise it sets the given options. Defining a tag this way also gives it a priority higher than any existing tag.

The supported tag options, all controlling the appearance of the tagged text, are:

font

The font to use for the text.

foreground

The color to use for the text.

background
The color to use for the area behind the text.

fgstipple, *bgstipple*
Bitmaps used to stipple the foreground (text) and the background; only well supported on X11.

borderwidth
The width of the border drawn around the text according to *relief* (default `0`).

relief
The 3-D appearance of the text's border: `'flat'` (the default), `'raised'`, `'sunken'`, `'ridge'`, `'groove'` or `'solid'`.

offset
How far the text is raised above (or, if negative, lowered below) the baseline, for superscripts and subscripts.

underline
Whether to underline the text.

underlinefg
The color of the underline; it defaults to the text color.

overstrike
Whether to draw a line through the middle of the text.

overstrikefg
The color of the overstrike line; it defaults to the text color.

elide
Whether the text is elided (hidden).

justify
How to justify the first character of a display line: `'left'` (the default), `'right'` or `'center'`.

wrap
How to wrap lines that are too long: `'char'`, `'word'` or `'none'`.

lmargin1, *lmargin2*
The indentation, in pixels, of the first display line of a logical line and of the remaining display lines.

lmargincolor
The color of the left margin area.

rmargin
The right-hand margin, in pixels.

rmargincolor
The color of the right margin area.

spacing1, *spacing2*, *spacing3*
Extra space, in pixels, above the first display line of a logical line, between its display lines, and below its last display line.

tabs
The set of tab stops, in the same form as the widget's *tabs* option.

tabstyle
How tab stops are interpreted: `'tabular'` or `'wordprocessor'`.

selectbackground, *selectforeground*
The background and foreground colors used for the text while it is selected.

Nota

Tk 8.6 added the *lmargincolor*, *overstrikefg*, *rmargincolor*, *selectbackground*, *selectforeground* and *underlinefg* options.

`tag_config()` is an alias of **`tag_configure()`**.

tag_cget(*tagName*, *option*)

Return the current value of the configuration option *option* for the tag *tagName*.

tag_names(*index=None*)

If *index* is omitted, return a tuple of the names of all tags defined in the widget; otherwise return only the names of the tags applied to the character at *index*. The names are ordered from lowest to highest priority.

tag_ranges(*tagName*)

Return a tuple of indices describing all ranges of text tagged with *tagName*. The result alternates start and end indices, so that elements `2*i` and `2*i+1` bound the *i*-th range.

tag_nextrange(*tagName*, *index1*, *index2=None*)

Search forward from *index1* (up to *index2* if given) for the first range of characters tagged with *tagName*, and return a two-element tuple of its start and end indices, or an empty tuple if there is no such range.

tag_prevrange(*tagName*, *index1*, *index2=None*)

Search backward from *index1* (down to *index2* if given) for the nearest preceding range of characters tagged with *tagName*, and return a two-element tuple of its start and end indices, or an empty tuple if there is no such range.

tag_raise(*tagName*, *aboveThis=None*)

Raise the priority of tag *tagName* so that it is just above the priority of *aboveThis*, or to the highest priority of all tags if *aboveThis* is omitted. When the display options of overlapping tags conflict, the higher-priority tag wins.

tag_lower(*tagName*, *belowThis=None*)

Lower the priority of tag *tagName* so that it is just below the priority of *belowThis*, or to the lowest priority of all tags if *belowThis* is

omitted.

tag_bind(*tagName*, *sequence*, *func*, *add=None*)

Bind the event *sequence* on characters tagged with *tagName* to the callback *func*, so that *func* is invoked when that event occurs over such a character. If *add* is true the binding is added alongside any existing bindings for *sequence*, otherwise it replaces them. Works like `bind()` and returns the identifier of the new binding.

tag_unbind(*tagName*, *sequence*, *funcid=None*)

Remove the bindings of the event *sequence* on characters tagged with *tagName*. If *funcid* is given, only that binding (as returned by `tag_bind()`) is removed and its callback is unregistered.

Alterado na versão 3.13: If *funcid* is given, only that callback is unbound.

image_create(*index*, *cnf={}*, ***kw*)

Embed an image at *index* and return the name assigned to this image instance, which may then be used as an index or passed to the other `image_*` methods. The options, given in *cnf* and *kw*, include *image* (the Tk image to display), *name* (a base name for the instance), *align*, *padx* and *pady*.

image_cget(*index*, *option*)

Return the current value of the configuration option *option* for the embedded image at *index*.

image_configure(*index*, *cnf=None*, ***kw*)

Query or modify the configuration options of the embedded image at *index*, like `configure()` but applied to that image.

image_names()

Return a tuple of the names of all images embedded in the widget.

Nota

This shadows the inherited **`Misc.image_names()`**, which returns the names of all images in the Tcl interpreter; that method is not available on a **`Text`**.

window_create(*index*, *cnf={}*, ***kw*)

Embed a window (any widget) at *index*. The options, given in *cnf* and *kw*, include *window* (the widget to embed), *create* (a callback that creates the widget on demand), *align*, *stretch*, *padx* and *pady*. The embedded widget must be a descendant of the text widget's parent.

window_cget(*index*, *option*)

Return the current value of the configuration option *option* for the embedded window at *index*.

window_configure(*index*, *cnf=None*, ***kw*)

Query or modify the configuration options of the embedded window at *index*, like **`configure()`** but applied to that window.

`window_config()` is an alias of **`window_configure()`**.

window_names()

Return a tuple of the names of all windows embedded in the widget.

edit(**args*)

Low-level wrapper around the Tk `edit` widget command that controls the undo/redo mechanism and the modified flag; *args* is the `edit` subcommand and its arguments. The **`edit_*()`** methods below are thin wrappers around it and are usually more convenient.

edit_modified(*arg=None*)

If *arg* is omitted, return the current state of the modified flag as true or false; the flag is set automatically whenever the text is inserted or deleted. Otherwise set the flag to the boolean *arg*.

edit_undo()

Undo the most recent edit action, that is, all the inserts and deletes recorded on the undo stack since the previous separator, and move it to the redo stack. Raises `TclError` if the undo stack is empty. Has no effect unless the *undo* option is true. Since Tk 9.0, returns a tuple of indices delimiting the ranges of text that were changed.

edit_redo()

Reapply the most recently undone edit action, provided no further edits have been made since, and move it back to the undo stack. Raises `TclError` if the redo stack is empty. Has no effect unless the *undo* option is true. Since Tk 9.0, returns a tuple of indices delimiting the ranges of text that were changed.

edit_reset()

Clear the undo and redo stacks.

edit_separator()

Push a separator onto the undo stack, marking a boundary between edit actions for undo and redo. Has no effect unless the *undo* option is true. Separators are inserted automatically when the *autoseparators* option is true.

search(*pattern*, *index*, *stopindex=None*, *forwards=None*, *backwards=None*, *exact=None*, *regexp=None*, *nocase=None*, *count=None*, *elide=None*, *, *nolinestop=None*, *strictlimits=None*)

Search for *pattern* starting at *index* and return the index of the first character of the first match, or an empty string if there is no match. Searching stops at *stopindex* if given; otherwise it wraps around the ends of the text until the starting position is reached again. The following boolean keyword flags control the search: *forwards* or *backwards* select the direction (forward is the default); *exact* (the default) or *regexp* select literal or regular-expression matching; *nocase* makes the match case-insensitive; *elide* causes hidden text to be searched as well; *nolinestop* (regexp only) lets `.` and `[^` match newlines; and *strictlimits* requires the whole match to lie within

index and *stopindex*. If *count* is a `Variable`, the number of index positions in the match is stored in it.

Alterado na versão 3.15: Added the *nolinestop* and *strictlimits* parameters.

search_all(*pattern*, *index*, *stopindex=None*, *, *forwards=None*, *backwards=None*, *exact=None*, *regexp=None*, *nocase=None*, *count=None*, *elide=None*, *nolinestop=None*, *overlap=None*, *strictlimits=None*)

Like `search()`, but find every match in the searched range and return a tuple of the starting indices of all matches (empty if there are none). By default overlapping matches are not reported; passing a true *overlap* returns every match that is not wholly contained in another. If *count* is a `Variable`, it receives a list with one element per match.

Adicionado na versão 3.15.

scan_mark(*x*, *y*)

Record *x*, *y* and the current view, for use with later `scan dragto()` calls. This is typically bound to a mouse button press in the widget.

scan_dragto(*x*, *y*)

Scroll the widget by 10 times the difference between *x*, *y* and the coordinates passed to the last `scan mark()` call. This is typically bound to mouse motion events, producing the effect of dragging the text at high speed through the window.

debug(*boolean=None*)

If *boolean* is omitted, return whether internal consistency checks of the B-tree data structure are enabled. Otherwise enable or disable them. The setting is shared by all text widgets and may noticeably slow down widgets holding large amounts of text.

dump(*index1*, *index2=None*, *command=None*, ***kw*)

Return the contents of the widget from *index1* up to but not including *index2* (or just the segment at *index1* if *index2* is omitted), including text and information about marks, tags, images and windows. The result is a list of `(key, value, index)` triples, where *key* is one of `'text'`, `'mark'`, `'tagon'`, `'tagoff'`, `'image'` or `'window'`. By default all kinds are reported; passing any of the keyword arguments *all*, *text*, *mark*, *tag*, *image* or *window* as true restricts the dump to the selected kinds. If *command* is given, it is called once per triple with the three values as arguments and nothing is returned.

peer_create(*newPathName*, *cnf={}*, ***kw*)

Create a peer text widget with the path name *newPathName* that shares this widget's underlying data (text, marks, tags, images and the undo stack). Changes made through any peer are reflected in all of them. By default the peer covers the same lines as this widget; standard text options, including *startline* and *endline*, may be given to override this.

Adicionado na versão 3.3.

peer_names()

Return a tuple of the path names of this widget's peers, not including the widget itself.

Adicionado na versão 3.3.

yview_pickplace(**what*)

Adjust the view so that the location given by *what* is visible. This is an obsolete equivalent of `see()`, which should be used instead.

Variable classes

class tkinter.Variable(*master=None*, *value=None*, *name=None*)

The base class for the Tk variable wrappers. A Tk variable is a value stored in the Tcl interpreter that can be linked to widgets through their

variable or *textvariable* options (see Coupling widget variables), so that changes propagate both ways: updating the variable updates every widget bound to it, and a user editing such a widget updates the variable.

master is the widget whose Tcl interpreter owns the variable; if omitted, the default root window is used. *value* is the initial value; if omitted, a type-specific default is used. *name* is the name of the variable in the Tcl interpreter; if omitted, a unique name of the form `'PY_VARnum'` is generated. If *name* matches an existing variable and *value* is omitted, the existing value is retained.

In most cases you should use one of the typed subclasses below – **`StringVar`**, **`IntVar`**, **`DoubleVar`** or **`BooleanVar`** – rather than **`Variable`** directly.

Nota

When a **`Variable`** is garbage collected, its Tcl variable is unset. Keep a reference to it for as long as a widget is linked to it, for example by storing it as an attribute rather than in a local variable. Otherwise Tk recreates the Tcl variable to keep the widget working, but it is never unset again, leaking one Tcl variable per dropped wrapper.

Alterado na versão 3.10: Two variables now compare equal (==) only when they have the same name, are of the same class, and belong to the same Tcl interpreter.

get()

Return the current value of the variable. For the base class the value is returned as a string; the typed subclasses convert it to the appropriate Python type.

set(*value*)

Set the variable to *value*. **`initialize()`** is an alias of **`set()`**.

Adicionado na versão 3.3: The *initialize* spelling.

trace_add(*mode*, *callback*)

Register *callback* to be called when the variable is accessed according to *mode*. *mode* is one of the strings `'array'`, `'read'`, `'write'` or `'unset'`, or a list or tuple of such strings.

When triggered, *callback* is called with three arguments: the name of the Tcl variable, an index (or an empty string if the variable is not an element of an array), and the *mode* that triggered the call.

Return the internal name of the registered callback, which can be passed to **`trace_remove()`**.

Adicionado na versão 3.6.

trace_remove(*mode*, *cbname*)

Remove a trace callback from the variable. *mode* must match the *mode* that was passed to **`trace_add()`**, and *cbname* is the callback name returned by **`trace_add()`**.

Adicionado na versão 3.6.

trace_info()

Return a list of `(modes, cbname)` pairs describing all traces currently set on the variable, where *modes* is a tuple of mode strings and *cbname* is the internal callback name.

Adicionado na versão 3.6.

trace_variable(*mode*, *callback*)

Register *callback* to be called when the variable is accessed according to *mode*. *mode* is one of the strings `'r'`, `'w'` or `'u'`, for read, write or unset. Return the internal name of the registered callback. **`trace()`** is an alias of **`trace_variable()`**.

Descontinuado desde a versão 3.6: Use **`trace_add()`** instead. This method wraps a Tcl feature that was removed in Tcl 9.0.

trace_vdelete(*mode*, *cbname*)

Remove the trace callback named *cbname* registered for *mode* with `trace_variable()`.

Descontinuado desde a versão 3.6: Use `trace_remove()` instead. This method wraps a Tcl feature that was removed in Tcl 9.0.

trace_vinfo()

Return a list of `(mode, cbname)` pairs for all traces set on the variable with `trace_variable()`.

Descontinuado desde a versão 3.6: Use `trace_info()` instead. This method wraps a Tcl feature that was removed in Tcl 9.0.

class tkinter.StringVar(*master=None*, *value=None*, *name=None*)

A `Variable` subclass that holds a string. The default value is `''`.

get()

Return the value of the variable as a `str`.

class tkinter.IntVar(*master=None*, *value=None*, *name=None*)

A `Variable` subclass that holds an integer. The default value is `0`.

get()

Return the value of the variable as an `int`.

class tkinter.DoubleVar(*master=None*, *value=None*, *name=None*)

A `Variable` subclass that holds a float. The default value is `0.0`.

get()

Return the value of the variable as a `float`.

Nota

A floating-point value is always parsed with a period (`.`) as the decimal separator, but `Spinbox`, `Scale` and `ttk.Spinbox` format it

according to the `LC_NUMERIC` locale. Under a locale that uses a comma they produce a value that `get()` cannot read, raising `TclError`. Set `LC_NUMERIC` to a locale that uses a period (such as `'C'`) to avoid this.

class tkinter.BooleanVar(*master=None*, *value=None*, *name=None*)

A `Variable` subclass that holds a boolean. The default value is `False`.

get()

Return the value of the variable as a `bool`. Raise a `ValueError` if the value cannot be interpreted as a boolean.

set(*value*)

Set the variable to *value*, converting it to a boolean. `initialize()` is an alias of **`set()`**.

Adicionado na versão 3.3: The *initialize* spelling.

Image classes

class tkinter.Image(*imgtype*, *name=None*, *cnf={}*, *master=None*, ***kw*)

Base class for Tk images. *imgtype* is the Tk image type, one of `'photo'` or `'bitmap'`. An image is a named object that can be displayed by widgets through their *image* option; deleting all references to the **`Image`** object deletes the underlying Tk image. Usually you create a `PhotoImage` or `BitmapImage` rather than an **`Image`** directly.

The image's configuration options are given by *cnf* and *kw* and may be queried and changed later with the mapping protocol (using `image[key]`) or with the `configure()` method.

configure(***kw*)

Modify one or more configuration options of the image. The valid options depend on the image type; see `PhotoImage` and `BitmapImage`. `config()` is an alias of **`configure()`**.

height()

Return the height of the image, in pixels.

width()

Return the width of the image, in pixels.

type()

Return the type of the image, that is the value of *imgtype* with which it was created (for example `'photo'` or `'bitmap'`).

class tkinter.PhotoImage(*name=None*, *cnf={}*, *master=None*, ***kw*)

A full-color image (the Tk `photo` image type), stored internally with a varying degree of transparency per pixel. It can read and write GIF, PPM/PGM and (in Tk 8.6 and later) PNG files, read SVG files (in Tk 9.0 and later), and be drawn in widgets. Inherits from **`Image`**.

The configuration options include *data* (the image contents as a string), *file* (the name of a file to read the contents from), *format* (the name of the file format handler), *width* and *height* (the size of the image, used when building it up piece by piece), *gamma* and *palette*.

blank()

Blank the image; that is, set the entire image to have no data, so that it is displayed as transparent and the background of whatever window it is displayed in shows through.

cget(*option*)

Return the current value of the configuration option *option*.

copy(*, *from_coords=None*, *zoom=None*, *subsample=None*)

Return a new **`PhotoImage`** with a copy of this image.

from_coords specifies a rectangular sub-region of the source image to be copied. It must be a tuple or a list of 1 to 4 integers `(x1, y1, x2, y2)`. `(x1, y1)` and `(x2, y2)` specify diagonally opposite corners of the rectangle. If *x2* and *y2* are not specified, they default

to the bottom-right corner of the source image. The pixels copied include the left and top edges of the rectangle but not the bottom or right edges. If *from_coords* is not given, the whole source image is copied.

If *zoom* or *subsample* are specified, the image is transformed as in the **`zoom()`** or **`subsample()`** methods. The value must be a single integer or a pair of integers.

Alterado na versão 3.13: Added the *from_coords*, *zoom* and *subsample* parameters.

copy_replace(*sourceImage*, *, *from_coords=None*, *to=None*, *shrink=False*, *zoom=None*, *subsample=None*, *compositingrule=None*)

Copy a region from *sourceImage* (which must be a **`PhotoImage`**) into this image, possibly with pixel zooming and/or subsampling. If no options are specified, the whole of *sourceImage* is copied into this image, starting at coordinates `(0, 0)`.

from_coords specifies a rectangular sub-region of the source image to be copied, as in the **`copy()`** method.

to specifies a rectangular sub-region of the destination image to be affected. It must be a tuple or a list of 1 to 4 integers `(x1, y1, x2, y2)`. If *x2* and *y2* are not specified, they default to `(x1, y1)` plus the size of the source region (after subsampling and zooming, if specified). If *x2* and *y2* are specified, the source region is replicated if necessary to fill the destination region in a tiled fashion.

If *shrink* is true, the size of the destination image is reduced, if necessary, so that the region being copied into is at the bottom-right corner of the image.

If *zoom* or *subsample* are specified, the image is transformed as in the **`zoom()`** or **`subsample()`** methods. The value must be a single integer or a pair of integers.

compositingrule specifies how transparent pixels in the source image are combined with the destination image. With `'overlay'` (the default), the old contents of the destination image remain visible, as if the source image were printed on a piece of transparent film and placed over the top of the destination. With `'set'`, the old contents of the destination image are discarded and the source image is used as-is.

Adicionado na versão 3.13.

data(*format=None*, *, *from_coords=None*, *background=None*, *grayscale=False*)

Return the image data.

format specifies the name of the image file format handler to use. If it is not given, the data is returned as a tuple (one element per row) of strings containing space-separated (one element per pixel/column) colors in `#RRGGBB` format.

from_coords specifies a rectangular region of the image to be returned. It must be a tuple or a list of 1 to 4 integers `(x1, y1, x2, y2)`. If only *x1* and *y1* are specified, the region extends from `(x1, y1)` to the bottom-right corner of the image. If all four coordinates are given, they specify diagonally opposite corners of the region, including `(x1, y1)` and excluding `(x2, y2)`. If *from_coords* is not given, the whole image is returned.

If *background* is specified, the data does not contain any transparency information; in all transparent pixels the color is replaced by the specified color.

If *grayscale* is true, the data does not contain color information; all pixel data is transformed into grayscale.

Adicionado na versão 3.13.

get(*x*, *y*)

Return the color of the pixel at coordinates (*x*, *y*) as an `(r, g, b)` tuple of three integers between 0 and 255, representing the red, green and blue components respectively.

put(*data*, *to=None*)

Set pixels of the image to the colors given in *data*, which must be a string or a nested sequence of horizontal rows of pixel colors (for example `"{red green} {blue yellow}"`).

to specifies the coordinates of the region of the image into which the data are copied. It must be a tuple or a list of 2 or 4 integers `(x1, y1)` or `(x1, y1, x2, y2)` giving the top-left corner, and optionally the bottom-right corner, of the region. The default position is `(0, 0)`.

read(*filename*, *format=None*, *, *from_coords=None*, *to=None*, *shrink=False*)

Read image data from the file named *filename* into the image.

format specifies the format of the image data in the file.

from_coords specifies a rectangular sub-region of the image file data to be copied to the destination image. It must be a tuple or a list of 1 to 4 integers `(x1, y1, x2, y2)`. If only *x1* and *y1* are specified, the region extends from `(x1, y1)` to the bottom-right corner of the image in the file. If all four coordinates are given, they specify diagonally opposite corners of the region. If *from_coords* is not given, the whole of the image in the file is read.

to specifies the coordinates of the top-left corner of the region of the image into which the data are read. The default is `(0, 0)`.

If *shrink* is true, the size of the image is reduced, if necessary, so that the region into which the file data are read is at the bottom-right corner of the image.

Adicionado na versão 3.13.

subsample(*x*, *y=''*, *, *from_coords=None*)

Return a new **`PhotoImage`** based on this image but using only every *x*-th pixel in the X direction and every *y*-th pixel in the Y direction. If *y* is not given, it defaults to the same value as *x*.

from_coords specifies a rectangular sub-region of the source image to be copied, as in the **`copy()`** method.

Alterado na versão 3.13: Added the *from_coords* parameter.

transparency_get(*x*, *y*)

Return `True` if the pixel at coordinates (*x*, *y*) is fully transparent, `False` otherwise.

Adicionado na versão 3.8.

transparency_set(*x*, *y*, *boolean*)

Make the pixel at coordinates (*x*, *y*) fully transparent if *boolean* is true, fully opaque otherwise.

Adicionado na versão 3.8.

write(*filename*, *format=None*, *from_coords=None*, *, *background=None*, *grayscale=False*)

Write image data from the image to the file named *filename*.

format specifies the name of the image file format handler to use. If it is not given, the format is guessed from the file extension.

from_coords specifies a rectangular region of the image to be written. It must be a tuple or a list of 1 to 4 integers `(x1, y1, x2, y2)`. If only *x1* and *y1* are specified, the region extends from `(x1, y1)` to the bottom-right corner of the image. If all four coordinates are given, they specify diagonally opposite corners of the region. If *from_coords* is not given, the whole image is written.

If *background* is specified, the data does not contain any transparency information; in all transparent pixels the color is replaced by the specified color.

If *grayscale* is true, the data does not contain color information; all pixel data is transformed into grayscale.

Alterado na versão 3.13: Added the *background* and *grayscale* parameters.

zoom(*x*, *y=''*, **, from_coords=None*)

Return a new **`PhotoImage`** with this image magnified by a factor of *x* in the X direction and *y* in the Y direction. If *y* is not given, it defaults to the same value as *x*.

from_coords specifies a rectangular sub-region of the source image to be copied, as in the **`copy()`** method.

Alterado na versão 3.13: Added the *from_coords* parameter.

class tkinter.BitmapImage(*name=None, cnf={}, master=None, **kw*)

A two-color image (the Tk `bitmap` image type) created from an X11 bitmap. Each pixel displays a foreground color, a background color, or nothing (producing a transparent effect). Inherits from **`Image`**.

The configuration options are *data* or *file* (the source bitmap, given as a string in X11 bitmap format or as the name of a file in that format), *maskdata* or *maskfile* (the mask bitmap, in the same forms), and *foreground* and *background* (the two colors). For pixels where the mask is zero the image displays nothing; for other pixels it displays the foreground color where the source is one and the background color where the source is zero. If *background* is set to an empty string, the background pixels are transparent.

`BitmapImage` has no methods of its own beyond those inherited from **`Image`**.

Other classes

class tkinter.Event

A container for the attributes of an event passed to a callback bound with `Misc.bind()`. An `Event` instance has the following attributes, each corresponding to a field of the underlying Tk event; depending on the event type, some attributes may be set to the string `'??'` to indicate that they are not meaningful. See Bindings and events.

serial

The serial number of the event.

num

The mouse button that was pressed or released (for button events).

focus

Whether the window has the focus (for `Enter` and `Leave` events).

height
width

The new height and width of the window (for `Configure` and `Expose` events).

keycode

The keycode of the key that was pressed or released.

state

The state of the event, as a number (for most events) or a string (for `Visibility` events).

time

The timestamp of the event, in milliseconds.

x
y

The pointer position relative to the widget, in pixels.

x_root
y_root

The pointer position relative to the top-left corner of the screen, in pixels.

char

The character typed, as a string (for key events).

send_event

`True` if the event was sent by another application.

keysym

The symbolic name of the key that was pressed or released.

keysym_num

The numeric value of **`keysym`**.

type

The **`EventType`** of the event.

widget

The widget on which the event occurred.

delta

The amount the mouse wheel was rotated (for `MouseWheel` events).

user_data

The data string of a virtual event, as passed to the *data* option of **`Misc.event_generate()`**. It is `'??'` for non-virtual events.

Adicionado na versão 3.15.

detail

A fixed detail string for `Enter`, `Leave`, `FocusIn`, `FocusOut` and `ConfigureRequest` events (see the Tcl/Tk documentation). It is `'??'` for other events.

Adicionado na versão 3.15.

class tkinter.EventType(**values*)

An `enum.StrEnum` enumerating the Tk event types, used as the value of `Event.type`. Its members include, among others, `KeyPress`, `KeyRelease`, `ButtonPress`, `ButtonRelease`, `Motion`, `Enter`, `Leave`, `FocusIn`, `FocusOut`, `Configure`, `Map`, `Unmap`, `Expose`, `Destroy` and `MouseWheel`.

Adicionado na versão 3.6.

class tkinter.CallWrapper(*func*, *subst*, *widget*)

Internal helper that wraps a Python callback so that it can be invoked from Tcl. *func* is the Python function, *subst* is an optional function that pre-processes the Tcl arguments, and *widget* is the widget used for error reporting. Instances are created automatically by `Misc.register()`; this class is not normally used directly.

Module-level functions

tkinter.Tcl(*screenName=None*, *baseName=None*, *className='Tk'*, *useTk=False*)

The **`Tcl()`** function is a factory function which creates an object much like that created by the `Tk` class, except that it does not initialize the Tk subsystem. This is most often useful when driving the Tcl interpreter in an environment where one doesn't want to create extraneous toplevel windows, or where one cannot (such as Unix/Linux systems without an X server). An object created by the **`Tcl()`** object can have a Toplevel window created (and the Tk subsystem initialized) by calling its `loadtk()` method.

tkinter.NoDefaultRoot()

Inhibit the creation of an implicit default root window. Afterwards **`tkinter`** no longer creates a shared default root automatically, and operations that rely on one — such as constructing a widget without an

explicit *master* — raise a **RuntimeError**. Call this early in larger applications to make the root window explicit.

tkinter.mainloop(*n=0*)

Run the Tk main event loop on the default root window until all windows are destroyed. Equivalent to calling **Misc.mainloop()** on the default root.

tkinter.getboolean(*s*)

Convert the Tcl boolean string *s* (one of `'1'`, `'true'`, `'yes'`, `'on'` and similar, or their false counterparts) to a Python **bool**. Raise **TclError** for an invalid value.

tkinter.getdouble(*s*)

Convert *s* to a floating-point number. This is the built-in **float**.

tkinter.getint(*s*)

Convert *s* to an integer. This is the built-in **int**.

tkinter.image_names()

Return the names of all existing images in the default root's interpreter.

tkinter.image_types()

Return the available image types (such as `'photo'` and `'bitmap'`) in the default root's interpreter.

File handlers

O Tk permite que você registre e cancele o registro de uma função de retorno que será chamada a partir do laço central do Tk quando uma E/S for possível em um descritor de arquivo. Apenas um tratador pode ser registrado por descritor de arquivo. Código de exemplo:

```
import tkinter
widget = tkinter.Tk()
mask = tkinter.READABLE | tkinter.WRITABLE
widget.tk.createfilehandler(file, mask, callback)
```

```
...
widget.tk.deletefilehandler(file)
```

Este recurso não está disponível no Windows.

Já que você não sabe quantos bytes estão disponíveis para leitura, você pode não querer usar os métodos `read()` ou `readline()` de `BufferedIOBase` ou `TextIOBase`, já que eles insistirão em ler uma quantidade predefinida de bytes. Para sockets, os métodos `recv()` ou `recvfrom()` vão servir; para outros arquivos, use dados brutos ou `os.read(file.fileno(), maxbytecount)`.

Widget.tk.createfilehandler(*file*, *mask*, *func*)

Registra a função de retorno do tratador de arquivo *func*. O argumento *file* pode ser um objeto com um método `fileno()` (como um arquivo ou objeto soquete) ou um descritor de arquivo de inteiro. O argumento *mask* é uma combinação OR de qualquer uma das três constantes abaixo. O retorno de chamada é chamado da seguinte maneira:

```
callback(file, mask)
```

Widget.tk.deletefilehandler(*file*)

Cancela o registro de um tratador de arquivo.

tkinter.READABLE
tkinter.WRITABLE
tkinter.EXCEPTION

Constantes usadas nos argumentos *mask*.

Constantes

The following symbolic constants are available in both the **`tkinter`** and **`tkinter.constants`** namespaces.

tkinter.TRUE
tkinter.YES
tkinter.ON

Truthy values, all equal to the integer `1`.

tkinter.FALSE
tkinter.NO
tkinter.OFF

Falsy values, all equal to the integer `0`.

tkinter.N
tkinter.S
tkinter.E
tkinter.W
tkinter.NE
tkinter.NW
tkinter.SE
tkinter.SW
tkinter.NS
tkinter.EW
tkinter.NSEW
tkinter.CENTER

Compass directions (`'n'`, `'s'`, `'e'`, `'w'` and the diagonals and edges) plus `CENTER` (`'center'`), used as values for the *anchor* and *sticky* options and by methods such as `Misc.grid_anchor()`.

tkinter.LEFT
tkinter.RIGHT
tkinter.TOP
tkinter.BOTTOM

Sides for the *side* option of the packer (see `Pack.pack_configure()`).

tkinter.X
tkinter.Y
tkinter.BOTH
tkinter.NONE

Values for the *fill* option of the packer: `'x'`, `'y'`, `'both'` or `'none'`.

tkinter.RAISED
tkinter.SUNKEN

tkinter.FLAT
tkinter.RIDGE
tkinter.GROOVE
tkinter.SOLID

Values for the *relief* option, which controls a widget's 3-D border.

tkinter.HORIZONTAL
tkinter.VERTICAL

Values for the *orient* option of widgets such as `Scale`, `Scrollbar` and `PanedWindow`.

tkinter.CHAR
tkinter.WORD

Values for the *wrap* option of the `Text` widget, selecting line wrapping on character or word boundaries.

tkinter.BASELINE

The text-alignment value `'baseline'`.

tkinter.INSIDE
tkinter.OUTSIDE

Values for the *bordermode* option of the placer (see `Place.place_configure()`).

tkinter.INSERT
tkinter.CURRENT
tkinter.END
tkinter.ANCHOR
tkinter.SEL
tkinter.SEL_FIRST
tkinter.SEL_LAST

Symbolic indices used by the `Text`, `Entry`, `Listbox` and `Canvas` widgets, such as `'insert'` (the insertion cursor), `'current'`, `'end'`, `'anchor'` and the bounds of the selection (`'sel.first'` and `'sel.last'`).

tkinter.ALL

The special tag `'all'`, which matches every item of a `Canvas` or every character of a `Text` (for example `canvas.delete(ALL)`).

tkinter.NORMAL
tkinter.DISABLED
tkinter.ACTIVE
tkinter.HIDDEN

Values for the *state* option of various widgets and items.

tkinter.CASCADE
tkinter.CHECKBUTTON
tkinter.COMMAND
tkinter.RADIOBUTTON
tkinter.SEPARATOR

Menu entry types, used as the *itemType* argument of `Menu.add()` and `Menu.insert()`.

tkinter.SINGLE
tkinter.BROWSE
tkinter.MULTIPLE
tkinter.EXTENDED

Values for the *selectmode* option of the `Listbox` widget.

tkinter.PIESLICE
tkinter.CHORD
tkinter.ARC

Values for the *style* option of `Canvas` arc items.

tkinter.BUTT
tkinter.PROJECTING
tkinter.ROUND
tkinter.BEVEL
tkinter.MITER

Values for the *capstyle* (`'butt'`, `'projecting'`, `'round'`) and *joinstyle* (`'round'`, `'bevel'`, `'miter'`) options of `Canvas` line items.

tkinter.FIRST

tkinter.LAST

Values for the *arrow* option of `Canvas` line items, indicating which ends have arrowheads.

tkinter.MOVETO
tkinter.SCROLL

The first argument passed by a `Scrollbar` to the `XView.xview()` or `YView.yview()` method of the scrolled widget.

tkinter.UNITS
tkinter.PAGES

Values for the *what* argument of `XView.xview_scroll()` and `YView.yview_scroll()`.

tkinter.UNDERLINE
tkinter.NUMERIC
tkinter.DOTBOX

Other option values: `'underline'`, `'numeric'` and `'dotbox'`.

`tkinter.colorchooser` — Diálogo de escolha de cor

Código-fonte: Lib/tkinter/colorchooser.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/colorchooser.py]

O módulo **`tkinter.colorchooser`** fornece a classe **`Chooser`** como uma interface para o diálogo do seletor de cores nativo. `Chooser` implementa uma janela de diálogo de escolha de cores modal. A classe `Chooser` herda da classe **`Dialog`**.

class tkinter.colorchooser.Chooser(*master=None*, ***options*)

A classe que implementa a caixa de diálogo modal de seleção de cores. A maioria das aplicações utiliza a função de conveniência **`askcolor()`** em vez de instanciar essa classe diretamente.

tkinter.colorchooser.askcolor(*color=None*, ***options*)

Exibe uma caixa de diálogo modal de seleção de cores e retorna a cor escolhida. *color* é a cor selecionada quando a caixa de diálogo é aberta. O valor de retorno é uma tupla `((r, g, b), hexstr)`, onde `r`, `g` e `b` são os componentes vermelho, verde e azul como números inteiros na faixa de 0 a 255, e *hexstr* é a string de cor equivalente do Tk, como `'#ff8000'`. Se o usuário cancelar a caixa de diálogo, é retornado `(None, None)`.

Alterado na versão 3.10: Os valores RGB na cor retornada agora são números inteiros na faixa de 0 a 255, em vez de números de ponto flutuante.

Ver também

Módulo **`tkinter.commondialog`**

Módulo de diálogo padrão do Tkinter

`tkinter.font` — Invólucro de fontes Tkinter

Código-fonte: Lib/tkinter/font.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/font.py]

The `tkinter.font` module provides the `Font` class for creating and using named fonts.

Os diferentes pesos e inclinações de fontes são:

tkinter.font.NORMAL
tkinter.font.BOLD
tkinter.font.ITALIC
tkinter.font.ROMAN

class tkinter.font.Font(*root=None*, *font=None*, *name=None*, *exists=False*, ***options*)

A classe `Font` representa uma fonte nomeada. Instâncias *font* recebem nomes únicos e podem ser especificadas por família, tamanho e configuração de estilo. Fontes nomeadas são os métodos Tk para criação e identificação de fontes de um objeto singular, ao invés de especificar a fonte por seus atributos em cada ocorrência.

Alterado na versão 3.10: Two fonts now compare equal (==) only when both are `Font` instances with the same name belonging to the same Tcl interpreter.

argumentos:

font - tupla de especificação da fonte (família, tamanho, opções)
name - nome único da fonte
exists - aponta para uma fonte nomeada existente se for verdadeiro

palavras reservadas opcionais (ignoradas caso *font* seja especificado):

family - font family, for example, Courier, Times
size - tamanho da fonte
Se *size* for positivo, ele é interpretado como tamanho em pontos.
Caso *size* seja um número negativo, seu valor absoluto é tratado
como tamanho em pixels.
weight - ênfase da fonte (normal,negrito)
slant - ROMAN, ITALIC
underline - sublinhamento da fonte (0 - sem sublinhamento, 1 - sublinhado)
overstrike - fonte riscada (0 - sem risco, 1 - letras riscadas)

actual(*option=None*, *displayof=None*)

Return the actual attributes of the font, which may differ from the requested ones because of platform limitations. With no *option*, return a dictionary of all the attributes; if *option* is given, return the value of that single attribute. The attributes are resolved on the display of the *displayof* widget, or the main application window if it is not specified.

cget(*option*)

Recupera um atributo da fonte.

configure(**options*)

Modify one or more attributes of the font. With no arguments, return a dictionary of the current attributes.

`config()` is an alias of **`configure()`**.

copy()

Return a distinct copy of the current font: a new named font with the same attributes but a different name, which can be reconfigured independently of the original. If the current font wraps a font

description, the copy is instead a named font with its resolved attributes.

measure(*text*, *displayof=None*)

Return amount of space the text would occupy on the specified display when formatted in the current font, as an integer number of pixels. If no display is specified then the main application window is assumed.

metrics(**options*, ***kw*)

Return font-specific data. With no options, return a dictionary mapping each metric name to its integer value; if one option name is given, return that metric's value as an integer. Options include:

ascent - distância entre a linha de base e o ponto mais alto que um caractere da fonte pode ocupar

descent - distância entre a linha de base e o ponto mais baixo que um caractere da fonte pode ocupar

linespace - separação vertical mínima necessária entre dois caracteres da fonte que garante que não ocorra sobreposição entre as linhas.

fixed - 1 caso a fonte seja de largura fixa, 0 caso contrário

tkinter.font.families(*root=None*, *displayof=None*)

Return a tuple of the names of the available font families.

tkinter.font.names(*root=None*)

Return a tuple of the names of all the defined fonts.

tkinter.font.nametofont(*name*, *root=None*)

Return a `Font` representation of the existing named font *name*. *root* is the widget whose Tcl interpreter owns the font; if omitted, the default root window is used.

Alterado na versão 3.10: O parâmetro *root* foi adicionado.

Tkinter dialogs

`tkinter.simpledialog` — Standard Tkinter input dialogs

Código-fonte: Lib/tkinter/simpledialog.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/simpledialog.py]

The **`tkinter.simpledialog`** module contains convenience classes and functions for creating simple modal dialogs to get a value from the user.

tkinter.simpledialog.askfloat(*title*, *prompt*, ***, *initialvalue=None*, *minvalue=None*, *maxvalue=None*, *parent=None*)
tkinter.simpledialog.askinteger(*title*, *prompt*, ***, *initialvalue=None*, *minvalue=None*, *maxvalue=None*, *parent=None*)
tkinter.simpledialog.askstring(*title*, *prompt*, ***, *initialvalue=None*, *show=None*, *parent=None*)

Prompt the user to enter a value of the desired type and return it, or `None` if the dialog is cancelled.

title is the dialog title and *prompt* the message shown above the entry. *initialvalue* is the value initially placed in the entry. *parent* is the window over which the dialog is shown. **`askinteger()`** and **`askfloat()`** also accept *minvalue* and *maxvalue*, which bound the accepted value. **`askstring()`** also accepts *show*, a character used to mask the entered text, for example `'*'` to hide a password.

class tkinter.simpledialog.Dialog(*parent*, *title=None*)

The base class for custom dialogs. Instantiating it shows the dialog modally and returns once the user closes it; the entered value is then available in the **`result`** attribute.

result

The value produced by **apply()**, or `None` if the dialog was cancelled.

body(*master*)

Substitui para construir a interface da caixa de diálogo e retornar o widget que deve ter foco inicial.

buttonbox()

O comportamento padrão adiciona botões OK e Cancelar. Substitua para layouts de botão personalizados.

validate()

Validate the data entered by the user. Return true if it is valid, in which case the dialog proceeds to **apply()**; return false to keep the dialog open. The default implementation always returns true; override it to check the input.

apply()

Process the data entered by the user, for example by storing it in the **result** attribute. Called after **validate()** succeeds and just before the dialog is destroyed. The default implementation does nothing; override it to act on or store the result.

destroy()

Destroy the dialog window, clearing the reference to the widget that had the initial focus.

class tkinter.simpledialog.SimpleDialog(*master*, *text=''*, *buttons=[]*, *default=None*, *cancel=None*, *title=None*, *class_=None*)

A simple modal dialog that displays the message *text* above a row of push buttons whose labels are given by *buttons*, and returns the index of the button the user presses. *default* is the index of the button activated by the Return key, *cancel* the index returned when the window is closed through the window manager, *title* the window title, and *class_* the Tk class name of the window.

go()

Display the dialog, wait until the user presses a button or closes the window, and return the index of the chosen button.

`tkinter.filedialog` — File selection dialogs

Código-fonte: Lib/tkinter/filedialog.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/filedialog.py]

The `tkinter.filedialog` module provides classes and factory functions for creating file/directory selection windows.

Native load/save dialogs

As seguintes classes e funções fornecem janelas de diálogo de arquivo que combinam uma aparência nativa com opções de configuração para personalizar o comportamento. Os seguintes argumentos nomeados são aplicáveis às classes e funções listado abaixo:

parent - a janela para colocar a caixa de diálogo no topo

title - o título da janela

initialdir - o diretório no qual a caixa de diálogo começa

initialfile - o arquivo selecionado ao abrir a caixa de diálogo

filetypes - uma sequência de tuplas (rótulo, padrão), o caractere curinga '*' é permitido

defaultextension - extensão padrão para anexar ao arquivo (caixas de diálogo para salvar)

multiple - quando verdadeiro, a seleção de vários itens é permitida

Fábrica de funções estáticas

The below functions when called create a modal, native look-and-feel dialog, wait for the user's selection, and return it. The exact return value depends on the function (see below); when the dialog is cancelled it is an empty string, an empty tuple or `None`. The precise type of this empty value may vary between platforms and Tk versions, so test the result for truth rather than comparing it with a specific value.

tkinter.filedialog.askopenfile(*mode='r', **options*)

tkinter.filedialog.askopenfiles(*mode='r', **options*)

Create an **`Open`** dialog. **`askopenfile()`** returns the opened file object, or `None` if the dialog is cancelled. **`askopenfiles()`** returns a list of the opened file objects, or an empty tuple if cancelled. The files are opened in mode *mode* (read-only `'r'` by default).

tkinter.filedialog.asksaveasfile(*mode='w', **options*)

Create a **`SaveAs`** dialog and return the opened file object, or `None` if the dialog is cancelled. The file is opened in mode *mode* (`'w'` by default).

tkinter.filedialog.askopenfilename(***options*)

tkinter.filedialog.askopenfilenames(***options*)

Create an **`Open`** dialog. **`askopenfilename()`** returns the selected filename as a string, or an empty string if the dialog is cancelled. **`askopenfilenames()`** returns a tuple of the selected filenames, or an empty tuple if cancelled.

tkinter.filedialog.asksaveasfilename(***options*)

Create a **`SaveAs`** dialog and return the selected filename as a string, or an empty string if the dialog is cancelled.

tkinter.filedialog.askdirectory(***options*)

Prompt the user to select a directory, and return its path as a string, or an empty string if the dialog is cancelled. Additional keyword option: *mustexist* - if true, the user may only select an existing directory (false by default).

class tkinter.filedialog.Open(*master=None, **options*)

class tkinter.filedialog.SaveAs(*master=None, **options*)

class tkinter.filedialog.Directory(*master=None, **options*)

The above three classes provide native dialog windows for loading and saving files and for selecting a directory.

Classes de conveniência

As classes abaixo são usadas para criar janelas de arquivos/diretórios desde o início. Elas não emulam a aparência nativa da plataforma.

Nota

A classe *FileDialog* deve ser uma subclasse para manipulação e comportamento de eventos personalizados.

class tkinter.filedialog.FileDialog(*master, title=None*)

Cria uma caixa de diálogo básica de seleção de arquivo.

cancel_command(*event=None*)

Aciona o encerramento da janela de diálogo.

dirs_double_event(*event*)

Manipulador de eventos para evento de clique duplo no diretório.

dirs_select_event(*event*)

Manipulador de eventos para evento de clique no diretório.

files_double_event(*event*)

Manipulador de eventos para evento de clique duplo no arquivo.

files_select_event(*event*)

Manipulador de eventos para evento de clique único no arquivo.

filter_command(*event=None*)

Filtra os arquivos por diretório.

get_filter()

Recupera o filtro de arquivo atualmente em uso.

get_selection()

Recupera o item atualmente selecionado.

go(*dir_or_file=os.curdir*, *pattern='*'*, *default=''*, *key=None*)

Caixa de diálogo de renderização e inicia um laço de eventos.

ok_event(*event*)

Sai da caixa de diálogo retornando a seleção atual.

ok_command()

Called when the user confirms the current selection. The base implementation accepts the selection and closes the dialog; `LoadFileDialog` and `SaveFileDialog` override it to check the selection first.

quit(*how=None*)

Sai da caixa de diálogo retornando o nome do arquivo, se houver.

set_filter(*dir*, *pat*)

Define o filtro de arquivo.

set_selection(*file*)

Atualiza a seleção de arquivo atual para *file*.

class tkinter.filedialog.LoadFileDialog(*master*, *title=None*)

Uma subclasse de FileDialog que cria uma janela de diálogo para selecionar um arquivo existente.

ok_command()

Testa se um arquivo é fornecido e se a seleção indica um já arquivo existente.

class tkinter.filedialog.SaveFileDialog(*master*, *title=None*)

Uma subclasse de FileDialog que cria uma janela de diálogo para selecionar um arquivo de destino.

ok_command()

Testa se a seleção aponta ou não para um arquivo válido que não é um diretório. A confirmação é necessária se um arquivo já existente for selecionado.

`tkinter.commondialog` — Dialog window templates

Código-fonte: Lib/tkinter/commondialog.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/commondialog.py]

The `tkinter.commondialog` module provides the `Dialog` class that is the base class for dialogs defined in other supporting modules.

class tkinter.commondialog.Dialog(*master=None*, ***options*)

show(***options*)

Renderiza a janela de diálogo.

`tkinter.dialog` — Classic Tk dialog boxes

Source code: Lib/tkinter/dialog.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/dialog.py]

The `tkinter.dialog` module provides a simple modal dialog box built on the classic (non-themed) Tk widgets.

tkinter.dialog.DIALOG_ICON

The name of a bitmap (`'questhead'`) suitable for use as the *bitmap* of a **`Dialog`**.

class tkinter.dialog.Dialog(*master=None*, *cnf={}*, ***kw*)

Display a modal dialog box built from the classic (non-themed) Tk widgets and wait for the user to press one of its buttons. The options, given through *cnf* or as keyword arguments, are all required: *title* (the window title), *text* (the message), *bitmap* (the name of a bitmap icon, such as **`DIALOG_ICON`**), *default* (the index of the default button) and *strings* (the sequence of button labels). After construction, the **`num`** attribute holds the index of the button the user pressed.

destroy()

Do nothing. The dialog window is destroyed automatically before the constructor returns, so there is nothing left for this method to do.

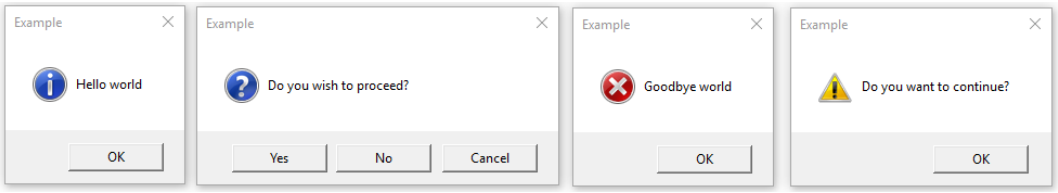
Ver também

Módulos **`tkinter.messagebox`**, Lendo e escrevendo arquivos

`tkinter.messagebox` — Prompts de mensagem do Tkinter

Código-fonte: Lib/tkinter/messagebox.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/messagebox.py]

The **`tkinter.messagebox`** module provides a template base class as well as a variety of convenience methods for commonly used configurations. The message boxes are modal: each blocks until the user responds, then returns a value that depends on the function. The `show*` functions and **`Message.show()`** return the symbolic name of the button the user pressed, as a string (such as **`OK`** or **`YES`**). Common message box styles and layouts include but are not limited to:



class tkinter.messagebox.Message(*master=None*, ***options*)

Criar uma janela de mensagem com uma mensagem específica da aplicação, um ícone e um conjunto de botões. Cada botão na janela é identificado com um nome simbólico único (veja as opções de “type”).

As seguintes opções são suportadas:

“command”

Especifica a função que será invocada quando o usuário fecha a janela de diálogo. O nome do botão clicado pelo usuário para fechar a janela é passada como um argumento. Esta funcionalidade está disponível apenas no macOS.

“default”

Fornece o nome simbólico do botão padrão para essa janela de mensagem (**OK**, **CANCEL**, e assim por diante). Se essa opção não for especificada, o primeiro botão da caixa de diálogo será o padrão.

“detail”
Especifica uma mensagem auxiliar para a mensagem principal fornecida pela opção *message*. Esta mensagem será apresentado abaixo da mensagem principal e, quando suportado pelo sistema operacional, em uma fonte menos enfatizada do que a mensagem principal.

“icon”
Especifica um icon a ser apresentado. Se essa opção não for especificada, o ícone **INFO** será exibido.

“message”
Especifica a mensagem para mostrar nessa caixa de mensagem. O padrão valor é uma string vazia.

“parent”
Torna a janela especificada a janela pai da caixa de mensagem. A caixa de mensagem é exibida na parte superior de sua janela pai.

“title”
Especifica uma string para mostrar como o título da caixa de mensagem. Essa opção é ignorada no macOS, onde a plataforma proíbe o uso de um título nesse tipo de caixa de diálogo.

“type”
Organiza um conjunto predefinido de botões a serem mostrados.

Nota

Tk 8.6 added the *command* option.

show(***options*)

Mostra uma janela de mensagem e aguarda que o usuário selecione um dos botões. Em seguida, retorna o nome simbólico do botão selecionado. Argumentos nomeados pode substituir opções especificadas no construtor.

Caixa de mensagem de informação

tkinter.messagebox.showinfo(*title=None*, *message=None*, ***options*)

Cria e exibe uma caixa de mensagem informativa com o título e a mensagem especificados.

Caixas de mensagem de atenção

tkinter.messagebox.showwarning(*title=None*, *message=None*, ***options*)

Cria e exibe uma caixa de mensagem de alerta com o título e a mensagem especificados.

tkinter.messagebox.showerror(*title=None*, *message=None*, ***options*)

Cria e exibe uma caixa de mensagem de erro com o título e a mensagem especificados.

Caixas de mensagem de dúvida

tkinter.messagebox.askquestion(*title=None*, *message=None*, ***, *type=YESNO*, ***options*)

Faz uma pergunta. Por padrão mostra os botões `YES` e `NO`. Retorna o nome simbólico do botão selecionado.

tkinter.messagebox.askokcancel(*title=None*, *message=None*, ***options*)

Pergunta se a operação deve prosseguir. Mostra os botões `OK` e `CANCEL`. Retorna `True` se a resposta for “OK” e `False` caso contrário.

tkinter.messagebox.askretrycancel(*title=None*, *message=None*, ***options*)

Ask if operation should be retried. Shows buttons **RETRY** and **CANCEL**. Return `True` if the answer is retry and `False` otherwise.

tkinter.messagebox.askyesno(*title=None*, *message=None*, ***options*)

Faz uma pergunta. Mostra os botões **YES** e **NO**. Retorna `True` se a resposta for sim e `False` caso contrário.

tkinter.messagebox.askyesnocancel(*title=None*, *message=None*, ***options*)

Faz uma pergunta. Mostra os botões **YES**, **NO** e **CANCEL**. Retorna `True` se a resposta for sim, `None` se cancelado, e `False` caso contrário.

Nomes simbólicos dos botões:

tkinter.messagebox.ABORT = *'abort'*

tkinter.messagebox.RETRY = *'retry'*

tkinter.messagebox.IGNORE = *'ignore'*

tkinter.messagebox.OK = *'ok'*

tkinter.messagebox.CANCEL = *'cancel'*

tkinter.messagebox.YES = *'yes'*

tkinter.messagebox.NO = *'no'*

Conjunto predefinido de botões:

tkinter.messagebox.ABORTRETRYIGNORE = *'abortretryignore'*

Exibe três botões cujos nomes simbólicos são **ABORT**, **RETRY** e **IGNORE**.

tkinter.messagebox.OK = *'ok'*

Exibe um botão cujo nome simbólico é **OK**.

tkinter.messagebox.OKCANCEL = *'okcancel'*

Exibe dois botões cujos nomes simbólicos são **OK** e **CANCEL**.

tkinter.messagebox.RETRYCANCEL = *'retrycancel'*

Exibe dois botões cujos nomes simbólicos são **RETRY** e **CANCEL**.

tkinter.messagebox.YESNO = *'yesno'*

Exibe dois botões cujos nomes simbólicos são **YES** e **NO**.

tkinter.messagebox.YESNOCANCEL = *'yesnocancel'*

Exibe três botões cujos nomes simbólicos são **YES**, **NO** e **CANCEL**.

Imagens de ícones:

tkinter.messagebox.ERROR = *'error'*

tkinter.messagebox.INFO = *'info'*

tkinter.messagebox.QUESTION = *'question'*

tkinter.messagebox.WARNING = *'warning'*

`tkinter.scrolledtext` — Widget de texto com *scrollbar*

Código-fonte: Lib/tkinter/scrolledtext.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/scrolledtext.py]

O módulo **`tkinter.scrolledtext`** fornece uma classe com o mesmo nome que implementa um widget de texto básico que possui uma barra de rolagem vertical configurada para fazer a "coisa certa". Usar a classe **`ScrolledText`** é muito mais fácil do que configurar um widget de texto e barra de rolagem diretamente.

O widget de texto e a barra de rolagem são agrupados em **`Frame`**, e os métodos dos gerenciadores de geometria **`Pack`**, **`Grid`** e **`Place`** são adquiridos do objeto **`Frame`**. Isso permite que o widget **`ScrolledText`** seja usado diretamente para obter o comportamento mais normal de gerenciamento de geometria.

Se um controle mais específico for necessário, os seguintes atributos estarão disponíveis:

class tkinter.scrolledtext.ScrolledText(*master=None*, ***kw*)

frame

O quadro que envolve os widgets de barra de rolagem e texto.

vbar

O widget da barra de rolagem.

`tkinter.dnd` — Suporte para arrastar e soltar

Código-fonte: Lib/tkinter/dnd.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/dnd.py]

Nota

Isto é experimental e deverá ser descontinuado quando for substituído pelo Tk DND.

The **`tkinter.dnd`** module provides drag-and-drop support for objects within a single application, within the same window or between windows. To enable an object to be dragged, you must create an event binding for it that starts the drag-and-drop process. Typically, you bind a ButtonPress event to a callback function that you write (see Bindings and events). The function should call **`dnd.start()`**, where *source* is the object to be dragged, and *event* is the event that invoked the call (the argument to your callback function).

A seleção de um objeto de destino ocorre da seguinte forma:

1. Top-down search of the area under the mouse for a target widget:
   - the target widget should have a callable *dnd_accept* attribute;
   - if *dnd_accept* is not present or returns `None`, the search moves to the parent widget;
   - if no target widget is found, the target object is `None`.
2. Call to `<old_target>.dnd_leave(source, event)`.
3. Call to `<new_target>.dnd_enter(source, event)`.
4. Call to `<target>.dnd_commit(source, event)` to notify of the drop.
5. Call to `<source>.dnd_end(target, event)` to signal the end of drag-and-drop.

class tkinter.dnd.DndHandler(*source*, *event*)

A classe *DndHandler* trata de eventos de arrastar e soltar que rastreiam eventos Motion e ButtonRelease na raiz do widget de evento.

cancel(*event=None*)

Cancela o processo de arrastar e soltar.

finish(*event*, *commit=0*)

Executa o fim das funções de arrastar e soltar.

on_motion(*event*)

Inspect area below mouse for target objects while a drag is performed.

on_release(*event*)

Sinaliza o fim do arrasto quando o padrão de liberação for acionado.

tkinter.dnd.dnd_start(*source*, *event*)

Factory function for the drag-and-drop process. Return the `DndHandler` instance managing the drag, or `None` if a drag could not be started.

Ver também

Bindings and events

`tkinter.ttk` — Widgets temáticos do Tk

Código-fonte: Lib/tkinter/ttk.py
[https://github.com/python/cpython/tree/3.15/Lib/tkinter/ttk.py]

The `tkinter.ttk` module provides access to the Tk themed widget set, introduced in Tk 8.5. Its widgets adapt their appearance to the platform's native theme, giving an application a better and more consistent look and feel than the classic `tkinter` widgets, whose appearance is fixed.

The basic idea for `tkinter.ttk` is to separate, to the extent possible, the code implementing a widget's behavior from the code implementing its appearance.

Ttk widgets are used just like the classic `tkinter` widgets and share the same machinery: the widget hierarchy, the geometry managers, variable coupling and event binding. Those foundational concepts are covered in the `tkinter` documentation and are not repeated here.

Adicionado na versão 3.1.

Ver também

Tk Widget Styling Support (TIP #48) **[https://tip.tcl-lang.org/48.html]**
The Tcl Improvement Proposal that introduced the themed widget styling engine.

Usando Ttk

Para começar a usar Ttk, importe seu módulo:

```
from tkinter import ttk
```

Para substituir os widgets básicos do Tk, a importação deve seguir a importação do Tk:

```
from tkinter import *
from tkinter.ttk import *
```

That code causes several **`tkinter.ttk`** widgets (**`Button`**, **`Checkbutton`**, **`Entry`**, **`Frame`**, **`Label`**, **`LabelFrame`**, **`Menubutton`**, **`OptionMenu`**, **`PanedWindow`**, **`Radiobutton`**, **`Scale`**, **`Scrollbar`** and **`Spinbox`**) to automatically replace the Tk widgets.

> **Nota**
>
> Overriding the classic widgets with `from tkinter.ttk import *` is convenient for adapting existing code, but new code is usually clearer if it imports the module as `from tkinter import ttk` and refers to the themed widgets explicitly, such as `ttk.Button`.

This has the direct benefit of using the new widgets which gives a better look and feel across platforms; however, the replacement widgets are not completely compatible. The main difference is that widget options such as `fg`, `bg` and others related to widget styling are no longer present in Ttk widgets. Instead, use the **`ttk.Style`** class for improved styling effects.

Ttk widgets

Ttk comes with 18 widgets, twelve of which already existed in tkinter: **`Button`**, **`Checkbutton`**, **`Entry`**, **`Frame`**, **`Label`**, **`LabelFrame`**, **`Menubutton`**, **`PanedWindow`**, **`Radiobutton`**, **`Scale`**, **`Scrollbar`**, and **`Spinbox`**. The other six are new: **`Combobox`**, **`Notebook`**, **`Progressbar`**, **`Separator`**, **`Sizegrip`** and **`Treeview`**. All of them are subclasses of **`Widget`**.

Utilizar os widgets Ttk melhora a experiência de uso e o visual da aplicação. Conforme discutido anteriormente, existem diferenças em como

a estilização é codificada.

Código Tk:

```
l1 = tkinter.Label(text="Test", fg="black", bg="white")
l2 = tkinter.Label(text="Test", fg="black", bg="white")
```

Código Ttk:

```
style = ttk.Style()
style.configure("BW.TLabel", foreground="black", background="whi

l1 = ttk.Label(text="Test", style="BW.TLabel")
l2 = ttk.Label(text="Test", style="BW.TLabel")
```

Para mais informações sobre TtkStyling, veja a documentação da classe `Style`.

Widget

`ttk.Widget` defines standard options and methods supported by Tk themed widgets and is not supposed to be directly instantiated.

Standard options

All the `ttk` Widgets accept the following options:

Opção	Descrição
class	Specifies the window class. The class is used when querying the option database for the window's other options, to determine the default bindtags for the window, and to select the widget's default layout and style. This option is read-only, and may only be specified when the window is created.

Opção	Descrição
cursor	Specifies the mouse cursor to be used for the widget. See the *cursor* option type under Tk option data types. If set to the empty string (the default), the cursor is inherited from the parent widget.
takefocus	Determina se a janela aceitará foco durante a navegação com o teclado. Retorna 0, 1 ou uma string vazia. Caso retorne 0, significa que a janela deve ser completamente ignorada durante a navegação com o teclado. Caso retorne 1, significa que a janela receberá as entradas do teclado enquanto estiver visível. E a string vazia significa que os scripts de navegação determinam quando a janela recebe foco ou não.
style	Pode ser utilizado para especificar um estilo customizado do widget.

Scrollable widget options

As opções a seguir são suportadas pelos widgets que são controlados por uma barra de rolagem.

Opção	Descrição

Opção	Descrição
xscrollcommand	Utilizado para comunicação com barras de rolagem horizontais. When the view in the widget's window changes, the widget calls the *xscrollcommand* callback. Usually this option consists of the method `Scrollbar.set` of some scrollbar. This will cause the scrollbar to be updated whenever the view in the window changes.
yscrollcommand	Utilizado para comunicação com barras de rolagem verticais. Para mais informações, veja acima.

Label options

As opções a seguir são suportadas por rótulos, botões e outros widgets semelhantes.

Opção	Descrição
text	Especifica uma string para exibição dentro do widget.
textvariable	Specifies a name whose value will be used in place of the text option resource.

Opção	Descrição
underline	Caso definido, especifica o índice (base-0) de um caractere para sublinhamento na string de texto. O caractere sublinhado é utilizado para ativação mnemônica.
image	Define uma imagem para exibição. Isso é uma lista de 1 ou mais elementos. O primeiro elemento é o nome da imagem padrão. O resto da lista é uma sequência de pares estado/valor conforme definido por `Style.map()`, definindo imagens diferentes para utilização quando o widget estiver em um determinado estado ou em uma combinação de estados. Todas as imagens da lista devem ter o mesmo tamanho.
compound	Define como exibir a imagem em relação ao texto, no caso em que ambas as opções de texto e imagem estejam presentes. Valores válidos são: • texto: exibir apenas texto • imagem: exibir apenas a imagem • top, bottom, left, right: exibem a imagem acima, abaixo, à esquerda, ou à direita do texto, respectivamente. • none: o padrão, exibe a imagem caso ela esteja presente, caso contrário , exibe o texto.
width	Caso maior que zero, especifica quanto espaço, em largura de caracteres, alocar para o rótulo do texto, se menor que zero, especifica a largura mínima. Se zero ou inespecífico, é utilizada a largura normal do rótulo de texto.

Compatibility options

Opção	Descrição
state	May be set to “normal” or “disabled” to control the “disabled” state bit. This is a write-only option: setting it changes the widget state, but the `Widget.state()` method does not affect this option.

Widget states

O estado do widget é um bitmap de sinalizadores de estado independentes.

Sinalizador	Descrição
active	O cursor do mouse está sobre o widget e pressionar um botão do mouse causará alguma ação
disabled	Widget está desabilitado sob controle do programa.
focus	Widget tem foco do teclado
pressed	Widget está sendo pressionado
selected	“On”, “true”, ou “current” para coisas como botões de confirmação e botões de opção.

Sinalizador	Descrição
background	Windows e Mac possuem uma noção de janela “ativa” ou em primeiro plano. O estado de “plano de fundo” é definido para widgets em uma janela no plano de fundo, e limpo para aqueles na janela de primeiro plano
readonly	Widget não deve permitir modificação do usuário
alternate	Um formato de exibição específico do widget
invalid	O valor do widget é inválido

Uma especificação de estado é uma sequência de nomes de estado, opcionalmente prefixada por um ponto de exclamação indicando que o bit está desligado.

ttk.Widget

Besides the methods described below, the `ttk.Widget` supports the methods `tkinter.Widget.cget` and `tkinter.Widget.configure`.

class tkinter.ttk.Widget

identify(*x*, *y*)

Retorna o nome do elemento na posição *x y*, ou a string vazia caso o ponto não esteja dentro de algum elemento.

x e *y* são coordenadas de pixel relativas ao widget.

instate(*statespec*, *callback=None*, **args*, ***kw*)

Test the widget's state. If a callback is not specified, returns `True` if the widget state matches *statespec* and `False` otherwise. If callback is specified then it is called with args if widget state matches *statespec*.

state(*statespec=None*)

Modify or inquire widget state. If *statespec* is specified, sets the widget state according to it and return a new *statespec* indicating which flags were changed. If *statespec* is not specified, returns the currently enabled state flags.

Not to be confused with `Wm.state`.

statespec will usually be a list or a tuple.

Combobox

The `ttk.Combobox` widget combines a text field with a pop-down list of values. This widget is a subclass of `Entry`.

Besides the methods inherited from `Widget`: `cget()`, `configure()`, `identify()`, `instate()` and `state()`, and the following inherited from `Entry`: `bbox()`, `delete()`, `icursor()`, `index()`, `insert()`, `selection*`, `xview*`, it has some other methods, described at `ttk.Combobox`.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
exportselection	Boolean value. If set, the widget selection is linked to the X selection (which can be returned by invoking Misc.selection_get, for example).

Opção	Descrição
justify	Especifica como o texto é alinhado dentro do widget. Pode ser “left”, “center” ou “right”.
height	Especifica a altura, em linhas, da lista suspensa.
postcommand	Um script (possivelmente registrado com Misc.register) que é chamado imediatamente antes da exibição dos valores. Ele pode especificar quais valores exibir.
state	One of “normal”, “readonly”, or “disabled”. In the “readonly” state, the value may not be edited directly, and the user can only select one of the values from the dropdown list. In the “normal” state, the text field is directly editable. In the “disabled” state, no interaction is possible.
textvariable	Especifica um nome que terá seu valor vinculado ao valor do widget. Sempre que o valor associado a esse nome mudar, o valor do widget será atualizado, e vice versa. Veja `tkinter.StringVar`.
values	Especifica a lista de valores para exibição na lista suspensa.
width	Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget’s font.

Nota

Tk 9.1 added the *locale* option, which selects the locale used to determine word and character boundaries within the text (`"C"` by default).

Eventos virtuais

Os widgets combobox geram um evento virtual **<<ComboboxSelected>>** quando o usuário seleciona um elemento da lista de valores.

ttk.Combobox

class tkinter.ttk.Combobox

current(*newindex=None*)

If *newindex* is specified, sets the combobox value to the element position *newindex*. Otherwise, returns the index of the current value or -1 if the current value is not in the values list.

get()

Retorna o valor atual do combobox.

set(*value*)

Sets the value of the combobox to *value*.

Spinbox

The `ttk.Spinbox` widget is a `ttk.Entry` enhanced with increment and decrement arrows. It can be used for numbers or lists of string values. This widget is a subclass of `Entry`. Besides the methods inherited from `Widget`: `cget()`, `configure()`, `identify()`, `instate()` and `state()`, and the following inherited from `Entry`: `bbox()`, `delete()`, `icursor()`, `index()`, `insert()`, `xview*`, it has some other methods, described at `ttk.Spinbox`.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
from	Float value. If set, this is the minimum value to which the decrement button will decrement. Must be spelled as `from_` when used as an argument, since `from` is a Python keyword.
to	Float value. If set, this is the maximum value to which the increment button will increment.
increment	Float value. Specifies the amount which the increment/decrement buttons change the value. Defaults to 1.0.
values	Sequence of string or float values. If specified, the increment/decrement buttons will cycle through the items in this sequence rather than incrementing or decrementing numbers.
wrap	Boolean value. If `True`, increment and decrement buttons will cycle from the `to` value to the `from` value or the `from` value to the `to` value, respectively.
format	String value. This specifies the format of numbers set by the increment/decrement buttons. It must be in the form “%W.Pf”, where W is the padded width of the value, P is the precision, and ‘%’ and ‘f’ are literal.

Opção	Descrição
command	Python callable. Will be called with no arguments whenever either of the increment or decrement buttons are pressed.

Eventos virtuais

The spinbox widget generates an **<<Increment>>** virtual event when the user presses <Up>, and a **<<Decrement>>** virtual event when the user presses <Down>.

ttk.Spinbox

class tkinter.ttk.Spinbox

With a non-integer increment, see numeric values and the locale.

Adicionado na versão 3.8.

get()

Returns the current value of the spinbox.

set(*value*)

Sets the value of the spinbox to *value*.

Notebook

Ttk Notebook widget manages a collection of windows and displays a single one at a time. Each child window is associated with a tab, which the user may select to change the currently displayed window.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
height	If present and greater than zero, specifies the desired height of the pane area (not including internal padding or tabs). Otherwise, the maximum height of all panes is used.
padding	Specifies the amount of extra space to add around the outside of the notebook. The padding is a list up to four length specifications left top right bottom. If fewer than four elements are specified, bottom defaults to top, right defaults to left, and top defaults to left.
width	If present and greater than zero, specified the desired width of the pane area (not including internal padding). Otherwise, the maximum width of all panes is used.

Tab options

There are also specific options for tabs:

Opção	Descrição
state	Either “normal”, “disabled” or “hidden”. If “disabled”, then the tab is not selectable. If “hidden”, then the tab is not shown.

Opção	Descrição
sticky	Specifies how the child window is positioned within the pane area. Value is a string containing zero or more of the characters “n”, “s”, “e” or “w”. Each letter refers to a side (north, south, east or west) that the child window will stick to, as per the `grid` geometry manager.
padding	Specifies the amount of extra space to add between the notebook and this pane. Syntax is the same as for the option padding used by this widget.
text	Specifies a text to be displayed in the tab.
image	Specifies an image to display in the tab. See the option image described in `Widget`.
compound	Specifies how to display the image relative to the text, in the case both options text and image are present. See Label Options for legal values.
underline	Specifies the index (0-based) of a character to underline in the text string. The underlined character is used for mnemonic activation if `Notebook.enable_traversal()` is called.

Tab identifiers

The tab_id present in several methods of `ttk.Notebook` may take any of the following forms:

- An integer between zero and the number of tabs
- The name of a child window
- A positional specification of the form “@x,y”, which identifies the tab
- The literal string “current”, which identifies the currently selected tab
- The literal string “end”, which returns the number of tabs (only valid for `Notebook.index()`)

Eventos virtuais

This widget generates a <<**NotebookTabChanged**>> virtual event after a new tab is selected.

ttk.Notebook

class tkinter.ttk.Notebook

add(*child*, ***kw*)

Adds a new tab to the notebook.

If window is currently managed by the notebook but hidden, it is restored to its previous position.

See Tab Options for the list of available options.

forget(*tab_id*)

Removes the tab specified by *tab_id*, unmaps and unmanages the associated window.

This shadows the inherited geometry-manager `forget()`; use `pack_forget()`, `grid_forget()` or `place_forget()` to remove the widget itself from its manager.

hide(*tab_id*)

Hides the tab specified by *tab_id*.

The tab will not be displayed, but the associated window remains managed by the notebook and its configuration remembered. Hidden tabs may be restored with the `add()` command.

identify(*x*, *y*)

Returns the name of the tab element at position *x*, *y*, or the empty string if none.

index(*tab_id*)

Returns the numeric index of the tab specified by *tab_id*, or the total number of tabs if *tab_id* is the string “end”.

insert(*pos*, *child*, ***kw*)

Inserts a pane at the specified position.

pos is either the string “end”, an integer index, or the name of a managed child. If *child* is already managed by the notebook, moves it to the specified position.

See Tab Options for the list of available options.

select(*tab_id=None*)

Selects the specified *tab_id*.

The associated child window will be displayed, and the previously selected window (if different) is unmapped. If *tab_id* is omitted, returns the widget name of the currently selected pane.

tab(*tab_id*, *option=None*, ***kw*)

Query or modify the options of the specific *tab_id*.

If *kw* is not given, returns a dictionary of the tab option values. If *option* is specified, returns the value of that *option*. Otherwise, sets the options to the corresponding values.

tabs()

Returns a tuple of windows managed by the notebook.

enable_traversal()

Enable keyboard traversal for a toplevel window containing this notebook.

This will extend the bindings for the toplevel window containing the notebook as follows:

- `Control-Tab`: selects the tab following the currently selected one.
- `Shift-Control-Tab`: selects the tab preceding the currently selected one.
- `Alt-K`: where *K* is the mnemonic (underlined) character of any tab, will select that tab.

Multiple notebooks in a single toplevel may be enabled for traversal, including nested notebooks. However, notebook traversal only works properly if all panes are direct children of the notebook.

Progressbar

The `ttk.Progressbar` widget shows the status of a long-running operation. It can operate in two modes: 1) the determinate mode which shows the amount completed relative to the total amount of work to be done and 2) the indeterminate mode which provides an animated display to let the user know that work is progressing.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
orient	One of “horizontal” or “vertical”. Specifies the orientation of the progress bar.

Opção	Descrição
length	Specifies the length of the long axis of the progress bar (width if horizontal, height if vertical).
mode	One of “determinate” or “indeterminate”.
maximum	A number specifying the maximum value. Defaults to 100.
value	The current value of the progress bar. In “determinate” mode, this represents the amount of work completed. In “indeterminate” mode, it is interpreted as modulo *maximum*; that is, the progress bar completes one “cycle” when its value increases by *maximum*.
variable	A name which is linked to the option value. If specified, the value of the progress bar is automatically set to the value of this name whenever the latter is modified.
phase	Read-only option. The widget periodically increments the value of this option whenever its value is greater than 0 and, in determinate mode, less than maximum. This option may be used by the current theme to provide additional animation effects.

ttk.Progressbar

class tkinter.ttk.Progressbar

start(*interval=None*)

Begin autoincrement mode: schedules a recurring timer event that calls `Progressbar.step()` every *interval* milliseconds. If omitted, *interval* defaults to 50 milliseconds.

step(*amount=None*)

Increments the progress bar's value by *amount*.

amount defaults to 1.0 if omitted.

stop()

Stop autoincrement mode: cancels any recurring timer event initiated by `Progressbar.start()` for this progress bar.

Separator

The `ttk.Separator` widget displays a horizontal or vertical separator bar.

It has no other methods besides the ones inherited from `ttk.Widget`.

Opções

This widget accepts the following specific option:

Opção	Descrição
orient	One of "horizontal" or "vertical". Specifies the orientation of the separator.

Sizegrip

The `ttk.Sizegrip` widget (also known as a grow box) allows the user to resize the containing toplevel window by pressing and dragging the grip.

This widget has neither specific options nor specific methods, besides the ones inherited from `ttk.Widget`.

Platform-specific notes

- On macOS, toplevel windows automatically include a built-in size grip by default. Adding a `Sizegrip` is harmless, since the built-in grip will just mask the widget.

Bugs

- If the containing toplevel's position was specified relative to the right or bottom of the screen (for example, ….), the `Sizegrip` widget will not resize the window.
- This widget supports only "southeast" resizing.

Treeview

The `ttk.Treeview` widget displays a hierarchical collection of items. Each item has a textual label, an optional image, and an optional list of data values. The data values are displayed in successive columns after the tree label.

The order in which data values are displayed may be controlled by setting the widget option `displaycolumns`. The tree widget can also display column headings. Columns may be accessed by number or symbolic names listed in the widget option columns. See Column Identifiers.

Each item is identified by a unique name. The widget will generate item IDs if they are not supplied by the caller. There is a distinguished root item, named `{}`. The root item itself is not displayed; its children appear at the top level of the hierarchy.

Each item also has a list of tags, which can be used to associate event bindings with individual items and control the appearance of the item.

The Treeview widget supports horizontal and vertical scrolling, according to the options described in Scrollable Widget Options and the methods `Treeview.xview()` and `Treeview.yview()`.

Opções

Este widget aceita as seguintes opções específicas:

Opção	Descrição
columns	A list of column identifiers, specifying the number of columns and their names.
displaycolumns	A list of column identifiers (either symbolic or integer indices) specifying which data columns are displayed and the order in which they appear, or the string “#all”.
height	Specifies the number of rows which should be visible. Note: the requested width is determined from the sum of the column widths.
padding	Specifies the internal padding for the widget. The padding is a list of up to four length specifications.

Opção	Descrição
selectmode	Controls how the built-in class bindings manage the selection. One of “extended”, “browse” or “none”. If set to “extended” (the default), multiple items may be selected. If “browse”, only a single item will be selected at a time. If “none”, the selection will not be changed. Note that the application code and tag bindings can set the selection however they wish, regardless of the value of this option.
show	A list containing zero or more of the following values, specifying which elements of the tree to display. • tree: display tree labels in column #0. • headings: display the heading row. The default is “tree headings”, that is, show all elements. **Note**: Column #0 always refers to the tree column, even if show=”tree” is not specified.

Nota

Tk 9.0 added several `Treeview` features. The *selectmode* option gained the values `"single"` and `"multiple"`; the new widget options *selecttype* (`"item"` or `"cell"` selection), *striped* (zebra-striped rows), and *titlecolumns* / *titleitems* (columns or rows frozen against scrolling) were introduced; the column *separator* option was added; and items gained a *hidden* option. Tk 9.1 added the *rowheight* and *headingheight* options.

Item options

The following item options may be specified for items in the insert and item widget commands.

Opção	Descrição
text	The textual label to display for the item.
image	A Tk Image, displayed to the left of the label.
values	The list of values associated with the item. Each item should have the same number of values as the widget option columns. If there are fewer values than columns, the remaining values are assumed empty. If there are more values than columns, the extra values are ignored.
open	`True`/`False` value indicating whether the item's children should be displayed or hidden.
tags	A list of tags associated with this item.

Tag options

The following options may be specified on tags:

Opção	Descrição
foreground	Specifies the text foreground color.

Opção	Descrição
background	Specifies the cell or item background color.
font	Specifies the font to use when drawing text.
image	Specifies the item image, in case the item’s image option is empty.

Column identifiers

Column identifiers take any of the following forms:

- A symbolic name from the list of columns option.
- An integer n, specifying the nth data column.
- A string of the form #n, where n is an integer, specifying the nth display column.

Notas:

- Item’s option values may be displayed in a different order than the order in which they are stored.
- Column #0 always refers to the tree column, even if show=”tree” is not specified.

A data column number is an index into an item’s option values list; a display column number is the column number in the tree where the values are displayed. Tree labels are displayed in column #0. If option displaycolumns is not set, then data column n is displayed in column #n+1. Again, **column #0 always refers to the tree column**.

Eventos virtuais

The Treeview widget generates the following virtual events.

Evento	Descrição
<<TreeviewSelect>>	Generated whenever the selection changes.
<<TreeviewOpen>>	Generated just before settings the focus item to open=True.
<<TreeviewClose>>	Generated just after setting the focus item to open=False.

The `Treeview.focus()` and `Treeview.selection()` methods can be used to determine the affected item or items.

ttk.Treeview

class tkinter.ttk.Treeview

bbox(*item*, *column=None*)

Returns the bounding box (relative to the treeview widget's window) of the specified *item* in the form (x, y, width, height).

If *column* is specified, returns the bounding box of that cell. If the *item* is not visible (that is, if it is a descendant of a closed item or is scrolled offscreen), returns an empty string.

This shadows the inherited `Misc.bbox()`; use `grid bbox()` for the grid bounding box.

get_children(*item=None*)

Returns a tuple of children belonging to *item*.

If *item* is not specified, returns root children.

set_children(*item*, **newchildren*)

Replaces *item*'s children with *newchildren*.

Children present in *item* that are not present in *newchildren* are detached from the tree. No items in *newchildren* may be an ancestor of *item*. Note that not specifying *newchildren* results in detaching *item*'s children.

column(*column*, *option=None*, ***kw*)

Query or modify the options for the specified *column*.

If *kw* is not given, returns a dict of the column option values. If *option* is specified then the value for that *option* is returned. Otherwise, sets the options to the corresponding values.

The valid options/values are:

id

Returns the column name. This is a read-only option.

anchor: One of the standard Tk anchor values.

Specifies how the text in this column should be aligned with respect to the cell.

minwidth: width

The minimum width of the column in pixels. The treeview widget will not make the column any smaller than specified by this option when the widget is resized or the user drags a column.

separator: `True`/`False`

Specifies whether a column separator should be drawn to the right of the column.

stretch: `True`/`False`

Specifies whether the column's width should be adjusted when the widget is resized.

width: width
The width of the column in pixels.

To configure the tree column, call this with column = “#0”

delete(**items*)
Delete all specified *items* and all their descendants.

The root item may not be deleted.

detach(**items*)
Unlinks all of the specified *items* from the tree.

The items and all of their descendants are still present, and may be reinserted at another point in the tree, but will not be displayed.

The root item may not be detached.

exists(*item*)
Returns `True` if the specified *item* is present in the tree, `False` otherwise.

focus(*item=None*)
If *item* is specified, sets the focus item to *item*. Otherwise, returns the current focus item, or ‘’ if there is none.

This shadows the inherited **`Misc.focus()`**; use **`focus_set()`** to focus the widget itself.

heading(*column*, *option=None*, ***kw*)
Query or modify the heading options for the specified *column*.

If *kw* is not given, returns a dict of the heading option values. If *option* is specified then the value for that *option* is returned. Otherwise, sets the options to the corresponding values.

The valid options/values are:

text: text
The text to display in the column heading.

image: imageName
Specifies an image to display to the right of the column heading.

anchor: anchor
Specifies how the heading text should be aligned. One of the standard Tk anchor values.

command: callback
A callback to be invoked when the heading label is pressed.

To configure the tree column heading, call this with column = “#0”.

identify(*component*, *x*, *y*)
Returns a description of the specified *component* under the point given by *x* and *y*, or the empty string if no such *component* is present at that position.

identify_row(*y*)
Returns the item ID of the item at position *y*.

identify_column(*x*)
Returns the display column identifier of the cell at position *x*.

The tree column has ID #0.

identify_region(*x*, *y*)
Returns one of:

region	meaning
heading	Tree heading area.
separator	Space between two columns headings.

region	meaning
tree	The tree area.
cell	A data cell.

Availability: Tk 8.6.

identify_element(*x*, *y*)

Returns the element at position *x*, *y*.

Availability: Tk 8.6.

index(*item*)

Returns the integer index of *item* within its parent's list of children.

insert(*parent*, *index*, *iid=None*, ***kw*)

Creates a new item and returns the item identifier of the newly created item.

parent is the item ID of the parent item, or the empty string to create a new top-level item. *index* is an integer, or the value "end", specifying where in the list of parent's children to insert the new item. If *index* is less than or equal to zero, the new node is inserted at the beginning; if *index* is greater than or equal to the current number of children, it is inserted at the end. If *iid* is specified, it is used as the item identifier; *iid* must not already exist in the tree. Otherwise, a new unique identifier is generated.

See Item Options for the list of available options.

item(*item*, *option=None*, ***kw*)

Query or modify the options for the specified *item*.

If no options are given, a dict with options/values for the item is returned. If *option* is specified then the value for that option is

returned. Otherwise, sets the options to the corresponding values as given by *kw*.

move(*item*, *parent*, *index*)

Moves *item* to position *index* in *parent*'s list of children.

It is illegal to move an item under one of its descendants. If *index* is less than or equal to zero, *item* is moved to the beginning; if greater than or equal to the number of children, it is moved to the end. If *item* was detached it is reattached.

`reattach()` is an alias of **`move()`**.

next(*item*)

Returns the identifier of *item*'s next sibling, or '' if *item* is the last child of its parent.

parent(*item*)

Returns the ID of the parent of *item*, or '' if *item* is at the top level of the hierarchy.

prev(*item*)

Returns the identifier of *item*'s previous sibling, or '' if *item* is the first child of its parent.

see(*item*)

Ensure that *item* is visible.

Sets all of *item*'s ancestors open option to `True`, and scrolls the widget if necessary so that *item* is within the visible portion of the tree.

selection()

Returns a tuple of selected items.

Alterado na versão 3.8: `selection()` no longer takes arguments. For changing the selection state use the following selection

methods.

selection_set(**items*)

items becomes the new selection.

Alterado na versão 3.6: items can be passed as separate arguments, not just as a single tuple.

selection_add(**items*)

Add *items* to the selection.

Alterado na versão 3.6: items can be passed as separate arguments, not just as a single tuple.

selection_remove(**items*)

Remove *items* from the selection.

Alterado na versão 3.6: items can be passed as separate arguments, not just as a single tuple.

selection_toggle(**items*)

Toggle the selection state of each item in *items*.

Alterado na versão 3.6: items can be passed as separate arguments, not just as a single tuple.

set(*item*, *column=None*, *value=None*)

With one argument, returns a dictionary of column/value pairs for the specified *item*. With two arguments, returns the current value of the specified *column*. With three arguments, sets the value of given *column* in given *item* to the specified *value*.

tag_bind(*tagname*, *sequence=None*, *callback=None*)

Bind a callback for the given event *sequence* to the tag *tagname*. When an event is delivered to an item, the callbacks for each of the item's tags option are called.

tag_configure(*tagname*, *option=None*, ***kw*)

Query or modify the options for the specified *tagname*.

If *kw* is not given, returns a dict of the option settings for *tagname*. If *option* is specified, returns the value for that *option* for the specified *tagname*. Otherwise, sets the options to the corresponding values for the given *tagname*.

tag_has(*tagname*, *item=None*)

If *item* is specified, returns `True` if the specified *item* has the given *tagname* and `False` otherwise. Otherwise, returns a tuple of all items that have the specified tag.

Availability: Tk 8.6

xview(**args*)

Query or modify horizontal position of the treeview.

yview(**args*)

Query or modify vertical position of the treeview.

Ttk styling

Each widget in **`ttk`** is assigned a style, which specifies the set of elements making up the widget and how they are arranged, along with dynamic and default settings for element options. By default the style name is the same as the widget's class name, but it may be overridden by the widget's style option. If you don't know the class name of a widget, use the method `Misc.winfo_class` (somewidget.winfo_class()).

Ver também

Introduction to the Tk theme engine [https://www.tcl-lang.org/man/tcl9.0/TkCmd/ttk_intro.html]

The `ttk::intro` man page explains how the theme engine works.

The Tile Widget Set **[https://tktable.sourceforge.net/tile/tile-tcl2004.pdf]**

Joe English's 2004 paper introducing the theme engine (then the separate *Tile* extension), with diagrams of how elements and layouts make up a widget's appearance.

class tkinter.ttk.Style

This class is used to manipulate the style database.

configure(*style*, *query_opt=None*, ***kw*)

Query or set the default value of the specified option(s) in *style*.

Each key in *kw* is an option and each value is a string identifying the value for that option.

For example, to change every default button to be a flat button with some padding and a different background color:

```
from tkinter import ttk
import tkinter

root = tkinter.Tk()

ttk.Style().configure("TButton", padding=6, relief="flat",
   background="#ccc")

btn = ttk.Button(text="Sample")
btn.pack()

root.mainloop()
```

map(*style*, *query_opt=None*, ***kw*)

Query or sets dynamic values of the specified option(s) in *style*.

Each key in *kw* is an option and each value should be a list or a tuple (usually) containing statespecs grouped in tuples, lists, or some other preference. A statespec is a compound of one or more states and then a value.

An example may make it more understandable:

```
import tkinter
from tkinter import ttk

root = tkinter.Tk()

style = ttk.Style()
style.map("C.TButton",
    foreground=[('pressed', 'red'), ('active', 'blue')],
    background=[('pressed', '!disabled', 'black'), ('acti
    )

colored_btn = ttk.Button(text="Test", style="C.TButton").

root.mainloop()
```

Note that the order of the (states, value) sequences for an option does matter, if the order is changed to `[('active', 'blue'), ('pressed', 'red')]` in the foreground option, for example, the result would be a blue foreground when the widget were in active or pressed states.

When called to query the map (without specifying values to set), it returns a dictionary mapping each option to its list of statespecs.

Alterado na versão 3.10: The value returned when querying the map was corrected.

lookup(*style*, *option*, *state=None*, *default=None*)

Returns the value specified for *option* in *style*.

If *state* is specified, it is expected to be a sequence of one or more states. If the *default* argument is set, it is used as a fallback value in case no specification for option is found.

To check what font a Button uses by default:

```
from tkinter import ttk

print(ttk.Style().lookup("TButton", "font"))
```

layout(*style*, *layoutspec=None*)

Define the widget layout for given *style*. If *layoutspec* is omitted, return the layout specification for given style.

layoutspec, if specified, is expected to be a list or some other sequence type (excluding strings), where each item should be a tuple and the first item is the layout name and the second item should have the format described in Layouts.

To understand the format, see the following example (it is not intended to do anything useful):

```
from tkinter import ttk
import tkinter

root = tkinter.Tk()

style = ttk.Style()
style.layout("TMenubutton", [
   ("Menubutton.background", None),
   ("Menubutton.button", {"children":
       [("Menubutton.focus", {"children":
           [("Menubutton.padding", {"children":
               [("Menubutton.label", {"side": "left", "exp
           })]
       })]
   }),
])

mbtn = ttk.Menubutton(text='Text')
mbtn.pack()
root.mainloop()
```

element_create(*elementname*, *etype*, **args*, ***kw*)

Create a new element in the current theme, of the given *etype* which is expected to be either “image”, “from” or “vsapi”. The latter is

only available in Tk 8.6 on Windows.

If “image” is used, *args* should contain the default image name followed by statespec/value pairs (this is the imagespec), and *kw* may have the following options:

border=padding
: padding is a list of up to four integers, specifying the left, top, right, and bottom borders, respectively.

height=height
: Specifies a minimum height for the element. If less than zero, the base image’s height is used as a default.

padding=padding
: Specifies the element’s interior padding. Defaults to border’s value if not specified.

sticky=spec
: Specifies how the image is placed within the final parcel. spec contains zero or more characters “n”, “s”, “w”, or “e”.

width=width
: Specifies a minimum width for the element. If less than zero, the base image’s width is used as a default.

Exemplo:

```
img1 = tkinter.PhotoImage(master=root, file='button.png')
img1 = tkinter.PhotoImage(master=root, file='button-presse
img1 = tkinter.PhotoImage(master=root, file='button-active
style = ttk.Style(root)
style.element_create('Button.button', 'image',
                     img1, ('pressed', img2), ('active', 
                     border=(2, 4), sticky='we')
```

If “from” is used as the value of *etype*, **`element_create()`** will clone an existing element. *args* is expected to contain a themename, from which the element will be cloned, and optionally an element to

clone from. If this element to clone from is not specified, an empty element will be used. *kw* is discarded.

Exemplo:

```
style = ttk.Style(root)
style.element_create('plain.background', 'from', 'default
```

If “vsapi” is used as the value of *etype*, **`element_create()`** will create a new element in the current theme whose visual appearance is drawn using the Microsoft Visual Styles API which is responsible for the themed styles on Windows XP and Vista. *args* is expected to contain the Visual Styles class and part as given in the Microsoft documentation followed by an optional sequence of tuples of ttk states and the corresponding Visual Styles API state value. *kw* may have the following options:

padding=padding
: Specify the element’s interior padding. *padding* is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. If fewer than four elements are specified, bottom defaults to top, right defaults to left, and top defaults to left. In other words, a list of three numbers specify the left, vertical, and right padding; a list of two numbers specify the horizontal and the vertical padding; a single number specifies the same padding all the way around the widget. This option may not be mixed with any other options.

margins=padding
: Specifies the elements exterior padding. *padding* is a list of up to four integers specifying the left, top, right and bottom padding quantities respectively. This option may not be mixed with any other options.

width=width
: Specifies the width for the element. If this option is set then the Visual Styles API will not be queried for the recommended size or the part. If this option is set then *height* should also be set.

The *width* and *height* options cannot be mixed with the *padding* or *margins* options.

height=height

Specifies the height of the element. See the comments for *width*.

Exemplo:

```
style = ttk.Style(root)
style.element_create('pin', 'vsapi', 'EXPLORERBAR', 3, [
                     ('pressed', '!selected', 3),
                     ('active', '!selected', 2),
                     ('pressed', 'selected', 6),
                     ('active', 'selected', 5),
                     ('selected', 4),
                     ('', 1)])
style.layout('Explorer.Pin',
             [('Explorer.Pin.pin', {'sticky': 'news'})])
pin = ttk.Checkbutton(style='Explorer.Pin')
pin.pack(expand=True, fill='both')
```

Alterado na versão 3.13: Added support of the “vsapi” element factory.

element_names()

Returns a tuple of elements defined in the current theme.

element_options(*elementname*)

Returns a tuple of *elementname*’s options.

theme_create(*themename*, *parent=None*, *settings=None*)

Create a new theme.

It is an error if *themename* already exists. If *parent* is specified, the new theme will inherit styles, elements and layouts from the parent theme. If *settings* are present they are expected to have the same syntax used for `theme_settings()`.

theme_settings(*themename*, *settings*)

Temporarily sets the current theme to *themename*, apply specified *settings* and then restore the previous theme.

Each key in *settings* is a style and each value may contain the keys 'configure', 'map', 'layout' and 'element create' and they are expected to have the same format as specified by the methods `Style.configure()`, `Style.map()`, `Style.layout()` and `Style.element_create()` respectively.

As an example, let's change the Combobox for the default theme a bit:

```
from tkinter import ttk
import tkinter

root = tkinter.Tk()

style = ttk.Style()
style.theme_settings("default", {
   "TCombobox": {
       "configure": {"padding": 5},
       "map": {
           "background": [("active", "green2"),
                          ("!disabled", "green4")],
           "fieldbackground": [("!disabled", "green3")],
           "foreground": [("focus", "OliveDrab1"),
                          ("!disabled", "OliveDrab2")]
       }
   }
})

combo = ttk.Combobox().pack()

root.mainloop()
```

theme_names()

Returns a tuple of all known themes.

theme_use(*themename=None*)

If *themename* is not given, returns the theme in use. Otherwise, sets the current theme to *themename*, refreshes all widgets and emits a <<ThemeChanged>> event.

Layouts

A layout can be just `None`, if it takes no options, or a dict of options specifying how to arrange the element. The layout mechanism uses a simplified version of the pack geometry manager: given an initial cavity, each element is allocated a parcel.

The valid options/values are:

side: whichside
Specifies which side of the cavity to place the element; one of top, right, bottom or left. If omitted, the element occupies the entire cavity.

sticky: nswe
Specifies where the element is placed inside its allocated parcel.

unit: 0 ou 1
If set to 1, causes the element and all of its descendants to be treated as a single element for the purposes of `Widget.identify()` et al. It's used for things like scrollbar thumbs with grips.

children: [sublayout…]
Specifies a list of elements to place inside the element. Each element is a tuple (or other sequence type) where the first item is the layout name, and the other is a Layout.

Additional widgets

The following themed widgets complete the `tkinter.ttk` widget set. Each is the themed counterpart of the like-named classic `tkinter` widget and inherits the common methods of `Widget`.

class tkinter.ttk.Button(*master=None*, ***kw*)

Ttk **`Button`** widget, displays a textual label and/or image, and evaluates a command when pressed. It is the themed counterpart of **`tkinter.Button`** and inherits the common widget methods from **`Widget`**.

invoke()

Invoke the command associated with the button and return its result.

class tkinter.ttk.Checkbutton(*master=None*, ***kw*)

Ttk **`Checkbutton`** widget, used to control a boolean variable that is toggled on and off. It is the themed counterpart of **`tkinter.Checkbutton`** and inherits the common widget methods from **`Widget`**.

invoke()

Toggle the button between its selected and deselected states, invoke the command associated with the button, and return its result.

class tkinter.ttk.Entry(*master=None*, *widget=None*, ***kw*)

Ttk **`Entry`** widget, displays a one-line text string and allows the user to edit it. It is the themed counterpart of **`tkinter.Entry`** and inherits the common widget methods from **`Widget`** as well as the editing methods from **`tkinter.Entry`**.

bbox(*index*)

Return a tuple `(x, y, width, height)` giving the bounding box of the character at the given *index*.

This shadows the inherited **`Misc.bbox()`**; use **`grid bbox()`** for the grid bounding box.

identify(*x*, *y*)

Return the name of the element under the point given by *x* and *y*, or the empty string if no element is present at that location.

validate()

Force validation of the entry and return `True` if validation succeeded, and `False` otherwise.

class tkinter.ttk.Frame(*master=None*, ***kw*)

Ttk **`Frame`** widget, a container used to group and lay out other widgets. It is the themed counterpart of **`tkinter.Frame`** and inherits the common widget methods from **`Widget`**.

class tkinter.ttk.Label(*master=None*, ***kw*)

Ttk **`Label`** widget, displays a textual label and/or image. It is the themed counterpart of **`tkinter.Label`** and inherits the common widget methods from **`Widget`**.

class tkinter.ttk.Labelframe(*master=None*, ***kw*)

Ttk **`Labelframe`** widget, a container that draws a border and a title label around its contents. It is the themed counterpart of **`tkinter.LabelFrame`** and inherits the common widget methods from **`Widget`**.

class tkinter.ttk.Menubutton(*master=None*, ***kw*)

Ttk **`Menubutton`** widget, displays a textual label and/or image, and pops up a menu when pressed. It is the themed counterpart of **`tkinter.Menubutton`** and inherits the common widget methods from **`Widget`**.

class tkinter.ttk.OptionMenu(*master*, *variable*, *default=None*, **values*, ***kwargs*)

Ttk **`OptionMenu`** widget, a **`Menubutton`** that pops up a menu of mutually exclusive choices. *variable* is the variable that tracks the currently selected value, *default* is the value to set initially, and *values* are the entries to display in the menu. A *command* keyword argument may be given to specify a callable that is invoked with the selected value whenever the selection changes; the *style* keyword argument sets the style used by the underlying menubutton; the *direction* keyword argument sets where the menu is posted relative to the menubutton (one of `'above'`, `'below'` (the default), `'left'`, `'right'` or `'flush'`); and the *name* keyword argument sets the Tk widget name.

set_menu(*default=None, *values*)

Replace the entries of the menu with *values*. If *default* is given, also set it as the current value of the *variable*.

destroy()

Destroy this widget and its associated menu.

Alterado na versão 3.14: Added support for the *name* keyword argument.

class tkinter.ttk.Panedwindow(*master=None, **kw*)

Ttk **`Panedwindow`** widget, displays a number of subwindows stacked either vertically or horizontally. The user may adjust the relative sizes of the subwindows by dragging the sash between panes. It is the themed counterpart of **`tkinter.PanedWindow`** and inherits the common widget methods from **`Widget`**, as well as the **`add()`** and **`panes()`** methods from **`tkinter.PanedWindow`**.

insert(*pos, child, **kw*)

Insert a pane containing *child* at the position *pos*. *pos* is either the string `'end'`, an integer index, or the name of a managed subwindow. If *child* is already managed by the paned window, move it to the specified position. Any keyword arguments set pane options.

forget(*child*)

Remove *child*, which may be either an integer index or the name of a managed subwindow, from the panes.

This shadows the inherited geometry-manager **`forget()`**; use **`pack forget()`**, **`grid forget()`** or **`place forget()`** to remove the widget itself from its manager.

pane(*pane, option=None, **kw*)

Query or modify the options of the specified *pane*, where *pane* is either an integer index or the name of a managed subwindow. If no

arguments are given, return a dictionary of the pane option values. If *option* is specified, return the value of that option. Otherwise, set the options given as keyword arguments to their corresponding values.

sashpos(*index*, *newpos=None*)

If *newpos* is specified, set the position of sash number *index* and return its new position. This may adjust the positions of adjacent sashes to ensure that positions are monotonically increasing; positions are also constrained to be between 0 and the total size of the widget. If *newpos* is omitted, return the current position of the sash.

class tkinter.ttk.Radiobutton(*master=None*, ***kw*)

Ttk **`Radiobutton`** widget, used as part of a group to control a single shared variable by selecting one of several mutually exclusive values. It is the themed counterpart of **`tkinter.Radiobutton`** and inherits the common widget methods from **`Widget`**.

invoke()

Set the option variable to the button's value, select the button, invoke the command associated with the button, and return its result.

class tkinter.ttk.Scale(*master=None*, ***kw*)

Ttk **`Scale`** widget, displays a slider that lets the user select a numeric value from a range by moving the slider along a trough. It is the themed counterpart of **`tkinter.Scale`** and inherits the common widget methods from **`Widget`**.

configure(*cnf=None*, ***kw*)

Modify or query the widget options, like **`Widget.configure`**. In addition, this method clips the `from` and `to` values so that the current value stays within the range defined by them.

Alterado na versão 3.9: Now returns the configuration value, like **`Widget.configure`**.

get(*x=None*, *y=None*)

Return the current value of the scale. If *x* and *y* are given, return the value corresponding to the pixel coordinate *x*, *y* instead.

class tkinter.ttk.Scrollbar(*master=None*, ***kw*)

Ttk **`Scrollbar`** widget, controls the viewport of an associated scrollable widget such as a **`Treeview`**, **`Entry`** or **`tkinter.Text`**. It is the themed counterpart of **`tkinter.Scrollbar`** and inherits the common widget methods from **`Widget`**, as well as the **`set()`** and **`get()`** methods from **`tkinter.Scrollbar`**.

class tkinter.ttk.Separator(*master=None*, ***kw*)

Ttk **`Separator`** widget, displays a horizontal or vertical separator line. It has no direct counterpart in **`tkinter`** and inherits the common widget methods from **`Widget`**.

class tkinter.ttk.Sizegrip(*master=None*, ***kw*)

Ttk **`Sizegrip`** widget, displays a grip that allows the user to resize the containing toplevel window by pressing and dragging the grip, typically placed in the bottom-right corner. It has no direct counterpart in **`tkinter`** and inherits the common widget methods from **`Widget`**.

class tkinter.ttk.LabeledScale(*master=None*, *variable=None*, *from_=0*, *to=10*, ***kw*)

A **`Frame`** containing a **`Scale`** and a **`Label`** that shows the scale's current value. *variable* is the **`IntVar`** tracked by the scale (one is created if it is not given), and *from_* and *to* define the range of the scale.

destroy()

Destroy this widget and remove the trace callback registered on the associated variable.

class tkinter.ttk.LabelFrame(*master=None*, ***kw*)

Alias of **`Labelframe`**, kept for naming compatibility with **`tkinter.LabelFrame`**.

class tkinter.ttk.PanedWindow(*master=None*, ***kw*)

Alias of `Panedwindow`, kept for naming compatibility with `tkinter.PanedWindow`.

IDLE — Editor e console Python

Código-fonte: Lib/idlelib/ [https://github.com/python/cpython/tree/3.15/Lib/idlelib/]

IDLE é o Ambiente Integrado de Desenvolvimento e Aprendizagem (em inglês, Integrated Development and Learning Environment) do Python.

O IDLE tem os seguintes recursos:

- multiplataforma: funciona basicamente da mesma forma no Windows, Unix e macOS
- janela de console do Python (interpretador interativo) com colorização de entrada de código, saída e mensagens de erro
- editor de texto multijanela com múltiplos desfazer, colorização Python, indentação inteligente, dicas de chamada, preenchimento automático e outros recursos
- pesquisa em qualquer janela, substituição nas janelas do editor e pesquisa em vários arquivos (grep)
- depurador com pontos de interrupção persistentes, etapas e visualização de espaço de nomes global e local
- configuração, navegadores e outros diálogos

A aplicação IDLE está implementada no pacote `idlelib`.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Menus

O IDLE possui dois tipos principais de janelas: a janela do console e a janela do editor. É possível ter várias janelas do editor abertas simultaneamente. No Windows e no Linux, cada uma tem seu próprio menu

superior. Cada menu documentado abaixo indica a qual tipo de janela está associado.

Janelas de saída, como as usadas para Edit => Find in Files, são um subtipo de janela de editor. Atualmente, têm o mesmo menu superior, mas um título predefinido e menu de contexto diferentes.

No macOS, há um menu de aplicações. Ele muda dinamicamente conforme a janela selecionada no momento. Ele possui um menu de IDLE e algumas entradas descritas abaixo foram movidas para atender às diretrizes da Apple.

Menu File (Shell e Editor)

New File
: Cria uma nova janela de edição de arquivo.

Open…
: Abre um arquivo existente com uma caixa de diálogo Open.

Open Module…
: Abre um módulo existente (procura em sys.path).

Recent Files
: Abre uma lista de arquivos recentes. Clique em um para abri-lo.

Module Browser
: Mostra as funções, classes e métodos do arquivo atual do editor em uma estrutura de árvore. No console, abre primeiro um módulo.

Path Browser
: Mostra os diretórios do sys.path, módulos, funções, classes e métodos em uma estrutura de árvore.

Save
: Salva o conteúdo da janela atual no arquivo associado, se houver um. Janelas que foram alteradas desde que foram abertas ou salvas pela

última vez têm um * antes e depois do título da janela. Se não houver um arquivo associado, executa Save As.

Save As…
: Salva a janela atual com uma caixa de diálogo Save As. O arquivo salvo torna-se o novo arquivo associado à janela. (Se o seu explorador de arquivos estiver configurado para ocultar extensões, a extensão atual será omitida na caixa de nome do arquivo. Se o novo nome do arquivo não tiver '.', '.py' e '.txt' serão adicionados a arquivos Python e de texto, exceto no macOS Aqua, onde '.py' é adicionado a todos os arquivos.)

Save Copy As…
: Salva a janela atual em um arquivo diferente sem alterar o arquivo associado. (veja Save As acima sobre a extensão do nome do arquivo.)

Print Window
: Imprime a janela atual para a impressora padrão.

Close Window
: Fecha a janela atual (se for um editor não salvo, pergunta se quer salvar; se for um console não salvo, pergunta se quer sair da execução). Chamar `exit()` ou `close()` na janela do console também fecha o console. Se esta for a única janela, também sai do IDLE.

Exit IDLE
: Fecha todas as janelas e sai do IDLE (pergunta se quer salvar as janelas de edição não salvas).

Menu Edit (Shell e Editor)

Undo
: Desfaz a última alteração na janela atual. Um máximo de 1000 alterações pode ser desfeito.

Redo
: Refaz a última alteração desfeita na janela atual.

Select All
: Seleciona todo o conteúdo da janela atual.

Cut
: Copia a seleção para a área de transferência do sistema; depois apaga a seleção.

Copy
: Copia a seleção para a área de transferência do sistema.

Paste
: Insere o conteúdo da área de transferência do sistema na janela atual.

As funções da área de transferência também estão disponíveis nos menus de contexto.

Find…
: Abre uma caixa de diálogo de pesquisa com muitas opções

Find Again
: Repete a última pesquisa, se houver uma.

Find Selection
: Procura a string atualmente selecionada, se houver uma.

Find in Files…
: Abre uma caixa de diálogo de pesquisa de ficheiros. Coloca os resultados numa nova janela de saída.

Replace…
: Abre uma caixa de diálogo de pesquisa e substituição.

Go to Line
: Move o cursor para o início da linha solicitada e torna essa linha visível. Um pedido após o final do ficheiro vai para o final. Limpa qualquer seleção e atualiza o estado da linha e da coluna.

Show Completions

Open a scrollable list allowing selection of existing names. See Completions in the Editing and Navigation section below.

Expand Word
: Expande um prefixo que você digitou para corresponder a uma palavra completa na mesma janela; repita o processo para obter uma expansão diferente.

Show Call Tip
: After an unclosed parenthesis for a function, open a small window with function parameter hints. See Calltips in the Editing and Navigation section below.

Show Surrounding Parens
: Realça os parênteses envolventes.

Menu Format (somente na janela do Editor)

Format Paragraph
: Rewrap the text block containing the text insert cursor. Avoid code lines. See Format block in the Editing and Navigation section below.

Indent Region
: Desloca as linhas selecionadas à direita pela largura de indentação (4 espaços por padrão).

Dedent Region
: Desloca as linhas selecionadas à esquerda pela largura de indentação (4 espaços por padrão).

Comment Out Region
: Insere ## à frente das linhas selecionadas.

Uncomment Region
: Remove # ou ## iniciais das linhas selecionadas.

Tabify Region

Converte sequências iniciais de espaços em tabs. (Nota: recomendamos usar blocos de 4 espaços para indentar código Python.)

Untabify Region
: Converte *todos* os tabs para o número correto de espaços.

Toggle Tabs
: Abre uma caixa de diálogo para alternar entre espaços e tabs para indentar.

New Indent Width
: Abre uma caixa de diálogo para alterar a largura de indentação. A largura predefinida aceita pela comunidade Python é de 4 espaços.

Strip Trailing Whitespace
: Remove espaços e outros caracteres de espaço em branco após o último caractere não branco de uma linha aplicando `str.rstrip()` a cada linha, incluindo linhas dentro de strings multilinhas. Exceto para as janelas Shell, remove as novas linhas extras no final do arquivo.

Menu Run (somente na janela do Editor)

Run Module
: Executa Check Module. Se não houver erros, reinicia o console para limpar o ambiente, depois executa o módulo. A saída é exibida na janela Shell. Note que a saída requer o uso de `print` ou `write`. Quando a execução estiver completa, a Shell mantém o foco e exibe um prompt. Neste ponto, pode-se explorar interativamente o resultado da execução. Isto é semelhante a executar um ficheiro com `python -i file` na linha de comandos.

Run… Customized
: O mesmo que a Run Module, mas executa o módulo com configurações personalizadas. *Argumentos de Linha de Comandos* estendem `sys.argv` como se passados na linha de comandos. O módulo pode ser executado na Shell sem reiniciar.

Check Module

Verifica a sintaxe do módulo atualmente aberto na janela do Editor. Se o módulo não tiver sido salvo, o IDLE perguntará se quer salvar ou salvará automaticamente, conforme configurado na aba General da caixa de diálogo Settings. Se houver um erro de sintaxe, a localização aproximada é indicada na janela do Editor.

Python Shell
: Abre ou ativa a janela do Shell Python.

Menu Shell (somente na janela do Shell)

View Last Restart
: Desloca a janela para o último reinício do Shell.

Restart Shell
: Reinicia o shell para limpar o ambiente e redefinir a apresentação e o tratamento de exceções.

Previous History
: Percorre os comandos anteriores no histórico que correspondam à entrada atual.

Next History
: Percorre os comandos posteriores no histórico que correspondam à entrada atual.

Interrupt Execution
: Interrompe um programa em execução.

Menu Debug (somente na janela do Shell)

Go to File/Line
: Procura na linha atual com o cursor e na linha acima por um nome de arquivo e um número de linha. Se encontrado, abre o arquivo se não estiver já aberto e mostra a linha. Use isto para ver as linhas de origem referenciadas num traceback de exceção e as linhas encontradas por

Find in Files. Também está disponível no menu de contexto da janela Shell e das janelas de Saída.

Debugger (alternar)
: Quando ativado, o código introduzido na Shell ou executado a partir de um Editor será executado sob o depurador. No Editor, é possível definir pontos de interrupção pelo menu de contexto. Esta funcionalidade ainda está incompleta e é experimental.

Stack Viewer
: Mostra o traceback da pilha da última exceção em formato de árvore, incluindo acesso a locais e globais.

Auto-open Stack Viewer
: Alterna a abertura automática do visualizador de pilha para uma exceção não tratada.

Menu Options (Shell e Editor)

Configure IDLE
: Abre uma caixa de diálogo de configuração e alteração de preferências, tais como: tipos de letra, indentação, atalhos de teclado, temas de cor de texto, janelas de inicialização e tamanho, fontes de ajuda adicionais e extensões. No macOS, abre a caixa de diálogo de configuração selecionando Preferences no menu da aplicação. Para mais detalhes, veja Configurando preferências em Ajuda e preferências.

A maioria das opções de configuração aplica-se a todas as janelas ou a todas as janelas futuras. Os itens de opção abaixo aplicam-se apenas à janela ativa.

Show/Hide Code Context (somente janela do Editor)
: Abre um painel no topo da janela de edição que mostra o contexto de bloco de código exibido no topo da janela. Veja Contexto de código na seção de Edição e navegação abaixo.

Show/Hide Line Numbers (somente janela do Editor)

Abre uma coluna à esquerda da janela de edição que mostra o número de cada linha de texto. Por padrão, está desligado, o que pode ser alterado nas preferências (veja Configurando preferências).

Zoom/Restore Height
: Alterna a janela entre o tamanho normal e a altura máxima. O tamanho inicial é predefinido para 40 linhas por 80 caracteres, a menos que seja alterado na General da caixa de diálogo Configure IDLE. A altura máxima de uma tela é determinada ao maximizar momentaneamente uma janela pela primeira vez que ela é ampliada na tela. Alterar as configurações da tela pode invalidar a altura salva. Este alternar não tem efeito quando uma janela está maximizada.

Menu Window (Shell e Editor)

Lista os nomes de todas as janelas abertas; selecione uma para trazê-la ao primeiro plano (restaurando-a, se necessário).

Menu Help (Shell e Editor)

About IDLE
: Exibe versão, direitos de autor, licença, créditos e mais.

IDLE Help
: Exibe este documento do IDLE, detalhando as opções de menu, edição básica e navegação, e outras dicas.

Python Docs
: Acessa a documentação local do Python, se instalada, ou inicia um navegador web e abre docs.python.org mostrando a documentação mais recente do Python.

Turtle Demo
: Executa o módulo turtledemo com exemplos de código Python e gráficos de tartaruga.

Fontes de ajuda adicionais podem ser adicionadas aqui com a caixa de diálogo Configure IDLE na aba General. Veja a subsecção Fontes de ajuda abaixo para mais informações sobre as escolhas do menu Help.

Menus de contexto

Abra um menu de contexto clicando com o botão direito em uma janela (Control+clique no macOS). Os menus de contexto também têm as funções padrão da área de transferências no menu Edit.

Cut
: Copia a seleção para a área de transferência do sistema; depois apaga a seleção.

Copy
: Copia a seleção para a área de transferência do sistema.

Paste
: Insere o conteúdo da área de transferência do sistema na janela atual.

As janelas do Editor também têm funções de ponto de interrupção. As linhas com um ponto de interrupção definido são especialmente destacadas. Os pontos de interrupção só têm efeito quando executados no depurador. Os pontos de interrupção de um ficheiro são guardados no diretório `.idlerc` do usuário.

Set Breakpoint
: Define um ponto de interrupção na linha atual.

Clear Breakpoint
: Limpa o ponto de interrupção da linha.

As janelas Shell e de saída também têm os seguintes menus de contexto.

Go to file/line
: Igual ao menu Debug.

A janela Shell também tem uma funcionalidade de compressão de saída explicada na subsecção *Janela Shell Python* abaixo.

Squeeze
: Se o cursor estiver sobre uma linha de saída, comprime toda a saída entre o código acima e o prompt abaixo em um rótulo ‘Squeezed text’.

Edição e navegação

Janelas do Editor

O IDLE pode abrir janelas de editor ao iniciar, dependendo das configurações e de como é iniciado. Depois, use o menu File. Só pode haver uma janela de editor aberta para um determinado arquivo.

A barra de título contém o nome do arquivo, o caminho completo e as versões do Python e do IDLE que estão sendo executadas na janela. A barra de estado contém o número da linha (‘Ln’) e o da coluna (‘Col’). Os números de linha começam em 1; os de coluna, em 0.

O IDLE presume que os arquivos com uma extensão .py* conhecida contêm código Python e que os demais não contêm. Execute o código Python pelo menu Run.

Teclas de atalho

O cursor de inserção do IDLE é uma barra vertical fina entre posições de caracteres. Quando os caracteres são introduzidos, o cursor de inserção e tudo o que estiver à sua direita movem-se uma posição para a direita, e o novo carácter é inserido no espaço criado.

Várias teclas não-caractere movem o cursor e podem apagar caracteres. A eliminação não coloca texto na área de transferência, mas o IDLE tem uma lista de cancelamento. Sempre que este documento discute teclas, ‘C’ refere-se à tecla `Control` no Windows e Unix e à tecla `Command` no macOS. (E todas essas discussões assumem que as teclas não foram reatribuídas a outra coisa.)

- As teclas de seta movem o cursor um caractere ou uma linha.
- `C-Seta-para-esquerda` e `C-Seta-para-direita` movem uma palavra para a esquerda ou para a direita.
- `Home` e `End` vão para o início ou fim da linha.
- `Page Up` e `Page Down` sobem ou descem uma tela.
- `C-Home` e `C-End` vão para o início ou fim do arquivo.
- `Backspace` e `Del` (ou `C-d`) apagam o caractere anterior ou seguinte.
- `C-Backspace` e `C-Del` apagam uma palavra para a esquerda ou para a direita.
- `C-k` apaga ('mata') tudo à direita.

As associações de teclas padrão (como `C-c` para copiar e `C-v` para colar) podem funcionar. As associações de teclas são selecionadas na caixa de diálogo Configure IDLE.

Indentação automática

Depois de uma instrução de abertura de bloco, a linha seguinte é indentada por 4 espaços (na janela do console do Python por uma tabulação). Depois de certas palavras-chave (como break, return etc.), a linha seguinte é desindentada. Na indentação inicial, `Backspace` apaga até 4 espaços se houver. `Tab` insere espaços (na janela do console do Python, uma tabulação), o número depende da largura de indentação. Atualmente, as tabulações estão limitadas a quatro espaços devido a limitações do Tcl/Tk.

Veja também os comandos de indentar/desindentar região no menu Format.

Busca e substituição

Qualquer seleção torna-se um alvo de pesquisa. No entanto, apenas seleções dentro de uma linha funcionam, pois as pesquisas são realizadas apenas dentro de linhas, com a quebra ao final removida. Se `[x] Regular expression` estiver marcada, o alvo é interpretado de acordo com o módulo re do Python.

Autocomplementos

Os autocomplementos são fornecidos, quando solicitados e disponíveis, para nomes de módulos, atributos de classes ou funções, ou nomes de arquivos. Cada método de solicitação exibe uma caixa de complementos com os nomes disponíveis. (Veja os complementos de tabulação abaixo para uma exceção.) Para qualquer caixa, altere o nome que está a ser completado e o item realçado na caixa digitando e apagando caracteres; pressionando as teclas `Up`, `Down`, `PageUp`, `PageDown`, `Home` e `End`; e com um clique simples dentro da caixa. Feche a caixa com as teclas `Escape`, `Enter` e duplo `Tab` ou cliques fora da caixa. Um duplo clique dentro da caixa seleciona e fecha.

Uma forma de abrir uma caixa é pressionar uma tecla de caractere e aguardar um intervalo predefinido. Isto é predefinido para 2 segundos; personalize-o na caixa de diálogo de configurações. (Para evitar pop-ups automáticos, defina o atraso para um valor muito alto, como 100000000.) Para nomes de módulos importados, atributos de classes ou de funções, digite '.'. Para nomes de arquivos no diretório raiz, digite **`os.sep`** ou **`os.altsep`** imediatamente após uma aspa de abertura. (No Windows, pode-se especificar uma unidade antes.) Mova-se para subdiretórios digitando um nome de diretório seguido de um separador.

Em vez de esperar, ou depois de uma caixa ser fechada, abra uma caixa de autocomplemento imediatamente com Show Completions no menu Edit. A tecla de atalho predefinida é `C-space`. Se digitar um prefixo para o nome desejado antes de abrir a caixa, a primeira correspondência ou quase correspondência torna-se visível. O resultado é o mesmo que inserir um prefixo depois que a caixa é exibida. Show Completions depois de uma aspa completa nomes de arquivos no diretório atual em vez do diretório raiz.

Pressionar `Tab` após um prefixo geralmente tem o mesmo efeito que Show Completions. (Sem prefixo, faz indentação.) No entanto, se houver apenas uma correspondência para o prefixo, ela é imediatamente adicionada ao texto do editor, sem abrir uma caixa.

Invocar 'Show Completions', ou pressionar `Tab` após um prefixo, fora de uma string e sem um '.' anterior, abre uma caixa com palavras reservadas, nomes embutidos e nomes disponíveis ao nível do módulo.

Ao editar código em um editor (em oposição à Shell), expanda os nomes disponíveis ao nível do módulo executando o seu código e não reiniciando a Shell depois disso. Isto é especialmente útil depois de adicionar importações no topo de um arquivo. Isto também amplia os possíveis complementos de atributos.

As caixas de autocomplemento excluem inicialmente nomes que começam com '_' ou, para módulos, não incluídos em '__all__'. Os nomes ocultos podem ser acessados digitando '_' depois de '.', antes ou depois da caixa ser aberta.

Dicas de chamada

Uma dica de chamada é exibida automaticamente quando se digita `(` após o nome de uma função *acessível*. Uma expressão de nome de função pode incluir pontos e índices. Uma dica de chamada permanece até ser clicada, o cursor ser movido para fora da área de argumentos, ou `)` ser digitado. Sempre que o cursor estiver na parte de argumentos de uma definição, selecione Edit e "Show Call Tip" no menu ou insira o seu atalho para exibir uma dica de chamada.

A dica de chamada consiste na assinatura da função e na string de documentação até a primeira linha em branco desta ou a quinta linha não em branco. (Algumas funções embutidas não têm uma assinatura acessível.) Uma '/' ou '*' na assinatura indica que os argumentos precedentes ou seguintes são passados apenas por posição ou nome (palavra-chave). Os detalhes estão sujeitos a alterações.

Na Shell, as funções acessíveis dependem de quais módulos foram importados para o processo do usuário, incluindo aqueles importados pelo próprio IDLE, e quais definições foram executadas, tudo desde o último reinício.

Por exemplo, reinicie a Shell e introduza `itertools.count(`. Uma dica de chamada aparece porque o IDLE importa itertools para o processo do utilizador para o seu próprio uso. (Isto pode mudar.) Introduza `turtle.write(` e nada aparece. O IDLE não importa turtle por si só. A

entrada de menu e o atalho também não fazem nada. Introduza `import turtle`. Depois disso, `turtle.write(` exibirá uma dica de chamada.

Num editor, as instruções de importação não têm efeito até que o arquivo seja executado. Pode-se querer executar um arquivo depois de escrever instruções de importação, depois de adicionar definições de funções, ou depois de abrir um arquivo existente.

Formatação de bloco

A opção “Reformat Paragraph” reembrulha um bloco (‘parágrafo’) de comentários não em branco igualmente indentados e contíguos, um bloco semelhante de texto dentro de um string com várias linhas, ou um subconjunto selecionado de qualquer um deles. Se necessário, adicione uma linha em branco para separar string do código. Linhas parciais em uma seleção são expandidas para formar linhas completas. As linhas resultantes têm a mesma indentação de antes, mas apresentam um comprimento total máximo de N colunas (caracteres). Altere o padrão N de 72 na guia Windows da janela IDLE Settings.

Contexto de código

Dentro de uma janela de editor contendo código Python, o contexto de código pode ser alternado para mostrar ou ocultar um painel no topo da janela. Quando mostrado, este painel congela as linhas de abertura para código de bloco, como aquelas que começam com as palavras-chave `class`, `def` ou `if`, que de outra forma poderia passar despercebido em uma rolagem de página. O tamanho do painel será expandido e contraído conforme necessário para mostrar todos os níveis atuais de contexto, até ao número máximo de linhas definido na caixa de diálogo Configure IDLE (que é 15 por padrão). Se não houver linhas de contexto atuais e a funcionalidade for ativada, uma única linha em branco será exibida. Clicar numa linha no painel de contexto moverá essa linha para o topo do editor.

As cores de texto e fundo para o painel de contexto podem ser configuradas no separador Highlights na caixa de diálogo Configure IDLE.

Janela Shell

Na Shell do IDLE, introduza, edite e recupere instruções completas. (A maioria dos terminais e consolas só funciona com uma única linha física de cada vez).

Envie uma instrução de linha única para execução pressionando `Return` com o cursor em qualquer lugar na linha. Se uma linha for estendida com barra invertida (\), o cursor deve estar na última linha física. Envie uma instrução composta de várias linhas inserindo uma linha em branco após a instrução.

Quando se cola código na Shell, ele não é compilado e possivelmente executado até que se pressione `Return`, como especificado acima. Pode-se editar o código colado primeiro. Se colar mais do que uma instrução na Shell, o resultado será um **`SyntaxError`** quando várias instruções são compiladas como se fossem uma.

Linhas contendo `RESTART` significam que o processo de execução do usuário foi reiniciado. Isto ocorre quando o processo de execução do usuário falhou, quando se solicita um reinício no menu Shell, ou quando se executa código numa janela do editor.

As funcionalidades de edição descritas nas subsecções anteriores funcionam ao introduzir código interativamente. A janela Shell do IDLE também responde ao seguinte:

- `C-c` tenta interromper a execução da instrução (mas pode falhar).
- `C-d` fecha a Shell se digitado num prompt `>>>`.
- `Alt-p` e `Alt-n` (`C-p` e `C-n` no macOS) recuperam para o prompt atual a instrução anterior ou seguinte anteriormente introduzida que corresponda a algo já digitado.
- `Return` enquanto o cursor está em qualquer instrução anterior anexa esta ao que já foi digitado no prompt.

Text colors

Idle defaults to black on white text, but colors text with special meanings. For the shell, these are shell output, shell error, user output, and user error. For Python code, at the shell prompt or in an editor, these are keywords, builtin class and function names, names following `class` and `def`, strings, and comments. For any text window, these are the cursor (when present), found text (when possible), and selected text.

IDLE also highlights the soft keywords **`match`**, **`case`**, and `_` in pattern-matching statements. However, this highlighting is not perfect and will be incorrect in some rare cases, including some `_`-s in `case` patterns.

Text coloring is done in the background, so uncolorized text is occasionally visible. To change the color scheme, use the Configure IDLE dialog Highlighting tab. The marking of debugger breakpoint lines in the editor and text in popups and dialogs is not user-configurable.

Startup and Code Execution

Upon startup with the `-s` option, IDLE will execute the file referenced by the environment variables **`IDLESTARTUP`** or **`PYTHONSTARTUP`**. IDLE first checks for `IDLESTARTUP`; if `IDLESTARTUP` is present the file referenced is run. If `IDLESTARTUP` is not present, IDLE checks for `PYTHONSTARTUP`. Files referenced by these environment variables are convenient places to store functions that are used frequently from the IDLE shell, or for executing import statements to import common modules.

In addition, `Tk` also loads a startup file if it is present. Note that the Tk file is loaded unconditionally. This additional file is `.Idle.py` and is looked for in the user's home directory. Statements in this file will be executed in the Tk namespace, so this file is not useful for importing functions to be used from IDLE's Python shell.

Uso na linha de comando

IDLE can be invoked from the command line with various options. The general syntax is:

```
python -m idlelib [options] [file ...]
```

The following options are available:

-c <command>

Run the specified Python command in the shell window. For example, pass `-c "print('Hello, World!')"`. On Windows, the outer quotes must be double quotes as shown.

-d

Enable the debugger and open the shell window.

-e

Open an editor window.

-h

Print a help message with legal combinations of options and exit.

-i

Open a shell window.

-r <file>

Run the specified file in the shell window.

-s

Run the startup file (as defined by the environment variables **`IDLESTARTUP`** or **`PYTHONSTARTUP`**) before opening the shell window.

-t <title>

Set the title of the shell window.

-

Read and execute standard input in the shell window. This option must be the last one before any arguments.

If arguments are provided:

- If `-`, `-c`, or `-r` is used, all arguments are placed in `sys.argv[1:]`, and `sys.argv[0]` is set to `''`, `'-c'`, or `'-r'` respectively. No editor window is opened, even if that is the default set in the *Options* dialog.
- Otherwise, arguments are treated as files to be opened for editing, and `sys.argv` reflects the arguments passed to IDLE itself.

Startup failure

IDLE uses a socket to communicate between the IDLE GUI process and the user code execution process. A connection must be established whenever the Shell starts or restarts. (The latter is indicated by a divider line that says 'RESTART'). If the user process fails to connect to the GUI process, it usually displays a `Tk` error box with a 'cannot connect' message that directs the user here. It then exits.

One specific connection failure on Unix systems results from misconfigured masquerading rules somewhere in a system's network setup. When IDLE is started from a terminal, one will see a message starting with `** Invalid host:`. The valid value is `127.0.0.1 (idlelib.rpc.LOCALHOST)`. One can diagnose with `tcpconnect -irv 127.0.0.1 6543` in one terminal window and `tcplisten <same args>` in another.

A common cause of failure is a user-written file with the same name as a standard library module, such as *random.py* and *tkinter.py*. When such a file is located in the same directory as a file that is about to be run, IDLE cannot import the stdlib file. The current fix is to rename the user file.

Though less common than in the past, an antivirus or firewall program may stop the connection. If the program cannot be taught to allow the connection, then it must be turned off for IDLE to work. It is safe to allow this internal connection because no data is visible on external ports. A similar problem is a network mis-configuration that blocks connections.

Python installation issues occasionally stop IDLE: multiple versions can clash, or a single installation might need admin access. If one undo the clash, or cannot or does not want to run as admin, it might be easiest to completely remove Python and start over.

A zombie pythonw.exe process could be a problem. On Windows, use Task Manager to check for one and stop it if there is. Sometimes a restart initiated by a program crash or Keyboard Interrupt (control-C) may fail to connect. Dismissing the error box or using Restart Shell on the Shell menu may fix a temporary problem.

When IDLE first starts, it attempts to read user configuration files in `~/.idlerc/` (~ is one's home directory). If there is a problem, an error message should be displayed. Leaving aside random disk glitches, this can be prevented by never editing the files by hand. Instead, use the configuration dialog, under Options. Once there is an error in a user configuration file, the best solution may be to delete it and start over with the settings dialog.

If IDLE quits with no message, and it was not started from a console, try starting it from a console or terminal (`python -m idlelib`) and see if this results in an error message.

On Unix-based systems with tcl/tk older than `8.6.11` (see `About IDLE`) certain characters of certain fonts can cause a tk failure with a message to the terminal. This can happen either if one starts IDLE to edit a file with such a character or later when entering such a character. If one cannot upgrade tcl/tk, then re-configure IDLE to use a font that works better.

Running user code

With rare exceptions, the result of executing Python code with IDLE is intended to be the same as executing the same code by the default method, directly with Python in a text-mode system console or terminal window. However, the different interface and operation occasionally affect visible results. For instance, `sys.modules` starts with more entries, and `threading.active_count()` returns 2 instead of 1.

By default, IDLE runs user code in a separate OS process rather than in the user interface process that runs the shell and editor. In the execution process, it replaces `sys.stdin`, `sys.stdout`, and `sys.stderr` with objects that get input from and send output to the Shell window. The original values

stored in `sys.__stdin__`, `sys.__stdout__`, and `sys.__stderr__` are not touched, but may be `None`.

Sending print output from one process to a text widget in another is slower than printing to a system terminal in the same process. This has the most effect when printing multiple arguments, as the string for each argument, each separator, the newline are sent separately. For development, this is usually not a problem, but if one wants to print faster in IDLE, format and join together everything one wants displayed together and then print a single string. Both format strings and **`str.join()`** can help combine fields and lines.

IDLE's standard stream replacements are not inherited by subprocesses created in the execution process, whether directly by user code or by modules such as multiprocessing. If such subprocess use `input` from sys.stdin or `print` or `write` to sys.stdout or sys.stderr, IDLE should be started in a command line window. (On Windows, use `python` or `py` rather than `pythonw` or `pyw`.) The secondary subprocess will then be attached to that window for input and output.

If `sys` is reset by user code, such as with `importlib.reload(sys)`, IDLE's changes are lost and input from the keyboard and output to the screen will not work correctly.

When Shell has the focus, it controls the keyboard and screen. This is normally transparent, but functions that directly access the keyboard and screen will not work. These include system-specific functions that determine whether a key has been pressed and if so, which.

The IDLE code running in the execution process adds frames to the call stack that would not be there otherwise. IDLE wraps `sys.getrecursionlimit` and `sys.setrecursionlimit` to reduce the effect of the additional stack frames.

When user code raises SystemExit either directly or by calling sys.exit, IDLE returns to a Shell prompt instead of exiting.

User output in Shell

When a program outputs text, the result is determined by the corresponding output device. When IDLE executes user code, `sys.stdout` and `sys.stderr` are connected to the display area of IDLE's Shell. Some of its features are inherited from the underlying Tk Text widget. Others are programmed additions. Where it matters, Shell is designed for development rather than production runs.

For instance, Shell never throws away output. A program that sends unlimited output to Shell will eventually fill memory, resulting in a memory error. In contrast, some system text windows only keep the last n lines of output. A Windows console, for instance, keeps a user-settable 1 to 9999 lines, with 300 the default.

A Tk Text widget, and hence IDLE's Shell, displays characters (codepoints) in the BMP (Basic Multilingual Plane) subset of Unicode. Which characters are displayed with a proper glyph and which with a replacement box depends on the operating system and installed fonts. Tab characters cause the following text to begin after the next tab stop. (They occur every 8 'characters'). Newline characters cause following text to appear on a new line. Other control characters are ignored or displayed as a space, box, or something else, depending on the operating system and font. (Moving the text cursor through such output with arrow keys may exhibit some surprising spacing behavior.)

```
>>> s = 'a\tb\a<\x02><\r>\bc\nd'  # Enter 22 chars.
>>> len(s)
14
>>> s  # Display repr(s)
'a\tb\x07<\x02><\r>\x08c\nd'
>>> print(s, end='')  # Display s as is.
# Result varies by OS and font.  Try it.
```

The `repr` function is used for interactive echo of expression values. It returns an altered version of the input string in which control codes, some BMP codepoints, and all non-BMP codepoints are replaced with escape codes. As demonstrated above, it allows one to identify the characters in a string, regardless of how they are displayed.

Normal and error output are generally kept separate (on separate lines) from code input and each other. They each get different highlight colors.

For SyntaxError tracebacks, the normal '^' marking where the error was detected is replaced by coloring the text with an error highlight. When code run from a file causes other exceptions, one may right click on a traceback line to jump to the corresponding line in an IDLE editor. The file will be opened if necessary.

Shell has a special facility for squeezing output lines down to a 'Squeezed text' label. This is done automatically for output over N lines (N = 50 by default). N can be changed in the PyShell section of the General page of the Settings dialog. Output with fewer lines can be squeezed by right clicking on the output. This can be useful lines long enough to slow down scrolling.

Squeezed output is expanded in place by double-clicking the label. It can also be sent to the clipboard or a separate view window by right-clicking the label.

Developing tkinter applications

IDLE is intentionally different from standard Python in order to facilitate development of tkinter programs. Enter `import tkinter as tk; root = tk.Tk()` in standard Python and nothing appears. Enter the same in IDLE and a tk window appears. In standard Python, one must also enter `root.update()` to see the window. IDLE does the equivalent in the background, about 20 times a second, which is about every 50 milliseconds. Next enter `b = tk.Button(root, text='button'); b.pack()`. Again, nothing visibly changes in standard Python until one enters `root.update()`.

Most tkinter programs run `root.mainloop()`, which usually does not return until the tk app is destroyed. If the program is run with `python -i` or from an IDLE editor, a `>>>` shell prompt does not appear until `mainloop()` returns, at which time there is nothing left to interact with.

When running a tkinter program from an IDLE editor, one can comment out the mainloop call. One then gets a shell prompt immediately and can

interact with the live application. One just has to remember to re-enable the mainloop call when running in standard Python.

Running without a subprocess

By default, IDLE executes user code in a separate subprocess via a socket, which uses the internal loopback interface. This connection is not externally visible and no data is sent to or received from the internet. If firewall software complains anyway, you can ignore it.

If the attempt to make the socket connection fails, Idle will notify you. Such failures are sometimes transient, but if persistent, the problem may be either a firewall blocking the connection or misconfiguration of a particular system. Until the problem is fixed, one can run Idle with the -n command line switch.

If IDLE is started with the -n command line switch it will run in a single process and will not create the subprocess which runs the RPC Python execution server. This can be useful if Python cannot create the subprocess or the RPC socket interface on your platform. However, in this mode user code is not isolated from IDLE itself. Also, the environment is not restarted when Run/Run Module (F5) is selected. If your code has been modified, you must reload() the affected modules and re-import any specific items (e.g. from foo import baz) if the changes are to take effect. For these reasons, it is preferable to run IDLE with the default subprocess if at all possible.

Descontinuado desde a versão 3.4.

Ajuda e preferências

Help sources

Help menu entry “IDLE Help” displays a formatted html version of the IDLE chapter of the Library Reference. The result, in a read-only tkinter text window, is close to what one sees in a web browser. Navigate through the text with a mousewheel, the scrollbar, or up and down arrow keys held

down. Or click the TOC (Table of Contents) button and select a section header in the opened box.

Help menu entry “Python Docs” opens the extensive sources of help, including tutorials, available at `docs.python.org/x.y`, where ‘x.y’ is the currently running Python version. If your system has an off-line copy of the docs (this may be an installation option), that will be opened instead.

Selected URLs can be added or removed from the help menu at any time using the General tab of the Configure IDLE dialog.

Configurando preferências

The font preferences, highlighting, keys, and general preferences can be changed via Configure IDLE on the Option menu. Non-default user settings are saved in a `.idlerc` directory in the user’s home directory. Problems caused by bad user configuration files are solved by editing or deleting one or more of the files in `.idlerc`.

On the Font tab, see the text sample for the effect of font face and size on multiple characters in multiple languages. Edit the sample to add other characters of personal interest. Use the sample to select monospaced fonts. If particular characters have problems in Shell or an editor, add them to the top of the sample and try changing first size and then font.

On the Highlights and Keys tab, select a built-in or custom color theme and key set. To use a newer built-in color theme or key set with older IDLEs, save it as a new custom theme or key set and it well be accessible to older IDLEs.

IDLE on macOS

Under System Preferences: Dock, one can set “Prefer tabs when opening documents” to “Always”. This setting is not compatible with the tk/tkinter GUI framework used by IDLE, and it breaks a few IDLE features.

Extensions

IDLE contains an extension facility. Preferences for extensions can be changed with the Extensions tab of the preferences dialog. See the beginning of config-extensions.def in the idlelib directory for further information. The only current default extension is zzdummy, an example also used for testing.

idlelib — implementation of IDLE application

Source code: Lib/idlelib [https://github.com/python/cpython/tree/3.15/Lib/idlelib]

The Lib/idlelib package implements the IDLE application. See the rest of this page for how to use IDLE.

The files in idlelib are described in idlelib/README.txt. Access it either in idlelib or click Help => About IDLE on the IDLE menu. This file also maps IDLE menu items to the code that implements the item. Except for files listed under ‘Startup’, the idlelib code is ‘private’ in sense that feature changes can be backported (see **PEP 434** [https://peps.python.org/pep-0434/]).

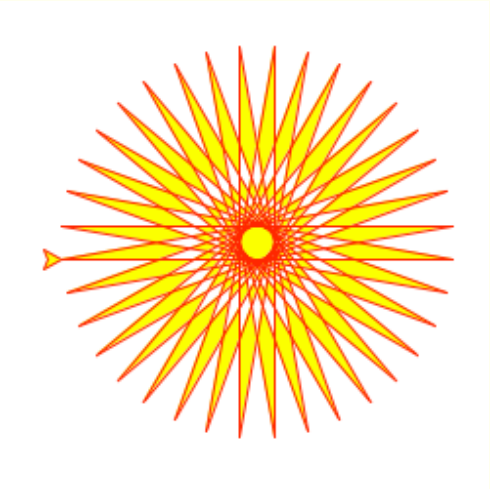
`turtle` — Gráficos tartaruga

Código-fonte: Lib/turtle.py [https://github.com/python/cpython/tree/3.15/Lib/turtle.py]

Imagine uma tartaruga robótica começando em (0, 0) no plano x-y. Depois de um `import turtle`, execute o comando `turtle.forward(15)`, e ela moverá (na tela!) 15 pixels na direção em que está virada (para frente), desenhando uma linha à medida que se move. Execute o comando `turtle.right(25)`, e ela fará uma rotação, de 25 graus no sentido horário, sem sair do lugar.

Turtle star

A tartaruga pode desenhar formas intrincadas usando programas que repetem movimentos simples.



Os gráficos tartaruga são uma implementação das ferramentas de desenho introduzidas na linguagem Logo em 1967 [https://en.wikipedia.org/wiki/Turtle_(robot)]. A ferramenta foi criada para fins educacionais, e seu feedback visual imediato a torna um meio eficaz para os alunos entrarem em contato com conceitos de programação. É também uma maneira prática de gerar saídas gráficas simples sem a necessidade de bibliotecas externas.

Esse documento inclui quatro seções principais:

- Tutorial ensina os básicos do desenho tartaruga.
- Referência descreve as funções, métodos e classes que este módulo define.
- Guias de como fazer detalha como lidar com tarefas específicas.
- Explicação fornece informações básicas sobre a interface orientada a objetos.

Nota

Os gráficos tartaruga requerem o módulo opcional `tkinter`. Os instaladores do python.org para Windows e macOS o incluem, mas algumas distribuições Linux e outras plataformas podem empacotá-lo separadamente. Se `import turtle` falhar com um erro mencionando `_tkinter`, consulte a documentação da sua distribuidora (ou seja, de quem forneceu o Python para você). Verifique isso com antecedência se você planeja usar o gráficos tartaruga com um estudante.

Tutorial

As pessoas recém-chegadas devem começar por aqui. Neste tutorial, exploraremos alguns dos conceitos básicos do desenho de tartarugas.

Iniciando um ambiente da tartaruga

Em um console do Python, faça a importação de todos os objetos do módulo `turtle`:

```
from turtle import *
```

Se você se deparar com a seguinte mensagem de erro `Standard library module '_tkinter' was not found`, será necessário instalar o `pacote de interface gráfica Tk` em seu sistema.

Desenho básico

Envie a tartaruga para frente 100 passos:

```
forward(100)
```

Você deverá ver (provavelmente, em uma nova janela na sua tela) uma linha desenhada pela tartaruga, em direção ao leste (→) . Mude a direção da tartaruga, de modo que ela gire 120 graus para a esquerda (sentido anti-horário):

```
left(120)
```

Vamos continuar desenhando um triângulo:

```
forward(100)
left(120)
forward(100)
```

Observe como a tartaruga, representada por uma seta, aponta em diferentes direções à medida que você a guia.

Experimente esses comandos, bem como `backward()` e `right()`. Muitos comandos também possuem apelidos mais curtos, como `fd()` para `forward()`.

Controle da Caneta

Tente alterar a cor - por exemplo, `color('blue')` - e a largura da linha - por exemplo, `width(3)` - e, em seguida, desenhe novamente.

Você também pode mover a tartaruga sem desenhar, levantando a caneta: `up()` antes de mover. Para começar a desenhar novamente, use `down()`.

A posição da tartaruga

Envie sua tartaruga de volta ao ponto de partida (útil se ela tiver desaparecido da tela):

```
home()
```

A posição inicial da tartaruga está no centro da tela. Se você precisar conhecê-la, obtenha as coordenadas x-y da tartaruga com:

```
pos()
```

A a posição inicial fica em `(0, 0)`.

E, depois de um tempo, provavelmente vai querer ajudar a limpar a janela aberta para que possamos começar de novo:

```
clearscreen()
```

Criação de padrões algorítmicos

Usando laços de repetição, é possível construir criar padrões geométricos:

```
for steps in range(100):
    for c in ('blue', 'red', 'green'):
        color(c)
        forward(steps)
        right(30)
```

- que, é claro, são limitados apenas pela imaginação!

Vamos desenhar o formato de estrela na parte superior da página. Queremos linhas de contorno vermelhas, com preenchimento em amarelo:

```
color('red')
fillcolor('yellow')
```

Assim como `up()` e `down()` determinam se as linhas serão desenhadas, o preenchimento pode ser ativado e desativado:

```
begin_fill()
```

Em seguida, vamos criar um laço de repetição:

```
start = pos()

while True:
    forward(200)
    left(170)
    if distance(start) < 1:
        break
```

`distance(start) < 1` é uma boa maneira de saber quando a tartaruga está de volta à sua posição inicial.

Por fim, complete o preenchimento:

```
end_fill()
```

(Observe que o preenchimento só ocorre de fato quando você fornece o comando `end_fill()` .)

Referência

Métodos e funções Turtle

A maioria dos exemplos desta seção referem-se a uma instância Turtle chamada `turtle`.

Movimento e desenho

turtle.forward(*distance*)
turtle.fd(*distance*)

Parâmetros:: **distance** – um número

Move a tartaruga para frente pela *distance* especificada, na direção em que a tartaruga está indo.

```
>>> turtle.position()
(0.00,0.00)
>>> turtle.forward(25)
>>> turtle.position()
(25.00,0.00)
>>> turtle.forward(-75)
>>> turtle.position()
(-50.00,0.00)
```

turtle.back(*distance*)
turtle.bk(*distance*)
turtle.backward(*distance*)

Parâmetros:: **distance** – um número

Move a tartaruga para trás por *distance*, na direção oposta à direção em que a tartaruga está indo. A direção da tartaruga não muda.

```
>>> turtle.position()
(0.00,0.00)
```

```
>>> turtle.backward(30)
>>> turtle.position()
(-30.00,0.00)
```

turtle.right(*angle*)
turtle.rt(*angle*)

Parâmetros:: **angle** – um número

Gira a tartaruga para direita pelo ângulo *angle* especificado. Por padrão, o ângulo é medido em graus; a unidade pode ser alterada com `degrees()` ou `radians()`. A forma como a direção é medida depende do modo da tartaruga; consulte `mode()`.

```
>>> turtle.heading()
22.0
>>> turtle.right(45)
>>> turtle.heading()
337.0
```

turtle.left(*angle*)
turtle.lt(*angle*)

Parâmetros:: **angle** – um número

Gira a tartaruga para esquerda pelo ângulo *angle* especificado. Por padrão, o ângulo é medido em graus; a unidade pode ser alterada com `degrees()` ou `radians()`. A forma como a direção é medida depende do modo da tartaruga; consulte `mode()`.

```
>>> turtle.heading()
22.0
>>> turtle.left(45)
>>> turtle.heading()
67.0
```

turtle.goto(*x*, *y=None*)
turtle.setpos(*x*, *y=None*)
turtle.setposition(*x*, *y=None*)

Parâmetros::
- **x** – um número ou um par/vetor de números
- **y** – um número ou `None`

Move a tartaruga para uma posição absoluta. Se *y* for `None`, *x* deve ser um par de coordenadas ou um **`Vec2D`**, como, por exemplo, o valor retornado por **`pos()`**. Se a caneta estiver abaixada, uma linha é desenhada. A orientação da tartaruga não muda.

```
>>> tp = turtle.pos()
>>> tp
(0.00,0.00)
>>> turtle.goto(60,30)
>>> turtle.pos()
(60.00,30.00)
>>> turtle.goto((20,80))
>>> turtle.pos()
(20.00,80.00)
>>> turtle.goto(tp)
>>> turtle.pos()
(0.00,0.00)
```

turtle.teleport(*x*, *y=None*, *, *fill_gap=False*)

Parâmetros::

- **x** – um número ou `None`
- **y** – um número ou `None`
- **fill_gap** – um valor booleano

Move a tartaruga para uma posição absoluta. Ao contrário de goto(x, y), uma linha não será desenhada. A orientação da tartaruga não muda. Se estiver sendo preenchido no momento, o(s) polígono(s) teletransportado(s) será(ão) preenchido(s) após sair, e o preenchimento começará novamente após o teletransporte. Isso pode ser desabilitado com fill_gap=True, o que faz com que a linha imaginária percorrida durante o teletransporte atue como uma barreira de preenchimento como em goto(x, y).

```
>>> tp = turtle.pos()
>>> tp
(0.00,0.00)
>>> turtle.teleport(60)
>>> turtle.pos()
(60.00,0.00)
>>> turtle.teleport(y=10)
>>> turtle.pos()
(60.00,10.00)
```

```
>>> turtle.teleport(20, 30)
>>> turtle.pos()
(20.00,30.00)
```

Adicionado na versão 3.12.

turtle.setx(*x*)

Parâmetros:: **x** – um número

Define a coordenada x da tartaruga para *x*. A coordenada y permanece inalterada.

```
>>> turtle.position()
(0.00,240.00)
>>> turtle.setx(10)
>>> turtle.position()
(10.00,240.00)
```

turtle.sety(*y*)

Parâmetros:: **y** – um número

Define a coordenada y da tartaruga para *y*. A coordenada x permanece inalterada.

```
>>> turtle.position()
(0.00,40.00)
>>> turtle.sety(-10)
>>> turtle.position()
(0.00,-10.00)
```

turtle.setheading(*to_angle*)
turtle.seth(*to_angle*)

Parâmetros:: **to_angle** – um número

Define a direção da tartaruga para *to_angle*. Aqui estão algumas direções mais comuns em graus:

modo padrão	modo logo
0 - leste	0 - norte

modo padrão	modo logo
90 - norte	90 - leste
180 - oeste	180 - sul
270 - sul	270 - oeste

```
>>> turtle.setheading(90)
>>> turtle.heading()
90.0
```

turtle.home()

Move a tartaruga para a origem, coordenadas (0,0). A direção da tartaruga é defina para sua orientação inicial, que depende do modo. Veja `mode()`.

```
>>> turtle.heading()
90.0
>>> turtle.position()
(0.00,-10.00)
>>> turtle.home()
>>> turtle.position()
(0.00,0.00)
>>> turtle.heading()
0.0
```

turtle.circle(*radius*, *extent=None*, *steps=None*)

Parâmetros::

- **radius** – um número
- **extent** – um número ou `None`
- **steps** – um inteiro ou `None`

Desenha um círculo com o *radius* dado. O centro são as unidades de *radius* à esquerda da tartaruga; *extent*, um ângulo, determina qual parte do círculo é desenhada. Se *extent* não for fornecida, desenha o círculo inteiro. Se *extent* não for um círculo completo, uma extremidade do arco será a posição atual da caneta. Desenha o arco no sentido anti-horário se *radius* for positivo, caso contrário, no sentido horário. Finalmente, a direção da tartaruga é alterada da *extent*.

Como o círculo é aproximado por um polígono regular inscrito, *steps* determina o número de passos a serem usados. Caso não seja informado, será calculado automaticamente. Pode ser usado para desenhar polígonos regulares.

```
>>> turtle.home()
>>> turtle.position()
(0.00,0.00)
>>> turtle.heading()
0.0
>>> turtle.circle(50)
>>> turtle.position()
(-0.00,0.00)
>>> turtle.heading()
0.0
>>> turtle.circle(120, 180)  # draw a semicircle
>>> turtle.position()
(0.00,240.00)
>>> turtle.heading()
180.0
```

turtle.dot()
turtle.dot(*size*)
turtle.dot(*color*, *..*)
turtle.dot(*size*, *color*, *..*)
turtle.dot(*size*, *r*, *g*, *b*, *..*)

Parâmetros::

- **size** – um inteiro >= 1 (caso seja fornecido)
- **color** – uma string de cores ou uma tupla de cores numéricas

Desenha um ponto circular com diâmetro *size*, usando *color*. Se *size* não for fornecido, o máximo de `pensize+4` e `2*pensize` será usado.

```
>>> turtle.home()
>>> turtle.dot()
>>> turtle.fd(50); turtle.dot(20, "blue"); turtle.fd(50)
>>> turtle.position()
(100.00,-0.00)
>>> turtle.heading()
0.0
```

turtle.stamp()

Carimba uma cópia da forma da tartaruga na tela na posição atual da tartaruga. Retorna um stamp_id para esse carimbo, que pode ser usado para excluí-lo chamando `clearstamp(stamp_id)`.

```
>>> turtle.color("blue")
>>> stamp_id = turtle.stamp()
>>> turtle.fd(50)
```

turtle.clearstamp(*stampid*)

Parâmetros:: **stampid** – um inteiro, deve ser o valor de retorno da chamada de `stamp()` anterior

Exclui o carimbo com o *stamp* fornecido.

```
>>> turtle.position()
(150.00,-0.00)
>>> turtle.color("blue")
>>> astamp = turtle.stamp()
>>> turtle.fd(50)
>>> turtle.position()
(200.00,-0.00)
>>> turtle.clearstamp(astamp)
>>> turtle.position()
(200.00,-0.00)
```

turtle.clearstamps(*n=None*)

Parâmetros:: **n** – um inteiro (ou `None`)

Exclui todos ou o primeiro/último *n* dos selos da tartaruga. Se *n* for `None`, exclui todos os carimbos, se $n > 0$ exclui os primeiros *n* carimbos, senão se $n < 0$ exclui os últimos *n* carimbos.

```
>>> for i in range(8):
...     unused_stamp_id = turtle.stamp()
...     turtle.fd(30)
>>> turtle.clearstamps(2)
>>> turtle.clearstamps(-2)
>>> turtle.clearstamps()
```

turtle.undo()

Desfaz (repetidamente) a(s) última(s) ação(ões) da tartaruga. O número de ações de desfazer disponíveis é determinado pelo tamanho do buffer de desfazer.

```
>>> for i in range(4):
...     turtle.fd(50); turtle.lt(80)
...
>>> for i in range(8):
...     turtle.undo()
```

turtle.speed(*speed=None*)

Parâmetros:: **speed** – um inteiro no intervalo 0..10 ou uma string de velocidade (veja abaixo)

Define a velocidade da tartaruga para um valor inteiro no intervalo 0..10. Se nenhum argumento for fornecido, retorna a velocidade atual.

Se a entrada for um número maior que 10 ou menor que 0,5, a velocidade é definida como 0. As strings de velocidade são mapeadas para valores de velocidade da seguinte forma:

- “fastest”: 0
- “fast”: 10
- “normal”: 6
- “slow”: 3
- “slowest”: 1

Velocidades de 1 a 10 tornam a animação cada vez mais rápida, tanto para o desenho da linha como para a rotação da tartaruga.

Atenção: *speed* = 0 significa que *nenhuma* animação ocorre. forward/back faz a tartaruga pular e da mesma forma para left/right faz a tartaruga girar instantaneamente.

```
>>> turtle.speed()
3
>>> turtle.speed('normal')
>>> turtle.speed()
6
>>> turtle.speed(9)
```

```
>>> turtle.speed()
9
```

Fala o estado de Turtle

turtle.position()
turtle.pos()

Retorna a localização atual da tartaruga (x,y) como um vetor **Vec2D**.

```
>>> turtle.pos()
(440.00,-0.00)
```

turtle.towards(*x*, *y=None*)

Parâmetros::

- **x** – um número ou um par/vetor de números ou uma instância de tartaruga
- **y** – um número caso *x* seja um número, senão None

Retorna o ângulo da linha que vai da posição da tartaruga até (x,y). Se *y* for None, *x* deve ser um par de coordenadas — um **Vec2D**, por exemplo, como o retornado por **pos()** — ou outra tartaruga. O ângulo é medido a partir da orientação inicial da tartaruga, a qual depende do modo da tartaruga; consulte **mode()**.

```
>>> turtle.goto(10, 10)
>>> turtle.towards(0,0)
225.0
```

turtle.xcor()

Retorna a coordenada X da tartaruga.

```
>>> turtle.home()
>>> turtle.left(50)
>>> turtle.forward(100)
>>> turtle.pos()
(64.28,76.60)
>>> print(round(turtle.xcor(), 5))
64.27876
```

turtle.ycor()

Retorna a coordenada Y da tartaruga.

```
>>> turtle.home()
>>> turtle.left(60)
>>> turtle.forward(100)
>>> print(turtle.pos())
(50.00,86.60)
>>> print(round(turtle.ycor(), 5))
86.60254
```

turtle.heading()

Retorna o título atual da tartaruga. O valor depende do modo da tartaruga, veja `mode()`.

```
>>> turtle.home()
>>> turtle.left(67)
>>> turtle.heading()
67.0
```

turtle.distance(*x*, *y=None*)

Parâmetros::

- **x** – um número ou um par/vetor de números ou uma instância de tartaruga
- **y** – um número caso *x* seja um número, senão `None`

Retorna a distância da tartaruga para (x,y) em unidades de passo de tartaruga. Se *y* for `None`, *x* deve ser um par de coordenadas ou um `Vec2D`, como, por exemplo, o valor retornado por `pos()` ou outra tartaruga.

```
>>> turtle.home()
>>> turtle.distance(30,40)
50.0
>>> turtle.distance((30,40))
50.0
>>> joe = Turtle()
>>> joe.forward(77)
>>> turtle.distance(joe)
77.0
```

Configurações de medida

turtle.degrees(*fullcircle=360.0*)

Parâmetros:: **fullcircle** – um número

Define as unidades de medida de ângulo como graus. O número de graus em um círculo completo é definido como *fullcircle*, cujo valor padrão é 360.

```
>>> turtle.home()
>>> turtle.left(90)
>>> turtle.heading()
90.0

>>> # Alterar unidade de medida de ângulo para grad (também
>>> # conhecido como gon, grau ou grado e é igual a 1/100
>>> # do ângulo reto).
>>> turtle.degrees(400.0)
>>> turtle.heading()
100.0
>>> turtle.degrees(360)
>>> turtle.heading()
90.0
```

turtle.radians()

Define as unidades de medida de ângulo para radianos. Equivalente a `degrees(2 * math.pi)`.

```
>>> turtle.home()
>>> turtle.left(90)
>>> turtle.heading()
90.0
>>> turtle.radians()
>>> turtle.heading()
1.5707963267948966
```

Controle da Caneta

Estado do Desenho

turtle.pendown()

turtle.pd()

turtle.down()

Desce a caneta - desenha ao se mover.

turtle.penup()
turtle.pu()
turtle.up()

Levanta a caneta – sem qualquer desenho ao se mover.

turtle.pensize(*width=None*)
turtle.width(*width=None*)

Parâmetros:: **width** – um número positivo

Define a espessura da linha para *width* ou retorna-a. Se resizemode estiver definido como "auto" e a forma de tartaruga for um polígono, esse polígono será desenhado com a mesma espessura de linha. Se nenhum argumento for fornecido, a espessura atual da caneta será retornada.

```
>>> turtle.pensize()
1
>>> turtle.pensize(10)   # from here on lines of width 10 are
```

turtle.pen(*pen=None, **pendict*)

Parâmetros::
- **pen** – um dicionário com algumas ou todas as chaves listadas abaixo
- **pendict** – um ou mais argumentos nomeados com as chaves listadas abaixo como palavras-chave

Retorna ou define os atributos da caneta em um "dicionário da caneta" com os seguintes pares de chave/valor:

- "shown": True/False
- "pendown": True/False
- "pencolor": string de cores ou tupla de cores
- "fillcolor": string de cores ou tupla de cores
- "pensize": número positivo
- "speed": número na faixa de 0..10.
- "resizemode": "auto", "user" ou "noresize"

- “stretchfactor”: (número positivo, número positivo)
- “outline”: número positivo
- “tilt”: número

Este dicionário pode ser usado como argumento para uma chamada subsequente para `pen()` para restaurar o estado da caneta anterior. Além disso, um ou mais desses atributos podem ser fornecidos como argumentos nomeados. Isso pode ser usado para definir vários atributos de caneta em uma instrução.

```
>>> turtle.pen(fillcolor="black", pencolor="red", pensize=10)
>>> sorted(turtle.pen().items())
[('fillcolor', 'black'), ('outline', 1), ('pencolor', 'red'),
 ('pendown', True), ('pensize', 10), ('resizemode', 'noresize'
 ('shearfactor', 0.0), ('shown', True), ('speed', 9),
 ('stretchfactor', (1.0, 1.0)), ('tilt', 0.0)]
>>> penstate=turtle.pen()
>>> turtle.color("yellow", "")
>>> turtle.penup()
>>> sorted(turtle.pen().items())[:3]
[('fillcolor', ''), ('outline', 1), ('pencolor', 'yellow')]
>>> turtle.pen(penstate, fillcolor="green")
>>> sorted(turtle.pen().items())[:3]
[('fillcolor', 'green'), ('outline', 1), ('pencolor', 'red')]
```

turtle.isdown()

Retorna `True` se a caneta estiver abaixada, `False` se estiver levantada.

```
>>> turtle.penup()
>>> turtle.isdown()
False
>>> turtle.pendown()
>>> turtle.isdown()
True
```

Controle da Cor

turtle.pencolor()

turtle.pencolor(*color*, /)

turtle.pencolor(*r*, *g*, *b*, /)

Retorna ou define a cor da caneta ou pencolor.

São permitidos quatro formatos de entrada:

`pencolor()`
: Retorna a cor da caneta atual como string de especificação de cor ou como uma tupla (veja o exemplo). Pode ser usado como entrada para outra chamada color/pencolor/fillcolor/bgcolor.

`pencolor(colorstring)`
: Define pencolor como *colorstring*, que é uma string de especificação de cor Tk, como `"red"`, `"yellow"` ou `"#33cc8c"`.

`pencolor((r, g, b))`
: Define a cor da caneta como a cor RGB representada pela tupla *r*, *g*, e *b*. Os valores de *r*, *g*, and *b* precisam estar na faixa 0..colormode, onde colormode é 1.0 ou 255 (ver **`colormode()`**).

`pencolor(r, g, b)`
: Define a cor da caneta como a cor RGB representada por *r*, *g*, e *b*. Os valores de *r*, *g*, and *b* precisam estar na faixa 0..colormode.

Se a forma da tartaruga for um polígono, o contorno desse polígono será desenhado com a nova cor de caneta definida.

```
>>> colormode()
1.0
>>> turtle.pencolor()
'red'
>>> turtle.pencolor("brown")
>>> turtle.pencolor()
'brown'
>>> tup = (0.2, 0.8, 0.55)
>>> turtle.pencolor(tup)
>>> turtle.pencolor()
(0.2, 0.8, 0.5490196078431373)
>>> colormode(255)
>>> turtle.pencolor()
(51.0, 204.0, 140.0)
>>> turtle.pencolor('#32c18f')
```

```
>>> turtle.pencolor()
(50.0, 193.0, 143.0)
```

turtle.fillcolor()
turtle.fillcolor(*color*, /)
turtle.fillcolor(*r*, *g*, *b*, /)

Retorna ou define o fillcolor.

São permitidos quatro formatos de entrada:

`fillcolor()`
Retorna a cor de preenchimento atual como string, ou em formato de tupla (veja o exemplo). Pode ser usado como entrada para outra chamada color/pencolor/fillcolor/bgcolor.

`fillcolor(colorstring)`
Define fillcolor como *colorstring*, que é uma especificação de cor do Tk em formato de string, como `"red"`, `"yellow"`, ou `"#33cc8c"`.

`fillcolor((r, g, b))`
Define fillcolor como a cor no padrão RGB representada por tupla de *r*, *g* e *b*. Cada um de *r*, *g* e *b* deve estar no limite 0..colormode, em que colormode é 1,0 ou 255 (consulte **`colormode()`**).

`fillcolor(r, g, b)`
Define fillcolor como uma cor no formato RGB representada por *r*, *g* e *b*. Cada um dos elementos *r*, *g* e *b* devem estar no limite 0..colormode.

Se turtleshape for um polígono, o interior desse polígono será desenhado com a nova cor de preenchimento definida.

```
>>> turtle.fillcolor("violet")
>>> turtle.fillcolor()
'violet'
>>> turtle.pencolor()
(50.0, 193.0, 143.0)
>>> turtle.fillcolor((50, 193, 143))  # Inteiros, não pontos f
>>> turtle.fillcolor()
(50.0, 193.0, 143.0)
```

```
>>> turtle.fillcolor('#ffffff')
>>> turtle.fillcolor()
(255.0, 255.0, 255.0)
```

turtle.color()
turtle.color(*color*, /)
turtle.color(*r*, *g*, *b*, /)
turtle.color(*pencolor*, *fillcolor*, /)

Retorna ou define os valores de pencolor e fillcolor.

Diversos formatos de entrada são permitidos. Eles usam de 0 a 3 argumentos da seguinte forma:

`color()`
Retorna os valores atuais de pencolor e fillcollor como um par de cores como strings ou tuplas conforme retornado por **`pencolor()`** e **`fillcolor()`**.

`color(colorstring)`, `color((r,g,b))`, `color(r,g,b)`
Entradas como em **`pencolor()`**, define ambos, fillcolor e pencolor, para o valor informado.

`color(colorstring1, colorstring2)`, `color((r1,g1,b1), (r2,g2,b2))`
Equivalente a `pencolor(colorstring1)` e `fillcolor(colorstring2)` e de forma análoga se a outra formatação de entrada for usada.

Se o turtleshape for um polígono, o contorno e o interior desse polígono serão desenhados com as cores recém-definidas.

```
>>> turtle.color("red", "green")
>>> turtle.color()
('red', 'green')
>>> color("#285078", "#a0c8f0")
>>> color()
((40.0, 80.0, 120.0), (160.0, 200.0, 240.0))
```

Veja também: Método da tela **`colormode()`**.

Preenchimento

turtle.filling()

Retorna fillstate (`True` se estiver preenchido, `False` caso contrário).

```
>>> turtle.begin_fill()
>>> if turtle.filling():
...     turtle.pensize(5)
... else:
...     turtle.pensize(3)
```

turtle.fill()

Preenche a forma desenhada no bloco `with turtle.fill():`.

```
>>> turtle.color("black", "red")
>>> with turtle.fill():
...      turtle.circle(80)
```

Usar **`fill()`** equivale a adicionar **`begin_fill()`** antes do bloco de preenchimento e **`end_fill()`** depois do bloco de preenchimento:

```
>>> turtle.color("black", "red")
>>> turtle.begin_fill()
>>> turtle.circle(80)
>>> turtle.end_fill()
```

Adicionado na versão 3.14.

turtle.begin_fill()

Deve ser chamado antes de desenhar uma forma a ser preenchida.

turtle.end_fill()

Preenche a forma desenhada após a última chamada para **`begin_fill()`**.

O preenchimento ou não de regiões sobrepostas para polígonos que se cruzam ou formas múltiplas depende dos gráficos do sistema operacional, do tipo de sobreposição e do número de sobreposições. Por exemplo, a estrela da Tartaruga acima pode ser toda amarela ou ter algumas regiões brancas.

```
>>> turtle.color("black", "red")
>>> turtle.begin_fill()
>>> turtle.circle(80)
>>> turtle.end_fill()
```

Mais sobre o Controle do Desenho

turtle.reset()

Remove os desenhos da tartaruga da tela, centraliza novamente a tartaruga e define as variáveis para os valores padrões.

```
>>> turtle.goto(0,-22)
>>> turtle.left(100)
>>> turtle.position()
(0.00,-22.00)
>>> turtle.heading()
100.0
>>> turtle.reset()
>>> turtle.position()
(0.00,0.00)
>>> turtle.heading()
0.0
```

turtle.clear()

Exclui os desenhos da tartaruga da tela e não move a tartaruga. O atual estado e posição da tartaruga, bem como os desenhos de outras tartarugas, não são afetados.

turtle.write(*arg*, *move=False*, *align='left'*, *font=('Arial', 8, 'normal')*)

Parâmetros::

- **arg** – objeto a ser escrito na TurtleScreen
- **move** – True/False
- **align** – uma das Strings “left”, “center” ou right”
- **font** – a triple (fontname, fontsize, fonttype)

Escreve o texto - a representação string de *arg* - na posição atual da tartaruga de acordo com *align* (“left”, “center” ou “right”) e com a fonte fornecida. Se *move* for verdadeiro, a caneta será movida para o canto inferior direito do texto. Por padrão, *move* é `False`.

```
>>> turtle.write("Home = ", True, align="center")
>>> turtle.write((0,0), True)
```

Estado da tartaruga

Visibilidade

turtle.hideturtle()
turtle.ht()

Torna a tartaruga invisível. É uma boa ideia fazer isso enquanto estiver fazendo algum desenho complexo, pois ocultar a tartaruga acelera o desenho de forma visível.

```
>>> turtle.hideturtle()
```

turtle.showturtle()
turtle.st()

Tornar a tartaruga visível.

```
>>> turtle.showturtle()
```

turtle.isvisible()

Retorna `True` se a Tartaruga for exibida, `False` se estiver oculta.

```
>>> turtle.hideturtle()
>>> turtle.isvisible()
False
>>> turtle.showturtle()
>>> turtle.isvisible()
True
```

Aparência

turtle.shape(*name=None*)

Parâmetros:: **name** – uma string que é um shapename válido

Define a forma da tartaruga para a forma com o *nome* fornecido ou, se o nome não for fornecido, retorna nome da forma atual. A forma com *name* deve existir no dicionário TurtleScreen. Inicialmente, há as seguintes

formas de polígono: “arrow” (seta), “turtle” (tartaruga), “circle” (círculo), “square” (quadrado), “triangle” (triângulo), “classic” (clássico). Para saber mais sobre como lidar com formas, consulte o método Screen `register_shape()` .

```
>>> turtle.shape()
'classic'
>>> turtle.shape("turtle")
>>> turtle.shape()
'turtle'
```

turtle.resizemode(*rmode=None*)

Parâmetros:: **rmode** – uma das Strings “auto”, “user”, “noresize”

Define resizemode como um dos valores: “auto”, “user”, “noresize”. Se *rmode* não for fornecido, retorna o modo de redimensionamento atual. Os diferentes modos de redimensionamento têm os seguintes efeitos:

- “auto”: adapta a aparência da tartaruga correspondente ao valor do pensize.
- “user”: adapta a aparência da tartaruga de acordo com os valores de stretchfactor e outlinewidth (outline), que são definidos por `shapesize()`.
- “noresize”: não acontece nenhuma adaptação na aparência da tartaruga.

`resizemode("user")` é chamado por `shapesize()` quando usado com argumentos.

```
>>> turtle.resizemode()
'noresize'
>>> turtle.resizemode("auto")
>>> turtle.resizemode()
'auto'
```

turtle.shapesize(*stretch_wid=None*, *stretch_len=None*, *outline=None*)
turtle.turtlesize(*stretch_wid=None*, *stretch_len=None*, *outline=None*)

Parâmetros::

- **stretch_wid** – número positivo
- **stretch_len** – número positivo
- **outline** – número positivo

Retorna ou define os atributos x/y-stretchfactors e/ou contorno da caneta. Define o modo de redimensionamento como “usuário”. Se e somente se resizemode estiver definido como “user”, a tartaruga será exibida esticada de acordo com seus stretchfactors: *stretch_wid* é stretchfactor perpendicular à sua orientação, *stretch_len* é stretchfactor na direção de sua orientação, *outline* determina a largura do contorno da forma.

```
>>> turtle.shapesize()
(1.0, 1.0, 1)
>>> turtle.resizemode("user")
>>> turtle.shapesize(5, 5, 12)
>>> turtle.shapesize()
(5, 5, 12)
>>> turtle.shapesize(outline=8)
>>> turtle.shapesize()
(5, 5, 8)
```

turtle.shearfactor(*shear=None*)

Parâmetros:: **shear** – número (opcional)

Define ou retorna o fator de shearfactor atual. Corta a forma de tartaruga de acordo com o fator de shearfactor fornecido, que é a tangente do ângulo de shearfactor. *Não* muda a direção da tartaruga (direção do movimento). Se o shearfactor não for fornecido: retorna o fator de shearfactor atual, i. Isso é. a tangente do ângulo de shearfactor, pelo qual as linhas paralelas à direção da tartaruga são cortadas.

```
>>> turtle.shape("circle")
>>> turtle.shapesize(5,2)
>>> turtle.shearfactor(0.5)
>>> turtle.shearfactor()
0.5
```

turtle.tilt(*angle*)

Parâmetros:: **angle** – um número

Gira a forma de tartaruga em um *ângulo* a partir de seu ângulo de inclinação atual, mas *não* altera o rumo da tartaruga (direção do movimento).

```
>>> turtle.reset()
>>> turtle.shape("circle")
>>> turtle.shapesize(5,2)
>>> turtle.tilt(30)
>>> turtle.fd(50)
>>> turtle.tilt(30)
>>> turtle.fd(50)
```

turtle.tiltangle(*angle=None*)

Parâmetros:: **angle** – um número (opcional)

Define ou retorna o ângulo de inclinação atual. Se o ângulo for fornecido, gira a forma de tartaruga para apontar na direção especificada pelo ângulo, independentemente do seu ângulo de inclinação atual. *Não* muda a direção da tartaruga (direção do movimento). Se o ângulo não for fornecido: retorna o ângulo de inclinação atual, que é o ângulo entre a orientação da forma da tartaruga e a direção da tartaruga (sua direção de movimento).

```
>>> turtle.reset()
>>> turtle.shape("circle")
>>> turtle.shapesize(5,2)
>>> turtle.tilt(45)
>>> turtle.tiltangle()
45.0
```

turtle.shapetransform(*t11=None*, *t12=None*, *t21=None*, *t22=None*)

Parâmetros::
- **t11** – um número (opcional)
- **t12** – um número (opcional)
- **t21** – um número (opcional)
- **t12** – um número (opcional)

Define ou retorna a matriz de transformação atual da forma da tartaruga.

Se nenhum dos elementos da matriz for fornecido, retorna a matriz de transformação como uma tupla de 4 elementos. Caso contrário, define os

elementos fornecidos e transforma a forma da tartaruga de acordo com a matriz que consiste na primeira linha t11, t12 e na segunda linha t21, t22. O resultado de t11 * t22 - t12 * t21 não deve ser zero, caso contrário será gerado um erro. Modifica stretchfactor, shearfactor e tiltangle de acordo com a matriz fornecida.

```
>>> turtle = Turtle()
>>> turtle.shape("square")
>>> turtle.shapesize(4,2)
>>> turtle.shearfactor(-0.5)
>>> turtle.shapetransform()
(4.0, -1.0, -0.0, 2.0)
```

turtle.get_shapepoly()

Retorna o polígono da forma atual como uma tupla de pares de coordenadas. Isto pode ser usado para definir uma nova forma ou componentes de uma forma composta.

```
>>> turtle.shape("square")
>>> turtle.shapetransform(4, -1, 0, 2)
>>> turtle.get_shapepoly()
((50, -20), (30, 20), (-50, 20), (-30, -20))
```

Eventos Utilizados

turtle.onclick(*fun*, *btn=1*, *add=None*)

Parâmetros::

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado na tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá uma ligação antiga

Vincula *fun* a eventos de clique do mouse nessa tartaruga. Se *fun* for `None`, as associações existentes serão removidas. Exemplo para a tartaruga anônima, ou seja, a forma processual:

```
>>> def turn(x, y):
...     left(180)
...
>>> onclick(turn)  # Agora, clicar na tartaruga irá girá-la.
>>> onclick(None)  # a associação de eventos será removida
```

turtle.onrelease(*fun*, *btn=1*, *add=None*)

Parâmetros::

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado n tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá um ligação antiga

Vincule *fun* aos eventos de liberação do botão do mouse nesta tartaruga. Se *fun* for `None`, as associações existentes serão removidas.

```
>>> class MyTurtle(Turtle):
...     def glow(self,x,y):
...         self.fillcolor("red")
...     def unglow(self,x,y):
...         self.fillcolor("")
...
>>> turtle = MyTurtle()
>>> turtle.onclick(turtle.glow)     # clicar na tartaruga torn
>>> turtle.onrelease(turtle.unglow) # .liberar a transforma em
```

turtle.ondrag(*fun*, *btn=1*, *add=None*)

Parâmetros::

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado n tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá um ligação antiga

Vincule *fun* a eventos de movimentação do mouse nessa tartaruga. Se *fun* for `None`, as associações existentes serão removidas.

Observação: Cada sequência de eventos de movimento do mouse em uma tartaruga é precedida por um evento de clique do mouse naquela tartaruga.

```
>>> turtle.ondrag(turtle.goto)
```

Posteriormente, clicar e arrastar a Tartaruga a moverá pela tela, produzindo desenhos à mão (se a caneta estiver deitada).

Métodos Especiais da Tartaruga

turtle.poly()

Registra os vértices de um polígono desenhado no bloco `with turtle.poly():`. O primeiro e o último vértices serão conectados.

```
>>> with turtle.poly():
...     turtle.forward(100)
...     turtle.right(60)
...     turtle.forward(100)
```

Adicionado na versão 3.14.

turtle.begin_poly()

Começa a registrar os vértices de um polígono. A posição atual da tartaruga é o primeiro vértice do polígono.

turtle.end_poly()

Para de registrar os vértices de um polígono. A posição atual da tartaruga é o último vértice do polígono. Isto será conectado ao primeiro vértice.

turtle.get_poly()

Retorna o último polígono registrado.

```
>>> turtle.home()
>>> turtle.begin_poly()
>>> turtle.fd(100)
```

```
>>> turtle.left(20)
>>> turtle.fd(30)
>>> turtle.left(60)
>>> turtle.fd(50)
>>> turtle.end_poly()
>>> p = turtle.get_poly()
>>> register_shape("myFavouriteShape", p)
```

turtle.clone()

Cria e retorna um clone da tartaruga com a mesma posição, direção e propriedades da tartaruga original.

```
>>> mick = Turtle()
>>> joe = mick.clone()
```

turtle.getturtle()

turtle.getpen()

Retorna o próprio objeto Turtle. Uso recomendado: como função para retornar a “tartaruga anônima”:

```
>>> pet = getturtle()
>>> pet.fd(50)
>>> pet
<turtle.Turtle object at 0x...>
```

turtle.getscreen()

Retorna o objeto `TurtleScreen` no qual a tartaruga está sendo desenhada. Os métodos de TurtleScreen podem então ser chamados para este objeto.

```
>>> ts = turtle.getscreen()
>>> ts
<turtle._Screen object at 0x...>
>>> ts.bgcolor("pink")
```

turtle.setundobuffer(*size*)

Parâmetros:: **size** – um inteiro ou `None`

Define ou desativa o undobuffer. Se *size* for um inteiro, um undobuffer vazio de determinado tamanho será instalado. *size* fornece o número

máximo de ações da tartaruga que podem ser desfeitas pelo método/função **undo()** . Se *size* for `None`, o undobuffer será desativado.

```
>>> turtle.setundobuffer(42)
```

turtle.undobufferentries()

Retorna o número de entradas no undobuffer.

```
>>> while undobufferentries():
...     undo()
```

Formas compostas

Para usar formas de tartaruga compostas, que consistem em vários polígonos de cores diferentes, você deve usar a classe auxiliar **Shape** explicitamente, conforme descrito abaixo:

1. Crie um objeto Shape vazio do tipo “compound”.

2. Adicione quantos componentes desejar a esse objeto, usando o método **addcomponent()**.

   Por exemplo:

   ```
   >>> s = Shape("compound")
   >>> poly1 = ((0,0),(10,-5),(0,10),(-10,-5))
   >>> s.addcomponent(poly1, "red", "blue")
   >>> poly2 = ((0,0),(10,-5),(-10,-5))
   >>> s.addcomponent(poly2, "blue", "red")
   ```

3. Agora, adicione o Shape à lista de formas da tela e use-o:

   ```
   >>> register_shape("myshape", s)
   >>> shape("myshape")
   ```

Nota

A classe **Shape** é usada internamente pelo método **register_shape()** de diferentes maneiras. O programador da aplicação precisa lidar com a classe Shape *somente* ao usar formas compostas como as mostradas acima!

Métodos e funções de janela

A maioria dos exemplos desta seção se refere a uma instância de TurtleScreen chamada `screen`.

Controle da Janela

turtle.bgcolor()
turtle.bgcolor(*color*, /)
turtle.bgcolor(*r*, *g*, *b*, /)

Retorna ou define a cor de fundo do TurtleScreen.

São permitidos quatro formatos de entrada:

`bgcolor()`
Retorna a cor de fundo atual como string de especificação de cor ou como uma tupla (veja o exemplo). Pode ser usado como entrada para outra chamada color/pencolor/fillcolor/bgcolor.

`bgcolor(colorstring)`
Define a cor de fundo como *colorstring*, que é uma string de especificação de cor Tk, como `"red"`, `"yellow"` ou `"#33cc8c"`.

`bgcolor((r, g, b))`
Define a cor de fundo como a cor RGB representada pela tupla *r*, *g*, e *b*. Os valores de *r*, *g*, and *b* precisam estar na faixa 0..colormode, onde colormode é 1.0 ou 255 (ver `colormode()`).

`bgcolor(r, g, b)`
Define a cor de fundo como a cor RGB representada por *r*, *g*, e *b*. Os valores de *r*, *g*, and *b* precisam estar na faixa 0..colormode.

```
>>> screen.bgcolor("orange")
>>> screen.bgcolor()
'orange'
>>> screen.bgcolor("#800080")
```

```
>>> screen.bgcolor()
(128.0, 0.0, 128.0)
```

turtle.bgpic(*picname=None*)

Parâmetros:: **picname** – um string, um nome de um arquivo de imagem (PNG, GIF, PGM e PPM) ou `"nopic"`, ou `None`

Configura a imagem de fundo ou retorna nome da imagem de fundo atual. Se *picname* for um nome de arquivo, definirá a imagem correspondente como plano de fundo. Se *picname* for `"nopic"`, exclui a imagem de fundo, caso haja. Se *picname* for `None`, retorna o nome do arquivo da imagem de fundo atual:

```
>>> screen.bgpic()
'nopic'
>>> screen.bgpic("landscape.gif")
>>> screen.bgpic()
"landscape.gif"
```

turtle.clear()

Nota

Este método de TurtleScreen está disponível como uma função global somente sob o nome `clearscreen`. A função global `clear` é derivada de forma diferente do método de Turtle `clear` .

turtle.clearscreen()

Remove todos os desenhos e todas as tartarugas da TurtleScreen. Redefine a TurtleScreen, agora vazia, para seu estado inicial: fundo branco, sem imagem de fundo, sem vínculos de eventos e rastreamento ativado.

turtle.reset()

Nota

Este método de TurtleScreen está disponível como uma função global somente com o nome `resetscreen`. A função global `reset` é outra

derivada do método de Turtle `reset` .

turtle.resetscreen()

Reinicia todas as Turtles em Screen para seu estado inicial.

turtle.screensize(*canvwidth=None*, *canvheight=None*, *bg=None*)

Parâmetros::

- **canvwidth** – positivo inteiro, a nova largura da tel em pixels
- **canvheight** – positivo inteiro, a nova altura da tela em pixels
- **bg** – Uma string com a cor ou uma tupla, a nova cor de fundo

Se nenhum argumento for fornecido, retorna o atual (canvaswidth, canvasheight). Caso contrário, redimensiona a tela em que as tartarugas estão desenhadas. Não altera a janela de desenho. Para observar as partes ocultas da tela, use as barras de rolagem. Com este método, é possível tornar visíveis as partes de um desenho que antes estavam fora da tela.

```
>>> screen.screensize()
(400, 300)
>>> screen.screensize(2000,1500)
>>> screen.screensize()
(2000, 1500)
```

Por exemplo, para procurar uma tartaruga que escapou por engano ;-)

turtle.setworldcoordinates(*llx*, *lly*, *urx*, *ury*)

Parâmetros::

- **llx** – um número representando a coordenada x do canto inferior esquerdo da tela
- **lly** – um número representando a coordenada y do canto inferior esquerdo da tela
- **urx** – um número representando a coordenada x d canto superior direito da tela
- **ury** – um número representando a coordenada y d canto superior direito da tela

Configura o sistema de coordenadas definido pelo usuário e muda para o modo “world”, se necessário. Isso executa um `screen.reset()`. Se o modo “world” já estiver ativo, todos os desenhos atuais serão redesenhados de acordo com as novas coordenadas.

ATENÇÃO: em sistemas de coordenadas definidos pelo usuário, os ângulos podem aparecer distorcidos.

```
>>> screen.reset()
>>> screen.setworldcoordinates(-50,-7.5,50,7.5)
>>> for _ in range(72):
...     left(10)
...
>>> for _ in range(8):
...     left(45); fd(2)   # a regular octagon
```

Controle da animação

turtle.no_animation()

Desabilita temporariamente a animação da tartaruga. O código escrito dentro do bloco `no_animation` não será animado; assim que o bloco de código for fechado, o desenho aparecerá.

```
>>> with screen.no_animation():
...     for dist in range(2, 400, 2):
...         fd(dist)
...         rt(90)
```

Adicionado na versão 3.14.

turtle.delay(*delay=None*)

Parâmetros:: **delay** – um número inteiro positivo

Define ou retorna o *delay* do desenho em milissegundos. (Esse é aproximadamente o intervalo de tempo entre duas atualizações consecutivas da tela). Quanto maior o atraso do desenho, mais lenta será a animação.

Argumentos opcionais:

```
>>> screen.delay()
10
>>> screen.delay(5)
>>> screen.delay()
5
```

turtle.tracer(*n=None*, *delay=None*)

Parâmetros::
- **n** – inteiro não-negativo
- **delay** – inteiro não-negativo

Ative ou desative a animação da tartaruga e defina o atraso para atualização dos desenhos. Se *n* for fornecido, apenas cada n-ésima atualização regular da tela será realmente executada. (Pode ser usado para acelerar o desenho de gráficos complexos.) Quando chamado sem argumentos, retorna o valor atualmente armazenado de n. O segundo argumento define o valor do atraso (consulte `delay()`).

```
>>> screen.tracer(8, 25)
>>> dist = 2
>>> for i in range(200):
...     fd(dist)
...     rt(90)
...     dist += 2
```

turtle.update()

Executa uma atualização do TurtleScreen. Para ser usado quando o rastreador estiver desligado.

Veja também o método RawTurtle/Turtle `speed()`.

Usando os eventos de tela

turtle.listen(*xdummy=None*, *ydummy=None*)

Define o foco no TurtleScreen (para coletar eventos-chave). Argumentos fictícios são fornecidos para que seja possível passar `listen()` para o método onclick.

turtle.onkey(*fun*, *key*)
turtle.onkeyrelease(*fun*, *key*)

Parâmetros::

- **fun** – uma função sem argumentos ou `None`
- **key** – uma string: uma tecla (por exemplo, “a”) ou o nome de uma tecla (por exemplo, “space”)

Vincula *fun* ao evento de liberação da tecla. Se *fun* for `None`, as ligações de eventos serão removidas. Observação: para poder registrar eventos-chave, o TurtleScreen deve ter o foco. (Veja o método **`listen()`**.)

```
>>> def f():
...     fd(50)
...     lt(60)
...
>>> screen.onkey(f, "Up")
>>> screen.listen()
```

turtle.onkeypress(*fun*, *key=None*)

Parâmetros::

- **fun** – uma função sem argumentos ou `None`
- **key** – uma string: uma tecla (por exemplo, “a”) ou o nome de uma tecla (por exemplo, “space”)

Vincula *fun* ao evento de pressionamento de tecla se a tecla for fornecida, ou a qualquer evento de pressionamento de tecla se nenhuma tecla for fornecida. Observação: para poder registrar eventos-chave, o TurtleScreen deve ter foco. (Veja o método **`listen()`**.)

```
>>> def f():
...     fd(50)
...
>>> screen.onkey(f, "Up")
>>> screen.listen()
```

turtle.onclick(*fun*, *btn=1*, *add=None*)

turtle.onscreenclick(*fun*, *btn=1*, *add=None*)

Parâmetros::

- **fun** – um função com dois argumento que serão chamados com as coordenadas do ponto clicado na tela
- **btn** – número do botão do mouse, o padrão é 1 (botão esquerdo do mouse)
- **add** – `True` ou `False` – se `True`, uma nova ligação será adicionada; caso contrário, ela substituirá uma ligação antiga

Vincula *fun* a eventos de clique do mouse na tela. Se *fun* for `None`, as associações existentes serão removidas.

Exemplo de uma instância de TurtleScreen chamada `screen` e uma instância de Turtle chamada `turtle`:

```
>>> screen.onclick(turtle.goto) # Em seguida, clicar no Turtle
>>>                             # fará a tartaruga se mover at
>>> screen.onclick(None)        # remove associação de evento
```

Nota

Este método TurtleScreen está disponível como uma função global somente com o nome `onscreenclick`. A função global `onclick` é outra derivada do método de Turtle `onclick` .

turtle.ontimer(*fun*, *t=0*)

Parâmetros::

- **fun** – um função sem nenhum argumento
- **t** – um número >= 0

Instala um cronômetro que chama *fun* após *t* milissegundos.

```
>>> running = True
>>> def f():
...     if running:
...         fd(50)
...         lt(60)
...         screen.ontimer(f, 250)
>>> f()   ### faz a tartaruga marchar
>>> running = False
```

turtle.mainloop()

turtle.done()

Inicia o loop de eventos - chamando a função principal do loop do Tkinter. Deve ser a última instrução em um programa gráfico de tartaruga. *Não* deve ser usado se um script for executado no IDLE no modo -n (sem subprocesso) - para uso interativo de gráficos de tartaruga.

```
>>> screen.mainloop()
```

Métodos de entrada

turtle.textinput(*title*, *prompt*)

Parâmetros::
- **title** – string
- **prompt** – string

Abre uma janela de diálogo para a entrada de uma string. O parâmetro *title* é o título da janela de diálogo, prompt é um texto que descreve as informações a serem inseridas. Se a caixa de diálogo for preenchida, retorna a string que foi informada. Se a caixa de diálogo for cancelada, retorna `None`.

```
>>> screen.textinput("NIM", "Name of first player:")
```

turtle.numinput(*title*, *prompt*, *default=None*, *minval=None*, *maxval=None*)

Parâmetros::
- **title** – string
- **prompt** – string
- **default** – número (opcional)
- **minval** – número (opcional)
- **maxval** – número (opcional)

Abre uma janela de diálogo para a entrada de um número. O parâmetro *title* é o título da janela de diálogo, prompt é um texto que descreve principalmente quais informações numéricas devem ser inseridas. default: valor padrão , minval: mínimo valor para entrada, maxval: máximo valor para entrada. O número inserido deve estar no intervalo minval .. maxval, se este for fornecido. Caso contrário, será emitida uma dica e a caixa de diálogo permanecerá aberta para correção. Se a caixa de diálogo for

preenchida, retorna o número informado. Se a caixa de diálogo for cancelada, retorna `None`.

```
>>> screen.numinput("Poker", "Your stakes:", 1000, minval=10,
```

Configurações e métodos especiais

turtle.mode(*mode=None*)

Parâmetros:: **mode** – uma das strings “standard”, “logo” ou “world”

Define o modo da tartaruga (“padrão”, “logotipo” ou “mundo”) e redefine a posição. Se o modo não for fornecido, o modo atual será retornado.

O modo “standard” é compatível com o antigo **`turtle`**. O modo “logo” é compatível com a maioria dos gráficos de tartaruga. O modo “world” usa “coordenadas mundiais” definidas pelo usuário. **Atenção**: nesse modo, os ângulos aparecem distorcidos se a proporção de `x/y` não for igual a 1.

Modo	Título inicial da tartaruga	ângulos positivos
“standard”	para a direita (east)	counterclockwise
“logo”	upward (north)	sentido horário

```
>>> mode("logo")   # resets turtle heading to north
>>> mode()
'logo'
```

turtle.colormode(*cmode=None*)

Parâmetros:: **cmode** – um dos valores 1.0 ou 255

Retorna o modo de cor ou defini-lo como 1,0 ou 255. Posteriormente, os valores *r*, *g*, *b* das triplas de cores devem estar no limite 0..*cmode*.

```
>>> screen.colormode(1)
>>> turtle.pencolor(240, 160, 80)
Traceback (most recent call last):
```

```
     ...
TurtleGraphicsError: bad color sequence: (240, 160, 80)
>>> screen.colormode()
1.0
>>> screen.colormode(255)
>>> screen.colormode()
255
>>> turtle.pencolor(240,160,80)
```

turtle.getcanvas()

Retorna a tela desse TurtleScreen. Útil para pessoas que sabem o que fazer com uma tela do Tkinter.

```
>>> cv = screen.getcanvas()
>>> cv
<turtle.ScrolledCanvas object ...>
```

turtle.getshapes()

Retorna uma lista dos nomes de todas as formas de tartarugas disponíveis no momento.

```
>>> screen.getshapes()
['arrow', 'blank', 'circle', ..., 'turtle']
```

turtle.register_shape(*name*, *shape=None*)

turtle.addshape(*name*, *shape=None*)

Há quatro maneiras diferentes de chamar esta função:

1. *name* é o nome de um arquivo de imagem (PNG, GIF, PGM e PPM) e *shape* é `None`: Instala a forma de imagem correspondente.

   ```
   >>> screen.register_shape("turtle.gif")
   ```

   Nota

   As formas de imagem *não* giram ao girar a tartaruga, portanto, elas não exibem o rumo da tartaruga!

2. *name* é uma string arbitrária e *shape* é o nome de um arquivo de imagem (PNG, GIF, PGM e PPM): Instala a forma de imagem

correspondente.

```
>>> screen.register_shape("turtle", "turtle.gif")
```

Nota

As formas de imagem *não* giram ao girar a tartaruga, portanto, elas não exibem o rumo da tartaruga!

3. *name* é uma string arbitrária e *shape* é um tupla de pares de coordenadas: Instala a forma de polígono correspondente.

```
>>> screen.register_shape("triangle", ((5,-3), (0,5), (-5,
```

4. *name* é uma string arbitrária e *shape* é um objeto (composto) `Shape`: Instala a forma composta correspondente.

Adiciona uma forma de tartaruga à lista de formas do TurtleScreen. Somente as formas registradas dessa forma podem ser usadas com o comando `shape(shapename)`.

Alterado na versão 3.14: Adicionado suporte para os formatos de imagem PNG, PGM e PPM. É possível especificar tanto o nome da forma quanto o nome do arquivo de imagem.

turtle.turtles()

Retorne uma lista de tartarugas na tela.

```
>>> for turtle in screen.turtles():
...     turtle.color("red")
```

turtle.window_height()

Retorna a altura da janela da tartaruga.

```
>>> screen.window_height()
480
```

turtle.window_width()

Retorna a largura da janela da tartaruga.

```
>>> screen.window_width()
640
```

Métodos apenas de janela

turtle.bye()

Fecha a janela do gráficos de tartaruga.

turtle.exitonclick()

Vincula o método `bye()` aos cliques do mouse na tela.

Se o valor “using_IDLE” na configuração dicionário for `False` (valor padrão), entra também no loop principal. Observação: Se IDLE for executado com a chave `-n` (sem subprocesso), esse valor deverá ser definido como `True` em `turtle.cfg`. Nesse caso, o loop principal do próprio IDLE também estará ativo para o script do cliente.

turtle.save(*filename*, *overwrite=False*)

Salva o desenho atual da tartaruga (e tartarugas) como um arquivo PostScript.

Parâmetros::

- **filename** – o caminho do arquivo PostScript salvo
- **overwrite** – Se for `False` e já existir um arquivo com o nome fornecido, a função levantará uma `FileExistsError`. Se for `True`, o arquivo será sobrescrito.

```
>>> screen.save("my_drawing.ps")
>>> screen.save("my_drawing.ps", overwrite=True)
```

Adicionado na versão 3.14.

turtle.setup(*width=_CFG['width']*, *height=_CFG['height']*, *startx=_CFG['leftright']*, *starty=_CFG['topbottom']*)

Define o tamanho e a posição da janela principal. Os valores padrões dos argumentos são armazenados na configuração dicionário e podem ser alterados por meio de um arquivo `turtle.cfg`.

Parâmetros::

- **width** – se for um inteiro, o tamanho em pixels; se for um float, uma fração da tela; o padrão é 50% d tela
- **height** – se for um inteiro, a altura em pixels; se fo um float, uma fração da tela; o padrão é 75% da tela
- **startx** – se positivo, este parâmetro define a posição inicial em pixels a partir da borda esquerd da tela; se negativo, define a partir da borda direita se `None`, centraliza a janela horizontalmente
- **starty** – se positivo, este parâmetro define a posição inicial, em pixels, a partir da borda superior da tela; se negativo, define a partir da borda inferior; se for `None`, centraliza a janela verticalmente

```
>>> screen.setup (width=200, height=200, startx=0, starty=0)
>>>              # define a janela para 200x200 pixels, no can
>>> screen.setup(width=.75, height=0.5, startx=None, starty=No
>>>              # define a janela para 75% da tela por 50% da
```

turtle.title(*titlestring*)

Parâmetros:: **titlestring** – uma string que é exibida na barra de título da janela de gráficos da tartaruga

Define o título da janela da tartaruga como *titlestring*.

```
>>> screen.title("Welcome to the turtle zoo!")
```

Classes Públicas

class turtle.RawTurtle(*canvas*)

class turtle.RawPen(*canvas*)

Parâmetros:: **canvas** – um **`tkinter.Canvas`**, um **`ScrolledCanvas`** ou um **`TurtleScreen`**

Cria uma tartaruga. A tartaruga tem todos os métodos descritos acima como “métodos de Turtle/RawTurtle”.

class turtle.Turtle

Subclasse do RawTurtle, tem a mesma interface, mas desenha em um objeto padrão `Screen`, que é criado automaticamente, quando necessário, pela primeira vez.

class turtle.TurtleScreen(*cv*)

Parâmetros:: **cv** – uma `tkinter.Canvas`

Fornece métodos orientado para a tela, como `bgcolor()` etc., que são descritos acima.

class turtle.Screen

Subclasse do TurtleScreen, com quatro métodos adicionados.

class turtle.ScrolledCanvas(*master*)

Parâmetros:: **master** – algum widget do Tkinter para conter o ScrolledCanvas. Exemplo: um Tkinter-canvas com barras de rolagem adicionadas

Usado pela classe Screen, que, portanto, fornece automaticamente um ScrolledCanvas como playground para as tartarugas.

class turtle.Shape(*type_*, *data*)

Parâmetros:: **type_** – uma das strings “polygon”, “image”, “compound”

Estrutura de dados para modelar a forma. O par `(type_, data)` deve seguir essa especificação:

type_	***data***
“polygon”	Uma tupla com o par de coordenadas do polígono
“image”	uma imagem (nesse formato, usada apenas internamente!)

*type_*	*data*
"compound"	`None` (uma forma composta deve ser construída usando o método **`addcomponent()`**)

addcomponent(*poly*, *fill*, *outline=None*)

Parâmetros::

- **poly** – um polígono, ou seja, um tupla de pares de números
- **fill** – uma cor com a qual o *poly* será preenchido
- **outline** – uma cor para o contorno do polígono (se fornecido)

Exemplo:

```
>>> poly = ((0,0),(10,-5),(0,10),(-10,-5))
>>> s = Shape("compound")
>>> s.addcomponent(poly, "red", "blue")
>>> # ... adicione mais componentes e então use register_sh
```

Veja Formas compostas.

class turtle.Vec2D(*x*, *y*)

Uma classe de vetor bidimensional, usada como classe auxiliar para implementar gráficos de tartaruga. Pode ser útil também para programas de gráficos de tartaruga. Derivado de uma tupla, portanto, um vetor é um tupla!

Fornece (para vetores *a*, *b*, número *k*):

- `a + b` vetor adicional
- `a - b` subtração de vetor
- `a * b` produto interno
- `k * a` e `a * k` multiplicação com escalar
- `abs(a)` valor absoluto de um
- rotação `a.rotate(angle)`

Exceções

O módulo **`turtle`** define a seguinte exceção:

exception turtle.TurtleGraphicsError

Levantada para argumentos ou operações inválidas. Por exemplo, uma string de cor malformada:

```
>>> turtle.color("blau")
Traceback (most recent call last):
    ...
turtle.TurtleGraphicsError: bad color string: blau
```

Guias de como fazer

Esta seção aborda alguns casos de uso e abordagens típicos do módulo “turtle”.

Inicie e encerre o preenchimento automaticamente

A partir do Python 3.14, você pode usar o gerenciador de contexto **`fill()`** em vez de **`begin_fill()`** e **`end_fill()`** para iniciar e encerrar o preenchimento automaticamente. Aqui está um exemplo:

```
with fill():
    for i in range(4):
        forward(100)
        right(90)

forward(200)
```

O código acima equivale a:

```
begin_fill()
for i in range(4):
    forward(100)
    right(90)
end_fill()

forward(200)
```

Use o espaço de nomes do módulo `turtle`

Usar `from turtle import *` é conveniente, mas lembre-se de que ele importa um coleção bastante grande de objetos e, se você estiver fazendo qualquer coisa no seu código que não esteja relacionada com o módulo gráficos de tartaruga, corre o risco de um conflito de nomes de objetos (isso se torna um problema ainda maior se você estiver usando o módulo de gráficos de tartaruga em um script em que outros módulos possam ser também importados).

A solução é usar o identificador/domínio do próprio módulo `import turtle` sem o asterisco no final da importação - assim a chamada do objeto/função `fd()` torna-se `turtle.fd()`, da mesma forma como `width()` torna-se `turtle.width()` e assim por diante. (Se digitar “turtle” várias vezes se tornar tedioso, use `import turtle as t`, por exemplo, para fornecer um apelido mais conciso e objetivo ao identificador/domínio dentro do seu código).

Use o módulo gráfico de tartaruga dentro de um script com código específico

Recomenda-se usar o módulo `turtle` com uma apelido para identificador/domínio, conforme descrito acima, por exemplo:

```
import turtle as t
from random import random

for i in range(100):
    steps = int(random() * 100)
    angle = int(random() * 360)
    t.right(angle)
    t.fd(steps)
```

No entanto, outra etapa também é necessária ou o Python também fechará a janela da tartaruga assim que o código acima terminar de ser executado. Adicione:

```
t.mainloop()
```

ao final do código anterior. Assim, o código anterior agora conterá uma instrução que aguardará ser dispensado e não sairá até que seja encerrado, por exemplo, fechando a janela aberta pelo módulo gráficos da tartaruga.

Use o módulo gráficos da tartaruga orientado a objetos

Ver também

Explicação de interface orientada a objetos

Exceto para fins introdutórios muito básicos ou para experimentar coisas o mais rápido possível, é mais comum e muito mais eficiente usar a abordagem orientada a objetos para gráficos de tartarugas. Por exemplo, isso permite várias tartarugas na tela ao mesmo tempo.

Nessa abordagem, os vários comandos do módulo turtle são métodos de objetos (principalmente objetos de `Turtle`). Você *pode* usar a abordagem orientada a objetos em console, mas ela seria mais típica em um script Python.

O exemplo acima se torna então:

```
from turtle import Turtle
from random import random

t = Turtle()
for i in range(100):
    steps = int(random() * 100)
    angle = int(random() * 360)
    t.right(angle)
    t.fd(steps)

t.screen.mainloop()
```

Observe a última linha. `t.screen` é um instância da subclasse **`Screen`** que existe numa instância da classe tartaruga ou “turtle”; ela é criada automaticamente junto com a instância de tartaruga.

A tela da tartaruga pode ser personalizada, por exemplo:

```
t.screen.title('Object-oriented turtle demo')
t.screen.bgcolor("orange")
```

Como usar a Ajuda

Os métodos públicos das classes Screen e Turtle estão amplamente documentado por meio de docstrings. Eles podem ser usados como ajuda on-line por meio dos recursos de ajuda do Python:

- Ao usar o IDLE, as dicas de ferramentas mostram as assinaturas e as primeiras linhas das docstrings das chamadas de função/método digitadas.

- Chamar **`help()`** em métodos ou funções exibe o seguinte docstring:

```
>>> help(Screen.bgcolor)
Help on method bgcolor in module turtle:

bgcolor(self, *args) unbound turtle.Screen method
    Set or return backgroundcolor of the TurtleScreen.

    Arguments (if given): a color string or three numbers
    in the range 0..colormode or a 3-tuple of such numbers.


    >>> screen.bgcolor("orange")
    >>> screen.bgcolor()
    "orange"
    >>> screen.bgcolor(0.5,0,0.5)
    >>> screen.bgcolor()
    "#800080"

>>> help(Turtle.penup)
Help on method penup in module turtle:

penup(self) unbound turtle.Turtle method
    Pull the pen up -- no drawing when moving.

    Aliases: penup | pu | up

    No argument

    >>> turtle.penup()
```

- Os docstrings de funções que são derivados do métodos têm um formato diferente:

```
>>> help(bgcolor)
Help on function bgcolor in module turtle:

bgcolor(*args)
    Set or return backgroundcolor of the TurtleScreen.

    Arguments (if given): a color string or three numbers
    in the range 0..colormode or a 3-tuple of such numbers.

    Example::

      >>> bgcolor("orange")
      >>> bgcolor()
      "orange"
      >>> bgcolor(0.5,0,0.5)
      >>> bgcolor()
      "#800080"

>>> help(penup)
Help on function penup in module turtle:

penup()
    Pull the pen up -- no drawing when moving.

    Aliases: penup | pu | up

    No argument

    Example:
    >>> penup()
```

Esses docstrings modificados são criados automaticamente junto com as definições de função que são derivadas de métodos ao importar.

Tradução de docstrings em diferentes idiomas

Há um utilitário para criar a dicionário cujas chaves são os nomes de método e cujos valores são os docstrings dos métodos públicos das classes Screen e Turtle.

turtle.write_docstringdict(*filename='turtle_docstringdict'*)

Parâmetros:: **filename** – uma string, usado como nome de arquivo

Cria e escreve um dicionário de docstring em um script Python com o nome de arquivo fornecido. Esta função deve ser chamada explicitamente (ela não é usada pelas classes gráficas da tartaruga). O dicionário docstring será gravado no script Python `filename.py`. O objetivo é servir como modelo para traduzir os documentos para diferentes idiomas.

Se você (ou seus alunos) quiserem usar o **`turtle`** com a ajuda on-line em seu idioma nativo, será necessário traduzir os docstrings e salvar o arquivo resultante como, por exemplo, `turtle_docstringdict_german.py`.

Se você tiver uma entrada específica em seu arquivo `turtle.cfg`, esse dicionário será lido no momento do importar e substituirá as documentações originais em inglês.

No momento da redação deste texto, há docstring dicionários em alemão e italiano. (Pedidos devem ser enviados para glingl@aon.at.)

Como configurar Screen and Turtles

A configuração padrão integrada imita a aparência e o comportamento do antigo módulo Turtle para manter a melhor compatibilidade possível com ele.

Se quiser usar uma configuração diferente que reflita melhor o recurso deste módulo ou que se adapte melhor às suas necessidades, por exemplo, para uso em uma sala de aula, você pode criar um arquivo de configuração `turtle.cfg` que será lido no momento de importar e modifica a configuração de acordo com suas definições.

A configuração que definida no arquivo `turtle.cfg`:

```
width = 0.5
height = 0.75
leftright = None
topbottom = None
canvwidth = 400
```

```
canvheight = 300
mode = standard
colormode = 1.0
delay = 10
undobuffersize = 1000
shape = classic
pencolor = black
fillcolor = black
resizemode = noresize
visible = True
language = english
exampleturtle = turtle
examplescreen = screen
title = Python Turtle Graphics
using_IDLE = False
```

Breve explicação das entradas selecionadas:

- As quatro primeiras linhas correspondem aos argumentos do método **`Screen.setup`**.
- As linhas 5 e 6 correspondem a argumento do método **`Screen.screensize`**.
- *shape* pode ser qualquer uma das formas pré-definidas, como por exemplo: seta, tartaruga, etc. Para obter mais informações, consulte `help(shape)`.
- Se você não quiser usar nenhuma cor de preenchimento (ou seja, tornar a tartaruga transparente), você deve usar `fillcolor = ""` (todas as strings não vazias não devem ter aspas no arquivo cfg).
- Se você quiser refletir o estado da tartaruga, você deve usar `resizemode = auto`.
- Se você definir, por exemplo, `language = italian`, o docstringdict `turtle_docstringdict_italian.py` será carregado ao importar (se estiver presente no caminho de importação, por exemplo, no mesmo diretório que **`turtle`**).
- As entradas *exampleturtle* e *examplescreen* definem os nomes desses objetos conforme ocorrem nas docstrings. A transformação de "método-docstrings" para "função-docstrings" excluirá esses nomes dos docstrings.
- *using_IDLE*: Defina isso como `True` se você trabalha regularmente com IDLE e sua chave `-n` ("sem subprocesso"). Isso impedirá que o

exitonclick() entre no loop principal.

Pode haver um arquivo `turtle.cfg` no diretório em que o **`turtle`** está armazenado e um arquivo adicional no diretório de trabalho atual. O arquivo armazenado no diretório de trabalho atual tem preferência e vai substituir as configurações do primeiro.

O diretório `Lib/turtledemo` contém um arquivo chamado `turtle.cfg`. Você pode usá-lo como um exemplo e ver seus efeitos ao executar as demonstrações (de preferência, não utilize visualizador de demonstrações).

Explicação

Um objeto tartaruga é baseado em um objeto canvas, e há uma série de classes principais na interface orientada a objetos da tartaruga que podem ser usadas para criá-los e relacioná-los entre si.

Uma instância da classe **`Turtle`** cria automaticamente uma nova instância de **`Screen`**, caso esta não esteja presente.

`Turtle` é um subclasse de **`RawTurtle`**, que *não* cria automaticamente uma superfície de desenho - uma *tela* precisará ser fornecida ou criada para ele. A *tela* pode ser uma **`tkinter.Canvas`**, **`ScrolledCanvas`** ou **`TurtleScreen`**.

`TurtleScreen` é a superfície de desenho básica para uma tartaruga. **`Screen`** é uma subclasse de `TurtleScreen` e inclui alguns métodos adicionais para gerenciar sua aparência (incluindo tamanho e título) e comportamento. O construtor `TurtleScreen` precisa de um **`tkinter.Canvas`** ou um **`ScrolledCanvas`** como um argumento.

A interface funcional para gráficos de tartaruga usa os vários métodos de `Turtle` e `TurtleScreen`/`Screen`. Nos bastidores, um objeto de tela é criado automaticamente sempre que um função derivada de um método de `Screen` é chamado. Da mesma forma, um objeto de tartaruga é criado automaticamente sempre que qualquer uma das funções derivadas de um método de tartaruga é chamado.

Para usar várias tartarugas em uma tela, a interface orientada a objetos deve ser usada.

`turtledemo` — Scripts de Demonstração

O pacote **`turtledemo`** inclui um conjunto de scripts de demonstração. Esses scripts podem ser executados e visualizados usando o visualizador de demonstração fornecido da seguinte forma:

```
python -m turtledemo
```

Como alternativa, você pode executar os scripts de demonstração individualmente. Por exemplo,

```
python -m turtledemo.bytedesign
```

O diretório do pacote **`turtledemo`** contém:

- Um arquivo de exemplo `__main__.py` que pode ser usado para visualizar o código-fonte dos scripts e executá-los ao mesmo tempo.
- Vários scripts que demonstram diferentes recursos do módulo **`turtle`**. Os exemplos podem ser acessados no menu Exemplos. Eles também podem ser executados de forma isolada.
- Um arquivo `turtle.cfg` que serve como exemplo de como escrever e usar esse tipo de arquivos.

Os scripts de demonstração são:

Nome	Descrição	Recursos
`bytedesign`	Padrão de gráficos de tartaruga clássico complexo	`tracer()`, `delay()`, `update()`

Nome	Descrição	Recursos
`chaos`	gráficos do modelo de Verhulst, mostram que os cálculos do computador podem gerar resultados às vezes contra as expectativas do bom senso	coordenadas mundiais
`clock`	Relógio analógico que mostra o horário do seu computador	tartarugas como ponteiros de relógio, `ontimer()`
`colormixer`	experimento com r, g, b	`ondrag()`
`forest`	3 breadth-first trees	randomização
`fractalcurves`	Curvas de Hilbert & Koch	recursão
`lindenmayer`	ethnomathematics (indian kolams)	L-System
`minimal_hanoi`	Torres de Hanoi	Tartarugas retângulos como discos de Hanói (`shape()`, `shapesize()`)

Nome	Descrição	Recursos
`nim`	jogue o clássico jogo nim com três montes de gravetos contra o computador.	tartarugas como gravetos, acionadas por eventos (mouse, teclado)
`paint`	programa de desenho super minimalista	`onclick()`
`peace`	elementar	tartaruga: aparência e animação
`penrose`	ladrilhos irregulares com pipas e dardos	`stamp()`
`planet_and_moon`	simulação de um sistema gravitacional	formas compostas, `Vec2D`
`rosette`	um padrão do artigo Wikipédia sobre gráficos de tartaruga	`clone()`, `undo()`
`round_dance`	tartarugas dançantes girando em pares na direção oposta	formas compostas, `clone()` `shapesize()`, `tilt()`, `get_shapepoly()`, `update()`

Nome	Descrição	Recursos
`sorting_animate`	demonstração visual de diferentes tipos de métodos de ordenação	alinhamento simples, randomização
`tree`	uma árvore (gráfica) da largura inicial (usando geradores)	`clone()`
`two_canvases`	desenho simples	tartarugas em duas telas
`yinyang`	outro exemplo elementar	`circle()`

Divirta-se!

Ferramentas de Desenvolvimento

Os módulos descritos neste capítulo ajudam você a escrever softwares. Por exemplo, o módulo `pydoc` recebe um módulo e gera documentação com base no conteúdo do módulo. Os módulos `doctest` e `unittest` contêm frameworks para escrever testes unitários que automaticamente exercitam código e verificam se a saída esperada é produzida.

A lista de módulos descritos neste capítulo é:

- `typing` — Suporte para dicas de tipo
  - Especificação para o sistema de tipos do Python
  - Apelidos de tipo
  - NewType
  - Anotações de objetos chamáveis
  - Genéricos
  - Anotando tuplas
  - O tipo de objetos de classe
  - Anotando geradores e corrotinas
  - Tipos genéricos definidos pelo usuário
  - O tipo `Any`
  - Subtipagem nominal vs estrutural
  - Conteúdo do módulo
    - Tipos primitivos especiais
      - Tipos especiais
      - Formas especiais
      - Criando tipos genéricos e apelidos de tipo
      - Outras diretivas especiais
    - Protocolos
    - ABCs e protocolos para trabalhar com E/S
    - Funções e decoradores
    - Auxiliares de introspecção
    - Constante
    - Apelidos descontinuados
      - Apelidos de tipos embutidos

        - Apelidos para tipos em `collections`
        - Apelidos de outros tipos concretos
        - Apelidos de contêineres ABC em `collections.abc`.
        - Apelidos para ABCs assíncronas em `collections.abc`
        - Apelidos para outros ABCs em `collections.abc`.
        - Apelidos de ABCs da `contextlib`
    - Cronograma de descontinuação dos principais recursos
- `pydoc` — Gerador de documentação e sistema de ajuda online
- Modo de Desenvolvimento do Python
    - Efeitos do Modo de Desenvolvimento do Python
    - Exemplo de ResourceWarning
    - Exemplo de erro de descritor de arquivo inválido
- `doctest` — Teste exemplos interativos de Python
    - Uso simples: verificando exemplos em Docstrings
    - Utilização comum: Verificando exemplos em um arquivo texto
    - Uso na linha de comando
    - Como ele funciona
        - Quais docstrings são examinadas?
        - Como os exemplos de docstrings são reconhecidos?
        - Qual é o contexto de execução?
        - E quanto às exceções?
        - Flags opcionais
        - Directives
        - Avisos
    - Basic API
    - API do Unittest
    - Advanced API
        - DocTest Objects
        - Example Objects
        - DocTestFinder objects
        - DocTestParser objects
        - TestResults objects
        - DocTestRunner objects
        - OutputChecker objects
    - Depuração
    - Soapbox
- `unittest` — Framework de testes unitários

- Exemplo Básico
- Interface de Linha de Comando
  - Opções de linha de comando
- Test Discovery
- Organizando código teste
- Reutilizando códigos de teste antigos
- Ignorando testes e falhas esperadas
- Distinguindo iterações de teste utilizando subtestes
- Classes e funções
  - Casos de teste
  - Grouping tests
  - Carregando e executando testes
    - load_tests Protocol
- Classes e Módulos de Definição de Contexto
  - setUpClass and tearDownClass
  - setUpModule and tearDownModule
- Tratamento de sinal

- `unittest.mock` — mock object library
  - Guia Rápido
  - A classe Mock
    - Fazendo chamadas
    - Deletando Atributos
    - Nomes de Mock e o atributo *name*
    - Anexando Mocks como Atributos
  - Os criadores de patches
    - patch
    - patch.object
    - patch.dict
    - patch.multiple
    - métodos do patch: *start* e *stop*
    - patch de embutidos
    - TEST_PREFIX
    - Aninhando Decoradores Patch
    - Onde fazer patch
    - Patching Descriptors and Proxy Objects
  - MagicMock and magic method support
    - Simulando Métodos Mágicos

        - Magic Mock
    - Ajudantes
        - sentinel
        - DEFAULT
        - chamada
        - create_autospec
        - ANY
        - FILTER_DIR
        - mock_open
        - Especificação automática
        - Vedando mocks
    - Order of precedence of `side_effect`, `return_value` and *wraps*
- `unittest.mock` — primeiros passos
    - Usando Mock
        - Mock patching methods
        - Mock for method calls on an object
        - Mocking classes
        - Nomeando os mocks
        - Tracking all calls
        - Setting return values and attributes
        - Raising exceptions with mocks
        - Side effect functions and iterables
        - Mocking asynchronous iterators
        - Mocking asynchronous context manager
        - Creating a mock from an existing object
        - Using side_effect to return per file content
    - Patch decorators
    - Further examples
        - Mocking chained calls
        - Partial mocking
        - Mocking a generator method
        - Applying the same patch to every test method
        - Mocking unbound methods
        - Checking multiple calls with mock
        - Coping with mutable arguments
        - Nesting patches
        - Mocking a dictionary with MagicMock

        - Mock subclasses and their attributes
        - Mocking imports with patch.dict
        - Tracking order of calls and less verbose call assertions
        - More complex argument matching
- `test` — Pacote de Testes de Regressão do Python
    - Writing Unit Tests for the `test` package
    - Executando testes usando a interface de linha de comando
- `test.support` — Utilities for the Python test suite
- `test.support.socket_helper` — Utilities for socket tests
- `test.support.script_helper` — Utilities for the Python execution tests
- `test.support.bytecode_helper` — Support tools for testing correct bytecode generation
- `test.support.threading_helper` — Utilities for threading tests
- `test.support.os_helper` — Utilities for os tests
- `test.support.import_helper` — Utilities for import tests
- `test.support.warnings_helper` — Utilities for warnings tests

`typing` — Suporte para dicas de tipo

Adicionado na versão 3.5.

Código-fonte: Lib/typing.py [https://github.com/python/cpython/tree/3.15/Lib/typing.py]

> **Nota**
>
> O ambiente de execução do Python não força anotações de tipos de variáveis e funções. Elas podem ser usadas por ferramentas de terceiros como verificadores de tipo, IDEs, linters, etc.

Este módulo fornece suporte em tempo de execução para dicas de tipo.

Considere a função abaixo:

```
def surface_area_of_cube(edge_length: float) -> str:
    return f"A área superficial de um cubo é {6 * edge_length **
```

A função `surface_area_of_cube` recebe um argumento que se espera ser uma instância de **`float`**, conforme indicado pela dica de tipo `edge_length: float`. Espera-se que a função retorne uma instância de **`str`**, conforme indicado pela dica `-> str`.

Embora as dicas de tipo possam ser classes simples como **`float`** ou **`str`**, elas também podem ser mais complexas. O módulo **`typing`** fornece um vocabulário de dicas de tipo mais avançadas.

Novos recursos são frequentemente adicionados ao módulo `typing`. O pacote typing_extensions [https://pypi.org/project/typing_extensions/] fornece backports desses novos recursos para versões mais antigas do Python.

Ver também

Folha de dicas sobre tipagem
[https://mypy.readthedocs.io/en/stable/cheat_sheet_py3.html]
Uma visão geral das dicas de tipo (hospedado por mypy docs, em inglês).

Seção de referência do sistema de tipos da documentação do mypy
[https://mypy.readthedocs.io/en/stable/index.html]
O sistema de tipagem do Python é padronizado pelas PEPs, portanto esta referência deve se aplicar a maioria do verificadores de tipo do Python. (Alguns trechos podem se referir especificamente ao mypy. Documento em inglês).

Static Typing with Python **[https://typing.python.org/en/latest/]**
Documentação independente de verificador de tipo escrita pela comunidade, detalhando os recursos do sistema de tipo, ferramentas úteis de tipagem e melhores práticas.

Especificação para o sistema de tipos do Python

A especificação canônica e atualizada do sistema de tipos Python pode ser encontrada em Specification for the Python type system [https://typing.python.org/en/latest/spec/index.html].

Apelidos de tipo

Um apelido de tipo é definido utilizando a instrução `type`, que por sua vez cria uma instância da classe `TypeAliasType`. Neste exemplo, `Vector` e `list[float]` serão tratados de maneira equivalente pelos verificadores de tipo estático:

```
type Vector = list[float]

def scale(scalar: float, vector: Vector) -> Vector:
    return [scalar * num for num in vector]
```

```
# passa na verificação de tipos; uma lista de floats é qualificad
new_vector = scale(2.0, [1.0, -4.2, 5.4])
```

Apelidos de tipo são úteis para simplificar assinaturas de tipo complexas. Por exemplo:

```
from collections.abc import Sequence

type ConnectionOptions = dict[str, str]
type Address = tuple[str, int]
type Server = tuple[Address, ConnectionOptions]

def broadcast_message(message: str, servers: Sequence[Server]) ->
    ...

# O verificador de tipo tratará a assinatura de tipo anterior
# como exatamente equivalente à assinatura abaixo.
def broadcast_message(
    message: str,
    servers: Sequence[tuple[tuple[str, int], dict[str, str]]]
) -> None:
    ...
```

A instrução `type` é nova no Python 3.12. Para manter retrocompatibilidade, apelidos de tipo também podem ser criados usando atribuição:

```
Vector = list[float]
```

Ou marcado com `TypeAlias` para tornar explícito que se trata de um apelido de tipo e não uma atribuição de variável comum:

```
from typing import TypeAlias

Vector: TypeAlias = list[float]
```

NewType

Utilize o auxiliar `NewType` para criar tipos únicos:

```
from typing import NewType

UserId = NewType('UserId', int)
some_id = UserId(524313)
```

O verificador de tipo estático tratará o novo tipo como se fosse uma subclasse do tipo original. Isso é útil para ajudar a encontrar erros de lógica:

```
def get_user_name(user_id: UserId) -> str:
    ...

# passa na verificação de tipo
user_a = get_user_name(UserId(42351))

# falha na verificação de tipo: um int não é um UserId
user_b = get_user_name(-1)
```

Você ainda pode executar todas as operações `int` em uma variável do tipo `UserId`, mas o resultado sempre será do tipo `int`. Isso permite que você passe um `UserId` em qualquer ocasião que `int` possa ser esperado, mas previne que você acidentalmente crie um `UserId` de uma forma inválida:

```
# 'output' é do tipo 'int', em vez de 'UserId'
output = UserId(23413) + UserId(54341)
```

Note que essas verificações são aplicadas apenas pelo verificador de tipo estático. Em tempo de execução, a instrução `Derived = NewType('Derived', Base)` irá tornar `Derived` um chamável que retornará imediatamente qualquer parâmetro que você passar. Isso significa que a expressão `Derived(some_value)` não cria uma nova classe ou introduz sobrecarga além de uma chamada regular de função.instrução

Mais precisamente, a expressão `some_value is Derived(some_value)` é sempre verdadeira em tempo de execução.

É inválido criar um subtipo de `Derived`:

```
from typing import NewType

UserId = NewType('UserId', int)
```

```
# Falha em tempo de execução e não passa na verificação de tipos
class AdminUserId(UserId): pass
```

No entanto, é possível criar um **`NewType`** baseado em um ‘derivado’ `NewType`:

```
from typing import NewType

UserId = NewType('UserId', int)

ProUserId = NewType('ProUserId', UserId)
```

e a verificação de tipos para `ProUserId` funcionará como esperado.

Veja **PEP 484** [https://peps.python.org/pep-0484/] para mais detalhes.

Nota

Lembre-se que o uso de um apelido de tipo declara que dois tipos serão *equivalentes* entre si. Efetuar `type Alias = Original` fará o verificador de tipo estático tratar `Alias` como sendo *exatamente equivalente* a `Original` em todos os casos. Isso é útil quando você deseja simplificar assinaturas de tipo complexas.

Em contraste, `NewType` declara que um tipo será *subtipo* de outro. Efetuando `Derived = NewType('Derived', Original)` irá fazer o verificador de tipo estático tratar `Derived` como uma *subclasse* de `Original`, o que significa que um valor do tipo `Original` não pode ser utilizado onde um valor do tipo `Derived` é esperado. Isso é útil quando você deseja evitar erros de lógica com custo mínimo de tempo de execução.

Adicionado na versão 3.5.2.

Alterado na versão 3.10: `NewType` agora é uma classe em vez de uma função. Consequentemente, existem alguns custos em tempo de execução ao chamar `NewType` em vez de uma função comum.

Alterado na versão 3.11: O desempenho de chamar `NewType` voltou ao mesmo nível da versão Python 3.9.

Anotações de objetos chamáveis

Funções – ou outros objetos chamáveis – podem ser anotados com **`collections.abc.Callable`** ou com o tipo descontinuado **`typing.Callable`**. `Callable[[int], str]` expressa uma função que recebe um único parâmetro de tipo **`int`** e retorna uma **`str`**.

Por exemplo:

```
from collections.abc import Callable, Awaitable

def feeder(get_next_item: Callable[[], str]) -> None:
    ...  # Corpo

def async_query(on_success: Callable[[int], None],
                on_error: Callable[[int, Exception], None]) -> No
    ...  # Corpo

async def on_update(value: str) -> None:
    ...  # Corpo

callback: Callable[[str], Awaitable[None]] = on_update
```

A sintaxe de subscrição deve sempre ser usada com exatamente dois valores: a lista de argumentos e o tipo de retorno. A lista de argumentos deve ser uma lista de tipos, uma **`ParamSpec`**, **`Concatenate`** ou reticências (. . .). O tipo de retorno deve ser um tipo único.

Se uma reticências literal . . . é passada no lugar de uma lista de argumentos, ela indica que um chamável com uma lista de qualquer parâmetro arbitrário seria aceita.

```
def concat(x: str, y: str) -> str:
    return x + y

x: Callable[..., str]
x = str     # OK
x = concat  # OK também
```

`Callable` não pode representar assinaturas complexas, como funções que aceitam um número variado de argumentos, funções sobrecarregadas, or funções que recebem apenas parâmetros somente-nomeados. No entanto, essas assinaturas podem ser expressas ao se definir uma **`Protocol`** com um método **`__call__()`**:

```
from collections.abc import Iterable
from typing import Protocol

class Combiner(Protocol):
    def __call__(self, *vals: bytes, maxlen: int | None = None) -

def batch_proc(data: Iterable[bytes], cb_results: Combiner) -> by
    for item in data:
        ...

def good_cb(*vals: bytes, maxlen: int | None = None) -> list[byte
    ...
def bad_cb(*vals: bytes, maxitems: int | None) -> list[bytes]:
    ...

batch_proc([], good_cb)  # OK
batch_proc([], bad_cb)   # Erro! Argumento 2 tem tipo incompatíve
                         # nome e tipo diferente do callback
```

Chamáveis que recebem outros chamáveis como argumentos podem indicar que seus tipos de parâmetro dependem uns dos outros usando **`ParamSpec`**. Além disso, se esse chamável adiciona ou remove argumentos de outros chamáveis, o operador **`Concatenate`** pode ser usado. Eles assumem a forma de `Callable[ParamSpecVariable, ReturnType]` e `Callable[Concatenate[Arg1Type, Arg2Type, ..., ParamSpecVariable], ReturnType]`, respectivamente.

Alterado na versão 3.10: `Callable` agora oferece suporte a **`ParamSpec`** e **`Concatenate`**. Veja **PEP 612** [https://peps.python.org/pep-0612/] para mais detalhes.

Ver também

A documentação para **`ParamSpec`** e **`Concatenate`** contém exemplos de uso em `Callable`.

Genéricos

Como a informação de tipo sobre objetos mantidos em contêineres não pode ser inferida estaticamente de uma maneira genérica, muitas classes de contêineres na biblioteca padrão permitem usar subscrição para denotar o tipo esperado dos elementos dos contêineres.

```
from collections.abc import Mapping, Sequence

class Employee: ...

# Sequence[Employee] indica que todos os elementos na sequência
# devem ser instâncias de "Employee".
# Mapping[str, str] indica que todas as chaves e valores no mapea
# devem ser strings.
def notify_by_email(employees: Sequence[Employee],
                    overrides: Mapping[str, str]) -> None: ...
```

Funções e classes genéricas podem ser parametrizadas utilizando a sintaxe de parâmetro de tipos:

```
from collections.abc import Sequence

def first[T](l: Sequence[T]) -> T:  # Função é genérica sobre Typ
    return l[0]
```

Ou utilizando a fábrica `TypeVar` diretamente:

```
from collections.abc import Sequence
from typing import TypeVar

U = TypeVar('U')                  # Declara tipo variável "U"

def second(l: Sequence[U]) -> U:  # Função é genérica sobre a Typ
    return l[1]
```

Alterado na versão 3.12: O suporte sintático para genéricos é novo no Python 3.12.

Anotando tuplas

Para a maior parte dos tipos contêineres em Python, o sistema de tipagem presume que todos os elementos do contêiner são do mesmo tipo. Por exemplo:

```
from collections.abc import Mapping

# Verificador de tipos irá inferir que todos os elementos em ``x`
x: list[int] = []

# Erro no verificador: ``list`` aceita um único tipo de argumento
y: list[int, str] = [1, 'foo']

# Verificador de tipos irá inferir que todas as chaves em ``z`` d
# e todos os valores em ``z`` devem ser strings ou ints
z: Mapping[str, str | int] = {}
```

`list` aceita apenas um tipo de argumento, e assim o verificador de tipos emitirá um erro na atribuição `y` acima. Da mesma forma, `Mapping` aceita apenas dois tipos de argumento: O primeiro indica o tipo das chaves, e o segundo indica o tipo dos valores.

Ao contrário da maioria dos outros contêineres Python, é comum no código Python idiomático que as tuplas tenham elementos que não sejam todos do mesmo tipo. Por esse motivo, as tuplas têm um caso especial no sistema de tipagem do Python. `tuple` aceita *qualquer quantidade* de argumentos de tipo:

```
# OK: ``x`` foi atribuída a uma tupla de comprimento 1, onde o ún
x: tuple[int] = (5,)

# OK: ``y`` foi atribuída a uma tupla de comprimento 2;
# o elemento 1 é um int, o elemento 2 é uma str
y: tuple[int, str] = (5, "foo")

# Erro: a anotação de tipo indica uma tupla de comprimento 1,
```

```
# mas ``z`` foi atribuída a uma tupla de comprimento 3
z: tuple[int] = (1, 2, 3)
```

Para denotar uma tupla que pode ter *qualquer* comprimento e na qual todos os elementos são do mesmo tipo `T`, use a literal de reticências `...`: `tuple[T, ...]`. Para denotar uma tupla vazia, use `tuple[()]`. Usar `tuple` simples como anotação equivale a usar `tuple[Any, ...]`:

```
x: tuple[int, ...] = (1, 2)
# Essas reatribuições são OK: ``tuple[int, ...]`` indica que x po
x = (1, 2, 3)
x = ()
# Essa reatribuição é um erro: todos os elementos de ``x`` devem
x = ("foo", "bar")

# ``y`` só pode ser atribuída a uma tupla vazia
y: tuple[()] = ()

z: tuple = ("foo", "bar")
# Essas reatribuições são OK: o tipo ``tuple`` equivale a ``tuple
z = (1, 2, 3)
z = ()
```

O tipo de objetos de classe

Uma variável anotada com `C` pode aceitar um valor de tipo `C`. Por outro lado, uma variável anotada com `type[C]` (ou o tipo descontinuado **`typing.Type[C]`**) pode aceitar valores que são as próprias classes – mais especificamente, ela aceitará o *objeto classe* de `C`. Por exemplo:

```
a = 3         # Tem tipo ``int``
b = int       # Tem tipo ``type[int]``
c = type(a)   # Também tem tipo ``type[int]``
```

Observe que `type[C]` é covariante:

```
class User: ...
class ProUser(User): ...
class TeamUser(User): ...
```

```
def make_new_user(user_class: type[User]) -> User:
    # ...
    return user_class()

make_new_user(User)      # OK
make_new_user(ProUser)   # Também OK: ``type[ProUser]`` é subtipo
make_new_user(TeamUser)  # Tudo bem
make_new_user(User())    # Erro: esperava-se ``type[User]`` mas r
make_new_user(int)       # Erro: ``type[int]`` não é subtipo de `
```

Os únicos parâmetros válidos para `type` são classes, `Any`, type variables e uniões de qualquer um desses tipos. Por exemplo:

```
def new_non_team_user(user_class: type[BasicUser | ProUser]): ...

new_non_team_user(BasicUser)  # OK
new_non_team_user(ProUser)    # OK
new_non_team_user(TeamUser)   # Erro: ``type[TeamUser]`` não é su
                              # de ``type[BasicUser | ProUser]``
new_non_team_user(User)       # Também é um erro
```

`type[Any]` é equivalente a `type`, que é a raiz da hierarquia de metaclasses do Python.

Anotando geradores e corrotinas

Um gerador pode ser anotado usando um tipo genérico `Generator[YieldType, SendType, ReturnType]`. Por exemplo:

```
def echo_round() -> Generator[int, float, str]:
    sent = yield 0
    while sent >= 0:
        sent = yield round(sent)
    return 'Done'
```

Note que, diferentemente de outras classes genéricas na biblioteca padrão, o `SendType` da classe `Generator` se comporta contravariantemente, em vez de covariantemente ou invariantemente.

Os parâmetros `SendType` e `ReturnType` são `None` por padrão:

```
def infinite_stream(start: int) -> Generator[int]:
    while True:
        yield start
        start += 1
```

Também é possível definir esses tipos explicitamente:

```
def infinite_stream(start: int) -> Generator[int, None, None]:
    while True:
        yield start
        start += 1
```

Geradores simples que só produzem valores também podem ser anotados com tipo de retorno `Iterable[YieldType]` ou `Iterator[YieldType]`:

```
def infinite_stream(start: int) -> Iterator[int]:
    while True:
        yield start
        start += 1
```

Geradores assíncronos são manipulados similarmente, mas não espere um argumento de tipo `ReturnType` (`AsyncGenerator[YieldType, SendType]`). O argumento `SendType` é **`None`** por padrão, então as seguintes definições são equivalentes:

```
async def infinite_stream(start: int) -> AsyncGenerator[int]:
    while True:
        yield start
        start = await increment(start)

async def infinite_stream(start: int) -> AsyncGenerator[int, None
    while True:
        yield start
        start = await increment(start)
```

Como no caso síncrono, `AsyncIterable[YieldType]` e `AsyncIterator[YieldType]` também estão disponíveis:

```
async def infinite_stream(start: int) -> AsyncIterator[int]:
    while True:
        yield start
        start = await increment(start)
```

Corrotinas podem ser anotadas usando `Coroutine[YieldType, SendType, ReturnType]`. Argumentos genéricos correspondem àqueles da classe `Generator`, por exemplo:

```
from collections.abc import Coroutine
c: Coroutine[list[str], str, int]  # Uma corrotina defina em outr
x = c.send('hi')                   # Tipo inferido de 'x' é list[
async def bar() -> None:
    y = await c                    # Tipo inferido de 'y' é int
```

Tipos genéricos definidos pelo usuário

Uma classe definida pelo usuário pode ser definida como uma classe genérica.

```
from logging import Logger

class LoggedVar[T]:
    def __init__(self, value: T, name: str, logger: Logger) -> No
        self.name = name
        self.logger = logger
        self.value = value

    def set(self, new: T) -> None:
        self.log('Set ' + repr(self.value))
        self.value = new

    def get(self) -> T:
        self.log('Get ' + repr(self.value))
        return self.value

    def log(self, message: str) -> None:
        self.logger.info('%s: %s', self.name, message)
```

Esta sintaxe indica que o classe `LoggedVar` é parametrizada em torno de uma única type variable `T`. Isso também torna `T` válido como um tipo dentro do corpo da classe.

Classes genéricas implicitamente herdar de `Generic`. Para compatibilidade com Python 3.11 e versões inferiores, também é possível herdar

explicitamente de **`Generic`** para indicar uma classe genérica:

```
from typing import TypeVar, Generic

T = TypeVar('T')

class LoggedVar(Generic[T]):
    ...
```

Classes genéricas têm métodos **`__class_getitem__()`**, o que significa que podem ser parametrizadas em tempo de execução (por exemplo, `LoggedVar[int]` abaixo):

```
from collections.abc import Iterable

def zero_all_vars(vars: Iterable[LoggedVar[int]]) -> None:
    for var in vars:
        var.set(0)
```

Um tipo genérico pode ter qualquer número de tipos de variáveis. Todas as variedades de **`TypeVar`** são permitidas como parâmetros para um tipo genérico:

```
from typing import TypeVar, Generic, Sequence

class WeirdTrio[T, B: Sequence[bytes], S: (int, str)]:
    ...

OldT = TypeVar('OldT', contravariant=True)
OldB = TypeVar('OldB', bound=Sequence[bytes], covariant=True)
OldS = TypeVar('OldS', int, str)

class OldWeirdTrio(Generic[OldT, OldB, OldS]):
    ...
```

Cada tipo dos argumentos para **`Generic`** devem ser distintos. Assim, os seguintes exemplos são inválidos:

```
from typing import TypeVar, Generic
...

class Pair[M, M]:  # SyntaxError
```

```
    ...

T = TypeVar('T')

class Pair(Generic[T, T]):   # INVÁLIDO
    ...
```

Classes genéricas podem também herdar de outras classes:

```
from collections.abc import Sized

class LinkedList[T](Sized):
    ...
```

Ao herdar das classes genérico, alguns tipos podem ser fixos:

```
from collections.abc import Mapping

class MyDict[T](Mapping[str, T]):
    ...
```

Neste caso `MyDict` possui um único parâmetro, `T`.

O uso de uma classe genérica sem especificar tipos pressupõe `Any` para cada posição. No exemplo a seguir, `MyIterable` não é genérico, mas herda implicitamente de `Iterable[Any]`:

```
from collections.abc import Iterable

class MyIterable(Iterable): # Igual a Iterable[Any]
    ...
```

Também há suporte para tipos genéricos definidos pelo usuário. Exemplos:

```
from collections.abc import Iterable

type Response[S] = Iterable[S] | int

# Tipo de retorno aqui é igual a Iterable[str] | int
def response(query: str) -> Response[str]:
    ...

type Vec[T] = Iterable[tuple[T, T]]
```

```
def inproduct[T: (int, float, complex)](v: Vec[T]) -> T: # Igual
    return sum(x*y for x, y in v)
```

Para manter retrocompatibilidade, os apelidos de tipos genéricos também podem ser criados usando atribuição:

```
from collections.abc import Iterable
from typing import TypeVar

S = TypeVar("S")
Response = Iterable[S] | int
```

Alterado na versão 3.7: **`Generic`** não possui mais uma metaclasse personalizada.

Alterado na versão 3.12: Suporte sintático para apelidos de tipo e genéricos é novo na versão 3.12. Anteriormente, as classes genéricas precisavam explicitamente herdar de **`Generic`** ou conter um tipo de variável em uma de suas bases.

Genéricos definidos pelo usuário para expressões de parâmetros também oferecem suporte por meio de variáveis de especificação de parâmetros no formato `[**P]`. O comportamento é consistente com as variáveis de tipo descritas acima, pois as variáveis de especificação de parâmetro são tratadas pelo módulo **`typing`** como uma variável de tipo especializada. A única exceção a isso é que uma lista de tipos pode ser usada para substituir um **`ParamSpec`**:

```
>>> class Z[T, **P]: ...  # T é um TypeVar; P é um ParamSpec
...
>>> Z[int, [dict, float]]
__main__.Z[int, [dict, float]]
```

Classes genéricas sobre um **`ParamSpec`** também podem ser criadas usando herança explícita de **`Generic`**. Neste caso, `**` não é usado:

```
from typing import ParamSpec, Generic

P = ParamSpec('P')
```

```
class Z(Generic[P]):
    ...
```

Outra diferença entre `TypeVar` e `ParamSpec` é que um genérico com apenas uma variável de especificação de parâmetro aceitará listas de parâmetros nos formatos `X[[Type1, Type2, ...]]` e também `X[Type1, Type2, ...]` por razões estéticas. Internamente, o último é convertido no primeiro, portanto são equivalentes:

```
>>> class X[**P]: ...
...
>>> X[int, str]
__main__.X[[int, str]]
>>> X[[int, str]]
__main__.X[[int, str]]
```

Observe que genéricos com `ParamSpec` podem não ter `__parameters__` corretos após a substituição em alguns casos porque eles são destinados principalmente à verificação de tipo estático.

Alterado na versão 3.10: `Generic` agora pode ser parametrizado através de expressões de parâmetros. Veja `ParamSpec` e **PEP 612** [https://peps.python.org/pep-0612/] para mais detalhes.

Uma classe genérica definida pelo usuário pode ter ABCs como classes base sem conflito de metaclasse. Não há suporte a metaclasses genéricas. O resultado da parametrização de genéricos é armazenado em cache, e a maioria dos tipos no módulo **`typing`** são hasheáveis e comparáveis em termos de igualdade.

O tipo `Any`

A special kind of type is `Any`. A static type checker will treat every type as assignable to **`Any`** and **`Any`** as assignable to every type.

Isso significa que é possível realizar qualquer operação ou chamada de método sobre um valor do tipo `Any` e atribuí-lo a qualquer variável:

```
from typing import Any

a: Any = None
a = []          # OK
a = 2           # OK

s: str = ''
s = a           # OK

def foo(item: Any) -> int:
    # Passa na verificação de tipos; 'item' pode ser qualquer tip
    # e esse tipo pode ter um método 'bar'
    item.bar()
    ...
```

Observe que nenhuma verificação de tipo é realizada ao atribuir um valor do tipo `Any` a um tipo mais preciso. Por exemplo, o verificador de tipo estático não relatou um erro ao atribuir `a` a `s` mesmo que `s` tenha sido declarado como sendo do tipo `str` e receba um valor `int` em tempo de execução!

Além disso, todas as funções sem um tipo de retorno ou tipos de parâmetro terão como padrão implicitamente o uso de `Any`:

```
def legacy_parser(text):
    ...
    return data

# Um verificador de tipo estático tratará a função acima
# como tendo a mesma assinatura da função abaixo:
def legacy_parser(text: Any) -> Any:
    ...
    return data
```

Este comportamento permite que `Any` seja usado como uma *saída de emergência* quando você precisar misturar código tipado dinamicamente e estaticamente.

Compare o comportamento de `Any` com o comportamento de `object`. Semelhante a **`Any`**, todo tipo é um subtipo de **`object`**. No entanto, ao contrário de **`Any`**, o inverso não é verdadeiro: **`object`** *não* é um subtipo de qualquer outro tipo.

Isso significa que quando o tipo de um valor é `object`, um verificador de tipo rejeitará quase todas as operações nele, e atribuí-lo a uma variável (ou usá-la como valor de retorno) de um tipo mais especializado é um tipo erro. Por exemplo:

```
def hash_a(item: object) -> int:
    # Fails type checking; an object does not have a 'magic' meth
    item.magic()
    ...

def hash_b(item: Any) -> int:
    # Passes type checking
    item.magic()
    ...

# Passes type checking, since ints and strs are subclasses of obj
hash_a(42)
hash_a("foo")

# Passes type checking, since Any is assignable to all types
hash_b(42)
hash_b("foo")
```

Use `object` para indicar que um valor pode ser de qualquer tipo de maneira segura. Use `Any` para indicar que um valor é tipado dinamicamente.

Subtipagem nominal vs estrutural

Inicialmente a **PEP 484** [https://peps.python.org/pep-0484/] definiu o sistema de tipos estáticos do Python como usando *subtipagem nominal*. Isto significa que uma classe `A` é permitida onde uma classe `B` é esperada se e somente se `A` for uma subclasse de `B`.

Este requisito anteriormente também se aplicava a classes base abstratas, como `Iterable`. O problema com essa abordagem é que uma classe teve que ser marcada explicitamente para suportá-los, o que não é pythônico e diferente do que normalmente seria feito em código Python de tipo dinamicamente idiomático. Por exemplo, isso está em conformidade com **PEP 484** [https://peps.python.org/pep-0484/]:

```
from collections.abc import Sized, Iterable, Iterator

class Bucket(Sized, Iterable[int]):
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...
```

PEP 544 [https://peps.python.org/pep-0544/] solves this problem by allowing users to write the above code without explicit base classes in the class definition, allowing `Bucket` to be implicitly considered a subtype of both `Sized` and `Iterable[int]` by static type checkers. This is known as *structural subtyping* (or static duck-typing):

```
from collections.abc import Iterator, Iterable

class Bucket:  # Nota: sem classes base
    ...
    def __len__(self) -> int: ...
    def __iter__(self) -> Iterator[int]: ...

def collect(items: Iterable[int]) -> int: ...
result = collect(Bucket())  # Passa na verificação de tipo
```

Além disso, ao criar uma subclasse de uma classe especial **`Protocol`**, um usuário pode definir novos protocolos personalizados para aproveitar ao máximo a subtipagem estrutural (veja exemplos abaixo).

Conteúdo do módulo

O módulo `typing` define as seguintes classes, funções e decoracores.

Tipos primitivos especiais

Tipos especiais

Eles podem ser usados como tipos em anotações. Eles não oferecem suporte a subscrição usando `[]`.

typing.Any

Tipo especial que indica um tipo irrestrito.

- Every type is assignable to `Any`.
- `Any` is assignable to every type.

Alterado na versão 3.11: `Any` agora pode ser usado como classe base. Isso pode ser útil para evitar erros do verificador de tipo com classes que podem digitar em qualquer lugar ou são altamente dinâmicas.

typing.AnyStr

Uma tipo variável restrito.

Definição:

```
AnyStr = TypeVar('AnyStr', str, bytes)
```

`AnyStr` deve ser usado para funções que podem aceitar argumentos `str` ou `bytes` mas não podem permitir que os dois se misturem.

Por exemplo:

```
def concat(a: AnyStr, b: AnyStr) -> AnyStr:
    return a + b

concat("foo", "bar")    # OK: saída tem tipo 'str'
concat(b"foo", b"bar")  # OK: saída tem tipo 'bytes'
concat("foo", b"bar")   # Erro: não se mistura str com bytes
```

Note que, apesar do nome, `AnyStr` não tem nada a ver com o tipo `Any`, nem significa “qualquer string”. Em particular, `AnyStr` e `str | bytes` são diferentes entre si e têm casos de uso diferentes:

```
# Uso inválido de AnyStr:
# A variável de tipo é usada somente uma vez na assinatura da
# então não pode ser "resolvida" pelo verificador de tipo
def greet_bad(cond: bool) -> AnyStr:
    return "hi there!" if cond else b"greetings!"

# O jeito melhor de anotar esta função
def greet_proper(cond: bool) -> str | bytes:
    return "hi there!" if cond else b"greetings!"
```

Descontinuado desde a versão 3.13, será removido na versão 3.18: Descontinuado em favor da nova sintaxe de parâmetro de tipo. Use `class A[T: (str, bytes)]: ...` em vez de importar `AnyStr`. Veja a **PEP 695** [https://peps.python.org/pep-0695/] para mais detalhes.

No Python 3.16, `AnyStr` será removido de `typing.__all__`, e avisos de descontinuação serão emitidos em tempo de execução quando ele for acessado ou importado de `typing`. `AnyStr` será removido do `typing` no Python 3.18.

typing.LiteralString

Tipo especial que inclui apenas strings literais.

Qualquer literal de string é compatível com `LiteralString`, assim como outro `LiteralString`. Entretanto, um objeto digitado apenas `str` não é. Uma string criada pela composição de objetos do tipo `LiteralString` também é aceitável como uma `LiteralString`.

Exemplo:

```
def run_query(sql: LiteralString) -> None:
    ...

def caller(arbitrary_string: str, literal_string: LiteralStrin
    run_query("SELECT * FROM students")  # OK
    run_query(literal_string)  # OK
    run_query("SELECT * FROM " + literal_string)  # OK
    run_query(arbitrary_string)  # erro no verificador
    run_query(  # erro no verificador
        f"SELECT * FROM students WHERE name = {arbitrary_strin
    )
```

`LiteralString` é útil para APIs sensíveis onde strings arbitrárias geradas pelo usuário podem gerar problemas. Por exemplo, os dois casos acima que geram erros no verificador de tipo podem ser vulneráveis a um ataque de injeção de SQL.

Veja **PEP 675** [https://peps.python.org/pep-0675/] para mais detalhes.

Adicionado na versão 3.11.

typing.Never
typing.NoReturn

`Never` e **`NoReturn`** representam o tipo inferior [https://en.wikipedia.org/wiki/Bottom_type], um tipo que não possui membros.

Eles podem ser usados para indicar que uma função nunca retorna, como **`sys.exit()`**:

```
from typing import Never  # ou NoReturn

def stop() -> Never:
    raise RuntimeError('no way')
```

Ou para definir uma função que nunca deve ser chamada, pois não existem argumentos válidos, como **`assert never()`**:

```
from typing import Never  # or NoReturn

def never_call_me(arg: Never) -> None:
    pass

def int_or_str(arg: int | str) -> None:
    never_call_me(arg)  # erro no verificador
    match arg:
        case int():
            print("It's an int")
        case str():
            print("It's a str")
        case _:
            never_call_me(arg)  # OK, arg é do tipo Never (ou
```

`Never` e **`NoReturn`** têm o mesmo significado no sistema de tipos e os verificadores de tipo estático tratam ambos de forma equivalente.

Adicionado na versão 3.6.2: Adicionado **`NoReturn`**.

Adicionado na versão 3.11: Adicionado **`Never`**.

typing.Self

Tipo especial para representar a classe atual inclusa.

Por exemplo:

```
from typing import Self, reveal_type

class Foo:
    def return_self(self) -> Self:
        ...
        return self

class SubclassOfFoo(Foo): pass

reveal_type(Foo().return_self())  # Tipo revelado é "Foo"
reveal_type(SubclassOfFoo().return_self())  # Tipo revelado é
```

Esta anotação é semanticamente equivalente à seguinte, embora de forma mais sucinta:

```
from typing import TypeVar

Self = TypeVar("Self", bound="Foo")

class Foo:
    def return_self(self: Self) -> Self:
        ...
        return self
```

Em geral, se algo retorna `self`, como nos exemplos acima, você deve usar `Self` como anotação de retorno. Se `Foo.return_self` foi anotado como retornando `"Foo"`, então o verificador de tipo inferiria o objeto retornado de `SubclassOfFoo.return_self` como sendo do tipo `Foo` em vez de `SubclassOfFoo`.

Outros casos de uso comuns incluem:

- `classmethod`s que são usados como construtores alternativos e retornam instâncias do parâmetro `cls`.
- Anotando um método `__enter__()` que retorna self.

Você não deveria usar `Self` como a anotação de retorno se não for garantido que o método retorne uma instância de uma subclasse quando a classe for subclassificada:

```
class Eggs:
    # Self seria uma anotação de retorno incorreta aqui,
    # já que o objeto retornado sempre é instância de Eggs,
    # inclusive em subclasses
    def returns_eggs(self) -> "Eggs":
        return Eggs()
```

Veja **PEP 673** [https://peps.python.org/pep-0673/] para mais detalhes.

Adicionado na versão 3.11.

typing.TypeAlias

Anotações especiais para declarar explicitamente um apelido de tipo.

Por exemplo:

```
from typing import TypeAlias

Factors: TypeAlias = list[int]
```

`TypeAlias` é particularmente útil em versões mais antigas do Python para anotar apelidos que fazem uso de referências futuras, pois pode ser difícil para os verificadores de tipo distingui-los das atribuições normais de variáveis:

```
from typing import Generic, TypeAlias, TypeVar

T = TypeVar("T")

# "Box" não existe ainda,
# então temos que usar aspas para a referência futura em Pytho
# Usar ``TypeAlias`` diz ao verificador de tipo que isso é uma
# e não uma atribuição de variável a string.
BoxOfStrings: TypeAlias = "Box[str]"

class Box(Generic[T]):
    @classmethod
    def make_box_of_strings(cls) -> BoxOfStrings: ...
```

Veja **PEP 613** [https://peps.python.org/pep-0613/] para mais detalhes.

Adicionado na versão 3.10.

Descontinuado desde a versão 3.12: **`TypeAlias`** foi descontinuado em favor da instrução **`type`**, que cria instâncias de **`TypeAliasType`** e oferece suporte nativo a referências futuras. Observe que embora **`TypeAlias`** e **`TypeAliasType`** sirvam propósitos semelhantes e tenham nomes semelhantes, eles são distintos e o último não é o tipo do primeiro. A remoção de **`TypeAlias`** não está planejada atualmente, mas os usuários são encorajados a migrar para instruções **`type`**.

Formas especiais

Eles podem ser usados como tipos em anotações. Todos eles oferecem suporte a subscrição usando `[]`, mas cada um tem uma sintaxe única.

class typing.Union

Tipo de união; `Union[X, Y]` é equivalente a `X | Y` e significa X ou Y.

Para definir uma união, use, por exemplo. `Union[int, str]` ou a abreviatura `int | str`. Usar essa abreviação é recomendado. Detalhes:

- Os argumentos devem ser tipos e deve haver pelo menos um.
- As uniões de uniões são achatadas, por exemplo:

  ```
  Union[Union[int, str], float] == Union[int, str, float]
  ```

  Entretanto, isso não se aplica a uniões referenciadas por meio de um apelido de tipo, para evitar forçar a avaliação da **`TypeAliasType`** subjacente:

  ```
  type A = Union[int, str]
  Union[A, float] != Union[int, str, float]
  ```

- As uniões de um único argumento desaparecem, por exemplo:

  ```
  Union[int] == int  # O construtor retorna um int
  ```

- Argumento redundantes são pulados, e.g.:

  ```
  Union[int, str, int] == Union[int, str] == int | str
  ```

- Ao comparar uniões, a ordem de argumentos é ignorada. Por exemplo:

  ```
  Union[int, str] == Union[str, int]
  ```

- Você não pode estender ou instanciar uma `Union`

- Você não pode escrever `Union[X][Y]`.

Alterado na versão 3.7: Não remova subclasses explícitas de uniões em tempo de execução.

Alterado na versão 3.10: Uniões agora podem ser escritas com `X | Y`. Veja expressões de união de tipos.

Alterado na versão 3.14: **`types.UnionType`** agora é um apelido para **`Union`**, e tanto `Union[int, str]` quanto `int | str` criam instâncias da mesma classe. Para verificar se um objeto é um `Union` em tempo de execução, use `isinstance(obj, Union)`. Para compatibilidade com versões anteriores do Python, use `get_origin(obj) is typing.Union or get_origin(obj) is types.UnionType`.

typing.Optional

`Optional[X]` equivale a `X | None` (ou `Union[X, None]`).

Note que isso não é o mesmo conceito de um argumento opcional, que possui um valor por padrão. Um argumento opcional com padrão não requer o qualificador `Optional` em sua anotação de tipo só por ser opcional. Por exemplo:

```
def foo(arg: int = 0) -> None:
    ...
```

Por outro lado, se um valor explícito de `None` for permitido, o uso de `Optional` é apropriado, seja o argumento opcional ou não. Por exemplo:

```
def foo(arg: Optional[int] = None) -> None:
    ...
```

Alterado na versão 3.10: Optional agora pode ser escrito como `X | None`. Veja expressões de união de tipos.

typing.Concatenate

Forma especial para anotar funções de ordem superior.

`Concatenate` can be used in conjunction with Callable and **`ParamSpec`** to annotate a higher-order callable which adds, removes, or transforms parameters of another callable. Usage is in the form `Concatenate[Arg1Type, Arg2Type, ..., ParamSpecVariable]`. `Concatenate` is valid when used in Callable type hints and when instantiating user-defined generic classes with **`ParamSpec`** parameters. The last parameter to `Concatenate` must be a **`ParamSpec`** or ellipsis (`...`).

Por exemplo, para anotar um decorador `with_lock` que oferece uma instância de **`threading.Lock`** para a função decorada, `Concatenate` pode ser usado para indicar que `with_lock` espera um chamável cujo primeiro argumento tem tipo `Lock`, e retorna um chamável com uma assinatura de tipos diferente. Neste caso, o **`ParamSpec`** indica que os tipos dos parâmetros do chamável retornado dependem dos tipos dos parâmetros do chamável de entrada:

```
from collections.abc import Callable
from threading import Lock
from typing import Concatenate

# Use esta trava para se certificar de que
# uma única thread a executa a qualquer momento.
my_lock = Lock()

def with_lock[**P, R](f: Callable[Concatenate[Lock, P], R]) ->
    '''Um decorador com segurança de tipos que oferece uma tra
    def inner(*args: P.args, **kwargs: P.kwargs) -> R:
        # Oferece a trava como primeiro argumento.
        return f(my_lock, *args, **kwargs)
    return inner

@with_lock
def sum_threadsafe(lock: Lock, numbers: list[float]) -> float:
    '''Somatório de uma lista de números de modo seguro para t
    with lock:
```

```
        return sum(numbers)

# We don't need to pass in the lock ourselves thanks to the de
sum_threadsafe([1.1, 2.2, 3.3])
```

Adicionado na versão 3.10.

Ver também

- **PEP 612** [https://peps.python.org/pep-0612/] – Variáveis de especificação de parâmetro (a PEP que introduz `ParamSpec` e `Concatenate`)
- **`ParamSpec`**
- Anotações de objetos chamáveis

typing.Literal

Forma especial de tipagem para definir "tipos literais".

`Literal` pode ser usado para indicar aos verificadores de tipo que o objeto anotado tem valor equivalente a algum dos literais oferecidos.

Por exemplo:

```
def validate_simple(data: Any) -> Literal[True]:  # sempre ret
    ...

type Mode = Literal['r', 'rb', 'w', 'wb']
def open_helper(file: str, mode: Mode) -> str:
    ...

open_helper('/some/path', 'r')      # Passa verificação de tip
open_helper('/other/path', 'typo')  # Erro em verificação de t
```

`Literal[...]` não é subclasse. Em tempo de execução, permite-se um valor arbitrário como argumento de tipo para `Literal[...]`, mas verificadores de tipo podem impor restrições. Veja **PEP 586** [https://peps.python.org/pep-0586/] para mais detalhes sobre tipos literais.

Detalhes adicionais:

- Os argumentos devem ser valores literais e deve haver pelo menos um.

- Tipos `Literal` aninhados são achatados, por exemplo:

  ```
  assert Literal[Literal[1, 2], 3] == Literal[1, 2, 3]
  ```

  Entretanto, isso não se aplica a tipos `Literal` referenciadas por meio de um apelido de tipo, para evitar forçar a avaliação da **`TypeAliasType`** subjacente:

  ```
  type A = Literal[1, 2]
  assert Literal[A, 3] != Literal[1, 2, 3]
  ```

- Argumento redundantes são pulados, e.g.:

  ```
  assert Literal[1, 2, 1] == Literal[1, 2]
  ```

- Ao comparar literais, a ordem de argumentos é ignorada. Por exemplo:

  ```
  assert Literal[1, 2] == Literal[2, 1]
  ```

- Você não pode estender ou instanciar uma `Literal`.

- Você não pode escrever `Literal[X][Y]`.

Adicionado na versão 3.8.

Alterado na versão 3.9.1: `Literal` agora remove parâmetros duplicados. Comparações de igualdade entre objetos `Literal` não dependem da ordem. Objetos `Literal` agora levantam uma exceção **`TypeError`** durante comparações de igualdade se um de seus parâmetros não for hasheável.

typing.ClassVar

Uma construção especial de tipagem para marcar variáveis de classe.

Como introduzido na **PEP 526** [https://peps.python.org/pep-0526/], uma variável cuja anotação de tipo tem um invólucro ClassVar indica que um dado atributo deve ser usado como uma variável de classe, e que ele não deve ser definido em instâncias dessa classe. Modo de usar:

```
class Starship:
    stats: ClassVar[dict[str, int]] = {} # variável de classe
    damage: int = 10                     # variável de instânc
```

`ClassVar` aceita apenas tipos e não pode ser subscrita posteriormente.

`ClassVar` is not a class itself, and cannot be used with `isinstance()` or `issubclass()`. **`ClassVar`** does not change Python runtime behavior, but it can be used by static type checkers. For example, a type checker might flag the following code as an error:

```
enterprise_d = Starship(3000)
enterprise_d.stats = {} # Erro: atribuindo valor a variável de
Starship.stats = {}     # Isso é OK
```

Adicionado na versão 3.5.3.

Alterado na versão 3.13: `ClassVar` agora pode ser aninhada em `Final` e vice-versa.

typing.Final

Uma construção especial de tipagem para indicar nomes finais a verificadores de tipos

Nomes finais não podem ser reatribuídos em qualquer escopo. Nomes finais declarados em escopos de classe não podem ser substituídos em subclasses.

Por exemplo:

```
MAX_SIZE: Final = 9000
MAX_SIZE += 1  # Erro reportado pelo verificador de tipos

class Connection:
    TIMEOUT: Final[int] = 10

class FastConnector(Connection):
    TIMEOUT = 1  # Error reportado pelo verificador de tipos
```

Não há verificação em tempo de execução dessas propriedades. Veja **PEP 591** [https://peps.python.org/pep-0591/] para mais detalhes.

Adicionado na versão 3.8.

Alterado na versão 3.13: `Final` agora pode ser aninhada em `ClassVar` e vice-versa.

typing.Required

Uma construção especial de tipagem para marcar uma chave de `TypedDict` como necessária.

Isso é útil principalmente para TypedDicts com `total=False`. Veja `TypedDict` e **PEP 655** [https://peps.python.org/pep-0655/] para obter mais detalhes.

Adicionado na versão 3.11.

typing.NotRequired

Uma construção especial de tipagem para marcar uma chave de `TypedDict` como potencialmente ausente.

Veja `TypedDict` e **PEP 655** [https://peps.python.org/pep-0655/] para obter mais detalhes.

Adicionado na versão 3.11.

typing.ReadOnly

Uma construção especial de tipagem para marcar um item de um `TypedDict` como somente leitura.

Por exemplo:

```
class Movie(TypedDict):
   title: ReadOnly[str]
   year: int

def mutate_movie(m: Movie) -> None:
   m["year"] = 1999  # allowed
   m["title"] = "The Matrix"  # type checker error
```

Não há verificação em tempo de execução para esta propriedade.

Consulte `TypedDict` e **PEP 705** [https://peps.python.org/pep-0705/] para obter mais detalhes.

Adicionado na versão 3.13.

typing.Annotated

Forma especial de tipagem para adicionar metadados específicos de contexto para uma anotação.

Adiciona metadados `x` a um determinado tipo `T` usando a anotação `Annotated[T, x]`. Os metadados adicionados usando `Annotated` podem ser usados por ferramentas de análise estática ou em tempo de execução. Em tempo de execução, os metadados são armazenados em um atributo **`__metadata__`**.

Se uma biblioteca ou ferramenta encontrar uma anotação `Annotated[T, x]` e ela não tiver lógica especial para os metadados, ela deverá ignorar os metadados e simplesmente tratar a anotação como `T`. Dessa forma, `Annotated` pode ser útil em códigos que desejarem usar a anotação para propósitos fora do sistema de tipagem estática do Python.

Usar `Annotated[T, x]` como um anotação ainda permite a verificação estática do tipo de `T`, pois os verificadores de tipo simplesmente ignoram os metadados em `x`. Dessa forma, o `Annotated` difere do decorador `@no_type_check`, que também pode ser usado para adicionar anotações fora do escopo do sistema de tipagem, mas desativa completamente a verificação de tipos para uma função ou classe.

A responsabilidade de como interpretar os metadados é da ferramenta ou da biblioteca que encontrar uma anotação `Annotated`. Uma ferramenta ou biblioteca que encontrar um tipo `Annotated` poderá examinar os elementos de metadados para determinar se eles são de interesse (por exemplo, usando `isinstance()`).

Annotated[<type>, <metadata>]

Aqui está um exemplo de como você pode usar `Annotated` para adicionar metadados a anotações de tipos se estiver fazendo uma análise de intervalos:

```
@dataclass
class ValueRange:
    lo: int
    hi: int

T1 = Annotated[int, ValueRange(-10, 5)]
T2 = Annotated[T1, ValueRange(-20, 3)]
```

O primeiro argumento para `Annotated` deve ser um tipo válido. Vários elementos de metadados podem ser fornecidos, pois `Annotated` suporta argumentos variádicos. A ordem dos elementos de metadados é preservada e importa para verificações de igualdade:

```
@dataclass
class ctype:
     kind: str

a1 = Annotated[int, ValueRange(3, 10), ctype("char")]
a2 = Annotated[int, ctype("char"), ValueRange(3, 10)]

assert a1 != a2  # a ordem importa
```

Cabe à ferramenta que consome as anotações decidir se o cliente tem permissão de adicionar vários elementos de metadados a uma anotação e como mesclar essas anotações.

Os tipos `Annotated` aninhados são achatados. A ordem dos elementos de metadados começa com a anotação mais interna:

```
assert Annotated[Annotated[int, ValueRange(3, 10)], ctype("cha
    int, ValueRange(3, 10), ctype("char")
]
```

Entretanto, isso não se aplica a tipos `Annotated` referenciadas por meio de um apelido de tipo, para evitar forçar a avaliação da **`TypeAliasType`** subjacente:

```
type From3To10[T] = Annotated[T, ValueRange(3, 10)]
assert Annotated[From3To10[int], ctype("char")] != Annotated[
   int, ValueRange(3, 10), ctype("char")
]
```

Elementos duplicados de metadata não são removidos:

```
assert Annotated[int, ValueRange(3, 10)] != Annotated[
    int, ValueRange(3, 10), ValueRange(3, 10)
]
```

`Annotated` pode ser usado com apelidos aninhados e apelidos genéricos:

```
@dataclass
class MaxLen:
    value: int

type Vec[T] = Annotated[list[tuple[T, T]], MaxLen(10)]

# Quando usado em uma anotação de tipo, um verificador de
# ``Annotated[list[tuple[int, int]], MaxLen(10)]``:
type V = Vec[int]
```

`Annotated` não pode ser usado com um **`TypeVarTuple`** desempacotado:

```
type Variadic[*Ts] = Annotated[*Ts, Ann1] = Annotated[T1, T2,
```

onde `T1`, `T2`, … são **`TypeVars`**. Isso é inválido, pois somente um tipo deve ser passado para Annotated.

Por padrão, **`get_type_hints()`** retira os metadados de anotações. Passe `include_extras=True` para preservar os metadados:

```
>>> from typing import Annotated, get_type_hints
>>> def func(x: Annotated[int, "metadata"]) -> None: pass
...
>>> get_type_hints(func)
{'x': <class 'int'>, 'return': <class 'NoneType'>}
>>> get_type_hints(func, include_extras=True)
{'x': typing.Annotated[int, 'metadata'], 'return': <class
```

Em tempo de execução, os metadados associados a um tipo `Annotated` podem ser recuperados por meio do atributo **`__metadata__`**:

```
>>> from typing import Annotated
>>> X = Annotated[int, "very", "important", "metadata"]
>>> X
typing.Annotated[int, 'very', 'important', 'metadata']
>>> X.__metadata__
('very', 'important', 'metadata')
```

Se você quiser recuperar o tipo original envolvido por `Annotated`, use o atributo **`__origin__`**:

```
>>> from typing import Annotated, get_origin
>>> Password = Annotated[str, "secret"]
>>> Password.__origin__
<class 'str'>
```

Note que usar **`get_origin()`** retornará o próprio `Annotated`:

```
>>> get_origin(Password)
typing.Annotated
```

Ver também

PEP 593 [https://peps.python.org/pep-0593/] - Anotações flexíveis para funções e variáveis

A PEP que introduz `Annotated` à biblioteca padrão.

Adicionado na versão 3.9.

typing.TypeForm

A special form representing the value that results from evaluating a type expression.

This value encodes the information supplied in the type expression, and it represents the type described by that type expression.

When used in a type expression, `TypeForm` describes a set of type form objects. It accepts a single type argument, which must be a valid type

expression. `TypeForm[T]` describes the set of all type form objects that represent the type `T` or types assignable to `T`.

`TypeForm(obj)` simply returns `obj` unchanged. This is useful for explicitly marking a value as a type form for static type checkers.

Exemplo:

```
from typing import Any, TypeForm

def cast[T](typ: TypeForm[T], value: Any) -> T: ...

reveal_type(cast(int, "x"))  # Revealed type is "int"
```

See **PEP 747** [https://peps.python.org/pep-0747/] for details.

Adicionado na versão 3.15.

typing.TypeIs

Uma construção especial de tipagem para marcar funções de predicado de tipo definidas pelo usuário.

`TypeIs` pode ser usado para anotar o tipo de retorno de uma função de predicado de tipo definida pelo usuário. `TypeIs` aceita apenas um único argumento de tipo. Em tempo de execução, funções marcadas dessa forma devem retornar um booleano e receber, no mínimo, um argumento posicional.

`TypeIs` visa beneficiar *estreitamento de tipo* – uma técnica usada por verificador de tipo estático para determinar um tipo mais preciso de expressão dentro do fluxo de código de um programa. Normalmente, o estreitamento de tipo é feito por meio da análise do fluxo de código condicional e da aplicação do estreitamento a um bloquear código. A condicional expressão aqui é às vezes chamada de “predicado de tipo”:

```
def is_str(val: str | float):
    # predicado de tipos "isinstance"
    if isinstance(val, str):
        # Tipo de ``val`` é restrito a ``str``
        ...
    else:
```

```
        # Senão, tipo de ``val`` é restrito a ``float``.
        ...
```

Às vezes, seria conveniente usar uma função booleana definida pelo usuário como um predicado de tipo. Essa função deveria usar `TypeIs[...]` ou **`TypeGuard`** como seu tipo de retorno para alertar o verificador de tipo estático sobre essa intenção. `TypeIs` geralmente tem um comportamento mais intuitivo do que `TypeGuard`, mas não pode ser usado quando os tipos de entrada e saída são incompatíveis (por exemplo, de `list[object]` para `list[int]`) ou quando a função não retorna `True` para todas as instâncias do tipo restrito.

Usar `-> TypeIs[NarrowedType]` informa ao verificador de tipo estático que, para uma determinada função:

1. O valor de retorno é um booleano.
2. Se o valor de retorno for `True`, o tipo de seu argumento será a interseção entre o tipo original do argumento e o `NarrowedType`.
3. Se o valor de retorno for `False`, o tipo de seu argumento será restrito para excluir `NarrowedType`.

Por exemplo:

```
from typing import assert_type, final, TypeIs

class Parent: pass
class Child(Parent): pass
@final
class Unrelated: pass

def is_parent(val: object) -> TypeIs[Parent]:
    return isinstance(val, Parent)

def run(arg: Child | Unrelated):
    if is_parent(arg):
        # Tipo de ``arg`` é restrito à interseção
        # de ``Parent`` e ``Child``, o que equivale a
        # ``Child``.
        assert_type(arg, Child)
    else:
        # Tipo de ``arg`` é restrito para excluir ``Parent``,
```

```
# então sobrou o tipo ``Unrelated``.
assert_type(arg, Unrelated)
```

O tipo dentro de `TypeIs` deve ser consistente com o tipo do argumento da função; caso contrário, o verificador de tipo estático levantará um erro. Uma função `TypeIs` escrita incorretamente pode ocasionar um comportamento insensato pelo sistema de tipos; é de responsabilidade do usuário escrever tais funções de forma segura para tipos.

Se uma função `TypeIs` for um método de classe ou instância, então o tipo em `TypeIs` mapeia para o tipo do segundo parâmetro (depois de `cls` ou `self`).

Em resumo, a forma `def foo(arg: TypeA) -> TypeIs[TypeB]: ...`, significa que se `foo(arg)` retorna `True`, então `arg` é um instância de `TypeB`, e se retorna `False`, não é um instância de `TypeB`.

`TypeIs` também funciona com variáveis de tipo. Para mais informações, consulte a **PEP 742** [https://peps.python.org/pep-0742/] (Restringindo tipos com `TypeIs`).

Adicionado na versão 3.13.

typing.TypeGuard

Uma construção especial de tipagem para marcar funções de predicado de tipo definidas pelo usuário.

Funções de predicados de tipo são funções definidas pelo usuário que retornam se seu argumento é uma instância de um tipo específico. O `TypeGuard` funciona como **`TypeIs`**, mas tem efeitos sutilmente diferentes sobre o comportamento da verificação de tipos (veja abaixo).

Usar `-> TypeGuard` informa ao verificador de tipo estático que, para uma determinada função:

1. O valor de retorno é um booleano.
2. Se o valor de retorno for `True`, o tipo de seu argumento é o tipo dentro de `TypeGuard`.

`TypeGuard` também funciona com tipos variáveis. Consulte a **PEP 647** [https://peps.python.org/pep-0647/] para obter mais detalhes.

Por exemplo:

```
def is_str_list(val: list[object]) -> TypeGuard[list[str]]:
    '''Determina se todos os objetos na lista são strings'''
    return all(isinstance(x, str) for x in val)

def func1(val: list[object]):
    if is_str_list(val):
        # Tipo de ``val`` é restrito a ``list[str]``.
        print(" ".join(val))
    else:
        # Tipo de ``val`` continua sendo ``list[object]``.
        print("Not a list of strings!")
```

`TypeIs` e `TypeGuard` diferem das seguintes maneiras:

- `TypeIs` exige que o tipo restrito seja um subtipo do tipo de entrada, enquanto o `TypeGuard`, não. O motivo principal é permitir coisas como restringir `list[object]` a `list[str]`, embora este não seja um subtipo daquele, já que `list` é invariante.
- Quando uma função `TypeGuard` retorna `True`, os verificadores de tipo restringem o tipo da variável exatamente ao tipo `TypeGuard`. Quando uma função `TypeIs` retorna `True`, os verificadores de tipo podem inferir um tipo mais preciso combinando o tipo previamente conhecido da variável com o tipo `TypeIs`. (Tecnicamente, isso é conhecido como um tipo de interseção).
- Quando uma função `TypeGuard` retorna `False`, os verificadores de tipo não podem restringir o tipo da variável. Quando uma função `TypeIs` retorna `False`, os verificadores de tipo podem restringir o tipo da variável para excluir o tipo `TypeIs`.

Adicionado na versão 3.10.

typing.Unpack

Operador de tipagem para marcar conceitualmente um objeto como tendo sido desempacotado.

Por exemplo, usar o operador de desempacotamento `*` em uma tupla de tipos variáveis equivale a usar o `Unpack` para marcar a tupla de tipos variáveis como tendo sido desempacotada:

```
Ts = TypeVarTuple('Ts')
tup: tuple[*Ts]
# Efetivamente faz:
tup: tuple[Unpack[Ts]]
```

Na verdade, `Unpack` pode ser usado de forma intercambiável com `*` no contexto dos tipos **`typing.TypeVarTuple`** e **`builtins.tuple`**. Você pode ver `Unpack` sendo usado explicitamente em versões mais antigas do Python, onde `*` não podia ser usado em certos lugares:

```
# Em versões mais antigas do Python, TypeVarTuple e Unpack
# estão localizados no pacote de backports `typing_extensions`
from typing_extensions import TypeVarTuple, Unpack

Ts = TypeVarTuple('Ts')
tup: tuple[*Ts]         # Erro de sintaxe no Python <= 3.10!
tup: tuple[Unpack[Ts]]  # Semanticamente equivalente e compatí
```

`Unpack` também pode ser usado com **`typing.TypedDict`** para a tipagem de `**kwargs` em uma assinatura de função:

```
from typing import TypedDict, Unpack

class Movie(TypedDict):
    name: str
    year: int

# Esta função espera dois argumentos nomeados: `name` de tipo
# e `year` de tipo `int`.
def foo(**kwargs: Unpack[Movie]): ...
```

Consulte **PEP 692** [https://peps.python.org/pep-0692/] para obter mais detalhes sobre como usar `Unpack` para tipagem de `**kwargs`.

Adicionado na versão 3.11.

Criando tipos genéricos e apelidos de tipo

As classes a seguir não devem ser usadas diretamente como anotações. propósito O objetivo delas é construir bloquear para criar genérico e digitar apelido.

Esses objetos podem ser criados por meio de sintaxe especial (listas de parâmetros de tipo e a instrução **`type`**). Para compatibilidade com Python 3.11 e versões anteriores, eles também podem ser criados sem a sintaxe dedicada, como documentado abaixo.

class typing.Generic

Classe base abstrata para tipos genéricos

Normalmente, um tipo genérico é declarado adicionando-se uma lista de parâmetros de tipo após o nome da classe:

```
class Mapping[KT, VT]:
    def __getitem__(self, key: KT) -> VT:
        ...
        # Etc.
```

Essa classe herda implicitamente de `Generic`. A semântica em tempo de execução dessa sintaxe é discutida na Referência da Linguagem.

Esta classe pode ser utilizada como segue:

```
def lookup_name[X, Y](mapping: Mapping[X, Y], key: X, default:
    try:
        return mapping[key]
    except KeyError:
        return default
```

Aqui os colchetes depois no nome da função indica uma função genérica.

Para manter retrocompatibilidade, as classes genéricas também podem ser declaradas herdando explicitamente de `Generic`. Nesse caso, os parâmetros de tipo devem ser declarados separadamente:

```
KT = TypeVar('KT')
VT = TypeVar('VT')

class Mapping(Generic[KT, VT]):
    def __getitem__(self, key: KT) -> VT:
        ...
        # Etc.
```

class typing.TypeVar(*name*, **constraints*, *bound=None*, *covariant=False*, *contravariant=False*, *infer_variance=False*, *default=typing.NoDefault*)

Tipo variável.

A maneira preferida de construir um tipo variável é por meio da sintaxe dedicada para funções genéricas, classes genéricas e apelidos de tipo genérico:

```
class Sequence[T]:  # T é um TypeVar
    ...
```

Essa sintaxe também pode ser usada para criar tipos variáveis delimitados e tipos variáveis restritos:

```
class StrSequence[S: str]:  # S é a TypeVar com como limite su
    ...                     # podemos dizer que S é "delimitad


class StrOrBytesSequence[A: (str, bytes)]:  # A é uma TypeVar
    ...
```

No entanto, se desejado, tipos variáveis reutilizáveis também podem ser construídas manualmente, como:

```
T = TypeVar('T') # Pode ser qualquer coisa
S = TypeVar('S', bound=str) # Pode ser qualquer subtipo de str
A = TypeVar('A', str, bytes) # Deve ser exatamente str ou byte
```

Tipos variáveis existem principalmente para o benefício de verificadores de tipo estático. Eles servem como parâmetros para tipos genéricos, bem como para definições de função genérica e apelidos de tipo. Consulte

`Generic` para obter mais informações sobre genérico. Funções genéricas funcionam da seguinte forma:

```
def repeat[T](x: T, n: int) -> Sequence[T]:
    """Retorna uma lista contendo n referências para x."""
    return [x]*n


def print_capitalized[S: str](x: S) -> S:
    """Exibe x capitalizado, e retorna x."""
    print(x.capitalize())
    return x


def concatenate[A: (str, bytes)](x: A, y: A) -> A:
    """Concatena dois objetos string ou bytes."""
    return x + y
```

Observe que tipos variáveis podem ser *delimitados*, *restritos*, nenhum dos dois, mas não podem ser *ambos*.

A variância de tipos variáveis é inferida pelos verificadores de tipo quando eles são criados por meio do sintaxe de parâmetro de tipo ou quando `infer_variance=True` é passado. Os tipos variáveis criados manualmente podem ser explicitamente marcados como covariante ou contravariantes ao passar `covariant=True` ou `contravariant=True`. Por padrão, os tipos variáveis criados manualmente são invariante. Consulte **PEP 484** [https://peps.python.org/pep-0484/] e **PEP 695** [https://peps.python.org/pep-0695/] para obter mais detalhes.

Tipos variáveis delimitados e tipos variáveis restritos têm semânticas diferentes de várias formas importantes. Usar um tipo variável *delimitado* significa que a `TypeVar` será resolvido usando o tipo mais específico possível:

```
x = print_capitalized('a string')
reveal_type(x)  # tipo revelado é str

class StringSubclass(str):
    pass
```

```
y = print_capitalized(StringSubclass('another string'))
reveal_type(y)  # tipo revelado é StringSubclass

z = print_capitalized(45)  # erro: int não é subtipo de str
```

O limite superior de um tipo variável pode ser um tipo concreto, um tipo abstrato (classe base abstrata ou protocolo) ou até mesmo uma união de tipos:

```
# Pode ser qualquer coisa com um método __abs__
def print_abs[T: SupportsAbs](arg: T) -> None:
    print("Absolute value:", abs(arg))

U = TypeVar('U', bound=str|bytes)  # Pode ser qualquer subtipo
V = TypeVar('V', bound=SupportsAbs)  # Pode ser qualquer coisa
```

Porém, usar um tipo variável *restrito* significa que a `TypeVar` só poderá ser resolvida como sendo exatamente uma das restrições dadas:

```
a = concatenate('one', 'two')
reveal_type(a) # o tipo revelado é str

b = concatenate(StringSubclass('one'), StringSubclass('two'))
reveal_type(b) # o tipo revelado é str, apesar de StringSubcla

c = concatenate('one', b'two') # erro: tipo variável 'A' pode
```

Em tempo de execução, `isinstance(x, T)` levantará **`TypeError`**.

__name__

O nome do tipo variável.

__covariant__

Se o tipo variável foi explicitamente marcado como covariante.

__contravariant__

Se o tipo variável foi explicitamente marcado como contravariante.

__infer_variance__

Se a variância de tipo variável deve ser inferida por verificadores de tipo.

Adicionado na versão 3.12.

__bound__

A limite superior do tipo variável, se houver.

Alterado na versão 3.12: Para tipos variáveis criado por meio da sintaxe de parâmetro de tipo , a limite é avaliado somente quando o atributo é acessado, não quando o tipo variável é criado (consulte Avaliação preguiçosa).

evaluate_bound()

Uma função de avaliação correspondente ao atributo `__bound__`. Quando chamado diretamente, este método oferece suporte apenas ao formato `VALUE`, que equivale a acessar o atributo `__bound__` diretamente, mas o objeto do método pode ser passado para `annotationlib.call_evaluate_function()` para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

__constraints__

Um tupla contendo as restrições do tipo variável, se houver.

Alterado na versão 3.12: Para tipos variáveis criado por meio da sintaxe de parâmetro de tipo , a restrição é avaliada somente quando o atributo é acessado, não quando o tipo variável é criado (consulte Avaliação preguiçosa).

evaluate_constraints()

Uma função de avaliação correspondente ao atributo `__constraints__`. Quando chamado diretamente, este método oferece suporte apenas ao formato `VALUE`, que equivale a acessar o atributo `__constraints__` diretamente, mas o objeto do método pode ser passado para `annotationlib.call_evaluate_function()` para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

__default__

O padrão valor do tipo variável, ou `typing.NoDefault` se ele não tiver padrão.

Adicionado na versão 3.13.

evaluate_default()

Uma função de avaliação correspondente ao atributo `__default__`. Quando chamado diretamente, este método oferece suporte apenas ao formato `VALUE`, que equivale a acessar o atributo `__default__` diretamente, mas o objeto do método pode ser passado para `annotationlib.call_evaluate_function()` para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

has_default()

Retorna se o tipo variável tem ou não um valor padrão. Isso equivale a verificar se `__default__` não é o singleton `typing.NoDefault`, exceto por não forçar a avaliação instantânea do valor padrão que é avaliado preguiçosamente.

Adicionado na versão 3.13.

Alterado na versão 3.12: Tipos variáveis agora podem ser declarados usando a sintaxe de parâmetro de tipo introduzida por **PEP 695** [https://peps.python.org/pep-0695/]. O parâmetro `infer_variance` foi adicionado.

Alterado na versão 3.13: Adiciona suporte a valores padrão.

class typing.TypeVarTuple(*name*, **, bound=None, covariant=False, contravariant=False, infer_variance=False, default=typing.NoDefault*)

Tupla de tipo variável. Uma forma especializada de tipo variável que permite genéricos *variádicos*.

Tuplas de tipo variável podem ser declarada em listas de parâmetros de tipo usando um único asterisco (`*`) antes do nome:

```
def move_first_element_to_last[T, *Ts](tup: tuple[T, *Ts]) ->
    return (*tup[1:], tup[0])
```

Ou invocando o construtor `TypeVarTuple` explicitamente:

```
T = TypeVar("T")
Ts = TypeVarTuple("Ts")

def move_first_element_to_last(tup: tuple[T, *Ts]) -> tuple[*T
    return (*tup[1:], tup[0])
```

Um tipo variável normal permite a parametrização com um único tipo. Uma tupla de tipos variáveis, por outro lado, permite a parametrização com um número *arbitrário* de tipos, agindo como um número *arbitrário* de tipos variáveis envolvidos em um tupla. Por exemplo:

```
# T é limitado a int, Ts é limitado a ()
# Valor retornado é (1,), que tem tipo tuple[int]
move_first_element_to_last(tup=(1,))

# T é limitado a int, Ts é limitado a (str,)
# Valor retornado é ('spam', 1), que tem tipo tuple[str, int]
move_first_element_to_last(tup=(1, 'spam'))

# T é limitado a int, Ts é limitado a (str, float)
# Valor retornado é ('spam', 3.0, 1), que tem tipo tuple[str,
move_first_element_to_last(tup=(1, 'spam', 3.0))

# Isso é um erro para o verificador de tipos (e falha em tempo
# pois tuple[()] não é compatível com tuple[T, *Ts]
# (pelo menos um elemento é necessário)
move_first_element_to_last(tup=())
```

Observe o uso do operador de desempacotamento `*` em `tuple[T, *Ts]`. Conceitualmente, você pode interpretar `Ts` como uma tupla de tipos variáveis `(T1, T2, ...)`. Então o tipo `tuple[T, *Ts]` se tornaria `tuple[T, *(T1, T2, ...)]`, que equivale a `tuple[T, T1, T2, ...]`.

(Note que, em versões mais antigas de Python, você pode encontrar isso escrito com **`Unpack`** em vez de `Unpack[Ts]`.)

Tuplas de tipos variáveis devem *sempre* ser desempacotadas. Isso ajuda a distinguir entre tuplas de tipos variáveis e tipos variáveis normais:

```
x: Ts          # Inválido
x: tuple[Ts]   # Inválido
x: tuple[*Ts]  # O jeito correto
```

As tuplas de tipos variáveis podem ser usadas no mesmo contexto que tipos variáveis normais. Por exemplo, em argumentos, tipos de retorno e definições de classes:

```
class Array[*Shape]:
    def __getitem__(self, key: tuple[*Shape]) -> float: ...
    def __abs__(self) -> "Array[*Shape]": ...
    def get_shape(self) -> tuple[*Shape]: ...
```

As tuplas de tipos variáveis podem ser combinadas com tipos variáveis normais:

```
class Array[DType, *Shape]:  # Tudo certo aqui
    pass

class Array2[*Shape, DType]:  # Tudo certo aqui também
    pass

class Height: ...
class Width: ...

float_array_1d: Array[float, Height] = Array()     # Totalment
int_array_2d: Array[int, Height, Width] = Array()  # Sim, cert
```

No entanto, observe que no máximo uma tupla de tipos variáveis pode aparecer em uma única lista de argumentos de tipo ou parâmetros de tipo:

```
x: tuple[*Ts, *Ts]             # Inválido
class Array[*Shape, *Shape]:  # Inválido
    pass
```

Por fim, uma tupla de tipos variáveis desempacotada pode ser usada como anotação de tipo de `*args`:

```
def call_soon[*Ts](
    callback: Callable[[*Ts], None],
    *args: *Ts
) -> None:
    ...
    callback(*args)
```

Ao contrário do que acontece com anotações não desempacotadas de `*args` - por exemplo, `*args: int`, que especificaria que *todos* os argumentos são `int` -, `*args: *Ts` permite referenciar os tipos de cada argumento em `*args` *individualmente*. Isso nos permite garantir que os tipos em `*args` passados para `call_soon` correspondem aos tipos dos argumentos (posicionais) de `callback`.

Consulte a **PEP 646** [https://peps.python.org/pep-0646/] para mais detalhes sobre tuplas de tipos variáveis.

__name__

O nome da tupla de tipos variáveis.

__covariant__

Whether the type variable tuple has been explicitly marked as covariant.

Adicionado na versão 3.15.

__contravariant__

Whether the type variable tuple has been explicitly marked as contravariant.

Adicionado na versão 3.15.

__infer_variance__

Whether the type variable tuple's variance should be inferred by type checkers.

Adicionado na versão 3.15.

__default__

O valor padrão da tupla de tipos variáveis, ou `typing.NoDefault` se não existir padrão.

Adicionado na versão 3.13.

evaluate_default()

Uma função de avaliação correspondente ao atributo `__default__`. Quando chamado diretamente, este método oferece suporte apenas ao formato `VALUE`, que equivale a acessar o atributo `__default__` diretamente, mas o objeto do método pode ser passado para `annotationlib.call_evaluate_function()` para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

has_default()

Retorna se a tupla de tipos variáveis tem ou não um valor padrão. Isso equivale a verificar se `__default__` não é o singleton `typing.NoDefault`, exceto por não forçar a avaliação instantânea do valor padrão que é avaliado preguiçosamente.

Adicionado na versão 3.13.

Type variable tuples created with `covariant=True` or `contravariant=True` can be used to declare covariant or contravariant generic types. The `bound` argument is also accepted, similar to `TypeVar`, but its actual semantics are yet to be decided.

Adicionado na versão 3.11.

Alterado na versão 3.12: Tuplas de tipos variáveis agora podem ser declaradas usando a sintaxe de parâmetros de tipo introduzido pela **PEP 695** [https://peps.python.org/pep-0695/].

Alterado na versão 3.13: Adiciona suporte a valores padrão.

Alterado na versão 3.15: Added support for the `bound`, `covariant`, `contravariant`, and `infer_variance` parameters.

class typing.ParamSpec(*name*, ***, *bound=None*, *covariant=False*, *contravariant=False*, *infer_variance=False*, *default=typing.NoDefault*)

Variável de especificação de parâmetro. Uma versão especializada de tipos variáveis.

Em listas de parâmetros de tipo as especificações de parâmetros podem ser declaradas com dois asteriscos (`**`):

```
type IntFunc[**P] = Callable[P, int]
```

Para compatibilidade com Python 3.11 e versões anteriores, os objetos `ParamSpec` também podem ser criados da seguinte forma:

```
P = ParamSpec('P')
```

Variáveis de especificação de parâmetro existem principalmente para o benefício de verificadores de tipo estático. São usadas para encaminhar os tipos de parâmetros de um chamável para outro chamável – um padrão comumente encontrado em funções e decoradores de ordem superior. Só são válidas quando usados em `Concatenate`, ou como o primeiro argumento para `Callable`, ou como parâmetro para genéricos definidos pelo usuário. Consulte **`Generic`** para obter mais informações sobre tipos genéricos.

Por exemplo, para adicionar um registro básico de eventos a uma função, é possível criar um decorador `add_logging` para registrar chamadas de função. A variável de especificação de parâmetro informa ao verificador de tipos que o chamável passado para o decorador e o novo chamável retornado por ele têm parâmetros de tipo interdependentes:

```
from collections.abc import Callable
import logging

def add_logging[T, **P](f: Callable[P, T]) -> Callable[P, T]:
    '''Um decorador com segurança de tipos que acrescenta regi
    def inner(*args: P.args, **kwargs: P.kwargs) -> T:
        logging.info(f'{f.__name__} was called')
```

```
        return f(*args, **kwargs)
    return inner

@add_logging
def add_two(x: float, y: float) -> float:
    '''Soma dois números.'''
    return x + y
```

Sem o `ParamSpec`, a maneira mais simples de anotar isso anteriormente era usar um **`TypeVar`** com o limite superior `Callable[..., Any]`. No entanto, isso causa dois problemas:

1. O verificador de tipos não consegue verificar a função `inner`, porque `*args` e `**kwargs` precisam ter tipo **`Any`**.
2. **`cast()`** pode ser exigida no corpo do decorador `add_logging` ao retornar a função `inner`, ou o verificador de tipo estático deverá ser instruído a ignorar o `return inner`.

args

kwargs

Como `ParamSpec` captura tanto parâmetros posicionais quanto parâmetros nomeados, `P.args` e `P.kwargs` podem ser usados para dividir um `ParamSpec` em seus componentes. `P.args` representa a tupla de parâmetros posicionais em uma determinada chamada e só deve ser usada para anotar `*args`. `P.kwargs` representa o mapeamento de parâmetros nomeados para seus valores em uma determinada chamada, e só deve ser usado para anotar `**kwargs`. Ambos os atributos exigem que o parâmetro anotado esteja em escopo. Em tempo de execução, `P.args` e `P.kwargs` são instâncias, respectivamente, de **`ParamSpecArgs`** e **`ParamSpecKwargs`**.

__name__

O nome da especificação de parâmetros.

__covariant__

Whether the parameter specification has been explicitly marked as covariant.

__contravariant__

Whether the parameter specification has been explicitly marked as contravariant.

__infer_variance__

Whether the parameter specification's variance should be inferred by type checkers.

Adicionado na versão 3.12.

__default__

O valor padrão da especificação de parâmetro, ou `typing.NoDefault` se não tiver padrão.

Adicionado na versão 3.13.

evaluate_default()

Uma função de avaliação correspondente ao atributo `__default__`. Quando chamado diretamente, este método oferece suporte apenas ao formato `VALUE`, que equivale a acessar o atributo `__default__` diretamente, mas o objeto do método pode ser passado para `annotationlib.call_evaluate_function()` para avaliar o valor em um formato diferente.

Adicionado na versão 3.14.

has_default()

Retorna se a especificação de parâmetros tem ou não um valor padrão. Isso equivale a verificar se `__default__` não é o singleton `typing.NoDefault`, exceto por não forçar a avaliação instantânea do valor padrão que é avaliado preguiçosamente.

Adicionado na versão 3.13.

Variáveis de especificação de parâmetros criadas com `covariant=True` ou `contravariant=True` podem ser usadas para declarar tipos genéricos covariantes ou contravariantes. O argumento `bound` também é aceito, semelhante ao `TypeVar`. Porém, a semântica real dessas palavras reservadas ainda não foi decidida.

Adicionado na versão 3.10.

Alterado na versão 3.12: Especificações de parâmetros agora podem ser declaradas usando a sintaxe de parâmetros de tipo introduzido pela **PEP 695** [https://peps.python.org/pep-0695/].

Alterado na versão 3.13: Adiciona suporte a valores padrão.

Nota

Somente variáveis de especificação de parâmetro definidas em escopo global podem ser serializadas com pickle.

Ver também

- **PEP 612** [https://peps.python.org/pep-0612/] – Variáveis de especificação de parâmetro (a PEP que introduz `ParamSpec` e `Concatenate`)
- **`Concatenate`**
- Anotações de objetos chamáveis

class typing.ParamSpecArgs
class typing.ParamSpecKwargs

Tipos dos argumentos e dos argumentos nomeados de um **`ParamSpec`**. O atributo `P.args` de um `ParamSpec` é uma instância de `ParamSpecArgs`, e o atributo `P.kwargs` é uma instância de `ParamSpecKwargs`. São destinados à introspecção em tempo de execução, e não têm nenhum significado especial para o verificador de tipo estático.

Chamar **`get_origin()`** em qualquer um desses objetos retornará o `ParamSpec` original:

```
>>> from typing import ParamSpec, get_origin
>>> P = ParamSpec("P")
>>> get_origin(P.args) is P
True
>>> get_origin(P.kwargs) is P
True
```

Adicionado na versão 3.10.

class typing.TypeAliasType(*name*, *value*, ***, *type_params=()*, *qualname=None*)

O tipo de apelidos de tipo criados pela instrução `type`.

Exemplo:

```
>>> type Alias = int
>>> type(Alias)
<class 'typing.TypeAliasType'>
```

Adicionado na versão 3.12.

__name__

O nome do apelido de tipo:

```
>>> type Alias = int
>>> Alias.__name__
'Alias'
```

__qualname__

The qualified name of the type alias:

```
>>> class Class:
...     type Alias = int
...
>>> Class.Alias.__qualname__
'Class.Alias'
```

Adicionado na versão 3.15.

__module__

O nome do módulo no qual o apelido de tipo foi definido:

```
>>> type Alias = int
>>> Alias.__module__
'__main__'
```

This attribute is writable.

Alterado na versão 3.15: The attribute is now writable.

__type_params__

Os parâmetros de tipo do apelido de tipo, ou uma tupla vazia se o apelido não for genérico:

```
>>> type ListOrSet[T] = list[T] | set[T]
>>> ListOrSet.__type_params__
(T,)
>>> type NotGeneric = int
>>> NotGeneric.__type_params__
()
```

__value__

O valor do apelido de tipo. Isso é uma avaliação preguiçosa, portanto, os nomes usados na definição do apelido não são resolvidos até que o atributo `__value__` seja acessado:

```
>>> type Mutually = Recursive
>>> type Recursive = Mutually
>>> Mutually
Mutually
>>> Recursive
Recursive
>>> Mutually.__value__
Recursive
>>> Recursive.__value__
Mutually
```

evaluate_value()

Uma função de avaliação correspondente ao atributo **`__value__`**. Quando chamado diretamente, este método oferece suporte apenas ao formato **`VALUE`**, que equivale a acessar o atributo **`__value__`** diretamente, mas o objeto do método pode ser passado para **`annotationlib.call_evaluate_function()`** para avaliar o valor em um formato diferente.

```
>>> type Alias = undefined
>>> Alias.__value__
Traceback (most recent call last):
...
NameError: name 'undefined' is not defined
>>> from annotationlib import Format, call_evaluate_functic
```

```
>>> Alias.evaluate_value(Format.VALUE)
Traceback (most recent call last):
...
NameError: name 'undefined' is not defined
>>> call_evaluate_function(Alias.evaluate_value, Format.FOR
ForwardRef('undefined')
```

Adicionado na versão 3.14.

Descompactando

Apelidos de tipo oferecem suporte a desempacotamento em estrela usando a sintaxe `*Alias`. Isso equivale a usar `Unpack[Alias]` diretamente:

```
>>> type Alias = tuple[int, str]
>>> type Unpacked = tuple[bool, *Alias]
>>> Unpacked.__value__
tuple[bool, typing.Unpack[Alias]]
```

Adicionado na versão 3.14.

Outras diretivas especiais

Essas funções e classes não devem ser usadas diretamente como anotações. O objetivo é que sejam blocos de construção para criar e declarar tipos.

class typing.NamedTuple

Versão tipada de `collections.namedtuple()`.

Uso:

```
class Employee(NamedTuple):
    name: str
    id: int
```

Isso equivale a:

```
Employee = collections.namedtuple('Employee', ['name', 'id'])
```

Para dar um valor padrão a um campo, você pode atribuir um valor a ele no corpo da classe:

```
class Employee(NamedTuple):
    name: str
    id: int = 3

employee = Employee('Guido')
assert employee.id == 3
```

Campos com valor padrão devem vir depois de quaisquer campos sem valor padrão.

The types for each field name can be retrieved by calling `annotationlib.get_annotations()` on the resulting class. (The field names are in the `_fields` attribute and the default values are in the `_field_defaults` attribute, both of which are part of the `namedtuple()` API.)

Subclasses de `NamedTuple` também podem ter strings de documentação e métodos:

```
class Employee(NamedTuple):
    """Representa um funcionário."""
    name: str
    id: int = 3

    def __repr__(self) -> str:
        return f'<Employee {self.name}, id={self.id}>'
```

Subclasses de `NamedTuple` podem ser genéricas:

```
class Group[T](NamedTuple):
    key: T
    group: list[T]
```

Uso retrocompatível:

```
# Para criar uma NamedTuple genérica em Python 3.11
T = TypeVar("T")

class Group(NamedTuple, Generic[T]):
```

```
    key: T
    group: list[T]

# Também há suporte para a sintaxe funcional
Employee = NamedTuple('Employee', [('name', str), ('id', int)]
```

Alterado na versão 3.6: Adiciona suporte à sintaxe de anotação de variáveis da **PEP 526** [https://peps.python.org/pep-0526/].

Alterado na versão 3.6.1: Adiciona suporte a valores padrões, métodos, e docstrings.

Alterado na versão 3.8: Os atributos `_field_types` e `__annotations__` agora são dicionários regulares em vez de instâncias de `OrderedDict`.

Alterado na versão 3.9: Remove o atributo `_field_types` em favor do atributo mais padronizado `__annotations__` que tem as mesmas informações.

Alterado na versão 3.9: `NamedTuple` is now a function rather than a class. It can still be used as a class base, as described above.

Alterado na versão 3.11: Adiciona suporte a tuplas nomeadas genéricas.

Alterado na versão 3.14: O uso de **`super()`** (e a variável de clausura `__class__`) em métodos de subclasses de `NamedTuple` não é suportado e causa um **`TypeError`**.

class typing.NewType(*name*, *tp*)

Classe auxiliar para criar tipos únicos com baixo custo.

A `NewType` is considered a distinct type by a type checker. At runtime, however, calling a `NewType` returns its argument unchanged.

Uso:

```
UserId = NewType('UserId', int) # Declare o NewType "UserId"
first_user = UserId(1) # "UserId" retorna o argumento inaltera
```

__module__
O nome do módulo no qual o novo tipo é definido.

__name__
O nome do novo tipo.

__supertype__
O tipo na qual o novo tipo é baseado.

Adicionado na versão 3.5.2.

Alterado na versão 3.10: `NewType` agora é uma classe em vez de uma função.

class typing.Protocol(*Generic*)
Classe base para classes de protocolo.

Classes de protocolo são definidas assim:

```
class Proto(Protocol):
    def meth(self) -> int:
        ...
```

Essas classes são usadas principalmente com verificadores de tipo estático que reconhecem a subtipagem estrutural (tipagem pato estática). Por exemplo:

```
class C:
    def meth(self) -> int:
        return 0

def func(x: Proto) -> int:
    return x.meth()

func(C())  # Passa na verificação de tipos
```

See **PEP 544** [https://peps.python.org/pep-0544/] for more details. Protocol classes decorated with `@runtime_checkable` (described later) act as simple-minded runtime protocols that check only the presence of given attributes, ignoring their type signatures. Protocol classes without this decorator

cannot be used as the second argument to **isinstance()** or **issubclass()**.

Classes de protocolo podem ser genéricas. Por exemplo:

```
class GenProto[T](Protocol):
    def meth(self) -> T:
        ...
```

Em códigos que precisam ser compatíveis com Python 3.11 ou versões anteriores, protocolos genéricos podem ser escritos da seguinte forma:

```
T = TypeVar("T")

class GenProto(Protocol[T]):
    def meth(self) -> T:
        ...
```

Adicionado na versão 3.8.

Descontinuado desde a versão 3.15, será removido na versão 3.20: It is deprecated to call **isinstance()** and **issubclass()** checks on protocol classes that were not explicitly decorated with **runtime_checkable()** but that inherit from a runtime-checkable protocol class. This will throw a **TypeError** in Python 3.20.

@typing.runtime_checkable

Marca uma classe de protocolo como um protocolo de tempo de execução.

Such a protocol can be used with **isinstance()** and **issubclass()**. This allows a simple-minded structural check, very similar to “one-trick ponies” in **collections.abc** such as **Iterable**. For example:

```
@runtime_checkable
class Closable(Protocol):
    def close(self): ...

assert isinstance(open('/some/file'), Closable)

@runtime_checkable
```

```
class Named(Protocol):
    name: str

import threading
assert isinstance(threading.Thread(name='Bob'), Named)
```

Runtime checkability of protocols is not inherited. A subclass of a runtime-checkable protocol is only runtime-checkable if it is explicitly marked as such, regardless of class hierarchy:

```
@runtime_checkable
class Iterable(Protocol):
    def __iter__(self): ...

# Without @runtime_checkable, Reversible would no longer be ru
@runtime_checkable
class Reversible(Iterable, Protocol):
    def __reversed__(self): ...
```

Este decorador levanta `TypeError` quando aplicado a uma classe não-protocolo.

Nota

`@runtime_checkable` will check only the presence of the required methods or attributes, not their type signatures or types. For example, `ssl.SSLObject` is a class, therefore it passes an `issubclass()` check against Callable. However, the `ssl.SSLObject.__init__` method exists only to raise a `TypeError` with a more informative message, therefore making it impossible to call (instantiate) `ssl.SSLObject`.

Nota

uma verificação com `isinstance()` sobre um protocolo verificável em tempo de execução pode ser surpreendentemente lenta se comparada a uma verificação `isinstance()` sobre outros tipos de classe. Considere usar expressões alternativas, como chamar a função `hasattr()` para realizar verificações estruturais em código sensível a desempenho.

Adicionado na versão 3.8.

Alterado na versão 3.12: A implementação interna do `isinstance()` agora verifica os protocolos verificáveis em tempo de execução usando `inspect.getattr_static()` para procurar o atributo (anteriormente, era usado `hasattr()`). Como resultado, alguns objetos que costumavam ser considerados instâncias de um protoloco verifiável em tempo de execução podem não ser mais considerados instâncias desse protocolo em Python 3.12+, e vice-versa. É improvável que a maioria dos usuários seja afetada por essa alteração.

Alterado na versão 3.12: Os membros de um protocolo verificável em tempo de execução agora serão considerados "congelados" em tempo de execução assim que a classe for criada. A alteração dinâmica de atributos de um protocolo em tempo de execução ainda funcionará, mas não terá impacto nas verificações de `isinstance()` ao comparar objetos com o protocolo. Consulte o O que há de novo no Python 3.12 para mais detalhes.

Descontinuado desde a versão 3.15, será removido na versão 3.20: It is deprecated to call `isinstance()` and `issubclass()` checks on protocol classes that were not explicitly decorated with **`runtime_checkable()`** but that inherit from a runtime-checkable protocol class. This will throw a `TypeError` in Python 3.20.

class typing.TypedDict(*dict*)

Special construct to add type hints to a dictionary. At runtime "**`TypedDict`** instances" are simply `dicts`.

`TypedDict` declara um tipo dicionário que espera que todas as suas instâncias tenham um determinado conjunto de chaves, onde cada chave está associada a um valor de um tipo consistente. Essa expectativa não é verificada em tempo de execução, mas é imposta apenas por verificadores de tipos. Modo de usar:

```
class Point2D(TypedDict):
    x: int
    y: int
    label: str

a: Point2D = {'x': 1, 'y': 2, 'label': 'good'}  # OK
```

```
b: Point2D = {'z': 3, 'label': 'bad'}           # Falha na ver

assert Point2D(x=1, y=2, label='first') == dict(x=1, y=2, labe
```

Um modo alternativo de criar um `TypedDict` é usando a sintaxe de chamada de função. O segundo argumento deve ser um **`dict`**:

```
Point2D = TypedDict('Point2D', {'x': int, 'y': int, 'label': s
```

Esta sintaxe funcional permite definirmos chaves usando identificadores inválidos, por exemplo, por serem palavras-chave ou conterem hífens, ou quando os nomes das chaves não devem ser desconfigurados como nomes privados comuns:

```
# levanta SyntaxError
class Point2D(TypedDict):
    in: int  # 'in' é uma palavra reservada
    x-y: int  # nome com hífen

class Definition(TypedDict):
    __schema: str  # desconfigurado para `_Definition__schema`

# OK, sintaxe funcional
Point2D = TypedDict('Point2D', {'in': int, 'x-y': int})
Definition = TypedDict('Definition', {'__schema': str})  # não
```

Por padrão, todas as chaves devem estar presentes em um `TypedDict`. É possível marcar chaves individuais como não-obrigatórias usando **`NotRequired`**:

```
class Point2D(TypedDict):
    x: int
    y: int
    label: NotRequired[str]

# Sintaxe alternativa
Point2D = TypedDict('Point2D', {'x': int, 'y': int, 'label': N
```

Isso significa que um `TypedDict` `Point2D` pode ter a chave `label` omitida.

Também é possível marcar todas as chaves como não necessárias por padrão, especificando a totalidade como `False`:

```
class Point2D(TypedDict, total=False):
    x: int
    y: int

# Sintaxe alternativa
Point2D = TypedDict('Point2D', {'x': int, 'y': int}, total=Fal
```

Isso significa que um `TypedDict` `Point2D` pode ter qualquer uma de suas chaves omitidas. Espera-se que um verificador de tipos apenas permita os literais `False` ou `True` como valores do argumento `total`. `True` é o padrão, e todos os itens definidos no corpo da classe tornam-se obrigatórios.

As chaves individuais de um `TypedDict` com `total=False` podem ser marcadas como obrigatórias usando **`Required`**:

```
class Point2D(TypedDict, total=False):
    x: Required[int]
    y: Required[int]
    label: str

# Sintaxe alternativa
Point2D = TypedDict('Point2D', {
    'x': Required[int],
    'y': Required[int],
    'label': str
}, total=False)
```

É possível que um tipo `TypedDict` herde de um ou mais tipos `TypedDict` usando a sintaxe baseada em classes. Modo de usar:

```
class Point3D(Point2D):
    z: int
```

`Point3D` tem três itens: `x`, `y` e `z`. Equivale a esta definição:

```
class Point3D(TypedDict):
    x: int
```

```
    y: int
    z: int
```

By default, a `TypedDict` is open, meaning that it may contain additional keys at runtime beyond those defined in the class body. The *closed* class argument can be used to control this; if `closed=True`, the `TypedDict` cannot contain additional keys.

```
class ClosedPoint(TypedDict, closed=True):
    x: int
    y: int

class ClosedPoint3D(ClosedPoint):  # type checker error: canno
    z: int
```

Setting `closed=False` explicitly requests the default open behavior. If the argument is not passed, this state is inherited from the parent class.

In addition to being open or closed, a `TypedDict` can also be configured to have extra items. If the *extra_items* class argument is set to a type, the `TypedDict` can contain arbitrary additional keys, but the values of those keys must be of the specified type.

```
class ExtraItemsPoint(TypedDict, extra_items=int):
    x: int
    y: int

point: ExtraItemsPoint = {'x': 1, 'y': 2, 'anything': 3}  # OK
```

The *extra_items* argument is also inherited through subclassing. It is unset by default, and it may not be used together with the *closed* argument.

Um `TypedDict` não pode herdar de uma classe não `TypedDict`, exceto **`Generic`**. Por exemplo:

```
class X(TypedDict):
    x: int

class Y(TypedDict):
    y: int
```

```
class Z(object): pass  # Uma classe não TypedDict

class XY(X, Y): pass  # OK

class XZ(X, Z): pass  # levanta TypeError
```

Um `TypedDict` pode ser genérico:

```
class Group[T](TypedDict):
    key: T
    group: list[T]
```

Para criar um `TypedDict` genérico que seja compatível com Python 3.11 ou inferior, herde de **`Generic`** explicitamente:

```
T = TypeVar("T")

class Group(TypedDict, Generic[T]):
    key: T
    group: list[T]
```

A `TypedDict` can be introspected via **`annotationlib.get_annotations()`** (see Boas práticas para anotações for more information on annotations best practices) and the following attributes:

__total__

`Point2D.__total__` fornece o valor do argumento `total`. Exemplo:

```
>>> from typing import TypedDict
>>> class Point2D(TypedDict): pass
>>> Point2D.__total__
True
>>> class Point2D(TypedDict, total=False): pass
>>> Point2D.__total__
False
>>> class Point3D(Point2D): pass
>>> Point3D.__total__
True
```

Esse atributo reflete *apenas* o valor do argumento `total` para a classe `TypedDict` atual, e não que a classe é semanticamente total. Por

exemplo, um `TypedDict` com `__total__` definido como `True` pode ter chaves marcadas com **`NotRequired`**, ou pode herdar de outro `TypedDict` com `total=False`. Portanto, geralmente é melhor usar **`__required_keys__`** e **`__optional_keys__`** para introspecção.

__required_keys__

Adicionado na versão 3.9.

__optional_keys__

`Point2D.__required_keys__` e `Point2D.__optional_keys__` retornam objetos **`frozenset`** contendo chaves obrigatórias e opcionais, respectivamente.

As chaves marcadas com **`Required`** sempre aparecerão em `__required_keys__` e as chaves marcadas com **`NotRequired`** sempre aparecerão em `__optional_keys__`.

For backwards compatibility with Python 3.10 and below, it is also possible to use inheritance to declare both required and non-required keys in the same `TypedDict`. This is done by declaring a `TypedDict` with one value for the `total` argument and then inheriting from it in another `TypedDict` with a different value for `total`:

```
>>> class Point2D(TypedDict, total=False):
...     x: int
...     y: int
...
>>> class Point3D(Point2D):
...     z: int
...
>>> Point3D.__required_keys__ == frozenset({'z'})
True
>>> Point3D.__optional_keys__ == frozenset({'x', 'y'})
True
```

Adicionado na versão 3.9.

Nota

Se `from __future__ import annotations` for usado ou se anotações forem fornecidas como strings, as anotações não serão

avaliadas quando o `TypedDict` for definido. Portanto, a introspecção em tempo de execução da qual `__required_keys__` e `__optional_keys__` dependem pode não funcionar corretamente, e os valores dos atributos podem estar incorretos.

__readonly_keys__

Um **`frozenset`** contendo os nomes de todas as chaves somente para leitura. As chaves são somente para leitura se tiverem o qualificador **`ReadOnly`**.

Adicionado na versão 3.13.

__mutable_keys__

Um **`frozenset`** contendo os nomes de todas as chaves mutáveis. As chaves são mutáveis se não tiverem o qualificador **`ReadOnly`**.

Adicionado na versão 3.13.

__closed__

The value of the *closed* class argument. It can be `True`, `False`, or **`None`**.

__extra_items__

The value of the *extra_items* class argument. It can be a valid type or **`NoExtraItems`**.

Consulte a seção TypedDict [https://typing.python.org/en/latest/spec/typeddict.html#typeddict] na documentação de tipagem para obter mais exemplos e regras detalhadas.

Adicionado na versão 3.8.

Alterado na versão 3.9: `TypedDict` is now a function rather than a class. It can still be used as a class base, as described above.

Alterado na versão 3.11: Adicionado suporte para marcar chaves individuais como **`Required`** ou **`NotRequired`**. Consulte **PEP 655** [https://peps.python.org/pep-0655/].

Alterado na versão 3.11: Adicionado suporte para `TypedDicts` genéricos.

Alterado na versão 3.13: Removeu suporte para o método de argumento nomeado para criar `TypedDicts`.

Alterado na versão 3.13: Support for the **`ReadOnly`** qualifier was added. See **PEP 705** [https://peps.python.org/pep-0705/].

Alterado na versão 3.15: Support for the *closed* and *extra_items* class arguments was added. See **PEP 728** [https://peps.python.org/pep-0728/].

Protocolos

Os protocolos a seguir são fornecidos pelo módulo **`typing`**. Todos são decorados com **`@runtime_checkable`**.

class typing.SupportsAbs

A protocol with one abstract method `__abs__` that is covariant in its return type.

class typing.SupportsBytes

A protocol with one abstract method `__bytes__`.

class typing.SupportsComplex

A protocol with one abstract method `__complex__`.

class typing.SupportsFloat

A protocol with one abstract method `__float__`.

class typing.SupportsIndex

A protocol with one abstract method `__index__`.

Adicionado na versão 3.8.

class typing.SupportsInt

A protocol with one abstract method `__int__`.

class typing.SupportsRound

A protocol with one abstract method `__round__` that is covariant in its return type.

ABCs e protocolos para trabalhar com E/S

class typing.IO[*AnyStr*]
class typing.TextIO
class typing.BinaryIO

A classe genérica `IO[AnyStr]` e suas subclasses `TextIO(IO[str])` e `BinaryIO(IO[bytes])` representam os tipos de fluxos de E/S, como os retornados por **`open()`**. Note que essas classes não são protocolos e sua interface é bem ampla.

Os protocolos **`io.Reader`** e **`io.Writer`** oferecem uma alternativa mais simples para tipos de argumentos, quando apenas os métodos `read()` ou `write()` são acessados, respectivamente:

```
def read_and_write(reader: Reader[str], writer: Writer[bytes]):
    data = reader.read()
    writer.write(data.encode())
```

Considere também usar **`collections.abc.Iterable`** para iterar sobre as linhas de um fluxo de entrada:

```
def read_config(stream: Iterable[str]):
    for line in stream:
        ...
```

Funções e decoradores

typing.cast(*typ*, *val*)

Converta um valor em um tipo.

Isso retorna o valor inalterado. Para o verificador de tipos, isso indica que o valor de retorno tem o tipo designado, mas em tempo de execução não verificamos nada intencionalmente (queremos que isso seja o mais rápido possível).

typing.assert_type(*val*, *typ*, /)

Pede para um verificador de tipo estático confirmar se *val* tem tipo inferido *typ*.

Essa função faz nada em tempo de execução: ela retorna o primeiro argumento inalterado, sem verificações ou efeitos colaterais, independentemente do tipo real do argumento.

Quando um verificador de tipo estático encontra uma chamada para `assert_type()`, ele emite um erro se o valor não for do tipo especificado:

```
def greet(name: str) -> None:
    assert_type(name, str) # OK, o tipo inferido de `name` é `
    assert_type(name, int) # erro no verificador de tipos
```

Esse função é útil para garantir que o verificador de tipos entende um script conforme as intenções do desenvolvedor:

```
def complex_function(arg: object):
    # Realiza alguma lógica complexa de restrição de tipos
    # após a qual esperamos que o tipo seja `int`
    ...
    # Testa se o verificador de tipos entendeu nossa função co
    assert_type(arg, int)
```

Adicionado na versão 3.11.

typing.assert_never(*arg*, /)

Pede ao verificador de tipo estático para confirmar se uma linha de código é inalcançável.

Exemplo:

```
def int_or_str(arg: int | str) -> None:
    match arg:
        case int():
            print("É um int")
        case str():
            print("É um str")
```

```
        case _ as unreachable:
            assert_never(unreachable)
```

Aqui, as anotações permitem que o verificador de tipos deduza que o último caso nunca será executado, pois `arg` é um **`int`** ou uma **`str`**, e ambas as opções são cobertas por casos anteriores.

Se um verificador de tipos descobrir que uma chamada para `assert_never()` é alcançável, ele emitirá um erro. Por exemplo, se a anotação de tipo para `arg` fosse `int | str | float`, o verificador de tipos emitiria um erro indicando que `unreachable` é do tipo **`float`**. Para que uma chamada para `assert_never` passe verificação de tipos, o tipo inferido do argumento passado deve ser o tipo inferior, **`Never`**, e nada mais.

Em tempo de execução, essa função levanta uma exceção quando chamada.

Ver também

Unreachable Code and Exhaustiveness Checking [https://tipagem.Python.org/en/latest/guides/unreachable.html] tem mais informações sobre a verificação de exaustividade com tipagem estática.

Adicionado na versão 3.11.

typing.reveal_type(*obj*, /)

Pede para um verificador de tipo estático revelar o tipo inferido de uma expressão.

Quando um verificador de tipo estático encontra uma chamada para essa função, ele emite um diagnóstico com o tipo inferido do argumento. Por exemplo:

```
x: int = 1
reveal_type(x) # O tipo revelado é "builtins.int"
```

Isso pode ser útil quando você deseja depurar como o verificador de tipos lida com um determinado trecho de código.

Em tempo de execução, esta função envia o tipo de seu argumento para **sys.stderr** e retorna o argumento inalterado (permitindo que a chamada seja usada em uma expressão):

```
x = reveal_type(1)  # exibe "Runtime type is int"
print(x)  # exibe "1"
```

Observe que o tipo em tempo de execução pode ser diferente (mais ou menos específico) do tipo inferido estaticamente por um verificador de tipos.

A maioria dos verificadores de tipos dá suporte a `reveal_type()` em qualquer lugar, mesmo que o nome não seja importado de `typing`. Porém, importar o nome de `typing` permite que o código seja executado sem erros em tempo de execução e comunica a intenção com mais clareza.

Adicionado na versão 3.11.

@typing.dataclass_transform(*, *eq_default=True*, *order_default=False*, *kw_only_default=False*, *frozen_default=False*, *field_specifiers=()*, ***kwargs*)

Um decorador que marca um objeto como tendo comportamento similar a uma **dataclass**.

`@dataclass_transform` may be used to decorate a class, metaclass, or a function that is itself a decorator. The presence of `@dataclass_transform()` tells a static type checker that the decorated object performs runtime “magic” that transforms a class in a similar way to **@dataclasses.dataclass**.

Exemplo de uso com a função decoradora:

```
@dataclass_transform()
def create_model[T](cls: type[T]) -> type[T]:
    ...
    return cls

@create_model
class CustomerModel:
```

```
    id: int
    name: str
```

Em uma classe base:

```
@dataclass_transform()
class ModelBase: ...

class CustomerModel(ModelBase):
    id: int
    name: str
```

Em uma metaclasse:

```
@dataclass_transform()
class ModelMeta(type): ...

class ModelBase(metaclass=ModelMeta): ...

class CustomerModel(ModelBase):
    id: int
    name: str
```

As classes `CustomerModel` definidas acima serão tratadas pelos verificadores de tipos semelhantemente às classes criadas com **`@dataclasses.dataclass`**. Por exemplo, os verificadores de tipo presumirão que essas classes têm métodos `__init__` que aceitam `id` e `name`.

A classe, metaclasse ou função decorada pode aceitar os seguintes argumentos booleanos tal que verificadores de tipo presumirão ter o mesmo efeito que teriam no decorador **`@dataclasses.dataclass`**: `init`, `eq`, `order`, `unsafe_hash`, `frozen`, `match_args`, `kw_only`, e `slots`. É necessário que o valor desses argumentos (`True` ou `False`) seja avaliado estaticamente.

The arguments to the `@dataclass_transform` decorator can be used to customize the default behaviors of the decorated class, metaclass, or function:

Parâmetros::

- **eq_default** (*bool*) – Indica se o parâmetro `eq` é presumido como `True` ou `False` se for omitido pel chamador. O padrão é `True`.
- **order_default** (*bool*) – Indica se o parâmetro `order` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `False`.
- **kw_only_default** (*bool*) – Indica se o parâmetro `kw_only` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `False`.
- **frozen_default** (*bool*) – Indica se o parâmetro `frozen` é presumido como `True` ou `False` se for omitido pelo chamador. O padrão é `False`. Added in version 3.12
- **field_specifiers** (*tuple[Callable[..., Any], ...]*) – Especifica uma lista estática de classes ou funções compatíveis que descrevem campos, semelhante a `dataclasses.field()`. O valor padrão é `()`.
- ****kwargs** (*Any*) – Outros argumentos nomeados arbitrários são aceitos para permitir possíveis futuras extensões.

Verificadores de tipo reconhecem os seguintes parâmetros opcionais em especificadores de campos:

Parâmetros reconhecidos para especificadores de campos

Nome do parâmetro	Descrição
`init`	Indica se o campo deve ser incluído no método `__init__` sintetizado. Se não for especificado, o valor padrão de `init` é `True`.
`default`	Fornece o valor padrão do campo.

Nome do parâmetro	Descrição
`default_factory`	Fornece uma função de retorno de tempo de execução que retorna o valor padrão do campo. Se nem `default` nem `default_factory` forem especificados, o campo é presumido como sem valor padrão e deverá receber um valor quando a classe for instanciada.
`factory`	Um apelido para o parâmetro `default_factory` em especificadores de campos.
`kw_only`	Indicates whether the field should be marked as keyword-only. If `True`, the field will be keyword-only. If `False`, it will not be keyword-only. If unspecified, the value of the `kw_only` parameter on the object decorated with `@dataclass_transform` will be used, or if that is unspecified, the value of `kw_only_default` on `@dataclass_transform` will be used.
`alias`	Fornece um nome alternativo para o campo. Esse nome alternativo é usado no método `__init__` sintetizado.

Em tempo de execução, esse decorador registra seu argumento no atributo `__dataclass_transform__` no objeto decorado. Ele não tem nenhum outro efeito em tempo de execução.

Consulte **PEP 681** [https://peps.python.org/pep-0681/] para mais detalhes.

Adicionado na versão 3.11.

@typing.overload

Decorador para criar funções e métodos sobrecarregados.

O decorador `@overload` permite descrever funções e métodos com suporte a várias combinações de tipos de argumento. Uma sequência de definições decoradas com `@overload` deve preceder uma única definição não decorada por `@overload` (para a mesma função/método).

Definições decoradas com `@overload` são usadas somente para benefício do verificador de tipos, já que serão sobrescritas por definições sem decoração de `@overload`. Enquanto isso, definições sem decoração de `@overload` serão usadas em tempo de execução, mas devem ser ignoradas pelo verificador de tipos. Em tempo de execução, chamar uma função decorada por `@overload` diretamente levantará **`NotImplementedError`**.

Um exemplo de sobrecarga que fornece um tipo mais preciso do que o expressar o tipo expresso por uma união ou um tipo variável:

```
@overload
def process(response: None) -> None:
    ...
@overload
def process(response: int) -> tuple[int, str]:
    ...
@overload
def process(response: bytes) -> str:
    ...
def process(response):
    ...  # a implementação real vem aqui
```

Consulte **PEP 484** [https://peps.python.org/pep-0484/] para mais detalhes e uma comparação com outras semânticas de tipagem.

Alterado na versão 3.11: Funções sobrecarregadas agora podem ser inspecionadas em tempo de execução usando **`get_overloads()`**.

typing.get_overloads(*func*)

Retorna uma sequência de definições de *func* decoradas com **`@overload`**.

func é o objeto função para a implementação da função sobrecarregada. Por exemplo, dada a definição de `process` na documentação de **`@overload`**, `get_overloads(process)` retornará uma sequência de três objetos função para as três sobrecargas definidas. Se for chamada em uma

função sem sobrecargas, `get_overloads()` retornará uma sequência vazia.

A função `get_overloads()` pode ser usada em tempo de execução para introspecção de uma função sobrecarregada.

Adicionado na versão 3.11.

typing.clear_overloads()

Apaga todas as sobrecargas registradas no registro interno.

Isso pode ser usado para recuperar a memória usada pelo registro.

Adicionado na versão 3.11.

@typing.final

Um decorador para indicar métodos e classes finais.

Decorar um método com `@final` indica a um verificador de tipos que o método não pode ser substituído em uma subclasse. Decorar uma classe com `@final` indica que ela não pode ser herdada.

Por exemplo:

```
class Base:
    @final
    def done(self) -> None:
        ...
class Sub(Base):
    def done(self) -> None:  # Erro reportado pelo verificador
        ...

@final
class Leaf:
    ...
class Other(Leaf):  # Erro reportado pelo verificador de tipos
    ...
```

Não há verificação em tempo de execução dessas propriedades. Veja **PEP 591** [https://peps.python.org/pep-0591/] para mais detalhes.

Adicionado na versão 3.8.

Alterado na versão 3.11: O decorador tentará definir um atributo `__final__` como `True` no objeto decorado. Assim, uma verificação como `if getattr(obj, "__final__", False)` pode ser usada em tempo de execução para determinar se um objeto `obj` foi marcado como final. Se o objeto decorado não tem suporte a definição de atributos, o decorador retorna o objeto inalterado sem levantar uma exceção.

@typing.no_type_check

Decorador para indicar que anotações não são dicas de tipo.

Isso funciona como decorator de uma classe ou função. Com uma classe, ele se aplica recursivamente a todos os métodos e classes definidos nessa classe (mas não a métodos definidos em suas superclasses ou subclasses). Os verificadores de tipos ignorarão todas as anotações em uma função ou classe com este decorador.

`@no_type_check` modifica o objeto decorado internamente.

@typing.override

Decorador para indicar que um método em uma subclasse deve substituir um método ou atributo em uma superclasse.

Os verificadores de tipo devem emitir um erro se um método decorado com `@override` substitui nada. Isso ajuda a evitar erros que podem ocorrer quando uma classe base muda sem uma mudança equivalente em uma classe filha.

Por exemplo:

```
class Base:
    def log_status(self) -> None:
        ...

class Sub(Base):
    @override
    def log_status(self) -> None:  # OK: substitui Base.log_st
        ...
```

```
    @override
    def done(self) -> None:  # Erro reportado pelo verificador
        ...
```

Não há verificação desta propriedade em tempo de execução.

O decorador tentará definir um atributo `__override__` como `True` no objeto decorado. Assim, uma verificação como `if getattr(obj, "__override__", False)` poderá ser usada em tempo de execução para determinar se um objeto `obj` foi marcado como uma substituição. Se o objeto decorado não der suporte à atribuição de atributos, o decorador retorna o objeto inalterado sem levantar uma exceção.

Consulte **PEP 698** [https://peps.python.org/pep-0698/] para obter mais detalhes.

Adicionado na versão 3.12.

@typing.disjoint_base

Decorator to mark a class as a disjoint base.

Type checkers do not allow child classes of a disjoint base `C` to inherit from other disjoint bases that are not parent or child classes of `C`.

Por exemplo:

```
@disjoint_base
class Disjoint1: pass

@disjoint_base
class Disjoint2: pass

class Disjoint3(Disjoint1, Disjoint2): pass  # Type checker er
```

Type checkers can use knowledge of disjoint bases to detect unreachable code and determine when two types can overlap.

The corresponding runtime concept is a solid base (see Herança múltipla). Classes that are solid bases at runtime can be marked with `@disjoint_base` in stub files. Users may also mark other classes as

disjoint bases to indicate to type checkers that multiple inheritance with other disjoint bases should not be allowed.

Note that the concept of a solid base is a CPython implementation detail, and the exact set of standard library classes that are disjoint bases at runtime may change in future versions of Python.

Adicionado na versão 3.15.

@typing.type_check_only

Decorador para marcar uma classe ou função como indisponível em tempo de execução.

Este decorador em si não está disponível em tempo de execução. Seu objetivo principal é marcar classes definidas em arquivos de tipo stub se uma implementação retornar uma instância de uma classe privada:

```
@type_check_only
class Response:  # privado ou indisponível em tempo de execuçã
    code: int
    def get_header(self, name: str) -> str: ...

def fetch_response() -> Response: ...
```

Observe que retornar instâncias de classes privadas não é recomendado. Normalmente, é preferível tornar essas classes públicas.

Auxiliares de introspecção

typing.get_type_hints(*obj*, *globalns=None*, *localns=None*, *include_extras=False*, *, *format=Format.VALUE*)

Return a dictionary containing type hints for a function, method, module, class object, or other callable object.

This is often the same as `annotationlib.get_annotations()`, but this function makes the following changes to the annotations dictionary:

- Referências futuras codificadas como literais de string ou objetos `ForwardRef` são tratados avaliando-as em *globalns*, *localns* e (quando aplicável) no espaço de nomes de parâmetro de tipo de *obj*. Se *globalns* ou *localns* não forem fornecidos, os dicionários de espaço de nomes apropriados serão inferidos de *obj*.
- `None` é substituído por `types.NoneType`.
- Se `@no_type_check` tiver sido aplicada a *obj*, será retornado um dicionário vazio.
- If *obj* is a class `C`, the function returns a dictionary that merges annotations from `C`'s base classes with those on `C` directly. This is done by traversing `C.__mro__` and iteratively combining annotations of each base class. Annotations on classes appearing earlier in the method resolution order always take precedence over annotations on classes appearing later in the method resolution order.
- A função substitui recursivamente todas as ocorrências de `Annotated[T, ...]`, `Required[T]`, `NotRequired[T]` e `ReadOnly[T]` por `T`, a menos que *include_extras* seja definido como `True` (consulte `Annotated` para obter mais informações).

Cuidado

Esta função pode executar código arbitrário contido em anotações. Consulte Implicações de segurança de anotações de introspecção para obter mais informações.

Nota

If `Format.VALUE` is used and any forward references in the annotations of *obj* are not resolvable, a `NameError` exception is raised. For example, this can happen with names imported under `if TYPE_CHECKING`. More generally, any kind of exception can be raised if an annotation contains invalid Python code.

Nota

Calling **`get_type_hints()`** on an instance is not supported. To retrieve annotations for an instance, call **`get_type_hints()`** on the instance's class instead (for example, `get_type_hints(type(obj))`).

Alterado na versão 3.9: Adiciona o parâmetro `include_extras` como parte da **PEP 593** [https://peps.python.org/pep-0593/]. Consulte a documentação em `Annotated` para mais informações.

Alterado na versão 3.11: Anteriormente, `Optional[t]` era adicionado a anotações de funções e métodos se um valor padrão igual a `None` fosse definido. Agora, a anotação é retornada inalterada.

Alterado na versão 3.14: Added the `format` parameter. See the documentation on `annotationlib.get_annotations()` for more information.

Alterado na versão 3.14: Calling **`get_type_hints()`** on instances is no longer supported. Some instances were accepted in earlier versions as an undocumented implementation detail.

typing.get_origin(*tp*)

Obtenha o versão sem subscrição de um tipo: para um objeto cujo tipo tem a forma `X[Y, Z, ...]`, retorna `X`.

Se `X` for um apelido do módulo typing para uma classe embutida ou classe de `collections`, ele será normalizado para a classe original. Se `X` for uma instância de `ParamSpecArgs` ou `ParamSpecKwargs`, retorna o `ParamSpec` subjacente. Retorna `None` para objetos incompatíveis.

Exemplos:

```
assert get_origin(str) is None
assert get_origin(Dict[str, int]) is dict
assert get_origin(Union[int, str]) is Union
assert get_origin(Annotated[str, "metadata"]) is Annotated
P = ParamSpec('P')
assert get_origin(P.args) is P
assert get_origin(P.kwargs) is P
```

Adicionado na versão 3.8.

typing.get_args(*tp*)

Obtenha os argumentos de tipos com todas as substituições realizadas: para um objeto cujo tipo tem a forma `X[Y, Z, ...]`, retorna `(Y, Z, ...)`.

Se `X` for uma união ou um **`Literal`**, e estiver contido em outro tipo genérico, a ordem de `(Y, Z, ...)` poderá ser diferente da ordem dos argumentos originais `[Y, Z, ...]` devido ao armazenamento dos tipos em cache. Retorna `()` para objetos sem suporte.

Exemplos:

```
assert get_args(int) == ()
assert get_args(Dict[int, str]) == (int, str)
assert get_args(Union[int, str]) == (int, str)
```

Adicionado na versão 3.8.

typing.get_protocol_members(*tp*)

Retorna o conjunto de membros definidos em um **`Protocol`**.

```
>>> from typing import Protocol, get_protocol_members
>>> class P(Protocol):
...     def a(self) -> str: ...
...     b: int
>>> get_protocol_members(P) == frozenset({'a', 'b'})
True
```

Levanta **`TypeError`** para argumentos que não são protocolos.

Adicionado na versão 3.13.

typing.is_protocol(*tp*)

Determina se um tipo é um **`Protocol`**.

Por exemplo:

```
class P(Protocol):
    def a(self) -> str: ...
    b: int
```

```
assert is_protocol(P)
assert not is_protocol(int)
```

This function only returns true for `Protocol` classes, not for generic aliases of them:

```
class GenericP[T](Protocol):
    def a(self) -> T: ...
    b: int

assert not is_protocol(GenericP[int])
```

Adicionado na versão 3.13.

typing.is_typeddict(*tp*)

Verifica se um tipo é um **`TypedDict`**.

Por exemplo:

```
class Film(TypedDict):
    title: str
    year: int

assert is_typeddict(Film)
assert not is_typeddict(list | str)

# TypedDict é uma fábrica para criar dicionários tipados
# e não um dicionário tipado em si.
assert not is_typeddict(TypedDict)
```

This function only returns true for `TypedDict` classes, not for generic aliases of them:

```
class GenericFilm[T](TypedDict):
    title: str
    year: T

assert not is_typeddict(GenericFilm[int])
```

Adicionado na versão 3.10.

class typing.ForwardRef

Classe usada para representação de tipagem interna de referências futuras como strings.

Por exemplo, `List["AlgumaClasse"]` é implicitamente transformado em `List[ForwardRef("AlgumaClasse")]`. **`ForwardRef`** não deve ser instanciada por um usuário, mas pode ser usada por ferramentas de introspecção.

Nota

Tipos genéricos da **PEP 585** [https://peps.python.org/pep-0585/] como `list["SomeClass"]` não serão transformados implicitamente em `list[ForwardRef("SomeClass")]` e, portanto, não serão resolvidos automaticamente para `list[SomeClass]`.

Adicionado na versão 3.7.4.

Alterado na versão 3.14: Este agora é um apelido para **`annotationlib.ForwardRef`**. Vários comportamentos não documentados desta classe foram alterados; por exemplo, após a avaliação de um `ForwardRef`, o valor avaliado não é mais armazenado em cache.

typing.evaluate_forward_ref(*forward_ref*, ***, *owner=None*, *globals=None*, *locals=None*, *type_params=None*, *format=annotationlib.Format.VALUE*)

Avalia um **`annotationlib.ForwardRef`** como um dica de tipo.

Isso é semelhante a chamar **`annotationlib.ForwardRef.evaluate()`**, mas diferentemente desse método, **`evaluate_forward_ref()`** também avalia recursivamente referências futuras aninhadas dentro da dica de tipo.

Consulte a documentação de **`annotationlib.ForwardRef.evaluate()`** para saber o significado dos parâmetros *owner*, *globals*, *locals*, *type_params* e *format*.

Cuidado

Esta função pode executar código arbitrário contido em anotações. Consulte Implicações de segurança de anotações de introspecção para

obter mais informações.

Adicionado na versão 3.14.

typing.NoDefault

Um objeto sinalizador usado para indicar que um parâmetro de tipo não possui valor-padrão. Por exemplo:

```
>>> T = TypeVar("T")
>>> T.__default__ is typing.NoDefault
True
>>> S = TypeVar("S", default=None)
>>> S.__default__ is None
True
```

Adicionado na versão 3.13.

typing.NoExtraItems

A **sentinel** object used to indicate that a **TypedDict** does not have the *extra_items* class argument.

```
>>> from typing import TypedDict, NoExtraItems
>>> class Point(TypedDict):
...     x: int
...     y: int
...
>>> Point.__extra_items__ is NoExtraItems
True
```

Constante

typing.TYPE_CHECKING

A special constant that is assumed to be `True` by static type checkers. It's `False` at runtime.

Um módulo cuja importação é cara e que contém apenas tipos usados para anotações de tipagem pode ser importado com segurança dentro de um bloco `if TYPE_CHECKING:`. Isso impede que o módulo seja importado em tempo de execução; anotações não são avaliadas de forma ansiosa

(veja **PEP 649** [https://peps.python.org/pep-0649/]), portanto, usar símbolos indefinidos em anotações é inofensivo – desde que você não os examine posteriormente. Sua ferramenta de análise de tipos estáticos definirá `TYPE_CHECKING` como `True` durante a análise de tipos estáticos, o que significa que o módulo será importado e os tipos serão verificados corretamente durante essa análise.

Uso:

```
if TYPE_CHECKING:
    import expensive_mod

def fun(arg: expensive_mod.SomeType) -> None:
    local_var: expensive_mod.AnotherType = other_fun()
```

Se você ocasionalmente precisar examinar anotações de tipo em tempo de execução que podem conter símbolos indefinidos, use **`annotationlib.get_annotations()`** com um parâmetro `format` de **`annotationlib.Format.STRING`** ou **`annotationlib.Format.FORWARDREF`** para recuperar as anotações com segurança sem levantar **`NameError`**.

Adicionado na versão 3.5.2.

Apelidos descontinuados

Este módulo define vários apelidos descontinuados de classes pré-existentes da biblioteca padrão. Originalmente, elas eram incluídas no módulo **`typing`** para permitir a parametrização dessas classes genéricas usando `[]`. Porém, os apelidos se tornaram redundantes no Python 3.9, quando as classes pré-existentes correspondentes passaram a dar suporte a `[]` (consulte a **PEP 585** [https://peps.python.org/pep-0585/]).

Os tipos redundantes estão descontinuados desde Python 3.9. No entanto, enquanto os apelidos podem ser removidos em algum momento, essa remoção desses apelidos não está planejada. Assim, nenhum aviso de descontinuação será enviado pelo interpretador para esses apelidos.

Se em algum momento decidirem remover esses apelidos descontinuados, o interpretador emitirá um aviso de descontinuação por, no mínimo, duas

versões de lançamento antes da remoção. Os apelidos são garantidos a permanecerem no módulo **typing** sem avisos de descontinuação até, no mínimo, Python 3.14.

Verificadores de tipos são encorajados a sinalizar o uso de tipos descontinuados se o programa que estão verificando respeita uma versão mínima de Python 3.9 ou mais nova.

Apelidos de tipos embutidos

class typing.Dict(*dict, MutableMapping[KT, VT]*)

Apelido descontinuado de **dict**.

Note que, para anotar argumentos, prefere-se usar um tipo de coleção abstrata como **Mapping** em vez de **dict** ou **typing.Dict**.

Descontinuado desde a versão 3.9: **builtins.dict** agora oferece suporte a subscrição (**[]**). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.List(*list, MutableSequence[T]*)

Apelido descontinuado de **list**.

Note que, para anotar argumentos, é preferível usar um tipo de coleção abstrata, como **Sequence** ou **Iterable**, em vez de **list** ou **typing.List**.

Descontinuado desde a versão 3.9: **builtins.list** agora oferece suporte a subscrição (**[]**). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Set(*set, MutableSet[T]*)

Apelido descontinuado de **builtins.set**.

Note que, para anotar argumentos, prefere-se usar um tipo de coleção abstrata como **collections.abc.Set** em vez de **set** ou **typing.Set**.

Descontinuado desde a versão 3.9: **builtins.set** agora oferece suporte a subscrição (**[]**). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo

Generic Alias.

class typing.FrozenSet(*frozenset, AbstractSet[T_co]*)

Apelido descontinuado de `builtins.frozenset`.

Descontinuado desde a versão 3.9: `builtins.frozenset` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

typing.Tuple

Apelido descontinuado para `tuple`.

`tuple` e `Tuple` são casos especiais no sistema de tipos; consulte Anotando tuplas para obter mais detalhes.

Descontinuado desde a versão 3.9: `builtins.tuple` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Type(*Generic[CT_co]*)

Apelido descontinuado de `type`.

Consulte O tipo de objetos de classe para obter detalhes sobre como usar `type` ou `typing.Type` em anotações de tipos.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: `builtins.type` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Apelidos para tipos em `collections`

class typing.DefaultDict(*collections.defaultdict, MutableMapping[KT, VT]*)

Apelido descontinuado de `collections.defaultdict`.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: `collections.defaultdict` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.OrderedDict(*collections.OrderedDict, MutableMapping[KT, VT]*)

Apelido descontinuado de `collections.OrderedDict`.

Adicionado na versão 3.7.2.

Descontinuado desde a versão 3.9: `collections.OrderedDict` agora oferece suporte a subscrição (`[]`). consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.ChainMap(*collections.ChainMap, MutableMapping[KT, VT]*)

Apelido descontinuado de `collections.ChainMap`.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: `collections.ChainMap` agora oferece suporte a subscrição (`[]`). consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Counter(*collections.Counter, Dict[T, int]*)

Apelido descontinuado de `collections.Counter`.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: `collections.Counter` agora oferece suporte a subscrição (`[]`). consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Deque(*deque, MutableSequence[T]*)

Apelido descontinuado de `collections.deque`.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: `collections.deque` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Apelidos de outros tipos concretos

class typing.Pattern
class typing.Match

Deprecated aliases corresponding to the return types from `re.compile()` and `re.search()`.

Esses tipos (e as funções correspondentes) são genéricas sobre `AnyStr`. `Pattern` pode ser especializado como `Pattern[str]` ou `Pattern[bytes]`; `Match` pode ser especializado como `Match[str]` ou `Match[bytes]`.

Descontinuado desde a versão 3.9: Classes `Pattern` e `Match` de `re` agora suporte `[]`. Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Text

Apelido descontinuado de `str`.

`Text` é fornecida para servir como um caminho compatível com versões futuras para código em Python 2: no Python 2, `Text` é um apelido para `unicode`.

Use `Text` para indicar que um valor deve conter uma string unicode de forma compatível com Python 2 e Python 3:

```
def add_unicode_checkmark(text: Text) -> Text:
    return text + u' \u2713'
```

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.11: Python 2 deixou de receber suporte, e a maioria dos verificadores de tipos também não oferece suporte à verificação de tipos de código de Python 2. A remoção do apelido não está planejada no momento, mas os usuários são incentivados a usar `str` em vez de `Text`.

Apelidos de contêineres ABC em `collections.abc`.

class typing.AbstractSet(*Collection[T_co]*)

Apelido descontinuado de `collections.abc.Set`.

Descontinuado desde a versão 3.9: `collections.abc.Set` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.ByteString(*Sequence[int]*)

Apelido descontinuado para `collections.abc.ByteString`.

Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use `Buffer` ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

`ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de `bytes` e `bytearray`. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como `memoryview`, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

See **PEP 688** [https://peps.python.org/pep-0688/#current-options] for more details.

Descontinuado desde a versão 3.9, será removido na versão 3.17.

class typing.Collection(*Sized, Iterable[T_co], Container[T_co]*)

Apelido descontinuado de `collections.abc.Collection`.

Adicionado na versão 3.6.

Descontinuado desde a versão 3.9: `collections.abc.Collection` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Container(*Generic[T_co]*)

Apelido descontinuado de `collections.abc.Container`.

Descontinuado desde a versão 3.9: `collections.abc.Container` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.ItemsView(*MappingView, AbstractSet[tuple[KT_co, VT_co]]*)

Apelido descontinuado de `collections.abc.ItemsView`.

Descontinuado desde a versão 3.9: `collections.abc.ItemsView` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.KeysView(*MappingView, AbstractSet[KT_co]*)

Apelido descontinuado de `collections.abc.KeysView`.

Descontinuado desde a versão 3.9: `collections.abc.KeysView` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Mapping(*Collection[KT], Generic[KT, VT_co]*)

Apelido descontinuado de `collections.abc.Mapping`.

Descontinuado desde a versão 3.9: `collections.abc.Mapping` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.MappingView(*Sized*)

Apelido descontinuado de `collections.abc.MappingView`.

Descontinuado desde a versão 3.9: `collections.abc.MappingView` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.MutableMapping(*Mapping[KT, VT]*)

Apelido descontinuado de `collections.abc.MutableMapping`.

Descontinuado desde a versão 3.9: `collections.abc.MutableMapping` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.MutableSequence(*Sequence[T]*)

Apelido descontinuado de `collections.abc.MutableSequence`.

Descontinuado desde a versão 3.9: `collections.abc.MutableSequence` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.MutableSet(*AbstractSet[T]*)

Apelido descontinuado de `collections.abc.MutableSet`.

Descontinuado desde a versão 3.9: `collections.abc.MutableSet` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Sequence(*Reversible[T_co], Collection[T_co]*)

Apelido descontinuado de `collections.abc.Sequence`.

Descontinuado desde a versão 3.9: `collections.abc.Sequence` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.ValuesView(*MappingView, Collection[_VT_co]*)

Apelido descontinuado de `collections.abc.ValuesView`.

Descontinuado desde a versão 3.9: `collections.abc.ValuesView` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Apelidos para ABCs assíncronas em `collections.abc`

class typing.Coroutine(*Awaitable[ReturnType], Generic[YieldType, SendType, ReturnType]*)

Apelido descontinuado de `collections.abc.Coroutine`.

Consulte Anotando geradores e corrotinas para obter detalhes sobre como usar `collections.abc.Coroutine` e `typing.Coroutine` em anotações de tipos.

Adicionado na versão 3.5.3.

Descontinuado desde a versão 3.9: `collections.abc.Coroutine` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.AsyncGenerator(*AsyncIterator[YieldType], Generic[YieldType, SendType]*)

Apelido descontinuado de `collections.abc.AsyncGenerator`.

Consulte Anotando geradores e corrotinas para obter detalhes sobre como usar `collections.abc.AsyncGenerator` e `typing.AsyncGenerator` em anotações de tipos.

Adicionado na versão 3.6.1.

Descontinuado desde a versão 3.9: `collections.abc.AsyncGenerator` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Alterado na versão 3.13: O parâmetro `SendType` agora tem um valor-padrão.

class typing.AsyncIterable(*Generic[T_co]*)

Apelido descontinuado de `collections.abc.AsyncIterable`.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: `collections.abc.AsyncIterable` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.AsyncIterator(*AsyncIterable[T_co]*)

Apelido descontinuado de `collections.abc.AsyncIterator`.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: `collections.abc.AsyncIterator` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Awaitable(*Generic[T_co]*)

Apelido descontinuado de `collections.abc.Awaitable`.

Adicionado na versão 3.5.2.

Descontinuado desde a versão 3.9: `collections.abc.Awaitable` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Apelidos para outros ABCs em `collections.abc`.

class typing.Iterable(*Generic[T_co]*)

Apelido descontinuado de `collections.abc.Iterable`.

Descontinuado desde a versão 3.9: `collections.abc.Iterable` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Iterator(*Iterable[T_co]*)

Apelido descontinuado de `collections.abc.Iterator`.

Descontinuado desde a versão 3.9: `collections.abc.Iterator` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

typing.Callable

Apelido descontinuado de `collections.abc.Callable`.

Veja Anotações de objetos chamáveis para detalhes sobre como usar `collections.abc.Callable` e `typing.Callable` em anotações de tipo.

Descontinuado desde a versão 3.9: `collections.abc.Callable` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Alterado na versão 3.10: `Callable` agora oferece suporte a `ParamSpec` e `Concatenate`. Veja **PEP 612** [https://peps.python.org/pep-0612/] para mais detalhes.

class typing.Generator(*Iterator[YieldType], Generic[YieldType, SendType, ReturnType]*)

Apelido descontinuado de `collections.abc.Generator`.

Consulte Anotando geradores e corrotinas para obter detalhes sobre como usar `collections.abc.Generator` e `typing.Generator` em anotações de tipos.

Descontinuado desde a versão 3.9: `collections.abc.Generator` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Alterado na versão 3.13: Adiciona valores-padrão para os tipos de envio e retorno.

class typing.Hashable

Apelido descontinuado de `collections.abc.Hashable`.

Descontinuado desde a versão 3.12: Use `collections.abc.Hashable` diretamente.

class typing.Reversible(*Iterable[T_co]*)

Apelido descontinuado de `collections.abc.Reversible`.

Descontinuado desde a versão 3.9: `collections.abc.Reversible` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

class typing.Sized

Apelido descontinuado de `collections.abc.Sized`.

Descontinuado desde a versão 3.12: Use `collections.abc.Sized` diretamente.

Apelidos de ABCs da `contextlib`

class typing.ContextManager(*Generic[T_co, ExitT_co]*)

Apelido descontinuado de `contextlib.AbstractContextManager`.

O primeiro parâmetro de tipo, `T_co`, representa o tipo retornado pelo método `__enter__()`. O segundo parâmetro opcional de tipo, `ExitT_co`, cujo valor-padrão é `bool | None`, representa o tipo retornado pelo método `__exit__()`.

Adicionado na versão 3.5.4.

Descontinuado desde a versão 3.9: `contextlib.AbstractContextManager` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Alterado na versão 3.13: Adiciona o segundo parâmetro opcional de tipo, `ExitT_co`.

class typing.AsyncContextManager(*Generic[T_co, AExitT_co]*)

Apelido descontinuado de `contextlib.AbstractAsyncContextManager`.

O primeiro parâmetro de tipo, `T_co`, representa o tipo retornado pelo método `__aenter__()`. O segundo parâmetro opcional de tipo, `AExitT_co`, cujo valor-padrão é `bool | None`, representa o tipo retornado pelo método `__aexit__()`.

Adicionado na versão 3.6.2.

Descontinuado desde a versão 3.9: `contextlib.AbstractAsyncContextManager` agora oferece suporte a subscrição (`[]`). Consulte **PEP 585** [https://peps.python.org/pep-0585/] e Tipo Generic Alias.

Alterado na versão 3.13: Adiciona o segundo parâmetro opcional de tipo, `AExitT_co`.

Cronograma de descontinuação dos principais recursos

Alguns recursos em `typing` estão descontinuados e podem ser removidos em uma versão futura de Python. A tabela a seguir resume as principais descontinuações para sua conveniência. Ela está sujeita a alterações, e nem todas as descontinuações estão listadas.

Recurso	Descontinuado em	Remoção planejada	PEP/issue
versões `typing` de coleções-padrão	3.9	Não definido (consulte apelidos descontinuados para mais informações)	**PEP 585**
`typing.ByteString`	3.9	3.17	gh-91896
`typing.Text`	3.11	Não definido	gh-92332
`typing.Hashable` e `typing.Sized`	3.12	Não definido	gh-94309
`typing.TypeAlias`	3.12	Não definido	**PEP 695**
`typing.AnyStr`	3.13	3.18	gh-105578

pydoc — Gerador de documentação e sistema de ajuda online

Código-fonte: Lib/pydoc.py [https://github.com/python/cpython/tree/3.15/Lib/pydoc.py]

O módulo **pydoc** gera automaticamente a documentação dos módulos Python. A documentação pode ser apresentada como páginas de texto no console, servida em um navegador web ou salva em arquivos HTML.

Para módulos, classes, funções e métodos, a documentação exibida é derivada da docstring (ou seja, o atributo `__doc__`) do objeto, e recursivamente de seus membros documentáveis. Se não houver docstring, **pydoc** tenta obter uma descrição do bloco de linhas de comentário logo acima da definição da classe, função ou método no arquivo fonte, ou no topo do módulo (consulte `inspect.getcomments()`).

A função embutida `help()` invoca o sistema de ajuda online no interpretador interativo, que usa **pydoc** para gerar sua documentação como texto no console. A mesma documentação de texto também pode ser vista de fora do interpretador Python executando **pydoc** como um script no prompt de comando do sistema operacional. Por exemplo, executar

```
python -m pydoc sys
```

em um prompt de console exibirá a documentação do módulo `sys`, em um estilo semelhante às páginas de manual mostradas pelo comando Unix **man**. O argumento para **pydoc** pode ser o nome de uma função, módulo ou pacote, ou uma referência pontilhada a uma classe, método ou função dentro de um módulo ou módulo em um pacote. Se o argumento para **pydoc** parecer um caminho (ou seja, ele contém o separador de caminho para o seu sistema operacional, como uma barra no Unix) e se refere a um arquivo fonte Python existente, então a documentação é produzida para esse arquivo.

Nota

Para encontrar objetos e sua documentação, **`pydoc`** importa os módulos a serem documentados. Portanto, qualquer código no nível do módulo será executado nessa ocasião. Use uma proteção `if __name__ == '__main__':` para executar código apenas quando um arquivo é chamado como um script e não apenas importado.

Ao imprimir a saída para o console, **pydoc** tenta paginar a saída para facilitar a leitura. Se a variável de ambiente **`MANPAGER`** ou a **`PAGER`** estiver definida, **pydoc** usará seu valor como um programa de paginação. Quando ambas estiverem definidas, **`MANPAGER`** é usada.

Especificar um sinalizador `-w` antes do argumento fará com que a documentação HTML seja escrita em um arquivo no diretório atual, ao invés de exibir texto no console.

Especificar um sinalizador `-k` antes do argumento irá pesquisar as linhas de sinopse de todos os módulos disponíveis para a palavra reservada fornecida como o argumento, novamente de uma maneira semelhante ao comando Unix **man**. A linha de sinopse de um módulo é a primeira linha de sua string de documentação.

Você também pode usar **pydoc** para iniciar um servidor HTTP na máquina local que servirá a documentação para os navegadores web visitantes. **python -m pydoc -p 1234** irá iniciar um servidor HTTP na porta 1234, permitindo que você navegue pela documentação em `http://localhost:1234/` em seu navegador preferido. Especificar `0` como o número da porta irá selecionar uma porta não utilizada arbitrária.

Aviso

O servidor HTTP **`pydoc`** destina-se ao uso local durante o desenvolvimento e não é adequado para uso em produção.

python -m pydoc -n <hostname> irá iniciar o servidor ouvindo no nome de host fornecido. Por padrão, o nome de host é “localhost”, mas se você deseja que o servidor seja acessado por outras máquinas, você pode alterar o nome de host ao qual o servidor responde. Durante o desenvolvimento, isso é especialmente útil se você deseja executar o pydoc de dentro de um contêiner.

python -m pydoc -b irá iniciar o servidor e, adicionalmente, abrir um navegador da web para uma página de índice do módulo. Cada página exibida tem uma barra de navegação na parte superior onde você pode escolher *Get* para obter ajuda em um item individual, *Search* para pesquisar todos os módulos com uma palavra reservada em sua linha de sinopse e ir para as páginas de índice do módulo em *Module index*, tópicos em *Topics* e palavras reservadas em *Keywords*.

Quando **pydoc** gera documentação, ele usa o ambiente atual e o caminho para localizar os módulos. Assim, invocar **pydoc spam** documenta precisamente a versão do módulo que você obteria se iniciasse o interpretador Python e digitasse `import spam`.

Os documentos do módulo para os módulos principais são presumidos como residindo em `https://docs.python.org/X.Y/library/`, sendo `X` e `Y` os números de versão principal e secundária do interpretador Python. Isso pode ser substituído definindo a variável de ambiente **`PYTHONDOCS`** para uma URL diferente ou para um diretório local contendo as páginas do Manual de Referência da Biblioteca.

Alterado na versão 3.2: Adicionada a opção `-b`.

Alterado na versão 3.3: A opção de linha de comando `-g` foi removida.

Alterado na versão 3.4: **`pydoc`** agora usa **`inspect.signature()`** em vez de **`inspect.getfullargspec()`** para extrair informações de assinatura de chamáveis.

Alterado na versão 3.7: Adicionada a opção `-n`.

Modo de Desenvolvimento do Python

Adicionado na versão 3.7.

O Modo de Desenvolvimento do Python introduz verificações de tempo de execução adicionais que são muito custosas para serem ativadas por padrão. Não deve ser mais detalhado que o padrão se o código estiver correto; novos avisos são emitidos somente quando um problema é detectado.

Ele pode ser ativado usando a opção de linha de comando **`-X dev`** ou configurando a variável de ambiente **`PYTHONDEVMODE`** como `1`.

Veja também a compilação de depuração do Python.

Efeitos do Modo de Desenvolvimento do Python

A ativação do Modo de Desenvolvimento do Python é semelhante ao comando a seguir, mas com efeitos adicionais descritos abaixo:

```
PYTHONMALLOC=debug PYTHONASYNCIODEBUG=1 python -W default -X fau
```

Efeitos do Modo de Desenvolvimento do Python:

- Adiciona o filtro de avisos `default`. Os seguintes avisos são exibidos:
  - **`DeprecationWarning`**
  - **`ImportWarning`**
  - **`PendingDeprecationWarning`**
  - **`ResourceWarning`**

  Normalmente, os avisos acima são filtrados pelos filtros de avisos padrão.

Ele se comporta como se a opção de linha de comando `-W default` fosse usada.

Use a opção de linha de comando `-W error` ou defina a variável de ambiente `PYTHONWARNINGS` com `error` para tratar avisos como erros.

- Instala ganchos de depuração nos alocadores de memória para verificar por:

  - Estouro negativo de buffer
  - Estouro de buffer
  - Violação de API de alocador de memória
  - Uso inseguro da GIL

  Consulte a função C `PyMem_SetupDebugHooks()`.

  Se comporta como se a variável de ambiente `PYTHONMALLOC` estivesse definida com `debug`.

  Para habilitar o Modo de Desenvolvimento do Python sem instalar ganchos de depuração nos alocadores de memória, defina a variável de ambiente `PYTHONMALLOC` como `default`.

- Chama `faulthandler.enable()` na inicialização do Python para instalar manipuladores para os sinais `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS` e `SIGILL` para despejar o traceback (situação da pilha de execução) do Python no caso de travamento.

  Ele se comporta como se a opção de linha de comando `-X faulthandler` fosse usada ou se a variável de ambiente `PYTHONFAULTHANDLER` estivesse definida como `1`.

- Ativa o modo de depuração de asyncio. Por exemplo, `asyncio` verifica as corrotinas que não foram aguardadas (*await*) e as registra.

  Ele se comporta como se a variável de ambiente `PYTHONASYNCIODEBUG` estivesse definida como `1`.

- Verifica os argumentos *encoding* e *errors* para operações de codificação e decodificação de strings. Exemplos: `open()`, `str.encode()` e `bytes.decode()`.

  Por padrão, para obter o melhor desempenho, o argumento *errors* é verificado apenas no primeiro erro de codificação/decodificação, e o argumento *encoding* às vezes é ignorado para strings vazias.

- O destrutor de `io.IOBase` registra exceções `close()`.

- Define o atributo `dev_mode` de `sys.flags` como `True`.

O Modo de Desenvolvimento do Python não ativa o módulo `tracemalloc` por padrão porque o custo adicional (para desempenho e memória) seria muito grande. A ativação do módulo **`tracemalloc`** fornece informações adicionais sobre a origem de alguns erros. Por exemplo, `ResourceWarning` registra o retorno ao local onde o recurso foi alocado, e um erro de estouro de buffer registra o retorno ao local onde o bloco de memória foi alocado.

O Modo de Desenvolvimento do Python não impede que a opção de linha de comando `-O` remova as instruções `assert` nem configure `__debug__` como `False`.

O Modo de Desenvolvimento do Python só pode ser ativado na inicialização do Python. Seu valor pode ser lido de `sys.flags.dev_mode`.

Alterado na versão 3.8: O destrutor de `io.IOBase` agora registra exceções `close()`.

Alterado na versão 3.9: Os argumentos *encoding* e *errors* agora são verificados para operações de codificação e decodificação de strings.

Exemplo de ResourceWarning

Exemplo de um script que conta o número de linhas do arquivo texto especificado na linha de comando:

```
import sys

def main():
    fp = open(sys.argv[1])
    nlines = len(fp.readlines())
    print(nlines)
    # O arquivo é fechado implicitamente

if __name__ == "__main__":
    main()
```

O script não fecha o arquivo explicitamente. Por padrão, o Python não emite nenhum aviso. Exemplo usando README.txt, que possui 269 linhas:

```
$ python script.py README.txt
269
```

A ativação do Modo de Desenvolvimento do Python exibe um aviso **`ResourceWarning`**:

```
$ python -X dev script.py README.txt
269
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper
  main()
ResourceWarning: Enable tracemalloc to get the object allocatior
```

Além disso, ativar **`tracemalloc`** mostra a linha em que o arquivo foi aberto:

```
$ python -X dev -X tracemalloc=5 script.py README.rst
269
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper
  main()
Object allocated at (most recent call last):
  File "script.py", lineno 10
    main()
  File "script.py", lineno 4
    fp = open(sys.argv[1])
```

A correção é fechar explicitamente o arquivo. Exemplo usando um gerenciador de contexto:

```
def main():
    # Fecha o arquivo explicitamente ao sair do bloco with
    with open(sys.argv[1]) as fp:
        nlines = len(fp.readlines())
    print(nlines)
```

Não fechar um recurso explicitamente pode deixá-lo aberto por muito mais tempo do que o esperado; isso pode causar problemas graves ao sair do Python. É ruim no CPython, mas é ainda pior no PyPy. Fechar recursos explicitamente torna uma aplicação mais determinística e mais confiável.

Exemplo de erro de descritor de arquivo inválido

Script exibindo sua própria primeira linha:

```
import os

def main():
    fp = open(__file__)
    firstline = fp.readline()
    print(firstline.rstrip())
    os.close(fp.fileno())
    # O arquivo é fechado explicitamente

main()
```

Por padrão, o Python não emite qualquer aviso:

```
$ python script.py
import os
```

O Modo de Desenvolvimento do Python mostra uma `ResourceWarning` e registra um erro “Bad file descriptor” ao finalizar o objeto arquivo:

```
$ python -X dev script.py
import os
script.py:10: ResourceWarning: unclosed file <_io.TextIOWrapper
  main()
ResourceWarning: Enable tracemalloc to get the object allocatior
Exception ignored in: <_io.TextIOWrapper name='script.py' mode='
```

```
Traceback (most recent call last):
  File "script.py", line 10, in <module>
    main()
OSError: [Errno 9] Bad file descriptor
```

`os.close(fp.fileno())` fecha o descritor de arquivo. Quando o finalizador de objeto arquivo tenta fechar o descritor de arquivo novamente, ele falha com o erro `Bad file descriptor`. Um descritor de arquivo deve ser fechado apenas uma vez. Na pior das hipóteses, fechá-lo duas vezes pode causar um acidente (consulte bpo-18748 [https://bugs.python.org/issue?@action=redirect&bpo=18748] para um exemplo).

A correção é remover a linha `os.close(fp.fileno())` ou abrir o arquivo com `closefd=False`.

`doctest` — Teste exemplos interativos de Python

Código-fonte: Lib/doctest.py [https://github.com/python/cpython/tree/3.15/Lib/doctest.py]

The `doctest` module searches for pieces of text that look like interactive Python sessions, and then executes those sessions to verify that they work exactly as shown. There are several common ways to use doctest:

- Para verificar se as docstrings de um módulo estão atualizadas, verificando que todos os exemplos interativos ainda funcionam conforme documentado.
- Para executar testes de regressão, verificando que os exemplos interativos de um arquivo de teste ou um objeto teste funcionam como esperado.
- Para escrever a documentação do tutorial para um pacote, ilustrado de forma liberal com exemplos de entrada e saída. Dependendo se os exemplos ou o texto expositivo são enfatizados, isso tem o sabor do “teste ilustrado” ou “documentação executável”.

Aqui temos um pequeno exemplo, porém, completo:

```
"""
This is the "example" module.

The example module supplies one function, factorial().  For exam

>>> factorial(5)
120
"""

def factorial(n):
    """Return the factorial of n, an exact integer >= 0.

    >>> [factorial(n) for n in range(6)]
```

```
    [1, 1, 2, 6, 24, 120]
    >>> factorial(30)
    265252859812191058636308480000000
    >>> factorial(-1)
    Traceback (most recent call last):
        ...
    ValueError: n must be >= 0

    Factorials of floats are OK, but the float must be an exact
    >>> factorial(30.1)
    Traceback (most recent call last):
        ...
    ValueError: n must be exact integer
    >>> factorial(30.0)
    265252859812191058636308480000000

    It must also not be ridiculously large:
    >>> factorial(1e100)
    Traceback (most recent call last):
        ...
    OverflowError: n too large
    """

    import math
    if not n >= 0:
        raise ValueError("n must be >= 0")
    if math.floor(n) != n:
        raise ValueError("n must be exact integer")
    if n+1 == n:  # catch a value like 1e300
        raise OverflowError("n too large")
    result = 1
    factor = 2
    while factor <= n:
        result *= factor
        factor += 1
    return result


if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

If you run `example.py` directly from the command line, **`doctest`** works its magic:

```
$ python example.py
$
```

There's no output! That's normal, and it means all the examples worked. Pass `-v` to the script, and **`doctest`** prints a detailed log of what it's trying, and prints a summary at the end:

```
$ python example.py -v
Trying:
    factorial(5)
Expecting:
    120
ok
Trying:
    [factorial(n) for n in range(6)]
Expecting:
    [1, 1, 2, 6, 24, 120]
ok
```

E assim por diante, eventualmente terminando com:

```
Trying:
    factorial(1e100)
Expecting:
    Traceback (most recent call last):
        ...
    OverflowError: n too large
ok
2 items passed all tests:
   1 test in __main__
   6 tests in __main__.factorial
7 tests in 2 items.
7 passed.
Test passed.
$
```

That's all you need to know to start making productive use of **`doctest`**! Jump in. The following sections provide full details. Note that there are

many examples of doctests in the standard Python test suite and libraries. Especially useful examples can be found in the standard test file `Lib/test/test_doctest/test_doctest.py`.

Adicionado na versão 3.13: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

Uso simples: verificando exemplos em Docstrings

A maneira mais simples de começar a usar o doctest (mas não necessariamente a maneira como você continuará fazendo isso) é encerrar cada módulo **`M`** com:

```
if __name__ == "__main__":
    import doctest
    doctest.testmod()
```

`doctest` e então examine a docstrings no módulo **`M`**.

Executar o módulo como um script faz com que os exemplos nas docstrings sejam executados e verificados:

```
python M.py
```

Isso não exibirá nada, a menos que um exemplo falhe, caso em que o(s) exemplo(s) falhando(s) e a(s) causa(s) da(s) falha(s) são impressas em stdout e a linha final de saída será `***Test Failed*** N failures.`, onde *N* é o número de exemplos que falharam.

Ao invés disso, execute agora com a opção `-v`:

```
python M.py -v
```

e um relatório detalhado de todos os exemplos testados é impresso na saída padrão, junto com diversos resumos no final.

Você pode forçar o modo verboso passando `verbose=True` para **`testmod()`**, ou proibi-lo passando `verbose=False`. Em qualquer um desses casos,

`sys.argv` não é examinado por **`testmod()`** (então passar `-v` ou não não tem efeito).

Há também um atalho de linha de comando para executar **`testmod()`**, veja a seção Uso na linha de comando.

Para mais informações sobre **`testmod()`**, veja a seção Basic API.

Utilização comum: Verificando exemplos em um arquivo texto

Outra aplicação simples do doctest é testar exemplos interativos em um arquivo texto. Isso pode ser feito com a função **`testfile()`**:

```
import doctest
doctest.testfile("example.txt")
```

Esse pequeno script executa e verifica quaisquer exemplos interativos do Python contidos no arquivo `example.txt`. O conteúdo do arquivo é tratado como se fosse uma única docstring gigante; o arquivo não precisa conter um programa Python! Por exemplo, talvez `example.txt` contenha isto:

```
The ``example`` module
======================

Using ``factorial``
-------------------

This is an example text file in reStructuredText format.  First
``factorial`` from the ``example`` module:

    >>> from example import factorial

Now use it:

    >>> factorial(6)
    120
```

Executar `doctest.testfile("example.txt")` então encontra o erro nesta documentação:

```
File "./example.txt", line 14, in example.txt
Failed example:
    factorial(6)
Expected:
    120
Got:
    720
```

Assim como **`testmod()`**, **`testfile()`** não vai exibir nada a menos que um exemplo falhe. Se um exemplo falhar, então o(s) exemplo(s) com falha e a(s) causa(s) da(s) falha(s) são impressos em stdout, usando o mesmo formato que **`testmod()`**.

Por padrão, **`testfile()`** procura por arquivos no diretório do módulo chamador. Veja a seção Basic API para uma descrição dos argumentos opcionais que podem ser usados para dizer para procurar por arquivos em outros locais.

Assim como **`testmod()`**, a verbosidade de **`testfile()`** pode ser definida com a opção de linha de comando `-v` ou com o argumento nomeado opcional *verbose*.

Há também um atalho de linha de comando para executar **`testfile()`**, veja a seção Uso na linha de comando.

Para maiores informações em **`testfile()`**, veja a seção Basic API.

Uso na linha de comando

The **`doctest`** module can be invoked as a script from the command line:

```
python -m doctest [-v] [-o OPÇÃO] [-f] arquivo [arquivo ...]
```

-v, --verbose

Relatório detalhado de todos os exemplos testados é impresso na saída padrão, junto com diversos resumos no final.

```
python -m doctest -v example.py
```

Isso importará `example.py` como um módulo independente e executará **`testmod()`** nele. Observe que isso pode não funcionar corretamente se o arquivo fizer parte de um pacote e importar outros submódulos desse pacote.

Se o nome do arquivo não termina com `.py`, **`doctest`** infere que ele deve ser executado com **`testfile()`**:

```
python -m doctest -v example.txt
```

-o, --option <option>

Os sinalizadores de opção controlam vários aspectos do comportamento do doctest, consulte a seção Flags opcionais.

Adicionado na versão 3.4.

-f, --fail-fast

Este é um atalho para `-o FAIL_FAST`.

Adicionado na versão 3.4.

Como ele funciona

Esta seção examina detalhadamente como o doctest funciona: quais docstrings ele analisa, como encontra exemplos interativos, qual contexto de execução ele usa, como ele lida com exceções e como sinalizadores de opção podem ser usados para controlar seu comportamento. Esta é a informação que você precisa saber para escrever exemplos de doctest; para obter informações sobre como realmente executar o doctest nesses exemplos, consulte as seções a seguir.

Quais docstrings são examinadas?

A docstring do módulo e todas as docstrings de funções, classes e métodos são pesquisadas. Os objetos importados para o módulo não são pesquisados.

Além disso, há casos em que você deseja que os testes façam parte de um módulo, mas não do texto de ajuda, o que exige que os testes não sejam incluídos na documentação. Doctest procura uma variável em nível de módulo chamada `__test__` e a utiliza para localizar outros testes. Se `M.__test__` existir, deve ser um dicionário, e cada entrada mapeia um nome (string) para um objeto função, objeto classe ou string. As docstrings de funções e objetos classe encontradas em `M.__test__` são pesquisadas e as strings são tratadas como se fossem docstrings. Na saída, uma chave `K` em `M.__test__` aparece com o nome `M.__test__.K`.

Por exemplo, coloque este bloco de código no topo de `example.py`:

```
__test__ = {
    'numbers': """
>>> factorial(6)
720

>>> [factorial(n) for n in range(6)]
[1, 1, 2, 6, 24, 120]
"""
}
```

O valor de `example.__test__["numbers"]` será tratado como uma docstring e todos os testes dentro dele serão executados. É importante observar que o valor pode ser mapeado para uma função, objeto classe ou módulo; se sim, **`doctest`** pesquisa recursivamente em busca de docstrings, que são então escaneados em busca de testes.

Quaisquer classes encontradas são pesquisadas recursivamente de forma semelhante, para testar docstrings em seus métodos contidos e classes aninhadas.

Nota

`doctest` can only automatically discover classes and functions that are defined at the module level or inside other classes.

Since nested classes and functions only exist when an outer function is called, they cannot be discovered. Define them outside to make them

visible.

Como os exemplos de docstrings são reconhecidos?

Na maioria dos casos, copiar e colar de uma sessão de console interativo funciona bem, mas o doctest não está tentando fazer uma emulação exata de qualquer shell Python específico.

```
>>> # comentários são ignorados
>>> x = 12
>>> x
12
>>> if x == 13:
...     print("yes")
... else:
...     print("no")
...     print("NO")
...     print("NO!!!")
...
no
NO
NO!!!
>>>
```

Qualquer saída esperada deve seguir imediatamente a linha final `'>>> '` ou `'... '` contendo o código, e a saída esperada (se houver) se estende até a próxima `'>>> '` ou linha com apenas espaços em branco.

A saída formatada:

- A saída esperada não pode conter uma linha com apenas espaços em branco, uma vez que tal linha é usada para sinalizar o fim da saída esperada. Se a saída esperada contiver uma linha vazia, coloque `<BLANKLINE>` em seu exemplo doctest em cada local onde uma linha em branco é esperada.

- Todos os caracteres de tabulação rígidos são expandidos para espaços, usando paradas de tabulação de 8 colunas. As guias na saída gerada

pelo código testado não são modificadas. Como quaisquer tabulações rígidas na saída de amostra *são* expandidas, isso significa que se a saída do código incluir tabulações rígidas, a única maneira de o doctest passar é se a opção **`NORMALIZE_WHITESPACE`** ou a diretiva estiver em vigor. Alternativamente, o teste pode ser reescrito para capturar a saída e compará-la com um valor esperado como parte do teste. Esse tratamento das guias na fonte foi obtido por tentativa e erro e provou ser a maneira menos propensa a erros de lidar com elas. É possível usar um algoritmo diferente para lidar com guias escrevendo uma classe **`DocTestParser`** personalizada.

- A saída para stdout é capturada, mas não para stderr (os tracebacks de exceção são capturados por um meio diferente).

- Se você continuar uma linha através de barra invertida em uma sessão interativa, ou por qualquer outro motivo usar uma barra invertida, você deverá usar uma docstring bruta, que preservará suas barras invertidas exatamente como você as digita:

```
>>> def f(x):
...     r'''Backslashes in a raw docstring: m\n'''
...
>>> print(f.__doc__)
Backslashes in a raw docstring: m\n
```

  Caso contrário, a barra invertida será interpretada como parte da string. Por exemplo, o `\n` acima seria interpretado como um caractere de nova linha. Alternativamente, você pode duplicar cada barra invertida na versão doctest (e não usar uma string bruta):

```
>>> def f(x):
...     '''Backslashes in a raw docstring: m\\n'''
...
>>> print(f.__doc__)
Backslashes in a raw docstring: m\n
```

- A coluna inicial não importa:

```
>>> assert "Easy!"
      >>> import math
```

```
>>> math.floor(1.9)
1
```

e tantos caracteres de espaço em branco iniciais são removidos da saída esperada quantos apareceram na linha inicial `'>>> '` que iniciou o exemplo.

Qual é o contexto de execução?

By default, each time **`doctest`** finds a docstring to test, it uses a *shallow copy* of **`M`**'s globals, so that running tests doesn't change the module's real globals, and so that one test in **`M`** can't leave behind crumbs that accidentally allow another test to work. This means examples can freely use any names defined at top-level in **`M`**, and names defined earlier in the docstring being run. Examples cannot see names defined in other docstrings.

Você pode forçar o uso de seu próprio dicionário como contexto de execução passando `globs=seu_dicionario` para **`testmod()`** ou **`testfile()`**.

E quanto às exceções?

Não tem problema, desde que o traceback seja a única saída produzida pelo exemplo: basta colar o traceback. [[1]] Como os tracebacks contêm detalhes que provavelmente mudarão rapidamente (por exemplo, caminhos exatos de arquivos e números de linha), este é um caso em que o doctest trabalha duro para ser flexível no que aceita.

Exemplo simples:

```
>>> [1, 2, 3].remove(42)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: list.remove(x): x not in list
```

Esse doctest será bem-sucedido se **`ValueError`** for levantada, com o detalhe `list.remove(x): x not in list`, conforme mostrado.

A saída esperada para uma exceção deve começar com um cabeçalho do traceback, que pode ser qualquer uma das duas linhas a seguir, indentadas da mesma forma que a primeira linha do exemplo:

```
Traceback (most recent call last):
Traceback (innermost last):
```

The traceback header is followed by an optional traceback stack, whose contents are ignored by doctest. The traceback stack is typically omitted, or copied verbatim from an interactive session.

The traceback stack is followed by the most interesting part: the line(s) containing the exception type and detail. This is usually the last line of a traceback, but can extend across multiple lines if the exception has a multi-line detail:

```
>>> raise ValueError('multi\n    line\ndetail')
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ValueError: multi
    line
detail
```

The last three lines (starting with `ValueError`) are compared against the exception's type and detail, and the rest are ignored.

Best practice is to omit the traceback stack, unless it adds significant documentation value to the example. So the last example is probably better as:

```
>>> raise ValueError('multi\n    line\ndetail')
Traceback (most recent call last):
    ...
ValueError: multi
    line
detail
```

Note that tracebacks are treated very specially. In particular, in the rewritten example, the use of `...` is independent of doctest's `ELLIPSIS` option. The ellipsis in that example could be left out, or could just as well be three (or

three hundred) commas or digits, or an indented transcript of a Monty Python skit.

Some details you should read once, but won't need to remember:

- Doctest can't guess whether your expected output came from an exception traceback or from ordinary printing. So, e.g., an example that expects `ValueError: 42 is prime` will pass whether **`ValueError`** is actually raised or if the example merely prints that traceback text. In practice, ordinary output rarely begins with a traceback header line, so this doesn't create real problems.
- Each line of the traceback stack (if present) must be indented further than the first line of the example, *or* start with a non-alphanumeric character. The first line following the traceback header indented the same and starting with an alphanumeric is taken to be the start of the exception detail. Of course this does the right thing for genuine tracebacks.
- When the **`IGNORE_EXCEPTION_DETAIL`** doctest option is specified, everything following the leftmost colon and any module information in the exception name is ignored.
- The interactive shell omits the traceback header line for some **`SyntaxError`**s. But doctest uses the traceback header line to distinguish exceptions from non-exceptions. So in the rare case where you need to test a **`SyntaxError`** that omits the traceback header, you will need to manually add the traceback header line to your test example.

- For some exceptions, Python displays the position of the error using `^` markers and tildes:

  ```
  >>> 1 + None
    File "<stdin>", line 1
      1 + None
      ~~^~~~~~
  TypeError: unsupported operand type(s) for +: 'int' and 'Nor
  ```

  Since the lines showing the position of the error come before the exception type and detail, they are not checked by doctest. For

example, the following test would pass, even though it puts the ^ marker in the wrong location:

```
>>> 1 + None
  File "<stdin>", line 1
    1 + None
    ^~~~~~~~
TypeError: unsupported operand type(s) for +: 'int' and 'Nor
```

Flags opcionais

A number of option flags control various aspects of doctest's behavior. Symbolic names for the flags are supplied as module constants, which can be bitwise ORed together and passed to various functions. The names can also be used in doctest directives, and may be passed to the doctest command line interface via the `-o` option.

The first group of options define test semantics, controlling aspects of how doctest decides whether actual output matches an example's expected output:

doctest.DONT_ACCEPT_TRUE_FOR_1

By default, if an expected output block contains just `1`, an actual output block containing just `1` or just `True` is considered to be a match, and similarly for `0` versus `False`. When **`DONT_ACCEPT_TRUE_FOR_1`** is specified, neither substitution is allowed. The default behavior caters to that Python changed the return type of many functions from integer to boolean; doctests expecting "little integer" output still work in these cases. This option will probably go away, but not for several years.

doctest.DONT_ACCEPT_BLANKLINE

By default, if an expected output block contains a line containing only the string `<BLANKLINE>`, then that line will match a blank line in the actual output. Because a genuinely blank line delimits the expected output, this is the only way to communicate that a blank line is expected. When **`DONT_ACCEPT_BLANKLINE`** is specified, this substitution is not allowed.

doctest.NORMALIZE_WHITESPACE

When specified, all sequences of whitespace (blanks and newlines) are treated as equal. Any sequence of whitespace within the expected output will match any sequence of whitespace within the actual output. By default, whitespace must match exactly. **`NORMALIZE_WHITESPACE`** is especially useful when a line of expected output is very long, and you want to wrap it across multiple lines in your source.

doctest.ELLIPSIS

When specified, an ellipsis marker (`...`) in the expected output can match any substring in the actual output. This includes substrings that span line boundaries, and empty substrings, so it's best to keep usage of this simple. Complicated uses can lead to the same kinds of "oops, it matched too much!" surprises that `.*` is prone to in regular expressions.

doctest.IGNORE_EXCEPTION_DETAIL

When specified, doctests expecting exceptions pass so long as an exception of the expected type is raised, even if the details (message and fully qualified exception name) don't match.

For example, an example expecting `ValueError: 42` will pass if the actual exception raised is `ValueError: 3*14`, but will fail if, say, a **`TypeError`** is raised instead. It will also ignore any fully qualified name included before the exception class, which can vary between implementations and versions of Python and the code/libraries in use. Hence, all three of these variations will work with the flag specified:

```
>>> raise Exception('message')
Traceback (most recent call last):
Exception: message

>>> raise Exception('message')
Traceback (most recent call last):
builtins.Exception: message

>>> raise Exception('message')
Traceback (most recent call last):
__main__.Exception: message
```

Note that **ELLIPSIS** can also be used to ignore the details of the exception message, but such a test may still fail based on whether the module name is present or matches exactly.

Alterado na versão 3.2: **IGNORE_EXCEPTION_DETAIL** now also ignores any information relating to the module containing the exception under test.

doctest.SKIP

When specified, do not run the example at all. This can be useful in contexts where doctest examples serve as both documentation and test cases, and an example should be included for documentation purposes, but should not be checked. E.g., the example's output might be random; or the example might depend on resources which would be unavailable to the test driver.

The SKIP flag can also be used for temporarily "commenting out" examples.

doctest.COMPARISON_FLAGS

A bitmask or'ing together all the comparison flags above.

The second group of options controls how test failures are reported:

doctest.REPORT_UDIFF

When specified, failures that involve multi-line expected and actual outputs are displayed using a unified diff.

doctest.REPORT_CDIFF

When specified, failures that involve multi-line expected and actual outputs will be displayed using a context diff.

doctest.REPORT_NDIFF

When specified, differences are computed by `difflib.Differ`, using the same algorithm as the popular `ndiff.py` utility. This is the only method that marks differences within lines as well as across lines. For example, if a line of expected output contains digit `1` where actual

output contains letter `l`, a line is inserted with a caret marking the mismatching column positions.

doctest.REPORT_ONLY_FIRST_FAILURE

When specified, display the first failing example in each doctest, but suppress output for all remaining examples. This will prevent doctest from reporting correct examples that break because of earlier failures; but it might also hide incorrect examples that fail independently of the first failure. When **`REPORT_ONLY_FIRST_FAILURE`** is specified, the remaining examples are still run, and still count towards the total number of failures reported; only the output is suppressed.

doctest.FAIL_FAST

When specified, exit after the first failing example and don't attempt to run the remaining examples. Thus, the number of failures reported will be at most 1. This flag may be useful during debugging, since examples after the first failure won't even produce debugging output.

doctest.REPORTING_FLAGS

A bitmask or'ing together all the reporting flags above.

There is also a way to register new option flag names, though this isn't useful unless you intend to extend **`doctest`** internals via subclassing:

doctest.register_optionflag(*name*)

Create a new option flag with a given name, and return the new flag's integer value. **`register_optionflag()`** can be used when subclassing **`OutputChecker`** or **`DocTestRunner`** to create new options that are supported by your subclasses. **`register_optionflag()`** should always be called using the following idiom:

```
MY_FLAG = register_optionflag('MY_FLAG')
```

Directives

Doctest directives may be used to modify the option flags for an individual example. Doctest directives are special Python comments following an

example's source code:

```
directive:              "#" "doctest:" directive_options
directive_options:      directive_option ("," directive_option)*
directive_option:       on_or_off directive_option_name
on_or_off:              "+" | "-"
directive_option_name:  "DONT_ACCEPT_BLANKLINE" | "NORMALIZE_WHI
```

Whitespace is not allowed between the `+` or `-` and the directive option name. The directive option name can be any of the option flag names explained above.

An example's doctest directives modify doctest's behavior for that single example. Use `+` to enable the named behavior, or `-` to disable it.

Por exemplo, este teste é aprovado:

```
>>> print(list(range(20)))  # doctest: +NORMALIZE_WHITESPACE
[0,   1,  2,  3,  4,  5,  6,  7,  8,  9,
10,  11, 12, 13, 14, 15, 16, 17, 18, 19]
```

Without the directive it would fail, both because the actual output doesn't have two blanks before the single-digit list elements, and because the actual output is on a single line. This test also passes, and also requires a directive to do so:

```
>>> print(list(range(20)))  # doctest: +ELLIPSIS
[0, 1, ..., 18, 19]
```

Multiple directives can be used on a single physical line, separated by commas:

```
>>> print(list(range(20)))  # doctest: +ELLIPSIS, +NORMALIZE_WHI
[0,    1, ...,   18,    19]
```

If multiple directive comments are used for a single example, then they are combined:

```
>>> print(list(range(20)))  # doctest: +ELLIPSIS
...                         # doctest: +NORMALIZE_WHITESPACE
[0,    1, ...,   18,    19]
```

As the previous example shows, you can add `...` lines to your example containing only directives. This can be useful when an example is too long for a directive to comfortably fit on the same line:

```
>>> print(list(range(5)) + list(range(10, 20)) + list(range(30,
... # doctest: +ELLIPSIS
[0, ..., 4, 10, ..., 19, 30, ..., 39]
```

Note that since all options are disabled by default, and directives apply only to the example they appear in, enabling options (via `+` in a directive) is usually the only meaningful choice. However, option flags can also be passed to functions that run doctests, establishing different defaults. In such cases, disabling an option via `-` in a directive can be useful.

Avisos

`doctest` is serious about requiring exact matches in expected output. If even a single character doesn’t match, the test fails. This will probably surprise you a few times, as you learn exactly what Python does and doesn’t guarantee about output. For example, when printing a set, Python doesn’t guarantee that the element is printed in any particular order, so a test like

```
>>> foo()
{"spam", "eggs"}
```

is vulnerable! One workaround is to do

```
>>> foo() == {"spam", "eggs"}
True
```

instead. Another is to do

```
>>> d = sorted(foo())
>>> d
['eggs', 'spam']
```

There are others, but you get the idea.

Another bad idea is to print things that embed an object address, like

```
>>> id(1.0)  # certain to fail some of the time
7948648
>>> class C: pass
>>> C()  # the default repr() for instances embeds an address
<C object at 0x00AC18F0>
```

The **ELLIPSIS** directive gives a nice approach for the last example:

```
>>> C()  # doctest: +ELLIPSIS
<C object at 0x...>
```

Floating-point numbers are also subject to small output variations across platforms, because Python defers to the platform C library for some floating-point calculations, and C libraries vary widely in quality here.

```
>>> 1000**0.1  # risky
1.9952623149688797
>>> round(1000**0.1, 9) # safer
1.995262315
>>> print(f'{1000**0.1:.4f}') # much safer
1.9953
```

Numbers of the form `I/2.**J` are safe across all platforms, and I often contrive doctest examples to produce numbers of that form:

```
>>> 3./4  # utterly safe
0.75
```

Simple fractions are also easier for people to understand, and that makes for better documentation.

Basic API

The functions **testmod()** and **testfile()** provide a simple interface to doctest that should be sufficient for most basic uses. For a less formal introduction to these two functions, see sections Uso simples: verificando

exemplos em Docstrings and Utilização comum: Verificando exemplos em um arquivo texto.

doctest.testfile(*filename*, *module_relative=True*, *name=None*, *package=None*, *globs=None*, *verbose=None*, *report=True*, *optionflags=0*, *extraglobs=None*, *raise_on_error=False*, *parser=DocTestParser()*, *encoding=None*)

All arguments except *filename* are optional, and should be specified in keyword form.

Test examples in the file named *filename*. Return `(failure_count, test_count)`.

Optional argument *module_relative* specifies how the filename should be interpreted:

- If *module_relative* is `True` (the default), then *filename* specifies an OS-independent module-relative path. By default, this path is relative to the calling module's directory; but if the *package* argument is specified, then it is relative to that package. To ensure OS-independence, *filename* should use `/` characters to separate path segments, and may not be an absolute path (i.e., it may not begin with `/`).
- If *module_relative* is `False`, then *filename* specifies an OS-specific path. The path may be absolute or relative; relative paths are resolved with respect to the current working directory.

Optional argument *name* gives the name of the test; by default, or if `None`, `os.path.basename(filename)` is used.

Optional argument *package* is a Python package or the name of a Python package whose directory should be used as the base directory for a module-relative filename. If no package is specified, then the calling module's directory is used as the base directory for module-relative filenames. It is an error to specify *package* if *module_relative* is `False`.

Optional argument *globs* gives a dict to be used as the globals when executing examples. A new shallow copy of this dict is created for the doctest, so its examples start with a clean slate. By default, or if `None`, a new empty dict is used.

Optional argument *extraglobs* gives a dict merged into the globals used to execute examples. This works like `dict.update()`: if *globs* and *extraglobs* have a common key, the associated value in *extraglobs* appears in the combined dict. By default, or if `None`, no extra globals are used. This is an advanced feature that allows parameterization of doctests. For example, a doctest can be written for a base class, using a generic name for the class, then reused to test any number of subclasses by passing an *extraglobs* dict mapping the generic name to the subclass to be tested.

Optional argument *verbose* prints lots of stuff if true, and prints only failures if false; by default, or if `None`, it's true if and only if `'-v'` is in `sys.argv`.

Optional argument *report* prints a summary at the end when true, else prints nothing at the end. In verbose mode, the summary is detailed, else the summary is very brief (in fact, empty if all tests passed).

Optional argument *optionflags* (default value `0`) takes the bitwise OR of option flags. See section Flags opcionais.

Optional argument *raise_on_error* defaults to false. If true, an exception is raised upon the first failure or unexpected exception in an example. This allows failures to be post-mortem debugged. Default behavior is to continue running examples.

Optional argument *parser* specifies a `DocTestParser` (or subclass) that should be used to extract tests from the files. It defaults to a normal parser (i.e., `DocTestParser()`).

Optional argument *encoding* specifies an encoding that should be used to convert the file to unicode.

doctest.testmod(*m=None*, *name=None*, *globs=None*, *verbose=None*, *report=True*, *optionflags=0*, *extraglobs=None*, *raise_on_error=False*, *exclude_empty=False*)

All arguments are optional, and all except for *m* should be specified in keyword form.

Test examples in docstrings in functions and classes reachable from module *m* (or module `__main__` if *m* is not supplied or is `None`), starting with `m.__doc__`.

Also test examples reachable from dict `m.__test__`, if it exists. `m.__test__` maps names (strings) to functions, classes and strings; function and class docstrings are searched for examples; strings are searched directly, as if they were docstrings.

Only docstrings attached to objects belonging to module *m* are searched.

Return `(failure_count, test_count)`.

Optional argument *name* gives the name of the module; by default, or if `None`, `m.__name__` is used.

Optional argument *exclude_empty* defaults to false. If true, objects for which no doctests are found are excluded from consideration. The default is a backward compatibility hack, so that code still using `doctest.master.summarize` in conjunction with **`testmod()`** continues to get output for objects with no tests. The *exclude_empty* argument to the newer `DocTestFinder` constructor defaults to true.

Optional arguments *extraglobs*, *verbose*, *report*, *optionflags*, *raise_on_error*, and *globs* are the same as for function `testfile()` above, except that *globs* defaults to `m.__dict__`.

doctest.run_docstring_examples(*f*, *globs*, *verbose=False*, *name='NoName'*, *compileflags=None*, *optionflags=0*)

Test examples associated with object *f*; for example, *f* may be a string, a module, a function, or a class object.

A shallow copy of dictionary argument *globs* is used for the execution context.

Optional argument *name* is used in failure messages, and defaults to `"NoName"`.

If optional argument *verbose* is true, output is generated even if there are no failures. By default, output is generated only in case of an example failure.

Optional argument *compileflags* gives the set of flags that should be used by the Python compiler when running the examples. By default, or if `None`, flags are deduced corresponding to the set of future features found in *globs*.

Optional argument *optionflags* works as for function **`testfile()`** above.

API do Unittest

As your collection of doctest'ed modules grows, you'll want a way to run all their doctests systematically. **`doctest`** provides two functions that can be used to create **`unittest`** test suites from modules and text files containing doctests. To integrate with **`unittest`** test discovery, include a load_tests function in your test module:

```
import unittest
import doctest
import my_module_with_doctests

def load_tests(loader, tests, ignore):
    tests.addTests(doctest.DocTestSuite(my_module_with_doctests)
    return tests
```

There are two main functions for creating **`unittest.TestSuite`** instances from text files and modules with doctests:

doctest.DocFileSuite(**paths*, *module_relative=True*, *package=None*, *setUp=None*, *tearDown=None*, *globs=None*, *optionflags=0*,

parser=DocTestParser(), encoding=None)

Convert doctest tests from one or more text files to a `unittest.TestSuite`.

The returned `unittest.TestSuite` is to be run by the unittest framework and runs the interactive examples in each file. Each file is run as a separate unit test, and each example in a file is run as a subtest. If any example in a file fails, then the synthesized unit test fails. The traceback for failure or error contains the name of the file containing the test and a (sometimes approximate) line number. If all the examples in a file are skipped, then the synthesized unit test is also marked as skipped.

Pass one or more paths (as strings) to text files to be examined.

Options may be provided as keyword arguments:

Optional argument *module_relative* specifies how the filenames in *paths* should be interpreted:

- If *module_relative* is `True` (the default), then each filename in *paths* specifies an OS-independent module-relative path. By default, this path is relative to the calling module's directory; but if the *package* argument is specified, then it is relative to that package. To ensure OS-independence, each filename should use `/` characters to separate path segments, and may not be an absolute path (i.e., it may not begin with `/`).
- If *module_relative* is `False`, then each filename in *paths* specifies an OS-specific path. The path may be absolute or relative; relative paths are resolved with respect to the current working directory.

Optional argument *package* is a Python package or the name of a Python package whose directory should be used as the base directory for module-relative filenames in *paths*. If no package is specified, then the calling module's directory is used as the base directory for module-relative filenames. It is an error to specify *package* if *module_relative* is `False`.

Optional argument *setUp* specifies a set-up function for the test suite. This is called before running the tests in each file. The *setUp* function will be passed a **`DocTest`** object. The *setUp* function can access the test globals as the **`globs`** attribute of the test passed.

Optional argument *tearDown* specifies a tear-down function for the test suite. This is called after running the tests in each file. The *tearDown* function will be passed a **`DocTest`** object. The *tearDown* function can access the test globals as the **`globs`** attribute of the test passed.

Optional argument *globs* is a dictionary containing the initial global variables for the tests. A new copy of this dictionary is created for each test. By default, *globs* is a new empty dictionary.

Optional argument *optionflags* specifies the default doctest options for the tests, created by or-ing together individual option flags. See section Flags opcionais. See function **`set_unittest_reportflags()`** below for a better way to set reporting options.

Optional argument *parser* specifies a **`DocTestParser`** (or subclass) that should be used to extract tests from the files. It defaults to a normal parser (i.e., `DocTestParser()`).

Optional argument *encoding* specifies an encoding that should be used to convert the file to unicode.

The global `__file__` is added to the globals provided to doctests loaded from a text file using **`DocFileSuite()`**.

Alterado na versão 3.15: Run each example as a subtest.

doctest.DocTestSuite(*module=None*, *globs=None*, *extraglobs=None*, *test_finder=None*, *setUp=None*, *tearDown=None*, *optionflags=0*, *checker=None*)

Convert doctest tests for a module to a **`unittest.TestSuite`**.

The returned **`unittest.TestSuite`** is to be run by the unittest framework and runs each doctest in the module. Each docstring is run as

a separate unit test, and each example in a docstring is run as a subtest. If any of the doctests fail, then the synthesized unit test fails. The traceback for failure or error contains the name of the file containing the test and a (sometimes approximate) line number. If all the examples in a docstring are skipped, then the synthesized unit test is also marked as skipped.

Optional argument *module* provides the module to be tested. It can be a module object or a (possibly dotted) module name. If not specified, the module calling this function is used.

Optional argument *globs* is a dictionary containing the initial global variables for the tests. A new copy of this dictionary is created for each test. By default, *globs* is the module's `__dict__`.

Optional argument *extraglobs* specifies an extra set of global variables, which is merged into *globs*. By default, no extra globals are used.

Optional argument *test_finder* is the `DocTestFinder` object (or a drop-in replacement) that is used to extract doctests from the module.

Optional arguments *setUp*, *tearDown*, and *optionflags* are the same as for function `DocFileSuite()` above, but they are called for each docstring.

This function uses the same search technique as `testmod()`.

Alterado na versão 3.5: **`DocTestSuite()`** returns an empty `unittest.TestSuite` if *module* contains no docstrings instead of raising `ValueError`.

Alterado na versão 3.15: Run each example as a subtest.

Under the covers, `DocTestSuite()` creates a `unittest.TestSuite` out of **`doctest.DocTestCase`** instances, and **`DocTestCase`** is a subclass of `unittest.TestCase`. **`DocTestCase`** isn't documented here (it's an internal detail), but studying its code can answer questions about the exact details of `unittest` integration.

Similarly, **DocFileSuite()** creates a **unittest.TestSuite** out of **doctest.DocFileCase** instances, and **DocFileCase** is a subclass of **DocTestCase**.

So both ways of creating a **unittest.TestSuite** run instances of **DocTestCase**. This is important for a subtle reason: when you run **doctest** functions yourself, you can control the **doctest** options in use directly, by passing option flags to **doctest** functions. However, if you're writing a **unittest** framework, **unittest** ultimately controls when and how tests get run. The framework author typically wants to control **doctest** reporting options (perhaps, e.g., specified by command line options), but there's no way to pass options through **unittest** to **doctest** test runners.

For this reason, **doctest** also supports a notion of **doctest** reporting flags specific to **unittest** support, via this function:

doctest.set_unittest_reportflags(*flags*)

Set the **doctest** reporting flags to use.

Argument *flags* takes the bitwise OR of option flags. See section Flags opcionais. Only "reporting flags" can be used.

This is a module-global setting, and affects all future doctests run by module **unittest**: the **runTest()** method of **DocTestCase** looks at the option flags specified for the test case when the **DocTestCase** instance was constructed. If no reporting flags were specified (which is the typical and expected case), **doctest**'s **unittest** reporting flags are bitwise ORed into the option flags, and the option flags so augmented are passed to the **DocTestRunner** instance created to run the doctest. If any reporting flags were specified when the **DocTestCase** instance was constructed, **doctest**'s **unittest** reporting flags are ignored.

The value of the **unittest** reporting flags in effect before the function was called is returned by the function.

Advanced API

The basic API is a simple wrapper that's intended to make doctest easy to use. It is fairly flexible, and should meet most users' needs; however, if you require more fine-grained control over testing, or wish to extend doctest's capabilities, then you should use the advanced API.

The advanced API revolves around two container classes, which are used to store the interactive examples extracted from doctest cases:

- **`Example`**: A single Python statement, paired with its expected output.
- **`DocTest`**: A collection of **`Example`**s, typically extracted from a single docstring or text file.

Additional processing classes are defined to find, parse, and run, and check doctest examples:

- **`DocTestFinder`**: Finds all docstrings in a given module, and uses a **`DocTestParser`** to create a **`DocTest`** from every docstring that contains interactive examples.
- **`DocTestParser`**: Creates a **`DocTest`** object from a string (such as an object's docstring).
- **`DocTestRunner`**: Executes the examples in a **`DocTest`**, and uses an **`OutputChecker`** to verify their output.
- **`OutputChecker`**: Compares the actual output from a doctest example with the expected output, and decides whether they match.

The relationships among these processing classes are summarized in the following diagram:

```
                             list of:
+------+                   +---------+
|module| --DocTestFinder-> | DocTest | --DocTestRunner-> results
+------+    |        ^     +---------+     |       ^    (printed
            |        |     | Example |     |       |
            v        |     |   ...   |     v       |
           DocTestParser   | Example |   OutputChecker
                           +---------+
```

DocTest Objects

class doctest.DocTest(*examples*, *globs*, *name*, *filename*, *lineno*, *docstring*)

A collection of doctest examples that should be run in a single namespace. The constructor arguments are used to initialize the attributes of the same names.

`DocTest` defines the following attributes. They are initialized by the constructor, and should not be modified directly.

examples

A list of **`Example`** objects encoding the individual interactive Python examples that should be run by this test.

globs

The namespace (aka globals) that the examples should be run in. This is a dictionary mapping names to values. Any changes to the namespace made by the examples (such as binding new variables) will be reflected in **`globs`** after the test is run.

name

A string name identifying the **`DocTest`**. Typically, this is the name of the object or file that the test was extracted from.

filename

The name of the file that this **`DocTest`** was extracted from; or `None` if the filename is unknown, or if the **`DocTest`** was not extracted from a file.

lineno

The line number within **`filename`** where this **`DocTest`** begins, or `None` if the line number is unavailable. This line number is zero-based with respect to the beginning of the file.

docstring

The string that the test was extracted from, or `None` if the string is unavailable, or if the test was not extracted from a string.

Example Objects

class doctest.Example(*source*, *want*, *exc_msg=None*, *lineno=0*, *indent=0*, *options=None*)

A single interactive example, consisting of a Python statement and its expected output. The constructor arguments are used to initialize the attributes of the same names.

`Example` defines the following attributes. They are initialized by the constructor, and should not be modified directly.

source

A string containing the example's source code. This source code consists of a single Python statement, and always ends with a newline; the constructor adds a newline when necessary.

want

The expected output from running the example's source code (either from stdout, or a traceback in case of exception). `want` ends with a newline unless no output is expected, in which case it's an empty string. The constructor adds a newline when necessary.

exc_msg

The exception message generated by the example, if the example is expected to generate an exception; or `None` if it is not expected to generate an exception. This exception message is compared against the return value of `traceback.format_exception_only()`. `exc_msg` ends with a newline unless it's `None`. The constructor adds a newline if needed.

lineno

The line number within the string containing this example where the example begins. This line number is zero-based with respect to the beginning of the containing string.

indent

The example's indentation in the containing string, i.e., the number of space characters that precede the example's first prompt.

options

A dictionary mapping from option flags to `True` or `False`, which is used to override default options for this example. Any option flags not contained in this dictionary are left at their default value (as specified by the **`DocTestRunner`**'s optionflags). By default, no options are set.

DocTestFinder objects

class doctest.DocTestFinder(*verbose=False*, *parser=DocTestParser()*, *recurse=True*, *exclude_empty=True*)

A processing class used to extract the **`DocTest`**s that are relevant to a given object, from its docstring and the docstrings of its contained objects. **`DocTest`**s can be extracted from modules, classes, functions, methods, staticmethods, classmethods, and properties.

The optional argument *verbose* can be used to display the objects searched by the finder. It defaults to `False` (no output).

The optional argument *parser* specifies the **`DocTestParser`** object (or a drop-in replacement) that is used to extract doctests from docstrings.

If the optional argument *recurse* is false, then **`DocTestFinder.find()`** will only examine the given object, and not any contained objects.

If the optional argument *exclude_empty* is false, then **`DocTestFinder.find()`** will include tests for objects with empty docstrings.

`DocTestFinder` defines the following method:

find(*obj[, name][, module][, globs][, extraglobs]*)

Return a list of the **`DocTest`**s that are defined by *obj*'s docstring, or by any of its contained objects' docstrings.

The optional argument *name* specifies the object's name; this name will be used to construct names for the returned **`DocTest`**s. If *name*

is not specified, then `obj.__name__` is used.

The optional parameter *module* is the module that contains the given object. If the module is not specified or is `None`, then the test finder will attempt to automatically determine the correct module. The object's module is used:

- As a default namespace, if *globs* is not specified.
- To prevent the DocTestFinder from extracting DocTests from objects that are imported from other modules. (Contained objects with modules other than *module* are ignored.)
- To find the name of the file containing the object.
- To help find the line number of the object within its file.

If *module* is `False`, no attempt to find the module will be made. This is obscure, of use mostly in testing doctest itself: if *module* is `False`, or is `None` but cannot be found automatically, then all objects are considered to belong to the (non-existent) module, so all contained objects will (recursively) be searched for doctests.

The globals for each **`DocTest`** is formed by combining *globs* and *extraglobs* (bindings in *extraglobs* override bindings in *globs*). A new shallow copy of the globals dictionary is created for each **`DocTest`**. If *globs* is not specified, then it defaults to the module's **`__dict__`**, if specified, or `{}` otherwise. If *extraglobs* is not specified, then it defaults to `{}`.

DocTestParser objects

class doctest.DocTestParser

A processing class used to extract interactive examples from a string, and use them to create a **`DocTest`** object.

`DocTestParser` defines the following methods:

get_doctest(*string*, *globs*, *name*, *filename*, *lineno*)

Extract all doctest examples from the given string, and collect them into a **`DocTest`** object.

globs, *name*, *filename*, and *lineno* are attributes for the new **`DocTest`** object. See the documentation for **`DocTest`** for more information.

get_examples(*string*, *name='<string>'*)

Extract all doctest examples from the given string, and return them as a list of **`Example`** objects. Line numbers are 0-based. The optional argument *name* is a name identifying this string, and is only used for error messages.

parse(*string*, *name='<string>'*)

Divide the given string into examples and intervening text, and return them as a list of alternating **`Example`**s and strings. Line numbers for the **`Example`**s are 0-based. The optional argument *name* is a name identifying this string, and is only used for error messages.

TestResults objects

class doctest.TestResults(*failed*, *attempted*)

failed

Number of failed tests.

attempted

Number of attempted tests.

skipped

Number of skipped tests.

Adicionado na versão 3.13.

DocTestRunner objects

class doctest.DocTestRunner(*checker=None*, *verbose=None*, *optionflags=0*)

A processing class used to execute and verify the interactive examples in a **`DocTest`**.

The comparison between expected outputs and actual outputs is done by an **`OutputChecker`**. This comparison may be customized with a number of option flags; see section Flags opcionais for more information. If the option flags are insufficient, then the comparison may also be customized by passing a subclass of **`OutputChecker`** to the constructor.

The test runner's display output can be controlled in two ways. First, an output function can be passed to **`run()`**; this function will be called with strings that should be displayed. It defaults to `sys.stdout.write`. If capturing the output is not sufficient, then the display output can be also customized by subclassing DocTestRunner, and overriding the methods **`report_skip()`**, **`report_start()`**, **`report_success()`**, **`report_unexpected_exception()`**, and **`report_failure()`**.

The optional keyword argument *checker* specifies the **`OutputChecker`** object (or drop-in replacement) that should be used to compare the expected outputs to the actual outputs of doctest examples.

The optional keyword argument *verbose* controls the **`DocTestRunner`**'s verbosity. If *verbose* is `True`, then information is printed about each example, as it is run. If *verbose* is `False`, then only failures are printed. If *verbose* is unspecified, or `None`, then verbose output is used iff the command-line switch `-v` is used.

The optional keyword argument *optionflags* can be used to control how the test runner compares expected output to actual output, and how it displays failures. For more information, see section Flags opcionais.

The test runner accumulates statistics. The aggregated number of attempted, failed and skipped examples is also available via the **`tries`**, **`failures`** and **`skips`** attributes. The **`run()`** and **`summarize()`** methods return a **`TestResults`** instance.

`DocTestRunner` defines the following methods:

report_skip(*out*, *test*, *example*)

Report that the given example was skipped. This method is provided to allow subclasses of **`DocTestRunner`** to customize their output; it

should not be called directly.

example is the example about to be processed. *test* is the test containing *example*. *out* is the output function that was passed to `DocTestRunner.run()`.

Adicionado na versão 3.15.

report_start(*out*, *test*, *example*)

Report that the test runner is about to process the given example. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly.

example is the example about to be processed. *test* is the test containing *example*. *out* is the output function that was passed to `DocTestRunner.run()`.

report_success(*out*, *test*, *example*, *got*)

Report that the given example ran successfully. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly.

example is the example about to be processed. *got* is the actual output from the example. *test* is the test containing *example*. *out* is the output function that was passed to `DocTestRunner.run()`.

report_failure(*out*, *test*, *example*, *got*)

Report that the given example failed. This method is provided to allow subclasses of `DocTestRunner` to customize their output; it should not be called directly.

example is the example about to be processed. *got* is the actual output from the example. *test* is the test containing *example*. *out* is the output function that was passed to `DocTestRunner.run()`.

report_unexpected_exception(*out*, *test*, *example*, *exc_info*)

Report that the given example raised an unexpected exception. This method is provided to allow subclasses of **`DocTestRunner`** to customize their output; it should not be called directly.

example is the example about to be processed. *exc_info* is a tuple containing information about the unexpected exception (as returned by **`sys.exc_info()`**). *test* is the test containing *example*. *out* is the output function that was passed to **`DocTestRunner.run()`**.

run(*test*, *compileflags=None*, *out=None*, *clear_globs=True*)

Run the examples in *test* (a **`DocTest`** object), and display the results using the writer function *out*. Return a **`TestResults`** instance.

The examples are run in the namespace `test.globs`. If *clear_globs* is true (the default), then this namespace will be cleared after the test runs, to help with garbage collection. If you would like to examine the namespace after the test completes, then use *clear_globs=False*.

compileflags gives the set of flags that should be used by the Python compiler when running the examples. If not specified, then it will default to the set of future-import flags that apply to *globs*.

The output of each example is checked using the **`DocTestRunner`**'s output checker, and the results are formatted by the **`DocTestRunner.report_*()`** methods.

summarize(*verbose=None*)

Print a summary of all the test cases that have been run by this DocTestRunner, and return a **`TestResults`** instance.

The optional *verbose* argument controls how detailed the summary is. If the verbosity is not specified, then the **`DocTestRunner`**'s verbosity is used.

`DocTestParser` has the following attributes:

tries

Number of attempted examples.

failures
Number of failed examples.

skips
Number of skipped examples.

Adicionado na versão 3.13.

OutputChecker objects

class doctest.OutputChecker
A class used to check the whether the actual output from a doctest example matches the expected output. **`OutputChecker`** defines two methods: **`check_output()`**, which compares a given pair of outputs, and returns `True` if they match; and **`output_difference()`**, which returns a string describing the differences between two outputs.

`OutputChecker` define os seguintes métodos:

check_output(*want*, *got*, *optionflags*)
Return `True` iff the actual output from an example (*got*) matches the expected output (*want*). These strings are always considered to match if they are identical; but depending on what option flags the test runner is using, several non-exact match types are also possible. See section Flags opcionais for more information about option flags.

output_difference(*example*, *got*, *optionflags*)
Return a string describing the differences between the expected output for a given example (*example*) and the actual output (*got*). *optionflags* is the set of option flags used to compare *want* and *got*.

Depuração

Doctest provides several mechanisms for debugging doctest examples:

- Several functions convert doctests to executable Python programs, which can be run under the Python debugger, **pdb**.

- The **DebugRunner** class is a subclass of **DocTestRunner** that raises an exception for the first failing example, containing information about that example. This information can be used to perform post-mortem debugging on the example.

- The **unittest** cases generated by **DocTestSuite()** support the **debug()** method defined by **unittest.TestCase**.

- You can add a call to **pdb.set_trace()** in a doctest example, and you'll drop into the Python debugger when that line is executed. Then you can inspect current values of variables, and so on. For example, suppose `a.py` contains just this module docstring:

```
"""
>>> def f(x):
...     g(x*2)
>>> def g(x):
...     print(x+3)
...     import pdb; pdb.set_trace()
>>> f(3)
9
"""
```

Then an interactive Python session may look like this:

```
>>> import a, doctest
>>> doctest.testmod(a)
--Return--
> <doctest a[1]>(3)g()->None
-> import pdb; pdb.set_trace()
(Pdb) list
  1     def g(x):
  2         print(x+3)
  3  ->     import pdb; pdb.set_trace()
[EOF]
(Pdb) p x
6
(Pdb) step
```

```
--Return--
> <doctest a[0]>(2)f()->None
-> g(x*2)
(Pdb) list
  1     def f(x):
  2  ->     g(x*2)
[EOF]
(Pdb) p x
3
(Pdb) step
--Return--
> <doctest a[2]>(1)?()->None
-> f(3)
(Pdb) cont
(0, 3)
>>>
```

Functions that convert doctests to Python code, and possibly run the synthesized code under the debugger:

doctest.script_from_examples(*s*)

Convert text with examples to a script.

Argument *s* is a string containing doctest examples. The string is converted to a Python script, where doctest examples in *s* are converted to regular code, and everything else is converted to Python comments. The generated script is returned as a string. For example,

```
import doctest
print(doctest.script_from_examples(r"""
    Set x and y to 1 and 2.
    >>> x, y = 1, 2

    Print their sum:
    >>> print(x+y)
    3
"""))
```

displays:

```
# Set x and y to 1 and 2.
x, y = 1, 2
#
# Print their sum:
print(x+y)
# Expected:
## 3
```

This function is used internally by other functions (see below), but can also be useful when you want to transform an interactive Python session into a Python script.

doctest.testsource(*module*, *name*)

Convert the doctest for an object to a script.

Argument *module* is a module object, or dotted name of a module, containing the object whose doctests are of interest. Argument *name* is the name (within the module) of the object with the doctests of interest. The result is a string, containing the object's docstring converted to a Python script, as described for **`script_from_examples()`** above. For example, if module `a.py` contains a top-level function **`f()`**, then

```
import a, doctest
print(doctest.testsource(a, "a.f"))
```

prints a script version of function **`f()`**'s docstring, with doctests converted to code, and the rest placed in comments.

doctest.debug(*module*, *name*, *pm=False*)

Debug the doctests for an object.

The *module* and *name* arguments are the same as for function **`testsource()`** above. The synthesized Python script for the named object's docstring is written to a temporary file, and then that file is run under the control of the Python debugger, **`pdb`**.

A shallow copy of `module.__dict__` is used for both local and global execution context.

Optional argument *pm* controls whether post-mortem debugging is used. If *pm* has a true value, the script file is run directly, and the debugger gets involved only if the script terminates via raising an unhandled exception. If it does, then post-mortem debugging is invoked, via `pdb.post_mortem()`, passing the traceback object from the unhandled exception. If *pm* is not specified, or is false, the script is run under the debugger from the start, via passing an appropriate `exec()` call to `pdb.run()`.

doctest.debug_src(*src*, *pm=False*, *globs=None*)

Debug the doctests in a string.

This is like function `debug()` above, except that a string containing doctest examples is specified directly, via the *src* argument.

Optional argument *pm* has the same meaning as in function `debug()` above.

Optional argument *globs* gives a dictionary to use as both local and global execution context. If not specified, or `None`, an empty dictionary is used. If specified, a shallow copy of the dictionary is used.

The `DebugRunner` class, and the special exceptions it may raise, are of most interest to testing framework authors, and will only be sketched here. See the source code, and especially `DebugRunner`'s docstring (which is a doctest!) for more details:

class doctest.DebugRunner(*checker=None*, *verbose=None*, *optionflags=0*)

A subclass of `DocTestRunner` that raises an exception as soon as a failure is encountered. If an unexpected exception occurs, an `UnexpectedException` exception is raised, containing the test, the example, and the original exception. If the output doesn't match, then a `DocTestFailure` exception is raised, containing the test, the example, and the actual output.

For information about the constructor parameters and methods, see the documentation for `DocTestRunner` in section Advanced API.

There are two exceptions that may be raised by `DebugRunner` instances:

exception doctest.DocTestFailure(*test*, *example*, *got*)

An exception raised by `DocTestRunner` to signal that a doctest example's actual output did not match its expected output. The constructor arguments are used to initialize the attributes of the same names.

`DocTestFailure` defines the following attributes:

DocTestFailure.test

The `DocTest` object that was being run when the example failed.

DocTestFailure.example

The `Example` that failed.

DocTestFailure.got

The example's actual output.

exception doctest.UnexpectedException(*test*, *example*, *exc_info*)

An exception raised by `DocTestRunner` to signal that a doctest example raised an unexpected exception. The constructor arguments are used to initialize the attributes of the same names.

`UnexpectedException` define os seguintes atributos:

UnexpectedException.test

The `DocTest` object that was being run when the example failed.

UnexpectedException.example

The `Example` that failed.

UnexpectedException.exc_info

A tuple containing information about the unexpected exception, as returned by `sys.exc_info()`.

Soapbox

As mentioned in the introduction, **`doctest`** has grown to have three primary uses:

1. Checking examples in docstrings.
2. Regression testing.
3. Executable documentation / literate testing.

These uses have different requirements, and it is important to distinguish them. In particular, filling your docstrings with obscure test cases makes for bad documentation.

When writing a docstring, choose docstring examples with care. There's an art to this that needs to be learned—it may not be natural at first. Examples should add genuine value to the documentation. A good example can often be worth many words. If done with care, the examples will be invaluable for your users, and will pay back the time it takes to collect them many times over as the years go by and things change. I'm still amazed at how often one of my **`doctest`** examples stops working after a "harmless" change.

Doctest also makes an excellent tool for regression testing, especially if you don't skimp on explanatory text. By interleaving prose and examples, it becomes much easier to keep track of what's actually being tested, and why. When a test fails, good prose can make it much easier to figure out what the problem is, and how it should be fixed. It's true that you could write extensive comments in code-based testing, but few programmers do. Many have found that using doctest approaches instead leads to much clearer tests. Perhaps this is simply because doctest makes writing prose a little easier than writing code, while writing comments in code is a little harder. I think it goes deeper than just that: the natural attitude when writing a doctest-based test is that you want to explain the fine points of your software, and illustrate them with examples. This in turn naturally leads to test files that start with the simplest features, and logically progress to complications and edge cases. A coherent narrative is the result, instead of a collection of isolated functions that test isolated bits of functionality seemingly at random. It's a different attitude, and produces different results, blurring the distinction between testing and explaining.

Regression testing is best confined to dedicated objects or files. There are several options for organizing tests:

- Write text files containing test cases as interactive examples, and test the files using `testfile()` or `DocFileSuite()`. This is recommended, although is easiest to do for new projects, designed from the start to use doctest.
- Define functions named `_regrtest_topic` that consist of single docstrings, containing test cases for the named topics. These functions can be included in the same file as the module, or separated out into a separate test file.
- Define a `__test__` dictionary mapping from regression test topics to docstrings containing test cases.

When you have placed your tests in a module, the module can itself be the test runner. When a test fails, you can arrange for your test runner to re-run only the failing doctest while you debug the problem. Here is a minimal example of such a test runner:

```
if __name__ == '__main__':
    import doctest
    flags = doctest.REPORT_NDIFF|doctest.FAIL_FAST
    if len(sys.argv) > 1:
        name = sys.argv[1]
        if name in globals():
            obj = globals()[name]
        else:
            obj = __test__[name]
        doctest.run_docstring_examples(obj, globals(), name=name
                                       optionflags=flags)
    else:
        fail, total = doctest.testmod(optionflags=flags)
        print(f"{fail} failures out of {total} tests")
```

Notas de rodapé

[1]

Examples containing both expected output and an exception are not supported. Trying to guess where one ends and the other begins is too error-prone, and that also makes for a confusing test.

`unittest` — Framework de testes unitários

Código-fonte: Lib/unittest/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/unittest/__init__.py]

(Caso já estejas familiarizado com os conceitos básicos de testes, poderás querer ignorar a lista de métodos assertivos.)

The **`unittest`** unit testing framework was originally inspired by JUnit and has a similar flavor as major unit testing frameworks in other languages. It supports test automation, sharing of setup and shutdown code for tests, aggregation of tests into collections, and independence of the tests from the reporting framework.

To achieve this, **`unittest`** supports some important concepts in an object-oriented way:

definição de contexto de teste
: Uma *definição de contexto de teste* representa a preparação necessária pra performar um ou mais testes, além de quaisquer ações de limpeza relacionadas. Isso pode envolver, por exemplo, criar bancos de dados proxy ou temporários, diretórios ou iniciar um processo de servidor.

caso de teste
: A *test case* is the individual unit of testing. It checks for a specific response to a particular set of inputs. **`unittest`** provides a base class, **`TestCase`**, which may be used to create new test cases.

Suíte de Testes
: Uma *test suite* é uma coleção de casos de teste, conjuntos de teste ou ambos. O mesmo é usado para agregar testes que devem ser executados juntos.

test runner

Um *test runner* é um componente que orquestra a execução de testes e fornece o resultado para o usuário. O runner pode usar uma interface gráfica, uma interface textual ou retornar um valor especial para indicar os resultados da execução dos testes.

Ver também

Módulo `doctest`

Outro módulo de suporte a testes com um sabor muito diferente.

Simple Smalltalk Testing: With Patterns
[https://web.archive.org/web/20150315073817/http://www.xprogramming.com/testfram.htm]

Kent Beck's original paper on testing frameworks using the pattern shared by **`unittest`**.

pytest **[https://docs.pytest.org/]**

É um framework externo do unittest com uma sintaxe mais leve para escrever testes. Por exemplo, `assert func(10) == 42`.

The Python Testing Tools Taxonomy
[https://wiki.python.org/moin/PythonTestingToolsTaxonomy]

Uma extensa lista de ferramentas para testar código Python, incluindo estruturas de teste funcionais e bibliotecas de objetos simulados.

Testing in Python Mailing List **[http://lists.idyll.org/listinfo/testing-in-python]**

Um grupo de interesse especial para discussão de testes e ferramentas de teste, em Python.

The script `Tools/unittestgui/unittestgui.py` in the Python source distribution is a GUI tool for test discovery and execution. This is intended largely for ease of use for those new to unit testing. For production environments it is recommended that tests be driven by a continuous integration system such as Buildbot [https://buildbot.net/], Jenkins [https://www.jenkins.io/], GitHub Actions [https://github.com/features/actions], or AppVeyor [https://www.appveyor.com/].

Exemplo Básico

The **`unittest`** module provides a rich set of tools for constructing and running tests. This section demonstrates that a small subset of the tools suffice to meet the needs of most users.

Aqui temos um simples Script para testar três métodos de String:

```
import unittest

class TestStringMethods(unittest.TestCase):

    def test_upper(self):
        self.assertEqual('foo'.upper(), 'FOO')

    def test_isupper(self):
        self.assertTrue('FOO'.isupper())
        self.assertFalse('Foo'.isupper())

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', 'world'])
        # check that s.split fails when the separator is not a s
        with self.assertRaises(TypeError):
            s.split(2)

if __name__ == '__main__':
    unittest.main()
```

A test case is created by subclassing **`unittest.TestCase`**. The three individual tests are defined with methods whose names start with the letters `test`. This naming convention informs the test runner about which methods represent tests.

O cerne de cada teste é a invocação de um método **`assertEqual()`** para verificar se há um resultado esperado; **`assertTrue()`** ou **`assertFalse()`** para verificar uma condição; ou **`assertRaises()`** para verificar se uma exceção específica será levantada. Esses métodos são usados ao invés de

utilizar a expressão **`assert`** para que o runner de teste possa acumular todos os resultados do teste e produzir um relatório.

Os métodos **`setUp()`** e **`tearDown()`** permitem que você defina instruções que serão executadas antes e depois de cada método de teste. Eles são abordados em mais detalhes na seção Organizando código teste.

O bloco final mostra uma maneira simples de executar os testes. A função **`unittest.main()`** fornece uma interface de linha de comando para o Script de teste. Quando executado a partir da linha de comando, o Script acima produz uma saída que se parece com isso:

```
...
----------------------------------------------------------------------
Ran 3 tests in 0.000s

OK
```

Passando a opção `-v` para o nosso Script de teste instruirá a função **`unittest.main()`** a habilitar um nível mais alto de verbosidade e produzirá a seguinte saída:

```
test_isupper (__main__.TestStringMethods.test_isupper) ... ok
test_split (__main__.TestStringMethods.test_split) ... ok
test_upper (__main__.TestStringMethods.test_upper) ... ok

----------------------------------------------------------------------
Ran 3 tests in 0.001s

OK
```

The above examples show the most commonly used **`unittest`** features which are sufficient to meet many everyday testing needs. The remainder of the documentation explores the full feature set from first principles.

Alterado na versão 3.11: The behavior of returning a value from a test method (other than the default `None` value), is now deprecated.

Interface de Linha de Comando

O módulo unittest pode ser usado diretamente da linha de comando para executar testes de módulos, classes ou mesmo testes de métodos individuais:

```
python -m unittest test_module1 test_module2
python -m unittest test_module.TestClass
python -m unittest test_module.TestClass.test_method
```

Você pode passar uma lista com qualquer combinação de nomes de módulos e nomes de classes ou métodos totalmente qualificados.

Os módulos de teste podem ser especificados por caminhos de arquivo também:

```
python -m unittest tests/test_something.py
```

Isso permite que você use o auto-completar do console/shell para especificar o módulo de teste. O arquivo especificado precisa ser "importável" como um módulo. O caminho é convertido para um nome de módulo ao remover a extensão *.py* e conversando os separadores do caminho em .. Se você quer executar um arquivo de teste que não é importável como um módulo, você deve executar o arquivo diretamente.

Você pode executar os testes com mais detalhes (maior verbosidade) ao usar o sinalizador -v:

```
python -m unittest -v test_module
```

Quando executado sem argumentos Test Discovery é iniciado:

```
python -m unittest
```

Para uma lista de todas as opções de linha de comando:

```
python -m unittest -h
```

Alterado na versão 3.2: Em versões mais antigas, era possível de executar apenas testes de métodos individuais e não de módulos ou classes.

Adicionado na versão 3.14: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

Opções de linha de comando

unittest suporta as seguintes opções de linha de comando:

-b, --buffer

Os streams da saída padrão e do erro padrão são carregados durante a execução do teste. A saída de um teste que passou é descartada. A saída geralmente é mostrada quando um teste falha e é adicionada às mensagens de falha.

-c, --catch

`Control-C` durante a execução do teste aguarda até que o teste corrente termine e a partir disso mostra todos os resultados até o momento. Um segundo `Control-C` invoca uma exceção `KeyboardInterrupt` comum.

Veja Signal Handling para as funções que provêm essa funcionalidade.

-f, --failfast

Parar a execução do teste no primeiro erro ou falha.

-k

Only run test methods and classes that match the pattern or substring. This option may be used multiple times, in which case all test cases that match any of the given patterns are included.

Padrões que contém um caractere curinga (`*`) são combinados com os testes pelo método `fnmatch.fnmatchcase()`; caso contrário, é utilizada uma combinação simples de substrings, diferenciando-se letras maiúsculas e minúsculas.

Padrões são combinados com o nome completo qualificado do método de teste no formato que ele é importado pelo carregador.

Por exemplo, `-k foo` combina com `foo_tests.SomeTest.test_something`, `bar_tests.SomeTest.test_foo`, mas não com `bar_tests.FooTest.test_something`.

--locals
Mostra variáveis locais no traceback.

--durations N
Show the N slowest test cases (N=0 for all).

Adicionado na versão 3.2: As opções de linha de comando `-b`, `-c` e `-f` foram adicionadas.

Adicionado na versão 3.5: A opção de linha de comando `--locals`.

Adicionado na versão 3.7: A opção de linha de comando `-k`.

Adicionado na versão 3.12: The command-line option `--durations`.

A linha de comando também pode ser usada para descobrir testes, para executar todos os testes de um projeto ou apenas de um subconjunto.

Test Discovery

Adicionado na versão 3.2.

Unittest supports simple test discovery. In order to be compatible with test discovery, all of the test files must be modules or packages importable from the top-level directory of the project (this means that their filenames must be valid identifiers).

O descobrimento de testes é implementado no **`TestLoader.discover()`**, mas também pode ser utilizado a partir da linha de comando. O comando básico para uso é:

```
cd project_directory
python -m unittest discover
```

Nota

Como um atalho, `python -m unittest` é o equivalente a `python -m unittest discover`. Se você deseja passar argumentos para a descoberta de testes, o subcomando `discover` deve ser usado explicitamente.

O sub-comando `discover` (descubra) tem as seguintes opções:

-v, --verbose
: Saída verbosa

-s, --start-directory directory
: Diretório no qual se inicia o descobrimento (`.` por padrão)

-p, --pattern pattern
: Padrão de texto para se descobrir os arquivos de teste (`test*.py` por padrão)

-t, --top-level-directory directory
: Diretório raiz do projeto (diretório de início por padrão)

As opções **`-s`**, **`-p`** e **`-t`** podem ser passadas como argumentos posicionais nessa ordem. As duas linhas de comando seguintes são equivalentes:

```
python -m unittest discover -s project_directory -p "*_test.py"
python -m unittest discover project_directory "*_test.py"
```

Além de aceitar caminhos, também é possível passar o nome de um pacote, como `myproject.subpackage.test`, como diretório de início. O nome do pacote que for passado será importado e sua localização no sistema de arquivos será utilizada como diretório de início.

Cuidado

O descobridor de testes importa os testes para carregá-los. Uma vez que o descobridor tiver encontrado todos os arquivos de teste a partir do diretório de início especificado, ele transforma os caminhos em nomes de pacotes para conseguir importá-los. Por exemplo, `foo/bar/baz.py` será importado como `foo.bar.baz`.

Se você possuir um pacote instalado globalmente e tentar executar o descobrimento em uma versão diferente deste mesmo pacote, a importação *pode* acontecer do lugar errado. Se isso acontecer, o descobridor de testes irá emitir um alerta e encerrar a execução.

Se você configurar o diretório de início como sendo um nome de pacote, não um caminho para um diretório, o descobridor irá presumir que qualquer local do qual ele importar é o local correto. Neste caso, nenhum alerta será emitido.

Módulos de testes e pacotes podem conter customizações no carregamento de testes utilizando load_tests protocol.

Alterado na versão 3.4: O descobrimento de testes possui suporte para pacotes de espaço de nomes.

Alterado na versão 3.11: Test discovery dropped the namespace packages support. It has been broken since Python 3.7. Start directory and its subdirectories containing tests must be regular package that have `__init__.py` file.

If the start directory is the dotted name of the package, the ancestor packages can be namespace packages.

Alterado na versão 3.14: Test discovery supports namespace package as start directory again. To avoid scanning directories unrelated to Python, tests are not searched in subdirectories that do not contain `__init__.py`.

Organizando código teste

The basic building blocks of unit testing are *test cases* — single scenarios that must be set up and checked for correctness. In **`unittest`**, test cases are represented by **`unittest.TestCase`** instances. To make your own test cases you must write subclasses of **`TestCase`** or use **`FunctionTestCase`**.

O código de teste em uma instância da classe **`TestCase`** deve ser completamente auto-contido, de maneira que ele possa ser executado isoladamente ou combinado, de forma arbitrária, com quaisquer outros casos de teste.

A mais simples subclasse de **`TestCase`** irá implementar um método de teste (i.e. um método cujo nome começa com `test`) para executar um teste:

```
import unittest

class DefaultWidgetSizeTestCase(unittest.TestCase):
    def test_default_widget_size(self):
        widget = Widget('The widget')
        self.assertEqual(widget.size(), (50, 50))
```

Note that in order to test something, we use one of the assert* methods provided by the **`TestCase`** base class. If the test fails, an exception will be raised with an explanatory message, and **`unittest`** will identify the test case as a *failure*. Any other exceptions will be treated as *errors*.

Os testes podem ser muitos, e as configurações podem ser repetitivas. Por sorte, temos como reaproveitar estas configurações implementando um método chamado **`setUp()`**, o qual será automaticamente chamado pelo framework de teste para cada teste único que executarmos:

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def test_default_widget_size(self):
        self.assertEqual(self.widget.size(), (50,50),
                         'incorrect default size')
```

```
    def test_widget_resize(self):
        self.widget.resize(100,150)
        self.assertEqual(self.widget.size(), (100,150),
                         'wrong size after resize')
```

Nota

A ordem de execução dos testes será feita com base na ordenação dos nomes dos métodos de teste respeitando os critérios da ordenação de strings embutida.

Se o método `setUp()` levantar uma exceção durante a sua execução, o framework irá considerar que o teste sofreu um erro e o método de teste não será executado.

De maneira similar, pode-se definir um método `tearDown()` que limpa o ambiente após a execução do método de teste:

```
import unittest

class WidgetTestCase(unittest.TestCase):
    def setUp(self):
        self.widget = Widget('The widget')

    def tearDown(self):
        self.widget.dispose()
```

Se o método `setUp()` for bem sucedido, o método `tearDown()` será executado, independente do resultado do método de teste.

Such a working environment for the testing code is called a *test fixture*. A new TestCase instance is created as a unique test fixture used to execute each individual test method. Thus `setUp()`, `tearDown()`, and **`TestCase.__init__()`** will be called once per test.

It is recommended that you use TestCase implementations to group tests together according to the features they test. **`unittest`** provides a mechanism for this: the *test suite*, represented by **`unittest`**'s `TestSuite` class. In most

cases, calling `unittest.main()` will do the right thing and collect all the module's test cases for you and execute them.

Entretanto, caso você queira customizar a construção da sua suíte de testes, é possível fazê-la desta forma:

```
def suite():
    suite = unittest.TestSuite()
    suite.addTest(WidgetTestCase('test_default_widget_size'))
    suite.addTest(WidgetTestCase('test_widget_resize'))
    return suite

if __name__ == '__main__':
    runner = unittest.TextTestRunner()
    runner.run(suite())
```

Você pode colocar as definições dos casos de teste e suíte de testes em um mesmo módulo que contém o código a ser testado (tal como `widget.py`), mas existem várias vantagens ao colocar o código dos testes em um módulo separado, como `test_widget.py`:

- O módulo de teste pode ser executado de maneira isolada a partir da linha de comando.
- O código de teste pode ser mais facilmente separado do código a ser entregue.
- Há uma menor tentação para modificar o código de teste para que ele se adéque ao código testado sem que haja uma boa razão.
- O código de teste deve ser modificado muito menos frequentemente do que o código que ele testa.
- Código testado pode ser reformulado mais facilmente.
- Testes para módulos escritos em C devem ser, obrigatoriamente, colocados em módulos separados, então por que não manter a consistência?
- Se as estratégias de teste mudarem, não há a necessidade de mudar o código-fonte.

Reutilizando códigos de teste antigos

Some users will find that they have existing test code that they would like to run from **unittest**, without converting every old test function to a **TestCase** subclass.

For this reason, **unittest** provides a **FunctionTestCase** class. This subclass of **TestCase** can be used to wrap an existing test function. Set-up and tear-down functions can also be provided.

Dadas as seguintes funções de teste:

```
def testSomething():
    something = makeSomething()
    assert something.name is not None
    # ...
```

é possível criar uma instância de caso de teste, com métodos opcionais de configuração inicial e final (set-up e tear-down), como mostrado a seguir:

```
testcase = unittest.FunctionTestCase(testSomething,
                                     setUp=makeSomethingDB,
                                     tearDown=deleteSomethingDB)
```

Nota

Even though **FunctionTestCase** can be used to quickly convert an existing test base over to a **unittest**-based system, this approach is not recommended. Taking the time to set up proper **TestCase** subclasses will make future test refactorings infinitely easier.

In some cases, the existing tests may have been written using the **doctest** module. If so, **doctest** provides a **DocTestSuite** class that can automatically build **unittest.TestSuite** instances from the existing **doctest**-based tests.

Ignorando testes e falhas esperadas

Adicionado na versão 3.1.

Unittest suporta ignorar métodos de teste individuais e, até mesmo, classes de teste inteiras. Além disso, há suporte para a marcação de um teste como uma “falha esperada”, um teste que está incorreto e irá falhar, mas não deve ser considerado como uma falha no `TestResult`.

Skipping a test is simply a matter of using the `@skip` decorator or one of its conditional variants, calling `TestCase.skipTest()` within a `setUp()` or test method, or raising `SkipTest` directly.

Ignorar se parece basicamente com:

```
class MyTestCase(unittest.TestCase):

    @unittest.skip("demonstrating skipping")
    def test_nothing(self):
        self.fail("shouldn't happen")

    @unittest.skipIf(mylib.__version__ < (1, 3),
                     "not supported in this library version")
    def test_format(self):
        # Tests that work for only a certain version of the libr
        pass

    @unittest.skipUnless(sys.platform.startswith("win"), "requir
    def test_windows_support(self):
        # windows specific testing code
        pass

    def test_maybe_skipped(self):
        if not external_resource_available():
            self.skipTest("external resource not available")
        # test code that depends on the external resource
        pass
```

Esta é a saída da execução do exemplo acima em modo verboso:

```
test_format (__main__.MyTestCase.test_format) ... skipped 'not s
test_nothing (__main__.MyTestCase.test_nothing) ... skipped 'dem
test_maybe_skipped (__main__.MyTestCase.test_maybe_skipped) ...
test_windows_support (__main__.MyTestCase.test_windows_support)
```

```
----------------------------------------------------------------------
Ran 4 tests in 0.005s

OK (skipped=4)
```

Classes podem ser puladas assim como métodos:

```
@unittest.skip("showing class skipping")
class MySkippedTestCase(unittest.TestCase):
    def test_not_run(self):
        pass
```

`TestCase.setUp()` também pode ignorar o teste. Isso é útil quando um recurso que precisa ser configurado não está disponível.

Expected failures use the `@expectedFailure` decorator.

```
class ExpectedFailureTestCase(unittest.TestCase):
    @unittest.expectedFailure
    def test_fail(self):
        self.assertEqual(1, 0, "broken")
```

É simples ignorar com decoradores customizados, basta fazer um decorador que chama `skip()` no teste quando ele deve ser ignorado. Este decorador ignora o teste a não ser que o objeto fornecido tenha um determinado atributo:

```
def skipUnlessHasattr(obj, attr):
    if hasattr(obj, attr):
        return lambda func: func
    return unittest.skip("{!r} doesn't have {!r}".format(obj, at
```

Os decoradores e exceção seguintes ignoram testes e falhas esperadas:

@unittest.skip(*reason*)

Ignora incondicionalmente o teste decorado. *reason* deve descrever a razão pela qual o teste está sendo ignorado.

@unittest.skipIf(*condition*, *reason*)

Ignora o teste decorado se *condition* for verdadeiro.

@unittest.skipUnless(*condition*, *reason*)

Ignora o teste decorado a não ser que *condition* seja verdadeiro.

@unittest.expectedFailure

Marca o teste como uma falha ou erro esperado. Se o teste falhar ou ocorrerem erros na função de teste mesmo (ao invés de em um dos métodos *test fixture*), então ele será considerado como executado com sucesso. Se o teste passar, ele será considerado como uma falha.

exception unittest.SkipTest(*reason*)

Esta exceção é levantada para ignorar um teste.

Normalmente, você pode utilizar `TestCase.skipTest()` ou um dos decoradores para ignorar sem ter de levantar esta exceção diretamente.

Testes ignorados não terão seus métodos `setUp()` ou `tearDown()` executados. Classes ignoradas não terão seus métodos `setUpClass()` ou `tearDownClass()` executados. Módulos ignorados não terão seus métodos `setUpModule()` ou `tearDownModule()` executados.

Distinguindo iterações de teste utilizando subtestes

Adicionado na versão 3.4.

Quando existem pequenas diferenças entre os seus testes, por exemplo alguns parâmetros, unittest permite que você distinga-os dentro do corpo de um método de teste utilizando o gerenciador de contexto `subTest()`.

Por exemplo, o teste seguinte:

```
class NumbersTest(unittest.TestCase):

    def test_even(self):
        """
```

```
        Test that numbers between 0 and 5 are all even.
        """
        for i in range(0, 6):
            with self.subTest(i=i):
                self.assertEqual(i % 2, 0)
```

irá produzir a seguinte saída:

```
======================================================================
FAIL: test_even (__main__.NumbersTest.test_even) (i=1)
Test that numbers between 0 and 5 are all even.
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0

======================================================================
FAIL: test_even (__main__.NumbersTest.test_even) (i=3)
Test that numbers between 0 and 5 are all even.
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0

======================================================================
FAIL: test_even (__main__.NumbersTest.test_even) (i=5)
Test that numbers between 0 and 5 are all even.
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 11, in test_even
    self.assertEqual(i % 2, 0)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError: 1 != 0
```

Sem usar um subteste, a execução para depois da primeira falha e o erro será menos fácil de ser diagnosticado porque o valor de `i` não será mostrado:

```
======================================================================
FAIL: test_even (__main__.NumbersTest.test_even)
----------------------------------------------------------------------
Traceback (most recent call last):
  File "subtests.py", line 32, in test_even
    self.assertEqual(i % 2, 0)
AssertionError: 1 != 0
```

Classes e funções

This section describes in depth the API of **`unittest`**.

Casos de teste

class unittest.TestCase(*methodName='runTest'*)

Instances of the **`TestCase`** class represent the logical test units in the **`unittest`** universe. This class is intended to be used as a base class, with specific tests being implemented by concrete subclasses. This class implements the interface needed by the test runner to allow it to drive the tests, and methods that the test code can use to check for and report various kinds of failure.

Cada instância da classe **`TestCase`** irá executar um único método base: o método chamado *methodName*. Em muitos casos de uso da classe **`TestCase`**, você não precisará modificar o *methodName* ou reimplementar o método `runTest()` padrão.

Alterado na versão 3.2: **`TestCase`** pode ser instanciada com sucesso sem fornecer um *methodName*. Isso torna mais fácil experimentar com **`TestCase`** a partir de um interpretador interativo.

Instâncias **`TestCase`** possuem três grupos de métodos: um grupo utilizado para executar o teste, outro utilizado pela implementação do teste para checar as condições e reportar falhas, e alguns métodos de investigação para permitir coletar dados sobre o teste em si.

Os métodos do primeiro grupo (que executam os testes) são:

setUp()

Método chamado para preparar a definição de contexto de teste. Chamado imediatamente após chamar o método de teste; exceto pelo `AssertionError` ou `SkipTest`, qualquer exceção levantada por este método será considerada um erro além de uma simples falha de teste. A implementação padrão não faz nada.

tearDown()

Método chamado imediatamente após o método de teste ter sido chamado e o resultado registrado. Este método é chamado mesmo se o método de teste tiver levantado uma exceção, então a implementação em subclasses deve ser feita com cautela ao verificar o estado interno. Qualquer exceção além de `AssertionError` ou `SkipTest` levantada por este método será considerado um erro adicional, e não uma simples falha de teste (que incrementaria o número total de erros no relatório final de testes). Este método será executado apenas se o método `setUp()` for bem sucedido, independente do resultado do método de teste. A implementação padrão não faz nada.

setUpClass()

A class method called before tests in an individual class are run. `setUpClass` is called with the class as the only argument and must be decorated as a `@classmethod`:

```
@classmethod
def setUpClass(cls):
    ...
```

Veja Classes e Módulos de Definição de Contexto para mais detalhes.

Adicionado na versão 3.2.

tearDownClass()

A class method called after tests in an individual class have run. `tearDownClass` is called with the class as the only argument and

must be decorated as a `@classmethod`:

```
@classmethod
def tearDownClass(cls):
    ...
```

Veja Classes e Módulos de Definição de Contexto para mais detalhes.

Adicionado na versão 3.2.

run(*result=None*)

Executa o teste, registrando as informações no objeto resultado `TestResult`, passado como *result*. Se *result* for omitido, ou definido como `None`, um objeto resultado temporário é criado (chamando o método `defaultTestResult()`) e utilizado. O objeto resultado é retornado para quem chamou o método `run()`.

O mesmo efeito pode ser obtido ao chamar uma instância da classe `TestCase`.

Alterado na versão 3.3: Versões anteriores de `run` não retornavam o resultado. Nem chamando por uma instância.

skipTest(*reason*)

Ao se executar durante um método de teste ou `setUp()`, pula o teste em execução. Veja Ignorando testes e falhas esperadas para mais informações.

Adicionado na versão 3.1.

subTest(*msg=None*, ***params*)

Retorna um gerenciador de contexto que executa, como subteste, o bloco de código englobado. *msg* e *params* são valores opcionais e arbitrários que são mostrados sempre quando um subteste falha, permitindo identificá-los claramente.

Um caso de teste pode conter inúmeras declarações de subteste e elas podem ser aninhadas de forma arbitrária.

Veja Distinguindo iterações de teste utilizando subtestes para mais informações.

Adicionado na versão 3.4.

debug()

Executa o teste sem coletar o resultado. Permite propagar exceções levantadas pelo teste e pode ser utilizado para oferecer suporte aos testes sob um depurador.

A classe `TestCase` oferece diversos métodos de asserção para checar e reportar falhas. A tabela a seguir lista os métodos mais utilizados (veja as tabelas abaixo para mais métodos de asserção).

Método	Avalia se	Novo em
`assertEqual(a, b)`	`a == b`	
`assertNotEqual(a, b)`	`a != b`	
`assertTrue(x)`	`bool(x) is True`	
`assertFalse(x)`	`bool(x) is False`	
`assertIs(a, b)`	`a is b`	3.1
`assertIsNot(a, b)`	`a is not b`	3.1
`assertIsNone(x)`	`x is None`	3.1
`assertIsNotNone(x)`	`x is not None`	3.1
`assertIn(a, b)`	`a in b`	3.1

Método	Avalia se	Novo em
`assertNotIn(a, b)`	`a not in b`	3.1
`assertIsInstance(a, b)`	`isinstance(a, b)`	3.2
`assertNotIsInstance(a, b)`	`not isinstance(a, b)`	3.2
`assertIsSubclass(a, b)`	`issubclass(a, b)`	3.14
`assertNotIsSubclass(a, b)`	`not issubclass(a, b)`	3.14

Todos os métodos de asserção aceitam um argumento *msg* que, se especificado, é utilizado como a mensagem de erro em caso de falha (veja também `longMessage`). Note que o argumento nomeado *msg* pode ser passado para `assertRaises()`, `assertRaisesRegex()`, `assertWarns()` e `assertWarnsRegex()` apenas quando eles são utilizados como gerenciador de contexto.

assertEqual(*first*, *second*, *msg=None*)

Testa se *first* e *second* são iguais. Se o resultado da comparação não indicar igualdade, o teste irá falhar.

Além disso, se *first* e *second* são exatamente do mesmo tipo e são uma lista, tupla, dict, set, frozenset, str, ou qualquer outro tipo que é registrado na subclasse com `addTypeEqualityFunc()`, a função de igualdade específica do tipo é será chamada para gerar uma mensagem de erro mais útil (veja também lista de métodos específicos por tipo)

Alterado na versão 3.1: Adição da chamada automática da função específica por tipo.

Alterado na versão 3.2: método `assertMultiLineEqual()` adicionado como função de igualdade padrão para o tipo string.

assertNotEqual(*first*, *second*, *msg=None*)

Testa se *first* e *second* não são iguais. Se o resultado da comparação indicar igualdade, o teste irá falhar.

assertTrue(*expr*, *msg=None*)
assertFalse(*expr*, *msg=None*)

Testa se a expressão *expr* é verdadeira (ou falsa).

Note que isso é equivalente a `bool(expr) is True` e não a `expr is True` (use `assertIs(expr, True)` neste último caso). Este método também deve ser evitado quando outros métodos mais específicos estão disponíveis (e.g. `assertEqual(a, b)` no lugar de `assertTrue(a == b)`), já que estes podem ter uma melhor mensagem de erro em caso de falha.

assertIs(*first*, *second*, *msg=None*)
assertIsNot(*first*, *second*, *msg=None*)

Testa que *first* e *second* são (ou não são) o mesmo objeto.

Adicionado na versão 3.1.

assertIsNone(*expr*, *msg=None*)
assertIsNotNone(*expr*, *msg=None*)

Testa se *expr* é (ou não é) `None`.

Adicionado na versão 3.1.

assertIn(*member*, *container*, *msg=None*)
assertNotIn(*member*, *container*, *msg=None*)

Testa se *member* está (ou não está) em *container*.

Adicionado na versão 3.1.

assertIsInstance(*obj*, *cls*, *msg=None*)
assertNotIsInstance(*obj*, *cls*, *msg=None*)

Testa se *obj* é (ou não é) uma instância de *cls* (que pode ser uma classe ou uma tupla de classes, como suportado pela função

`isinstance()`). Para checar pelo tipo exato, use `assertIs(type(obj), cls)`.

Adicionado na versão 3.2.

assertIsSubclass(*cls*, *superclass*, *msg=None*)
assertNotIsSubclass(*cls*, *superclass*, *msg=None*)

Test that *cls* is (or is not) a subclass of *superclass* (which can be a class or a tuple of classes, as supported by `issubclass()`). To check for the exact type, use `assertIs(cls, superclass)`.

Adicionado na versão 3.14.

Também é possível checar a produção de exceções, avisos e mensagens de log usando os seguintes métodos:

Método	Avalia se	Novo em
`assertRaises(exc, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *exc*	
`assertRaisesRegex(exc, r, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *exc* e a mensagem casa com a expressão regular *r*	3.1
`assertWarns(warn, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *warn*	3.2
`assertWarnsRegex(warn, r, fun, *args, **kwds)`	`fun(*args, **kwds)` levanta *warn* e a mensagem casa com a expressão regular *r*	3.2

Método	Avalia se	Novo em
`assertLogs(logger, level)`	O bloco `with` registra logs no *logger* com pelo menos um nível *level*	Módulos para processamento de XML
`assertNoLogs(logger, level)`	O bloco `with` não loga em *logger* com *level* mínimo	3.10

assertRaises(*exception*, *callable*, **args*, ***kwds*)
assertRaises(*exception*, ***, *msg=None*)

Testa se uma exceção é levantada quando *callable* é chamado com quaisquer argumentos nomeados ou posicionais que também são passados para `assertRaises()`. O teste passa se *exception* for levantada e falha se outra exceção ou nenhuma exceção for levantada.

Se somente os argumentos *exception* e, possivelmente, *msg* forem passados, retorna um gerenciador de contexto para que o código sob teste possa ser escrito de forma embutida ao invés de ser definido como uma função:

```
with self.assertRaises(SomeException):
    do_something()
```

Quando utilizado como gerenciador de contexto, `assertRaises()` aceita o argumento nomeado adicional *msg*.

The context manager will store the caught exception object in its **`exception`** attribute. This can be useful if the intention is to perform additional checks on the exception raised:

```
with self.assertRaises(SomeException) as cm:
    do_something()
```

```
the_exception = cm.exception
self.assertEqual(the_exception.error_code, 3)
```

Alterado na versão 3.1: Adicionada a possibilidade de se utilizar **`assertRaises()`** como gerenciador de contexto.

Alterado na versão 3.2: Added the **`exception`** attribute.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

assertRaisesRegex(*exception*, *regex*, *callable*, **args*, ***kwds*)
assertRaisesRegex(*exception*, *regex*, ***, *msg=None*)

Similar ao **`assertRaises()`** mas também testa se *regex* casa com a representação em string da exceção levantada. *regex* pode ser um objeto expressão regular ou uma string contendo uma expressão regular compatível com o uso pela função **`re.search()`**. Exemplos:

```
self.assertRaisesRegex(ValueError, "invalid literal for.*
                       int, 'XYZ')
```

ou:

```
with self.assertRaisesRegex(ValueError, 'literal'):
   int('XYZ')
```

Adicionado na versão 3.1: Adicionado com o nome `assertRaisesRegexp`.

Alterado na versão 3.2: Renomeado para **`assertRaisesRegex()`**.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

assertWarns(*warning*, *callable*, **args*, ***kwds*)
assertWarns(*warning*, ***, *msg=None*)

Testa se o aviso *warning* é acionado quando *callable* é chamado com quaisquer argumentos nomeados ou posicionais que também

são passados para **assertWarns()**. O teste passa se *warning* for acionado e falha se não for. Qualquer exceção é considerada como falha. Para capturar qualquer aviso presente em um grupo de avisos, uma tupla contendo as classes de aviso podem ser passadas como *warning*.

Se apenas os argumentos *warning* e, possivelmente, *msg* forem passados, retorna um gerenciador de contexto para que o código sob teste possa ser escrito de forma embutida ao invés de ser definido como uma função:

```
with self.assertWarns(SomeWarning):
    do_something()
```

Quando usado como gerenciador de contexto, **assertWarns()** aceita o argumento nomeado adicional *msg*.

The context manager will store the caught warning object in its **warning** attribute, and the source line which triggered the warnings in the **filename** and **lineno** attributes. This can be useful if the intention is to perform additional checks on the warning caught:

```
with self.assertWarns(SomeWarning) as cm:
    do_something()

self.assertIn('myfile.py', cm.filename)
self.assertEqual(320, cm.lineno)
```

The context managers can be nested to test that multiple different warnings are emitted:

```
with (self.assertWarns(SomeWarning),
      self.assertWarns(OtherWarning)):
    do_something()
```

Este método funciona independente dos filtros de aviso configurados no momento em que é chamado.

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

Alterado na versão 3.15: Warnings that do not match the specified category are no longer swallowed. Nested context managers are now supported.

assertWarnsRegex(*warning*, *regex*, *callable*, **args*, ***kwds*)
assertWarnsRegex(*warning*, *regex*, ***, *msg=None*)

Similar ao `assertWarns()` mas também testa se *regex* casa com a mensagem do aviso acionado. *regex* pode ser um objeto expressão regular ou uma string contendo uma expressão regular compatível para uso pela função `re.search()`. Exemplo:

```
self.assertWarnsRegex(DeprecationWarning,
                      r'legacy_function\(\) is deprecated
                      legacy_function, 'XYZ')
```

ou:

```
with self.assertWarnsRegex(RuntimeWarning, 'unsafe frobnic
    frobnicate('/etc/passwd')
```

The context managers can be nested to test that multiple different warnings are emitted:

```
with (self.assertWarns(SomeWarning, regex1),
      self.assertWarns(OtherWarning, regex2)):
    do_something()
```

Adicionado na versão 3.2.

Alterado na versão 3.3: Adicionado o argumento nomeado *msg* quando utilizado como gerenciador de contexto.

Alterado na versão 3.15: Warnings that do not match the specified category or regex are no longer swallowed. Nested context managers are now supported.

assertLogs(*logger=None*, *level=None*, *formatter=None*)

Um gerenciador de contexto para testar se pelo menos uma mensagem é inserida em *logger* ou em um de seus filhos, com pelo menos um nível *level*.

Se fornecido, *logger* deve ser um objeto `logging.Logger` ou uma `str` fornecendo o nome de um logger. O padrão é o logger root, o qual irá capturar todas as mensagens que não foram bloqueadas por um logger descendente não-propagante.

If given, *level* should be either a numeric logging level or its string equivalent (for example either `"ERROR"` or `logging.ERROR`). The default is `logging.INFO`.

If given, *formatter* should be a `logging.Formatter` object. The default is a formatter with format string `"%(levelname)s:%(name)s:%(message)s"`

O teste passa se pelo menos uma das mensagens emitidas dentro do bloco `with` casa com as condições dadas por *logger* e *level*, falhando caso contrário.

O objeto retornado pelo gerenciador de contexto é um registro auxiliar que mantém os rastros das mensagens de log capturadas de acordo com os critérios dados. Ele possui dois atributos:

records

Uma lista de objetos de log da classe `logging.LogRecord` que foram compatíveis com os critérios dados.

output

Uma lista de objetos da classe `str` com a saída formatada das mensagens de log compatíveis com os critérios dados.

Exemplo:

```
with self.assertLogs('foo', level='INFO') as cm:
    logging.getLogger('foo').info('first message')
```

```
    logging.getLogger('foo.bar').error('second message')
self.assertEqual(cm.output, ['INFO:foo:first message',
                             'ERROR:foo.bar:second message
```

Adicionado na versão 3.4.

Alterado na versão 3.15: Now accepts a *formatter* to control how messages are formatted.

assertNoLogs(*logger=None*, *level=None*)

Um gerenciador de contexto para testar que nenhuma mensagem é logada no *logger* ou um dos seus filhos, com pelo menos o *level* fornecido.

Se passado, *logger* deve ser um objeto da classe `logging.Logger` ou um objeto da classe `str` com o nome do registrador de logs. O padrão é o registrador de log raiz, que irá capturar todas as mensagens.

If given, *level* should be either a numeric logging level or its string equivalent (for example either `"ERROR"` or `logging.ERROR`). The default is `logging.INFO`.

Ao contrário de `assertLogs()`, nada será retornado pelo gerenciador de contexto.

Adicionado na versão 3.10.

Existem também outros métodos usados para executar verificações mais específicas, como:

Método	Avalia se	Novo em
`assertAlmostEqual(a, b)`	`round(a-b, 7) == 0`	

Método	Avalia se	Novo em
`assertNotAlmostEqual(a, b)`	`round(a-b, 7) != 0`	
`assertGreater(a, b)`	`a > b`	3.1
`assertGreaterEqual(a, b)`	`a >= b`	3.1
`assertLess(a, b)`	`a < b`	3.1
`assertLessEqual(a, b)`	`a <= b`	3.1
`assertRegex(s, r)`	`r.search(s)`	3.1
`assertNotRegex(s, r)`	`not r.search(s)`	3.2
`assertCountEqual(a, b)`	*a* contains the same elements as *b*, regardless of their order.	3.2
`assertStartsWith(a, b)`	`a.startswith(b)`	3.14
`assertNotStartsWith(a, b)`	`not a.startswith(b)`	3.14
`assertEndsWith(a, b)`	`a.endswith(b)`	3.14
`assertNotEndsWith(a, b)`	`not a.endswith(b)`	3.14
`assertHasAttr(a, b)`	`hasattr(a, b)`	3.14
`assertNotHasAttr(a, b)`	`not hasattr(a, b)`	3.14

assertAlmostEqual(*first*, *second*, *places=7*, *msg=None*, *delta=None*)

assertNotAlmostEqual(*first*, *second*, *places=7*, *msg=None*, *delta=None*)

Testa se *first* e *second* são (ou não são) aproximadamente iguais considerando a diferença entre eles, arredondando para o número de casas decimais dado (7 por padrão), e comparando a zero. Note que estes métodos arredondam os valores considerando o número de casas decimais (i.e. como a função `round()`) e não o número de algarismos significativos.

Se *delta* é fornecido no lugar de *places*, a diferença entre *first* e *second* deve ser menor ou igual a (ou maior que) *delta*.

Passar *delta* e *places* ao mesmo tempo levanta a exceção `TypeError`.

Alterado na versão 3.2: `assertAlmostEqual()` considera, automaticamente, objetos quase iguais que possuem a comparação de igualdade dada como verdadeira. `assertNotAlmostEqual()` falha automaticamente se os objetos possuem a comparação de igualdade dada como verdadeira. Adicionado o argumento nomeado *delta*.

assertGreater(*first*, *second*, *msg=None*)

assertGreaterEqual(*first*, *second*, *msg=None*)

assertLess(*first*, *second*, *msg=None*)

assertLessEqual(*first*, *second*, *msg=None*)

Testa que *first* é respectivamente >, >=, < ou <= que *second*, dependendo do nome do método. Se não for, o teste irá falhar:

```
>>> self.assertGreaterEqual(3, 4)
AssertionError: "3" unexpectedly not greater than or equal
```

Adicionado na versão 3.1.

assertRegex(*text*, *regex*, *msg=None*)

assertNotRegex(*text*, *regex*, *msg=None*)

Testa que uma procura por *regex* corresponde (ou não corresponde) a *text*. Em caso de falha, a mensagem de erro irá incluir o padrão e o

text (ou o padrão e a parte do *text* que inesperadamente correspondeu). *regex* pode ser um objeto de expressão regular ou uma string contendo uma expressão regular adequada para uso por `re.search()`.

Adicionado na versão 3.1: Adicionada abaixo do nome `assertRegexpMatches`.

Alterado na versão 3.2: O método `assertRegexpMatches()` foi renomeado para `assertRegex()`.

Adicionado na versão 3.2: **`assertNotRegex()`**.

assertCountEqual(*first*, *second*, *msg=None*)

Test that sequence *first* contains the same elements as *second*, regardless of their order. When they don't, an error message listing the differences between the sequences will be generated.

Elementos duplicados *não* são ignorados ao comparar *first* e *second*. É verificado se cada elemento tem a mesma contagem em ambas sequências. Equivalente a: `assertEqual(Counter(list(first)), Counter(list(second)))`, mas também funciona com sequências de objetos não hasheáveis.

Adicionado na versão 3.2.

assertStartsWith(*s*, *prefix*, *msg=None*)

assertNotStartsWith(*s*, *prefix*, *msg=None*)

Test that the Unicode or byte string *s* starts (or does not start) with a *prefix*. *prefix* can also be a tuple of strings to try.

Adicionado na versão 3.14.

assertEndsWith(*s*, *suffix*, *msg=None*)

assertNotEndsWith(*s*, *suffix*, *msg=None*)

Test that the Unicode or byte string *s* ends (or does not end) with a *suffix*. *suffix* can also be a tuple of strings to try.

Adicionado na versão 3.14.

assertHasAttr(*obj*, *name*, *msg=None*)

assertNotHasAttr(*obj*, *name*, *msg=None*)

Test that the object *obj* has (or has not) an attribute *name*.

Adicionado na versão 3.14.

The `assertEqual()` method dispatches the equality check for objects of the same type to different type-specific methods. These methods are already implemented for most of the built-in types, but it's also possible to register new methods using `addTypeEqualityFunc()`:

addTypeEqualityFunc(*typeobj*, *function*)

Registers a type-specific method called by `assertEqual()` to check if two objects of exactly the same *typeobj* (not subclasses) compare equal. *function* must take two positional arguments and a third msg=None keyword argument just as **`assertEqual()`** does. It must raise `self.failureException(msg)` when inequality between the first two parameters is detected – possibly providing useful information and explaining the inequalities in details in the error message.

Adicionado na versão 3.1.

The list of type-specific methods automatically used by `assertEqual()` are summarized in the following table. Note that it's usually not necessary to invoke these methods directly.

Método	Usado para comparar	Novo em
`assertMultiLineEqual(a, b)`	strings	3.1

Método	Usado para comparar	Novo em
`assertSequenceEqual(a, b)`	sequências	3.1
`assertListEqual(a, b)`	listas	3.1
`assertTupleEqual(a, b)`	tuplas	3.1
`assertSetEqual(a, b)`	sets ou frozensets	3.1
`assertDictEqual(a, b)`	dicionários	3.1

assertMultiLineEqual(*first*, *second*, *msg=None*)

Test that the multiline string *first* is equal to the string *second*. When not equal a diff of the two strings highlighting the differences will be included in the error message. This method is used by default when comparing strings with `assertEqual()`.

Adicionado na versão 3.1.

assertSequenceEqual(*first*, *second*, *msg=None*, *seq_type=None*)

Tests that two sequences are equal. If a *seq_type* is supplied, both *first* and *second* must be instances of *seq_type* or a failure will be raised. If the sequences are different an error message is constructed that shows the difference between the two.

This method is not called directly by `assertEqual()`, but it's used to implement `assertListEqual()` and `assertTupleEqual()`.

Adicionado na versão 3.1.

assertListEqual(*first*, *second*, *msg=None*)

assertTupleEqual(*first*, *second*, *msg=None*)

Tests that two lists or tuples are equal. If not, an error message is constructed that shows only the differences between the two. An error is also raised if either of the parameters are of the wrong type.

These methods are used by default when comparing lists or tuples with **`assertEqual()`**.

Adicionado na versão 3.1.

assertSetEqual(*first*, *second*, *msg=None*)

Tests that two sets are equal. If not, an error message is constructed that lists the differences between the sets. This method is used by default when comparing sets or frozensets with **`assertEqual()`**.

Fails if either of *first* or *second* does not have a **`difference()`** method.

Adicionado na versão 3.1.

assertDictEqual(*first*, *second*, *msg=None*)

Test that two dictionaries are equal. If not, an error message is constructed that shows the differences in the dictionaries. This method will be used by default to compare dictionaries in calls to **`assertEqual()`**.

Adicionado na versão 3.1.

Finally the **`TestCase`** provides the following methods and attributes:

fail(*msg=None*)

Signals a test failure unconditionally, with *msg* or `None` for the error message.

failureException

This class attribute gives the exception raised by the test method. If a test framework needs to use a specialized exception, possibly to carry additional information, it must subclass this exception in order to “play fair” with the framework. The initial value of this attribute is **`AssertionError`**.

longMessage

This class attribute determines what happens when a custom failure message is passed as the msg argument to an assertXYY call that fails. `True` is the default value. In this case, the custom message is appended to the end of the standard failure message. When set to `False`, the custom message replaces the standard message.

The class setting can be overridden in individual test methods by assigning an instance attribute, self.longMessage, to `True` or `False` before calling the assert methods.

The class setting gets reset before each test call.

Adicionado na versão 3.1.

maxDiff

This attribute controls the maximum length of diffs output by assert methods that report diffs on failure. It defaults to 80*8 characters. Assert methods affected by this attribute are **`assertSequenceEqual()`** (including all the sequence comparison methods that delegate to it), **`assertDictEqual()`** and **`assertMultiLineEqual()`**.

Setting `maxDiff` to `None` means that there is no maximum length of diffs.

Adicionado na versão 3.2.

Frameworks de teste podem usar os seguintes métodos para coletar informações sobre o teste:

countTestCases()

Retorna o número de testes representados por esse objeto de teste. Para instâncias **`TestCase`** será sempre `1`.

defaultTestResult()

Return an instance of the test result class that should be used for this test case class (if no other result instance is provided to the **`run()`** method).

For **`TestCase`** instances, this will always be an instance of **`TestResult`**; subclasses of **`TestCase`** should override this as necessary.

id()

Retorna uma string identificando o caso de teste específico. Geralmente é o nome completo do método do teste, incluindo o módulo e o nome da classe.

shortDescription()

Retorna uma descrição do teste ou `None` se nenhuma descrição for fornecida. A implementação padrão desse método retorna a primeira linha da docstring do método do teste, se disponível, ou `None`.

Alterado na versão 3.1: In 3.1 this was changed to add the test name to the short description even in the presence of a docstring. This caused compatibility issues with unittest extensions and adding the test name was moved to the **`TextTestResult`** in Python 3.2.

addCleanup(*function*, /, **args*, ***kwargs*)

Add a function to be called after **`tearDown()`** to cleanup resources used during the test. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into **`addCleanup()`** when they are added.

If **`setUp()`** fails, meaning that **`tearDown()`** is not called, then any cleanup functions added will still be called.

Adicionado na versão 3.1.

enterContext(*cm*)

Enter the supplied context manager. If successful, also add its **`__exit__()`** method as a cleanup function by **`addCleanup()`** and return the result of the **`__enter__()`** method.

Adicionado na versão 3.11.

doCleanups()

This method is called unconditionally after `tearDown()`, or after `setUp()` if `setUp()` raises an exception.

It is responsible for calling all the cleanup functions added by `addCleanup()`. If you need cleanup functions to be called *prior* to `tearDown()` then you can call `doCleanups()` yourself.

`doCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time.

Adicionado na versão 3.1.

classmethod addClassCleanup(*function*, /, **args*, ***kwargs*)

Add a function to be called after `tearDownClass()` to cleanup resources used during the test class. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into `addClassCleanup()` when they are added.

If `setUpClass()` fails, meaning that `tearDownClass()` is not called, then any cleanup functions added will still be called.

Adicionado na versão 3.8.

classmethod enterClassContext(*cm*)

Enter the supplied context manager. If successful, also add its `__exit__()` method as a cleanup function by `addClassCleanup()` and return the result of the `__enter__()` method.

Adicionado na versão 3.11.

classmethod doClassCleanups()

This method is called unconditionally after `tearDownClass()`, or after `setUpClass()` if `setUpClass()` raises an exception.

It is responsible for calling all the cleanup functions added by `addClassCleanup()`. If you need cleanup functions to be called *prior* to `tearDownClass()` then you can call `doClassCleanups()` yourself.

`doClassCleanups()` pops methods off the stack of cleanup functions one at a time, so it can be called at any time.

Adicionado na versão 3.8.

class unittest.IsolatedAsyncioTestCase(*methodName='runTest'*)

This class provides an API similar to `TestCase` and also accepts coroutines as test functions.

Adicionado na versão 3.8.

loop_factory

The *loop_factory* passed to `asyncio.Runner`. Override in subclasses with `asyncio.EventLoop` to avoid using the asyncio policy system.

Adicionado na versão 3.13.

async asyncSetUp()

Method called to prepare the test fixture. This is called after `TestCase.setUp()`. This is called immediately before calling the test method; other than `AssertionError` or `SkipTest`, any exception raised by this method will be considered an error rather than a test failure. The default implementation does nothing.

async asyncTearDown()

Method called immediately after the test method has been called and the result recorded. This is called before `tearDown()`. This is called even if the test method raised an exception, so the implementation in subclasses may need to be particularly careful about checking internal state. Any exception, other than `AssertionError` or `SkipTest`, raised by this method will be considered an additional error rather than a test failure (thus increasing the total number of

reported errors). This method will only be called if the **asyncSetUp()** succeeds, regardless of the outcome of the test method. The default implementation does nothing.

addAsyncCleanup(*function*, /, **args*, ***kwargs*)

This method accepts a coroutine that can be used as a cleanup function.

async enterAsyncContext(*cm*)

Enter the supplied asynchronous context manager. If successful, also add its **__aexit__()** method as a cleanup function by **addAsyncCleanup()** and return the result of the **__aenter__()** method.

Adicionado na versão 3.11.

run(*result=None*)

Sets up a new event loop to run the test, collecting the result into the **TestResult** object passed as *result*. If *result* is omitted or `None`, a temporary result object is created (by calling the **defaultTestResult()** method) and used. The result object is returned to **run()**'s caller. At the end of the test all the tasks in the event loop are cancelled.

An example illustrating the order:

```
from unittest import IsolatedAsyncioTestCase

events = []


class Test(IsolatedAsyncioTestCase):


    def setUp(self):
        events.append("setUp")

    async def asyncSetUp(self):
        self._async_connection = await AsyncConnection()
```

```
        events.append("asyncSetUp")

    async def test_response(self):
        events.append("test_response")
        response = await self._async_connection.get("https://
        self.assertEqual(response.status_code, 200)
        self.addAsyncCleanup(self.on_cleanup)

    def tearDown(self):
        events.append("tearDown")

    async def asyncTearDown(self):
        await self._async_connection.close()
        events.append("asyncTearDown")

    async def on_cleanup(self):
        events.append("cleanup")

if __name__ == "__main__":
    unittest.main()
```

After running the test, `events` would contain `["setUp", "asyncSetUp", "test_response", "asyncTearDown", "tearDown", "cleanup"]`.

class unittest.FunctionTestCase(*testFunc*, *setUp=None*, *tearDown=None*, *description=None*)

This class implements the portion of the **`TestCase`** interface which allows the test runner to drive the test, but does not provide the methods which test code can use to check and report errors. This is used to create test cases using legacy test code, allowing it to be integrated into a **`unittest`**-based test framework.

Grouping tests

class unittest.TestSuite(*tests=()*)

This class represents an aggregation of individual test cases and test suites. The class presents the interface needed by the test runner to

allow it to be run as any other test case. Running a **`TestSuite`** instance is the same as iterating over the suite, running each test individually.

If *tests* is given, it must be an iterable of individual test cases or other test suites that will be used to build the suite initially. Additional methods are provided to add test cases and suites to the collection later on.

`TestSuite` objects behave much like **`TestCase`** objects, except they do not actually implement a test. Instead, they are used to aggregate tests into groups of tests that should be run together. Some additional methods are available to add tests to **`TestSuite`** instances:

addTest(*test*)

Add a **`TestCase`** or **`TestSuite`** to the suite.

addTests(*tests*)

Add all the tests from an iterable of **`TestCase`** and **`TestSuite`** instances to this test suite.

This is equivalent to iterating over *tests*, calling **`addTest()`** for each element.

`TestSuite` shares the following methods with **`TestCase`**:

run(*result*)

Run the tests associated with this suite, collecting the result into the test result object passed as *result*. Note that unlike **`TestCase.run()`**, **`TestSuite.run()`** requires the result object to be passed in.

debug()

Run the tests associated with this suite without collecting the result. This allows exceptions raised by the test to be propagated to the caller and can be used to support running tests under a debugger.

countTestCases()

Return the number of tests represented by this test object, including all individual tests and sub-suites.

__iter__()

Tests grouped by a **`TestSuite`** are always accessed by iteration. Subclasses can lazily provide tests by overriding **`__iter__()`**. Note that this method may be called several times on a single suite (for example when counting tests or comparing for equality) so the tests returned by repeated iterations before **`TestSuite.run()`** must be the same for each call iteration. After **`TestSuite.run()`**, callers should not rely on the tests returned by this method unless the caller uses a subclass that overrides **`TestSuite._removeTestAtIndex()`** to preserve test references.

Alterado na versão 3.2: In earlier versions the **`TestSuite`** accessed tests directly rather than through iteration, so overriding **`__iter__()`** wasn't sufficient for providing tests.

Alterado na versão 3.4: In earlier versions the **`TestSuite`** held references to each **`TestCase`** after **`TestSuite.run()`**. Subclasses can restore that behavior by overriding **`TestSuite._removeTestAtIndex()`**.

In the typical usage of a **`TestSuite`** object, the **`run()`** method is invoked by a **`TestRunner`** rather than by the end-user test harness.

Carregando e executando testes

class unittest.TestLoader

The **`TestLoader`** class is used to create test suites from classes and modules. Normally, there is no need to create an instance of this class; the **`unittest`** module provides an instance that can be shared as **`unittest.defaultTestLoader`**. Using a subclass or instance, however, allows customization of some configurable properties.

Objetos da classe **`TestLoader`** possuem os seguintes atributos:

errors

A list of the non-fatal errors encountered while loading tests. Not reset by the loader at any point. Fatal errors are signalled by the relevant method raising an exception to the caller. Non-fatal errors are also indicated by a synthetic test that will raise the original error when run.

Adicionado na versão 3.5.

Objetos da classe `TestLoader` possuem os seguintes métodos:

loadTestsFromTestCase(*testCaseClass*)

Return a suite of all test cases contained in the `TestCase`-derived `testCaseClass`.

A test case instance is created for each method named by `getTestCaseNames()`. By default these are the method names beginning with `test`. If `getTestCaseNames()` returns no methods, but the `runTest()` method is implemented, a single test case is created for that method instead.

loadTestsFromModule(*module*, *, *pattern=None*)

Return a suite of all test cases contained in the given module. This method searches *module* for classes derived from `TestCase` and creates an instance of the class for each test method defined for the class.

Nota

While using a hierarchy of `TestCase`-derived classes can be convenient in sharing fixtures and helper functions, defining test methods on base classes that are not intended to be instantiated directly does not play well with this method. Doing so, however, can be useful when the fixtures are different and defined in subclasses.

If a module provides a `load_tests` function it will be called to load the tests. This allows modules to customize test loading. This is the load_tests protocol. The *pattern* argument is passed as the third argument to `load_tests`.

Alterado na versão 3.2: Suporte para `load_tests` adicionado.

Alterado na versão 3.5: Support for a keyword-only argument *pattern* has been added.

Alterado na versão 3.12: The undocumented and unofficial *use_load_tests* parameter has been removed.

loadTestsFromName(*name*, *module=None*)

Return a suite of all test cases given a string specifier.

The specifier *name* is a "dotted name" that may resolve either to a module, a test case class, a test method within a test case class, a **`TestSuite`** instance, or a callable object which returns a **`TestCase`** or **`TestSuite`** instance. These checks are applied in the order listed here; that is, a method on a possible test case class will be picked up as "a test method within a test case class", rather than "a callable object".

For example, if you have a module **`SampleTests`** containing a **`TestCase`**-derived class **`SampleTestCase`** with three test methods (**`test_one()`**, **`test_two()`**, and **`test_three()`**), the specifier `'SampleTests.SampleTestCase'` would cause this method to return a suite which will run all three test methods. Using the specifier `'SampleTests.SampleTestCase.test_two'` would cause it to return a test suite which will run only the **`test_two()`** test method. The specifier can refer to modules and packages which have not been imported; they will be imported as a side-effect.

O método opcionalmente resolve o *nome* relativo ao *módulo* dado.

Alterado na versão 3.5: If an **`ImportError`** or **`AttributeError`** occurs while traversing *name* then a synthetic test that raises that

error when run will be returned. These errors are included in the errors accumulated by self.errors.

loadTestsFromNames(*names*, *module=None*)

Similar to **`loadTestsFromName()`**, but takes a sequence of names rather than a single name. The return value is a test suite which supports all the tests defined for each name.

getTestCaseNames(*testCaseClass*)

Return a sorted sequence of method names found within *testCaseClass*; this should be a subclass of **`TestCase`**.

discover(*start_dir*, *pattern='test*.py'*, *top_level_dir=None*)

Find all the test modules by recursing into subdirectories from the specified start directory, and return a TestSuite object containing them. Only test files that match *pattern* will be loaded. (Using shell style pattern matching.) Only module names that are importable (i.e. are valid Python identifiers) will be loaded.

All test modules must be importable from the top level of the project. If the start directory is not the top level directory then *top_level_dir* must be specified separately.

If importing a module fails, for example due to a syntax error, then this will be recorded as a single error and discovery will continue. If the import failure is due to **`SkipTest`** being raised, it will be recorded as a skip instead of an error.

If a package (a directory containing a file named `__init__.py`) is found, the package will be checked for a `load_tests` function. If this exists then it will be called `package.load_tests(loader, tests, pattern)`. Test discovery takes care to ensure that a package is only checked for tests once during an invocation, even if the load_tests function itself calls `loader.discover`.

If `load_tests` exists then discovery does *not* recurse into the package, `load_tests` is responsible for loading all tests in the

package.

The pattern is deliberately not stored as a loader attribute so that packages can continue discovery themselves.

top_level_dir is stored internally, and used as a default to any nested calls to `discover()`. That is, if a package's `load_tests` calls `loader.discover()`, it does not need to pass this argument.

start_dir can be a dotted module name as well as a directory.

Adicionado na versão 3.2.

Alterado na versão 3.4: Modules that raise **`SkipTest`** on import are recorded as skips, not errors.

start_dir can be a namespace packages.

Paths are sorted before being imported so that execution order is the same even if the underlying file system's ordering is not dependent on file name.

Alterado na versão 3.5: Found packages are now checked for `load_tests` regardless of whether their path matches *pattern*, because it is impossible for a package name to match the default pattern.

Alterado na versão 3.11: start_dir can not be a namespace packages. It has been broken since Python 3.7, and Python 3.11 officially removes it.

Alterado na versão 3.13: top_level_dir is only stored for the duration of *discover* call.

Alterado na versão 3.14: start_dir can once again be a namespace package.

The following attributes of a **`TestLoader`** can be configured either by subclassing or assignment on an instance:

testMethodPrefix

String giving the prefix of method names which will be interpreted as test methods. The default value is `'test'`.

This affects `getTestCaseNames()` and all the `loadTestsFrom*` methods.

sortTestMethodsUsing

Function to be used to compare method names when sorting them in `getTestCaseNames()` and all the `loadTestsFrom*` methods.

suiteClass

Callable object that constructs a test suite from a list of tests. No methods on the resulting object are needed. The default value is the `TestSuite` class.

This affects all the `loadTestsFrom*` methods.

testNamePatterns

List of Unix shell-style wildcard test name patterns that test methods have to match to be included in test suites (see `-k` option).

If this attribute is not `None` (the default), all test methods to be included in test suites must match one of the patterns in this list. Note that matches are always performed using `fnmatch.fnmatchcase()`, so unlike patterns passed to the `-k` option, simple substring patterns will have to be converted using `*` wildcards.

This affects all the `loadTestsFrom*` methods.

Adicionado na versão 3.7.

class unittest.TestResult

This class is used to compile information about which tests have succeeded and which have failed.

A **`TestResult`** object stores the results of a set of tests. The **`TestCase`** and **`TestSuite`** classes ensure that results are properly recorded; test authors do not need to worry about recording the outcome of tests.

Testing frameworks built on top of **`unittest`** may want access to the **`TestResult`** object generated by running a set of tests for reporting purposes; a **`TestResult`** instance is returned by the **`TestRunner.run()`** method for this purpose.

`TestResult` instances have the following attributes that will be of interest when inspecting the results of running a set of tests:

errors

A list containing 2-tuples of **`TestCase`** instances and strings holding formatted tracebacks. Each tuple represents a test which raised an unexpected exception.

failures

A list containing 2-tuples of **`TestCase`** instances and strings holding formatted tracebacks. Each tuple represents a test where a failure was explicitly signalled using the assert* methods.

skipped

A list containing 2-tuples of **`TestCase`** instances and strings holding the reason for skipping the test.

Adicionado na versão 3.1.

expectedFailures

A list containing 2-tuples of **`TestCase`** instances and strings holding formatted tracebacks. Each tuple represents an expected failure or error of the test case.

unexpectedSuccesses

A list containing **`TestCase`** instances that were marked as expected failures, but succeeded.

collectedDurations

A list containing 2-tuples of test case names and floats representing the elapsed time of each test which was run.

Adicionado na versão 3.12.

shouldStop

Set to `True` when the execution of tests should stop by **`stop()`**.

testsRun

The total number of tests run so far.

buffer

If set to true, `sys.stdout` and `sys.stderr` will be buffered in between **`startTest()`** and **`stopTest()`** being called. Collected output will only be echoed onto the real `sys.stdout` and `sys.stderr` if the test fails or errors. Any output is also attached to the failure / error message.

Adicionado na versão 3.2.

failfast

Se definido como true (verdadeiro) **`stop()`** será chamado na primeira falha ou erro, interrompendo a execução do teste.

Adicionado na versão 3.2.

tb_locals

If set to true then local variables will be shown in tracebacks.

Adicionado na versão 3.5.

wasSuccessful()

Return `True` if all tests run so far have passed, otherwise returns `False`.

Alterado na versão 3.4: Returns `False` if there were any **`unexpectedSuccesses`** from tests marked with the **`@expectedFailure`** decorator.

stop()

This method can be called to signal that the set of tests being run should be aborted by setting the **shouldStop** attribute to `True`. **`TestRunner`** objects should respect this flag and return without running any additional tests.

For example, this feature is used by the **`TextTestRunner`** class to stop the test framework when the user signals an interrupt from the keyboard. Interactive tools which provide **`TestRunner`** implementations can use this in a similar manner.

The following methods of the **`TestResult`** class are used to maintain the internal data structures, and may be extended in subclasses to support additional reporting requirements. This is particularly useful in building tools which support interactive reporting while tests are being run.

startTest(*test*)

Called when the test case *test* is about to be run.

stopTest(*test*)

Called after the test case *test* has been executed, regardless of the outcome.

startTestRun()

Called once before any tests are executed.

Adicionado na versão 3.1.

stopTestRun()

Called once after all tests are executed.

Adicionado na versão 3.1.

addError(*test*, *err*)

Called when the test case *test* raises an unexpected exception. *err* is a tuple of the form returned by **`sys.exc_info()`**: `(type, value,`

`traceback)`.

The default implementation appends a tuple `(test, formatted_err)` to the instance's **`errors`** attribute, where *formatted_err* is a formatted traceback derived from *err*.

addFailure(*test*, *err*)

Called when the test case *test* signals a failure. *err* is a tuple of the form returned by **`sys.exc_info()`**: `(type, value, traceback)`.

The default implementation appends a tuple `(test, formatted_err)` to the instance's **`failures`** attribute, where *formatted_err* is a formatted traceback derived from *err*.

addSuccess(*test*)

Called when the test case *test* succeeds.

The default implementation does nothing.

addSkip(*test*, *reason*)

Called when the test case *test* is skipped. *reason* is the reason the test gave for skipping.

The default implementation appends a tuple `(test, reason)` to the instance's **`skipped`** attribute.

addExpectedFailure(*test*, *err*)

Called when the test case *test* fails or errors, but was marked with the **`@expectedFailure`** decorator.

The default implementation appends a tuple `(test, formatted_err)` to the instance's **`expectedFailures`** attribute, where *formatted_err* is a formatted traceback derived from *err*.

addUnexpectedSuccess(*test*)

Called when the test case *test* was marked with the **`@expectedFailure`** decorator, but succeeded.

The default implementation appends the test to the instance's `unexpectedSuccesses` attribute.

addSubTest(*test*, *subtest*, *outcome*)

Called when a subtest finishes. *test* is the test case corresponding to the test method. *subtest* is a custom `TestCase` instance describing the subtest.

If *outcome* is `None`, the subtest succeeded. Otherwise, it failed with an exception where *outcome* is a tuple of the form returned by `sys.exc_info()`: `(type, value, traceback)`.

The default implementation does nothing when the outcome is a success, and records subtest failures as normal failures.

Adicionado na versão 3.4.

addDuration(*test*, *elapsed*)

Called when the test case finishes. *elapsed* is the time represented in seconds, and it includes the execution of cleanup functions.

Adicionado na versão 3.12.

class unittest.TextTestResult(*stream*, *descriptions*, *verbosity*, ***, *durations=None*)

A concrete implementation of `TestResult` used by the `TextTestRunner`. Subclasses should accept `**kwargs` to ensure compatibility as the interface changes.

Adicionado na versão 3.2.

Alterado na versão 3.12: Added the *durations* keyword parameter.

unittest.defaultTestLoader

Instance of the `TestLoader` class intended to be shared. If no customization of the `TestLoader` is needed, this instance can be used instead of repeatedly creating new instances.

class unittest.TextTestRunner(*stream=None*, *descriptions=True*, *verbosity=1*, *failfast=False*, *buffer=False*, *resultclass=None*, *warnings=None*, *, *tb_locals=False*, *durations=None*)

A basic test runner implementation that outputs results to a stream. If *stream* is `None`, the default, **`sys.stderr`** is used as the output stream. This class has a few configurable parameters, but is essentially very simple. Graphical applications which run test suites should provide alternate implementations. Such implementations should accept `**kwargs` as the interface to construct runners changes when features are added to unittest.

By default this runner shows **`DeprecationWarning`**, **`PendingDeprecationWarning`**, **`ResourceWarning`** and **`ImportWarning`** even if they are ignored by default. This behavior can be overridden using Python's **`-Wd`** or **`-Wa`** options (see Warning control) and leaving *warnings* to `None`.

Alterado na versão 3.2: Added the *warnings* parameter.

Alterado na versão 3.2: The default stream is set to **`sys.stderr`** at instantiation time rather than import time.

Alterado na versão 3.5: Added the *tb_locals* parameter.

Alterado na versão 3.12: Added the *durations* parameter.

_makeResult()

This method returns the instance of `TestResult` used by **`run()`**. It is not intended to be called directly, but can be overridden in subclasses to provide a custom `TestResult`.

`_makeResult()` instantiates the class or callable passed in the `TextTestRunner` constructor as the `resultclass` argument. It defaults to **`TextTestResult`** if no `resultclass` is provided. The result class is instantiated with the following arguments:

```
stream, descriptions, verbosity
```

run(*test*)

This method is the main public interface to the `TextTestRunner`. This method takes a **`TestSuite`** or **`TestCase`** instance. A **`TestResult`** is created by calling **`_makeResult()`** and the test(s) are run and the results printed to stdout.

unittest.main(*module='__main__', defaultTest=None, argv=None, testRunner=None, testLoader=unittest.defaultTestLoader, exit=True, verbosity=1, failfast=None, catchbreak=None, buffer=None, warnings=None*)

A command-line program that loads a set of tests from *module* and runs them; this is primarily for making test modules conveniently executable. The simplest use for this function is to include the following line at the end of a test script:

```
if __name__ == '__main__':
    unittest.main()
```

You can run tests with more detailed information by passing in the verbosity argument:

```
if __name__ == '__main__':
    unittest.main(verbosity=2)
```

The *defaultTest* argument is either the name of a single test or an iterable of test names to run if no test names are specified via *argv*. If not specified or `None` and no test names are provided via *argv*, all tests found in *module* are run.

O argumento *argv* pode ser uma lista de opções passada para o programa, com o primeiro elemento sendo o nome do programa. Se não for especificado ou for `None`, os valores de **`sys.argv`** são usados.

The *testRunner* argument can either be a test runner class or an already created instance of it. By default `main` calls **`sys.exit()`** with an exit code indicating success (0) or failure (1) of the tests run. An exit code of 5 indicates that no tests were run or skipped.

The *testLoader* argument has to be a **`TestLoader`** instance, and defaults to **`defaultTestLoader`**.

`main` supports being used from the interactive interpreter by passing in the argument `exit=False`. This displays the result on standard output without calling **`sys.exit()`**:

```
>>> from unittest import main
>>> main(module='test_module', exit=False)
```

The *failfast*, *catchbreak* and *buffer* parameters have the same effect as the same-name command-line options.

The *warnings* argument specifies the warning filter that should be used while running the tests. If it's not specified, it will remain `None` if a **`-W`** option is passed to **python** (see Warning control), otherwise it will be set to `'default'`.

Calling `main` returns an object with the `result` attribute that contains the result of the tests run as a **`unittest.TestResult`**.

Alterado na versão 3.1: The *exit* parameter was added.

Alterado na versão 3.2: The *verbosity*, *failfast*, *catchbreak*, *buffer* and *warnings* parameters were added.

Alterado na versão 3.4: The *defaultTest* parameter was changed to also accept an iterable of test names.

load_tests Protocol

Adicionado na versão 3.2.

Modules or packages can customize how tests are loaded from them during normal test runs or test discovery by implementing a function called `load_tests`.

If a test module defines `load_tests` it will be called by **`TestLoader.loadTestsFromModule()`** with the following arguments:

```
load_tests(loader, standard_tests, pattern)
```

where *pattern* is passed straight through from `loadTestsFromModule`. It defaults to `None`.

It should return a **`TestSuite`**.

loader is the instance of **`TestLoader`** doing the loading. *standard_tests* are the tests that would be loaded by default from the module. It is common for test modules to only want to add or remove tests from the standard set of tests. The third argument is used when loading packages as part of test discovery.

A typical `load_tests` function that loads tests from a specific set of **`TestCase`** classes may look like:

```
test_cases = (TestCase1, TestCase2, TestCase3)

def load_tests(loader, tests, pattern):
    suite = TestSuite()
    for test_class in test_cases:
        tests = loader.loadTestsFromTestCase(test_class)
        suite.addTests(tests)
    return suite
```

If discovery is started in a directory containing a package, either from the command line or by calling **`TestLoader.discover()`**, then the package `__init__.py` will be checked for `load_tests`. If that function does not exist, discovery will recurse into the package as though it were just another directory. Otherwise, discovery of the package's tests will be left up to `load_tests` which is called with the following arguments:

```
load_tests(loader, standard_tests, pattern)
```

This should return a **`TestSuite`** representing all the tests from the package. (`standard_tests` will only contain tests collected from `__init__.py`.)

Because the pattern is passed into `load_tests` the package is free to continue (and potentially modify) test discovery. A ‘do nothing’ `load_tests` function for a test package would look like:

```
def load_tests(loader, standard_tests, pattern):
    # top level directory cached on loader instance
    this_dir = os.path.dirname(__file__)
    package_tests = loader.discover(start_dir=this_dir, pattern=
    standard_tests.addTests(package_tests)
    return standard_tests
```

Alterado na versão 3.5: Discovery no longer checks package names for matching *pattern* due to the impossibility of package names matching the default pattern.

Classes e Módulos de Definição de Contexto

Class and module level fixtures are implemented in **`TestSuite`**. When the test suite encounters a test from a new class then **`tearDownClass()`** from the previous class (if there is one) is called, followed by **`setUpClass()`** from the new class.

Similarly if a test is from a different module from the previous test then `tearDownModule` from the previous module is run, followed by `setUpModule` from the new module.

Após executar todos os testes, haverá a execução final do `tearDownClass` e do `tearDownModule`.

Note that shared fixtures do not play well with [potential] features like test parallelization and they break test isolation. They should be used with care.

The default ordering of tests created by the unittest test loaders is to group all tests from the same modules and classes together. This will lead to `setUpClass` / `setUpModule` (etc) being called exactly once per class and module. If you randomize the order, so that tests from different modules

and classes are adjacent to each other, then these shared fixture functions may be called multiple times in a single test run.

Shared fixtures are not intended to work with suites with non-standard ordering. A `BaseTestSuite` still exists for frameworks that don't want to support shared fixtures.

If there are any exceptions raised during one of the shared fixture functions the test is reported as an error. Because there is no corresponding test instance an `_ErrorHolder` object (that has the same interface as a `TestCase`) is created to represent the error. If you are just using the standard unittest test runner then this detail doesn't matter, but if you are a framework author it may be relevant.

setUpClass and tearDownClass

These must be implemented as class methods:

```
import unittest

class Test(unittest.TestCase):
    @classmethod
    def setUpClass(cls):
        cls._connection = createExpensiveConnectionObject()

    @classmethod
    def tearDownClass(cls):
        cls._connection.destroy()
```

If you want the `setUpClass` and `tearDownClass` on base classes called then you must call up to them yourself. The implementations in `TestCase` are empty.

If an exception is raised during a `setUpClass` then the tests in the class are not run and the `tearDownClass` is not run. Skipped classes will not have `setUpClass` or `tearDownClass` run. If the exception is a `SkipTest` exception then the class will be reported as having been skipped instead of as an error.

setUpModule and tearDownModule

These should be implemented as functions:

```
def setUpModule():
    createConnection()

def tearDownModule():
    closeConnection()
```

If an exception is raised in a `setUpModule` then none of the tests in the module will be run and the `tearDownModule` will not be run. If the exception is a **`SkipTest`** exception then the module will be reported as having been skipped instead of as an error.

To add cleanup code that must be run even in the case of an exception, use `addModuleCleanup`:

unittest.addModuleCleanup(*function*, /, **args*, ***kwargs*)

Add a function to be called after **`tearDownModule()`** to cleanup resources used during the test class. Functions will be called in reverse order to the order they are added (LIFO). They are called with any arguments and keyword arguments passed into **`addModuleCleanup()`** when they are added.

If **`setUpModule()`** fails, meaning that **`tearDownModule()`** is not called, then any cleanup functions added will still be called.

Adicionado na versão 3.8.

unittest.enterModuleContext(*cm*)

Enter the supplied context manager. If successful, also add its **`__exit__()`** method as a cleanup function by **`addModuleCleanup()`** and return the result of the **`__enter__()`** method.

Adicionado na versão 3.11.

unittest.doModuleCleanups()

This function is called unconditionally after **tearDownModule()**, or after **setUpModule()** if **setUpModule()** raises an exception.

It is responsible for calling all the cleanup functions added by **addModuleCleanup()**. If you need cleanup functions to be called *prior* to **tearDownModule()** then you can call **doModuleCleanups()** yourself.

doModuleCleanups() pops methods off the stack of cleanup functions one at a time, so it can be called at any time.

Adicionado na versão 3.8.

Tratamento de sinal

Adicionado na versão 3.2.

The **-c/--catch** command-line option to unittest, along with the `catchbreak` parameter to **unittest.main()**, provide more friendly handling of control-C during a test run. With catch break behavior enabled control-C will allow the currently running test to complete, and the test run will then end and report all the results so far. A second control-c will raise a **KeyboardInterrupt** in the usual way.

The control-c handling signal handler attempts to remain compatible with code or tests that install their own **signal.SIGINT** handler. If the `unittest` handler is called but *isn't* the installed **signal.SIGINT** handler, i.e. it has been replaced by the system under test and delegated to, then it calls the default handler. This will normally be the expected behavior by code that replaces an installed handler and delegates to it. For individual tests that need `unittest` control-c handling disabled the **removeHandler()** decorator can be used.

There are a few utility functions for framework authors to enable control-c handling functionality within test frameworks.

unittest.installHandler()

Install the control-c handler. When a `signal.SIGINT` is received (usually in response to the user pressing control-c) all registered results have `stop()` called.

unittest.registerResult(*result*)

Register a `TestResult` object for control-c handling. Registering a result stores a weak reference to it, so it doesn't prevent the result from being garbage collected.

Registering a `TestResult` object has no side-effects if control-c handling is not enabled, so test frameworks can unconditionally register all results they create independently of whether or not handling is enabled.

unittest.removeResult(*result*)

Remove um resultado registrado. Dado que um resultado for removido, então `stop()` não será mais chamado no objeto resultado em resposta a um Ctrl+C

unittest.removeHandler(*function=None*)

When called without arguments this function removes the control-c handler if it has been installed. This function can also be used as a test decorator to temporarily remove the handler while the test is being executed:

```
@unittest.removeHandler
def test_signal_handling(self):
    ...
```

`unittest.mock` — mock object library

Adicionado na versão 3.3.

Código-fonte: Lib/unittest/mock.py
[https://github.com/python/cpython/tree/3.15/Lib/unittest/mock.py]

`unittest.mock` is a library for testing in Python. It allows you to replace parts of your system under test with mock objects and make assertions about how they have been used.

`unittest.mock` provides a core **`Mock`** class removing the need to create a host of stubs throughout your test suite. After performing an action, you can make assertions about which methods / attributes were used and arguments they were called with. You can also specify return values and set needed attributes in the normal way.

Adicionalmente, o mock fornece um decorador **`patch()`** que lida com os atributos do módulo de patch e do nível de classe no escopo de um teste, junto com **`sentinel`** para criar objetos únicos. Veja o guia rápido para alguns exemplos de como usar **`Mock`**, **`MagicMock`** e **`patch()`**.

Mock foi projetado para uso com **`unittest`** e é baseado no padrão 'ação -> asserção' em vez de 'gravar -> reproduzir' usado por muitas estruturas de simulação.

There is a backport of **`unittest.mock`** for earlier versions of Python, available as mock [https://pypi.org/project/mock/] on PyPI.

Guia Rápido

Os objetos `Mock` e `MagicMock` criam todos os atributos e métodos à medida que você os acessa e armazena detalhes de como eles foram usados. Você pode configurá-los, especificar valores de retorno ou limitar quais atributos estão disponíveis e, em seguida, fazer afirmações sobre como eles foram usados:

```
>>> from unittest.mock import MagicMock
>>> thing = ProductionClass()
>>> thing.method = MagicMock(return_value=3)
>>> thing.method(3, 4, 5, key='value')
3
>>> thing.method.assert_called_with(3, 4, 5, key='value')
```

`side_effect` allows you to perform side effects, including raising an exception when a mock is called:

```
>>> from unittest.mock import Mock
>>> mock = Mock(side_effect=KeyError('foo'))
>>> mock()
Traceback (most recent call last):
 ...
KeyError: 'foo'

>>> values = {'a': 1, 'b': 2, 'c': 3}
>>> def side_effect(arg):
...     return values[arg]
...
>>> mock.side_effect = side_effect
>>> mock('a'), mock('b'), mock('c')
(1, 2, 3)
>>> mock.side_effect = [5, 4, 3, 2, 1]
>>> mock(), mock(), mock()
(5, 4, 3)
```

O Mock tem muitas outras maneiras de configurá-lo e controlar seu comportamento. Por exemplo, o argumento *spec* configura o mock para obter sua especificação de outro objeto. Tentar acessar atributos ou métodos no mock que não existem no spec falhará com um `AttributeError`.

O gerenciador de contexto / decorador `patch()` facilita a simulação de classes ou objetos em um módulo em teste. O objeto que você especificar

será substituído por um mock (ou outro objeto) durante o teste e restaurado quando o teste terminar:

```
>>> from unittest.mock import patch
>>> @patch('module.ClassName2')
... @patch('module.ClassName1')
... def test(MockClass1, MockClass2):
...     module.ClassName1()
...     module.ClassName2()
...     assert MockClass1 is module.ClassName1
...     assert MockClass2 is module.ClassName2
...     assert MockClass1.called
...     assert MockClass2.called
...
>>> test()
```

Nota

Quando você aninha decoradores de patches, as simulações são passadas para a função decorada na mesma ordem em que foram aplicadas (a ordem normal *Python* em que os decoradores são aplicados). Isso significa de baixo para cima, portanto, no exemplo acima, a simulação para `module.ClassName1` é passada primeiro.

Com **`patch()`**, é importante que você faça o patch de objetos no espaço de nomes onde eles são procurados. Normalmente, isso é simples, mas para um guia rápido, leia onde fazer o patch.

Assim como um decorador **`patch()`** pode ser usado como um gerenciador de contexto em uma instrução with:

```
>>> with patch.object(ProductionClass, 'method', return_value=Nc
...     thing = ProductionClass()
...     thing.method(1, 2, 3)
...
>>> mock_method.assert_called_once_with(1, 2, 3)
```

Também existe `patch.dict()` para definir valores em um dicionário apenas durante um escopo e restaurar o dicionário ao seu estado original quando o teste termina:

```
>>> foo = {'key': 'value'}
>>> original = foo.copy()
>>> with patch.dict(foo, {'newkey': 'newvalue'}, clear=True):
...     assert foo == {'newkey': 'newvalue'}
...
>>> assert foo == original
```

Mock possui suporte a simulação de métodos mágicos de Python. A maneira mais fácil de usar métodos mágicos é com a classe `MagicMock`. Ele permite que você faça coisas como:

```
>>> mock = MagicMock()
>>> mock.__str__.return_value = 'foobarbaz'
>>> str(mock)
'foobarbaz'
>>> mock.__str__.assert_called_with()
```

Mock permite atribuir funções (ou outras instâncias do Mock) a métodos mágicos e elas serão chamadas apropriadamente. A classe `MagicMock` é apenas uma variante do Mock que possui todos os métodos mágicos pré-criados para você (bem, todos os úteis de qualquer maneira).

A seguir, é apresentado um exemplo do uso de métodos mágicos com a classe Mock comum:

```
>>> mock = Mock()
>>> mock.__str__ = Mock(return_value='wheeeeee')
>>> str(mock)
'wheeeeee'
```

Para garantir que os objetos mock em seus testes tenham a mesmo API que os objetos que eles estão substituindo, você pode usar especificação automática. A especificação automática pode ser feita por meio do argumento *autospec* para fazer patch ou pela função `create_autospec()`. A especificação automática cria objetos mock que têm os mesmos atributos e métodos que os objetos que estão substituindo, e qualquer funções e

métodos (incluindo construtores) têm a mesma assinatura de chamada que o objeto real.

Isso garante que seus mocks falharão da mesma forma que o código de produção se forem usados incorretamente:

```
>>> from unittest.mock import create_autospec
>>> def function(a, b, c):
...     pass
...
>>> mock_function = create_autospec(function, return_value='fish
>>> mock_function(1, 2, 3)
'fishy'
>>> mock_function.assert_called_once_with(1, 2, 3)
>>> mock_function('wrong arguments')
Traceback (most recent call last):
 ...
TypeError: missing a required argument: 'b'
```

`create_autospec()` também pode ser usada com classes, onde copia a assinatura do método `__init__`, e com objetos chamáveis onde copia a assinatura do método `__call__`.

A classe Mock

`Mock` é um objeto simulado flexível destinado a substituir o uso de *stubs* e dublês de teste em todo o seu código. Os mocks são chamáveis e cria atributos como novos mocks à medida que você os acessa [[1]]. Acessar o mesmo atributo sempre retorna o mesmo mock. Os mocks registram como você os utiliza, permitindo que você faça asserções sobre o que o seu código fez com eles.

`MagicMock` é uma subclasse de `Mock` com todos os métodos mágicos pré-criados e prontos para uso. Existem também variantes não chamáveis, úteis quando você está simulando objetos que não são chamáveis: `NonCallableMock` e `NonCallableMagicMock`

The `patch()` decorators makes it easy to temporarily replace classes in a particular module with a `Mock` object. By default `patch()` will create a `MagicMock` for you. You can specify an alternative class of `Mock` using the *new_callable* argument to `patch()`.

class unittest.mock.Mock(*spec=None*, *side_effect=None*, *return_value=DEFAULT*, *wraps=None*, *name=None*, *spec_set=None*, *unsafe=False*, ***kwargs*)

Create a new `Mock` object. `Mock` takes several optional arguments that specify the behaviour of the Mock object:

- *spec*: This can be either a list of strings or an existing object (a class or instance) that acts as the specification for the mock object. If you pass in an object then a list of strings is formed by calling dir on the object (excluding unsupported magic attributes and methods). Accessing any attribute not in this list will raise an `AttributeError`.

  If *spec* is an object (rather than a list of strings) then `__class__` returns the class of the spec object. This allows mocks to pass `isinstance()` tests.

- *spec_set*: A stricter variant of *spec*. If used, attempting to *set* or get an attribute on the mock that isn't on the object passed as *spec_set* will raise an `AttributeError`.

- *side_effect*: A function to be called whenever the Mock is called. See the `side_effect` attribute. Useful for raising exceptions or dynamically changing return values. The function is called with the same arguments as the mock, and unless it returns `DEFAULT`, the return value of this function is used as the return value.

  Alternatively *side_effect* can be an exception class or instance. In this case the exception will be raised when the mock is called.

  If *side_effect* is an iterable then each call to the mock will return the next value from the iterable.

A *side_effect* can be cleared by setting it to `None`.

- *return_value*: The value returned when the mock is called. By default this is a new Mock (created on first access). See the `return_value` attribute.

- *unsafe*: By default, accessing any attribute whose name starts with *assert*, *assret*, *asert*, *aseert* or *assrt* will raise an `AttributeError`. Passing `unsafe=True` will allow access to these attributes.

  Adicionado na versão 3.5.

- *wraps*: Item for the mock object to wrap. If *wraps* is not `None` then calling the Mock will pass the call through to the wrapped object (returning the real result). Attribute access on the mock will return a Mock object that wraps the corresponding attribute of the wrapped object (so attempting to access an attribute that doesn't exist will raise an `AttributeError`).

  If the mock has an explicit *return_value* set then calls are not passed to the wrapped object and the *return_value* is returned instead.

- *name*: If the mock has a name then it will be used in the repr of the mock. This can be useful for debugging. The name is propagated to child mocks.

Mocks can also be called with arbitrary keyword arguments. These will be used to set attributes on the mock after it is created. See the `configure_mock()` method for details.

assert_called()

Afirmar que o mock foi chamado pelo menos uma vez.

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called()
```

Adicionado na versão 3.6.

assert_called_once()

Afirma que o mock foi chamado exatamente uma vez.

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called_once()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called_once()
Traceback (most recent call last):
...
AssertionError: Expected 'method' to have been called once
Calls: [call(), call()].
```

Adicionado na versão 3.6.

assert_called_with(**args*, ***kwargs*)

This method is a convenient way of asserting that the last call has been made in a particular way:

```
>>> mock = Mock()
>>> mock.method(1, 2, 3, test='wow')
<Mock name='mock.method()' id='...'>
>>> mock.method.assert_called_with(1, 2, 3, test='wow')
```

assert_called_once_with(**args*, ***kwargs*)

Assert that the mock was called exactly once and that call was with the specified arguments.

```
>>> mock = Mock(return_value=None)
>>> mock('foo', bar='baz')
>>> mock.assert_called_once_with('foo', bar='baz')
>>> mock('other', bar='values')
>>> mock.assert_called_once_with('other', bar='values')
Traceback (most recent call last):
  ...
```

```
AssertionError: Expected 'mock' to be called once. Called
Calls: [call('foo', bar='baz'), call('other', bar='values
```

assert_any_call(**args*, ***kwargs*)

Assert the mock has been called with the specified arguments.

The assert passes if the mock has *ever* been called, unlike **assert_called_with()** and **assert_called_once_with()** that only pass if the call is the most recent one, and in the case of **assert_called_once_with()** it must also be the only call.

```
>>> mock = Mock(return_value=None)
>>> mock(1, 2, arg='thing')
>>> mock('some', 'thing', 'else')
>>> mock.assert_any_call(1, 2, arg='thing')
```

assert_has_calls(*calls*, *any_order=False*)

Assert the mock has been called with the specified calls. The **mock_calls** list is checked for the calls.

If *any_order* is false then the calls must be sequential. There can be extra calls before or after the specified calls.

If *any_order* is true then the calls can be in any order, but they must all appear in **mock_calls**.

```
>>> mock = Mock(return_value=None)
>>> mock(1)
>>> mock(2)
>>> mock(3)
>>> mock(4)
>>> calls = [call(2), call(3)]
>>> mock.assert_has_calls(calls)
>>> calls = [call(4), call(2), call(3)]
>>> mock.assert_has_calls(calls, any_order=True)
```

assert_not_called()

Afirma que o mock nunca foi chamado.

```
>>> m = Mock()
>>> m.hello.assert_not_called()
>>> obj = m.hello()
>>> m.hello.assert_not_called()
Traceback (most recent call last):
  ...
AssertionError: Expected 'hello' to not have been called.
Calls: [call()].
```

Adicionado na versão 3.5.

reset_mock(*, *return_value=False*, *side_effect=False*)

The reset_mock method resets all the call attributes on a mock object:

```
>>> mock = Mock(return_value=None)
>>> mock('hello')
>>> mock.called
True
>>> mock.reset_mock()
>>> mock.called
False
```

This can be useful where you want to make a series of assertions that reuse the same object.

return_value parameter when set to `True` resets **`return_value`**:

```
>>> mock = Mock(return_value=5)
>>> mock('hello')
5
>>> mock.reset_mock(return_value=True)
>>> mock('hello')
<Mock name='mock()' id='...'>
```

side_effect parameter when set to `True` resets **`side_effect`**:

```
>>> mock = Mock(side_effect=ValueError)
>>> mock('hello')
Traceback (most recent call last):
  ...
```

```
ValueError
>>> mock.reset_mock(side_effect=True)
>>> mock('hello')
<Mock name='mock()' id='...'>
```

Note that **`reset_mock()`** *doesn't* clear the **`return_value`**, **`side_effect`** or any child attributes you have set using normal assignment by default.

Child mocks are reset as well.

Alterado na versão 3.6: Foram adicionados dois argumentos somente-nomeado à função reset_mock.

mock_add_spec(*spec*, *spec_set=False*)

Add a spec to a mock. *spec* can either be an object or a list of strings. Only attributes on the *spec* can be fetched as attributes from the mock.

If *spec_set* is true then only attributes on the spec can be set.

attach_mock(*mock*, *attribute*)

Attach a mock as an attribute of this one, replacing its name and parent. Calls to the attached mock will be recorded in the **`method_calls`** and **`mock_calls`** attributes of this one.

configure_mock(***kwargs*)

Define atributos no mock por meio de argumentos nomeados.

Attributes plus return values and side effects can be set on child mocks using standard dot notation and unpacking a dictionary in the method call:

```
>>> mock = Mock()
>>> attrs = {'method.return_value': 3, 'other.side_effect
>>> mock.configure_mock(**attrs)
>>> mock.method()
3
>>> mock.other()
```

```
Traceback (most recent call last):
  ...
KeyError
```

The same thing can be achieved in the constructor call to mocks:

```
>>> attrs = {'method.return_value': 3, 'other.side_effect
>>> mock = Mock(some_attribute='eggs', **attrs)
>>> mock.some_attribute
'eggs'
>>> mock.method()
3
>>> mock.other()
Traceback (most recent call last):
  ...
KeyError
```

`configure_mock()` exists to make it easier to do configuration after the mock has been created.

__dir__()

`Mock` objects limit the results of `dir(some_mock)` to useful results. For mocks with a *spec* this includes all the permitted attributes for the mock.

See **`FILTER_DIR`** for what this filtering does, and how to switch it off.

_get_child_mock(**_kw_)

Create the child mocks for attributes and return value. By default child mocks will be the same type as the parent. Subclasses of Mock may want to override this to customize the way child mocks are made.

For non-callable mocks the callable variant will be used (rather than any custom subclass).

called

A boolean representing whether or not the mock object has been called:

```
>>> mock = Mock(return_value=None)
>>> mock.called
False
>>> mock()
>>> mock.called
True
```

call_count

An integer telling you how many times the mock object has been called:

```
>>> mock = Mock(return_value=None)
>>> mock.call_count
0
>>> mock()
>>> mock()
>>> mock.call_count
2
```

return_value

Set this to configure the value returned by calling the mock:

```
>>> mock = Mock()
>>> mock.return_value = 'fish'
>>> mock()
'fish'
```

The default return value is a mock object and you can configure it in the normal way:

```
>>> mock = Mock()
>>> mock.return_value.attribute = sentinel.Attribute
>>> mock.return_value()
<Mock name='mock()()' id='...'>
>>> mock.return_value.assert_called_with()
```

`return_value` também pode ser definido no construtor:

```
>>> mock = Mock(return_value=3)
>>> mock.return_value
3
>>> mock()
3
```

side_effect

This can either be a function to be called when the mock is called, an iterable or an exception (class or instance) to be raised.

If you pass in a function it will be called with same arguments as the mock and unless the function returns the **DEFAULT** singleton the call to the mock will then return whatever the function returns. If the function returns **DEFAULT** then the mock will return its normal value (from the **return_value**).

If you pass in an iterable, it is used to retrieve an iterator which must yield a value on every call. This value can either be an exception instance to be raised, or a value to be returned from the call to the mock (**DEFAULT** handling is identical to the function case).

An example of a mock that raises an exception (to test exception handling of an API):

```
>>> mock = Mock()
>>> mock.side_effect = Exception('Boom!')
>>> mock()
Traceback (most recent call last):
  ...
Exception: Boom!
```

Usando **side_effect** para retornar um sequência de valores:

```
>>> mock = Mock()
>>> mock.side_effect = [3, 2, 1]
>>> mock(), mock(), mock()
(3, 2, 1)
```

Usando um chamável:

```
>>> mock = Mock(return_value=3)
>>> def side_effect(*args, **kwargs):
...     return DEFAULT
...
>>> mock.side_effect = side_effect
>>> mock()
3
```

`side_effect` can be set in the constructor. Here's an example that adds one to the value the mock is called with and returns it:

```
>>> side_effect = lambda value: value + 1
>>> mock = Mock(side_effect=side_effect)
>>> mock(3)
4
>>> mock(-8)
-7
```

Configuração `side_effect` para `None` limpa isso:

```
>>> m = Mock(side_effect=KeyError, return_value=3)
>>> m()
Traceback (most recent call last):
 ...
KeyError
>>> m.side_effect = None
>>> m()
3
```

call_args

This is either `None` (if the mock hasn't been called), or the arguments that the mock was last called with. This will be in the form of a tuple: the first member, which can also be accessed through the `args` property, is any positional arguments the mock was called with (or an empty tuple) and the second member, which can also be accessed through the `kwargs` property, is any keyword arguments (or an empty dictionary).

```
>>> mock = Mock(return_value=None)
>>> print(mock.call_args)
None
>>> mock()
```

```
>>> mock.call_args
call()
>>> mock.call_args == ()
True
>>> mock(3, 4)
>>> mock.call_args
call(3, 4)
>>> mock.call_args == ((3, 4),)
True
>>> mock.call_args.args
(3, 4)
>>> mock.call_args.kwargs
{}
>>> mock(3, 4, 5, key='fish', next='w00t!')
>>> mock.call_args
call(3, 4, 5, key='fish', next='w00t!')
>>> mock.call_args.args
(3, 4, 5)
>>> mock.call_args.kwargs
{'key': 'fish', 'next': 'w00t!'}
```

call_args, along with members of the lists **call_args_list**, **method_calls** and **mock_calls** are **call** objects. These are tuples, so they can be unpacked to get at the individual arguments and make more complex assertions. See calls as tuples.

Alterado na versão 3.8: Adicionadas propriedades `args` e `kwargs`.

call_args_list

This is a list of all the calls made to the mock object in sequence (so the length of the list is the number of times it has been called). Before any calls have been made it is an empty list. The **call** object can be used for conveniently constructing lists of calls to compare with **call_args_list**.

```
>>> mock = Mock(return_value=None)
>>> mock()
>>> mock(3, 4)
>>> mock(key='fish', next='w00t!')
>>> mock.call_args_list
[call(), call(3, 4), call(key='fish', next='w00t!')]
```

```
>>> expected = [(), ((3, 4),), ({'key': 'fish', 'next': 'w
>>> mock.call_args_list == expected
True
```

Members of `call_args_list` are `call` objects. These can be unpacked as tuples to get at the individual arguments. See calls as tuples.

method_calls

As well as tracking calls to themselves, mocks also track calls to methods and attributes, and *their* methods and attributes:

```
>>> mock = Mock()
>>> mock.method()
<Mock name='mock.method()' id='...'>
>>> mock.property.method.attribute()
<Mock name='mock.property.method.attribute()' id='...'>
>>> mock.method_calls
[call.method(), call.property.method.attribute()]
```

Members of `method_calls` are `call` objects. These can be unpacked as tuples to get at the individual arguments. See calls as tuples.

mock_calls

`mock_calls` records *all* calls to the mock object, its methods, magic methods *and* return value mocks.

```
>>> mock = MagicMock()
>>> result = mock(1, 2, 3)
>>> mock.first(a=3)
<MagicMock name='mock.first()' id='...'>
>>> mock.second()
<MagicMock name='mock.second()' id='...'>
>>> int(mock)
1
>>> result(1)
<MagicMock name='mock()()' id='...'>
>>> expected = [call(1, 2, 3), call.first(a=3), call.secon
... call.__int__(), call()(1)]
```

```
>>> mock.mock_calls == expected
True
```

Members of **mock_calls** are **call** objects. These can be unpacked as tuples to get at the individual arguments. See calls as tuples.

Nota

The way **mock_calls** are recorded means that where nested calls are made, the parameters of ancestor calls are not recorded and so will always compare equal:

```
>>> mock = MagicMock()
>>> mock.top(a=3).bottom()
<MagicMock name='mock.top().bottom()' id='...'>
>>> mock.mock_calls
[call.top(a=3), call.top().bottom()]
>>> mock.mock_calls[-1] == call.top(a=-1).bottom()
True
```

__class__

Normally the **__class__** attribute of an object will return its type. For a mock object with a **spec**, **__class__** returns the spec class instead. This allows mock objects to pass **isinstance()** tests for the object they are replacing / masquerading as:

```
>>> mock = Mock(spec=3)
>>> isinstance(mock, int)
True
```

__class__ is assignable to, this allows a mock to pass an **isinstance()** check without forcing you to use a spec:

```
>>> mock = Mock()
>>> mock.__class__ = dict
>>> isinstance(mock, dict)
True
```

class unittest.mock.NonCallableMock(*spec=None*, *wraps=None*, *name=None*, *spec_set=None*, ***kwargs*)

A non-callable version of **Mock**. The constructor parameters have the same meaning of **Mock**, with the exception of *return_value* and *side_effect* which have no meaning on a non-callable mock.

Mock objects that use a class or an instance as a **spec** or **spec_set** are able to pass **isinstance()** tests:

```
>>> mock = Mock(spec=SomeClass)
>>> isinstance(mock, SomeClass)
True
>>> mock = Mock(spec_set=SomeClass())
>>> isinstance(mock, SomeClass)
True
```

The **Mock** classes have support for mocking magic methods. See magic methods for the full details.

The mock classes and the **patch()** decorators all take arbitrary keyword arguments for configuration. For the **patch()** decorators the keywords are passed to the constructor of the mock being created. The keyword arguments are for configuring attributes of the mock:

```
>>> m = MagicMock(attribute=3, other='fish')
>>> m.attribute
3
>>> m.other
'fish'
```

The return value and side effect of child mocks can be set in the same way, using dotted notation. As you can't use dotted names directly in a call you have to create a dictionary and unpack it using `**`:

```
>>> attrs = {'method.return_value': 3, 'other.side_effect': KeyE
>>> mock = Mock(some_attribute='eggs', **attrs)
>>> mock.some_attribute
'eggs'
>>> mock.method()
3
```

```
>>> mock.other()
Traceback (most recent call last):
  ...
KeyError
```

A callable mock which was created with a *spec* (or a *spec_set*) will introspect the specification object's signature when matching calls to the mock. Therefore, it can match the actual call's arguments regardless of whether they were passed positionally or by name:

```
>>> def f(a, b, c): pass
...
>>> mock = Mock(spec=f)
>>> mock(1, 2, c=3)
<Mock name='mock()' id='140161580456576'>
>>> mock.assert_called_with(1, 2, 3)
>>> mock.assert_called_with(a=1, b=2, c=3)
```

This applies to **`assert_called_with()`**, **`assert_called_once_with()`**, **`assert_has_calls()`** and **`assert_any_call()`**. When Especificação automática, it will also apply to method calls on the mock object.

Alterado na versão 3.4: Adicionada introspecção de assinatura em objetos mock especificados e auto-especificados.

class unittest.mock.PropertyMock(**args*, ***kwargs*)

A mock intended to be used as a **`property`**, or other descriptor, on a class. **`PropertyMock`** provides **`__get__()`** and **`__set__()`** methods so you can specify a return value when it is fetched.

Fetching a **`PropertyMock`** instance from an object calls the mock, with no args. Setting it calls the mock with the value being set.

```
>>> class Foo:
...     @property
...     def foo(self):
...         return 'something'
...     @foo.setter
...     def foo(self, value):
```

```
...         pass
...
>>> with patch('__main__.Foo.foo', new_callable=PropertyMock)
...     mock_foo.return_value = 'mockity-mock'
...     this_foo = Foo()
...     print(this_foo.foo)
...     this_foo.foo = 6
...
mockity-mock
>>> mock_foo.mock_calls
[call(), call(6)]
```

Because of the way mock attributes are stored you can't directly attach a `PropertyMock` to a mock object. Instead you can attach it to the mock type object:

```
>>> m = MagicMock()
>>> p = PropertyMock(return_value=3)
>>> type(m).foo = p
>>> m.foo
3
>>> p.assert_called_once_with()
```

Cuidado

If an `AttributeError` is raised by `PropertyMock`, it will be interpreted as a missing descriptor and `__getattr__()` will be called on the parent mock:

```
>>> m = MagicMock()
>>> no_attribute = PropertyMock(side_effect=AttributeError)
>>> type(m).my_property = no_attribute
>>> m.my_property
<MagicMock name='mock.my_property' id='140165240345424'>
```

See `__getattr__()` for details.

class unittest.mock.AsyncMock(*spec=None*, *side_effect=None*, *return_value=DEFAULT*, *wraps=None*, *name=None*, *spec_set=None*,

unsafe=False, ***kwargs*)

An asynchronous version of **MagicMock**. The **AsyncMock** object will behave so the object is recognized as an async function, and the result of a call is an awaitable.

```
>>> mock = AsyncMock()
>>> inspect.iscoroutinefunction(mock)
True
>>> inspect.isawaitable(mock())
True
```

The result of `mock()` is an async function which will have the outcome of `side_effect` or `return_value` after it has been awaited:

- if `side_effect` is a function, the async function will return the result of that function,
- if `side_effect` is an exception, the async function will raise the exception,
- if `side_effect` is an iterable, the async function will return the next value of the iterable, however, if the sequence of result is exhausted, `StopAsyncIteration` is raised immediately,
- if `side_effect` is not defined, the async function will return the value defined by `return_value`, hence, by default, the async function returns a new **AsyncMock** object.

Setting the *spec* of a **Mock** or **MagicMock** to an async function will result in a coroutine object being returned after calling.

```
>>> async def async_func(): pass
...
>>> mock = MagicMock(async_func)
>>> mock
<MagicMock spec='function' id='...'>
>>> mock()
<coroutine object AsyncMockMixin._mock_call at ...>
```

Setting the *spec* of a **Mock**, **MagicMock**, or **AsyncMock** to a class with asynchronous and synchronous functions will automatically detect the synchronous functions and set them as **MagicMock** (if the parent mock is

AsyncMock or **MagicMock**) or **Mock** (if the parent mock is **Mock**). All asynchronous functions will be **AsyncMock**.

```
>>> class ExampleClass:
...     def sync_foo():
...         pass
...     async def async_foo():
...         pass
...
>>> a_mock = AsyncMock(ExampleClass)
>>> a_mock.sync_foo
<MagicMock name='mock.sync_foo' id='...'>
>>> a_mock.async_foo
<AsyncMock name='mock.async_foo' id='...'>
>>> mock = Mock(ExampleClass)
>>> mock.sync_foo
<Mock name='mock.sync_foo' id='...'>
>>> mock.async_foo
<AsyncMock name='mock.async_foo' id='...'>
```

Adicionado na versão 3.8.

assert_awaited()

Assert that the mock was awaited at least once. Note that this is separate from the object having been called, the `await` keyword must be used:

```
>>> mock = AsyncMock()
>>> async def main(coroutine_mock):
...     await coroutine_mock
...
>>> coroutine_mock = mock()
>>> mock.called
True
>>> mock.assert_awaited()
Traceback (most recent call last):
...
AssertionError: Expected mock to have been awaited.
>>> asyncio.run(main(coroutine_mock))
>>> mock.assert_awaited()
```

assert_awaited_once()

Afirme que o mock foi aguardado exatamente uma vez.

```
>>> mock = AsyncMock()
>>> async def main():
...     await mock()
...
>>> asyncio.run(main())
>>> mock.assert_awaited_once()
>>> asyncio.run(main())
>>> mock.assert_awaited_once()
Traceback (most recent call last):
...
AssertionError: Expected mock to have been awaited once. A
```

assert_awaited_with(**args*, ***kwargs*)

Assert that the last await was with the specified arguments.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> asyncio.run(main('foo', bar='bar'))
>>> mock.assert_awaited_with('foo', bar='bar')
>>> mock.assert_awaited_with('other')
Traceback (most recent call last):
...
AssertionError: expected await not found.
Expected: mock('other')
Actual: mock('foo', bar='bar')
```

assert_awaited_once_with(**args*, ***kwargs*)

Assert that the mock was awaited exactly once and with the specified arguments.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> asyncio.run(main('foo', bar='bar'))
```

```
>>> mock.assert_awaited_once_with('foo', bar='bar')
>>> asyncio.run(main('foo', bar='bar'))
>>> mock.assert_awaited_once_with('foo', bar='bar')
Traceback (most recent call last):
...
AssertionError: Expected mock to have been awaited once. A
```

assert_any_await(**args*, ***kwargs*)

Assert the mock has ever been awaited with the specified arguments.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> asyncio.run(main('foo', bar='bar'))
>>> asyncio.run(main('hello'))
>>> mock.assert_any_await('foo', bar='bar')
>>> mock.assert_any_await('other')
Traceback (most recent call last):
...
AssertionError: mock('other') await not found
```

assert_has_awaits(*calls*, *any_order=False*)

Assert the mock has been awaited with the specified calls. The `await_args_list` list is checked for the awaits.

If *any_order* is false then the awaits must be sequential. There can be extra calls before or after the specified awaits.

If *any_order* is true then the awaits can be in any order, but they must all appear in `await_args_list`.

```
>>> mock = AsyncMock()
>>> async def main(*args, **kwargs):
...     await mock(*args, **kwargs)
...
>>> calls = [call("foo"), call("bar")]
>>> mock.assert_has_awaits(calls)
Traceback (most recent call last):
```

```
...
AssertionError: Awaits not found.
Expected: [call('foo'), call('bar')]
Actual: []
>>> asyncio.run(main('foo'))
>>> asyncio.run(main('bar'))
>>> mock.assert_has_awaits(calls)
```

assert_not_awaited()

Afirma que o mock nunca foi aguardado.

```
>>> mock = AsyncMock()
>>> mock.assert_not_awaited()
```

reset_mock(**args*, ***kwargs*)

See `Mock.reset_mock()`. Also sets `await_count` to 0, `await_args` to None, and clears the `await_args_list`.

await_count

An integer keeping track of how many times the mock object has been awaited.

```
>>> mock = AsyncMock()
>>> async def main():
...     await mock()
...
>>> asyncio.run(main())
>>> mock.await_count
1
>>> asyncio.run(main())
>>> mock.await_count
2
```

await_args

This is either `None` (if the mock hasn't been awaited), or the arguments that the mock was last awaited with. Functions the same as `Mock.call_args`.

```
>>> mock = AsyncMock()
>>> async def main(*args):
...     await mock(*args)
```

```
...
>>> mock.await_args
>>> asyncio.run(main('foo'))
>>> mock.await_args
call('foo')
>>> asyncio.run(main('bar'))
>>> mock.await_args
call('bar')
```

await_args_list

This is a list of all the awaits made to the mock object in sequence (so the length of the list is the number of times it has been awaited). Before any awaits have been made it is an empty list.

```
>>> mock = AsyncMock()
>>> async def main(*args):
...     await mock(*args)
...
>>> mock.await_args_list
[]
>>> asyncio.run(main('foo'))
>>> mock.await_args_list
[call('foo')]
>>> asyncio.run(main('bar'))
>>> mock.await_args_list
[call('foo'), call('bar')]
```

class unittest.mock.ThreadingMock(*spec=None*, *side_effect=None*, *return_value=DEFAULT*, *wraps=None*, *name=None*, *spec_set=None*, *unsafe=False*, ***, *timeout=UNSET*, ***kwargs*)

A version of **MagicMock** for multithreading tests. The **ThreadingMock** object provides extra methods to wait for a call to be invoked, rather than assert on it immediately.

The default timeout is specified by the `timeout` argument, or if unset by the **ThreadingMock.DEFAULT_TIMEOUT** attribute, which defaults to blocking (`None`).

You can configure the global default timeout by setting **ThreadingMock.DEFAULT_TIMEOUT**.

wait_until_called(*, *timeout=UNSET*)

Waits until the mock is called.

If a timeout was passed at the creation of the mock or if a timeout argument is passed to this function, the function raises an `AssertionError` if the call is not performed in time.

```
>>> mock = ThreadingMock()
>>> thread = threading.Thread(target=mock)
>>> thread.start()
>>> mock.wait_until_called(timeout=1)
>>> thread.join()
```

wait_until_any_call_with(**args*, ***kwargs*)

Waits until the mock is called with the specified arguments.

If a timeout was passed at the creation of the mock the function raises an `AssertionError` if the call is not performed in time.

```
>>> mock = ThreadingMock()
>>> thread = threading.Thread(target=mock, args=("arg1", '
>>> thread.start()
>>> mock.wait_until_any_call_with("arg1", "arg2", arg="th:
>>> thread.join()
```

DEFAULT_TIMEOUT

Global default timeout in seconds to create instances of **`ThreadingMock`**.

Adicionado na versão 3.13.

Fazendo chamadas

Mock objects are callable. The call will return the value set as the `return_value` attribute. The default return value is a new Mock object; it is created the first time the return value is accessed (either explicitly or by calling the Mock) - but it is stored and the same one returned each time.

Calls made to the object will be recorded in the attributes like **call_args** and **call_args_list**.

If **side_effect** is set then it will be called after the call has been recorded, so if **side_effect** raises an exception the call is still recorded.

The simplest way to make a mock raise an exception when called is to make **side_effect** an exception class or instance:

```
>>> m = MagicMock(side_effect=IndexError)
>>> m(1, 2, 3)
Traceback (most recent call last):
  ...
IndexError
>>> m.mock_calls
[call(1, 2, 3)]
>>> m.side_effect = KeyError('Bang!')
>>> m('two', 'three', 'four')
Traceback (most recent call last):
  ...
KeyError: 'Bang!'
>>> m.mock_calls
[call(1, 2, 3), call('two', 'three', 'four')]
```

If **side_effect** is a function then whatever that function returns is what calls to the mock return. The **side_effect** function is called with the same arguments as the mock. This allows you to vary the return value of the call dynamically, based on the input:

```
>>> def side_effect(value):
...     return value + 1
...
>>> m = MagicMock(side_effect=side_effect)
>>> m(1)
2
>>> m(2)
3
>>> m.mock_calls
[call(1), call(2)]
```

If you want the mock to still return the default return value (a new mock), or any set return value, then there are two ways of doing this. Either return `return_value` from inside `side_effect`, or return `DEFAULT`:

```
>>> m = MagicMock()
>>> def side_effect(*args, **kwargs):
...     return m.return_value
...
>>> m.side_effect = side_effect
>>> m.return_value = 3
>>> m()
3
>>> def side_effect(*args, **kwargs):
...     return DEFAULT
...
>>> m.side_effect = side_effect
>>> m()
3
```

To remove a `side_effect`, and return to the default behaviour, set the `side_effect` to `None`:

```
>>> m = MagicMock(return_value=6)
>>> def side_effect(*args, **kwargs):
...     return 3
...
>>> m.side_effect = side_effect
>>> m()
3
>>> m.side_effect = None
>>> m()
6
```

The `side_effect` can also be any iterable object. Repeated calls to the mock will return values from the iterable (until the iterable is exhausted and a `StopIteration` is raised):

```
>>> m = MagicMock(side_effect=[1, 2, 3])
>>> m()
1
>>> m()
2
```

```
>>> m()
3
>>> m()
Traceback (most recent call last):
  ...
StopIteration
```

If any members of the iterable are exceptions they will be raised instead of returned:

```
>>> iterable = (33, ValueError, 66)
>>> m = MagicMock(side_effect=iterable)
>>> m()
33
>>> m()
Traceback (most recent call last):
 ...
ValueError
>>> m()
66
```

Deletando Atributos

Mock objects create attributes on demand. This allows them to pretend to be objects of any type.

You may want a mock object to return `False` to a **`hasattr()`** call, or raise an **`AttributeError`** when an attribute is fetched. You can do this by providing an object as a **`spec`** for a mock, but that isn't always convenient.

You "block" attributes by deleting them. Once deleted, accessing an attribute will raise an **`AttributeError`**.

```
>>> mock = MagicMock()
>>> hasattr(mock, 'm')
True
>>> del mock.m
>>> hasattr(mock, 'm')
False
>>> del mock.f
>>> mock.f
```

```
Traceback (most recent call last):
    ...
AttributeError: f
```

Nomes de Mock e o atributo *name*

Since “name” is an argument to the `Mock` constructor, if you want your mock object to have a “name” attribute you can’t just pass it in at creation time. There are two alternatives. One option is to use `configure_mock()`:

```
>>> mock = MagicMock()
>>> mock.configure_mock(name='my_name')
>>> mock.name
'my_name'
```

A simpler option is to simply set the “name” attribute after mock creation:

```
>>> mock = MagicMock()
>>> mock.name = "foo"
```

Anexando Mocks como Atributos

When you attach a mock as an attribute of another mock (or as the return value) it becomes a “child” of that mock. Calls to the child are recorded in the `method_calls` and `mock_calls` attributes of the parent. This is useful for configuring child mocks and then attaching them to the parent, or for attaching mocks to a parent that records all calls to the children and allows you to make assertions about the order of calls between mocks:

```
>>> parent = MagicMock()
>>> child1 = MagicMock(return_value=None)
>>> child2 = MagicMock(return_value=None)
>>> parent.child1 = child1
>>> parent.child2 = child2
>>> child1(1)
>>> child2(2)
>>> parent.mock_calls
[call.child1(1), call.child2(2)]
```

The exception to this is if the mock has a name. This allows you to prevent the “parenting” if for some reason you don’t want it to happen.

```
>>> mock = MagicMock()
>>> not_a_child = MagicMock(name='not-a-child')
>>> mock.attribute = not_a_child
>>> mock.attribute()
<MagicMock name='not-a-child()' id='...'>
>>> mock.mock_calls
[]
```

Mocks created for you by `patch()` are automatically given names. To attach mocks that have names to a parent you use the `attach_mock()` method:

```
>>> thing1 = object()
>>> thing2 = object()
>>> parent = MagicMock()
>>> with patch('__main__.thing1', return_value=None) as child1:
...     with patch('__main__.thing2', return_value=None) as chil
...         parent.attach_mock(child1, 'child1')
...         parent.attach_mock(child2, 'child2')
...         child1('one')
...         child2('two')
...
>>> parent.mock_calls
[call.child1('one'), call.child2('two')]
```

[1]

The only exceptions are magic methods and attributes (those that have leading and trailing double underscores). Mock doesn’t create these but instead raises an `AttributeError`. This is because the interpreter will often implicitly request these methods, and gets *very* confused to get a new Mock object when it expects a magic method. If you need magic method support see magic methods.

Os criadores de patches

The patch decorators are used for patching objects only within the scope of the function they decorate. They automatically handle the unpatching for you, even if exceptions are raised. All of these functions can also be used in with statements or as class decorators.

patch

Nota

The key is to do the patching in the right namespace. See the section where to patch.

unittest.mock.patch(*target*, *new=DEFAULT*, *spec=None*, *create=False*, *spec_set=None*, *autospec=None*, *new_callable=None*, ***kwargs*)

`patch()` acts as a function decorator, class decorator or a context manager. Inside the body of the function or with statement, the *target* is patched with a *new* object. When the function/with statement exits the patch is undone.

If *new* is omitted, then the target is replaced with an **`AsyncMock`** if the patched object is an async function or a **`MagicMock`** otherwise. If **`patch()`** is used as a decorator and *new* is omitted, the created mock is passed in as an extra argument to the decorated function. If **`patch()`** is used as a context manager the created mock is returned by the context manager.

target should be a string in the form `'package.module.ClassName'`. The *target* is imported and the specified object replaced with the *new* object, so the *target* must be importable from the environment you are calling **`patch()`** from. The target is imported when the decorated function is executed, not at decoration time.

The *spec* and *spec_set* keyword arguments are passed to the **`MagicMock`** if patch is creating one for you.

In addition you can pass `spec=True` or `spec_set=True`, which causes patch to pass in the object being mocked as the spec/spec_set object.

new_callable allows you to specify a different class, or callable object, that will be called to create the *new* object. By default **`AsyncMock`** is used for async functions and **`MagicMock`** for the rest.

A more powerful form of *spec* is *autospec*. If you set `autospec=True` then the mock will be created with a spec from the object being replaced. All attributes of the mock will also have the spec of the corresponding attribute of the object being replaced. Methods and functions being mocked will have their arguments checked and will raise a **`TypeError`** if they are called with the wrong signature. For mocks replacing a class, their return value (the 'instance') will have the same spec as the class. See the **`create_autospec()`** function and Especificação automática.

Instead of `autospec=True` you can pass `autospec=some_object` to use an arbitrary object as the spec instead of the one being replaced.

By default **`patch()`** will fail to replace attributes that don't exist. If you pass in `create=True`, and the attribute doesn't exist, patch will create the attribute for you when the patched function is called, and delete it again after the patched function has exited. This is useful for writing tests against attributes that your production code creates at runtime. It is off by default because it can be dangerous. With it switched on you can write passing tests against APIs that don't actually exist!

Nota

Alterado na versão 3.5: If you are patching builtins in a module then you don't need to pass `create=True`, it will be added by default.

Patch can be used as a **`TestCase`** class decorator. It works by decorating each test method in the class. This reduces the boilerplate code when your test methods share a common patchings set. **`patch()`** finds tests by looking for method names that start with `patch.TEST_PREFIX`. By

default this is `'test'`, which matches the way **unittest** finds tests. You can specify an alternative prefix by setting `patch.TEST_PREFIX`.

Patch can be used as a context manager, with the with statement. Here the patching applies to the indented block after the with statement. If you use “as” then the patched object will be bound to the name after the “as”; very useful if **patch()** is creating a mock object for you.

patch() takes arbitrary keyword arguments. These will be passed to **AsyncMock** if the patched object is asynchronous, to **MagicMock** otherwise or to *new_callable* if specified.

`patch.dict(...)`, `patch.multiple(...)` e `patch.object(...)` estão disponíveis para casos de uso alternativos.

patch() as function decorator, creating the mock for you and passing it into the decorated function:

```
>>> @patch('__main__.SomeClass')
... def function(normal_argument, mock_class):
...     print(mock_class is SomeClass)
...
>>> function(None)
True
```

Patching a class replaces the class with a **MagicMock** *instance*. If the class is instantiated in the code under test then it will be the **return value** of the mock that will be used.

If the class is instantiated multiple times you could use **side effect** to return a new mock each time. Alternatively you can set the *return_value* to be anything you want.

To configure return values on methods of *instances* on the patched class you must do this on the **return value**. For example:

```
>>> class Class:
...     def method(self):
...         pass
...
```

```
>>> with patch('__main__.Class') as MockClass:
...     instance = MockClass.return_value
...     instance.method.return_value = 'foo'
...     assert Class() is instance
...     assert Class().method() == 'foo'
...
```

If you use *spec* or *spec_set* and `patch()` is replacing a *class*, then the return value of the created mock will have the same spec.

```
>>> Original = Class
>>> patcher = patch('__main__.Class', spec=True)
>>> MockClass = patcher.start()
>>> instance = MockClass()
>>> assert isinstance(instance, Original)
>>> patcher.stop()
```

The *new_callable* argument is useful where you want to use an alternative class to the default `MagicMock` for the created mock. For example, if you wanted a `NonCallableMock` to be used:

```
>>> thing = object()
>>> with patch('__main__.thing', new_callable=NonCallableMock) a
...     assert thing is mock_thing
...     thing()
...
Traceback (most recent call last):
  ...
TypeError: 'NonCallableMock' object is not callable
```

Another use case might be to replace an object with an `io.StringIO` instance:

```
>>> from io import StringIO
>>> def foo():
...     print('Something')
...
>>> @patch('sys.stdout', new_callable=StringIO)
... def test(mock_stdout):
...     foo()
...     assert mock_stdout.getvalue() == 'Something\n'
```

```
...
>>> test()
```

When **patch()** is creating a mock for you, it is common that the first thing you need to do is to configure the mock. Some of that configuration can be done in the call to patch. Any arbitrary keywords you pass into the call will be used to set attributes on the created mock:

```
>>> patcher = patch('__main__.thing', first='one', second='two')
>>> mock_thing = patcher.start()
>>> mock_thing.first
'one'
>>> mock_thing.second
'two'
```

As well as attributes on the created mock attributes, like the **return_value** and **side_effect**, of child mocks can also be configured. These aren't syntactically valid to pass in directly as keyword arguments, but a dictionary with these as keys can still be expanded into a **patch()** call using `**`:

```
>>> config = {'method.return_value': 3, 'other.side_effect': Key
>>> patcher = patch('__main__.thing', **config)
>>> mock_thing = patcher.start()
>>> mock_thing.method()
3
>>> mock_thing.other()
Traceback (most recent call last):
  ...
KeyError
```

By default, attempting to patch a function in a module (or a method or an attribute in a class) that does not exist will fail with **AttributeError**:

```
>>> @patch('sys.non_existing_attribute', 42)
... def test():
...     assert sys.non_existing_attribute == 42
...
>>> test()
Traceback (most recent call last):
```

```
  ...
AttributeError: <module 'sys' (built-in)> does not have the attr
```

but adding `create=True` in the call to **`patch()`** will make the previous example work as expected:

```
>>> @patch('sys.non_existing_attribute', 42, create=True)
... def test(mock_stdout):
...     assert sys.non_existing_attribute == 42
...
>>> test()
```

Alterado na versão 3.8: **`patch()`** now returns an **`AsyncMock`** if the target is an async function.

patch.object

patch.object(*target*, *attribute*, *new=DEFAULT*, *spec=None*, *create=False*, *spec_set=None*, *autospec=None*, *new_callable=None*, ***kwargs*)

patch the named member (*attribute*) on an object (*target*) with a mock object.

`patch.object()` can be used as a decorator, class decorator or a context manager. Arguments *new*, *spec*, *create*, *spec_set*, *autospec* and *new_callable* have the same meaning as for **`patch()`**. Like **`patch()`**, **`patch.object()`** takes arbitrary keyword arguments for configuring the mock object it creates.

When used as a class decorator **`patch.object()`** honours `patch.TEST_PREFIX` for choosing which methods to wrap.

You can either call **`patch.object()`** with three arguments or two arguments. The three argument form takes the object to be patched, the attribute name and the object to replace the attribute with.

When calling with the two argument form you omit the replacement object, and a mock is created for you and passed in as an extra argument to the decorated function:

```
>>> @patch.object(SomeClass, 'class_method')
... def test(mock_method):
...     SomeClass.class_method(3)
...     mock_method.assert_called_with(3)
...
>>> test()
```

spec, *create* and the other arguments to **patch.object()** have the same meaning as they do for **patch()**.

patch.dict

patch.dict(*in_dict*, *values=()*, *clear=False*, ***kwargs*)

Patch a dictionary, or dictionary like object, and restore the dictionary to its original state after the test, where the restored dictionary is a copy of the dictionary as it was before the test.

in_dict can be a dictionary or a mapping like container. If it is a mapping then it must at least support getting, setting and deleting items plus iterating over keys.

in_dict can also be a string specifying the name of the dictionary, which will then be fetched by importing it.

values can be a dictionary of values to set in the dictionary. *values* can also be an iterable of `(key, value)` pairs.

If *clear* is true then the dictionary will be cleared before the new values are set.

patch.dict() can also be called with arbitrary keyword arguments to set values in the dictionary.

Alterado na versão 3.8: **patch.dict()** now returns the patched dictionary when used as a context manager.

patch.dict() can be used as a context manager, decorator or class decorator:

```
>>> foo = {}
>>> @patch.dict(foo, {'newkey': 'newvalue'})
... def test():
...     assert foo == {'newkey': 'newvalue'}
...
>>> test()
>>> assert foo == {}
```

When used as a class decorator **`patch.dict()`** honours `patch.TEST_PREFIX` (default to `'test'`) for choosing which methods to wrap:

```
>>> import os
>>> import unittest
>>> from unittest.mock import patch
>>> @patch.dict('os.environ', {'newkey': 'newvalue'})
... class TestSample(unittest.TestCase):
...     def test_sample(self):
...         self.assertEqual(os.environ['newkey'], 'newvalue')
```

If you want to use a different prefix for your test, you can inform the patchers of the different prefix by setting `patch.TEST_PREFIX`. For more details about how to change the value of see TEST_PREFIX.

`patch.dict()` can be used to add members to a dictionary, or simply let a test change a dictionary, and ensure the dictionary is restored when the test ends.

```
>>> foo = {}
>>> with patch.dict(foo, {'newkey': 'newvalue'}) as patched_foo:
...     assert foo == {'newkey': 'newvalue'}
...     assert patched_foo == {'newkey': 'newvalue'}
...     # You can add, update or delete keys of foo (or patched_
...     patched_foo['spam'] = 'eggs'
...
>>> assert foo == {}
>>> assert patched_foo == {}
```

```
>>> import os
>>> with patch.dict('os.environ', {'newkey': 'newvalue'}):
...     print(os.environ['newkey'])
...
```

```
newvalue
>>> assert 'newkey' not in os.environ
```

Keywords can be used in the `patch.dict()` call to set values in the dictionary:

```
>>> mymodule = MagicMock()
>>> mymodule.function.return_value = 'fish'
>>> with patch.dict('sys.modules', mymodule=mymodule):
...     import mymodule
...     mymodule.function('some', 'args')
...
'fish'
```

`patch.dict()` can be used with dictionary like objects that aren't actually dictionaries. At the very minimum they must support item getting, setting, deleting and either iteration or membership test. This corresponds to the magic methods `__getitem__()`, `__setitem__()`, `__delitem__()` and either `__iter__()` or `__contains__()`.

```
>>> class Container:
...     def __init__(self):
...         self.values = {}
...     def __getitem__(self, name):
...         return self.values[name]
...     def __setitem__(self, name, value):
...         self.values[name] = value
...     def __delitem__(self, name):
...         del self.values[name]
...     def __iter__(self):
...         return iter(self.values)
...
>>> thing = Container()
>>> thing['one'] = 1
>>> with patch.dict(thing, one=2, two=3):
...     assert thing['one'] == 2
...     assert thing['two'] == 3
...
>>> assert thing['one'] == 1
>>> assert list(thing) == ['one']
```

patch.multiple

patch.multiple(*target*, *spec=None*, *create=False*, *spec_set=None*, *autospec=None*, *new_callable=None*, ***kwargs*)

Perform multiple patches in a single call. It takes the object to be patched (either as an object or a string to fetch the object by importing) and keyword arguments for the patches:

```
with patch.multiple(settings, FIRST_PATCH='one', SECOND_PATCH
    ...
```

Use **DEFAULT** as the value if you want **patch.multiple()** to create mocks for you. In this case the created mocks are passed into a decorated function by keyword, and a dictionary is returned when **patch.multiple()** is used as a context manager.

patch.multiple() can be used as a decorator, class decorator or a context manager. The arguments *spec*, *spec_set*, *create*, *autospec* and *new_callable* have the same meaning as for **patch()**. These arguments will be applied to *all* patches done by **patch.multiple()**.

When used as a class decorator **patch.multiple()** honours `patch.TEST_PREFIX` for choosing which methods to wrap.

If you want **patch.multiple()** to create mocks for you, then you can use **DEFAULT** as the value. If you use **patch.multiple()** as a decorator then the created mocks are passed into the decorated function by keyword.

```
>>> thing = object()
>>> other = object()

>>> @patch.multiple('__main__', thing=DEFAULT, other=DEFAULT)
... def test_function(thing, other):
...     assert isinstance(thing, MagicMock)
...     assert isinstance(other, MagicMock)
...
>>> test_function()
```

patch.multiple() can be nested with other `patch` decorators, but put arguments passed by keyword *after* any of the standard arguments created by **patch()**:

```
>>> @patch('sys.exit')
... @patch.multiple('__main__', thing=DEFAULT, other=DEFAULT)
... def test_function(mock_exit, other, thing):
...     assert 'other' in repr(other)
...     assert 'thing' in repr(thing)
...     assert 'exit' in repr(mock_exit)
...
>>> test_function()
```

If **patch.multiple()** is used as a context manager, the value returned by the context manager is a dictionary where created mocks are keyed by name:

```
>>> with patch.multiple('__main__', thing=DEFAULT, other=DEFAULT
...     assert 'other' in repr(values['other'])
...     assert 'thing' in repr(values['thing'])
...     assert values['thing'] is thing
...     assert values['other'] is other
...
```

métodos do patch: *start* e *stop*

All the patchers have **`start()`** and **`stop()`** methods. These make it simpler to do patching in `setUp` methods or where you want to do multiple patches without nesting decorators or with statements.

To use them call **patch()**, **patch.object()** or **patch.dict()** as normal and keep a reference to the returned `patcher` object. You can then call **`start()`** to put the patch in place and **`stop()`** to undo it.

If you are using **patch()** to create a mock for you then it will be returned by the call to `patcher.start`.

```
>>> patcher = patch('package.module.ClassName')
>>> from package import module
```

```
>>> original = module.ClassName
>>> new_mock = patcher.start()
>>> assert module.ClassName is not original
>>> assert module.ClassName is new_mock
>>> patcher.stop()
>>> assert module.ClassName is original
>>> assert module.ClassName is not new_mock
```

A typical use case for this might be for doing multiple patches in the `setUp` method of a **`TestCase`**:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         self.patcher1 = patch('package.module.Class1')
...         self.patcher2 = patch('package.module.Class2')
...         self.MockClass1 = self.patcher1.start()
...         self.MockClass2 = self.patcher2.start()
...
...     def tearDown(self):
...         self.patcher1.stop()
...         self.patcher2.stop()
...
...     def test_something(self):
...         assert package.module.Class1 is self.MockClass1
...         assert package.module.Class2 is self.MockClass2
...
>>> MyTest('test_something').run()
```

Cuidado

If you use this technique you must ensure that the patching is "undone" by calling `stop`. This can be fiddlier than you might think, because if an exception is raised in the `setUp` then `tearDown` is not called. **`unittest.TestCase.addCleanup()`** makes this easier:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         patcher = patch('package.module.Class')
...         self.MockClass = patcher.start()
...         self.addCleanup(patcher.stop)
...
...     def test_something(self):
```

```
...         assert package.module.Class is self.MockClass
...
```

As an added bonus you no longer need to keep a reference to the `patcher` object.

It is also possible to stop all patches which have been started by using **`patch.stopall()`**.

patch.stopall()

Stop all active patches. Only stops patches started with `start`.

patch de embutidos

You can patch any builtins within a module. The following example patches builtin **`ord()`**:

```
>>> @patch('__main__.ord')
... def test(mock_ord):
...     mock_ord.return_value = 101
...     print(ord('c'))
...
>>> test()
101
```

TEST_PREFIX

All of the patchers can be used as class decorators. When used in this way they wrap every test method on the class. The patchers recognise methods that start with `'test'` as being test methods. This is the same way that the **`unittest.TestLoader`** finds test methods by default.

It is possible that you want to use a different prefix for your tests. You can inform the patchers of the different prefix by setting `patch.TEST_PREFIX`:

```
>>> patch.TEST_PREFIX = 'foo'
>>> value = 3
>>>
```

```
>>> @patch('__main__.value', 'not three')
... class Thing:
...     def foo_one(self):
...         print(value)
...     def foo_two(self):
...         print(value)
...
>>>
>>> Thing().foo_one()
not three
>>> Thing().foo_two()
not three
>>> value
3
```

Aninhando Decoradores Patch

If you want to perform multiple patches then you can simply stack up the decorators.

You can stack up multiple patch decorators using this pattern:

```
>>> @patch.object(SomeClass, 'class_method')
... @patch.object(SomeClass, 'static_method')
... def test(mock1, mock2):
...     assert SomeClass.static_method is mock1
...     assert SomeClass.class_method is mock2
...     SomeClass.static_method('foo')
...     SomeClass.class_method('bar')
...     return mock1, mock2
...
>>> mock1, mock2 = test()
>>> mock1.assert_called_once_with('foo')
>>> mock2.assert_called_once_with('bar')
```

Note that the decorators are applied from the bottom upwards. This is the standard way that Python applies decorators. The order of the created mocks passed into your test function matches this order.

Onde fazer patch

patch() works by (temporarily) changing the object that a *name* points to with another one. There can be many names pointing to any individual object, so for patching to work you must ensure that you patch the name used by the system under test.

The basic principle is that you patch where an object is *looked up*, which is not necessarily the same place as where it is defined. A couple of examples will help to clarify this.

Imagine we have a project that we want to test with the following structure:

```
a.py
    -> Defines SomeClass

b.py
    -> from a import SomeClass
    -> some_function instantiates SomeClass
```

Now we want to test `some_function` but we want to mock out `SomeClass` using **patch()**. The problem is that when we import module b, which we will have to do when it imports `SomeClass` from module a. If we use **`patch()`** to mock out `a.SomeClass` then it will have no effect on our test; module b already has a reference to the *real* `SomeClass` and it looks like our patching had no effect.

The key is to patch out `SomeClass` where it is used (or where it is looked up). In this case `some_function` will actually look up `SomeClass` in module b, where we have imported it. The patching should look like:

```
@patch('b.SomeClass')
```

However, consider the alternative scenario where instead of `from a import SomeClass` module b does `import a` and `some_function` uses `a.SomeClass`. Both of these import forms are common. In this case the class we want to patch is being looked up in the module and so we have to patch `a.SomeClass` instead:

```
@patch('a.SomeClass')
```

Patching Descriptors and Proxy Objects

Both patch and patch.object correctly patch and restore descriptors: class methods, static methods and properties. You should patch these on the *class* rather than an instance. They also work with *some* objects that proxy attribute access, like the django settings object [https://web.archive.org/web/20200603181648/http://www.voidspace.org.uk/python/weblog/arch_d7_2010_12_04.shtml#e1198].

MagicMock and magic method support

Simulando Métodos Mágicos

`Mock` supports mocking the Python protocol methods, also known as "magic methods". This allows mock objects to replace containers or other objects that implement Python protocols.

Because magic methods are looked up differently from normal methods [[2]], this support has been specially implemented. This means that only specific magic methods are supported. The supported list includes *almost* all of them. If there are any missing that you need please let us know.

You mock magic methods by setting the method you are interested in to a function or a mock instance. If you are using a function then it *must* take `self` as the first argument [[3]].

```
>>> def __str__(self):
...     return 'fooble'
...
>>> mock = Mock()
>>> mock.__str__ = __str__
>>> str(mock)
'fooble'

>>> mock = Mock()
>>> mock.__str__ = Mock()
>>> mock.__str__.return_value = 'fooble'
>>> str(mock)
'fooble'
```

```
>>> mock = Mock()
>>> mock.__iter__ = Mock(return_value=iter([]))
>>> list(mock)
[]
```

One use case for this is for mocking objects used as context managers in a `with` statement:

```
>>> mock = Mock()
>>> mock.__enter__ = Mock(return_value='foo')
>>> mock.__exit__ = Mock(return_value=False)
>>> with mock as m:
...     assert m == 'foo'
...
>>> mock.__enter__.assert_called_with()
>>> mock.__exit__.assert_called_with(None, None, None)
```

Calls to magic methods do not appear in `method_calls`, but they are recorded in `mock_calls`.

Nota

If you use the *spec* keyword argument to create a mock then attempting to set a magic method that isn't in the spec will raise an `AttributeError`.

A lista completa de métodos mágicos compatíveis é:

- `__hash__`, `__sizeof__`, `__repr__` e `__str__`
- `__dir__`, `__format__` e `__subclasses__`
- `__round__`, `__floor__`, `__trunc__` e `__ceil__`
- Comparações: `__lt__`, `__gt__`, `__le__`, `__ge__`, `__eq__` e `__ne__`
- Container methods: `__getitem__`, `__setitem__`, `__delitem__`, `__contains__`, `__len__`, `__iter__`, `__reversed__` and `__missing__`
- Gerenciador de contexto: `__enter__`, `__exit__`, `__aenter__` e `__aexit__`
- Métodos numéricos unários: `__neg__`, `__pos__` e `__invert__`
- The numeric methods (including right hand and in-place variants): `__add__`, `__sub__`, `__mul__`, `__matmul__`, `__truediv__`,

`__floordiv__`, `__mod__`, `__divmod__`, `__lshift__`, `__rshift__`, `__and__`, `__xor__`, `__or__`, and `__pow__`
- Métodos de conversão numérica: `__complex__`, `__int__`, `__float__` e `__index__`
- Métodos descritores: `__get__`, `__set__` e `__delete__`
- Pickling: `__reduce__`, `__reduce_ex__`, `__getinitargs__`, `__getnewargs__`, `__getstate__` e `__setstate__`
- File system path representation: `__fspath__`
- Métodos de iteração assíncrona: `__aiter__` e `__anext__`

Alterado na versão 3.8: Adicionado suporte para **`os.PathLike.__fspath__()`**.

Alterado na versão 3.8: Adicionado suporte para `__aenter__`, `__aexit__`, `__aiter__` e `__anext__`.

The following methods exist but are *not* supported as they are either in use by mock, can't be set dynamically, or can cause problems:

- `__getattr__`, `__setattr__`, `__init__` e `__new__`
- `__prepare__`, `__instancecheck__`, `__subclasscheck__`, `__del__`

Magic Mock

Existem duas variantes de `MagicMock`: **`MagicMock`** e **`NonCallableMagicMock`**.

class unittest.mock.MagicMock(**args*, ***kw*)

`MagicMock` is a subclass of **`Mock`** with default implementations of most of the magic methods. You can use `MagicMock` without having to configure the magic methods yourself.

The constructor parameters have the same meaning as for **`Mock`**.

If you use the *spec* or *spec_set* arguments then *only* magic methods that exist in the spec will be created.

class unittest.mock.NonCallableMagicMock(**args*, ***kw*)

Uma versão não-chamável de **MagicMock**.

The constructor parameters have the same meaning as for **MagicMock**, with the exception of *return_value* and *side_effect* which have no meaning on a non-callable mock.

The magic methods are setup with **MagicMock** objects, so you can configure them and use them in the usual way:

```
>>> mock = MagicMock()
>>> mock[3] = 'fish'
>>> mock.__setitem__.assert_called_with(3, 'fish')
>>> mock.__getitem__.return_value = 'result'
>>> mock[2]
'result'
```

By default many of the protocol methods are required to return objects of a specific type. These methods are preconfigured with a default return value, so that they can be used without you having to do anything if you aren't interested in the return value. You can still *set* the return value manually if you want to change the default.

Métodos e seus padrões:

- `__lt__`: **NotImplemented**
- `__gt__`: **NotImplemented**
- `__le__`: **NotImplemented**
- `__ge__`: **NotImplemented**
- `__int__`: `1`
- `__contains__`: `False`
- `__len__`: `0`
- `__iter__`: `iter([])`
- `__exit__`: `False`
- `__aexit__`: `False`
- `__complex__`: `1j`
- `__float__`: `1.0`
- `__bool__`: `True`
- `__index__`: `1`
- `__hash__`: hash padrão para o mock

- `__str__`: *str* padrão para o mock
- `__sizeof__`: *sizeof* padrão para o mock

Por exemplo:

```
>>> mock = MagicMock()
>>> int(mock)
1
>>> len(mock)
0
>>> list(mock)
[]
>>> object() in mock
False
```

The two equality methods, **`__eq__()`** and **`__ne__()`**, are special. They do the default equality comparison on identity, using the **`side_effect`** attribute, unless you change their return value to return something else:

```
>>> MagicMock() == 3
False
>>> MagicMock() != 3
True
>>> mock = MagicMock()
>>> mock.__eq__.return_value = True
>>> mock == 3
True
```

The return value of **`__iter__()`** can be any iterable object and isn't required to be an iterator:

```
>>> mock = MagicMock()
>>> mock.__iter__.return_value = ['a', 'b', 'c']
>>> list(mock)
['a', 'b', 'c']
>>> list(mock)
['a', 'b', 'c']
```

If the return value *is* an iterator, then iterating over it once will consume it and subsequent iterations will result in an empty list:

```
>>> mock.__iter__.return_value = iter(['a', 'b', 'c'])
>>> list(mock)
['a', 'b', 'c']
>>> list(mock)
[]
```

`MagicMock` has all of the supported magic methods configured except for some of the obscure and obsolete ones. You can still set these up if you want.

Magic methods that are supported but not setup by default in `MagicMock` are:

- `__subclasses__`
- `__dir__`
- `__format__`
- `__get__`, `__set__` e `__delete__`
- `__reversed__` e `__missing__`
- `__reduce__`, `__reduce_ex__`, `__getinitargs__`, `__getnewargs__`, `__getstate__` e `__setstate__`
- `__getformat__`

[2]

Magic methods *should* be looked up on the class rather than the instance. Different versions of Python are inconsistent about applying this rule. The supported protocol methods should work with all supported versions of Python.

[3]

The function is basically hooked up to the class, but each `Mock` instance is kept isolated from the others.

Ajudantes

sentinel

unittest.mock.sentinel

The `sentinel` object provides a convenient way of providing unique objects for your tests.

Attributes are created on demand when you access them by name. Accessing the same attribute will always return the same object. The objects returned have a sensible repr so that test failure messages are readable.

Alterado na versão 3.7: The `sentinel` attributes now preserve their identity when they are **`copied`** or **`pickled`**.

Sometimes when testing you need to test that a specific object is passed as an argument to another method, or returned. It can be common to create named sentinel objects to test this. **`sentinel`** provides a convenient way of creating and testing the identity of objects like this.

In this example we monkey patch `method` to return `sentinel.some_object`:

```
>>> real = ProductionClass()
>>> real.method = Mock(name="method")
>>> real.method.return_value = sentinel.some_object
>>> result = real.method()
>>> assert result is sentinel.some_object
>>> result
sentinel.some_object
```

DEFAULT

unittest.mock.DEFAULT

The **`DEFAULT`** object is a pre-created sentinel (actually `sentinel.DEFAULT`). It can be used by **`side_effect`** functions to indicate that the normal return value should be used.

chamada

unittest.mock.call(**args, **kwargs*)

call() is a helper object for making simpler assertions, for comparing with **call_args**, **call_args_list**, **mock_calls** and **method_calls**. **call()** can also be used with **assert_has_calls()**.

```
>>> m = MagicMock(return_value=None)
>>> m(1, 2, a='foo', b='bar')
>>> m()
>>> m.call_args_list == [call(1, 2, a='foo', b='bar'), call(
True
```

call.call_list()

For a call object that represents multiple calls, **call_list()** returns a list of all the intermediate calls as well as the final call.

`call_list` is particularly useful for making assertions on "chained calls". A chained call is multiple calls on a single line of code. This results in multiple entries in **mock_calls** on a mock. Manually constructing the sequence of calls can be tedious.

call_list() can construct the sequence of calls from the same chained call:

```
>>> m = MagicMock()
>>> m(1).method(arg='foo').other('bar')(2.0)
<MagicMock name='mock().method().other()()' id='...'>
>>> kall = call(1).method(arg='foo').other('bar')(2.0)
>>> kall.call_list()
[call(1),
 call().method(arg='foo'),
 call().method().other('bar'),
 call().method().other()(2.0)]
>>> m.mock_calls == kall.call_list()
True
```

A `call` object is either a tuple of (positional args, keyword args) or (name, positional args, keyword args) depending on how it was constructed. When you construct them yourself this isn't particularly interesting, but the `call` objects that are in the **Mock.call_args**, **Mock.call_args_list** and

`Mock.mock_calls` attributes can be introspected to get at the individual arguments they contain.

The `call` objects in `Mock.call_args` and `Mock.call_args_list` are two-tuples of (positional args, keyword args) whereas the `call` objects in `Mock.mock_calls`, along with ones you construct yourself, are three-tuples of (name, positional args, keyword args).

You can use their "tupleness" to pull out the individual arguments for more complex introspection and assertions. The positional arguments are a tuple (an empty tuple if there are no positional arguments) and the keyword arguments are a dictionary:

```
>>> m = MagicMock(return_value=None)
>>> m(1, 2, 3, arg='one', arg2='two')
>>> kall = m.call_args
>>> kall.args
(1, 2, 3)
>>> kall.kwargs
{'arg': 'one', 'arg2': 'two'}
>>> kall.args is kall[0]
True
>>> kall.kwargs is kall[1]
True

>>> m = MagicMock()
>>> m.foo(4, 5, 6, arg='two', arg2='three')
<MagicMock name='mock.foo()' id='...'>
>>> kall = m.mock_calls[0]
>>> name, args, kwargs = kall
>>> name
'foo'
>>> args
(4, 5, 6)
>>> kwargs
{'arg': 'two', 'arg2': 'three'}
>>> name is m.mock_calls[0][0]
True
```

create_autospec

unittest.mock.create_autospec(*spec*, *spec_set=False*, *instance=False*, ***kwargs*)

Create a mock object using another object as a spec. Attributes on the mock will use the corresponding attribute on the *spec* object as their spec.

Functions or methods being mocked will have their arguments checked to ensure that they are called with the correct signature.

If *spec_set* is `True` then attempting to set attributes that don't exist on the spec object will raise an **`AttributeError`**.

If a class is used as a spec then the return value of the mock (the instance of the class) will have the same spec. You can use a class as the spec for an instance object by passing `instance=True`. The returned mock will only be callable if instances of the mock are callable.

`create_autospec()` also takes arbitrary keyword arguments that are passed to the constructor of the created mock.

See Especificação automática for examples of how to use auto-speccing with **`create_autospec()`** and the *autospec* argument to **`patch()`**.

Alterado na versão 3.8: **`create_autospec()`** now returns an **`AsyncMock`** if the target is an async function.

ANY

unittest.mock.ANY

Sometimes you may need to make assertions about *some* of the arguments in a call to mock, but either not care about some of the arguments or want to pull them individually out of **`call_args`** and make more complex assertions on them.

To ignore certain arguments you can pass in objects that compare equal to *everything*. Calls to **`assert_called_with()`** and

assert_called_once_with() will then succeed no matter what was passed in.

```
>>> mock = Mock(return_value=None)
>>> mock('foo', bar=object())
>>> mock.assert_called_once_with('foo', bar=ANY)
```

ANY can also be used in comparisons with call lists like **mock_calls**:

```
>>> m = MagicMock(return_value=None)
>>> m(1)
>>> m(1, 2)
>>> m(object())
>>> m.mock_calls == [call(1), call(1, 2), ANY]
True
```

ANY is not limited to comparisons with call objects and so can also be used in test assertions:

```
class TestStringMethods(unittest.TestCase):

    def test_split(self):
        s = 'hello world'
        self.assertEqual(s.split(), ['hello', ANY])
```

FILTER_DIR

unittest.mock.FILTER_DIR

FILTER_DIR is a module level variable that controls the way mock objects respond to **dir()**. The default is `True`, which uses the filtering described below, to only show useful members. If you dislike this filtering, or need to switch it off for diagnostic purposes, then set `mock.FILTER_DIR = False`.

With filtering on, `dir(some_mock)` shows only useful attributes and will include any dynamically created attributes that wouldn't normally be shown. If the mock was created with a *spec* (or *autospec* of course) then all the attributes from the original are shown, even if they haven't been accessed yet:

```
>>> dir(Mock())
['assert_any_call',
 'assert_called',
 'assert_called_once',
 'assert_called_once_with',
 'assert_called_with',
 'assert_has_calls',
 'assert_not_called',
 'attach_mock',
 ...
>>> from urllib import request
>>> dir(Mock(spec=request))
['AbstractBasicAuthHandler',
 'AbstractDigestAuthHandler',
 'AbstractHTTPHandler',
 'BaseHandler',
 ...
```

Many of the not-very-useful (private to **`Mock`** rather than the thing being mocked) underscore and double underscore prefixed attributes have been filtered from the result of calling **`dir()`** on a **`Mock`**. If you dislike this behaviour you can switch it off by setting the module level switch **`FILTER_DIR`**:

```
>>> from unittest import mock
>>> mock.FILTER_DIR = False
>>> dir(mock.Mock())
['_NonCallableMock__get_return_value',
 '_NonCallableMock__get_side_effect',
 '_NonCallableMock__return_value_doc',
 '_NonCallableMock__set_return_value',
 '_NonCallableMock__set_side_effect',
 '__call__',
 '__class__',
 ...
```

Alternatively you can just use `vars(my_mock)` (instance members) and `dir(type(my_mock))` (type members) to bypass the filtering irrespective of **`FILTER_DIR`**.

mock_open

unittest.mock.mock_open(*mock=None*, *read_data=''*)

A helper function to create a mock to replace the use of `open()`. It works for `open()` called directly or used as a context manager.

The *mock* argument is the mock object to configure. If `None` (the default) then a `MagicMock` will be created for you, with the API limited to methods or attributes available on standard file handles.

read_data is a string for the `read()`, `readline()`, and `readlines()` methods of the file handle to return. Calls to those methods will take data from *read_data* until it is depleted. The mock of these methods is pretty simplistic: every time the *mock* is called, the *read_data* is rewound to the start. If you need more control over the data that you are feeding to the tested code you will need to customize this mock for yourself. When that is insufficient, one of the in-memory filesystem packages on PyPI [https://pypi.org] can offer a realistic filesystem for testing.

Alterado na versão 3.4: Added `readline()` and `readlines()` support. The mock of `read()` changed to consume *read_data* rather than returning it on each call.

Alterado na versão 3.5: read_data is now reset on each call to the *mock*.

Alterado na versão 3.8: Added `__iter__()` to implementation so that iteration (such as in for loops) correctly consumes *read_data*.

Using `open()` as a context manager is a great way to ensure your file handles are closed properly and is becoming common:

```
with open('/some/path', 'w') as f:
    f.write('something')
```

The issue is that even if you mock out the call to `open()` it is the *returned object* that is used as a context manager (and has `__enter__()` and `__exit__()` called).

Mocking context managers with a `MagicMock` is common enough and fiddly enough that a helper function is useful.

```
>>> m = mock_open()
>>> with patch('__main__.open', m):
...     with open('foo', 'w') as h:
...         h.write('some stuff')
...
>>> m.mock_calls
[call('foo', 'w'),
 call().__enter__(),
 call().write('some stuff'),
 call().__exit__(None, None, None)]
>>> m.assert_called_once_with('foo', 'w')
>>> handle = m()
>>> handle.write.assert_called_once_with('some stuff')
```

E para ler arquivos:

```
>>> with patch('__main__.open', mock_open(read_data='bibble')) a
...     with open('foo') as h:
...         result = h.read()
...
>>> m.assert_called_once_with('foo')
>>> assert result == 'bibble'
```

Especificação automática

Autospeccing is based on the existing `spec` feature of mock. It limits the api of mocks to the api of an original object (the spec), but it is recursive (implemented lazily) so that attributes of mocks only have the same api as the attributes of the spec. In addition mocked functions / methods have the same call signature as the original so they raise a `TypeError` if they are called incorrectly.

Before I explain how auto-speccing works, here's why it is needed.

`Mock` is a very powerful and flexible object, but it suffers from a flaw which is general to mocking. If you refactor some of your code, rename members

and so on, any tests for code that is still using the *old api* but uses mocks instead of the real objects will still pass. This means your tests can all pass even though your code is broken.

Alterado na versão 3.5: Before 3.5, tests with a typo in the word assert would silently pass when they should raise an error. You can still achieve this behavior by passing `unsafe=True` to Mock.

Note that this is another reason why you need integration tests as well as unit tests. Testing everything in isolation is all fine and dandy, but if you don't test how your units are "wired together" there is still lots of room for bugs that tests might have caught.

`unittest.mock` already provides a feature to help with this, called speccing. If you use a class or instance as the **`spec`** for a mock then you can only access attributes on the mock that exist on the real class:

```
>>> from urllib import request
>>> mock = Mock(spec=request.Request)
>>> mock.assret_called_with  # Intentional typo!
Traceback (most recent call last):
 ...
AttributeError: Mock object has no attribute 'assret_called_with
```

The spec only applies to the mock itself, so we still have the same issue with any methods on the mock:

```
>>> mock.header_items()
<mock.Mock object at 0x...>
>>> mock.header_items.assret_called_with()  # Intentional typo!
```

Auto-speccing solves this problem. You can either pass `autospec=True` to **`patch()`** / **`patch.object()`** or use the **`create_autospec()`** function to create a mock with a spec. If you use the `autospec=True` argument to **`patch()`** then the object that is being replaced will be used as the spec object. Because the speccing is done "lazily" (the spec is created as attributes on the mock are accessed) you can use it with very complex or deeply nested objects (like modules that import modules that import modules) without a big performance hit.

Aqui está um exemplo disso em uso:

```
>>> from urllib import request
>>> patcher = patch('__main__.request', autospec=True)
>>> mock_request = patcher.start()
>>> request is mock_request
True
>>> mock_request.Request
<MagicMock name='request.Request' spec='Request' id='...'>
```

You can see that **`request.Request`** has a spec. **`request.Request`** takes two arguments in the constructor (one of which is *self*). Here's what happens if we try to call it incorrectly:

```
>>> req = request.Request()
Traceback (most recent call last):
 ...
TypeError: <lambda>() takes at least 2 arguments (1 given)
```

The spec also applies to instantiated classes (i.e. the return value of specced mocks):

```
>>> req = request.Request('foo')
>>> req
<NonCallableMagicMock name='request.Request()' spec='Request' id
```

`Request` objects are not callable, so the return value of instantiating our mocked out **`request.Request`** is a non-callable mock. With the spec in place any typos in our asserts will raise the correct error:

```
>>> req.add_header('spam', 'eggs')
<MagicMock name='request.Request().add_header()' id='...'>
>>> req.add_header.assret_called_with  # Intentional typo!
Traceback (most recent call last):
 ...
AttributeError: Mock object has no attribute 'assret_called_with
>>> req.add_header.assert_called_with('spam', 'eggs')
```

In many cases you will just be able to add `autospec=True` to your existing **`patch()`** calls and then be protected against bugs due to typos and api

changes.

As well as using *autospec* through **patch()** there is a **create_autospec()** for creating autospecced mocks directly:

```
>>> from urllib import request
>>> mock_request = create_autospec(request)
>>> mock_request.Request('foo', 'bar')
<NonCallableMagicMock name='mock.Request()' spec='Request' id='.
```

This isn't without caveats and limitations however, which is why it is not the default behaviour. In order to know what attributes are available on the spec object, autospec has to introspect (access attributes) the spec. As you traverse attributes on the mock a corresponding traversal of the original object is happening under the hood. If any of your specced objects have properties or descriptors that can trigger code execution then you may not be able to use autospec. On the other hand it is much better to design your objects so that introspection is safe [[4]].

A more serious problem is that it is common for instance attributes to be created in the **__init__()** method and not to exist on the class at all. *autospec* can't know about any dynamically created attributes and restricts the api to visible attributes.

```
>>> class Something:
...   def __init__(self):
...     self.a = 33
...
>>> with patch('__main__.Something', autospec=True):
...   thing = Something()
...   thing.a
...
Traceback (most recent call last):
  ...
AttributeError: Mock object has no attribute 'a'
```

There are a few different ways of resolving this problem. The easiest, but not necessarily the least annoying, way is to simply set the required attributes on the mock after creation. Just because *autospec* doesn't allow

you to fetch attributes that don't exist on the spec it doesn't prevent you setting them:

```
>>> with patch('__main__.Something', autospec=True):
...   thing = Something()
...   thing.a = 33
...
```

There is a more aggressive version of both *spec* and *autospec* that *does* prevent you setting non-existent attributes. This is useful if you want to ensure your code only *sets* valid attributes too, but obviously it prevents this particular scenario:

```
>>> with patch('__main__.Something', autospec=True, spec_set=Tru
...   thing = Something()
...   thing.a = 33
...
Traceback (most recent call last):
 ...
AttributeError: Mock object has no attribute 'a'
```

Probably the best way of solving the problem is to add class attributes as default values for instance members initialised in **`__init__()`**. Note that if you are only setting default attributes in **`__init__()`** then providing them via class attributes (shared between instances of course) is faster too. e.g.

```
class Something:
    a = 33
```

This brings up another issue. It is relatively common to provide a default value of `None` for members that will later be an object of a different type. `None` would be useless as a spec because it wouldn't let you access *any* attributes or methods on it. As `None` is *never* going to be useful as a spec, and probably indicates a member that will normally of some other type, autospec doesn't use a spec for members that are set to `None`. These will just be ordinary mocks (well - MagicMocks):

```
>>> class Something:
...     member = None
```

```
...
>>> mock = create_autospec(Something)
>>> mock.member.foo.bar.baz()
<MagicMock name='mock.member.foo.bar.baz()' id='...'>
```

If modifying your production classes to add defaults isn't to your liking then there are more options. One of these is simply to use an instance as the spec rather than the class. The other is to create a subclass of the production class and add the defaults to the subclass without affecting the production class. Both of these require you to use an alternative object as the spec. Thankfully **`patch()`** supports this - you can simply pass the alternative object as the *autospec* argument:

```
>>> class Something:
...   def __init__(self):
...     self.a = 33
...
>>> class SomethingForTest(Something):
...   a = 33
...
>>> p = patch('__main__.Something', autospec=SomethingForTest)
>>> mock = p.start()
>>> mock.a
<NonCallableMagicMock name='Something.a' spec='int' id='...'>
```

[4]

This only applies to classes or already instantiated objects. Calling a mocked class to create a mock instance *does not* create a real instance. It is only attribute lookups - along with calls to **`dir()`** - that are done.

Vedando mocks

unittest.mock.seal(*mock*)

Seal will disable the automatic creation of mocks when accessing an attribute of the mock being sealed or any of its attributes that are already mocks recursively.

If a mock instance with a name or a spec is assigned to an attribute it won't be considered in the sealing chain. This allows one to prevent seal from fixing part of the mock object.

```
>>> mock = Mock()
>>> mock.submock.attribute1 = 2
>>> mock.not_submock = mock.Mock(name="sample_name")
>>> seal(mock)
>>> mock.new_attribute  # This will raise AttributeError.
>>> mock.submock.attribute2  # This will raise AttributeError
>>> mock.not_submock.attribute2  # This won't raise.
```

Adicionado na versão 3.7.

Order of precedence of `side_effect`, `return_value` and *wraps*

The order of their precedence is:

1. `side_effect`
2. `return_value`
3. *wraps*

If all three are set, mock will return the value from `side_effect`, ignoring `return_value` and the wrapped object altogether. If any two are set, the one with the higher precedence will return the value. Regardless of the order of which was set first, the order of precedence remains unchanged.

```
>>> from unittest.mock import Mock
>>> class Order:
...     @staticmethod
...     def get_value():
...         return "third"
...
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.get_value.side_effect = ["first"]
>>> order_mock.get_value.return_value = "second"
>>> order_mock.get_value()
'first'
```

As `None` is the default value of **`side_effect`**, if you reassign its value back to `None`, the order of precedence will be checked between **`return_value`** and the wrapped object, ignoring **`side_effect`**.

```
>>> order_mock.get_value.side_effect = None
>>> order_mock.get_value()
'second'
```

If the value being returned by **`side_effect`** is **`DEFAULT`**, it is ignored and the order of precedence moves to the successor to obtain the value to return.

```
>>> from unittest.mock import DEFAULT
>>> order_mock.get_value.side_effect = [DEFAULT]
>>> order_mock.get_value()
'second'
```

When **`Mock`** wraps an object, the default value of **`return_value`** will be **`DEFAULT`**.

```
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.return_value
sentinel.DEFAULT
>>> order_mock.get_value.return_value
sentinel.DEFAULT
```

The order of precedence will ignore this value and it will move to the last successor which is the wrapped object.

As the real call is being made to the wrapped object, creating an instance of this mock will return the real instance of the class. The positional arguments, if any, required by the wrapped object must be passed.

```
>>> order_mock_instance = order_mock()
>>> isinstance(order_mock_instance, Order)
True
>>> order_mock_instance.get_value()
'third'
```

```
>>> order_mock.get_value.return_value = DEFAULT
>>> order_mock.get_value()
'third'
```

```
>>> order_mock.get_value.return_value = "second"
>>> order_mock.get_value()
'second'
```

But if you assign `None` to it, this will not be ignored as it is an explicit assignment. So, the order of precedence will not move to the wrapped object.

```
>>> order_mock.get_value.return_value = None
>>> order_mock.get_value() is None
True
```

Even if you set all three at once when initializing the mock, the order of precedence remains the same:

```
>>> order_mock = Mock(spec=Order, wraps=Order,
...                   **{"get_value.side_effect": ["first"],
...                      "get_value.return_value": "second"}
...                   )
...
>>> order_mock.get_value()
'first'
>>> order_mock.get_value.side_effect = None
>>> order_mock.get_value()
'second'
>>> order_mock.get_value.return_value = DEFAULT
>>> order_mock.get_value()
'third'
```

If **`side_effect`** is exhausted, the order of precedence will not cause a value to be obtained from the successors. Instead, `StopIteration` exception is raised.

```
>>> order_mock = Mock(spec=Order, wraps=Order)
>>> order_mock.get_value.side_effect = ["first side effect value
...                                     "another side effect val
>>> order_mock.get_value.return_value = "second"
```

```
>>> order_mock.get_value()
'first side effect value'
```

```
>>> order_mock.get_value()
'another side effect value'

>>> order_mock.get_value()
Traceback (most recent call last):
 ...
StopIteration
```

`unittest.mock` — primeiros passos

Adicionado na versão 3.3.

Usando Mock

Mock patching methods

Usos comuns para objetos `Mock` incluem:

- Métodos de aplicação de patches
- Método de gravação que invoca objetos

Talvez você queira substituir um método em um objeto para verificar se ele foi chamado com os argumentos corretos por outra parte do sistema:

```
>>> real = SomeClass()
>>> real.method = MagicMock(name='method')
>>> real.method(3, 4, 5, key='value')
<MagicMock name='method()' id='...'>
```

Depois que nosso mock for usado (`real.method` neste exemplo), ele terá métodos e atributos que permitem que você faça afirmações sobre como ele foi usado.

> **Nota**
>
> Na maioria desses exemplos, as classes `Mock` e `MagicMock` são intercambiáveis. Como a `MagicMock` é a classe mais capaz, ela faz sentido para ser usada por padrão.

Uma vez que o mock foi chamado, seu atributo `called` é definido como `True`. Mais importante, podemos usar o método `assert_called_with()` ou

`assert_called_once_with()` para verificar se ele foi chamado com os argumentos corretos.

Este exemplo testa se a chamada de `ProductionClass().method` resulta em uma chamada para o método `something`:

```
>>> class ProductionClass:
...     def method(self):
...         self.something(1, 2, 3)
...     def something(self, a, b, c):
...         pass
...
>>> real = ProductionClass()
>>> real.something = MagicMock()
>>> real.method()
>>> real.something.assert_called_once_with(1, 2, 3)
```

Mock for method calls on an object

No último exemplo, aplicamos um patch em um método diretamente em um objeto para verificar se ele foi chamado corretamente. Outro caso de uso comum é passar um objeto para um método (ou alguma parte do sistema em teste) e então verificar se ele é usado da maneira correta.

O `ProductionClass` simples abaixo tem um método `closer`. Se ele for chamado com um objeto, então ele chama `close` nele.

```
>>> class ProductionClass:
...     def closer(self, something):
...         something.close()
...
```

Então, para testá-lo, precisamos passar um objeto com um método `close` e verificar se ele foi chamado corretamente.

```
>>> real = ProductionClass()
>>> mock = Mock()
>>> real.closer(mock)
>>> mock.close.assert_called_with()
```

Não precisamos fazer nenhum trabalho para fornecer o método ‘close’ em nosso mock. Acessar close o cria. Então, se ‘close’ ainda não foi chamado, acessá-lo no teste o criará, mas `assert_called_with()` levantará uma exceção de falha.

Mocking classes

A common use case is to mock out classes instantiated by your code under test. When you patch a class, then that class is replaced with a mock. Instances are created by *calling the class*. This means you access the “mock instance” by looking at the return value of the mocked class.

In the example below we have a function `some_function` that instantiates `Foo` and calls a method on it. The call to `patch()` replaces the class `Foo` with a mock. The `Foo` instance is the result of calling the mock, so it is configured by modifying the mock `return_value`.

```
>>> def some_function():
...     instance = module.Foo()
...     return instance.method()
...
>>> with patch('module.Foo') as mock:
...     instance = mock.return_value
...     instance.method.return_value = 'the result'
...     result = some_function()
...     assert result == 'the result'
```

Nomeando os mocks

It can be useful to give your mocks a name. The name is shown in the repr of the mock and can be helpful when the mock appears in test failure messages. The name is also propagated to attributes or methods of the mock:

```
>>> mock = MagicMock(name='foo')
>>> mock
<MagicMock name='foo' id='...'>
>>> mock.method
<MagicMock name='foo.method' id='...'>
```

Tracking all calls

Frequentemente você quer rastrear mais de uma chamada para um método. O atributo **mock_calls** registra todas as chamadas para atributos filhos do mock - e também para seus filhos.

```
>>> mock = MagicMock()
>>> mock.method()
<MagicMock name='mock.method()' id='...'>
>>> mock.attribute.method(10, x=53)
<MagicMock name='mock.attribute.method()' id='...'>
>>> mock.mock_calls
[call.method(), call.attribute.method(10, x=53)]
```

If you make an assertion about `mock_calls` and any unexpected methods have been called, then the assertion will fail. This is useful because as well as asserting that the calls you expected have been made, you are also checking that they were made in the right order and with no additional calls:

You use the **call** object to construct lists for comparing with `mock_calls`:

```
>>> expected = [call.method(), call.attribute.method(10, x=53)]
>>> mock.mock_calls == expected
True
```

However, parameters to calls that return mocks are not recorded, which means it is not possible to track nested calls where the parameters used to create ancestors are important:

```
>>> m = Mock()
>>> m.factory(important=True).deliver()
<Mock name='mock.factory().deliver()' id='...'>
>>> m.mock_calls[-1] == call.factory(important=False).deliver()
True
```

Setting return values and attributes

Setting the return values on a mock object is trivially easy:

```
>>> mock = Mock()
>>> mock.return_value = 3
>>> mock()
3
```

Of course you can do the same for methods on the mock:

```
>>> mock = Mock()
>>> mock.method.return_value = 3
>>> mock.method()
3
```

The return value can also be set in the constructor:

```
>>> mock = Mock(return_value=3)
>>> mock()
3
```

If you need an attribute setting on your mock, just do it:

```
>>> mock = Mock()
>>> mock.x = 3
>>> mock.x
3
```

Sometimes you want to mock up a more complex situation, like for example `mock.connection.cursor().execute("SELECT 1")`. If we wanted this call to return a list, then we have to configure the result of the nested call.

We can use **`call`** to construct the set of calls in a “chained call” like this for easy assertion afterwards:

```
>>> mock = Mock()
>>> cursor = mock.connection.cursor.return_value
>>> cursor.execute.return_value = ['foo']
>>> mock.connection.cursor().execute("SELECT 1")
['foo']
>>> expected = call.connection.cursor().execute("SELECT 1").call
>>> mock.mock_calls
[call.connection.cursor(), call.connection.cursor().execute('SEL
```

```
>>> mock.mock_calls == expected
True
```

It is the call to `.call_list()` that turns our call object into a list of calls representing the chained calls.

Raising exceptions with mocks

A useful attribute is **`side_effect`**. If you set this to an exception class or instance then the exception will be raised when the mock is called.

```
>>> mock = Mock(side_effect=Exception('Boom!'))
>>> mock()
Traceback (most recent call last):
  ...
Exception: Boom!
```

Side effect functions and iterables

`side_effect` can also be set to a function or an iterable. The use case for `side_effect` as an iterable is where your mock is going to be called several times, and you want each call to return a different value. When you set `side_effect` to an iterable every call to the mock returns the next value from the iterable:

```
>>> mock = MagicMock(side_effect=[4, 5, 6])
>>> mock()
4
>>> mock()
5
>>> mock()
6
```

For more advanced use cases, like dynamically varying the return values depending on what the mock is called with, `side_effect` can be a function. The function will be called with the same arguments as the mock. Whatever the function returns is what the call returns:

```
>>> vals = {(1, 2): 1, (2, 3): 2}
>>> def side_effect(*args):
...     return vals[args]
...
>>> mock = MagicMock(side_effect=side_effect)
>>> mock(1, 2)
1
>>> mock(2, 3)
2
```

Mocking asynchronous iterators

Since Python 3.8, `AsyncMock` and `MagicMock` have support to mock Iteradores assíncronos through `__aiter__`. The **`return_value`** attribute of `__aiter__` can be used to set the return values to be used for iteration.

```
>>> mock = MagicMock()  # AsyncMock also works here
>>> mock.__aiter__.return_value = [1, 2, 3]
>>> async def main():
...     return [i async for i in mock]
...
>>> asyncio.run(main())
[1, 2, 3]
```

Mocking asynchronous context manager

Since Python 3.8, `AsyncMock` and `MagicMock` have support to mock Gerenciadores de contexto assíncronos through `__aenter__` and `__aexit__`. By default, `__aenter__` and `__aexit__` are `AsyncMock` instances that return an async function.

```
>>> class AsyncContextManager:
...     async def __aenter__(self):
...         return self
...     async def __aexit__(self, exc_type, exc, tb):
...         pass
...
>>> mock_instance = MagicMock(AsyncContextManager())  # AsyncMoc
>>> async def main():
...     async with mock_instance as result:
```

```
...         pass
...
>>> asyncio.run(main())
>>> mock_instance.__aenter__.assert_awaited_once()
>>> mock_instance.__aexit__.assert_awaited_once()
```

Creating a mock from an existing object

One problem with over use of mocking is that it couples your tests to the implementation of your mocks rather than your real code. Suppose you have a class that implements `some_method`. In a test for another class, you provide a mock of this object that *also* provides `some_method`. If later you refactor the first class, so that it no longer has `some_method` - then your tests will continue to pass even though your code is now broken!

`Mock` allows you to provide an object as a specification for the mock, using the *spec* keyword argument. Accessing methods / attributes on the mock that don't exist on your specification object will immediately raise an attribute error. If you change the implementation of your specification, then tests that use that class will start failing immediately without you having to instantiate the class in those tests.

```
>>> mock = Mock(spec=SomeClass)
>>> mock.old_method()
Traceback (most recent call last):
   ...
AttributeError: Mock object has no attribute 'old_method'. Did y
```

Using a specification also enables a smarter matching of calls made to the mock, regardless of whether some parameters were passed as positional or named arguments:

```
>>> def f(a, b, c): pass
...
>>> mock = Mock(spec=f)
>>> mock(1, 2, 3)
<Mock name='mock()' id='140161580456576'>
>>> mock.assert_called_with(a=1, b=2, c=3)
```

If you want this smarter matching to also work with method calls on the mock, you can use auto-speccing.

If you want a stronger form of specification that prevents the setting of arbitrary attributes as well as the getting of them then you can use *spec_set* instead of *spec*.

Using side_effect to return per file content

`mock_open()` is used to patch `open()` method. `side_effect` can be used to return a new Mock object per call. This can be used to return different contents per file stored in a dictionary:

```
DEFAULT = "default"
data_dict = {"file1": "data1",
             "file2": "data2"}

def open_side_effect(name):
    return mock_open(read_data=data_dict.get(name, DEFAULT))()

with patch("builtins.open", side_effect=open_side_effect):
    with open("file1") as file1:
        assert file1.read() == "data1"

    with open("file2") as file2:
        assert file2.read() == "data2"

    with open("file3") as file2:
        assert file2.read() == "default"
```

Patch decorators

Nota

Com `patch()`, é importante que você faça o patch de objetos no espaço de nomes onde eles são procurados. Normalmente, isso é simples, mas para um guia rápido, leia onde fazer o patch.

A common need in tests is to patch a class attribute or a module attribute, for example patching a builtin or patching a class in a module to test that it is instantiated. Modules and classes are effectively global, so patching on them has to be undone after the test or the patch will persist into other tests and cause hard to diagnose problems.

mock provides three convenient decorators for this: **`patch()`**, **`patch.object()`** and **`patch.dict()`**. `patch` takes a single string, of the form `package.module.Class.attribute` to specify the attribute you are patching. It also optionally takes a value that you want the attribute (or class or whatever) to be replaced with. 'patch.object' takes an object and the name of the attribute you would like patched, plus optionally the value to patch it with.

`patch.object`:

```
>>> original = SomeClass.attribute
>>> @patch.object(SomeClass, 'attribute', sentinel.attribute)
... def test():
...     assert SomeClass.attribute == sentinel.attribute
...
>>> test()
>>> assert SomeClass.attribute == original

>>> @patch('package.module.attribute', sentinel.attribute)
... def test():
...     from package.module import attribute
...     assert attribute is sentinel.attribute
...
>>> test()
```

If you are patching a module (including **`builtins`**) then use **`patch()`** instead of **`patch.object()`**:

```
>>> mock = MagicMock(return_value=sentinel.file_handle)
>>> with patch('builtins.open', mock):
...     handle = open('filename', 'r')
...
>>> mock.assert_called_with('filename', 'r')
>>> assert handle == sentinel.file_handle, "incorrect file handl
```

The module name can be ‘dotted’, in the form `package.module` if needed:

```
>>> @patch('package.module.ClassName.attribute', sentinel.attrib
... def test():
...     from package.module import ClassName
...     assert ClassName.attribute == sentinel.attribute
...
>>> test()
```

A nice pattern is to actually decorate test methods themselves:

```
>>> class MyTest(unittest.TestCase):
...     @patch.object(SomeClass, 'attribute', sentinel.attribute
...     def test_something(self):
...         self.assertEqual(SomeClass.attribute, sentinel.attri
...
>>> original = SomeClass.attribute
>>> MyTest('test_something').test_something()
>>> assert SomeClass.attribute == original
```

If you want to patch with a Mock, you can use `patch()` with only one argument (or `patch.object()` with two arguments). The mock will be created for you and passed into the test function / method:

```
>>> class MyTest(unittest.TestCase):
...     @patch.object(SomeClass, 'static_method')
...     def test_something(self, mock_method):
...         SomeClass.static_method()
...         mock_method.assert_called_with()
...
>>> MyTest('test_something').test_something()
```

You can stack up multiple patch decorators using this pattern:

```
>>> class MyTest(unittest.TestCase):
...     @patch('package.module.ClassName1')
...     @patch('package.module.ClassName2')
...     def test_something(self, MockClass2, MockClass1):
...         self.assertIs(package.module.ClassName1, MockClass1)
...         self.assertIs(package.module.ClassName2, MockClass2)
```

```
...
>>> MyTest('test_something').test_something()
```

When you nest patch decorators the mocks are passed in to the decorated function in the same order they applied (the normal *Python* order that decorators are applied). This means from the bottom up, so in the example above the mock for `test_module.ClassName2` is passed in first.

Também existe **`patch.dict()`** para definir valores em um dicionário apenas durante um escopo e restaurar o dicionário ao seu estado original quando o teste termina:

```
>>> foo = {'key': 'value'}
>>> original = foo.copy()
>>> with patch.dict(foo, {'newkey': 'newvalue'}, clear=True):
...     assert foo == {'newkey': 'newvalue'}
...
>>> assert foo == original
```

`patch`, `patch.object` and `patch.dict` can all be used as context managers.

Where you use **`patch()`** to create a mock for you, you can get a reference to the mock using the “as” form of the with statement:

```
>>> class ProductionClass:
...     def method(self):
...         pass
...
>>> with patch.object(ProductionClass, 'method') as mock_method:
...     mock_method.return_value = None
...     real = ProductionClass()
...     real.method(1, 2, 3)
...
>>> mock_method.assert_called_with(1, 2, 3)
```

As an alternative `patch`, `patch.object` and `patch.dict` can be used as class decorators. When used in this way it is the same as applying the decorator individually to every method whose name starts with “test”.

Further examples

Here are some more examples for some slightly more advanced scenarios.

Mocking chained calls

Mocking chained calls is actually straightforward with mock once you understand the **`return_value`** attribute. When a mock is called for the first time, or you fetch its `return_value` before it has been called, a new **`Mock`** is created.

This means that you can see how the object returned from a call to a mocked object has been used by interrogating the `return_value` mock:

```
>>> mock = Mock()
>>> mock().foo(a=2, b=3)
<Mock name='mock().foo()' id='...'>
>>> mock.return_value.foo.assert_called_with(a=2, b=3)
```

From here it is a simple step to configure and then make assertions about chained calls. Of course another alternative is writing your code in a more testable way in the first place…

So, suppose we have some code that looks a little bit like this:

```
>>> class Something:
...     def __init__(self):
...         self.backend = BackendProvider()
...     def method(self):
...         response = self.backend.get_endpoint('foobar').creat
...         # more code
```

Assuming that `BackendProvider` is already well tested, how do we test `method()`? Specifically, we want to test that the code section `# more code` uses the response object in the correct way.

As this chain of calls is made from an instance attribute we can monkey patch the `backend` attribute on a `Something` instance. In this particular case

we are only interested in the return value from the final call to `start_call` so we don't have much configuration to do. Let's assume the object it returns is 'file-like', so we'll ensure that our response object uses the builtin `open()` as its `spec`.

To do this we create a mock instance as our mock backend and create a mock response object for it. To set the response as the return value for that final `start_call` we could do this:

```
mock_backend.get_endpoint.return_value.create_call.return_value.
```

We can do that in a slightly nicer way using the `configure_mock()` method to directly set the return value for us:

```
>>> something = Something()
>>> mock_response = Mock(spec=open)
>>> mock_backend = Mock()
>>> config = {'get_endpoint.return_value.create_call.return_valu
>>> mock_backend.configure_mock(**config)
```

With these we monkey patch the "mock backend" in place and can make the real call:

```
>>> something.backend = mock_backend
>>> something.method()
```

Using `mock_calls` we can check the chained call with a single assert. A chained call is several calls in one line of code, so there will be several entries in `mock_calls`. We can use `call.call_list()` to create this list of calls for us:

```
>>> chained = call.get_endpoint('foobar').create_call('spam', 'e
>>> call_list = chained.call_list()
>>> assert mock_backend.mock_calls == call_list
```

Partial mocking

For some tests, you may want to mock out a call to **`datetime.date.today()`** to return a known date, but don't want to prevent the code under test from creating new date objects. Unfortunately **`datetime.date`** is written in C, so you cannot just monkey-patch out the static **`datetime.date.today()`** method.

Instead, you can effectively wrap the date class with a mock, while passing through calls to the constructor to the real class (and returning real instances).

The **`patch decorator`** is used here to mock out the `date` class in the module under test. The **`side_effect`** attribute on the mock date class is then set to a lambda function that returns a real date. When the mock date class is called a real date will be constructed and returned by `side_effect`.

```
>>> import datetime as dt
>>> with patch('mymodule.date') as mock_date:
...     mock_date.today.return_value = dt.date(2010, 10, 8)
...     mock_date.side_effect = lambda *args, **kw: dt.date(*arg
...
...     assert mymodule.date.today() == dt.date(2010, 10, 8)
...     assert mymodule.date(2009, 6, 8) == dt.date(2009, 6, 8)
```

Note that we don't patch **`datetime.date`** globally, we patch `date` in the module that *uses* it. See where to patch.

When `date.today()` is called a known date is returned, but calls to the `date(...)` constructor still return normal dates. Without this you can find yourself having to calculate an expected result using exactly the same algorithm as the code under test, which is a classic testing anti-pattern.

Calls to the date constructor are recorded in the `mock_date` attributes (`call_count` and friends) which may also be useful for your tests.

Uma forma alternativa de lidar com datas de mock, ou outras classes embutidas, é discutida nesta entrada de blog [https://williambert.online/2011/07/how-to-unit-testing-in-django-with-mocking-and-patching/].

Mocking a generator method

A Python generator is a function or method that uses the `yield` statement to return a series of values when iterated over [[1]].

A generator method / function is called to return the generator object. It is the generator object that is then iterated over. The protocol method for iteration is `__iter__()`, so we can mock this using a `MagicMock`.

Here's an example class with an "iter" method implemented as a generator:

```
>>> class Foo:
...     def iter(self):
...         for i in [1, 2, 3]:
...             yield i
...
>>> foo = Foo()
>>> list(foo.iter())
[1, 2, 3]
```

How would we mock this class, and in particular its "iter" method?

To configure the values returned from the iteration (implicit in the call to `list`), we need to configure the object returned by the call to `foo.iter()`.

```
>>> mock_foo = MagicMock()
>>> mock_foo.iter.return_value = iter([1, 2, 3])
>>> list(mock_foo.iter())
[1, 2, 3]
```

[1]

There are also generator expressions and more advanced uses [http://www.dabeaz.com/coroutines/index.html] of generators, but we aren't concerned about them here. A very good introduction to generators and how powerful they are is: Generator Tricks for Systems Programmers [http://www.dabeaz.com/generators/].

Applying the same patch to every test method

If you want several patches in place for multiple test methods the obvious way is to apply the patch decorators to every method. This can feel like unnecessary repetition. Instead, you can use **`patch()`** (in all its various forms) as a class decorator. This applies the patches to all test methods on the class. A test method is identified by methods whose names start with `test`:

```
>>> @patch('mymodule.SomeClass')
... class MyTest(unittest.TestCase):
...
...     def test_one(self, MockSomeClass):
...         self.assertIs(mymodule.SomeClass, MockSomeClass)
...
...     def test_two(self, MockSomeClass):
...         self.assertIs(mymodule.SomeClass, MockSomeClass)
...
...     def not_a_test(self):
...         return 'something'
...
>>> MyTest('test_one').test_one()
>>> MyTest('test_two').test_two()
>>> MyTest('test_two').not_a_test()
'something'
```

An alternative way of managing patches is to use the métodos do patch: start e stop. These allow you to move the patching into your `setUp` and `tearDown` methods.

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         self.patcher = patch('mymodule.foo')
...         self.mock_foo = self.patcher.start()
...
...     def test_foo(self):
...         self.assertIs(mymodule.foo, self.mock_foo)
...
...     def tearDown(self):
...         self.patcher.stop()
...
>>> MyTest('test_foo').run()
```

If you use this technique you must ensure that the patching is “undone” by calling `stop`. This can be fiddlier than you might think, because if an exception is raised in the setUp then tearDown is not called. `unittest.TestCase.addCleanup()` makes this easier:

```
>>> class MyTest(unittest.TestCase):
...     def setUp(self):
...         patcher = patch('mymodule.foo')
...         self.addCleanup(patcher.stop)
...         self.mock_foo = patcher.start()
...
...     def test_foo(self):
...         self.assertIs(mymodule.foo, self.mock_foo)
...
>>> MyTest('test_foo').run()
```

Mocking unbound methods

Sometimes a test needs to patch an *unbound method*, which means patching the method on the class rather than on the instance. In order to make assertions about which objects were calling this particular method, you need to pass `self` as the first argument. The issue is that you can’t patch with a mock for this, because if you replace an unbound method with a mock it doesn’t become a bound method when fetched from the instance, and so it doesn’t get `self` passed in. The workaround is to patch the unbound method with a real function instead. The `patch()` decorator makes it so simple to patch out methods with a mock that having to create a real function becomes a nuisance.

If you pass `autospec=True` to patch then it does the patching with a *real* function object. This function object has the same signature as the one it is replacing, but delegates to a mock under the hood. You still get your mock auto-created in exactly the same way as before. What it means though, is that if you use it to patch out an unbound method on a class the mocked function will be turned into a bound method if it is fetched from an instance. It will have `self` passed in as the first argument, which is exactly what was needed:

```
>>> class Foo:
...   def foo(self):
...     pass
...
>>> with patch.object(Foo, 'foo', autospec=True) as mock_foo:
...   mock_foo.return_value = 'foo'
...   foo = Foo()
...   foo.foo()
...
'foo'
>>> mock_foo.assert_called_once_with(foo)
```

If we don't use `autospec=True` then the unbound method is patched out with a Mock instance instead, and isn't called with `self`.

Checking multiple calls with mock

mock has a nice API for making assertions about how your mock objects are used.

```
>>> mock = Mock()
>>> mock.foo_bar.return_value = None
>>> mock.foo_bar('baz', spam='eggs')
>>> mock.foo_bar.assert_called_with('baz', spam='eggs')
```

If your mock is only being called once you can use the **`assert_called_once_with()`** method that also asserts that the **`call_count`** is one.

```
>>> mock.foo_bar.assert_called_once_with('baz', spam='eggs')
>>> mock.foo_bar()
>>> mock.foo_bar.assert_called_once_with('baz', spam='eggs')
Traceback (most recent call last):
    ...
AssertionError: Expected 'foo_bar' to be called once. Called 2 t
Calls: [call('baz', spam='eggs'), call()].
```

Both `assert_called_with` and `assert_called_once_with` make assertions about the *most recent* call. If your mock is going to be called several times,

and you want to make assertions about *all* those calls you can use **`call_args_list`**:

```
>>> mock = Mock(return_value=None)
>>> mock(1, 2, 3)
>>> mock(4, 5, 6)
>>> mock()
>>> mock.call_args_list
[call(1, 2, 3), call(4, 5, 6), call()]
```

The **`call`** helper makes it easy to make assertions about these calls. You can build up a list of expected calls and compare it to `call_args_list`. This looks remarkably similar to the repr of the `call_args_list`:

```
>>> expected = [call(1, 2, 3), call(4, 5, 6), call()]
>>> mock.call_args_list == expected
True
```

Coping with mutable arguments

Another situation is rare, but can bite you, is when your mock is called with mutable arguments. `call_args` and `call_args_list` store *references* to the arguments. If the arguments are mutated by the code under test then you can no longer make assertions about what the values were when the mock was called.

Here's some example code that shows the problem. Imagine the following functions defined in 'mymodule':

```
def frob(val):
    pass

def grob(val):
    "First frob and then clear val"
    frob(val)
    val.clear()
```

When we try to test that `grob` calls `frob` with the correct argument look what happens:

```
>>> with patch('mymodule.frob') as mock_frob:
...     val = {6}
...     mymodule.grob(val)
...
>>> val
set()
>>> mock_frob.assert_called_with({6})
Traceback (most recent call last):
    ...
AssertionError: Expected: (({6},), {})
Called with: ((set(),), {})
```

One possibility would be for mock to copy the arguments you pass in. This could then cause problems if you do assertions that rely on object identity for equality.

Here's one solution that uses the **`side_effect`** functionality. If you provide a `side_effect` function for a mock then `side_effect` will be called with the same args as the mock. This gives us an opportunity to copy the arguments and store them for later assertions. In this example we're using *another* mock to store the arguments so that we can use the mock methods for doing the assertion. Again a helper function sets this up for us.

```
>>> from copy import deepcopy
>>> from unittest.mock import Mock, patch, DEFAULT
>>> def copy_call_args(mock):
...     new_mock = Mock()
...     def side_effect(*args, **kwargs):
...         args = deepcopy(args)
...         kwargs = deepcopy(kwargs)
...         new_mock(*args, **kwargs)
...         return DEFAULT
...     mock.side_effect = side_effect
...     return new_mock
...
>>> with patch('mymodule.frob') as mock_frob:
...     new_mock = copy_call_args(mock_frob)
...     val = {6}
...     mymodule.grob(val)
...
>>> new_mock.assert_called_with({6})
```

```
>>> new_mock.call_args
call({6})
```

`copy_call_args` is called with the mock that will be called. It returns a new mock that we do the assertion on. The `side_effect` function makes a copy of the args and calls our `new_mock` with the copy.

Nota

If your mock is only going to be used once there is an easier way of checking arguments at the point they are called. You can simply do the checking inside a `side_effect` function.

```
>>> def side_effect(arg):
...     assert arg == {6}
...
>>> mock = Mock(side_effect=side_effect)
>>> mock({6})
>>> mock(set())
Traceback (most recent call last):
    ...
AssertionError
```

An alternative approach is to create a subclass of **`Mock`** or **`MagicMock`** that copies (using **`copy.deepcopy()`**) the arguments. Here's an example implementation:

```
>>> from copy import deepcopy
>>> class CopyingMock(MagicMock):
...     def __call__(self, /, *args, **kwargs):
...         args = deepcopy(args)
...         kwargs = deepcopy(kwargs)
...         return super().__call__(*args, **kwargs)
...
>>> c = CopyingMock(return_value=None)
>>> arg = set()
>>> c(arg)
>>> arg.add(1)
>>> c.assert_called_with(set())
>>> c.assert_called_with(arg)
```

```
Traceback (most recent call last):
    ...
AssertionError: expected call not found.
Expected: mock({1})
Actual: mock(set())
>>> c.foo
<CopyingMock name='mock.foo' id='...'>
```

When you subclass `Mock` or `MagicMock` all dynamically created attributes, and the `return_value` will use your subclass automatically. That means all children of a `CopyingMock` will also have the type `CopyingMock`.

Nesting patches

Using patch as a context manager is nice, but if you do multiple patches you can end up with nested with statements indenting further and further to the right:

```
>>> class MyTest(unittest.TestCase):
...
...     def test_foo(self):
...         with patch('mymodule.Foo') as mock_foo:
...             with patch('mymodule.Bar') as mock_bar:
...                 with patch('mymodule.Spam') as mock_spam:
...                     assert mymodule.Foo is mock_foo
...                     assert mymodule.Bar is mock_bar
...                     assert mymodule.Spam is mock_spam
...
>>> original = mymodule.Foo
>>> MyTest('test_foo').test_foo()
>>> assert mymodule.Foo is original
```

With unittest `cleanup` functions and the métodos do patch: start e stop we can achieve the same effect without the nested indentation. A simple helper method, `create_patch`, puts the patch in place and returns the created mock for us:

```
>>> class MyTest(unittest.TestCase):
...
...     def create_patch(self, name):
...         patcher = patch(name)
```

```
...         thing = patcher.start()
...         self.addCleanup(patcher.stop)
...         return thing
...
...     def test_foo(self):
...         mock_foo = self.create_patch('mymodule.Foo')
...         mock_bar = self.create_patch('mymodule.Bar')
...         mock_spam = self.create_patch('mymodule.Spam')
...
...         assert mymodule.Foo is mock_foo
...         assert mymodule.Bar is mock_bar
...         assert mymodule.Spam is mock_spam
...
>>> original = mymodule.Foo
>>> MyTest('test_foo').run()
>>> assert mymodule.Foo is original
```

Mocking a dictionary with MagicMock

You may want to mock a dictionary, or other container object, recording all access to it whilst having it still behave like a dictionary.

We can do this with `MagicMock`, which will behave like a dictionary, and using `side_effect` to delegate dictionary access to a real underlying dictionary that is under our control.

When the `__getitem__()` and `__setitem__()` methods of our `MagicMock` are called (normal dictionary access) then `side_effect` is called with the key (and in the case of `__setitem__` the value too). We can also control what is returned.

After the `MagicMock` has been used we can use attributes like `call_args_list` to assert about how the dictionary was used:

```
>>> my_dict = {'a': 1, 'b': 2, 'c': 3}
>>> def getitem(name):
...      return my_dict[name]
...
>>> def setitem(name, val):
...     my_dict[name] = val
```

```
...
>>> mock = MagicMock()
>>> mock.__getitem__.side_effect = getitem
>>> mock.__setitem__.side_effect = setitem
```

Nota

An alternative to using `MagicMock` is to use `Mock` and *only* provide the magic methods you specifically want:

```
>>> mock = Mock()
>>> mock.__getitem__ = Mock(side_effect=getitem)
>>> mock.__setitem__ = Mock(side_effect=setitem)
```

A *third* option is to use `MagicMock` but passing in `dict` as the *spec* (or *spec_set*) argument so that the `MagicMock` created only has dictionary magic methods available:

```
>>> mock = MagicMock(spec_set=dict)
>>> mock.__getitem__.side_effect = getitem
>>> mock.__setitem__.side_effect = setitem
```

With these side effect functions in place, the `mock` will behave like a normal dictionary but recording the access. It even raises a **`KeyError`** if you try to access a key that doesn't exist.

```
>>> mock['a']
1
>>> mock['c']
3
>>> mock['d']
Traceback (most recent call last):
    ...
KeyError: 'd'
>>> mock['b'] = 'fish'
>>> mock['d'] = 'eggs'
>>> mock['b']
'fish'
>>> mock['d']
'eggs'
```

After it has been used you can make assertions about the access using the normal mock methods and attributes:

```
>>> mock.__getitem__.call_args_list
[call('a'), call('c'), call('d'), call('b'), call('d')]
>>> mock.__setitem__.call_args_list
[call('b', 'fish'), call('d', 'eggs')]
>>> my_dict
{'a': 1, 'b': 'fish', 'c': 3, 'd': 'eggs'}
```

Mock subclasses and their attributes

There are various reasons why you might want to subclass **`Mock`**. One reason might be to add helper methods. Here's a silly example:

```
>>> class MyMock(MagicMock):
...     def has_been_called(self):
...         return self.called
...
>>> mymock = MyMock(return_value=None)
>>> mymock
<MyMock id='...'>
>>> mymock.has_been_called()
False
>>> mymock()
>>> mymock.has_been_called()
True
```

The standard behaviour for `Mock` instances is that attributes and the return value mocks are of the same type as the mock they are accessed on. This ensures that `Mock` attributes are `Mocks` and `MagicMock` attributes are `MagicMocks` [[2]]. So if you're subclassing to add helper methods then they'll also be available on the attributes and return value mock of instances of your subclass.

```
>>> mymock.foo
<MyMock name='mock.foo' id='...'>
>>> mymock.foo.has_been_called()
False
>>> mymock.foo()
<MyMock name='mock.foo()' id='...'>
```

```
>>> mymock.foo.has_been_called()
True
```

Sometimes this is inconvenient. For example, one user [https://code.google.com/archive/p/mock/issues/105] is subclassing mock to created a Twisted adaptor [https://twisted.org/documents/11.0.0/api/twisted.python.components.html]. Having this applied to attributes too actually causes errors.

`Mock` (in all its flavours) uses a method called `_get_child_mock` to create these "sub-mocks" for attributes and return values. You can prevent your subclass being used for attributes by overriding this method. The signature is that it takes arbitrary keyword arguments (`**kwargs`) which are then passed onto the mock constructor:

```
>>> class Subclass(MagicMock):
...     def _get_child_mock(self, /, **kwargs):
...         return MagicMock(**kwargs)
...
>>> mymock = Subclass()
>>> mymock.foo
<MagicMock name='mock.foo' id='...'>
>>> assert isinstance(mymock, Subclass)
>>> assert not isinstance(mymock.foo, Subclass)
>>> assert not isinstance(mymock(), Subclass)
```

[2]

An exception to this rule are the non-callable mocks. Attributes use the callable variant because otherwise non-callable mocks couldn't have callable methods.

Mocking imports with patch.dict

One situation where mocking can be hard is where you have a local import inside a function. These are harder to mock because they aren't using an object from the module namespace that we can patch out.

Generally local imports are to be avoided. They are sometimes done to prevent circular dependencies, for which there is *usually* a much better way

to solve the problem (refactor the code) or to prevent “up front costs” by delaying the import. This can also be solved in better ways than an unconditional local import (store the module as a class or module attribute and only do the import on first use).

That aside there is a way to use `mock` to affect the results of an import. Importing fetches an *object* from the **`sys.modules`** dictionary. Note that it fetches an *object*, which need not be a module. Importing a module for the first time results in a module object being put in `sys.modules`, so usually when you import something you get a module back. This need not be the case however.

This means you can use **`patch.dict()`** to *temporarily* put a mock in place in **`sys.modules`**. Any imports whilst this patch is active will fetch the mock. When the patch is complete (the decorated function exits, the with statement body is complete or `patcher.stop()` is called) then whatever was there previously will be restored safely.

Here’s an example that mocks out the ‘fooble’ module.

```
>>> import sys
>>> mock = Mock()
>>> with patch.dict('sys.modules', {'fooble': mock}):
...     import fooble
...     fooble.blob()
...
<Mock name='mock.blob()' id='...'>
>>> assert 'fooble' not in sys.modules
>>> mock.blob.assert_called_once_with()
```

As you can see the `import fooble` succeeds, but on exit there is no ‘fooble’ left in **`sys.modules`**.

This also works for the `from module import name` form:

```
>>> mock = Mock()
>>> with patch.dict('sys.modules', {'fooble': mock}):
...     from fooble import blob
...     blob.blip()
...
```

```
<Mock name='mock.blob.blip()' id='...'>
>>> mock.blob.blip.assert_called_once_with()
```

With slightly more work you can also mock package imports:

```
>>> mock = Mock()
>>> modules = {'package': mock, 'package.module': mock.module}
>>> with patch.dict('sys.modules', modules):
...     from package.module import fooble
...     fooble()
...
<Mock name='mock.module.fooble()' id='...'>
>>> mock.module.fooble.assert_called_once_with()
```

Tracking order of calls and less verbose call assertions

The `Mock` class allows you to track the *order* of method calls on your mock objects through the `method_calls` attribute. This doesn't allow you to track the order of calls between separate mock objects, however we can use `mock_calls` to achieve the same effect.

Because mocks track calls to child mocks in `mock_calls`, and accessing an arbitrary attribute of a mock creates a child mock, we can create our separate mocks from a parent one. Calls to those child mock will then all be recorded, in order, in the `mock_calls` of the parent:

```
>>> manager = Mock()
>>> mock_foo = manager.foo
>>> mock_bar = manager.bar

>>> mock_foo.something()
<Mock name='mock.foo.something()' id='...'>
>>> mock_bar.other.thing()
<Mock name='mock.bar.other.thing()' id='...'>

>>> manager.mock_calls
[call.foo.something(), call.bar.other.thing()]
```

We can then assert about the calls, including the order, by comparing with the `mock_calls` attribute on the manager mock:

```
>>> expected_calls = [call.foo.something(), call.bar.other.thing
>>> manager.mock_calls == expected_calls
True
```

If `patch` is creating, and putting in place, your mocks then you can attach them to a manager mock using the **`attach_mock()`** method. After attaching calls will be recorded in `mock_calls` of the manager.

```
>>> manager = MagicMock()
>>> with patch('mymodule.Class1') as MockClass1:
...     with patch('mymodule.Class2') as MockClass2:
...         manager.attach_mock(MockClass1, 'MockClass1')
...         manager.attach_mock(MockClass2, 'MockClass2')
...         MockClass1().foo()
...         MockClass2().bar()
<MagicMock name='mock.MockClass1().foo()' id='...'>
<MagicMock name='mock.MockClass2().bar()' id='...'>
>>> manager.mock_calls
[call.MockClass1(),
call.MockClass1().foo(),
call.MockClass2(),
call.MockClass2().bar()]
```

If many calls have been made, but you're only interested in a particular sequence of them then an alternative is to use the **`assert_has_calls()`** method. This takes a list of calls (constructed with the **`call`** object). If that sequence of calls are in **`mock_calls`** then the assert succeeds.

```
>>> m = MagicMock()
>>> m().foo().bar().baz()
<MagicMock name='mock().foo().bar().baz()' id='...'>
>>> m.one().two().three()
<MagicMock name='mock.one().two().three()' id='...'>
>>> calls = call.one().two().three().call_list()
>>> m.assert_has_calls(calls)
```

Even though the chained call `m.one().two().three()` aren't the only calls that have been made to the mock, the assert still succeeds.

Sometimes a mock may have several calls made to it, and you are only interested in asserting about *some* of those calls. You may not even care about the order. In this case you can pass `any_order=True` to `assert_has_calls`:

```
>>> m = MagicMock()
>>> m(1), m.two(2, 3), m.seven(7), m.fifty('50')
(...)
>>> calls = [call.fifty('50'), call(1), call.seven(7)]
>>> m.assert_has_calls(calls, any_order=True)
```

More complex argument matching

Using the same basic concept as **`ANY`** we can implement matchers to do more complex assertions on objects used as arguments to mocks.

Suppose we expect some object to be passed to a mock that by default compares equal based on object identity (which is the Python default for user defined classes). To use **`assert_called_with()`** we would need to pass in the exact same object. If we are only interested in some of the attributes of this object then we can create a matcher that will check these attributes for us.

You can see in this example how a 'standard' call to `assert_called_with` isn't sufficient:

```
>>> class Foo:
...     def __init__(self, a, b):
...         self.a, self.b = a, b
...
>>> mock = Mock(return_value=None)
>>> mock(Foo(1, 2))
>>> mock.assert_called_with(Foo(1, 2))
Traceback (most recent call last):
    ...
AssertionError: expected call not found.
Expected: mock(<__main__.Foo object at 0x...>)
Actual: mock(<__main__.Foo object at 0x...>)
```

A comparison function for our `Foo` class might look something like this:

```
>>> def compare(self, other):
...     if not type(self) == type(other):
...         return False
...     if self.a != other.a:
...         return False
...     if self.b != other.b:
...         return False
...     return True
...
```

And a matcher object that can use comparison functions like this for its equality operation would look something like this:

```
>>> class Matcher:
...     def __init__(self, compare, some_obj):
...         self.compare = compare
...         self.some_obj = some_obj
...     def __eq__(self, other):
...         return self.compare(self.some_obj, other)
...
```

Putting all this together:

```
>>> match_foo = Matcher(compare, Foo(1, 2))
>>> mock.assert_called_with(match_foo)
```

The `Matcher` is instantiated with our compare function and the `Foo` object we want to compare against. In `assert_called_with` the `Matcher` equality method will be called, which compares the object the mock was called with against the one we created our matcher with. If they match then `assert_called_with` passes, and if they don't an **`AssertionError`** is raised:

```
>>> match_wrong = Matcher(compare, Foo(3, 4))
>>> mock.assert_called_with(match_wrong)
Traceback (most recent call last):
    ...
AssertionError: Expected: ((<Matcher object at 0x...>,), {})
Called with: ((<Foo object at 0x...>,), {})
```

With a bit of tweaking you could have the comparison function raise the `AssertionError` directly and provide a more useful failure message.

As of version 1.5, the Python testing library PyHamcrest [https://pyhamcrest.readthedocs.io/] provides similar functionality, that may be useful here, in the form of its equality matcher (hamcrest.library.integration.match_equality [https://pyhamcrest.readthedocs.io/en/release-1.8/integration/#module-hamcrest.library.integration.match_equality]).

`test` — Pacote de Testes de Regressão do Python

Nota

The **`test`** package is meant for internal use by Python only. It is documented for the benefit of the core developers of Python. Any use of this package outside of Python's standard library is discouraged as code mentioned here can change or be removed without notice between releases of Python.

The **`test`** package contains all regression tests for Python as well as the modules **`test.support`** and **`test.regrtest`**. **`test.support`** is used to enhance your tests while **`test.regrtest`** drives the testing suite.

Each module in the **`test`** package whose name starts with `test_` is a testing suite for a specific module or feature. All new tests should be written using the **`unittest`** or **`doctest`** module. Some older tests are written using a "traditional" testing style that compares output printed to `sys.stdout`; this style of test is considered deprecated.

Ver também

Módulo `unittest`
Escrevendo testes de regressão PyUnit.

Módulo `doctest`
Testes embutidos em Strings da documentação.

Writing Unit Tests for the `test` package

É preferível que os testes que usam o módulo **unittest** sigam algumas diretrizes. Uma é nomear o módulo de teste iniciando-o com `test_` e termine com o nome do módulo que está sendo testado. Os métodos de teste no módulo de teste deve começar com `test_` e terminar com uma descrição do que o método está testando. Isso é necessário para que os métodos sejam reconhecidos pelo driver de teste como métodos de teste. Além disso, na string de documentação para o método deve ser incluído. Um comentário (como os `# Tests function returns only True or False`) deve ser usado para fornecer documentação para testar métodos. Isso é feito porque as strings de documentação são impressas se existem e, portanto, qual teste está sendo executado não é indicado.

Um boilerplate básico é muitas vezes usado:

```
import unittest
from test import support

class MyTestCase1(unittest.TestCase):

    # Só use setUp() e tearDown() se necessário

    def setUp(self):
        ... código para executar na preparação de testes ...

    def tearDown(self):
        ... código para executar a limpeza após os testes ...

    def test_feature_one(self):
        # Testa o recurso um.
        ... código de teste ...

    def test_feature_two(self):
        # Testa o recurso dois.
        ... código de teste ...

    ... mais métodos de teste ...

class MyTestCase2(unittest.TestCase):
    ... mesma estrutura que MyTestCase1 ...

... mais classes de teste ...
```

```
if __name__ == '__main__':
    unittest.main()
```

Este padrão de código permite que o conjunto de testes seja executado pelo **`test.regrtest`**, por conta própria, como um script que suporte o **`unittest`** CLI, ou através do `python -m unittest` CLI.

O objetivo do teste de regressão é tentar quebrar o código. Isso leva a algumas diretrizes que devemos seguir:

- O conjunto de testes deve exercitar todas as classes, funções e constantes. Isso inclui não apenas a API externa que deve ser apresentada ao mundo exterior, mas também o código "privado".

- Os testes de Whitebox (que examinam o código que está sendo testado quando os testes estão sendo gravados) são preferidos. O teste Blackbox (que testa apenas a interface do público de usuário) não é completo o suficiente para garantir que todos os casos de limite e extremos sejam testados.

- Certifique-se de que todos os valores possíveis sejam testados, incluindo os inválidos. Isso garante que não apenas todos os valores válidos são aceitos, mas também, que os valores impróprios são tratados corretamente.

- Esgote o maior número possível de caminhos de código. Teste onde ocorre a ramificação e, assim, personalize a entrada para garantir que tantos caminhos diferentes pelo código sejam tomados.

- Adicione um teste explícito para quaisquer bugs descobertos ao código testado. Isso garantirá que o erro não apareça novamente se o código for alterado no futuro.

- Certifique-se de limpar após seus testes (como fechar e remover todos os arquivos temporários).

- Se um teste depende de uma condição específica do sistema operacional, então verifica se a condição já existe antes de tentar o

teste.

- Importa o menor número de módulos possível e faça isso o mais rápido possível. Isso minimiza dependências externas de testes, e também minimiza possíveis comportamento anômalo dos efeitos colaterais da importação de um módulo.

- Tente maximizar a reutilização de código. Ocasionalmente, os testes variam em algo tão pequeno quanto o tipo de entrada é usado. Minimize a duplicação de código criando uma subclasse básica de testes com uma classe que especifica o input:

```
class TestFuncAcceptsSequencesMixin:

    func = mySuperWhammyFunction

    def test_func(self):
        self.func(self.arg)

class AcceptLists(TestFuncAcceptsSequencesMixin, unittest.Te
    arg = [1, 2, 3]

class AcceptStrings(TestFuncAcceptsSequencesMixin, unittest.
    arg = 'abc'

class AcceptTuples(TestFuncAcceptsSequencesMixin, unittest.T
    arg = (1, 2, 3)
```

  Ao usar este padrão, lembre-se que todas as classes que herdam de `unittest.TestCase` são executadas como testes. A classe **`TestFuncAcceptsSequencesMixin`** no exemplo acima não tem nenhum dado e, portanto, não pode ser executada sozinha, portanto, ela não herda de **`unittest.TestCase`**.

Ver também

Test Driven Development

Um livro de Kent Beck sobre escrita de testes antes do código.

Executando testes usando a interface de linha de comando

The **`test`** package can be run as a script to drive Python's regression test suite, thanks to the `-m` option: **python -m test**. Under the hood, it uses **`test.regrtest`**; the call **python -m test.regrtest** used in previous Python versions still works. Running the script by itself automatically starts running all regression tests in the **`test`** package. It does this by finding all modules in the package whose name starts with `test_`, importing them, and executing the function **`test_main()`** if present or loading the tests via unittest.TestLoader.loadTestsFromModule if `test_main` does not exist. The names of tests to execute may also be passed to the script. Specifying a single regression test (**python -m test test_spam**) will minimize output and only print whether the test passed or failed.

Running **`test`** directly allows what resources are available for tests to use to be set. You do this by using the `-u` command-line option. Specifying `all` as the value for the `-u` option enables all possible resources: **python -m test -uall**. If all but one resource is desired (a more common case), a comma-separated list of resources that are not desired may be listed after `all`. The command **python -m test -uall,-audio,-largefile** will run **`test`** with all resources except the `audio` and `largefile` resources. For a list of all resources and more command-line options, run **python -m test -h**.

Alguns outros meios para executar os testes de regressão dependem em qual plataforma os testes estão sendo executados. No Unix, você pode executar: programa: ' make test' no diretório de mais alto nível onde o Python foi construído. No Windows, executar: programa 'rt.bat' do seu diretório: file: 'PCbuild' executará todos os testes de regressão.

Adicionado na versão 3.14: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

`test.support` — Utilities for the Python test suite

The **`test.support`** module provides support for Python's regression test suite.

> **Nota**
>
> **`test.support`** is not a public module. It is documented here to help Python developers write tests. The API of this module is subject to change without backwards compatibility concerns between releases.

Esse módulo define as seguintes exceções:

exception test.support.TestFailed

Exceção a ser levantada quando um teste falha. Isto foi descontinuado em favor dos testes baseados em **`unittest`** e métodos de asserção de **`unittest.TestCase`**.

exception test.support.ResourceDenied

Subclasse de **`unittest.SkipTest`**. Levantada quando um recurso (como uma conexão de rede) não está disponível. Levantada pela função **`requires()`**.

The **`test.support`** module defines the following constants:

test.support.verbose

`True` quando a saída detalhada está habilitada. Deve ser verificado quando informações mais detalhadas são desejadas sobre um teste em execução. *verbose* é definido por **`test.regrtest`**.

test.support.is_jython

`True` se o interpretador em execução for Jython.

test.support.is_android

`True` if `sys.platform` is `android`.

test.support.is_emscripten

`True` if `sys.platform` is `emscripten`.

test.support.is_wasi

`True` if `sys.platform` is `wasi`.

test.support.is_apple_mobile

`True` if `sys.platform` is `ios`, `tvos`, or `watchos`.

test.support.is_apple

`True` if `sys.platform` is `darwin` or `is_apple_mobile` is `True`.

test.support.unix_shell

Caminho para o console se não estiver no Windows; por outro lado `None`

test.support.LOOPBACK_TIMEOUT

Tempo limite em segundos para testes usando um servidor de rede escutando na interface de loopback local da rede como `127.0.0.1`.

O tempo limite é longo o suficiente para evitar a falha do teste: leva em consideração que o cliente e o servidor podem ser executados em diferentes threads ou mesmo processos diferentes.

O tempo limite deve ser longo o suficiente para os métodos **`connect()`**, **`recv()`** e **`send()`** de **`socket.socket`**.

Its default value is 10 seconds.

Veja também **`INTERNET_TIMEOUT`**.

test.support.INTERNET_TIMEOUT

Tempo limite em segundos para solicitações de rede indo para a Internet.

O tempo limite é curto o suficiente para evitar que um teste espere muito tempo se a requisição da Internet for bloqueada por qualquer motivo.

Normalmente, um tempo limite usando **INTERNET_TIMEOUT** não deve marcar um teste como falhado, mas ignorá-lo: veja **transient_internet()**.

Seu valor padrão é 1 minuto.

Veja também **LOOPBACK_TIMEOUT**.

test.support.SHORT_TIMEOUT

Tempo limite em segundos para marcar um teste como falho se o teste demorar “muito tempo”.

O valor do tempo limite depende da opção da linha de comando regrtest `--timeout`.

Se um teste usando **SHORT_TIMEOUT** começar a falhar aleatoriamente em buildbots lentos, use **LONG_TIMEOUT**.

Seu valor padrão é 30 segundos.

test.support.LONG_TIMEOUT

Tempo limite em segundos para detectar quando um teste trava.

É longo o suficiente para reduzir o risco de falha de teste nos buildbots Python mais lentos. Não deve ser usado para marcar um teste como reprovado se o teste demorar “muito tempo”. O valor do tempo limite depende da opção de linha de comando regrtest `--timeout`.

Seu valor padrão é 5 minutos.

Veja também **LOOPBACK_TIMEOUT**, **INTERNET_TIMEOUT** e **SHORT_TIMEOUT**.

test.support.PGO

Definido quando os testes podem ser ignorados quando não são úteis para PGO.

test.support.PIPE_MAX_SIZE

Uma constante que provavelmente é maior que o tamanho do buffer de canal do sistema operacional subjacente, para fazer o bloqueio de escritas.

test.support.Py_DEBUG

`True` se o Python foi criado com a macro **`Py_DEBUG`** definida, ou seja, se o Python foi criado no modo de depuração.

Adicionado na versão 3.12.

test.support.SOCK_MAX_SIZE

Uma constante que provavelmente é maior que o tamanho do buffer de soquete do sistema operacional subjacente, para fazer o bloqueio de escritas.

test.support.TEST_SUPPORT_DIR

Set to the top level directory that contains **`test.support`**.

test.support.TEST_HOME_DIR

Define o diretório de mais alto nível para o pacote de teste.

test.support.TEST_DATA_DIR

Define o diretório `data` de dentro do pacote de teste.

test.support.MAX_Py_ssize_t

Define **`sys.maxsize`** para grandes testes de memória.

test.support.max_memuse

Set by **`set_memlimit()`** as the memory limit for big memory tests. Limited by **`MAX_Py_ssize_t`**.

test.support.real_max_memuse

Set by `set_memlimit()` as the memory limit for big memory tests. Not limited by `MAX_Py_ssize_t`.

test.support.MISSING_C_DOCSTRINGS

Set to `True` if Python is built without docstrings (the **`WITH_DOC_STRINGS`** macro is not defined). See the `configure --without-doc-strings` option.

See also the `HAVE_DOCSTRINGS` variable.

test.support.HAVE_DOCSTRINGS

Set to `True` if function docstrings are available. See the `python -OO` option, which strips docstrings of functions implemented in Python.

See also the `MISSING_C_DOCSTRINGS` variable.

test.support.TEST_HTTP_URL

Define the URL of a dedicated HTTP server for the network tests.

test.support.ALWAYS_EQ

Object that is equal to anything. Used to test mixed type comparison.

test.support.NEVER_EQ

Object that is not equal to anything (even to `ALWAYS_EQ`). Used to test mixed type comparison.

test.support.LARGEST

Object that is greater than anything (except itself). Used to test mixed type comparison.

test.support.SMALLEST

Object that is less than anything (except itself). Used to test mixed type comparison.

The **`test.support`** module defines the following functions:

test.support.busy_retry(*timeout*, *err_msg=None*, /, *, *error=True*)

Run the loop body until `break` stops the loop.

After *timeout* seconds, raise an **`AssertionError`** if *error* is true, or just stop the loop if *error* is false.

Exemplo:

```
for _ in support.busy_retry(support.SHORT_TIMEOUT):
    if check():
        break
```

Example of error=False usage:

```
for _ in support.busy_retry(support.SHORT_TIMEOUT, error=Fals
    if check():
        break
else:
    raise RuntimeError('my custom error')
```

test.support.sleeping_retry(*timeout*, *err_msg=None*, /, *, *init_delay=0.010*, *max_delay=1.0*, *error=True*)

Wait strategy that applies exponential backoff.

Run the loop body until `break` stops the loop. Sleep at each loop iteration, but not at the first iteration. The sleep delay is doubled at each iteration (up to *max_delay* seconds).

See **`busy_retry()`** documentation for the parameters usage.

Example raising an exception after SHORT_TIMEOUT seconds:

```
for _ in support.sleeping_retry(support.SHORT_TIMEOUT):
    if check():
        break
```

Example of error=False usage:

```
for _ in support.sleeping_retry(support.SHORT_TIMEOUT, error=
    if check():
        break
```

```
else:
    raise RuntimeError('my custom error')
```

test.support.is_resource_enabled(*resource*)

Return `True` if *resource* is enabled and available. The list of available resources is only set when **`test.regrtest`** is executing the tests.

test.support.get_resource_value(*resource*)

Return the value specified for *resource* (as `-u resource=value`). Return `None` if *resource* is disabled or no value is specified.

test.support.python_is_optimized()

Return `True` if Python was not built with `-O0` or `-Og`.

test.support.with_pymalloc()

Return **`_testcapi.WITH_PYMALLOC`**.

test.support.requires(*resource*, *msg=None*)

Raise **`ResourceDenied`** if *resource* is not available. *msg* is the argument to **`ResourceDenied`** if it is raised. Always returns `True` if called by a function whose `__name__` is `'__main__'`. Used when tests are executed by **`test.regrtest`**.

test.support.sortdict(*dict*)

Return a repr of *dict* with keys sorted.

test.support.findfile(*filename*, *subdir=None*)

Return the path to the file named *filename*. If no match is found *filename* is returned. This does not equal a failure since it could be the path to the file.

Setting *subdir* indicates a relative path to use to find the file rather than looking directly in the path directories.

test.support.get_pagesize()

Get size of a page in bytes.

Adicionado na versão 3.12.

test.support.setswitchinterval(*interval*)

Set the **sys.setswitchinterval()** to the given *interval*. Defines a minimum interval for Android systems to prevent the system from hanging.

test.support.check_impl_detail(**guards*)

Use this check to guard CPython's implementation-specific tests or to run them only on the implementations guarded by the arguments. This function returns `True` or `False` depending on the host platform. Example usage:

```
check_impl_detail()               # Only on CPython (default)
check_impl_detail(jython=True)    # Only on Jython.
check_impl_detail(cpython=False)  # Everywhere except CPython
```

test.support.set_memlimit(*limit*)

Define os valores para **max_memuse** e **real_max_memuse** para grandes testes de memória.

test.support.record_original_stdout(*stdout*)

Armazena o valor de *stdout*. Destina-se a manter o stdout no momento em que o registro começou.

test.support.get_original_stdout()

Retorna o stdout original definido por **record_original_stdout()** ou `sys.stdout` se não estiver definido.

test.support.args_from_interpreter_flags()

Retorna uma lista de argumentos de linha de comando reproduzindo as configurações em `sys.flags` e `sys.warnoptions`.

test.support.optim_args_from_interpreter_flags()

Retorna a lista de argumentos da linha de comando reproduzindo as configurações de otimização atuais em `sys.flags`.

test.support.captured_stdin()

test.support.captured_stdout()

test.support.captured_stderr()

Um gerenciador de contexto que substitui temporariamente o fluxo nomeado pelo objeto `io.StringIO`.

Exemplo do uso com fluxos de saída:

```
with captured_stdout() as stdout, captured_stderr() as stderr
    print("hello")
    print("error", file=sys.stderr)
assert stdout.getvalue() == "hello\n"
assert stderr.getvalue() == "error\n"
```

Exemplo de uso com fluxo de entrada:

```
with captured_stdin() as stdin:
    stdin.write('hello\n')
    stdin.seek(0)
    # call test code that consumes from sys.stdin
    captured = input()
self.assertEqual(captured, "hello")
```

test.support.disable_faulthandler()

Um gerenciador de contexto que desativa temporariamente `faulthandler`.

test.support.gc_collect()

Força a coleta de tantos objetos quanto possível. Isso é necessário porque a desalocação oportuna não é garantida pelo coletor de lixo. Isso significa que os métodos `__del__` podem ser chamados mais tarde do que o esperado e as referência fraca podem permanecer ativas por mais tempo do que o esperado.

test.support.disable_gc()

A context manager that disables the garbage collector on entry. On exit, the garbage collector is restored to its prior state.

test.support.swap_attr(*obj*, *attr*, *new_val*)

Context manager to swap out an attribute with a new object.

Uso:

```
with swap_attr(obj, "attr", 5):
    ...
```

This will set `obj.attr` to 5 for the duration of the `with` block, restoring the old value at the end of the block. If `attr` doesn't exist on `obj`, it will be created and then deleted at the end of the block.

The old value (or `None` if it doesn't exist) will be assigned to the target of the "as" clause, if there is one.

test.support.swap_item(*obj*, *attr*, *new_val*)

Context manager to swap out an item with a new object.

Uso:

```
with swap_item(obj, "item", 5):
    ...
```

This will set `obj["item"]` to 5 for the duration of the `with` block, restoring the old value at the end of the block. If `item` doesn't exist on `obj`, it will be created and then deleted at the end of the block.

The old value (or `None` if it doesn't exist) will be assigned to the target of the "as" clause, if there is one.

test.support.flush_std_streams()

Call the `flush()` method on `sys.stdout` and then on `sys.stderr`. It can be used to make sure that the logs order is consistent before writing into stderr.

Adicionado na versão 3.11.

test.support.print_warning(*msg*)

Print a warning into `sys.stderr`. Format the message as: `f"Warning -- {msg}"`. If *msg* is made of multiple lines, add `"Warning -- "` prefix to each line.

Adicionado na versão 3.9.

test.support.wait_process(*pid*, *, *exitcode*, *timeout=None*)

Wait until process *pid* completes and check that the process exit code is *exitcode*.

Raise an `AssertionError` if the process exit code is not equal to *exitcode*.

If the process runs longer than *timeout* seconds (`SHORT_TIMEOUT` by default), kill the process and raise an `AssertionError`. The timeout feature is not available on Windows.

Adicionado na versão 3.9.

test.support.calcobjsize(*fmt*)

Return the size of the `PyObject` whose structure members are defined by *fmt*. The returned value includes the size of the Python object header and alignment.

test.support.calcvobjsize(*fmt*)

Return the size of the `PyVarObject` whose structure members are defined by *fmt*. The returned value includes the size of the Python object header and alignment.

test.support.checksizeof(*test*, *o*, *size*)

For testcase *test*, assert that the `sys.getsizeof` for *o* plus the GC header size equals *size*.

@test.support.anticipate_failure(*condition*)

A decorator to conditionally mark tests with `@unittest.expectedFailure`. Any use of this decorator should have an associated comment identifying the relevant tracker issue.

test.support.system_must_validate_cert(*f*)

A decorator that skips the decorated test on TLS certification validation failures.

@test.support.run_with_locale(*catstr*, **locales*)

A decorator for running a function in a different locale, correctly resetting it after it has finished. *catstr* is the locale category as a string (for example `"LC_ALL"`). The *locales* passed will be tried sequentially, and the first valid locale will be used.

@test.support.run_with_tz(*tz*)

A decorator for running a function in a specific timezone, correctly resetting it after it has finished.

@test.support.requires_freebsd_version(**min_version*)

Decorator for the minimum version when running test on FreeBSD. If the FreeBSD version is less than the minimum, the test is skipped.

@test.support.requires_linux_version(**min_version*)

Decorator for the minimum version when running test on Linux. If the Linux version is less than the minimum, the test is skipped.

@test.support.requires_mac_version(**min_version*)

Decorator for the minimum version when running test on macOS. If the macOS version is less than the minimum, the test is skipped.

@test.support.requires_gil_enabled

Decorador para pular testes na construção com threads livres. Se a GIL estiver desabilitada, o teste será ignorado.

@test.support.requires_IEEE_754

Decorator for skipping tests on non-IEEE 754 platforms.

@test.support.requires_zlib

Decorator for skipping tests if `zlib` doesn't exist.

@test.support.requires_gzip

Decorator for skipping tests if `gzip` doesn't exist.

@test.support.requires_bz2

Decorator for skipping tests if `bz2` doesn't exist.

@test.support.requires_lzma

Decorator for skipping tests if `lzma` doesn't exist.

@test.support.requires_resource(*resource*)

Decorator for skipping tests if *resource* is not available.

@test.support.requires_docstrings

Decorator for only running the test if `HAVE_DOCSTRINGS`.

@test.support.requires_limited_api

Decorator for only running the test if Limited C API is available.

@test.support.cpython_only

Decorator for tests only applicable to CPython.

@test.support.impl_detail(*msg=None*, ***guards*)

Decorator for invoking `check_impl_detail()` on *guards*. If that returns `False`, then uses *msg* as the reason for skipping the test.

@test.support.thread_unsafe(*reason=None*)

Decorator for marking tests as thread-unsafe. This test always runs in one thread even when invoked with `--parallel-threads`.

@test.support.no_tracing

Decorator to temporarily turn off tracing for the duration of the test.

@test.support.refcount_test

Decorator for tests which involve reference counting. The decorator does not run the test if it is not run by CPython. Any trace function is unset for the duration of the test to prevent unexpected refcounts caused by the trace function.

@test.support.bigmemtest(*size*, *memuse*, *dry_run=True*)

Decorator for bigmem tests.

size is a requested size for the test (in arbitrary, test-interpreted units.) *memuse* is the number of bytes per unit for the test, or a good estimate of it. For example, a test that needs two byte buffers, of 4 GiB each, could be decorated with `@bigmemtest(size=_4G, memuse=2)`.

The *size* argument is normally passed to the decorated test method as an extra argument. If *dry_run* is `True`, the value passed to the test method may be less than the requested value. If *dry_run* is `False`, it means the test doesn't support dummy runs when `-M` is not specified.

@test.support.bigaddrspacetest

Decorador para testes que preenche o espaço do endereço.

test.support.linked_to_musl()

Return `False` if there is no evidence the interpreter was compiled with `musl`, otherwise return a version triple, either `(0, 0, 0)` if the version is unknown, or the actual version if it is known. Intended for use in `skip` decorators. `emscripten` and `wasi` are assumed to be compiled with `musl`; otherwise `platform.libc_ver` is checked.

test.support.check_syntax_error(*testcase*, *statement*, *errtext=''*, ***, *lineno=None*, *offset=None*)

Test for syntax errors in *statement* by attempting to compile *statement*. *testcase* is the **`unittest`** instance for the test. *errtext* is the regular expression which should match the string representation of the raised **`SyntaxError`**. If *lineno* is not `None`, compares to the line of the exception. If *offset* is not `None`, compares to the offset of the exception.

test.support.open_urlresource(*url*, **args*, ***kw*)

Abre *url*. Se falhar em abrir, levanta **`TestFailed`**.

test.support.reap_children()

Use this at the end of `test_main` whenever sub-processes are started. This will help ensure that no extra children (zombies) stick around to hog resources and create problems when looking for refleaks.

test.support.get_attribute(*obj*, *name*)

Get an attribute, raising **`unittest.SkipTest`** if **`AttributeError`** is raised.

test.support.catch_unraisable_exception()

Context manager catching unraisable exception using **`sys.unraisablehook()`**.

Storing the exception value (`cm.unraisable.exc_value`) creates a reference cycle. The reference cycle is broken explicitly when the context manager exits.

Storing the object (`cm.unraisable.object`) can resurrect it if it is set to an object which is being finalized. Exiting the context manager clears the stored object.

Uso:

```
with support.catch_unraisable_exception() as cm:
    # code creating an "unraisable exception"
    ...

    # check the unraisable exception: use cm.unraisable
    ...

# cm.unraisable attribute no longer exists at this point
# (to break a reference cycle)
```

Adicionado na versão 3.8.

test.support.load_package_tests(*pkg_dir*, *loader*, *standard_tests*, *pattern*)

Generic implementation of the **unittest** `load_tests` protocol for use in test packages. *pkg_dir* is the root directory of the package; *loader*, *standard_tests*, and *pattern* are the arguments expected by `load_tests`. In simple cases, the test package's `__init__.py` can be the following:

```
import os
from test.support import load_package_tests

def load_tests(*args):
    return load_package_tests(os.path.dirname(__file__), *arg
```

test.support.detect_api_mismatch(*ref_api*, *other_api*, ***, *ignore=()*)

Returns the set of attributes, functions or methods of *ref_api* not found on *other_api*, except for a defined list of items to be ignored in this check specified in *ignore*.

By default this skips private attributes beginning with '_' but includes all magic methods, i.e. those starting and ending in '__'.

Adicionado na versão 3.5.

test.support.patch(*test_instance*, *object_to_patch*, *attr_name*, *new_value*)

Override *object_to_patch.attr_name* with *new_value*. Also add cleanup procedure to *test_instance* to restore *object_to_patch* for *attr_name*. The *attr_name* should be a valid attribute for *object_to_patch*.

test.support.run_in_subinterp(*code*)

Run *code* in subinterpreter. Raise **unittest.SkipTest** if **tracemalloc** is enabled.

@test.support.isolation.runInSubprocess(***, *options=()*, *env=None*, *timeout=None*)

Decorator that runs the decorated test in a fresh interpreter subprocess, in isolation, so that it does not share global or interpreter state with the rest of the test run. It can decorate a test method or a whole **TestCase** subclass. Decorated methods must take no extra arguments. A failure, error or skip in the subprocess is reported for the corresponding test, and

individual **subtests** that fail or are skipped are reported individually. A reported failure or error shows the original subprocess traceback as the cause of the exception.

When a **method** is decorated, only that method runs in a subprocess; all fixtures (**setUp()** / **tearDown()**, **setUpClass()** / **tearDownClass()** and `setUpModule()` / `tearDownModule()`) run both in the parent process (as usual) and in the subprocess around the method.

When a **class** is decorated, the whole class runs in a single subprocess, and **setUpClass()**, **tearDownClass()**, **setUp()** and **tearDown()** run once each in the subprocess and are skipped in the parent process. A failure or skip of **`setUpClass()`** in the subprocess is reported for the whole class. `setUpModule()` cannot be controlled by a class decorator, so it still runs in the parent process too; test it with **runningInSubprocess** if needed.

The subprocess inherits the enabled resources (`-u`), memory limit (`-M`) and verbosity (`-v`) of the parent test run, so that **requires_resource()**, **requires()**, **bigmemtest()** and the like behave consistently in both processes.

options is a sequence of interpreter command line options to run the subprocess with, and *env* is a mapping of environment variables to set in it, on top of the inherited environment. A value of `None` in *env* unsets the variable. Note that **-E** and **-I** make the subprocess ignore the `PYTHON*` environment variables, including **PYTHONPATH**.

timeout is the number of seconds to wait for the subprocess; the test is reported as an error if it does not complete in time. By default there is no timeout, and a hung test is left to the timeout of the test runner.

The test is skipped on platforms without subprocess support.

test.support.isolation.runningInSubprocess

`True` while the code runs in the isolated subprocess spawned by **runInSubprocess()**, and `False` otherwise (including in the parent process and in a normal, non-isolated test run). Fixtures such as

setUp(), **tearDown()**, **setUpClass()**, **tearDownClass()**, `setUpModule()` and `tearDownModule()` can test it to choose which code to run in the subprocess.

test.support.check_free_after_iterating(*test*, *iter*, *cls*, *args=()*)

Assert instances of *cls* are deallocated after iterating.

test.support.missing_compiler_executable(*cmd_names=[]*)

Check for the existence of the compiler executables whose names are listed in *cmd_names* or all the compiler executables when *cmd_names* is empty and return the first missing executable or `None` when none is found missing.

test.support.check__all__(*test_case*, *module*, *name_of_module=None*, *extra=()*, *not_exported=()*)

Assert that the `__all__` variable of *module* contains all public names.

The module's public names (its API) are detected automatically based on whether they match the public name convention and were defined in *module*.

The *name_of_module* argument can specify (as a string or tuple thereof) what module(s) an API could be defined in order to be detected as a public API. One case for this is when *module* imports part of its public API from other modules, possibly a C backend (like `csv` and its `_csv`).

The *extra* argument can be a set of names that wouldn't otherwise be automatically detected as "public", like objects without a proper **`__module__`** attribute. If provided, it will be added to the automatically detected ones.

The *not_exported* argument can be a set of names that must not be treated as part of the public API even though their names indicate otherwise.

Exemplo de uso:

```
import bar
import foo
import unittest
from test import support

class MiscTestCase(unittest.TestCase):
    def test__all__(self):
        support.check__all__(self, foo)

class OtherTestCase(unittest.TestCase):
    def test__all__(self):
        extra = {'BAR_CONST', 'FOO_CONST'}
        not_exported = {'baz'}  # Undocumented name.
        # bar imports part of its API from _bar.
        support.check__all__(self, bar, ('bar', '_bar'),
                             extra=extra, not_exported=not_ex
```

Adicionado na versão 3.6.

test.support.skip_if_broken_multiprocessing_synchronize()

Skip tests if the **`multiprocessing.synchronize`** module is missing, if there is no available semaphore implementation, or if creating a lock raises an **`OSError`**.

Adicionado na versão 3.10.

test.support.check_disallow_instantiation(*test_case*, *tp*, **args*, ***kwds*)

Assert that type *tp* cannot be instantiated using *args* and *kwds*.

Adicionado na versão 3.10.

test.support.adjust_int_max_str_digits(*max_digits*)

This function returns a context manager that will change the global **`sys.set_int_max_str_digits()`** setting for the duration of the context to allow execution of test code that needs a different limit on the number of digits when converting between an integer and string.

Adicionado na versão 3.11.

The **`test.support`** module defines the following classes:

class test.support.SuppressCrashReport

A context manager used to try to prevent crash dialog popups on tests that are expected to crash a subprocess.

No Windows, desativa as caixas de diálogo Relatório de Erros do Windows usando SetErrorMode <https://msdn.microsoft.com/en-us/library/windows/desktop/ms680621.aspx>_.

On UNIX, **`resource.setrlimit()`** is used to set **`resource.RLIMIT_CORE`**'s soft limit to 0 to prevent coredump file creation.

On both platforms, the old value is restored by **`__exit__()`**.

class test.support.SaveSignals

Class to save and restore signal handlers registered by the Python signal handler.

save(*self*)

Save the signal handlers to a dictionary mapping signal numbers to the current signal handler.

restore(*self*)

Set the signal numbers from the **`save()`** dictionary to the saved handler.

class test.support.Matcher

matches(*self*, *d*, ***kwargs*)

Tenta corresponder um único dicionário com os argumentos fornecidos.

match_value(*self*, *k*, *dv*, *v*)

Tente combinar um único valor armazenado (*dv*) com um valor fornecido (*v*).

`test.support.socket_helper` — Utilities for socket tests

The **`test.support.socket_helper`** module provides support for socket tests.

Adicionado na versão 3.9.

test.support.socket_helper.IPV6_ENABLED

Set to `True` if IPv6 is enabled on this host, `False` otherwise.

test.support.socket_helper.find_unused_port(*family=socket.AF_INET*, *socktype=socket.SOCK_STREAM*)

Returns an unused port that should be suitable for binding. This is achieved by creating a temporary socket with the same family and type as the `sock` parameter (default is **`AF_INET`**, **`SOCK_STREAM`**), and binding it to the specified host address (defaults to `0.0.0.0`) with the port set to 0, eliciting an unused ephemeral port from the OS. The temporary socket is then closed and deleted, and the ephemeral port is returned.

Either this method or **`bind_port()`** should be used for any tests where a server socket needs to be bound to a particular port for the duration of the test. Which one to use depends on whether the calling code is creating a Python socket, or if an unused port needs to be provided in a constructor or passed to an external program (i.e. the `-accept` argument to openssl's s_server mode). Always prefer **`bind_port()`** over **`find_unused_port()`** where possible. Using a hard coded port is discouraged since it can make multiple instances of the test impossible to run simultaneously, which is a problem for buildbots.

test.support.socket_helper.bind_port(*sock*, *host=HOST*)

Bind the socket to a free port and return the port number. Relies on ephemeral ports in order to ensure we are using an unbound port. This is important as many tests may be running simultaneously, especially in a

buildbot environment. This method raises an exception if the `sock.family` is **AF_INET** and `sock.type` is **SOCK_STREAM**, and the socket has **SO_REUSEADDR** or **SO_REUSEPORT** set on it. Tests should never set these socket options for TCP/IP sockets. The only case for setting these options is testing multicasting via multiple UDP sockets.

Additionally, if the **SO_EXCLUSIVEADDRUSE** socket option is available (i.e. on Windows), it will be set on the socket. This will prevent anyone else from binding to our host/port for the duration of the test.

test.support.socket_helper.bind_unix_socket(*sock*, *addr*)

Bind a Unix socket, raising **unittest.SkipTest** if **PermissionError** is raised.

@test.support.socket_helper.skip_unless_bind_unix_socket

A decorator for running tests that require a functional `bind()` for Unix sockets.

test.support.socket_helper.transient_internet(*resource_name*, ***, *timeout=30.0*, *errnos=()*)

A context manager that raises **ResourceDenied** when various issues with the internet connection manifest themselves as exceptions.

test.support.script_helper — Utilities for the Python execution tests

The **test.support.script_helper** module provides support for Python's script execution tests.

test.support.script_helper.interpreter_requires_environment()

Return `True` if `sys.executable interpreter` requires environment variables in order to be able to run at all.

This is designed to be used with `@unittest.skipIf()` to annotate tests that need to use an `assert_python*()` function to launch an isolated mode (`-I`) or no environment mode (`-E`) sub-interpreter process.

A normal build & test does not run into this situation but it can happen when trying to run the standard library test suite from an interpreter that doesn't have an obvious home with Python's current home finding logic.

Setting **PYTHONHOME** is one way to get most of the testsuite to run in that situation. **PYTHONPATH** or **PYTHONUSERSITE** are other common environment variables that might impact whether or not the interpreter can start.

test.support.script_helper.run_python_until_end(**args*, ***env_vars*)

Set up the environment based on *env_vars* for running the interpreter in a subprocess. The values can include `__isolated`, `__cleanenv`, `__cwd`, and `TERM`.

Alterado na versão 3.9: The function no longer strips whitespaces from *stderr*.

test.support.script_helper.assert_python_ok(**args*, ***env_vars*)

Assert that running the interpreter with *args* and optional environment variables *env_vars* succeeds (`rc == 0`) and return a `(return code, stdout, stderr)` tuple.

If the *__cleanenv* keyword-only parameter is set, *env_vars* is used as a fresh environment.

Python is started in isolated mode (command line option `-I`), except if the *__isolated* keyword-only parameter is set to `False`.

Alterado na versão 3.9: The function no longer strips whitespaces from *stderr*.

test.support.script_helper.assert_python_failure(**args*, ***env_vars*)

Assert that running the interpreter with *args* and optional environment variables *env_vars* fails (`rc != 0`) and return a `(return code, stdout, stderr)` tuple.

See **`assert_python_ok()`** for more options.

Alterado na versão 3.9: The function no longer strips whitespaces from *stderr*.

test.support.script_helper.spawn_python(**args*, *stdout=subprocess.PIPE*, *stderr=subprocess.STDOUT*, ***kw*)

Run a Python subprocess with the given arguments.

kw is extra keyword args to pass to **`subprocess.Popen()`**. Returns a **`subprocess.Popen`** object.

test.support.script_helper.kill_python(*p*)

Run the given **`subprocess.Popen`** process until completion and return stdout.

test.support.script_helper.make_script(*script_dir*, *script_basename*, *source*, *omit_suffix=False*)

Cria um script contendo *source* no caminho *script_dir* e *script_basename*. Se *omit_suffix* `False`, acrescente `.py` ao nome. Retorna o caminho completo do script.

test.support.script_helper.make_zip_script(*zip_dir*, *zip_basename*, *script_name*, *name_in_zip=None*)

Cria um arquivo zip em *zip_dir* e *zip_basename* com a extensão `zip` que contém os arquivos em *script_name*. *name_in_zip* é o nome do arquivo. Retorna uma tupla contendo `(full path, full path of archive name)`.

test.support.script_helper.make_pkg(*pkg_dir*, *init_source=''*)

Cria um diretório nomeado *pkg_dir* contendo um arquivo `__init__` com *init_source* como seus conteúdos.

test.support.script_helper.make_zip_pkg(*zip_dir*, *zip_basename*, *pkg_name*, *script_basename*, *source*, *depth=1*, *compiled=False*)

Create a zip package directory with a path of *zip_dir* and *zip_basename* containing an empty `__init__` file and a file *script_basename* containing the *source*. If *compiled* is `True`, both source files will be compiled and added to the zip package. Return a tuple of the full zip path and the archive name for the zip file.

`test.support.bytecode_helper` — Support tools for testing correct bytecode generation

The **`test.support.bytecode_helper`** module provides support for testing and inspecting bytecode generation.

Adicionado na versão 3.9.

O módulo define a seguinte classe:

class test.support.bytecode_helper.BytecodeTestCase(*unittest.TestCase*)

This class has custom assertion methods for inspecting bytecode.

BytecodeTestCase.get_disassembly_as_string(*co*)

Return the disassembly of *co* as string.

BytecodeTestCase.assertInBytecode(*x*, *opname*, *argval=_UNSPECIFIED*)

Return instr if *opname* is found, otherwise throws **`AssertionError`**.

BytecodeTestCase.assertNotInBytecode(*x*, *opname*, *argval=_UNSPECIFIED*)

Throws **`AssertionError`** if *opname* is found.

`test.support.threading_helper` — Utilities for threading tests

The **`test.support.threading_helper`** module provides support for threading tests.

Adicionado na versão 3.10.

test.support.threading_helper.join_thread(*thread*, *timeout=None*)

Join a *thread* within *timeout*. Raise an **`AssertionError`** if thread is still alive after *timeout* seconds.

@test.support.threading_helper.reap_threads

Decorator to ensure the threads are cleaned up even if the test fails.

test.support.threading_helper.start_threads(*threads*, *unlock=None*)

Context manager to start *threads*, which is a sequence of threads. *unlock* is a function called after the threads are started, even if an exception was raised; an example would be **`threading.Event.set()`**. `start_threads` will attempt to join the started threads upon exit.

test.support.threading_helper.threading_cleanup(**original_values*)

Cleanup up threads not specified in *original_values*. Designed to emit a warning if a test leaves running threads in the background.

test.support.threading_helper.threading_setup()

Return current thread count and copy of dangling threads.

test.support.threading_helper.wait_threads_exit(*timeout=None*)

Context manager to wait until all threads created in the `with` statement exit.

test.support.threading_helper.catch_threading_exception()

Context manager catching `threading.Thread` exception using `threading.excepthook()`.

Attributes set when an exception is caught:

- `exc_type`
- `exc_value`
- `exc_traceback`
- `thread`

See `threading.excepthook()` documentation.

These attributes are deleted at the context manager exit.

Uso:

```
with threading_helper.catch_threading_exception() as cm:
    # code spawning a thread which raises an exception
    ...

    # check the thread exception, use cm attributes:
    # exc_type, exc_value, exc_traceback, thread
    ...

# exc_type, exc_value, exc_traceback, thread attributes of cm
# exists at this point
# (to avoid reference cycles)
```

Adicionado na versão 3.8.

test.support.threading_helper.run_concurrently(*worker_func*, *nthreads*, *args=()*, *kwargs={}*)

Run the worker function concurrently in multiple threads. Re-raises an exception if any thread raises one, after all threads have finished.

`test.support.os_helper` — Utilities for os tests

The **`test.support.os_helper`** module provides support for os tests.

Adicionado na versão 3.10.

test.support.os_helper.FS_NONASCII

A non-ASCII character encodable by **`os.fsencode()`**.

test.support.os_helper.SAVEDCWD

Set to **`os.getcwd()`**.

test.support.os_helper.TESTFN

Set to a name that is safe to use as the name of a temporary file. Any temporary file that is created should be closed and unlinked (removed).

test.support.os_helper.TESTFN_NONASCII

Set to a filename containing the **`FS_NONASCII`** character, if it exists. This guarantees that if the filename exists, it can be encoded and decoded with the default filesystem encoding. This allows tests that require a non-ASCII filename to be easily skipped on platforms where they can't work.

test.support.os_helper.TESTFN_UNENCODABLE

Define o nome de arquivo (tipo str) que não pode ser codificado pela codificação do sistema de arquivos no modo estrito. Ele pode ser `None` se não for possível gerar como um nome de arquivo.

test.support.os_helper.TESTFN_UNDECODABLE

Define o nome de arquivo (tipo str) que não pode ser codificado pela codificação do sistema de arquivos no modo estrito. Ele pode ser `None` se não for possível ser gerado com um nome de arquivo.

test.support.os_helper.TESTFN_UNICODE

Define um nome não-ASCII para o arquivo temporário.

class test.support.os_helper.EnvironmentVarGuard

Class used to temporarily set or unset environment variables. Instances can be used as a context manager and have a complete dictionary

interface for querying/modifying the underlying `os.environ`. After exit from the context manager all changes to environment variables done through this instance will be rolled back.

Alterado na versão 3.1: Adicionada uma interface para dicionário.

class test.support.os_helper.FakePath(*path*)

Simple path-like object. It implements the **`__fspath__()`** method which just returns the *path* argument. If *path* is an exception, it will be raised in **`__fspath__()`**.

EnvironmentVarGuard.set(*envvar*, *value*)

Temporariamente define a variável de ambiente `envvar` para o valor `value`.

EnvironmentVarGuard.unset(*envvar*, **others*)

Temporarily unset one or more environment variables.

Alterado na versão 3.14: More than one environment variable can be unset.

test.support.os_helper.can_symlink()

Return `True` if the OS supports symbolic links, `False` otherwise.

test.support.os_helper.can_xattr()

Return `True` if the OS supports xattr, `False` otherwise.

test.support.os_helper.change_cwd(*path*, *quiet=False*)

A context manager that temporarily changes the current working directory to *path* and yields the directory.

If *quiet* is `False`, the context manager raises an exception on error. Otherwise, it issues only a warning and keeps the current working directory the same.

test.support.os_helper.create_empty_file(*filename*)

Create an empty file with *filename*. If it already exists, truncate it.

test.support.os_helper.fd_count()

Conta o número de descritores de arquivos abertos.

test.support.os_helper.fs_is_case_insensitive(*directory*)

Return `True` if the file system for *directory* is case-insensitive.

test.support.os_helper.make_bad_fd()

Cria um descritor de arquivo inválido abrindo e fechando um arquivo temporário e retornando seu descritor.

test.support.os_helper.rmdir(*filename*)

Call `os.rmdir()` on *filename*. On Windows platforms, this is wrapped with a wait loop that checks for the existence of the file, which is needed due to antivirus programs that can hold files open and prevent deletion.

test.support.os_helper.rmtree(*path*)

Call `shutil.rmtree()` on *path* or call `os.lstat()` and `os.rmdir()` to remove a path and its contents. As with `rmdir()`, on Windows platforms this is wrapped with a wait loop that checks for the existence of the files.

@test.support.os_helper.skip_unless_symlink

Um decorador para executar testes que requerem suporte para links simbólicos.

@test.support.os_helper.skip_unless_xattr

Um decorador para execução de testes que requerem suporte para xattr.

test.support.os_helper.temp_cwd(*name='tempcwd', quiet=False*)

Um gerenciador de contexto que cria temporariamente um novo diretório e altera o diretório de trabalho atual (CWD).

The context manager creates a temporary directory in the current directory with name *name* before temporarily changing the current working directory. If *name* is `None`, the temporary directory is created using `tempfile.mkdtemp()`.

Se *quiet* é `False` e ele não possibilita criar ou alterar o CWD, um erro é levantado. Por outro lado, somente um aviso surge e o CWD original é utilizado.

test.support.os_helper.temp_dir(*path=None*, *quiet=False*)

Um gerenciador de contexto que cria um diretório temporário no *path* e produz o diretório.

If *path* is `None`, the temporary directory is created using `tempfile.mkdtemp()`. If *quiet* is `False`, the context manager raises an exception on error. Otherwise, if *path* is specified and cannot be created, only a warning is issued.

test.support.os_helper.temp_umask(*umask*)

Um gerenciador de contexto que temporariamente define o umask do processo.

test.support.os_helper.unlink(*filename*)

Call `os.unlink()` on *filename*. As with `rmdir()`, on Windows platforms, this is wrapped with a wait loop that checks for the existence of the file.

@test.support.os_helper.with_source_date_epoch(*, *epoch=123456789*)

A decorator for running tests with the **`SOURCE_DATE_EPOCH`** environment variable set to *epoch*.

@test.support.os_helper.without_source_date_epoch

A decorator for running tests with the **`SOURCE_DATE_EPOCH`** environment variable unset.

class test.support.os_helper.SourceDateEpochTestMeta

Metaclass wrapping all test methods of the class with **with_source_date_epoch()** if the *source_date_epoch* keyword class argument is true, or with **without_source_date_epoch()** otherwise. For example:

```
class TestsWithSourceEpoch(Tests,
                           metaclass=SourceDateEpochTestMeta,
                           source_date_epoch=True):
    pass
```

test.support.import_helper — Utilities for import tests

The **test.support.import_helper** module provides support for import tests.

Adicionado na versão 3.10.

test.support.import_helper.forget(*module_name*)

Remove the module named *module_name* from `sys.modules` and delete any byte-compiled files of the module.

test.support.import_helper.import_fresh_module(*name*, *fresh=()*, *blocked=()*, *deprecated=False*)

This function imports and returns a fresh copy of the named Python module by removing the named module from `sys.modules` before doing the import. Note that unlike **reload()**, the original module is not affected by this operation.

fresh is an iterable of additional module names that are also removed from the `sys.modules` cache before doing the import.

blocked is an iterable of module names that are replaced with `None` in the module cache during the import to ensure that attempts to import

them raise **ImportError**.

The named module and any modules named in the *fresh* and *blocked* parameters are saved before starting the import and then reinserted into `sys.modules` when the fresh import is complete.

Module and package deprecation messages are suppressed during this import if *deprecated* is `True`.

This function will raise **ImportError** if the named module cannot be imported.

Exemplo de uso:

```
# Get copies of the warnings module for testing without affec
# version being used by the rest of the test suite. One copy
# C implementation, the other is forced to use the pure Pytho
# implementation
py_warnings = import_fresh_module('warnings', blocked=['_warr
c_warnings = import_fresh_module('warnings', fresh=['_warning
```

Adicionado na versão 3.1.

test.support.import_helper.import_module(*name*, *deprecated=False*, ***, *required_on=()*)

This function imports and returns the named module. Unlike a normal import, this function raises **unittest.SkipTest** if the module cannot be imported.

Module and package deprecation messages are suppressed during this import if *deprecated* is `True`. If a module is required on a platform but optional for others, set *required_on* to an iterable of platform prefixes which will be compared against **sys.platform**.

Adicionado na versão 3.1.

test.support.import_helper.modules_setup()

Retorna a cópia de **sys.modules**.

test.support.import_helper.modules_cleanup(*oldmodules*)

Remove modules except for *oldmodules* and `encodings` in order to preserve internal cache.

test.support.import_helper.unload(*name*)

Exclui o *name* de `sys.modules`.

test.support.import_helper.make_legacy_pyc(*source*)

Move a **PEP 3147** [https://peps.python.org/pep-3147/]/**PEP 488** [https://peps.python.org/pep-0488/] pyc file to its legacy pyc location and return the file system path to the legacy pyc file. The *source* value is the file system path to the source file. It does not need to exist, however the PEP 3147/488 pyc file must exist.

class test.support.import_helper.CleanImport(**module_names*)

A context manager to force import to return a new module reference. This is useful for testing module-level behaviors, such as the emission of a `DeprecationWarning` on import. Example usage:

```
with CleanImport('foo'):
    importlib.import_module('foo')  # New reference.
```

class test.support.import_helper.DirsOnSysPath(**paths*)

A context manager to temporarily add directories to `sys.path`.

This makes a copy of `sys.path`, appends any directories given as positional arguments, then reverts **`sys.path`** to the copied settings when the context ends.

Note that *all* `sys.path` modifications in the body of the context manager, including replacement of the object, will be reverted at the end of the block.

`test.support.warnings_helper` — Utilities for warnings tests

The **`test.support.warnings_helper`** module provides support for warnings tests.

Adicionado na versão 3.10.

test.support.warnings_helper.ignore_warnings(*, *category*)

Suppress warnings that are instances of *category*, which must be `Warning` or a subclass. Roughly equivalent to `warnings.catch_warnings()` with `warnings.simplefilter('ignore', category=category)`. For example:

```
@warning_helper.ignore_warnings(category=DeprecationWarning)
def test_suppress_warning():
    # do something
```

Adicionado na versão 3.8.

test.support.warnings_helper.check_no_resource_warning(*testcase*)

Context manager to check that no `ResourceWarning` was raised. You must remove the object which may emit **`ResourceWarning`** before the end of the context manager.

test.support.warnings_helper.check_syntax_warning(*testcase*, *statement*, *errtext=''*, *, *lineno=1*, *offset=None*)

Test for syntax warning in *statement* by attempting to compile *statement*. Test also that the `SyntaxWarning` is emitted only once, and that it will be converted to a `SyntaxError` when turned into error. *testcase* is the `unittest` instance for the test. *errtext* is the regular expression which should match the string representation of the emitted **`SyntaxWarning`** and raised **`SyntaxError`**. If *lineno* is not `None`, compares

to the line of the warning and exception. If *offset* is not `None`, compares to the offset of the exception.

Adicionado na versão 3.8.

test.support.warnings_helper.check_warnings(**filters*, *quiet=True*)

A convenience wrapper for **`warnings.catch_warnings()`** that makes it easier to test that a warning was correctly raised. It is approximately equivalent to calling `warnings.catch_warnings(record=True)` with **`warnings.simplefilter()`** set to `always` and with the option to automatically validate the results that are recorded.

`check_warnings` accepts 2-tuples of the form `("message regexp", WarningCategory)` as positional arguments. If one or more *filters* are provided, or if the optional keyword argument *quiet* is `False`, it checks to make sure the warnings are as expected: each specified filter must match at least one of the warnings raised by the enclosed code or the test fails, and if any warnings are raised that do not match any of the specified filters the test fails. To disable the first of these checks, set *quiet* to `True`.

Se nenhum argumento é especificado, o padrão é:

```
check_warnings(("", Warning), quiet=True)
```

Nesse caso, todos os avisos são capturados e nenhum erro é gerado.

On entry to the context manager, a **`WarningRecorder`** instance is returned. The underlying warnings list from **`catch_warnings()`** is available via the recorder object's **`warnings`** attribute. As a convenience, the attributes of the object representing the most recent warning can also be accessed directly through the recorder object (see example below). If no warning has been raised, then any of the attributes that would otherwise be expected on an object representing a warning will return `None`.

The recorder object also has a **`reset()`** method, which clears the warnings list.

O gerenciador de contexto é desenhado para ser utilizado dessa forma:

```
with check_warnings(("assertion is always true", SyntaxWarnir
                    ("", UserWarning)):
    exec('assert(False, "Hey!")')
    warnings.warn(UserWarning("Hide me!"))
```

No caso, se um aviso não foi levantado, ou algum outro aviso não foi levantado, `check_warnings()` deveria aparecer como um erro.

When a test needs to look more deeply into the warnings, rather than just checking whether or not they occurred, code like this can be used:

```
with check_warnings(quiet=True) as w:
    warnings.warn("foo")
    assert str(w.args[0]) == "foo"
    warnings.warn("bar")
    assert str(w.args[0]) == "bar"
    assert str(w.warnings[0].args[0]) == "foo"
    assert str(w.warnings[1].args[0]) == "bar"
    w.reset()
    assert len(w.warnings) == 0
```

Aqui todos os avisos serão capturados e o código de teste testa os avisos diretamente capturados.

Alterado na versão 3.2: Novos argumentos opcionais *filters* e *quiet*.

class test.support.warnings_helper.WarningsRecorder

Class used to record warnings for unit tests. See documentation of `check_warnings()` above for more details.

Depuração e perfilamento

Essas bibliotecas ajudam no desenvolvimento do Python: o depurador permite que você percorra o código, analise os quadros de pilha e defina pontos de interrupção etc., e os criadores de perfil executam o código e fornecem uma análise detalhada dos tempos de execução, permitindo identificar gargalos em seus programas. Os eventos de auditoria fornecem visibilidade dos comportamentos de tempo de execução que, de outra forma, exigiriam depuração ou correção intrusiva.

- Tabela de eventos de auditoria
- `bdb` — Framework do depurador
- `faulthandler` — Dump the Python traceback
  - Dumping the traceback
  - Dumping the C stack
    - C Stack Compatibility
  - Fault handler state
  - Dumping the tracebacks after a timeout
  - Dumping the traceback on a user signal
  - Issue with file descriptors
  - Exemplo
- `pdb` — O depurador do Python
  - Interface de linha de comando
  - Comandos de depuração
- `profiling` — Python profilers
  - Introduction to profiling
  - Choosing a profiler
    - When to use statistical sampling
    - When to use deterministic tracing
  - Quick start
    - Statistical profiling
    - Deterministic profiling
  - Understanding profile output
  - Legacy compatibility
    - `profiling.tracing` — Deterministic profiler

- What is deterministic profiling?
- Command-line interface
- Programmatic usage examples
  - Basic profiling
  - Using the `Profile` class
- Module reference
- Using a custom timer
- Limitations

- `profiling.sampling` — Statistical profiler
  - What is statistical profiling?
    - How time is estimated
    - When to use a different approach
  - Quick examples
  - Commands
    - The `run` command
    - The `attach` command
    - The `dump` command
    - The `replay` command
    - Profiling in production
    - Platform requirements
    - Version compatibility
  - Sampling configuration
    - Sampling rate and duration
    - Thread selection
    - Blocking mode
    - Special frames
    - Opcode-aware profiling
    - Real-time statistics
    - Subprocess profiling
    - Sampling efficiency
  - Profiling modes
    - Wall-clock mode
    - CPU mode
    - Comparing wall-clock and CPU profiles
    - GIL mode
    - Exception mode
  - Output formats

        - pstats format
        - Collapsed stacks format
        - Flame graph format
            - Differential flame graphs
        - Gecko format
        - Heatmap format
        - Binary format
    - Record and replay workflow
    - Live mode
        - Keyboard commands
    - Async-aware profiling
        - Async modes
        - Task markers and stack reconstruction
        - Option restrictions
    - Command-line interface
        - Global options
        - Dump options
        - Sampling options
        - Mode options
        - Output options
        - pstats display options
        - Run command options
- `pstats` — Statistics for profilers
    - Reading and displaying profile data
        - Working with statistics
        - Filtering output
        - Analyzing call relationships
        - Combining multiple profiles
    - The `Stats` class
    - Interface de linha de comando
- `timeit` — Mede o tempo de execução de pequenos trechos de código
    - Basic examples
    - Python interface
    - Interface de linha de comando
    - Exemplos
- `trace` — Rastreia ou acompanha a execução de instruções Python
    - Uso na linha de comando

        - Opções principais
        - Modificadores
        - Filtros
    - Interface programática
- `tracemalloc` — Trace memory allocations
    - Exemplos
        - Exibe o top 10
        - Compute differences
        - Get the traceback of a memory block
        - Pretty top
            - Record the current and peak size of all traced memory blocks
    - API
        - Funções
        - DomainFilter
        - Filter
        - Frame
        - Snapshot
        - Statistic
        - StatisticDiff
        - Trace
        - Traceback

Tabela de eventos de auditoria

Esta tabela contém todos os eventos levantados por chamadas de `sys.audit()` ou `PySys_Audit()` durante todo o tempo de execução do CPython e da biblioteca padrão. Essas chamadas foram adicionadas na versão 3.8 ou posterior (veja **PEP 578** [https://peps.python.org/pep-0578/]).

Veja `sys.addaudithook()` e `PySys_AddAuditHook()` para informações sobre como tratar estes eventos.

Esta tabela é gerada a partir da documentação do CPython e pode não representar eventos levantados por outras implementações. Consulte a documentação específica de tempo de execução para obter eventos reais levantados.

Audit event	Arguments	References
_thread.start_new_thread	`function`, `args`, `kwargs`	[1]
array.__new__	`typecode`, `initializer`	[1]
builtins.breakpoint	`breakpointhook`	[1]
builtins.id	`id`	[1]
builtins.input	`prompt`	[1]
builtins.input/result	`result`	[1]

Audit event	Arguments	References
code.__new__	`code`, `filename`, `name`, `argcount`, `posonlyargcount`, `kwonlyargcount`, `nlocals`, `stacksize`, `flags`	[1]
compile	`source`, `filename`	[1]
cpython.PyConfig_Set	`name`, `value`	[1]
cpython.PyInterpreterState_Clear		[1]
cpython.PyInterpreterState_New		[1]
cpython._PySys_ClearAuditHooks		[1]
cpython.remote_debugger_script	`script_path`	[1]
cpython.run_command	`command`	[1]
cpython.run_file	`filename`	[1]
cpython.run_interactivehook	`hook`	[1]

Audit event	Arguments	References
cpython.run_module	`module-name`	[1]
cpython.run_startup	`filename`	[1]
cpython.run_stdin		[1][2][3]
ctypes.addressof	`obj`	[1]
ctypes.call_function	`func_pointer`, `arguments`	[1]
ctypes.cdata	`address`	[1]
ctypes.cdata/buffer	`pointer`, `size`, `offset`	[1][2]
ctypes.create_string_buffer	`init`, `size`	[1]
ctypes.create_unicode_buffer	`init`, `size`	[1]
ctypes.dlopen	`name`	[1]
ctypes.dlsym	`library`, `name`	[1]

Audit event	Arguments	References
ctypes.dlsym/handle	`handle`, `name`	[1]
ctypes.get_errno		[1]
ctypes.get_last_error		[1]
ctypes.memoryview_at	`address`, `size`, `readonly`	[1]
ctypes.set_errno	`errno`	[1]
ctypes.set_exception	`code`	[1]
ctypes.set_last_error	`error`	[1]
ctypes.string_at	`ptr`, `size`	[1]
ctypes.wstring_at	`ptr`, `size`	[1]
ensurepip.bootstrap	`root`	[1]
exec	`code_object`	[1][2]

Audit event	Arguments	References
fcntl.fcntl	`fd`, `cmd`, `arg`	[1]
fcntl.flock	`fd`, `operation`	[1]
fcntl.ioctl	`fd`, `request`, `arg`	[1]
fcntl.lockf	`fd`, `cmd`, `len`, `start`, `whence`	[1]
ftplib.connect	`self`, `host`, `port`	[1]
ftplib.sendcmd	`self`, `cmd`	[1][2]
function.__new__	`code`	[1]
gc.get_objects	`generation`	[1]
gc.get_referents	`objs`	[1]
gc.get_referrers	`objs`	[1]
glob.glob	`pathname`, `recursive`	[1][2]

Audit event	Arguments	References
glob.glob/2	`pathname`, `recursive`, `root_dir`, `dir_fd`	[1][2]
http.client.connect	`self`, `host`, `port`	[1]
http.client.send	`self`, `data`	[1]
imaplib.open	`self`, `host`, `port`	[1]
imaplib.send	`self`, `data`	[1]
import	`module`, `filename`, `sys.path`, `sys.meta_path`, `sys.path_hooks`	[1]
marshal.dumps	`value`, `version`	[1]
marshal.load		[1]
marshal.loads	`bytes`	[1]
mmap.__new__	`fileno`, `length`, `access`, `offset`	[1]

Audit event	Arguments	References
msvcrt.get_osfhandle	`fd`	[1]
msvcrt.locking	`fd`, `mode`, `nbytes`	[1]
msvcrt.open_osfhandle	`handle`, `flags`	[1]
object.__delattr__	`obj`, `name`	[1]
object.__getattr__	`obj`, `name`	[1]
object.__setattr__	`obj`, `name`, `value`	[1]
open	`path`, `mode`, `flags`	[1][2][3]
os._clearenv		[1]
os.add_dll_directory	`path`	[1]
os.chdir	`path`	[1][2]
os.chflags	`path`, `flags`	[1][2]
os.chmod	`path`, `mode`, `dir_fd`	[1][2][3]

Audit event	Arguments	References
os.chown	`path`, `uid`, `gid`, `dir_fd`	[1][2][3]
os.exec	`path`, `args`, `env`	[1]
os.fork		[1]
os.forkpty		[1]
os.fwalk	`top`, `topdown`, `onerror`, `follow_symlinks`, `dir_fd`	[1]
os.getxattr	`path`, `attribute`	[1]
os.kill	`pid`, `sig`	[1]
os.killpg	`pgid`, `sig`	[1]
os.link	`src`, `dst`, `src_dir_fd`, `dst_dir_fd`	[1]
os.listdir	`path`	[1]
os.listdrives		[1]

Audit event	Arguments	References
os.listmounts	`volume`	[1]
os.listvolumes		[1]
os.listxattr	`path`	[1]
os.lockf	`fd`, `cmd`, `len`	[1]
os.mkdir	`path`, `mode`, `dir_fd`	[1][2]
os.posix_spawn	`path`, `argv`, `env`	[1][2]
os.putenv	`key`, `value`	[1]
os.remove	`path`, `dir_fd`	[1][2][3]
os.removexattr	`path`, `attribute`	[1]
os.rename	`src`, `dst`, `src_dir_fd`, `dst_dir_fd`	[1][2][3]
os.rmdir	`path`, `dir_fd`	[1]

Audit event	Arguments	References
os.scandir	`path`	[1]
os.setxattr	`path`, `attribute`, `value`, `flags`	[1]
os.spawn	`mode`, `path`, `args`, `env`	[1]
os.startfile	`path`, `operation`	[1]
os.startfile/2	`path`, `operation`, `arguments`, `cwd`, `show_cmd`	[1]
os.symlink	`src`, `dst`, `dir_fd`	[1]
os.system	`command`	[1]
os.truncate	`fd`, `length`	[1][2]
os.unsetenv	`key`	[1]
os.utime	`path`, `times`, `ns`, `dir_fd`	[1]

Audit event	Arguments	References
os.walk	`top`, `topdown`, `onerror`, `followlinks`	[1]
pathlib.Path.glob	`self`, `pattern`	[1]
pathlib.Path.rglob	`self`, `pattern`	[1]
pdb.Pdb		[1]
pickle.find_class	`module`, `name`	[1]
poplib.connect	`self`, `host`, `port`	[1][2]
poplib.putline	`self`, `line`	[1][2]
pty.spawn	`argv`	[1]
resource.prlimit	`pid`, `resource`, `limits`	[1]
resource.setrlimit	`resource`, `limits`	[1]
setopencodehook		[1]

Audit event	Arguments	References
shutil.chown	`path`, `user`, `group`	[1]
shutil.copyfile	`src`, `dst`	[1][2][3]
shutil.copymode	`src`, `dst`	[1][2]
shutil.copystat	`src`, `dst`	[1][2]
shutil.copytree	`src`, `dst`	[1]
shutil.make_archive	`base_name`, `format`, `root_dir`, `base_dir`	[1]
shutil.move	`src`, `dst`	[1]
shutil.rmtree	`path`, `dir_fd`	[1]
shutil.unpack_archive	`filename`, `extract_dir`, `format`	[1]
signal.pthread_kill	`thread_id`, `signalnum`	[1]
smtplib.connect	`self`, `host`, `port`	[1]

Audit event	Arguments	References
smtplib.send	`self`, `data`	[1]
socket.__new__	`self`, `family`, `type`, `protocol`	[1]
socket.bind	`self`, `address`	[1]
socket.connect	`self`, `address`	[1][2]
socket.getaddrinfo	`host`, `port`, `family`, `type`, `protocol`	[1]
socket.gethostbyaddr	`ip_address`	[1]
socket.gethostbyname	`hostname`	[1][2]
socket.gethostname		[1]
socket.getnameinfo	`sockaddr`	[1]
socket.getservbyname	`servicename`, `protocolname`	[1]
socket.getservbyport	`port`, `protocolname`	[1]

Audit event	Arguments	References
socket.sendmsg	`self`, `address`	[1]
socket.sendto	`self`, `address`	[1]
socket.sethostname	`name`	[1]
sqlite3.connect	`database`	[1]
sqlite3.connect/handle	`connection_handle`	[1]
sqlite3.enable_load_extension	`connection`, `enabled`	[1]
sqlite3.load_extension	`connection`, `path`	[1]
subprocess.Popen	`executable`, `args`, `cwd`, `env`	[1]
sys._current_exceptions		[1]
sys._current_frames		[1]
sys._getframe	`frame`	[1]

Audit event	Arguments	References
sys._getframemodulename	`depth`	[1]
sys.addaudithook		[1][2]
sys.excepthook	`hook`, `type`, `value`, `traceback`	[1]
sys.monitoring.register_callback	`func`	[1]
sys.remote_exec	`pid`, `script_path`	[1]
sys.set_asyncgen_hooks_finalizer		[1]
sys.set_asyncgen_hooks_firstiter		[1]
sys.setprofile		[1]
sys.settrace		[1]
sys.unraisablehook	`hook`, `unraisable`	[1]
syslog.closelog		[1]

Audit event	Arguments	References
syslog.openlog	`ident`, `logoption`, `facility`	[1]
syslog.setlogmask	`maskpri`	[1]
syslog.syslog	`priority`, `message`	[1]
tempfile.mkdtemp	`fullpath`	[1][2]
tempfile.mkstemp	`fullpath`	[1][2][3]
time.sleep	`seconds`	[1]
urllib.Request	`fullurl`, `data`, `headers`, `method`	[1]
webbrowser.open	`url`	[1]
winreg.ConnectRegistry	`computer_name`, `key`	[1]
winreg.CreateKey	`key`, `sub_key`, `access`	[1][2]
winreg.DeleteKey	`key`, `sub_key`, `access`	[1][2]

Audit event	Arguments	References
winreg.DeleteTree	`key`, `sub_key`	[1]
winreg.DeleteValue	`key`, `value`	[1]
winreg.DisableReflectionKey	`key`	[1]
winreg.EnableReflectionKey	`key`	[1]
winreg.EnumKey	`key`, `index`	[1]
winreg.EnumValue	`key`, `index`	[1]
winreg.ExpandEnvironmentStrings	`str`	[1]
winreg.LoadKey	`key`, `sub_key`, `file_name`	[1]
winreg.OpenKey	`key`, `sub_key`, `access`	[1]
winreg.OpenKey/result	`key`	[1][2][3]
winreg.PyHKEY.Detach	`key`	[1]

Audit event	Arguments	References
winreg.QueryInfoKey	`key`	[1]
winreg.QueryReflectionKey	`key`	[1]
winreg.QueryValue	`key`, `sub_key`, `value_name`	[1][2]
winreg.SaveKey	`key`, `file_name`	[1]
winreg.SetValue	`key`, `sub_key`, `type`, `value`	[1][2]

Os eventos a seguir são levantados internamente e não correspondem a nenhuma API pública de CPython:

Evento de auditoria	Argumentos
_winapi.CreateFile	`file_name`, `desired_access`, `share_mode`, `creation_disposition`, `flags_and_attributes`
_winapi.CreateJunction	`src_path`, `dst_path`
_winapi.CreateNamedPipe	`name`, `open_mode`, `pipe_mode`

Evento de auditoria	Argumentos
_winapi.CreatePipe	
_winapi.CreateProcess	`application_name`, `command_line`, `current_directory`
_winapi.OpenProcess	`process_id`, `desired_access`
_winapi.TerminateProcess	`handle`, `exit_code`
_posixsubprocess.fork_exec	`exec_list`, `args`, `env`
ctypes.PyObj_FromPtr	`obj`

Adicionado na versão 3.14: O evento de auditoria interno `_posixsubprocess.fork_exec`.

bdb — Framework do depurador

Código-fonte: Lib/bdb.py [https://github.com/python/cpython/tree/3.15/Lib/bdb.py]

The **bdb** module handles basic debugger functions, like setting breakpoints or managing execution via the debugger.

A seguinte exceção é definida:

exception bdb.BdbQuit

Exceção levantada pela classe **Bdb** para sair do depurador.

The **bdb** module also defines two classes:

class bdb.Breakpoint(*self*, *file*, *line*, *temporary=False*, *cond=None*, *funcname=None*)

Esta classe implementa pontos de interrupção temporários, contagens de ignorar, desativação e (re)ativação, além de condicionais.

Os pontos de interrupção são indexados por número por meio de uma lista chamada **bpbynumber** e por pares `(file, line)` por meio de **bplist**. O primeiro aponta para uma única instância da classe **Breakpoint**. O segundo aponta para uma lista dessas instâncias, já que pode haver mais de um ponto de interrupção por linha.

Ao criar um ponto de interrupção, seu **nome do arquivo** associado deve estar no formato canônico. Se um **funcname** for definido, um ponto de interrupção **atingido** será contado quando a primeira linha dessa função for executada. Um ponto de interrupção **condicional** sempre conta um **atingido**.

As instâncias de **Breakpoint** têm os seguintes métodos:

deleteMe()

Exclui o ponto de interrupção da lista associada a um arquivo/linha. Se for o último ponto de interrupção nessa posição, ele também excluirá a entrada do arquivo/linha.

enable()

Marca o ponto de interrupção como ativado.

disable()

Marca o ponto de interrupção como desativado.

bpformat()

Retorna uma string com todas as informações sobre o ponto de interrupção, formatadas de maneira clara:

- Número do ponto de interrupção.
- Status temporário (del ou keep).
- Posição do arquivo/linha.
- Condição de interrupção.
- Número de vezes para ignorar.
- Número de vezes atingido.

Adicionado na versão 3.2.

bpprint(*out=None*)

Exibe a saída de `bpformat()` no arquivo *out* ou, se for `None`, na saída padrão.

Instâncias `Breakpoint` possuem os seguintes atributos:

file

Nome do arquivo do `Breakpoint`.

line

Número da linha do `Breakpoint` dentro de `file`.

temporary

`True` se um `Breakpoint` em (file, line) for temporário.

cond

Condição para avaliar um `Breakpoint` em (file, line).

funcname

Nome da função que define se um `Breakpoint` é atingido ao entrar na função.

enabled

`True` se `Breakpoint` estiver ativado.

bpbynumber

Índice numérico de uma única instância de `Breakpoint`.

bplist

Dicionário de instâncias de `Breakpoint` indexadas por tuplas (`file`, `line`).

ignore

Número de vezes para ignorar o `Breakpoint`.

hits

Contagem do número de vezes que um `Breakpoint` foi atingido.

class bdb.Bdb(*skip=None*, *backend='settrace'*)

A `Bdb` atua como uma classe base genérica de depurador em Python.

Esta classe cuida dos detalhes do mecanismo de rastreamento; uma classe derivada deve implementar a interação com o usuário. A classe de depuração padrão (`pdb.Pdb`) é um exemplo.

O argumento *skip*, se fornecido, deve ser um iterável de padrões de nomes de módulos no estilo glob. O depurador não entrará em frames que se originam em um módulo que corresponda a um desses padrões. Se um frame é considerado como originado em determinado módulo é determinado pelo valor de `__name__` nos globals do frame.

O argumento *backend* especifica o backend a ser usado pela **Bdb**. Ele pode ser `'settrace'` ou `'monitoring'`. `'settrace'` usa **sys.settrace()**, que oferece a melhor retrocompatibilidade. O backend `'monitoring'` usa o novo **sys.monitoring**, introduzido no Python 3.12, que pode ser muito mais eficiente porque pode desabilitar eventos não utilizados. Estamos tentando manter interfaces idênticas para ambos os backends, mas existem algumas diferenças. Os desenvolvedores de depuradores são incentivados a usar o backend `'monitoring'` para obter melhor desempenho.

Alterado na versão 3.1: Adicionado o parâmetro *skip*.

Alterado na versão 3.14: Adicionado o parâmetro *backend*.

Os seguintes métodos de **Bdb** normalmente não precisam ser substituídos.

canonic(*filename*)

Retorna a forma canônica de *filename*.

Para nomes de arquivos reais, a forma canônica é um **caminho absoluto**, **normalizado para distinguir maiúsculas e minúsculas**, dependente do sistema operacional. Um *filename* com colchetes angulares, como `"<stdin>"` gerado no modo interativo, é retornado sem alterações.

start_trace(*self*)

Inicia o rastreamento. Para o backend `'settrace'`, este método equivale a `sys.settrace(self.trace_dispatch)`

Adicionado na versão 3.14.

stop_trace(*self*)

Para o rastreamento. Para o backend `'settrace'`, este método equivale a `sys.settrace(None)`

Adicionado na versão 3.14.

reset()

Define os atributos **`botframe`**, **`stopframe`**, **`returnframe`** e **`quitting`** com valores prontos para iniciar a depuração.

trace_dispatch(*frame*, *event*, *arg*)

Esta função é instalada como a função de rastreamento dos frames depurados. Seu valor de retorno é a nova função de rastreamento (na maioria dos casos, ela própria).

A implementação padrão decide como despachar um frame, dependendo do tipo de evento (passado como string) que está prestes a ser executado. *event* pode ser um dos seguintes:

- `"line"`: Uma nova linha de código será executada.
- `"call"`: Um função está prestes a ser chamada, ou outro bloco de código foi inserido.
- `"retorno"`: Um função ou outro bloco de código está prestes a retornar.
- `"exception"`: Ocorreu uma exceção.

For all the events, specialized functions (see below) are called.

O parâmetro *arg* depende do evento anterior.

Consulte a documentação de **`sys.settrace()`** para mais informações sobre a função de rastreamento. Para mais informações sobre objetos de código e de frame, consulte A hierarquia de tipos padrão.

dispatch_line(*frame*)

Se o depurador deve parar na linha atual, invoca o método **`user_line()`** (que deve ser substituído em subclasses). Levanta a exceção **`BdbQuit`** se o sinalizador **`quitting`** estiver definido (ele pode ser definido a partir de **`user_line()`**). Retorna uma referência ao método **`trace_dispatch()`** para continuar o rastreamento nesse escopo.

dispatch_call(*frame*, *arg*)

Se o depurador deve parar nesta chamada de função, invoca o método `user_call()` (que deve ser substituído em subclasses). Levanta a exceção `BdbQuit` se o sinalizador `quitting` estiver definido (ele pode ser definido a partir de **`user_call()`**). Retorna uma referência ao método `trace_dispatch()` para continuar o rastreamento nesse escopo.

dispatch_return(*frame*, *arg*)

Se o depurador deve parar no retorno desta função, invoca o método `user_return()` (que deve ser substituído em subclasses). Levanta a exceção `BdbQuit` se o sinalizador `quitting` estiver definido (ele pode ser definido a partir de **`user_return()`**). Retorna uma referência ao método `trace_dispatch()` para continuar o rastreamento nesse escopo.

dispatch_exception(*frame*, *arg*)

Se o depurador deve parar nesta exceção, invoca o método `user_exception()` (que deve ser substituído em subclasses). Levanta a exceção `BdbQuit` se o sinalizador `quitting` estiver definido (ele pode ser definido a partir de **`user_exception()`**). Retorna uma referência ao método `trace_dispatch()` para continuar o rastreamento nesse escopo.

Normalmente, as classes derivadas não substituem o seguinte métodos, mas podem fazê-lo se quiserem redefinir a definição de parada e de pontos de interrupção.

is_skipped_module(*module_name*)

Retorna `True` se *module_name* corresponder a algum padrão de skip.

stop_here(*frame*)

Retorna `True` se frame estiver abaixo do frame inicial na pilha.

break_here(*frame*)

Retorna `True` se houver um ponto de interrupção efetivo para essa linha.

Verifica se uma linha ou função ponto de interrupção existe e está em vigor. Exclua os pontos de interrupção temporários com base nas informações de `effective()`.

break_anywhere(*frame*)

Retorna `True` se existir algum ponto de interrupção para o nome de arquivo de *frame*'s.

As classes derivadas devem substituir esses métodos para obter controle sobre a operação da depurador.

user_call(*frame*, *argument_list*)

Chamado a partir de `dispatch_call()` se um ponto de interrupção puder parar dentro da função chamada.

argument_list não é mais usado e sempre será `None`. O argumento é mantido por retrocompatibilidade.

user_line(*frame*)

Chamado a partir de `dispatch_line()` quando `stop_here()` ou `break_here()` retornam `True`.

user_return(*frame*, *return_value*)

Chamado a partir de `dispatch_return()` quando `stop_here()` retorna `True`.

user_exception(*frame*, *exc_info*)

Chamado a partir de `dispatch_exception()` quando `stop_here()` retorna `True`.

do_clear(*arg*)

Manipula como um ponto de interrupção deve ser removido quando for temporário.

Esse método deve ser implementado por classes derivadas.

As classes derivadas e os clientes podem chamar os métodos a seguir para afetar o estado de stepping.

set_step()

Parar após uma linha de código.

set_next(*frame*)

Parar na próxima linha no frame fornecido ou abaixo dele.

set_return(*frame*)

Parar ao retornar do frame fornecido.

set_until(*frame*, *lineno=None*)

Parar quando a linha com *lineno* maior que a atual for alcançada ou ao retornar do frame atual.

set_trace([*frame*])

Inicia a depuração a partir de *frame*. Se *frame* não for especificado, a depuração começa a partir do frame do chamador.

Alterado na versão 3.13: `set_trace()` entrará no depurador imediatamente, em vez de na próxima linha de código a ser executada.

set_continue()

Pare somente nos pontos de interrupção ou quando terminar. Se não houver pontos de interrupção, defina o sistema função de rastreamento como `None`.

set_quit()

Defina o atributo **`quitting`** como `True`. Isso levanta `BdbQuit` na próxima chamada de um dos métodos **`dispatch_*()`**.

Classes derivadas e clientes podem chamar os métodos a seguir para manipular pontos de interrupção. Esses métodos retornam uma string contendo uma mensagem de erro se algo der errado, ou `None` se tudo estiver correto.

set_break(*filename*, *lineno*, *temporary=False*, *cond=None*, *funcname=None*)

Define um novo ponto de interrupção. Se a linha *lineno* não existir para o *filename* passado como argumento, retorna uma mensagem de erro. O *filename* deve estar na forma canônica, conforme descrito no método `canonic()`.

clear_break(*filename*, *lineno*)

Exclui os pontos de interrupção em *filename* e *lineno*. Se nenhum tiver sido definido, retorna uma mensagem de erro.

clear_bpbynumber(*arg*)

Exclui o ponto de interrupção cujo índice é *arg* em `Breakpoint.bpbynumber`. Se *arg* não for numérico ou estiver fora do intervalo, retorna uma mensagem de erro.

clear_all_file_breaks(*filename*)

Exclui todos os pontos de interrupção em *filename*. Se nenhum tiver sido definido, retorna uma mensagem de erro.

clear_all_breaks()

Exclui todos os pontos de interrupção existentes. Se nenhum tiver sido definido, retorna uma mensagem de erro.

get_bpbynumber(*arg*)

Retorna o ponto de interrupção especificado pelo número fornecido. Se *arg* for uma string, ele será convertido para um número. Se *arg* for uma string não numérica, se o ponto de interrupção nunca tiver existido ou tiver sido excluído, uma `ValueError` será levantada.

Adicionado na versão 3.2.

get_break(*filename*, *lineno*)

Retorna `True` se houver um ponto de interrupção para *lineno* em *filename*.

get_breaks(*filename*, *lineno*)

Retorna todos os pontos de interrupção para *lineno* em *filename*, ou uma lista vazia se nenhum estiver definido.

get_file_breaks(*filename*)

Retorna todos os pontos de interrupção em *filename*, ou uma lista vazia se nenhum estiver definido.

get_all_breaks()

Retorna todos os pontos de interrupção definidos.

Classes derivadas e clientes podem chamar os métodos a seguir para desabilitar e reiniciar eventos a fim de obter melhor desempenho. Esses métodos só funcionam ao usar o backend `'monitoring'`.

disable_current_event()

Desabilita o evento atual até a próxima vez que **`restart_events()`** for chamada. Isso é útil quando o depurador não está interessado na linha atual.

Adicionado na versão 3.14.

restart_events()

Reinicia todos os eventos desabilitados. Essa função é chamada automaticamente nos métodos `dispatch_*` após a chamada dos métodos `user_*`. Se os métodos `dispatch_*` não forem substituídos, os eventos desabilitados serão reiniciados após cada interação do usuário.

Adicionado na versão 3.14.

Classes derivadas e clientes podem chamar os métodos a seguir para obter uma estrutura de dados que represente um stack trace (situação da pilha de execução).

get_stack(*f*, *t*)

Retorna uma lista de tuplas (frame, lineno) em um stack trace e um tamanho.

O frame chamado mais recentemente é o último da lista. O tamanho é o número de frames abaixo do frame onde o depurador foi invocado.

format_stack_entry(*frame_lineno*, *lprefix=': '*)

Retorna uma string com informações sobre uma entrada da pilha, que é uma tupla `(frame, lineno)`. A string retornada contém:

- O nome de arquivo canônico que contém o quadro.
- O nome da função ou `"<lambda>"`.
- O argumento de entrada.
- O valor de retorno.
- A linha de código (se existir).

Os dois métodos a seguir podem ser chamados pelos clientes para usar um depurador e depurar uma instrução, fornecida como uma string.

run(*cmd*, *globals=None*, *locals=None*)

Depura uma instrução executada por meio da função **`exec()`**. globals assume por padrão **`__main__.__dict__`**, locals assume por padrão globals.

runeval(*expr*, *globals=None*, *locals=None*)

Depura uma expressão executada por meio da função **`eval()`**. globals e locals têm o mesmo significado que em **`run()`**.

runctx(*cmd*, *globals*, *locals*)

Por retrocompatibilidade. Chama o método **`run()`**.

runcall(*func*, /, **args*, ***kwds*)

Depura uma única chamada de função e retorna seu resultado.

Por fim, o módulo define as funções a seguir:

bdb.checkfuncname(*b*, *frame*)

Retorna `True` se devemos interromper aqui, dependendo da forma como o **`Breakpoint`** foi configurado.

If it was set via line number, it checks if **`b.line`** is the same as the one in *frame*. If the breakpoint was set via **`function name`**, we have to check we are in the right *frame* (the right function) and if we are on its first executable line.

bdb.effective(*file*, *line*, *frame*)

Retorna `(active breakpoint, delete temporary flag)` ou `(None, None)` como o ponto de interrupção sobre o qual agir.

The *active breakpoint* is the first entry in **`bplist`** for the (**`file`**, **`line`**) (which must exist) that is **`enabled`**, for which **`checkfuncname()`** is true, and that has neither a false **`condition`** nor positive **`ignore`** count. The *flag*, meaning that a temporary breakpoint should be deleted, is `False` only when the **`cond`** cannot be evaluated (in which case, **`ignore`** count is ignored).

Se essa entrada não existir, será retornado `(None, None)`.

bdb.set_trace()

Inicie a depuração com uma instância de **`Bdb`** a partir do frame do chamador.

`faulthandler` — Dump the Python traceback

Adicionado na versão 3.3.

This module contains functions to dump Python tracebacks explicitly, on a fault, after a timeout, or on a user signal. Call `faulthandler.enable()` to install fault handlers for the `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS`, and `SIGILL` signals. You can also enable them at startup by setting the `PYTHONFAULTHANDLER` environment variable or by using the `-X` `faulthandler` command line option.

The fault handler is compatible with system fault handlers like Apport or the Windows fault handler. The module uses an alternative stack for signal handlers if the **`sigaltstack()`** function is available. This allows it to dump the traceback even on a stack overflow.

The fault handler is called on catastrophic cases and therefore can only use signal-safe functions (e.g. it cannot allocate memory on the heap). Because of this limitation traceback dumping is minimal compared to normal Python tracebacks:

- Only ASCII is supported. The `backslashreplace` error handler is used on encoding.
- Each string is limited to 500 characters.
- Only the filename, the function name and the line number are displayed. (no source code)
- It is limited to 100 frames per thread, and 100 threads (configurable via *max_threads*).
- The order is reversed: the most recent call is shown first.

By default, the Python traceback is written to `sys.stderr`. To see tracebacks, applications must be run in the terminal. A log file can

alternatively be passed to `faulthandler.enable()`.

The module is implemented in C, so tracebacks can be dumped on a crash or when Python is deadlocked.

The Python Development Mode calls `faulthandler.enable()` at Python startup.

Ver também

Module `pdb`
Interactive source code debugger for Python programs.

Módulo `traceback`
Interface padrão para extrair, formatar e imprimir rastreamentos de pilha de programas Python.

Dumping the traceback

faulthandler.dump_traceback(*file=sys.stderr*, *all_threads=True*, ***, *max_threads=100*)

Dump the tracebacks of all threads into *file*. If *all_threads* is `False`, dump only the current thread. *max_threads* caps the number of threads dumped.

Ver também

`traceback.print_tb()`, which can be used to print a traceback object.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

Alterado na versão 3.15: Added the *max_threads* keyword argument.

Dumping the C stack

Adicionado na versão 3.14.

faulthandler.dump_c_stack(*file=sys.stderr*)

Dump the C stack trace of the current thread into *file*.

If the Python build does not support it or the operating system does not provide a stack trace, then this prints an error in place of a dumped C stack.

C Stack Compatibility

If the system does not support the C-level *backtrace(3)* *[https://manpages.debian.org/backtrace(3)]* or *dladdr1(3)* *[https://manpages.debian.org/dladdr1(3)]*, then C stack dumps will not work. An error will be printed instead of the stack.

Additionally, some compilers do not support CPython's implementation of C stack dumps. As a result, a different error may be printed instead of the stack, even if the operating system supports dumping stacks.

Nota

Dumping C stacks can be arbitrarily slow, depending on the DWARF level of the binaries in the call stack.

Fault handler state

faulthandler.enable(*file=sys.stderr*, *all_threads=True*, *c_stack=True*, *, *max_threads=100*)

Enable the fault handler: install handlers for the `SIGSEGV`, `SIGFPE`, `SIGABRT`, `SIGBUS` and `SIGILL` signals to dump the Python traceback. If

all_threads is `True`, produce tracebacks for every running thread. Otherwise, dump only the current thread.

The *file* must be kept open until the fault handler is disabled: see issue with file descriptors.

If *c_stack* is `True`, then the C stack trace is printed after the Python traceback, unless the system does not support it. See **`dump_c_stack()`** for more information on compatibility.

max_threads caps the number of threads dumped when a fatal signal fires.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

Alterado na versão 3.6: On Windows, a handler for Windows exception is also installed.

Alterado na versão 3.10: The dump now mentions if a garbage collector collection is running if *all_threads* is true.

Alterado na versão 3.14: Only the current thread is dumped if the GIL is disabled to prevent the risk of data races.

Alterado na versão 3.14: The dump now displays the C stack trace if *c_stack* is true.

Alterado na versão 3.15: Added the *max_threads* keyword argument.

faulthandler.disable()

Disable the fault handler: uninstall the signal handlers installed by **`enable()`**.

faulthandler.is_enabled()

Check if the fault handler is enabled.

Dumping the tracebacks after a timeout

faulthandler.dump_traceback_later(*timeout*, *repeat=False*, *file=sys.stderr*, *exit=False*, *, *max_threads=100*)

Dump the tracebacks of all threads, after a timeout of *timeout* seconds, or every *timeout* seconds if *repeat* is `True`. If *exit* is `True`, call **`_exit()`** with status=1 after dumping the tracebacks. (Note **`_exit()`** exits the process immediately, which means it doesn't do any cleanup like flushing file buffers.) If the function is called twice, the new call replaces previous parameters and resets the timeout. The timer has a sub-second resolution. *max_threads* caps the number of threads dumped.

The *file* must be kept open until the traceback is dumped or **`cancel_dump_traceback_later()`** is called: see issue with file descriptors.

This function is implemented using a watchdog thread.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

Alterado na versão 3.7: This function is now always available.

Alterado na versão 3.15: Added the *max_threads* keyword argument.

faulthandler.cancel_dump_traceback_later()

Cancel the last call to **`dump_traceback_later()`**.

Dumping the traceback on a user signal

faulthandler.register(*signum*, *file=sys.stderr*, *all_threads=True*, *chain=False*, *, *max_threads=100*)

Register a user signal: install a handler for the *signum* signal to dump the traceback of all threads, or of the current thread if *all_threads* is

`False`, into *file*. Call the previous handler if chain is `True`. *max_threads* caps the number of threads dumped.

The *file* must be kept open until the signal is unregistered by **`unregister()`**: see issue with file descriptors.

Não disponível no Windows.

Alterado na versão 3.5: Added support for passing file descriptor to this function.

Alterado na versão 3.15: Added the *max_threads* keyword argument.

faulthandler.unregister(*signum*)

Unregister a user signal: uninstall the handler of the *signum* signal installed by **`register()`**. Return `True` if the signal was registered, `False` otherwise.

Não disponível no Windows.

Issue with file descriptors

`enable()`, **`dump_traceback_later()`** and **`register()`** keep the file descriptor of their *file* argument. If the file is closed and its file descriptor is reused by a new file, or if **`os.dup2()`** is used to replace the file descriptor, the traceback will be written into a different file. Call these functions again each time that the file is replaced.

Exemplo

Example of a segmentation fault on Linux with and without enabling the fault handler:

```
$ python -c "import ctypes; ctypes.string_at(0)"
Segmentation fault

$ python -q -X faulthandler
```

```
>>> import ctypes
>>> ctypes.string_at(0)
Fatal Python error: Segmentation fault

Current thread 0x00007fb899f39700 (most recent call first):
  File "/opt/python/Lib/ctypes/__init__.py", line 486 in string_
  File "<stdin>", line 1 in <module>

Current thread's C stack trace (most recent call first):
  Binary file "/opt/python/python", at _Py_DumpStack+0x42 [0x5b2
  Binary file "/opt/python/python", at +0x32dcbd [0x5b27f7d85cbc
  Binary file "/opt/python/python", at +0x32df8a [0x5b27f7d85f8a
  Binary file "/usr/lib/libc.so.6", at +0x3def0 [0x77b73226bef0]
  Binary file "/usr/lib/libc.so.6", at +0x17ef9c [0x77b7323acf9c
  Binary file "/opt/python/build/lib.linux-x86_64-3.15/_ctypes.c
  Binary file "/usr/lib/libffi.so.8", at +0x7976 [0x77b73158f976
  Binary file "/usr/lib/libffi.so.8", at +0x413c [0x77b73158c13c
  Binary file "/usr/lib/libffi.so.8", at ffi_call+0x12e [0x77b73
  Binary file "/opt/python/build/lib.linux-x86_64-3.15/_ctypes.c
  Binary file "/opt/python/build/lib.linux-x86_64-3.15/_ctypes.c
  Binary file "/opt/python/build/lib.linux-x86_64-3.15/_ctypes.c
  Binary file "/opt/python/python", at _PyObject_MakeTpCall+0xce
  Binary file "/opt/python/python", at +0x11bab6 [0x5b27f7b73ab6
  Binary file "/opt/python/python", at PyObject_Vectorcall+0x23
  Binary file "/opt/python/python", at _PyEval_EvalFrameDefault+
  Binary file "/opt/python/python", at +0x2818e6 [0x5b27f7cd98e6
  Binary file "/opt/python/python", at +0x281aab [0x5b27f7cd9aab
  Binary file "/opt/python/python", at PyEval_EvalCode+0xc5 [0x5
  Binary file "/opt/python/python", at +0x255957 [0x5b27f7cad957
  Binary file "/opt/python/python", at +0x255ab4 [0x5b27f7cadab4
  Binary file "/opt/python/python", at _PyEval_EvalFrameDefault+
  Binary file "/opt/python/python", at +0x2818e6 [0x5b27f7cd98e6
  Binary file "/opt/python/python", at +0x281aab [0x5b27f7cd9aab
  Binary file "/opt/python/python", at +0x11b6e1 [0x5b27f7b736e1
  Binary file "/opt/python/python", at +0x11d348 [0x5b27f7b7534
  Binary file "/opt/python/python", at +0x11d626 [0x5b27f7b7562
  Binary file "/opt/python/python", at PyObject_Call+0x20 [0x5b2
  Binary file "/opt/python/python", at +0x32a67a [0x5b27f7d8267a
  Binary file "/opt/python/python", at +0x32a7f8 [0x5b27f7d827f8
  Binary file "/opt/python/python", at +0x32ac1b [0x5b27f7d82c1b
  Binary file "/opt/python/python", at Py_RunMain+0x31 [0x5b27f7
  <truncated rest of calls>
Segmentation fault
```

pdb — O depurador do Python

Código-fonte: Lib/pdb.py [https://github.com/python/cpython/tree/3.15/Lib/pdb.py]

O módulo **pdb** define um depurador de código-fonte interativo para programas Python. Ele possui suporte a definição de pontos de interrupção (condicionais) e passo único no nível da linha de origem, inspeção de quadros de pilha, listagem de código-fonte e avaliação de código Python arbitrário no contexto de qualquer quadro de pilha. Ele também tem suporte a depuração *post-mortem* e pode ser chamado sob controle do programa.

O depurador é extensível – na verdade, ele é definido como a classe **Pdb**. Atualmente, isso não está documentado, mas é facilmente compreendido pela leitura do código-fonte. A interface de extensão usa os módulos **bdb** e **cmd**.

Ver também

Módulo faulthandler
Usado para despejar tracebacks (situação da pilha de execução) do Python explicitamente, em uma falha, após um tempo limite ou em um sinal do usuário.

Módulo traceback
Interface padrão para extrair, formatar e imprimir rastreamentos de pilha de programas Python.

O uso típico para entrar no depurador é inserir:

```
import pdb; pdb.set_trace()
```

ou:

```
breakpoint()
```

no local que você deseja entrar o depurador e, em seguida, executar o programa. Você pode percorrer o código seguindo esta instrução e continuar executando sem o depurador usando o comando **continue**.

Alterado na versão 3.7: A função embutida **breakpoint()**, quando chamada com valores padrão, pode ser usada em vez de `import pdb; pdb.set_trace()`.

```
def double(x):
   breakpoint()
   return x * 2
val = 3
print(f"{val} * 2 is {double(val)}")
```

O prompt do depurador é `(Pdb)`, que é o indicador de que você está no modo de depuração:

```
> ...(2)double()
-> breakpoint()
(Pdb) p x
3
(Pdb) continue
3 * 2 is 6
```

Alterado na versão 3.3: O preenchimento por tabulação através do módulo **readline** está disponível para comandos e argumentos de comando, por exemplo os nomes globais e locais atuais são oferecidos como argumentos do comando `p`.

Interface de linha de comando

Você também pode invocar **pdb** na linha de comando para depurar outros scripts. Por exemplo:

```
python -m pdb [-c comando] (-m módulo | -p pid | arquivo_python)
```

Quando invocado como um módulo, o pdb entra automaticamente na depuração *post-mortem* se o programa que está sendo depurado for

encerrado de forma anormal. Após a depuração *post-mortem* (ou após a saída normal do programa), o pdb reiniciará o programa. A reinicialização automática preserva o estado do pdb (p.ex., pontos de interrupção) e, na maioria dos casos, é mais útil do que encerrar o depurador na saída do programa.

-c, --command <command>

Para executar comandos como se fossem dados em um arquivo `.pdbrc`. Veja Comandos de depuração.

Alterado na versão 3.2: Adicionada a opção `-c`.

-m <module>

Para executar módulos de maneira similar ao `python -m`. Assim como em um script, o depurador pausará a execução logo antes da primeira linha do módulo.

Alterado na versão 3.7: Adicionada a opção `-m`.

-p, --pid <pid>

Anexa ao processo com o PID especificado.

Adicionado na versão 3.14.

Para anexar a um processo Python em execução para depuração remota, use a opção `-p` ou `--pid` com o PID do processo de destino:

```
python -m pdb -p 1234
```

Nota

Anexar a um processo remoto bloqueado em uma chamada de sistema ou aguardando E/S só funcionará quando a próxima instrução de bytecode for executada ou quando o processo receber um sinal.

O uso típico para executar uma instrução sob o controle do depurador é:

```
>>> import pdb
>>> def f(x):
...     print(1 / x)
>>> pdb.run("f(2)")
> <string>(1)<module>()
(Pdb) continue
0.5
>>>
```

O uso típico para inspecionar um programa com falha é:

```
>>> import pdb
>>> def f(x):
...     print(1 / x)
...
>>> f(0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<stdin>", line 2, in f
ZeroDivisionError: division by zero
>>> pdb.pm()
> <stdin>(2)f()
(Pdb) p x
0
(Pdb)
```

Alterado na versão 3.13: A implementação da **PEP 667** [https://peps.python.org/pep-0667/] significa que atribuições de nomes feitas via `pdb` afetarão imediatamente o escopo ativo, mesmo quando executado dentro de um escopo otimizado.

O módulo define as seguintes funções; cada uma entra no depurador de uma maneira ligeiramente diferente:

pdb.run(*statement*, *globals=None*, *locals=None*)

Executa a instrução *statement* (fornecida como uma string ou um objeto código) sob controle do depurador. O prompt do depurador aparece antes de qualquer código ser executado; você pode definir pontos de interrupção e digitar **`continue`** ou pode percorrer a instrução usando **`step`** ou **`next`** (todos esses comandos são explicados abaixo). Os

argumentos opcionais *globals* e *locals* especificam o ambiente em que o código é executado; por padrão, o dicionário do módulo `__main__` é usado. (Veja a explicação das funções embutidas `exec()` ou `eval()`.)

pdb.runeval(*expression*, *globals=None*, *locals=None*)

Avalia a expressão *expression* (fornecida como uma string ou um objeto código) sob controle do depurador. Quando `runeval()` retorna, ele retorna o valor de *expression*. Caso contrário, esta função é semelhante a `run()`.

pdb.runcall(*function*, **args*, ***kwds*)

Chama a função *function* (um objeto função ou método, não uma string) com os argumentos fornecidos. Quando `runcall()` retorna, ele retorna qualquer que seja a chamada de função retornada. O prompt do depurador aparece assim que a função é inserida.

pdb.set_trace(*, *header=None*, *commands=None*)

Entra no depurador no quadro da pilha de chamada. Isso é útil para codificar um ponto de interrupção em um determinado ponto de um programa, mesmo que o código não esteja sendo depurado de outra forma (por exemplo, quando uma asserção falha). Se fornecido, *header* é impresso no console imediatamente antes do início da depuração. O argumento *commands*, se fornecido, é uma lista de comandos para executar quando o depurador é iniciado.

Alterado na versão 3.7: O argumento somente-nomeado *header*.

Alterado na versão 3.13: `set_trace()` entrará no depurador imediatamente, em vez da próxima linha de código a ser executada.

Adicionado na versão 3.14: O argumento *commands*.

awaitable pdb.set_trace_async(*, *header=None*, *commands=None*)

Versão async de `set_trace()`. Esta função deve ser usada dentro de uma função async com `await`.

```
async def f():
    await pdb.set_trace_async()
```

As instruções **`await`** são suportadas se o depurador for invocado por esta função.

Adicionado na versão 3.14.

pdb.post_mortem(*t=None*)

Entra em depuração post-mortem da exceção fornecida ou objeto traceback. Se nenhum valor for fornecido, usa a exceção que está sendo tratada no momento ou levanta `ValueError` se não houver nenhuma.

Alterado na versão 3.13: Adiciona suporte a objetos exceção.

pdb.pm()

Entra a depuração post-mortem da exceção encontrada em **`sys.last_exc`**.

pdb.set_default_backend(*backend*)

Há dois backends suportados para pdb: `'settrace'` e `'monitoring'`. Consulte **`bdb.Bdb`** para obter detalhes. O usuário pode definir o backend padrão a ser usado caso nenhum seja especificado ao instanciar **`Pdb`**. Se nenhum backend for especificado, o padrão será `'settrace'`.

Nota

`breakpoint()` e **`set_trace()`** não serão afetados por esta função. Eles sempre usam o backend `'monitoring'`.

Adicionado na versão 3.14.

pdb.get_default_backend()

Retorna o backend padrão para pdb.

Adicionado na versão 3.14.

As funções `run*` e a `set_trace()` são aliases, ou apelidos, para instanciar a classe `Pdb` e chamar o método com o mesmo nome. Se você deseja acessar outros recursos, faça você mesmo:

class pdb.Pdb(*completekey='tab', stdin=None, stdout=None, skip=None, nosigint=False, readrc=True, mode=None, backend=None, colorize=False*)

`Pdb` é a classe do depurador.

Os argumentos *completekey*, *stdin* e *stdout* são passados para a classe subjacente `cmd.Cmd`; veja a descrição lá.

O argumento *skip*, se fornecido, deve ser um iterável de padrões de nome de módulo no estilo glob. O depurador não entrará nos quadros que se originam em um módulo que corresponde a um desses padrões. [[1]]

Por padrão, o Pdb define um manipulador para o sinal SIGINT (que é enviado quando o usuário pressiona `Ctrl-C` no console) quando você dá um comando `continue`. Isso permite que você entre no depurador novamente pressionando `Ctrl-C`. Se você deseja que o Pdb não toque no manipulador SIGINT, defina *nosigint* como verdadeiro.

O argumento *readrc* é padronizado como verdadeiro e controla se o Pdb carregará arquivos .pdbrc do sistema de arquivos.

O argumento *mode* especifica como o depurador foi invocado. Ele afeta o funcionamento de alguns comandos do depurador. Os valores válidos são `'inline'` (usado pelo comando embutido breakpoint()), `'cli'` (usado pela invocação da linha de comando) ou `None` (para comportamento retrocompatível, como antes da adição do argumento *mode*).

O argumento *backend* especifica o backend a ser usado pelo depurador. Se `None` for passado, o backend padrão será usado. Consulte `set_default_backend()`. Caso contrário, os backends implementados são `'settrace'` e `'monitoring'`.

O argumento *colorize*, se definido como `True`, habilitará a saída colorida no depurador, se houver suporte para cores. Isso destacará o código-fonte exibido no pdb.

Exemplo de chamada para habilitar rastreamento com *skip*:

```
import pdb; pdb.Pdb(skip=['django.*']).set_trace()
```

Levanta um evento de auditoria `pdb.Pdb` sem argumentos.

Alterado na versão 3.1: Adicionado o parâmetro *skip*.

Alterado na versão 3.2: Adicionado o parâmetro *nosigint*. Anteriormente, um manipulador de SIGINT nunca era definido por Pdb.

Alterado na versão 3.6: O argumento *readrc*.

Adicionado na versão 3.14: Adicionado o argumento *mode*.

Adicionado na versão 3.14: Adicionado o argumento *backend*.

Adicionado na versão 3.14: Adicionado o argumento *colorize*.

Alterado na versão 3.14: Pontos de interrupção em linha como `breakpoint()` ou `pdb.set_trace()` sempre interromperão o programa ao chamar o quadro, ignorando o padrão *skip* (se houver).

run(*statement*, *globals=None*, *locals=None*)
runeval(*expression*, *globals=None*, *locals=None*)
runcall(*function*, **args*, ***kwds*)
set_trace()

Consulte a documentação para as funções explicadas acima.

Comandos de depuração

Os comandos reconhecidos pelo depurador estão listados abaixo. A maioria dos comandos pode ser abreviada para uma ou duas letras, conforme

indicado; por exemplo, `h(elp)` significa que `h` ou `help` podem ser usados para inserir o comando de ajuda (mas não `he` ou `hel`, nem `H` ou `Help` ou `HELP`). Os argumentos para os comandos devem ser separados por espaços em branco (espaços ou tabulações). Os argumentos opcionais estão entre colchetes (`[]`) na sintaxe do comando; os colchetes não devem ser digitados. As alternativas na sintaxe de comando são separadas por uma barra vertical (`|`).

Digitar uma linha em branco repete o último comando digitado. Exceção: se o último comando foi um comando **`list`**, as próximas 11 linhas serão listadas.

Os comandos que o depurador não reconhece são presumidos como instruções Python e são executados no contexto do programa que está sendo depurado. As instruções Python também podem ser prefixadas com um ponto de exclamação (`!`). Essa é uma maneira poderosa de inspecionar o programa que está sendo depurado; é até possível alterar uma variável ou chamar uma função. Quando ocorre uma exceção em uma instrução, o nome da exceção é impresso, mas o estado do depurador não é alterado.

Alterado na versão 3.13: Expressões/instruções, cujo prefixo é um comando pdb, agora são identificadas e executadas corretamente.

O depurador possui suporte a aliases. Os aliases podem ter parâmetros que permitem um certo nível de adaptabilidade ao contexto em exame.

Vários comandos podem ser inseridos em uma única linha, separados por `;;`. (Um único `;` não é usado, pois é o separador para vários comandos em uma linha que é passada para o analisador Python.) Nenhuma inteligência é aplicada para separar os comandos; a entrada é dividida no primeiro par `;;`, mesmo que esteja no meio de uma string entre aspas. Uma solução alternativa para strings com caractere de ponto e vírgula duplo é usar a concatenação de string implícita `';''';'` ou `";"";"`.

Para definir uma variável global temporária, use uma *variável de conveniência*. Uma *variável de conveniência* é uma variável cujo nome começa com `$`. Por exemplo, `$foo = 1` define uma variável global `$foo` que você pode usar na sessão do depurador. As *variáveis de conveniência* são

limpas quando o programa retoma a execução, portanto é menos provável que interfira em seu programa em comparação ao uso de variáveis normais como `foo = 1`.

Existem quatro *variáveis de conveniência* predefinidas:

- `$_frame`: o quadro atual que você está depurando
- `$_retval`: o valor de retorno se o quadro estiver retornando
- `$_exception`: a exceção se o quadro estiver levantando uma exceção
- `$_asynctask`: a tarefa asyncio se o pdb parar em uma função assíncrona

Adicionado na versão 3.12: Adicionado o recurso de *variável de conveniência*.

Adicionado na versão 3.14: Adicionada a variável de conveniência `$_asynctask`.

Se um arquivo `.pdbrc` existe no diretório inicial do usuário ou no diretório atual, ele é lido com a codificação `'utf-8'` e executado como se tivesse sido digitado no prompt do depurador, com a exceção de que linhas vazias e linhas iniciando com `#` são ignoradas. Isso é particularmente útil para aliases. Se ambos os arquivos existirem, aquele no diretório inicial será lido primeiro e os aliases definidos poderão ser substituídos pelo arquivo local.

Alterado na versão 3.2: `.pdbrc` agora pode conter comandos que continuam a depuração, como **`continue`** ou **`next`**. Anteriormente, esses comandos não tinham efeito.

Alterado na versão 3.11: `.pdbrc` agora é lido com a codificação `'utf-8'`. Anteriormente, ele era lido com a codificação da localidade do sistema.

h(elp) [command]

Sem argumento, imprime a lista de comandos disponíveis. Com um *command* como argumento, imprime ajuda sobre esse comando. `help pdb` exibe a documentação completa (a docstring do módulo **`pdb`**). Como o argumento *command* deve ser um identificador, `help exec` deve ser inserido para obter ajuda sobre o comando `!`.

w(here) [count]

Exibe um stack trace (situação da pilha de execução), com o quadro mais recente na parte inferior. Se *count* for 0, exibe a entrada do quadro atual. Se *count* for negativo, exibe os quadros menos recentes - *count*. Se *count* for positivo, exibe os quadros *count* mais recentes. Uma seta (>) indica o quadro atual, que determina o contexto da maioria dos comandos.

Alterado na versão 3.14: o argumento *count* foi adicionado

d(own) [count]

Move os níveis do quadro atual *count* (padrão 1) para baixo no stack trace (para um quadro mais recente).

u(p) [count]

Move os níveis do quadro atual na *count* (padrão 1) para cima no stack trace (para um quadro mais antigo).

b(reak) [([filename:]lineno | function) [, condition]]

Com um argumento *lineno*, define uma quebra na linha *lineno* no arquivo atual. O número da linha pode ser prefixado com *filename* e dois pontos, para especificar um ponto de interrupção em outro arquivo (possivelmente um que ainda não tenha sido carregado). O arquivo é pesquisado em `sys.path`. Formas aceitáveis de *filename* são `/caminhoabsoluto/para/arquivo.py`, `caminhorelativo/do/arquivo.py`, `módulo` e `pacote.módulo`.

Com um argumento *function*, define uma interrupção na primeira instrução executável dentro dessa função. *function* pode ser qualquer expressão avaliada como uma função no espaço de nomes atual.

Se um segundo argumento estiver presente, é uma expressão que deve ser avaliada como verdadeira antes que o ponto de interrupção seja respeitado.

Sem argumento, lista todas as quebras, inclusive para cada ponto de interrupção, o número de vezes que o ponto de interrupção foi atingido,

a contagem atual de ignorados e a condição associada, se houver.

Cada ponto de interrupção recebe um número ao qual todos os outros comandos de ponto de interrupção se referem.

tbreak [([filename:]lineno | function) [, condition]]

Ponto de interrupção temporário, que é removido automaticamente quando é atingido pela primeira vez. Os argumentos são os mesmos que para **break**.

cl(ear) [filename:lineno | bpnumber ...]

Com um argumento *filename:lineno*, limpa todos os pontos de interrupção nessa linha. Com uma lista separada por espaços de números de ponto de interrupção, limpa esses pontos de interrupção. Sem argumento, limpa todas as quebras (mas primeiro pede a confirmação).

disable bpnumber [bpnumber ...]

Desativa os pontos de interrupção fornecidos como uma lista separada por espaços de números de pontos de interrupção. Desabilitar um ponto de interrupção significa que ele não pode interromper a execução do programa, mas, ao contrário de limpar um ponto de interrupção, ele permanece na lista de pontos de interrupção e pode ser (re)ativado.

enable bpnumber [bpnumber ...]

Ativa o ponto de interrupção especificado.

ignore bpnumber [count]

Define a contagem de ignorados para o número do ponto de interrupção especificado. Se *count* for omitida, a contagem de ignorados será definida como 0. Um ponto de interrupção se torna ativo quando a contagem de ignorados é zero. Quando diferente de zero, a contagem é decrementada cada vez que o ponto de interrupção é atingido e o ponto de interrupção não é desativado e qualquer condição associada é avaliada como verdadeira.

condition bpnumber [condition]

Define uma nova *condition* para o ponto de interrupção, uma expressão que deve ser avaliada como verdadeira antes que o ponto de interrupção seja respeitado. Se *condition* for omitida, qualquer condição existente será removida; isto é, o ponto de interrupção é tornado incondicional.

commands [bpnumber]

Especifica uma lista de comandos para o número do ponto de interrupção *bpnumber*. Os próprios comandos aparecem nas linhas seguintes. Digite em uma linha contendo apenas `end` para finalizar os comandos. Um exemplo:

```
(Pdb) commands 1
(com) p alguma_variável
(com) end
(Pdb)
```

Para remover todos os comandos de um ponto de interrupção, digite `commands` e siga-o imediatamente com `end`; isto é, não dê comandos.

Sem o argumento *bpnumber*, `commands` refere-se ao ponto de interrupção definido mais recentemente que ainda existe.

Você pode usar comandos de ponto de interrupção para iniciar seu programa novamente. Simplesmente use o comando **`continue`**, ou **`step`**, ou qualquer outro comando que reinicie a execução.

Especificar qualquer comando que retome a execução (atualmente **`continue`**, **`step`**, **`next`**, **`return`**, **`until`**, **`jump`**, **`quit`** e suas abreviações) finaliza a lista de comandos (como se esse comando fosse imediatamente seguido pelo end). Isso ocorre porque sempre que você retoma a execução (mesmo com um simples next ou step), você pode encontrar outro ponto de interrupção — que pode ter sua própria lista de comandos, levando a ambiguidades sobre qual lista executar.

Se a lista de comandos contiver o comando `silent` ou um comando que retome a execução, a mensagem de ponto de interrupção contendo informações sobre o quadro não será exibida.

Alterado na versão 3.14: As informações do quadro não serão exibidas se um comando que retome a execução estiver presente na lista de comandos.

s(tep)

Executa a linha atual, interrompe na primeira ocasião possível (em uma função chamada ou na próxima linha na função atual).

n(ext)

Continua a execução até que a próxima linha na função atual seja atingida ou ela retorne. (A diferença entre `next` e `step` é que `step` para dentro de uma função chamada, enquanto `next` executa funções chamadas em (quase) velocidade máxima, parando apenas na próxima linha da função atual.)

unt(il) [lineno]

Sem argumento, continua a execução até que a linha com um número maior que o atual seja atingida.

Com *lineno*, continua a execução até que uma linha com um número maior ou igual a *lineno* ser alcançada. Nos dois casos, também interrompe quando o quadro atual retornar.

Alterado na versão 3.2: Permite fornecer um número de linha explícito.

r(eturn)

Continua a execução até que a função atual retorne.

c(ont(inue))

Continua a execução, interrompe apenas quando um ponto de interrupção for encontrado.

j(ump) lineno

Define a próxima linha que será executada. Disponível apenas no quadro mais inferior. Isso permite voltar e executar o código novamente ou avançar para pular o código que você não deseja executar.

Deve-se notar que nem todos os saltos são permitidos – por exemplo, não é possível pular para o meio de um loop de `for` ou sair de uma cláusula `finally`.

l(ist) [first[, last]]

Lista o código-fonte do arquivo atual. Sem argumentos, lista 11 linhas ao redor da linha atual ou continue a listagem anterior. Com `.` como argumento, lista 11 linhas ao redor da linha atual. Com um argumento, lista 11 linhas nessa linha. Com dois argumentos, lista o intervalo especificado; se o segundo argumento for menor que o primeiro, ele será interpretado como uma contagem.

A linha atual no quadro atual é indicada por `->`. Se uma exceção estiver sendo depurada, a linha em que a exceção foi originalmente gerada ou propagada é indicada por `>>`, se for diferente da linha atual.

Alterado na versão 3.2: Adicionado o marcador `>>`.

ll | longlist

Lista todo o código-fonte da função ou quadro atual. As linhas interessantes estão marcadas como para `list`.

Adicionado na versão 3.2.

a(rgs)

Imprime os argumentos da função atual e seus valores atuais.

p expression

Avalia *expression* no contexto atual e imprima seu valor.

Nota

`print()` também pode ser usado, mas não é um comando de depuração — isso executa a função Python `print()`.

pp expression

Como o comando **p**, exceto que o valor de *expression* é bastante impresso usando o módulo **pprint**.

whatis expression

Exibe o tipo de *expression*.

source expression

Tenta obter o código-fonte de *expression* e exibe-o.

Adicionado na versão 3.2.

display [expression]

Exibe o valor de *expression* caso ela tenha sido alterada, sempre que a execução for interrompida no quadro atual.

Sem *expression*, lista todas as expressões de exibição para o quadro atual.

Nota

Display avalia *expression* e compara com o resultado da avaliação anterior de *expression*, portanto, quando o resultado é mutável, display pode não ser capaz de captar as alterações.

Exemplo:

```
lst = []
breakpoint()
pass
lst.append(1)
print(lst)
```

Display não perceberá que `lst` foi alterado porque o resultado da avaliação foi modificado internamente por `lst.append(1)` antes de ser comparado:

```
> example.py(3)<module>()
-> pass
(Pdb) display lst
```

```
display lst: []
(Pdb) n
> example.py(4)<module>()
-> lst.append(1)
(Pdb) n
> example.py(5)<module>()
-> print(lst)
(Pdb)
```

Você pode fazer alguns truques com o mecanismo de cópia para fazê-lo funcionar:

```
> example.py(3)<module>()
-> pass
(Pdb) display lst[:]
display lst[:]: []
(Pdb) n
> example.py(4)<module>()
-> lst.append(1)
(Pdb) n
> example.py(5)<module>()
-> print(lst)
display lst[:]: [1]  [old: []]
(Pdb)
```

Adicionado na versão 3.2.

undisplay [expression]

Não exibe mais *expression* no quadro atual. Sem expressão, limpa todas as expressões de exibição para o quadro atual.

Adicionado na versão 3.2.

interact

Inicia um interpretador interativo (usando o módulo `code`) em um novo espaço de nomes global inicializado a partir dos espaços de nomes locais e globais para o escopo atual. Use `exit()` ou `quit()` para sair do interpretador e retornar ao depurador.

Nota

Como `interact` cria um novo espaço de nomes dedicado para execução de código, atribuições a variáveis não afetarão os espaços de nomes originais. No entanto, as modificações em quaisquer objetos mutáveis referenciados serão refletidas nos espaços de nomes originais, como de costume.

Adicionado na versão 3.2.

Alterado na versão 3.13: `exit()` e `quit()` podem ser usados para sair do comando **`interact`**.

Alterado na versão 3.13: **`interact`** direciona sua saída para o canal de saída do depurador em vez de **`sys.stderr`**.

alias [name [command]]

Cria um apelido chamado *name* que executa *command*. O comando *command não* deve ser colocado entre aspas. Parâmetros substituíveis podem ser indicados por `%1`, `%2`, … e `%9`, enquanto `%*` é substituído por todos os parâmetros. Se *command* for omitido, o apelido atual para *name* será mostrado. Se nenhum argumento for fornecido, todos os apelidos serão listados.

Os aliases podem ser aninhados e podem conter qualquer coisa que possa ser digitada legalmente no prompt do pdb. Observe que os comandos internos do pdb *podem* ser substituídos por aliases. Esse comando é oculto até que o alias seja removido. O alias é aplicado recursivamente à primeira palavra da linha de comando; todas as outras palavras da linha são deixadas em paz.

Como exemplo, aqui estão dois aliases úteis (especialmente quando colocados no arquivo `.pdbrc`):

```
# Exibe variáveis de instância (use em "pi classInst")
alias pi for k in %1.__dict__.keys(): print(f"%1.{k} = {%1.__
# Exibe variáveis de instância em self
alias ps pi self
```

unalias name

Executa o alias *name* especificado.

! statement

Executa a instrução *statement* (de uma só linha) no contexto do quadro de pilha atual. O ponto de exclamação pode ser omitido, a menos que a primeira palavra da instrução seja semelhante a um comando de depuração, por exemplo:

```
(Pdb) ! n=42
(Pdb)
```

Para definir uma variável global, você pode prefixar o comando de atribuição com uma instrução **`global`** na mesma linha, por exemplo:

```
(Pdb) global list_options; list_options = ['-l']
(Pdb)
```

run [args ...]
restart [args ...]

Reinicia o programa Python depurado. Se *args* for fornecido, ele é dividido com **`shlex`** e o resultado é usado como o novo **`sys.argv`**. Histórico, pontos de interrupção, ações e opções do depurador são preservados. **`restart`** é um apelido para **`run`**.

Alterado na versão 3.14: Os comandos **`run`** e **`restart`** são desabilitados quando o depurador é invocado no modo `'inline'`.

q(uit)

Sai do depurador. O programa que está sendo executado é abortado. Uma entrada com fim de arquivo equivale a **`quit`**.

Um prompt de confirmação será exibido se o depurador for invocado no modo `'inline'`. `y`, `Y`, `<Enter>` ou `EOF` confirmarão o encerramento.

Alterado na versão 3.14: Um prompt de confirmação será exibido se o depurador for invocado no modo `'inline'`. Após a confirmação, o

depurador chamará `sys.exit()` imediatamente, em vez de levantar `bdb.BdbQuit` no próximo evento de rastreamento.

debug code

Entra em um depurador recursivo que percorre *code* (que é uma expressão ou instrução arbitrária a ser executada no ambiente atual).

retval

Exibe o valor de retorno para o último retorno de a função atual.

exceptions [excnumber]

Lista ou salta entre exceções encadeadas.

Ao usar `pdb.pm()` ou `Pdb.post_mortem(...)` com uma exceção encadeada em vez de um traceback, permite ao usuário mover entre as exceções encadeadas usando o comando `exceptions` para listar exceções e `exceptions <número>` para mudar para essa exceção.

Exemplo:

```
def out():
    try:
        middle()
    except Exception as e:
        raise ValueError("reraise middle() error") from e

def middle():
    try:
        return inner(0)
    except Exception as e:
        raise ValueError("Middle fail")

def inner(x):
    1 / x

out()
```

chamar `pdb.pm()` permitirá mover entre exceções:

```
> example.py(5)out()
-> raise ValueError("reraise middle() error") from e

(Pdb) exceptions
  0 ZeroDivisionError('division by zero')
  1 ValueError('Middle fail')
> 2 ValueError('reraise middle() error')

(Pdb) exceptions 0
> example.py(16)inner()
-> 1 / x

(Pdb) up
> example.py(10)middle()
-> return inner(0)
```

Adicionado na versão 3.13.

Notas de rodapé

[1]

Se um quadro é considerado originário de um determinado módulo é determinado pelo `__name__` nos globais do quadro.

`profiling` — Python profilers

Adicionado na versão 3.15.

Source code: Lib/profiling/ [https://github.com/python/cpython/tree/3.15/Lib/profiling/]

Introduction to profiling

A *profile* is a set of statistics that describes how often and for how long various parts of a program execute. These statistics help identify performance bottlenecks and guide optimization efforts. Python provides two fundamentally different approaches to collecting this information: statistical sampling and deterministic tracing.

The `profiling` package organizes Python's built-in profiling tools under a single namespace. It contains two submodules, each implementing a different profiling methodology:

`profiling.sampling`
: A statistical profiler that periodically samples the call stack. Run scripts directly or attach to running processes by PID. Provides multiple output formats (flame graphs, heatmaps, Firefox Profiler), GIL analysis, GC tracking, and multiple profiling modes (wall-clock, CPU, GIL) with virtually no overhead.

`profiling.tracing`
: A deterministic profiler that traces every function call, return, and exception event. Provides exact call counts and precise timing information, capturing every invocation including very fast functions.

Nota

The profiler modules are designed to provide an execution profile for a given program, not for benchmarking purposes. For benchmarking, use

the `timeit` module, which provides reasonably accurate timing measurements. This distinction is particularly important when comparing Python code against C code: deterministic profilers introduce overhead for Python code but not for C-level functions, which can skew comparisons.

Choosing a profiler

For most performance analysis, use the statistical profiler (`profiling.sampling`). It has minimal overhead, works for both development and production, and provides rich visualization options including flame graphs, heatmaps, GIL analysis, and more.

Use the deterministic profiler (`profiling.tracing`) when you need **exact call counts** and cannot afford to miss any function calls. Since it instruments every function call and return, it will capture even very fast functions that complete between sampling intervals. The tradeoff is higher overhead.

The following table summarizes the key differences:

Recurso	**Statistical sampling (`profiling.sampling`)**	**Deterministic (`profiling.tracing`)**
Overhead	Virtually none	Moderate
Accuracy	Statistical estimate	Exact call counts
Output formats	pstats, flame graph, heatmap, gecko, collapsed	pstats

Recurso	Statistical sampling (`profiling.sampling`)	Deterministic (`profiling.tracing`)
Profiling modes	Wall-clock, CPU, GIL	Wall-clock
Special frames	GC, native (C extensions)	N/D
Attach to PID	Yes	No

When to use statistical sampling

The statistical profiler (`profiling.sampling`) is recommended for most performance analysis tasks. Use it the same way you would use `profiling.tracing`:

```
python -m profiling.sampling run script.py
```

One of the main strengths of the sampling profiler is its variety of output formats. Beyond traditional pstats tables, it can generate interactive flame graphs that visualize call hierarchies, line-level source heatmaps that show exactly where time is spent in your code, and Firefox Profiler output for timeline-based analysis.

The profiler also provides insight into Python interpreter behavior that deterministic profiling cannot capture. Use `--mode gil` to identify GIL contention in multi-threaded code, `--mode cpu` to measure actual CPU time excluding I/O waits, or inspect `<GC>` frames to understand garbage collection overhead. The `--native` option reveals time spent in C extensions, helping distinguish Python overhead from library performance.

For multi-threaded applications, the `-a` option samples all threads simultaneously, showing how work is distributed. And for production debugging, the `attach` command connects to any running Python process by PID without requiring a restart or code changes.

When to use deterministic tracing

The deterministic profiler (`profiling.tracing`) instruments every function call and return. This approach has higher overhead than sampling, but guarantees complete coverage of program execution.

The primary reason to choose deterministic tracing is when you need exact call counts. Statistical profiling estimates frequency based on sampling, which may undercount short-lived functions that complete between samples. If you need to verify that an optimization actually reduced the number of function calls, or if you want to trace the complete call graph to understand caller-callee relationships, deterministic tracing is the right choice.

Deterministic tracing also excels at capturing functions that execute in microseconds. Such functions may not appear frequently enough in statistical samples, but deterministic tracing records every invocation regardless of duration.

Quick start

This section provides the minimal steps needed to start profiling. For complete documentation, see the dedicated pages for each profiler.

Statistical profiling

To profile a script, use the `profiling.sampling` module with the `run` command:

```
python -m profiling.sampling run script.py
python -m profiling.sampling run -m mypackage.module
```

This runs the script under the profiler and prints a summary of where time was spent. For an interactive flame graph:

```
python -m profiling.sampling run --flamegraph script.py
```

To profile an already-running process, use the `attach` command with the process ID:

```
python -m profiling.sampling attach 1234
```

For custom settings, specify the sampling interval (in microseconds) and duration (in seconds):

```
python -m profiling.sampling run -i 50 -d 30 script.py
```

Deterministic profiling

To profile a script from the command line:

```
python -m profiling.tracing script.py
```

To profile a piece of code programmatically:

```
import profiling.tracing
profiling.tracing.run('my_function()')
```

This executes the given code under the profiler and prints a summary showing exact function call counts and timing.

Understanding profile output

Both profilers collect function-level statistics, though they present them in different formats. The sampling profiler offers multiple visualizations (flame graphs, heatmaps, Firefox Profiler, pstats tables), while the deterministic profiler produces pstats-compatible output. Regardless of format, the underlying concepts are the same.

Key profiling concepts:

Direct time (also called *self time* or *tottime*)
: Time spent executing code in the function itself, excluding time spent in functions it called. High direct time indicates the function contains expensive operations.

Cumulative time (also called *total time* or *cumtime*)
: Time spent in the function and all functions it called. This measures the total cost of calling a function, including its entire call subtree.

Call count (also called *ncalls* or *samples*)
: How many times the function was called (deterministic) or sampled (statistical). In deterministic profiling, this is exact. In statistical profiling, it represents the number of times the function appeared in a stack sample.

Primitive calls
: Calls that are not induced by recursion. When a function recurses, the total call count includes recursive invocations, but primitive calls counts only the initial entry. Displayed as `total/primitive` (for example, `3/1` means three total calls, one primitive).

Caller/Callee relationships
: Which functions called a given function (callers) and which functions it called (callees). Flame graphs visualize this as nested rectangles; pstats can display it via the **`print_callers()`** and **`print_callees()`** methods.

Legacy compatibility

For backward compatibility, the `cProfile` module remains available as an alias to **`profiling.tracing`**. Existing code using `import cProfile` will continue to work without modification in all future Python versions.

Descontinuado desde a versão 3.15: The pure Python **`profile`** module is deprecated and will be removed in Python 3.17. Use **`profiling.tracing`** (or its alias `cProfile`) instead. See **`profile`** for migration guidance.

Ver também

`profiling.sampling`
: Statistical sampling profiler with flame graphs, heatmaps, and GIL analysis. Recommended for most users.

`profiling.tracing`
: Deterministic tracing profiler for exact call counts.

`pstats`
: Statistics analysis and formatting for profile data.

`timeit`
: Module for measuring execution time of small code snippets.

Submódulos

- `profiling.tracing` — Deterministic profiler
- `profiling.sampling` — Statistical profiler

`profiling.tracing` — Deterministic profiler

Adicionado na versão 3.15.

Source code: Lib/profiling/tracing/
[https://github.com/python/cpython/tree/3.15/Lib/profiling/tracing/]

The **`profiling.tracing`** module provides deterministic profiling of Python programs. It monitors every function call, function return, and exception event, recording precise timing for each. This approach provides exact call counts and complete visibility into program execution, making it ideal for development and testing scenarios.

Nota

This module is also available as `cProfile` for backward compatibility. The `cProfile` name will continue to work in all future Python versions. Use whichever import style suits your codebase:

```
# Preferred (new style)
import profiling.tracing
profiling.tracing.run('my_function()')

# Also works (backward compatible)
import cProfile
cProfile.run('my_function()')
```

What is deterministic profiling?

Deterministic profiling captures every function call, function return, and exception event during program execution. The profiler measures the

precise time intervals between these events, providing exact statistics about how the program behaves.

In contrast to statistical profiling, which samples the call stack periodically to estimate where time is spent, deterministic profiling records every event. This means you get exact call counts rather than statistical approximations. The trade-off is that instrumenting every event introduces overhead that can slow down program execution.

Python's interpreted nature makes deterministic profiling practical. The interpreter already dispatches events for function calls and returns, so the profiler can hook into this mechanism without requiring code modification. The overhead tends to be moderate relative to the inherent cost of interpretation, making deterministic profiling suitable for most development workflows.

Deterministic profiling helps answer questions like:

- How many times was this function called?
- What is the complete call graph of my program?
- Which functions are called by a particular function?
- Are there unexpected function calls happening?

Call count statistics can identify bugs (surprising counts) and inline expansion opportunities (high call counts). Internal time statistics reveal "hot loops" that warrant optimization. Cumulative time statistics help identify algorithmic inefficiencies. The handling of cumulative times in this profiler allows direct comparison of recursive and iterative implementations.

Command-line interface

The **`profiling.tracing`** module can be invoked as a script to profile another script or module:

```
python -m profiling.tracing [-o output_file] [-s sort_order] (-m
```

This runs the specified script or module under the profiler and prints the results to standard output (or saves them to a file).

-o <output_file>

Write the profile results to a file instead of standard output. The output file can be read by the **pstats** module for later analysis.

-s <sort_order>

Sort the output by the specified key. This accepts any of the sort keys recognized by **pstats.Stats.sort_stats()**, such as `cumulative`, `time`, `calls`, or `name`. This option only applies when **-o** is not specified.

-m <module>

Profile a module instead of a script. The module is located using the standard import mechanism.

Adicionado na versão 3.7: The `-m` option for `cProfile`.

Adicionado na versão 3.8: The `-m` option for **profile**.

Programmatic usage examples

For more control over profiling, use the module's functions and classes directly.

Basic profiling

The simplest approach uses the **run()** function:

```
import profiling.tracing
profiling.tracing.run('my_function()')
```

This profiles the given code string and prints a summary to standard output. To save results for later analysis:

```
profiling.tracing.run('my_function()', 'output.prof')
```

Using the `Profile` class

The **`Profile`** class provides fine-grained control:

```
import profiling.tracing
import pstats
from io import StringIO

pr = profiling.tracing.Profile()
pr.enable()
# ... code to profile ...
pr.disable()

# Print results
s = StringIO()
ps = pstats.Stats(pr, stream=s).sort_stats(pstats.SortKey.CUMULA
ps.print_stats()
print(s.getvalue())
```

The **`Profile`** class also works as a context manager:

```
import profiling.tracing

with profiling.tracing.Profile() as pr:
    # ... code to profile ...

pr.print_stats()
```

Module reference

profiling.tracing.run(*command*, *filename=None*, *sort=-1*)

Profile execution of a command and print or save the results.

This function executes the *command* string using **`exec()`** in the `__main__` module's namespace:

```
exec(command, __main__.__dict__, __main__.__dict__)
```

If *filename* is not provided, the function creates a `pstats.Stats` instance and prints a summary to standard output. If *filename* is provided, the raw profile data is saved to that file for later analysis with `pstats`.

The *sort* argument specifies the sort order for printed output, accepting any value recognized by `pstats.Stats.sort_stats()`.

profiling.tracing.runctx(*command*, *globals*, *locals*, *filename=None*, *sort=-1*)

Profile execution of a command with explicit namespaces.

Like `run()`, but executes the command with the specified *globals* and *locals* mappings instead of using the `__main__` module's namespace:

```
exec(command, globals, locals)
```

class profiling.tracing.Profile(*timer=None*, *timeunit=0.0*, *subcalls=True*, *builtins=True*)

A profiler object that collects execution statistics.

The optional *timer* argument specifies a custom timing function. If not provided, the profiler uses a platform-appropriate default timer. When supplying a custom timer, it must return a single number representing the current time. If the timer returns integers, use *timeunit* to specify the duration of one time unit (for example, `0.001` for milliseconds).

The *subcalls* argument controls whether the profiler tracks call relationships between functions. The *builtins* argument controls whether built-in functions are profiled.

Alterado na versão 3.8: Added context manager support.

enable()

Start collecting profiling data.

disable()

Stop collecting profiling data.

create_stats()

Stop collecting data and record the results internally as the current profile.

print_stats(*sort=-1*)

Create a `pstats.Stats` object from the current profile and print the results to standard output.

The *sort* argument specifies the sorting order. It accepts a single key or a tuple of keys for multi-level sorting, using the same values as `pstats.Stats.sort_stats()`.

Adicionado na versão 3.13: Support for a tuple of sort keys.

dump_stats(*filename*)

Write the current profile data to *filename*. The file can be read by `pstats.Stats` for later analysis.

run(*cmd*)

Profile the command string via `exec()`.

runctx(*cmd*, *globals*, *locals*)

Profile the command string via `exec()` with the specified namespaces.

runcall(*func*, */*, **args*, ***kwargs*)

Profile a function call. Returns whatever *func* returns:

```
result = pr.runcall(my_function, arg1, arg2, keyword=valu
```

Nota

Profiling requires that the profiled code returns normally. If the interpreter terminates (for example, via `sys.exit()`) during profiling, no results will

be available.

Using a custom timer

The `Profile` class accepts a custom timing function, allowing you to measure different aspects of execution such as wall-clock time or CPU time. Pass the timing function to the constructor:

```
pr = profiling.tracing.Profile(my_timer_function)
```

The timer function must return a single number representing the current time. If it returns integers, also specify *timeunit* to indicate the duration of one unit:

```
# Timer returns time in milliseconds
pr = profiling.tracing.Profile(my_ms_timer, 0.001)
```

For best performance, the timer function should be as fast as possible. The profiler calls it frequently, so timer overhead directly affects profiling overhead.

The `time` module provides several functions suitable for use as custom timers:

- `time.perf_counter()` for high-resolution wall-clock time
- `time.process_time()` for CPU time (excluding sleep)
- `time.monotonic()` for monotonic clock time

Limitations

Deterministic profiling has inherent limitations related to timing accuracy.

The underlying timer typically has a resolution of about one millisecond. Measurements cannot be more accurate than this resolution. With enough measurements, timing errors tend to average out, but individual measurements may be imprecise.

There is also latency between when an event occurs and when the profiler captures the timestamp. Similarly, there is latency after reading the timestamp before user code resumes. Functions called frequently accumulate this latency, which can make them appear slower than they actually are. This error is typically less than one clock tick per call but can become significant for functions called many times.

The **`profiling.tracing`** module (and its `cProfile` alias) is implemented as a C extension with low overhead, so these timing issues are less pronounced than with the deprecated pure Python **`profile`** module.

Ver também

`profiling`
: Overview of Python profiling tools and guidance on choosing a profiler.

`profiling.sampling`
: Statistical sampling profiler for production use.

`pstats`
: Statistics analysis and formatting for profile data.

`profile`
: Deprecated pure Python profiler (includes calibration documentation).

`profiling.sampling` — Statistical profiler

Adicionado na versão 3.15.

Source code: [Lib/profiling/sampling/](https://github.com/python/cpython/tree/3.15/Lib/profiling/sampling/)
[https://github.com/python/cpython/tree/3.15/Lib/profiling/sampling/]

The **`profiling.sampling`** module, named **Tachyon**, provides statistical profiling of Python programs through periodic stack sampling. Tachyon can run scripts directly or attach to any running Python process without requiring code changes or restarts. Because sampling occurs externally to the target process, overhead is virtually zero, making Tachyon suitable for both development and production environments.

What is statistical profiling?

Statistical profiling builds a picture of program behavior by periodically capturing snapshots of the call stack. Rather than instrumenting every

function call and return as deterministic profilers do, Tachyon reads the call stack at regular intervals to record what code is currently running.

This approach rests on a simple principle: functions that consume significant CPU time will appear frequently in the collected samples. By gathering thousands of samples over a profiling session, Tachyon constructs an accurate statistical estimate of where time is spent. The more samples collected, the more precise this estimate becomes.

The following interactive visualization demonstrates how sampling profiling works. Press **Play** to watch a Python program execute, and observe how the profiler periodically captures snapshots of the call stack. Adjust the **sample interval** to see how sampling frequency affects the results.

How time is estimated

The time values shown in Tachyon's output are **estimates derived from sample counts**, not direct measurements. Tachyon counts how many times each function appears in the collected samples, then multiplies by the sampling interval to estimate time.

For example, with a 10 kHz sampling rate over a 10-second profile, Tachyon collects approximately 100,000 samples. If a function appears in 5,000 samples (5% of total), Tachyon estimates it consumed 5% of the 10-second duration, or about 500 milliseconds. This is a statistical estimate, not a precise measurement.

The accuracy of these estimates depends on sample count. With 100,000 samples, a function showing 5% has a margin of error of roughly ±0.5%. With only 1,000 samples, the same 5% measurement could actually represent anywhere from 3% to 7% of real time.

This is why longer profiling durations and shorter sampling intervals produce more reliable results—they collect more samples. For most

performance analysis, the default settings provide sufficient accuracy to identify bottlenecks and guide optimization efforts.

Because sampling is statistical, results will vary slightly between runs. A function showing 12% in one run might show 11% or 13% in the next. This is normal and expected. Focus on the overall pattern rather than exact percentages, and don't worry about small variations between runs.

When to use a different approach

Statistical sampling is not ideal for every situation.

For very short scripts that complete in under one second, the profiler may not collect enough samples for reliable results. Use `profiling.tracing` instead, or run the script in a loop to extend profiling time.

When you need exact call counts, sampling cannot provide them. Sampling estimates frequency from snapshots, so if you need to know precisely how many times a function was called, use `profiling.tracing`.

When comparing two implementations where the difference might be only 1-2%, sampling noise can obscure real differences. Use `timeit` for micro-benchmarks or `profiling.tracing` for precise measurements.

The key difference from `profiling.tracing` is how measurement happens. A tracing profiler instruments your code, recording every function call and return. This provides exact call counts and precise timing but adds overhead to every function call. A sampling profiler, by contrast, observes the program from outside at fixed intervals without modifying its execution. Think of the difference like this: tracing is like having someone follow you and write down every step you take, while sampling is like taking photographs every second and inferring your path from those snapshots.

This external observation model is what makes sampling profiling practical for production use. The profiled program runs at full speed because there is no instrumentation code running inside it, and the target process is never stopped or paused during sampling—Tachyon reads the call stack directly from the process's memory while it continues to run. You can attach to a

live server, collect data, and detach without the application ever knowing it was observed. The trade-off is that very short-lived functions may be missed if they happen to complete between samples.

Statistical profiling excels at answering the question, “Where is my program spending time?” It reveals hotspots and bottlenecks in production code where deterministic profiling overhead would be unacceptable. For exact call counts and complete call graphs, use `profiling.tracing` instead.

Quick examples

Profile a script and see the results immediately:

```
python -m profiling.sampling run script.py
```

Profile a module with arguments:

```
python -m profiling.sampling run -m mypackage.module arg1 arg2
```

Generate an interactive flame graph:

```
python -m profiling.sampling run --flamegraph -o profile.html sc
```

Attach to a running process by PID:

```
python -m profiling.sampling attach 12345
```

Print a single snapshot of a running process’s stack:

```
python -m profiling.sampling dump 12345
```

Use live mode for real-time monitoring (press `q` to quit):

```
python -m profiling.sampling run --live script.py
```

Profile for 60 seconds with a faster sampling rate:

```
python -m profiling.sampling run -d 60 -r 20khz script.py
```

Generate a line-by-line heatmap:

```
python -m profiling.sampling run --heatmap script.py
```

Enable opcode-level profiling to see which bytecode instructions are executing:

```
python -m profiling.sampling run --opcodes --flamegraph script.p
```

Commands

Tachyon operates through several subcommands. `run` and `attach` collect samples over time; `dump` captures a single snapshot; `replay` converts binary profiles to other formats.

The `run` command

The `run` command launches a Python script or module and profiles it from startup:

```
python -m profiling.sampling run script.py
python -m profiling.sampling run -m mypackage.module
```

When profiling a script, the profiler starts the target in a subprocess, waits for it to initialize, then begins collecting samples. The `-m` flag indicates that the target should be run as a module (equivalent to `python -m`). Arguments after the target are passed through to the profiled program:

```
python -m profiling.sampling run script.py --config settings.yam
```

The `attach` command

The `attach` command connects to an already-running Python process by its process ID:

```
python -m profiling.sampling attach 12345
```

This command is particularly valuable for investigating performance issues in production systems. The target process requires no modification and need not be restarted. The profiler attaches, collects samples for the specified duration, then detaches and produces output.

```
python -m profiling.sampling attach --live 12345
python -m profiling.sampling attach --flamegraph -d 30 -o profil
```

On most systems, attaching to another process requires appropriate permissions. See Platform requirements for platform-specific requirements.

The `dump` command

The `dump` command prints a single snapshot of a running process's Python stack and exits, similar to a traceback:

```
python -m profiling.sampling dump 12345
```

Unlike `attach`, `dump` does not run a sampling loop: it reads the stack once. This is useful for investigating hung or unresponsive processes, or for answering "what is this process doing right now?".

The output mirrors a traceback (most recent call last) and annotates each thread with its current state (main thread, has GIL, on CPU, waiting for GIL, has exception, or idle):

```
Stack dump for PID 12345, thread 140735 (main thread, has GIL, c
  File "server.py", line 28, in serve
    await handle_request(req)
  File "handler.py", line 91, in handle_request
    result = expensive_call(req)
```

When the target's source files are readable, `dump` prints the source line for each frame and highlights the executing expression.

Like `attach`, `dump` requires permission to read the target process's memory. See Platform requirements.

The `dump` command supports the following options:

`-a`, `--all-threads`
: Dump every thread in the target process. Without this flag only the main thread is shown.

`--native`
: Include synthetic `<native>` frames marking transitions into C extensions or other non-Python code.

`--no-gc`
: Hide the synthetic `<GC>` frames that mark active garbage collection.

`--opcodes`
: Annotate each frame with the bytecode opcode the thread is currently executing (for example, `opcode=CALL_KW`). Useful for instruction-level investigation, including identifying specializations chosen by the adaptive interpreter.

`--async-aware`
: Reconstruct stacks across `await` boundaries. `dump` walks the task graph and emits one section per task, with `<task>` markers separating coroutines awaiting each other.

`--async-mode {running,all}`
: Controls which tasks are included when `--async-aware` is enabled. `running` shows only the task currently executing on each thread; `all` (the default for `dump`) also includes tasks suspended on a wait. `attach`'s default for this flag is `running`; `dump` defaults to `all` because a single snapshot is most useful when it shows the full task graph.

`--blocking`
: Pause every thread in the target while reading its stack and resume them after. Guarantees a fully consistent snapshot at the cost of briefly stopping the target. Without it, `dump` reads memory while the target keeps running, which is faster but can occasionally produce a torn stack.

The `replay` command

The `replay` command converts binary profile files to other output formats:

```
python -m profiling.sampling replay profile.bin
python -m profiling.sampling replay --flamegraph -o profile.html
```

This command is useful when you have captured profiling data in binary format and want to analyze it later or convert it to a visualization format. Binary profiles can be replayed multiple times to different formats without re-profiling.

```
# Convert binary to pstats (default, prints to stdout)
python -m profiling.sampling replay profile.bin

# Convert binary to flame graph
python -m profiling.sampling replay --flamegraph -o output.html

# Convert binary to gecko format for Firefox Profiler
python -m profiling.sampling replay --gecko -o profile.json prof

# Convert binary to heatmap
python -m profiling.sampling replay --heatmap -o my_heatmap prof
```

Profiling in production

The sampling profiler is designed for production use. It imposes no measurable overhead on the target process because it reads memory externally rather than instrumenting code. The target application continues running at full speed and is unaware it is being profiled.

When profiling production systems, keep these guidelines in mind:

Start with shorter durations (10-30 seconds) to get quick results, then extend if you need more statistical accuracy. By default, profiling runs until the target process completes, which is usually sufficient to identify major hotspots.

If possible, profile during representative load rather than peak traffic. Profiles collected during normal operation are easier to interpret than those

collected during unusual spikes.

The profiler itself consumes some CPU on the machine where it runs (not on the target process). On the same machine, this is typically negligible. When profiling remote processes, network latency does not affect the target.

Results from production may differ from development due to different data sizes, concurrent load, or caching effects. This is expected and is often exactly what you want to capture.

Platform requirements

The profiler reads the target process's memory to capture stack traces. This requires elevated permissions on most operating systems.

Linux

On Linux, the profiler uses `ptrace` or `process_vm_readv` to read the target process's memory. This typically requires one of:

- Running as root
- Having the `CAP_SYS_PTRACE` capability
- Adjusting the Yama ptrace scope: `/proc/sys/kernel/yama/ptrace_scope`

The default ptrace_scope of 1 restricts ptrace to parent processes only. To allow attaching to any process owned by the same user, set it to 0:

```
echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope
```

macOS

On macOS, the profiler uses `task_for_pid()` to access the target process. This requires one of:

- Running as root
- The profiler binary having the `com.apple.security.cs.debugger` entitlement

- System Integrity Protection (SIP) being disabled (not recommended)

Windows

On Windows, the profiler requires administrative privileges or the `SeDebugPrivilege` privilege to read another process's memory.

Version compatibility

The profiler and target process must run the same Python minor version (for example, both Python 3.15). Attaching from Python 3.14 to a Python 3.15 process is not supported.

Additional restrictions apply to pre-release Python versions: if either the profiler or target is running a pre-release (alpha, beta, or release candidate), both must run the exact same version.

On free-threaded Python builds, the profiler cannot attach from a free-threaded build to a standard build, or vice versa.

Sampling configuration

Before exploring the various output formats and visualization options, it is important to understand how to configure the sampling process itself. The profiler offers several options that control how frequently samples are collected, how long profiling runs, which threads are observed, and what additional context is captured in each sample.

The default configuration works well for most use cases:

Option	Default
Default for `--sampling-rate` / `-r`	1 kHz

Option	Default
Default for `--duration` / `-d`	Run to completion
Default for `--all-threads` / `-a`	Main thread only
Default for `--native`	No `<native>` frames (C code time attributed to caller)
Default for `--no-gc`	`<GC>` frames included when garbage collection is active
Default for `--mode`	Wall-clock mode (all samples recorded)
Default for `--realtime-stats`	Disabled
Default for `--subprocesses`	Disabled
Default for `--blocking`	Disabled (non-blocking sampling)

Sampling rate and duration

The two most fundamental parameters are the sampling rate and duration. Together, these determine how many samples will be collected during a profiling session.

The `--sampling-rate` option (`-r`) sets how frequently samples are collected. The default is 1 kHz (1,000 samples per second):

```
python -m profiling.sampling run -r 20khz script.py
```

Higher rates capture more samples and provide finer-grained data at the cost of slightly higher profiler CPU usage. Lower rates reduce profiler overhead but may miss short-lived functions. For most applications, the default rate provides a good balance between accuracy and overhead.

The `--duration` option (`-d`) sets how long to profile in seconds. By default, profiling continues until the target process exits or is interrupted:

```
python -m profiling.sampling run -d 60 script.py
```

Specifying a duration is useful when attaching to long-running processes or when you want to limit profiling to a specific time window. When profiling a script, the default behavior of running to completion is usually what you want.

Thread selection

Python programs often use multiple threads, whether explicitly through the `threading` module or implicitly through libraries that manage thread pools.

By default, the profiler samples only the main thread. The `--all-threads` option (`-a`) enables sampling of all threads in the process:

```
python -m profiling.sampling run -a script.py
```

Multi-thread profiling reveals how work is distributed across threads and can identify threads that are blocked or starved. Each thread's samples are combined in the output, with the ability to filter by thread in some formats. This option is particularly useful when investigating concurrency issues or when work is distributed across a thread pool.

Blocking mode

By default, Tachyon reads the target process's memory without stopping it. This non-blocking approach is ideal for most profiling scenarios because it imposes virtually zero overhead on the target application: the profiled program runs at full speed and is unaware it is being observed.

However, non-blocking sampling can occasionally produce incomplete or inconsistent stack traces in applications with many generators or coroutines that rapidly switch between yield points, or in programs with very fast-changing call stacks where functions enter and exit between the start and end of a single stack read, resulting in reconstructed stacks that mix frames from different execution states or that never actually existed.

For these cases, the `--blocking` option stops the target process during each sample:

```
python -m profiling.sampling run --blocking script.py
python -m profiling.sampling attach --blocking 12345
```

When blocking mode is enabled, the profiler suspends the target process, reads its stack, then resumes it. This guarantees that each captured stack represents a real, consistent snapshot of what the process was doing at that instant. The trade-off is that the target process runs slower because it is repeatedly paused.

Aviso

Do not use very high sample rates (low `--interval` values) with blocking mode. Suspending and resuming a process takes time, and if the sampling interval is too short, the target will spend more time stopped than running. For blocking mode, intervals of 1000 microseconds (1 millisecond) or higher are recommended. The default 100 microsecond interval may cause noticeable slowdown in the target application.

Use blocking mode only when you observe inconsistent stacks in your profiles, particularly with generator-heavy or coroutine-heavy code. For

most applications, the default non-blocking mode provides accurate results with zero impact on the target process.

Special frames

The profiler can inject artificial frames into the captured stacks to provide additional context about what the interpreter is doing at the moment each sample is taken. These synthetic frames help distinguish different types of execution that would otherwise be invisible.

The `--native` option adds `<native>` frames to indicate when Python has called into C code (extension modules, built-in functions, or the interpreter itself):

```
python -m profiling.sampling run --native script.py
```

These frames help distinguish time spent in Python code versus time spent in native libraries. Without this option, native code execution appears as time in the Python function that made the call. This is useful when optimizing code that makes heavy use of C extensions like NumPy or database drivers.

By default, the profiler includes `<GC>` frames when garbage collection is active. The `--no-gc` option suppresses these frames:

```
python -m profiling.sampling run --no-gc script.py
```

GC frames help identify programs where garbage collection consumes significant time, which may indicate memory allocation patterns worth optimizing. If you see substantial time in `<GC>` frames, consider investigating object allocation rates or using object pooling.

Opcode-aware profiling

The `--opcodes` option enables instruction-level profiling that captures which Python bytecode instructions are executing at each sample:

```
python -m profiling.sampling run --opcodes --flamegraph script.p
```

This feature provides visibility into Python's bytecode execution, including adaptive specialization optimizations. When a generic instruction like `LOAD_ATTR` is specialized at runtime into a more efficient variant like `LOAD_ATTR_INSTANCE_VALUE`, the profiler shows both the specialized name and the base instruction.

Opcode information appears in several output formats:

- **Flame graphs**: Hovering over a frame displays a tooltip with a bytecode instruction breakdown, showing which opcodes consumed time in that function
- **Heatmap**: Expandable bytecode panels per source line show instruction breakdown with specialization percentages
- **Live mode**: An opcode panel shows instruction-level statistics for the selected function, accessible via keyboard navigation
- **Gecko format**: Opcode transitions are emitted as interval markers in the Firefox Profiler timeline

This level of detail is particularly useful for:

- Understanding the performance impact of Python's adaptive specialization
- Identifying hot bytecode instructions that might benefit from optimization
- Analyzing the effectiveness of different code patterns at the instruction level
- Debugging performance issues that occur at the bytecode level

The `--opcodes` option is compatible with `--live`, `--flamegraph`, `--heatmap`, and `--gecko` formats. It requires additional memory to store opcode information and may slightly reduce sampling performance, but provides unprecedented visibility into Python's execution model.

Real-time statistics

The `--realtime-stats` option displays sampling rate statistics during profiling:

```
python -m profiling.sampling run --realtime-stats script.py
```

This shows the actual achieved sampling rate, which may be lower than requested if the profiler cannot keep up. The statistics help verify that profiling is working correctly and that sufficient samples are being collected. See Sampling efficiency for details on interpreting these metrics.

Subprocess profiling

The `--subprocesses` option enables automatic profiling of subprocesses spawned by the target:

```
python -m profiling.sampling run --subprocesses script.py
python -m profiling.sampling attach --subprocesses 12345
```

When enabled, the profiler monitors the target process for child process creation. When a new Python child process is detected, a separate profiler instance is automatically spawned to profile it. This is useful for applications that use `multiprocessing`, `subprocess`, `concurrent.futures` with `ProcessPoolExecutor`, or other process spawning mechanisms.

worker_pool.py

```
from concurrent.futures import ProcessPoolExecutor
import math

def compute_factorial(n):
    total = 0
    for i in range(50):
        total += math.factorial(n)
    return total

if __name__ == "__main__":
    numbers = [5000 + i * 100 for i in range(50)]
    with ProcessPoolExecutor(max_workers=4) as executor:
        results = list(executor.map(compute_factorial, numbe
    print(f"Computed {len(results)} factorials")
```

```
python -m profiling.sampling run --subprocesses --flamegraph wor
```

This produces separate flame graphs for the main process and each worker process: `flamegraph_<main_pid>.html`, `flamegraph_<worker1_pid>.html`, and so on.

Each subprocess receives its own output file. The filename is derived from the specified output path (or the default) with the subprocess's process ID appended:

- If you specify `-o profile.html`, subprocesses produce `profile_12345.html`, `profile_12346.html`, and so on
- With default output, subprocesses produce files like `flamegraph_12345.html` or directories like `heatmap_12345`
- For pstats format (which defaults to stdout), subprocesses produce files like `profile_12345.pstats`

The subprocess profilers inherit most sampling options from the parent (sampling rate, duration, thread selection, native frames, GC frames, async-aware mode, and output format). All Python descendant processes are profiled recursively, including grandchildren and further descendants.

Subprocess detection works by periodically scanning for new descendants of the target process and checking whether each new process is a Python process by probing the process memory for Python runtime structures. Non-Python subprocesses (such as shell commands or external tools) are ignored.

There is a limit of 100 concurrent subprocess profilers to prevent resource exhaustion in programs that spawn many processes. If this limit is reached, additional subprocesses are not profiled and a warning is printed.

The **`--subprocesses`** option is incompatible with **`--live`** mode because live mode uses an interactive terminal interface that cannot accommodate multiple concurrent profiler displays.

Sampling efficiency

Sampling efficiency metrics help assess the quality of the collected data. These metrics appear in the profiler's terminal output and in the flame graph sidebar.

Sampling efficiency is the percentage of sample attempts that succeeded. Each sample attempt reads the target process's call stack from memory. An attempt can fail if the process is in an inconsistent state at the moment of reading, such as during a context switch or while the interpreter is updating its internal structures. A low efficiency may indicate that the profiler could not keep up with the requested sampling rate, often due to system load or an overly aggressive interval setting.

Missed samples is the percentage of expected samples that were not collected. Based on the configured interval and duration, the profiler expects to collect a certain number of samples. Some samples may be missed if the profiler falls behind schedule, for example when the system is under heavy load. A small percentage of missed samples is normal and does not significantly affect the statistical accuracy of the profile.

Both metrics are informational. Even with some failed attempts or missed samples, the profile remains statistically valid as long as enough samples were collected. The profiler reports the actual number of samples captured, which you can use to judge whether the data is sufficient for your analysis.

Profiling modes

The sampling profiler supports four modes that control which samples are recorded. The mode determines what the profile measures: total elapsed time, CPU execution time, time spent holding the global interpreter lock, or exception handling.

Wall-clock mode

Wall-clock mode (`--mode=wall`) captures all samples regardless of what the thread is doing. This is the default mode and provides a complete picture of where time passes during program execution:

```
python -m profiling.sampling run --mode=wall script.py
```

In wall-clock mode, samples are recorded whether the thread is actively executing Python code, waiting for I/O, blocked on a lock, or sleeping. This makes wall-clock profiling ideal for understanding the overall time distribution in your program, including time spent waiting.

If your program spends significant time in I/O operations, network calls, or sleep, wall-clock mode will show these waits as time attributed to the calling function. This is often exactly what you want when optimizing end-to-end latency.

CPU mode

CPU mode (`--mode=cpu`) records samples only when the thread is actually executing on a CPU core:

```
python -m profiling.sampling run --mode=cpu script.py
```

Samples taken while the thread is sleeping, blocked on I/O, or waiting for a lock are discarded. The resulting profile shows where CPU cycles are consumed, filtering out idle time.

CPU mode is useful when you want to focus on computational hotspots without being distracted by I/O waits. If your program alternates between computation and network calls, CPU mode reveals which computational sections are most expensive.

Comparing wall-clock and CPU profiles

Running both wall-clock and CPU mode profiles can reveal whether a function's time is spent computing or waiting.

If a function appears prominently in both profiles, it is a true computational hotspot—actively using the CPU. Optimization should focus on algorithmic improvements or more efficient code.

If a function is high in wall-clock mode but low or absent in CPU mode, it is I/O-bound or waiting. The function spends most of its time waiting for network, disk, locks, or sleep. CPU optimization won't help here; consider async I/O, connection pooling, or reducing wait time instead.

```
import time

def do_sleep():
    time.sleep(2)

def do_compute():
    sum(i**2 for i in range(1000000))

if __name__ == "__main__":
    do_sleep()
    do_compute()
```

```
python -m profiling.sampling run --mode=wall script.py  # do_sle
python -m profiling.sampling run --mode=cpu script.py   # do_sle
```

GIL mode

GIL mode (`--mode=gil`) records samples only when the thread holds Python's global interpreter lock:

```
python -m profiling.sampling run --mode=gil script.py
```

The GIL is held only while executing Python bytecode. When Python calls into C extensions, performs I/O operations, or executes native code, the GIL is typically released. This means GIL mode effectively measures time spent running Python code specifically, filtering out time in native libraries.

In multi-threaded programs, GIL mode reveals which code is preventing other threads from running Python bytecode. Since only one thread can hold the GIL at a time, functions that appear frequently in GIL mode profiles are monopolizing the interpreter.

GIL mode helps answer questions like "which functions are monopolizing the GIL?" and "why are my other threads starving?" It can also be useful in

single-threaded programs to distinguish Python execution time from time spent in C extensions or I/O.

```
import hashlib

def hash_work():
    # C extension - releases GIL during computation
    for _ in range(200):
        hashlib.sha256(b"data" * 250000).hexdigest()

def python_work():
    # Pure Python - holds GIL during computation
    for _ in range(3):
        sum(i**2 for i in range(1000000))

if __name__ == "__main__":
    hash_work()
    python_work()
```

```
python -m profiling.sampling run --mode=cpu script.py  # hash_wo
python -m profiling.sampling run --mode=gil script.py  # hash_wo
```

Exception mode

Exception mode (`--mode=exception`) records samples only when a thread has an active exception:

```
python -m profiling.sampling run --mode=exception script.py
```

Samples are recorded in two situations: when an exception is being propagated up the call stack (after `raise` but before being caught), or when code is executing inside an `except` block where exception information is still present in the thread state.

The following example illustrates which code regions are captured:

```
def example():
    try:
        raise ValueError("error")    # Captured: exception being
    except ValueError:
```

```
        process_error()              # Captured: inside except b
    finally:
        cleanup()                    # NOT captured: exception a

def example_propagating():
    try:
        try:
            raise ValueError("error")
        finally:
            cleanup()                # Captured: exception propa
    except ValueError:
        pass

def example_no_exception():
    try:
        do_work()
    finally:
        cleanup()                    # NOT captured: no exceptio
```

Note that `finally` blocks are only captured when an exception is actively propagating through them. Once an `except` block finishes executing, Python clears the exception information before running any subsequent `finally` block. Similarly, `finally` blocks that run during normal execution (when no exception was raised) are not captured because no exception state is present.

This mode is useful for understanding where your program spends time handling errors. Exception handling can be a significant source of overhead in code that uses exceptions for flow control (such as `StopIteration` in iterators) or in applications that process many error conditions (such as network servers handling connection failures).

Exception mode helps answer questions like “how much time is spent handling exceptions?” and “which exception handlers are the most expensive?” It can reveal hidden performance costs in code that catches and processes many exceptions, even when those exceptions are handled gracefully. For example, if a parsing library uses exceptions internally to signal format errors, this mode will capture time spent in those handlers even if the calling code never sees the exceptions.

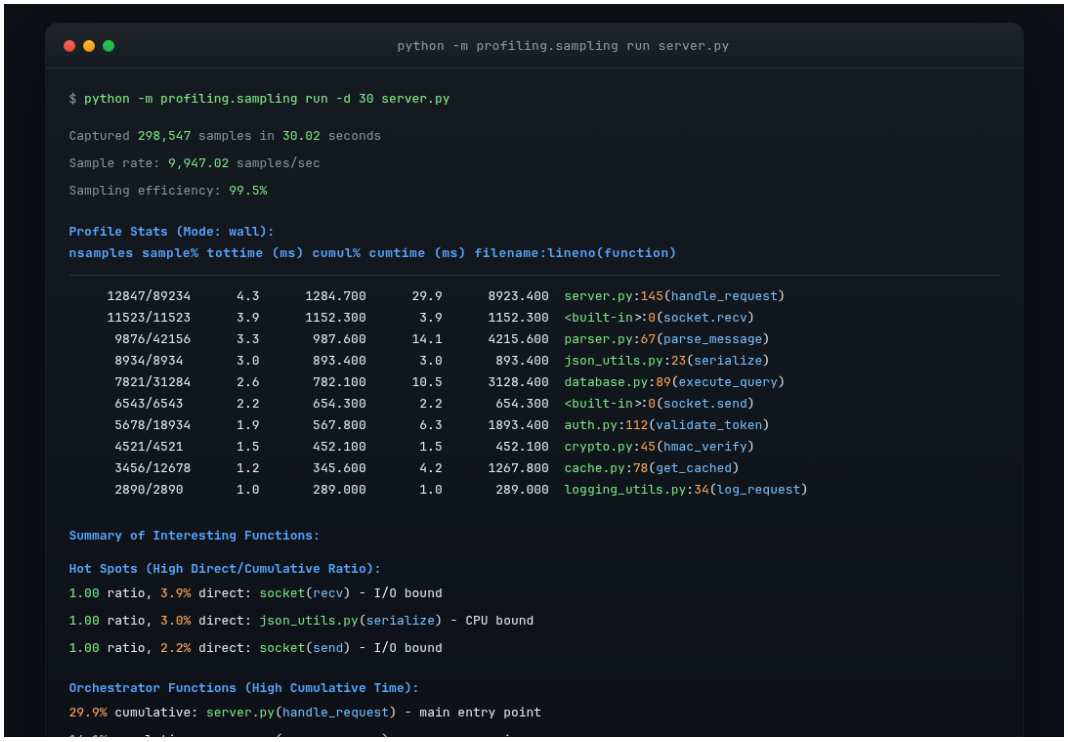
Output formats

The profiler produces output in several formats, each suited to different analysis workflows. The format is selected with a command-line flag, and output goes to stdout, a file, or a directory depending on the format.

pstats format

The pstats format (`--pstats`) produces a text table similar to what deterministic profilers generate. This is the default output format:

```
python -m profiling.sampling run script.py
python -m profiling.sampling run --pstats script.py
```



The pstats format displays profiling results in a color-coded table showing function hotspots, sample counts, and timing estimates.

Output appears on stdout by default:

```
Profile Stats (Mode: wall):
     nsamples  sample%    tottime (ms)  cumul%   cumtime (ms)  f
       234/892    11.7%       234.00     44.6%       892.00    s
       156/156     7.8%       156.00      7.8%       156.00    <
        98/421     4.9%        98.00     21.1%       421.00    p
```

The columns show sampling counts and estimated times:

- **nsamples**: Displayed as `direct/cumulative` (for example, `10/50`). Direct samples are when the function was at the top of the stack, actively executing. Cumulative samples are when the function appeared anywhere on the stack, including when it was waiting for functions it called. If a function shows `10/50`, it was directly executing in 10 samples and was on the call stack in 50 samples total.
- **sample%** and **cumul%**: Percentages of total samples for direct and cumulative counts respectively.
- **tottime** and **cumtime**: Estimated wall-clock time based on sample counts and the profiling duration. Time units are selected automatically based on the magnitude: seconds for large values, milliseconds for moderate values, or microseconds for small values.

The output includes a legend explaining each column and a summary of interesting functions that highlights:

- **Hot spots**: Functions with high direct/cumulative sample ratio (ratio close to 1.0). These functions spend most of their time executing their own code rather than waiting for callees. High ratios indicate where CPU time is actually consumed.
- **Indirect calls**: Functions with large differences between cumulative and direct samples. These are orchestration functions that delegate work to other functions. They appear frequently on the stack but rarely at the top.
- **Call magnification**: Functions where cumulative samples far exceed direct samples (high cumulative/direct multiplier). These are frequently-nested functions that appear deep in many call chains.

Use `--no-summary` to suppress both the legend and summary sections.

To save pstats output to a binary file instead of stdout:

```
python -m profiling.sampling run -o profile.pstats script.py
```

The pstats format supports several options for controlling the display. The `--sort` option determines the column used for ordering results:

```
python -m profiling.sampling run --sort=tottime script.py
python -m profiling.sampling run --sort=cumtime script.py
python -m profiling.sampling run --sort=nsamples script.py
```

The `--limit` option restricts output to the top N entries:

```
python -m profiling.sampling run --limit=30 script.py
```

The `--no-summary` option suppresses the header summary that precedes the statistics table.

Collapsed stacks format

Collapsed stacks format (`--collapsed`) produces one line per unique call stack, with a count of how many times that stack was sampled:

```
python -m profiling.sampling run --collapsed script.py
```

The output looks like:

```
main;process_data;parse_json;decode_utf8 42
main;process_data;parse_json 156
main;handle_request;send_response 89
```

Each line contains semicolon-separated function names representing the call stack from bottom to top, followed by a space and the sample count. This format is designed for compatibility with external flame graph tools, particularly Brendan Gregg's `flamegraph.pl` script.

To generate a flame graph from collapsed stacks:

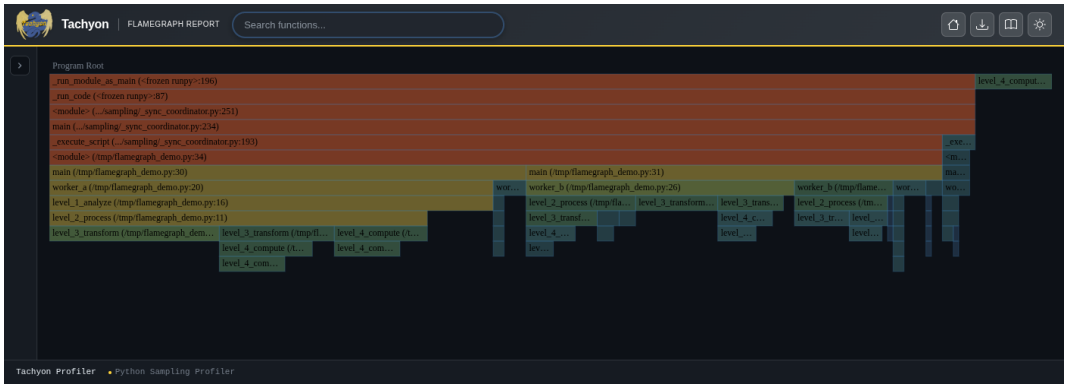
```
python -m profiling.sampling run --collapsed script.py > stacks.
flamegraph.pl stacks.txt > profile.svg
```

The resulting SVG can be viewed in any web browser and provides an interactive visualization where you can click to zoom into specific call paths.

Flame graph format

Flame graph format (`--flamegraph`) produces a self-contained HTML file with an interactive flame graph visualization:

```
python -m profiling.sampling run --flamegraph script.py
python -m profiling.sampling run --flamegraph -o profile.html sc
```



The flame graph visualization shows call stacks as nested rectangles, with width proportional to time spent. The sidebar displays runtime statistics, GIL metrics, and hotspot functions.

Try the interactive example!

If no output file is specified, the profiler generates a filename based on the process ID (for example, `flamegraph.12345.html`).

The generated HTML file requires no external dependencies and can be opened directly in a web browser. The visualization displays call stacks as nested rectangles, with width proportional to time spent. Hovering over a rectangle shows details about that function including source code context, and clicking zooms into that portion of the call tree.

The flame graph interface includes:

- A sidebar showing profile summary, thread statistics, sampling efficiency metrics (see Sampling efficiency), and top hotspot functions
- Search functionality supporting both function name matching and `file.py:42` line patterns
- Per-thread filtering via dropdown
- Dark/light theme toggle (preference saved across sessions)
- SVG export for saving the current view

The thread statistics section shows runtime behavior metrics:

- **GIL Held**: percentage of samples where a thread held the global interpreter lock (actively running Python code)
- **GIL Released**: percentage of samples where no thread held the GIL
- **Waiting GIL**: percentage of samples where a thread was waiting to acquire the GIL
- **GC**: percentage of samples during garbage collection

These statistics help identify GIL contention and understand how time is distributed between Python execution, native code, and waiting.

Flame graphs are particularly effective for identifying deep call stacks and understanding the hierarchical structure of time consumption. Wide rectangles at the top indicate functions that consume significant time either directly or through their callees.

Differential flame graphs

Differential flame graphs compare two profiling runs to highlight where performance changed. This helps identify regressions introduced by code changes and validate that optimizations achieved their intended effect:

```
# Capture baseline profile
python -m profiling.sampling run --binary -o baseline.bin script

# After modifying code, generate differential flamegraph
python -m profiling.sampling run --diff-flamegraph baseline.bin
```

The visualization draws the current profile with frame widths showing current time consumption, then applies color to indicate how each function changed relative to the baseline.

Color coding:

- **Red**: Functions consuming more time (regressions). Lighter shades indicate modest increases, while darker shades show severe regressions.
- **Blue**: Functions consuming less time (improvements). Lighter shades for modest reductions, darker shades for significant speedups.
- **Gray**: Minimal or no change.
- **Purple**: New functions not present in the baseline.

Frame colors indicate changes in **direct time** (time when the function was at the top of the stack, actively executing), not cumulative time including callees. Hovering over a frame shows comparison details including baseline time, current time, and the percentage change.

Some call paths may disappear entirely between profiles. These are called **elided stacks** and occur when optimizations eliminate code paths or certain branches stop executing. If elided stacks are present, an elided toggle appears allowing you to switch between the main differential view and an elided-only view that shows just the removed paths (colored purple).

Gecko format

Gecko format (`--gecko`) produces JSON output compatible with the Firefox Profiler:

```
python -m profiling.sampling run --gecko script.py
python -m profiling.sampling run --gecko -o profile.json script.
```

The Firefox Profiler [https://profiler.firefox.com] is a sophisticated web-based tool originally built for profiling Firefox itself. It provides features beyond basic flame graphs, including a timeline view, call tree exploration, and marker

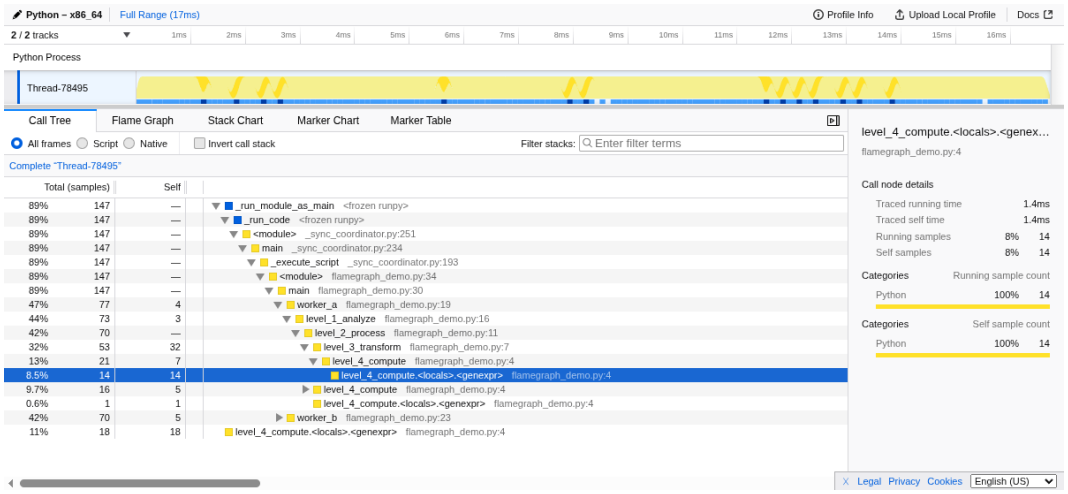
visualization. See the [Firefox Profiler documentation](https://profiler.firefox.com/docs/#/) [https://profiler.firefox.com/docs/#/] for detailed usage instructions.

To use the output, open the Firefox Profiler in your browser and load the JSON file. The profiler runs entirely client-side, so your profiling data never leaves your machine.

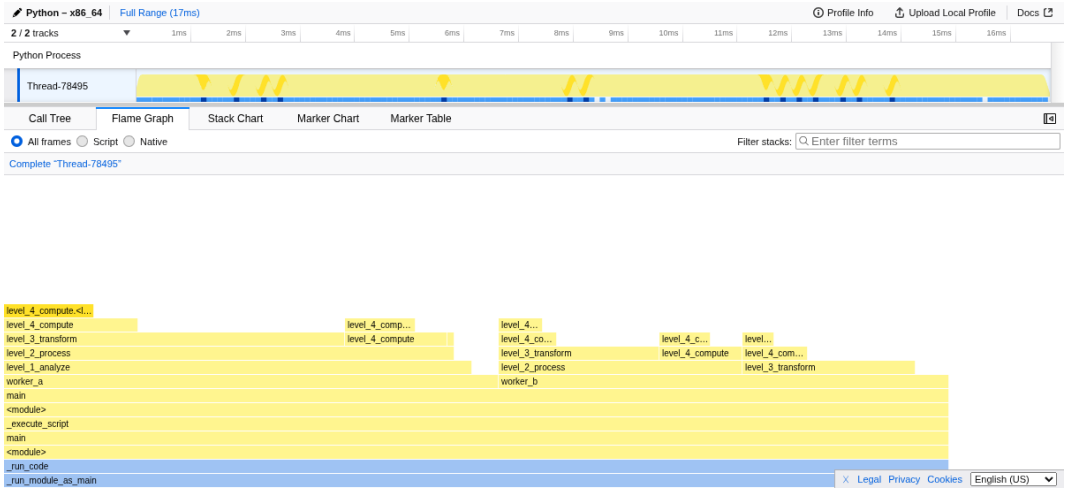
Gecko format automatically collects additional metadata about GIL state and CPU activity, enabling analysis features specific to Python's threading model. The profiler emits interval markers that appear as colored bands in the Firefox Profiler timeline:

- **GIL markers**: show when threads hold or release the global interpreter lock
- **CPU markers**: show when threads are executing on CPU versus idle
- **Code type markers**: distinguish Python code from native (C extension) code
- **GC markers**: indicate garbage collection activity

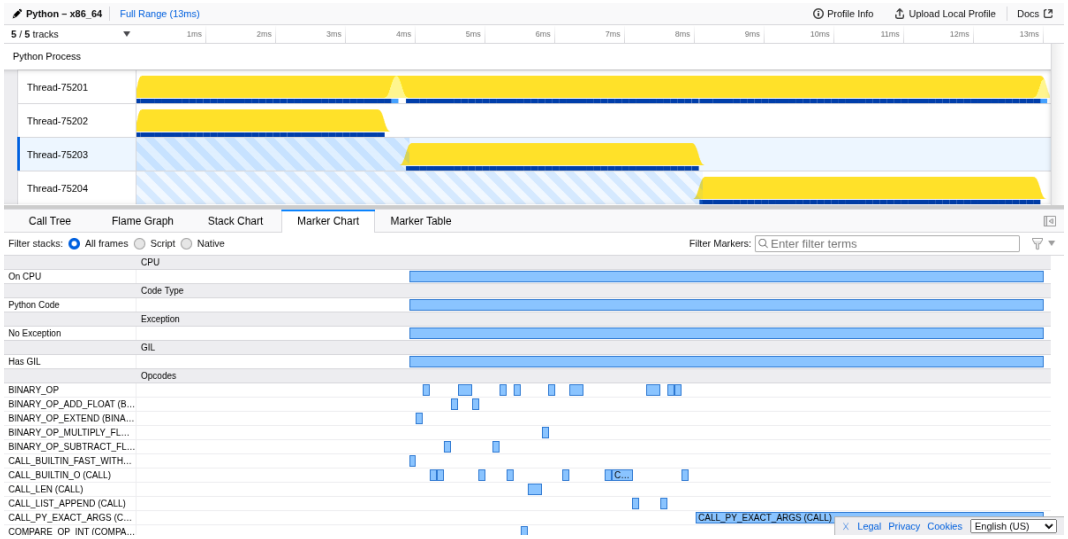
For this reason, the `--mode` option is not available with Gecko format; all relevant data is captured automatically.



The Call Tree view shows the complete call hierarchy with sample counts and percentages. The sidebar displays detailed statistics for the selected function including running time and sample distribution.



The Flame Graph visualization shows call stacks as nested rectangles. Functions names are visible in the call hierarchy.

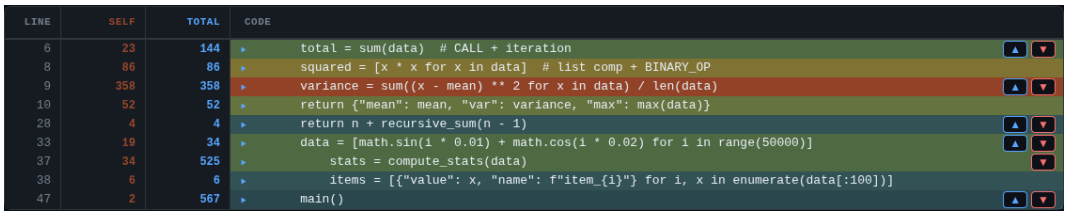


The Marker Chart displays interval markers including CPU state, GIL status, and opcodes. With `--opcodes` enabled, bytecode instructions like `BINARY_OP_ADD_FLOAT`, `CALL_PY_EXACT_ARGS`, and `CALL_LIST_APPEND` appear as markers showing execution over time.

Heatmap format

Heatmap format (`--heatmap`) generates an interactive HTML visualization showing sample counts at the source line level:

```
python -m profiling.sampling run --heatmap script.py
python -m profiling.sampling run --heatmap -o my_heatmap script.
```



The heatmap overlays sample counts directly on your source code. Lines are color-coded from cool (few samples) to hot (many samples). Navigation buttons (▲ ▼) let you jump between callers and callees.

Unlike other formats that produce a single file, heatmap output creates a directory containing HTML files for each profiled source file. If no output path is specified, the directory is named `heatmap_PID`.

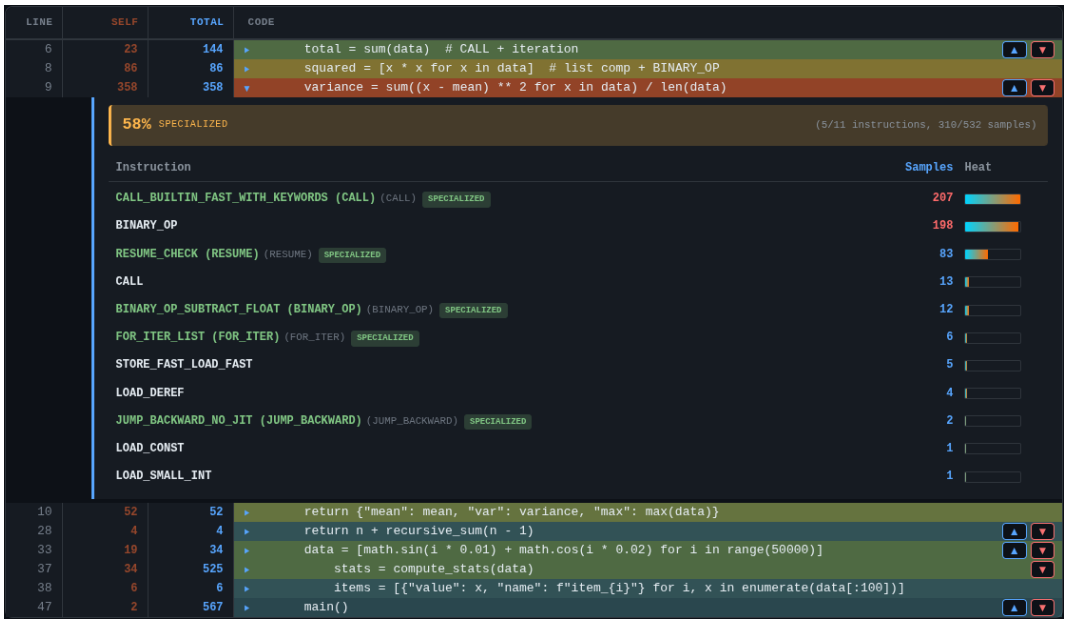
The heatmap visualization displays your source code with a color gradient indicating how many samples were collected at each line. Hot lines (many samples) appear in warm colors, while cold lines (few or no samples) appear in cool colors. This view helps pinpoint exactly which lines of code are responsible for time consumption.

The heatmap interface provides several interactive features:

- **Coloring modes**: toggle between “Self Time” (direct execution) and “Total Time” (cumulative, including time in called functions)
- **Cold code filtering**: show all lines or only lines with samples
- **Call graph navigation**: each line shows navigation buttons (▲ for callers, ▼ for callees) that let you trace execution paths through your code. When multiple functions called or were called from a line, a menu appears showing all options with their sample counts.
- **Scroll minimap**: a vertical overview showing the heat distribution across the entire file
- **Hierarchical index**: files organized by type (stdlib, site-packages, project) with aggregate sample counts per folder
- **Dark/light theme**: toggle with preference saved across sessions

- **Line linking**: click line numbers to create shareable URLs

When opcode-level profiling is enabled with `--opcodes`, each hot line can be expanded to show which bytecode instructions consumed time:



Expanding a hot line reveals the bytecode instructions executed, including specialized variants. The panel shows sample counts per instruction and the overall specialization percentage for the line.

Try the interactive example!

Heatmaps are especially useful when you know which file contains a performance issue but need to identify the specific lines. Many developers prefer this format because it maps directly to their source code, making it easy to read and navigate. For smaller scripts and focused analysis, heatmaps provide an intuitive view that shows exactly where time is spent without requiring interpretation of hierarchical visualizations.

Binary format

Binary format (`--binary`) produces a compact binary file for efficient storage of profiling data:

```
python -m profiling.sampling run --binary -o profile.bin script.
python -m profiling.sampling attach --binary -o profile.bin 1234
```

The `--compression` option controls data compression:

- `auto` (default): Use zstd compression if available, otherwise no compression
- `zstd`: Force zstd compression (requires `compression.zstd` support)
- `none`: Disable compression

```
python -m profiling.sampling run --binary --compression=zstd -o
```

To analyze binary profiles, use the The replay command to convert them to other formats like flame graphs or pstats output.

Record and replay workflow

The binary format combined with the replay command enables a record-and-replay workflow that separates data capture from analysis. Rather than generating visualizations during profiling, you capture raw data to a compact binary file and convert it to different formats later.

This approach has three main benefits:

- Sampling runs faster because the work of building data structures for visualization is deferred until replay.
- A single binary capture can be converted to multiple output formats without re-profiling: pstats for a quick overview, flame graph for visual exploration, heatmap for line-level detail.
- Binary files are compact and easy to share with colleagues who can convert them to their preferred format.

A typical workflow:

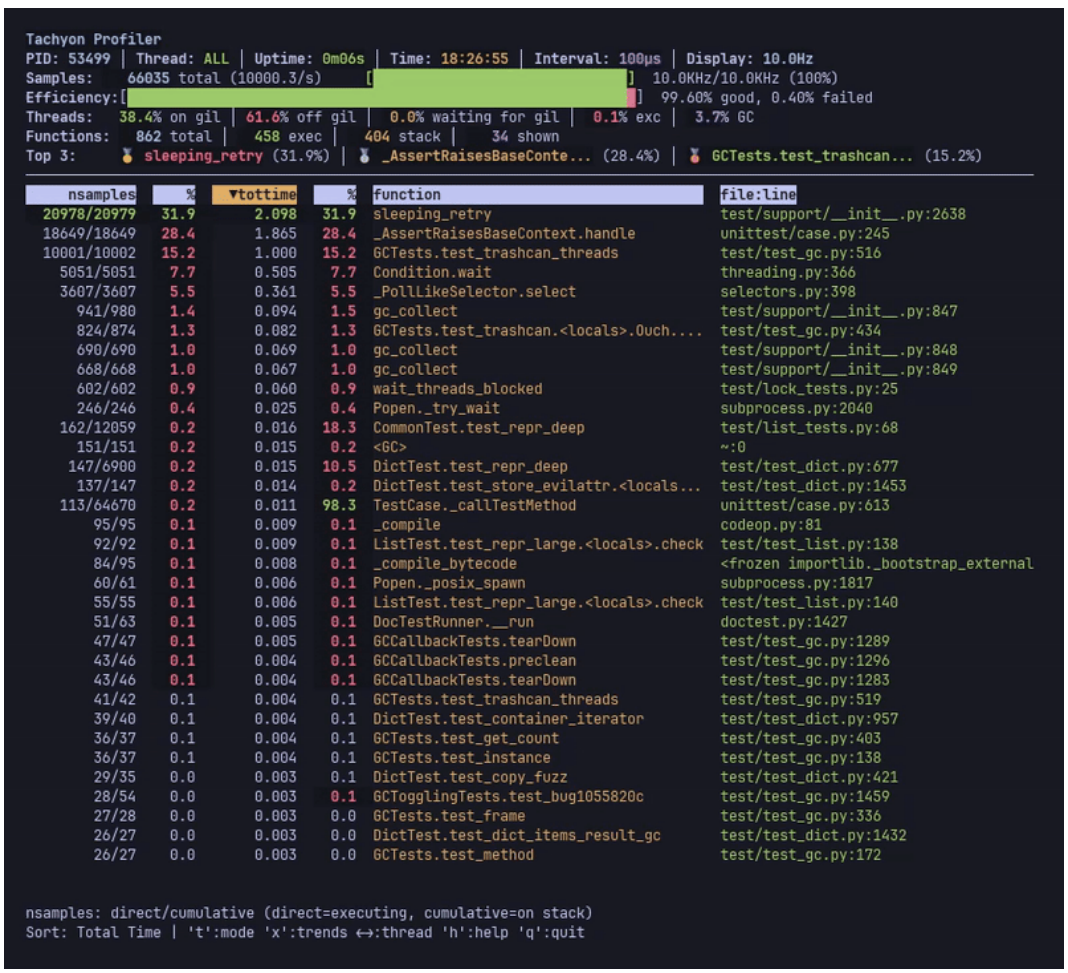
```
# Capture profile in production or during tests
python -m profiling.sampling attach --binary -o profile.bin 1234
```

```
# Later, analyze with different formats
python -m profiling.sampling replay profile.bin
python -m profiling.sampling replay --flamegraph -o profile.html
python -m profiling.sampling replay --heatmap -o heatmap profile
```

Live mode

Live mode (`--live`) provides a terminal-based real-time view of profiling data, similar to the `top` command for system processes:

```
python -m profiling.sampling run --live script.py
python -m profiling.sampling attach --live 12345
```



Live mode displays real-time profiling statistics, showing combined data from multiple threads in a multi-threaded application.

The display updates continuously as new samples arrive, showing the current hottest functions. This mode requires the `curses` module, which is available on Unix-like systems but not on Windows. The terminal must be at least 60 columns wide and 12 lines tall; larger terminals display more columns.

The header displays the top 3 hottest functions, sampling efficiency metrics, and thread status statistics (GIL held percentage, CPU usage, GC time). The main table shows function statistics with the currently sorted column indicated by an arrow (▼).

When `--opcodes` is enabled, an additional opcode panel appears below the main table, showing instruction-level statistics for the currently selected function. This panel displays which bytecode instructions are executing most frequently, including specialized variants and their base opcodes.

```
Tachyon Profiler
PID: 52146 | Thread: ALL | Uptime: 0m07s | Time: 18:26:25 | Interval: 100µs | Display: 10.0Hz
Samples:    71038 total (10000.4/s)      [                                  ]  10.0KHz/10.0KHz (100%)
Efficiency:[                                                                ]  99.69% good, 0.31% failed
Threads:   36.3% on gil | 63.7% off gil |  0.0% waiting for gil |  0.1% exc |  4.4% GC
Functions:   881 total |   478 exec |   403 stack |    34 shown
Top 3:     🥇 sleeping_retry (35.6%) | 🥈 _AssertRaisesBaseConte... (26.0%) | 🥉 GCTests.test_trashcan... (14.1%)

    nsamples    %   ▼ttotime    %  function                                  file:line
►  25188/25189  35.6    2.519  35.6  sleeping_retry                            test/support/__init__.py:2638
   18430/18430  26.0    1.843  26.0  _AssertRaisesBaseContext.handle           unittest/case.py:245
   10001/10015  14.1    1.000  14.1  GCTests.test_trashcan_threads             test/test_gc.py:516
    5053/5053    7.1    0.505   7.1  Condition.wait                            threading.py:366
    3466/3466    4.9    0.347   4.9  _PollLikeSelector.select                  selectors.py:398
    1221/1267    1.7    0.122   1.8  gc_collect                                test/support/__init__.py:847
    1105/1105    1.6    0.111   1.6  wait_threads_blocked                      test/lock_tests.py:25
     876/876     1.2    0.088   1.2  gc_collect                                test/support/__init__.py:849
     856/856     1.2    0.086   1.2  gc_collect                                test/support/__init__.py:848
     816/868     1.2    0.082   1.2  GCTests.test_trashcan.<locals>.Ouch...    test/test_gc.py:434
     237/237     0.3    0.024   0.3  Popen._try_wait                           subprocess.py:2040
   160/11966     0.2    0.016  16.9  CommonTest.test_repr_deep                 test/list_tests.py:68
     153/153     0.2    0.015   0.2  <GC>                                      ~:0
    148/6772     0.2    0.015   9.6  DictTest.test_repr_deep                   test/test_dict.py:677
     137/147     0.2    0.014   0.2  DictTest.test_store_evilattr.<local...    test/test_dict.py:1453
   122/69721     0.2    0.012  98.4  TestCase._callTestMethod                  unittest/case.py:613
     100/100     0.1    0.010   0.1  ListTest.test_repr_large.<locals>.c...    test/test_list.py:138
       95/95     0.1    0.009   0.1  _compile                                  codeop.py:81
       83/83     0.1    0.008   0.1  _compile_bytecode                         <frozen importlib._bootstrap_externa
       66/67     0.1    0.007   0.1  Popen._posix_spawn                        subprocess.py:1817
       58/58     0.1    0.006   0.1  ListTest.test_repr_large.<locals>.c...    test/test_list.py:140
       55/79     0.1    0.006   0.1  DocTestRunner.__run                       doctest.py:1427
       47/47     0.1    0.005   0.1  GCCallbackTests.tearDown                  test/test_gc.py:1289
       45/48     0.1    0.005   0.1  GCCallbackTests.preclean                  test/test_gc.py:1296
── Opcodes for sleeping_retry() ──
  [                    ] CALL_BUILTIN_O (CALL)              99.6% ( 25088)
  [                    ] CALL                                0.4% (   100)

nsamples: direct/cumulative (direct=executing, cumulative=on stack)
Sort: Total Time | 't':mode 'x':trends ↔:thread 'h':help 'q':quit
```

Live mode with `--opcodes` enabled shows an opcode panel with a bytecode instruction breakdown for the selected function.

Keyboard commands

Within live mode, keyboard commands control the display:

`q`
: Quit the profiler and return to the shell.

`s` / `S`
: Cycle through sort orders forward/backward (sample count, percentage, total time, cumulative percentage, cumulative time).

`p`
: Pause or resume display updates. Sampling continues in the background while the display is paused, so you can freeze the view to examine results without stopping data collection.

`r`
: Reset all statistics and start fresh. This is disabled after profiling finishes to prevent accidental data loss.

`/`
: Enter filter mode to search for functions by name. The filter uses case-insensitive substring matching against the filename and function name. Type a pattern and press Enter to apply, or Escape to cancel. Glob patterns and regular expressions are not supported.

`c`
: Clear the current filter and show all functions again.

`t`
: Toggle between viewing all threads combined or per-thread statistics. In per-thread mode, a thread counter (for example, `1/4`) appears showing your position among the available threads.

`←` `→` or `↑` `↓`

In per-thread view, navigate between threads. Navigation wraps around from the last thread to the first and vice versa.

`+` / `-`
: Increase or decrease the display refresh rate. The range is 0.05 seconds (20 Hz, very responsive) to 1.0 second (1 Hz, lower overhead). Faster refresh rates use more CPU. The default is 0.1 seconds (10 Hz).

`x`
: Toggle trend indicators that show whether functions are becoming hotter or cooler over time. When enabled, increasing metrics appear in green and decreasing metrics appear in red, comparing each update to the previous one.

`h` or `?`
: Show the help screen with all available commands.

`j` / `k` (or `Up` / `Down`)
: Navigate through opcode entries in the opcode panel (when `--opcodes` is enabled). These keys scroll through the instruction-level statistics for the currently selected function.

When profiling finishes (duration expires or target process exits), the display shows a “PROFILING COMPLETE” banner and freezes the final results. You can still navigate, sort, and filter the results before pressing `q` to exit.

Live mode is incompatible with output format options (`--collapsed`, `--flamegraph`, and so on) because it uses an interactive terminal interface rather than producing file output.

Async-aware profiling

For programs using `asyncio`, the profiler offers async-aware mode (`--async-aware`) that reconstructs call stacks based on the task structure rather than the raw Python frames:

```
python -m profiling.sampling run --async-aware async_script.py
```

Standard profiling of async code can be confusing because the physical call stack often shows event loop internals rather than the logical flow of your coroutines. Async-aware mode addresses this by tracking which task is running and presenting stacks that reflect the `await` chain.

```
import asyncio

async def fetch(url):
    await asyncio.sleep(0.1)
    return url

async def main():
    for _ in range(50):
        await asyncio.gather(fetch("a"), fetch("b"), fetch("c"))

if __name__ == "__main__":
    asyncio.run(main())
```

```
python -m profiling.sampling run --async-aware --flamegraph -o c
```

Nota

Async-aware profiling requires the target process to have the **`asyncio`** module loaded. If you profile a script before it imports asyncio, async-aware mode will not be able to capture task information.

Async modes

The **`--async-mode`** option controls which tasks appear in the profile:

```
python -m profiling.sampling run --async-aware --async-mode=runr
python -m profiling.sampling run --async-aware --async-mode=all
```

With **`--async-mode`**`=running` (the default), only the task currently executing on the CPU is profiled. This shows where your program is actively spending time and is the typical choice for performance analysis.

With `--async-mode=all`, tasks that are suspended (awaiting I/O, locks, or other tasks) are also included. This mode is useful for understanding what your program is waiting on, but produces larger profiles since every suspended task appears in each sample.

Task markers and stack reconstruction

In async-aware profiles, you will see `<task>` frames that mark boundaries between asyncio tasks. These are synthetic frames inserted by the profiler to show the task structure. The task name appears as the function name in these frames.

When a task awaits another task, the profiler reconstructs the logical call chain by following the `await` relationships. Only “leaf” tasks (tasks that no other task is currently awaiting) generate their own stack entries. Tasks being awaited by other tasks appear as part of their awaiter’s stack instead.

If a task has multiple awaiters (a diamond pattern in the task graph), the profiler deterministically selects one parent and annotates the task marker with the number of parents, for example `MyTask (2 parents)`. This indicates that alternate execution paths exist but are not shown in this particular stack.

Option restrictions

Async-aware mode uses a different stack reconstruction mechanism and is incompatible with: `--native`, `--no-gc`, `--all-threads`, and `--mode=cpu` or `--mode=gil`.

Command-line interface

The complete command-line interface for reference.

Global options

run

Run and profile a Python script or module.

attach
: Attach to and profile a running process by PID.

dump
: Print a single one-shot snapshot of a running process's Python stack.

replay
: Convert a binary profile file to another output format.

Dump options

The following options apply to the `dump` subcommand:

-a, --all-threads
: Dump all threads in the target process instead of just the main thread.

--native
: Include `<native>` frames for non-Python code.

--no-gc
: Exclude `<GC>` frames for active garbage collection.

--opcodes
: Show bytecode opcode names when available.

--async-aware
: Reconstruct the stack across `await` boundaries for asyncio applications.

--async-mode <mode>
: Async stack mode: `running` (only the running task) or `all` (all tasks including waiting). Defaults to `all` for `dump`. Requires `--async-aware`.

--blocking
: Pause all threads in the target process while reading the stack.

Sampling options

-r <rate>, --sampling-rate <rate>
: Sampling rate (for example, `10000`, `10khz`, `10k`). Default: `1khz`.

-d <seconds>, --duration <seconds>
: Profiling duration in seconds. Default: run to completion.

-a, --all-threads
: Sample all threads, not just the main thread.

--realtime-stats
: Display sampling statistics during profiling.

--native
: Include `<native>` frames for non-Python code.

--no-gc
: Exclude `<GC>` frames for garbage collection.

--async-aware
: Enable async-aware profiling for asyncio programs.

--opcodes
: Gather bytecode opcode information for instruction-level profiling. Shows which bytecode instructions are executing, including specializations. Compatible with `--live`, `--flamegraph`, `--heatmap`, and `--gecko` formats only.

--subprocesses
: Also profile subprocesses. Each subprocess gets its own profiler instance and output file. Incompatible with `--live`.

--blocking
: Pause the target process during each sample. This ensures consistent stack traces at the cost of slowing down the target. Use with longer

intervals (1000 µs or higher) to minimize impact. See Blocking mode for details.

Mode options

--mode <mode>
: Sampling mode: `wall` (default), `cpu`, `gil`, or `exception`. The `cpu`, `gil`, and `exception` modes are incompatible with `--async-aware`.

--async-mode <mode>
: Async profiling mode: `running` (default) or `all`. Requires `--async-aware`.

Output options

--pstats
: Generate pstats statistics. This is the default. When written to stdout, the output is a text table; with `-o`, it is a binary pstats file.

--collapsed
: Generate collapsed stack format for external flame graph tools.

--flamegraph
: Generate self-contained HTML flame graph.

--diff-flamegraph <baseline.bin>
: Generate differential flamegraph comparing to a baseline binary profile.

--gecko
: Generate Gecko JSON format for Firefox Profiler.

--heatmap
: Generate HTML heatmap with line-level sample counts.

--binary
: Generate high-performance binary format for later conversion with the `replay` command.

--compression <type>

Compression for binary format: `auto` (use zstd if available, default), `zstd`, or `none`.

-o <path>, --output <path>

Output file or directory path. Default behavior varies by format: **`--pstats`** prints a text table to stdout, while `-o` writes a binary pstats file. Other formats generate a file named `<format>_<PID>.<ext>` (for example, `flamegraph_12345.html`). **`--heatmap`** creates a directory named `heatmap_<PID>`.

--browser

Automatically open HTML output (**`--flamegraph`** and **`--heatmap`**) in your default web browser after generation. When profiling with **`--subprocesses`**, only the main process opens the browser; subprocess outputs are never auto-opened.

pstats display options

These options apply only to pstats format output.

--sort <key>

Sort order: `nsamples`, `tottime`, `cumtime`, `sample-pct`, `cumul-pct`, `nsamples-cumul`, or `name`. Default: `nsamples`.

-l <count>, --limit <count>

Maximum number of entries to display. Default: 15.

--no-summary

Omit the Legend and Summary of Interesting Functions sections from output.

Run command options

-m, --module

Treat the target as a module name rather than a script path.

--live

Start interactive terminal interface instead of batch profiling.

Ver também

`profiling`

Overview of Python profiling tools and guidance on choosing a profiler.

`profiling.tracing`

Deterministic tracing profiler for exact call counts and timing.

`pstats`

Statistics analysis for profile data.

Firefox Profiler **[https://profiler.firefox.com]**

Web-based profiler that accepts Gecko format output. See the documentation [https://profiler.firefox.com/docs/#/] for usage details.

FlameGraph **[https://github.com/brendangregg/FlameGraph]**

Tools for generating flame graphs from collapsed stack format.

`pstats` — Statistics for profilers

Source code: Lib/pstats.py [https://github.com/python/cpython/tree/3.15/Lib/pstats.py]

The `pstats` module provides tools for reading, manipulating, and displaying profiling statistics generated by Python's profilers. It reads output from both `profiling.tracing` (deterministic profiler) and `profiling.sampling` (statistical profiler).

Reading and displaying profile data

The `Stats` class is the primary interface for working with profile data. It can read statistics from files or directly from a `Profile` object.

Load statistics from a file and print a basic report:

```
import pstats

p = pstats.Stats('profile_output.prof')
p.print_stats()
```

The `Stats` object provides methods for sorting and filtering the data before printing. For example, to see the ten functions with the highest cumulative time:

```
from pstats import SortKey

p = pstats.Stats('profile_output.prof')
p.sort_stats(SortKey.CUMULATIVE).print_stats(10)
```

Working with statistics

The `Stats` class supports method chaining, making it convenient to perform multiple operations:

```
p = pstats.Stats('restats')
p.strip_dirs().sort_stats(-1).print_stats()
```

The `strip_dirs()` method removes directory paths from filenames, making the output more compact. The `sort_stats()` method accepts various keys to control the sort order.

Different sort keys highlight different aspects of performance:

```
from pstats import SortKey

# Functions that consume the most cumulative time
p.sort_stats(SortKey.CUMULATIVE).print_stats(10)

# Functions that consume the most time in their own code
p.sort_stats(SortKey.TIME).print_stats(10)

# Functions sorted by name
p.sort_stats(SortKey.NAME).print_stats()
```

Filtering output

The `print_stats()` method accepts restrictions that filter which functions are displayed. Restrictions can be integers (limiting the count), floats between 0 and 1 (selecting a percentage), or strings (matching function names via regular expression).

Print only the top 10%:

```
p.print_stats(.1)
```

Print only functions whose names contain “init”:

```
p.print_stats('init')
```

Combine restrictions (they apply sequentially):

```
# Top 10%, then only those containing "init"
p.print_stats(.1, 'init')
```

```
# Functions in files matching "foo:", limited to top 50%
p.sort_stats(SortKey.FILENAME).print_stats('foo:', .5)
```

Analyzing call relationships

The `print_callers()` method shows which functions called each displayed function:

```
p.print_callers()
```

The `print_callees()` method shows the opposite relationship, listing which functions each displayed function called:

```
p.print_callees()
```

Both methods accept the same restriction arguments as `print_stats()`.

Combining multiple profiles

Statistics from multiple profiling runs can be combined into a single `Stats` object:

```
# Load multiple files at once
p = pstats.Stats('run1.prof', 'run2.prof', 'run3.prof')

# Or add files incrementally
p = pstats.Stats('run1.prof')
p.add('run2.prof')
p.add('run3.prof')
```

When files are combined, statistics for identical functions (same file, line, and name) are accumulated, giving an aggregate view across all profiling runs.

The `Stats` class

class pstats.Stats(**filenames_or_profile*, *stream=sys.stdout*)

Create a statistics object from profile data.

The arguments can be filenames (strings or path-like objects) or `Profile` objects. If multiple sources are provided, their statistics are combined.

The *stream* argument specifies where output from `print_stats()` and related methods is written. It defaults to `sys.stdout`.

The profile data format is specific to the Python version that created it. There is no compatibility guarantee between Python versions or between different profilers.

strip_dirs()

Remove leading path information from all filenames.

This method modifies the object in place and returns it for method chaining. After stripping, the statistics are considered to be in random order.

If stripping causes two functions to become indistinguishable (same filename, line number, and function name), their statistics are combined into a single entry.

add(**filenames*)

Add profiling data from additional files.

The files must have been created by the same profiler type. Statistics for identical functions are accumulated.

dump_stats(*filename*)

Save the current statistics to a file.

The file is created if it does not exist and overwritten if it does. The saved data can be loaded by creating a new `Stats` object.

sort_stats(**keys*)

Sort the statistics according to the specified criteria.

Each key can be a string or a `SortKey` enum member. When multiple keys are provided, later keys break ties in earlier keys.

Using `SortKey` enum members is preferred over strings as it provides better error checking:

```
from pstats import SortKey
p.sort_stats(SortKey.CUMULATIVE)
```

Valid sort keys:

String	Enum	Significado
`'calls'`	`SortKey.CALLS`	call count
`'cumulative'`	`SortKey.CUMULATIVE`	cumulative time
`'cumtime'`	N/D	cumulative time
`'file'`	N/D	file name
`'filename'`	`SortKey.FILENAME`	file name
`'module'`	N/D	file name
`'ncalls'`	N/D	call count
`'pcalls'`	`SortKey.PCALLS`	primitive call count
`'line'`	`SortKey.LINE`	line number
`'name'`	`SortKey.NAME`	function name
`'nfl'`	`SortKey.NFL`	name/file/line
`'stdname'`	`SortKey.STDNAME`	standard name

String	Enum	Significado
`'time'`	`SortKey.TIME`	internal time
`'tottime'`	N/D	internal time

All sorts on statistics are in descending order (most time consuming first), while name, file, and line number sorts are ascending (alphabetical).

The difference between `SortKey.NFL` and `SortKey.STDNAME` is that NFL sorts line numbers numerically while STDNAME sorts them as strings. `sort_stats(SortKey.NFL)` is equivalent to `sort_stats(SortKey.NAME, SortKey.FILENAME, SortKey.LINE)`.

For backward compatibility, the numeric arguments `-1`, `0`, `1`, and `2` are also accepted, meaning `'stdname'`, `'calls'`, `'time'`, and `'cumulative'` respectively.

Adicionado na versão 3.7: The **`SortKey`** enum.

reverse_order()

Reverse the current sort order.

By default, the sort direction is chosen appropriately for the sort key (descending for time-based keys, ascending for name-based keys). This method inverts that choice.

print_stats(**restrictions*)

Print a report of the profiling statistics.

The output includes a header line summarizing the data, followed by a table of function statistics sorted according to the last **`sort_stats()`** call.

Restrictions filter the output. Each restriction is either:

- An integer: limits output to that many entries
- A float between 0.0 and 1.0: selects that fraction of entries
- A string: matches function names via regular expression

Restrictions are applied sequentially. For example:

```
print_stats(.1, 'foo:')
```

First limits to the top 10%, then filters to functions matching ‘foo:’.

print_callers(**restrictions*)

Print the callers of each function in the statistics.

For each function in the filtered results, shows which functions called it and how often.

With **`profiling.tracing`** (or `cProfile`), each caller line shows three numbers: the number of calls from that caller, and the total and cumulative times for those specific calls.

Accepts the same restriction arguments as **`print_stats()`**.

print_callees(**restrictions*)

Print the functions called by each function in the statistics.

This is the inverse of **`print_callers()`**, showing which functions each listed function called.

Accepts the same restriction arguments as **`print_stats()`**.

get_stats_profile()

Return a `StatsProfile` object containing the statistics.

The returned object provides programmatic access to the profile data, with function names mapped to `FunctionProfile` objects containing timing and call count information.

Adicionado na versão 3.9.

class pstats.SortKey

An enumeration of valid sort keys for `Stats.sort_stats()`.

CALLS

Sort by call count.

CUMULATIVE

Sort by cumulative time.

FILENAME

Sort by file name.

LINE

Sort by line number.

NAME

Sort by function name.

NFL

Sort by name, then file, then line number (numeric line sort).

PCALLS

Sort by primitive (non-recursive) call count.

STDNAME

Sort by standard name (string-based line sort).

TIME

Sort by internal time (time in function excluding subcalls).

Interface de linha de comando

The **pstats** module can be invoked as a script to interactively browse profile data:

```
python -m pstats profile_output.prof
```

This opens a line-oriented interface (built on `cmd`) for examining the statistics. Type `help` at the prompt for available commands.

Ver também

`profiling`
: Overview of Python profiling tools.

`profiling.tracing`
: Deterministic tracing profiler.

`profiling.sampling`
: Statistical sampling profiler.

`timeit` — Mede o tempo de execução de pequenos trechos de código

Código-fonte: Lib/timeit.py [https://github.com/python/cpython/tree/3.15/Lib/timeit.py]

Este módulo fornece uma maneira simples de cronometrar pequenos trechos do código Python. Ele tem uma Interface de linha de comando e um chamável. Ele evita uma série de armadilhas comuns para medir os tempos de execução de um script. Veja também o capítulo “Algorithms” de Tim Peters, na segunda edição do *Python Cookbook*, publicado pela O’Reilly.

Basic examples

O exemplo a seguir mostra como a Interface de linha de comando pode ser usado para comparar três expressões diferentes:

```
$ python -m timeit "'-'.join(str(n) for n in range(100))"
10000 loops, best of 5: 30.2 usec per loop
$ python -m timeit "'-'.join([str(n) for n in range(100)])"
10000 loops, best of 5: 27.5 usec per loop
$ python -m timeit "'-'.join(map(str, range(100)))"
10000 loops, best of 5: 23.2 usec per loop
```

Isso pode ser obtido da interface Python interface

```
>>> import timeit
>>> timeit.timeit('"-".join(str(n) for n in range(100))', number
0.3018611848820001
>>> timeit.timeit('"-".join([str(n) for n in range(100)])', numb
0.2727368790656328
>>> timeit.timeit('"-".join(map(str, range(100)))', number=10000
0.23702679807320237
```

Um chamável também pode ser passado para a Python interface:

```
>>> timeit.timeit(lambda: "-".join(map(str, range(100))), number
0.19665591977536678
```

Observe, entretanto, que `timeit()` determinará automaticamente o número de repetições somente quando a interface de linha de comando for usada. Na seção Exemplos você encontrará exemplos mais avançados.

Python interface

Este módulo define três funções e uma classe pública:

timeit.timeit(*stmt='pass', setup='pass', timer=<default timer>, number=1000000, globals=None*)

Cria uma instância de `Timer` com o código de *setup* e a função *timer* função para executar o método `timeit()` com o total de execuções informado em *number*. O argumento opcional *globals* especifica um espaço de nomes no qual o código será executado.

Alterado na versão 3.5: O parâmetro opcional *globals* foi adicionado.

timeit.repeat(*stmt='pass', setup='pass', timer=<default timer>, repeat=5, number=1000000, globals=None*)

Cria uma instância `Timer` com a instrução fornecida, o código de *setup* e a função *timer* e para executar o método `repeat()` com o total de execuções informando em *repeat* e o *número* de execuções fornecidos. O argumento opcional *globals* especifica um espaço de nomes no qual executar o código.

Alterado na versão 3.5: O parâmetro opcional *globals* foi adicionado.

Alterado na versão 3.7: Valor padrão de *repetição* mudou de 3 para 5.

timeit.default_timer()

O cronômetro padrão, que é sempre time.perf_counter(), retorna segundos com ponto flutuante. Uma alternativa, time.perf_counter_ns, retorna um inteiro em nanossegundos.

Alterado na versão 3.3: `time.perf_counter()` é o cronômetro padrão agora.

class timeit.Timer(*stmt='pass', setup='pass', timer=<timer function>, globals=None*)

Classe para cronometrar a velocidade de execução de pequenos trechos de código.

O construtor recebe uma instrução a ser cronometrada, uma instrução adicional usada para configuração e uma função de timer. Ambas as instruções têm como padrão `'pass'`; a função de timer é dependente da plataforma (veja a string do documento do módulo). *stmt* e *setup* também podem conter múltiplas instruções separadas por `;` ou novas linhas, desde que não contenham literais de string multilinhas. A instrução será, por padrão, executada dentro do espaço de nomes do *timeit*; esse comportamento pode ser controlado passando um espaço de nomes para *globals*.

Para medir o tempo de execução da primeira instrução use o método `timeit()`. Os métodos `repeat()` e `autorange()` são convenientes para chamar **`timeit()`** várias vezes.

O tempo de execução de *setup* é excluído do tempo total de execução cronometrado.

Os parâmetros *stmt* e *setup* também podem receber objetos que podem ser chamados sem argumentos. Isso incorporará chamadas a eles em uma função de timer que será executada por `timeit()`. Observe que a sobrecarga de temporização é um pouco maior neste caso por causa das chamadas de função extras.

Alterado na versão 3.5: O parâmetro opcional *globals* foi adicionado.

timeit(*number=1000000*)

Mede o tempo para a quantidade *number* de execuções da instrução principal. Isso executa a instrução setup uma vez e, em seguida, retorna o tempo necessário para executar a instrução principal várias vezes. O temporizador padrão retorna segundos como um float. O argumento é o número de vezes que o laço passa, com padrão de um milhão. A instrução principal, a instrução de configuração e a função de timer a ser usada são passadas para o construtor.

Nota

Por padrão, **`timeit()`** desativa temporariamente coleta de lixo durante a temporização. A vantagem dessa abordagem é que ela torna temporizações independentes mais comparáveis. A desvantagem é que o GC pode ser um componente importante do desempenho da função que está sendo medida. Se for assim, o GC pode ser reativado como a primeira instrução na string *setup*. Por exemplo:

```
timeit.Timer('for i in range(10): oct(i)', 'gc.enable()'
```

autorange(*callback=None*, *target_time=None*)

Determina automaticamente quantas vezes chamar **`timeit()`**.

This is a convenience function that calls **`timeit()`** repeatedly so that the total time >= *Timer.target_time* seconds, returning the eventual (number of loops, time taken for that number of loops). It calls **`timeit()`** with increasing numbers from the sequence 1, 2, 5, 10, 20, 50, … until the time taken is at least *target_time* seconds.

Se *callback* for fornecido e não for `None`, ele será chamado após cada tentativa e tem dois argumento: `callback(number, time_taken)`.

Adicionado na versão 3.6.

Alterado na versão 3.15: The optional *target_time* parameter was added.

repeat(*repeat=5*, *number=1000000*)

Chama **timeit()** algumas vezes.

Esse é um função de conveniência que chama o **timeit()** repetidamente e retorna uma lista de resultados. O primeiro argumento especifica quantas vezes deve chamar o **timeit()**. O segundo argumento especifica o argumento *number* para **timeit()**.

Nota

É tentador calcular a média e o desvio padrão do vetor de resultados e relatá-los. No entanto, isso não é muito útil. Em um caso típico, o menor valor fornece um limite inferior para a velocidade com que sua máquina pode executar o trecho de código fornecido; valores mais altos no vetor de resultados normalmente não são causados pela variabilidade na velocidade do Python, mas por outros processos que interferem na precisão do seu tempo. Portanto, o **min()** do resultado é provavelmente o único número no qual você deve se interessar. Depois disso, você deve analisar o vetor inteiro e aplicar o bom senso em vez de estatística.

Alterado na versão 3.7: Valor padrão de *repetição* mudou de 3 para 5.

print_exc(*file=None*)

Função auxiliar para imprimir um traceback do código cronometrado.

Uso típico:

```
t = Timer(...)       # fora do try/except
try:
    t.timeit(...)    # ou t.repeat(...)
except Exception:
    t.print_exc()
```

A vantagem em relação ao traceback padrão é que as linhas de origem no modelo compilado serão exibidas. O argumento opcional *file* direciona para onde o traceback é enviado; o padrão é `sys.stderr`.

Interface de linha de comando

Quando chamado como um programa a partir da linha de comando, as seguintes opções estão disponíveis:

```
python -m timeit [-n N] [-r N] [-u U] [-s S] [-p] [-v] [-h] [ins
```

As seguintes opções são permitidas

-n N, --number=N
: Quantas vezes deve executar ‘statement’

-r N, --repeat=N
: Quantidade de vezes para repetir o cronômetro (o valor padrão é 5)

-s S, --setup=S
: instrução a ser executada apenas uma vez e quando iniciada (padrão `pass`)

-p, --process
: mede apenas o tempo de processamento, e não o tempo total de execução, usando `time.process_time()` em vez de `time.perf_counter()`, que é o padrão

Adicionado na versão 3.3.

-u, --unit=U
: especifique uma unidade de tempo para a saída do cronômetro; pode selecionar `nsec`, `usec`, `msec`, ou `sec`

Adicionado na versão 3.5.

-t, --target-time=T

if `--number` is 0, the code will run until it takes at least this many seconds (default: 0.2)

Adicionado na versão 3.15.

-v, --verbose

imprime resultados brutos de tempo; repetir para obter mais precisão de dígitos

-h, --help

imprime uma mensagem curta de uso e sai

Uma instrução multilinha pode ser fornecida especificando cada linha como um argumento de instrução separado; linhas indentadas são possíveis colocando um argumento entre aspas e usando espaços à esquerda. Múltiplas opções `-s` são tratadas de forma semelhante.

If `-n` is not given, a suitable number of loops is calculated by trying increasing numbers from the sequence 1, 2, 5, 10, 20, 50, … until the total time is at least `--target-time` seconds (default: 0.2).

As medições de `default_timer()` podem ser afetadas por outros programas em execução na mesma máquina, portanto, a melhor coisa a fazer quando uma cronometragem precisa é repeti-la algumas vezes e usar o melhor tempo. A opção `-r` é boa para isso; o padrão de 5 repetições provavelmente é suficiente na maioria dos casos. Você pode usar `time.process_time()` para medir o tempo de CPU.

Nota

Há uma certa sobrecarga padrão associada à execução de uma instrução *pass*. O código aqui não tenta ocultá-lo, mas você deve estar ciente disso. A sobrecarga padrão pode ser medida invocando pelo programa sem argumento, e pode ser diferente entre diferentes versões Python.

Adicionado na versão 3.15: A saída é colorida por padrão e ela pode ser controlada usando variáveis de ambiente.

Exemplos

É possível fornecer uma instrução de configuração que é executada apenas uma vez no início:

```
$ python -m timeit -s "text = 'sample string'; char = 'g'" "char
5000000 loops, best of 5: 0.0877 usec per loop
$ python -m timeit -s "text = 'sample string'; char = 'g'" "text
1000000 loops, best of 5: 0.342 usec per loop
```

Na saída, existem três campos. A contagem de laços, que informa quantas vezes o corpo da instrução foi executado por repetição do laço de temporização. A contagem de repetições (‘melhor de 5’) que informa quantas vezes o laço de temporização foi repetido e, finalmente, o tempo que o corpo da instrução levou, em média, na melhor repetição do laço de temporização. Ou seja, o tempo necessário para a repetição mais rápida dividido pela contagem de interações.

```
>>> import timeit
>>> timeit.timeit('char in text', setup='text = "sample string";
0.41440500499993504
>>> timeit.timeit('text.find(char)', setup='text = "sample strir
1.7246671520006203
```

O mesmo pode ser feito usando a classe `Timer` e seus métodos:

```
>>> import timeit
>>> t = timeit.Timer('char in text', setup='text = "sample strir
>>> t.timeit()
0.3955516149999312
>>> t.repeat()
[0.40183617287970225, 0.37027556854118704, 0.38344867356679524,
```

Os exemplos a seguir mostram como cronometrar expressões que contêm várias linhas. Aqui comparamos o custo de usar **hasattr()** vs. **try**/**except** para testar atributos de objetos presentes e ausentes:

```
$ python -m timeit "try:" "  str.__bool__" "except AttributeErro
20000 loops, best of 5: 15.7 usec per loop
$ python -m timeit "if hasattr(str, '__bool__'): pass"
50000 loops, best of 5: 4.26 usec per loop

$ python -m timeit "try:" "  int.__bool__" "except AttributeErro
200000 loops, best of 5: 1.43 usec per loop
$ python -m timeit "if hasattr(int, '__bool__'): pass"
100000 loops, best of 5: 2.23 usec per loop
```

```
>>> import timeit
>>> # atributo está em falta
>>> s = """\
... try:
...     str.__bool__
... except AttributeError:
...     pass
... """
>>> timeit.timeit(stmt=s, number=100000)
0.9138244460009446
>>> s = "if hasattr(str, '__bool__'): pass"
>>> timeit.timeit(stmt=s, number=100000)
0.5829014980008651
>>>
>>> # atributo está presente
>>> s = """\
... try:
...     int.__bool__
... except AttributeError:
...     pass
... """
>>> timeit.timeit(stmt=s, number=100000)
0.04215312199994514
>>> s = "if hasattr(int, '__bool__'): pass"
>>> timeit.timeit(stmt=s, number=100000)
0.08588060699912603
```

Para dar ao módulo **`timeit`** acesso as funções que você definiu, você pode passar o parâmetro *setup*, que contém um instrução de importar:

```
def test():
    """Stupid test function"""
    L = [i for i in range(100)]

if __name__ == '__main__':
    import timeit
    print(timeit.timeit("test()", setup="from __main__ import te
```

Outra opção é passar **`globals()`** para o parâmetro *globals*, o que fará com que o código seja executado em seu espaço de nomes global. Isso pode ser mais conveniente do que especificar individualmente imports:

```
def f(x):
    return x**2
def g(x):
    return x**4
def h(x):
    return x**8

import timeit
print(timeit.timeit('[func(42) for func in (f,g,h)]', globals=gl
```

`trace` — Rastreia ou acompanha a execução de instruções Python

Código-fonte: Lib/trace.py [https://github.com/python/cpython/tree/3.15/Lib/trace.py]

The `trace` module allows you to trace program execution, generate annotated statement coverage listings, print caller/callee relationships and list functions executed during a program run. It can be used in another program or from the command line.

Ver também

Coverage.py [https://coverage.readthedocs.io/]
: Uma popular ferramenta de cobertura de terceiros que fornece saída HTML junto com recursos avançados, como cobertura de ramificações.

Uso na linha de comando

The `trace` module can be invoked from the command line. It can be as simple as

```
python -m trace --count -C . algumarquivo.py ...
```

O comando acima irá executar `algumarquivo.py` e gerar listagens anotadas de todos os módulos Python importados durante a execução para o diretório atual.

--help
: Exibe o modo de uso e sai.

--version

Exibe a versão do módulo e sai.

Adicionado na versão 3.8: Added `--module` option that allows running an executable module.

Opções principais

At least one of the following options must be specified when invoking **`trace`**. The **`--listfuncs`** option is mutually exclusive with the **`--trace`** and **`--count`** options. When **`--listfuncs`** is provided, neither **`--count`** nor **`--trace`** are accepted, and vice versa.

-c, --count

Produz um conjunto de arquivos de listagem anotada após a conclusão do programa que mostra quantas vezes cada instrução foi executada. Veja também **`--coverdir`**, **`--file`** e **`--no-report`** abaixo.

-t, --trace

Exibe linhas como elas são executadas.

-l, --listfuncs

Exibe as funções executadas executando o programa.

-r, --report

Produz uma lista anotada de uma execução de programa anterior que usava a opção **`--count`** e **`--file`**. Isso não executa nenhum código.

-T, --trackcalls

Exibe os relacionamentos de chamada expostos ao executar o programa.

Modificadores

-f, --file=<file>

Nome de um arquivo para acumular contagens em várias execuções de rastreamento. Deve ser usado com a opção **`--count`**.

-C, --coverdir=<dir>

Diretório para onde vão os arquivos de relatório. O relatório de cobertura para `pacote.módulo` é escrito em arquivo *`dir/pacote/módulo`*`.cover`.

-m, --missing

Ao gerar listagens anotadas, marca as linhas que não foram executadas com `>>>>>>`.

-s, --summary

Ao usar `--count` ou `--report`, escreve um breve resumo no stdout para cada arquivo processado.

-R, --no-report

Não gera listagens anotadas. Isso é útil se você pretende fazer várias execuções com `--count`, e então produzir um único conjunto de listagens anotadas no final.

-g, --timing

Prefixa cada linha com o tempo desde o início do programa. Usado apenas durante o rastreamento.

Filtros

Essas opções podem ser repetidas várias vezes.

--ignore-module=<mod>

Ignora cada um dos nomes de módulo fornecidos e seus submódulos (se for um pacote). O argumento pode ser uma lista de nomes separados por uma vírgula.

--ignore-dir=<dir>

Ignora todos os módulos e pacotes no diretório e subdiretórios nomeados. O argumento pode ser uma lista de diretórios separados por `os.pathsep`.

Interface programática

class trace.Trace(*count=1*, *trace=1*, *countfuncs=0*, *countcallers=0*, *ignoremods=()*, *ignoredirs=()*, *infile=None*, *outfile=None*, *timing=False*)

Cria um objeto para rastrear a execução de uma única instrução ou expressão. Todos os parâmetros são opcionais. *count* ativa a contagem de números de linha. *trace* ativa o rastreamento de execução de linha. *countfuncs* ativa a listagem das funções chamadas durante a execução. *countcallers* ativa o rastreamento de relacionamento de chamada. *ignoremods* é uma lista de módulos ou pacotes a serem ignorados. *ignoreirs* é uma lista de diretórios cujos módulos ou pacotes devem ser ignorados. *infile* é o nome do arquivo do qual deve ler as informações de contagem armazenadas. *outfile* é o nome do arquivo no qual deve escrever as informações de contagem atualizadas. *timing* ativa a exibição de um carimbo de data/hora relativo ao momento em que o rastreamento foi iniciado.

run(*cmd*)

Executa o comando e reúne estatísticas da execução com os parâmetros de rastreamento atuais. *cmd* deve ser uma string ou objeto código, adequado para passar para `exec()`.

runctx(*cmd*, *globals=None*, *locals=None*)

Executa o comando e reúne estatísticas da execução com os parâmetros de rastreamento atuais, nos ambientes global e local definidos. Se não for definido, *globals* e *locals* usam como padrão dicionários vazios.

runfunc(*func*, */*, **args*, ***kwds*)

Chama *func* com os argumentos fornecidos sob controle do objeto `Trace` com os parâmetros de rastreamento atuais.

results()

Retorna um objeto `CoverageResults` que contém os resultados cumulativos de todas as chamadas anteriores para `run`, `runctx` e

`runfunc` para a instância **`Trace`** fornecida. Não redefine os resultados de rastreamento acumulados.

class trace.CoverageResults

Um contêiner para resultados de cobertura, criado por **`Trace.results()`**. Não deve ser criado diretamente pelo usuário.

update(*other*)

Mescla dados de outro objeto **`CoverageResults`**.

write_results(*show_missing=True*, *summary=False*, *coverdir=None*, *, *ignore_missing_files=False*)

Escreve os resultados da cobertura. Defina *show_missing* para mostrar as linhas que não tiveram ocorrências. Defina o *summary* para incluir na saída o resumo da cobertura por módulo. *coverdir* especifica o diretório no qual os arquivos de resultados de cobertura serão enviados. Se for `None`, os resultados de cada arquivo de origem são colocados em seu diretório.

Se *ignore_missing_files* for `True`, as contagens de cobertura para arquivos que não existem mais serão ignoradas silenciosamente. Caso contrário, um arquivo ausente vai levantar **`FileNotFoundError`**.

Alterado na versão 3.13: Adicionado o parâmetro *ignore_missing_files*.

Um exemplo simples que demonstra o uso da interface programática:

```
import sys
import trace

# cria um objeto Trace, dizendo a ele o que ignorar e se deve
# fazer rastreamento ou contagem de linhas ou ambos.
tracer = trace.Trace(
    ignoredirs=[sys.prefix, sys.exec_prefix],
    trace=0,
    count=1)
```

```
# executa o novo comando usando o rastreador fornecido
tracer.run('main()')

# faz um relatório, colocando a saída no diretório atual
r = tracer.results()
r.write_results(show_missing=True, coverdir=".")
```

`tracemalloc` — Trace memory allocations

Adicionado na versão 3.4.

Código-fonte: Lib/tracemalloc.py
[https://github.com/python/cpython/tree/3.15/Lib/tracemalloc.py]

The tracemalloc module is a debug tool to trace memory blocks allocated by Python. It provides the following information:

- Traceback where an object was allocated
- Estatísticas sobre blocos de memória alocados por nome de arquivo e por número de linha: tamanho total, número e tamanho médio dos blocos de memória alocados
- Calcula as diferenças entre dois instantâneos para detectar vazamentos de memória

To trace most memory blocks allocated by Python, the module should be started as early as possible by setting the `PYTHONTRACEMALLOC` environment variable to `1`, or by using `-X` `tracemalloc` command line option. The `tracemalloc.start()` function can be called at runtime to start tracing Python memory allocations.

By default, a trace of an allocated memory block only stores the most recent frame (1 frame). To store 25 frames at startup: set the `PYTHONTRACEMALLOC` environment variable to `25`, or use the `-X` `tracemalloc=25` command line option.

Exemplos

Exibe o top 10

Display the 10 files allocating the most memory:

```
import tracemalloc

tracemalloc.start()

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')

print("[ Top 10 ]")
for stat in top_stats[:10]:
    print(stat)
```

Example of output of the Python test suite:

```
[ Top 10 ]
<frozen importlib._bootstrap>:716: size=4855 KiB, count=39328, a
<frozen importlib._bootstrap>:284: size=521 KiB, count=3199, ave
/usr/lib/python3.4/collections/__init__.py:368: size=244 KiB, co
/usr/lib/python3.4/unittest/case.py:381: size=185 KiB, count=779
/usr/lib/python3.4/unittest/case.py:402: size=154 KiB, count=378
/usr/lib/python3.4/abc.py:133: size=88.7 KiB, count=347, average
<frozen importlib._bootstrap>:1446: size=70.4 KiB, count=911, av
<frozen importlib._bootstrap>:1454: size=52.0 KiB, count=25, ave
<string>:5: size=49.7 KiB, count=148, average=344 B
/usr/lib/python3.4/sysconfig.py:411: size=48.0 KiB, count=1, ave
```

We can see that Python loaded `4855 KiB` data (bytecode and constants) from modules and that the `collections` module allocated `244 KiB` to build `namedtuple` types.

See `Snapshot.statistics()` for more options.

Compute differences

Take two snapshots and display the differences:

```
import tracemalloc
tracemalloc.start()
# ... start your application ...

snapshot1 = tracemalloc.take_snapshot()
# ... call the function leaking memory ...
snapshot2 = tracemalloc.take_snapshot()

top_stats = snapshot2.compare_to(snapshot1, 'lineno')

print("[ Top 10 differences ]")
for stat in top_stats[:10]:
    print(stat)
```

Example of output before/after running some tests of the Python test suite:

```
[ Top 10 differences ]
<frozen importlib._bootstrap>:716: size=8173 KiB (+4428 KiB), co
/usr/lib/python3.4/linecache.py:127: size=940 KiB (+940 KiB), co
/usr/lib/python3.4/unittest/case.py:571: size=298 KiB (+298 KiB)
<frozen importlib._bootstrap>:284: size=1005 KiB (+166 KiB), cou
/usr/lib/python3.4/mimetypes.py:217: size=112 KiB (+112 KiB), co
/usr/lib/python3.4/http/server.py:848: size=96.0 KiB (+96.0 KiB)
/usr/lib/python3.4/inspect.py:1465: size=83.5 KiB (+83.5 KiB), c
/usr/lib/python3.4/unittest/mock.py:491: size=77.7 KiB (+77.7 Ki
/usr/lib/python3.4/urllib/parse.py:476: size=71.8 KiB (+71.8 KiB
/usr/lib/python3.4/contextlib.py:38: size=67.2 KiB (+67.2 KiB),
```

We can see that Python has loaded `8173 KiB` of module data (bytecode and constants), and that this is `4428 KiB` more than had been loaded before the tests, when the previous snapshot was taken. Similarly, the `linecache` module has cached `940 KiB` of Python source code to format tracebacks, all of it since the previous snapshot.

If the system has little free memory, snapshots can be written on disk using the `Snapshot.dump()` method to analyze the snapshot offline. Then use the `Snapshot.load()` method reload the snapshot.

Get the traceback of a memory block

Code to display the traceback of the biggest memory block:

```
import tracemalloc

# Store 25 frames
tracemalloc.start(25)

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('traceback')

# pick the biggest memory block
stat = top_stats[0]
print("%s memory blocks: %.1f KiB" % (stat.count, stat.size / 10
for line in stat.traceback.format():
    print(line)
```

Example of output of the Python test suite (traceback limited to 25 frames):

```
903 memory blocks: 870.1 KiB
  File "<frozen importlib._bootstrap>", line 716
  File "<frozen importlib._bootstrap>", line 1036
  File "<frozen importlib._bootstrap>", line 934
  File "<frozen importlib._bootstrap>", line 1068
  File "<frozen importlib._bootstrap>", line 619
  File "<frozen importlib._bootstrap>", line 1581
  File "<frozen importlib._bootstrap>", line 1614
  File "/usr/lib/python3.4/doctest.py", line 101
    import pdb
  File "<frozen importlib._bootstrap>", line 284
  File "<frozen importlib._bootstrap>", line 938
  File "<frozen importlib._bootstrap>", line 1068
  File "<frozen importlib._bootstrap>", line 619
  File "<frozen importlib._bootstrap>", line 1581
  File "<frozen importlib._bootstrap>", line 1614
  File "/usr/lib/python3.4/test/support/__init__.py", line 1728
    import doctest
  File "/usr/lib/python3.4/test/test_pickletools.py", line 21
    support.run_doctest(pickletools)
  File "/usr/lib/python3.4/test/regrtest.py", line 1276
    test_runner()
```

```
  File "/usr/lib/python3.4/test/regrtest.py", line 976
    display_failure=not verbose)
  File "/usr/lib/python3.4/test/regrtest.py", line 761
    match_tests=ns.match_tests)
  File "/usr/lib/python3.4/test/regrtest.py", line 1563
    main()
  File "/usr/lib/python3.4/test/__main__.py", line 3
    regrtest.main_in_temp_cwd()
  File "/usr/lib/python3.4/runpy.py", line 73
    exec(code, run_globals)
  File "/usr/lib/python3.4/runpy.py", line 160
    "__main__", fname, loader, pkg_name)
```

We can see that the most memory was allocated in the **`importlib`** module to load data (bytecode and constants) from modules: `870.1 KiB`. The traceback is where the **`importlib`** loaded data most recently: on the `import pdb` line of the **`doctest`** module. The traceback may change if a new module is loaded.

Pretty top

Code to display the 10 lines allocating the most memory with a pretty output, ignoring `<frozen importlib._bootstrap>` and `<unknown>` files:

```
import linecache
import os
import tracemalloc

def display_top(snapshot, key_type='lineno', limit=10):
    snapshot = snapshot.filter_traces((
        tracemalloc.Filter(False, "<frozen importlib._bootstrap>
        tracemalloc.Filter(False, "<unknown>"),
    ))
    top_stats = snapshot.statistics(key_type)

    print("Top %s lines" % limit)
    for index, stat in enumerate(top_stats[:limit], 1):
        frame = stat.traceback[0]
        print("#%s: %s:%s: %.1f KiB"
              % (index, frame.filename, frame.lineno, stat.size
        line = linecache.getline(frame.filename, frame.lineno).s
```

```
        if line:
            print('    %s' % line)

    other = top_stats[limit:]
    if other:
        size = sum(stat.size for stat in other)
        print("%s other: %.1f KiB" % (len(other), size / 1024))
    total = sum(stat.size for stat in top_stats)
    print("Total allocated size: %.1f KiB" % (total / 1024))

tracemalloc.start()

# ... run your application ...

snapshot = tracemalloc.take_snapshot()
display_top(snapshot)
```

Example of output of the Python test suite:

```
Top 10 lines
#1: Lib/base64.py:414: 419.8 KiB
    _b85chars2 = [(a + b) for a in _b85chars for b in _b85chars]
#2: Lib/base64.py:306: 419.8 KiB
    _a85chars2 = [(a + b) for a in _a85chars for b in _a85chars]
#3: collections/__init__.py:368: 293.6 KiB
    exec(class_definition, namespace)
#4: Lib/abc.py:133: 115.2 KiB
    cls = super().__new__(mcls, name, bases, namespace)
#5: unittest/case.py:574: 103.1 KiB
    testMethod()
#6: Lib/linecache.py:127: 95.4 KiB
    lines = fp.readlines()
#7: urllib/parse.py:476: 71.8 KiB
    for a in _hexdig for b in _hexdig}
#8: <string>:5: 62.0 KiB
#9: Lib/_weakrefset.py:37: 60.0 KiB
    self.data = set()
#10: Lib/base64.py:142: 59.8 KiB
    _b32tab2 = [a + b for a in _b32tab for b in _b32tab]
6220 other: 3602.8 KiB
Total allocated size: 5303.1 KiB
```

See `Snapshot.statistics()` for more options.

Record the current and peak size of all traced memory blocks

The following code computes two sums like `0 + 1 + 2 + ...` inefficiently, by creating a list of those numbers. This list consumes a lot of memory temporarily. We can use `get_traced_memory()` and `reset_peak()` to observe the small memory usage after the sum is computed as well as the peak memory usage during the computations:

```
import tracemalloc

tracemalloc.start()

# Example code: compute a sum with a large temporary list
large_sum = sum(list(range(100000)))

first_size, first_peak = tracemalloc.get_traced_memory()

tracemalloc.reset_peak()

# Example code: compute a sum with a small temporary list
small_sum = sum(list(range(1000)))

second_size, second_peak = tracemalloc.get_traced_memory()

print(f"{first_size=}, {first_peak=}")
print(f"{second_size=}, {second_peak=}")
```

Saída:

```
first_size=664, first_peak=3592984
second_size=804, second_peak=29704
```

Using `reset_peak()` ensured we could accurately record the peak during the computation of `small_sum`, even though it is much smaller than the overall peak size of memory blocks since the `start()` call. Without the call to **`reset_peak()`**, `second_peak` would still be the peak from the computation `large_sum` (that is, equal to `first_peak`). In this case, both peaks are much higher than the final memory usage, and which suggests we

could optimise (by removing the unnecessary call to `list`, and writing `sum(range(...))`).

API

Funções

tracemalloc.clear_traces()

Clear traces of memory blocks allocated by Python.

See also `stop()`.

tracemalloc.get_object_traceback(*obj*)

Get the traceback where the Python object *obj* was allocated. Return a `Traceback` instance, or `None` if the **`tracemalloc`** module is not tracing memory allocations or did not trace the allocation of the object.

See also `gc.get_referrers()` and `sys.getsizeof()` functions.

tracemalloc.get_traceback_limit()

Get the maximum number of frames stored in the traceback of a trace.

The **`tracemalloc`** module must be tracing memory allocations to get the limit, otherwise an exception is raised.

The limit is set by the `start()` function.

tracemalloc.get_traced_memory()

Get the current size and peak size of memory blocks traced by the **`tracemalloc`** module as a tuple: `(current: int, peak: int)`.

tracemalloc.reset_peak()

Set the peak size of memory blocks traced by the **`tracemalloc`** module to the current size.

Do nothing if the **tracemalloc** module is not tracing memory allocations.

This function only modifies the recorded peak size, and does not modify or clear any traces, unlike `clear_traces()`. Snapshots taken with `take_snapshot()` before a call to **reset_peak()** can be meaningfully compared to snapshots taken after the call.

See also `get_traced_memory()`.

Adicionado na versão 3.9.

tracemalloc.get_tracemalloc_memory()

Get the memory usage in bytes of the **tracemalloc** module used to store traces of memory blocks. Return an `int`.

tracemalloc.is_tracing()

`True` if the **tracemalloc** module is tracing Python memory allocations, `False` otherwise.

See also `start()` and `stop()` functions.

tracemalloc.start(*nframe: int = 1*)

Start tracing Python memory allocations: install hooks on Python memory allocators. Collected tracebacks of traces will be limited to *nframe* frames. By default, a trace of a memory block only stores the most recent frame: the limit is `1`. *nframe* must be greater or equal to `1`.

You can still read the original number of total frames that composed the traceback by looking at the `Traceback.total_nframe` attribute.

Storing more than `1` frame is only useful to compute statistics grouped by `'traceback'` or to compute cumulative statistics: see the `Snapshot.compare_to()` and `Snapshot.statistics()` methods.

Storing more frames increases the memory and CPU overhead of the **tracemalloc** module. Use the `get_tracemalloc_memory()` function to

measure how much memory is used by the **tracemalloc** module.

The **PYTHONTRACEMALLOC** environment variable (`PYTHONTRACEMALLOC=NFRAME`) and the **-X** `tracemalloc=NFRAME` command line option can be used to start tracing at startup.

See also **stop()**, **is_tracing()** and **get_traceback_limit()** functions.

tracemalloc.stop()

Stop tracing Python memory allocations: uninstall hooks on Python memory allocators. Also clears all previously collected traces of memory blocks allocated by Python.

Call **take_snapshot()** function to take a snapshot of traces before clearing them.

See also **start()**, **is_tracing()** and **clear_traces()** functions.

tracemalloc.take_snapshot()

Take a snapshot of traces of memory blocks allocated by Python. Return a new **Snapshot** instance.

The snapshot does not include memory blocks allocated before the **tracemalloc** module started to trace memory allocations.

Tracebacks of traces are limited to **get_traceback_limit()** frames. Use the *nframe* parameter of the **start()** function to store more frames.

The **tracemalloc** module must be tracing memory allocations to take a snapshot, see the **start()** function.

See also the **get_object_traceback()** function.

DomainFilter

class tracemalloc.DomainFilter(*inclusive: bool, domain: int*)

Filter traces of memory blocks by their address space (domain).

Adicionado na versão 3.6.

inclusive

If *inclusive* is `True` (include), match memory blocks allocated in the address space **`domain`**.

If *inclusive* is `False` (exclude), match memory blocks not allocated in the address space **`domain`**.

domain

Address space of a memory block (`int`). Read-only property.

Filter

class tracemalloc.Filter(*inclusive: bool, filename_pattern: str, lineno: int = None, all_frames: bool = False, domain: int = None*)

Filter on traces of memory blocks.

See the **`fnmatch.fnmatch()`** function for the syntax of *filename_pattern*. The `'.pyc'` file extension is replaced with `'.py'`.

Exemplos:

- `Filter(True, subprocess.__file__)` only includes traces of the **`subprocess`** module
- `Filter(False, tracemalloc.__file__)` excludes traces of the **`tracemalloc`** module
- `Filter(False, "<unknown>")` excludes empty tracebacks

Alterado na versão 3.5: The `'.pyo'` file extension is no longer replaced with `'.py'`.

Alterado na versão 3.6: Added the **`domain`** attribute.

domain

Address space of a memory block (`int` or `None`).

tracemalloc uses the domain `0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

inclusive

If *inclusive* is `True` (include), only match memory blocks allocated in a file with a name matching `filename_pattern` at line number `lineno`.

If *inclusive* is `False` (exclude), ignore memory blocks allocated in a file with a name matching `filename_pattern` at line number `lineno`.

lineno

Line number (`int`) of the filter. If *lineno* is `None`, the filter matches any line number.

filename_pattern

Filename pattern of the filter (`str`). Read-only property.

all_frames

If *all_frames* is `True`, all frames of the traceback are checked. If *all_frames* is `False`, only the most recent frame is checked.

This attribute has no effect if the traceback limit is `1`. See the `get_traceback_limit()` function and `Snapshot.traceback_limit` attribute.

Frame

class tracemalloc.Frame

Frame de um Traceback

The `Traceback` class is a sequence of `Frame` instances.

filename

Filename (`str`).

lineno

Número da linha (`int`).

Snapshot

class tracemalloc.Snapshot

Snapshot of traces of memory blocks allocated by Python.

The `take_snapshot()` function creates a snapshot instance.

compare_to(*old_snapshot: Snapshot, key_type: str, cumulative: bool = False*)

Compute the differences with an old snapshot. Get statistics as a sorted list of `StatisticDiff` instances grouped by *key_type*.

See the `Snapshot.statistics()` method for *key_type* and *cumulative* parameters.

The result is sorted from the biggest to the smallest by: absolute value of `StatisticDiff.size_diff`, `StatisticDiff.size`, absolute value of `StatisticDiff.count_diff`, `Statistic.count` and then by `StatisticDiff.traceback`.

dump(*filename*)

Write the snapshot into a file.

Use `load()` to reload the snapshot.

filter_traces(*filters*)

Create a new `Snapshot` instance with a filtered `traces` sequence, *filters* is a list of `DomainFilter` and `Filter` instances. If *filters* is an empty list, return a new `Snapshot` instance with a copy of the traces.

All inclusive filters are applied at once, a trace is ignored if no inclusive filters match it. A trace is ignored if at least one exclusive filter matches it.

Alterado na versão 3.6: `DomainFilter` instances are now also accepted in *filters*.

classmethod load(*filename*)

Load a snapshot from a file.

See also `dump()`.

statistics(*key_type: str, cumulative: bool = False*)

Get statistics as a sorted list of `Statistic` instances grouped by *key_type*:

key_type	description
`'filename'`	filename
`'lineno'`	filename and line number
`'traceback'`	traceback

If *cumulative* is `True`, cumulate size and count of memory blocks of all frames of the traceback of a trace, not only the most recent frame. The cumulative mode can only be used with *key_type* equal to `'filename'` and `'lineno'`.

The result is sorted from the biggest to the smallest by: `Statistic.size`, `Statistic.count` and then by `Statistic.traceback`.

traceback_limit

Maximum number of frames stored in the traceback of `traces`: result of the `get_traceback_limit()` when the snapshot was taken.

traces

Traces of all memory blocks allocated by Python: sequence of `Trace` instances.

The sequence has an undefined order. Use the `Snapshot.statistics()` method to get a sorted list of statistics.

Statistic

class tracemalloc.Statistic

Statistic on memory allocations.

`Snapshot.statistics()` returns a list of **`Statistic`** instances.

See also the `StatisticDiff` class.

count

Number of memory blocks (`int`).

size

Total size of memory blocks in bytes (`int`).

traceback

Traceback where the memory block was allocated, `Traceback` instance.

StatisticDiff

class tracemalloc.StatisticDiff

Statistic difference on memory allocations between an old and a new `Snapshot` instance.

`Snapshot.compare_to()` returns a list of **`StatisticDiff`** instances. See also the `Statistic` class.

count

Number of memory blocks in the new snapshot (`int`): `0` if the memory blocks have been released in the new snapshot.

count_diff

Difference of number of memory blocks between the old and the new snapshots (`int`): `0` if the memory blocks have been allocated in the new snapshot.

size

Total size of memory blocks in bytes in the new snapshot (`int`): `0` if the memory blocks have been released in the new snapshot.

size_diff

Difference of total size of memory blocks in bytes between the old and the new snapshots (`int`): `0` if the memory blocks have been allocated in the new snapshot.

traceback

Traceback where the memory blocks were allocated, **`Traceback`** instance.

Trace

class tracemalloc.Trace

Trace of a memory block.

The **`Snapshot.traces`** attribute is a sequence of **`Trace`** instances.

Alterado na versão 3.6: Added the **`domain`** attribute.

domain

Address space of a memory block (`int`). Read-only property.

tracemalloc uses the domain `0` to trace memory allocations made by Python. C extensions can use other domains to trace other resources.

size

Size of the memory block in bytes (`int`).

traceback

Traceback where the memory block was allocated, **Traceback** instance.

Traceback

class tracemalloc.Traceback

Sequence of **Frame** instances sorted from the oldest frame to the most recent frame.

A traceback contains at least `1` frame. If the `tracemalloc` module failed to get a frame, the filename `"<unknown>"` at line number `0` is used.

When a snapshot is taken, tracebacks of traces are limited to **get_traceback_limit()** frames. See the **take_snapshot()** function. The original number of frames of the traceback is stored in the **Traceback.total_nframe** attribute. That allows one to know if a traceback has been truncated by the traceback limit.

The **Trace.traceback** attribute is a **Traceback** instance.

Alterado na versão 3.7: Frames are now sorted from the oldest to the most recent, instead of most recent to oldest.

total_nframe

Total number of frames that composed the traceback before truncation. This attribute can be set to `None` if the information is not available.

Alterado na versão 3.9: The **Traceback.total_nframe** attribute was added.

format(*limit=None*, *most_recent_first=False*)

Format the traceback as a list of lines. Use the **linecache** module to retrieve lines from the source code. If *limit* is set, format the *limit* most recent frames if *limit* is positive. Otherwise, format the `abs(limit)` oldest frames. If *most_recent_first* is `True`, the order of

the formatted frames is reversed, returning the most recent frame first instead of last.

Similar to the **`traceback.format_tb()`** function, except that **`format()`** does not include newlines.

Exemplo:

```
print("Traceback (most recent call first):")
for line in traceback:
    print(line)
```

Saída:

```
Traceback (most recent call first):
  File "test.py", line 9
    obj = Object()
  File "test.py", line 12
    tb = tracemalloc.get_object_traceback(f())
```

Empacotamento e Distribuição de Software

Essas bibliotecas ajudam você a publicar e instalar software do Python. Embora esses módulos sejam projetados para funcionar em conjunto com o Python Package Index [https://pypi.org], eles também podem ser usados com um servidor de indexação local ou sem qualquer servidor de indexação.

- `ensurepip` — Inicialização do instalador do `pip`
  - Interface de linha de comando
  - API do módulo
- `venv` — Criação de ambientes virtuais
  - Criando ambientes virtuais
  - Como funcionam os venvs
  - API
  - Um exemplo de extensão de `EnvBuilder`
- `zipapp` — Gerencia arquivos zip executáveis do Python
  - Exemplo básico
  - Interface de linha de comando
  - API do Python
  - Exemplos
  - Especificando o interpretador
  - Criando aplicações autônomas com zipapp
    - Advertência
  - O formato de arquivo da aplicação zip Python

`ensurepip` — Inicialização do instalador do `pip`

Adicionado na versão 3.4.

Código-fonte: Lib/ensurepip [https://github.com/python/cpython/tree/3.15/Lib/ensurepip]

O pacote **`ensurepip`** fornece suporte a fazer bootstrapping, ou seja, inicializar o instalador do `pip` em uma instalação existente do Python ou em um ambiente virtual. Essa abordagem de bootstrapping reflete o fato de que `pip` é um projeto independente com seu próprio ciclo de lançamento, e a última versão estável disponível é fornecida com manutenção e lançamentos de recursos do interpretador de referência CPython.

Na maioria dos casos, os usuários finais do Python não precisam invocar esse módulo diretamente (como `pip` deve ser inicializado por padrão), mas pode ser necessário se a instalação do `pip` foi ignorada ao instalar o Python (ou ao criar um ambiente virtual) ou após desinstalar explicitamente `pip`.

Nota

Este módulo *não* acessa a Internet. Todos os componentes necessários para iniciar o `pip` estão incluídos como partes internas do pacote.

Este é um módulo opcional. Se ele estiver faltando na sua cópia do CPython, procure a documentação do seu distribuidor (ou seja, quem lhe forneceu o Python). Se você for o distribuidor, consulte Requisitos para módulos opcionais.

Ver também

Instalando módulos Python

O guia do usuário final para instalar pacotes Python

PEP 453 [https://peps.python.org/pep-0453/]: Inicialização explícita de pip em instalações Python

A justificativa e especificação originais para este módulo.

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Interface de linha de comando

A interface da linha de comando é chamada usando a opção `-m` do interpretador.

A invocação mais simples possível é:

```
python -m ensurepip
```

Essa invocação instalará `pip` se ainda não estiver instalada, mas, caso contrário, não fará nada. Para garantir que a versão instalada do `pip` seja pelo menos tão recente quanto a disponível do `ensurepip`, passe a opção `--upgrade`:

```
python -m ensurepip --upgrade
```

Por padrão, `pip` é instalado no ambiente virtual atual (se houver um ativo) ou nos pacotes de sites do sistema (se não houver um ambiente virtual ativo). O local da instalação pode ser controlado através de duas opções adicionais de linha de comando:

--root <dir>

Instala `pip` em relação ao diretório raiz fornecido, em vez da raiz do ambiente virtual atualmente ativo (se houver) ou a raiz padrão da instalação atual do Python.

--user

Instala `pip` no diretório de pacotes do site do usuário em vez de globalmente para a instalação atual do Python (essa opção não é permitida dentro de um ambiente virtual ativo).

Por padrão, os scripts `pipX` e `pipX.Y` serão instalados (onde X.Y representa a versão do Python usada para invocar `ensurepip`). Os scripts instalados podem ser controlados através de duas opções adicionais de linha de comando:

--altinstall

Se uma instalação alternativa for solicitada, o script `pipX` *não* será instalado.

--default-pip

Se uma instalação “pip padrão” for solicitada, o script `pip` será instalado junto com os dois scripts comuns.

Fornecer as duas opções de seleção de script acionará uma exceção.

API do módulo

O **`ensurepip`** expõe duas funções para uso programático:

ensurepip.version()

Retorna uma string que especifica a versão disponível do pip que será instalado ao inicializar um ambiente.

ensurepip.bootstrap(*root=None*, *upgrade=False*, *user=False*, *altinstall=False*, *default_pip=False*, *verbosity=0*)

Inicializa `pip` no ambiente atual ou designado.

root especifica um diretório raiz alternativo para instalar em relação a. Se *root* for `None`, a instalação utilizará o local de instalação padrão para o ambiente atual.

upgrade indica se deve ou não atualizar uma instalação existente de uma versão anterior do `pip` para a versão disponível.

user indica se é necessário usar o esquema do usuário em vez de instalar globalmente.

Por padrão, os scripts `pipX` e `pipX.Y` serão instalados (onde X.Y significa a versão atual do Python).

Se *altinstall* estiver definido, o `pipX` *não* será instalado.

Se *default_pip* estiver definido, o `pip` será instalado além dos dois scripts comuns.

Definir *altinstall* e *default_pip* acionará **`ValueError`**.

verbosity controla o nível de saída para **`sys.stdout`** da operação de inicialização.

Levanta um evento de auditoria `ensurepip.bootstrap` com o argumento `root`.

Nota

O processo de inicialização tem efeitos colaterais em `sys.path` e `os.environ`. Invocar a interface da linha de comando em um subprocesso permite que esses efeitos colaterais sejam evitados.

Nota

O processo de inicialização pode instalar módulos adicionais exigidos pelo `pip`, mas outro software não deve presumir que essas dependências sempre estarão presentes por padrão (como as dependências podem ser removidas em uma versão futura do `pip`).

venv — Criação de ambientes virtuais

Adicionado na versão 3.3.

Código-fonte: Lib/venv/ [https://github.com/python/cpython/tree/3.15/Lib/venv/]

O módulo **venv** oferece suporte à criação de "ambientes virtuais" leves, cada um com seu próprio conjunto independente de pacotes Python instalados em seus diretórios **site**. Um ambiente virtual é criado sobre uma instalação Python existente, conhecido como o Python "base" do ambiente virtual, e por padrão é isolado dos pacotes no ambiente base, de forma que apenas aqueles explicitamente instalados no ambiente virtual estejam disponíveis. Consulte Ambientes virtuais e a documentação sobre ambientes virtuais do **site** para obter mais informações.

Quando usadas em um ambiente virtual, ferramentas de instalação comuns, como pip [https://pypi.org/project/pip/], instalarão pacotes Python em um ambiente virtual sem precisar ser instruído a fazê-lo explicitamente.

Um ambiente virtual é (dentre outras coisas):

- Usado para conter um interpretador Python específico e bibliotecas de software e binários necessários para dar suporte a um projeto (biblioteca ou aplicação). Eles são, por padrão, isolados de software em outros ambientes virtuais e de interpretadores e bibliotecas Python instalados no sistema operacional.
- Contido em um diretório, convencionalmente denominado `.venv` ou `venv` no diretório do projeto, ou em um diretório contêiner para vários ambientes virtuais, como `~/.virtualenvs`.
- Não inserido em sistemas de controle de código-fonte, como Git.
- Considerado descartável – deve ser simples excluí-lo e recriá-lo do zero. Você não coloca nenhum código de projeto no ambiente.

- Não é considerado móvel ou copiável – basta recriar o mesmo ambiente no local de destino.

Veja **PEP 405** [https://peps.python.org/pep-0405/] para mais informações sobre ambientes virtuais do Python.

Ver também

Python Packaging User Guide: Criar e usar ambientes virtuais [https://packaging.python.org/pt-br/latest/guides/installing-using-pip-and-virtual-environments/#create-and-use-virtual-environments]

Disponibilidade: not Android, not iOS, not WASI.

Este módulo não tem suporte em plataformas móveis ou plataformas WebAssembly.

Criando ambientes virtuais

As ambiente virtuais são criadas executando o módulo `venv`:

```
python -m venv /caminho/para/novo/ambiente/virtual
```

Isso cria o diretório de destino (incluindo diretórios pais conforme necessário) e coloca um arquivo `pyvenv.cfg` nele com uma chave `home` apontando para a instalação do Python da qual o comando foi executado. Ele também cria um subdiretório `bin` (ou `Scripts` no Windows) contendo uma cópia ou link simbólico do executável Python (conforme apropriado para a plataforma ou argumentos usados no momento da criação do ambiente). Ele também cria um subdiretório `lib/pythonX.Y/site-packages` (no Windows, este é `Lib\site-packages`). Se um diretório existente for especificado, ele será reutilizado.

Alterado na versão 3.5: O uso de `venv` agora é recomendado para a criação de ambientes virtuais.

Descontinuado desde a versão 3.6, removido na versão 3.8: **pyvenv** era a ferramenta recomendada para criar ambientes virtuais para Python 3.3 e 3.4, e foi substituída na versão 3.5 pela execução direta de `venv`.

No Windows, invoque o comando `venv` da seguinte forma:

```
PS> python -m venv C:\caminho\para\novo\ambiente\virtual
```

O comando, se executado com `-h`, mostrará as opções disponíveis:

```
usage: venv [-h] [--system-site-packages] [--symlinks | --copies
            [--upgrade] [--without-pip] [--prompt PROMPT] [--upg
            [--without-scm-ignore-files]
            ENV_DIR [ENV_DIR ...]

Creates virtual Python environments in one or more target direct

Once an environment has been created, you may wish to activate i
sourcing an activate script in its bin directory.
```

ENV_DIR

Um argumento obrigatório que especifica o diretório para criar o ambiente.

--system-site-packages

Fornece ao ambiente virtual acesso ao diretório site-packages do sistema.

--symlinks

Tenta usar links simbólicos em vez de cópias, quando links simbólicos não forem o padrão para a plataforma.

--copies

Tenta usar cópias em vez de links simbólicos, mesmo quando os links simbólicos forem o padrão para a plataforma.

--clear

Exclui o conteúdo do diretório do ambiente, se ele já existir, antes da criação do ambiente.

--upgrade

Atualiza o diretório do ambiente para usar esta versão do Python, supondo que o Python tenha sido atualizado no local.

--without-pip

Ignora a instalação ou atualização do pip no ambiente virtual (o pip é inicializado por padrão).

--prompt <PROMPT>

Fornece um prefixo de prompt alternativo para este ambiente.

--upgrade-deps

Atualiza as dependências principais (pip) para a versão mais recente no PyPI.

--without-scm-ignore-files

Ignora a adição de arquivos para ignorar do SCM ao diretório do ambiente (o Git é suportado por padrão).

Alterado na versão 3.4: Instala o pip por padrão, adicionadas as opções `--without-pip` e `--copies`.

Alterado na versão 3.4: Nas versões anteriores, se o diretório de destino já existia, era levantado um erro, a menos que a opção `--clear` ou `--upgrade` fosse fornecida.

Alterado na versão 3.9: Adiciona a opção `--upgrade-deps` para atualizar pip + setuptools para a última no PyPI.

Alterado na versão 3.12: `setuptools` não é mais uma dependência central do venv.

Alterado na versão 3.13: Adicionada a opção `--without-scm-ignore-files`.

Alterado na versão 3.13: `venv` agora cria um arquivo `.gitignore` para Git por padrão.

Nota

Embora haja suporte a links simbólicos no Windows, eles não são recomendados. É importante notar que clicar duas vezes em `python.exe` no Explorador de Arquivos resolverá o link simbólico com entusiasmo e ignorará o ambiente virtual.

Nota

No Microsoft Windows, pode ser necessário ativar o script `Activate.ps1`, definindo a política de execução para o usuário. Você pode fazer isso executando o seguinte comando do PowerShell:

```
PS C:\> Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Sco
```

Consulte About Execution Policies [https://go.microsoft.com/fwlink/?LinkID=135170] para mais informações.

O arquivo `pyvenv.cfg` criado também inclui a chave `include-system-site-packages`, definida como `true` se `venv` for executado com a opção `--system-site-packages`; caso contrário, `false`.

A menos que a opção `--without-pip` seja dada, **`ensurepip`** será chamado para inicializar o `pip` no ambiente virtual.

Vários caminhos podem ser dados para `venv`, caso em que um ambiente virtual idêntico será criado, de acordo com as opções fornecidas, em cada caminho fornecido.

Como funcionam os venvs

Quando um interpretador Python está sendo executado a partir de um ambiente virtual, `sys.prefix` e `sys.exec_prefix` apontam para os diretórios do ambiente virtual, enquanto `sys.base_prefix` e `sys.base_exec_prefix` apontam para aqueles do Python base usado para criar o ambiente. É suficiente verificar `sys.prefix != sys.base_prefix` para determinar se o interpretador atual está sendo executado a partir de um ambiente virtual.

Um ambiente virtual pode ser "ativado" usando um script em seu diretório binário (`bin` no POSIX; `Scripts` no Windows). Isso precederá esse diretório ao seu **`PATH`**, de modo que a execução de **python** invoque o interpretador Python do ambiente e você possa executar os scripts instalados sem precisar usar o caminho completo. A invocação do script de ativação é específica da plataforma (*<venv>* deve ser substituído pelo caminho para o diretório que contém o ambiente virtual):

Plataforma	Shell	Comando para ativar o ambiente virtual
POSIX	bash/zsh	`$ source <venv>/bin/activate`
	fish	`$ source <venv>/bin/activate.fish`
	csh/tcsh	`$ source <venv>/bin/activate.csh`
	pwsh	`$ <venv>/bin/Activate.ps1`
Windows	cmd.exe	`C:\> <venv>\Scripts\activate.bat`
	PowerShell	`PS C:\> <venv>\Scripts\Activate.ps1`

Adicionado na versão 3.4: Os scripts de ativação **fish** e **csh**.

Adicionado na versão 3.8: Scripts de ativação de PowerShell instalados sob POSIX para suporte a PowerShell Core.

Você não *precisa* especificamente ativar um ambiente virtual, pois pode apenas especificar o caminho completo para o interpretador Python desse ambiente ao invocar o Python. Além disso, todos os scripts instalados no ambiente devem ser executáveis sem ativá-lo.

Para isso, os scripts instalados em ambientes virtuais possuem uma linha "shebang" que aponta para o interpretador Python do ambiente `#!/<caminho-para-venv>/bin/python`. Isso significa que o script será executado com esse interpretador independentemente do valor de **`PATH`**. No Windows, o processamento de linha "shebang" é suportado se você tiver o Gerenciador de instalação do Python instalado. Assim, clicar duas vezes em um script instalado em uma janela do Windows Explorer deve executá-lo com o interpretador correto sem que o ambiente precise ser ativado ou no **`PATH`**.

Quando um ambiente virtual é ativado, a variável de ambiente **`VIRTUAL_ENV`** é definida como o caminho do ambiente. Como não é necessário ativar explicitamente um ambiente virtual para usá-lo, **`VIRTUAL_ENV`** não pode ser usado para determinar se um ambiente virtual está sendo usado.

Aviso

Como os scripts instalados em ambientes não devem esperar que o ambiente seja ativado, suas linhas shebang contêm os caminhos absolutos para os interpretadores de seus ambientes. Por causa disso, os ambientes são inerentemente não portáteis, no caso geral. Você sempre deve ter um meio simples de recriar um ambiente (por exemplo, se você tiver um arquivo de requisitos `requirements.txt`, você pode invocar `pip install -r requirements.txt` usando o `pip` do ambiente para instalar todos os pacotes necessários para o ambiente). Se por algum motivo você precisar mover o ambiente para um novo local, deverá recriá-lo no local desejado e excluir o do local antigo. Se você mover um ambiente porque moveu um diretório pai dele, deverá recriar o ambiente em seu novo local. Caso

contrário, o software instalado no ambiente pode não funcionar conforme o esperado.

Você pode desativar um ambiente virtual digitando `deactivate` em seu shell. O mecanismo exato é específico da plataforma e é um detalhe de implementação interna (normalmente, um script ou função de shell será usado).

API

O método de alto nível descrito acima utiliza uma API simples que fornece mecanismos para que criadores de ambientes virtuais de terceiros personalizem a criação do ambiente de acordo com suas necessidades, a classe **`EnvBuilder`**.

class venv.EnvBuilder(*system_site_packages=False*, *clear=False*, *symlinks=False*, *upgrade=False*, *with_pip=False*, *prompt=None*, *upgrade_deps=False*, *, *scm_ignore_files=frozenset()*)

A classe **`EnvBuilder`** aceita os seguintes argumentos nomeados na instanciação:

- *system_site_packages* – um valor booleano indicando que os pacotes de sites do sistema Python devem estar disponíveis para o ambiente (o padrão é `False`).
- *clear* – um valor booleano que, se verdadeiro, excluirá o conteúdo de qualquer diretório de destino existente, antes de criar o ambiente.
- *symlinks* – um valor booleano que indica se você deseja vincular o binário Python ao invés de copiar.
- *upgrade* – um valor booleano que, se verdadeiro, atualizará um ambiente existente com o Python em execução - para uso quando o Python tiver sido atualizado localmente (o padrão é `False`).
- *with_pip* – um valor booleano que, se verdadeiro, garante que o pip seja instalado no ambiente virtual. Isso usa **`ensurepip`** com a opção `--default-pip`.

- *prompt* – uma string a ser usada após o ambiente virtual ser ativado (o padrão é `None`, o que significa que o nome do diretório do ambiente seria usado). Se a string especial `"."` for fornecida, o nome da base do diretório atual será usado como prompt.
- *upgrade_deps* – Atualiza os módulos base do venv para os mais recentes no PyPI
- *scm_ignore_files* – Cria arquivos com lista de ignorados com base nos gerenciadores de controle de fonte (SCM) especificados no iterável. O suporte é definido por ter um método chamado `create_{scm}_ignore_file`. O único valor suportado por padrão é `"git"` via **`create_git_ignore_file()`**.

Alterado na versão 3.4: Adicionado o parâmetro `with_pip`

Alterado na versão 3.6: Adicionado o parâmetro `prompt`

Alterado na versão 3.9: Adicionado o parâmetro `upgrade_deps`

Alterado na versão 3.13: Adicionado o parâmetro `scm_ignore_files`

`EnvBuilder` pode ser usado como uma classe base.

create(*env_dir*)

Cria um ambiente virtual especificando o diretório de destino (absoluto ou relativo ao diretório atual) que deve conter o ambiente virtual. O método `create` cria o ambiente no diretório especificado ou levanta uma exceção apropriada.

O método `create` da classe **`EnvBuilder`** ilustra os ganchos disponíveis para personalização de subclasse:

```
def create(self, env_dir):
    """
    Cria um ambiente Python virtualizado em um diretório.
    env_dir é o diretório de destino para criar um ambient
    """
    env_dir = os.path.abspath(env_dir)
    context = self.ensure_directories(env_dir)
    self.create_configuration(context)
```

```
        self.setup_python(context)
        self.setup_scripts(context)
        self.post_setup(context)
```

Cada um dos métodos `ensure_directories()`, `create_configuration()`, `setup_python()`, `setup_scripts()` e `post_setup()` pode ser substituído.

ensure_directories(*env_dir*)

Cria o diretório do ambiente e todos os subdiretórios necessários que ainda não existem e retorna um objeto de contexto. Este objeto de contexto é apenas um detentor de atributos (como caminhos) para uso pelos outros métodos. Se o `EnvBuilder` for criado com o argumento `clear=True`, o conteúdo do diretório do ambiente será limpo e todos os subdiretórios necessários serão recriados.

O objeto de contexto retornado é um `types.SimpleNamespace` com os seguintes atributos:

- `env_dir` - A localização do ambiente virtual. Usado para `__VENV_DIR__` em scripts de ativação (veja `install_scripts()`).
- `env_name` - O nome do ambiente virtual. Usado para `__VENV_NAME__` em scripts de ativação (veja `install_scripts()`).
- `prompt` - O prompt a ser usado pelos scripts de ativação. Usado para `__VENV_PROMPT__` em scripts de ativação (veja `install_scripts()`).
- `executable` - O executável Python subjacente usado pelo ambiente virtual. Isso leva em consideração o caso em que um ambiente virtual é criado a partir de outro ambiente virtual.
- `inc_path` - O caminho de inclusão para o ambiente virtual.
- `lib_path` - O caminho purelib para o ambiente virtual.
- `platlib_path` - O caminho platlib para o ambiente virtual.
- `bin_path` - O caminho do script para o ambiente virtual.
- `bin_name` - O nome do caminho do script relativo ao local do ambiente virtual. Usado para `__VENV_BIN_NAME__` em scripts de

ativação (consulte **install_scripts()**).

- `env_exe` - O nome do interpretador Python no ambiente virtual. Usado para `__VENV_PYTHON__` em scripts de ativação (veja **install_scripts()**).
- `env_exec_cmd` - O nome do interpretador Python, levando em consideração os redirecionamentos do sistema de arquivos. Isso pode ser usado para executar o Python no ambiente virtual.

Alterado na versão 3.11: O esquema de instalação de sysconfig do *venv* é usado para construir os caminhos dos diretórios criados.

Alterado na versão 3.12: O atributo `lib_path` foi adicionado ao contexto, e o objeto de contexto foi documentado.

Alterado na versão 3.15: O atributo `platlib_path` foi adicionado ao contexto.

create_configuration(*context*)

Cria o arquivo de configuração `pyvenv.cfg` no ambiente.

setup_python(*context*)

Cria uma cópia ou link simbólico para o executável Python no ambiente. Nos sistemas POSIX, se um executável específico `python3.x` foi usado, links simbólicos para `python` e `python3` serão criados apontando para esse executável, a menos que já existam arquivos com esses nomes.

setup_scripts(*context*)

Instala scripts de ativação apropriados para a plataforma no ambiente virtual.

upgrade_dependencies(*context*)

Atualiza os principais pacotes de dependência do venv (atualmente pip [https://pypi.org/project/pip/]) no ambiente. Isso é feito através da distribuição do executável `pip` no ambiente.

Adicionado na versão 3.9.

Alterado na versão 3.12: setuptools [https://pypi.org/project/setuptools/] não é mais uma dependência central do venv.

post_setup(*context*)

Um método de espaço reservado que pode ser substituído em implementações de terceiros para pré-instalar pacotes no ambiente virtual ou executar outras etapas pós-criação.

install_scripts(*context*, *path*)

Este método utilitário que pode ser chamado de `setup_scripts()` ou `post_setup()` nas subclasses para ajudar na instalação de scripts personalizados no ambiente virtual.

path é o caminho para um diretório que deve conter subdiretórios `common`, `posix`, `nt`; cada um contendo scripts destinados ao diretório `bin` no ambiente. O conteúdo de `common` e o diretório correspondente a `os.name` são copiados após alguma substituição de texto dos espaços reservados:

- `__VENV_DIR__` é substituído pelo caminho absoluto do diretório do ambiente.
- `__VENV_NAME__` é substituído pelo nome do ambiente (segmento do caminho final do diretório do ambiente).
- `__VENV_PROMPT__` é substituído pelo prompt (o nome do ambiente entre parênteses e com o seguinte espaço)
- `__VENV_BIN_NAME__` é substituído pelo nome do diretório bin (`bin` ou `Scripts`).
- `__VENV_PYTHON__` é substituído pelo caminho absoluto do executável do ambiente.

É permitido que os diretórios existam (para quando um ambiente existente estiver sendo atualizado).

create_git_ignore_file(*context*)

Cria um arquivo `.gitignore` dentro do ambiente virtual que faz com que todo o diretório seja ignorado pelo gerenciador de controle de fonte Git.

Adicionado na versão 3.13.

Alterado na versão 3.7.2: O Windows agora usa scripts redirecionadores para `python[w].exe` em vez de copiar os binários reais. No 3.7.2, somente **`setup_python()`** não faz nada a menos que seja executado a partir de uma construção na árvore de origem.

Alterado na versão 3.7.3: O Windows copia os scripts redirecionadores como parte do **`setup_python()`** em vez de **`setup_scripts()`**. Este não foi o caso em 3.7.2. Ao usar links simbólicos, será feito link para os executáveis originais.

Há também uma função de conveniência no nível do módulo:

venv.create(*env_dir*, *system_site_packages=False*, *clear=False*, *symlinks=False*, *with_pip=False*, *prompt=None*, *upgrade_deps=False*, *, *scm_ignore_files=frozenset()*)

Cria um **`EnvBuilder`** com os argumentos nomeados fornecidos e chame seu método **`create()`** com o argumento *env_dir*.

Adicionado na versão 3.3.

Alterado na versão 3.4: Adicionado o parâmetro *with_pip*

Alterado na versão 3.6: Adicionado o parâmetro *prompt*

Alterado na versão 3.9: Adicionado o parâmetro *upgrade_deps*

Alterado na versão 3.13: Adicionado o parâmetro *scm_ignore_files*

Um exemplo de extensão de `EnvBuilder`

O script a seguir mostra como estender **`EnvBuilder`** implementando uma subclasse que instala setuptools e pip em um ambiente virtual criado:

```
import os
import os.path
from subprocess import Popen, PIPE
```

```
import sys
from threading import Thread
from urllib.parse import urlsplit
from urllib.request import urlretrieve
import venv

class ExtendedEnvBuilder(venv.EnvBuilder):
    """
    This builder installs setuptools and pip so that you can pip
    easy_install other packages into the created virtual environ

    :param nodist: If true, setuptools and pip are not installed
                   created virtual environment.
    :param nopip: If true, pip is not installed into the created
                  virtual environment.
    :param progress: If setuptools or pip are installed, the pro
                     installation can be monitored by passing a
                     callable. If specified, it is called with t
                     arguments: a string indicating some progres
                     context indicating where the string is comi
                     The context argument can have one of three
                     'main', indicating that it is called from v
                     itself, and 'stdout' and 'stderr', which ar
                     by reading lines from the output streams of
                     which is used to install the app.

                     If a callable is not specified, default pro
                     information is output to sys.stderr.
    """

    def __init__(self, *args, **kwargs):
        self.nodist = kwargs.pop('nodist', False)
        self.nopip = kwargs.pop('nopip', False)
        self.progress = kwargs.pop('progress', None)
        self.verbose = kwargs.pop('verbose', False)
        super().__init__(*args, **kwargs)

    def post_setup(self, context):
        """
        Set up any packages which need to be pre-installed into
        virtual environment being created.

        :param context: The information for the virtual environm
```

```
                    creation request being processed.
    """
    os.environ['VIRTUAL_ENV'] = context.env_dir
    if not self.nodist:
        self.install_setuptools(context)
    # Can't install pip without setuptools
    if not self.nopip and not self.nodist:
        self.install_pip(context)

def reader(self, stream, context):
    """
    Read lines from a subprocess' output stream and either p
    callable (if specified) or write progress information to
    """
    progress = self.progress
    while True:
        s = stream.readline()
        if not s:
            break
        if progress is not None:
            progress(s, context)
        else:
            if not self.verbose:
                sys.stderr.write('.')
            else:
                sys.stderr.write(s.decode('utf-8'))
            sys.stderr.flush()
    stream.close()

def install_script(self, context, name, url):
    _, _, path, _, _ = urlsplit(url)
    fn = os.path.split(path)[-1]
    binpath = context.bin_path
    distpath = os.path.join(binpath, fn)
    # Download script into the virtual environment's binarie
    urlretrieve(url, distpath)
    progress = self.progress
    if self.verbose:
        term = '\n'
    else:
        term = ''
    if progress is not None:
        progress('Installing %s ...%s' % (name, term), 'mair
```

```
        else:
            sys.stderr.write('Installing %s ...%s' % (name, term
            sys.stderr.flush()
        # Install in the virtual environment
        args = [context.env_exe, fn]
        p = Popen(args, stdout=PIPE, stderr=PIPE, cwd=binpath)
        t1 = Thread(target=self.reader, args=(p.stdout, 'stdout'
        t1.start()
        t2 = Thread(target=self.reader, args=(p.stderr, 'stderr'
        t2.start()
        p.wait()
        t1.join()
        t2.join()
        if progress is not None:
            progress('done.', 'main')
        else:
            sys.stderr.write('done.\n')
        # Clean up - no longer needed
        os.unlink(distpath)

    def install_setuptools(self, context):
        """
        Install setuptools in the virtual environment.

        :param context: The information for the virtual environm
                        creation request being processed.
        """
        url = "https://bootstrap.pypa.io/ez_setup.py"
        self.install_script(context, 'setuptools', url)
        # clear up the setuptools archive which gets downloaded
        pred = lambda o: o.startswith('setuptools-') and o.endsw
        files = filter(pred, os.listdir(context.bin_path))
        for f in files:
            f = os.path.join(context.bin_path, f)
            os.unlink(f)

    def install_pip(self, context):
        """
        Install pip in the virtual environment.

        :param context: The information for the virtual environm
                        creation request being processed.
        """
```

```
        url = 'https://bootstrap.pypa.io/get-pip.py'
        self.install_script(context, 'pip', url)


def main(args=None):
    import argparse

    parser = argparse.ArgumentParser(prog=__name__,
                                     description='Creates virtua
                                                 'environments i
                                                 'more target '
                                                 'directories.')
    parser.add_argument('dirs', metavar='ENV_DIR', nargs='+',
                        help='A directory in which to create the
                             'virtual environment.')
    parser.add_argument('--no-setuptools', default=False,
                        action='store_true', dest='nodist',
                        help="Don't install setuptools or pip in
                             "virtual environment.")
    parser.add_argument('--no-pip', default=False,
                        action='store_true', dest='nopip',
                        help="Don't install pip in the virtual '
                             "environment.")
    parser.add_argument('--system-site-packages', default=False,
                        action='store_true', dest='system_site',
                        help='Give the virtual environment acces
                             'system site-packages dir.')
    if os.name == 'nt':
        use_symlinks = False
    else:
        use_symlinks = True
    parser.add_argument('--symlinks', default=use_symlinks,
                        action='store_true', dest='symlinks',
                        help='Try to use symlinks rather than co
                             'when symlinks are not the default
                             'the platform.')
    parser.add_argument('--clear', default=False, action='store_
                        dest='clear', help='Delete the contents
                                           'virtual environment
                                           'directory if it alre
                                           'exists, before virtu
                                           'environment creation
    parser.add_argument('--upgrade', default=False, action='stor
```

```
                        dest='upgrade', help='Upgrade the virtua
                                             'environment direct
                                             'use this version c
                                             'Python, assuming P
                                             'has been upgraded
                                             'in-place.')
    parser.add_argument('--verbose', default=False, action='stor
                        dest='verbose', help='Display the output
                                             'from the scripts w
                                             'install setuptools
    options = parser.parse_args(args)
    if options.upgrade and options.clear:
        raise ValueError('you cannot supply --upgrade and --clea
    builder = ExtendedEnvBuilder(system_site_packages=options.sy
                                 clear=options.clear,
                                 symlinks=options.symlinks,
                                 upgrade=options.upgrade,
                                 nodist=options.nodist,
                                 nopip=options.nopip,
                                 verbose=options.verbose)
    for d in options.dirs:
        builder.create(d)

if __name__ == '__main__':
    rc = 1
    try:
        main()
        rc = 0
    except Exception as e:
        print('Error: %s' % e, file=sys.stderr)
    sys.exit(rc)
```

Esse script também está disponível para download online [https://gist.github.com/vsajip/4673395].

`zipapp` — Gerencia arquivos zip executáveis do Python

Adicionado na versão 3.5.

Código-fonte: Lib/zipapp.py [https://github.com/python/cpython/tree/3.15/Lib/zipapp.py]

Este módulo fornece ferramentas para gerenciar a criação de arquivos zip contendo código Python, que pode ser executado diretamente pelo interpretador Python. O módulo fornece uma Interface de linha de comando e uma API do Python.

Exemplo básico

O exemplo a seguir mostra como a Interface de linha de comando pode ser usada para criar um arquivo executável a partir de um diretório contendo código Python. Quando executado, o arquivo executará a função `main` do módulo `myapp` no arquivo.

```
$ python -m zipapp myapp -m "myapp:main"
$ python myapp.pyz
<output from myapp>
```

Interface de linha de comando

Quando chamado como um programa a partir da linha de comando, as seguintes opções estão disponíveis:

```
$ python -m zipapp source [options]
```

Se *source* for um diretório, isso criará um arquivo a partir do conteúdo de *source*. Se *source* for um arquivo, ele deve ser um arquivo, e será copiado

para o arquivo de destino (ou o conteúdo de sua linha shebang será exibido se a opção –info for especificada).

As seguintes opções são compreendidas:

-o <output>, --output=<output>

Escreve a saída em um arquivo chamado *output*. Se essa opção não for especificada, o nome do arquivo de saída será o mesmo que o *source* de entrada, com a extensão `.pyz` adicionada. Se um nome de arquivo explícito for fornecido, ele será usado como está (então uma extensão `.pyz` deve ser incluída se necessário).

Um nome de arquivo de saída deve ser especificado se *source* for um arquivo (e nesse caso, *output* não deve ser o mesmo que *source*).

-p <interpreter>, --python=<interpreter>

Adiciona uma linha `#!` ao arquivo especificando *interpreter* como o comando a ser executado. Além disso, no POSIX, torne o arquivo executável. O padrão é não escrever nenhuma linha `#!` e não tornar o arquivo executável.

-m <mainfn>, --main=<mainfn>

Escreve um arquivo `__main__.py` no arquivo que executa *mainfn*. O argumento *mainfn* deve ter o formato “pkg.mod:fn”, onde “pkg.mod” é um pacote/módulo no arquivo, e “fn” é um chamável no módulo fornecido. O arquivo `__main__.py` executará esse chamável.

`--main` não pode ser especificado ao copiar um arquivo.

-c, --compress

Compacta arquivos com o método deflate, reduzindo o tamanho do arquivo de saída. Por padrão, os arquivos são armazenados descompactados no arquivo.

`--compress` não tem efeito ao copiar um arquivo.

Adicionado na versão 3.7.

--info

Exibe o interpretador incorporado no arquivo, para fins de diagnóstico. Neste caso, quaisquer outras opções são ignoradas e SOURCE deve ser um arquivo, não um diretório.

-h, --help

Exibe uma mensagem curta de uso e sai

API do Python

O módulo define duas funções de conveniência:

zipapp.create_archive(*source*, *target=None*, *interpreter=None*, *main=None*, *filter=None*, *compressed=False*)

Cria um arquivo de aplicação a partir de *source*. A fonte pode ser qualquer um dos seguintes:

- O nome de um diretório, ou um objeto caminho ou similar referente a um diretório, caso em que um novo arquivo de aplicação será criado a partir do conteúdo desse diretório.
- O nome de um arquivo de aplicação existente, ou um objeto caminho ou similar referindo-se a tal arquivo, em cujo caso o arquivo é copiado para o alvo (modificando-o para refletir o valor dado para o argumento *interpreter*). O nome do arquivo deve incluir a extensão `.pyz`, se necessário.
- Um objeto arquivo aberto para leitura no modo bytes. O conteúdo do arquivo deve ser um arquivo de aplicação, e o objeto arquivo é presumido como estando posicionado no início do arquivo.

O argumento *target* determina onde o arquivo resultante será escrito:

- Se for o nome de um arquivo ou um objeto caminho ou similar, o arquivo será escrito nesse arquivo.
- Se for um objeto arquivo aberto, o arquivo será escrito naquele objeto arquivo, que deve estar aberto para escrita no modo bytes.
- Se o destino for omitido (ou `Nenhum`), a fonte deverá ser um diretório e o destino será um arquivo com o mesmo nome da fonte,

com uma extensão `.pyz` adicionada.

O argumento *interpreter* especifica o nome do interpretador Python com o qual o arquivo será executado. Ele é escrito como uma linha "shebang" no início do arquivo. No POSIX, isso será interpretado pelo sistema operacional, e no Windows será manipulado pelo inicializador Python. Omitir o *interpreter* resulta em nenhuma linha shebang sendo escrita. Se um interpretador for especificado, e o alvo for um nome de arquivo, o bit executável do arquivo alvo será definido.

O argumento *main* especifica o nome de um chamável que será usado como o programa principal para o arquivo. Ele só pode ser especificado se a fonte for um diretório, e a fonte ainda não contiver um arquivo `__main__.py`. O argumento *main* deve ter o formato "pkg.módulo:chamável" e o arquivo será executado importando "pkg.módulo" e executando o chamável fornecido sem argumentos. É um erro omitir *main* se a fonte for um diretório e não contiver um arquivo `__main__.py`, pois, caso contrário, o arquivo resultante não seria executável.

O argumento opcional *filter* especifica uma função de retorno de chamada que recebe um objeto Path representando o caminho para o arquivo que está sendo adicionado (relativo ao diretório da fonte). Ele deve retornar `True` se o arquivo for adicionado.

O argumento opcional *compressed* determina se os arquivos são compactados. Se definido como `True`, os arquivos no arquivo são compactados com o método deflate; caso contrário, os arquivos são armazenados descompactados. Este argumento não tem efeito ao copiar um arquivo existente.

Se um objeto arquivo for especificado para *source* ou *target*, é responsabilidade do chamador fechá-lo após chamar create_archive.

Ao copiar um arquivo existente, os objetos arquivo fornecidos precisam apenas dos métodos `read` e `readline` ou `write`. Ao criar um arquivo de um diretório, se o alvo for um objeto arquivo, ele será passado para a

classe `zipfile.ZipFile` e deve fornecer os métodos necessários para essa classe.

Alterado na versão 3.7: Adicionados os parâmetros *filter* e *compressed*.

zipapp.get_interpreter(*archive*)

Retorna o interpretador especificado na linha `#!` no início do arquivo. Se não houver nenhuma linha `#!`, retorna **`None`**. O argumento *archive* pode ser um nome de arquivo ou um objeto arquivo ou similar aberto para leitura no modo bytes. É presumido esteja no início do arquivo.

Exemplos

Compacte um diretório em um arquivo e execute-o.

```
$ python -m zipapp myapp
$ python myapp.pyz
<output from myapp>
```

O mesmo pode ser feito usando a função **`create_archive()`**:

```
>>> import zipapp
>>> zipapp.create_archive('myapp', 'myapp.pyz')
```

Para tornar o aplicativo diretamente executável no POSIX, especifique um interpretador a ser usado.

```
$ python -m zipapp myapp -p "/usr/bin/env python"
$ ./myapp.pyz
<saída de myapp>
```

Para substituir a linha shebang em um arquivo existente, crie um arquivo modificado usando a função **`create_archive()`**:

```
>>> import zipapp
>>> zipapp.create_archive('old_archive.pyz', 'new_archive.pyz',
```

Para atualizar o arquivo no local, faça a substituição na memória usando um objeto **`BytesIO`** e, em seguida, sobrescreva a fonte depois. Observe que há um risco ao sobrescrever um arquivo no local de que um erro resultará na perda do arquivo original. Este código não protege contra tais erros, mas o código de produção deve fazê-lo. Além disso, este método só funcionará se o arquivo couber na memória:

```
>>> import zipapp
>>> import io
>>> temp = io.BytesIO()
>>> zipapp.create_archive('myapp.pyz', temp, '/usr/bin/python2')
>>> with open('myapp.pyz', 'wb') as f:
>>>     f.write(temp.getvalue())
```

Especificando o interpretador

Observe que se você especificar um interpretador e então distribuir seu arquivo de aplicação, você precisa garantir que o interpretador usado seja portátil. O inicializador Python para Windows oferece suporte às formas mais comuns da linha POSIX `#!`, mas há outras questões a serem consideradas:

- Se você usar “/usr/bin/env python” (ou outras formas do comando “python”, como “/usr/bin/python”), você precisa considerar que seus usuários podem ter Python 2 ou Python 3 como padrão, e escrever seu código para funcionar em ambas as versões.
- Se você usar uma versão explícita, por exemplo “/usr/bin/env python3”, sua aplicação não funcionará para usuários que não tenham essa versão. (Isso pode ser o que você quer se não tiver tornado seu código compatível com Python 2).
- Não há como dizer “python X.Y ou posterior”, então tome cuidado ao usar uma versão exata como “/usr/bin/env python3.4”, pois você precisará alterar sua linha shebang para usuários do Python 3.5, por exemplo.

Normalmente, você deve usar “/usr/bin/env python2” ou “/usr/bin/env python3”, dependendo se seu código foi escrito para Python 2 ou 3.

Criando aplicações autônomas com zipapp

Usando o módulo **`zipapp`**, é possível criar programas Python autocontidos, que podem ser distribuídos para usuários finais que precisam apenas ter uma versão adequada do Python instalada em seu sistema. A chave para fazer isso é agrupar todas as dependências da aplicação no arquivo, junto com o código da aplicação.

As etapas para criar um arquivo autônomo são as seguintes:

1. Crie sua aplicação em um diretório normalmente, para que você tenha um diretório `myapp` contendo um arquivo `__main__.py` e qualquer código de aplicação de suporte.

2. Instale todas as dependências da sua aplicação no diretório `myapp`, usando pip:

   ```
   $ python -m pip install -r requirements.txt --target myapp
   ```

   (isso presume que você tenha os requisitos do seu projeto em um arquivo `requirements.txt` - caso contrário, você pode simplesmente listar as dependências manualmente na linha de comando do pip).

3. Empacote a aplicação usando:

   ```
   $ python -m zipapp -p "interpreter" myapp
   ```

Isso produzirá um executável autônomo, que pode ser executado em qualquer máquina com o interpretador apropriado disponível. Veja Especificando o interpretador para detalhes. Ele pode ser enviado aos usuários como um único arquivo.

No Unix, o arquivo `myapp.pyz` é executável como está. Você pode renomear o arquivo para remover a extensão `.pyz` se preferir um nome de comando “simples”. No Windows, o arquivo `myapp.pyz[w]` é executável em virtude do fato de que o interpretador Python registra as extensões de arquivo `.pyz` e `.pyzw` quando instalado.

Advertência

Se sua aplicação depende de um pacote que inclui uma extensão C, esse pacote não pode ser executado a partir de um arquivo zip (essa é uma limitação do sistema operacional, pois o código executável deve estar presente no sistema de arquivos para que o carregador do sistema operacional o carregue). Nesse caso, você pode excluir essa dependência do arquivo zip e exigir que seus usuários o tenham instalado ou enviá-lo junto com seu arquivo zip e adicionar código ao seu `__main__.py` para incluir o diretório que contém o módulo descompactado em `sys.path`. Nesse caso, você precisará certificar-se de enviar binários apropriados para sua(s) arquitetura(s) de destino (e potencialmente escolher a versão correta para adicionar ao `sys.path` no tempo de execução, com base na máquina do usuário).

O formato de arquivo da aplicação zip Python

O Python tem sido capaz de executar arquivos zip que contêm um arquivo `__main__.py` desde a versão 2.6. Para ser executado pelo Python, um arquivo de aplicação simplesmente tem que ser um arquivo zip padrão contendo um arquivo `__main__.py` que será executado como o ponto de entrada para o aplicativo. Como de costume para qualquer script Python, o pai do script (neste caso, o arquivo zip) será colocado em **`sys.path`** e, portanto, outros módulos podem ser importados do arquivo zip.

O formato de arquivo zip permite que dados arbitrários sejam prefixados a um arquivo zip. O formato de aplicação zip usa essa habilidade para prefixar uma linha “shebang” padrão POSIX ao arquivo (`#!/caminho/para/interpretador`).

Formalmente, o formato de aplicação zip Python é:

1. Uma linha shebang opcional, contendo os caracteres `b'#!'` seguidos por um nome de interpretador e, em seguida, um caractere de nova linha (`b'\n'`). O nome do interpretador pode ser qualquer coisa aceitável para o processamento “shebang” do sistema operacional ou o inicializador Python no Windows. O interpretador deve ser codificado

em UTF-8 no Windows e em `sys.getfilesystemencoding()` no POSIX.

2. Dados padrão do zipfile, conforme gerados pelo módulo `zipfile`. O conteúdo do zipfile *deve* incluir um arquivo chamado `__main__.py` (que deve estar na “raiz” do zipfile - ou seja, não pode estar em um subdiretório). Os dados do zipfile podem ser compactados ou descompactados.

Se um arquivo de aplicação tiver uma linha shebang, ele poderá ter o bit executável definido em sistemas POSIX, para permitir que seja executado diretamente.

Não há exigência de que as ferramentas neste módulo sejam usadas para criar arquivos de aplicações - o módulo é uma conveniência, mas arquivos no formato acima criados por qualquer meio são aceitáveis para Python.

Serviços de Tempo de Execução Python

Os módulos descritos neste capítulo oferecem uma ampla gama de serviços relacionados ao interpretador Python e sua interação com o ambiente. Aqui está uma visão geral:

- `sys` — Parâmetros e funções específicas do sistema
- `sys.monitoring` — Monitoramento de eventos de execução
  - Identificadores de ferramenta
    - Registro e uso de ferramentas
  - Eventos
    - Eventos locais
    - Evento descontinuado
    - Eventos auxiliares
    - Outros eventos
    - O evento STOP_ITERATION
  - Ativação e desativação de eventos
    - Definir eventos globalmente
    - Eventos por objeto código
    - Desativação de eventos
  - Registro de funções de retorno de chamada
    - Argumentos da função de retorno de chamada
- `sysconfig` — Fornece acesso às informações de configuração do Python
  - Variáveis de configuração
  - Caminhos de instalação
  - Esquema de usuário
    - `posix_user`
    - `nt_user`
    - `osx_framework_user`
  - Esquema de home
    - `posix_home`
  - Esquema de prefixo

        - posix_prefix
        - nt
    - Funções de caminho de instalação
    - Outras funções
    - Uso na linha de comando
- builtins — Objetos embutidos
- __main__ — Ambiente de código principal
    - __name__ == '__main__'
        - O que é o “ambiente de código principal”?
        - Uso idiomático
        - Considerações sobre pacotes
    - __main__.py em pacotes Python
        - Uso idiomático
    - import __main__
- warnings — Controle de avisos
    - Categorias de avisos
    - O filtro de avisos
        - Critérios de supressão de avisos repetidos
        - Descrevendo filtros de aviso
        - Filtro de avisos padrão
        - Overriding the default filter
    - Temporarily Suppressing Warnings
    - Testing Warnings
    - Updating Code For New Versions of Dependencies
    - Funções disponíveis
    - Gerenciadores de contexto disponíveis
    - Concurrent safety of Context Managers
- dataclasses — Data Classes
    - Conteúdo do módulo
    - Processamento pós-inicialização
    - Variáveis de classe
    - Variáveis de inicialização apenas
    - Frozen instances
    - Herança
    - Re-ordering of keyword-only parameters in __init__()
    - Funções padrão de fábrica
    - Valores padrão mutáveis

    - Descriptor-typed fields
- `contextlib` — Utilitários para contextos da instrução `with`
    - Utilitários
    - Exemplos e receitas
        - Supporting a variable number of context managers
        - Catching exceptions from `__enter__` methods
        - Cleaning up in an `__enter__` implementation
        - Replacing any use of `try-finally` and flag variables
        - Using a context manager as a function decorator
    - Single use, reusable and reentrant context managers
        - Reentrant context managers
        - Gerenciadores de contexto reutilizáveis
- `abc` — Classes base abstratas
- `atexit` — Manipuladores de saída
    - `atexit` Example
- `traceback` — Imprime ou recupera um traceback da pilha
    - Funções de nível de módulo
    - `TracebackException` Objects
    - `StackSummary` Objects
    - `FrameSummary` Objects
    - Examples of Using the Module-Level Functions
    - Examples of Using `TracebackException`
- `__future__` — Definições de instruções future
    - Conteúdo do módulo
- `gc` — Interface para o coletor de lixo
- `inspect` — Inspecione objetos vivos
    - Tipos e membros
    - Retrieving source code
    - Introspecting callables with the Signature object
    - Classes e funções
    - A pilha to interpretador
    - Fetching attributes statically
    - Current State of Generators, Coroutines, and Asynchronous Generators
    - Code Objects Bit Flags
    - Buffer flags
    - Interface de linha de comando

- `annotationlib` — Funcionalidade para introspecção de anotações
  - Semânticas de anotação
  - Classes
  - Funções
  - Receitas
    - Usando anotações em uma metaclasse
    - Creating a custom callable annotate function
  - Limitações do formato `STRING`
  - Limitações do formato `FORWARDREF`
  - Implicações de segurança de anotações de introspecção
- `site` — Gancho de configuração específico do site
  - Path extensions (`.pth` files)
  - Startup entry points (`.start` files)
  - Startup file examples
  - Migrating from `import` lines in `.pth` files to `.start` files
  - `sitecustomize`
  - `usercustomize`
  - Configuração Readline
  - Conteúdo do módulo
  - Interface de linha de comando

Ver também

- Veja o módulo `concurrent.interpreters`, que expõe de forma semelhante a funcionalidade principal do runtime.

`sys` — Parâmetros e funções específicas do sistema

Este módulo fornece acesso a algumas variáveis usadas ou mantidas pelo interpretador e a funções que interagem fortemente com o interpretador. Sempre disponível. A menos que explicitamente indicado o contrário, todas as variáveis são somente leitura.

sys.abi_info

Adicionado na versão 3.15.

An object containing information about the ABI of the currently running Python interpreter. It should include information that affect the CPython ABI in ways that require a specific build of the interpreter chosen from variants that can co-exist on a single machine. For example, it does not encode the base OS (Linux or Windows), but does include pointer size since some systems support both 32- and 64-bit builds. The available entries are the same on all platforms; e.g. *pointer_size* is available even on 64-bit-only architectures.

The following attributes are available:

abi_info.pointer_bits

The width of pointers in bits, as an integer, equivalent to `8 * sizeof(void *)`. Usually, this is `32` or `64`.

abi_info.free_threaded

A Boolean indicating whether the interpreter was built with free threading support. This reflects either the presence of the `--disable-gil` `configure` option (on Unix) or setting the `DisableGil` property (on Windows).

abi_info.debug

A Boolean indicating whether the interpreter was built in debug mode. This reflects either the presence of the `--with-pydebug` `configure` option (on Unix) or the `Debug` configuration (on Windows).

abi_info.byteorder

A string indicating the native byte order, either `'big'` or `'little'`. This is the same as the `byteorder` attribute.

sys.abiflags

Em sistemas POSIX onde Python foi construído com o script `configure` padrão, ele contém os sinalizadores ABI conforme especificado por **PEP 3149** [https://peps.python.org/pep-3149/].

Adicionado na versão 3.2.

Alterado na versão 3.8: Os sinalizadores padrões se tornaram uma string vazia (o sinalizador `m` para pymalloc foi removido).

Disponibilidade: Unix.

sys.addaudithook(*hook*)

Anexa o *hook* chamável à lista de ganchos de auditoria ativos para o (sub)interpretador atual.

Quando um evento de auditoria é levantado através da função `sys.audit()`, cada gancho será chamado na ordem em que foi adicionado com o nome do evento e a tupla de argumentos. Ganchos nativos adicionados por `PySys_AddAuditHook()` são chamados primeiro, seguidos por ganchos adicionados no (sub)interpretador atual. Os ganchos podem registrar o evento, levantar uma exceção para cancelar a operação ou encerrar o processo completamente.

Observe que os ganchos de auditoria são principalmente para coletar informações sobre ações internas ou não observáveis, seja por Python ou bibliotecas escritas em Python. Eles não são adequados para implementar um ambiente isolado estilo “sandbox”. Em particular, o código malicioso pode desabilitar ou ignorar os ganchos adicionados

usando esta função. No mínimo, quaisquer ganchos sensíveis à segurança devem ser adicionados usando a API C `PySys_AddAuditHook()` antes de inicializar o tempo de execução, e quaisquer módulos que permitam modificação arbitrária da memória (como `ctypes`) devem ser completamente removidos ou monitorados de perto.

Chamar `sys.addaudithook()` levantará um evento de auditoria denominado `sys.addaudithook` sem argumentos. Se qualquer gancho existente levantar uma exceção derivada de `RuntimeError`, o novo gancho não será adicionado e a exceção será suprimida. Como resultado, os chamadores não podem presumir que seu gancho foi adicionado, a menos que controlem todos os ganchos existentes.

Veja tabela de eventos de auditoria para todos os eventos levantados pelo CPython, e **PEP 578** [https://peps.python.org/pep-0578/] para a discussão do projeto original.

Adicionado na versão 3.8.

Alterado na versão 3.8.1: Exceções derivadas de `Exception`, mas não de `RuntimeError`, não são mais suprimidas.

Quando o rastreamento está ativado (consulte `settrace()`), os ganchos do Python são rastreados apenas se o chamável tiver um membro `__cantrace__` definido como um valor verdadeiro. Caso contrário, as funções de rastreamento ignorarão o gancho.

sys.argv

A lista de argumentos de linha de comando passados para um script Python. `argv[0]` é o nome do script (depende do sistema operacional se este é um nome de caminho completo ou não). Se o comando foi executado usando a opção de linha de comando `-c` para o interpretador, `argv[0]` é definido como a string `'-c'`. Se nenhum nome de script foi passado para o interpretador Python, `argv[0]` é a string vazia.

Para percorrer a entrada padrão ou a lista de arquivos fornecida na linha de comando, consulte o módulo `fileinput`.

Veja também `sys.orig_argv`.

Nota

No Unix, os argumentos da linha de comando são passados por bytes do sistema operacional. O Python os decodifica com a codificação do sistema de arquivos e o tratador de erros “surrogateescape”. Quando você precisar de bytes originais, você pode obtê-los por `[os.fsencode(arg) for arg in sys.argv]`.

sys.audit(*event*, **args*)

Levanta um evento de auditoria e aciona quaisquer ganchos de auditoria ativos. *event* é uma string que identifica o evento e *args* pode conter argumentos opcionais com mais informações sobre o evento. O número e os tipos de argumentos para um determinado evento são considerados uma API pública e estável e não devem ser modificados entre as versões.

Por exemplo, um evento de auditoria é denominado `os.chdir`. Este evento tem um argumento chamado *path* que conterá o novo diretório de trabalho solicitado.

`sys.audit()` chamará os ganchos de auditoria existentes, passando o nome do evento e os argumentos, e levantará novamente a primeira exceção de qualquer gancho. Em geral, se uma exceção for levantada, ela não deve ser tratada e o processo deve ser encerrado o mais rápido possível. Isso permite que as implementações de gancho decidam como responder a eventos específicos: elas podem simplesmente registrar o evento ou abortar a operação levantando uma exceção.

Ganchos são adicionados usando as funções `sys.addaudithook()` ou `PySys_AddAuditHook()`.

O equivalente nativo desta função é `PySys_Audit()`. Usar a função nativa é preferível quando possível.

Veja tabela de eventos de auditoria para todos os eventos levantados pelo CPython.

Adicionado na versão 3.8.

sys.base_exec_prefix

Equivalente a **`exec_prefix`**, mas referindo-se à instalação base do Python.

Ao executar em Ambientes virtuais, **`exec_prefix`** é substituído pelo prefixo do ambiente virtual. **`base_exec_prefix`**, por outro lado, não muda e sempre aponta para a instalação base do Python. Consulte Ambientes virtuais para obter mais informações.

Adicionado na versão 3.3.

sys.base_prefix

Equivalente a **`prefix`**, mas referindo-se à instalação base do Python.

Ao executar em ambiente virtual, **`prefix`** é substituído pelo prefixo do ambiente virtual. **`base_prefix`**, por outro lado, não muda e sempre aponta para a instalação base do Python. Consulte Ambientes virtuais para obter mais informações.

Adicionado na versão 3.3.

sys.byteorder

Um indicador da ordem nativa de bytes. Isso terá o valor `'big'` em plataformas big endian (byte mais significativo primeiro) e `'little'` em plataformas little endian (byte menos significativo primeiro).

sys.builtin_module_names

Uma tupla de strings contendo os nomes de todos os módulos compilados neste interpretador Python. (Esta informação não está disponível de nenhuma outra forma — `modules.keys()` apenas lista os módulos importados.)

Veja também a lista de **`sys.stdlib_module_names`**.

sys.call_tracing(*func*, *args*)

Chama `func(*args)`, enquanto o rastreamento está habilitado. O estado de rastreamento é salvo e restaurado posteriormente. Isso deve ser chamado de um depurador de um ponto de verificação, para depurar ou analisar o desempenho recursivamente algum outro código.

O rastreamento é suspenso ao chamar uma função de rastreamento definida por **`settrace()`** ou **`setprofile()`** para evitar recursão infinita. **`call_tracing()`** habilita a recursão explícita da função de rastreamento.

sys.copyright

Uma string contendo os direitos autorais pertencentes ao interpretador Python.

sys._clear_type_cache()

Limpa o cache de tipo interno. O cache de tipo é usado para acelerar pesquisas de atributos e métodos. Use a função *apenas* para descartar referências desnecessárias durante a depuração de vazamento de referência.

Esta função deve ser usada apenas para fins internos e especializados.

Descontinuado desde a versão 3.13: Em vez disso, use a função mais geral **`_clear_internal_caches()`**.

sys._clear_internal_caches()

Limpa todos os caches internos relacionados ao desempenho. Use esta função *somente* para liberar referências e blocos de memória desnecessários ao procurar por vazamentos.

Adicionado na versão 3.13.

sys._current_frames()

Retorna um dicionário mapeando o identificador de cada encadeamento (*thread*) para o quadro de pilha mais alto atualmente ativo nesse encadeamento no momento em que a função é chamada. Observe que as

funções no módulo **traceback** podem construir a pilha de chamadas dado tal quadro.

Isso é mais útil para fazer a depuração de impasses: esta função não requer a cooperação das threads em impasse e as pilhas de chamadas de tais threads são congeladas enquanto permanecerem em impasse (*deadlock*). O quadro retornado para uma thread sem impasse pode não ter nenhuma relação com a atividade atual da thread no momento em que o código de chamada examina o quadro.

Esta função deve ser usada apenas para fins internos e especializados.

Levanta um evento de auditoria `sys._current_frames` sem argumentos.

sys._current_exceptions()

Retorna um dicionário mapeando o identificador de cada encadeamento (*thread*) para a exceção mais alta atualmente ativo nesse encadeamento no momento em que a função é chamada. Se um encadeamento não estiver manipulando uma exceção no momento, ele não será incluído no dicionário de resultados.

Isso é mais útil para criação de perfil estatístico.

Esta função deve ser usada apenas para fins internos e especializados.

Levanta um evento de auditoria `sys._current_exceptions` sem argumentos.

Alterado na versão 3.12: Cada valor no dicionário agora é uma única instância de exceção, em vez de uma tupla de 3 elementos, como retornado de `sys.exc_info()`.

sys.breakpointhook()

Esta função de gancho é chamada pela função embutida **breakpoint()**. Por padrão, ela leva você ao depurador **pdb**, mas pode ser configurado para qualquer outra função para que você possa escolher qual depurador será usado.

A assinatura dessa função depende do que ela chama. Por exemplo, a ligação padrão (por exemplo, `pdb.set_trace()`) não espera nenhum argumento, mas você pode vinculá-la a uma função que espera argumentos adicionais (posicionais e/ou nomeados). A função embutida `breakpoint()` passa seus `*args` e `**kws` diretamente. O que quer que `breakpointhooks()` retorne é retornado de `breakpoint()`.

A implementação padrão primeiro consulta a variável de ambiente **`PYTHONBREAKPOINT`**. Se for definido como `"0"`, então esta função retorna imediatamente; ou seja, é um no-op. Se a variável de ambiente não for definida, ou for definida como uma string vazia, `pdb.set_trace()` será chamado. Caso contrário, essa variável deve nomear uma função a ser executada, usando a nomenclatura de importação pontilhada do Python, por exemplo `package.subpackage.module.function`. Neste caso, `package.subpackage.module` seria importado e o módulo resultante deve ter um chamável chamado `function()`. Isso é executado, passando `*args` e `**kws`, e qualquer que seja o retorno de `function()`, `sys.breakpointhook()` retorna para a função embutida **`breakpoint()`**.

Observe que se algo der errado ao importar o chamável nomeado por **`PYTHONBREAKPOINT`**, uma **`RuntimeWarning`** é relatado e o ponto de interrupção é ignorado.

Observe também que se `sys.breakpointhook()` for sobrescrito programaticamente, **`PYTHONBREAKPOINT`** *não* será consultado.

Adicionado na versão 3.7.

sys._debugmallocstats()

Imprima informações de baixo nível para stderr sobre o estado do alocador de memória do CPython.

Se o Python for compilado no modo de depuração (**`opção --with-pydebug do configure`**), ele também executa algumas verificações de consistência interna custosas.

Adicionado na versão 3.3.

Esta função é específica para CPython. O formato de saída exato não é definido aqui e pode mudar.

sys.dllhandle

Número inteiro que especifica o identificador da DLL do Python.

Disponibilidade: Windows.

sys.displayhook(*value*)

Se *value* não for `None`, esta função imprime `repr(value)` em `sys.stdout`, e salva *value* em `builtins._`. Se `repr(value)` não for codificável para `sys.stdout.encoding` com o tratador de erros `sys.stdout.errors` (que provavelmente é `'strict'`), codifica-o para `sys.stdout.encoding` com tratador de erros `'backslashreplace'`.

`sys.displayhook` é chamado no resultado da avaliação de uma expressão inserida em uma sessão interativa do Python. A exibição desses valores pode ser personalizada atribuindo outra função de um argumento a `sys.displayhook`.

Pseudocódigo:

```
def displayhook(value):
    if value is None:
        return
    # Define '_' como None para evitar recursão
    builtins._ = None
    text = repr(value)
    try:
        sys.stdout.write(text)
    except UnicodeEncodeError:
        bytes = text.encode(sys.stdout.encoding, 'backslashre
        if hasattr(sys.stdout, 'buffer'):
            sys.stdout.buffer.write(bytes)
        else:
            text = bytes.decode(sys.stdout.encoding, 'strict'
            sys.stdout.write(text)
    sys.stdout.write("\n")
    builtins._ = value
```

Alterado na versão 3.2: Usa o tratador de erros `'backslashreplace'` ao ser levantada **`UnicodeEncodeError`**.

sys.dont_write_bytecode

Se isso for true, o Python não tentará escrever arquivos `.pyc` na importação de módulos fonte. Este valor é inicialmente definido como `True` ou `False` dependendo da opção de linha de comando **`-B`** e da variável de ambiente **`PYTHONDONTWRITEBYTECODE`**, mas você mesmo pode configurá-lo para controlar geração de arquivo bytecode.

sys._emscripten_info

Um tupla nomeada contendo informações sobre o ambiente na plataforma *wasm32-emscripten*. A tupla nomeada é provisória e pode mudar no futuro.

_emscripten_info.emscripten_version

Versão emscripten como tupla de ints (maior, menor, micro), por exemplo `(3, 1, 8)`.

_emscripten_info.runtime

String em tempo de execução; por exemplo, user-agent do navegador, `'Node.js v14.18.2'`, ou `'UNKNOWN'`.

_emscripten_info.pthreads

`True` se o Python for compilado com suporte a pthreads do Emscripten.

_emscripten_info.shared_memory

`True` se o Python for compilado com suporte a memória compartilhada.

Disponibilidade: Emscripten.

Adicionado na versão 3.11.

sys.pycache_prefix

Se for definido (não `None`), o Python escreverá arquivos de cache de bytecode `.pyc` para (e os lerá de) uma árvore de diretórios paralela com raiz neste diretório, em vez de diretórios `__pycache__` na árvore de código-fonte. Quaisquer diretórios `__pycache__` na árvore de código-fonte serão ignorados e novos arquivos `.pyc` gravados dentro do prefixo pycache. Portanto, se você usar **`compileall`** como uma etapa de pré-compilação, certifique-se de executá-lo com o mesmo prefixo pycache (se houver) que usará em tempo de execução.

Um caminho relativo é interpretado em relação ao diretório de trabalho atual.

Este valor é definido inicialmente com base no valor da opção de linha de comando **`-X`** `pycache_prefix=PATH` ou na variável de ambiente **`PYTHONPYCACHEPREFIX`** (a linha de comando tem precedência). Se nenhum estiver definido, é `None`.

Adicionado na versão 3.8.

sys.excepthook(*type*, *value*, *traceback*)

Esta função imprime um determinado traceback (situação da pilha de execução) e exceção para `sys.stderr`.

Quando uma exceção além da **`SystemExit`** é levantada e não capturada, o interpretador chama `sys.excepthook` com três argumentos, a classe de exceção, a instância de exceção e um objeto traceback. Em uma sessão interativa, isso acontece logo antes de o controle retornar ao prompt; em um programa Python, isso acontece pouco antes de o programa ser encerrado. A manipulação de tais exceções de nível superior pode ser personalizada atribuindo outra função de três argumentos a `sys.excepthook`.

Levanta um evento de auditoria `sys.excepthook` com os argumentos `hook`, `type`, `value`, `traceback` quando ocorre uma exceção não capturada. Se nenhum gancho foi definido, `hook` pode ser `None`. Se algum gancho gerar uma exceção derivada de **`RuntimeError`**, a chamada para o gancho será suprimida. Caso contrário, a exceção de gancho de

auditoria será relatada como não aumentável e `sys.excepthook` será chamado.

Ver também

A função `sys.unraisablehook()` lida com exceções que não podem ser levantadas e a função `threading.excepthook()` trata exceções levantadas por `threading.Thread.run()`.

sys.__breakpointhook__
sys.__displayhook__
sys.__excepthook__
sys.__unraisablehook__

Esses objetos contêm os valores originais de `breakpointhook`, `displayhook`, `excepthook` e `unraisablehook` no início do programa. Eles são salvos para que `breakpointhook`, `displayhook` e `excepthook`, `unraisablehook` possam ser restaurados caso sejam substituídos por objetos quebrados ou alternativos.

Adicionado na versão 3.7: __breakpointhook__

Adicionado na versão 3.8: __unraisablehook__

sys.exception()

Esta função, quando chamada enquanto um tratador de exceção está em execução (como uma cláusula `except` ou `except*`), retorna a instância da exceção que foi capturada por este tratador. Quando os manipuladores de exceção estão aninhados um dentro do outro, apenas a exceção tratada pelo tratador mais interno é acessível.

Se nenhum tratador de exceções estiver em execução, esta função retornará `None`.

Adicionado na versão 3.11.

sys.exc_info()

Essa função retorna a representação de estilo antigo da exceção tratada. Se uma exceção `e` é tratada atualmente (de forma que **`exception()`** retornaria `e`), **`exc_info()`** retorna a tupla `(type(e), e, e.__traceback__)`. Ou seja, uma tupla contendo o tipo da exceção (uma subclasse de **`BaseException`**), a própria exceção e um objeto de traceback que normalmente encapsula a pilha de chamadas no ponto em que a última exceção ocorreu.

Se nenhuma exceção estiver sendo tratada em qualquer lugar da pilha, esta função retornará uma tupla contendo três valores `None`.

Alterado na versão 3.11: Os campos `type` e `traceback` agora são derivados do `value` (a instância da exceção), portanto, quando uma exceção é modificada enquanto está sendo tratada, as alterações são refletidas nos resultados das subsequentes chamadas para **`exc_info()`**.

sys.exec_prefix

Uma string que fornece o prefixo do diretório específico do site onde os arquivos Python dependentes da plataforma são instalados; por padrão, também é `'/usr/local'`. Isso pode ser definido em tempo de compilação com o argumento `--exec-prefix` para o script **configure**. Especificamente, todos os arquivos de configuração (por exemplo, o arquivo de cabeçalho `pyconfig.h`) são instalados no diretório *`exec_prefix`*`/lib/python`*`X.Y`*`/config`, e os módulos da biblioteca compartilhada são instalados em *`exec_prefix`*`/lib/python`*`X.Y`*`/lib-dynload`, onde *X.Y* é o número da versão do Python, por exemplo `3.2`.

Nota

Se um ambiente virtual estiver em vigor, este **`exec_prefix`** vai apontar para o ambiente virtual. O valor para a instalação do Python ainda estará disponível, via **`base_exec_prefix`**. Confira Ambientes virtuais para mais informações.

Alterado na versão 3.14: Ao executar em um ambiente virtual, **`prefix`** e **`exec_prefix`** agora são definidos como o prefixo do ambiente virtual pela inicialização do caminho, em vez de **`site`**. Isso significa que

prefix e **exec_prefix** sempre apontam para o ambiente virtual, mesmo quando **site** está desabilitado (**-S**).

sys.executable

Uma string que fornece o caminho absoluto do binário executável para o interpretador Python, em sistemas onde isso faz sentido. Se o Python não conseguir recuperar o caminho real para seu executável, **sys.executable** será uma string vazia ou `None`.

sys.exit([*arg*])

Levanta uma exceção **SystemExit**, sinalizando a intenção de sair do interpretador.

The optional argument *arg* can be an integer giving the exit status (defaulting to zero), or another type of object. If it is an integer, zero is considered “successful termination” and any nonzero value is considered “abnormal termination” by shells and the like. Most systems require it to be in the range 0–127, and produce undefined results otherwise. Some systems have a convention for assigning specific meanings to specific exit codes, but these are generally underdeveloped; Unix programs generally use 2 for command line syntax errors and 1 for all other kinds of errors. If another type of object is passed, `None` is equivalent to passing zero, and any other object is printed to **stderr** and results in an exit code of 1. In particular, `sys.exit("some error message")` is a quick way to exit a program when an error occurs.

Since **exit()** ultimately “only” raises an exception, it will only exit the process when called from the main thread, and the exception is not intercepted. Cleanup actions specified by **finally** clauses of **try** statements are honored, and it is possible to intercept the exit attempt at an outer level.

Alterado na versão 3.6: Se ocorrer um erro na limpeza após o interpretador Python ter capturado **SystemExit** (como um erro ao liberar dados em buffer nos fluxos padrão), o status de saída é alterado para 120.

sys.flags

A tupla nomeada *flags* expõe o status dos sinalizadores de linha de comando. Os sinalizadores devem ser acessados somente pelo nome e não pelo índice. Os atributos são somente leitura.

flags.debug	`-d`
flags.inspect	`-i`
flags.interactive	`-i`
flags.isolated	`-I`
flags.optimize	`-O` ou `-OO`
flags.dont_write_bytecode	`-B`
flags.no_user_site	`-s`
flags.no_site	`-S`
flags.ignore_environment	`-E`
flags.verbose	`-v`
flags.bytes_warning	`-b`
flags.quiet	`-q`
flags.hash_randomization	`-R`
flags.dev_mode	`-X dev` (Modo de Desenvolvimento do Python)
flags.utf8_mode	`-X utf8`
flags.safe_path	`-P`

flags.int_max_str_digits	`-X int_max_str_digits` (limitação de comprimento de string na conversão para inteiro)
flags.warn_default_encoding	`-X warn_default_encoding`
flags.gil	`-X gil` e `PYTHON_GIL`
flags.thread_inherit_context	`-X thread_inherit_context` e `PYTHON_THREAD_INHERIT_CONTEXT`
flags.context_aware_warnings	`-X context_aware_warnings` e `PYTHON_CONTEXT_AWARE_WARNINGS`

Alterado na versão 3.2: Adicionado o atributo `quiet` para o novo sinalizador `-q`.

Adicionado na versão 3.2.3: O atributo `hash_randomization`.

Alterado na versão 3.3: Removido o atributo obsoleto `division_warning`.

Alterado na versão 3.4: Adicionado o atributo `isolated` para o sinalizador `-I` `isolated`.

Alterado na versão 3.7: Adicionado o atributo `dev_mode` para o novo Modo de Desenvolvimento do Python e o atributo `utf8_mode` para o novo sinalizador `-X` `utf8`.

Alterado na versão 3.10: Adicionado o atributo `warn_default_encoding` para o sinalizador `-X` `warn_default_encoding`.

Alterado na versão 3.11: Adicionado o atributo `safe_path` para a opção `-P`.

Alterado na versão 3.11: Adicionado o atributo `int_max_str_digits`.

Alterado na versão 3.13: Adicionado o atributo `gil`.

Alterado na versão 3.14: Adicionado o atributo `thread_inherit_context`.

Alterado na versão 3.14: Adicionado o atributo `context_aware_warnings`.

sys.float_info

Uma tupla nomeada contendo informações sobre o tipo float, ponto flutuante. Ele contém informações de baixo nível sobre a precisão e a representação interna. Os valores correspondem às várias constantes de ponto flutuante definidas no arquivo de cabeçalho padrão `float.h` para a linguagem de programação 'C'; consulte a seção 5.2.4.2.2 do padrão ISO/IEC C de 1999 [C99], 'Características dos tipos flutuantes', para obter detalhes.

Atributos da tupla nomeada **`float_info`**

atributo	**macro em float.h**	**explicação**
float_info.epsilon	**`DBL_EPSILON`**	diferença entre 1,0 e o menor valor maior que 1,0 que pode ser representado como ponto flutuante. Veja também **`math.ulp.()`**
float_info.dig	**`DBL_DIG`**	O número máximo de dígitos decimais que podem ser fielmente representados em um ponto flutuante; veja abaixo.

atributo	macro em float.h	explicação
float_info.mant_dig	**`DBL_MANT_DIG`**	Precisão do ponto flutuante: o número de dígitos de base `radix` no significando de um ponto flutuante.
float_info.max	**`DBL_MAX`**	O ponto flutuante finito positivo máximo representável.
float_info.max_exp	**`DBL_MAX_EXP`**	O inteiro máximo *e* de tal modo que `radix**(e-1)` é um ponto flutuante finito representável.
float_info.max_10_exp	**`DBL_MAX_10_EXP`**	O inteiro máximo *e* de tal modo que `10**e` é um intervalo de pontos flutuantes finitos representáveis.
float_info.min	**`DBL_MIN`**	O ponto flutuante *normalizado* positivo mínimo representável. Use **`math.ulp(0.0)`** para obter o menor ponto flutuante representável positivo *desnormalizado*.

atributo	macro em float.h	explicação
float_info.min_exp	**`DBL_MIN_EXP`**	O inteiro mínimo *e* de tal modo que `radix**(e-1)` é um ponto flutuante normalizado.
float_info.min_10_exp	**`DBL_MIN_10_EXP`**	O inteiro mínimo *e* de tal modo que `10**e` é um ponto flutuante normalizado.
float_info.radix	**`FLT_RADIX`**	A raiz da representação do expoente.

atributo	macro em float.h	explicação
float_info.rounds	**`FLT_ROUNDS`**	Um inteiro representando o modo de arredondamento para aritmética de ponto flutuante. Isso reflete o valor da macro do sistema **`FLT_ROUNDS`** no momento da inicialização do interpretador: • `-1`: indeterminável • `0`: em direção a zero • `1`: para o mais próximo • `2`: em direção ao infinito positivo • `3`: em direção ao infinito negativo Todos os outros valores para **`FLT_ROUNDS`** caracterizam o comportamento de arredondamento definido pela implementação.

O atributo **`sys.float_info.dig`** precisa de mais explicações. Se `s` for qualquer string representando um número decimal com no máximo **`sys.float_info.dig`** dígitos significativos, então converter `s` para ponto flutuante e vice-versa recuperará uma string representando o mesmo decimal valor:

```
>>> import sys
>>> sys.float_info.dig
15
```

```
>>> s = '3.14159265358979'    # string decimal com 15 dígitos
>>> format(float(s), '.15g')  # converter para float e descor
'3.14159265358979'
```

Mas para strings com mais de `sys.float_info.dig` dígitos significativos, isso nem sempre é verdade:

```
>>> s = '9876543211234567'    # 16 dígitos significativos é r
>>> format(float(s), '.16g')  # a conversão muda o valor
'9876543211234568'
```

sys.float_repr_style

Uma string indicando como a função `repr()` se comporta para floats. Se a string tem valor `'short'` então para um ponto flutuante finito `x`, `repr(x)` visa produzir uma string curta com a propriedade que `float(repr(x)) == x`. Este é o comportamento usual no Python 3.1 e posterior. Caso contrário, `float_repr_style` tem valor `'legacy'` e `repr(x)` se comporta da mesma forma que nas versões do Python anteriores a 3.1.

Adicionado na versão 3.1.

sys.getallocatedblocks()

Retorna a quantidade de blocos de memória alocados atualmente pelo interpretador, independentemente do tamanho. Esta função é útil principalmente ao acompanhar e depurar vazamentos de memória. Devido aos caches internos do interpretador, o resultado pode variar de chamada a chamada; você pode ter que chamar `_clear_internal_caches()` e `gc.collect()` para obter resultados mais previsíveis.

Se uma construção ou implementação do Python não puder computar razoavelmente essas informações, `getallocatedblocks()` poderá retornar 0 em seu lugar.

Adicionado na versão 3.4.

sys.getunicodeinternedsize()

Retorna o número de objetos unicode que foram internalizados.

Adicionado na versão 3.12.

sys.getandroidapilevel()

Retorna o nível de API de tempo de construção do Android como um inteiro. Isso representa a versão mínima do Android em que esta construção do Python pode ser executada. Para obter informações sobre a versão de tempo de execução, consulte **`platform.android_ver()`**.

Disponibilidade: Android.

Adicionado na versão 3.7.

sys.getdefaultencoding()

Retorna `'utf-8'`. Este é o nome da codificação de string padrão, usada em métodos como **`str.encode()`**.

sys.getdlopenflags()

Retorna o valor atual dos sinalizadores que são usados para chamadas **`dlopen()`**. Nomes simbólicos para os valores dos sinalizadores podem ser encontrados no módulo **`os`** (constantes `RTLD_`*`xxx`*, por exemplo **`os.RTLD_LAZY`**).

Disponibilidade: Unix.

sys.getfilesystemencoding()

Obtém o codificação do sistema de arquivos: a codificação usada com o manipulador de erros do sistema de arquivos para converter entre nomes de arquivos Unicode e nomes de arquivos de bytes. O manipulador de erros do sistema de arquivos é retornado de **`getfilesystemencodeerrors()`**.

Para melhor compatibilidade, str deve ser usado para nomes de arquivos em todos os casos, embora a representação de nomes de arquivos como bytes também seja suportada. As funções que aceitam ou retornam

nomes de arquivo devem suportar str ou bytes e converter internamente para a representação preferencial do sistema.

`os.fsencode()` e `os.fsdecode()` devem ser usados para garantir que a codificação correta e o modo de erros sejam usados.

O tratador de erros e codificação do sistema de arquivos são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Alterado na versão 3.2: O resultado de `getfilesystemencoding()` não pode mais ser `None`.

Alterado na versão 3.6: O Windows não tem mais garantia de retornar `'mbcs'`. Veja **PEP 529** [https://peps.python.org/pep-0529/] e `_enablelegacywindowsfsencoding()` para mais informações.

Alterado na versão 3.7: Retorna `'utf-8'` se o Modo UTF-8 do Python estiver ativado.

sys.getfilesystemencodeerrors()

Obtém o manipulador de erros do sistema de arquivos: o manipulador de erros usado com a codificação do sistema de arquivos para converter entre nomes de arquivos Unicode e nomes de arquivos de bytes. A codificação do sistema de arquivos é retornado de `getfilesystemencoding()`.

`os.fsencode()` e `os.fsdecode()` devem ser usados para garantir que a codificação correta e o modo de erros sejam usados.

O tratador de erros e codificação do sistema de arquivos são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Adicionado na versão 3.6.

sys.get_int_max_str_digits()

Retorna o valor atual para a limitação de comprimento de string na conversão para inteiro. Veja também `set_int_max_str_digits()`.

Adicionado na versão 3.11.

sys.get_lazy_imports()

Returns the current lazy imports mode as a string.

- `"normal"`: Only imports explicitly marked with the `lazy` keyword are lazy
- `"all"`: All top-level imports are potentially lazy

See also `set_lazy_imports()` and **PEP 810** [https://peps.python.org/pep-0810/].

Adicionado na versão 3.15.

sys.get_lazy_imports_filter()

Returns the current lazy imports filter function, or `None` if no filter is set.

The filter function is called for every potentially lazy import to determine whether it should actually be lazy. See `set_lazy_imports_filter()` for details on the filter function signature.

Adicionado na versão 3.15.

sys.getrefcount(*object*)

Retorna a contagem de referências do *object*. A contagem retornada é geralmente um valor maior do que o esperado, porque inclui a referência (temporária) passada como argumento para `getrefcount()`.

Observe que o valor retornado pode não refletir realmente quantas referências ao objeto são realmente mantidas. Por exemplo, alguns objetos são imortais e têm uma contagem de referências muito alta, que não reflete o número real de referências. Consequentemente, não confie no valor retornado para ser preciso, exceto um valor de 0 ou 1.

Objetos imortais com uma grande contagem de referências podem ser identificados via `_is_immortal()`.

Alterado na versão 3.12: Objetos imortais têm contagens de referência muito grandes que não correspondem ao número real de referências ao objeto.

sys.getrecursionlimit()

Retorna o valor atual do limite de recursão, a profundidade máxima da pilha do interpretador Python. Esse limite evita que a recursão infinita cause um estouro da pilha C e falhe o Python. Pode ser definido por `setrecursionlimit()`.

sys.getsizeof(*object*[, *default*])

Retorna o tamanho de um objeto em bytes. O objeto pode ser qualquer tipo de objeto. Todos os objetos embutidos retornarão resultados corretos, mas isso não precisa ser verdadeiro para extensões de terceiros, pois é específico da implementação.

Apenas o consumo de memória diretamente atribuído ao objeto é contabilizado, não o consumo de memória dos objetos a que ele se refere.

Se fornecido, *default* será retornado se o objeto não fornecer meios para recuperar o tamanho. Caso contrário, uma exceção `TypeError` será levantada.

`getsizeof()` chama o método `__sizeof__` do objeto e adiciona uma sobrecarga adicional do coletor de lixo se o objeto for gerenciado pelo coletor de lixo.

Consulte receita de tamanho recursivo [https://code.activestate.com/recipes/577504-compute-memory-footprint-of-an-object-and-its-cont/] para obter um exemplo de uso de `getsizeof()` recursivamente para encontrar o tamanho dos contêineres e todo o seu conteúdo.

sys.getswitchinterval()

Retorna o “intervalo de troca de thread” do interpretador em segundos; veja `setswitchinterval()`.

Adicionado na versão 3.2.

sys._getframe([*depth*])

Retorna um objeto quadro da pilha de chamadas. Se o inteiro opcional *depth* for fornecido, retorna o objeto de quadro que muitos chamam abaixo do topo da pilha. Se for mais profundo do que a pilha de chamadas, `ValueError` é levantada. O padrão para *depth* é zero, retornando o quadro no topo da pilha de chamadas.

Levanta um evento de auditoria `sys._getframe` com o argumento `frame`.

Esta função deve ser usada apenas para fins internos e especializados. Não é garantido que exista em todas as implementações do Python.

sys._getframemodulename([*depth*])

Retorna o nome de um módulo da pilha de chamadas. Se o inteiro opcional *depth* for fornecido, retorna o módulo que muitas chamadas abaixo do topo da pilha. Se for mais profundo do que a pilha de chamadas, ou se o módulo não for identificável, `None` é retornado. O padrão para *depth* é zero, retornando o módulo no topo da pilha de chamadas.

Levanta um evento de auditoria `sys._getframemodulename` com o argumento `depth`.

Esta função deve ser usada apenas para fins internos e especializados. Não é garantido que exista em todas as implementações do Python.

Adicionado na versão 3.12.

sys.getobjects(*limit*[, *type*])

Esta função só existe se o CPython foi construído usando a opção de configuração especializada `--with-trace-refs`. Ela é destinada apenas

para depurar problemas de coleta de lixo.

Retorna uma lista de até *limit* objetos Python alocados dinamicamente. Se *type* for fornecido, somente objetos daquele tipo exato (não subtipos) serão incluídos.

Objetos da lista não são seguros para uso. Especificamente, o resultado incluirá objetos de todos os interpretadores que compartilham seu estado de alocador de objetos (ou seja, aqueles criados com `PyInterpreterConfig.use_main_obmalloc` definido como 1 ou usando `Py_NewInterpreter()` e o interpretador principal). Misturar objetos de diferentes iinterpretadores pode levar a travamentos ou outros comportamentos inesperados.

Esta função deve ser usada apenas para fins especializados. Não é garantido que exista em todas as implementações do Python.

Alterado na versão 3.14: O resultado pode incluir objetos de outros interpretadores.

sys.getprofile()

Obtém a função do criador de perfil conforme definido por `setprofile()`.

sys.gettrace()

Obtém a função trace conforme definido por `settrace()`.

A função `gettrace()` destina-se apenas à implementação de depuradores, perfiladores, ferramentas de cobertura e similares. Seu comportamento faz parte da plataforma de implementação, e não da definição da linguagem e, portanto, pode não estar disponível em todas as implementações do Python.

sys.getwindowsversion()

Retorna uma tupla nomeada que descreve a versão do Windows em execução no momento. Os elementos nomeados são *major*, *minor*, *build*, *platform*, *service_pack*, *service_pack_minor*,

service_pack_major, *suite_mask*, *product_type* e *platform_version*. *service_pack* contém uma string, *platform_version* uma tupla de 3 elementos e todos os outros valores são inteiros. Os componentes também podem ser acessados pelo nome, então `sys.getwindowsversion()[0]` é equivalente a `sys.getwindowsversion().major`. Para compatibilidade com versões anteriores, apenas os primeiros 5 elementos são recuperáveis por indexação.

platform será `2` (VER_PLATFORM_WIN32_NT).

product_type pode ser um dos seguintes valores:

Constante	Significado
`1` (VER_NT_WORKSTATION)	O sistema é uma estação de trabalho
`2` (VER_NT_DOMAIN_CONTROLLER)	O sistema é um controlador de domínio.
`3` (VER_NT_SERVER)	O sistema é um servidor, mas não um controlador de domínio.

Esta função atua como um envólucro em volta da função Win32 **`GetVersionEx()`**; consulte a documentação da Microsoft em **`OSVERSIONINFOEX()`** para obter mais informações sobre esses campos.

platform_version retorna a versão principal, a versão secundária e o número de compilação do sistema operacional atual, em vez da versão que está sendo emulada para o processo. Destina-se a ser usado no registro, e não na detecção de recursos.

Nota

platform_version deriva a versão de kernel32.dll que pode ser de uma versão diferente da versão do sistema operacional. Use o módulo

`platform` para obter a versão precisa do sistema operacional.

Disponibilidade: Windows.

Alterado na versão 3.2: Alterado para uma tupla nomeada e adicionado *service_pack_minor*, *service_pack_major*, *suite_mask* e *product_type*.

Alterado na versão 3.6: Adicionado *platform_version*

sys.get_asyncgen_hooks()

Retorna um objeto *asyncgen_hooks*, que é semelhante a um `namedtuple` no formato `(firstiter, finalizer)`, onde *firstiter* e *finalizer* devem ser `None` ou funções que recebem um iterador gerador assíncrono como argumento e são usadas para agendar a finalização de um gerador assíncrono por um laço de eventos.

Adicionado na versão 3.6: Veja **PEP 525** [https://peps.python.org/pep-0525/] para mais detalhes.

Nota

Esta função foi adicionada provisoriamente (veja **PEP 411** [https://peps.python.org/pep-0411/] para detalhes.)

sys.get_coroutine_origin_tracking_depth()

Obtém a profundidade de rastreamento da origem da corrotina atual, conforme definido por `set_coroutine_origin_tracking_depth()`.

Adicionado na versão 3.7.

Nota

Esta função foi adicionada provisoriamente (veja **PEP 411** [https://peps.python.org/pep-0411/] para detalhes.) Use-a apenas para propósitos de depuração.

sys.hash_info

Uma tupla nomeada fornecendo parâmetros da implementação de hash numérico. Para mais detalhes sobre hashing de tipos numéricos, veja Hashing de tipos numéricos.

hash_info.width

A largura em bits usada para fazer hash de valores

hash_info.modulus

O módulo primo P usado para esquema de hash numérico

hash_info.inf

O valor de hash retornado para um infinito positivo

hash_info.nan

(Este atributo não é mais usado)

hash_info.imag

O multiplicador usado para a parte imaginária de um número complexo

hash_info.algorithm

O nome do algoritmo para hash de str, bytes e memoryview

hash_info.hash_bits

O tamanho da saída interna do algoritmo de hash

hash_info.seed_bits

O tamanho da chave semente do algoritmo hash

hash_info.cutoff

Limite para otimização de strings pequenas DJBX33A no intervalo `[1, cutoff)`.

Adicionado na versão 3.2.

Alterado na versão 3.4: Adicionados *algorithm*, *hash_bits*, *seed_bits* e *cutoff*.

sys.hexversion

O número da versão codificado como um único inteiro. Isso é garantido para aumentar com cada versão, incluindo suporte adequado para lançamentos de não produção. Por exemplo, para testar se o interpretador Python é pelo menos a versão 1.5.2, use:

```
if sys.hexversion >= 0x010502F0:
    # usa algum recurso avançado
    ...
else:
    # usa uma implementação alternativa ou adverte o usuário
    ...
```

Isso é chamado `hexversion` já que só parece realmente significativo quando visto como o resultado de passá-lo para a função embutida **`hex()`**. A tupla nomeada **`sys.version_info`** pode ser usada para uma codificação mais amigável das mesmas informações.

Mais detalhes sobre `hexversion` podem ser encontrados em API e Versionamento de ABI.

sys.implementation

Um objeto que contém informações sobre a implementação do interpretador Python em execução no momento. Os atributos a seguir devem existir em todas as implementações do Python.

name é o identificador da implementação, por exemplo `'cpython'`. A string real é definida pela implementação do Python, mas é garantido que seja minúscula.

version é uma tupla nomeada, no mesmo formato que **`sys.version_info`**. Ela representa a versão da implementação Python em *implementation*. Isso tem um significado distinto da versão específica da *linguagem* Python à qual o interpretador em execução no momento está em conformidade, que `sys.version_info` representa. Por exemplo, para PyPy 1.8 `sys.implementation.version` pode ser

`sys.version_info(1, 8, 0, 'final', 0)`, enquanto `sys.version_info` seria `sys.version_info(2, 7, 2, 'final', 0)`. Para CPython, eles são o mesmo valor, pois é a implementação de referência.

hexversion é a versão de implementação em formato hexadecimal, como **`sys.hexversion`**.

cache_tag é a tag usada pelo mecanismo de importação nos nomes de arquivo dos módulos em cache. Por convenção, seria um composto do nome e da versão da implementação, como `'cpython-33'`. No entanto, uma implementação Python pode usar algum outro valor, se apropriado. Se `cache_tag` for definido como `None`, isso indica que o cache do módulo deve ser desabilitado.

supports_isolated_interpreters é um valor booleano que indica se esta implementação oferece suporte a múltiplos interpretadores isolados. É `True` para CPython na maioria das plataformas. Plataformas com este suporte implementam o módulo de baixo nível **`_interpreters`**.

Ver também

PEP 684 [https://peps.python.org/pep-0684/], **PEP 734** [https://peps.python.org/pep-0734/] e **`concurrent.interpreters`**.

`sys.implementation` pode conter atributos adicionais específicos para a implementação do Python. Esses atributos não padrão devem começar com um sublinhado e não são descritos aqui. Independentemente do seu conteúdo, **`sys.implementation`** não mudará durante uma execução do interpretador, nem entre versões de implementação. (No entanto, ele pode mudar entre versões da linguagem Python.) Veja a **PEP 421** [https://peps.python.org/pep-0421/] para mais informações.

Adicionado na versão 3.3.

Alterado na versão 3.14: Adicionado o campo `supports_isolated_interpreters`.

Nota

A adição de novos atributos obrigatórios deve passar pelo processo normal de PEPs. Veja a **PEP 421** [https://peps.python.org/pep-0421/] para mais informações.

sys.int_info

Uma tupla nomeada que contém informações sobre a representação interna de inteiros do Python. Os atributos são somente leitura.

int_info.bits_per_digit

O número de bits mantidos em cada dígito. Os inteiros do Python são armazenados internamente na base `2**int_info.bits_per_digit`.

int_info.sizeof_digit

O tamanho em bytes do tipo C usado para representar um dígito.

int_info.default_max_str_digits

O valor padrão para `sys.get_int_max_str_digits()` quando não estiver explicitamente configurado.

int_info.str_digits_check_threshold

O valor mínimo diferente de zero para `sys.set_int_max_str_digits()`, `PYTHONINTMAXSTRDIGITS` ou `-X int_max_str_digits`.

Adicionado na versão 3.1.

Alterado na versão 3.11: Adicionados `default_max_str_digits` e `str_digits_check_threshold`.

sys.__interactivehook__

Quando esse atributo existe, seu valor é chamado automaticamente (sem argumentos) quando o interpretador é iniciado em modo interativo. Isso é feito após o arquivo `PYTHONSTARTUP` ser lido, para que você possa definir esse gancho lá. O módulo `site` define isso.

Levanta um evento de auditoria `cpython.run_interactivehook` com o objeto gancho como o argumento quando o gancho é chamado na inicialização.

Adicionado na versão 3.4.

sys.intern(*string*)

Insere *string* na tabela de strings “internalizadas” e retorna a string internalizada – que é a própria *string* ou uma cópia. Strings internalizadas são úteis para obter um pouco de desempenho na pesquisa de dicionário – se as chaves em um dicionário foram internalizadas, e a chave de pesquisa para internalizada, já que comparações de chaves (após o hash) podem ser feitas por uma comparação de ponteiros em vez de uma comparação de strings. Normalmente, os nomes usados em programas Python são internalizados automaticamente, e os dicionários usados para armazenar atributos de módulo, classe ou instância têm chaves internalizadas.

Strings internalizadas não são imortais; você deve manter uma referência ao valor de retorno de **`intern()`** para se beneficiar dela.

sys._is_gil_enabled()

Retorna **`True`** se a GIL estiver habiltada e **`False`** caso contrário.

Adicionado na versão 3.13.

Não há garantia de que exista em todas as implementações do Python.

sys.is_finalizing()

Retorna **`True`** se o interpretador Python principal estiver desligando. Retorna **`False`** caso contrário.

Veja também a exceção **`PythonFinalizationError`**.

Adicionado na versão 3.5.

sys._jit

Utilitários para observar compilação just-in-time.

A compilação JIT é um *detalhe de implementação experimental* do CPython. Não há garantia de que `sys._jit` exista ou se comporte da mesma maneira em todas as implementações, versões ou configurações de construção do Python.

Adicionado na versão 3.14.

_jit.is_available()

Retorna `True` se o executável Python atual oferecer suporte à compilação JIT e `False` caso contrário. Isso pode ser controlado compilando o CPython com a opção `--experimental-jit` no Windows e a opção **`--enable-experimental-jit`** em todas as outras plataformas.

_jit.is_enabled()

Retorna `True` se a compilação JIT estiver habilitada para o processo Python atual (implica **`sys._jit.is_available()`**), e `False` caso contrário. Se a compilação JIT estiver disponível, isso pode ser controlado definindo a variável de ambiente **`PYTHON_JIT`** como `0` (desabilitada) ou `1` (habilitada) na inicialização do interpretador.

_jit.is_active()

Retorna `True` se o quadro Python mais alto estiver executando código JIT (implica **`sys._jit.is_enabled()`**), e `False` caso contrário.

Nota

Esta função destina-se a testar e depurar o próprio JIT. Deve ser evitada para qualquer outro propósito.

Nota

Devido à natureza do rastreamento de compiladores JIT, chamadas repetidas a esta função podem gerar resultados surpreendentes. Por

exemplo, ramificar seu valor de retorno provavelmente levará a um comportamento inesperado (se isso fizer com que o código JIT seja inserido ou encerrado):

```
>>> for warmup in range(BIG_NUMBER):
...     # Esta linha está "quente" e, eventualmente, é c
...     if sys._jit.is_active():
...         # Esta linha está "fria" e é executada no in
...         assert sys._jit.is_active()
...
Traceback (most recent call last):
  File "<stdin>", line 5, in <module>
    assert sys._jit.is_active()
           ~~~~~~~~~~~~~~~~~~^^
AssertionError
```

sys.last_exc

Esta variável nem sempre está definida; ela é definida para a instância de exceção quando uma exceção não é tratada e o interpretador imprime uma mensagem de erro e um stack traceback. Seu uso pretendido é permitir que um usuário interativo importe um módulo depurador e se envolva na depuração post-mortem sem ter que reexecutar o comando que causou o erro. (O uso típico é `import pdb; pdb.pm()` para entrar no depurador post-mortem; veja o módulo **pdb** para mais informações.)

Adicionado na versão 3.12.

sys._is_immortal(*op*)

Retorna **True** se o objeto fornecido for imortal, **False** caso contrário.

Nota

Objetos que são imortais (e, portanto, retornam `True` ao serem passados para esta função) não têm garantia de serem imortais em versões futuras, e vice-versa para objetos mortais.

Adicionado na versão 3.14.

Esta função deve ser usada apenas para fins especializados. Não é garantido que exista em todas as implementações do Python.

sys._is_interned(*string*)

Retorna `True` se a string fornecida for “internalizada”, `False` caso contrário.

Adicionado na versão 3.13.

Não há garantia de que exista em todas as implementações do Python.

sys.last_type
sys.last_value
sys.last_traceback

Essas três variáveis estão descontinuadas; use `sys.last_exc` em vez disso. Elas mantêm a representação legada de `sys.last_exc`, conforme retornada de `exc_info()` acima.

sys.lazy_modules

A `set` of fully qualified module name strings that have been lazily imported in the current interpreter but not yet loaded. When a lazily imported module is accessed for the first time, its name is typically removed from this set.

The set may contain some additional strings. It is intended for debugging and introspection, and consumers are expected to verify each entry’s status.

Detalhes da implementação do CPython: Currently, `lazy_modules` may also contain:

- names of *attributes* (non-modules), such as `"pathlib.Path"` after running `lazy from pathlib import Path`, and
- names of items than have already been accessed.

In future versions of Python, these may be removed, and/or additional extras may be added.

See also `set_lazy_imports()` and **PEP 810** [https://peps.python.org/pep-0810/].

Adicionado na versão 3.15.

sys.maxsize

Um inteiro que fornece o valor máximo que uma variável do tipo `Py_ssize_t` pode assumir. Geralmente é `2**31 - 1` em uma plataforma de 32 bits e `2**63 - 1` em uma plataforma de 64 bits.

sys.maxunicode

Um inteiro que fornece o valor do maior ponto de código Unicode, ou seja, `1114111` (`0x10FFFF` em hexadecimal).

Alterado na versão 3.3: Antes da **PEP 393** [https://peps.python.org/pep-0393/], `sys.maxunicode` costumava ser `0xFFFF` ou `0x10FFFF`, dependendo da opção de configuração que especificava se os caracteres Unicode eram armazenados como UCS-2 ou UCS-4.

sys.meta_path

Uma lista de objetos localizadores de metacaminho que têm seus métodos `find_spec()` chamados para que algum deles localize o módulo a ser importado. Por padrão, armazena entradas que implementam a semântica de importação padrão de Python. O método **`find_spec()`** é chamado com pelo menos o nome absoluto do módulo a ser importado. Se o módulo a ser importado estiver contido num pacote, o atributo `__path__` do pacote pai é passado como segundo argumento. O método retorna um spec de módulo, ou `None` se o módulo não for encontrado.

Ver também

`importlib.abc.MetaPathFinder`

A classe base abstrata que define a interface dos objetos localizadores em `meta_path`.

`importlib.machinery.ModuleSpec`

A classe concreta da qual `find_spec()` deve retornar instâncias.

Alterado na versão 3.4: Especificações de módulo foram introduzidas no Python 3.4, pela **PEP 451** [https://peps.python.org/pep-0451/].

Alterado na versão 3.12: Foi removido o alternativa que procurava um método **`find_module()`** se uma entrada `meta_path` não tivesse um método `find_spec()`.

sys.modules

Este é um dicionário que mapeia nomes de módulos para módulos que já foram carregados. Isso pode ser manipulado para forçar o recarregamento de módulos e outros truques. No entanto, substituir o dicionário não vai funcionar necessariamente como esperado e excluir itens essenciais do dicionário pode fazer com que o Python falhe. Se você quiser iterar sobre este dicionário global, sempre use `sys.modules.copy()` ou `tuple(sys.modules)` para evitar exceções, pois seu tamanho pode mudar durante a iteração como um efeito colateral do código ou atividade em outros threads.

sys.orig_argv

A lista dos argumentos originais da linha de comando passados para o executável Python.

Os elementos de `sys.orig_argv` são os argumentos para o interpretador Python, enquanto os elementos de `sys.argv` são os argumentos para o programa do usuário. Os argumentos consumidos pelo próprio interpretador estarão presentes em **`sys.orig_argv`** e ausentes em **`sys.argv`**.

Adicionado na versão 3.10.

sys.path

Uma lista de strings que especifica o caminho de pesquisa para módulos. Inicializado a partir da variável de ambiente `PYTHONPATH`, mais um padrão dependente da instalação.

Por padrão, conforme inicializado na inicialização do programa, um caminho potencialmente inseguro é adicionado ao `sys.path` (*antes* das entradas inseridas como resultado de `PYTHONPATH`):

- A linha de comando `python -m módulo`: adiciona o diretório de trabalho atual
- A linha de comando `python script.py`: prefixa o diretório do script. Se for um link simbólico, resolve links simbólicos.
- As linhas de comando `python -c código` e `python` (REPL): adiciona uma string vazia, que indica o diretório de trabalho atual.

Para não acrescentar esse caminho potencialmente inseguro, use a opção de linha de comando **`-P`** ou a variável de ambiente **`PYTHONSAFEPATH`**.

Um programa é livre para modificar esta lista para seus próprios propósitos. Somente strings devem ser adicionadas a **`sys.path`**; todos os outros tipos de dados são ignorados durante a importação.

Ver também

- Módulo **`site`** Descreve como usar arquivos .pth para estender **`sys.path`**.

sys.path_hooks

Uma lista de chamáveis que recebem um argumento de caminho para tentar criar um localizador para o caminho. Se um localizador puder ser criado, ele deve ser retornado pelo chamável, senão, ele levanta **`ImportError`**.

Originalmente especificado na **PEP 302** [https://peps.python.org/pep-0302/].

sys.path_importer_cache

Um dicionário atuando como um cache para objetos do localizador. As chaves são caminhos que foram passados para **`sys.path_hooks`** e os valores são os localizadores que são encontrados. Se um caminho for um caminho de sistema de arquivo válido, mas nenhum localizador for encontrado em **`sys.path_hooks`**, então `None` é armazenado.

Originalmente especificado na **PEP 302** [https://peps.python.org/pep-0302/].

sys.platform

Uma string contendo um identificador da plataforma. Valores conhecidos são:

Sistema	Valor de `platform`
AIX	`'aix'`
Android	`'android'`
Emscripten	`'emscripten'`
FreeBSD	`'freebsd'`
iOS	`'ios'`
Linux	`'linux'`
macOS	`'darwin'`
Windows	`'win32'`
Windows/Cygwin	`'cygwin'`
WASI	`'wasi'`

Em sistemas Unix não listados na tabela, o valor é o nome do sistema operacional em letras minúsculas, conforme retornado por `uname -s`, com a primeira parte da versão, conforme retornado por `uname -r` anexada, por exemplo, `'sunos5'`, *no momento em que o Python foi construído*. A menos que você queira testar uma versão específica do sistema, é recomendável usar o seguinte idioma:

```
if sys.platform.startswith('sunos'):
    # código específico do SunOS aqui...
```

Alterado na versão 3.3: No Linux, `sys.platform` não contém mais a versão principal. É sempre `'linux'`, em vez de `'linux2'` ou `'linux3'`.

Alterado na versão 3.8: No AIX, `sys.platform` não contém mais a versão principal. É sempre `'aix'`, em vez de `'aix5'` ou `'aix7'`.

Alterado na versão 3.13: No Android, `sys.platform` agora retorna `'android'` em vez de `'linux'`.

Alterado na versão 3.14: No FreeBSD, `sys.platform` não contém mais a versão principal. É sempre `'freebsd'`, em vez de `'freebsd13'` ou `'freebsd14'`.

Ver também

`os.name` tem uma granularidade mais substancial. `os.uname()` fornece informações de versão dependentes do sistema.

O módulo `platform` fornece verificações detalhadas sobre a identificação do sistema.

sys.platlibdir

Nome do diretório da biblioteca específica da plataforma. É usado para construir o caminho da biblioteca padrão e os caminhos dos módulos de extensão instalados.

É igual a `"lib"` na maioria das plataformas. No Fedora e SuSE, é igual a `"lib64"` em plataformas de 64 bits, o que fornece os seguintes caminhos `sys.path` (onde `X.Y` é a versão `major.minor` do Python):

- `/usr/lib64/pythonX.Y/`: Biblioteca padrão (como `os.py` do módulo `os`)
- `/usr/lib64/pythonX.Y/lib-dynload/`: Módulos de extensão C da biblioteca padrão (como o módulo `errno`, o nome exato do arquivo é específico da plataforma)
- `/usr/lib/pythonX.Y/site-packages/` (sempre usa `lib`, não `sys.platlibdir`): Módulos de terceiros
- `/usr/lib64/pythonX.Y/site-packages/`: Módulos de extensão C de pacotes de terceiros

Adicionado na versão 3.9.

sys.prefix

Uma string que fornece o prefixo do diretório específico do site onde os arquivos Python independentes de plataforma são instalados; no Unix, o padrão é `/usr/local`. Isso pode ser definido no momento da construção com o argumento **`--prefix`** para o script **configure**. Veja Caminhos de instalação para caminhos derivados.

Nota

Se um ambiente virtual estiver em vigor, este **`prefix`** vai apontar para o ambiente virtual. O valor para a instalação do Python ainda estará disponível, via **`base_prefix`**. Confira Ambientes virtuais para mais informações.

Alterado na versão 3.14: Ao executar em um ambiente virtual, **`prefix`** e **`exec_prefix`** agora são definidos como o prefixo do ambiente virtual pela inicialização do caminho, em vez de **`site`**. Isso significa que **`prefix`** e **`exec_prefix`** sempre apontam para o ambiente virtual, mesmo quando **`site`** está desabilitado (**`-S`**).

sys.ps1
sys.ps2

Strings especificando o prompt primário e secundário do interpretador. Elas são definidas somente se o interpretador estiver no modo interativo. Seus valores iniciais neste caso são `'>>> '` e `'... '`. Se um objeto não string for atribuído a qualquer variável, seu **`str()`** é reavaliado cada vez que o interpretador se prepara para ler um novo comando interativo; isso pode ser usado para implementar um prompt dinâmico.

sys.setdlopenflags(*n*)

Define os sinalizadores usados pelo interpretador para chamadas **`dlopen()`**, como quando o interpretador carrega módulos de extensão. Entre outras coisas, isso habilitará uma resolução preguiçosa de símbolos ao importar um módulo, se chamado como `sys.setdlopenflags(0)`. Para compartilhar símbolos entre módulos de extensão, chame como `sys.setdlopenflags(os.RTLD_GLOBAL)`. Nomes

simbólicos para os valores de sinalizadores podem ser encontrados no módulo **os** (constantes `RTLD_`*xxx*, por exemplo, **os.RTLD_LAZY**).

Disponibilidade: Unix.

sys.set_int_max_str_digits(*maxdigits*)

Define a limitação de comprimento de string na conversão para inteiro usada por este interpretador. Veja também **get_int_max_str_digits()**.

Adicionado na versão 3.11.

sys.set_lazy_imports(*mode*)

Sets the global lazy imports mode. The *mode* parameter must be one of the following strings:

- `"normal"`: Only imports explicitly marked with the `lazy` keyword are lazy
- `"all"`: All top-level imports become potentially lazy

This function is intended for advanced users who need to control lazy imports across their entire application. Library developers should generally not use this function as it affects the runtime execution of applications.

In addition to the mode, lazy imports can be controlled via the filter provided by **set_lazy_imports_filter()**.

See also **get_lazy_imports()** and **PEP 810** [https://peps.python.org/pep-0810/].

Adicionado na versão 3.15.

sys.set_lazy_imports_filter(*filter*)

Sets the lazy imports filter callback. The *filter* parameter must be a callable or `None` to clear the filter.

The filter function is called for every potentially lazy import to determine whether it should actually be lazy. It should have the following signature:

```
def filter(importing_module: str, imported_module: str,
           fromlist: tuple[str, ...] | None) -> bool
```

The function is called with three positional arguments:

- *importing_module* is the name of the module doing the import
- *imported_module* is the resolved name of the module being imported (for example, `lazy from .spam import eggs` passes `package.spam`)
- *fromlist* is the tuple of names being imported (for `from ... import` statements), or `None` for regular imports

The filter should return `True` to allow the import to be lazy, or `False` to force an eager import.

This is an advanced feature intended for specialized users who need fine-grained control over lazy import behavior.

See also **`get_lazy_imports_filter()`** and **PEP 810** [https://peps.python.org/pep-0810/].

Adicionado na versão 3.15.

sys.setprofile(*profilefunc*)

Define a função de perfil do sistema, que permite implementar um perfilador de código-fonte Python em Python. Veja o capítulo profile — Pure Python profiler para mais informações sobre o perfilador Python. A função de perfil do sistema é chamada de forma semelhante à função de rastreamento do sistema (veja **`settrace()`**), mas é chamada com eventos diferentes, por exemplo, não é chamada para cada linha de código executada (apenas na chamada e retorno, mas o evento de retorno é relatado mesmo quando uma exceção foi definida). A função é específica do thread, mas não há como o perfilador saber sobre trocas de contexto entre threads, então não faz sentido usá-la na presença de vários threads. Além disso, seu valor de retorno não é usado, então ele pode simplesmente retornar `None`. Um erro na função de perfil fará com que sua própria definição seja removida.

Nota

O mesmo mecanismo de rastreamento é usado para **`setprofile()`** como **`settrace()`**. Para rastrear chamadas com **`setprofile()`** dentro de uma função de rastreamento (por exemplo, em um ponto de interrupção do depurador), consulte **`call_tracing()`**.

As funções de perfil devem ter três argumentos: *frame*, *event* e *arg*. *frame* é o quadro de pilha atual. *event* é uma string: `'call'`, `'return'`, `'c_call'`, `'c_return'` ou `'c_exception'`. *arg* depende do tipo de evento.

Os eventos têm o seguinte significado:

`'call'`
: Uma função é chamada (ou algum outro bloco de código é inserido). A função de perfil é chamada; *arg* é `None`.

`'return'`
: Uma função (ou outro bloco de código) está prestes a retornar. A função de perfilação é chamada; *arg* é o valor que será retornado, ou `None` se o evento for causado por uma exceção sendo levantada.

`'c_call'`
: Uma função C está prestes a ser chamada. Pode ser uma função de extensão ou uma embutida. *arg* é o objeto da função C.

`'c_return'`
: Uma função C foi retornada. *arg* é o objeto da função C.

`'c_exception'`
: Uma função C levantou uma exceção. *arg* é o objeto da função C.

Levanta um evento de auditoria `sys.setprofile` sem argumentos.

sys.setrecursionlimit(*limit*)

Define a profundidade máxima da pilha do interpretador Python para *limit*. Esse limite impede que a recursão infinita cause um estouro da

pilha C e trave o Python.

O limite mais alto possível depende da plataforma. Um usuário pode precisar definir o limite mais alto quando tiver um programa que exija recursão profunda e uma plataforma que dê suporte a um limite mais alto. Isso deve ser feito com cuidado, porque um limite muito alto pode levar a uma falha.

Se o novo limite for muito baixo na profundidade de recursão atual, uma exceção `RecursionError` será levantada.

Alterado na versão 3.5.1: Uma exceção `RecursionError` agora é levantada se o novo limite for muito baixo na profundidade de recursão atual.

sys.setswitchinterval(*interval*)

Define o intervalo de troca de thread do interpretador (em segundos). Este valor de ponto flutuante determina a duração ideal das "fatias de tempo" alocadas para threads Python em execução simultânea. Observe que o valor real pode ser maior, especialmente se funções ou métodos internos de execução longa forem usados. Além disso, qual thread se torna agendada no final do intervalo é uma decisão do sistema operacional. O interpretador não tem seu próprio escalonador.

Adicionado na versão 3.2.

sys.settrace(*tracefunc*)

Define a função trace do sistema, que permite que você implemente um depurador de código-fonte Python em Python. A função é específica de thread; para um depurador suportar múltiplas threads, ele deve registrar uma função trace usando `settrace()` para cada thread que está sendo depurada ou usar `threading.settrace()`.

As funções de rastreamento devem ter três argumentos: *frame*, *event* e *arg*. *frame* é o quadro de pilha atual. *event* é uma string: `'call'`, `'line'`, `'return'`, `'exception'` ou `'opcode'`. *arg* depende do tipo de evento.

A função de rastreamento é invocada (com *event* definido como `'call'`) sempre que um novo escopo local é inserido; ela deve retornar uma referência a uma função de rastreamento local a ser usada para o novo escopo, ou `None` se o escopo não deve ser rastreado.

A função de rastreamento local deve retornar uma referência a si mesma ou a outra função que seria então usada como a função de rastreamento local para o escopo.

Se ocorrer algum erro na função de rastreamento, ela será desativada, assim como `settrace(None)` é chamado.

Nota

O rastreamento é desabilitado ao chamar a função de rastreamento (por exemplo, uma função definida por **`settrace()`**). Para rastreamento recursivo, veja **`call_tracing()`**.

Os eventos têm o seguinte significado:

`'call'`
: Uma função é chamada (ou algum outro bloco de código é inserido). A função de rastreamento global é chamada; *arg* é `None`; o valor de retorno especifica a função de rastreamento local.

`'line'`
: The interpreter is about to execute a new line of code or re-execute the condition of a loop. The local trace function is called; *arg* is `None`; the return value specifies the new local trace function. See InternalDocs/code_objects.md [https://github.com/python/cpython/tree/3.15/InternalDocs/code_objects.md] for a detailed explanation of how this works. Per-line events may be disabled for a frame by setting **`f_trace_lines`** to **`False`** on that frame.

`'return'`
: Uma função (ou outro bloco de código) está prestes a retornar. A função de rastreamento local é chamada; *arg* é o valor que será

retornado, ou `None` se o evento for causado por uma exceção sendo levantada. O valor de retorno da função de rastreamento é ignorado.

`'exception'`
Ocorreu uma exceção. A função de rastreamento local é chamada; *arg* é uma tupla `(exception, value, traceback)`; o valor de retorno especifica a nova função de rastreamento local.

`'opcode'`
O interpretador está prestes a executar um novo opcode (veja **`dis`** para detalhes do opcode). A função de rastreamento local é chamada; *arg* é `None`; o valor de retorno especifica a nova função de rastreamento local. Eventos por opcode não são emitidos por padrão: eles devem ser explicitamente solicitados definindo **`f_trace_opcodes`** como **`True`** no quadro.

Observe que, à medida que uma exceção é propagada pela cadeia de chamadores, um evento `'exception'` é gerado em cada nível.

Para um uso mais refinado, é possível definir uma função de rastreamento atribuindo `frame.f_trace = tracefunc` explicitamente, em vez de depender de ser definida indiretamente por meio do valor de retorno de uma função de rastreamento já instalada. Isso também é necessário para ativar a função de rastreamento no quadro atual, o que **`settrace()`** não faz. Observe que, para que isso funcione, uma função de rastreamento global deve ter sido instalada com **`settrace()`** para habilitar o maquinário de rastreamento em tempo de execução, mas não precisa ser a mesma função de rastreamento (por exemplo, pode ser uma função de rastreamento de baixa sobrecarga que simplesmente retorna `None` para se desabilitar imediatamente em cada quadro).

Para mais informações sobre objetos de código e quadro, consulte A hierarquia de tipos padrão.

Levanta um evento de auditoria `sys.settrace` sem argumentos.

A função **`settrace()`** destina-se apenas à implementação de depuradores, perfiladores, ferramentas de cobertura e similares. Seu

comportamento faz parte da plataforma de implementação, e não da definição da linguagem e, portanto, pode não estar disponível em todas as implementações do Python.

Alterado na versão 3.7: Tipo de evento `'opcode'` adicionado; atributos **`f_trace_lines`** e **`f_trace_opcodes`** adicionados a quadros

sys.set_asyncgen_hooks(*[firstiter] [, finalizer]*)

Aceita dois argumentos nomeados opcionais que são chamáveis que aceitam um iterador gerador assíncrono como argumento. O chamável *firstiter* será chamado quando um gerador assíncrono for iterado pela primeira vez. O *finalizer* será chamado quando um gerador assíncrono estiver prestes a ser coletado como lixo.

Levanta um evento de auditoria `sys.set_asyncgen_hooks_firstiter` sem argumentos.

Levanta um evento de auditoria `sys.set_asyncgen_hooks_finalizer` sem argumentos.

Dois eventos de auditoria são gerados porque a API subjacente consiste em duas chamadas, cada uma das quais deve levantar seu próprio evento.

Adicionado na versão 3.6: Veja a **PEP 525** [https://peps.python.org/pep-0525/] para mais detalhes, e para um exemplo de referência de um método *finalizer* veja a implementação de `asyncio.Loop.shutdown_asyncgens` em Lib/asyncio/base_events.py [https://github.com/python/cpython/tree/3.15/Lib/asyncio/base_events.py]

Nota

Esta função foi adicionada provisoriamente (veja **PEP 411** [https://peps.python.org/pep-0411/] para detalhes.)

sys.set_coroutine_origin_tracking_depth(*depth*)

Permite habilitar ou desabilitar o rastreamento de origem de corrotina. Quando habilitado, o atributo `cr_origin` em objetos de corrotina conterá uma tupla de tuplas (nome do arquivo, número da linha, nome da função) descrevendo o traceback onde o objeto corrotina foi criado, com a chamada mais recente primeiro. Quando desabilitado, `cr_origin` será `None`.

Para habilitar, passe um valor *depth* maior que zero; isso define o número de quadros cujas informações serão capturadas. Para desabilitar, defina *depth* para zero.

Esta configuração é específica do thread.

Adicionado na versão 3.7.

Nota

Esta função foi adicionada provisoriamente (veja **PEP 411** [https://peps.python.org/pep-0411/] para detalhes.) Use-a apenas para propósitos de depuração.

sys.activate_stack_trampoline(*backend*, /)

Ativa o trampolim do perfilador de pilha *backend*. O único backend suportado é `"perf"`.

Os trampolins empilháveis não podem ser ativados se o JIT estiver ativo.

Disponibilidade: Linux.

Adicionado na versão 3.12.

Ver também

- Python support for the perf map compatible profilers
- https://perf.wiki.kernel.org

sys.deactivate_stack_trampoline()

Desativa o backend trampolim do perfilador de pilha atual.

Se nenhum perfilador de pilha estiver ativado, esta função não terá efeito.

Disponibilidade: Linux.

Adicionado na versão 3.12.

sys.is_stack_trampoline_active()

Retorna `True` se um trampolim de perfilador de pilha estiver ativo.

Disponibilidade: Linux.

Adicionado na versão 3.12.

sys.remote_exec(*pid*, *script*)

Executa *script*, um arquivo contendo código Python no processo remoto com o *pid* fornecido.

Esta função retorna imediatamente, e o código será executado pela thread principal do processo de destino na próxima oportunidade disponível, de forma semelhante à manipulação de sinais. Não há interface para determinar quando o código foi executado. O chamador é responsável por garantir que o arquivo ainda exista sempre que o processo remoto tentar lê-lo e que ele não tenha sido sobrescrito.

O processo remoto deve estar executando um interpretador CPython da mesma versão principal e secundária que o processo local. Se o interpretador local ou remoto for de pré-lançamento (alfa, beta ou candidato a lançamento), os interpretadores local e remoto devem ser exatamente da mesma versão.

See Protocolo de anexação de depuração remota for more information about the remote debugging mechanism.

Quando o código é executado no processo remoto, um evento de auditoria `sys.remote_exec` é gerado com o *pid* e o caminho para o arquivo de script. Este evento é levantado no processo que chamou **`sys.remote_exec()`**.

Quando o script é executado no processo remoto, um evento de auditoria `cpython.remote_debugger_script` é levantado com o caminho no processo remoto. Este evento é levantado no processo remoto, não naquele que chamou **`sys.remote_exec()`**.

Disponibilidade: Unix, Windows.

Adicionado na versão 3.14: See **PEP 768** [https://peps.python.org/pep-0768/] for more details.

sys._enablelegacywindowsfsencoding()

Altera tratador de erros e codificação do sistema de arquivos para 'replace' e 'mbcs' respectivamente, para consistência com versões do Python anteriores à 3.6.

Isso é equivalente a definir a variável de ambiente **`PYTHONLEGACYWINDOWSFSENCODING`** antes de iniciar o Python.

Veja também **`sys.getfilesystemencoding()`** e **`sys.getfilesystemencodeerrors()`**.

Disponibilidade: Windows.

Nota

Alterar a codificação do sistema de arquivos após a inicialização do Python é arriscado porque o fsencoding antigo ou os caminhos codificados pelo fsencoding antigo podem estar armazenados em cache em algum lugar. Use **`PYTHONLEGACYWINDOWSFSENCODING`** em vez disso.

Adicionado na versão 3.6: Veja **PEP 529** [https://peps.python.org/pep-0529/] para mais detalhes.

Descontinuado desde a versão 3.13, será removido na versão 3.16: Use **PYTHONLEGACYWINDOWSFSENCODING**.

sys.stdin
sys.stdout
sys.stderr

Objetos arquivo usados pelo interpretador para entrada padrão, saída e erros:

- `stdin` é usado para todas as entradas interativas (incluindo chamadas para **input()**);
- `stdout` é usado para a saída das instruções **print()** e expressões e para os prompts de **input()**;
- Os prompts do próprio interpretador e suas mensagens de erro vão para `stderr`.

Esses fluxos são arquivos texto regulares como aqueles retornados pela função **open()**. Seus parâmetros são escolhidos da seguinte forma:

- The encoding and error handling are initialized from **PyConfig.stdio_encoding** and **PyConfig.stdio_errors**.

  No Windows, o UTF-8 é usado para o dispositivo de console. Dispositivos não-caracteres, como arquivos de disco e pipes, usam a codificação da localidade do sistema (ou seja, a página de código ANSI). Dispositivos de caracteres não-console, como NUL (ou seja, onde `isatty()` retorna `True`) usam o valor das páginas de código de entrada e saída do console na inicialização, respectivamente para stdin e stdout/stderr. Isso assume como padrão a codificação da localidade do sistema se o processo não for inicialmente anexado a um console.

  O comportamento especial do console pode ser substituído definindo a variável de ambiente PYTHONLEGACYWINDOWSSTDIO antes de iniciar o Python. Nesse caso, as páginas de código do console são usadas como para qualquer outro dispositivo de caractere.

Em todas as plataformas, você pode substituir a codificação de caracteres definindo a variável de ambiente **PYTHONIOENCODING** antes de iniciar o Python ou usando a nova opção de linha de comando **-X** `utf8` e a variável de ambiente **PYTHONUTF8**. No entanto, para o console do Windows, isso só se aplica quando **PYTHONLEGACYWINDOWSSTDIO** também está definido.

- Quando interativo, o fluxo `stdout` é armazenado em buffer de linha. Caso contrário, ele é armazenado em buffer de bloco como arquivos texto comuns. O fluxo `stderr` é armazenado em buffer de linha em ambos os casos. Você pode tornar ambos os fluxos sem buffer passando a opção de linha de comando **-u** ou definindo a variável de ambiente **PYTHONUNBUFFERED**.

Alterado na versão 3.9: `stderr` não interativo agora é armazenado em buffer de linha em vez de totalmente armazenado em buffer.

Nota

Para escrever ou ler dados binários de/para os fluxos padrão, use o objeto binário subjacente **buffer**. Por exemplo, para escrever bytes em **stdout**, use `sys.stdout.buffer.write(b'abc')`.

Entretanto, se você estiver escrevendo uma biblioteca (e não controlar em qual contexto seu código será executado), esteja ciente de que os fluxos padrão podem ser substituídos por objetos arquivo ou similares, como **io.StringIO**, que não oferecem suporte ao atributo **buffer**.

sys.__stdin__
sys.__stdout__
sys.__stderr__

Esses objetos contêm os valores originais de `stdin`, `stderr` e `stdout` no início do programa. Eles são usados durante a finalização e podem ser úteis para imprimir no fluxo padrão real, não importa se o objeto `sys.std*` foi redirecionado.

Ele também pode ser usado para restaurar os arquivos reais para objetos arquivo de trabalho conhecidos, caso tenham sido substituídos por um objeto quebrado. No entanto, a maneira preferida de fazer isso é salvar explicitamente o fluxo anterior antes de substituí-lo e restaurar o objeto salvo.

Nota

Sob algumas condições, `stdin`, `stdout` e `stderr`, bem como os valores originais de `__stdin__`, `__stdout__` e `__stderr__` podem ser `None`. Geralmente é o caso de aplicativos GUI do Windows que não estão conectados a um console e aplicativos Python iniciados com **pythonw**.

sys.stdlib_module_names

Um frozenset de strings contendo os nomes dos módulos da biblioteca padrão.

É o mesmo em todas as plataformas. Módulos que não estão disponíveis em algumas plataformas e módulos desabilitados na construção do Python também são listados. Todos os tipos de módulos são listados: Python puro, módulos embutidos, congelados e de extensão. Módulos de teste são excluídos.

Para pacotes, somente o pacote principal é listado: subpacotes e submódulos não são listados. Por exemplo, o pacote `email` é listado, mas o subpacote `email.mime` e o submódulo `email.message` não são listados.

Veja também a lista de **`sys.builtin_module_names`**.

Adicionado na versão 3.10.

sys.thread_info

Uma tupla nomeada contendo informações sobre a implementação de threads.

thread_info.name

O nome da implementação da thread:

- `"nt"`: Threads do Windows
- `"pthread"`: Threads do POSIX
- `"pthread-stubs"`: stubs de threads POSIX (em plataformas WebAssembly que não implementam threads).
- `"solaris"`: Threads do Solaris

thread_info.lock

O nome da implementação de trava.

- `"semaphore"`: a lock uses a semaphore (Python 3.14 and older)
- `"mutex+cond"`: a lock uses a mutex and a condition variable (Python 3.14 and older)
- `"pymutex"`: a lock uses the **`PyMutex`** implementation (Python 3.15 and newer)
- `None` se essa informação for desconhecida

thread_info.version

O nome e a versão da biblioteca de threads. É uma string, ou `None` se essa informação for desconhecida.

Adicionado na versão 3.3.

sys.tracebacklimit

Quando essa variável é definida como um valor inteiro, ela determina o número máximo de níveis de informações de traceback impressas quando ocorre uma exceção não tratada. O padrão é `1000`. Quando definida como `0` ou menos, todas as informações de traceback são suprimidas e apenas o tipo e o valor da exceção são impressos.

sys.unraisablehook(*unraisable*, /)

Manipula uma exceção não levantada

Chamada quando uma exceção ocorreu, mas não há como o Python manipulá-la. Por exemplo, quando um destrutor levanta uma exceção ou durante a coleta de lixo (**`gc.collect()`**).

O argumento *unraisable* tem os seguintes atributos:

- **`exc_type`**: Tipo de exceção.
- **`exc_value`**: Valor da exceção, pode ser `None`.
- **`exc_traceback`**: Traceback da exceção, pode ser `None`.
- **`err_msg`**: Mensagem de erro, pode ser `None`.
- **`object`**: O objeto que causa a exceção pode ser `None`.

The default hook formats **`err_msg`** and **`object`** as: `f'{err_msg}: {object!r}'`; use "Exception ignored in" error message if **`err_msg`** is `None`. Similar to the **`traceback`** module, this adds color to exceptions by default. This can be disabled using environment variables.

`sys.unraisablehook()` pode ser substituída para controlar como exceções não levantáveis são manipuladas.

Alterado na versão 3.15: Exceptions are now printed with colorful text.

Ver também

`excepthook()`, que manipula exceções não capturadas.

Aviso

Armazena **`exc_value`** usando um gancho personalizado pode criar um ciclo de referência. Ele deve ser limpo explicitamente para quebrar o ciclo de referência quando a exceção não for mais necessária.

Armazenar **`object`** usando um gancho personalizado pode ressuscitá-lo se ele for definido como um objeto que está sendo finalizado. Evite armazenar **`object`** após o gancho personalizado ser concluído para evitar ressuscitar objetos.

Levanta um evento de auditoria `sys.unraisablehook` com argumentos *hook* e *unraisable* quando ocorrer uma exceção que não puder ser manipulada. O objeto *unraisable* é o mesmo que será passado para o gancho. Se nenhum gancho tiver sido definido, *hook* pode ser `None`.

Adicionado na versão 3.8.

sys.version

Uma string contendo o número da versão do interpretador Python mais informações adicionais sobre o número da construção e o compilador usado. Esta string é exibida quando o interpretador interativo é iniciado. Não extraia informações de versão dela, em vez disso, use `version_info` e as funções fornecidas pelo módulo `platform`.

sys.api_version

A versão da API C, equivalente à macro C `PYTHON_API_VERSION`. Definido para retrocompatibilidade.

Atualmente, essa constante não é atualizada nas novas versões de Python e não é útil para controle de versão. Isso pode mudar no futuro.

sys.version_info

Uma tupla contendo os cinco componentes do número da versão: *major*, *minor*, *micro*, *releaselevel* e *serial*. Todos os valores, exceto *releaselevel*, são inteiros; o nível de versão é `'alpha'`, `'beta'`, `'candidate'` ou `'final'`. O valor `version_info` correspondente à versão 2.0 do Python é `(2, 0, 0, 'final', 0)`. Os componentes também podem ser acessados por nome, então `sys.version_info[0]` é equivalente a `sys.version_info.major` e assim por diante.

Alterado na versão 3.1: Adiciona componentes nomeados como atributos.

sys.warnoptions

Este é um detalhe de implementação do framework de avisos; não modifique este valor. Consulte o módulo `warnings` para obter mais informações sobre o framework de avisos.

sys.winver

The version number used to form registry keys on Windows platforms. This is stored as string resource 1000 in the Python DLL. The value is normally the major and minor versions of the running Python

interpreter. It is provided in the **sys** module for informational purposes; modifying this value has no effect on the registry keys used by Python.

Disponibilidade: Windows.

sys.monitoring

Espaço de nomes contendo funções e constantes para função de retorno de registro e controle de eventos de monitoramento. Veja **sys.monitoring** para detalhes.

sys._xoptions

Um dicionário dos vários sinalizadores específicos de implementação passados pela opção de linha de comando **-X**. Os nomes de opção são mapeados para seus valores, se fornecidos explicitamente, ou para **True**. Exemplo:

```
$ ./python -Xa=b -Xc
Python 3.2a3+ (py3k, Oct 16 2010, 20:14:50)
[GCC 4.4.3] on linux2
Type "help", "copyright", "credits" or "license" for more in
>>> import sys
>>> sys._xoptions
{'a': 'b', 'c': True}
```

Esta é uma maneira específica do CPython de acessar opções passadas por **-X**. Outras implementações podem exportá-las por outros meios, ou não exportá-las.

Adicionado na versão 3.2.

Citações

[C99]

ISO/IEC 9899:1999. “Programming languages – C.” Um rascunho público desta norma está disponível em https://www.open-std.org/jtc1/sc22/wg14/www/docs/n1256.pdf.

`sys.monitoring` — Monitoramento de eventos de execução

Adicionado na versão 3.12.

Nota

`sys.monitoring` é um espaço de nomes dentro do módulo **`sys`**, não um módulo independente, e `import sys.monitoring` falharia com **`ModuleNotFoundError`**. Em vez disso, basta usar `import sys` e então usar `sys.monitoring`.

Esse espaço de nomes fornece acesso às funções e constantes necessárias para ativar e controlar o monitoramento de eventos.

À medida que os programas são executados, ocorrem eventos que podem ser de interesse para as ferramentas que monitoram a execução. O espaço de nomes **`sys.monitoring`** fornece meios para receber retornos de chamada quando ocorrem eventos de interesse.

A API de monitoramento consiste em três componentes:

- Identificadores de ferramenta
- Eventos
- Funções de retorno de chamadas

Identificadores de ferramenta

Um identificador de ferramenta é um número inteiro e o nome associado. Os identificadores de ferramenta são usados para evitar que as ferramentas interfiram umas nas outras e para permitir que várias ferramentas operem ao mesmo tempo. Atualmente, as ferramentas são totalmente independentes e

não podem ser usadas para monitorar umas às outras. Essa restrição poderá ser suspensa no futuro.

Antes de registrar ou ativar eventos, uma ferramenta deve escolher um identificador. Os identificadores são números inteiros no intervalo de 0 a 5, inclusive.

Registro e uso de ferramentas

sys.monitoring.use_tool_id(*tool_id: int, name: str, /*) → None

Deve ser chamado antes que *tool_id* possa ser usado. *tool_id* deve estar no intervalo de 0 a 5, inclusive. Levanta um `ValueError` se *tool_id* estiver em uso.

sys.monitoring.clear_tool_id(*tool_id: int, /*) → None

Cancela o registro de todos os eventos e funções de retorno de chamada associados a *tool_id*.

Adicionado na versão 3.14.

sys.monitoring.free_tool_id(*tool_id: int, /*) → None

Deve ser chamado quando uma ferramenta não precisar mais de *tool_id*. Chamará `clear_tool_id()` antes de liberar *tool_id*.

Alterado na versão 3.14: Agora chama `clear_tool_id()` antes de liberar *tool_id*. Anteriormente, não desabilitava eventos globais ou locais associados a *tool_id*, nem cancelava o registro de quaisquer funções de callback.

sys.monitoring.get_tool(*tool_id: int, /*) → str | None

Retorna o nome da ferramenta se *tool_id* estiver em uso; caso contrário, retorna `None`. *tool_id* deve estar no intervalo de 0 a 5, inclusive.

Todas as IDs são tratadas da mesma forma pela VM com relação aos eventos, mas as seguintes IDs são predefinidas para facilitar a cooperação entre as ferramentas:

```
sys.monitoring.DEBUGGER_ID = 0
sys.monitoring.COVERAGE_ID = 1
sys.monitoring.PROFILER_ID = 2
sys.monitoring.OPTIMIZER_ID = 5
```

Eventos

Os seguintes eventos são suportados:

sys.monitoring.events.BRANCH_LEFT
: Uma ramificação condicional vai para a esquerda.

  Cabe à ferramenta determinar como apresentar os ramos “esquerdo” e “direito”. Não há garantia de qual ramo é “esquerdo” e qual é “direito”, exceto que será consistente durante toda a duração do programa.

sys.monitoring.events.BRANCH_RIGHT
: Uma ramificação condicional vai para a direita.

sys.monitoring.events.CALL
: Uma chamada no código Python (o evento ocorre antes da chamada).

sys.monitoring.events.C_RAISE
: Uma exceção levantada por qualquer chamável, exceto funções Python (o evento ocorre após a saída).

sys.monitoring.events.C_RETURN
: Retorno de qualquer chamável, exceto por funções Python (o evento ocorre após o retorno).

sys.monitoring.events.EXCEPTION_HANDLED
: Uma exceção é tratada.

sys.monitoring.events.INSTRUCTION
: Uma instrução VM está prestes a ser executada.

sys.monitoring.events.JUMP

É feito um salto incondicional no gráfico do fluxo de controle.

sys.monitoring.events.LINE

Está prestes a ser executada uma instrução que tem um número de linha diferente da instrução anterior.

sys.monitoring.events.PY_RESUME

Retomada de uma função Python (para funções geradoras e de corrotina), exceto para chamadas `throw()`.

sys.monitoring.events.PY_RETURN

Retorno de uma função Python (ocorre imediatamente antes do retorno, o quadro do receptor estará na pilha).

sys.monitoring.events.PY_START

Início de uma função Python (ocorre imediatamente após a chamada, o quadro do receptor da chamada estará na pilha)

sys.monitoring.events.PY_THROW

Uma função Python é retomada por uma chamada `throw()`.

sys.monitoring.events.PY_UNWIND

Saída de uma função Python durante o desenrolamento de uma exceção. Isso inclui exceções levantadas diretamente na função e que podem continuar a se propagar.

sys.monitoring.events.PY_YIELD

Produz de uma função Python (ocorre imediatamente antes do yield, o quadro do receptor estará na pilha).

sys.monitoring.events.RAISE

Uma exceção é levantada, exceto aquelas que causam um evento `STOP_ITERATION`.

sys.monitoring.events.RERAISE

Uma exceção é levantada novamente, por exemplo, no final de um bloco `finally`.

sys.monitoring.events.STOP_ITERATION

Uma exceção artificial **`StopIteration`** é levantada; consulte o evento STOP_ITERATION.

Mais eventos poderão ser adicionados no futuro.

Esses eventos são atributos do espaço de nomes **`sys.monitoring.events`**. Cada evento é representado como uma constante de potência de 2 inteiros. Para definir um conjunto de eventos, basta aplicar OU (OR) bit a bit nos eventos individuais juntos. Por exemplo, para especificar os eventos **`PY_RETURN`** e **`PY_START`**, use a expressão `PY_RETURN | PY_START`.

sys.monitoring.events.NO_EVENTS

Um apelido para `0` para que os usuários possam fazer comparações explícitas como:

```
if get_events(DEBUGGER_ID) == NO_EVENTS:
    ...
```

Definir este evento desativa todos os eventos.

Eventos locais

Os eventos locais estão associados à execução normal do programa e ocorrem em locais claramente definidos. Todos os eventos locais podem ser desativados por local. Os eventos locais são:

- **`PY_START`**
- **`PY_RESUME`**
- **`PY_RETURN`**
- **`PY_YIELD`**
- **`CALL`**
- **`LINE`**
- **`INSTRUCTION`**
- **`JUMP`**
- **`BRANCH_LEFT`**
- **`BRANCH_RIGHT`**
- **`STOP_ITERATION`**

Evento descontinuado

- `BRANCH`

O evento `BRANCH` foi descontinuado na versão 3.14. Usar os eventos **`BRANCH_LEFT`** e **`BRANCH_RIGHT`** proporcionará um desempenho muito melhor, pois podem ser desabilitados independentemente.

Eventos auxiliares

Os eventos auxiliares podem ser monitorados como outros eventos, mas são controlados por outro evento:

- **`C_RAISE`**
- **`C_RETURN`**

Os eventos **`C_RETURN`** e **`C_RAISE`** são controlados pelo evento **`CALL`**. Os eventos **`C_RETURN`** e **`C_RAISE`** só serão vistos se o evento **`CALL`** correspondente estiver sendo monitorado.

Outros eventos

Outros eventos não estão necessariamente vinculados a um local específico no programa e não podem ser desativados individualmente por local.

Os outros eventos que podem ser monitorados são:

- **`PY_THROW`**
- **`PY_UNWIND`**
- **`RAISE`**
- **`EXCEPTION_HANDLED`**
- **`RERAISE`**

Alterado na versão 3.15: Outros eventos agora podem ser ativados e desativados individualmente para cada objeto de código. Retornar **`DISABLE`** a partir de um função de retorno (callback) desativa o evento para todo o objeto de código (para a ferramenta atual).

O evento STOP_ITERATION

PEP 380 [https://peps.python.org/pep-0380/#use-of-stopiteration-to-return-values] especifica que uma exceção `StopIteration` é levantada ao retornar um valor de um gerador ou corrotina. No entanto, essa é uma maneira muito ineficiente de retornar um valor, portanto, algumas implementações do Python, especialmente o CPython 3.12+, não levantam uma exceção, a menos que ela seja visível para outro código.

Para permitir que as ferramentas monitorem exceções reais sem reduzir a velocidade dos geradores e das corrotinas, o evento `STOP_ITERATION` é fornecido. O `STOP_ITERATION` pode ser desativado localmente.

Observe que o evento `STOP_ITERATION` e o evento `RAISE` para uma exceção `StopIteration` são equivalentes e tratados como intercambiáveis ao gerar eventos. As implementações favorecerão `STOP_ITERATION` por questões de desempenho, mas podem gerar um evento `RAISE` com uma `StopIteration`.

Ativação e desativação de eventos

Para monitorar um evento, ele deve ser ativado e uma função de retorno correspondente deve ser registrada. Os eventos podem ser ativados ou desativados definindo-os globalmente e/ou para um objeto código específico. Um evento vai acionar apenas uma vez, mesmo se ele for ligado globalmente e localmente.

Definir eventos globalmente

Os eventos podem ser controlados globalmente, modificando o conjunto de eventos que estão sendo monitorados.

sys.monitoring.get_events(*tool_id: int, /*) → int

Retorna o endereço `int` que representa todos os eventos ativos.

sys.monitoring.set_events(*tool_id: int, event_set: int, /*) → None

Ativa todos os eventos definidos em *event_set*. Levanta um `ValueError` se *tool_id* não estiver em uso.

Nenhum evento está ativo por padrão.

Eventos por objeto código

Os eventos também podem ser controlados por objeto código. As funções definidas abaixo que aceitam um `types.CodeType` devem estar preparadas para aceitar um objeto semelhante de funções que não são definidas no Python (veja API C de monitoramento).

sys.monitoring.get_local_events(*tool_id: int, code: CodeType, /*) → int

Retorna todos os eventos locais para *code*

sys.monitoring.set_local_events(*tool_id: int, code: CodeType, event_set: int, /*) → None

Ativa todos os eventos locais para *code* que estão definidos em *event_set*. Levanta a `ValueError` se *tool_id* não estiver em uso.

Desativação de eventos

sys.monitoring.DISABLE

Um valor especial que pode ser retornado de um função de retorno função para desativar eventos para o local do código atual.

Os eventos locais podem ser desativados para um local de código específico, retornando `sys.monitoring.DISABLE` de uma função de retorno de chamada. Isso não altera quais eventos são definidos ou quaisquer outros locais de código para o mesmo evento.

Outros eventos podem ser desabilitados individualmente para cada objeto código, fazendo com que uma função de retorno de chamada (callback) retorne `sys.monitoring.DISABLE`. Isso desabilita o evento para todo o objeto código (para a ferramenta atual).

A desativação de eventos para locais específicos é muito importante para o monitoramento de alto desempenho. Por exemplo, um programa pode ser executado em um depurador sem sobrecarga se o depurador desativar todo o monitoramento, exceto alguns pontos de interrupção.

sys.monitoring.restart_events() → None

Habilita todos os eventos que foram desabilitados por **sys.monitoring.DISABLE** para todas as ferramentas.

Registro de funções de retorno de chamada

sys.monitoring.register_callback(*tool_id: int*, *event: int*, *func: Callable | None*, /) → Callable | None

Registra o chamável *func* para o *event* com o *tool_id* fornecido

Se outra função de retorno tiver sido registrada para o *tool_id* e o *event* fornecidos, ela será cancelada e retornada. Caso contrário, **register_callback()** retorna `None`.

Levanta um evento de auditoria `sys.monitoring.register_callback` com o argumento `func`.

As funções podem ser canceladas chamando `sys.monitoring.register_callback(tool_id, event, None)`.

As funções de retorno de chamada podem ser registradas e canceladas a qualquer momento.

Os retornos de chamada são chamados apenas uma vez, independentemente de o evento estar ativado global e localmente. Portanto, se um evento puder ser ativado tanto para eventos globais quanto locais pelo seu código, o retorno de chamada precisa ser escrito para lidar com qualquer um dos gatilhos.

Argumentos da função de retorno de chamada

sys.monitoring.MISSING

Um valor especial que é passado para uma função de retorno para indicar que não há argumento para a chamada.

Quando ocorre um evento ativo, a função de retorno de chamada registrada é chamada. Funções de retorno retornando um objeto diferente de **DISABLE** vai ter nenhum efeito. Eventos diferentes fornecerão à função de retorno de chamada argumentos diferentes, como segue:

- **PY_START** e **PY_RESUME**:

  ```
  func(code: CodeType, instruction_offset: int) -> object
  ```

- **PY_RETURN** e **PY_YIELD**:

  ```
  func(code: CodeType, instruction_offset: int, retval: object
  ```

- **CALL**, **C_RAISE** e **C_RETURN** (*arg0* pode ser **MISSING** especificamente):

  ```
  func(code: CodeType, instruction_offset: int, callable: obje
  ```

  code representa o objeto de código onde a chamada está sendo feita, enquanto *callable* é o objeto que está prestes a ser chamado (e, portanto, acionou o evento). Se não houver argumentos, *arg0* é definido como **sys.monitoring.MISSING**.

  Para métodos de instância, *callable* será o objeto função encontrado na classe com *arg0* definido para a instância (ou seja, o argumento `self` para o método).

- **RAISE**, **RERAISE**, **EXCEPTION_HANDLED**, **PY_UNWIND**, **PY_THROW** e **STOP_ITERATION**:

  ```
  func(code: CodeType, instruction_offset: int, exception: Bas
  ```

- **LINE**:

```
func(code: CodeType, line_number: int) -> object
```

- **`BRANCH_LEFT`**, **`BRANCH_RIGHT`** e **`JUMP`**:

  ```
  func(code: CodeType, instruction_offset: int, destination_of
  ```

  Observe que o *destination_offset* é onde o código será executado em seguida.

- **`INSTRUCTION`**:

  ```
  func(code: CodeType, instruction_offset: int) -> object
  ```

`sysconfig` — Fornece acesso às informações de configuração do Python

Adicionado na versão 3.2.

Código-fonte: Lib/sysconfig [https://github.com/python/cpython/tree/3.15/Lib/sysconfig]

O módulo **`sysconfig`** fornece acesso às informações de configuração do Python, como a lista de caminhos de instalação e as variáveis de configuração relevantes para a plataforma atual.

Variáveis de configuração

Uma distribuição do Python contém um arquivo de cabeçalho `Makefile` e `pyconfig.h` que são necessários para construir o próprio binário do Python e extensões C de terceiros compiladas usando `setuptools`.

`sysconfig` coloca todas as variáveis encontradas nestes arquivos em um dicionário que pode ser acessado usando **`get_config_vars()`** ou **`get_config_var()`**.

Note que no Windows, é um conjunto muito menor.

sysconfig.get_config_vars(**args*)

Sem argumentos, retorna um dicionário de todas as variáveis de configuração relevantes para a plataforma atual.

Com argumentos, retorna uma lista de valores resultantes da pesquisa de cada argumento no dicionário de variáveis de configuração.

Para cada argumento, se o valor não for encontrado, retorna `None`.

sysconfig.get_config_var(*name*)

Retorna o valor de uma única variável *name*. Equivalente a `get_config_vars().get(name)`.

Se *name* não for encontrado, retorna `None`.

Exemplo de uso:

```
>>> import sysconfig
>>> sysconfig.get_config_var('Py_ENABLE_SHARED')
0
>>> sysconfig.get_config_var('LIBDIR')
'/usr/local/lib'
>>> sysconfig.get_config_vars('AR', 'CXX')
['ar', 'g++']
```

Caminhos de instalação

O Python usa um esquema de instalação que difere dependendo da plataforma e das opções de instalação. Esses esquemas são armazenados em **`sysconfig`** sob identificadores únicos baseados no valor retornado por **`os.name`**. Os esquemas são usados por instaladores de pacote para determinar para onde copiar arquivos.

Python atualmente oferece suporte a nove esquemas:

- *posix_prefix*: esquema para plataformas POSIX como Linux ou macOS. Este é o esquema padrão usado quando o Python ou um componente é instalado.
- *posix_home*: esquema para plataformas POSIX, quando a opção *home* é usada. Esse esquema define caminhos localizados sob um prefixo inicial específico.
- *posix_user*: esquema para plataformas POSIX, quando a opção *user* é usada. Esse esquema define caminhos localizados sob o diretório pessoal (home) do usuário (**`site.USER_BASE`**).
- *posix_venv*: esquema para **`ambientes virtuais do Python`** em plataformas POSIX; por padrão, é o mesmo que *posix_prefix*.

- *nt*: esquema para Windows. Este é o esquema padrão usado quando o Python ou um componente é instalado.
- *nt_user*: esquema para Windows, quando utilizada a opção *user*.
- *nt_venv*: esquema para `ambientes virtuais do Python` no Windows; por padrão, é o mesmo que *nt_prefix*.
- *venv*: um esquema com valores de *posix_venv* ou *nt_venv* dependendo da plataforma em que o Python é executado.
- *osx_framework_user*: esquema para plataformas macOS, quando utilizada a opção *user*.

Cada esquema é composto por uma série de caminhos e cada caminho possui um identificador único. Python atualmente usa oito caminhos:

- *stdlib*: diretório que contém os arquivos da biblioteca Python padrão que não são específicos da plataforma.
- *platstdlib*: diretório que contém os arquivos da biblioteca Python padrão que são específicos da plataforma.
- *platlib*: diretório para arquivos específicos do site e específicos da plataforma.
- *purelib*: diretório para arquivos específicos do site e não específicos da plataforma (Python 'pure').
- *include*: diretório para arquivos de cabeçalho não específicos da plataforma para a API C do Python.
- *platinclude*: diretório para arquivos de cabeçalho específicos da plataforma para a API C do Python.
- *scripts*: diretório para arquivos de script.
- *data*: diretório para arquivos de dados.

Esquema de usuário

Este esquema foi projetado para ser a solução mais conveniente para usuários que não têm permissão de escrita no diretório global de pacotes de sites ou não desejam instalar nele.

Os arquivos serão instalados em subdiretórios de `site.USER_BASE` (escrito como `userbase` daqui em diante). Este esquema instala módulos Python

puros e módulos de extensão no mesmo local (também conhecido como `site.USER_SITE`).

posix_user

Caminho	Diretório de instalação
stdlib	*userbase*/lib/python*X.Y*
platstdlib	*userbase*/lib/python*X.Y*
platlib	*userbase*/lib/python*X.Y*/site-packages
purelib	*userbase*/lib/python*X.Y*/site-packages
include	*userbase*/include/python*X.Y*
scripts	*userbase*/bin
data	*userbase*

nt_user

Caminho	Diretório de instalação
stdlib	*userbase*\Python*XY*

Caminho	Diretório de instalação
platstdlib	*userbase*\Python*XY*
platlib	*userbase*\Python*XY*\site-packages
purelib	*userbase*\Python*XY*\site-packages
include	*userbase*\Python*XY*\Include
scripts	*userbase*\Python*XY*\Scripts
data	*userbase*

osx_framework_user

Caminho	Diretório de instalação
stdlib	*userbase*/lib/python
platstdlib	*userbase*/lib/python
platlib	*userbase*/lib/python/site-packages
purelib	*userbase*/lib/python/site-packages

Caminho	Diretório de instalação
include	`userbase/include/pythonX.Y`
scripts	`userbase/bin`
data	`userbase`

Esquema de home

A ideia por trás do “esquema home” é que você construa e mantenha um estoque pessoal de módulos Python. O nome deste esquema é derivado da ideia de um diretório “home” no Unix, uma vez que não é incomum para um usuário Unix fazer seu diretório home ter um layout semelhante a `/usr/` ou `/usr/local/`. Este esquema pode ser usado por qualquer pessoa, independentemente do sistema operacional para o qual está instalando.

`posix_home`

Caminho	Diretório de instalação
stdlib	`home/lib/python`
platstdlib	`home/lib/python`
platlib	`home/lib/python`

Caminho	Diretório de instalação
purelib	`home/lib/python`
include	`home/include/python`
platinclude	`home/include/python`
scripts	`home/bin`
data	`home`

Esquema de prefixo

O “esquema prefixo” é útil quando você deseja usar uma instalação Python para realizar a compilação/instalação (ou seja, para executar o script de configuração), mas instalar módulos no diretório de módulo de terceiros de uma instalação Python diferente (ou algo que parece uma instalação diferente do Python). Se isso parece um pouco incomum, então é – é por isso que os esquemas usuário e home vêm antes. No entanto, existem pelo menos dois casos conhecidos em que o esquema prefixo será útil.

Primeiro, considere que muitas distribuições Linux colocam Python em `/usr`, ao invés do mais tradicional `/usr/local`. Isso é totalmente apropriado, já que, nesses casos, o Python é parte do “sistema” em vez de um complemento local. No entanto, se você estiver instalando módulos Python a partir do código-fonte, provavelmente deseja que eles entrem em `/usr/local/lib/python2.X` em vez de `/usr/lib/python2.X`.

Outra possibilidade é um sistema de arquivos de rede onde o nome usado para escrever em um diretório remoto é diferente do nome usado para lê-lo:

por exemplo, o interpretador Python acessado como `/usr/local/bin/python` pode pesquisar por módulos em `/usr/local/lib/python2.X`, mas esses módulos teriam que ser instalados em, digamos, `/mnt/@server/export/lib/python2.X`.

posix_prefix

Caminho	Diretório de instalação
stdlib	*prefix*/lib/python*X.Y*
platstdlib	*prefix*/lib/python*X.Y*
platlib	*prefix*/lib/python*X.Y*/site-packages
purelib	*prefix*/lib/python*X.Y*/site-packages
include	*prefix*/include/python*X.Y*
platinclude	*prefix*/include/python*X.Y*
scripts	*prefix*/bin
data	*prefix*

nt

Caminho	Diretório de instalação
stdlib	*prefix*\Lib
platstdlib	*prefix*\Lib
platlib	*prefix*\Lib\site-packages
purelib	*prefix*\Lib\site-packages
include	*prefix*\Include
platinclude	*prefix*\Include
scripts	*prefix*\Scripts
data	*prefix*

Funções de caminho de instalação

`sysconfig` fornece algumas funções para determinar esses caminhos de instalação.

sysconfig.get_scheme_names()

Retorna uma tupla contendo todos os esquemas atualmente suportados em **`sysconfig`**.

sysconfig.get_default_scheme()

Retorna o nome do esquema padrão para a plataforma atual.

Adicionado na versão 3.10: Esta função era chamada de `_get_default_scheme()` e considerada um detalhe de implementação.

Alterado na versão 3.11: Quando o Python é executado em um ambiente virtual, o esquema *venv* é retornado.

sysconfig.get_preferred_scheme(*key*)

Retorna um nome de esquema preferido para um layout de instalação especificado por *key*.

key deve ser `"prefix"`, `"home"` ou `"user"`.

O valor de retorno é um nome de esquema listado em **`get_scheme_names()`**. Ele pode ser passado para as funções **`sysconfig`** que recebem um argumento *scheme*, como **`get_paths()`**.

Adicionado na versão 3.10.

Alterado na versão 3.11: Quando o Python é executado em um ambiente virtual e `key="prefix"`, o esquema *venv* é retornado.

sysconfig._get_preferred_schemes()

Retorna um dict contendo nomes de esquema preferidos na plataforma atual. Os implementadores e redistribuidores do Python podem adicionar seus esquemas preferidos ao valor global de nível de módulo `_INSTALL_SCHEMES` e modificar esta função para retornar esses nomes de esquema. Por exemplo, fornecer esquemas diferentes para os gerenciadores de pacotes de sistema e idioma usarem, de modo que os pacotes instalados por um não se misturem com os do outro.

Os usuários finais não devem usar esta função, mas **`get_default_scheme()`** e **`get_preferred_scheme()`**.

Adicionado na versão 3.10.

sysconfig.get_path_names()

Retorna uma tupla contendo todos os nomes de caminhos atualmente suportados em **`sysconfig`**.

sysconfig.get_path(*name*[, *scheme*[, *vars*[, *expand*]]])

Retorna um caminho de instalação correspondente ao caminho *name*, do esquema de instalação denominado *scheme*.

name deve ser um valor da lista retornada por **`get_path_names()`**.

`sysconfig` armazena os caminhos de instalação correspondentes a cada nome de caminho, para cada plataforma, com variáveis a serem expandidas. Por exemplo, o caminho *stdlib* para o esquema *nt* é: `{base}/Lib`.

`get_path()` usará as variáveis retornadas por **`get_config_vars()`** para expandir o caminho. Todas as variáveis possuem valores padrão para cada plataforma, portanto, pode-se chamar esta função e obter o valor padrão.

Se *scheme* for fornecido, deve ser um valor da lista retornada por **`get_scheme_names()`**. Caso contrário, o esquema padrão para a plataforma atual é usado.

Se *vars* for fornecido, deve ser um dicionário de variáveis que atualizará o dicionário retornado por **`get_config_vars()`**.

Se *expand* for definido como `False`, o caminho não será expandido usando as variáveis.

Se *name* não for encontrado, levanta uma **`KeyError`**.

sysconfig.get_paths([*scheme*[, *vars*[, *expand*]]])

Retorna um dicionário contendo todos os caminhos de instalação correspondentes a um esquema de instalação. Veja **`get_path()`** para mais informações.

Se *esquema* não for fornecido, usará o esquema padrão para a plataforma atual.

Se *vars* for fornecido, deve ser um dicionário de variáveis que atualizará o dicionário usado para expandir os caminhos.

Se *expand* for definido como falso, os caminhos não serão expandidos.

Se *scheme* não for um esquema existente, `get_paths()` vai levantar uma `KeyError`.

Outras funções

sysconfig.get_python_version()

Retorna o número da versão Python `MAJOR.MINOR` como uma string. Semelhante a `'%d.%d' % sys.version_info[:2]`.

sysconfig.get_platform()

Retorna uma string que identifica a plataforma atual.

Isso é usado principalmente para distinguir diretórios de construção específicos da plataforma e distribuições construídas específicas da plataforma. Geralmente inclui o nome e a versão do sistema operacional e a arquitetura (conforme fornecido por `os.uname()`), embora as informações exatas incluídas dependam do sistema operacional; por exemplo, no Linux, a versão do kernel não é particularmente importante.

Exemplos de valores retornados:

- linux-x86_64
- linux-aarch64
- solaris-2.6-sun4u

Windows:

- win-amd64 (Windows 64 bits no AMD64, isto é, x86_64, Intel64 e EM64T)

- win-arm64 (Windows 64 bits no ARM64, isto é, AArch64)
- win32 (todos os demais - especificamente, sys.platform é retornado)

SO baseado em POSIX:

- linux-x86_64
- macosx-15.5-arm64
- macosx-26.0-universal2 (macOS sobre Apple Silicon ou Intel)
- android-24-arm64_v8a

Para outras plataformas não POSIX, é retornado apenas `sys.platform`.

sysconfig.is_python_build()

Retorna `True` se o interpretador Python em execução foi construído a partir do código-fonte e está sendo executado a partir de seu local de construção, e não de um local resultante de, por exemplo, executando `make install` ou instalando através de um instalador binário.

sysconfig.parse_config_h(*fp*[, *vars*])

Analisa um arquivo no estilo `config.h`.

fp é um objeto arquivo ou similar apontando para o arquivo `config.h` ou similar.

Um dicionário contendo pares nome/valor é retornado. Se um dicionário opcional for passado como segundo argumento, ele será usado no lugar de um novo dicionário e atualizado com os valores lidos no arquivo.

sysconfig.get_config_h_filename()

Retorna o caminho do `pyconfig.h`.

sysconfig.get_makefile_filename()

Retorna o caminho do `Makefile`.

Uso na linha de comando

Você pode usar **`sysconfig`** como um script com a opção *-m* do Python:

```
$ python -m sysconfig
Platform: "macosx-10.4-i386"
Python version: "3.2"
Current installation scheme: "posix_prefix"

Paths:
        data = "/usr/local"
        include = "/Users/tarek/Dev/svn.python.org/py3k/Include'
        platinclude = "."
        platlib = "/usr/local/lib/python3.2/site-packages"
        platstdlib = "/usr/local/lib/python3.2"
        purelib = "/usr/local/lib/python3.2/site-packages"
        scripts = "/usr/local/bin"
        stdlib = "/usr/local/lib/python3.2"

Variables:
        AC_APPLE_UNIVERSAL_BUILD = "0"
        AIX_GENUINE_CPLUSPLUS = "0"
        AR = "ar"
        ARFLAGS = "rc"
        ...
```

Esta chamada imprimirá na saída padrão as informações retornadas por `get_platform()`, `get_python_version()`, `get_path()` e `get_config_vars()`.

`builtins` — Objetos embutidos

Este módulo fornece acesso direto a todos os identificadores embutidos do Python; Por exemplo, `builtins.open` é o nome completo para a função embutida **`open()`**.

Este módulo normalmente não é acessado explicitamente pela maioria dos aplicativos, mas pode ser útil em módulos que fornecem objetos com o mesmo nome como um valor embutido, mas em que o objeto embutido desse nome também é necessário. Por exemplo, em um módulo que deseja implementar uma função **`open()`** que envolve o embutido **`open()`**, este módulo pode ser usado diretamente:

```
import builtins

def open(path):
    f = builtins.open(path, 'r')
    return UpperCaser(f)

class UpperCaser:
    '''Envoltório em volta de um arquivo que converte saída para

    def __init__(self, f):
        self._f = f

    def read(self, count=-1):
        return self._f.read(count).upper()

    # ...
```

Como um detalhe de implementação, a maioria dos módulos tem o nome `__builtins__` disponibilizados como parte de seus globais. O valor de `__builtins__` normalmente, este é o módulo ou o valor desse módulo **`__dict__`** atributo. Uma vez que este é um detalhe de implementação, ele não pode ser usado por implementações alternativas do Python.

Ver também

- Constantes embutidas
- Exceções embutidas
- Funções embutidas
- Tipos embutidos

`__main__` — Ambiente de código principal

Em Python, o nome especial `__main__` é usado em duas construções importantes:

1. O nome do ambiente principal do programa, que pode ser verificado usando a expressão `__name__ == '__main__'`; e
2. o arquivo `__main__.py` em pacotes Python.

Ambas as formas estão relacionadas aos módulos Python; como os usuários interagem com eles e como eles interagem entre si. Eles serão explicados em detalhes abaixo. Se você ainda não conhece módulos Python, veja a seção Módulos para uma introdução.

`__name__ == '__main__'`

Quando um pacote ou módulo Python é importado, `__name__` é definido como o nome do módulo. Normalmente, este é o nome do próprio arquivo Python sem a extensão `.py`:

```
>>> import configparser
>>> configparser.__name__
'configparser'
```

Caso o arquivo seja parte de um pacote, `__name__` também incluirá o nome da pasta raiz do pacote.

```
>>> from concurrent.futures import process
>>> process.__name__
'concurrent.futures.process'
```

Porém, se o módulo for executado como o código principal, `__name__` passa a ser definido como a string `'__main__'`

O que é o “ambiente de código principal”?

`__main__` é o nome do ambiente principal no qual o código é executado. “Ambiente de código principal” é o primeiro módulo Python definido pelo usuário que começa a ser executado. É considerado principal porque ele importa todos os outros módulos que o programa precisa. As vezes, “ambiente principal” também pode ser chamado de “ponto de entrada” da aplicação.

O ambiente de código principal pode ser:

- o escopo de um prompt de comando interativo:

  ```
  >>> __name__
  '__main__'
  ```

- o módulo Python passado ao interpretador do Python como um argumento correspondente ao nome do arquivo:

  ```
  $ python helloworld.py
  Hello, world!
  ```

- o módulo ou pacote Python passado ao interpretador do Python com o argumento `-m`:

  ```
  $ python -m tarfile
  usage: tarfile.py [-h] [-v] (...)
  ```

- código Python lido pelo interpretador através da entrada padrão de linha de comando:

  ```
  $ echo "import this" | python
  The Zen of Python, by Tim Peters

  Beautiful is better than ugly.
  Explicit is better than implicit.
  ...
  ```

- código Python passado ao interpretador do Python com o argumento `-c`:

```
$ python -c "import this"
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
...
```

Em cada uma destas situações, a variável especial `__name__` passa a ser definida como `'__main__'`.

Como resultado, um módulo pode saber se está ou não sendo executado no ambiente principal verificando seu próprio `__name__`, que habilita um termo comum para executar código condicionalmente quando o módulo não é inicializado a partir de uma instrução de importação:

```
if __name__ == '__main__':
    # Executa quando o módulo não é inicializado por uma instruç
    ...
```

Ver também

Para uma visão mais detalhada sobre como `__name__` é definido em todas as situações, veja a sessão Módulos.

Uso idiomático

Alguns módulos contém códigos que são específicos para serem usados como scripts, como análise de argumentos através de linha de comando ou leitura de dados da entrada padrão. Se um módulo como o citado for importado em um outro módulo diferente, por exemplo em testes unitários, o código do script também seria executado de forma indesejada.

É aqui onde o uso do trecho de código `if __name__ == '__main__'` revela-se útil. Os códigos dentro desta condicional não rodarão ao não ser que o módulo seja executado através do ambiente principal.

Colocar o mínimo de instruções possível no bloco abaixo de `if __name__ == '__main__'` pode melhorar a clareza e a precisão do código. Na maioria das vezes, uma função chamada de `main` encapsula o comportamento principal do programa:

```
# echo.py

import shlex
import sys

def echo(phrase: str) -> None:
   """Um invólucro fictício sobre print."""
   # com propósito de demonstração, imagine que existe
   # uma lógica reutilizável valiosa nesta função
   print(phrase)

def main() -> int:
    """Ecoa os argumentos de entrada para a saída padrão."""
    phrase = shlex.join(sys.argv)
    echo(phrase)
    return 0

if __name__ == '__main__':
    sys.exit(main())  # a próxima seção explica o uso de sys.exi
```

Repare quem se o módulo, ao invés de ter encapsulado o código dentro da função `main`, fosse colocado direto dentro do bloco `if __name__ == '__main__'`, a variável `phrase` seria global para todo o módulo. Isto é suscetível à erros pois outras funções dentro do módulo poderiam inadvertidamente usar a variável global ao invés da local. A função `main` resolve este problema.

O uso da função `main` tem o benefício adicional de a própria função `echo` ser isolada e importável em outro lugar. Quando `echo.py` é importado, as funções `echo` e `main` serão definidas, mas nenhuma delas será chamada por conta do bloco `__name__ != '__main__'`.

Considerações sobre pacotes

`main` são funções frequentemente usadas para criar ferramentas de linha de comando especificando-as como pontos de entrada para scripts de console. Quando isto é feito, pip [https://pip.pypa.io/] insere a chamada da função em um modelo de script, onde o valor de retorno de `main` é passado para **`sys.exit()`**. Por exemplo:

```
sys.exit(main())
```

Uma vez que a chamada à `main` está embutida dentro de **`sys.exit()`**, a expectativa é que a sua função retorne somente valores aceitável para **`sys.exit()`**; normalmente um inteiro ou `None` (retornado de forma implícita, caso a sua função não tenha uma instrução de retorno).

Seguindo proativamente essa convenção, nosso módulo terá o mesmo comportamento quando executado diretamente (ou seja, `python echo.py`) como terá também se posteriormente criarmos um pacote como um ponto de entrada de script de console em um pacote instalável via pip.

Em particular, tenha cuidado ao retornar strings de sua função `main`. **`sys.exit()`** interpretará um argumento de string como uma mensagem de falha, então seu programa terá um código de saída `1`, indicando falha, e a string será escrita em **`sys.stderr`**. O exemplo anterior de `echo.py` exemplifica o uso da convenção `sys.exit(main())`.

Ver também

O Guia de Usuário para Empacotamento de Python [https://packaging.python.org/] contém uma coleção de tutoriais e referências sobre como distribuir e instalar pacotes Python com ferramentas modernas.

`__main__.py` em pacotes Python

Se você não estiver familiarizado com pacotes Python, veja a seção Pacotes do tutorial. Mais comumente, o arquivo `__main__.py` é usado para fornecer

uma interface de linha de comando para um pacote. Considere o seguinte pacote hipotético, “bandclass”:

```
bandclass
  ├── __init__.py
  ├── __main__.py
  └── student.py
```

`__main__.py` será executado quando o próprio pacote for invocado diretamente da linha de comando usando o sinalizador `-m`. Por exemplo:

```
$ python -m bandclass
```

Este comando fará com que `__main__.py` seja executado. Como você utiliza esse mecanismo dependerá da natureza do pacote que você está escrevendo, mas neste caso hipotético, pode fazer sentido permitir que o professor procure alunos:

```
# bandclass/__main__.py

import sys
from .student import search_students

student_name = sys.argv[1] if len(sys.argv) >= 2 else ''
print(f'Estudante encontrado(a): {search_students(student_name)}
```

Observe que `from .student import search_students` é um exemplo de importação relativa. Esse estilo de importação pode ser usado ao fazer referência a módulos em um pacote. Para mais detalhes, veja Referências em um mesmo pacote na seção Módulos do tutorial.

Uso idiomático

O conteúdo do `__main__.py` normalmente não é protegido por um bloco `if __name__ == '__main__'` . Em vez disso, esses arquivos são mantidos curtos e importa funções para serem executados a partir de outros módulos. Esses outros módulos podem então ter suas unidades facilmente testadas e são adequadamente reutilizáveis.

Se usado, o bloco condicional `if __name__ == '__main__'` ainda funcionará como esperado para o arquivo `__main__.py` dentro de um pacote, pois seu atributo `__name__` incluirá o caminho do pacote se importado:

```
>>> import asyncio.__main__
>>> asyncio.__main__.__name__
'asyncio.__main__'
```

Porém, isso não funcionará para arquivos `__main__.py` no diretório raiz de um arquivo `.zip`. Portanto, para consistência, é preferível um `__main__.py` mínimo sem uma verificação de `__name__`.

Ver também

Veja **`venv`**, da biblioteca padrão, para um exemplo de pacote com um `__main__.py` minimalista. Ele não contém um bloco `if __name__ == '__main__'`. Você pode invocá-lo com `python -m venv [directory]`.

Veja **`runpy`** para mais detalhes sobre o sinalizador **`-m`** para o executável do interpretador.

Veja **`zipapp`** para saber como executar aplicativos compactados como arquivos *.zip*. Nesse caso, o Python procura um arquivo `__main__.py` no diretório raiz do arquivo.

`import __main__`

Independentemente do módulo com o qual um programa Python foi iniciado, outros módulos executados no mesmo programa podem importar o escopo do ambiente principal (espaço de nomes) importando o módulo `__main__`. Isso não faz a importação de um arquivo `__main__.py`, mas sim qualquer módulo que recebeu o nome especial `'__main__'`.

Aqui está um módulo de exemplo que consome o espaço de nomes `__main__`:

```
# namely.py

import __main__

def did_user_define_their_name():
    return 'my_name' in dir(__main__)

def print_user_name():
    if not did_user_define_their_name():
        raise ValueError('Defina a variável `my_name`!')

    print(__main__.my_name)
```

Exemplo de uso deste módulo pode ser como abaixo:

```
# start.py

import sys

from namely import print_user_name

# my_name = "Dinsdale"

def main():
    try:
        print_user_name()
    except ValueError as ve:
        return str(ve)

if __name__ == "__main__":
    sys.exit(main())
```

Agora, se iniciarmos nosso programa, o resultado seria assim:

```
$ python start.py
Defina a variável `my_name`!
```

O código de saída do programa seria 1, indicando um erro. Descomentar a linha com `my_name = "Dinsdale"` corrige o programa e agora ele sai com o código de status 0, indicando sucesso:

```
$ python start.py
Dinsdale
```

Observe que a importação de `__main__` não causa nenhum problema com a execução involuntária de código principal destinado ao uso de script que é colocado no bloco `if __name__ == "__main__"` do módulo `start`. Por que isso funciona?

O Python insere um módulo `__main__` vazio em **`sys.modules`** na inicialização do interpretador e o preenche executando o código principal. Em nosso exemplo, este é o módulo `start` que executa linha por linha e importa `namely`. Por sua vez, `namely` importa `__main__` (que é realmente `start`). Isso é um ciclo de importação! Felizmente, como o módulo `__main__` parcialmente preenchido está presente em **`sys.modules`**, o Python o passa para `namely`. Veja Considerações especiais sobre __main__ na referência do sistema de importação para detalhes sobre como isso funciona.

O REPL do Python é outro exemplo de um "ambiente principal", então qualquer coisa definida no REPL se torna parte do escopo do `__main__`:

```
>>> import namely
>>> namely.did_user_define_their_name()
False
>>> namely.print_user_name()
Traceback (most recent call last):
...
ValueError: Defina a variável `my_name`!
>>> my_name = 'Jabberwocky'
>>> namely.did_user_define_their_name()
True
>>> namely.print_user_name()
Jabberwocky
```

O escopo `__main__` é usado na implementação de **`pdb`** e **`rlcompleter`**.

`warnings` — Controle de avisos

Código-fonte: Lib/warnings.py
[https://github.com/python/cpython/tree/3.15/Lib/warnings.py]

As mensagens de aviso são normalmente emitidas em situações em que é útil alertar o usuário sobre alguma condição em um programa, em que essa condição (normalmente) não garante o levantamento de uma exceção nem o encerramento do programa. Por exemplo, pode-se querer emitir um aviso quando um programa usa um módulo obsoleto.

Pessoas que programam em Python emitem avisos chamando a função `warn()` definida neste módulo. (Com C usam `PyErr_WarnEx()`; veja Tratamento de exceções para detalhes).

Mensagens de aviso são normalmente escritas no `sys.stderr`, mas sua disposição pode ser alterada de forma flexível, desde ignorar todos os avisos até transformá-los em exceções. A disposição dos avisos pode variar de acordo com a categoria de aviso, o texto da mensagem de aviso e o local de origem em que ela é emitida. As repetições de um aviso específico para o mesmo local de origem são normalmente suprimidas.

Existem duas etapas no controle de avisos: primeiro, cada vez que um aviso é emitido, determina-se se uma mensagem deve ser emitida ou não; a seguir, se uma mensagem deve ser emitida, ela é formatada e impressa usando um gancho configurável pelo usuário.

A determinação de emitir ou não uma mensagem de aviso é controlada pelo filtro de aviso, que é uma sequência de regras e ações correspondentes. As regras podem ser adicionadas ao filtro chamando `filterwarnings()` e redefinidas para o estado padrão chamando `resetwarnings()`.

A exibição de mensagens de aviso é feita chamando `showwarning()`, que pode ser substituída; a implementação padrão desta função formata a

mensagem chamando `formatwarning()`, que também está disponível para uso por implementações personalizadas.

Ver também

`logging.captureWarnings()` permite que você manipule todos os avisos com a infraestrutura de registro padrão.

Categorias de avisos

Existem várias exceções embutidas que representam categorias de aviso. Essa categorização é útil para filtrar grupos de avisos.

Embora sejam tecnicamente exceções embutidas, elas são documentadas aqui, porque conceitualmente pertencem ao mecanismo de avisos.

O código do usuário pode definir categorias de aviso adicionais criando uma subclasse de uma das categorias de aviso padrão. Uma categoria de aviso deve ser sempre uma subclasse da classe `Warning`.

As seguintes classes de categorias de avisos estão definidas atualmente:

Classe	Descrição
`Warning`	Base class for warning categories. It is a subclass of `Exception`.
`UserWarning`	Base class for warnings generated by user code. The default category for `warn()`.

Classe	Descrição
`DeprecationWarning`	Base class for warnings about deprecated features when those warnings are intended for other Python developers (ignored by default, unless triggered by code in `__main__`).
`PendingDeprecationWarning`	Base class for warnings about features that will be deprecated in the future (ignored by default).
`SyntaxWarning`	Base class for warnings about dubious syntax (typically emitted when compiling Python source code, and hence may not be suppressed by runtime filters).
`RuntimeWarning`	Classe base para avisos sobre comportamento duvidoso de tempo de execução.
`FutureWarning`	Classe base para avisos sobre recursos descontinuados destinados a usuários finais de aplicações escritas em Python.
`ImportWarning`	Base class for warnings triggered during the process of importing a module (ignored by default).

Classe	Descrição
`UnicodeWarning`	Classe base para avisos relacionados a Unicode.
`EncodingWarning`	Base class for warnings related to encodings. See Ativar EncodingWarning for details.
`BytesWarning`	Classe base para avisos relacionados a `bytes` e `bytearray`.
`ResourceWarning`	Base class for warnings related to resource usage (ignored by default).

Alterado na versão 3.7: Anteriormente, `DeprecationWarning` e `FutureWarning` eram diferenciadas com base em se um recurso estava sendo removido completamente ou mudando seu comportamento. Elas agora são diferenciadas com base em seu público-alvo e na maneira como são tratadas pelos filtros de avisos padrão.

O filtro de avisos

O filtro de avisos controla se os avisos são ignorados, exibidos ou transformados em erros (levantando uma exceção).

Conceitualmente, o filtro de avisos mantém uma lista ordenada de especificações de filtro; qualquer aviso específico é comparado a cada especificação de filtro na lista, até que uma correspondência seja encontrada; o filtro determina a disposição da correspondência. Cada

entrada é uma tupla no formato (*action*, *message*, *category*, *module*, *lineno*), sendo:

- *action* é uma das seguintes strings:

Valor	Disposição
`"default"`	exibe a primeira ocorrência de avisos correspondentes para cada local (módulo + número da linha) onde o aviso é emitido
`"error"`	transforma avisos correspondentes em exceções
`"ignore"`	nunca exibe avisos correspondentes
`"always"`	sempre exibe avisos correspondentes
`"all"`	apelido para “always”
`"module"`	exibe a primeira ocorrência de avisos correspondentes para cada módulo onde o aviso é emitido (independentemente do número da linha)
`"once"`	exibe apenas a primeira ocorrência de avisos correspondentes, independentemente da localização

- *message* is a string containing a regular expression that the start of the warning message must match, case-insensitively. In `-W` and `PYTHONWARNINGS`, if *message* starts and ends with a forward slash (`/`), it specifies a regular expression as above; otherwise it is a literal string

that the start of the warning message must match (case-insensitively), ignoring any whitespace at the start or end of *message*.

- *category* é uma classe (uma subclasse de `Warning`) da qual a categoria de aviso deve ser uma subclasse para corresponder.

- *module* is a string containing a regular expression that the start of the fully qualified module name must match, case-sensitively. In `-W` and `PYTHONWARNINGS`, if *module* starts and ends with a forward slash (`/`), it specifies a regular expression as above; otherwise it is a literal string that the fully qualified module name must be equal to (case-sensitively), ignoring any whitespace at the start or end of *module*.

- *lineno* é um número inteiro que deve corresponder ao número da linha onde ocorreu o aviso, ou `0` para corresponder a todos os números de linha.

Como a classe `Warning` é derivada da classe embutida `Exception`, para transformar um aviso em um erro, basta levantarmos `category(message)`.

Se um aviso for relatado e não corresponder a nenhum filtro registrado, a ação “default” (padrão em inglês) será aplicada (daí o nome).

Critérios de supressão de avisos repetidos

Os filtros que suprimem avisos repetidos aplicam os seguintes critérios para determinar se um aviso é considerado uma repetição:

- `"default"`: Um aviso é considerado uma repetição somente se (*message*, *category*, *module*, *lineno*) forem todos iguais.
- `"module"`: Um aviso é considerado uma repetição se (*message*, *category*, *module*) forem os mesmos, ignorando o número da linha.
- `"once"`: Um aviso é considerado uma repetição se (*message*, *category*) forem os mesmos, ignorando o módulo e o número da linha.

Descrevendo filtros de aviso

The warnings filter is initialized by `-W` options passed to the Python interpreter command line and the `PYTHONWARNINGS` environment variable. The interpreter saves the arguments for all supplied entries without interpretation in `sys.warnoptions`; the `warnings` module parses these when it is first imported (invalid options are ignored, after printing a message to `sys.stderr`).

Os filtros de avisos individuais são especificados como uma sequência de campos separados por dois pontos:

```
action:message:category:module:line
```

The meaning of each of these fields is as described in O filtro de avisos. When listing multiple filters on a single line (as for `PYTHONWARNINGS`), the individual filters are separated by commas and the filters listed later take precedence over those listed before them (as they're applied left-to-right, and the most recently applied filters take precedence over earlier ones).

Commonly used warning filters apply to either all warnings, warnings in a particular category, or warnings raised by particular modules or packages. Some examples:

```
default                      # Show all warnings (even those igr
ignore                       # Ignore all warnings
error                        # Convert all warnings to errors
error::ResourceWarning       # Treat ResourceWarning messages as
default::DeprecationWarning  # Show DeprecationWarning messages
ignore,default:::mymodule    # Only report warnings triggered by
error:::mymodule             # Convert warnings to errors in "my
```

Filtro de avisos padrão

By default, Python installs several warning filters, which can be overridden by the `-W` command-line option, the `PYTHONWARNINGS` environment variable and calls to `filterwarnings()`.

In regular release builds, the default warning filter has the following entries (in order of precedence):

```
default::DeprecationWarning:__main__
ignore::DeprecationWarning
ignore::PendingDeprecationWarning
ignore::ImportWarning
ignore::ResourceWarning
```

In a debug build, the list of default warning filters is empty.

Alterado na versão 3.2: `DeprecationWarning` is now ignored by default in addition to `PendingDeprecationWarning`.

Alterado na versão 3.7: `DeprecationWarning` is once again shown by default when triggered directly by code in `__main__`.

Alterado na versão 3.7: `BytesWarning` no longer appears in the default filter list and is instead configured via `sys.warnoptions` when `-b` is specified twice.

Overriding the default filter

Developers of applications written in Python may wish to hide *all* Python level warnings from their users by default, and only display them when running tests or otherwise working on the application. The `sys.warnoptions` attribute used to pass filter configurations to the interpreter can be used as a marker to indicate whether or not warnings should be disabled:

```
import sys

if not sys.warnoptions:
    import warnings
    warnings.simplefilter("ignore")
```

Developers of test runners for Python code are advised to instead ensure that *all* warnings are displayed by default for the code under test, using code like:

```
import sys

if not sys.warnoptions:
```

```
    import os, warnings
    warnings.simplefilter("default") # Change the filter in this
    os.environ["PYTHONWARNINGS"] = "default" # Also affect subpr
```

Finally, developers of interactive shells that run user code in a namespace other than `__main__` are advised to ensure that **`DeprecationWarning`** messages are made visible by default, using code like the following (where `user_ns` is the module used to execute code entered interactively):

```
import warnings
warnings.filterwarnings("default", category=DeprecationWarning,
                                   module=user_ns.get("__name__'
```

Temporarily Suppressing Warnings

If you are using code that you know will raise a warning, such as a deprecated function, but do not want to see the warning (even when warnings have been explicitly configured via the command line), then it is possible to suppress the warning using the **`catch_warnings`** context manager:

```
import warnings

def fxn():
    warnings.warn("deprecated", DeprecationWarning)

with warnings.catch_warnings():
    warnings.simplefilter("ignore")
    fxn()
```

While within the context manager all warnings will simply be ignored. This allows you to use known-deprecated code without having to see the warning while not suppressing the warning for other code that might not be aware of its use of deprecated code.

Nota

See Concurrent safety of Context Managers for details on the concurrency-safety of the **`catch_warnings`** context manager when used in programs using multiple threads or async functions.

Testing Warnings

To test warnings raised by code, use the **`catch_warnings`** context manager. With it you can temporarily mutate the warnings filter to facilitate your testing. For instance, do the following to capture all raised warnings to check:

```
import warnings

def fxn():
    warnings.warn("deprecated", DeprecationWarning)

with warnings.catch_warnings(record=True) as w:
    # Cause all warnings to always be triggered.
    warnings.simplefilter("always")
    # Trigger a warning.
    fxn()
    # Verify some things
    assert len(w) == 1
    assert issubclass(w[-1].category, DeprecationWarning)
    assert "deprecated" in str(w[-1].message)
```

One can also cause all warnings to be exceptions by using `error` instead of `always`. One thing to be aware of is that if a warning has already been raised because of a `once`/`default` rule, then no matter what filters are set the warning will not be seen again unless the warnings registry related to the warning has been cleared.

Once the context manager exits, the warnings filter is restored to its state when the context was entered. This prevents tests from changing the warnings filter in unexpected ways between tests and leading to indeterminate test results.

Nota

See Concurrent safety of Context Managers for details on the concurrency-safety of the `catch_warnings` context manager when used in programs using multiple threads or async functions.

When testing multiple operations that raise the same kind of warning, it is important to test them in a manner that confirms each operation is raising a new warning (e.g. set warnings to be raised as exceptions and check the operations raise exceptions, check that the length of the warning list continues to increase after each operation, or else delete the previous entries from the warnings list before each new operation).

Updating Code For New Versions of Dependencies

Warning categories that are primarily of interest to Python developers (rather than end users of applications written in Python) are ignored by default.

Notably, this "ignored by default" list includes `DeprecationWarning` (for every module except `__main__`), which means developers should make sure to test their code with typically ignored warnings made visible in order to receive timely notifications of future breaking API changes (whether in the standard library or third party packages).

In the ideal case, the code will have a suitable test suite, and the test runner will take care of implicitly enabling all warnings when running tests (the test runner provided by the `unittest` module does this).

In less ideal cases, applications can be checked for use of deprecated interfaces by passing `-Wd` to the Python interpreter (this is shorthand for `-W default`) or setting `PYTHONWARNINGS=default` in the environment. This enables default handling for all warnings, including those that are ignored by default. To change what action is taken for encountered warnings you can change what argument is passed to `-W` (e.g. `-W error`). See the `-W` flag for more details on what is possible.

Funções disponíveis

warnings.warn(*message*, *category=None*, *stacklevel=1*, *source=None*, ***, *skip_file_prefixes=()*)

Issue a warning, or maybe ignore it or raise an exception. The *category* argument, if given, must be a warning category class; it defaults to `UserWarning`. Alternatively, *message* can be a `Warning` instance, in which case *category* will be ignored and `message.__class__` will be used. In this case, the message text will be `str(message)`. This function raises an exception if the particular warning issued is changed into an error by the warnings filter. The *stacklevel* argument can be used by wrapper functions written in Python, like this:

```
def deprecated_api(message):
    warnings.warn(message, DeprecationWarning, stacklevel=2)
```

This makes the warning refer to `deprecated_api`'s caller, rather than to the source of `deprecated_api` itself (since the latter would defeat the purpose of the warning message).

The *skip_file_prefixes* keyword argument can be used to indicate which stack frames are ignored when counting stack levels. This can be useful when you want the warning to always appear at call sites outside of a package when a constant *stacklevel* does not fit all call paths or is otherwise challenging to maintain. If supplied, it must be a tuple of strings. When prefixes are supplied, stacklevel is implicitly overridden to be `max(2, stacklevel)`. To cause a warning to be attributed to the caller from outside of the current package you might write:

```
# example/lower.py
_warn_skips = (os.path.dirname(__file__),)

def one_way(r_luxury_yacht=None, t_wobbler_mangrove=None):
    if r_luxury_yacht:
        warnings.warn("Please migrate to t_wobbler_mangrove=
                      skip_file_prefixes=_warn_skips)

# example/higher.py
```

```
from . import lower

def another_way(**kw):
    lower.one_way(**kw)
```

This makes the warning refer to both the `example.lower.one_way()` and `example.higher.another_way()` call sites only from calling code living outside of `example` package.

source, if supplied, is the destroyed object which emitted a **`ResourceWarning`**.

Alterado na versão 3.6: Adicionado o parâmetro *source*.

Alterado na versão 3.12: Added *skip_file_prefixes*.

warnings.warn_explicit(*message*, *category*, *filename*, *lineno*, *module=None*, *registry=None*, *module_globals=None*, *source=None*)

This is a low-level interface to the functionality of **`warn()`**, passing in explicitly the message, category, filename and line number, and optionally other arguments. *message* must be a string and *category* a subclass of **`Warning`** or *message* may be a **`Warning`** instance, in which case *category* will be ignored.

module, if supplied, should be the module name. If no module is passed, the module regular expression in warnings filter will be tested against the module names constructed from the path components starting from all parent directories (with `/__init__.py`, `.py` and, on Windows, `.pyw` stripped) and against the filename with `.py` stripped. For example, when the filename is `'/path/to/package/module.py'`, it will be tested against `'path.to.package.module'`, `'to.package.module'` `'package.module'`, `'module'`, and `'/path/to/package/module'`.

registry, if supplied, should be the `__warningregistry__` dictionary of the module. If no registry is passed, each warning is treated as the first occurrence, that is, filter actions `"default"`, `"module"` and `"once"` are handled as `"always"`.

module_globals, if supplied, should be the global namespace in use by the code for which the warning is issued. (This argument is used to support displaying source for modules found in zipfiles or other non-filesystem import sources).

source, if supplied, is the destroyed object which emitted a **`ResourceWarning`**.

Alterado na versão 3.6: Adiciona o parâmetro *source*.

Alterado na versão 3.15: If no module is passed, test the filter regular expression against module names created from the path, not only the path itself.

warnings.showwarning(*message*, *category*, *filename*, *lineno*, *file=None*, *line=None*)

Write a warning to a file. The default implementation calls `formatwarning(message, category, filename, lineno, line)` and writes the resulting string to *file*, which defaults to **`sys.stderr`**. You may replace this function with any callable by assigning to `warnings.showwarning`. *line* is a line of source code to be included in the warning message; if *line* is not supplied, **`showwarning()`** will try to read the line specified by *filename* and *lineno*.

warnings.formatwarning(*message*, *category*, *filename*, *lineno*, *line=None*)

Format a warning the standard way. This returns a string which may contain embedded newlines and ends in a newline. *line* is a line of source code to be included in the warning message; if *line* is not supplied, **`formatwarning()`** will try to read the line specified by *filename* and *lineno*.

warnings.filterwarnings(*action*, *message=''*, *category=Warning*, *module=''*, *lineno=0*, *append=False*)

Insert an entry into the list of warnings filter specifications. The entry is inserted at the front by default; if *append* is true, it is inserted at the end. This checks the types of the arguments, compiles the *message* and *module* regular expressions, and inserts them as a tuple in the list of

warnings filters. Entries closer to the front of the list override entries later in the list, if both match a particular warning. Omitted arguments default to a value that matches everything.

warnings.simplefilter(*action*, *category=Warning*, *lineno=0*, *append=False*)

Insert a simple entry into the list of warnings filter specifications. The meaning of the function parameters is as for `filterwarnings()`, but regular expressions are not needed as the filter inserted always matches any message in any module as long as the category and line number match.

warnings.resetwarnings()

Reset the warnings filter. This discards the effect of all previous calls to `filterwarnings()`, including that of the `-W` command line options and calls to `simplefilter()`.

@warnings.deprecated(*message*, */*, ***, *category=DeprecationWarning*, *stacklevel=1*)

Decorator to indicate that a class, function or overload is deprecated.

When this decorator is applied to an object, deprecation warnings may be emitted at runtime when the object is used. static type checkers will also generate a diagnostic on usage of the deprecated object.

Uso:

```
from warnings import deprecated
from typing import overload

@deprecated("Use B instead")
class A:
    pass

@deprecated("Use g instead")
def f():
    pass

@overload
```

```
@deprecated("int support is deprecated")
def g(x: int) -> int: ...
@overload
def g(x: str) -> int: ...
```

The warning specified by *category* will be emitted at runtime on use of deprecated objects. For functions, that happens on calls; for classes, on instantiation and on creation of subclasses. If the *category* is `None`, no warning is emitted at runtime. The *stacklevel* determines where the warning is emitted. If it is `1` (the default), the warning is emitted at the direct caller of the deprecated object; if it is higher, it is emitted further up the stack. Static type checker behavior is not affected by the *category* and *stacklevel* arguments.

The deprecation message passed to the decorator is saved in the `__deprecated__` attribute on the decorated object. If applied to an overload, the decorator must be after the `@~typing.overload` decorator for the attribute to exist on the overload as returned by `typing.get_overloads()`.

Adicionado na versão 3.13: See **PEP 702** [https://peps.python.org/pep-0702/].

Gerenciadores de contexto disponíveis

class warnings.catch_warnings(*, *record=False*, *module=None*, *action=None*, *category=Warning*, *lineno=0*, *append=False*)

A context manager that copies and, upon exit, restores the warnings filter and the `showwarning()` function. If the *record* argument is `False` (the default) the context manager returns `None` on entry. If *record* is `True`, a list is returned that is progressively populated with objects as seen by a custom **`showwarning()`** function (which also suppresses output to `sys.stderr`). Each object in the list is guaranteed to have the following attributes:

- `message`: the warning message (an instance of `Warning`)
- `category`: the warning category (a subclass of `Warning`)
- `filename`: the file name where the warning occurred (`str`)

- `lineno`: the line number in the file (`int`)
- `file`: the file object used for output (if any), or `None`
- `line`: the line of source code (if available), or `None`
- `source`: the original object that generated the warning (if available), or `None`
- `module`: the module name where the warning occurred (`str`), or `None`

Alterado na versão 3.6: The `source` attribute was added.

Alterado na versão 3.15: The `module` attribute was added.

The type of these objects is not specified and may change; only the presence of these attributes is guaranteed.

The *module* argument takes a module that will be used instead of the module returned when you import **`warnings`** whose filter will be protected. This argument exists primarily for testing the **`warnings`** module itself.

Se o argumento *action* não for `None`, os argumentos restantes serão passados para **`simplefilter()`** como se fossem chamados imediatamente ao entrar no contexto.

Consulte O filtro de avisos para saber o significado de *category* e *lineno*.

Nota

See Concurrent safety of Context Managers for details on the concurrency-safety of the **`catch_warnings`** context manager when used in programs using multiple threads or async functions.

Alterado na versão 3.11: Adicionados os parâmetros *action*, *category*, *lineno* e *append*.

Concurrent safety of Context Managers

The behavior of `catch_warnings` context manager depends on the `sys.flags.context_aware_warnings` flag. If the flag is true, the context manager behaves in a concurrent-safe fashion and otherwise not. Concurrent-safe means that it is both thread-safe and safe to use within asyncio coroutines and tasks. Being thread-safe means that behavior is predictable in a multi-threaded program. The flag defaults to true for free-threaded builds and false otherwise.

If the `context_aware_warnings` flag is false, then `catch_warnings` will modify the global attributes of the **`warnings`** module. This is not safe if used within a concurrent program (using multiple threads or using asyncio coroutines). For example, if two or more threads use the **`catch_warnings`** class at the same time, the behavior is undefined.

If the flag is true, `catch_warnings` will not modify global attributes and will instead use a `ContextVar` to store the newly established warning filtering state. A context variable provides thread-local storage and it makes the use of **`catch_warnings`** thread-safe.

The *record* parameter of the context handler also behaves differently depending on the value of the flag. When *record* is true and the flag is false, the context manager works by replacing and then later restoring the module's `showwarning()` function. That is not concurrent-safe.

When *record* is true and the flag is true, the `showwarning()` function is not replaced. Instead, the recording status is indicated by an internal property in the context variable. In this case, the **`showwarning()`** function will not be restored when exiting the context handler.

The `context_aware_warnings` flag can be set the `-X context_aware_warnings` command-line option or by the `PYTHON_CONTEXT_AWARE_WARNINGS` environment variable.

> **Nota**
>
> It is likely that most programs that desire thread-safe behaviour of the warnings module will also want to set the `thread_inherit_context` flag to true. That flag causes threads

created by `threading.Thread` to start with a copy of the context variables from the thread starting it. When true, the context established by `catch_warnings` in one thread will also apply to new threads started by it. If false, new threads will start with an empty warnings context variable, meaning that any filtering that was established by a **`catch_warnings`** context manager will no longer be active.

Alterado na versão 3.14: Added the `sys.flags.context_aware_warnings` flag and the use of a context variable for `catch_warnings` if the flag is true. Previous versions of Python acted as if the flag was always set to false.

`dataclasses` — Data Classes

Código-fonte: Lib/dataclasses.py
[https://github.com/python/cpython/tree/3.15/Lib/dataclasses.py]

Este módulo fornece um decorador e funções para adicionar automaticamente métodos especiais tais como `__init__()` e `__repr__()` a classes definidas pelo usuário. Foi originalmente descrita em **PEP 557** [https://peps.python.org/pep-0557/].

Variáveis-membro a serem usadas nesses métodos gerados são definidas usando as anotações de tipo da **PEP 526** [https://peps.python.org/pep-0526/]. Por exemplo, esse código:

```
from dataclasses import dataclass

@dataclass
class InventoryItem:
    """Class for keeping track of an item in inventory."""
    name: str
    unit_price: float
    quantity_on_hand: int = 0

    def total_cost(self) -> float:
        return self.unit_price * self.quantity_on_hand
```

adicionará, entre outras coisas, um **`__init__()`** como esse:

```
def __init__(self, name: str, unit_price: float, quantity_on_har
    self.name = name
    self.unit_price = unit_price
    self.quantity_on_hand = quantity_on_hand
```

Observe que este método é adicionado automaticamente à classe: ele não é especificado diretamente na definição **`InventoryItem`** mostrada acima.

Adicionado na versão 3.7.

Conteúdo do módulo

@dataclasses.dataclass(*, *init=True*, *repr=True*, *eq=True*, *order=False*, *unsafe_hash=False*, *frozen=False*, *match_args=True*, *kw_only=False*, *slots=False*, *weakref_slot=False*)

Esta função é um decorador que é usado para adicionar métodos especiais para classes, conforme descrito abaixo.

O decorador `@dataclass` examina a classe para encontrar campos (`field`. Um `field` é definido como uma variável de classe que tem uma anotação de tipo. Com duas exceções, descritas mais adiante, `@dataclass` não examina o tipo especificado na anotação de variável.

A ordem dos campos em todos os métodos gerados é a ordem em que eles aparecem na definição de classe.

O decorador `@dataclass` adicionará vários métodos "dunder" à classe, descritos abaixo. Se algum dos métodos adicionados já existir na classe, o comportamento dependerá do parâmetro, conforme documentado abaixo. O decorador retorna a mesma classe decorada; nenhuma nova classe é criada.

Se `@dataclass` for usado apenas como um simples decorador, sem parâmetros, ele age como se tivesse os valores padrão documentados nessa assinatura. Ou seja, esses três usos de `@dataclass` são equivalentes:

```
@dataclass
class C:
    ...

@dataclass()
class C:
    ...

@dataclass(init=True, repr=True, eq=True, order=False, unsafe
```

```
            match_args=True, kw_only=False, slots=False, weakr
class C:
    ...
```

Os parâmetros do `@dataclass` são:

- *init*: Se verdadeiro (o padrão), o método **`__init__()`** será gerado.

  Se a classe do usuário definir **`__init__()`** esse parâmetro é ignorado.

- *repr*: Se verdadeiro (o padrão), um método **`__repr__()`** será gerado. A sequência de string de representação gerada terá o nome da classe e o nome e representação de cada campo, na ordem em que são definidos na classe. Os campos marcados como excluídos da representação não são incluídos. Por exemplo: `InventoryItem(name='widget', unit_price=3.0, quantity_on_hand=10)`.

  Se a classe do usuário já define **`__repr__()`** esse parâmetro é ignorado.

- *eq*: If true (the default), an **`__eq__()`** method will be generated.

  This method compares the class by comparing each field in order. Both instances in the comparison must be of the identical type.

  Se a classe do usuário já define **`__eq__()`** esse parâmetro é ignorado.

  Alterado na versão 3.13: The generated `__eq__` method now compares each field individually (for example, `self.a == other.a and self.b == other.b`), rather than comparing tuples of fields as in previous versions.

  This change makes the comparison faster but it may alter results in cases where attributes compare equal by identity but not by value (such as `float('nan')`).

In Python 3.12 and earlier, the comparison was performed by creating tuples of the fields and comparing them (for example, `(self.a, self.b) == (other.a, other.b)`).

- *order*: Se verdadeiro (o padrão é `False`), os métodos `__lt__()`, `__le__()`, `__gt__()` e `__ge__()` serão gerados. Comparam a classe como se fosse uma tupla de campos, em ordem. Ambas instâncias na comparação devem ser de tipo idêntico. Se *order* é verdadeiro e *eq* é falso, a exceção `ValueError` é levantada.

  Se a classe do usuário já define algum dentre **`__lt__()`**, **`__le__()`**, **`__gt__()`** ou **`__ge__()`**, então `TypeError` é levantada.

- *unsafe_hash*: Se verdadeiro, força `dataclasses` a criar um método `__hash__()`, mesmo que não seja seguro fazê-lo. Caso contrário, gera um método **`__hash__()`** de acordo com a configuração de *eq* e *frozen*. O valor padrão é `False`.

  `__hash__()` é usado para prover o método embutido `hash()`, e quando objetos são adicionados a coleções do tipo dicionário ou conjunto. Ter um método **`__hash__()`** implica que instâncias da classe serão imutáveis. Mutabilidade é uma propriedade complicada, que depende da intenção do programador, da existência e comportamento do método **`__eq__()`**, e dos valores dos parâmetros *eq* e *frozen* no decorador `@dataclass`.

  Por padrão, `@dataclass` não vai adicionar implicitamente um método `__hash__()`, a menos que seja seguro fazê-lo. Nem irá adicionar ou modificar um método **`__hash__()`** existente, definido explicitamente. Configurar o atributo de classe `__hash__ = None` tem um significado específico para o Python, conforme descrito na documentação de **`__hash__()`**.

  Se **`__hash__()`** não é definido explicitamente, ou se é configurado como `None`, então `@dataclass` *pode* adicionar um método **`__hash__()`** implícito. Mesmo que não seja recomendado, pode-se forçar `@dataclass` a criar um método **`__hash__()`** com `unsafe_hash=True`. Este pode ser o caso se sua classe é

logicamente imutável, mas na prática pode ser mudada. Esse é um caso de uso específico e deve ser considerado com muito cuidado.

Essas são as regras governando a criação implícita de um método **__hash__()**. Observe que não pode ter um método **__hash__()** explícito na dataclass e configurar `unsafe_hash=True`; isso resultará em um **TypeError**.

Se *eq* e *frozen* são ambos verdadeiros, por padrão `@dataclass` vai gerar um método **__hash__()**. Se *eq* é verdadeiro e *frozen* é falso, **__hash__()** será configurado para `None`, marcando a classe como não-hasheável (já que é mutável). Se *eq* é falso, **__hash__()** será deixado intocado, o que significa que o método **__hash__()** da superclasse será usado (se a superclasse é **object**, significa que voltará para o hash baseado em id).

- *frozen*: If true (the default is `False`), assigning to fields will generate an exception. This emulates read-only frozen instances. See the discussion below.

  If **__setattr__()** or **__delattr__()** is defined in the class and *frozen* is true, then **TypeError** is raised.

- *match_args*: Se verdadeiro (o padrão é `True`), a tupla **__match_args__** será criada a partir da lista de parâmetros não-somente-nomeado para o método **__init__()** gerado (mesmo se **__init__()** não for gerado, veja acima). Se falso, ou se **__match_args__** já estiver definido na classe, então **__match_args__** não será gerado.

Adicionado na versão 3.10.

- *kw_only*: Se verdadeiro (o valor padrão é `False`), então todos os campos serão marcados como somente-nomeado. Se um campo for marcado como somente-nomeado, então o único efeito é que o parâmetro **__init__()** gerado a partir de um campo somente-nomeado deve ser especificado com um campo quando

__init__() é chamado. Veja a entrada parâmetro do glossário para detalhes. Veja também a seção **KW_ONLY**.

Campos somente-nomeados não são incluídos em **__match_args__**.

Adicionado na versão 3.10.

- *slots*: Se true (o padrão é `False`), o atributo **__slots__** será gerado e uma nova classe será retornada no lugar da original. Se **__slots__** já estiver definido na classe, então **TypeError** será levantada.

Aviso

Passar parâmetros para uma classe base **__init_subclass__()** ao usar `slots=True` resultará em um **TypeError**. Use `__init_subclass__` sem parâmetros ou use valores padrão como solução alternativa. Consulte gh-91126 [https://github.com/python/cpython/issues/91126] para obter detalhes completos.

Adicionado na versão 3.10.

Alterado na versão 3.11: Se um nome de campo já estiver incluído no **__slots__** de uma classe base, ele não será incluído no **__slots__** gerado para evitar substitui-los. Portanto, não use **__slots__** para recuperar os nomes dos campos de uma classe de dados. Use **fields()** em vez disso. Para poder determinar os slots herdados, a classe base **__slots__** pode ser qualquer iterável, mas *não* um iterador.

- *weakref_slot*: Se verdadeiro (o padrão é `False`), adicione um slot chamado "__weakref__", que é necessário para tornar uma instância **fraca**. É um erro especificar `weakref_slot=True` sem também especificar `slots=True`.

Adicionado na versão 3.11.

`fields` pode opcionalmente especificar um valor padrão, usando sintaxe Python normal:

```
@dataclass
class C:
    a: int       # 'a' tem nenhum valor mínimo
    b: int = 0   # atribui um valor padrão para 'b'
```

Nesse exemplo, **`a`** e **`b`** serão incluídos no método **`__init__()`** adicionado, que será definido como:

```
def __init__(self, a: int, b: int = 0):
```

`TypeError` será levantada se um campo sem valor padrão for definido após um campo com valor padrão. Isso é verdadeiro se ocorrer numa classe simples, ou como resultado de uma herança de classe.

dataclasses.field(*, *default=MISSING*, *default_factory=MISSING*, *init=True*, *repr=True*, *hash=None*, *compare=True*, *metadata=None*, *kw_only=MISSING*, *doc=None*)

Para casos de uso comuns e simples, nenhuma outra funcionalidade é necessária. Existem, no entanto, alguns recursos que requerem informações adicionais por campo. Para satisfazer essa necessidade de informações adicionais, você pode substituir o valor do campo padrão por uma chamada para a função **`field()`** fornecida. Por exemplo:

```
@dataclass
class C:
    mylist: list[int] = field(default_factory=list)

c = C()
c.mylist += [1, 2, 3]
```

As shown above, the **`MISSING`** value is a **`sentinel`** object used to detect if some parameters are provided by the user. This sentinel is used because `None` is a valid value for some parameters with a distinct meaning. No code should directly use the **`MISSING`** value.

Os parâmetros de **`field()`** são:

- *default*: Se fornecido, este será o valor padrão para este campo. Isso é necessário porque a própria chamada **`field()`** substitui a posição normal do valor padrão.

- *default_factory*: Se fornecido, deve ser um chamável sem argumento que será chamado quando um valor padrão for necessário para este campo. Entre outras finalidades, isso pode ser usado para especificar campos com valores padrão mutáveis, conforme discutido abaixo. É um erro especificar ambos *default* e *default_factory*.

- *init*: Se verdadeiro (o padrão), este campo é incluído como um parâmetro para o método **`__init__()`** gerado.

- *repr*: Se verdadeiro (o padrão), este campo é incluído na string retornada pelo método **`__repr__()`** gerado.

- *hash*: Pode ser um bool ou `None`. Se verdadeiro, este campo é incluído no método **`__hash__()`** gerado. Se false, este campo é excluído do **`__hash__()`** gerado. Se `None` (o padrão), usa o valor de *compare*: este seria normalmente o comportamento esperado, já que um campo deve ser incluído no hash se for usado para comparações. Definir este valor para algo diferente de `None` é desencorajado.

  Uma possível razão para definir `hash=False` mas `compare=True` seria se um campo for caro para calcular um valor de hash, esse campo for necessário para teste de igualdade e houver outros campos que contribuem para o valor de hash do tipo. Mesmo que um campo seja excluído do hash, ele ainda será usado para comparações.

- *compare*: Se verdadeiro (o padrão), este campo é incluído nos métodos de igualdade e comparação gerados (**`__eq__()`**, **`__gt__()`**, etc.).

- *metadata*: Pode ser um mapeamento ou `None`. `None` é tratado como um dicionário vazio. Este valor é agrupado em

`MappingProxyType()` para torná-lo somente leitura e exposto no objeto `Field`. Ele não é usado por Data Classes e é fornecido como um mecanismo de extensão de terceiros. Vários terceiros podem ter sua própria chave, para usar como um espaço de nomes nos metadados.

- *kw_only*: Se verdadeiro, este campo será marcado como somente-nomeado. Isso é usado quando os parâmetros do método `__init__()` gerados são calculados.

  Campos somente-nomeados também não são incluídos em **`__match_args__`**.

Adicionado na versão 3.10.

- *doc*: docstring opcional para este campo.

Adicionado na versão 3.14.

If the default value of a field is specified by a call to **`field()`**, then the class attribute for this field will be replaced by the specified *default* value. If *default* is not provided, then the class attribute will be deleted. The intent is that after the `@dataclass` decorator runs, the class attributes will all contain the default values for the fields, just as if the default value itself were specified. For example, after:

```
@dataclass
class C:
    x: int
    y: int = field(repr=False)
    z: int = field(repr=False, default=10)
    t: int = 20
```

The class attribute **`C.z`** will be `10`, the class attribute **`C.t`** will be `20`, and the class attributes **`C.x`** and **`C.y`** will not be set.

Alterado na versão 3.15: If *metadata* is `None`, use an empty `frozendict`, instead of a `MappingProxyType()` of an empty `dict`.

class dataclasses.Field

`Field` objects describe each defined field. These objects are created internally, and are returned by the **`fields()`** module-level method (see below). Users should never instantiate a **`Field`** object directly. Its documented attributes are:

- **`name`**: The name of the field.
- **`type`**: The type of the field.
- **`default`**, **`default_factory`**, **`init`**, **`repr`**, **`hash`**, **`compare`**, **`metadata`**, and **`kw_only`** have the identical meaning and values as they do in the **`field()`** function.

Outros atributos podem existir, mas são privados e não devem ser inspecionados ou confiáveis.

class dataclasses.InitVar

`InitVar[T]` type annotations describe variables that are init-only. Fields annotated with **`InitVar`** are considered pseudo-fields, and thus are neither returned by the **`fields()`** function nor used in any way except adding them as parameters to **`__init__()`** and an optional **`__post_init__()`**.

dataclasses.fields(*class_or_instance*)

Retorna uma tupla de objetos **`Field`** que definem os campos para esta classe de dados. Aceita uma classe de dados ou uma instância de uma classe de dados. Levanta **`TypeError`** se não for passada uma classe de dados ou instância de uma. Não retorna pseudocampos que são `ClassVar` ou `InitVar`.

dataclasses.asdict(*obj*, ***, *dict_factory=dict*)

Converts the dataclass *obj* to a dict (by using the factory function *dict_factory*). Each dataclass is converted to a dict of its fields, as `name: value` pairs. dataclasses, dicts, frozendicts, lists, and tuples are recursed into. Other objects are copied with **`copy.deepcopy()`**.

Example of using **`asdict()`** on nested dataclasses:

```
@dataclass
class Point:
     x: int
     y: int

@dataclass
class C:
     mylist: list[Point]

p = Point(10, 20)
assert asdict(p) == {'x': 10, 'y': 20}

c = C([Point(0, 0), Point(10, 4)])
assert asdict(c) == {'mylist': [{'x': 0, 'y': 0}, {'x': 10,
```

Para criar uma cópia rasa, a seguinte solução alternativa pode ser usada:

```
{field.name: getattr(obj, field.name) for field in fields(obj
```

`asdict()` raises **`TypeError`** if *obj* is not a dataclass instance.

dataclasses.astuple(*obj*, *, *tuple_factory=tuple*)

Converts the dataclass *obj* to a tuple (by using the factory function *tuple_factory*). Each dataclass is converted to a tuple of its field values. dataclasses, dicts, frozendicts, lists, and tuples are recursed into. Other objects are copied with **`copy.deepcopy()`**.

Continuando a partir do exemplo anterior:

```
assert astuple(p) == (10, 20)
assert astuple(c) == ([(0, 0), (10, 4)],)
```

Para criar uma cópia rasa, a seguinte solução alternativa pode ser usada:

```
tuple(getattr(obj, field.name) for field in dataclasses.field
```

`astuple()` raises **`TypeError`** if *obj* is not a dataclass instance.

dataclasses.make_dataclass(*cls_name*, *fields*, ***, *bases=()*, *namespace=None*, *init=True*, *repr=True*, *eq=True*, *order=False*, *unsafe_hash=False*, *frozen=False*, *match_args=True*, *kw_only=False*, *slots=False*, *weakref_slot=False*, *module=None*, *decorator=dataclass*)

Creates a new dataclass with name *cls_name*, fields as defined in *fields*, base classes as given in *bases*, and initialized with a namespace as given in *namespace*. *fields* is an iterable whose elements are each either `name`, `(name, type)`, or `(name, type, Field)`. If just `name` is supplied, **`typing.Any`** is used for `type`. The values of *init*, *repr*, *eq*, *order*, *unsafe_hash*, *frozen*, *match_args*, *kw_only*, *slots*, and *weakref_slot* have the same meaning as they do in **`@dataclass`**.

If *module* is defined, the **`__module__`** attribute of the dataclass is set to that value. By default, it is set to the module name of the caller.

The *decorator* parameter is a callable that will be used to create the dataclass. It should take the class object as a first argument and the same keyword arguments as **`@dataclass`**. By default, the **`@dataclass`** function is used.

This function is not strictly required, because any Python mechanism for creating a new class with **`__annotations__`** can then apply the **`@dataclass`** function to convert that class to a dataclass. This function is provided as a convenience. For example:

```
C = make_dataclass('C',
                   [('x', int),
                     'y',
                    ('z', int, field(default=5))],
                   namespace={'add_one': lambda self: self.x
```

É equivalente a:

```
@dataclass
class C:
    x: int
    y: 'typing.Any'
    z: int = 5
```

```
    def add_one(self):
        return self.x + 1
```

Adicionado na versão 3.14: Added the *decorator* parameter.

dataclasses.replace(*obj*, /, ***changes*)

Creates a new object of the same type as *obj*, replacing fields with values from *changes*. If *obj* is not a Data Class, raises **`TypeError`**. If keys in *changes* are not field names of the given dataclass, raises **`TypeError`**.

The newly returned object is created by calling the **`__init__()`** method of the dataclass. This ensures that **`__post_init__()`**, if present, is also called.

Init-only variables without default values, if any exist, must be specified on the call to **`replace()`** so that they can be passed to **`__init__()`** and **`__post_init__()`**.

It is an error for *changes* to contain any fields that are defined as having `init=False`. A **`ValueError`** will be raised in this case.

Be forewarned about how `init=False` fields work during a call to **`replace()`**. They are not copied from the source object, but rather are initialized in **`__post_init__()`**, if they're initialized at all. It is expected that `init=False` fields will be rarely and judiciously used. If they are used, it might be wise to have alternate class constructors, or perhaps a custom **`replace()`** (or similarly named) method which handles instance copying.

Dataclass instances are also supported by generic function **`copy.replace()`**.

dataclasses.is_dataclass(*obj*)

Return `True` if its parameter is a dataclass (including subclasses of a dataclass, but not including generic aliases) or an instance of one, otherwise return `False`.

Se você precisa saber se a classe é uma instância de dataclass (e não a dataclass de fato), então adicione uma verificação para `not isinstance(obj, type)`:

```
def is_dataclass_instance(obj):
    return is_dataclass(obj) and not isinstance(obj, type)
```

dataclasses.MISSING

A **sentinel** object signifying a missing default or *default_factory*.

Alterado na versão 3.15: **MISSING** is now a **sentinel** object.

dataclasses.KW_ONLY

A **sentinel** object used as a type annotation. Any fields after a pseudo-field with the type of **KW_ONLY** are marked as keyword-only fields. Note that a pseudo-field of type **KW_ONLY** is otherwise completely ignored. This includes the name of such a field. By convention, a name of _ is used for a **KW_ONLY** field. Keyword-only fields signify **__init__()** parameters that must be specified as keywords when the class is instantiated.

In this example, the fields `y` and `z` will be marked as keyword-only fields:

```
@dataclass
class Point:
    x: float
    _: KW_ONLY
    y: float
    z: float

p = Point(0, y=1.5, z=2.0)
```

In a single dataclass, it is an error to specify more than one field whose type is **KW_ONLY**.

Adicionado na versão 3.10.

Alterado na versão 3.15: **KW_ONLY** is now a **sentinel** object.

exception dataclasses.FrozenInstanceError

Raised when an implicitly defined **__setattr__()** or **__delattr__()** is called on a dataclass which was defined with `frozen=True`. It is a subclass of **AttributeError**.

Processamento pós-inicialização

dataclasses.__post_init__()

When defined on the class, it will be called by the generated **__init__()**, normally as **self.__post_init__()**. However, if any `InitVar` fields are defined, they will also be passed to **__post_init__()** in the order they were defined in the class. If no **__init__()** method is generated, then **__post_init__()** will not automatically be called.

Among other uses, this allows for initializing field values that depend on one or more other fields. For example:

```
@dataclass
class C:
    a: float
    b: float
    c: float = field(init=False)

    def __post_init__(self):
        self.c = self.a + self.b
```

The **__init__()** method generated by **@dataclass** does not call base class **__init__()** methods. If the base class has an **__init__()** method that has to be called, it is common to call this method in a **__post_init__()** method:

```
class Rectangle:
    def __init__(self, height, width):
        self.height = height
        self.width = width

@dataclass
class Square(Rectangle):
    side: float
```

```
    def __post_init__(self):
        super().__init__(self.side, self.side)
```

Note, however, that in general the dataclass-generated **`__init__()`** methods don't need to be called, since the derived dataclass will take care of initializing all fields of any base class that is a dataclass itself.

See the section below on init-only variables for ways to pass parameters to **`__post_init__()`**. Also see the warning about how **`replace()`** handles `init=False` fields.

Variáveis de classe

One of the few places where **`@dataclass`** actually inspects the type of a field is to determine if a field is a class variable as defined in **PEP 526** [https://peps.python.org/pep-0526/]. It does this by checking if the type of the field is **`typing.ClassVar`**. If a field is a `ClassVar`, it is excluded from consideration as a field and is ignored by the dataclass mechanisms. Such `ClassVar` pseudo-fields are not returned by the module-level **`fields()`** function.

Variáveis de inicialização apenas

Another place where **`@dataclass`** inspects a type annotation is to determine if a field is an init-only variable. It does this by seeing if the type of a field is of type **`InitVar`**. If a field is an **`InitVar`**, it is considered a pseudo-field called an init-only field. As it is not a true field, it is not returned by the module-level **`fields()`** function. Init-only fields are added as parameters to the generated **`__init__()`** method, and are passed to the optional **`__post_init__()`** method. They are not otherwise used by dataclasses.

For example, suppose a field will be initialized from a database, if a value is not provided when creating the class:

```
@dataclass
class C:
    i: int
    j: int | None = None
```

```
    database: InitVar[DatabaseType | None] = None

    def __post_init__(self, database):
        if self.j is None and database is not None:
            self.j = database.lookup('j')

c = C(10, database=my_database)
```

In this case, `fields()` will return `Field` objects for **`i`** and **`j`**, but not for **`database`**.

Frozen instances

It is not possible to create truly immutable Python objects. However, by passing `frozen=True` to the `@dataclass` decorator you can emulate immutability. In that case, dataclasses will add `__setattr__()` and `__delattr__()` methods to the class. These methods will raise a `FrozenInstanceError` when invoked.

There is a tiny performance penalty when using `frozen=True`: `__init__()` cannot use simple assignment to initialize fields, and must use **`object.__setattr__()`**.

Herança

When the dataclass is being created by the `@dataclass` decorator, it looks through all of the class's base classes in reverse MRO (that is, starting at `object`) and, for each dataclass that it finds, adds the fields from that base class to an ordered mapping of fields. After all of the base class fields are added, it adds its own fields to the ordered mapping. All of the generated methods will use this combined, calculated ordered mapping of fields. Because the fields are in insertion order, derived classes override base classes. An example:

```
@dataclass
class Base:
    x: Any = 15.0
    y: int = 0
```

```
@dataclass
class C(Base):
    z: int = 10
    x: int = 15
```

The final list of fields is, in order, **x**, **y**, **z**. The final type of **x** is **int**, as specified in class **C**.

The generated **__init__()** method for **C** will look like:

```
def __init__(self, x: int = 15, y: int = 0, z: int = 10):
```

Re-ordering of keyword-only parameters in __init__()

After the parameters needed for **__init__()** are computed, any keyword-only parameters are moved to come after all regular (non-keyword-only) parameters. This is a requirement of how keyword-only parameters are implemented in Python: they must come after non-keyword-only parameters.

In this example, **Base.y**, **Base.w**, and **D.t** are keyword-only fields, and **Base.x** and **D.z** are regular fields:

```
@dataclass
class Base:
    x: Any = 15.0
    _: KW_ONLY
    y: int = 0
    w: int = 1

@dataclass
class D(Base):
    z: int = 10
    t: int = field(kw_only=True, default=0)
```

The generated **__init__()** method for **D** will look like:

```
def __init__(self, x: Any = 15.0, z: int = 10, *, y: int = 0, w:
```

Note that the parameters have been re-ordered from how they appear in the list of fields: parameters derived from regular fields are followed by parameters derived from keyword-only fields.

The relative ordering of keyword-only parameters is maintained in the re-ordered **`__init__()`** parameter list.

Funções padrão de fábrica

If a **`field()`** specifies a *default_factory*, it is called with zero arguments when a default value for the field is needed. For example, to create a new instance of a list, use:

```
mylist: list = field(default_factory=list)
```

If a field is excluded from **`__init__()`** (using `init=False`) and the field also specifies *default_factory*, then the default factory function will always be called from the generated **`__init__()`** function. This happens because there is no other way to give the field an initial value.

Valores padrão mutáveis

Python stores default member variable values in class attributes. Consider this example, not using dataclasses:

```
class C:
    x = []
    def add(self, element):
        self.x.append(element)

o1 = C()
o2 = C()
o1.add(1)
o2.add(2)
assert o1.x == [1, 2]
assert o1.x is o2.x
```

Note that the two instances of class **`C`** share the same class variable **`x`**, as expected.

Usando dataclasses, *se* este código fosse válido:

```
@dataclass
class D:
    x: list = []      # This code raises ValueError
    def add(self, element):
        self.x.append(element)
```

Geraria código similar a:

```
class D:
    x = []
    def __init__(self, x=x):
        self.x = x
    def add(self, element):
        self.x.append(element)

assert D().x is D().x
```

This has the same issue as the original example using class **`C`**. That is, two instances of class **`D`** that do not specify a value for **`x`** when creating a class instance will share the same copy of **`x`**. Because dataclasses just use normal Python class creation they also share this behavior. There is no general way for Data Classes to detect this condition. Instead, the **`@dataclass`** decorator will raise a **`ValueError`** if it detects an unhashable default parameter. The assumption is that if a value is unhashable, it is mutable. This is a partial solution, but it does protect against many common errors.

Using default factory functions is a way to create new instances of mutable types as default values for fields:

```
@dataclass
class D:
    x: list = field(default_factory=list)

assert D().x is not D().x
```

Alterado na versão 3.11: Instead of looking for and disallowing objects of type `list`, `dict`, or `set`, unhashable objects are now not allowed as default values. Unhashability is used to approximate mutability.

Descriptor-typed fields

Fields that are assigned descriptor objects as their default value have the following special behaviors:

- The value for the field passed to the dataclass's `__init__()` method is passed to the descriptor's `__set__()` method rather than overwriting the descriptor object.
- Similarly, when getting or setting the field, the descriptor's `__get__()` or `__set__()` method is called rather than returning or overwriting the descriptor object.
- To determine whether a field contains a default value, `@dataclass` will call the descriptor's `__get__()` method using its class access form: `descriptor.__get__(obj=None, type=cls)`. If the descriptor returns a value in this case, it will be used as the field's default. On the other hand, if the descriptor raises `AttributeError` in this situation, no default value will be provided for the field.

```
class IntConversionDescriptor:
    def __init__(self, *, default):
        self._default = default

    def __set_name__(self, owner, name):
        self._name = "_" + name

    def __get__(self, obj, type):
        if obj is None:
            return self._default

        return getattr(obj, self._name, self._default)

    def __set__(self, obj, value):
        setattr(obj, self._name, int(value))

@dataclass
```

```
class InventoryItem:
    quantity_on_hand: IntConversionDescriptor = IntConversionDes

i = InventoryItem()
print(i.quantity_on_hand)   # 100
i.quantity_on_hand = 2.5    # calls __set__ with 2.5
print(i.quantity_on_hand)   # 2
```

Note that if a field is annotated with a descriptor type, but is not assigned a descriptor object as its default value, the field will act like a normal field.

`contextlib` — Utilitários para contextos da instrução `with`

Código-fonte: Lib/contextlib.py
[https://github.com/python/cpython/tree/3.15/Lib/contextlib.py]

Este módulo fornece utilitários para tarefas comuns envolvendo a instrução **`with`**. Para mais informações, veja também Tipos de Gerenciador de Contexto e Gerenciadores de contexto da instrução with.

Utilitários

Funções e classes fornecidas:

class contextlib.AbstractContextManager

An abstract base class for classes that implement **`__enter__()`** and **`__exit__()`**. A default implementation for **`__enter__()`** is provided which returns `self` while **`__exit__()`** is an abstract method which by default returns `None`. See also the definition of Tipos de Gerenciador de Contexto.

Adicionado na versão 3.6.

class contextlib.AbstractAsyncContextManager

An abstract base class for classes that implement **`__aenter__()`** and **`__aexit__()`**. A default implementation for **`__aenter__()`** is provided which returns `self` while **`__aexit__()`** is an abstract method which by default returns `None`. See also the definition of Gerenciadores de contexto assíncronos.

Adicionado na versão 3.7.

@contextlib.contextmanager

Esta função é um decorador que pode ser usado para definir uma função de fábrica para gerenciadores de contexto de instrução **`with`**, sem precisar criar uma classe ou separar os métodos **`__enter__()`** e **`__exit__()`**.

While many objects natively support use in with statements, sometimes a resource needs to be managed that isn't a context manager in its own right, and doesn't implement a `close()` method for use with `contextlib.closing`.

Um exemplo abstrato seria o seguinte para garantir o gerenciamento correto dos recursos:

```
from contextlib import contextmanager

@contextmanager
def managed_resource(*args, **kwds):
    # Códigopara obter recursos como, por exemplo:
    resource = acquire_resource(*args, **kwds)
    try:
        yield resource
    finally:
        # Código para liberar recursos como, por exemplo:
        release_resource(resource)
```

A função pode, então, ser usada da seguinte forma:

```
>>> with managed_resource(timeout=3600) as resource:
...     # O recurso é liberado no final deste bloco,
...     # mesmo que o código no bloco levante uma exceção
```

A função que está sendo decorada deve retornar um iterador gerador quando chamada. Este iterador deve produzir exatamente um valor, que será vinculado aos alvos na cláusula **`as`** da instrução **`with`**, se houver.

No ponto em que o gerador é produzido, o bloco aninhado na instrução **`with`** é executado. O gerador é então retomado após o bloco ser encerrado. Se uma exceção não tratada ocorrer no bloco, ela será levantada novamente dentro do gerador no ponto em que a produção ocorreu. Assim, você pode usar uma instrução **`try`**…**`except`**…**`finally`**

para capturar o erro (se houver), ou garantir que alguma limpeza ocorra. Se uma exceção for capturada apenas para registrá-la ou para executar alguma ação (em vez de suprimi-la completamente), o gerador deve levantar novamente essa exceção. Caso contrário, o gerenciador de contexto do gerador indicará à instrução **`with`** que a exceção foi tratada, e a execução será retomada com a instrução imediatamente após a instrução **`with`**.

`@contextmanager` uses **`ContextDecorator`** so the context managers it creates can be used as decorators as well as in **`with`** statements. When used as a decorator, a new generator instance is implicitly created on each function call (this allows the otherwise "one-shot" context managers created by **`@contextmanager`** to meet the requirement that context managers support multiple invocations in order to be used as decorators).

Alterado na versão 3.2: Uso de **`ContextDecorator`**.

@contextlib.asynccontextmanager

Similar to **`@~contextlib.contextmanager`**, but creates an asynchronous context manager.

Esta função é um decorador que pode ser usado para definir uma função de fábrica para gerenciadores de contexto assíncronos de instrução **`async with`**, sem precisar criar uma classe ou separar os métodos **`__aenter__()`** e **`__aexit__()`**. Ela deve ser aplicada a uma função que atua como gerador assíncrono.

Um exemplo simples:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def get_connection():
    conn = await acquire_db_connection()
    try:
        yield conn
    finally:
        await release_db_connection(conn)
```

```
async def get_all_users():
    async with get_connection() as conn:
        return conn.query('SELECT ...')
```

Adicionado na versão 3.7.

Context managers defined with `@asynccontextmanager` can be used either as decorators or with `async with` statements:

```
import time
from contextlib import asynccontextmanager

@asynccontextmanager
async def timeit():
    now = time.monotonic()
    try:
        yield
    finally:
        print(f'it took {time.monotonic() - now}s to run')

@timeit()
async def main():
    # ... código do async ...
```

When used as a decorator, a new generator instance is implicitly created on each function call. This allows the otherwise “one-shot” context managers created by `@asynccontextmanager` to meet the requirement that context managers support multiple invocations in order to be used as decorators.

Alterado na versão 3.10: Async context managers created with `@asynccontextmanager` can be used as decorators.

contextlib.closing(*thing*)

Retorna um gerenciador de contexto que fecha *thing* após a conclusão do bloco. Isso basicamente equivale a:

```
from contextlib import contextmanager

@contextmanager
```

```
def closing(thing):
    try:
        yield thing
    finally:
        thing.close()
```

E permite que você escreva código como isso:

```
from contextlib import closing
from urllib.request import urlopen

with closing(urlopen('https://www.python.org')) as page:
    for line in page:
        print(line)
```

sem precisar fechar explicitamente `page`. Mesmo se ocorrer um erro, `page.close()` será chamado quando o bloco **`with`** for encerrado.

Nota

A maioria dos tipos que gerenciam recursos suporta o protocolo de gerenciador de contexto, que fecha *thing* ao sair da declaração **`with`**. Como tal, **`closing()`** é mais útil para tipos de terceiros que não oferecem suporte a gerenciadores de contexto. Este exemplo é puramente para fins ilustrativos, pois **`urlopen()`** normalmente seria usado em um gerenciador de contexto.

contextlib.aclosing(*thing*)

Retorna um gerenciador de contexto async que chama o método `aclose()` de *thing* após a conclusão do bloco. Isso basicamente equivale a:

```
from contextlib import asynccontextmanager

@asynccontextmanager
async def aclosing(thing):
    try:
        yield thing
    finally:
        await thing.aclose()
```

De forma significativa, `aclosing()` oferece suporte a limpeza determinística de geradores assíncronos quando eles são encerrados mais cedo por **`break`** ou uma exceção. Por exemplo:

```
from contextlib import aclosing

async with aclosing(my_generator()) as values:
    async for value in values:
        if value == 42:
            break
```

Esse padrão garante que o código de saída assíncrono do gerador seja executado no mesmo contexto que suas iterações (para que exceções e variáveis de contexto funcionem conforme o esperado, e o código de saída não seja executado após o tempo de vida de alguma tarefa da qual ele depende).

Adicionado na versão 3.10.

contextlib.nullcontext(*enter_result=None*)

Return a context manager that returns *enter_result* from **`__enter__()`**, but otherwise does nothing. It is intended to be used as a stand-in for an optional context manager, for example:

```
def myfunction(arg, ignore_exceptions=False):
    if ignore_exceptions:
        # Usa suppress para ignorar todas as exceções.
        cm = contextlib.suppress(Exception)
    else:
        # Não ignora quaisquer exceções, cm não tem efeito.
        cm = contextlib.nullcontext()
    with cm:
        # Faz algo
```

Um exemplo usando *enter_result*:

```
def process_file(file_or_path):
    if isinstance(file_or_path, str):
        # Se for uma string, abre o arquivo
        cm = open(file_or_path)
    else:
```

```
        # O chamador é responsável por fechar o arquivo
        cm = nullcontext(file_or_path)

    with cm as file:
        # Efetua um processamento no arquivo
```

Também pode ser usado como um substituto para gerenciadores de contexto assíncronos:

```
async def send_http(session=None):
    if not session:
        # Se houver nenhuma sessão http, cria-a com aiohttp
        cm = aiohttp.ClientSession()
    else:
        # O chamador é responsável por fechar a sessão
        cm = nullcontext(session)

    async with cm as session:
        # Envia requisições http com sessão
```

Adicionado na versão 3.7.

Alterado na versão 3.10: Suporte a gerenciador de contexto assíncrono foi adicionado.

contextlib.suppress(**exceptions*)

Retorna um gerenciador de contexto que suprime qualquer uma das exceções especificadas se elas ocorrerem no corpo de uma instrução **`with`** e então retoma a execução com a primeira instrução após o final da instrução **`with`**.

Como qualquer outro mecanismo que suprime completamente exceções, este gerenciador de contexto deve ser usado apenas para cobrir erros muito específicos, onde continuar silenciosamente com a execução do programa é considerado a coisa certa a fazer.

Por exemplo:

```
from contextlib import suppress

with suppress(FileNotFoundError):
```

```
    os.remove('algumarquivo.tmp')

with suppress(FileNotFoundError):
    os.remove('outroarquivo.tmp')
```

Este código equivale a:

```
try:
    os.remove('algumarquivo.tmp')
except FileNotFoundError:
    pass

try:
    os.remove('outroarquivo.tmp')
except FileNotFoundError:
    pass
```

O gerenciador de contexto é reentrante.

Se o código dentro do bloco **`with`** levantar uma exceção **`BaseExceptionGroup`**, exceções suprimidas são removidas do grupo. Quaisquer exceções do grupo que não forem suprimidas são levantadas novamente em um novo grupo que é criado usando o método **`derive()`** do grupo original.

Adicionado na versão 3.4.

Alterado na versão 3.12: `suppress` now supports suppressing exceptions raised as part of a **`BaseExceptionGroup`**.

contextlib.redirect_stdout(*new_target*)

Context manager for temporarily redirecting **`sys.stdout`** to another file object.

This tool adds flexibility to existing functions or classes whose output is hardwired to **`sys.stdout`**.

For example, the output of **`help()`** normally is sent to *sys.stdout*. You can capture that output in a string by redirecting the output to an **`io.StringIO`** object. The replacement stream is returned from the

`__enter__()` method and so is available as the target of the `with` statement:

```
with redirect_stdout(io.StringIO()) as f:
    help(pow)
s = f.getvalue()
```

To send the output of `help()` to a file on disk, redirect the output to a regular file:

```
with open('help.txt', 'w') as f:
    with redirect_stdout(f):
        help(pow)
```

To send the output of `help()` to *sys.stderr*:

```
with redirect_stdout(sys.stderr):
    help(pow)
```

Note that the global side effect on `sys.stdout` means that this context manager is not suitable for use in library code and most threaded applications. It also has no effect on the output of subprocesses. However, it is still a useful approach for many utility scripts.

O gerenciador de contexto é reentrante.

Adicionado na versão 3.4.

contextlib.redirect_stderr(*new_target*)

Similar to `redirect_stdout()` but redirecting the global `sys.stderr` to another file object.

O gerenciador de contexto é reentrante.

Adicionado na versão 3.5.

contextlib.chdir(*path*)

Non parallel-safe context manager to change the current working directory. As this changes a global state, the working directory, it is not suitable for use in most threaded or async contexts. It is also not suitable

for most non-linear code execution, like generators, where the program execution is temporarily relinquished – unless explicitly desired, you should not yield when this context manager is active.

This is a simple wrapper around `chdir()`, it changes the current working directory upon entering and restores the old one on exit.

O gerenciador de contexto é reentrante.

Adicionado na versão 3.11.

class contextlib.ContextDecorator

A base class that enables a context manager to also be used as a decorator.

Context managers inheriting from `ContextDecorator` have to implement `__enter__()` and `__exit__()` as normal. `__exit__` retains its optional exception handling even when used as a decorator.

`ContextDecorator` is used by `@contextmanager`, so you get this functionality automatically.

Example of `ContextDecorator`:

```
from contextlib import ContextDecorator

class mycontext(ContextDecorator):
    def __enter__(self):
        print('Starting')
        return self

    def __exit__(self, *exc):
        print('Finishing')
        return False
```

The class can then be used like this:

```
>>> @mycontext()
... def function():
...     print('The bit in the middle')
```

```
...
>>> function()
Starting
The bit in the middle
Finishing

>>> with mycontext():
...     print('The bit in the middle')
...
Starting
The bit in the middle
Finishing
```

This change is just syntactic sugar for any construct of the following form:

```
def f():
    with cm():
        # Do stuff
```

`ContextDecorator` lets you instead write:

```
@cm()
def f():
    # Do stuff
```

It makes it clear that the `cm` applies to the whole function, rather than just a piece of it (and saving an indentation level is nice, too).

Existing context managers that already have a base class can be extended by using `ContextDecorator` as a mixin class:

```
from contextlib import ContextDecorator

class mycontext(ContextBaseClass, ContextDecorator):
    def __enter__(self):
        return self

    def __exit__(self, *exc):
        return False
```

Nota

As the decorated function must be able to be called multiple times, the underlying context manager must support use in multiple `with` statements. If this is not the case, then the original construct with the explicit `with` statement inside the function should be used.

When the decorated callable is a generator function, coroutine function, or asynchronous generator function, the returned wrapper is of the same kind and keeps the context manager open for the lifetime of the iteration or await rather than only for the call that creates the generator or coroutine object. Wrapped generators and asynchronous generators are explicitly closed when iteration ends, as if by `closing()` or `aclosing()`.

Nota

For asynchronous generators the wrapper re-yields each value with `async for`; values sent with `asend()` and exceptions thrown with `athrow()` are not forwarded to the wrapped generator.

Adicionado na versão 3.2.

Alterado na versão 3.15: Decorating a generator function, coroutine function, or asynchronous generator function now keeps the context manager open across iteration or await. Previously the context manager exited as soon as the generator or coroutine object was created.

class contextlib.AsyncContextDecorator

Similar to `ContextDecorator`, but the context manager is entered and exited with `async with`. Decorate coroutine functions and asynchronous generator functions with this class; the returned wrapper is of the same kind.

Nota

Synchronous functions and generators are accepted, but the wrapper is always asynchronous, so the decorated callable must then be awaited

or iterated with `async for`. If that change of calling convention is not intended, use **`ContextDecorator`** instead.

Example of `AsyncContextDecorator`:

```
from asyncio import run
from contextlib import AsyncContextDecorator

class mycontext(AsyncContextDecorator):
    async def __aenter__(self):
        print('Starting')
        return self

    async def __aexit__(self, *exc):
        print('Finishing')
        return False
```

The class can then be used like this:

```
>>> @mycontext()
... async def function():
...     print('The bit in the middle')
...
>>> run(function())
Starting
The bit in the middle
Finishing

>>> async def function():
...    async with mycontext():
...         print('The bit in the middle')
...
>>> run(function())
Starting
The bit in the middle
Finishing
```

Adicionado na versão 3.10.

Alterado na versão 3.15: Decorating an asynchronous generator function now keeps the context manager open across iteration.

Previously the context manager exited as soon as the generator object was created. Synchronous functions and synchronous generator functions are also now accepted, with an asynchronous wrapper returned.

class contextlib.ExitStack

A context manager that is designed to make it easy to programmatically combine other context managers and cleanup functions, especially those that are optional or otherwise driven by input data.

For example, a set of files may easily be handled in a single with statement as follows:

```
with ExitStack() as stack:
    files = [stack.enter_context(open(fname)) for fname in fi
    # All opened files will automatically be closed at the er
    # the with statement, even if attempts to open files late
    # in the list raise an exception
```

The **`__enter__()`** method returns the **`ExitStack`** instance, and performs no additional operations.

Each instance maintains a stack of registered callbacks that are called in reverse order when the instance is closed (either explicitly or implicitly at the end of a **`with`** statement). Note that callbacks are *not* invoked implicitly when the context stack instance is garbage collected.

This stack model is used so that context managers that acquire their resources in their `__init__` method (such as file objects) can be handled correctly.

Since registered callbacks are invoked in the reverse order of registration, this ends up behaving as if multiple nested **`with`** statements had been used with the registered set of callbacks. This even extends to exception handling - if an inner callback suppresses or replaces an exception, then outer callbacks will be passed arguments based on that updated state.

This is a relatively low level API that takes care of the details of correctly unwinding the stack of exit callbacks. It provides a suitable foundation for higher level context managers that manipulate the exit stack in application specific ways.

Adicionado na versão 3.3.

enter_context(*cm*)

Enters a new context manager and adds its `__exit__()` method to the callback stack. The return value is the result of the context manager's own `__enter__()` method.

These context managers may suppress exceptions just as they normally would if used directly as part of a `with` statement.

Alterado na versão 3.11: Raises `TypeError` instead of `AttributeError` if *cm* is not a context manager.

Alterado na versão 3.15: Added support for arbitrary descriptors `__enter__()` and `__exit__()`.

push(*exit*)

Adds a context manager's `__exit__()` method to the callback stack.

As `__enter__` is *not* invoked, this method can be used to cover part of an `__enter__()` implementation with a context manager's own `__exit__()` method.

If passed an object that is not a context manager, this method assumes it is a callback with the same signature as a context manager's `__exit__()` method and adds it directly to the callback stack.

By returning true values, these callbacks can suppress exceptions the same way context manager `__exit__()` methods can.

The passed in object is returned from the function, allowing this method to be used as a function decorator.

Alterado na versão 3.15: Added support for arbitrary descriptors **`__exit__()`**.

callback(*callback*, /, **args*, ***kwds*)

Accepts an arbitrary callback function and arguments and adds it to the callback stack.

Unlike the other methods, callbacks added this way cannot suppress exceptions (as they are never passed the exception details).

The passed in callback is returned from the function, allowing this method to be used as a function decorator.

pop_all()

Transfers the callback stack to a fresh **`ExitStack`** instance and returns it. No callbacks are invoked by this operation - instead, they will now be invoked when the new stack is closed (either explicitly or implicitly at the end of a **`with`** statement).

For example, a group of files can be opened as an “all or nothing” operation as follows:

```
with ExitStack() as stack:
    files = [stack.enter_context(open(fname)) for fname i
    # Hold onto the close method, but don't call it yet.
    close_files = stack.pop_all().close
    # If opening any file fails, all previously opened fi
    # closed automatically. If all files are opened succe
    # they will remain open even after the with statement
    # close_files() can then be invoked explicitly to clo
```

close()

Immediately unwinds the callback stack, invoking callbacks in the reverse order of registration. For any context managers and exit callbacks registered, the arguments passed in will indicate that no exception occurred.

class contextlib.AsyncExitStack

An asynchronous context manager, similar to **ExitStack**, that supports combining both synchronous and asynchronous context managers, as well as having coroutines for cleanup logic.

The **close()** method is not implemented; **aclose()** must be used instead.

async enter_async_context(*cm*)

Similar to **ExitStack.enter_context()** but expects an asynchronous context manager.

Alterado na versão 3.11: Raises **TypeError** instead of **AttributeError** if *cm* is not an asynchronous context manager.

Alterado na versão 3.15: Added support for arbitrary descriptors **__aenter__()** and **__aexit__()**.

push_async_exit(*exit*)

Similar to **ExitStack.push()** but expects either an asynchronous context manager or a coroutine function.

Alterado na versão 3.15: Added support for arbitrary descriptors **__aexit__()**.

push_async_callback(*callback*, /, **args*, ***kwds*)

Similar to **ExitStack.callback()** but expects a coroutine function.

async aclose()

Similar to **ExitStack.close()** but properly handles awaitables.

Continuing the example for **@asynccontextmanager**:

```
async with AsyncExitStack() as stack:
    connections = [await stack.enter_async_context(get_connec
        for i in range(5)]
    # All opened connections will automatically be released a
    # the async with statement, even if attempts to open a co
    # later in the list raise an exception.
```

Adicionado na versão 3.7.

Exemplos e receitas

This section describes some examples and recipes for making effective use of the tools provided by **`contextlib`**.

Supporting a variable number of context managers

The primary use case for **`ExitStack`** is the one given in the class documentation: supporting a variable number of context managers and other cleanup operations in a single **`with`** statement. The variability may come from the number of context managers needed being driven by user input (such as opening a user specified collection of files), or from some of the context managers being optional:

```
with ExitStack() as stack:
    for resource in resources:
        stack.enter_context(resource)
    if need_special_resource():
        special = acquire_special_resource()
        stack.callback(release_special_resource, special)
    # Perform operations that use the acquired resources
```

As shown, **`ExitStack`** also makes it quite easy to use **`with`** statements to manage arbitrary resources that don't natively support the context management protocol.

Catching exceptions from `__enter__` methods

It is occasionally desirable to catch exceptions from an **`__enter__()`** method implementation, *without* inadvertently catching exceptions from the **`with`** statement body or the context manager's **`__exit__()`** method. By using **`ExitStack`** the steps in the context management protocol can be separated slightly in order to allow this:

```
stack = ExitStack()
try:
    x = stack.enter_context(cm)
except Exception:
    # handle __enter__ exception
else:
    with stack:
        # Handle normal case
```

Actually needing to do this is likely to indicate that the underlying API should be providing a direct resource management interface for use with `try`/`except`/`finally` statements, but not all APIs are well designed in that regard. When a context manager is the only resource management API provided, then `ExitStack` can make it easier to handle various situations that can't be handled directly in a `with` statement.

Cleaning up in an `__enter__` implementation

As noted in the documentation of `ExitStack.push()`, this method can be useful in cleaning up an already allocated resource if later steps in the `__enter__()` implementation fail.

Here's an example of doing this for a context manager that accepts resource acquisition and release functions, along with an optional validation function, and maps them to the context management protocol:

```
from contextlib import contextmanager, AbstractContextManager, E

class ResourceManager(AbstractContextManager):

    def __init__(self, acquire_resource, release_resource, check
        self.acquire_resource = acquire_resource
        self.release_resource = release_resource
        if check_resource_ok is None:
            def check_resource_ok(resource):
                return True
        self.check_resource_ok = check_resource_ok

    @contextmanager
    def _cleanup_on_error(self):
```

```
        with ExitStack() as stack:
            stack.push(self)
            yield
            # The validation check passed and didn't raise an ex
            # Accordingly, we want to keep the resource, and pas
            # back to our caller
            stack.pop_all()

    def __enter__(self):
        resource = self.acquire_resource()
        with self._cleanup_on_error():
            if not self.check_resource_ok(resource):
                msg = "Failed validation for {!r}"
                raise RuntimeError(msg.format(resource))
        return resource

    def __exit__(self, *exc_details):
        # We don't need to duplicate any of our resource release
        self.release_resource()
```

Replacing any use of `try-finally` and flag variables

A pattern you will sometimes see is a `try-finally` statement with a flag variable to indicate whether or not the body of the `finally` clause should be executed. In its simplest form (that can't already be handled just by using an `except` clause instead), it looks something like this:

```
cleanup_needed = True
try:
    result = perform_operation()
    if result:
        cleanup_needed = False
finally:
    if cleanup_needed:
        cleanup_resources()
```

As with any `try` statement based code, this can cause problems for development and review, because the setup code and the cleanup code can end up being separated by arbitrarily long sections of code.

`ExitStack` makes it possible to instead register a callback for execution at the end of a `with` statement, and then later decide to skip executing that callback:

```
from contextlib import ExitStack

with ExitStack() as stack:
    stack.callback(cleanup_resources)
    result = perform_operation()
    if result:
        stack.pop_all()
```

This allows the intended cleanup behaviour to be made explicit up front, rather than requiring a separate flag variable.

If a particular application uses this pattern a lot, it can be simplified even further by means of a small helper class:

```
from contextlib import ExitStack

class Callback(ExitStack):
    def __init__(self, callback, /, *args, **kwds):
        super().__init__()
        self.callback(callback, *args, **kwds)

    def cancel(self):
        self.pop_all()

with Callback(cleanup_resources) as cb:
    result = perform_operation()
    if result:
        cb.cancel()
```

If the resource cleanup isn't already neatly bundled into a standalone function, then it is still possible to use the decorator form of `ExitStack.callback()` to declare the resource cleanup in advance:

```
from contextlib import ExitStack

with ExitStack() as stack:
    @stack.callback
```

```
    def cleanup_resources():
        ...
    result = perform_operation()
    if result:
        stack.pop_all()
```

Devido à maneira como o protocolo decorador funciona, uma função de retorno de chamada declarada dessa forma não pode receber nenhum parâmetro. Em vez disso, quaisquer recursos a serem liberados devem ser acessados como variáveis de clausura.

Using a context manager as a function decorator

`ContextDecorator` makes it possible to use a context manager in both an ordinary `with` statement and also as a function decorator.

For example, it is sometimes useful to wrap functions or groups of statements with a logger that can track the time of entry and time of exit. Rather than writing both a function decorator and a context manager for the task, inheriting from `ContextDecorator` provides both capabilities in a single definition:

```
from contextlib import ContextDecorator
import logging

logging.basicConfig(level=logging.INFO)

class track_entry_and_exit(ContextDecorator):
    def __init__(self, name):
        self.name = name

    def __enter__(self):
        logging.info('Entering: %s', self.name)

    def __exit__(self, exc_type, exc, exc_tb):
        logging.info('Exiting: %s', self.name)
```

Instances of this class can be used as both a context manager:

```
with track_entry_and_exit('widget loader'):
    print('Some time consuming activity goes here')
    load_widget()
```

And also as a function decorator:

```
@track_entry_and_exit('widget loader')
def activity():
    print('Some time consuming activity goes here')
    load_widget()
```

Note that there is one additional limitation when using context managers as function decorators: there's no way to access the return value of `__enter__()`. If that value is needed, then it is still necessary to use an explicit `with` statement.

Ver também

PEP 343 **[https://peps.python.org/pep-0343/]** **- A instrução "with"**
A especificação, o histórico e os exemplos para a instrução Python `with`.

Single use, reusable and reentrant context managers

Most context managers are written in a way that means they can only be used effectively in a `with` statement once. These single use context managers must be created afresh each time they're used - attempting to use them a second time will trigger an exception or otherwise not work correctly.

This common limitation means that it is generally advisable to create context managers directly in the header of the `with` statement where they are used (as shown in all of the usage examples above).

Files are an example of effectively single use context managers, since the first `with` statement will close the file, preventing any further IO operations using that file object.

Context managers created using `@contextmanager` are also single use context managers, and will complain about the underlying generator failing to yield if an attempt is made to use them a second time:

```
>>> from contextlib import contextmanager
>>> @contextmanager
... def singleuse():
...     print("Before")
...     yield
...     print("After")
...
>>> cm = singleuse()
>>> with cm:
...     pass
...
Before
After
>>> with cm:
...     pass
...
Traceback (most recent call last):
    ...
RuntimeError: generator didn't yield
```

Reentrant context managers

More sophisticated context managers may be “reentrant”. These context managers can not only be used in multiple `with` statements, but may also be used *inside* a **`with`** statement that is already using the same context manager.

`threading.RLock` is an example of a reentrant context manager, as are `suppress()`, `redirect_stdout()`, and `chdir()`. Here’s a very simple example of reentrant use:

```
>>> from contextlib import redirect_stdout
>>> from io import StringIO
>>> stream = StringIO()
>>> write_to_stream = redirect_stdout(stream)
>>> with write_to_stream:
...     print("This is written to the stream rather than stdout'
...     with write_to_stream:
...         print("This is also written to the stream")
...
>>> print("This is written directly to stdout")
This is written directly to stdout
>>> print(stream.getvalue())
This is written to the stream rather than stdout
This is also written to the stream
```

Real world examples of reentrancy are more likely to involve multiple functions calling each other and hence be far more complicated than this example.

Note also that being reentrant is *not* the same thing as being thread safe. `redirect_stdout()`, for example, is definitely not thread safe, as it makes a global modification to the system state by binding `sys.stdout` to a different stream.

Gerenciadores de contexto reutilizáveis

Distinct from both single use and reentrant context managers are “reusable” context managers (or, to be completely explicit, “reusable, but not reentrant” context managers, since reentrant context managers are also reusable). These context managers support being used multiple times, but will fail (or otherwise not work correctly) if the specific context manager instance has already been used in a containing with statement.

`threading.Lock` is an example of a reusable, but not reentrant, context manager (for a reentrant lock, it is necessary to use `threading.RLock` instead).

Another example of a reusable, but not reentrant, context manager is `ExitStack`, as it invokes *all* currently registered callbacks when leaving any

with statement, regardless of where those callbacks were added:

```
>>> from contextlib import ExitStack
>>> stack = ExitStack()
>>> with stack:
...     stack.callback(print, "Callback: from first context")
...     print("Leaving first context")
...
Leaving first context
Callback: from first context
>>> with stack:
...     stack.callback(print, "Callback: from second context")
...     print("Leaving second context")
...
Leaving second context
Callback: from second context
>>> with stack:
...     stack.callback(print, "Callback: from outer context")
...     with stack:
...         stack.callback(print, "Callback: from inner context'
...         print("Leaving inner context")
...     print("Leaving outer context")
...
Leaving inner context
Callback: from inner context
Callback: from outer context
Leaving outer context
```

As the output from the example shows, reusing a single stack object across multiple with statements works correctly, but attempting to nest them will cause the stack to be cleared at the end of the innermost with statement, which is unlikely to be desirable behaviour.

Using separate **`ExitStack`** instances instead of reusing a single instance avoids that problem:

```
>>> from contextlib import ExitStack
>>> with ExitStack() as outer_stack:
...     outer_stack.callback(print, "Callback: from outer contex
...     with ExitStack() as inner_stack:
...         inner_stack.callback(print, "Callback: from inner cc
```

```
...         print("Leaving inner context")
...     print("Leaving outer context")
...
Leaving inner context
Callback: from inner context
Leaving outer context
Callback: from outer context
```

abc — Classes base abstratas

Código-fonte: Lib/abc.py [https://github.com/python/cpython/tree/3.15/Lib/abc.py]

Este módulo fornece a infraestrutura para definir classes base abstratas (ABCs, do inglês *abstract base class*) em Python, como delineado em **PEP 3119** [https://peps.python.org/pep-3119/]; veja o PEP para entender o porquê isto foi adicionado ao Python. (Veja também **PEP 3141** [https://peps.python.org/pep-3141/] e o módulo `numbers` sobre uma hierarquia de tipo para números baseado nas ABCs.)

O módulo `collections` tem algumas classes concretas que derivam de ABCs; essas podem, evidentemente, ser ainda mais derivadas. Além disso, o submódulo `collections.abc` tem algumas ABCs que podem ser usadas para testar se uma classe ou instância oferece uma interface particular, por exemplo, se é hasheável ou se é um mapeamento.

Este módulo fornece a metaclasse `ABCMeta` para definir ABCs e uma classe auxiliar `ABC` para alternativamente definir ABCs através de herança:

class abc.ABC

Uma classe auxiliar que tem `ABCMeta` como sua metaclasse. Com essa classe, uma classe base abstrata pode ser criada simplesmente derivando da `ABC` evitando às vezes confundir o uso da metaclasse, por exemplo:

```
from abc import ABC

class MyABC(ABC):
    pass
```

Note que o tipo da classe `ABC` ainda é `ABCMeta`, portanto herdar da `ABC` requer as precauções usuais a respeito do uso da metaclasse, pois herança múltipla pode levar a conflitos de metaclasse. Pode-se também definir uma classe base abstrata ao passar a palavra reservada metaclasse e usar `ABCMeta` diretamente, por exemplo:

```
from abc import ABCMeta

class MyABC(metaclass=ABCMeta):
    pass
```

Adicionado na versão 3.4.

class abc.ABCMeta

Metaclasse para definir Classe Base Abstrata (ABCs).

Use esta metaclasse para criar uma ABC. Uma ABC pode ser diretamente estendida, e então agir como uma classe misturada. Você também pode registrar classes concretas não relacionadas (até mesmo classes embutidas) e ABCs não relacionadas como "subclasses virtuais" – estas e suas descendentes serão consideradas subclasses da ABC de registro pela função embutida **`issubclass()`**, mas a ABC de registro não irá aparecer na MRO (Ordem de Resolução do Método) e nem as implementações do método definidas pela ABC de registro será chamável (nem mesmo via **`super()`**). [[1]]

Classes criadas com a metaclasse de **`ABCMeta`** tem o seguinte método:

register(*subclass*)

Registra *subclass* como uma "subclasse virtual" desta ABC. Por exemplo:

```
from abc import ABC

class MyABC(ABC):
    pass

MyABC.register(tuple)

assert issubclass(tuple, MyABC)
assert isinstance((), MyABC)
```

Alterado na versão 3.3: Retorna a subclasse registrada, para permitir o uso como um decorador de classe.

Alterado na versão 3.4: Para detectar chamadas para **`register()`**, você pode usar a função `get_cache_token()`.

Você também pode sobrepor este método em uma classe base abstrata:

__subclasshook__(*subclass*)

(Deve obrigatoriamente ser definido como um método de classe.)

Verifica se *subclass* é considerada uma subclasse desta ABC. Isto significa que você pode personalizar ainda mais o comportamento de `issubclass()` sem a necessidade de chamar `register()` em toda classe que você queira considerar uma subclasse da ABC. (Este método de classe é chamado do método `__subclasscheck__()` da ABC.)

Este método deve retornar `True`, `False` ou `NotImplemented`. Se retornar `True`, *subclass* é considerada uma subclasse desta ABC. Se retornar `False`, *subclass* não é considerada uma subclasse desta ABC, mesmo que normalmente seria uma. Se retornar **`NotImplemented`**, a verificação da subclasse é continuada com o mecanismo usual.

Para uma demonstração destes conceitos, veja este exemplo de definição ABC:

```
class Foo:
    def __getitem__(self, index):
        ...
    def __len__(self):
        ...
    def get_iterator(self):
        return iter(self)

class MyIterable(ABC):

    @abstractmethod
    def __iter__(self):
        while False:
            yield None
```

```
    def get_iterator(self):
        return self.__iter__()

    @classmethod
    def __subclasshook__(cls, C):
        if cls is MyIterable:
            if any("__iter__" in B.__dict__ for B in C.__mro_
                return True
        return NotImplemented

MyIterable.register(Foo)
```

A `MyIterable` da ABC define o método iterável padrão, `__iter__()`, como um método abstrato. A implementação dada aqui pode ainda ser chamada da subclasse. O método **`get_iterator()`** é também parte da classe base abstrata `MyIterable`, mas não precisa ser substituído nas classes derivadas não abstratas.

O método de classe `__subclasshook__()` definido aqui diz que qualquer classe que tenha um método `__iter__()` em seu `__dict__` (ou no de uma de suas classes base, acessados via lista `__mro__`) é considerada uma `MyIterable` também.

Finalmente, a última linha faz de `Foo` uma subclasse virtual da `MyIterable`, apesar de não definir um método `__iter__()` (ela usa o protocolo iterável antigo, definido em termos de `__len__()` e `__getitem__()`). Note que isto não fará o `get_iterator` disponível como um método de `Foo`, então ele é fornecido separadamente.

O módulo **`abc`** também fornece o seguinte decorador:

@abc.abstractmethod

Um decorador indicando métodos abstratos.

Using this decorator requires that the class's metaclass is `ABCMeta` or is derived from it. A class that has a metaclass derived from **`ABCMeta`** cannot be instantiated unless all of its abstract methods and properties are overridden. The abstract methods can be called using any of the

normal ‘super’ call mechanisms. **`@abstractmethod`** may be used to declare abstract methods for properties and descriptors.

Dynamically adding abstract methods to a class, or attempting to modify the abstraction status of a method or class once it is created, are only supported using the **`update_abstractmethods()`** function. The **`@abstractmethod`** only affects subclasses derived using regular inheritance; “virtual subclasses” registered with the ABC’s **`register()`** method are not affected.

When **`@abstractmethod`** is applied in combination with other method descriptors, it should be applied as the innermost decorator, as shown in the following usage examples:

```
class C(ABC):
    @abstractmethod
    def my_abstract_method(self, arg1):
        ...
    @classmethod
    @abstractmethod
    def my_abstract_classmethod(cls, arg2):
        ...
    @staticmethod
    @abstractmethod
    def my_abstract_staticmethod(arg3):
        ...

    @property
    @abstractmethod
    def my_abstract_property(self):
        ...
    @my_abstract_property.setter
    @abstractmethod
    def my_abstract_property(self, val):
        ...

    @abstractmethod
    def _get_x(self):
        ...
    @abstractmethod
    def _set_x(self, val):
```

```
        ...
    x = property(_get_x, _set_x)
```

In order to correctly interoperate with the abstract base class machinery, the descriptor must identify itself as abstract using **`__isabstractmethod__`**. In general, this attribute should be `True` if any of the methods used to compose the descriptor are abstract. For example, Python's built-in **`@property`** does the equivalent of:

```
class Descriptor:
    ...
    @property
    def __isabstractmethod__(self):
        return any(getattr(f, '__isabstractmethod__', False)
                   f in (self._fget, self._fset, self._fdel))
```

Nota

Diferente de métodos abstratos Java, esses métodos abstratos podem ter uma implementação. Esta implementação pode ser chamada via mecanismo da **`super()`** da classe que a substitui. Isto pode ser útil como um ponto final para uma super chamada em um framework que usa herança múltipla cooperativa.

O módulo **`abc`** também suporta os seguintes decoradores herdados:

@abc.abstractclassmethod

Adicionado na versão 3.2.

Descontinuado desde a versão 3.3: It is now possible to use **`@classmethod`** with **`@abstractmethod`**, making this decorator redundant.

A subclass of the built-in **`classmethod`**, indicating an abstract classmethod. Otherwise it is similar to **`@abstractmethod`**.

This special case is deprecated, as the **`@classmethod`** decorator is now correctly identified as abstract when applied to an abstract method:

```
class C(ABC):
    @classmethod
    @abstractmethod
    def my_abstract_classmethod(cls, arg):
        ...
```

@abc.abstractstaticmethod

Adicionado na versão 3.2.

Descontinuado desde a versão 3.3: It is now possible to use `@staticmethod` with `@abstractmethod`, making this decorator redundant.

A subclass of the built-in `staticmethod`, indicating an abstract staticmethod. Otherwise it is similar to `@abstractmethod`.

This special case is deprecated, as the `@staticmethod` decorator is now correctly identified as abstract when applied to an abstract method:

```
class C(ABC):
    @staticmethod
    @abstractmethod
    def my_abstract_staticmethod(arg):
        ...
```

@abc.abstractproperty

Descontinuado desde a versão 3.3: It is now possible to use `@property`, `@property.getter`, `@property.setter` and `@property.deleter` with `@abstractmethod`, making this decorator redundant.

A subclass of the built-in `property`, indicating an abstract property.

This special case is deprecated, as the `@property` decorator is now correctly identified as abstract when applied to an abstract method:

```
class C(ABC):
    @property
    @abstractmethod
    def my_abstract_property(self):
        ...
```

O exemplo acima define uma propriedade somente leitura; você também pode definir uma propriedade abstrata de leitura e escrita marcando apropriadamente um ou mais dos métodos subjacentes como abstratos:

```
class C(ABC):
    @property
    def x(self):
        ...

    @x.setter
    @abstractmethod
    def x(self, val):
        ...
```

Se apenas alguns componentes são abstratos, apenas estes componentes precisam ser atualizados para criar uma propriedade concreta em uma subclasse:

```
class D(C):
    @C.x.setter
    def x(self, val):
        ...
```

O módulo **abc** também fornece as seguintes funções:

abc.get_cache_token()

Retorna o token de cache da classe base abstrata atual.

O token é um objeto opaco (que suporta teste de igualdade) identificando a versão atual do cache da classe base abstrata para subclasses virtuais. O token muda a cada chamada ao `ABCMeta.register()` em qualquer CBA.

Adicionado na versão 3.4.

abc.update_abstractmethods(*cls*)

Uma função para recalcular o status de abstração de uma classe abstrata. Esta função deve ser chamada se os métodos abstratos de uma classe

foram implementados ou alterados após sua criação. Normalmente, essa função deve ser chamada de dentro de um decorador de classe.

Retorna *cls*, para permitir o uso como decorador de classe.

Se *cls* não for uma instância de `ABCMeta`, não faz nada.

Nota

Esta função presume que as superclasses de *cls* já estão atualizadas. Ele não atualiza nenhuma subclasse.

Adicionado na versão 3.10.

Notas de rodapé

[1]

Programadores C++ devem notar que o conceito da classe base virtual do Python não é o mesmo que o de C++.

`atexit` — Manipuladores de saída

The **`atexit`** module defines functions to register and unregister *exit handlers*: functions that are automatically executed “at exit”, that is, upon normal program termination (for instance, if **`sys.exit()`** is called or the main module’s execution completes) or, more generally, upon interpreter shutdown.

At exit, all registered exit handlers are called in the *reverse* order in which they were registered. If you register `A`, `B`, and `C`, at interpreter shutdown time they will be run in the order `C`, `B`, `A`. The assumption is that lower level modules will normally be imported before higher level modules and thus must be cleaned up later.

If an exception is raised during execution of an exit handler, a traceback is printed (unless **`SystemExit`** is raised) and the exception information is saved. After all exit handlers have had a chance to run, the last exception to be raised is re-raised.

In programs that use multiple interpreters, each interpreter has its own stack of exit handlers, which are executed when the interpreter shuts down (for example, with **`concurrent.interpreters.Interpreter.close()`** or the C API **`Py_EndInterpreter()`**). Registration functions in this module only affect the interpreter they are called from.

Note: Exit handlers are not called when the program is killed by a signal not handled by Python, when a Python fatal internal error is detected, or when **`os._exit()`** is called.

Nota: O efeito de registrar ou cancelar o registro de funções dentro de uma função de limpeza é indefinido.

Aviso

When writing exit handlers, especially in C API extensions, keep in mind that other exit handlers may still run arbitrary Python code after you clean up. Such code should succeed or fail with an exception, rather than crash.

Alterado na versão 3.12: Attempts to start a new thread or `os.fork()` a new process in an exit handler now leads to `RuntimeError`. Previously, this could cause race conditions between the main Python runtime thread freeing thread states while internal `threading` routines or the new process try to use that state, which could lead to crashes rather than clean shutdown.

Alterado na versão 3.7: When used with subinterpreters, registered functions are local to the interpreter they were registered in.

atexit.register(*func*, **args*, ***kwargs*)

Register *func* as an exit handler. Any optional arguments that are to be passed to *func* must be passed as arguments to `register()`. It is possible to register the same function and arguments more than once.

Esta função retorna *func*, o que torna possível usá-la como um decorador.

atexit.unregister(*func*)

Remove *func* from the list of exit handlers. `unregister()` silently does nothing if *func* was not previously registered. If *func* has been registered more than once, every occurrence of that function in the `atexit` call stack will be removed. Equality comparisons (==) are used internally during unregistration, so function references do not need to have matching identities.

Ver também

Módulo `readline`

Useful example of `atexit` to read and write `readline` history files.

`atexit` Example

O exemplo simples a seguir demonstra como um módulo pode inicializar um contador de um arquivo quando ele é importado e salvar automaticamente o valor atualizado do contador quando o programa termina, sem depender que a aplicação faça uma chamada explícita nesse módulo na finalização.

```
try:
    with open('counterfile') as infile:
        _count = int(infile.read())
except FileNotFoundError:
    _count = 0

def incrcounter(n):
    global _count
    _count = _count + n

def savecounter():
    with open('counterfile', 'w') as outfile:
        outfile.write('%d' % _count)

import atexit

atexit.register(savecounter)
```

Os argumentos posicional e de palavra reservada também podem ser passados para **`register()`** para ser passada para a função registrada quando é chamada

```
def goodbye(name, adjective):
    print('Goodbye %s, it was %s to meet you.' % (name, adjectiv

import atexit

atexit.register(goodbye, 'Donny', 'nice')
# ou:
atexit.register(goodbye, adjective='nice', name='Donny')
```

Utilizado como um decorador:

```
import atexit

@atexit.register
def goodbye():
    print('You are now leaving the Python sector.')
```

Isso só funciona com funções que podem ser invocadas sem argumentos.

`traceback` — Imprime ou recupera um traceback da pilha

Código-fonte: Lib/traceback.py
[https://github.com/python/cpython/tree/3.15/Lib/traceback.py]

This module provides a standard interface to extract, format and print stack traces of Python programs. It is more flexible than the interpreter's default traceback display, and therefore makes it possible to configure certain aspects of the output. Finally, it contains a utility for capturing enough information about an exception to print it later, without the need to save a reference to the actual exception. Since exceptions can be the roots of large objects graph, this utility can significantly improve memory management.

The module uses traceback objects — these are objects of type `types.TracebackType`, which are assigned to the `__traceback__` field of `BaseException` instances.

Ver também

Módulo `faulthandler`
Usado para despejar tracebacks (situação da pilha de execução) do Python explicitamente, em uma falha, após um tempo limite ou em um sinal do usuário.

Module `pdb`
Interactive source code debugger for Python programs.

The module's API can be divided into two parts:

- Module-level functions offering basic functionality, which are useful for interactive inspection of exceptions and tracebacks.

- **`TracebackException`** class and its helper classes **`StackSummary`** and **`FrameSummary`**. These offer both more flexibility in the output generated and the ability to store the information necessary for later formatting without holding references to actual exception and traceback objects.

Adicionado na versão 3.13: A saída é colorizada por padrão e ela pode ser controlada usando variáveis de ambiente.

Funções de nível de módulo

traceback.print_tb(*tb*, *limit=None*, *file=None*)

Print up to *limit* stack trace entries from traceback object *tb* (starting from the caller's frame) if *limit* is positive. Otherwise, print the last `abs(limit)` entries. If *limit* is omitted or `None`, all entries are printed. If *file* is omitted or `None`, the output goes to **`sys.stderr`**; otherwise it should be an open file or file-like object to receive the output.

Nota

The meaning of the *limit* parameter is different than the meaning of **`sys.tracebacklimit`**. A negative *limit* value corresponds to a positive value of **`sys.tracebacklimit`**, whereas the behaviour of a positive *limit* value cannot be achieved with **`sys.tracebacklimit`**.

Alterado na versão 3.5: Added negative *limit* support.

traceback.print_exception(*exc*, */*, [*value*, *tb*,]*limit=None*, *file=None*, *chain=True*)

Print exception information and stack trace entries from traceback object *tb* to *file*. This differs from **`print_tb()`** in the following ways:

- if *tb* is not `None`, it prints a header `Traceback (most recent call last):`
- it prints the exception type and *value* after the stack trace

- if *type(value)* is `SyntaxError` and *value* has the appropriate format, it prints the line where the syntax error occurred with a caret indicating the approximate position of the error.

Since Python 3.10, instead of passing *value* and *tb*, an exception object can be passed as the first argument. If *value* and *tb* are provided, the first argument is ignored in order to provide backwards compatibility.

The optional *limit* argument has the same meaning as for `print_tb()`. If *chain* is true (the default), then chained exceptions (the `__cause__` or `__context__` attributes of the exception) will be printed as well, like the interpreter itself does when printing an unhandled exception.

Alterado na versão 3.5: The *etype* argument is ignored and inferred from the type of *value*.

Alterado na versão 3.10: The *etype* parameter has been renamed to *exc* and is now positional-only.

traceback.print_exc(*limit=None*, *file=None*, *chain=True*)

This is a shorthand for `print_exception(sys.exception(), limit=limit, file=file, chain=chain)`.

traceback.print_last(*limit=None*, *file=None*, *chain=True*)

This is a shorthand for `print_exception(sys.last_exc, limit=limit, file=file, chain=chain)`. In general it will work only after an exception has reached an interactive prompt (see `sys.last_exc`).

traceback.print_stack(*f=None*, *limit=None*, *file=None*)

Print up to *limit* stack trace entries (starting from the invocation point) if *limit* is positive. Otherwise, print the last `abs(limit)` entries. If *limit* is omitted or `None`, all entries are printed. The optional *f* argument can be used to specify an alternate stack frame to start. The optional *file* argument has the same meaning as for `print_tb()`.

Alterado na versão 3.5: Added negative *limit* support.

traceback.extract_tb(*tb*, *limit=None*)

Return a `StackSummary` object representing a list of "pre-processed" stack trace entries extracted from the traceback object *tb*. It is useful for alternate formatting of stack traces. The optional *limit* argument has the same meaning as for `print_tb()`. A "pre-processed" stack trace entry is a `FrameSummary` object with attributes representing the information that is usually printed for a stack trace.

traceback.extract_stack(*f=None*, *limit=None*)

Extract the raw traceback from the current stack frame. The return value has the same format as for `extract_tb()`. The optional *f* and *limit* arguments have the same meaning as for `print_stack()`.

traceback.print_list(*extracted_list*, *file=None*)

Print the list of tuples as returned by `extract_tb()` or `extract_stack()` as a formatted stack trace to the given file. If *file* is `None`, the output is written to `sys.stderr`.

traceback.format_list(*extracted_list*)

Given a list of tuples or `FrameSummary` objects as returned by `extract_tb()` or `extract_stack()`, return a list of strings ready for printing. Each string in the resulting list corresponds to the item with the same index in the argument list. Each string ends in a newline; the strings may contain internal newlines as well, for those items whose source text line is not `None`.

traceback.format_exception_only(*exc*, */*, [*value*,]*, *show_group=False*)

Format the exception part of a traceback using an exception value such as given by `sys.last_exc`. The return value is a list of strings, each ending in a newline. The list contains the exception's message, which is normally a single string; however, for `SyntaxError` exceptions, it contains several lines that (when printed) display detailed information about where the syntax error occurred. Following the message, the list contains the exception's `notes`.

Since Python 3.10, instead of passing *value*, an exception object can be passed as the first argument. If *value* is provided, the first argument is ignored in order to provide backwards compatibility.

When *show_group* is `True`, and the exception is an instance of **`BaseExceptionGroup`**, the nested exceptions are included as well, recursively, with indentation relative to their nesting depth.

Alterado na versão 3.10: The *etype* parameter has been renamed to *exc* and is now positional-only.

Alterado na versão 3.11: The returned list now includes any **`notes`** attached to the exception.

Alterado na versão 3.13: show_group parameter was added.

traceback.format_exception(*exc*, */*, [*value*, *tb*,]*limit=None*, *chain=True*)

Format a stack trace and the exception information. The arguments have the same meaning as the corresponding arguments to **`print_exception()`**. The return value is a list of strings, each ending in a newline and some containing internal newlines. When these lines are concatenated and printed, exactly the same text is printed as does **`print_exception()`**.

Alterado na versão 3.5: The *etype* argument is ignored and inferred from the type of *value*.

Alterado na versão 3.10: This function's behavior and signature were modified to match **`print_exception()`**.

traceback.format_exc(*limit=None*, *chain=True*)

This is like `print_exc(limit)` but returns a string instead of printing to a file.

traceback.format_tb(*tb*, *limit=None*)

A shorthand for `format_list(extract_tb(tb, limit))`.

traceback.format_stack(*f=None*, *limit=None*)

A shorthand for `format_list(extract_stack(f, limit))`.

traceback.clear_frames(*tb*)

Clears the local variables of all the stack frames in a traceback *tb* by calling the **`clear()`** method of each frame object.

Adicionado na versão 3.4.

traceback.walk_stack(*f*)

Walk a stack following **`f.f_back`** from the given frame, yielding the frame and line number for each frame. If *f* is `None`, the current stack is used. This helper is used with **`StackSummary.extract()`**.

Adicionado na versão 3.5.

Alterado na versão 3.14: This function previously returned a generator that would walk the stack when first iterated over. The generator returned now is the state of the stack when `walk_stack` is called.

traceback.walk_tb(*tb*)

Walk a traceback following **`tb_next`** yielding the frame and line number for each frame. This helper is used with **`StackSummary.extract()`**.

Adicionado na versão 3.5.

`TracebackException` Objects

Adicionado na versão 3.5.

`TracebackException` objects are created from actual exceptions to capture data for later printing. They offer a more lightweight method of storing this information by avoiding holding references to traceback and frame objects. In addition, they expose more options to configure the output compared to the module-level functions described above.

class traceback.TracebackException(*exc_type*, *exc_value*, *exc_traceback*, ***, *limit=None*, *lookup_lines=True*, *capture_locals=False*, *compact=False*, *max_group_width=15*, *max_group_depth=10*)

Capture an exception for later rendering. The meaning of *limit*, *lookup_lines* and *capture_locals* are as for the **StackSummary** class.

If *compact* is true, only data that is required by **TracebackException**'s **format()** method is saved in the class attributes. In particular, the **__context__** field is calculated only if **__cause__** is `None` and **__suppress_context__** is false.

Note that when locals are captured, they are also shown in the traceback.

max_group_width and *max_group_depth* control the formatting of exception groups (see **BaseExceptionGroup**). The depth refers to the nesting level of the group, and the width refers to the size of a single exception group's exceptions array. The formatted output is truncated when either limit is exceeded.

Alterado na versão 3.10: Adicionado o parâmetro *compact*.

Alterado na versão 3.11: Added the *max_group_width* and *max_group_depth* parameters.

__cause__

A **TracebackException** of the original **__cause__**.

__context__

A **TracebackException** of the original **__context__**.

exceptions

If `self` represents an **ExceptionGroup**, this field holds a list of **TracebackException** instances representing the nested exceptions. Otherwise it is `None`.

Adicionado na versão 3.11.

__suppress_context__

The `__suppress_context__` value from the original exception.

__notes__

The `__notes__` value from the original exception, or `None` if the exception does not have any notes. If it is not `None` is it formatted in the traceback after the exception string.

Adicionado na versão 3.11.

stack

A `StackSummary` representing the traceback.

exc_type

The class of the original exception.

Descontinuado desde a versão 3.13.

exc_type_str

String display of the class of the original exception.

Adicionado na versão 3.13.

filename

For syntax errors - the file name where the error occurred.

lineno

For syntax errors - the line number where the error occurred.

end_lineno

For syntax errors - the end line number where the error occurred. Can be `None` if not present.

Adicionado na versão 3.10.

text

For syntax errors - the text where the error occurred.

offset

For syntax errors - the offset into the text where the error occurred.

end_offset

For syntax errors - the end offset into the text where the error occurred. Can be `None` if not present.

Adicionado na versão 3.10.

msg

For syntax errors - the compiler error message.

classmethod from_exception(*exc*, *, *limit=None*, *lookup_lines=True*, *capture_locals=False*, *compact=False*, *max_group_width=15*, *max_group_depth=10*)

Capture an exception for later rendering. *limit*, *lookup_lines* and *capture_locals* are as for the `StackSummary` class.

Note that when locals are captured, they are also shown in the traceback.

print(*, *file=None*, *chain=True*)

Print to *file* (default `sys.stderr`) the exception information returned by `format()`.

Adicionado na versão 3.11.

format(*, *chain=True*)

Format the exception.

If *chain* is not `True`, `__cause__` and `__context__` will not be formatted.

The return value is a generator of strings, each ending in a newline and some containing internal newlines. `print_exception()` is a wrapper around this method which just prints the lines to a file.

format_exception_only(*, *show_group=False*)

Format the exception part of the traceback.

The return value is a generator of strings, each ending in a newline.

When *show_group* is `False`, the generator emits the exception's message followed by its notes (if it has any). The exception message is normally a single string; however, for **`SyntaxError`** exceptions, it consists of several lines that (when printed) display detailed information about where the syntax error occurred.

When *show_group* is `True`, and the exception is an instance of **`BaseExceptionGroup`**, the nested exceptions are included as well, recursively, with indentation relative to their nesting depth.

Alterado na versão 3.11: The exception's **`notes`** are now included in the output.

Alterado na versão 3.13: Added the *show_group* parameter.

`StackSummary` Objects

Adicionado na versão 3.5.

`StackSummary` objects represent a call stack ready for formatting.

class traceback.StackSummary

classmethod extract(*frame_gen*, *, *limit=None*, *lookup_lines=True*, *capture_locals=False*)

Construct a **`StackSummary`** object from a frame generator (such as is returned by **`walk_stack()`** or **`walk_tb()`**).

If *limit* is supplied, only this many frames are taken from *frame_gen*. If *lookup_lines* is `False`, the returned **`FrameSummary`** objects will not have read their lines in yet, making the cost of creating the **`StackSummary`** cheaper (which may be valuable if it may not actually get formatted). If *capture_locals* is `True` the local

variables in each **`FrameSummary`** are captured as object representations.

Alterado na versão 3.12: Exceptions raised from **`repr()`** on a local variable (when *capture_locals* is `True`) are no longer propagated to the caller.

classmethod from_list(*a_list*)

Construct a **`StackSummary`** object from a supplied list of **`FrameSummary`** objects or old-style list of tuples. Each tuple should be a 4-tuple with *filename*, *lineno*, *name*, *line* as the elements.

format()

Returns a list of strings ready for printing. Each string in the resulting list corresponds to a single frame from the stack. Each string ends in a newline; the strings may contain internal newlines as well, for those items with source text lines.

For long sequences of the same frame and line, the first few repetitions are shown, followed by a summary line stating the exact number of further repetitions.

Alterado na versão 3.6: Long sequences of repeated frames are now abbreviated.

format_frame_summary(*frame_summary*)

Returns a string for printing one of the frames involved in the stack. This method is called for each **`FrameSummary`** object to be printed by **`StackSummary.format()`**. If it returns `None`, the frame is omitted from the output.

Adicionado na versão 3.11.

`FrameSummary` Objects

Adicionado na versão 3.5.

A **`FrameSummary`** object represents a single frame in a traceback.

class traceback.FrameSummary(*filename*, *lineno*, *name*, ***, *lookup_line=True*, *locals=None*, *line=None*, *end_lineno=None*, *colno=None*, *end_colno=None*)

Represents a single frame in the traceback or stack that is being formatted or printed. It may optionally have a stringified version of the frame's locals included in it. If *lookup_line* is `False`, the source code is not looked up until the **`FrameSummary`** has the **`line`** attribute accessed (which also happens when casting it to a **`tuple`**). **`line`** may be directly provided, and will prevent line lookups happening at all. *locals* is an optional local variable mapping, and if supplied the variable representations are stored in the summary for later display.

`FrameSummary` instances have the following attributes:

filename

The filename of the source code for this frame. Equivalent to accessing **`f.f_code.co_filename`** on a frame object *f*.

lineno

The line number of the source code for this frame.

name

Equivalent to accessing **`f.f_code.co_name`** on a frame object *f*.

line

A string representing the source code for this frame, with leading and trailing whitespace stripped. If the source is not available, it is `None`.

end_lineno

The last line number of the source code for this frame. By default, it is set to `lineno` and indexation starts from 1.

Alterado na versão 3.13: The default value changed from `None` to `lineno`.

colno

The column number of the source code for this frame. By default, it is `None` and indexation starts from 0.

end_colno

The last column number of the source code for this frame. By default, it is `None` and indexation starts from 0.

Examples of Using the Module-Level Functions

This simple example implements a basic read-eval-print loop, similar to (but less useful than) the standard Python interactive interpreter loop. For a more complete implementation of the interpreter loop, refer to the **code** module.

```
import sys, traceback

def run_user_code(envdir):
    source = input(">>> ")
    try:
        exec(source, envdir)
    except Exception:
        print("Exception in user code:")
        print("-"*60)
        traceback.print_exc(file=sys.stdout)
        print("-"*60)

envdir = {}
while True:
    run_user_code(envdir)
```

The following example demonstrates the different ways to print and format the exception and traceback:

```
import sys, traceback

def lumberjack():
    bright_side_of_life()

def bright_side_of_life():
```

```
    return tuple()[0]

try:
    lumberjack()
except IndexError as exc:
    print("*** print_tb:")
    traceback.print_tb(exc.__traceback__, limit=1, file=sys.stdo
    print("*** print_exception:")
    traceback.print_exception(exc, limit=2, file=sys.stdout)
    print("*** print_exc:")
    traceback.print_exc(limit=2, file=sys.stdout)
    print("*** format_exc, first and last line:")
    formatted_lines = traceback.format_exc().splitlines()
    print(formatted_lines[0])
    print(formatted_lines[-1])
    print("*** format_exception:")
    print(repr(traceback.format_exception(exc)))
    print("*** extract_tb:")
    print(repr(traceback.extract_tb(exc.__traceback__)))
    print("*** format_tb:")
    print(repr(traceback.format_tb(exc.__traceback__)))
    print("*** tb_lineno:", exc.__traceback__.tb_lineno)
```

The output for the example would look similar to this:

```
*** print_tb:
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~~~~~~^^
*** print_exception:
Traceback (most recent call last):
  File "<doctest...>", line 10, in <module>
    lumberjack()
    ~~~~~~~~~~^^
  File "<doctest...>", line 4, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~~~^^
IndexError: tuple index out of range
*** print_exc:
Traceback (most recent call last):
  File "<doctest...>", line 10, in <module>
    lumberjack()
```

```
    ~~~~~~~~~~^^
  File "<doctest...>", line 4, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~~^^
IndexError: tuple index out of range
*** format_exc, first and last line:
Traceback (most recent call last):
IndexError: tuple index out of range
*** format_exception:
['Traceback (most recent call last):\n',
 '  File "<doctest default[0]>", line 10, in <module>\n    lumbe
 '  File "<doctest default[0]>", line 4, in lumberjack\n    brig
 '  File "<doctest default[0]>", line 7, in bright_side_of_life\
 'IndexError: tuple index out of range\n']
*** extract_tb:
[<FrameSummary file <doctest...>, line 10 in <module>>,
 <FrameSummary file <doctest...>, line 4 in lumberjack>,
 <FrameSummary file <doctest...>, line 7 in bright_side_of_life>
*** format_tb:
['  File "<doctest default[0]>", line 10, in <module>\n    lumbe
 '  File "<doctest default[0]>", line 4, in lumberjack\n    brig
 '  File "<doctest default[0]>", line 7, in bright_side_of_life\
*** tb_lineno: 10
```

The following example shows the different ways to print and format the stack:

```
>>> import traceback
>>> def another_function():
...     lumberstack()
...
>>> def lumberstack():
...     traceback.print_stack()
...     print(repr(traceback.extract_stack()))
...     print(repr(traceback.format_stack()))
...
>>> another_function()
  File "<doctest>", line 10, in <module>
    another_function()
  File "<doctest>", line 3, in another_function
    lumberstack()
  File "<doctest>", line 6, in lumberstack
```

```
    traceback.print_stack()
[('<doctest>', 10, '<module>', 'another_function()'),
 ('<doctest>', 3, 'another_function', 'lumberstack()'),
 ('<doctest>', 7, 'lumberstack', 'print(repr(traceback.extract_s
['  File "<doctest>", line 10, in <module>\n    another_functior
 '  File "<doctest>", line 3, in another_function\n    lumbersta
 '  File "<doctest>", line 8, in lumberstack\n    print(repr(tra
```

This last example demonstrates the final few formatting functions:

```
>>> import traceback
>>> traceback.format_list([('spam.py', 3, '<module>', 'spam.eggs
...                        ('eggs.py', 42, 'eggs', 'return "bacc
['  File "spam.py", line 3, in <module>\n    spam.eggs()\n',
 '  File "eggs.py", line 42, in eggs\n    return "bacon"\n']
>>> an_error = IndexError('tuple index out of range')
>>> traceback.format_exception_only(an_error)
['IndexError: tuple index out of range\n']
```

Examples of Using `TracebackException`

With the helper class, we have more options:

```
>>> import sys
>>> from traceback import TracebackException
>>>
>>> def lumberjack():
...     bright_side_of_life()
...
>>> def bright_side_of_life():
...     t = "bright", "side", "of", "life"
...     return t[5]
...
>>> try:
...     lumberjack()
... except IndexError as e:
...     exc = e
...
>>> try:
...     try:
```

```
...         lumberjack()
...     except:
...         1/0
... except Exception as e:
...     chained_exc = e
...
>>> # limit works as with the module-level functions
>>> TracebackException.from_exception(exc, limit=-2).print()
Traceback (most recent call last):
  File "<python-input-1>", line 6, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~~^^
  File "<python-input-1>", line 10, in bright_side_of_life
    return t[5]
           ~^^^
IndexError: tuple index out of range

>>> # capture_locals adds local variables in frames
>>> TracebackException.from_exception(exc, limit=-2, capture_loc
Traceback (most recent call last):
  File "<python-input-1>", line 6, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~~^^
  File "<python-input-1>", line 10, in bright_side_of_life
    return t[5]
           ~^^^
    t = ("bright", "side", "of", "life")
IndexError: tuple index out of range

>>> # The *chain* kwarg to print() controls whether chained
>>> # exceptions are displayed
>>> TracebackException.from_exception(chained_exc).print()
Traceback (most recent call last):
  File "<python-input-19>", line 4, in <module>
    lumberjack()
    ~~~~~~~~~~^^
  File "<python-input-8>", line 7, in lumberjack
    bright_side_of_life()
    ~~~~~~~~~~~~~~~~~~~^^
  File "<python-input-8>", line 11, in bright_side_of_life
    return t[5]
           ~^^^
IndexError: tuple index out of range
```

```
During handling of the above exception, another exception occurr

Traceback (most recent call last):
  File "<python-input-19>", line 6, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero

>>> TracebackException.from_exception(chained_exc).print(chain=F
Traceback (most recent call last):
  File "<python-input-19>", line 6, in <module>
    1/0
    ~^~
ZeroDivisionError: division by zero
```

`__future__` — Definições de instruções future

Código-fonte: Lib/__future__.py
[https://github.com/python/cpython/tree/3.15/Lib/__future__.py]

Instruções do tipo `from __future__ import feature` são chamadas de instruções future. Esses são casos especiais do compilador do Python para permitir o uso de novos recursos do Python em módulos que contêm a instrução future antes da versão em que o recurso se torna padrão.

Embora o compilador Python dê um significado especial adicional a essas instruções future, elas ainda são executadas como qualquer outra instrução de importação. O módulo **`__future__`** existe e é tratado pelo sistema de importação da mesma forma que qualquer outro módulo Python. Isso serve a três propósitos:

- Para evitar confundir as ferramentas existentes que analisam as instruções de importação e esperam encontrar os módulos sendo importados.
- Para documentar quando as mudanças incompatíveis foram introduzidas e quando elas serão — ou foram — obrigatórias. Esta é uma forma de documentação executável e pode ser inspecionada programaticamente por meio da importação **`__future__`** e da análise de seus conteúdos.
- Para garantir que instruções future sejam executadas em versões anteriores a Python 2.1, pelo menos, processe exceções de tempo de execução (a importação de **`__future__`** falhará, porque não havia nenhum módulo com esse nome antes de 2.1).

Conteúdo do módulo

Nenhuma descrição de característica será excluída de `__future__`. Desde a sua introdução no Python 2.1, os seguintes recursos encontraram o caminho para o idioma usando esse mecanismo:

característica	opcional em	obrigatório em	efeito
__future__.nested_scopes	2.1.0b1	2.2	**PEP 227**: *Statically Nested Scopes*
__future__.generators	2.2.0a1	2.3	**PEP 255**: *Simple Generators*
__future__.division	2.2.0a2	3.0	**PEP 238**: *Changing the Division Operator*
__future__.absolute_import	2.5.0a1	3.0	**PEP 328**: *Imports: Multi-Line and Absolute/Relative*
__future__.with_statement	2.5.0a1	2.6	**PEP 343**: *The "with" Statement*
__future__.print_function	2.6.0a2	3.0	**PEP 3105**: *Make print a function*

característica	opcional em	obrigatório em	efeito
__future__.unicode_literals	2.6.0a2	3.0	**PEP 3112**: *Bytes literals in Python 3000*
__future__.generator_stop	3.5.0b1	3.7	**PEP 479**: *StopIteration handling inside generators*
__future__.annotations	3.7.0b1	Nunca [[1]]	**PEP 563**: *Avaliação adiada de anotações*, **PEP 649**: *Avaliação diferida de anotações usando descritores*

class __future__._Feature

Cada instrução em `__future__.py` é da forma:

```
FeatureName = _Feature(OptionalRelease, MandatoryRelease,
                       CompilerFlag)
```

Onde, normalmente, *OptionalRelease* é inferior a *MandatoryRelease*, e ambos são tuplas de 5 entradas da mesma forma que **`sys.version_info`**:

```
(PY_MAJOR_VERSION, # o 2 em 2.1.0a3; um int
 PY_MINOR_VERSION, # o 1; um int
 PY_MICRO_VERSION, # o 0; um int
 PY_RELEASE_LEVEL, # "alpha", "beta", "candidate" ou "final";
```

```
 PY_RELEASE_SERIAL # o 3; um int
)
```

_Feature.getOptionalRelease()

OptionalRelease registra o primeiro lançamento no qual o recurso foi aceito.

_Feature.getMandatoryRelease()

No caso de um *MandatoryRelease* que ainda não ocorreu, *MandatoryRelease* prevê o lançamento em que o recurso se tornará parte da linguagem.

Senão *MandatoryRelease* registra quando o recurso se tornou parte do idioma; Em versões em ou depois disso, os módulos não precisam mais de uma instrução future para usar o recurso em questão, mas podem continuar a usar essas importações.

MandatoryRelease também pode ser `None`, o que significa que uma característica planejada foi descartada ou que isso ainda não está decidido.

_Feature.compiler_flag

CompilerFlag é o sinalizador (bitfield) que deve ser passado no quarto argumento para a função embutida **`compile()`** para habilitar o recurso no código compilado dinamicamente. Este sinalizador é armazenado no atributo **`_Feature.compiler_flag`** em instâncias de **`_Feature`**.

[1]

`from __future__ import annotations` estava programado para se tornar obrigatório no Python 3.10, mas a alteração foi adiada e, por fim, cancelada. Este recurso será eventualmente descontinuado e removido. Veja **PEP 649** [https://peps.python.org/pep-0649/] e **PEP 749** [https://peps.python.org/pep-0749/].

Ver também

Instruções future

Como o compilador trata as importações de future.

PEP 236 [https://peps.python.org/pep-0236/] - De volta ao __future__

A proposta original para o mecanismo do __future__.

gc — Interface para o coletor de lixo

Este módulo fornece uma interface para o coletor de lixo opcional. Ele permite desabilitar o coletor, ajustar a frequência de coleta e configurar as opções de depuração. Ele também fornece acesso a objetos inacessíveis que o coletor encontra, mas não pode “limpar”. Como o coletor complementa a contagem de referência já usada em Python, você pode desabilitá-lo se tem certeza de que seu programa não cria ciclos de referência. A coleta automática pode ser desativada pela chamada `gc.disable()`. Para depurar um programa vazando, chame `gc.set_debug(gc.DEBUG_LEAK)`. Perceba que isto inclui `gc.DEBUG_SAVEALL`, fazendo com que os objetos coletados pelo coletor de lixo sejam salvos para inspeção em gc.garbage.

The **gc** module provides the following functions:

gc.enable()

Habilita a coleta de lixo automática.

gc.disable()

Desabilita a coleta de lixo automática.

gc.isenabled()

Retorna `True` se a coleta automática estiver habilitada.

gc.collect(*generation=2*)

Sem argumentos, execute uma coleta completa. O argumento opcional *generation* pode ser um inteiro especificando qual geração coletar (de 0 a 2). Uma exceção `ValueError` é levantada se o número de geração for inválido. A soma dos objetos coletados e dos objetos incoletáveis é retornada.

As listas livres mantidas para vários tipos embutidos são limpas sempre que uma coleta completa ou a da geração mais alta (2) é executada. Nem todos os itens em algumas listas livres podem ser liberados devido à implementação particular, em particular `float`.

O efeito de chamar `gc.collect()` enquanto o interpretador já está realizando uma coleta é indefinido.

Alterado na versão 3.14: `generation=1` executa um incremento de coleta.

Alterado na versão 3.14.5: `generation=1` performs collection of the middle generation.

gc.set_debug(*flags*)

Define os sinalizadores de depuração da coleta de lixo. As informações de depuração serão gravadas em `sys.stderr`. Veja abaixo uma lista de sinalizadores de depuração que podem ser combinados usando operações de bit para controlar a depuração.

gc.get_debug()

Retorna os sinalizadores de depuração atualmente definidos.

gc.get_objects(*generation=None*)

Retorna uma lista de todos os objetos rastreados pelo coletor, excluindo a lista retornada. Se *generation* não for `None`, retorna apenas os objetos rastreados pelo coletor que estão nessa geração.

Alterado na versão 3.8: Novo parâmetro *generation*.

Alterado na versão 3.14: A geração 1 foi removida

Alterado na versão 3.14.5: Generation 1 is reintroduced to maintain GC behavior from 3.13.

Levanta um evento de auditoria `gc.get_objects` com o argumento `generation`.

gc.get_stats()

Retorna uma lista de três dicionários por geração contendo estatísticas de coleta desde o início do interpretador. O número de chaves pode mudar no futuro, mas atualmente cada dicionário conterá os seguintes itens:

- `collections` é o número de vezes que esta geração foi coletada;
- `collected` é o número total de objetos coletados nesta geração;
- `uncollectable` is the total number of objects which were found to be uncollectable (and were therefore moved to the **`garbage`** list) inside this generation;
- `candidates` is the total number of objects in this generation which were considered for collection and traversed;
- `duration` is the total time in seconds spent in collections for this generation.

Adicionado na versão 3.4.

Alterado na versão 3.15: Add `duration` and `candidates`.

gc.set_threshold(*threshold0*[, *threshold1*[, *threshold2*]])

Define os limites de coleta de lixo (a frequência de coleta). Definir *threshold0* como zero desativa a coleta.

O GC classifica os objetos em três gerações, dependendo de quantas varreduras de coleta eles sobreviveram. Novos objetos são colocados na geração mais nova (geração `0`). Se um objeto sobreviver a uma coleção, ele será movido para a próxima geração mais antiga. Como a geração `2` é a geração mais antiga, os objetos dessa geração permanecem lá após uma coleta. Para decidir quando executar, o coletor acompanha o número de alocações e desalocações de objetos desde a última coleta. Quando o número de alocações menos o número de desalocações exceder *threshold0*, a coleta será iniciada. Inicialmente, apenas a geração `0` é examinada. Se a geração `0` foi examinada mais de *threshold1* vezes desde que a geração `1` foi examinada, então a geração `1` também é examinada. Com a terceira geração, as coisas são um pouco mais complicadas, veja Coletando a geração mais antiga

[https://github.com/python/cpython/blob/ff0ef0a54bef26fc507fbf9b7a6009eb7d3f17f5/InternalDocs/garbage_collector.md#collecting-the-oldest-generation] para mais informações.

See Garbage collector design [https://github.com/python/cpython/blob/3.15/InternalDocs/garbage_collector.md] for more information.

Alterado na versão 3.14: threshold2 é ignorado

Alterado na versão 3.14.5: threshold2 is restored to match Python 3.13 behavior.

gc.get_count()

Retorna as contagens da coleta atual como uma tupla de `(count0, count1, count2)`.

gc.get_threshold()

Retorna os limites da coleta atual como uma tupla de `(threshold0, threshold1, threshold2)`.

gc.get_referrers(**objs*)

Retorna a lista de objetos que se referem diretamente a qualquer um dos objs. Esta função localizará apenas os contêineres que suportam coleta de lixo; tipos de extensão que se referem a outros objetos, mas não suportam coleta de lixo, não serão encontrados.

Observe que os objetos que já foram desreferenciados, mas que vivem em ciclos e ainda não foram coletados pelo coletor de lixo podem ser listados entre os referenciadores resultantes. Para obter apenas os objetos atualmente ativos, chame `collect()` antes de chamar `get_referrers()`.

Aviso

Deve-se tomar cuidado ao usar objetos retornados por `get_referrers()` porque alguns deles ainda podem estar em construção e, portanto, em um estado temporariamente inválido. Evite

usar **`get_referrers()`** para qualquer finalidade que não seja depuração.

Levanta um evento de auditoria `gc.get_referrers` com o argumento `objs`.

gc.get_referents(**objs*)

Retorna uma lista de objetos diretamente referenciados por qualquer um dos argumentos. Os referentes retornados são aqueles objetos visitados pelos métodos a nível do C **`tp_traverse`** dos argumentos (se houver), e podem não ser todos os objetos diretamente alcançáveis. Os métodos **`tp_traverse`** são suportados apenas por objetos que suportam coleta de lixo e são necessários apenas para visitar objetos que possam estar envolvidos em um ciclo. Assim, por exemplo, se um número inteiro pode ser acessado diretamente de um argumento, esse objeto inteiro pode ou não aparecer na lista de resultados.

Levanta um evento de auditoria `gc.get_referents` com o argumento `objs`.

gc.is_tracked(*obj*)

Retorna `True` se o objeto está atualmente rastreado pelo coletor de lixo, `False` caso contrário. Como regra geral, as instâncias de tipos atômicos não são rastreadas e as instâncias de tipos não atômicos (contêineres, objetos definidos pelo usuário…) são. No entanto, algumas otimizações específicas do tipo podem estar presentes para suprimir a pegada do coletor de lixo de instâncias simples (por exemplo, dicts contendo apenas chaves e valores atômicos):

```
>>> gc.is_tracked(0)
False
>>> gc.is_tracked("a")
False
>>> gc.is_tracked([])
True
>>> gc.is_tracked({})
False
```

```
>>> gc.is_tracked({"a": 1})
True
```

Adicionado na versão 3.1.

gc.is_finalized(*obj*)

Retorna `True` se o objeto fornecido foi finalizado pelo coletor de lixo, `False` caso contrário.

```
>>> x = None
>>> class Lazarus:
...     def __del__(self):
...         global x
...         x = self
...
>>> lazarus = Lazarus()
>>> gc.is_finalized(lazarus)
False
>>> del lazarus
>>> gc.is_finalized(x)
True
```

Adicionado na versão 3.9.

gc.freeze()

Congela todos os objetos rastreados pelo coletor de lixo; move-os para uma geração permanente e ignora-os em todas as coleções futuras.

Se um processo for `fork()` sem `exec()`, evitar cópia em gravação (copy-on-write) desnecessário em processos filho maximizará o compartilhamento de memória e reduzirá o uso geral de memória. Isso requer evitar a criação de “buracos” liberados nas páginas de memória no processo pai e garantir que as coleções GC nos processos filho não toquem no contador `gc_refs` de objetos de vida longa originados no processo pai. Para realizar ambos, chame `gc.disable()` no início do processo pai, `gc.freeze()` logo antes de `fork()` e `gc.enable()` no início em processos filhos.

Adicionado na versão 3.7.

gc.unfreeze()

Descongela os objetos na geração permanente, coloca-os de volta na geração mais antiga.

Adicionado na versão 3.7.

gc.get_freeze_count()

Retorna o número de objetos na geração permanente.

Adicionado na versão 3.7.

As seguintes variáveis são fornecidas para acesso somente leitura (você pode alterar os valores, mas não deve revinculá-los):

gc.garbage

Uma lista de objetos que o coletor considerou inacessíveis, mas não puderam ser liberados (objetos não coletáveis). A partir do Python 3.4, esta lista deve estar vazia na maioria das vezes, exceto ao usar instâncias de tipos de extensão C com um slot `tp_del` não-`NULL`.

Se **`DEBUG_SAVEALL`** for definido, todos os objetos inacessíveis serão adicionados a esta lista ao invés de liberados.

Alterado na versão 3.2: Se esta lista não estiver vazia no desligamento do interpretador, um **`ResourceWarning`** é emitido, que é silencioso por padrão. Se **`DEBUG_UNCOLLECTABLE`** for definido, além disso, todos os objetos não coletáveis serão impressos.

Alterado na versão 3.4: Seguindo a **PEP 442** [https://peps.python.org/pep-0442/], objetos com um método **`__del__()`** não vão mais para **`gc.garbage`**.

gc.callbacks

Uma lista de retornos de chamada que serão invocados pelo coletor de lixo antes e depois da coleta. As funções de retorno serão chamadas com dois argumentos, *phase* e *info*.

phase pode ser um dos dois valores:

“start”: A coleta de lixo está prestes a começar.

“stop”: A coleta de lixo terminou.

info é um ditado que fornece mais informações para a função de retorno. As seguintes chaves estão atualmente definidas:

“generation”: A geração mais antiga sendo coletada.

“collected”: Quando *phase* é “stop”, o número de objetos coletados com sucesso.

“uncollectable”: Quando *phase* é “stop”, o número de objetos que não puderam ser coletados e foram colocados em `garbage`.

“candidates”: When *phase* is “stop”, the total number of objects in this generation which were considered for collection and traversed.

“duration”: When *phase* is “stop”, the time in seconds spent in the collection.

As aplicações podem adicionar suas próprias funções de retorno a essa lista. Os principais casos de uso são:

Reunir estatísticas sobre coleta de lixo, como com que frequência várias gerações são coletadas e quanto tempo leva a coleta.

Permitindo que os aplicativos identifiquem e limpem seus próprios tipos não colecionáveis quando eles aparecem em `garbage`.

Adicionado na versão 3.3.

Alterado na versão 3.15: Add “duration” and “candidates”.

As seguintes constantes são fornecidas para uso com `set_debug()`:

gc.DEBUG_STATS

Imprimir estatísticas durante a coleta. Esta informação pode ser útil ao sintonizar a frequência de coleta.

gc.DEBUG_COLLECTABLE

Imprimir informações sobre objetos colecionáveis encontrados.

gc.DEBUG_UNCOLLECTABLE

Imprime informações de objetos não colecionáveis encontrados (objetos que não são alcançáveis, mas não podem ser liberados pelo coletor). Esses objetos serão adicionados à lista `garbage`.

Alterado na versão 3.2: Imprime também o conteúdo da lista **`garbage`** em desligamento do interpretador, se não estiver vazia.

gc.DEBUG_SAVEALL

Quando definido, todos os objetos inacessíveis encontrados serão anexados ao *lixo* em vez de serem liberados. Isso pode ser útil para depurar um programa com vazamento.

gc.DEBUG_LEAK

Os sinalizadores de depuração necessários para o coletor imprimir informações sobre um programa com vazamento (igual a `DEBUG_COLLECTABLE | DEBUG_UNCOLLECTABLE | DEBUG_SAVEALL`).

`inspect` — Inspecione objetos vivos

Código-fonte: Lib/inspect.py [https://github.com/python/cpython/tree/3.15/Lib/inspect.py]

The `inspect` module provides several useful functions to help get information about live objects such as modules, classes, methods, functions, tracebacks, frame objects, and code objects. For example, it can help you examine the contents of a class, retrieve the source code of a method, extract and format the argument list for a function, or get all the information you need to display a detailed traceback.

Há quatro tipos principais de serviços fornecidos por este módulo: verificação de tipos, obtenção de código-fonte, inspeção de classes e funções e exame da pilha do interpretador.

Tipos e membros

A função `getmembers()` recupera os membros de um objeto, como uma classe ou módulo. As funções cujos nomes começam com “is” são fornecidas principalmente como opções convenientes para o segundo argumento de `getmembers()`. Elas também ajudam a determinar quando você pode esperar encontrar os seguintes atributos especiais (consulte Atributos relacionados à importação em objetos de módulo para atributos de módulo):

Tipo	Atributo	Descrição
class	__doc__	string de documentação
	__name__	nome com o qual esta classe foi definida
	__qualname__	nome qualificado
	__module__	nome do módulo no qual esta classe foi definida

Tipo	Atributo	Descrição
	__type_params__	Uma tupla contendo os parâmetros de tipo de uma classe genérica
método	__doc__	string de documentação
	__name__	nome com o qual este método foi definido
	__qualname__	nome qualificado
	__func__	objeto função contendo implementação de método
	__self__	instância para o qual este método está vinculado, ou `None`.
	__module__	nome do módulo no qual este método foi definido
função	__doc__	string de documentação
	__name__	nome com o qual esta função foi definida
	__qualname__	nome qualificado
	__code__	objeto código contendo bytecode da função compilada

Tipo	Atributo	Descrição
	__defaults__	tupla de quaisquer valores padrão para parâmetros posicionais ou de parâmetros nomeados
	__kwdefaults__	mapeamento de quaisquer valores padrão para parâmetros somente-nomeados
	__globals__	espaço de nomes global no qual esta função foi definida
	__builtins__	espaço de nomes de métodos e funções embutidas
	__annotations__	mapeamento de nomes de parâmetros para anotações; a chave `"return"` é reservada para anotações de retorno.
	__type_params__	Uma tupla contendo os parâmetros de tipo de uma função genérica
	__module__	nome do módulo no qual esta função foi definida
traceback	tb_frame	objeto quadro neste nível
	tb_lasti	índice da última instrução tentada no bytecode
	tb_lineno	número da linha atual no código-fonte do Python

Tipo	Atributo	Descrição
	tb_next	próximo objeto de traceback interno (chamado por este nível)
quadro	f_back	próximo objeto de quadro externo (chamador deste quadro)
	f_builtins	espaço de nomes de métodos e funçõess embutidas visto por este quadro
	f_code	objeto código sendo executado neste quadro
	f_globals	espaço de nomes global visto por este quadro
	f_lasti	índice da última instrução tentada no bytecode
	f_lineno	número da linha atual no código-fonte do Python
	f_locals	espaço de nomes local visto por este quadro
	f_generator	retorna o gerador ou objeto de corrotina que possui este quadro, ou `None` se o quadro for de uma função regular
	f_trace	função de rastreamento para este quadro, ou `None`

Tipo	Atributo	Descrição
	f_trace_lines	indica se um evento de rastreamento é acionado para cada linha de origem
	f_trace_opcodes	indica se eventos por código de operação são solicitados
	clear()	usado para limpar todas as referências a variáveis locais
código	co_argcount	número de argumentos (não incluindo argumentos somente-nomeados, args * ou **)
	co_code	string de bytecode compilado bruto
	co_cellvars	tuple of names of cell variables (referenced by containing scopes)
	co_consts	tuple of constants used in the bytecode
	co_filename	name of file in which this code object was created
	co_firstlineno	number of first line in Python source code
	co_flags	bitmap of `co_*` flags, read more here

Tipo	Atributo	Descrição
	co_freevars	tuple of names of free variables (referenced via a function’s closure)
	co_posonlyargcount	number of positional only arguments
	co_kwonlyargcount	number of keyword only arguments (not including ** arg)
	co_name	name with which this code object was defined
	co_qualname	fully qualified name with which this code object was defined
	co_names	tuple of names other than arguments and function locals
	co_nlocals	number of local variables
	co_stacksize	virtual machine stack space required
	co_varnames	tuple of names of arguments and local variables
	co_lines()	returns an iterator that yields successive bytecode ranges
	co_positions()	returns an iterator of source code positions for each bytecode instruction

Tipo	Atributo	Descrição
	replace()	returns a copy of the code object with new values
gerador	__name__	nome
	__qualname__	nome qualificado
	gi_frame	quadro
	gi_running	is the generator running?
	gi_suspended	is the generator suspended?
	gi_code	código
	gi_yieldfrom	object being iterated by `yield from`, or `None`
	gi_state	state of the generator, one of `GEN_CREATED`, `GEN_RUNNING`, `GEN_SUSPENDED`, or `GEN_CLOSED`
async generator	__name__	nome
	__qualname__	nome qualificado
	ag_await	object being awaited on, or `None`

Tipo	Atributo	Descrição
	ag_frame	quadro
	ag_running	is the generator running?
	ag_suspended	is the generator suspended?
	ag_code	código
	ag_state	state of the async generator, one of `AGEN_CREATED`, `AGEN_RUNNING`, `AGEN_SUSPENDED`, or `AGEN_CLOSED`
corrotina	__name__	nome
	__qualname__	nome qualificado
	cr_await	object being awaited on, or `None`
	cr_frame	quadro
	cr_running	is the coroutine running?
	cr_suspended	is the coroutine suspended?
	cr_code	código

Tipo	Atributo	Descrição
	cr_origin	where coroutine was created, or `None`. See `sys.set_coroutine_origin_tracking_depth()`
	cr_state	state of the coroutine, one of `CORO_CREATED`, `CORO_RUNNING`, `CORO_SUSPENDED`, or `CORO_CLOSED`
builtin	__doc__	string de documentação
	__name__	original name of this function or method
	__qualname__	nome qualificado
	__self__	instance to which a method is bound, or `None`

Alterado na versão 3.5: Add `__qualname__` and `gi_yieldfrom` attributes to generators.

The `__name__` attribute of generators is now set from the function name, instead of the code name, and it can now be modified.

Alterado na versão 3.7: Add `cr_origin` attribute to coroutines.

Alterado na versão 3.10: Add `__builtins__` attribute to functions.

Alterado na versão 3.11: Add `gi_suspended` attribute to generators.

Alterado na versão 3.11: Add `cr_suspended` attribute to coroutines.

Alterado na versão 3.12: Add `ag_suspended` attribute to async generators.

Alterado na versão 3.14: Add `f_generator` attribute to frames.

Alterado na versão 3.15: Add `gi_state` attribute to generators, `cr_state` attribute to coroutines, and `ag_state` attribute to async generators.

inspect.getmembers(*object*[, *predicate*])

Return all the members of an object in a list of `(name, value)` pairs sorted by name. If the optional *predicate* argument—which will be called with the `value` object of each member—is supplied, only members for which the predicate returns a true value are included.

Nota

`getmembers()` will only return class attributes defined in the metaclass when the argument is a class and those attributes have been listed in the metaclass' custom **`__dir__()`**.

inspect.getmembers_static(*object*[, *predicate*])

Return all the members of an object in a list of `(name, value)` pairs sorted by name without triggering dynamic lookup via the descriptor protocol, __getattr__ or __getattribute__. Optionally, only return members that satisfy a given predicate.

Nota

`getmembers_static()` may not be able to retrieve all members that getmembers can fetch (like dynamically created attributes) and may find members that getmembers can't (like descriptors that raise AttributeError). It can also return descriptor objects instead of instance members in some cases.

Adicionado na versão 3.11.

inspect.getmodulename(*path*)

Return the name of the module named by the file *path*, without including the names of enclosing packages. The file extension is checked against all of the entries in **`importlib.machinery.all_suffixes()`**. If it matches, the final path component is returned with the extension removed. Otherwise, `None` is returned.

Note that this function *only* returns a meaningful name for actual Python modules - paths that potentially refer to Python packages will still return `None`.

Alterado na versão 3.3: The function is based directly on **`importlib`**.

inspect.ismodule(*object*)

Return `True` if the object is a module.

inspect.isclass(*object*)

Return `True` if the object is a class, whether built-in or created in Python code.

This function returns `False` for generic aliases of classes, such as `list[int]`.

inspect.ismethod(*object*)

Return `True` if the object is a bound method written in Python.

Nota

For example, given this class:

```
>>> class Greeter:
...     def say_hello(self):
...         print('hello!')
```

A bound method (also known as an *instance method*) is created when accessing `say_hello` (a function defined in the `Greeter` namespace) through an instance of the `Greeter` class:

```
>>> instance = Greeter()

>>> instance.say_hello
<bound method Greeter.say_hello of <__main__.Greeter object ...>>
>>> ismethod(instance.say_hello)
True
>>> isfunction(instance.say_hello)
False
```

Accessing `say_hello` through the `Greeter` class will return the function itself. For this function, **`ismethod()`** will return `False`, but **`isfunction()`** will return `True`:

```
>>> Greeter.say_hello
<function Greeter.say_hello at 0x7f7503854a90>
>>> ismethod(Greeter.say_hello)
False
```

```
>>> isfunction(Greeter.say_hello)
True
```

See Métodos for details.

inspect.isfunction(*object*)

Return `True` if the object is a Python function, which includes functions created by a lambda expression.

See the note for **`ismethod()`** for an example.

inspect.ispackage(*object*)

Return `True` if the object is a package.

Adicionado na versão 3.14.

inspect.isgeneratorfunction(*object*)

Return `True` if the object is a Python generator function.

It also returns `True` for bound methods created from Python generator functions (see Métodos for more information).

Alterado na versão 3.8: Functions wrapped in **`functools.partial()`** now return `True` if the wrapped function is a Python generator function.

Alterado na versão 3.10.6: Duck-typed function-like objects now return `True` if their code object has the **`CO_GENERATOR`** flag.

Alterado na versão 3.13: Functions wrapped in **`functools.partialmethod()`** now return `True` if the wrapped function is a Python generator function.

inspect.isgenerator(*object*)

Return `True` if the object is a generator.

inspect.iscoroutinefunction(*object*)

Return `True` if the object is a coroutine function (a function defined with an **`async def`** syntax), a **`functools.partial()`** wrapping a coroutine function, or a sync function marked with **`markcoroutinefunction()`**.

Adicionado na versão 3.5.

Alterado na versão 3.8: Functions wrapped in **`functools.partial()`** now return `True` if the wrapped function is a coroutine function.

Alterado na versão 3.10.6: Duck-typed function-like objects now return `True` if their code object has the **`CO_COROUTINE`** flag.

Alterado na versão 3.12: Sync functions marked with **`markcoroutinefunction()`** now return `True`.

Alterado na versão 3.13: Functions wrapped in **`functools.partialmethod()`** now return `True` if the wrapped function is a coroutine function.

inspect.markcoroutinefunction(*func*)

Decorator to mark a callable as a coroutine function if it would not otherwise be detected by **`iscoroutinefunction()`**.

This may be of use for sync functions that return a coroutine, if the function is passed to an API that requires **`iscoroutinefunction()`**.

When possible, using an **`async def`** function is preferred. Also acceptable is calling the function and testing the return with **`iscoroutine()`**.

Adicionado na versão 3.12.

inspect.iscoroutine(*object*)

Return `True` if the object is a coroutine created by an **`async def`** function.

Adicionado na versão 3.5.

inspect.isawaitable(*object*)

Return `True` if the object can be used in **`await`** expression.

Can also be used to distinguish generator-based coroutines from regular generators:

```
import types

def gen():
    yield
@types.coroutine
def gen_coro():
    yield
```

```
assert not isawaitable(gen())
assert isawaitable(gen_coro())
```

Adicionado na versão 3.5.

inspect.isasyncgenfunction(*object*)

Return `True` if the object is an asynchronous generator function, for example:

```
>>> async def agen():
...     yield 1
...
>>> inspect.isasyncgenfunction(agen)
True
```

Adicionado na versão 3.6.

Alterado na versão 3.8: Functions wrapped in **`functools.partial()`** now return `True` if the wrapped function is an asynchronous generator function.

Alterado na versão 3.10.6: Duck-typed function-like objects now return `True` if their code object has the **`CO_ASYNC_GENERATOR`** flag.

Alterado na versão 3.13: Functions wrapped in **`functools.partialmethod()`** now return `True` if the wrapped function is a asynchronous generator function.

inspect.isasyncgen(*object*)

Return `True` if the object is an asynchronous generator iterator created by an asynchronous generator function.

Adicionado na versão 3.6.

inspect.istraceback(*object*)

Return `True` if the object is a traceback.

inspect.isframe(*object*)

Return `True` if the object is a frame.

inspect.iscode(*object*)

Return `True` if the object is a code.

inspect.isbuiltin(*object*)

Return `True` if the object is a built-in function or a bound built-in method.

inspect.ismethodwrapper(*object*)

Return `True` if the type of object is a `MethodWrapperType`.

These are instances of `MethodWrapperType`, such as `__str__()`, `__eq__()` and `__repr__()`.

Adicionado na versão 3.11.

inspect.isroutine(*object*)

Return `True` if the object is a user-defined or built-in function or method.

inspect.isabstract(*object*)

Return `True` if the object is an abstract base class.

inspect.ismethoddescriptor(*object*)

Return `True` if the object is a method descriptor, but not if `isclass()`, `ismethod()` or `isfunction()` is true.

This, for example, is true of `int.__add__`. An object passing this test has a `__get__()` method, but not a `__set__()` method or a `__delete__()` method. Beyond that, the set of attributes varies. A `__name__` attribute is usually sensible, and `__doc__` often is.

Method descriptors that also pass any of the other tests (`isclass()`, `ismethod()` or `isfunction()`) make this function return `False`, simply because those other tests promise more – you can, for example, count on having the `__func__` attribute when an object passes `ismethod()`.

Alterado na versão 3.13: This function no longer incorrectly reports objects with `__get__()` and `__delete__()`, but not `__set__()`, as being method descriptors (such objects are data descriptors, not method descriptors).

inspect.isdatadescriptor(*object*)

Return `True` if the object is a data descriptor, but not if `isclass()`, `ismethod()` or `isfunction()` is true.

Data descriptors always have a `__set__()` method and/or a `__delete__()` method. Optionally, they may also have a `__get__()` method.

Examples of data descriptors are `properties`, getsets and member descriptors. Note that for the latter two (defined only in C extension modules), more specific tests are available: `isgetsetdescriptor()` and `ismemberdescriptor()`, respectively.

While data descriptors may also have `__name__` and `__doc__` attributes (as properties, getsets and member descriptors do), this is not necessarily the case in general.

Alterado na versão 3.8: This function now reports objects with only a `__set__()` method as being data descriptors (the presence of `__get__()` is no longer required for that). Moreover, objects with `__delete__()`, but not `__set__()`, are now properly recognized as data descriptors as well, which was not the case previously.

inspect.isgetsetdescriptor(*object*)

Return `True` if the object is a getset descriptor.

Detalhes da implementação do CPython: getsets are attributes defined in extension modules via `PyGetSetDef` structures. For Python implementations without such types, this method will always return `False`.

inspect.ismemberdescriptor(*object*)

Return `True` if the object is a member descriptor.

Detalhes da implementação do CPython: Member descriptors are attributes defined in extension modules via `PyMemberDef` structures. For Python implementations without such types, this method will always return `False`.

Retrieving source code

inspect.getdoc(*object*, *, *inherit_class_doc=True*, *fallback_to_class_doc=True*)

Get the documentation string for an object, cleaned up with `cleandoc()`. If the documentation string for an object is not provided:

- if the object is a class and *inherit_class_doc* is true (by default), retrieve the documentation string from the inheritance hierarchy;
- if the object is a method, a property or a descriptor, retrieve the documentation string from the inheritance hierarchy;

- otherwise, if *fallback_to_class_doc* is true (by default), retrieve the documentation string from the class of the object.

Return `None` if the documentation string is invalid or missing.

Alterado na versão 3.5: Strings de documentação agora são herdadas, se não forem sobrescritas.

Alterado na versão 3.15: Added parameters *inherit_class_doc* and *fallback_to_class_doc*.

Documentation strings on `cached_property` objects are now inherited if not overridden.

inspect.getcomments(*object*)

Return in a single string any lines of comments immediately preceding the object's source code (for a class, function, or method), or at the top of the Python source file (if the object is a module). If the object's source code is unavailable, return `None`. This could happen if the object has been defined in C or the interactive shell.

inspect.getfile(*object*)

Return the name of the (text or binary) file in which an object was defined. An `OSError` is raised if the source code cannot be retrieved. This will fail with a `TypeError` if the object is a built-in module, class, or function.

inspect.getmodule(*object*)

Try to guess which module an object was defined in. Return `None` if the module cannot be determined.

inspect.getsourcefile(*object*)

Return the name of the Python source file in which an object was defined or `None` if no way can be identified to get the source. An `OSError` is raised if the source code cannot be retrieved. This will fail with a `TypeError` if the object is a built-in module, class, or function.

inspect.getsourcelines(*object*)

Return a list of source lines and starting line number for an object. The argument may be a module, class, method, function, traceback, frame, or code object. The source code is returned as a list of the lines corresponding to the

object and the line number indicates where in the original source file the first line of code was found. An `OSError` is raised if the source code cannot be retrieved. A `TypeError` is raised if the object is a built-in module, class, or function.

Alterado na versão 3.3: `OSError` is raised instead of `IOError`, now an alias of the former.

inspect.getsource(*object*)

Return the text of the source code for an object. The argument may be a module, class, method, function, traceback, frame, or code object. The source code is returned as a single string. An `OSError` is raised if the source code cannot be retrieved. A `TypeError` is raised if the object is a built-in module, class, or function.

Alterado na versão 3.3: `OSError` is raised instead of `IOError`, now an alias of the former.

inspect.cleandoc(*doc*)

Clean up indentation from docstrings that are indented to line up with blocks of code.

All leading whitespace is removed from the first line. Any leading whitespace that can be uniformly removed from the second line onwards is removed. Empty lines at the beginning and end are subsequently removed. Also, all tabs are expanded to spaces.

Introspecting callables with the Signature object

Adicionado na versão 3.3.

The `Signature` object represents the call signature of a callable object and its return annotation. To retrieve a `Signature` object, use the `signature()` function.

inspect.signature(*callable*, *, *follow_wrapped=True*, *globals=None*, *locals=None*, *eval_str=False*, *annotation_format=Format.VALUE*)

Return a `Signature` object for the given *callable*:

```
>>> from inspect import signature
>>> def foo(a, *, b:int, **kwargs):
```

```
...     pass

>>> sig = signature(foo)

>>> str(sig)
'(a, *, b: int, **kwargs)'

>>> str(sig.parameters['b'])
'b: int'

>>> sig.parameters['b'].annotation
<class 'int'>
```

Accepts a wide range of Python callables, from plain functions and classes to **functools.partial()** objects.

If some of the annotations are strings (e.g., because `from __future__ import annotations` was used), **signature()** will attempt to automatically un-stringize the annotations using **annotationlib.get_annotations()**. The *globals*, *locals*, and *eval_str* parameters are passed into **annotationlib.get_annotations()** when resolving the annotations; see the documentation for **annotationlib.get_annotations()** for instructions on how to use these parameters. A member of the **annotationlib.Format** enum can be passed to the *annotation_format* parameter to control the format of the returned annotations. For example, use `annotation_format=annotationlib.Format.STRING` to return annotations in string format.

Raises **ValueError** if no signature can be provided, and **TypeError** if that type of object is not supported. Also, if the annotations are stringized, and *eval_str* is not false, the `eval()` call(s) to un-stringize the annotations in **annotationlib.get_annotations()** could potentially raise any kind of exception.

A slash (/) in the signature of a function denotes that the parameters prior to it are positional-only. For more info, see the FAQ entry on positional-only parameters.

Alterado na versão 3.5: The *follow_wrapped* parameter was added. Pass `False` to get a signature of *callable* specifically (`callable.__wrapped__` will not be used to unwrap decorated callables.)

Alterado na versão 3.10: The *globals*, *locals*, and *eval_str* parameters were added.

Alterado na versão 3.14: The *annotation_format* parameter was added.

Nota

Some callables may not be introspectable in certain implementations of Python. For example, in CPython, some built-in functions defined in C provide no metadata about their arguments.

Detalhes da implementação do CPython: If the passed object has a `__signature__` attribute, we may use it to create the signature. The exact semantics are an implementation detail and are subject to unannounced changes. Consult the source code for current semantics.

class inspect.Signature(*parameters=None*, ***, *return_annotation=Signature.empty*)

A `Signature` object represents the call signature of a function and its return annotation. For each parameter accepted by the function it stores a `Parameter` object in its `parameters` collection.

The optional *parameters* argument is a sequence of `Parameter` objects, which is validated to check that there are no parameters with duplicate names, and that the parameters are in the right order, i.e. positional-only first, then positional-or-keyword, and that parameters with defaults follow parameters without defaults.

The optional *return_annotation* argument can be an arbitrary Python object. It represents the "return" annotation of the callable.

`Signature` objects are *immutable*. Use `Signature.replace()` or `copy.replace()` to make a modified copy.

Alterado na versão 3.5: `Signature` objects are now picklable and hashable.

empty

A special class-level marker to specify absence of a return annotation.

parameters

An ordered mapping of parameters' names to the corresponding `Parameter` objects. Parameters appear in strict definition order, including keyword-

only parameters.

Alterado na versão 3.7: Python only explicitly guaranteed that it preserved the declaration order of keyword-only parameters as of version 3.7, although in practice this order had always been preserved in Python 3.

return_annotation

The "return" annotation for the callable. If the callable has no "return" annotation, this attribute is set to `Signature.empty`.

bind(**args*, ***kwargs*)

Create a mapping from positional and keyword arguments to parameters. Returns `BoundArguments` if `*args` and `**kwargs` match the signature, or raises a `TypeError`.

bind_partial(**args*, ***kwargs*)

Works the same way as `Signature.bind()`, but allows the omission of some required arguments (mimics `functools.partial()` behavior.) Returns `BoundArguments`, or raises a `TypeError` if the passed arguments do not match the signature.

replace(**[, parameters][, return_annotation]*)

Create a new **`Signature`** instance based on the instance **`replace()`** was invoked on. It is possible to pass different *parameters* and/or *return_annotation* to override the corresponding properties of the base signature. To remove `return_annotation` from the copied **`Signature`**, pass in `Signature.empty`.

```
>>> def test(a, b):
...     pass
...
>>> sig = signature(test)
>>> new_sig = sig.replace(return_annotation="new return anno")
>>> str(new_sig)
"(a, b) -> 'new return anno'"
```

`Signature` objects are also supported by the generic function `copy.replace()`.

format(**, max_width=None, quote_annotation_strings=True*)

Create a string representation of the **`Signature`** object.

If *max_width* is passed, the method will attempt to fit the signature into lines of at most *max_width* characters. If the signature is longer than *max_width*, all parameters will be on separate lines.

If *quote_annotation_strings* is False, annotations in the signature are displayed without opening and closing quotation marks if they are strings. This is useful if the signature was created with the **`STRING`** format or if `from __future__ import annotations` was used.

Adicionado na versão 3.13.

Alterado na versão 3.14: The *unquote_annotations* parameter was added.

classmethod from_callable(*obj*, **, follow_wrapped=True, globals=None, locals=None, eval_str=False*)

Return a **`Signature`** (or its subclass) object for a given callable *obj*.

This method simplifies subclassing of **`Signature`**:

```
class MySignature(Signature):
    pass
sig = MySignature.from_callable(sum)
assert isinstance(sig, MySignature)
```

Its behavior is otherwise identical to that of **`signature()`**.

Adicionado na versão 3.5.

Alterado na versão 3.10: The *globals*, *locals*, and *eval_str* parameters were added.

class inspect.Parameter(*name*, *kind*, **, default=Parameter.empty, annotation=Parameter.empty*)

Objetos **`Parameter`** são *imutáveis*. Em vez de modificar um objeto **`Parameter`**, você pode usar **`Parameter.replace()`** ou **`copy.replace()`** para criar uma cópia modificada.

Alterado na versão 3.5: Objetos Parameter agora são compatíveis com pickle e hasheável.

empty

Um marcador de nível de classe especial para especificar a ausência de valor padrão e anotação.

name

O nome do parâmetro como uma string. O nome deve ser um identificador Python válido.

CPython gera nomes de parâmetro implícito na forma `.0` no código objeto usado para implementar compreensão e expressão geradora.

Alterado na versão 3.6: Esses nomes de parâmetro agora são expostos por este módulo como nomes como `implicit0`.

default

O padrão valor para o parâmetro. Se o parâmetro não tiver padrão valor, esse atributo será definido como **`Parameter.empty`**.

annotation

A anotação para o parâmetro. Se o parâmetro não tiver anotação, esse atributo será definido como **`Parameter.empty`**.

kind

Descreve como valores de argumento são vinculados ao parâmetro. Os possíveis valores são acessíveis via **`Parameter`** (como `Parameter.KEYWORD_ONLY`) e tem suporte para comparação e ordenação, na seguinte ordem:

Nome	**Significado**
POSITIONAL_ONLY	Valor deve ser fornecido como argumento posicional. Parâmetros somente-posicionais são aqueles que aparecem antes de uma entrada `/` (se presente) em uma definição de função Python.

Nome	Significado
POSITIONAL_OR_KEYWORD	Valor pode ser fornecido como argumento posicional ou nomeado (esse é o comportamento de ligação padrão para funções implementadas em Python).
`*VAR_POSITIONAL*`	Uma tupla de argumentos posicionais que não estão vinculados a nenhum outro parâmetro. Isso corresponde a um parâmetro `*args` em uma definição de função Python.
KEYWORD_ONLY	Valor deve ser fornecido como argumento nomeado. Parâmetros somente-nomeados são aqueles que aparecem após uma entrada `*` ou `*args` em uma definição de função Python.
VAR_KEYWORD	Um dicionário de argumentos nomeados que não estão vinculados a nenhum outro parâmetro. Isso corresponde a um parâmetro `**kwargs` em uma definição de função Python.

Exemplo: exibir todos argumentos somente-nomeados sem padrão valor:

```
>>> def foo(a, b, *, c, d=10):
...     pass

>>> sig = signature(foo)
>>> for param in sig.parameters.values():
...     if (param.kind == param.KEYWORD_ONLY and
...                        param.default is param.empty):
...         print('Parameter:', param)
Parameter: c
```

kind.description

Descreve um valor enumerado de `Parameter.kind`.

Adicionado na versão 3.8.

Exemplo: exibir as descrições de todos os argumento:

```
>>> def foo(a, b, *, c, d=10):
...     pass

>>> sig = signature(foo)
>>> for param in sig.parameters.values():
...     print(param.kind.description)
positional or keyword
positional or keyword
keyword-only
keyword-only
```

replace(**[, name][, kind][, default][, annotation]*)

Cria uma nova instância de **Parameter** com base na instância em que o método foi chamado. Para substituir um atributo de **Parameter**, passe o argumento correspondente. Para remover um valor padrão ou/e uma anotação de um **Parameter**, passe **Parameter.empty**.

```
>>> from inspect import Parameter
>>> param = Parameter('foo', Parameter.KEYWORD_ONLY, default=42
>>> str(param)
'foo=42'

>>> str(param.replace()) # Criará uma cópia rasa de 'param'
'foo=42'

>>> str(param.replace(default=Parameter.empty, annotation='spam
"foo: 'spam'"
```

Objetos **Parameter** também são suportados pela função genérica **copy.replace()**.

Alterado na versão 3.4: Em Python 3.3, era permitido objetos **Parameter** terem `name` definido como `None` se `kind` estivesse definido como `POSITIONAL_ONLY`. Isso não é mais permitido.

class inspect.BoundArguments

Result of a **Signature.bind()** or **Signature.bind_partial()** call. Holds the mapping of arguments to the function's parameters.

arguments

A mutable mapping of parameters' names to arguments' values. Contains only explicitly bound arguments. Changes in `arguments` will reflect in `args` and `kwargs`.

Should be used in conjunction with `Signature.parameters` for any argument processing purposes.

> **Nota**
>
> Arguments for which `Signature.bind()` or `Signature.bind_partial()` relied on a default value are skipped. However, if needed, use `BoundArguments.apply_defaults()` to add them.

Alterado na versão 3.9: `arguments` is now of type `dict`. Formerly, it was of type `collections.OrderedDict`.

args

A tuple of positional arguments values. Dynamically computed from the `arguments` attribute.

kwargs

A dict of keyword arguments values. Dynamically computed from the `arguments` attribute. Arguments that can be passed positionally are included in `args` instead.

signature

A reference to the parent `Signature` object.

apply_defaults()

Set default values for missing arguments.

For variable-positional arguments (`*args`) the default is an empty tuple.

For variable-keyword arguments (`**kwargs`) the default is an empty dict.

```
>>> def foo(a, b='ham', *args): pass
>>> ba = inspect.signature(foo).bind('spam')
>>> ba.apply_defaults()
>>> ba.arguments
{'a': 'spam', 'b': 'ham', 'args': ()}
```

Adicionado na versão 3.5.

The **`args`** and **`kwargs`** properties can be used to invoke functions:

```
def test(a, *, b):
    ...

sig = signature(test)
ba = sig.bind(10, b=20)
test(*ba.args, **ba.kwargs)
```

Ver também

PEP 362 [https://peps.python.org/pep-0362/] - Function Signature Object.
The detailed specification, implementation details and examples.

Classes e funções

inspect.getclasstree(*classes*, *unique=False*)

Arrange the given list of classes into a hierarchy of nested lists. Where a nested list appears, it contains classes derived from the class whose entry immediately precedes the list. Each entry is a 2-tuple containing a class and a tuple of its base classes. If the *unique* argument is true, exactly one entry appears in the returned structure for each class in the given list. Otherwise, classes using multiple inheritance and their descendants will appear multiple times.

inspect.getfullargspec(*func*, *, *annotation_format=Format.VALUE*)

Get the names and default values of a Python function's parameters. A named tuple is returned:

```
FullArgSpec(args, varargs, varkw, defaults, kwonlyargs,
kwonlydefaults, annotations)
```

args is a list of the positional parameter names. *varargs* is the name of the `*` parameter or `None` if arbitrary positional arguments are not accepted. *varkw* is the name of the `**` parameter or `None` if arbitrary keyword arguments are not accepted. *defaults* is an *n*-tuple of default argument values corresponding to the last *n* positional parameters, or `None` if there are no such defaults defined. *kwonlyargs* is a list of keyword-only parameter names in declaration order. *kwonlydefaults* is a dictionary mapping parameter names from *kwonlyargs* to the default values used if no argument is supplied. *annotations* is a dictionary

mapping parameter names to annotations. The special key `"return"` is used to report the function return value annotation (if any).

Note that **`signature()`** and Signature Object provide the recommended API for callable introspection, and support additional behaviours (like positional-only arguments) that are sometimes encountered in extension module APIs. This function is retained primarily for use in code that needs to maintain compatibility with the Python 2 `inspect` module API.

A member of the **`annotationlib.Format`** enum can be passed to the *annotation_format* parameter to control the format of the returned annotations. For example, use `annotation_format=annotationlib.Format.STRING` to return annotations in string format. Note that with the default `VALUE` format, creation of some argspecs may raise an exception.

Alterado na versão 3.4: This function is now based on **`signature()`**, but still ignores `__wrapped__` attributes and includes the already bound first parameter in the signature output for bound methods.

Alterado na versão 3.6: This method was previously documented as deprecated in favour of **`signature()`** in Python 3.5, but that decision has been reversed in order to restore a clearly supported standard interface for single-source Python 2/3 code migrating away from the legacy **`getargspec()`** API.

Alterado na versão 3.7: Python only explicitly guaranteed that it preserved the declaration order of keyword-only parameters as of version 3.7, although in practice this order had always been preserved in Python 3.

Alterado na versão 3.15: The *annotation_format* parameter was added.

inspect.getargvalues(*frame*)

Get information about arguments passed into a particular frame. A named tuple `ArgInfo(args, varargs, keywords, locals)` is returned. *args* is a list of the argument names. *varargs* and *keywords* are the names of the `*` and `**` arguments or `None`. *locals* is the locals dictionary of the given frame.

Nota

This function was inadvertently marked as deprecated in Python 3.5.

inspect.formatargvalues(*args*[, *varargs*, *varkw*, *locals*, *formatarg*, *formatvarargs*, *formatvarkw*, *formatvalue*])

Format a pretty argument spec from the four values returned by `getargvalues()`. The format* arguments are the corresponding optional formatting functions that are called to turn names and values into strings.

Nota

This function was inadvertently marked as deprecated in Python 3.5.

inspect.getmro(*cls*)

Return a tuple of class cls's base classes, including cls, in method resolution order. No class appears more than once in this tuple. Note that the method resolution order depends on cls's type. Unless a very peculiar user-defined metatype is in use, cls will be the first element of the tuple.

inspect.getcallargs(*func*, /, **args*, ***kwds*)

Bind the *args* and *kwds* to the argument names of the Python function or method *func*, as if it was called with them. For bound methods, bind also the first argument (typically named `self`) to the associated instance. A dict is returned, mapping the argument names (including the names of the `*` and `**` arguments, if any) to their values from *args* and *kwds*. In case of invoking *func* incorrectly, i.e. whenever `func(*args, **kwds)` would raise an exception because of incompatible signature, an exception of the same type and the same or similar message is raised. For example:

```
>>> from inspect import getcallargs
>>> def f(a, b=1, *pos, **named):
...     pass
...
>>> getcallargs(f, 1, 2, 3) == {'a': 1, 'named': {}, 'b': 2, 'pos'
True
>>> getcallargs(f, a=2, x=4) == {'a': 2, 'named': {'x': 4}, 'b': 1
True
>>> getcallargs(f)
Traceback (most recent call last):
...
TypeError: f() missing 1 required positional argument: 'a'
```

Adicionado na versão 3.2.

Descontinuado desde a versão 3.5: Use `Signature.bind()` and `Signature.bind_partial()` instead.

inspect.getclosurevars(*func*)

Get the mapping of external name references in a Python function or method *func* to their current values. A named tuple `ClosureVars(nonlocals, globals, builtins, unbound)` is returned. *nonlocals* maps referenced names to lexical closure variables, *globals* to the function's module globals and *builtins* to the builtins visible from the function body. *unbound* is the set of names referenced in the function that could not be resolved at all given the current module globals and builtins.

`TypeError` is raised if *func* is not a Python function or method.

Adicionado na versão 3.3.

inspect.unwrap(*func*, **, stop=None*)

Get the object wrapped by *func*. It follows the chain of `__wrapped__` attributes returning the last object in the chain.

stop is an optional callback accepting an object in the wrapper chain as its sole argument that allows the unwrapping to be terminated early if the callback returns a true value. If the callback never returns a true value, the last object in the chain is returned as usual. For example, `signature()` uses this to stop unwrapping if any object in the chain has a `__signature__` attribute defined.

`ValueError` is raised if a cycle is encountered.

Adicionado na versão 3.4.

inspect.get_annotations(*obj*, **, globals=None, locals=None, eval_str=False, format=annotationlib.Format.VALUE*)

Calcula o dicionário de anotações para um objeto.

This is an alias for `annotationlib.get_annotations()`; see the documentation of that function for more information.

Cuidado

Esta função pode executar código arbitrário contido em anotações. Consulte Implicações de segurança de anotações de introspecção para obter mais

informações.

Adicionado na versão 3.10.

Alterado na versão 3.14: This function is now an alias for `annotationlib.get_annotations()`. Calling it as `inspect.get_annotations` will continue to work.

A pilha to interpretador

Some of the following functions return `FrameInfo` objects. For backwards compatibility these objects allow tuple-like operations on all attributes except `positions`. This behavior is considered deprecated and may be removed in the future.

class inspect.FrameInfo

frame

The frame object that the record corresponds to.

filename

The file name associated with the code being executed by the frame this record corresponds to.

lineno

The line number of the current line associated with the code being executed by the frame this record corresponds to.

function

The function name that is being executed by the frame this record corresponds to.

code_context

A list of lines of context from the source code that's being executed by the frame this record corresponds to.

index

The index of the current line being executed in the `code_context` list.

positions

A `dis.Positions` object containing the start line number, end line number, start column offset, and end column offset associated with the instruction being executed by the frame this record corresponds to.

Alterado na versão 3.5: Return a named tuple instead of a `tuple`.

Alterado na versão 3.11: `FrameInfo` is now a class instance (that is backwards compatible with the previous named tuple).

class inspect.Traceback

filename

The file name associated with the code being executed by the frame this traceback corresponds to.

lineno

The line number of the current line associated with the code being executed by the frame this traceback corresponds to.

function

The function name that is being executed by the frame this traceback corresponds to.

code_context

A list of lines of context from the source code that's being executed by the frame this traceback corresponds to.

index

The index of the current line being executed in the `code_context` list.

positions

A `dis.Positions` object containing the start line number, end line number, start column offset, and end column offset associated with the instruction being executed by the frame this traceback corresponds to.

Alterado na versão 3.11: `Traceback` is now a class instance (that is backwards compatible with the previous named tuple).

Nota

Keeping references to frame objects, as found in the first element of the frame records these functions return, can cause your program to create reference cycles.

Once a reference cycle has been created, the lifespan of all objects which can be accessed from the objects which form the cycle can become much longer even if Python's optional cycle detector is enabled. If such cycles must be created, it is important to ensure they are explicitly broken to avoid the delayed destruction of objects and increased memory consumption which occurs.

Though the cycle detector will catch these, destruction of the frames (and local variables) can be made deterministic by removing the cycle in a **`finally`** clause. This is also important if the cycle detector was disabled when Python was compiled or using **`gc.disable()`**. For example:

```
def handle_stackframe_without_leak():
    frame = inspect.currentframe()
    try:
        # do something with the frame
    finally:
        del frame
```

If you want to keep the frame around (for example to print a traceback later), you can also break reference cycles by using the **`frame.clear()`** method.

The optional *context* argument supported by most of these functions specifies the number of lines of context to return, which are centered around the current line.

inspect.getframeinfo(*frame*, *context=1*)

Get information about a frame or traceback object. A **`Traceback`** object is returned.

Alterado na versão 3.11: A **`Traceback`** object is returned instead of a named tuple.

inspect.getouterframes(*frame*, *context=1*)

Get a list of **`FrameInfo`** objects for a frame and all outer frames. These frames represent the calls that lead to the creation of *frame*. The first entry in the returned list represents *frame*; the last entry represents the outermost call on *frame*'s stack.

Alterado na versão 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of **`FrameInfo`** objects is returned.

inspect.getinnerframes(*traceback*, *context=1*)

Get a list of **`FrameInfo`** objects for a traceback's frame and all inner frames. These frames represent calls made as a consequence of *frame*. The first entry in the list represents *traceback*; the last entry represents where the exception was raised.

Alterado na versão 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of **`FrameInfo`** objects is returned.

inspect.currentframe()

Return the frame object for the caller's stack frame.

Detalhes da implementação do CPython: This function relies on Python stack frame support in the interpreter, which isn't guaranteed to exist in all implementations of Python. If running in an implementation without Python stack frame support this function returns `None`.

inspect.stack(*context=1*)

Return a list of **`FrameInfo`** objects for the caller's stack. The first entry in the returned list represents the caller; the last entry represents the outermost call on the stack.

Alterado na versão 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of **`FrameInfo`** objects is returned.

inspect.trace(*context=1*)

Return a list of **`FrameInfo`** objects for the stack between the current frame and the frame in which an exception currently being handled was raised in. The first entry in the list represents the caller; the last entry represents where the exception was raised.

Alterado na versão 3.5: A list of named tuples `FrameInfo(frame, filename, lineno, function, code_context, index)` is returned.

Alterado na versão 3.11: A list of **`FrameInfo`** objects is returned.

Fetching attributes statically

Both `getattr()` and `hasattr()` can trigger code execution when fetching or checking for the existence of attributes. Descriptors, like properties, will be invoked and `__getattr__()` and `__getattribute__()` may be called.

For cases where you want passive introspection, like documentation tools, this can be inconvenient. `getattr_static()` has a similar signature as `getattr()` but avoids executing code when it fetches attributes.

inspect.getattr_static(*obj*, *attr*)
inspect.getattr_static(*obj*, *attr*, *default*)

Retrieve attributes without triggering dynamic lookup via the descriptor protocol, `__getattr__()` or `__getattribute__()`.

Note: this function may not be able to retrieve all attributes that getattr can fetch (like dynamically created attributes) and may find attributes that getattr can't (like descriptors that raise AttributeError). It can also return descriptors objects instead of instance members.

If the instance `__dict__` is shadowed by another member (for example a property) then this function will be unable to find instance members.

Adicionado na versão 3.2.

`getattr_static()` does not resolve descriptors, for example slot descriptors or getset descriptors on objects implemented in C. The descriptor object is returned instead of the underlying attribute.

You can handle these with code like the following. Note that for arbitrary getset descriptors invoking these may trigger code execution:

```
# example code for resolving the builtin descriptor types
class _foo:
    __slots__ = ['foo']

slot_descriptor = type(_foo.foo)
getset_descriptor = type(type(open(__file__)).name)
wrapper_descriptor = type(str.__dict__['__add__'])
descriptor_types = (slot_descriptor, getset_descriptor, wrapper_descr

result = getattr_static(some_object, 'foo')
```

```
if type(result) in descriptor_types:
    try:
        result = result.__get__()
    except AttributeError:
        # descriptors can raise AttributeError to
        # indicate there is no underlying value
        # in which case the descriptor itself will
        # have to do
        pass
```

Current State of Generators, Coroutines, and Asynchronous Generators

When implementing coroutine schedulers and for other advanced uses of generators, it is useful to determine whether a generator is currently executing, is waiting to start or resume or execution, or has already terminated. `getgeneratorstate()` allows the current state of a generator to be determined easily.

inspect.getgeneratorstate(*generator*)

Get current state of a generator-iterator.

Possible states are:

- GEN_CREATED: Waiting to start execution.
- GEN_RUNNING: Currently being executed by the interpreter.
- GEN_SUSPENDED: Currently suspended at a yield expression.
- GEN_CLOSED: Execution has completed.

Adicionado na versão 3.2.

inspect.getcoroutinestate(*coroutine*)

Get current state of a coroutine object. The function is intended to be used with coroutine objects created by `async def` functions, but will accept any coroutine-like object that has `cr_running` and `cr_frame` attributes.

Possible states are:

- CORO_CREATED: Waiting to start execution.
- CORO_RUNNING: Currently being executed by the interpreter.
- CORO_SUSPENDED: Currently suspended at an await expression.

- CORO_CLOSED: Execution has completed.

Adicionado na versão 3.5.

inspect.getasyncgenstate(*agen*)

Get current state of an asynchronous generator object. The function is intended to be used with asynchronous iterator objects created by `async def` functions which use the `yield` statement, but will accept any asynchronous generator-like object that has `ag_running` and `ag_frame` attributes.

Possible states are:

- AGEN_CREATED: Waiting to start execution.
- AGEN_RUNNING: Currently being executed by the interpreter.
- AGEN_SUSPENDED: Currently suspended at a yield expression.
- AGEN_CLOSED: Execution has completed.

Adicionado na versão 3.12.

The current internal state of the generator can also be queried. This is mostly useful for testing purposes, to ensure that internal state is being updated as expected:

inspect.getgeneratorlocals(*generator*)

Get the mapping of live local variables in *generator* to their current values. A dictionary is returned that maps from variable names to values. This is the equivalent of calling `locals()` in the body of the generator, and all the same caveats apply.

If *generator* is a generator with no currently associated frame, then an empty dictionary is returned. `TypeError` is raised if *generator* is not a Python generator object.

Detalhes da implementação do CPython: This function relies on the generator exposing a Python stack frame for introspection, which isn't guaranteed to be the case in all implementations of Python. In such cases, this function will always return an empty dictionary.

Adicionado na versão 3.3.

inspect.getcoroutinelocals(*coroutine*)

This function is analogous to `getgeneratorlocals()`, but works for coroutine objects created by `async def` functions.

Adicionado na versão 3.5.

inspect.getasyncgenlocals(*agen*)

This function is analogous to `getgeneratorlocals()`, but works for asynchronous generator objects created by `async def` functions which use the `yield` statement.

Adicionado na versão 3.12.

Code Objects Bit Flags

Python code objects have a `co_flags` attribute, which is a bitmap of the following flags:

inspect.CO_OPTIMIZED

The code object is optimized, using fast locals.

inspect.CO_NEWLOCALS

If set, a new dict will be created for the frame's `f_locals` when the code object is executed.

inspect.CO_VARARGS

The code object has a variable positional parameter (`*args`-like).

inspect.CO_VARKEYWORDS

The code object has a variable keyword parameter (`**kwargs`-like).

inspect.CO_NESTED

The flag is set when the code object is a nested function.

inspect.CO_GENERATOR

The flag is set when the code object is a generator function, i.e. a generator object is returned when the code object is executed.

inspect.CO_COROUTINE

The flag is set when the code object is a coroutine function. When the code object is executed it returns a coroutine object. See **PEP 492**

[https://peps.python.org/pep-0492/] for more details.

Adicionado na versão 3.5.

inspect.CO_ITERABLE_COROUTINE

The flag is used to transform generators into generator-based coroutines. Generator objects with this flag can be used in `await` expression, and can `yield from` coroutine objects. See **PEP 492** [https://peps.python.org/pep-0492/] for more details.

Adicionado na versão 3.5.

inspect.CO_ASYNC_GENERATOR

The flag is set when the code object is an asynchronous generator function. When the code object is executed it returns an asynchronous generator object. See **PEP 525** [https://peps.python.org/pep-0525/] for more details.

Adicionado na versão 3.6.

inspect.CO_HAS_DOCSTRING

The flag is set when there is a docstring for the code object in the source code. If set, it will be the first item in **`co_consts`**.

Adicionado na versão 3.14.

inspect.CO_METHOD

The flag is set when the code object is a function defined in class scope.

Adicionado na versão 3.14.

Nota

The flags are specific to CPython, and may not be defined in other Python implementations. Furthermore, the flags are an implementation detail, and can be removed or deprecated in future Python releases. It's recommended to use public APIs from the **`inspect`** module for any introspection needs.

Buffer flags

class inspect.BufferFlags

This is an `enum.IntFlag` that represents the flags that can be passed to the `__buffer__()` method of objects implementing the buffer protocol.

The meaning of the flags is explained at Tipos de solicitação do buffer.

SIMPLE

WRITABLE

FORMAT

ND

STRIDES

C_CONTIGUOUS

F_CONTIGUOUS

ANY_CONTIGUOUS

INDIRECT

CONTIG

CONTIG_RO

STRIDED

STRIDED_RO

RECORDS

RECORDS_RO

FULL

FULL_RO

READ

WRITE

Adicionado na versão 3.12.

Interface de linha de comando

The **`inspect`** module also provides a basic introspection capability from the command line.

By default, accepts the name of a module and prints the source of that module. A class or function within the module can be printed instead by appending a colon and the qualified name of the target object.

--details

Print information about the specified object rather than the source code

Alterado na versão 3.15: The `--details` option now supports basic introspection for modules without available source code and indicates when modules are frozen. It also indicates when the given target reference is not the canonical name of the referenced object.

`annotationlib` — Funcionalidade para introspecção de anotações

Adicionado na versão 3.14.

Código-fonte: Lib/annotationlib.py
[https://github.com/python/cpython/tree/3.15/Lib/annotationlib.py]

O módulo **`annotationlib`** fornece ferramentas para introspecção de anotações em módulos, classes e funções.

As anotações são avaliadas preguiçosamente e frequentemente contêm referências futuras a objetos que ainda não foram definidos quando a anotação é criada. Este módulo fornece um conjunto de ferramentas de baixo nível que podem ser usadas para recuperar anotações de forma confiável, mesmo na presença de referências futuras e em outros casos extremos.

Este módulo oferece suporte à recuperação de anotações em três formatos principais (veja **`Format`**), cada um dos quais funciona melhor para diferentes casos de uso:

- **`VALUE`** avalia as anotações e retorna seus valores. Isso é mais simples de usar, mas pode levantar erros, por exemplo, se as anotações contiverem referências a nomes indefinidos.
- **`FORWARDREF`** retorna objetos **`ForwardRef`** para anotações que não podem ser resolvidas, permitindo que você inspecione as anotações sem avaliá-las. Isso é útil quando você precisa trabalhar com anotações que podem conter referências de encaminhamento não resolvidas.
- **`STRING`** retorna as anotações como uma string, semelhante a como apareceriam no arquivo-fonte. Isso é útil para geradores de documentação que desejam exibir anotações de forma legível.

A função `get_annotations()` é o principal ponto de entrada para recuperar anotações. Dada uma função, classe ou módulo, ela retorna um dicionário de anotações no formato solicitado. Este módulo também fornece funcionalidade para trabalhar diretamente com a função de anotação, usada para avaliar anotações, como `get_annotate_from_class_namespace()` e `call_annotate_function()`, bem como a função `call_evaluate_function()` para trabalhar com funções de avaliação.

Cuidado

A maioria das funcionalidades deste módulo pode executar código arbitrário; veja a seção de segurança para mais informações.

Ver também

PEP 649 [https://peps.python.org/pep-0649/] propôs o modelo atual de como as anotações funcionam em Python.

PEP 749 [https://peps.python.org/pep-0749/] expandiu vários aspectos de **PEP 649** [https://peps.python.org/pep-0649/] e introduziu o módulo `annotationlib`.

Boas práticas para anotações fornece práticas recomendadas para trabalhar com anotações.

typing-extensions [https://pypi.org/project/typing-extensions/] fornece uma função `get_annotations()`, com compatibilidade retroativa, que funciona em versões anteriores do Python.

Semânticas de anotação

A forma como as anotações são avaliadas mudou ao longo da história do Python 3 e atualmente ainda depende de uma importação futura. Houve modelos de execução para anotações:

- *Semântica de estoque* (padrão no Python 3.0 a 3.13; veja **PEP 3107** [https://peps.python.org/pep-3107/] e **PEP 526** [https://peps.python.org/pep-0526/]): As anotações são avaliadas avidamente, à medida que são encontradas no código-fonte.
- *Anotações em string* (usadas com `from __future__ import annotations` no Python 3.7 e versões mais recentes; veja **PEP 563** [https://peps.python.org/pep-0563/]): As anotações são armazenadas apenas como strings.
- *Avaliação adiada* (padrão no Python 3.14 e versões mais recentes; veja **PEP 649** [https://peps.python.org/pep-0649/] e **PEP 749** [https://peps.python.org/pep-0749/]): As anotações são avaliadas preguiçosamente, somente quando são acessadas.

Como exemplo, considere o seguinte programa:

```
def func(a: Cls) -> None:
    print(a)

class Cls: pass

print(func.__annotations__)
```

Isso se comportará da seguinte maneira:

- De acordo com a semântica de estoque (Python 3.13 e versões anteriores), ele levantará uma `NameError` na linha onde `func` está definido, porque `Cls` é um nome indefinido naquele ponto.
- Em anotações stringificadas (se `from __future__ import annotations` for usado), exibirá `{'a': 'Cls', 'return': 'None'}`.
- Em avaliação adiada (Python 3.14 e posterior), exibirá `{'a': <class 'Cls'>, 'return': None}`.

A semântica de estoque foi usada quando as anotações de função foram introduzidas pela primeira vez no Python 3.0 (por **PEP 3107** [https://peps.python.org/pep-3107/]) porque esta era a maneira mais simples e óbvia de implementar anotações. O mesmo modelo de execução foi usado quando as anotações de variáveis foram introduzidas no Python 3.6 (por **PEP 526** [https://peps.python.org/pep-0526/]). No entanto, a semântica de estoque causou problemas ao usar anotações como dicas de tipo, como a necessidade de se

referir a nomes que ainda não estavam definidos quando a anotação era encontrada. Além disso, havia problemas de desempenho com a execução de anotações no momento da importação do módulo. Portanto, no Python 3.7, **PEP 563** [https://peps.python.org/pep-0563/] introduziu a capacidade de armazenar anotações como strings usando a sintaxe `from __future__ import annotations`. O plano na época era eventualmente tornar esse comportamento o padrão, mas um problema surgiu: anotações em string são mais difíceis de processar para aqueles que inspecionam anotações em tempo de execução. Uma proposta alternativa, **PEP 649** [https://peps.python.org/pep-0649/], introduziu o terceiro modelo de execução, avaliação adiada, e foi implementada no Python 3.14. Anotações em string ainda são usadas se `from __future__ import annotations` estiver presente, mas esse comportamento será eventualmente removido.

Classes

class annotationlib.Format

Uma `IntEnum` que descreve os formatos nos quais as anotações podem ser retornadas. Membros da enumeração, ou seus valores inteiros equivalentes, podem ser passados para `get_annotations()` e outras funções neste módulo, bem como para funções `__annotate__`.

VALUE = *1*

Os valores são o resultado da avaliação das expressões de anotação.

VALUE_WITH_FAKE_GLOBALS = *2*

Valor especial usado para sinalizar que uma função de anotação está sendo avaliada em um ambiente especial com variáveis globais falsas. Ao receber este valor, as funções de anotação devem retornar o mesmo valor do formato `Format.VALUE` ou levantar `NotImplementedError` para sinalizar que não suportam execução neste ambiente. Este formato é usado apenas internamente e não deve ser passado para as funções deste módulo.

FORWARDREF = *3*

Valores são valores de anotação reais (conforme o formato `Format.VALUE`) para valores definidos e proxies `ForwardRef` para valores indefinidos. Objetos reais podem conter referências a objetos proxy `ForwardRef`.

STRING = *4*

Valores são a string de texto da anotação como ela aparece no código-fonte, até modificações, incluindo, mas não se limitando a, normalizações de espaços em branco e otimizações de valores constantes.

Os valores exatos dessas strings podem mudar em versões futuras do Python.

Adicionado na versão 3.14.

class annotationlib.ForwardRef

Um objeto proxy para referências futuras em anotações.

Instâncias desta classe são retornadas quando o formato `FORWARDREF` é usado e as anotações contêm um nome que não pode ser resolvido. Isso pode acontecer quando uma referência de avanço é usada em uma anotação, como quando uma classe é referenciada antes de ser definida.

__forward_arg__

Uma string contendo o código que foi avaliado para produzir `ForwardRef`. A string pode não ser exatamente equivalente à fonte original.

evaluate(*, *owner=None*, *globals=None*, *locals=None*, *type_params=None*, *format=Format.VALUE*)

Avalia a referência futura, retornando seu valor.

Se o argumento *format* for `VALUE` (o padrão), este método poderá levantar uma exceção, como `NameError`, caso a referência futura se refira a um nome que não pode ser resolvido. Os argumentos deste método podem ser usados para fornecer ligações para nomes que, de

outra forma, seriam indefinidos. Se o argumento *format* for **FORWARDREF**, o método nunca levantará uma exceção, mas poderá retornar uma instância de **ForwardRef**. Por exemplo, se o objeto de referência futura contiver o código `list[undefined]`, onde `undefined` é um nome que não está definido, avaliá-lo com o formato **FORWARDREF** retornará `list[ForwardRef('undefined')]`. Se o argumento *format* for **STRING**, o método retornará **__forward_arg__**.

O parâmetro *owner* fornece o mecanismo preferencial para passar informações de escopo para este método. O proprietário de uma **ForwardRef** é o objeto que contém a anotação da qual a **ForwardRef** deriva, como um objeto módulo, objeto tipo ou objeto função.

Os parâmetros *globals*, *locals* e *type_params* fornecem um mecanismo mais preciso para influenciar os nomes disponíveis quando **ForwardRef** é avaliado. *globals* e *locals* são passados para **eval()**, representando os espaços de nomes global e local nos quais o nome é avaliado. O parâmetro *type_params* é relevante para objetos criados usando a sintaxe nativa para classes genéricas e funções. É uma tupla de parâmetros de tipo que estão no escopo enquanto a referência futura está sendo avaliada. Por exemplo, ao avaliar um **ForwardRef** recuperado de uma anotação encontrada no espaço de nomes de classe de uma classe genérica `C`, *type_params* deve ser definido como `C.__type_params__`.

Instâncias de **ForwardRef** retornadas por **get_annotations()** retêm referências a informações sobre o escopo de onde se originaram, portanto, chamar esse método sem argumentos adicionais pode ser suficiente para avaliar tais objetos. Instâncias de **ForwardRef** criadas por outros meios podem não ter nenhuma informação sobre seu escopo, portanto, passar argumentos para esse método pode ser necessário para avaliá-las com sucesso.

Se nenhum entre *owner*, *globals*, *locals* ou *type_params* for fornecido e **ForwardRef** não contiver informações sobre sua origem, dicionários globals e locals vazios serão usados.

Adicionado na versão 3.14.

Funções

annotationlib.annotations_to_string(*annotations*)

Converte um dicionário de anotações contendo valores de tempo de execução em um dicionário contendo apenas strings. Se os valores ainda não forem strings, eles serão convertidos usando `type_repr()`. Isso serve como um auxiliar para funções de anotação fornecidas pelo usuário que oferecem suporte ao formato `STRING`, mas não têm acesso ao código que cria as anotações.

For example, this is used to implement the `STRING` format for `typing.TypedDict` classes created through the functional syntax:

```
>>> from typing import TypedDict
>>> Movie = TypedDict("movie", {"name": str, "year": int})
>>> get_annotations(Movie, format=Format.STRING)
{'name': 'str', 'year': 'int'}
```

Adicionado na versão 3.14.

annotationlib.call_annotate_function(*annotate*, *format*, ***, *owner=None*)

Chama a função de anotação *annotate* com o *format* fornecido, um membro da enumeração `Format` e retorne o dicionário de anotações produzido pela função.

Esta função auxiliar é necessária porque as funções de anotação geradas pelo compilador para funções, classes e módulos oferecem suporte a apenas o formato `VALUE` quando chamadas diretamente. Para oferecer suporte a outros formatos, esta função chama a função de anotação em um ambiente especial que permite a produção de anotações nos outros formatos. Este é um bloco de construção útil ao implementar funcionalidades que precisam avaliar anotações parcialmente enquanto uma classe está sendo construída.

owner é o objeto que possui a função de anotação, geralmente uma função, classe ou módulo. Se fornecido, é usado no formato `FORWARDREF` para produzir um objeto `ForwardRef` que contém mais informações.

Ver também

PEP 649 [https://peps.python.org/pep-0649/#the-stringizer-and-the-fake-globals-environment] contém uma explicação da técnica de implementação usada por esta função.

Adicionado na versão 3.14.

annotationlib.call_evaluate_function(*evaluate*, *format*, *, *owner=None*)

Chama a função de avaliação *evaluate* com o *format* fornecido, um membro da enumeração `Format` e retorna o valor produzido pela função. Isso é semelhante a `call_annotate_function()`, mas esta última sempre retorna um dicionário que mapeia strings para anotações, enquanto esta função retorna um único valor.

Isso se destina ao uso com as funções de avaliação geradas para elementos avaliados preguiçosamente relacionados a apelidos de tipo e parâmetros de tipo:

- `typing.TypeAliasType.evaluate_value()`, o valor dos apelidos de tipo
- `typing.TypeVar.evaluate_bound()`, a delimitação das variáveis de tipo
- `typing.TypeVar.evaluate_constraints()`, as restrições de tipos variáveis
- `typing.TypeVar.evaluate_default()`, o valor padrão de tipos variáveis
- `typing.ParamSpec.evaluate_default()`, o valor padrão de especificações de parâmetros
- `typing.TypeVarTuple.evaluate_default()`, o valor padrão de tuplas de tipos variáveis

owner é o objeto que possui a função de avaliação, como o apelido de tipo ou o objeto de tipo variável.

format pode ser usado para controlar o formato no qual o valor é retornado:

```
>>> type Alias = undefined
>>> call_evaluate_function(Alias.evaluate_value, Format.VALUE
Traceback (most recent call last):
...
NameError: name 'undefined' is not defined
>>> call_evaluate_function(Alias.evaluate_value, Format.FORWA
ForwardRef('undefined')
>>> call_evaluate_function(Alias.evaluate_value, Format.STRIN
'undefined'
```

Adicionado na versão 3.14.

annotationlib.get_annotate_from_class_namespace(*namespace*)

Recupera a função de anotação de um dicionário de espaço de nomes de classe *namespace*. Retorna **`None`** se o espaço de nomes não contiver uma função de anotação. Isso é útil principalmente antes da classe ser totalmente criada (por exemplo, em uma metaclasse); após a classe existir, a função de anotação pode ser recuperada com `cls.__annotate__`. Veja abaixo para um exemplo usando esta função em uma metaclasse.

Adicionado na versão 3.14.

annotationlib.get_annotations(*obj*, **, globals=None, locals=None, eval_str=False, format=Format.VALUE*)

Calcula o dicionário de anotações para um objeto.

obj pode ser um objeto chamável, classe, módulo ou outro objeto com os atributos **`__annotate__`** ou **`__annotations__`**. Passar qualquer outro objeto levanta **`TypeError`**.

O parâmetro *format* controla o formato em que as anotações são retornadas e deve ser um membro da enumeração **`Format`** ou seu equivalente inteiro. Os diferentes formatos funcionam da seguinte forma:

- VALUE: **`object.__annotations__`** é tentado primeiro; se não existir, a função **`object.__annotate__`** é chamada, se existir.
- FORWARDREF: Se **`object.__annotations__`** existir e puder ser avaliado com sucesso, ele será usado; caso contrário, a função **`object.__annotate__`** será chamada. Se também não existir, **`object.__annotations__`** será tentado novamente e qualquer erro ao acessá-lo será levantado novamente.
    - Ao chamar **`object.__annotate__`**, ele é chamado primeiro com **`FORWARDREF`**. Se isso não for implementado, ele verificará se **`VALUE_WITH_FAKE_GLOBALS`** é suportado e o utilizará no ambiente de globais falsos. Se nenhum desses formatos for suportado, ele retornará para **`VALUE`**. Se **`VALUE`** falhar, o erro desta chamada será gerado.
- STRING: Se **`object.__annotate__`** existir, ele será chamado primeiro; caso contrário, **`object.__annotations__`** será usado e transformado em string usando **`annotations_to_string()`**.
    - Ao chamar **`object.__annotate__`**, ele é chamado primeiro com **`STRING`**. Se isso não for implementado, ele verificará se **`VALUE_WITH_FAKE_GLOBALS`** é suportado e o utilizará no ambiente de globais falsos. Se nenhum desses formatos for suportado, ele recorrerá a **`VALUE`** com o resultado convertido usando **`annotations_to_string()`**. Se **`VALUE`** falhar, o erro desta chamada será levantado.

Retorna um dict. **`get_annotations()`** retorna um novo dict toda vez que é chamado; chamá-lo duas vezes no mesmo objeto retornará dois dicts diferentes, mas equivalentes.

Esta função cuida de vários detalhes para você:

- Se *eval_str* for verdadeiro, valores do tipo **`str`** serão descodificados usando **`eval()`**. Isso se destina ao uso com anotações transformadas em string (`from __future__ import annotations`). É um erro definir *eval_str* como true com formatos diferentes de **`Format.VALUE`**.
- Se *obj* não tiver um dicionário de anotações, retorna um dicionário vazio. (Funções e métodos sempre têm um dicionário de

anotações; classes, módulos e outros tipos de chamáveis podem não ter.)
- Ignora anotações herdadas em classes, bem como anotações em metaclasses. Se uma classe não tiver seu próprio dicionário de anotações, retorna um dicionário vazio.
- Todos os acessos aos membros do objeto e valores do dicionário são feitos usando `getattr()` e `dict.get()` por segurança.

eval_str controla se valores do tipo **`str`** são substituídos ou não pelo resultado da chamada de **`eval()`** nesses valores:

- Se eval_str for verdadeiro, **`eval()`** será chamado em valores do tipo **`str`**. (Observe que **`get_annotations()`** não captura exceções; se **`eval()`** levanta uma exceção, ele desenrolará a pilha após a chamada de **`get_annotations()`**.)
- Se *eval_str* for falso (o padrão), os valores do tipo **`str`** não serão alterados.

globals e *locals* são passados para **`eval()`**; consulte a documentação de **`eval()`** para mais informações. Se *globals* ou *locals* for **`None`**, esta função pode substituir esse valor por um padrão específico do contexto, dependendo de `type(obj)`:

- Se *obj* for um módulo, *globals* assume como padrão `obj.__dict__`.
- Se *obj* for uma classe, *globals* assume como padrão `sys.modules[obj.__module__].__dict__` e *locals* assume como padrão o espaço de nomes da classe *obj*.
- Se *obj* for um chamável, *globals* assume como padrão **`obj.__globals__`**, embora se *obj* for uma função encapsulada (usando **`functools.update_wrapper()`**) ou um objeto **`functools.partial`**, ela será desencapsulada até que uma função não encapsulada seja encontrada.

Chamar **`get_annotations()`** é uma boa prática para acessar o dicionário de anotações de qualquer objeto. Consulte Boas práticas para anotações para obter mais informações sobre as práticas recomendadas para anotações.

```
>>> def f(a: int, b: str) -> float:
...     pass
>>> get_annotations(f)
{'a': <class 'int'>, 'b': <class 'str'>, 'return': <class 'fl
```

Adicionado na versão 3.14.

annotationlib.type_repr(*value*)

Converte um valor Python arbitrário para um formato adequado para uso pelo formato **STRING**. Isso chama **repr()** para a maioria dos objetos, mas tem um tratamento especial para alguns objetos, como objetos tipo.

Isto serve como um auxiliar para funções de anotação fornecidas pelo usuário que oferecem suporte ao formato **STRING**, mas não têm acesso ao código que cria as anotações. Também pode ser usado para fornecer uma representação de string amigável para outros objetos que contêm valores comumente encontrados em anotações.

Adicionado na versão 3.14.

Receitas

Usando anotações em uma metaclasse

Uma metaclasse pode querer inspecionar ou até mesmo modificar as anotações no corpo de uma classe durante a criação da classe. Isso requer a recuperação das anotações do dicionário de espaços de nomes da classe. Para classes criadas com `from __future__ import annotations`, as anotações estarão na chave `__annotations__` do dicionário. Para outras classes com anotações, **get_annotate_from_class_namespace()** pode ser usado para obter a função annotate, e **call_annotate_function()** pode ser usado para chamá-la e recuperar as anotações. Usar o formato **FORWARDREF** geralmente é a melhor opção, pois permite que as anotações se refiram a nomes que ainda não podem ser resolvidos quando a classe é criada.

Para modificar as anotações, é melhor criar uma função de anotação que chame a função de anotação original, faça os ajustes necessários e retorne o

resultado.

Abaixo está um exemplo de uma metaclasse que filtra todas as anotações `typing.ClassVar` da classe e as coloca em um atributo separado:

```
import annotationlib
import typing

class ClassVarSeparator(type):
   def __new__(mcls, name, bases, ns):
      if "__annotations__" in ns:  # from __future__ import anno
         annotations = ns["__annotations__"]
         classvar_keys = {
            key for key, value in annotations.items()
            # Use comparação de strings para simplificar; uma so
            # mais robusta poderia usar annotationlib.ForwardRef
            if value.startswith("ClassVar")
         }
         classvars = {key: annotations[key] for key in classvar_
         ns["__annotations__"] = {
            key: value for key, value in annotations.items()
            if key not in classvar_keys
         }
         wrapped_annotate = None
      elif annotate := annotationlib.get_annotate_from_class_nam
         annotations = annotationlib.call_annotate_function(
            annotate, format=annotationlib.Format.FORWARDREF
         )
         classvar_keys = {
            key for key, value in annotations.items()
            if typing.get_origin(value) is typing.ClassVar
         }
         classvars = {key: annotations[key] for key in classvar_

         def wrapped_annotate(format):
            annos = annotationlib.call_annotate_function(annotat
            return {key: value for key, value in annos.items() i

      else:  # nenhuma anotação
         classvars = {}
         wrapped_annotate = None
      typ = super().__new__(mcls, name, bases, ns)
```

```
    if wrapped_annotate is not None:
       # Encapsula o __annotate__ original com um invólucro qu
       typ.__annotate__ = wrapped_annotate
    typ.classvars = classvars  # Armazena as ClassVars em um a
    return typ
```

Creating a custom callable annotate function

Custom annotate functions may be literal functions like those automatically generated for functions, classes, and modules. Or, they may wish to utilise the encapsulation provided by classes, in which case any callable can be used as an annotate function.

To provide the **`VALUE`**, **`STRING`**, or **`FORWARDREF`** formats directly, an annotate function must provide the following attribute:

- A callable `__call__` with signature `__call__(format, /) -> dict`, that does not raise a **`NotImplementedError`** when called with a supported format.

To provide the **`VALUE_WITH_FAKE_GLOBALS`** format, which is used to automatically generate **`STRING`** or **`FORWARDREF`** if they are not supported directly, annotate functions must provide the following attributes:

- A callable `__call__` with signature `__call__(format, /) -> dict`, that does not raise a **`NotImplementedError`** when called with **`VALUE_WITH_FAKE_GLOBALS`**.
- A code object `__code__` containing the compiled code for the annotate function.
- Optional: A tuple of the function's positional defaults `__kwdefaults__`, if the function represented by `__code__` uses any positional defaults.
- Optional: A dict of the function's keyword defaults `__defaults__`, if the function represented by `__code__` uses any keyword defaults.
- Optional: All other function attributes.

```
class Annotate:
    called_formats = []

    def __call__(self, format=None, /, *, _self=None):
        # When called with fake globals, `_self` will be the
        # actual self value, and `self` will be the format.
        if _self is not None:
            self, format = _self, self

        self.called_formats.append(format)
        if format <= 2:  # VALUE or VALUE_WITH_FAKE_GLOBALS
            return {"x": MyType}
        raise NotImplementedError

    __code__ = __call__.__code__
    __defaults__ = (None,)
    __kwdefaults__ = property(lambda self: dict(_self=self))

    __globals__ = {}
    __builtins__ = {}
    __closure__ = None
```

This can then be called with:

```
>>> from annotationlib import call_annotate_function, Format
>>> call_annotate_function(Annotate(), format=Format.STRING)
{'x': 'MyType'}
```

Or used as the annotate function for an object:

```
>>> from annotationlib import get_annotations, Format
>>> class C:
...   pass
>>> C.__annotate__ = Annotate()
>>> get_annotations(Annotate(), format=Format.STRING)
{'x': 'MyType'}
```

Limitações do formato `STRING`

O formato `STRING` tem como objetivo aproximar o código-fonte da anotação, mas a estratégia de implementação usada significa que nem

sempre é possível recuperar o código-fonte exato.

Primeiro, a transformação em string, é claro, não pode recuperar nenhuma informação que não esteja presente no código compilado, incluindo comentários, espaços em branco, parênteses e operações que são simplificadas pelo compilador.

Em segundo lugar, a transformação em string pode interceptar quase todas as operações que envolvem nomes pesquisados em algum escopo, mas não pode interceptar operações que operem totalmente em constantes. Como corolário, isso também significa que não é seguro solicitar o formato `STRING` em código não confiável: Python é poderoso o suficiente para permitir a execução arbitrária de código mesmo sem acesso a nenhuma variável global ou embutida. Por exemplo:

```
>>> def f(x: (1).__class__.__base__.__subclasses__()[-1].__init_
...
>>> annotationlib.get_annotations(f, format=annotationlib.Format
Hello world
{'x': 'None'}
```

Nota

Este exemplo específico funciona no momento em que este artigo foi escrito, mas depende de detalhes de implementação e não há garantia de que funcionará no futuro.

Entre os diferentes tipos de expressões que existem em Python, conforme representado pelo módulo `ast`, algumas expressões são suportadas, o que significa que o formato `STRING` geralmente pode recuperar o código-fonte original; outras não são suportadas, o que significa que podem resultar em saída incorreta ou erro.

Os seguintes são aceitos (às vezes com ressalvas):

- `ast.BinOp`
- `ast.UnaryOp`

    - `ast.Invert` (~), `ast.UAdd` (+) e `ast.USub` (-) são suportadas
    - `ast.Not` (not) não é suportada
- `ast.Dict` (exceto ao usar desempacotamento **)
- `ast.Set`
- `ast.Compare`
    - `ast.Eq` e `ast.NotEq` são suportadas
    - `ast.Lt`, `ast.LtE`, `ast.Gt` e `ast.GtE` são suportadas, mas o operando pode ser invertido
    - `ast.Is`, `ast.IsNot`, `ast.In` e `ast.NotIn` não são suportadas
- `ast.Call` (exceto ao usar desempacotamento **)
- `ast.Constant` (embora não seja a representação exata da constante; por exemplo, sequências de escape em strings são perdidas; números hexadecimais são convertidos em decimais)
- `ast.Attribute` (presumindo que o valor não é uma constante)
- `ast.Subscript` (presumindo que o valor não é uma constante)
- `ast.Starred` (desempacotamento *)
- `ast.Name`
- `ast.List`
- `ast.Tuple`
- `ast.Slice`

Os seguintes não são suportados, mas geram um erro informativo quando encontrados pela transformação em string:

- `ast.FormattedValue` (f-strings; o erro não é detectado se especificadores de conversões como !r forem usados)
- `ast.JoinedStr` (f-strings)

Os seguintes itens não são suportados e resultam em saída incorreta:

- `ast.BoolOp` (and e or)
- `ast.IfExp`
- `ast.Lambda`
- `ast.ListComp`
- `ast.SetComp`
- `ast.DictComp`
- `ast.GeneratorExp`

Os seguintes itens não são permitidos em escopos de anotação e, portanto, não são relevantes:

- `ast.NamedExpr` (`:=`)
- `ast.Await`
- `ast.Yield`
- `ast.YieldFrom`

Limitações do formato `FORWARDREF`

O formato `FORWARDREF` visa produzir valores reais o máximo possível, com tudo o que não puder ser resolvido sendo substituído por objetos `ForwardRef`. Ele é afetado, em linhas gerais, pelas mesmas limitações do formato `STRING`: anotações que realizam operações em literais ou que usam tipos de expressão não suportados podem levantar exceções quando avaliadas usando o formato `FORWARDREF`.

Abaixo estão alguns exemplos do comportamento com expressões não suportadas:

```
>>> from annotationlib import get_annotations, Format
>>> def zerodiv(x: 1 / 0): ...
>>> get_annotations(zerodiv, format=Format.STRING)
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> get_annotations(zerodiv, format=Format.FORWARDREF)
Traceback (most recent call last):
  ...
ZeroDivisionError: division by zero
>>> def ifexp(x: 1 if y else 0): ...
>>> get_annotations(ifexp, format=Format.STRING)
{'x': '1'}
```

Implicações de segurança de anotações de introspecção

Grande parte da funcionalidade deste módulo envolve a execução de código relacionado a anotações, que podem então realizar ações arbitrárias. Por exemplo, `get_annotations()` pode chamar uma função de anotação arbitrária, e `ForwardRef.evaluate()` pode chamar `eval()` em uma string arbitrária. O código contido em uma anotação pode realizar chamadas de sistema arbitrárias, entrar em um laço infinito ou executar qualquer outra operação. Isso também se aplica a qualquer acesso ao atributo `__annotations__` e a várias funções no módulo `typing` que trabalham com anotações, como `typing.get_type_hints()`.

Qualquer problema de segurança decorrente disso também se aplica imediatamente após a importação de código que possa conter anotações não confiáveis: a importação de código sempre pode causar a execução de operações arbitrárias. No entanto, não é seguro aceitar strings ou outras entradas de uma fonte não confiável e passá-las para qualquer uma das APIs para introspecção de anotações, por exemplo, editando um dicionário `__annotations__` ou criando diretamente um objeto `ForwardRef`.

`site` — Gancho de configuração específico do site

Código-fonte: Lib/site.py [https://github.com/python/cpython/tree/3.15/Lib/site.py]

Este módulo é importado automaticamente durante a inicialização. A importação automática pode ser suprimida usando a opção `-S` do interpretador.

Importing this module normally appends site-specific paths to the module search path and adds callables, including `help()` to the built-in namespace. However, Python startup option `-S` blocks this, and this module can be safely imported with no automatic modifications to the module search path or additions to the builtins. To explicitly trigger the usual site-specific additions, call the `main()` function.

Alterado na versão 3.3: A importação do módulo usado para acionar a manipulação de caminhos, mesmo ao usar `-S`.

It starts by constructing up to four directories from a head and a tail part. For the head part, it uses `sys.prefix` and `sys.exec_prefix`; empty heads are skipped. For the tail part, it uses the empty string and then `lib/site-packages` (on Windows) or `lib/python`*`X.Y[t]`*`/site-packages` (on Unix and macOS). (The optional suffix "t" indicates the free-threaded build, and is appended if `"t"` is present in the `sys.abiflags` constant.) For each of the distinct head-tail combinations, it sees if it refers to an existing directory, and if so, adds it to `sys.path` and also inspects the newly added path for configuration files.

Alterado na versão 3.5: Suporte para o diretório "site-python" foi removido.

Alterado na versão 3.13: No Unix, as instalações do Python threads livres são identificadas pelo sufixo “t” no nome do diretório específico da versão, como `lib/python3.13t/`.

Alterado na versão 3.14: **`site`** is no longer responsible for updating **`sys.prefix`** and **`sys.exec_prefix`** on Ambientes virtuais. This is now done during the path initialization. As a result, under Ambientes virtuais, **`sys.prefix`** and **`sys.exec_prefix`** no longer depend on the **`site`** initialization, and are therefore unaffected by **`-S`**.

When running under a virtual environment, the `pyvenv.cfg` file in **`sys.prefix`** is checked for site-specific configurations. If the `include-system-site-packages` key exists and is set to `true` (case-insensitive), the system-level prefixes will be searched for site-packages, otherwise they won’t. If the system-level prefixes are not searched then the user site prefixes are also implicitly not searched for site-packages.

The **`site`** module recognizes two startup configuration files of the form *`name`*`.pth` for path configurations, and *`name`*`.start` for pre-first-line code execution. Both files can exist in one of the four directories mentioned above. Within each directory, these files are sorted alphabetically by filename, then parsed in sorted order.

Path extensions (`.pth` files)

`name``.pth` contains additional items (one per line) to be appended to `sys.path`. Items that name non-existing directories are never added to `sys.path`, and no check is made that the item refers to a directory rather than a file. No item is added to `sys.path` more than once. Blank lines and lines beginning with `#` are skipped.

For backward compatibility, lines starting with `import` (followed by space or tab) are executed with **`exec()`**.

Alterado na versão 3.13: Os arquivos `.pth` agora são decodificados primeiro por UTF-8 e depois pela codificação da localidade se falhar.

Alterado na versão 3.15: `.pth` file lines starting with `import` are deprecated. During the deprecation period, such lines are still executed (except in the case below), but a diagnostic message is emitted only when the **`-v`** flag is given.

`import` lines in *`name`*`.pth` are silently ignored when a matching *`name`*`.start` file exists.

Errors on individual lines no longer abort processing of the rest of the file. Each error is reported and the remaining lines continue to be processed.

Descontinuado desde a versão 3.15, será removido na versão 3.20: Decoding *`name`*`.pth` files in any encoding other than `utf-8-sig` is deprecated in Python 3.15, and support for decoding from the locale encoding will be removed in Python 3.20.

`import` lines in *`name`*`.pth` files are deprecated and will be silently ignored in Python 3.18 and 3.19. In Python 3.20 a warning will be produced for `import` lines in *`name`*`.pth` files.

Startup entry points (`.start` files)

Adicionado na versão 3.15.

A startup entry point file is a file whose name has the form *`name`*`.start` and exists in one of the site-packages directories described above. Each file specifies entry points to be called during interpreter startup, using the `pkg.mod:callable` syntax understood by **`pkgutil.resolve_name()`**.

Each non-blank line that does not begin with `#` must contain an entry point reference in the form `pkg.mod:callable`. The colon and callable portion are mandatory. Each callable is invoked with no arguments, and any return value is discarded.

`.start` files are processed after all `.pth` path extensions have been applied to **`sys.path`**, ensuring that paths are available before any startup code runs.

Unlike `sys.path` extensions from `.pth` files, duplicate entry points are **not** de-duplicated — if an entry point appears more than once, it will be called more than once.

If an exception occurs during resolution or invocation of an entry point, a traceback is printed to `sys.stderr` and processing continues with the remaining entry points.

`.start` files must be encoded in UTF-8.

PEP 829 [https://peps.python.org/pep-0829/] defined the original specification for these features.

Nota

If a *name*`.start` file exists alongside a *name*`.pth` file with the same base name, any `import` lines in the `.pth` file are ignored in favor of the entry points in the `.start` file.

Nota

Executable lines (`import` lines in *name*`.pth` files and *name*`.start` file entry points) are always run at Python startup (unless `-S` is given to disable the `site.py` module entirely), regardless of whether a particular module is actually going to be used.

Nota

name`.start` files invoke `pkgutil.resolve_name()` with `strict=True`, which requires the full `pkg.mod:callable` form.

Startup file examples

For example, suppose `sys.prefix` and `sys.exec_prefix` are set to `/usr/local`. The Python X.Y library is then installed in `/usr/local/lib/python`*`X.Y`*. Suppose this has a subdirectory `/usr/local/lib/python`*`X.Y`*`/site-packages` with three sub-subdirectories, `foo`, `bar` and `spam`, and two path configuration files, `foo.pth` and `bar.pth`. Assume `foo.pth` contains the following:

```
# configuração do pacote foo

foo
bar
bletch
```

e que `bar.pth` contém:

```
# configuração do pacote bar

bar
```

Em seguida, os seguintes diretórios específicos da versão são adicionados a `sys.path`, nesta ordem:

```
/usr/local/lib/pythonX.Y/site-packages/bar
/usr/local/lib/pythonX.Y/site-packages/foo
```

Observe que `bletch` é omitido porque não existe; o diretório `bar` precede o diretório `foo` porque `bar.pth` vem em ordem alfabética antes de `foo.pth`; e `spam` é omitido porque não é mencionado em nenhum dos arquivos de configuração de caminho.

Let’s say that there is also a `foo.start` file containing the following:

```
# foo package startup code

foo.submod:initialize
```

Now, after `sys.path` has been extended as above, and before Python turns control over to user code, the `foo.submod` module is imported and the `initialize()` function from that module is called.

Migrating from `import` lines in `.pth` files to `.start` files

If your package currently ships a *`name`*`.pth` file, you can keep all `sys.path` extension lines unchanged. Only `import` lines need to be migrated.

To migrate, create a callable (taking zero arguments) within an importable module in your package. Reference it as a `pkg.mod:callable` entry point in a matching *`name`*`.start` file. Move everything on your `import` line after the first semi-colon into the `callable()` function.

If your package must straddle older Pythons that do not support **PEP 829** [https://peps.python.org/pep-0829/] and newer Pythons that do, change the `import` lines in your *`name`*`.pth` to use the following form:

```
import pkg.mod; pkg.mod.callable()
```

Older Pythons will execute these `import` lines, while newer Pythons will ignore them in favor of the *`name`*`.start` file. After the straddling period, remove all `import` lines from your `.pth` files.

`sitecustomize`

After these path manipulations, an attempt is made to import a module named **`sitecustomize`**, which can perform arbitrary site-specific customizations. It is typically created by a system administrator in the site-packages directory. If this import fails with an **`ImportError`** or its subclass exception, and the exception's **`name`** attribute equals `'sitecustomize'`, it is silently ignored. If Python is started without output streams available, as with `pythonw.exe` on Windows (which is used by default to start IDLE), attempted output from **`sitecustomize`** is ignored. Any other exception causes a silent and perhaps mysterious failure of the process.

`usercustomize`

After this, an attempt is made to import a module named **usercustomize**, which can perform arbitrary user-specific customizations, if **ENABLE_USER_SITE** is true. This file is intended to be created in the user site-packages directory (see below), which is part of `sys.path` unless disabled by **-s**. If this import fails with an **ImportError** or its subclass exception, and the exception's **name** attribute equals `'usercustomize'`, it is silently ignored.

Note that for some non-Unix systems, `sys.prefix` and `sys.exec_prefix` are empty, and the path manipulations are skipped; however the import of **sitecustomize** and **usercustomize** is still attempted.

Configuração Readline

On systems that support **readline**, this module will also import and configure the **rlcompleter** module, if Python is started in interactive mode and without the **-S** option. The default behavior is to enable tab completion and to use `~/.python_history` as the history save file. To disable it, delete (or override) the **sys.__interactivehook__** attribute in your **sitecustomize** or **usercustomize** module or your **PYTHONSTARTUP** file.

Alterado na versão 3.4: A ativação do rlcompleter e do histórico foi feita automaticamente.

Conteúdo do módulo

site.PREFIXES

Uma lista de prefixos para diretórios site-package.

site.ENABLE_USER_SITE

Sinalizador mostrando o status do diretório site-packages do usuário. `True` significa que ele está habilitado e foi adicionado a `sys.path`. `False` significa que ele foi desabilitado por solicitação do usuário (com **-s** ou **PYTHONNOUSERSITE**). `None` significa que ele foi desabilitado por motivos de segurança (incompatibilidade entre o ID do usuário ou grupo e o ID efetivo) ou por um administrador.

site.USER_SITE

Caminho para o usuário site-packages para o Python em execução. Pode ser `None` se **`getusersitepackages()`** ainda não tiver sido chamado. O valor padrão é `~/.local/lib/python`*`X.Y`*`[t]/site-packages` para construções UNIX e macOS não-framework, `~/Library/Python/`*`X.Y`*`/lib/python/site-packages` para construções macOS framework e *`%APPDATA%`*`\Python\Python`*`XY`*`\site-packages` no Windows. O "t" opcional indica a construção com threads livres. Este diretório é um diretório de site, o que significa que os arquivos `.pth` nele serão processados.

site.USER_BASE

Caminho para o diretório base para os pacotes do site do usuário. Pode ser `None` se **`getuserbase()`** ainda não tiver sido chamado. O valor padrão é `~/.local` para construções não-framework UNIX e macOS, `~/Library/Python/`*`X.Y`* para construções de framework macOS e *`%APPDATA%`*`\Python` para Windows. Este valor é usado para calcular os diretórios de instalação para scripts, arquivos de dados, módulos Python, etc. para o esquema de instalação de usuário. Veja também **`PYTHONUSERBASE`**.

site.main()

Adiciona todos os diretórios padrão específicos do site ao caminho de pesquisa de módulos. Esta função é chamada automaticamente quando este módulo é importado, a menos que o interpretador Python tenha sido iniciado com o sinalizador **`-S`**.

Alterado na versão 3.3: Esta função costumava ser chamada incondicionalmente.

site.makepath(**paths*)

Join *paths* with **`os.path.join()`**, attempt to make the result absolute with **`os.path.abspath()`**, and return a 2-tuple containing the absolute path and its case-normalized form as produced by **`os.path.normcase()`**. If **`os.path.abspath()`** raises **`OSError`**, the joined path is used unchanged for the case-normalization step.

The second element of the returned tuple is the form used throughout the **site** module to compare paths on case-insensitive file systems, and is what populates the `known_paths` sets that prevent duplicate **sys.path** entries in various APIs within this module.

class site.StartupState(*known_paths=None*)

Instances of this class accumulate interpreter startup configuration data from one or more site directories. They are the preferred interface for batching the processing of `.pth` and `.start` files across multiple site directories, so that every **sys.path** extension is visible before any startup code runs.

The optional *known_paths* argument is a set of case-normalized paths (which can be produced by **makepath()**) used to prevent duplicate **sys.path** entries. When `None` (the default), the set is built from the current **sys.path**. **main()** implicitly uses an instance of this class.

Typical use:

```
state = site.StartupState()
for sitedir in site_dirs:
    state.addsitedir(sitedir)
state.process()
```

Adicionado na versão 3.15.

addsitedir(*sitedir*)

Read the `.pth` and `.start` files in *sitedir* and record their **sys.path** extensions, deprecated `.pth import` lines, and `.start` entry points on this state. The recorded data is not applied until **process()** is called.

addusersitepackages()

Add the per-user site-packages directory, if enabled and if it exists. The directory's startup data is accumulated for later processing by **process()**.

addsitepackages(*prefixes=None*)

Add global site-packages directories, computed from *prefixes* or from the global **PREFIXES** when *prefixes* is `None`. Each directory's startup data is accumulated for later processing by **process()**.

process()

Apply the accumulated state by first adding the path extensions to **sys.path**, then executing the `.start` file entry points and `.pth` file `import` lines (deprecated).

This method is not idempotent and must not be called more than once on the same instance. Doing so will apply the accumulated state more than once, re-running entry points and `import` lines.

site.addsitedir(*sitedir*, *known_paths=None*)

Add a directory to sys.path and parse the `.pth` and `.start` files found in that directory. Typically used in **sitecustomize** or **usercustomize** (see above).

The *known_paths* argument is an optional set of case-normalized paths used to prevent duplicate **sys.path** entries. When `None` (the default), the set is built from the current **sys.path**.

For batched processing across multiple site directories, build a **StartupState** explicitly and call **StartupState.addsitedir()** on it; that defers `.pth` and `.start` processing until a single **StartupState.process()** call, ensuring every **sys.path** extension is visible before any startup code runs.

Alterado na versão 3.15: Also processes `.start` files. See Startup entry points (.start files).

site.getsitepackages(*prefixes=None*)

Retorna uma lista contendo todos os diretórios globais de pacotes de sites.

For each directory given in *prefixes* (or **PREFIXES** if *prefixes* is `None`), this function will compute its site-packages subdirectory depending on

the system environment, and will return a list of full paths, which are not checked for existence.

Adicionado na versão 3.2.

Alterado na versão 3.3: Added the optional *prefixes* parameter.

site.getuserbase()

Retorna o caminho do diretório base do usuário, **USER_BASE**. Se ele ainda não foi inicializado, esta função também o definirá, respeitando **PYTHONUSERBASE**.

Adicionado na versão 3.2.

site.getusersitepackages()

Retorna o caminho do diretório site-packages específico do usuário, **USER_SITE**. Se ele ainda não foi inicializado, esta função também o definirá, respeitando **USER_BASE**. Para determinar se o site-packages específico do usuário foi adicionado a `sys.path`, **ENABLE_USER_SITE** deve ser usado.

Adicionado na versão 3.2.

Interface de linha de comando

The **site** module also provides a way to get the user directories from the command line:

```
$ python -m site --user-site
/home/user/.local/lib/python3.11/site-packages
```

Se for chamado sem argumentos, ele vai exibir o conteúdo de **sys.path** na saída padrão, seguido pelo valor de **USER_BASE** e se o diretório existe, depois o mesmo para **USER_SITE** e, finalmente, o valor de **ENABLE_USER_SITE**.

--user-base

Exibe o caminho para o diretório base do usuário.

--user-site

Exibe o caminho para o diretório site-packages do usuário.

Se ambas as opções forem fornecidas, a base do usuário e o site do usuário serão exibidos (sempre nesta ordem), separados por `os.pathsep`.

Se qualquer opção for fornecida, o script sairá com um destes valores: `0` se o diretório site-packages do usuário estiver habilitado, `1` se ele foi desabilitado pelo usuário, `2` se ele foi desabilitado por motivos de segurança ou por um administrador, e um valor maior que 2 se houver um erro.

Ver também

- **PEP 370** [https://peps.python.org/pep-0370/] – Diretório site-packages por usuário.
- **PEP 829** [https://peps.python.org/pep-0829/] – Startup entry points and the deprecation of import lines in `.pth` files
- A inicialização do caminho de pesquisa de módulos sys.path – A inicialização de `sys.path`.

Interpretadores Python Personalizados

Os módulos descritos neste capítulo permitem a escrita de interfaces semelhantes ao interpretador interativo da Python. Se você quer um interpretador de Python que suporte algum recurso especial, além do idioma de Python, você deve olhar para o módulo `code`. (O módulo `codeop` é de nível inferior, usado para suportar compilação de um pedaço possivelmente incompleto do código Python).

A lista completa de módulos descritos neste capítulo é:

- `code` — Classes bases do interpretador
  - Objetos de interpretador interativo
  - Objetos de console Interativo
- `codeop` — Compila código Python

code — Classes bases do interpretador

Código-fonte: Lib/code.py [https://github.com/python/cpython/tree/3.15/Lib/code.py]

O módulo `code` fornece facilidades para implementar laços de leitura-execução-escrita no código Python. São incluídas duas classes e funções de conveniência que podem ser usadas para criar aplicações que fornecem um prompt de interpretador interativo.

class code.InteractiveInterpreter(*locals=None*)

Esta classe trata de análise e estado do interpretador (espaço de nomes do usuário); o mesmo não lida com buffer de entrada ou solicitação ou nomeação de arquivo de entrada (o nome do arquivo é sempre passado explicitamente). O argumento opcional *locals* especifica um mapeamento para usar como espaço de nomes no qual o código será executado; ele é padrão para um dicionário recém-criado com a chave `'__name__'` definida com `'__console__'` e a chave `'__doc__'` definida com `None`.

Observe que objetos funções e classes criados sob uma instância **`InteractiveInterpreter`** pertencerão ao espaço de nomes especificado por *locals*. Só serão selecionáveis se *locals* for o espaço de nomes de um módulo existente.

class code.InteractiveConsole(*locals=None*, *filename='<console>'*, ***, *local_exit=False*)

Emula de forma muito semelhante o comportamento do interpretador Python interativo. Esta classe baseia-se em **`InteractiveInterpreter`** e adiciona prompts usando os familiares `sys.ps1` e `sys.ps2`, e buffer de entrada. Se *local_exit* for verdadeiro, `exit()` e `quit()` no console não levantarão **`SystemExit`**, mas retornarão ao código de chamada.

Alterado na versão 3.13: Adicionado o parâmetro *local_exit*.

code.interact(*banner=None*, *readfunc=None*, *local=None*, *exitmsg=None*, *local_exit=False*)

Função de conveniência para executar um laço de leitura-execução-impressão. Isto cria uma nova instância de **`InteractiveConsole`** e define *readfunc* para ser usado como o método **`InteractiveConsole.raw_input()`**, se fornecido. Se *local* for fornecido, ele será passado para o construtor **`InteractiveConsole`** para uso como espaço de nomes padrão para o laço do interpretador. Se *local_exit* for fornecido, ele será passado para o construtor **`InteractiveConsole`**. O método **`interact()`** da instância é então executado com *banner* e *exitmsg* passados como banner e mensagem de saída a serem usados, se fornecidos. O objeto console é descartado após o uso.

Alterado na versão 3.6: Parâmetro adicionado *exitmsg*.

Alterado na versão 3.13: Adicionado o parâmetro *local_exit*.

code.compile_command(*source*, *filename='<input>'*, *symbol='single'*)

Esta função é útil para programas que desejam emular o laço principal do interpretador Python (também conhecido como laço de leitura-execução-impressão). A parte complicada é determinar quando o usuário digitou um comando incompleto que pode ser concluído inserindo mais texto (em vez de um comando completo ou um erro de sintaxe). Esta função *quase* sempre toma a mesma decisão que o laço principal do interpretador real.

source é a string fonte; *filename* é o nome do arquivo opcional do qual a fonte foi lida, sendo o padrão `'<input>'`; e *symbol* é o símbolo opcional de início da gramática, que deve ser `'single'` (o padrão), `'eval'` ou `'exec'`.

Retorna um objeto código (o mesmo que `compile(source, filename, symbol)`) se o comando for completo e válido; `None` se o comando estiver incompleto; levanta **`SyntaxError`** se o comando estiver completo

e contém um erro de sintaxe, ou levanta `OverflowError` ou `ValueError` se o comando contiver um literal inválido.

Objetos de interpretador interativo

InteractiveInterpreter.runsource(*source*, *filename='<input>'*, *symbol='single'*)

Compila e executa alguma fonte no interpretador. Os argumentos são os mesmos de `compile_command()`; o padrão para *filename* é `'<input>'`, e para *symbol* é `'single'`. Uma de várias coisas pode acontecer:

- The input is incorrect; `compile_command()` raised an exception (usually `SyntaxError`). A syntax traceback will be printed by calling the `showsyntaxerror()` method. **`runsource()`** returns `False`.
- A entrada está incompleta e são necessárias mais entradas; `compile_command()` retornou `None`. `runsource()` retorna `True`.
- A entrada está completa; `compile_command()` retornou um objeto código. O código é executado chamando `runcode()` (que também lida com exceções de tempo de execução, exceto `SystemExit`). `runsource()` retorna `False`.

O valor de retorno pode ser usado para decidir se usar `sys.ps1` ou `sys.ps2` para solicitar a próxima linha.

InteractiveInterpreter.runcode(*code*)

Executa um objeto código. Quando ocorre uma exceção, `showtraceback()` é chamado para exibir um traceback. Todas as exceções são capturadas, exceto `SystemExit`, que pode ser propagada.

Uma observação sobre `KeyboardInterrupt`: esta exceção pode ocorrer em outro lugar neste código e nem sempre pode ser detectada. O chamador deve estar preparado para lidar com isso.

InteractiveInterpreter.showsyntaxerror(*filename=None*)

Exibe o erro de sintaxe que acabou de ocorrer. Isso não exibe um stack trace (situação da pilha de execução) porque não há um para erros de

sintaxe. Se *filename* for fornecido, ele será inserido na exceção em vez do nome de arquivo padrão fornecido pelo analisador sintático do Python, porque ele sempre usa `'<string>'` ao ler uma string. A saída é escrita pelo método `write()`.

InteractiveInterpreter.showtraceback()

Exibe a exceção que acabou de ocorrer. Removemos o primeiro item da pilha porque ele está dentro da implementação do objeto interpretador. A saída é escrita pelo método `write()`.

Alterado na versão 3.5: O traceback encadeado completo é exibido em vez de apenas o traceback primário.

InteractiveInterpreter.write(*data*)

Escreve uma string no fluxo de erro padrão (`sys.stderr`). As classes derivadas devem substituir isso para fornecer o tratamento de saída apropriado conforme necessário.

Objetos de console Interativo

A classe `InteractiveConsole` é uma subclasse de `InteractiveInterpreter` e, portanto, oferece todos os métodos dos objetos interpretadores, bem como as seguintes adições.

InteractiveConsole.interact(*banner=None*, *exitmsg=None*)

Emula de forma muito semelhante o console interativo do Python. O argumento opcional *banner* especifica o banner a ser impresso antes da primeira interação; por padrão ele imprime um banner semelhante ao impresso pelo interpretador Python padrão, seguido pelo nome da classe do objeto console entre parênteses (para não confundir isso com o interpretador real – já que está tão próximo!).

O argumento opcional *exitmsg* especifica uma mensagem de saída impressa ao sair. Passe a string vazia para suprimir a mensagem de saída. Se *exitmsg* não for fornecido ou `None`, uma mensagem padrão será impressa.

Alterado na versão 3.4: Para suprimir a impressão de qualquer banner, passe uma string vazia.

Alterado na versão 3.6: Imprime uma mensagem de saída ao sair.

InteractiveConsole.push(*line*)

Envia uma linha do texto fonte para o interpretador. A linha não deve ter uma nova linha à direita; pode ter novas linhas internas. A linha é anexada a um buffer e o método **`runsource()`** do interpretador é chamado com o conteúdo concatenado do buffer como fonte. Se isso indicar que o comando foi executado ou é inválido, o buffer será redefinido; caso contrário, o comando estará incompleto e o buffer permanecerá como estava após a linha ser anexada. O valor de retorno é `True` se mais entrada for necessária, `False` se a linha foi tratada de alguma forma (isto é o mesmo que **`runsource()`**).

InteractiveConsole.resetbuffer()

Remove qualquer texto fonte não tratado do buffer de entrada.

InteractiveConsole.raw_input(*prompt=''*)

Escreve um prompt e leia uma linha. A linha retornada não inclui a nova linha final. Quando o usuário insere a sequência de teclas de fim de linha, uma exceção **`EOFError`** é levantada. A implementação base lê `sys.stdin`; uma subclasse pode substituir isso por uma implementação diferente.

codeop — Compila código Python

Código-fonte: Lib/codeop.py [https://github.com/python/cpython/tree/3.15/Lib/codeop.py]

O módulo **codeop** fornece utilitários sobre os quais o loop de leitura-execução-exibição do Python pode ser emulado, como é feito no módulo **code**. Como resultado, você provavelmente não deseja usar o módulo diretamente; se você deseja incluir tal loop em seu programa, você provavelmente deseja usar o módulo **code**.

Há duas partes para esta tarefa:

1. Ser capaz de dizer se uma linha de entrada completa uma instrução Python: em suma, dizer se deve exibir '>>>' ou '. . .' em seguida.
2. Lembrar quais instruções futuras o usuário inseriu, para que as entradas subsequentes possam ser compiladas com essas declarações em vigor.

O módulo **codeop** fornece uma maneira de fazer cada uma dessas coisas e uma maneira de fazer as duas coisas.

Para fazer apenas a primeira:

codeop.compile_command(*source*, *filename='<input>'*, *symbol='single'*)

Tenta compilar *source*, que deve ser uma string de código Python e retornar um objeto código se *source* for um código Python válido. Nesse caso, o atributo de nome de arquivo do objeto código será *filename*, cujo padrão é `'<input>'`. Retorna `None` se *source não* é um código Python válido, mas é um prefixo de código Python válido.

Se houver um problema com *source*, uma exceção será levantada. **SyntaxError** é levantada se houver sintaxe Python inválida, e **OverflowError** ou **ValueError** se houver um literal inválido.

O argumento *symbol* determina se *source* é compilado como uma instrução (`'single'`, o padrão), como uma sequência de instruções (`'exec'`) ou como uma expressão (`'eval'`). Qualquer outro valor fará com que `ValueError` seja levantada.

Nota

É possível (mas não provável) que o analisador sintático pare de analisar com um resultado bem-sucedido antes de chegar ao final da fonte; neste caso, os símbolos finais podem ser ignorados em vez de causar um erro. Por exemplo, uma barra invertida seguida por duas novas linhas pode ser seguida por lixo arbitrário. Isso será corrigido quando a API para o analisador for melhor.

class codeop.Compile

Instâncias desta classe têm métodos `__call__()` idênticos em assinatura à função embutida `compile()`, mas com a diferença de que se a instância compilar o texto do programa contendo uma instrução `__future__`, a instância se “lembra” e compila todos os textos de programa subsequentes com a instrução em vigor.

class codeop.CommandCompiler

Instâncias desta classe têm métodos `__call__()` idênticos em assinatura a `compile_command()`; a diferença é que se a instância compila o texto do programa contendo uma instrução `__future__`, a instância se “lembra” e compila todos os textos do programa subsequentes com a instrução em vigor.

Importando módulos

Os módulos descritos neste capítulo fornecem novas maneiras de importar outros módulos Python e hooks para personalizar o processo de importação.

A lista completa de módulos descritos neste capítulo é:

- `zipimport` - Importa módulos de arquivos Zip
  - Objetos zipimporter
  - Exemplos
- `pkgutil` — Utilitário de extensão de pacote
- `modulefinder` — Procura módulos usados por um script
  - Exemplo de uso de `ModuleFinder`
- `runpy` — Localizando e executando módulos Python
- `importlib` — A implementação de `import`
  - Introdução
  - Funções
  - `importlib.abc` – Abstract base classes related to import
  - `importlib.machinery` – Importers and path hooks
  - `importlib.util` – Utility code for importers
  - Exemplos
    - Importing programmatically
    - Checando se o módulo pode ser importado
    - Importa o arquivo de origem diretamente
    - Implementing lazy imports
    - Setting up an importer
    - Approximating `importlib.import_module()`
- `importlib.resources` – Leitura, abertura e acesso a recursos de pacotes
  - API funcional
- `importlib.resources.abc` – Classes base abstratas para recursos
- `importlib.metadata` – Acessando metadados do pacote
  - Visão Geral
  - API funcional
    - Pontos de entrada

        - Metadados de distribuição
        - Versões de distribuição
        - Arquivos de distribuição
        - Requisitos de distribuição
        - Mapeando importação pra pacotes de distribuição
    - Distribuições
    - Descoberta de distribuição
    - Implementando provedores personalizados
        - Exemplo
- A inicialização do caminho de pesquisa de módulos `sys.path`
    - Ambientes virtuais
    - Arquivos _pth
    - Python embarcado

`zipimport` - Importa módulos de arquivos Zip

Código-fonte: Lib/zipimport.py
[https://github.com/python/cpython/tree/3.15/Lib/zipimport.py]

Este módulo adiciona a capacidade de importar módulos Python (`*.py`, `*.pyc`) e pacotes de arquivos em formato ZIP. Normalmente, não é necessário usar o módulo **`zipimport`** explicitamente; ele é usado automaticamente pelo mecanismo embutido no **`import`** para itens **`sys.path`** que são caminhos para arquivos ZIP.

Normalmente, **`sys.path`** é uma lista de nomes de diretórios como strings. Este módulo também permite que um item de **`sys.path`** seja uma string que nomeia um arquivo ZIP. O arquivo ZIP pode conter uma estrutura de subdiretório para suportar importações de pacotes, e um caminho dentro do arquivo pode ser especificado para importar apenas de um subdiretório. Por exemplo, o caminho `example.zip/lib/` importaria apenas do subdiretório `lib/` dentro do arquivo.

Quaisquer arquivos podem estar presentes no arquivo ZIP, mas os importadores são invocados apenas para os arquivos `.py` e `.pyc`. A importação ZIP de módulos dinâmicos (`.pyd`, `.so`) não é permitida. Observe que, se um arquivo contiver apenas arquivos `.py`, o Python não tentará modificá-lo adicionando o arquivo `.pyc` correspondente, o que significa que, se um arquivo ZIP não contiver arquivos `.pyc`, a importação poderá ser bastante lenta.

Alterado na versão 3.15: Há suporte para entradas de arquivos zip compactadas com Zstandard (*zstd*).

Alterado na versão 3.13: Suporte para ZIP64.

Alterado na versão 3.8: Anteriormente, não havia suporte para arquivos ZIP com um comentário de arquivo.

Ver também

Nota da aplicação PKZIP **[https://pkware.cachefly.net/webdocs/casestudies/APPNOTE.TXT]**
Documentação do formato de arquivo ZIP feita por Phil Katz, criador do formato e dos algoritmos usados.

PEP 273 **[https://peps.python.org/pep-0273/]** **- Importar módulos de arquivos Zip**
Escrito por James C. Ahlstrom, que também forneceu uma implementação. O Python 2.3 segue a especificação em **PEP 273** [https://peps.python.org/pep-0273/], mas usa uma implementação escrita por Just van Rossum que utiliza os ganchos de importação descritos em **PEP 302** [https://peps.python.org/pep-0302/].

`importlib` **- A importação do maquinário de importação**
Pacote que fornece os protocolos relevantes para todos os importadores implementarem.

Este módulo define uma exceção:

exception zipimport.ZipImportError
Exceção levantada por objetos zipimporter. É uma subclasse de `ImportError`, portanto, também pode ser capturada como `ImportError`.

Objetos zipimporter

`zipimporter` é a classe para importar arquivos ZIP.

class zipimport.zipimporter(*archivepath*)
Cria uma nova instância de zipimporter. *archivepath* deve ser um caminho para um arquivo ZIP ou para um caminho específico dentro de um arquivo ZIP. Por exemplo, um *archivepath* de `foo/bar.zip/lib`

procurará módulos no diretório `lib` dentro do arquivo ZIP `foo/bar.zip` (desde que exista).

`ZipImportError` é levantada se *archivepath* não apontar para um arquivo ZIP válido.

Alterado na versão 3.12: Os métodos `find_loader()` e `find_module()`, descontinuados na versão 3.10, foram removidos. Em vez disso, use `find_spec()`.

create_module(*spec*)

Implementação de `importlib.abc.Loader.create_module()` que retorna `None` para solicitar explicitamente a semântica padrão.

Adicionado na versão 3.10.

exec_module(*module*)

Implementação de `importlib.abc.Loader.exec_module()`.

Adicionado na versão 3.10.

find_spec(*fullname*, *target=None*)

Uma implementação de `importlib.abc.PathEntryFinder.find_spec()`.

Adicionado na versão 3.10.

get_code(*fullname*)

Retorna o objeto código para o módulo especificado. Levanta `ZipImportError` se o módulo não puder ser importado.

get_data(*pathname*)

Retorna os dados associados a *pathname*. Levanta `OSError` se o arquivo não for encontrado.

Alterado na versão 3.3: `IOError` costumava ser levantada, agora ela é um apelido para `OSError`.

get_filename(*fullname*)

Retorna o valor que `__file__` seria definido se o módulo especificado fosse importado. Levanta **`ZipImportError`** se o módulo não puder ser importado.

Adicionado na versão 3.1.

get_source(*fullname*)

Retorna o código-fonte do módulo especificado. Levanta **`ZipImportError`** se o módulo não puder ser encontrado, e retorna **`None`** se o arquivo contiver o módulo, mas não tiver o código-fonte para ele.

is_package(*fullname*)

Retorna `True` se o módulo especificado por *fullname* é um pacote. Levanta **`ZipImportError`** se o módulo não pode ser localizado.

invalidate_caches()

Limpa o cache interno de informações sobre arquivos encontrados no arquivo ZIP.

Adicionado na versão 3.10.

archive

O nome do arquivo ZIP associado ao importador, sem um possível subcaminho.

prefix

O subcaminho dentro do arquivo ZIP onde os módulos são pesquisados. Esta é a string vazia para objetos zipimporter que apontam para a raiz do arquivo ZIP.

Os atributos **`archive`** e **`prefix`**, quando combinados com uma barra, são iguais ao argumento original *archivepath* fornecido ao construtor da **`zipimporter`**.

Exemplos

Aqui está um exemplo que importa um módulo de um arquivo ZIP - observe que o módulo **`zipimport`** não é usado explicitamente.

```
$ unzip -l arquivo_exemplo.zip
Archive:  arquivo_exemplo.zip
  Length     Date   Time    Name
 --------    ----   ----    ----
     8467  01-01-00 12:30   exemplo.py
 --------                   -------
     8467                   1 file
```

```
>>> import sys
>>> # Adiciona o arquivo para a frente do caminho de pesquisa de
>>> sys.path.insert(0, 'arquivo_exemplo.zip')
>>> import exemplo
>>> exemplo.__file__
'arquivo_exemplo.zip/exemplo.py'
```

`pkgutil` — Utilitário de extensão de pacote

Código-fonte: Lib/pkgutil.py [https://github.com/python/cpython/tree/3.15/Lib/pkgutil.py]

Este módulo fornece utilitários para o sistema de importação, em particular suporte a pacotes.

class pkgutil.ModuleInfo(*module_finder*, *name*, *ispkg*)

Um namedtuple que contém um breve resumo das informações de um módulo.

Adicionado na versão 3.6.

pkgutil.extend_path(*path*, *name*)

Estende o caminho de pesquisa para os módulos que compõem um pacote. O uso pretendido é colocar o seguinte código no `__init__.py` de um pacote:

```
from pkgutil import extend_path
__path__ = extend_path(__path__, __name__)
```

Para cada diretório em `sys.path` que tenha um subdiretório que corresponda ao nome do pacote, adiciona o subdiretório ao `__path__` do pacote. Isto é útil se quisermos distribuir diferentes partes de um único pacote lógico como múltiplos diretórios.

Também procura por arquivos `*.pkg` que começam onde `*` corresponde ao argumento *name*. Este recurso é semelhante aos arquivos `*.pth` (consulte o módulo `site` para mais informações), exceto que não considera casos especiais para linhas que começam com `import`. Um arquivo `*.pkg` é confiável por natureza: além de pular linhas em branco e ignorar comentários, todas as entradas encontradas em um arquivo

`*.pkg` são adicionadas ao caminho, independentemente de existirem ou não no sistema de arquivos (este é um recurso).

Se o caminho de entrada não for uma lista (como é o caso de pacotes congelados), ele será retornado inalterado. O caminho de entrada não é modificado; uma cópia estendida é retornada. Os itens são anexados à cópia apenas no final.

Presume-se que `sys.path` seja uma sequência. Itens de `sys.path` que não sejam strings referentes a diretórios existentes são ignorados. Itens Unicode em `sys.path` que causam erros quando usados como nomes de arquivo podem fazer com que esta função levante uma exceção (em linha com o comportamento de `os.path.isdir()`).

pkgutil.get_importer(*path_item*)

Recupera um localizador para o *path_item* fornecido.

O localizador retornado é armazenado em cache em `sys.path_importer_cache` se ele foi criado recentemente por um gancho de caminho.

O cache (ou parte dele) pode ser limpo manualmente se uma nova varredura de `sys.path_hooks` for necessária.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em `importlib` em vez de depender da emulação de importação interna do pacote da **PEP 302** [https://peps.python.org/pep-0302/].

pkgutil.iter_importers(*fullname=''*)

Produz objetos localizador para o nome do módulo fornecido.

Se *fullname* contiver `'.'`, os localizadores serão para o pacote que contém *fullname*, caso contrário, serão todos os localizadores de nível superior registrados (ou seja, aqueles em `sys.meta_path` e `sys.path_hooks`).

Se o módulo nomeado estiver em um pacote, esse pacote será importado como um efeito colateral da invocação desta função.

Se nenhum nome de módulo for especificado, todos os localizadores de nível superior serão produzidos.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em `importlib` em vez de depender da emulação de importação interna do pacote da **PEP 302** [https://peps.python.org/pep-0302/].

pkgutil.iter_modules(*path=None*, *prefix=''*)

Levanta `ModuleInfo` para todos os submódulos em *path* ou, se *path* for `None`, todos os módulos de nível superior em `sys.path`.

path deve ser `None` ou uma lista de caminhos para procurar módulos.

prefix é uma string a ser exibida na frente de cada nome de módulo na saída.

Nota

Funciona apenas para um localizador que define um método `iter_modules()`. Esta interface não é padrão, portanto, o módulo também fornece implementações para `importlib.machinery.FileFinder` e `zipimport.zipimporter`.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em `importlib` em vez de depender da emulação de importação interna do pacote da **PEP 302** [https://peps.python.org/pep-0302/].

pkgutil.walk_packages(*path=None*, *prefix=''*, *onerror=None*)

Levanta `ModuleInfo` para todos os módulos recursivamente em *path*, ou, se *path* for `None`, todos os módulos acessíveis.

path deve ser `None` ou uma lista de caminhos para procurar módulos.

prefix é uma string a ser exibida na frente de cada nome de módulo na saída.

Observe que esta função deve importar todos os *pacotes* (*não* todos os módulos!) no caminho *path* fornecido, para acessar o atributo `__path__`

para encontrar submódulos.

onerror é uma função que é chamada com um argumento (o nome do pacote que estava sendo importado) caso ocorra alguma exceção durante a tentativa de importação de um pacote. Se nenhuma função *onerror* for fornecida, `ImportError`s são capturadas e ignoradas, enquanto todas as outras exceções são propagadas, encerrando a busca.

Exemplos:

```
# lista todos os módulos que o python consegue acessars
walk_packages()

# lista todos os submódulos de ctypes
walk_packages(ctypes.__path__, ctypes.__name__ + '.')
```

Nota

Funciona apenas para um localizador que define um método `iter_modules()`. Esta interface não é padrão, portanto, o módulo também fornece implementações para `importlib.machinery.FileFinder` e `zipimport.zipimporter`.

Alterado na versão 3.3: Atualizado para ser baseado diretamente em `importlib` em vez de depender da emulação de importação interna do pacote da **PEP 302** [https://peps.python.org/pep-0302/].

pkgutil.get_data(*package*, *resource*)

Obtém um recurso de um pacote.

Este é um invólucro para a API `get_data` carregadora. O argumento *package* deve ser o nome de um pacote, no formato de módulo padrão (`foo.bar`). O argumento *resource* deve estar no formato de um nome de arquivo relativo, usando `/` como separador de caminho.

A função retorna uma string binária que é o conteúdo do recurso especificado.

Esta função utiliza o método carregador **`get_data()`** para dar suporte a módulos instalados no sistema de arquivos, mas também em arquivos zip, bancos de dados ou em outros locais.

Para pacotes localizados no sistema de arquivos, que já foram importados, isso é o equivalente aproximado de:

```
d = os.path.dirname(sys.modules[package].__file__)
data = open(os.path.join(d, resource), 'rb').read()
```

Assim como a função **`open()`**, **`get_data()`** pode seguir diretórios pai (`../`) e caminhos absolutos (começando com `/` ou `C:/`, por exemplo). Ela pode abrir artefatos de compilação/instalação como arquivos `.py` e `.pyc` ou arquivos com **`nomes_reservados`**. Para ser compatível com carregadores que não utilizam o sistema de arquivos, evite usar esses recursos.

Aviso

Esta função destina-se a entradas confiáveis. Ela não verifica se o *resource* "pertence" ao *package*.

Se você usar um caminho de *resource* fornecido pelo usuário, considere verificá-lo. Por exemplo, exija um nome de arquivo alfanumérico com uma extensão conhecida ou instale e verifique uma lista de recursos conhecidos.

Se o pacote não puder ser localizado ou carregado, ou se usar um carregador que não oferece suporte a **`get_data`**, `None` será retornado. Em particular, o carregador para pacotes de espaço de nomes não oferece suporte a **`get_data`**.

Ver também

O módulo **`importlib.resources`** fornece acesso estruturado para recursos de módulo.

pkgutil.resolve_name(*name*, *, *strict=False*)

Resolve um nome a um objeto.

Essa funcionalidade é usada em vários lugares na biblioteca padrão (veja bpo-12915 [https://bugs.python.org/issue?@action=redirect&bpo=12915]) - e funcionalidades equivalentes também estão amplamente utilizadas em pacotes de terceiros, como setuptools, Django e Pyramid.

Espera-se que *name* seja uma string em um dos seguintes formatos, onde W é uma abreviação para um identificador Python válido e ponto representa um ponto literal nessas pseudo-regexes:

- `W(.W)*`
- `W(.W)*:(W(.W)*)?`
- `W(.W)*:(W(.W)*)`

A primeira forma destina-se apenas à retrocompatibilidade. Ela presume que parte do nome pontilhado seja um pacote e o restante seja um objeto em algum lugar dentro desse pacote, possivelmente aninhado dentro de outros objetos. Como o ponto onde o pacote termina e a hierarquia de objetos começa não pode ser inferido por inspeção, tentativas repetidas de importação devem ser feitas com esta forma.

Na segunda forma, o chamador deixa claro o ponto de divisão por meio de um caractere de dois pontos: o nome pontilhado à esquerda do caractere de dois pontos representa um pacote a ser importado, e o nome pontilhado à direita representa a hierarquia de objetos dentro desse pacote. Apenas uma importação é necessária nesta forma. Se terminar com caractere de dois pontos, um objeto de módulo será retornado.

As duas primeiras formas são aceitas quando `strict=False` (o padrão).

A terceira forma requer tanto o nome do módulo quanto o objeto chamável, separados por dois-pontos. Apenas esta forma é aceita quando `strict=True`.

A função retornará um objeto (que pode ser um módulo) ou levantará uma das seguintes exceções:

`ValueError` – se *name* não estiver em um formato reconhecido.

`ImportError` – se uma importação falhou quando não deveria.

`AttributeError` – Se ocorrer uma falha ao percorrer a hierarquia de objetos dentro do pacote importado para chegar ao objeto desejado.

Adicionado na versão 3.9.

Alterado na versão 3.15: A opção `strict` somente-nomeada opcional foi adicionada.

`modulefinder` — Procura módulos usados por um script

Código-fonte: Lib/modulefinder.py
[https://github.com/python/cpython/tree/3.15/Lib/modulefinder.py]

Este módulo fornece uma classe **`ModuleFinder`** que pode ser usada para determinar o conjunto de módulos importados por um script. O `modulefinder.py` também pode ser executado como um script, fornecendo o nome do arquivo de um script Python como seu argumento, após o qual um relatório dos módulos importados será impresso.

modulefinder.AddPackagePath(*pkg_name*, *path*)

Registra que o pacote chamado *pkg_name* pode ser encontrado no caminho especificado em *path*

modulefinder.ReplacePackage(*oldname*, *newname*)

Permite especificar que o módulo chamado *oldname* é de fato o pacote chamado *newname*.

class modulefinder.ModuleFinder(*path=None*, *debug=0*, *excludes=[]*, *replace_paths=[]*)

Esta classe fornece os métodos **`run_script()`** e **`report()`** para determinar o conjunto de módulos importados por um script. *path* pode ser uma lista de diretórios para procurar por módulos; se não especificado, `sys.path` é usado. *debug* define o nível de depuração; valores mais altos fazem a classe imprimir mensagens de depuração sobre o que está fazendo. *excludes* é uma lista de nomes de módulos a serem excluídos da análise. *replace_paths* é uma lista de tuplas `(oldpath, newpath)` que serão substituídas nos caminhos dos módulos.

report()

Imprime um relatório na saída padrão que lista os módulos importados pelo script e seus caminhos, bem como os módulos que estão faltando ou parecem estar ausentes.

run_script(*pathname*)

Analisa o conteúdo do arquivo *pathname*, que deve conter o código Python.

modules

Um nome de módulo de mapeamento de dicionário para módulos. Veja Exemplo de uso de ModuleFinder.

Exemplo de uso de `ModuleFinder`

O script que será analisado posteriormente (bacon.py):

```
import re, itertools

try:
    import baconhameggs
except ImportError:
    pass

try:
    import guido.python.ham
except ImportError:
    pass
```

O script que irá gerar o relatório de bacon.py:

```
from modulefinder import ModuleFinder

finder = ModuleFinder()
finder.run_script('bacon.py')

print('Loaded modules:')
for name, mod in finder.modules.items():
    print('%s: ' % name, end='')
    print(','.join(list(mod.globalnames.keys())[:3]))
```

```
print('-'*50)
print('Modules not imported:')
print('\n'.join(finder.badmodules.keys()))
```

Exemplo de saída (pode variar dependendo da arquitetura):

```
Loaded modules:
_types:
copyreg:  _inverted_registry,_slotnames,__all__
re._compiler:  isstring,_sre,_optimize_unicode
_sre:
re._constants:  REPEAT_ONE,makedict,AT_END_LINE
sys:
re:  __module__,finditer,_expand
itertools:
__main__:  re,itertools,baconhameggs
re._parser:  _PATTERNENDERS,SRE_FLAG_UNICODE
array:
types:  __module__,IntType,TypeType
--------------------------------------------------
Modules not imported:
guido.python.ham
baconhameggs
```

`runpy` — Localizando e executando módulos Python

Código-fonte: Lib/runpy.py [https://github.com/python/cpython/tree/3.15/Lib/runpy.py]

The `runpy` module is used to locate and run Python modules without importing them first. Its main use is to implement the `-m` command line switch that allows scripts to be located using the Python module namespace rather than the filesystem.

Observe que este *não* é um módulo isolado - todo o código é executado no processo atual, e quaisquer efeitos colaterais (como importações em cache de outros módulos) irão permanecer em vigor após o retorno da função.

Furthermore, any functions and classes defined by the executed code are not guaranteed to work correctly after a `runpy` function has returned. If that limitation is not acceptable for a given use case, `importlib` is likely to be a more suitable choice than this module.

The `runpy` module provides two functions:

runpy.run_module(*mod_name*, *init_globals=None*, *run_name=None*, *alter_sys=False*)

Executa o código do módulo especificado e retorna o dicionário de globals do módulo resultante. O código do módulo é localizado primeiro usando o mecanismo de importação padrão (consulte **PEP 302** [https://peps.python.org/pep-0302/] para detalhes) e então executado em um novo espaço de nomes de módulo.

O argumento *mod_name* deve ser um nome de módulo absoluto. Se o nome do módulo se referir a um pacote ao invés de um módulo normal, então esse pacote é importado e o submódulo `__main__` dentro desse

pacote é então executado e o dicionário global do módulo resultante retornado.

O argumento opcional de dicionário *init_globals* pode ser usado para preencher previamente o dicionário global do módulo antes do código ser executado. *init_globals* não será modificado. Se qualquer uma das variáveis globais especiais abaixo for definida em *init_globals*, estas definições serão substituídas por **`run_module()`**.

The special global variables `__name__`, `__spec__`, `__file__`, `__loader__` and `__package__` are set in the globals dictionary before the module code is executed. (Note that this is a minimal set of variables - other variables may be set implicitly as an interpreter implementation detail.)

`__name__` é definido como *run_name* se este argumento opcional não for **`None`**, para `mod_name + '.__main__'` se o módulo nomeado for um pacote e para o argumento *mod_name* caso contrário .

`__spec__` será definido adequadamente para o módulo *efetivamente* importado (isto é, `__spec__.name` vai sempre ser *mod_name* ou `mod_name + '.__main__'`, nunca *run_name*).

`__file__`, `__loader__` and `__package__` are set as normal based on the module spec.

Se o argumento *alter_sys* for fornecido e for avaliado como **`True`**, então `sys.argv[0]` será atualizado com o valor de `__file__` e `sys.modules[__name__]` é atualizado com um objeto de módulo temporário para o módulo que está sendo executado. Ambos `sys.argv[0]` e `sys.modules[__name__]` são restaurados para seus valores originais antes que a função retorne.

Note que esta manipulação de **`sys`** não é segura para thread. Outras threads podem ver o módulo parcialmente inicializado, bem como a lista alterada de argumentos. É recomendado que o módulo `sys` seja deixado sozinho ao invocar esta função a partir do código encadeado.

Ver também

A opção `-m` oferece funcionalidade equivalente na linha de comando.

Alterado na versão 3.1: Adicionada capacidade de executar pacotes procurando por um submódulo `__main__`.

Alterado na versão 3.2: Adicionada a variável global `__cached__` (veja **PEP 3147** [https://peps.python.org/pep-3147/]).

Alterado na versão 3.4: Atualizado para aproveitar o recurso de spec de módulo adicionado por **PEP 451** [https://peps.python.org/pep-0451/]. Isso permite que `__cached__` seja configurado corretamente para módulos executados desta forma, assim como garante que o nome real do módulo esteja sempre acessível como `__spec__.name`.

Alterado na versão 3.12: A definição de `__cached__`, `__loader__` e `__package__` foi descontinuada. Veja `ModuleSpec` para alternativas.

Alterado na versão 3.15: The global variable `__cached__` is no longer set.

runpy.run_path(*path_name, init_globals=None, run_name=None*)

Executa o código no local do sistema de arquivos nomeado e retorna o dicionário de globals do módulo resultante. Assim como um nome de script fornecido à linha de comando CPython, *file_path* pode se referir a um arquivo fonte Python, um arquivo compilado em bytecode ou uma entrada válida de `sys.path` contendo um módulo `__main__` (por exemplo, um arquivo zip contendo um arquivo de nível superior `__main__.py`).

Para um script simples, o código especificado é simplesmente executado em um novo espaço de nomes de módulo. Para uma entrada válida `sys.path` (normalmente um arquivo zip ou diretório), a entrada é primeiro adicionada ao início de `sys.path`. A função então procura e executa um módulo `__main__` usando o caminho atualizado. Observe que não há proteção especial contra invocar uma entrada `__main__`

existente localizada em outro lugar em `sys.path` se não houver tal módulo no local especificado.

O argumento opcional de dicionário *init_globals* pode ser usado para preencher previamente o dicionário de globals do módulo antes do código ser executado. *init_globals* não será modificado. Se qualquer uma das variáveis globais especiais abaixo for definida em *init_globals*, estas definições serão substituídas por **`run_path()`**.

The special global variables `__name__`, `__spec__`, `__file__`, `__loader__` and `__package__` are set in the globals dictionary before the module code is executed. (Note that this is a minimal set of variables - other variables may be set implicitly as an interpreter implementation detail.)

`__name__` é definido como *run_name* se este argumento opcional não for **`None`** e como `'<run_path>'` caso contrário.

If *file_path* directly references a script file (whether as source or as precompiled byte code), then `__file__` will be set to *file_path*, and `__spec__`, `__loader__` and `__package__` will all be set to **`None`**.

If *file_path* is a reference to a valid **`sys.path`** entry, then `__spec__` will be set appropriately for the imported **`__main__`** module (that is, `__spec__.name` will always be `__main__`). `__file__`, `__loader__` and `__package__` will be set as normal based on the module spec.

Várias alterações também são feitas no módulo **`sys`**. Primeiramente, **`sys.path`** pode ser alterado conforme descrito acima. `sys.argv[0]` é atualizado com o valor de *file_path* e `sys.modules[__name__]` é atualizado com um objeto de módulo temporário para o módulo que está sendo executado. Todas as modificações em itens em **`sys`** são revertidas antes que a função retorne.

Note que, diferentemente de **`run_module()`**, as alterações feitas em **`sys`** não são opcionais nesta função, pois esses ajustes são essenciais para permitir a execução das entradas do **`sys.path`**. Como as limitações de segurança de threads ainda se aplicam, o uso dessa função no código

encadeado deve ser serializado com a trava de importação ou delegado a um processo separado.

Ver também

Opções de interface para funcionalidade equivalente na linha de comando (`python path/to/script`).

Adicionado na versão 3.2.

Alterado na versão 3.4: Atualizado para aproveitar o recurso de spec de módulo adicionado por **PEP 451** [https://peps.python.org/pep-0451/]. Isso permite que `__cached__` seja definido corretamente no caso em que `__main__` é importado de uma entrada `sys.path` válida em vez de ser executado diretamente.

Alterado na versão 3.12: A definição de `__cached__`, `__loader__` e `__package__` foi descontinuada.

Alterado na versão 3.15: The global variable `__cached__` is no longer set.

Ver também

PEP 338 [https://peps.python.org/pep-0338/] – Executando módulos como scripts
PEP escrita e implementada por Nick Coghlan.

PEP 366 [https://peps.python.org/pep-0366/] – Importações relativas explícitas do módulo principal
PEP escrita e implementada por Nick Coghlan.

PEP 451 [https://peps.python.org/pep-0451/] – Um tipo ModuleSpec para o sistema de importação
PEP escrita e implementada por Eric Snow

Linha de comando e ambiente - Detalhes da linha de comando do CPython

A função `importlib.import_module()`

`importlib` — A implementação de `import`

Adicionado na versão 3.1.

Código-fonte: Lib/importlib/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/importlib/__init__.py]

Introdução

The purpose of the **`importlib`** package is three-fold.

Uma é fornecer a implementação da instrução **`import`** (e, portanto, por extensão, a função **`__import__()`**) no código-fonte Python. Isso fornece uma implementação de **`import`** que é portável para qualquer interpretador Python. Isso também fornece uma implementação que é mais fácil de compreender do que aquela implementada em uma linguagem de programação diferente do Python.

Dois, os componentes para implementar **`import`** são expostos neste pacote, tornando mais fácil para os usuários criarem seus próprios objetos personalizados (conhecidos genericamente como importador) para participar do processo de importação.

Terceiro, o pacote contém módulos que expõem funcionalidades adicionais para gerenciar aspectos de pacotes Python:

- **`importlib.metadata`** apresenta acesso a metadados de distribuições de terceiros.
- **`importlib.resources`** fornece rotinas para acessar “recursos” não codificados de pacotes Python.

Ver também

A instrução import

A referência da linguagem para a instrução `import`.

Especificação dos pacotes **[https://www.python.org/doc/essays/packages/]**

Especificação original dos pacotes. Algumas semânticas mudaram desde a redação deste documento (por exemplo, redirecionamento baseado em `None` em `sys.modules`).

A função `__import__()`

A instrução `import` é um açúcar sintático para esta função.

A inicialização do caminho de pesquisa de módulos sys.path

A inicialização de `sys.path`.

PEP 235 **[https://peps.python.org/pep-0235/]**

Importação em plataformas que diferenciam maiúsculo de minúsculo

PEP 263 **[https://peps.python.org/pep-0263/]**

Definindo codificações do código-fonte do Python

PEP 302 **[https://peps.python.org/pep-0302/]**

Novos ganchos de importação

PEP 328 **[https://peps.python.org/pep-0328/]**

Importações: Multilinha e Absoluto/Relativo

PEP 366 **[https://peps.python.org/pep-0366/]**

Importações relativas explícitas do módulo principal

PEP 420 **[https://peps.python.org/pep-0420/]**

Pacotes de espaço de nomes implícitos

PEP 451 **[https://peps.python.org/pep-0451/]**

Um tipo de ModuleSpec para o sistema de importação

PEP 488 **[https://peps.python.org/pep-0488/]**

Eliminação de arquivos PYO

PEP 489 **[https://peps.python.org/pep-0489/]**

Inicialização de módulo extensão multifase

PEP 552 **[https://peps.python.org/pep-0552/]**
pycs determinísticos

PEP 3120 **[https://peps.python.org/pep-3120/]**
Usando UTF-8 como fonte padrão de codificação

PEP 3147 **[https://peps.python.org/pep-3147/]**
Diretórios de repositório de PYC

Funções

importlib.__import__(*name, globals=None, locals=None, fromlist=(), level=0*)

Uma implementação da função embutida `__import__()`.

Nota

A importação programática de módulos deve usar `import_module()` em vez desta função.

importlib.import_module(*name, package=None*)

Importa um módulo. O argumento *name* especifica qual módulo importar em termos absolutos ou relativos (por exemplo, `pkg.mod` ou `..mod`). Se o nome for especificado em termos relativos, então o argumento *package* deve ser definido como o nome do pacote que atuará como âncora para resolver o nome do pacote (por exemplo, `import_module('..mod', 'pkg.subpkg')` importará `pkg.mod`).

A função `import_module()` atua como um wrapper simplificador em torno de `importlib.__import__()`. Isso significa que toda a semântica da função é derivada de **`importlib.__import__()`**. A diferença mais importante entre essas duas funções é que **`import_module()`** retorna o pacote ou módulo especificado (por exemplo, `pkg.mod`), enquanto

`__import__()` retorna o pacote ou módulo de nível superior (por exemplo, `pkg`).

Se você estiver importando dinamicamente um módulo que foi criado desde que o interpretador iniciou a execução (por exemplo, criou um arquivo fonte Python), você pode precisar chamar `invalidate_caches()` para que o novo módulo seja notado pelo sistema de importação.

If the module cannot be imported, **`import_module()`** will raise `ImportError` or an appropriate subclass like `ModuleNotFoundError`.

Alterado na versão 3.3: Os pacotes pai são importados automaticamente.

importlib.invalidate_caches()

Invalida os caches internos dos localizadores armazenados em `sys.meta_path`. Se um localizador implementar `invalidate_caches()` então ele será chamado para realizar a invalidação. Esta função deve ser chamada se algum módulo for criado/instalado enquanto seu programa estiver em execução para garantir que todos os localizadores notarão a existência do novo módulo.

Adicionado na versão 3.3.

Alterado na versão 3.10: Pacotes de espaço de nomes criados/instalados em um local `sys.path` diferente após o mesmo espaço de nomes já ter sido importado são notados.

importlib.reload(*module*)

Recarrega um *module* importado anteriormente. O argumento deve ser um objeto módulo, portanto deve ter sido importado com êxito antes. Isso é útil se você editou o arquivo fonte do módulo usando um editor externo e deseja experimentar a nova versão sem sair do interpretador Python. O valor de retorno é o objeto módulo (que pode ser diferente se a reimportação fizer com que um objeto diferente seja colocado em `sys.modules`).

Quando `reload()` é executado:

- O código do módulo Python é recompilado e o código em nível de módulo é reexecutado, definindo um novo conjunto de objetos que são vinculados a nomes no dicionário do módulo reutilizando o carregador que originalmente carregou o módulo. A função `init` dos módulos de extensão não é chamada uma segunda vez.
- Tal como acontece com todos os outros objetos em Python, os objetos antigos só são recuperados depois que suas contagens de referências caem para zero.
- Os nomes no espaço de nomes do módulo são atualizados para apontar para quaisquer objetos novos ou alterados.
- Outras referências aos objetos antigos (como nomes externos ao módulo) não são religadas para se referir aos novos objetos e devem ser atualizadas em cada espaço de nomes onde ocorrem, se isso for desejado.

Existem várias outras ressalvas:

Quando um módulo é recarregado, seu dicionário (contendo as variáveis globais do módulo) é retido. As redefinições de nomes vão substituir as definições antigas, portanto isso geralmente não é um problema. Se a nova versão de um módulo não definir um nome definido pela versão antiga, a definição antiga permanecerá. Este recurso pode ser usado para vantagem do módulo se ele mantiver uma tabela global ou cache de objetos – com uma instrução `try` ele pode testar a presença da tabela e pular sua inicialização se desejar:

```
try:
    cache
except NameError:
    cache = {}
```

Geralmente não é muito útil recarregar módulos embutidos ou carregados dinamicamente. Recarregar `sys`, `__main__`, `builtins` e outros módulos principais não é recomendado. Em muitos casos, os módulos de extensão não são projetados para serem inicializados mais de uma vez e podem falhar de maneiras arbitrárias quando recarregados.

Se um módulo importa objetos de outro módulo usando `from` … `import` …, chamar `reload()` para o outro módulo não redefine os objetos importados dele – uma maneira de contornar isso é executar novamente a instrução **`from`**, outra é usar **`import`** e nomes qualificados (*module.name*).

Se um módulo instancia instâncias de uma classe, recarregar o módulo que define a classe não afeta as definições de método das instâncias – elas continuam a usar a definição de classe antiga. O mesmo se aplica às classes derivadas.

Adicionado na versão 3.4.

Alterado na versão 3.7: `ModuleNotFoundError` é levantada quando o módulo que está sendo recarregado não possui um `ModuleSpec`.

Aviso

This function is not thread-safe. Calling it from multiple threads can result in unexpected behavior. It's recommended to use the `threading.Lock` or other synchronization primitives for thread-safe module reloading.

`importlib.abc` – Abstract base classes related to import

Código-fonte: Lib/importlib/abc.py
[https://github.com/python/cpython/tree/3.15/Lib/importlib/abc.py]

The **`importlib.abc`** module contains all of the core abstract base classes used by `import`. Some subclasses of the core abstract base classes are also provided to help in implementing the core ABCs.

Hierarquia de ABC:

```
object
 +-- MetaPathFinder
 +-- PathEntryFinder
 +-- Loader
      +-- ResourceLoader --------+
      +-- InspectLoader          |
           +-- ExecutionLoader --+
                                 +-- FileLoader
                                 +-- SourceLoader
```

class importlib.abc.MetaPathFinder

Uma classe base abstrata que representa um localizador de metacaminho.

Adicionado na versão 3.3.

Alterado na versão 3.10: Não é mais uma subclasse de **`Finder`**.

find_spec(*fullname*, *path*, *target=None*)

Um método abstrato para encontrar um spec para o módulo especificado. Se esta for uma importação de nível superior, *path* será `None`. Caso contrário, esta é uma busca por um subpacote ou módulo e *path* será o valor de **`__path__`** do pacote pai. Se um spec não puder ser encontrado, `None` será retornado. Quando passado, `target` é um objeto de módulo que o localizador pode usar para fazer uma estimativa mais informada sobre qual spec retornar. **`importlib.util.spec_from_loader()`** pode ser útil para implementar `MetaPathFinders` concretos.

Adicionado na versão 3.4.

invalidate_caches()

Um método opcional que, quando chamado, deve invalidar qualquer cache interno usado pelo localizador. Usado por **`importlib.invalidate_caches()`** ao invalidar os caches de todos os localizadores em **`sys.meta_path`**.

Alterado na versão 3.4: Retorna `None` quando chamado em vez de **`NotImplemented`**.

discover(*parent=None*)

An optional method which searches for possible specs with given *parent* module spec. If *parent* is *None*, **`MetaPathFinder.discover()`** will search for top-level modules.

Returns an iterable of possible specs.

Raises **`ValueError`** if *parent* is not a package module.

Aviso

This method can potentially yield a very large number of objects, and it may carry out IO operations when computing these values.

Because of this, it will generally be desirable to compute the result values on-the-fly, as they are needed. As such, the returned object is only guaranteed to be an **`iterable`**, instead of a **`list`** or other **`collection`** type.

Adicionado na versão 3.15.

class importlib.abc.PathEntryFinder

Uma classe base abstrata que representa um localizador de entrada de caminho. Embora tenha algumas semelhanças com **`MetaPathFinder`**, `PathEntryFinder` deve ser usado apenas dentro do subsistema de importação baseado em caminho fornecido por **`importlib.machinery.PathFinder`**.

Adicionado na versão 3.3.

Alterado na versão 3.10: Não é mais uma subclasse de **`Finder`**.

find_spec(*fullname*, *target=None*)

Um método abstrato para encontrar um spec para o módulo especificado. O localizador irá procurar pelo módulo apenas dentro do entrada de caminho ao qual ele está atribuído. Se um spec não puder ser encontrado, `None` será retornado. Quando passado, `target` é um objeto de módulo que o localizador pode usar para fazer uma estimativa mais informada sobre qual spec retornar. **`importlib.util.spec_from_loader()`** pode ser útil para implementar `PathEntryFinders` concretos.

Adicionado na versão 3.4.

invalidate_caches()

Um método opcional que, quando chamado, deve invalidar qualquer cache interno usado pelo localizador. Usado por **`importlib.machinery.PathFinder.invalidate_caches()`** ao invalidar os caches de todos os localizadores em cache.

discover(*parent=None*)

An optional method which searches for possible specs with given *parent* module spec. If *parent* is *None*, **`PathEntryFinder.discover()`** will search for top-level modules.

Returns an iterable of possible specs.

Raises **`ValueError`** if *parent* is not a package module.

Aviso

This method can potentially yield a very large number of objects, and it may carry out IO operations when computing these values.

Because of this, it will generally be desirable to compute the result values on-the-fly, as they are needed. As such, the returned object is only guaranteed to be an **`iterable`**, instead of a **`list`** or other **`collection`** type.

Adicionado na versão 3.15.

class importlib.abc.Loader

Uma classe base abstrata para um carregador. Veja **PEP 302** [https://peps.python.org/pep-0302/] para a definição exata de um carregador.

Loaders that wish to support resource reading should implement a **`get_resource_reader()`** method as specified by **`importlib.resources.abc.ResourceReader`**.

Alterado na versão 3.7: Introduced the optional **`get_resource_reader()`** method.

Alterado na versão 3.15: Removed the `load_module()` method.

create_module(*spec*)

Um método que retorna o objeto do módulo a ser usado ao importar um módulo. Este método pode retornar `None`, indicando que a semântica padrão de criação do módulo deve ocorrer.

Adicionado na versão 3.4.

Alterado na versão 3.6: Este método não é mais opcional quando **`exec_module()`** é definido.

exec_module(*module*)

Um método abstrato que executa o módulo em seu próprio espaço de nomes quando um módulo é importado ou recarregado. O módulo já deve estar inicializado quando **`exec_module()`** for chamado. Quando este método existir, **`create_module()`** deve ser definido.

Adicionado na versão 3.4.

Alterado na versão 3.6: **`create_module()`** deve ser definido.

class importlib.abc.ResourceLoader

Substituída por TraversableResources

Uma classe base abstrata para um carregador que implementa o protocolo opcional **PEP 302** [https://peps.python.org/pep-0302/] para carregar recursos arbitrários do back-end de armazenamento.

Descontinuado desde a versão 3.7: This ABC is deprecated in favour of supporting resource loading through `importlib.resources.abc.TraversableResources`. This class exists for backwards compatibility only with other ABCs in this module.

abstractmethod get_data(*path*)

Um método abstrato para retornar os bytes dos dados localizados em *path*. Carregadores que possuem um backend de armazenamento arquivo ou similar que permite o armazenamento de dados arbitrários podem implementar esse método abstrato para fornecer acesso direto aos dados armazenados. `OSError` deve ser levantada se o *path* não puder ser encontrado. Espera-se que o *path* seja construído usando o atributo `__file__` de um módulo ou um item de `__path__` de um pacote.

Alterado na versão 3.4: Levanta `OSError` em vez de `NotImplementedError`.

class importlib.abc.InspectLoader

Uma classe base abstrata para um carregador que implementa o protocolo opcional da **PEP 302** [https://peps.python.org/pep-0302/] para carregadores que inspecionam módulos.

get_code(*fullname*)

Retorna o objeto de código para um módulo, ou `None` se o módulo não tiver um objeto código (como seria o caso, por exemplo, para um módulo embutido). Levanta um `ImportError` se o carregador não conseguir encontrar o módulo solicitado.

Nota

Embora o método tenha uma implementação padrão, sugere-se que ele seja substituído, se possível, para desempenho.

Alterado na versão 3.4: Não é mais abstrato e uma implementação concreta é fornecida.

abstractmethod get_source(*fullname*)

Um método abstrato para retornar a fonte de um módulo. Ele é retornado como uma string de texto usando novas linhas universais, traduzindo todos os separadores de linha reconhecidos em caracteres `'\n'`. Retorna `None` se nenhuma fonte estiver disponível (por exemplo, um módulo embutido). Levanta **`ImportError`** se o carregador não puder encontrar o módulo especificado.

Alterado na versão 3.4: Levanta **`ImportError`** em vez de **`NotImplementedError`**.

is_package(*fullname*)

Um método opcional para retornar um valor verdadeiro se o módulo for um pacote, um valor falso caso contrário. **`ImportError`** é levantada se o carregador não puder encontrar o módulo.

Alterado na versão 3.4: Levanta **`ImportError`** em vez de **`NotImplementedError`**.

static source_to_code(*data*, *path='<string>'*, *fullname=None*)

Cria um objeto código a partir de código-fonte Python

O argumento *data* pode ser qualquer coisa a qual a função **`compile()`** tenha suporte (por exemplo, string ou bytes). O argumento *path* deve ser o “caminho” de onde o código-fonte se originou, o que pode ser um conceito abstrato (por exemplo, local em um arquivo zip).

Com o objeto de código subsequente, é possível executá-lo em um módulo executando `exec(code, module.__dict__)`.

The optional argument *fullname* specifies the module name. It is needed to unambiguous filter syntax warnings by module name.

Adicionado na versão 3.4.

Alterado na versão 3.5: O método foi tornado estático.

Adicionado na versão 3.15: Added the *fullname* parameter.

exec_module(*module*)

Implementação de **`Loader.exec_module()`**.

Adicionado na versão 3.4.

class importlib.abc.ExecutionLoader

Uma classe base abstrata que herda de **`InspectLoader`** que, quando implementada, ajuda um módulo a ser executado como um script. A ABC representa um protocolo opcional da **PEP 302** [https://peps.python.org/pep-0302/].

abstractmethod get_filename(*fullname*)

Um método abstrato que deve retornar o valor de **`__file__`** para o módulo especificado. Se nenhum caminho estiver disponível, **`ImportError`** é levantada.

Se o código-fonte estiver disponível, o método deverá retornar o caminho para o arquivo fonte, independentemente de um bytecode ter sido usado para carregar o módulo.

Alterado na versão 3.4: Levanta **`ImportError`** em vez de **`NotImplementedError`**.

class importlib.abc.FileLoader(*fullname*, *path*)

Uma classe base abstrata que herda de `ResourceLoader` e `ExecutionLoader`, fornecendo implementações concretas de `ResourceLoader.get_data()` e `ExecutionLoader.get_filename()`.

O argumento *fullname* é um nome totalmente resolvido do módulo que o carregador deve manipular. O argumento *path* é o caminho para o arquivo do módulo.

Adicionado na versão 3.3.

Alterado na versão 3.15: Removed the `load_module()` method.

name
: O nome do módulo que o carregador pode manipular.

path
: Caminho para o arquivo do módulo

abstractmethod get_filename(*fullname*)
: Retorna `path`.

abstractmethod get_data(*path*)
: Lê *caminho* como um arquivo binário e retorna os bytes dele.

class importlib.abc.SourceLoader

Uma classe base abstrata para implementar o carregamento de arquivo fonte (e opcionalmente bytecode). A classe herda de `ResourceLoader` e `ExecutionLoader`, exigindo a implementação de:

- `ResourceLoader.get_data()`
- `ExecutionLoader.get_filename()`
  : Deve retornar apenas o caminho para o arquivo fonte; o carregamento sem fonte não é suportado.

Os métodos abstratos definidos por esta classe são para adicionar suporte opcional ao arquivo bytecode. Não implementar esses métodos opcionais (ou fazer com que eles levantem `NotImplementedError`) faz com que o carregador funcione apenas com código-fonte. Implementar

os métodos permite que o carregador funcione com arquivos de código-fonte *e* bytecode; ele não permite carregamento *sem fonte* onde apenas o bytecode é fornecido. Arquivos de bytecode são uma otimização para acelerar o carregamento removendo a etapa de análise do compilador do Python e, portanto, nenhuma API específica de bytecode é exposta.

Alterado na versão 3.15: Removed the `load_module()` method.

path_stats(*path*)

Método abstrato opcional que retorna um **`dict`** contendo metadados sobre o caminho especificado. As chaves de dicionário suportadas são:

- `'mtime'` (obrigatório): um número inteiro ou de ponto flutuante que representa o tempo de modificação do código-fonte;
- `'size'` (opcional): o tamanho em bytes do código-fonte.

Quaisquer outras chaves no dicionário são ignoradas, para permitir extensões futuras. Se o caminho não puder ser manipulado, **`OSError`** é levantada.

Adicionado na versão 3.3.

Alterado na versão 3.4: Levanta **`OSError`** em vez de **`NotImplementedError`**.

path_mtime(*path*)

Método abstrato opcional que retorna o horário de modificação do caminho especificado.

Descontinuado desde a versão 3.3: Este método foi descontinuado em favor de **`path_stats()`**. Você não precisa implementá-lo, mas ele ainda está disponível para fins de compatibilidade. Levanta **`OSError`** se o caminho não puder ser manipulado.

Alterado na versão 3.4: Levanta **`OSError`** em vez de **`NotImplementedError`**.

set_data(*path*, *data*)

Método abstrato opcional que grava os bytes especificados em um caminho de arquivo. Quaisquer diretórios intermediários que não existam devem ser criados automaticamente.

Quando a gravação no caminho falhar porque o caminho é somente leitura (`errno.EACCES`/`PermissionError`), não propaga a exceção.

Alterado na versão 3.4: Não mais levanta `NotImplementedError` quando chamado.

get_code(*fullname*)

Implementação concreta de `InspectLoader.get_code()`.

exec_module(*module*)

Implementação concreta de `Loader.exec_module()`.

Adicionado na versão 3.4.

get_source(*fullname*)

Implementação concreta de `InspectLoader.get_source()`.

is_package(*fullname*)

Implementação concreta de `InspectLoader.is_package()`. Um módulo é determinado como um pacote se seu caminho de arquivo (conforme fornecido por `ExecutionLoader.get_filename()`) for um arquivo chamado `__init__` quando a extensão do arquivo for removida **e** o nome do módulo em si não terminar em `__init__`.

`importlib.machinery` – Importers and path hooks

Código-fonte: Lib/importlib/machinery.py
[https://github.com/python/cpython/tree/3.15/Lib/importlib/machinery.py]

Este módulo contém vários objetos que ajudam o `import` a localizar e carregar módulos.

importlib.machinery.SOURCE_SUFFIXES

Uma lista de strings representando os sufixos de arquivo reconhecidos para módulos em código-fonte.

Adicionado na versão 3.3.

importlib.machinery.DEBUG_BYTECODE_SUFFIXES

Uma lista de strings representando os sufixos de arquivo para módulos em bytecode não otimizado.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.5: Use `BYTECODE_SUFFIXES` instead.

importlib.machinery.OPTIMIZED_BYTECODE_SUFFIXES

Uma lista de strings representando os sufixos de arquivo para módulos em bytecode otimizado.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.5: Use `BYTECODE_SUFFIXES` instead.

importlib.machinery.BYTECODE_SUFFIXES

Uma lista de strings representando os sufixos de arquivo reconhecidos para módulos em bytecode (incluindo com ponto no início).

Adicionado na versão 3.3.

Alterado na versão 3.5: O valor não mais depende de `__debug__`.

importlib.machinery.EXTENSION_SUFFIXES

Uma lista de strings representando os sufixos de arquivo reconhecidos para módulos de extensão.

Adicionado na versão 3.3.

importlib.machinery.all_suffixes()

Retorna uma lista combinada de strings representando todos os sufixos de arquivo para módulos reconhecidos pelo maquinário de importação padrão. Este é um auxiliar para código que simplesmente precisa saber se um caminho de sistema de arquivos potencialmente se refere a um módulo sem precisar de detalhes sobre o tipo de módulo (por exemplo, `inspect.getmodulename()`).

Adicionado na versão 3.3.

class importlib.machinery.BuiltinImporter

Um importador para módulos embutidos. Todos os módulos embutidos conhecidos estão listados em `sys.builtin_module_names`. Esta classe implementa os ABCs `importlib.abc.MetaPathFinder` e `importlib.abc.InspectLoader`.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

Alterado na versão 3.5: As part of **PEP 489** [https://peps.python.org/pep-0489/], the builtin importer now implements `Loader.create_module` and `Loader.exec_module`

class importlib.machinery.FrozenImporter

Um importador para módulos congelados. Esta classe implementa os ABCs `importlib.abc.MetaPathFinder` e `importlib.abc.InspectLoader`.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

Alterado na versão 3.4: Gained `create_module()` and `exec_module()` methods.

class importlib.machinery.WindowsRegistryFinder

Localizador para módulos declarados no registro do Windows. Esta classe implementa a ABC `importlib.abc.MetaPathFinder`.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.6: Use a configuração de `site` em vez disso. Versões futuras do Python podem não habilitar esse localizador por padrão.

class importlib.machinery.PathFinder

Um Localizador para atributos `sys.path` e pacote `__path__`. Esta classe implementa a ABC `importlib.abc.MetaPathFinder`.

Somente métodos de classe são definidos por esta classe para aliviar a necessidade de instanciação.

classmethod find_spec(*fullname*, *path=None*, *target=None*)

Método de classe que tenta encontrar um spec para o módulo especificado por *fullname* em `sys.path` ou, se definido, em *path*. Para cada entrada de caminho pesquisada, `sys.path_importer_cache` é verificado. Se um objeto não falso for encontrado, ele será usado como o localizador de entrada de caminho para procurar o módulo que está sendo pesquisado. Se nenhuma entrada for encontrada em **`sys.path_importer_cache`**, então `sys.path_hooks` será pesquisado para um localizador para a entrada de caminho e, se encontrado, será armazenado em **`sys.path_importer_cache`** junto com a consulta sobre o módulo. Se nenhum localizador for encontrado, então `None` será armazenado no cache e retornado.

Adicionado na versão 3.4.

Alterado na versão 3.5: Se o diretório de trabalho atual — representado por uma string vazia — não for mais válido, `None` será retornado, mas nenhum valor será armazenado em cache em `sys.path_importer_cache`.

classmethod invalidate_caches()

Chama `importlib.abc.PathEntryFinder.invalidate_caches()` em todos os localizadores armazenados em `sys.path_importer_cache` que definem o método. Caso contrário, as entradas em **`sys.path_importer_cache`** definidas como `None` são excluídas.

Alterado na versão 3.7: Entradas de `None` em `sys.path_importer_cache` são excluídas.

Alterado na versão 3.4: Chama objetos em `sys.path_hooks` com o diretório de trabalho atual para `''` (ou seja, a string vazia).

class importlib.machinery.FileFinder(*path*, **loader_details*)

Uma implementação concreta de `importlib.abc.PathEntryFinder` que armazena em cache os resultados do sistema de arquivos.

O argumento *path* é o diretório que o localizador é responsável por pesquisar.

O argumento *loader_details* é um número variável de tuplas de 2 itens, cada uma contendo um carregador e uma sequência de sufixos de arquivo que o carregador reconhece. Espera-se que os carregadores sejam chamáveis que aceitam dois argumentos do nome do módulo e o caminho para o arquivo encontrado.

O localizador armazenará em cache o conteúdo do diretório conforme necessário, fazendo chamadas de “stat” para cada pesquisa de módulo para verificar se o cache não está desatualizado. Como a obsolescência do cache depende da granularidade das informações de estado do sistema operacional do sistema de arquivos, há uma condição de corrida potencial de busca por um módulo, criação de um novo arquivo e, em seguida, busca pelo módulo que o novo arquivo representa. Se as operações ocorrerem rápido o suficiente para caber na granularidade das chamadas de “stat”, a pesquisa do módulo falhará. Para evitar que isso aconteça, ao criar um módulo dinamicamente, certifique-se de chamar `importlib.invalidate_caches()`.

Adicionado na versão 3.3.

path

O caminho que o localizador irá procurar.

find_spec(*fullname*, *target=None*)

Tenta encontrar o spec para manipular *fullname* dentro de `path`.

Adicionado na versão 3.4.

invalidate_caches()

Limpa o cache interno.

classmethod path_hook(**loader_details*)

Um método de classe que retorna uma clausura para uso em `sys.path_hooks`. Uma instância de `FileFinder` é retornada pela clausura usando o argumento de caminho dado à clausura diretamente e *loader_details* indiretamente.

Se o argumento para a clausura não for um diretório existente, `ImportError` será levantada.

class importlib.machinery.NamespacePath(*name*, *path*, *path_finder*)

Represents a namespace package's path (`module.__path__`).

When its `__path__` value is accessed it will be recomputed if necessary. This keeps it in-sync with the global state (`sys.modules`).

The *name* argument is the name of the namespace module.

The *path* argument is the initial path value.

The *path_finder* argument is the callable used to recompute the path value. The callable has the same signature as `importlib.abc.MetaPathFinder.find_spec()`.

When the parent's `module.__path__` attribute is updated, the path value is recomputed.

If the parent module is missing from `sys.modules`, then `ModuleNotFoundError` will be raised.

For top-level modules, the parent module's path is `sys.path`.

Nota

`PathFinder.invalidate_caches()` invalidates `NamespacePath`, forcing the path value to be recomputed next time it is accessed.

Adicionado na versão 3.15.

class importlib.machinery.SourceFileLoader(*fullname*, *path*)

Uma implementação concreta de `importlib.abc.SourceLoader` subclassificando `importlib.abc.FileLoader` e fornecendo algumas implementações concretas de outros métodos.

Adicionado na versão 3.3.

Alterado na versão 3.15: Removed the `load_module()` method.

name

O nome do módulo que este carregador irá manipular.

path

O caminho para o arquivo fonte.

is_package(*fullname*)

Retorna `True` se `path` parecer ser para um pacote.

path_stats(*path*)

Implementação concreta de `importlib.abc.SourceLoader.path_stats()`.

set_data(*path*, *data*)

Implementação concreta de `importlib.abc.SourceLoader.set_data()`.

class importlib.machinery.SourcelessFileLoader(*fullname*, *path*)

Uma implementação concreta de `importlib.abc.FileLoader` que pode importar arquivos de bytecode (ou seja, não existem arquivos de código-fonte).

Observe que o uso direto de arquivos de bytecode (e, portanto, não de arquivos de código-fonte) impede que seus módulos sejam utilizáveis por todas as implementações do Python ou novas versões do Python que alteram o formato do bytecode.

Adicionado na versão 3.3.

Alterado na versão 3.15: Removed the `load_module()` method.

name

O nome do módulo que o carregador vai manipular.

path

O caminho para o arquivo de bytecode.

is_package(*fullname*)

Determina se o módulo é um pacote baseado em `path`.

get_code(*fullname*)

Retorna o objeto código para `name` criado a partir de `path`.

get_source(*fullname*)

Retorna `None`, pois os arquivos de bytecode não têm código-fonte quando este carregador é usado.

class importlib.machinery.ExtensionFileLoader(*fullname*, *path*)

Uma implementação concreta de `importlib.abc.ExecutionLoader` para módulos de extensão.

O argumento *fullname* especifica o nome do módulo ao qual o carregador deve dar suporte. O argumento *path* é o caminho para o arquivo do módulo de extensão.

Observe que, por padrão, a importação de um módulo de extensão falhará em subinterpretadores se ele não implementar inicialização multifásica (consulte a **PEP 489** [https://peps.python.org/pep-0489/]), mesmo que ele seja importado com sucesso.

Adicionado na versão 3.3.

Alterado na versão 3.12: A inicialização multifásica agora é necessária para uso em subinterpretadores.

name

Nome do módulo ao qual o carregador dá suporte.

path

Caminho para o módulo de extensão.

create_module(*spec*)

Cria o objeto do módulo a partir da especificação fornecida de acordo com a **PEP 489** [https://peps.python.org/pep-0489/].

Adicionado na versão 3.5.

exec_module(*module*)

Inicializa o objeto do módulo fornecido de acordo com a **PEP 489** [https://peps.python.org/pep-0489/].

Adicionado na versão 3.5.

is_package(*fullname*)

Returns `True` if the file path points to a package's `__init__` module based on **`EXTENSION_SUFFIXES`**.

get_code(*fullname*)

Retorna `None`, pois os módulos de extensão não possuem um objeto código.

get_source(*fullname*)

Retorna `None` pois os módulos de extensão não possuem código-fonte.

get_filename(*fullname*)

Retorna **`path`**.

Adicionado na versão 3.4.

class importlib.machinery.NamespaceLoader(*name*, *path*, *path_finder*)

Uma implementação concreta de **`importlib.abc.InspectLoader`** para pacotes de espaço de nomes. Este é um apelido para uma classe privada e só é tornado público para introspecção do atributo `__loader__` em pacotes de espaço de nomes:

```
>>> from importlib.machinery import NamespaceLoader
>>> import my_namespace
>>> isinstance(my_namespace.__loader__, NamespaceLoader)
True
>>> import importlib.abc
>>> isinstance(my_namespace.__loader__, importlib.abc.Loader)
True
```

Adicionado na versão 3.11.

class importlib.machinery.ModuleSpec(*name*, *loader*, ***, *origin=None*, *loader_state=None*, *is_package=None*)

A specification for a module's import-system-related state. This is typically exposed as the module's **`__spec__`** attribute. Many of these attributes are also available directly on a module: for example, `module.__spec__.origin == module.__file__`. Note, however, that while the *values* are usually equivalent, they can differ since there is no synchronization between the two objects. For example, it is possible to update the module's **`__file__`** at runtime and this will not be automatically reflected in the module's **`__spec__.origin`**, and vice versa.

Adicionado na versão 3.4.

name

The module's fully qualified name (see `module.__name__`). The finder should always set this attribute to a non-empty string.

loader

The loader used to load the module (see `module.__loader__`). The finder should always set this attribute.

origin

The location the loader should use to load the module (see `module.__file__`). For example, for modules loaded from a `.py` file this is the filename. The finder should always set this attribute to a meaningful value for the loader to use. In the uncommon case that there is not one (like for namespace packages), it should be set to `None`.

submodule_search_locations

A (possibly empty) sequence of strings enumerating the locations in which a package's submodules will be found (see `module.__path__`). Most of the time there will only be a single directory in this list.

The finder should set this attribute to a sequence, even an empty one, to indicate to the import system that the module is a package. It should be set to `None` for non-package modules. It is set automatically later to a special object for namespace packages.

loader_state

The finder may set this attribute to an object containing additional, module-specific data to use when loading the module. Otherwise it should be set to `None`.

cached

The filename of a compiled version of the module's code. The finder should always set this attribute but it may be `None` for modules that do not need compiled code stored.

parent

(Read-only) The fully qualified name of the package the module is in (or the empty string for a top-level module). See `module.__package__`. If the module is a package then this is the same as `name`.

has_location

`True` if the spec's `origin` refers to a loadable location, `False` otherwise. This value impacts how **`origin`** is interpreted and how the module's `__file__` is populated.

class importlib.machinery.AppleFrameworkLoader(*name*, *path*)

A specialization of `importlib.machinery.ExtensionFileLoader` that is able to load extension modules in Framework format.

For compatibility with the iOS App Store, *all* binary modules in an iOS app must be dynamic libraries, contained in a framework with appropriate metadata, stored in the `Frameworks` folder of the packaged app. There can be only a single binary per framework, and there can be no executable binary material outside the Frameworks folder.

To accommodate this requirement, when running on iOS, extension module binaries are *not* packaged as `.so` files on `sys.path`, but as individual standalone frameworks. To discover those frameworks, this loader is registered against the `.fwork` file extension, with a `.fwork` file acting as a placeholder in the original location of the binary on `sys.path`. The `.fwork` file contains the path of the actual binary in the `Frameworks` folder, relative to the app bundle. To allow for resolving a framework-packaged binary back to the original location, the framework is expected to contain a `.origin` file that contains the location of the `.fwork` file, relative to the app bundle.

Por exemplo, considere o caso de uma importação `from foo.bar import _whiz`, onde `_whiz` é implementado com o módulo binário `sources/foo/bar/_whiz.abi3.so`, com `sources` sendo o local registrado em `sys.path`, relativo ao pacote da aplicação. Este módulo *deve* ser distribuído como

`Frameworks/foo.bar._whiz.framework/foo.bar._whiz` (criando o nome do framework a partir do caminho completo de importação do módulo), com um arquivo `Info.plist` no diretório `.framework` identificando o binário como um framework. O módulo `foo.bar._whiz` seria representado no local original com um arquivo marcador `sources/foo/bar/_whiz.abi3.fwork`, contendo o caminho `Frameworks/foo.bar._whiz/foo.bar._whiz`. O framework também conteria `Frameworks/foo.bar._whiz.framework/foo.bar._whiz.origin`, contendo o caminho para o arquivo `.fwork`.

When a module is loaded with this loader, the `__file__` for the module will report as the location of the `.fwork` file. This allows code to use the `__file__` of a module as an anchor for file system traversal. However, the spec origin will reference the location of the *actual* binary in the `.framework` folder.

The Xcode project building the app is responsible for converting any `.so` files from wherever they exist in the `PYTHONPATH` into frameworks in the `Frameworks` folder (including stripping extensions from the module file, the addition of framework metadata, and signing the resulting framework), and creating the `.fwork` and `.origin` files. This will usually be done with a build step in the Xcode project; see the iOS documentation for details on how to construct this build step.

Adicionado na versão 3.13.

Disponibilidade: iOS.

name
: Nome do módulo ao qual o carregador dá suporte.

path
: Path to the `.fwork` file for the extension module.

`importlib.util` – Utility code for importers

Source code: Lib/importlib/util.py
[https://github.com/python/cpython/tree/3.15/Lib/importlib/util.py]

This module contains the various objects that help in the construction of an importer.

importlib.util.MAGIC_NUMBER

The bytes which represent the bytecode version number. If you need help with loading/writing bytecode then consider `importlib.abc.SourceLoader`.

Adicionado na versão 3.4.

importlib.util.cache_from_source(*path*, ***, *optimization=None*)

Return the **PEP 3147** [https://peps.python.org/pep-3147/]/**PEP 488** [https://peps.python.org/pep-0488/] path to the byte-compiled file associated with the source *path*. For example, if *path* is `/foo/bar/baz.py` the return value would be `/foo/bar/__pycache__/baz.cpython-32.pyc` for Python 3.2. The `cpython-32` string comes from the current magic tag (see `sys.implementation.cache_tag`; if it is not defined then `NotImplementedError` will be raised).

The *optimization* parameter is used to specify the optimization level of the bytecode file. An empty string represents no optimization, so `/foo/bar/baz.py` with an *optimization* of `''` will result in a bytecode path of `/foo/bar/__pycache__/baz.cpython-32.pyc`. `None` causes the interpreter's optimization level to be used. Any other value's string representation is used, so `/foo/bar/baz.py` with an *optimization* of `2` will lead to the bytecode path of `/foo/bar/__pycache__/baz.cpython-32.opt-2.pyc`. The string representation of *optimization* can only be alphanumeric, else `ValueError` is raised.

Adicionado na versão 3.4.

Alterado na versão 3.5: The *optimization* parameter was added and the *debug_override* parameter was deprecated.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.15: The *debug_override* parameter was removed.

importlib.util.source_from_cache(*path*)

Given the *path* to a **PEP 3147** [https://peps.python.org/pep-3147/] file name, return the associated source code file path. For example, if *path* is `/foo/bar/__pycache__/baz.cpython-32.pyc` the returned path would be `/foo/bar/baz.py`. *path* need not exist, however if it does not conform to **PEP 3147** [https://peps.python.org/pep-3147/] or **PEP 488** [https://peps.python.org/pep-0488/] format, a `ValueError` is raised. If `sys.implementation.cache_tag` is not defined, `NotImplementedError` is raised.

Adicionado na versão 3.4.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

importlib.util.decode_source(*source_bytes*)

Decode the given bytes representing source code and return it as a string with universal newlines (as required by `importlib.abc.InspectLoader.get_source()`).

Adicionado na versão 3.4.

importlib.util.resolve_name(*name*, *package*)

Resolve a relative module name to an absolute one.

If **name** has no leading dots, then **name** is simply returned. This allows for usage such as `importlib.util.resolve_name('sys', __spec__.parent)` without doing a check to see if the **package** argument is needed.

`ImportError` is raised if **name** is a relative module name but **package** is a false value (e.g. `None` or the empty string). `ImportError` is also raised if a relative name would escape its containing package (e.g. requesting `..bacon` from within the `spam` package).

Adicionado na versão 3.3.

Alterado na versão 3.9: To improve consistency with import statements, raise `ImportError` instead of `ValueError` for invalid relative import attempts.

importlib.util.find_spec(*name*, *package=None*)

Localiza o spec para um módulo, opcionalmente relativo ao nome do **package** especificado. Se o módulo estiver em `sys.modules`, então `sys.modules[name].__spec__` será retornado (a menos que o spec seja `None` ou não esteja definido, nesse caso `ValueError` é levantada). Caso contrário, uma pesquisa usando `sys.meta_path` é feita. `None` é retornado se nenhum spec for encontrada.

If **name** is for a submodule (contains a dot), the parent module is automatically imported.

name and **package** work the same as for `importlib.import_module()`.

Adicionado na versão 3.4.

Alterado na versão 3.7: Raises `ModuleNotFoundError` instead of `AttributeError` if **package** is in fact not a package (i.e. lacks a `__path__` attribute).

importlib.util.module_from_spec(*spec*)

Create a new module based on **spec** and `spec.loader.create_module`.

If `spec.loader.create_module` does not return `None`, then any pre-existing attributes will not be reset. Also, no `AttributeError` will be raised if triggered while accessing **spec** or setting an attribute on the module.

This function is preferred over using `types.ModuleType` to create a new module as **spec** is used to set as many import-controlled attributes on the module as possible.

Adicionado na versão 3.5.

importlib.util.spec_from_loader(*name*, *loader*, *, *origin=None*, *is_package=None*)

A factory function for creating a **ModuleSpec** instance based on a loader. The parameters have the same meaning as they do for ModuleSpec. The function uses available loader APIs, such as **InspectLoader.is_package**, to fill in any missing information on the spec.

Adicionado na versão 3.4.

importlib.util.spec_from_file_location(*name*, *location*, *, *loader=None*, *submodule_search_locations=None*)

A factory function for creating a **ModuleSpec** instance based on the path to a file. Missing information will be filled in on the spec by making use of loader APIs and by the implication that the module will be file-based.

Adicionado na versão 3.4.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

importlib.util.source_hash(*source_bytes*)

Return the hash of *source_bytes* as bytes. A hash-based `.pyc` file embeds the **source_hash()** of the corresponding source file's contents in its header.

Adicionado na versão 3.7.

importlib.util._incompatible_extension_module_restrictions(*, *disable_check*)

A context manager that can temporarily skip the compatibility check for extension modules. By default the check is enabled and will fail when a single-phase init module is imported in a subinterpreter. It will also fail for a multi-phase init module that doesn't explicitly support a per-interpreter GIL, when imported in an interpreter with its own GIL.

Note that this function is meant to accommodate an unusual case; one which is likely to eventually go away. There's is a pretty good chance this is not what you were looking for.

You can get the same effect as this function by implementing the basic interface of multi-phase init (**PEP 489** [https://peps.python.org/pep-0489/]) and lying about support for multiple interpreters (or per-interpreter GIL).

Aviso

Using this function to disable the check can lead to unexpected behavior and even crashes. It should only be used during extension module development.

Adicionado na versão 3.12.

class importlib.util.LazyLoader(*loader*)

A class which postpones the execution of the loader of a module until the module has an attribute accessed.

This class **only** works with loaders that define `exec_module()` as control over what module type is used for the module is required. For those same reasons, the loader's `create_module()` method must return `None` or a type for which its `__class__` attribute can be mutated along with not using slots. Finally, modules which substitute the object placed into `sys.modules` will not work as there is no way to properly replace the module references throughout the interpreter safely; `ValueError` is raised if such a substitution is detected.

Nota

For projects where startup time is critical, this class allows for potentially minimizing the cost of loading a module if it is never used. For projects where startup time is not essential then use of this class is **heavily** discouraged due to error messages created during loading being postponed and thus occurring out of context.

Adicionado na versão 3.5.

Alterado na versão 3.6: Began calling `create_module()`, removing the compatibility warning for `importlib.machinery.BuiltinImporter` and `importlib.machinery.ExtensionFileLoader`.

classmethod factory(*loader*)

A class method which returns a callable that creates a lazy loader. This is meant to be used in situations where the loader is passed by class instead of by instance.

```
suffixes = importlib.machinery.SOURCE_SUFFIXES
loader = importlib.machinery.SourceFileLoader
lazy_loader = importlib.util.LazyLoader.factory(loader)
finder = importlib.machinery.FileFinder(path, (lazy_loade
```

Exemplos

Importing programmatically

To programmatically import a module, use `importlib.import_module()`.

```
import importlib

itertools = importlib.import_module('itertools')
```

Checando se o módulo pode ser importado

If you need to find out if a module can be imported without actually doing the import, then you should use `importlib.util.find_spec()`.

Note that if `name` is a submodule (contains a dot), `importlib.util.find_spec()` will import the parent module.

```
import importlib.util
import sys

# For illustrative purposes.
```

```
name = 'itertools'

if name in sys.modules:
    print(f"{name!r} already in sys.modules")
elif (spec := importlib.util.find_spec(name)) is not None:
    # If you chose to perform the actual import ...
    module = importlib.util.module_from_spec(spec)
    sys.modules[name] = module
    spec.loader.exec_module(module)
    print(f"{name!r} has been imported")
else:
    print(f"can't find the {name!r} module")
```

Importa o arquivo de origem diretamente

This recipe should be used with caution: it is an approximation of an import statement where the file path is specified directly, rather than **`sys.path`** being searched. Alternatives should first be considered first, such as modifying **`sys.path`** when a proper module is required, or using **`runpy.run_path()`** when the global namespace resulting from running a Python file is appropriate.

To import a Python source file directly from a path, use the following recipe:

```
import importlib.util
import sys


def import_from_path(module_name, file_path):
    spec = importlib.util.spec_from_file_location(module_name, f
    module = importlib.util.module_from_spec(spec)
    sys.modules[module_name] = module
    spec.loader.exec_module(module)
    return module


# For illustrative purposes only (use of `json` is arbitrary).
import json
file_path = json.__file__
module_name = json.__name__
```

```
# Similar outcome as `import json`.
json = import_from_path(module_name, file_path)
```

Implementing lazy imports

The example below shows how to implement lazy imports:

```
>>> import importlib.util
>>> import sys
>>> def lazy_import(name):
...     spec = importlib.util.find_spec(name)
...     loader = importlib.util.LazyLoader(spec.loader)
...     spec.loader = loader
...     module = importlib.util.module_from_spec(spec)
...     sys.modules[name] = module
...     loader.exec_module(module)
...     return module
...
>>> lazy_typing = lazy_import("typing")
>>> #lazy_typing is a real module object,
>>> #but it is not loaded in memory yet.
>>> lazy_typing.TYPE_CHECKING
False
```

Setting up an importer

For deep customizations of import, you typically want to implement an importer. This means managing both the finder and loader side of things. For finders there are two flavours to choose from depending on your needs: a meta path finder or a path entry finder. The former is what you would put on **`sys.meta_path`** while the latter is what you create using a path entry hook on **`sys.path_hooks`** which works with **`sys.path`** entries to potentially create a finder. This example will show you how to register your own importers so that import will use them (for creating an importer for yourself, read the documentation for the appropriate classes defined within this package):

```
import importlib.machinery
import sys

# For illustrative purposes only.
SpamMetaPathFinder = importlib.machinery.PathFinder
SpamPathEntryFinder = importlib.machinery.FileFinder
loader_details = (importlib.machinery.SourceFileLoader,
                  importlib.machinery.SOURCE_SUFFIXES)

# Setting up a meta path finder.
# Make sure to put the finder in the proper location in the list
# priority.
sys.meta_path.append(SpamMetaPathFinder)

# Setting up a path entry finder.
# Make sure to put the path hook in the proper location in the l
# of priority.
sys.path_hooks.append(SpamPathEntryFinder.path_hook(loader_detai
```

Approximating `importlib.import_module()`

Import itself is implemented in Python code, making it possible to expose most of the import machinery through importlib. The following helps illustrate the various APIs that importlib exposes by providing an approximate implementation of `importlib.import_module()`:

```
import importlib.util
import sys

def import_module(name, package=None):
    """An approximate implementation of import."""
    absolute_name = importlib.util.resolve_name(name, package)
    try:
        return sys.modules[absolute_name]
    except KeyError:
        pass

    path = None
    if '.' in absolute_name:
        parent_name, _, child_name = absolute_name.rpartition('.
        parent_module = import_module(parent_name)
```

```
        path = parent_module.__spec__.submodule_search_locations
    for finder in sys.meta_path:
        spec = finder.find_spec(absolute_name, path)
        if spec is not None:
            break
    else:
        msg = f'No module named {absolute_name!r}'
        raise ModuleNotFoundError(msg, name=absolute_name)
    module = importlib.util.module_from_spec(spec)
    sys.modules[absolute_name] = module
    spec.loader.exec_module(module)
    if path is not None:
        setattr(parent_module, child_name, module)
    return module
```

`importlib.resources` – Leitura, abertura e acesso a recursos de pacotes

Código-fonte: Lib/importlib/resources/__init__.py
[https://github.com/python/cpython/tree/3.15/Lib/importlib/resources/__init__.py]

Adicionado na versão 3.7.

Esse módulo aproveita o sistema de importação do Python para fornecer acesso a *recursos* dentro de *pacotes*.

"Recursos" são recursos arquivo ou similares associados a um módulo ou pacote em Python. Os recursos podem estar contidos diretamente em um pacote, em um subdiretório contido nesse pacote ou adjacentes a módulos fora de um pacote. Os recursos podem ser de texto ou binários. Como resultado, as fontes de módulos Python de um pacote (.py), artefatos de compilação (pycache) e artefatos de instalação (como `nomes de arquivos reservados` em diretórios) são tecnicamente recursos de fato desse pacote. Na prática, entretanto, os recursos são principalmente os artefatos não-Python expostos especificamente pelo autor do pacote.

Os recursos podem ser abertos ou lidos no modo binário ou de texto.

Os recursos são mais ou menos semelhantes a arquivos dentro de diretórios, embora seja importante ter em mente que isso é apenas uma metáfora. Recursos e pacotes **não** precisam existir como arquivos e diretórios físicos no sistema de arquivos: por exemplo, um pacote e seus recursos podem ser importados de um arquivo zip usando `zipimport`.

Aviso

importlib.resources follows the same security model as the built-in **open()** function. Passing untrusted inputs to the functions in this module is unsafe.

Nota

O backport autônomo desse módulo fornece mais informações sobre uso do importlib.resources [https://importlib-resources.readthedocs.io/en/latest/using.html] e migração do pkg_resources para o importlib.resources [https://importlib-resources.readthedocs.io/en/latest/migration.html].

Loaders que desejam oferecer suporte à leitura de recursos devem implementar um método `get_resource_reader(fullname)` conforme especificado por **importlib.resources.abc.ResourceReader**.

class importlib.resources.Anchor

Representa uma âncora para recursos, seja um **objeto módulo** ou um nome de módulo como uma string. Definido como `Union[str, ModuleType]`.

importlib.resources.files(*anchor: Anchor | None = None*)

Retorna um objeto **Traversable** que representa o contêiner de recursos (pense em diretório) e seus recursos (pense em arquivos). Um Traversable pode conter outros contêineres (pense em subdiretórios).

anchor é um **Anchor** opcional. Se a âncora for um pacote, os recursos são resolvidos a partir desse pacote. Se for um módulo, os recursos são resolvidos ao lado desse módulo (no mesmo pacote ou na raiz do pacote). Se a âncora for omitida, o módulo do chamador é usado.

Adicionado na versão 3.9.

Alterado na versão 3.12: package parameter was renamed to *anchor*. *package* was still accepted, but deprecated.

Alterado na versão 3.15: *package* parameter was fully removed. *anchor* can now be a non-package module and if omitted will default to the caller's module. *package* is no longer accepted since Python 3.15. Consider passing the anchor positionally or using `importlib_resources >= 5.10` for a compatible interface on older Pythons.

importlib.resources.as_file(*traversable*)

Dado um objeto **`Traversable`** que representa um arquivo ou diretório, normalmente de **`importlib.resources.files()`**, retorna um gerenciador de contexto para uso em uma instrução **`with`**. O gerenciador de contexto fornece um objeto **`pathlib.Path`**.

Sair do gerenciador de contexto limpa qualquer arquivo ou diretório temporário criado quando o recurso foi extraído, por exemplo, de um arquivo zip.

Use `as_file` quando os métodos de Traversable (`read_text`, etc.) forem insuficientes e for necessário um arquivo ou diretório real no sistema de arquivos.

Adicionado na versão 3.9.

Alterado na versão 3.12: Adicionado suporte para *traversable* representando um diretório.

API funcional

Um conjunto de auxiliares simplificados e compatíveis com versões anteriores está disponível. Estes permitem operações comuns em uma única chamada de função.

Para todas as funções a seguir:

- *anchor* é um **`Anchor`**, assim como em **`files()`**. Ao contrário do comportamento em `files`, *anchor* aqui não pode ser omitido.

- *path_names* são componentes do nome do caminho de um recurso, relativo à âncora. Por exemplo, para obter o texto de um recurso chamado `info.txt`, use:

  ```
  importlib.resources.read_text(my_module, "info.txt")
  ```

  Assim como em **`Traversable.joinpath`**, os componentes individuais devem usar barras ordinárias (`/`) como separadores do caminho. Por exemplo, as seguintes chamadas são equivalentes:

  ```
  importlib.resources.read_binary(my_module, "pics/painting.pr
  importlib.resources.read_binary(my_module, "pics", "paintin
  ```

  Por motivos de compatibilidade com versões anteriores, funções que leem texto requerem um argumento *encoding* explícito se múltiplos *path_names* forem fornecidos. Por exemplo, para obter o texto de `info/chapter1.txt`, use:

  ```
  importlib.resources.read_text(my_module, "info", "chapter1.t
                                encoding='utf-8')
  ```

importlib.resources.open_binary(*anchor*, **path_names*)

Abre o recurso nomeado para leitura binária.

Veja a introdução para detalhes sobre *anchor* e *path_names*.

Esta função retorna um objeto **`BinaryIO`**, isto é, um fluxo binário aberto para leitura.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).open('rb')
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.open_text(*anchor*, **path_names*, *encoding='utf-8'*, *errors='strict'*)

Abre o recurso nomeado para leitura de texto. Por padrão, o conteúdo é lido como UTF-8 estrito.

Veja a introdução para detalhes sobre *anchor* e *path_names*. *encoding* e *errors* têm o mesmo significado que na embutida `open()`.

Por motivos de compatibilidade com versões anteriores, o argumento *encoding* precisa ser passado explicitamente se houver múltiplos *path_names*. Essa limitação está agendada para ser removida no Python 3.15.

Esta função retorna um objeto `TextIO`, isto é, um fluxo de texto aberto para leitura.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).open('r', encoding=encodi
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos. *encoding* e *errors* precisam ser fornecidos como argumentos nomeados.

importlib.resources.read_binary(*anchor*, **path_names*)

Lê e retorna o conteúdo do recurso nomeado como `bytes`.

Veja a introdução para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).read_bytes()
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.read_text(*anchor*, **path_names*, *encoding='utf-8'*, *errors='strict'*)

Lê e retorna o conteúdo do recurso nomeado como `str`. Por padrão, o conteúdo é lido como UTF-8 estrito.

Veja a introdução para detalhes sobre *anchor* e *path_names*. *encoding* e *errors* têm o mesmo significado que na embutida `open()`.

Por motivos de compatibilidade com versões anteriores, o argumento *encoding* precisa ser passado explicitamente se houver múltiplos *path_names*. Essa limitação está agendada para ser removida no Python 3.15.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).read_text(encoding=encodi
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos. *encoding* e *errors* precisam ser fornecidos como argumentos nomeados.

importlib.resources.path(*anchor*, **path_names*)

Fornece o caminho para o *resource* como um caminho real do sistema de arquivos. Essa função retorna um gerenciador de contexto para uso em uma instrução `with`. O gerenciador de contexto fornece um objeto `pathlib.Path`.

Sair do gerenciador de contexto limpa qualquer arquivo temporário criado, por exemplo quando o recurso precisa ser extraído de um arquivo zip.

Por exemplo, o método `stat()` requer um caminho real em um sistema de arquivos; ele pode ser usado assim:

```
with importlib.resources.path(anchor, "resource.txt") as fspa
    result = fspath.stat()
```

Veja a introdução para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
as_file(files(anchor).joinpath(*path_names))
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.is_resource(*anchor*, **path_names*)

Retorna `True` se o recurso nomeado existe, senão `False`. Esta função não considera diretórios como sendo recursos.

Veja a introdução para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
files(anchor).joinpath(*path_names).is_file()
```

Alterado na versão 3.13: Múltiplos *path_names* são aceitos.

importlib.resources.contents(*anchor*, **path_names*)

Retorna um iterável sobre os itens nomeados no pacote ou caminho. O iterável retorna nomes de recursos (por exemplo, arquivos) e outros itens que não recursos (por exemplo, diretórios) como `str`. O iterável não recorre em subdiretórios.

Veja a introdução para detalhes sobre *anchor* e *path_names*.

Esta função é mais ou menos equivalente a:

```
for resource in files(anchor).joinpath(*path_names).iterdir(
    yield resource.name
```

Descontinuado desde a versão 3.11: Prefira `iterdir()` como acima, que oferece mais controle sobre os resultados e funcionalidade mais rica.

`importlib.resources.abc` – Classes base abstratas para recursos

Código-fonte: Lib/importlib/resources/abc.py
[https://github.com/python/cpython/tree/3.15/Lib/importlib/resources/abc.py]

Adicionado na versão 3.11.

class importlib.resources.abc.ResourceReader

Substituída por TraversableResources

Uma classe base abstrata para fornecer a capacidade de ler *recursos*.

Da perspectiva deste ABC, um *recurso* é um artefato binário que é enviado dentro de um pacote. Normalmente isso é algo como um arquivo de dados que fica próximo ao arquivo `__init__.py` do pacote. O objetivo desta classe é ajudar a abstrair o acesso a tais arquivos de dados para que não importe se o pacote e seu(s) arquivo(s) de dados estão armazenados, por exemplo, em um arquivo zip versus no sistema de arquivos.

Para qualquer um dos métodos desta classe, espera-se que o argumento *resource* seja um objeto caminho ou similar que representa conceitualmente apenas um nome de arquivo. Isso significa que nenhum caminho de subdiretório deve ser incluído no argumento *resource*. Isso ocorre porque a localização do pacote para o qual o leitor se destina, atua como o “diretório”. Portanto, a metáfora para diretórios e nomes de arquivos são pacotes e recursos, respectivamente. É também por isso que se espera que as instâncias dessa classe se correlacionem diretamente a um pacote específico (em vez de representar potencialmente vários pacotes ou um módulo).

Carregadores que desejam oferecer suporte à leitura de recursos devem fornecer um método chamado `get_resource_reader(nomecompleto)` que retorna um objeto implementando esta interface ABC. Se o módulo especificado por nomecompleto não for um pacote, este método deve retornar **`None`**. Um objeto compatível com este ABC só deve ser retornado quando o módulo especificado for um pacote.

Descontinuado desde a versão 3.12: Use **`importlib.resources.abc.TraversableResources`**.

abstractmethod open_resource(*resource*)

Retorna um objeto arquivo ou similar aberto para leitura binária de *resource*.

Se o recurso não puder ser encontrado, **`FileNotFoundError`** é levantada.

abstractmethod resource_path(*resource*)

Retorna o caminho do sistema de arquivos para *resource*.

Se o recurso não existir concretamente no sistema de arquivos, levanta **`FileNotFoundError`**.

abstractmethod is_resource(*path*)

Retorna `True` se o *path* nomeado for considerado um recurso. **`FileNotFoundError`** é levantada se *path* não existir.

Alterado na versão 3.10: O argumento *name* foi renomeado para *path*.

abstractmethod contents()

Retorna um iterável de strings sobre o conteúdo do pacote. Observe que não é necessário que todos os nomes retornados pelo iterador sejam recursos reais, por exemplo, é aceitável retornar nomes para os quais **`is_resource()`** seria falso.

Permitir que nomes que não são recursos sejam retornados é permitir situações em que a forma como um pacote e seus recursos são armazenados é conhecida a priori e os nomes que não são recursos seriam úteis. Por exemplo, o retorno de nomes de subdiretórios é permitido para que, quando se souber que o pacote e os recursos estão armazenados no sistema de arquivos, esses nomes de subdiretórios possam ser usados diretamente.

O método abstrato retorna um iterável sem itens.

class importlib.resources.abc.Traversable

Um objeto com um subconjunto de métodos de **`pathlib.Path`** adequados para percorrer diretórios e abrir arquivos.

Para uma representação do objeto no sistema de arquivos, use **`importlib.resources.as_file()`**.

name

Abstrato. O nome base deste objeto sem nenhuma referência pai.

abstractmethod iterdir()

Produz objetos Traversable em self.

abstractmethod is_dir()

Retorna `True` se self for um diretório.

abstractmethod is_file()

Retorna `True` se self for um arquivo.

abstractmethod joinpath(**pathsegments*)

Percorre os diretórios de acordo com *pathsegments* e retorna o resultado como **`Traversable`**.

Cada argumento *pathsegments* pode conter vários nomes separados por barras (`/`, `posixpath.sep`). Por exemplo, os seguintes são equivalentes:

```
files.joinpath('subdir', 'subsuddir', 'arquivo.txt')
files.joinpath('subdir/subsuddir/arquivo.txt')
```

Observe que algumas implementações de **`Traversable`** podem não ser atualizadas para a versão mais recente do protocolo. Para compatibilidade com tais implementações, forneça um único argumento sem separadores de caminho para cada chamada para `joinpath`. Por exemplo:

```
files.joinpath('subdir').joinpath('subsubdir').joinpath('a
```

Alterado na versão 3.11: `joinpath` aceita múltiplos *pathsegments*, e estes segmentos podem conter barras como separadores de caminho. Anteriormente, apenas um único argumento *child* era aceito.

abstractmethod __truediv__(*child*)

Retorna um filho Traversable em si mesmo. Equivalente a `joinpath(child)`.

abstractmethod open(*mode='r', *args, **kwargs*)

mode pode ser ‘r’ ou ‘rb’ para abrir como texto ou binário. Retorna um manipulador adequado para leitura (o mesmo que **`pathlib.Path.open`**).

Ao abrir como texto, aceita parâmetros de codificação como os aceitos por **`io.TextIOWrapper`**.

read_bytes()

Lê o conteúdo de self como bytes.

read_text(*encoding=None*)

Lê o conteúdo de self como texto.

class importlib.resources.abc.TraversableResources

Uma classe base abstrata para leitores de recursos capaz de servir a interface **`importlib.resources.files()`**. É uma subclasse de **`ResourceReader`** e fornece implementações concretas dos métodos

abstratos de **`ResourceReader`**. Portanto, qualquer carregador que forneça **`TraversableResources`** também fornece **`ResourceReader`**.

Espera-se que os carregadores que desejam oferecer suporte à leitura de recursos implementem essa interface.

abstractmethod files()

Retorna um objeto **`importlib.resources.abc.Traversable`** para o pacote carregado.

`importlib.metadata` – Acessando metadados do pacote

Adicionado na versão 3.8.

Alterado na versão 3.10: `importlib.metadata` não é mais provisório.

Código-fonte: Lib/importlib/metadata/__init__.py [https://github.com/python/cpython/tree/3.15/Lib/importlib/metadata/__init__.py]

`importlib.metadata` é uma biblioteca que fornece acesso aos metadados de um Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Import-Package] instalado, como seus pontos de entrada ou seus nomes de nível superior (Pacotes de Importação [https://packaging.python.org/pt_BR/latest/glossary/#term-Import-Package], módulos, se houver). Construída em parte no sistema de importação do Python, esta biblioteca fornece as APIs de ponto de entrada e de metadados que eram anteriormente expostas pelo agora removido pacote `pkg_resources`. Junto com **`importlib.resources`**, agora substitui `pkg_resources`.

`importlib.metadata` opera em *pacotes de distribuição* de terceiros instalados no diretório `site-packages` do Python através de ferramentas como pip [https://pypi.org/project/pip/]. Especificamente, ele funciona com distribuições com diretórios `dist-info` ou `egg-info` detectáveis, e metadados definidos pelas Especificações de metadados principais [https://packaging.python.org/pt_BR/latest/specifications/core-metadata/#core-metadata].

Importante

Eles *não* são necessariamente equivalentes ou correspondem 1:1 aos nomes de *pacotes de importação* de nível superior que podem ser importados dentro do código Python. Um *pacote de distribuição* pode conter vários *pacotes de importação* (e módulos únicos), e um *pacote de importação* de nível superior pode ser mapeado para vários *pacotes de*

distribuição se for um pacote de espaço de nomes. Você pode usar packages_distributions() para obter um mapeamento entre eles.

Por padrão, os metadados de distribuição podem residir no sistema de arquivos ou em arquivos zip em **`sys.path`**. Através de um mecanismo de extensão, os metadados podem residir em praticamente qualquer lugar.

Ver também

https://importlib-metadata.readthedocs.io/

A documentação para `importlib_metadata`, que fornece um backport de `importlib.metadata`. Isso inclui uma referência de API [https://importlib-metadata.readthedocs.io/en/latest/api.html] para as classes e funções deste módulo, bem como um guia de migração [https://importlib-metadata.readthedocs.io/en/latest/migration.html] para usuários existentes de `pkg_resources`.

Visão Geral

Digamos que você queira obter a string da versão de um Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package] que você instalou usando `pip`. Começamos criando um ambiente virtual e instalando algo nele:

```
$ python -m venv exemplo
$ source exemplo/bin/activate
(exemplo) $ python -m pip install wheel
```

Você pode obter a string de versão para `wheel` executando o seguinte:

```
(example) $ python
>>> from importlib.metadata import version
>>> version('wheel')
'0.32.3'
```

Você também pode obter uma coleção de pontos de entrada selecionáveis pelas propriedades do EntryPoint (normalmente ‘group’ ou ‘name’), como `console_scripts`, `distutils.commands` e outros. Cada grupo contém uma coleção de objetos EntryPoint.

Você pode obter os metadados para uma distribuição:

```
>>> from importlib.metadata import metadata
>>> list(metadata('wheel'))
['Metadata-Version', 'Name', 'Version', 'Summary', 'Home-page',
```

Você também pode obter uma número da versão da distribuição, listar seus arquivos constituintes e obter uma lista dos Requisitos de distribuição da distribuição.

exception importlib.metadata.PackageNotFoundError

Subclasse de **`ModuleNotFoundError`** levantada por diversas funções neste módulo quando consultada sobre um pacote de distribuição que não está instalado no ambiente Python atual.

exception importlib.metadata.MetadataNotFound

Subclass of **`FileNotFoundError`** raised when attempting to load metadata from a distribution folder that is empty or otherwise does not contain a metadata file.

API funcional

Este pacote fornece a seguinte funcionalidade por meio de sua API pública.

Pontos de entrada

importlib.metadata.entry_points(****select_params*)

Retorna uma instância de **`EntryPoints`** descrevendo pontos de entrada para o ambiente atual. Quaisquer parâmetros nomeados fornecidos são passados para o método **`select()`** para comparação com os atributos das definições de pontos de entrada individuais.

Note: to query for entry points based on **`EntryPoint.dist`** attribute, use **`Distribution.entry_points()`** instead (as different **`Distribution`** instances do not currently compare equal, even if they have the same attributes)

class importlib.metadata.EntryPoints

Detalhes de uma coleção de pontos de entrada instalados.

Também fornece um atributo `.groups` que relata todos os grupos de pontos de entrada identificados e um atributo `.names` que relata todos os nomes de pontos de entrada identificados.

class importlib.metadata.EntryPoint

Detalhes de um ponto de entrada instalado.

Cada instância de **`EntryPoint`** tem os atributos `.name`, `.group` e `.value` e um método `.load()` para resolver o valor. Há também os atributos `.module`, `.attr` e `.extras` para obter os componentes do atributo `.value` e `.dist` para obter informações sobre o pacote de distribuição que fornece o ponto de entrada.

Consultar todos os pontos de entrada:

```
>>> eps = entry_points()
```

A função **`entry_points()`** retorna um objeto **`EntryPoints`**, uma coleção de todos os objetos **`EntryPoint`** com atributos `names` e `groups` por conveniência:

```
>>> sorted(eps.groups)
['console_scripts', 'distutils.commands', 'distutils.setup_keywo
```

`EntryPoints` possui um método **`select()`** para selecionar pontos de entrada correspondendo a propriedades específicas. Selecione pontos de entrada no grupo `console_scripts`:

```
>>> scripts = eps.select(group='console_scripts')
```

Equivalentemente, já que **`entry_points()`** passa argumentos nomeados para a seleção:

```
>>> scripts = entry_points(group='console_scripts')
```

Escolha um script específico chamado “wheel” (encontrado no projeto wheel):

```
>>> 'wheel' in scripts.names
True
>>> wheel = scripts['wheel']
```

De forma equivalente, consulte esse ponto de entrada durante a seleção:

```
>>> (wheel,) = entry_points(group='console_scripts', name='wheel
>>> (wheel,) = entry_points().select(group='console_scripts', na
```

Inspecione o ponto de entrada resolvido:

```
>>> wheel
EntryPoint(name='wheel', value='wheel.cli:main', group='console_
>>> wheel.module
'wheel.cli'
>>> wheel.attr
'main'
>>> wheel.extras
[]
>>> main = wheel.load()
>>> main
<function main at 0x103528488>
```

O `group` e `name` são valores arbitrários definidos pelo autor do pacote e normalmente um cliente desejará resolver todos os pontos de entrada para um grupo específico. Leia a documentação do setuptools [https://setuptools.pypa.io/en/latest/userguide/entry_point.html] para obter mais informações sobre pontos de entrada, sua definição e uso.

Alterado na versão 3.12: Os pontos de entrada “selecionáveis” foram introduzidos em `importlib_metadata` 3.6 e Python 3.10. Antes dessas

mudanças, `entry_points` não aceitava parâmetros e sempre retornava um dicionário de pontos de entrada, chaveado por grupo. Com `importlib_metadata` 5.0 e Python 3.12, `entry_points` sempre retorna um objeto `EntryPoints`. Veja backports.entry_points_selectable [https://pypi.org/project/backports.entry_points_selectable/] para opções de compatibilidade.

Alterado na versão 3.13: Objetos `EntryPoint` não apresentam mais uma interface tupla ou similar (**`__getitem__()`**).

Metadados de distribuição

importlib.metadata.metadata(*distribution_name*)

Retorna os metadados de distribuição correspondentes ao pacote de distribuição nomeado como uma instância de **`PackageMetadata`**.

Levanta **`PackageNotFoundError`** se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

Raises **`MetadataNotFound`** if a distribution package is present but no METADATA file is present.

class importlib.metadata.PackageMetadata

Uma implementação concreta do protocolo PackageMetadata [https://importlib-metadata.readthedocs.io/en/latest/api.html#importlib_metadata.PackageMetadata].

Além de fornecer os métodos e atributos de protocolo definidos, subscrever a instância é equivalente a chamar o método **`get()`**.

Cada Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package] inclui alguns metadados, que você pode extrair usando a função **`metadata()`**:

```
>>> wheel_metadata = metadata('wheel')
```

As chaves da estrutura de dados retornada nomeiam as palavras reservadas dos metadados, e os valores são retornados sem análise dos metadados de

distribuição:

```
>>> wheel_metadata['Requires-Python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
```

`PackageMetadata` também apresenta um atributo **`json`** que retorna todos os metadados em um formato compatível com JSON pela **PEP 566** [https://peps.python.org/pep-0566/]:

```
>>> wheel_metadata.json['requires_python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
```

O conjunto completo de metadados disponíveis não é descrito aqui. Consulte a especificação de metadados principais do PyPA [https://packaging.python.org/pt_BR/latest/specifications/core-metadata/#core-metadata] para obter detalhes adicionais.

Alterado na versão 3.15: Previously and incidentally, if a METADATA file was missing from a distribution, an empty `PackageMetadata` would be returned, indistinguishable from an empty METADATA file. Now, a missing METADATA file triggers a `MetadataNotFound` exception.

Alterado na versão 3.10: A `Description` agora é incluída nos metadados quando apresentada através do payload. Os caracteres de continuação de linha foram removidos.

O atributo `json` foi adicionado.

Versões de distribuição

importlib.metadata.version(*distribution_name*)

Retorna a versão [https://packaging.python.org/pt-br/latest/specifications/core-metadata/#version] do pacote de distribuição instalado para o pacote de distribuição nomeado.

Levanta **`PackageNotFoundError`** se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

A função **`version()`** é a maneira mais rápida de obter o número de versão de um Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package], como uma string:

```
>>> version('wheel')
'0.32.3'
```

Arquivos de distribuição

importlib.metadata.files(*distribution_name*)

Return the full set of files contained within the named distribution package as **`PackagePath`** instances.

Levanta **`PackageNotFoundError`** se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

Retorna **`None`** se a distribuição for encontrada, mas os registros do banco de dados de instalação que relatam os arquivos associados ao pacote de distribuição estiverem ausentes.

class importlib.metadata.PackagePath

A **`pathlib.PurePath`** derived object with additional `dist`, `size`, and `hash` properties corresponding to the distribution package's installation metadata for that file, also:

locate()

If possible, return the concrete **`SimplePath`** allowing to access data, or raise a **`NotImplementedError`** otherwise.

class importlib.metadata.SimplePath

A protocol representing a minimal subset of **`pathlib.Path`** that allows to check if it `exists()`, to traverse using `joinpath()` and `parent`, and to retrieve data using `read_text()` and `read_bytes()`.

The **`files()`** function takes a Distribution Package [https://packaging.python.org/en/latest/glossary/#term-Distribution-Package] name and returns all of the files installed by this distribution. For example:

```
>>> util = [p for p in files('wheel') if 'util.py' in str(p)][0]
>>> util
PackagePath('wheel/util.py')
>>> util.size
859
>>> util.dist
<importlib.metadata._hooks.PathDistribution object at 0x101e0cef
>>> util.hash
<FileHash mode: sha256 value: bYkw5oMccfazVCoYQwKkkemoVyMAFoR34m
```

Uma vez que tenha o arquivo, você também pode ler seu conteúdo:

```
>>> print(util.read_text())
import base64
import sys
...
def as_bytes(s):
    if isinstance(s, text_type):
        return s.encode('utf-8')
    return s
```

Você também pode usar o método **`locate()`** para obter o caminho absoluto para o arquivo:

```
>>> util.locate()
PosixPath('/home/gustav/example/lib/site-packages/wheel/util.py'
```

No caso em que o arquivo de metadados que lista os arquivos (`RECORD` ou `SOURCES.txt`) estiver faltando, **`files()`** retornará **`None`**. O chamador pode querer agrupar chamadas para **`files()`** em always_iterable [https://more-itertools.readthedocs.io/en/stable/api.html#more_itertools.always_iterable] ou de outra forma se proteger contra isso condição se a distribuição de destino não for conhecida por ter os metadados presentes.

Requisitos de distribuição

importlib.metadata.requires(*distribution_name*)

Retorna os especificadores de dependência declarados para o pacote de distribuição nomeado.

Levanta **`PackageNotFoundError`** se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

Para obter o conjunto completo de requisitos para um Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package], use a função **`requires()`**:

```
>>> requires('wheel')
["pytest (>=3.0.0) ; extra == 'test'", "pytest-cov ; extra == 't
```

Mapeando importação pra pacotes de distribuição

importlib.metadata.packages_distributions()

Retorna um mapeamento do módulo de nível superior e importa os nomes dos pacotes encontrados via **`sys.meta_path`** para os nomes dos pacotes de distribuição (se houver) que fornecem os arquivos correspondentes.

Para permitir pacotes de espaço de nomes (que podem ter membros fornecidos por vários pacotes de distribuição), cada nome de importação de nível superior é mapeado para uma lista de nomes de distribuição em vez de mapear diretamente para um único nome.

Um método conveniente para resolver o nome do Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package] (ou nomes, no caso de um pacote de espaço de nomes) que fornece cada módulo Python de nível superior importável ou Pacote de Importação [https://packaging.python.org/pt_BR/latest/glossary/#term-Import-Package]:

```
>>> packages_distributions()
{'importlib_metadata': ['importlib-metadata'], 'yaml': ['PyYAML'
```

Algumas instalações editáveis não fornecem nomes de nível superior [https://github.com/pypa/packaging-problems/issues/609] e, portanto, esta função não é

confiável com tais instalações.

Adicionado na versão 3.10.

Distribuições

While the module level API described above is the most common and convenient usage, all that information is accessible from the **Distribution** class. **Distribution** is an abstract object that represents the metadata for a Python Distribution Package [https://packaging.python.org/en/latest/glossary/#term-Distribution-Package]. Get the concrete **Distribution** subclass instance for an installed distribution package by calling the **distribution()** function:

```
>>> from importlib.metadata import distribution
>>> dist = distribution('wheel')
>>> type(dist)
<class 'importlib.metadata.PathDistribution'>
```

importlib.metadata.distribution(*distribution_name*)

Retorna uma instância de **Distribution** descrevendo o pacote de distribuição nomeado.

Levanta **PackageNotFoundError** se o pacote de distribuição nomeado não estiver instalado no ambiente Python atual.

Thus, an alternative way to get e.g. the version number is through the **Distribution.version** attribute:

```
>>> dist.version
'0.32.3'
```

The same applies for **entry_points()** and **files()**.

class importlib.metadata.Distribution

Detalhes de um pacote de distribuição instalado.

Observação: instâncias diferentes de **Distribution** não são atualmente consideradas iguais em uma comparação, mesmo que estejam

relacionadas à mesma distribuição instalada e, portanto, tenham os mesmos atributos.

static at(*path*)

classmethod from_name(*name*)

Return a **`Distribution`** instance at the given path or with the given name.

classmethod discover(*, *context=None*, ***kwargs*)

Returns an iterable of **`Distribution`** instances for all packages (see distribution-discovery).

The optional argument *context* is a **`DistributionFinder.Context`** instance, used to modify the search for distributions. Alternatively, *kwargs* may contain keyword arguments for constructing a new **`DistributionFinder.Context`**.

metadata*: PackageMetadata*

Raises **`MetadataNotFound`** if the METADATA file is not present in the distribution.

There are all kinds of additional metadata available on **`Distribution`** instances as a **`PackageMetadata`** instance:

```
>>> dist.metadata['Requires-Python']
'>=2.7, !=3.0.*, !=3.1.*, !=3.2.*, !=3.3.*'
>>> dist.metadata['License']
'MIT'
```

O conjunto completo de metadados disponíveis não é descrito aqui. Consulte a especificação de metadados principais do PyPA [https://packaging.python.org/pt_BR/latest/specifications/core-metadata/#core-metadata] para obter detalhes adicionais.

name*: str*

requires*: list[str]*

version*: str*

A few metadata fields are also available as shortcut properties.

Adicionado na versão 3.10: The `name` shortcut was added.

origin

For editable packages, an `origin` property may present **PEP 610** [https://peps.python.org/pep-0610/] metadata (for non-editable packages, `origin` is **`None`**):

```
>>> dist.origin.url
'file:///caminho/para/wheel-0.32.3.editable-py3-none-any.w
```

The `origin` object follows the Direct URL Data Structure [https://packaging.python.org/en/latest/specifications/direct-url-data-structure/].

Adicionado na versão 3.13.

entry_points*: EntryPoints*

The entry points provided by this distribution package.

files*: list[PackagePath] | None*

All files contained in this distribution package. Like **`files()`**, this returns **`None`** if there are no records.

The following two abstract methods need to be implemented when implementing-custom-providers:

locate_file(*path*)

Like **`PackagePath.locate()`**, return a **`SimplePath`** for the given path. Takes a **`os.PathLike`** or a **`str`**.

read_text(*filename*)

A shortcut for `distribution.locate_file(filename).read_text()`.

Descoberta de distribuição

Por padrão, este pacote fornece suporte embutido para descoberta de metadados para sistema de arquivos e arquivo zip Pacotes de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package]. Esta pesquisa do localizador de metadados tem como padrão `sys.path`, mas varia um pouco na maneira como ela interpreta esses valores em relação a outras mecanismo de importação. Em particular:

- `importlib.metadata` não honra objetos **`bytes`** em `sys.path`.
- `importlib.metadata` irá incidentalmente honrar objetos **`pathlib.Path`** em `sys.path` mesmo que tais valores sejam ignorados para importações.

class importlib.metadata.DistributionFinder

A **`MetaPathFinder`** subclass capable of discovering installed distributions.

Custom providers should implement this interface in order to supply metadata.

class Context(***kwargs*)

A **`Context`** gives a custom provider a means to solicit additional details from the callers of distribution discovery functions like **`distributions()`** or **`Distribution.discover()`** beyond **`.name`** and **`.path`** when searching for distributions.

For example, a provider could expose suites of packages in either a “public” or “private” `realm`. A caller of distribution discovery functions may wish to query only for distributions in a particular realm and could call `distributions(realm="private")` to signal to the custom provider to only include distributions from that realm.

Each **`DistributionFinder`** must expect any parameters and should attempt to honor the canonical parameters defined below when appropriate.

See the section on Implementando provedores personalizados for more details.

name

Specific name for which a distribution finder should match.

A `.name` of `None` matches all distributions.

path

A property providing the sequence of directory paths that a distribution finder should search.

Typically refers to Python installed package paths such as "site-packages" directories and defaults to `sys.path`.

importlib.metadata.distributions(***kwargs*)

Returns an iterable of `Distribution` instances for all packages.

The *kwargs* argument may contain either a keyword argument `context`, a `DistributionFinder.Context` instance, or pass keyword arguments for constructing a new `DistributionFinder.Context`. The `DistributionFinder.Context` is used to modify the search for distributions.

Implementando provedores personalizados

`importlib.metadata` aborda duas superfícies de API, uma para *consumidores* e outra para *provedores*. A maioria dos usuários são consumidores, consumindo metadados fornecidos pelos pacotes. Existem outros casos de uso, no entanto, em que os usuários desejam expor metadados por meio de algum outro mecanismo, como junto com um importador personalizado. Esse caso de uso exige um *provedor personalizado*.

Como os metadados de Pacote de Distribuição [https://packaging.python.org/pt_BR/latest/glossary/#term-Distribution-Package] não estão disponíveis por meio de pesquisas a `sys.path` ou carregadores de pacotes diretamente, os metadados de uma distribuição são encontrados através de localizadores do sistema de importação. Para encontrar os metadados de um

pacote de distribuição, `importlib.metadata` consulta a lista de localizadores de metacaminho em **`sys.meta_path`**.

A implementação possui ganchos integrados ao `PathFinder`, servindo metadados para pacotes de distribuição encontrados no sistema de arquivos.

The abstract class **`importlib.abc.MetaPathFinder`** defines the interface expected of finders by Python's import system. `importlib.metadata` extends this protocol by looking for an optional `find_distributions` callable on the finders from **`sys.meta_path`** and presents this extended interface as the **`DistributionFinder`** abstract base class, which defines this abstract method:

```
@abc.abstractmethod
def find_distributions(context=DistributionFinder.Context()) ->
    """Return an iterable of all Distribution instances capable
    loading the metadata for packages for the indicated ``contex
    """
```

The **`DistributionFinder.Context`** object provides **`path`** and **`name`** properties indicating the path to search and name to match and may supply other relevant context sought by the consumer.

In practice, to support finding distribution package metadata in locations other than the file system, subclass **`Distribution`** and implement the abstract methods. Then from a custom finder, return instances of this derived **`Distribution`** in the `find_distributions()` method.

Exemplo

Imagine um localizador personalizado que carrega módulos Python de um banco de dados:

```
class DatabaseImporter(importlib.abc.MetaPathFinder):
    def __init__(self, db):
        self.db = db

    def find_spec(self, fullname, target=None) -> ModuleSpec:
        return self.db.spec_from_name(fullname)
```

```
sys.meta_path.append(DatabaseImporter(connect_db(...)))
```

That importer now presumably provides importable modules from a database, but it provides no metadata or entry points. For this custom importer to provide metadata, it would also need to implement **DistributionFinder**:

```
from importlib.metadata import DistributionFinder

class DatabaseImporter(DistributionFinder):
    ...

    def find_distributions(self, context=DistributionFinder.Cont
        query = dict(name=context.name) if context.name else {}
        for dist_record in self.db.query_distributions(query):
            yield DatabaseDistribution(dist_record)
```

Desta forma, `query_distributions` retornaria registros para cada distribuição servida pelo banco de dados correspondente à consulta. Por exemplo, se `requests-1.0` estiver no banco de dados, `find_distributions` produziria um `DatabaseDistribution` para `Context(name='requests')` ou `Context(name=None)`.

Por uma questão de simplicidade, este exemplo ignora `context.path`. O atributo `path` tem como padrão `sys.path` e é o conjunto de caminhos de importação a serem considerados na pesquisa. Um `DatabaseImporter` poderia funcionar potencialmente sem qualquer preocupação com um caminho de pesquisa. Supondo que o importador não faça particionamento, o caminho “path” seria irrelevante. Para ilustrar o propósito de `path`, o exemplo precisaria ilustrar um `DatabaseImporter` mais complexo cujo comportamento variasse dependendo de `sys.path`/`PYTHONPATH`. Nesse caso, `find_distributions` deve respeitar o `context.path` e produzir apenas `Distributions` pertinentes a esse caminho.

`DatabaseDistribution`, então, se pareceria com algo como:

```
class DatabaseDistribution(importlib.metadata.Distribution):
    def __init__(self, record):
```

```
        self.record = record

    def read_text(self, filename):
        """
        Lê um arquivo como "METADATA" para a atual distribuição.
        """
        if filename == "METADATA":
            return f"""Name: {self.record.name}
Version: {self.record.version}
"""
        if filename == "entry_points.txt":
            return "\n".join(
              f"""[{ep.group}]\n{ep.name}={ep.value}"""
              for ep in self.record.entry_points)

    def locate_file(self, path):
        raise RuntimeError("This distribution has no file system
```

Esta implementação básica deve fornecer metadados e pontos de entrada para pacotes servidos pelo `DatabaseImporter`, presumindo que o `record` forneça os atributos `.name`, `.version` e `.entry_points` adequados.

The `DatabaseDistribution` may also provide other metadata files, like `RECORD` (required for **`Distribution.files`**) or override the implementation of **`Distribution.files`**. See the source for more inspiration.

A inicialização do caminho de pesquisa de módulos `sys.path`

Um caminho de pesquisa de módulos é inicializado quando o Python é iniciado. Este caminho de pesquisa de módulos pode ser acessado em `sys.path`.

A primeira entrada no caminho de pesquisa de módulos é o diretório que contém o script de entrada, se houver. Caso contrário, a primeira entrada é o diretório atual, que é o caso ao executar o console interativo, um comando com `-c` ou um módulo com `-m`.

A variável de ambiente `PYTHONPATH` é frequentemente usada para adicionar diretórios ao caminho de pesquisa. Se esta variável de ambiente for encontrada, o conteúdo será adicionado ao caminho de pesquisa de módulos.

Nota

`PYTHONPATH` afetará todas as versões/ambientes Python instalados. Tenha cuidado ao definir isso em seu perfil do shell ou em variáveis de ambiente globais. O módulo `site` oferece técnicas mais diferenciadas, conforme mencionado abaixo.

Os próximos itens adicionados são os diretórios contendo módulos padrão do Python, bem como quaisquer módulos de extensão dos quais esses módulos dependem. Módulos de extensão são arquivos `.pyd` no Windows e arquivos `.so` em outras plataformas. O diretório com os módulos Python independentes de plataforma é chamado `prefix`. O diretório com os módulos de extensão é chamado `exec_prefix`.

A variável de ambiente `PYTHONHOME` pode ser usada para definir os locais `prefix` e `exec_prefix`. Caso contrário, esses diretórios serão encontrados

usando o executável Python como ponto de partida e, em seguida, procurando por vários arquivos e diretórios “de referência”. Observe que todos os links simbólicos são seguidos para que o local real do executável do Python seja usado como ponto de partida da pesquisa. O local do executável do Python é chamado `home`.

Uma vez que `home` é determinado, o diretório `prefix` é encontrado procurando primeiro por `python`*`majorversionminorversion`*`.zip` (`python311.zip`). No Windows, o arquivo zip é procurado em `home` e no Unix espera-se que o arquivo esteja em `lib`. Observe que o local esperado do arquivo zip é adicionado ao caminho de pesquisa de módulos, mesmo que o arquivo não exista. Se nenhum arquivo for encontrado, o Python no Windows continuará a pesquisa por `prefix` procurando por `Lib\os.py`. Python no Unix procurará por `lib/python`*`majorversion`*`.`*`minorversion`*`/os.py` (`lib/python3.11/os.py`). No Windows, `prefix` e `exec_prefix` são iguais, porém em outras plataformas `lib/python`*`majorversion`*`.`*`minorversion`*`/lib-dynload` (`lib/python3.11/lib-dynload`) é pesquisado e usado como âncora para `exec_prefix`. Em algumas plataformas, `lib` pode ser `lib64` ou outro valor, veja **`sys.platlibdir`** e **`PYTHONPLATLIBDIR`**.

Uma vez encontrados, `prefix` e `exec_prefix` estão disponíveis em **`sys.base_prefix`** e **`sys.base_exec_prefix`**, respectivamente.

Se **`PYTHONHOME`** não estiver definido e um arquivo `pyvenv.cfg` for encontrado ao lado do executável principal ou em seu diretório pai, **`sys.prefix`** e **`sys.exec_prefix`** serão definidos para o diretório que contém `pyvenv.cfg`; caso contrário, serão definidos com o mesmo valor que **`sys.base_prefix`** e **`sys.base_exec_prefix`**, respectivamente. Isso é usado por Ambientes virtuais.

Finally, the **`site`** module is processed and `site-packages` directories are added to the module search path. The **`PYTHONUSERBASE`** environment variable controls where is searched for user site-packages and the **`PYTHONNOUSERSITE`** environment variable prevents searching for user site-packages all together. A common way to customize the search path is to create **`sitecustomize`** or **`usercustomize`** modules as described in the **`site`** module documentation.

Nota

The command line options `-E`, `-P`, `-I`, `-S` and `-s` further affect path calculations, see their documentation for details.

Alterado na versão 3.14: `sys.prefix` e `sys.exec_prefix` agora são definidos para o diretório `pyvenv.cfg` durante a inicialização do caminho. Isso era feito anteriormente por `site` e, portanto, afetado por `-S`.

Ambientes virtuais

Ambientes virtuais colocam um arquivo `pyvenv.cfg` em seu prefixo, o que faz com que `sys.prefix` e `sys.exec_prefix` apontem para eles, em vez da instalação base.

Os valores `prefix` e `exec_prefix` da instalação base estão disponíveis em `sys.base_prefix` e `sys.base_exec_prefix`.

Além de ser usado como um marcador para identificar ambientes virtuais, `pyvenv.cfg` também pode ser usado para configurar a inicialização do `site`. Consulte a documentação de ambientes virtuais do `site`.

Nota

`PYTHONHOME` substitui a detecção `pyvenv.cfg`.

Nota

There are other ways “virtual environments” could be implemented. This documentation refers to implementations based on the `pyvenv.cfg` mechanism, such as `venv`, that many virtual environment implementations follow.

Arquivos _pth

Para substituir completamente **sys.path**, crie um arquivo `._pth` com o mesmo nome da biblioteca compartilhada ou executável (`python._pth` ou `python311._pth`). O caminho da biblioteca compartilhada é sempre conhecido no Windows, mas pode não estar disponível em outras plataformas. No arquivo `._pth`, especifique uma linha para cada caminho a ser adicionado a **sys.path**. O arquivo baseado no nome da biblioteca compartilhada substitui aquele baseado no executável, o que permite que os caminhos sejam restritos para qualquer programa que carregue o tempo de execução, se desejado.

Quando o arquivo existe, todos os registros e variáveis de ambiente são ignorados, o modo isolado é ativado, e **site** não é importado a menos que uma linha do arquivo especifique `import site`. Caminhos em branco e linhas começando com `#` são ignorados. Cada caminho pode ser absoluto ou relativo ao local do arquivo. Instruções de importação diferente de `site` não são permitidas, e código arbitrário não pode ser especificado.

Note que arquivos `.pth` (sem o sublinhado no início) serão processados normalmente pelo módulo **site** quando `import site` tiver sido especificado.

Python embarcado

Se o Python estiver incorporado em outra aplicação, **Py_InitializeFromConfig()** e a estrutura **PyConfig** podem ser usados para inicializar o Python. Os detalhes específicos do caminho estão descritos em Python Path Configuration.

Ver também

- Encontrando módulos para observações detalhadas do Windows.
- Utilizando Python em plataformas Unix para detalhes do Unix.

Serviços da Linguagem Python

O Python fornece vários módulos para ajudar a trabalhar com a linguagem Python. Estes módulos suportam tokenizing, parsing, análise de sintaxe, disassembly de bytecode e várias outras facilidades.

Esses módulos incluem:

- `ast` — Árvores de sintaxe abstrata
  - Gramática abstrata
  - Classes de nós
    - Nós raízes
    - Literais
    - Variáveis
    - Expressões
      - Subscrição
      - Compreensões
    - Instruções
      - Importações
    - Fluxo de controle
    - Correspondência de padrões
    - Anotações de tipos
    - Parâmetros de tipo
    - Definições de função e classe
    - Async e await
  - Auxiliares de `ast`
  - Sinalizadores do compilador
  - Uso na linha de comando
- `symtable` — Acesso a tabela de símbolos do compilador
  - Generating Symbol Tables
  - Examining Symbol Tables
  - Uso na linha de comando
- `token` — Constantes usadas com árvores de análises do Python
- `keyword` — Testando palavras reservadas do Python
- `tokenize` — Tokenizador para código-fonte Python

    - Tokenizing input
    - Uso na linha de comando
    - Exemplos
- `tabnanny` — Detecção de indentação ambígua
- `pyclbr` — Suporte a navegador de módulos do Python
    - Objetos Function
    - Objetos Class
- `py_compile` — Compila arquivos fonte do Python
    - Interface de Linha de Comando
- `compileall` — Compilar bibliotecas do Python para bytecode
    - Uso na linha de comando
    - Funções públicas
- `dis` — Disassembler de bytecode do Python
    - Interface de linha de comando
    - Análise de bytecode
    - Funções de análise
    - Instruções em bytecode do Python
    - Opcode collections
- `pickletools` — Ferramentas para desenvolvedores pickle
    - Uso na linha de comando
        - Opções de linha de comando
    - Interface programática

`ast` — Árvores de sintaxe abstrata

Código-fonte: Lib/ast.py [https://github.com/python/cpython/tree/3.15/Lib/ast.py]

O módulo **`ast`** ajuda as aplicações Python a processar árvores da gramática de sintaxe abstrata do Python. A sintaxe abstrata em si pode mudar em cada lançamento do Python; este módulo ajuda a descobrir programaticamente como é a gramática atual.

Uma árvore de sintaxe abstrata pode ser gerada passando **`ast.PyCF_ONLY_AST`** como um sinalizador para a função embutida **`compile()`**, ou usando o auxiliar **`parse()`** fornecido neste módulo. O resultado será uma árvore de objetos cujas classes herdam de **`ast.AST`**. Uma árvore de sintaxe abstrata pode ser compilada em um objeto código Python usando a função embutida **`compile()`**.

Gramática abstrata

A gramática abstrata está atualmente definida da seguinte forma:

```
-- ASDL's 4 builtin types are:
-- identifier, int, string, constant

module Python
{
    mod = Module(stmt* body, type_ignore* type_ignores)
        | Interactive(stmt* body)
        | Expression(expr body)
        | FunctionType(expr* argtypes, expr returns)

    stmt = FunctionDef(identifier name, arguments args,
                       stmt* body, expr* decorator_list, expr? r
                       string? type_comment, type_param* type_pa
          | AsyncFunctionDef(identifier name, arguments args,
                             stmt* body, expr* decorator_list, e
                             string? type_comment, type_param* t
```

```
| ClassDef(identifier name,
   expr* bases,
   keyword* keywords,
   stmt* body,
   expr* decorator_list,
   type_param* type_params)
| Return(expr? value)

| Delete(expr* targets)
| Assign(expr* targets, expr value, string? type_comme
| TypeAlias(expr name, type_param* type_params, expr v
| AugAssign(expr target, operator op, expr value)
-- 'simple' indicates that we annotate simple name wit
| AnnAssign(expr target, expr annotation, expr? value,

-- use 'orelse' because else is a keyword in target la
| For(expr target, expr iter, stmt* body, stmt* orelse
| AsyncFor(expr target, expr iter, stmt* body, stmt* o
| While(expr test, stmt* body, stmt* orelse)
| If(expr test, stmt* body, stmt* orelse)
| With(withitem* items, stmt* body, string? type_comme
| AsyncWith(withitem* items, stmt* body, string? type_

| Match(expr subject, match_case* cases)

| Raise(expr? exc, expr? cause)
| Try(stmt* body, excepthandler* handlers, stmt* orels
| TryStar(stmt* body, excepthandler* handlers, stmt* o
| Assert(expr test, expr? msg)

| Import(alias* names, int? is_lazy)
| ImportFrom(identifier? module, alias* names, int? le

| Global(identifier* names)
| Nonlocal(identifier* names)
| Expr(expr value)
| Pass | Break | Continue

-- col_offset is the byte offset in the utf8 string th
attributes (int lineno, int col_offset, int? end_liner

-- BoolOp() can use left & right?
```

```
expr = BoolOp(boolop op, expr* values)
     | NamedExpr(expr target, expr value)
     | BinOp(expr left, operator op, expr right)
     | UnaryOp(unaryop op, expr operand)
     | Lambda(arguments args, expr body)
     | IfExp(expr test, expr body, expr orelse)
     | Dict(expr?* keys, expr* values)
     | Set(expr* elts)
     | ListComp(expr elt, comprehension* generators)
     | SetComp(expr elt, comprehension* generators)
     | DictComp(expr key, expr? value, comprehension* genera
     | GeneratorExp(expr elt, comprehension* generators)
     -- the grammar constrains where yield expressions can c
     | Await(expr value)
     | Yield(expr? value)
     | YieldFrom(expr value)
     -- need sequences for compare to distinguish between
     -- x < 4 < 3 and (x < 4) < 3
     | Compare(expr left, cmpop* ops, expr* comparators)
     | Call(expr func, expr* args, keyword* keywords)
     | FormattedValue(expr value, int conversion, expr? form
     | Interpolation(expr value, constant str, int conversic
     | JoinedStr(expr* values)
     | TemplateStr(expr* values)
     | Constant(constant value, string? kind)

     -- the following expression can appear in assignment co
     | Attribute(expr value, identifier attr, expr_context c
     | Subscript(expr value, expr slice, expr_context ctx)
     | Starred(expr value, expr_context ctx)
     | Name(identifier id, expr_context ctx)
     | List(expr* elts, expr_context ctx)
     | Tuple(expr* elts, expr_context ctx)

     -- can appear only in Subscript
     | Slice(expr? lower, expr? upper, expr? step)

      -- col_offset is the byte offset in the utf8 string th
      attributes (int lineno, int col_offset, int? end_liner

expr_context = Load | Store | Del

boolop = And | Or
```

```
operator = Add | Sub | Mult | MatMult | Div | Mod | Pow | LS
             | RShift | BitOr | BitXor | BitAnd | FloorDiv

unaryop = Invert | Not | UAdd | USub

cmpop = Eq | NotEq | Lt | LtE | Gt | GtE | Is | IsNot | In |

comprehension = (expr target, expr iter, expr* ifs, int is_a

excepthandler = ExceptHandler(expr? type, identifier? name,
                attributes (int lineno, int col_offset, int?

arguments = (arg* posonlyargs, arg* args, arg? vararg, arg*
             expr?* kw_defaults, arg? kwarg, expr* defaults)

arg = (identifier arg, expr? annotation, string? type_commer
       attributes (int lineno, int col_offset, int? end_line

-- keyword arguments supplied to call (NULL identifier for *
keyword = (identifier? arg, expr value)
           attributes (int lineno, int col_offset, int? end_

-- import name with optional 'as' alias.
alias = (identifier name, identifier? asname)
         attributes (int lineno, int col_offset, int? end_li

withitem = (expr context_expr, expr? optional_vars)

match_case = (pattern pattern, expr? guard, stmt* body)

pattern = MatchValue(expr value)
        | MatchSingleton(constant value)
        | MatchSequence(pattern* patterns)
        | MatchMapping(expr* keys, pattern* patterns, identi
        | MatchClass(expr cls, pattern* patterns, identifier

        | MatchStar(identifier? name)
        -- The optional "rest" MatchMapping parameter handle

        | MatchAs(pattern? pattern, identifier? name)
        | MatchOr(pattern* patterns)
```

```
            attributes (int lineno, int col_offset, int end_lir

    type_ignore = TypeIgnore(int lineno, string tag)

    type_param = TypeVar(identifier name, expr? bound, expr? def
               | ParamSpec(identifier name, expr? default_value)
               | TypeVarTuple(identifier name, expr? default_val
               attributes (int lineno, int col_offset, int end_l
}
```

Classes de nós

class ast.AST

Esta é a base abstrata de todas as classes de nós de AST. As classes de nós reais são derivadas do arquivo `Parser/Python.asdl`, que é reproduzido acima. Elas são definidas no módulo C **`_ast`** e reexportadas no **`ast`**.

Há uma classe definida para cada símbolo do lado esquerdo na gramática abstrata (por exemplo, **`ast.stmt`** ou **`ast.expr`**). Além disso, existe uma classe definida para cada construtor no lado direito; essas classes herdam das classes para as árvores do lado esquerdo. Por exemplo, **`ast.BinOp`** herda de **`ast.expr`**. Para regras de produção com alternativas ("somas"), a classe do lado esquerdo é abstrata: apenas instâncias de nós construtores específicos são criadas.

_fields

Cada classe concreta possui um atributo **`_fields`** que fornece os nomes de todos os nós filhos.

Cada instância de uma classe concreta tem um atributo para cada nó filho, do tipo definido na gramática. Por exemplo, as instâncias **`ast.BinOp`** possuem um atributo **`left`** do tipo **`ast.expr`**.

Se estes atributos estiverem marcados como opcionais na gramática (usando um ponto de interrogação), o valor pode ser `None`. Se os atributos puderem ter valor zero ou mais (marcados com um

asterisco), os valores serão representados como listas do Python. Todos os atributos possíveis devem estar presentes e ter valores válidos ao compilar uma AST com **compile()**.

_field_types

O atributo **_field_types** em cada classe concreta é um dicionário que mapeia nomes de campos (como também listado em **_fields**) para seus tipos.

```
>>> ast.TypeVar._field_types
{'name': <class 'str'>, 'bound': ast.expr | None, 'defaul
```

Adicionado na versão 3.13.

lineno
col_offset
end_lineno
end_col_offset

As instâncias das subclasses **ast.expr** e **ast.stmt** possuem os atributos **lineno**, **col_offset**, **end_lineno** e **end_col_offset**. O **lineno** e **end_lineno** são o primeiro e o último número de linha do intervalo do texto de origem (indexado em 1, para que a primeira linha seja a linha 1) e o **col_offset** e **end_col_offset** são os deslocamentos de byte UTF-8 correspondentes do primeiro e do último tokens que geraram o nó. O deslocamento UTF-8 é registrado porque o analisador sintático usa UTF-8 internamente.

Observe que as posições finais não são exigidas pelo compilador e, portanto, são opcionais. O deslocamento final está *após* o último símbolo, por exemplo, é possível obter o segmento de origem de um nó de expressão de uma linha usando `source_line[node.col_offset : node.end_col_offset]`.

O construtor de uma classe **ast.T** analisa seus argumentos da seguinte forma:

- Se houver argumentos posicionais, deve haver tantos quanto houver itens em **T._fields**; eles serão atribuídos como atributos

desses nomes.
- Se houver argumentos nomeados, eles definirão os atributos dos mesmos nomes para os valores fornecidos.

Por exemplo, para criar e popular um nó **ast.UnaryOp**, você poderia usar

```
node = ast.UnaryOp(ast.USub(), ast.Constant(5, lineno=0, col_
                   lineno=0, col_offset=0)
```

Se um campo que é opcional na gramática for omitido do construtor, o padrão é `None`. Se um campo de lista for omitido, o padrão será a lista vazia. Se um campo do tipo **ast.expr_context** for omitido, o padrão é **Load()**. Se qualquer outro campo for omitido, uma **DeprecationWarning** será levantada e o nó AST não terá este campo. No Python 3.15, esta condição levantará um erro.

Alterado na versão 3.8: A classe **ast.Constant** é agora usada para todas as constantes.

Alterado na versão 3.9: Os índices simples são representados por seus valores, as fatias estendidas são representadas como tuplas.

Alterado na versão 3.13: Os construtores de nós AST foram alterados para fornecer valores padrão sensatos para campos omitidos: campos opcionais agora têm `None` como padrão, campos do tipo lista têm uma lista vazia como padrão e campos do tipo **ast.expr_context** têm **Load()** como padrão. Anteriormente, atributos omitidos não existiam nos nós construídos (tentar acessá-los levantava **AttributeError**).

Alterado na versão 3.14: A saída **__repr__()** dos nós **AST** inclui os valores dos campos dos nós.

Descontinuado desde a versão 3.8, removido na versão 3.14: Versões anteriores do Python forneciam as classes AST **ast.Num**, **ast.Str**, **ast.Bytes**, **ast.NameConstant** e **ast.Ellipsis**, que foram descontinuadas no Python 3.8. Essas classes foram removidas no Python 3.14, e sua funcionalidade foi substituída por **ast.Constant**.

Descontinuado desde a versão 3.9: Classes antigas **`ast.Index`** e **`ast.ExtSlice`** ainda estão disponíveis, mas serão removidos em versões futuras do Python. Enquanto isso, instanciá-las retornará uma instância de uma classe diferente.

Descontinuado desde a versão 3.13, removido na versão 3.15: Versões anteriores do Python permitiam a criação de nós AST sem campos obrigatórios. Da mesma forma, os construtores do nó AST permitiam argumentos nomeados arbitrários que eram definidos como atributos do nó AST, mesmo que não correspondessem a nenhum dos campos do nó AST. Estes casos agora levantam **`TypeError`**.

Descontinuado desde a versão 3.15, será removido na versão 3.20: Na gramática acima, as classes de nós da AST que correspondem a regras de produção com variantes (também conhecidas como "somas") são classes abstratas. Versões anteriores do Python permitiam a criação de instâncias diretas dessas classes de nós abstratas. Esse comportamento foi descontinuado e será removido no Python 3.20.

Nota

As descrições das classes de nós específicas exibidas aqui foram inicialmente adaptadas do fantástico projeto Green Tree Snakes [https://greentreesnakes.readthedocs.io/en/latest/] e de todos os seus contribuidores.

Nós raízes

class ast.Module(*body*, *type_ignores*)

Um módulo Python, como entrada de arquivo. Tipo de nó gerado por **`ast.parse()`** com *mode* no padrão `"exec"`.

`body` é uma **`list`** das Instruções do módulo.

`type_ignores` é uma **`list`** dos comentários de ignorar tipo do módulo; veja **`ast.parse()`** para mais detalhes.

```
>>> print(ast.dump(ast.parse('x = 1'), indent=4))
Module(
    body=[
        Assign(
            targets=[
                Name(id='x', ctx=Store())],
            value=Constant(value=1))])
```

class ast.Expression(*body*)

Uma única entrada de expressão Python. Tipo de nó gerado por **`ast.parse()`** quando *mode* é `"eval"`.

`body` é um nó único, um dos tipos de expressão.

```
>>> print(ast.dump(ast.parse('123', mode='eval'), indent=4))
Expression(
    body=Constant(value=123))
```

class ast.Interactive(*body*)

Uma única entrada interativa, como em Modo interativo. Tipo de nó gerado por **`ast.parse()`** quando *mode* é `"single"`.

`body` é uma **`list`** de nós de instrução.

```
>>> print(ast.dump(ast.parse('x = 1; y = 2', mode='single'),
Interactive(
    body=[
        Assign(
            targets=[
                Name(id='x', ctx=Store())],
            value=Constant(value=1)),
        Assign(
            targets=[
                Name(id='y', ctx=Store())],
            value=Constant(value=2))])
```

class ast.FunctionType(*argtypes*, *returns*)

Uma representação de comentários de tipo antigo para funções, já que as versões do Python anteriores a 3.5 não davam suporte às anotações

da **PEP 484** [https://peps.python.org/pep-0484/]. Tipo de nó gerado por **`ast.parse()`** quando *mode* é `"func_type"`.

Esses comentários de tipo ficariam assim:

```
def sum_two_number(a, b):
    # tipo: (int, int) -> int
    return a + b
```

`argtypes` é uma **`list`** de nós de expressão.

`returns` é um único nó de expressão.

```
>>> print(ast.dump(ast.parse('(int, str) -> List[int]', mode=
FunctionType(
    argtypes=[
        Name(id='int'),
        Name(id='str')],
    returns=Subscript(
        value=Name(id='List'),
        slice=Name(id='int')))
```

Adicionado na versão 3.8.

Literais

class ast.Constant(*value*, *kind*)

Um valor constante. O atributo `value` do literal `Constant` contém o objeto Python que ele representa. Os valores representados podem ser instâncias de **`str`**, **`bytes`**, **`int`**, **`float`**, **`complex`** e **`bool`**, e as constantes **`None`** e **`Ellipsis`**.

O atributo `kind` é uma string opcional. Para literais de string com o prefixo `u`, `kind` é definido como `'u'`. Para todas as outras constantes, `kind` é `None`.

```
>>> print(ast.dump(ast.parse('123', mode='eval'), indent=4))
Expression(
    body=Constant(value=123))
```

```
>>> print(ast.dump(ast.parse("u'hello'", mode='eval'), indent
Expression(
    body=Constant(value='hello', kind='u'))
```

class ast.FormattedValue(*value*, *conversion*, *format_spec*)

Nó que representa um único campo de formatação em uma f-string. Se a string contiver um único campo de formatação e nada mais, o nó poderá ser isolado, caso contrário ele aparecerá em `JoinedStr`.

- `value` é qualquer nó de expressão (como um literal, uma variável ou uma chamada de função).
- `conversion` é um inteiro:
  - -1: sem formatação
  - 97 (`ord('a')`): formatação e `ASCII` com `!a`
  - 114 (`ord('r')`): formatação em `repr()` com `!r`
  - 115 (`ord('s')`): formatação em `string` com `!s`
- `format_spec` é um nó `JoinedStr` que representa a formatação do valor, ou `None` se nenhum formato foi especificado. Tanto `conversion` quanto `format_spec` podem ser configurados ao mesmo tempo.

class ast.JoinedStr(*values*)

Uma f-string, compreendendo uma série de nós `FormattedValue` e `Constant`.

```
>>> print(ast.dump(ast.parse('f"sin({a}) is {sin(a):.3}"', mo
Expression(
    body=JoinedStr(
        values=[
            Constant(value='sin('),
            FormattedValue(
                value=Name(id='a'),
                conversion=-1),
            Constant(value=') is '),
            FormattedValue(
                value=Call(
                    func=Name(id='sin'),
                    args=[
                        Name(id='a')]),
```

```
                conversion=-1,
                format_spec=JoinedStr(
                    values=[
                        Constant(value='.3')]))]))
```

class ast.TemplateStr(*values*, ***)

Adicionado na versão 3.14.

Nó que representa um literal de string template, compreendendo uma série de nós `Interpolation` e `Constant`. Esses nós podem estar em qualquer ordem e não precisam ser intercalados.

```
>>> expr = ast.parse('t"{name} finished {place:ordinal}"', mo
>>> print(ast.dump(expr, indent=4))
Expression(
    body=TemplateStr(
        values=[
            Interpolation(
                value=Name(id='name'),
                str='name',
                conversion=-1),
            Constant(value=' finished '),
            Interpolation(
                value=Name(id='place'),
                str='place',
                conversion=-1,
                format_spec=JoinedStr(
                    values=[
                        Constant(value='ordinal')]))]))
```

class ast.Interpolation(*value*, *str*, *conversion*, *format_spec=None*)

Adicionado na versão 3.14.

Nó que representa um único campo de interpolação em um literal de string template.

- `value` é qualquer nó de expressão (como um literal, uma variável ou uma chamada de função). Tem o mesmo significado que `FormattedValue.value`.

- `str` é uma constante que contém o texto da expressão de interpolação.

  Se `str` for definido como `None`, `value` será usado para gerar código ao chamar **`ast.unparse()`**. Isso não garante mais que o código gerado seja idêntico ao original e se destina à geração de código.

- `conversion` é um inteiro:

  - -1: sem formatação
  - 97 (`ord('a')`): conversão em **`ASCII`** com `!a`
  - 114 (`ord('r')`): conversão em **`repr()`** com `!r`
  - 115 (`ord('s')`): conversão em **`string`** com `!s`

  Isso tem o mesmo significado que `FormattedValue.conversion`.

- `format_spec` é um nó **`JoinedStr`** que representa a formatação do valor, ou `None` se nenhum formato foi especificado. Tanto `conversion` quanto `format_spec` podem ser definidos ao mesmo tempo. Isso tem o mesmo significado que `FormattedValue.format_spec`.

class ast.List(*elts*, *ctx*)

class ast.Tuple(*elts*, *ctx*)

Uma lista ou tupla. `elts` contém uma lista de nós que representam os elementos. `ctx` é **`Store`** se o contêiner for um alvo de atribuição (ou seja, `(x,y)=algumacoisa`), e **`Load`** caso contrário.

```
>>> print(ast.dump(ast.parse('[1, 2, 3]', mode='eval'), inder
Expression(
    body=List(
        elts=[
            Constant(value=1),
            Constant(value=2),
            Constant(value=3)]))
>>> print(ast.dump(ast.parse('(1, 2, 3)', mode='eval'), inder
Expression(
    body=Tuple(
        elts=[
            Constant(value=1),
```

```
            Constant(value=2),
            Constant(value=3)]))
```

class ast.Set(*elts*)

Um conjunto. `elts` contém uma lista de nós que representam os elementos do conjunto.

```
>>> print(ast.dump(ast.parse('{1, 2, 3}', mode='eval'), inder
Expression(
    body=Set(
        elts=[
            Constant(value=1),
            Constant(value=2),
            Constant(value=3)]))
```

class ast.Dict(*keys*, *values*)

Um dicionário. `keys` e `values` contêm listas de nós que representam as chaves e os valores respectivamente, em ordem correspondente (o que seria retornado ao chamar `dictionary.keys()` e `dictionary.values()`).

Ao desempacotar o dicionário usando literais de dicionário, a expressão a ser expandida vai para a lista `values`, com um `None` na posição correspondente em `keys`.

```
>>> print(ast.dump(ast.parse('{"a":1, **d}', mode='eval'), ir
Expression(
    body=Dict(
        keys=[
            Constant(value='a'),
            None],
        values=[
            Constant(value=1),
            Name(id='d')]))
```

Variáveis

class ast.Name(*id*, *ctx*)

Um nome de variável. `id` contém o nome como uma string e `ctx` é um dos seguintes tipos.

class ast.Load
class ast.Store
class ast.Del

As referências de variáveis podem ser usadas para carregar o valor de uma variável, para atribuir um novo valor a ela ou para excluí-la. As referências de variáveis recebem um contexto para distinguir esses casos.

```
>>> print(ast.dump(ast.parse('a'), indent=4))
Module(
    body=[
        Expr(
            value=Name(id='a'))])

>>> print(ast.dump(ast.parse('a = 1'), indent=4))
Module(
    body=[
        Assign(
            targets=[
                Name(id='a', ctx=Store())],
            value=Constant(value=1))])

>>> print(ast.dump(ast.parse('del a'), indent=4))
Module(
    body=[
        Delete(
            targets=[
                Name(id='a', ctx=Del())])])
```

class ast.Starred(*value*, *ctx*)

Uma referência de variável `*var`. `value` contém a variável, normalmente um nó **`Name`**. Este tipo deve ser usado ao construir um nó **`Call`** com `*args`.

```
>>> print(ast.dump(ast.parse('a, *b = it'), indent=4))
Module(
```

```
    body=[
        Assign(
            targets=[
                Tuple(
                    elts=[
                        Name(id='a', ctx=Store()),
                        Starred(
                            value=Name(id='b', ctx=Store()),
                            ctx=Store())],
                    ctx=Store())],
            value=Name(id='it'))])
```

Expressões

class ast.Expr(*value*)

Quando uma expressão, como uma chamada de função, aparece como uma instrução por si só com seu valor de retorno não usado ou armazenado, ela é encapsulada neste contêiner. `value` contém um dos outros nós nesta seção, um nó **`Constant`**, um **`Name`**, um **`Lambda`**, um **`Yield`** ou **`YieldFrom`**.

```
>>> print(ast.dump(ast.parse('-a'), indent=4))
Module(
    body=[
        Expr(
            value=UnaryOp(
                op=USub(),
                operand=Name(id='a')))])
```

class ast.UnaryOp(*op*, *operand*)

Uma operação unária. `op` é o operador e `operand` qualquer nó de expressão.

class ast.UAdd
class ast.USub
class ast.Not
class ast.Invert

Tokens de operador unário. **`Not`** é a palavra reservada `not`, **`Invert`** é o operador ~.

```
>>> print(ast.dump(ast.parse('not x', mode='eval'), indent=4)
Expression(
    body=UnaryOp(
        op=Not(),
        operand=Name(id='x')))
```

class ast.BinOp(*left*, *op*, *right*)

Uma operação binária (como adição ou divisão). `op` é o operador, e `left` e `right` são quaisquer nós de expressão.

```
>>> print(ast.dump(ast.parse('x + y', mode='eval'), indent=4)
Expression(
    body=BinOp(
        left=Name(id='x'),
        op=Add(),
        right=Name(id='y')))
```

class ast.Add
class ast.Sub
class ast.Mult
class ast.Div
class ast.FloorDiv
class ast.Mod
class ast.Pow
class ast.LShift
class ast.RShift
class ast.BitOr
class ast.BitXor
class ast.BitAnd
class ast.MatMult

Tokens de operador binário.

class ast.BoolOp(*op*, *values*)

Uma operação booleana, ‘or’ ou ‘and’. `op` é **`Or`** ou **`And`**. `values` são os valores envolvidos. Operações consecutivas com o mesmo operador, como `a or b or c`, são recolhidas em um nó com vários valores.

Isso não inclui `not`, que é um **`UnaryOp`**.

```
>>> print(ast.dump(ast.parse('x or y', mode='eval'), indent=4
Expression(
    body=BoolOp(
        op=Or(),
        values=[
            Name(id='x'),
            Name(id='y')]))
```

class ast.And
class ast.Or

Tokens de operador booleano.

class ast.Compare(*left*, *ops*, *comparators*)

Uma comparação de dois ou mais valores. `left` é o primeiro valor na comparação, `ops` a lista de operadores e `comparators` a lista de valores após o primeiro elemento na comparação.

```
>>> print(ast.dump(ast.parse('1 <= a < 10', mode='eval'), inc
Expression(
    body=Compare(
        left=Constant(value=1),
        ops=[
            LtE(),
            Lt()],
        comparators=[
            Name(id='a'),
            Constant(value=10)]))
```

class ast.Eq
class ast.NotEq
class ast.Lt
class ast.LtE
class ast.Gt
class ast.GtE
class ast.Is
class ast.IsNot

class ast.In
class ast.NotIn

Tokens de operador de comparação.

class ast.Call(*func*, *args*, *keywords*)

Uma chamada de função. `func` é a função, que geralmente será um objeto **`Name`** ou **`Attribute`**. Dos argumentos:

- `args` contém uma lista dos argumentos passados por posição.
- `keywords` contém uma lista de objetos **`keyword`** representando argumentos passados como nomeados.

Os argumentos `args` e `keywords` são opcionais e padrão para listas vazias.

```
>>> print(ast.dump(ast.parse('func(a, b=c, *d, **e)', mode='e
Expression(
    body=Call(
        func=Name(id='func'),
        args=[
            Name(id='a'),
            Starred(
                value=Name(id='d'))],
        keywords=[
            keyword(
                arg='b',
                value=Name(id='c')),
            keyword(
                value=Name(id='e'))]))
```

class ast.keyword(*arg*, *value*)

Um argumento nomeado para uma chamada de função ou definição de classe. `arg` é uma string bruta do nome do parâmetro, `value` é um nó para passar.

class ast.IfExp(*test*, *body*, *orelse*)

Uma expressão como `a if b else c`. Cada campo contém um único nó, portanto, no exemplo a seguir, todos os três são nós **`Name`**.

```
>>> print(ast.dump(ast.parse('a if b else c', mode='eval'), i
Expression(
    body=IfExp(
        test=Name(id='b'),
        body=Name(id='a'),
        orelse=Name(id='c')))
```

class ast.Attribute(*value*, *attr*, *ctx*)

Acesso a atributo como, por exemplo, `d.keys`. `value` é um nó, normalmente um **`Name`**. `attr` é uma string simples fornecendo o nome do atributo, e `ctx` é **`Load`**, **`Store`** ou **`Del`** de acordo com como o atributo é acionado sobre.

```
>>> print(ast.dump(ast.parse('snake.colour', mode='eval'), in
Expression(
    body=Attribute(
        value=Name(id='snake'),
        attr='colour'))
```

class ast.NamedExpr(*target*, *value*)

Uma expressão nomeada. Este nó de AST é produzido pelo operador de expressões de atribuição (também conhecido como operador morsa). Ao contrário do nó **`Assign`** no qual o primeiro argumento pode ser múltiplos nós, neste caso ambos `target` e `value` devem ser nós únicos.

```
>>> print(ast.dump(ast.parse('(x := 4)', mode='eval'), indent
Expression(
    body=NamedExpr(
        target=Name(id='x', ctx=Store()),
        value=Constant(value=4)))
```

Adicionado na versão 3.8.

Subscrição

class ast.Subscript(*value*, *slice*, *ctx*)

Um subscrito, como `l[1]`. `value` é o objeto subscrito (geralmente sequência ou mapeamento). `slice` é um índice, fatia ou chave. Pode ser uma **`Tuple`** e conter uma **`Slice`**. `ctx` é **`Load`**, **`Store`** ou **`Del`** de acordo com a ação realizada com o subscrito.

```
>>> print(ast.dump(ast.parse('l[1:2, 3]', mode='eval'), inder
Expression(
    body=Subscript(
        value=Name(id='l'),
        slice=Tuple(
            elts=[
                Slice(
                    lower=Constant(value=1),
                    upper=Constant(value=2)),
                Constant(value=3)])))
```

class ast.Slice(*lower*, *upper*, *step*)

Fatiamento regular (no formato `lower:upper` ou `lower:upper:step`). Pode ocorrer apenas dentro do campo *slice* de **`Subscript`**, diretamente ou como um elemento de **`Tuple`**.

```
>>> print(ast.dump(ast.parse('l[1:2]', mode='eval'), indent=4
Expression(
    body=Subscript(
        value=Name(id='l'),
        slice=Slice(
            lower=Constant(value=1),
            upper=Constant(value=2))))
```

Compreensões

class ast.ListComp(*elt*, *generators*)

class ast.SetComp(*elt*, *generators*)

class ast.GeneratorExp(*elt*, *generators*)

class ast.DictComp(*key*, *value*, *generators*)

Lista e define compreensões, expressões geradoras e compreensões de dicionário. `elt` (ou `key` e `value`) é um único nó que representa a parte

que será avaliada para cada item.

Para compreensões de dicionário que utilizam desempacotamento, por exemplo `{**item for item in items}`, a expressão a ser expandida fica na posição da `key` (chave), e o `value` (valor) é `None`.

`generators` é uma lista de nós de **`comprehension`**.

```
>>> print(ast.dump(
...     ast.parse('[x for x in numbers]', mode='eval'),
...     indent=4,
... ))
Expression(
    body=ListComp(
        elt=Name(id='x'),
        generators=[
            comprehension(
                target=Name(id='x', ctx=Store()),
                iter=Name(id='numbers'),
                is_async=0)]))
>>> print(ast.dump(
...     ast.parse('{x: x**2 for x in numbers}', mode='eval'),
...     indent=4,
... ))
Expression(
    body=DictComp(
        key=Name(id='x'),
        value=BinOp(
            left=Name(id='x'),
            op=Pow(),
            right=Constant(value=2)),
        generators=[
            comprehension(
                target=Name(id='x', ctx=Store()),
                iter=Name(id='numbers'),
                is_async=0)]))
>>> print(ast.dump(
...     ast.parse('{x for x in numbers}', mode='eval'),
...     indent=4,
... ))
Expression(
    body=SetComp(
```

```
        elt=Name(id='x'),
        generators=[
            comprehension(
                target=Name(id='x', ctx=Store()),
                iter=Name(id='numbers'),
                is_async=0)]))
```

class ast.comprehension(*target*, *iter*, *ifs*, *is_async*)

Uma cláusula `for` em uma compreensão. `target` é a referência a ser usada para cada elemento - normalmente um nó **`Name`** ou **`Tuple`**. `iter` é o objeto sobre o qual iterar. `ifs` é uma lista de expressões de teste: cada cláusula `for` pode ter múltiplos `ifs`.

`is_async` indica que uma compreensão é assíncrona (usando um `async for` em vez de `for`). O valor é um número inteiro (0 ou 1).

```
>>> print(ast.dump(ast.parse('[ord(c) for line in file for c
...                 indent=4)) # Várias compreensões em uma.
Expression(
    body=ListComp(
        elt=Call(
            func=Name(id='ord'),
            args=[
                Name(id='c')]),
        generators=[
            comprehension(
                target=Name(id='line', ctx=Store()),
                iter=Name(id='file'),
                is_async=0),
            comprehension(
                target=Name(id='c', ctx=Store()),
                iter=Name(id='line'),
                is_async=0)]))

>>> print(ast.dump(ast.parse('(n**2 for n in it if n>5 if n<
...                 indent=4)) # compreensão do gerador
Expression(
    body=GeneratorExp(
        elt=BinOp(
            left=Name(id='n'),
            op=Pow(),
```

```
            right=Constant(value=2)),
        generators=[
            comprehension(
                target=Name(id='n', ctx=Store()),
                iter=Name(id='it'),
                ifs=[
                    Compare(
                        left=Name(id='n'),
                        ops=[
                            Gt()],
                        comparators=[
                            Constant(value=5)]),
                    Compare(
                        left=Name(id='n'),
                        ops=[
                            Lt()],
                        comparators=[
                            Constant(value=10)])],
                is_async=0)]))

>>> print(ast.dump(ast.parse('[i async for i in soc]', mode='
...                 indent=4)) # compreensão de async
Expression(
    body=ListComp(
        elt=Name(id='i'),
        generators=[
            comprehension(
                target=Name(id='i', ctx=Store()),
                iter=Name(id='soc'),
                is_async=1)]))
```

Instruções

class ast.Assign(*targets*, *value*, *type_comment*)

Uma atribuição. `targets` é uma lista de nós e `value` é um único nó.

Vários nós em `targets` representam a atribuição do mesmo valor a cada um. O desempacotamento é representada colocando uma **`Tuple`** ou **`List`** dentro de `targets`.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

```
>>> print(ast.dump(ast.parse('a = b = 1'), indent=4)) # Multi
Module(
    body=[
        Assign(
            targets=[
                Name(id='a', ctx=Store()),
                Name(id='b', ctx=Store())],
            value=Constant(value=1))])

>>> print(ast.dump(ast.parse('a,b = c'), indent=4)) # Unpacki
Module(
    body=[
        Assign(
            targets=[
                Tuple(
                    elts=[
                        Name(id='a', ctx=Store()),
                        Name(id='b', ctx=Store())],
                    ctx=Store())],
            value=Name(id='c'))])
```

class ast.AnnAssign(*target*, *annotation*, *value*, *simple*)

Uma atribuição com uma anotação de tipo. `target` é um nó único e pode ser uma classe **`Name`**, **`Attribute`** ou **`Subscript`**. `annotation` é a anotação, como um nó **`Constant`** ou **`Name`**. `value` é um único nó opcional.

`simple` é sempre 0 (indicando um alvo “complexo”) ou 1 (indicando um alvo “simples”). Um alvo “simples” consiste apenas em um nó **`Name`** que não aparece entre parênteses; todos os outros alvos são considerados complexos. Apenas alvos simples aparecem no dicionário **`__annotations__`** de módulos e classes.

```
>>> print(ast.dump(ast.parse('c: int'), indent=4))
Module(
    body=[
```

```
        AnnAssign(
            target=Name(id='c', ctx=Store()),
            annotation=Name(id='int'),
            simple=1)])

>>> print(ast.dump(ast.parse('(a): int = 1'), indent=4)) # An
Module(
    body=[
        AnnAssign(
            target=Name(id='a', ctx=Store()),
            annotation=Name(id='int'),
            value=Constant(value=1),
            simple=0)])

>>> print(ast.dump(ast.parse('a.b: int'), indent=4)) # Attrib
Module(
    body=[
        AnnAssign(
            target=Attribute(
                value=Name(id='a'),
                attr='b',
                ctx=Store()),
            annotation=Name(id='int'),
            simple=0)])

>>> print(ast.dump(ast.parse('a[1]: int'), indent=4)) # Subsc
Module(
    body=[
        AnnAssign(
            target=Subscript(
                value=Name(id='a'),
                slice=Constant(value=1),
                ctx=Store()),
            annotation=Name(id='int'),
            simple=0)])
```

class ast.AugAssign(*target*, *op*, *value*)

Atribuição aumentada, como `a += 1`. No exemplo a seguir, `target` é um nó **`Name`** para `x` (com o contexto **`Store`**), `op` é **`Add`**, e `value` é uma **`Constant`** com valor para 1.

O atributo `target` não pode ser da classe **`Tuple`** ou **`List`**, diferentemente dos alvos de **`Assign`**.

```
>>> print(ast.dump(ast.parse('x += 2'), indent=4))
Module(
    body=[
        AugAssign(
            target=Name(id='x', ctx=Store()),
            op=Add(),
            value=Constant(value=2))])
```

class ast.Raise(*exc*, *cause*)

Uma instrução `raise`. `exc` é o objeto de exceção a ser levantado, normalmente uma **`Call`** ou **`Name`**, ou `None` para um `raise` independente. `cause` é a parte opcional para `y` em `raise x from y`.

```
>>> print(ast.dump(ast.parse('raise x from y'), indent=4))
Module(
    body=[
        Raise(
            exc=Name(id='x'),
            cause=Name(id='y'))])
```

class ast.Assert(*test*, *msg*)

Uma asserção. `test` contém a condição, como um nó **`Compare`**. `msg` contém a mensagem de falha.

```
>>> print(ast.dump(ast.parse('assert x,y'), indent=4))
Module(
    body=[
        Assert(
            test=Name(id='x'),
            msg=Name(id='y'))])
```

class ast.Delete(*targets*)

Representa uma instrução `del`. `targets` é uma lista de nós, como nós **`Name`**, **`Attribute`** ou **`Subscript`**.

```
>>> print(ast.dump(ast.parse('del x,y,z'), indent=4))
Module(
```

```
    body=[
        Delete(
            targets=[
                Name(id='x', ctx=Del()),
                Name(id='y', ctx=Del()),
                Name(id='z', ctx=Del())])])
```

class ast.Pass

Uma instrução `pass`.

```
>>> print(ast.dump(ast.parse('pass'), indent=4))
Module(
    body=[
        Pass()])
```

class ast.TypeAlias(*name*, *type_params*, *value*)

Um apelido de tipo criado através da instrução **`type`**. `name` é o nome do apelido, `type_params` é uma lista de parâmetros de tipo, e `value` é o valor do apelido do tipo.

```
>>> print(ast.dump(ast.parse('type Alias = int'), indent=4))
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
            value=Name(id='int'))])
```

Adicionado na versão 3.12.

Outras instruções que são aplicáveis apenas dentro de funções ou laços são descritas em outras seções.

Importações

class ast.Import(*names*)

Uma instrução de importação. `names` é uma lista de nós de **`alias`**.

```
>>> print(ast.dump(ast.parse('import x,y,z'), indent=4))
Module(
    body=[
```

```
        Import(
            names=[
                alias(name='x'),
                alias(name='y'),
                alias(name='z')],
            is_lazy=0)])
```

class ast.ImportFrom(*module*, *names*, *level*)

Representa `from x import y`. `module` é uma string bruta do nome ‘from’, sem quaisquer pontos iniciais, ou `None` para instruções como `from . import foo`. `level` é um número inteiro que contém o nível da importação relativa (0 significa importação absoluta).

```
>>> print(ast.dump(ast.parse('from y import x,y,z'), indent=4
Module(
    body=[
        ImportFrom(
            module='y',
            names=[
                alias(name='x'),
                alias(name='y'),
                alias(name='z')],
            level=0,
            is_lazy=0)])
```

class ast.alias(*name*, *asname*)

Ambos os parâmetros são strings brutas dos nomes. `asname` pode ser `None` se o nome normal for usado.

```
>>> print(ast.dump(ast.parse('from ..foo.bar import a as b, c
Module(
    body=[
        ImportFrom(
            module='foo.bar',
            names=[
                alias(name='a', asname='b'),
                alias(name='c')],
            level=2,
            is_lazy=0)])
```

Fluxo de controle

Nota

Cláusulas opcionais como `else` são armazenadas como uma lista vazia se não estiverem presentes.

class ast.If(*test*, *body*, *orelse*)

Uma instrução `if`. `test` contém um único nó, como um nó **`Compare`**. `body` e `orelse` contêm, cada um, uma lista de nós.

As cláusulas `elif` não têm uma representação especial no AST, mas aparecem como nós extras de **`If`** dentro da seção `orelse` da cláusula anterior.

```
>>> print(ast.dump(ast.parse("""
... if x:
...     ...
... elif y:
...     ...
... else:
...     ...
... """), indent=4))
Module(
    body=[
        If(
            test=Name(id='x'),
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            orelse=[
                If(
                    test=Name(id='y'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))],
                    orelse=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

class ast.For(*target*, *iter*, *body*, *orelse*, *type_comment*)

Um laço `for`. `target` contém as variáveis às quais o laço atribui, como um único nó de **`Name`**, **`Tuple`**, **`List`**, **`Attribute`** ou **`Subscript`**. `iter` contém o item a ser repetido, novamente como um único nó. `body` e `orelse` contêm listas de nós para executar. Aqueles em `orelse` são executados se o laço terminar normalmente, ao invés de através de uma instrução `break`.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

```
>>> print(ast.dump(ast.parse("""
... for x in y:
...     ...
... else:
...     ...
... """), indent=4))
Module(
    body=[
        For(
            target=Name(id='x', ctx=Store()),
            iter=Name(id='y'),
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            orelse=[
                Expr(
                    value=Constant(value=Ellipsis))])])
```

class ast.While(*test*, *body*, *orelse*)

Um laço `while`. `test` contém a condição, como um nó de **`Compare`**.

```
>>> print(ast.dump(ast.parse("""
... while x:
...    ...
... else:
...    ...
... """), indent=4))
```

```
Module(
    body=[
        While(
            test=Name(id='x'),
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            orelse=[
                Expr(
                    value=Constant(value=Ellipsis))])])
```

class ast.Break

class ast.Continue

As instruções `break` e `continue`.

```
>>> print(ast.dump(ast.parse("""\
... for a in b:
...     if a > 5:
...         break
...     else:
...         continue
...
... """), indent=4))
Module(
    body=[
        For(
            target=Name(id='a', ctx=Store()),
            iter=Name(id='b'),
            body=[
                If(
                    test=Compare(
                        left=Name(id='a'),
                        ops=[
                            Gt()],
                        comparators=[
                            Constant(value=5)]),
                    body=[
                        Break()],
                    orelse=[
                        Continue()])])])
```

class ast.Try(*body*, *handlers*, *orelse*, *finalbody*)

Blocos `try`. Todos os atributos são uma lista de nós a serem executados, exceto `handlers`, que é uma lista de nós de **`ExceptHandler`**.

```
>>> print(ast.dump(ast.parse("""
... try:
...     ...
... except Exception:
...     ...
... except OtherException as e:
...     ...
... else:
...     ...
... finally:
...     ...
... """), indent=4))
Module(
    body=[
        Try(
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            handlers=[
                ExceptHandler(
                    type=Name(id='Exception'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
                ExceptHandler(
                    type=Name(id='OtherException'),
                    name='e',
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
            orelse=[
                Expr(
                    value=Constant(value=Ellipsis))],
            finalbody=[
                Expr(
                    value=Constant(value=Ellipsis))])])
```

class ast.TryStar(*body*, *handlers*, *orelse*, *finalbody*)

Blocos `try` que são seguidos por cláusulas `except*`. Os atributos são os mesmos de **`Try`**, mas os nós **`ExceptHandler`** em `handlers` são interpretados como blocos `except*` em vez de `except`.

```
>>> print(ast.dump(ast.parse("""
... try:
...     ...
... except* Exception:
...     ...
... """), indent=4))
Module(
    body=[
        TryStar(
            body=[
                Expr(
                    value=Constant(value=Ellipsis))],
            handlers=[
                ExceptHandler(
                    type=Name(id='Exception'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.11.

class ast.ExceptHandler(*type*, *name*, *body*)

Uma única cláusula `except`. `type` é o tipo de exceção que irá corresponder, normalmente um nó de **`Name`** (ou `None` para uma cláusula abrangente `except:`). `name` é uma string bruta para o nome conter a exceção, ou `None` se a cláusula não tiver `as foo`. `body` é uma lista de nós.

```
>>> print(ast.dump(ast.parse("""\
... try:
...     a + 1
... except TypeError:
...     pass
... """), indent=4))
Module(
    body=[
```

```
        Try(
            body=[
                Expr(
                    value=BinOp(
                        left=Name(id='a'),
                        op=Add(),
                        right=Constant(value=1)))],
            handlers=[
                ExceptHandler(
                    type=Name(id='TypeError'),
                    body=[
                        Pass()])])])
```

class ast.With(*items*, *body*, *type_comment*)

Um bloco `with`. `items` é uma lista de nós **`withitem`** representando os gerenciadores de contexto, e `body` é o bloco indentado dentro do contexto.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

class ast.withitem(*context_expr*, *optional_vars*)

Um único gerenciador de contexto em um bloco `with`. `context_expr` é o gerenciador de contexto, geralmente um nó de **`Call`**. `optional_vars` é um **`Name`**, **`Tuple`** ou **`List`** para a parte `as foo`, ou `None` se não for usado.

```
>>> print(ast.dump(ast.parse("""\
... with a as b, c as d:
...    something(b, d)
... """), indent=4))
Module(
    body=[
        With(
            items=[
                withitem(
                    context_expr=Name(id='a'),
                    optional_vars=Name(id='b', ctx=Store())),
                withitem(
                    context_expr=Name(id='c'),
```

```
                optional_vars=Name(id='d', ctx=Store()))]
        body=[
            Expr(
                value=Call(
                    func=Name(id='something'),
                    args=[
                        Name(id='b'),
                        Name(id='d')])])])
```

Correspondência de padrões

class ast.Match(*subject*, *cases*)

Uma instrução `match`. `subject` contém o assunto da correspondência (o objeto que está sendo comparado com os casos) e `cases` contém um iterável de nós de **`match case`** com os diferentes casos.

Adicionado na versão 3.10.

class ast.match_case(*pattern*, *guard*, *body*)

Um padrão de caso único em uma instrução `match`. `pattern` contém o padrão de correspondência com o qual o assunto será comparado. Observe que os nós **`AST`** produzidos para padrões diferem daqueles produzidos para expressões, mesmo quando compartilham a mesma sintaxe.

O atributo `guard` contém uma expressão que será avaliada se o padrão corresponder ao assunto.

`body` contém uma lista de nós a serem executados se o padrão corresponder e o resultado da avaliação da expressão de guarda for verdadeiro.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [x] if x>0:
...         ...
...     case tuple():
...         ...
```

```
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchAs(name='x')]),
                    guard=Compare(
                        left=Name(id='x'),
                        ops=[
                            Gt()],
                        comparators=[
                            Constant(value=0)]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
                match_case(
                    pattern=MatchClass(
                        cls=Name(id='tuple')),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

class ast.MatchValue(*value*)

Um literal de correspondência ou padrão de valor que é comparado por igualdade. `value` é um nó de expressão. Os nós de valor permitido são restritos conforme descrito na documentação da instrução de correspondência. Este padrão será bem-sucedido se o assunto da correspondência for igual ao valor avaliado.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case "Relevant":
...         ...
... """), indent=4))
Module(
```

```
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchValue(
                        value=Constant(value='Relevant')),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

class ast.MatchSingleton(*value*)

Um padrão literal de correspondência que compara por identidade. `value` é o singleton a ser comparado com: `None`, `True` ou `False`. Este padrão será bem-sucedido se o assunto da correspondência for a constante fornecida.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case None:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchSingleton(value=None),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

class ast.MatchSequence(*patterns*)

Um padrão de sequência de correspondência. `patterns` contém os padrões a serem comparados aos elementos do assunto se o assunto for uma sequência. Corresponde a uma sequência de comprimento variável se um dos subpadrões for um nó `MatchStar`, caso contrário corresponde a uma sequência de comprimento fixo.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [1, 2]:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchValue(
                                value=Constant(value=1)),
                            MatchValue(
                                value=Constant(value=2))]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])
```

Adicionado na versão 3.10.

class ast.MatchStar(*name*)

Corresponde ao restante da sequência em um padrão de sequência de correspondência de comprimento variável. Se `name` não for `None`, uma lista contendo os elementos restantes da sequência será vinculada a esse nome se o padrão de sequência geral for bem-sucedido.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [1, 2, *rest]:
...         ...
...     case [*_]:
```

```
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchValue(
                                value=Constant(value=1)),
                            MatchValue(
                                value=Constant(value=2)),
                            MatchStar(name='rest')]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
                match_case(
                    pattern=MatchSequence(
                        patterns=[
                            MatchStar()]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

class ast.MatchMapping(*keys*, *patterns*, *rest*)

Um padrão de mapeamento de correspondência. `keys` é uma sequência de nós de expressão. `patterns` é uma sequência correspondente de nós padrão. `rest` é um nome opcional que pode ser especificado para capturar os elementos restantes do mapeamento. As expressões-chave permitidas são restritas conforme descrito na documentação da instrução match.

Este padrão será bem-sucedido se o assunto for um mapeamento, todas as expressões-chave avaliadas estiverem presentes no mapeamento e o valor correspondente a cada chave corresponder ao subpadrão correspondente. Se `rest` não for `None`, um dict contendo os elementos

de mapeamento restantes será vinculado a esse nome se o padrão de mapeamento geral for bem-sucedido.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case {1: _, 2: _}:
...         ...
...     case {**rest}:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchMapping(
                        keys=[
                            Constant(value=1),
                            Constant(value=2)],
                        patterns=[
                            MatchAs(),
                            MatchAs()]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
                match_case(
                    pattern=MatchMapping(rest='rest'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

class ast.MatchClass(*cls*, *patterns*, *kwd_attrs*, *kwd_patterns*)

Um padrão de classe de correspondência. `cls` é uma expressão que fornece a classe nominal a ser correspondida. `patterns` é uma sequência de nós padrão a serem comparados com a sequência definida pela classe de atributos de correspondência de padrões. `kwd_attrs` é uma sequência de atributos adicionais a serem correspondidos

(especificados como argumentos nomeados no padrão de classe), `kwd_patterns` são os padrões correspondentes (especificados como valores de argumentos nomeados no padrão de classe).

Esse padrão será bem-sucedido se o assunto for uma instância da classe indicada, todos os padrões posicionais corresponderem aos atributos correspondentes definidos pela classe e quaisquer atributos, passados como argumentos nomeados, especificados corresponderem ao seu padrão correspondente.

Nota: as classes podem definir uma propriedade que retorna self para corresponder um nó padrão à instância que está sendo correspondida. Vários tipos internos também são combinados dessa forma, conforme descrito na documentação da instrução match.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case Point2D(0, 0):
...         ...
...     case Point3D(x=0, y=0, z=0):
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchClass(
                        cls=Name(id='Point2D'),
                        patterns=[
                            MatchValue(
                                value=Constant(value=0)),
                            MatchValue(
                                value=Constant(value=0))]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
                match_case(
                    pattern=MatchClass(
                        cls=Name(id='Point3D'),
```

```
                        kwd_attrs=[
                            'x',
                            'y',
                            'z'],
                        kwd_patterns=[
                            MatchValue(
                                value=Constant(value=0)),
                            MatchValue(
                                value=Constant(value=0)),
                            MatchValue(
                                value=Constant(value=0))]),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))])
```

Adicionado na versão 3.10.

class ast.MatchAs(*pattern*, *name*)

Uma correspondência “como padrão”, padrão de captura ou padrão curinga. `pattern` contém o padrão de correspondência com o qual o assunto será comparado. Se o padrão for `None`, o nó representa um padrão de captura (ou seja, um nome simples) e sempre terá sucesso.

O atributo `name` contém o nome que será vinculado se o padrão for bem-sucedido. Se `name` for `None`, `pattern` também deverá ser `None` e o nó representa o padrão curinga.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [x] as y:
...         ...
...     case _:
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
```

```
                    pattern=MatchAs(
                        pattern=MatchSequence(
                            patterns=[
                                MatchAs(name='x')]),
                        name='y'),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
                match_case(
                    pattern=MatchAs(),
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

class ast.MatchOr(*patterns*)

Uma correspondência de “padrão ou”. Um “padrão ou” corresponde cada um de seus subpadrões com o assunto, até que um seja bem-sucedido. O “padrão ou” é então considerado bem-sucedido. Se nenhum dos subpadrões for bem-sucedido, o padrão or falhará. O atributo `patterns` contém uma lista de nós de padrões de correspondência que serão comparados com o assunto.

```
>>> print(ast.dump(ast.parse("""
... match x:
...     case [x] | (y):
...         ...
... """), indent=4))
Module(
    body=[
        Match(
            subject=Name(id='x'),
            cases=[
                match_case(
                    pattern=MatchOr(
                        patterns=[
                            MatchSequence(
                                patterns=[
                                    MatchAs(name='x')]),
                            MatchAs(name='y')]),
```

```
                    body=[
                        Expr(
                            value=Constant(value=Ellipsis))]
```

Adicionado na versão 3.10.

Anotações de tipos

class ast.TypeIgnore(*lineno*, *tag*)

Um comentário `# type: ignore` localizado em *lineno*. *tag* é a tag opcional especificada pelo formato `# type: ignore <tag>`.

```
>>> print(ast.dump(ast.parse('x = 1 # type: ignore', type_com
Module(
    body=[
        Assign(
            targets=[
                Name(id='x', ctx=Store())],
            value=Constant(value=1))],
    type_ignores=[
        TypeIgnore(lineno=1, tag='')])
>>> print(ast.dump(ast.parse('x: bool = 1 # type: ignore[assi
Module(
    body=[
        AnnAssign(
            target=Name(id='x', ctx=Store()),
            annotation=Name(id='bool'),
            value=Constant(value=1),
            simple=1)],
    type_ignores=[
        TypeIgnore(lineno=1, tag='[assignment]')])
```

Nota

Os nós **`TypeIgnore`** não são gerados quando o parâmetro *type_comments* está definido como `False` (padrão). Consulte **`ast.parse()`** para obter mais detalhes.

Adicionado na versão 3.8.

Parâmetros de tipo

Parâmetros de tipo podem existir em classes, funções e apelidos de tipo.

class ast.TypeVar(*name*, *bound*, *default_value*)

Uma **`typing.TypeVar`**. `name` é o nome da variável de tipo. `bound` é o limite ou as restrições, se houver. Se `bound` for uma **`Tuple`**, ele representa restrições; caso contrário, representa o limite. `default_value` é o valor padrão; se **`TypeVar`** não tiver padrão, este atributo será definido como `None`.

```
>>> print(ast.dump(ast.parse("type Alias[T: int = bool] = lis
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
            type_params=[
                TypeVar(
                    name='T',
                    bound=Name(id='int'),
                    default_value=Name(id='bool'))],
            value=Subscript(
                value=Name(id='list'),
                slice=Name(id='T')))])
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionado o parâmetro *default_value*.

class ast.ParamSpec(*name*, *default_value*)

Uma **`typing.ParamSpec`**. `name` é o nome da especificação do parâmetro. `default_value` é o valor padrão; se **`ParamSpec`** não tiver padrão, este atributo será definido como `None`.

```
>>> print(ast.dump(ast.parse("type Alias[**P = [int, str]] =
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
```

```
            type_params=[
                ParamSpec(
                    name='P',
                    default_value=List(
                        elts=[
                            Name(id='int'),
                            Name(id='str')]))],
            value=Subscript(
                value=Name(id='Callable'),
                slice=Tuple(
                    elts=[
                        Name(id='P'),
                        Name(id='int')]))])
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionado o parâmetro *default_value*.

class ast.TypeVarTuple(*name*, *default_value*)

Uma `typing.TypeVarTuple`. `name` é o nome da tupla da variável de tipo. `default_value` é o valor padrão; se **`TypeVarTuple`** não tiver padrão, este atributo será definido como `None`.

```
>>> print(ast.dump(ast.parse("type Alias[*Ts = ()] = tuple[*T
Module(
    body=[
        TypeAlias(
            name=Name(id='Alias', ctx=Store()),
            type_params=[
                TypeVarTuple(name='Ts', default_value=Tuple(
            value=Subscript(
                value=Name(id='tuple'),
                slice=Tuple(
                    elts=[
                        Starred(
                            value=Name(id='Ts'))]))])
```

Adicionado na versão 3.12.

Alterado na versão 3.13: Adicionado o parâmetro *default_value*.

Definições de função e classe

class ast.FunctionDef(*name*, *args*, *body*, *decorator_list*, *returns*, *type_comment*, *type_params*)

Uma definição de função

- `name` é uma string bruta do nome da função.
- `args` é um nó **`arguments`**.
- `body` é a lista de nós dentro da função.
- `decorator_list` é a lista de decoradores a serem aplicados, armazenados primeiro na parte externa (ou seja, o primeiro da lista será aplicado por último).
- `returns` é a anotação de retorno.
- `type_params` é uma lista de parâmetros de tipo.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário.

Alterado na versão 3.12: Adicionado `type_params`.

class ast.Lambda(*args*, *body*)

`lambda` é uma definição mínima de função que pode ser usada dentro de uma expressão. Ao contrário de **`FunctionDef`**, `body` contém um único nó.

```
>>> print(ast.dump(ast.parse('lambda x,y: ...'), indent=4))
Module(
    body=[
        Expr(
            value=Lambda(
                args=arguments(
                    args=[
                        arg(arg='x'),
                        arg(arg='y')]),
                body=Constant(value=Ellipsis)))])
```

class ast.arguments(*posonlyargs*, *args*, *vararg*, *kwonlyargs*, *kw_defaults*, *kwarg*, *defaults*)

Os argumentos para uma função.

- `posonlyargs`, `args` e `kwonlyargs` são listas de nós **`arg`**.
- `vararg` e `kwarg` são nós **`arg`** únicos, referindo-se aos parâmetros `*args, **kwargs`.
- `kw_defaults` é uma lista de valores padrão para argumentos somente-nomeados. Se um for `None`, o argumento correspondente é necessário.
- `defaults` é uma lista de valores padrão para argumentos que podem ser passados posicionalmente. Se houver menos padrões, eles corresponderão aos últimos n argumentos.

class ast.arg(*arg*, *annotation*, *type_comment*)

Um único argumento em uma lista. `arg` é uma string bruta do nome do argumento; `annotation` é sua anotação, como um nó **`Name`**.

type_comment

`type_comment` é uma string opcional com a anotação de tipo como comentário

```
>>> print(ast.dump(ast.parse("""\
... @decorator1
... @decorator2
... def f(a: 'annotation', b=1, c=2, *d, e, f=3, **g) -> 'ret
...     pass
... """), indent=4))
Module(
    body=[
        FunctionDef(
            name='f',
            args=arguments(
                args=[
                    arg(
                        arg='a',
                        annotation=Constant(value='annotatior
                    arg(arg='b'),
                    arg(arg='c')],
```

```
                vararg=arg(arg='d'),
                kwonlyargs=[
                    arg(arg='e'),
                    arg(arg='f')],
                kw_defaults=[
                    None,
                    Constant(value=3)],
                kwarg=arg(arg='g'),
                defaults=[
                    Constant(value=1),
                    Constant(value=2)]),
            body=[
                Pass()],
            decorator_list=[
                Name(id='decorator1'),
                Name(id='decorator2')],
            returns=Constant(value='return annotation'))])
```

class ast.Return(*value*)

Uma instrução `return`.

```
>>> print(ast.dump(ast.parse('return 4'), indent=4))
Module(
    body=[
        Return(
            value=Constant(value=4))])
```

class ast.Yield(*value*)

class ast.YieldFrom(*value*)

Uma expressão `yield` ou `yield from`. Por serem expressões, elas devem ser agrupadas em um nó **`Expr`** se o valor enviado de volta não for usado.

```
>>> print(ast.dump(ast.parse('yield x'), indent=4))
Module(
    body=[
        Expr(
            value=Yield(
                value=Name(id='x')))])
```

```
>>> print(ast.dump(ast.parse('yield from x'), indent=4))
Module(
    body=[
        Expr(
            value=YieldFrom(
                value=Name(id='x')))])
```

class ast.Global(*names*)

class ast.Nonlocal(*names*)

Instruções `global` e `nonlocal`. `names` é uma lista de strings brutas.

```
>>> print(ast.dump(ast.parse('global x,y,z'), indent=4))
Module(
    body=[
        Global(
            names=[
                'x',
                'y',
                'z'])])
```

```
>>> print(ast.dump(ast.parse('nonlocal x,y,z'), indent=4))
Module(
    body=[
        Nonlocal(
            names=[
                'x',
                'y',
                'z'])])
```

class ast.ClassDef(*name*, *bases*, *keywords*, *body*, *decorator_list*, *type_params*)

Uma definição de classe

- `name` é uma string bruta para o nome da classe
- `bases` é uma lista de nós para classes base especificadas explicitamente.
- `keywords` é uma lista de nós **`keyword`**, principalmente para 'metaclass'. Outras argumentos nomeados serão passadas para a metaclasse, conforme a **PEP 3115** [https://peps.python.org/pep-3115/].

- `body` é uma lista de nós que representam o código dentro da definição de classe.
- `decorator_list` é uma lista de nós, como em **`FunctionDef`**.
- `type_params` é uma lista de parâmetros de tipo.

```
>>> print(ast.dump(ast.parse("""\
... @decorator1
... @decorator2
... class Foo(base1, base2, metaclass=meta):
...     pass
... """), indent=4))
Module(
    body=[
        ClassDef(
            name='Foo',
            bases=[
                Name(id='base1'),
                Name(id='base2')],
            keywords=[
                keyword(
                    arg='metaclass',
                    value=Name(id='meta'))],
            body=[
                Pass()],
            decorator_list=[
                Name(id='decorator1'),
                Name(id='decorator2')])])
```

Alterado na versão 3.12: Adicionado `type_params`.

Async e await

class ast.AsyncFunctionDef(*name*, *args*, *body*, *decorator_list*, *returns*, *type_comment*, *type_params*)

Uma definição de função `async def`. Possui os mesmos campos que **`FunctionDef`**.

Alterado na versão 3.12: Adicionado `type_params`.

class ast.Await(*value*)

Uma expressão `await`. `value` é o que ela espera. Válido apenas no corpo de um **`AsyncFunctionDef`**.

```
>>> print(ast.dump(ast.parse("""\
... async def f():
...     await other_func()
... """), indent=4))
Module(
    body=[
        AsyncFunctionDef(
            name='f',
            args=arguments(),
            body=[
                Expr(
                    value=Await(
                        value=Call(
                            func=Name(id='other_func'))))])])
```

class ast.AsyncFor(*target*, *iter*, *body*, *orelse*, *type_comment*)

class ast.AsyncWith(*items*, *body*, *type_comment*)

Laços `async for` e gerenciadores de contexto `async with`. Eles têm os mesmos campos que **`For`** e **`With`**, respectivamente. Válido apenas no corpo de **`AsyncFunctionDef`**.

Nota

Quando uma string é analisada por **`ast.parse()`**, os nós operadores (subclasses de **`ast.operator`**, **`ast.unaryop`**, **`ast.cmpop`**, **`ast.boolop`** e **`ast.expr_context`**) na árvore retornada serão singletons. As alterações em um serão refletidas em todas as outras ocorrências do mesmo valor (por exemplo, **`ast.Add`**).

Auxiliares de `ast`

Além das classes de nós, o módulo **`ast`** define essas funções e classes utilitárias para percorrer árvores de sintaxe abstrata:

ast.parse(*source*, *filename='<unknown>'*, *mode='exec'*, ***, *type_comments=False*, *feature_version=None*, *optimize=-1*, *module=None*)

Analisa a fonte como um nó AST. Equivalente a `compile(source, filename, mode, flags=FLAGS_VALUE, optimize=optimize, module=module)`, onde `FLAGS_VALUE` é `ast.PyCF_ONLY_AST` se `optimize <= 0` e `ast.PyCF_OPTIMIZED_AST` caso contrário.

Se `type_comments=True` é fornecido, o analisador é modificado para verificar e retornar comentários do tipo, conforme especificado por **PEP 484** [https://peps.python.org/pep-0484/] e **PEP 526** [https://peps.python.org/pep-0526/]. Isso é equivalente a adicionar **`ast.PyCF_TYPE_COMMENTS`** aos sinalizadores passados para **`compile()`**. Isso relatará erros de sintaxe para comentários do tipo extraviado. Sem esse sinalizador, os comentários do tipo serão ignorados e o campo `type_comment` nos nós AST selecionados sempre será `None`. Além disso, os locais dos comentários `# type: ignore` serão retornados como o atributo `type_ignores` de **`Module`** (caso contrário, é sempre uma lista vazia).

Além disso, se `mode` for `'func_type'`, a sintaxe de entrada é modificada para corresponder a “comentários de tipo de assinatura” de **PEP 484** [https://peps.python.org/pep-0484/], por exemplo, `(str, int) -> List[str]`.

Definir `feature_version` como uma tupla `(major, minor)` resultará em uma tentativa de “melhor esforço” de análise usando a gramática daquela versão do Python. Por exemplo, definir `feature_version=(3, 9)` tentará proibir a análise de instruções **`match`**. Atualmente `major` deve ser igual a `3`. A versão mais baixa suportada é `(3, 7)` (e isso pode aumentar em versões futuras do Python); o mais alto é `sys.version_info[0:2]`. A tentativa de “melhor esforço” significa que não há garantia de que a análise (ou sucesso da análise) seja a mesma que quando executada na versão Python correspondente a `feature_version`.

Se a fonte contém um caractere nulo (`\0`), **`ValueError`** é levantada.

Aviso

Observe que a análise bem-sucedida do código-fonte em um objeto AST não garante que o código-fonte fornecido seja um código Python válido que pode ser executado, pois a etapa de compilação pode levantar mais exceções `SyntaxError`. Por exemplo, a fonte `return 42` gera um nó AST válido para uma instrução return, mas não pode ser compilado sozinho (precisa estar dentro de um nó de função).

Em particular, `ast.parse()` não fará nenhuma verificação de escopo, o que a etapa de compilação faz.

Aviso

É possível travar o interpretador Python com uma string suficientemente grande/complexa devido às limitações de profundidade da pilha no compilador de AST do Python.

Alterado na versão 3.8: Adicionado `type_comments`, `mode='func_type'` e `feature_version`.

Alterado na versão 3.13: A versão mínima suportada para `feature_version` agora é `(3, 7)`. O argumento `optimize` foi adicionado.

Adicionado na versão 3.15: Adicionado o parâmetro *module*.

ast.unparse(*ast_obj*)

Desfaz análise de um objeto `ast.AST` e gera uma string com código que produziria um objeto `ast.AST` equivalente se analisado novamente com `ast.parse()`.

Aviso

A string de código produzida não será necessariamente igual ao código original que gerou o objeto `ast.AST` (sem quaisquer otimizações do compilador, como tuplas/frozensets constantes).

Aviso

Tentar desfazer análise de uma expressão altamente complexa resultaria em `RecursionError`.

Adicionado na versão 3.9.

ast.literal_eval(*node_or_string*)

Avalia um nó de expressão ou uma string contendo apenas um literal Python ou uma exibição de contêiner. A string ou nó fornecido pode consistir apenas nas seguintes estruturas literais Python: strings, bytes, números, tuplas, listas, dicionários, conjuntos, booleanos, `None` e `Ellipsis`.

Isso pode ser usado para avaliar strings contendo valores Python sem a necessidade de analisar os valores por conta própria. Não é capaz de avaliar expressões arbitrariamente complexas, por exemplo, envolvendo operadores ou indexação.

Esta função foi documentada como “segura” no passado sem definir o que isso significava. Isso foi enganoso. Isso foi projetado especificamente para não executar código Python, ao contrário do `eval()` mais geral. Não há espaço de nomes, pesquisas de nome ou capacidade de chamada. Mas não está livre de ataques: uma entrada relativamente pequena pode levar ao esgotamento da memória ou ao esgotamento da pilha C, travando o processo. Também existe a possibilidade de negação de serviço por consumo excessivo de CPU em algumas entradas. Portanto, não é recomendado chamá-la em dados não confiáveis.

Aviso

É possível travar o interpretador Python devido às limitações de profundidade da pilha no compilador AST do Python.

Pode levantar `ValueError`, `TypeError`, `SyntaxError`, `MemoryError` e `RecursionError` dependendo da entrada malformada.

Alterado na versão 3.2: Agora permite bytes e literais de conjuntos.

Alterado na versão 3.9: Agora oferece suporte à criação de conjuntos vazios com `'set()'`.

Alterado na versão 3.10: Para entradas de string, os espaços iniciais e tabulações agora são removidos.

ast.get_docstring(*node*, *clean=True*)

Retorna a docstring do *node* dado (que deve ser um nó **`FunctionDef`**, **`AsyncFunctionDef`**, **`ClassDef`** ou **`Module`**) ou `None` se não tiver uma docstring. Se *clean* for verdadeiro, limpa o recuo da docstring com **`inspect.cleandoc()`**.

Alterado na versão 3.5: Não há suporte a **`AsyncFunctionDef`**.

ast.get_source_segment(*source*, *node*, ***, *padded=False*)

Obtém o segmento de código-fonte do *source* que gerou o *node*. Se alguma informação de localização (**`lineno`**, **`end_lineno`**, **`col_offset`** ou **`end_col_offset`**) está faltando, retorna `None`.

Se *padded* for `True`, a primeira linha de uma instrução multilinha será preenchida com espaços para corresponder à sua posição original.

Adicionado na versão 3.8.

ast.fix_missing_locations(*node*)

Quando você compila uma árvore de nós com **`compile()`**, o compilador espera atributos **`lineno`** e **`col_offset`** para cada nó que os suporta. Isso é tedioso para preencher nós gerados, portanto, esse auxiliar adiciona esses atributos recursivamente, onde ainda não estão definidos, definindo-os para os valores do nó pai. Ele funciona recursivamente a partir do *node*.

ast.increment_lineno(*node*, *n=1*)

Incrementa o número da linhas e o número da linha final de cada nó na árvore começando em *node* em *n*. Isso é útil para “mover código” para

um local diferente em um arquivo.

ast.copy_location(*new_node*, *old_node*)

Copia o local de origem (**`lineno`**, **`col_offset`**, **`end_lineno`** e **`end_col_offset`**) de *old_node* para *new_node* se possível e, então, retorna *new_node*.

ast.iter_fields(*node*)

Produz uma tupla de `(fieldname, value)` para cada campo em `node._fields` que esteja presente em *node*.

ast.iter_child_nodes(*node*)

Produz todos os nós filhos diretos de *node*, ou seja, todos os campos que são nós e todos os itens de campos que são listas de nós.

ast.walk(*node*)

Produz recursivamente todos os nós descendentes na árvore começando em *node* (incluindo o próprio *node*), em nenhuma ordem especificada. Isso é útil se você quiser apenas modificar nós no lugar e não se importar com o contexto.

class ast.NodeVisitor

Uma classe base de visitante de nó que percorre a árvore de sintaxe abstrata e chama uma função de visitante para cada nó encontrado. Esta função pode retornar um valor que é encaminhado pelo método **`visit()`**.

Esta classe deve ser uma subclasse, com a subclasse adicionando métodos visitantes.

visit(*node*)

Visita um nó. A implementação padrão chama o método chamado `self.visit_`*`nomedaclasse`* sendo *nomedaclasse* o nome da classe do nó, ou **`generic_visit()`** se aquele método não existir.

generic_visit(*node*)

Este visitante chama `visit()` em todos os filhos do nó.

Observe que nós filhos de nós que possuem um método de visitante personalizado não serão visitados, a menos que o visitante chame `generic_visit()` ou os visite por conta própria.

visit_Constant(*node*)

Manipula todos os nós constantes.

Não use o `NodeVisitor` se você quiser aplicar mudanças nos nós durante a travessia. Para isso existe um visitante especial (`NodeTransformer`) que permite modificações.

Descontinuado desde a versão 3.8, removido na versão 3.14: Os métodos **`visit_Num()`**, **`visit_Str()`**, **`visit_Bytes()`**, **`visit_NameConstant()`** e **`visit_Ellipsis()`** não serão chamados nas versões 3.14 e mais novas do Python. Adicione um método `visit_Constant()` para lidar com nós de constantes.

class ast.NodeTransformer

A subclasse `NodeVisitor` que percorre a árvore de sintaxe abstrata e permite a modificação de nós.

O `NodeTransformer` percorrerá a AST e usará o valor de retorno dos métodos do visitante para substituir ou remover o nó antigo. Se o valor de retorno do método visitante for `None`, o nó será removido de seu local, caso contrário, ele será substituído pelo valor de retorno. O valor de retorno pode ser o nó original, caso em que não há substituição.

Aqui está um exemplo de transformador que rescreve todas as ocorrências de procuras por nome (`foo`) para `data['foo']`:

```
class RewriteName(NodeTransformer):

    def visit_Name(self, node):
        return Subscript(
            value=Name(id='data'),
            slice=Constant(value=node.id),
```

```
            ctx=node.ctx
        )
```

Tenha em mente que, se o nó em que você está operando tiver nós filhos, você deve transformar os nós filhos por conta própria ou chamar o método `generic_visit()` para o nó primeiro.

Para nós que faziam parte de uma coleção de instruções (que se aplica a todos os nós de instrução), o visitante também pode retornar uma lista de nós em vez de apenas um único nó.

Se `NodeTransformer` introduz novos nós (que não faziam parte da árvore original) sem fornecer informações de localização (como `lineno`), `fix_missing_locations()` deve ser chamado com o novo subárvore para recalcular as informações de localização:

```
tree = ast.parse('foo', mode='eval')
new_tree = fix_missing_locations(RewriteName().visit(tree))
```

Normalmente você usa o transformador assim:

```
node = YourTransformer().visit(node)
```

ast.dump(*node*, *annotate_fields=True*, *include_attributes=False*, *, *color=False*, *indent=None*, *show_empty=False*)

Retorne um despejo formatado da árvore em *node*. Isso é útil principalmente para fins de depuração. Se *annotate_fields* for verdadeiro (por padrão), a sequência retornada mostrará os nomes e os valores para os campos. Se *annotate_fields* for falso, a sequência de resultados será mais compacta ao omitir nomes de campos não ambíguos. Atributos como números de linha e deslocamentos de coluna não são despejados por padrão. Se isso for desejado, *include_attributes* pode ser definido como verdadeiro.

Se *color* for `True`, a string retornada terá destaque de sintaxe usando sequências de escape ANSI. Se for `False` (o padrão), a saída colorida estará sempre desativada.

Se *indent* for um inteiro não negativo ou uma string, então a árvore terá uma saída formatada com este nível de indentação. Um nível de indentação 0, negativo ou `""` apenas colocará novas linhas. `None` (o padrão) seleciona a representação de uma única linha. Usando um inteiro positivo a indentação terá alguns espaços por nível. Se *indent* for uma string (como `"\t"`), essa string será usada para indentar cada nível.

Se *show_empty* for falso (o padrão), listas vazias opcionais e valores opcionais de `Load()` serão omitidos da saída. Valores opcionais `None` são sempre omitidos.

```
>>> tree = ast.parse('print(None)', '?', 'eval')
>>> print(ast.dump(tree, indent=4))
Expression(
    body=Call(
        func=Name(id='print'),
        args=[
            Constant(value=None)]))
>>> print(ast.dump(tree, indent=4, show_empty=True))
Expression(
    body=Call(
        func=Name(id='print', ctx=Load()),
        args=[
            Constant(value=None)],
        keywords=[]))
```

Alterado na versão 3.9: Adicionada a opção *indent*.

Alterado na versão 3.13: Adicionada a opção *show_empty*.

Alterado na versão 3.15: Omite valores opcionais de `Load()` por padrão.

Alterado na versão 3.15: Adicionado o parâmetro *color*.

ast.compare(*a*, *b*, */*, ***, *compare_attributes=False*)

Compara recursivamente duas ASTs.

compare_attributes afeta se os atributos da AST são considerados na comparação. Se *compare_attributes* for `False` (padrão), os atributos

serão ignorados. Caso contrário, todos devem ser iguais. Esta opção é útil para verificar se os ASTs são estruturalmente iguais, mas diferem em espaços em branco ou em detalhes semelhantes. Os atributos incluem números de linha e deslocamentos de coluna.

Adicionado na versão 3.14.

Sinalizadores do compilador

Os seguintes sinalizadores podem ser passados para `compile()` para alterar os efeitos na compilação de um programa:

ast.PyCF_ALLOW_TOP_LEVEL_AWAIT

Habilita suporte para `await`, `async for`, `async with` e compreensões assíncronas de nível superior.

Adicionado na versão 3.8.

ast.PyCF_ONLY_AST

Gera e retorna uma árvore de sintaxe abstrata em vez de retornar um objeto de código compilado.

ast.PyCF_OPTIMIZED_AST

O AST retornado é otimizado de acordo com o argumento *optimize* em `compile()` ou `ast.parse()`.

Adicionado na versão 3.13.

ast.PyCF_TYPE_COMMENTS

Habilita suporte para comentários do tipo **PEP 484** [https://peps.python.org/pep-0484/] e **PEP 526** [https://peps.python.org/pep-0526/] (`# type: <type>`, `# type: ignore <stuff>`).

Adicionado na versão 3.8.

Uso na linha de comando

Adicionado na versão 3.9.

Alterado na versão 3.15: A saída agora possui realce de sintaxe por padrão. Isso pode ser controlado usando variáveis de ambiente.

O módulo **`ast`** pode ser executado como um script na linha de comando. É tão simples quanto:

```
python -m ast [-m <mode>] [-a] [infile]
```

As seguintes opções são aceitas:

-h, --help
: Mostra a mensagem de ajuda e sai.

-m <mode>
--mode <mode>
: Especifica que tipo de código deve ser compilado, como o argumento *mode* em **`parse()`**.

--no-type-comments
: Não analisa comentários de tipo.

-a, --include-attributes
: Inclui atributos como números de linha e deslocamentos de colunas.

-i <indent>
--indent <indent>
: indentação de nós em AST (número de espaços).

--feature-version <version>
: Versão do Python no formato 3.x (por exemplo, 3.10). O padrão é a versão atual do interpretador.

  Adicionado na versão 3.14.

-O <level>
--optimize <level>

Nível de otimização do analisador. O padrão é sem otimização.

Adicionado na versão 3.14.

--show-empty

Exibe listas e campos vazios com o valor `None`. O padrão é não exibir objetos vazios.

Adicionado na versão 3.14.

Se `infile` for especificado, seu conteúdo será analisado no AST e despejado no stdout. Caso contrário, o conteúdo será lido em stdin.

Ver também

Green Tree Snakes [https://greentreesnakes.readthedocs.io/], um recurso de documentação externo, possui bons detalhes sobre trabalhar com ASTs do Python.

ASTTokens [https://asttokens.readthedocs.io/en/latest/user-guide.html] anota ASTs do Python com as posições de tokens e texto no código-fonte que as gerou. Isso é útil para ferramentas que fazem transformações de código-fonte.

leoAst.py [https://leo-editor.github.io/leo-editor/appendices.html#leoast-py] unifica as visualizações baseadas em token e em árvore de análise de programas python, inserindo links duas vias entre tokens e nós de ast.

LibCST [https://libcst.readthedocs.io/] analisa o código como uma árvore de sintaxe concreta que se parece com uma árvore ast e mantém todos os detalhes de formatação. É útil para construir linters e aplicações de refatoração automatizada (codemod).

Parso [https://parso.readthedocs.io] é um analisador Python que oferece suporte a recuperação de erros e análise de ida e volta para diferentes versões do Python (em várias versões do Python). Parso também é capaz de listar vários erros de sintaxe em seu arquivo Python.

`symtable` — Acesso a tabela de símbolos do compilador

Código-fonte: Lib/symtable.py
[https://github.com/python/cpython/tree/3.15/Lib/symtable.py]

Symbol tables are generated by the compiler from AST just before bytecode is generated. The symbol table is responsible for calculating the scope of every identifier in the code. `symtable` provides an interface to examine these tables.

Generating Symbol Tables

symtable.symtable(*code*, *filename*, *compile_type*, *, *module=None*)

Return the toplevel `SymbolTable` for the Python source *code*. *filename* is the name of the file containing the code. *compile_type* is like the *mode* argument to `compile()`. The optional argument *module* specifies the module name. It is needed to unambiguous filter syntax warnings by module name.

Adicionado na versão 3.15: Adicionado o parâmetro *module*.

Examining Symbol Tables

class symtable.SymbolTableType

An enumeration indicating the type of a `SymbolTable` object.

MODULE = *"module"*

Used for the symbol table of a module.

FUNCTION = *"function"*

Used for the symbol table of a function.

CLASS = *"class"*
Used for the symbol table of a class.

The following members refer to different flavors of annotation scopes.

ANNOTATION = *"annotation"*
Used for annotations if `from __future__ import annotations` is active.

TYPE_ALIAS = *"type alias"*
Used for the symbol table of **`type`** constructions.

TYPE_PARAMETERS = *"type parameters"*
Used for the symbol table of generic functions or generic classes.

TYPE_VARIABLE = *"type variable"*
Used for the symbol table of the bound, the constraint tuple or the default value of a single type variable in the formal sense, i.e., a TypeVar, a TypeVarTuple or a ParamSpec object (the latter two do not support a bound or a constraint tuple).

Adicionado na versão 3.13.

class symtable.SymbolTable
A namespace table for a block. The constructor is not public.

get_type()
Return the type of the symbol table. Possible values are members of the **`SymbolTableType`** enumeration.

Alterado na versão 3.12: Added `'annotation'`, `'TypeVar bound'`, `'type alias'`, and `'type parameter'` as possible return values.

Alterado na versão 3.13: Return values are members of the **`SymbolTableType`** enumeration.

The exact values of the returned string may change in the future, and thus, it is recommended to use `SymbolTableType` members instead of hard-coded strings.

get_id()

Return the table's identifier.

get_name()

Return the table's name. This is the name of the class if the table is for a class, the name of the function if the table is for a function, or `'top'` if the table is global (`get_type()` returns `'module'`). For type parameter scopes (which are used for generic classes, functions, and type aliases), it is the name of the underlying class, function, or type alias. For type alias scopes, it is the name of the type alias. For `TypeVar` bound scopes, it is the name of the `TypeVar`.

get_lineno()

Return the number of the first line in the block this table represents.

is_optimized()

Return `True` if the locals in this table can be optimized.

is_nested()

Return `True` if the block is a nested class or function.

has_children()

Return `True` if the block has nested namespaces within it. These can be obtained with `get_children()`.

get_identifiers()

Return a view object containing the names of symbols in the table. See the documentation of view objects.

lookup(*name*)

Lookup *name* in the table and return a `Symbol` instance.

get_symbols()

Return a list of `Symbol` instances for names in the table.

get_children()

Return a list of the nested symbol tables.

class symtable.Function

A namespace for a function or method. This class inherits from `SymbolTable`.

get_parameters()

Return a tuple containing names of parameters to this function.

get_locals()

Return a tuple containing names of locals in this function.

get_globals()

Return a tuple containing names of globals in this function.

get_nonlocals()

Return a tuple containing names of explicitly declared nonlocals in this function.

get_frees()

Return a tuple containing names of free (closure) variables in this function.

get_cells()

Return a tuple containing names of cell variables in this table.

Adicionado na versão 3.15.

class symtable.Class

A namespace of a class. This class inherits from `SymbolTable`.

get_methods()

Return a tuple containing the names of method-like functions declared in the class.

Here, the term ‘method’ designates *any* function defined in the class body via **`def`** or **`async def`**.

Functions defined in a deeper scope (e.g., in an inner class) are not picked up by **`get_methods()`**.

Por exemplo:

```
>>> import symtable
>>> st = symtable.symtable('''
... def outer(): pass
...
... class A:
...    def f():
...        def w(): pass
...
...    def g(self): pass
...
...    @classmethod
...    async def h(cls): pass
...
...    global outer
...    def outer(self): pass
... ''', 'test', 'exec')
>>> class_A = st.get_children()[2]
>>> class_A.get_methods()
('f', 'g', 'h')
```

Although `A().f()` raises **`TypeError`** at runtime, `A.f` is still considered as a method-like function.

Descontinuado desde a versão 3.14, será removido na versão 3.16.

class symtable.Symbol

An entry in a **`SymbolTable`** corresponding to an identifier in the source. The constructor is not public.

get_name()

Return the symbol's name.

is_referenced()

Return `True` if the symbol is used in its block.

is_imported()

Return `True` if the symbol is created from an import statement.

is_parameter()

Return `True` if the symbol is a parameter.

is_type_parameter()

Return `True` if the symbol is a type parameter.

Adicionado na versão 3.14.

is_global()

Return `True` if the symbol is global.

is_nonlocal()

Return `True` if the symbol is nonlocal.

is_declared_global()

Return `True` if the symbol is declared global with a global statement.

is_local()

Return `True` if the symbol is local to its block.

is_annotated()

Return `True` if the symbol is annotated.

Adicionado na versão 3.6.

is_free()

Return `True` if the symbol is referenced in its block, but not assigned to.

is_cell()

Return `True` if the symbol is referenced but not assigned in a nested block.

Adicionado na versão 3.15.

is_free_class()

Return *True* if a class-scoped symbol is free from the perspective of a method.

Considere o exemplo a seguir:

```
def f():
    x = 1  # function-scoped
    class C:
        x = 2  # class-scoped
        def method(self):
            return x
```

In this example, the class-scoped symbol `x` is considered to be free from the perspective of `C.method`, thereby allowing the latter to return *1* at runtime and not *2*.

Adicionado na versão 3.14.

is_assigned()

Return `True` if the symbol is assigned to in its block.

is_comp_iter()

Return `True` if the symbol is a comprehension iteration variable.

Adicionado na versão 3.14.

is_comp_cell()

Return `True` if the symbol is a cell in an inlined comprehension.

Adicionado na versão 3.14.

is_namespace()

Return `True` if name binding introduces new namespace.

If the name is used as the target of a function or class statement, this will be true.

Por exemplo:

```
>>> table = symtable.symtable("def some_func(): pass", "s
>>> table.lookup("some_func").is_namespace()
True
```

Note that a single name can be bound to multiple objects. If the result is `True`, the name may also be bound to other objects, like an int or list, that does not introduce a new namespace.

get_namespaces()

Return a list of namespaces bound to this name.

get_namespace()

Return the namespace bound to this name. If more than one or no namespace is bound to this name, a **`ValueError`** is raised.

Uso na linha de comando

Adicionado na versão 3.13.

The **`symtable`** module can be executed as a script from the command line.

```
python -m symtable [infile...]
```

Symbol tables are generated for the specified Python source files and dumped to stdout. If no input file is specified, the content is read from stdin.

`token` — Constantes usadas com árvores de análises do Python

Código-fonte: Lib/token.py [https://github.com/python/cpython/tree/3.15/Lib/token.py]

Este módulo fornece constantes que representam os valores numéricos dos nós das folhas da árvore de análise (tokens terminais). Consulte o arquivo `Grammar/Tokens` na distribuição Python para obter as definições dos nomes no contexto da gramática da linguagem. Os valores numéricos específicos para os quais os nomes são mapeados podem mudar entre as versões do Python.

O módulo também fornece um mapeamento de códigos numéricos para nomes e algumas funções. As funções espelham definições nos arquivos de cabeçalho do Python C.

Observe que o valor de um token pode depender das opções do tokenizador. Por exemplo, um token `"+"` pode ser relatado como **`PLUS`** ou **`OP`**, ou um token `"match"` pode ser **`NAME`** ou **`SOFT_KEYWORD`**.

token.tok_name

Dicionário que mapeia os valores numéricos das constantes definidas neste módulo de volta para cadeias de nomes, permitindo que seja gerada uma representação mais legível de árvores de análise.

token.ISTERMINAL(*x*)

Retorna `True` para valores de tokens terminais.

token.ISNONTERMINAL(*x*)

Retorna `True` para valores de tokens não terminais.

token.ISEOF(*x*)

Retorna `True` se *x* for o marcador que indica o final da entrada.

Os constantes de tokens são:

token.NAME
: Valor de token que indica um literal ou palavra reservada

token.NUMBER
: Valor de token que indica um literal numérico

token.STRING
: Valor de token que indica um literal de string ou byte, excluindo literais de strings formatadas. A string de token não é interpretada: ela inclui as aspas ao redor e o prefixo (se fornecido); contrabarras são incluídas literalmente, sem processar sequências de escape.

token.OP
: Um valor de token genérico que indica um operador ou delimitador.

  Este valor é relatado apenas pelo módulo **`tokenize`**. Internamente, o tokenizador usa tipos exatos de tokens em vez disso.

token.COMMENT
: Valor de token usado para indicar um comentário. O analisador sintático ignora tokens **`COMMENT`**.

token.NEWLINE
: Valor de token que indica o fim de uma linha lógica.

token.NL
: Valor de token usado para indicar uma nova linha não terminante. Os tokens **`NL`** são gerados quando uma linha lógica de código é continuada em várias linhas físicas. O analisador sintático ignora os tokens **`NL`**.

token.INDENT
: Valor de token usado no início de uma linha lógica para indicar o início de um bloco indentado.

token.DEDENT

Valor de token usado no início de uma linha lógica para indicar o fim de um bloco indentado.

token.FSTRING_START

Valor de token usado para indicar o início de um literal de f-string.

A string de token inclui o prefixo e as aspas iniciais, mas nenhum conteúdo do literal.

token.FSTRING_MIDDLE

Valor de token usado para texto literal dentro de um literal de f-string, incluindo especificações de formato.

Campos de substituição (ou seja, as partes não literais de f-strings) usam os mesmos tokens que outras expressões e são delimitados pelos tokens `LBRACE`, `RBRACE`, `EXCLAMATION` e `COLON`.

token.FSTRING_END

Valor de token usado para indicar o fim de um literal de f-string.

A string do token contém as aspas de fechamento.

token.TSTRING_START

Valor de token usado para indicar o início de um literal de string template.

A string de token inclui o prefixo e as aspas iniciais, mas nenhum conteúdo do literal.

Adicionado na versão 3.14.

token.TSTRING_MIDDLE

Valor de token usado para texto literal dentro de um literal de string template, incluindo especificações de formato.

Campos de substituição (ou seja, as partes não literais de t-strings) usam os mesmos tokens que outras expressões e são delimitados pelos tokens

LBRACE, **RBRACE**, **EXCLAMATION** e **COLON**.

Adicionado na versão 3.14.

token.TSTRING_END

Valor de token usado para indicar o fim de um literal de string template.

A string do token contém as aspas de fechamento.

Adicionado na versão 3.14.

token.ENDMARKER

Valor de token que indica o fim da entrada. Usado em regras de gramática de nível superior.

token.ENCODING

Valor de token que indica a codificação usada para decodificar os bytes de origem em texto. O primeiro token retornado por **tokenize.tokenize()** sempre será um token `ENCODING`.

Este tipo de token não é usado pelo tokenizador do C, mas é necessário para o módulo **tokenize**.

Os seguintes tipos de token não são produzidos pelo módulo **tokenize** e são definidos para usos especiais no tokenizador ou analisador:

token.TYPE_IGNORE

Valor de token indicando que um comentário `type: ignore` foi reconhecido. Tais tokens são produzidos em vez de tokens **COMMENT** regulares somente com o sinalizador **PyCF_TYPE_COMMENTS**.

token.TYPE_COMMENT

Valor de token indicando que um comentário de tipo foi reconhecido. Tais tokens são produzidos em vez de tokens **COMMENT** regulares somente com o sinalizador **PyCF_TYPE_COMMENTS**.

token.SOFT_KEYWORD

Valor de token indicando uma palavra reservada contextual.

O tokenizador nunca produz esse valor. Para verificar uma palavra reservada contextual, passe uma string do token **NAME** para **keyword.issoftkeyword()**.

token.ERRORTOKEN

Valor de token usado para indicar uma entrada errada.

O módulo **tokenize** geralmente indica erros levantando exceções em vez de emitir este token. Ele também pode emitir tokens como **OP** ou **NAME** com strings que são rejeitadas posteriormente pelo parser.

Os tokens restantes representam operadores e delimitadores específicos. (O módulo **tokenize** os relata como **OP**; consulte `exact_type` na documentação **tokenize** para obter detalhes.)

Token	Valor
token.LPAR	`"("`
token.RPAR	`")"`
token.LSQB	`"["`
token.RSQB	`"]"`
token.COLON	`":"`
token.COMMA	`","`
token.SEMI	`";"`

Token	Valor
token.PLUS	`"+"`
token.MINUS	`"-"`
token.STAR	`"*"`
token.SLASH	`"/"`
token.VBAR	`"|"`
token.AMPER	`"&"`
token.LESS	`"<"`
token.GREATER	`">"`
token.EQUAL	`"="`
token.DOT	`"."`
token.PERCENT	`"%"`
token.LBRACE	`"{"`

Token	Valor
token.RBRACE	"}"
token.EQEQUAL	"=="
token.NOTEQUAL	"!="
token.LESSEQUAL	"<="
token.GREATEREQUAL	">="
token.TILDE	"~"
token.CIRCUMFLEX	"^"
token.LEFTSHIFT	"<<"
token.RIGHTSHIFT	">>"
token.DOUBLESTAR	"**"
token.PLUSEQUAL	"+="
token.MINEQUAL	"-="

Token	Valor
token.STAREQUAL	`"*="`
token.SLASHEQUAL	`"/="`
token.PERCENTEQUAL	`"%="`
token.AMPEREQUAL	`"&="`
token.VBAREQUAL	`"|="`
token.CIRCUMFLEXEQUAL	`"^="`
token.LEFTSHIFTEQUAL	`"<<="`
token.RIGHTSHIFTEQUAL	`">>="`
token.DOUBLESTAREQUAL	`"**="`
token.DOUBLESLASH	`"//"`
token.DOUBLESLASHEQUAL	`"//="`
token.AT	`"@"`

Token	Valor
token.ATEQUAL	`"@="`
token.RARROW	`"->"`
token.ELLIPSIS	`"..."`
token.COLONEQUAL	`":="`
token.EXCLAMATION	`"!"`

As seguintes constantes não-tokens são fornecidas:

token.N_TOKENS
: O número de tipos de token definidos neste módulo.

token.EXACT_TOKEN_TYPES
: Um dicionário que mapeia a representação de string de um token para seu código numérico.

Adicionado na versão 3.8.

Alterado na versão 3.5: Adicionados os tokens **`AWAIT`** e **`ASYNC`**.

Alterado na versão 3.7: Adicionados os tokens **`COMMENT`**, **`NL`** e **`ENCODING`**.

Alterado na versão 3.7: Removido os tokens **`AWAIT`** e **`ASYNC`**. “async” e “await” são agora tokenizados como tokens **`NAME`**.

Alterado na versão 3.8: Adicionados **`TYPE_COMMENT`**, **`TYPE_IGNORE`**, **`COLONEQUAL`**. Adicionados os tokens **`AWAIT`** e **`ASYNC`** (eles são necessários

para dar suporte à análise de versões mais antigas do Python para **`ast.parse()`** com `feature_version` definido como 6 ou inferior).

Alterado na versão 3.12: Adicionado **`EXCLAMATION`**.

Alterado na versão 3.13: Removidos novamente os tokens **`AWAIT`** e **`ASYNC`**.

`keyword` — Testando palavras reservadas do Python

Código-fonte: Lib/keyword.py
[https://github.com/python/cpython/tree/3.15/Lib/keyword.py]

Este módulo permite a um programa Python determinar se uma string é uma palavra reservada ou palavra reservada contextual.

keyword.iskeyword(*s*)

Retorna `True` se *s* for uma palavra reservada do Python.

keyword.kwlist

Sequência contendo todas as palavras reservadas definidas para o interpretador. Se alguma palavra reservada estiver definida para apenas estar ativa quando instruções `__future__` específicas tiverem efeito, estas serão incluídas também.

keyword.issoftkeyword(*s*)

Retorna `True` se *s* for uma palavra reservada contextual do Python.

Adicionado na versão 3.9.

keyword.softkwlist

Sequência contendo todas as palavras reservadas contextuais definidas para o interpretador. Se alguma palavra reservada contextual estiver definida para apenas estar ativo quando instruções `__future__` tiverem efeito, estas serão incluídas também.

Adicionado na versão 3.9.

`tokenize` — Tokenizador para código-fonte Python

Código-fonte: Lib/tokenize.py
[https://github.com/python/cpython/tree/3.15/Lib/tokenize.py]

The **`tokenize`** module provides a lexical scanner for Python source code, implemented in Python. The scanner in this module returns comments as tokens as well, making it useful for implementing “pretty-printers”, including colorizers for on-screen displays.

Para simplificar o tratamento do fluxo de tokens, todos os tokens operadores e delimitadores e **`Ellipsis`** são retornados usando o tipo de token genérico **`OP`**. O tipo exato pode ser determinado verificando a propriedade `exact_type` na tupla nomeada retornada de **`tokenize.tokenize()`**.

> **Aviso**
>
> Observe que as funções neste módulo são projetadas apenas para analisar código Python sintaticamente válido (código que não levanta exceção quando analisado usando **`ast.parse()`**). O comportamento das funções neste módulo é **indefinido** ao fornecer código Python inválido e pode mudar a qualquer momento.

Tokenizing input

O ponto de entrada principal é um gerador:

tokenize.tokenize(*readline*)

O gerador **tokenize()** requer um argumento, *readline*, que deve ser um objeto chamável que fornece a mesma interface que o método **io.IOBase.readline()** de objetos arquivo. Cada chamada para a função deve retornar uma linha de entrada como bytes.

O gerador produz tuplas de 5 elementos com estes membros: o tipo de token; a string de token; uma tupla de 2 elementos `(srow, scol)` de ints especificando a linha e a coluna onde o token começa na fonte; uma tupla de 2 elementos `(erow, ecol)` de ints especificando a linha e a coluna onde o token termina na fonte; e a linha na qual o token foi encontrado. A linha passada (o último item da tupla) é a linha *física*. A tupla 5 elementos é retornada como uma tupla nomeada com os nomes dos campos: `tipo string início fim linha`.

A tupla nomeada retornada tem uma propriedade adicional chamada `exact_type` que contém o tipo exato do operador para tokens **OP**. Para todos os outros tipos de token, `exact_type` é igual ao campo `type` da tupla nomeada.

Alterado na versão 3.1: Adiciona suporte para tuplas nomeadas.

Alterado na versão 3.3: Adicionado suporte para `exact_type`.

tokenize() determina a codificação de origem do arquivo procurando por um BOM UTF-8 ou cookie de codificação, de acordo com **PEP 263** [https://peps.python.org/pep-0263/].

tokenize.generate_tokens(*readline*)

Tokeniza uma fonte lendo strings unicode em vez de bytes.

Assim como **tokenize()**, o argumento *readline* é um chamável que retorna uma única linha de entrada. No entanto, **generate_tokens()** espera que *readline* retorne um objeto str em vez de bytes.

O resultado é um iterador produzindo tuplas nomeadas, exatamente como **tokenize()**. Ele não produz um token **ENCODING**.

All constants from the **token** module are also exported from **tokenize**.

Outra função é fornecida para reverter o processo de tokenização. Isso é útil para criar ferramentas que tokenizam um script, modificam o fluxo de tokens e escrevem de volta o script modificado.

tokenize.untokenize(*iterable*)

Converte tokens de volta para o código-fonte Python. O *iterable* deve retornar sequências com pelo menos dois elementos, o tipo de token e a string do token. Quaisquer elementos de sequência adicionais são ignorados.

O resultado é garantido para tokenizar de volta para corresponder à entrada, de modo que a conversão seja sem perdas e as viagens de ida e volta sejam garantidas. A garantia se aplica somente ao tipo de token e à sequência de tokens, pois o espaçamento entre os tokens (posições de coluna) pode mudar.

Retorna bytes, codificados usando o token `ENCODING`, que é a primeira sequência de tokens gerada por `tokenize()`. Se não houver nenhum token de codificação na entrada, ele retorna um str em vez disso.

`tokenize()` precisa detectar a codificação dos arquivos fonte que ele tokeniza. A função que ele usa para fazer isso está disponível:

tokenize.detect_encoding(*readline*)

A função `detect_encoding()` é usada para detectar a codificação que deve ser usada para decodificar um arquivo de origem Python. Ela requer um argumento, readline, da mesma forma que o gerador `tokenize()`.

Ele chamará readline no máximo duas vezes e retornará a codificação usada (como uma string) e uma lista de todas as linhas (não decodificadas de bytes) que ele leu.

Ele detecta a codificação a partir da presença de um BOM UTF-8 ou um cookie de codificação conforme especificado em **PEP 263** [https://peps.python.org/pep-0263/]. Se um BOM e um cookie estiverem presentes, mas discordarem, uma exceção `SyntaxError` será levantada.

Observe que se o BOM for encontrado, `'utf-8-sig'` será retornado como uma codificação.

Se nenhuma codificação for especificada, o padrão `'utf-8'` será retornado.

Use **`open()`** para abrir arquivos de código-fonte Python: ele usa **`detect_encoding()`** para detectar a codificação do arquivo.

tokenize.open(*filename*)

Abre um arquivo no modo somente leitura usando a codificação detectada por **`detect_encoding()`**.

Adicionado na versão 3.2.

exception tokenize.TokenError

Levantada quando uma docstring ou expressão que pode ser dividida em várias linhas não é concluída em nenhum lugar do arquivo, por exemplo:

```
"""Início da
docstring
```

ou:

```
[1,
 2,
 3
```

Uso na linha de comando

Adicionado na versão 3.3.

The **`tokenize`** module can be executed as a script from the command line. It is as simple as:

```
python -m tokenize [-e] [nome-do-arquivo.py]
```

As seguintes opções são aceitas:

-h, --help
    mostra esta mensagem de ajuda e sai

-e, --exact
    exibe nomes de tokens usando o tipo exato

Se `nome-de-arquivo.py` for especificado, seu conteúdo será tokenizado para stdout. Caso contrário, a tokenização será realizada em stdin.

Adicionado na versão 3.15: A saída é colorida por padrão e ela pode ser controlada usando variáveis de ambiente.

Exemplos

Exemplo de um reescritor de script que transforma literais float em objetos Decimal:

```
from tokenize import tokenize, untokenize, NUMBER, STRING, NAME,
from io import BytesIO

def decistmt(s):
    """Substitui Decimals por floats em uma string de instruções

    >>> from decimal import Decimal
    >>> s = 'print(+21.3e-5*-.1234/81.7)'
    >>> decistmt(s)
    "print (+Decimal ('21.3e-5')*-Decimal ('.1234')/Decimal ('81

    O formato do expoente é herdado da biblioteca da plataforma
    conhecidos são "e-007" (Windows) e "e-07" (não Windows). Com
    mostrando apenas 12 dígitos, e o 13º não está próximo de 5,
    da saída deve ser independente da plataforma.

    >>> exec(s)  #doctest: +ELLIPSIS
    -3.21716034272e-0...7

    A saída dos cálculos com Decimal deve ser idêntica em todas
    plataformas.

    >>> exec(decistmt(s))
```

```
    -3.217160342717258261933904529E-7
    """
    result = []
    g = tokenize(BytesIO(s.encode('utf-8')).readline)  # tokeniz
    for toknum, tokval, _, _, _ in g:
        if toknum == NUMBER and '.' in tokval:  # replace NUMBER
            result.extend([
                (NAME, 'Decimal'),
                (OP, '('),
                (STRING, repr(tokval)),
                (OP, ')')
            ])
        else:
            result.append((toknum, tokval))
    return untokenize(result).decode('utf-8')
```

Exemplo de tokenização a partir da linha de comando. O script:

```
def say_hello():
    print("Hello, World!")

say_hello()
```

will be tokenized to the following output where the first column is the range of the line/column coordinates where the token is found, the second column is the name of the token, and the final column is the value of the token (if any):

```
$ python -m tokenize hello.py
0,0-0,0:            ENCODING       'utf-8'
1,0-1,3:            NAME           'def'
1,4-1,13:           NAME           'say_hello'
1,13-1,14:          OP             '('
1,14-1,15:          OP             ')'
1,15-1,16:          OP             ':'
1,16-1,17:          NEWLINE        '\n'
2,0-2,4:            INDENT         '    '
2,4-2,9:            NAME           'print'
2,9-2,10:           OP             '('
2,10-2,25:          STRING         '"Hello, World!"'
2,25-2,26:          OP             ')'
```

```
2,26-2,27:          NEWLINE        '\n'
3,0-3,1:            NL             '\n'
4,0-4,0:            DEDENT         ''
4,0-4,9:            NAME           'say_hello'
4,9-4,10:           OP             '('
4,10-4,11:          OP             ')'
4,11-4,12:          NEWLINE        '\n'
5,0-5,0:            ENDMARKER      ''
```

Os nomes exatos dos tipos de token podem ser exibidos usando a opção `-e`:

```
$ python -m tokenize -e hello.py
0,0-0,0:            ENCODING       'utf-8'
1,0-1,3:            NAME           'def'
1,4-1,13:           NAME           'say_hello'
1,13-1,14:          LPAR           '('
1,14-1,15:          RPAR           ')'
1,15-1,16:          COLON          ':'
1,16-1,17:          NEWLINE        '\n'
2,0-2,4:            INDENT         '    '
2,4-2,9:            NAME           'print'
2,9-2,10:           LPAR           '('
2,10-2,25:          STRING         '"Hello, World!"'
2,25-2,26:          RPAR           ')'
2,26-2,27:          NEWLINE        '\n'
3,0-3,1:            NL             '\n'
4,0-4,0:            DEDENT         ''
4,0-4,9:            NAME           'say_hello'
4,9-4,10:           LPAR           '('
4,10-4,11:          RPAR           ')'
4,11-4,12:          NEWLINE        '\n'
5,0-5,0:            ENDMARKER      ''
```

Exemplo de tokenização de um arquivo programaticamente, lendo strings unicode em vez de bytes com `generate_tokens()`:

```
import tokenize

with tokenize.open('hello.py') as f:
    tokens = tokenize.generate_tokens(f.readline)
    for token in tokens:
        print(token)
```

Ou lendo bytes diretamente com `tokenize()`:

```
import tokenize

with open('hello.py', 'rb') as f:
    tokens = tokenize.tokenize(f.readline)
    for token in tokens:
        print(token)
```

`tabnanny` — Detecção de indentação ambígua

Código-fonte: Lib/tabnanny.py
[https://github.com/python/cpython/tree/3.15/Lib/tabnanny.py]

Por enquanto, este módulo deve ser chamado como um script. No entanto, é possível importá-lo para um IDE e usar a função `check()` descrita abaixo.

> **Nota**
>
> A API fornecida por este módulo provavelmente mudará em versões futuras; essas mudanças podem não ser compatíveis com versões anteriores.

tabnanny.check(*file_or_dir*)

Se *file_or_dir* for um diretório e não um link simbólico, desce recursivamente a árvore de diretórios nomeada por *file_or_dir*, verificando todos os arquivos `.py` ao longo do caminho. Se *file_or_dir* for um arquivo-fonte comum do Python, ele será verificado quanto a problemas relacionados ao espaço em branco. As mensagens de diagnóstico são gravadas na saída padrão usando a função `print()`.

tabnanny.verbose

Sinalizador indicando se as mensagens detalhadas devem ser impressas. Isso é incrementado pela opção `-v` se chamado como um script.

tabnanny.filename_only

Sinalizador indicando se os nomes dos arquivos devem ser impressos apenas com problemas relacionados a espaços em branco. Isso é definido como true pela opção `-q` se chamado como um script.

exception tabnanny.NannyNag

Levantada por `process_tokens()` se detectar um recuo ambíguo. Capturado e manipulado em `check()`.

tabnanny.process_tokens(*tokens*)

Esta função é usada por `check()` para processar os tokens gerados pelo módulo `tokenize`.

Ver também

Módulo `tokenize`

Scanner léxico para código-fonte Python.

`pyclbr` — Suporte a navegador de módulos do Python

Código-fonte: Lib/pyclbr.py [https://github.com/python/cpython/tree/3.15/Lib/pyclbr.py]

O módulo **`pyclbr`** fornece informações limitadas sobre as funções, classes e métodos definidos em um módulo codificado em Python. As informações são suficientes para implementar um navegador de módulos. As informações são extraídas do código-fonte do Python em vez de importar o módulo, portanto, este módulo é seguro para uso com código não confiável. Essa restrição torna impossível o uso deste módulo com módulos não implementados no Python, incluindo todos os módulos de extensão padrão e opcionais.

pyclbr.readmodule(*module*, *path=None*)

Retorna um dicionário que mapeia os nomes de classe no nível do módulo aos descritores de classe. Se possível, descritores para classes base importadas estão incluídos. O parâmetro *module* é uma string com o nome do módulo a ser lido; pode ser o nome de um módulo dentro de um pacote. Se fornecido, *path* é uma sequência de caminhos de diretório anexada a `sys.path`, que é usada para localizar o código-fonte do módulo.

Esta função é a interface original e é mantida apenas para compatibilidade reversa. Ela retorna uma versão filtrada da seguinte.

pyclbr.readmodule_ex(*module*, *path=None*)

Retorna uma árvore baseada em dicionário que contém uma função ou descritores de classe para cada função e classe definida no módulo com uma instrução `def` ou `class`. O dicionário retornado mapeia os nomes das funções e das classes no nível do módulo para seus descritores. Objetos aninhados são inseridos no dicionário filho de seus pais. Como

em readmodule, *module* nomeia o módulo a ser lido e *path* é anexado ao sys.path. Se o módulo que está sendo lido for um pacote, o dicionário retornado terá uma chave `'__path__'` cujo valor é uma lista que contém o caminho de pesquisa do pacote.

Adicionado na versão 3.7: Descritores para definições aninhadas. Eles são acessados através do novo atributo filho. Cada um tem um novo atributo pai.

Os descritores retornados por essas funções são instâncias das classes Function e Class. Não se espera que os usuários criem instâncias dessas classes.

Objetos Function

class pyclbr.Function

Instâncias da classe **`Function`** descrevem funções definidas por instruções def. Elas têm os seguintes atributos:

file

Nome do arquivo no qual a função está definida.

module

O nome do módulo que define a função descrita.

name

O nome da função.

lineno

O número da linha no arquivo em que a definição é iniciada.

parent

Para funções de nível superior, `None`. Para funções aninhadas, o pai.

Adicionado na versão 3.7.

children

Um **dicionário** que mapeia nomes para descritores para funções e classes aninhadas.

Adicionado na versão 3.7.

is_async

`True` para funções que são definidas com o prefixo **async**. Caso contrário, `False`.

Adicionado na versão 3.10.

Objetos Class

class pyclbr.Class

Instâncias da classe **Class** descrevem classes definidas por instruções class. Elas têm os mesmos atributos que **Functions** e mais dois.

file

Nome do arquivo no qual a classe está definida.

module

O nome do módulo que define a classe descrita.

name

O nome da classe.

lineno

O número da linha no arquivo em que a definição é iniciada.

parent

Para classes de nível superior, `None`. Para classes aninhadas, o pai.

Adicionado na versão 3.7.

children

Um dicionário que mapeia nomes para descritores para funções e classes aninhadas.

Adicionado na versão 3.7.

super

Uma lista de objetos **`Class`** que descreve as classes base imediatas da classe que está sendo descrita. Classes nomeadas como superclasses, mas que não podem ser descobertas por **`readmodule_ex()`** são listadas como uma string com o nome da classe em vez de como objetos de **`Class`**.

methods

Um **`dicionário`** que mapeia nomes de métodos para números de linha. Isso pode ser derivado do dicionário **`children`** mais novo, mas permanece para compatibilidade retroativa.

`py_compile` — Compila arquivos fonte do Python

Código-fonte: Lib/py_compile.py
[https://github.com/python/cpython/tree/3.15/Lib/py_compile.py]

O módulo **`py_compile`** fornece uma função para gerar um arquivo de bytecode a partir de um arquivo fonte, e outra função usada quando o arquivo fonte do módulo é invocado como um script.

Embora nem sempre seja necessária, essa função pode ser útil ao instalar módulos para uso compartilhado, especialmente se alguns usuários não tiverem permissão para gravar os arquivos de cache de bytecodes no diretório que contém o código-fonte.

exception py_compile.PyCompileError

Exceção levantada quando ocorre um erro ao tentar compilar o arquivo.

py_compile.compile(*file*, *cfile=None*, *dfile=None*, *doraise=False*, *optimize=-1*, *invalidation_mode=PycInvalidationMode.TIMESTAMP*, *quiet=0*)

Compila um arquivo fonte para bytecode e grava o arquivo de cache de bytecode. O código-fonte é carregado a partir do arquivo chamado *file*. O bytecode é gravado em *cfile*, cujo padrão é o caminho **PEP 3147** [https://peps.python.org/pep-3147/]/**PEP 488** [https://peps.python.org/pep-0488/], terminando em `.pyc`. Por exemplo, se *file* for `/foo/bar/baz.py`, o *cfile* será padronizado como `/foo/bar/__pycache__/baz.cpython-32.pyc` para o Python 3.2. Se *dfile* for especificado, ele será usado em vez de *file* como o nome do arquivo fonte do qual as linhas de origem são obtidas para exibição em tracebacks de exceção. Se *doraise* for verdadeiro, a **`PyCompileError`** será levantada quando um erro for encontrado durante a compilação de *file*. Se *doraise* for false (o padrão),

uma string de erros será gravada em `sys.stderr`, mas nenhuma exceção será levantada. Essa função retorna o caminho para o arquivo compilado em bytes, ou seja, qualquer valor *cfile* foi usado.

Os argumentos *doraise* e *quiet* determinam como os erros são tratados durante a compilação do arquivo. Se *quiet* for 0 ou 1 e *doraise* for false, o comportamento padrão será ativado: uma string de erros será gravada em `sys.stderr` e a função retornará `None` em vez de um caminho. Se *doraise* for verdadeiro, uma **`PyCompileError`** será levantada. No entanto, se *quiet* for 2, nenhuma mensagem será escrita e *doraise* não terá efeito.

Se o caminho que *cfile* se tornar (especificado ou computado explicitamente) for um link simbólico ou um arquivo não regular, **`FileExistsError`** será levantada. Isso serve como um aviso de que a importação transformará esses caminhos em arquivos regulares se for permitido gravar arquivos compilados em bytes nesses caminhos. Esse é um efeito colateral da importação usando a renomeação de arquivo para colocar o arquivo final compilado em bytecode para evitar problemas de gravação simultânea de arquivos.

optimize controla o nível de otimização e é passado para a função embutida **`compile()`**. O padrão de `-1` seleciona o nível de otimização do interpretador atual.

invalidation_mode deve ser um membro da enum **`PycInvalidationMode`** e controla como o cache do bytecode gerado é invalidado em tempo de execução. O padrão é **`PycInvalidationMode.CHECKED_HASH`** se a variável de ambiente **`SOURCE_DATE_EPOCH`** estiver configurada, caso contrário, o padrão é **`PycInvalidationMode.TIMESTAMP`**.

Alterado na versão 3.2: Alterado o valor padrão de *cfile* para ficar em conformidade com a **PEP 3147** [https://peps.python.org/pep-3147/]. O padrão anterior era *file* + `'c'` (`'o'` se a otimização estivesse ativada). Também foi adicionado o parâmetro *optimize*.

Alterado na versão 3.4: Alterado o código para usar **importlib** para a gravação do arquivo de cache do bytecode. Isso significa que a semântica de criação/gravação de arquivo agora corresponde ao que **importlib** faz, por exemplo, permissões, semântica de gravação e movimentação, etc. Também foi adicionada a ressalva de que **FileExistsError** é levantada se *cfile* for um link simbólico ou um arquivo não regular.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado conforme especificado em **PEP 552** [https://peps.python.org/pep-0552/]. Se a variável de ambiente **SOURCE_DATE_EPOCH** estiver configurada, *invalidation_mode* será forçado a **PycInvalidationMode.CHECKED_HASH**.

Alterado na versão 3.7.2: A variável de ambiente **SOURCE_DATE_EPOCH** não substitui mais o valor do argumento *invalidation_mode* e, em vez disso, determina seu valor padrão.

Alterado na versão 3.8: O parâmetro *quiet* foi adicionado.

class py_compile.PycInvalidationMode

Uma enumeração de métodos possíveis que o interpretador pode usar para determinar se um arquivo de bytecode está atualizado com um arquivo fonte. O arquivo `.pyc` indica o modo de invalidação desejado em seu cabeçalho. Veja Invalidação de bytecode em cache para obter mais informações sobre como o Python invalida arquivos `.pyc` em tempo de execução.

Adicionado na versão 3.7.

TIMESTAMP

O arquivo `.pyc` inclui o carimbo de data e hora e o tamanho do arquivo fonte, que o Python comparará com os metadados do arquivo fonte no tempo de execução para determinar se o arquivo `.pyc` precisa ser gerado novamente.

CHECKED_HASH

O arquivo `.pyc` inclui um hash do conteúdo do arquivo fonte, com o qual o Python comparará o fonte em tempo de execução para determinar se o arquivo `.pyc` precisa ser gerado novamente.

UNCHECKED_HASH

Como **`CHECKED_HASH`**, o arquivo `.pyc` inclui um hash do conteúdo do arquivo fonte. No entanto, em tempo de execução, o Python presumirá que o arquivo `.pyc` está atualizado e não validará o `.pyc` contra o arquivo fonte.

Essa opção é útil quando os `.pycs` são atualizados por algum sistema externo ao Python, como um sistema de compilação.

Interface de Linha de Comando

Este módulo pode ser invocado como um script para compilar vários arquivos fonte. Os arquivos nomeados em *filenames* são compilados e o bytecode resultante é armazenado em cache da maneira normal. Este programa não pesquisa uma estrutura de diretórios para localizar arquivos de origem; ele apenas compila arquivos nomeados explicitamente. O status de saída será diferente de zero se um dos arquivos não puder ser compilado.

<file> ... <fileN>
-

Argumentos posicionais são arquivos para compilar. Se `-` for o único parâmetro, a lista de arquivos será obtida da entrada padrão.

-q, --quiet

Suprime a saída de erros.

Alterado na versão 3.2: Adicionado suporte a `-`.

Alterado na versão 3.10: Adicionado suporte a **`-q`**.

Ver também

Módulo `compileall`

Utilitários para compilar todos os arquivos fontes Python em uma árvore de diretórios.

`compileall` — Compilar bibliotecas do Python para bytecode

Código-fonte: Lib/compileall.py
[https://github.com/python/cpython/tree/3.15/Lib/compileall.py]

Este módulo fornece algumas funções utilitárias para dar suporte à instalação de bibliotecas Python. Essas funções compilam arquivos fonte Python em uma árvore de diretórios. Este módulo pode ser usado para criar os arquivos de bytecodes em cache no momento da instalação da biblioteca, o que os torna disponíveis para uso mesmo por usuários que não têm permissão de gravação nos diretórios da biblioteca.

Disponibilidade: not WASI.

Este módulo não funciona ou não está disponível em WebAssembly. Veja Plataformas WebAssembly para mais informações.

Uso na linha de comando

Este módulo pode funcionar como um script (usando **python -m compileall**) para compilar fontes do Python.

directory ...
file ...

Argumentos posicionais são arquivos a serem compilados ou diretórios que contêm arquivos de origem, percorridos recursivamente. Se nenhum argumento for fornecido, comporta-se como se a linha de comando fosse `-l <diretórios do sys.path>`.

-l

Não atua recursivamente em subdiretórios, apenas compila arquivos de código-fonte diretamente contidos nos diretórios nomeados ou implícitos.

-f

Força a recompilação, mesmo que os carimbos de data e hora estejam atualizados.

-q

Não imprime a lista de arquivos compilados. Se passado uma vez, as mensagens de erro ainda serão impressas. Se passado duas vezes (`-qq`), toda a saída é suprimida.

-d destdir

Diretório anexado ao caminho de cada arquivo que está sendo compilado. Isso aparecerá nos tracebacks em tempo de compilação e também será compilado no arquivo de bytecode, onde será usado em tracebacks e outras mensagens nos casos em que o arquivo de origem não exista no momento em que o arquivo de bytecode for executado.

-s strip_prefix

Remove o prefixo especificado dos caminhos gravados nos arquivos `.pyc`. Caminhos são tornados relativos ao prefixo.

Esta opção pode ser usada com `-p`, mas não com `-d`.

-p prepend_prefix

Adiciona o prefixo fornecido aos caminhos registrados nos arquivos `.pyc`. Use `-p /` para tornar os caminhos absolutos.

Esta opção pode ser usada com `-s`, mas não com `-d`.

-x regex

A expressão regular `regex` é usada para pesquisar o caminho completo para cada arquivo considerado para compilação e, se a `regex` produzir uma correspondência, o arquivo será ignorado.

-i list

Lê o arquivo `list` e adicione cada linha que ele contém à lista de arquivos e diretórios a serem compilados. Se `list` for `-`, lê as linhas do `stdin`.

-b

Escreve os arquivos de bytecode em seus locais e nomes legados, que podem sobrescrever arquivos de bytecode criados por outra versão do Python. O padrão é gravar arquivos em seus locais e nomes do **PEP 3147** [https://peps.python.org/pep-3147/], o que permite que arquivos de bytecode de várias versões do Python coexistam.

-r

Controla o nível máximo de recursão para subdiretórios. Se isso for dado, a opção `-l` não será levada em consideração. **python -m compileall <diretório> -r 0** é equivalente a **python -m compileall <diretório> -l**.

-j N

Usa *N* workers para compilar os arquivos dentro do diretório especificado. Se `0` for usado, o resultado de **`os.process_cpu_count()`** será usado.

--invalidation-mode [timestamp|checked-hash|unchecked-hash]

Controla como os arquivos de bytecode gerados são invalidados no tempo de execução. O valor `timestamp` significa que os arquivos `.pyc` com o carimbo de data/hora do fonte e o tamanho incorporado serão gerados. Os valores `selected-hash` e `unchecked-hash` fazem com que os pycs baseados em hash sejam gerados. Arquivos pycs baseados em hash incorporam um hash do conteúdo do arquivo fonte em vez de um carimbo de data/hora. Veja Invalidação de bytecode em cache para obter mais informações sobre como o Python valida os arquivos de cache do bytecode em tempo de execução. O padrão é `timestamp` se a variável de ambiente **`SOURCE_DATE_EPOCH`** não estiver configurada e `selected-hash` se a variável de ambiente `SOURCE_DATE_EPOCH` estiver configurada.

-o level

Compila com o nível de otimização fornecido. Pode ser usado várias vezes para compilar para vários níveis por vez (por exemplo, `compileall -o 1 -o 2`).

-e dir

Ignora links simbólicos que apontam para fora do diretório especificado.

--hardlink-dupes

Se dois arquivos `.pyc` com nível de otimização diferente tiverem o mesmo conteúdo, usa links físicos para consolidar arquivos duplicados.

Alterado na versão 3.2: Adicionadas as opções `-i`, `-b` e `-h`.

Alterado na versão 3.5: Adicionadas as opções `-j`, `-r` e `-qq`. A opção `-q` foi alterada para um valor multinível. `-b` sempre produzirá um arquivo de bytecodes que termina em `.pyc`, nunca em `.pyo`.

Alterado na versão 3.7: Adicionada a opção `--invalidation-mode`.

Alterado na versão 3.9: Adicionadas as opções `-s`, `-p`, `-e` e `--hardlink-dupes`. Aumentado o limite de recursão padrão de 10 para `sys.getrecursionlimit()`. Adicionada a possibilidade de especificar a opção `-o` várias vezes.

Não há opção na linha de comando para controlar o nível de otimização usado pela função `compile()` porque o próprio interpretador Python já fornece a opção: **python -O -m compileall**.

Da mesma forma, a função `compile()` respeita a configuração `sys.pycache_prefix`. O cache do bytecode gerado somente será útil se `compile()` for executado com o mesmo `sys.pycache_prefix` (se houver) que será usado em tempo de execução.

Funções públicas

compileall.compile_dir(*dir*, *maxlevels=sys.getrecursionlimit()*, *ddir=None*, *force=False*, *rx=None*, *quiet=0*, *legacy=False*, *optimize=-1*, *workers=1*, *invalidation_mode=None*, *, *stripdir=None*, *prependdir=None*, *limit_sl_dest=None*, *hardlink_dupes=False*)

Desce recursivamente a árvore de diretórios nomeada por *dir*, compilando todos os arquivos `.py` ao longo do caminho. Retorna um valor verdadeiro se todos os arquivos forem compilados com êxito e um valor falso caso contrário.

O parâmetro *maxlevels* é usado para limitar a profundidade da recursão; o padrão é `sys.getrecursionlimit()`.

Se *ddir* for fornecido, ele será anexado ao caminho de cada arquivo que está sendo compilado para uso em tracebacks em tempo de compilação e também será compilado no arquivo de bytecode, onde será usado em tracebacks e outras mensagens nos casos em que o arquivo de origem não existe no momento em que o arquivo de bytecode é executado.

Se *force* for verdadeiro, os módulos serão recompilados, mesmo que os carimbos de data e hora estejam atualizados.

Se *rx* for fornecido, seu método `search` será chamado no caminho completo para cada arquivo considerado para compilação e, se retornar um valor verdadeiro, o arquivo será ignorado. Isso pode ser usado para excluir arquivos correspondendo a uma expressão regular, dado como um objeto re.Pattern.

Se *quiet* for `False` ou `0` (o padrão), os nomes dos arquivos e outras informações serão impressos com o padrão. Definido como `1`, apenas os erros são impressos. Definido como `2`, toda a saída é suprimida.

Se *legacy* for verdadeiro, os arquivos de bytecodes serão gravados em seus locais e nomes herdados, o que poderá sobrescrever arquivos de bytecodes criados por outra versão do Python. O padrão é gravar arquivos em seus locais e nomes do **PEP 3147** [https://peps.python.org/pep-3147/], o que permite que arquivos de bytecodes de várias versões do Python coexistam.

optimize especifica o nível de otimização para o compilador. Ele é passado para a função embutida `compile()`. Aceita também uma sequência de níveis de otimização que levam a várias compilações de um arquivo `.py` em uma chamada.

O argumento *workers* especifica quantos workers são usados para compilar arquivos em paralelo. O padrão é não usar vários workers. Se a plataforma não puder usar vários workers e o argumento *workers* for fornecido, a compilação sequencial será usada como reserva. Se *workers* for 0, o número de núcleos no sistema é usado. Se *workers* for menor que `0`, a `ValueError` será levantada.

invalidation_mode deve ser um membro de enum `py_compile.PycInvalidationMode` e controla como os pycs gerados são invalidados em tempo de execução.

Os argumentos *stripdir*, *prependdir* e *limit_sl_dest* correspondem às opções `-s`, `-p` e `-e` descrita acima. eles podem ser especificados como `str` ou `os.PathLike`.

Se *hardlink_dupes* for verdadeiro e dois arquivos `.pyc` com nível de otimização diferente tiverem o mesmo conteúdo, usa links físicos para consolidar arquivos duplicados.

Alterado na versão 3.2: Adicionado os parâmetros *legacy* e *optimize*.

Alterado na versão 3.5: Adicionado o parâmetro *workers*.

Alterado na versão 3.5: O parâmetro *quiet* foi alterado para um valor multinível.

Alterado na versão 3.5: O parâmetro *legacy* grava apenas arquivos `.pyc`, não os arquivos `.pyo`, independentemente do valor de *optimize*.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado.

Alterado na versão 3.7.2: O valor padrão do parâmetro *invalidation_mode* é atualizado para `None`.

Alterado na versão 3.8: A definição de *workers* como 0 agora escolhe o número ideal de núcleos.

Alterado na versão 3.9: Adicionados os argumentos *stripdir*, *prependdir*, *limit_sl_dest* e *hardlink_dupes*. O valor padrão de *maxlevels* foi alterado de `10` para `sys.getrecursionlimit()`

compileall.compile_file(*fullname, ddir=None, force=False, rx=None, quiet=0, legacy=False, optimize=-1, invalidation_mode=None, *, stripdir=None, prependdir=None, limit_sl_dest=None, hardlink_dupes=False*)

Compila o arquivo com o caminho *fullname*. Retorna um valor verdadeiro se o arquivo compilado com êxito e um valor falso caso contrário.

Se *ddir* for fornecido, ele será anexado ao caminho do arquivo que está sendo compilado para uso em rastreamentos em tempo de compilação e também será compilado no arquivo de bytecode, onde será usado em tracebacks e outras mensagens nos casos em que o arquivo fonte não existe no momento em que o arquivo de bytecode é executado.

Se *rx* for fornecido, seu método `search` passará o nome do caminho completo para o arquivo que está sendo compilado e, se retornar um valor verdadeiro, o arquivo não será compilado e `True` será retornado. Isso pode ser usado para excluir arquivos correspondendo a uma expressão regular, dado como um objeto re.Pattern.

Se *quiet* for `False` ou `0` (o padrão), os nomes dos arquivos e outras informações serão impressos com o padrão. Definido como `1`, apenas os erros são impressos. Definido como `2`, toda a saída é suprimida.

Se *legacy* for verdadeiro, os arquivos de bytecodes serão gravados em seus locais e nomes herdados, o que poderá sobrescrever arquivos de bytecodes criados por outra versão do Python. O padrão é gravar arquivos em seus locais e nomes do **PEP 3147** [https://peps.python.org/pep-

3147/], o que permite que arquivos de bytecodes de várias versões do Python coexistam.

optimize especifica o nível de otimização para o compilador. Ele é passado para a função embutida `compile()`. Aceita também uma sequência de níveis de otimização que levam a várias compilações de um arquivo `.py` em uma chamada.

invalidation_mode deve ser um membro de enum `py_compile.PycInvalidationMode` e controla como os pycs gerados são invalidados em tempo de execução.

Os argumentos *stripdir*, *prependdir* e *limit_sl_dest* correspondem às opções `-s`, `-p` e `-e` descrita acima. eles podem ser especificados como `str` ou `os.PathLike`.

Se *hardlink_dupes* for verdadeiro e dois arquivos `.pyc` com nível de otimização diferente tiverem o mesmo conteúdo, usa links físicos para consolidar arquivos duplicados.

Adicionado na versão 3.2.

Alterado na versão 3.5: O parâmetro *quiet* foi alterado para um valor multinível.

Alterado na versão 3.5: O parâmetro *legacy* grava apenas arquivos `.pyc`, não os arquivos `.pyo`, independentemente do valor de *optimize*.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado.

Alterado na versão 3.7.2: O valor padrão do parâmetro *invalidation_mode* é atualizado para `None`.

Alterado na versão 3.9: Adicionados os argumentos *stripdir*, *prependdir*, *limit_sl_dest* e *hardlink_dupes*.

compileall.compile_path(*skip_curdir=True*, *maxlevels=0*, *force=False*, *quiet=0*, *legacy=False*, *optimize=-1*, *invalidation_mode=None*)

Compila Byte para bytecodes todos os arquivos `.py` encontrados ao longo de `sys.path`. Retorna um valor verdadeiro se todos os arquivos forem compilados com êxito e um valor falso caso contrário.

Se *skip_curdir* for verdadeiro (o padrão), o diretório atual não será incluído na pesquisa. Todos os outros parâmetros são passados para a função **`compile_dir()`**. Note que, ao contrário das outras funções de compilação, `maxlevels` é padronizado como `0`.

Alterado na versão 3.2: Adicionado os parâmetros *legacy* e *optimize*.

Alterado na versão 3.5: O parâmetro *quiet* foi alterado para um valor multinível.

Alterado na versão 3.5: O parâmetro *legacy* grava apenas arquivos `.pyc`, não os arquivos `.pyo`, independentemente do valor de *optimize*.

Alterado na versão 3.7: O parâmetro *invalidation_mode* foi adicionado.

Alterado na versão 3.7.2: O valor padrão do parâmetro *invalidation_mode* é atualizado para `None`.

Para forçar uma recompilação de todos os arquivos `.py` no subdiretório `Lib/` e todos os seus subdiretórios:

```
import compileall

compileall.compile_dir('Lib/', force=True)

# Efetua a mesma compilação, excluindo arquivos em diretórios .s
import re
compileall.compile_dir('Lib/', rx=re.compile(r'[/\\][.]svn'), fo

# Objetos pathlib.Path também podem ser usados.
import pathlib
compileall.compile_dir(pathlib.Path('Lib/'), force=True)
```

Ver também

Módulo `py_compile`

Compila para bytecode um único arquivo fonte.

`dis` — Disassembler de bytecode do Python

Código-fonte: Lib/dis.py [https://github.com/python/cpython/tree/3.15/Lib/dis.py]

The **`dis`** module supports the analysis of CPython bytecode by disassembling it. The CPython bytecode which this module takes as an input is defined in the file `Include/opcode.h` and used by the compiler and the interpreter.

O bytecode é um detalhe de implementação do interpretador CPython. Não há garantias de que bytecodes não serão adicionados, removidos ou alterados entre as versões do Python. O uso deste módulo não deve ser considerado que funcionará em todas as VMs do Python ou mesmo versões do Python.

Alterado na versão 3.6: Cada instrução ocupa 2 bytes. Anteriormente, o número de bytes variava de acordo com a instrução.

Alterado na versão 3.10: O argumento para instruções de pulo, tratamento de exceção e laço é agora o deslocamento em instruções, ao invés de em bytes.

Alterado na versão 3.11: Some instructions are accompanied by one or more inline cache entries, which take the form of **`CACHE`** instructions. These instructions are hidden by default, but can be shown by passing `show_caches=True` to any **`dis`** utility. Furthermore, the interpreter now adapts the bytecode to specialize it for different runtime conditions. The adaptive bytecode can be shown by passing `adaptive=True`.

Alterado na versão 3.12: O argumento de um pulo é o deslocamento da instrução alvo relativo à instrução que aparece imediatamente após as entradas **`CACHE`** da instrução de pulo.

Como consequência, a presença de instruções **CACHE** é transparente para pulos adiante, mas precisa ser considerada ao lidar com pulos para trás.

Alterado na versão 3.13: A saída agora mostra rótulos lógicos ao invés dos deslocamentos das instruções para alvos de pulos e de tratadores de exceções. A opção de linha de comando `-O` e o argumento `show_offsets` foram adicionados.

Alterado na versão 3.14: A opção de linha de comando **-P** e o argumento `show_positions` foram adicionados.

A opção de linha de comando **-S** foi adicionada.

Exemplo: Dada a função **myfunc()**:

```
def myfunc(alist):
    return len(alist)
```

o comando a seguir pode ser usado para mostrar a desconstrução de **myfunc()**:

```
>>> dis.dis(myfunc)
  2           RESUME                   0

  3           LOAD_GLOBAL              1 (len + NULL)
              LOAD_FAST_BORROW         0 (alist)
              CALL                     1
              RETURN_VALUE
```

(O “2” é o número da linha).

Interface de linha de comando

The **dis** module can be invoked as a script from the command line:

```
python -m dis [-h] [-C] [-O] [-P] [-S] [infile]
```

As seguintes opções são aceitas:

-h, --help
Exibe o modo de uso e sai.

-C, --show-caches
Mostra caches em linha

Adicionado na versão 3.13.

-O, --show-offsets
Mostra os deslocamentos das instruções

Adicionado na versão 3.13.

-P, --show-positions
Mostra posições de instruções no código-fonte.

Adicionado na versão 3.14.

-S, --specialized
Mostra bytecode especializado.

Adicionado na versão 3.14.

Se `infile` for especificada, o seu código desmontado será escrito no stdout. Caso contrário, será feito o desmonte da compilação do código-fonte recebido do stdin.

Análise de bytecode

Adicionado na versão 3.4.

A API de análise de bytecode permite que partes do código Python sejam encapsuladas em um objeto **`Bytecode`** que facilite o acesso aos detalhes do código compilado.

class dis.Bytecode(*x*, ***, *first_line=None*, *current_offset=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*,

show_positions=False)

Analisa o bytecode correspondente a uma função, um gerador, um gerador assíncrono, uma corrotina, um método, uma string de código-fonte, ou um objeto de código (conforme retornado por `compile()`).

Esta é um invólucro de conveniência que encapsula muitas das funções listadas abaixo, principalmente a `get_instructions()`, já que iterar sobre sobre uma instância de `Bytecode` produz operações bytecode como instâncias de `Instruction`.

Se *first_line* não for `None`, ele indica o número de linha que deve ser reportado para a primeira linha de código-fonte no código desmontado. Caso contrário, a informação de linha de código-fonte (se houver) é extraída diretamente da desconstrução do objeto de código.

Se *current_offset* não for `None`, ele é um deslocamento em instruções no código desconstruído. Definir este argumento significa que o `dis()` vai mostrar um marcador de "instrução atual" sobre o opcode especificado.

Se *show_caches* for `True`, o `dis()` vai exibir entradas de cache em linha usadas pelo interpretador para especializar o bytecode.

Se *adaptive* for `True`, o `dis()` vai exibir bytecode especializado que pode ser diferente do bytecode original.

Se *show_offsets* for `True`, o `dis()` vai incluir deslocamentos em instruções na saída.

Se *show_positions* for `True`, `dis()` incluirá posições do código-fonte da instrução na saída.

classmethod from_traceback(*tb*, ***, *show_caches=False*)

Constrói uma instância de `Bytecode` a partir do traceback fornecido, definindo *current_offset* apontando para a instrução responsável pela exceção.

codeobj

O objeto de código compilado.

first_line

A primeira linha de código-fonte do objeto de código (caso disponível).

dis()

Retorna uma visualização formatada das operações em bytecode (as mesmas que seriam impressas pela `dis.dis()`, mas retornadas como uma string multilinha).

info()

Retorna uma string multilinha formatada com informação detalhada sobre o objeto de código, como `code_info()`.

Alterado na versão 3.7: Este método agora lida com objetos de corrotina e de gerador assíncrono.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Adicionado o parâmetro *show_offsets*

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

Exemplo:

```
>>> bytecode = dis.Bytecode(myfunc)
>>> for instr in bytecode:
...     print(instr.opname)
...
RESUME
LOAD_GLOBAL
LOAD_FAST_BORROW
CALL
RETURN_VALUE
```

Funções de análise

The **`dis`** module also defines the following analysis functions that convert the input directly to the desired output. They can be useful if only a single operation is being performed, so the intermediate analysis object isn't useful:

dis.code_info(*x*)

Retorna uma string multilinha formatada com informação detalhada sobre o objeto de código correspondente à função, gerador, gerador assíncrono, corrotina, método, string de código-fonte ou objeto de código fornecido.

Observe que o conteúdo exato de strings de informação de código são altamente dependentes da implementação e podem mudar de forma arbitrária através de VMs Python ou lançamentos do Python.

Adicionado na versão 3.2.

Alterado na versão 3.7: Este método agora lida com objetos de corrotina e de gerador assíncrono.

dis.show_code(*x*, *, *file=None*)

Imprime no arquivo *file* (ou `sys.stdout` caso *file* não seja especificado) informações detalhadas sobre o objeto de código correspondente à função, método, string de código-fonte fornecido.

Este é um atalho conveniente para `print(code_info(x), file=file)`, destinado à exploração interativa no prompt do interpretador.

Adicionado na versão 3.2.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

dis.dis(*x=None*, *, *file=None*, *depth=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)

Desmonta o objeto *x*. *x* pode denotar um módulo, uma classe, um método, uma função, um gerador, um gerador assíncrono, uma corrotina, um objeto de código, uma string de código-fonte ou uma

sequência de bytes contendo bytecode bruto. Para um módulo, são desmontadas todas as funções. Para uma classe, são desmontados todos os métodos (incluindo métodos de classe e estáticos). Para um objeto de código ou sequência de bytecodes brutos, é impressa uma linha para cada instrução de bytecode. Além disso, objetos de código aninhados são desmontados recursivamente. Estes podem incluir expressões geradoras, funções aninhadas, corpos de classes aninhadas, e objetos de código usados para escopos de anotação. Strings são compiladas para objetos de código com a função embutida **`compile()`** antes de serem desmontadas. Se nenhum objeto for fornecido, o último traceback é desmontado.

O resultado é escrito como texto no arquivo *file* caso tenha sido fornecido como argumento, ou para `sys.stdout` caso contrário.

A profundidade máxima de recursão é limitada por *depth* a menos que seja `None`. `depth=0` significa não fazer recursão.

Se *show_caches* for `True`, essa função vai exibir entradas de cache em linha usadas pelo interpretador para especializar o bytecode.

Se *adaptive* for `True`, essa função vai exibir bytecode especializado que pode ser diferente do bytecode original.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

Alterado na versão 3.7: Foi implementada a desmontagem recursiva, e adicionado o parâmetro *depth*.

Alterado na versão 3.7: Este método agora lida com objetos de corrotina e de gerador assíncrono.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Foi adicionado o parâmetro *show_offsets*.

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

dis.distb(*tb=None*, *, *file=None*, *show_caches=False*, *adaptive=False*, *show_offset=False*, *show_positions=False*)

Desmonta a função no topo da pilha de um traceback, usando o último traceback caso nenhum tenha sido passado. A instrução que causou a exceção é indicada.

O resultado é escrito como texto no arquivo *file* caso tenha sido fornecido como argumento, ou para `sys.stdout` caso contrário.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Foi adicionado o parâmetro *show_offsets*.

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

dis.disassemble(*code*, *lasti=-1*, *, *file=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)
dis.disco(*code*, *lasti=-1*, *, *file=None*, *show_caches=False*, *adaptive=False*, *show_offsets=False*, *show_positions=False*)

Desmonta um objeto de código, indicando a última instrução se *lasti* tiver sido fornecido. A saída é dividida em colunas da seguinte forma:

1. a localização do código-fonte da instrução. Informações completas sobre a localização são exibidas se *show_positions* for verdadeiro. Caso contrário (o padrão), apenas o número da linha é exibido.
2. a instrução atual, indicada por `-->`,
3. um rótulo da instrução, indicado com `>>`,
4. o endereço da instrução
5. o nome do código da operação,
6. os parâmetros da operação, e
7. a interpretação dos parâmetros, em parênteses.

A interpretação dos parâmetros reconhece nomes de variáveis locais e globais, valores de constantes, alvos de ramificações, e operadores de comparação.

O resultado é escrito como texto no arquivo *file* caso tenha sido fornecido como argumento, ou para `sys.stdout` caso contrário.

Alterado na versão 3.4: Adicionado o parâmetro *file*.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: Foi adicionado o parâmetro *show_offsets*.

Alterado na versão 3.14: Foi adicionado o parâmetro *show_positions*.

dis.get_instructions(*x*, *, *first_line=None*, *show_caches=False*, *adaptive=False*)

Retorna um iterador sobre as instruções na função, método, string de código-fonte ou objeto de código fornecido.

O iterador gera uma série de tuplas nomeadas **`Instruction`** contendo detalhes de cada operação no código fornecido.

Se *first_line* não for `None`, ele indica o número de linha que deve ser reportado para a primeira linha de código-fonte no código desmontado. Caso contrário, a informação de linha de código-fonte (se houver) é extraída diretamente da desconstrução do objeto de código.

O parâmetro *adaptive* funciona assim como na função **`dis()`**.

Adicionado na versão 3.4.

Alterado na versão 3.11: Adicionados os parâmetros *show_caches* e *adaptive*.

Alterado na versão 3.13: O parâmetro *show_caches* foi descontinuado e não tem mais efeito. O iterador gera as instâncias da **`Instruction`** com o campo *cache_info* populado (independentemente do valor de *show_caches*) e não gera mais items separados para as entradas de cache.

dis.findlinestarts(*code*)

Essa função geradora usa o método `co_lines()` do objeto de código *code* para encontrar as posições que correspondem aos inícios de cada linha do código-fonte. Elas são geradas em pares `(offset, lineno)`.

Alterado na versão 3.6: Números de linhas podem ser decrescentes. Antes, eles eram sempre crescentes.

Alterado na versão 3.10: The **PEP 626** [https://peps.python.org/pep-0626/] `co_lines()` method is used instead of the `co_firstlineno` and **`codeobject.co_lnotab`** attributes of the code object.

Alterado na versão 3.13: Números de linha podem ser `None` para bytecode que não corresponde a um código-fonte.

dis.findlabels(*code*)

Detecta todas as posições na string de bytecode compilado bruto *code* que são alvos de pulos, e as retorna em uma lista.

dis.stack_effect(*opcode*, *oparg=None*, *, *jump=None*)

Calcula o efeito que o *opcode* com argumento *oparg* tem na pilha.

Se a operação tiver um alvo de pulo e *jump* for `True`, `stack_effect()` vai retornar o efeito na pilha de realizar o pulo. Se *jump* for `False`, ela vai retornar o efeito na pilha de não pular. E se *jump* for `None` (o padrão), vai retornar o efeito máximo na pilha dentre os dois casos.

Adicionado na versão 3.4.

Alterado na versão 3.8: Adicionado o parâmetro *jump*.

Alterado na versão 3.13: Se `oparg` for omitido (ou `None`), agora é retornado o efeito na pilha para o caso `oparg=0`. Anteriormente isso era um erro caso o opcode usasse o seu argumento. Além disso, agora não é mais um erro passar um inteiro como `oparg` quando o `opcode` não o usa; o `oparg` nesse caso é ignorado.

Instruções em bytecode do Python

A função `get_instructions()` e a classe `Bytecode` fornecem detalhes de instruções de bytecode como instâncias de `Instruction`:

class dis.Instruction

Detalhes de uma operação em bytecode

opcode

código numérico da operação, correspondendo aos valores dos opcodes listados abaixo e aos valores dos bytecodes nas Opcode collections.

opname

nome legível por humanos para a operação

baseopcode

código numérico para a operação base caso a operação seja especializada; caso contrário, igual ao `opcode`

baseopname

nome legível por humanos para a operação base caso a operação seja especializada; caso contrário, igual ao `opname`

arg

argumento numérico para a operação (se houver), caso contrário `None`

oparg

apelido para `arg`

argval

valor resolvido do argumento (se houver), caso contrário `None`

argrepr

descrição legível por humanos do argumento da operação (se houver), caso contrário uma string vazia.

offset
: índice de início da operação dentro da sequência de bytecodes

start_offset
: índice de início da operação dentro da sequência de bytecodes, incluindo as operações de `EXTENDED_ARG` prefixadas, caso presentes; caso contrário, igual a **`offset`**

cache_offset
: índice de início das entradas de cache que seguem a operação

end_offset
: índice de fim das entradas de cache que seguem a operação

starts_line
: `True` se esse opcode inicia uma linha, `False` caso contrário

line_number
: linha de código-fonte associada a esse opcode (se houver), senão `None`

is_jump_target
: `True` se algum outro código pula para cá, senão `False`

jump_target
: índice do bytecode que é alvo do pulo caso essa seja uma operação de pulo, caso contrário `None`

positions
: objeto **`dis.Positions`** contendo os pontos de início e fim cobertos por esta instrução.

cache_info
: Informações sobre as entradas de cache desta instrução, como tripletos do formato `(nome, tamanho, dados)`, onde `nome` e `tamanho` descrevem o formato do cache e dados são o conteúdo do cache. `cache_info` é `None` se a instrução não tiver caches.

Adicionado na versão 3.4.

Alterado na versão 3.11: Adicionado o campo `positions`.

Alterado na versão 3.13: Alterado o campo `starts_line`.

Adicionados os campos `start_offset`, `cache_offset`, `end_offset`, `baseopname`, `baseopcode`, `jump_target`, `oparg`, `line_number` e `cache_info`.

class dis.Positions

Caso a informação não esteja disponível, alguns campos podem ser `None`.

lineno

end_lineno

col_offset

end_col_offset

Adicionado na versão 3.11.

O compilador de Python atualmente gera as seguintes instruções de bytecode.

Instruções gerais

A seguir, vamos usar `STACK` para nos referirmos à pilha do interpretador, e vamos descrever operações nela como se ela fosse uma lista do Python. Nessa linguagem, `STACK[-1]` é o topo da pilha.

NOP

Código para não fazer nada. Usado como espaço reservado pelo otimizador de bytecode, e para gerar eventos de rastreamento de linha.

NOT_TAKEN

Código sem ação. Usado pelo interpretador para registrar os eventos **BRANCH_LEFT** e **BRANCH_RIGHT** para **sys.monitoring**.

Adicionado na versão 3.14.

POP_ITER

Remove o iterador do topo da pilha.

Adicionado na versão 3.14.

POP_TOP

Remove o item no topo da pilha:

```
STACK.pop()
```

END_FOR

Remove o item no topo da pilha. Equivalente a `POP_TOP`. Usado como limpeza ao final de laços, o que explica o nome.

Adicionado na versão 3.12.

END_SEND

Implementa `del STACK[-2]`. Usado como limpeza quando um gerador termina.

Adicionado na versão 3.12.

COPY(*i*)

Coloca o i-ésimo item no topo da pilha sem removê-lo da sua posição original:

```
assert i > 0
STACK.append(STACK[-i])
```

Adicionado na versão 3.11.

SWAP(*i*)

Troca o topo da pilha de lugar com o i-ésimo elemento.

`STACK[-i], STACK[-1] = STACK[-1], STACK[-i]`

Adicionado na versão 3.11.

CACHE

Ao invés de ser uma instrução de fato, este opcode é usado para demarcar espaço extra para o interpretador armazenar dados úteis diretamente no próprio bytecode. É escondido automaticamente por todas as utilidades do `dis`, mas pode ser visualizado com `show_caches=True`.

Do ponto de vista lógico, este espaço faz parte da instrução anterior. Muitos opcodes esperam ser seguidos por um número exato de caches, e instruem o interpretador a pulá-los em tempo de execução.

Caches populados podem se parecer com qualquer instrução, de forma que ler ou modificar bytecode adaptativo bruto contendo dados "quickened" requer muito cuidado.

Adicionado na versão 3.11.

Operações unárias

Operações unárias tiram o topo da pilha, aplicam a operação, e põem o resultado de volta na pilha.

UNARY_NEGATIVE

Implementa `STACK[-1] = -STACK[-1]`.

UNARY_NOT

Implementa `STACK[-1] = not STACK[-1]`.

Alterado na versão 3.13: Essa instrução agora requer que o operando seja exatamente do tipo **`bool`**.

UNARY_INVERT

Implementa `STACK[-1] = ~STACK[-1]`.

GET_ITER

Implementa `STACK[-1] = iter(STACK[-1])`.

GET_YIELD_FROM_ITER

Se `STACK[-1]` for um iterador gerador ou um objeto corrotina, nada acontece. Caso contrário, implementa `STACK[-1] = iter(STACK[-1])`.

Adicionado na versão 3.5.

TO_BOOL

Implementa `STACK[-1] = bool(STACK[-1])`.

Adicionado na versão 3.13.

Operações binárias e internas

Operações binárias removem os dois itens no topo da pilha (`STACK[-1]` e `STACK[-2]`). A operação é realizada, e o resultado é colocado de volta na pilha.

As operações internas são como as operações binárias, só que a operação é feita internamente caso suportado por `STACK[-2]`, e o `STACK[-1]` resultante pode ser (mas não necessariamente é) o `STACK[-2]` original.

BINARY_OP(*op*)

Implementa os operadores binários e locais (depende do valor de *op*):

```
rhs = STACK.pop()
lhs = STACK.pop()
STACK.append(lhs op rhs)
```

Adicionado na versão 3.11.

Alterado na versão 3.14: Com oparg :`NB_SUBSCR`, implementa subscrito binário (substitui o opcode `BINARY_SUBSCR`)

STORE_SUBSCR

Implementa:

```
key = STACK.pop()
container = STACK.pop()
value = STACK.pop()
container[key] = value
```

DELETE_SUBSCR

Implementa:

```
key = STACK.pop()
container = STACK.pop()
del container[key]
```

BINARY_SLICE

Implementa:

```
end = STACK.pop()
start = STACK.pop()
container = STACK.pop()
STACK.append(container[start:end])
```

Adicionado na versão 3.12.

STORE_SLICE

Implementa:

```
end = STACK.pop()
start = STACK.pop()
container = STACK.pop()
value = STACK.pop()
container[start:end] = value
```

Adicionado na versão 3.12.

Opcodes para corrotinas

GET_AWAITABLE(*where*)

Implementa `STACK[-1] = get_awaitable(STACK[-1])`, onde `get_awaitable(o)` retorna `o` se `o` for um objeto de corrotina ou um gerador com o sinalizador **`CO_ITERABLE_COROUTINE`**, ou então resolve `o.__await__`.

Se o operando `where` não for zero, ele indica onde a instrução ocorre:

- `1`: Após uma chamada a `__aenter__`
- `2`: Após uma chamada a `__aexit__`

Adicionado na versão 3.5.

Alterado na versão 3.11: Anteriormente, esta instrução não tinha um oparg.

GET_AITER

Implementa `STACK[-1] = STACK[-1].__aiter__()`.

Adicionado na versão 3.5.

Alterado na versão 3.7: Não é mais aceitado que o `__aiter__` retorne objetos aguardáveis.

GET_ANEXT

Implementa `STACK.append(get_awaitable(STACK[-1].__anext__()))`. Veja `GET_AWAITABLE` para o significado de `get_awaitable`.

Adicionado na versão 3.5.

END_ASYNC_FOR

Termina um laço **`async for`**. Trata exceções levantadas ao aguardar um item seguinte. A pilha contém o iterável async em `STACK[-2]` e a exceção levantada em `STACK[-1]`. Ambos são retirados. Se a exceção não for **`StopAsyncIteration`**, ela é re-levantada.

Adicionado na versão 3.8.

Alterado na versão 3.11: A representação da exceção na pilha consiste agora de um item, ao invés de três.

CLEANUP_THROW

Trata uma exceção levantada durante um chamada a `throw()` ou `close()` através do quadro atual. Se `STACK[-1]` for uma instância de `StopIteration`, remove três valores da pilha e põe de volta o seu membro `value`. Caso contrário, re-levanta `STACK[-1]`.

Adicionado na versão 3.12.

Opcodes genéricos

SET_ADD(*i*)

Implementa:

```
item = STACK.pop()
set.add(STACK[-i], item)
```

Usado para implementar compreensões de conjuntos.

LIST_APPEND(*i*)

Implementa:

```
item = STACK.pop()
list.append(STACK[-i], item)
```

Usado para implementar compreensões de lista.

MAP_ADD(*i*)

Implementa:

```
value = STACK.pop()
key = STACK.pop()
dict.__setitem__(STACK[-i], key, value)
```

Usado para implementar compreensões de dicionário.

Adicionado na versão 3.1.

Alterado na versão 3.8: O valor do mapa é `STACK[-1]`, e a sua chave, `STACK[-2]`. Antes, eles estavam ao contrário.

Para as instruções **SET_ADD**, **LIST_APPEND** e **MAP_ADD**, o valor ou par chave/valor é removido da pilha, mas o objeto de contêiner continua na pilha para que ele esteja disponível para as iterações seguintes do laço.

RETURN_VALUE

Retorna `STACK[-1]` para quem chamou a função.

YIELD_VALUE

Gera `STACK.pop()` a partir de um gerador.

Alterado na versão 3.11: oparg definido como sendo a profundidade da pilha.

Alterado na versão 3.12: oparg definido como sendo a profundidade do bloco exception, para o fechamento eficiente de geradores.

Alterado na versão 3.13: oparg é `1` caso esta instrução seja parte de um yield-from ou await, e `0` caso contrário.

SETUP_ANNOTATIONS

Verifica se `__annotations__` está definido em `locals()` e, se não estiver, é inicializado como um `dict` vazio. Este opcode é emitido somente se o corpo de uma classe ou módulo contém anotações de variáveis estaticamente.

Adicionado na versão 3.6.

POP_EXCEPT

Remove o valor no topo da pilha, o qual é usado para restaurar o estado de exceção.

Alterado na versão 3.11: A representação da exceção na pilha consiste agora de um item, ao invés de três.

RERAISE

Re-levanta a exceção que se encontra no topo da pilha. Se o oparg não for zero, remove um valor adicional do topo da pilha, o qual é atribuído ao **f_lasti** do quadro atual.

Adicionado na versão 3.9.

Alterado na versão 3.11: A representação da exceção na pilha consiste agora de um item, ao invés de três.

PUSH_EXC_INFO

Remove um valor do topo da pilha. Põe a exceção atual no topo da pilha. Põe de volta no topo da pilha o valor que foi removido inicialmente. Usado em tratadores de exceções.

Adicionado na versão 3.11.

CHECK_EXC_MATCH

Verifica correspondências de exceções em `except`. Testa de `STACK[-2]` é uma exceção que corresponde a `STACK[-1]`. Remove `STACK[-1]` do topo da pilha, e põe no seu lugar o resultado booleano do teste.

Adicionado na versão 3.11.

CHECK_EG_MATCH

Verifica correspondências de exceções em `except*`. Aplica `split(STACK[-1])` no grupo de exceções que representa `STACK[-2]`.

No caso de uma correspondência, remove dois itens do topo da pilha e põe nela o subgrupo que falhou a correspondência (`None` caso a correspondência tenha sido total), seguido pelo subgrupo que correspondeu. Quando não há correspondência nenhuma, remove um item (o tipo da correspondência) e põe `None` no seu lugar.

Adicionado na versão 3.11.

WITH_EXCEPT_START

Chama a função na posição 4 da pilha com argumentos (tipo, val, tb) representando a exceção no topo da pilha. Usado para implementar a chamada `context_manager.__exit__(*exc_info())` quando uma exceção ocorreu em uma instrução **`with`**.

Adicionado na versão 3.9.

Alterado na versão 3.11: A função `__exit__` fica agora na posição 4 pilha, ao invés da 7. A representação da exceção pilha consiste agora de um item, não três.

LOAD_COMMON_CONSTANT

Envia uma constante comum para a pilha. O interpretador contém uma lista codificada de constantes suportadas por esta instrução. Usado pela instrução **`assert`** para carregar **`AssertionError`**.

Adicionado na versão 3.14.

LOAD_BUILD_CLASS

Põe a função **`builtins.__build_class__()`** no topo da pilha. Ela será chamada posteriormente para construir uma classe.

GET_LEN

Executa `STACK.append(len(STACK[-1]))`. Usado em instruções **`match`** onde a comparação com a estrutura do padrão é necessária.

Adicionado na versão 3.10.

MATCH_MAPPING

Se `STACK[-1]` for uma instância de **`collections.abc.Mapping`** (ou, de forma mais técnica: se tiver o sinalizador **`Py_TPFLAGS_MAPPING`** definido no seu **`tp_flags`**), põe `True` no topo da pilha. Caso contrário, põe `False`.

Adicionado na versão 3.10.

MATCH_SEQUENCE

Se `STACK[-1]` for uma instância de **`collections.abc.Sequence`** e *não* for uma instância de **`str`**/**`bytes`**/**`bytearray`** (ou, de forma mais técnica: se tiver o sinalizador **`Py_TPFLAGS_SEQUENCE`** definido no seu **`tp_flags`**), põe `True` no topo da pilha. Caso contrário, põe `False`.

Adicionado na versão 3.10.

MATCH_KEYS

`STACK[-1]` é uma tupla de chaves de um mapeamento, e `STACK[-2]` é o sujeito de uma correspondência. Se `STACK[-2]` contiver todas as chaves em `STACK[-1]`, põe no topo da pilha um **`tuple`** contendo os valores correspondentes. Caso contrário, põe `None`.

Adicionado na versão 3.10.

Alterado na versão 3.11: Anteriormente, essa instrução também colocava na pilha um valor booleano indicando sucesso (`True`) ou falha (`False`).

STORE_NAME(*namei*)

Implementa `name = STACK.pop()`. *namei* é o índice de *name* no atributo **`co_names`** do objeto de código. O compilador tenta usar **`STORE_FAST`** ou **`STORE_GLOBAL`** se possível.

DELETE_NAME(*namei*)

Implementa `del name`, onde *namei* é o índice no atributo **`co_names`** do objeto de código.

UNPACK_SEQUENCE(*count*)

Desempacota `STACK[-1]` em *count* valores individuais, os quais são postos na pilha da direita para a esquerda. Requer que haja exatamente *count* valores:

```
assert(len(STACK[-1]) == count)
STACK.extend(STACK.pop()[:-count-1:-1])
```

UNPACK_EX(*counts*)

Implementa atribuição com um alvo estrelado: desempacota o iterável `STACK[-1]` em valores individuais, sendo que pode haver menos valores do que itens no iterável: um dos novos valores será a lista de todos os itens que sobraram.

A quantidade de valores antes e após o valor que será a lista é limitada a 255.

A quantidade de valores antes do valor lista é passada no argumento do opcode. A quantidade de valores após a lista, se houver, é passada usando um `EXTENDED_ARG`. A consequência é que o argumento pode ser visto como um valor de dois bytes, onde o byte “de baixo” de *counts* é a quantidade de valores antes do valor lista, e o byte “de cima” de *counts*, a quantidade após.

Os valores extraídos são postos na pilha da direita para a esquerda, ou seja, após executar `a, *b, c = d` os valores serão armazenados como `STACK.extend((a, b, c))`.

STORE_ATTR(*namei*)

Implementa:

```
obj = STACK.pop()
value = STACK.pop()
obj.name = value
```

onde *namei* é o índice do nome no **`co_names`** do objeto de código.

DELETE_ATTR(*namei*)

Implementa:

```
obj = STACK.pop()
del obj.name
```

onde *namei* é o índice do nome no **`co_names`** do objeto de código.

STORE_GLOBAL(*namei*)

Funciona como o **`STORE_NAME`**, mas o nome é armazenado com um nome global.

DELETE_GLOBAL(*namei*)

Funciona como o **`DELETE_NAME`**, mas deleta um nome global.

LOAD_CONST(*consti*)

Põe `co_consts[consti]` no topo da pilha.

LOAD_SMALL_INT(*i*)

Coloca o inteiro `i` na pilha. `i` deve estar em `range(256)`

Adicionado na versão 3.14.

LOAD_NAME(*namei*)

Põe no topo da pilha o valor associado a `co_names[namei]`. O nome é procurado nos locais, nos globais, e então nos embutidos.

LOAD_LOCALS

Envia uma referência ao dicionário local para a pilha. Isso é usado para preparar dicionários de espaço de nomes para **`LOAD_FROM_DICT_OR_DEREF`** e **`LOAD_FROM_DICT_OR_GLOBALS`**.

Adicionado na versão 3.12.

LOAD_FROM_DICT_OR_GLOBALS(*i*)

Extrai um mapeamento da pilha e procura o valor de `co_names[namei]`. Se o nome não for encontrado, procura-o nas variáveis globais e, em seguida, nas variáveis embutidas, semelhante a **`LOAD_GLOBAL`**. Isso é usado para carregar variáveis globais em escopos de anotação dentro dos corpos de classe.

Adicionado na versão 3.12.

BUILD_TEMPLATE

Constrói uma nova instância **`Template`** a partir de uma tupla de strings e uma tupla de interpolações e envia o objeto resultante para a pilha:

```
interpolations = STACK.pop()
strings = STACK.pop()
STACK.append(_build_template(strings, interpolations))
```

Adicionado na versão 3.14.

BUILD_INTERPOLATION(*format*)

Constrói uma nova instância **`Interpolation`** a partir de um valor e sua expressão de origem e envia o objeto resultante para a pilha.

Se nenhuma especificação de conversão ou formato estiver presente, `format` será definido como `2`.

If the low bit of `format` is set, it indicates that the interpolation contains a format specification.

If `format >> 2` is non-zero, it indicates that the interpolation contains a conversion. The value of `format >> 2` is the conversion type (`0` for no conversion, `1` for `!s`, `2` for `!r`, and `3` for `!a`):

```
conversion = format >> 2
if format & 1:
    format_spec = STACK.pop()
else:
    format_spec = None
expression = STACK.pop()
value = STACK.pop()
STACK.append(_build_interpolation(value, expression, convers
```

Adicionado na versão 3.14.

BUILD_TUPLE(*count*)

Creates a tuple consuming *count* items from the stack, and pushes the resulting tuple onto the stack:

```
if count == 0:
    value = ()
else:
    value = tuple(STACK[-count:])
    STACK = STACK[:-count]

STACK.append(value)
```

BUILD_LIST(*count*)

Works as **`BUILD_TUPLE`**, but creates a list.

BUILD_SET(*count*)

Works as **`BUILD TUPLE`**, but creates a set.

BUILD_MAP(*count*)

Pushes a new dictionary object onto the stack. Pops `2 * count` items so that the dictionary holds *count* entries: `{..., STACK[-4]: STACK[-3], STACK[-2]: STACK[-1]}`.

Alterado na versão 3.5: The dictionary is created from stack items instead of creating an empty dictionary pre-sized to hold *count* items.

BUILD_STRING(*count*)

Concatenates *count* strings from the stack and pushes the resulting string onto the stack.

Adicionado na versão 3.6.

LIST_EXTEND(*i*)

Implementa:

```
seq = STACK.pop()
list.extend(STACK[-i], seq)
```

Used to build lists.

Adicionado na versão 3.9.

SET_UPDATE(*i*)

Implementa:

```
seq = STACK.pop()
set.update(STACK[-i], seq)
```

Used to build sets.

Adicionado na versão 3.9.

DICT_UPDATE(*i*)

Implementa:

```
map = STACK.pop()
dict.update(STACK[-i], map)
```

Used to build dicts.

Adicionado na versão 3.9.

DICT_MERGE(*i*)

Like **DICT_UPDATE** but raises an exception for duplicate keys.

Adicionado na versão 3.9.

LOAD_ATTR(*namei*)

If the low bit of `namei` is not set, this replaces `STACK[-1]` with `getattr(STACK[-1], co_names[namei>>1])`.

If the low bit of `namei` is set, this will attempt to load a method named `co_names[namei>>1]` from the `STACK[-1]` object. `STACK[-1]` is popped. This bytecode distinguishes two cases: if `STACK[-1]` has a method with the correct name, the bytecode pushes the unbound method and `STACK[-1]`. `STACK[-1]` will be used as the first argument (`self`) by **CALL** or **CALL_KW** when calling the unbound method. Otherwise, `NULL` and the object returned by the attribute lookup are pushed.

Alterado na versão 3.12: If the low bit of `namei` is set, then a `NULL` or `self` is pushed to the stack before the attribute or unbound method respectively.

LOAD_SUPER_ATTR(*namei*)

This opcode implements **super()**, both in its zero-argument and two-argument forms (e.g. `super().method()`, `super().attr` and `super(cls, self).method()`, `super(cls, self).attr`).

It pops three values from the stack (from top of stack down):

- `self`: the first argument to the current method

- `cls`: the class within which the current method was defined
- the global `super`

With respect to its argument, it works similarly to **`LOAD_ATTR`**, except that `namei` is shifted left by 2 bits instead of 1.

The low bit of `namei` signals to attempt a method load, as with **`LOAD_ATTR`**, which results in pushing `NULL` and the loaded method. When it is unset a single value is pushed to the stack.

The second-low bit of `namei`, if set, means that this was a two-argument call to **`super()`** (unset means zero-argument).

Adicionado na versão 3.12.

COMPARE_OP(*opname*)

Performs a Boolean operation. The operation name can be found in `cmp_op[opname >> 5]`. If the fifth-lowest bit of `opname` is set (`opname & 16`), the result should be coerced to `bool`.

Alterado na versão 3.13: The fifth-lowest bit of the oparg now indicates a forced conversion to **`bool`**.

IS_OP(*invert*)

Performs `is` comparison, or `is not` if `invert` is 1.

Adicionado na versão 3.9.

CONTAINS_OP(*invert*)

Performs `in` comparison, or `not in` if `invert` is 1.

Adicionado na versão 3.9.

IMPORT_NAME(*namei*)

Imports the module `co_names[namei]`. `STACK[-1]` and `STACK[-2]` are popped and provide the *fromlist* and *level* arguments of **`__import__()`**. The module object is pushed onto the stack. The current namespace is

not affected: for a proper import statement, a subsequent **STORE_FAST** instruction modifies the namespace.

IMPORT_FROM(*namei*)

Loads the attribute `co_names[namei]` from the module found in `STACK[-1]`. The resulting object is pushed onto the stack, to be subsequently stored by a **STORE_FAST** instruction.

JUMP_FORWARD(*delta*)

Increments bytecode counter by *delta*.

JUMP_BACKWARD(*delta*)

Decrements bytecode counter by *delta*. Checks for interrupts.

Adicionado na versão 3.11.

JUMP_BACKWARD_NO_INTERRUPT(*delta*)

Decrements bytecode counter by *delta*. Does not check for interrupts.

Adicionado na versão 3.11.

POP_JUMP_IF_TRUE(*delta*)

If `STACK[-1]` is true, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Alterado na versão 3.11: The oparg is now a relative delta rather than an absolute target. This opcode is a pseudo-instruction, replaced in final bytecode by the directed versions (forward/backward).

Alterado na versão 3.12: This is no longer a pseudo-instruction.

Alterado na versão 3.13: Essa instrução agora requer que o operando seja exatamente do tipo **bool**.

POP_JUMP_IF_FALSE(*delta*)

If `STACK[-1]` is false, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Alterado na versão 3.11: The oparg is now a relative delta rather than an absolute target. This opcode is a pseudo-instruction, replaced in final bytecode by the directed versions (forward/backward).

Alterado na versão 3.12: This is no longer a pseudo-instruction.

Alterado na versão 3.13: Essa instrução agora requer que o operando seja exatamente do tipo `bool`.

POP_JUMP_IF_NOT_NONE(*delta*)

If `STACK[-1]` is not `None`, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Adicionado na versão 3.11.

Alterado na versão 3.12: This is no longer a pseudo-instruction.

POP_JUMP_IF_NONE(*delta*)

If `STACK[-1]` is `None`, increments the bytecode counter by *delta*. `STACK[-1]` is popped.

Adicionado na versão 3.11.

Alterado na versão 3.12: This is no longer a pseudo-instruction.

FOR_ITER(*delta*)

`STACK[-1]` is an iterator. Call its `__next__()` method. If this yields a new value, push it on the stack (leaving the iterator below it). If the iterator indicates it is exhausted then the byte code counter is incremented by *delta*.

Alterado na versão 3.12: Up until 3.11 the iterator was popped when it was exhausted.

LOAD_GLOBAL(*namei*)

Loads the global named `co_names[namei>>1]` onto the stack.

Alterado na versão 3.11: If the low bit of `namei` is set, then a `NULL` is pushed to the stack before the global variable.

LOAD_FAST(*var_num*)

Pushes a reference to the local `co_varnames[var_num]` onto the stack.

Alterado na versão 3.12: This opcode is now only used in situations where the local variable is guaranteed to be initialized. It cannot raise `UnboundLocalError`.

LOAD_FAST_BORROW(*var_num*)

Pushes a borrowed reference to the local `co_varnames[var_num]` onto the stack.

Adicionado na versão 3.14.

LOAD_FAST_LOAD_FAST(*var_nums*)

Pushes references to `co_varnames[var_nums >> 4]` and `co_varnames[var_nums & 15]` onto the stack.

Adicionado na versão 3.13.

LOAD_FAST_BORROW_LOAD_FAST_BORROW(*var_nums*)

Pushes borrowed references to `co_varnames[var_nums >> 4]` and `co_varnames[var_nums & 15]` onto the stack.

Adicionado na versão 3.14.

LOAD_FAST_CHECK(*var_num*)

Pushes a reference to the local `co_varnames[var_num]` onto the stack, raising an `UnboundLocalError` if the local variable has not been initialized.

Adicionado na versão 3.12.

LOAD_FAST_AND_CLEAR(*var_num*)

Pushes a reference to the local `co_varnames[var_num]` onto the stack (or pushes `NULL` onto the stack if the local variable has not been initialized) and sets `co_varnames[var_num]` to `NULL`.

Adicionado na versão 3.12.

STORE_FAST(*var_num*)

Stores `STACK.pop()` into the local `co_varnames[var_num]`.

STORE_FAST_STORE_FAST(*var_nums*)

Stores `STACK[-1]` into `co_varnames[var_nums >> 4]` and `STACK[-2]` into `co_varnames[var_nums & 15]`.

Adicionado na versão 3.13.

STORE_FAST_LOAD_FAST(*var_nums*)

Stores `STACK.pop()` into the local `co_varnames[var_nums >> 4]` and pushes a reference to the local `co_varnames[var_nums & 15]` onto the stack.

Adicionado na versão 3.13.

DELETE_FAST(*var_num*)

Deletes local `co_varnames[var_num]`.

MAKE_CELL(*i*)

Creates a new cell in slot `i`. If that slot is nonempty then that value is stored into the new cell.

Adicionado na versão 3.11.

LOAD_DEREF(*i*)

Loads the cell contained in slot `i` of the “fast locals” storage. Pushes a reference to the object the cell contains on the stack.

Alterado na versão 3.11: `i` is no longer offset by the length of `co_varnames`.

LOAD_FROM_DICT_OR_DEREF(*i*)

Pops a mapping off the stack and looks up the name associated with slot `i` of the “fast locals” storage in this mapping. If the name is not found there, loads it from the cell contained in slot `i`, similar to **`LOAD_DEREF`**. This is used for loading closure variables in class bodies (which previously used **`LOAD_CLASSDEREF`**) and in annotation scopes within class bodies.

Adicionado na versão 3.12.

STORE_DEREF(*i*)

Stores `STACK.pop()` into the cell contained in slot `i` of the “fast locals” storage.

Alterado na versão 3.11: `i` is no longer offset by the length of **`co_varnames`**.

DELETE_DEREF(*i*)

Empties the cell contained in slot `i` of the “fast locals” storage. Used by the **`del`** statement.

Adicionado na versão 3.2.

Alterado na versão 3.11: `i` is no longer offset by the length of **`co_varnames`**.

COPY_FREE_VARS(*n*)

Copia as `n` variáveis livres (de clausura) do fechamento para o quadro. Remove a necessidade de código especial do lado do chamador ao chamar closures.

Adicionado na versão 3.11.

RAISE_VARARGS(*argc*)

Raises an exception using one of the 3 forms of the `raise` statement, depending on the value of *argc*:

- 0: `raise` (re-raise previous exception)
- 1: `raise STACK[-1]` (raise exception instance or type at `STACK[-1]`)
- 2: `raise STACK[-2] from STACK[-1]` (raise exception instance or type at `STACK[-2]` with `__cause__` set to `STACK[-1]`)

CALL(*argc*)

Calls a callable object with the number of arguments specified by `argc`. On the stack are (in ascending order):

- The callable
- `self` or `NULL`
- The remaining positional arguments

`argc` is the total of the positional arguments, excluding `self`.

`CALL` pops all arguments and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object.

Adicionado na versão 3.11.

Alterado na versão 3.13: The callable now always appears at the same position on the stack.

Alterado na versão 3.13: Calls with keyword arguments are now handled by **`CALL_KW`**.

CALL_KW(*argc*)

Calls a callable object with the number of arguments specified by `argc`, including one or more named arguments. On the stack are (in ascending order):

- The callable
- `self` or `NULL`
- The remaining positional arguments
- The named arguments
- A **`tuple`** of keyword argument names

`argc` is the total of the positional and named arguments, excluding `self`. The length of the tuple of keyword argument names is the number of named arguments.

`CALL_KW` pops all arguments, the keyword names, and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object.

Adicionado na versão 3.13.

CALL_FUNCTION_EX(*flags*)

Calls a callable object with variable set of positional and keyword arguments. If the lowest bit of *flags* is set, the top of the stack contains a mapping object containing additional keyword arguments. Before the callable is called, the mapping object and iterable object are each "unpacked" and their contents passed in as keyword and positional arguments respectively. `CALL_FUNCTION_EX` pops all arguments and the callable object off the stack, calls the callable object with those arguments, and pushes the return value returned by the callable object.

Adicionado na versão 3.6.

PUSH_NULL

Pushes a `NULL` to the stack. Used in the call sequence to match the `NULL` pushed by **`LOAD_ATTR`** for non-method calls.

Adicionado na versão 3.11.

MAKE_FUNCTION

Pushes a new function object on the stack built from the code object at `STACK[-1]`.

Alterado na versão 3.10: Flag value `0x04` is a tuple of strings instead of dictionary

Alterado na versão 3.11: Qualified name at `STACK[-1]` was removed.

Alterado na versão 3.13: Extra function attributes on the stack, signaled by oparg flags, were removed. They now use **`SET_FUNCTION_ATTRIBUTE`**.

SET_FUNCTION_ATTRIBUTE(*flag*)

Sets an attribute on a function object. Expects the function at `STACK[-1]` and the attribute value to set at `STACK[-2]`; consumes both and leaves the function at `STACK[-1]`. The flag determines which attribute to set:

- `0x01` a tuple of default values for positional-only and positional-or-keyword parameters in positional order
- `0x02` a dictionary of keyword-only parameters' default values
- `0x04` a tuple of strings containing parameters' annotations
- `0x08` a tuple containing cells for free variables, making a closure
- `0x10` the annotate function for the function object

Adicionado na versão 3.13.

Alterado na versão 3.14: Added `0x10` to indicate the annotate function for the function object.

BUILD_SLICE(*argc*)

Pushes a slice object on the stack. *argc* must be 2 or 3. If it is 2, implements:

```
end = STACK.pop()
start = STACK.pop()
STACK.append(slice(start, end))
```

if it is 3, implements:

```
step = STACK.pop()
end = STACK.pop()
start = STACK.pop()
STACK.append(slice(start, end, step))
```

See the **`slice()`** built-in function for more information.

EXTENDED_ARG(*ext*)

Prefixes any opcode which has an argument too big to fit into the default one byte. *ext* holds an additional byte which act as higher bits in the argument. For each opcode, at most three prefixal `EXTENDED_ARG` are allowed, forming an argument from two-byte to four-byte.

CONVERT_VALUE(*oparg*)

Convert value to a string, depending on `oparg`:

```
value = STACK.pop()
result = func(value)
STACK.append(result)
```

- `oparg == 1`: call **`str()`** on *value*
- `oparg == 2`: call **`repr()`** on *value*
- `oparg == 3`: call **`ascii()`** on *value*

Used for implementing formatted string literals (f-strings).

Adicionado na versão 3.13.

FORMAT_SIMPLE

Formats the value on top of stack:

```
value = STACK.pop()
result = value.__format__("")
STACK.append(result)
```

Used for implementing formatted string literals (f-strings).

Adicionado na versão 3.13.

FORMAT_WITH_SPEC

Formats the given value with the given format spec:

```
spec = STACK.pop()
value = STACK.pop()
result = value.__format__(spec)
STACK.append(result)
```

Used for implementing formatted string literals (f-strings).

Adicionado na versão 3.13.

MATCH_CLASS(*count*)

`STACK[-1]` is a tuple of keyword attribute names, `STACK[-2]` is the class being matched against, and `STACK[-3]` is the match subject. *count* is the number of positional sub-patterns.

Pop `STACK[-1]`, `STACK[-2]`, and `STACK[-3]`. If `STACK[-3]` is an instance of `STACK[-2]` and has the positional and keyword attributes required by *count* and `STACK[-1]`, push a tuple of extracted attributes. Otherwise, push `None`.

Adicionado na versão 3.10.

Alterado na versão 3.11: Anteriormente, essa instrução também colocava na pilha um valor booleano indicando sucesso (`True`) ou falha (`False`).

RESUME(*context*)

A no-op. Performs internal tracing, debugging and optimization checks.

The `context` operand consists of two parts. The lowest two bits indicate where the `RESUME` occurs:

- `0` The start of a function, which is neither a generator, coroutine nor an async generator
- `1` Depois de uma expressão `yield`
- `2` After a `yield from` expression
- `3` Depois de uma expressão `await`

The next bit is `1` if the RESUME is at except-depth `1`, and `0` otherwise.

Adicionado na versão 3.11.

Alterado na versão 3.13: The oparg value changed to include information about except-depth

RETURN_GENERATOR

Create a generator, coroutine, or async generator from the current frame. Used as first opcode of in code object for the above mentioned callables. Clear the current frame and return the newly created generator.

Adicionado na versão 3.11.

SEND(*delta*)

Equivalent to `STACK[-1] = STACK[-2].send(STACK[-1])`. Used in `yield from` and `await` statements.

If the call raises **`StopIteration`**, pop the top value from the stack, push the exception's `value` attribute, and increment the bytecode counter by *delta*.

Adicionado na versão 3.11.

HAVE_ARGUMENT

This is not really an opcode. It identifies the dividing line between opcodes in the range [0,255] which don't use their argument and those that do (`< HAVE_ARGUMENT` and `>= HAVE_ARGUMENT`, respectively).

If your application uses pseudo instructions or specialized instructions, use the **`hasarg`** collection instead.

Alterado na versão 3.6: Now every instruction has an argument, but opcodes `< HAVE_ARGUMENT` ignore it. Before, only opcodes `>= HAVE_ARGUMENT` had an argument.

Alterado na versão 3.12: Pseudo instructions were added to the **`dis`** module, and for them it is not true that comparison with `HAVE_ARGUMENT` indicates whether they use their arg.

Descontinuado desde a versão 3.13: Use **`hasarg`** instead.

CALL_INTRINSIC_1

Calls an intrinsic function with one argument. Passes `STACK[-1]` as the argument and sets `STACK[-1]` to the result. Used to implement

functionality that is not performance critical.

The operand determines which intrinsic function is called:

Operand	Descrição
`INTRINSIC_1_INVALID`	Not valid
`INTRINSIC_PRINT`	Prints the argument to standard out. Used in the REPL.
`INTRINSIC_IMPORT_STAR`	Performs `import *` for the named module.
`INTRINSIC_STOPITERATION_ERROR`	Extracts the return value from a `StopIteration` exception.
`INTRINSIC_ASYNC_GEN_WRAP`	Wraps an async generator value
`INTRINSIC_UNARY_POSITIVE`	Performs the unary `+` operation
`INTRINSIC_LIST_TO_TUPLE`	Converts a list to a tuple
`INTRINSIC_TYPEVAR`	Cria um `typing.TypeVar`
`INTRINSIC_PARAMSPEC`	Cria um `typing.ParamSpec`
`INTRINSIC_TYPEVARTUPLE`	Cria um `typing.TypeVarTuple`
`INTRINSIC_SUBSCRIPT_GENERIC`	Returns `typing.Generic` subscripted with the argument

Operand	Descrição
`INTRINSIC_TYPEALIAS`	Creates a `typing.TypeAliasType`; used in the `type` statement. The argument is a tuple of the type alias's name, type parameters, and value.

Adicionado na versão 3.12.

CALL_INTRINSIC_2

Calls an intrinsic function with two arguments. Used to implement functionality that is not performance critical:

```
arg2 = STACK.pop()
arg1 = STACK.pop()
result = intrinsic2(arg1, arg2)
STACK.append(result)
```

The operand determines which intrinsic function is called:

Operand	Descrição
`INTRINSIC_2_INVALID`	Not valid
`INTRINSIC_PREP_RERAISE_STAR`	Calculates the `ExceptionGroup` to raise from a `try-except*`.
`INTRINSIC_TYPEVAR_WITH_BOUND`	Creates a `typing.TypeVar` with a bound.
`INTRINSIC_TYPEVAR_WITH_CONSTRAINTS`	Creates a `typing.TypeVar` with constraints.
`INTRINSIC_SET_FUNCTION_TYPE_PARAMS`	Sets the `__type_params__` attribute of a function.

Adicionado na versão 3.12.

LOAD_SPECIAL

Performs special method lookup on `STACK[-1]`. If `type(STACK[-1]).__xxx__` is a method, leave `type(STACK[-1]).__xxx__; STACK[-1]` on the stack. If `type(STACK[-1]).__xxx__` is not a method, leave `STACK[-1].__xxx__; NULL` on the stack.

Adicionado na versão 3.14.

Pseudo-instructions

These opcodes do not appear in Python bytecode. They are used by the compiler but are replaced by real opcodes or removed before bytecode is generated.

SETUP_FINALLY(*target*)

Set up an exception handler for the following code block. If an exception occurs, the value stack level is restored to its current state and control is transferred to the exception handler at `target`.

SETUP_CLEANUP(*target*)

Like `SETUP_FINALLY`, but in case of an exception also pushes the last instruction (`lasti`) to the stack so that `RERAISE` can restore it. If an exception occurs, the value stack level and the last instruction on the frame are restored to their current state, and control is transferred to the exception handler at `target`.

SETUP_WITH(*target*)

Like `SETUP_CLEANUP`, but in case of an exception one more item is popped from the stack before control is transferred to the exception handler at `target`.

This variant is used in **`with`** and **`async with`** constructs, which push the return value of the context manager's **`__enter__()`** or **`__aenter__()`** to the stack.

POP_BLOCK

Marks the end of the code block associated with the last `SETUP_FINALLY`, `SETUP_CLEANUP` or `SETUP_WITH`.

JUMP
JUMP_NO_INTERRUPT

Undirected relative jump instructions which are replaced by their directed (forward/backward) counterparts by the assembler.

JUMP_IF_TRUE
JUMP_IF_FALSE

Conditional jumps which do not impact the stack. Replaced by the sequence `COPY 1`, `TO_BOOL`, `POP_JUMP_IF_TRUE/FALSE`.

LOAD_CLOSURE(*i*)

Pushes a reference to the cell contained in slot `i` of the “fast locals” storage.

Note that `LOAD_CLOSURE` is replaced with `LOAD_FAST` in the assembler.

Alterado na versão 3.13: This opcode is now a pseudo-instruction.

Opcode collections

These collections are provided for automatic introspection of bytecode instructions:

Alterado na versão 3.12: The collections now contain pseudo instructions and instrumented instructions as well. These are opcodes with values `>= MIN_PSEUDO_OPCODE` and `>= MIN_INSTRUMENTED_OPCODE`.

dis.opname

Sequence of operation names, indexable using the bytecode.

dis.opmap

Dictionary mapping operation names to bytecodes.

dis.cmp_op

Sequence of all compare operation names.

dis.hasarg

Sequence of bytecodes that use their argument.

Adicionado na versão 3.12.

dis.hasconst

Sequence of bytecodes that access a constant.

dis.hasfree

Sequence of bytecodes that access a free (closure) variable. ‘free’ in this context refers to names in the current scope that are referenced by inner scopes or names in outer scopes that are referenced from this scope. It does *not* include references to global or builtin scopes.

dis.hasname

Sequence of bytecodes that access an attribute by name.

dis.hasjump

Sequence of bytecodes that have a jump target. All jumps are relative.

Adicionado na versão 3.13.

dis.haslocal

Sequence of bytecodes that access a local variable.

dis.hascompare

Sequence of bytecodes of Boolean operations.

dis.hasexc

Sequence of bytecodes that set an exception handler.

Adicionado na versão 3.12.

dis.hasjrel

Sequence of bytecodes that have a relative jump target.

Descontinuado desde a versão 3.13: All jumps are now relative. Use `hasjump`.

dis.hasjabs

Sequence of bytecodes that have an absolute jump target.

Descontinuado desde a versão 3.13: All jumps are now relative. This list is empty.

`pickletools` — Ferramentas para desenvolvedores pickle

Código-fonte: Lib/pickletools.py
[https://github.com/python/cpython/tree/3.15/Lib/pickletools.py]

Este módulo contém várias constantes relacionadas aos detalhes íntimos do módulo **`pickle`**, alguns comentários extensos sobre a implementação e algumas funções úteis para analisar dados em conserva. O conteúdo deste módulo é útil para desenvolvedores do núcleo Python que estão trabalhando no **`pickle`**; usuários comuns do módulo **`pickle`** provavelmente não acharão o módulo **`pickletools`** relevante.

Uso na linha de comando

Adicionado na versão 3.2.

Quando chamado a partir da linha de comando, `python -m pickletools` irá desmontar o conteúdo de um ou mais arquivos pickle. Note que se você quiser ver o objeto Python armazenado serializado em pickle ao invés dos detalhes do formato pickle, você pode usar `-m pickle`. No entanto, quando o arquivo serializado em pickle que você deseja examinar vem de uma fonte não confiável, `-m pickletools` é uma opção mais segura porque não executa bytecode pickle.

Por exemplo, com uma tupla `(1, 2)` tratada com pickling no arquivo `x.pickle`:

```
$ python -m pickle x.pickle
(1, 2)

$ python -m pickletools x.pickle
    0: \x80 PROTO      3
```

```
    2: K    BININT1    1
    4: K    BININT1    2
    6: \x86 TUPLE2
    7: q    BINPUT     0
    9: .    STOP
highest protocol among opcodes = 2
```

Opções de linha de comando

-a, --annotate
: Anota cada linha com uma descrição curta do código de operação.

-o, --output=<file>
: Nome de um arquivo no qual a saída deve ser escrita.

-l, --indentlevel=<num>
: O número de espaços em branco para indentar um novo nível MARK.

-m, --memo
: Quando vários objetos são desmontados, preserva memo entre as desmontagens.

-p, --preamble=<preamble>
: Quando mais de um arquivo serializado com pickle for especificado, exibe o preâmbulo fornecido antes de cada desmontagem.

pickle_file
: Um arquivo serializado com pickle para ler, ou `-` para indicar leitura da entrada padrão.

Adicionado na versão 3.15: A saída é colorida por padrão e ela pode ser controlada usando variáveis de ambiente.

Interface programática

pickletools.dis(*pickle*, *out=None*, *memo=None*, *indentlevel=4*, *annotate=0*)

Produz uma desmontagem simbólica do pickle para o objeto arquivo ou similar *out*, tendo como padrão `sys.stdout`. *pickle* pode ser uma string ou um objeto arquivo ou similar. *memo* pode ser um dicionário Python que será usado como memo do pickle; ele pode ser usado para realizar desmontagens em várias serializações com pickle criadas pelo mesmo pickler. Níveis sucessivos, indicados por códigos de operação `MARK` no fluxo, são indentados por espaços *indentlevel*. Se um valor diferente de zero for fornecido para *annotate*, cada código de operação na saída será anotado com uma breve descrição. O valor de *annotate* é usado como uma dica para a coluna onde a anotação deve começar.

Alterado na versão 3.2: Adicionado o parâmetro *annotate*.

pickletools.genops(*pickle*)

Fornece um iterador sobre todos os códigos de operação em uma serialização com pickle, retornando uma sequência de triplos `(opcode, arg, pos)`. *opcode* é uma instância de uma classe **`OpcodeInfo`**; *arg* é o valor decodificado, como um objeto Python, do argumento do opcode; *pos* é a posição em que este código de operação está localizado. *pickle* pode ser uma string ou um objeto arquivo ou similar.

pickletools.optimize(*picklestring*)

Retorna uma nova string pickle equivalente após eliminar códigos de operação `PUT` não utilizados. O pickle otimizado é mais curto, leva menos tempo de transmissão, requer menos espaço de armazenamento e efetua unpickling com mais eficiência.

Serviços Específicos do MS Windows

Este capítulo descreve módulos que estão disponíveis somente na plataformas MS Windows.

- `msvcrt` — Rotinas úteis do runtime MS VC++
  - Operações com arquivos
  - E/S de console
  - Outras funções
- `winreg` — Acesso aos registros do Windows
  - Funções
  - Constantes
    - Constantes HKEY_*
    - Direitos de acesso
      - Específico de 64 bits
    - Tipos de valores
  - Objetos identificador de registro
- `winsound` — Interface de reprodução de som para o Windows
  - Funções
  - Constantes

`msvcrt` — Rotinas úteis do runtime MS VC++

Código-fonte: PC/msvcrtmodule.c
[https://github.com/python/cpython/tree/3.15/PC/msvcrtmodule.c]

Essas funções fornecem acesso a alguns recursos úteis em plataformas Windows. Alguns módulos de nível superior usam essas funções para construir as implementações de seus serviços no Windows. Por exemplo, o módulo `getpass` usa isso na implementação da função `getpass()`.

Mais documentação sobre essas funções pode ser encontrada na documentação da API da plataforma.

O módulo implementa as variantes normal e ampla de caracteres da API de E/S do console. A API normal lida apenas com caracteres ASCII e é de uso limitado para aplicativos internacionalizados. A API ampla de caracteres deve ser usada sempre que possível.

Disponibilidade: Windows.

Alterado na versão 3.3: As operações neste módulo agora levantam `OSError` onde `IOError` foi levantado.

Operações com arquivos

msvcrt.locking(*fd*, *mode*, *nbytes*)

Trava parte de um arquivo com base no descritor de arquivo *fd* do runtime do C. Levanta `OSError` em caso de falha. A região travada do arquivo se estende da posição atual do arquivo por *nbytes* bytes e pode continuar além do final do arquivo. *mode* deve ser uma das constantes `LK_*` listadas abaixo. Várias regiões em um arquivo podem ser travadas

ao mesmo tempo, mas não podem se sobrepor. Regiões adjacentes não são mescladas; elas devem ser destravadas individualmente.

Levanta um evento de auditoria `msvcrt.locking` com os argumentos `fd`, `mode`, `nbytes`.

msvcrt.LK_LOCK
msvcrt.LK_RLCK

Trava os bytes especificados. Se os bytes não puderem ser travados, o programa tenta novamente imediatamente após 1 segundo. Se, após 10 tentativas, os bytes não puderem ser travados, **`OSError`** é levantada.

msvcrt.LK_NBLCK
msvcrt.LK_NBRLCK

Trava os bytes especificados. Se os bytes não puderem ser travados, uma exceção **`OSError`** é levantada.

msvcrt.LK_UNLCK

Destrava os bytes especificados, que devem ter sido travados anteriormente.

msvcrt.setmode(*fd*, *flags*)

Defina o modo de conversão de final de linha para o descritor de arquivo *fd*. Para configurá-lo no modo de texto, *flags* deve ser **`os.O_TEXT`**; para binário, deve ser **`os.O_BINARY`**.

msvcrt.open_osfhandle(*handle*, *flags*)

Crie um descritor de arquivo de tempo de execução do C a partir do identificador de arquivo *handle*. O parâmetro *flags* deve ser um OU bit a bit de **`os.O_APPEND`**, **`os.O_RDONLY`**, **`os.O_TEXT`** e **`os.O_NOINHERIT`**. O descritor de arquivo retornado pode ser usado como parâmetro para **`os.fdopen()`** para criar um objeto de arquivo.

O descritor de arquivo é herdável por padrão. Passe o sinalizador **`os.O_NOINHERIT`** para torná-lo não herdável.

Levanta um evento de auditoria `msvcrt.open_osfhandle` com os argumentos `handle`, `flags`.

msvcrt.get_osfhandle(*fd*)

Retorna o identificador de arquivo para o descritor de arquivo *fd*. Levanta **`OSError`** se *fd* não for reconhecido.

Levanta um evento de auditoria `msvcrt.get_osfhandle` com o argumento `fd`.

E/S de console

msvcrt.kbhit()

Retorna um valor diferente de zero se houver uma tecla pressionada aguardando leitura. Caso contrário, retorna 0.

msvcrt.getch()

Lê um pressionamento de tecla e retorna o caractere resultante como uma sequência de bytes. Nada é ecoado no console. Essa chamada será bloqueada se um pressionamento de tecla ainda não estiver disponível, mas não esperará que `Enter` seja pressionado. Se a tecla pressionada for uma tecla de função especial, ela retornará `'\000'` ou `'\xe0'`; a próxima chamada retornará o código da chave. A tecla `Control-C` não pode ser lida com esta função.

msvcrt.getwch()

Variante com caractere largo de **`getch()`**, retornando um valor Unicode.

msvcrt.getche()

Semelhante a **`getch()`**, mas o pressionamento de tecla será repetido se representar um caractere imprimível.

msvcrt.getwche()

Variante com caractere largo de **`getche()`**, retornando um valor Unicode.

msvcrt.putch(*char*)

Imprime a string de bytes *char* no console sem armazenar em buffer.

msvcrt.putwch(*unicode_char*)

Variante com caractere largo de `putch()`, retornando um valor Unicode.

msvcrt.ungetch(*char*)

Faz com que a string de bytes *char* seja “empurrada” para o buffer do console; será o próximo caractere lido por `getch()` ou `getche()`.

msvcrt.ungetwch(*unicode_char*)

Variante com caractere largo de `ungetch()`, retornando um valor Unicode.

Outras funções

msvcrt.heapmin()

Força o heap **`malloc()`** a ser limpado e retorna os blocos não utilizados ao sistema operacional. Em caso de falha, isso levanta `OSError`.

msvcrt.set_error_mode(*mode*)

Altera o local onde o ambiente de execução C grava uma mensagem de erro para um erro que pode encerrar o programa. *mode* deve ser uma das constantes **`OUT_*`** listadas abaixo ou `REPORT_ERRMODE`. Retorna a configuração anterior ou -1 se ocorrer um erro. Disponível apenas na construção de depuração do Python.

msvcrt.OUT_TO_DEFAULT

O destino dos erros é determinado pelo tipo da aplicação. Disponível apenas na construção de depuração do Python.

msvcrt.OUT_TO_STDERR

O destino dos erros é um de erro padrão. Disponível apenas na construção de depuração do Python.

msvcrt.OUT_TO_MSGBOX

O destino dos erros é um de caixa de mensagem. Disponível apenas na construção de depuração do Python.

msvcrt.REPORT_ERRMODE

Relata o valor atual do modo de erro. Disponível apenas na construção de depuração do Python.

msvcrt.CrtSetReportMode(*type*, *mode*)

Especifica o destino ou destinos para um tipo de relatório específico gerado por **`_CrtDbgReport()`** no ambiente de execução do MS VC++. *type* deve ser uma das constantes **`CRT_*`** listadas abaixo. *mode* deve ser uma das constantes **`CRTDBG_*`** listadas abaixo. Disponível apenas na construção de depuração do Python.

msvcrt.CrtSetReportFile(*type*, *file*)

Após usar **`CrtSetReportMode()`** para especificar **`CRTDBG_MODE_FILE`**, você pode especificar o identificador do arquivo que receberá o texto da mensagem. *type* deve ser uma das constantes **`CRT_*`** listadas abaixo. *file* deve ser o identificador do arquivo que você deseja especificar. Disponível apenas na construção de depuração do Python.

msvcrt.CRT_WARN

Avisos, mensagens e informações que não exigem atenção imediata.

msvcrt.CRT_ERROR

Erros, problemas irrecuperáveis e questões que exigem atenção imediata.

msvcrt.CRT_ASSERT

Falhas de asserção.

msvcrt.CRTDBG_MODE_DEBUG

Escreve a mensagem na janela de saída do depurador.

msvcrt.CRTDBG_MODE_FILE

Escreve a mensagem em um descritor de arquivo fornecido pelo usuário. `CrtSetReportFile()` deve ser chamada para definir o arquivo ou fluxo específico a ser usado como destino.

msvcrt.CRTDBG_MODE_WNDW

Cria uma caixa de mensagem para exibir a mensagem juntamente com os botões `Abort`, `Retry` e `Ignore`.

msvcrt.CRTDBG_REPORT_MODE

Retorna o *modole* atual para o *type* especificado.

msvcrt.CRT_ASSEMBLY_VERSION

A versão do CRT Assembly, do arquivo de cabeçalho `crtassem.h`.

msvcrt.VC_ASSEMBLY_PUBLICKEYTOKEN

O token da chave pública do VC Assembly, do arquivo de cabeçalho `crtassem.h`.

msvcrt.LIBRARIES_ASSEMBLY_NAME_PREFIX

O prefixo do nome de Libraries Assembly, do arquivo de cabeçalho `crtassem.h`.

`winreg` — Acesso aos registros do Windows

Source code: PC/winreg.c [https://github.com/python/cpython/tree/3.15/PC/winreg.c]

Essas funções expõem a API de registro do Windows ao Python. Em vez de usar um inteiro como identificador de registro, um objeto identificador é usado para garantir que os identificadores sejam fechados corretamente, mesmo que o programador não os feche explicitamente.

Disponibilidade: Windows.

Alterado na versão 3.3: Várias funções neste módulo costumavam levantar um `WindowsError` , que agora é um apelido de `OSError`.

Funções

Este módulo oferece as seguintes funções:

winreg.CloseKey(*hkey*)

Fecha uma chave de registro aberta anteriormente. O argumento *hkey* especifica uma chave aberta anteriormente.

> **Nota**
>
> Se *hkey* não for fechado usando este método (ou via `hkey.Close()`), ele será fechado quando o objeto *hkey* for destruído pelo Python.

winreg.ConnectRegistry(*computer_name*, *key*)

Estabelece uma conexão com um identificador de registro predefinido em outro computador e retorna um objeto identificador.

computer_name é o nome do computador remoto, no formato `r"\\computername"`. Se `None`, o computador local será usado.

key é o identificador predefinido para conexão.

O valor de retorno é o identificador da chave aberta. Se a função falhar, uma exceção **`OSError`** será levantada.

Levanta um evento de auditoria `winreg.ConnectRegistry` com os argumentos `computer_name`, `key`.

Alterado na versão 3.3: Veja acima.

winreg.CreateKey(*key*, *sub_key*)

Cria ou abre a chave especificada, retornando um objeto identificador.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que nomeia a chave que este método abre ou cria.

Se *key* for uma das chaves predefinidas, *sub_key* poderá ser `None`. Nesse caso, o identificador retornado será o mesmo identificador de chave passado para a função.

Se a chave já existir, esta função abre a chave existente.

O valor de retorno é o identificador da chave aberta. Se a função falhar, uma exceção **`OSError`** será levantada.

Levanta um evento de auditoria `winreg.CreateKey` com os argumentos `key`, `sub_key`, `access`.

Levanta um evento de auditoria `winreg.OpenKey/result` com o argumento `key`.

Alterado na versão 3.3: Veja acima.

winreg.CreateKeyEx(*key*, *sub_key*, *reserved=0*, *access=KEY_WRITE*)

Cria ou abre a chave especificada, retornando um objeto identificador.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que nomeia a chave que este método abre ou cria.

reserved é um inteiro reservado e deve ser zero. O padrão é zero.

access é um inteiro que especifica uma máscara de acesso que descreve o acesso de segurança desejado para a chave. O padrão é **`KEY_WRITE`**. Consulte Direitos de acesso para outros valores permitidos.

Se *key* for uma das chaves predefinidas, *sub_key* poderá ser `None`. Nesse caso, o identificador retornado será o mesmo identificador de chave passado para a função.

Se a chave já existir, esta função abre a chave existente.

O valor de retorno é o identificador da chave aberta. Se a função falhar, uma exceção **`OSError`** será levantada.

Levanta um evento de auditoria `winreg.CreateKey` com os argumentos `key`, `sub_key`, `access`.

Levanta um evento de auditoria `winreg.OpenKey/result` com o argumento `key`.

Adicionado na versão 3.2.

Alterado na versão 3.3: Veja acima.

winreg.DeleteKey(*key*, *sub_key*)

Exclui a chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que deve ser uma subchave da chave identificada pelo parâmetro *key*. Este valor não pode ser `None`, e a chave não pode ter subchaves.

Este método não pode excluir chaves com subchaves.

Se o método for bem-sucedido, a chave inteira, incluindo todos os seus valores, será removida. Se o método falhar, uma exceção **`OSError`** será levantada.

Levanta um evento de auditoria `winreg.DeleteKey` com os argumentos `key`, `sub_key`, `access`.

Alterado na versão 3.3: Veja acima.

winreg.DeleteKeyEx(*key*, *sub_key*, *access=KEY_WOW64_64KEY*, *reserved=0*)

Exclui a chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que deve ser uma subchave da chave identificada pelo parâmetro *key*. Este valor não pode ser `None`, e a chave não pode ter subchaves.

reserved é um inteiro reservado e deve ser zero. O padrão é zero.

access é um inteiro que especifica uma máscara de acesso que descreve o acesso de segurança desejado para a chave. O padrão é **`KEY_WOW64_64KEY`**. No Windows de 32 bits, as constantes WOW64 são ignoradas. Consulte Direitos de acesso para outros valores permitidos.

Este método não pode excluir chaves com subchaves.

Se o método for bem-sucedido, a chave inteira, incluindo todos os seus valores, será removida. Se o método falhar, uma exceção **`OSError`** será levantada.

Em versões do Windows sem suporte, **`NotImplementedError`** é levantada.

Levanta um evento de auditoria `winreg.DeleteKey` com os argumentos `key`, `sub_key`, `access`.

Adicionado na versão 3.2.

Alterado na versão 3.3: Veja acima.

winreg.DeleteTree(*key*, *sub_key=None*)

Deletes the specified key and all its subkeys and values recursively.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key is a string that names the subkey to delete. If `None`, deletes all subkeys and values of the specified key.

This function deletes a key and all its descendants. If *sub_key* is `None`, all subkeys and values of the specified key are deleted.

Raises an auditing event `winreg.DeleteTree` with arguments `key`, `sub_key`.

Adicionado na versão 3.15.

winreg.DeleteValue(*key*, *value*)

Remove um valor nomeado de uma chave de registro.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

value é uma string que identifica o valor a ser removido.

Levanta um evento de auditoria `winreg.DeleteValue` com os argumentos `key`, `value`.

winreg.EnumKey(*key*, *index*)

Enumera subchaves de uma chave de registro aberta, retornando uma string.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

index é um inteiro que identifica o índice da chave a ser recuperada.

A função recupera o nome de uma subchave cada vez que é chamada. Normalmente, ela é chamada repetidamente até que uma exceção `OSError` seja levantada, indicando que não há mais valores disponíveis.

Levanta um evento de auditoria `winreg.EnumKey` com os argumentos `key`, `index`.

Alterado na versão 3.3: Veja acima.

winreg.EnumValue(*key*, *index*)

Enumera valores de uma chave de registro aberta, retornando uma tupla.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

index é um inteiro que identifica o índice do valor a ser recuperado.

A função recupera o nome de uma subchave cada vez que é chamada. Normalmente, ela é chamada repetidamente, até que uma exceção `OSError` seja levantada, indicando que não há mais valores.

O resultado é uma tupla de 3 itens:

Índice	Significado
`0`	Uma string que identifica o nome do valor

Índice	Significado
1	Um objeto que contém os dados de valor e cujo tipo depende do tipo de registro subjacente
2	Um inteiro que identifica o tipo dos dados do valor (veja a tabela na documentação para **SetValueEx()**)

Levanta um evento de auditoria `winreg.EnumValue` com os argumentos `key`, `index`.

Alterado na versão 3.3: Veja acima.

winreg.ExpandEnvironmentStrings(*str*)

Expande os espaços reservados de posição de variáveis de ambiente `%NAME%` em strings como **REG_EXPAND_SZ**:

```
>>> ExpandEnvironmentStrings('%windir%')
'C:\\Windows'
```

Levanta um evento de auditoria `winreg.ExpandEnvironmentStrings` com o argumento `str`.

winreg.FlushKey(*key*)

Grava todos os atributos de uma chave no registro.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

Não é necessário chamar **FlushKey()** para alterar uma chave. As alterações do registro são descarregadas no disco pelo registro usando seu limpador lento. As alterações do registro também são descarregadas no disco no desligamento do sistema. Ao contrário de **CloseKey()**, o método **FlushKey()** retorna somente quando todos os dados foram gravados no registro. Uma aplicação só deve chamar **FlushKey()** se precisar de certeza absoluta de que as alterações do registro estão no disco.

Nota

Se você não sabe se uma chamada **`FlushKey()`** é necessária, provavelmente não é.

winreg.LoadKey(*key*, *sub_key*, *file_name*)

Cria uma subchave sob a chave especificada e armazena informações de registro de um arquivo especificado nessa subchave.

key é um identificador retornado por **`ConnectRegistry()`** ou uma das constantes **`HKEY_USERS`** ou **`HKEY_LOCAL_MACHINE`**.

sub_key é uma string que identifica a subchave a ser carregada.

file_name é o nome do arquivo do qual serão carregados os dados do registro. Este arquivo deve ter sido criado com a função **`SaveKey()`**. No sistema de arquivos da tabela de alocação de arquivos (FAT), o nome do arquivo não pode ter extensão.

Uma chamada para **`LoadKey()`** falhará se o processo de chamada não tiver o privilégio **`SE_RESTORE_PRIVILEGE`**. Observe que privilégios são diferentes de permissões — consulte a documentação do RegLoadKey [https://msdn.microsoft.com/en-us/library/ms724889%28v=VS.85%29.aspx] para mais detalhes.

Se *key* for um identificador retornado por **`ConnectRegistry()`**, então o caminho especificado em *file_name* será relativo ao computador remoto.

Levanta um evento de auditoria `winreg.LoadKey` com os argumentos `key`, `sub_key`, `file_name`.

winreg.OpenKey(*key*, *sub_key*, *reserved=0*, *access=KEY_READ*)
winreg.OpenKeyEx(*key*, *sub_key*, *reserved=0*, *access=KEY_READ*)

Abre a chave especificada, retornando um objeto identificador.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que identifica a sub_key a ser aberta.

reserved é um inteiro reservado e deve ser zero. O padrão é zero.

access é um inteiro que especifica uma máscara de acesso que descreve o acesso de segurança desejado para a chave. O padrão é **KEY_READ**. Consulte Direitos de acesso para outros valores permitidos.

O resultado é um novo identificador para a chave especificada.

Se a função falhar, **OSError** será levantada.

Levanta um evento de auditoria `winreg.OpenKey` com os argumentos `key`, `sub_key`, `access`.

Levanta um evento de auditoria `winreg.OpenKey/result` com o argumento `key`.

Alterado na versão 3.2: Permite o uso de argumentos nomeados.

Alterado na versão 3.3: Veja acima.

winreg.QueryInfoKey(*key*)

Retorna informações sobre uma chave, como uma tupla.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

O resultado é uma tupla de 3 itens:

Índice	Significado
`0`	Um inteiro que fornece o número de subchaves que esta chave possui.

Índice	Significado
`1`	Um inteiro que fornece o número de valores que esta chave possui.
`2`	Um número inteiro que indica quando a chave foi modificada pela última vez (se disponível) em centenas de nanossegundos desde 1º de janeiro de 1601.

Levanta um evento de auditoria `winreg.QueryInfoKey` com o argumento `key`.

winreg.QueryValue(*key*, *sub_key*)

Recupera o valor sem nome de uma chave, como uma string.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que contém o nome da subchave à qual o valor está associado. Se este parâmetro for `None` ou estiver vazio, a função recupera o valor definido pelo método **`SetValue()`** para a chave identificada por *key*.

Os valores no registro têm componentes de nome, tipo e dados. Este método recupera os dados do primeiro valor de uma chave que possui um nome `NULL`. Mas a chamada de API subjacente não retorna o tipo, portanto, sempre use **`QueryValueEx()`** se possível.

Levanta um evento de auditoria `winreg.QueryValue` com os argumentos `key`, `sub_key`, `value_name`.

winreg.QueryValueEx(*key*, *value_name*)

Recupera o tipo e os dados de um nome de valor especificado associado a uma chave de registro aberta.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

value_name é uma string que indica o valor a ser consultado.

O resultado é uma tupla de 2 itens:

Índice	Significado
`0`	O valor do item do registro.
`1`	Um inteiro que fornece o tipo de registro para este valor (veja a tabela na documentação para **`SetValueEx()`**)

Levanta um evento de auditoria `winreg.QueryValue` com os argumentos `key`, `sub_key`, `value_name`.

winreg.SaveKey(*key*, *file_name*)

Salva a chave especificada e todas as suas subchaves no arquivo especificado.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

file_name é o nome do arquivo para salvar os dados do registro. Este arquivo não pode existir. Se este nome de arquivo incluir uma extensão, ele não poderá ser usado em sistemas de arquivos FAT (tabela de alocação de arquivos) pelo método **`LoadKey()`**.

Se *key* representa uma chave em um computador remoto, o caminho descrito por *file_name* é relativo ao computador remoto. O chamador deste método deve possuir o privilégio de segurança **SeBackupPrivilege**. Observe que privilégios são diferentes de permissões — consulte a documentação Conflitos entre Direitos e Permissões do Usuário [https://msdn.microsoft.com/en-us/library/ms724878%28v=VS.85%29.aspx] para obter mais detalhes.

Esta função passa `NULL` para *security_attributes* para a API.

Levanta um evento de auditoria `winreg.SaveKey` com os argumentos `key`, `file_name`.

winreg.SetValue(*key*, *sub_key*, *type*, *value*)

Associa um valor a uma chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

sub_key é uma string que nomeia a subchave à qual o valor está associado.

type é um inteiro que especifica o tipo dos dados. Atualmente, deve ser **`REG_SZ`**, o que significa que apenas strings são suportadas. Use a função **`SetValueEx()`** para suporte a outros tipos de dados.

valor é uma string que especifica o novo valor.

Se a chave especificada pelo parâmetro *sub_key* não existir, a função SetValue a criará.

O comprimento dos valores é limitado pela memória disponível. Valores longos (mais de 2048 bytes) devem ser armazenados como arquivos com os nomes dos arquivos armazenados no registro de configuração. Isso ajuda o registro a funcionar com eficiência.

A chave identificada pelo parâmetro *key* deve ter sido aberta com acesso **`KEY_SET_VALUE`**.

Levanta um evento de auditoria `winreg.SetValue` com os argumentos `key`, `sub_key`, `type`, `value`.

winreg.SetValueEx(*key*, *value_name*, *reserved*, *type*, *value*)

Armazena dados no campo de valor de uma chave de registro aberta.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

value_name é uma string que nomeia a subchave à qual o valor está associado.

reserved pode ser qualquer coisa – zero é sempre passado para a API.

type é um inteiro que especifica o tipo dos dados. Consulte Tipos de valor para os tipos disponíveis.

valor é uma string que especifica o novo valor.

Este método também pode definir informações adicionais de valor e tipo para a chave especificada. A chave identificada pelo parâmetro key deve ter sido aberta com o acesso **`KEY_SET_VALUE`**.

Para abrir a chave, use os métodos **`CreateKey()`** ou **`OpenKey()`**.

O comprimento dos valores é limitado pela memória disponível. Valores longos (mais de 2048 bytes) devem ser armazenados como arquivos com os nomes dos arquivos armazenados no registro de configuração. Isso ajuda o registro a funcionar com eficiência.

Levanta um evento de auditoria `winreg.SetValue` com os argumentos `key`, `sub_key`, `type`, `value`.

winreg.DisableReflectionKey(*key*)

Desabilita a reflexão do registro para processos de 32 bits em execução em um sistema operacional de 64 bits.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

Geralmente levantará **`NotImplementedError`** se executado em um sistema operacional de 32 bits.

Se a chave não estiver na lista de reflexão, a função será bem-sucedida, mas não terá efeito. Desabilitar a reflexão para uma chave não afeta a reflexão de nenhuma subchave.

Levanta um evento de auditoria `winreg.DisableReflectionKey` com o argumento `key`.

winreg.EnableReflectionKey(*key*)

Restaura a reflexão do registro para a chave desabilitada especificada.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

Geralmente levantará **`NotImplementedError`** se executado em um sistema operacional de 32 bits.

Restaura a reflexão de uma chave não afeta a reflexão de nenhuma subchave.

Levanta um evento de auditoria `winreg.EnableReflectionKey` com o argumento `key`.

winreg.QueryReflectionKey(*key*)

Determina o estado de reflexão para a chave especificada.

key é uma chave já aberta, ou uma das constantes predefinidas HKEY_*.

Retorna `True` se a reflexão estiver desabilitada.

Geralmente levantará **`NotImplementedError`** se executado em um sistema operacional de 32 bits.

Levanta um evento de auditoria `winreg.QueryReflectionKey` com o argumento `key`.

Constantes

As seguintes constantes são definidas para uso em muitas funções do módulo **`winreg`**.

Constantes HKEY_*

winreg.HKEY_CLASSES_ROOT

Entradas de registro subordinadas a esta chave definem tipos (ou classes) de documentos e as propriedades associadas a esses tipos. Aplicações Shell e COM utilizam as informações armazenadas sob esta chave.

winreg.HKEY_CURRENT_USER

As entradas de registro subordinadas a esta chave definem as preferências do usuário atual. Essas preferências incluem as configurações de variáveis de ambiente, dados sobre grupos de programas, cores, impressoras, conexões de rede e preferências de aplicações.

winreg.HKEY_LOCAL_MACHINE

As entradas de registro subordinadas a esta chave definem o estado físico do computador, incluindo dados sobre o tipo de barramento, memória do sistema e hardware e software instalados.

winreg.HKEY_USERS

As entradas de registro subordinadas a esta chave definem a configuração de usuário padrão para novos usuários no computador local e a configuração de usuário para o usuário atual.

winreg.HKEY_PERFORMANCE_DATA

Entradas de registro subordinadas a esta chave permitem que você acesse dados de desempenho. Os dados não são armazenados no registro; as funções de registro fazem com que o sistema colete os dados de sua fonte.

winreg.HKEY_CURRENT_CONFIG

Contém informações sobre o perfil de hardware atual do sistema de computador local.

winreg.HKEY_DYN_DATA

Esta chave não é usada em versões do Windows posteriores à 98.

Direitos de acesso

Para obter mais informações, consulte Segurança e acesso a chaves do registro [https://msdn.microsoft.com/en-us/library/ms724878%28v=VS.85%29.aspx].

winreg.KEY_ALL_ACCESS
: Combina os direitos de acesso STANDARD_RIGHTS_REQUIRED, `KEY_QUERY_VALUE`, `KEY_SET_VALUE`, `KEY_CREATE_SUB_KEY`, `KEY_ENUMERATE_SUB_KEYS`, `KEY_NOTIFY` e `KEY_CREATE_LINK`.

winreg.KEY_WRITE
: Combina os direitos de acesso STANDARD_RIGHTS_WRITE, `KEY_SET_VALUE` e `KEY_CREATE_SUB_KEY`.

winreg.KEY_READ
: Combina os valores STANDARD_RIGHTS_READ, `KEY_QUERY_VALUE`, `KEY_ENUMERATE_SUB_KEYS` e `KEY_NOTIFY`.

winreg.KEY_EXECUTE
: Equivalente a `KEY_READ`.

winreg.KEY_QUERY_VALUE
: Necessário para consultar os valores de uma chave de registro.

winreg.KEY_SET_VALUE
: Necessário para criar, excluir ou definir um valor de registro.

winreg.KEY_CREATE_SUB_KEY
: Necessário para criar uma subchave de uma chave de registro.

winreg.KEY_ENUMERATE_SUB_KEYS
: Necessário para enumerar as subchaves de uma chave de registro.

winreg.KEY_NOTIFY
: Necessário para solicitar notificações de alteração para uma chave de registro ou para subchaves de uma chave de registro.

winreg.KEY_CREATE_LINK
Reservado para uso pelo sistema.

Específico de 64 bits

Para obter mais informações, consulte Acessando uma exibição alternativa do Registro [https://msdn.microsoft.com/en-us/library/aa384129(v=VS.85).aspx].

winreg.KEY_WOW64_64KEY
Indica que uma aplicação no Windows 64 bits deve operar na visualização de registro de 64 bits. No Windows de 32 bits, esta constante é ignorada.

winreg.KEY_WOW64_32KEY
Indica que uma aplicação no Windows 64 bits deve operar na visualização de registro de 32 bits. No Windows de 32 bits, esta constante é ignorada.

Tipos de valores

Para obter mais informações, consulte Tipos de valor do Registro [https://msdn.microsoft.com/en-us/library/ms724884%28v=VS.85%29.aspx].

winreg.REG_BINARY
Dados binários em qualquer formato.

winreg.REG_DWORD
Número de 32 bits.

winreg.REG_DWORD_LITTLE_ENDIAN
Um número de 32 bits no formato little-endian. Equivalente a `REG_DWORD`.

winreg.REG_DWORD_BIG_ENDIAN
Um número de 32 bits no formato big-endian.

winreg.REG_EXPAND_SZ

String terminada em nulo contendo referências a variáveis de ambiente (`%PATH%`).

winreg.REG_LINK
Um link simbólico Unicode.

winreg.REG_MULTI_SZ
Uma sequência de strings terminadas em nulo, terminadas por dois caracteres nulos. (O Python lida com essa terminação automaticamente.)

winreg.REG_NONE
Nenhum tipo de valor definido.

winreg.REG_QWORD
Um número de 64 bits.

Adicionado na versão 3.6.

winreg.REG_QWORD_LITTLE_ENDIAN
Um número de 64 bits no formato little-endian. Equivalente a **`REG_QWORD`**.

Adicionado na versão 3.6.

winreg.REG_RESOURCE_LIST
Uma lista de recursos de driver de dispositivo.

winreg.REG_FULL_RESOURCE_DESCRIPTOR
Uma configuração de hardware.

winreg.REG_RESOURCE_REQUIREMENTS_LIST
Uma lista de recursos de hardware.

winreg.REG_SZ
Uma string terminada em nulo.

Objetos identificador de registro

Este objeto encapsula um objeto HKEY do Windows, fechando-o automaticamente quando o objeto é destruído. Para garantir a limpeza, você pode chamar o método `Close()` no objeto ou a função `CloseKey()`.

Todas as funções de registro neste módulo retornam um destes objetos.

Todas as funções de registro neste módulo que aceitam um objeto identificador também aceitam um inteiro; no entanto, o uso do objeto identificador é incentivado.

Objetos identificador fornecem semântica para `__bool__()` – portanto

```
if handle:
    print("Sim")
```

vai exibir `Sim` se o identificador for válido no momento (não foi fechado ou desanexado).

The object also support equality comparison semantics, so handle objects will compare equal if they both reference the same underlying Windows handle value. Closed handle objects (those with a handle value of zero) always compare equal.

Objetos identificador podem ser convertidos em um inteiro (por exemplo, usando a função embutida `int()`), caso em que o valor do identificador do Windows subjacente é retornado. Você também pode usar o método `Detach()` para retornar o identificador inteiro e também desconectar o identificador do Windows do objeto identificador.

PyHKEY.Close()

Fecha o identificador subjacente do Windows.

Se o identificador já estiver fechado, nenhum erro será levantado.

PyHKEY.Detach()

Separa o identificador do Windows do objeto identificador.

O resultado é um inteiro que contém o valor do identificador antes de ser destacado. Se o identificador já estiver destacado ou fechado, isso retornará zero.

Após chamar esta função, o identificador é efetivamente invalidado, mas não é fechado. Você chamaria esta função quando precisasse que o identificador subjacente de Win32 existisse além do tempo de vida do objeto identificador.

Levanta um evento de auditoria `winreg.PyHKEY.Detach` com o argumento `key`.

PyHKEY.__enter__()

PyHKEY.__exit__(**exc_info*)

O objeto HKEY implementa `__enter__()` e `__exit__()` e, portanto, oferece suporte ao protocolo de contexto para a instrução `with`:

```
with OpenKey(HKEY_LOCAL_MACHINE, "foo") as key:
    ...  # trabalha com a chave "key"
```

vai fechar automaticamente *key* quando o controle sair do bloco `with`.

Alterado na versão 3.15: Handle objects are now compared by their underlying Windows handle value instead of object identity for equality comparisons.

`winsound` — Interface de reprodução de som para o Windows

Código-fonte: PC/winsound.c
[https://github.com/python/cpython/tree/3.15/PC/winsound.c]

O módulo **`winsound`** fornece acesso ao mecanismo básico de reprodução de som fornecido pelas plataformas Windows. Inclui funções e várias constantes.

Disponibilidade: Windows.

Funções

winsound.Beep(*frequency*, *duration*)

Emite um bipe no alto-falante do PC. O parâmetro *frequency* especifica a frequência, em hertz, do som e deve estar no intervalo de 37 a 32.767. O parâmetro *duration* especifica o número de milissegundos que o som deve durar. Se o sistema não conseguir emitir um bipe no alto-falante, **`RuntimeError`** é levantado.

winsound.PlaySound(*sound*, *flags*)

Chama a função subjacente **`PlaySound()`** da API da plataforma. O parâmetro *sound* pode ser um nome de arquivo, um apelido de som do sistema, dados de áudio como um objeto bytes ou similar ou `None`. Sua interpretação depende do valor de *flags*, que pode ser uma combinação OU bit a bit das constantes descritas abaixo. Se o parâmetro *sound* for `None`, qualquer som de forma de onda em execução no momento será interrompido. Se o sistema indicar um erro, **`RuntimeError`** será levantada.

winsound.MessageBeep(*type=MB_OK*)

Chama a função subjacente **`MessageBeep()`** da API da plataforma. Isso reproduz um som conforme especificado no registro. O argumento *type* especifica qual som reproduzir; os valores possíveis são `-1`, `MB_ICONASTERISK`, `MB_ICONEXCLAMATION`, `MB_ICONHAND`, `MB_ICONQUESTION` e `MB_OK`, todos descritos abaixo. O valor `-1` produz um “bipe simples”; este é o fallback final se um som não puder ser reproduzido de outra forma. Se o sistema indicar um erro, **`RuntimeError`** será levantada.

Constantes

winsound.SND_FILENAME

O parâmetro *sound* é o nome de um arquivo WAV. Não use com **`SND_ALIAS`**.

winsound.SND_ALIAS

O parâmetro *sound* é um nome de associação de som do registro. Se o registro não contiver tal nome, reproduza o som padrão do sistema, a menos que **`SND_NODEFAULT`** também seja especificado. Se nenhum som padrão for registrado, levanta **`RuntimeError`**. Não use com **`SND_FILENAME`**.

Todos os sistemas Win32 oferecem suporte a pelo menos o seguinte; a maioria dos sistemas oferecem suporte a muito mais:

name **`PlaySound()`**	**Nome do som correspondente no Painel de Controle**
`'SystemAsterisk'`	Asterisco
`'SystemExclamation'`	Exclamação
`'SystemExit'`	Logoff do Windows

name `PlaySound()`	**Nome do som correspondente no Painel de Controle**
`'SystemHand'`	Parada crítica
`'SystemQuestion'`	Pergunta

Por exemplo:

```
import winsound
# Reproduz o som de logoff do Windows.
winsound.PlaySound("SystemExit", winsound.SND_ALIAS)

# Provavelmente reproduz o som padrão do Windows, se algum
# estiver registrado (porque "*" provavelmente não é o nome r
# de algum som).
winsound.PlaySound("*", winsound.SND_ALIAS)
```

winsound.SND_LOOP

Reproduz o som repetidamente. O sinalizador `SND_ASYNC` também deve ser usado para evitar bloqueio. Não pode ser usado com `SND_MEMORY`.

winsound.SND_MEMORY

O parâmetro *sound* para `PlaySound()` é uma imagem de memória de um arquivo WAV, como um objeto bytes ou similar.

Nota

Este módulo não oferece suporte à reprodução de uma imagem de memória de forma assíncrona, portanto, uma combinação deste sinalizador e `SND_ASYNC` levantará `RuntimeError`.

winsound.SND_PURGE

Para de reproduzir todas as instâncias do som especificado.

Nota

Não há suporte a este sinalizador em plataformas Windows modernas.

winsound.SND_ASYNC

Retorna imediatamente, permitindo que os sons sejam reproduzidos de forma assíncrona.

winsound.SND_NODEFAULT

Se o som especificado não for encontrado, o som padrão do sistema não será reproduzido.

winsound.SND_NOSTOP

Não interrompe os sons que estão sendo reproduzidos.

winsound.SND_NOWAIT

Retorna imediatamente se o driver de som estiver ocupado.

Nota

Não há suporte a este sinalizador em plataformas Windows modernas.

winsound.SND_APPLICATION

O parâmetro *sound* é um apelido específico da aplicação no registro. Este sinalizador pode ser combinado com o sinalizador `SND_ALIAS` para especificar um apelido de som definido pela aplicação.

winsound.SND_SENTRY

Aciona um evento SoundSentry quando o som é reproduzido.

Adicionado na versão 3.14.

winsound.SND_SYNC

O som é reproduzido de forma sincronizada. Este é o comportamento padrão.

Adicionado na versão 3.14.

winsound.SND_SYSTEM

Atribui o som à sessão de áudio para sons de notificação do sistema.

Adicionado na versão 3.14.

winsound.MB_ICONASTERISK

Reproduz o som `SystemDefault`.

winsound.MB_ICONEXCLAMATION

Reproduz o som `SystemExclamation`.

winsound.MB_ICONHAND

Reproduz o som `SystemHand`.

winsound.MB_ICONQUESTION

Reproduz o som `SystemQuestion`.

winsound.MB_OK

Reproduz o som `SystemDefault`.

winsound.MB_ICONERROR

Reproduz o som `SystemHand`.

Adicionado na versão 3.14.

winsound.MB_ICONINFORMATION

Reproduz o som `SystemDefault`.

Adicionado na versão 3.14.

winsound.MB_ICONSTOP

Reproduz o som `SystemHand`.

Adicionado na versão 3.14.

winsound.MB_ICONWARNING

Reproduz o som `SystemExclamation`.

Adicionado na versão 3.14.

Serviços específicos do Unix

Os módulos descritos neste capítulo fornecem interfaces para recursos que são exclusivos do sistema operacional Unix ou, em alguns casos, para algumas ou várias variantes do mesmo. Aqui está uma visão geral:

- `shlex` — Simple lexical analysis
  - shlex Objects
  - Regras de análise
  - Improved Compatibility with Shells
- `posix` — As chamadas de sistema mais comuns do POSIX
  - Suporte a arquivos grandes
  - Conteúdo notável do módulo
- `pwd` — A senha do banco de dados
- `grp` — O banco de dados de grupos
- `termios` — Controle de tty no estilo POSIX
  - Exemplo
- `tty` — Funções de controle de terminal
- `pty` — Utilitários de pseudoterminal
  - Exemplo
- `fcntl` — as chamadas de sistema `fcntl` e `ioctl`
- `resource` — Informação de uso de recursos
  - Limites de recursos
  - Uso de recursos
- `syslog` — Rotinas da biblioteca syslog do Unix
  - Exemplos
    - Exemplo simples

`shlex` — Simple lexical analysis

Código-fonte: Lib/shlex.py [https://github.com/python/cpython/tree/3.15/Lib/shlex.py]

The `shlex` class makes it easy to write lexical analyzers for simple syntaxes resembling that of the Unix shell. This will often be useful for writing minilanguages, (for example, in run control files for Python applications) or for parsing quoted strings.

The `shlex` module defines the following functions:

shlex.split(*s*, *comments=False*, *posix=True*)

Split the string *s* using shell-like syntax. If *comments* is `False` (the default), the parsing of comments in the given string will be disabled (setting the `commenters` attribute of the `shlex` instance to the empty string). This function operates in POSIX mode by default, but uses non-POSIX mode if the *posix* argument is false.

Alterado na versão 3.12: Passing `None` for *s* argument now raises an exception, rather than reading `sys.stdin`.

shlex.join(*split_command*)

Concatenate the tokens of the list *split_command* and return a string. This function is the inverse of `split()`.

```
>>> from shlex import join
>>> print(join(['echo', '-n', 'Multiple words']))
echo -n 'Multiple words'
```

The returned value is shell-escaped to protect against injection vulnerabilities (see `quote()`).

Adicionado na versão 3.8.

shlex.quote(*s*)

Return a shell-escaped version of the string *s*. The returned value is a string that can safely be used as one token in a shell command line, for cases where you cannot use a list.

Aviso

The `shlex` module is **only designed for Unix shells**.

The **`quote()`** function is not guaranteed to be correct on non-POSIX compliant shells or shells from other operating systems such as Windows. Executing commands quoted by this module on such shells can open up the possibility of a command injection vulnerability.

Consider using functions that pass command arguments with lists such as **`subprocess.run()`** with `shell=False`.

Este idioma seria inseguro:

```
>>> filename = 'somefile; rm -rf ~'
>>> command = 'ls -l {}'.format(filename)
>>> print(command)  # executed by a shell: boom!
ls -l somefile; rm -rf ~
```

`quote()` lets you plug the security hole:

```
>>> from shlex import quote
>>> command = 'ls -l {}'.format(quote(filename))
>>> print(command)
ls -l 'somefile; rm -rf ~'
>>> remote_command = 'ssh home {}'.format(quote(command))
>>> print(remote_command)
ssh home 'ls -l '"'"'somefile; rm -rf ~'"'"'
```

The quoting is compatible with UNIX shells and with **`split()`**:

```
>>> from shlex import split
>>> remote_command = split(remote_command)
>>> remote_command
['ssh', 'home', "ls -l 'somefile; rm -rf ~'"]
>>> command = split(remote_command[-1])
```

```
>>> command
['ls', '-l', 'somefile; rm -rf ~']
```

Adicionado na versão 3.3.

The **shlex** module defines the following class:

class shlex.shlex(*instream=None*, *infile=None*, *posix=False*, *punctuation_chars=False*)

A **shlex** instance or subclass instance is a lexical analyzer object. The initialization argument, if present, specifies where to read characters from. It must be a file-/stream-like object with **read()** and **readline()** methods, or a string. If no argument is given, input will be taken from `sys.stdin`. The second optional argument is a filename string, which sets the initial value of the **infile** attribute. If the *instream* argument is omitted or equal to `sys.stdin`, this second argument defaults to "stdin". The *posix* argument defines the operational mode: when *posix* is not true (default), the **shlex** instance will operate in compatibility mode. When operating in POSIX mode, **shlex** will try to be as close as possible to the POSIX shell parsing rules. The *punctuation_chars* argument provides a way to make the behaviour even closer to how real shells parse. This can take a number of values: the default value, `False`, preserves the behaviour seen under Python 3.5 and earlier. If set to `True`, then parsing of the characters `();<>|&` is changed: any run of these characters (considered punctuation characters) is returned as a single token. If set to a non-empty string of characters, those characters will be used as the punctuation characters. Any characters in the **wordchars** attribute that appear in *punctuation_chars* will be removed from **wordchars**. See Improved Compatibility with Shells for more information. *punctuation_chars* can be set only upon **shlex** instance creation and can't be modified later.

Alterado na versão 3.6: The *punctuation_chars* parameter was added.

Ver também

Module configparser

Parser for configuration files similar to the Windows `.ini` files.

shlex Objects

A `shlex` instance has the following methods:

shlex.get_token()

Return a token. If tokens have been stacked using `push_token()`, pop a token off the stack. Otherwise, read one from the input stream. If reading encounters an immediate end-of-file, `eof` is returned (the empty string (`''`) in non-POSIX mode, and `None` in POSIX mode).

shlex.push_token(*str*)

Push the argument onto the token stack.

shlex.read_token()

Read a raw token. Ignore the pushback stack, and do not interpret source requests. (This is not ordinarily a useful entry point, and is documented here only for the sake of completeness.)

shlex.sourcehook(*filename*)

When `shlex` detects a source request (see `source` below) this method is given the following token as argument, and expected to return a tuple consisting of a filename and an open file-like object.

Normally, this method first strips any quotes off the argument. If the result is an absolute pathname, or there was no previous source request in effect, or the previous source was a stream (such as `sys.stdin`), the result is left alone. Otherwise, if the result is a relative pathname, the directory part of the name of the file immediately before it on the source inclusion stack is prepended (this behavior is like the way the C preprocessor handles `#include "file.h"`).

The result of the manipulations is treated as a filename, and returned as the first component of the tuple, with `open()` called on it to yield the

second component. (Note: this is the reverse of the order of arguments in instance initialization!)

This hook is exposed so that you can use it to implement directory search paths, addition of file extensions, and other namespace hacks. There is no corresponding ‘close’ hook, but a shlex instance will call the `close()` method of the sourced input stream when it returns EOF.

For more explicit control of source stacking, use the `push_source()` and `pop_source()` methods.

shlex.push_source(*newstream*, *newfile=None*)

Push an input source stream onto the input stack. If the filename argument is specified it will later be available for use in error messages. This is the same method used internally by the `sourcehook()` method.

shlex.pop_source()

Pop the last-pushed input source from the input stack. This is the same method used internally when the lexer reaches EOF on a stacked input stream.

shlex.error_leader(*infile=None*, *lineno=None*)

This method generates an error message leader in the format of a Unix C compiler error label; the format is `'"%s", line %d: '`, where the `%s` is replaced with the name of the current source file and the `%d` with the current input line number (the optional arguments can be used to override these).

This convenience is provided to encourage **`shlex`** users to generate error messages in the standard, parseable format understood by Emacs and other Unix tools.

Instances of `shlex` subclasses have some public instance variables which either control lexical analysis or can be used for debugging:

shlex.commenters

The string of characters that are recognized as comment beginners. All characters from the comment beginner to end of line are ignored. Includes just `'#'` by default.

shlex.wordchars

The string of characters that will accumulate into multi-character tokens. By default, includes all ASCII alphanumerics and underscore. In POSIX mode, the accented characters in the Latin-1 set are also included. If **`punctuation_chars`** is not empty, the characters `~-./*?=`, which can appear in filename specifications and command line parameters, will also be included in this attribute, and any characters which appear in `punctuation_chars` will be removed from `wordchars` if they are present there. If **`whitespace_split`** is set to `True`, this will have no effect.

shlex.whitespace

Characters that will be considered whitespace and skipped. Whitespace bounds tokens. By default, includes space, tab, linefeed and carriage-return.

shlex.escape

Characters that will be considered as escape. This will be only used in POSIX mode, and includes just `'\'` by default.

shlex.quotes

Characters that will be considered string quotes. The token accumulates until the same quote is encountered again (thus, different quote types protect each other as in the shell.) By default, includes ASCII single and double quotes.

shlex.escapedquotes

Characters in **`quotes`** that will interpret escape characters defined in **`escape`**. This is only used in POSIX mode, and includes just `'"'` by default.

shlex.whitespace_split

If `True`, tokens will only be split in whitespaces. This is useful, for example, for parsing command lines with **`shlex`**, getting tokens in a similar way to shell arguments. When used in combination with **`punctuation_chars`**, tokens will be split on whitespace in addition to those characters.

Alterado na versão 3.8: The **`punctuation_chars`** attribute was made compatible with the **`whitespace_split`** attribute.

shlex.infile

The name of the current input file, as initially set at class instantiation time or stacked by later source requests. It may be useful to examine this when constructing error messages.

shlex.instream

The input stream from which this **`shlex`** instance is reading characters.

shlex.source

This attribute is `None` by default. If you assign a string to it, that string will be recognized as a lexical-level inclusion request similar to the `source` keyword in various shells. That is, the immediately following token will be opened as a filename and input will be taken from that stream until EOF, at which point the **`close()`** method of that stream will be called and the input source will again become the original input stream. Source requests may be stacked any number of levels deep.

shlex.debug

If this attribute is numeric and `1` or more, a **`shlex`** instance will print verbose progress output on its behavior. If you need to use this, you can read the module source code to learn the details.

shlex.lineno

Source line number (count of newlines seen so far plus one).

shlex.token

The token buffer. It may be useful to examine this when catching exceptions.

shlex.eof

Token used to determine end of file. This will be set to the empty string (`''`), in non-POSIX mode, and to `None` in POSIX mode.

shlex.punctuation_chars

A read-only property. Characters that will be considered punctuation. Runs of punctuation characters will be returned as a single token. However, note that no semantic validity checking will be performed: for example, '>>>' could be returned as a token, even though it may not be recognised as such by shells.

Adicionado na versão 3.6.

Regras de análise

When operating in non-POSIX mode, **`shlex`** will try to obey the following rules.

- Quote characters are not recognized within words (`Do"Not"Separate` is parsed as the single word `Do"Not"Separate`);
- Escape characters are not recognized;
- Enclosing characters in quotes preserve the literal value of all characters within the quotes;
- Closing quotes separate words (`"Do"Separate` is parsed as `"Do"` and `Separate`);
- If **`whitespace_split`** is `False`, any character not declared to be a word character, whitespace, or a quote will be returned as a single-character token. If it is `True`, **`shlex`** will only split words in whitespaces;
- EOF is signaled with an empty string (`''`);
- It's not possible to parse empty strings, even if quoted.

When operating in POSIX mode, **`shlex`** will try to obey the following parsing rules.

- Quotes are stripped out, and do not separate words (`"Do"Not"Separate"` is parsed as the single word `DoNotSeparate`);

- Non-quoted escape characters (e.g. `'\'`) preserve the literal value of the next character that follows;
- Enclosing characters in quotes which are not part of **escapedquotes** (e.g. `"'"`) preserve the literal value of all characters within the quotes;
- Enclosing characters in quotes which are part of **escapedquotes** (e.g. `'"'`) preserves the literal value of all characters within the quotes, with the exception of the characters mentioned in **escape**. The escape characters retain their special meaning only when followed by the quote in use, or the escape character itself. Otherwise the escape character will be considered a normal character.
- EOF is signaled with a **None** value;
- Quoted empty strings (`''`) are allowed.

Improved Compatibility with Shells

Adicionado na versão 3.6.

The **shlex** class provides compatibility with the parsing performed by common Unix shells like `bash`, `dash`, and `sh`. To take advantage of this compatibility, specify the `punctuation_chars` argument in the constructor. This defaults to `False`, which preserves pre-3.6 behaviour. However, if it is set to `True`, then parsing of the characters `();<>|&` is changed: any run of these characters is returned as a single token. While this is short of a full parser for shells (which would be out of scope for the standard library, given the multiplicity of shells out there), it does allow you to perform processing of command lines more easily than you could otherwise. To illustrate, you can see the difference in the following snippet:

```
>>> import shlex
>>> text = "a && b; c && d || e; f >'abc'; (def \"ghi\")"
>>> s = shlex.shlex(text, posix=True)
>>> s.whitespace_split = True
>>> list(s)
['a', '&&', 'b;', 'c', '&&', 'd', '||', 'e;', 'f', '>abc;', '(de
>>> s = shlex.shlex(text, posix=True, punctuation_chars=True)
>>> s.whitespace_split = True
>>> list(s)
```

```
['a', '&&', 'b', ';', 'c', '&&', 'd', '||', 'e', ';', 'f', '>',
'(', 'def', 'ghi', ')']
```

Of course, tokens will be returned which are not valid for shells, and you'll need to implement your own error checks on the returned tokens.

Instead of passing `True` as the value for the punctuation_chars parameter, you can pass a string with specific characters, which will be used to determine which characters constitute punctuation. For example:

```
>>> import shlex
>>> s = shlex.shlex("a && b || c", punctuation_chars="|")
>>> list(s)
['a', '&', '&', 'b', '||', 'c']
```

Nota

When `punctuation_chars` is specified, the **`wordchars`** attribute is augmented with the characters `~-./*?=`. That is because these characters can appear in file names (including wildcards) and command-line arguments (e.g. `--color=auto`). Hence:

```
>>> import shlex
>>> s = shlex.shlex('~/a && b-c --color=auto || d *.py?',
...                 punctuation_chars=True)
>>> list(s)
['~/a', '&&', 'b-c', '--color=auto', '||', 'd', '*.py?']
```

However, to match the shell as closely as possible, it is recommended to always use `posix` and **`whitespace_split`** when using **`punctuation_chars`**, which will negate **`wordchars`** entirely.

For best effect, `punctuation_chars` should be set in conjunction with `posix=True`. (Note that `posix=False` is the default for **`shlex`**.)

`posix` — As chamadas de sistema mais comuns do POSIX

Este módulo fornece acesso à funcionalidade do sistema operacional padronizada pelo padrão C e pelo padrão POSIX (uma interface Unix levemente disfarçada).

Disponibilidade: Unix.

Não importe este módulo diretamente. Em vez disso, importe o módulo **`os`**, que fornece uma versão *portátil* dessa interface. No Unix, o módulo **`os`** fornece um superconjunto da interface **`posix`**. Em sistemas operacionais não Unix, o módulo **`posix`** não está disponível, mas um subconjunto está sempre disponível na interface **`os`**. Uma vez que **`os`** é importado, seu uso *não* causa penalidade de desempenho em comparação com **`posix`**. Além disso, **`os`** fornece algumas funcionalidades adicionais, como chamar automaticamente **`putenv()`** quando uma entrada em `os.environ` é alterada.

Erros são relatados como exceções. As exceções usuais são dadas para erros de tipo, enquanto os erros relatados pelas chamadas do sistema levantam **`OSError`**.

Suporte a arquivos grandes

Vários sistemas operacionais (incluindo AIX e Solaris) fornecem suporte a arquivos maiores que 2 GiB a partir de um modelo de programação C em que int e long são valores de 32 bits. Isso geralmente é realizado definindo o tamanho relevante e os tipos de deslocamento como valores de 64 bits. Esses arquivos às vezes são chamados de *arquivos grandes*.

O suporte a arquivos grandes é ativado no Python quando o tamanho de um **`off_t`** é maior que a long e long long é pelo menos tão grande quanto um **`off_t`**. Pode ser necessário configurar e compilar o Python com certos

sinalizadores do compilador para ativar esse modo. Por exemplo, com o Solaris 2.6 e 2.7, você precisa fazer algo como:

```
CFLAGS="`getconf LFS_CFLAGS`" OPT="-g -O2 $CFLAGS" \
        ./configure
```

Em sistemas Linux com capacidade para arquivos grandes, isso pode funcionar:

```
CFLAGS='-D_LARGEFILE64_SOURCE -D_FILE_OFFSET_BITS=64' OPT="-g -O
        ./configure
```

Conteúdo notável do módulo

Além de muitas funções descritas na documentação do módulo **os**, **posix** define o seguinte item de dados:

posix.environ

Um dicionário que representa o ambiente de strings no momento em que o interpretador foi iniciado. Chaves e valores são bytes no Unix e str no Windows. Por exemplo, `environ[b'HOME']` (`environ['HOME']` no Windows) é o nome do caminho do diretório inicial, equivalente a `getenv("HOME")` em C .

A modificação deste dicionário não afeta o ambiente de strings passado por **execv()**, **popen()** ou **system()**. Se você precisar alterar o ambiente, passe `environ` para **execve()** ou adicione atribuições de variável e instruções de exportação para a string de comando para **system()** ou **popen()**.

Alterado na versão 3.2: No Unix, chaves e valores são bytes.

Nota

O módulo **os** fornece uma implementação alternativa de `environ` que atualiza o ambiente ao ocorrerem modificações. Observe também que a atualização de **os.environ** tornará este dicionário obsoleto. O uso do módulo **os** é recomendado sobre o acesso direto ao módulo **posix**.

`pwd` — A senha do banco de dados

Este módulo provê acesso ao banco de dados das contas de usuário do sistema e suas respectivas senhas. Isto está disponível para todas as versões do Unix.

Disponibilidade: Unix, not WASI, not iOS.

As entradas do banco de dados de senhas são reportadas como um objeto do tipo tupla, cujos atributos correspondem aos membros da estrutura `passwd` (Campos dos atributos abaixo, veja `<pwd.h>`):

Índice	Atributo	Significado
0	`pw_name`	Nome de login
1	`pw_passwd`	Senha encriptada opcional
2	`pw_uid`	ID numérico do usuário
3	`pw_gid`	ID numérico do grupo
4	`pw_gecos`	Nome do usuário ou campo de comentário
5	`pw_dir`	Diretório home do usuário

Índice	Atributo	Significado
6	`pw_shell`	Interpretador de comandos do usuário

O uid e o gid são números inteiros, e os outros são strings. `KeyError` é levando se o campo requerido não puder ser encontrado.

Nota

Em Unix tradicional, o campo `pw_passwd` geralmente contém uma senha encriptada com um algoritmo derivado de DES. No entanto, a maioria dos sistemas tipo Unix modernos usam o chamado sistema *shadow password*. Nesses Unixes o campo *pw_passwd* só contém um asterisco (`'*'`) ou a letra `'x'` e a senha encriptada é guardada no arquivo `/etc/shadow` o qual não é permitido o acesso irrestrito a leitura. Se o campo *pw_passwd* contém alguma coisa útil dependerá do sistema.

Isto define os seguintes itens

pwd.getpwuid(*uid*)

Retorna a entrada do banco de dados de senhas para um dado ID de usuário

pwd.getpwnam(*name*)

Retorna a entrada do banco de dados de senhas para um dado nome de usuário

pwd.getpwall()

Retorna uma lista de todos as entradas disponíveis no banco de dados de senhas, em uma ordem arbitrária.

Ver também

Módulo `grp`

Uma interface para o banco de dados do grupo, similar a esta.

`grp` — O banco de dados de grupos

Este módulo fornece acesso ao banco de dados de grupos Unix. Ele está disponível em todas as versões do Unix.

Disponibilidade: Unix, not WASI, not Android, not iOS.

As entradas do banco de dados de grupos são relatadas como um objeto tupla ou similar, cujos atributos correspondem aos membros da estrutura `group` (campo Atributo abaixo, consulte `<grp.h>`):

Índice	Atributo	Significado
0	gr_name	o nome do grupo
1	gr_passwd	a senha do grupo (criptografada); geralmente vazia
2	gr_gid	O ID numérico do grupo
3	gr_mem	todos os nomes de usuários dos membros do grupo

O gid é um inteiro, nome e senha são strings, e a lista de membros é uma lista de strings. (Observe que a maioria dos usuários não são listados explicitamente como membros do grupo em que estão, de acordo com o banco de dados de senhas. Verifique ambos os bancos de dados para obter informações completas sobre associação. Observe também que um `gr_name` que começa com `+` ou `-` provavelmente é uma referência YP/NIS e pode não ser acessível via `getgrnam()` ou `getgrgid()`.)

Isto define os seguintes itens

grp.getgrgid(*id*)

Retorna a entrada do banco de dados de grupos para o ID do grupo numérico fornecido. `KeyError` é levantada se a entrada solicitada não puder ser encontrada.

Alterado na versão 3.10: `TypeError` é levantada para argumentos não inteiros, como pontos flutuantes ou strings.

grp.getgrnam(*name*)

Retorna a entrada do banco de dados de grupos para o nome do grupo fornecido. `KeyError` é levantada se a entrada solicitada não puder ser encontrada.

grp.getgrall()

Retorna uma lista de todas as entradas de grupo disponíveis, em ordem arbitrária.

Ver também

Módulo `pwd`

Uma interface para o banco de dados de usuários, semelhante a esta.

`termios` — Controle de tty no estilo POSIX

Este módulo fornece uma interface para as chamadas POSIX para controle de E/S do tty. Para uma descrição completa dessas chamadas, consulte a página de manual Unix *termios(3)* *[https://manpages.debian.org/termios(3)]*. Está disponível apenas para as versões Unix que tenham suporte ao controle de E/S de tty no estilo POSIX do *termios* configurado durante a instalação.

Disponibilidade: Unix.

Todas as funções neste módulo usam um descritor de arquivo *fd* como seu primeiro argumento. Pode ser um descritor de arquivo de tipo inteiro, como retornado por `sys.stdin.fileno()`, ou um objeto arquivo, como o próprio `sys.stdin`.

Este módulo também define todas as constantes necessárias para trabalhar com as funções fornecidas aqui; estes têm o mesmo nome de seus equivalentes em C. Consulte a documentação do sistema para mais informações sobre o uso dessas interfaces de controle de terminal.

O módulo define as seguintes funções:

termios.tcgetattr(*fd*)

Retorna uma lista contendo os atributos tty para o descritor de arquivo *fd*, da seguinte forma: `[iflag, oflag, cflag, lflag, ispeed, ospeed, cc]` onde *cc* é uma lista dos caracteres especiais do tty (cada uma string de comprimento 1, exceto os itens com índices **`VMIN`** e **`VTIME`**, que são números inteiros quando esses campos são definidos). A interpretação dos sinalizadores e as velocidades, bem como a indexação no vetor *cc*, devem ser feitas usando as constantes simbólicas definidas no módulo **`termios`**.

termios.tcsetattr(*fd*, *when*, *attributes*)

Define os atributos do tty para descritor de arquivo *fd* a partir de *attributes*, que é uma lista como a retornada por `tcgetattr()`. O argumento *when* determina quando os atributos são alterados:

termios.TCSANOW

Altera os atributos imediatamente.

termios.TCSADRAIN

Altera os atributos depois de transmitir todas as saídas enfileiradas.

termios.TCSAFLUSH

Altera os atributos depois de transmitir todas as saídas enfileiradas e descartar todas as entradas enfileiradas.

termios.tcsendbreak(*fd*, *duration*)

Envia uma quebra no descritor de arquivo *fd*. Uma duração zero, representada por *duration*, envia uma pausa por 0,25 a 0,5 segundos; *duration* com valor diferente de zero tem um significado dependente do sistema.

termios.tcdrain(*fd*)

Aguarda até que toda a saída escrita no descritor de arquivo *fd* seja transmitida.

termios.tcflush(*fd*, *queue*)

Descarta dados na fila no descritor de arquivo *fd*. O seletor *queue* especifica qual fila: **`TCIFLUSH`** para a fila de entrada, **`TCOFLUSH`** para a fila de saída ou **`TCIOFLUSH`** para as duas filas.

termios.tcflow(*fd*, *action*)

Suspende ou retoma a entrada ou saída no descritor de arquivo *fd*. O argumento *action* pode ser **`TCOOFF`** para suspender a saída, **`TCOON`** para reiniciar a saída, **`TCIOFF`** para suspender a entrada ou **`TCION`** para reiniciar a entrada.

termios.tcgetwinsize(*fd*)

Retorna uma tupla `(ws_row, ws_col)` contendo o tamanho da janela de terminal para o descritor de arquivo *fd*. Requer **`termios.TIOCGWINSZ`** ou **`termios.TIOCGSIZE`**.

Adicionado na versão 3.11.

termios.tcsetwinsize(*fd*, *winsize*)

Define o tamanho da janela de terminal para o descritor de arquivo *fd* de *winsize*, que é uma tupla de dois elementos `(ws_row, ws_col)` como a retornada por **`tcgetwinsize()`**. Requer pelo menos um dos pares (**`termios.TIOCGWINSZ`**, **`termios.TIOCSWINSZ`**); (**`termios.TIOCGSIZE`**, **`termios.TIOCSSIZE`**) a ser definido.

Adicionado na versão 3.11.

Ver também

Módulo `tty`

Funções de conveniência para operações comuns de controle de terminal.

Exemplo

Aqui está uma função que solicita uma senha com o eco desativado. Observe a técnica usando uma chamada separada **`tcgetattr()`** e uma instrução **`try`** … **`finally`** para garantir que os atributos tty antigos sejam restaurados exatamente, aconteça o que acontecer:

```
def getpass(prompt="Senha: "):
    import termios, sys
    fd = sys.stdin.fileno()
    old = termios.tcgetattr(fd)
    new = termios.tcgetattr(fd)
    new[3] = new[3] & ~termios.ECHO          # lflags
    try:
```

```
    termios.tcsetattr(fd, termios.TCSADRAIN, new)
    passwd = input(prompt)
finally:
    termios.tcsetattr(fd, termios.TCSADRAIN, old)
return passwd
```

`tty` — Funções de controle de terminal

Código-fonte: Lib/tty.py [https://github.com/python/cpython/tree/3.15/Lib/tty.py]

O módulo **`tty`** define funções para colocar o tty nos modos de cbreak e não tratados (raw).

Disponibilidade: Unix.

Por requerer o módulo **`termios`**, ele funcionará apenas no Unix.

O módulo **`tty`** define as seguintes funções:

tty.cfmakeraw(*mode*)

Converte a lista de atributos de tty *mode*, que é uma lista como a retornada por **`termios.tcgetattr()`**, para a de um tty em modo raw.

Adicionado na versão 3.12.

tty.cfmakecbreak(*mode*)

Converte a lista de atributos de tty *mode*, que é uma lista como a retornada por **`termios.tcgetattr()`**, para a de um tty em modo cbreak.

Isto limpa os sinalizadores de modo local `ECHO` e `ICANON` em *mode* bem como configura a entrada mínima para 1 byte sem atraso.

Adicionado na versão 3.12.

Alterado na versão 3.12.2: O sinalizador `ICRNL` não está mais limpo. Isso corresponde ao comportamento `stty cbreak` do Linux e macOS e ao que **`setcbreak()`** historicamente fez.

tty.setraw(*fd*, *when=termios.TCSAFLUSH*)

Altera o modo do descritor de arquivo *fd* para raw. Se *when* for omitido, o padrão é `termios.TCSAFLUSH`, e é passado para `termios.tcsetattr()`. O retorna valor de `termios.tcgetattr()` é salvo antes de definir *fd* para o modo raw; esse valor é retornado.

Alterado na versão 3.12: O valor de retorno agora é o atributos originais do tty, em vez de `None`.

tty.setcbreak(*fd*, *when=termios.TCSAFLUSH*)

Altera o modo de descritor de arquivo *fd* para cbreak. Se *when* for omitido, o padrão é `termios.TCSAFLUSH`, e é passado para `termios.tcsetattr()`. O retorna valor de `termios.tcgetattr()` é salvo antes de definir *fd* para o modo cbreak; esse valor é retornado.

Isto limpa os sinalizadores de modo local `ECHO` e `ICANON` bem como configura a entrada mínima para 1 byte sem atraso.

Alterado na versão 3.12: O valor de retorno agora é o atributos originais do tty, em vez de `None`.

Alterado na versão 3.12.2: O sinalizador `ICRNL` não está mais limpo. Isso restaura o comportamento do Python 3.11 e anteriores, além de corresponder ao que Linux, macOS e BSDs descrevem em suas páginas de manual `stty(1)` em relação ao modo cbreak.

Ver também

Módulo `termios`

Interface baixo nível para controle de terminal.

`pty` — Utilitários de pseudoterminal

Código-fonte: Lib/pty.py [https://github.com/python/cpython/tree/3.15/Lib/pty.py]

O módulo **`pty`** define operações para lidar com o conceito de pseudoterminal: iniciar outro processo e poder gravar e ler de seu terminal de controle programaticamente.

Disponibilidade: Unix.

O tratamento do pseudoterminal é altamente dependente da plataforma. Esse código foi testado principalmente no Linux, no FreeBSD e no macOS (supõe-se que funcione em outras plataformas POSIX, mas não foi testado exaustivamente).

O módulo **`pty`** define as seguintes funções:

pty.fork()

Faz um fork. Conecta o terminal de controle do filho a um pseudoterminal. O valor de retorno é `(pid, fd)`. Observe que a criança recebe *pid* 0 e o *fd* é *inválido*. O valor de retorno do pai é o *pid* do filho, e o *fd* é um descritor de arquivo conectado ao terminal de controle do filho (e também à entrada e à saída padrão do filho).

O descritor de arquivo *fd* retornado é não herdável.

Aviso

No macOS, o uso desta função é inseguro quando combinado com o uso de APIs de sistema de nível mais alto, e isso inclui o uso de **`urllib.request`**.

Alterado na versão 3.15: O descritor de arquivo retornado agora for tornado não-herdável.

pty.openpty()

Abre um novo par de pseudoterminais, usando `os.openpty()`, se possível, ou código de emulação para sistemas genérico Unix . Retorna um par de descritores de arquivo `(master, slave)`, para a extremidade mestre e escrava, respectivamente.

pty.spawn(*argv*[, *master_read*[, *stdin_read*]])

Gera um processo e conecta seu terminal de controle com o E/S padrão do processo atual. Isso é frequentemente usado para confundir programas que insistem em ler no terminal de controle. Espera-se que o processo gerado por trás do pty acabe sendo encerrado e, quando isso acontecer, o *spawn* é retornado.

Um laço copia o STDIN do processo atual para o filho e os dados recebidos do filho para o STDOUT do site processar atual. Não é sinalizado para o filho se o STDIN do processar atual fechar.

As funções *master_read* e o *stdin_read* recebem um descritor de arquivo do qual devem ler e devem sempre retorna uma string de bytes. Para forçar *spawn* retornar antes que o processo filho saia, um byte vazio vetor deve ser retornado para sinalizar o fim do arquivo.

A implementação padrão para ambos os funções vai ler e retornar até 1024 bytes cada vez que a função for chamada. A função de retorno *master_read* é passada para o descritor de arquivo mestre do pseudoterminal para ler a saída do processo filho, e ao *stdin_read* é passado o descritor de arquivo 0, para ler a entrada padrão do processo pai.

Retornar uma string de bytes vazia de qualquer uma das funções de retorno é interpretado como uma condição de fim de vida (EOF), e que a função de retorno não será chamada depois disso. Se *stdin_read* sinalizar EOF, o terminal de controle não poderá mais se comunicar com o processo pai OU o processo filho. A menos que o filho processar

seja encerrado sem nenhuma entrada, *spawn* vai então fazer o laço para sempre. Se *master_read* sinalizar EOF, os mesmos comportamento resultados (pelo menos no Linux).

Retorna o valor de status de saída de **os.waitpid()** no processo filho.

os.waitstatus_to_exitcode() pode ser usado para converter o status de saída em um código de saída.

Levanta um evento de auditoria `pty.spawn` com o argumento `argv`.

Alterado na versão 3.4: **spawn()** agora retorna o valor de status de **os.waitpid()** no processo filho.

Exemplo

O programa a seguir funciona como o comando Unix *script(1)* *[https://manpages.debian.org/script(1)]* , usando um pseudoterminal para registrar todas as entradas e saídas de uma sessão de terminal em um “script”.

```
import argparse
import os
import pty
import sys
import time

parser = argparse.ArgumentParser()
parser.add_argument('-a', dest='append', action='store_true')
parser.add_argument('-p', dest='use_python', action='store_true'
parser.add_argument('filename', nargs='?', default='typescript')
options = parser.parse_args()

shell = sys.executable if options.use_python else os.environ.get
filename = options.filename
mode = 'ab' if options.append else 'wb'

with open(filename, mode) as script:
    def read(fd):
        data = os.read(fd, 1024)
        script.write(data)
```

```
    return data

print('Script started, file is', filename)
script.write(('Script started on %s\n' % time.asctime()).enc

pty.spawn(shell, read)

script.write(('Script done on %s\n' % time.asctime()).encode
print('Script done, file is', filename)
```

`fcntl` — as chamadas de sistema `fcntl` e `ioctl`

Este módulo executa o controle de arquivos e de E/S em descritores de arquivos. É uma interface para as rotinas **`fcntl()`** e **`ioctl()`** do Unix. Veja as páginas de manual do Unix *fcntl(2)* *[https://manpages.debian.org/fcntl(2)]* e *ioctl(2)* *[https://manpages.debian.org/ioctl(2)]* para mais detalhes.

Disponibilidade: Unix, not WASI.

Todas as funções neste módulo recebem um descritor de arquivo *fd* como seu primeiro argumento. Este pode ser um descritor de arquivo inteiro, como retornado por `sys.stdin.fileno()`, ou um objeto **`io.IOBase`**, como o próprio `sys.stdin`, que fornece um **`fileno()`** que retorna um descritor de arquivo genuíno.

Alterado na versão 3.3: As operações neste módulo costumavam levantar um **`IOError`**, mas agora levantam um **`OSError`**.

Alterado na versão 3.8: O módulo **`fcntl`** agora contém constantes `F_ADD_SEALS`, `F_GET_SEALS` e `F_SEAL_*` para selar descritores de arquivo **`os.memfd_create()`**.

Alterado na versão 3.9: No macOS, o módulo **`fcntl`** expõe a constante `F_GETPATH`, que obtém o caminho de um arquivo de um descritor de arquivo. No Linux(>=3.15), o módulo **`fcntl`** expõe as constantes `F_OFD_GETLK`, `F_OFD_SETLK` e `F_OFD_SETLKW`, que são usadas ao trabalhar com travas de descrição de arquivo aberto.

Alterado na versão 3.10: No Linux >= 2.6.11, o módulo **`fcntl`** expõe as constantes `F_GETPIPE_SZ` e `F_SETPIPE_SZ`, que permitem verificar e modificar o tamanho de um encadeamento, respectivamente.

Alterado na versão 3.11: No FreeBSD, o módulo **`fcntl`** expõe as constantes `F_DUP2FD` e `F_DUP2FD_CLOEXEC`, que permitem duplicar um descritor de arquivo, sendo que esta última define também o sinalizador `FD_CLOEXEC`.

Alterado na versão 3.12: On Linux >= 4.5, the **`fcntl`** module exposes the `FICLONE` and `FICLONERANGE` constants, which allow to share some data of one file with another file by reflinking on some filesystems (e.g., btrfs, OCFS2, and XFS). This behavior is commonly referred to as “copy-on-write”.

Alterado na versão 3.13: No Linux >= 2.6.32, o módulo **`fcntl`** expõe as constantes `F_GETOWN_EX`, `F_SETOWN_EX`, `F_OWNER_TID`, `F_OWNER_PID`, `F_OWNER_PGRP`, que permitem direcionar sinais de disponibilidade de E/S para uma thread, processo ou grupo de processos específico. No Linux >= 4.13, o módulo **`fcntl`** expõe as constantes `F_GET_RW_HINT`, `F_SET_RW_HINT`, `F_GET_FILE_RW_HINT`, `F_SET_FILE_RW_HINT` e `RWH_WRITE_LIFE_*`, que permitem informar o kernel sobre o tempo de vida relativo esperado de gravações em um determinado nó-i ou por meio de uma descrição de arquivo aberto específica. No Linux >= 5.1 e NetBSD, o módulo **`fcntl`** expõe a constante `F_SEAL_FUTURE_WRITE` para uso com as operações `F_ADD_SEALS` e `F_GET_SEALS`. No FreeBSD, o módulo **`fcntl`** expõe as constantes `F_READAHEAD`, `F_ISUNIONSTACK` e `F_KINFO`. No macOS e FreeBSD, o módulo **`fcntl`** expõe a constante `F_RDAHEAD`. No NetBSD e AIX, o módulo **`fcntl`** expõe a constante `F_CLOSEM`. No NetBSD, o módulo **`fcntl`** expõe a constante `F_MAXFD`. No macOS e no NetBSD, o módulo **`fcntl`** expõe as constantes `F_GETNOSIGPIPE` e `F_SETNOSIGPIPE`.

Alterado na versão 3.14: No Linux >= 6.1, o módulo **`fcntl`** expõe o `F_DUPFD_QUERY` para consultar um descritor de arquivo apontando para o mesmo arquivo.

O módulo define as seguintes funções:

fcntl.fcntl(*fd*, *cmd*, *arg=0*, /)

Perform the operation *cmd* on file descriptor *fd* (file objects providing a **`fileno()`** method are accepted as well). The values used for *cmd* are operating system dependent, and are available as constants in the **`fcntl`**

module, using the same names as used in the relevant C header files. The argument *arg* can either be an integer value, a bytes-like object, or a string. The type and size of *arg* must match the type and size of the argument of the operation as specified in the relevant C documentation.

Quando *arg* é um inteiro, a função retorna o valor de retorno inteiro da chamada C **`fcntl()`**.

When the argument is bytes-like object, it represents a binary structure, for example, created by **`struct.pack()`**. A string value is encoded to binary using the UTF-8 encoding. The binary data is copied to a buffer whose address is passed to the C **`fcntl()`** call. The return value after a successful call is the contents of the buffer, converted to a **`bytes`** object. The length of the returned object will be the same as the length of the *arg* argument.

Se a chamada a **`fcntl()`** falhar, um exceção **`OSError`** é levantada.

Nota

If the type or size of *arg* does not match the type or size of the operation's argument (for example, if an integer is passed when a pointer is expected, or the information returned in the buffer by the operating system is larger than the size of *arg*), this is most likely to result in a segmentation violation or a more subtle data corruption.

Levanta um evento de auditoria `fcntl.fcntl` com os argumentos `fd`, `cmd`, `arg`.

Alterado na versão 3.14: Adicione suporte a objetos bytes ou similares arbitrários, não apenas **`bytes`**.

Alterado na versão 3.15: The size of bytes-like objects is no longer limited to 1024 bytes.

fcntl.ioctl(*fd*, *request*, *arg=0*, *mutate_flag=True*, /)

Esta função é idêntica à função **`fcntl()`**, exceto que o tratamento de argumentos é ainda mais complicado.

O parâmetro *request* é limitado a valores que cabem em 32 ou 64 bits, dependendo da plataforma. Constantes adicionais de interesse para uso como argumento *request* podem ser encontradas no módulo **termios**, com os mesmos nomes usados nos arquivos de cabeçalho C relevantes.

O parâmetro *arg* pode ser um inteiro, um objeto bytes ou similar ou uma string. O tipo e o tamanho de *arg* devem corresponder ao tipo e ao tamanho do argumento da operação, conforme especificado na documentação C relevante.

Se *arg* não oferecer suporte à interface de buffer de leitura e escrita ou o *mutate_flag* for falso, o comportamento será o mesmo da função **fcntl()**.

Se *arg* oferece suporte à interface de buffer de leitura e escrita (como **bytearray**) e *mutate_flag* for true (o padrão), o buffer será (na prática) passado para a chamada de sistema subjacente **ioctl()**, o código de retorno desta última será passado de volta para o Python que fez a chamada, e o novo conteúdo do buffer refletirá a ação de **ioctl()**. Esta é uma pequena simplificação, pois se o buffer fornecido tiver menos de 1024 bytes, ele será primeiro copiado para um buffer estático de 1024 bytes, que será então passado para **ioctl()** e copiado de volta para o buffer fornecido.

Se a chamada a **ioctl()** falhar, uma exceção **OSError** é levantada.

Nota

If the type or size of *arg* does not match the type or size of the operation's argument (for example, if an integer is passed when a pointer is expected, or the information returned in the buffer by the operating system is larger than the size of *arg*), this is most likely to result in a segmentation violation or a more subtle data corruption.

Um exemplo:

```
>>> import array, fcntl, struct, termios, os
>>> os.getpgrp()
```

```
13341
>>> struct.unpack('h', fcntl.ioctl(0, termios.TIOCGPGRP, "  '
13341
>>> buf = array.array('h', [0])
>>> fcntl.ioctl(0, termios.TIOCGPGRP, buf, 1)
0
>>> buf
array('h', [13341])
```

Levanta um evento de auditoria `fcntl.ioctl` com os argumentos `fd`, `request`, `arg`.

Alterado na versão 3.14: A GIL é sempre liberada durante uma chamada de sistema. Chamadas de sistema que falham com EINTR são repetidas automaticamente.

Alterado na versão 3.15: The size of not mutated bytes-like objects is no longer limited to 1024 bytes.

fcntl.flock(*fd*, *operation*, /)

Executa a operação de trava *operation* no descritor de arquivo *fd* (objetos arquivo que fornecem um método **`fileno()`** também são aceitos). Consulte o manual do Unix *flock(2)* *[https://manpages.debian.org/flock(2)]* para obter detalhes. (Em alguns sistemas, esta função é emulada usando **`fcntl()`**.)

Se a chamada **`flock()`** falhar, uma exceção **`OSError`** será levantada.

Levanta um evento de auditoria `fcntl.flock` com os argumentos `fd` e `operation`.

fcntl.lockf(*fd*, *cmd*, *len=0*, *start=0*, *whence=0*, /)

Este é essencialmente um invólucro em torno das chamadas de trava **`fcntl()`**. *fd* é o descritor de arquivo (objetos arquivo que fornecem um método **`fileno()`** também são aceitos) do arquivo para travar ou destravar, e *cmd* é um dos seguintes valores:

fcntl.LOCK_UN

Libera uma trava existente.

fcntl.LOCK_SH
Adquire uma trava compartilhada.

fcntl.LOCK_EX
Adquire uma trava exclusiva.

fcntl.LOCK_NB
Aplica OU (OR) bit a bit com qualquer uma das outras três constantes `LOCK_*` para tornar a solicitação não bloqueante.

Se **`LOCK_NB`** for usado e a trava não puder ser obtido, uma exceção **`OSError`** será levantada e terá um atributo *errno* definido como **`EACCES`** ou **`EAGAIN`** (dependendo do sistema operacional; para portabilidade, verifique ambos os valores). Em pelo menos alguns sistemas, **`LOCK_EX`** só pode ser usado se o descritor de arquivo se referir a um arquivo aberto para escrita.

len é o número de bytes para travar, *start* é o deslocamento de bytes em que a trava começa, em relação a *whence*, e *whence* é como em **`io.IOBase.seek()`**, especificamente:

- `0` – relativo ao início do arquivo (**`os.SEEK_SET`**)
- `1` – relativo à posição atual do buffer (**`os.SEEK_CUR`**)
- `2` – relativo ao fim do arquivo (**`os.SEEK_END`**)

O padrão para *start* é 0, o que significa iniciar no início do arquivo. O padrão para *len* é 0, o que significa travar no final do arquivo. O padrão para *whence* também é 0.

Levanta um evento de auditoria `fcntl.lockf` com os argumentos `fd`, `cmd`, `len`, `start`, `whence`.

Exemplos (todos em um sistema compatível com SVR4):

```
import struct, fcntl, os

f = open(...)
```

```
rv = fcntl.fcntl(f, fcntl.F_SETFL, os.O_NDELAY)

lockdata = struct.pack('hhllhh', fcntl.F_WRLCK, 0, 0, 0, 0, 0)
rv = fcntl.fcntl(f, fcntl.F_SETLKW, lockdata)
```

Observe que, no primeiro exemplo, a variável de valor de retorno *rv* conterá um valor inteiro; no segundo exemplo, ela conterá um objeto `bytes`. O layout da estrutura da variável *lockdata* depende do sistema — portanto, usar a chamada `flock()` pode ser melhor.

Ver também

Módulo `os`

Se os sinalizadores de trava `O_SHLOCK` e `O_EXLOCK` estiverem presentes no módulo `os` (somente no BSD), a função `os.open()` fornece uma alternativa às funções `lockf()` e `flock()`.

`resource` — Informação de uso de recursos

Este módulo fornece mecanismos básicos para medir e controlar os recursos do sistema usados por um programa.

Disponibilidade: Unix, not WASI.

Constantes simbólicas são usadas para especificar recursos específicos do sistema e para solicitar informações de uso sobre o processo atual ou seus filhos.

Uma exceção `OSError` é levantada em caso de falha de chamada de sistema.

exception resource.error

Um apelido descontinuado de `OSError`.

Alterado na versão 3.3: Seguindo a **PEP 3151** [https://peps.python.org/pep-3151/], esta classe foi transformada em um apelido de `OSError`.

Limites de recursos

O uso de recursos pode ser limitado usando a função `setrlimit()` descrita abaixo. Cada recurso é controlado por um par de limites: um limite suave e um limite rígido. O limite suave é o limite atual e pode ser reduzido ou aumentado por um processo ao longo do tempo. O limite suave nunca pode exceder o limite rígido. O limite rígido pode ser reduzido para qualquer valor maior que o limite suave, mas não aumentado. (Somente processos com o UID efetivo do superusuário podem aumentar um limite rígido.)

Os recursos específicos que podem ser limitados dependem do sistema. Eles são descritos na página man *getrlimit(2)* *[https://manpages.debian.org/getrlimit(2)]*. Os recursos listados abaixo são suportados quando o sistema operacional

subjacente os suporta; recursos que não podem ser verificados ou controlados pelo sistema operacional não são definidos neste módulo para essas plataformas.

resource.RLIM_INFINITY

Constante usada para representar o limite de um recurso ilimitado.

Alterado na versão 3.15: It is now always positive. Previously, it could be negative, such as -1 or -3.

resource.RLIM_SAVED_CUR

resource.RLIM_SAVED_MAX

Constants used to represent the soft and hard limit values if they cannot be represented in the `rlim_t` value in C. Can be equal to **`RLIM_INFINITY`**.

Adicionado na versão 3.15.

resource.getrlimit(*resource*)

Retorna uma tupla `(suave, rígido)` com os limites soft e hard atuais de *resource*. Levanta **`ValueError`** se um recurso inválido for especificado, ou **`error`** se a chamada do sistema subjacente falhar inesperadamente.

resource.setrlimit(*resource*, *limits*)

Define novos limites de consumo de *resource*. O argumento *limits* deve ser uma tupla `(suave, rígido)` de dois inteiros descrevendo os novos limites. Um valor de **`RLIM_INFINITY`** pode ser usado para solicitar um limite ilimitado.

Levanta **`ValueError`** se um recurso inválido for especificado, se o novo limite suave exceder o limite rígido ou se um processo tentar aumentar seu limite rígido. Especificar um limite de **`RLIM_INFINITY`** quando o limite rígido ou do sistema para esse recurso não for ilimitado resultará em uma exceção **`ValueError`**. Um processo com o UID efetivo de superusuário pode solicitar qualquer valor limite válido, incluindo

ilimitado, mas **`ValueError`** ainda será levantada se o limite solicitado exceder o limite imposto pelo sistema.

`setrlimit` também pode levantar **`error`** se a chamada do sistema subjacente falhar.

VxWorks só oferece suporte a definir **`RLIMIT_NOFILE`**.

Levanta um evento de auditoria `resource.setrlimit` com os argumentos `resource`, `limits`.

resource.prlimit(*pid*, *resource*[, *limits*])

Combina **`setrlimit()`** e **`getrlimit()`** em uma única função e permite obter e definir os limites de recursos de um processo arbitrário. Se *pid* for 0, a chamada se aplica ao processo atual. *resource* e *limits* têm o mesmo significado que em **`setrlimit()`**, exceto que *limits* é opcional.

Quando *limits* não é fornecido, a função retorna o limite de *resource* do processo *pid*. Quando *limits* é fornecido, o limite de *resource* do processo é definido e o limite de recurso anterior é retornado.

Levanta **`ProcessLookupError`** quando *pid* não pode ser encontrado e **`PermissionError`** quando o usuário não tem `CAP_SYS_RESOURCE` para o processo.

Levanta um evento de auditoria `resource.prlimit` com os argumentos `pid`, `resource`, `limits`.

Disponibilidade: Linux >= 2.6.36 with glibc >= 2.13.

Adicionado na versão 3.4.

Esses símbolos definem recursos cujo consumo pode ser controlado usando as funções **`setrlimit()`** e **`getrlimit()`** descritas abaixo. Os valores desses símbolos são exatamente as constantes usadas por programas em C.

A página man do Unix para *getrlimit(2)* *[https://manpages.debian.org/getrlimit(2)]* lista os recursos disponíveis. Observe que nem todos os sistemas usam o

mesmo símbolo ou o mesmo valor para denotar o mesmo recurso. Este módulo não tenta mascarar diferenças de plataforma — símbolos não definidos para uma plataforma não estarão disponíveis neste módulo para essa plataforma.

resource.RLIMIT_CORE

O tamanho máximo (em bytes) de um arquivo de núcleo (core file) que o processo atual pode criar. Isso pode resultar na criação de um arquivo de núcleo parcial se um núcleo maior for necessário para conter a imagem completa do processo.

resource.RLIMIT_CPU

O tempo máximo de processador (em segundos) que um processo pode usar. Se esse limite for excedido, um sinal `SIGXCPU` é enviado ao processo. (Consulte a documentação do módulo `signal` para obter informações sobre como capturar esse sinal e fazer algo útil, por exemplo, liberar arquivos abertos no disco.)

resource.RLIMIT_FSIZE

O tamanho máximo de um arquivo que o processo pode criar.

resource.RLIMIT_DATA

O tamanho máximo (em bytes) da heap do processo.

resource.RLIMIT_STACK

O tamanho máximo (em bytes) da pilha de chamadas para o processo atual. Isso afeta apenas a pilha da thread principal em um processo multithread.

resource.RLIMIT_RSS

O tamanho máximo do conjunto de residentes que deve ser disponibilizado para o processo.

resource.RLIMIT_NPROC

O número máximo de processos que o processo atual pode criar.

resource.RLIMIT_NOFILE

O número máximo de descritores de arquivo abertos para o processo atual.

resource.RLIMIT_OFILE

O nome BSD para `RLIMIT_NOFILE`.

resource.RLIMIT_MEMLOCK

O espaço máximo de endereço que pode ser travado na memória.

resource.RLIMIT_VMEM

A maior área de memória mapeada que o processo pode ocupar. Geralmente um apelido de `RLIMIT_AS`.

Disponibilidade: Solaris, FreeBSD, NetBSD.

resource.RLIMIT_AS

A área máxima (em bytes) de espaço de endereço que pode ser ocupada pelo processo.

resource.RLIMIT_MSGQUEUE

O número de bytes que podem ser alocados para filas de mensagens POSIX.

Disponibilidade: Linux >= 2.6.8.

Adicionado na versão 3.4.

resource.RLIMIT_NICE

O teto para o nível agradável do processo (calculado como 20 - rlim_cur).

Disponibilidade: Linux >= 2.6.12.

Adicionado na versão 3.4.

resource.RLIMIT_RTPRIO

O teto da prioridade em tempo real.

Disponibilidade: Linux >= 2.6.12.

Adicionado na versão 3.4.

resource.RLIMIT_RTTIME

O limite de tempo (em microssegundos) no tempo de CPU que um processo pode gastar sob agendamento em tempo real sem fazer uma chamada de sistema de bloqueio.

Disponibilidade: Linux >= 2.6.25.

Adicionado na versão 3.4.

resource.RLIMIT_SIGPENDING

O número de sinais que o processo pode enfileirar.

Disponibilidade: Linux >= 2.6.8.

Adicionado na versão 3.4.

resource.RLIMIT_SBSIZE

O tamanho máximo (em bytes) de uso do buffer de soquete para este usuário. Isso limita a quantidade de memória de rede e, portanto, a quantidade de mbufs que este usuário pode armazenar a qualquer momento.

Disponibilidade: FreeBSD, NetBSD.

Adicionado na versão 3.4.

resource.RLIMIT_SWAP

O tamanho máximo (em bytes) do espaço de swap que pode ser reservado ou utilizado por todos os processos deste ID de usuário. Este limite é aplicado somente se o bit 1 do sysctl vm.overcommit estiver definido. Consulte tuning(7) [https://man.freebsd.org/cgi/man.cgi?query=tuning&sektion=7] para uma descrição completa deste sysctl.

Disponibilidade: FreeBSD >= 8.

Adicionado na versão 3.4.

resource.RLIMIT_NPTS

O número máximo de pseudoterminais criados por este ID de usuário.

Disponibilidade: FreeBSD >= 8.

Adicionado na versão 3.4.

resource.RLIMIT_KQUEUES

O número máximo de kqueues que esse ID de usuário tem permissão para criar.

Disponibilidade: FreeBSD >= 11.

Adicionado na versão 3.10.

resource.RLIMIT_NTHR

The maximum number of threads for this user id, not counting the main and kernel threads.

Disponibilidade: NetBSD >= 7.0.

Adicionado na versão 3.15.

resource.RLIMIT_PIPEBUF

The maximum total size of in-kernel buffers for bi-directional pipes/fifos that this user id is allowed to consume.

Disponibilidade: FreeBSD >= 14.2.

Adicionado na versão 3.15.

resource.RLIMIT_THREADS

The maximum number of threads each process can create.

Disponibilidade: AIX.

Adicionado na versão 3.15.

resource.RLIMIT_UMTXP

The limit of the number of process-shared Posix thread library objects allocated by user id.

Disponibilidade: FreeBSD >= 11.

Adicionado na versão 3.15.

Uso de recursos

Essas funções são usadas para recuperar informações de uso de recursos:

resource.getrusage(*who*)

Esta função retorna um objeto que descreve os recursos consumidos pelo processo atual ou seus filhos, conforme especificado pelo parâmetro *who*. O parâmetro *who* deve ser especificado usando uma das constantes **RUSAGE_*** descritas abaixo.

Um exemplo simples:

```
from resource import *
import time

# uma tarefa não vinculada à CPU
time.sleep(3)
print(getrusage(RUSAGE_SELF))

# uma tarefa vinculada à CPU
for i in range(10 ** 8):
   _ = 1 + 1
print(getrusage(RUSAGE_SELF))
```

Os campos do valor de retorno descrevem como um recurso específico do sistema foi utilizado, por exemplo, a quantidade de tempo gasto em execução no modo de usuário ou o número de vezes que o processo foi trocado da memória principal. Alguns valores dependem do intervalo de

tique do relógio, por exemplo, a quantidade de memória que o processo está usando.

Para retrocompatibilidade, o valor de retorno também pode ser acessado como uma tupla de 16 elementos.

Os campos **`ru_utime`** e **`ru_stime`** do valor de retorno são valores de ponto flutuante que representam o tempo gasto na execução no modo usuário e o tempo gasto na execução no modo sistema, respectivamente. Os valores restantes são inteiros. Consulte a página man *getrusage(2)* *[https://manpages.debian.org/getrusage(2)]* para obter informações detalhadas sobre esses valores. Um breve resumo é apresentado aqui:

Índice	Campo	Recurso
`0`	**`ru_utime`**	tempo no modo de usuário (segundos em pontos flutuantes)
`1`	**`ru_stime`**	tempo no modo de sistema (segundos em pontos flutuantes)
`2`	**`ru_maxrss`**	tamanho máximo do conjunto residente
`3`	**`ru_ixrss`**	tamanho da memória compartilhada
`4`	**`ru_idrss`**	tamanho da memória não compartilhada
`5`	**`ru_isrss`**	tamanho da pilha não compartilhada
`6`	**`ru_minflt`**	falhas de página que não requerem E/S
`7`	**`ru_majflt`**	falhas de página que requerem E/S
`8`	**`ru_nswap`**	número de trocas para a memórias de swap
`9`	**`ru_inblock`**	operações de entrada de bloco

Índice	Campo	Recurso
10	**ru_oublock**	operações de saída de bloco
11	**ru_msgsnd**	mensagens enviadas
12	**ru_msgrcv**	mensagens recebidas
13	**ru_nsignals**	sinais recebidos
14	**ru_nvcsw**	trocas voluntárias de contexto
15	**ru_nivcsw**	trocas involuntárias de contexto

Esta função levantará uma `ValueError` se um parâmetro *who* inválido for especificado. Também pode levantar uma exceção `error` em circunstâncias incomuns.

resource.getpagesize()

Retorna o número de bytes em uma página do sistema. (Não precisa ser o mesmo que o tamanho da página de hardware.)

Os seguintes símbolos **RUSAGE_*** são passados para a função `getrusage()` para especificar quais informações de processos devem ser fornecidas.

resource.RUSAGE_SELF

Passeapara `getrusage()` para solicitar recursos consumidos pelo processo de chamada, que é a soma dos recursos usados por todas as threads no processo.

resource.RUSAGE_CHILDREN

Passa para `getrusage()` para solicitar recursos consumidos por processos filhos do processo de chamada que foram encerrados e aguardados.

resource.RUSAGE_BOTH

Passa para `getrusage()` para solicitar recursos consumidos pelo processo atual e pelos processos filhos. Pode não estar disponível em todos os sistemas.

resource.RUSAGE_THREAD

Passa para `getrusage()` para solicitar recursos consumidos pela thread atual. Pode não estar disponível em todos os sistemas.

Adicionado na versão 3.2.

`syslog` — Rotinas da biblioteca syslog do Unix

Este módulo fornece uma interface para as rotinas da biblioteca `syslog` do Unix. Consulte as páginas de manual do Unix para uma descrição detalhada do recurso `syslog`.

Disponibilidade: Unix, not WASI, not iOS.

Este módulo é uma camada para a família de rotinas `syslog` do sistema. Uma biblioteca Python pura que pode se comunicar com um servidor syslog está disponível no módulo **`logging.handlers`** como **`SysLogHandler`**.

O módulo define as seguintes funções:

syslog.syslog(*message*)
syslog.syslog(*priority*, *message*)

Envia a string *message* para o registrador do sistema. Uma nova linha final é adicionada, se necessário. Cada mensagem é marcada com uma prioridade composta por um *facility* e um *level*. O argumento opcional *priority*, cujo padrão é **`LOG_INFO`**, determina a prioridade da mensagem. Se a facilidade não está codificada em *priority* usando o OU lógico (`LOG_INFO | LOG_USER`), o valor dado na chamada **`openlog()`** é usado.

Se **`openlog()`** não foi chamado antes da chamada para **`syslog()`**, **`openlog()`** será chamado sem argumentos.

Levanta um evento de auditoria `syslog.syslog` com os argumentos `priority`, `message`.

Alterado na versão 3.2: Nas versões anteriores, **`openlog()`** não seria chamado automaticamente se não fosse chamado antes da chamada para **`syslog()`**, adiando a implementação do syslog para chamar `openlog()`.

Alterado na versão 3.12: Esta função é restrita em subinterpretadores. (Apenas o código que executa em vários interpretadores é afetado e a restrição não é relevante para a maioria dos usuários.) **`openlog()`** deve ser chamado no interpretador principal antes de **`syslog()`** pode ser usado em um subinterpretador. Caso contrário, vai levantar **`RuntimeError`**.

syslog.openlog([*ident*[, *logoption*[, *facility*]]])

As opções de log das chamadas subsequentes **`syslog()`** podem ser definidas chamando **`openlog()`**. **`syslog()`** irá chamar **`openlog()`** sem argumentos se o log não estiver aberto no momento.

O argumento nomeado opcional *ident* é uma string que é prefixada a cada mensagem, e o padrão é `sys.argv[0]` com os componentes do caminho inicial removidos. O argumento nomeado opcional *logoption* (o padrão é 0) é um campo de bits – veja abaixo os valores possíveis para combinar. O argumento nomeado opcional *facility* (o padrão é **`LOG_USER`**) define o recurso padrão para mensagens que não possuem um recurso explicitamente codificado.

Levanta um evento de auditoria `syslog.openlog` com os argumentos `ident`, `logoption`, `facility`.

Alterado na versão 3.2: Nas versões anteriores, os argumentos nomeados não eram permitidos e *ident* era obrigatório.

Alterado na versão 3.12: Esta função é restrita em subinterpretadores. (Apenas o código que executa em vários interpretadores é afetado e a restrição não é relevante para a maioria dos usuários.) Isso deve ser chamado no interpretador principal.Vai levantar **`RuntimeError`** se chamado em um subinterpretador.

syslog.closelog()

Redefine os valores do módulo syslog e chama a biblioteca de sistema `closelog()`.

Isso faz com que o módulo se comporte como quando importado inicialmente. Por exemplo, **`openlog()`** será chamado na primeira chamada **`syslog()`** (se **`openlog()`** ainda não foi chamado), e *ident* e outro **`openlog()`** os parâmetros são redefinidos para os padrões.

Levanta um evento de auditoria `syslog.closelog` sem argumentos.

Alterado na versão 3.12: Esta função é restrita em subinterpretadores. (Apenas o código que executa em vários interpretadores é afetado e a restrição não é relevante para a maioria dos usuários.) Isso deve ser chamado no interpretador principal.Vai levantar **`RuntimeError`** se chamado em um subinterpretador.

syslog.setlogmask(*maskpri*)

Define a máscara de prioridade como *maskpri* e retorna o valor da máscara anterior. Chamadas para **`syslog()`** com um nível de prioridade não definido em *maskpri* são ignoradas. O padrão é registrar todas as prioridades. A função `LOG_MASK(pri)` calcula a máscara para a prioridade individual *pri*. A função `LOG_UPTO(pri)` calcula a máscara para todas as prioridades até e incluindo *pri*.

Levanta um evento de auditoria `syslog.setlogmask` com o argumento `maskpri`.

O módulo define as seguintes constantes:

syslog.LOG_EMERG
syslog.LOG_ALERT
syslog.LOG_CRIT
syslog.LOG_ERR
syslog.LOG_WARNING
syslog.LOG_NOTICE
syslog.LOG_INFO
syslog.LOG_DEBUG

Níveis de prioridade (alto a baixo).

syslog.LOG_AUTH
syslog.LOG_AUTHPRIV

syslog.LOG_CRON
syslog.LOG_DAEMON
syslog.LOG_FTP
syslog.LOG_INSTALL
syslog.LOG_KERN
syslog.LOG_LAUNCHD
syslog.LOG_LPR
syslog.LOG_MAIL
syslog.LOG_NETINFO
syslog.LOG_NEWS
syslog.LOG_RAS
syslog.LOG_REMOTEAUTH
syslog.LOG_SYSLOG
syslog.LOG_USER
syslog.LOG_UUCP
syslog.LOG_LOCAL0
syslog.LOG_LOCAL1
syslog.LOG_LOCAL2
syslog.LOG_LOCAL3
syslog.LOG_LOCAL4
syslog.LOG_LOCAL5
syslog.LOG_LOCAL6
syslog.LOG_LOCAL7

Facilidades, dependendo da disponibilidade em `<syslog.h>` para **`LOG_AUTHPRIV`**, **`LOG_FTP`**, **`LOG_NETINFO`**, **`LOG_REMOTEAUTH`**, **`LOG_INSTALL`** e **`LOG_RAS`**.

Alterado na versão 3.13: Adicionados **`LOG_FTP`**, **`LOG_NETINFO`**, **`LOG_REMOTEAUTH`**, **`LOG_INSTALL`**, **`LOG_RAS`** e **`LOG_LAUNCHD`**.

syslog.LOG_PID
syslog.LOG_CONS
syslog.LOG_NDELAY
syslog.LOG_ODELAY
syslog.LOG_NOWAIT
syslog.LOG_PERROR

Opções de log, dependendo da disponibilidade em `<syslog.h>` para **`LOG_ODELAY`**, **`LOG_NOWAIT`** e **`LOG_PERROR`**.

Exemplos

Exemplo simples

Um conjunto simples de exemplos:

```
import syslog

syslog.syslog('Processing started')
if error:
    syslog.syslog(syslog.LOG_ERR, 'Processing started')
```

Um exemplo de configuração de algumas opções de log, isso incluiria o ID do processo nas mensagens registradas e escreveria as mensagens no recurso de destino usado para o log de correio:

```
syslog.openlog(logoption=syslog.LOG_PID, facility=syslog.LOG_MAI
syslog.syslog('E-mail processing initiated...')
```

Interface de linha de comando (CLI) de módulos

Os seguintes módulos têm uma interface de linha de comando.

- ast
- asyncio
- **`base64`**
- calendar
- **`code`**
- compileall
- `cProfile`: veja profiling.tracing
- dis
- doctest
- **`encodings.rot_13`**
- ensurepip
- **`filecmp`**
- **`fileinput`**
- **`ftplib`**
- gzip
- http.server
- idlelib
- inspect
- json
- mimetypes
- pdb
- pickle
- pickletools
- platform
- **`poplib`**
- profiling.sampling
- profiling.tracing
- pstats
- py_compile

- **pyclbr**
- **pydoc**
- **quopri**
- random
- **runpy**
- site
- sqlite3
- symtable
- sysconfig
- **tabnanny**
- tarfile
- **this**
- timeit
- tokenize
- trace
- **turtledemo**
- unittest
- uuid
- venv
- webbrowser
- zipapp
- zipfile

Consulte também a interface de linha de comando do Python.

Módulos substituídos

Os módulos descritos neste capítulo foram substituídos por outros módulos para a maioria dos casos de uso e são mantidos principalmente para preservar a retrocompatibilidade.

Os módulos podem aparecer neste capítulo porque eles cobrem apenas um subconjunto limitado de um espaço de problema, e uma solução mais geralmente aplicável está disponível em outro lugar na biblioteca padrão (por exemplo, **`getopt`** cobre a tarefa muito específica de "imitar a API C **`getopt()`** no Python", em vez dos recursos mais amplos de análise de opções de linha de comando e análise de argumentos oferecidos por **`optparse`** e **`argparse`**).

Alternativamente, módulos podem aparecer neste capítulo porque estão totalmente descontinuados e aguardando remoção em uma versão futura, ou estão suavemente descontinuados e seu uso é ativamente desencorajado em novos projetos. Com a remoção de vários módulos obsoletos por meio da **PEP 594** [https://peps.python.org/pep-0594/], atualmente não há módulos nesta última categoria.

- **`getopt`** — Analisador sintático no estilo C para opções de linha de comando
- **`profile`** — Pure Python profiler

`getopt` — Analisador sintático no estilo C para opções de linha de comando

Código-fonte: Lib/getopt.py [https://github.com/python/cpython/tree/3.15/Lib/getopt.py]

> **Nota**
>
> Este módulo é considerado completo em recursos. Uma alternativa mais declarativa e extensível para esta API é fornecida no módulo **`optparse`**. Outros aprimoramentos funcionais para processamento de parâmetros de linha de comando são fornecidos como módulos de terceiros no PyPI ou como recursos no módulo **`argparse`**.

Este módulo ajuda os scripts a analisar os argumentos da linha de comando em `sys.argv`. Ele suporta as mesmas convenções da função Unix **`getopt()`** (incluindo os significados especiais de argumentos da forma '`-`' e '`--`'). Longas opções semelhantes às suportadas pelo software GNU também podem ser usadas por meio de um terceiro argumento opcional.

Usuários que não estão familiarizados com a função Unix **`getopt()`** devem considerar usar o módulo **`argparse`** em vez disso. Usuários que estão familiarizados com a função Unix **`getopt()`**, mas gostariam de obter um comportamento equivalente enquanto escrevem menos código e obtêm melhor ajuda e mensagens de erro devem considerar usar o módulo **`optparse`**. Veja Escolhendo uma biblioteca de análise de linha de comando para detalhes adicionais.

Este módulo fornece duas funções e uma exceção:

getopt.getopt(*args*, *shortopts*, *longopts=[]*)

Analisa opções de linha de comando e lista de parâmetros. *args* é a lista de argumentos a ser analisada, sem a referência inicial para o programa em execução. Normalmente, isso significa `sys.argv[1:]`. *shortopts* é a string de letras de opção que o script deseja reconhecer, com opções que requerem um argumento seguido por dois pontos (`':'`) e opções que aceitam o argumento opcional seguido por dois caracteres de dois pontos (`'::'`); ou seja, o mesmo formato que Unix **`getopt()`** usa.

Nota

Ao contrário do GNU **`getopt()`**, após um argumento sem opção, todos os argumentos adicionais são considerados também sem opção. Isso é semelhante à maneira como os sistemas Unix não GNU funcionam.

longopts, se especificado, deve ser uma lista de strings com os nomes das opções longas que devem ser suportadas. Os caracteres `'--'` no início não devem ser incluídos no nome da opção. Opções longas que requerem um argumento devem ser seguidas por um sinal de igual (`'='`). Opções longas que aceitam um argumento opcional devem ser seguidas por um sinal de igualdade e ponto de interrogação (`'=?'`). Para aceitar apenas opções longas, *shortopts* deve ser uma string vazia. Opções longas na linha de comando podem ser reconhecidas, desde que forneçam um prefixo do nome da opção que corresponda exatamente a uma das opções aceitas. Por exemplo, se *longopts* for `['foo', 'frob']`, a opção `--fo` irá corresponder a `--foo`, mas `--f` não corresponderá exclusivamente, então **`GetoptError`** será levantada.

Se *longopts* for uma string, ela é tratada como uma lista de um único elemento.

O valor de retorno consiste em dois elementos: o primeiro é uma lista de pares `(option, value)`; a segunda é a lista de argumentos de programa restantes depois que a lista de opções foi removida (esta é uma fatia ao final de *args*). Cada par de opção e valor retornado tem a opção como seu primeiro elemento, prefixado com um hífen para opções curtas (por exemplo, `'-x'`) ou dois hifenes para opções longas (por exemplo, `'--`

`long-option'`), e o argumento da opção como seu segundo elemento, ou uma string vazia se a opção não tiver argumento. As opções ocorrem na lista na mesma ordem em que foram encontradas, permitindo assim múltiplas ocorrências. Opções longas e curtas podem ser misturadas.

Alterado na versão 3.14: Argumentos opcionais são suportados.

getopt.gnu_getopt(*args*, *shortopts*, *longopts=[]*)

Esta função funciona como **`getopt()`**, exceto que o modo de digitalização do estilo GNU é usado por padrão. Isso significa que os argumentos de opção e não opção podem ser misturados. A função **`getopt()`** interrompe o processamento das opções assim que um argumento não opcional é encontrado.

Se o primeiro caractere da string de opção for `'+'`, ou se a variável de ambiente **`POSIXLY_CORRECT`** estiver definida, então o processamento da opção para assim que um argumento não opcional for encontrado.

Se o primeiro caractere da string de opção for `'-'`, argumentos não opcionais seguidos por opções serão adicionados à lista de pares opção-valor como um par que tem `None` como primeiro elemento e a lista de argumentos não opcionais como segundo elemento. O segundo elemento do resultado **`gnu_getopt()`** é uma lista de argumentos do programa após a última opção.

Alterado na versão 3.14: Suporte para retornar opções misturadas e argumentos não opcionais em ordem.

exception getopt.GetoptError

Isso é levantado quando uma opção não reconhecida é encontrada na lista de argumentos ou quando uma opção que requer um argumento não é fornecida. O argumento para a exceção é uma string que indica a causa do erro. Para opções longas, um argumento dado a uma opção que não requer uma também fará com que essa exceção seja levantada. Os atributos **`msg`** e **`opt`** fornecem a mensagem de erro e a opção relacionada; se não houver uma opção específica à qual a exceção se relaciona, **`opt`** é uma string vazia.

exception getopt.error

Apelido para **GetoptError**; para compatibilidade reversa.

Um exemplo usando apenas opções de estilo Unix:

```
>>> import getopt
>>> args = '-a -b -cfoo -d bar a1 a2'.split()
>>> args
['-a', '-b', '-cfoo', '-d', 'bar', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'abc:d:')
>>> optlist
[('-a', ''), ('-b', ''), ('-c', 'foo'), ('-d', 'bar')]
>>> args
['a1', 'a2']
```

Usar nomes de opções longos é igualmente fácil:

```
>>> s = '--condition=foo --testing --output-file abc.def -x a1 a
>>> args = s.split()
>>> args
['--condition=foo', '--testing', '--output-file', 'abc.def', '-x
>>> optlist, args = getopt.getopt(args, 'x', [
...     'condition=', 'output-file=', 'testing'])
>>> optlist
[('--condition', 'foo'), ('--testing', ''), ('--output-file', 'a
>>> args
['a1', 'a2']
```

Argumentos opcionais devem ser especificados explicitamente:

```
>>> s = '-Con -C --color=off --color a1 a2'
>>> args = s.split()
>>> args
['-Con', '-C', '--color=off', '--color', 'a1', 'a2']
>>> optlist, args = getopt.getopt(args, 'C::', ['color=?'])
>>> optlist
[('-C', 'on'), ('-C', ''), ('--color', 'off'), ('--color', '')]
>>> args
['a1', 'a2']
```

A ordem dos argumentos opcionais e não opcionais pode ser preservada:

```
>>> s = 'a1 -x a2 a3 a4 --long a5 a6'
>>> args = s.split()
>>> args
['a1', '-x', 'a2', 'a3', 'a4', '--long', 'a5', 'a6']
>>> optlist, args = getopt.gnu_getopt(args, '-x:', ['long='])
>>> optlist
[(None, ['a1']), ('-x', 'a2'), (None, ['a3', 'a4']), ('--long',
>>> args
['a6']
```

Em um script, o uso típico é algo assim:

```
import getopt, sys

def main():
    try:
        opts, args = getopt.getopt(sys.argv[1:], "ho:v", ["help'
    except getopt.GetoptError as err:
        # print help information and exit:
        print(err)  # will print something like "option -a not r
        usage()
        sys.exit(2)
    output = None
    verbose = False
    for o, a in opts:
        if o == "-v":
            verbose = True
        elif o in ("-h", "--help"):
            usage()
            sys.exit()
        elif o in ("-o", "--output"):
            output = a
        else:
            assert False, "unhandled option"
    process(args, output=output, verbose=verbose)

if __name__ == "__main__":
    main()
```

Observe que uma interface de linha de comando equivalente pode ser produzida com menos código e mais mensagens de erro de ajuda e erro

informativas usando o módulo **`optparse`**:

```
import optparse

if __name__ == '__main__':
    parser = optparse.OptionParser()
    parser.add_option('-o', '--output')
    parser.add_option('-v', dest='verbose', action='store_true')
    opts, args = parser.parse_args()
    process(args, output=opts.output, verbose=opts.verbose)
```

Uma interface de linha de comando aproximadamente equivalente para este caso também pode ser produzida usando o módulo **`argparse`**:

```
import argparse

if __name__ == '__main__':
    parser = argparse.ArgumentParser()
    parser.add_argument('-o', '--output')
    parser.add_argument('-v', dest='verbose', action='store_true
    parser.add_argument('rest', nargs='*')
    args = parser.parse_args()
    process(args.rest, output=args.output, verbose=args.verbose)
```

Veja Escolhendo uma biblioteca de análise de linha de comando para detalhes sobre como a versão `argparse` deste código difere em comportamento da versão `optparse` (e `getopt`).

Ver também

Módulo `optparse`
Análise de opções de linha de comando declarativa.

Módulo `argparse`
Opção de linha de comando e biblioteca de análise de argumentos mais opinativa.

`profile` — Pure Python profiler

Source code: Lib/profile.py [https://github.com/python/cpython/tree/3.15/Lib/profile.py]

Descontinuado desde a versão 3.15, será removido na versão 3.17.

The **`profile`** module is deprecated and will be removed in Python 3.17. Use **`profiling.tracing`** instead.

The **`profile`** module provides a pure Python implementation of a deterministic profiler. While useful for understanding profiler internals or extending profiler behavior through subclassing, its pure Python implementation introduces significant overhead compared to the C-based **`profiling.tracing`** module.

For most profiling tasks, use:

- **`profiling.sampling`** for production debugging with zero overhead
- **`profiling.tracing`** for development and testing

Migration

Migrating from **`profile`** to **`profiling.tracing`** is straightforward. The APIs are compatible:

```
# Old (deprecated)
import profile
profile.run('my_function()')

# New (recommended)
import profiling.tracing
profiling.tracing.run('my_function()')
```

For most code, replacing `import profile` with `import profiling.tracing` (and using `profiling.tracing` instead of `profile`

throughout) provides a straightforward migration path.

Nota

The `cProfile` module remains available as a backward-compatible alias to `profiling.tracing`. Existing code using `import cProfile` will continue to work without modification.

`profile` and `profiling.tracing` module reference

Both the `profile` and `profiling.tracing` modules provide the following functions:

profile.run(*command*, *filename=None*, *sort=-1*)

This function takes a single argument that can be passed to the `exec()` function, and an optional file name. In all cases this routine executes:

```
exec(command, __main__.__dict__, __main__.__dict__)
```

and gathers profiling statistics from the execution. If no file name is present, then this function automatically creates a `Stats` instance and prints a simple profiling report. If the sort value is specified, it is passed to this `Stats` instance to control how the results are sorted.

profile.runctx(*command*, *globals*, *locals*, *filename=None*, *sort=-1*)

This function is similar to `run()`, with added arguments to supply the globals and locals mappings for the *command* string. This routine executes:

```
exec(command, globals, locals)
```

and gathers profiling statistics as in the `run()` function above.

class profile.Profile(*timer=None*, *timeunit=0.0*, *subcalls=True*, *builtins=True*)

This class is normally only used if more precise control over profiling is needed than what the **`profiling.tracing.run()`** function provides.

A custom timer can be supplied for measuring how long code takes to run via the *timer* argument. This must be a function that returns a single number representing the current time. If the number is an integer, the *timeunit* specifies a multiplier that specifies the duration of each unit of time. For example, if the timer returns times measured in thousands of seconds, the time unit would be `.001`.

Directly using the **`Profile`** class allows formatting profile results without writing the profile data to a file:

```
import profiling.tracing
import pstats
import io
from pstats import SortKey

pr = profiling.tracing.Profile()
pr.enable()
# ... do something ...
pr.disable()
s = io.StringIO()
sortby = SortKey.CUMULATIVE
ps = pstats.Stats(pr, stream=s).sort_stats(sortby)
ps.print_stats()
print(s.getvalue())
```

The **`Profile`** class can also be used as a context manager (supported only in **`profiling.tracing`**, not in the deprecated **`profile`** module; see Tipos de Gerenciador de Contexto):

```
import profiling.tracing

with profiling.tracing.Profile() as pr:
    # ... do something ...

    pr.print_stats()
```

Alterado na versão 3.8: Adicionado suporte a gerenciador de contexto.

enable()

Start collecting profiling data. Only in `profiling.tracing`.

disable()

Stop collecting profiling data. Only in `profiling.tracing`.

create_stats()

Stop collecting profiling data and record the results internally as the current profile.

print_stats(*sort=-1*)

Cria um objeto `Stats` com base no perfil atual e imprime os resultados para stdout.

The *sort* parameter specifies the sorting order of the displayed statistics. It accepts a single key or a tuple of keys to enable multi-level sorting, as in `pstats.Stats.sort_stats()`.

Adicionado na versão 3.13: **`print_stats()`** now accepts a tuple of keys.

dump_stats(*filename*)

Escreve os resultados do profile atual para *filename*.

run(*cmd*)

Profile the cmd via `exec()`.

runctx(*cmd*, *globals*, *locals*)

Profile the cmd via `exec()` with the specified global and local environment.

runcall(*func*, /, **args*, ***kwargs*)

Profile `func(*args, **kwargs)`

Note that profiling will only work if the called command/function actually returns. If the interpreter is terminated (e.g. via a `sys.exit()` call during

the called command/function execution) no profiling results will be printed.

Differences from `profiling.tracing`

The `profile` module differs from `profiling.tracing` in several ways:

Higher overhead. The pure Python implementation is significantly slower than the C implementation, making it unsuitable for profiling long-running programs or performance-sensitive code.

Calibration support. The `profile` module supports calibration to compensate for profiling overhead. This is not needed in `profiling.tracing` because the C implementation has negligible overhead.

Custom timers. Both modules support custom timers, but `profile` accepts timer functions that return tuples (like `os.times()`), while `profiling.tracing` requires a function returning a single number.

Subclassing. The pure Python implementation is easier to subclass and extend for custom profiling behavior.

What is deterministic profiling?

Deterministic profiling is meant to reflect the fact that all *function call*, *function return*, and *exception* events are monitored, and precise timings are made for the intervals between these events (during which time the user's code is executing). In contrast, *statistical profiling* (which is provided by the `profiling.sampling` module) periodically samples the effective instruction pointer, and deduces where time is being spent. The latter technique traditionally involves less overhead (as the code does not need to be instrumented), but provides only relative indications of where time is being spent.

In Python, since there is an interpreter active during execution, the presence of instrumented code is not required in order to do deterministic profiling. Python automatically provides a *hook* (optional callback) for each event. In addition, the interpreted nature of Python tends to add so much overhead to

execution, that deterministic profiling tends to only add small processing overhead in typical applications. The result is that deterministic profiling is not that expensive, yet provides extensive run time statistics about the execution of a Python program.

Call count statistics can be used to identify bugs in code (surprising counts), and to identify possible inline-expansion points (high call counts). Internal time statistics can be used to identify “hot loops” that should be carefully optimized. Cumulative time statistics should be used to identify high level errors in the selection of algorithms. Note that the unusual handling of cumulative times in this profiler allows statistics for recursive implementations of algorithms to be directly compared to iterative implementations.

Limitações

One limitation has to do with accuracy of timing information. There is a fundamental problem with deterministic profilers involving accuracy. The most obvious restriction is that the underlying “clock” is only ticking at a rate (typically) of about .001 seconds. Hence no measurements will be more accurate than the underlying clock. If enough measurements are taken, then the “error” will tend to average out. Unfortunately, removing this first error induces a second source of error.

The second problem is that it “takes a while” from when an event is dispatched until the profiler’s call to get the time actually *gets* the state of the clock. Similarly, there is a certain lag when exiting the profiler event handler from the time that the clock’s value was obtained (and then squirreled away), until the user’s code is once again executing. As a result, functions that are called many times, or call many functions, will typically accumulate this error. The error that accumulates in this fashion is typically less than the accuracy of the clock (less than one clock tick), but it *can* accumulate and become very significant.

The problem is more important with the deprecated **`profile`** module than with the lower-overhead `profiling.tracing`. For this reason, **`profile`** provides a means of calibrating itself for a given platform so that this error

can be probabilistically (on the average) removed. After the profiler is calibrated, it will be more accurate (in a least square sense), but it will sometimes produce negative numbers (when call counts are exceptionally low, and the gods of probability work against you :-).) Do *not* be alarmed by negative numbers in the profile. They should *only* appear if you have calibrated your profiler, and the results are actually better than without calibration.

Calibration

The profiler of the **`profile`** module subtracts a constant from each event handling time to compensate for the overhead of calling the time function, and socking away the results. By default, the constant is 0. The following procedure can be used to obtain a better constant for a given platform (see Limitações).

```
import profile
pr = profile.Profile()
for i in range(5):
    print(pr.calibrate(10000))
```

The method executes the number of Python calls given by the argument, directly and again under the profiler, measuring the time for both. It then computes the hidden overhead per profiler event, and returns that as a float. For example, on a 1.8Ghz Intel Core i5 running macOS, and using Python's time.process_time() as the timer, the magical number is about 4.04e-6.

The object of this exercise is to get a fairly consistent result. If your computer is *very* fast, or your timer function has poor resolution, you might have to pass 100000, or even 1000000, to get consistent results.

When you have a consistent answer, there are three ways you can use it:

```
import profile

# 1. Apply computed bias to all Profile instances created hereaf
profile.Profile.bias = your_computed_bias

# 2. Apply computed bias to a specific Profile instance.
```

```
pr = profile.Profile()
pr.bias = your_computed_bias

# 3. Specify computed bias in instance constructor.
pr = profile.Profile(bias=your_computed_bias)
```

If you have a choice, you are better off choosing a smaller constant, and then your results will “less often” show up as negative in profile statistics.

Using a custom timer

If you want to change how current time is determined (for example, to force use of wall-clock time or elapsed process time), pass the timing function you want to the **`Profile`** class constructor:

```
pr = profile.Profile(your_time_func)
```

The resulting profiler will then call `your_time_func`. Depending on whether you are using **`profile.Profile`** or **`profiling.tracing.Profile`**, `your_time_func`’s return value will be interpreted differently:

`profile.Profile`

> `your_time_func` should return a single number, or a list of numbers whose sum is the current time (like what **`os.times()`** returns). If the function returns a single time number, or the list of returned numbers has length 2, then you will get an especially fast version of the dispatch routine.
>
> Be warned that you should calibrate the profiler class for the timer function that you choose (see Calibration). For most machines, a timer that returns a lone integer value will provide the best results in terms of low overhead during profiling. (**`os.times()`** is *pretty* bad, as it returns a tuple of floating-point values). If you want to substitute a better timer in the cleanest fashion, derive a class and hardwire a replacement dispatch method that best handles your timer call, along with the appropriate calibration constant.

profiling.tracing.Profile

`your_time_func` should return a single number. If it returns integers, you can also invoke the class constructor with a second argument specifying the real duration of one unit of time. For example, if `your_integer_time_func` returns times measured in thousands of seconds, you would construct the **Profile** instance as follows:

```
pr = profiling.tracing.Profile(your_integer_time_func, 0.001)
```

As the **profiling.tracing.Profile** class cannot be calibrated, custom timer functions should be used with care and should be as fast as possible. For the best results with a custom timer, it might be necessary to hard-code it in the C source of the internal **_lsprof** module.

Python 3.3 adds several new functions in **time** that can be used to make precise measurements of process or wall-clock time. For example, see **time.perf_counter()**.

Ver também

profiling

Overview of Python profiling tools.

profiling.tracing

Recommended replacement for this module.

pstats

Statistical analysis and formatting for profile data.

Módulos removidos

Os módulos descritos neste capítulo foram removidos da biblioteca padrão do Python. Eles estão documentados aqui para ajudar as pessoas a encontrarem substituições.

- **aifc** — Lê e escreve arquivos AIFF e AIFC
- **asynchat** — Manipulador de comando/resposta de socket assíncrono
- **asyncore** — Manipulador de socket assíncrono
- **audioop** — Manipulando dados de áudio original
- **cgi** — Suporte a Common Gateway Interface
- **cgitb** — Gerenciador de traceback para scripts de CGI
- **chunk** — Lê dados em blocos IFF
- **crypt** — Função para verificar senhas Unix
- **distutils** — Criação e instalação de módulos do Python
- **imghdr** — Determina o tipo de uma imagem
- **imp** — Acessa os componentes internos de importação
- **mailcap** — Manipulação de arquivo Mailcap
- **msilib** — Lê e escreve arquivos do Microsoft Installer
- **nis** — Interface para NIS da Sun (Yellow Pages)
- **nntplib** — Cliente de protocolo NNTP
- **ossaudiodev** — Acesso a dispositivos de áudio compatíveis com OSS
- **pipes** — Interface para pipelines do shell
- **smtpd** — Serviços SMTP
- **sndhdr** — Determina o tipo de arquivos de som
- **spwd** — O banco de dados de senhas shadow
- **sunau** — Lê e escreve arquivos AU da Sun
- **telnetlib** — cliente Telnet
- **uu** — Codifica e decodifica arquivos uuencode
- **xdrlib** — Codifica e decodifica dados XDR

Considerações de segurança

Os módulos a seguir têm considerações de segurança específicas:

- `base64`: considerações de segurança do base64 na **RFC 4648** [https://datatracker.ietf.org/doc/html/rfc4648.html]
- `hashlib`: todos os construtores usam um argumento somente-nomeado "usedforsecurity", desativando algoritmos conhecidos não seguros e bloqueados
- `http.server` não é adequado para uso em produção, implementando apenas verificações básicas de segurança. Veja as considerações de segurança.
- `logging`: configuração do Logging usa eval()
- `multiprocessing`: Connection.recv() usa pickle
- `pickle`: Restringindo globais no pickle
- `random` não deve ser usado para o propósito de segurança. Em vez disso, use `secrets`
- `shelve`: shelve é baseado no pickle e, portanto, inadequado para lidar com fontes não confiáveis
- `ssl`: Considerações de segurança de SSL/TLS
- `subprocess`: Considerações de segurança de Subprocess
- `tempfile`: mktemp foi descontinuado em razão de vulnerabilidade de condições de corrida
- `xml`: Segurança no XML
- `zipfile`: Arquivos .zip preparados de forma maliciosa podem causar esgotamento do volume do disco

A opção de linha de comando `-I` pode ser usada para executar o Python no modo isolado. Quando não pode ser usado, a opção `-P` ou a variável de ambiente `PYTHONSAFEPATH` pode ser usada para não preceder um caminho potencialmente inseguro para `sys.path` como o diretório atual, o diretório do script ou uma string vazia.

Estendendo e incorporando o interpretador Python

Este documento descreve o desenvolvimento de módulos com C ou C++ para adicionar recursos ao interpretador Python criando novos módulos. Esses módulos podem fazer o que o código Python faz – definir funções, tipos de objeto e métodos –, mas também interagir com bibliotecas nativas ou obter melhor desempenho ao evitar a sobrecarga de um interpretador. O documento também descreve como embutir o interpretador Python em outra aplicação, de forma a utilizá-lo como sendo uma linguagem de extensão. Por fim, é mostrado como se pode compilar e fazer a vinculação dos módulos de extensão para que estes possam ser carregados dinamicamente (em tempo de execução) pelo interpretador, caso o sistema operacional subjacente suportar esse recurso.

Este documento pressupõe conhecimentos básicos sobre C e Python. Para uma introdução informal ao Python, consulte O tutorial do Python. A Referência da Linguagem Python fornece uma definição mais formal da linguagem. Referência da bibliotecas embutidas do Python documenta as funções embutidas e tipos de objetos, e A biblioteca padrão do Python documenta os módulos de objetos existentes (embutidos e escritos em Python) que dão à linguagem sua ampla gama de aplicações.

Para uma descrição detalhada de toda a API Python/C, consulte o Manual de referência da API C/Python separado.

Para dar suporte a extensões, a API do Python API (Application Programmers Interface) define um conjunto de funções, macros e variáveis que fornecem acesso à maior parte dos aspectos do sistema de tempo de execução do Python. A API do Python pode ser incorporada em um arquivo fonte em C com a inclusão do cabeçalho `"Python.h"`.

Nota

A interface de extensões em C é específica para o CPython, e módulos de extensão não funcionam em outras implementações do Python. Em muitos casos, é possível evitar a criação destas extensões em C e preservar a portabilidade para outras implementações. Por exemplo, se o seu caso de uso for o de fazer chamadas a funções em bibliotecas C ou chamadas de sistema, considere utilizar o módulo `ctypes` ou a biblioteca cffi [https://cffi.readthedocs.io/] ao invés de escrever código C personalizado. Esses módulos permitem escrever código Python que pode interoperar com código C e que é mais portável entre implementações do Python do que escrever e compilar um módulo de extensão em C.

Ferramentas de terceiros recomendadas

Este documenta aborda apenas as ferramentas básicas para a criação de extensões fornecidas como parte desta versão do CPython. Algumas ferramentas de terceiros oferecem abordagens mais simples e mais sofisticadas para a criação de extensões em C e C++ para Python.

Embora este documento seja voltado para autores de extensões, ele também deve ser útil para os autores de tais ferramentas. Por exemplo, o módulo de tutorial pode servir como um caso de teste simples para uma ferramenta de build ou como uma amostra da saída esperada de um gerador de código.

Tutorial da API C

Este tutorial descreve como escrever um módulo simples em C ou C++ usando a API C do Python — isto é, utilizando as ferramentas básicas fornecidas como parte desta versão do CPython.

1. Seu primeiro módulo de extensão da API C

Guias para tópicos intermediários

Esta seção do guia aborda a criação de extensões C e C++ sem assistência de ferramentas de terceiros. Destina-se principalmente aos criadores dessas

ferramentas, em vez de ser uma maneira recomendada de criar suas próprias extensões C.

- Using the C API: Assorted topics
- Definindo Tipos de Extensão: Tutorial
- Defining Extension Types: Assorted Topics
- Construindo extensões C e C++
- Construindo Extensões C e C++ no Windows

Incorporando o tempo de execução do CPython em uma aplicação maior

Às vezes, em vez de criar uma extensão que é executada dentro do interpretador Python como a aplicação principal, é desejável incorporar o tempo de execução do CPython em uma aplicação maior. Esta seção aborda alguns dos detalhes envolvidos para fazer isso com êxito.

- Incorporando o Python numa Outra Aplicação

Seu primeiro módulo de extensão da API C

Este tutorial guiará você pela criação de um módulo de extensão simples para Python, escrito em C ou C++.

Utilizaremos diretamente a API C do Python de baixo nível. Para formas mais simples de criar módulos de extensão, consulte as ferramentas de terceiros recomendadas.

O tutorial pressupõe conhecimentos básicos de Python: você deve ser capaz de definir funções em código Python antes de começar a escrevê-las em C. Consulte O tutorial do Python para uma introdução ao próprio Python.

O tutorial deve ser acessível a qualquer pessoa capaz de escrever uma biblioteca básica em C. Embora mencionemos vários conceitos que não se espera que um iniciante em C conheça — como funções *static* ou declarações de vinculação (*linkage*) —, compreendê-los não é necessário para obter êxito.

Nosso foco será proporcionar a você uma noção de como é a API C do Python. Isso não ensinará conceitos importantes, como tratamento de erros e contagem de referências, que são abordados em capítulos posteriores.

Vamos supor que você utilize um sistema do tipo Unix (incluindo macOS e Linux) ou Windows. Em outros sistemas, talvez seja necessário ajustar alguns detalhes — por exemplo, o nome de um comando do sistema.

Você precisa ter um compilador C adequado e os cabeçalhos de desenvolvimento do Python instalados. No Linux, esses cabeçalhos geralmente estão em um pacote como `python3-dev` ou `python3-devel`.

Você precisa ser capaz de instalar pacotes Python. Este tutorial utiliza o *pip <https://pip.pypa.io/>`__* (`pip install`), mas você pode substituí-lo por

qualquer ferramenta capaz de realizar a construção e a instalação de projetos baseados em `pyproject.toml`, como uv [https://docs.astral.sh/uv/] (`uv setup.py install`). De preferência, mantenha um ambiente virtual ativado.

Nota

Este tutorial utiliza APIs que foram adicionadas no CPython 3.15. Para criar uma extensão compatível com versões anteriores do CPython, consulte uma versão anterior desta documentação.

Este tutorial utiliza sintaxe de C introduzida no C11 e no C++20. Se a sua extensão precisar ser compatível com padrões anteriores, consulte os tutoriais na documentação para o Python 3.14 ou versões anteriores.

O que faremos

Vamos criar um módulo de extensão chamado `spam` [[1]], que incluirá uma interface Python para a função **`system()`** da biblioteca padrão C. Essa função é definida em `stdlib.h`. Ela recebe uma string C como argumento, executa o argumento como um comando do sistema e retorna um valor de resultado como um inteiro. Uma página de manual para **`system()`** poderia resumi-la da seguinte forma:

```
#include <stdlib.h>
int system(const char *command);
```

Observe que, assim como muitas funções da biblioteca padrão da linguagem C, esta função já está disponível no Python. Em ambientes de produção, utilize **`os.system()`** ou **`subprocess.run()`** em vez do módulo que você escreverá aqui.

Queremos que esta função seja chamável a partir do Python da seguinte forma:

```
>>> import spam
>>> status = spam.system("whoami")
```

```
Nome do usuário
>>> status
0
```

Nota

O comando de sistema *whoami* exibe o seu nome de usuário. Ele é útil em tutoriais como este porque possui o mesmo nome tanto no Unix quanto no Windows.

Comece com os cabeçalhos

Comece criando um diretório para este tutorial e acessando-o na linha de comando. Em seguida, crie um arquivo chamado `spammodule.c` no seu diretório. [[2]]

Neste arquivo, incluiremos dois cabeçalhos: `Python.h`, para importar todas as declarações da API C do Python, e `stdlib.h`, para a função **`system()`**. [[3]]

Adicione as seguintes linhas a `spammodule.c`:

```
#include <Python.h>
#include <stdlib.h>     // para system()
```

Certifique-se de colocar `stdlib.h` — e quaisquer outras inclusões à biblioteca padrão — *após* `Python.h`. Em alguns sistemas, o Python pode definir diretivas de pré-processador que afetam os cabeçalhos padrão.

Executando sua ferramenta de construção

Com apenas as diretivas de inclusões (includes) implementadas, sua extensão não fará nada. Ainda assim, é um bom momento para compilá-la e tentar importá-la. Isso garantirá que sua ferramenta de construção esteja funcionando, permitindo que você faça e teste alterações incrementais à medida que acompanha o restante do texto.

O próprio CPython não inclui uma ferramenta para construir módulos de extensão; recomenda-se utilizar um projeto de terceiros para essa finalidade. Neste tutorial, utilizaremos o meson-python [https://mesonbuild.com/meson-python/]. (Caso queira utilizar outra ferramenta, consulte Appendix: Other build tools.)

O `meson-python` requer a definição de um "projeto" utilizando dois arquivos extras.

Primeiro, adicione um arquivo `pyproject.toml` com o seguinte conteúdo:

```
[build-system]
build-backend = 'mesonpy'
requires = ['meson-python']

[project]
# Placeholders de informações de projeto
# (altere isso antes de distribuir o módulo)
name = 'projetoexemplo'
version = '0'
```

Então, cria `meson.build` contendo o seguinte:

```
project('projetoexemplo', 'c')

py = import('python').find_installation(pure: false)

py.extension_module(
   'spam',          # nome do módulo Python importável
   'spammodule.c',  # o arquivo-fonte C
   install: true,
)
```

Nota

Consulte a documentação do meson-python [https://mesonbuild.com/meson-python/] para detalhes sobre a configuração.

Agora, construa e instale o *projeto no diretório atual* (`.`) via `pip`:

```
python -m pip -v install .
```

A opção `-v` (`--verbose`) faz com que o `pip` mostre a saída do compilador, o que é frequentemente útil durante o desenvolvimento.

Dica

Se você não tiver o `pip` instalado, execute `python -m ensurepip`, de preferência em um ambiente virtual. (Ou, se preferir outra ferramenta capaz de construir e instalar projetos baseados em `pyproject.toml`, utilize-a.)

Observe que precisará de executar este comando novamente sempre que alterar a sua extensão. Ao contrário do Python, o C tem um passo de compilação explícito.

Quando a sua extensão estiver compilada e instalada, inicie o Python e tente importá-la. Isto deverá falhar com a seguinte exceção:

```
>>> import spam
Traceback (most recent call last):
   ...
ImportError: dynamic module does not define module export functi
```

Gancho de exportação de módulo

A exceção que você recebeu ao tentar importar o módulo indicava que o Python está procurando uma “função de exportação de módulo”, também conhecida como gancho de exportação de módulo. Vamos definir uma.

Primeiro, adicione um protótipo embaixo das linhas `#include`:

```
PyMODEXPORT_FUNC PyModExport_spam(void);
```

Dica

O protótipo não é estritamente necessário, mas alguns compiladores modernos emitem avisos se ele não estiver presente. Geralmente, é melhor adicionar o protótipo do que desativar o aviso.

A macro `PyMODEXPORT_FUNC` declara o tipo de retorno da função e adiciona quaisquer declarações de vinculação especiais necessárias para tornar a função visível e utilizável quando o CPython a carrega.

Após o protótipo, adicione a função em si. Por enquanto, faça com que ela retorne `NULL`:

```
PyMODEXPORT_FUNC
PyModExport_spam(void)
{
   return NULL;
}
```

Compile e carregue o módulo novamente. Você deverá receber um erro diferente desta vez.

```
>>> import spam
Traceback (most recent call last):
   ...
SystemError: module export hook for module 'spam' failed without
```

Simplesmente retornar `NULL` *não* é o comportamento correto para um gancho de exportação, e o CPython reclama disso. Isso é bom — significa que o CPython encontrou a função! Agora, vamos fazer com que ela realize alguma tarefa útil.

A tabela de slots

Em vez de `NULL`, o gancho de exportação deve retornar as informações necessárias para criar um módulo. Vamos começar pelo básico: o nome e a docstring.

As informações devem ser definidas em um vetor de entradas **PySlot**, que são essencialmente pares chave-valor. Defina esse array logo antes do seu gancho de exportação:

```
PyABIInfo_VAR(abi_info);

static PySlot spam_slots[] = {
   PySlot_STATIC_DATA(Py_mod_abi, &abi_info),
   PySlot_STATIC_DATA(Py_mod_name, "spam"),
   PySlot_STATIC_DATA(Py_mod_doc, "Um módulo maravilhoso com um
   PySlot_END
};
```

A macro **PySlot_STATIC_DATA** é usada quando o valor do slot (aqui: `&abi_info`, `"spam"` e a docstring) é um ponteiro para dados constantes alocados estaticamente.

A macro `PyABIInfo_VAR(abi_info);` e o slot **Py_mod_abi** constituem um código padrão que ajuda a evitar que extensões compiladas para uma versão diferente do Python causem a falha do interpretador.

Para ambos **Py_mod_name** e **Py_mod_doc**, os valores são strings C — isto é, vetores de bytes terminados em NUL e codificados em UTF-8.

Observe a entrada sentinela `PySlot_END` ao final. Ela marca o fim do vetor. Se você esquecê-la, provocará um comportamento indefinido.

The array is defined as `static` – that is, not visible outside this `.c` file. This will be a common theme. CPython only needs to access the export hook; all global variables and all other functions should generally be `static`, so that they don't clash with other extensions.

Return this array from your export hook instead of `NULL`:

```
PyMODEXPORT_FUNC
PyModExport_spam(void)
{
   return spam_slots;
}
```

Now, recompile and try it out:

```
>>> import spam
>>> print(spam)
<module 'spam' from '/home/encukou/dev/cpython/spam.so'>
```

You have an extension module! Try `help(spam)` to see the docstring.

The next step will be adding a function.

Exposing a function

To expose the **`system()`** C function directly to Python, we'll need to write a layer of glue code to convert arguments from Python objects to C values, and the C return value back to Python.

One of the simplest ways to write glue code is a "**`METH_O`**" function, which takes two Python objects and returns one. All Python objects – regardless of the Python type – are represented in C as pointers to the **`PyObject`** structure.

Add such a function above the slots array:

```
static PyObject *
spam_system(PyObject *self, PyObject *arg)
{
   Py_RETURN_NONE;
}
```

For now, we ignore the arguments, and use the **`Py_RETURN_NONE`** macro, which expands to a `return` statement that properly returns a Python **`None`** object.

Recompile your extension to make sure you don't have syntax errors. We haven't yet added `spam_system` to the module, so you might get a warning that `spam_system` is unused.

Method definitions

To expose the C function to Python, you will need to provide several pieces of information in a structure called **`PyMethodDef`** [[4]]:

- `ml_name`: the name of the Python function;
- `ml_doc`: a docstring;
- `ml_meth`: the C function to be called; and
- `ml_flags`: a set of flags describing details like how Python arguments are passed to the C function. We'll use **`METH_O`** here – the flag that matches our `spam_system` function's signature.

Because modules typically create several functions, these definitions need to be collected in an array, with a zero-filled sentinel at the end. Add this array just below the `spam_system` function:

```
static PyMethodDef spam_methods[] = {
    {
        .ml_name="system",
        .ml_meth=spam_system,
        .ml_flags=METH_O,
        .ml_doc="Execute a shell command.",
    },
    {NULL, NULL, 0, NULL}        /* Sentinel */
};
```

As with module slots, a zero-filled sentinel marks the end of the array.

Next, we'll add the method to the module. Add a **`Py_mod_methods`** slot to your **`PyMethodDef`** array:

```
PyABIInfo_VAR(abi_info);

static PySlot spam_slots[] = {
    PySlot_STATIC_DATA(Py_mod_abi, &abi_info),
    PySlot_STATIC_DATA(Py_mod_name, "spam"),
    PySlot_STATIC_DATA(Py_mod_doc, "A wonderful module with an e
    PySlot_STATIC_DATA(Py_mod_methods, spam_methods),
    PySlot_END
```

```
};
```

Recompile your extension again, and test it. Be sure to restart the Python interpreter, so that `import spam` picks up the new version of the module.

You should now be able to call the function:

```
>>> import spam
>>> print(spam.system)
<built-in function system>
>>> print(spam.system('whoami'))
None
```

Note that our `spam.system` does not yet run the `whoami` command; it only returns `None`.

Check that the function accepts exactly one argument, as specified by the **`METH_O`** flag:

```
>>> print(spam.system('too', 'many', 'arguments'))
Traceback (most recent call last):
   ...
TypeError: spam.system() takes exactly one argument (3 given)
```

Returning an integer

Now, let's take a look at the return value. Instead of `None`, we'll want `spam.system` to return a number – that is, a Python **`int`** object. Eventually this will be the exit code of a system command, but let's start with a fixed value, say, `3`.

The Python C API provides a function to create a Python **`int`** object from a C `int` value: **`PyLong_FromLong()`**. [[5]]

To call it, replace the `Py_RETURN_NONE` with the following 3 lines:

```
static PyObject *
spam_system(PyObject *self, PyObject *arg)
```

```
{
   int status = 3;
   PyObject *result = PyLong_FromLong(status);
   return result;
}
```

Recompile, restart the Python interpreter again, and check that the function now returns 3:

```
>>> import spam
>>> spam.system('whoami')
3
```

Accepting a string

Finally, let's handle the function argument.

Our C function, **`spam_system()`**, takes two arguments. The first one, `PyObject *self`, will be set to the `spam` module object. This isn't useful in our case, so we'll ignore it.

The other one, `PyObject *arg`, will be set to the object that the user passed from Python. We expect that it should be a Python string. In order to use the information in it, we will need to convert it to a C value – in this case, a C string (`const char *`).

There's a slight type mismatch here: Python's **`str`** objects store Unicode text, but C strings are arrays of bytes. So, we'll need to *encode* the data, and we'll use the UTF-8 encoding for it. (UTF-8 might not always be correct for system commands, but it's what **`str.encode()`** uses by default, and the C API has special support for it.)

The function to encode a Python string into a UTF-8 buffer is named **`PyUnicode_AsUTF8AndSize()`** [[6]]. Call it like this:

```
static PyObject *
spam_system(PyObject *self, PyObject *arg)
{
   const char *command = PyUnicode_AsUTF8AndSize(arg, NULL);
```

```
    int status = 3;
    PyObject *result = PyLong_FromLong(status);
    return result;
}
```

If **PyUnicode_AsUTF8AndSize()** is successful, *command* will point to the resulting C string – a zero-terminated array of bytes [[7]]. This buffer is managed by the *arg* object, which means we don't need to free it, but we must follow some rules:

- We should only use the buffer inside the `spam_system` function. After `spam_system` returns, *arg* and the buffer it manages might be garbage-collected.
- We must not modify it. This is why we use `const`.

If **PyUnicode_AsUTF8AndSize()** was *not* successful, it returns a `NULL` pointer. When calling *any* Python C API, we always need to handle such error cases. The way to do this in general is left for later chapters of this documentation. For now, be assured that we are already handling errors from **PyLong_FromLong()** correctly.

For the **PyUnicode_AsUTF8AndSize()** call, the correct way to handle errors is returning `NULL` from `spam_system`. Add an `if` block for this:

```
static PyObject *
spam_system(PyObject *self, PyObject *arg)
{
    const char *command = PyUnicode_AsUTF8AndSize(arg);
    if (command == NULL) {
        return NULL;
    }
    int status = 3;
    PyObject *result = PyLong_FromLong(status);
    return result;
}
```

To test that error handling works, compile again, restart Python so that `import spam` picks up the new version of your module, and try passing a non-string value to your function:

```
>>> import spam
>>> spam.system(3)
Traceback (most recent call last):
   ...
TypeError: bad argument type for built-in operation
```

Now, all that is left is calling the C library function **`system()`** with the `char *` buffer, and using its result instead of the `3`:

```
static PyObject *
spam_system(PyObject *self, PyObject *arg)
{
   const char *command = PyUnicode_AsUTF8AndSize(arg);
   if (command == NULL) {
      return NULL;
   }
   int status = system(command);
   PyObject *result = PyLong_FromLong(status);
   return result;
}
```

Compile your module, restart Python, and test. This time, you should see your username – the output of the `whoami` system command:

```
>>> import spam
>>> result = spam.system('whoami')
User Name
>>> result
0
```

You can also test with other commands, like `ls`, `dir`, or one that doesn't exist:

```
>>> import spam
>>> result = spam.system('nonexistent-command')
sh: line 1: nonexistent-command: command not found
>>> result
32512
```

The result

Congratulations! You have written a complete Python C API extension module, and completed this tutorial!

Here is the entire source file, for your convenience:

```
/// Includes

#include <Python.h>
#include <stdlib.h>     // for system()

/// Implementation of spam.system

static PyObject *
spam_system(PyObject *self, PyObject *arg)
{
   const char *command = PyUnicode_AsUTF8AndSize(arg, NULL);
   if (command == NULL) {
      return NULL;
   }
   int status = system(command);
   PyObject *result = PyLong_FromLong(status);
   return result;
}

/// Module method table

static PyMethodDef spam_methods[] = {
    {
        .ml_name="system",
        .ml_meth=spam_system,
        .ml_flags=METH_O,
        .ml_doc="Execute a shell command.",
    },
    {NULL, NULL, 0, NULL}        /* Sentinel */
};

/// Module slot table

PyABIInfo_VAR(abi_info);

static PySlot spam_slots[] = {
    PySlot_STATIC_DATA(Py_mod_abi, &abi_info),
```

```
    PySlot_STATIC_DATA(Py_mod_name, "spam"),
    PySlot_STATIC_DATA(Py_mod_doc, "A wonderful module with an e
    PySlot_STATIC_DATA(Py_mod_methods, spam_methods),
    PySlot_END
};

/// Export hook prototype

PyMODEXPORT_FUNC PyModExport_spam(void);

/// Module export hook

PyMODEXPORT_FUNC
PyModExport_spam(void)
{
   return spam_slots;
}
```

Appendix: Other build tools

You should be able to follow this tutorial – except the *Running your build tool* section itself – with a build tool other than `meson-python`.

The Python Packaging User Guide has a list of recommended tools [https://packaging.python.org/en/latest/guides/tool-recommendations/#build-backends-for-extension-modules]; be sure to choose one for the C language.

Workaround for missing PyInit function

If your build tool output complains about missing `PyInit_spam`, add the following function to your module for now:

```
// A workaround
void *PyInit_spam(void) { return NULL; }
```

This is a shim for an old-style initialization function, which was required in extension modules for CPython 3.14 and below. Current CPython does not need it, but some build tools may still assume that all extension modules need to define it.

If you use this workaround, you will get the exception `SystemError: initialization of spam failed without raising an exception` instead of `ImportError: dynamic module does not define module export function`.

Compiling directly

Using a third-party build tool is heavily recommended, as it will take care of various details of your platform and Python installation, of naming the resulting extension, and, later, of distributing your work.

If you are building an extension for as *specific* system, or for yourself only, you might instead want to run your compiler directly. The way to do this is system-specific; be prepared for issues you will need to solve yourself.

Linux

On Linux, the Python development package may include a `python3-config` command that prints out the required compiler flags. If you use it, check that it corresponds to the CPython interpreter you'll use to load the module. Then, start with the following command:

```
gcc --shared $(python3-config --cflags --ldflags) spammodule.c -
```

This should generate a `spam.so` file that you need to put in a directory on `sys.path`.

Notas de rodapé

[1]

`spam` is the favorite food of Monty Python fans…

[2]

The source file name is entirely up to you, though some tools can be picky about the `.c` extension. This tutorial uses the traditional `*module.c` suffix.

Some people would just use `spam.c` to implement a module named `spam`, projects where Python isn't the primary language might use `py_spam.c`, and so on.

[3]

Including `stdlib.h` is technically not necessary, since `Python.h` includes it and several other standard headers for its own use or for backwards compatibility. However, it is good practice to explicitly include what you need.

[4]

The **`PyMethodDef`** structure is also used to create methods of classes, so there's no separate "**`PyFunctionDef`**".

[5]

The name **`PyLong_FromLong()`** might not seem obvious. `PyLong` refers to a the Python **`int`**, which was originally called `long`; the `FromLong` refers to the C `long` (or `long int`) type.

[6]

Here, `PyUnicode` refers to the original name of the Python **`str`** class: `unicode`.

The `AndSize` part of the name refers to the fact that this function can also retrieve the size of the buffer, using an output argument. We don't need this, so we set the second argument to NULL.

[7]

We're ignoring the fact that Python strings can also contain NUL bytes, which terminate a C string. In other words, our function will treat `spam.system("foo\0bar")` as `spam.system("foo")`. This possibility can lead to security issues, so the real `os.system` function size checks for this case and raises an error.

Using the C API: Assorted topics

The tutorial walked you through creating a C API extension module, but left many areas unexplained. This document looks at several concepts that you'll need to learn in order to write more complex extensions.

Erros e exceções

An important convention throughout the Python interpreter is the following: when a function fails, it should set an exception condition and return an error value (usually `-1` or a `NULL` pointer). Exception information is stored in three members of the interpreter's thread state. These are `NULL` if there is no exception. Otherwise they are the C equivalents of the members of the Python tuple returned by `sys.exc_info()`. These are the exception type, exception instance, and a traceback object. It is important to know about them to understand how errors are passed around.

A API Python define uma série de funções para definir vários tipos de exceções.

The most common one is `PyErr_SetString()`. Its arguments are an exception object and a C string. The exception object is usually a predefined object like `PyExc_ZeroDivisionError`. The C string indicates the cause of the error and is converted to a Python string object and stored as the "associated value" of the exception.

Another useful function is `PyErr_SetFromErrno()`, which only takes an exception argument and constructs the associated value by inspection of the global variable **`errno`**. The most general function is `PyErr_SetObject()`, which takes two object arguments, the exception and its associated value. You don't need to `Py_INCREF()` the objects passed to any of these functions.

You can test non-destructively whether an exception has been set with `PyErr_Occurred()`. This returns the current exception object, or `NULL` if no exception has occurred. You normally don't need to call **`PyErr_Occurred()`**

to see whether an error occurred in a function call, since you should be able to tell from the return value.

When a function *f* that calls another function *g* detects that the latter fails, *f* should itself return an error value (usually `NULL` or `-1`). It should *not* call one of the `PyErr_*` functions — one has already been called by *g*. *f*'s caller is then supposed to also return an error indication to *its* caller, again *without* calling `PyErr_*`, and so on — the most detailed cause of the error was already reported by the function that first detected it. Once the error reaches the Python interpreter's main loop, this aborts the currently executing Python code and tries to find an exception handler specified by the Python programmer.

(There are situations where a module can actually give a more detailed error message by calling another `PyErr_*` function, and in such cases it is fine to do so. As a general rule, however, this is not necessary, and can cause information about the cause of the error to be lost: most operations can fail for a variety of reasons.)

To ignore an exception set by a function call that failed, the exception condition must be cleared explicitly by calling **`PyErr_Clear()`**. The only time C code should call **`PyErr_Clear()`** is if it doesn't want to pass the error on to the interpreter but wants to handle it completely by itself (possibly by trying something else, or pretending nothing went wrong).

Every failing **`malloc()`** call must be turned into an exception — the direct caller of **`malloc()`** (or **`realloc()`**) must call **`PyErr_NoMemory()`** and return a failure indicator itself. All the object-creating functions (for example, **`PyLong_FromLong()`**) already do this, so this note is only relevant to those who call **`malloc()`** directly.

Also note that, with the important exception of **`PyArg_ParseTuple()`** and friends, functions that return an integer status usually return a positive value or zero for success and `-1` for failure, like Unix system calls.

Finally, be careful to clean up garbage (by making **`Py_XDECREF()`** or **`Py_DECREF()`** calls for objects you have already created) when you return an error indicator!

The choice of which exception to raise is entirely yours. There are predeclared C objects corresponding to all built-in Python exceptions, such as `PyExc_ZeroDivisionError`, which you can use directly. Of course, you should choose exceptions wisely — don't use `PyExc_TypeError` to mean that a file couldn't be opened (that should probably be `PyExc_OSError`). If something's wrong with the argument list, the `PyArg_ParseTuple()` function usually raises **`PyExc_TypeError`**. If you have an argument whose value must be in a particular range or must satisfy other conditions, `PyExc_ValueError` is appropriate.

You can also define a new exception that is unique to your module. The simplest way to do this is to declare a static global object variable at the beginning of the file:

```
static PyObject *SpamError = NULL;
```

and initialize it by calling `PyErr_NewException()` in the module's `Py_mod_exec` function (**`spam_module_exec()`**):

```
SpamError = PyErr_NewException("spam.error", NULL, NULL);
```

Since **`SpamError`** is a global variable, it will be overwritten every time the module is reinitialized, when the `Py_mod_exec` function is called.

For now, let's avoid the issue: we will block repeated initialization by raising an `ImportError`:

```
static PyObject *SpamError = NULL;

static int
spam_module_exec(PyObject *m)
{
    if (SpamError != NULL) {
        PyErr_SetString(PyExc_ImportError,
                        "cannot initialize spam module more thar
        return -1;
    }
    SpamError = PyErr_NewException("spam.error", NULL, NULL);
    if (PyModule_AddObjectRef(m, "SpamError", SpamError) < 0) {
        return -1;
```

```
    }

    return 0;
}

static PyModuleDef_Slot spam_module_slots[] = {
    {Py_mod_exec, spam_module_exec},
    {0, NULL}
};

static struct PyModuleDef spam_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "spam",
    .m_size = 0,  // non-negative
    .m_slots = spam_module_slots,
};

PyMODINIT_FUNC
PyInit_spam(void)
{
    return PyModuleDef_Init(&spam_module);
}
```

Note that the Python name for the exception object is **`spam.error`**. The **`PyErr_NewException()`** function may create a class with the base class being **`Exception`** (unless another class is passed in instead of `NULL`), described in Exceções embutidas.

Note also that the **`SpamError`** variable retains a reference to the newly created exception class; this is intentional! Since the exception could be removed from the module by external code, an owned reference to the class is needed to ensure that it will not be discarded, causing **`SpamError`** to become a dangling pointer. Should it become a dangling pointer, C code which raises the exception could cause a core dump or other unintended side effects.

For now, the **`Py_DECREF()`** call to remove this reference is missing. Even when the Python interpreter shuts down, the global **`SpamError`** variable will not be garbage-collected. It will “leak”. We did, however, ensure that this will happen at most once per process.

We discuss the use of **PyMODINIT_FUNC** as a function return type later in this sample.

The **spam.error** exception can be raised in your extension module using a call to **PyErr_SetString()** as shown below:

```
static PyObject *
spam_system(PyObject *self, PyObject *args)
{
    const char *command;
    int sts;

    if (!PyArg_ParseTuple(args, "s", &command))
        return NULL;
    sts = system(command);
    if (sts < 0) {
        PyErr_SetString(SpamError, "System command failed");
        return NULL;
    }
    return PyLong_FromLong(sts);
}
```

Embedding an extension

If you want to make your module a permanent part of the Python interpreter, you will have to change the configuration setup and rebuild the interpreter. On Unix, place your file (`spammodule.c` for example) in the `Modules/` directory of an unpacked source distribution, add a line to the file `Modules/Setup.local` describing your file:

```
spam spammodule.o
```

and rebuild the interpreter by running **make** in the toplevel directory. You can also run **make** in the `Modules/` subdirectory, but then you must first rebuild `Makefile` there by running '**make** Makefile'. (This is necessary each time you change the `Setup` file.)

If your module requires additional libraries to link with, these can be listed on the line in the configuration file as well, for instance:

```
spam spammodule.o -lX11
```

Chamando funções Python de C

The tutorial concentrated on making C functions callable from Python. The reverse is also useful: calling Python functions from C. This is especially the case for libraries that support so-called “callback” functions. If a C interface makes use of callbacks, the equivalent Python often needs to provide a callback mechanism to the Python programmer; the implementation will require calling the Python callback functions from a C callback. Other uses are also imaginable.

Fortunately, the Python interpreter is easily called recursively, and there is a standard interface to call a Python function. (If you’re interested in how to call the Python parser with a particular string as input, see A camada de nível muito alto.)

Calling a Python function is easy. First, the Python program must somehow pass you the Python function object. You should provide a function (or some other interface) to do this. When this function is called, save a pointer to the Python function object (be careful to `Py_INCREF()` it!) in a global variable — or wherever you see fit. For example, the following function might be part of a module definition:

```
static PyObject *my_callback = NULL;

static PyObject *
my_set_callback(PyObject *dummy, PyObject *args)
{
    PyObject *result = NULL;
    PyObject *temp;

    if (PyArg_ParseTuple(args, "O:set_callback", &temp)) {
        if (!PyCallable_Check(temp)) {
            PyErr_SetString(PyExc_TypeError, "parameter must be
            return NULL;
        }
        Py_XINCREF(temp);         /* Add a reference to new call
        Py_XDECREF(my_callback);  /* Dispose of previous callbac
```

```
        my_callback = temp;        /* Remember new callback */
        /* Boilerplate to return "None" */
        Py_INCREF(Py_None);
        result = Py_None;
    }
    return result;
}
```

This function must be registered with the interpreter using the **METH_VARARGS** flag in **PyMethodDef.ml_flags**. The **PyArg_ParseTuple()** function and its arguments are documented in section Extraindo parâmetros em funções de extensão.

The macros **Py_XINCREF()** and **Py_XDECREF()** increment/decrement the reference count of an object and are safe in the presence of `NULL` pointers (but note that *temp* will not be `NULL` in this context). More info on them in section Contagens de referências.

Later, when it is time to call the function, you call the C function **PyObject_CallObject()**. This function has two arguments, both pointers to arbitrary Python objects: the Python function, and the argument list. The argument list must always be a tuple object, whose length is the number of arguments. To call the Python function with no arguments, pass in `NULL`, or an empty tuple; to call it with one argument, pass a singleton tuple. **Py_BuildValue()** returns a tuple when its format string consists of zero or more format codes between parentheses. For example:

```
int arg;
PyObject *arglist;
PyObject *result;
...
arg = 123;
...
/* Time to call the callback */
arglist = Py_BuildValue("(i)", arg);
result = PyObject_CallObject(my_callback, arglist);
Py_DECREF(arglist);
```

PyObject_CallObject() returns a Python object pointer: this is the return value of the Python function. **PyObject_CallObject()** is "reference-count-neutral" with respect to its arguments. In the example a new tuple was created to serve as the argument list, which is **Py_DECREF()**-ed immediately after the **PyObject_CallObject()** call.

The return value of **PyObject_CallObject()** is "new": either it is a brand new object, or it is an existing object whose reference count has been incremented. So, unless you want to save it in a global variable, you should somehow **Py_DECREF()** the result, even (especially!) if you are not interested in its value.

Before you do this, however, it is important to check that the return value isn't `NULL`. If it is, the Python function terminated by raising an exception. If the C code that called **PyObject_CallObject()** is called from Python, it should now return an error indication to its Python caller, so the interpreter can print a stack trace, or the calling Python code can handle the exception. If this is not possible or desirable, the exception should be cleared by calling **PyErr_Clear()**. For example:

```
if (result == NULL)
    return NULL; /* Pass error back */
...use result...
Py_DECREF(result);
```

Depending on the desired interface to the Python callback function, you may also have to provide an argument list to **PyObject_CallObject()**. In some cases the argument list is also provided by the Python program, through the same interface that specified the callback function. It can then be saved and used in the same manner as the function object. In other cases, you may have to construct a new tuple to pass as the argument list. The simplest way to do this is to call **Py_BuildValue()**. For example, if you want to pass an integral event code, you might use the following code:

```
PyObject *arglist;
...
arglist = Py_BuildValue("(l)", eventcode);
result = PyObject_CallObject(my_callback, arglist);
Py_DECREF(arglist);
```

```
if (result == NULL)
    return NULL; /* Pass error back */
/* Here maybe use the result */
Py_DECREF(result);
```

Note the placement of `Py_DECREF(arglist)` immediately after the call, before the error check! Also note that strictly speaking this code is not complete: **`Py_BuildValue()`** may run out of memory, and this should be checked.

You may also call a function with keyword arguments by using **`PyObject_Call()`**, which supports arguments and keyword arguments. As in the above example, we use **`Py_BuildValue()`** to construct the dictionary.

```
PyObject *dict;
...
dict = Py_BuildValue("{s:i}", "name", val);
result = PyObject_Call(my_callback, NULL, dict);
Py_DECREF(dict);
if (result == NULL)
    return NULL; /* Pass error back */
/* Here maybe use the result */
Py_DECREF(result);
```

Extraindo parâmetros em funções de extensão

The tutorial uses a "**`METH_O`**" function, which is limited to a single Python argument. If you want more, you can use **`METH_VARARGS`** instead. With this flag, the C function will receive a **`tuple`** of arguments instead of a single object.

For unpacking the tuple, CPython provides the **`PyArg_ParseTuple()`** function, declared as follows:

```
int PyArg_ParseTuple(PyObject *arg, const char *format, ...);
```

O argumento *arg* deve ser um objeto tupla contendo uma lista de argumentos passada de Python para uma função C. O argumento *format* deve ser uma string de formato, cuja sintaxe é explicada em Análise de

argumentos e construção de valores no Manual de Referência da API C/Python. Os argumentos restantes devem ser endereços de variáveis cujo tipo é determinado pela string de formato.

For example, to receive a single Python **`str`** object and turn it into a C buffer, you would use `"s"` as the format string:

```
const char *command;
if (!PyArg_ParseTuple(args, "s", &command)) {
    return NULL;
}
```

If an error is detected in the argument list, **`PyArg_ParseTuple()`** returns `NULL` (the error indicator for functions returning object pointers); your function may return `NULL`, relying on the exception set by **`PyArg_ParseTuple()`**.

Note that while **`PyArg_ParseTuple()`** checks that the Python arguments have the required types, it cannot check the validity of the addresses of C variables passed to the call: if you make mistakes there, your code will probably crash or at least overwrite random bits in memory. So be careful!

Note que quaisquer referências a objeto Python que são fornecidas ao chamador são referências *emprestadas*; não decremente a contagem de referências delas!

Alguns exemplos de chamadas:

```
#include <Python.h>

int ok;
int i, j;
long k, l;
const char *s;
Py_ssize_t size;

ok = PyArg_ParseTuple(args, ""); /* No arguments */
    /* Python call: f() */

ok = PyArg_ParseTuple(args, "s", &s); /* A string */
    /* Possible Python call: f('whoops!') */
```

```
ok = PyArg_ParseTuple(args, "lls", &k, &l, &s); /* Two longs and
    /* Possible Python call: f(1, 2, 'three') */
```

```
ok = PyArg_ParseTuple(args, "(ii)s#", &i, &j, &s, &size);
    /* A pair of ints and a string, whose size is also returned
    /* Possible Python call: f((1, 2), 'three') */
```

```
{
    const char *file;
    const char *mode = "r";
    int bufsize = 0;
    ok = PyArg_ParseTuple(args, "s|si", &file, &mode, &bufsize);
    /* A string, and optionally another string and an integer */
    /* Possible Python calls:
       f('spam')
       f('spam', 'w')
       f('spam', 'wb', 100000) */
}
```

```
{
    int left, top, right, bottom, h, v;
    ok = PyArg_ParseTuple(args, "((ii)(ii))(ii)",
             &left, &top, &right, &bottom, &h, &v);
    /* A rectangle and a point */
    /* Possible Python call:
       f(((0, 0), (400, 300)), (10, 10)) */
}
```

```
{
    Py_complex c;
    ok = PyArg_ParseTuple(args, "D:myfunction", &c);
    /* a complex, also providing a function name for errors */
    /* Possible Python call: myfunction(1+2j) */
}
```

Parâmetros nomeados para funções de extensão

If you also want your function to accept keyword arguments, use the **METH_KEYWORDS** flag in combination with **METH_VARARGS**. (**METH_KEYWORDS**

can also be used with other flags; see its documentation for the allowed combinations.)

In this case, the C function should accept a third `PyObject *` parameter which will be a dictionary of keywords. Use **`PyArg_ParseTupleAndKeywords()`** to parse the arguments to such a function.

The **`PyArg_ParseTupleAndKeywords()`** function is declared as follows:

```
int PyArg_ParseTupleAndKeywords(PyObject *arg, PyObject *kwdict,
                                const char *format, char * const
```

The *arg* and *format* parameters are identical to those of the **`PyArg_ParseTuple()`** function. The *kwdict* parameter is the dictionary of keywords received as the third parameter from the Python runtime. The *kwlist* parameter is a `NULL`-terminated list of strings which identify the parameters; the names are matched with the type information from *format* from left to right. On success, **`PyArg_ParseTupleAndKeywords()`** returns true, otherwise it returns false and raises an appropriate exception.

Nota

Nested tuples cannot be parsed when using keyword arguments! Keyword parameters passed in which are not present in the *kwlist* will cause **`TypeError`** to be raised.

Aqui está um módulo de exemplo que usa parâmetros nomeados, baseado em um exemplo de Geoff Philbrick (philbrick@hks.com):

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>

static PyObject *
keywdarg_parrot(PyObject *self, PyObject *args, PyObject *keywds
{
    int voltage;
```

```
    const char *state = "a stiff";
    const char *action = "voom";
    const char *type = "Norwegian Blue";

    static char *kwlist[] = {"voltage", "state", "action", "type

    if (!PyArg_ParseTupleAndKeywords(args, keywds, "i|sss", kwli
                                     &voltage, &state, &action,
        return NULL;

    printf("-- This parrot wouldn't %s if you put %i Volts throu
           action, voltage);
    printf("-- Lovely plumage, the %s -- It's %s!\n", type, stat

    Py_RETURN_NONE;
}

static PyMethodDef keywdarg_methods[] = {
    /* The cast of the function is necessary since PyCFunction v
     * only take two PyObject* parameters, and keywdarg_parrot()
     * three.
     */
    {"parrot", (PyCFunction)(void(*)(void))keywdarg_parrot, METH
     "Print a lovely skit to standard output."},
    {NULL, NULL, 0, NULL}   /* sentinel */
};
```

Construindo valores arbitrários

This function is the counterpart to `PyArg_ParseTuple()`. It is declared as follows:

```
PyObject *Py_BuildValue(const char *format, ...);
```

It recognizes a set of format units similar to the ones recognized by `PyArg_ParseTuple()`, but the arguments (which are input to the function, not output) must not be pointers, just values. It returns a new Python object, suitable for returning from a C function called from Python.

One difference with **PyArg_ParseTuple()**: while the latter requires its first argument to be a tuple (since Python argument lists are always represented as tuples internally), **Py_BuildValue()** does not always build a tuple. It builds a tuple only if its format string contains two or more format units. If the format string is empty, it returns `None`; if it contains exactly one format unit, it returns whatever object is described by that format unit. To force it to return a tuple of size 0 or one, parenthesize the format string.

Examples (to the left the call, to the right the resulting Python value):

```
Py_BuildValue("")                        None
Py_BuildValue("i", 123)                  123
Py_BuildValue("iii", 123, 456, 789)      (123, 456, 789)
Py_BuildValue("s", "hello")              'hello'
Py_BuildValue("y", "hello")              b'hello'
Py_BuildValue("ss", "hello", "world")    ('hello', 'world')
Py_BuildValue("s#", "hello", 4)          'hell'
Py_BuildValue("y#", "hello", 4)          b'hell'
Py_BuildValue("()")                      ()
Py_BuildValue("(i)", 123)                (123,)
Py_BuildValue("(ii)", 123, 456)          (123, 456)
Py_BuildValue("(i,i)", 123, 456)         (123, 456)
Py_BuildValue("[i,i]", 123, 456)         [123, 456]
Py_BuildValue("{s:i,s:i}",
              "abc", 123, "def", 456)    {'abc': 123, 'def': 456
Py_BuildValue("((ii)(ii)) (ii)",
              1, 2, 3, 4, 5, 6)          (((1, 2), (3, 4)), (5,
```

Contagens de referências

In languages like C or C++, the programmer is responsible for dynamic allocation and deallocation of memory on the heap. In C, this is done using the functions **`malloc()`** and **`free()`**. In C++, the operators `new` and `delete` are used with essentially the same meaning and we'll restrict the following discussion to the C case.

Every block of memory allocated with **`malloc()`** should eventually be returned to the pool of available memory by exactly one call to **`free()`**. It is important to call **`free()`** at the right time. If a block's address is forgotten

but **`free()`** is not called for it, the memory it occupies cannot be reused until the program terminates. This is called a *memory leak*. On the other hand, if a program calls **`free()`** for a block and then continues to use the block, it creates a conflict with reuse of the block through another **`malloc()`** call. This is called *using freed memory*. It has the same bad consequences as referencing uninitialized data — core dumps, wrong results, mysterious crashes.

Causas comuns de vazamentos de memória são caminhos incomuns no código. Por exemplo, uma função pode alocar um bloco de memória, executar algum cálculo e, em seguida, liberar o bloco novamente. Agora, uma alteração nos requisitos da função pode adicionar um teste ao cálculo que detecta uma condição de erro e pode retornar prematuramente da função. É fácil esquecer de liberar o bloco de memória alocado ao executar essa saída prematura, especialmente quando ela é adicionada posteriormente ao código. Esses vazamentos, uma vez introduzidos, geralmente passam despercebidos por um longo tempo: a saída de erro é executada apenas em uma pequena fração de todas as chamadas, e a maioria das máquinas modernas tem bastante memória virtual, portanto, o vazamento só se torna aparente em um processo de longa duração que usa a função com vazamento com frequência. Portanto, é importante evitar que vazamentos aconteçam por meio de uma convenção ou estratégia de codificação que minimize esse tipo de erro.

Since Python makes heavy use of **`malloc()`** and **`free()`**, it needs a strategy to avoid memory leaks as well as the use of freed memory. The chosen method is called *reference counting*. The principle is simple: every object contains a counter, which is incremented when a reference to the object is stored somewhere, and which is decremented when a reference to it is deleted. When the counter reaches zero, the last reference to the object has been deleted and the object is freed.

An alternative strategy is called *automatic garbage collection*. (Sometimes, reference counting is also referred to as a garbage collection strategy, hence the use of “automatic” to distinguish the two.) The big advantage of automatic garbage collection is that the user doesn’t need to call **`free()`** explicitly. (Another claimed advantage is an improvement in speed or

memory usage — this is no hard fact however.) The disadvantage is that for C, there is no truly portable automatic garbage collector, while reference counting can be implemented portably (as long as the functions **`malloc()`** and **`free()`** are available — which the C Standard guarantees). Maybe some day a sufficiently portable automatic garbage collector will be available for C. Until then, we'll have to live with reference counts.

Embora o Python utilize a implementação tradicional de contagem de referências, ele também oferece um detector de ciclos que funciona para detectar ciclos de referência. Isso permite que as aplicações não se preocupem em criar referências circulares diretas ou indiretas; esses são os pontos fracos da coleta de lixo implementada usando apenas a contagem de referências. Ciclos de referência consistem em objetos que contêm referências (possivelmente indiretas) a si mesmos, de modo que cada objeto no ciclo tem uma contagem de referências diferente de zero. Implementações típicas de contagem de referências não são capazes de recuperar a memória pertencente a nenhum objeto em um ciclo de referência, ou referenciada a partir dos objetos no ciclo, mesmo que não haja mais referências ao próprio ciclo.

The cycle detector is able to detect garbage cycles and can reclaim them. The **`gc`** module exposes a way to run the detector (the **`collect()`** function), as well as configuration interfaces and the ability to disable the detector at runtime.

Contagem de referências no Python

There are two macros, `Py_INCREF(x)` and `Py_DECREF(x)`, which handle the incrementing and decrementing of the reference count. **`Py_DECREF()`** also frees the object when the count reaches zero. For flexibility, it doesn't call **`free()`** directly — rather, it makes a call through a function pointer in the object's *type object*. For this purpose (and others), every object also contains a pointer to its type object.

The big question now remains: when to use `Py_INCREF(x)` and `Py_DECREF(x)`? Let's first introduce some terms. Nobody "owns" an object; however, you can *own a reference* to an object. An object's reference count

is now defined as the number of owned references to it. The owner of a reference is responsible for calling `Py_DECREF()` when the reference is no longer needed. Ownership of a reference can be transferred. There are three ways to dispose of an owned reference: pass it on, store it, or call `Py_DECREF()`. Forgetting to dispose of an owned reference creates a memory leak.

It is also possible to *borrow* [[1]] a reference to an object. The borrower of a reference should not call `Py_DECREF()`. The borrower must not hold on to the object longer than the owner from which it was borrowed. Using a borrowed reference after the owner has disposed of it risks using freed memory and should be avoided completely [[2]].

The advantage of borrowing over owning a reference is that you don't need to take care of disposing of the reference on all possible paths through the code — in other words, with a borrowed reference you don't run the risk of leaking when a premature exit is taken. The disadvantage of borrowing over owning is that there are some subtle situations where in seemingly correct code a borrowed reference can be used after the owner from which it was borrowed has in fact disposed of it.

A borrowed reference can be changed into an owned reference by calling `Py_INCREF()`. This does not affect the status of the owner from which the reference was borrowed — it creates a new owned reference, and gives full owner responsibilities (the new owner must dispose of the reference properly, as well as the previous owner).

Regras de propriedade

Sempre que uma referência de objeto é passada para dentro ou para fora de uma função, faz parte da especificação da interface da função se a propriedade é transferida com a referência ou não.

Most functions that return a reference to an object pass on ownership with the reference. In particular, all functions whose function it is to create a new object, such as `PyLong_FromLong()` and `Py_BuildValue()`, pass ownership to the receiver. Even if the object is not actually new, you still receive

ownership of a new reference to that object. For instance, **PyLong_FromLong()** maintains a cache of popular values and can return a reference to a cached item.

Many functions that extract objects from other objects also transfer ownership with the reference, for instance **PyObject_GetAttrString()**. The picture is less clear, here, however, since a few common routines are exceptions: **PyTuple_GetItem()**, **PyList_GetItem()**, **PyDict_GetItem()**, and **PyDict_GetItemString()** all return references that you borrow from the tuple, list or dictionary.

The function **PyImport_AddModule()** also returns a borrowed reference, even though it may actually create the object it returns: this is possible because an owned reference to the object is stored in `sys.modules`.

When you pass an object reference into another function, in general, the function borrows the reference from you — if it needs to store it, it will use **Py_INCREF()** to become an independent owner. There are exactly two important exceptions to this rule: **PyTuple_SetItem()** and **PyList_SetItem()**. These functions take over ownership of the item passed to them — even if they fail! (Note that **PyDict_SetItem()** and friends don't take over ownership — they are "normal.")

When a C function is called from Python, it borrows references to its arguments from the caller. The caller owns a reference to the object, so the borrowed reference's lifetime is guaranteed until the function returns. Only when such a borrowed reference must be stored or passed on, it must be turned into an owned reference by calling **Py_INCREF()**.

A referência de objeto retornada de uma função C chamada do Python deve ser uma referência própria — a propriedade é transferida da função para seu chamador.

Gelo fino

There are a few situations where seemingly harmless use of a borrowed reference can lead to problems. These all have to do with implicit

invocations of the interpreter, which can cause the owner of a reference to dispose of it.

The first and most important case to know about is using **Py_DECREF()** on an unrelated object while borrowing a reference to a list item. For instance:

```
void
bug(PyObject *list)
{
    PyObject *item = PyList_GetItem(list, 0);

    PyList_SetItem(list, 1, PyLong_FromLong(0L));
    PyObject_Print(item, stdout, 0); /* BUG! */
}
```

Esta função primeiro toma emprestada uma referência a `list[0]`, depois substitui `list[1]` pelo valor `0` e, por fim, imprime a referência emprestada. Parece inofensivo, certo? Mas não é!

Let's follow the control flow into **PyList_SetItem()**. The list owns references to all its items, so when item 1 is replaced, it has to dispose of the original item 1. Now let's suppose the original item 1 was an instance of a user-defined class, and let's further suppose that the class defined a **__del__()** method. If this class instance has a reference count of 1, disposing of it will call its **__del__()** method. Internally, **PyList_SetItem()** calls **Py_DECREF()** on the replaced item, which invokes replaced item's corresponding **tp_dealloc** function. During deallocation, **tp_dealloc** calls **tp_finalize**, which is mapped to the **__del__()** method for class instances (see **PEP 442** [https://peps.python.org/pep-0442/]). This entire sequence happens synchronously within the **PyList_SetItem()** call.

Since it is written in Python, the **__del__()** method can execute arbitrary Python code. Could it perhaps do something to invalidate the reference to `item` in **bug()**? You bet! Assuming that the list passed into **bug()** is accessible to the **__del__()** method, it could execute a statement to the effect of `del list[0]`, and assuming this was the last reference to that object, it would free the memory associated with it, thereby invalidating `item`.

A solução, depois de descobrir a origem do problema, é fácil: incrementar temporariamente a contagem de referências. A versão correta da função é:

```
void
no_bug(PyObject *list)
{
    PyObject *item = PyList_GetItem(list, 0);

    Py_INCREF(item);
    PyList_SetItem(list, 1, PyLong_FromLong(0L));
    PyObject_Print(item, stdout, 0);
    Py_DECREF(item);
}
```

This is a true story. An older version of Python contained variants of this bug and someone spent a considerable amount of time in a C debugger to figure out why his **`__del__()`** methods would fail…

The second case of problems with a borrowed reference is a variant involving threads. Normally, multiple threads in the Python interpreter can't get in each other's way, because there is a global lock protecting Python's entire object space. However, it is possible to temporarily release this lock using the macro **`Py_BEGIN_ALLOW_THREADS`**, and to re-acquire it using **`Py_END_ALLOW_THREADS`**. This is common around blocking I/O calls, to let other threads use the processor while waiting for the I/O to complete. Obviously, the following function has the same problem as the previous one:

```
void
bug(PyObject *list)
{
    PyObject *item = PyList_GetItem(list, 0);
    Py_BEGIN_ALLOW_THREADS
    ...some blocking I/O call...
    Py_END_ALLOW_THREADS
    PyObject_Print(item, stdout, 0); /* BUG! */
}
```

Ponteiros NULL

In general, functions that take object references as arguments do not expect you to pass them `NULL` pointers, and will dump core (or cause later core dumps) if you do so. Functions that return object references generally return `NULL` only to indicate that an exception occurred. The reason for not testing for `NULL` arguments is that functions often pass the objects they receive on to other function — if each function were to test for `NULL`, there would be a lot of redundant tests and the code would run more slowly.

It is better to test for `NULL` only at the "source:" when a pointer that may be `NULL` is received, for example, from **`malloc()`** or from a function that may raise an exception.

The macros **`Py_INCREF()`** and **`Py_DECREF()`** do not check for `NULL` pointers — however, their variants **`Py_XINCREF()`** and **`Py_XDECREF()`** do.

The macros for checking for a particular object type (`Pytype_Check()`) don't check for `NULL` pointers — again, there is much code that calls several of these in a row to test an object against various different expected types, and this would generate redundant tests. There are no variants with `NULL` checking.

The C function calling mechanism guarantees that the argument list passed to C functions (`args` in the examples) is never `NULL` — in fact it guarantees that it is always a tuple [[3]].

It is a severe error to ever let a `NULL` pointer "escape" to the Python user.

Escrevendo extensões em C++

É possível escrever módulos de extensão em C++. Algumas restrições se aplicam. Se o programa principal (o interpretador Python) for compilado e vinculado pelo compilador C, objetos globais ou estáticos com construtores não poderão ser usados. Isso não é um problema se o programa principal for vinculado pelo compilador C++. Funções que serão chamadas pelo interpretador Python (em particular, funções de inicialização de módulos) devem ser declaradas usando `extern "C"`. Não é necessário incluir os arquivos de cabeçalho Python entre `extern "C" {...}` — eles já usam esta

forma se o símbolo `__cplusplus` estiver definido (todos os compiladores C++ recentes definem este símbolo).

Fornecendo uma API C para um módulo de extensão

Muitos módulos de extensão apenas fornecem novas funções e tipos para serem usados em Python, mas às vezes o código em um módulo de extensão pode ser útil para outros módulos de extensão. Por exemplo, um módulo de extensão poderia implementar um tipo “collection”, que funciona como listas sem ordem. Assim como o tipo de lista padrão do Python possui uma API C que permite que os módulos de extensão criem e manipulem listas, esse novo tipo de coleção deve ter um conjunto de funções em C para manipulação direta de outros módulos de extensão.

À primeira vista, isso parece fácil: basta escrever as funções (sem declará-las `static`, é claro), fornecer um arquivo de cabeçalho apropriado e documentar a API C. E, de fato, isso funcionaria se todos os módulos de extensão estivessem sempre vinculados estaticamente ao interpretador Python. Quando os módulos são usados como bibliotecas compartilhadas, no entanto, os símbolos definidos em um módulo podem não ser visíveis para outro módulo. Os detalhes de visibilidade dependem do sistema operacional; alguns sistemas usam um espaço de nomes global para o interpretador Python e todos os módulos de extensão (Windows, por exemplo), enquanto outros exigem uma lista explícita de símbolos importados no momento da vinculação do módulo (AIX é um exemplo) ou oferecem uma escolha de estratégias diferentes (a maioria dos Unices). E mesmo que os símbolos sejam globalmente visíveis, o módulo cujas funções se deseja chamar pode não ter sido carregado ainda!

Portability therefore requires not to make any assumptions about symbol visibility. This means that all symbols in extension modules should be declared `static`, except for the module’s initialization function, in order to avoid name clashes with other extension modules. And it means that symbols that *should* be accessible from other extension modules must be exported in a different way.

Python provides a special mechanism to pass C-level information (pointers) from one extension module to another one: Capsules. A Capsule is a Python data type which stores a pointer (void*). Capsules can only be created and accessed via their C API, but they can be passed around like any other Python object. In particular, they can be assigned to a name in an extension module's namespace. Other extension modules can then import this module, retrieve the value of this name, and then retrieve the pointer from the Capsule.

Existem muitas maneiras de usar Cápsulas para exportar a API C de um módulo de extensão. Cada função pode obter sua própria Cápsula, ou todos os ponteiros da API C podem ser armazenados em um array cujo endereço é publicado em uma Cápsula. E as diversas tarefas de armazenamento e recuperação dos ponteiros podem ser distribuídas de diferentes maneiras entre o módulo que fornece o código e os módulos clientes.

Whichever method you choose, it's important to name your Capsules properly. The function `PyCapsule_New()` takes a name parameter (const char*); you're permitted to pass in a `NULL` name, but we strongly encourage you to specify a name. Properly named Capsules provide a degree of runtime type-safety; there is no feasible way to tell one unnamed Capsule from another.

Em particular, as Cápsulas usadas para expor APIs C devem receber um nome seguindo esta convenção:

```
modulename.attributename
```

The convenience function `PyCapsule_Import()` makes it easy to load a C API provided via a Capsule, but only if the Capsule's name matches this convention. This behavior gives C API users a high degree of certainty that the Capsule they load contains the correct C API.

The following example demonstrates an approach that puts most of the burden on the writer of the exporting module, which is appropriate for commonly used library modules. It stores all C API pointers (just one in the example!) in an array of void pointers which becomes the value of a Capsule. The header file corresponding to the module provides a macro that

takes care of importing the module and retrieving its C API pointers; client modules only have to call this macro before accessing the C API.

The exporting module is a modification of the **spam** module from the tutorial. The function **spam.system()** does not call the C library function **system()** directly, but a function **PySpam_System()**, which would of course do something more complicated in reality (such as adding “spam” to every command). This function **PySpam_System()** is also exported to other extension modules.

The function **PySpam_System()** is a plain C function, declared `static` like everything else:

```
static int
PySpam_System(const char *command)
{
    return system(command);
}
```

The function **spam_system()** is modified in a trivial way:

```
static PyObject *
spam_system(PyObject *self, PyObject *args)
{
    const char *command;
    int sts;

    if (!PyArg_ParseTuple(args, "s", &command))
        return NULL;
    sts = PySpam_System(command);
    return PyLong_FromLong(sts);
}
```

No início do módulo, logo após a linha

```
#include <Python.h>
```

mais duas linhas devem ser adicionadas:

```
#define SPAM_MODULE
#include "spammodule.h"
```

The `#define` is used to tell the header file that it is being included in the exporting module, not a client module. Finally, the module's **`mod_exec`** function must take care of initializing the C API pointer array:

```
static int
spam_module_exec(PyObject *m)
{
    static void *PySpam_API[PySpam_API_pointers];
    PyObject *c_api_object;

    /* Initialize the C API pointer array */
    PySpam_API[PySpam_System_NUM] = (void *)PySpam_System;

    /* Create a Capsule containing the API pointer array's addre
    c_api_object = PyCapsule_New((void *)PySpam_API, "spam._C_AP

    if (PyModule_Add(m, "_C_API", c_api_object) < 0) {
        return -1;
    }

    return 0;
}
```

Note that `PySpam_API` is declared `static`; otherwise the pointer array would disappear when **`PyInit_spam()`** terminates!

A maior parte do trabalho está no arquivo de cabeçalho `spammodule.h`, que se parece com isso:

```
#ifndef Py_SPAMMODULE_H
#define Py_SPAMMODULE_H
#ifdef __cplusplus
extern "C" {
#endif

/* Header file for spammodule */

/* C API functions */
#define PySpam_System_NUM 0
#define PySpam_System_RETURN int
#define PySpam_System_PROTO (const char *command)
```

```
/* Total number of C API pointers */
#define PySpam_API_pointers 1


#ifdef SPAM_MODULE
/* This section is used when compiling spammodule.c */

static PySpam_System_RETURN PySpam_System PySpam_System_PROTO;

#else
/* This section is used in modules that use spammodule's API */

static void **PySpam_API;

#define PySpam_System \
 (*(PySpam_System_RETURN (*)PySpam_System_PROTO) PySpam_API[PySp

/* Return -1 on error, 0 on success.
 * PyCapsule_Import will set an exception if there's an error.
 */
static int
import_spam(void)
{
    PySpam_API = (void **)PyCapsule_Import("spam._C_API", 0);
    return (PySpam_API != NULL) ? 0 : -1;
}

#endif

#ifdef __cplusplus
}
#endif

#endif /* !defined(Py_SPAMMODULE_H) */
```

All that a client module must do in order to have access to the function **`PySpam_System()`** is to call the function (or rather macro) **`import_spam()`** in its **`mod_exec`** function:

```
static int
client_module_exec(PyObject *m)
```

```
{
    if (import_spam() < 0) {
        return -1;
    }
    /* additional initialization can happen here */
    return 0;
}
```

The main disadvantage of this approach is that the file `spammodule.h` is rather complicated. However, the basic structure is the same for each function that is exported, so it has to be learned only once.

Finally it should be mentioned that Capsules offer additional functionality, which is especially useful for memory allocation and deallocation of the pointer stored in a Capsule. The details are described in the Python/C API Reference Manual in the section Capsules and in the implementation of Capsules (files `Include/pycapsule.h` and `Objects/capsule.c` in the Python source code distribution).

Notas de rodapé

[1]

A metáfora de “pegar emprestado” uma referência não é completamente correta: o proprietário ainda tem uma cópia da referência.

[2]

Checking that the reference count is at least 1 **does not work** — the reference count itself could be in freed memory and may thus be reused for another object!

[3]

These guarantees don’t hold when you use the “old” style calling convention — this is still found in much existing code.

Definindo Tipos de Extensão: Tutorial

O Python permite que o desenvolvedor de um módulo de extensão em C defina novos tipos que podem ser manipulados a partir do código Python, da mesma forma que os tipos embutidos `str` e `list`. O código para de todos tipos de extensão segue um padrão, mas há alguns detalhes que você precisa entender antes de começar. Este documento é uma introdução suave ao tópico.

O básico

O tempo de execução do CPython trata todos os objetos Python como variáveis do tipo PyObject*, que funciona como um "tipo base" para todos os objetos Python. A estrutura `PyObject` em si contém apenas a contagem de referências do objeto e um ponteiro para o "objeto de tipo" correspondente. É nesse objeto de tipo que tudo acontece: ele determina quais funções em C o interpretador chama quando, por exemplo, um atributo é acessado em um objeto, um método é chamado ou o objeto é multiplicado por outro. Essas funções em C são chamadas de "métodos de tipo".

Então, se você quiser definir um novo tipo de extensão, você precisa criar um novo objeto de tipo.

Esse tipo de coisa só pode ser explicado com exemplo, então aqui está um módulo mínimo, mas completo, que define um novo tipo nomeado `Custom` dentro de um módulo de extensão C `custom`:

Nota

O que estamos mostrando aqui é a maneira tradicional de definir tipos de extensão *estáticos*. Deve ser adequada para a maioria dos usos. A API C

também permite definir tipos de extensão alocados em heap usando a função `PyType_FromSpec()`, que não é abordada neste tutorial.

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>

typedef struct {
    PyObject_HEAD
    /* Campos específico do tipo vão aqui. */
} CustomObject;

static PyTypeObject CustomType = {
    .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "custom.Custom",
    .tp_doc = PyDoc_STR("Custom objects"),
    .tp_basicsize = sizeof(CustomObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT,
    .tp_new = PyType_GenericNew,
};

static int
custom_module_exec(PyObject *m)
{
    if (PyType_Ready(&CustomType) < 0) {
        return -1;
    }

    if (PyModule_AddObjectRef(m, "Custom", (PyObject *) &Custom
        return -1;
    }

    return 0;
}

static PyModuleDef_Slot custom_module_slots[] = {
    {Py_mod_exec, custom_module_exec},
    // Só use isso quando estiver usando tipos estáticos
    {Py_mod_multiple_interpreters, Py_MOD_MULTIPLE_INTERPRETERS_
    {0, NULL}
};
```

```
static PyModuleDef custom_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "custom",
    .m_doc = "Example module that creates an extension type.",
    .m_size = 0,
    .m_slots = custom_module_slots,
};

PyMODINIT_FUNC
PyInit_custom(void)
{
    return PyModuleDef_Init(&custom_module);
}
```

Isso é bastante informação para assimilar de uma só vez, mas algumas partes devem parecer familiares pelo que foi visto no capítulo anterior. Este arquivo define três elementos:

1. O que um **objeto `Custom`** contém: esta é uma estrutura `CustomObject`, alocada uma vez para cada instância de **`Custom`**.
2. Como o **tipo `Custom`** se comporta: esta é a estrutura `CustomType`, que define um conjunto de sinalizadores e ponteiros de função que o interpretador inspeciona quando operações específicas são solicitadas.
3. Como definir e executar o módulo **`custom`**: esta é a função `PyInit_custom` e a estrutura associada `custom_module` para definir o módulo, e a função `custom_module_exec` para configurar um novo objeto de módulo.

O primeiro bit é

```
typedef struct {
    PyObject_HEAD
} CustomObject;
```

Isto é o que um objeto Custom conterá. `PyObject_HEAD` é obrigatório no início de cada estrutura de objeto e define um campo chamado `ob_base` do tipo **`PyObject`**, contendo um ponteiro para um objeto de tipo e uma contagem de referências (esses podem ser acessados usando os macros

`Py_TYPE` e `Py_REFCNT`, respectivamente). O objetivo do macro é abstrair o layout e permitir campos adicionais em construções de depuração.

Nota

Não há ponto e vírgula acima após o macro `PyObject_HEAD`. Tenha cuidado para não adicionar um por engano: alguns compiladores podem emitir erros.

Obviamente, objetos geralmente armazenam dados adicionais além do padrão `PyObject_HEAD` boilerplate; por exemplo, aqui está a definição do padrão Python floats:

```
typedef struct {
    PyObject_HEAD
    double ob_fval;
} PyFloatObject;
```

O segundo bit é a definição do objeto de tipo.

```
estático PyTypeObject CustomType = {
    .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "custom.Custom",
    .tp_doc = PyDoc_STR("Custom objeto"),
    .tp_basicsize = sizeof(CustomObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT,
    .tp_new = PyType_GenericNew,
};
```

Nota

Recomendamos usar inicializadores nomeados no estilo C99, como mostrado acima, para evitar listar todos os campos de `PyTypeObject` com os quais você não precisa se preocupar e também para não depender da ordem de declaração desses campos

A definição real de **PyTypeObject** no `object.h` possui muito mais campos do que a definição acima. Os campos restantes serão preenchidos com zeros pelo compilador C, e é prática comum não especificá-los explicitamente, a menos que sejam necessários.

Vamos separá-lo, um campo de cada vez

```
.ob_base = PyVarObject_HEAD_INIT(NULL, 0)
```

Essa linha é um boilerplate obrigatório para inicializar o campo `ob_base` mencionado acima.

```
.tp_name = "custom.Custom",
```

O nome do nosso tipo. Ele aparecerá na representação textual padrão do nosso objeto e em algumas mensagens erro, por exemplo:

```
>>> "" + custom.Custom()
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can only concatenate str (not "custom.Custom") to str
```

Observe que o nome é um nome pontilhado que inclui o nome módulo e o nome do tipo dentro do módulo. O módulo, neste caso, é **`custom`** e o tipo é **`Custom`**, portanto, definimos o nome do tipo como **`custom.Custom`**. O uso do pontilhado real caminho de importação é importante para tornar seu tipo compatível com o **pydoc** e os módulos **pickle**.

```
.tp_basicsize = sizeof(CustomObject),
.tp_itemsize = 0,
```

Isso serve para que o Python saiba quanta memória deve ser alocada ao criar novas instâncias do **`Custom`**. O **tp_itemsize** é usado somente para objeto de tamanho variável e, caso contrário, deve ser zero.

Nota

Se você quiser que seu tipo possa ser subclassificado a partir do Python, e seu tipo tiver o mesmo **tp_basicsize** que seu tipo base, você poderá ter

problemas com herança múltipla. Uma subclasse Python do seu tipo terá de listar o seu tipo primeiro em **__bases__**, caso contrário ela não conseguirá chamar o método **__new__()** do seu tipo sem gerar um erro. Você pode evitar esse problema garantindo que seu tipo tenha um valor de **tp_basicsize** maior que o do tipo base. Na maior parte do tempo, isso já será verdadeiro, pois ou o tipo base será **object**, ou você estará adicionando membros de dados ao seu tipo base e, portanto, aumentando seu tamanho.

Definimos os sinalizadores da classe como **Py_TPFLAGS_DEFAULT**.

```
.tp_flags = Py_TPFLAGS_DEFAULT,
```

Todos os tipos devem incluir essa constante em seus sinalizadores. Ela habilita todos os membros definidos até pelo menos o Python 3.3. Se você precisar de membros adicionais, será necessário fazer um OR com os sinalizadores correspondentes.

Fornecemos uma docstring para o tipo em tp_doc **tp_doc**.

```
.tp_doc = PyDoc_STR("Custom objects"),
```

Para habilitar a criação de objetos, precisamos fornecer um manipulador para **tp_new**. Esse é o equivalente ao método Python **__new__()**, mas precisa ser especificado explicitamente. Neste caso, podemos simplesmente usar a implementação padrão fornecida pela função da API **PyType_GenericNew()**.

```
.tp_new = PyType_GenericNew,
```

Todo o restante do arquivo deve ser familiar, exceto por alguns códigos em **custom_module_exec()**:

```
se (PyType_Ready(&CustomType) < 0) {
    retorno -1;
}
```

Isso inicializa o tipo **`Custom`**, preenchendo vários membros com os valores padrão apropriados, incluindo **`ob_type`**, que definimos inicialmente como `NULL`.

```
if (PyModule_AddObjectRef(m, "Custom", (PyObject *) &CustomType)
return -1;
}
```

Isso adiciona o tipo ao dicionário do módulo. Isso nos permite criar instâncias de **`Custom`** chamando a classe **`Custom`**:

```
>>> import custom
>>> mycustom = custom.Custom()
```

É isso! Só resta compilá-lo; coloque o código acima em um arquivo chamado `custom.c`,

```
[build-system]
requires = ["setuptools"]
build-backend = "setuptools.build_meta"

[project]
name = "custom"
version = "1"
```

Em um arquivo chamado `pyproject.toml`, e

```
from setuptools import Extension, setup
setup(ext_modules=[Extension("custom", ["custom.c"])])
```

Em um arquivo chamado `setup.py`; e então digitando

```
$ python -m pip install .
```

no console deverá gerar um arquivo `custom.so` em um subdiretório e instalá-lo; agora abra o Python — você já deve conseguir `import custom` e experimentar objetos `Custom`.

Isso não foi tão difícil, foi?

Naturalmente, o tipo Custom atual é bastante desinteressante. Não tem dados e não faz nada. Não pode nem ser subclassificado.

Adicionando dados e métodos ao exemplo básico

Vamos estender o exemplo básico para adicionar alguns dados e métodos. Também vamos tornar o tipo utilizável como uma classe base. Iremos criar um novo módulo, **`custom2`**, que adiciona esses recursos:

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include <stddef.h> /* for offsetof() */

typedef struct {
    PyObject_HEAD
    PyObject *first; /* first name */
    PyObject *last;  /* last name */
    int number;
} CustomObject;

static void
Custom_dealloc(PyObject *op)
{
    CustomObject *self = (CustomObject *) op;
    Py_XDECREF(self->first);
    Py_XDECREF(self->last);
    Py_TYPE(self)->tp_free(self);
}

static PyObject *
Custom_new(PyTypeObject *type, PyObject *args, PyObject *kwds)
{
    CustomObject *self;
    self = (CustomObject *) type->tp_alloc(type, 0);
    if (self != NULL) {
        self->first = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
        if (self->first == NULL) {
            Py_DECREF(self);
            return NULL;
        }
        self->last = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
```

```
        if (self->last == NULL) {
            Py_DECREF(self);
            return NULL;
        }
        self->number = 0;
    }
    return (PyObject *) self;
}

static int
Custom_init(PyObject *op, PyObject *args, PyObject *kwds)
{
    CustomObject *self = (CustomObject *) op;
    static char *kwlist[] = {"first", "last", "number", NULL};
    PyObject *first = NULL, *last = NULL;

    if (!PyArg_ParseTupleAndKeywords(args, kwds, "|OOi", kwlist,
                                     &first, &last,
                                     &self->number))
        return -1;

    if (first) {
        Py_XSETREF(self->first, Py_NewRef(first));
    }
    if (last) {
        Py_XSETREF(self->last, Py_NewRef(last));
    }
    return 0;
}

static PyMemberDef Custom_members[] = {
    {"first", Py_T_OBJECT_EX, offsetof(CustomObject, first), 0,
     "first name"},
    {"last", Py_T_OBJECT_EX, offsetof(CustomObject, last), 0,
     "last name"},
    {"number", Py_T_INT, offsetof(CustomObject, number), 0,
     "custom number"},
    {NULL}  /* Sentinel */
};

static PyObject *
Custom_name(PyObject *op, PyObject *Py_UNUSED(dummy))
{
```

```
    CustomObject *self = (CustomObject *) op;
    if (self->first == NULL) {
        PyErr_SetString(PyExc_AttributeError, "first");
        return NULL;
    }
    if (self->last == NULL) {
        PyErr_SetString(PyExc_AttributeError, "last");
        return NULL;
    }
    return PyUnicode_FromFormat("%S %S", self->first, self->last
}

static PyMethodDef Custom_methods[] = {
    {"name", Custom_name, METH_NOARGS,
     "Return the name, combining the first and last name"
    },
    {NULL}  /* Sentinel */
};

static PyTypeObject CustomType = {
    .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "custom2.Custom",
    .tp_doc = PyDoc_STR("Custom objects"),
    .tp_basicsize = sizeof(CustomObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .tp_new = Custom_new,
    .tp_init = Custom_init,
    .tp_dealloc = Custom_dealloc,
    .tp_members = Custom_members,
    .tp_methods = Custom_methods,
};

static int
custom_module_exec(PyObject *m)
{
    if (PyType_Ready(&CustomType) < 0) {
        return -1;
    }

    if (PyModule_AddObjectRef(m, "Custom", (PyObject *) &Custom
        return -1;
    }
```

```
    return 0;
}

static PyModuleDef_Slot custom_module_slots[] = {
    {Py_mod_exec, custom_module_exec},
    {Py_mod_multiple_interpreters, Py_MOD_MULTIPLE_INTERPRETERS_
    {0, NULL}
};

static PyModuleDef custom_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "custom2",
    .m_doc = "Example module that creates an extension type.",
    .m_size = 0,
    .m_slots = custom_module_slots,
};

PyMODINIT_FUNC
PyInit_custom2(void)
{
    return PyModuleDef_Init(&custom_module);
}
```

Esta versão do módulo possui várias alterações.

O tipo **`Custom`** agora possui três atributos de dados em sua estrutura C: first, last e number. As variáveis first e last são strings Python contendo o primeiro e o último nome. O atributo number é um inteiro em C.

A estrutura do objeto é atualizada de acordo

```
typedef struct {
PyObject_HEAD
PyObject *first; /* first name */
PyObject *last; /* last name */
int number;
} CustomObject;
```

Como agora temos dados para gerenciar, precisamos ter mais cuidado com a alocação e a desalocação do objeto. No mínimo, precisamos de método de

desalocação:

```
static void
Custom_dealloc(PyObject *op)
{
CustomObject *self = (CustomObject *) op;
Py_XDECREF(self->first);
Py_XDECREF(self->last);
Py_TYPE(self)->tp_free(self);
}
```

que é atribuído ao membro **tp_dealloc**:

```
tp_dealloc = Custom_dealloc,
```

Esse método primeiro limpa a contagem de referências dos dois atributos Python. **Py_XDECREF()** lida corretamente com o caso em que seu argumento é `NULL` (o que pode acontecer aqui se `tp_new` falhou no meio do processo). Em seguida, ele chama o membro **tp_free** do tipo do objeto (obtido por `Py_TYPE(self)`) para liberar a memória do objeto. Observe que o tipo do objeto pode não ser **`CustomType`**, pois o objeto pode ser uma instância de uma subclasse.

Nota

O cast explícito para `CustomObject *` acima é necessário porque definimos `Custom_dealloc` para receber um argumento do tipo `PyObject *`, enquanto o ponteiro de função `tp_dealloc` espera receber um argumento `PyObject *`. Ao atribuir nossa função ao slot `tp_dealloc` de um tipo, estamos declarando que ela só pode ser chamada com instâncias da classe `CustomObject`; portanto, o cast para `(CustomObject *)` é seguro. Isso é polimorfismo orientado a objetos, em C!

Em códigos existentes, ou em versões anteriores deste tutorial, você pode consultar funções semelhantes receberem diretamente um ponteiro para estrutura de objeto do subtipo (`CustomObject*`), assim:

```
Custom_dealloc(CustomObject *self)
{
```

```
Py_XDECREF(self->first);
Py_XDECREF(self->last);
Py_TYPE(self)->tp_free((PyObject *) self);
}
...
.tp_dealloc = (destructor) Custom_dealloc,
```

Isso faz a mesma coisa em todas as arquiteturas que o CPython oferece suporte, mas, de acordo com o padrão da linguagem C, isso invoca um comportamento indefinido.

Queremos nos certificar de que o primeiro e o último nome sejam inicializados como strings vazias, portanto, fornecemos uma implementação:: `tp_new`:

```
static PyObject *
Custom_new(PyTypeObject *type, PyObject *args, PyObject *kwds)
{
CustomObject *self;
self = (CustomObject *) type->tp_alloc(type, 0);
if (self != NULL) {
self->first = PyUnicode_FromString("");
if (self->first == NULL) {
Py_DECREF(self);
return NULL;
}
self->last = PyUnicode_FromString("");
if (self->last == NULL) {
Py_DECREF(self);
return NULL;
}
self->number = 0;
}
return (PyObject *) self;
}
```

e instale-o no membro:: **`tp_new`**

```
.tp_new = Custom_new,
```

O manipulador `tp_new` é responsável por criar (em oposição a inicializar) os objetos do tipo. Ele é exposto no Python como o método **`__new__()`**. Não é obrigatório definir um membro `tp_new` e, de fato, muitos tipos de extensão simplesmente reutilizam **`PyType_GenericNew()`**, como na primeira versão do tipo **`Custom`** acima. Neste caso, usamos o manipulador `tp_new` para inicializar os atributos `first` e `last` com valores padrão que não sejam `NULL`.

O `tp_new` recebe o tipo que está sendo instanciado (não necessariamente `CustomType`, caso uma subclasse esteja sendo instanciada) e quaisquer argumentos passados quando o tipo foi chamado, e deve retornar a instância criada. Manipuladores `tp_new` sempre aceitam argumentos posicionais e argumentos nomeados, mas frequentemente os ignoram, deixando o tratamento dos argumentos para os métodos inicializadores (ou seja, `tp_init` em C ou `__init__` em Python).

Nota

`tp_new` não deve chamar `tp_init` explicitamente, pois o interpretador fará isso por conta própria.

A implementação de `tp_new` chama o slot **`tp_alloc`** para alocar memória:

```
self = (CustomObject *) type->tp_alloc(type, 0);
```

Como a alocação de memória pode falhar, precisamos verificar se o resultado de **`tp_alloc`** é `NULL` antes de prosseguir.

Nota

Nós não preenchemos o slot **`tp_alloc`** por conta própria. Em vez disso, **`PyType_Ready()`** o preenche herdando-o da nossa classe base, que por padrão é **`object`**. A maioria dos tipos usa a estratégia de alocação padrão.

Nota

> Se você estiver criando um **tp_new** cooperativo (um que chama o **tp_new** ou **__new__()** de um tipo base), você *não* deve tentar determinar qual método chamar usando a ordem de resolução de métodos em tempo de execução. Sempre determine estaticamente qual tipo você vai chamar e chame seu **tp_new** diretamente ou via `type->tp_base->tp_new`. Se você não fizer isso, as subclasses Python do seu tipo que também herdam de outras classes definidas em Python podem não funcionar corretamente. (Especificamente, você pode não conseguir criar instâncias dessas subclasses sem receber um **TypeError**.)

Também definimos uma função de inicialização que aceita argumentos para fornecer valores iniciais para nosso instância:

```
static intCustom_init(PyObject *op, PyObject *args, PyObject *kw
```

preenchendo o slot :: **tp_init**.

```
.tp_init = Custom_init,
```

O slot **tp_init** é exposto em Python como o método **__init__()**. Ele é usado para inicializar um objeto depois que ele é criado. Inicializadores sempre aceitam argumentos posicionais e argumentos nomeados, e devem retornar `0` em caso de sucesso ou `-1` em caso de erro.

Ao contrário do manipulador `tp_new`, não há garantia de que seja chamado (por exemplo, o módulo **pickle** por padrão não chama **__init__()** em instâncias desserializadas). Além disso, `tp_init` pode ser chamado várias vezes. Qualquer um pode chamar o método **__init__()** em nossos objetos. Por esse motivo, precisamos ter cuidado redobrado ao atribuir os novos valores de atributos. Poderíamos ficar tentados, por exemplo, a atribuir o membro `first` assim:

```
if (first) { Py_XDECREF(self->first); Py_INCREF(first); self->fi
```

Mas isso seria arriscado. Nosso tipo não restringe o tipo do membro `first`, então ele poderia ser qualquer tipo de objeto. Ele poderia ter um destrutor

que executasse código que tentasse acessar o membro `first`; ou esse destrutor poderia desanexar o estado de thread e permitir que código arbitrário fosse executado em outras threads, acessando e modificando nosso objeto.

Para sermos extremamente cautelosos e nos proteger dessa possibilidade, quase sempre atribuímos novamente os membros antes de decrementar a contagem de referências. Quando não precisamos fazer isso?

- quando sabemos absolutamente que a contagem de referência é maior que 1;
- quando sabemos que a desalocação do objeto [[1]] não desvinculará o estado de thread nem causará quaisquer chamadas de volta no código do nosso tipo;
- quando estamos decrementando uma contagem de referências em um manipulador **`tp_dealloc`** de um tipo que não oferece suporte à coleta de lixo cíclica [[2]].

Queremos expor nossas variáveis de instância como atributos. Há várias maneiras de fazer isso. A forma mais simples é estabelecer definições de membros:

```
static PyMemberDef Custom_members[] = { {"first", Py_T_OBJECT_EX
```

e colocar as definições no slot **`tp_members`**:

```
.tp_members = Custom_members,
```

Cada definição de membro possui um nome de membro, um tipo, um deslocamento, sinalizadores de acesso e uma string de documentação. Consulte a seção Generic Attribute Management abaixo para mais detalhes.

Uma desvantagem dessa abordagem é que ela não fornece uma forma de restringir os tipos de objetos que podem ser atribuídos aos atributos em Python. Esperamos que os nomes first e last sejam strings, mas qualquer objeto Python pode ser atribuído. Além disso, os atributos podem ser excluídos, fazendo com que os ponteiros em C sejam definidos como `NULL`. Mesmo que possamos garantir que os membros sejam inicializados com

valores não `NULL`, eles ainda podem acabar sendo definidos como `NULL` caso os atributos sejam apagados.

Definimos um único método,:meth:*!Custom.name*, que exibe o nome do objeto como a concatenação do primeiro nome com o último nome

```
static PyObject *Custom_name(PyObject *op, PyObject *Py_UNUSED(c
```

O método é implementado como uma função em C que recebe uma instância de **`Custom`** (ou de uma subclasse de **`Custom`**) como primeiro argumento. Métodos sempre recebem a instância como primeiro argumento. Eles também costumam aceitar argumentos posicionais e nomeados, mas, neste caso, não recebemos nenhum e não precisamos aceitar uma tupla de argumentos posicionais nem um dicionário de argumentos nomeados. Este método equivale ao método Python:

```
def name(self): return "%s %s" % (self.first, self.last)
```

Observe que precisamos verificar a possibilidade de que nossos atributos **`first`** e **`last`** estejam com valor `NULL`. Isso ocorre porque eles podem ser excluídos e, nesse caso, ficam definidos como `NULL`. Seria melhor impedir a exclusão desses atributos e restringir seus valores para que sejam sempre strings. Veremos como fazer isso na próxima seção.

Agora que definimos o método, precisamos criar uma array de definições de métodos:

```
static PyMethodDef Custom_methods[] = {
{"name", Custom_name, METH_NOARGS,
"Return the name, combining the first and last name"
},
{NULL} /* Sentinel */
};
```

note que usamos o sinalizador **`METH_NOARGS`** para indicar que o método não espera argumentos além de *self*)

e atribuí-lo para o slot:: **`tp_methods`**

```
.tp_methods = Custom_methods,
```

Por fim, tornaremos nosso tipo utilizável como classe base para herança. Escrevemos nossos métodos com cuidado até agora, garantindo que eles não façam suposições sobre o tipo do objeto que está sendo criado ou utilizado; portanto, tudo o que precisamos fazer é adicionar a **Py_TPFLAGS_BASETYPE** à definição dos sinalizadores da nossa classe:

```
.tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
```

Renomeamos **PyInit_custom()** para **PyInit_custom2()**, atualizamos o nome do módulo na estrutura **PyModuleDef**, e atualizamos o nome completo da classe na estrutura **PyTypeObject**.

Finalmente, atualizamos nosso arquivo `setup.py` para incluir o novo módulo,

```
from setuptools import Extension, setupsetup(ext_modules=[ Exter
```

então podemos instalar novamente para realizarmos o `import custom2`

```
$ python -m pip install .
```

Fornecendo controle mais preciso sobre atributos de dados

Nesta seção, forneceremos um controle mais preciso sobre como os atributos **first** e **last** são definidos no exemplo **Custom**. Na versão anterior do nosso módulo, as variáveis de instância **first** e **last** podiam receber valores que não fossem strings ou até mesmo ser excluídas. Queremos garantir que esses atributos sempre contenham strings.

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include <stddef.h> /* for offsetof() */

typedef struct {
    PyObject_HEAD
```

```
    PyObject *first; /* first name */
    PyObject *last;  /* last name */
    int number;
} CustomObject;

static void
Custom_dealloc(PyObject *op)
{
    CustomObject *self = (CustomObject *) op;
    Py_XDECREF(self->first);
    Py_XDECREF(self->last);
    Py_TYPE(self)->tp_free(self);
}

static PyObject *
Custom_new(PyTypeObject *type, PyObject *args, PyObject *kwds)
{
    CustomObject *self;
    self = (CustomObject *) type->tp_alloc(type, 0);
    if (self != NULL) {
        self->first = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
        if (self->first == NULL) {
            Py_DECREF(self);
            return NULL;
        }
        self->last = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
        if (self->last == NULL) {
            Py_DECREF(self);
            return NULL;
        }
        self->number = 0;
    }
    return (PyObject *) self;
}

static int
Custom_init(PyObject *op, PyObject *args, PyObject *kwds)
{
    CustomObject *self = (CustomObject *) op;
    static char *kwlist[] = {"first", "last", "number", NULL};
    PyObject *first = NULL, *last = NULL;

    if (!PyArg_ParseTupleAndKeywords(args, kwds, "|UUi", kwlist,
```

```
                                        &first, &last,
                                        &self->number))
        return -1;

    if (first) {
        Py_SETREF(self->first, Py_NewRef(first));
    }
    if (last) {
        Py_SETREF(self->last, Py_NewRef(last));
    }
    return 0;
}

static PyMemberDef Custom_members[] = {
    {"number", Py_T_INT, offsetof(CustomObject, number), 0,
     "custom number"},
    {NULL}  /* Sentinel */
};

static PyObject *
Custom_getfirst(PyObject *op, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    return Py_NewRef(self->first);
}

static int
Custom_setfirst(PyObject *op, PyObject *value, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    if (value == NULL) {
        PyErr_SetString(PyExc_TypeError, "Cannot delete the firs
        return -1;
    }
    if (!PyUnicode_Check(value)) {
        PyErr_SetString(PyExc_TypeError,
                        "The first attribute value must be a str
        return -1;
    }
    Py_SETREF(self->first, Py_NewRef(value));
    return 0;
}
```

```
static PyObject *
Custom_getlast(PyObject *op, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    return Py_NewRef(self->last);
}

static int
Custom_setlast(PyObject *op, PyObject *value, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    if (value == NULL) {
        PyErr_SetString(PyExc_TypeError, "Cannot delete the last
        return -1;
    }
    if (!PyUnicode_Check(value)) {
        PyErr_SetString(PyExc_TypeError,
                        "The last attribute value must be a stri
        return -1;
    }
    Py_SETREF(self->last, Py_NewRef(value));
    return 0;
}

static PyGetSetDef Custom_getsetters[] = {
    {"first", Custom_getfirst, Custom_setfirst,
     "first name", NULL},
    {"last", Custom_getlast, Custom_setlast,
     "last name", NULL},
    {NULL}  /* Sentinel */
};

static PyObject *
Custom_name(PyObject *op, PyObject *Py_UNUSED(dummy))
{
    CustomObject *self = (CustomObject *) op;
    return PyUnicode_FromFormat("%S %S", self->first, self->last
}

static PyMethodDef Custom_methods[] = {
    {"name", Custom_name, METH_NOARGS,
     "Return the name, combining the first and last name"
    },
```

```
    {NULL}  /* Sentinel */
};

static PyTypeObject CustomType = {
    .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "custom3.Custom",
    .tp_doc = PyDoc_STR("Custom objects"),
    .tp_basicsize = sizeof(CustomObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .tp_new = Custom_new,
    .tp_init = Custom_init,
    .tp_dealloc = Custom_dealloc,
    .tp_members = Custom_members,
    .tp_methods = Custom_methods,
    .tp_getset = Custom_getsetters,
};

static int
custom_module_exec(PyObject *m)
{
    if (PyType_Ready(&CustomType) < 0) {
        return -1;
    }

    if (PyModule_AddObjectRef(m, "Custom", (PyObject *) &Custom1
        return -1;
    }

    return 0;
}

static PyModuleDef_Slot custom_module_slots[] = {
    {Py_mod_exec, custom_module_exec},
    {Py_mod_multiple_interpreters, Py_MOD_MULTIPLE_INTERPRETERS_
    {0, NULL}
};

static PyModuleDef custom_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "custom3",
    .m_doc = "Example module that creates an extension type.",
    .m_size = 0,
```

```
    .m_slots = custom_module_slots,
};

PyMODINIT_FUNC
PyInit_custom3(void)
{
    return PyModuleDef_Init(&custom_module);
}
```

Para oferecer um controle maior sobre os atributos **`first`** e **`last`**, usaremos funções personalizadas de acesso (getters) e modificação (setters). A seguir estão as funções para obter e definir o atributo **`first`**:

```
static PyObject *
Custom_getfirst(PyObject *op, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    Py_INCREF(self->first);
    return self->first;
}

static int
Custom_setfirst(PyObject *op, PyObject *value, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    PyObject *tmp;
    if (value == NULL) {
        PyErr_SetString(PyExc_TypeError, "Cannot delete the firs
        return -1;
    }
    if (!PyUnicode_Check(value)) {
        PyErr_SetString(PyExc_TypeError,
                        "The first attribute value must be a str
        return -1;
    }
    tmp = self->first;
    Py_INCREF(value);
    self->first = value;
    Py_DECREF(tmp);
    return 0;
}
```

A função getter recebe um objeto **`Custom`** e um closure, que é um ponteiro para void. Neste caso, o closure é ignorado. (O closure oferece suporte a um modo de usar mais avançado, no qual dados de definição são passados ao getter e ao setter. Isso pode, por exemplo, permitir que um único conjunto de funções getter e setter decida qual atributo obter ou definir com base nos dados presentes no closure.)

A função setter recebe o objeto **`Custom`**, o novo valor e o closure. O novo valor pode ser `NULL`, caso em que o atributo está sendo excluído. No nosso setter, levantamos um erro se o atributo for excluído ou se o novo valor não for uma string.

Nós criamos um vetor de estruturas **`PyGetSetDef`**:

```
static PyGetSetDef Custom_getsetters[] = {
    {"first", Custom_getfirst, Custom_setfirst,
     "first name", NULL},
    {"last", Custom_getlast, Custom_setlast,
     "last name", NULL},
    {NULL}  /* Sentinel */
};
```

e registra isso num slot **`tp_getset`**:

```
.tp_getset = Custom_getsetters,
```

A última entrada em uma estrutura **`PyGetSetDef`** é o “closure” mencionado acima. Neste caso, não estamos usando um closure, então simplesmente passamos `NULL`.

Também removemos as definições de membros para esses atributos:

```
static PyMemberDef Custom_members[] = {
    {"number", Py_T_INT, offsetof(CustomObject, number), 0,
     "custom number"},
    {NULL}  /* Sentinel */
};
```

Também precisamos atualizar o manipulador **`tp_init`** para permitir apenas strings [[3]] como valores passados:

```
static int
Custom_init(PyObject *op, PyObject *args, PyObject *kwds)
{
    CustomObject *self = (CustomObject *) op;
    static char *kwlist[] = {"first", "last", "number", NULL};
    PyObject *first = NULL, *last = NULL, *tmp;

    if (!PyArg_ParseTupleAndKeywords(args, kwds, "|UUi", kwlist,
                                     &first, &last,
                                     &self->number))
        return -1;

    if (first) {
        tmp = self->first;
        Py_INCREF(first);
        self->first = first;
        Py_DECREF(tmp);
    }
    if (last) {
        tmp = self->last;
        Py_INCREF(last);
        self->last = last;
        Py_DECREF(tmp);
    }
    return 0;
}
```

Com essas alterações, podemos garantir que os membros `first` e `last` nunca sejam `NULL`, de modo que podemos remover quase todas as verificações de valores `NULL`. Isso significa que a maioria das chamadas para **`Py_XDECREF()`** pode ser substituída por chamadas para **`Py_DECREF()`**. O único lugar em que não podemos substituir essas chamadas é na implementação de `tp_dealloc`, onde existe a possibilidade de que a inicialização desses membros tenha falhado em `tp_new`.

Também renomeamos a função de inicialização do módulo e o nome do módulo dentro da função de inicialização, como fizemos anteriormente, e adicionamos uma definição extra ao arquivo `setup.py`.

Apoiando a coleta de lixo cíclica

O Python possui um coletor de lixo (GC) cíclico que pode identificar objetos que não são mais necessários mesmo quando suas contagens de referências não são zero. Isso pode acontecer quando os objetos participam de ciclos. Por exemplo, considere:

```
>>> l = []
>>> l.append(l)
>>> del l
```

Neste exemplo, criamos uma lista que contém a si mesma. Quando a apagamos, ela ainda possui uma referência apontando para si própria. Sua contagem de referências não cai para zero. Felizmente, o coletor de lixo cíclico do Python acabará percebendo que a lista é lixo e irá liberá-la.

No segunda versão do exemplo de **`Custom`**, permitimos que qualquer tipo de objeto fosse armazenado nos atributos **`first`** ou **`last`** [[4]]. Além disso, na segunda e na terceira versões, permitimos estender **`Custom`**, e subclasses podem adicionar atributos arbitrários. Por qualquer um desses dois motivos, instâncias de **`Custom`** podem participar de ciclos:

```
>>> import custom3
>>> class Derived(custom3.Custom): pass
...
>>> n = Derived()
>>> n.some_attribute = n
```

Para permitir que uma instância de **`Custom`** que participa de um ciclo de referência seja corretamente detectada e coletada pelo coletor de lixo cíclico, nosso tipo **`Custom`** precisa preencher dois slots adicionais e ativar um sinalizador que habilita esses slots:

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
#include <stddef.h> /* for offsetof() */

typedef struct {
    PyObject_HEAD
    PyObject *first; /* first name */
    PyObject *last;  /* last name */
    int number;
```

```
} CustomObject;

static int
Custom_traverse(PyObject *op, visitproc visit, void *arg)
{
    CustomObject *self = (CustomObject *) op;
    Py_VISIT(self->first);
    Py_VISIT(self->last);
    return 0;
}

static int
Custom_clear(PyObject *op)
{
    CustomObject *self = (CustomObject *) op;
    Py_CLEAR(self->first);
    Py_CLEAR(self->last);
    return 0;
}

static void
Custom_dealloc(PyObject *op)
{
    PyObject_GC_UnTrack(op);
    (void)Custom_clear(op);
    Py_TYPE(op)->tp_free(op);
}

static PyObject *
Custom_new(PyTypeObject *type, PyObject *args, PyObject *kwds)
{
    CustomObject *self;
    self = (CustomObject *) type->tp_alloc(type, 0);
    if (self != NULL) {
        self->first = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
        if (self->first == NULL) {
            Py_DECREF(self);
            return NULL;
        }
        self->last = Py_GetConstant(Py_CONSTANT_EMPTY_STR);
        if (self->last == NULL) {
            Py_DECREF(self);
            return NULL;
```

```
        }
        self->number = 0;
    }
    return (PyObject *) self;
}

static int
Custom_init(PyObject *op, PyObject *args, PyObject *kwds)
{
    CustomObject *self = (CustomObject *) op;
    static char *kwlist[] = {"first", "last", "number", NULL};
    PyObject *first = NULL, *last = NULL;

    if (!PyArg_ParseTupleAndKeywords(args, kwds, "|UUi", kwlist,
                                     &first, &last,
                                     &self->number))
        return -1;

    if (first) {
        Py_SETREF(self->first, Py_NewRef(first));
    }
    if (last) {
        Py_SETREF(self->last, Py_NewRef(last));
    }
    return 0;
}

static PyMemberDef Custom_members[] = {
    {"number", Py_T_INT, offsetof(CustomObject, number), 0,
     "custom number"},
    {NULL}  /* Sentinel */
};

static PyObject *
Custom_getfirst(PyObject *op, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    return Py_NewRef(self->first);
}

static int
Custom_setfirst(PyObject *op, PyObject *value, void *closure)
{
```

```
    CustomObject *self = (CustomObject *) op;
    if (value == NULL) {
        PyErr_SetString(PyExc_TypeError, "Cannot delete the firs
        return -1;
    }
    if (!PyUnicode_Check(value)) {
        PyErr_SetString(PyExc_TypeError,
                        "The first attribute value must be a str
        return -1;
    }
    Py_XSETREF(self->first, Py_NewRef(value));
    return 0;
}

static PyObject *
Custom_getlast(PyObject *op, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    return Py_NewRef(self->last);
}

static int
Custom_setlast(PyObject *op, PyObject *value, void *closure)
{
    CustomObject *self = (CustomObject *) op;
    if (value == NULL) {
        PyErr_SetString(PyExc_TypeError, "Cannot delete the last
        return -1;
    }
    if (!PyUnicode_Check(value)) {
        PyErr_SetString(PyExc_TypeError,
                        "The last attribute value must be a stri
        return -1;
    }
    Py_XSETREF(self->last, Py_NewRef(value));
    return 0;
}

static PyGetSetDef Custom_getsetters[] = {
    {"first", Custom_getfirst, Custom_setfirst,
     "first name", NULL},
    {"last", Custom_getlast, Custom_setlast,
     "last name", NULL},
```

```
    {NULL}  /* Sentinel */
};

static PyObject *
Custom_name(PyObject *op, PyObject *Py_UNUSED(dummy))
{
    CustomObject *self = (CustomObject *) op;
    return PyUnicode_FromFormat("%S %S", self->first, self->last
}

static PyMethodDef Custom_methods[] = {
    {"name", Custom_name, METH_NOARGS,
     "Return the name, combining the first and last name"
    },
    {NULL}  /* Sentinel */
};

static PyTypeObject CustomType = {
    .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "custom4.Custom",
    .tp_doc = PyDoc_STR("Custom objects"),
    .tp_basicsize = sizeof(CustomObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE | Py_TP
    .tp_new = Custom_new,
    .tp_init = Custom_init,
    .tp_dealloc = Custom_dealloc,
    .tp_traverse = Custom_traverse,
    .tp_clear = Custom_clear,
    .tp_members = Custom_members,
    .tp_methods = Custom_methods,
    .tp_getset = Custom_getsetters,
};

static int
custom_module_exec(PyObject *m)
{
    if (PyType_Ready(&CustomType) < 0) {
        return -1;
    }

    if (PyModule_AddObjectRef(m, "Custom", (PyObject *) &CustomT
        return -1;
```

```
    }

    return 0;
}

static PyModuleDef_Slot custom_module_slots[] = {
    {Py_mod_exec, custom_module_exec},
    {Py_mod_multiple_interpreters, Py_MOD_MULTIPLE_INTERPRETERS_
    {0, NULL}
};

static PyModuleDef custom_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "custom4",
    .m_doc = "Example module that creates an extension type.",
    .m_size = 0,
    .m_slots = custom_module_slots,
};

PyMODINIT_FUNC
PyInit_custom4(void)
{
    return PyModuleDef_Init(&custom_module);
}
```

Primeiro, o método de travessia informa ao coletor de lixo cíclico quais subobjetos podem participar de ciclos:

```
static int
Custom_traverse(PyObject *op, visitproc visit, void *arg)
{
    CustomObject *self = (CustomObject *) op;
    int vret;
    if (self->first) {
        vret = visit(self->first, arg);
        if (vret != 0)
            return vret;
    }
    if (self->last) {
        vret = visit(self->last, arg);
        if (vret != 0)
            return vret;
```

```
    }
    return 0;
}
```

Para cada subobjeto que pode participar de ciclos, precisamos chamar a função **`visit()`**, que é passada para o método de travessia. A função **`visit()`** recebe como argumentos o subobjeto e o argumento extra *arg* fornecido ao método de travessia. Ela retorna um valor inteiro que deve ser retornado se for diferente de zero.

O Python fornece o macro **`Py_VISIT()`**, que automatiza as chamadas às funções visit. Com **`Py_VISIT()`**, podemos minimizar a quantidade de boilerplate em `Custom_traverse`:

```
static int
Custom_traverse(PyObject *op, visitproc visit, void *arg)
{
    CustomObject *self = (CustomObject *) op;
    Py_VISIT(self->first);
    Py_VISIT(self->last);
    return 0;
}
```

Nota

A implementação de **`tp_traverse`** deve nomear seus argumentos exatamente como *visit* e *arg* para que seja possível usar **`Py_VISIT()`**.

Segundo, precisamos fornecer um método para limpar quaisquer subobjetos que possam participar de ciclos:

```
static int
Custom_clear(PyObject *op)
{
    CustomObject *self = (CustomObject *) op;
    Py_CLEAR(self->first);
    Py_CLEAR(self->last);
    return 0;
}
```

Observe o uso do macro `Py_CLEAR()`. Ele é a forma recomendada e segura de limpar atributos de dados de tipos arbitrários enquanto decrementa suas contagens de referências. Se você chamasse `Py_XDECREF()` no atributo antes de defini-lo como `NULL`, haveria a possibilidade de que o destrutor do atributo chamasse novamente algum código que lesse o atributo (especialmente se houver um ciclo de referência).

Nota

Você poderia emular `Py_CLEAR()` ao escrever:

```
PyObject *tmp;
tmp = self->first;
self->first = NULL;
Py_XDECREF(tmp);
```

Apesar disso, é muito mais fácil e menos propenso a erros, usar sempre `Py_CLEAR()` ao excluir um atributo. Não tente micro-otimizar às custas da robustez!

O desalocador `Custom_dealloc` pode executar código arbitrário ao limpar atributos. Isso significa que o GC pode ser acionado dentro da função. Como o GC presume que a contagem de referências não é zero, precisamos remover o objeto do rastreamento do GC chamando `PyObject_GC_UnTrack()` antes de limpar os membros. A seguir está nossa versão reimplementada do desalocador usando **`PyObject_GC_UnTrack()`** e `Custom_clear`:

```
static void
Custom_dealloc(PyObject *op)
{
    PyObject_GC_UnTrack(op);
    (void)Custom_clear(op);
    Py_TYPE(op)->tp_free(op);
}
```

Por fim, adicionamos o macro `Py_TPFLAGS_HAVE_GC` aos sinalizadores da classe:

```
.tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE | Py_TPFLAG
```

Isso é praticamente tudo. Se tivéssemos escrito manipuladores personalizados para `tp_alloc` ou `tp_free`, precisaríamos adaptá-los para a coleta de lixo cíclica. A maior parte das extensões usará as versões fornecidas automaticamente.

Criando subclasses de outros tipos

É possível criar novos tipos de extensão derivados de tipos já existentes. É mais fácil herdar dos tipos embutidos, já que uma extensão pode usar diretamente o `PyTypeObject` de que precisa. Pode ser difícil compartilhar essas estruturas **`PyTypeObject`** entre módulos de extensão distintos.

Neste exemplo, criaremos um tipo **`SubList`** que herda do tipo embutido `list`. O novo tipo será totalmente compatível com listas comuns, mas terá um método adicional, **`increment()`**, que incrementa um contador interno:

```
>>> import sublist
>>> s = sublist.SubList(range(3))
>>> s.extend(s)
>>> print(len(s))
6
>>> print(s.increment())
1
>>> print(s.increment())
2
```

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>

typedef struct {
    PyListObject list;
    int state;
} SubListObject;

static PyObject *
SubList_increment(PyObject *op, PyObject *Py_UNUSED(dummy))
{
```

```
    SubListObject *self = (SubListObject *) op;
    self->state++;
    return PyLong_FromLong(self->state);
}

static PyMethodDef SubList_methods[] = {
    {"increment", SubList_increment, METH_NOARGS,
     PyDoc_STR("increment state counter")},
    {NULL},
};

static int
SubList_init(PyObject *op, PyObject *args, PyObject *kwds)
{
    SubListObject *self = (SubListObject *) op;
    if (PyList_Type.tp_init(op, args, kwds) < 0)
        return -1;
    self->state = 0;
    return 0;
}

static PyTypeObject SubListType = {
    .ob_base = PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "sublist.SubList",
    .tp_doc = PyDoc_STR("SubList objects"),
    .tp_basicsize = sizeof(SubListObject),
    .tp_itemsize = 0,
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE,
    .tp_init = SubList_init,
    .tp_methods = SubList_methods,
};

static int
sublist_module_exec(PyObject *m)
{
    SubListType.tp_base = &PyList_Type;
    if (PyType_Ready(&SubListType) < 0) {
        return -1;
    }

    if (PyModule_AddObjectRef(m, "SubList", (PyObject *) &SubLis
        return -1;
    }
```

```
    return 0;
}

static PyModuleDef_Slot sublist_module_slots[] = {
    {Py_mod_exec, sublist_module_exec},
    {Py_mod_multiple_interpreters, Py_MOD_MULTIPLE_INTERPRETERS_
    {0, NULL}
};

static PyModuleDef sublist_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "sublist",
    .m_doc = "Example module that creates an extension type.",
    .m_size = 0,
    .m_slots = sublist_module_slots,
};

PyMODINIT_FUNC
PyInit_sublist(void)
{
    return PyModuleDef_Init(&sublist_module);
}
```

Como você pode consultar, o código-fonte se assemelha bastante aos exemplos de **`Custom`** das seções anteriores. Vamos detalhar as principais diferenças entre eles.

```
typedef struct {
    PyListObject list;
    int state;
} SubListObject;
```

A diferença principal para objetos de tipos derivados é que a estrutura de objeto do tipo base deve ser o primeiro valor. O tipo base já inclui o **`PyObject_HEAD()`** no início de sua própria estrutura.

Quando um objeto Python é uma instância de **`SubList`**, seu ponteiro `PyObject *` pode ser convertido com segurança tanto para `PyListObject *` quanto para `SubListObject *`:

```
static int
SubList_init(PyObject *op, PyObject *args, PyObject *kwds)
{
    SubListObject *self = (SubListObject *) op;
    if (PyList_Type.tp_init(op, args, kwds) < 0)
        return -1;
    self->state = 0;
    return 0;
}
```

Vemos acima como chamar o método `__init__()` do tipo base.

Esse padrão é importante ao escrever um tipo que possui membros personalizados em `tp_new` e `tp_dealloc`. O manipulador **`tp_new`** não deve realmente criar a memória do objeto usando o seu próprio `tp_alloc`; em vez disso, deve permitir que a classe base faça isso, chamando o seu próprio **`tp_new`**.

A estrutura `PyTypeObject` provê o campo `tp_base`, que especifica a classe base concreta do tipo. Devido a problemas de compilação multiplataforma, não se deve preencher esse campo diretamente com uma referência para `PyList_Type`; isso deve ser feito dentro da função `Py_mod_exec`:

```
static int
sublist_module_exec(PyObject *m)
{
    SubListType.tp_base = &PyList_Type;
    if (PyType_Ready(&SubListType) < 0) {
        return -1;
    }

    if (PyModule_AddObjectRef(m, "SubList", (PyObject *) &SubLis
        return -1;
    }

    return 0;
}
```

Antes de chamar `PyType_Ready()`, a estrutura do tipo deve ter o slot `tp_base` preenchido. Ao derivarmos um tipo existente, não é necessário

preencher o slot `tp_alloc` com `PyType_GenericNew()` — a função de alocação do tipo base será herdada.

Depois disso, chamar `PyType_Ready()` e adicionar o objeto de tipo ao módulo é o mesmo processo usado nos exemplos básicos com `Custom`.

Notas de rodapé

[1]

Isso é verdade quando sabemos que o objeto é um tipo básico, como uma string ou um float.

[2]

Confiamos nisso no manipulador `tp_dealloc` neste exemplo, porque o nosso tipo não oferece suporte à coleta de lixo.

[3]

Agora sabemos que o primeiro e último membros são strings, então talvez pudéssemos ter menos cuidado com a diminuição de suas contagens de referência, no entanto, aceitamos instâncias de subclasses de string. Mesmo que a desalocação de strings normais não retorne aos nossos objetos, não podemos garantir que a desalocação de uma instância de uma subclasse de cadeias de caracteres não retornará aos nossos objetos.

[4]

Além disso, mesmo com nossos atributos restritos a instâncias de strings, o usuário poderia passar arbitrariamente subclasses `str` e, portanto, ainda criar ciclos de referência.

Defining Extension Types: Assorted Topics

This section aims to give a quick fly-by on the various type methods you can implement and what they do.

Here is the definition of `PyTypeObject`, with some fields only used in debug builds omitted:

```
typedef struct _typeobject {
    PyObject_VAR_HEAD
    const char *tp_name; /* For printing, in format "<module>.<r
    Py_ssize_t tp_basicsize, tp_itemsize; /* For allocation */

    /* Methods to implement standard operations */

    destructor tp_dealloc;
    Py_ssize_t tp_vectorcall_offset;
    getattrfunc tp_getattr;
    setattrfunc tp_setattr;
    PyAsyncMethods *tp_as_async; /* formerly known as tp_compare
                                    or tp_reserved (Python 3) */
    reprfunc tp_repr;

    /* Method suites for standard classes */

    PyNumberMethods *tp_as_number;
    PySequenceMethods *tp_as_sequence;
    PyMappingMethods *tp_as_mapping;

    /* More standard operations (here for binary compatibility)

    hashfunc tp_hash;
    ternaryfunc tp_call;
    reprfunc tp_str;
    getattrofunc tp_getattro;
    setattrofunc tp_setattro;
```

```
/* Functions to access object as input/output buffer */
PyBufferProcs *tp_as_buffer;

/* Flags to define presence of optional/expanded features */
unsigned long tp_flags;

const char *tp_doc; /* Documentation string */

/* Assigned meaning in release 2.0 */
/* call function for all accessible objects */
traverseproc tp_traverse;

/* delete references to contained objects */
inquiry tp_clear;

/* Assigned meaning in release 2.1 */
/* rich comparisons */
richcmpfunc tp_richcompare;

/* weak reference enabler */
Py_ssize_t tp_weaklistoffset;

/* Iterators */
getiterfunc tp_iter;
iternextfunc tp_iternext;

/* Attribute descriptor and subclassing stuff */
PyMethodDef *tp_methods;
PyMemberDef *tp_members;
PyGetSetDef *tp_getset;
// Strong reference on a heap type, borrowed reference on a
PyTypeObject *tp_base;
PyObject *tp_dict;
descrgetfunc tp_descr_get;
descrsetfunc tp_descr_set;
Py_ssize_t tp_dictoffset;
initproc tp_init;
allocfunc tp_alloc;
newfunc tp_new;
freefunc tp_free; /* Low-level free-memory routine */
inquiry tp_is_gc; /* For PyObject_IS_GC */
PyObject *tp_bases;
PyObject *tp_mro; /* method resolution order */
```

```
    PyObject *tp_cache; /* no longer used */
    void *tp_subclasses;  /* for static builtin types this is ar
    PyObject *tp_weaklist; /* not used for static builtin types
    destructor tp_del;

    /* Type attribute cache version tag. Added in version 2.6.
     * If zero, the cache is invalid and must be initialized.
     */
    unsigned int tp_version_tag;

    destructor tp_finalize;
    vectorcallfunc tp_vectorcall;

    /* bitset of which type-watchers care about this type */
    unsigned char tp_watched;

    /* Number of tp_version_tag values used.
     * Set to _Py_ATTR_CACHE_UNUSED if the attribute cache is
     * disabled for this type (e.g. due to custom MRO entries).
     * Otherwise, limited to MAX_VERSIONS_PER_CLASS (defined els
     */
    uint16_t tp_versions_used;
} PyTypeObject;
```

Now that's a *lot* of methods. Don't worry too much though – if you have a type you want to define, the chances are very good that you will only implement a handful of these.

As you probably expect by now, we're going to go over this and give more information about the various handlers. We won't go in the order they are defined in the structure, because there is a lot of historical baggage that impacts the ordering of the fields. It's often easiest to find an example that includes the fields you need and then change the values to suit your new type.

```
const char *tp_name; /* For printing */
```

The name of the type – as mentioned in the previous chapter, this will appear in various places, almost entirely for diagnostic purposes. Try to choose something that will be helpful in such a situation!

```
Py_ssize_t tp_basicsize, tp_itemsize; /* For allocation */
```

These fields tell the runtime how much memory to allocate when new objects of this type are created. Python has some built-in support for variable length structures (think: strings, tuples) which is where the `tp_itemsize` field comes in. This will be dealt with later.

```
const char *tp_doc;
```

Here you can put a string (or its address) that you want returned when the Python script references `obj.__doc__` to retrieve the doc string.

Now we come to the basic type methods – the ones most extension types will implement.

Finalization and De-allocation

```
destructor tp_dealloc;
```

This function is called when the reference count of the instance of your type is reduced to zero and the Python interpreter wants to reclaim it. If your type has memory to free or other clean-up to perform, you can put it here. The object itself needs to be freed here as well. Here is an example of this function:

```
static void
newdatatype_dealloc(PyObject *op)
{
    newdatatypeobject *self = (newdatatypeobject *) op;
    free(self->obj_UnderlyingDatatypePtr);
    Py_TYPE(self)->tp_free(self);
}
```

If your type supports garbage collection, the destructor should call `PyObject_GC_UnTrack()` before clearing any member fields:

```
static void
newdatatype_dealloc(PyObject *op)
{
```

```
    newdatatypeobject *self = (newdatatypeobject *) op;
    PyObject_GC_UnTrack(op);
    Py_CLEAR(self->other_obj);
    ...
    Py_TYPE(self)->tp_free(self);
}
```

One important requirement of the deallocator function is that it leaves any pending exceptions alone. This is important since deallocators are frequently called as the interpreter unwinds the Python stack; when the stack is unwound due to an exception (rather than normal returns), nothing is done to protect the deallocators from seeing that an exception has already been set. Any actions which a deallocator performs which may cause additional Python code to be executed may detect that an exception has been set. This can lead to misleading errors from the interpreter. The proper way to protect against this is to save a pending exception before performing the unsafe action, and restoring it when done. This can be done using the **PyErr_Fetch()** and **PyErr_Restore()** functions:

```
static void
my_dealloc(PyObject *obj)
{
    MyObject *self = (MyObject *) obj;
    PyObject *cbresult;

    if (self->my_callback != NULL) {
        PyObject *err_type, *err_value, *err_traceback;

        /* This saves the current exception state */
        PyErr_Fetch(&err_type, &err_value, &err_traceback);

        cbresult = PyObject_CallNoArgs(self->my_callback);
        if (cbresult == NULL) {
           PyErr_WriteUnraisable(self->my_callback);
        }
        else {
            Py_DECREF(cbresult);
        }

        /* This restores the saved exception state */
        PyErr_Restore(err_type, err_value, err_traceback);
```

```
        Py_DECREF(self->my_callback);
    }
    Py_TYPE(self)->tp_free(self);
}
```

Nota

There are limitations to what you can safely do in a deallocator function. First, if your type supports garbage collection (using `tp_traverse` and/or `tp_clear`), some of the object's members can have been cleared or finalized by the time `tp_dealloc` is called. Second, in **`tp_dealloc`**, your object is in an unstable state: its reference count is equal to zero. Any call to a non-trivial object or API (as in the example above) might end up calling **`tp_dealloc`** again, causing a double free and a crash.

Starting with Python 3.4, it is recommended not to put any complex finalization code in `tp_dealloc`, and instead use the new `tp_finalize` type method.

Ver também

PEP 442 [https://peps.python.org/pep-0442/] explains the new finalization scheme.

Object Presentation

In Python, there are two ways to generate a textual representation of an object: the `repr()` function, and the `str()` function. (The `print()` function just calls **`str()`**.) These handlers are both optional.

```
reprfunc tp_repr;
reprfunc tp_str;
```

The `tp_repr` handler should return a string object containing a representation of the instance for which it is called. Here is a simple

example:

```
static PyObject *
newdatatype_repr(PyObject *op)
{
    newdatatypeobject *self = (newdatatypeobject *) op;
    return PyUnicode_FromFormat("Repr-ified_newdatatype{{size:%d
                                self->obj_UnderlyingDatatypePtr-
}
```

If no `tp_repr` handler is specified, the interpreter will supply a representation that uses the type's `tp_name` and a uniquely identifying value for the object.

The `tp_str` handler is to `str()` what the `tp_repr` handler described above is to `repr()`; that is, it is called when Python code calls **`str()`** on an instance of your object. Its implementation is very similar to the **`tp_repr`** function, but the resulting string is intended for human consumption. If **`tp_str`** is not specified, the **`tp_repr`** handler is used instead.

Here is a simple example:

```
static PyObject *
newdatatype_str(PyObject *op)
{
    newdatatypeobject *self = (newdatatypeobject *) op;
    return PyUnicode_FromFormat("Stringified_newdatatype{{size:%
                                self->obj_UnderlyingDatatypePtr-
}
```

Attribute Management

For every object which can support attributes, the corresponding type must provide the functions that control how the attributes are resolved. There needs to be a function which can retrieve attributes (if any are defined), and another to set attributes (if setting attributes is allowed). Removing an attribute is a special case, for which the new value passed to the handler is `NULL`.

Python supports two pairs of attribute handlers; a type that supports attributes only needs to implement the functions for one pair. The difference is that one pair takes the name of the attribute as a char*, while the other accepts a PyObject*. Each type can use whichever pair makes more sense for the implementation's convenience.

```
getattrfunc  tp_getattr;        /* char * version */
setattrfunc  tp_setattr;
/* ... */
getattrofunc tp_getattro;       /* PyObject * version */
setattrofunc tp_setattro;
```

If accessing attributes of an object is always a simple operation (this will be explained shortly), there are generic implementations which can be used to provide the PyObject* version of the attribute management functions. The actual need for type-specific attribute handlers almost completely disappeared starting with Python 2.2, though there are many examples which have not been updated to use some of the new generic mechanism that is available.

Generic Attribute Management

Most extension types only use *simple* attributes. So, what makes the attributes simple? There are only a couple of conditions that must be met:

1. The name of the attributes must be known when `PyType_Ready()` is called.
2. No special processing is needed to record that an attribute was looked up or set, nor do actions need to be taken based on the value.

Note that this list does not place any restrictions on the values of the attributes, when the values are computed, or how relevant data is stored.

When `PyType_Ready()` is called, it uses three tables referenced by the type object to create descriptors which are placed in the dictionary of the type object. Each descriptor controls access to one attribute of the instance object. Each of the tables is optional; if all three are `NULL`, instances of the type will only have attributes that are inherited from their base type, and

should leave the **tp_getattro** and **tp_setattro** fields NULL as well, allowing the base type to handle attributes.

The tables are declared as three fields of the type object:

```
struct PyMethodDef *tp_methods;
struct PyMemberDef *tp_members;
struct PyGetSetDef *tp_getset;
```

If **tp_methods** is not NULL, it must refer to an array of **PyMethodDef** structures. Each entry in the table is an instance of this structure:

```
typedef struct PyMethodDef {
    const char  *ml_name;       /* method name */
    PyCFunction  ml_meth;       /* implementation function */
    int          ml_flags;      /* flags */
    const char  *ml_doc;        /* docstring */
} PyMethodDef;
```

One entry should be defined for each method provided by the type; no entries are needed for methods inherited from a base type. One additional entry is needed at the end; it is a sentinel that marks the end of the array. The **ml_name** field of the sentinel must be NULL.

The second table is used to define attributes which map directly to data stored in the instance. A variety of primitive C types are supported, and access may be read-only or read-write. The structures in the table are defined as:

```
typedef struct PyMemberDef {
    const char *name;
    int         type;
    int         offset;
    int         flags;
    const char *doc;
} PyMemberDef;
```

For each entry in the table, a descriptor will be constructed and added to the type which will be able to extract a value from the instance structure. The **type** field should contain a type code like **Py_T_INT** or **Py_T_DOUBLE**; the

value will be used to determine how to convert Python values to and from C values. The `flags` field is used to store flags which control how the attribute can be accessed: you can set it to `Py_READONLY` to prevent Python code from setting it.

An interesting advantage of using the `tp_members` table to build descriptors that are used at runtime is that any attribute defined this way can have an associated doc string simply by providing the text in the table. An application can use the introspection API to retrieve the descriptor from the class object, and get the doc string using its `__doc__` attribute.

As with the `tp_methods` table, a sentinel entry with a `ml_name` value of `NULL` is required.

Type-specific Attribute Management

For simplicity, only the char* version will be demonstrated here; the type of the name parameter is the only difference between the char* and PyObject* flavors of the interface. This example effectively does the same thing as the generic example above, but does not use the generic support added in Python 2.2. It explains how the handler functions are called, so that if you do need to extend their functionality, you'll understand what needs to be done.

The `tp_getattr` handler is called when the object requires an attribute look-up. It is called in the same situations where the `__getattr__()` method of a class would be called.

Aqui está um exemplo:

```
static PyObject *
newdatatype_getattr(PyObject *op, char *name)
{
    newdatatypeobject *self = (newdatatypeobject *) op;
    if (strcmp(name, "data") == 0) {
        return PyLong_FromLong(self->data);
    }

    PyErr_Format(PyExc_AttributeError,
```

```
                 "'%.100s' object has no attribute '%.400s'",
                 Py_TYPE(self)->tp_name, name);
    return NULL;
}
```

The **tp_setattr** handler is called when the **__setattr__()** or **__delattr__()** method of a class instance would be called. When an attribute should be deleted, the third parameter will be `NULL`. Here is an example that simply raises an exception; if this were really all you wanted, the **tp_setattr** handler should be set to `NULL`.

```
static int
newdatatype_setattr(PyObject *op, char *name, PyObject *v)
{
    PyErr_Format(PyExc_RuntimeError, "Read-only attribute: %s",
    return -1;
}
```

Object Comparison

```
richcmpfunc tp_richcompare;
```

The **tp_richcompare** handler is called when comparisons are needed. It is analogous to the rich comparison methods, like **__lt__()**, and also called by **PyObject_RichCompare()** and **PyObject_RichCompareBool()**.

This function is called with two Python objects and the operator as arguments, where the operator is one of `Py_EQ`, `Py_NE`, `Py_LE`, `Py_GE`, `Py_LT` or `Py_GT`. It should compare the two objects with respect to the specified operator and return `Py_True` or `Py_False` if the comparison is successful, `Py_NotImplemented` to indicate that comparison is not implemented and the other object's comparison method should be tried, or `NULL` if an exception was set.

Here is a sample implementation, for a datatype that is considered equal if the size of an internal pointer is equal:

```
static PyObject *
newdatatype_richcmp(PyObject *lhs, PyObject *rhs, int op)
{
    newdatatypeobject *obj1 = (newdatatypeobject *) lhs;
    newdatatypeobject *obj2 = (newdatatypeobject *) rhs;
    PyObject *result;
    int c, size1, size2;

    /* code to make sure that both arguments are of type
       newdatatype omitted */

    size1 = obj1->obj_UnderlyingDatatypePtr->size;
    size2 = obj2->obj_UnderlyingDatatypePtr->size;

    switch (op) {
    case Py_LT: c = size1 <  size2; break;
    case Py_LE: c = size1 <= size2; break;
    case Py_EQ: c = size1 == size2; break;
    case Py_NE: c = size1 != size2; break;
    case Py_GT: c = size1 >  size2; break;
    case Py_GE: c = size1 >= size2; break;
    }
    result = c ? Py_True : Py_False;
    return Py_NewRef(result);
 }
```

Abstract Protocol Support

Python supports a variety of *abstract* ‘protocols;’ the specific interfaces provided to use these interfaces are documented in Camada de Objetos Abstratos.

A number of these abstract interfaces were defined early in the development of the Python implementation. In particular, the number, mapping, and sequence protocols have been part of Python since the beginning. Other protocols have been added over time. For protocols which depend on several handler routines from the type implementation, the older protocols have been defined as optional blocks of handlers referenced by the type object. For newer protocols there are additional slots in the main type object, with a flag bit being set to indicate that the slots are present and

should be checked by the interpreter. (The flag bit does not indicate that the slot values are non-`NULL`. The flag may be set to indicate the presence of a slot, but a slot may still be unfilled.)

```
PyNumberMethods   *tp_as_number;
PySequenceMethods *tp_as_sequence;
PyMappingMethods  *tp_as_mapping;
```

If you wish your object to be able to act like a number, a sequence, or a mapping object, then you place the address of a structure that implements the C type **`PyNumberMethods`**, **`PySequenceMethods`**, or **`PyMappingMethods`**, respectively. It is up to you to fill in this structure with appropriate values. You can find examples of the use of each of these in the `Objects` directory of the Python source distribution.

```
hashfunc tp_hash;
```

This function, if you choose to provide it, should return a hash number for an instance of your data type. Here is a simple example:

```
static Py_hash_t
newdatatype_hash(PyObject *op)
{
    newdatatypeobject *self = (newdatatypeobject *) op;
    Py_hash_t result;
    result = self->some_size + 32767 * self->some_number;
    if (result == -1) {
        result = -2;
    }
    return result;
}
```

`Py_hash_t` is a signed integer type with a platform-varying width. Returning `-1` from **`tp_hash`** indicates an error, which is why you should be careful to avoid returning it when hash computation is successful, as seen above.

```
ternaryfunc tp_call;
```

This function is called when an instance of your data type is “called”, for example, if `obj1` is an instance of your data type and the Python script contains `obj1('hello')`, the **`tp_call`** handler is invoked.

This function takes three arguments:

1. *self* is the instance of the data type which is the subject of the call. If the call is `obj1('hello')`, then *self* is `obj1`.
2. *args* is a tuple containing the arguments to the call. You can use **`PyArg_ParseTuple()`** to extract the arguments.
3. *kwds* is a dictionary of keyword arguments that were passed. If this is non-`NULL` and you support keyword arguments, use **`PyArg_ParseTupleAndKeywords()`** to extract the arguments. If you do not want to support keyword arguments and this is non-`NULL`, raise a **`TypeError`** with a message saying that keyword arguments are not supported.

Here is a toy `tp_call` implementation:

```
static PyObject *
newdatatype_call(PyObject *op, PyObject *args, PyObject *kwds)
{
    newdatatypeobject *self = (newdatatypeobject *) op;
    PyObject *result;
    const char *arg1;
    const char *arg2;
    const char *arg3;

    if (!PyArg_ParseTuple(args, "sss:call", &arg1, &arg2, &arg3)
        return NULL;
    }
    result = PyUnicode_FromFormat(
        "Returning -- value: [%d] arg1: [%s] arg2: [%s] arg3: [%
        self->obj_UnderlyingDatatypePtr->size,
        arg1, arg2, arg3);
    return result;
}
```

```
/* Iterators */
getiterfunc tp_iter;
```

```
iternextfunc tp_iternext;
```

These functions provide support for the iterator protocol. Both handlers take exactly one parameter, the instance for which they are being called, and return a new reference. In the case of an error, they should set an exception and return `NULL`. `tp_iter` corresponds to the Python `__iter__()` method, while `tp_iternext` corresponds to the Python `__next__()` method.

Any iterable object must implement the `tp_iter` handler, which must return an iterator object. Here the same guidelines apply as for Python classes:

- For collections (such as lists and tuples) which can support multiple independent iterators, a new iterator should be created and returned by each call to `tp_iter`.
- Objects which can only be iterated over once (usually due to side effects of iteration, such as file objects) can implement `tp_iter` by returning a new reference to themselves – and should also therefore implement the `tp_iternext` handler.

Any iterator object should implement both `tp_iter` and `tp_iternext`. An iterator's **`tp_iter`** handler should return a new reference to the iterator. Its **`tp_iternext`** handler should return a new reference to the next object in the iteration, if there is one. If the iteration has reached the end, **`tp_iternext`** may return `NULL` without setting an exception, or it may set **`StopIteration`** *in addition* to returning `NULL`; avoiding the exception can yield slightly better performance. If an actual error occurs, **`tp_iternext`** should always set an exception and return `NULL`.

Weak Reference Support

One of the goals of Python's weak reference implementation is to allow any type to participate in the weak reference mechanism without incurring the overhead on performance-critical objects (such as numbers).

Ver também

Documentação do módulo **`weakref`**.

For an object to be weakly referenceable, the extension type must set the `Py_TPFLAGS_MANAGED_WEAKREF` bit of the **`tp_flags`** field. The legacy **`tp_weaklistoffset`** field should be left as zero.

If this flag is set, **`Py_TPFLAGS_HAVE_GC`** should also be set.

Concretely, here is how the statically declared type object would look:

```
static PyTypeObject TrivialType = {
    PyVarObject_HEAD_INIT(NULL, 0)
    /* ... other members omitted for brevity ... */
    .tp_flags = Py_TPFLAGS_MANAGED_WEAKREF | ...,
};
```

The only further addition is that `tp_dealloc` needs to clear any weak references (by calling **`PyObject_ClearWeakRefs()`**):

```
static void
Trivial_dealloc(PyObject *op)
{
    /* Clear weakrefs first before calling any destructors */
    PyObject_ClearWeakRefs(op);
    /* ... remainder of destruction code omitted for brevity ...
    Py_TYPE(op)->tp_free(op);
}
```

More Suggestions

In order to learn how to implement any specific method for your new data type, get the CPython source code. Go to the `Objects` directory, then search the C source files for `tp_` plus the function you want (for example, `tp_richcompare`). You will find examples of the function you want to implement.

When you need to verify that an object is a concrete instance of the type you are implementing, use the **`PyObject_TypeCheck()`** function. A sample of its use might be something like the following:

```
if (!PyObject_TypeCheck(some_object, &MyType)) {
    PyErr_SetString(PyExc_TypeError, "arg #1 not a mything");
    return NULL;
}
```

Ver também

Download CPython source releases.
https://www.python.org/downloads/source/

The CPython project on GitHub, where the CPython source code is developed.
https://github.com/python/cpython

Construindo extensões C e C++

Uma extensão C para CPython é uma biblioteca compartilhada (por exemplo, um arquivo `.so` no Linux, `.pyd` no Windows), que exporta uma *função de inicialização*.

Veja Definindo módulos de extensão para detalhes.

Construindo extensões C e C ++ com setuptools

A construção, o empacotamento e a distribuição de módulos de extensão são melhor realizados com ferramentas de terceiros e estão fora do escopo deste documento. Uma ferramenta adequada é o Setuptools, cuja documentação pode ser encontrada em https://setuptools.pypa.io/en/latest/setuptools.html.

O módulo **`distutils`**, que estava incluído na biblioteca padrão até o Python 3.12, agora é mantido como parte do Setuptools.

Construindo Extensões C e C++ no Windows

Este capítulo explica brevemente como criar um módulo de extensão do Windows para Python usando o Microsoft Visual C++ e segue com informações mais detalhadas sobre como ele funciona. O material explicativo é útil para o programador do Windows aprender a construir extensões Python e o programador Unix interessado em produzir software que possa ser construído com sucesso no Unix e no Windows.

Os autores de módulos são encorajados a usar a abordagem distutils para construir módulos de extensão, em vez daquele descrito nesta seção. Você ainda precisará do compilador C que foi usado para construir o Python; normalmente o Microsoft Visual C++.

Nota

Este capítulo menciona vários nomes de arquivos que incluem um número de versão do Python codificado. Esses nomes de arquivos são representados com o número da versão mostrado como `XY`; na prática, `'X'` será o número da versão principal e `'Y'` será o número da versão secundária da versão do Python com a qual você está trabalhando. Por exemplo, se você estiver usando o Python 2.2.1, `XY` será `22`.

Uma abordagem de livro de receitas

Existem duas abordagens para construir módulos de extensão no Windows, assim como no Unix: use o pacote `setuptools` para controlar o processo de construção ou faça as coisas manualmente. A abordagem setuptools funciona bem para a maioria das extensões; documentação sobre o uso de `setuptools` para construir e empacotar módulos de extensão está disponível em Construindo extensões C e C ++ com setuptools. Se você

achar que realmente precisa fazer as coisas manualmente, pode ser instrutivo estudar o arquivo do projeto para o módulo de biblioteca padrão winsound [https://github.com/python/cpython/tree/3.15/PCbuild/winsound.vcxproj].

Diferenças entre o Unix e o Windows

O Unix e o Windows usam paradigmas completamente diferentes para o carregamento do código em tempo de execução. Antes de tentar construir um módulo que possa ser carregado dinamicamente, esteja ciente de como o seu sistema funciona.

No Unix, um arquivo de objeto compartilhado (`.so`) contém código a ser usado pelo programa e também os nomes de funções e dados que ele espera encontrar no programa. Quando o arquivo é associado ao programa, todas as referências a essas funções e dados no código do arquivo são alteradas para apontar para os locais reais no programa em que as funções e os dados são colocados na memória. Isso é basicamente uma operação de vinculação.

No Windows, um arquivo de biblioteca de vínculo dinâmico (`.dll`) não possui referências pendentes. Em vez disso, um acesso a funções ou dados passa por uma tabela de pesquisa. Portanto, o código DLL não precisa ser corrigido no tempo de execução para se referir à memória do programa; em vez disso, o código já usa a tabela de pesquisa da DLL e a tabela de pesquisa é modificada em tempo de execução para apontar para as funções e dados.

No Unix, existe apenas um tipo de arquivo de biblioteca (`.a`) que contém código de vários arquivos de objetos (`.o`). Durante a etapa da vinculação para criar um arquivo de objeto compartilhado (`.so`), o vinculador pode achar que não sabe onde um identificador está definido. O vinculador procurará nos arquivos de objeto nas bibliotecas; se encontrar, incluirá todo o código desse arquivo de objeto.

No Windows, existem dois tipos de biblioteca, uma biblioteca estática e uma biblioteca de importação (ambas chamadas `.lib`). Uma biblioteca estática é como um arquivo Unix `.a`; contém código a ser incluído conforme necessário. Uma biblioteca de importação é basicamente usada

apenas para garantir ao vinculador que um determinado identificador é legal e estará presente no programa quando a DLL for carregada. Portanto, o vinculador usa as informações da biblioteca de importação para construir a tabela de pesquisa para o uso de identificadores que não estão incluídos na DLL. Quando uma aplicação ou uma DLL é vinculado, pode ser gerada uma biblioteca de importação, que precisará ser usada para todas as DLLs futuras que dependem dos símbolos na aplicação ou DLL.

Suponha que você esteja construindo dois módulos de carregamento dinâmico, B e C, que devem compartilhar outro bloco de código A. No Unix, você *não* passaria `A.a` ao ligador para `B.so` e `C.so`; isso faria com que fosse incluído duas vezes, para que B e C tivessem sua própria cópia. No Windows, a construção `A.dll` também construirá `A.lib`. Você *passa* `A.lib` ao ligador para B e C. `A.lib` não contém código; apenas contém informações que serão usadas em tempo de execução para acessar o código de A.

No Windows, usar uma biblioteca de importação é como usar `import spam`; fornece acesso aos nomes de spam, mas não cria uma cópia separada. No Unix, vincular a uma biblioteca é mais como `from spam import *`; ele cria uma cópia separada.

Py_NO_LINK_LIB

Desativa a vinculação implícita baseada em `#pragma` com a biblioteca Python, realizada dentro dos arquivos de cabeçalho CPython.

Adicionado na versão 3.14.

Usando DLLs na prática

O Python para Windows é criado no Microsoft Visual C++; o uso de outros compiladores pode ou não funcionar. O restante desta seção é específico do MSVC++.

Ao criar DLLs no Windows, você pode usar a biblioteca CPython de duas maneiras:

1. Por padrão, a inclusão de `PC/pyconfig.h` diretamente ou via `Python.h` aciona um link implícito e sensível à configuração com a biblioteca. O arquivo de cabeçalho escolhe `pythonXY_d.lib` para Debug, `pythonXY.lib` para Release e `pythonX.lib` para Release com a API Limitada habilitada.

   Para construir duas DLLs, spam e ni (que usa funções C encontradas em spam), você pode usar estes comandos:

   ```
   cl /LD /I/python/include spam.c
   cl /LD /I/python/include ni.c spam.lib
   ```

   O primeiro comando criou três arquivos: `spam.obj`, `spam.dll` e `spam.lib`. O `Spam.dll` não contém nenhuma função Python (como **`PyArg_ParseTuple()`**), mas sabe como encontrar o código Python graças ao `pythonXY.lib` implicitamente ligado.

   O segundo comando criou `ni.dll` (e `.obj` e `.lib`), que sabe como encontrar as funções necessárias do spam e também do executável do Python.

2. Manualmente, definindo a macro **`Py_NO_LINK_LIB`** antes de incluir `Python.h`. Você deve passar `pythonXY.lib` para o ligador.

   Para construir duas DLLs, spam e ni (que usa funções C encontradas em spam), você pode usar estes comandos:

   ```
   cl /LD /DPy_NO_LINK_LIB /I/python/include spam.c ../libs/pyt
   cl /LD /DPy_NO_LINK_LIB /I/python/include ni.c spam.lib ../l
   ```

   O primeiro comando criou três arquivos: `spam.obj`, `spam.dll` e `spam.lib`. O `spam.dll` não contém nenhuma função Python (como **`PyArg_ParseTuple()`**), mas sabe como encontrar o código Python graças a `pythonXY.lib`.

   O segundo comando criou `ni.dll` (e `.obj` e `.lib`), que sabe como encontrar as funções necessárias do spam e também do executável do Python.

Nem todo identificador é exportado para a tabela de pesquisa. Se você deseja que outros módulos (incluindo Python) possam ver seus identificadores, é necessário dizer `_declspec(dllexport)`, como em `void _declspec(dllexport) initspam(void)` ou `PyObject _declspec(dllexport) *NiGetSpamData(void)`.

O Developer Studio incluirá muitas bibliotecas importadas que você realmente não precisa, adicionando cerca de 100K ao seu executável. Para se livrar delas, use a caixa de diálogo de configurações do projeto, na aba vincular, para especificar *ignorar bibliotecas padrão*. Adicione o `msvcrt`*`xx`*`.lib` correto à lista de bibliotecas.

Incorporando o Python numa Outra Aplicação

Os capítulos anteriores discutiram como estender o Python, ou seja, como expandir a funcionalidade do Python anexando uma biblioteca de funções em C a ele. Também é possível fazer o inverso: enriquecer sua aplicação em C/C++ incorporando o Python nela. A incorporação fornece à sua aplicação a capacidade de implementar parte da funcionalidade da aplicação em Python em vez de C ou C++. Isso pode ser usado para diversos propósitos; um exemplo seria permitir que os usuários personalizem a aplicação de acordo com suas necessidades escrevendo alguns scripts em Python. Você também pode usá-la se parte da funcionalidade puder ser escrita em Python mais facilmente.

Incorporar o Python é semelhante a estendê-lo, mas não exatamente. A diferença é que, ao estender o Python, o programa principal da aplicação ainda é o interpretador Python, enquanto que, se você incorporar o Python, o programa principal pode não ter nada a ver com o Python — em vez disso, algumas partes da aplicação chamam ocasionalmente o interpretador Python para executar algum código Python.

So if you are embedding Python, you are providing your own main program. One of the things this main program has to do is initialize the Python interpreter. At the very least, you have to call the function `Py_Initialize()`. There are optional calls to pass command line arguments to Python. Then later you can call the interpreter from any part of the application.

There are several different ways to call the interpreter: you can pass a string containing Python statements to `PyRun_SimpleString()`, or you can pass a stdio file pointer and a file name (for identification in error messages only) to `PyRun_SimpleFile()`. You can also call the lower-level operations described in the previous chapters to construct and use Python objects.

Ver também

Manual de referência da API C/Python
The details of Python's C interface are given in this manual. A great deal of necessary information can be found here.

Very High Level Embedding

The simplest form of embedding Python is the use of the very high level interface. This interface is intended to execute a Python script without needing to interact with the application directly. This can for example be used to perform some operation on a file.

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>

int
main(int argc, char *argv[])
{
    PyStatus status;
    PyConfig config;
    PyConfig_InitPythonConfig(&config);

    /* optional but recommended */
    status = PyConfig_SetBytesString(&config, &config.program_na
    if (PyStatus_Exception(status)) {
        goto exception;
    }

    status = Py_InitializeFromConfig(&config);
    if (PyStatus_Exception(status)) {
        goto exception;
    }
    PyConfig_Clear(&config);

    PyRun_SimpleString("from time import time,ctime\n"
                       "print('Today is', ctime(time()))\n");
    if (Py_FinalizeEx() < 0) {
        exit(120);
```

```
    }
    return 0;

  exception:
     PyConfig_Clear(&config);
     Py_ExitStatusException(status);
}
```

Nota

`#define PY_SSIZE_T_CLEAN` was used to indicate that `Py_ssize_t` should be used in some APIs instead of `int`. It is not necessary since Python 3.13, but we keep it here for backward compatibility. See Strings e buffers for a description of this macro.

Setting **`PyConfig.program_name`** should be called before **`Py_InitializeFromConfig()`** to inform the interpreter about paths to Python run-time libraries. Next, the Python interpreter is initialized with **`Py_Initialize()`**, followed by the execution of a hard-coded Python script that prints the date and time. Afterwards, the **`Py_FinalizeEx()`** call shuts the interpreter down, followed by the end of the program. In a real program, you may want to get the Python script from another source, perhaps a text-editor routine, a file, or a database. Getting the Python code from a file can better be done by using the **`PyRun_SimpleFile()`** function, which saves you the trouble of allocating memory space and loading the file contents.

Beyond Very High Level Embedding: An overview

The high level interface gives you the ability to execute arbitrary pieces of Python code from your application, but exchanging data values is quite cumbersome to say the least. If you want that, you should use lower level calls. At the cost of having to write more C code, you can achieve almost anything.

It should be noted that extending Python and embedding Python is quite the same activity, despite the different intent. Most topics discussed in the previous chapters are still valid. To show this, consider what the extension code from Python to C really does:

1. Convert data values from Python to C,
2. Perform a function call to a C routine using the converted values, and
3. Convert the data values from the call from C to Python.

When embedding Python, the interface code does:

1. Convert data values from C to Python,
2. Perform a function call to a Python interface routine using the converted values, and
3. Convert the data values from the call from Python to C.

As you can see, the data conversion steps are simply swapped to accommodate the different direction of the cross-language transfer. The only difference is the routine that you call between both data conversions. When extending, you call a C routine, when embedding, you call a Python routine.

This chapter will not discuss how to convert data from Python to C and vice versa. Also, proper use of references and dealing with errors is assumed to be understood. Since these aspects do not differ from extending the interpreter, you can refer to earlier chapters for the required information.

Pure Embedding

The first program aims to execute a function in a Python script. Like in the section about the very high level interface, the Python interpreter does not directly interact with the application (but that will change in the next section).

The code to run a function defined in a Python script is:

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
```

```
int
main(int argc, char *argv[])
{
    PyObject *pName, *pModule, *pFunc;
    PyObject *pArgs, *pValue;
    int i;

    if (argc < 3) {
        fprintf(stderr,"Usage: call pythonfile funcname [args]\r
        return 1;
    }

    Py_Initialize();
    pName = PyUnicode_DecodeFSDefault(argv[1]);
    /* Error checking of pName left out */

    pModule = PyImport_Import(pName);
    Py_DECREF(pName);

    if (pModule != NULL) {
        pFunc = PyObject_GetAttrString(pModule, argv[2]);
        /* pFunc is a new reference */

        if (pFunc && PyCallable_Check(pFunc)) {
            pArgs = PyTuple_New(argc - 3);
            for (i = 0; i < argc - 3; ++i) {
                pValue = PyLong_FromLong(atoi(argv[i + 3]));
                if (!pValue) {
                    Py_DECREF(pArgs);
                    Py_DECREF(pModule);
                    fprintf(stderr, "Cannot convert argument\n")
                    return 1;
                }
                /* pValue reference stolen here: */
                PyTuple_SetItem(pArgs, i, pValue);
            }
            pValue = PyObject_CallObject(pFunc, pArgs);
            Py_DECREF(pArgs);
            if (pValue != NULL) {
                printf("Result of call: %ld\n", PyLong_AsLong(pV
                Py_DECREF(pValue);
            }
```

```
            else {
                Py_DECREF(pFunc);
                Py_DECREF(pModule);
                PyErr_Print();
                fprintf(stderr,"Call failed\n");
                return 1;
            }
        }
        else {
            if (PyErr_Occurred())
                PyErr_Print();
            fprintf(stderr, "Cannot find function \"%s\"\n", arg
        }
        Py_XDECREF(pFunc);
        Py_DECREF(pModule);
    }
    else {
        PyErr_Print();
        fprintf(stderr, "Failed to load \"%s\"\n", argv[1]);
        return 1;
    }
    if (Py_FinalizeEx() < 0) {
        return 120;
    }
    return 0;
}
```

This code loads a Python script using `argv[1]`, and calls the function named in `argv[2]`. Its integer arguments are the other values of the `argv` array. If you compile and link this program (let's call the finished executable **call**), and use it to execute a Python script, such as:

```
def multiply(a,b):
    print("Will compute", a, "times", b)
    c = 0
    for i in range(0, a):
        c = c + b
    return c
```

then the result should be:

```
$ call multiply multiply 3 2
Will compute 3 times 2
Result of call: 6
```

Although the program is quite large for its functionality, most of the code is for data conversion between Python and C, and for error reporting. The interesting part with respect to embedding Python starts with

```
Py_Initialize();
pName = PyUnicode_DecodeFSDefault(argv[1]);
/* Error checking of pName left out */
pModule = PyImport_Import(pName);
```

After initializing the interpreter, the script is loaded using **`PyImport_Import()`**. This routine needs a Python string as its argument, which is constructed using the **`PyUnicode_DecodeFSDefault()`** data conversion routine.

```
pFunc = PyObject_GetAttrString(pModule, argv[2]);
/* pFunc is a new reference */

if (pFunc && PyCallable_Check(pFunc)) {
    ...
}
Py_XDECREF(pFunc);
```

Once the script is loaded, the name we're looking for is retrieved using **`PyObject_GetAttrString()`**. If the name exists, and the object returned is callable, you can safely assume that it is a function. The program then proceeds by constructing a tuple of arguments as normal. The call to the Python function is then made with:

```
pValue = PyObject_CallObject(pFunc, pArgs);
```

Upon return of the function, `pValue` is either `NULL` or it contains a reference to the return value of the function. Be sure to release the reference after examining the value.

Extending Embedded Python

Until now, the embedded Python interpreter had no access to functionality from the application itself. The Python API allows this by extending the embedded interpreter. That is, the embedded interpreter gets extended with routines provided by the application. While it sounds complex, it is not so bad. Simply forget for a while that the application starts the Python interpreter. Instead, consider the application to be a set of subroutines, and write some glue code that gives Python access to those routines, just like you would write a normal Python extension. For example:

```
static int numargs=0;

/* Return the number of arguments of the application command lir
static PyObject*
emb_numargs(PyObject *self, PyObject *args)
{
    if(!PyArg_ParseTuple(args, ":numargs"))
        return NULL;
    return PyLong_FromLong(numargs);
}

static PyMethodDef emb_module_methods[] = {
    {"numargs", emb_numargs, METH_VARARGS,
     "Return the number of arguments received by the process."},
    {NULL, NULL, 0, NULL}
};

static struct PyModuleDef emb_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "emb",
    .m_size = 0,
    .m_methods = emb_module_methods,
};

static PyObject*
PyInit_emb(void)
{
    return PyModuleDef_Init(&emb_module);
}
```

Insert the above code just above the **`main()`** function. Also, insert the following two statements before the call to **`Py_Initialize()`**:

```
numargs = argc;
PyImport_AppendInittab("emb", &PyInit_emb);
```

These two lines initialize the `numargs` variable, and make the **`emb.numargs()`** function accessible to the embedded Python interpreter. With these extensions, the Python script can do things like

```
import emb
print("Number of arguments", emb.numargs())
```

Em uma aplicação real, os métodos exporão uma API da aplicação para Python.

Embedding Python in C++

It is also possible to embed Python in a C++ program; precisely how this is done will depend on the details of the C++ system used; in general you will need to write the main program in C++, and use the C++ compiler to compile and link your program. There is no need to recompile Python itself using C++.

Compiling and Linking under Unix-like systems

It is not necessarily trivial to find the right flags to pass to your compiler (and linker) in order to embed the Python interpreter into your application, particularly because Python needs to load library modules implemented as C dynamic extensions (`.so` files) linked against it.

To find out the required compiler and linker flags, you can execute the `python`*`X.Y`*`-config` script which is generated as part of the installation process (a `python3-config` script may also be available). This script has several options, of which the following will be directly useful to you:

- `pythonX.Y-config --cflags` will give you the recommended flags when compiling:

  ```
  $ /opt/bin/python3.11-config --cflags
  -I/opt/include/python3.11 -I/opt/include/python3.11 -Wsign-c
  ```

- `pythonX.Y-config --ldflags --embed` will give you the recommended flags when linking:

  ```
  $ /opt/bin/python3.11-config --ldflags --embed
  -L/opt/lib/python3.11/config-3.11-x86_64-linux-gnu -L/opt/li
  ```

Nota

To avoid confusion between several Python installations (and especially between the system Python and your own compiled Python), it is recommended that you use the absolute path to `python*X.Y*-config`, as in the above example.

If this procedure doesn't work for you (it is not guaranteed to work for all Unix-like platforms; however, we welcome bug reports) you will have to read your system's documentation about dynamic linking and/or examine Python's `Makefile` (use **`sysconfig.get_makefile_filename()`** to find its location) and compilation options. In this case, the **`sysconfig`** module is a useful tool to programmatically extract the configuration values that you will want to combine together. For example:

```
>>> import sysconfig
>>> sysconfig.get_config_var('LIBS')
'-lpthread -ldl  -lutil'
>>> sysconfig.get_config_var('LINKFORSHARED')
'-Xlinker -export-dynamic'
```

Manual de referência da API C/Python

Este manual documenta a API usada por programadores C e C++ que desejam escrever módulos de extensões ou embutir Python. É um complemento para Estendendo e incorporando o interpretador Python, que descreve os princípios gerais da escrita de extensões mas não documenta as funções da API em detalhes.

- Introdução
  - Compatibilidade com a versão da linguagem
  - Padrões de codificação
  - Arquivos de inclusão
  - Macros úteis
  - Objetos, tipos e contagens de referências
  - Exceções
  - Incorporando Python
  - Compilações de depuração
  - Ferramentas de terceiros recomendadas
- C API and ABI Stability
  - API C Instável
  - Stable Application Binary Interfaces
  - Considerações da plataforma
  - ABI Checking
  - Conteúdo da API Limitada
- A camada de nível muito alto
  - Símbolos iniciais disponíveis
  - Stack Effects
- Contagem de referências
- Tratamento de exceções
  - Impressão e limpeza
  - Levantando exceções
  - Emitindo avisos
  - Consultando o indicador de erro

    - Tratamento de sinal
    - Classes de exceção
    - Objeto Exceção
    - Objetos de exceção Unicode
    - Controle de recursão
    - Exception and warning types
    - Tracebacks
- Definindo módulos de extensão
    - Extension export hook
    - Inicialização multifásica
    - Várias instâncias de módulo
    - `PyInit` function
- Definition slots
    - Convenience macros
    - Common slot IDs
- Utilitários
    - Utilitários do sistema operacional
    - Funções de sistema
    - Controle de processos
    - Importando módulos
    - Suporte a *marshalling* de dados
    - Análise de argumentos e construção de valores
    - Conversão e formação de strings
    - Classificação e conversão de caracteres
    - API do PyHash
    - Reflexão
    - Registro de codec e funções de suporte
    - API C PyTime
    - Suporte a Mapas do Perf
- Camada de Objetos Abstratos
    - Protocolo de objeto
    - Protocolo de chamada
    - Protocolo de número
    - Protocolo de sequência
    - Protocolo de mapeamento
    - Protocolo Iterador
    - Protocolo de buffer

- Camada de Objetos Concretos
  - Objetos Fundamentais
  - Objetos Numéricos
  - Objetos Sequência
  - Coleções
  - Objetos Função
  - Outros Objetos
  - API C para módulos de extensão
- Interpreter initialization and finalization
  - Before Python initialization
  - Variáveis de configuração global
  - Inicializando e encerrando o interpretador
  - Cautions regarding interpreter finalization
  - Interpreter views
  - Parâmetros de todo o processo
- Thread states and the global interpreter lock
  - Desvinculação do estado de thread do código de extensão
  - Using the C API from foreign threads
  - GIL-state APIs
  - Cuidados com o uso de fork()
  - APIs de alto nível
  - APIs de baixo nível
- Asynchronous notifications
- Operating system thread APIs
- Primitivas de sincronização
  - Python critical section API
  - Legacy locking APIs
- Thread-local storage support
  - Thread-specific storage API
  - Dynamic allocation
  - Métodos
  - Legacy APIs
- Multiple interpreters in a Python process
  - A per-interpreter GIL
  - Bugs and caveats
  - APIs de alto nível
  - APIs de baixo nível

    - Advanced debugger support
- Profiling and tracing
- Reference tracing
- Configuração de inicialização do Python
    - API C PyInitConfig
    - Opções de configuração
    - API de configuração Python em tempo de execução
    - API C PyConfig
    - Py_GetArgcArgv()
    - Multi-Phase Initialization Private Provisional API
- Gerenciamento de Memória
    - Visão Geral
    - Allocator Domains
    - Raw Memory Interface
    - Interface da Memória
    - Alocadores de objeto
    - Alocadores de memória padrão
    - Alocadores de memória
    - Debug hooks on the Python memory allocators
    - The pymalloc allocator
    - The mimalloc allocator
    - tracemalloc C API
    - Exemplos
- Suporte a implementação de Objetos
    - Alocando objetos na heap
    - Ciclo de vida do objeto
    - Estruturas comuns de objetos
    - Estruturas de objetos tipo
    - Suporte a Coleta Cíclica de Lixo
- API e Versionamento de ABI
    - Constantes de versão de tempo de construção
    - Versão de tempo de execução
    - Macros de empacotamento de bits
- API C de monitoramento
- Gerando eventos de execução
    - Gerenciando o Estado de um Monitoramento

Introdução

A Interface de Programação de Aplicações (API) para Python fornece aos programadores C e C++ acesso ao interpretador Python em uma variedade de níveis. A API pode ser usada igualmente em C++, mas, para abreviar, geralmente é chamada de API Python/C. Existem dois motivos fundamentalmente diferentes para usar a API Python/C. A primeira razão é escrever *módulos de extensão* para propósitos específicos; esses são módulos C que estendem o interpretador Python. Este é provavelmente o uso mais comum. O segundo motivo é usar Python como um componente em uma aplicação maior; esta técnica é geralmente referida como *incorporação* Python em uma aplicação.

Escrever um módulo de extensão é um processo relativamente bem compreendido, no qual uma abordagem de “livro de receitas” funciona bem. Existem várias ferramentas que automatizam o processo até certo ponto. Embora as pessoas tenham incorporado o Python em outras aplicações desde sua existência inicial, o processo de incorporação do Python é menos direto do que escrever uma extensão.

Muitas funções da API são úteis independentemente de você estar incorporando ou estendendo o Python; além disso, a maioria das aplicações que incorporam Python também precisará fornecer uma extensão customizada, portanto, é provavelmente uma boa ideia se familiarizar com a escrita de uma extensão antes de tentar incorporar Python em uma aplicação real.

Compatibilidade com a versão da linguagem

A API C do Python é compatível com as versões C11 e C++11 do C e C++.

Este é um limite inferior: a API C não requer recursos de versões posteriores de C/C++. Você *não* precisa habilitar o “modo c11” do seu compilador.

Padrões de codificação

Se você estiver escrevendo código C para inclusão no CPython, **deve** seguir as diretrizes e padrões definidos na **PEP 7** [https://peps.python.org/pep-0007/]. Essas diretrizes se aplicam independentemente da versão do Python com a qual você está contribuindo. Seguir essas convenções não é necessário para seus próprios módulos de extensão de terceiros, a menos que você eventualmente espere contribuí-los para o Python.

Arquivos de inclusão

Todas as definições de função, tipo e macro necessárias para usar a API Python/C estão incluídas em seu código pela seguinte linha:

```
#define PY_SSIZE_T_CLEAN
#include <Python.h>
```

Isso implica a inclusão dos seguintes cabeçalhos padrão: `<stdio.h>`, `<string.h>`, `<errno.h>`, `<limits.h>`, `<assert.h>` e `<stdlib.h>` (se disponível).

> **Nota**
>
> Uma vez que Python pode definir algumas definições de pré-processador que afetam os cabeçalhos padrão em alguns sistemas, você *deve* incluir `Python.h` antes de quaisquer cabeçalhos padrão serem incluídos.
>
> É recomendável sempre definir `PY_SSIZE_T_CLEAN` antes de incluir `Python.h`. Veja Análise de argumentos e construção de valores para uma descrição desta macro.

Todos os nomes visíveis ao usuário definidos por Python.h (exceto aqueles definidos pelos cabeçalhos padrão incluídos) têm um dos prefixos `Py` ou `_Py`. Nomes começando com `_Py` são para uso interno pela implementação Python e não devem ser usados por escritores de extensão. Os nomes dos membros da estrutura não têm um prefixo reservado.

Nota

O código do usuário nunca deve definir nomes que começam com `Py` ou `_Py`. Isso confunde o leitor e coloca em risco a portabilidade do código do usuário para versões futuras do Python, que podem definir nomes adicionais começando com um desses prefixos.

Os arquivos de cabeçalho são normalmente instalados com Python. No Unix, eles estão localizados nos diretórios *prefix*`/include/pythonversion/` e *exec_prefix*`/include/pythonversion/`, onde **prefix** e **exec_prefix** são definidos pelos parâmetros correspondentes ao script **configure** e *version* do Python é `'%d.%d' % sys.version_info[:2]`. No Windows, os cabeçalhos são instalados em *prefix*`/include`, onde `prefix` é o diretório de instalação especificado para o instalador.

Para incluir os cabeçalhos, coloque os dois diretórios (se diferentes) no caminho de pesquisa do compilador para as inclusões. *Não* coloque os diretórios pais no caminho de busca e então use `#include <pythonX.Y/Python.h>`; isto irá quebrar em compilações multiplataforma, uma vez que os cabeçalhos independentes da plataforma em **prefix** incluem os cabeçalhos específicos da plataforma de **exec_prefix**.

Os usuários de C++ devem notar que embora a API seja definida inteiramente usando C, os arquivos de cabeçalho declaram apropriadamente os pontos de entrada como `extern "C"`. Como resultado, não há necessidade de fazer nada especial para usar a API do C++.

System includes

`Python.h` includes several standard header files. C extensions should include the standard headers that they use, and should not rely on these implicit includes. The implicit includes are:

- `<assert.h>`
- `<intrin.h>` (on Windows)

- `<inttypes.h>`
- `<limits.h>`
- `<math.h>`
- `<stdarg.h>`
- `<string.h>`
- `<wchar.h>`
- `<sys/types.h>` (if present)

The following are included for backwards compatibility, unless using Limited API 3.13 or newer:

- `<ctype.h>`
- `<unistd.h>` (on POSIX)

The following are included for backwards compatibility, unless using Limited API 3.11 or newer:

- `<errno.h>`
- `<stdio.h>`
- `<stdlib.h>`

Nota

Uma vez que Python pode definir algumas definições de pré-processador que afetam os cabeçalhos padrão em alguns sistemas, você *deve* incluir `Python.h` antes de quaisquer cabeçalhos padrão serem incluídos.

Macros úteis

Diversas macros úteis são definidas nos arquivos de cabeçalho do Python. Muitas são definidas mais próximas de onde são úteis (por exemplo, **`Py_RETURN_NONE`**, **`PyMODINIT_FUNC`**). Outras de utilidade mais geral são definidas aqui. Esta não é necessariamente uma lista completa.

Py_CAN_START_THREADS

If this macro is defined, then the current system is able to start threads.

Currently, all systems supported by CPython (per **PEP 11** [https://peps.python.org/pep-0011/]), with the exception of some WebAssembly platforms, support starting threads.

Adicionado na versão 3.13.

Py_GETENV(s)

Like `getenv(s)`, but returns `NULL` if `-E` was passed on the command line (see `PyConfig.use_environment`).

Docstring macros

PyDoc_STRVAR(name, str)

Creates a variable with name *name* that can be used in docstrings. If Python is built without docstrings (`--without-doc-strings`), the value will be an empty string.

Exemplo:

```
PyDoc_STRVAR(pop_doc, "Remove e retorna o elemento mais à dir

static PyMethodDef deque_methods[] = {
    // ...
    {"pop", (PyCFunction)deque_pop, METH_NOARGS, pop_doc},
    // ...
}
```

Expands to `PyDoc_VAR(name) = PyDoc_STR(str)`.

PyDoc_STR(str)

Expands to the given input string, or an empty string if docstrings are disabled (`--without-doc-strings`).

Exemplo:

```
static PyMethodDef pysqlite_row_methods[] = {
    {"keys", (PyCFunction)pysqlite_row_keys, METH_NOARGS,
        PyDoc_STR("Retorna as chaves da linha.")},
```

```
    {NULL, NULL}
};
```

PyDoc_VAR(name)

Declares a static character array variable with the given *name*. Expands to `static const char name[]`

Por exemplo:

```
PyDoc_VAR(python_doc) = PyDoc_STR(
   "A genus of constricting snakes in the Pythonidae family r
   "to the tropics and subtropics of the Eastern Hemisphere.'
```

General utility macros

The following macros are for common tasks not specific to Python.

Py_UNUSED(arg)

Use isso para argumentos não usados em uma definição de função para silenciar avisos do compilador. Exemplo: `int func(int a, int Py_UNUSED(b)) { return a; }`.

Adicionado na versão 3.4.

Py_GCC_ATTRIBUTE(name)

Use a GCC attribute *name*, hiding it from compilers that don't support GCC attributes (such as MSVC).

This expands to `__attribute__((name))` on a GCC compiler, and expands to nothing on compilers that don't support GCC attributes.

Numeric utilities

Py_ABS(x)

Retorna o valor absoluto de `x`.

The argument may be evaluated more than once. Consequently, do not pass an expression with side-effects directly to this macro.

Se o resultado não puder ser representado (por exemplo, se `x` tiver o valor **`INT_MIN`** para o tipo int), o comportamento é indefinido.

Corresponds roughly to `((x) < 0 ? -(x) : (x))`

Adicionado na versão 3.3.

Py_MAX(x, y)
Py_MIN(x, y)

Return the larger or smaller of the arguments, respectively.

Any arguments may be evaluated more than once. Consequently, do not pass an expression with side-effects directly to this macro.

`Py_MAX` corresponds roughly to `(((x) > (y)) ? (x) : (y))`.

Adicionado na versão 3.3.

Py_ARITHMETIC_RIGHT_SHIFT(type, integer, positions)

Similar to *`integer >> positions`*, but forces sign extension, as the C standard does not define whether a right-shift of a signed integer will perform sign extension or a zero-fill.

integer should be any signed integer type. *positions* is the number of positions to shift to the right.

Both *integer* and *positions* can be evaluated more than once; consequently, avoid directly passing a function call or some other operation with side-effects to this macro. Instead, store the result as a variable and then pass it.

type is unused and only kept for backwards compatibility. Historically, *type* was used to cast *integer*.

Alterado na versão 3.1: This macro is now valid for all signed integer types, not just those for which `unsigned type` is legal. As a result, *type* is no longer used.

Py_CHARMASK(c)

O argumento deve ser um caractere ou um número inteiro no intervalo [-128, 127] ou [0, 255]. Esta macro retorna `c` convertido em um `unsigned char`.

Assertion utilities

Py_UNREACHABLE()

Use isso quando você tiver um caminho de código que não pode ser alcançado por design. Por exemplo, na cláusula `default:` em uma instrução `switch` para a qual todos os valores possíveis são incluídos nas instruções `case`. Use isto em lugares onde você pode ficar tentado a colocar uma chamada `assert(0)` ou `abort()`.

No modo de lançamento, a macro ajuda o compilador a otimizar o código e evita um aviso sobre código inacessível. Por exemplo, a macro é implementada com `__builtin_unreachable()` no GCC em modo de lançamento.

In debug mode, and on unsupported compilers, the macro expands to a call to **`Py_FatalError()`**.

A use for `Py_UNREACHABLE()` is following a call to a function that never returns but that is not declared `_Noreturn`.

Se um caminho de código for um código muito improvável, mas puder ser alcançado em casos excepcionais, esta macro não deve ser usada. Por exemplo, sob condição de pouca memória ou se uma chamada de sistema retornar um valor fora do intervalo esperado. Nesse caso, é melhor relatar o erro ao chamador. Se o erro não puder ser reportado ao chamador, **`Py_FatalError()`** pode ser usada.

Adicionado na versão 3.7.

Py_SAFE_DOWNCAST(value, larger, smaller)

Cast *value* to type *smaller* from type *larger*, validating that no information was lost.

On release builds of Python, this is roughly equivalent to `((smaller) value)` (in C++, `static_cast<smaller>(value)` will be used instead).

On debug builds (implying that **`Py_DEBUG`** is defined), this asserts that no information was lost with the cast from *larger* to *smaller*.

value, *larger*, and *smaller* may all be evaluated more than once in the expression; consequently, do not pass an expression with side-effects directly to this macro.

Py_BUILD_ASSERT(cond)

Realiza asserção de uma condição de tempo de compilação *cond*, como uma instrução. A construção falhará se a condição for falsa ou não puder ser avaliada em tempo de compilação.

Corresponds roughly to `static_assert(cond)` on C23 and above.

Por exemplo:

```
Py_BUILD_ASSERT(sizeof(PyTime_t) == sizeof(int64_t));
```

Adicionado na versão 3.3.

Py_BUILD_ASSERT_EXPR(cond)

Realiza asserção de uma condição de tempo de compilação *cond*, como uma expressão que é avaliada com `0`. A construção falhará se a condição for falsa ou não puder ser avaliada em tempo de compilação.

Por exemplo:

```
#define foo_to_char(foo) \
    ((char *)(foo) + Py_BUILD_ASSERT_EXPR(offsetof(struct foo
```

Adicionado na versão 3.3.

Type size utilities

Py_ARRAY_LENGTH(array)

Calcula o comprimento de um vetor C alocado estaticamente em tempo de compilação.

O argumento *array* deve ser um vetor C com tamanho conhecido em tempo de compilação. Passar um vetor com tamanho desconhecido, como um vetor alocado na heap, resultará em um erro de compilação em alguns compiladores ou produzirá resultados incorretos.

Isso é aproximadamente equivalente a:

```
sizeof(array) / sizeof((array)[0])
```

Py_MEMBER_SIZE(type, member)

Return the size of a structure (*type*) *member* in bytes.

Corresponds roughly to `sizeof(((type *)NULL)->member)`.

Adicionado na versão 3.6.

Macro definition utilities

Py_FORCE_EXPANSION(X)

This is equivalent to *X*, which is useful for token-pasting in macros, as macro expansions in *X* are forcefully evaluated by the preprocessor.

Py_STRINGIFY(x)

Convert `x` to a C string. For example, `Py_STRINGIFY(123)` returns `"123"`.

Adicionado na versão 3.4.

Declaration utilities

The following macros can be used in declarations. They are most useful for defining the C API itself, and have limited use for extension authors. Most of them expand to compiler-specific spellings of common extensions to the C language.

Py_ALWAYS_INLINE

Pede ao compilador para sempre embutir uma função em linha estática. O compilador pode ignorá-lo e decide não inserir a função.

Corresponds to `always_inline` attribute in GCC and `__forceinline` in MSVC.

Ele pode ser usado para inserir funções em linha estáticas críticas de desempenho ao compilar Python no modo de depuração com função de inserir em linha desabilitada. Por exemplo, o MSC desabilita a função de inserir em linha ao compilar no modo de depuração.

Marcar cegamente uma função em linha estática com Py_ALWAYS_INLINE pode resultar em desempenhos piores (devido ao aumento do tamanho do código, por exemplo). O compilador geralmente é mais inteligente que o desenvolvedor para a análise de custo/benefício.

Se o Python tiver sido compilado em modo de depuração (se a macro **`Py_DEBUG`** estiver definida), a macro **`Py_ALWAYS_INLINE`** não fará nada.

Deve ser especificado antes do tipo de retorno da função. Uso:

```
static inline Py_ALWAYS_INLINE int random(void) { return 4; }
```

Adicionado na versão 3.11.

Py_NO_INLINE

Desabilita a inserção em linha em uma função. Por exemplo, isso reduz o consumo da pilha C: útil em compilações LTO+PGO que faz uso intenso de inserção em linha de código (veja bpo-33720 [https://bugs.python.org/issue?@action=redirect&bpo=33720]).

Corresponds to the `noinline` attribute/specification on GCC and MSVC.

Uso:

```
Py_NO_INLINE static int random(void) { return 4; }
```

Adicionado na versão 3.11.

Py_DEPRECATED(version)

Use this to declare APIs that were deprecated in a specific CPython version. The macro must be placed before the symbol name.

Exemplo:

```
Py_DEPRECATED(3.8) PyAPI_FUNC(int) Py_OldFunction(void);
```

Alterado na versão 3.8: Suporte a MSVC foi adicionado.

Py_LOCAL(type)

Declare a function returning the specified *type* using a fast-calling qualifier for functions that are local to the current file. Semantically, this is equivalent to `static type`.

Py_LOCAL_INLINE(type)

Equivalente a **`Py_LOCAL`**, mas que também exige que a função seja inserida localmente.

Py_LOCAL_SYMBOL

Macro used to declare a symbol as local to the shared library (hidden). On supported platforms, it ensures the symbol is not exported.

On compatible versions of GCC/Clang, it expands to `__attribute__((visibility("hidden")))`.

Py_EXPORTED_SYMBOL

Macro used to declare a symbol (function or data) as exported. On Windows, this expands to `__declspec(dllexport)`. On compatible

versions of GCC/Clang, it expands to `__attribute__((visibility("default")))`. This macro is for defining the C API itself; extension modules should not use it.

Py_IMPORTED_SYMBOL

Macro used to declare a symbol as imported. On Windows, this expands to `__declspec(dllimport)`. This macro is for defining the C API itself; extension modules should not use it.

PyAPI_FUNC(type)

Macro used by CPython to declare a function as part of the C API. Its expansion depends on the platform and build configuration. This macro is intended for defining CPython's C API itself; extension modules should not use it for their own symbols.

PyAPI_DATA(type)

Macro used by CPython to declare a public global variable as part of the C API. Its expansion depends on the platform and build configuration. This macro is intended for defining CPython's C API itself; extension modules should not use it for their own symbols.

Outdated macros

The following soft deprecated macros have been used to features that have been standardized in C11 (or previous standards).

Py_ALIGNED(num)

On some GCC-like compilers, specify alignment to *num* bytes. This does nothing on other compilers.

Use the standard `alignas` specifier rather than this macro.

Suavemente descontinuado desde a versão 3.15.

PY_FORMAT_SIZE_T

The **`printf()`** formatting modifier for **`size_t`**. Use `"z"` directly instead.

Suavemente descontinuado desde a versão 3.15.

Py_LL(number)
Py_ULL(number)

Use *number* as a `long long` or `unsigned long long` integer literal, respectively.

Expands to *number* followed by `LL` or `LLU`, respectively, but will expand to some compiler-specific suffixes on some older compilers.

Consider using the C99 standard suffixes `LL` and `LLU` directly.

Suavemente descontinuado desde a versão 3.15.

PY_LONG_LONG
PY_INT32_T
PY_UINT32_T
PY_INT64_T
PY_UINT64_T

Aliases for the types **`long long`**, **`int32_t`**, **`uint32_t`**. **`int64_t`** and **`uint64_t`**, respectively. Historically, these types needed compiler-specific extensions.

Suavemente descontinuado desde a versão 3.15.

PY_LLONG_MIN
PY_LLONG_MAX
PY_ULLONG_MAX
PY_SIZE_MAX

Aliases for the values **`LLONG_MIN`**, **`LLONG_MAX`**, **`ULLONG_MAX`**, and **`SIZE_MAX`**, respectively. Use these standard names instead.

The required header, `<limits.h>`, is included in `Python.h`.

Suavemente descontinuado desde a versão 3.15.

Py_MEMCPY(dest, src, n)

This is an alias to **memcpy()**.

Suavemente descontinuado desde a versão 3.14: Use **memcpy()** directly instead.

Py_UNICODE_SIZE

Size of the **wchar_t** type. Use `sizeof(wchar_t)` or `WCHAR_WIDTH/8` instead.

The required header for the latter, `<limits.h>`, is included in `Python.h`.

Suavemente descontinuado desde a versão 3.15.

Py_UNICODE_WIDE

Defined if `wchar_t` can hold a Unicode character (UCS-4). Use `sizeof(wchar_t) >= 4` instead

Suavemente descontinuado desde a versão 3.15.

Py_VA_COPY

This is an alias to the C99-standard `va_copy` function.

Historically, this would use a compiler-specific method to copy a `va_list`.

Alterado na versão 3.6: This is now an alias to `va_copy`.

Suavemente descontinuado desde a versão 3.15.

Objetos, tipos e contagens de referências

A maioria das funções da API Python/C tem um ou mais argumentos, bem como um valor de retorno do tipo PyObject*. Este tipo é um ponteiro para um tipo de dados opaco que representa um objeto Python arbitrário. Como todos os tipos de objeto Python são tratados da mesma maneira pela linguagem Python na maioria das situações (por exemplo, atribuições, regras de escopo e passagem de argumento), é adequado que eles sejam

representados por um único tipo C. Quase todos os objetos Python vivem na pilha: você nunca declara uma variável automática ou estática do tipo **`PyObject`**, variáveis de apenas ponteiro do tipo PyObject* podem ser declaradas. A única exceção são os objetos de tipo; uma vez que estes nunca devem ser desalocados, eles são normalmente objetos estáticos **`PyTypeObject`**.

Todos os objetos Python (mesmo inteiros Python) têm um *tipo* e uma *contagem de referências*. O tipo de um objeto determina que tipo de objeto ele é (por exemplo, um número inteiro, uma lista ou uma função definida pelo usuário; existem muitos mais, conforme explicado em A hierarquia de tipos padrão). Para cada um dos tipos conhecidos, há uma macro para verificar se um objeto é desse tipo; por exemplo, `PyList_Check(a)` é verdadeiro se (e somente se) o objeto apontado por *a* for uma lista Python.

Contagens de referências

A contagem de referências é importante porque os computadores atuais têm um tamanho de memória finito (e frequentemente severamente limitado); ela conta quantos lugares diferentes existem que possuem uma referência forte a um objeto. Tal lugar pode ser outro objeto, ou uma variável C global (ou estática), ou uma variável local em alguma função C. Quando a última referência forte a um objeto é liberada (ou seja, sua contagem de referências se torna zero), o objeto é desalocado. Se ele contiver referências a outros objetos, essas referências são liberadas. Esses outros objetos podem ser desalocados por sua vez, se não houver mais referências a eles, e assim por diante. (Há um problema óbvio com objetos que referenciam uns aos outros aqui; por enquanto, a solução é “não faça isso”.)

Contagens de referências são sempre manipuladas explicitamente. A maneira normal é usar a macro **`Py_INCREF()`** para obter uma nova referência a um objeto (ou seja, incrementar sua contagem de referências em um), e **`Py_DECREF()`** para liberar essa referência (ou seja, decrementar a contagem de referências em um). A macro **`Py_DECREF()`** é consideravelmente mais complexa do que a incref, pois deve verificar se a contagem de referências se torna zero e, em seguida, fazer com que o desalocador do objeto seja chamado. O desalocador é um ponteiro para

função contido na estrutura de tipo do objeto. O desalocador específico do tipo cuida da liberação de referências para outros objetos contidos no objeto se este for um tipo de objeto composto, como uma lista, bem como de executar qualquer finalização adicional necessária. Não há chance de a contagem de referências transbordar; pelo menos tantos bits são usados para armazenar a contagem de referência quantos forem os locais de memória distintos na memória virtual (presumindo `sizeof(Py_ssize_t) >= sizeof(void*)`). Portanto, o incremento da contagem de referências é uma operação simples.

Não é necessário manter uma referência forte (ou seja, incrementar a contagem de referências) para cada variável local que contém um ponteiro para um objeto. Em teoria, a contagem de referências do objeto aumenta em um quando a variável é feita para apontar para ele e diminui em um quando a variável sai do escopo. No entanto, esses dois se cancelam, portanto, no final, a contagem de referências não mudou. A única razão real para usar a contagem de referências é evitar que o objeto seja desalocado enquanto nossa variável estiver apontando para ele. Se sabemos que existe pelo menos uma outra referência ao objeto que vive pelo menos tanto quanto nossa variável, não há necessidade de tomar uma nova referência forte (ou seja, incrementar a contagem de referências) temporariamente. Uma situação importante em que isso ocorre é em objetos que são passados como argumentos para funções C em um módulo de extensão que são chamados de Python; o mecanismo de chamada garante manter uma referência a todos os argumentos durante a chamada.

No entanto, uma armadilha comum é extrair um objeto de uma lista e mantê-lo por um tempo, sem tomar uma nova referência. Alguma outra operação poderia remover o objeto da lista, liberando essa referência e possivelmente desalocando-o. O perigo real é que operações aparentemente inocentes podem invocar código Python arbitrário que poderia fazer isso; existe um caminho de código que permite que o controle flua de volta para o usuário a partir de um **`Py_DECREF()`**, então quase qualquer operação é potencialmente perigosa.

Uma abordagem segura é sempre usar as operações genéricas (funções cujo nome começa com `PyObject_`, `PyNumber_`, `PySequence_` ou `PyMapping_`).

Essas operações sempre criam uma nova referência forte (ou seja, incrementam a contagem de referências) do objeto que retornam. Isso deixa o chamador com a responsabilidade de chamar `Py_DECREF()` quando terminar com o resultado; isso logo se torna uma segunda natureza.

Detalhes da contagem de referências

O comportamento da contagem de referências de funções na API C/Python é melhor explicado em termos de *propriedade de referências*. A propriedade pertence às referências, nunca aos objetos (os objetos não são possuídos: eles são sempre compartilhados). "Possuir uma referência" significa ser responsável por chamar Py_DECREF nela quando a referência não for mais necessária. A propriedade também pode ser transferida, o que significa que o código que recebe a propriedade da referência torna-se responsável por eventualmente efetuar um liberando ela chamando `Py_DECREF()` ou `Py_XDECREF()` quando não é mais necessário – ou passando essa responsabilidade (geralmente para o responsável pela chamada). Quando uma função passa a propriedade de uma referência para seu chamador, diz-se que o chamador recebe uma *nova* referência. Quando nenhuma propriedade é transferida, diz-se que o chamador *toma emprestado* a referência. Nada precisa ser feito para uma referência emprestada.

Conversely, when a calling function passes in a reference to an object, there are two possibilities: the function *steals* a reference to the object, or it does not.

Stealing a reference means that when you pass a reference to a function, that function assumes that it now owns that reference. Since the new owner can use `Py_DECREF()` at its discretion, you (the caller) must not use that reference after the call.

Poucas funções roubam referências; as duas exceções notáveis são `PyList_SetItem()` e `PyTuple_SetItem()`, que roubam uma referência para o item (mas não para a tupla ou lista na qual o item é colocado!). Essas funções foram projetadas para roubar uma referência devido a um idioma comum para preencher uma tupla ou lista com objetos recém-criados; por

exemplo, o código para criar a tupla `(1, 2, "three")` pode ser parecido com isto (esquecendo o tratamento de erros por enquanto; uma maneira melhor de codificar isso é mostrada abaixo):

```
PyObject *t;

t = PyTuple_New(3);
PyTuple_SetItem(t, 0, PyLong_FromLong(1L));
PyTuple_SetItem(t, 1, PyLong_FromLong(2L));
PyTuple_SetItem(t, 2, PyUnicode_FromString("três"));
```

Aqui, **`PyLong_FromLong()`** retorna uma nova referência que é imediatamente roubada por **`PyTuple_SetItem()`**. Quando você quiser continuar usando um objeto, embora a referência a ele seja roubada, use **`Py_INCREF()`** para obter outra referência antes de chamar a função de roubo de referência.

A propósito, **`PyTuple_SetItem()`** é a *única* maneira de definir itens de tupla; **`PySequence_SetItem()`** e **`PyObject_SetItem()`** se recusam a fazer isso, pois tuplas são um tipo de dados imutável. Você só deve usar **`PyTuple_SetItem()`** para tuplas que você mesmo está criando.

O código equivalente para preencher uma lista pode ser escrita usando **`PyList_New()`** e **`PyList_SetItem()`**.

No entanto, na prática, você raramente usará essas maneiras de criar e preencher uma tupla ou lista. Existe uma função genérica, **`Py_BuildValue()`**, que pode criar objetos mais comuns a partir de valores C, dirigidos por uma *string de formato*. Por exemplo, os dois blocos de código acima podem ser substituídos pelos seguintes (que também cuidam da verificação de erros):

```
PyObject *tuple, *list;

tuple = Py_BuildValue("(iis)", 1, 2, "três");
list = Py_BuildValue("[iis]", 1, 2, "três");
```

É muito mais comum usar **`PyObject_SetItem()`** e amigos com itens cujas referências você está apenas pegando emprestado, como argumentos que

foram passados para a função que você está escrevendo. Nesse caso, o comportamento deles em relação às referências é muito mais são, já que você não precisa tomar uma nova referência só para poder doá-la ("mande-a ser roubada"). Por exemplo, esta função define todos os itens de uma lista (na verdade, qualquer sequência mutável) para um determinado item:

```
int
set_all(PyObject *target, PyObject *item)
{
    Py_ssize_t i, n;

    n = PyObject_Length(target);
    if (n < 0)
        return -1;
    for (i = 0; i < n; i++) {
        PyObject *index = PyLong_FromSsize_t(i);
        if (!index)
            return -1;
        if (PyObject_SetItem(target, index, item) < 0) {
            Py_DECREF(index);
            return -1;
        }
        Py_DECREF(index);
    }
    return 0;
}
```

A situação é ligeiramente diferente para os valores de retorno da função. Embora passar uma referência para a maioria das funções não altere suas responsabilidades de propriedade para aquela referência, muitas funções que retornam uma referência a um objeto fornecem a propriedade da referência. O motivo é simples: em muitos casos, o objeto retornado é criado instantaneamente e a referência que você obtém é a única referência ao objeto. Portanto, as funções genéricas que retornam referências a objetos, como **`PyObject_GetItem()`** e **`PySequence_GetItem()`**, sempre retornam uma nova referência (o chamador torna-se o dono da referência).

É importante perceber que se você possui uma referência retornada por uma função depende de qual função você chama apenas — *a plumagem* (o tipo do objeto passado como um argumento para a função) *não entra nela!*

Assim, se você extrair um item de uma lista usando **PyList_GetItem()**, você não possui a referência — mas se obtiver o mesmo item da mesma lista usando **PySequence_GetItem()** (que leva exatamente os mesmos argumentos), você possui uma referência ao objeto retornado.

Aqui está um exemplo de como você poderia escrever uma função que calcula a soma dos itens em uma lista de inteiros; uma vez usando **PyList_GetItem()**, e uma vez usando **PySequence_GetItem()**.

```
long
sum_list(PyObject *list)
{
    Py_ssize_t i, n;
    long total = 0, value;
    PyObject *item;

    n = PyList_Size(list);
    if (n < 0)
        return -1; /* Não é uma lista */
    for (i = 0; i < n; i++) {
        item = PyList_GetItem(list, i); /* Não pode falhar */
        if (!PyLong_Check(item)) continue; /* Ignora não-inteirc
        value = PyLong_AsLong(item);
        if (value == -1 && PyErr_Occurred())
            /* Inteiro muito grande para caber em um C longo, sa
            return -1;
        total += value;
    }
    return total;
}
```

```
long
sum_sequence(PyObject *sequence)
{
    Py_ssize_t i, n;
    long total = 0, value;
    PyObject *item;
    n = PySequence_Length(sequence);
    if (n < 0)
        return -1; /* Não tem comprimento */
    for (i = 0; i < n; i++) {
```

```
        item = PySequence_GetItem(sequence, i);
        if (item == NULL)
            return -1; /* Não é uma sequência ou outra falha */
        if (PyLong_Check(item)) {
            value = PyLong_AsLong(item);
            Py_DECREF(item);
            if (value == -1 && PyErr_Occurred())
                /* Inteiro muito grande para caber em um C longo
                return -1;
            total += value;
        }
        else {
            Py_DECREF(item); /* Descartar propriedade de referên
        }
    }
    return total;
}
```

Tipos

Existem alguns outros tipos de dados que desempenham um papel significativo na API Python/C; a maioria são tipos C simples, como int, long, double e char*. Alguns tipos de estrutura são usados para descrever tabelas estáticas usadas para listar as funções exportadas por um módulo ou os atributos de dados de um novo tipo de objeto, e outro é usado para descrever o valor de um número complexo. Eles serão discutidos junto com as funções que os utilizam.

type Py_ssize_t

Parte da ABI Estável.

Um tipo integral assinado tal que `sizeof(Py_ssize_t) == sizeof(size_t)`. C99 não define tal coisa diretamente (size_t é um tipo integral não assinado). Veja **PEP 353** [https://peps.python.org/pep-0353/] para mais detalhes. `PY_SSIZE_T_MAX` é o maior valor positivo do tipo `Py_ssize_t`.

Exceções

O programador Python só precisa lidar com exceções se o tratamento de erros específico for necessário; as exceções não tratadas são propagadas automaticamente para o chamador, depois para o chamador e assim por diante, até chegarem ao interpretador de nível superior, onde são relatadas ao usuário acompanhadas por um traceback (situação da pilha de execução).

Para programadores C, entretanto, a verificação de erros sempre deve ser explícita. Todas as funções na API Python/C podem levantar exceções, a menos que uma declaração explícita seja feita de outra forma na documentação de uma função. Em geral, quando uma função encontra um erro, ela define uma exceção, descarta todas as referências de objeto de sua propriedade e retorna um indicador de erro. Se não for documentado de outra forma, este indicador é `NULL` ou `-1`, dependendo do tipo de retorno da função. Algumas funções retornam um resultado booleano verdadeiro/falso, com falso indicando um erro. Muito poucas funções não retornam nenhum indicador de erro explícito ou têm um valor de retorno ambíguo e requerem teste explícito para erros com `PyErr_Occurred()`. Essas exceções são sempre documentadas explicitamente.

O estado de exceção é mantido no armazenamento por thread (isso é equivalente a usar o armazenamento global em uma aplicação sem thread). Uma thread pode estar em um de dois estados: ocorreu uma exceção ou não. A função `PyErr_Occurred()` pode ser usada para verificar isso: ela retorna uma referência emprestada ao objeto do tipo de exceção quando uma exceção ocorreu, e `NULL` caso contrário. Existem várias funções para definir o estado de exceção: `PyErr_SetString()` é a função mais comum (embora não a mais geral) para definir o estado de exceção, e `PyErr_Clear()` limpa o estado da exceção.

O estado de exceção completo consiste em três objetos (todos os quais podem ser `NULL`): o tipo de exceção, o valor de exceção correspondente e o traceback. Eles têm os mesmos significados que o resultado no Python de `sys.exc_info()`; no entanto, eles não são os mesmos: os objetos Python representam a última exceção sendo tratada por uma instrução Python `try` … `except`, enquanto o estado de exceção de nível C só existe enquanto uma exceção está sendo transmitido entre funções C até atingir o laço de

repetição principal do interpretador de bytecode Python, que se encarrega de transferi-lo para `sys.exc_info()` e amigos.

Observe que a partir do Python 1.5, a maneira preferida e segura para thread para acessar o estado de exceção do código Python é chamar a função **`sys.exc_info()`**, que retorna o estado de exceção por thread para o código Python. Além disso, a semântica de ambas as maneiras de acessar o estado de exceção mudou, de modo que uma função que captura uma exceção salvará e restaurará o estado de exceção de seu segmento de modo a preservar o estado de exceção de seu chamador. Isso evita bugs comuns no código de tratamento de exceções causados por uma função aparentemente inocente sobrescrevendo a exceção sendo tratada; também reduz a extensão da vida útil frequentemente indesejada para objetos que são referenciados pelos quadros de pilha no traceback.

Como princípio geral, uma função que chama outra função para realizar alguma tarefa deve verificar se a função chamada levantou uma exceção e, em caso afirmativo, passar o estado da exceção para seu chamador. Ele deve descartar todas as referências de objeto que possui e retornar um indicador de erro, mas *não* deve definir outra exceção — que sobrescreveria a exceção que acabou de ser gerada e perderia informações importantes sobre a causa exata do erro.

Um exemplo simples de detecção de exceções e transmiti-las é mostrado no exemplo **`sum_sequence()`** acima. Acontece que este exemplo não precisa limpar nenhuma referência de propriedade quando detecta um erro. A função de exemplo a seguir mostra alguma limpeza de erro. Primeiro, para lembrar por que você gosta de Python, mostramos o código Python equivalente:

```
def incr_item(dict, key):
    try:
        item = dict[key]
    except KeyError:
        item = 0
    dict[key] = item + 1
```

Aqui está o código C correspondente, em toda sua glória:

```
int
incr_item(PyObject *dict, PyObject *key)
{
    /* Objetos todos inicializados para NULL para Py_XDECREF */
    PyObject *item = NULL, *const_one = NULL, *incremented_item
    int rv = -1; /* Retorna valor inicializado para -1 (falha) *

    item = PyObject_GetItem(dict, key);
    if (item == NULL) {
        /* Trata de KeyError apenas: */
        if (!PyErr_ExceptionMatches(PyExc_KeyError))
            goto error;

        /* Limpa o erro e usa zero: */
        PyErr_Clear();
        item = PyLong_FromLong(0L);
        if (item == NULL)
            goto error;
    }
    const_one = PyLong_FromLong(1L);
    if (const_one == NULL)
        goto error;

    incremented_item = PyNumber_Add(item, const_one);
    if (incremented_item == NULL)
        goto error;

    if (PyObject_SetItem(dict, key, incremented_item) < 0)
        goto error;
    rv = 0; /* Success */
    /* Continua com o código de limpeza */

 error:
    /* Código de limpeza, compartilhado pelo caminho sucesso e f

    /* Usa Py_XDECREF() para ignorar referências NULL */
    Py_XDECREF(item);
    Py_XDECREF(const_one);
    Py_XDECREF(incremented_item);

    return rv; /* -1 para erro, 0 para sucesso */
}
```

Este exemplo representa um uso endossado da instrução `goto` em C! Ele ilustra o uso de **`PyErr_ExceptionMatches()`** e **`PyErr_Clear()`** para lidar com exceções específicas, e o uso de **`Py_XDECREF()`** para descartar referências de propriedade que podem ser `NULL` (observe o `'X'` no nome; **`Py_DECREF()`** travaria quando confrontado com uma referência `NULL`). É importante que as variáveis usadas para manter as referências de propriedade sejam inicializadas com `NULL` para que isso funcione; da mesma forma, o valor de retorno proposto é inicializado para `-1` (falha) e apenas definido para sucesso após a chamada final feita ser bem sucedida.

Incorporando Python

A única tarefa importante com a qual apenas os incorporadores (em oposição aos escritores de extensão) do interpretador Python precisam se preocupar é a inicialização e, possivelmente, a finalização do interpretador Python. A maior parte da funcionalidade do interpretador só pode ser usada após a inicialização do interpretador.

A função de inicialização básica é **`Py_Initialize()`**. Isso inicializa a tabela de módulos carregados e cria os módulos fundamentais **`builtins`**, **`__main__`** e **`sys`**. Ela também inicializa o caminho de pesquisa de módulos (`sys.path`).

`Py_Initialize()` não define a “lista de argumentos de script” (`sys.argv`). Se esta variável for necessária para o código Python que será executado posteriormente, **`PyConfig.argv`** e **`PyConfig.parse_argv`** devem estar definidas; veja Configuração de inicialização do Python.

Na maioria dos sistemas (em particular, no Unix e no Windows, embora os detalhes sejam ligeiramente diferentes), **`Py_Initialize()`** calcula o caminho de pesquisa de módulos com base em sua melhor estimativa para a localização do executável do interpretador Python padrão, presumindo que a biblioteca Python é encontrada em um local fixo em relação ao executável do interpretador Python. Em particular, ele procura por um diretório chamado `lib/python`*`X.Y`* relativo ao diretório pai onde o executável chamado `python` é encontrado no caminho de pesquisa de comandos do shell (a variável de ambiente **`PATH`**).

Por exemplo, se o executável Python for encontrado em `/usr/local/bin/python`, ele presumirá que as bibliotecas estão em `/usr/local/lib/python`*X.Y*. (Na verdade, este caminho particular também é o local reserva, usado quando nenhum arquivo executável chamado `python` é encontrado ao longo de **`PATH`**.) O usuário pode substituir este comportamento definindo a variável de ambiente **`PYTHONHOME`**, ou insira diretórios adicionais na frente do caminho padrão definindo **`PYTHONPATH`**.

The embedding application can steer the search by setting **`PyConfig.program_name`** *before* calling **`Py_InitializeFromConfig()`**. Note that **`PYTHONHOME`** still overrides this and **`PYTHONPATH`** is still inserted in front of the standard path.

Às vezes, é desejável "desinicializar" o Python. Por exemplo, a aplicação pode querer iniciar novamente (fazer outra chamada para **`Py_Initialize()`**) ou a aplicação simplesmente termina com o uso de Python e deseja liberar memória alocada pelo Python. Isso pode ser feito chamando **`Py_FinalizeEx()`**. A função **`Py_IsInitialized()`** retorna verdadeiro se o Python está atualmente no estado inicializado. Mais informações sobre essas funções são fornecidas em um capítulo posterior. Observe que **`Py_FinalizeEx()`** *não* libera toda a memória alocada pelo interpretador Python, por exemplo, a memória alocada por módulos de extensão atualmente não pode ser liberada.

Compilações de depuração

Python pode ser compilado com várias macros para permitir verificações extras do interpretador e módulos de extensão. Essas verificações tendem a adicionar uma grande quantidade de sobrecarga ao tempo de execução, portanto, não são habilitadas por padrão.

Uma lista completa dos vários tipos de compilações de depuração está no arquivo `Misc/SpecialBuilds.txt` na distribuição do código-fonte do Python. Estão disponíveis compilações que oferecem suporte ao rastreamento de contagens de referências, depuração do alocador de memória ou criação de perfil de baixo nível do laço do interpretador

principal. Apenas as compilações usadas com mais frequência serão descritas no restante desta seção.

Py_DEBUG

Compiling the interpreter with the **`Py_DEBUG`** macro defined produces what is generally meant by a debug build of Python.

On Unix, **`Py_DEBUG`** can be enabled by adding **`--with-pydebug`** to the `./configure` command. This will also disable compiler optimization.

On Windows, selecting a debug build (e.g., by passing the **`-d`** option to `PCbuild/build.bat`) automatically enables **`Py_DEBUG`**. Additionally, the presence of the not-Python-specific **`_DEBUG`** macro, when defined by the compiler, will also implicitly enable **`Py_DEBUG`**.

In addition to the reference count debugging described below, extra checks are performed. See Python Debug Build for more details.

Defining `Py_TRACE_REFS` enables reference tracing (see the **`configure --with-trace-refs option`**). When defined, a circular doubly linked list of active objects is maintained by adding two extra fields to every **`PyObject`**. Total allocations are tracked as well. Upon exit, all existing references are printed. (In interactive mode this happens after every statement run by the interpreter.)

Consulte `Misc/SpecialBuilds.txt` na distribuição do código-fonte Python para informações mais detalhadas.

Ferramentas de terceiros recomendadas

As seguintes ferramentas de terceiros oferecem abordagens mais simples e mais sofisticadas para criar extensões C, C++ e Rust para Python:

- Cython [https://cython.org/]
- cffi [https://cffi.readthedocs.io]
- HPy [https://hpyproject.org/]
- nanobind [https://github.com/wjakob/nanobind] (C++)

- Numba [https://numba.pydata.org/]
- pybind11 [https://pybind11.readthedocs.io/] (C++)
- PyO3 [https://pyo3.rs/] (Rust)
- SWIG [https://www.swig.org]

O uso de ferramentas como essas pode ajudar a evitar a criação de código estritamente vinculado a uma versão específica do CPython, evitar erros de contagem de referências e focar mais no seu próprio código do que em usar a API do CPython. Em geral, novas versões do Python podem ser suportadas atualizando a ferramenta, e seu código frequentemente usará APIs mais novas e eficientes automaticamente. Algumas ferramentas também oferecem suporte à compilação para outras implementações do Python a partir de um único conjunto de fontes.

Esses projetos não são suportados pelas mesmas pessoas que mantêm o Python, e quaisquer problemas precisam ser relatados diretamente aos projetos. Lembre-se de verificar se o projeto ainda é mantido e tem suporte, pois a lista acima pode ficar desatualizada.

Ver também

Guia do Usuário de Empacotamento do Python: Extensões Binárias
[https://packaging.python.org/guides/packaging-binary-extensions/]

O Guia do Usuário de Empacotamento do Python não abrange apenas várias ferramentas disponíveis que simplificam a criação de extensões binárias, mas também discute os vários motivos pelos quais a criação de um módulo de extensão pode ser desejável em primeiro lugar.

C API and ABI Stability

A menos que documentado de outra forma, a API C do Python é coberta pela Política de Compatibilidade com versões anteriores, **PEP 387** [https://peps.python.org/pep-0387/]. A maioria das alterações são compatíveis com a fonte (normalmente adicionando apenas uma nova API). A alteração ou remoção da API existente só é feita após um período de descontinuação ou para corrigir problemas sérios.

A Interface Binária de Aplicação (ABI) do CPython é compatível para frente e para trás através de uma versão menor (se elas forem compiladas da mesma maneira; veja Considerações da plataforma abaixo). Portanto, o código compilado para Python 3.10.0 funcionará em 3.10.8 e vice-versa, mas precisará ser compilado separadamente para 3.9.x e 3.11.x.

Existem dois níveis de API C com diferentes expectativas de estabilidade:

- API Instável ("Unstable API"), pode mudar em versões menores sem período de depreciação. É marcado pelo prefixo `PyUnstable` nos nomes.
- API Limitada ("Limited API"), é compatível em várias versões menores. Quando **`Py_LIMITED_API`** é definido, apenas este subconjunto é exposto de `Python.h`.

Elas são discutidas em mais detalhes abaixo.

Nomes prefixados por um sublinhado, como `_Py_InternalState`, são APIs privadas que podem ser alteradas sem aviso prévio, mesmo em lançamentos de correção. Se você precisa usar essa API, considere entrar em contato com os desenvolvedores do CPython [https://discuss.python.org/c/core-dev/c-api/30] para discutir a adição de uma API pública para o seu caso de uso.

API C Instável

Qualquer API nomeada com o prefixo “PyUnstable” expõe detalhes de implementação do CPython e pode mudar em cada versão menor (por exemplo, de 3.9 para 3.10) sem nenhum aviso de depreciação. No entanto, não mudará em uma versão de correção de bugs (por exemplo, de 3.10.0 para 3.10.1).

É geralmente destinado a ferramentas especializadas de baixo nível, como depuradores.

Projetos que utilizam esta API são esperados para seguir o desenvolvimento do CPython e dedicar esforço extra para se ajustar às mudanças.

Stable Application Binary Interfaces

Python’s *Stable ABI* allows extensions to be compatible with multiple versions of Python, without recompilation.

Nota

For simplicity, this document talks about *extensions*, but Stable ABI works the same way for all uses of the API – for example, embedding Python.

There are two Stable ABIs:

- `abi3`, introduced in Python 3.2, is compatible with **non-**free-threaded builds of CPython.
- `abi3t`, introduced in Python 3.15, is compatible with free-threaded builds of CPython. It has stricter API limitations than `abi3`.

  Adicionado na versão 3.15: `abi3t` was added in **PEP 803** [https://peps.python.org/pep-0803/]

It is possible for an extension to be compiled for *both* `abi3` and `abi3t` at the same time; the result will be compatible with both free-threaded and non-

free-threaded builds of Python. Currently, this has no downsides compared to compiling for `abi3t` only.

Each Stable ABI is versioned using the first two numbers of the Python version. For example, Stable ABI 3.14 corresponds to Python 3.14. An extension compiled for Stable ABI 3.x is ABI-compatible with Python 3.x and above.

Extensions that target a stable ABI must only use a limited subset of the C API. This subset is known as the *Limited API*; its contents are listed below.

On Windows, extensions that use a Stable ABI should be linked against `python3.dll` rather than a version-specific library such as `python39.dll`. This library only exposes the relevant symbols.

On some platforms, Python will look for and load shared library files named with the `abi3` or `abi3t` tag (for example, `mymodule.abi3.so`). Free-threaded interpreters only recognize the `abi3t` tag, while non-free-threaded ones will prefer `abi3` but fall back to `abi3t`. Thus, extensions compatible with both ABIs should use the `abi3t` tag.

Python does not necessarily check that extensions it loads have compatible ABI. Extension authors are encouraged to add a check using the **`Py_mod_abi`** slot or the **`PyABIInfo_Check()`** function, but the user (or their packaging tool) is ultimately responsible for ensuring that, for example, extensions built for Stable ABI 3.10 are not installed for lower versions of Python.

All functions in Stable ABI are present as functions in Python's shared library, not solely as macros. This makes them usable in languages that don't use the C preprocessor, including Python's **`ctypes`**.

Compiling for Stable ABI

Nota

Build tools (such as, for example, meson-python, scikit-build-core, or Setuptools) often have a mechanism for setting macros and synchronizing

them with extension filenames and other metadata. Prefer using such a mechanism, if it exists, over defining the macros manually.

The rest of this section is mainly relevant for tool authors, and for people who compile extensions manually.

Ver também

list of recommended tools [https://packaging.python.org/en/latest/guides/tool-recommendations/#build-backends-for-extension-modules] in the Python Packaging User Guide

To compile for a Stable ABI, define one or both of the following macros to the lowest Python version your extension should support, in `Py_PACK_VERSION` format. Typically, you should choose a specific value rather than the version of the Python headers you are compiling against.

The macros must be defined before including `Python.h`. Since `Py_PACK_VERSION` is not available at this point, you will need to use the numeric value directly. For reference, the values for a few recent Python versions are:

```
0x30a0000  /* Py_PACK_VERSION(3,10) */
0x30b0000  /* Py_PACK_VERSION(3,11) */
0x30c0000  /* Py_PACK_VERSION(3,12) */
0x30d0000  /* Py_PACK_VERSION(3,13) */
0x30e0000  /* Py_PACK_VERSION(3,14) */
0x30f0000  /* Py_PACK_VERSION(3,15) */
```

When one of the macros is defined, `Python.h` will only expose API that is compatible with the given Stable ABI – that is, the Limited API plus some definitions that need to be visible to the compiler but should not be used directly. When both are defined, `Python.h` will only expose API compatible with both Stable ABIs.

Py_LIMITED_API

Target `abi3`, that is, non-free-threaded builds of CPython. See above for common information.

Py_TARGET_ABI3T

Target `abi3t`, that is, free-threaded builds of CPython. See above for common information.

Adicionado na versão 3.15.

Both macros specify a target ABI; the different naming style is due to backwards compatibility.

Historical note

You can also define `Py_LIMITED_API` as `3`. This works the same as `0x03020000` (Python 3.2, the version that introduced Stable ABI).

When both are defined, `Python.h` may, or may not, redefine **`Py_LIMITED_API`** to match **`Py_TARGET_ABI3T`**.

On a free-threaded build – that is, when **`Py_GIL_DISABLED`** is defined – **`Py_TARGET_ABI3T`** defaults to the value of **`Py_LIMITED_API`**. This means that there are two ways to build for both `abi3` and `abi3t`:

- define both **`Py_LIMITED_API`** and **`Py_TARGET_ABI3T`**, or
- define only **`Py_LIMITED_API`** and:
  - on Windows, define **`Py_GIL_DISABLED`**;
  - on other systems, use the headers of free-threaded build of Python.

Stable ABI Scope and Performance

The goal for Stable ABI is to allow everything that is possible with the full C API, but possibly with a performance penalty. Generally, compatibility with Stable ABI will require some changes to an extension's source code.

For example, while `PyList_GetItem()` is available, its “unsafe” macro variant `PyList_GET_ITEM()` is not. The macro can be faster because it can rely on version-specific implementation details of the list object.

For another example, when *not* compiling for Stable ABI, some C API functions are inlined or replaced by macros. Compiling for Stable ABI disables this inlining, allowing stability as Python’s data structures are improved, but possibly reducing performance.

By leaving out the **`Py_LIMITED_API`** or **`Py_TARGET_ABI3T`** definition, it is possible to compile Stable-ABI-compatible source for a version-specific ABI. A potentially faster version-specific extension can then be distributed alongside a version compiled for Stable ABI – a slower but more compatible fallback.

Stable ABI Caveats

Note that compiling for Stable ABI is *not* a complete guarantee that code will be compatible with the expected Python versions. Stable ABI prevents *ABI* issues, like linker errors due to missing symbols or data corruption due to changes in structure layouts or function signatures. However, other changes in Python can change the *behavior* of extensions.

One issue that the `Py_TARGET_ABI3T` and `Py_LIMITED_API` macros do not guard against is calling a function with arguments that are invalid in a lower Python version. For example, consider a function that starts accepting `NULL` for an argument. In Python 3.9, `NULL` now selects a default behavior, but in Python 3.8, the argument will be used directly, causing a `NULL` dereference and crash. A similar argument works for fields of structs.

For these reasons, we recommend testing an extension with *all* minor Python versions it supports.

Também recomendamos revisar a documentação de todas as APIs utilizadas para verificar se ela faz parte explicitamente da API Limitada. Mesmo com a definição de `Py_LIMITED_API`, algumas declarações privadas são expostas por razões técnicas (ou até mesmo acidentalmente, como bugs).

Also note that while compiling with `Py_LIMITED_API` 3.8 means that the extension should *load* on Python 3.12, and *compile* with Python 3.12, the same source will not necessarily compile with `Py_LIMITED_API` set to 3.12. In general, parts of the Limited API may be deprecated and removed, provided that Stable ABI stays stable.

Considerações da plataforma

ABI stability depends not only on Python, but also on the compiler used, lower-level libraries and compiler options. For the purposes of the Stable ABIs, these details define a "platform". They usually depend on the OS type and processor architecture

It is the responsibility of each particular distributor of Python to ensure that all Python versions on a particular platform are built in a way that does not break the Stable ABIs, or the version-specific ABIs. This is the case with Windows and macOS releases from `python.org` and many third-party distributors.

ABI Checking

Adicionado na versão 3.15.

Python includes a rudimentary check for ABI compatibility.

This check is not comprehensive. It only guards against common cases of incompatible modules being installed for the wrong interpreter. It also does not take platform incompatibilities into account. It can only be done after an extension is successfully loaded.

Despite these limitations, it is recommended that extension modules use this mechanism, so that detectable incompatibilities raise exceptions rather than crash.

Most modules can use this check via the **`Py_mod_abi`** slot and the **`PyABIInfo_VAR`** macro, for example like this:

```
PyABIInfo_VAR(abi_info);

static PyModuleDef_Slot mymodule_slots[] = {
   {Py_mod_abi, &abi_info},
   ...
};
```

The full API is described below for advanced use cases.

int PyABIInfo_Check(PyABIInfo *info, const char *module_name)

Parte da ABI Estável desde a versão 3.15.

Verify that the given *info* is compatible with the currently running interpreter.

Return 0 on success. On failure, raise an exception and return -1.

If the ABI is incompatible, the raised exception will be **`ImportError`**.

The *module_name* argument can be `NULL`, or point to a NUL-terminated UTF-8-encoded string used for error messages.

Note that if *info* describes the ABI that the current code uses (as defined by **`PyABIInfo_VAR`**, for example), using any other Python C API may lead to crashes. In particular, it is not safe to examine the raised exception.

Adicionado na versão 3.15.

PyABIInfo_VAR(NAME)

Parte da ABI Estável desde a versão 3.15.

Define a static **`PyABIInfo`** variable with the given *NAME* that describes the ABI that the current code will use. This macro expands to:

```
static PyABIInfo NAME = {
    1, 0,
    PyABIInfo_DEFAULT_FLAGS,
    PY_VERSION_HEX,
    PyABIInfo_DEFAULT_ABI_VERSION
}
```

Adicionado na versão 3.15.

type PyABIInfo

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.15.

uint8_t abiinfo_major_version

The major version of **`PyABIInfo`**. Can be set to:

- `0` to skip all checking, or
- `1` to specify this version of **`PyABIInfo`**.

uint8_t abiinfo_minor_version

The minor version of **`PyABIInfo`**. Must be set to `0`; larger values are reserved for backwards-compatible future versions of **`PyABIInfo`**.

uint16_t flags

This field is usually set to the following macro:

PyABIInfo_DEFAULT_FLAGS

Parte da ABI Estável desde a versão 3.15.

Default flags, based on current values of macros such as **`Py_LIMITED_API`** and **`Py_GIL_DISABLED`**.

Alternately, the field can be set to the following flags, combined by bitwise OR. Unused bits must be set to zero.

ABI variant – one of:

PyABIInfo_STABLE

Parte da ABI Estável desde a versão 3.15.

Specifies that Stable ABI is used.

PyABIInfo_INTERNAL

Specifies ABI specific to a particular build of CPython. Internal use only.

Free-threading compatibility – one of:

PyABIInfo_FREETHREADED

Parte da ABI Estável desde a versão 3.15.
Specifies ABI compatible with free-threaded builds of CPython. (That is, ones compiled with `--disable-gil`; with `t` in `sys.abiflags`)

PyABIInfo_GIL

Parte da ABI Estável desde a versão 3.15.
Specifies ABI compatible with non-free-threaded builds of CPython (ones compiled *without* `--disable-gil`).

PyABIInfo_FREETHREADING_AGNOSTIC

Parte da ABI Estável desde a versão 3.15.
Specifies ABI compatible with both free-threaded and non-free-threaded builds of CPython, that is, both `abi3` and `abi3t`.

uint32_t build_version

The version of the Python headers used to build the code, in the format used by `PY_VERSION_HEX`.

This can be set to `0` to skip any checks related to this field. This option is meant mainly for projects that do not use the CPython headers directly, and do not emulate a specific version of them.

uint32_t abi_version

The ABI version.

For Stable ABI, this field should be the value of `Py_LIMITED_API` or `Py_TARGET_ABI3T`. If both are defined, use the smaller value. (If `Py_LIMITED_API` is `3`; use Py_PACK_VERSION(3, 2) instead of `3`.)

Otherwise, it should be set to `PY_VERSION_HEX`.

It can also be set to `0` to skip any checks related to this field.

PyABIInfo_DEFAULT_ABI_VERSION

Parte da ABI Estável desde a versão 3.15.
The value that should be used for this field, based on current values of macros such as `Py_LIMITED_API`, `PY_VERSION_HEX` and `Py_GIL_DISABLED`.

Adicionado na versão 3.15.

Conteúdo da API Limitada

This is the definitive list of Limited API for Python 3.15:

- `METH_CLASS`
- `METH_COEXIST`
- `METH_FASTCALL`
- `METH_METHOD`
- `METH_NOARGS`
- `METH_O`
- `METH_STATIC`
- `METH_VARARGS`
- `PY_VECTORCALL_ARGUMENTS_OFFSET`
- `PyABIInfo`
- `PyABIInfo_Check()`
- `PyABIInfo_DEFAULT_ABI_VERSION`
- `PyABIInfo_DEFAULT_FLAGS`
- `PyABIInfo_FREETHREADED`
- `PyABIInfo_FREETHREADING_AGNOSTIC`
- `PyABIInfo_GIL`
- `PyABIInfo_STABLE`
- `PyABIInfo_VAR`
- `PyAIter_Check()`
- `PyArg_Parse()`
- `PyArg_ParseTuple()`
- `PyArg_ParseTupleAndKeywords()`
- `PyArg_UnpackTuple()`
- `PyArg_VaParse()`
- `PyArg_VaParseTupleAndKeywords()`
- `PyArg_ValidateKeywordArguments()`

- PyBUF_ANY_CONTIGUOUS
- PyBUF_CONTIG
- PyBUF_CONTIG_RO
- PyBUF_C_CONTIGUOUS
- PyBUF_FORMAT
- PyBUF_FULL
- PyBUF_FULL_RO
- PyBUF_F_CONTIGUOUS
- PyBUF_INDIRECT
- PyBUF_MAX_NDIM
- PyBUF_ND
- PyBUF_READ
- PyBUF_RECORDS
- PyBUF_RECORDS_RO
- PyBUF_SIMPLE
- PyBUF_STRIDED
- PyBUF_STRIDED_RO
- PyBUF_STRIDES
- PyBUF_WRITABLE
- PyBUF_WRITE
- PyBaseObject_Type
- PyBool_FromLong()
- PyBool_Type
- PyBuffer_FillContiguousStrides()
- PyBuffer_FillInfo()
- PyBuffer_FromContiguous()
- PyBuffer_GetPointer()
- PyBuffer_IsContiguous()
- PyBuffer_Release()
- PyBuffer_SizeFromFormat()
- PyBuffer_ToContiguous()
- PyByteArrayIter_Type
- PyByteArray_AsString()
- PyByteArray_Concat()
- PyByteArray_FromObject()
- PyByteArray_FromStringAndSize()
- PyByteArray_Resize()

- PyByteArray_Size()
- PyByteArray_Type
- PyBytesIter_Type
- PyBytes_AsString()
- PyBytes_AsStringAndSize()
- PyBytes_Concat()
- PyBytes_ConcatAndDel()
- PyBytes_DecodeEscape()
- PyBytes_FromFormat()
- PyBytes_FromFormatV()
- PyBytes_FromObject()
- PyBytes_FromString()
- PyBytes_FromStringAndSize()
- PyBytes_Repr()
- PyBytes_Size()
- PyBytes_Type
- PyCFunction
- PyCFunctionFast
- PyCFunctionFastWithKeywords
- PyCFunctionWithKeywords
- PyCFunction_GetFlags()
- PyCFunction_GetFunction()
- PyCFunction_GetSelf()
- PyCFunction_New()
- PyCFunction_NewEx()
- PyCFunction_Type
- PyCMethod_New()
- PyCallIter_New()
- PyCallIter_Type
- PyCallable_Check()
- PyCapsule_Destructor
- PyCapsule_GetContext()
- PyCapsule_GetDestructor()
- PyCapsule_GetName()
- PyCapsule_GetPointer()
- PyCapsule_Import()
- PyCapsule_IsValid()

- PyCapsule_New()
- PyCapsule_SetContext()
- PyCapsule_SetDestructor()
- PyCapsule_SetName()
- PyCapsule_SetPointer()
- PyCapsule_Type
- PyClassMethodDescr_Type
- PyCodec_BackslashReplaceErrors()
- PyCodec_Decode()
- PyCodec_Decoder()
- PyCodec_Encode()
- PyCodec_Encoder()
- PyCodec_IgnoreErrors()
- PyCodec_IncrementalDecoder()
- PyCodec_IncrementalEncoder()
- PyCodec_KnownEncoding()
- PyCodec_LookupError()
- PyCodec_NameReplaceErrors()
- PyCodec_Register()
- PyCodec_RegisterError()
- PyCodec_ReplaceErrors()
- PyCodec_StreamReader()
- PyCodec_StreamWriter()
- PyCodec_StrictErrors()
- PyCodec_Unregister()
- PyCodec_XMLCharRefReplaceErrors()
- PyComplex_FromDoubles()
- PyComplex_ImagAsDouble()
- PyComplex_RealAsDouble()
- PyComplex_Type
- PyCriticalSection
- PyCriticalSection2
- PyCriticalSection2_Begin()
- PyCriticalSection2_End()
- PyCriticalSection_Begin()
- PyCriticalSection_End()
- PyDescr_NewClassMethod()

- PyDescr_NewGetSet()
- PyDescr_NewMember()
- PyDescr_NewMethod()
- PyDictItems_Type
- PyDictIterItem_Type
- PyDictIterKey_Type
- PyDictIterValue_Type
- PyDictKeys_Type
- PyDictProxy_New()
- PyDictProxy_Type
- PyDictRevIterItem_Type
- PyDictRevIterKey_Type
- PyDictRevIterValue_Type
- PyDictValues_Type
- PyDict_Clear()
- PyDict_Contains()
- PyDict_Copy()
- PyDict_DelItem()
- PyDict_DelItemString()
- PyDict_GetItem()
- PyDict_GetItemRef()
- PyDict_GetItemString()
- PyDict_GetItemStringRef()
- PyDict_GetItemWithError()
- PyDict_Items()
- PyDict_Keys()
- PyDict_Merge()
- PyDict_MergeFromSeq2()
- PyDict_New()
- PyDict_Next()
- PyDict_SetDefaultRef()
- PyDict_SetItem()
- PyDict_SetItemString()
- PyDict_Size()
- PyDict_Type
- PyDict_Update()
- PyDict_Values()

- PyEllipsis_Type
- PyEnum_Type
- PyErr_BadArgument()
- PyErr_BadInternalCall()
- PyErr_CheckSignals()
- PyErr_Clear()
- PyErr_Display()
- PyErr_DisplayException()
- PyErr_ExceptionMatches()
- PyErr_Fetch()
- PyErr_Format()
- PyErr_FormatV()
- PyErr_GetExcInfo()
- PyErr_GetHandledException()
- PyErr_GetRaisedException()
- PyErr_GivenExceptionMatches()
- PyErr_NewException()
- PyErr_NewExceptionWithDoc()
- PyErr_NoMemory()
- PyErr_NormalizeException()
- PyErr_Occurred()
- PyErr_Print()
- PyErr_PrintEx()
- PyErr_ProgramText()
- PyErr_ResourceWarning()
- PyErr_Restore()
- PyErr_SetExcFromWindowsErr()
- PyErr_SetExcFromWindowsErrWithFilename()
- PyErr_SetExcFromWindowsErrWithFilenameObject()
- PyErr_SetExcFromWindowsErrWithFilenameObjects()
- PyErr_SetExcInfo()
- PyErr_SetFromErrno()
- PyErr_SetFromErrnoWithFilename()
- PyErr_SetFromErrnoWithFilenameObject()
- PyErr_SetFromErrnoWithFilenameObjects()
- PyErr_SetFromWindowsErr()
- PyErr_SetFromWindowsErrWithFilename()

- PyErr_SetHandledException()
- PyErr_SetImportError()
- PyErr_SetImportErrorSubclass()
- PyErr_SetInterrupt()
- PyErr_SetInterruptEx()
- PyErr_SetNone()
- PyErr_SetObject()
- PyErr_SetRaisedException()
- PyErr_SetString()
- PyErr_SyntaxLocation()
- PyErr_SyntaxLocationEx()
- PyErr_WarnEx()
- PyErr_WarnExplicit()
- PyErr_WarnFormat()
- PyErr_WriteUnraisable()
- PyEval_AcquireThread()
- PyEval_EvalCode()
- PyEval_EvalCodeEx()
- PyEval_EvalFrame()
- PyEval_EvalFrameEx()
- PyEval_GetBuiltins()
- PyEval_GetFrame()
- PyEval_GetFrameBuiltins()
- PyEval_GetFrameGlobals()
- PyEval_GetFrameLocals()
- PyEval_GetFuncDesc()
- PyEval_GetFuncName()
- PyEval_GetGlobals()
- PyEval_GetLocals()
- PyEval_InitThreads()
- PyEval_ReleaseThread()
- PyEval_RestoreThread()
- PyEval_SaveThread()
- PyExc_ArithmeticError
- PyExc_AssertionError
- PyExc_AttributeError
- PyExc_BaseException

- PyExc_BaseExceptionGroup
- PyExc_BlockingIOError
- PyExc_BrokenPipeError
- PyExc_BufferError
- PyExc_BytesWarning
- PyExc_ChildProcessError
- PyExc_ConnectionAbortedError
- PyExc_ConnectionError
- PyExc_ConnectionRefusedError
- PyExc_ConnectionResetError
- PyExc_DeprecationWarning
- PyExc_EOFError
- PyExc_EncodingWarning
- PyExc_EnvironmentError
- PyExc_Exception
- PyExc_FileExistsError
- PyExc_FileNotFoundError
- PyExc_FloatingPointError
- PyExc_FutureWarning
- PyExc_GeneratorExit
- PyExc_IOError
- PyExc_ImportError
- PyExc_ImportWarning
- PyExc_IndentationError
- PyExc_IndexError
- PyExc_InterruptedError
- PyExc_IsADirectoryError
- PyExc_KeyError
- PyExc_KeyboardInterrupt
- PyExc_LookupError
- PyExc_MemoryError
- PyExc_ModuleNotFoundError
- PyExc_NameError
- PyExc_NotADirectoryError
- PyExc_NotImplementedError
- PyExc_OSError
- PyExc_OverflowError

- PyExc_PendingDeprecationWarning
- PyExc_PermissionError
- PyExc_ProcessLookupError
- PyExc_RecursionError
- PyExc_ReferenceError
- PyExc_ResourceWarning
- PyExc_RuntimeError
- PyExc_RuntimeWarning
- PyExc_StopAsyncIteration
- PyExc_StopIteration
- PyExc_SyntaxError
- PyExc_SyntaxWarning
- PyExc_SystemError
- PyExc_SystemExit
- PyExc_TabError
- PyExc_TimeoutError
- PyExc_TypeError
- PyExc_UnboundLocalError
- PyExc_UnicodeDecodeError
- PyExc_UnicodeEncodeError
- PyExc_UnicodeError
- PyExc_UnicodeTranslateError
- PyExc_UnicodeWarning
- PyExc_UserWarning
- PyExc_ValueError
- PyExc_Warning
- PyExc_WindowsError
- PyExc_ZeroDivisionError
- PyExceptionClass_Name()
- PyException_GetArgs()
- PyException_GetCause()
- PyException_GetContext()
- PyException_GetTraceback()
- PyException_SetArgs()
- PyException_SetCause()
- PyException_SetContext()
- PyException_SetTraceback()

- PyFile_FromFd()
- PyFile_GetLine()
- PyFile_WriteObject()
- PyFile_WriteString()
- PyFilter_Type
- PyFloat_AsDouble()
- PyFloat_FromDouble()
- PyFloat_FromString()
- PyFloat_GetInfo()
- PyFloat_GetMax()
- PyFloat_GetMin()
- PyFloat_Type
- PyFrameObject
- PyFrame_GetCode()
- PyFrame_GetLineNumber()
- PyFrozenSet_New()
- PyFrozenSet_Type
- PyGC_Collect()
- PyGC_Disable()
- PyGC_Enable()
- PyGC_IsEnabled()
- PyGILState_Ensure()
- PyGILState_GetThisThreadState()
- PyGILState_Release()
- PyGILState_STATE
- PyGetSetDef
- PyGetSetDescr_Type
- PyImport_AddModule()
- PyImport_AddModuleObject()
- PyImport_AddModuleRef()
- PyImport_AppendInittab()
- PyImport_ExecCodeModule()
- PyImport_ExecCodeModuleEx()
- PyImport_ExecCodeModuleObject()
- PyImport_ExecCodeModuleWithPathnames()
- PyImport_GetImporter()
- PyImport_GetMagicNumber()

- PyImport_GetMagicTag()
- PyImport_GetModule()
- PyImport_GetModuleDict()
- PyImport_Import()
- PyImport_ImportFrozenModule()
- PyImport_ImportFrozenModuleObject()
- PyImport_ImportModule()
- PyImport_ImportModuleLevel()
- PyImport_ImportModuleLevelObject()
- PyImport_ReloadModule()
- PyIndex_Check()
- PyInterpreterGuard
- PyInterpreterGuard_Close()
- PyInterpreterGuard_FromCurrent()
- PyInterpreterGuard_FromView()
- PyInterpreterState
- PyInterpreterState_Clear()
- PyInterpreterState_Delete()
- PyInterpreterState_Get()
- PyInterpreterState_GetDict()
- PyInterpreterState_GetID()
- PyInterpreterState_New()
- PyInterpreterView
- PyInterpreterView_Close()
- PyInterpreterView_FromCurrent()
- PyInterpreterView_FromMain()
- PyIter_Check()
- PyIter_Next()
- PyIter_NextItem()
- PyIter_Send()
- PyListIter_Type
- PyListRevIter_Type
- PyList_Append()
- PyList_AsTuple()
- PyList_GetItem()
- PyList_GetItemRef()
- PyList_GetSlice()

- PyList_Insert()
- PyList_New()
- PyList_Reverse()
- PyList_SetItem()
- PyList_SetSlice()
- PyList_Size()
- PyList_Sort()
- PyList_Type
- PyLongExport
- PyLongLayout
- PyLongObject
- PyLongRangeIter_Type
- PyLongWriter
- PyLongWriter_Create()
- PyLongWriter_Discard()
- PyLongWriter_Finish()
- PyLong_AsDouble()
- PyLong_AsInt()
- PyLong_AsInt32()
- PyLong_AsInt64()
- PyLong_AsLong()
- PyLong_AsLongAndOverflow()
- PyLong_AsLongLong()
- PyLong_AsLongLongAndOverflow()
- PyLong_AsNativeBytes()
- PyLong_AsSize_t()
- PyLong_AsSsize_t()
- PyLong_AsUInt32()
- PyLong_AsUInt64()
- PyLong_AsUnsignedLong()
- PyLong_AsUnsignedLongLong()
- PyLong_AsUnsignedLongLongMask()
- PyLong_AsUnsignedLongMask()
- PyLong_AsVoidPtr()
- PyLong_Export()
- PyLong_FreeExport()
- PyLong_FromDouble()

- PyLong_FromInt32()
- PyLong_FromInt64()
- PyLong_FromLong()
- PyLong_FromLongLong()
- PyLong_FromNativeBytes()
- PyLong_FromSize_t()
- PyLong_FromSsize_t()
- PyLong_FromString()
- PyLong_FromUInt32()
- PyLong_FromUInt64()
- PyLong_FromUnsignedLong()
- PyLong_FromUnsignedLongLong()
- PyLong_FromUnsignedNativeBytes()
- PyLong_FromVoidPtr()
- PyLong_GetInfo()
- PyLong_GetNativeLayout()
- PyLong_Type
- PyMODEXPORT_FUNC
- PyMap_Type
- PyMapping_Check()
- PyMapping_GetItemString()
- PyMapping_GetOptionalItem()
- PyMapping_GetOptionalItemString()
- PyMapping_HasKey()
- PyMapping_HasKeyString()
- PyMapping_HasKeyStringWithError()
- PyMapping_HasKeyWithError()
- PyMapping_Items()
- PyMapping_Keys()
- PyMapping_Length()
- PyMapping_SetItemString()
- PyMapping_Size()
- PyMapping_Values()
- PyMem_Calloc()
- PyMem_Free()
- PyMem_Malloc()
- PyMem_RawCalloc()

- PyMem_RawFree()
- PyMem_RawMalloc()
- PyMem_RawRealloc()
- PyMem_Realloc()
- PyMemberDef
- PyMemberDescr_Type
- PyMember_GetOne()
- PyMember_SetOne()
- PyMemoryView_FromBuffer()
- PyMemoryView_FromMemory()
- PyMemoryView_FromObject()
- PyMemoryView_GetContiguous()
- PyMemoryView_Type
- PyMethodDef
- PyMethodDescr_Type
- PyModuleDef
- PyModuleDef_Base
- PyModuleDef_Init()
- PyModuleDef_Slot
- PyModuleDef_Type
- PyModule_Add()
- PyModule_AddFunctions()
- PyModule_AddIntConstant()
- PyModule_AddObject()
- PyModule_AddObjectRef()
- PyModule_AddStringConstant()
- PyModule_AddType()
- PyModule_Create2()
- PyModule_Exec()
- PyModule_ExecDef()
- PyModule_FromDefAndSpec2()
- PyModule_FromSlotsAndSpec()
- PyModule_GetDef()
- PyModule_GetDict()
- PyModule_GetFilename()
- PyModule_GetFilenameObject()
- PyModule_GetName()

- PyModule_GetNameObject()
- PyModule_GetState()
- PyModule_GetStateSize()
- PyModule_GetState_DuringGC()
- PyModule_GetToken()
- PyModule_GetToken_DuringGC()
- PyModule_New()
- PyModule_NewObject()
- PyModule_SetDocString()
- PyModule_Type
- PyNumber_Absolute()
- PyNumber_Add()
- PyNumber_And()
- PyNumber_AsSsize_t()
- PyNumber_Check()
- PyNumber_Divmod()
- PyNumber_Float()
- PyNumber_FloorDivide()
- PyNumber_InPlaceAdd()
- PyNumber_InPlaceAnd()
- PyNumber_InPlaceFloorDivide()
- PyNumber_InPlaceLshift()
- PyNumber_InPlaceMatrixMultiply()
- PyNumber_InPlaceMultiply()
- PyNumber_InPlaceOr()
- PyNumber_InPlacePower()
- PyNumber_InPlaceRemainder()
- PyNumber_InPlaceRshift()
- PyNumber_InPlaceSubtract()
- PyNumber_InPlaceTrueDivide()
- PyNumber_InPlaceXor()
- PyNumber_Index()
- PyNumber_Invert()
- PyNumber_Long()
- PyNumber_Lshift()
- PyNumber_MatrixMultiply()
- PyNumber_Multiply()

- PyNumber_Negative()
- PyNumber_Or()
- PyNumber_Positive()
- PyNumber_Power()
- PyNumber_Remainder()
- PyNumber_Rshift()
- PyNumber_Subtract()
- PyNumber_ToBase()
- PyNumber_TrueDivide()
- PyNumber_Xor()
- PyOS_AfterFork()
- PyOS_AfterFork_Child()
- PyOS_AfterFork_Parent()
- PyOS_BeforeFork()
- PyOS_CheckStack()
- PyOS_FSPath()
- PyOS_InputHook
- PyOS_InterruptOccurred()
- PyOS_double_to_string()
- PyOS_getsig()
- PyOS_mystricmp()
- PyOS_mystrnicmp()
- PyOS_setsig()
- PyOS_sighandler_t
- PyOS_snprintf()
- PyOS_string_to_double()
- PyOS_strtol()
- PyOS_strtoul()
- PyOS_vsnprintf()
- PyObject
- PyObject.ob_refcnt
- PyObject.ob_type
- PyObject_ASCII()
- PyObject_AsFileDescriptor()
- PyObject_Bytes()
- PyObject_Call()
- PyObject_CallFinalizerFromDealloc()

- PyObject_CallFunction()
- PyObject_CallFunctionObjArgs()
- PyObject_CallMethod()
- PyObject_CallMethodObjArgs()
- PyObject_CallNoArgs()
- PyObject_CallObject()
- PyObject_Calloc()
- PyObject_CheckBuffer()
- PyObject_ClearWeakRefs()
- PyObject_CopyData()
- PyObject_DelAttr()
- PyObject_DelAttrString()
- PyObject_DelItem()
- PyObject_DelItemString()
- PyObject_Dir()
- PyObject_Format()
- PyObject_Free()
- PyObject_GC_Del()
- PyObject_GC_IsFinalized()
- PyObject_GC_IsTracked()
- PyObject_GC_Track()
- PyObject_GC_UnTrack()
- PyObject_GenericGetAttr()
- PyObject_GenericGetDict()
- PyObject_GenericSetAttr()
- PyObject_GenericSetDict()
- PyObject_GetAIter()
- PyObject_GetAttr()
- PyObject_GetAttrString()
- PyObject_GetBuffer()
- PyObject_GetItem()
- PyObject_GetIter()
- PyObject_GetOptionalAttr()
- PyObject_GetOptionalAttrString()
- PyObject_GetTypeData()
- PyObject_GetTypeData_DuringGC()
- PyObject_HasAttr()

- PyObject_HasAttrString()
- PyObject_HasAttrStringWithError()
- PyObject_HasAttrWithError()
- PyObject_Hash()
- PyObject_HashNotImplemented()
- PyObject_Init()
- PyObject_InitVar()
- PyObject_IsInstance()
- PyObject_IsSubclass()
- PyObject_IsTrue()
- PyObject_Length()
- PyObject_Malloc()
- PyObject_Not()
- PyObject_Realloc()
- PyObject_Repr()
- PyObject_RichCompare()
- PyObject_RichCompareBool()
- PyObject_SelfIter()
- PyObject_SetAttr()
- PyObject_SetAttrString()
- PyObject_SetItem()
- PyObject_Size()
- PyObject_Str()
- PyObject_Type()
- PyObject_Vectorcall()
- PyObject_VectorcallMethod()
- PyProperty_Type
- PyRangeIter_Type
- PyRange_Type
- PyReversed_Type
- PySeqIter_New()
- PySeqIter_Type
- PySequence_Check()
- PySequence_Concat()
- PySequence_Contains()
- PySequence_Count()
- PySequence_DelItem()

- PySequence_DelSlice()
- PySequence_Fast()
- PySequence_GetItem()
- PySequence_GetSlice()
- PySequence_In()
- PySequence_InPlaceConcat()
- PySequence_InPlaceRepeat()
- PySequence_Index()
- PySequence_Length()
- PySequence_List()
- PySequence_Repeat()
- PySequence_SetItem()
- PySequence_SetSlice()
- PySequence_Size()
- PySequence_Tuple()
- PySetIter_Type
- PySet_Add()
- PySet_Clear()
- PySet_Contains()
- PySet_Discard()
- PySet_New()
- PySet_Pop()
- PySet_Size()
- PySet_Type
- PySlice_AdjustIndices()
- PySlice_GetIndices()
- PySlice_GetIndicesEx()
- PySlice_New()
- PySlice_Type
- PySlice_Unpack()
- PySlot
- PySlot_DATA
- PySlot_END
- PySlot_FUNC
- PySlot_INT64
- PySlot_INTPTR
- PySlot_OPTIONAL

- PySlot_PTR
- PySlot_PTR_STATIC
- PySlot_SIZE
- PySlot_STATIC
- PySlot_STATIC_DATA
- PySlot_UINT64
- PyState_AddModule()
- PyState_FindModule()
- PyState_RemoveModule()
- PyStructSequence_Desc
- PyStructSequence_Field
- PyStructSequence_GetItem()
- PyStructSequence_New()
- PyStructSequence_NewType()
- PyStructSequence_SetItem()
- PyStructSequence_UnnamedField
- PySuper_Type
- PySys_Audit()
- PySys_AuditTuple()
- PySys_FormatStderr()
- PySys_FormatStdout()
- PySys_GetAttr()
- PySys_GetAttrString()
- PySys_GetObject()
- PySys_GetOptionalAttr()
- PySys_GetOptionalAttrString()
- PySys_GetXOptions()
- PySys_SetArgv()
- PySys_SetArgvEx()
- PySys_SetObject()
- PySys_WriteStderr()
- PySys_WriteStdout()
- PyThreadState
- PyThreadStateToken
- PyThreadState_Clear()
- PyThreadState_Delete()
- PyThreadState_Ensure()

- PyThreadState_EnsureFromView()
- PyThreadState_Get()
- PyThreadState_GetDict()
- PyThreadState_GetFrame()
- PyThreadState_GetID()
- PyThreadState_GetInterpreter()
- PyThreadState_New()
- PyThreadState_Release()
- PyThreadState_SetAsyncExc()
- PyThreadState_Swap()
- PyThread_GetInfo()
- PyThread_ReInitTLS()
- PyThread_acquire_lock()
- PyThread_acquire_lock_timed()
- PyThread_allocate_lock()
- PyThread_create_key()
- PyThread_delete_key()
- PyThread_delete_key_value()
- PyThread_exit_thread()
- PyThread_free_lock()
- PyThread_get_key_value()
- PyThread_get_stacksize()
- PyThread_get_thread_ident()
- PyThread_get_thread_native_id()
- PyThread_init_thread()
- PyThread_release_lock()
- PyThread_set_key_value()
- PyThread_set_stacksize()
- PyThread_start_new_thread()
- PyThread_tss_alloc()
- PyThread_tss_create()
- PyThread_tss_delete()
- PyThread_tss_free()
- PyThread_tss_get()
- PyThread_tss_is_created()
- PyThread_tss_set()
- PyTraceBack_Here()

- PyTraceBack_Print()
- PyTraceBack_Type
- PyTupleIter_Type
- PyTuple_GetItem()
- PyTuple_GetSlice()
- PyTuple_New()
- PyTuple_Pack()
- PyTuple_SetItem()
- PyTuple_Size()
- PyTuple_Type
- PyTypeObject
- PyType_ClearCache()
- PyType_Freeze()
- PyType_FromMetaclass()
- PyType_FromModuleAndSpec()
- PyType_FromSlots()
- PyType_FromSpec()
- PyType_FromSpecWithBases()
- PyType_GenericAlloc()
- PyType_GenericNew()
- PyType_GetBaseByToken()
- PyType_GetBaseByToken_DuringGC()
- PyType_GetFlags()
- PyType_GetFullyQualifiedName()
- PyType_GetModule()
- PyType_GetModuleByDef()
- PyType_GetModuleByToken()
- PyType_GetModuleByToken_DuringGC()
- PyType_GetModuleName()
- PyType_GetModuleState()
- PyType_GetModuleState_DuringGC()
- PyType_GetModule_DuringGC()
- PyType_GetName()
- PyType_GetQualName()
- PyType_GetSlot()
- PyType_GetTypeDataSize()
- PyType_IsSubtype()

- PyType_Modified()
- PyType_Ready()
- PyType_Slot
- PyType_Spec
- PyType_Type
- PyUnicodeDecodeError_Create()
- PyUnicodeDecodeError_GetEncoding()
- PyUnicodeDecodeError_GetEnd()
- PyUnicodeDecodeError_GetObject()
- PyUnicodeDecodeError_GetReason()
- PyUnicodeDecodeError_GetStart()
- PyUnicodeDecodeError_SetEnd()
- PyUnicodeDecodeError_SetReason()
- PyUnicodeDecodeError_SetStart()
- PyUnicodeEncodeError_GetEncoding()
- PyUnicodeEncodeError_GetEnd()
- PyUnicodeEncodeError_GetObject()
- PyUnicodeEncodeError_GetReason()
- PyUnicodeEncodeError_GetStart()
- PyUnicodeEncodeError_SetEnd()
- PyUnicodeEncodeError_SetReason()
- PyUnicodeEncodeError_SetStart()
- PyUnicodeIter_Type
- PyUnicodeTranslateError_GetEnd()
- PyUnicodeTranslateError_GetObject()
- PyUnicodeTranslateError_GetReason()
- PyUnicodeTranslateError_GetStart()
- PyUnicodeTranslateError_SetEnd()
- PyUnicodeTranslateError_SetReason()
- PyUnicodeTranslateError_SetStart()
- PyUnicode_Append()
- PyUnicode_AppendAndDel()
- PyUnicode_AsASCIIString()
- PyUnicode_AsCharmapString()
- PyUnicode_AsEncodedString()
- PyUnicode_AsLatin1String()
- PyUnicode_AsMBCSString()

- PyUnicode_AsRawUnicodeEscapeString()
- PyUnicode_AsUCS4()
- PyUnicode_AsUCS4Copy()
- PyUnicode_AsUTF16String()
- PyUnicode_AsUTF32String()
- PyUnicode_AsUTF8AndSize()
- PyUnicode_AsUTF8String()
- PyUnicode_AsUnicodeEscapeString()
- PyUnicode_AsWideChar()
- PyUnicode_AsWideCharString()
- PyUnicode_BuildEncodingMap()
- PyUnicode_Compare()
- PyUnicode_CompareWithASCIIString()
- PyUnicode_Concat()
- PyUnicode_Contains()
- PyUnicode_Count()
- PyUnicode_Decode()
- PyUnicode_DecodeASCII()
- PyUnicode_DecodeCharmap()
- PyUnicode_DecodeCodePageStateful()
- PyUnicode_DecodeFSDefault()
- PyUnicode_DecodeFSDefaultAndSize()
- PyUnicode_DecodeLatin1()
- PyUnicode_DecodeLocale()
- PyUnicode_DecodeLocaleAndSize()
- PyUnicode_DecodeMBCS()
- PyUnicode_DecodeMBCSStateful()
- PyUnicode_DecodeRawUnicodeEscape()
- PyUnicode_DecodeUTF16()
- PyUnicode_DecodeUTF16Stateful()
- PyUnicode_DecodeUTF32()
- PyUnicode_DecodeUTF32Stateful()
- PyUnicode_DecodeUTF7()
- PyUnicode_DecodeUTF7Stateful()
- PyUnicode_DecodeUTF8()
- PyUnicode_DecodeUTF8Stateful()
- PyUnicode_DecodeUnicodeEscape()

- PyUnicode_EncodeCodePage()
- PyUnicode_EncodeFSDefault()
- PyUnicode_EncodeLocale()
- PyUnicode_Equal()
- PyUnicode_EqualToUTF8()
- PyUnicode_EqualToUTF8AndSize()
- PyUnicode_FSConverter()
- PyUnicode_FSDecoder()
- PyUnicode_Find()
- PyUnicode_FindChar()
- PyUnicode_Format()
- PyUnicode_FromEncodedObject()
- PyUnicode_FromFormat()
- PyUnicode_FromFormatV()
- PyUnicode_FromObject()
- PyUnicode_FromOrdinal()
- PyUnicode_FromString()
- PyUnicode_FromStringAndSize()
- PyUnicode_FromWideChar()
- PyUnicode_GetDefaultEncoding()
- PyUnicode_GetLength()
- PyUnicode_InternFromString()
- PyUnicode_InternInPlace()
- PyUnicode_IsIdentifier()
- PyUnicode_Join()
- PyUnicode_Partition()
- PyUnicode_RPartition()
- PyUnicode_RSplit()
- PyUnicode_ReadChar()
- PyUnicode_Replace()
- PyUnicode_Resize()
- PyUnicode_RichCompare()
- PyUnicode_Split()
- PyUnicode_Splitlines()
- PyUnicode_Substring()
- PyUnicode_Tailmatch()
- PyUnicode_Translate()

- PyUnicode_Type
- PyUnicode_WriteChar()
- PyVarObject
- PyVarObject.ob_base
- PyVarObject.ob_size
- PyVectorcall_Call()
- PyVectorcall_NARGS()
- PyWeakReference
- PyWeakref_GetRef()
- PyWeakref_NewProxy()
- PyWeakref_NewRef()
- PyWrapperDescr_Type
- PyWrapper_New()
- PyZip_Type
- Py_ASNATIVEBYTES_ALLOW_INDEX
- Py_ASNATIVEBYTES_BIG_ENDIAN
- Py_ASNATIVEBYTES_DEFAULTS
- Py_ASNATIVEBYTES_LITTLE_ENDIAN
- Py_ASNATIVEBYTES_NATIVE_ENDIAN
- Py_ASNATIVEBYTES_REJECT_NEGATIVE
- Py_ASNATIVEBYTES_UNSIGNED_BUFFER
- Py_AUDIT_READ
- Py_AddPendingCall()
- Py_AtExit()
- Py_BEGIN_ALLOW_THREADS
- Py_BEGIN_CRITICAL_SECTION
- Py_BEGIN_CRITICAL_SECTION2
- Py_BLOCK_THREADS
- Py_BuildValue()
- Py_BytesMain()
- Py_CompileString()
- Py_DecRef()
- Py_DecodeLocale()
- Py_END_ALLOW_THREADS
- Py_END_CRITICAL_SECTION
- Py_END_CRITICAL_SECTION2
- Py_EncodeLocale()

- Py_EndInterpreter()
- Py_EnterRecursiveCall()
- Py_Exit()
- Py_FatalError()
- Py_FileSystemDefaultEncodeErrors
- Py_FileSystemDefaultEncoding
- Py_Finalize()
- Py_FinalizeEx()
- Py_GenericAlias()
- Py_GenericAliasType
- Py_GetBuildInfo()
- Py_GetCompiler()
- Py_GetConstant()
- Py_GetConstantBorrowed()
- Py_GetCopyright()
- Py_GetPlatform()
- Py_GetRecursionLimit()
- Py_GetVersion()
- Py_HasFileSystemDefaultEncoding
- Py_IS_TYPE()
- Py_IncRef()
- Py_Initialize()
- Py_InitializeEx()
- Py_Is()
- Py_IsFalse()
- Py_IsFinalizing()
- Py_IsInitialized()
- Py_IsNone()
- Py_IsTrue()
- Py_LeaveRecursiveCall()
- Py_Main()
- Py_MakePendingCalls()
- Py_NewInterpreter()
- Py_NewRef()
- Py_PACK_FULL_VERSION()
- Py_PACK_VERSION()
- Py_READONLY

- Py_REFCNT()
- Py_RELATIVE_OFFSET
- Py_ReprEnter()
- Py_ReprLeave()
- Py_SET_SIZE()
- Py_SIZE()
- Py_SetProgramName()
- Py_SetPythonHome()
- Py_SetRecursionLimit()
- Py_TPFLAGS_BASETYPE
- Py_TPFLAGS_DEFAULT
- Py_TPFLAGS_HAVE_GC
- Py_TPFLAGS_HAVE_VECTORCALL
- Py_TPFLAGS_ITEMS_AT_END
- Py_TPFLAGS_METHOD_DESCRIPTOR
- Py_TP_USE_SPEC
- Py_TYPE()
- Py_T_BOOL
- Py_T_BYTE
- Py_T_CHAR
- Py_T_DOUBLE
- Py_T_FLOAT
- Py_T_INT
- Py_T_LONG
- Py_T_LONGLONG
- Py_T_OBJECT_EX
- Py_T_PYSSIZET
- Py_T_SHORT
- Py_T_STRING
- Py_T_STRING_INPLACE
- Py_T_UBYTE
- Py_T_UINT
- Py_T_ULONG
- Py_T_ULONGLONG
- Py_T_USHORT
- Py_UCS4
- Py_UNBLOCK_THREADS

- Py_UTF8Mode
- Py_VaBuildValue()
- Py_Version
- Py_XNewRef()
- Py_am_aiter
- Py_am_anext
- Py_am_await
- Py_am_send
- Py_bf_getbuffer
- Py_bf_releasebuffer
- Py_buffer
- Py_intptr_t
- Py_mod_abi
- Py_mod_create
- Py_mod_doc
- Py_mod_exec
- Py_mod_gil
- Py_mod_methods
- Py_mod_multiple_interpreters
- Py_mod_name
- Py_mod_slots
- Py_mod_state_clear
- Py_mod_state_free
- Py_mod_state_size
- Py_mod_state_traverse
- Py_mod_token
- Py_mp_ass_subscript
- Py_mp_length
- Py_mp_subscript
- Py_nb_absolute
- Py_nb_add
- Py_nb_and
- Py_nb_bool
- Py_nb_divmod
- Py_nb_float
- Py_nb_floor_divide
- Py_nb_index

- Py_nb_inplace_add
- Py_nb_inplace_and
- Py_nb_inplace_floor_divide
- Py_nb_inplace_lshift
- Py_nb_inplace_matrix_multiply
- Py_nb_inplace_multiply
- Py_nb_inplace_or
- Py_nb_inplace_power
- Py_nb_inplace_remainder
- Py_nb_inplace_rshift
- Py_nb_inplace_subtract
- Py_nb_inplace_true_divide
- Py_nb_inplace_xor
- Py_nb_int
- Py_nb_invert
- Py_nb_lshift
- Py_nb_matrix_multiply
- Py_nb_multiply
- Py_nb_negative
- Py_nb_or
- Py_nb_positive
- Py_nb_power
- Py_nb_remainder
- Py_nb_rshift
- Py_nb_subtract
- Py_nb_true_divide
- Py_nb_xor
- Py_slot_end
- Py_slot_invalid
- Py_slot_subslots
- Py_sq_ass_item
- Py_sq_concat
- Py_sq_contains
- Py_sq_inplace_concat
- Py_sq_inplace_repeat
- Py_sq_item
- Py_sq_length

- Py_sq_repeat
- Py_ssize_t
- Py_tp_alloc
- Py_tp_base
- Py_tp_bases
- Py_tp_basicsize
- Py_tp_call
- Py_tp_clear
- Py_tp_dealloc
- Py_tp_del
- Py_tp_descr_get
- Py_tp_descr_set
- Py_tp_doc
- Py_tp_extra_basicsize
- Py_tp_finalize
- Py_tp_flags
- Py_tp_free
- Py_tp_getattr
- Py_tp_getattro
- Py_tp_getset
- Py_tp_hash
- Py_tp_init
- Py_tp_is_gc
- Py_tp_itemsize
- Py_tp_iter
- Py_tp_iternext
- Py_tp_members
- Py_tp_metaclass
- Py_tp_methods
- Py_tp_module
- Py_tp_name
- Py_tp_new
- Py_tp_repr
- Py_tp_richcompare
- Py_tp_setattr
- Py_tp_setattro
- Py_tp_slots

- Py_tp_str
- Py_tp_token
- Py_tp_traverse
- Py_tp_vectorcall
- Py_uintptr_t
- allocfunc
- binaryfunc
- descrgetfunc
- descrsetfunc
- destructor
- getattrfunc
- getattrofunc
- getbufferproc
- getiterfunc
- getter
- hashfunc
- initproc
- inquiry
- iternextfunc
- lenfunc
- newfunc
- objobjargproc
- objobjproc
- releasebufferproc
- reprfunc
- richcmpfunc
- setattrfunc
- setattrofunc
- setter
- ssizeargfunc
- ssizeobjargproc
- ssizessizeargfunc
- ssizessizeobjargproc
- symtable
- ternaryfunc
- traverseproc
- unaryfunc

- vectorcallfunc
- visitproc

A camada de nível muito alto

As funções neste capítulo permitirão que você execute um código-fonte Python fornecido em um arquivo ou buffer, mas não permitirão que você interaja de forma mais detalhada com o interpretador.

Várias dessas funções aceitam um símbolo de início da gramática como parâmetro. Os símbolos de início disponíveis são `Py_eval_input`, `Py_file_input` e `Py_single_input` e `Py_func_type_input`. Eles são descritos seguindo as funções que os aceitam como parâmetros.

Note também que várias dessas funções aceitam parâmetros FILE*. Um problema específico que precisa ser tratado com cuidado é que a estrutura **`FILE`** para diferentes bibliotecas C pode ser diferente e incompatível. No Windows (pelo menos), é possível que extensões vinculadas dinamicamente usem bibliotecas diferentes, então deve-se tomar cuidado para que os parâmetros FILE* sejam passados para essas funções somente se for certo que elas foram criadas pela mesma biblioteca que o tempo de execução do Python está usando.

int PyRun_AnyFile(FILE *fp, const char *filename)

Esta é uma interface simplificada para `PyRun_AnyFileExFlags()` abaixo, deixando *closeit* definido como `0` e *flags* definido como `NULL`.

int PyRun_AnyFileFlags(FILE *fp, const char *filename, PyCompilerFlags *flags)

Esta é uma interface simplificada para `PyRun_AnyFileExFlags()` abaixo, deixando o argumento *closeit* definido como `0`.

int PyRun_AnyFileEx(FILE *fp, const char *filename, int closeit)

Esta é uma interface simplificada para `PyRun_AnyFileExFlags()` abaixo, deixando o argumento *flags* definido como `NULL`.

int PyRun_AnyFileExFlags(FILE *fp, const char *filename, int closeit, PyCompilerFlags *flags)

Se *fp* se referir a um arquivo associado a um dispositivo interativo (entrada de console ou terminal ou pseudo-terminal Unix), retorna o valor de **PyRun_InteractiveLoop()**, caso contrário, retorna o resultado de **PyRun_SimpleFile()**. *filename* é decodificado da codificação do sistema de arquivos (**sys.getfilesystemencoding()**). Se *filename* for `NULL`, esta função usa `"???"` como o nome do arquivo. Se *closeit* for verdadeiro, o arquivo será fechado antes de `PyRun_SimpleFileExFlags()` retornar.

int PyRun_SimpleString(const char *command)

Esta é uma interface simplificada para **PyRun_SimpleStringFlags()** abaixo, deixando o argumento **PyCompilerFlags*** definido como `NULL`.

int PyRun_SimpleStringFlags(const char *command, PyCompilerFlags *flags)

Executa o código-fonte Python de *command* no módulo **__main__** de acordo com o argumento *flags*. Se **__main__** ainda não existir, ele será criado. Retorna `0` em caso de sucesso ou `-1` se uma exceção foi gerada. Se houve um erro, não há como obter as informações da exceção. Para o significado de *flags*, veja abaixo.

Observe que se uma exceção **SystemExit** não tratada for levantada, esta função não retornará `-1`, mas sairá do processo, desde que **PyConfig.inspect** seja zero.

int PyRun_SimpleFile(FILE *fp, const char *filename)

Esta é uma interface simplificada para **PyRun_SimpleFileExFlags()** abaixo, deixando *closeit* definido como `0` e *flags* definido como `NULL`.

int PyRun_SimpleFileEx(FILE *fp, const char *filename, int closeit)

Esta é uma interface simplificada para **PyRun_SimpleFileExFlags()** abaixo, deixando o *flags* definido como `NULL`.

int PyRun_SimpleFileExFlags(FILE *fp, const char *filename, int closeit, PyCompilerFlags *flags)

Semelhante a **PyRun_SimpleStringFlags()**, mas o código-fonte Python é lido de *fp* em vez de uma string na memória. *filename* deve ser o nome do arquivo, ele é decodificado a partir do tratador de erros e codificação do sistema de arquivos. Se *closeit* for true, o arquivo será fechado antes que `PyRun_SimpleFileExFlags()` retorne.

Nota

No Windows, *fp* deve ser aberto como modo binário (por exemplo, `fopen(filename, "rb")`). Caso contrário, o Python pode não manipular corretamente o arquivo de script com final de linha LF.

int PyRun_InteractiveOneObject(FILE *fp, PyObject *filename, PyCompilerFlags *flags)

Lê e executa uma única instrução de um arquivo associado a um dispositivo interativo de acordo com o argumento *flags*. O usuário será solicitado usando `sys.ps1` e `sys.ps2`. *filename* deve ser um objeto **str** Python.

Retorna `0` quando a entrada foi executada com sucesso, `-1` se houve uma exceção ou um código de erro do arquivo de inclusão `errcode.h` distribuído como parte do Python se houve um erro de análise. (Observe que `errcode.h` não é incluído por `Python.h`, então deve ser incluído especificamente se necessário.)

int PyRun_InteractiveOne(FILE *fp, const char *filename)

Esta é uma interface simplificada para **PyRun_InteractiveOneFlags()** abaixo, deixando *flags* definido como `NULL`.

int PyRun_InteractiveOneFlags(FILE *fp, const char *filename, PyCompilerFlags *flags)

Semelhante a **PyRun_InteractiveOneObject()**, mas *filename* é um const char*, que é decodificado a partir do tratador de erros e

codificação do sistema de arquivos.

int PyRun_InteractiveLoop(FILE *fp, const char *filename)

Esta é uma interface simplificada para **`PyRun_InteractiveLoopFlags()`** abaixo, deixando *flags* definido como `NULL`.

int PyRun_InteractiveLoopFlags(FILE *fp, const char *filename, PyCompilerFlags *flags)

Lê e executa instruções de um arquivo associado a um dispositivo interativo até que o EOF seja atingido. O usuário será consultado usando `sys.ps1` e `sys.ps2`. *filename* é decodificado a partir do tratador de erros e codificação do sistema de arquivos. Retorna `0` no EOF ou um número negativo em caso de falha.

int (*PyOS_InputHook)(void)

Parte da ABI Estável.

Pode ser definido para apontar para uma função com o protótipo `int func(void)`. A função será chamada quando o prompt do interpretador do Python estiver prestes a ficar ocioso e aguardar a entrada do usuário no terminal. O valor de retorno é ignorado. A substituição deste hook pode ser usada para integrar o prompt do interpretador com outros laços de eventos, como feito em `Modules/_tkinter.c` no código-fonte do Python.

Alterado na versão 3.12: Esta função só é chamada pelo interpretador principal.

char *(*PyOS_ReadlineFunctionPointer)(FILE*, FILE*, const char*)

Pode ser definido para apontar para uma função com o protótipo `char *func(FILE *stdin, FILE *stdout, char *prompt)`, substituindo a função padrão usada para ler uma única linha de entrada no prompt do interpretador. Espera-se que a função produza a string *prompt* se não for `NULL` e, em seguida, leia uma linha de entrada do arquivo de entrada padrão fornecido, retornando a string resultante. Por exemplo, o módulo **`readline`** define este gancho para fornecer recursos de edição de linha e preenchimento de tabulação.

O resultado deve ser uma string alocada por `PyMem_RawMalloc()` ou `PyMem_RawRealloc()`, ou `NULL` se ocorrer um erro.

Alterado na versão 3.4: O resultado deve ser alocado por `PyMem_RawMalloc()` ou `PyMem_RawRealloc()`, em vez de ser alocado por `PyMem_Malloc()` ou `PyMem_Realloc()`.

Alterado na versão 3.12: Esta função só é chamada pelo interpretador principal.

PyObject *PyRun_String(const char *str, int start, PyObject *globals, PyObject *locals)

Retorna valor: Nova referência.
Esta é uma interface simplificada para `PyRun_StringFlags()` abaixo, deixando *flags* definido como `NULL`.

PyObject *PyRun_StringFlags(const char *str, int start, PyObject *globals, PyObject *locals, PyCompilerFlags *flags)

Retorna valor: Nova referência.
Execute Python source code from *str* in the context specified by the objects *globals* and *locals* with the compiler flags specified by *flags*. *globals* must be a dictionary; *locals* can be any object that implements the mapping protocol. The parameter *start* specifies the start symbol and must be one of the available start symbols.

Retorna o resultado da execução do código como um objeto Python, ou `NULL` se uma exceção foi levantada.

PyObject *PyRun_File(FILE *fp, const char *filename, int start, PyObject *globals, PyObject *locals)

Retorna valor: Nova referência.
Esta é uma interface simplificada para `PyRun_FileExFlags()` abaixo, deixando *closeit* definido como `0` e *flags* definido como `NULL`.

PyObject *PyRun_FileEx(FILE *fp, const char *filename, int start, PyObject *globals, PyObject *locals, int closeit)

Retorna valor: Nova referência.
Esta é uma interface simplificada para **`PyRun_FileExFlags()`** abaixo, deixando *flags* definido como `NULL`.

PyObject *PyRun_FileFlags(FILE *fp, const char *filename, int start, PyObject *globals, PyObject *locals, PyCompilerFlags *flags)

Retorna valor: Nova referência.
Esta é uma interface simplificada para **`PyRun_FileExFlags()`** abaixo, deixando *closeit* definido como `0`.

PyObject *PyRun_FileExFlags(FILE *fp, const char *filename, int start, PyObject *globals, PyObject *locals, int closeit, PyCompilerFlags *flags)

Retorna valor: Nova referência.
Semelhante a **`PyRun_StringFlags()`**, mas o código-fonte Python é lido de *fp* em vez de uma string na memória. *filename* deve ser o nome do arquivo, ele é decodificado a partir do tratador de erros e codificação do sistema de arquivos. Se *closeit* for true, o arquivo será fechado antes que **`PyRun_FileExFlags()`** retorne.

PyObject *Py_CompileString(const char *str, const char *filename, int start)

Retorna valor: Nova referência. Parte da ABI Estável.
Esta é uma interface simplificada para **`Py_CompileStringFlags()`** abaixo, deixando *flags* definido como `NULL`.

PyObject *Py_CompileStringFlags(const char *str, const char *filename, int start, PyCompilerFlags *flags)

Retorna valor: Nova referência.
Esta é uma interface simplificada para **`Py_CompileStringExFlags()`** abaixo, com *optimize* definido como `-1`.

PyObject *Py_CompileStringObject(const char *str, PyObject *filename, int start, PyCompilerFlags *flags, int optimize)

Retorna valor: Nova referência.

Analisa e compila o código-fonte Python em *str*, retornando o objeto de código resultante. O símbolo inicial é fornecido por *start*; isso pode ser usado para restringir o código que pode ser compilado e deve ser símbolos iniciais disponíveis. O nome do arquivo especificado por *filename* é usado para construir o objeto de código e pode aparecer em tracebacks ou mensagens de exceção **`SyntaxError`**. Isso retorna `NULL` se o código não puder ser analisado ou compilado.

O inteiro *optimize* especifica o nível de otimização do compilador; um valor de `-1` seleciona o nível de otimização do interpretador dado pela opção **`-O`**. Níveis explícitos são `0` (nenhuma otimização; `__debug__` é verdadeiro), `1` (instruções `assert` são removidas, `__debug__` é falso) ou `2` (strings de documentação também são removidas).

Adicionado na versão 3.4.

PyObject *Py_CompileStringExFlags(const char *str, const char *filename, int start, PyCompilerFlags *flags, int optimize)

Retorna valor: Nova referência.

Como **`Py_CompileStringObject()`**, mas *filename* é uma string de bytes decodificada a partir do tratador de erros e codificação do sistema de arquivos.

Adicionado na versão 3.2.

PyObject *PyEval_EvalCode(PyObject *co, PyObject *globals, PyObject *locals)

Retorna valor: Nova referência. Parte da ABI Estável.

Esta é uma interface simplificada para **`PyEval_EvalCodeEx()`**, com apenas o objeto código e variáveis locais e globais. Os outros argumentos são definidos como `NULL`.

PyObject *PyEval_EvalCodeEx(PyObject *co, PyObject *globals, PyObject *locals, PyObject *const *args, int argcount, PyObject *const *kws, int kwcount, PyObject *const *defs, int defcount, PyObject *kwdefs, PyObject *closure)

Retorna valor: Nova referência. Parte da ABI Estável.
Avalia um objeto código pré-compilado, dado um ambiente particular para sua avaliação. Este ambiente consiste em um dicionário de variáveis globais, um objeto mapeamento de variáveis locais, vetores de argumentos, nomeados e padrões, um dicionário de valores padrões para argumentos somente-nomeados e uma tupla de clausura de células.

PyObject *PyEval_EvalFrame(PyFrameObject *f)

Retorna valor: Nova referência. Parte da ABI Estável.
Avalia um quadro de execução. Esta é uma interface simplificada para **`PyEval_EvalFrameEx()`**, para retrocompatibilidade.

PyObject *PyEval_EvalFrameEx(PyFrameObject *f, int throwflag)

Retorna valor: Nova referência. Parte da ABI Estável.
Esta é a função principal e sem retoques da interpretação Python. O objeto código associado ao quadro de execução *f* é executado, interpretando bytecode e executando chamadas conforme necessário. O parâmetro adicional *throwflag* pode ser ignorado na maioria das vezes - se verdadeiro, ele faz com que uma exceção seja levantada imediatamente; isso é usado para os métodos **`throw()`** de objetos geradores.

Alterado na versão 3.4: Essa função agora inclui uma asserção de depuração para ajudar a garantir que ela não descarte silenciosamente uma exceção ativa.

int PyEval_MergeCompilerFlags(PyCompilerFlags *cf)

Esta função altera os sinalizadores do quadro de avaliação atual e retorna verdadeiro em caso de sucesso e falso em caso de falha.

struct PyCompilerFlags

Esta é a estrutura usada para manter os sinalizadores do compilador. Em casos onde o código está apenas sendo compilado, ele é passado como `int flags`, e em casos onde o código está sendo executado, ele é passado como `PyCompilerFlags *flags`. Neste caso, `from __future__ import` pode modificar *flags*.

Sempre que `PyCompilerFlags *flags` for `NULL`, **`cf_flags`** é tratado como igual a `0`, e qualquer modificação devido a `from __future__ import` é descartada.

int cf_flags
: Sinalizadores do compilador.

int cf_feature_version
: *cf_feature_version* é a versão secundária do Python. Deve ser inicializada como `PY_MINOR_VERSION`.

  O campo é ignorado por padrão, ele é usado se e somente se o sinalizador `PyCF_ONLY_AST` estiver definido em **`cf_flags`**.

Alterado na versão 3.8: Adicionado campo *cf_feature_version*.

Os sinalizadores do compilador disponíveis são acessíveis como macros:

PyCF_ALLOW_TOP_LEVEL_AWAIT
PyCF_ONLY_AST
PyCF_OPTIMIZED_AST
PyCF_TYPE_COMMENTS
: Veja sinalizadores do compilador na documentação do módulo Python **`ast`**, que exporta essas constantes com os mesmos nomes.

Low-level flags

The following flags and masks serve narrow needs of the standard library and interactive interpreters. Code outside the standard library rarely has a reason to use them. They are considered implementation details and may change at any time.

PyCF_ALLOW_INCOMPLETE_INPUT
: This flag is a private interface between the compiler and the **`codeop`** module. Do not use it; its behavior is unsupported and may change without warning.

With this flag set, when compilation fails because the source text ends where more input is expected, for example in the middle of an indented block or an unterminated string literal, the error raised is the undocumented `_IncompleteInputError`, a subclass of **`SyntaxError`**. The **`codeop`** module sets this flag, together with **`PyCF_DONT_IMPLY_DEDENT`**, to tell input that is incomplete apart from input with a real syntax error, so that interactive interpreters know when to prompt for another line instead of reporting an error.

Adicionado na versão 3.11.

PyCF_DONT_IMPLY_DEDENT

By default, when compiling with the **`Py_single_input`** start symbol, reaching the end of the source text implicitly closes any open indented blocks. With this flag set, open blocks are only closed if the last line of the source ends with a newline; otherwise, compilation fails with a **`SyntaxError`**:

```
PyCompilerFlags flags = {
    .cf_flags = 0,
    .cf_feature_version = PY_MINOR_VERSION,
};
const char *source = "if a:\n    pass";

/* The "if" block is closed implicitly;
   this returns a code object: */
Py_CompileStringFlags(source, "<input>", Py_single_input,

/* With the flag, this fails with a SyntaxError,
   because the last line does not end with a newline: */
flags.cf_flags = PyCF_DONT_IMPLY_DEDENT;
Py_CompileStringFlags(source, "<input>", Py_single_input,
```

The **`codeop`** module uses this flag to detect incomplete interactive input. While the user is still typing inside an indented block, the source does not yet end with a newline, so it fails to compile and the user is prompted for another line.

PyCF_IGNORE_COOKIE

Read the source text as UTF-8, ignoring its **PEP 263** [https://peps.python.org/pep-0263/] encoding declaration ("coding cookie"), if any:

```
PyCompilerFlags flags = {
    .cf_flags = 0,
    .cf_feature_version = PY_MINOR_VERSION,
};
const char *source = "# coding: latin-1\ns = '\xe9'\n";

/* The coding cookie is honored: byte 0xE9 is decoded as
   Latin-1, and this returns a code object that sets s to
Py_CompileStringFlags(source, "<input>", Py_file_input, &f

/* With the flag, the cookie is ignored and compilation fa
   with a SyntaxError, because 0xE9 is not valid UTF-8: */
flags.cf_flags = PyCF_IGNORE_COOKIE;
Py_CompileStringFlags(source, "<input>", Py_file_input, &f
```

The `compile()`, `eval()` and `exec()` built-in functions set this flag when the source is a `str` object, because they pass the text to the parser encoded as UTF-8.

PyCF_SOURCE_IS_UTF8

Mark the source text as known to be UTF-8 encoded. The `compile()`, `eval()` and `exec()` built-in functions set this flag, but it currently has no effect.

Os sinalizadores "`PyCF`" acima podem ser combinados com sinalizadores "`CO_FUTURE`", como `CO_FUTURE_ANNOTATIONS`, para habilitar recursos normalmente selecionáveis usando instruções future. Consulte Sinalizadores de objetos código para obter uma lista completa.

The following masks combine several flags:

PyCF_MASK

Bitmask of all `CO_FUTURE` flags (see Sinalizadores de objetos código), which select features normally enabled by future statements. When code compiled with a `PyCompilerFlags *flags`

argument contains a `from __future__ import` statement, the flag for the imported feature is added to *flags*, so that code executed later in the same context inherits it.

PyCF_MASK_OBSOLETE

Do not use this mask in new code. It is kept only so that old code passing its flags to `compile()` keeps working.

Bitmask of flags for obsolete future features that no longer have any effect.

PyCF_COMPILE_MASK

Bitmask of all `PyCF` flags that change how the source is compiled, such as `PyCF_ONLY_AST`. The `compile()` built-in function uses this mask to validate its *flags* argument.

Símbolos iniciais disponíveis

int Py_eval_input

O símbolo inicial da gramática Python para expressões isoladas; para uso com `Py_CompileString()`.

int Py_file_input

O símbolo de início da gramática Python para sequências de instruções conforme lidas de um arquivo ou outra fonte; para uso com `Py_CompileString()`. Este é o símbolo a ser usado ao compilar código-fonte Python arbitrariamente longo.

int Py_single_input

O símbolo de início da gramática Python para uma única declaração; para uso com `Py_CompileString()`. Este é o símbolo usado para o laço do interpretador interativo.

int Py_func_type_input

O símbolo inicial da gramática Python para um tipo de função; para uso com `Py_CompileString()`. É usado para analisar “comentários de tipo de assinatura” de **PEP 484** [https://peps.python.org/pep-0484/].

Isso requer que o sinalizador `PyCF_ONLY_AST` seja definido.

Ver também

- `ast.FunctionType`
- **PEP 484** [https://peps.python.org/pep-0484/]

Adicionado na versão 3.8.

Stack Effects

Ver também

`dis.stack_effect()`

PY_INVALID_STACK_EFFECT

Sentinel value representing an invalid stack effect.

This is currently equivalent to `INT_MAX`.

Adicionado na versão 3.8.

int PyCompile_OpcodeStackEffect(int opcode, int oparg)

Calcula o efeito que o *opcode* com argumento *oparg* tem na pilha.

On success, this function returns the stack effect; on failure, this returns `PY_INVALID_STACK_EFFECT`.

Adicionado na versão 3.4.

int PyCompile_OpcodeStackEffectWithJump(int opcode, int oparg, int jump)

Similar to `PyCompile_OpcodeStackEffect()`, but don't include the stack effect of jumping if *jump* is zero.

If *jump* is `0`, this will not include the stack effect of jumping, but if *jump* is `1` or `-1`, this will include it.

On success, this function returns the stack effect; on failure, this returns `PY_INVALID_STACK_EFFECT`.

Adicionado na versão 3.8.

Contagem de referências

As funções e macros nesta sessão são usadas para o gerenciamento de contagem de referências de objetos Python.

Py_ssize_t Py_REFCNT(PyObject *o)

Parte da ABI Estável desde a versão 3.14.
Obtêm a contagem de referências do objeto Python *o*.

Observe que o valor retornado pode não refletir realmente quantas referências ao objeto são realmente mantidas. Por exemplo, alguns objetos são imortais e têm uma refcount muito alta que não reflete o número real de referências. Consequentemente, não confie no valor retornado para ser preciso, exceto um valor de 0 ou 1.

Use a função `Py_SET_REFCNT()` para definir uma contagem de referências de um objeto.

Nota

On free-threaded builds of Python, returning 1 isn't sufficient to determine if it's safe to treat *o* as having no access by other threads. Use `PyUnstable_Object_IsUniquelyReferenced()` for that instead.

Veja também a função `PyUnstable_Object_IsUniqueReferencedTemporary()`.

Alterado na versão 3.10: `Py_REFCNT()` is changed to the inline static function.

Alterado na versão 3.11: The parameter type is no longer const PyObject*.

void Py_SET_REFCNT(PyObject *o, Py_ssize_t refcnt)

Set the object *o* reference counter to *refcnt*.

Em uma construção do Python com threads threads livres, se *refcnt* for maior que `UINT32_MAX`, o objeto será transformado em imortal.

Esta função não tem efeito sobre objetos imortais.

Adicionado na versão 3.9.

Alterado na versão 3.12: Os objetos imortais não são modificados.

void Py_INCREF(PyObject *o)

Indicate taking a new strong reference to object *o*, indicating it is in use and should not be destroyed.

Esta função não tem efeito sobre objetos imortais.

Esta função é geralmente usada para transformar uma referência emprestada em uma referência forte local. A função **`Py_NewRef()`** pode ser usada para criar uma nova referência forte.

When done using the object, release is by calling **`Py_DECREF()`**.

The object must not be `NULL`; if you aren't sure that it isn't `NULL`, use **`Py_XINCREF()`**.

Do not expect this function to actually modify *o* in any way. For at least **some objects** [https://peps.python.org/pep-0683/], this function has no effect.

Alterado na versão 3.12: Os objetos imortais não são modificados.

void Py_XINCREF(PyObject *o)

Semelhante a **`Py_INCREF()`**, mas o objeto *o* pode ser `NULL`, caso em que isso não tem efeito.

Veja também **`Py_XNewRef()`**.

PyObject *Py_NewRef(PyObject *o)

Parte da ABI Estável desde a versão 3.10.

Create a new strong reference to an object: call **Py_INCREF()** on *o* and return the object *o*.

When the strong reference is no longer needed, **Py_DECREF()** should be called on it to release the reference.

The object *o* must not be `NULL`; use **Py_XNewRef()** if *o* can be `NULL`.

Por exemplo:

```
Py_INCREF(obj);
self->attr = obj;
```

can be written as:

```
self->attr = Py_NewRef(obj);
```

See also **Py_INCREF()**.

Adicionado na versão 3.10.

PyObject *Py_XNewRef(PyObject *o)

Parte da ABI Estável desde a versão 3.10.
Semelhante a **Py_NewRef()**, mas o objeto *o* pode ser NULL.

Se o objeto *o* é `NULL`, a função apenas retorna `NULL`.

Adicionado na versão 3.10.

void Py_DECREF(PyObject *o)

Release a strong reference to object *o*, indicating the reference is no longer used.

Esta função não tem efeito sobre objetos imortais.

Once the last strong reference is released (i.e. the object's reference count reaches 0), the object's type's deallocation function (which must not be `NULL`) is invoked.

This function is usually used to delete a strong reference before exiting its scope.

The object must not be `NULL`; if you aren't sure that it isn't `NULL`, use `Py_XDECREF()`.

Do not expect this function to actually modify *o* in any way. For at least **some objects** [https://peps.python.org/pep-0683/], this function has no effect.

Aviso

The deallocation function can cause arbitrary Python code to be invoked (e.g. when a class instance with a `__del__()` method is deallocated). While exceptions in such code are not propagated, the executed code has free access to all Python global variables. This means that any object that is reachable from a global variable should be in a consistent state before `Py_DECREF()` is invoked. For example, code to delete an object from a list should copy a reference to the deleted object in a temporary variable, update the list data structure, and then call `Py_DECREF()` for the temporary variable.

Alterado na versão 3.12: Os objetos imortais não são modificados.

void Py_XDECREF(PyObject *o)

Similar to `Py_DECREF()`, but the object *o* can be `NULL`, in which case this has no effect. The same warning from `Py_DECREF()` applies here as well.

void Py_CLEAR(PyObject *o)

Release a strong reference for object *o*. The object may be `NULL`, in which case the macro has no effect; otherwise the effect is the same as for `Py_DECREF()`, except that the argument is also set to `NULL`. The warning for `Py_DECREF()` does not apply with respect to the object passed because the macro carefully uses a temporary variable and sets the argument to `NULL` before releasing the reference.

It is a good idea to use this macro whenever releasing a reference to an object that might be traversed during garbage collection.

Alterado na versão 3.12: The macro argument is now only evaluated once. If the argument has side effects, these are no longer duplicated.

void Py_IncRef(PyObject *o)

Parte da ABI Estável.
Indicate taking a new strong reference to object *o*. A function version of **Py_XINCREF()**. It can be used for runtime dynamic embedding of Python.

void Py_DecRef(PyObject *o)

Parte da ABI Estável.
Release a strong reference to object *o*. A function version of **Py_XDECREF()**. It can be used for runtime dynamic embedding of Python.

Py_SETREF(dst, src)

Macro safely releasing a strong reference to object *dst* and setting *dst* to *src*.

As in case of **Py_CLEAR()**, “the obvious” code can be deadly:

```
Py_DECREF(dst);
dst = src;
```

The safe way is:

```
Py_SETREF(dst, src);
```

That arranges to set *dst* to *src before* releasing the reference to the old value of *dst*, so that any code triggered as a side-effect of *dst* getting torn down no longer believes *dst* points to a valid object.

Adicionado na versão 3.6.

Alterado na versão 3.12: The macro arguments are now only evaluated once. If an argument has side effects, these are no longer duplicated.

Py_XSETREF(dst, src)

Variant of `Py_SETREF` macro that uses `Py_XDECREF()` instead of `Py_DECREF()`.

Adicionado na versão 3.6.

Alterado na versão 3.12: The macro arguments are now only evaluated once. If an argument has side effects, these are no longer duplicated.

Tratamento de exceções

As funções descritas nesse capítulo permitem você tratar e gerar exceções em Python. É importante entender alguns princípios básicos no tratamento de exceções no Python. Funciona de forma parecida com a variável POSIX **`errno`**: existe um indicador global (por thread) do último erro ocorrido. A maioria das funções da API C não o zera ao obter sucesso, mas o define para indicar a causa do erro ao falhar. A maioria das funções da API retorna um indicador de erro, geralmente, `NULL` se eles devem retornar um ponteiro, ou `-1` se retornarem um inteiro (exceção: as funções `PyArg_*` retornam `1` para sucesso e `0` para falha).

De forma concreta, o indicador de erro consiste em três ponteiros de objeto: o tipo da exceção, o valor da exceção e o objeto de traceback. Qualquer um desses ponteiros pode ser `NULL` se não definido (embora algumas combinações sejam proibidas, por exemplo, você não pode ter um retorno não `NULL` se o tipo de exceção for `NULL`).

Quando uma função deve falhar porque devido à falha de alguma função que ela chamou, ela geralmente não define o indicador de erro; a função que ela chamou já o definiu. Ela é responsável por manipular o erro e limpar a exceção ou retornar após limpar todos os recursos que possui (como referências a objetos ou alocações de memória); ela *não* deve continuar normalmente se não estiver preparada para lidar com o erro. Se estiver retornando devido a um erro, é importante indicar ao chamador que um erro foi definido. Se o erro não for manipulado ou propagado com cuidado, chamadas adicionais para a API Python/C podem não se comportar conforme o esperado e podem falhar de maneiras misteriosas.

Nota

O indicador de erro **não** é resultado de `sys.exc_info()`. O primeiro corresponde a uma exceção que ainda não foi capturada (e, portanto, ainda está se propagando), enquanto o segundo retorna uma exceção após ser capturada (e, portanto, parou de se propagar).

Impressão e limpeza

void PyErr_Clear()

Parte da ABI Estável.
Limpe o indicador de erro. Se o indicador de erro não estiver definido, não haverá efeito.

void PyErr_PrintEx(int set_sys_last_vars)

Parte da ABI Estável.
Exibe um traceback padrão para `sys.stderr` e limpe o indicador de erro. **A menos que** o erro seja um `SystemExit`, nesse caso, nenhum traceback será impresso e o processo Python será encerrado com o código de erro especificado pela instância `SystemExit`.

Chame esta função **apenas** quando o indicador de erro estiver definido. Caso contrário, causará um erro fatal!

Se *set_sys_last_vars* for diferente de zero, a variável `sys.last_exc` será definida com a exceção exibida. Para manter a retrocompatibilidade, as variáveis descontinuadas `sys.last_type`, `sys.last_value` e `sys.last_traceback` também serão definidas com o tipo, valor e traceback dessa exceção, respectivamente.

Alterado na versão 3.12: A configuração de `sys.last_exc` foi adicionada.

void PyErr_Print()

Parte da ABI Estável.
Apelido para `PyErr_PrintEx(1)`.

void PyErr_WriteUnraisable(PyObject *obj)

Parte da ABI Estável.
Chama `sys.unraisablehook()` usando a exceção atual e o argumento *obj*.

Esta função utilitária imprime uma mensagem de aviso para `sys.stderr` quando uma exceção foi definida, mas é impossível para o interpretador realmente gerar a exceção. É usada, por exemplo, quando ocorre uma exceção em um método **`__del__()`**.

A função é chamada com um único argumento *obj* que identifica o contexto em que ocorreu a exceção não gerável. Se possível, o repr de *obj* será impresso na mensagem de aviso. Se *obj* for `NULL`, apenas o traceback será impresso.

Uma exceção deve ser definida ao chamar essa função.

Alterado na versão 3.4: Exibe o traceback (situação da pilha de execução). Exibe apenas o traceback se *obj* for `NULL`.

Alterado na versão 3.8: Utiliza **`sys.unraisablehook()`**.

void PyErr_FormatUnraisable(const char *format, ...)

Similar a **`PyErr_WriteUnraisable()`**, mas *format* e parâmetros subsequentes ajudam a formatar a mensagem de aviso; eles têm o mesmo significado e valores que em **`PyUnicode_FromFormat()`**. `PyErr_WriteUnraisable(obj)` é aproximadamente equivalente à `PyErr_FormatUnraisable("Exception ignored in: %R", obj)`. Se *format* é `NULL`, somente o rastreamento é impresso.

Adicionado na versão 3.13.

void PyErr_DisplayException(PyObject *exc)

Parte da ABI Estável desde a versão 3.12.
Imprime a exibição padrão do traceback de `exc` em `sys.stderr`, incluindo as exceções encadeadas e notas.

Adicionado na versão 3.12.

void PyErr_Display(PyObject *unused, PyObject *value, PyObject *tb)

Parte da ABI Estável.
Legacy variant of **`PyErr_DisplayException()`**.

Print the exception *value* with its traceback to `sys.stderr`. If *value* has no traceback set, *tb* is used as its traceback. The first argument is ignored.

If `sys.stderr` is `None`, nothing is printed. If `sys.stderr` is not set, the exception is dumped to the C `stderr` stream instead.

Descontinuado desde a versão 3.12: Use `PyErr_DisplayException()` instead.

Levantando exceções

Essas funções ajudam a definir o indicador de erro do thread. Por conveniência, algumas dessas funções sempre retornam um ponteiro `NULL` ao usar instrução com `return`.

void PyErr_SetString(PyObject *type, const char *message)

Parte da ABI Estável.
Esse é o modo mais comum de definir o indicador de erro. O primeiro argumento especifica o tipo da exceção; é normalmente uma das exceções padrão, como `PyExc_RuntimeError`. Você não precisa criar um novo strong reference para ela (como em `Py_INCREF()`). O segundo argumento é uma mensagem de erro; ela é codificada com `'utf-8'`.

void PyErr_SetObject(PyObject *type, PyObject *value)

Parte da ABI Estável.
Essa função é semelhante à `PyErr_SetString()` mas permite especificar um objeto Python arbitrário para o valor da exceção.

PyObject *PyErr_Format(PyObject *exception, const char *format, ...)

Retorna valor: Sempre NULL. Parte da ABI Estável.
Essa função define o indicador de erro e retorna `NULL`. *exception* deve ser uma classe Python de exceção. O *format* e parâmetros subsequentes ajudam a formatar a mensagem de erro; eles têm o mesmo significado e valores que em `PyUnicode_FromFormat()`. *format* é uma string codificada em ASCII.

PyObject *PyErr_FormatV(PyObject *exception, const char *format, va_list vargs)

Retorna valor: Sempre NULL. Parte da ABI Estável desde a versão 3.5.
Igual a **PyErr_Format()**, mas usando o argumento **va_list** em vez de um número variável de argumentos.

Adicionado na versão 3.5.

void PyErr_SetNone(PyObject *type)

Parte da ABI Estável.
Isso é uma abreviação para `PyErr_SetObject(type, Py_None)`.

int PyErr_BadArgument()

Parte da ABI Estável.
Essa é uma abreviação de `PyErr_SetString(PyExc_TypeError, message)`, na qual *message* indica que uma operação embutida foi invocada com um argumento ilegal. Ela é principalmente para uso interno.

PyObject *PyErr_NoMemory()

Retorna valor: Sempre NULL. Parte da ABI Estável.
Essa é uma abreviação para `PyErr_SetNone(PyExc_MemoryError)`; que retorna `NULL` para que uma função de alocação de objeto possa escrever `return PyErr_NoMemory();` quando ficar sem memória.

PyObject *PyErr_SetFromErrno(PyObject *type)

Retorna valor: Sempre NULL. Parte da ABI Estável.
Esta é uma função conveniente para levantar uma exceção quando uma função da biblioteca C retornou um erro e definir a variável C **errno**. Ela constrói um objeto tupla cujo primeiro item é o valor inteiro **errno** e cujo segundo item é a mensagem de erro correspondente (obtida de **strerror()**), e então chama `PyErr_SetObject(type, object)`. No Unix, quando o valor **errno** é **EINTR**, indicando uma chamada de sistema interrompida, isso chama **PyErr_CheckSignals()**, e se isso definir o indicador de erro, deixa-o definido como tal. A função retorna sempre `NULL`, de forma que uma função que envolve uma chamada de

sistema pode retornar `return PyErr_SetFromErrno(type);` quando a chamada de sistema retornar um erro.

PyObject *PyErr_SetFromErrnoWithFilenameObject(PyObject *type, PyObject *filenameObject)

Retorna valor: Sempre NULL. Parte da ABI Estável.

Similar à **`PyErr_SetFromErrno()`**, com o comportamento adicional de que, se *filenameObject* não é `NULL`, ele é passado para o construtor de *type* como um terceiro parâmetro. No caso da exceção **`OSError`**, isso é usado para definir o atributo **`filename`** da instância da exceção.

PyObject *PyErr_SetFromErrnoWithFilenameObjects(PyObject *type, PyObject *filenameObject, PyObject *filenameObject2)

Retorna valor: Sempre NULL. Parte da ABI Estável desde a versão 3.7.

Similar à **`PyErr_SetFromErrnoWithFilenameObject()`**, mas recebe um segundo objeto filename, para levantar erros quando uma função que recebe dois nomes de arquivos falha.

Adicionado na versão 3.4.

PyObject *PyErr_SetFromErrnoWithFilename(PyObject *type, const char *filename)

Retorna valor: Sempre NULL. Parte da ABI Estável.

Similar à **`PyErr_SetFromErrnoWithFilenameObject()`**, mas o nome do arquivo é fornecido como uma string C. *filename* é decodificado do filesystem encoding and error handler.

PyObject *PyErr_SetFromWindowsErr(int ierr)

Retorna valor: Sempre NULL. Parte da ABI Estável on Windows desde a versão 3.7.

Essa é uma função conveniente para levantar **`OSError`**. Se chamada com *ierr* de `0`, o código de erro retornado por uma chamada para **`GetLastError()`** é usado instantaneamente. Ela chama a função Win32 **`FormatMessage()`** para retornar a descrição Windows do código de erro dado por *ierr* ou **`GetLastError()`**, e então constrói um objeto **`OSError`** com o atributo **`winerror`** definido para o código de erro, o atributo

strerror definido para a mensagem de erro correspondente (lida de **FormatMessage()**), e então chama `PyErr_SetObject(PyExc_OSError, object)`. Essa função sempre retorna `NULL`.

Disponibilidade: Windows.

PyObject *PyErr_SetExcFromWindowsErr(PyObject *type, int ierr)

Retorna valor: Sempre NULL. Parte da ABI Estável on Windows desde a versão 3.7.

Semelhante a **PyErr_SetFromWindowsErr()**, com um parâmetro adicional que especifica o tipo de exceção a ser levantada.

Disponibilidade: Windows.

PyObject *PyErr_SetFromWindowsErrWithFilename(int ierr, const char *filename)

Retorna valor: Sempre NULL. Parte da ABI Estável on Windows desde a versão 3.7.

Semelhante a **PyErr_SetFromWindowsErr()**, com o comportamento adicional de que, se *filename* não for `NULL`, ele é decodificado a partir da codificação do sistema de arquivos (**os.fsdecode()**) e passado para o construtor de **OSError** como um terceiro parâmetro a ser usado para definir o atributo **filename** da instância da exceção.

Disponibilidade: Windows.

PyObject *PyErr_SetExcFromWindowsErrWithFilenameObject(PyObject *type, int ierr, PyObject *filename)

Retorna valor: Sempre NULL. Parte da ABI Estável on Windows desde a versão 3.7.

Semelhante a **PyErr_SetExcFromWindowsErr()**, com o comportamento adicional de que, se *filename* não for `NULL`, ele é passado para o construtor de **OSError** como um terceiro parâmetro a ser usado para definir o atributo **filename** da instância da exceção.

Disponibilidade: Windows.

PyObject *PyErr_SetExcFromWindowsErrWithFilenameObjects(PyObject *type, int ierr, PyObject *filename, PyObject *filename2)

Retorna valor: Sempre NULL. Parte da ABI Estável on Windows desde a versão 3.7.

Similar à **`PyErr_SetExcFromWindowsErrWithFilenameObject()`**, mas aceita um segundo caminho do objeto.

Disponibilidade: Windows.

Adicionado na versão 3.4.

PyObject *PyErr_SetExcFromWindowsErrWithFilename(PyObject *type, int ierr, const char *filename)

Retorna valor: Sempre NULL. Parte da ABI Estável on Windows desde a versão 3.7.

Similar à **`PyErr_SetFromWindowsErrWithFilename()`**, com um parâmetro adicional especificando o tipo de exceção a ser gerado.

Disponibilidade: Windows.

PyObject *PyErr_SetImportError(PyObject *msg, PyObject *name, PyObject *path)

Retorna valor: Sempre NULL. Parte da ABI Estável desde a versão 3.7.

Essa é uma função conveniente para levantar **`ImportError`**. *msg* será definido como a string da mensagem de exceção. *name* e *path*, ambos os quais podem ser `NULL`, serão definidos como, respectivamente, os atributos `name` e `path` de **`ImportError`**.

Adicionado na versão 3.3.

PyObject *PyErr_SetImportErrorSubclass(PyObject *exception, PyObject *msg, PyObject *name, PyObject *path)

Retorna valor: Sempre NULL. Parte da ABI Estável desde a versão 3.6.

Muito parecido com **`PyErr_SetImportError()`** mas a função permite especificar uma subclasse de **`ImportError`** para levantar uma exceção.

Adicionado na versão 3.6.

void PyErr_SyntaxLocationObject(PyObject *filename, int lineno, int col_offset)

Define informações de arquivo, linha e deslocamento para a atual exceção. Se a exceção não é **`SyntaxError`**, define atributos adicionais, os quais fazem o subsistema de impressão de exceções pensar que trata-se de um **`SyntaxError`**.

Adicionado na versão 3.4.

void PyErr_RangedSyntaxLocationObject(PyObject *filename, int lineno, int col_offset, int end_lineno, int end_col_offset)

Similar a **`PyErr_SyntaxLocationObject()`**, mas também define as informações *end_lineno* e *end_col_offset* para a exceção atual.

Adicionado na versão 3.10.

void PyErr_SyntaxLocationEx(const char *filename, int lineno, int col_offset)

Parte da ABI Estável desde a versão 3.7.
Como **`PyErr_SyntaxLocationObject()`**, mas *filename* é uma bytestring descodificada a partir do tratador de erros e codificação do sistema de arquivos.

Adicionado na versão 3.2.

void PyErr_SyntaxLocation(const char *filename, int lineno)

Parte da ABI Estável.
Como **`PyErr_SyntaxLocationEx()`**, mas o parâmetro *col_offset* é omitido.

void PyErr_BadInternalCall()

Parte da ABI Estável.
Essa é uma abreviação de `PyErr_SetString(PyExc_SystemError, message)`, onde *message* indica que uma operação interna (por exemplo,

uma função API C/Python) foi invocada com um argumento ilegal. Ela é principalmente para uso interno.

PyObject *PyErr_ProgramTextObject(PyObject *filename, int lineno)

Obtém a linha de código-fonte em *filename* na linha *lineno*. *filename* deve ser um objeto `str` Python.

Em caso de sucesso, esta função retorna um objeto string do Python com a linha encontrada. Em caso de falha, esta função retorna `NULL` sem definir uma exceção.

PyObject *PyErr_ProgramText(const char *filename, int lineno)

Parte da ABI Estável.

Semelhante a `PyErr_ProgramTextObject()`, mas *filename* é um const char*, que é decodificado com tratador de erros e codificação do sistema de arquivos, em vez de uma referência de objeto Python.

Emitindo avisos

Use essas funções para emitir avisos a partir de código C. Elas espelham funções semelhantes exportadas pelo módulo `warnings` do Python. Normalmente, elas exibem uma mensagem de aviso em *sys.stderr*; no entanto, também é possível que o usuário tenha especificado que os avisos devem ser transformados em erros, e nesse caso, uma exceção será lançada. Também é possível que as funções levantem uma exceção devido a um problema com o mecanismo de avisos. O valor de retorno é `0` se nenhuma exceção for lançada, ou `-1` se uma exceção for lançada. (Não é possível determinar se uma mensagem de aviso é realmente impressa, nem qual é o motivo da exceção; isso é intencional.) Se uma exceção for levantada, a função que chamou a função deve realizar seu tratamento de exceções normal (por exemplo, liberar as referências pertencentes a `Py_DECREF()` e retornar um valor de erro).

int PyErr_WarnEx(PyObject *category, const char *message, Py_ssize_t stack_level)

Parte da ABI Estável.

Emite uma mensagem de aviso. O argumento *category* é uma categoria de aviso (veja abaixo) ou `NULL`; o argumento *message* é uma string codificada em UTF-8. *stack_level* é um número positivo que indica o número de quadros da pilha; o aviso será emitido a partir da linha de código atualmente em execução nesse quadro da pilha. Um *stack_level* de 1 é a função que chama **`PyErr_WarnEx()`**, 2 é a função acima dessa e assim por diante.

As categorias de aviso devem ser subclasses de **`PyExc_Warning`**; **`PyExc_Warning`** é uma subclasse de **`PyExc_Exception`**; a categoria de aviso padrão é **`PyExc_RuntimeWarning`**. As categorias de aviso padrão do Python estão disponíveis como variáveis globais cujos nomes estão enumerados em Warning types.

Para obter informações sobre o controle de avisos, consulte a documentação do módulo **`warnings`** e a opção **`-W`** na documentação da linha de comando. Não existe uma API C para o controle de avisos.

int PyErr_WarnExplicitObject(PyObject *category, PyObject *message, PyObject *filename, int lineno, PyObject *module, PyObject *registry)

Emite uma mensagem de aviso com controle explícito sobre todos os atributos de aviso. Trata-se de um invólucro simples em torno da função Python **`warnings.warn_explicit()`**; consulte-a para obter mais informações. Os argumentos *module* e *registry* podem ser definidos como `NULL` para obter o efeito padrão descrito nessa função.

Adicionado na versão 3.4.

int PyErr_WarnExplicit(PyObject *category, const char *message, const char *filename, int lineno, const char *module, PyObject *registry)

Parte da ABI Estável.

Semelhante a **`PyErr_WarnExplicitObject()`**, exceto que *message* e *module* são strings codificadas em UTF-8 e *filename* é decodificado a partir do tratador de erros e codificação do sistema de arquivos.

int PyErr_WarnFormat(PyObject *category, Py_ssize_t stack_level, const char *format, ...)

Parte da ABI Estável.
Function similar to `PyErr_WarnEx()`, but uses `PyUnicode_FromFormat()` to format the warning message. *format* is an ASCII-encoded string.

Adicionado na versão 3.2.

int PyErr_WarnExplicitFormat(PyObject *category, const char *filename, int lineno, const char *module, PyObject *registry, const char *format, ...)

Similar to `PyErr_WarnExplicit()`, but uses `PyUnicode_FromFormat()` to format the warning message. *format* is an ASCII-encoded string.

Adicionado na versão 3.2.

int PyErr_ResourceWarning(PyObject *source, Py_ssize_t stack_level, const char *format, ...)

Parte da ABI Estável desde a versão 3.6.
Função similar a `PyErr_WarnFormat()`, mas a *category* é `ResourceWarning` e ela passa *source* para `warnings.WarningMessage`.

Adicionado na versão 3.6.

Consultando o indicador de erro

PyObject *PyErr_Occurred()

Retorna valor: Referência emprestada. Parte da ABI Estável.
Test whether the error indicator is set. If set, return the exception *type* (the first argument to the last call to one of the `PyErr_Set*` functions or to `PyErr_Restore()`). If not set, return `NULL`. You do not own a reference to the return value, so you do not need to `Py_DECREF()` it.

O chamador deve ter um estado de thread anexado.

Nota

Do not compare the return value to a specific exception; use `PyErr_ExceptionMatches()` instead, shown below. (The comparison

could easily fail since the exception may be an instance instead of a class, in the case of a class exception, or it may be a subclass of the expected exception.)

int PyErr_ExceptionMatches(PyObject *exc)

Parte da ABI Estável.

Equivalent to `PyErr_GivenExceptionMatches(PyErr_Occurred(), exc)`. This should only be called when an exception is actually set; a memory access violation will occur if no exception has been raised.

int PyErr_GivenExceptionMatches(PyObject *given, PyObject *exc)

Parte da ABI Estável.

Return true if the *given* exception matches the exception type in *exc*. If *exc* is a class object, this also returns true when *given* is an instance of a subclass. If *exc* is a tuple, all exception types in the tuple (and recursively in subtuples) are searched for a match.

PyObject *PyErr_GetRaisedException(void)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.12.

Return the exception currently being raised, clearing the error indicator at the same time. Return `NULL` if the error indicator is not set.

This function is used by code that needs to catch exceptions, or code that needs to save and restore the error indicator temporarily.

Por exemplo:

```
{
   PyObject *exc = PyErr_GetRaisedException();

   /* ... code that might produce other errors ... */

   PyErr_SetRaisedException(exc);
}
```

Ver também

`PyErr_GetHandledException()`, to save the exception currently being handled.

Adicionado na versão 3.12.

void PyErr_SetRaisedException(PyObject *exc)

Parte da ABI Estável desde a versão 3.12.

Set *exc* as the exception currently being raised, clearing the existing exception if one is set. If *exc* is `NULL`, just clear the existing exception.

exc must be a valid exception or `NULL`.

This call "steals" a reference to *exc*.

Adicionado na versão 3.12.

void PyErr_Fetch(PyObject **ptype, PyObject **pvalue, PyObject **ptraceback)

Parte da ABI Estável.

Descontinuado desde a versão 3.12: Use `PyErr_GetRaisedException()` em vez disso.

Retrieve the error indicator into three variables whose addresses are passed. If the error indicator is not set, set all three variables to `NULL`. If it is set, it will be cleared and you own a reference to each object retrieved. The value and traceback object may be `NULL` even when the type object is not.

Nota

This function is normally only used by legacy code that needs to catch exceptions or save and restore the error indicator temporarily.

Por exemplo:

```
{
PyObject *type, *value, *traceback;
PyErr_Fetch(&type, &value, &traceback);
```

```
/* ... código que pode reproduzir outros erros ... */

PyErr_Restore(type, value, traceback);
}
```

void PyErr_Restore(PyObject *type, PyObject *value, PyObject *traceback)

Parte da ABI Estável.
Descontinuado desde a versão 3.12: Use **`PyErr_SetRaisedException()`** em vez disso.

Set the error indicator from the three objects, *type*, *value*, and *traceback*, clearing the existing exception if one is set. If the objects are `NULL`, the error indicator is cleared. Do not pass a `NULL` type and non-`NULL` value or traceback. The exception type should be a class. Do not pass an invalid exception type or value. (Violating these rules will cause subtle problems later.) This call takes away a reference to each object: you must own a reference to each object before the call and after the call you no longer own these references. (If you don't understand this, don't use this function. I warned you.)

Nota

This function is normally only used by legacy code that needs to save and restore the error indicator temporarily. Use **`PyErr_Fetch()`** to save the current error indicator.

void PyErr_NormalizeException(PyObject **exc, PyObject **val, PyObject **tb)

Parte da ABI Estável.
Descontinuado desde a versão 3.12: Use **`PyErr_GetRaisedException()`** em vez disso, para evitar qualquer possível desnormalização.

Under certain circumstances, the values returned by **PyErr_Fetch()** below can be “unnormalized”, meaning that `*exc` is a class object but `*val` is not an instance of the same class. This function can be used to instantiate the class in that case. If the values are already normalized, nothing happens. The delayed normalization is implemented to improve performance.

Nota

This function *does not* implicitly set the **__traceback__** attribute on the exception value. If setting the traceback appropriately is desired, the following additional snippet is needed:

```
if (tb != NULL) {
PyException_SetTraceback(val, tb);
}
```

PyObject *PyErr_GetHandledException(void)

Parte da ABI Estável desde a versão 3.11.

Retrieve the active exception instance, as would be returned by **sys.exception()**. This refers to an exception that was *already caught*, not to an exception that was freshly raised. Returns a new reference to the exception or `NULL`. Does not modify the interpreter’s exception state.

Nota

This function is not normally used by code that wants to handle exceptions. Rather, it can be used when code needs to save and restore the exception state temporarily. Use **PyErr_SetHandledException()** to restore or clear the exception state.

Adicionado na versão 3.11.

void PyErr_SetHandledException(PyObject *exc)

Parte da ABI Estável desde a versão 3.11.

Set the active exception, as known from `sys.exception()`. This refers to an exception that was *already caught*, not to an exception that was

freshly raised. To clear the exception state, pass `NULL`.

Nota

This function is not normally used by code that wants to handle exceptions. Rather, it can be used when code needs to save and restore the exception state temporarily. Use **`PyErr_GetHandledException()`** to get the exception state.

Adicionado na versão 3.11.

void PyErr_GetExcInfo(PyObject **ptype, PyObject **pvalue, PyObject **ptraceback)

Parte da ABI Estável desde a versão 3.7.

Retrieve the old-style representation of the exception info, as known from **`sys.exc_info()`**. This refers to an exception that was *already caught*, not to an exception that was freshly raised. Returns new references for the three objects, any of which may be `NULL`. Does not modify the exception info state. This function is kept for backwards compatibility. Prefer using **`PyErr_GetHandledException()`**.

Nota

This function is not normally used by code that wants to handle exceptions. Rather, it can be used when code needs to save and restore the exception state temporarily. Use **`PyErr_SetExcInfo()`** to restore or clear the exception state.

Adicionado na versão 3.3.

void PyErr_SetExcInfo(PyObject *type, PyObject *value, PyObject *traceback)

Parte da ABI Estável desde a versão 3.7.

Set the exception info, as known from `sys.exc_info()`. This refers to an exception that was *already caught*, not to an exception that was freshly raised. This function "steals" the references of the arguments. To

clear the exception state, pass `NULL` for all three arguments. This function is kept for backwards compatibility. Prefer using **`PyErr_SetHandledException()`**.

Nota

This function is not normally used by code that wants to handle exceptions. Rather, it can be used when code needs to save and restore the exception state temporarily. Use **`PyErr_GetExcInfo()`** to read the exception state.

Adicionado na versão 3.3.

Alterado na versão 3.11: The `type` and `traceback` arguments are no longer used and can be NULL. The interpreter now derives them from the exception instance (the `value` argument). The function still "steals" references of all three arguments.

Tratamento de sinal

int PyErr_CheckSignals()

Parte da ABI Estável.

Handle external interruptions, such as signals or activating a debugger, whose processing has been delayed until it is safe to run Python code and/or raise exceptions.

For example, pressing `Ctrl-C` causes a terminal to send the **`signal.SIGINT`** signal. This function executes the corresponding Python signal handler, which, by default, raises the **`KeyboardInterrupt`** exception.

`PyErr_CheckSignals()` should be called by long-running C code frequently enough so that the response appears immediate to humans.

Handlers invoked by this function currently include:

- Signal handlers, including Python functions registered using the `signal` module.

  Signal handlers are only run in the main thread of the main interpreter.

  (This is where the function got the name: originally, signals were the only way to interrupt the interpreter.)

- Running the garbage collector, if necessary.

- Executing a pending remote debugger script.

- Raise the exception set by `PyThreadState_SetAsyncExc()`.

If any handler raises an exception, immediately return `-1` with that exception set. Any remaining interruptions are left to be processed on the next `PyErr_CheckSignals()` invocation, if appropriate.

If all handlers finish successfully, or there are no handlers to run, return `0`.

Alterado na versão 3.12: This function may now invoke the garbage collector.

Alterado na versão 3.14: This function may now execute a remote debugger script, if remote debugging is enabled.

Alterado na versão 3.15: The exception set by `PyThreadState_SetAsyncExc()` is now raised.

void PyErr_SetInterrupt()

Parte da ABI Estável.

Simula o efeito de um sinal `SIGINT` chegando. Isto é equivalente a `PyErr_SetInterruptEx(SIGINT)`.

Nota

This function is async-signal-safe. It can be called without an attached thread state and from a C signal handler.

int PyErr_SetInterruptEx(int signum)

Parte da ABI Estável desde a versão 3.10.

Simulate the effect of a signal arriving. The next time `PyErr_CheckSignals()` is called, the Python signal handler for the given signal number will be called.

This function can be called by C code that sets up its own signal handling and wants Python signal handlers to be invoked as expected when an interruption is requested (for example when the user presses Ctrl-C to interrupt an operation).

Se o sinal fornecido não for tratado pelo Python (foi definido como `signal.SIG_DFL` ou `signal.SIG_IGN`), ele será ignorado.

If *signum* is outside of the allowed range of signal numbers, `-1` is returned. Otherwise, `0` is returned. The error indicator is never changed by this function.

Nota

This function is async-signal-safe. It can be called without an attached thread state and from a C signal handler.

Adicionado na versão 3.10.

int PySignal_SetWakeupFd(int fd)

This utility function specifies a file descriptor to which the signal number is written as a single byte whenever a signal is received. *fd* must be non-blocking. It returns the previous such file descriptor.

O valor `-1` desabilita o recurso; este é o estado inicial. Isso é equivalente à `signal.set_wakeup_fd()` em Python, mas sem nenhuma verificação

de erro. *fd* deve ser um descritor de arquivo válido. A função só deve ser chamada a partir da thread principal.

Alterado na versão 3.5: No Windows, a função agora também suporta manipuladores de socket.

Classes de exceção

PyObject *PyErr_NewException(const char *name, PyObject *base, PyObject *dict)

Retorna valor: Nova referência. Parte da ABI Estável.
This utility function creates and returns a new exception class. The *name* argument must be the name of the new exception, a C string of the form `module.classname`. The *base* and *dict* arguments are normally `NULL`. This creates a class object derived from **`Exception`** (accessible in C as **`PyExc_Exception`**).

The **`__module__`** attribute of the new class is set to the first part (up to the last dot) of the *name* argument, and the class name is set to the last part (after the last dot). The *base* argument can be used to specify alternate base classes; it can either be only one class or a tuple of classes. The *dict* argument can be used to specify a dictionary of class variables and methods.

PyObject *PyErr_NewExceptionWithDoc(const char *name, const char *doc, PyObject *base, PyObject *dict)

Retorna valor: Nova referência. Parte da ABI Estável.
Same as **`PyErr_NewException()`**, except that the new exception class can easily be given a docstring: If *doc* is non-`NULL`, it will be used as the docstring for the exception class.

Adicionado na versão 3.2.

int PyExceptionClass_Check(PyObject *ob)

Return non-zero if *ob* is an exception class, zero otherwise. This function always succeeds.

const char *PyExceptionClass_Name(PyObject *ob)

Parte da ABI Estável desde a versão 3.8.
Retorna `tp_name` da classe de exceção *ob*.

PyException_HEAD

This is a macro including the base fields for an exception object.

This was included in Python's C API by mistake and is not designed for use in extensions. For creating custom exception objects, use `PyErr_NewException()` or otherwise create a class inheriting from `PyExc_BaseException`.

Suavemente descontinuado desde a versão 3.15.

Objeto Exceção

int PyExceptionInstance_Check(PyObject *op)

Return true if *op* is an instance of `BaseException`, false otherwise. This function always succeeds.

PyExceptionInstance_Class(op)

Equivalent to `Py_TYPE(op)`.

PyObject *PyException_GetTraceback(PyObject *ex)

Retorna valor: Nova referência. Parte da ABI Estável.
Return the traceback associated with the exception as a new reference, as accessible from Python through the `__traceback__` attribute. If there is no traceback associated, this returns `NULL`.

int PyException_SetTraceback(PyObject *ex, PyObject *tb)

Parte da ABI Estável.
Defina o retorno traceback (situação da pilha de execução) associado à exceção como *tb*. Use `Py_None` para limpá-lo.

PyObject *PyException_GetContext(PyObject *ex)

Retorna valor: Nova referência. Parte da ABI Estável.

Return the context (another exception instance during whose handling *ex* was raised) associated with the exception as a new reference, as accessible from Python through the **`__context__`** attribute. If there is no context associated, this returns `NULL`.

void PyException_SetContext(PyObject *ex, PyObject *ctx)

Parte da ABI Estável.

Set the context associated with the exception to *ctx*. Use `NULL` to clear it. There is no type check to make sure that *ctx* is an exception instance. This "steals" a reference to *ctx*.

PyObject *PyException_GetCause(PyObject *ex)

Retorna valor: Nova referência. Parte da ABI Estável.

Return the cause (either an exception instance, or `None`, set by `raise ... from ...`) associated with the exception as a new reference, as accessible from Python through the **`__cause__`** attribute.

void PyException_SetCause(PyObject *ex, PyObject *cause)

Parte da ABI Estável.

Set the cause associated with the exception to *cause*. Use `NULL` to clear it. There is no type check to make sure that *cause* is either an exception instance or `None`. This "steals" a reference to *cause*.

O atributo **`__suppress_context__`** é implicitamente definido como `True` por esta função.

PyObject *PyException_GetArgs(PyObject *ex)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.12.

Retorna **`args`** da exceção *ex*.

void PyException_SetArgs(PyObject *ex, PyObject *args)

Parte da ABI Estável desde a versão 3.12.

Set **`args`** of exception *ex* to *args*.

PyObject *PyUnstable_Exc_PrepReraiseStar(PyObject *orig, PyObject *excs)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Implement part of the interpreter's implementation of **`except*`**. *orig* is the original exception that was caught, and *excs* is the list of the exceptions that need to be raised. This list contains the unhandled part of *orig*, if any, as well as the exceptions that were raised from the **`except*`** clauses (so they have a different traceback from *orig*) and those that were reraised (and have the same traceback as *orig*). Return the **`ExceptionGroup`** that needs to be reraised in the end, or `None` if there is nothing to reraise.

Adicionado na versão 3.12.

Objetos de exceção Unicode

As seguintes funções são usadas para criar e modificar exceções Unicode de C.

PyObject *PyUnicodeDecodeError_Create(const char *encoding, const char *object, Py_ssize_t length, Py_ssize_t start, Py_ssize_t end, const char *reason)

Retorna valor: Nova referência. Parte da ABI Estável.
Cria um objeto **`UnicodeDecodeError`** com os atributos *encoding*, *object*, *length*, *start*, *end* e *reason*. O *encoding* e *reason* são strings codificadas em UTF-8.

PyObject *PyUnicodeDecodeError_GetEncoding(PyObject *exc)
PyObject *PyUnicodeEncodeError_GetEncoding(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o atributo * encoding* dado no objeto da exceção.

PyObject *PyUnicodeDecodeError_GetObject(PyObject *exc)

PyObject *PyUnicodeEncodeError_GetObject(PyObject *exc)
PyObject *PyUnicodeTranslateError_GetObject(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o atributo *object* dado no objeto da exceção.

int PyUnicodeDecodeError_GetStart(PyObject *exc, Py_ssize_t *start)
int PyUnicodeEncodeError_GetStart(PyObject *exc, Py_ssize_t *start)
int PyUnicodeTranslateError_GetStart(PyObject *exc, Py_ssize_t *start)

Parte da ABI Estável.
Obtém o atributo *start* do objeto da exceção coloca-o em **start*. *start* não deve ser `NULL`. Retorna `0` se não der erro, `-1` caso dê erro.

If the **`UnicodeError.object`** is an empty sequence, the resulting *start* is `0`. Otherwise, it is clipped to `[0, len(object) - 1]`.

Ver também

`UnicodeError.start`

int PyUnicodeDecodeError_SetStart(PyObject *exc, Py_ssize_t start)
int PyUnicodeEncodeError_SetStart(PyObject *exc, Py_ssize_t start)
int PyUnicodeTranslateError_SetStart(PyObject *exc, Py_ssize_t start)

Parte da ABI Estável.
Set the *start* attribute of the given exception object to *start*. Return `0` on success, `-1` on failure.

Nota

While passing a negative *start* does not raise an exception, the corresponding getters will not consider it as a relative offset.

int PyUnicodeDecodeError_GetEnd(PyObject *exc, Py_ssize_t *end)
int PyUnicodeEncodeError_GetEnd(PyObject *exc, Py_ssize_t *end)
int PyUnicodeTranslateError_GetEnd(PyObject *exc, Py_ssize_t *end)

Parte da ABI Estável.
Obtenha o atributo *end* dado no objeto de exceção e coloque **end*. O *end* não deve ser `NULL`. Em caso de sucesso, retorna `0`, em caso de falha, retorna `-1`.

If the **`UnicodeError.object`** is an empty sequence, the resulting *end* is `0`. Otherwise, it is clipped to `[1, len(object)]`.

int PyUnicodeDecodeError_SetEnd(PyObject *exc, Py_ssize_t end)
int PyUnicodeEncodeError_SetEnd(PyObject *exc, Py_ssize_t end)
int PyUnicodeTranslateError_SetEnd(PyObject *exc, Py_ssize_t end)

Parte da ABI Estável.
Defina o atributo *end* do objeto de exceção fornecido para *end*. Retorna `0` em caso de sucesso e `-1` em caso de falha.

Ver também

`UnicodeError.end`

PyObject *PyUnicodeDecodeError_GetReason(PyObject *exc)
PyObject *PyUnicodeEncodeError_GetReason(PyObject *exc)
PyObject *PyUnicodeTranslateError_GetReason(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o atributo *reason* dado no objeto da exceção.

int PyUnicodeDecodeError_SetReason(PyObject *exc, const char *reason)
int PyUnicodeEncodeError_SetReason(PyObject *exc, const char *reason)
int PyUnicodeTranslateError_SetReason(PyObject *exc, const char *reason)

Parte da ABI Estável.
Defina o atributo *reason* do objeto de exceção fornecido para *reason*. Retorna `0` em caso de sucesso e `-1` em caso de falha.

Controle de recursão

These two functions provide a way to perform safe recursive calls at the C level, both in the core and in extension modules. They are needed if the recursive code does not necessarily invoke Python code (which tracks its recursion depth automatically). They are also not needed for *tp_call* implementations because the call protocol takes care of recursion handling.

int Py_EnterRecursiveCall(const char *where)

Parte da ABI Estável desde a versão 3.9.
Marca um ponto em que a chamada recursiva em nível C está prestes a ser executada.

The function then checks if the stack limit is reached. If this is the case, a `RecursionError` is set and a nonzero value is returned. Otherwise, zero is returned.

The check is based on the remaining C stack space of the current thread, not on a count of calls, so it is unaffected by `Py_SetRecursionLimit()` and `sys.setrecursionlimit()`.

where should be a UTF-8 encoded string such as `" in instance check"` to be concatenated to the `RecursionError` message caused by the recursion depth limit.

Ver também

The `PyUnstable_ThreadState_SetStackProtection()` function.

Alterado na versão 3.9: Esta função agora também está disponível na API limitada.

Alterado na versão 3.14: The check is based on the remaining C stack space. Previously, a separate counter of C-level calls was used.

void Py_LeaveRecursiveCall(void)

Parte da ABI Estável desde a versão 3.9.
Termina com `Py_EnterRecursiveCall()`. Deve ser chamada uma vez para cada invocação *successful* de `Py_EnterRecursiveCall()`.

Alterado na versão 3.9: Esta função agora também está disponível na API limitada.

Properly implementing `tp_repr` for container types requires special recursion handling. In addition to protecting the stack, `tp_repr` also needs to track objects to prevent cycles. The following two functions facilitate this functionality. Effectively, these are the C equivalent to `@reprlib.recursive_repr`.

int Py_ReprEnter(PyObject *object)

Parte da ABI Estável.
Chamado no início da implementação `tp_repr` para detectar ciclos.

If the object has already been processed, the function returns a positive integer. In that case the `tp_repr` implementation should return a string object indicating a cycle. As examples, `dict` objects return `{...}` and `list` objects return `[...]`.

A função retornará um inteiro negativo se o limite da recursão for atingido. Nesse caso a implementação `tp_repr` deverá, normalmente,. retornar `NULL`.

Caso contrário, a função retorna zero e a implementação `tp_repr` poderá continuar normalmente.

void Py_ReprLeave(PyObject *object)

Parte da ABI Estável.
Termina a `Py_ReprEnter()`. Deve ser chamado uma vez para cada chamada de `Py_ReprEnter()` que retorna zero.

int Py_GetRecursionLimit(void)

Parte da ABI Estável.
Get the recursion limit for the current interpreter. It can be set with `Py_SetRecursionLimit()`. The recursion limit prevents the Python interpreter stack from growing infinitely.

This function cannot fail, and the caller must hold an attached thread state.

Ver também

`sys.getrecursionlimit()`

void Py_SetRecursionLimit(int new_limit)

Parte da ABI Estável.

Set the recursion limit for the current interpreter.

This function cannot fail, and the caller must hold an attached thread state.

Ver também

`sys.setrecursionlimit()`

Exception and warning types

All standard Python exceptions and warning categories are available as global variables whose names are `PyExc_` followed by the Python exception name. These have the type PyObject*; they are all class objects.

For completeness, here are all the variables:

Tipos de exceção

Nome C	Nome Python
PyObject *PyExc_BaseException *Parte da ABI Estável.*	`BaseException`

Nome C	Nome Python
PyObject *PyExc_BaseExceptionGroup *Parte da ABI Estável desde a versão 3.11.*	`BaseExceptionGroup`
PyObject *PyExc_Exception *Parte da ABI Estável.*	`Exception`
PyObject *PyExc_ArithmeticError *Parte da ABI Estável.*	`ArithmeticError`
PyObject *PyExc_AssertionError *Parte da ABI Estável.*	`AssertionError`
PyObject *PyExc_AttributeError *Parte da ABI Estável.*	`AttributeError`
PyObject *PyExc_BlockingIOError *Parte da ABI Estável desde a versão 3.7.*	`BlockingIOError`
PyObject *PyExc_BrokenPipeError *Parte da ABI Estável desde a versão 3.7.*	`BrokenPipeError`
PyObject *PyExc_BufferError *Parte da ABI Estável.*	`BufferError`

Nome C	Nome Python
PyObject *PyExc_ChildProcessError *Parte da ABI Estável desde a versão 3.7.*	`ChildProcessError`
PyObject *PyExc_ConnectionAbortedError *Parte da ABI Estável desde a versão 3.7.*	`ConnectionAbortedError`
PyObject *PyExc_ConnectionError *Parte da ABI Estável desde a versão 3.7.*	`ConnectionError`
PyObject *PyExc_ConnectionRefusedError *Parte da ABI Estável desde a versão 3.7.*	`ConnectionRefusedError`
PyObject *PyExc_ConnectionResetError *Parte da ABI Estável desde a versão 3.7.*	`ConnectionResetError`
PyObject *PyExc_EOFError *Parte da ABI Estável.*	`EOFError`
PyObject *PyExc_FileExistsError *Parte da ABI Estável desde a versão 3.7.*	`FileExistsError`
PyObject *PyExc_FileNotFoundError *Parte da ABI Estável desde a versão 3.7.*	`FileNotFoundError`
PyObject *PyExc_FloatingPointError *Parte da ABI Estável.*	`FloatingPointError`

Nome C	Nome Python
PyObject *PyExc_GeneratorExit *Parte da ABI Estável.*	`GeneratorExit`
PyObject *PyExc_ImportCycleError	`ImportCycleError`
PyObject *PyExc_ImportError *Parte da ABI Estável.*	`ImportError`
PyObject *PyExc_IndentationError *Parte da ABI Estável.*	`IndentationError`
PyObject *PyExc_IndexError *Parte da ABI Estável.*	`IndexError`
PyObject *PyExc_InterruptedError *Parte da ABI Estável desde a versão 3.7.*	`InterruptedError`
PyObject *PyExc_IsADirectoryError *Parte da ABI Estável desde a versão 3.7.*	`IsADirectoryError`
PyObject *PyExc_KeyError *Parte da ABI Estável.*	`KeyError`
PyObject *PyExc_KeyboardInterrupt *Parte da ABI Estável.*	`KeyboardInterrupt`

Nome C	Nome Python
PyObject *PyExc_LookupError *Parte da ABI Estável.*	`LookupError`
PyObject *PyExc_MemoryError *Parte da ABI Estável.*	`MemoryError`
PyObject *PyExc_ModuleNotFoundError *Parte da ABI Estável desde a versão 3.6.*	`ModuleNotFoundError`
PyObject *PyExc_NameError *Parte da ABI Estável.*	`NameError`
PyObject *PyExc_NotADirectoryError *Parte da ABI Estável desde a versão 3.7.*	`NotADirectoryError`
PyObject *PyExc_NotImplementedError *Parte da ABI Estável.*	`NotImplementedError`
PyObject *PyExc_OSError *Parte da ABI Estável.*	`OSError`
PyObject *PyExc_OverflowError *Parte da ABI Estável.*	`OverflowError`
PyObject *PyExc_PermissionError *Parte da ABI Estável desde a versão 3.7.*	`PermissionError`

Nome C	Nome Python
PyObject *PyExc_ProcessLookupError *Parte da ABI Estável desde a versão 3.7.*	`ProcessLookupError`
PyObject *PyExc_PythonFinalizationError	`PythonFinalizationError`
PyObject *PyExc_RecursionError *Parte da ABI Estável desde a versão 3.7.*	`RecursionError`
PyObject *PyExc_ReferenceError *Parte da ABI Estável.*	`ReferenceError`
PyObject *PyExc_RuntimeError *Parte da ABI Estável.*	`RuntimeError`
PyObject *PyExc_StopAsyncIteration *Parte da ABI Estável desde a versão 3.7.*	`StopAsyncIteration`
PyObject *PyExc_StopIteration *Parte da ABI Estável.*	`StopIteration`
PyObject *PyExc_SyntaxError *Parte da ABI Estável.*	`SyntaxError`
PyObject *PyExc_SystemError *Parte da ABI Estável.*	`SystemError`

Nome C	Nome Python
PyObject *PyExc_SystemExit *Parte da ABI Estável.*	`SystemExit`
PyObject *PyExc_TabError *Parte da ABI Estável.*	`TabError`
PyObject *PyExc_TimeoutError *Parte da ABI Estável desde a versão 3.7.*	`TimeoutError`
PyObject *PyExc_TypeError *Parte da ABI Estável.*	`TypeError`
PyObject *PyExc_UnboundLocalError *Parte da ABI Estável.*	`UnboundLocalError`
PyObject *PyExc_UnicodeDecodeError *Parte da ABI Estável.*	`UnicodeDecodeError`
PyObject *PyExc_UnicodeEncodeError *Parte da ABI Estável.*	`UnicodeEncodeError`
PyObject *PyExc_UnicodeError *Parte da ABI Estável.*	`UnicodeError`
PyObject *PyExc_UnicodeTranslateError *Parte da ABI Estável.*	`UnicodeTranslateError`

Nome C	Nome Python
PyObject *PyExc_ValueError *Parte da ABI Estável.*	`ValueError`
PyObject *PyExc_ZeroDivisionError *Parte da ABI Estável.*	`ZeroDivisionError`

Adicionado na versão 3.3: `PyExc_BlockingIOError`, `PyExc_BrokenPipeError`, `PyExc_ChildProcessError`, `PyExc_ConnectionError`, `PyExc_ConnectionAbortedError`, `PyExc_ConnectionRefusedError`, `PyExc_ConnectionResetError`, `PyExc_FileExistsError`, `PyExc_FileNotFoundError`, `PyExc_InterruptedError`, `PyExc_IsADirectoryError`, `PyExc_NotADirectoryError`, `PyExc_PermissionError`, `PyExc_ProcessLookupError` e `PyExc_TimeoutError` foram introduzidos seguindo a **PEP 3151** [https://peps.python.org/pep-3151/].

Adicionado na versão 3.5: `PyExc_StopAsyncIteration` e `PyExc_RecursionError`.

Adicionado na versão 3.6: `PyExc_ModuleNotFoundError`.

Adicionado na versão 3.11: `PyExc_BaseExceptionGroup`.

OSError aliases

The following are a compatibility aliases to `PyExc_OSError`.

Alterado na versão 3.3: Esses aliases costumavam ser tipos de exceção separados.

Nome C	Nome Python	Notas
PyObject *PyExc_EnvironmentError *Parte da ABI Estável.*	`OSError`	
PyObject *PyExc_IOError *Parte da ABI Estável.*	`OSError`	
PyObject *PyExc_WindowsError *Parte da ABI Estável on Windows desde a versão 3.7.*	`OSError`	[win]

Notas:

[win]

`PyExc_WindowsError` is only defined on Windows; protect code that uses this by testing that the preprocessor macro `MS_WINDOWS` is defined.

Warning types

Nome C	Nome Python
PyObject *PyExc_Warning *Parte da ABI Estável.*	`Warning`
PyObject *PyExc_BytesWarning *Parte da ABI Estável.*	`BytesWarning`

Nome C	Nome Python
PyObject *PyExc_DeprecationWarning *Parte da ABI Estável.*	`DeprecationWarning`
PyObject *PyExc_EncodingWarning *Parte da ABI Estável desde a versão 3.10.*	`EncodingWarning`
PyObject *PyExc_FutureWarning *Parte da ABI Estável.*	`FutureWarning`
PyObject *PyExc_ImportWarning *Parte da ABI Estável.*	`ImportWarning`
PyObject *PyExc_PendingDeprecationWarning *Parte da ABI Estável.*	`PendingDeprecationWarning`
PyObject *PyExc_ResourceWarning *Parte da ABI Estável desde a versão 3.7.*	`ResourceWarning`
PyObject *PyExc_RuntimeWarning *Parte da ABI Estável.*	`RuntimeWarning`
PyObject *PyExc_SyntaxWarning *Parte da ABI Estável.*	`SyntaxWarning`

Nome C	Nome Python
PyObject *PyExc_UnicodeWarning *Parte da ABI Estável.*	`UnicodeWarning`
PyObject *PyExc_UserWarning *Parte da ABI Estável.*	`UserWarning`

Adicionado na versão 3.2: `PyExc_ResourceWarning`.

Adicionado na versão 3.10: `PyExc_EncodingWarning`.

Tracebacks

PyTypeObject PyTraceBack_Type

Parte da ABI Estável.

Type object for traceback objects. This is available as `types.TracebackType` in the Python layer.

int PyTraceBack_Check(PyObject *op)

Return true if *op* is a traceback object, false otherwise. This function does not account for subtypes.

int PyTraceBack_Here(PyFrameObject *f)

Parte da ABI Estável.

Replace the `__traceback__` attribute on the current exception with a new traceback prepending *f* to the existing chain.

Calling this function without an exception set is undefined behavior.

Esta função retorna “0” em caso de sucesso e retorna “-1” com uma exceção definida em caso de falha.

int PyTraceBack_Print(PyObject *tb, PyObject *f)

Parte da ABI Estável.

Write the traceback *tb* into the file *f*.

Esta função retorna “0” em caso de sucesso e retorna “-1” com uma exceção definida em caso de falha.

const char *PyUnstable_DumpTraceback(int fd, PyThreadState *tstate)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Write a trace of the Python stack in *tstate* into the file *fd*. The format looks like:

```
Traceback (most recent call first):
  File "xxx", line xxx in <xxx>
  File "xxx", line xxx in <xxx>
  ...
  File "xxx", line xxx in <xxx>
```

This function is meant to debug situations such as segfaults, fatal errors, and similar. The file and function names it outputs are encoded to ASCII with backslashreplace and truncated to 500 characters. It writes only the first 100 frames; further frames are truncated with the line . . . .

This function will return `NULL` on success, or an error message on error.

This function is intended for use in crash scenarios such as signal handlers for SIGSEGV, where the interpreter may be in an inconsistent state. Given that it reads interpreter data structures that may be partially modified, the function might produce incomplete output or it may even crash itself.

The caller does not need to hold an attached thread state, nor does *tstate* need to be attached.

Adicionado na versão 3.15.

const char *PyUnstable_DumpTracebackThreads(int fd, PyInterpreterState *interp, PyThreadState *current_tstate, Py_ssize_t max_threads)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Write the traces of all Python threads in *interp* into the file *fd*.

If *interp* is `NULL` then this function will try to identify the current interpreter using thread-specific storage. If it cannot, it will return an error.

If *current_tstate* is not `NULL` then it will be used to identify what the current thread is in the written output. If it is `NULL` then this function will identify the current thread using thread-specific storage. It is not an error if the function is unable to get the current Python thread state.

This function will return `NULL` on success, or an error message on error.

This function is meant to debug situations such as segfaults, fatal errors, and similar. It calls **`PyUnstable_DumpTraceback()`** for each thread. It only writes the tracebacks of the first *max_threads* threads, further output is truncated with the line `....` If *max_threads* is 0, the function will use a default value of 100 for the argument.

This function is intended for use in crash scenarios such as signal handlers for SIGSEGV, where the interpreter may be in an inconsistent state. Given that it reads interpreter data structures that may be partially modified, the function might produce incomplete output or it may even crash itself.

The caller does not need to hold an attached thread state, nor does *current_tstate* need to be attached.

Aviso

On the free-threaded build, this function is not thread-safe. If another thread deletes its thread state while this function is being called, the process will likely crash.

Adicionado na versão 3.15.

Definindo módulos de extensão

A C extension for CPython is a shared library (for example, a `.so` file on Linux, `.pyd` DLL on Windows), which is loadable into the Python process (for example, it is compiled with compatible compiler settings), and which exports an *export hook* function (or an old-style initialization function).

Para ser importável por padrão (ou seja, por `importlib.machinery.ExtensionFileLoader`), a biblioteca compartilhada deve estar disponível em `sys.path` e deve ser nomeada após o nome do módulo mais uma extensão listada em `importlib.machinery.EXTENSION_SUFFIXES`.

Nota

A construção, o empacotamento e a distribuição de módulos de extensão são melhor realizados com ferramentas de terceiros e estão fora do escopo deste documento. Uma ferramenta adequada é o Setuptools, cuja documentação pode ser encontrada em https://setuptools.pypa.io/en/latest/setuptools.html.

Extension export hook

Adicionado na versão 3.15: Support for the `PyModExport_<name>` export hook was added in Python 3.15. The older way of defining modules is still available: consult either the PyInit function section or earlier versions of this documentation if you plan to support earlier Python versions.

The export hook must be an exported function with the following signature:

PySlot *PyModExport_modulename(void)

For modules with ASCII-only names, the export hook must be named `PyModExport_<name>`, with `<name>` replaced by the module's name.

For non-ASCII module names, the export hook must instead be named `PyModExportU_<name>` (note the `U`), with `<name>` encoded using Python's *punycode* encoding with hyphens replaced by underscores. In Python:

```
def hook_name(name):
    try:
        suffix = b'_' + name.encode('ascii')
    except UnicodeEncodeError:
        suffix = b'U_' + name.encode('punycode').replace(b'-', b
    return b'PyModExport' + suffix
```

The export hook returns an array of **`PySlot`** entries, terminated by an entry with a slot ID of `0`. These slots describe how the module should be created and initialized.

This array must remain valid and constant until interpreter shutdown. Typically, it should use `static` storage. Prefer using the **`Py_mod_create`** and **`Py_mod_exec`** slots for any dynamic behavior.

The export hook may return `NULL` with an exception set to signal failure.

It is recommended to define the export hook function using a helper macro:

PyMODEXPORT_FUNC

Parte da ABI Estável desde a versão 3.15.

Declare an extension module export hook. This macro:

- specifies the PySlot* return type,
- adiciona quaisquer declarações de vinculação especiais exigidas pela plataforma e
- para C++, declara a função como `extern "C"`.

Por exemplo, um módulo chamado `spam` seria definido assim:

```
PyABIInfo_VAR(abi_info);
```

```
static PySlot spam_slots[] = {
    PySlot_STATIC_DATA(Py_mod_abi, &abi_info),
    PySlot_STATIC_DATA(Py_mod_name, "spam"),
    PySlot_FUNC(Py_mod_init, spam_init_function),
    ...
    PySlot_END
};

PyMODEXPORT_FUNC
PyModExport_spam(void)
{
    return spam_slots;
}
```

The export hook is typically the only non-`static` item defined in the module's C source.

The hook should be kept short. If it does more than `return` a static array, several caveats apply:

- If you need to use any Python C API, it is recommended to call **`PyABIInfo_Check()`** first to raise an exception, rather than crash, in common cases of ABI mismatch.
- Code in the export hook must never rely on the GIL: free-threaded builds of Python can only check the **`Py_mod_gil`** slot (or the lack of it) after the hook returns,
- Similarly, the hook may be called in any subinterpreter, since the **`Py_mod_multiple_interpreters`** slot (or lack of it) is only checked after the hook returns.

Por exemplo:

```
PyMODEXPORT_FUNC
PyModExport_modulename(void)
{
   if (PyABIInfo_Check(&abi_info, "modulename") < 0) {
      /* ABI mismatch. It's not safe to examine the raised excep
      return NULL;
   }

   /* use Python API (as little as possible); don't rely on GIL
```

```
    return modulename_slots;
}
```

Nota

It is possible to export multiple modules from a single shared library by defining multiple export hooks. However, importing them requires a custom importer or suitably named copies/links of the extension file, because Python's import machinery only finds the function corresponding to the filename. See the Multiple modules in one library [https://peps.python.org/pep-0489/#multiple-modules-in-one-library] section in **PEP 489** [https://peps.python.org/pep-0489/] for details.

Inicialização multifásica

The process of creating an extension module follows several phases:

- Python finds and calls the export hook to get information on how to create the module.
- Before any substantial code is executed, Python can determine which capabilities the module supports, and it can adjust the environment or refuse loading an incompatible extension. Slots like `Py_mod_abi`, `Py_mod_gil` and `Py_mod_multiple_interpreters` influence this step.
- By default, Python itself then creates the module object – that is, it does the equivalent of calling `__new__()` when creating an object. This step can be overridden using the `Py_mod_create` slot.
- Python sets initial module attributes like `__package__` and `__loader__`, and inserts the module object into `sys.modules`.
- Afterwards, the module object is initialized in an extension-specific way – the equivalent of `__init__()` when creating an object, or of executing top-level code in a Python-language module. The behavior is specified using the `Py_mod_exec` slot.

This is called *multi-phase initialization* to distinguish it from the legacy (but still supported) single-phase initialization, where an initialization function

returns a fully constructed module.

Alterado na versão 3.5: Adicionado suporte para inicialização multifásica (**PEP 489** [https://peps.python.org/pep-0489/]).

Várias instâncias de módulo

By default, extension modules are not singletons. For example, if the `sys.modules` entry is removed and the module is re-imported, a new module object is created and, typically, populated with fresh method and type objects. The old module is subject to normal garbage collection. This mirrors the behavior of pure-Python modules.

Instâncias adicionais do módulo podem ser criadas em subinterpretadores ou após a reinicialização do tempo de execução do Python (`Py_Finalize()` e `Py_Initialize()`). Nesses casos, compartilhar objetos Python entre instâncias do módulo provavelmente causaria travamentos ou comportamento indefinido.

Para evitar tais problemas, cada instância de um módulo de extensão deve ser *isolada*: alterações em uma instância não devem afetar implicitamente as outras, e todos os estados pertencentes ao módulo, incluindo referências a objetos Python, devem ser específicos de uma instância específica do módulo. Consulte Isolando módulos de extensão para obter mais detalhes e um guia prático.

Uma maneira mais simples de evitar esses problemas é levantar um erro na inicialização repetida.

Espera-se que todos os módulos ofereçam suporte a subinterpretador, ou então sinalizem explicitamente a falta de suporte. Isso geralmente é obtido por isolamento ou bloqueio de inicializações repetidas, como acima. Um módulo também pode ser limitado ao interpretador principal usando o slot `Py_mod_multiple_interpreters`.

`PyInit` function

Suavemente descontinuado desde a versão 3.15: This functionality will not get new features, but there are no plans to remove it.

Instead of **`PyModExport_modulename()`**, an extension module can define an older-style *initialization function* with the signature:

PyObject *PyInit_modulename(void)

Its name should be `PyInit_<name>`, with `<name>` replaced by the name of the module. For non-ASCII module names, use `PyInitU_<name>` instead, with `<name>` encoded in the same way as for the export hook (that is, using Punycode with underscores).

If a module exports both `PyInit_<name>` and `PyModExport_<name>`, the `PyInit_<name>` function is ignored.

Like with **`PyMODEXPORT_FUNC`**, it is recommended to define the initialization function using a helper macro:

PyMODINIT_FUNC

Declara uma função de inicialização do módulo de extensão. Esta macro:

- especifica o tipo de retorno PyObject*,
- adiciona quaisquer declarações de vinculação especiais exigidas pela plataforma e
- para C++, declara a função como `extern "C"`.

Normally, the initialization function (`PyInit_modulename`) returns a **`PyModuleDef`** instance with non-`NULL` **`m_slots`**. This allows Python to use multi-phase initialization.

Before it is returned, the `PyModuleDef` instance must be initialized using the following function:

PyObject *PyModuleDef_Init(PyModuleDef *def)

Parte da ABI Estável desde a versão 3.5.

Garante que uma definição de módulo é um objeto Python devidamente inicializado que reporta corretamente seu tipo e uma contagem de referências.

Retorna *def* convertido para `PyObject*`, ou `NULL` se ocorrer um erro.

Calling this function is required before returning a **`PyModuleDef`** from a module initialization function. It should not be used in other contexts.

Observe que o Python assume que as estruturas `PyModuleDef` são alocadas estaticamente. Esta função pode retornar uma nova referência ou uma referência emprestada; esta referência não deve ser liberada.

Adicionado na versão 3.5.

Por exemplo, um módulo chamado `spam` seria definido assim:

```
static struct PyModuleDef spam_module = {
    .m_base = PyModuleDef_HEAD_INIT,
    .m_name = "spam",
    ...
};

PyMODINIT_FUNC
PyInit_spam(void)
{
    return PyModuleDef_Init(&spam_module);
}
```

Inicialização monofásica legada

Suavemente descontinuado desde a versão 3.15: A inicialização monofásica é um mecanismo legado para inicializar módulos de extensão, com desvantagens e falhas de projeto conhecidas. Os autores de módulos de extensão são incentivados a utilizar a inicialização multifásica.

However, there are no plans to remove support for it.

In single-phase initialization, the old-style initialization function (`PyInit_modulename`) should create, populate and return a module object. This is typically done using **`PyModule_Create()`** and functions like **`PyModule_AddObjectRef()`**.

A inicialização monofásica difere do padrão nas seguintes maneiras:

- Módulos monofásicos são, ou melhor, *contêm*, “singletons”.

  Quando o módulo é inicializado pela primeira vez, o Python salva o conteúdo do `__dict__` do módulo (ou seja, normalmente, as funções e os tipos do módulo).

  Para importações subsequentes, o Python não chama a função de inicialização novamente. Em vez disso, ele cria um novo objeto de módulo com um novo `__dict__` e copia o conteúdo salvo para ele. Por exemplo, dado um módulo monofásico `_testsinglephase` [[1]] que define uma função `sum` e uma classe de exceção `error`:

  ```
  >>> import sys
  >>> import _testsinglephase as one
  >>> del sys.modules['_testsinglephase']
  >>> import _testsinglephase as two
  >>> one is two
  False
  >>> one.__dict__ is two.__dict__
  False
  >>> one.sum is two.sum
  True
  >>> one.error is two.error
  True
  ```

  O comportamento exato deve ser considerado um detalhe da implementação do CPython.

- Para contornar o fato de que `PyInit_modulename` não aceita um argumento *spec*, parte do estado do mecanismo de importação é salvo e aplicado ao primeiro módulo adequado criado durante a chamada `PyInit_modulename`. Especificamente, quando um submódulo é

importado, esse mecanismo adiciona o nome do pacote pai ao nome do módulo.

Uma função monofásica `PyInit_modulename` deve criar “seu” objeto de módulo o mais rápido possível, antes que qualquer outro objeto de módulo possa ser criado.

- Nomes de módulos não ASCII (`PyInitU_modulename`) não são suportados.
- Módulos monofásicos oferecem suporte a funções de pesquisa de módulos como **`PyState_FindModule()`**.
- The module’s **`PyModuleDef.m_slots`** must be NULL.

[1]

`_testsinglephase` é um módulo interno usado no conjunto de autotestes do CPython; sua instalação pode ou não incluí-lo.

Definition slots

To define module objects and classes using the C API, you may use an array of *slots* – essentially, key-value pairs that describe features of the object to create. This decouples the data from the structures used at runtime, allowing CPython – and other Python C API implementations – to update the structures without breaking backwards compatibility.

This section documents slots in general. For object-specific behavior and slot values, see documentation for functions that apply slots:

- `PyType_FromSlots()` for types;
- `PyModule_FromSlotsAndSpec()` and Extension export hook for modules.

When slots are passed to a function that applies them, the function will not modify the slot array, nor any data it points to (recursively). After the function is done, the caller is allowed to modify or deallocate the array and any data it points to (recursively), except data explicitly marked with `PySlot_STATIC`.

Except when documented otherwise, multiple slots with the same ID (`sl_id`) may not occur in a single slots array.

Adicionado na versão 3.15: Slot arrays generalize an earlier way of defining objects: using `PyType_Spec` with `PyType_Slot` for types, and `PyModuleDef` with `PyModuleDef_Slot` for modules. The earlier API is soft deprecated; there are no plans to remove it.

Entries of the slots array use the following structure:

type PySlot

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.15.

An entry in a slots array. Defined as:

```
typedef struct {
    uint16_t sl_id;
    uint16_t sl_flags;
    uint32_t _reserved;  // must be 0
    union {
        void *sl_ptr;
        void (*sl_func)(void);
        Py_ssize_t sl_size;
        int64_t sl_int64;
        uint64_t sl_uint64;
    };
} PySlot;
```

uint16_t sl_id

A slot ID, chosen from:

- `Py_slot_*` values documented in Common slot IDs below;
- `Py_mod_*` values for modules, as documented in Module definition;
- Values for types, as documented in Type slot IDs.

A **`sl_id`** of zero (**`Py_slot_end`**) marks the end of a slots array.

void *sl_ptr
void (*sl_func)(void)
Py_ssize_t sl_size
int64_t sl_int64
uint64_t sl_uint64

The data for the slot. These members are part of an anonymous union; the member to use depends on which data type is required by the slot ID: data pointer, function pointer, size, signed or unsigned integer, respectively.

Except when documented otherwise for a specific slot ID, pointers (that is **`sl_ptr`** and **`sl_func`**) may not be NULL.

uint16_t sl_flags

Zero or more of the following flags, OR-ed together:

PySlot_STATIC

Parte da ABI Estável desde a versão 3.15.
All data the slot points to is statically allocated and constant. Thus, the interpreter does not need to copy the information.

This flag is implied for function pointers.

The flag applies even to data the slot points to "indirectly", except for slots nested via `Py_slot_subslots` which may have their own **`PySlot_STATIC`** flags. For example, if applied to a `Py_tp_members` slot that points to an array of `PyMemberDef` structures, then the entire array, as well as the name and doc strings in its elements, must be static and constant.

PySlot_INTPTR

Parte da ABI Estável desde a versão 3.15.
The data is stored in `sl_ptr`; CPython will cast it to the appropriate type.

This flag can simplify porting from the older `PyType_Slot` and `PyModuleDef_Slot` structures.

PySlot_OPTIONAL

Parte da ABI Estável desde a versão 3.15.
If the slot ID is unknown, the interpreter should ignore the slot, rather than fail.

For example, if Python 3.16 adds a new feature with a new slot ID,attr the corresponding slot may be marked **`PySlot_OPTIONAL`** so that Python 3.15 ignores it.

Note that the "optionality" only applies to unknown slot IDs. This flag does not make Python skip invalid values of known slots.

Adicionado na versão 3.15.

Convenience macros

PySlot_DATA(name, value)

PySlot_FUNC(name, value)

PySlot_SIZE(name, value)

PySlot_INT64(name, value)

PySlot_UINT64(name, value)

PySlot_STATIC_DATA(name, value)

Parte da ABI Estável desde a versão 3.15.

Convenience macros to define **`PySlot`** structures with **`sl_id`** and a particular union member set.

`PySlot_STATIC_DATA` sets the **`PySlot_STATIC`** flag; others set no flags.

Note that these macros use *designated initializers*, a C language feature that C++ added in the 2020 version of the standard. If your code needs to be compatible with C++11 or older, use **`PySlot_PTR`** instead.

Defined as:

```
#define PySlot_DATA(NAME, VALUE) \
   {.sl_id=NAME, .sl_ptr=(void*)(VALUE)}

#define PySlot_FUNC(NAME, VALUE) \
   {.sl_id=NAME, .sl_func=(VALUE)}

#define PySlot_SIZE(NAME, VALUE) \
   {.sl_id=NAME, .sl_size=(VALUE)}

#define PySlot_INT64(NAME, VALUE) \
   {.sl_id=NAME, .sl_int64=(VALUE)}

#define PySlot_UINT64(NAME, VALUE) \
   {.sl_id=NAME, .sl_uint64=(VALUE)}

#define PySlot_STATIC_DATA(NAME, VALUE) \
   {.sl_id=NAME, .sl_flags=PySlot_STATIC, .sl_ptr=(VALUE)}
```

Adicionado na versão 3.15.

PySlot_END

Parte da ABI Estável desde a versão 3.15.
Convenience macro to mark the end of a **`PySlot`** array.

Defined as:

```
#define PySlot_END {0}
```

Adicionado na versão 3.15.

PySlot_PTR(name, value)
PySlot_PTR_STATIC(name, value)

Parte da ABI Estável desde a versão 3.15.
Convenience macros for use in C++11-compatible code. This version of C++ does not allow setting arbitrary union members in literals; instead, these macros set the **`PySlot_INTPTR`** flag and cast the value to `(void*)`.

Defined as:

```
#define PySlot_PTR(NAME, VALUE) \
   {NAME, PySlot_INTPTR, {0}, {(void*)(VALUE)}}

#define PySlot_PTR_STATIC(NAME, VALUE) \
   {NAME, PySlot_INTPTR|Py_SLOT_STATIC, {0}, {(void*)(VALUE)}
```

Adicionado na versão 3.15.

Common slot IDs

The following slot IDs may be used in both type and module definitions.

Py_slot_end

Parte da ABI Estável desde a versão 3.15.
Marks the end of a slots array. Defined as zero.

Adicionado na versão 3.15.

Py_slot_subslots

Parte da ABI Estável desde a versão 3.15.

Nested slots array.

The value (**sl_ptr**) should point to an array of **PySlot** structures. The slots in the array (up to but not including the zero-ID terminator) will be treated as if they were inserted if the current slot array, at the point **Py_slot_subslots** appears.

Slot nesting depth is limited to 5 levels. This restriction may be lifted in the future.

Adicionado na versão 3.15.

Py_slot_invalid

Parte da ABI Estável desde a versão 3.15.
Reserved; will always be treated as an unknown slot ID. Defined as `UINT16_MAX` (`0xFFFF`).

When used with the **PySlot_OPTIONAL** flag, defines a slot with no effect. Without the flag, processing a slot with this ID will fail.

Adicionado na versão 3.15.

Utilitários

As funções neste capítulo executam várias tarefas de utilidade pública, desde ajudar o código C a ser mais portátil em plataformas, usando módulos Python de C, como também, a analise de argumentos de função e a construção de valores Python a partir de valores C.

- Utilitários do sistema operacional
- Funções de sistema
- Controle de processos
- Importando módulos
- Suporte a *marshalling* de dados
- Análise de argumentos e construção de valores
  - Análise de argumentos
    - Strings e buffers
    - Números
    - Outros objetos
    - Funções da API
  - Construindo valores
- Conversão e formação de strings
- Classificação e conversão de caracteres
- API do PyHash
- Reflexão
- Registro de codec e funções de suporte
  - API de pesquisa de codec
  - API de registro de tratamentos de erros de decodificação Unicode
  - Variáveis utilitárias de codec
- API C PyTime
  - Tipos
  - Funções de relógio
  - Funções de relógio brutas
  - Funções de conversão
- Suporte a Mapas do Perf

Utilitários do sistema operacional

PyObject *PyOS_FSPath(PyObject *path)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.6.

Retorna a representação do sistema de arquivos para *path*. Se o objeto for um objeto **`str`** ou **`bytes`**, então uma nova referência forte é retornada. Se o objeto implementa a interface **`os.PathLike`**, então **`__fspath__()`** é retornado desde que seja um objeto **`str`** ou **`bytes`**. Caso contrário, **`TypeError`** é levantada e `NULL` é retornado.

Adicionado na versão 3.6.

int Py_FdIsInteractive(FILE *fp, const char *filename)

Retorna verdadeiro (não zero) se o arquivo padrão de E/S *fp* com o nome *filename* for considerado interativo. Este é o caso dos arquivos para os quais `isatty(fileno(fp))` é verdade. Se **`PyConfig.interactive`** for não zero, esta função também retorna true se o ponteiro *filename* for `NULL` ou se o nome for igual a uma das strings `'<stdin>'` ou `'???'`.

Esta função não deve ser chamada antes da inicialização do Python.

void PyOS_BeforeFork()

Parte da ABI Estável on platforms with fork() desde a versão 3.7.

Função para preparar algum estado interno antes de ser feito um fork do processo. Isso deve ser chamado antes de chamar **`fork()`** ou qualquer função semelhante que clone o processo atual. Disponível apenas em sistemas onde **`fork()`** é definido.

Aviso

A chamada C **`fork()`** só deve ser feita a partir da thread “main” (do interpretador “main”). O mesmo vale para `PyOS_BeforeFork()`.

Adicionado na versão 3.7.

void PyOS_AfterFork_Parent()

Parte da ABI Estável on platforms with fork() desde a versão 3.7.
Função para atualizar algum estado interno depois de ser feito um fork do processo. Isso deve ser chamado a partir do processo pai depois de chamar **`fork()`** ou qualquer função semelhante que clone o processo atual, independentemente da clonagem do processo ter sido bem-sucedida ou não. Disponível apenas em sistemas onde **`fork()`** é definido.

Aviso

A chamada C **`fork()`** só deve ser feita a partir da thread "main" (do interpretador "main"). O mesmo vale para `PyOS_AfterFork_Parent()`.

Adicionado na versão 3.7.

void PyOS_AfterFork_Child()

Parte da ABI Estável on platforms with fork() desde a versão 3.7.
Função para atualizar o estado interno do interpretador depois de ser feito um fork do processo. Isso deve ser chamado a partir do processo filho depois de chamar **`fork()`** ou qualquer função semelhante que clone o processo atual, se houver alguma chance do processo ter uma chamada de retorno para o interpretador Python. Disponível apenas em sistemas onde **`fork()`** é definido.

Aviso

A chamada C **`fork()`** só deve ser feita a partir da thread "main" (do interpretador "main"). O mesmo vale para `PyOS_AfterFork_Child()`.

Adicionado na versão 3.7.

Ver também

`os.register_at_fork()` permite registrar funções personalizadas do Python para serem chamadas por `PyOS_BeforeFork()`, `PyOS_AfterFork_Parent()` e `PyOS_AfterFork_Child()`.

void PyOS_AfterFork()

Parte da ABI Estável on platforms with fork().

Função para atualizar algum estado interno após ser feito um fork de processo; isso deve ser chamado no novo processo se o interpretador do Python continuar a ser usado. Se um novo executável é carregado no novo processo, esta função não precisa ser chamada.

Descontinuado desde a versão 3.7: Esta função foi sucedida por `PyOS_AfterFork_Child()`.

int PyOS_CheckStack()

Parte da ABI Estável on platforms with USE_STACKCHECK desde a versão 3.7.

Retorna verdadeiro quando o interpretador fica sem espaço na pilha. Esta é uma verificação confiável, mas só está disponível quando `USE_STACKCHECK` é definido (atualmente em certas versões do Windows usando o compilador Microsoft Visual C++). `USE_STACKCHECK` será definido automaticamente; você nunca deve alterar a definição em seu próprio código.

typedef void (*PyOS_sighandler_t)(int)

Parte da ABI Estável.

PyOS_sighandler_t PyOS_getsig(int i)

Parte da ABI Estável.

Retorna o manipulador de sinal atual para o sinal *i*. Este é um invólucro fino em torno de `sigaction()` ou `signal()`. Não chame essas funções diretamente!

PyOS_sighandler_t PyOS_setsig(int i, PyOS_sighandler_t h)

Parte da ABI Estável.

Define o manipulador de sinal para o sinal *i* como *h*; retornar o manipulador de sinal antigo. Este é um invólucro fino em torno de **`sigaction()`** ou **`signal()`**. Não chame essas funções diretamente!

int PyOS_InterruptOccurred(void)

Parte da ABI Estável.

Verifica se um sinal **`SIGINT`** foi recebido.

Retorna `1` se um **`SIGINT`** ocorreu e limpa o sinalizador de sinal, ou `0` caso contrário.

Na maioria dos casos, você deve preferir **`PyErr_CheckSignals()`** em vez desta função. **`PyErr_CheckSignals()`** invoca os manipuladores de sinal apropriados para todos os sinais pendentes, permitindo que o código Python lide com o sinal corretamente. Esta função detecta apenas **`SIGINT`** e não invoca nenhum manipulador de sinal do Python.

Esta função é segura para sinais de async e não pode falhar. O chamador deve manter um estado de thread anexado.

wchar_t *Py_DecodeLocale(const char *arg, size_t *size)

Parte da ABI Estável desde a versão 3.7.

Aviso

Esta função não deve ser chamada diretamente: use a API **`PyConfig`** com a função **`PyConfig_SetBytesString()`** que garante que Python esteja pré-inicializado.

Esta função não deve ser chamada antes de Python estar pré-inicializado e para que a localidade LC_CTYPE seja configurada corretamente: consulte a função **`Py_PreInitialize()`**.

Decodifica uma string de bytes do tratador de erros e codificação do sistema de arquivos. Se o tratador de erros for o tratador de errors surrogateescape, bytes não decodificáveis são decodificados como caracteres no intervalo U+DC80..U+DCFF; e se uma string de bytes puder ser decodificada como um caractere substituto, os bytes são

escapados usando o tratador de erros surrogateescape em vez de decodificá-los.

Retorna um ponteiro para uma string de caracteres largos recém-alocada, usa `PyMem_RawFree()` para liberar a memória. Se o tamanho não for `NULL`, escreve o número de caracteres largos excluindo o caractere nulo em `*size`

Retorna `NULL` em erro de decodificação ou erro de alocação de memória. Se *size* não for `NULL`, `*size` é definido como `(size_t)-1` em erro de memória ou definido como `(size_t)-2` em erro de decodificação.

tratador de erros e codificação do sistema de arquivos são selecionados por `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` de `PyConfig`.

Erros de decodificação nunca devem acontecer, a menos que haja um bug na biblioteca C.

Use a função `Py_EncodeLocale()` para codificar a string de caracteres de volta para uma string de bytes.

Ver também

As funções `PyUnicode_DecodeFSDefaultAndSize()` e `PyUnicode_DecodeLocaleAndSize()`.

Adicionado na versão 3.5.

Alterado na versão 3.7: A função agora usa a codificação UTF-8 no Modo UTF-8 do Python.

Alterado na versão 3.8: A função agora usa a codificação UTF-8 no Windows se `PyPreConfig.legacy_windows_fs_encoding` for zero;

char *Py_EncodeLocale(const wchar_t *text, size_t *error_pos)

Parte da ABI Estável desde a versão 3.7.

Codifica uma string de caracteres largos para o tratador de erros e codificação do sistema de arquivos. Se o tratador de erros for um tratador de erros substituto, os caracteres substitutos no intervalo U+DC80..U+DCFF são convertidos em bytes 0x80..0xFF.

Retorna um ponteiro para uma string de bytes recém-alocada; use `PyMem_Free()` para liberar a memória. Retorna `NULL` em caso de erro de codificação ou de erro de alocação de memória.

Se error_pos não for `NULL`, `*error_pos` será definido como `(size_t)-1` em caso de sucesso, ou definido como o índice do caractere inválido em caso de erro de codificação.

tratador de erros e codificação do sistema de arquivos são selecionados por `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` de `PyConfig`.

Use a função `Py_DecodeLocale()` para decodificar a string de bytes de volta para uma string de caracteres largos.

Aviso

Esta função não deve ser chamada antes de Python estar pré-inicializado e para que a localidade LC_CTYPE seja configurada corretamente: consulte a função `Py_PreInitialize()`.

Ver também

As funções `PyUnicode_EncodeFSDefault()` e `PyUnicode_EncodeLocale()`.

Adicionado na versão 3.5.

Alterado na versão 3.7: A função agora usa a codificação UTF-8 no Modo UTF-8 do Python.

Alterado na versão 3.8: A função agora usa a codificação UTF-8 no Windows se `PyPreConfig.legacy_windows_fs_encoding` for zero.

FILE *Py_fopen(PyObject *path, const char *mode)

Semelhante a **fopen()**, mas *path* é um objeto Python e uma exceção é definida em caso de erro.

path deve ser um objeto **str**, um objeto **bytes** ou um objeto caminho ou similar.

Em caso de sucesso, retorna um novo ponteiro para arquivo. Em caso de erro, define uma exceção e retorna `NULL`.

O arquivo deve ser fechado por **Py_fclose()** em vez de chamar diretamente **fclose()**.

O descritor de arquivo é criado não herdável (**PEP 446** [https://peps.python.org/pep-0446/]).

O chamador deve ter um estado de thread anexado.

Adicionado na versão 3.14.

int Py_fclose(FILE *file)

Feche um arquivo que foi aberto por **Py_fopen()**.

Em caso de sucesso, retorna `0`. Em caso de erro, retorna `EOF` e `errno` é definido para indicar o erro. Em ambos os casos, qualquer acesso subsequente (incluindo outra chamada a **Py_fclose()**) ao fluxo resulta em comportamento indefinido.

Adicionado na versão 3.14.

Funções de sistema

Essas são funções utilitárias que tornam a funcionalidade do módulo **sys** acessível ao código C. Todas elas funcionam com o dicionário do módulo **sys** da thread do interpretador atual, que está contido na estrutura interna do estado de thread.

PyObject *PySys_GetAttr(PyObject *name)

Parte da ABI Estável desde a versão 3.15.

Obtém o atributo *name* do módulo **`sys`**. Retorna uma referência forte. Levanta **`RuntimeError`** e retorna `NULL` se ele não existir ou se o módulo **`sys`** não puder ser encontrado.

Se o objeto inexistente não deve ser tratado como uma falha, você pode usar **`PySys_GetOptionalAttr()`** em vez disso.

Adicionado na versão 3.15.

PyObject *PySys_GetAttrString(const char *name)

Parte da ABI Estável desde a versão 3.15.

Isso é o mesmo que **`PySys_GetAttr()`**, mas *name* é especificado como uma string de bytes codificada em UTF-8 do tipo const char*, em vez de um PyObject*.

Se o objeto inexistente não deve ser tratado como uma falha, você pode usar **`PySys_GetOptionalAttrString()`** em vez disso.

Adicionado na versão 3.15.

int PySys_GetOptionalAttr(PyObject *name, PyObject **result)

Parte da ABI Estável desde a versão 3.15.

Variante de **`PySys_GetAttr()`** que não levanta uma exceção se o objeto não existir.

- Define **result* como uma nova referência forte para o objeto e retorna `1` se o objeto existir.
- Define **result* como `NULL` e retorna `0` sem definir uma exceção caso o objeto não exista.
- Define uma exceção, atribui `NULL` a **result* e retorna `-1` se ocorrer um erro.

Adicionado na versão 3.15.

int PySys_GetOptionalAttrString(const char *name, PyObject **result)

Parte da ABI Estável desde a versão 3.15.
Isso é o mesmo que `PySys_GetOptionalAttr()`, mas *name* é especificado como uma string de bytes codificada em UTF-8 do tipo const char*, em vez de um PyObject*.

Adicionado na versão 3.15.

PyObject *PySys_GetObject(const char *name)

Retorna valor: Referência emprestada. Parte da ABI Estável.
Semelhante a `PySys_GetAttrString()`, mas retorna uma referência emprestada e retorna `NULL` *sem* definir uma exceção em caso de falha.

Preserva a exceção que foi definida antes da chamada.

int PySys_SetObject(const char *name, PyObject *v)

Parte da ABI Estável.
Define *name* no módulo `sys` como *v*, a menos que *v* seja `NULL`, caso em que *name* é excluído do módulo sys. Retorna `0` em caso de sucesso e `-1` em caso de erro.

void PySys_WriteStdout(const char *format, ...)

Parte da ABI Estável.
Escreve a string de saída descrita por *format* em `sys.stdout`. Nenhuma exceção será levantada, mesmo que ocorra truncamento (veja abaixo).

format deve limitar o tamanho total da string de saída formatada a 1000 bytes ou menos – após 1000 bytes, a string de saída é truncada. Em particular, isso significa que não devem ocorrer formatos “%s” irrestritos; estes devem ser limitados usando “%.1s”, onde 2 é um número decimal calculado de forma que 3 mais o tamanho máximo de outros textos formatados não exceda 1000 bytes. Também fique atento a “%f”, que pode exibir centenas de dígitos para números muito grandes.

Se ocorrer um problema, ou se `sys.stdout` não estiver definido, a mensagem formatada será escrita na saída padrão (nível C) *stdout*.

void PySys_WriteStderr(const char *format, ...)

Parte da ABI Estável.
Como `PySys_WriteStdout()`, mas escreve em `sys.stderr` ou *stderr* em vez disso.

void PySys_FormatStdout(const char *format, ...)

Parte da ABI Estável.
Função semelhante a PySys_WriteStdout(), mas formata a mensagem usando `PyUnicode_FromFormatV()` e não trunca a mensagem para um comprimento arbitrário.

Adicionado na versão 3.2.

void PySys_FormatStderr(const char *format, ...)

Parte da ABI Estável.
Como `PySys_FormatStdout()`, mas escreve em `sys.stderr` ou *stderr* em vez disso.

Adicionado na versão 3.2.

PyObject *PySys_GetXOptions()

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.7.
Retorna o dicionário atual de opções `-X`, de forma semelhante a `sys.xoptions`. Em caso de erro, retorna `NULL` e uma exceção é definida.

Adicionado na versão 3.2.

int PySys_Audit(const char *event, const char *format, ...)

Parte da ABI Estável desde a versão 3.13.
Levanta um evento de auditoria com todos os ganchos ativos. Retorna zero em caso de sucesso e um valor diferente de zero com uma exceção definida em caso de falha.

O argumento de string *event* não pode ser *NULL*.

Se algum gancho tiver sido adicionado, o argumento *format* e outros serão usados para construir uma tupla a ser passada. Além de `N`, os mesmos caracteres de formatação usados em `Py_BuildValue()` estão disponíveis. Se o valor construído não for uma tupla, ele será adicionado a uma tupla de um único elemento.

A opção de formato `N` não deve ser usada. Ela consome uma referência, mas como não há como saber se os argumentos dessa função serão consumidos, usá-la pode causar vazamentos de referência.

Observe que os caracteres de formato `#` devem sempre ser tratados como `Py_ssize_t`, independentemente de `PY_SSIZE_T_CLEAN` ter sido definido.

`sys.audit()` executa a mesma função a partir do código Python.

Veja também `PySys_AuditTuple()`.

Adicionado na versão 3.8.

Alterado na versão 3.8.2: É necessário o tipo `Py_ssize_t` para caracteres de formato `#`. Anteriormente, um aviso de descontinuação era levantado.

int PySys_AuditTuple(const char *event, PyObject *args)

Parte da ABI Estável desde a versão 3.13.
Semelhante a `PySys_Audit()`, mas passa argumentos como um objeto Python. *args* deve ser uma `tuple`. Para não passar argumentos, *args* pode ser *NULL*.

Adicionado na versão 3.13.

int PySys_AddAuditHook(Py_AuditHookFunction hook, void *userData)

Adiciona o *gancho* chamável à lista de ganchos de auditoria ativos. Retorna zero em caso de sucesso e um valor diferente de zero em caso de falha. Se o ambiente de execução já tiver sido inicializado, também define um erro em caso de falha. Os ganchos adicionados por meio

desta API são chamados para todos os interpretadores criados pelo ambiente de execução.

O ponteiro *userData* é passado para a função de gancho. Como as funções de gancho podem ser chamadas de diferentes tempos de execução, esse ponteiro não deve se referir diretamente ao estado do Python.

Esta função pode ser chamada com segurança antes de `Py_Initialize()`. Quando chamada após a inicialização em tempo de execução, os ganchos de auditoria existentes são notificados e podem abortar silenciosamente a operação, levantando um erro da classe `Exception` (outros erros não serão silenciados).

A função de gancho é sempre chamada com um estado de thread anexado pelo interpretador Python que levantou o evento.

Consulte a **PEP 578** [https://peps.python.org/pep-0578/] para uma descrição detalhada da auditoria. As funções no ambiente de execução e na biblioteca padrão que levantam eventos estão listadas na tabela de eventos de auditoria. Os detalhes estão na documentação de cada função.

Se o interpretador estiver inicializado, esta função levanta um evento de auditoria `sys.addaudithook` sem argumentos. Se algum gancho existente levantar uma exceção derivada de `Exception`, o novo gancho não será adicionado e a exceção será tratada. Consequentemente, os chamadores não podem presumir que seu gancho foi adicionado, a menos que controlem todos os ganchos existentes.

typedef int (*Py_AuditHookFunction)(const char *event, PyObject *args, void *userData)

O tipo da função de gancho. *event* é o argumento de evento em string C passado para `PySys_Audit()` ou `PySys_AuditTuple()`. *args* tem a garantia de ser um `PyTupleObject`. *userData* é o argumento passado para PySys_AddAuditHook().

Adicionado na versão 3.8.

Controle de processos

void Py_FatalError(const char *message)

Parte da ABI Estável.

Exibe uma mensagem de erro fatal e encerra forçadamente o processo. Nenhuma limpeza é realizada. Esta função só deve ser invocada quando uma condição for detectada que torne perigoso continuar usando o interpretador Python; por exemplo, quando a administração de objetos parecer estar corrompida. No Unix, a função da biblioteca padrão C **`abort()`** é chamada, que tentará produzir um arquivo `core`.

A função `Py_FatalError()` é substituída por uma macro que registra automaticamente o nome da função atual, a menos que a macro `Py_LIMITED_API` esteja definida.

Alterado na versão 3.9: Registra o nome da função automaticamente.

void Py_Exit(int status)

Parte da ABI Estável.

Encerra o processo atual. Isso chama **`Py_FinalizeEx()`** e, em seguida, chama a função da biblioteca padrão C `exit(status)`. Se **`Py_FinalizeEx()`** indicar um erro, o código de status de saída será definido como 120.

Alterado na versão 3.6: Erros da finalização não são mais ignorados.

int Py_AtExit(void (*func)())

Parte da ABI Estável.

Registra uma função de limpeza para ser chamada por **`Py_FinalizeEx()`**. A função de limpeza será chamada sem argumentos e não deve retornar nenhum valor. No máximo 32 funções de limpeza podem ser registradas. Quando o registro for bem-sucedido, **`Py_AtExit()`** retorna `0`; em caso de falha, retorna `-1`. A última função de limpeza registrada é chamada primeiro. Cada função de limpeza será chamada no máximo uma vez. Como a finalização interna do Python

terá sido concluída antes da função de limpeza, nenhuma API do Python deve ser chamada por *func*.

Ver também

`PyUnstable_AtExit()` para passar um argumento `void *data`.

Importando módulos

PyObject *PyImport_ImportModule(const char *name)

Retorna valor: Nova referência. Parte da ABI Estável.
Esta é uma função auxiliar em torno de **PyImport_Import()**, que recebe um const char* como argumento em vez de um PyObject*.

PyObject *PyImport_ImportModuleEx(const char *name, PyObject *globals, PyObject *locals, PyObject *fromlist)

Retorna valor: Nova referência.
Importa um módulo. Isso é melhor descrito referindo-se à função embutida do Python **__import__()**.

O valor de retorno é uma nova referência ao módulo importado ou pacote de nível superior, ou `NULL` com uma exceção definida em caso de falha. Como para **__import__()**, o valor de retorno quando um submódulo de um pacote é solicitado é normalmente o pacote de nível superior, a menos que um *fromlist* não vazio seja fornecido.

As importações com falhas removem objetos incompletos do módulo, como em **PyImport_ImportModule()**.

PyObject *PyImport_ImportModuleLevelObject(PyObject *name, PyObject *globals, PyObject *locals, PyObject *fromlist, int level)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.
Importa um módulo. Isso é melhor descrito referindo-se à função embutida do Python **__import__()**, já que a função padrão **__import__()** chama essa função diretamente.

O valor de retorno é uma nova referência ao módulo importado ou pacote de nível superior, ou `NULL` com uma exceção definida em caso de falha. Como para **__import__()**, o valor de retorno quando um

submódulo de um pacote é solicitado é normalmente o pacote de nível superior, a menos que um *fromlist* não vazio seja fornecido.

Adicionado na versão 3.3.

PyObject *PyImport_ImportModuleLevel(const char *name, PyObject *globals, PyObject *locals, PyObject *fromlist, int level)

Retorna valor: Nova referência. Parte da ABI Estável.
Semelhante para **`PyImport_ImportModuleLevelObject()`**, mas o nome é uma string codificada em UTF-8 de um objeto Unicode.

Alterado na versão 3.3: Valores negativos para *level* não são mais aceitos.

PyObject *PyImport_Import(PyObject *name)

Retorna valor: Nova referência. Parte da ABI Estável.
Essa é uma interface de alto nível que chama a atual "função auxiliar de importação" (com um *level* explícito de 0, significando importação absoluta). Invoca a função **`__import__()`** a partir de `__builtins__` da global atual. Isso significa que a importação é feita usando quaisquer extras de importação instalados no ambiente atual.

Esta função sempre usa importações absolutas.

PyObject *PyImport_ReloadModule(PyObject *m)

Retorna valor: Nova referência. Parte da ABI Estável.
Recarrega um módulo. Retorna uma nova referência para o módulo recarregado, ou `NULL` com uma exceção definida em caso de falha (o módulo ainda existe neste caso).

PyObject *PyImport_AddModuleRef(const char *name)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.13.
Retorna o objeto do módulo correspondente ao nome do módulo.

O argumento *name* pode ter o formato `pacote.módulo`. Primeiro, verifica se existe um módulo no dicionário de módulos e, caso

contrário, cria um novo e insere-o no dicionário de módulos.

Retorna uma referência forte ao módulo em caso de sucesso. Retorna `NULL` com uma exceção definida em caso de falha.

O nome do módulo *name* é decodificado de UTF-8.

Esta função não carrega nem importa o módulo; se o módulo ainda não estiver carregado, você receberá um objeto de módulo vazio. Use **`PyImport_ImportModule()`** ou uma de suas variantes para importar um módulo. Estruturas de pacotes implícitas por um nome com pontos para *name* não são criadas se ainda não existirem.

Adicionado na versão 3.13.

PyObject *PyImport_AddModuleObject(PyObject *name)

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.7.
Semelhante a **`PyImport_AddModuleRef()`**, mas retorna uma referência emprestada e *name* é um objeto Python **`str`**.

Adicionado na versão 3.3.

PyObject *PyImport_AddModule(const char *name)

Retorna valor: Referência emprestada. Parte da ABI Estável.
Semelhante a **`PyImport_AddModuleRef()`**, mas retorna uma referência emprestada.

PyObject *PyImport_ExecCodeModule(const char *name, PyObject *co)

Retorna valor: Nova referência. Parte da ABI Estável.
Dado um nome de módulo (possivelmente no formato `pacote.módulo`) e um objeto código lido de um arquivo bytecode Python ou obtido da função embutida **`compile()`**, carrea o módulo. Retorna uma nova referência ao objeto do módulo ou `NULL` com uma exceção definida se ocorrer um erro. *name* é removido de **`sys.modules`** em casos de erro, mesmo que *name* já estivesse em **`sys.modules`** na entrada de **`PyImport_ExecCodeModule()`**. Deixar módulos incompletamente

inicializados em **sys.modules** é perigoso, pois as importações desses módulos não têm como saber que o objeto do módulo está em um estado desconhecido (e provavelmente corrompido em relação às intenções do autor do módulo).

Os atributos **__spec__** e **__loader__** do módulo serão definidos, caso ainda não estejam, com os valores apropriados. O carregador do spec será definido como o atributo **__loader__** do módulo (se definido) e, caso contrário, como uma instância de **SourceFileLoader**.

The module's **__file__** attribute will be set to the code object's **co_filename**.

Esta função recarregará o módulo se este já tiver sido importado. Veja **PyImport_ReloadModule()** para forma desejada de recarregar um módulo.

Se *name* apontar para um nome pontilhado no formato de `package.module`, quaisquer estruturas de pacote ainda não criadas ainda não serão criadas.

Veja também **PyImport_ExecCodeModuleEx()** e **PyImport_ExecCodeModuleWithPathnames()**.

Alterado na versão 3.12: The setting of `__cached__` and **__loader__** is deprecated. See **ModuleSpec** for alternatives.

Alterado na versão 3.15: The `__cached__` attribute is no longer set.

PyObject *PyImport_ExecCodeModuleEx(const char *name, PyObject *co, const char *pathname)

Retorna valor: Nova referência. Parte da ABI Estável.

Como **PyImport_ExecCodeModule()**, mas o atributo **__file__** do objeto módulo é definido como *pathname* se não for `NULL`.

Veja também **PyImport_ExecCodeModuleWithPathnames()**.

PyObject *PyImport_ExecCodeModuleObject(PyObject *name, PyObject *co, PyObject *pathname, PyObject *cpathname)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Like **PyImport_ExecCodeModuleEx()**, but the path to any compiled file via *cpathname* is used appropriately when non-`NULL`. Of the three functions, this is the preferred one to use.

Adicionado na versão 3.3.

Alterado na versão 3.12: Setting `__cached__` is deprecated. See **ModuleSpec** for alternatives.

Alterado na versão 3.15: The `__cached__` attribute no longer set.

PyObject *PyImport_ExecCodeModuleWithPathnames(const char *name, PyObject *co, const char *pathname, const char *cpathname)

Retorna valor: Nova referência. Parte da ABI Estável.

Como **PyImport_ExecCodeModuleObject()**, mas *name*, *pathname* e *cpathname* são strings codificadas em UTF-8. Também são feitas tentativas para descobrir qual valor para *pathname* deve ser de *cpathname* se o primeiro estiver definido como `NULL`.

Adicionado na versão 3.2.

Alterado na versão 3.3: Usa **imp.source_from_cache()** no cálculo do caminho de origem se apenas o caminho do bytecode for fornecido.

Alterado na versão 3.12: Não usa mais o módulo **imp** removido.

long PyImport_GetMagicNumber()

Parte da ABI Estável.

Retorna o número mágico para arquivos de bytecode Python (também conhecido como arquivo `.pyc`). O número mágico deve estar presente nos primeiros quatro bytes do arquivo bytecode, na ordem de bytes little-endian. Retorna `-1` em caso de erro.

Alterado na versão 3.3: Retorna o valor de `-1` no caso de falha.

const char *PyImport_GetMagicTag()

Parte da ABI Estável.

Retorna a string de tag mágica para nomes de arquivo de bytecode Python no formato de **PEP 3147** [https://peps.python.org/pep-3147/]. Tenha em mente que o valor em `sys.implementation.cache_tag` é autoritativo e deve ser usado no lugar desta função.

Adicionado na versão 3.2.

PyObject *PyImport_GetModuleDict()

Retorna valor: Referência emprestada. Parte da ABI Estável.

Retorna o dicionário usado para a administração do módulo (também conhecido como `sys.modules`). Observe que esta é uma variável por interpretador.

PyObject *PyImport_GetModule(PyObject *name)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.8.

Retorna o módulo já importado com o nome fornecido. Se o módulo ainda não foi importado, retorna `NULL`, mas não define um erro. Retorna `NULL` e define um erro se a pesquisa falhar.

Adicionado na versão 3.7.

PyObject *PyImport_GetImporter(PyObject *path)

Retorna valor: Nova referência. Parte da ABI Estável.

Return a finder object for a **`sys.path`**/**`pkg.__path__`** item *path*, possibly by fetching it from the **`sys.path_importer_cache`** dict. If it wasn't yet cached, traverse **`sys.path_hooks`** until a hook is found that can handle the path item. Return `None` if no hook could; this tells our caller that the path based finder could not find a finder for this path item. Cache the result in **`sys.path_importer_cache`**. Return a new reference to the finder object.

int PyImport_ImportFrozenModuleObject(PyObject *name)

Parte da ABI Estável desde a versão 3.7.
Carrega um módulo congelado chamado *name*. Retorna `1` para sucesso, `0` se o módulo não for encontrado e `-1` com uma exceção definida se a inicialização falhar. Para acessar o módulo importado em um carregamento bem-sucedido, use **`PyImport_ImportModule()`**. (Observe o nome incorreto — esta função recarregaria o módulo se ele já tivesse sido importado.)

Adicionado na versão 3.3.

Alterado na versão 3.4: O atributo `__file__` não está mais definido no módulo.

int PyImport_ImportFrozenModule(const char *name)

Parte da ABI Estável.
Semelhante a **`PyImport_ImportFrozenModuleObject()`**, mas o nome é uma string codificada em UTF-8 em vez de um objeto Unicode.

struct _frozen

Esta é a definição do tipo de estrutura para descritores de módulo congelados, conforme gerado pelo utilitário **freeze** (veja `Tools/freeze/` na distribuição fonte do Python). Sua definição, encontrada em `Include/import.h`, é:

```
struct _frozen {
    const char *name;
    const unsigned char *code;
    int size;
    bool is_package;
};
```

Alterado na versão 3.11: O novo campo `is_package` indica se o módulo é um pacote ou não. Isso substitui a configuração do campo `size` para um valor negativo.

const struct _frozen *PyImport_FrozenModules

Este ponteiro é inicializado para apontar para um vetor de registros de **`_frozen`**, terminado por um cujos membros são todos `NULL` ou zero.

Quando um módulo congelado é importado, ele é pesquisado nesta tabela. O código de terceiros pode fazer truques com isso para fornecer uma coleção criada dinamicamente de módulos congelados.

int PyImport_AppendInittab(const char *name, PyObject *(*initfunc)(void))

Parte da ABI Estável.

Adiciona um único módulo à tabela existente de módulos embutidos. Este é um invólucro prático em torno de **`PyImport_ExtendInittab()`**, retornando `-1` se a tabela não puder ser estendida. O novo módulo pode ser importado pelo nome *name* e usa a função *initfunc* como a função de inicialização chamada na primeira tentativa de importação. Deve ser chamado antes de **`Py_Initialize()`**.

struct _inittab

Structure describing a single entry in the list of built-in modules. Programs which embed Python may use an array of these structures in conjunction with **`PyImport_ExtendInittab()`** to provide additional built-in modules. The structure consists of two members:

const char *name

The module name, as an ASCII encoded string.

PyObject *(*initfunc)(void)

Initialization function for a module built into the interpreter.

Note that the inittab uses "`PyInit`" initialization functions; there is currently no way to include "`PyModExport_`" export hooks.

int PyImport_ExtendInittab(struct _inittab *newtab)

Add a collection of modules to the table of built-in modules. The *newtab* array must end with a sentinel entry which contains `NULL` for the **`name`** field; failure to provide the sentinel value can result in a memory fault. Returns `0` on success or `-1` if insufficient memory could be allocated to extend the internal table. In the event of failure, no modules are added to the internal table. This must be called before **`Py_Initialize()`**.

Se Python é inicializado várias vezes, `PyImport_AppendInittab()` ou `PyImport_ExtendInittab()` devem ser chamados antes de cada inicialização do Python.

struct _inittab *PyImport_Inittab

The table of built-in modules used by Python initialization. Do not use this directly; use `PyImport_AppendInittab()` and `PyImport_ExtendInittab()` instead.

PyObject *PyImport_ImportModuleAttr(PyObject *mod_name, PyObject *attr_name)

Retorna valor: Nova referência.

Import the module *mod_name* and get its attribute *attr_name*.

Names must be Python `str` objects.

Helper function combining `PyImport_Import()` and `PyObject_GetAttr()`. For example, it can raise `ImportError` if the module is not found, and `AttributeError` if the attribute doesn't exist.

Adicionado na versão 3.14.

PyObject *PyImport_ImportModuleAttrString(const char *mod_name, const char *attr_name)

Retorna valor: Nova referência.

Similar to `PyImport_ImportModuleAttr()`, but names are UTF-8 encoded strings instead of Python `str` objects.

Adicionado na versão 3.14.

PyImport_LazyImportsMode PyImport_GetLazyImportsMode()

Gets the current lazy imports mode.

Adicionado na versão 3.15.

PyObject *PyImport_GetLazyImportsFilter()

Return a strong reference to the current lazy imports filter, or `NULL` if none exists. This function always succeeds.

Adicionado na versão 3.15.

int PyImport_SetLazyImportsMode(PyImport_LazyImportsMode mode)

Similar to **`PyImport_ImportModuleAttr()`**, but names are UTF-8 encoded strings instead of Python **`str`** objects.

This function always returns `0`.

Adicionado na versão 3.15.

int PyImport_SetLazyImportsFilter(PyObject *filter)

Sets the current lazy imports filter. The *filter* should be a callable that will receive `(importing_module_name, imported_module_name, [fromlist])` when an import can potentially be lazy. The `imported_module_name` value is the resolved module name, so `lazy from .spam import eggs` passes `package.spam`. The callable must return `True` if the import should be lazy and `False` otherwise.

Return `0` on success and `-1` with an exception set otherwise.

Adicionado na versão 3.15.

type PyImport_LazyImportsMode

Enumeration of possible lazy import modes.

enumerator PyImport_LAZY_NORMAL

Respect the `lazy` keyword in source code. This is the default mode.

enumerator PyImport_LAZY_ALL

Make all imports lazy by default.

Adicionado na versão 3.15.

PyObject *PyImport_CreateModuleFromInitfunc(PyObject *spec, PyObject *(*initfunc)(void))

This function is a building block that enables embedders to implement the `create_module()` step of custom static extension importers (e.g. of statically-linked extensions).

spec must be a `ModuleSpec` object.

initfunc must be an initialization function, the same as for `PyImport_AppendInittab()`.

On success, create and return a module object. This module will not be initialized; call `PyModule_Exec()` to initialize it. (Custom importers should do this in their `exec_module()` method.)

On error, return NULL with an exception set.

Adicionado na versão 3.15.

Suporte a *marshalling* de dados

Essas rotinas permitem que o código C trabalhe com objetos serializados usando o mesmo formato de dados que o módulo `marshal`. Existem funções para gravar dados no formato de serialização e funções adicionais que podem ser usadas para ler os dados novamente. Os arquivos usados para armazenar dados empacotados devem ser abertos no modo binário.

Os valores numéricos são armazenados primeiro com o byte menos significativo.

O módulo oferece suporte a várias versões do formato de dados; consulte a `documentação do módulo Python` para obter detalhes.

Py_MARSHAL_VERSION

A versão atual do formato. Veja `marshal.version`.

void PyMarshal_WriteLongToFile(long value, FILE *file, int version)

Aplica *marshalling* em um inteiro long, *value*, para *file*. Isso escreverá apenas os 32 bits menos significativos de *value*; independentemente do tamanho do tipo nativo long. *version* indica o formato do arquivo.

Esta função pode falhar, caso em que define o indicador de erro. Use `PyErr_Occurred()` para verificar isso.

void PyMarshal_WriteObjectToFile(PyObject *value, FILE *file, int version)

Aplica *marshalling* em um objeto Python, *value*, para *file*. *version* indica o formato do arquivo.

Esta função pode falhar, caso em que define o indicador de erro. Use `PyErr_Occurred()` para verificar isso.

PyObject *PyMarshal_WriteObjectToString(PyObject *value, int version)

Retorna valor: Nova referência.
Retorna um objeto de bytes que contém a representação pós-*marshalling* de *value*. *version* indica o formato do arquivo.

As seguintes funções permitem que os valores pós-*marshalling* sejam lidos novamente.

long PyMarshal_ReadLongFromFile(FILE *file)

Retorna um long C do fluxo de dados em um FILE* aberto para leitura. Somente um valor de 32 bits pode ser lido usando essa função, independentemente do tamanho nativo de long.

Em caso de erro, define a exceção apropriada (`EOFError`) e retorna `-1`.

int PyMarshal_ReadShortFromFile(FILE *file)

Retorna um short C do fluxo de dados em um FILE* aberto para leitura. Somente um valor de 16 bits pode ser lido usando essa função, independentemente do tamanho nativo de short.

Em caso de erro, define a exceção apropriada (`EOFError`) e retorna `-1`.

PyObject *PyMarshal_ReadObjectFromFile(FILE *file)

Retorna valor: Nova referência.
Retorna um objeto Python do fluxo de dados em um FILE* aberto para leitura.

Em caso de erro, define a exceção apropriada (`EOFError`, `ValueError` ou `TypeError`) e retorna `NULL`.

PyObject *PyMarshal_ReadLastObjectFromFile(FILE *file)

Retorna valor: Nova referência.
Retorna um objeto Python do fluxo de dados em um FILE* aberto para leitura. Diferentemente de `PyMarshal_ReadObjectFromFile()`, essa função presume que nenhum objeto adicional será lido do arquivo, permitindo que ela carregue agressivamente os dados do arquivo na memória, para que a desserialização possa operar a partir de dados na

memória em vez de ler um byte por vez do arquivo. Use essa variante apenas se tiver certeza de que não estará lendo mais nada do arquivo.

Em caso de erro, define a exceção apropriada (`EOFError`, `ValueError` ou `TypeError`) e retorna `NULL`.

PyObject *PyMarshal_ReadObjectFromString(const char *data, Py_ssize_t len)

Retorna valor: Nova referência.
Retorna um objeto Python do fluxo de dados em um buffer de bytes contendo *len* bytes apontados por *data*.

Em caso de erro, define a exceção apropriada (`EOFError`, `ValueError` ou `TypeError`) e retorna `NULL`.

Análise de argumentos e construção de valores

Essas funções são úteis ao criar suas próprias funções e métodos de extensão. Informações adicionais e exemplos estão disponíveis em Estendendo e incorporando o interpretador Python.

As três primeiras funções descritas, `PyArg_ParseTuple()`, `PyArg_ParseTupleAndKeywords()`, e `PyArg_Parse()`, todas usam a *string de formatação* que informam à função sobre os argumentos esperados. As strings de formato usam a mesma sintaxe para cada uma dessas funções.

Análise de argumentos

Uma string de formato consiste em zero ou mais "unidades de formato". Uma unidade de formato descreve um objeto Python; geralmente é um único caractere ou uma sequência entre parênteses de unidades de formato. Com algumas poucas exceções, uma unidade de formato que não é uma sequência entre parênteses normalmente corresponde a um único argumento de endereço para essas funções. Na descrição a seguir, a forma citada é a unidade de formato; a entrada em parênteses ( ) é o tipo de objeto Python que corresponde à unidade de formato; e a entrada em colchetes [ ] é o tipo da variável(s) C cujo endereço deve ser passado.

Strings e buffers

Nota

No Python 3.12 e anteriores, a macro **`PY_SSIZE_T_CLEAN`** deve ser definida antes da inclusão de `Python.h` para usar todas as variantes no formato `#` (`s#`, `y#`, etc.) explicadas abaixo. Isso não é necessário no Python 3.13 e posteriores.

Esses formatos permitem acessar um objeto como um pedaço contíguo de memória. Você não precisa fornecer armazenamento bruto para a área de unicode ou bytes retornada.

Salvo indicação em contrário, os buffers não são terminados em NUL.

Existem três maneiras pelas quais strings e buffers podem ser convertidos em C:

- Formatos como `y*` e `s*` estão dentro de uma estrutura `Py_buffer`. Isso bloqueia o buffer subjacente para que o chamador possa posteriormente usar o buffer, mesmo dentro de um bloco `Py_BEGIN_ALLOW_THREADS` sem que haja o risco de que dados mutáveis sejam redimensionados ou destruídos. Dessa forma, **você precisa chamar** `PyBuffer_Release()` depois de ter concluído o processamento de dados (ou em qualquer caso de interrupção precoce).

- Os formatos `es`, `es#`, `et` e `et#` alocam o buffer resultante. **Você precisa chamar** `PyMem_Free()` depois de ter concluído o processamento de dados (ou em qualquer caso de interrupção precoce).

- Outros formatos usam um `str` ou um objeto bytes ou similar somente leitura, como `bytes`, e fornecem um ponteiro `const char *` para seu buffer. Nesse caso, o buffer é "emprestado": ele é gerenciado pelo objeto Python correspondente e compartilha o tempo de vida desse objeto. Você mesmo não precisará liberar nenhuma memória.

  Para garantir que o buffer subjacente possa ser emprestado com segurança, o campo `PyBufferProcs.bf_releasebuffer` do objeto deve ser `NULL`. Isso não permite objetos mutáveis comuns, como `bytearray`, mas também alguns objetos somente leitura, como `memoryview` ou `bytes`.

  Além desse requisito `bf_releasebuffer`, não há nenhuma verificação para saber se o objeto de entrada é imutável (por exemplo, se ele atenderia a uma solicitação de um buffer gravável ou se outro thread pode alterar os dados).

`s` (`str`) [const char *]

Converte um objeto Unicode para um ponteiro em C para uma string. Um ponteiro para uma string existente é armazenado na variável do ponteiro do caractere cujo o endereço que você está passando. A string em C é terminada em NULO. A string no Python não deve conter pontos de código nulo embutidos; se isso acontecer, uma exceção `ValueError` é levantada. Objetos Unicode são convertidos para strings em C usando a codificação `'utf-8'`. Se essa conversão falhar, uma exceção `UnicodeError` é levantada.

Nota

Esse formato não aceita objetos bytes ou similares. Se você quer aceitar caminhos de sistema de arquivos e convertê-los para strings de caracteres em C, é preferível que use o formato `O&` com `PyUnicode_FSConverter()` como *conversor*.

Alterado na versão 3.5: Anteriormente, a exceção `TypeError` era levantada quando pontos de código nulo embutidos em string Python eram encontrados.

`s*` (`str` ou objeto bytes ou similar) [Py_buffer]

Esse formato aceita tanto objetos Unicode quanto objetos bytes ou similares. Preenche uma estrutura `Py_buffer` fornecida pelo chamador. Nesse caso, a string em C resultante pode conter bytes NUL embutidos. Objetos Unicode são convertidos para strings em C usando codificação `'utf-8'`.

`s#` (`str`, objeto bytes ou similar somente leitura) [const char *, `Py_ssize_t`]

Como `s*`, exceto que não fornece um buffer emprestado. O resultado é armazenado em duas variáveis em C, a primeira é um ponteiro para uma string em C, a segunda é o tamanho. A string pode conter bytes nulos embutidos. Objetos Unicode são convertidos para strings em C usando codificação `'utf-8'`.

`z` (`str` ou `None`) [const char *]

Como `s`, mas o objeto Python também pode ser `None`, caso em que o ponteiro C é definido como `NULL`.

`z*` (**`str`**, objeto bytes ou similar ou `None`) [Py_buffer]

Como `s*`, mas o objeto Python também pode ser `None`, caso em que o membro `buf` da estrutura **`Py_buffer`** é definida como `NULL`.

`z#` (**`str`**, objeto bytes ou similar somente leitura ou `None`) [const char *, **`Py_ssize_t`**]

Como `s#`, mas o objeto Python também pode ser `None`, caso em que o ponteiro C é definido como `NULL`.

`y` (objeto bytes ou similar somente leitura) [const char *]

Este formato converte um objeto bytes ou similar para um ponteiro C para uma string de caracteres emprestada; não aceita objetos Unicode. O buffer de bytes não pode conter bytes nulos embutidos; se isso ocorrer uma exceção **`ValueError`** será levantada.

Alterado na versão 3.5: Anteriormente, a exceção **`TypeError`** era levantada quando pontos de código nulo embutidos em string Python eram encontrados no buffer de bytes.

`y*` (objeto bytes ou similar) [Py_buffer]

Esta variante em `s*` não aceita objetos unicode, apenas objetos bytes ou similares. **Esta é a maneira recomendada para aceitar dados binários.**

`y#` (objeto bytes ou similar somente leitura) [const char *, **`Py_ssize_t`**]

Esta variação de `s#` não aceita objetos Unicode, apenas objetos bytes ou similares.

`S` (**`bytes`**) [PyBytesObject *]

Exige que o objeto Python seja um objeto **`bytes`**, sem tentar nenhuma conversão. Levanta **`TypeError`** se o objeto não for um objeto byte. A variável C pode ser declarada como PyObject*.

`Y` (**`bytearray`**) [PyByteArrayObject *]

Exige que o objeto Python seja um objeto **bytearray**, sem aceitar qualquer conversão. Levanta **TypeError** se o objeto não é um objeto **bytearray**. A variável C apenas pode ser declarada como PyObject*.

`U` (**str**) [PyObject *]

Exige que o objeto python seja um objeto Unicode, sem tentar alguma conversão. Levanta **TypeError** se o objeto não for um objeto Unicode. A variável C deve ser declarada como PyObject*.

`w*` (objeto bytes ou similar de leitura e escrita) [Py_buffer]

Este formato aceita qualquer objeto que implemente a interface do buffer de leitura e escrita. Ele preenche uma estrutura **Py_buffer** fornecida pelo chamador. O buffer pode conter bytes nulos incorporados. O chamador deve chamar **PyBuffer_Release()** quando isso for feito com o buffer.

`es` (**str**) [const char *encoding, char **buffer]

Esta variante em `s` é utilizada para codificação do Unicode em um buffer de caracteres. Ele só funciona para dados codificados sem NUL bytes incorporados.

Este formato exige dois argumentos. O primeiro é usado apenas como entrada e deve ser a const char* que aponta para o nome de uma codificação como uma string terminada em NUL ou `NULL`, nesse caso a codificação `'utf-8'` é usada. Uma exceção é levantada se a codificação nomeada não for conhecida pelo Python. O segundo argumento deve ser um char**; o valor do ponteiro a que ele faz referência será definido como um buffer com o conteúdo do texto do argumento. O texto será codificado na codificação especificada pelo primeiro argumento.

PyArg_ParseTuple() alocará um buffer do tamanho necessário, copiará os dados codificados nesse buffer e ajustará **buffer* para referenciar o armazenamento recém-alocado. O chamador é responsável por chamar **PyMem_Free()** para liberar o buffer alocado após o uso.

`et` (**str**, **bytes** ou **bytearray**) [const char *encoding, char **buffer]

O mesmo que `es`, exceto que os objetos strings de bytes são passados sem os recodificar. Em vez disso, a implementação presume que o objeto string de bytes usa a codificação passada como parâmetro.

`es#` (`str`) [const char *encoding, char **buffer, `Py_ssize_t` *buffer_length]

Essa variante em `s#` é usada para codificar Unicode em um buffer de caracteres. Diferente do formato `es`, essa variante permite a entrada de dados que contêm caracteres NUL.

Exige três argumentos. O primeiro é usado apenas como entrada e deve ser a const char* que aponta para o nome de uma codificação como uma string terminada em NUL ou `NULL`, nesse caso a codificação `'utf-8'` é usada. Uma exceção será gerada se a codificação nomeada não for conhecida pelo Python. O segundo argumento deve ser um char**; o valor do ponteiro a que ele faz referência será definido como um buffer com o conteúdo do texto do argumento. O texto será codificado na codificação especificada pelo primeiro argumento. O terceiro argumento deve ser um ponteiro para um número inteiro; o número inteiro referenciado será definido como o número de bytes no buffer de saída.

Há dois modos de operação:

Se **buffer* apontar um ponteiro `NULL`, a função irá alocar um buffer do tamanho necessário, copiar os dados codificados para dentro desse buffer e configurar **buffer* para referenciar o novo armazenamento alocado. O chamador é responsável por chamar `PyMem_Free()` para liberar o buffer alocado após o uso.

Se **buffer* apontar para um ponteiro que não seja `NULL` (um buffer já alocado), `PyArg_ParseTuple()` irá usar essa localização como buffer e interpretar o valor inicial de **buffer_length* como sendo o tamanho do buffer. Depois ela vai copiar os dados codificados para dentro do buffer e terminá-lo com NUL. Se o buffer não for suficientemente grande, um `ValueError` será definido.

Em ambos os casos, o **buffer_length* é definido como o comprimento dos dados codificados sem o byte NUL à direita.

`et#` (`str`, `bytes` ou `bytearray`) [const char *encoding, char **buffer, `Py_ssize_t` *buffer_length]

O mesmo que `es#`, exceto que os objetos strings de bytes são passados sem que sejam recodificados. Em vez disso, a implementação presume que o objeto string de bytes usa a codificação passada como parâmetro.

Alterado na versão 3.12: `u`, `u#`, `Z` e `Z#` foram removidos porque usavam uma representação herdada de `Py_UNICODE*`.

Números

Esses formatos permitem representar números Python ou caracteres únicos como números C. Formatos que exigem `int`, `float` ou `complex` também podem usar os métodos especiais correspondentes `__index__()`, `__float__()` ou `__complex__()` para converter o objeto Python para o tipo necessário.

Para formatos de inteiro com sinal, uma `OverflowError` é levantada se o valor estiver fora da faixa permitida para o tipo C. Para formatos de inteiro sem sinal, os bits mais significativos são truncados silenciosamente quando o campo de destino é pequeno demais para receber o valor, e uma `DeprecationWarning` é emitido quando o valor é maior que o valor máximo para o tipo C ou menor que o valor mínimo para o tipo de inteiro com sinal correspondente de mesmo tamanho.

`b` (`int`) [unsigned char]

Converte um inteiro Python não negativo em um inteiro pequeno sem sinal (unsigned tiny integer), armazenado em um unsigned char do C.

`B` (`int`) [unsigned char]

Converte um inteiro Python para um inteiro pequeno (tiny integer) sem verificação de estouro, armazenado em um unsigned char do C.
Converte um inteiro Python para unsigned char do C.

`h` (`int`) [short int]

Converte um inteiro Python para um short int do C.

`H` (`int`) [unsigned short int]
Converte um inteiro Python para C unsigned short int.

`i` (`int`) [int]
Converte um inteiro Python para um int simples do C.

`I` (`int`) [unsigned int]
Converte um inteiro Python para C unsigned int.

`l` (`int`) [long int]
Converte um inteiro Python para um long int do C.

`k` (`int`) [unsigned long]
Converte um inteiro Python para C unsigned long.

Alterado na versão 3.14: Usa `__index__()`, se disponível.

`L` (`int`) [longo longo]
Converte um inteiro Python para um long long do C.

`K` (`int`) [unsigned long long]
Converte um inteiro Python para C unsigned long long.

Alterado na versão 3.14: Usa `__index__()`, se disponível.

`n` (`int`) [`Py_ssize_t`]
Converte um inteiro Python para um `Py_ssize_t` do C.

`c` (`bytes` ou `bytearray` de comprimento 1) [char]
Converte um byte Python, representado com um objeto **`byte`** ou `bytearray` de comprimento 1, para um char do C.

Alterado na versão 3.3: Permite objetos `bytearray`.

`C` (`str` de comprimento 1) [int]
Converte um caractere Python, representado como uma `str` objeto de comprimento 1, para um int do C

`f` (**`float`**) [float]
Converte um número de ponto flutuante Python para um float do C.

`d` (**`float`**) [double]
Converte um número de ponto flutuante Python para um double do C.

`D` (**`complex`**) [Py_complex]
Converte um número complexo Python para uma estrutura C **`Py_complex`**

Descontinuado desde a versão 3.15: Para os formatos de inteiro sem sinal `B`, `H`, `I`, `k` e `K`, é emitida uma **`DeprecationWarning`** quando o valor é maior que o valor máximo para o tipo C ou menor que o valor mínimo para o tipo de inteiro com sinal correspondente de mesmo tamanho.

Outros objetos

`O` (objeto) [PyObject*]
Armazena um objeto Python (sem qualquer conversão) em um ponteiro de objeto C. O programa C então recebe o objeto real que foi passado. Uma nova referência forte ao objeto não é criado (isto é sua contagem de referências não é aumentada). O ponteiro armazenado não é `NULL`.

`O!` (objeto) [*typeobject*, PyObject *]
Armazena um objeto Python em um ponteiro de objeto C. Isso é similar a `O`, mas usa dois argumentos C: o primeiro é o endereço de um objeto do tipo Python, o segundo é um endereço da variável C (de tipo PyObject*) no qual o ponteiro do objeto está armazenado. Se o objeto Python não tiver o tipo necessário, **`TypeError`** é levantada.

`O&` (objeto) [*converter*, *address*]
Converte um objeto Python em uma variável C através de uma função *converter*. Isso leva dois argumentos: o primeiro é a função, o segundo é o endereço da variável C (de tipo arbitrário), convertendo para void*. A função *converter* por sua vez, é chamada da seguinte maneira:

```
status = converter(object, address);
```

onde *object* é o objeto Python a ser convertido e *address* é o argumento void* que foi passado para a função `PyArg_Parse*`. O *status* retornado deve ser `1` para uma conversão bem-sucedida e `0` se a conversão falhar. Quando a conversão falha, a função *converter* deve levantar uma exceção e deixar o conteúdo de *address* inalterado.

Se o *converter* retornar **`Py_CLEANUP_SUPPORTED`**, ele poderá ser chamado uma segunda vez se a análise do argumento eventualmente falhar, dando ao conversor a chance de liberar qualquer memória que já havia alocado. Nesta segunda chamada, o parâmetro *object* será `NULL`; *address* terá o mesmo valor que na chamada original.

Exemplos de conversores: **`PyUnicode_FSConverter()`** e **`PyUnicode_FSDecoder()`**.

Alterado na versão 3.1: **`Py_CLEANUP_SUPPORTED`** foi adicionado.

`p` (**`bool`**) [int]

Testa o valor transmitido para a verdade (um booleano **p**redicado) e converte o resultado em seu valor inteiro C verdadeiro/falso equivalente. Define o int como `1` se a expressão for verdadeira e `0` se for falsa. Isso aceita qualquer valor válido do Python. Veja Teste do valor verdade para obter mais informações sobre como o Python testa valores para a verdade.

Adicionado na versão 3.3.

`(items)` (sequência) [*itens-correspondentes*]

O objeto deve ser uma sequência Python (exceto **`str`**, **`bytes`** ou **`bytearray`**) cujo comprimento é o número de unidades de formato em *itens*. Os argumentos C devem corresponder às unidades de formato individuais em *itens*. As unidades de formato para sequências podem ser aninhadas.

Se *items* contiver unidades de formato que armazenam um buffer emprestado (`s`, `s#`, `z`, `z#`, `y` ou `y#`) ou uma referência emprestada (`S`, `Y`, `U`, `O` ou `O!`), o objeto deve ser uma tupla Python. O *conversor* para a

unidade de formato `O&` em *items* não deve armazenar um buffer emprestado ou uma referência emprestada.

Alterado na versão 3.14: Objetos `str` e `bytearray` não são mais aceitos como sequência.

Descontinuado desde a versão 3.14: Sequências não-tuplas serão descontinuadas se *items* contiver unidades de formato que armazenem um buffer emprestado ou uma referência emprestada.

Alguns outros caracteres possuem significados na string de formatação. Isso pode não ocorrer dentro de parênteses aninhados. Eles são:

`|`

Indica que os argumentos restantes na lista de argumentos do Python são opcionais. As variáveis C correspondentes a argumentos opcionais devem ser inicializadas para seus valores padrão — quando um argumento opcional não é especificado, `PyArg_ParseTuple()` não toca no conteúdo da(s) variável(eis) C correspondente(s). Por exemplo, a string de formato `"OO|OO"` corresponde à assinatura Python `f(a, b, c=None, d=None)`.

`$`

Apenas para `PyArg_ParseTupleAndKeywords()`: Indica que os argumentos restantes na lista de argumentos do Python são argumentos somente-nomeado. Eles são opcionais se `|` tiver sido especificado antes de `$` e obrigatórios caso contrário. `|` não pode ser especificado após `$`. Por exemplo, a string de formato `"O|O$O"` corresponde à assinatura Python `f(a, b=None, *, c=None)`, e a string de formato `"OO$OO"` corresponde a `f(a, b, *, c, d)`.

Adicionado na versão 3.3.

`:`

A lista de unidades de formatação acaba aqui; a string após os dois pontos é usada como o nome da função nas mensagens de erro (o “valor associado” da exceção que `PyArg_ParseTuple()` levanta).

`;`

A lista de unidades de formatação acaba aqui; a string após o ponto e vírgula é usada como a mensagem de erro *ao invés* da mensagem de erro padrão. `:` e `;` se excluem mutuamente.

Note que quaisquer referências a objeto Python que são fornecidas ao chamador são referências *emprestadas*; não libera-as (isto é, não decremente a contagem de referências delas)!

Argumentos adicionais passados para essas funções devem ser endereços de variáveis cujo tipo é determinado pela string de formatação; estes são usados para armazenar valores vindos da tupla de entrada. Existem alguns casos, como descrito na lista de unidades de formatação acima, onde esses parâmetros são usados como valores de entrada; eles devem concordar com o que é especificado para a unidade de formatação correspondente nesse caso.

Para a conversão funcionar, o objeto *arg* deve corresponder ao formato e o formato deve estar completo. Em caso de sucesso, as funções `PyArg_Parse*` retornam verdadeiro, caso contrário retornam falso e levantam uma exceção apropriada. Quando as funções `PyArg_Parse*` falham devido a uma falha de conversão em uma das unidades de formatação, as variáveis nos endereços correspondentes àquela unidade e às unidades de formatação seguintes são deixadas intocadas.

Funções da API

int PyArg_ParseTuple(PyObject *args, const char *format, ...)

Parte da ABI Estável.

Analisa os parâmetros de uma função que recebe apenas parâmetros posicionais em variáveis locais. Retorna verdadeiro em caso de sucesso; em caso de falha, retorna falso e levanta a exceção apropriada.

int PyArg_VaParse(PyObject *args, const char *format, va_list vargs)

Parte da ABI Estável.

Idêntico a **`PyArg_ParseTuple()`**, exceto que aceita uma va_list ao invés de um número variável de argumentos.

int PyArg_ParseTupleAndKeywords(PyObject *args, PyObject *kw, const char *format, char *const *keywords, ...)

Parte da ABI Estável.

Analisa os parâmetros de uma função que aceita tanto parâmetros posicionais quando parâmetros nomeados e os converte em variáveis locais. O argumento *keywords* é um array terminado em `NULL` de nomes de parâmetros nomeados especificados como strings C codificadas em ASCII ou UTF-8 e terminadas com nulo. Nomes vazios representam parâmetros somente-posicional. Retorna verdadeiro em caso de sucesso; em caso de falha, retorna falso e levanta a exceção apropriada.

Nota

A declaração do parâmetro *keywords* é char *const* em C e const char *const* em C++. Isso pode ser substituído com a macro **`PY_CXX_CONST`**.

Alterado na versão 3.6: Adicionado suporte para positional-only parameters.

Alterado na versão 3.13: O parâmetro *keywords* é agora do tipo char *const* em C e const char *const* em C++, no lugar de char**. Adicionado suporte para nomes de parâmetros nomeados não-ASCII.

int PyArg_VaParseTupleAndKeywords(PyObject *args, PyObject *kw, const char *format, char *const *keywords, va_list vargs)

Parte da ABI Estável.

Idêntico a **`PyArg_ParseTupleAndKeywords()`**, exceto que aceita uma va_list ao invés de um número variável de argumentos.

int PyArg_ValidateKeywordArguments(PyObject*)

Parte da ABI Estável.

Garante que as chaves no dicionário de argumento de palavras reservadas são strings. Isso só é necessário se **`PyArg_ParseTupleAndKeywords()`** não é usado, já que o último já faz essa checagem.

Adicionado na versão 3.2.

int PyArg_Parse(PyObject *args, const char *format, ...)

Parte da ABI Estável.

Analisa o parâmetro de uma função que recebe um único parâmetro posicional e o converte em uma variável local. Retorna verdadeiro em caso de sucesso; em caso de falha, retorna falso e levanta a exceção apropriada.

Exemplo:

```
// Função usando a convenção de chamada ao método METH_O
static PyObject*
my_function(PyObject *module, PyObject *arg)
{
    int value;
    if (!PyArg_Parse(arg, "i:my_function", &value)) {
        return NULL;
    }
    // ... usar valor ...
}
```

int PyArg_ParseArray(PyObject *const *args, Py_ssize_t nargs, const char *format, ...)

Analisa os parâmetros de uma função que aceita somente parâmetros de vetor para variáveis locais (ou seja, uma função que use a convenção de chamada **METH_FASTCALL**). Retorna verdadeiro em caso de sucesso; em caso de falha, retorna falso e levanta a exceção apropriada.

Adicionado na versão 3.15.

int PyArg_ParseArrayAndKeywords(PyObject *const *args, Py_ssize_t nargs, PyObject *kwnames, const char *format, const char *const *kwlist, ...)

Analisa os parâmetros de uma função que aceite tanto parâmetros de vetor e parâmetros nomeados para variáveis locais (ou seja, uma função que use **METH_FASTCALL** | **METH_KEYWORDS** convenções de chamada).

Retorna verdadeiro em caso de sucesso; em caso de falha, retorna falso e levanta a exceção apropriada.

Adicionado na versão 3.15.

int PyArg_UnpackTuple(PyObject *args, const char *name, Py_ssize_t min, Py_ssize_t max, ...)

Parte da ABI Estável.

Uma forma mais simples de recuperação de parâmetro que não usa uma string de formato para especificar os tipos de argumentos. Funções que usam este método para recuperar seus parâmetros devem ser declaradas como **METH_VARARGS** em tabelas de função ou método. A tupla contendo os parâmetros reais deve ser passada como *args*; deve realmente ser uma tupla. O comprimento da tupla deve ser de pelo menos *min* e não mais do que *max*; *min* e *max* podem ser iguais. Argumentos adicionais devem ser passados para a função, cada um dos quais deve ser um ponteiro para uma variável PyObject*; eles serão preenchidos com os valores de *args*; eles conterão referências emprestadas. As variáveis que correspondem a parâmetros opcionais não fornecidos por *args* não serão preenchidas; estes devem ser inicializados pelo chamador. Esta função retorna verdadeiro em caso de sucesso e falso se *args* não for uma tupla ou contiver o número incorreto de elementos; uma exceção será definida se houver uma falha.

Este é um exemplo do uso dessa função, tirado das fontes do módulo auxiliar para referências fracas **_weakref**:

```
static PyObject *
weakref_ref(PyObject *self, PyObject *args)
{
    PyObject *object;
    PyObject *callback = NULL;
    PyObject *result = NULL;

    if (PyArg_UnpackTuple(args, "ref", 1, 2, &object, &callba
        result = PyWeakref_NewRef(object, callback);
    }
    return result;
}
```

A chamada à **PyArg_UnpackTuple()** neste exemplo é inteiramente equivalente à chamada para **PyArg_ParseTuple()**:

```
PyArg_ParseTuple(args, "O|O:ref", &object, &callback)
```

PY_CXX_CONST

O valor para ser inserido, se houver, antes de char *const* na declaração do parâmetro *keywords* das funções **PyArg_ParseTupleAndKeywords()** e **PyArg_VaParseTupleAndKeywords()**. Por padrão, é vazio para C e `const` for C++ (const char *const*). Para substituir, defina o valor desejado antes de incluir `Python.h`.

Adicionado na versão 3.13.

Construindo valores

PyObject *Py_BuildValue(const char *format, ...)

Retorna valor: Nova referência. Parte da ABI Estável.

Cria um novo valor baseado em uma string de formatação similar àquelas aceitas pela família de funções `PyArg_Parse*` e uma sequência de valores. Retorna o valor ou `NULL` em caso de erro; uma exceção será levantada se `NULL` for retornado.

Py_BuildValue() não constrói sempre uma tupla. Ela constrói uma tupla apenas se a sua string de formatação contém duas ou mais unidades de formatação. Se a string de formatação estiver vazia, ela retorna `None`; se ela contém exatamente uma unidade de formatação, ela retorna qualquer que seja o objeto que for descrito pela unidade de formatação. Para forçar ela a retornar uma tupla de tamanho 0 ou um, use parênteses na string de formatação.

Quando buffers de memória são passados como parâmetros para fornecer dados para construir objetos, como nos formatos `s` e `s#`, os dados necessários são copiados. Buffers fornecidos pelo chamador nunca são referenciados pelos objetos criados por **Py_BuildValue()**. Em outras palavras, se o seu código invoca **`malloc()`** e passa a memória

alocada para **`Py_BuildValue()`**, seu código é responsável por chamar **`free()`** para aquela memória uma vez que **`Py_BuildValue()`** tiver retornado.

Na descrição a seguir, a forma entre aspas é a unidade de formatação; a entrada em parênteses (arredondado) é o tipo do objeto Python que a unidade de formatação irá retornar; e a entrada em colchetes [quadrado] é o tipo do(s) valor(es) C a ser(em) passado(s).

Os caracteres de espaço, tab, dois pontos e vírgula são ignorados em strings de formatação (mas não dentro de unidades de formatação como `s#`). Isso pode ser usado para tornar strings de formatação longas um pouco mais legíveis.

`s` (**`str`** ou `None`) [const char *]
: Converte uma string C terminada em NULL em um objeto Python **`str`** usando codificação `'utf-8'`. Se o ponteiro da string C é `NULL`, `None` é usado.

`s#` (**`str`** ou `None`) [const char *, **`Py_ssize_t`**]
: Converte uma string C e seu comprimento em um objeto Python **`str`** usando a codificação `'utf-8'`. Se o ponteiro da string C é `NULL`, o comprimento é ignorado e `None` é retornado.

`y` (**`bytes`**) [const char *]
: Isso converte uma string C para um objeto Python **`bytes`**. Se o ponteiro da string C é `NULL`, `None` é retornado.

`y#` (**`bytes`**) [const char *, **`Py_ssize_t`**]
: Isso converte uma string C e seu comprimento para um objeto Python. Se o ponteiro da string C é `NULL`, `None` é retornado.

`z` (**`str`** ou `None`) [const char *]
: O mesmo de `s`.

`z#` (**`str`** ou `None`) [const char *, **`Py_ssize_t`**]
: O mesmo de `s#`.

`u` (**`str`**) [const wchar_t *]
Converte um buffer terminado por null **`wchar_t`** de dados Unicode (UTF-16 ou UCS-4) para um objeto Python Unicode. Se o ponteiro do buffer Unicode é `NULL`, `None` é retornado.

`u#` (**`str`**) [const wchar_t *, **`Py_ssize_t`**]
Converte um buffer de dados Unicode (UTF-17 ou UCS-4) e seu comprimento em um objeto Python Unicode. Se o ponteiro do buffer Unicode é `NULL`, o comprimento é ignorado e `None` é retornado.

`U` (**`str`** ou `None`) [const char *]
O mesmo de `s`.

`U#` (**`str`** ou `None`) [const char *, **`Py_ssize_t`**]
O mesmo de `s#`.

`i` (**`int`**) [int]
Converte um simples int do C em um objeto inteiro do Python.

`b` (**`int`**) [char]
Converte um simples char do C em um objeto inteiro do Python.

`h` (**`int`**) [short int]
Converte um simples short int do C em um objeto inteiro do Python.

`l` (**`int`**) [long int]
Converte um long int do C em um objeto inteiro do Python.

`B` (**`int`**) [unsigned char]
Converte um unsigned char do C em um objeto inteiro do Python.

`H` (**`int`**) [unsigned short int]
Converte um unsigned short int do C em um objeto inteiro do Python.

`I` (**`int`**) [unsigned int]
Converte um unsigned int do C em um objeto inteiro do Python.

`k` (`int`) [unsigned long]
Converte um unsigned long do C em um objeto inteiro do Python.

`L` (`int`) [longo longo]
Converte um long long do C em um objeto inteiro do Python.

`K` (`int`) [unsigned long long]
Converte um unsigned long long do C em um objeto inteiro do Python.

`n` (`int`) [`Py_ssize_t`]
Converte um `Py_ssize_t` do C em um objeto inteiro do Python.

`p` (`bool`) [int]
Converte um int C para um objeto `bool` Python.

Esteja ciente de que este formato requer um argumento `int`. Ao contrário da maioria dos outros contextos em C, argumentos variádicos não são convertidos automaticamente para um tipo adequado. Você pode converter outro tipo (por exemplo, um ponteiro ou um float) para um valor `int` adequado usando `(x) ? 1 : 0` ou `!!x`.

Adicionado na versão 3.14.

`c` (`bytes` de comprimento 1) [char]
Converte um int representando um byte do C em um objeto `bytes` de comprimento 1 do Python.

`C` (`str` de comprimento 1) [int]
Converte um int representando um caractere do C em um objeto `str` de comprimento 1 do Python.

`d` (`float`) [double]
Converte um double do C em um número ponto flutuante do Python.

`f` (`float`) [float]

Converte um float do C em um número ponto flutuante do Python.

`D` (**`complex`**) [Py_complex *]

Converte uma estrutura **`Py_complex`** do C em um número complexo do Python.

`O` (objeto) [PyObject*]

Passa um objeto Python intocado, mas cria uma nova referência forte a ele (isto é, sua contagem de referências é incrementada por um). Se o objeto passado é um ponteiro `NULL`, presume-se que isso foi causado porque a chamada que produziu o argumento encontrou um erro e definiu uma exceção. Portanto, **`Py_BuildValue()`** irá retornar `NULL` mas não irá levantar uma exceção. Se nenhuma exceção foi levantada ainda, **`SystemError`** é definida.

`S` (objeto) [PyObject *]

O mesmo que `O`.

`N` (objeto) [PyObject *]

O mesmo que `O`, exceto que não cria uma nova referência forte. Útil quando o objeto é criado por uma chamada a um construtor de objeto na lista de argumento.

`O&` (objeto) [*converter*, *anything*]

Converte *anything* para um objeto Python através de uma função *converter*. A função é chamada com *anything* (que deve ser compatível com o void*) como argumento e deve retornar um "novo" objeto Python, ou `NULL` se um erro ocorreu.

`(items)` (**`tuple`**) [*matching-items*]

Converte uma sequência de valores C para uma tupla Python com o mesmo número de itens.

`[items]` (**`list`**) [*matching-items*]

Converte uma sequência de valores C para uma lista Python com o mesmo número de itens.

`{items}` (**`dict`**) [*matching-items*]

Converte uma sequência de valores C para um dicionário Python. Cada par de valores consecutivos do C adiciona um item ao dicionário, servindo como chave e valor, respectivamente.

Se existir um erro na string de formatação, a exceção **`SystemError`** é definida e `NULL` é retornado.

PyObject *Py_VaBuildValue(const char *format, va_list vargs)

Retorna valor: Nova referência. Parte da ABI Estável.

Idêntico a **`Py_BuildValue()`**, exceto que aceita uma va_list ao invés de um número variável de argumentos.

Conversão e formação de strings

Funções para conversão de números e saída formatada de strings.

int PyOS_snprintf(char *str, size_t size, const char *format, ...)

Parte da ABI Estável.
Saída não superior a *size* bytes para *str* de acordo com a string de formato *format* e os argumentos extras. Veja a página man do Unix *snprintf(3)* *[https://manpages.debian.org/snprintf(3)]*.

int PyOS_vsnprintf(char *str, size_t size, const char *format, va_list va)

Parte da ABI Estável.
Saída não superior a *size* bytes para *str* de acordo com o string de formato *format* e a variável argumento de lista *va*. Página man do Unix *vsnprintf(3)* *[https://manpages.debian.org/vsnprintf(3)]*.

`PyOS_snprintf()` e **`PyOS_vsnprintf()`** envolvem as funções **`snprintf()`** e **`vsnprintf()`** da biblioteca Standard C. Seu objetivo é garantir um comportamento consistente em casos extremos, o que as funções do Standard C não garantem.

Os invólucros garantem que `str[size-1]` seja sempre `'\0'` no retorno. Eles nunca escrevem mais do que *size* bytes (incluindo o `'\0'` ao final) em str. Ambas as funções exigem que `str != NULL`, `size > 0`, `format != NULL` e `size < INT_MAX`. Note que isso significa que não há equivalente ao `n = snprintf(NULL, 0, ...)` do C99 que determinaria o tamanho de buffer necessário.

O valor de retorno (*rv*) para essas funções deve ser interpretado da seguinte forma:

- Quando `0 <= rv < size`, a conversão de saída foi bem-sucedida e os caracteres de *rv* foram escritos em *str* (excluindo o `'\0'` byte em `str[rv]`).

- Quando `rv >= size`, a conversão de saída foi truncada e um buffer com `rv + 1` bytes teria sido necessário para ter sucesso. `str[size-1]` é `'\0'` neste caso.
- Quando `rv < 0`, a conversão da saída falhou e `str[size-1]` é `'\0'` neste caso também, mas o resto de *str* é indefinido. A causa exata do erro depende da plataforma subjacente.

As funções a seguir fornecem strings independentes de localidade para conversões de números.

unsigned long PyOS_strtoul(const char *str, char **ptr, int base)

Parte da ABI Estável.

Converte a parte inicial da string em `str` para um valor unsigned long de acordo com a `base` fornecida, que deve estar entre `2` e `36` inclusive, ou ser o valor especial `0`.

Espaços em branco iniciais e diferenciação entre caracteres maiúsculos e minúsculos são ignorados. Se `base` for zero, procura um `0b`, `0o` ou `0x` inicial para informar qual base. Se estes estiverem ausentes, o padrão é `10`. Base deve ser 0 ou entre 2 e 36 (inclusive). Se `ptr` não for `NULL`, conterá um ponteiro para o fim da varredura.

Se o valor convertido ficar fora do intervalo do tipo de retorno correspondente, ocorrerá um erro de intervalo (**`errno`** é definido como **`ERANGE`**) e **`ULONG_MAX`** será retornado. Se nenhuma conversão puder ser realizada, `0` será retornado.

Veja também a página man do Unix *strtoul(3)* *[https://manpages.debian.org/strtoul(3)]*.

Adicionado na versão 3.2.

long PyOS_strtol(const char *str, char **ptr, int base)

Parte da ABI Estável.

Converte a parte inicial da string em `str` para um valor long de acordo com a `base` fornecida, que deve estar entre `2` e `36` inclusive, ou ser o valor especial `0`.

O mesmo que **PyOS_strtoul()**, mas retorna um valor long e **LONG_MAX** em caso de estouro.

Veja também a página man do Unix *strtol(3)* *[https://manpages.debian.org/strtol(3)]*.

Adicionado na versão 3.2.

double PyOS_string_to_double(const char *s, char **endptr, PyObject *overflow_exception)

Parte da ABI Estável.

Converte uma string `s` em double, levantando uma exceção Python em caso de falha. O conjunto de strings aceitas corresponde ao conjunto de strings aceito pelo construtor **float()** do Python, exceto que `s` não deve ter espaços em branco à esquerda ou à direita. A conversão é independente da localidade atual.

Se `endptr` for `NULL`, converte a string inteira. Levanta **ValueError** e retorna `-1.0` se a string não for uma representação válida de um número de ponto flutuante.

Se endptr não for `NULL`, converte o máximo possível da string e define `*endptr` para apontar para o primeiro caractere não convertido. Se nenhum segmento inicial da string for a representação válida de um número de ponto flutuante, define `*endptr` para apontar para o início da string, levanta ValueError e retorna `-1.0`.

If `s` represents a value that is too large to store in a float (for example, `"1e500"` is such a string on many platforms) then if `overflow_exception` is `NULL` return **INFINITY** (with an appropriate sign) and don't set any exception. Otherwise, `overflow_exception` must point to a Python exception object; raise that exception and return `-1.0`. In both cases, set `*endptr` to point to the first character after the converted value.

Se qualquer outro erro ocorrer durante a conversão (por exemplo, um erro de falta de memória), define a exceção Python apropriada e retorna `-1.0`.

Adicionado na versão 3.1.

char *PyOS_double_to_string(double val, char format_code, int precision, int flags, int *ptype)

Parte da ABI Estável.

Converte um *val* double para uma string usando *format_code*, *precision* e *flags* fornecidos.

format_code deve ser um entre `'e'`, `'E'`, `'f'`, `'F'`, `'g'`, `'G'` ou `'r'`. Para `'r'`, a precisão *precision* fornecida deve ser 0 e é ignorada. O código de formato `'r'` especifica o formato padrão de **`repr()`**.

flags podem ser zero ou mais dos seguintes valores combinados por meio do operador OR:

Py_DTSF_SIGN

Sempre preceda a string retornada com um caractere de sinal, mesmo que *val* não seja negativo.

Py_DTSF_ADD_DOT_0

Garante que a string retornada não se pareça com um número inteiro.

Py_DTSF_ALT

Aplica as regras de formatação “alternativas”. Veja a documentação para o especificador `'#'` de **`PyOS_snprintf()`** para detalhes.

Py_DTSF_NO_NEG_0

Zero negativo é convertido para zerp positivo.

Adicionado na versão 3.11.

If *ptype* is non-`NULL`, then the value it points to will be set to one of the following constants depending on the type of *val*:

ptype	**type of *val***

**ptype*	**type of *val***
Py_DTST_FINITE	finite number
Py_DTST_INFINITE	infinite number
Py_DTST_NAN	not a number

O valor de retorno é um ponteiro para *buffer* com a string convertida ou `NULL` se a conversão falhou. O chamador é responsável por liberar a string retornada chamando **`PyMem_Free()`**.

Adicionado na versão 3.1.

int PyOS_mystricmp(const char *str1, const char *str2)
int PyOS_mystrnicmp(const char *str1, const char *str2, Py_ssize_t size)

Parte da ABI Estável.
Comparação de strings sem distinção entre maiúsculas e minúsculas. Essas funções funcionam de maneira quase idêntica a **`strcmp()`** e **`strncmp()`** (respectivamente), exceto pelo fato de ignorarem maiúsculas e minúsculas em caracteres ASCII.

Retorna `0` se as strings forem iguais, um valor negativo se *str1* for ordenada lexicograficamente antes de *str2*, ou um valor positivo se for ordenada depois.

Nos argumentos *str1* ou *str2*, um byte NUL marca o fim da string. Para **`PyOS_mystrnicmp()`**, o argumento *size* fornece o tamanho máximo da string, como se NUL estivesse presente no índice fornecido por *size*.

Estas funções não usam a localidade.

int PyOS_stricmp(const char *str1, const char *str2)
int PyOS_strnicmp(const char *str1, const char *str2, Py_ssize_t size)

Comparação de strings sem distinção entre maiúsculas e minúsculas.

No Windows, estes são apelidos de **`stricmp()`** e **`strnicmp()`**, respectivamente.

Em outras plataformas, são apelidos de **`PyOS_mystricmp()`** e **`PyOS_mystrnicmp()`**, respectivamente.

Classificação e conversão de caracteres

As macros a seguir fornecem classificação e conversão de caracteres independentes de localidade (ao contrário da biblioteca padrão C `ctype.h`). O argumento deve ser um char com ou sem sinal.

Py_ISALNUM(c)

Retorna verdadeiro se o caractere *c* for um caractere alfanumérico.

Py_ISALPHA(c)

Retorna verdadeiro se o caractere *c* for um caractere alfabético (`a-z` e `A-Z`).

Py_ISDIGIT(c)

Retorna verdadeiro se o caractere *c* for um dígito decimal (`0-9`).

Py_ISLOWER(c)

Retorna verdadeiro se o caractere *c* for uma letra minúscula do ASCII (`a-z`).

Py_ISUPPER(c)

Retorna verdadeiro se o caractere *c* for uma letra maiúscula do ASCII (`A-Z`).

Py_ISSPACE(c)

Retorna verdadeiro se o caractere *c* for um caractere de espaço em branco (espaço, tabulação, retorno de carro, nova linha, tabulação

vertical ou alimentação de formulário).

Py_ISXDIGIT(c)

Retorna verdadeiro se o caractere *c* for um dígito hexadecimal (`0-9`, `a-f` e `A-F`).

Py_TOLOWER(c)

Retorna o equivalente em minúsculas do caractere *c*.

Py_TOUPPER(c)

Retorna o equivalente em maiúsculas do caractere *c*.

API do PyHash

Veja também o membro `PyTypeObject.tp_hash` e Hashing de tipos numéricos.

type Py_hash_t

Tipo de valor do hash: inteiro com sinal.

Adicionado na versão 3.2.

type Py_uhash_t

Tipo de valor do hash: inteiro sem sinal.

Adicionado na versão 3.2.

Py_HASH_ALGORITHM

Um valor numérico que indica o algoritmo de hash para `str`, `bytes` e `memoryview`.

O nome do algoritmo é exposto por `sys.hash_info.algorithm`.

Adicionado na versão 3.4.

Py_HASH_FNV
Py_HASH_SIPHASH24
Py_HASH_SIPHASH13

Valores numéricos a serem comparados com `Py_HASH_ALGORITHM` para determinar qual algoritmo é usado para o hashing. O algoritmo de hash pode ser configurado através da opção `--with-hash-algorithm` do comando configure.

Adicionado na versão 3.4: Adiciona `Py_HASH_FNV` e `Py_HASH_SIPHASH24`.

Adicionado na versão 3.11: Adiciona `Py_HASH_SIPHASH13`.

Py_HASH_CUTOFF

Buffers de comprimento no intervalo `[1, Py_HASH_CUTOFF)` são hasheados usando DJBX33A em vez do algoritmo descrito por **`Py_HASH_ALGORITHM`**.

- Uma **`Py_HASH_CUTOFF`** de 0 desabilita a otimização.
- **`Py_HASH_CUTOFF`** deve ser não-negativo ou menor ou igual a 7.

Plataformas de 32 bits devem usar um limite de corte menor do que plataformas de 64 bits, pois é mais fácil criar strings que colidam. Um limite de corte de 7 em plataformas de 64 bits e 5 em plataformas de 32 bits deve fornecer uma margem de segurança adequada.

Isso corresponde à constante **`sys.hash_info.cutoff`**.

Adicionado na versão 3.4.

PyHASH_MODULUS

O primo de Mersenne [https://pt.wikipedia.org/wiki/Primo_de_Mersenne] `P = 2**n -1`, usado para esquema de hash numérico.

Isso corresponde à constante **`sys.hash_info.modulus`**.

Adicionado na versão 3.13.

PyHASH_BITS

O expoente `n` de `P` em **`PyHASH_MODULUS`**.

Adicionado na versão 3.13.

PyHASH_MULTIPLIER

Multiplicador de primo usado em strings e vários outros hashes.

Adicionado na versão 3.13.

PyHASH_INF

O valor de hash retornado para um infinito positivo.

Isso corresponde à constante **`sys.hash_info.inf`**.

Adicionado na versão 3.13.

PyHASH_IMAG

O multiplicador usado para a parte imaginária de um número complexo.

Isso corresponde à constante `sys.hash_info.imag`.

Adicionado na versão 3.13.

type PyHash_FuncDef

Definição de função de hash usada por `PyHash_GetFuncDef()`.

Py_hash_t (*const hash)(const void*, Py_ssize_t)

Função de hash.

const char *name

Nome de função hash (string codificada em UTF-8).

Isso corresponde à constante `sys.hash_info.algorithm`.

const int hash_bits

Tamanho interno do valor do hash em bits.

Isso corresponde à constante `sys.hash_info.hash_bits`.

const int seed_bits

Tamanho da entrada de seed em bits.

Isso corresponde à constante `sys.hash_info.seed_bits`.

Adicionado na versão 3.4.

PyHash_FuncDef *PyHash_GetFuncDef(void)

Obtém a definição de função de hash.

Ver também

PEP 456 [https://peps.python.org/pep-0456/] “Algoritmo de hash seguro e intercambiável”.

Adicionado na versão 3.4.

Py_hash_t Py_HashPointer(const void *ptr)

Hash de um valor de ponteiro: processa o valor do ponteiro como um inteiro (converte-o para `uintptr_t` internamente). O ponteiro não é desreferenciado.

A função não pode falhar: ela não pode retornar `-1`.

Adicionado na versão 3.13.

Py_hash_t Py_HashBuffer(const void *ptr, Py_ssize_t len)

Calcula e retorna o valor de hash de um buffer de *len* bytes, começando no endereço *ptr*. O hash tem garantia de corresponder ao de **`bytes`**, **`memoryview`** e outros objetos embutidos que implementam o protocolo de buffer.

Usa esta função para implementar hash para objetos imutáveis cuja função **`tp_richcompare`** compara ao buffer de outro objeto.

len deve ser maior que ou igual a `0`.

Esta função sempre tem sucesso.

Adicionado na versão 3.14.

Py_hash_t PyObject_GenericHash(PyObject *obj)

Função de hash genérica que deve ser colocada no slot `tp_hash` de um objeto de tipo. Seu resultado depende apenas da identidade do objeto.

No CPython, é equivalente a **`Py_HashPointer()`**.

Adicionado na versão 3.13.

Reflexão

PyObject *PyEval_GetBuiltins(void)

Retorna valor: Referência emprestada. Parte da ABI Estável. Descontinuado desde a versão 3.13: Use `PyEval_GetFrameBuiltins()`.

Retorna um dicionário dos componentes embutidos no quadro de execução atual ou o interpretador do estado da thread, se nenhum quadro estiver em execução no momento.

PyObject *PyEval_GetLocals(void)

Retorna valor: Referência emprestada. Parte da ABI Estável. Descontinuado desde a versão 3.13: Use `PyEval_GetFrameLocals()` para obter o mesmo comportamento que chamar `locals()` no código Python, ou então chame `PyFrame_GetLocals()` no resultado de `PyEval_GetFrame()` para acessar o atributo `f_locals` do quadro atualmente em execução.

Retorna um mapeamento fornecendo acesso às variáveis locais no quadro de execução atual ou `NULL` se nenhum quadro estiver sendo executado no momento.

Consulte `locals()` para detalhes do mapeamento retornado em diferentes escopos.

Como esta função retorna uma referência emprestada, o dicionário retornado para escopos otimizados é armazenado em cache no objeto frame e permanecerá ativo enquanto o objeto frame o fizer. Ao contrário de `PyEval_GetFrameLocals()` e `locals()`, chamadas subsequentes para esta função no mesmo quadro atualizarão o conteúdo do dicionário em cache para refletir as mudanças no estado das variáveis locais em vez de retornar um novo snapshot.

Alterado na versão 3.13: Como parte da **PEP 667** [https://peps.python.org/pep-0667/], `PyFrame_GetLocals()`, `locals()` e `FrameType.f_locals` não

fazem mais uso do dicionário de cache compartilhado. Consulte a entrada de O Que Há de Novo para detalhes adicionais.

PyObject *PyEval_GetGlobals(void)

Retorna valor: Referência emprestada. Parte da ABI Estável.
Descontinuado desde a versão 3.13: Use `PyEval_GetFrameGlobals()`.

Retorna um dicionário das variáveis globais no quadro de execução atual ou `NULL` se nenhum quadro estiver sendo executado no momento.

PyFrameObject *PyEval_GetFrame(void)

Retorna valor: Referência emprestada. Parte da ABI Estável.
Retorna o quadro do estado de thread anexado, que é `NULL` se nenhum quadro estiver em execução no momento.

Veja também `PyThreadState_GetFrame()`.

PyObject *PyEval_GetFrameBuiltins(void)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.13.
Retorna um dicionário dos componentes embutidos no quadro de execução atual ou o interpretador do estado da thread, se nenhum quadro estiver em execução no momento.

Adicionado na versão 3.13.

PyObject *PyEval_GetFrameLocals(void)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.13.
Retorna um dicionário das variáveis locais no quadro de execução atual ou `NULL` se nenhum quadro estiver sendo executado no momento. Equivalente a chamar `locals()` em código Python.

Para acessar `f_locals` no quadro atual sem fazer um snapshot independente em escopos otimizados, chame `PyFrame_GetLocals()` no resultado de `PyEval_GetFrame()`.

Adicionado na versão 3.13.

PyObject *PyEval_GetFrameGlobals(void)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.13.

Retorna um dicionário das variáveis globais no quadro de execução atual ou `NULL` se nenhum quadro estiver sendo executado no momento. Equivalente a chamar **`globals()`** em código Python.

Adicionado na versão 3.13.

const char *PyEval_GetFuncName(PyObject *func)

Parte da ABI Estável.

Retorna o nome de *func* se for uma função, classe ou objeto de instância, senão o nome do tipo da *func*.

const char *PyEval_GetFuncDesc(PyObject *func)

Parte da ABI Estável.

Retorna uma sequência de caracteres de descrição, dependendo do tipo de *func*. Os valores de retorno incluem “()” para funções e métodos, “ constructor”, “ instance” e “ object”.. Concatenado com o resultado de **`PyEval_GetFuncName()`**, o resultado será uma descrição de *func*.

Registro de codec e funções de suporte

int PyCodec_Register(PyObject *search_function)

Parte da ABI Estável.
Registra uma nova função de busca de codec.

Como efeito colateral, tenta carregar o pacote **`encodings`**, se isso ainda não tiver sido feito, com o propósito de garantir que ele sempre seja o primeiro na lista de funções de busca.

int PyCodec_Unregister(PyObject *search_function)

Parte da ABI Estável desde a versão 3.10.
Cancela o registro de uma função de busca de codec e limpa o cache de registro. Se a função de busca não está registrada, não faz nada. Retorna 0 no sucesso. Levanta uma exceção e retorna -1 em caso de erro.

Adicionado na versão 3.10.

int PyCodec_KnownEncoding(const char *encoding)

Parte da ABI Estável.
Retorna `1` ou `0` dependendo se há um codec registrado para a dada codificação *encoding*. Essa função sempre é bem-sucedida.

PyObject *PyCodec_Encode(PyObject *object, const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
API de codificação baseada em codec genérico.

object é passado através da função de codificação encontrada para a codificação fornecida por meio de *encoding*, usando o método de tratamento de erros definido por *errors*. *errors* pode ser `NULL` para usar o

método padrão definido para o codec. Levanta um `LookupError` se nenhum codificador puder ser encontrado.

PyObject *PyCodec_Decode(PyObject *object, const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
API de decodificação baseada em decodificador genérico.

object é passado através da função de decodificação encontrada para a codificação fornecida por meio de *encoding*, usando o método de tratamento de erros definido por *errors*. *errors* pode ser `NULL` para usar o método padrão definido para o codec. Levanta um `LookupError` se nenhum decodificador puder ser encontrado.

API de pesquisa de codec

Nas funções a seguir, a string *encoding* é pesquisada com todos os caracteres sendo convertidos para minúsculo, o que faz com que as codificações pesquisadas por esse mecanismo não façam distinção entre maiúsculas e minúsculas. Se nenhum codec for encontrado, um `KeyError` é definido e `NULL` é retornado.

PyObject *PyCodec_Encoder(const char *encoding)

Retorna valor: Nova referência. Parte da ABI Estável.
Obtém uma função de codificação para o *encoding* dado.

PyObject *PyCodec_Decoder(const char *encoding)

Retorna valor: Nova referência. Parte da ABI Estável.
Obtém uma função de decodificação para o *encoding* dado.

PyObject *PyCodec_IncrementalEncoder(const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Obtém um objeto `IncrementalEncoder` para o *encoding* dado.

PyObject *PyCodec_IncrementalDecoder(const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Obtém um objeto `IncrementalDecoder` para o *encoding* dado.

PyObject *PyCodec_StreamReader(const char *encoding, PyObject *stream, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Obtém uma função de fábrica `StreamReader` para o *encoding* dado.

PyObject *PyCodec_StreamWriter(const char *encoding, PyObject *stream, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Obtém uma função de fábrica `StreamWriter` para o *encoding* dado.

API de registro de tratamentos de erros de decodificação Unicode

int PyCodec_RegisterError(const char *name, PyObject *error)

Parte da ABI Estável.
Registra a função de retorno de chamada de tratamento de *erro* para o *nome* fornecido. Esta chamada de função é invocada por um codificador quando encontra caracteres/bytes indecodificáveis e *nome* é especificado como o parâmetro de erro na chamada da função de codificação/decodificação.

O retorno de chamada obtém um único argumento, uma instância de `UnicodeEncodeError`, `UnicodeDecodeError` ou `UnicodeTranslateError` que contém informações sobre a sequencia problemática de caracteres ou bytes e seu deslocamento na string original (consulte Objetos de exceção Unicode para funções que extraem essa informação). A função de retorno de chamada deve levantar a exceção dada, ou retornar uma tupla de dois itens contendo a substituição para a sequência problemática, e um inteiro fornecendo o

deslocamento na string original na qual a codificação/decodificação deve ser retomada.

Retorna `0` em caso de sucesso, `-1` em caso de erro.

PyObject *PyCodec_LookupError(const char *name)

Retorna valor: Nova referência. Parte da ABI Estável.
Pesquisa a função de retorno de chamada de tratamento de erros registrada em *name*. Como um caso especial, `NULL` pode ser passado; nesse caso, o erro no tratamento de retorno de chamada para “strict” será retornado.

PyObject *PyCodec_StrictErrors(PyObject *exc)

Retorna valor: Sempre NULL. Parte da ABI Estável.
Levanta *exc* como uma exceção.

PyObject *PyCodec_IgnoreErrors(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Ignora o erro de unicode, ignorando a entrada que causou o erro.

PyObject *PyCodec_ReplaceErrors(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Substitui o erro de unicode por `?` ou `U+FFFD`.

PyObject *PyCodec_XMLCharRefReplaceErrors(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Substitui o erro de unicode por caracteres da referência XML.

PyObject *PyCodec_BackslashReplaceErrors(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável.
Substitui o erro de unicode com escapes de barra invertida (`\x`, `\u` e `\U`).

PyObject *PyCodec_NameReplaceErrors(PyObject *exc)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.
Substitui os erros de codificação unicode com escapes `\N{...}`.

Adicionado na versão 3.5.

Variáveis utilitárias de codec

const char *Py_hexdigits

Uma constante string contendo os dígitos hexadecimais minúsculos: `"0123456789abcdef"`.

Adicionado na versão 3.3.

API C PyTime

Adicionado na versão 3.13.

A API C de relógios provê acesso a relógios do sistema. Ela é similar ao módulo Python `time`.

Para uma API C relacionada ao módulo `datetime`, veja Objetos DateTime.

Tipos

type PyTime_t

Um registro de data e hora ou uma duração em nanossegundos, representados como um inteiro de 64 bits com sinal.

O ponto de referência para registros de data e hora depende do relógio usado. Por exemplo, `PyTime_Time()` retorna registros relativos à época UNIX.

É suportada uma amplitude em torno de [-292.3 anos; +292.3 anos]. Usando o início da época UNIX (1º de Janeiro de 1970) como referência, a gama de datas suportadas é em torno de [1677-09-21; 2262-04-11]. Os limites exatos são expostos como constantes:

PyTime_t PyTime_MIN

Valor mínimo de `PyTime_t`.

PyTime_t PyTime_MAX

Valor máximo de `PyTime_t`.

Funções de relógio

As funções a seguir aceitam um ponteiro para PyTime_t ao qual elas atribuem o valor de um determinado relógio. Detalhes de cada relógio estão

disponíveis nas documentações das funções Python correspondentes.

As funções retornam `0` em caso de sucesso, ou `-1` (com uma exceção definida) em caso de falha.

Em caso de estouro de inteiros, elas definem a exceção **`PyExc_OverflowError`** e definem `*result` como o valor limitado ao intervalo `[PyTime_MIN; PyTime_MAX]`. (Em sistemas atuais, estouros de inteiros são provavelmente causados por uma má configuração do tempo do sistema).

Como qualquer outra API C (se não especificado o contrário), as funções devem ser chamadas com um estado de thread anexado.

int PyTime_Monotonic(PyTime_t *result)

Lê o relógio monótono. Veja **`time.monotonic()`** para detalhes importantes sobre este relógio.

int PyTime_PerfCounter(PyTime_t *result)

Lê o contador de desempenho. Veja **`time.perf_counter()`** para detalhes importantes sobre este relógio.

int PyTime_Time(PyTime_t *result)

Lê o “relógio de parede”. Veja **`time.time()`** para detalhes importantes sobre este relógio.

Funções de relógio brutas

Similar às funções de relógio, mas não definem uma exceção em condições de erro, e não requerem que o chamador tenha um estado de thread anexado.

Em caso de sucesso, as funções retornam `0`.

Em caso de falha, elas definem `*result` como `0` e retornam `-1`, *sem* definir uma exceção. Para acessar a causa do erro, anexe um estado de thread e

chame a função regular (sem o sufixo `Raw`). Observe que a função regular pode ser bem-sucedida mesmo após o `Raw` falhar.

int PyTime_MonotonicRaw(PyTime_t *result)

Similar a **`PyTime_Monotonic()`**, mas não define uma exceção em caso de erro, e não exige um estado de thread anexado.

int PyTime_PerfCounterRaw(PyTime_t *result)

Similar a **`PyTime_PerfCounter()`**, mas não define uma exceção em caso de erro e não requer um estado de thread anexado.

int PyTime_TimeRaw(PyTime_t *result)

Similar a **`PyTime_Time()`**, mas não define uma exceção em caso de erro e não requer um estado de thread anexado.

Funções de conversão

double PyTime_AsSecondsDouble(PyTime_t t)

Converte um registro de data e hora para uma quantidade de segundos como um double C.

Esta função nunca falha, mas note que double tem acurácia limitada para valores grandes.

Suporte a Mapas do Perf

On supported platforms (Linux and macOS), the runtime can take advantage of *perf map files* to make Python functions visible to an external profiling tool (such as perf [https://perf.wiki.kernel.org/index.php/Main_Page] or samply [https://github.com/mstange/samply/]). A running process may create a file in the `/tmp` directory, which contains entries that can map a section of executable code to a name. This interface is described in the documentation of the Linux Perf tool [https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/tree/tools/perf/Documentation/jit-interface.txt].

Em Python, essas APIs auxiliares podem ser usadas por bibliotecas e recursos que dependem da geração de código de máquina dinamicamente.

Observe que manter o estado de thread anexado não é necessário para essas APIs.

int PyUnstable_PerfMapState_Init(void)

> *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*

Abre o arquivo `/tmp/perf-$pid.map`, a menos que já esteja aberto, e cria uma trava para garantir escritas seguras para thread no arquivo (desde que as escritas sejam feitas através de **`PyUnstable_WritePerfMapEntry()`**). Normalmente, não há necessidade de chamar isso explicitamente; basta usar **`PyUnstable_WritePerfMapEntry()`** e ele inicializará o estado na primeira chamada.

Retorna `0` em caso de sucesso, `-1` em caso de falha ao criar/abrir o arquivo de mapa de desempenho ou `-2` em caso de falha na criação de uma trava. Verifique `errno` para mais informações sobre a causa de uma falha.

int PyUnstable_WritePerfMapEntry(const void *code_addr, size_t code_size, const char *entry_name)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Escreve uma única entrada no arquivo `/tmp/perf-$pid.map`. Esta função é segura para thread. Aqui está a aparência de um exemplo de entrada:

```
# endereço  tamanho  nome
7f3529fcf759 b     py::bar:/run/t.py
```

Chamará **`PyUnstable_PerfMapState_Init()`** antes de escrever a entrada, se o arquivo de mapa de desempenho ainda não estiver aberto. Retorna `0` em caso de sucesso ou os mesmos códigos de erro que **`PyUnstable_PerfMapState_Init()`** em caso de falha.

void PyUnstable_PerfMapState_Fini(void)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Fecha o arquivo de mapa do perf aberto por **`PyUnstable_PerfMapState_Init()`**. Isso é chamado pelo próprio tempo de execução durante o desligamento do interpretador. Em geral, não deve haver motivo para chamar isso explicitamente, exceto para lidar com cenários específicos, como bifurcação.

int PyUnstable_CopyPerfMapFile(const char *parent_filename)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Open the `/tmp/perf-$pid.map` file and append the content of *parent_filename* to it.

This function is available on all platforms but only generates output on platforms that support perf maps (currently only Linux). On other platforms, it does nothing.

Adicionado na versão 3.13.

int PyUnstable_PerfTrampoline_CompileCode(PyCodeObject *code)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Compile the given code object using the current perf trampoline.

The "current" trampoline is the one set by the runtime or the most recent `PyUnstable_PerfTrampoline_SetPersistAfterFork()` call.

If no trampoline is set, falls back to normal compilation (no perf map entry).

Parâmetros:: • **code** – The code object to compile.
Retorna:: 0 on success, -1 on failure.

Adicionado na versão 3.13.

int PyUnstable_PerfTrampoline_SetPersistAfterFork(int enable)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Set whether the perf trampoline should persist after a fork.

- If `enable` is true (non-zero): perf map file remains open/valid post-fork. Child process inherits all existing perf map entries.
- If `enable` is false (zero): perf map closes post-fork. Child process gets empty perf map.

Default: false (clears on fork).

Parâmetros:: • **enable** – 1 to enable, 0 to disable.
Retorna:: 0 on success, -1 on failure.

Adicionado na versão 3.13.

Camada de Objetos Abstratos

As funções neste capítulo interagem com objetos Python independentemente do seu tipo, ou com classes amplas de tipos de objetos (por exemplo, todos os tipos numéricos ou todos os tipos de sequência). Quando usadas em tipos de objetos para os quais não se aplicam, elas irão levantar uma exceção Python.

Não é possível usar essas funções em objetos que não foram inicializados corretamente, como um objeto de lista criado por `PyList_New()`, mas cujos itens ainda não foram definidos para algum valor diferente de `NULL`.

- Protocolo de objeto
- Protocolo de chamada
  - O protocolo *tp_call*
  - O protocolo vectorcall
    - Controle de recursão
    - API de suporte à chamada de vetores
  - API de chamada de objetos
  - API de suporte a chamadas
- Protocolo de número
- Protocolo de sequência
- Protocolo de mapeamento
- Protocolo Iterador
- Protocolo de buffer
  - Estrutura de Buffer
  - Tipos de solicitação do buffer
    - campos independentes do pedido
    - apenas em formato
    - forma, avanços, suboffsets
    - requisições contíguas
    - requisições compostas
  - Vetores Complexos
    - Estilo NumPy: forma e avanços
    - Estilo-PIL: forma, avanços e suboffsets

- Funções relacionadas ao Buffer

Protocolo de objeto

PyObject *Py_GetConstant(unsigned int constant_id)

Parte da ABI Estável desde a versão 3.13.

Obtém uma referência forte para uma constante.

Define uma exceção e retorna `NULL` se *constant_id* for inválido.

constant_id deve ser um destes identificadores constantes:

Identificador da constante	Valor	Objeto retornado
Py_CONSTANT_NONE	`0`	`None`
Py_CONSTANT_FALSE	`1`	`False`
Py_CONSTANT_TRUE	`2`	`True`
Py_CONSTANT_ELLIPSIS	`3`	`Ellipsis`
Py_CONSTANT_NOT_IMPLEMENTED	`4`	`NotImplemented`
Py_CONSTANT_ZERO	`5`	`0`
Py_CONSTANT_ONE	`6`	`1`
Py_CONSTANT_EMPTY_STR	`7`	`''`
Py_CONSTANT_EMPTY_BYTES	`8`	`b''`
Py_CONSTANT_EMPTY_TUPLE	`9`	`()`

Valores numéricos são fornecidos apenas para projetos que não podem usar identificadores constantes.

Adicionado na versão 3.13.

No CPython, todas essas constantes são imortais.

PyObject *Py_GetConstantBorrowed(unsigned int constant_id)

Parte da ABI Estável desde a versão 3.13.
Semelhante a `Py_GetConstant()`, mas retorna uma referência emprestada.

Esta função destina-se principalmente à compatibilidade com versões anteriores: usar `Py_GetConstant()` é recomendado para novo código.

A referência é emprestada do interpretador e é válida até a finalização do interpretador.

Adicionado na versão 3.13.

PyObject *Py_NotImplemented

O singleton `NotImplemented`, usado para sinalizar que uma operação não foi implementada para a combinação de tipo fornecida.

Py_RETURN_NOTIMPLEMENTED

Manipula adequadamente o retorno de `Py_NotImplemented` de dentro de uma função C (ou seja, cria uma nova referência forte para `NotImplemented` e retorna-a).

Py_PRINT_RAW

Flag to be used with multiple functions that print the object (like `PyObject_Print()` and `PyFile_WriteObject()`). If passed, these functions use the `str()` of the object instead of the `repr()`.

int PyObject_Print(PyObject *o, FILE *fp, int flags)

Print an object *o*, on file *fp*. Returns `-1` on error. The flags argument is used to enable certain printing options. The only option currently supported is `Py_PRINT_RAW`; if given, the `str()` of the object is written instead of the `repr()`.

void PyObject_Dump(PyObject *op)

Dump an object *op* to `stderr`. This should only be used for debugging.

The output is intended to try dumping objects even after memory corruption:

- Information is written starting with fields that are the least likely to crash when accessed.
- This function can be called without an attached thread state, but it's not recommended to do so: it can cause deadlocks.
- An object that does not belong to the current interpreter may be dumped, but this may also cause crashes or unintended behavior.
- Implement a heuristic to detect if the object memory has been freed. Don't display the object contents in this case, only its memory address.
- The output format may change at any time.

Example of output:

```
object address  : 0x7f80124702c0
object refcount : 2
object type     : 0x9902e0
object type name: str
object repr     : 'abcdef'
```

Adicionado na versão 3.15.

int PyObject_HasAttrWithError(PyObject *o, PyObject *attr_name)

Parte da ABI Estável desde a versão 3.13.

Returns `1` if *o* has the attribute *attr_name*, and `0` otherwise. This is equivalent to the Python expression `hasattr(o, attr_name)`. On failure, return `-1`.

Adicionado na versão 3.13.

int PyObject_HasAttrStringWithError(PyObject *o, const char *attr_name)

Parte da ABI Estável desde a versão 3.13.

This is the same as `PyObject_HasAttrWithError()`, but *attr_name* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*.

Adicionado na versão 3.13.

int PyObject_HasAttr(PyObject *o, PyObject *attr_name)

Parte da ABI Estável.

Returns `1` if *o* has the attribute *attr_name*, and `0` otherwise. This function always succeeds.

Nota

Exceptions that occur when this calls `__getattr__()` and `__getattribute__()` methods aren't propagated, but instead given to `sys.unraisablehook()`. For proper error handling, use `PyObject_HasAttrWithError()`, `PyObject_GetOptionalAttr()` or `PyObject_GetAttr()` instead.

int PyObject_HasAttrString(PyObject *o, const char *attr_name)

Parte da ABI Estável.

This is the same as `PyObject_HasAttr()`, but *attr_name* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*.

Nota

Exceptions that occur when this calls `__getattr__()` and `__getattribute__()` methods or while creating the temporary `str` object are silently ignored. For proper error handling, use `PyObject_HasAttrStringWithError()`, `PyObject_GetOptionalAttrString()` or `PyObject_GetAttrString()` instead.

PyObject *PyObject_GetAttr(PyObject *o, PyObject *attr_name)

Retorna valor: Nova referência. Parte da ABI Estável.

Retrieve an attribute named *attr_name* from object *o*. Returns the attribute value on success, or `NULL` on failure. This is the equivalent of the Python expression `o.attr_name`.

If the missing attribute should not be treated as a failure, you can use **`PyObject_GetOptionalAttr()`** instead.

PyObject *PyObject_GetAttrString(PyObject *o, const char *attr_name)

Retorna valor: Nova referência. Parte da ABI Estável.
This is the same as **`PyObject_GetAttr()`**, but *attr_name* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*.

If the missing attribute should not be treated as a failure, you can use **`PyObject_GetOptionalAttrString()`** instead.

int PyObject_GetOptionalAttr(PyObject *obj, PyObject *attr_name, PyObject **result);

Parte da ABI Estável desde a versão 3.13.
Variant of **`PyObject_GetAttr()`** which doesn't raise **`AttributeError`** if the attribute is not found.

If the attribute is found, return `1` and set **result* to a new strong reference to the attribute. If the attribute is not found, return `0` and set **result* to `NULL`; the **`AttributeError`** is silenced. If an error other than **`AttributeError`** is raised, return `-1` and set **result* to `NULL`.

Adicionado na versão 3.13.

int PyObject_GetOptionalAttrString(PyObject *obj, const char *attr_name, PyObject **result);

Parte da ABI Estável desde a versão 3.13.
This is the same as **`PyObject_GetOptionalAttr()`**, but *attr_name* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*.

Adicionado na versão 3.13.

PyObject *PyObject_GenericGetAttr(PyObject *o, PyObject *name)

Retorna valor: Nova referência. Parte da ABI Estável.

Generic attribute getter function that is meant to be put into a type object's `tp_getattro` slot. It looks for a descriptor in the dictionary of classes in the object's MRO as well as an attribute in the object's **`__dict__`** (if present). As outlined in Implementando descritores, data descriptors take preference over instance attributes, while non-data descriptors don't. Otherwise, an **`AttributeError`** is raised.

int PyObject_SetAttr(PyObject *o, PyObject *attr_name, PyObject *v)

Parte da ABI Estável.

Set the value of the attribute named *attr_name*, for object *o*, to the value *v*. Raise an exception and return `-1` on failure; return `0` on success. This is the equivalent of the Python statement `o.attr_name = v`.

If *v* is `NULL`, the attribute is deleted. This behaviour is deprecated in favour of using **`PyObject_DelAttr()`**, but there are currently no plans to remove it.

The function must not be called with a `NULL` *v* and an exception set. This case can arise from forgetting `NULL` checks and would delete the attribute.

Alterado na versão 3.15: Must not be called with NULL value if an exception is set.

int PyObject_SetAttrString(PyObject *o, const char *attr_name, PyObject *v)

Parte da ABI Estável.

This is the same as **`PyObject_SetAttr()`**, but *attr_name* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*.

If *v* is `NULL`, the attribute is deleted, but this feature is deprecated in favour of using **`PyObject_DelAttrString()`**.

The function must not be called with a `NULL` *v* and an exception set. This case can arise from forgetting `NULL` checks and would delete the

attribute.

The number of different attribute names passed to this function should be kept small, usually by using a statically allocated string as *attr_name*. For attribute names that aren't known at compile time, prefer calling **PyUnicode_FromString()** and **PyObject_SetAttr()** directly. For more details, see **PyUnicode_InternFromString()**, which may be used internally to create a key object.

Alterado na versão 3.15: Must not be called with NULL value if an exception is set.

int PyObject_GenericSetAttr(PyObject *o, PyObject *name, PyObject *value)

Parte da ABI Estável.
Generic attribute setter and deleter function that is meant to be put into a type object's **tp_setattro** slot. It looks for a data descriptor in the dictionary of classes in the object's MRO, and if found it takes preference over setting or deleting the attribute in the instance dictionary. Otherwise, the attribute is set or deleted in the object's **__dict__** (if present). On success, `0` is returned, otherwise an **AttributeError** is raised and `-1` is returned.

int PyObject_DelAttr(PyObject *o, PyObject *attr_name)

Parte da ABI Estável desde a versão 3.13.
Delete attribute named *attr_name*, for object *o*. Returns `-1` on failure. This is the equivalent of the Python statement `del o.attr_name`.

int PyObject_DelAttrString(PyObject *o, const char *attr_name)

Parte da ABI Estável desde a versão 3.13.
This is the same as **PyObject_DelAttr()**, but *attr_name* is specified as a const char* UTF-8 encoded bytes string, rather than a PyObject*.

The number of different attribute names passed to this function should be kept small, usually by using a statically allocated string as *attr_name*. For attribute names that aren't known at compile time, prefer calling **PyUnicode_FromString()** and **PyObject_DelAttr()** directly. For more

details, see **PyUnicode_InternFromString()**, which may be used internally to create a key object for lookup.

PyObject *PyObject_GenericGetDict(PyObject *o, void *context)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.10.

A generic implementation for the getter of a `__dict__` descriptor. It creates the dictionary if necessary.

This function may also be called to get the **__dict__** of the object *o*. Pass `NULL` for *context* when calling it. Since this function may need to allocate memory for the dictionary, it may be more efficient to call **PyObject_GetAttr()** when accessing an attribute on the object.

On failure, returns `NULL` with an exception set.

Adicionado na versão 3.3.

int PyObject_GenericSetDict(PyObject *o, PyObject *value, void *context)

Parte da ABI Estável desde a versão 3.7.

A generic implementation for the setter of a `__dict__` descriptor. This implementation does not allow the dictionary to be deleted.

Adicionado na versão 3.3.

PyObject **_PyObject_GetDictPtr(PyObject *obj)

Return a pointer to **__dict__** of the object *obj*. If there is no `__dict__`, return `NULL` without setting an exception.

This function may need to allocate memory for the dictionary, so it may be more efficient to call **PyObject_GetAttr()** when accessing an attribute on the object.

PyObject *PyObject_RichCompare(PyObject *o1, PyObject *o2, int opid)

Retorna valor: Nova referência. Parte da ABI Estável.

Compare the values of *o1* and *o2* using the operation specified by *opid*, which must be one of **Py_LT**, **Py_LE**, **Py_EQ**, **Py_NE**, **Py_GT**, or **Py_GE**,

corresponding to `<`, `<=`, `==`, `!=`, `>`, or `>=` respectively. This is the equivalent of the Python expression `o1 op o2`, where `op` is the operator corresponding to *opid*. Returns the value of the comparison on success, or `NULL` on failure.

int PyObject_RichCompareBool(PyObject *o1, PyObject *o2, int opid)

Parte da ABI Estável.

Compare the values of *o1* and *o2* using the operation specified by *opid*, like **`PyObject_RichCompare()`**, but returns `-1` on error, `0` if the result is false, `1` otherwise.

Nota

If *o1* and *o2* are the same object, **`PyObject_RichCompareBool()`** will always return `1` for **`Py_EQ`** and `0` for **`Py_NE`**.

PyObject *PyObject_Format(PyObject *obj, PyObject *format_spec)

Parte da ABI Estável.

Format *obj* using *format_spec*. This is equivalent to the Python expression `format(obj, format_spec)`.

format_spec may be `NULL`. In this case the call is equivalent to `format(obj)`. Returns the formatted string on success, `NULL` on failure.

PyObject *PyObject_Repr(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

Compute a string representation of object *o*. Returns the string representation on success, `NULL` on failure. This is the equivalent of the Python expression `repr(o)`. Called by the **`repr()`** built-in function.

If argument is `NULL`, return the string `'<NULL>'`.

Alterado na versão 3.4: Essa função agora inclui uma asserção de depuração para ajudar a garantir que ela não descarte silenciosamente uma exceção ativa.

PyObject *PyObject_ASCII(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

As **PyObject_Repr()**, compute a string representation of object *o*, but escape the non-ASCII characters in the string returned by **PyObject_Repr()** with `\x`, `\u` or `\U` escapes. This generates a string similar to that returned by **PyObject_Repr()** in Python 2. Called by the **ascii()** built-in function.

If argument is `NULL`, return the string `'<NULL>'`.

PyObject *PyObject_Str(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

Compute a string representation of object *o*. Returns the string representation on success, `NULL` on failure. This is the equivalent of the Python expression `str(o)`. Called by the **str()** built-in function and, therefore, by the **print()** function.

If argument is `NULL`, return the string `'<NULL>'`.

Alterado na versão 3.4: Essa função agora inclui uma asserção de depuração para ajudar a garantir que ela não descarte silenciosamente uma exceção ativa.

PyObject *PyObject_Bytes(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

Compute a bytes representation of object *o*. `NULL` is returned on failure and a bytes object on success. This is equivalent to the Python expression `bytes(o)`, when *o* is not an integer. Unlike `bytes(o)`, a TypeError is raised when *o* is an integer instead of a zero-initialized bytes object.

If argument is `NULL`, return the **bytes** object `b'<NULL>'`.

int PyObject_IsSubclass(PyObject *derived, PyObject *cls)

Parte da ABI Estável.

Return `1` if the class *derived* is identical to or derived from the class *cls*, otherwise return `0`. In case of an error, return `-1`.

If *cls* is a tuple, the check will be done against every entry in *cls*. The result will be `1` when at least one of the checks returns `1`, otherwise it will be `0`.

If *cls* has a `__subclasscheck__()` method, it will be called to determine the subclass status as described in **PEP 3119** [https://peps.python.org/pep-3119/]. Otherwise, *derived* is a subclass of *cls* if it is a direct or indirect subclass, i.e. contained in `cls.__mro__`.

Normally only class objects, i.e. instances of `type` or a derived class, are considered classes. However, objects can override this by having a `__bases__` attribute (which must be a tuple of base classes).

int PyObject_IsInstance(PyObject *inst, PyObject *cls)

Parte da ABI Estável.

Return `1` if *inst* is an instance of the class *cls* or a subclass of *cls*, or `0` if not. On error, returns `-1` and sets an exception.

If *cls* is a tuple, the check will be done against every entry in *cls*. The result will be `1` when at least one of the checks returns `1`, otherwise it will be `0`.

If *cls* has a `__instancecheck__()` method, it will be called to determine the subclass status as described in **PEP 3119** [https://peps.python.org/pep-3119/]. Otherwise, *inst* is an instance of *cls* if its class is a subclass of *cls*.

An instance *inst* can override what is considered its class by having a `__class__` attribute.

An object *cls* can override if it is considered a class, and what its base classes are, by having a `__bases__` attribute (which must be a tuple of base classes).

Py_hash_t PyObject_Hash(PyObject *o)

Parte da ABI Estável.

Compute and return the hash value of an object *o*. On failure, return `-1`. This is the equivalent of the Python expression `hash(o)`.

Alterado na versão 3.2: The return type is now Py_hash_t. This is a signed integer the same size as **`Py_ssize_t`**.

Py_hash_t PyObject_HashNotImplemented(PyObject *o)

Parte da ABI Estável.

Set a **`TypeError`** indicating that `type(o)` is not hashable and return `-1`. This function receives special treatment when stored in a `tp_hash` slot, allowing a type to explicitly indicate to the interpreter that it is not hashable.

int PyObject_IsTrue(PyObject *o)

Parte da ABI Estável.

Returns `1` if the object *o* is considered to be true, and `0` otherwise. This is equivalent to the Python expression `not not o`. On failure, return `-1`.

int PyObject_Not(PyObject *o)

Parte da ABI Estável.

Returns `0` if the object *o* is considered to be true, and `1` otherwise. This is equivalent to the Python expression `not o`. On failure, return `-1`.

PyObject *PyObject_Type(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

When *o* is non-`NULL`, returns a type object corresponding to the object type of object *o*. On failure, raises **`SystemError`** and returns `NULL`. This is equivalent to the Python expression `type(o)`. This function creates a new strong reference to the return value. There's really no reason to use this function instead of the **`Py_TYPE()`** function, which returns a pointer of type PyTypeObject*, except when a new strong reference is needed.

int PyObject_TypeCheck(PyObject *o, PyTypeObject *type)

Return non-zero if the object *o* is of type *type* or a subtype of *type*, and `0` otherwise. Both parameters must be non-`NULL`.

Py_ssize_t PyObject_Size(PyObject *o)

Py_ssize_t PyObject_Length(PyObject *o)

Parte da ABI Estável.

Return the length of object *o*. If the object *o* provides either the sequence and mapping protocols, the sequence length is returned. On error, `-1` is returned. This is the equivalent to the Python expression `len(o)`.

Py_ssize_t PyObject_LengthHint(PyObject *o, Py_ssize_t defaultvalue)

Return an estimated length for the object *o*. First try to return its actual length, then an estimate using **`__length_hint__()`**, and finally return the default value. On error return `-1`. This is the equivalent to the Python expression `operator.length_hint(o, defaultvalue)`.

Adicionado na versão 3.4.

PyObject *PyObject_GetItem(PyObject *o, PyObject *key)

Retorna valor: Nova referência. Parte da ABI Estável.

Return element of *o* corresponding to the object *key* or `NULL` on failure. This is the equivalent of the Python expression `o[key]`.

int PyObject_SetItem(PyObject *o, PyObject *key, PyObject *v)

Parte da ABI Estável.

Map the object *key* to the value *v*. Raise an exception and return `-1` on failure; return `0` on success. This is the equivalent of the Python statement `o[key] = v`. This function *does not* steal a reference to *v*.

int PyObject_DelItem(PyObject *o, PyObject *key)

Parte da ABI Estável.

Remove the mapping for the object *key* from the object *o*. Return `-1` on failure. This is equivalent to the Python statement `del o[key]`.

int PyObject_DelItemString(PyObject *o, const char *key)

Parte da ABI Estável.

É o mesmo que **`PyObject_DelItem()`**, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

PyObject *PyObject_Dir(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
This is equivalent to the Python expression `dir(o)`, returning a (possibly empty) list of strings appropriate for the object argument, or `NULL` if there was an error. If the argument is `NULL`, this is like the Python `dir()`, returning the names of the current locals; in this case, if no execution frame is active then `NULL` is returned but **`PyErr_Occurred()`** will return false.

PyObject *PyObject_GetIter(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
This is equivalent to the Python expression `iter(o)`. It returns a new iterator for the object argument, or the object itself if the object is already an iterator. Raises **`TypeError`** and returns `NULL` if the object cannot be iterated.

PyObject *PyObject_SelfIter(PyObject *obj)

Retorna valor: Nova referência. Parte da ABI Estável.
This is equivalent to the Python `__iter__(self): return self` method. It is intended for iterator types, to be used in the **`PyTypeObject.tp_iter`** slot.

PyObject *PyObject_GetAIter(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.10.
This is the equivalent to the Python expression `aiter(o)`. Takes an **`AsyncIterable`** object and returns an **`AsyncIterator`** for it. This is typically a new iterator but if the argument is an **`AsyncIterator`**, this returns itself. Raises **`TypeError`** and returns `NULL` if the object cannot be iterated.

Adicionado na versão 3.10.

void *PyObject_GetTypeData(PyObject *o, PyTypeObject *cls)

Parte da ABI Estável desde a versão 3.12.
Get a pointer to subclass-specific data reserved for *cls*.

The object *o* must be an instance of *cls*, and *cls* must have been created using negative `PyType_Spec.basicsize`. Python does not check this.

On error, set an exception and return `NULL`.

Adicionado na versão 3.12.

Py_ssize_t PyType_GetTypeDataSize(PyTypeObject *cls)

Parte da ABI Estável desde a versão 3.12.

Return the size of the instance memory space reserved for *cls*, i.e. the size of the memory `PyObject_GetTypeData()` returns.

This may be larger than requested using `-PyType_Spec.basicsize`; it is safe to use this larger size (e.g. with `memset()`).

The type *cls* **must** have been created using negative `PyType_Spec.basicsize`. Python does not check this.

On error, set an exception and return a negative value.

Adicionado na versão 3.12.

void *PyObject_GetItemData(PyObject *o)

Get a pointer to per-item data for a class with `Py_TPFLAGS_ITEMS_AT_END`.

On error, set an exception and return `NULL`. `TypeError` is raised if *o* does not have `Py_TPFLAGS_ITEMS_AT_END` set.

Adicionado na versão 3.12.

int PyObject_VisitManagedDict(PyObject *obj, visitproc visit, void *arg)

Visit the managed dictionary of *obj*.

This function must only be called in a traverse function of the type which has the `Py_TPFLAGS_MANAGED_DICT` flag set.

Adicionado na versão 3.13.

void PyObject_ClearManagedDict(PyObject *obj)

Clear the managed dictionary of *obj*.

This function must only be called in a clear function of the type which has the `Py_TPFLAGS_MANAGED_DICT` flag set.

Adicionado na versão 3.13.

int PyUnstable_Object_EnableDeferredRefcount(PyObject *obj)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Enable deferred reference counting [https://peps.python.org/pep-0703/#deferred-reference-counting] on *obj*, if supported by the runtime. In the free-threaded build, this allows the interpreter to avoid reference count adjustments to *obj*, which may improve multi-threaded performance. The tradeoff is that *obj* will only be deallocated by the tracing garbage collector, and not when the interpreter no longer has any references to it.

This function returns `1` if deferred reference counting is enabled on *obj*, and `0` if deferred reference counting is not supported or if the hint was ignored by the interpreter, such as when deferred reference counting is already enabled on *obj*. This function is thread-safe, and cannot fail.

This function does nothing on builds with the GIL enabled, which do not support deferred reference counting. This also does nothing if *obj* is not an object tracked by the garbage collector (see `gc.is_tracked()` and `PyObject_GC_IsTracked()`).

This function is intended to be used soon after *obj* is created, by the code that creates it, such as in the object's `tp_new` slot.

Adicionado na versão 3.14.

int PyUnstable_Object_IsUniqueReferencedTemporary(PyObject *obj)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Check if *obj* is a unique temporary object. Returns `1` if *obj* is known to be a unique temporary object, and `0` otherwise. This function cannot fail, but the check is conservative, and may return `0` in some cases even if *obj* is a unique temporary object.

If an object is a unique temporary, it is guaranteed that the current code has the only reference to the object. For arguments to C functions, this should be used instead of checking if the reference count is `1`. Starting with Python 3.14, the interpreter internally avoids some reference count modifications when loading objects onto the operands stack by borrowing references when possible, which means that a reference count of `1` by itself does not guarantee that a function argument uniquely referenced.

In the example below, `my_func` is called with a unique temporary object as its argument:

```
my_func([1, 2, 3])
```

In the example below, `my_func` is **not** called with a unique temporary object as its argument, even if its refcount is `1`:

```
my_list = [1, 2, 3]
my_func(my_list)
```

See also the function **`Py_REFCNT()`**.

Adicionado na versão 3.14.

int PyUnstable_IsImmortal(PyObject *obj)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Essa função retorna um valor diferente de zero se *obj* for immortal, e zero caso contrário. Essa função nunca falha.

Nota

Objetos que são imortais em um versão do CPython não têm garantia de ser imortal em outro.

Adicionado na versão 3.14.

int PyUnstable_TryIncRef(PyObject *obj)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Increments the reference count of *obj* if it is not zero. Returns `1` if the object's reference count was successfully incremented. Otherwise, this function returns `0`.

`PyUnstable_EnableTryIncRef()` must have been called earlier on *obj* or this function may spuriously return `0` in the free-threaded build.

This function is logically equivalent to the following C code, except that it behaves atomically in the free-threaded build:

```
if (Py_REFCNT(op) > 0) {
   Py_INCREF(op);
   return 1;
}
return 0;
```

This is intended as a building block for managing weak references without the overhead of a Python weak reference object.

Typically, correct use of this function requires support from *obj*'s deallocator (`tp_dealloc`). For example, the following sketch could be adapted to implement a "weakmap" that works like a `WeakValueDictionary` for a specific type:

```
PyMutex mutex;

PyObject *
add_entry(weakmap_key_type *key, PyObject *value)
```

```
{
    PyUnstable_EnableTryIncRef(value);
    weakmap_type weakmap = ...;
    PyMutex_Lock(&mutex);
    weakmap_add_entry(weakmap, key, value);
    PyMutex_Unlock(&mutex);
    Py_RETURN_NONE;
}

PyObject *
get_value(weakmap_key_type *key)
{
    weakmap_type weakmap = ...;
    PyMutex_Lock(&mutex);
    PyObject *result = weakmap_find(weakmap, key);
    if (PyUnstable_TryIncRef(result)) {
        // `result` is safe to use
        PyMutex_Unlock(&mutex);
        return result;
    }
    // if we get here, `result` is starting to be garbage-col
    // but has not been removed from the weakmap yet
    PyMutex_Unlock(&mutex);
    return NULL;
}

// tp_dealloc function for weakmap values
void
value_dealloc(PyObject *value)
{
    weakmap_type weakmap = ...;
    PyMutex_Lock(&mutex);
    weakmap_remove_value(weakmap, value);

    ...
    PyMutex_Unlock(&mutex);
}
```

Adicionado na versão 3.14.

void PyUnstable_EnableTryIncRef(PyObject *obj)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Enables subsequent uses of `PyUnstable_TryIncRef()` on *obj*. The caller must hold a strong reference to *obj* when calling this.

Adicionado na versão 3.14.

int PyUnstable_Object_IsUniquelyReferenced(PyObject *op)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Determine if *op* only has one reference.

On GIL-enabled builds, this function is equivalent to Py_REFCNT(op) == 1.

On a free-threaded build, this checks if *op*'s reference count is equal to one and additionally checks if *op* is only used by this thread. Py_REFCNT(op) == 1 is **not** thread-safe on free-threaded builds; prefer this function.

The caller must hold an attached thread state, despite the fact that this function doesn't call into the Python interpreter. This function cannot fail.

Adicionado na versão 3.14.

int PyUnstable_SetImmortal(PyObject *op)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Marks the object *op* immortal. The argument should be uniquely referenced by the calling thread. This is intended to be used for reducing reference counting contention in the free-threaded build for objects which are shared across threads.

This is a one-way process: objects can only be made immortal; they cannot be made mortal once again. Immortal objects do not participate in reference counting and will never be garbage collected. If the object is GC-tracked, it is untracked.

This function is intended to be used soon after *op* is created, by the code that creates it, such as in the object's `tp_new` slot. Returns 1 if the object was made immortal and returns 0 if it was not. This function cannot fail.

Adicionado na versão 3.15.

Protocolo de chamada

O CPython permite dois protocolos de chamada: *tp_call* e vectorcall.

O protocolo *tp_call*

Instâncias de classe que definem `tp_call` são chamáveis. A assinatura do slot é:

```
PyObject *tp_call(PyObject *callable, PyObject *args, PyObject *
```

Uma chamada é feita usando uma tupla para os argumentos posicionais e um dicionário para os argumentos nomeados, similar a `callable(*args, **kwargs)` no Python. *args* não pode ser nulo (utilize uma tupla vazia se não houver argumentos), mas *kwargs* pode ser *NULL* se não houver argumentos nomeados.

Esta convenção não é somente usada por *tp_call*: `tp_new` e `tp_init` também passam argumento dessa forma.

Para chamar um objeto, use `PyObject_Call()` ou outra API de chamada.

O protocolo vectorcall

Adicionado na versão 3.9.

O protocolo vectorcall foi introduzido pela **PEP 590** [https://peps.python.org/pep-0590/] como um protocolo adicional para tornar invocações mais eficientes.

Como regra de ouro, CPython vai preferir o vectorcall para chamadas internas se o chamável suportar. Entretanto, isso não é uma regra rígida. Ademais, alguma extensões de terceiros usam *tp_call* diretamente (em vez de utilizar `PyObject_Call()`). Portanto, uma classe que suporta vectorcall precisa também implementar `tp_call`. Além disso, o chamável precisa se

comportar da mesma forma independe de qual protocolo é utilizado. A forma recomendada de alcançar isso é definindo **`tp_call`** para `PyVectorcall_Call()`. Vale a pena repetir:

Aviso

Uma classe que suporte vectorcall também **precisa** implementar `tp_call` com a mesma semântica.

Alterado na versão 3.12: O sinalizador `Py_TPFLAGS_HAVE_VECTORCALL` agora é removido da classe quando o método `__call__()` está reatribuído. (Internamente, isso apenas define `tp_call`, e portanto, pode fazê-lo comportar-se de forma diferente da função vectorcall. Em versões anteriores do Python, vectorcall só deve ser usado com tipos `imutáveis` ou estáticos.

Uma classe não deve implementar vectorcall se for mais lento que *tp_call*. Por exemplo, se o chamador precisa converter os argumentos para uma tupla args e um dicionário kwargs de qualquer forma, então não é necessário implementar vectorcall.

Classes podem implementar o protocolo vectorcall ativando o sinalizador `Py_TPFLAGS_HAVE_VECTORCALL` e configurando `tp_vectorcall_offset` para o offset dentro da estrutura do objeto onde uma *vectorcallfunc* aparece. Este é um ponteiro para uma função com a seguinte assinatura:

typedef PyObject *(*vectorcallfunc)(PyObject *callable, PyObject *const *args, size_t nargsf, PyObject *kwnames)

Parte da ABI Estável desde a versão 3.12.

- *callable* é o objeto sendo chamado.
- *args* é um array C formado pelos argumentos posicionais seguidos de valores dos argumentos nomeados. Este pode ser *NULL* se não existirem argumentos.
- *nargsf* é o número de argumentos posicionais somado á possível

Sinalizador `PY_VECTORCALL_ARGUMENTS_OFFSET`. Para obter o número real de argumentos posicionais de *nargsf*, use `PyVectorcall_NARGS()`.

- *kwnames* é uma tupla contendo os nomes dos argumentos nomeados; em outras palavras, as chaves do dicionário kwargs. Estes nomes devem ser strings (instâncias de `str` ou uma subclasse) e eles devem ser únicos. Se não existem argumentos nomeados, então *kwnames* deve então ser *NULL*.

PY_VECTORCALL_ARGUMENTS_OFFSET

Parte da ABI Estável desde a versão 3.12.

Se esse sinalizador é definido em um argumento *nargsf* do vectorcall, deve ser permitido ao chamado temporariamente mudar `args[-1]`. Em outras palavras, *args* aponta para o argumento 1 (não 0) no vetor alocado. O chamado deve restaurar o valor de `args[-1]` antes de retornar.

Para `PyObject_VectorcallMethod()`, este sinalizador significa que `args[0]` pode ser alterado.

Sempre que podem realizar a um custo tão baixo (sem alocações adicionais), invocadores são encorajados a usar `PY_VECTORCALL_ARGUMENTS_OFFSET`. Isso permitirá invocados como métodos vinculados a instâncias fazerem suas próprias invocações (o que inclui um argumento *self*) muito eficientemente.

Adicionado na versão 3.8.

Para invocar um objeto que implementa vectorcall, utilize a função call API como qualquer outra invocável. `PyObject_Vectorcall()` será normalmente mais eficiente.

Controle de recursão

Quando utilizando *tp_call*, invocadores não precisam se preocupar sobre recursão: CPython usa `Py_EnterRecursiveCall()` e `Py_LeaveRecursiveCall()` para chamadas utilizando *tp_call*.

Por questão de eficiência, este não é o caso de chamadas utilizando o vectorcall: o que chama deve utilizar *Py_EnterRecursiveCall* e *Py_LeaveRecursiveCall* se necessário.

API de suporte à chamada de vetores

Py_ssize_t PyVectorcall_NARGS(size_t nargsf)

Parte da ABI Estável desde a versão 3.12.
Dado um argumento de chamada de vetor *nargsf*, retorna o número real de argumentos. Atualmente equivalente a:

```
(Py_ssize_t)(nargsf & ~PY_VECTORCALL_ARGUMENTS_OFFSET)
```

Entretanto, a função `PyVectorcall_NARGS` deve ser usada para permitir para futuras extensões.

Adicionado na versão 3.8.

vectorcallfunc PyVectorcall_Function(PyObject *op)

Se *op* não suporta o protocolo de chamada de vetor (seja porque o tipo ou a instância específica não suportam), retorne *NULL*. Se não, retorne o ponteiro da função chamada de vetor armazenado em *op*. Esta função nunca levanta uma exceção.

É mais útil checar se *op* suporta ou não chamada de vetor, o que pode ser feito checando `PyVectorcall_Function(op) != NULL`.

Adicionado na versão 3.9.

PyObject *PyVectorcall_Call(PyObject *callable, PyObject *tuple, PyObject *dict)

Parte da ABI Estável desde a versão 3.12.
Chama o **`vectorcallfunc`** de *callable* com argumentos posicionais e nomeados dados em uma tupla e dicionário, respectivamente.

Esta é uma função especializada, feita para ser colocada no slot **`tp_call`** ou usada em uma implementação de `tp_call`. Ela não verifica o

sinalizador `Py_TPFLAGS_HAVE_VECTORCALL` e não retorna para `tp_call`.

Adicionado na versão 3.8.

API de chamada de objetos

Várias funções estão disponíveis para chamar um objeto Python. Cada uma converte seus argumentos para uma convenção suportada pelo objeto chamado – seja *tp_call* ou chamada de vetor. Para fazer o mínimo possível de conversões, escolha um que melhor se adapte ao formato de dados que você tem disponível.

A tabela a seguir resume as funções disponíveis; por favor, veja a documentação individual para detalhes.

Função	chamável	args	kwargs
`PyObject_Call()`	`PyObject *`	tupla	dict/`NULL`
`PyObject_CallNoArgs()`	`PyObject *`	—	—
`PyObject_CallOneArg()`	`PyObject *`	1 objeto	—
`PyObject_CallObject()`	`PyObject *`	tupla/`NULL`	—
`PyObject_CallFunction()`	`PyObject *`	formato	—
`PyObject_CallMethod()`	obj + `char*`	formato	—

Função	chamável	args	kwargs
`PyObject_CallFunctionObjArgs()`	`PyObject *`	variádica	—
`PyObject_CallMethodObjArgs()`	obj + nome	variádica	—
`PyObject_CallMethodNoArgs()`	obj + nome	—	—
`PyObject_CallMethodOneArg()`	obj + nome	1 objeto	—
`PyObject_Vectorcall()`	`PyObject *`	vectorcall	vectorcall
`PyObject_VectorcallDict()`	`PyObject *`	vectorcall	dict/`NULL`
`PyObject_VectorcallMethod()`	arg + nome	vectorcall	vectorcall

PyObject *PyObject_Call(PyObject *callable, PyObject *args, PyObject *kwargs)

Retorna valor: Nova referência. Parte da ABI Estável.

Chama um objeto Python chamável de *callable*, com argumentos dados pela tupla *args*, e argumentos nomeados dados pelo dicionário *kwargs*.

args não deve ser *NULL*; use uma tupla vazia se não precisar de argumentos. Se nenhum argumento nomeado é necessário, *kwargs* pode ser *NULL*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Esse é o equivalente da expressão Python: `callable(*args, **kwargs)`.

PyObject *PyObject_CallNoArgs(PyObject *callable)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.10.

Chama um objeto Python chamável de *callable* sem nenhum argumento. É o jeito mais eficiente de chamar um objeto Python sem nenhum argumento.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Adicionado na versão 3.9.

PyObject *PyObject_CallOneArg(PyObject *callable, PyObject *arg)

Retorna valor: Nova referência.

Chama um objeto Python chamável de *callable* com exatamente 1 argumento posicional *arg* e nenhum argumento nomeado.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Adicionado na versão 3.9.

PyObject *PyObject_CallObject(PyObject *callable, PyObject *args)

Retorna valor: Nova referência. Parte da ABI Estável.

Chama um objeto Python chamável de *callable* com argumentos dados pela tupla *args*. Se nenhum argumento é necessário, *args* pode ser *NULL*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Este é o equivalente da expressão Python: `callable(*args)`.

PyObject *PyObject_CallFunction(PyObject *callable, const char *format, ...)

Retorna valor: Nova referência. Parte da ABI Estável.
Chama um objeto Python chamável de *callable*, com um número variável de argumentos C. Os argumentos C são descritos usando uma string de estilo no formato **`Py_BuildValue()`**. O formato pode ser *NULL*, indicando que nenhum argumento foi provido.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Este é o equivalente da expressão Python: `callable(*args)`.

Note que se você apenas passa argumentos PyObject*, **`PyObject_CallFunctionObjArgs()`** é uma alternativa mais rápida.

Alterado na versão 3.4: O tipo de *format* foi mudado de `char *`.

PyObject *PyObject_CallMethod(PyObject *obj, const char *name, const char *format, ...)

Retorna valor: Nova referência. Parte da ABI Estável.
Chama o método chamado *name* do objeto *obj* com um número variável de argumentos C. Os argumentos C são descritos com uma string de formato **`Py_BuildValue()`** que deve produzir uma tupla.

O formato pode ser *NULL*, indicado que nenhum argumento foi provido.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Este é o equivalente da expressão Python: `obj.name(arg1, arg2, ...)`.

Note que se você apenas passa argumentos PyObject*, **`PyObject_CallMethodObjArgs()`** é uma alternativa mais rápida.

Alterado na versão 3.4: Os tipos de *name* e *format* foram mudados de `char *`.

PyObject *PyObject_CallFunctionObjArgs(PyObject *callable, ...)

Retorna valor: Nova referência. Parte da ABI Estável.
Chama um objeto Python chamável de *callable*, com um número variável de argumentos PyObject*. Os argumentos são providos como um número variável de parâmetros seguidos por um *NULL*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Este é o equivalente da expressão Python: `callable(arg1, arg2, ...)`.

PyObject *PyObject_CallMethodObjArgs(PyObject *obj, PyObject *name, ...)

Retorna valor: Nova referência. Parte da ABI Estável.
Chama um método do objeto Python *obj*, onde o nome do método é dado como um objeto string Python em *name*. É chamado com um número variável de argumentos PyObject*. Os argumentos são providos como um número variável de parâmetros seguidos por um *NULL*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

PyObject *PyObject_CallMethodNoArgs(PyObject *obj, PyObject *name)

Chama um método do objeto Python *obj* sem argumentos, onde o nome do método é fornecido como um objeto string do Python em *name*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Adicionado na versão 3.9.

PyObject *PyObject_CallMethodOneArg(PyObject *obj, PyObject *name, PyObject *arg)

Chama um método do objeto Python *obj* com um argumento posicional *arg*, onde o nome do método é fornecido como um objeto string do Python em *name*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Adicionado na versão 3.9.

PyObject *PyObject_Vectorcall(PyObject *callable, PyObject *const *args, size_t nargsf, PyObject *kwnames)

Parte da ABI Estável desde a versão 3.12.
Chama um objeto Python chamável *callable*. Os argumentos são os mesmos de **`vectorcallfunc`**. Se *callable* tiver suporte a vectorcall, isso chamará diretamente a função vectorcall armazenada em *callable*.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Adicionado na versão 3.8: como `_PyObject_Vectorcall`

Alterado na versão 3.9: Renomeado para o nome atual, sem o sublinhado no início. O nome provisório antigo está suavemente descontinuado.

PyObject *PyObject_VectorcallDict(PyObject *callable, PyObject *const *args, size_t nargsf, PyObject *kwdict)

Chama *callable* com argumentos posicionais passados exatamente como no protocolo vectorcall, mas com argumentos nomeados passados como um dicionário *kwdict*. O array *args* contém apenas os argumentos posicionais.

Independentemente de qual protocolo é usado internamente, uma conversão de argumentos precisa ser feita. Portanto, esta função só deve ser usada se o chamador já tiver um dicionário pronto para usar para os argumentos nomeados, mas não uma tupla para os argumentos posicionais.

Adicionado na versão 3.9.

PyObject *PyObject_VectorcallMethod(PyObject *name, PyObject *const *args, size_t nargsf, PyObject *kwnames)

Parte da ABI Estável desde a versão 3.12.

Chama um método usando a convenção de chamada vectorcall. O nome do método é dado como uma string Python *name*. O objeto cujo método é chamado é *args[0]*, e o array *args* começando em *args[1]* representa os argumentos da chamada. Deve haver pelo menos um argumento posicional. *nargsf* é o número de argumentos posicionais incluindo *args[0]*, mais `PY_VECTORCALL_ARGUMENTS_OFFSET` se o valor de `args[0]` puder ser alterado temporariamente. Argumentos nomeados podem ser passados como em `PyObject_Vectorcall()`.

Se o objeto tem o recurso `Py_TPFLAGS_METHOD_DESCRIPTOR`, isso irá chamar o objeto de método não vinculado com o vetor *args* inteiro como argumentos.

Retorna o resultado da chamada em sucesso, ou levanta uma exceção e retorna *NULL* em caso de falha.

Adicionado na versão 3.9.

API de suporte a chamadas

int PyCallable_Check(PyObject *o)

Parte da ABI Estável.

Determine se o objeto *o* é chamável. Devolva `1` se o objeto é chamável e `0` caso contrário. Esta função sempre tem êxito.

Protocolo de número

int PyNumber_Check(PyObject *o)

Parte da ABI Estável.
Retorna `1` se o objeto *o* fornece protocolos numéricos; caso contrário, retorna falso. Esta função sempre tem sucesso.

Alterado na versão 3.8: Retorna `1` se *o* for um número inteiro de índice.

PyObject *PyNumber_Add(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da adição de *o1* e *o2*, ou `NULL` em caso de falha. Este é o equivalente da expressão Python `o1 + o2`.

PyObject *PyNumber_Subtract(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da subtração de *o2* por *o1*, ou `NULL` em caso de falha. Este é o equivalente da expressão Python `o1 - o2`.

PyObject *PyNumber_Multiply(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da multiplicação de *o1* e *o2*, ou `NULL` em caso de falha. Este é o equivalente da expressão Python `o1 * o2`.

PyObject *PyNumber_MatrixMultiply(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.
Retorna o resultado da multiplicação da matriz em *o1* e *o2*, ou `NULL` em caso de falha. Este é o equivalente da expressão Python `o1 @ o2`.

Adicionado na versão 3.5.

PyObject *PyNumber_FloorDivide(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna o piso de *o1* dividido por *o2* ou `NULL` em caso de falha. Isso é equivalente à expressão Python `o1 // o2`.

PyObject *PyNumber_TrueDivide(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna uma aproximação razoável para o valor matemático de *o1* dividido por *o2*, ou `NULL` em caso de falha. O valor devolvido é "aproximado" porque os números binários de ponto flutuante são aproximados; não é possível representar todos os números reais na base dois. Esta função pode devolver um valor de ponto flutuante quando lhe são passados dois inteiros. Isso é o equivalente à expressão Python `o1 / o2`.

PyObject *PyNumber_Remainder(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resto da divisão de *o1* por *o2* ou `NULL` em caso de falha. Isso é equivalente à expressão Python `o1 % o2`.

PyObject *PyNumber_Divmod(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Consulte a função embutida **`divmod()`**. Retorna `NULL` em caso de falha. Isso é equivalente à expressão Python `divmod(o1, o2)`.

PyObject *PyNumber_Power(PyObject *o1, PyObject *o2, PyObject *o3)

Retorna valor: Nova referência. Parte da ABI Estável.
Consulte a função embutida **`pow()`**. Retorna `NULL` em caso de falha. Isso é equivalente à expressão Python `pow(o1, o2, o3)`, na qual *o3* é opcional. Se *o3* deve ser ignorado, passe **`Py_None`** em seu lugar (passar `NULL` para *o3* causaria um acesso ilegal à memória).

PyObject *PyNumber_Negative(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna a negação de *o* em caso de sucesso ou `NULL` em caso de falha. Isso é equivalente à expressão Python `-o`.

PyObject *PyNumber_Positive(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna *o* em caso de sucesso ou `NULL` em caso de falha. Isso é equivalente à expressão Python `+o`.

PyObject *PyNumber_Absolute(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o valor absoluto de *o* ou `NULL` em caso de falha. Isso é equivalente à expressão Python `abs(o)`.

PyObject *PyNumber_Invert(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna a negação bit a bit de *o* em caso de sucesso, ou `NULL` em caso de falha. Isso é equivalente à expressão Python `~o`.

PyObject *PyNumber_Lshift(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado do deslocamento à esquerda de *o1* por *o2* em caso de sucesso ou `NULL` em caso de falha. Isso é equivalente à expressão Python `o1 << o2`.

PyObject *PyNumber_Rshift(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado do deslocamento à direita de *o1* por *o2* em caso de sucesso ou `NULL` em caso de falha. Isso é equivalente à expressão Python `o1 >> o2`.

PyObject *PyNumber_And(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o bit a bit “e” de *o1* e *o2* em caso de sucesso e `NULL` em caso de falha. Isso é equivalente à expressão Python `o1 & o2`.

PyObject *PyNumber_Xor(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o bit a bit “ou exclusivo” de *o1* por *o2* em caso de sucesso ou `NULL` em caso de falha. Isso é equivalente à expressão Python `o1 ^ o2`.

PyObject *PyNumber_Or(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o bit a bit "ou" de *o1* e *o2* em caso de sucesso ou `NULL` em caso de falha. Isto é equivalente à expressão Python `o1 | o2`.

PyObject *PyNumber_InPlaceAdd(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da adição de *o1* e *o2* ou `NULL` em caso de falha. A operação é feita *no local* quando *o1* dá suporte a isso. Isto é equivalente à instrução Python `o1 += o2`.

PyObject *PyNumber_InPlaceSubtract(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da subtração de *o2* de *o1* ou `NULL` em caso de falha. A operação é feita *no local* quando *o1* dá suporte a isso. Isto é equivalente à instrução Python `o1 -= o2`.

PyObject *PyNumber_InPlaceMultiply(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da multiplicação de *o1* e *o2* ou `NULL` em caso de falha. A operação é feita *no local* quando *o1* dá suporte a isso. Isto é equivalente à instrução Python `o1 *= o2`.

PyObject *PyNumber_InPlaceMatrixMultiply(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.
Retorna o resultado da multiplicação de matrizes entre *o1* e *o2*, ou `NULL` em caso de falha. A operação é realizada no local quando *o1* oferece suporte a isso. Isso é equivalente à instrução Python `o1 @= o2`.

Adicionado na versão 3.5.

PyObject *PyNumber_InPlaceFloorDivide(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna o valor matemático mínimo da divisão de *o1* por *o2* ou `NULL` em caso de falha. A operação é feita *no local* quando *o1* dá suporte a isso. Isto é equivalente à instrução Python `o1 //= o2`.

PyObject *PyNumber_InPlaceTrueDivide(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna uma aproximação razoável para o valor matemático de *o1* dividido por *o2*, ou `NULL` em caso de falha. O valor de retorno é "aproximado" porque os números binários de ponto flutuante são aproximados; não é possível representar todos os números reais na base dois. Esta função pode retornar um valor de ponto flutuante quando lhe são passados dois inteiros. A operação é feita no local quando *o1* oferece suporte a isso. Isso é o equivalente à instrução Python `o1 /= o2`.

PyObject *PyNumber_InPlaceRemainder(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resto da divisão de *o1* por *o2* ou `NULL` em caso de falha. A operação é feita *no local* quando *o1* dá suporte a isso. Isto é equivalente à instrução Python `o1 %= o2`.

PyObject *PyNumber_InPlacePower(PyObject *o1, PyObject *o2, PyObject *o3)

Retorna valor: Nova referência. Parte da ABI Estável.
Consulte a função embutida **`pow()`**. Retorna `NULL` em caso de falha. A operação é feita no lugar quando *o1* oferece suporte a isso. Isso é equivalente à instrução Python `o1 **= o2` quando o3 é **`Py_None`** ou, do contrário, uma variante no local `pow(o1, o2, o3)`. Se *o3* deve ser ignorado, passe **`Py_None`** em seu lugar (passar `NULL` para *o3* causaria um acesso ilegal à memória).

PyObject *PyNumber_InPlaceLshift(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado do deslocamento à esquerda de *o1* por *o2* em caso de sucesso, ou `NULL` em caso de falha. A operação é feita no local

quando *o1* oferece suporte a isso. Isto é equivalente à instrução Python `o1 <<= o2`.

PyObject *PyNumber_InPlaceRshift(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado do deslocamento à direita de *o1* por *o2* em caso de sucesso, ou `NULL` em caso de falha. A operação é feita no local quando *o1* oferece suporte a isso. Isto é equivalente à instrução Python `o1 >>= o2`.

PyObject *PyNumber_InPlaceAnd(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o "bitwise and" de *o1* e *o2* em caso de sucesso e `NULL` em caso de falha. A operação é feita no local quando *o1* oferece suporte a isso. Isto é equivalente à instrução Python `o1 &= o2`.

PyObject *PyNumber_InPlaceXor(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o "bitwise exclusive or" de *o1* por *o2* em caso de sucesso, ou `NULL` em caso de falha. A operação é feita no local quando *o1* oferece suporte a isso. Isto é equivalente à instrução Python `o1 ^= o2`.

PyObject *PyNumber_InPlaceOr(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o "bitwise or" de *o1* e *o2* em caso de sucesso, ou `NULL` em caso de falha. A operação é feita no local quando *o1* oferece suporte a isso. Isto é equivalente à instrução Python `o1 |= o2`.

PyObject *PyNumber_Long(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o *o* convertido para um objeto inteiro em caso de sucesso ou `NULL` em caso de falha. Isto é equivalente à expressão Python `int(o)`.

PyObject *PyNumber_Float(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna o *o* convertido para um objeto float em caso de sucesso ou `NULL` em caso de falha. Isto é equivalente à expressão Python `float(o)`.

PyObject *PyNumber_Index(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna *o* convertido para int Python em caso de sucesso ou `NULL` com um **`TypeError`** exceção levantada ao falhar.

Alterado na versão 3.10: O resultado sempre tem o tipo exato **`int`**. Anteriormente, o resultado poderia ter sido uma instância de uma subclasse de `int`.

PyObject *PyNumber_ToBase(PyObject *n, int base)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o inteiro *n* convertido para a base *base* como uma string. O argumento *base* deve ser 2, 8, 10 ou 16. Para as bases 2, 8 ou 16, a string retornada é prefixada com um marcador de base `'0b'`, `'0o'` ou `'0x'`, respectivamente. Se *n* não for um inteiro Python, ele é convertido primeiro com **`PyNumber_Index()`**.

Py_ssize_t PyNumber_AsSsize_t(PyObject *o, PyObject *exc)

Parte da ABI Estável.
Retorna *o* convertido para um valor **`Py_ssize_t`** se *o* puder ser interpretado como um inteiro. Se a chamada falhar, uma exceção é levantada e `-1` é retornado.

Se *o* puder ser convertido para um int Python, mas a tentativa de conversão para um valor **`Py_ssize_t`** levantar um **`OverflowError`**, então o argumento *exc* é o tipo de exceção que será gerada (geralmente **`IndexError`** ou **`OverflowError`**). Se *exc* for `NULL`, então a exceção é limpa e o valor é cortado para `PY_SSIZE_T_MIN` para um inteiro negativo ou `PY_SSIZE_T_MAX` para um inteiro positivo.

int PyIndex_Check(PyObject *o)

Parte da ABI Estável desde a versão 3.8.
Retorna `1` se *o* for um índice inteiro (tem o slot `nb_index` da estrutura `tp_as_number` preenchido) e `0` caso contrário. Esta função sempre tem

sucesso.

Protocolo de sequência

int PySequence_Check(PyObject *o)

Parte da ABI Estável.

Retorna `1` se o objeto fornecer o protocolo de sequência e `0` caso contrário. Note que ele retorna `1` para classes Python com um método `__getitem__()` a menos que sejam subclasses de `dict` já que no caso geral é impossível determinar que tipo de chaves a classe provê. Esta função sempre obtém sucesso.

Py_ssize_t PySequence_Size(PyObject *o)

Py_ssize_t PySequence_Length(PyObject *o)

Parte da ABI Estável.

Retorna o número de objetos na sequência *o* em caso de sucesso, ou `-1` em caso de falha. Equivale à expressão Python `len(o)`.

PyObject *PySequence_Concat(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna a concatenação de *o1* e *o2* em caso de sucesso, e `NULL` em caso de falha. Equivale à expressão Python `o1 + o2`.

PyObject *PySequence_Repeat(PyObject *o, Py_ssize_t count)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna o resultado da repetição do objeto sequência *o* *count* vezes ou `NULL` em caso de falha. Equivale à expressão Python `o * count`.

PyObject *PySequence_InPlaceConcat(PyObject *o1, PyObject *o2)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna a concatenação de *o1* e *o2* em caso de sucesso, e `NULL` em caso de falha. A operação é feita *localmente* se *o1* permitir. Equivale à expressão Python `o1 += o2`.

PyObject *PySequence_InPlaceRepeat(PyObject *o, Py_ssize_t count)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o resultado da repetição do objeto sequência *o* *count* vezes ou `NULL` em caso de falha. A operação é feita *localmente* se *o* permitir. Equivale à expressão Python `o *= count`.

PyObject *PySequence_GetItem(PyObject *o, Py_ssize_t i)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o elemento *i* de *o* ou `NULL` em caso de falha. Equivale à expressão Python `o[i]`.

PyObject *PySequence_GetSlice(PyObject *o, Py_ssize_t i1, Py_ssize_t i2)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna a fatia do objeto sequência *o* entre *i1* e *i2*, ou `NULL` em caso de falha. Equivale à expressão Python `o[i1:i2]`.

int PySequence_SetItem(PyObject *o, Py_ssize_t i, PyObject *v)

Parte da ABI Estável.
Atribui o objeto *v* ao elemento *i* de *o*. Levanta uma exceção e retorna `-1` em caso de falha; retorna `0` em caso de sucesso. Isso é o equivalente à instrução Python `o[i] = v`. Esta função *não* "rouba" uma referência a *v*.

Se *v* for `NULL`, o elemento será removido, mas este recurso foi descontinuado em favor do uso de **`PySequence_DelItem()`**.

int PySequence_DelItem(PyObject *o, Py_ssize_t i)

Parte da ABI Estável.
Exclui o elemento *i* do objeto *o*. Retorna `-1` em caso de falha. Equivale à instrução Python `del o[i]`.

int PySequence_SetSlice(PyObject *o, Py_ssize_t i1, Py_ssize_t i2, PyObject *v)

Parte da ABI Estável.
Atribui o objeto sequência *v* à fatia no objeto sequência *o* de *i1* a *i2*. Equivale à instrução Python `o[i1:i2] = v`.

int PySequence_DelSlice(PyObject *o, Py_ssize_t i1, Py_ssize_t i2)

Parte da ABI Estável.

Exclui a fatia no objeto sequência *o* de *i1* a *i2*. Retorna `-1` em caso de falha. Equivale à instrução Python `del o[i1:i2]`.

Py_ssize_t PySequence_Count(PyObject *o, PyObject *value)

Parte da ABI Estável.

Retorna a quantidade de ocorrências de *value* em *o*, isto é, retorna a quantidade de chaves onde `o[key] == value`. Em caso de falha, retorna `-1`. Equivale à expressão Python `o.count(value)`.

int PySequence_Contains(PyObject *o, PyObject *value)

Parte da ABI Estável.

Determina se *o* contém *value*. Se um item em *o* for igual a *value*, retorna `1`, senão, retorna `0`. Em caso de erro, retorna `-1`. Equivale à expressão Python `value in o`.

int PySequence_In(PyObject *o, PyObject *value)

Parte da ABI Estável.

Apelido para **`PySequence_Contains()`**.

Suavemente descontinuado desde a versão 3.14: A função não deve mais ser usada para escrever novos códigos.

Py_ssize_t PySequence_Index(PyObject *o, PyObject *value)

Parte da ABI Estável.

Retorna o primeiro índice *i* tal que `o[i] == value`. Em caso de erro, retorna `-1`. Equivale à expressão Python `o.index(value)`.

PyObject *PySequence_List(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um objeto lista com o mesmo conteúdo da sequência ou iterável *o*, ou `NULL` em caso de falha. Garante-se que a lista retornada será nova. Equivale à expressão Python `list(o)`.

PyObject *PySequence_Tuple(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o objeto tupla com o mesmo conteúdo da sequência ou iterável *o*, ou `NULL` em caso de falha. Se *o* for uma tupla, retorna uma nova referência. Senão, uma tupla será construída com o conteúdo apropriado. Equivale à expressão Python `tuple(o)`.

PyObject *PySequence_Fast(PyObject *o, const char *m)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna a sequência ou iterável *o* como um objeto usável por outras funções da família `PySequence_Fast*`. Se o objeto não for uma sequência ou iterável, levanta **`TypeError`** com *m* sendo o texto da mensagem. Retorna `NULL` em caso de falha.

As funções `PySequence_Fast*` têm esse nome porque presumem que *o* é um **`PyTupleObject`** ou um **`PyListObject`** e porque acessam os campos de dados de *o* diretamente.

Como detalhe de implementação de CPython, se *o* já for uma sequência ou lista, ele será retornado.

Py_ssize_t PySequence_Fast_GET_SIZE(PyObject *o)

Retorna o comprimento de *o*, presumindo que *o* foi retornado por **`PySequence_Fast()`** e que *o* não seja `NULL`. O tamanho também pode ser obtido ao chamar **`PySequence_Size()`** em *o*, mas **`PySequence_Fast_GET_SIZE()`** é mais rápida, ao supor que *o* é uma lista ou tupla.

PyObject *PySequence_Fast_GET_ITEM(PyObject *o, Py_ssize_t i)

Retorna valor: Referência emprestada.
Retorna o elemento *i* de *o*, presumindo que *o* foi retornado por **`PySequence_Fast()`**, que *o* seja `NULL`, e que *i* esteja nos limites.

PyObject **PySequence_Fast_ITEMS(PyObject *o)

Retorna o vetor subjacente de ponteiros PyObject. Presume que *o* foi retornado por **`PySequence_Fast()`** e que *o* não seja `NULL`.

Note que, se uma lista for redimensionada, a realocação poderá reposicionar o vetor de itens. Portanto, só use o ponteiro de vetor subjacente em contextos onde a sequência não mudará.

PyObject *PySequence_ITEM(PyObject *o, Py_ssize_t i)

Retorna valor: Nova referência.

Retorna o elemento *i* de *o*, ou `NULL` em caso de falha. É uma forma mais rápida de **`PySequence_GetItem()`**, mas sem verificar se **`PySequence_Check()`** em *o* é verdadeiro e sem ajustar índices negativos.

Protocolo de mapeamento

Veja também `PyObject_GetItem()`, `PyObject_SetItem()` e `PyObject_DelItem()`.

int PyMapping_Check(PyObject *o)

Parte da ABI Estável.

Retorna `1` se o objeto fornece protocolo de mapeamento ou suporta fatiamento e `0` caso contrário. Note que ele retorna `1` para classes Python com um método `__getitem__()` visto que geralmente é impossível determinar a que tipo de chaves a classe tem suporte. Esta função sempre tem sucesso.

Py_ssize_t PyMapping_Size(PyObject *o)

Py_ssize_t PyMapping_Length(PyObject *o)

Parte da ABI Estável.

Retorna o número de chaves no objeto *o* em caso de sucesso e `-1` em caso de falha. Isso é equivalente à expressão Python `len(o)`.

PyObject *PyMapping_GetItemString(PyObject *o, const char *key)

Retorna valor: Nova referência. Parte da ABI Estável.

É o mesmo que `PyObject_GetItem()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

int PyMapping_GetOptionalItem(PyObject *obj, PyObject *key, PyObject **result)

Parte da ABI Estável desde a versão 3.13.

Variante de `PyObject_GetItem()` que não levanta `KeyError` se a chave não for encontrada.

Se a chave for encontrada, retorna `1` e define **result* como uma nova referência forte para o valor correspondente. Se a chave não for encontrada, retorna `0` e define **result* como `NULL`; o `KeyError` é

silenciado. Se um erro diferente de **`KeyError`** for levanta, retorna `-1` e define **result* como `NULL`.

Adicionado na versão 3.13.

int PyMapping_GetOptionalItemString(PyObject *obj, const char *key, PyObject **result)

Parte da ABI Estável desde a versão 3.13.
É o mesmo que **`PyMapping_GetOptionalItem()`**, mas *key* é especificado como uma string de bytes codificada em UTF-8 const char*, em vez de um PyObject*.

Adicionado na versão 3.13.

int PyMapping_SetItemString(PyObject *o, const char *key, PyObject *v)

Parte da ABI Estável.
É o mesmo que **`PyObject_SetItem()`**, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

int PyMapping_DelItem(PyObject *o, PyObject *key)

Este é um apelido de **`PyObject_DelItem()`**.

int PyMapping_DelItemString(PyObject *o, const char *key)

É o mesmo que **`PyObject_DelItem()`**, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

int PyMapping_HasKeyWithError(PyObject *o, PyObject *key)

Parte da ABI Estável desde a versão 3.13.
Retorna `1` se o objeto de mapeamento tiver a chave *key* e `0` caso contrário. Isso é equivalente à expressão Python `key in o`. Em caso de falha, retorna `-1`.

Adicionado na versão 3.13.

int PyMapping_HasKeyStringWithError(PyObject *o, const char *key)

Parte da ABI Estável desde a versão 3.13.

É o mesmo que `PyMapping_HasKeyWithError()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

Adicionado na versão 3.13.

int PyMapping_HasKey(PyObject *o, PyObject *key)

Parte da ABI Estável.

Retorna `1` se o objeto de mapeamento tiver a chave *key* e `0` caso contrário. Isso é equivalente à expressão Python `key in o`. Esta função sempre tem sucesso.

Nota

As exceções que ocorrem quando este construtor chama o método `__getitem__()` são silenciosamente ignoradas. Para o tratamento adequado de erros, use `PyMapping_HasKeyWithError()`, `PyMapping_GetOptionalItem()` ou `PyObject_GetItem()` em vez disso.

int PyMapping_HasKeyString(PyObject *o, const char *key)

Parte da ABI Estável.

É o mesmo que `PyMapping_HasKey()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

Nota

As exceções que ocorrem quando este construtor chama o método `__getitem__()` ou ao criar o objeto temporário `str` são silenciosamente ignoradas. Para o tratamento adequado de erros, use `PyMapping_HasKeyStringWithError()`, `PyMapping_GetOptionalItemString()` ou `PyMapping_GetItemString()` em vez disso.

PyObject *PyMapping_Keys(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Em caso de sucesso, retorna uma lista das chaves no objeto *o*. Em caso de falha, retorna `NULL`.

Alterado na versão 3.7: Anteriormente, a função retornava uma lista ou tupla.

PyObject *PyMapping_Values(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Em caso de sucesso, retorna uma lista dos valores no objeto *o*. Em caso de falha, retorna `NULL`.

Alterado na versão 3.7: Anteriormente, a função retornava uma lista ou tupla.

PyObject *PyMapping_Items(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Em caso de sucesso, retorna uma lista dos itens no objeto *o*, onde cada item é uma tupla contendo um par de valores-chave. Em caso de falha, retorna `NULL`.

Alterado na versão 3.7: Anteriormente, a função retornava uma lista ou tupla.

Protocolo Iterador

Existem duas funções especificas para trabalhar com iteradores.

int PyIter_Check(PyObject *o)

Parte da ABI Estável desde a versão 3.8.
Retorna valor diferente de zero se o objeto *o* puder ser passado com segurança para **`PyIter_NextItem()`**, e `0` caso contrário. Esta função sempre tem sucesso.

int PyAIter_Check(PyObject *o)

Parte da ABI Estável desde a versão 3.10.
Retorna valor diferente de zero se o objeto *o* fornecer o protocolo **`AsyncIterator`** e `0` caso contrário. Esta função sempre é bem-sucedida.

Adicionado na versão 3.10.

int PyIter_NextItem(PyObject *iter, PyObject **item)

Parte da ABI Estável desde a versão 3.14.
Retorna `1` e define *item* como uma referência forte do próximo valor do iterador *iter* em caso de sucesso. Retorna `0` e define *item* como `NULL` se não houver valores restantes. Retorna `-1`, define *item* como `NULL` e define uma exceção em caso de erro.

Adicionado na versão 3.14.

PyObject *PyIter_Next(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável.
Esta é uma versão mais antiga de **`PyIter_NextItem()`**, que foi mantida para retrocompatibilidade. Prefira **`PyIter_NextItem()`**.

Retorna o próximo valor do iterador *o*. O objeto deve ser um iterador de acordo com **`PyIter_Check()`** (cabe ao chamador verificar isso). Se não

houver valores restantes, retorna `NULL` sem nenhuma exceção definida. Se ocorrer um erro ao recuperar o item, retorna `NULL` e passa a exceção.

type PySendResult

O valor de enum usado para representar diferentes resultados de **`PyIter_Send()`**.

Adicionado na versão 3.10.

PySendResult PyIter_Send(PyObject *iter, PyObject *arg, PyObject **presult)

Parte da ABI Estável desde a versão 3.10.

Envia o valor *arg* para o iterador *iter*. Retorna:

- `PYGEN_RETURN` se o iterador retornar. O valor de retorno é retornado via *presult*.
- `PYGEN_NEXT` se o iterador render. O valor produzido é retornado via *presult*.
- `PYGEN_ERROR` se o iterador tiver levantado uma exceção. *presult* é definido como `NULL`.

Adicionado na versão 3.10.

Protocolo de buffer

Certos objetos disponíveis no Python envolvem o acesso a um vetor ou *buffer* de memória subjacente. Esses objetos incluem as classes **`bytes`** e **`bytearray`** embutidas, e alguns tipos de extensão como **`array.array`**. As bibliotecas de terceiros podem definir seus próprios tipos para fins especiais, como processamento de imagem ou análise numérica.

Embora cada um desses tipos tenha sua própria semântica, eles compartilham a característica comum de serem suportados por um buffer de memória possivelmente grande. É desejável, em algumas situações, acessar esse buffer diretamente e sem cópia intermediária.

Python fornece essa facilidade no nível C e Python sob a forma de protocolo de buffer. Este protocolo tem dois lados:

- do lado do produtor, um tipo pode exportar uma "interface de buffer" que permite que objetos desse tipo exponham informações sobre o buffer subjacente. Esta interface é descrita na seção Buffer Object Structures; para Python, consulte Emulando tipos buffer.
- do lado do consumidor, vários meios estão disponíveis para obter o ponteiro para os dados subjacentes de um objeto (por exemplo, um parâmetro de método). Para Python, consulte Emulando tipos buffer.

Objetos simples como **`bytes`** e **`bytearray`** expõem seu buffer subjacente em uma forma orientada a byte. Outras formas são possíveis; por exemplo, os elementos expostos por uma **`array.array`** podem ser valores de vários bytes.

Um exemplo de interface de um consumidor de buffer é o método **`write()`** de objetos arquivo: qualquer objeto que possa exportar uma série de bytes por meio da interface de buffer pode ser gravado em um arquivo. Enquanto o **`write()`** precisa apenas de acesso somente de leitura ao conteúdo interno do objeto passado, outros métodos, como **`readinto()`**, precisam de acesso somente de escrita ao conteúdo interno. A interface de buffer permite que os objetos possam permitir ou rejeitar a exportação para buffers de leitura e escrita ou somente leitura.

Existem duas maneiras para um consumidor da interface de buffer adquirir um buffer em um objeto alvo:

- chamar **`PyObject_GetBuffer()`** com os parâmetros certos;
- chamar **`PyArg_ParseTuple()`** (ou um dos seus irmãos) com um dos códigos de formatação `y*`, `w*` ou `s*`.

Em ambos os casos, `PyBuffer_Release()` deve ser chamado quando o buffer não é mais necessário. A falta de tal pode levar a vários problemas, tais como vazamentos de recursos.

Adicionado na versão 3.12: O protocolo de buffer agora está acessível em Python, veja Emulando tipos buffer e `memoryview`.

Estrutura de Buffer

As estruturas de buffer (ou simplesmente "buffers") são úteis como uma maneira de expor os dados binários de outro objeto para o programador Python. Eles também podem ser usados como um mecanismo de cópia silenciosa. Usando sua capacidade de fazer referência a um bloco de memória, é possível expor facilmente qualquer dado ao programador Python. A memória pode ser uma matriz grande e constante em uma extensão C, pode ser um bloco bruto de memória para manipulação antes de passar para uma biblioteca do sistema operacional, ou pode ser usado para transmitir dados estruturados no formato nativo e formato de memória.

Ao contrário da maioria dos tipos de dados expostos pelo interpretador Python, os buffers não são ponteiros `PyObject` mas sim estruturas C simples. Isso permite que eles sejam criados e copiados de forma muito simples. Quando um invólucro genérico em torno de um buffer é necessário, um objeto memoryview pode ser criado.

Para obter instruções curtas sobre como escrever um objeto exportador, consulte Buffer Object Structures. Para obter um buffer, veja `PyObject_GetBuffer()`.

type Py_buffer

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.11.

void *buf

Um ponteiro para o início da estrutura lógica descrita pelos campos do buffer. Este pode ser qualquer local dentro do bloco de memória física subjacente do exportador. Por exemplo, com negativo `strides` o valor pode apontar para o final do bloco de memória.

Para vetores contíguos, o valor aponta para o início do bloco de memória.

PyObject *obj

Uma nova referência ao objeto sendo exporta. A referência pertence ao consumidor e é automaticamente liberada (por exemplo, a contagem de referências é decrementada) e é atribuída para `NULL` por `PyBuffer_Release()`. O campo equivale ao valor de retorno de qualquer função padrão da API C.

Como um caso especial, para buffers *temporários* que são encapsulados por `PyMemoryView_FromBuffer()` ou `PyBuffer_FillInfo()` esse campo é `NULL`. Em

geral, objetos exportadores NÃO DEVEM usar esse esquema.

Py_ssize_t len

`product(shape) * itemsize`. Para matrizes contíguas, este é o comprimento do bloco de memória subjacente. Para matrizes não contíguas, é o comprimento que a estrutura lógica teria se fosse copiado para uma representação contígua.

Acessando `((char *)buf)[0] up to ((char *)buf)[len-1]` só é válido se o buffer tiver sido obtido por uma solicitação que garanta a contiguidade. Na maioria dos casos, esse pedido será **PyBUF_SIMPLE** ou **PyBUF_WRITABLE**.

int readonly

Um indicador de se o buffer é somente leitura. Este campo é controlado pelo sinalizador **PyBUF_WRITABLE**.

Py_ssize_t itemsize

O tamanho do item em bytes de um único elemento. O mesmo que o valor de **struct.calcsize()** chamado em valores não `NULL` de **format**.

Exceção importante: Se um consumidor requisita um buffer sem sinalizador **PyBUF_FORMAT**, **format** será definido como `NULL`, mas **itemsize** ainda terá seu valor para o formato original.

Se **shape** está presente, a igualdade `product(shape) * itemsize == len` ainda é válida e o usuário pode usar **itemsize** para navegar o buffer.

Se **shape** é `NULL` como resultado de uma **PyBUF_SIMPLE** ou uma requisição **PyBUF_WRITABLE**, o consumidor deve ignorar **itemsize** e presumir `itemsize == 1`.

char *format

Uma string terminada por *NULL* no estilo de sintaxe de módulo **struct** descrevendo os conteúdos de um único item. Se isso é `NULL`, `"B"` (unsigned bytes) é presumido.

Este campo é controlado pelo sinalizador **PyBUF_FORMAT**.

int ndim

O número de dimensões de memória representado como um array n-dimensional. Se for `0`, **buf** aponta para um único elemento representando um escalar. Neste caso, **shape**, **strides** e **suboffsets** DEVEM ser `NULL`. O número máximo de dimensões é dado por **PyBUF_MAX_NDIM**.

Py_ssize_t *shape

Uma matriz de **Py_ssize_t** do comprimento **ndim** indicando a forma da memória como uma matriz n-dimensional. Observe que a forma `shape[0] * ... *`

`shape[ndim-1] * itemsize` DEVE ser igual a **`len`**.

Os valores da forma são restritos a `shape[n] >= 0`. The case `shape[n] == 0` requer atenção especial. Veja complex arrays para mais informações.

A forma de acesso a matriz é de somente leitura para o usuário.

Py_ssize_t *strides

Um vetor de **`Py_ssize_t`** de comprimento **`ndim`** dando o número de bytes para saltar para obter um novo elemento em cada dimensão.

Os valores de Stride podem ser qualquer número inteiro. Para arrays regulares, os passos são geralmente positivos, mas um consumidor DEVE ser capaz de lidar com o caso `strides[n] <= 0`. Veja complex arrays para mais informações.

A matriz de passos é somente leitura para o consumidor.

Py_ssize_t *suboffsets

Uma matriz de **`Py_ssize_t`** de comprimento **`ndim`**. Se `suboffsets[n] >= 0`, os valores armazenados ao longo da n-ésima dimensão são ponteiros e o valor suboffset determina quantos bytes para adicionar a cada ponteiro após desreferenciar. Um valor de suboffset que é negativo indica que não deve ocorrer desreferenciação (caminhando em um bloco de memória contíguo).

Se todos os subconjuntos forem negativos (ou seja, não é necessário fazer referência), então este campo deve ser `NULL` (o valor padrão).

Esse tipo de representação de matriz é usado pela Python Imaging Library (PIL). Veja complex arrays para obter mais informações sobre como acessar elementos dessa matriz.a matriz.

A matriz de subconjuntos é somente leitura para o consumidor.

void *internal

Isso é para uso interno pelo objeto exportador. Por exemplo, isso pode ser re-moldado como um número inteiro pelo exportador e usado para armazenar bandeiras sobre se os conjuntos de forma, passos e suboffsets devem ou não ser liberados quando o buffer é liberado. O consumidor NÃO DEVE alterar esse valor.

Constantes:

PyBUF_MAX_NDIM

Parte da ABI Estável desde a versão 3.11.

O número máximo de dimensões que a memória representa. Exportadores DEVEM respeitar esse limite, consumidores de buffers multidimensionais DEVEM ser capazes

de lidar com até **PyBUF_MAX_NDIM** dimensões. Atualmente definido como 64.

Tipos de solicitação do buffer

Os buffers geralmente são obtidos enviando uma solicitação de buffer para um objeto exportador via **PyObject_GetBuffer()**. Uma vez que a complexidade da estrutura lógica da memória pode variar drasticamente, o consumidor usa o argumento *flags* para especificar o tipo de buffer exato que pode manipular.

Todos os campos **Py_buffer** são definidos de forma não-ambígua pelo tipo de requisição.

campos independentes do pedido

Os seguintes campos não são influenciados por *flags* e devem sempre ser preenchidos com os valores corretos: **obj**, **buf**, **len**, **itemsize**, **ndim**.

apenas em formato

PyBUF_WRITABLE

Parte da ABI Estável desde a versão 3.11.
Controla o campo **readonly** . Se configurado, o exportador DEVE fornecer um buffer gravável ou então relatar falha. Do contrário, o exportador PODE fornecer um buffer somente leitura ou um buffer gravável , mas a escolha DEVE ser consistente para todos os consumidores. Por exemplo, PyBUF_SIMPLE | PyBUF_WRITABLE pode ser usado para requisitar um buffer simples gravável.

PyBUF_WRITEABLE

Isto é um apelido para **PyBUF_WRITABLE**.

Suavemente descontinuado desde a versão 3.13.

PyBUF_FORMAT

Parte da ABI Estável desde a versão 3.11.
Controla o campo **format**. Se configurado, este campo DEVE ser preenchido corretamente. Caso contrário, este campo DEVE ser `NULL`.

:**PyBUF_WRITABLE** pode ser |'d para qualquer um dos sinalizadores na próxima seção. Uma vez que **PyBUF_WRITABLE** é definido como 0, **PyBUF_WRITABLE** pode ser usado como uma bandeira autônoma para solicitar um buffer simples gravável.

PyBUF_FORMAT deve ser |'d para qualquer um dos sinalizadores exceto **PyBUF_SIMPLE**, uma vez que este já indica que o formato `B` (unsigned bytes). **PyBUF_FORMAT** não pode ser utilizado sozinho.

forma, avanços, suboffsets

As bandeiras que controlam a estrutura lógica da memória estão listadas em ordem decrescente de complexidade. Observe que cada bandeira contém todos os bits das bandeiras abaixo.

Solicitação	Forma	Avanços	subconjuntos
PyBUF_INDIRECT *Parte da ABI Estável desde a versão 3.11.*	sim	sim	se necessário
PyBUF_STRIDES *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL
PyBUF_ND *Parte da ABI Estável desde a versão 3.11.*	sim	NULL	NULL
PyBUF_SIMPLE *Parte da ABI Estável desde a versão 3.11.*	NULL	NULL	NULL

requisições contíguas

contiguity do C ou Fortran podem ser explicitamente solicitadas, com ou sem informação de avanço. Sem informação de avanço, o buffer deve ser C-contíguo.

Solicitação	Forma	Avanços	subconjuntos	contig
PyBUF_C_CONTIGUOUS *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	C
PyBUF_F_CONTIGUOUS *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	F

Solicitação	Forma	Avanços	subconjuntos	contig
PyBUF_ANY_CONTIGUOUS *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	C ou F
`PyBUF_ND`	sim	NULL	NULL	C

requisições compostas

Todas as requisições possíveis foram completamente definidas por alguma combinação dos sinalizadores na seção anterior. Por conveniência, o protocolo do buffer fornece combinações frequentemente utilizadas como sinalizadores únicos.

Na seguinte tabela *U* significa contiguidade indefinida. O consumidor deve chamar `PyBuffer_IsContiguous()` para determinar a contiguidade.

Solicitação	Forma	Avanços	subconjuntos	contig	readonly	formato
PyBUF_FULL *Parte da ABI Estável desde a versão 3.11.*	sim	sim	se necessário	U	0	sim
PyBUF_FULL_RO *Parte da ABI Estável desde a versão 3.11.*	sim	sim	se necessário	U	1 ou 0	sim
PyBUF_RECORDS *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	U	0	sim
PyBUF_RECORDS_RO *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	U	1 ou 0	sim

Solicitação	Forma	Avanços	subconjuntos	contig	readonly	formato
PyBUF_STRIDED *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	U	0	NULL
PyBUF_STRIDED_RO *Parte da ABI Estável desde a versão 3.11.*	sim	sim	NULL	U	1 ou 0	NULL
PyBUF_CONTIG *Parte da ABI Estável desde a versão 3.11.*	sim	NULL	NULL	C	0	NULL
PyBUF_CONTIG_RO *Parte da ABI Estável desde a versão 3.11.*	sim	NULL	NULL	C	1 ou 0	NULL

Vetores Complexos

Estilo NumPy: forma e avanços

A estrutura lógica de vetores do estilo NumPy é definida por **`itemsize`**, **`ndim`**, **`shape`** e **`strides`**.

Se `ndim == 0`, a localização da memória apontada para **`buf`** é interpretada como um escalar de tamanho **`itemsize`**. Nesse caso, ambos **`shape`** e **`strides`** são `NULL`.

Se **`strides`** é `NULL`, o vetor é interpretado como um vetor C n-dimensional padrão. Caso contrário, o consumidor deve acessar um vetor n-dimensional como a seguir:

```
ptr = (char *)buf + indices[0] * strides[0] + ... + indices[n-1] * strides[n
item = *((typeof(item) *)ptr);
```

Como notado acima, **`buf`** pode apontar para qualquer localização dentro do bloco de memória em si. Um exportador pode verificar a validade de um buffer com essa função:

```
def verify_structure(memlen, itemsize, ndim, shape, strides, offset):
    """Verifica se os parâmetros representa um vetor válido dentro
```

```
       dos limites da memória alocada:
           char *mem: início do bloco de memória física
           memlen: comprimento do bloco de memória física
           offset: (char *)buf - mem
    """
    if offset % itemsize:
        return False
    if offset < 0 or offset+itemsize > memlen:
        return False
    if any(v % itemsize for v in strides):
        return False

    if ndim <= 0:
        return ndim == 0 and not shape and not strides
    if 0 in shape:
        return True

    imin = sum(strides[j]*(shape[j]-1) for j in range(ndim)
               if strides[j] <= 0)
    imax = sum(strides[j]*(shape[j]-1) for j in range(ndim)
               if strides[j] > 0)

    return 0 <= offset+imin and offset+imax+itemsize <= memlen
```

Estilo-PIL: forma, avanços e suboffsets

Além dos itens normais, uma matriz em estilo PIL pode conter ponteiros que devem ser seguidos para se obter o próximo elemento em uma dimensão. Por exemplo, a matriz tridimensional em C `char v[2][2][3]` também pode ser vista como um vetor de 2 ponteiros para duas matrizes bidimensionais: `char (*v[2])[2][3]`. Na representação por suboffsets, esses dois ponteiros podem ser embutidos no início de **`buf`**, apontando para duas matrizes `char x[2][3]` que podem estar localizadas em qualquer lugar na memória.

Esta é uma função que retorna um ponteiro para o elemento em uma matriz N-D apontada por um índice N-dimensional onde existem ambos passos e subconjuntos não-`NULL`:

```
void *get_item_pointer(int ndim, void *buf, Py_ssize_t *strides,
                       Py_ssize_t *suboffsets, Py_ssize_t *indices) {
    char *pointer = (char*)buf;
    int i;
    for (i = 0; i < ndim; i++) {
        pointer += strides[i] * indices[i];
        if (suboffsets[i] >=0 ) {
            pointer = *((char**)pointer) + suboffsets[i];
        }
    }
    return (void*)pointer;
}
```

Funções relacionadas ao Buffer

int PyObject_CheckBuffer(PyObject *obj)

Parte da ABI Estável desde a versão 3.11.
Retorna `1` se *obj* oferece suporte à interface de buffer, se não, `0`. Quando `1` é retornado, isso não garante que **`PyObject_GetBuffer()`** será bem sucedida. Esta função é sempre bem sucedida.

int PyObject_GetBuffer(PyObject *exporter, Py_buffer *view, int flags)

Parte da ABI Estável desde a versão 3.11.
Envia uma requisição ao *exporter* para preencher a *view* conforme especificado por *flags*. Se o exporter não conseguir prover um buffer do tipo especificado, ele DEVE levantar **`BufferError`**, definir `view->obj` para `NULL` e retornar `-1`.

Em caso de sucesso, preenche *view*, define `view->obj` para uma nova referência para *exporter* e retorna 0. No caso de provedores de buffer encadeados que redirecionam requisições para um único objeto, `view->obj` DEVE se referir a este objeto em vez de *exporter* (Veja Buffer Object Structures).

Chamadas bem sucedidas para **`PyObject_GetBuffer()`** devem ser emparelhadas a chamadas para **`PyBuffer_Release()`**, similar para **`malloc()`** e **`free()`**. Assim, após o consumidor terminar com o buffer, **`PyBuffer_Release()`** deve ser chamado exatamente uma vez.

void PyBuffer_Release(Py_buffer *view)

Parte da ABI Estável desde a versão 3.11.
Libera o buffer de *view* e libera o strong reference (por exemplo, decrementa o contador de referências) para o objeto de suporte da view, `view->obj`. Esta função DEVE ser chamada quando o buffer não estiver mais sendo usado, ou o vazamento de referências pode acontecer.

É um erro chamar essa função em um buffer que não foi obtido via **`PyObject_GetBuffer()`**.

Py_ssize_t PyBuffer_SizeFromFormat(const char *format)

Parte da ABI Estável desde a versão 3.11.
Retorna o **`itemsize`** implícito de **`format`**. Em erro, levantar e exceção e retornar -1.

Adicionado na versão 3.9.

int PyBuffer_IsContiguous(const Py_buffer *view, char order)

Parte da ABI Estável desde a versão 3.11.

Retorna `1` se a memória definida pela *view* é contígua no estilo C (*order* é `'C'`) ou no estilo Fortran (*order* é `'F'`) ou qualquer outra (*order* é `'A'`). Retorna `0` caso contrário. Essa função é sempre bem sucedida.

void *PyBuffer_GetPointer(const Py_buffer *view, const Py_ssize_t *indices)

Parte da ABI Estável desde a versão 3.11.
Recebe a área de memória apontada pelos *indices* dentro da *view* dada. *indices* deve apontar para um array de `view->ndim` índices.

int PyBuffer_FromContiguous(const Py_buffer *view, const void *buf, Py_ssize_t len, char order)

Parte da ABI Estável desde a versão 3.11.
Copia *len* bytes contíguos de *buf* para *view*. *order* pode ser `'C'` ou `'F'` ou `'A'` (para ordenação estilo C ou estilo Fortran ou qualquer um deles). Retorna `0` em caso de sucesso e `-1` em caso de erro.

int PyBuffer_ToContiguous(void *buf, const Py_buffer *src, Py_ssize_t len, char order)

Parte da ABI Estável desde a versão 3.11.
Copia *len* bytes de *src* para sua representação contígua em *buf*. *order* pode ser `'C'` ou `'F'` ou `'A'` (para ordenação estilo C, Fortran ou qualquer uma). O retorno é `0` em caso de sucesso e `-1` em caso de falha.

Esta função falha se *len* != *src->len*.

int PyObject_CopyData(PyObject *dest, PyObject *src)

Parte da ABI Estável desde a versão 3.11.
Copia os dados do buffer *src* para o buffer *dest*. Pode converter entre buffers de estilo C e/ou estilo Fortran.

`0` é retornado em caso de sucesso, `-1` em caso de erro.

void PyBuffer_FillContiguousStrides(int ndims, Py_ssize_t *shape, Py_ssize_t *strides, int itemsize, char order)

Parte da ABI Estável desde a versão 3.11.
Preenche o array *strides* com byte-strides de um array contíguo (estilo C se *order* é `'C'` ou estilo Fortran se *order* for `'F'`) da forma dada com o número dado de bytes por elemento.

int PyBuffer_FillInfo(Py_buffer *view, PyObject *exporter, void *buf, Py_ssize_t len, int readonly, int flags)

Parte da ABI Estável desde a versão 3.11.
Manipula requisições de buffer para um exportador que quer expor *buf* de tamanho *len* com capacidade de escrita definida de acordo com *readonly*. *buf* é interpretada como

uma sequência de bytes sem sinal.

O argumento *flags* indica o tipo de requisição. Esta função sempre preenche *view* como especificado por *flags*, a não ser que *buf* seja designado como somente leitura e **`PyBUF_WRITABLE`** esteja definido em *flags*.

Em caso de sucesso, defina `view->obj` como um novo referência para *exporter* e retorna 0. Caso contrário, levante **`BufferError`** , defina `view->obj` para `NULL` e retorne `-1` ;

Se esta função é usada como parte de um getbufferproc, *exporter* DEVE ser definida para o objeto de exportação e *flags* deve ser passado sem modificações. Caso contrário, *exporter* DEVE ser `NULL`.

Camada de Objetos Concretos

As funções neste capítulo são específicas para certos tipos de objetos Python. Passar para eles um objeto do tipo errado não é uma boa ideia; se você receber um objeto de um programa Python e não tiver certeza de que ele tem o tipo certo, primeiro execute uma verificação de tipo; por exemplo, para verificar se um objeto é um dicionário, use `PyDict_Check()`. O capítulo está estruturado como a "árvore genealógica" dos tipos de objetos Python.

Aviso

Enquanto as funções descritas neste capítulo verificam cuidadosamente o tipo de objetos passados, muitos deles não verificam a passagem de `NULL` em vez de um objeto válido. Permitir a passagem de `NULL` pode causar violações ao acesso à memória e encerramento imediato do interpretador.

Objetos Fundamentais

Esta seção descreve os objetos de tipo Python e o objeto singleton `None`.

- Objetos tipo
  - Creating Heap-Allocated Types
    - Type slot IDs
  - Soft-deprecated API
- O Objeto `None`

Objetos Numéricos

- Objetos Inteiros
  - Export API
  - PyLongWriter API
  - Deprecated API

- Objetos Booleanos
- Objetos de ponto flutuante
  - As funções Pack e Unpack
    - Funções Pack
    - Funções de Unpack
- Objetos números complexos
  - Números complexos como estruturas C.

Objetos Sequência

Operações genéricas em objetos de sequência foram discutidas no capítulo anterior; Esta seção lida com os tipos específicos de objetos sequência que são intrínsecos à linguagem Python.

- Objetos Bytes
- PyBytesWriter
  - Create, Finish, Discard
  - High-level API
  - Getters
  - Low-level API
- Objetos byte array
  - Macros para verificação de tipo
  - Funções diretas da API
  - Macros
- Objetos Unicode e Codecs
  - Objetos Unicode
    - Tipo Unicode
    - Propriedade de caracteres Unicode
    - Creating and accessing Unicode strings
    - Locale Encoding
    - File System Encoding
    - wchar_t Support
  - Built-in Codecs
    - Generic Codecs
    - UTF-8 Codecs
    - UTF-32 Codecs
    - UTF-16 Codecs

        - UTF-7 Codecs
        - Unicode-Escape Codecs
        - Raw-Unicode-Escape Codecs
        - Latin-1 Codecs
        - ASCII Codecs
        - Character Map Codecs
        - MBCS codecs for Windows
    - Methods and Slot Functions
    - PyUnicodeWriter
    - Deprecated API
- Objeto tupla
- Objetos sequência de estrutura
- Objeto List

Coleções

- Dictionary objects
    - Objetos de visão de dicionário
    - Frozen dictionary objects
    - Ordered dictionaries
- Objeto Set
    - Deprecated API

Objetos Função

- Objetos Função
- Objetos de Método de Instância
- Objetos método
- Objeto célula
- Objetos código
- Sinalizadores de objetos código
- Informação adicional

Outros Objetos

- File objects

    - Soft-deprecated API
- Objetos do Módulo
    - Module definition
        - Metadata slots
        - Feature slots
        - Creation and initialization slots
    - Module state
        - Slots for defining module state
        - Module token
    - Creating extension modules dynamically
    - Module definition struct
    - Support functions
        - Module lookup (single-phase initialization)
- Objetos Iteradores
    - Objetos intervalo
    - Tipos de iteradores embutidos
    - Outros objetos iteradores
- Objetos descritores
    - Descritores embutidos
- Objetos Slice
    - Objeto Ellipsis
- Objetos MemoryView
- Objetos buffer serializados com pickle
- Objetos referência fraca
- Capsules
- Objetos sentinel
- Objetos quadro
    - Frame locals proxies
    - Legacy local variable APIs
    - Internal frames
- Objetos Geradores
    - Objetos geradores assíncronos
    - Deprecated API
- Objetos corrotina
- Objetos de variáveis de contexto
- Objetos de indicação de tipos

API C para módulos de extensão

- API C de curses
- Dados internos
- Objetos DateTime
- Dados internos

Objetos tipo

type PyTypeObject

Parte da ABI Estável (como uma estrutura opaca).

A estrutura C dos objetos usados para descrever tipos embutidos.

PyTypeObject PyType_Type

Parte da ABI Estável.

Este é o objeto de tipo para objetos tipo; é o mesmo objeto que `type` na camada Python.

int PyType_Check(PyObject *o)

Retorna valor diferente de zero se o objeto *o* for um objeto tipo, incluindo instâncias de tipos derivados do objeto tipo padrão. Retorna 0 em todos os outros casos. Esta função sempre tem sucesso.

int PyType_CheckExact(PyObject *o)

Retorna valor diferente de zero se o objeto *o* for um objeto tipo, mas não um subtipo do objeto tipo padrão. Retorna 0 em todos os outros casos. Esta função sempre tem sucesso.

unsigned int PyType_ClearCache()

Parte da ABI Estável.

Limpa o cache de pesquisa interno. Retorna a marcação de versão atual.

unsigned long PyType_GetFlags(PyTypeObject *type)

Parte da ABI Estável.

Return the `tp_flags` member of *type*. This function is primarily meant for use with `Py_LIMITED_API`; the individual flag bits are guaranteed to be stable across Python releases, but access to `tp_flags` itself is not part of the limited API.

Adicionado na versão 3.2.

Alterado na versão 3.4: O tipo de retorno é agora um `unsigned long` em vez de um `long`.

PyObject *PyType_GetDict(PyTypeObject *type)

Return the type object's internal namespace, which is otherwise only exposed via a read-only proxy (`cls.__dict__`). This is a replacement for accessing `tp_dict` directly. The returned dictionary must be treated as read-only.

This function is meant for specific embedding and language-binding cases, where direct access to the dict is necessary and indirect access (e.g. via the proxy or `PyObject_GetAttr()`) isn't adequate.

Extension modules should continue to use `tp_dict`, directly or indirectly, when setting up their own types.

Adicionado na versão 3.12.

void PyType_Modified(PyTypeObject *type)

Parte da ABI Estável.
Invalida o cache de pesquisa interna para o tipo e todos os seus subtipos. Esta função deve ser chamada após qualquer modificação manual dos atributos ou classes bases do tipo.

int PyType_AddWatcher(PyType_WatchCallback callback)

Register *callback* as a type watcher. Return a non-negative integer ID which must be passed to future calls to `PyType_Watch()`. In case of error (e.g. no more watcher IDs available), return `-1` and set an exception.

In free-threaded builds, `PyType_AddWatcher()` is not thread-safe, so it must be called at start up (before spawning the first thread).

Adicionado na versão 3.12.

int PyType_ClearWatcher(int watcher_id)

Clear watcher identified by *watcher_id* (previously returned from **PyType_AddWatcher()**). Return `0` on success, `-1` on error (e.g. if *watcher_id* was never registered.)

An extension should never call `PyType_ClearWatcher` with a *watcher_id* that was not returned to it by a previous call to **PyType_AddWatcher()**.

Adicionado na versão 3.12.

int PyType_Watch(int watcher_id, PyObject *type)

Mark *type* as watched. The callback granted *watcher_id* by **PyType_AddWatcher()** will be called whenever **PyType_Modified()** reports a change to *type*. (The callback may be called only once for a series of consecutive modifications to *type*, if **_PyType_Lookup()** is not called on *type* between the modifications; this is an implementation detail and subject to change.)

The callback is also invoked when a watched heap type is deallocated.

An extension should never call `PyType_Watch` with a *watcher_id* that was not returned to it by a previous call to **PyType_AddWatcher()**.

Adicionado na versão 3.12.

Alterado na versão 3.15: The callback is now also invoked when a watched heap type is deallocated.

int PyType_Unwatch(int watcher_id, PyObject *type)

Mark *type* as not watched. This undoes a previous call to **PyType_Watch()**. *type* must not be `NULL`.

An extension should never call this function with a *watcher_id* that was not returned to it by a previous call to **PyType_AddWatcher()**.

On success, this function returns `0`. On failure, this function returns `-1` with an exception set.

Adicionado na versão 3.12.

typedef int (*PyType_WatchCallback)(PyObject *type)

Type of a type-watcher callback function.

The callback must not modify *type* or cause `PyType_Modified()` to be called on *type* or any type in its MRO; violating this rule could cause infinite recursion.

The callback may be called during type deallocation. In this case, the type object is temporarily resurrected (its reference count is at least 1) and all its attributes are still valid. However, the callback should not store new strong references to the type, as this would resurrect the object and prevent its deallocation.

Adicionado na versão 3.12.

Alterado na versão 3.15: The callback may now be called during deallocation of a watched heap type.

int PyType_HasFeature(PyTypeObject *o, int feature)

Retorna valor diferente de zero se o objeto tipo *o* define o recurso *feature*. Os recursos de tipo são denotados por sinalizadores de bit único.

int PyType_FastSubclass(PyTypeObject *type, int flag)

Return non-zero if the type object *type* sets the subclass flag *flag*. Subclass flags are denoted by `Py_TPFLAGS_*_SUBCLASS`. This function is used by many `_Check` functions for common types.

Ver também

`PyObject_TypeCheck()`, which is used as a slower alternative in `_Check` functions for types that don't come with subclass flags.

int PyType_IS_GC(PyTypeObject *o)

Return true if the type object includes support for the cycle detector; this tests the type flag `Py_TPFLAGS_HAVE_GC`.

int PyType_IsSubtype(PyTypeObject *a, PyTypeObject *b)

Parte da ABI Estável.
Retorna verdadeiro se *a* for um subtipo de *b*.

This function only checks for actual subtypes, which means that `__subclasscheck__()` is not called on *b*. Call `PyObject_IsSubclass()` to do the same check that `issubclass()` would do.

PyObject *PyType_GenericAlloc(PyTypeObject *type, Py_ssize_t nitems)

Retorna valor: Nova referência. Parte da ABI Estável.
Generic handler for the `tp_alloc` slot of a type object. Uses Python's default memory allocation mechanism to allocate memory for a new instance, zeros the memory, then initializes the memory as if by calling `PyObject_Init()` or `PyObject_InitVar()`.

Do not call this directly to allocate memory for an object; call the type's `tp_alloc` slot instead.

For types that support garbage collection (i.e., the `Py_TPFLAGS_HAVE_GC` flag is set), this function behaves like `PyObject_GC_New` or `PyObject_GC_NewVar` (except the memory is guaranteed to be zeroed before initialization), and should be paired with `PyObject_GC_Del()` in `tp_free`. Otherwise, it behaves like `PyObject_New` or `PyObject_NewVar` (except the memory is guaranteed to be zeroed before initialization) and should be paired with `PyObject_Free()` in `tp_free`.

PyObject *PyType_GenericNew(PyTypeObject *type, PyObject *args, PyObject *kwds)

Retorna valor: Nova referência. Parte da ABI Estável.
Generic handler for the `tp_new` slot of a type object. Creates a new instance using the type's `tp_alloc` slot and returns the resulting object.

int PyType_Ready(PyTypeObject *type)

Parte da ABI Estável.

Finaliza um objeto tipo. Isso deve ser chamado em todos os objetos tipo para finalizar sua inicialização. Esta função é responsável por adicionar slots herdados da classe base de um tipo. Retorna `0` em caso de sucesso, ou retorna `-1` e define uma exceção em caso de erro.

Nota

If some of the base classes implements the GC protocol and the provided type does not include the **`Py_TPFLAGS_HAVE_GC`** in its flags, then the GC protocol will be automatically implemented from its parents. On the contrary, if the type being created does include **`Py_TPFLAGS_HAVE_GC`** in its flags then it **must** implement the GC protocol itself by at least implementing the **`tp_traverse`** handle.

PyObject *PyType_GetName(PyTypeObject *type)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.11.

Retorna o nome do tipo. Equivalente a pegar o atributo **`__name__`** do tipo.

Adicionado na versão 3.11.

PyObject *PyType_GetQualName(PyTypeObject *type)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.11.

Retorna o nome qualificado do tipo. Equivalente a pegar o atributo **`__qualname__`** do tipo.

Adicionado na versão 3.11.

PyObject *PyType_GetFullyQualifiedName(PyTypeObject *type)

Parte da ABI Estável desde a versão 3.13.

Retorna o nome qualificado do tipo. Equivalente à `f"{type.__module__}.{type.__qualname__}"`, ou **`type.__qualname__`** se **`type.__module__`** não é uma string ou é igual a `"builtins"`.

Adicionado na versão 3.13.

PyObject *PyType_GetModuleName(PyTypeObject *type)

Parte da ABI Estável desde a versão 3.13.
Retorna o nome do tipo do módulo. Equivalente a pegar o atributo `type.__module__`.

Adicionado na versão 3.13.

void *PyType_GetSlot(PyTypeObject *type, int slot)

Parte da ABI Estável desde a versão 3.4.
Retorna o ponteiro de função armazenado no slot fornecido. Se o resultado for `NULL`, isso indica que o slot é `NULL` ou que a função foi chamada com parâmetros inválidos. Os chamadores normalmente lançarão o ponteiro do resultado no tipo de função apropriado.

Veja `PyType_Slot.slot` por possíveis valores do argumento *slot*.

Adicionado na versão 3.4.

Alterado na versão 3.10: `PyType_GetSlot()` agora aceita todos os tipos. Antes, estava limitada a tipos heap.

PyObject *PyType_GetModule(PyTypeObject *type)

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.10.
Retorna o objeto de módulo associado ao tipo fornecido quando o tipo foi criado usando `PyType_FromModuleAndSpec()`.

The returned reference is borrowed from *type*, and will be valid as long as you hold a reference to *type*. Do not release it with `Py_DECREF()` or similar.

Se nenhum módulo estiver associado com o tipo fornecido, define `TypeError` e retorna `NULL`.

This function is usually used to get the module in which a method is defined. Note that in such a method, `PyType_GetModule(Py_TYPE(self))` may not return the intended result. `Py_TYPE(self)` may be a *subclass* of the intended class, and subclasses are not necessarily defined in the same module as their superclass. See **`PyCMethod`** to get the class that defines the method. See **`PyType_GetModuleByToken()`** for cases when **`PyCMethod`** cannot be used.

Adicionado na versão 3.9.

void *PyType_GetModuleState(PyTypeObject *type)

Parte da ABI Estável desde a versão 3.10.

Return the state of the module object associated with the given type. This is a shortcut for calling **`PyModule_GetState()`** on the result of **`PyType_GetModule()`**.

Se nenhum módulo estiver associado com o tipo fornecido, define **`TypeError`** e retorna `NULL`.

Se o *tipo* tem um módulo associado mas o seu estado é `NULL`, retorna `NULL` sem definir uma exceção.

Adicionado na versão 3.9.

PyObject *PyType_GetModuleByToken(PyTypeObject *type, const void *mod_token)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.15.

Find the first superclass whose module has the given module token, and return that module.

Se nenhum módulo é encontrado, levanta um **`TypeError`** e retorna `NULL`.

This function is intended to be used together with **`PyModule_GetState()`** to get module state from slot methods (such as

tp_init or **nb_add**) and other places where a method's defining class cannot be passed using the **PyCMethod** calling convention.

Adicionado na versão 3.15.

PyObject *PyType_GetModuleByDef(PyTypeObject *type, struct PyModuleDef *def)

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.13.

Find the first superclass whose module was created from the given **PyModuleDef** *def*, or whose module token is equal to *def*, and return that module.

Note that modules created from a **PyModuleDef** always have their token set to the **PyModuleDef**'s address. In other words, this function is equivalent to **PyType_GetModuleByToken()**, except that it:

- returns a borrowed reference, and
- has a non-`void*` argument type (which is a cosmetic difference in C).

The returned reference is borrowed from *type*, and will be valid as long as you hold a reference to *type*. Do not release it with **Py_DECREF()** or similar.

Adicionado na versão 3.11.

int PyType_GetBaseByToken(PyTypeObject *type, void *tp_token, PyTypeObject **result)

Parte da ABI Estável desde a versão 3.14.

Find the first superclass in *type*'s method resolution order whose **Py_tp_token** token is equal to *tp_token*.

- If found, set **result* to a new strong reference to it and return `1`.
- If not found, set **result* to `NULL` and return `0`.
- On error, set **result* to `NULL` and return `-1` with an exception set.

The *result* argument may be `NULL`, in which case **result* is not set. Use this if you need only the return value.

The *tp_token* argument may not be `NULL`.

Adicionado na versão 3.14.

int PyUnstable_Type_AssignVersionTag(PyTypeObject *type)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Attempt to assign a version tag to the given type.

Returns 1 if the type already had a valid version tag or a new one was assigned, or 0 if a new tag could not be assigned.

Adicionado na versão 3.12.

int PyType_SUPPORTS_WEAKREFS(PyTypeObject *type)

Return true if instances of *type* support creating weak references, false otherwise. This function always succeeds. *type* must not be `NULL`.

Ver também

- Objetos referência fraca
- **`weakref`**

Creating Heap-Allocated Types

The following function is used to create heap types:

PyObject *PyType_FromSlots(const PySlot *slots)

Parte da ABI Estável desde a versão 3.15.

Create and return a heap type from a **`PySlot`** array. See Definition slots for general information on slots, and Type slot IDs for slots specific to type creation.

This function calls **PyType_Ready()** on the new type.

Note that this function does *not* fully match the behavior of calling **type()** or using the **class** statement. With user-provided base types or metaclasses, prefer calling **type** (or the metaclass) over `PyType_From*` functions. Specifically:

- **__new__()** is not called on the new class (and it must be set to `type.__new__`).
- **__init__()** is not called on the new class.
- **__init_subclass__()** is not called on any bases.
- **__set_name__()** is not called on new descriptors.

Slots are typically defined as a global static constant arrays. However, sometimes slot values are not statically known at compile time. For example, slots like **Py_tp_bases**, **Py_tp_metaclass** and **Py_tp_module** require live Python objects. In this case, it is recommended to put such slots on the stack, and use **Py_slot_subslots** to refer to an array of static slots. For example:

```
static const PySlot my_slots[] = {
   PySlot_STATIC_DATA(Py_tp_name, "MyClass"),
   PySlot_FUNC(Py_tp_repr, my_repr_func),
   ...
   PySlot_END
};

PyObject *make_my_class(PyObject *module) {
   PySlot all_slots[] = {
      PySlot_STATIC_DATA(Py_slot_subslots, my_slots),
      PySlot_DATA(Py_tp_module, module),
      PySlot_END
   };
   return PyType_FromSlots(all_slots);
}
```

Heap types created without the **Py_TPFLAGS_IMMUTABLETYPE** flag may be modified, for example by setting attributes on them, as with classes defined in Python code. Sometimes, such modifications are necessary to fully

initialize a type, but you may wish to prevent users from changing the type after the initialization is done:

int PyType_Freeze(PyTypeObject *type)

Parte da ABI Estável desde a versão 3.14.

Make a type immutable: set the `Py_TPFLAGS_IMMUTABLETYPE` flag.

All base classes of *type* must be immutable.

On success, return `0`. On error, set an exception and return `-1`.

The type must not be used before it's made immutable. For example, type instances must not be created before the type is made immutable.

Adicionado na versão 3.14.

Type slot IDs

Most type slot IDs are named like the field names of the structures `PyTypeObject`, `PyNumberMethods`, `PySequenceMethods`, `PyMappingMethods` and `PyAsyncMethods` with an added `Py_` prefix. For example, use:

- `Py_tp_dealloc` to set `PyTypeObject.tp_dealloc`
- `Py_nb_add` to set `PyNumberMethods.nb_add`
- `Py_sq_length` to set `PySequenceMethods.sq_length`

The following slots need additional considerations when specified as slots:

- `Py_tp_name`
- `Py_tp_basicsize` and `Py_tp_extra_basicsize`
- `Py_tp_itemsize`
- `Py_tp_flags`

Additional slots do not directly correspond to a `PyTypeObject` struct field:

- `Py_tp_token`
- `Py_tp_metaclass`
- `Py_tp_module`

The following “offset” fields cannot be set using `PyType_Slot`:

- `tp_weaklistoffset` (use `Py_TPFLAGS_MANAGED_WEAKREF` instead if possible)
- `tp_dictoffset` (use `Py_TPFLAGS_MANAGED_DICT` instead if possible)
- `tp_vectorcall_offset` (use `"__vectorcalloffset__"` in PyMemberDef)

If it is not possible to switch to a `MANAGED` flag (for example, for vectorcall or to support Python older than 3.12), specify the offset in `Py_tp_members`. See PyMemberDef documentation for details.

The following internal fields cannot be set at all when creating a heap type:

- `tp_dict`, `tp_mro`, `tp_cache`, `tp_subclasses`, and `tp_weaklist`.

The `Py_tp_base` slot is equivalent to `Py_tp_bases`; both may be set either to a type or a tuple of types. If both are specified, the value of **`Py_tp_bases`** is used.

Slot values may not be `NULL`, except for the following:

- `Py_tp_doc`
- `Py_tp_token` (for clarity, prefer `Py_TP_USE_SPEC` rather than `NULL`)

Alterado na versão 3.9: Slots in `PyBufferProcs` may be set in the unlimited API.

Alterado na versão 3.11: `bf_getbuffer` and `bf_releasebuffer` are now available under the limited API.

Alterado na versão 3.14: The field `tp_vectorcall` can now be set using `Py_tp_vectorcall`. See the field’s documentation for details.

Alterado na versão 3.15: The `Py_tp_bases` slot may be set to a single type object, making it equivalent to the `Py_tp_base` slot. Previously, a tuple of types was required.

The following slots correspond to fields in the underlying type structure, but need extra remarks for use as slots:

Py_tp_name

Parte da ABI Estável desde a versão 3.15.

`Slot ID` for the name of the type, used to set `PyTypeObject.tp_name`.

This slot (or `PyType_Spec.name`) is required to create a type.

This may not be used in `PyType_Spec.slots`. Use `PyType_Spec.name` instead.

Detalhes da implementação do CPython: CPython processes slots in order. It is recommended to put `Py_tp_name` at the beginning of the slots array, so that if processing of a later slots fails, error messages can include the name.

Adicionado na versão 3.15.

Py_tp_basicsize

Parte da ABI Estável desde a versão 3.15.

`Slot ID` for the size of the instance in bytes. It is used to set `PyTypeObject.tp_basicsize`.

The value must be positive.

This may not be used in `PyType_Spec.slots`. Use `PyType_Spec.basicsize` instead.

This slot may not be used with `PyType_GetSlot()`. Use `PyTypeObject.tp_basicsize` instead if needed, but be aware that a type's size is often considered an implementation detail.

Adicionado na versão 3.15.

Py_tp_extra_basicsize

Parte da ABI Estável desde a versão 3.15.

Slot ID for type data size in bytes, that is, how much space instances of the class need *in addition* to space needed for superclasses.

The value is used, together with the size of superclasses, to set **PyTypeObject.tp_basicsize**. Python will insert padding as needed to meet **tp_basicsize**'s alignment requirements.

Use **PyObject_GetTypeData()** to get a pointer to subclass-specific memory reserved this way.

The value must be positive. To specify that instances need no additional size (that is, size should be inherited), omit the **Py_tp_extra_basicsize** slot rather than set it to zero.

Specifying both **Py_tp_basicsize** and **Py_tp_extra_basicsize** is an error.

This may not be used in **PyType_Spec.slots**. Use negative **PyType_Spec.basicsize** instead.

This slot may not be used with **PyType_GetSlot()**.

Adicionado na versão 3.15.

Py_tp_itemsize

Parte da ABI Estável desde a versão 3.15.
Slot ID for the size of one element of a variable-size type, in bytes. Used to set **PyTypeObject.tp_itemsize**. See **tp_itemsize** documentation for caveats.

The value must be positive.

If this slot is missing, **tp_itemsize** is inherited. Extending arbitrary variable-sized classes is dangerous, since some types use a fixed offset for variable-sized memory, which can then overlap fixed-sized memory used by a subclass. To help prevent mistakes, inheriting `itemsize` is only possible in the following situations:

- The base is not variable-sized (its `tp_itemsize` is zero).
- The requested `PyType_Spec.basicsize` is positive, suggesting that the memory layout of the base class is known.
- The requested `PyType_Spec.basicsize` is zero, suggesting that the subclass does not access the instance's memory directly.
- With the `Py_TPFLAGS_ITEMS_AT_END` flag.

This may not be used in `PyType_Spec.slots`. Use `PyType_Spec.itemsize` instead.

This slot may not be used with `PyType_GetSlot()`.

Adicionado na versão 3.15.

Py_tp_flags

Parte da ABI Estável desde a versão 3.15.
`Slot_ID` for type flags, used to set `PyTypeObject.tp_flags`.

The `Py_TPFLAGS_HEAPTYPE` flag is not set, `PyType_FromSpecWithBases()` sets it automatically.

This may not be used in `PyType_Spec.slots`. Use negative `PyType_Spec.basicsize` instead.

This slot may not be used with `PyType_GetSlot()`. Use `PyType_GetFlags()` instead.

Adicionado na versão 3.15.

Py_tp_bases

Parte da ABI Estável.
`Slot_ID` for type flags, used to set `PyTypeObject.tp_bases`.

The slot can be set to a tuple of type objects which the newly created type should inherit from, like the "positional arguments" of a Python class definition.

Alternately, the slot can be set to a single type object to specify a single base. The effect is the same as specifying a one-element tuple.

Alterado na versão 3.15: Previously, **Py_tp_bases** required a tuple of types.

Py_tp_base

Parte da ABI Estável.
Equivalent to **Py_tp_bases** (with `s` at the end). If both are specified, **Py_tp_bases** takes priority and this slot is ignored.

Alterado na versão 3.15: Previously, **Py_tp_base** required a single type, not a tuple.

Suavemente descontinuado desde a versão 3.15: When not targeting older Python versions, prefer **Py_tp_bases**.

The following slots do not correspond to public fields in the underlying structures:

Py_tp_metaclass

Parte da ABI Estável desde a versão 3.15.
Slot ID for the metaclass used to construct the resulting type object. When omitted the metaclass is derived from bases (**Py_tp_bases** or the *bases* argument of **PyType_FromMetaclass()**).

Metaclasses that override **tp_new** are not supported, except if `tp_new` is `NULL`.

This may not be used in **PyType_Spec.slots**. Use **PyType_FromMetaclass()** to specify a metaclass with **PyType_Spec**.

This slot may not be used with **PyType_GetSlot()**. Use **Py_TYPE()** on the type object instead.

Adicionado na versão 3.15.

Py_tp_module

Parte da ABI Estável desde a versão 3.15.
Slot ID for recording the module in which the new class is defined.

The value must be a module object. The module is associated with the new type and can later be retrieved with **PyType_GetModule()**. The associated module is not inherited by subclasses; it must be specified for each class individually.

This may not be used in **PyType_Spec.slots**. Use **PyType_FromMetaclass()** to specify a module with **PyType_Spec**.

This slot may not be used with **PyType_GetSlot()**. Use **PyType_GetModule()** instead.

Adicionado na versão 3.15.

Py_tp_token

Parte da ABI Estável desde a versão 3.14.

Slot_ID for recording a static memory layout ID for a class.

If the class is defined using a **PyType_Spec**, and that spec is statically allocated, the token can be set to the spec using the special value **Py_TP_USE_SPEC**:

```
static PyType_Slot foo_slots[] = {
   {Py_tp_token, Py_TP_USE_SPEC},
```

It can also be set to an arbitrary pointer, but you must ensure that:

- The pointer outlives the class, so it's not reused for something else while the class exists.
- It "belongs" to the extension module where the class lives, so it will not clash with other extensions.

Use **PyType_GetBaseByToken()** to check if a class's superclass has a given token – that is, check whether the memory layout is compatible.

To get the token for a given class (without considering superclasses), use **PyType_GetSlot()** with `Py_tp_token`.

Adicionado na versão 3.14.

Py_TP_USE_SPEC

Parte da ABI Estável desde a versão 3.14.

Used as a value with **Py_tp_token** to set the token to the class's **PyType_Spec**. May only be used for classes defined using **PyType_Spec**.

Expands to `NULL`.

Adicionado na versão 3.14.

Py_tp_slots

Parte da ABI Estável desde a versão 3.15.

Slot_ID that works like **Py_slot_subslots**, except it specifies an array of **PyType_Slot** structures.

Adicionado na versão 3.15.

Soft-deprecated API

The following functions are soft deprecated. They will continue to work, but new features will be added as slots for **PyType_FromSlots()**, not as arguments to new `PyType_From*` functions.

PyObject *PyType_FromMetaclass(PyTypeObject *metaclass, PyObject *module, PyType_Spec *spec, PyObject *bases)

Parte da ABI Estável desde a versão 3.12.

Create and return a heap type from the *spec* (see **Py_TPFLAGS_HEAPTYPE**).

A non-`NULL` *metaclass* argument corresponds to the **Py_tp_metaclass** slot.

A non-`NULL` *bases* argument corresponds to the **Py_tp_bases** slot, and takes precedence over **Py_tp_bases** and **Py_tp_bases** slots.

A non-`NULL` *module* argument corresponds to the **Py_tp_module** slot.

This function calls **PyType_Ready()** on the new type.

Note that this function does *not* fully match the behavior of calling **type()** or using the **class** statement. See the note in **PyType_FromSlots()** documentation for details.

Adicionado na versão 3.12.

Suavemente descontinuado desde a versão 3.15: Prefer **PyType_FromSlots()** in new code.

PyObject *PyType_FromModuleAndSpec(PyObject *module, PyType_Spec *spec, PyObject *bases)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.10.

Equivalent to `PyType_FromMetaclass(NULL, module, spec, bases)`.

Adicionado na versão 3.9.

Alterado na versão 3.10: The function now accepts a single class as the *bases* argument and `NULL` as the `tp_doc` slot.

Alterado na versão 3.12: The function now finds and uses a metaclass corresponding to the provided base classes. Previously, only **type** instances were returned.

The **tp_new** of the metaclass is *ignored*. which may result in incomplete initialization. Creating classes whose metaclass overrides **tp_new** is deprecated.

Alterado na versão 3.14: Creating classes whose metaclass overrides **tp_new** is no longer allowed.

Suavemente descontinuado desde a versão 3.15: Prefer **PyType_FromSlots()** in new code.

PyObject *PyType_FromSpecWithBases(PyType_Spec *spec, PyObject *bases)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.3.
Equivalent to `PyType_FromMetaclass(NULL, NULL, spec, bases)`.

Adicionado na versão 3.3.

Alterado na versão 3.12: The function now finds and uses a metaclass corresponding to the provided base classes. Previously, only **type** instances were returned.

The **tp_new** of the metaclass is *ignored*. which may result in incomplete initialization. Creating classes whose metaclass overrides **tp_new** is deprecated.

Alterado na versão 3.14: Creating classes whose metaclass overrides **tp_new** is no longer allowed.

Suavemente descontinuado desde a versão 3.15: Prefer **PyType_FromSlots()** in new code.

PyObject *PyType_FromSpec(PyType_Spec *spec)

Retorna valor: Nova referência. Parte da ABI Estável.
Equivalent to `PyType_FromMetaclass(NULL, NULL, spec, NULL)`.

Alterado na versão 3.12: The function now finds and uses a metaclass corresponding to the base classes provided in *Py_tp_base[s]* slots. Previously, only **type** instances were returned.

The **tp_new** of the metaclass is *ignored*. which may result in incomplete initialization. Creating classes whose metaclass overrides **tp_new** is deprecated.

Alterado na versão 3.14: Creating classes whose metaclass overrides **tp_new** is no longer allowed.

Suavemente descontinuado desde a versão 3.15: Prefer **PyType_FromSlots()** in new code.

type PyType_Spec

Parte da ABI Estável (incluindo todos os membros).
Structure defining a type's behavior, used for soft-deprecated functions like `PyType_FromMetaclass()`.

This structure contains several members that can instead be specified as slots for `PyType_FromSlots()`, and an array of slot entries with a simpler structure.

const char *name

Corresponds to `Py_tp_name`.

int basicsize

If positive, corresponds to `Py_tp_basicsize`.

If negative, corresponds to `Py_tp_extra_basicsize` set to the absolute value.

Alterado na versão 3.12: Previously, this field could not be negative.

int itemsize

Corresponds to `Py_tp_itemsize`.

unsigned int flags

Corresponds to `Py_tp_flags`.

PyType_Slot *slots

Array of `PyType_Slot` (not `PySlot`) structures.

Terminated by the special slot value `{0, NULL}`. Each slot ID should be specified at most once.

type PyType_Slot

Parte da ABI Estável (incluindo todos os membros).
Structure defining optional functionality of a type, used for soft-deprecated functions like `PyType_FromMetaclass()`.

Note that a **PyType_Slot** array may be included in a **PySlot** array using **Py_tp_slots**, and vice versa using **Py_slot_subslots**.

Each **PyType_Slot** structure `tpslot` is interpreted as the following **PySlot** structure:

```
(PySlot){
   .sl_id=tpslot.slot,
   .sl_flags=PySlot_INTPTR | sub_static,
   .sl_ptr=tpslot.func
}
```

where `sub_static` is `PySlot_STATIC` if the slot requires the flag (such as for **Py_tp_methods**), or if this flag is present on the "parent" **Py_tp_slots** slot (if any).

int slot

Corresponds to **PySlot.sl_id**.

void *pfunc

Corresponds to **PySlot.sl_ptr**.

O Objeto `None`

Observe que o **`PyTypeObject`** para `None` não está diretamente exposto pela API Python/C. Como `None` é um singleton, é suficiente testar a identidade do objeto (usando `==` em C). Não há nenhuma função **`PyNone_Check()`** pela mesma razão.

PyObject *Py_None

O objeto Python `None`, denotando falta de valor. Este objeto não tem métodos e é imortal.

Alterado na versão 3.12: **`Py_None`** é imortal.

Py_RETURN_NONE

Retorna **`Py_None`** de uma função.

Objetos Inteiros

Todos os inteiros são implementados como objetos inteiros “longos” de tamanho arbitrário.

Em caso de erro, a maioria das APIs `PyLong_As*` retorna `(tipo de retorno)-1` que não pode ser distinguido de um número. Use **`PyErr_Occurred()`** para desambiguar.

type PyLongObject

Parte da ABI Estável (como uma estrutura opaca).
Este subtipo de **`PyObject`** representa um objeto inteiro Python.

PyTypeObject PyLong_Type

Parte da ABI Estável.
Esta instância de **`PyTypeObject`** representa o tipo inteiro Python. Este é o mesmo objeto que **`int`** na camada Python.

int PyLong_Check(PyObject *p)

Retorna true se seu argumento é um **`PyLongObject`** ou um subtipo de **`PyLongObject`**. Esta função sempre tem sucesso.

int PyLong_CheckExact(PyObject *p)

Retorna true se seu argumento é um **`PyLongObject`**, mas não um subtipo de **`PyLongObject`**. Esta função sempre tem sucesso.

PyObject *PyLong_FromLong(long v)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna um novo objeto **`PyLongObject`** de *v* ou `NULL` em caso de falha.

Detalhes da implementação do CPython: CPython keeps an array of integer objects for all integers between `-5` and `1024`. When you create an int in that range you actually just get back a reference to the existing object.

PyObject *PyLong_FromUnsignedLong(unsigned long v)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto **PyLongObject** de um unsigned long C ou NULL em caso de falha.

PyObject *PyLong_FromSsize_t(Py_ssize_t v)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto **PyLongObject** de um **Py_ssize_t** C ou NULL em caso de falha.

PyObject *PyLong_FromSize_t(size_t v)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto **PyLongObject** de um **size_t** C ou NULL em caso de falha.

PyObject *PyLong_FromLongLong(long long v)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto **PyLongObject** de um long long C ou NULL em caso de falha.

PyObject *PyLong_FromUnsignedLongLong(unsigned long long v)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto **PyLongObject** de um unsigned long long C ou NULL em caso de falha.

PyObject *PyLong_FromInt32(int32_t value)

PyObject *PyLong_FromInt64(int64_t value)

Parte da ABI Estável desde a versão 3.14.

Retorna um novo objeto **PyLongObject** de um int32_t ou int64_t C com sinal, ou NULL com uma exceção definida no caso de falha.

Adicionado na versão 3.14.

PyObject *PyLong_FromUInt32(uint32_t value)

PyObject *PyLong_FromUInt64(uint64_t value)

Parte da ABI Estável desde a versão 3.14.

Retorna um novo objeto **`PyLongObject`** de um uint32_t ou uint64_t C sem sinal, ou `NULL` com uma exceção definida no caso de falha.

Adicionado na versão 3.14.

PyObject *PyLong_FromDouble(double v)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna um novo objeto **`PyLongObject`** da parte inteira de *v* ou `NULL` em caso de falha.

PyObject *PyLong_FromString(const char *str, char **pend, int base)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna um novo **`PyLongObject`** com base no valor da string em *str*, que é interpretado de acordo com a raiz em *base* ou `NULL` em caso de falha. Se *pend* não for `NULL`, **pend* apontará para o fim do *str* em caso de sucesso ou para seu primeiro caractere que não pode ser processado em caso de erro. Se *base* for `0`, *str* é interpretado usando a definição de Inteiros literais; neste caso, zeros à esquerda em um número decimal diferente de zero aumenta um **`ValueError`**. Se *base* não for `0`, deve estar entre `2` e `36`, inclusive. Espaços em branco no início ou no final e sublinhados simples após um especificador de base e entre dígitos são ignorados. Se não houver dígitos ou *str* não for terminado em NULL após os dígitos e espaços em branco à direita, **`ValueError`** será levantada.

Ver também

As funções **`PyLong_AsNativeBytes()`** e **`PyLong_FromNativeBytes()`** podem ser usadas para converter um **`PyLongObject`** de/para uma vetor de bytes na base `256`.

PyObject *PyLong_FromUnicodeObject(PyObject *u, int base)

Retorna valor: Nova referência.
Converte uma sequência de dígitos Unicode na string *u* para um valor inteiro Python.

Adicionado na versão 3.3.

PyObject *PyLong_FromVoidPtr(void *p)

Retorna valor: Nova referência. Parte da ABI Estável.

Cria um inteiro Python a partir do ponteiro *p*. O valor do ponteiro pode ser recuperado do valor resultante usando **PyLong_AsVoidPtr()**.

PyObject *PyLong_FromNativeBytes(const void *buffer, size_t n_bytes, int flags)

Parte da ABI Estável desde a versão 3.14.

Create a Python integer from the value contained in the first *n_bytes* of *buffer*, interpreted as a two's-complement signed number.

flags are as for **PyLong_AsNativeBytes()**. Passing `-1` will select the native endian that CPython was compiled with and assume that the most-significant bit is a sign bit. Passing `Py_ASNATIVEBYTES_UNSIGNED_BUFFER` will produce the same result as calling **PyLong_FromUnsignedNativeBytes()**. Other flags are ignored.

Adicionado na versão 3.13.

PyObject *PyLong_FromUnsignedNativeBytes(const void *buffer, size_t n_bytes, int flags)

Parte da ABI Estável desde a versão 3.14.

Create a Python integer from the value contained in the first *n_bytes* of *buffer*, interpreted as an unsigned number.

flags are as for **PyLong_AsNativeBytes()**. Passing `-1` will select the native endian that CPython was compiled with and assume that the most-significant bit is not a sign bit. Flags other than endian are ignored.

Adicionado na versão 3.13.

PyLong_FromPid(pid)

Macro for creating a Python integer from a process identifier.

This can be defined as an alias to **PyLong_FromLong()** or **PyLong_FromLongLong()**, depending on the size of the system's PID type.

Adicionado na versão 3.2.

long PyLong_AsLong(PyObject *obj)

Parte da ABI Estável.

Return a C long representation of *obj*. If *obj* is not an instance of **PyLongObject**, first call its **__index__()** method (if present) to convert it to a **PyLongObject**.

Raise **OverflowError** if the value of *obj* is out of range for a long.

Retorna `-1` no caso de erro. Use **PyErr_Occurred()** para desambiguar.

Alterado na versão 3.8: Usa **__index__()**, se disponível.

Alterado na versão 3.10: This function will no longer use **__int__()**.

long PyLong_AS_LONG(PyObject *obj)

Exactly equivalent to the preferred `PyLong_AsLong`. In particular, it can fail with **OverflowError** or another exception.

Suavemente descontinuado desde a versão 3.14.

int PyLong_AsInt(PyObject *obj)

Parte da ABI Estável desde a versão 3.13.

Similar to **PyLong_AsLong()**, but store the result in a C int instead of a C long.

Adicionado na versão 3.13.

long PyLong_AsLongAndOverflow(PyObject *obj, int *overflow)

Parte da ABI Estável.

Return a C long representation of *obj*. If *obj* is not an instance of **PyLongObject**, first call its **__index__()** method (if present) to convert

it to a **PyLongObject**.

If the value of *obj* is greater than **LONG_MAX** or less than **LONG_MIN**, set **overflow* to `1` or `-1`, respectively, and return `-1`; otherwise, set **overflow* to `0`. If any other exception occurs set **overflow* to `0` and return `-1` as usual.

Retorna `-1` no caso de erro. Use **PyErr_Occurred()** para desambiguar.

Alterado na versão 3.8: Usa **__index__()**, se disponível.

Alterado na versão 3.10: This function will no longer use **__int__()**.

long long PyLong_AsLongLong(PyObject *obj)

Parte da ABI Estável.

Return a C long long representation of *obj*. If *obj* is not an instance of **PyLongObject**, first call its **__index__()** method (if present) to convert it to a **PyLongObject**.

Raise **OverflowError** if the value of *obj* is out of range for a long long.

Retorna `-1` no caso de erro. Use **PyErr_Occurred()** para desambiguar.

Alterado na versão 3.8: Usa **__index__()**, se disponível.

Alterado na versão 3.10: This function will no longer use **__int__()**.

long long PyLong_AsLongLongAndOverflow(PyObject *obj, int *overflow)

Parte da ABI Estável.

Return a C long long representation of *obj*. If *obj* is not an instance of **PyLongObject**, first call its **__index__()** method (if present) to convert it to a **PyLongObject**.

If the value of *obj* is greater than **LLONG_MAX** or less than **LLONG_MIN**, set **overflow* to `1` or `-1`, respectively, and return `-1`; otherwise, set **overflow* to `0`. If any other exception occurs set **overflow* to `0` and return `-1` as usual.

Retorna `-1` no caso de erro. Use `PyErr_Occurred()` para desambiguar.

Adicionado na versão 3.2.

Alterado na versão 3.8: Usa `__index__()`, se disponível.

Alterado na versão 3.10: This function will no longer use `__int__()`.

Py_ssize_t PyLong_AsSsize_t(PyObject *pylong)

Parte da ABI Estável.
Retorna uma representação de `Py_ssize_t` C de *pylong*. *pylong* deve ser uma instância de `PyLongObject`.

Levanta `OverflowError` se o valor de *pylong* estiver fora do intervalo de um `Py_ssize_t`.

Retorna `-1` no caso de erro. Use `PyErr_Occurred()` para desambiguar.

unsigned long PyLong_AsUnsignedLong(PyObject *pylong)

Parte da ABI Estável.
Return a C unsigned long representation of *pylong*. *pylong* must be an instance of `PyLongObject`.

Raise `OverflowError` if the value of *pylong* is out of range for a unsigned long.

Retorna `(unsigned long)-1` no caso de erro. Use `PyErr_Occurred()` para desambiguar.

size_t PyLong_AsSize_t(PyObject *pylong)

Parte da ABI Estável.
Retorna uma representação de `size_t` C de *pylong*. *pylong* deve ser uma instância de `PyLongObject`.

Levanta `OverflowError` se o valor de *pylong* estiver fora do intervalo de um `size_t`.

Retorna `(size)-1` no caso de erro. Use **`PyErr_Occurred()`** para desambiguar.

unsigned long long PyLong_AsUnsignedLongLong(PyObject *pylong)

Parte da ABI Estável.

Return a C unsigned long long representation of *pylong*. *pylong* must be an instance of **`PyLongObject`**.

Raise **`OverflowError`** if the value of *pylong* is out of range for an unsigned long long.

Retorna `(unsigned long long)-1` no caso de erro. Use **`PyErr_Occurred()`** para desambiguar.

Alterado na versão 3.1: Um *pylong* negativo agora levanta **`OverflowError`**, não **`TypeError`**.

unsigned long PyLong_AsUnsignedLongMask(PyObject *obj)

Parte da ABI Estável.

Return a C unsigned long representation of *obj*. If *obj* is not an instance of **`PyLongObject`**, first call its **`__index__()`** method (if present) to convert it to a **`PyLongObject`**.

If the value of *obj* is out of range for an unsigned long, return the reduction of that value modulo `ULONG_MAX + 1`.

Retorna `(unsigned long)-1` no caso de erro. Use **`PyErr_Occurred()`** para desambiguar.

Alterado na versão 3.8: Usa **`__index__()`**, se disponível.

Alterado na versão 3.10: This function will no longer use **`__int__()`**.

unsigned long long PyLong_AsUnsignedLongLongMask(PyObject *obj)

Parte da ABI Estável.

Return a C unsigned long long representation of *obj*. If *obj* is not an instance of **`PyLongObject`**, first call its **`__index__()`** method (if present)

to convert it to a **PyLongObject**.

If the value of *obj* is out of range for an unsigned long long, return the reduction of that value modulo `ULLONG_MAX + 1`.

Retorna `(unsigned long long)-1` no caso de erro. Use **PyErr_Occurred()** para desambiguar.

Alterado na versão 3.8: Usa **__index__()**, se disponível.

Alterado na versão 3.10: This function will no longer use **__int__()**.

int PyLong_AsInt32(PyObject *obj, int32_t *value)
int PyLong_AsInt64(PyObject *obj, int64_t *value)

Parte da ABI Estável desde a versão 3.14.
Set **value* to a signed C int32_t or int64_t representation of *obj*.

If *obj* is not an instance of **PyLongObject**, first call its **__index__()** method (if present) to convert it to a **PyLongObject**.

If the *obj* value is out of range, raise an **OverflowError**.

Set **value* and return `0` on success. Set an exception and return `-1` on error.

value must not be `NULL`.

Adicionado na versão 3.14.

int PyLong_AsUInt32(PyObject *obj, uint32_t *value)
int PyLong_AsUInt64(PyObject *obj, uint64_t *value)

Parte da ABI Estável desde a versão 3.14.
Set **value* to an unsigned C uint32_t or uint64_t representation of *obj*.

If *obj* is not an instance of **PyLongObject**, first call its **__index__()** method (if present) to convert it to a **PyLongObject**.

- If *obj* is negative, raise a **ValueError**.

- If the *obj* value is out of range, raise an `OverflowError`.

Set **value* and return `0` on success. Set an exception and return `-1` on error.

value must not be `NULL`.

Adicionado na versão 3.14.

double PyLong_AsDouble(PyObject *pylong)

Parte da ABI Estável.
Return a C double representation of *pylong*. *pylong* must be an instance of `PyLongObject`.

Raise `OverflowError` if the value of *pylong* is out of range for a double.

Retorna `-1.0` no caso de erro. Use `PyErr_Occurred()` para desambiguar.

void *PyLong_AsVoidPtr(PyObject *pylong)

Parte da ABI Estável.
Convert a Python integer *pylong* to a C void pointer. If *pylong* cannot be converted, an `OverflowError` will be raised. This is only assured to produce a usable void pointer for values created with `PyLong_FromVoidPtr()`.

Retorna `NULL` no caso de erro. Use `PyErr_Occurred()` para desambiguar.

Py_ssize_t PyLong_AsNativeBytes(PyObject *pylong, void *buffer, Py_ssize_t n_bytes, int flags)

Parte da ABI Estável desde a versão 3.14.
Copy the Python integer value *pylong* to a native *buffer* of size *n_bytes*. The *flags* can be set to `-1` to behave similarly to a C cast, or to values documented below to control the behavior.

Returns `-1` with an exception raised on error. This may happen if *pylong* cannot be interpreted as an integer, or if *pylong* was negative and the

`Py_ASNATIVEBYTES_REJECT_NEGATIVE` flag was set.

Otherwise, returns the number of bytes required to store the value. If this is equal to or less than *n_bytes*, the entire value was copied. All *n_bytes* of the buffer are written: remaining bytes filled by copies of the sign bit.

If the returned value is greater than *n_bytes*, the value was truncated: as many of the lowest bits of the value as could fit are written, and the higher bits are ignored. This matches the typical behavior of a C-style downcast.

> **Nota**
>
> Overflow is not considered an error. If the returned value is larger than *n_bytes*, most significant bits were discarded.

`0` will never be returned.

Values are always copied as two's-complement.

Usage example:

```
int32_t value;
Py_ssize_t bytes = PyLong_AsNativeBytes(pylong, &value, sizec
if (bytes < 0) {
    // Failed. A Python exception was set with the reason.
    return NULL;
}
else if (bytes <= (Py_ssize_t)sizeof(value)) {
    // Success!
}
else {
    // Overflow occurred, but 'value' contains the truncated
    // lowest bits of pylong.
}
```

Passing zero to *n_bytes* will return the size of a buffer that would be large enough to hold the value. This may be larger than technically

necessary, but not unreasonably so. If *n_bytes=0*, *buffer* may be `NULL`.

Nota

Passing *n_bytes=0* to this function is not an accurate way to determine the bit length of the value.

To get at the entire Python value of an unknown size, the function can be called twice: first to determine the buffer size, then to fill it:

```
// Ask how much space we need.
Py_ssize_t expected = PyLong_AsNativeBytes(pylong, NULL, 0, -
if (expected < 0) {
    // Failed. A Python exception was set with the reason.
    return NULL;
}
assert(expected != 0);  // Impossible per the API definition.
uint8_t *bignum = malloc(expected);
if (!bignum) {
    PyErr_SetString(PyExc_MemoryError, "bignum malloc failed.
    return NULL;
}
// Safely get the entire value.
Py_ssize_t bytes = PyLong_AsNativeBytes(pylong, bignum, expec
if (bytes < 0) {  // Exception has been set.
    free(bignum);
    return NULL;
}
else if (bytes > expected) {  // This should not be possible.
    PyErr_SetString(PyExc_RuntimeError,
        "Unexpected bignum truncation after a size check.");
    free(bignum);
    return NULL;
}
// The expected success given the above pre-check.
// ... use bignum ...
free(bignum);
```

flags is either `-1` (`Py_ASNATIVEBYTES_DEFAULTS`) to select defaults that behave most like a C cast, or a combination of the other flags in the

table below. Note that `-1` cannot be combined with other flags.

Currently, `-1` corresponds to `Py_ASNATIVEBYTES_NATIVE_ENDIAN | Py_ASNATIVEBYTES_UNSIGNED_BUFFER`.

Sinalizador	Valor
Py_ASNATIVEBYTES_DEFAULTS *Parte da ABI Estável desde a versão 3.14.*	`-1`
Py_ASNATIVEBYTES_BIG_ENDIAN *Parte da ABI Estável desde a versão 3.14.*	`0`
Py_ASNATIVEBYTES_LITTLE_ENDIAN *Parte da ABI Estável desde a versão 3.14.*	`1`
Py_ASNATIVEBYTES_NATIVE_ENDIAN *Parte da ABI Estável desde a versão 3.14.*	`3`
Py_ASNATIVEBYTES_UNSIGNED_BUFFER *Parte da ABI Estável desde a versão 3.14.*	`4`
Py_ASNATIVEBYTES_REJECT_NEGATIVE *Parte da ABI Estável desde a versão 3.14.*	`8`
Py_ASNATIVEBYTES_ALLOW_INDEX *Parte da ABI Estável desde a versão 3.14.*	`16`

Specifying `Py_ASNATIVEBYTES_NATIVE_ENDIAN` will override any other endian flags. Passing `2` is reserved.

By default, sufficient buffer will be requested to include a sign bit. For example, when converting 128 with *n_bytes=1*, the function will return 2 (or more) in order to store a zero sign bit.

If `Py_ASNATIVEBYTES_UNSIGNED_BUFFER` is specified, a zero sign bit will be omitted from size calculations. This allows, for example, 128 to fit in a single-byte buffer. If the destination buffer is later treated as signed, a

positive input value may become negative. Note that the flag does not affect handling of negative values: for those, space for a sign bit is always requested.

Specifying `Py_ASNATIVEBYTES_REJECT_NEGATIVE` causes an exception to be set if *pylong* is negative. Without this flag, negative values will be copied provided there is enough space for at least one sign bit, regardless of whether `Py_ASNATIVEBYTES_UNSIGNED_BUFFER` was specified.

If `Py_ASNATIVEBYTES_ALLOW_INDEX` is specified and a non-integer value is passed, its `__index__()` method will be called first. This may result in Python code executing and other threads being allowed to run, which could cause changes to other objects or values in use. When *flags* is `-1`, this option is not set, and non-integer values will raise `TypeError`.

Nota

With the default *flags* (`-1`, or *UNSIGNED_BUFFER* without *REJECT_NEGATIVE*), multiple Python integers can map to a single value without overflow. For example, both `255` and `-1` fit a single-byte buffer and set all its bits. This matches typical C cast behavior.

Adicionado na versão 3.13.

PyLong_AsPid(pid)

Macro for converting a Python integer into a process identifier.

This can be defined as an alias to `PyLong_AsLong()`, `PyLong_FromLongLong()`, or `PyLong_AsInt()`, depending on the size of the system's PID type.

Adicionado na versão 3.2.

int PyLong_GetSign(PyObject *obj, int *sign)

Get the sign of the integer object *obj*.

On success, set *sign to the integer sign (0, -1 or +1 for zero, negative or positive integer, respectively) and return 0.

On failure, return -1 with an exception set. This function always succeeds if *obj* is a `PyLongObject` or its subtype.

Adicionado na versão 3.14.

int PyLong_IsPositive(PyObject *obj)

Check if the integer object *obj* is positive (`obj > 0`).

If *obj* is an instance of `PyLongObject` or its subtype, return `1` when it's positive and `0` otherwise. Else set an exception and return `-1`.

Adicionado na versão 3.14.

int PyLong_IsNegative(PyObject *obj)

Check if the integer object *obj* is negative (`obj < 0`).

If *obj* is an instance of `PyLongObject` or its subtype, return `1` when it's negative and `0` otherwise. Else set an exception and return `-1`.

Adicionado na versão 3.14.

int PyLong_IsZero(PyObject *obj)

Check if the integer object *obj* is zero.

If *obj* is an instance of `PyLongObject` or its subtype, return `1` when it's zero and `0` otherwise. Else set an exception and return `-1`.

Adicionado na versão 3.14.

PyObject *PyLong_GetInfo(void)

Parte da ABI Estável.

On success, return a read only named tuple, that holds information about Python's internal representation of integers. See `sys.int_info` for description of individual fields.

Em caso de falha, retorna `NULL` com uma exceção definida.

Adicionado na versão 3.1.

int PyUnstable_Long_IsCompact(const PyLongObject *op)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Return 1 if *op* is compact, 0 otherwise.

This function makes it possible for performance-critical code to implement a "fast path" for small integers. For compact values use `PyUnstable_Long_CompactValue()`; for others fall back to a `PyLong_As*` function or `PyLong_AsNativeBytes()`.

The speedup is expected to be negligible for most users.

Exactly what values are considered compact is an implementation detail and is subject to change.

Adicionado na versão 3.12.

Py_ssize_t PyUnstable_Long_CompactValue(const PyLongObject *op)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

If *op* is compact, as determined by `PyUnstable_Long_IsCompact()`, return its value.

Otherwise, the return value is undefined.

Adicionado na versão 3.12.

Export API

Adicionado na versão 3.14.

type PyLongLayout

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.15.

Layout of an array of “digits” (“limbs” in the GMP terminology), used to represent absolute value for arbitrary precision integers.

Use `PyLong_GetNativeLayout()` to get the native layout of Python `int` objects, used internally for integers with “big enough” absolute value.

See also `sys.int_info` which exposes similar information in Python.

uint8_t bits_per_digit

Bits per digit. For example, a 15 bit digit means that bits 0-14 contain meaningful information.

uint8_t digit_size

Digit size in bytes. For example, a 15 bit digit will require at least 2 bytes.

int8_t digits_order

Digits order:

- `1` for most significant digit first
- `-1` for least significant digit first

int8_t digit_endianness

Digit endianness:

- `1` for most significant byte first (big endian)
- `-1` for least significant byte first (little endian)

const PyLongLayout *PyLong_GetNativeLayout(void)

Parte da ABI Estável desde a versão 3.15.

Get the native layout of Python `int` objects.

See the `PyLongLayout` structure.

The function must not be called before Python initialization nor after Python finalization. The returned layout is valid until Python is finalized. The layout is the same for all Python sub-interpreters in a process, and so it can be cached.

type PyLongExport

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.15.

Export of a Python `int` object.

There are two cases:

- If `digits` is `NULL`, only use the `value` member.
- If `digits` is not `NULL`, use `negative`, `ndigits` and `digits` members.

int64_t value

The native integer value of the exported `int` object. Only valid if `digits` is `NULL`.

uint8_t negative

`1` if the number is negative, `0` otherwise. Only valid if `digits` is not `NULL`.

Py_ssize_t ndigits

Number of digits in `digits` array. Only valid if `digits` is not `NULL`.

const void *digits

Read-only array of unsigned digits. Can be `NULL`.

int PyLong_Export(PyObject *obj, PyLongExport *export_long)

Parte da ABI Estável desde a versão 3.15.

Export a Python `int` object.

export_long must point to a `PyLongExport` structure allocated by the caller. It must not be `NULL`.

On success, fill in **export_long* and return `0`. On error, set an exception and return `-1`.

`PyLong_FreeExport()` must be called when the export is no longer needed.

Detalhes da implementação do CPython: This function always succeeds if *obj* is a Python `int` object or a subclass.

void PyLong_FreeExport(PyLongExport *export_long)

Parte da ABI Estável desde a versão 3.15.
Release the export *export_long* created by `PyLong_Export()`.

Detalhes da implementação do CPython: Calling `PyLong_FreeExport()` is optional if *export_long->digits* is `NULL`.

PyLongWriter API

The `PyLongWriter` API can be used to import an integer.

Adicionado na versão 3.14.

type PyLongWriter

Parte da ABI Estável (como uma estrutura opaca) desde a versão 3.15.
A Python `int` writer instance.

The instance must be destroyed by `PyLongWriter_Finish()` or `PyLongWriter_Discard()`.

PyLongWriter *PyLongWriter_Create(int negative, Py_ssize_t ndigits, void **digits)

Parte da ABI Estável desde a versão 3.15.
Create a `PyLongWriter`.

On success, allocate **digits* and return a writer. On error, set an exception and return `NULL`.

negative is `1` if the number is negative, or `0` otherwise.

ndigits is the number of digits in the *digits* array. It must be greater than 0.

digits must not be NULL.

After a successful call to this function, the caller should fill in the array of digits *digits* and then call **`PyLongWriter_Finish()`** to get a Python **`int`**. The layout of *digits* is described by **`PyLong_GetNativeLayout()`**.

Digits must be in the range [`0`; `(1 << bits_per_digit) - 1`] (where the **`bits_per_digit`** is the number of bits per digit). Any unused most significant digits must be set to `0`.

Alternately, call **`PyLongWriter_Discard()`** to destroy the writer instance without creating an **`int`** object.

PyObject *PyLongWriter_Finish(PyLongWriter *writer)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.15.

Finish a **`PyLongWriter`** created by **`PyLongWriter_Create()`**.

On success, return a Python **`int`** object. On error, set an exception and return `NULL`.

The function takes care of normalizing the digits and converts the object to a compact integer if needed.

The writer instance and the *digits* array are invalid after the call.

void PyLongWriter_Discard(PyLongWriter *writer)

Parte da ABI Estável desde a versão 3.15.

Discard a **`PyLongWriter`** created by **`PyLongWriter_Create()`**.

If *writer* is `NULL`, no operation is performed.

The writer instance and the *digits* array are invalid after the call.

Deprecated API

These macros are soft deprecated. They describe parameters of the internal representation of `PyLongObject` instances.

Use `PyLong_GetNativeLayout()` instead, along with `PyLong_Export()` to read integer data or `PyLongWriter` to write it. These currently use the same layout, but are designed to continue working correctly even if CPython's internal integer representation changes.

PyLong_SHIFT

This is equivalent to `bits_per_digit` in the output of `PyLong_GetNativeLayout()`.

PyLong_BASE

This is currently equivalent to 1 << PyLong_SHIFT.

PyLong_MASK

This is currently equivalent to (1 << PyLong_SHIFT) - 1

Objetos Booleanos

Os booleanos no Python são implementados como um subclasse de inteiros. Há apenas dois booleanos, `Py_False` e `Py_True`. Assim sendo, as funções de criação e a exclusão normais não se aplicam aos booleanos. No entanto, as seguintes macros estão disponíveis.

PyTypeObject PyBool_Type

Parte da ABI Estável.

Este instância de `PyTypeObject` representa o tipo booleano em Python; é o mesmo objeto que `bool` na camada Python.

int PyBool_Check(PyObject *o)

Retorna verdadeiro se *o* for do tipo `PyBool_Type`. Esta função sempre tem sucesso.

PyObject *Py_False

O objeto Python `False`. Este objeto não tem métodos e é imortal.

Alterado na versão 3.12: `Py_False` é imortal.

PyObject *Py_True

O objeto Python `True`. Este objeto não tem métodos e é imortal.

Alterado na versão 3.12: `Py_True` é imortal.

Py_RETURN_FALSE

Retorna `Py_False` de uma função.

Py_RETURN_TRUE

Retorna `Py_True` de uma função.

PyObject *PyBool_FromLong(long v)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna `Py_True` ou `Py_False`, dependendo do valor verdade de *v*.

Objetos de ponto flutuante

type PyFloatObject

Este subtipo de **`PyObject`** representa um objeto de ponto flutuante do Python.

PyTypeObject PyFloat_Type

Parte da ABI Estável.

Esta instância do **`PyTypeObject`** representa o tipo de ponto flutuante do Python. Este é o mesmo objeto **`float`** na camada do Python.

int PyFloat_Check(PyObject *p)

Retorna true se seu argumento é um **`PyFloatObject`** ou um subtipo de **`PyFloatObject`**. Esta função sempre tem sucesso.

int PyFloat_CheckExact(PyObject *p)

Retorna true se seu argumento é um **`PyFloatObject`**, mas um subtipo de **`PyFloatObject`**. Esta função sempre tem sucesso.

PyObject *PyFloat_FromString(PyObject *str)

Retorna valor: Nova referência. Parte da ABI Estável.

Cria um objeto **`PyFloatObject`** baseado em uma string de valor “str” ou `NULL` em falha.

PyObject *PyFloat_FromDouble(double v)

Retorna valor: Nova referência. Parte da ABI Estável.

Cria um objeto **`PyFloatObject`** de *v* ou `NULL` em falha.

double PyFloat_AsDouble(PyObject *pyfloat)

Parte da ABI Estável.

Retorna uma representação C double do conteúdo de *pyfloat*. Se *pyfloat* não é um objeto de ponto flutuante do Python, mas possui o método **`__float__()`**, esse método será chamado primeiro para converter *pyfloat* em um ponto flutuante. Se **`__float__()`** não estiver definido,

será usado **`__index__()`**. Este método retorna `-1.0` em caso de falha, portanto, deve-se chamar **`PyErr_Occurred()`** para verificar se há erros.

Alterado na versão 3.8: Usa **`__index__()`**, se disponível.

double PyFloat_AS_DOUBLE(PyObject *pyfloat)

Retorna uma representação C double do conteúdo de *pyfloat*, mas sem verificação de erro.

PyObject *PyFloat_GetInfo(void)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna uma instância de structseq que contém informações sobre a precisão, os valores mínimo e máximo de um ponto flutuante. É um invólucro fino em torno do arquivo de cabeçalho `float.h`.

double PyFloat_GetMax()

Parte da ABI Estável.

Retorna o ponto flutuante finito máximo representável *DBL_MAX* como double do C.

double PyFloat_GetMin()

Parte da ABI Estável.

Retorna o ponto flutuante positivo mínimo normalizado *DBL_MIN* como double do C.

Py_INFINITY

This macro expands to a constant expression of type double, that represents the positive infinity.

It is equivalent to the **`INFINITY`** macro from the C11 standard `<math.h>` header.

Suavemente descontinuado desde a versão 3.15.

Py_NAN

This macro expands to a constant expression of type double, that represents a quiet not-a-number (qNaN) value.

Na maioria das plataformas, isso equivale à macro **NAN** do cabeçalho `<math.h>` do padrão C11.

Py_HUGE_VAL

Equivalente a **INFINITY**.

Suavemente descontinuado desde a versão 3.14.

Py_MATH_E

A definição (precisa para um tipo double) da constante **math.e**.

Py_MATH_El

Definição de alta precisão (long double) da constante **e**.

Descontinuado desde a versão 3.15, será removido na versão 3.20.

Py_MATH_PI

A definição (precisa para um tipo double) da constante **math.pi**.

Py_MATH_PIl

Definição de alta precisão (long double) da constante **pi**.

Descontinuado desde a versão 3.15, será removido na versão 3.20.

Py_MATH_TAU

A definição (precisa para um tipo double) da constante **math.tau**.

Adicionado na versão 3.6.

Py_RETURN_NAN

Retorna **math.nan** de uma função.

Na maioria das plataformas, isso equivale a `return PyFloat_FromDouble(NAN)`.

Py_RETURN_INF(sign)

Retorna **math.inf** ou **-math.inf** de uma função, dependendo do sinal de *sign*.

Na maioria das plataformas, isso equivale ao seguinte:

```
return PyFloat_FromDouble(copysign(INFINITY, sign));
```

Py_IS_FINITE(X)

Retorna `1` se o número de ponto flutuante *X* fornecido for finito, ou seja, for normal, subnormal ou zero, mas não infinito ou NaN. Caso contrário, retorna `0`.

Suavemente descontinuado desde a versão 3.14: Use **`isfinite`** instead.

Py_IS_INFINITY(X)

Retorna `1` se o número de ponto flutuante *X* fornecido for infinito positivo ou negativo. Caso contrário, retorna `0`.

Suavemente descontinuado desde a versão 3.14: Use **`isinf`** instead.

Py_IS_NAN(X)

Retorna `1` se o número de ponto flutuante fornecido *X* for um valor não numérico (NaN). Caso contrário, retorna `0`.

Suavemente descontinuado desde a versão 3.14: Use **`isnan`** instead.

As funções Pack e Unpack

The pack and unpack functions provide an efficient platform-independent way to store floating-point values as byte strings. The Pack routines produce a bytes string from a C double, and the Unpack routines produce a C double from such a bytes string. The suffix (2, 4 or 8) specifies the number of bytes in the bytes string:

- The 2-byte format is the IEEE 754 binary16 half-precision format.
- The 4-byte format is the IEEE 754 binary32 single-precision format.
- The 8-byte format is the IEEE 754 binary64 double-precision format.

The NaN type may not be preserved on some platforms while unpacking (signaling NaNs become quiet NaNs), for example on x86 systems in 32-bit

mode.

It's assumed that the double type has the IEEE 754 binary64 double precision format. What happens if it's not true is partly accidental (alas). On non-IEEE platforms with more precision, or larger dynamic range, than IEEE 754 supports, not all values can be packed; on non-IEEE platforms with less precision, or smaller dynamic range, not all values can be unpacked. The packing of special numbers like INFs and NaNs (if such things exist on the platform) may not be handled correctly, and attempting to unpack a bytes string containing an IEEE INF or NaN may raise an exception.

Adicionado na versão 3.11.

Funções Pack

The pack routines write 2, 4 or 8 bytes, starting at *p*. *le* is an int argument, non-zero if you want the bytes string in little-endian format (exponent last, at `p+1`, `p+3`, or `p+6` and `p+7`), zero if you want big-endian format (exponent first, at *p*). Use the **`PY_LITTLE_ENDIAN`** constant to select the native endian: it is equal to `0` on big endian processor, or `1` on little endian processor.

Valor de retorno: `0` se tudo estiver OK, `-1` se houver erro (e uma exceção for definida, provavelmente **`OverflowError`**).

int PyFloat_Pack2(double x, char *p, int le)

Empacota um C duplo como o formato de meia precisão binário16 IEEE 754.

int PyFloat_Pack4(double x, char *p, int le)

Empacota um C duplo como o formato de precisão simples binário32 IEEE 754.

int PyFloat_Pack8(double x, char *p, int le)

Empacota um C duplo como o formato de precisão dupla binária 64 IEEE 754.

Funções de Unpack

The unpack routines read 2, 4 or 8 bytes, starting at *p*. *le* is an int argument, non-zero if the bytes string is in little-endian format (exponent last, at `p+1`, `p+3` or `p+6` and `p+7`), zero if big-endian (exponent first, at *p*). Use the **`PY_LITTLE_ENDIAN`** constant to select the native endian: it is equal to `0` on big endian processor, or `1` on little endian processor.

Valor de retorno: O double descompactado. Em caso de erro, é `-1.0` e **`PyErr_Occurred()`** é verdadeiro (e uma exceção é definida, provavelmente **`OverflowError`**).

double PyFloat_Unpack2(const char *p, int le)

Descompacta o formato de meia precisão IEEE 754 binary16 como um double C.

double PyFloat_Unpack4(const char *p, int le)

Descompacta o formato IEEE 754 binary32 de precisão simples como um C double.

double PyFloat_Unpack8(const char *p, int le)

Descompacta o formato IEEE 754 binary64 de precisão dupla como um C double.

Objetos números complexos

type PyComplexObject

Este subtipo de **PyObject** representa um objeto Python de número complexo.

Py_complex cval

The complex number value, using the C **Py_complex** representation.

Descontinuado desde a versão 3.15, será removido na versão 3.20: Use **PyComplex_AsCComplex()** and **PyComplex_FromCComplex()** to convert a Python complex number to/from the C **Py_complex** representation.

PyTypeObject PyComplex_Type

Parte da ABI Estável.

Esta instância de **PyTypeObject** representa o tipo de número complexo Python. É o mesmo objeto que **complex** na camada Python.

int PyComplex_Check(PyObject *p)

Retorna true se seu argumento é um **PyComplexObject** ou um subtipo de **PyComplexObject**. Esta função sempre tem sucesso.

int PyComplex_CheckExact(PyObject *p)

Retorna true se seu argumento é um **PyComplexObject**, mas não um subtipo de **PyComplexObject**. Esta função sempre tem sucesso.

PyObject *PyComplex_FromDoubles(double real, double imag)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto **PyComplexObject** de *real* e *imag*. Retorna `NULL` com uma exceção definida ao ocorrer um erro.

double PyComplex_RealAsDouble(PyObject *op)

Parte da ABI Estável.

Retorna a parte real de *op* como um double C.

Se *op* não é um objeto de número complexo Python, mas tem um método `__complex__()`, este método será primeiro chamado para converter *op* em um objeto de número complexo Python. Se **`__complex__()`** não estiver definido, ele volta a chamar `PyFloat_AsDouble()` e retorna seu resultado.

Em caso de falha, este método retorna `-1.0` com uma exceção definida, então deve-se chamar `PyErr_Occurred()` para verificar se há erros.

Alterado na versão 3.13: Usa `__complex__()`, se disponível.

double PyComplex_ImagAsDouble(PyObject *op)

Parte da ABI Estável.

Retorna a parte imaginária de *op* como um double C.

Se *op* não é um objeto de número complexo Python, mas tem um método `__complex__()`, este método será primeiro chamado para converter *op* em um objeto de número complexo Python. Se **`__complex__()`** não estiver definido, ele volta a chamar `PyFloat_AsDouble()` e retorna `0.0` em caso de sucesso.

Em caso de falha, este método retorna `-1.0` com uma exceção definida, então deve-se chamar `PyErr_Occurred()` para verificar se há erros.

Alterado na versão 3.13: Usa `__complex__()`, se disponível.

type Py_complex

This C structure defines an export format for a Python complex number object.

double real
double imag

A estrutura é definida como:

```
typedef struct {
    double real;
```

```
    double imag;
} Py_complex;
```

PyObject *PyComplex_FromCComplex(Py_complex v)

Retorna valor: Nova referência.

Cria um novo objeto de número complexo Python a partir de um valor C `Py_complex`. Retorna `NULL` com uma exceção definida ao ocorrer um erro.

Py_complex PyComplex_AsCComplex(PyObject *op)

Retorna o valor `Py_complex` do número complexo *op*.

Se *op* não é um objeto de número complexo Python, mas tem um método `__complex__()`, este método será primeiro chamado para converter *op* em um objeto de número complexo Python. Se `__complex__()` não for definido, então ele recorre a `__float__()`. Se `__float__()` não estiver definido, então ele volta para `__index__()`.

Em caso de falha, este método retorna `Py_complex` com `real` definido para `-1.0` e com uma exceção definida, então deve-se chamar `PyErr_Occurred()` para verificar se há erros.

Alterado na versão 3.8: Usa `__index__()`, se disponível.

Números complexos como estruturas C.

The API also provides functions for working with complex numbers, using the `Py_complex` representation. Note that the functions which accept these structures as parameters and return them as results do so *by value* rather than dereferencing them through pointers.

Please note, that these functions are soft deprecated since Python 3.15. Avoid using this API in a new code to do complex arithmetic: either use the Number Protocol API or use native complex types, like double complex.

Py_complex _Py_c_sum(Py_complex left, Py_complex right)

Retorna a soma de dois números complexos, utilizando a representação C `Py_complex`.

Descontinuado desde a versão 3.15.

Py_complex _Py_c_diff(Py_complex left, Py_complex right)

Retorna a diferença entre dois números complexos, utilizando a representação C `Py_complex`.

Descontinuado desde a versão 3.15.

Py_complex _Py_c_neg(Py_complex num)

Retorna a negação do número complexo *num*, utilizando a representação C `Py_complex`.

Descontinuado desde a versão 3.15.

Py_complex _Py_c_prod(Py_complex left, Py_complex right)

Retorna o produto de dois números complexos, utilizando a representação C `Py_complex`.

Descontinuado desde a versão 3.15.

Py_complex _Py_c_quot(Py_complex dividend, Py_complex divisor)

Retorna o quociente de dois números complexos, utilizando a representação C `Py_complex`.

Se *divisor* é nulo, este método retorna zero e define `errno` para `EDOM`.

Descontinuado desde a versão 3.15.

Py_complex _Py_c_pow(Py_complex num, Py_complex exp)

Retorna a exponenciação de *num* por *exp*, utilizando a representação C `Py_complex`

Se *num* for nulo e *exp* não for um número real positivo, este método retorna zero e define `errno` para `EDOM`.

Define **`errno`** para **`ERANGE`** em caso de estouros.

Descontinuado desde a versão 3.15.

double _Py_c_abs(Py_complex num)

Return the absolute value of the complex number *num*.

Define **`errno`** para **`ERANGE`** em caso de estouros.

Descontinuado desde a versão 3.15.

Objetos Bytes

Estas funções levantam `TypeError` quando se espera um parâmetro bytes e são chamados com um parâmetro que não é bytes.

Detalhes da implementação do CPython: The internal buffer of `PyBytesObject` always includes an extra trailing null byte for compatibility with null terminated C strings. This extra byte is not counted in `PyBytes_Size()` nor in the various *length* and *size* arguments of the functions below.

type PyBytesObject

Esta é uma instância de `PyObject` representando o objeto bytes do Python.

PyTypeObject PyBytes_Type

Parte da ABI Estável.

Esta instância de `PyTypeObject` representa o tipo de bytes Python; é o mesmo objeto que `bytes` na camada de Python.

int PyBytes_Check(PyObject *o)

Retorna verdadeiro se o objeto *o* for um objeto bytes ou se for uma instância de um subtipo do tipo bytes. Esta função sempre tem sucesso.

int PyBytes_CheckExact(PyObject *o)

Retorna verdadeiro se o objeto *o* for um objeto bytes, mas não uma instância de um subtipo do tipo bytes. Esta função sempre tem sucesso.

PyObject *PyBytes_FromString(const char *v)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo objeto de bytes com uma cópia da string *v* como valor em caso de sucesso e `NULL` em caso de falha. O parâmetro *v* não deve ser `NULL` e isso não será verificado.

PyObject *PyBytes_FromStringAndSize(const char *v, Py_ssize_t len)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo objeto de bytes com uma cópia da string *v* como valor e comprimento *len* em caso de sucesso e `NULL` em caso de falha. Se *v* for `NULL`, o conteúdo do objeto bytes não será inicializado.

Suavemente descontinuado desde a versão 3.15: Use the **`PyBytesWriter`** API instead of `PyBytes_FromStringAndSize(NULL, len)`.

PyObject *PyBytes_FromFormat(const char *format, ...)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Leva uma string tipo **`printf()`** do C *format* e um número variável de argumentos, calcula o tamanho do objeto bytes do Python resultante e retorna um objeto bytes com os valores formatados nela. Os argumentos da variável devem ser tipos C e devem corresponder exatamente aos caracteres de formato na string *format*. Os seguintes formatos de caracteres são permitidos:

Caracteres Formatados	Tipo	Comentário
`%%`	*n/d*	O caractere literal %.
`%c`	int	Um único byte, representado como um C int.
`%d`	int	Equivalente a `printf("%d")`. [[1]]
`%u`	unsigned int	Equivalente a `printf("%u")`. [[1]]
`%ld`	long	Equivalente a `printf("%ld")`. [[1]]

Caracteres Formatados	Tipo	Comentário
`%lu`	unsigned long	Equivalente a `printf("%lu")`. [[1]]
`%zd`	`Py_ssize_t`	Equivalente a `printf("%zd")`. [[1]]
`%zu`	size_t	Equivalente a `printf("%zu")`. [[1]]
`%i`	int	Equivalente a `printf("%i")`. [[1]]
`%x`	int	Equivalente a `printf("%x")`. [[1]]
`%s`	const char*	Uma matriz de caracteres C com terminação nula.
`%p`	const void*	A representação hexadecimal de um ponteiro C. Principalmente equivalente a `printf("%p")` exceto que é garantido que comece com o literal `0x` independentemente do que o `printf` da plataforma ceda.

Um caractere de formato não reconhecido faz com que todo o resto da string de formato seja copiado como é para o objeto resultante e todos os argumentos extras sejam descartados.

[1] (1,2,3,4,5,6,7,8)
Para especificadores de número inteiro (d, u, ld, lu, zd, zu, i, x): o sinalizador de conversão 0 tem efeito mesmo quando uma precisão é fornecida.

PyObject *PyBytes_FromFormatV(const char *format, va_list vargs)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Idêntico a **`PyBytes_FromFormat()`** exceto que é preciso exatamente dois argumentos.

PyObject *PyBytes_FromObject(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.
Retorna a representação de bytes do objeto *o* que implementa o protocolo de buffer.

Nota

If the object implements the buffer protocol, then the buffer must not be mutated while the bytes object is being created.

Py_ssize_t PyBytes_Size(PyObject *o)

Parte da ABI Estável. Thread safety: Atomic.
Retorna o comprimento dos bytes em objeto bytes *o*.

Py_ssize_t PyBytes_GET_SIZE(PyObject *o)

Thread safety: Atomic.
Similar a **`PyBytes_Size()`**, mas sem verificação de erro.

char *PyBytes_AsString(PyObject *o)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.
Retorna um ponteiro para o conteúdo de *o*. O ponteiro se refere ao buffer interno de *o*, que consiste em `len(o) + 1` bytes. O último byte no buffer é sempre nulo, independentemente de haver outros bytes nulos. Os dados não devem ser modificados de forma alguma, a menos que o objeto tenha sido criado usando `PyBytes_FromStringAndSize(NULL, size)`. Não deve ser desalocado. Se *o* não é um objeto de bytes, **`PyBytes_AsString()`** retorna `NULL` e levanta **`TypeError`**.

char *PyBytes_AS_STRING(PyObject *string)

Thread safety: Safe to call from multiple threads with external synchronization only.
Similar a **PyBytes_AsString()**, mas sem verificação de erro.

int PyBytes_AsStringAndSize(PyObject *obj, char **buffer, Py_ssize_t *length)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.
Retorna os conteúdos terminados nulos do objeto *obj* através das variáveis de saída *buffer* e *length*. Retorna `0` em caso de sucesso.

Se *length* for `NULL`, o objeto bytes não poderá conter bytes nulos incorporados; se isso acontecer, a função retornará `-1` e a **ValueError** será levantado.

O buffer refere-se a um buffer interno de *obj*, que inclui um byte nulo adicional no final (não contado em *length*). Os dados não devem ser modificados de forma alguma, a menos que o objeto tenha sido criado apenas usando `PyBytes_FromStringAndSize(NULL, size)`. Não deve ser desalinhado. Se *obj* não é um objeto bytes, **PyBytes_AsStringAndSize()** retorna `-1` e levanta **TypeError**.

Alterado na versão 3.5: Anteriormente **TypeError** era levantado quando os bytes nulos incorporados eram encontrados no objeto bytes.

void PyBytes_Concat(PyObject **bytes, PyObject *newpart)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.
Create a new bytes object in **bytes* containing the contents of *newpart* appended to *bytes*; the caller will own the new reference. The reference to the old value of *bytes* will be "stolen". If the new object cannot be created, the old reference to *bytes* will still be "stolen", the value of **bytes* will be set to `NULL`, and the appropriate exception will be set.

Nota

If *newpart* implements the buffer protocol, then the buffer must not be mutated while the new bytes object is being created.

void PyBytes_ConcatAndDel(PyObject **bytes, PyObject *newpart)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

"Crie um novo objeto bytes em **bytes* contendo o conteúdo de newpart anexado a bytes. Esta versão libera a strong reference (referência forte) para newpart (ou seja, decrementa a contagem de referências a ele)."

Nota

If *newpart* implements the buffer protocol, then the buffer must not be mutated while the new bytes object is being created.

PyObject *PyBytes_Join(PyObject *sep, PyObject *iterable)

Thread safety: Safe for concurrent use on the same object.

Similar a `sep.join(iterable)` no Python.

sep deve ser um objeto Python **`bytes`**. (Observe que **`PyUnicode_Join()`** aceita o separador `NULL` e o trata como um espaço, enquanto **`PyBytes_Join()`** não aceita o separador `NULL`.)

iterable deve ser um objeto iterável que produz objetos que implementam o protocolo de buffer.

Em caso de sucesso, retorna um novo objeto **`bytes`**. Em caso de erro, define uma exceção e retorna `NULL`.

Adicionado na versão 3.14.

Nota

If *iterable* objects implement the buffer protocol, then the buffers must not be mutated while the new bytes object is being created.

int _PyBytes_Resize(PyObject **bytes, Py_ssize_t newsize)

Thread safety: *Safe to call without external synchronization on distinct objects*.
Redimensiona um objeto de bytes. *newsize* será o novo tamanho do objeto bytes. Você pode pensar nisso como se estivesse criando um novo objeto de bytes e destruindo o antigo, só que de forma mais eficiente. Passe o endereço de um objeto de bytes existente como um lvalue (pode ser gravado) e o novo tamanho desejado. Em caso de sucesso, **bytes* mantém o objeto de bytes redimensionados e `0` é retornado; o endereço em **bytes* pode diferir do seu valor de entrada. Se a realocação falhar, o objeto de bytes originais em **bytes* é desalocado, **bytes* é definido como `NULL`, **`MemoryError`** é definido e `-1` é retornado.

Suavemente descontinuado desde a versão 3.15: Use the **`PyBytesWriter`** API instead.

PyObject *PyBytes_Repr(PyObject *bytes, int smartquotes)

Parte da ABI Estável. *Thread safety:* *Atomic*.
Obtém a representação em string de *bytes*. Esta função é atualmente usada para implementar **`bytes.__repr__()`** em Python.

Esta função não realiza verificação de tipo; é um comportamento indefinido passar *bytes* como um objeto que não seja do tipo bytes ou `NULL`.

Se *smartquotes* for verdadeiro, a representação usará uma string entre aspas duplas em vez de aspas simples quando aspas simples estiverem presentes em *bytes*. Por exemplo, a string de bytes `'Python'` seria representada como `b"'Python'"` quando *smartquotes* for verdadeiro, ou `b'\'Python\''` quando for falso.

Em caso de sucesso, esta função retorna uma referência forte a um objeto **`str`** contendo a representação. Em caso de falha, ela retorna `NULL` com uma exceção definida.

PyObject *PyBytes_DecodeEscape(const char *s, Py_ssize_t len, const char *errors, Py_ssize_t unicode, const char *recode_encoding)

Parte da ABI Estável. *Thread safety:* *Atomic*.

Remove um string *s* que foi escapada com contrabarra. *s* não pode ser `NULL`. *len* deve ser o tamanho de *s*.

errors deve ser um dos seguintes valores: `"strict"`, `"replace"` ou `"ignore"`. Se *errors* for `NULL`, o valor padrão será `"strict"`.

Em caso de sucesso, esta função retorna uma referência forte a um objeto Python **`bytes`** contendo a string sem escape. Em caso de falha, esta função retorna `NULL` com uma exceção definida.

Alterado na versão 3.9: unicode e *recode_encoding* não são mais usadas.

PyBytesWriter

The **`PyBytesWriter`** API can be used to create a Python **`bytes`** object.

Adicionado na versão 3.15.

type PyBytesWriter

A bytes writer instance.

The API is **not thread safe**: a writer should only be used by a single thread at the same time.

The instance must be destroyed by **`PyBytesWriter_Finish()`** on success, or **`PyBytesWriter_Discard()`** on error.

Create, Finish, Discard

PyBytesWriter *PyBytesWriter_Create(Py_ssize_t size)

Create a **`PyBytesWriter`** to write *size* bytes.

If *size* is greater than zero, allocate *size* bytes, and set the writer size to *size*. The caller is responsible to write *size* bytes using **`PyBytesWriter_GetData()`**. This function does not overallocate.

On error, set an exception and return `NULL`.

size must be positive or zero.

PyObject *PyBytesWriter_Finish(PyBytesWriter *writer)

Finish a **`PyBytesWriter`** created by **`PyBytesWriter_Create()`**.

On success, return a Python **`bytes`** object. On error, set an exception and return `NULL`.

The writer instance is invalid after the call in any case. No API can be called on the writer after **`PyBytesWriter_Finish()`**.

PyObject *PyBytesWriter_FinishWithSize(PyBytesWriter *writer, Py_ssize_t size)

Similar to **`PyBytesWriter_Finish()`**, but resize the writer to *size* bytes before creating the **`bytes`** object.

PyObject *PyBytesWriter_FinishWithPointer(PyBytesWriter *writer, void *buf)

Similar to **`PyBytesWriter_Finish()`**, but resize the writer using *buf* pointer before creating the **`bytes`** object.

Set an exception and return `NULL` if *buf* pointer is outside the internal buffer bounds.

Function pseudo-code:

```
Py_ssize_t size = (char*)buf - (char*)PyBytesWriter_GetData(w
return PyBytesWriter_FinishWithSize(writer, size);
```

void PyBytesWriter_Discard(PyBytesWriter *writer)

Discard a **`PyBytesWriter`** created by **`PyBytesWriter_Create()`**.

Do nothing if *writer* is `NULL`.

The writer instance is invalid after the call. No API can be called on the writer after `PyBytesWriter_Discard()`.

High-level API

int PyBytesWriter_WriteBytes(PyBytesWriter *writer, const void *bytes, Py_ssize_t size)

Grow the *writer* internal buffer by *size* bytes, write *size* bytes of *bytes* at the *writer* end, and add *size* to the *writer* size.

If *size* is equal to `-1`, call `strlen(bytes)` to get the string length.

On success, return `0`. On error, set an exception and return `-1`.

int PyBytesWriter_Format(PyBytesWriter *writer, const char *format, ...)

Similar to `PyBytes_FromFormat()`, but write the output directly at the writer end. Grow the writer internal buffer on demand. Then add the written size to the writer size.

On success, return `0`. On error, set an exception and return `-1`.

Getters

Py_ssize_t PyBytesWriter_GetSize(PyBytesWriter *writer)

Get the writer size.

The function does not invalidate pointers returned by `PyBytesWriter_GetData()`.

The function cannot fail.

void *PyBytesWriter_GetData(PyBytesWriter *writer)

Get the writer data: start of the internal buffer.

The pointer remains valid until a `PyBytesWriter` function other than `PyBytesWriter_GetData()` or `PyBytesWriter_GetSize()` is called on

writer.

The function cannot fail.

Low-level API

int PyBytesWriter_Resize(PyBytesWriter *writer, Py_ssize_t size)

Resize the writer to *size* bytes. It can be used to enlarge or to shrink the writer. This function typically overallocates to achieve amortized performance when resizing multiple times.

Newly allocated bytes are left uninitialized.

On success, return `0`. On error, set an exception and return `-1`.

size must be positive or zero.

int PyBytesWriter_Grow(PyBytesWriter *writer, Py_ssize_t grow)

Resize the writer by adding *grow* bytes to the current writer size. This function typically overallocates to achieve amortized performance when resizing multiple times.

Newly allocated bytes are left uninitialized.

On success, return `0`. On error, set an exception and return `-1`.

size can be negative to shrink the writer.

void *PyBytesWriter_GrowAndUpdatePointer(PyBytesWriter *writer, Py_ssize_t size, void *buf)

Similar to **`PyBytesWriter_Grow()`**, but update also the *buf* pointer.

The *buf* pointer is moved if the internal buffer is moved in memory. The *buf* relative position within the internal buffer is left unchanged.

On error, set an exception and return `NULL`.

buf must not be `NULL`.

Function pseudo-code:

```
Py_ssize_t pos = (char*)buf - (char*)PyBytesWriter_GetData(wr
if (PyBytesWriter_Grow(writer, size) < 0) {
    return NULL;
}
return (char*)PyBytesWriter_GetData(writer) + pos;
```

Objetos byte array

type PyByteArrayObject

Esse subtipo de `PyObject` representa um objeto Python bytearray.

O buffer interno de `PyByteArrayObject` sempre inclui um byte nulo adicional ao final para compatibilidade com strings C terminadas em nulo. Esse byte extra não é contabilizado em `PyByteArray_Size()` nem nos argumentos *len* das funções abaixo.

PyTypeObject PyByteArray_Type

Parte da ABI Estável.

Essa instância de `PyTypeObject` representa um tipo Python bytearray; é o mesmo objeto que o `bytearray` na camada Python.

Macros para verificação de tipo

int PyByteArray_Check(PyObject *o)

Retorna verdadeiro se o objeto *o* for um objeto bytearray ou se for uma instância de um subtipo do tipo bytearray. Esta função sempre tem sucesso.

int PyByteArray_CheckExact(PyObject *o)

Retorna verdadeiro se o objeto *o* for um objeto bytearray, mas não uma instância de um subtipo do tipo bytearray. Esta função sempre tem sucesso.

Funções diretas da API

PyObject *PyByteArray_FromObject(PyObject *o)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Retorna um novo objeto bytearray, *o*, que implementa o protocolo de buffer.

Em caso de falha, retorna `NULL` com uma exceção definida.

Nota

Se o objeto implementar o protocolo de buffer, o buffer não deve ser modificado enquanto o objeto bytearray estiver a ser criado.

PyObject *PyByteArray_FromStringAndSize(const char *string, Py_ssize_t len)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Cria um novo objeto bytearray a partir de *string* e seu comprimento, *len*.

Em caso de falha, retorna `NULL` com uma exceção definida.

PyObject *PyByteArray_Concat(PyObject *a, PyObject *b)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Concatena os bytearrays *a* e *b* e retorna um novo bytearray com o resultado.

Em caso de falha, retorna `NULL` com uma exceção definida.

Nota

Se o objeto implementar o protocolo de buffer, o buffer não deve ser modificado enquanto o objeto bytearray estiver a ser criado.

Py_ssize_t PyByteArray_Size(PyObject *bytearray)

Parte da ABI Estável. Thread safety: Atomic.

Retorna o tamanho de *bytearray* após verificar se há um ponteiro `NULL`.

char *PyByteArray_AsString(PyObject *bytearray)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.
Retorna o conteúdo de *bytearray* como uma matriz de caracteres após verificar um ponteiro `NULL`. O array retornado sempre tem um byte nulo extra acrescentado.

Nota

Não é seguro para thread modificar o objeto bytearray enquanto se utiliza o vetor de caracteres retornado.

int PyByteArray_Resize(PyObject *bytearray, Py_ssize_t len)

Parte da ABI Estável.
Redimensiona o buffer interno de *bytearray* para o tamanho *len*. Falha é um retorno `-1` com um conjunto de exceções.

Alterado na versão 3.14: Um *len* negativo agora resultará na definição de uma exceção e no retorno de -1.

Macros

Estas macros trocam segurança por velocidade e não verificam os ponteiros.

char *PyByteArray_AS_STRING(PyObject *bytearray)

Thread safety: Safe to call from multiple threads with external synchronization only.
Similar a **`PyByteArray_AsString()`**, mas sem verificação de erro.

Nota

Não é seguro para thread modificar o objeto bytearray enquanto se utiliza o vetor de caracteres retornado.

Py_ssize_t PyByteArray_GET_SIZE(PyObject *bytearray)

Thread safety: Atomic.

Similar a `PyByteArray_Size()`, mas sem verificação de erro.

Objetos Unicode e Codecs

Objetos Unicode

Desde a implementação da **PEP 393** [https://peps.python.org/pep-0393/] no Python 3.3, os objetos Unicode usam internamente uma variedade de representações para permitir o processamento de toda a gama de caracteres Unicode, mantendo a eficiência de memória. Há casos especiais para strings em que todos os pontos de código estão abaixo de 128, 256 ou 65536; caso contrário, os pontos de código devem estar abaixo de 1114112 (que é a gama completa de caracteres Unicode).

A representação UTF-8 é criada sob demanda e armazenada em cache no objeto Unicode.

Nota

A representação `Py_UNICODE` foi removida desde o Python 3.12 com APIs descontinuadas. Consulte **PEP 623** [https://peps.python.org/pep-0623/] para obter mais informações.

Tipo Unicode

Estes são os tipos básicos de objetos Unicode usados para a implementação Unicode em Python:

PyTypeObject PyUnicode_Type

Parte da ABI Estável.

This instance of `PyTypeObject` represents the Python Unicode type. It is exposed to Python code as `str`.

PyTypeObject PyUnicodeIter_Type

Parte da ABI Estável.

Esta instância de `PyTypeObject` representa o tipo iterador Unicode do Python. É usada para iterar sobre objetos string Unicode.

type Py_UCS4
type Py_UCS2
type Py_UCS1

Parte da ABI Estável.
Esses tipos são definições de tipo para tipos inteiros sem sinal, amplos o suficiente para conter caracteres de 32 bits, 16 bits e 8 bits, respectivamente. Ao lidar com caracteres Unicode simples, use `Py_UCS4`.

Adicionado na versão 3.3.

type PyASCIIObject
type PyCompactUnicodeObject
type PyUnicodeObject

Esses subtipos de `PyObject` representam um objeto Unicode do Python. Em quase todos os casos, eles não devem ser usados diretamente, pois todas as funções da API que lidam com objetos Unicode recebem e retornam ponteiros `PyObject`.

Adicionado na versão 3.3.

The structure of a particular object can be determined using the following macros. The macros cannot fail; their behavior is undefined if their argument is not a Python Unicode object.

PyUnicode_IS_COMPACT(o)

True if *o* uses the `PyCompactUnicodeObject` structure.

Adicionado na versão 3.3.

PyUnicode_IS_COMPACT_ASCII(o)

True if *o* uses the `PyASCIIObject` structure.

Adicionado na versão 3.3.

As seguintes APIs são macros C e funções estáticas embutidas para verificações rápidas e acesso a dados internos somente leitura de objetos Unicode:

int PyUnicode_Check(PyObject *obj)

Retorna verdadeiro se o objeto *obj* for um objeto Unicode ou uma instância de um subtipo Unicode. Esta função sempre tem sucesso.

int PyUnicode_CheckExact(PyObject *obj)

Retorna verdadeiro se o objeto *obj* for um objeto Unicode, mas não uma instância de um subtipo. Esta função sempre tem sucesso.

Py_ssize_t PyUnicode_GET_LENGTH(PyObject *unicode)

Retorna o comprimento da string Unicode, em pontos de código. *unicode* deve ser um objeto Unicode na representação "canônica" (não marcado).

Adicionado na versão 3.3.

Py_UCS1 *PyUnicode_1BYTE_DATA(PyObject *unicode)
Py_UCS2 *PyUnicode_2BYTE_DATA(PyObject *unicode)
Py_UCS4 *PyUnicode_4BYTE_DATA(PyObject *unicode)

Retorna um ponteiro para a representação canônica convertida para tipos inteiros UCS1, UCS2 ou UCS4 para acesso direto aos caracteres. Nenhuma verificação é realizada se a representação canônica tem o tamanho de caractere correto; use `PyUnicode_KIND()` para selecionar a função correta.

Adicionado na versão 3.3.

PyUnicode_1BYTE_KIND
PyUnicode_2BYTE_KIND
PyUnicode_4BYTE_KIND

Valores de retorno da macro `PyUnicode_KIND()`.

Adicionado na versão 3.3.

Alterado na versão 3.12: `PyUnicode_WCHAR_KIND` foi removida.

int PyUnicode_KIND(PyObject *unicode)

Retorna uma das constantes do tipo PyUnicode (veja acima) que indicam quantos bytes por caractere este objeto Unicode usa para armazenar seus dados. *unicode* precisa ser um objeto Unicode na representação “canônica” (não marcado).

Adicionado na versão 3.3.

void *PyUnicode_DATA(PyObject *unicode)

Retorna um ponteiro vazio para o buffer Unicode bruto. *unicode* deve ser um objeto Unicode na representação “canônica” (não marcado).

Adicionado na versão 3.3.

void PyUnicode_WRITE(int kind, void *data, Py_ssize_t index, Py_UCS4 value)

Write the code point *value* to the given zero-based *index* in a string.

The *kind* value and *data* pointer must have been obtained from a string using **PyUnicode_KIND()** and **PyUnicode_DATA()** respectively. You must hold a reference to that string while calling **PyUnicode_WRITE()**. All requirements of **PyUnicode_WriteChar()** also apply.

The function performs no checks for any of its requirements, and is intended for usage in loops.

Adicionado na versão 3.3.

Py_UCS4 PyUnicode_READ(int kind, void *data, Py_ssize_t index)

Lê um ponto de código de uma representação canônica *data* (conforme obtido com **PyUnicode_DATA()**). Nenhuma verificação ou chamada pronta é realizada.

Adicionado na versão 3.3.

Py_UCS4 PyUnicode_READ_CHAR(PyObject *unicode, Py_ssize_t index)

Lê um caractere de um objeto Unicode *unicode*, que deve estar na representação “canônica”. Isso é menos eficiente do que **PyUnicode_READ()** se você fizer várias leituras consecutivas.

Adicionado na versão 3.3.

Py_UCS4 PyUnicode_MAX_CHAR_VALUE(PyObject *unicode)

Retorna o ponto de código máximo adequado para criar outra string baseada em *unicode*, que deve estar na representação “canônica”. Isso é sempre uma aproximação, mas é mais eficiente do que iterar sobre a string.

Adicionado na versão 3.3.

int PyUnicode_IsIdentifier(PyObject *unicode)

Parte da ABI Estável.

Retorna `1` se a string é um identificador válido conforme a definição da linguagem, seção Nomes (identificadores e palavras reservadas). Do contrário, retorna `0`.

Alterado na versão 3.9: A função não chama mais **Py_FatalError()** se a string não estiver pronta.

unsigned int PyUnicode_IS_ASCII(PyObject *unicode)

Return true if the string only contains ASCII characters. Equivalent to **str.isascii()**.

Adicionado na versão 3.2.

Py_hash_t PyUnstable_Unicode_GET_CACHED_HASH(PyObject *str)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

If the hash of *str*, as returned by `PyObject_Hash()`, has been cached and is immediately available, return it. Otherwise, return `-1` *without* setting an exception.

If *str* is not a string (that is, if `PyUnicode_Check(obj)` is false), the behavior is undefined.

This function never fails with an exception.

Note that there are no guarantees on when an object's hash is cached, and the (non-)existence of a cached hash does not imply that the string has any other properties.

Propriedade de caracteres Unicode

O Unicode fornece muitas propriedades de caracteres diferentes. As mais frequentemente necessárias estão disponíveis por meio destas macros, que são mapeadas para funções C, dependendo da configuração do Python.

int Py_UNICODE_ISSPACE(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere de espaço em branco.

int Py_UNICODE_ISLOWER(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere minúsculo.

int Py_UNICODE_ISUPPER(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere maiúsculo.

int Py_UNICODE_ISTITLE(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere em TitleCase.

int Py_UNICODE_ISLINEBREAK(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere de quebra de linha.

int Py_UNICODE_ISDECIMAL(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere decimal.

int Py_UNICODE_ISDIGIT(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere de dígito.

int Py_UNICODE_ISNUMERIC(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere numérico.

int Py_UNICODE_ISALPHA(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere alfabético.

int Py_UNICODE_ISALNUM(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere alfanumérico.

int Py_UNICODE_ISPRINTABLE(Py_UCS4 ch)

Retorna `1` ou `0` dependendo se *ch* é um caractere imprimível, no sentido de **`str.isprintable()`**.

These APIs can be used for fast direct character conversions:

Py_UCS4 Py_UNICODE_TOLOWER(Py_UCS4 ch)

Return the character *ch* converted to lower case.

Py_UCS4 Py_UNICODE_TOUPPER(Py_UCS4 ch)

Return the character *ch* converted to upper case.

Py_UCS4 Py_UNICODE_TOTITLE(Py_UCS4 ch)

Return the character *ch* converted to title case.

int Py_UNICODE_TODECIMAL(Py_UCS4 ch)

Return the character *ch* converted to a decimal positive integer. Return `-1` if this is not possible. This function does not raise exceptions.

int Py_UNICODE_TODIGIT(Py_UCS4 ch)

Return the character *ch* converted to a single digit integer. Return `-1` if this is not possible. This function does not raise exceptions.

double Py_UNICODE_TONUMERIC(Py_UCS4 ch)

Return the character *ch* converted to a double. Return `-1.0` if this is not possible. This function does not raise exceptions.

These APIs can be used to work with surrogates:

int Py_UNICODE_IS_SURROGATE(Py_UCS4 ch)

Check if *ch* is a surrogate (`0xD800 <= ch <= 0xDFFF`).

int Py_UNICODE_IS_HIGH_SURROGATE(Py_UCS4 ch)

Check if *ch* is a high surrogate (`0xD800 <= ch <= 0xDBFF`).

int Py_UNICODE_IS_LOW_SURROGATE(Py_UCS4 ch)

Check if *ch* is a low surrogate (`0xDC00 <= ch <= 0xDFFF`).

Py_UCS4 Py_UNICODE_HIGH_SURROGATE(Py_UCS4 ch)

Return the high UTF-16 surrogate (`0xD800` to `0xDBFF`) for a Unicode code point in the range `[0x10000; 0x10FFFF]`.

Py_UCS4 Py_UNICODE_LOW_SURROGATE(Py_UCS4 ch)

Return the low UTF-16 surrogate (`0xDC00` to `0xDFFF`) for a Unicode code point in the range `[0x10000; 0x10FFFF]`.

Py_UCS4 Py_UNICODE_JOIN_SURROGATES(Py_UCS4 high, Py_UCS4 low)

Join two surrogate code points and return a single **`Py_UCS4`** value. *high* and *low* are respectively the leading and trailing surrogates in a surrogate pair. *high* must be in the range `[0xD800; 0xDBFF]` and *low* must be in the range `[0xDC00; 0xDFFF]`.

Creating and accessing Unicode strings

To create Unicode objects and access their basic sequence properties, use these APIs:

PyObject *PyUnicode_New(Py_ssize_t size, Py_UCS4 maxchar)

Retorna valor: Nova referência.
Create a new Unicode object. *maxchar* should be the true maximum code point to be placed in the string. As an approximation, it can be rounded up to the nearest value in the sequence 127, 255, 65535, 1114111.

On error, set an exception and return `NULL`.

After creation, the string can be filled by **`PyUnicode_WriteChar()`**, **`PyUnicode_CopyCharacters()`**, **`PyUnicode_Fill()`**, **`PyUnicode_WRITE()`** or similar. Since strings are supposed to be immutable, take care to not "use" the result while it is being modified. In particular, before it's filled with its final contents, a string:

- must not be hashed,
- must not be **`converted to UTF-8`**, or another non-"canonical" representation,
- must not have its reference count changed,
- must not be shared with code that might do one of the above.

This list is not exhaustive. Avoiding these uses is your responsibility; Python does not always check these requirements.

To avoid accidentally exposing a partially-written string object, prefer using the **`PyUnicodeWriter`** API, or one of the `PyUnicode_From*` functions below.

Adicionado na versão 3.3.

PyObject *PyUnicode_FromKindAndData(int kind, const void *buffer, Py_ssize_t size)

Retorna valor: Nova referência.
Cria um novo objeto Unicode com o tipo *type* fornecido (os valores possíveis são **`PyUnicode_1BYTE_KIND`** etc., conforme retornado por **`PyUnicode_KIND()`**). O *buffer* deve apontar para um array de unidades com *size* de 1, 2 ou 4 bytes por caractere, conforme fornecido pelo tipo.

If necessary, the input *buffer* is copied and transformed into the canonical representation. For example, if the *buffer* is a UCS4 string (`PyUnicode_4BYTE_KIND`) and it consists only of codepoints in the UCS1 range, it will be transformed into UCS1 (`PyUnicode_1BYTE_KIND`).

Adicionado na versão 3.3.

PyObject *PyUnicode_FromStringAndSize(const char *str, Py_ssize_t size)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object from the char buffer *str*. The bytes will be interpreted as being UTF-8 encoded. The buffer is copied into the new object. The return value might be a shared object, i.e. modification of the data is not allowed.

This function raises `SystemError` when:

- *size* < 0,
- *str* is `NULL` and *size* > 0

Alterado na versão 3.12: str == `NULL` with *size* > 0 is not allowed anymore.

PyObject *PyUnicode_FromString(const char *str)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object from a UTF-8 encoded null-terminated char buffer *str*.

PyObject *PyUnicode_FromFormat(const char *format, ...)

Retorna valor: Nova referência. Parte da ABI Estável.
Take a C `printf()`-style *format* string and a variable number of arguments, calculate the size of the resulting Python Unicode string and return a string with the values formatted into it. The variable arguments must be C types and must correspond exactly to the format characters in the *format* ASCII-encoded string.

Um especificador de conversão contém dois ou mais caracteres e tem os seguintes componentes, que devem aparecer nesta ordem:

1. O caractere `'%'`, que determina o início do especificador.
2. Flags de conversão (opcional), que afetam o resultado de alguns tipos de conversão.
3. Minimum field width (optional). If specified as an `'*'` (asterisk), the actual width is given in the next argument, which must be of type int, and the object to convert comes after the minimum field width and optional precision.
4. Precision (optional), given as a `'.'` (dot) followed by the precision. If specified as `'*'` (an asterisk), the actual precision is given in the next argument, which must be of type int, and the value to convert comes after the precision.
5. Modificador de comprimento (opcional).
6. Tipo de conversão.

Os caracteres flags de conversão são:

Sinalizador	Significado
`0`	A conversão será preenchida por zeros para valores numéricos.
`-`	The converted value is left adjusted (overrides the `0` flag if both are given).

The length modifiers for following integer conversions (`d`, `i`, `o`, `u`, `x`, or `X`) specify the type of the argument (int by default):

Modifier	Tipos
`l`	long or unsigned long
`ll`	long long or unsigned long long
`j`	**`intmax_t`** or **`uintmax_t`**

Modifier	Tipos
`z`	**`size_t`** or **`ssize_t`**
`t`	**`ptrdiff_t`**

The length modifier `l` for following conversions `s` or `v` specify that the type of the argument is const wchar_t*.

The conversion specifiers are:

Conversion Specifier	Tipo	Comentário
`%`	*n/d*	The literal `%` character.
`d`, `i`	Specified by the length modifier	The decimal representation of a signed C integer.
`u`	Specified by the length modifier	The decimal representation of an unsigned C integer.
`o`	Specified by the length modifier	The octal representation of an unsigned C integer.
`x`	Specified by the length modifier	The hexadecimal representation of an unsigned C integer (lowercase).
`X`	Specified by the length modifier	The hexadecimal representation of an unsigned C integer (uppercase).

Conversion Specifier	Tipo	Comentário
`c`	int	A single character.
`s`	const char* or const wchar_t*	Uma matriz de caracteres C com terminação nula.
`p`	const void*	The hex representation of a C pointer. Mostly equivalent to `printf("%p")` except that it is guaranteed to start with the literal `0x` regardless of what the platform's `printf` yields.
`A`	PyObject*	The result of calling `ascii()`.
`U`	PyObject*	A Unicode object.
`V`	PyObject*, const char* or const wchar_t*	A Unicode object (which may be `NULL`) and a null-terminated C character array as a second parameter (which will be used, if the first parameter is `NULL`).
`S`	PyObject*	The result of calling `PyObject_Str()`.
`R`	PyObject*	The result of calling `PyObject_Repr()`.
`T`	PyObject*	Get the fully qualified name of an object type; call `PyType_GetFullyQualifiedName()`.
`#T`	PyObject*	Similar to `T` format, but use a colon (`:`) as separator between the module name and the qualified name.

Conversion Specifier	Tipo	Comentário
`N`	PyTypeObject*	Get the fully qualified name of a type; call **`PyType_GetFullyQualifiedName()`**.
`#N`	PyTypeObject*	Similar to `N` format, but use a colon (`:`) as separator between the module name and the qualified name.

Nota

The width formatter unit is number of characters rather than bytes. The precision formatter unit is number of bytes or **`wchar_t`** items (if the length modifier `l` is used) for `"%s"` and `"%V"` (if the `PyObject*` argument is `NULL`), and a number of characters for `"%A"`, `"%U"`, `"%S"`, `"%R"` and `"%V"` (if the `PyObject*` argument is not `NULL`).

Nota

Unlike to C **`printf()`** the `0` flag has effect even when a precision is given for integer conversions (`d`, `i`, `u`, `o`, `x`, or `X`).

Alterado na versão 3.2: Suporte adicionado para `"%lld"` e `"%llu"`.

Alterado na versão 3.3: Support for `"%li"`, `"%lli"` and `"%zi"` added.

Alterado na versão 3.4: Support width and precision formatter for `"%s"`, `"%A"`, `"%U"`, `"%V"`, `"%S"`, `"%R"` added.

Alterado na versão 3.12: Support for conversion specifiers `o` and `X`. Support for length modifiers `j` and `t`. Length modifiers are now applied to all integer conversions. Length modifier `l` is now applied to conversion specifiers `s` and `V`. Support for variable width and precision `*`. Support for flag `-`.

An unrecognized format character now sets a **SystemError**. In previous versions it caused all the rest of the format string to be copied as-is to the result string, and any extra arguments discarded.

Alterado na versão 3.13: Support for `%T`, `%#T`, `%N` and `%#N` formats added.

PyObject *PyUnicode_FromFormatV(const char *format, va_list vargs)

Retorna valor: Nova referência. Parte da ABI Estável.
Identical to **PyUnicode_FromFormat()** except that it takes exactly two arguments.

PyObject *PyUnicode_FromObject(PyObject *obj)

Retorna valor: Nova referência. Parte da ABI Estável.
Copy an instance of a Unicode subtype to a new true Unicode object if necessary. If *obj* is already a true Unicode object (not a subtype), return a new strong reference to the object.

Objects other than Unicode or its subtypes will cause a **TypeError**.

PyObject *PyUnicode_FromOrdinal(int ordinal)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode Object from the given Unicode code point *ordinal*.

The ordinal must be in `range(0x110000)`. A **ValueError** is raised in the case it is not.

PyObject *PyUnicode_FromEncodedObject(PyObject *obj, const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Decode an encoded object *obj* to a Unicode object.

bytes, **bytearray** and other bytes-like objects are decoded according to the given *encoding* and using the error handling defined by *errors*. Both can be `NULL` to have the interface use the default values (see Built-in Codecs for details).

All other objects, including Unicode objects, cause a **`TypeError`** to be set.

The API returns `NULL` if there was an error. The caller is responsible for decref'ing the returned objects.

void PyUnicode_Append(PyObject **p_left, PyObject *right)

Parte da ABI Estável.
Append the string *right* to the end of *p_left*. *p_left* must point to a strong reference to a Unicode object; **`PyUnicode_Append()`** releases ("steals") this reference.

On error, set **p_left* to `NULL` and set an exception.

On success, set **p_left* to a new strong reference to the result.

void PyUnicode_AppendAndDel(PyObject **p_left, PyObject *right)

Parte da ABI Estável.
The function is similar to **`PyUnicode_Append()`**, with the only difference being that it decrements the reference count of *right* by one.

PyObject *PyUnicode_BuildEncodingMap(PyObject *string)

Retorna valor: Nova referência. Parte da ABI Estável.
Return a mapping suitable for decoding a custom single-byte encoding. Given a Unicode string *string* of up to 256 characters representing an encoding table, returns either a compact internal mapping object or a dictionary mapping character ordinals to byte values. Raises a **`TypeError`** and return `NULL` on invalid input.

Adicionado na versão 3.2.

const char *PyUnicode_GetDefaultEncoding(void)

Parte da ABI Estável.
Return the name of the default string encoding, `"utf-8"`. See **`sys.getdefaultencoding()`**.

The returned string does not need to be freed, and is valid until interpreter shutdown.

Py_ssize_t PyUnicode_GetLength(PyObject *unicode)

Parte da ABI Estável desde a versão 3.7.
Return the length of the Unicode object, in code points.

On error, set an exception and return `-1`.

Adicionado na versão 3.3.

Py_ssize_t PyUnicode_CopyCharacters(PyObject *to, Py_ssize_t to_start, PyObject *from, Py_ssize_t from_start, Py_ssize_t how_many)

Copy characters from one Unicode object into another. This function performs character conversion when necessary and falls back to **`memcpy()`** if possible. Returns `-1` and sets an exception on error, otherwise returns the number of copied characters.

The string must not have been "used" yet. See **`PyUnicode_New()`** for details.

Adicionado na versão 3.3.

int PyUnicode_Resize(PyObject **unicode, Py_ssize_t length);

Parte da ABI Estável.
Resize a Unicode object **unicode* to the new *length* in code points.

Try to resize the string in place (which is usually faster than allocating a new string and copying characters), or create a new string.

**unicode* is modified to point to the new (resized) object and `0` is returned on success. Otherwise, `-1` is returned and an exception is set, and **unicode* is left untouched.

The function doesn't check string content, the result may not be a string in canonical representation.

Py_ssize_t PyUnicode_Fill(PyObject *unicode, Py_ssize_t start, Py_ssize_t length, Py_UCS4 fill_char)

Fill a string with a character: write *fill_char* into `unicode[start:start+length]`.

Fail if *fill_char* is bigger than the string maximum character, or if the string has more than 1 reference.

The string must not have been "used" yet. See **`PyUnicode_New()`** for details.

Return the number of written characters, or return `-1` and raise an exception on error.

Adicionado na versão 3.3.

int PyUnicode_WriteChar(PyObject *unicode, Py_ssize_t index, Py_UCS4 character)

Parte da ABI Estável desde a versão 3.7.
Write a *character* to the string *unicode* at the zero-based *index*. Return `0` on success, `-1` on error with an exception set.

This function checks that *unicode* is a Unicode object, that the index is not out of bounds, and that the object's reference count is one. See **`PyUnicode_WRITE()`** for a version that skips these checks, making them your responsibility.

The string must not have been "used" yet. See **`PyUnicode_New()`** for details.

Adicionado na versão 3.3.

Py_UCS4 PyUnicode_ReadChar(PyObject *unicode, Py_ssize_t index)

Parte da ABI Estável desde a versão 3.7.
Read a character from a string. This function checks that *unicode* is a Unicode object and the index is not out of bounds, in contrast to **`PyUnicode_READ_CHAR()`**, which performs no error checking.

Return character on success, `-1` on error with an exception set.

Adicionado na versão 3.3.

PyObject *PyUnicode_Substring(PyObject *unicode, Py_ssize_t start, Py_ssize_t end)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Return a substring of *unicode*, from character index *start* (included) to character index *end* (excluded). Negative indices are not supported. On error, set an exception and return `NULL`.

Adicionado na versão 3.3.

Py_UCS4 *PyUnicode_AsUCS4(PyObject *unicode, Py_UCS4 *buffer, Py_ssize_t buflen, int copy_null)

Parte da ABI Estável desde a versão 3.7.

Copy the string *unicode* into a UCS4 buffer, including a null character, if *copy_null* is set. Returns `NULL` and sets an exception on error (in particular, a **`SystemError`** if *buflen* is smaller than the length of *unicode*). *buffer* is returned on success.

Adicionado na versão 3.3.

Py_UCS4 *PyUnicode_AsUCS4Copy(PyObject *unicode)

Parte da ABI Estável desde a versão 3.7.

Copy the string *unicode* into a new UCS4 buffer that is allocated using **`PyMem_Malloc()`**. If this fails, `NULL` is returned with a **`MemoryError`** set. The returned buffer always has an extra null code point appended.

Adicionado na versão 3.3.

Locale Encoding

The current locale encoding can be used to decode text from the operating system.

PyObject *PyUnicode_DecodeLocaleAndSize(const char *str, Py_ssize_t length, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Decode a string from UTF-8 on Android and VxWorks, or from the current locale encoding on other platforms. The supported error handlers are `"strict"` and `"surrogateescape"` (**PEP 383** [https://peps.python.org/pep-0383/]). The decoder uses `"strict"` error handler if *errors* is `NULL`. *str* must end with a null character but cannot contain embedded null characters.

Use **`PyUnicode_DecodeFSDefaultAndSize()`** to decode a string from the filesystem encoding and error handler.

This function ignores the Python UTF-8 Mode.

Ver também

The **`Py_DecodeLocale()`** function.

Adicionado na versão 3.3.

Alterado na versão 3.7: The function now also uses the current locale encoding for the `surrogateescape` error handler, except on Android. Previously, **`Py_DecodeLocale()`** was used for the `surrogateescape`, and the current locale encoding was used for `strict`.

PyObject *PyUnicode_DecodeLocale(const char *str, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Similar to **`PyUnicode_DecodeLocaleAndSize()`**, but compute the string length using **`strlen()`**.

Adicionado na versão 3.3.

PyObject *PyUnicode_EncodeLocale(PyObject *unicode, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Encode a Unicode object to UTF-8 on Android and VxWorks, or to the current locale encoding on other platforms. The supported error handlers are `"strict"` and `"surrogateescape"` (**PEP 383** [https://peps.python.org/pep-0383/]). The encoder uses `"strict"` error handler if *errors* is `NULL`. Return a `bytes` object. *unicode* cannot contain embedded null characters.

Use `PyUnicode_EncodeFSDefault()` to encode a string to the filesystem encoding and error handler.

This function ignores the Python UTF-8 Mode.

Ver também

The `Py_EncodeLocale()` function.

Adicionado na versão 3.3.

Alterado na versão 3.7: The function now also uses the current locale encoding for the `surrogateescape` error handler, except on Android. Previously, `Py_EncodeLocale()` was used for the `surrogateescape`, and the current locale encoding was used for `strict`.

File System Encoding

Functions encoding to and decoding from the filesystem encoding and error handler (**PEP 383** [https://peps.python.org/pep-0383/] and **PEP 529** [https://peps.python.org/pep-0529/]).

To encode file names to `bytes` during argument parsing, the `"O&"` converter should be used, passing `PyUnicode_FSConverter()` as the conversion function:

int PyUnicode_FSConverter(PyObject *obj, void *result)

Parte da ABI Estável.

PyArg_Parse* converter: encode **str** objects – obtained directly or through the **os.PathLike** interface – to **bytes** using **PyUnicode_EncodeFSDefault()**; **bytes** objects are output as-is. *result* must be an address of a C variable of type PyObject* (or PyBytesObject*). On success, set the variable to a new strong reference to a bytes object which must be released when it is no longer used and return a non-zero value (**Py_CLEANUP_SUPPORTED**). Embedded null bytes are not allowed in the result. On failure, return `0` with an exception set.

If *obj* is `NULL`, the function releases a strong reference stored in the variable referred by *result* and returns `1`.

Adicionado na versão 3.1.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

To decode file names to **str** during argument parsing, the `"O&"` converter should be used, passing **PyUnicode_FSDecoder()** as the conversion function:

int PyUnicode_FSDecoder(PyObject *obj, void *result)

Parte da ABI Estável.

PyArg_Parse* converter: decode **bytes** objects – obtained either directly or indirectly through the **os.PathLike** interface – to **str** using **PyUnicode_DecodeFSDefaultAndSize()**; **str** objects are output as-is. *result* must be an address of a C variable of type PyObject* (or PyUnicodeObject*). On success, set the variable to a new strong reference to a Unicode object which must be released when it is no longer used and return a non-zero value (**Py_CLEANUP_SUPPORTED**). Embedded null characters are not allowed in the result. On failure, return `0` with an exception set.

If *obj* is `NULL`, release the strong reference to the object referred to by *result* and return `1`.

Adicionado na versão 3.2.

Alterado na versão 3.6: Aceita um objeto caminho ou similar.

PyObject *PyUnicode_DecodeFSDefaultAndSize(const char *str, Py_ssize_t size)

Retorna valor: Nova referência. Parte da ABI Estável.
Decode a string from the filesystem encoding and error handler.

If you need to decode a string from the current locale encoding, use **PyUnicode_DecodeLocaleAndSize()**.

Ver também

The **Py_DecodeLocale()** function.

Alterado na versão 3.6: The filesystem error handler is now used.

PyObject *PyUnicode_DecodeFSDefault(const char *str)

Retorna valor: Nova referência. Parte da ABI Estável.
Decode a null-terminated string from the filesystem encoding and error handler.

If the string length is known, use **PyUnicode_DecodeFSDefaultAndSize()**.

Alterado na versão 3.6: The filesystem error handler is now used.

PyObject *PyUnicode_EncodeFSDefault(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object to the filesystem encoding and error handler, and return **bytes**. Note that the resulting **bytes** object can contain null bytes.

If you need to encode a string to the current locale encoding, use **PyUnicode_EncodeLocale()**.

Ver também

The **Py_EncodeLocale()** function.

Adicionado na versão 3.2.

Alterado na versão 3.6: The filesystem error handler is now used.

wchar_t Support

`wchar_t` support for platforms which support it:

PyObject *PyUnicode_FromWideChar(const wchar_t *wstr, Py_ssize_t size)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object from the **`wchar_t`** buffer *wstr* of the given *size*. Passing `-1` as the *size* indicates that the function must itself compute the length, using **`wcslen()`**. Return `NULL` on failure.

Py_ssize_t PyUnicode_AsWideChar(PyObject *unicode, wchar_t *wstr, Py_ssize_t size)

Parte da ABI Estável.
Copy the Unicode object contents into the **`wchar_t`** buffer *wstr*. At most *size* **`wchar_t`** characters are copied (excluding a possibly trailing null termination character). Return the number of **`wchar_t`** characters copied or `-1` in case of an error.

When *wstr* is `NULL`, instead return the *size* that would be required to store all of *unicode* including a terminating null.

Note that the resulting wchar_t* string may or may not be null-terminated. It is the responsibility of the caller to make sure that the wchar_t* string is null-terminated in case this is required by the application. Also, note that the wchar_t* string might contain null characters, which would cause the string to be truncated when used with most C functions.

wchar_t *PyUnicode_AsWideCharString(PyObject *unicode, Py_ssize_t *size)

Parte da ABI Estável desde a versão 3.7.

Convert the Unicode object to a wide character string. The output string always ends with a null character. If *size* is not `NULL`, write the number of wide characters (excluding the trailing null termination character) into **size*. Note that the resulting **`wchar_t`** string might contain null characters, which would cause the string to be truncated when used with most C functions. If *size* is `NULL` and the wchar_t* string contains null characters a **`ValueError`** is raised.

Returns a buffer allocated by **`PyMem_New`** (use **`PyMem_Free()`** to free it) on success. On error, returns `NULL` and **size* is undefined. Raises a **`MemoryError`** if memory allocation is failed.

Adicionado na versão 3.2.

Alterado na versão 3.7: Raises a **`ValueError`** if *size* is `NULL` and the wchar_t* string contains null characters.

Built-in Codecs

Python provides a set of built-in codecs which are written in C for speed. All of these codecs are directly usable via the following functions.

Many of the following APIs take two arguments encoding and errors, and they have the same semantics as the ones of the built-in **`str()`** string object constructor.

Setting encoding to `NULL` causes the default encoding to be used which is UTF-8. The file system calls should use **`PyUnicode_FSConverter()`** for encoding file names. This uses the filesystem encoding and error handler internally.

Error handling is set by errors which may also be set to `NULL` meaning to use the default handling defined for the codec. Default error handling for all built-in codecs is “strict” (**`ValueError`** is raised).

The codecs all use a similar interface. Only deviations from the following generic ones are documented for simplicity.

Generic Codecs

The following macro is provided:

Py_UNICODE_REPLACEMENT_CHARACTER

The Unicode code point `U+FFFD` (replacement character).

This Unicode character is used as the replacement character during decoding if the *errors* argument is set to “replace”.

These are the generic codec APIs:

PyObject *PyUnicode_Decode(const char *str, Py_ssize_t size, const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.

Create a Unicode object by decoding *size* bytes of the encoded string *str*. *encoding* and *errors* have the same meaning as the parameters of the same name in the **`str()`** built-in function. The codec to be used is looked up using the Python codec registry. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_AsEncodedString(PyObject *unicode, const char *encoding, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.

Encode a Unicode object and return the result as Python bytes object. *encoding* and *errors* have the same meaning as the parameters of the same name in the Unicode **`encode()`** method. The codec to be used is looked up using the Python codec registry. Return `NULL` if an exception was raised by the codec.

UTF-8 Codecs

These are the UTF-8 codec APIs:

PyObject *PyUnicode_DecodeUTF8(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object by decoding *size* bytes of the UTF-8 encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_DecodeUTF8Stateful(const char *str, Py_ssize_t size, const char *errors, Py_ssize_t *consumed)

Retorna valor: Nova referência. Parte da ABI Estável.
If *consumed* is `NULL`, behave like **`PyUnicode_DecodeUTF8()`**. If *consumed* is not `NULL`, trailing incomplete UTF-8 byte sequences will not be treated as an error. Those bytes will not be decoded and the number of bytes that have been decoded will be stored in *consumed*.

PyObject *PyUnicode_AsUTF8String(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object using UTF-8 and return the result as Python bytes object. Error handling is "strict". Return `NULL` if an exception was raised by the codec.

The function fails if the string contains surrogate code points (`U+D800` - `U+DFFF`).

const char *PyUnicode_AsUTF8AndSize(PyObject *unicode, Py_ssize_t *size)

Parte da ABI Estável desde a versão 3.10.
Return a pointer to the UTF-8 encoding of the Unicode object, and store the size of the encoded representation (in bytes) in *size*. The *size* argument can be `NULL`; in this case no size will be stored. The returned buffer always has an extra null byte appended (not included in *size*), regardless of whether there are any other null code points.

On error, set an exception, set *size* to `-1` (if it's not NULL) and return `NULL`.

The function fails if the string contains surrogate code points (`U+D800` - `U+DFFF`).

This caches the UTF-8 representation of the string in the Unicode object, and subsequent calls will return a pointer to the same buffer. The caller is not responsible for deallocating the buffer. The buffer is deallocated and pointers to it become invalid when the Unicode object is garbage collected.

Adicionado na versão 3.3.

Alterado na versão 3.7: The return type is now `const char *` rather than `char *`.

Alterado na versão 3.10: This function is a part of the limited API.

const char *PyUnicode_AsUTF8(PyObject *unicode)

As **`PyUnicode_AsUTF8AndSize()`**, but does not store the size.

Aviso

This function does not have any special behavior for null characters [https://en.wikipedia.org/wiki/Null_character] embedded within *unicode*. As a result, strings containing null characters will remain in the returned string, which some C functions might interpret as the end of the string, leading to truncation. If truncation is an issue, it is recommended to use **`PyUnicode_AsUTF8AndSize()`** instead.

Adicionado na versão 3.3.

Alterado na versão 3.7: The return type is now `const char *` rather than `char *`.

UTF-32 Codecs

These are the UTF-32 codec APIs:

PyObject *PyUnicode_DecodeUTF32(const char *str, Py_ssize_t size, const char *errors, int *byteorder)

Retorna valor: Nova referência. Parte da ABI Estável.

Decode *size* bytes from a UTF-32 encoded buffer string and return the corresponding Unicode object. *errors* (if non-`NULL`) defines the error handling. It defaults to “strict”.

If *byteorder* is non-`NULL`, the decoder starts decoding using the given byte order:

```
*byteorder == -1: little endian
*byteorder == 0:  native order
*byteorder == 1:  big endian
```

If `*byteorder` is zero, and the first four bytes of the input data are a byte order mark (BOM), the decoder switches to this byte order and the BOM is not copied into the resulting Unicode string. If `*byteorder` is `-1` or `1`, any byte order mark is copied to the output.

After completion, **byteorder* is set to the current byte order at the end of input data.

If *byteorder* is `NULL`, the codec starts in native order mode.

Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_DecodeUTF32Stateful(const char *str, Py_ssize_t size, const char *errors, int *byteorder, Py_ssize_t *consumed)

Retorna valor: Nova referência. Parte da ABI Estável.
If *consumed* is `NULL`, behave like **`PyUnicode_DecodeUTF32()`**. If *consumed* is not `NULL`, **`PyUnicode_DecodeUTF32Stateful()`** will not treat trailing incomplete UTF-32 byte sequences (such as a number of bytes not divisible by four) as an error. Those bytes will not be decoded and the number of bytes that have been decoded will be stored in *consumed*.

PyObject *PyUnicode_AsUTF32String(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Return a Python byte string using the UTF-32 encoding in native byte order. The string always starts with a BOM mark. Error handling is “strict”. Return `NULL` if an exception was raised by the codec.

UTF-16 Codecs

These are the UTF-16 codec APIs:

PyObject *PyUnicode_DecodeUTF16(const char *str, Py_ssize_t size, const char *errors, int *byteorder)

Retorna valor: Nova referência. Parte da ABI Estável.

Decode *size* bytes from a UTF-16 encoded buffer string and return the corresponding Unicode object. *errors* (if non-`NULL`) defines the error handling. It defaults to "strict".

If *byteorder* is non-`NULL`, the decoder starts decoding using the given byte order:

```
*byteorder == -1: little endian
*byteorder == 0:  native order
*byteorder == 1:  big endian
```

If `*byteorder` is zero, and the first two bytes of the input data are a byte order mark (BOM), the decoder switches to this byte order and the BOM is not copied into the resulting Unicode string. If `*byteorder` is `-1` or `1`, any byte order mark is copied to the output (where it will result in either a `\ufeff` or a `\ufffe` character).

After completion, `*byteorder` is set to the current byte order at the end of input data.

If *byteorder* is `NULL`, the codec starts in native order mode.

Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_DecodeUTF16Stateful(const char *str, Py_ssize_t size, const char *errors, int *byteorder, Py_ssize_t *consumed)

Retorna valor: Nova referência. Parte da ABI Estável.

If *consumed* is `NULL`, behave like **`PyUnicode_DecodeUTF16()`**. If *consumed* is not `NULL`, **`PyUnicode_DecodeUTF16Stateful()`** will not treat trailing incomplete UTF-16 byte sequences (such as an odd number of bytes or a split surrogate pair) as an error. Those bytes will

not be decoded and the number of bytes that have been decoded will be stored in *consumed*.

PyObject *PyUnicode_AsUTF16String(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Return a Python byte string using the UTF-16 encoding in native byte order. The string always starts with a BOM mark. Error handling is "strict". Return `NULL` if an exception was raised by the codec.

UTF-7 Codecs

These are the UTF-7 codec APIs:

PyObject *PyUnicode_DecodeUTF7(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object by decoding *size* bytes of the UTF-7 encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_DecodeUTF7Stateful(const char *str, Py_ssize_t size, const char *errors, Py_ssize_t *consumed)

Retorna valor: Nova referência. Parte da ABI Estável.
If *consumed* is `NULL`, behave like **`PyUnicode_DecodeUTF7()`**. If *consumed* is not `NULL`, trailing incomplete UTF-7 base-64 sections will not be treated as an error. Those bytes will not be decoded and the number of bytes that have been decoded will be stored in *consumed*.

Unicode-Escape Codecs

These are the "Unicode Escape" codec APIs:

PyObject *PyUnicode_DecodeUnicodeEscape(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object by decoding *size* bytes of the Unicode-Escape encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_AsUnicodeEscapeString(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object using Unicode-Escape and return the result as a bytes object. Error handling is “strict”. Return `NULL` if an exception was raised by the codec.

Raw-Unicode-Escape Codecs

These are the “Raw Unicode Escape” codec APIs:

PyObject *PyUnicode_DecodeRawUnicodeEscape(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object by decoding *size* bytes of the Raw-Unicode-Escape encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_AsRawUnicodeEscapeString(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object using Raw-Unicode-Escape and return the result as a bytes object. Error handling is “strict”. Return `NULL` if an exception was raised by the codec.

Latin-1 Codecs

These are the Latin-1 codec APIs: Latin-1 corresponds to the first 256 Unicode ordinals and only these are accepted by the codecs during encoding.

PyObject *PyUnicode_DecodeLatin1(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object by decoding *size* bytes of the Latin-1 encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_AsLatin1String(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object using Latin-1 and return the result as Python bytes object. Error handling is “strict”. Return `NULL` if an exception was raised by the codec.

ASCII Codecs

These are the ASCII codec APIs. Only 7-bit ASCII data is accepted. All other codes generate errors.

PyObject *PyUnicode_DecodeASCII(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a Unicode object by decoding *size* bytes of the ASCII encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_AsASCIIString(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object using ASCII and return the result as Python bytes object. Error handling is “strict”. Return `NULL` if an exception was raised by the codec.

Character Map Codecs

This codec is special in that it can be used to implement many different codecs (and this is in fact what was done to obtain most of the standard codecs included in the **`encodings`** package). The codec uses mappings to encode and decode characters. The mapping objects provided must support the **`__getitem__()`** mapping interface; dictionaries and sequences work well.

These are the mapping codec APIs:

PyObject *PyUnicode_DecodeCharmap(const char *str, Py_ssize_t length, PyObject *mapping, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.

Create a Unicode object by decoding *size* bytes of the encoded string *str* using the given *mapping* object. Return `NULL` if an exception was raised by the codec.

If *mapping* is `NULL`, Latin-1 decoding will be applied. Else *mapping* must map bytes ordinals (integers in the range from 0 to 255) to Unicode strings, integers (which are then interpreted as Unicode ordinals) or `None`. Unmapped data bytes – ones which cause a **`LookupError`**, as well as ones which get mapped to `None`, `0xFFFE` or `'\ufffe'`, are treated as undefined mappings and cause an error.

PyObject *PyUnicode_AsCharmapString(PyObject *unicode, PyObject *mapping)

Retorna valor: Nova referência. Parte da ABI Estável.
Encode a Unicode object using the given *mapping* object and return the result as a bytes object. Error handling is "strict". Return `NULL` if an exception was raised by the codec.

The *mapping* object must map Unicode ordinal integers to bytes objects, integers in the range from 0 to 255 or `None`. Unmapped character ordinals (ones which cause a **`LookupError`**) as well as mapped to `None` are treated as "undefined mapping" and cause an error.

The following codec API is special in that maps Unicode to Unicode.

PyObject *PyUnicode_Translate(PyObject *unicode, PyObject *table, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável.
Translate a string by applying a character mapping table to it and return the resulting Unicode object. Return `NULL` if an exception was raised by the codec.

The mapping table must map Unicode ordinal integers to Unicode ordinal integers or `None` (causing deletion of the character).

Mapping tables need only provide the **`__getitem__()`** interface; dictionaries and sequences work well. Unmapped character ordinals

(ones which cause a `LookupError`) are left untouched and are copied as-is.

errors has the usual meaning for codecs. It may be `NULL` which indicates to use the default error handling.

MBCS codecs for Windows

These are the MBCS codec APIs. They are currently only available on Windows and use the Win32 MBCS converters to implement the conversions. Note that MBCS (or DBCS) is a class of encodings, not just one. The target encoding is defined by the user settings on the machine running the codec.

PyObject *PyUnicode_DecodeMBCS(const char *str, Py_ssize_t size, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável on Windows desde a versão 3.7.

Create a Unicode object by decoding *size* bytes of the MBCS encoded string *str*. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_DecodeMBCSStateful(const char *str, Py_ssize_t size, const char *errors, Py_ssize_t *consumed)

Retorna valor: Nova referência. Parte da ABI Estável on Windows desde a versão 3.7.

If *consumed* is `NULL`, behave like `PyUnicode_DecodeMBCS()`. If *consumed* is not `NULL`, `PyUnicode_DecodeMBCSStateful()` will not decode trailing lead byte and the number of bytes that have been decoded will be stored in *consumed*.

PyObject *PyUnicode_DecodeCodePageStateful(int code_page, const char *str, Py_ssize_t size, const char *errors, Py_ssize_t *consumed)

Retorna valor: Nova referência. Parte da ABI Estável on Windows desde a versão 3.7.

Similar to `PyUnicode_DecodeMBCSStateful()`, except uses the code page specified by *code_page*.

PyObject *PyUnicode_AsMBCSString(PyObject *unicode)

Retorna valor: Nova referência. Parte da ABI Estável on Windows desde a versão 3.7.

Encode a Unicode object using MBCS and return the result as Python bytes object. Error handling is “strict”. Return `NULL` if an exception was raised by the codec.

PyObject *PyUnicode_EncodeCodePage(int code_page, PyObject *unicode, const char *errors)

Retorna valor: Nova referência. Parte da ABI Estável on Windows desde a versão 3.7.

Encode the Unicode object using the specified code page and return a Python bytes object. Return `NULL` if an exception was raised by the codec. Use **`CP_ACP`** code page to get the MBCS encoder.

Adicionado na versão 3.3.

Methods and Slot Functions

The following APIs are capable of handling Unicode objects and strings on input (we refer to them as strings in the descriptions) and return Unicode objects or integers as appropriate.

They all return `NULL` or `-1` if an exception occurs.

PyObject *PyUnicode_Concat(PyObject *left, PyObject *right)

Retorna valor: Nova referência. Parte da ABI Estável.

Concat two strings giving a new Unicode string.

PyObject *PyUnicode_Split(PyObject *unicode, PyObject *sep, Py_ssize_t maxsplit)

Retorna valor: Nova referência. Parte da ABI Estável.

Split a string giving a list of Unicode strings. If *sep* is `NULL`, splitting will be done at all whitespace substrings. Otherwise, splits occur at the given separator. At most *maxsplit* splits will be done. If negative, no limit is set. Separators are not included in the resulting list.

On error, return `NULL` with an exception set.

Equivalent to **`str.split()`**.

PyObject *PyUnicode_RSplit(PyObject *unicode, PyObject *sep, Py_ssize_t maxsplit)

Retorna valor: Nova referência. Parte da ABI Estável.
Similar to **`PyUnicode_Split()`**, but splitting will be done beginning at the end of the string.

On error, return `NULL` with an exception set.

Equivalent to **`str.rsplit()`**.

PyObject *PyUnicode_Splitlines(PyObject *unicode, int keepends)

Retorna valor: Nova referência. Parte da ABI Estável.
Split a Unicode string at line breaks, returning a list of Unicode strings. CRLF is considered to be one line break. If *keepends* is `0`, the Line break characters are not included in the resulting strings.

PyObject *PyUnicode_Partition(PyObject *unicode, PyObject *sep)

Retorna valor: Nova referência. Parte da ABI Estável.
Split a Unicode string at the first occurrence of *sep*, and return a 3-tuple containing the part before the separator, the separator itself, and the part after the separator. If the separator is not found, return a 3-tuple containing the string itself, followed by two empty strings.

sep must not be empty.

On error, return `NULL` with an exception set.

Equivalent to **`str.partition()`**.

PyObject *PyUnicode_RPartition(PyObject *unicode, PyObject *sep)

Retorna valor: Nova referência. Parte da ABI Estável.
Similar to **`PyUnicode_Partition()`**, but split a Unicode string at the last occurrence of *sep*. If the separator is not found, return a 3-tuple

containing two empty strings, followed by the string itself.

sep must not be empty.

On error, return `NULL` with an exception set.

Equivalent to **`str.rpartition()`**.

PyObject *PyUnicode_Join(PyObject *separator, PyObject *seq)

Retorna valor: Nova referência. Parte da ABI Estável.
Join a sequence of strings using the given *separator* and return the resulting Unicode string.

Py_ssize_t PyUnicode_Tailmatch(PyObject *unicode, PyObject *substr, Py_ssize_t start, Py_ssize_t end, int direction)

Parte da ABI Estável.
Return `1` if *substr* matches `unicode[start:end]` at the given tail end (*direction* == `-1` means to do a prefix match, *direction* == `1` a suffix match), `0` otherwise. Return `-1` if an error occurred.

Py_ssize_t PyUnicode_Find(PyObject *unicode, PyObject *substr, Py_ssize_t start, Py_ssize_t end, int direction)

Parte da ABI Estável.
Return the first position of *substr* in `unicode[start:end]` using the given *direction* (*direction* == `1` means to do a forward search, *direction* == `-1` a backward search). The return value is the index of the first match; a value of `-1` indicates that no match was found, and `-2` indicates that an error occurred and an exception has been set.

Py_ssize_t PyUnicode_FindChar(PyObject *unicode, Py_UCS4 ch, Py_ssize_t start, Py_ssize_t end, int direction)

Parte da ABI Estável desde a versão 3.7.
Return the first position of the character *ch* in `unicode[start:end]` using the given *direction* (*direction* == `1` means to do a forward search, *direction* == `-1` a backward search). The return value is the index of the

first match; a value of `-1` indicates that no match was found, and `-2` indicates that an error occurred and an exception has been set.

Adicionado na versão 3.3.

Alterado na versão 3.7: start and *end* are now adjusted to behave like `unicode[start:end]`.

Py_ssize_t PyUnicode_Count(PyObject *unicode, PyObject *substr, Py_ssize_t start, Py_ssize_t end)

Parte da ABI Estável.
Return the number of non-overlapping occurrences of *substr* in `unicode[start:end]`. Return `-1` if an error occurred.

PyObject *PyUnicode_Replace(PyObject *unicode, PyObject *substr, PyObject *replstr, Py_ssize_t maxcount)

Retorna valor: Nova referência. Parte da ABI Estável.
Replace at most *maxcount* occurrences of *substr* in *unicode* with *replstr* and return the resulting Unicode object. *maxcount* == `-1` means replace all occurrences.

int PyUnicode_Compare(PyObject *left, PyObject *right)

Parte da ABI Estável.
Compare two strings and return `-1`, `0`, `1` for less than, equal, and greater than, respectively.

This function returns `-1` upon failure, so one should call **`PyErr_Occurred()`** to check for errors.

Ver também

The **`PyUnicode_Equal()`** function.

int PyUnicode_Equal(PyObject *a, PyObject *b)

Parte da ABI Estável desde a versão 3.14.
Test if two strings are equal:

- Return `1` if *a* is equal to *b*.
- Return `0` if *a* is not equal to *b*.
- Set a `TypeError` exception and return `-1` if *a* or *b* is not a `str` object.

The function always succeeds if *a* and *b* are `str` objects.

The function works for `str` subclasses, but does not honor custom `__eq__()` method.

Ver também

The `PyUnicode_Compare()` function.

Adicionado na versão 3.14.

int PyUnicode_EqualToUTF8AndSize(PyObject *unicode, const char *string, Py_ssize_t size)

Parte da ABI Estável desde a versão 3.13.

Compare a Unicode object with a char buffer which is interpreted as being UTF-8 or ASCII encoded and return true (`1`) if they are equal, or false (`0`) otherwise. If the Unicode object contains surrogate code points (`U+D800` - `U+DFFF`) or the C string is not valid UTF-8, false (`0`) is returned.

This function does not raise exceptions.

Adicionado na versão 3.13.

int PyUnicode_EqualToUTF8(PyObject *unicode, const char *string)

Parte da ABI Estável desde a versão 3.13.

Similar to `PyUnicode_EqualToUTF8AndSize()`, but compute *string* length using `strlen()`. If the Unicode object contains null characters, false (`0`) is returned.

Adicionado na versão 3.13.

int PyUnicode_CompareWithASCIIString(PyObject *unicode, const char *string)

Parte da ABI Estável.

Compare a Unicode object, *unicode*, with *string* and return `-1`, `0`, `1` for less than, equal, and greater than, respectively. It is best to pass only ASCII-encoded strings, but the function interprets the input string as ISO-8859-1 if it contains non-ASCII characters.

This function does not raise exceptions.

PyObject *PyUnicode_RichCompare(PyObject *left, PyObject *right, int op)

Retorna valor: Nova referência. Parte da ABI Estável.

Rich compare two Unicode strings and return one of the following:

- `NULL` in case an exception was raised
- **`Py_True`** or **`Py_False`** for successful comparisons
- **`Py_NotImplemented`** in case the type combination is unknown

Possible values for *op* are **`Py_GT`**, **`Py_GE`**, **`Py_EQ`**, **`Py_NE`**, **`Py_LT`**, and **`Py_LE`**.

PyObject *PyUnicode_Format(PyObject *format, PyObject *args)

Retorna valor: Nova referência. Parte da ABI Estável.

Return a new string object from *format* and *args*; this is analogous to `format % args`.

int PyUnicode_Contains(PyObject *unicode, PyObject *substr)

Parte da ABI Estável.

Check whether *substr* is contained in *unicode* and return true or false accordingly.

substr has to coerce to a one element Unicode string. `-1` is returned if there was an error.

void PyUnicode_InternInPlace(PyObject **p_unicode)

Parte da ABI Estável.

Intern the argument *p_unicode in place. The argument must be the address of a pointer variable pointing to a Python Unicode string object. If there is an existing interned string that is the same as *p_unicode, it sets *p_unicode to it (releasing the reference to the old string object and creating a new strong reference to the interned string object), otherwise it leaves *p_unicode alone and interns it.

(Clarification: even though there is a lot of talk about references, think of this function as reference-neutral. You must own the object you pass in; after the call you no longer own the passed-in reference, but you newly own the result.)

This function never raises an exception. On error, it leaves its argument unchanged without interning it.

Instances of subclasses of `str` may not be interned, that is, PyUnicode_CheckExact(*p_unicode) must be true. If it is not, then – as with any other error – the argument is left unchanged.

Note that interned strings are not "immortal". You must keep a reference to the result to benefit from interning.

PyObject *PyUnicode_InternFromString(const char *str)

Retorna valor: Nova referência. Parte da ABI Estável.

A combination of `PyUnicode_FromString()` and `PyUnicode_InternInPlace()`, meant for statically allocated strings.

Return a new ("owned") reference to either a new Unicode string object that has been interned, or an earlier interned string object with the same value.

Python may keep a reference to the result, or make it immortal, preventing it from being garbage-collected promptly. For interning an unbounded number of different strings, such as ones coming from user input, prefer calling `PyUnicode_FromString()` and `PyUnicode_InternInPlace()` directly.

unsigned int PyUnicode_CHECK_INTERNED(PyObject *str)

Return a non-zero value if *str* is interned, zero if not. The *str* argument must be a string; this is not checked. This function always succeeds.

Detalhes da implementação do CPython: A non-zero return value may carry additional information about *how* the string is interned. The meaning of such non-zero values, as well as each specific string's intern-related details, may change between CPython versions.

PyUnicodeWriter

The `PyUnicodeWriter` API can be used to create a Python `str` object.

Adicionado na versão 3.14.

type PyUnicodeWriter

A Unicode writer instance.

The instance must be destroyed by `PyUnicodeWriter_Finish()` on success, or `PyUnicodeWriter_Discard()` on error.

PyUnicodeWriter *PyUnicodeWriter_Create(Py_ssize_t length)

Create a Unicode writer instance.

length must be greater than or equal to `0`.

If *length* is greater than `0`, preallocate an internal buffer of *length* characters.

Define uma exceção e retorna `NULL` em caso de erro.

PyObject *PyUnicodeWriter_Finish(PyUnicodeWriter *writer)

Return the final Python `str` object and destroy the writer instance.

Define uma exceção e retorna `NULL` em caso de erro.

The writer instance is invalid after this call.

void PyUnicodeWriter_Discard(PyUnicodeWriter *writer)

Discard the internal Unicode buffer and destroy the writer instance.

If *writer* is `NULL`, no operation is performed.

The writer instance is invalid after this call.

int PyUnicodeWriter_WriteChar(PyUnicodeWriter *writer, Py_UCS4 ch)

Write the single Unicode character *ch* into *writer*.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

int PyUnicodeWriter_WriteUTF8(PyUnicodeWriter *writer, const char *str, Py_ssize_t size)

Decode the string *str* from UTF-8 in strict mode and write the output into *writer*.

size is the string length in bytes. If *size* is equal to `-1`, call `strlen(str)` to get the string length.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

See also **`PyUnicodeWriter_DecodeUTF8Stateful()`**.

int PyUnicodeWriter_WriteASCII(PyUnicodeWriter *writer, const char *str, Py_ssize_t size)

Write the ASCII string *str* into *writer*.

size is the string length in bytes. If *size* is equal to `-1`, call `strlen(str)` to get the string length.

str must only contain ASCII characters. The behavior is undefined if *str* contains non-ASCII characters.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

int PyUnicodeWriter_WriteWideChar(PyUnicodeWriter *writer, const wchar_t *str, Py_ssize_t size)

Write the wide string *str* into *writer*.

size is a number of wide characters. If *size* is equal to `-1`, call `wcslen(str)` to get the string length.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

int PyUnicodeWriter_WriteUCS4(PyUnicodeWriter *writer, const Py_UCS4 *str, Py_ssize_t size)

Writer the UCS4 string *str* into *writer*.

size is a number of UCS4 characters.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

int PyUnicodeWriter_WriteStr(PyUnicodeWriter *writer, PyObject *obj)

Call **`PyObject_Str()`** on *obj* and write the output into *writer*.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

To write a **`str`** subclass which overrides the **`__str__()`** method, **`PyUnicode_FromObject()`** can be used to get the original string.

int PyUnicodeWriter_WriteRepr(PyUnicodeWriter *writer, PyObject *obj)

Call **`PyObject_Repr()`** on *obj* and write the output into *writer*.

If *obj* is `NULL`, write the string `"<NULL>"` into *writer*.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

Alterado na versão 3.14.4: Added support for `NULL`.

int PyUnicodeWriter_WriteSubstring(PyUnicodeWriter *writer, PyObject *str, Py_ssize_t start, Py_ssize_t end)

Write the substring `str[start:end]` into *writer*.

str must be Python **`str`** object. *start* must be greater than or equal to 0, and less than or equal to *end*. *end* must be less than or equal to *str* length.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

int PyUnicodeWriter_Format(PyUnicodeWriter *writer, const char *format, ...)

Similar to **`PyUnicode_FromFormat()`**, but write the output directly into *writer*.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

int PyUnicodeWriter_DecodeUTF8Stateful(PyUnicodeWriter *writer, const char *string, Py_ssize_t length, const char *errors, Py_ssize_t *consumed)

Decode the string *str* from UTF-8 with *errors* error handler and write the output into *writer*.

size is the string length in bytes. If *size* is equal to `-1`, call `strlen(str)` to get the string length.

errors is an error handler name, such as `"replace"`. If *errors* is `NULL`, use the strict error handler.

If *consumed* is not `NULL`, set **consumed* to the number of decoded bytes on success. If *consumed* is `NULL`, treat trailing incomplete UTF-8 byte

sequences as an error.

On success, return `0`. On error, set an exception, leave the writer unchanged, and return `-1`.

See also **`PyUnicodeWriter_WriteUTF8()`**.

Deprecated API

The following API is deprecated.

type Py_UNICODE

This is a typedef of **`wchar_t`**, which is a 16-bit type or 32-bit type depending on the platform. Please use **`wchar_t`** directly instead.

Alterado na versão 3.3: Em versões anteriores, esse era um tipo de 16 bits ou de 32 bits, dependendo se você selecionava uma versão Unicode "estreita" ou "ampla" do Python no momento da construção.

Descontinuado desde a versão 3.13, removido na versão 3.15.

int PyUnicode_READY(PyObject *unicode)

Do nothing and return `0`. This API is kept only for backward compatibility, but there are no plans to remove it.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.10: This API does nothing since Python 3.12. Previously, this needed to be called for each string created using the old API (**`PyUnicode_FromUnicode()`** or similar).

unsigned int PyUnicode_IS_READY(PyObject *unicode)

Do nothing and return `1`. This API is kept only for backward compatibility, but there are no plans to remove it.

Adicionado na versão 3.3.

Descontinuado desde a versão 3.14: This API does nothing since Python 3.12. Previously, this could be called to check if `PyUnicode_READY()` is necessary.

Objeto tupla

type PyTupleObject

Este subtipo de **`PyObject`** representa um objeto tupla em Python.

PyTypeObject PyTuple_Type

Parte da ABI Estável.

Esta instância de **`PyTypeObject`** representa o tipo tupla de Python; é o mesmo objeto que **`tuple`** na camada Python.

int PyTuple_Check(PyObject *p)

Retorna verdadeiro se *p* é um objeto tupla ou uma instância de um subtipo do tipo tupla. Esta função sempre tem sucesso.

int PyTuple_CheckExact(PyObject *p)

Retorna verdadeiro se *p* é um objeto tupla, mas não uma instância de um subtipo do tipo tupla. Esta função sempre tem sucesso.

PyObject *PyTuple_New(Py_ssize_t len)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo objeto tupla de tamanho *len*, ou `NULL` com uma exceção definida em caso de falha.

PyObject *PyTuple_FromArray(PyObject *const *array, Py_ssize_t size)

Thread safety: Atomic.

Create a tuple of *size* items and copy references from *array* to the new tuple.

array can be NULL if *size* is `0`.

On success, return a new reference. On error, set an exception and return `NULL`.

Adicionado na versão 3.15.

PyObject *PyTuple_Pack(Py_ssize_t n, ...)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo objeto tupla de tamanho *n*, ou `NULL` com uma exceção em caso de falha. Os valores da tupla são inicializados para os *n* argumentos C subsequentes apontando para objetos Python. `PyTuple_Pack(2, a, b)` é equivalente a `Py_BuildValue("(OO)", a, b)`.

Py_ssize_t PyTuple_Size(PyObject *p)

Parte da ABI Estável. Thread safety: Atomic.

Pega um ponteiro para um objeto tupla e retorna o tamanho dessa tupla. No erro, retorna `-1` com uma exceção definida.

Py_ssize_t PyTuple_GET_SIZE(PyObject *p)

Thread safety: Atomic.

Como **`PyTuple_Size()`**, mas sem verificação de erro.

PyObject *PyTuple_GetItem(PyObject *p, Py_ssize_t pos)

Retorna valor: Referência emprestada. Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

Retorna o objeto na posição *pos* na tupla apontada por *p*. Se *pos* estiver fora dos limites, retorna `NULL` e define uma exceção **`IndexError`**.

A referência retornada é emprestada da tupla *p* (ou seja: ela só é válida enquanto você mantém uma referência a *p*). Para obter uma referência forte, use **`Py_NewRef(PyTuple_GetItem(...))`** ou **`PySequence_GetItem()`**.

PyObject *PyTuple_GET_ITEM(PyObject *p, Py_ssize_t pos)

Retorna valor: Referência emprestada. Thread safety: Safe to call from multiple threads with external synchronization only.

Como **PyTuple_GetItem()**, mas faz nenhuma verificação de seus argumentos.

PyObject *PyTuple_GetSlice(PyObject *p, Py_ssize_t low, Py_ssize_t high)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna a fatia da tupla apontada por *p* entre *low* e *high*, ou `NULL` com uma exceção definida em caso de falha.

Isso é o equivalente da expressão Python `p[low:high]`. A indexação do final da tupla não é suportada.

int PyTuple_SetItem(PyObject *p, Py_ssize_t pos, PyObject *o)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

Insert a reference to object *o* at position *pos* of the tuple pointed to by *p*. Return `0` on success. If *pos* is out of bounds, return `-1` and set an **IndexError** exception. This function should only be used to fill in brand new tuples; using it on an existing tuple is thread-unsafe.

Nota

This function "steals" a reference to *o* and discards a reference to an item already in the tuple at the affected position (unless it was NULL).

void PyTuple_SET_ITEM(PyObject *p, Py_ssize_t pos, PyObject *o)

Thread safety: Safe to call from multiple threads with external synchronization only.

Like **PyTuple_SetItem()**, but does no error checking, and should *only* be used to fill in brand new tuples, using it on an existing tuple is thread-unsafe.

A verificação de limites é realizada como uma asserção se o Python for construído em modo de depuração ou **com asserções**.

Nota

This function "steals" a reference to *o*, and, unlike `PyTuple_SetItem()`, does *not* discard a reference to any item that is being replaced; any reference in the tuple at position *pos* will be leaked.

Aviso

Esta macro deve ser usada *somente* em tuplas recém-criadas. Usar esta macro em uma tupla que já está em uso (ou, em outras palavras, com uma contagem de referências > 1) pode levar a um comportamento indefinido.

int _PyTuple_Resize(PyObject **p, Py_ssize_t newsize)

Thread safety: Safe to call from multiple threads with external synchronization only.

Pode ser usado para redimensionar uma tupla. *newsize* será o novo comprimento da tupla. Como as tuplas são *supostamente* imutáveis, isso só deve ser usado se houver apenas uma referência ao objeto. *Não* use isto se a tupla já for conhecida por alguma outra parte do código. A tupla sempre aumentará ou diminuirá no final. Pense nisso como destruir a tupla antiga e criar uma nova, mas com mais eficiência. Retorna `0` em caso de sucesso. O código do cliente nunca deve presumir que o valor resultante de `*p` será o mesmo de antes de chamar esta função. Se o objeto referenciado por `*p` for substituído, o `*p` original será destruído. Em caso de falha, retorna `-1` e define `*p` para `NULL`, e levanta `MemoryError` ou `SystemError`.

Objetos sequência de estrutura

A struct sequence object is a named tuple, that is, a sequence whose items can also be accessed through attributes. It is similar to `collections.namedtuple()`, but provides a slightly different interface.

To create a struct sequence, you first have to create a specific struct sequence type.

PyTypeObject *PyStructSequence_NewType(PyStructSequence_Desc *desc)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Cria um novo tipo de sequência de estrutura a partir dos dados em *desc*, descrito abaixo. Instâncias do tipo resultante podem ser criadas com **`PyStructSequence_New()`**.

Retorna `NULL` com uma exceção definida em caso de falha.

void PyStructSequence_InitType(PyTypeObject *type, PyStructSequence_Desc *desc)

Thread safety: Safe to call without external synchronization on distinct objects.

Inicializa um tipo de sequência de estrutura *type* de *desc* no lugar.

int PyStructSequence_InitType2(PyTypeObject *type, PyStructSequence_Desc *desc)

Thread safety: Safe to call without external synchronization on distinct objects.

Como **`PyStructSequence_InitType()`**, mas retorna `0` em caso de sucesso e `-1` com uma exceção definida em caso de falha.

Adicionado na versão 3.4.

type PyStructSequence_Desc

Parte da ABI Estável (incluindo todos os membros).

Contém as metainformações de um tipo de sequência de estrutura a ser criado.

const char *name

Nome totalmente qualificado do tipo; codificado em UTF-8 terminado em nulo. O nome deve conter o nome do módulo.

const char *doc

Ponteiro para docstring para o tipo ou `NULL` para omitir

PyStructSequence_Field *fields

Ponteiro para um vetor terminado em `NULL` com nomes de campos do novo tipo

int n_in_sequence

Número de campos visíveis para o lado Python (se usado como tupla)

type PyStructSequence_Field

Parte da ABI Estável (incluindo todos os membros).

Descreve um campo de uma sequência de estrutura. Como uma sequência de estrutura é modelada como uma tupla, todos os campos são digitados como PyObject*. O índice no vetor **`fields`** do **`PyStructSequence_Desc`** determina qual campo da sequência de estrutura é descrito.

const char *name

Nome do campo ou `NULL` para terminar a lista de campos nomeados; definida para **`PyStructSequence_UnnamedField`** para deixar sem nome.

const char *doc

Campo docstring ou `NULL` para omitir.

const char *const PyStructSequence_UnnamedField

Parte da ABI Estável desde a versão 3.11.

Valor especial para um nome de campo para deixá-lo sem nome.

Alterado na versão 3.9: O tipo foi alterado de `char *`.

PyObject *PyStructSequence_New(PyTypeObject *type)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Cria um instância de *type*, que deve ser criada com `PyStructSequence_NewType()`.

Retorna `NULL` com uma exceção definida em caso de falha.

PyObject *PyStructSequence_GetItem(PyObject *p, Py_ssize_t pos)

Retorna valor: Referência emprestada. Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

Return the object at position *pos* in the struct sequence pointed to by *p*. The returned reference is borrowed from the struct sequence *p* (that is: it is only valid as long as you hold a reference to *p*).

A verificação de limites é realizada como uma asserção se o Python for construído em modo de depuração ou `com_asserções`.

PyObject *PyStructSequence_GET_ITEM(PyObject *p, Py_ssize_t pos)

Retorna valor: Referência emprestada. Thread safety: Safe to call from multiple threads with external synchronization only.

Apelido para `PyStructSequence_GetItem()`.

Alterado na versão 3.13: Agora implementada como um apelido para `PyStructSequence_GetItem()`.

void PyStructSequence_SetItem(PyObject *p, Py_ssize_t pos, PyObject *o)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

Define o campo no índice *pos* da sequência de estrutura *p* para o valor *o*. Como `PyTuple_SET_ITEM()`, isto só deve ser usado para preencher novas instâncias.

A verificação de limites é realizada como uma asserção se o Python for construído em modo de depuração ou `com_asserções`.

Nota

This function “steals” a reference to *o*.

void PyStructSequence_SET_ITEM(PyObject *p, Py_ssize_t *pos, PyObject *o)

Thread safety: Safe to call from multiple threads with external synchronization only.

Apelido para `PyStructSequence_SetItem()`.

Alterado na versão 3.13: Agora implementada como um apelido para `PyStructSequence_SetItem()`.

Objeto List

type PyListObject

Este subtipo de **`PyObject`** representa um objeto de lista Python.

PyTypeObject PyList_Type

Parte da ABI Estável.

Esta instância de **`PyTypeObject`** representa o tipo de lista Python. Este é o mesmo objeto que **`list`** na camada Python.

int PyList_Check(PyObject *p)

Thread safety: Atomic.

Retorna verdadeiro se *p* é um objeto lista ou uma instância de um subtipo do tipo lista. Esta função sempre tem sucesso.

int PyList_CheckExact(PyObject *p)

Thread safety: Atomic.

Retorna verdadeiro se *p* é um objeto lista, mas não uma instância de um subtipo do tipo lista. Esta função sempre tem sucesso.

PyObject *PyList_New(Py_ssize_t len)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna uma nova lista de comprimento *len* em caso de sucesso, ou `NULL` em caso de falha.

Nota

Se *len* for maior que zero, os itens do objeto da lista retornada são definidos como `NULL`. Portanto, você não pode usar funções API abstratas, como **`PySequence_SetItem()`** ou expor o objeto ao código Python antes de definir todos os itens para um objeto real com **`PyList_SetItem()`** ou **`PyList_SET_ITEM()`**. As seguintes APIs são seguras desde que antes as listas tenham sido inicializadas: **`PyList_SetItem()`** e **`PyList_SET_ITEM()`**.

Py_ssize_t PyList_Size(PyObject *list)

Parte da ABI Estável. Thread safety: Atomic.
Retorna o comprimento do objeto de lista em *list*; isto é equivalente a `len(list)` em um objeto lista.

Py_ssize_t PyList_GET_SIZE(PyObject *list)

Thread safety: Atomic.
Similar a **`PyList_Size()`**, mas sem verificação de erro.

PyObject *PyList_GetItemRef(PyObject *list, Py_ssize_t index)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.13. Thread safety: Atomic.
Retorna o objeto na posição *index* na lista apontada por *list*. A posição deve ser não negativa; não há suporte à indexação do final da lista. Se *index* estiver fora dos limites (`<0 or >=len(list)`), retorna `NULL` e levanta uma exceção **`IndexError`**.

Adicionado na versão 3.13.

PyObject *PyList_GetItem(PyObject *list, Py_ssize_t index)

Retorna valor: Referência emprestada. Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.
Como **`PyList_GetItemRef()`**, mas retorna uma referência emprestada em vez de uma referência forte.

Nota

In the free-threaded build, the returned borrowed reference may become invalid if another thread modifies the list concurrently. Prefer **`PyList_GetItemRef()`**, which returns a strong reference.

PyObject *PyList_GET_ITEM(PyObject *list, Py_ssize_t i)

Retorna valor: Referência emprestada. Thread safety: Safe to call from multiple threads with external synchronization only.
Similar a `PyList_GetItem()`, mas sem verificação de erro.

Nota

In the free-threaded build, the returned borrowed reference may become invalid if another thread modifies the list concurrently. Prefer `PyList_GetItemRef()`, which returns a strong reference.

int PyList_SetItem(PyObject *list, Py_ssize_t index, PyObject *item)

Parte da ABI Estável. Thread safety: Atomic.
Define o item no índice *index* na lista como *item*. Retorna `0` em caso de sucesso. Se *index* estiver fora dos limites, retorna `-1` e levanta uma exceção `IndexError`.

Nota

This function "steals" a reference to *item*, even on error. On success, it discards a reference to an item already in the list at the affected position (unless it was `NULL`).

void PyList_SET_ITEM(PyObject *list, Py_ssize_t i, PyObject *o)

Thread safety: Safe to call from multiple threads with external synchronization only.
Forma macro de `PyList_SetItem()` sem verificação de erro. Este é normalmente usado apenas para preencher novas listas onde não há conteúdo anterior.

A verificação de limites é realizada como uma asserção se o Python for construído em modo de depuração ou `com asserções`.

Nota

This macro "steals" a reference to *item*, and, unlike `PyList_SetItem()`, does *not* discard a reference to any item that is

being replaced; any reference in *list* at position *i* will be leaked.

Nota

In the free-threaded build, this macro has no internal synchronization. It is normally only used to fill in new lists where no other thread has a reference to the list. If the list may be shared, use `PyList_SetItem()` instead, which uses a per-object lock.

int PyList_Insert(PyObject *list, Py_ssize_t index, PyObject *item)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Insere o item *item* na lista *list* na frente do índice *index*. Retorna `0` se for bem-sucedido; retorna `-1` e levanta uma exceção se malsucedido. Análogo a `list.insert(index, item)`.

int PyList_Append(PyObject *list, PyObject *item)

Parte da ABI Estável. Thread safety: Atomic.

Adiciona o item *item* ao final da lista *list*. Retorna `0` se for bem-sucedido; retorna `-1` e levanta uma exceção se malsucedido. Análogo a `list.insert(index, item)`.

PyObject *PyList_GetSlice(PyObject *list, Py_ssize_t low, Py_ssize_t high)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna uma lista dos objetos em *list* contendo os objetos *entre low* e *alto*. Retorne `NULL` e levanta uma exceção se malsucedido. Análogo a `list[low:high]`. Não há suporte à indexação do final da lista.

int PyList_SetSlice(PyObject *list, Py_ssize_t low, Py_ssize_t high, PyObject *itemlist)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Define a fatia de *list* entre *low* e *high* para o conteúdo de *itemlist*. Análogo a `list[low:high] = itemlist`. *itemlist* pode ser `NULL`, indicando a atribuição de uma lista vazia (exclusão de fatia). Retorna `0` em caso de sucesso, `-1` em caso de falha. Não há suporte à indexação do final da lista.

Nota

In the free-threaded build, when *itemlist* is a **`list`**, both *list* and *itemlist* are locked for the duration of the operation. For other iterables (or `NULL`), only *list* is locked.

int PyList_Extend(PyObject *list, PyObject *iterable)

Thread safety: Safe for concurrent use on the same object.
Estende *list* com o conteúdo de *iterable*. Isto é o mesmo que `PyList_SetSlice(list, PY_SSIZE_T_MAX, PY_SSIZE_T_MAX, iterable)` e análogo a `list.extend(iterable)` ou `list += iterable`.

Levanta uma exceção e retorna `-1` se *list* não for um objeto **`list`**. Retorna 0 em caso de sucesso.

Adicionado na versão 3.13.

Nota

In the free-threaded build, when *iterable* is a **`list`**, **`set`**, **`dict`**, or dict view, both *list* and *iterable* (or its underlying dict) are locked for the duration of the operation. For other iterables, only *list* is locked; *iterable* may be concurrently modified by another thread.

int PyList_Clear(PyObject *list)

Thread safety: Atomic.
Remove todos os itens da *lista*. Isto é o mesmo que `PyList_SetSlice(list, 0, PY_SSIZE_T_MAX, NULL)` e análogo a `list.clear()` ou `del list[:]`.

Levanta uma exceção e retorna `-1` se *list* não for um objeto `list`. Retorna 0 em caso de sucesso.

Adicionado na versão 3.13.

int PyList_Sort(PyObject *list)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Ordena os itens de *list* no mesmo lugar. Retorna `0` em caso de sucesso, e `-1` em caso de falha. Isso é o equivalente de `list.sort()`.

Nota

In the free-threaded build, element comparison via `__lt__()` can execute arbitrary Python code, during which the per-object lock may be temporarily released. For built-in types (`str`, `int`, `float`), the lock is not released during comparison.

int PyList_Reverse(PyObject *list)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Inverte os itens de *list* no mesmo lugar. Retorna `0` em caso de sucesso, e `-1` em caso de falha. Isso é o equivalente de `list.reverse()`.

PyObject *PyList_AsTuple(PyObject *list)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo objeto tupla contendo os conteúdos de *list*; equivale a `tuple(list)`.

Dictionary objects

type PyDictObject

Este subtipo do `PyObject` representa um objeto dicionário Python.

PyTypeObject PyDict_Type

Parte da ABI Estável.

Esta instância do `PyTypeObject` representa o tipo do dicionário Python. Este é o mesmo objeto `dict` na camada do Python.

int PyDict_Check(PyObject *p)

Thread safety: Atomic.

Retorna verdadeiro se *p* é um objeto dicionário ou uma instância de um subtipo do tipo dicionário. Esta função sempre tem sucesso.

int PyDict_CheckExact(PyObject *p)

Thread safety: Atomic.

Retorna verdadeiro se *p* é um objeto dicionário, mas não uma instância de um subtipo do tipo dicionário. Esta função sempre tem sucesso.

PyObject *PyDict_New()

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo dicionário vazio ou `NULL` em caso de falha.

PyObject *PyDictProxy_New(PyObject *mapping)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um objeto `types.MappingProxyType` para um mapeamento que reforça o comportamento somente leitura. Isso normalmente é usado para criar uma visão para evitar a modificação do dicionário para tipos de classes não dinâmicas.

The first argument can be a `dict`, a `frozendict`, or a mapping.

Alterado na versão 3.15: Also accept `frozendict`.

PyTypeObject PyDictProxy_Type

Parte da ABI Estável.

O objeto de tipo para mapear objetos proxy criados por `PyDictProxy_New()` e para o atributo somente leitura `__dict__` de muitos tipos embutidos. Uma instância de `PyDictProxy_Type` fornece uma visão dinâmica e somente leitura de um dicionário subjacente: as alterações no dicionário subjacente são refletidas no proxy, mas o próprio proxy não oferece suporte a operações de mutação. Isso corresponde a `types.MappingProxyType` no Python.

void PyDict_Clear(PyObject *p)

Parte da ABI Estável. Thread safety: Atomic.

Esvazia um dicionário existente de todos os pares chave-valor.

Do nothing if the argument is not a `dict` or a `dict` subclass.

int PyDict_Contains(PyObject *p, PyObject *key)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Determina se o dicionário *p* contém *key*. Se um item em *p* corresponder à *key*, retorna `1`, caso contrário, retorna `0`. Em caso de erro, retorna `-1`. Isso equivale à expressão Python `key in p`.

The first argument can be a `dict` or a `frozendict`.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

Alterado na versão 3.15: Also accept `frozendict`.

int PyDict_ContainsString(PyObject *p, const char *key)

Thread safety: Atomic.

É o mesmo que `PyDict_Contains()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

The first argument can be a `dict` or a `frozendict`.

Adicionado na versão 3.13.

Alterado na versão 3.15: Also accept `frozendict`.

PyObject *PyDict_Copy(PyObject *p)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um novo dicionário que contém o mesmo chave-valor como *p*.

int PyDict_SetItem(PyObject *p, PyObject *key, PyObject *val)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Insere *val* no dicionário *p* com a chave *key*. *key* deve ser hasheável; se não for, `TypeError` será levantada. Retorna `0` em caso de sucesso ou `-1` em caso de falha. Esta função *não* "rouba" uma referência para *val*.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

int PyDict_SetItemString(PyObject *p, const char *key, PyObject *val)

Parte da ABI Estável. Thread safety: Atomic.

É o mesmo que `PyDict_SetItem()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

int PyDict_DelItem(PyObject *p, PyObject *key)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Remove a entrada no dicionário *p* com a chave *key*. *key* deve ser hasheável; se não for, `TypeError` é levantada. Se *key* não estiver no dicionário, `KeyError` é levantada. Retorna `0` em caso de sucesso ou `-1` em caso de falha.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

int PyDict_DelItemString(PyObject *p, const char *key)

Parte da ABI Estável. Thread safety: Atomic.
É o mesmo que `PyDict_DelItem()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

int PyDict_GetItemRef(PyObject *p, PyObject *key, PyObject **result)

Parte da ABI Estável desde a versão 3.13. Thread safety: Safe for concurrent use on the same object.
Retorna uma nova referência forte para o objeto dicionário *p* que possui uma chave *key*:

- Se a chave estiver presente, define **result* como uma nova referência forte para o valor e retorna `1`.
- Se a chave estiver ausente, define **result* como `NULL` e retorna `0`.
- Em caso de erro, levanta uma exceção, define **result* como `NULL` e retorna `-1`.

The first argument can be a `dict` or a `frozendict`.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

Adicionado na versão 3.13.

Alterado na versão 3.15: Also accept `frozendict`.

Veja também a função `PyObject_GetItem()`.

PyObject *PyDict_GetItem(PyObject *p, PyObject *key)

Retorna valor: Referência emprestada. Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

Retorna um referência emprestada para o objeto do dicionário *p* que possui uma chave *key*. Retorna `NULL` se a chave *key* não estiver presente, mas *sem* definir uma exceção.

The first argument can be a `dict` or a `frozendict`.

Nota

Exceções que ocorrem ao chamar os métodos `__hash__()` e `__eq__()` são ignoradas silenciosamente. Ao invés disso, use a função `PyDict_GetItemWithError()`.

Nota

In the free-threaded build, the returned borrowed reference may become invalid if another thread modifies the dictionary concurrently. Prefer `PyDict_GetItemRef()`, which returns a strong reference.

Alterado na versão 3.10: Chamar esta API sem um estado de thread anexado era permitido por motivos históricos. Não é mais permitido.

Alterado na versão 3.15: Also accept `frozendict`.

PyObject *PyDict_GetItemWithError(PyObject *p, PyObject *key)

Retorna valor: Referência emprestada. Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

Variante de `PyDict_GetItem()` que não suprime exceções. Retorna `NULL` **com** uma exceção definida se uma exceção ocorreu. Retorna `NULL` **

sem ** uma exceção definida se a chave não estiver presente.

Nota

In the free-threaded build, the returned borrowed reference may become invalid if another thread modifies the dictionary concurrently. Prefer `PyDict_GetItemRef()`, which returns a strong reference.

Alterado na versão 3.15: Also accept `frozendict`.

PyObject *PyDict_GetItemString(PyObject *p, const char *key)

Retorna valor: Referência emprestada. Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.

É o mesmo que `PyDict_GetItem()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

Nota

Exceções que ocorrem ao chamar os métodos `__hash__()` e `__eq__()` ou ao criar objetos temporários da classe `str` são ignoradas silenciosamente. Ao invés disso, prefira usar a função `PyDict_GetItemWithError()` com sua própria *key* de `PyUnicode_FromString()`.

Nota

In the free-threaded build, the returned borrowed reference may become invalid if another thread modifies the dictionary concurrently. Prefer `PyDict_GetItemStringRef()`, which returns a strong reference.

Alterado na versão 3.15: Also accept `frozendict`.

int PyDict_GetItemStringRef(PyObject *p, const char *key, PyObject **result)

Parte da ABI Estável desde a versão 3.13. Thread safety: Atomic.
Similar a `PyDict_GetItemRef()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

Adicionado na versão 3.13.

Alterado na versão 3.15: Also accept `frozendict`.

PyObject *PyDict_SetDefault(PyObject *p, PyObject *key, PyObject *defaultobj)

Retorna valor: Referência emprestada. Thread safety: Safe to call from multiple threads with external synchronization only.
Isso é o mesmo que o `dict.setdefault()` de nível Python. Se presente, ele retorna o valor correspondente a *key* do dicionário *p*. Se a chave não estiver no dict, ela será inserida com o valor *defaultobj* e *defaultobj* será retornado. Esta função avalia a função hash de *key* apenas uma vez, em vez de avaliá-la independentemente para a pesquisa e a inserção.

Adicionado na versão 3.4.

Nota

In the free-threaded build, the returned borrowed reference may become invalid if another thread modifies the dictionary concurrently. Prefer `PyDict_SetDefaultRef()`, which returns a strong reference.

int PyDict_SetDefaultRef(PyObject *p, PyObject *key, PyObject *default_value, PyObject **result)

Parte da ABI Estável desde a versão 3.15. Thread safety: Safe for concurrent use on the same object.
Insere *default_value* no dicionário *p* com uma chave *key* se a chave ainda não estiver presente no dicionário. Se *result* não for `NULL`, então **result* é definido como uma referência forte para *default_value*, se a chave não estiver presente, ou para o valor existente, se *key* já estava presente no dicionário. Retorna `1` se a chave estava presente e

default_value não foi inserido, ou `0` se a chave não estava presente e *default_value* foi inserido. Em caso de falha, retorna `-1`, define uma exceção e define `*result` como `NULL`.

Para maior clareza: se você tiver uma referência forte para *default_value* antes de chamar esta função, então depois que ela retornar, você terá uma referência forte para *default_value* e **result* (se não for `NULL`). Estes podem referir-se ao mesmo objeto: nesse caso você mantém duas referências separadas para ele.

Nota

The operation is atomic on free threading when *key* is **`str`**, **`int`**, **`float`**, **`bool`** or **`bytes`**.

Adicionado na versão 3.13.

int PyDict_Pop(PyObject *p, PyObject *key, PyObject **result)

Thread safety: Safe for concurrent use on the same object.

Remove *key* do dicionário *p* e, opcionalmente, retorna o valor removido. Não levanta **`KeyError`** se a chave estiver ausente.

- Se a chave estiver presente, define **result* como uma nova referência para o valor se *result* não for `NULL` e retorna `1`.
- Se a chave estiver ausente, define **result* como `NULL` se *result* não for `NULL` e retorna `0`.
- Em caso de erro, levanta uma exceção e retorna `-1`.

Similar a **`dict.pop()`**, mas sem o valor padrão e sem levantar **`KeyError`** se a chave estiver ausente.

Nota

The operation is atomic on free threading when *key* is **`str`**, **`int`**, **`float`**, **`bool`** or **`bytes`**.

Adicionado na versão 3.13.

int PyDict_PopString(PyObject *p, const char *key, PyObject **result)

Thread safety: Atomic.

Similar a `PyDict_Pop()`, mas *key* é especificada como uma string de bytes const char* codificada em UTF-8, em vez de um PyObject*.

Adicionado na versão 3.13.

PyObject *PyDict_Items(PyObject *p)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um `PyListObject` contendo todos os itens do dicionário.

The first argument can be a `dict` or a `frozendict`.

Alterado na versão 3.15: Also accept `frozendict`.

PyObject *PyDict_Keys(PyObject *p)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um `PyListObject` contendo todas as chaves do dicionário.

The first argument can be a `dict` or a `frozendict`.

Alterado na versão 3.15: Also accept `frozendict`.

PyObject *PyDict_Values(PyObject *p)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna um `PyListObject` contendo todos os valores do dicionário *p*.

The first argument can be a `dict` or a `frozendict`.

Alterado na versão 3.15: Also accept `frozendict`.

Py_ssize_t PyDict_Size(PyObject *p)

Parte da ABI Estável. Thread safety: Atomic.

Retorna o número de itens no dicionário. Isso é equivalente a `len(p)` em um dicionário.

The argument can be a **`dict`** or a **`frozendict`**.

Alterado na versão 3.15: Also accept **`frozendict`**.

Py_ssize_t PyDict_GET_SIZE(PyObject *p)

Thread safety: Atomic.
Similar a **`PyDict_Size()`**, mas sem verificação de erro.

Alterado na versão 3.15: Also accept **`frozendict`**.

int PyDict_Next(PyObject *p, Py_ssize_t *ppos, PyObject **pkey, PyObject **pvalue)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.
Itera todos os pares de valores-chave no dicionário *p*. O **`Py_ssize_t`** referido por *ppos* deve ser inicializado para `0` antes da primeira chamada para esta função para iniciar a iteração; a função retorna true para cada par no dicionário e false quando todos os pares forem relatados. Os parâmetros *pkey* e *pvalue* devem apontar para variáveis de PyObject* que serão preenchidas com cada chave e valor, respectivamente, ou podem ser `NULL`. Todas as referências retornadas por meio deles são emprestadas. *ppos* não deve ser alterado durante a iteração. Seu valor representa deslocamentos dentro da estrutura do dicionário interno e, como a estrutura é esparsa, os deslocamentos não são consecutivos.

The first argument can be a **`dict`** or a **`frozendict`**.

Por exemplo:

```
PyObject *key, *value;
Py_ssize_t pos = 0;

while (PyDict_Next(self->dict, &pos, &key, &value)) {
    /* fazer algo de interessante com os valores... */
```

```
    ...
}
```

O dicionário *p* não deve sofrer mutação durante a iteração. É seguro modificar os valores das chaves à medida que você itera no dicionário, mas apenas enquanto o conjunto de chaves não mudar. Por exemplo:

```
PyObject *key, *value;
Py_ssize_t pos = 0;

while (PyDict_Next(self->dict, &pos, &key, &value)) {
    long i = PyLong_AsLong(value);
    if (i == -1 && PyErr_Occurred()) {
        return -1;
    }
    PyObject *o = PyLong_FromLong(i + 1);
    if (o == NULL)
        return -1;
    if (PyDict_SetItem(self->dict, key, o) < 0) {
        Py_DECREF(o);
        return -1;
    }
    Py_DECREF(o);
}
```

The function is not thread-safe in the free-threaded build without external synchronization for a mutable **dict**. You can use **Py_BEGIN_CRITICAL_SECTION** to lock the dictionary while iterating over it:

```
Py_BEGIN_CRITICAL_SECTION(self->dict);
while (PyDict_Next(self->dict, &pos, &key, &value)) {
    ...
}
Py_END_CRITICAL_SECTION();
```

The function is thread-safe on a **frozendict**.

Nota

Na construção com threads livres, esta função pode ser usada com segurança dentro de uma seção crítica. No entanto, as referências

retornadas para *pkey* e *pvalue* são emprestadas e só são válidas enquanto a seção crítica é mantida. Se precisar usar esses objetos fora da seção crítica ou quando a seção crítica puder ser suspensa, crie uma referência forte (por exemplo, usando `Py_NewRef()`).

Alterado na versão 3.15: Also accept `frozendict`.

int PyDict_Merge(PyObject *a, PyObject *b, int override)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Itera sobre o objeto de mapeamento *b* adicionando pares de valores-chave ao dicionário *a*. *b* pode ser um dicionário, ou qualquer objeto que suporte `PyMapping_Keys()` e `PyObject_GetItem()`. Se *override* for verdadeiro, os pares existentes em *a* serão substituídos se uma chave correspondente for encontrada em *b*, caso contrário, os pares serão adicionados apenas se não houver uma chave correspondente em *a*. Retorna `0` em caso de sucesso ou `-1` se uma exceção foi levantada.

Nota

In the free-threaded build, when *b* is a `dict` (with the standard iterator), both *a* and *b* are locked for the duration of the operation. When *b* is a non-dict mapping, only *a* is locked; *b* may be concurrently modified by another thread.

int PyDict_Update(PyObject *a, PyObject *b)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

É o mesmo que `PyDict_Merge(a, b, 1)` em C, e é semelhante a `a.update(b)` em Python, exceto que `PyDict_Update()` não cai na iteração em uma sequência de pares de valores de chave se o segundo argumento não tiver o atributo “keys”. Retorna `0` em caso de sucesso ou `-1` se uma exceção foi levantada.

Nota

In the free-threaded build, when *b* is a **`dict`** (with the standard iterator), both *a* and *b* are locked for the duration of the operation. When *b* is a non-dict mapping, only *a* is locked; *b* may be concurrently modified by another thread.

int PyDict_MergeFromSeq2(PyObject *a, PyObject *seq2, int override)

Parte da ABI Estável. *Thread safety: Safe for concurrent use on the same object*.

Atualiza ou mescla no dicionário *a*, a partir dos pares de chave-valor em *seq2*. *seq2* deve ser um objeto iterável produzindo objetos iteráveis de comprimento 2, vistos como pares chave-valor. No caso de chaves duplicadas, a última vence se *override* for verdadeiro, caso contrário, a primeira vence. Retorne `0` em caso de sucesso ou `-1` se uma exceção foi levantada. Python equivalente (exceto para o valor de retorno):

```
def PyDict_MergeFromSeq2(a, seq2, override):
    for key, value in seq2:
        if override or key not in a:
            a[key] = value
```

Nota

In the free-threaded build, only *a* is locked. The iteration over *seq2* is not synchronized; *seq2* may be concurrently modified by another thread.

int PyDict_AddWatcher(PyDict_WatchCallback callback)

Thread safety: Safe to call from multiple threads with external synchronization only.

Registra *callback* como um observador de dicionário. Retorna um ID inteiro não negativo que deve ser passado para futuras chamadas a **`PyDict_Watch()`**. Em caso de erro (por exemplo, não há mais IDs de observador disponíveis), retorna `-1` e define uma exceção.

Nota

This function is not internally synchronized. In the free-threaded build, callers should ensure no concurrent calls to `PyDict_AddWatcher()` or `PyDict_ClearWatcher()` are in progress.

Adicionado na versão 3.12.

int PyDict_ClearWatcher(int watcher_id)

Thread safety: Safe to call from multiple threads with external synchronization only.

Limpa o observador identificado por *watcher_id* retornado anteriormente de `PyDict_AddWatcher()`. Retorna `0` em caso de sucesso, `-1` em caso de erro (por exemplo, se o *watcher_id* fornecido nunca foi registrado).

Nota

This function is not internally synchronized. In the free-threaded build, callers should ensure no concurrent calls to `PyDict_AddWatcher()` or `PyDict_ClearWatcher()` are in progress.

Adicionado na versão 3.12.

int PyDict_Watch(int watcher_id, PyObject *dict)

Thread safety: Safe to call without external synchronization on distinct objects.

Marca o dicionário *dict* como observado. A função de retorno concedida a *watcher_id* por `PyDict_AddWatcher()` será chamada quando *dict* for modificado ou desalocado. Retorna `0` em caso de sucesso ou `-1` em caso de erro.

Adicionado na versão 3.12.

int PyDict_Unwatch(int watcher_id, PyObject *dict)

Thread safety: Safe to call without external synchronization on distinct objects.

Marca o dicionário *dict* como não mais observado. A função de retorno concedida a *watcher_id* por `PyDict_AddWatcher()` será chamada quando *dict* for modificado ou desalocado. O dicionário deve ter sido observado anteriormente por este observador. Retorna `0` em caso de sucesso ou `-1` em caso de erro.

Adicionado na versão 3.12.

type PyDict_WatchEvent

Enumeração de possíveis eventos de observador de dicionário: `PyDict_EVENT_ADDED`, `PyDict_EVENT_MODIFIED`, `PyDict_EVENT_DELETED`, `PyDict_EVENT_CLONED`, `PyDict_EVENT_CLEARED` ou `PyDict_EVENT_DEALLOCATED`.

Adicionado na versão 3.12.

typedef int (*PyDict_WatchCallback)(PyDict_WatchEvent event, PyObject *dict, PyObject *key, PyObject *new_value)

Tipo de uma função de retorno de chamada de observador de dicionário.

Se *event* for `PyDict_EVENT_CLEARED` ou `PyDict_EVENT_DEALLOCATED`, tanto *key* quanto *new_value* serão `NULL`. Se *event* for `PyDict_EVENT_ADDED` ou `PyDict_EVENT_MODIFIED`, *new_value* será o novo valor de *key*. Se *event* for `PyDict_EVENT_DELETED`, *key* estará sendo excluída do dicionário e *new_value* será `NULL`.

`PyDict_EVENT_CLONED` ocorre quando *dict* estava anteriormente vazio e outro dict é mesclado a ele. Para manter a eficiência dessa operação, os eventos `PyDict_EVENT_ADDED` por chave não são emitidos nesse caso; em vez disso, um único `PyDict_EVENT_CLONED` é emitido e *key* será o dicionário de origem.

A função de retorno pode inspecionar, mas não deve modificar o *dict*; isso pode ter efeitos imprevisíveis, inclusive recursão infinita. Não acione a execução do código Python na função de retorno, pois isso poderia modificar o dict como um efeito colateral.

Se *event* for `PyDict_EVENT_DEALLOCATED`, a obtenção de uma nova referência na função de retorno para o dicionário prestes a ser destruído o ressuscitará e impedirá que ele seja liberado nesse momento. Quando o objeto ressuscitado for destruído mais tarde, quaisquer funções de retorno do observador ativos naquele momento serão chamados novamente.

As funções de retorno ocorrem antes que a modificação notificada no *dict* ocorra, de modo que o estado anterior do *dict* possa ser inspecionado.

Se a função de retorno definir uma exceção, ela deverá retornar `-1`. Essa exceção será exibida como uma exceção não levantável usando **`PyErr_WriteUnraisable()`**. Caso contrário, deverá retornar `0`.

É possível que já exista uma exceção pendente definida na entrada da função de retorno. Nesse caso, a função de retorno deve retornar `0` com a mesma exceção ainda definida. Isso significa que a função de retorno não pode chamar nenhuma outra API que possa definir uma exceção, a menos que salve e limpe o estado da exceção primeiro e restaure a exceção antes de retornar.

Adicionado na versão 3.12.

Objetos de visão de dicionário

int PyDictViewSet_Check(PyObject *op)

Retorna verdadeiro se *op* for uma visão de um conjunto dentro de um dicionário. Isso atualmente equivale a PyDictKeys_Check(op) || PyDictItems_Check(op). Esta função sempre tem sucesso.

PyTypeObject PyDictKeys_Type

Parte da ABI Estável.

Objeto tipo para uma visualização das chaves do dicionário. Em Python, este é o tipo do objeto retornado por **`dict.keys()`**.

int PyDictKeys_Check(PyObject *op)

Retorna verdadeiro se *op* for uma instância de uma visualização de chaves de dicionário. Esta função sempre tem sucesso.

PyTypeObject PyDictValues_Type

Parte da ABI Estável.

Objeto tipo para uma visualização dos valores do dicionário. Em Python, este é o tipo do objeto retornado por `dict.values()`.

int PyDictValues_Check(PyObject *op)

Retorna verdadeiro se *op* for uma instância de uma visualização de valores de dicionário. Esta função sempre tem sucesso.

PyTypeObject PyDictItems_Type

Parte da ABI Estável.

Objeto tipo para uma visualização de itens de dicionário. Em Python, este é o tipo do objeto retornado por `dict.items()`.

int PyDictItems_Check(PyObject *op)

Retorna verdadeiro se *op* for uma instância de uma visualização de itens de dicionário. Esta função sempre tem sucesso.

Frozen dictionary objects

Adicionado na versão 3.15.

PyTypeObject PyFrozenDict_Type

This instance of `PyTypeObject` represents the Python frozen dictionary type. This is the same object as `frozendict` in the Python layer.

int PyAnyDict_Check(PyObject *p)

Return true if *p* is a `dict` object, a `frozendict` object, or an instance of a subtype of the `dict` or `frozendict` type. This function always succeeds.

int PyAnyDict_CheckExact(PyObject *p)

Return true if *p* is a **`dict`** object or a **`frozendict`** object, but not an instance of a subtype of the **`dict`** or **`frozendict`** type. This function always succeeds.

int PyFrozenDict_Check(PyObject *p)

Return true if *p* is a **`frozendict`** object or an instance of a subtype of the **`frozendict`** type. This function always succeeds.

int PyFrozenDict_CheckExact(PyObject *p)

Return true if *p* is a **`frozendict`** object, but not an instance of a subtype of the **`frozendict`** type. This function always succeeds.

PyObject *PyFrozenDict_New(PyObject *iterable)

Return a new **`frozendict`** from an iterable, or `NULL` on failure with an exception set.

Create an empty dictionary if *iterable* is `NULL`.

Ordered dictionaries

A API C do Python fornece uma interface para **`collections.OrderedDict`** do C. Desde o Python 3.7, os dicionários são ordenados por padrão, então geralmente há pouca necessidade dessas funções; prefira `PyDict*` sempre que possível.

PyTypeObject PyODict_Type

Objeto de tipo para dicionários ordenados. Este é o mesmo objeto que **`collections.OrderedDict`** na camada Python.

int PyODict_Check(PyObject *od)

Retorna verdadeiro se *od* for um objeto de dicionário ordenado ou uma instância de um subtipo do tipo **`OrderedDict`**. Esta função sempre tem sucesso..

int PyODict_CheckExact(PyObject *od)

Retorna verdadeiro se *od* for um objeto de dicionário ordenado, mas não uma instância de um subtipo do tipo **`OrderedDict`**. Esta função sempre tem sucesso.

PyTypeObject PyODictKeys_Type

Análogo a **`PyDictKeys_Type`** para dicionários ordenados.

PyTypeObject PyODictValues_Type

Análogo a **`PyDictValues_Type`** para dicionários ordenados.

PyTypeObject PyODictItems_Type

Análogo a **`PyDictItems_Type`** para dicionários ordenados.

PyObject *PyODict_New(void)

Retorna um novo dicionário ordenado vazio ou `NULL` em caso de falha.

Isso é análogo a **`PyDict_New()`**.

int PyODict_SetItem(PyObject *od, PyObject *key, PyObject *value)

Insere o valor *value* no dicionário ordenado *od* com a chave de *key*. Retorna `0` em caso de sucesso ou `-1` com uma exceção definida em caso de falha.

Isso é análogo a **`PyDict_SetItem()`**.

int PyODict_DelItem(PyObject *od, PyObject *key)

Remove a entrada do dicionário ordenado *od* com a chave *key*. Retorna `0` em caso de sucesso ou `-1` com uma exceção definida em caso de falha.

Isso é análogo a **`PyDict_DelItem()`**.

Estes são apelidos suavemente descontinuados a APIs de `PyDict`:

PyODict	**PyDict**

PyODict	PyDict
PyODict_GetItem(od, key)	PyDict_GetItem()
PyODict_GetItemWithError(od, key)	PyDict_GetItemWithError()
PyODict_GetItemString(od, key)	PyDict_GetItemString()
PyODict_Contains(od, key)	PyDict_Contains()
PyODict_Size(od)	PyDict_Size()
PyODict_SIZE(od)	PyDict_GET_SIZE()

Objeto Set

Esta seção detalha a API pública para os objetos `set` e `frozenset`. Qualquer funcionalidade não listada abaixo é melhor acessada usando o protocolo de objeto abstrato (incluindo `PyObject_CallMethod()`, `PyObject_RichCompareBool()`, `PyObject_Hash()`, `PyObject_Repr()`, `PyObject_IsTrue()`, `PyObject_Print()` e `PyObject_GetIter()`) ou o protocolo abstrato de número (incluindo `PyNumber_And()`, `PyNumber_Subtract()`, `PyNumber_Or()`, `PyNumber_Xor()`, `PyNumber_InPlaceAnd()`, `PyNumber_InPlaceSubtract()`, `PyNumber_InPlaceOr()` e `PyNumber_InPlaceXor()`).

type PySetObject

Este subtipo de `PyObject` é usado para manter os dados internos para ambos os objetos `set` e `frozenset`. É como um `PyDictObject` em que tem um tamanho fixo para conjuntos pequenos (muito parecido com o armazenamento de tupla) e apontará para um bloco de memória de tamanho variável separado para conjuntos de tamanho médio e grande (muito parecido com lista armazenamento). Nenhum dos campos desta estrutura deve ser considerado público e todos estão sujeitos a alterações. Todo o acesso deve ser feito por meio da API documentada, em vez de manipular os valores na estrutura.

PyTypeObject PySet_Type

Parte da ABI Estável.

Essa é uma instância de `PyTypeObject` representando o tipo Python `set`

PyTypeObject PyFrozenSet_Type

Parte da ABI Estável.

Esta é uma instância de `PyTypeObject` representando o tipo Python `frozenset`.

As macros de verificação de tipo a seguir funcionam em ponteiros para qualquer objeto Python. Da mesma forma, as funções construtoras funcionam com qualquer objeto Python iterável.

int PySet_Check(PyObject *p)

Retorna verdadeiro se *p* for um objeto `set` ou uma instância de um subtipo. Esta função sempre tem sucesso.

int PyFrozenSet_Check(PyObject *p)

Retorna verdadeiro se *p* for um objeto `frozenset` ou uma instância de um subtipo. Esta função sempre tem sucesso.

int PyAnySet_Check(PyObject *p)

Retorna verdadeiro se *p* for um objeto `set`, um objeto `frozenset` ou uma instância de um subtipo. Esta função sempre tem sucesso.

int PySet_CheckExact(PyObject *p)

Retorna verdadeiro se *p* for um objeto `set`, mas não uma instância de um subtipo. Esta função sempre tem sucesso.

Adicionado na versão 3.10.

int PyAnySet_CheckExact(PyObject *p)

Retorna verdadeiro se *p* for um objeto `set` ou um objeto `frozenset`, mas não uma instância de um subtipo. Esta função sempre tem sucesso.

int PyFrozenSet_CheckExact(PyObject *p)

Retorna verdadeiro se *p* for um objeto `frozenset`, mas não uma instância de um subtipo. Esta função sempre tem sucesso.

PyObject *PySet_New(PyObject *iterable)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Retorna uma nova `set` contendo objetos retornados pelo iterável *iterable*. O *iterable* pode ser `NULL` para criar um novo conjunto vazio. Retorna o novo conjunto em caso de sucesso ou `NULL` em caso de falha. Levanta `TypeError` se *iterable* não for realmente iterável. O construtor também é útil para copiar um conjunto (`c=set(s)`).

Nota

The operation is atomic on free threading when *iterable* is a **set**, **frozenset**, **dict** or **frozendict**.

PyObject *PyFrozenSet_New(PyObject *iterable)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Retorna uma nova **frozenset** contendo objetos retornados pelo iterável *iterable*. O *iterable* pode ser `NULL` para criar um novo frozenset vazio. Retorna o novo conjunto em caso de sucesso ou `NULL` em caso de falha. Levanta **TypeError** se *iterable* não for realmente iterável.

Nota

The operation is atomic on free threading when *iterable* is a **set**, **frozenset**, **dict** or **frozendict**.

As seguintes funções e macros estão disponíveis para instâncias de **set** ou **frozenset** ou instâncias de seus subtipos.

Py_ssize_t PySet_Size(PyObject *anyset)

Parte da ABI Estável. Thread safety: Atomic.

Retorna o comprimento de um objeto **set** ou **frozenset**. Equivalente a `len(anyset)`. Levanta um **SystemError** se *anyset* não for um **set**, **frozenset**, ou uma instância de um subtipo.

Py_ssize_t PySet_GET_SIZE(PyObject *anyset)

Thread safety: Atomic.

Forma macro de **PySet_Size()** sem verificação de erros.

int PySet_Contains(PyObject *anyset, PyObject *key)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Retorna `1` se encontrado, `0` se não encontrado, e `-1` se um erro é encontrado. Ao contrário do método Python **__contains__()**, esta função não converte automaticamente conjuntos não hasheáveis em

frozensets temporários. Levanta um `TypeError` se a *key* não for hasheável. Levanta `SystemError` se *anyset* não é um `set`, `frozenset`, ou uma instância de um subtipo.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

int PySet_Add(PyObject *set, PyObject *key)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Adiciona *key* a uma instância de `set`. Também funciona com instâncias de `frozenset` (como `PyTuple_SetItem()`, ele pode ser usado para preencher os valores de novos conjuntos de congelamentos antes que eles sejam expostos a outro código). Retorna `0` em caso de sucesso ou `-1` em caso de falha. Levanta um `TypeError` se a *key* não for hasheável. Levanta uma `MemoryError` se não houver espaço para crescer. Levanta uma `SystemError` se *set* não for uma instância de `set` ou seu subtipo.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

As seguintes funções estão disponíveis para instâncias de `set` ou seus subtipos, mas não para instâncias de `frozenset` ou seus subtipos.

int PySet_Discard(PyObject *set, PyObject *key)

Parte da ABI Estável. Thread safety: Safe for concurrent use on the same object.

Retorna `1` se encontrado e removido, `0` se não encontrado (nenhuma ação realizada) e `-1` se um erro for encontrado. Não levanta `KeyError` para chaves ausentes. Levanta uma `TypeError` se a *key* não for hasheável. Ao contrário do método Python `discard()`, esta função não converte automaticamente conjuntos não hasheáveis em frozensets

temporários. Levanta `SystemError` se *set* não é uma instância de `set` ou seu subtipo.

Nota

The operation is atomic on free threading when *key* is `str`, `int`, `float`, `bool` or `bytes`.

PyObject *PySet_Pop(PyObject *set)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Retorna uma nova referência a um objeto arbitrário no *set* e remove o objeto do *set*. Retorna `NULL` em caso de falha. Levanta `KeyError` se o conjunto estiver vazio. Levanta uma `SystemError` se *set* não for uma instância de `set` ou seu subtipo.

int PySet_Clear(PyObject *set)

Parte da ABI Estável. Thread safety: Atomic.

Esvazia um conjunto existente de todos os elementos. Retorna `0` em caso de sucesso. Retorna `-1` e levanta `SystemError` se *set* não for uma instância de `set` ou seu subtipo.

Nota

In the free-threaded build, the set is emptied before its entries are cleared, so other threads will observe an empty set rather than intermediate states.

Deprecated API

PySet_MINSIZE

A constant representing the size of an internal preallocated table inside `PySetObject` instances.

This is documented solely for completeness, as there are no guarantees that a given version of CPython uses preallocated tables with a fixed size. In code that does not deal with unstable set internals, **`PySet_MINSIZE`** can be replaced with a small constant like `8`.

If looking for the size of a set, use **`PySet_Size()`** instead.

Suavemente descontinuado desde a versão 3.14.

Objetos Função

Existem algumas funções específicas para as funções do Python.

type PyFunctionObject

A estrutura C usada para funções.

PyTypeObject PyFunction_Type

Esta é uma instância de **`PyTypeObject`** e representa o tipo de função Python. Está exposta a programadores Python como `types.FunctionType`.

int PyFunction_Check(PyObject *o)

Retorna verdadeiro se *o* for um objeto função (tem tipo **`PyFunction_Type`**). O parâmetro não deve ser `NULL`. Esta função sempre obtém sucesso.

PyObject *PyFunction_New(PyObject *code, PyObject *globals)

Retorna valor: Nova referência.

Retorna um novo objeto função associado ao objeto código *code*. *globals* deve ser um dicionário com as variáveis globais acessíveis à função.

O nome e *docstring* da função são adquiridos pelo objeto código. O atributo **`__module__`** é adquirido por meio de *globals*. Os valores-padrão de argumentos, anotações, e fechamento são definidos como `NULL`. O atributo **`__qualname__`** é definido com o mesmo valor do campo **`co_qualname`** de um objeto código.

PyObject *PyFunction_NewWithQualName(PyObject *code, PyObject *globals, PyObject *qualname)

Retorna valor: Nova referência.

Similar a **`PyFunction_New()`**, mas também permite definir o atributo **`__qualname__`** do objeto função. *qualname* deve ser um objeto Unicode

ou `NULL`. Se `NULL`, o atributo **`__qualname__`** é definido com o mesmo valor do campo **`co_qualname`** do objeto código.

Adicionado na versão 3.3.

PyObject *PyFunction_GetCode(PyObject *op)

Retorna valor: Referência emprestada.
Retorna o objeto código associado ao objeto função *op*.

PyObject *PyFunction_GetGlobals(PyObject *op)

Retorna valor: Referência emprestada.
Retorna o dicionário global associado ao objeto função *op*.

PyObject *PyFunction_GetModule(PyObject *op)

Retorna valor: Referência emprestada.
Retorna uma referência emprestada ao atributo **`__module__`** do objeto função *op*. Pode ser *NULL*.

Normalmente, trata-se de um **`string`** contendo o nome do módulo, mas pode ser definido como qualquer outro objeto pelo código Python.

PyObject *PyFunction_GetDefaults(PyObject *op)

Retorna valor: Referência emprestada.
Retorna os valores-padrão de argumentos do objeto função *op*. Pode ser uma tupla de argumentos ou `NULL`.

int PyFunction_SetDefaults(PyObject *op, PyObject *defaults)

Define os valores-padrão dos argumentos do objeto função *op*. *defaults* deve ser `Py_None` ou uma tupla.

Levanta **`SystemError`** e retorna `-1` em caso de falha.

void PyFunction_SetVectorcall(PyFunctionObject *func, vectorcallfunc vectorcall)

Define o campo *vectorcall* de um objeto função *func*.

Atenção: extensões que usam essa API devem preservar o comportamento inalterado (padrão) de uma função *vectorcall*!

Adicionado na versão 3.12.

PyObject *PyFunction_GetKwDefaults(PyObject *op)

Retorna valor: Referência emprestada.
Retorna os valores-padrão de argumentos somente-nomeados do objeto função *op*. Pode ser um dicionário de argumentos ou `NULL`.

int PyFunction_SetKwDefaults(PyObject *op, PyObject *defaults)

Define os valores padrão dos argumentos somente-nomeados do objeto função *op*. *defaults* deve ser um dicionário de argumentos somente-nomeados ou `Py_None`.

Esta função retorna `0` em caso de sucesso e retorna `-1` com uma exceção definida em caso de falha.

PyObject *PyFunction_GetClosure(PyObject *op)

Retorna valor: Referência emprestada.
Retorna o fechamento associado ao objeto função *op*. Pode ser `NULL` ou uma tupla de objetos célula.

int PyFunction_SetClosure(PyObject *op, PyObject *closure)

Define o fechamento associado ao objeto função *op*. *closure* deve ser `Py_None` ou uma tupla de objetos de célula.

Levanta **`SystemError`** e retorna `-1` em caso de falha.

PyObject *PyFunction_GetAnnotations(PyObject *op)

Retorna valor: Referência emprestada.
Retorna as anotações do objeto função *op*. Este pode ser um dicionário mutável ou `NULL`.

int PyFunction_SetAnnotations(PyObject *op, PyObject *annotations)

Define as anotações para o objeto função *op*. *annotations* deve ser um dicionário ou `Py_None`.

Levanta **`SystemError`** e retorna `-1` em caso de falha.

PyObject *PyFunction_GET_CODE(PyObject *op)
PyObject *PyFunction_GET_GLOBALS(PyObject *op)
PyObject *PyFunction_GET_MODULE(PyObject *op)
PyObject *PyFunction_GET_DEFAULTS(PyObject *op)
PyObject *PyFunction_GET_KW_DEFAULTS(PyObject *op)
PyObject *PyFunction_GET_CLOSURE(PyObject *op)
PyObject *PyFunction_GET_ANNOTATIONS(PyObject *op)

Retorna valor: Referência emprestada.
Essas funções são semelhantes às suas contrapartes `PyFunction_Get*`, mas não realizam verificação de tipos. Passar qualquer coisa que não seja uma instância de **`PyFunction_Type`** é um comportamento indefinido.

int PyFunction_AddWatcher(PyFunction_WatchCallback callback)

Registra *callback* como uma sentinela de função para o interpretador atual. Retorna um ID que pode ser passado para **`PyFunction_ClearWatcher()`**. Em caso de erro (por exemplo, sem novos IDs de sentinelas disponíveis), retorna `-1` e define uma exceção.

Adicionado na versão 3.12.

int PyFunction_ClearWatcher(int watcher_id)

Cancela o registro da sentinela identificada pelo *watcher_id* retornado por **`PyFunction_AddWatcher()`** para o interpretador atual. Retorna `0` em caso de sucesso, ou `-1` e define uma exceção em caso de erro (por exemplo, ao receber um *watcher_id* desconhecido.)

Adicionado na versão 3.12.

type PyFunction_WatchEvent

Enumeração de possíveis eventos do observador de funções:

- `PyFunction_EVENT_CREATE`
- `PyFunction_EVENT_DESTROY`
- `PyFunction_EVENT_MODIFY_CODE`
- `PyFunction_EVENT_MODIFY_DEFAULTS`
- `PyFunction_EVENT_MODIFY_KWDEFAULTS`

Adicionado na versão 3.12:

- `PyFunction_PYFUNC_EVENT_MODIFY_QUALNAME`

Adicionado na versão 3.15.

typedef int (*PyFunction_WatchCallback)(PyFunction_WatchEvent event, PyFunctionObject *func, PyObject *new_value)

Tipo de uma função de retorno de sentinela de função.

Se *event* for `PyFunction_EVENT_CREATE` ou `PyFunction_EVENT_DESTROY`, *new_value* será `NULL`. Senão, *new_value* portará uma referência emprestada ao novo valor prestes a ser guardado em *func* para o atributo que está sendo modificado.

A função de retorno poderá somente inspecionar, e não modificar *func*. Caso contrário, poderíamos ter efeitos imprevisíveis, incluindo recursão infinita.

Se *event* for `PyFunction_EVENT_CREATE`, a função de retorno será invocada após *func* ter sido completamente inicializada. Caso contrário, a função de retorno será invocada antes de modificar *func*, então o estado anterior de *func* poderá ser inspecionado. O ambiente de execução pode otimizar a criação de objetos função, quando possível, ao ignorá-las. Nesses casos, nenhum evento será emitido. Apesar de decisões de otimização criarem diferenças de comportamento em tempo de execução, elas não mudam a semântica do código Python sendo executado.

Se *event* for `PyFunction_EVENT_DESTROY`, então obter uma referência dentro da função de retorno para a função prestes a ser destruída irá

revivê-la, impedindo que esta função seja liberada nesse tempo. Quando o objeto revivido for destruído, quaisquer funções de retorno sentinelas ativas nesse momento poderão ser chamadas novamente.

Se a função de retorno definir uma exceção, ela deverá retornar `-1`. Essa exceção será exibida como uma exceção não levantável usando `PyErr_WriteUnraisable()`. Caso contrário, deverá retornar `0`.

É possível que já exista uma exceção pendente definida na entrada da função de retorno. Nesse caso, a função de retorno deve retornar `0` com a mesma exceção ainda definida. Isso significa que a função de retorno não pode chamar nenhuma outra API que possa definir uma exceção, a menos que salve e limpe o estado da exceção primeiro e restaure a exceção antes de retornar.

Adicionado na versão 3.12.

Objetos de Método de Instância

Um método de instância é um invólucro para um **`PyCFunction`** e a nova maneira de vincular um **`PyCFunction`** a um objeto classe. Ele substitui a chamada anterior `PyMethod_New(func, NULL, class)`.

PyTypeObject PyInstanceMethod_Type

Esta instância de **`PyTypeObject`** representa o tipo de método de instância Python. Não é exposto a programas Python.

int PyInstanceMethod_Check(PyObject *o)

Retorna verdadeiro se *o* é um objeto de método de instância (tem tipo **`PyInstanceMethod_Type`**). O parâmetro não deve ser `NULL`. Esta função sempre tem sucesso.

PyObject *PyInstanceMethod_New(PyObject *func)

Retorna valor: Nova referência.

Retorna um novo objeto de método de instância, com *func* sendo qualquer objeto chamável. *func* é a função que será chamada quando o método de instância for chamado.

PyObject *PyInstanceMethod_Function(PyObject *im)

Retorna valor: Referência emprestada.

Retorna o objeto função associado ao método de instância *im*.

PyObject *PyInstanceMethod_GET_FUNCTION(PyObject *im)

Retorna valor: Referência emprestada.

Versão macro de **`PyInstanceMethod_Function()`** que evita a verificação de erros.

Objetos método

Métodos são objetos função vinculados. Os métodos são sempre associados a uma instância de uma classe definida pelo usuário. Métodos não vinculados (métodos vinculados a um objeto de classe) não estão mais disponíveis.

PyTypeObject PyMethod_Type

Esta instância de **`PyTypeObject`** representa o tipo de método Python. Isso é exposto a programas Python como `types.MethodType`.

int PyMethod_Check(PyObject *o)

Retorna verdadeiro se *o* é um objeto de método (tem tipo **`PyMethod_Type`**). O parâmetro não deve ser `NULL`. Esta função sempre tem sucesso.

PyObject *PyMethod_New(PyObject *func, PyObject *self)

Retorna valor: Nova referência.

Retorna um novo objeto de método, com *func* sendo qualquer objeto chamável e *self* a instância à qual o método deve ser vinculado. *func* é a função que será chamada quando o método for chamado. *self* não deve ser `NULL`.

PyObject *PyMethod_Function(PyObject *meth)

Retorna valor: Referência emprestada.

Retorna o objeto função associado ao método *meth*.

PyObject *PyMethod_GET_FUNCTION(PyObject *meth)

Retorna valor: Referência emprestada.

Versão macro de **`PyMethod_Function()`** que evita a verificação de erros.

PyObject *PyMethod_Self(PyObject *meth)

Retorna valor: Referência emprestada.

Retorna a instância associada com o método *meth*.

PyObject *PyMethod_GET_SELF(PyObject *meth)

Retorna valor: Referência emprestada.

Versão macro de **`PyMethod_Self()`** que evita a verificação de erros.

Objeto célula

Objetos “cell” são usados para implementar variáveis referenciadas por múltiplos escopos. Para cada variável, um objeto célula é criado para armazenar o valor; as variáveis locais de cada quadro de pilha que referencia o valor contêm uma referência para as células de escopos externos que também usam essa variável. Quando o valor é acessado, o valor contido na célula é usado em vez do próprio objeto da célula. Essa des-referência do objeto da célula requer suporte do código de bytes gerado; estes não são automaticamente desprezados quando acessados. Objetos de células provavelmente não serão úteis em outro lugar.

type PyCellObject

A estrutura C usada para objetos célula.

PyTypeObject PyCell_Type

O objeto de tipo correspondente aos objetos célula.

int PyCell_Check(PyObject *ob)

Retorna verdadeiro se *ob* for um objeto célula; *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

PyObject *PyCell_New(PyObject *ob)

Retorna valor: Nova referência.

Cria e retorna um novo objeto célula contendo o valor *ob*. O parâmetro pode ser `NULL`.

PyObject *PyCell_Get(PyObject *cell)

Retorna valor: Nova referência.

Retorna o conteúdo da célula *cell*, que pode ser `NULL`. Se *cell* não for um objeto célula, retorna `NULL` com um conjunto de exceções.

PyObject *PyCell_GET(PyObject *cell)

Retorna valor: Referência emprestada.

Retorna o conteúdo da célula *cell*, mas sem verificar se *cell* não é `NULL` e um objeto célula.

int PyCell_Set(PyObject *cell, PyObject *value)

Define o conteúdo do objeto da célula *cell* como *value*. Isso libera a referência a qualquer conteúdo atual da célula. *value* pode ser `NULL`. *cell* não pode ser `NULL`.

Em caso de sucesso, retorna `0`. Se *cell* não for um objeto célula, define uma exceção e retorna `-1`.

void PyCell_SET(PyObject *cell, PyObject *value)

Define o valor do objeto da célula *cell* como *value*. Nenhuma contagem de referência é ajustada e nenhuma verificação é feita quanto à segurança; *cell* não pode ser `NULL` e deve ser um objeto célula.

Objetos código

Os objetos código são um detalhe de baixo nível da implementação do CPython. Cada um representa um pedaço de código executável que ainda não foi vinculado a uma função.

type PyCodeObject

A estrutura C dos objetos usados para descrever objetos código. Os campos deste tipo estão sujeitos a alterações a qualquer momento.

PyTypeObject PyCode_Type

Esta é uma instância de **`PyTypeObject`** representando o objeto código Python.

int PyCode_Check(PyObject *co)

Retorna verdadeiro se *co* for um objeto código. Esta função sempre tem sucesso.

Py_ssize_t PyCode_GetNumFree(PyCodeObject *co)

Retorna o número de variáveis livres (de clausura) em um objeto código.

int PyUnstable_Code_GetFirstFree(PyCodeObject *co)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Retorna a posição da primeira variável livre (de clausura) em um objeto código.

Alterado na versão 3.13: Renomeado de `PyCode_GetFirstFree` como parte da API C Instável. O nome antigo foi descontinuado, mas continuará disponível até que a assinatura mude novamente.

PyCodeObject *PyUnstable_Code_New(int argcount, int kwonlyargcount, int nlocals, int stacksize, int flags, PyObject *code, PyObject *consts,

PyObject *names, PyObject *varnames, PyObject *freevars, PyObject *cellvars, PyObject *filename, PyObject *name, PyObject *qualname, int firstlineno, PyObject *linetable, PyObject *exceptiontable)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Devolve um novo objeto código. Se precisar de um objeto código vazio para criar um quadro, use **PyCode_NewEmpty()**

Como a definição de bytecode muda constantemente, chamar **PyUnstable_Code_New()** diretamente pode vinculá-lo a uma versão de Python específica.

Os vários argumentos desta função são inter-dependentes de maneiras complexas, significando que mesmo alterações discretas de valor tem chances de resultar em execução incorreta ou erros fatais de VM. Tenha extremo cuidado ao usar esta função.

Alterado na versão 3.11: Adicionou os parâmetros `qualname` e `exceptiontable`

Alterado na versão 3.12: Renomeado de `PyCode_New` como parte da API C Instável. O nome antigo foi descontinuado, mas continuará disponível até que a assinatura mude novamente.

PyCodeObject *PyUnstable_Code_NewWithPosOnlyArgs(int argcount, int posonlyargcount, int kwonlyargcount, int nlocals, int stacksize, int flags, PyObject *code, PyObject *consts, PyObject *names, PyObject *varnames, PyObject *freevars, PyObject *cellvars, PyObject *filename, PyObject *name, PyObject *qualname, int firstlineno, PyObject *linetable, PyObject *exceptiontable)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Similar a **PyUnstable_Code_New()**, mas com um “posonlyargcount” extra para argumentos somente-posicionais. As mesmas ressalvas que se aplicam a `PyUnstable_Code_New` também se aplicam a essa função.

Adicionado na versão 3.8: Como `PyCode_NewWithPosOnlyArgs`

Alterado na versão 3.11: Adicionados os parâmetros `qualname` e `exceptiontable`

Alterado na versão 3.12: Renomeado para `PyUnstable_Code_NewWithPosOnlyArgs`. O nome antigo foi descontinuado, mas continuará disponível até que a assinatura mude novamente.

PyCodeObject *PyCode_NewEmpty(const char *filename, const char *funcname, int firstlineno)

Retorna valor: Nova referência.
Retorna um novo objeto código vazio com o nome de arquivo, nome da função e número da primeira linha especificados. O objeto código resultante irá levantar uma `Exception` se executado.

int PyCode_Addr2Line(PyCodeObject *co, int byte_offset)

Retorna o número da linha da instrução que ocorre em ou antes de `byte_offset` e termina depois disso. Se você só precisa do número da linha de um quadro, use **`PyFrame_GetLineNumber()`**.

Para iterar eficientemente sobre os números de linha em um objeto código, use **a API descrita na PEP 626** [https://peps.python.org/pep-0626/#out-of-process-debuggers-and-profilers] .

int PyCode_Addr2Location(PyObject *co, int byte_offset, int *start_line, int *start_column, int *end_line, int *end_column)

Define os ponteiros `int` passados para a linha do código-fonte e os números da coluna para a instrução em `byte_offset`. Define o valor como `0` quando as informações não estão disponíveis para nenhum elemento em particular.

Retorna `1` se a função for bem-sucedida e 0 caso contrário.

Adicionado na versão 3.11.

PyObject *PyCode_GetCode(PyCodeObject *co)

Equivalente ao código Python `getattr(co, 'co_code')`. Retorna uma referência forte a um **`PyBytesObject`** representando o bytecode em um objeto código. Em caso de erro, `NULL` é retornado e uma exceção é levantada.

Este `PyBytesObject` pode ser criado sob demanda pelo interpretador e não representa necessariamente o bytecode realmente executado pelo CPython. O caso de uso primário para esta função são depuradores e criadores de perfil.

Adicionado na versão 3.11.

PyObject *PyCode_GetVarnames(PyCodeObject *co)

Equivalente ao código Python `getattr(co, 'co_varnames')`. Retorna uma nova referência a um **`PyTupleObject`** contendo os nomes das variáveis locais. Em caso de erro, `NULL` é retornado e uma exceção é levantada.

Adicionado na versão 3.11.

PyObject *PyCode_GetCellvars(PyCodeObject *co)

Equivalente ao código Python `getattr(co, 'co_cellvars')`. Retorna uma nova referência a um **`PyTupleObject`** contendo os nomes das variáveis locais referenciadas por funções aninhadas. Em caso de erro, `NULL` é retornado e uma exceção é levantada.

Adicionado na versão 3.11.

PyObject *PyCode_GetFreevars(PyCodeObject *co)

Equivalente ao código Python `getattr(co, 'co_freevars')`. Retorna uma nova referência a um **`PyTupleObject`** contendo os nomes das variáveis livres (de clausura). Em caso de erro, `NULL` é retornado e uma exceção é levantada.

Adicionado na versão 3.11.

int PyCode_AddWatcher(PyCode_WatchCallback callback)

Registra *callback* como um observador do objeto código para o interpretador atual. Devolve um ID que pode ser passado para **`PyCode_ClearWatcher()`**. Em caso de erro (por exemplo, não há IDs de observadores disponíveis), devolve `-1` e define uma exceção.

Adicionado na versão 3.12.

int PyCode_ClearWatcher(int watcher_id)

Libera o observador identificado por *watcher_id* anteriormente retornado por **`PyCode_AddWatcher()`** para o interpretador atual. Retorna `0` em caso de sucesso ou `-1` em caso de erro e levanta uma exceção (ex., se o *watcher_id* dado não foi registrado.)

Adicionado na versão 3.12.

type PyCodeEvent

Enumeração dos possíveis eventos de observador do objeto código: `PY_CODE_EVENT_CREATE` - `PY_CODE_EVENT_DESTROY`

Adicionado na versão 3.12.

typedef int (*PyCode_WatchCallback)(PyCodeEvent event, PyCodeObject *co)

Tipo de uma função de callback de observador de objeto código.

Se *evento* é `PY_CODE_EVENT_CREATE`, então a função de retorno é invocada após *co* ter sido completamente inicializado. Senão, a função de retorno é invocada antes que a destruição de *co* ocorra, para que o estado anterior de *co* possa ser inspecionado.

Se *evento* for `PY_CODE_EVENT_DESTROY`, obter uma referência para a função de retorno do objeto-a-ser-destruído irá reativá-lo e impedirá que o objeto seja liberado. Quando o objeto reativado é posteriormente destruído, qualquer observador de funções de retorno ativos naquele momento serão chamados novamente.

Usuários desta API não devem depender de detalhes internos de implementação em tempo de execução. Tais detalhes podem incluir, mas não estão limitados a: o ordem e o momento exatos da criação e destruição de objetos código. Enquanto alterações nestes detalhes podem resultar em diferenças que são visíveis para os observadores (incluindo se uma função de retorno é chamada ou não), isso não muda a semântica do código Python executado.

Se a função de retorno definir uma exceção, ela deverá retornar `-1`; essa exceção será impressa como uma exceção não reprovável usando `PyErr_WriteUnraisable()`. Caso contrário, deverá retornar `0`.

É possível que já exista uma exceção pendente definida na entrada da função de retorno. Nesse caso, a função de retorno deve retornar `0` com a mesma exceção ainda definida. Isso significa que a função de retorno não pode chamar nenhuma outra API que possa definir uma exceção, a menos que salve e limpe o estado da exceção primeiro e o restaure antes de retornar.

Adicionado na versão 3.12.

PyObject *PyCode_Optimize(PyObject *code, PyObject *consts, PyObject *names, PyObject *lnotab_obj)

Esta é uma função que faz nada.

Antes do Python 3.10, essa função realizava otimizações básicas em um objeto código.

Alterado na versão 3.10: Esta função agora não faz nada.

Suavemente descontinuado desde a versão 3.13.

Sinalizadores de objetos código

Objetos código contêm um campo de bits de sinalizadores, que podem ser recuperados como o atributo Python `co_flags` (por exemplo, usando

`PyObject_GetAttrString()`) e definidos usando um argumento *flags* para `PyUnstable_Code_New()` e funções semelhantes.

Os sinalizadores cujos nomes começam com `CO_FUTURE_` correspondem a recursos normalmente selecionáveis por instruções future. Esses sinalizadores podem ser usados em `PyCompilerFlags.cf_flags`. Observe que muitos sinalizadores `CO_FUTURE_` são obrigatórios nas versões atuais do Python, e defini-los não tem efeito.

Os seguintes sinalizador estão disponíveis. Para saber o significado delas, consulte a documentação vinculada de seus equivalentes em Python.

Sinalizador	**Significado**
CO_OPTIMIZED	`inspect.CO_OPTIMIZED`
CO_NEWLOCALS	`inspect.CO_NEWLOCALS`
CO_VARARGS	`inspect.CO_VARARGS`
CO_VARKEYWORDS	`inspect.CO_VARKEYWORDS`
CO_NESTED	`inspect.CO_NESTED`
CO_GENERATOR	`inspect.CO_GENERATOR`
CO_COROUTINE	`inspect.CO_COROUTINE`
CO_ITERABLE_COROUTINE	`inspect.CO_ITERABLE_COROUTINE`

Sinalizador	Significado
CO_ASYNC_GENERATOR	`inspect.CO_ASYNC_GENERATOR`
CO_HAS_DOCSTRING	`inspect.CO_HAS_DOCSTRING`
CO_METHOD	`inspect.CO_METHOD`
CO_FUTURE_DIVISION	sem efeito (`__future__.division`)
CO_FUTURE_ABSOLUTE_IMPORT	sem efeito (`__future__.absolute_import`)
CO_FUTURE_WITH_STATEMENT	sem efeito (`__future__.with_statement`)
CO_FUTURE_PRINT_FUNCTION	sem efeito (`__future__.print_function`)
CO_FUTURE_UNICODE_LITERALS	sem efeito (`__future__.unicode_literals`)
CO_FUTURE_GENERATOR_STOP	sem efeito (`__future__.generator_stop`)
CO_FUTURE_ANNOTATIONS	`__future__.annotations`

Informação adicional

Para oferece suporte a extensões de baixo nível de avaliação de quadro (frame), tais como compiladores “just-in-time”, é possível anexar dados arbitrários adicionais a objetos código.

Estas funções são parte da camada instável da API C: Essa funcionalidade é um detalhe de implementação do CPython, e a API pode mudar sem avisos de descontinuidade.

Py_ssize_t PyUnstable_Eval_RequestCodeExtraIndex(freefunc free)

> *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*

Retorna um novo e opaco valor de índice usado para adicionar dados a objetos código.

Geralmente, você chama esta função apenas uma vez (por interpretador) e usa o resultado com `PyCode_GetExtra` e `PyCode_SetExtra` para manipular dados em objetos código individuais.

Se *free* não for `NULL`: quando o objeto código é desalocado, *free* será chamado em dados não-`NULL` armazenados sob o novo índice. Use **`Py_DecRef()`** quando armazenar **`PyObject`**.

Adicionado na versão 3.6: como `_PyEval_RequestCodeExtraIndex`

Alterado na versão 3.12: Renomeado para `PyUnstable_Eval_RequestCodeExtraIndex`. O nome antigo nome privado foi descontinuado, mas continuará disponível até a mudança da API.

int PyUnstable_Code_GetExtra(PyObject *code, Py_ssize_t index, void **extra)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Define *extra* para os dados adicionais armazenados sob o novo índice dado. Retorna 0 em caso de sucesso. Define uma exceção e retorna -1 em caso de erro.

Se nenhum dado foi determinado sob o índice, define *extra* como `NULL` e retorna 0 sem definir nenhuma exceção.

Adicionado na versão 3.6: como `_PyCode_GetExtra`

Alterado na versão 3.12: Renomeado para `PyUnstable_Code_GetExtra`. O nome antigo privado foi descontinuado, mas continuará disponível até a mudança da API.

int PyUnstable_Code_SetExtra(PyObject *code, Py_ssize_t index, void *extra)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Define os dados extras armazenados sob o índice dado a *extra*. Retorna 0 em caso de sucesso. Define uma exceção e retorna -1 em caso de erro.

Adicionado na versão 3.6: como `_PyCode_SetExtra`

Alterado na versão 3.12: Renomeado para `PyUnstable_Code_SetExtra`. O nome antigo privado foi descontinuado, mas continuará disponível até a mudança da API.

File objects

Essas APIs são uma emulação mínima da API C do Python 2 para objetos arquivo embutidos, que costumavam depender do suporte de E/S em buffer (FILE*) da biblioteca C padrão. No Python 3, arquivos e streams usam o novo módulo **`io`**, que define várias camadas sobre a E/S sem buffer de baixo nível do sistema operacional. As funções descritas a seguir são invólucros de conveniência para o C sobre essas novas APIs e são destinadas principalmente para relatórios de erros internos no interpretador; código de terceiros é recomendado para acessar as APIs de **`io`**.

PyObject *PyFile_FromFd(int fd, const char *name, const char *mode, int buffering, const char *encoding, const char *errors, const char *newline, int closefd)

Retorna valor: Nova referência. Parte da ABI Estável.
Cria um objeto arquivo Python a partir do descritor de arquivo de um arquivo já aberto *fd*. Os argumentos *name*, *encoding*, *errors* and *newline* podem ser `NULL` para usar os padrões; *buffering* pode ser *-1* para usar o padrão. *name* é ignorado e mantido para compatibilidade com versões anteriores. Retorna `NULL` em caso de falha. Para uma descrição mais abrangente dos argumentos, consulte a documentação da função **`io.open()`**.

Aviso

Como os streams do Python têm sua própria camada de buffer, combiná-los com os descritores de arquivo no nível do sistema operacional pode produzir vários problemas (como ordenação inesperada de dados).

Alterado na versão 3.2: Ignora atributo *name*.

int PyObject_AsFileDescriptor(PyObject *p)

Parte da ABI Estável.

Retorna o descritor de arquivo associado a *p* como um int. Se o objeto for um inteiro, seu valor será retornado. Caso contrário, o método **`fileno()`** do objeto será chamado se existir; o método deve retornar um inteiro, que é retornado como o valor do descritor de arquivo. Define uma exceção e retorna `-1` em caso de falha.

PyObject *PyFile_GetLine(PyObject *p, int n)

Retorna valor: Nova referência. Parte da ABI Estável.

Equivalente a `p.readline([n])`, esta função lê uma linha do objeto *p*. *p* pode ser um objeto arquivo ou qualquer objeto com um método **`readline()`**. Se *n* for `0`, exatamente uma linha é lida, independentemente do comprimento da linha. Se *n* for maior que `0`, não mais do que *n* bytes serão lidos do arquivo; uma linha parcial pode ser retornada. Em ambos os casos, uma string vazia é retornada se o final do arquivo for alcançado imediatamente. Se *n* for menor que `0`, entretanto, uma linha é lida independentemente do comprimento, mas **`EOFError`** é levantada se o final do arquivo for alcançado imediatamente.

int PyFile_SetOpenCodeHook(Py_OpenCodeHookFunction handler)

Substitui o comportamento normal de **`io.open_code()`** para passar seu parâmetro por meio do manipulador fornecido.

O *handler* é uma função do tipo:

typedef PyObject *(*Py_OpenCodeHookFunction)(PyObject*, void*)

Equivalente de PyObject *(*)(PyObject *path, void *userData), sendo *path* garantido como sendo **`PyUnicodeObject`**.

O ponteiro *userData* é passado para a função de gancho. Como as funções de gancho podem ser chamadas de diferentes tempos de execução, esse ponteiro não deve se referir diretamente ao estado do Python.

Como este gancho é usado intencionalmente durante a importação, evite importar novos módulos durante sua execução, a menos que eles estejam congelados ou disponíveis em `sys.modules`.

Uma vez que um gancho foi definido, ele não pode ser removido ou substituído, e chamadas posteriores para **PyFile_SetOpenCodeHook()** irão falhar. Em caso de falha, a função retorna -1 e define uma exceção se o interpretador foi inicializado.

É seguro chamar esta função antes **Py_Initialize()**.

Levanta um evento de auditoria `setopencodehook` sem argumentos.

Adicionado na versão 3.8.

PyObject *PyFile_OpenCodeObject(PyObject *path)

Abre o arquivo *path* com o modo `'rb'`. *path* deve ser um objeto Python da classe **str**. O comportamento desta função pode ser substituído por **PyFile_SetOpenCodeHook()** para permitir algum pré-processamento do texto.

Isso é análogo a **io.open_code()** no Python.

Em caso de sucesso, esta função retorna uma referência forte a um objeto arquivo Python. Em caso de falha, esta função retorna `NULL` com uma exceção definida.

Adicionado na versão 3.8.

PyObject *PyFile_OpenCode(const char *path)

Semelhante a **PyFile_OpenCodeObject()**, mas *path* é um const char* codificado em UTF-8.

Adicionado na versão 3.8.

int PyFile_WriteObject(PyObject *obj, PyObject *p, int flags)

Parte da ABI Estável.

Write object *obj* to file object *p*. The only supported flag for *flags* is **Py_PRINT_RAW**; if given, the **str()** of the object is written instead of the **repr()**.

If *obj* is `NULL`, write the string `"<NULL>"`.

Return `0` on success or `-1` on failure; the appropriate exception will be set.

int PyFile_WriteString(const char *s, PyObject *p)

Parte da ABI Estável.

Escreve a string *s* no objeto arquivo *p*. Retorna `0` em caso de sucesso ou `-1` em caso de falha; a exceção apropriada será definida.

Soft-deprecated API

Suavemente descontinuado desde a versão 3.15.

These are APIs that were included in Python's C API by mistake. They are documented solely for completeness; use other `PyFile*` APIs instead.

PyObject *PyFile_NewStdPrinter(int fd)

Use **`PyFile_FromFd()`** with defaults (`fd, NULL, "w", -1, NULL, NULL, NULL, 0`) instead.

PyTypeObject PyStdPrinter_Type

Type of file-like objects used internally at Python startup when **`io`** is not yet available. Use Python **`open()`** or **`PyFile_FromFd()`** to create file objects instead.

Objetos do Módulo

PyTypeObject PyModule_Type

Parte da ABI Estável.

This instance of `PyTypeObject` represents the Python module type. This is exposed to Python programs as `types.ModuleType`.

int PyModule_Check(PyObject *p)

Retorna true se *p* for um objeto de módulo ou um subtipo de um objeto de módulo. Esta função sempre é bem-sucedida.

int PyModule_CheckExact(PyObject *p)

Retorna true se *p* for um objeto de módulo, mas não um subtipo de `PyModule_Type`. Essa função é sempre bem-sucedida.

PyObject *PyModule_NewObject(PyObject *name)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Retorna um novo objeto de módulo com `module.__name__` definido como *name*. Os atributos `__name__`, `__doc__`, `__package__` e `__loader__` do módulo são preenchidos (todos, exceto `__name__`, são definidos como `None`). O chamador é responsável por definir um atributo `__file__`.

Retorna `NULL` com uma exceção definida em caso de erro.

Adicionado na versão 3.3.

Alterado na versão 3.4: `__package__` e `__loader__` agora estão definidos como `None`.

PyObject *PyModule_New(const char *name)

Retorna valor: Nova referência. Parte da ABI Estável.

Semelhante a `PyModule_NewObject()`, mas o nome é uma string codificada em UTF-8 em vez de um objeto Unicode.

PyObject *PyModule_GetDict(PyObject *module)

Retorna valor: Referência emprestada. Parte da ABI Estável.
Retorna o objeto dicionário que implementa o espaço de nomes de *module*; este objeto é o mesmo que o atributo `__dict__` do objeto de módulo. Se *module* não for um objeto de módulo (ou um subtipo de um objeto de módulo), `SystemError` é levantada e `NULL` é retornado.

É recomendado que as extensões usem outras funções `PyModule_*` e `PyObject_*` em vez de manipular diretamente o `__dict__` de um módulo.

The returned reference is borrowed from the module; it is valid until the module is destroyed.

PyObject *PyModule_GetNameObject(PyObject *module)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.
Retorna o valor `__name__` do *module*. Se o módulo não fornecer um, ou se não for uma string, `SystemError` é levantada e `NULL` é retornado.

Adicionado na versão 3.3.

const char *PyModule_GetName(PyObject *module)

Parte da ABI Estável.
Semelhante a `PyModule_GetNameObject()` mas retorna o nome codificado em `'utf-8'`

The returned buffer is only valid until the module is renamed or destroyed. Note that Python code may rename a module by setting its `__name__` attribute.

PyModuleDef *PyModule_GetDef(PyObject *module)

Parte da ABI Estável.

Retorna um ponteiro para a estrutura `PyModuleDef` da qual o módulo foi criado, ou `NULL` se o módulo não foi criado de uma definição.

On error, return `NULL` with an exception set. Use `PyErr_Occurred()` to tell this case apart from a missing `PyModuleDef`.

PyObject *PyModule_GetFilenameObject(PyObject *module)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna o nome do arquivo do qual o *module* foi carregado usando o atributo `__file__` do *module*. Se não estiver definido, ou se não for uma string unicode, levanta `SystemError` e retorna `NULL`; Caso contrário, retorna uma referência a um objeto Unicode.

Adicionado na versão 3.2.

const char *PyModule_GetFilename(PyObject *module)

Parte da ABI Estável.
Semelhante a `PyModule_GetFilenameObject()` mas retorna o nome do arquivo codificado em 'utf-8'.

The returned buffer is only valid until the module's `__file__` attribute is reassigned or the module is destroyed.

Descontinuado desde a versão 3.2: `PyModule_GetFilename()`: levanta `UnicodeEncodeError` quando há nomes de arquivos não codificáveis, use `PyModule_GetFilenameObject()`.

Module definition

Modules created using the C API are typically defined using an array of `PySlot` structs, which provides a "description" of how a module should be created. See Definition slots for more information on slots in general.

Alterado na versão 3.15: Previously, a `PyModuleDef` struct was necessary to define modules. The older way of defining modules is still available: consult either the Module definition struct section or earlier versions of this documentation if you plan to support earlier Python versions.

The slots array is usually used to define an extension module's "main" module object (see Definindo módulos de extensão for details). It can also be used to create extension modules dynamically.

Unless specified otherwise, the same slot ID may not be repeated in an array of slots.

Metadata slots

Py_mod_name

Parte da ABI Estável desde a versão 3.15.
`Slot ID` for the name of the new module, as a NUL-terminated UTF8-encoded `const char *`.

Note that modules are typically created using a **`ModuleSpec`**, and when they are, the name from the spec will be used instead of **`Py_mod_name`**. However, it is still recommended to include this slot for introspection and debugging purposes.

Adicionado na versão 3.15: Use **`PyModuleDef.m_name`** instead to support previous versions.

Py_mod_doc

Parte da ABI Estável desde a versão 3.15.
`Slot ID` for the docstring of the new module, as a NUL-terminated UTF8-encoded `const char *`.

Usually it is set to a variable created with **`PyDoc_STRVAR`**.

Adicionado na versão 3.15: Use **`PyModuleDef.m_doc`** instead to support previous versions.

Feature slots

Py_mod_abi

Parte da ABI Estável desde a versão 3.15.

Slot ID whose value points to a **PyABIInfo** structure describing the ABI that the extension is using.

A suitable **PyABIInfo** variable can be defined using the **PyABIInfo_VAR** macro, as in:

```
PyABIInfo_VAR(abi_info);

static PySlot mymodule_slots[] = {
   PySlot_DATA(Py_mod_abi, &abi_info),
   ...
};
```

When creating a module, Python checks the value of this slot using **PyABIInfo_Check()**.

This slot is required, except for modules created from **PyModuleDef**.

Adicionado na versão 3.15.

Py_mod_multiple_interpreters

Parte da ABI Estável desde a versão 3.12.

Slot ID whose value is one of:

Py_MOD_MULTIPLE_INTERPRETERS_NOT_SUPPORTED

The module does not support being imported in subinterpreters.

Py_MOD_MULTIPLE_INTERPRETERS_SUPPORTED

The module supports being imported in subinterpreters, but only when they share the main interpreter's GIL. (See Isolando módulos de extensão.)

Py_MOD_PER_INTERPRETER_GIL_SUPPORTED

The module supports being imported in subinterpreters, even when they have their own GIL. (See Isolando módulos de extensão.)

This slot determines whether or not importing this module in a subinterpreter will fail.

If `Py_mod_multiple_interpreters` is not specified, the import machinery defaults to `Py_MOD_MULTIPLE_INTERPRETERS_SUPPORTED`.

For historical reasons, the values are declared as pointers (`void *`). When using **`PySlot`** arrays, use **`PySlot_DATA`** for **`Py_mod_multiple_interpreters`**:

```
PySlot_DATA(Py_mod_multiple_interpreters,
            Py_MOD_PER_INTERPRETER_GIL_SUPPORTED)
```

Adicionado na versão 3.12.

Py_mod_gil

Parte da ABI Estável desde a versão 3.13.

`Slot_ID` whose value is one of:

Py_MOD_GIL_USED

The module depends on the presence of the global interpreter lock (GIL), and may access global state without synchronization.

Py_MOD_GIL_NOT_USED

The module is safe to run without an active GIL.

This slot is ignored by Python builds not configured with **`--disable-gil`**. Otherwise, it determines whether or not importing this module will cause the GIL to be automatically enabled. See CPython com threads livres for more detail.

If `Py_mod_gil` is not specified, the import machinery defaults to `Py_MOD_GIL_USED`.

For historical reasons, the values are declared as pointers (`void *`). When using **`PySlot`** arrays, use **`PySlot_DATA`** for **`Py_mod_gil`**:

```
PySlot_DATA(Py_mod_gil, Py_MOD_GIL_NOT_USED)
```

Adicionado na versão 3.13.

Creation and initialization slots

Py_mod_create

Parte da ABI Estável desde a versão 3.5.

Slot ID for a function that creates the module object itself. The function must have the signature:

PyObject *create_module(PyObject *spec, PyModuleDef *def)

The function will be called with:

- *spec*: a `ModuleSpec`-like object, meaning that any attributes defined for **importlib.machinery.ModuleSpec** have matching semantics. However, any of the attributes may be missing.
- *def*: `NULL`, or the module definition if the module is created from one.

The function should return a new module object, or set an error and return `NULL`.

This function should be kept minimal. In particular, it should not call arbitrary Python code, as trying to import the same module again may result in an infinite loop.

If `Py_mod_create` is not specified, the import machinery will create a normal module object using **PyModule_New()**. The name is taken from *spec*, not the definition, to allow extension modules to dynamically adjust to their place in the module hierarchy and be imported under different names through symlinks, all while sharing a single module definition.

There is no requirement for the returned object to be an instance of **PyModule_Type**. However, some slots may only be used with **PyModule_Type** instances; in particular:

- **Py_mod_exec**,
- module state slots (`Py_mod_state_*`),
- **Py_mod_token**.

Adicionado na versão 3.5.

Alterado na versão 3.15: The *slots* argument may be a `ModuleSpec`-like object, rather than a true **`ModuleSpec`** instance. Note that previous versions of CPython did not enforce this.

The *def* argument may now be `NULL`, since modules are not necessarily made from definitions.

Py_mod_exec

Parte da ABI Estável desde a versão 3.5.
`Slot ID` for a function that will *execute*, or initialize, the module. This function does the equivalent to executing the code of a Python module: typically, it adds classes and constants to the module. The signature of the function is:

int exec_module(PyObject *module)

See the Support functions section for some useful functions to call.

For backwards compatibility, the **`PyModuleDef.m_slots`** array may contain multiple **`Py_mod_exec`** slots; these are processed in the order they appear in the array. Elsewhere (that is, in arguments to **`PyModule_FromSlotsAndSpec()`** and in return values of `PyModExport_<name>`), repeating the slot is not allowed.

Adicionado na versão 3.5.

Alterado na versão 3.15: Repeated `Py_mod_exec` slots are disallowed, except in **`PyModuleDef.m_slots`**.

Py_mod_methods

Parte da ABI Estável desde a versão 3.15.
`Slot ID` for a table of module-level functions, as an array of **`PyMethodDef`** values suitable as the *functions* argument to **`PyModule_AddFunctions()`**.

Like other slot IDs, a slots array may only contain one **`Py_mod_methods`** entry. To add functions from multiple **`PyMethodDef`** arrays, call **`PyModule_AddFunctions()`** in the **`Py_mod_exec`** function.

The table must be statically allocated (or otherwise guaranteed to outlive the module object).

Adicionado na versão 3.15: Use `PyModuleDef.m_methods` instead to support previous versions.

Module state

Extension modules can have *module state* – a piece of memory that is allocated on module creation, and freed when the module object is deallocated. The module state is specified using dedicated slots.

A typical use of module state is storing an exception type – or indeed *any* type object defined by the module –

Unlike the module's Python attributes, Python code cannot replace or delete data stored in module state.

Keeping per-module information in attributes and module state, rather than in static globals, makes module objects *isolated* and safer for use in multiple sub-interpreters. It also helps Python do an orderly clean-up when it shuts down.

Extensions that keep references to Python objects as part of module state must implement `Py_mod_state_traverse` and `Py_mod_state_clear` functions to avoid reference leaks.

To retrieve the state from a given module, use the following functions:

void *PyModule_GetState(PyObject *module)

Parte da ABI Estável.
Return the "state" of the module, that is, a pointer to the block of memory allocated at module creation time, or `NULL`. See `Py_mod_state_size`.

On error, return `NULL` with an exception set. Use `PyErr_Occurred()` to tell this case apart from missing module state.

int PyModule_GetStateSize(PyObject *module, Py_ssize_t *result)

Parte da ABI Estável desde a versão 3.15.

Set **result* to the size of *module*'s state, as specified using `Py_mod_state_size` (or `PyModuleDef.m_size`), and return 0.

On error, set **result* to -1, and return -1 with an exception set.

Adicionado na versão 3.15.

Slots for defining module state

The following `slot IDs` are available for defining the module state.

Py_mod_state_size

Parte da ABI Estável desde a versão 3.15.

`Slot ID` for the size of the module state, in bytes.

Setting the value to a non-negative value means that the module can be re-initialized and specifies the additional amount of memory it requires for its state.

Ver **PEP 3121** [https://peps.python.org/pep-3121/] para mais detalhes.

Use `PyModule_GetStateSize()` to retrieve the size of a given module.

Adicionado na versão 3.15: Use `PyModuleDef.m_size` instead to support previous versions.

Py_mod_state_traverse

Parte da ABI Estável desde a versão 3.15.

`Slot ID` for a traversal function to call during GC traversal of the module object.

The signature of the function, and meanings of the arguments, is similar as for `PyTypeObject.tp_traverse`:

int traverse_module_state(PyObject *module, visitproc visit, void *arg)

This function is not called if the module state was requested but is not allocated yet. This is the case immediately after the module is created and before the module is executed (`Py_mod_exec` function). More precisely, this function is not called if the state size (`Py_mod_state_size`) is greater than 0 and the module state (as returned by `PyModule_GetState()`) is `NULL`.

Adicionado na versão 3.15: Use `PyModuleDef.m_traverse` instead to support previous versions.

Py_mod_state_clear

Parte da ABI Estável desde a versão 3.15.
`Slot_ID` for a clear function to call during GC clearing of the module object.

The signature of the function is:

int clear_module_state(PyObject *module)

This function is not called if the module state was requested but is not allocated yet. This is the case immediately after the module is created and before the module is executed (`Py_mod_exec` function). More precisely, this function is not called if the state size (`Py_mod_state_size`) is greater than 0 and the module state (as returned by `PyModule_GetState()`) is `NULL`.

Like `PyTypeObject.tp_clear`, this function is not *always* called before a module is deallocated. For example, when reference counting is enough to determine that an object is no longer used, the cyclic garbage collector is not involved and the `Py_mod_state_free` function is called directly.

Adicionado na versão 3.15: Use `PyModuleDef.m_clear` instead to support previous versions.

Py_mod_state_free

Parte da ABI Estável desde a versão 3.15.
`Slot_ID` for a function to call during deallocation of the module object.

The signature of the function is:

int free_module_state(PyObject *module)

This function is not called if the module state was requested but is not allocated yet. This is the case immediately after the module is created and before the module is executed (`Py_mod_exec` function). More precisely, this function is not called if the state size (`Py_mod_state_size`) is greater than 0 and the module state (as returned by `PyModule_GetState()`) is `NULL`.

Adicionado na versão 3.15: Use `PyModuleDef.m_free` instead to support previous versions.

Module token

Each module may have an associated *token*: a pointer-sized value intended to identify of the module state's memory layout. This means that if you have a module object, but you are not sure if it "belongs" to your extension, you can check using code like this:

```
PyObject *module = <the module in question>

void *module_token;
if (PyModule_GetToken(module, &module_token) < 0) {
    return NULL;
}
if (module_token != your_token) {
    PyErr_SetString(PyExc_ValueError, "unexpected module")
    return NULL;
}

// This module's state has the expected memory layout; it's safe
struct my_state state = (struct my_state*)PyModule_GetState(modu
```

A module's token – and the *your_token* value to use in the above code – is:

- For modules created with `PyModuleDef`: the address of that **`PyModuleDef`**;

- For modules defined with the `Py_mod_token` slot: the value of that slot;
- For modules created from an `PyModExport_*` export hook: the slots array that the export hook returned (unless overridden with `Py_mod_token`).

Py_mod_token

Parte da ABI Estável desde a versão 3.15.

`Slot_ID` for the module token.

If you use this slot to set the module token (rather than rely on the default), you must ensure that:

- The pointer outlives the class, so it's not reused for something else while the class exists.
- It "belongs" to the extension module where the class lives, so it will not clash with other extensions.
- If the token points to a `PyModuleDef` struct, the module should behave as if it was created from that **`PyModuleDef`**. In particular, the module state must have matching layout and semantics.

Modules created from `PyModuleDef` always use the address of the **`PyModuleDef`** as the token. This means that **`Py_mod_token`** cannot be used in `PyModuleDef.m_slots`.

Adicionado na versão 3.15.

int PyModule_GetToken(PyObject *module, void **result)

Parte da ABI Estável desde a versão 3.15.

Set **result* to the module token for *module* and return 0.

On error, set **result* to NULL, and return -1 with an exception set.

Adicionado na versão 3.15.

See also `PyType_GetModuleByToken()`.

Creating extension modules dynamically

The following functions may be used to create an extension module dynamically, rather than from an extension's export hook.

PyObject *PyModule_FromSlotsAndSpec(const PySlot *slots, PyObject *spec)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.15.

Create a new module object, given an array of slots and the **`ModuleSpec`** *spec*.

The *slots* argument must point to an array of **`PySlot`** structures, terminated by an entry with slot ID of 0 (typically written as **`PySlot_END`**). The array must include a **`Py_mod_abi`** entry.

The *spec* argument may be any `ModuleSpec`-like object, as described in **`Py_mod_create`** documentation. Currently, the *spec* must have a `name` attribute.

On success, return the new module. On error, return `NULL` with an exception set.

Note that this does not process the module's execution slot (**`Py_mod_exec`**). Both **`PyModule_FromSlotsAndSpec()`** and **`PyModule_Exec()`** must be called to fully initialize a module. (See also Inicialização multifásica.)

Adicionado na versão 3.15.

int PyModule_Exec(PyObject *module)

Parte da ABI Estável desde a versão 3.15.

Execute the **`Py_mod_exec`** slot(s) of *module*.

On success, return 0. On error, return -1 with an exception set.

For clarity: If *module* has no slots, for example if it uses legacy single-phase initialization, this function does nothing and returns 0.

Adicionado na versão 3.15.

Module definition struct

Traditionally, extension modules were defined using a *module definition* as the “description” of how a module should be created. Rather than using an array of slots directly, the definition has dedicated members for most common functionality, and allows additional slots as an extension mechanism.

This way of defining modules is still available and there are no plans to remove it.

type PyModuleDef

Parte da ABI Estável (see below).
The module definition struct, which holds information needed to create a module object.

This structure must be statically allocated (or be otherwise guaranteed to be valid while any modules created from it exist). Usually, there is only one variable of this type for each extension module defined this way.

The struct, including all members, is part of the Stable ABI for non-free-threaded builds (`abi3`). In the Stable ABI for free-threaded builds (`abi3t`), this struct is opaque, and unusable in practice; see Module definition for a replacement.

PyModuleDef_Base m_base

Always initialize this member to **`PyModuleDef_HEAD_INIT`**:

type PyModuleDef_Base

Parte da ABI Estável (see below).
The type of **`PyModuleDef.m_base`**.

The struct is part of the Stable ABI for non-free-threaded builds (`abi3`). In the Stable ABI for Free-Threaded Builds (`abi3t`), this struct is opaque, and unusable in practice.

PyModuleDef_HEAD_INIT

The required initial value for **`PyModuleDef.m_base`**.

const char *m_name

Corresponds to the **`Py_mod_name`** slot.

const char *m_doc

These members correspond to the **`Py_mod_doc`** slot. Setting this to NULL is equivalent to omitting the slot.

Py_ssize_t m_size

Corresponds to the **`Py_mod_state_size`** slot. Setting this to zero is equivalent to omitting the slot.

When using legacy single-phase initialization or when creating modules dynamically using **`PyModule_Create()`** or **`PyModule_Create2()`**, **`m_size`** may be set to -1. This indicates that the module does not support sub-interpreters, because it has global state.

PyMethodDef *m_methods

Corresponds to the **`Py_mod_methods`** slot. Setting this to NULL is equivalent to omitting the slot.

PyModuleDef_Slot *m_slots

An array of additional slots, terminated by a `{0, NULL}` entry. Note that the entries use the older **`PyModuleDef_Slot`** structure, rather than **`PySlot`**.

If the array contains slots corresponding to **`PyModuleDef`** members, the values must match. For example, if you use **`Py_mod_name`** in **`m_slots`**, **`PyModuleDef.m_name`** must be set to the same pointer (not just an equal string).

Alterado na versão 3.5: Antes da versão 3.5, esse membro era sempre definido como `NULL` e era definido como:

inquiry m_reload

type PyModuleDef_Slot

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.5.

Older structure defining additional slots of a module.

Note that a **PyModuleDef_Slot** array may be included in a **PySlot** array using **Py_mod_slots**, and vice versa using **Py_slot_subslots**.

Each **PyModuleDef_Slot** structure `modslot` is interpreted as the following **PySlot** structure:

```
(PySlot){
   .sl_id=modslot.slot,
   .sl_flags=PySlot_INTPTR | sub_static,
   .sl_ptr=modslot.value
}
```

where `sub_static` is `PySlot_STATIC` if the slot requires the flag (such as for **Py_mod_methods**), or if this flag is present on the "parent" **Py_mod_slots** slot (if any).

int slot

Corresponds to **PySlot.sl_id**.

void *value

Corresponds to **PySlot.sl_ptr**.

Adicionado na versão 3.5.

traverseproc m_traverse
inquiry m_clear
freefunc m_free

These members correspond to the **Py_mod_state_traverse**, **Py_mod_state_clear**, and **Py_mod_state_free** slots, respectively.

Setting these members to NULL is equivalent to omitting the corresponding slots.

Alterado na versão 3.9: **`m_traverse`**, **`m_clear`** and **`m_free`** functions are no longer called before the module state is allocated.

PyTypeObject PyModuleDef_Type

Parte da ABI Estável desde a versão 3.5.
The type of `PyModuleDef` objects.

Py_mod_slots

Parte da ABI Estável desde a versão 3.15.
`Slot_ID` that works like **`Py_slot_subslots`**, except it specifies an array of **`PyModuleDef_Slot`** structures.

Adicionado na versão 3.15.

The following API can be used to create modules from a **`PyModuleDef`** struct:

PyObject *PyModule_Create(PyModuleDef *def)

Retorna valor: Nova referência.
Create a new module object, given the definition in *def*. This is a macro that calls **`PyModule_Create2()`** with *module_api_version* set to **`PYTHON_API_VERSION`**, or to **`PYTHON_ABI_VERSION`** if using the limited API.

PyObject *PyModule_Create2(PyModuleDef *def, int module_api_version)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a new module object, given the definition in *def*, assuming the API version *module_api_version*. If that version does not match the version of the running interpreter, a **`RuntimeWarning`** is emitted.

Retorna `NULL` com uma exceção definida em caso de erro.

This function does not support slots. The `m_slots` member of *def* must be `NULL`.

Nota

A maioria dos usos dessa função deve ser feita com `PyModule_Create()`; use-o apenas se tiver certeza de que precisa.

PyObject *PyModule_FromDefAndSpec(PyModuleDef *def, PyObject *spec)

Retorna valor: Nova referência.

This macro calls `PyModule_FromDefAndSpec2()` with *module_api_version* set to `PYTHON_API_VERSION`, or to `PYTHON_ABI_VERSION` if using the limited API.

Adicionado na versão 3.5.

Suavemente descontinuado desde a versão 3.15: Prefer `PyModule_FromSlotsAndSpec()` in new code.

PyObject *PyModule_FromDefAndSpec2(PyModuleDef *def, PyObject *spec, int module_api_version)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Create a new module object, given the definition in *def* and the ModuleSpec *spec*, assuming the API version *module_api_version*. If that version does not match the version of the running interpreter, a `RuntimeWarning` is emitted.

Retorna `NULL` com uma exceção definida em caso de erro.

Note that this does not process execution slots (`Py_mod_exec`). Both `PyModule_FromDefAndSpec` and `PyModule_ExecDef` must be called to fully initialize a module.

Nota

Most uses of this function should be using `PyModule_FromDefAndSpec()` instead; only use this if you are sure you need it.

Adicionado na versão 3.5.

Suavemente descontinuado desde a versão 3.15: Prefer `PyModule_FromSlotsAndSpec()` in new code.

int PyModule_ExecDef(PyObject *module, PyModuleDef *def)

Parte da ABI Estável desde a versão 3.7.
Process any execution slots (`Py_mod_exec`) given in *def*.

Adicionado na versão 3.5.

Suavemente descontinuado desde a versão 3.15: To run a module's own execution slots, prefer `PyModule_Exec()`, which works on modules that were not created from a `PyModuleDef` structure.

PYTHON_API_VERSION
PYTHON_API_STRING

The C API version, as an integer (`1013`) and string (`"1013"`), respectively. Defined for backwards compatibility.

Atualmente, essa constante não é atualizada nas novas versões de Python e não é útil para controle de versão. Isso pode mudar no futuro.

PYTHON_ABI_VERSION
PYTHON_ABI_STRING

Defined as `3` and `"3"`, respectively, for backwards compatibility.

Atualmente, essa constante não é atualizada nas novas versões de Python e não é útil para controle de versão. Isso pode mudar no futuro.

Support functions

The following functions are provided to help initialize a module object. They are intended for a module's execution slot (**`Py_mod_exec`**), the initialization function for legacy single-phase initialization, or code that creates modules dynamically.

int PyModule_AddObjectRef(PyObject *module, const char *name, PyObject *value)

Parte da ABI Estável desde a versão 3.10.

Add an object to *module* as *name*. This is a convenience function which can be used from the module's initialization function.

On success, return `0`. On error, raise an exception and return `-1`.

Exemplo de uso:

```
static int
add_spam(PyObject *module, int value)
{
    PyObject *obj = PyLong_FromLong(value);
    if (obj == NULL) {
        return -1;
    }
    int res = PyModule_AddObjectRef(module, "spam", obj);
    Py_DECREF(obj);
    return res;
 }
```

To be convenient, the function accepts `NULL` *value* with an exception set. In this case, return `-1` and just leave the raised exception unchanged.

O exemplo também pode ser escrito sem verificar explicitamente se *obj* é `NULL`:

```
static int
add_spam(PyObject *module, int value)
{
    PyObject *obj = PyLong_FromLong(value);
    int res = PyModule_AddObjectRef(module, "spam", obj);
    Py_XDECREF(obj);
```

```
    return res;
}
```

Note that `Py_XDECREF()` should be used instead of `Py_DECREF()` in this case, since *obj* can be `NULL`.

The number of different *name* strings passed to this function should be kept small, usually by only using statically allocated strings as *name*. For names that aren't known at compile time, prefer calling **`PyUnicode_FromString()`** and **`PyObject_SetAttr()`** directly. For more details, see **`PyUnicode_InternFromString()`**, which may be used internally to create a key object.

Adicionado na versão 3.10.

int PyModule_Add(PyObject *module, const char *name, PyObject *value)

Parte da ABI Estável desde a versão 3.13.
Similar to **`PyModule_AddObjectRef()`**, but "steals" a reference to *value* (even on error). It can be called with a result of function that returns a new reference without bothering to check its result or even saving it to a variable.

Exemplo de uso:

```
if (PyModule_Add(module, "spam", PyBytes_FromString(value)) <
    goto error;
}
```

Adicionado na versão 3.13.

int PyModule_AddObject(PyObject *module, const char *name, PyObject *value)

Parte da ABI Estável.
Similar to **`PyModule_AddObjectRef()`**, but steals a reference to *value* on success (if it returns `0`).

The new **PyModule_Add()** or **PyModule_AddObjectRef()** functions are recommended, since it is easy to introduce reference leaks by misusing the **PyModule_AddObject()** function.

Nota

Unlike other functions that steal references, `PyModule_AddObject()` only releases the reference to *value* **on success**.

This means that its return value must be checked, and calling code must **Py_XDECREF()** *value* manually on error.

Exemplo de uso:

```
PyObject *obj = PyBytes_FromString(value);
if (PyModule_AddObject(module, "spam", obj) < 0) {
    // If 'obj' is not NULL and PyModule_AddObject() failed,
    // 'obj' strong reference must be deleted with Py_XDECREF
    // If 'obj' is NULL, Py_XDECREF() does nothing.
    Py_XDECREF(obj);
    goto error;
}
// PyModule_AddObject() stole a reference to obj:
// Py_XDECREF(obj) is not needed here.
```

Suavemente descontinuado desde a versão 3.13.

int PyModule_AddIntConstant(PyObject *module, const char *name, long value)

Parte da ABI Estável.

Add an integer constant to *module* as *name*. This convenience function can be used from the module's initialization function. Return `-1` with an exception set on error, `0` on success.

This is a convenience function that calls **PyLong_FromLong()** and **PyModule_AddObjectRef()**; see their documentation for details.

int PyModule_AddStringConstant(PyObject *module, const char *name, const char *value)

Parte da ABI Estável.

Add a string constant to *module* as *name*. This convenience function can be used from the module's initialization function. The string *value* must be `NULL`-terminated. Return `-1` with an exception set on error, `0` on success.

This is a convenience function that calls **`PyUnicode_InternFromString()`** and **`PyModule_AddObjectRef()`**; see their documentation for details.

PyModule_AddIntMacro(module, macro)

Add an int constant to *module*. The name and the value are taken from *macro*. For example `PyModule_AddIntMacro(module, AF_INET)` adds the int constant *AF_INET* with the value of *AF_INET* to *module*. Return `-1` with an exception set on error, `0` on success.

PyModule_AddStringMacro(module, macro)

Add a string constant to *module*.

int PyModule_AddType(PyObject *module, PyTypeObject *type)

Parte da ABI Estável desde a versão 3.10.

Add a type object to *module*. The type object is finalized by calling internally **`PyType_Ready()`**. The name of the type object is taken from the last component of **`tp_name`** after dot. Return `-1` with an exception set on error, `0` on success.

Adicionado na versão 3.9.

int PyModule_AddFunctions(PyObject *module, PyMethodDef *functions)

Parte da ABI Estável desde a versão 3.7.

Add the functions from the `NULL` terminated *functions* array to *module*. Refer to the **`PyMethodDef`** documentation for details on individual entries (due to the lack of a shared module namespace, module level

"functions" implemented in C typically receive the module as their first parameter, making them similar to instance methods on Python classes).

This function is called automatically when creating a module from `PyModuleDef` (such as when using Inicialização multifásica, `PyModule_Create`, or `PyModule_FromDefAndSpec`). Some module authors may prefer defining functions in multiple **`PyMethodDef`** arrays; in that case they should call this function directly.

The *functions* array must be statically allocated (or otherwise guaranteed to outlive the module object).

Adicionado na versão 3.5.

int PyModule_SetDocString(PyObject *module, const char *docstring)

Parte da ABI Estável desde a versão 3.7.
Set the docstring for *module* to *docstring*. This function is called automatically when creating a module from `PyModuleDef` (such as when using Inicialização multifásica, `PyModule_Create`, or `PyModule_FromDefAndSpec`).

Return `0` on success. Return `-1` with an exception set on error.

Adicionado na versão 3.5.

int PyUnstable_Module_SetGIL(PyObject *module, void *gil)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Indicate that *module* does or does not support running without the global interpreter lock (GIL), using one of the values from **`Py_mod_gil`**. It must be called during *module*'s initialization function when using Inicialização monofásica legada. If this function is not called during module initialization, the import machinery assumes the module does not support running without the GIL. This function is only available in Python builds configured with **`--disable-gil`**. Return `-1` with an exception set on error, `0` on success.

Adicionado na versão 3.13.

Module lookup (single-phase initialization)

The legacy single-phase initialization initialization scheme creates singleton modules that can be looked up in the context of the current interpreter. This allows the module object to be retrieved later with only a reference to the module definition.

These functions will not work on modules created using multi-phase initialization, since multiple such modules can be created from a single definition.

PyObject *PyState_FindModule(PyModuleDef *def)

Retorna valor: Referência emprestada. Parte da ABI Estável.

Returns the module object that was created from *def* for the current interpreter. This method requires that the module object has been attached to the interpreter state with **`PyState_AddModule()`** beforehand. In case the corresponding module object is not found or has not been attached to the interpreter state yet, it returns `NULL`.

int PyState_AddModule(PyObject *module, PyModuleDef *def)

Parte da ABI Estável desde a versão 3.3.

Attaches the module object passed to the function to the interpreter state. This allows the module object to be accessible via **`PyState_FindModule()`**.

Only effective on modules created using single-phase initialization.

Python calls `PyState_AddModule` automatically after importing a module that uses single-phase initialization, so it is unnecessary (but harmless) to call it from module initialization code. An explicit call is needed only if the module's own init code subsequently calls `PyState_FindModule`. The function is mainly intended for implementing alternative import mechanisms (either by calling it directly, or by referring to its implementation for details of the required state updates).

If a module was attached previously using the same *def*, it is replaced by the new *module*.

O chamador deve ter um estado de thread anexado.

Retorna `-1` com uma exceção definida em caso de erro, `0` em caso de sucesso.

Adicionado na versão 3.3.

int PyState_RemoveModule(PyModuleDef *def)

Parte da ABI Estável desde a versão 3.3.
Removes the module object created from *def* from the interpreter state.
Return `-1` with an exception set on error, `0` on success.

O chamador deve ter um estado de thread anexado.

Adicionado na versão 3.3.

Objetos Iteradores

O Python fornece dois objetos iteradores de propósito geral. O primeiro, um iterador de sequência, trabalha com uma sequência arbitrária suportando o método `__getitem__()`. O segundo trabalha com um objeto chamável e um valor de sentinela, chamando o chamável para cada item na sequência e finalizando a iteração quando o valor de sentinela é retornado.

PyTypeObject PySeqIter_Type

Parte da ABI Estável.

Objeto de tipo para objetos iteradores retornados por `PySeqIter_New()` e a forma de um argumento da função embutida `iter()` para os tipos de sequência embutidos.

int PySeqIter_Check(PyObject *op)

Retorna true se o tipo de *op* for `PySeqIter_Type`. Esta função sempre é bem-sucedida.

PyObject *PySeqIter_New(PyObject *seq)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um iterador que funcione com um objeto de sequência geral, *seq*. A iteração termina quando a sequência levanta `IndexError` para a operação de assinatura.

PyTypeObject PyCallIter_Type

Parte da ABI Estável.

Objeto de tipo para objetos iteradores retornados por `PyCallIter_New()` e a forma de dois argumentos da função embutida `iter()`.

int PyCallIter_Check(PyObject *op)

Retorna true se o tipo de *op* for `PyCallIter_Type`. Esta função sempre é bem-sucedida.

PyObject *PyCallIter_New(PyObject *callable, PyObject *sentinel)

Retorna valor: Nova referência. Parte da ABI Estável.
Retorna um novo iterador. O primeiro parâmetro, *callable*, pode ser qualquer objeto chamável do Python que possa ser chamado sem parâmetros; cada chamada deve retornar o próximo item na iteração. Quando *callable* retorna um valor igual a *sentinel*, a iteração será encerrada.

Objetos intervalo

PyTypeObject PyRange_Type
Parte da ABI Estável.
O objeto tipo para objetos `range`.

int PyRange_Check(PyObject *o)
Retorna verdadeiro se o objeto *op* for uma instância de um objeto `range`. Esta função sempre tem sucesso.

Tipos de iteradores embutidos

Esses são tipos de iteração embutidos que estão incluídos na API C do Python, mas não fornecem funções adicionais. Eles estão aqui para fins de completude.

Tipo em C	Tipo em Python
PyTypeObject PyEnum_Type *Parte da ABI Estável.*	`enumerate`
PyTypeObject PyFilter_Type *Parte da ABI Estável.*	`filter`

Tipo em C	Tipo em Python
PyTypeObject PyMap_Type *Parte da ABI Estável.*	`map`
PyTypeObject PyReversed_Type *Parte da ABI Estável.*	`reversed`
PyTypeObject PyZip_Type *Parte da ABI Estável.*	`zip`

Outros objetos iteradores

PyTypeObject PyByteArrayIter_Type
Parte da ABI Estável.

PyTypeObject PyBytesIter_Type
Parte da ABI Estável.

PyTypeObject PyListIter_Type
Parte da ABI Estável.

PyTypeObject PyListRevIter_Type
Parte da ABI Estável.

PyTypeObject PySetIter_Type
Parte da ABI Estável.

PyTypeObject PyTupleIter_Type
Parte da ABI Estável.

PyTypeObject PyRangeIter_Type

Parte da ABI Estável.

PyTypeObject PyLongRangeIter_Type
Parte da ABI Estável.

PyTypeObject PyDictIterKey_Type
Parte da ABI Estável.

PyTypeObject PyDictRevIterKey_Type
Parte da ABI Estável desde a versão 3.8.

PyTypeObject PyDictIterValue_Type
Parte da ABI Estável.

PyTypeObject PyDictRevIterValue_Type
Parte da ABI Estável desde a versão 3.8.

PyTypeObject PyDictIterItem_Type
Parte da ABI Estável.

PyTypeObject PyDictRevIterItem_Type
Parte da ABI Estável desde a versão 3.8.

PyTypeObject PyODictIter_Type
Objetos de tipo para iteradores de vários objetos embutidos.

Não cria instâncias dessas diretamente; prefira chamar `PyObject_GetIter()` em vez disso.

Note que não há garantia de que um determinado tipo embutido utilize um determinado tipo de iterador. Por exemplo, iterar sobre um `range` utilizará um de dois tipos de iterador, dependendo do tamanho do intervalo. Outros tipos podem começar a usar um esquema semelhante no futuro, sem aviso prévio.

Objetos descritores

“Descritores” são objetos que descrevem algum atributo de um objeto. Eles são encontrados no dicionário de objetos de tipo.

PyObject *PyDescr_NewGetSet(PyTypeObject *type, struct PyGetSetDef *getset)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a new get-set descriptor for extension type *type* from the **`PyGetSetDef`** structure *getset*.

Get-set descriptors expose attributes implemented by C getter and setter functions rather than stored directly in the instance. This is the same kind of descriptor created for entries in **`tp_getset`**, and it appears in Python as a **`types.GetSetDescriptorType`** object.

On success, return a strong reference to the descriptor. Return `NULL` with an exception set on failure.

PyObject *PyDescr_NewMember(PyTypeObject *type, struct PyMemberDef *member)

Retorna valor: Nova referência. Parte da ABI Estável.
Create a new member descriptor for extension type *type* from the **`PyMemberDef`** structure *member*.

Member descriptors expose fields in the type’s C struct as Python attributes. This is the same kind of descriptor created for entries in **`tp_members`**, and it appears in Python as a **`types.MemberDescriptorType`** object.

On success, return a strong reference to the descriptor. Return `NULL` with an exception set on failure.

PyTypeObject PyMemberDescr_Type

Parte da ABI Estável.
O objeto de tipo para descritores de membros criados a partir de estruturas `PyMemberDef`. Esses descritores expõem campos de uma estrutura C como atributos de um tipo e correspondem a objetos `types.MemberDescriptorType` em Python.

PyTypeObject PyGetSetDescr_Type
Parte da ABI Estável.
The type object for get/set descriptor objects created from `PyGetSetDef` structures. These descriptors implement attributes whose value is computed by C getter and setter functions, and are used for many built-in type attributes. They correspond to `types.GetSetDescriptorType` objects in Python.

PyObject *PyDescr_NewMethod(PyTypeObject *type, struct PyMethodDef *meth)
Retorna valor: Nova referência. Parte da ABI Estável.
Create a new method descriptor for extension type *type* from the `PyMethodDef` structure *meth*.

Method descriptors expose C functions as methods on a type. This is the same kind of descriptor created for entries in `tp_methods`, and it appears in Python as a `types.MethodDescriptorType` object.

On success, return a strong reference to the descriptor. Return `NULL` with an exception set on failure.

PyTypeObject PyMethodDescr_Type
Parte da ABI Estável.
The type object for method descriptor objects created from `PyMethodDef` structures. These descriptors expose C functions as methods on a type, and correspond to `types.MethodDescriptorType` objects in Python.

struct wrapperbase
Describes a slot wrapper used by `PyDescr_NewWrapper()`.

Each `wrapperbase` record stores the Python-visible name and metadata for a special method implemented by a type slot, together with the wrapper function used to adapt that slot to Python's calling convention.

PyObject *PyDescr_NewWrapper(PyTypeObject *type, struct wrapperbase *base, void *wrapped)

Retorna valor: Nova referência.

Create a new wrapper descriptor for extension type *type* from the **`wrapperbase`** structure *base* and the wrapped slot function pointer *wrapped*.

Wrapper descriptors expose special methods implemented by type slots. This is the same kind of descriptor that CPython creates for slot-based special methods such as `__repr__` or `__add__`, and it appears in Python as a **`types.WrapperDescriptorType`** object.

On success, return a strong reference to the descriptor. Return `NULL` with an exception set on failure.

PyTypeObject PyWrapperDescr_Type

Parte da ABI Estável.

O objeto de tipo para objetos descritores de encapsulamento criados por **`PyDescr_NewWrapper()`** e **`PyWrapper_New()`**. Os descritores de encapsulamento são usados internamente para expor métodos especiais implementados por meio de estruturas de encapsulamento e aparecem em Python como objetos **`types.WrapperDescriptorType`**.

PyObject *PyDescr_NewClassMethod(PyTypeObject *type, PyMethodDef *method)

Retorna valor: Nova referência. Parte da ABI Estável.

Create a new class method descriptor for extension type *type* from the **`PyMethodDef`** structure *method*.

Class method descriptors expose C methods that receive the class rather than an instance when accessed. This is the same kind of descriptor created for `METH_CLASS` entries in **`tp_methods`**, and it appears in Python as a **`types.ClassMethodDescriptorType`** object.

On success, return a strong reference to the descriptor. Return `NULL` with an exception set on failure.

int PyDescr_IsData(PyObject *descr)

Retorna não-zero se o objeto descritor *descr* descrevem um atributo de dados, ou `0` se os mesmos descrevem um método. *descr* deve ser um objeto descritor; não há verificação de erros.

PyObject *PyWrapper_New(PyObject *d, PyObject *self)

Retorna valor: Nova referência. Parte da ABI Estável.

Create a new bound wrapper object from the wrapper descriptor *d* and the instance *self*.

This is the bound form of a wrapper descriptor created by **`PyDescr_NewWrapper()`**. CPython creates these objects when a slot wrapper is accessed through an instance, and they appear in Python as **`types.MethodWrapperType`** objects.

On success, return a strong reference to the wrapper object. Return `NULL` with an exception set on failure.

PyDescr_COMMON

This is a macro including the common fields for a descriptor object.

This was included in Python's C API by mistake; do not use it in extensions. For creating custom descriptor objects, create a class implementing the descriptor protocol (**`tp_descr_get`** and **`tp_descr_set`**).

Suavemente descontinuado desde a versão 3.15.

Descritores embutidos

PyTypeObject PyProperty_Type

Parte da ABI Estável.

The type object for property objects. This is the same object as **`property`** in the Python layer.

PyTypeObject PySuper_Type

Parte da ABI Estável.

O objeto de tipo para superobjetos. Este é o mesmo objeto que `super` na camada Python.

PyTypeObject PyClassMethod_Type

O tipo de objeto de método de classe. Este é o mesmo objeto que `classmethod` na camada Python.

PyTypeObject PyClassMethodDescr_Type

Parte da ABI Estável.

The type object for C-level class method descriptor objects. This is the type of the descriptors created for `classmethod()` defined in C extension types, and corresponds to `types.ClassMethodDescriptorType` objects in Python.

PyObject *PyClassMethod_New(PyObject *callable)

Cria um novo objeto `classmethod` que envolve *callable*. *callable* deve ser um objeto chamável e não deve ser `NULL`.

Em caso de sucesso, esta função retorna uma referência forte a um novo descritor de método de classe. Em caso de falha, esta função retorna `NULL` com uma exceção definida.

PyTypeObject PyStaticMethod_Type

O tipo de objeto de método estático. Este é o mesmo objeto que `staticmethod` na camada Python.

PyObject *PyStaticMethod_New(PyObject *callable)

Cria um novo objeto `staticmethod` que envolve *callable*. *callable* deve ser um objeto chamável e não deve ser `NULL`.

Em caso de sucesso, esta função retorna uma referência forte a um novo descritor de método estático. Em caso de falha, esta função retorna `NULL` com uma exceção definida.

Objetos Slice

PyTypeObject PySlice_Type

Parte da ABI Estável.

Tipo de objeto para objetos fatia. Isso é o mesmo que `slice` na camada Python.

int PySlice_Check(PyObject *ob)

Retorna true se *ob* for um objeto fatia; *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

PyObject *PySlice_New(PyObject *start, PyObject *stop, PyObject *step)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um novo objeto fatia com os valores fornecidos. Os parâmetros *start*, *stop* e *step* são usados como os valores dos atributos do objeto fatia com os mesmos nomes. Qualquer um dos valores pode ser `NULL`, caso em que `None` será usado para o atributo correspondente.

Retorna `NULL` com uma exceção definida se o novo objeto não puder ser alocado.

int PySlice_GetIndices(PyObject *slice, Py_ssize_t length, Py_ssize_t *start, Py_ssize_t *stop, Py_ssize_t *step)

Parte da ABI Estável.

Recupera os índices de início, parada e intermediário do objeto fatia *slice*, presumindo uma sequência de comprimento *length*. Trata índices maiores que *length* como erros.

Retorna `0` em caso de sucesso e `-1` em caso de erro sem exceção definida (a menos que um dos índices não fosse `None` e falhou ao ser convertido para um inteiro, neste caso `-1` é retornado com uma exceção definida).

Você provavelmente não deseja usar esta função.

Alterado na versão 3.2: O tipo de parâmetro para o parâmetro *slice* era antes de `PySliceObject*`.

int PySlice_GetIndicesEx(PyObject *slice, Py_ssize_t length, Py_ssize_t *start, Py_ssize_t *stop, Py_ssize_t *step, Py_ssize_t *slicelength)

Parte da ABI Estável.

Usable replacement for **`PySlice_GetIndices()`**. Retrieve the start, stop, and step indices from the slice object *slice* assuming a sequence of length *length*, and store the length of the slice in *slicelength*. Out of bounds indices are clipped in a manner consistent with the handling of normal slices. *length* must not be negative.

Retorna `0` em caso de sucesso e `-1` em caso de erro com uma exceção definida.

Nota

Esta função não é considerada segura para sequências redimensionáveis. Sua invocação deve ser substituída por uma combinação de **`PySlice_Unpack()`** e **`PySlice_AdjustIndices()`** sendo

```
if (PySlice_GetIndicesEx(slice, length, &start, &stop, &ste
    // retorna erro
}
```

substituído por

```
if (PySlice_Unpack(slice, &start, &stop, &step) < 0) {
    // retorna erro
}
slicelength = PySlice_AdjustIndices(length, &start, &stop,
```

Alterado na versão 3.2: O tipo de parâmetro para o parâmetro *slice* era antes de `PySliceObject*`.

Alterado na versão 3.6.1: Se `Py_LIMITED_API` não estiver definido ou estiver definido com um valor entre `0x03050400` e `0x03060000` (não incluso) ou `0x03060100` ou mais alto, **`PySlice_GetIndicesEx()`** é implementado como uma macro usando **`PySlice_Unpack()`** e **`PySlice_AdjustIndices()`**. Os argumentos *start*, *stop* e *step* são avaliados mais de uma vez.

Descontinuado desde a versão 3.6.1: Se `Py_LIMITED_API` estiver definido para um valor menor que `0x03050400` ou entre `0x03060000` e `0x03060100` (não incluso), **`PySlice_GetIndicesEx()`** é uma função descontinuada.

int PySlice_Unpack(PyObject *slice, Py_ssize_t *start, Py_ssize_t *stop, Py_ssize_t *step)

Parte da ABI Estável desde a versão 3.7.
Extrai os membros de dados de início, parada e intermediário de um objeto fatia como C inteiros. Reduz silenciosamente os valores maiores do que `PY_SSIZE_T_MAX` para `PY_SSIZE_T_MAX`, aumenta silenciosamente os valores de início e parada menores que `PY_SSIZE_T_MIN` para `PY_SSIZE_T_MIN`, e silenciosamente aumenta os valores de intermediário menores que `-PY_SSIZE_T_MAX` para `-PY_SSIZE_T_MAX`.

Retorna `-1` com uma exceção definida em caso de erro, `0` em caso de sucesso.

Adicionado na versão 3.6.1.

Py_ssize_t PySlice_AdjustIndices(Py_ssize_t length, Py_ssize_t *start, Py_ssize_t *stop, Py_ssize_t step)

Parte da ABI Estável desde a versão 3.7.
Ajusta os índices de fatias inicial/final presumindo uma sequência do comprimento especificado. Índices fora dos limites são cortados de maneira consistente com o tratamento de fatias normais.

length must not be negative. *step* must not be zero and must not be less than `-PY_SSIZE_T_MAX`, as guaranteed by **`PySlice_Unpack()`**.

Retorna o comprimento da fatia. Sempre bem-sucedido. Não chama o código Python.

Adicionado na versão 3.6.1.

Objeto Ellipsis

PyTypeObject PyEllipsis_Type

Parte da ABI Estável.

O tipo do objeto Python `Ellipsis`. O mesmo que `types.EllipsisType` na camada Python.

PyObject *Py_Ellipsis

O objeto Python `Ellipsis`. Este objeto não tem métodos. Como `Py_None`, é um objeto singleton imortal.

Alterado na versão 3.12: `Py_Ellipsis` é imortal.

Objetos MemoryView

Um objeto `memoryview` expõe a interface de buffer a nível de C como um objeto Python que pode ser passado como qualquer outro objeto.

PyTypeObject PyMemoryView_Type

Parte da ABI Estável.

Esta instância de `PyTypeObject` representa o tipo memoryview Python. Este é o mesmo objeto que `memoryview` na camada Python.

PyObject *PyMemoryView_FromObject(PyObject *obj)

Retorna valor: Nova referência. Parte da ABI Estável.

Cria um objeto memoryview a partir de um objeto que fornece a interface do buffer. Se *obj* tiver suporte a exportações de buffer graváveis, o objeto memoryview será de leitura/gravação; caso contrário, poderá ser somente leitura ou leitura/gravação, a critério do exportador.

PyBUF_READ

Parte da ABI Estável desde a versão 3.11.

Sinalizador para solicitar um buffer de somente leitura.

PyBUF_WRITE

Parte da ABI Estável desde a versão 3.11.

Sinalizador para solicitar um buffer gravável.

PyObject *PyMemoryView_FromMemory(char *mem, Py_ssize_t size, int flags)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.7.

Cria um objeto memoryview usando *mem* como o buffer subjacente. *flags* pode ser um dos seguintes `PyBUF_READ` ou `PyBUF_WRITE`.

Adicionado na versão 3.3.

PyObject *PyMemoryView_FromBuffer(const Py_buffer *view)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.11.

Cria um objeto de memoryview envolvendo a estrutura de buffer *view* fornecida. Para buffers de bytes simples, `PyMemoryView_FromMemory()` é a função preferida.

PyObject *PyMemoryView_GetContiguous(PyObject *obj, int buffertype, char order)

Retorna valor: Nova referência. Parte da ABI Estável.

Cria um objeto memoryview para um pedaço contíguo de memória (na ordem 'C' ou 'F'ortran, representada por *order*) a partir de um objeto que define a interface do buffer. Se a memória for contígua, o objeto memoryview apontará para a memória original. Caso contrário, é feita uma cópia e a visualização da memória aponta para um novo objeto bytes.

buffertype pode ser um entre `PyBUF_READ` ou `PyBUF_WRITE`.

int PyMemoryView_Check(PyObject *obj)

Retorna true se o objeto *obj* for um objeto memoryview. Atualmente, não é permitido criar subclasses de `memoryview`. Esta função sempre tem sucesso.

Py_buffer *PyMemoryView_GET_BUFFER(PyObject *mview)

Retorna um ponteiro para a cópia privada da memória do buffer do exportador. *mview* **deve** ser uma instância de memoryview; Se essa macro não verificar seu tipo, faça você mesmo ou corre o risco de travar.

PyObject *PyMemoryView_GET_BASE(PyObject *mview)

Retorna um ponteiro para o objeto de exportação no qual a memória é baseada ou `NULL` se a memória tiver sido criada por uma das funções `PyMemoryView_FromMemory()` ou `PyMemoryView_FromBuffer()`. *mview* **deve** ser uma instância de memoryview.

Objetos buffer serializados com pickle

Adicionado na versão 3.8.

Um objeto `pickle.PickleBuffer` encapsula um objeto provedor de buffer para transferência de dados fora de banda com o módulo `pickle`.

PyTypeObject PyPickleBuffer_Type

Esta instância de `PyTypeObject` representa o tipo de buffer serializado com pickle do Python. Este é o mesmo objeto que `pickle.PickleBuffer` na camada Python.

int PyPickleBuffer_Check(PyObject *op)

Retorna verdadeiro se *op* for uma instância de buffer serializado com pickle. Esta função sempre tem sucesso.

PyObject *PyPickleBuffer_FromObject(PyObject *obj)

Cria um buffer serializado com pickle a partir do objeto *obj*.

Esta função falhará se *obj* não oferecer suporte ao protocolo de buffer.

Em caso de sucesso, retorna uma nova instância de buffer serializado com pickle. Em caso de falha, define uma exceção e retorna `NULL`.

Análogo a chamar `pickle.PickleBuffer` com *obj* em Python.

const Py_buffer *PyPickleBuffer_GetBuffer(PyObject *picklebuf)

Obtém um ponteiro para o `Py_buffer` subjacente que o buffer serializado com pickle encapsula.

O ponteiro retornado é válido enquanto *picklebuf* estiver ativo e não tiver sido liberado. O chamador não deve modificar ou liberar o

`Py_buffer` retornado. Se o buffer serializado com pickle tiver sido liberado, levanta uma exceção `ValueError`.

Em caso de sucesso, retorna um ponteiro para a visualização do buffer. Em caso de falha, define uma exceção e retorna `NULL`.

int PyPickleBuffer_Release(PyObject *picklebuf)

Libera o buffer subjacente mantido pelo buffer serializado com pickle.

Retorna `0` em caso de sucesso. Em caso de falha, define uma exceção e retorna `-1`.

Análogo a chamar `pickle.PickleBuffer.release()` em Python.

Objetos referência fraca

Python oferece suporte a *referências fracas* como objetos de primeira classe. Existem dois tipos de objetos específicos que implementam diretamente referências fracas. O primeiro é um objeto de referência simples, e o segundo atua como um intermediário entre o objeto original e o primeiro, tanto quanto possível.

int PyWeakref_Check(PyObject *ob)

Retorna não zero se *ob* for um objeto referência ou um objeto intermediário. Esta função sempre tem sucesso.

int PyWeakref_CheckRef(PyObject *ob)

Retorna não zero se *ob* for um objeto referência ou uma subclasse do tipo de referência. Esta função sempre tem sucesso.

int PyWeakref_CheckRefExact(PyObject *ob)

Retorna não zero se *ob* for um objeto referência, mas não uma subclasse do tipo de referência. Esta função sempre tem sucesso.

int PyWeakref_CheckProxy(PyObject *ob)

Retorna não zero se *ob* for um objeto intermediário. Esta função sempre tem sucesso.

PyObject *PyWeakref_NewRef(PyObject *ob, PyObject *callback)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um objeto de referência fraca para o objeto *ob*. Isso sempre retornará uma nova referência, mas não é garantido que um novo objeto seja criado; um objeto de referência existente pode ser retornado. O segundo parâmetro, *callback*, pode ser um objeto chamável que recebe uma notificação quando *ob* for coletado como lixo; ele deve aceitar um único parâmetro, que será o objeto de referência fraca propriamente dito. *callback* também pode ser `None` ou `NULL`. Se *ob* não for um objeto fracamente referenciável, isso retornará `NULL` e levantará **`TypeError`**.

Ver também

`PyType_SUPPORTS_WEAKREFS()` para verificar se *ob* é fracamente referenciável.

PyObject *PyWeakref_NewProxy(PyObject *ob, PyObject *callback)

Retorna valor: Nova referência. Parte da ABI Estável.

Retorna um objeto proxy de referência fraca para o objeto *ob*. Isso sempre retornará uma nova referência, mas não é garantido que um novo objeto seja criado; um objeto proxy de referência existente pode ser retornado. O segundo parâmetro, *callback*, pode ser um objeto chamável que recebe uma notificação quando *ob* for coletado como lixo; ele deve aceitar um único parâmetro, que será o objeto de referência fraca propriamente dito. *callback* também pode ser `None` ou `NULL`. Se *ob* for um objeto fracamente referenciável, isso retornará `NULL` e levantará `TypeError`.

Ver também

`PyType_SUPPORTS_WEAKREFS()` para verificar se *ob* é fracamente referenciável.

int PyWeakref_GetRef(PyObject *ref, PyObject **pobj)

Parte da ABI Estável desde a versão 3.13.

Obtém uma referência forte para o objeto referenciado a partir de uma referência fraca, *ref*, em **pobj*.

- Em caso de sucesso, define **pobj* como uma nova referência forte para o objeto referenciado e retorna 1.
- Se a referência estiver quebrada, define **pobj* como `NULL` e retorna 0.
- Em caso de erro, levanta uma exceção e retorna -1.

Adicionado na versão 3.13.

int PyWeakref_IsDead(PyObject *ref)

Testa se a referência fraca *ref* está inativa. Retorna 1 se a referência estiver inativa, 0 se estiver ativa e -1 com um erro definido se *ref* não for um objeto referência fraca.

Adicionado na versão 3.14.

void PyObject_ClearWeakRefs(PyObject *object)

Parte da ABI Estável.
Esta função é chamada pelo tratador `tp_dealloc` para limpar referências fracas.

Isso itera pelas referências fracas para *object* e chama as funções de retorno para as referências que possuem um. Ele retorna quando todas as funções de retorno tiverem sido executadas.

void PyUnstable_Object_ClearWeakRefsNoCallbacks(PyObject *object)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Limpa as referências fracas para *object* sem chamar as funções de retorno.

Esta função é chamada pelo manipulador `tp_dealloc` para tipos com finalizadores (por exemplo, `__del__()`). O manipulador para esses objetos primeiro chama `PyObject_ClearWeakRefs()` para limpar referências fracas e chamar suas funções de retorno, depois o finalizador e, finalmente, esta função para limpar quaisquer referências fracas que possam ter sido criadas pelo finalizador.

Na maioria das circunstâncias, é mais apropriado usar `PyObject_ClearWeakRefs()` para limpar referências fracas em vez desta função.

Adicionado na versão 3.13.

Capsules

Consulte Fornecendo uma API C para um módulo de extensão para obter mais informações sobre o uso desses objetos.

Adicionado na versão 3.1.

type PyCapsule

Este subtipo de `PyObject` representa um valor opaco, útil para módulos de extensão C que precisam passar um valor opaco (como ponteiro void*) através do código Python para outro código C . É frequentemente usado para disponibilizar um ponteiro de função C definido em um módulo para outros módulos, para que o mecanismo de importação regular possa ser usado para acessar APIs C definidas em módulos carregados dinamicamente.

PyTypeObject PyCapsule_Type

Parte da ABI Estável.

O objeto de tipo correspondente aos objetos cápsula. Este é o mesmo objeto que `types.CapsuleType` na camada Python.

type PyCapsule_Destructor

Parte da ABI Estável.

O tipo de um retorno de chamada destruidor para uma cápsula. Definido como:

```
typedef void (*PyCapsule_Destructor)(PyObject *);
```

Veja `PyCapsule_New()` para a semântica dos retornos de chamada PyCapsule_Destructor.

int PyCapsule_CheckExact(PyObject *p)

Thread safety: Atomic.

Retorna true se seu argumento é um `PyCapsule`. Esta função sempre tem sucesso.

PyObject *PyCapsule_New(void *pointer, const char *name, PyCapsule_Destructor destructor)

Retorna valor: Nova referência. Parte da ABI Estável. Thread safety: Atomic.

Cria um **`PyCapsule`** que encapsula o *ponteiro*. O argumento *pointer* pode não ser `NULL`.

Em caso de falha, define uma exceção e retorna `NULL`.

A string *name* pode ser `NULL` ou um ponteiro para uma string C válida. Se não for `NULL`, essa string deverá sobreviver à cápsula. (Embora seja permitido liberá-lo dentro do *descructor*.)

Se o argumento *destructor* não for `NULL`, ele será chamado com a cápsula como argumento quando for destruído.

Se esta cápsula for armazenada como um atributo de um módulo, o *name* deve ser especificado como `modulename.attributename`. Isso permitirá que outros módulos importem a cápsula usando **`PyCapsule_Import()`**.

void *PyCapsule_GetPointer(PyObject *capsule, const char *name)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.

Recupera o *pointer* armazenado na cápsula. Em caso de falha, define uma exceção e retorna `NULL`.

O parâmetro *name* deve ser comparado exatamente com o nome armazenado na cápsula. Se o nome armazenado na cápsula for `NULL`, o *name* passado também deve ser `NULL`. Python usa a função C **`strcmp()`** para comparar nomes de cápsulas.

PyCapsule_Destructor PyCapsule_GetDestructor(PyObject *capsule)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.

Retorna o destruidor atual armazenado na cápsula. Em caso de falha, define uma exceção e retorna `NULL`.

É legal para uma cápsula ter um destruidor `NULL`. Isso torna um código de retorno `NULL` um tanto ambíguo; use **`PyCapsule_IsValid()`** ou **`PyErr_Occurred()`** para desambiguar.

void *PyCapsule_GetContext(PyObject *capsule)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.
Retorna o contexto atual armazenado na cápsula. Em caso de falha, define uma exceção e retorna `NULL`.

É legal para uma cápsula ter um contexto `NULL`. Isso torna um código de retorno `NULL` um tanto ambíguo; use **`PyCapsule_IsValid()`** ou **`PyErr_Occurred()`** para desambiguar.

const char *PyCapsule_GetName(PyObject *capsule)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.
Retorna o nome atual armazenado na cápsula. Em caso de falha, define uma exceção e retorna `NULL`.

É legal para uma cápsula ter um nome `NULL`. Isso torna um código de retorno `NULL` um tanto ambíguo; use **`PyCapsule_IsValid()`** ou **`PyErr_Occurred()`** para desambiguar.

void *PyCapsule_Import(const char *name, int no_block)

Parte da ABI Estável. Thread safety: Safe to call from multiple threads with external synchronization only.
Importa um ponteiro para um objeto C de um atributo de cápsula em um módulo. O parâmetro *name* deve especificar o nome completo do atributo, como em `module.attribute`. O *name* armazenado na cápsula deve corresponder exatamente a essa string.

Esta função divide *name* no caractere `.` e importa o primeiro elemento. Em seguida, processa os elementos seguintes usando pesquisas de atributos.

Retorna o ponteiro interno *pointer* da cápsula com sucesso. Em caso de falha, define uma exceção e retorna `NULL`.

Nota

Se *name* apontar para um atributo de algum submódulo ou subpacote, esse submódulo ou subpacote deverá ser importado previamente usando outros meios (por exemplo, usando **`PyImport_ImportModule()`**) para que as pesquisas de atributos sejam bem-sucedidas.

Alterado na versão 3.3: no_block não tem mais efeito.

int PyCapsule_IsValid(PyObject *capsule, const char *name)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.

Determina se *capsule* é ou não uma cápsula válida. Uma cápsula válida é diferente de `NULL`, passa **`PyCapsule_CheckExact()`**, possui um ponteiro diferente de `NULL` armazenado e seu nome interno corresponde ao parâmetro *name*. (Consulte **`PyCapsule_GetPointer()`** para obter informações sobre como os nomes das cápsulas são comparados.)

Em outras palavras, se **`PyCapsule_IsValid()`** retornar um valor verdadeiro, as chamadas para qualquer um dos acessadores (qualquer função que comece com `PyCapsule_Get`) terão êxito garantido.

Retorna um valor diferente de zero se o objeto for válido e corresponder ao nome passado. Retorna `0` caso contrário. Esta função não falhará.

int PyCapsule_SetContext(PyObject *capsule, void *context)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.

Define o ponteiro de contexto dentro de *capsule* para *context*.

Retorna `0` em caso de sucesso. Retorna diferente de zero e define uma exceção em caso de falha.

int PyCapsule_SetDestructor(PyObject *capsule, PyCapsule_Destructor destructor)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.
Define o destrutor dentro de *capsule* para *destructor*.

Retorna `0` em caso de sucesso. Retorna diferente de zero e define uma exceção em caso de falha.

int PyCapsule_SetName(PyObject *capsule, const char *name)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.
Define o nome dentro de *capsule* como *name*. Se não for `NULL`, o nome deve sobreviver à cápsula. Se o *name* anterior armazenado na cápsula não era `NULL`, nenhuma tentativa será feita para liberá-lo.

Retorna `0` em caso de sucesso. Retorna diferente de zero e define uma exceção em caso de falha.

int PyCapsule_SetPointer(PyObject *capsule, void *pointer)

Parte da ABI Estável. Thread safety: Safe to call without external synchronization on distinct objects.
Define o ponteiro nulo dentro de *capsule* para *pointer*. O ponteiro não pode ser `NULL`.

Retorna `0` em caso de sucesso. Retorna diferente de zero e define uma exceção em caso de falha.

Objetos sentinel

PyTypeObject PySentinel_Type

Esta instância do **PyTypeObject** representa o tipo **sentinel** do Python. Este é o mesmo objeto que **sentinel**.

Adicionado na versão 3.15.

int PySentinel_Check(PyObject *o)

Retorna verdadeiro se *o* for um objeto **sentinel** ou um subtipo. O tipo **sentinel** atualmente não permite subclasses, portanto, essa verificação é exata. Versões futuras do Python poderão permitir a criação de subtipos. Esta função sempre tem sucesso.

Adicionado na versão 3.15.

int PySentinel_CheckExact(PyObject *o)

Retorna verdadeiro se *o* for um objeto **sentinel**, mas não um subtipo. O tipo **sentinel** atualmente não permite subclasses. Versões futuras do Python poderão permitir a criação de subtipos. Esta função sempre tem sucesso.

Adicionado na versão 3.15.

PyObject *PySentinel_New(const char *name, const char *module_name, const char *repr)

Retorna valor: Nova referência.

Retorna um novo objeto **sentinel** com **__name__** definido como *name* e **__module__** definido como *module_name*. *nome* não deve ser `NULL`. Se *module_name* for `NULL`, **__module__** será definido como `None`. Se *repr* for `NULL`, `repr()` retornará **__name__**. Retorna `NULL` com uma exceção definida em caso de falha.

Para que a serialização com *pickle* funcione, *module_name* deve ser o nome de um módulo importável, e o *sentinel* deve estar acessível a partir desse módulo por meio de um caminho que corresponda a *name*. O *pickle* trata *name* como o nome de uma variável global em *module_name* (veja `object.__reduce__()`).

Adicionado na versão 3.15.

Objetos quadro

type PyFrameObject

Parte da ABI Estável (como uma estrutura opaca).
A estrutura C dos objetos usados para descrever objetos frame.

Não há membros públicos nesta estrutura.

Alterado na versão 3.11: Os membros dessa estrutura foram removidos da API C pública. Consulte a entrada O Que há de Novo para detalhes.

As funções `PyEval_GetFrame()` e `PyThreadState_GetFrame()` podem ser utilizadas para obter um objeto frame.

Veja também Reflexão.

PyTypeObject PyFrame_Type

O tipo de objetos frame. É o mesmo objeto que `types.FrameType` na camada Python.

Alterado na versão 3.11: Anteriormente, este tipo só estava disponível após incluir `<frameobject.h>`.

PyFrameObject *PyFrame_New(PyThreadState *tstate, PyCodeObject *code, PyObject *globals, PyObject *locals)

Cria um novo objeto quadro. Esta função retorna uma referência forte ao novo objeto quadro em caso de sucesso e retorna `NULL` com uma exceção definida em caso de falha.

int PyFrame_Check(PyObject *obj)

Retorna diferente de zero se *obj* é um objeto frame

Alterado na versão 3.11: Anteriormente, esta função só estava disponível após incluir `<frameobject.h>`.

PyFrameObject *PyFrame_GetBack(PyFrameObject *frame)

Retorna valor: Nova referência.
Obtém o *frame* próximo ao quadro externo.

Return a strong reference, or `NULL` if *frame* has no outer frame. This raises no exceptions.

Adicionado na versão 3.9.

PyObject *PyFrame_GetBuiltins(PyFrameObject *frame)

Retorna valor: Nova referência.
Obtém o atributo **`f_builtins`** do *frame*.

Retorna uma referência forte. O resultado não pode ser `NULL`.

Adicionado na versão 3.11.

PyCodeObject *PyFrame_GetCode(PyFrameObject *frame)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.10.
Obtém o código de *frame*.

Retorna uma referência forte.

O resultado (código do frame) não pode ser `NULL`.

Adicionado na versão 3.9.

PyObject *PyFrame_GetGenerator(PyFrameObject *frame)

Retorna valor: Nova referência.
Obtém o gerador, corrotina ou gerador assíncrono que possui este frame, ou `NULL` se o frame não pertence a um gerador. Não levanta exceção, mesmo que o valor retornado seja `NULL`.

Retorna uma referência forte, ou `NULL`.

Adicionado na versão 3.11.

PyObject *PyFrame_GetGlobals(PyFrameObject *frame)

Retorna valor: Nova referência.
Obtenha o atributo **`f_globals`** do *frame*.

Retorna uma referência forte. O resultado não pode ser `NULL`.

Adicionado na versão 3.11.

int PyFrame_GetLasti(PyFrameObject *frame)

Obtenha o atributo `f_lasti` do *frame*.

Retorna -1 se `frame.f_lasti` é `None`.

Adicionado na versão 3.11.

PyObject *PyFrame_GetVar(PyFrameObject *frame, PyObject *name)

Retorna valor: Nova referência.
Obtém a variável *name* de *frame*.

- Retorna uma referência forte ao valor da variável em caso de sucesso.
- Levanta uma exceção `NameError` e retorna `NULL` se a variável não existir.
- Levanta uma exceção e retorna `NULL` em caso de erro.

Tipo de *name* deve ser um `str`.

Adicionado na versão 3.12.

PyObject *PyFrame_GetVarString(PyFrameObject *frame, const char *name)

Retorna valor: Nova referência.
Semelhante a `PyFrame_GetVar()`, mas o nome da variável é uma string C codificada em UTF-8.

Adicionado na versão 3.12.

PyObject *PyFrame_GetLocals(PyFrameObject *frame)

Retorna valor: Nova referência.
Obtém o atributo `f_locals` do *frame*. Se o quadro se referir a um escopo otimizado, isso retorna um objeto proxy de escrita direta que permite modificar as variáveis locais. Em todos os outros casos (classes, módulos, `exec()` e `eval()`), retorna o mapeamento que representa as variáveis locais do frame diretamente (como descrito para `locals()`).

Retorna uma referência forte.

Adicionado na versão 3.11.

Alterado na versão 3.13: Como parte de **PEP 667** [https://peps.python.org/pep-0667/], retorna uma instância de `PyFrameLocalsProxy_Type`.

int PyFrame_GetLineNumber(PyFrameObject *frame)

Parte da ABI Estável desde a versão 3.10.

Retorna o número da linha do *frame* atualmente em execução.

Frame locals proxies

Adicionado na versão 3.13.

The `f_locals` attribute on a frame object is an instance of a "frame-locals proxy". The proxy object exposes a write-through view of the underlying locals dictionary for the frame. This ensures that the variables exposed by `f_locals` are always up to date with the live local variables in the frame itself.

See **PEP 667** [https://peps.python.org/pep-0667/] for more information.

PyTypeObject PyFrameLocalsProxy_Type

The type of frame `locals()` proxy objects.

int PyFrameLocalsProxy_Check(PyObject *obj)

Return non-zero if *obj* is a frame `locals()` proxy.

Legacy local variable APIs

These APIs are soft deprecated. As of Python 3.13, they do nothing. They exist solely for backwards compatibility.

void PyFrame_LocalsToFast(PyFrameObject *f, int clear)

Prior to Python 3.13, this function would copy the `f_locals` attribute of *f* to the internal "fast" array of local variables, allowing changes in frame objects to be visible to the interpreter. If *clear* was true, this function would process variables that were unset in the locals dictionary.

Suavemente descontinuado desde a versão 3.13: Esta função agora não faz nada.

void PyFrame_FastToLocals(PyFrameObject *f)

Prior to Python 3.13, this function would copy the internal "fast" array of local variables (which is used by the interpreter) to the `f_locals` attribute of *f*, allowing changes in local variables to be visible to frame objects.

Suavemente descontinuado desde a versão 3.13: Esta função agora não faz nada.

int PyFrame_FastToLocalsWithError(PyFrameObject *f)

Prior to Python 3.13, this function was similar to `PyFrame_FastToLocals()`, but would return `0` on success, and `-1` with an exception set on failure.

Suavemente descontinuado desde a versão 3.13: Esta função agora não faz nada.

Ver também

PEP 667 [https://peps.python.org/pep-0667/]

Internal frames

Unless using **PEP 523** [https://peps.python.org/pep-0523/], you will not need this.

struct _PyInterpreterFrame

The interpreter's internal frame representation.

Adicionado na versão 3.11.

PyObject *PyUnstable_InterpreterFrame_GetCode(struct _PyInterpreterFrame *frame);

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Return a strong reference to the code object for the frame.

Adicionado na versão 3.12.

int PyUnstable_InterpreterFrame_GetLasti(struct _PyInterpreterFrame *frame);

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Return the byte offset into the last executed instruction.

Adicionado na versão 3.12.

int PyUnstable_InterpreterFrame_GetLine(struct _PyInterpreterFrame *frame);

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Return the currently executing line number, or -1 if there is no line number.

Adicionado na versão 3.12.

const PyTypeObject *PyUnstable_ExecutableKinds

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

An array of executable kinds (executor types) for frames, used for internal debugging and tracing.

Tools like debuggers and profilers can use this to identify the type of execution context associated with a frame (such as to filter out internal frames). The entries are indexed by the following constants:

Constante	Descrição
PyUnstable_EXECUTABLE_KIND_SKIP *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*	The frame is internal (For example: inlined) and should be skipped by tools.
PyUnstable_EXECUTABLE_KIND_PY_FUNCTION *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*	The frame corresponds to a standard Python function.

Constante	Descrição
PyUnstable_EXECUTABLE_KIND_BUILTIN_FUNCTION *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*	The frame corresponds to a function defined in native code.
PyUnstable_EXECUTABLE_KIND_METHOD_DESCRIPTOR *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*	The frame corresponds to a method on a class instance.

Note that reading the executable kind from a frame is currently only possible with undocumented internal APIs.

Adicionado na versão 3.13.

PyUnstable_EXECUTABLE_KINDS

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

The number of entries in `PyUnstable_ExecutableKinds`.

Adicionado na versão 3.13.

Objetos Geradores

Objetos geradores são o que o Python usa para implementar iteradores geradores. Eles são normalmente criados por iteração sobre uma função que produz valores, em vez de invocar explicitamente `PyGen_New()` ou `PyGen_NewWithQualName()`.

type PyGenObject

A estrutura C usada para objetos geradores.

PyTypeObject PyGen_Type

O objeto de tipo correspondendo a objetos geradores.

int PyGen_Check(PyObject *ob)

Retorna verdadeiro se *ob* for um objeto gerador; *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyGen_CheckExact(PyObject *ob)

Retorna verdadeiro se o tipo do *ob* é `PyGen_Type`; *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

PyObject *PyGen_New(PyFrameObject *frame)

Retorna valor: Nova referência.

Create and return a new generator object based on the *frame* object. A reference to *frame* is "stolen" by this function (even on error). The argument must not be `NULL`.

PyObject *PyGen_NewWithQualName(PyFrameObject *frame, PyObject *name, PyObject *qualname)

Retorna valor: Nova referência.

Create and return a new generator object based on the *frame* object, with `__name__` and `__qualname__` set to *name* and *qualname*. A reference to *frame* is "stolen" by this function (even on error). The *frame* argument must not be `NULL`.

PyCodeObject *PyGen_GetCode(PyGenObject *gen)

Retorna uma nova referência forte ao objeto de código encapsulado por *gen*. Esta função sempre tem sucesso.

Objetos geradores assíncronos

Ver também

PEP 525 [https://peps.python.org/pep-0525/]

PyTypeObject PyAsyncGen_Type

O objeto de tipo correspondente aos objetos geradores assíncronos. Está disponível como `types.AsyncGeneratorType` na camada Python.

Adicionado na versão 3.6.

PyObject *PyAsyncGen_New(PyFrameObject *frame, PyObject *name, PyObject *qualname)

Create a new asynchronous generator wrapping *frame*, with `__name__` and `__qualname__` set to *name* and *qualname*. *frame* is "stolen" by this function (even on error) and must not be `NULL`.

Em caso de sucesso, esta função retorna uma referência forte ao novo gerador assíncrono. Em caso de falha, esta função retorna `NULL` com uma exceção definida.

Adicionado na versão 3.6.

int PyAsyncGen_CheckExact(PyObject *op)

Retorna verdadeiro se *op* for um objeto gerador assíncrono, falso caso contrário. Esta função sempre tem sucesso.

Adicionado na versão 3.6.

Deprecated API

PyAsyncGenASend_CheckExact(op)

This is an API that was included in Python's C API by mistake.

It is solely here for completeness; do not use this API.

Suavemente descontinuado desde a versão 3.14.

Objetos corrotina

Adicionado na versão 3.5.

Os objetos corrotina são aquelas funções declaradas com um retorno de palavra-chave `async`.

type PyCoroObject

A estrutura C utilizada para objetos corrotinas.

PyTypeObject PyCoro_Type

O tipo de objeto correspondente a objetos corrotina.

int PyCoro_CheckExact(PyObject *ob)

Retorna true se o tipo do *ob* é **`PyCoro_Type`**; *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

PyObject *PyCoro_New(PyFrameObject *frame, PyObject *name, PyObject *qualname)

Retorna valor: Nova referência.

Cria e retorna um novo objeto de corrotina com base no objeto *frame*, com `__name__` e `__qualname__` definido como *name* e *qualname*. Uma referência a *frame* é roubada por esta função. O argumento *frame* não deve ser `NULL`.

Objetos de variáveis de contexto

Adicionado na versão 3.7.

Alterado na versão 3.7.1:

Nota

No Python 3.7.1, as assinaturas de todas as APIs C de variáveis de contexto foram **alteradas** para usar ponteiros `PyObject` em vez de `PyContext`, `PyContextVar` e `PyContextToken`. Por exemplo:

```
// no 3.7.0:
PyContext *PyContext_New(void);

// no 3.7.1+:
PyObject *PyContext_New(void);
```

Veja bpo-34762 [https://bugs.python.org/issue?@action=redirect&bpo=34762] para mais detalhes.

Esta seção detalha a API C pública para o módulo `contextvars`.

type PyContext
: A estrutura C usada para representar um objeto `contextvars.Context`.

type PyContextVar
: A estrutura C usada para representar um objeto `contextvars.ContextVar`.

type PyContextToken
: A estrutura C usada para representar um objeto `contextvars.Token`

PyTypeObject PyContext_Type
: O objeto de tipo que representa o tipo de *contexto*.

PyTypeObject PyContextVar_Type

O objeto de tipo que representa o tipo de *variável de contexto*.

PyTypeObject PyContextToken_Type

O objeto de tipo que representa o tipo de *token de variável de contexto*.

Macros de verificação de tipo:

int PyContext_CheckExact(PyObject *o)

Retorna verdadeiro se *o* for do tipo **`PyContext_Type`**. *o* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyContextVar_CheckExact(PyObject *o)

Retorna verdadeiro se *o* for do tipo **`PyContextVar_Type`**. *o* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyContextToken_CheckExact(PyObject *o)

Retorna verdadeiro se *o* for do tipo **`PyContextToken_Type`**. *o* não deve ser `NULL`. Esta função sempre tem sucesso.

Funções de gerenciamento de objetos de contexto:

PyObject *PyContext_New(void)

Retorna valor: Nova referência.

Cria um novo objeto de contexto vazio. Retorna `NULL` se um erro ocorreu.

PyObject *PyContext_Copy(PyObject *ctx)

Retorna valor: Nova referência.

Cria uma cópia rasa do objeto de contexto *ctx* passado. Retorna `NULL` se um erro ocorreu.

PyObject *PyContext_CopyCurrent(void)

Retorna valor: Nova referência.

Cria uma cópia rasa do contexto da thread atual. Retorna `NULL` se um erro ocorreu.

int PyContext_Enter(PyObject *ctx)

Defina *ctx* como o contexto atual para o thread atual. Retorna `0` em caso de sucesso e `-1` em caso de erro.

int PyContext_Exit(PyObject *ctx)

Desativa o contexto *ctx* e restaura o contexto anterior como o contexto atual para a thread atual. Retorna `0` em caso de sucesso e `-1` em caso de erro.

int PyContext_AddWatcher(PyContext_WatchCallback callback)

Registra *callback* como um observador do objeto contexto para o interpretador atual. Devolve um ID que pode ser passado para **`PyContext_ClearWatcher()`**. Em caso de erro (por exemplo, não há IDs de observadores disponíveis), devolve `-1` e define uma exceção.

Adicionado na versão 3.14.

int PyContext_ClearWatcher(int watcher_id)

Libera o observador identificado por *watcher_id* anteriormente retornado por **`PyContext_AddWatcher()`** para o interpretador atual. Retorna `0` em caso de sucesso ou `-1` em caso de erro e levanta uma exceção (ex., se o *watcher_id* dado não foi registrado.)

Adicionado na versão 3.14.

type PyContextEvent

Enumeração de possíveis eventos do observador de objeto contexto:

- `Py_CONTEXT_SWITCHED`: O contexto atual foi alternado para um contexto diferente. O objeto passado para o retorno de chamada de observação é o objeto **`contextvars.Context`**, que está atualmente em uso, ou None se não houver contexto atual.

Adicionado na versão 3.14.

typedef int (*PyContext_WatchCallback)(PyContextEvent event, PyObject *obj)

Função de retorno de chamada do observador de objetos contexto. O objeto passado para o retorno de chamada é específico do evento; consulte **PyContextEvent** para obter detalhes.

Se o retorno de chamada retornar com uma exceção definida, ele deverá retornar `-1`; essa exceção será impressa como uma exceção não levantável usando **PyErr_FormatUnraisable()**. Caso contrário, deverá retornar `0`.

É possível que já exista uma exceção pendente definida na entrada da função de retorno. Nesse caso, a função de retorno deve retornar `0` com a mesma exceção ainda definida. Isso significa que a função de retorno não pode chamar nenhuma outra API que possa definir uma exceção, a menos que salve e limpe o estado da exceção primeiro e restaure a exceção antes de retornar.

Adicionado na versão 3.14.

Funções de variável de contexto:

PyObject *PyContextVar_New(const char *name, PyObject *def)

Retorna valor: Nova referência.
Cria um novo objeto `ContextVar`. O parâmetro *name* é usado para fins de introspecção e depuração. O parâmetro *def* especifica um valor padrão para a variável de contexto, ou `NULL` para nenhum padrão. Se ocorrer um erro, esta função retorna `NULL`.

int PyContextVar_Get(PyObject *var, PyObject *default_value, PyObject **value)

Obtém o valor de uma variável de contexto. Retorna `-1` se um erro ocorreu durante a pesquisa, e `0` se nenhum erro ocorreu, se um valor foi encontrado ou não.

Se a variável de contexto foi encontrada, *value* será um ponteiro para ela. Se a variável de contexto *não* foi encontrada, *value* apontará para:

- *default_value*, se não for `NULL`;

- o valor padrão de *var*, se não for `NULL`;
- `NULL`

Exceto para `NULL`, a função retorna uma nova referência.

PyObject *PyContextVar_Set(PyObject *var, PyObject *value)

Retorna valor: Nova referência.

Define o valor de *var* como *value* no contexto atual. Retorna um novo objeto token para esta alteração, ou `NULL` se um erro ocorreu.

int PyContextVar_Reset(PyObject *var, PyObject *token)

Redefine o estado da variável de contexto *var* para o estado que anterior a **`PyContextVar_Set()`** que retornou o *token* foi chamado. Esta função retorna `0` em caso de sucesso e `-1` em caso de erro.

Objetos de indicação de tipos

São fornecidos vários tipos embutidos para sugestão de tipo. Atualmente, dois tipos existem – GenericAlias e Union. Apenas `GenericAlias` está exposto ao C.

PyObject *Py_GenericAlias(PyObject *origin, PyObject *args)

Parte da ABI Estável desde a versão 3.9.

Cria um objeto GenericAlias. Equivalente a chamar a classe Python **`types.GenericAlias`**. Os argumentos *origin* e *args* definem os atributos `__origin__` e `__args__` de `GenericAlias` respectivamente. *origin* deve ser um PyTypeObject*, e *args* pode ser um PyTupleObject* ou qualquer `PyObject*`. Se *args* passado não for uma tupla, uma tupla de 1 elemento é construída automaticamente e `__args__` é definido como `(args,)`. A verificação mínima é feita para os argumentos, então a função terá sucesso mesmo se *origin* não for um tipo. O atributo `__parameters__` de `GenericAlias` é construído lentamente a partir de `__args__`. Em caso de falha, uma exceção é levantada e `NULL` é retornado.

Aqui está um exemplo de como tornar um tipo de extensão genérico:

```
...
static PyMethodDef my_obj_methods[] = {
    // Outros métodos.
    ...
    {"__class_getitem__", Py_GenericAlias, METH_O|METH_CLASS,
    ...
}
```

Ver também

O método de modelo de dados **`__class_getitem__()`**.

Adicionado na versão 3.9.

PyTypeObject Py_GenericAliasType

Parte da ABI Estável desde a versão 3.9.

O tipo C do objeto retornado por `Py_GenericAlias()`. Equivalente a `types.GenericAlias` no Python.

Adicionado na versão 3.9.

API C de curses

curses expõe uma pequena interface C para módulos de extensão. Os usuários devem incluir o arquivo de cabeçalho `py_curses.h` (que não é incluído por padrão em `Python.h`) e `import curses()` deve ser invocada, geralmente como parte da função de inicialização do módulo, para popular `PyCurses_API`.

Aviso

Nem a API C nem o módulo `curses` do Python puro são compatíveis com subinterpretadores.

import_curses()

Importa a API C do curses. A macro não precisa de ponto e vírgula para ser chamada.

Em caso de sucesso, popula o ponteiro `PyCurses_API`.

Em caso de falha, define `PyCurses_API` como NULL e levanta uma exceção. O chamador deve verificar se ocorreu um erro através de `PyErr_Occurred()`:

```
import_curses();  // O ponto e vírgula é opcional, mas recome
if (PyErr_Occurred()) { /* cleanup */ }
```

void **PyCurses_API

Objeto alocado dinamicamente contendo a API C do curses. Esta variável só estará disponível após a execução bem-sucedida da macro `import_curses`.

`PyCurses_API[0]` corresponde a `PyCursesWindow_Type`.

`PyCurses_API[1]`, `PyCurses_API[2]` e `PyCurses_API[3]` são ponteiros para funções predicativas do tipo `int (*)(void)`.

Quando chamados, esses predicados retornam se **`curses.setupterm()`**, **`curses.initscr()`** e **`curses.start_color()`** foram chamados, respectivamente.

Veja também as macros de conveniência **`PyCursesSetupTermCalled`**, **`PyCursesInitialised`** e **`PyCursesInitialisedColor`**.

Nota

O número de entradas nesta estrutura está sujeito a alterações. Considere usar **`PyCurses_API_pointers`** para verificar se novos campos estão disponíveis.

PyCurses_API_pointers

O número de campos acessíveis (`4`) em **`PyCurses_API`**. Esse número é incrementado sempre que novos campos são adicionados.

PyTypeObject PyCursesWindow_Type

O tipo de heap correspondente a **`curses.window`**.

int PyCursesWindow_Check(PyObject *op)

Retorna verdadeiro se *op* for uma instância de **`curses.window`**, falso caso contrário.

As macros a seguir são macros de conveniência que se expandem em instruções C. Em particular, elas só podem ser usadas como `macro;` ou `macro`, mas não como `macro()` ou `macro();`.

PyCursesSetupTermCalled

Macro que verifica se **`curses.setupterm()`** foi chamada.

A expansão de macro é aproximadamente equivalente a:

```
{
    typedef int (*predicate_t)(void);
    predicate_t was_setupterm_called = (predicate_t)PyCurses_
    if (!was_setupterm_called()) {
        return NULL;
    }
}
```

PyCursesInitialised

Macro que verifica se `curses.initscr()` foi chamada.

A expansão de macro é aproximadamente equivalente a:

```
{
    typedef int (*predicate_t)(void);
    predicate_t was_initscr_called = (predicate_t)PyCurses_AP
    if (!was_initscr_called()) {
        return NULL;
    }
}
```

PyCursesInitialisedColor

Macro que verifica se `curses.start_color()` foi chamada.

A expansão de macro é aproximadamente equivalente a:

```
{
    typedef int (*predicate_t)(void);
    predicate_t was_start_color_called = (predicate_t)PyCurse
    if (!was_start_color_called()) {
        return NULL;
    }
}
```

Dados internos

Os seguintes objetos são expostos pela API C, mas devem ser considerados apenas para uso interno.

PyCurses_CAPSULE_NAME

Nome da cápsula do curses a ser passada para `PyCapsule_Import()`.

Uso interno apenas. Use `import curses` em vez disso.

Objetos DateTime

Vários objetos de data e hora são fornecidos pelo módulo `datetime`. Antes de usar qualquer uma dessas funções, o arquivo de cabeçalho `datetime.h` deve ser incluído na sua fonte (observe que isso não é incluído por `Python.h`) e a macro `PyDateTime_IMPORT` deve ser chamada, geralmente como parte da função de inicialização do módulo. A macro coloca um ponteiro para uma estrutura C em uma variável estática, `PyDateTimeAPI`, usada pelas macros a seguir.

PyDateTime_IMPORT()

Importa a API C de datetime.

Em caso de sucesso, preenche o ponteiro `PyDateTimeAPI`. Em caso de falha, define `PyDateTimeAPI` como `NULL` e lança uma exceção. O chamador deve verificar se ocorreu um erro através de `PyErr_Occurred()`:

```
PyDateTime_IMPORT;
if (PyErr_Occurred()) { /* limpeza */ }
```

Aviso

Isso não é compatível com subinterpretadores.

Alterado na versão 3.15: Esta macro agora é seguro para thread.

type PyDateTime_CAPI

Estrutura contendo os campos para a API C de datetime.

Os campos desta estrutura são privados e sujeitos a alterações.

Não use isto diretamente; prefira as APIs `PyDateTime_*`.

PyDateTime_CAPI *PyDateTimeAPI

Objeto alocado dinamicamente contendo a API C de datetime.

Esta variável só estará disponível após a execução bem-sucedida de `PyDateTime_IMPORT`.

Alterado na versão 3.15: Esta variável não deve ser acessada diretamente, pois acesso direto não é seguro para thread. Use `PyDateTime_IMPORT()` em vez disso.

type PyDateTime_Date

Esta é uma instância de `PyObject` representando o objeto data do Python.

type PyDateTime_DateTime

Esta é uma instância de `PyObject` representando o objeto datetime do Python.

type PyDateTime_Time

Esta é uma instância de `PyObject` representando o objeto hora do Python.

type PyDateTime_Delta

Esta é uma instância de `PyObject` representando a diferença entre dois valores de datetime.

PyTypeObject PyDateTime_DateType

Esta instância de `PyTypeObject` representa o tipo data do Python; é o mesmo objeto que `datetime.date` na camada de Python.

PyTypeObject PyDateTime_DateTimeType

Esta instância de `PyTypeObject` representa o tipo datetime do Python; é o mesmo objeto que `datetime.datetime` na camada de Python.

PyTypeObject PyDateTime_TimeType

Esta instância de `PyTypeObject` representa o tipo hora do Python; é o mesmo objeto que `datetime.time` na camada de Python.

PyTypeObject PyDateTime_DeltaType

Esta instância de **`PyTypeObject`** representa o tipo Python para a diferença entre dois valores de datetime; é o mesmo objeto que **`datetime.timedelta`** na camada de Python.

PyTypeObject PyDateTime_TZInfoType

Esta instância de **`PyTypeObject`** representa o tipo fuso horário do Python; é o mesmo objeto que **`datetime.tzinfo`** na camada de Python.

Macro para acesso ao singleton UTC:

PyObject *PyDateTime_TimeZone_UTC

Retorna um singleton do fuso horário representando o UTC, o mesmo objeto que **`datetime.timezone.utc`**.

Adicionado na versão 3.7.

Macros de verificação de tipo:

int PyDate_Check(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **`PyDateTime_DateType`** ou um subtipo de **`PyDateTime_DateType`**. *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyDate_CheckExact(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **`PyDateTime_DateType`**. *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyDateTime_Check(PyObject *ob)

Retorna verdadeiro se *ob* é do tipo **`PyDateTime_DateTimeType`** ou um subtipo de **`PyDateTime_DateTimeType`**. *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyDateTime_CheckExact(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **`PyDateTime_DateTimeType`**. *ob* não deve ser `NULL`. Esta função sempre tem sucesso.

int PyTime_Check(PyObject *ob)

Retorna verdadeiro se *ob* é do tipo **PyDateTime_TimeType** ou um subtipo de **PyDateTime_TimeType**. *ob* não deve ser NULL. Esta função sempre tem sucesso.

int PyTime_CheckExact(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **PyDateTime_TimeType**. *ob* não deve ser NULL. Esta função sempre tem sucesso.

int PyDelta_Check(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **PyDateTime_DeltaType** ou um subtipo de **PyDateTime_DeltaType**. *ob* não pode ser NULL. Esta função sempre tem sucesso.

int PyDelta_CheckExact(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **PyDateTime_DeltaType**. *ob* não deve ser NULL. Esta função sempre tem sucesso.

int PyTZInfo_Check(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **PyDateTime_TZInfoType** ou um subtipo de **PyDateTime_TZInfoType**. *ob* não pode ser NULL. Esta função sempre tem sucesso.

int PyTZInfo_CheckExact(PyObject *ob)

Retorna verdadeiro se *ob* for do tipo **PyDateTime_TZInfoType**. *ob* não deve ser NULL. Esta função sempre tem sucesso.

Macros para criar objetos:

PyObject *PyDate_FromDate(int year, int month, int day)

Retorna valor: Nova referência.

Retorna um objeto **datetime.date** com o ano, mês e dia especificados.

PyObject *PyDateTime_FromDateAndTime(int year, int month, int day, int hour, int minute, int second, int usecond)

Retorna valor: Nova referência.

Retorna um objeto **`datetime.datetime`** com o ano, mês, dia, hora, minuto, segundo, microssegundo especificados.

PyObject *PyDateTime_FromDateAndTimeAndFold(int year, int month, int day, int hour, int minute, int second, int usecond, int fold)

Retorna valor: Nova referência.

Retorna um objeto **`datetime.datetime`** com o ano, mês, dia, hora, minuto, segundo, microssegundo e a dobra especificados.

Adicionado na versão 3.6.

PyObject *PyTime_FromTime(int hour, int minute, int second, int usecond)

Retorna valor: Nova referência.

Retorna um objeto **`datetime.time`** com a hora, minuto, segundo e microssegundo especificados.

PyObject *PyTime_FromTimeAndFold(int hour, int minute, int second, int usecond, int fold)

Retorna valor: Nova referência.

Retorna um objeto **`datetime.time`** com a hora, minuto, segundo, microssegundo e a dobra especificados.

Adicionado na versão 3.6.

PyObject *PyDelta_FromDSU(int days, int seconds, int useconds)

Retorna valor: Nova referência.

Retorna um objeto **`datetime.timedelta`** representando o número especificado de dias, segundos e microssegundos. A normalização é realizada para que o número resultante de microssegundos e segundos esteja nos intervalos documentados para objetos de **`datetime.timedelta`**.

PyObject *PyTimeZone_FromOffset(PyObject *offset)

Retorna valor: Nova referência.

Retorna um objeto **`datetime.timezone`** com um deslocamento fixo sem nome representado pelo argumento *offset*.

Adicionado na versão 3.7.

PyObject *PyTimeZone_FromOffsetAndName(PyObject *offset, PyObject *name)

Retorna valor: Nova referência.

Retorna um objeto `datetime.timezone` com um deslocamento fixo representado pelo argumento *offset* e com tzname *name*.

Adicionado na versão 3.7.

Macros para extrair campos de objetos *date*. O argumento deve ser uma instância de `PyDateTime_Date`, inclusive subclasses (como `PyDateTime_DateTime`). O argumento não deve ser `NULL`, e o tipo não é verificado:

int PyDateTime_GET_YEAR(PyDateTime_Date *o)

Retorna o ano, como um inteiro positivo.

int PyDateTime_GET_MONTH(PyDateTime_Date *o)

Retorna o mês, como um inteiro de 1 a 12.

int PyDateTime_GET_DAY(PyDateTime_Date *o)

Retorna o dia, como um inteiro de 1 a 31.

Macros para extrair campos de objetos *datetime*. O argumento deve ser uma instância de `PyDateTime_DateTime`, inclusive subclasses. O argumento não deve ser `NULL`, e o tipo não é verificado:

int PyDateTime_DATE_GET_HOUR(PyDateTime_DateTime *o)

Retorna a hora, como um inteiro de 0 a 23.

int PyDateTime_DATE_GET_MINUTE(PyDateTime_DateTime *o)

Retorna o minuto, como um inteiro de 0 a 59.

int PyDateTime_DATE_GET_SECOND(PyDateTime_DateTime *o)

Retorna o segundo, como um inteiro de 0 a 59.

int PyDateTime_DATE_GET_MICROSECOND(PyDateTime_DateTime *o)

Retorna o microssegundo, como um inteiro de 0 a 999999.

int PyDateTime_DATE_GET_FOLD(PyDateTime_DateTime *o)

Retorna a dobra, como um inteiro de 0 a 1.

Adicionado na versão 3.6.

PyObject *PyDateTime_DATE_GET_TZINFO(PyDateTime_DateTime *o)

Retorna o tzinfo (que pode ser `None`).

Adicionado na versão 3.10.

Macros para extrair campos de objetos *time*. O argumento deve ser uma instância de **`PyDateTime_Time`**, inclusive subclasses. O argumento não deve ser `NULL`, e o tipo não é verificado:

int PyDateTime_TIME_GET_HOUR(PyDateTime_Time *o)

Retorna a hora, como um inteiro de 0 a 23.

int PyDateTime_TIME_GET_MINUTE(PyDateTime_Time *o)

Retorna o minuto, como um inteiro de 0 a 59.

int PyDateTime_TIME_GET_SECOND(PyDateTime_Time *o)

Retorna o segundo, como um inteiro de 0 a 59.

int PyDateTime_TIME_GET_MICROSECOND(PyDateTime_Time *o)

Retorna o microssegundo, como um inteiro de 0 a 999999.

int PyDateTime_TIME_GET_FOLD(PyDateTime_Time *o)

Retorna a dobra, como um inteiro de 0 a 1.

Adicionado na versão 3.6.

PyObject *PyDateTime_TIME_GET_TZINFO(PyDateTime_Time *o)

Retorna o tzinfo (que pode ser `None`).

Adicionado na versão 3.10.

Macros para extrair campos de objetos *time delta*. O argumento deve ser uma instância de **`PyDateTime_Delta`**, inclusive subclasses. O argumento não deve ser `NULL`, e o tipo não é verificado:

int PyDateTime_DELTA_GET_DAYS(PyDateTime_Delta *o)

Retorna o número de dias, como um inteiro de -999999999 a 999999999.

Adicionado na versão 3.3.

int PyDateTime_DELTA_GET_SECONDS(PyDateTime_Delta *o)

Retorna o número de segundos, como um inteiro de 0 a 86399.

Adicionado na versão 3.3.

int PyDateTime_DELTA_GET_MICROSECONDS(PyDateTime_Delta *o)

Retorna o número de microssegundos, como um inteiro de 0 a 999999.

Adicionado na versão 3.3.

Macros para a conveniência de módulos implementando a API de DB:

PyObject *PyDateTime_FromTimestamp(PyObject *args)

Retorna valor: Nova referência.

Cria e retorna um novo objeto **`datetime.datetime`** dado uma tupla de argumentos que possa ser passada ao método **`datetime.datetime.fromtimestamp()`**.

PyObject *PyDate_FromTimestamp(PyObject *args)

Retorna valor: Nova referência.

Cria e retorna um novo objeto **`datetime.date`** dado uma tupla de argumentos que possa ser passada ao método **`datetime.date.fromtimestamp()`**.

Dados internos

Os seguintes símbolos são expostos pela API C, mas devem ser considerados apenas para uso interno.

PyDateTime_CAPSULE_NAME

Nome da cápsula de datetime a ser passada para `PyCapsule_Import()`.

Uso interno apenas. Use `PyDateTime_IMPORT` em vez disso.

Interpreter initialization and finalization

Veja Configuração de inicialização do Python para detalhes sobre como configurar o interpretador antes da inicialização.

Before Python initialization

In an application embedding Python, the `Py_Initialize()` function must be called before using any other Python/C API functions; with the exception of a few functions and the global configuration variables.

As seguintes funções podem ser seguramente chamadas antes da inicialização do Python.

- Funções que inicializam o interpretador:
  - `Py_Initialize()`
  - `Py_InitializeEx()`
  - `Py_InitializeFromConfig()`
  - `Py_BytesMain()`
  - `Py_Main()`
  - as funções de pré-inicialização de tempo de execução cobertas em Configuração de inicialização do Python
- Funções de configuração
  - `PyImport_AppendInittab()`
  - `PyImport_ExtendInittab()`
  - `PyInitFrozenExtensions()`
  - `PyMem_SetAllocator()`
  - `PyMem_SetupDebugHooks()`
  - `PyObject_SetArenaAllocator()`
  - `Py_SetProgramName()`
  - `Py_SetPythonHome()`

    - as funções de configuração cobertas em Configuração de inicialização do Python
- Funções informativas:
    - `Py_IsInitialized()`
    - `PyMem_GetAllocator()`
    - `PyObject_GetArenaAllocator()`
    - `Py_GetBuildInfo()`
    - `Py_GetCompiler()`
    - `Py_GetCopyright()`
    - `Py_GetPlatform()`
    - `Py_GetVersion()`
    - `Py_IsInitialized()`
- Utilitários:
    - `Py_DecodeLocale()`
    - o relatório de status e funções utilitárias cobertas em Configuração de inicialização do Python
- Alocadores de memória:
    - `PyMem_RawMalloc()`
    - `PyMem_RawRealloc()`
    - `PyMem_RawCalloc()`
    - `PyMem_RawFree()`
- Sincronização:
    - `PyMutex_Lock()`
    - `PyMutex_Unlock()`

Nota

Despite their apparent similarity to some of the functions listed above, the following functions **should not be called** before the interpreter has been initialized: `Py_EncodeLocale()`, `PyEval_InitThreads()`, and `Py_RunMain()`.

Variáveis de configuração global

Python tem variáveis para a configuração global a fim de controlar diferentes características e opções. Por padrão, estes sinalizadores são controlados por opções de linha de comando.

Quando um sinalizador é definido por uma opção, o valor do sinalizador é o número de vezes que a opção foi definida. Por exemplo, `-b` define **`Py_BytesWarningFlag`** para 1 e `-bb` define **`Py_BytesWarningFlag`** para 2.

int Py_BytesWarningFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **`PyConfig.bytes_warning`** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Emite um aviso ao comparar **`bytes`** ou **`bytearray`** com **`str`** ou **`bytes`** com **`int`**. Emite um erro se for maior ou igual a `2`.

Definida pela opção **`-b`**.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_DebugFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **`PyConfig.parser_debug`** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Ativa a saída de depuração do analisador sintático (somente para especialistas, dependendo das opções de compilação).

Definida pela a opção **`-d`** e a variável de ambiente **`PYTHONDEBUG`**.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_DontWriteBytecodeFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **`PyConfig.write_bytecode`** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Se definida como diferente de zero, o Python não tentará escrever arquivos `.pyc` na importação de módulos fonte.

Definida pela opção **`-B`** e pela variável de ambiente **`PYTHONDONTWRITEBYTECODE`**.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_FrozenFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **`PyConfig.pathconfig_warnings`** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Sinalizador privado usado pelos programas `_freeze_module` e `frozenmain`.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_HashRandomizationFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração de **`PyConfig.hash_seed`** e **`PyConfig.use_hash_seed`** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Definida como `1` se a variável de ambiente **`PYTHONHASHSEED`** estiver definida como uma string não vazia.

Se o sinalizador for diferente de zero, lê a variável de ambiente **`PYTHONHASHSEED`** para inicializar a semente de hash secreta.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_IgnoreEnvironmentFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **`PyConfig.use_environment`** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Ignora todas as variáveis de ambiente **PYTHON***, por exemplo **PYTHONPATH** e **PYTHONHOME**, que podem estar definidas.

Definida pelas opções **-E** e **-I**.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_InspectFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **PyConfig.inspect** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Quando um script é passado como primeiro argumento ou a opção **-c** é usada, entre no modo interativo após executar o script ou o comando, mesmo quando **sys.stdin** não parece ser um terminal.

Definida pela opção **-i** e pela variável de ambiente **PYTHONINSPECT**.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_InteractiveFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **PyConfig.interactive** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Definida pela opção **-i**.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_IsolatedFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração **PyConfig.isolated** deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Executa o Python no modo isolado. No modo isolado, **sys.path** não contém nem o diretório do script nem o diretório de pacotes de sites do usuário.

Definida pela opção `-I`.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_LegacyWindowsFSEncodingFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyPreConfig.legacy_windows_fs_encoding` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Se o sinalizador for diferente de zero, use a codificação `mbcs` com o tratador de erros `replace`, em vez da codificação UTF-8 com o tratador de erros `surrogatepass`, para a codificação do sistema de arquivos e tratador de erros e codificação do sistema de arquivos.

Definida como `1` se a variável de ambiente `PYTHONLEGACYWINDOWSFSENCODING` estiver definida como uma string não vazia.

Veja **PEP 529** [https://peps.python.org/pep-0529/] para mais detalhes.

Disponibilidade: Windows.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_LegacyWindowsStdioFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.legacy_windows_stdio` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Se o sinalizador for diferente de zero, usa `io.FileIO` em vez de `io._WindowsConsoleIO` para fluxos padrão `sys`.

Definida como `1` se a variável de ambiente `PYTHONLEGACYWINDOWSSTDIO` estiver definida como uma string não vazia.

Veja a **PEP 528** [https://peps.python.org/pep-0528/] para mais detalhes.

Disponibilidade: Windows.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_NoSiteFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.site_import` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Desabilita a importação do módulo `site` e as manipulações dependentes do site de `sys.path` que isso acarreta. Também desabilita essas manipulações se `site` for explicitamente importado mais tarde (chame `site.main()` se você quiser que eles sejam acionados).

Definida pela opção `-S`.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_NoUserSiteDirectory

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.user_site_directory` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Não adiciona o **`diretório site-packages de usuário <site.USER_SITE>`** a `sys.path`.

Definida pelas opções `-s` e `-I`, e pela variável de ambiente `PYTHONNOUSERSITE`.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_OptimizeFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.optimization_level` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Definida pela opção `-O` e pela variável de ambiente `PYTHONOPTIMIZE`.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_QuietFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.quiet` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Não exibe as mensagens de direitos autorais e de versão nem mesmo no modo interativo.

Definida pela opção `-q`.

Adicionado na versão 3.2.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_UnbufferedStdioFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.buffered_stdio` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Força os fluxos stdout e stderr a não serem armazenados em buffer.

Definida pela opção `-u` e pela variável de ambiente `PYTHONUNBUFFERED`.

Descontinuado desde a versão 3.12, removido na versão 3.15.

int Py_VerboseFlag

Esta API é mantida para compatibilidade com versões anteriores: a configuração `PyConfig.verbose` deve ser usada em seu lugar, consulte Configuração de inicialização do Python.

Exibe uma mensagem cada vez que um módulo é inicializado, mostrando o local (nome do arquivo ou módulo embutido) de onde ele é carregado. Se maior ou igual a `2`, exibe uma mensagem para cada arquivo que é verificado durante a busca por um módulo. Também fornece informações sobre a limpeza do módulo na saída.

Definida pela a opção `-v` e a variável de ambiente `PYTHONVERBOSE`.

Descontinuado desde a versão 3.12, removido na versão 3.15.

Inicializando e encerrando o interpretador

void Py_Initialize()

Parte da ABI Estável.
Initialize the Python interpreter. In an application embedding Python, this should be called before using any other Python/C API functions; see Before Python Initialization for the few exceptions.

Isso inicializa a tabela de módulos carregados (`sys.modules`) e cria os módulos fundamentais `builtins`, `__main__` e `sys`. Também inicializa o caminho de pesquisa de módulos (`sys.path`). Isso não define `sys.argv`; use a API da Configuração de inicialização do Python para isso. Isso é um no-op quando chamado pela segunda vez (sem chamar `Py_FinalizeEx()` primeiro). Não há valor de retorno; é um erro fatal se a inicialização falhar.

Usa `Py_InitializeFromConfig()` para personalizar a Configuração de inicialização do Python.

Nota

No Windows, altera o modo do console de `O_TEXT` para `O_BINARY`, o que também afetará usos não Python do console usando o Runtime C.

void Py_InitializeEx(int initsigs)

Parte da ABI Estável.
Esta função funciona como `Py_Initialize()` se *initsigs* for `1`. Se *initsigs* for `0`, ela pula o registro de inicialização de manipuladores de sinal, o que pode ser útil quando o CPython é incorporado como parte de uma aplicação maior.

Usa `Py_InitializeFromConfig()` para personalizar a Configuração de inicialização do Python.

PyStatus Py_InitializeFromConfig(const PyConfig *config)

Inicializa o Python a partir da configuração *config*, conforme descrito em Initialization with PyConfig.

Consulte a seção Configuração de inicialização do Python para obter detalhes sobre como pré-inicializar o interpretador, preencher a estrutura de configuração do tempo de execução e consultar a estrutura de status retornada.

int Py_IsInitialized()

Parte da ABI Estável.
Retorna verdadeiro (diferente de zero) quando o interpretador Python foi inicializado, falso (zero) se não. Após `Py_FinalizeEx()` ser chamado, isso retorna falso até que `Py_Initialize()` seja chamado novamente.

Alterado na versão 3.15: This function no longer returns true until initialization has fully completed, including import of the `site` module. Previously it could return true while `Py_Initialize()` was still running.

int Py_IsFinalizing()

Parte da ABI Estável desde a versão 3.13.
Retorna verdadeiro (diferente de zero) se o interpretador Python principal estiver em desligamento. Retorna falso (zero) caso contrário.

Adicionado na versão 3.13.

int Py_FinalizeEx()

Parte da ABI Estável desde a versão 3.6.
Desfaz todas as inicializações feitas por `Py_Initialize()` e o uso subsequente de funções da API Python/C, e destrói todos os subinterpretadores (veja `Py_NewInterpreter()` abaixo) que foram criados e ainda não destruídos desde a última chamada a

`Py_Initialize()`. Esta operação é ineficaz quando chamada pela segunda vez (sem chamar **`Py_Initialize()`** novamente primeiro).

Como isso é o inverso de **`Py_Initialize()`**, ele deve ser chamado na mesma thread com o mesmo interpretador ativo. Isso significa a thread principal e o interpretador principal. Isso nunca deve ser chamado enquanto **`Py_RunMain()`** estiver em execução.

Normalmente, o valor de retorno é `0`. Se houver erros durante a finalização (limpeza de dados armazenados em buffer), `-1` será retornado.

Note that Python will do a best effort at freeing all memory allocated by the Python interpreter. Therefore, any C-Extension should make sure to correctly clean up all of the previously allocated PyObjects before using them in subsequent calls to **`Py_Initialize()`**. Otherwise it could introduce vulnerabilities and incorrect behavior.

Esta função é fornecida por vários motivos. Uma aplicação de incorporação pode querer reiniciar o Python sem precisar reiniciar a própria aplicação. Uma aplicação que carregou o interpretador Python de uma biblioteca carregável dinamicamente (ou DLL) pode querer liberar toda a memória alocada pelo Python antes de descarregar a DLL. Durante uma busca por vazamentos de memória em uma aplicação, um desenvolvedor pode querer liberar toda a memória alocada pelo Python antes de sair da aplicação.

Bugs e advertências: A destruição de módulos e objetos em módulos é feita em ordem aleatória; isso pode fazer com que destrutores (métodos **`__del__()`**) falhem quando dependem de outros objetos (até mesmo funções) ou módulos. Módulos de extensão carregados dinamicamente pelo Python não são descarregados. Pequenas quantidades de memória alocadas pelo interpretador Python podem não ser liberadas (se você encontrar um vazamento, por favor, reporte-o). Memória presa em referências circulares entre objetos não é liberada. Strings internadas serão todas desalocadas independentemente de sua contagem de referências. Parte da memória alocada por módulos de extensão pode não ser liberada. Algumas extensões podem não funcionar corretamente

se sua rotina de inicialização for chamada mais de uma vez; isso pode acontecer se uma aplicação chamar **Py_Initialize()** e **Py_FinalizeEx()** mais de uma vez. **Py_FinalizeEx()** não deve ser chamado recursivamente de dentro de si mesmo. Portanto, ele não deve ser chamado por nenhum código que possa ser executado como parte do processo de desligamento do interpretador, como manipuladores **atexit**, finalizadores de objetos ou qualquer código que possa ser executado durante a limpeza dos arquivos de stdout e stderr.

Levanta um evento de auditoria `cpython._PySys_ClearAuditHooks` sem argumentos.

Adicionado na versão 3.6.

void Py_Finalize()

Parte da ABI Estável.
Esta é uma versão retrocompatível de **Py_FinalizeEx()** que desconsidera o valor de retorno.

int Py_BytesMain(int argc, char **argv)

Parte da ABI Estável desde a versão 3.8.
Semelhante a **Py_Main()**, mas *argv* é um vetor de strings de bytes, permitindo que a aplicação chamadora delegue a etapa de decodificação de texto ao ambiente de execução do CPython.

Adicionado na versão 3.8.

int Py_Main(int argc, wchar_t **argv)

Parte da ABI Estável.
O programa principal para o interpretador padrão, encapsulando um ciclo completo de inicialização/finalização, bem como comportamento adicional para implementar a leitura de configurações do ambiente e da linha de comando e, em seguida, executar `__main__` de acordo com a Linha de comando.

Isso é disponibilizado para programas que desejam oferecer suporte à interface de linha de comando completa do CPython, em vez de apenas

incorporar um ambiente de execução do Python em uma aplicação maior.

Os parâmetros *argc* e *argv* são semelhantes aos passados para a função **`main()`** de um programa C, exceto que as entradas *argv* são primeiro convertidas para `wchar_t` usando **`Py_DecodeLocale()`**. Também é importante observar que as entradas da lista de argumentos podem ser modificadas para apontar para strings diferentes daquelas passadas (no entanto, o conteúdo das strings apontadas pela lista de argumentos não é modificado).

O valor de retorno é `2` se a lista de argumentos não representar uma linha de comando Python válida e, caso contrário, o mesmo que **`Py_RunMain()`**.

Em termos das APIs de configuração de tempo de execução do CPython documentadas na seção de configuração do ambiente de execução (e sem levar em conta o tratamento de erros), `Py_Main` é aproximadamente equivalente a:

```
PyConfig config;
PyConfig_InitPythonConfig(&config);
PyConfig_SetArgv(&config, argc, argv);
Py_InitializeFromConfig(&config);
PyConfig_Clear(&config);

Py_RunMain();
```

Em uso normal, uma aplicação de incorporação chamará esta função *em vez* de chamar **`Py_Initialize()`**, **`Py_InitializeEx()`** ou **`Py_InitializeFromConfig()`** diretamente, e todas as configurações serão aplicadas conforme descrito em outra parte desta documentação. Se esta função for chamada *após* uma chamada anterior à API de inicialização do ambiente de execução, as configurações de ambiente e de linha de comando que serão atualizadas dependem da versão (pois dependem de quais configurações oferecem suporte corretamente à modificação após já terem sido definidas uma vez na primeira inicialização do ambiente de execução).

int Py_RunMain(void)

Executa o módulo principal em um ambiente de execução CPython totalmente configurado.

Executa o comando (`PyConfig.run_command`), o script (`PyConfig.run_filename`) ou o módulo (`PyConfig.run_module`) especificado na linha de comando ou na configuração. Se nenhum desses valores estiver definido, executa o prompt interativo do Python (REPL) usando o espaço de nomes global do módulo `__main__`.

Se `PyConfig.inspect` não estiver definido (o padrão), o valor de retorno será `0` se o interpretador terminar normalmente (ou seja, sem levantar uma exceção), o status de saída de uma exceção `SystemExit` não tratada ou `1` para qualquer outra exceção não tratada.

Se `PyConfig.inspect` estiver definido (como quando a opção `-i` é usada), em vez de retornar quando o interpretador for encerrado, a execução será retomada em um prompt interativo do Python (REPL) usando o espaço de nomes global do módulo `__main__`. Se o interpretador for encerrado com uma exceção, ela será imediatamente levantada na sessão do REPL. O valor de retorno da função é então determinado pela forma como a *sessão do REPL* termina: `0`, `1` ou o status de uma exceção `SystemExit`, conforme especificado acima.

Essa função sempre finaliza o interpretador Python antes de retornar.

Consulte a Configuração do Python para obter um exemplo de um Python personalizado que sempre é executado em modo isolado usando `Py_RunMain()`.

int PyUnstable_AtExit(PyInterpreterState *interp, void (*func)(void*), void *data)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Registra uma função de retorno de chamada `atexit` para o interpretador de destino *interp*. Isso é semelhante a `Py_AtExit()`, mas recebe um

interpretador explícito e um ponteiro de dados para a função de retorno de chamada.

Deve haver um estado de thread anexado para *interp*.

Adicionado na versão 3.13.

Cautions regarding interpreter finalization

In the late stage of interpreter shutdown, after attempting to wait for non-daemon threads to exit (though this can be interrupted by `KeyboardInterrupt`) and running the `atexit` functions, the runtime is marked as finalizing, meaning that `Py_IsFinalizing()` and `sys.is_finalizing()` return true. At this point, only the finalization thread (the thread that initiated finalization; this is typically the main thread) is allowed to attach a thread state.

Other threads that attempt to attach during finalization, either explicitly (such as via `PyThreadState_Ensure()` or `Py_END_ALLOW_THREADS`) or implicitly (such as in-between bytecode instructions), will enter a **permanently blocked state**. Generally, this is harmless, but this can result in deadlocks. For example, a thread may be permanently blocked while holding a lock, meaning that the finalization thread can never acquire that lock.

Prior to CPython 3.13, the thread would exit instead of hanging, which led to other issues (see the warning note at `PyThread_exit_thread()`).

Gross? Yes. Starting in Python 3.15, there are a number of C APIs that make it possible to avoid these issues by temporarily preventing finalization:

Ver também

PEP 788 [https://peps.python.org/pep-0788/] explains the design, motivation and rationale for these APIs.

type PyInterpreterGuard

Parte da ABI Estável (como uma estrutura opaca) desde a versão 3.15.
An opaque interpreter guard structure.

By holding an interpreter guard, the caller can ensure that the interpreter will not finalize until the guard is closed (through `PyInterpreterGuard_Close()`).

When a guard is held, a thread attempting to finalize the interpreter will block until the guard is closed before starting finalization. After finalization has started, threads are forever unable to acquire guards for that interpreter. This means that if you forget to close an interpreter guard, the process will **permanently hang** during finalization!

Holding a guard for an interpreter is similar to holding a strong reference to a Python object, except finalization does not happen automatically after all guards are released: it requires an explicit `Py_EndInterpreter()` call.

Adicionado na versão 3.15.

PyInterpreterGuard *PyInterpreterGuard_FromCurrent(void)

Parte da ABI Estável desde a versão 3.15.
Create a finalization guard for the current interpreter. This will prevent finalization until the guard is closed.

Por exemplo:

```
// Temporarily prevent finalization.
PyInterpreterGuard *guard = PyInterpreterGuard_FromCurrent();
if (guard == NULL) {
   // Finalization has already started or we're out of memory
   return NULL;
}

Py_BEGIN_ALLOW_THREADS;
// Do some critical processing here. For example, we can safe
// locks that might be acquired by the finalization thread.
Py_END_ALLOW_THREADS;
```

```
// Now that we're done with our critical processing, the inte
// allowed to finalize again.
PyInterpreterGuard_Close(guard);
```

On success, this function returns a guard for the current interpreter; on failure, it returns `NULL` with an exception set.

This function will fail only if the current interpreter has already started finalizing, or if the process is out of memory.

The guard pointer returned by this function must be eventually closed with **`PyInterpreterGuard_Close()`**; failing to do so will result in the Python process infinitely hanging.

The caller must hold an attached thread state.

Adicionado na versão 3.15.

PyInterpreterGuard *PyInterpreterGuard_FromView(PyInterpreterView *view)

Parte da ABI Estável desde a versão 3.15.
Create a finalization guard for an interpreter through a view.

On success, this function returns a guard to the interpreter represented by *view*. The view is still valid after calling this function. The guard must eventually be closed with **`PyInterpreterGuard_Close()`**.

If the interpreter no longer exists, is already finalizing, or out of memory, then this function returns `NULL` without setting an exception.

The caller does not need to hold an attached thread state.

Adicionado na versão 3.15.

void PyInterpreterGuard_Close(PyInterpreterGuard *guard)

Parte da ABI Estável desde a versão 3.15.

Close an interpreter guard, allowing the interpreter to start finalization if no other guards remain. If an interpreter guard is never closed, the interpreter will infinitely wait when trying to enter finalization!

After an interpreter guard is closed, it may not be used in `PyThreadState_Ensure()`. Doing so will result in undefined behavior.

This function cannot fail, and the caller doesn't need to hold an attached thread state.

Adicionado na versão 3.15.

Interpreter views

In some cases, it may be necessary to access an interpreter that may have been deleted. This can be done using interpreter views.

type PyInterpreterView

Parte da ABI Estável (como uma estrutura opaca) desde a versão 3.15.
An opaque view of an interpreter.

This is a thread-safe way to access an interpreter that may have be finalizing or already destroyed.

Adicionado na versão 3.15.

PyInterpreterView *PyInterpreterView_FromCurrent(void)

Parte da ABI Estável desde a versão 3.15.
Create a view to the current interpreter.

This function is generally meant to be used alongside `PyInterpreterGuard_FromView()` or `PyThreadState_EnsureFromView()`.

On success, this function returns a view to the current interpreter; on failure, it returns `NULL` with an exception set.

The caller must hold an attached thread state.

Adicionado na versão 3.15.

void PyInterpreterView_Close(PyInterpreterView *view)

Parte da ABI Estável desde a versão 3.15.
Close an interpreter view.

If an interpreter view is never closed, the view's memory will never be freed, but there are no other consequences. (In contrast, forgetting to close a guard will infinitely hang the main thread during finalization.)

This function cannot fail, and the caller doesn't need to hold an attached thread state.

Adicionado na versão 3.15.

PyInterpreterView *PyInterpreterView_FromMain(void)

Parte da ABI Estável desde a versão 3.15.
Create a view for the main interpreter (the first and default interpreter in a Python process; see **`PyInterpreterState_Main()`**).

On success, this function returns a view to the main interpreter; on failure, it returns `NULL` without an exception set. Failure indicates that the process is out of memory.

Use this function when an interpreter pointer or view cannot be supplied by the caller, such as when a native threading library does not provide a `void *arg` parameter that could carry a **`PyInterpreterGuard`** or **`PyInterpreterView`**. In code that supports subinterpreters, prefer **`PyInterpreterView_FromCurrent()`** so the guard tracks the calling interpreter rather than the main one.

The caller does not need to hold an attached thread state.

Adicionado na versão 3.15.

Parâmetros de todo o processo

void Py_SetProgramName(const wchar_t *name)

Parte da ABI Estável.

Esta API é mantida para compatibilidade com versões anteriores: em vez disso, deve-se usar a configuração **`PyConfig.program_name`**, consulte Configuração de inicialização do Python.

This function should be called before **`Py_Initialize()`** is called for the first time, if it is called at all. It tells the interpreter the value of the `argv[0]` argument to the **`main()`** function of the program (converted to wide characters). This is used by some other functions below to find the Python run-time libraries relative to the interpreter executable. The default value is `'python'`. The argument should point to a zero-terminated wide character string in static storage whose contents will not change for the duration of the program's execution. No code in the Python interpreter will change the contents of this storage.

Use **`Py_DecodeLocale()`** para decodificar uma string de bytes e obter uma string wchar_t*.

Descontinuado desde a versão 3.11, removido na versão 3.15.

const char *Py_GetVersion()

Parte da ABI Estável.

Retorna a verão deste interpretador Python. Esta é uma string que se parece com

```
"3.0a5+ (py3k:63103M, May 12 2008, 00:53:55) \n[GCC 4.2.3]"
```

A primeira palavra (até o primeiro espaço em branco) representa a versão atual do Python; os primeiros caracteres indicam a versão principal e a versão secundária, separados por um ponto. A string retornada aponta para um armazenamento estático; o chamador não deve modificar seu valor. O valor fica disponível para o código Python como **`sys.version`**.

Veja também a constante `Py_Version`.

const char *Py_GetPlatform()

Parte da ABI Estável.

Retorna o identificador da plataforma atual. No Unix, este identificador é formado pelo nome “oficial” do sistema operacional, convertido para minúsculas, seguido pelo número da revisão principal; por exemplo, para Solaris 2.x, também conhecido como SunOS 5.x, o valor é `'sunos5'`. No macOS, é `'darwin'`. No Windows, é `'win'`. A string retornada aponta para um armazenamento estático; o chamador não deve modificar seu valor. O valor está disponível para o código Python como `sys.platform`.

const char *Py_GetCopyright()

Parte da ABI Estável.

Retorna a string oficial de direitos autoriais para a versão atual do Python, por exemplo

```
'Copyright 1991-1995 Stichting Mathematisch Centrum,
Amsterdam'
```

A string retornada aponta para o armazenamento estático; o chamador não deve modificar o seu valor. O valor está disponível para o código Python como `sys.copyright`.

const char *Py_GetCompiler()

Parte da ABI Estável.

Retorna uma indicação do compilador usado para construir a atual versão do Python, em colchetes, por exemplo:

```
"[GCC 2.7.2.2]"
```

A string retornada aponta para o armazenamento estático; o chamador não deve modificar o seu valor. O valor está disponível para o código Python como `sys.version`.

const char *Py_GetBuildInfo()

Parte da ABI Estável.
Return information about the sequence number and build date and time of the current Python interpreter instance, for example

```
"#67, Aug  1 1997, 22:34:28"
```

A string retornada aponta para o armazenamento estático; o chamador não deve modificar o seu valor. O valor está disponível para o código Python como `sys.version`.

void PySys_SetArgvEx(int argc, wchar_t **argv, int updatepath)

Parte da ABI Estável.
Esta API é mantida para compatibilidade com versões anteriores: em vez disso, deve-se usar a configuração **`PyConfig.argv`**, **`PyConfig.parse_argv`** e **`PyConfig.safe_path`**, consulte Configuração de inicialização do Python.

Define **`sys.argv`** com base em *argc* e *argv*. Esses parâmetros são semelhantes aos passados para a função **`main()`** do programa, com a diferença de que a primeira entrada deve se referir ao arquivo de script a ser executado, em vez do executável que hospeda o interpretador Python. Se não houver um script a ser executado, a primeira entrada em *argv* pode ser uma string vazia. Se esta função falhar ao inicializar **`sys.argv`**, uma condição fatal será sinalizada usando **`Py_FatalError()`**.

Se *updatepath* é zero, isto é tudo o que a função faz. Se *updatepath* não é zero, a função também modifica **`sys.path`** de acordo com o seguinte algoritmo:

- Se o nome de um script existente for passado em `argv[0]`, o caminho absoluto do diretório onde o script está localizado será adicionado a **`sys.path`**.
- Caso contrário (isto é, se *argc* for `0` ou `argv[0]` não apontar para um nome de arquivo existente), uma string vazia é adicionada ao início de **`sys.path`**, o que é o mesmo que adicionar o diretório de trabalho atual (`"."`).

Use `Py_DecodeLocale()` para decodificar uma string de bytes e obter uma string wchar_t*.

Veja também os membros `PyConfig.orig_argv` e `PyConfig.argv` da Configuração de inicialização do Python.

Nota

Recomenda-se que aplicações que incorporam o interpretador Python para fins que não sejam a execução de um único script passem `0` como *updatepath*, e atualizem `sys.path` por conta própria, se desejarem. Veja **CVE 2008-5983** [https://www.cve.org/CVERecord?id=CVE-2008-5983].

Em versões anteriores à 3.1.3, você pode obter o mesmo efeito removendo manualmente o primeiro elemento de `sys.path` após ter chamado `PySys_SetArgv()`, por exemplo, usando:

```
PyRun_SimpleString("import sys; sys.path.pop(0)\n");
```

Adicionado na versão 3.1.3.

Descontinuado desde a versão 3.11, removido na versão 3.15.

void PySys_SetArgv(int argc, wchar_t **argv)

Parte da ABI Estável.

Esta API é mantida para retrocompatibilidade: em vez disso, devem ser usadas as configurações `PyConfig.argv` e `PyConfig.parse_argv`, consulte a Configuração de inicialização do Python.

Esta função funciona como `PySys_SetArgvEx()` com *updatepath* definido como `1` a menos que o interpretador **python** tenha sido iniciado com a `-I`.

Use `Py_DecodeLocale()` para decodificar uma string de bytes e obter uma string wchar_t*.

Veja também os membros `PyConfig.orig_argv` e `PyConfig.argv` da Configuração de inicialização do Python.

Alterado na versão 3.4: O valor *updatepath* depende de `-I`.

Descontinuado desde a versão 3.11, removido na versão 3.15.

void Py_SetPythonHome(const wchar_t *home)

Parte da ABI Estável.
Esta API é mantida para retrocompatibilidade: em vez disso, deve-se usar a configuração `PyConfig.home`, consulte a Configuração de inicialização do Python.

Define o diretório pessoal (“home”) padrão, ou seja, o local das bibliotecas padrão do Python. Consulte `PYTHONHOME` para obter o significado da string do argumento.

O argumento deve apontar para uma string terminada em zero em um armazenamento estático, cujo conteúdo não será alterado durante a execução do programa. Nenhum código no interpretador Python alterará o conteúdo desse armazenamento.

Use `Py_DecodeLocale()` para decodificar uma string de bytes e obter uma string wchar_t*.

Descontinuado desde a versão 3.11, removido na versão 3.15.

Thread states and the global interpreter lock

Unless on a free-threaded build of CPython, the Python interpreter is generally not thread-safe. In order to support multi-threaded Python programs, there's a global lock, called the global interpreter lock or GIL, that must be held by a thread before accessing Python objects. Without the lock, even the simplest operations could cause problems in a multi-threaded program: for example, when two threads simultaneously increment the reference count of the same object, the reference count could end up being incremented only once instead of twice.

As such, only a thread that holds the GIL may operate on Python objects or invoke Python's C API.

In order to emulate concurrency, the interpreter regularly tries to switch threads between bytecode instructions (see `sys.setswitchinterval()`). This is why locks are also necessary for thread-safety in pure-Python code.

Additionally, the global interpreter lock is released around blocking I/O operations, such as reading or writing to a file. From the C API, this is done by detaching the thread state.

The Python interpreter keeps some thread-local information inside a data structure called `PyThreadState`, known as a thread state. Each thread has a thread-local pointer to a `PyThreadState`; a thread state referenced by this pointer is considered to be attached.

A thread can only have one attached thread state at a time. An attached thread state is typically analogous with holding the GIL, except on free-threaded builds. On builds with the GIL enabled, attaching a thread state will block until the GIL can be acquired. However, even on builds with the GIL disabled, it is still required to have an attached thread state, as the interpreter needs to keep track of which threads may access Python objects.

Nota

Even on the free-threaded build, attaching a thread state may block, as the GIL can be re-enabled or threads might be temporarily suspended (such as during a garbage collection).

Generally, there will always be an attached thread state when using Python's C API, including during embedding and when implementing methods, so it's uncommon to need to set up a thread state on your own. Only in some specific cases, such as in a `Py_BEGIN_ALLOW_THREADS` block or in a fresh thread, will the thread not have an attached thread state. If uncertain, check if `PyThreadState_GetUnchecked()` returns `NULL`.

If it turns out that you do need to create a thread state, it is recommended to use `PyThreadState_Ensure()` or `PyThreadState_EnsureFromView()`, which will manage the thread state for you.

Desvinculação do estado de thread do código de extensão

A maior parte do código de extensão que manipula o estado de thread tem a seguinte estrutura simples:

```
Salve o estado de thread em uma variável local.
... Faça alguma operação de E/S com bloqueio...
Restaure o estado de thread da variável local.
```

Isso é tão comum que existe um par de macros para simplificá-lo:

```
Py_BEGIN_ALLOW_THREADS
... Faça alguma operação de E/S com bloqueio...
Py_END_ALLOW_THREADS
```

A macro `Py_BEGIN_ALLOW_THREADS` abre um novo bloco e declara uma variável local oculta; a macro `Py_END_ALLOW_THREADS` fecha o bloco.

O bloco acima se expande para o seguinte código:

```
PyThreadState *_save;

_save = PyEval_SaveThread();
... Faça alguma operação de E/S com bloqueio...
PyEval_RestoreThread(_save);
```

Veja como essas funções atuam:

The attached thread state implies that the GIL is held for the interpreter. To detach it, `PyEval_SaveThread()` is called and the result is stored in a local variable.

By detaching the thread state, the GIL is released, which allows other threads to attach to the interpreter and execute while the current thread performs blocking I/O. When the I/O operation is complete, the old thread state is reattached by calling `PyEval_RestoreThread()`, which will wait until the GIL can be acquired.

Nota

Performing blocking I/O is the most common use case for detaching the thread state, but it is also useful to call it over long-running native code that doesn't need access to Python objects or Python's C API. For example, the standard `zlib` and `hashlib` modules detach the thread state when compressing or hashing data.

On a free-threaded build, the GIL is usually out of the question, but **detaching the thread state is still required**, because the interpreter periodically needs to block all threads to get a consistent view of Python objects without the risk of race conditions. For example, CPython currently suspends all threads for a short period of time while running the garbage collector.

Aviso

Detaching the thread state can lead to unexpected behavior during interpreter finalization. See Cautions regarding interpreter finalization for more details.

APIs

The following macros are normally used without a trailing semicolon; look for example usage in the Python source distribution.

Nota

These macros are still necessary on the free-threaded build to prevent deadlocks.

Py_BEGIN_ALLOW_THREADS

Parte da ABI Estável.

Esta macro se expande para `{ PyThreadState *_save; _save = PyEval_SaveThread();`. Observe que ele contém uma chave de abertura; ele deve ser combinado com a seguinte macro **`Py_END_ALLOW_THREADS`**. Veja acima para uma discussão mais aprofundada desta macro.

Py_END_ALLOW_THREADS

Parte da ABI Estável.

Esta macro se expande para `PyEval_RestoreThread(_save); }`. Observe que ele contém uma chave de fechamento; ele deve ser combinado com uma macro **`Py_BEGIN_ALLOW_THREADS`** anterior. Veja acima para uma discussão mais aprofundada desta macro.

Py_BLOCK_THREADS

Parte da ABI Estável.

Esta macro se expande para `PyEval_RestoreThread(_save);`: é equivalente a **`Py_END_ALLOW_THREADS`** sem a chave de fechamento.

Py_UNBLOCK_THREADS

Parte da ABI Estável.
Esta macro se expande para `_save = PyEval_SaveThread();`: é equivalente a **`Py_BEGIN_ALLOW_THREADS`** sem a chave de abertura e declaração de variável.

Using the C API from foreign threads

When threads are created using the dedicated Python APIs (such as the **`threading`** module), a thread state is automatically associated with them, However, when a thread is created from native code (for example, by a third-party library with its own thread management), it doesn't hold an attached thread state.

If you need to call Python code from these threads (often this will be part of a callback API provided by the aforementioned third-party library), you must first register these threads with the interpreter by creating a new thread state and attaching it.

The easiest way to do this is through **`PyThreadState_Ensure()`** or **`PyThreadState_EnsureFromView()`**.

Nota

These functions require an argument pointing to the desired interpreter; such a pointer can be acquired via a call to **`PyInterpreterGuard_FromCurrent()`** (for `PyThreadState_Ensure`) or **`PyInterpreterView_FromCurrent()`** (for `PyThreadState_EnsureFromView`) from the function that creates the thread. If no pointer is available (such as when the given native thread library doesn't provide a data argument), **`PyInterpreterView_FromMain()`** can be used to get a view for the main interpreter, but note that this will make the code incompatible with subinterpreters.

Por exemplo:

```
// The return value of PyInterpreterGuard_FromCurrent() from the
// function that created this thread.
PyInterpreterGuard *guard = thread_data->guard;

// Create a new thread state for the interpreter.
PyThreadStateToken *token = PyThreadState_Ensure(guard);
if (token == NULL) {
   PyInterpreterGuard_Close(guard);
   return;
}

// We have a valid thread state -- perform Python actions here.
result = CallSomeFunction();
// Evaluate result or handle exceptions.

// Release the thread state. No calls to the C API are allowed b
// point.
PyThreadState_Release(token);
PyInterpreterGuard_Close(guard);
```

Keep in mind that calling `PyThreadState_Ensure` might not always create a new thread state, and calling `PyThreadState_Release` might not always detach it. These functions may reuse an existing attached thread state, or may re-attach a thread state that was previously attached for the current thread.

Ver também

PEP 788 [https://peps.python.org/pep-0788/]

Reusing a thread state across repeated calls

Creating and destroying a **`PyThreadState`** is not free, and is more expensive on a free-threaded build. A foreign thread that calls into the interpreter many times – for example, a worker thread in a native thread pool – should avoid creating a fresh thread state on every entry and destroying it on every exit. Instead, set up one thread state when the thread starts (or lazily on its

first call into Python), attach and detach it around each call, and tear it down once when the thread exits.

Manage the thread state explicitly with **PyThreadState_New()**, attaching and detaching it with **PyEval_RestoreThread()** and **PyEval_SaveThread()**. This happens in three distinct phases, at different points in the thread's life.

When the thread starts, create one thread state for it. `interp` is the target interpreter, captured by the code that created this thread while it held an attached thread state (for example via **PyInterpreterState_Get()**):

```
PyThreadState *tstate = PyThreadState_New(interp);
```

Then, on each call into Python – which may happen many times over the thread's life – attach the thread state, make the Python C API calls that require it, and detach again so the thread does not hold the GIL while off doing non-Python work:

```
PyEval_RestoreThread(tstate);
result = CallSomeFunction();  /* your Python C API calls go here
PyEval_SaveThread();
```

When the thread is finished calling into Python, destroy the thread state once:

```
PyEval_RestoreThread(tstate);
PyThreadState_Clear(tstate);
PyThreadState_DeleteCurrent();
```

The general-purpose entry points for calling in from a foreign thread – **PyThreadState_Ensure()** and the older **PyGILState_Ensure()** – do *not* guarantee a persistent thread state: their thread-state lifetime is deliberately implementation-defined, so a matched acquire/release pair may create and destroy a thread state each time. Use **PyThreadState_New()**, as shown here, whenever you specifically want to reuse one thread state across calls.

The code that created the foreign thread must arrange for the shutdown sequence to run before the thread exits, and before **Py_FinalizeEx()** is

called. If interpreter finalization begins first, the shutdown `PyEval_RestoreThread()` call will hang the thread rather than return (see Cautions regarding interpreter finalization). If the thread exits without running the shutdown sequence, the thread state is leaked for the remainder of the process.

Attaching/detaching thread states

PyThreadStateToken *PyThreadState_Ensure(PyInterpreterGuard *guard)

Parte da ABI Estável desde a versão 3.15.

Ensure that the thread has an attached thread state for the interpreter protected by *guard*, and thus can safely invoke that interpreter.

It is OK to call this function if the thread already has an attached thread state, as long as there is a subsequent call to `PyThreadState_Release()` that matches this one (meaning that "nested" calls to this function are permitted).

The function's effect (if any) will be reversed by the matching call to `PyThreadState_Release()`.

On error, this function returns `NULL` *without* an exception set. Do not call **`PyThreadState_Release()`** in this case.

On success, this function returns a pointer value that must be passed to the matching call to **`PyThreadState_Release()`**.

The conditions in which this function creates a new thread state are considered unstable and implementation-dependent. If you need to control the exact lifetime of a thread state, consider using `PyThreadState_New()`. However, do not avoid this function solely on the basis that the lifetime of the thread state may be inconsistent across versions; changes to this function will be done with caution and in a backwards-compatible manner. In particular, the saving of thread-local variables and similar state will be retained across Python versions.

Detalhes da implementação do CPython: The exact behavior of whether this function creates a new thread state is described below, but be aware that this may change in the future.

First, this function checks if an attached thread state is present. If there is, this function then checks if the interpreter of that thread state matches the interpreter guarded by *guard*. If that is the case, this function simply marks the thread state as being used by a `PyThreadState_Ensure` call and returns.

If there is no attached thread state, then this function checks if any thread state has been used by the current OS thread. (This is returned by **`PyGILState_GetThisThreadState()`**.) If there was, then this function checks if that thread state's interpreter matches *guard*. If it does, it is re-attached and marked as used.

Otherwise, if both of the above cases fail, a new thread state is created for *guard*. It is then attached and marked as owned by `PyThreadState_Ensure`.

Adicionado na versão 3.15.

PyThreadStateToken *PyThreadState_EnsureFromView(PyInterpreterView *view)

Parte da ABI Estável desde a versão 3.15.
Get an attached thread state for the interpreter referenced by *view*.

The behavior and return value are the same as for **`PyThreadState_Ensure()`**; additionally, if the function succeeds, the interpreter referenced by *view* will be implicitly guarded. The guard will be released upon the corresponding **`PyThreadState_Release()`** call.

Adicionado na versão 3.15.

void PyThreadState_Release(PyThreadStateToken *token)

Parte da ABI Estável desde a versão 3.15.

Undo a `PyThreadState_Ensure()` or `PyThreadState_EnsureFromView()` call.

This must be called exactly once for each successful *Ensure* call, with *token* set to that call's return value.

The state that was attached before the corresponding *Ensure* call (if any) will be attached when `PyThreadState_Release()` returns.

The exact behavior of whether this function deletes a thread state is considered unstable and implementation-dependent.

Detalhes da implementação do CPython: Currently, this function will decrement an internal counter on the attached thread state. If this counter ever reaches below zero, this function emits a fatal error (via `Py_FatalError()`).

If the attached thread state is owned by `PyThreadState_Ensure`, then the attached thread state will be deallocated and deleted upon the internal counter reaching zero. Otherwise, nothing happens when the counter reaches zero.

Adicionado na versão 3.15.

type PyThreadStateToken

Parte da ABI Estável (como uma estrutura opaca) desde a versão 3.15. An opaque token retrieved from a `PyThreadState_Ensure()` call and passed to a corresponding `PyThreadState_Release()` call.

GIL-state APIs

The following APIs are generally not compatible with subinterpreters and will hang the process during interpreter finalization (see Cautions regarding interpreter finalization). As such, these APIs were soft deprecated in Python 3.15 in favor of the new APIs.

type PyGILState_STATE

Parte da ABI Estável.
The type of the value returned by **PyGILState_Ensure()** and passed to **PyGILState_Release()**.

enumerator PyGILState_LOCKED
The GIL was already held when **PyGILState_Ensure()** was called.

enumerator PyGILState_UNLOCKED
The GIL was not held when **PyGILState_Ensure()** was called.

PyGILState_STATE PyGILState_Ensure()
Parte da ABI Estável.
Ensure that the current thread is ready to call the Python C API regardless of the current state of Python, or of the attached thread state. This may be called as many times as desired by a thread as long as each call is matched with a call to **PyGILState_Release()**. In general, other thread-related APIs may be used between **PyGILState_Ensure()** and **PyGILState_Release()** calls as long as the thread state is restored to its previous state before the Release(). For example, normal usage of the **Py_BEGIN_ALLOW_THREADS** and **Py_END_ALLOW_THREADS** macros is acceptable.

The return value is an opaque "handle" to the attached thread state when **PyGILState_Ensure()** was called, and must be passed to **PyGILState_Release()** to ensure Python is left in the same state. Even though recursive calls are allowed, these handles *cannot* be shared - each unique call to **PyGILState_Ensure()** must save the handle for its call to **PyGILState_Release()**.

When the function returns, there will be an attached thread state and the thread will be able to call arbitrary Python code.

This function has no way to return an error. As such, errors are either fatal (that is, they send `SIGABRT` and crash the process; see **Py_FatalError()**), or the thread will be permanently blocked (such as during interpreter finalization).

Aviso

Calling this function when the interpreter is finalizing will infinitely hang the thread, which may cause deadlocks. Cautions regarding interpreter finalization for more details.

In addition, this function generally does not work with subinterpreters when used from foreign threads, because this function has no way of knowing which interpreter created the thread (and as such, will implicitly pick the main interpreter).

Alterado na versão 3.14: Hangs the current thread, rather than terminating it, if called while the interpreter is finalizing.

Suavemente descontinuado desde a versão 3.15: Use `PyThreadState_Ensure()` or `PyThreadState_EnsureFromView()` instead.

void PyGILState_Release(PyGILState_STATE)

Parte da ABI Estável.

Release any resources previously acquired. After this call, Python's state will be the same as it was prior to the corresponding `PyGILState_Ensure()` call (but generally this state will be unknown to the caller, hence the use of the GIL-state API).

Every call to `PyGILState_Ensure()` must be matched by a call to `PyGILState_Release()` on the same thread.

Suavemente descontinuado desde a versão 3.15: Use `PyThreadState_Release()` instead.

PyThreadState *PyGILState_GetThisThreadState()

Parte da ABI Estável.

Get the thread state that was most recently attached for this thread. (If the most recent thread state has been deleted, this returns `NULL`.)

If the caller has an attached thread state, it is returned.

In other terms, this function returns the thread state that will be used by **PyGILState_Ensure()**. If this returns `NULL`, then `PyGILState_Ensure` will create a new thread state.

This function cannot fail.

Suavemente descontinuado desde a versão 3.15: Use **PyThreadState_Get()** or **PyThreadState_GetUnchecked()** instead.

int PyGILState_Check()

Return `1` if the current thread has an attached thread state that matches the thread state returned by **PyGILState_GetThisThreadState()**. If the caller has no attached thread state or it otherwise doesn't match, then this returns `0`.

If the current Python process has ever created a subinterpreter, this function will *always* return `1`.

This is mainly a helper/diagnostic function.

Adicionado na versão 3.4.

Suavemente descontinuado desde a versão 3.15: Use `PyThreadState_GetUnchecked() != NULL` instead.

Cuidados com o uso de fork()

Outro aspecto importante a observar sobre threads é o seu comportamento diante da chamada de **fork()** da linguagem C. Na maioria dos sistemas com **fork()**, após um processo ser criado ("fork"), apenas a thread que emitiu o fork continuará existindo. Isso tem um impacto concreto tanto na forma como as travas devem ser gerenciados quanto em todo o estado armazenado no ambiente de execução do CPython.

The fact that only the "current" thread remains means any locks held by other threads will never be released. Python solves this for **os.fork()** by acquiring the locks it uses internally before the fork, and releasing them

afterwards. In addition, it resets any Lock objects in the child. When extending or embedding Python, there is no way to inform Python of additional (non-Python) locks that need to be acquired before or reset after a fork. OS facilities such as **`pthread_atfork()`** would need to be used to accomplish the same thing. Additionally, when extending or embedding Python, calling **`fork()`** directly rather than through **`os.fork()`** (and returning to or calling into Python) may result in a deadlock by one of Python's internal locks being held by a thread that is defunct after the fork. **`PyOS_AfterFork_Child()`** tries to reset the necessary locks, but is not always able to.

The fact that all other threads go away also means that CPython's runtime state there must be cleaned up properly, which **`os.fork()`** does. This means finalizing all other **`PyThreadState`** objects belonging to the current interpreter and all other **`PyInterpreterState`** objects. Due to this and the special nature of the "main" interpreter, **`fork()`** should only be called in that interpreter's "main" thread, where the CPython global runtime was originally initialized. The only exception is if **`exec()`** will be called immediately after.

APIs de alto nível

These are the most commonly used types and functions when writing multi-threaded C extensions.

type PyThreadState

Parte da ABI Estável (como uma estrutura opaca).
This data structure represents the state of a single thread. The only public data member is:

PyInterpreterState *interp

This thread's interpreter state.

void PyEval_InitThreads()

Parte da ABI Estável.
Função descontinuada que não faz nada.

In Python 3.6 and older, this function created the GIL if it didn't exist.

Alterado na versão 3.9: The function now does nothing.

Alterado na versão 3.7: Esta função agora é chamada por `Py_Initialize()`, então não há mais necessidade de você chamá-la.

Alterado na versão 3.2: Esta função não pode mais ser chamada antes de `Py_Initialize()`.

Descontinuado desde a versão 3.9.

PyThreadState *PyEval_SaveThread()

Parte da ABI Estável.
Detach the attached thread state and return it. The thread will have no thread state upon returning.

void PyEval_RestoreThread(PyThreadState *tstate)

Parte da ABI Estável.
Set the attached thread state to *tstate*. The passed thread state **should not** be attached, otherwise deadlock ensues. *tstate* will be attached upon returning.

Nota

Calling this function from a thread when the runtime is finalizing will hang the thread until the program exits, even if the thread was not created by Python. Refer to Cautions regarding interpreter finalization for more details.

Alterado na versão 3.14: Hangs the current thread, rather than terminating it, if called while the interpreter is finalizing.

PyThreadState *PyThreadState_Get()

Parte da ABI Estável.
Return the attached thread state. If the thread has no attached thread state, (such as when inside of `Py_BEGIN_ALLOW_THREADS` block), then

this issues a fatal error (so that the caller needn't check for `NULL`).

See also **`PyThreadState_GetUnchecked()`**.

PyThreadState *PyThreadState_GetUnchecked()

Similar to **`PyThreadState_Get()`**, but don't kill the process with a fatal error if it is NULL. The caller is responsible to check if the result is NULL.

Adicionado na versão 3.13: In Python 3.5 to 3.12, the function was private and known as `_PyThreadState_UncheckedGet()`.

PyThreadState *PyThreadState_Swap(PyThreadState *tstate)

Parte da ABI Estável.

Set the attached thread state to *tstate*, and return the thread state that was attached prior to calling.

This function is safe to call without an attached thread state; it will simply return `NULL` indicating that there was no prior thread state.

Ver também

`PyEval_ReleaseThread()`

Nota

Similar to **`PyGILState_Ensure()`**, this function will hang the thread if the runtime is finalizing.

APIs de baixo nível

PyThreadState *PyThreadState_New(PyInterpreterState *interp)

Parte da ABI Estável.

Create a new thread state object belonging to the given interpreter object. An attached thread state is not needed.

void PyThreadState_Clear(PyThreadState *tstate)

Parte da ABI Estável.
Reset all information in a thread state object. *tstate* must be attached

Alterado na versão 3.9: This function now calls the **`PyThreadState.on_delete`** callback. Previously, that happened in **`PyThreadState_Delete()`**.

Alterado na versão 3.13: The **`PyThreadState.on_delete`** callback was removed.

void PyThreadState_Delete(PyThreadState *tstate)

Parte da ABI Estável.
Destroy a thread state object. *tstate* should not be attached to any thread. *tstate* must have been reset with a previous call to **`PyThreadState_Clear()`**.

void PyThreadState_DeleteCurrent(void)

Detach the attached thread state (which must have been reset with a previous call to **`PyThreadState_Clear()`**) and then destroy it.

No thread state will be attached upon returning.

PyFrameObject *PyThreadState_GetFrame(PyThreadState *tstate)

Parte da ABI Estável desde a versão 3.10.
Get the current frame of the Python thread state *tstate*.

Return a strong reference. Return `NULL` if no frame is currently executing.

See also **`PyEval_GetFrame()`**.

tstate must not be `NULL`, and must be attached.

Adicionado na versão 3.9.

uint64_t PyThreadState_GetID(PyThreadState *tstate)

Parte da ABI Estável desde a versão 3.10.
Get the unique thread state identifier of the Python thread state *tstate*.

tstate must not be `NULL`, and must be attached.

Adicionado na versão 3.9.

PyInterpreterState *PyThreadState_GetInterpreter(PyThreadState *tstate)
Parte da ABI Estável desde a versão 3.10.
Get the interpreter of the Python thread state *tstate*.

tstate must not be `NULL`, and must be attached.

Adicionado na versão 3.9.

void PyThreadState_EnterTracing(PyThreadState *tstate)
Suspend tracing and profiling in the Python thread state *tstate*.

Resume them using the **`PyThreadState_LeaveTracing()`** function.

Adicionado na versão 3.11.

void PyThreadState_LeaveTracing(PyThreadState *tstate)
Resume tracing and profiling in the Python thread state *tstate* suspended by the **`PyThreadState_EnterTracing()`** function.

See also **`PyEval_SetTrace()`** and **`PyEval_SetProfile()`** functions.

Adicionado na versão 3.11.

int PyUnstable_ThreadState_SetStackProtection(PyThreadState *tstate, void *stack_start_addr, size_t stack_size)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Set the stack protection start address and stack protection size of a Python thread state.

On success, return `0`. On failure, set an exception and return `-1`.

CPython implements recursion control for C code by raising **`RecursionError`** when it notices that the machine execution stack is close to overflow. See for example the **`Py_EnterRecursiveCall()`** function. For this, it needs to know the location of the current thread's stack, which it normally gets from the operating system. When the stack is changed, for example using context switching techniques like the Boost library's `boost::context`, you must call **`PyUnstable_ThreadState_SetStackProtection()`** to inform CPython of the change.

Call **`PyUnstable_ThreadState_SetStackProtection()`** either before or after changing the stack. Do not call any other Python C API between the call and the stack change.

See **`PyUnstable_ThreadState_ResetStackProtection()`** for undoing this operation.

Adicionado na versão 3.15.

void PyUnstable_ThreadState_ResetStackProtection(PyThreadState *tstate)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Reset the stack protection start address and stack protection size of a Python thread state to the operating system defaults.

See **`PyUnstable_ThreadState_SetStackProtection()`** for an explanation.

Adicionado na versão 3.15.

PyObject *PyThreadState_GetDict()

Retorna valor: Referência emprestada. Parte da ABI Estável.

Return a dictionary in which extensions can store thread-specific state information. Each extension should use a unique key to use to store

state in the dictionary. It is okay to call this function when no thread state is attached. If this function returns `NULL`, no exception has been raised and the caller should assume no thread state is attached.

void PyEval_AcquireThread(PyThreadState *tstate)

Parte da ABI Estável.

Attach *tstate* to the current thread, which must not be `NULL` or already attached.

The calling thread must not already have an attached thread state.

Nota

Calling this function from a thread when the runtime is finalizing will hang the thread until the program exits, even if the thread was not created by Python. Refer to Cautions regarding interpreter finalization for more details.

Alterado na versão 3.8: Updated to be consistent with `PyEval_RestoreThread()`, `Py_END_ALLOW_THREADS()`, and `PyGILState_Ensure()`, and terminate the current thread if called while the interpreter is finalizing.

Alterado na versão 3.14: Hangs the current thread, rather than terminating it, if called while the interpreter is finalizing.

`PyEval_RestoreThread()` is a higher-level function which is always available (even when threads have not been initialized).

void PyEval_ReleaseThread(PyThreadState *tstate)

Parte da ABI Estável.

Detach the attached thread state. The *tstate* argument, which must not be `NULL`, is only used to check that it represents the attached thread state — if it isn't, a fatal error is reported.

`PyEval_SaveThread()` is a higher-level function which is always available (even when threads have not been initialized).

Asynchronous notifications

A mechanism is provided to make asynchronous notifications to the main interpreter thread. These notifications take the form of a function pointer and a void pointer argument.

int Py_AddPendingCall(int (*func)(void*), void *arg)

Parte da ABI Estável.

Schedule a function to be called from the main interpreter thread. On success, `0` is returned and *func* is queued for being called in the main thread. On failure, `-1` is returned without setting any exception.

When successfully queued, *func* will be *eventually* called from the main interpreter thread with the argument *arg*. It will be called asynchronously with respect to normally running Python code, but with both these conditions met:

- on a bytecode boundary;
- with the main thread holding an attached thread state (*func* can therefore use the full C API).

func must return `0` on success, or `-1` on failure with an exception set. *func* won't be interrupted to perform another asynchronous notification recursively, but it can still be interrupted to switch threads if the thread state is detached.

This function doesn't need an attached thread state. However, to call this function in a subinterpreter, the caller must have an attached thread state. Otherwise, the function *func* can be scheduled to be called from the wrong interpreter.

Aviso

This is a low-level function, only useful for very special cases. There is no guarantee that *func* will be called as quick as possible. If the main thread is busy executing a system call, *func* won't be called before the system call returns. This function is generally **not** suitable

for calling Python code from arbitrary C threads. Instead, use `PyThreadState_EnsureFromView()`.

Adicionado na versão 3.1.

Alterado na versão 3.9: If this function is called in a subinterpreter, the function *func* is now scheduled to be called from the subinterpreter, rather than being called from the main interpreter. Each subinterpreter now has its own list of scheduled calls.

Alterado na versão 3.12: This function now always schedules *func* to be run in the main interpreter.

int Py_MakePendingCalls(void)

Parte da ABI Estável.

Execute all pending calls. This is usually executed automatically by the interpreter.

Esta função retorna `0` em caso de sucesso e retorna `-1` com uma exceção definida em caso de falha.

If this is not called in the main thread of the main interpreter, this function does nothing and returns `0`. The caller must hold an attached thread state.

Adicionado na versão 3.1.

Alterado na versão 3.12: This function only runs pending calls in the main interpreter.

int PyThreadState_SetAsyncExc(unsigned long id, PyObject *exc)

Parte da ABI Estável.

Schedule an exception to be raised asynchronously in a thread. If the thread has a previously scheduled exception, it is overwritten.

The *id* argument is the thread id of the target thread, as returned by `PyThread_get_thread_ident()`. *exc* is the class of the exception to be

raised, or `NULL` to clear the pending exception (if any).

Return the number of affected thread states. This is normally `1` if *id* is found, even when no change was made (the given *exc* was already pending, or *exc* is `NULL` but no exception is pending). If the thread id isn't found, return `0`. This raises no exceptions.

To prevent naive misuse, you must write your own C extension to call this. This function must be called with an attached thread state. This function does not steal any references to *exc*. This function does not necessarily interrupt system calls such as **`sleep()`**.

Alterado na versão 3.7: The type of the *id* parameter changed from long to unsigned long.

Operating system thread APIs

PYTHREAD_INVALID_THREAD_ID

Sentinel value for an invalid thread ID.

This is currently equivalent to `(unsigned long)-1`.

unsigned long PyThread_start_new_thread(void (*func)(void*), void *arg)

Parte da ABI Estável.

Start function *func* in a new thread with argument *arg*. The resulting thread is not intended to be joined.

func must not be `NULL`, but *arg* may be `NULL`.

On success, this function returns the identifier of the new thread; on failure, this returns **`PYTHREAD_INVALID_THREAD_ID`**.

The caller does not need to hold an attached thread state.

unsigned long PyThread_get_thread_ident(void)

Parte da ABI Estável.

Return the identifier of the current thread, which will never be zero.

This function cannot fail, and the caller does not need to hold an attached thread state.

Ver também

`threading.get_ident()` and `threading.Thread.ident` expose this identifier to Python.

PyObject *PyThread_GetInfo(void)

Parte da ABI Estável desde a versão 3.3.

Get general information about the current thread in the form of a struct sequence object. This information is accessible as `sys.thread_info` in Python.

On success, this returns a new strong reference to the thread information; on failure, this returns `NULL` with an exception set.

The caller must hold an attached thread state.

PY_HAVE_THREAD_NATIVE_ID

This macro is defined when the system supports native thread IDs.

unsigned long PyThread_get_thread_native_id(void)

Parte da ABI Estável on platforms with native thread IDs.

Get the native identifier of the current thread as it was assigned by the operating system's kernel, which will never be less than zero.

This function is only available when `PY_HAVE_THREAD_NATIVE_ID` is defined.

This function cannot fail, and the caller does not need to hold an attached thread state.

Ver também

`threading.get_native_id()`

void PyThread_exit_thread(void)

Parte da ABI Estável.

Terminate the current thread. This function is generally considered unsafe and should be avoided. It is kept solely for backwards compatibility.

This function is only safe to call if all functions in the full call stack are written to safely allow it.

Aviso

If the current system uses POSIX threads (also known as "pthreads"), this calls *pthread_exit(3)* *[https://manpages.debian.org/pthread_exit(3)]*, which attempts to unwind the stack and call C++ destructors on some libc implementations. However, if a `noexcept` function is reached, it may terminate the process. Other systems, such as macOS, do unwinding.

On Windows, this function calls `_endthreadex()`, which kills the thread without calling C++ destructors.

In any case, there is a risk of corruption on the thread's stack.

Descontinuado desde a versão 3.14.

void PyThread_init_thread(void)

Parte da ABI Estável.

Initialize `PyThread*` APIs. Python executes this function automatically, so there's little need to call it from an extension module.

int PyThread_set_stacksize(size_t size)

Parte da ABI Estável.

Set the stack size of the current thread to *size* bytes.

This function returns `0` on success, `-1` if *size* is invalid, or `-2` if the system does not support changing the stack size. This function does not set exceptions.

The caller does not need to hold an attached thread state.

size_t PyThread_get_stacksize(void)

Parte da ABI Estável.

Return the stack size of the current thread in bytes, or `0` if the system's default stack size is in use.

The caller does not need to hold an attached thread state.

Primitivas de sincronização

The C-API provides a basic mutual exclusion lock.

type PyMutex

A mutual exclusion lock. The **PyMutex** should be initialized to zero to represent the unlocked state. For example:

```
PyMutex mutex = {0};
```

Instances of **PyMutex** should not be copied or moved. Both the contents and address of a **PyMutex** are meaningful, and it must remain at a fixed, writable location in memory.

> **Nota**
>
> A **PyMutex** currently occupies one byte, but the size should be considered unstable. The size may change in future Python releases without a deprecation period.

Adicionado na versão 3.13.

void PyMutex_Lock(PyMutex *m)

Thread safety: Atomic.

Lock mutex *m*. If another thread has already locked it, the calling thread will block until the mutex is unlocked. While blocked, the thread will temporarily detach the thread state if one exists.

Adicionado na versão 3.13.

void PyMutex_Unlock(PyMutex *m)

Thread safety: Atomic.

Unlock mutex *m*. The mutex must be locked — otherwise, the function will issue a fatal error.

Adicionado na versão 3.13.

int PyMutex_IsLocked(PyMutex *m)

Thread safety: Atomic.

Returns non-zero if the mutex *m* is currently locked, zero otherwise.

Nota

This function is intended for use in assertions and debugging only and should not be used to make concurrency control decisions, as the lock state may change immediately after the check.

Adicionado na versão 3.14.

Python critical section API

The critical section API provides a deadlock avoidance layer on top of per-object locks for free-threaded CPython. They are intended to replace reliance on the global interpreter lock, and are no-ops in versions of Python with the global interpreter lock.

Critical sections are intended to be used for custom types implemented in C-API extensions. They should generally not be used with built-in types like `list` and `dict` because their public C-APIs already use critical sections internally, with the notable exception of `PyDict_Next()`, which requires critical section to be acquired externally.

Critical sections avoid deadlocks by implicitly suspending active critical sections, hence, they do not provide exclusive access such as provided by traditional locks like `PyMutex`. When a critical section is started, the per-object lock for the object is acquired. If the code executed inside the critical section calls C-API functions then it can suspend the critical section thereby releasing the per-object lock, so other threads can acquire the per-object lock for the same object.

Variants that accept `PyMutex` pointers rather than Python objects are also available. Use these variants to start a critical section in a situation where there is no `PyObject` – for example, when working with a C type that does not extend or wrap `PyObject` but still needs to call into the C API in a manner that might lead to deadlocks.

Nota

Operations that need to lock two objects at once must use `Py_BEGIN_CRITICAL_SECTION2`. You *cannot* use nested critical sections to lock more than one object at once, because the inner critical section may suspend the outer critical sections. This API does not provide a way to lock more than two objects at once.

Exemplo de uso:

```
static PyObject *
set_field(MyObject *self, PyObject *value)
{
   Py_BEGIN_CRITICAL_SECTION(self);
   Py_SETREF(self->field, Py_XNewRef(value));
   Py_END_CRITICAL_SECTION();
   Py_RETURN_NONE;
}
```

In the above example, `Py_SETREF` calls `Py_DECREF`, which can call arbitrary code through an object's deallocation function. The critical section API avoids potential deadlocks due to reentrancy and lock ordering by allowing the runtime to temporarily suspend the critical section if the code triggered by the finalizer blocks and calls `PyEval_SaveThread()`.

Py_BEGIN_CRITICAL_SECTION(op)

Parte da ABI Estável desde a versão 3.15.

Acquires the per-object lock for the object *op* and begins a critical section.

In the free-threaded build, and when building for the Stable ABI, this macro expands to:

```
{
    PyCriticalSection _py_cs;
    PyCriticalSection_Begin(&_py_cs, (PyObject*)(op))
```

In the default build, this macro expands to `{`.

Adicionado na versão 3.13.

Py_BEGIN_CRITICAL_SECTION_MUTEX(m)

Locks the mutex *m* and begins a critical section.

In the free-threaded build, this macro expands to:

```
{
     PyCriticalSection _py_cs;
     PyCriticalSection_BeginMutex(&_py_cs, m)
```

Note that unlike `Py_BEGIN_CRITICAL_SECTION`, there is no cast for the argument of the macro - it must be a `PyMutex` pointer.

On the default build, this macro expands to `{`.

Adicionado na versão 3.14.

Py_END_CRITICAL_SECTION()

Parte da ABI Estável desde a versão 3.15.
Ends the critical section and releases the per-object lock.

In the free-threaded build, and when building for the Stable ABI, this macro expands to:

```
    PyCriticalSection_End(&_py_cs);
}
```

In the default build, this macro expands to `}`.

Adicionado na versão 3.13.

Py_BEGIN_CRITICAL_SECTION2(a, b)

Parte da ABI Estável desde a versão 3.15.
Acquires the per-object locks for the objects *a* and *b* and begins a critical section. The locks are acquired in a consistent order (lowest address first) to avoid lock ordering deadlocks.

In the free-threaded build, this macro expands to:

```
{
    PyCriticalSection2 _py_cs2;
    PyCriticalSection2_Begin(&_py_cs2, (PyObject*)(a), (PyOb
```

In the default build, this macro expands to `{`.

Adicionado na versão 3.13.

Py_BEGIN_CRITICAL_SECTION2_MUTEX(m1, m2)

Locks the mutexes *m1* and *m2* and begins a critical section.

In the free-threaded build, and when building for the Stable ABI, this macro expands to:

```
{
     PyCriticalSection2 _py_cs2;
     PyCriticalSection2_BeginMutex(&_py_cs2, m1, m2)
```

Note that unlike `Py_BEGIN_CRITICAL_SECTION2`, there is no cast for the arguments of the macro - they must be `PyMutex` pointers.

On the default build, this macro expands to `{`.

Adicionado na versão 3.14.

Py_END_CRITICAL_SECTION2()

Parte da ABI Estável desde a versão 3.15.
Ends the critical section and releases the per-object locks.

In the free-threaded build, and when building for the Stable ABI, this macro expands to:

```
    PyCriticalSection2_End(&_py_cs2);
}
```

In the default build, this macro expands to `}`.

Adicionado na versão 3.13.

Low-level critical section API

The following functions and structs are exposed for cases where C macros are not available.

void PyCriticalSection_Begin(PyCriticalSection *c, PyObject *op)
void PyCriticalSection_End(PyCriticalSection *c)

void PyCriticalSection2_Begin(PyCriticalSection2 *c, PyObject *a, PyObject *b)
void PyCriticalSection2_End(PyCriticalSection2 *c);

Parte da ABI Estável desde a versão 3.15.
To be used only as in the macro expansions listed earlier in this section.

In non-free-threaded builds of CPython, these functions do nothing.

Adicionado na versão 3.13.

type PyCriticalSection
type PyCriticalSection2

Parte da ABI Estável (incluindo todos os membros) desde a versão 3.15.
To be used only as in the macro expansions listed earlier in this section. Note that the contents of the structures are private and their meaning may change in future Python versions.

Adicionado na versão 3.13.

void PyCriticalSection_BeginMutex(PyCriticalSection *c, PyMutex *m);
void PyCriticalSection2_BeginMutex(PyCriticalSection2 *c, PyMutex *m1, PyMutex *m2);

To be used only as in the macro expansions listed earlier in this section.

In non-free-threaded builds of CPython, these functions do nothing.

Adicionado na versão 3.14.

Legacy locking APIs

These APIs are obsolete since Python 3.13 with the introduction of `PyMutex`.

Alterado na versão 3.15: These APIs are now a simple wrapper around `PyMutex`.

type PyThread_type_lock

A pointer to a mutual exclusion lock.

type PyLockStatus

The result of acquiring a lock with a timeout.

enumerator PY_LOCK_FAILURE

Failed to acquire the lock.

enumerator PY_LOCK_ACQUIRED

The lock was successfully acquired.

enumerator PY_LOCK_INTR

The lock was interrupted by a signal.

PyThread_type_lock PyThread_allocate_lock(void)

Parte da ABI Estável.

Allocate a new lock.

On success, this function returns a lock; on failure, this function returns `0` without an exception set.

The caller does not need to hold an attached thread state.

Alterado na versão 3.15: This function now always uses **`PyMutex`**. In prior versions, this would use a lock provided by the operating system.

void PyThread_free_lock(PyThread_type_lock lock)

Parte da ABI Estável.

Destroy *lock*. The lock should not be held by any thread when calling this.

The caller does not need to hold an attached thread state.

PyLockStatus PyThread_acquire_lock_timed(PyThread_type_lock lock, long long microseconds, int intr_flag)

Parte da ABI Estável.

Acquire *lock* with a timeout.

This will wait for *microseconds* microseconds to acquire the lock. If the timeout expires, this function returns **`PY_LOCK_FAILURE`**. If *microseconds* is `-1`, this will wait indefinitely until the lock has been released.

If *intr_flag* is `1`, acquiring the lock may be interrupted by a signal, in which case this function returns **`PY_LOCK_INTR`**. Upon interruption, it's generally expected that the caller makes a call to **`Py_MakePendingCalls()`** to propagate an exception to Python code.

If the lock is successfully acquired, this function returns **`PY_LOCK_ACQUIRED`**.

The caller does not need to hold an attached thread state.

int PyThread_acquire_lock(PyThread_type_lock lock, int waitflag)

Parte da ABI Estável.
Acquire *lock*.

If *waitflag* is `1` and another thread currently holds the lock, this function will wait until the lock can be acquired and will always return `1`.

If *waitflag* is `0` and another thread holds the lock, this function will not wait and instead return `0`. If the lock is not held by any other thread, then this function will acquire it and return `1`.

Unlike **`PyThread_acquire_lock_timed()`**, acquiring the lock cannot be interrupted by a signal.

The caller does not need to hold an attached thread state.

int PyThread_release_lock(PyThread_type_lock lock)

Parte da ABI Estável.
Release *lock*. If *lock* is not held, then this function issues a fatal error.

The caller does not need to hold an attached thread state.

Thread-local storage support

The Python interpreter provides low-level support for thread-local storage (TLS) which wraps the underlying native TLS implementation to support the Python-level thread-local storage API (`threading.local`). The CPython C level APIs are similar to those offered by pthreads and Windows: use a thread key and functions to associate a void* value per thread.

A thread state does *not* need to be attached when calling these functions; they supply their own locking.

Note that `Python.h` does not include the declaration of the TLS APIs, you need to include `pythread.h` to use thread-local storage.

Nota

None of these API functions handle memory management on behalf of the void* values. You need to allocate and deallocate them yourself. If the void* values happen to be PyObject*, these functions don't do refcount operations on them either.

Thread-specific storage API

The thread-specific storage (TSS) API was introduced to supersede the use of the existing TLS API within the CPython interpreter. This API uses a new type `Py_tss_t` instead of int to represent thread keys.

Adicionado na versão 3.7.

Ver também

"A New C-API for Thread-Local Storage in CPython" (**PEP 539** [https://peps.python.org/pep-0539/])

type Py_tss_t

This data structure represents the state of a thread key, the definition of which may depend on the underlying TLS implementation, and it has an internal field representing the key's initialization state. There are no public members in this structure.

Quando Py_LIMITED_API não é definido, a alocação estática deste tipo por `Py_tss_NEEDS_INIT` é permitida.

Py_tss_NEEDS_INIT

This macro expands to the initializer for `Py_tss_t` variables. Note that this macro won't be defined with Py_LIMITED_API.

Dynamic allocation

Dynamic allocation of the `Py_tss_t`, required in extension modules built with Py_LIMITED_API, where static allocation of this type is not possible due to its implementation being opaque at build time.

Py_tss_t *PyThread_tss_alloc()

Parte da ABI Estável desde a versão 3.7.

Retorna um valor que é o mesmo estado de um valor inicializado com `Py_tss_NEEDS_INIT`, ou `NULL` no caso de falha de alocação dinâmica.

void PyThread_tss_free(Py_tss_t *key)

Parte da ABI Estável desde a versão 3.7.

Free the given *key* allocated by `PyThread_tss_alloc()`, after first calling `PyThread_tss_delete()` to ensure any associated thread locals have been unassigned. This is a no-op if the *key* argument is `NULL`.

Nota

A freed key becomes a dangling pointer. You should reset the key to `NULL`.

Métodos

The parameter *key* of these functions must not be `NULL`. Moreover, the behaviors of `PyThread_tss_set()` and `PyThread_tss_get()` are undefined if the given `Py_tss_t` has not been initialized by `PyThread_tss_create()`.

int PyThread_tss_is_created(Py_tss_t *key)

Parte da ABI Estável desde a versão 3.7.
Return a non-zero value if the given `Py_tss_t` has been initialized by `PyThread_tss_create()`.

int PyThread_tss_create(Py_tss_t *key)

Parte da ABI Estável desde a versão 3.7.
Retorna um valor zero na inicialização bem-sucedida de uma chave TSS. O comportamento é indefinido se o valor apontado pelo argumento *key* não for inicializado por `Py_tss_NEEDS_INIT`. Essa função pode ser chamada repetidamente na mesma tecla – chamá-la em uma tecla já inicializada não funciona e retorna imediatamente com sucesso.

void PyThread_tss_delete(Py_tss_t *key)

Parte da ABI Estável desde a versão 3.7.
Destroy a TSS key to forget the values associated with the key across all threads, and change the key's initialization state to uninitialized. A destroyed key is able to be initialized again by `PyThread_tss_create()`. This function can be called repeatedly on the same key – calling it on an already destroyed key is a no-op.

int PyThread_tss_set(Py_tss_t *key, void *value)

Parte da ABI Estável desde a versão 3.7.
Return a zero value to indicate successfully associating a void* value with a TSS key in the current thread. Each thread has a distinct mapping of the key to a void* value.

void *PyThread_tss_get(Py_tss_t *key)

Parte da ABI Estável desde a versão 3.7.

Return the void* value associated with a TSS key in the current thread. This returns `NULL` if no value is associated with the key in the current thread.

Legacy APIs

Descontinuado desde a versão 3.7: This API is superseded by the thread-specific storage (TSS) API.

Nota

This version of the API does not support platforms where the native TLS key is defined in a way that cannot be safely cast to `int`. On such platforms, **`PyThread_create_key()`** will return immediately with a failure status, and the other TLS functions will all be no-ops on such platforms.

Due to the compatibility problem noted above, this version of the API should not be used in new code.

int PyThread_create_key()

Parte da ABI Estável.

void PyThread_delete_key(int key)

Parte da ABI Estável.

int PyThread_set_key_value(int key, void *value)

Parte da ABI Estável.

void *PyThread_get_key_value(int key)

Parte da ABI Estável.

void PyThread_delete_key_value(int key)

Parte da ABI Estável.

void PyThread_ReInitTLS()

Parte da ABI Estável.

Multiple interpreters in a Python process

While in most uses, you will only embed a single Python interpreter, there are cases where you need to create several independent interpreters in the same process and perhaps even in the same thread. Sub-interpreters allow you to do that.

The “main” interpreter is the first one created when the runtime initializes. It is usually the only Python interpreter in a process. Unlike sub-interpreters, the main interpreter has unique process-global responsibilities like signal handling. It is also responsible for execution during runtime initialization and is usually the active interpreter during runtime finalization. The `PyInterpreterState_Main()` function returns a pointer to its state.

You can switch between sub-interpreters using the `PyThreadState_Swap()` function. You can create and destroy them using the following functions:

type PyInterpreterConfig

Structure containing most parameters to configure a sub-interpreter. Its values are used only in `Py_NewInterpreterFromConfig()` and never modified by the runtime.

Adicionado na versão 3.12.

Campos de estrutura:

int use_main_obmalloc

If this is `0` then the sub-interpreter will use its own “object” allocator state. Otherwise it will use (share) the main interpreter’s.

If this is `0` then `check_multi_interp_extensions` must be `1` (non-zero). If this is `1` then `gil` must not be

`PyInterpreterConfig_OWN_GIL`.

int allow_fork

If this is `0` then the runtime will not support forking the process in any thread where the sub-interpreter is currently active. Otherwise fork is unrestricted.

Note that the `subprocess` module still works when fork is disallowed.

int allow_exec

If this is `0` then the runtime will not support replacing the current process via exec (e.g. `os.execv()`) in any thread where the sub-interpreter is currently active. Otherwise exec is unrestricted.

Note that the `subprocess` module still works when exec is disallowed.

int allow_threads

If this is `0` then the sub-interpreter's `threading` module won't create threads. Otherwise threads are allowed.

int allow_daemon_threads

If this is `0` then the sub-interpreter's `threading` module won't create daemon threads. Otherwise daemon threads are allowed (as long as `allow_threads` is non-zero).

int check_multi_interp_extensions

If this is `0` then all extension modules may be imported, including legacy (single-phase init) modules, in any thread where the sub-interpreter is currently active. Otherwise only multi-phase init extension modules (see **PEP 489** [https://peps.python.org/pep-0489/]) may be imported. (Also see `Py_mod_multiple_interpreters`.)

This must be `1` (non-zero) if `use_main_obmalloc` is `0`.

int gil

This determines the operation of the GIL for the sub-interpreter. It may be one of the following:

PyInterpreterConfig_DEFAULT_GIL
Use the default selection (`PyInterpreterConfig_SHARED_GIL`).

PyInterpreterConfig_SHARED_GIL
Use (share) the main interpreter's GIL.

PyInterpreterConfig_OWN_GIL
Use the sub-interpreter's own GIL.

If this is `PyInterpreterConfig_OWN_GIL` then `PyInterpreterConfig.use_main_obmalloc` must be `0`.

PyStatus Py_NewInterpreterFromConfig(PyThreadState **tstate_p, const PyInterpreterConfig *config)

Create a new sub-interpreter. This is an (almost) totally separate environment for the execution of Python code. In particular, the new interpreter has separate, independent versions of all imported modules, including the fundamental modules `builtins`, `__main__` and `sys`. The table of loaded modules (`sys.modules`) and the module search path (`sys.path`) are also separate. The new environment has no `sys.argv` variable. It has new standard I/O stream file objects `sys.stdin`, `sys.stdout` and `sys.stderr` (however these refer to the same underlying file descriptors).

The given *config* controls the options with which the interpreter is initialized.

Upon success, *tstate_p* will be set to the first thread state created in the new sub-interpreter. This thread state is attached. Note that no actual thread is created; see the discussion of thread states below. If creation of the new interpreter is unsuccessful, *tstate_p* is set to `NULL`; no exception is set since the exception state is stored in the attached thread state, which might not exist.

Like all other Python/C API functions, an attached thread state must be present before calling this function, but it might be detached upon returning. On success, the returned thread state will be attached. If the sub-interpreter is created with its own GIL then the attached thread state of the calling interpreter will be detached. When the function returns, the new interpreter's thread state will be attached to the current thread and the previous interpreter's attached thread state will remain detached.

Adicionado na versão 3.12.

Sub-interpreters are most effective when isolated from each other, with certain functionality restricted:

```
PyInterpreterConfig config = {
    .use_main_obmalloc = 0,
    .allow_fork = 0,
    .allow_exec = 0,
    .allow_threads = 1,
    .allow_daemon_threads = 0,
    .check_multi_interp_extensions = 1,
    .gil = PyInterpreterConfig_OWN_GIL,
};
PyThreadState *tstate = NULL;
PyStatus status = Py_NewInterpreterFromConfig(&tstate, &confi
if (PyStatus_Exception(status)) {
    Py_ExitStatusException(status);
}
```

Note that the config is used only briefly and does not get modified. During initialization the config's values are converted into various **`PyInterpreterState`** values. A read-only copy of the config may be stored internally on the **`PyInterpreterState`**.

Extension modules are shared between (sub-)interpreters as follows:

- For modules using multi-phase initialization, e.g. **`PyModule_FromDefAndSpec()`**, a separate module object is created and initialized for each interpreter. Only C-level static and global variables are shared between these module objects.

- For modules using legacy single-phase initialization, e.g. `PyModule_Create()`, the first time a particular extension is imported, it is initialized normally, and a (shallow) copy of its module's dictionary is squirreled away. When the same extension is imported by another (sub-)interpreter, a new module is initialized and filled with the contents of this copy; the extension's `init` function is not called. Objects in the module's dictionary thus end up shared across (sub-)interpreters, which might cause unwanted behavior (see Bugs and caveats below).

  Note that this is different from what happens when an extension is imported after the interpreter has been completely re-initialized by calling `Py_FinalizeEx()` and `Py_Initialize()`; in that case, the extension's `initmodule` function *is* called again. As with multi-phase initialization, this means that only C-level static and global variables are shared between these modules.

PyThreadState *Py_NewInterpreter(void)

Parte da ABI Estável.

Create a new sub-interpreter. This is essentially just a wrapper around `Py_NewInterpreterFromConfig()` with a config that preserves the existing behavior. The result is an unisolated sub-interpreter that shares the main interpreter's GIL, allows fork/exec, allows daemon threads, and allows single-phase init modules.

void Py_EndInterpreter(PyThreadState *tstate)

Parte da ABI Estável.

Destroy the (sub-)interpreter represented by the given thread state. The given thread state must be attached. When the call returns, there will be no attached thread state. All thread states associated with this interpreter are destroyed.

`Py_FinalizeEx()` will destroy all sub-interpreters that haven't been explicitly destroyed at that point.

A per-interpreter GIL

Adicionado na versão 3.12.

Using `Py_NewInterpreterFromConfig()` you can create a sub-interpreter that is completely isolated from other interpreters, including having its own GIL. The most important benefit of this isolation is that such an interpreter can execute Python code without being blocked by other interpreters or blocking any others. Thus a single Python process can truly take advantage of multiple CPU cores when running Python code. The isolation also encourages a different approach to concurrency than that of just using threads. (See **PEP 554** [https://peps.python.org/pep-0554/] and **PEP 684** [https://peps.python.org/pep-0684/].)

Using an isolated interpreter requires vigilance in preserving that isolation. That especially means not sharing any objects or mutable state without guarantees about thread-safety. Even objects that are otherwise immutable (e.g. `None`, `(1, 5)`) can't normally be shared because of the refcount. One simple but less-efficient approach around this is to use a global lock around all use of some state (or object). Alternately, effectively immutable objects (like integers or strings) can be made safe in spite of their refcounts by making them immortal. In fact, this has been done for the builtin singletons, small integers, and a number of other builtin objects.

Se você preservar o isolamento, terá acesso à computação multi-core adequada, sem as complicações que acompanham o uso de threads livres. A falha em preservar o isolamento traz a exposição a todas as consequências de threads livres, incluindo corridas e travamentos difíceis de depurar.

Aside from that, one of the main challenges of using multiple isolated interpreters is how to communicate between them safely (not break isolation) and efficiently. The runtime and stdlib do not provide any standard approach to this yet. A future stdlib module would help mitigate the effort of preserving isolation and expose effective tools for communicating (and sharing) data between interpreters.

Bugs and caveats

Because sub-interpreters (and the main interpreter) are part of the same process, the insulation between them isn't perfect — for example, using low-level file operations like `os.close()` they can (accidentally or maliciously) affect each other's open files. Because of the way extensions are shared between (sub-)interpreters, some extensions may not work properly; this is especially likely when using single-phase initialization or (static) global variables. It is possible to insert objects created in one sub-interpreter into a namespace of another (sub-)interpreter; this should be avoided if possible.

Special care should be taken to avoid sharing user-defined functions, methods, instances or classes between sub-interpreters, since import operations executed by such objects may affect the wrong (sub-)interpreter's dictionary of loaded modules. It is equally important to avoid sharing objects from which the above are reachable.

Also note that combining this functionality with `PyGILState_*` APIs is delicate, because these APIs assume a bijection between Python thread states and OS-level threads, an assumption broken by the presence of sub-interpreters. It is highly recommended that you don't switch sub-interpreters between a pair of matching `PyGILState_Ensure()` and `PyGILState_Release()` calls. Furthermore, extensions (such as `ctypes`) using these APIs to allow calling of Python code from non-Python created threads will probably be broken when using sub-interpreters.

APIs de alto nível

type PyInterpreterState

Parte da ABI Estável (como uma estrutura opaca).
This data structure represents the state shared by a number of cooperating threads. Threads belonging to the same interpreter share their module administration and a few other internal items. There are no public members in this structure.

Threads belonging to different interpreters initially share nothing, except process state like available memory, open file descriptors and

such. The global interpreter lock is also shared by all threads, regardless of to which interpreter they belong.

Alterado na versão 3.12: **PEP 684** [https://peps.python.org/pep-0684/] introduced the possibility of a per-interpreter GIL. See `Py_NewInterpreterFromConfig()`.

PyInterpreterState *PyInterpreterState_Get(void)

Parte da ABI Estável desde a versão 3.9.
Get the current interpreter.

Issue a fatal error if there is no attached thread state. It cannot return NULL.

Adicionado na versão 3.9.

int64_t PyInterpreterState_GetID(PyInterpreterState *interp)

Parte da ABI Estável desde a versão 3.7.
Return the interpreter's unique ID. If there was any error in doing so then `-1` is returned and an error is set.

O chamador deve ter um estado de thread anexado.

Adicionado na versão 3.7.

PyObject *PyInterpreterState_GetDict(PyInterpreterState *interp)

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.8.
Return a dictionary in which interpreter-specific data may be stored. If this function returns `NULL` then no exception has been raised and the caller should assume no interpreter-specific dict is available.

This is not a replacement for `PyModule_GetState()`, which extensions should use to store interpreter-specific state information.

The returned dictionary is borrowed from the interpreter and is valid until interpreter shutdown.

Adicionado na versão 3.8.

typedef PyObject *(*_PyFrameEvalFunction)(PyThreadState *tstate, _PyInterpreterFrame *frame, int throwflag)

Type of a frame evaluation function.

The *throwflag* parameter is used by the `throw()` method of generators: if non-zero, handle the current exception.

Alterado na versão 3.9: The function now takes a *tstate* parameter.

Alterado na versão 3.11: The *frame* parameter changed from `PyFrameObject*` to `_PyInterpreterFrame*`.

_PyFrameEvalFunction _PyInterpreterState_GetEvalFrameFunc(PyInterpreterState *interp)

Get the frame evaluation function.

See the **PEP 523** [https://peps.python.org/pep-0523/] "Adding a frame evaluation API to CPython".

Adicionado na versão 3.9.

void _PyInterpreterState_SetEvalFrameFunc(PyInterpreterState *interp, _PyFrameEvalFunction eval_frame)

Set the frame evaluation function.

See the **PEP 523** [https://peps.python.org/pep-0523/] "Adding a frame evaluation API to CPython".

Adicionado na versão 3.9.

void _PyInterpreterState_SetEvalFrameAllowSpecialization(PyInterpreterState *interp, int allow_specialization)

Enables or disables specialization why a custom frame evaluator is in place.

If *allow_specialization* is non-zero, the adaptive specializer will continue to specialize bytecodes even though a custom eval frame function is set. When *allow_specialization* is zero, setting a custom eval frame disables specialization. The standard interpreter loop will continue to deopt while a frame evaluation API is in place - the frame evaluation function needs to handle the specialized opcodes to take advantage of this.

Adicionado na versão 3.15.

int _PyInterpreterState_IsSpecializationEnabled(PyInterpreterState *interp)

Return non-zero if adaptive specialization is enabled for the interpreter. Specialization is enabled when no custom eval frame function is set, or when one is set with *allow_specialization* enabled.

Adicionado na versão 3.15.

APIs de baixo nível

All of the following functions must be called after `Py_Initialize()`.

Alterado na versão 3.7: `Py_Initialize()` now initializes the GIL and sets an attached thread state.

PyInterpreterState *PyInterpreterState_New()

Parte da ABI Estável.

Create a new interpreter state object. An attached thread state is not needed, but may optionally exist if it is necessary to serialize calls to this function.

Levanta um evento de auditoria `cpython.PyInterpreterState_New` sem argumentos.

void PyInterpreterState_Clear(PyInterpreterState *interp)

Parte da ABI Estável.

Reset all information in an interpreter state object. There must be an attached thread state for the interpreter.

Levanta um evento de auditoria `cpython.PyInterpreterState_Clear` sem argumentos.

void PyInterpreterState_Delete(PyInterpreterState *interp)

Parte da ABI Estável.

Destroy an interpreter state object. There **should not** be an attached thread state for the target interpreter. The interpreter state must have been reset with a previous call to **`PyInterpreterState_Clear()`**.

Advanced debugger support

These functions are only intended to be used by advanced debugging tools.

PyInterpreterState *PyInterpreterState_Head()

Return the interpreter state object at the head of the list of all such objects.

PyInterpreterState *PyInterpreterState_Main()

Return the main interpreter state object.

PyInterpreterState *PyInterpreterState_Next(PyInterpreterState *interp)

Return the next interpreter state object after *interp* from the list of all such objects.

PyThreadState *PyInterpreterState_ThreadHead(PyInterpreterState *interp)

Return the pointer to the first **`PyThreadState`** object in the list of threads associated with the interpreter *interp*.

PyThreadState *PyThreadState_Next(PyThreadState *tstate)

Return the next thread state object after *tstate* from the list of all such objects belonging to the same **`PyInterpreterState`** object.

Profiling and tracing

The Python interpreter provides some low-level support for attaching profiling and execution tracing facilities. These are used for profiling, debugging, and coverage analysis tools.

This C interface allows the profiling or tracing code to avoid the overhead of calling through Python-level callable objects, making a direct C function call instead. The essential attributes of the facility have not changed; the interface allows trace functions to be installed per-thread, and the basic events reported to the trace function are the same as had been reported to the Python-level trace functions in previous versions.

typedef int (*Py_tracefunc)(PyObject *obj, PyFrameObject *frame, int what, PyObject *arg)

The type of the trace function registered using `PyEval_SetProfile()` and `PyEval_SetTrace()`. The first parameter is the object passed to the registration function as *obj*, *frame* is the frame object to which the event pertains, *what* is one of the constants `PyTrace_CALL`, `PyTrace_EXCEPTION`, `PyTrace_LINE`, `PyTrace_RETURN`, `PyTrace_C_CALL`, `PyTrace_C_EXCEPTION`, `PyTrace_C_RETURN`, or `PyTrace_OPCODE`, and *arg* depends on the value of *what*:

Value of *what*	Meaning of *arg*
`PyTrace_CALL`	Always `Py_None`.
`PyTrace_EXCEPTION`	Exception information as returned by `sys.exc_info()`.
`PyTrace_LINE`	Always `Py_None`.
`PyTrace_RETURN`	Value being returned to the caller, or `NULL` if caused by an exception.

Value of *what*	Meaning of *arg*
`PyTrace_C_CALL`	Function object being called.
`PyTrace_C_EXCEPTION`	Function object being called.
`PyTrace_C_RETURN`	Function object being called.
`PyTrace_OPCODE`	Always `Py_None`.

int PyTrace_CALL

The value of the *what* parameter to a `Py_tracefunc` function when a new call to a function or method is being reported, or a new entry into a generator. Note that the creation of the iterator for a generator function is not reported as there is no control transfer to the Python bytecode in the corresponding frame.

int PyTrace_EXCEPTION

The value of the *what* parameter to a `Py_tracefunc` function when an exception has been raised. The callback function is called with this value for *what* when after any bytecode is processed after which the exception becomes set within the frame being executed. The effect of this is that as exception propagation causes the Python stack to unwind, the callback is called upon return to each frame as the exception propagates. Only trace functions receive these events; they are not needed by the profiler.

int PyTrace_LINE

The value passed as the *what* parameter to a `Py_tracefunc` function (but not a profiling function) when a line-number event is being reported. It may be disabled for a frame by setting `f_trace_lines` to *0* on that frame.

int PyTrace_RETURN

The value for the *what* parameter to `Py_tracefunc` functions when a call is about to return.

int PyTrace_C_CALL

The value for the *what* parameter to `Py_tracefunc` functions when a C function is about to be called.

int PyTrace_C_EXCEPTION

The value for the *what* parameter to `Py_tracefunc` functions when a C function has raised an exception.

int PyTrace_C_RETURN

The value for the *what* parameter to `Py_tracefunc` functions when a C function has returned.

int PyTrace_OPCODE

The value for the *what* parameter to `Py_tracefunc` functions (but not profiling functions) when a new opcode is about to be executed. This event is not emitted by default: it must be explicitly requested by setting `f_trace_opcodes` to *1* on the frame.

void PyEval_SetProfile(Py_tracefunc func, PyObject *obj)

Set the profiler function to *func*. The *obj* parameter is passed to the function as its first parameter, and may be any Python object, or `NULL`. If the profile function needs to maintain state, using a different value for *obj* for each thread provides a convenient and thread-safe place to store it. The profile function is called for all monitored events except `PyTrace_LINE` `PyTrace_OPCODE` and `PyTrace_EXCEPTION`.

See also the `sys.setprofile()` function.

O chamador deve ter um estado de thread anexado.

void PyEval_SetProfileAllThreads(Py_tracefunc func, PyObject *obj)

Like `PyEval_SetProfile()` but sets the profile function in all running threads belonging to the current interpreter instead of the setting it only on the current thread.

O chamador deve ter um estado de thread anexado.

As `PyEval_SetProfile()`, this function ignores any exceptions raised while setting the profile functions in all threads.

Adicionado na versão 3.12.

void PyEval_SetTrace(Py_tracefunc func, PyObject *obj)

Set the tracing function to *func*. This is similar to `PyEval_SetProfile()`, except the tracing function does receive line-number events and per-opcode events, but does not receive any event related to C function objects being called. Any trace function registered using `PyEval_SetTrace()` will not receive `PyTrace_C_CALL`, `PyTrace_C_EXCEPTION` or `PyTrace_C_RETURN` as a value for the *what* parameter.

See also the `sys.settrace()` function.

O chamador deve ter um estado de thread anexado.

void PyEval_SetTraceAllThreads(Py_tracefunc func, PyObject *obj)

Like `PyEval_SetTrace()` but sets the tracing function in all running threads belonging to the current interpreter instead of the setting it only on the current thread.

O chamador deve ter um estado de thread anexado.

As `PyEval_SetTrace()`, this function ignores any exceptions raised while setting the trace functions in all threads.

Adicionado na versão 3.12.

Reference tracing

Adicionado na versão 3.13.

typedef int (*PyRefTracer)(PyObject*, int event, void *data)

The type of the trace function registered using `PyRefTracer_SetTracer()`. The first parameter is a Python object that has been just created (when **event** is set to `PyRefTracer_CREATE`) or about to be destroyed (when **event** is set to `PyRefTracer_DESTROY`). The **data** argument is the opaque pointer that was provided when `PyRefTracer_SetTracer()` was called.

If a new tracing function is registered replacing the current one, a call to the trace function will be made with the object set to **NULL** and **event** set to `PyRefTracer_TRACKER_REMOVED`. This will happen just before the new function is registered.

Adicionado na versão 3.13.

int PyRefTracer_CREATE

The value for the *event* parameter to `PyRefTracer` functions when a Python object has been created.

int PyRefTracer_DESTROY

The value for the *event* parameter to `PyRefTracer` functions when a Python object has been destroyed.

int PyRefTracer_TRACKER_REMOVED

The value for the *event* parameter to `PyRefTracer` functions when the current tracer is about to be replaced by a new one.

Adicionado na versão 3.14.

int PyRefTracer_SetTracer(PyRefTracer tracer, void *data)

Register a reference tracer function. The function will be called when a new Python object has been created or when an object is going to be destroyed. If **data** is provided it must be an opaque pointer that will be provided when the tracer function is called. Return `0` on success. Set an exception and return `-1` on error.

Note that tracer functions **must not** create Python objects inside or otherwise the call will be re-entrant. The tracer also **must not** clear any

existing exception or set an exception. A thread state will be active every time the tracer function is called.

There must be an attached thread state when calling this function.

If another tracer function was already registered, the old function will be called with **event** set to `PyRefTracer_TRACKER_REMOVED` just before the new function is registered.

Adicionado na versão 3.13.

PyRefTracer PyRefTracer_GetTracer(void **data)

Get the registered reference tracer function and the value of the opaque data pointer that was registered when `PyRefTracer_SetTracer()` was called. If no tracer was registered this function will return NULL and will set the **data** pointer to NULL.

There must be an attached thread state when calling this function.

Adicionado na versão 3.13.

Configuração de inicialização do Python

API C PyInitConfig

Adicionado na versão 3.14.

O Python pode ser inicializado com `Py_InitializeFromInitConfig()`.

A função `Py_RunMain()` pode ser usada para escrever um programa Python personalizado.

Veja também Inicialização, Finalização e Threads.

Ver também

PEP 741 [https://peps.python.org/pep-0741/] "API C de configuração do Python".

Exemplo

Exemplo de Python personalizado sempre em execução com o Modo de Desenvolvimento Python ativado; retorna `-1` em caso de erro:

```
int init_python(void)
{
    PyInitConfig *config = PyInitConfig_Create();
    if (config == NULL) {
        printf("PYTHON INIT ERROR: memory allocation failed\n");
        return -1;
    }

    // Habilita o Modo de Desenvolvimento do Python
    if (PyInitConfig_SetInt(config, "dev_mode", 1) < 0) {
        goto error;
    }

    // Inicializa o Python com a configuração
    if (Py_InitializeFromInitConfig(config) < 0) {
        goto error;
    }
    PyInitConfig_Free(config);
    return 0;

error:
    {
        // Exibe a mensagem de erro.
        //
```

```
        // Esse estilo incomum de chaves é usado porque você não
        // fazer alvos goto apontarem para declarações de variáveis.
        const char *err_msg;
        (void)PyInitConfig_GetError(config, &err_msg);
        printf("PYTHON INIT ERROR: %s\n", err_msg);
        PyInitConfig_Free(config);
        return -1;
    }
}
```

Criar configuração

struct PyInitConfig

Estrutura opaca para configurar a inicialização do Python.

PyInitConfig *PyInitConfig_Create(void)

Cria uma nova configuração de inicialização usando os valores padrão de configuração isolada.

Deve ser liberada por **`PyInitConfig_Free()`**.

Retorna `NULL` em caso de falha na alocação de memória.

void PyInitConfig_Free(PyInitConfig *config)

Libera a memória da configuração de inicialização *config*.

Se *config* for `NULL`, nenhuma operação será realizada.

Tratamento de erros

int PyInitConfig_GetError(PyInitConfig *config, const char **err_msg)

Obtém a mensagem de erro *config*.

- Define **err_msg* e retorna `1` se um erro for definido.
- Define **err_msg* como `NULL` e retorna `0` caso contrário.

Uma mensagem de erro é uma string codificada em UTF-8.

Se *config* tiver um código de saída, formata o código de saída como uma mensagem de erro.

A mensagem de erro permanece válida até que outra função `PyInitConfig` seja chamada com *config*. O chamador não precisa liberar a mensagem de erro.

int PyInitConfig_GetExitCode(PyInitConfig *config, int *exitcode)

Obtém o código de saída *config*.

- Define **exitcode* e retorna `1` se *config* tiver um código de saída definido.
- Retorna `0` se *config* não tiver nenhum código de saída definido.

Somente a função `Py_InitializeFromInitConfig()` pode definir um código de saída se a opção `parse_argv` for diferente de zero.

Um código de saída pode ser definido quando a análise da linha de comando falhar (código de saída `2`) ou quando uma opção da linha de comando solicitar a exibição da ajuda da linha de comando (código de saída `0`).

Opções de get

O parâmetro *name* da opção de configuração deve ser uma string codificada em UTF-8 terminada em nulo e não nula. Consulte as opções de configuração.

int PyInitConfig_HasOption(PyInitConfig *config, const char *name)

Verifica se a configuração possui uma opção chamada *name*.

Retorna `1` se a opção existir, ou retorna `0` caso contrário.

int PyInitConfig_GetInt(PyInitConfig *config, const char *name, int64_t *value)

Obtém uma opção de configuração de número inteiro.

- Define **value* e retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

int PyInitConfig_GetStr(PyInitConfig *config, const char *name, char **value)

Obtém uma opção de configuração de string como uma string codificada em UTF-8 terminada em nulo.

- Define **value* e retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

**value* pode ser definido como `NULL` se a opção for uma string opcional e não estiver definida.

Em caso de sucesso, a string deve ser liberada com `free(value)` se não for `NULL`.

int PyInitConfig_GetStrList(PyInitConfig *config, const char *name, size_t *length, char ***items)

Obtém uma opção de configuração de lista de strgins como um vetor de strings codificadas em UTF-8 terminadas em null.

- Define **length* e **value* retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

Em caso de sucesso, a lista de strings deve ser liberada com `PyInitConfig_FreeStrList(length, items)`.

void PyInitConfig_FreeStrList(size_t length, char **items)

Libera memória de uma lista de strings criada por `PyInitConfig_GetStrList()`.

Opções de set

O parâmetro *name* da opção de configuração deve ser uma string codificada em UTF-8 terminada em nulo e não nula. Consulte as opções de configuração.

Algumas opções de configuração têm efeitos colaterais em outras opções. Essa lógica só é implementada quando `Py_InitializeFromInitConfig()` é chamada, e não pelas funções "Set" abaixo. Por exemplo, definir `dev_mode` como `1` não define `faulthandler` como `1`.

int PyInitConfig_SetInt(PyInitConfig *config, const char *name, int64_t value)

Define uma opção de configuração de inteiro.

- Retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

int PyInitConfig_SetStr(PyInitConfig *config, const char *name, const char *value)

Define uma opção de configuração de string a partir de uma string codificada em UTF-8 terminada em nulo. A string será copiada.

- Retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

int PyInitConfig_SetStrList(PyInitConfig *config, const char *name, size_t length, char *const *items)

Define uma opção de configuração de lista de strings a partir de um vetor de strings codificadas em UTF-8 terminadas em nulo. A lista de strings será copiada.

- Retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

Módulo

int PyInitConfig_AddModule(PyInitConfig *config, const char *name, PyObject *(*initfunc)(void))

Adiciona um módulo de extensão embutido à tabela de módulos embutidos.

O novo módulo pode ser importado pelo nome *name* e utiliza a função *initfunc* como função de inicialização, sendo chamada na primeira tentativa de importação.

- Retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.

Se o Python for inicializado várias vezes, `PyInitConfig_AddModule()` deve ser chamada em cada inicialização do Python.

Similar à função **`PyImport_AppendInittab()`**.

Inicializar o Python

int Py_InitializeFromInitConfig(PyInitConfig *config)

Inicializa o Python por meio da configuração de inicialização.

- Retorna `0` em caso de sucesso.
- Define um erro em *config* e retorna `-1` em caso de erro.
- Define um código de saída em *config* e retorna `-1` se o Python desejar sair.

Consulte `PyInitConfig_GetExitcode()` para o caso de código de saída.

Opções de configuração

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
`"allocator"`	**`allocator`**	`int`	Somente leitura
`"argv"`	**`argv`**	`list[str]`	Pública
`"base_exec_prefix"`	**`base_exec_prefix`**	`str`	Pública
`"base_executable"`	**`base_executable`**	`str`	Pública
`"base_prefix"`	**`base_prefix`**	`str`	Pública
`"buffered_stdio"`	**`buffered_stdio`**	`bool`	Somente leitura

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
"bytes_warning"	bytes_warning	int	Pública
"check_hash_pycs_mode"	check_hash_pycs_mode	str	Somente leitura
"code_debug_ranges"	code_debug_ranges	bool	Somente leitura
"coerce_c_locale"	coerce_c_locale	bool	Somente leitura
"coerce_c_locale_warn"	coerce_c_locale_warn	bool	Somente leitura
"configure_c_stdio"	configure_c_stdio	bool	Somente leitura
"configure_locale"	configure_locale	bool	Somente leitura
"cpu_count"	cpu_count	int	Pública
"dev_mode"	dev_mode	bool	Somente leitura
"dump_refs"	dump_refs	bool	Somente leitura
"dump_refs_file"	dump_refs_file	str	Somente leitura

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
"exec_prefix"	exec_prefix	str	Pública
"executable"	executable	str	Pública
"faulthandler"	faulthandler	bool	Somente leitura
"filesystem_encoding"	filesystem_encoding	str	Somente leitura
"filesystem_errors"	filesystem_errors	str	Somente leitura
"hash_seed"	hash_seed	int	Somente leitura
"home"	home	str	Somente leitura
"import_time"	import_time	int	Somente leitura
"inspect"	inspect	bool	Pública
"install_signal_handlers"	install_signal_handlers	bool	Somente leitura
"int_max_str_digits"	int_max_str_digits	int	Pública
"interactive"	interactive	bool	Pública

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
"isolated"	isolated	bool	Somente leitura
"legacy_windows_fs_encoding"	legacy_windows_fs_encoding	bool	Somente leitura
"legacy_windows_stdio"	legacy_windows_stdio	bool	Somente leitura
"malloc_stats"	malloc_stats	bool	Somente leitura
"module_search_paths"	module_search_paths	list[str]	Pública
"optimization_level"	optimization_level	int	Pública
"orig_argv"	orig_argv	list[str]	Somente leitura
"parse_argv"	parse_argv	bool	Somente leitura
"parser_debug"	parser_debug	bool	Pública
"pathconfig_warnings"	pathconfig_warnings	bool	Somente leitura
"perf_profiling"	perf_profiling	bool	Somente leitura

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
"platlibdir"	platlibdir	str	Pública
"prefix"	prefix	str	Pública
"program_name"	program_name	str	Somente leitura
"pycache_prefix"	pycache_prefix	str	Pública
"quiet"	quiet	bool	Pública
"run_command"	run_command	str	Somente leitura
"run_filename"	run_filename	str	Somente leitura
"run_module"	run_module	str	Somente leitura
"run_presite"	run_presite	str	Somente leitura
"safe_path"	safe_path	bool	Somente leitura
"show_ref_count"	show_ref_count	bool	Somente leitura

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
"site_import"	site_import	bool	Somente leitura
"skip_source_first_line"	skip_source_first_line	bool	Somente leitura
"stdio_encoding"	stdio_encoding	str	Somente leitura
"stdio_errors"	stdio_errors	str	Somente leitura
"stdlib_dir"	stdlib_dir	str	Pública
"tracemalloc"	tracemalloc	int	Somente leitura
"use_environment"	use_environment	bool	Pública
"use_frozen_modules"	use_frozen_modules	bool	Somente leitura
"use_hash_seed"	use_hash_seed	bool	Somente leitura
"use_system_logger"	use_system_logger	bool	Somente leitura
"user_site_directory"	user_site_directory	bool	Somente leitura

Opção	Membro de PyConfig/PyPreConfig	Tipo	Visibilidade
`"utf8_mode"`	`utf8_mode`	`bool`	Somente leitura
`"verbose"`	`verbose`	`int`	Pública
`"warn_default_encoding"`	`warn_default_encoding`	`bool`	Somente leitura
`"warnoptions"`	`warnoptions`	`list[str]`	Pública
`"write_bytecode"`	`write_bytecode`	`bool`	Pública
`"xoptions"`	`xoptions`	`dict[str, str]`	Pública
`"_pystats"`	`_pystats`	`bool`	Somente leitura

Visibilidade:

- Public: Can be retrieved by `PyConfig_Get()` and set by `PyConfig_Set()`.
- Read-only: Can be retrieved by `PyConfig_Get()`, but cannot be set by `PyConfig_Set()`.

API de configuração Python em tempo de execução

Em tempo de execução, é possível obter e definir opções de configuração usando as funções `PyConfig_Get()` e `PyConfig_Set()`.

O parâmetro *name* da opção de configuração deve ser uma string codificada em UTF-8 terminada em nulo e não nula. Consulte as opções de configuração.

Algumas opções são lidas a partir dos atributos `sys`. Por exemplo, a opção `"argv"` é lida a partir de `sys.argv`.

PyObject *PyConfig_Get(const char *name)

Obtém o valor de tempo de execução atual de uma opção de configuração como um objeto Python.

- Retorna uma nova referência em caso de sucesso.
- Define uma exceção e retorna `NULL` em caso de erro.

O tipo de objeto depende da opção de configuração. Pode ser:

- `bool`
- `int`
- `str`
- `list[str]`
- `dict[str, str]`

O chamador deve ter um estado de thread anexado. A função não pode ser chamada antes da inicialização do Python nem depois da finalização do Python.

Adicionado na versão 3.14.

int PyConfig_GetInt(const char *name, int *value)

Similar a **`PyConfig_Get()`**, mas obtém o valor como um int C.

- Retorna `0` em caso de sucesso.
- Define uma exceção e retorna `-1` em caso de erro.

Adicionado na versão 3.14.

PyObject *PyConfig_Names(void)

Obtém todos os nomes das opções de configuração como um `frozenset`.

- Retorna uma nova referência em caso de sucesso.
- Define uma exceção e retorna `NULL` em caso de erro.

O chamador deve ter um estado de thread anexado. A função não pode ser chamada antes da inicialização do Python nem depois da finalização do Python.

Adicionado na versão 3.14.

int PyConfig_Set(const char *name, PyObject *value)

Define o valor de tempo de execução atual de uma opção de configuração.

- Levanta uma **`ValueError`** se não houver uma opção *name*.
- Levanta uma **`ValueError`** se *value* foi um valor inválido.
- Levanta uma **`ValueError`** se a opção for somente leitura (não puder ser definida).
- Levanta um **`TypeError`** se *value* não tiver um tipo adequado.

O chamador deve ter um estado de thread anexado. A função não pode ser chamada antes da inicialização do Python nem depois da finalização do Python.

Levanta um evento de auditoria `cpython.PyConfig_Set` com os argumentos `name`, `value`.

Adicionado na versão 3.14.

Alterado na versão 3.15: The function now replaces **`sys.flags`** (create a new object), instead of modifying **`sys.flags`** in-place.

API C PyConfig

Adicionado na versão 3.8.

Python pode ser inicializado com **`Py_InitializeFromConfig()`** e a estrutura **`PyConfig`**. Pode ser pré-inicializado com **`Py_PreInitialize()`** e a estrutura **`PyPreConfig`**.

Existem dois tipos de configuração:

- A Python Configuration pode ser usada para construir um Python personalizado que se comporta como um Python comum. Por exemplo, variáveis de ambiente e argumento de linha de comando são usados para configurar Python.
- A Configuração isolada pode ser usada para incorporar Python em uma aplicação. Isso isola Python de um sistema. Por exemplo, variáveis de ambiente são ignoradas, a variável local LC_CTYPE fica inalterada e nenhum manipulador de sinal é registrado.

A função **`Py_RunMain()`** pode ser usada para escrever um programa Python personalizado.

Veja também Inicialização, Finalização e Threads.

Ver também

PEP 587 [https://peps.python.org/pep-0587/] “Configuração da inicialização do Python”.

Exemplo

Exemplo de Python personalizado sendo executado sempre em um modo isolado:

```
int main(int argc, char **argv)
{
    PyStatus status;

    PyConfig config;
    PyConfig_InitPythonConfig(&config);
    config.isolated = 1;

    /* Decode command line arguments.
       Implicitly preinitialize Python (in isolated mode). */
    status = PyConfig_SetBytesArgv(&config, argc, argv);
```

```
    if (PyStatus_Exception(status)) {
        goto exception;
    }

    status = Py_InitializeFromConfig(&config);
    if (PyStatus_Exception(status)) {
        goto exception;
    }
    PyConfig_Clear(&config);

    return Py_RunMain();

exception:
    PyConfig_Clear(&config);
    if (PyStatus_IsExit(status)) {
        return status.exitcode;
    }
    /* Display the error message and exit the process with
       non-zero exit code */
    Py_ExitStatusException(status);
}
```

PyWideStringList

type PyWideStringList

Lista de strings `wchar_t*`.

Se *length* é diferente de zero, *items* deve ser diferente de `NULL` e todas as strings devem ser diferentes de `NULL`.

Métodos:

PyStatus PyWideStringList_Append(PyWideStringList *list, const wchar_t *item)

Anexa *item* a *list*.

Python deve ser inicializado previamente antes de chamar essa função.

PyStatus PyWideStringList_Insert(PyWideStringList *list, Py_ssize_t index, const wchar_t *item)

Insere *item* na *list* na posição *index*.

Se *index* for maior ou igual ao comprimento da *list*, anexa o *item* a *list*.

index deve ser maior que ou igual a `0`.

Python deve ser inicializado previamente antes de chamar essa função.

Campos de estrutura:

Py_ssize_t length
Comprimento da lista.

wchar_t **items
Itens da lista.

PyStatus

type PyStatus
Estrutura para armazenar o status de uma função de inicialização: sucesso, erro ou saída.

Para um erro, ela pode armazenar o nome da função C que criou o erro.

Campos de estrutura:

int exitcode
Código de saída. Argumento passado para `exit()`.

const char *err_msg
Mensagem de erro.

const char *func
Nome da função que criou um erro. Pode ser `NULL`.

Funções para criar um status:

PyStatus PyStatus_Ok(void)
Sucesso.

PyStatus PyStatus_Error(const char *err_msg)
Erro de inicialização com uma mensagem.

err_msg não deve ser `NULL`.

PyStatus PyStatus_NoMemory(void)
Falha de alocação de memória (sem memória).

PyStatus PyStatus_Exit(int exitcode)
Sai do Python com o código de saída especificado.

Funções para manipular um status:

int PyStatus_Exception(PyStatus status)
O status é um erro ou uma saída? Se verdadeiro, a exceção deve ser tratada; chamando `Py_ExitStatusException()`, por exemplo.

int PyStatus_IsError(PyStatus status)

O resultado é um erro?

int PyStatus_IsExit(PyStatus status)

O resultado é uma saída?

void Py_ExitStatusException(PyStatus status)

Chama `exit(exitcode)` se *status* for uma saída. Exibe a mensagem de erro e sai com um código de saída diferente de zero se *status* for um erro. Deve ser chamado apenas se `PyStatus_Exception(status)` for diferente de zero.

Nota

Internamente, Python usa macros que definem `PyStatus.func`, enquanto funções para criar um status definem `func` para `NULL`.

Exemplo:

```
PyStatus alloc(void **ptr, size_t size)
{
    *ptr = PyMem_RawMalloc(size);
    if (*ptr == NULL) {
        return PyStatus_NoMemory();
    }
    return PyStatus_Ok();
}

int main(int argc, char **argv)
{
    void *ptr;
    PyStatus status = alloc(&ptr, 16);
    if (PyStatus_Exception(status)) {
        Py_ExitStatusException(status);
    }
    PyMem_Free(ptr);
    return 0;
}
```

PyPreConfig

type PyPreConfig

Estrutura usada para pré-inicializar o Python.

A função para inicializar uma pré-configuração:

void PyPreConfig_InitPythonConfig(PyPreConfig *preconfig)

Inicializa a pré-configuração com Configuração do Python.

void PyPreConfig_InitIsolatedConfig(PyPreConfig *preconfig)

Inicializa a pré-configuração com Configuração isolada.

Campos de estrutura:

int allocator

Nome de alocadores de memória em Python:

- `PYMEM_ALLOCATOR_NOT_SET` (`0`): não altera os alocadores de memória (usa o padrão).
- `PYMEM_ALLOCATOR_DEFAULT` (`1`): alocadores de memória padrão.
- `PYMEM_ALLOCATOR_DEBUG` (`2`): alocadores de memória padrão com ganchos de depuração.
- `PYMEM_ALLOCATOR_MALLOC` (`3`): usa `malloc()` da biblioteca C.
- `PYMEM_ALLOCATOR_MALLOC_DEBUG` (`4`): força uso de `malloc()` com ganchos de depuração.
- `PYMEM_ALLOCATOR_PYMALLOC` (`5`): alocador de memória do Python pymalloc.
- `PYMEM_ALLOCATOR_PYMALLOC_DEBUG` (`6`): alocador de memória deo Python pymalloc com ganchos de depuração.
- `PYMEM_ALLOCATOR_MIMALLOC` (`6`): usa `mimalloc`, um substituto rápido do malloc.
- `PYMEM_ALLOCATOR_MIMALLOC_DEBUG` (`7`): use `mimalloc`, um substituto rápido do malloc com ganchos de depuração.

`PYMEM_ALLOCATOR_PYMALLOC` e `PYMEM_ALLOCATOR_PYMALLOC_DEBUG` não são suportados se o Python estiver **`configurado usando --without-pymalloc`**.

`PYMEM_ALLOCATOR_MIMALLOC` e `PYMEM_ALLOCATOR_MIMALLOC_DEBUG` não são suportados se o Python estiver **`configurado usando --without-mimalloc`** ou se suporte atômico subjacente não estiver disponível.

Veja Gerenciamento de memória.

Padrão: `PYMEM_ALLOCATOR_NOT_SET`.

int configure_locale

Define a localidade LC_CTYPE para a localidade preferida do usuário.

Se igual a `0`, define os membros **`coerce_c_locale`** e **`coerce_c_locale_warn`** para `0`.

Veja a codificação da localidade.

Padrão: `1` na configuração do Python, `0` na configuração isolada.

int coerce_c_locale

Se igual a `2`, força a localidade para C.

Se for igual a `1`, lê a localidade LC_CTYPE para decidir se deve ser forçado.

Veja a codificação da localidade.

Padrão: `-1` na configuração do Python, `0` na configuração isolada.

int coerce_c_locale_warn

Se diferente de zero, emite um aviso se a localidade C for forçada.

Padrão: `-1` na configuração do Python, `0` na configuração isolada.

int dev_mode

Modo de desenvolvimento do Python: veja **`PyConfig.dev_mode`**.

Padrão: `-1` no modo do Python, `0` no modo isolado.

int isolated

Modo isolado: veja **`PyConfig.isolated`**.

Padrão: `0` no modo do Python, `1` no modo isolado.

int legacy_windows_fs_encoding

Se não zero:

- Define **`PyPreConfig.utf8_mode`** para `0`,
- Define **`PyConfig.filesystem_encoding`** para `"mbcs"`,
- Define **`PyConfig.filesystem_errors`** para `"replace"`.

Initialized from the **`PYTHONLEGACYWINDOWSFSENCODING`** environment variable value.

Disponível apenas no Windows. A macro `#ifdef MS_WINDOWS` pode ser usada para código específico do Windows.

Padrão: `0`.

int parse_argv

Se diferente de zero, **`Py_PreInitializeFromArgs()`** e **`Py_PreInitializeFromBytesArgs()`** analisam seu argumento `argv` da mesma forma que o Python regular analisa argumentos de linha de comando: veja Argumentos de linha de comando.

Padrão: `1` na configuração do Python, `0` na configuração isolada.

int use_environment

Usar variáveis de ambiente? Veja **`PyConfig.use_environment`**.

Padrão: `1` na configuração do Python e `0` na configuração isolada.

int utf8_mode

Se não zero, habilita o modo UTF-8 do Python.

Define para `0` ou `1` pela opção de linha de comando **`-X utf8`** e a variável de ambiente **`PYTHONUTF8`**.

Padrão: `1`.

Pré-inicializar Python com PyPreConfig

A pré-inicialização do Python:

- Define os alocadores de memória Python (**`PyPreConfig.allocator`**)
- Configura a localidade LC_CTYPE (codificação da localidade)
- Define o Modo UTF-8 do Python (**`PyPreConfig.utf8_mode`**)

A pré-configuração atual (tipo `PyPreConfig`) é armazenada em `_PyRuntime.preconfig`.

Funções para pré-inicializar Python:

PyStatus Py_PreInitialize(const PyPreConfig *preconfig)

Pré-inicializa o Python a partir da pré-configuração *preconfig*.

preconfig não pode ser `NULL`..

PyStatus Py_PreInitializeFromBytesArgs(const PyPreConfig *preconfig, int argc, char *const *argv)

Pré-inicializa o Python a partir da pré-configuração *preconfig*.

Analisa argumentos de linha de comando *argv* (strings de bytes) se **`parse_argv`** de *preconfig* for diferente de zero.

preconfig não pode ser `NULL`..

PyStatus Py_PreInitializeFromArgs(const PyPreConfig *preconfig, int argc, wchar_t *const *argv)

Pré-inicializa o Python a partir da pré-configuração *preconfig*.

Analisa argumentos de linha de comando *argv* (strings largas) se **`parse_argv`** de *preconfig* for diferente de zero.

preconfig não pode ser `NULL`..

O chamador é responsável por manipular exceções (erro ou saída) usando **`PyStatus_Exception()`** e **`Py_ExitStatusException()`**.

Para configuração do Python (**PyPreConfig_InitPythonConfig()**), se o Python for inicializado com argumentos de linha de comando, os argumentos de linha de comando também devem ser passados para pré-inicializar o Python, pois eles têm um efeito na pré-configuração como codificações. Por exemplo, a opção de linha de comando **-X utf8** habilita o Modo UTF-8 do Python.

`PyMem_SetAllocator()` pode ser chamado depois de **Py_PreInitialize()** e antes de **Py_InitializeFromConfig()** para instalar um alocador de memória personalizado. Ele pode ser chamado antes de **Py_PreInitialize()** se **PyPreConfig.allocator** estiver definido como `PYMEM_ALLOCATOR_NOT_SET`.

Funções de alocação de memória do Python como **PyMem_RawMalloc()** não devem ser usadas antes da pré-inicialização do Python, enquanto chamar diretamente `malloc()` e `free()` é sempre seguro. **Py_DecodeLocale()** não deve ser chamado antes da pré-inicialização do Python.

Exemplo usando a pré-inicialização para habilitar o modo UTF-8 do Python.

```
PyStatus status;
PyPreConfig preconfig;
PyPreConfig_InitPythonConfig(&preconfig);

preconfig.utf8_mode = 1;

status = Py_PreInitialize(&preconfig);
if (PyStatus_Exception(status)) {
    Py_ExitStatusException(status);
}

/* at this point, Python speaks UTF-8 */

Py_Initialize();
/* ... use Python API here ... */
Py_Finalize();
```

PyConfig

type PyConfig

Estrutura contendo a maioria dos parâmetros para configurar o Python.

Ao terminar, a função **PyConfig_Clear()** deve ser usada para liberar a memória de configuração.

Structure methods:

void PyConfig_InitPythonConfig(PyConfig *config)

Initialize configuration with the Python Configuration.

void PyConfig_InitIsolatedConfig(PyConfig *config)

Initialize configuration with the Isolated Configuration.

PyStatus PyConfig_SetString(PyConfig *config, wchar_t *const *config_str, const wchar_t *str)

Copy the wide character string *str* into `*config_str`.

Preinitialize Python if needed.

PyStatus PyConfig_SetBytesString(PyConfig *config, wchar_t *const *config_str, const char *str)

Decode *str* using **`Py_DecodeLocale()`** and set the result into `*config_str`.

Preinitialize Python if needed.

PyStatus PyConfig_SetArgv(PyConfig *config, int argc, wchar_t *const *argv)

Set command line arguments (**`argv`** member of *config*) from the *argv* list of wide character strings.

Preinitialize Python if needed.

PyStatus PyConfig_SetBytesArgv(PyConfig *config, int argc, char *const *argv)

Set command line arguments (**`argv`** member of *config*) from the *argv* list of bytes strings. Decode bytes using **`Py_DecodeLocale()`**.

Preinitialize Python if needed.

PyStatus PyConfig_SetWideStringList(PyConfig *config, PyWideStringList *list, Py_ssize_t length, wchar_t **items)

Set the list of wide strings *list* to *length* and *items*.

Preinitialize Python if needed.

PyStatus PyConfig_Read(PyConfig *config)

Lê toda a configuração do Python.

Campos que já estão inicializados permanecem inalterados.

Fields for path configuration are no longer calculated or modified when calling this function, as of Python 3.11.

The **`PyConfig_Read()`** function only parses **`PyConfig.argv`** arguments once: **`PyConfig.parse_argv`** is set to `2` after arguments are parsed. Since Python arguments are stripped from **`PyConfig.argv`**, parsing arguments twice would parse the application options as Python options.

Preinitialize Python if needed.

Alterado na versão 3.10: The **PyConfig.argv** arguments are now only parsed once, **PyConfig.parse_argv** is set to `2` after arguments are parsed, and arguments are only parsed if **PyConfig.parse_argv** equals `1`.

Alterado na versão 3.11: **PyConfig_Read()** no longer calculates all paths, and so fields listed under Python Path Configuration may no longer be updated until **Py_InitializeFromConfig()** is called.

void PyConfig_Clear(PyConfig *config)

Libera memória de configuração.

Most `PyConfig` methods preinitialize Python if needed. In that case, the Python preinitialization configuration (**PyPreConfig**) is based on the **PyConfig**. If configuration fields which are in common with **PyPreConfig** are tuned, they must be set before calling a **PyConfig** method:

- **PyConfig.dev_mode**
- **PyConfig.isolated**
- **PyConfig.parse_argv**
- **PyConfig.use_environment**

Moreover, if **PyConfig_SetArgv()** or **PyConfig_SetBytesArgv()** is used, this method must be called before other methods, since the preinitialization configuration depends on command line arguments (if **parse_argv** is non-zero).

The caller of these methods is responsible to handle exceptions (error or exit) using `PyStatus_Exception()` and `Py_ExitStatusException()`.

Campos de estrutura:

PyWideStringList argv

Set **sys.argv** command line arguments based on **argv**. These parameters are similar to those passed to the program's **main()** function with the difference that the first entry should refer to the script file to be executed rather than the executable hosting the Python interpreter. If there isn't a script that will be run, the first entry in **argv** can be an empty string.

Set **parse_argv** to `1` to parse **argv** the same way the regular Python parses Python command line arguments and then to strip Python arguments from **argv**.

If **argv** is empty, an empty string is added to ensure that **sys.argv** always exists and is never empty.

Padrão: `NULL`.

See also the **orig_argv** member.

int safe_path

If equals to zero, `Py_RunMain()` prepends a potentially unsafe path to **sys.path** at startup:

- If **argv[0]** is equal to `L"-m"` (`python -m module`), prepend the current working directory.
- If running a script (`python script.py`), prepend the script's directory. If it's a symbolic link, resolve symbolic links.
- Otherwise (`python -c code` and `python`), prepend an empty string, which means the current working directory.

Set to `1` by the **-P** command line option and the **PYTHONSAFEPATH** environment variable.

Default: `0` in Python config, `1` in isolated config.

Adicionado na versão 3.11.

wchar_t *base_exec_prefix

sys.base_exec_prefix.

Padrão: `NULL`.

Part of the Python Path Configuration output.

Veja também **PyConfig.exec_prefix**.

wchar_t *base_executable

Python base executable: `sys._base_executable`.

Set by the `__PYVENV_LAUNCHER__` environment variable.

Set from **PyConfig.executable** if `NULL`.

Padrão: `NULL`.

Part of the Python Path Configuration output.

See also **PyConfig.executable**.

wchar_t *base_prefix

sys.base_prefix.

Padrão: `NULL`.

Part of the Python Path Configuration output.

See also `PyConfig.prefix`.

int buffered_stdio

If equals to `0` and `configure_c_stdio` is non-zero, disable buffering on the C streams stdout and stderr.

Set to `0` by the `-u` command line option and the `PYTHONUNBUFFERED` environment variable.

stdin is always opened in buffered mode.

Padrão: `1`.

int bytes_warning

If equals to `1`, issue a warning when comparing `bytes` or `bytearray` with `str`, or comparing `bytes` with `int`.

If equal or greater to `2`, raise a `BytesWarning` exception in these cases.

Incremented by the `-b` command line option.

Padrão: `0`.

Descontinuado desde a versão 3.15, será removido na versão 3.17: The `-b` and `-bb` options will become no-op in 3.17. `bytes_warning` member will be removed in 3.17.

int warn_default_encoding

If non-zero, emit a `EncodingWarning` warning when `io.TextIOWrapper` uses its default encoding. See Ativar EncodingWarning for details.

Padrão: `0`.

Adicionado na versão 3.10.

int code_debug_ranges

If equals to `0`, disables the inclusion of the end line and column mappings in code objects. Also disables traceback printing carets to specific error locations.

Set to `0` by the `PYTHONNODEBUGRANGES` environment variable and by the `-X no_debug_ranges` command line option.

Padrão: `1`.

Adicionado na versão 3.11.

wchar_t *check_hash_pycs_mode

Control the validation behavior of hash-based `.pyc` files: value of the **`--check-hash-based-pycs`** command line option.

Valores válidos:

- `L"always"`: Hash the source file for invalidation regardless of value of the 'check_source' flag.
- `L"never"`: Assume that hash-based pycs always are valid.
- `L"default"`: The 'check_source' flag in hash-based pycs determines invalidation.

Default: `L"default"`.

See also **PEP 552** [https://peps.python.org/pep-0552/] "Deterministic pycs".

int configure_c_stdio

If non-zero, configure C standard streams:

- On Windows, set the binary mode (`O_BINARY`) on stdin, stdout and stderr.
- If **`buffered_stdio`** equals zero, disable buffering of stdin, stdout and stderr streams.
- If **`interactive`** is non-zero, enable stream buffering on stdin and stdout (only stdout on Windows).

Padrão: `1` na configuração do Python, `0` na configuração isolada.

int dev_mode

If non-zero, enable the Python Development Mode.

Set to `1` by the **`-X dev`** option and the **`PYTHONDEVMODE`** environment variable.

Padrão: `-1` no modo do Python, `0` no modo isolado.

int dump_refs

Dump Python references?

If non-zero, dump all objects which are still alive at exit.

Set to `1` by the **`PYTHONDUMPREFS`** environment variable.

Needs a special build of Python with the `Py_TRACE_REFS` macro defined: see the **`configure --with-trace-refs option`**.

Padrão: `0`.

wchar_t *dump_refs_file

Filename where to dump Python references.

Set by the **PYTHONDUMPREFSFILE** environment variable.

Padrão: `NULL`.

Adicionado na versão 3.11.

wchar_t *exec_prefix

The site-specific directory prefix where the platform-dependent Python files are installed: **sys.exec_prefix**.

Padrão: `NULL`.

Part of the Python Path Configuration output.

See also **PyConfig.base_exec_prefix**.

wchar_t *executable

The absolute path of the executable binary for the Python interpreter: **sys.executable**.

Padrão: `NULL`.

Part of the Python Path Configuration output.

See also **PyConfig.base_executable**.

int faulthandler

Enable faulthandler?

If non-zero, call **faulthandler.enable()** at startup.

Set to `1` by **-X faulthandler** and the **PYTHONFAULTHANDLER** environment variable.

Padrão: `-1` no modo do Python, `0` no modo isolado.

wchar_t *filesystem_encoding

Filesystem encoding: **sys.getfilesystemencoding()**.

On macOS, Android and VxWorks: use `"utf-8"` by default.

On Windows: use `"utf-8"` by default, or `"mbcs"` if **legacy_windows_fs_encoding** of **PyPreConfig** is non-zero.

Default encoding on other platforms:

- `"utf-8"` if **PyPreConfig.utf8_mode** is non-zero.
- `"ascii"` if Python detects that `nl_langinfo(CODESET)` announces the ASCII encoding, whereas the `mbstowcs()` function decodes from a different encoding

(usually Latin1).
- "utf-8" if nl_langinfo(CODESET) returns an empty string.
- Otherwise, use the locale encoding: nl_langinfo(CODESET) result.

At Python startup, the encoding name is normalized to the Python codec name. For example, "ANSI_X3.4-1968" is replaced with "ascii".

See also the **filesystem_errors** member.

wchar_t *filesystem_errors
Filesystem error handler: **sys.getfilesystemencodeerrors()**.

On Windows: use "surrogatepass" by default, or "replace" if **legacy_windows_fs_encoding** of **PyPreConfig** is non-zero.

On other platforms: use "surrogateescape" by default.

Supported error handlers:

- "strict"
- "surrogateescape"
- "surrogatepass" (only supported with the UTF-8 encoding)

See also the **filesystem_encoding** member.

int use_frozen_modules
If non-zero, use frozen modules.

Set by the **PYTHON_FROZEN_MODULES** environment variable.

Default: 1 in a release build, or 0 in a debug build.

unsigned long hash_seed

int use_hash_seed
Randomized hash function seed.

If **use_hash_seed** is zero, a seed is chosen randomly at Python startup, and **hash_seed** is ignored.

Set by the **PYTHONHASHSEED** environment variable.

Default *use_hash_seed* value: -1 in Python mode, 0 in isolated mode.

wchar_t *home
Set the default Python “home” directory, that is, the location of the standard Python libraries (see **PYTHONHOME**).

Set by the **PYTHONHOME** environment variable.

Padrão: `NULL`.

Part of the Python Path Configuration input.

int import_time

If `1`, profile import time. If `2`, include additional output that indicates when an imported module has already been loaded.

Set by the `-X importtime` option and the `PYTHONPROFILEIMPORTTIME` environment variable.

Padrão: `0`.

Alterado na versão 3.14: Added support for `import_time = 2`

int inspect

Enter interactive mode after executing a script or a command.

If greater than `0`, enable inspect: when a script is passed as first argument or the -c option is used, enter interactive mode after executing the script or the command, even when `sys.stdin` does not appear to be a terminal.

Incremented by the `-i` command line option. Set to `1` if the `PYTHONINSPECT` environment variable is non-empty.

Padrão: `0`.

int install_signal_handlers

Install Python signal handlers?

Default: `1` in Python mode, `0` in isolated mode.

int interactive

If greater than `0`, enable the interactive mode (REPL).

Incremented by the `-i` command line option.

Padrão: `0`.

int int_max_str_digits

Configures the integer string conversion length limitation. An initial value of `-1` means the value will be taken from the command line or environment or otherwise default to 4300 (`sys.int_info.default_max_str_digits`). A value of `0` disables the limitation. Values greater than zero but less than 640 (`sys.int_info.str_digits_check_threshold`) are unsupported and will produce an error.

Configured by the `-X int_max_str_digits` command line flag or the `PYTHONINTMAXSTRDIGITS` environment variable.

Default: `-1` in Python mode. 4300 (`sys.int_info.default_max_str_digits`) in isolated mode.

Adicionado na versão 3.12.

int cpu_count

If the value of `cpu_count` is not `-1` then it will override the return values of `os.cpu_count()`, `os.process_cpu_count()`, and `multiprocessing.cpu_count()`.

Configured by the `-X cpu_count=n|default` command line flag or the `PYTHON_CPU_COUNT` environment variable.

Default: `-1`.

Adicionado na versão 3.13.

int isolated

If greater than `0`, enable isolated mode:

- Set `safe_path` to `1`: don't prepend a potentially unsafe path to `sys.path` at Python startup, such as the current directory, the script's directory or an empty string.
- Set `use_environment` to `0`: ignore `PYTHON` environment variables.
- Set `user_site_directory` to `0`: don't add the user site directory to `sys.path`.
- Python REPL doesn't import `readline` nor enable default readline configuration on interactive prompts.

Set to `1` by the `-I` command line option.

Padrão: `0` no modo do Python, `1` no modo isolado.

See also the Isolated Configuration and `PyPreConfig.isolated`.

int legacy_windows_stdio

If non-zero, use `io.FileIO` instead of `io._WindowsConsoleIO` for `sys.stdin`, `sys.stdout` and `sys.stderr`.

Definida como `1` se a variável de ambiente `PYTHONLEGACYWINDOWSSTDIO` estiver definida como uma string não vazia.

Disponível apenas no Windows. A macro `#ifdef MS_WINDOWS` pode ser usada para código específico do Windows.

Padrão: `0`.

See also the **PEP 528** [https://peps.python.org/pep-0528/] (Change Windows console encoding to UTF-8).

int malloc_stats

If non-zero, dump statistics on Python pymalloc memory allocator at exit.

Set to `1` by the **`PYTHONMALLOCSTATS`** environment variable.

The option is ignored if Python is **`configured using the --without-pymalloc option`**.

Padrão: `0`.

wchar_t *platlibdir

Platform library directory name: **`sys.platlibdir`**.

Set by the **`PYTHONPLATLIBDIR`** environment variable.

Default: value of the `PLATLIBDIR` macro which is set by the **`configure --with-platlibdir option`** (default: `"lib"`, or `"DLLs"` on Windows).

Part of the Python Path Configuration input.

Adicionado na versão 3.9.

Alterado na versão 3.11: This macro is now used on Windows to locate the standard library extension modules, typically under `DLLs`. However, for compatibility, note that this value is ignored for any non-standard layouts, including in-tree builds and virtual environments.

wchar_t *pythonpath_env

Module search paths (**`sys.path`**) as a string separated by `DELIM` (**`os.pathsep`**).

Set by the **`PYTHONPATH`** environment variable.

Padrão: `NULL`.

Part of the Python Path Configuration input.

PyWideStringList module_search_paths

int module_search_paths_set

Module search paths: **`sys.path`**.

If **`module_search_paths_set`** is equal to `0`, **`Py_InitializeFromConfig()`** will replace **`module_search_paths`** and sets **`module_search_paths_set`** to `1`.

Default: empty list (`module_search_paths`) and `0` (`module_search_paths_set`).

Part of the Python Path Configuration output.

int optimization_level

Compilation optimization level:

- `0`: Peephole optimizer, set `__debug__` to `True`.
- `1`: Level 0, remove assertions, set `__debug__` to `False`.
- `2`: Level 1, strip docstrings.

Incremented by the `-O` command line option. Set to the `PYTHONOPTIMIZE` environment variable value.

Padrão: `0`.

PyWideStringList orig_argv

The list of the original command line arguments passed to the Python executable: `sys.orig_argv`.

If `orig_argv` list is empty and `argv` is not a list only containing an empty string, `PyConfig_Read()` copies **`argv`** into **`orig_argv`** before modifying **`argv`** (if `parse_argv` is non-zero).

See also the `argv` member and the `Py_GetArgcArgv()` function.

Padrão: lista vazia.

Adicionado na versão 3.10.

int parse_argv

Parse command line arguments?

If equals to `1`, parse `argv` the same way the regular Python parses command line arguments, and strip Python arguments from **`argv`**.

The `PyConfig_Read()` function only parses `PyConfig.argv` arguments once: `PyConfig.parse_argv` is set to `2` after arguments are parsed. Since Python arguments are stripped from **`PyConfig.argv`**, parsing arguments twice would parse the application options as Python options.

Default: `1` in Python mode, `0` in isolated mode.

Alterado na versão 3.10: The `PyConfig.argv` arguments are now only parsed if `PyConfig.parse_argv` equals to `1`.

int parser_debug

Parser debug mode. If greater than `0`, turn on parser debugging output (for expert only, depending on compilation options).

Incremented by the **-d** command line option. Set to the **PYTHONDEBUG** environment variable value.

Needs a debug build of Python (the `Py_DEBUG` macro must be defined).

Padrão: `0`.

int pathconfig_warnings

If non-zero, calculation of path configuration is allowed to log warnings into `stderr`. If equals to `0`, suppress these warnings.

Default: `1` in Python mode, `0` in isolated mode.

Part of the Python Path Configuration input.

Alterado na versão 3.11: Now also applies on Windows.

wchar_t *prefix

The site-specific directory prefix where the platform independent Python files are installed: **sys.prefix**.

Padrão: `NULL`.

Part of the Python Path Configuration output.

See also **PyConfig.base_prefix**.

wchar_t *program_name

Program name used to initialize **executable** and in early error messages during Python initialization.

- On macOS, use **PYTHONEXECUTABLE** environment variable if set.
- If the `WITH_NEXT_FRAMEWORK` macro is defined, use `__PYVENV_LAUNCHER__` environment variable if set.
- Use `argv[0]` of **argv** if available and non-empty.
- Otherwise, use `L"python"` on Windows, or `L"python3"` on other platforms.

Padrão: `NULL`.

Part of the Python Path Configuration input.

wchar_t *pycache_prefix

Directory where cached `.pyc` files are written: **sys.pycache_prefix**.

Set by the **-X pycache_prefix=PATH** command line option and the **PYTHONPYCACHEPREFIX** environment variable. The command-line option takes precedence.

If `NULL`, `sys.pycache_prefix` is set to `None`.

Padrão: `NULL`.

int quiet

Quiet mode. If greater than `0`, don't display the copyright and version at Python startup in interactive mode.

Incremented by the `-q` command line option.

Padrão: `0`.

wchar_t *run_command

Value of the `-c` command line option.

Used by `Py_RunMain()`.

Padrão: `NULL`.

wchar_t *run_filename

Filename passed on the command line: trailing command line argument without `-c` or `-m`. It is used by the `Py_RunMain()` function.

For example, it is set to `script.py` by the `python3 script.py arg` command line.

See also the `PyConfig.skip_source_first_line` option.

Padrão: `NULL`.

wchar_t *run_module

Value of the `-m` command line option.

Used by `Py_RunMain()`.

Padrão: `NULL`.

wchar_t *run_presite

`module` or `module:func` entry point that should be executed before the `site` module is imported.

Set by the `-X presite=module:func` command-line option and the `PYTHON_PRESITE` environment variable. The command-line option takes precedence.

Needs a debug build of Python (the `Py_DEBUG` macro must be defined).

Padrão: `NULL`.

int show_ref_count

Show total reference count at exit (excluding immortal objects)?

Set to `1` by `-X showrefcount` command line option.

Needs a debug build of Python (the `Py_REF_DEBUG` macro must be defined).

Padrão: `0`.

int site_import

Import the `site` module at startup?

If equal to zero, disable the import of the module site and the site-dependent manipulations of `sys.path` that it entails.

Also disable these manipulations if the `site` module is explicitly imported later (call `site.main()` if you want them to be triggered).

Set to `0` by the `-S` command line option.

`sys.flags.no_site` is set to the inverted value of `site_import`.

Padrão: `1`.

int skip_source_first_line

If non-zero, skip the first line of the `PyConfig.run_filename` source.

It allows the usage of non-Unix forms of `#!cmd`. This is intended for a DOS specific hack only.

Set to `1` by the `-x` command line option.

Padrão: `0`.

wchar_t *stdio_encoding

wchar_t *stdio_errors

Encoding and encoding errors of `sys.stdin`, `sys.stdout` and `sys.stderr` (but **`sys.stderr`** always uses `"backslashreplace"` error handler).

Use the `PYTHONIOENCODING` environment variable if it is non-empty.

Codificação padrão:

- `"UTF-8"` if `PyPreConfig.utf8_mode` is non-zero.
- Otherwise, use the locale encoding.

Tratador de erros padrão:

- On Windows: use `"surrogateescape"`.

- `"surrogateescape"` if `PyPreConfig.utf8_mode` is non-zero, or if the LC_CTYPE locale is “C” or “POSIX”.
- `"strict"` otherwise.

See also `PyConfig.legacy_windows_stdio`.

int tracemalloc

Enable tracemalloc?

If non-zero, call `tracemalloc.start()` at startup.

Set by `-X tracemalloc=N` command line option and by the `PYTHONTRACEMALLOC` environment variable.

Padrão: `-1` no modo do Python, `0` no modo isolado.

int perf_profiling

Enable the Linux `perf` profiler support?

If equals to `1`, enable support for the Linux `perf` profiler.

If equals to `2`, enable support for the Linux `perf` profiler with DWARF JIT support.

Set to `1` by `-X perf` command-line option and the `PYTHONPERFSUPPORT` environment variable.

Set to `2` by the `-X perf_jit` command-line option and the `PYTHON_PERF_JIT_SUPPORT` environment variable.

Default: `-1`.

Ver também

See Python support for the perf map compatible profilers for more information.

Adicionado na versão 3.12.

wchar_t *stdlib_dir

Directory of the Python standard library.

Padrão: `NULL`.

Adicionado na versão 3.11.

int use_environment

Use environment variables?

If equals to zero, ignore the environment variables.

Set to `0` by the `-E` environment variable.

Padrão: `1` na configuração do Python e `0` na configuração isolada.

int use_system_logger

If non-zero, `stdout` and `stderr` will be redirected to the system log.

Only available on macOS 10.12 and later, and on iOS.

Default: `0` (don't use the system log) on macOS; `1` on iOS (use the system log).

Adicionado na versão 3.14.

int user_site_directory

If non-zero, add the user site directory to `sys.path`.

Set to `0` by the `-s` and `-I` command line options.

Set to `0` by the `PYTHONNOUSERSITE` environment variable.

Default: `1` in Python mode, `0` in isolated mode.

int verbose

Verbose mode. If greater than `0`, print a message each time a module is imported, showing the place (filename or built-in module) from which it is loaded.

If greater than or equal to `2`, print a message for each file that is checked for when searching for a module. Also provides information on module cleanup at exit.

Incremented by the `-v` command line option.

Set by the `PYTHONVERBOSE` environment variable value.

Padrão: `0`.

PyWideStringList warnoptions

Options of the `warnings` module to build warnings filters, lowest to highest priority: `sys.warnoptions`.

The `warnings` module adds `sys.warnoptions` in the reverse order: the last `PyConfig.warnoptions` item becomes the first item of `warnings.filters` which is checked first (highest priority).

The `-W` command line options adds its value to `warnoptions`, it can be used multiple times.

The `PYTHONWARNINGS` environment variable can also be used to add warning options. Multiple options can be specified, separated by commas (`,`).

Padrão: lista vazia.

int write_bytecode
If equal to `0`, Python won't try to write `.pyc` files on the import of source modules.

Set to `0` by the `-B` command line option and the `PYTHONDONTWRITEBYTECODE` environment variable.

`sys.dont_write_bytecode` is initialized to the inverted value of `write_bytecode`.

Padrão: `1`.

PyWideStringList xoptions
Values of the `-X` command line options: `sys._xoptions`.

Padrão: lista vazia.

int _pystats
If non-zero, write performance statistics at Python exit.

Need a special build with the `Py_STATS` macro: see `--enable-pystats`.

Padrão: `0`.

If `parse_argv` is non-zero, `argv` arguments are parsed the same way the regular Python parses command line arguments, and Python arguments are stripped from **`argv`**.

The `xoptions` options are parsed to set other options: see the `-X` command line option.

Alterado na versão 3.9: The `show_alloc_count` field has been removed.

Initialization with PyConfig

Initializing the interpreter from a populated configuration struct is handled by calling `Py_InitializeFromConfig()`.

O chamador é responsável por manipular exceções (erro ou saída) usando `PyStatus_Exception()` e `Py_ExitStatusException()`.

If `PyImport_FrozenModules()`, `PyImport_AppendInittab()` or `PyImport_ExtendInittab()` are used, they must be set or called after Python preinitialization and before the Python initialization. If Python is initialized multiple times, **`PyImport_AppendInittab()`** or **`PyImport_ExtendInittab()`** must be called before each Python initialization.

The current configuration (`PyConfig` type) is stored in `PyInterpreterState.config`.

Example setting the program name:

```
void init_python(void)
{
    PyStatus status;

    PyConfig config;
    PyConfig_InitPythonConfig(&config);

    /* Set the program name. Implicitly preinitialize Python. */
    status = PyConfig_SetString(&config, &config.program_name,
                                L"/path/to/my_program");
    if (PyStatus_Exception(status)) {
        goto exception;
    }

    status = Py_InitializeFromConfig(&config);
    if (PyStatus_Exception(status)) {
        goto exception;
    }
    PyConfig_Clear(&config);
    return;

exception:
    PyConfig_Clear(&config);
    Py_ExitStatusException(status);
}
```

More complete example modifying the default configuration, read the configuration, and then override some parameters. Note that since 3.11, many parameters are not calculated until initialization, and so values cannot be read from the configuration structure. Any values set before initialize is called will be left unchanged by initialization:

```
PyStatus init_python(const char *program_name)
{
    PyStatus status;

    PyConfig config;
    PyConfig_InitPythonConfig(&config);

    /* Set the program name before reading the configuration
       (decode byte string from the locale encoding).

       Implicitly preinitialize Python. */
    status = PyConfig_SetBytesString(&config, &config.program_name,
                                     program_name);
    if (PyStatus_Exception(status)) {
        goto done;
    }

    /* Read all configuration at once */
    status = PyConfig_Read(&config);
    if (PyStatus_Exception(status)) {
        goto done;
```

```
    }

    /* Specify sys.path explicitly */
    /* If you want to modify the default set of paths, finish
       initialization first and then use PySys_GetAttrString("path") */
    config.module_search_paths_set = 1;
    status = PyWideStringList_Append(&config.module_search_paths,
                                     L"/path/to/stdlib");
    if (PyStatus_Exception(status)) {
        goto done;
    }
    status = PyWideStringList_Append(&config.module_search_paths,
                                     L"/path/to/more/modules");
    if (PyStatus_Exception(status)) {
        goto done;
    }

    /* Override executable computed by PyConfig_Read() */
    status = PyConfig_SetString(&config, &config.executable,
                                L"/path/to/my_executable");
    if (PyStatus_Exception(status)) {
        goto done;
    }

    status = Py_InitializeFromConfig(&config);

done:
    PyConfig_Clear(&config);
    return status;
}
```

Isolated Configuration

`PyPreConfig_InitIsolatedConfig()` and `PyConfig_InitIsolatedConfig()` functions create a configuration to isolate Python from the system. For example, to embed Python into an application.

This configuration ignores global configuration variables, environment variables, command line arguments (`PyConfig.argv` is not parsed) and user site directory. The C standard streams (ex: `stdout`) and the LC_CTYPE locale are left unchanged. Signal handlers are not installed.

Configuration files are still used with this configuration to determine paths that are unspecified. Ensure `PyConfig.home` is specified to avoid computing the default path configuration.

Configuração do Python

`PyPreConfig_InitPythonConfig()` and `PyConfig_InitPythonConfig()` functions create a configuration to build a customized Python which behaves as the regular Python.

Environments variables and command line arguments are used to configure Python, whereas global configuration variables are ignored.

This function enables C locale coercion (**PEP 538** [https://peps.python.org/pep-0538/]) and Python UTF-8 Mode (**PEP 540** [https://peps.python.org/pep-0540/]) depending on the LC_CTYPE locale, `PYTHONUTF8` and `PYTHONCOERCECLOCALE` environment variables.

Python Path Configuration

`PyConfig` contains multiple fields for the path configuration:

- Path configuration inputs:
  - `PyConfig.home`
  - `PyConfig.platlibdir`
  - `PyConfig.pathconfig_warnings`
  - `PyConfig.program_name`
  - `PyConfig.pythonpath_env`
  - current working directory: to get absolute paths
  - `PATH` environment variable to get the program full path (from `PyConfig.program_name`)
  - `__PYVENV_LAUNCHER__` environment variable
  - (Windows only) Application paths in the registry under "SoftwarePythonPythonCoreX.YPythonPath" of HKEY_CURRENT_USER and HKEY_LOCAL_MACHINE (where X.Y is the Python version).
- Path configuration output fields:
  - `PyConfig.base_exec_prefix`
  - `PyConfig.base_executable`
  - `PyConfig.base_prefix`
  - `PyConfig.exec_prefix`
  - `PyConfig.executable`
  - `PyConfig.module_search_paths_set`, `PyConfig.module_search_paths`
  - `PyConfig.prefix`

If at least one "output field" is not set, Python calculates the path configuration to fill unset fields. If `module_search_paths_set` is equal to `0`, `module_search_paths` is overridden and **`module_search_paths_set`** is set to `1`.

It is possible to completely ignore the function calculating the default path configuration by setting explicitly all path configuration output fields listed above. A string is considered as set even if it is non-empty. `module_search_paths` is considered as set if `module_search_paths_set` is set to `1`. In this case, `module_search_paths` will be used without modification.

Set `pathconfig_warnings` to `0` to suppress warnings when calculating the path configuration (Unix only, Windows does not log any warning).

If `base_prefix` or `base_exec_prefix` fields are not set, they inherit their value from `prefix` and `exec_prefix` respectively.

`Py_RunMain()` and `Py_Main()` modify `sys.path`:

- If `run_filename` is set and is a directory which contains a `__main__.py` script, prepend **`run_filename`** to `sys.path`.
- If `isolated` is zero:
  - If `run_module` is set, prepend the current directory to `sys.path`. Do nothing if the current directory cannot be read.
  - If `run_filename` is set, prepend the directory of the filename to `sys.path`.
  - Otherwise, prepend an empty string to `sys.path`.

If `site_import` is non-zero, `sys.path` can be modified by the `site` module. If `user_site_directory` is non-zero and the user's site-package directory exists, the **`site`** module appends the user's site-package directory to **`sys.path`**.

The following configuration files are used by the path configuration:

- `pyvenv.cfg`
- `._pth` file (ex: `python._pth`)
- `pybuilddir.txt` (Unix only)

If a `._pth` file is present:

- Set `isolated` to `1`.
- Set `use_environment` to `0`.
- Set `site_import` to `0`.
- Set `user_site_directory` to `0` (since 3.15).
- Set `safe_path` to `1`.

If `home` is not set and a `pyvenv.cfg` file is present in the same directory as `executable`, or its parent, `prefix` and `exec_prefix` are set that location. When this happens, `base_prefix` and `base_exec_prefix` still keep their value, pointing to the base installation. See Ambientes virtuais for more information.

The `__PYVENV_LAUNCHER__` environment variable is used to set `PyConfig.base_executable`.

Alterado na versão 3.14: `prefix`, and `exec_prefix`, are now set to the `pyvenv.cfg` directory. This was previously done by `site`, therefore affected by `-S`.

Alterado na versão 3.15: `user_site_directory` is now set to `0` when a `._pth` file is present.

Py_GetArgcArgv()

void Py_GetArgcArgv(int *argc, wchar_t ***argv)

Get the original command line arguments, before Python modified them.

See also `PyConfig.orig_argv` member.

Multi-Phase Initialization Private Provisional API

This section is a private provisional API introducing multi-phase initialization, the core feature of **PEP 432** [https://peps.python.org/pep-0432/]:

- "Core" initialization phase, "bare minimum Python":
  - Builtin types;
  - Builtin exceptions;
  - Builtin and frozen modules;
  - The `sys` module is only partially initialized (ex: `sys.path` doesn't exist yet).
- "Main" initialization phase, Python is fully initialized:
  - Install and configure `importlib`;
  - Apply the Path Configuration;
  - Install signal handlers;
  - Finish `sys` module initialization (ex: create `sys.stdout` and `sys.path`);
  - Enable optional features like `faulthandler` and `tracemalloc`;
  - Import the `site` module;
  - etc.

Private provisional API:

int PyConfig._init_main

If set to `0`, `Py_InitializeFromConfig()` stops at the "Core" initialization phase.

PyStatus _Py_InitializeMain(void)

Move to the "Main" initialization phase, finish the Python initialization.

No module is imported during the "Core" phase and the `importlib` module is not configured: the Path Configuration is only applied during the "Main" phase. It may allow to customize Python in Python to override or tune the Path Configuration, maybe install a custom `sys.meta_path` importer or an import hook, etc.

It may become possible to calculate the Path Configuration in Python, after the Core phase and before the Main phase, which is one of the **PEP 432** [https://peps.python.org/pep-0432/] motivation.

The "Core" phase is not properly defined: what should be and what should not be available at this phase is not specified yet. The API is marked as private and provisional: the API can be modified or even be removed anytime until a proper public API is designed.

Example running Python code between "Core" and "Main" initialization phases:

```
void init_python(void)
{
    PyStatus status;

    PyConfig config;
    PyConfig_InitPythonConfig(&config);
    config._init_main = 0;

    /* ... customize 'config' configuration ... */

    status = Py_InitializeFromConfig(&config);
    PyConfig_Clear(&config);
    if (PyStatus_Exception(status)) {
        Py_ExitStatusException(status);
    }

    /* Use sys.stderr because sys.stdout is only created
       by _Py_InitializeMain() */
    int res = PyRun_SimpleString(
        "import sys; "
        "print('Run Python code before _Py_InitializeMain', "
               "file=sys.stderr)");
    if (res < 0) {
        exit(1);
    }

    /* ... put more configuration code here ... */

    status = _Py_InitializeMain();
    if (PyStatus_Exception(status)) {
        Py_ExitStatusException(status);
    }
}
```

Gerenciamento de Memória

Visão Geral

O gerenciamento de memória em Python envolve um heap privado contendo todos os objetos e estruturas de dados Python. O gerenciamento desse heap privado é garantido internamente pelo *gerenciador de memória do Python*. O gerenciador de memória do Python possui diferentes componentes que lidam com diversos aspectos dinâmicos do gerenciamento de armazenamento, como compartilhamento, segmentação, pré-alocação ou cache.

No nível mais baixo, um alocador de memória bruta garante que haja espaço suficiente no heap privado para armazenar todos os dados relacionados ao Python, interagindo com o gerenciador de memória do sistema operacional. Além do alocador de memória bruta, vários alocadores específicos de objeto operam no mesmo heap e implementam políticas distintas de gerenciamento de memória, adaptadas às peculiaridades de cada tipo de objeto. Por exemplo, objetos inteiros são gerenciados dentro do heap de forma diferente de strings, tuplas ou dicionários, pois inteiros implicam diferentes requisitos de armazenamento e compensações entre velocidade e espaço. O gerenciador de memória do Python, portanto, delega parte do trabalho aos alocadores específicos de objeto, mas garante que estes operem dentro dos limites do heap privado.

It is important to understand that the management of the Python heap is performed by the interpreter itself and that the user has no control over it, even if they regularly manipulate object pointers to memory blocks inside that heap. The allocation of heap space for Python objects and other internal buffers is performed on demand by the Python memory manager through the Python/C API functions listed in this document.

To avoid memory corruption, extension writers should never try to operate on Python objects with the functions exported by the C library: **`malloc()`**, **`calloc()`**, **`realloc()`** and **`free()`**. This will result in mixed calls between the C allocator and the Python memory manager with fatal consequences, because they implement different algorithms and operate on different heaps. However, one may safely allocate and release memory blocks with the C library allocator for individual purposes, as shown in the following example:

```
PyObject *res;
char *buf = (char *) malloc(BUFSIZ); /* for I/O */

if (buf == NULL)
    return PyErr_NoMemory();
...Do some I/O operation involving buf...
res = PyBytes_FromString(buf);
free(buf); /* malloc'ed */
return res;
```

In this example, the memory request for the I/O buffer is handled by the C library allocator. The Python memory manager is involved only in the allocation of the bytes object returned as a result.

In most situations, however, it is recommended to allocate memory from the Python heap specifically because the latter is under control of the Python memory manager. For example, this is required when the interpreter is extended with new object types written in C. Another reason for using the Python heap is the desire to *inform* the Python memory manager about the memory needs of the extension module. Even when the requested memory is used exclusively for internal, highly specific purposes, delegating all memory requests to the Python memory manager causes the interpreter to have a more accurate image of its memory footprint as a whole. Consequently, under certain circumstances, the Python memory manager may or may not trigger appropriate actions, like garbage collection, memory compaction or other preventive procedures. Note that by using the C library allocator as shown in the previous example, the allocated memory for the I/O buffer completely escapes the Python memory manager.

Ver também

The `PYTHONMALLOC` environment variable can be used to configure the memory allocators used by Python.

The `PYTHONMALLOCSTATS` environment variable can be used to print statistics of the pymalloc memory allocator every time a new pymalloc object arena is created, and on shutdown.

Allocator Domains

All allocating functions belong to one of three different "domains" (see also `PyMemAllocatorDomain`). These domains represent different allocation strategies and are optimized for different purposes. The specific details on how every domain allocates memory or what internal functions each domain calls is considered an implementation detail, but for debugging purposes a simplified table can be found at Alocadores de memória padrão. The APIs used to allocate and free a block of memory must be from the same domain. For example, `PyMem_Free()` must be used to free memory allocated using `PyMem_Malloc()`.

The three allocation domains are:

- Raw domain: intended for allocating memory for general-purpose memory buffers where the allocation *must* go to the system allocator or where the allocator can operate without an attached thread state. The memory is requested directly from the system. See Raw Memory Interface.
- "Mem" domain: intended for allocating memory for Python buffers and general-purpose memory buffers where the allocation must be performed with an attached thread state. The memory is taken from the Python private heap. See Memory Interface.
- Object domain: intended for allocating memory for Python objects. The memory is taken from the Python private heap. See Object allocators.

Nota

The free-threaded build requires that only Python objects are allocated using the "object" domain and that all Python objects are allocated using that domain. This differs from the prior

Python versions, where this was only a best practice and not a hard requirement.

For example, buffers (non-Python objects) should be allocated using `PyMem_Malloc()`, `PyMem_RawMalloc()`, or `malloc()`, but not `PyObject_Malloc()`.

See Memory Allocation APIs.

Raw Memory Interface

The following function sets are wrappers to the system allocator. These functions are thread-safe, so a thread state does not need to be attached.

The default raw memory allocator uses the following functions: `malloc()`, `calloc()`, `realloc()` and `free()`; call `malloc(1)` (or `calloc(1, 1)`) when requesting zero bytes.

Adicionado na versão 3.4.

void *PyMem_RawMalloc(size_t n)

Parte da ABI Estável desde a versão 3.13.
Allocates *n* bytes and returns a pointer of type void* to the allocated memory, or `NULL` if the request fails.

Requesting zero bytes returns a distinct non-`NULL` pointer if possible, as if `PyMem_RawMalloc(1)` had been called instead. The memory will not have been initialized in any way.

void *PyMem_RawCalloc(size_t nelem, size_t elsize)

Parte da ABI Estável desde a versão 3.13.
Allocates *nelem* elements each of size *elsize* bytes and returns a pointer of type void* to the allocated memory, or `NULL` if the request fails. The memory is initialized to zeros.

Requesting zero elements or elements of size zero bytes returns a distinct non-`NULL` pointer if possible, as if `PyMem_RawCalloc(1, 1)` had been called instead.

Adicionado na versão 3.5.

void *PyMem_RawRealloc(void *p, size_t n)

Parte da ABI Estável desde a versão 3.13.
Resizes the memory block pointed to by *p* to *n* bytes. The contents will be unchanged to the minimum of the old and the new sizes.

If *p* is `NULL`, the call is equivalent to `PyMem_RawMalloc(n)`; else if *n* is equal to zero, the memory block is resized but is not freed, and the returned pointer is non-`NULL`.

Unless *p* is `NULL`, it must have been returned by a previous call to `PyMem_RawMalloc()`, `PyMem_RawRealloc()` or `PyMem_RawCalloc()`.

If the request fails, `PyMem_RawRealloc()` returns `NULL` and *p* remains a valid pointer to the previous memory area.

void PyMem_RawFree(void *p)

Parte da ABI Estável desde a versão 3.13.

Frees the memory block pointed to by *p*, which must have been returned by a previous call to `PyMem_RawMalloc()`, `PyMem_RawRealloc()` or `PyMem_RawCalloc()`. Otherwise, or if `PyMem_RawFree(p)` has been called before, undefined behavior occurs.

If *p* is `NULL`, no operation is performed.

Interface da Memória

The following function sets, modeled after the ANSI C standard, but specifying behavior when requesting zero bytes, are available for allocating and releasing memory from the Python heap.

In the GIL-enabled build (default build) the default memory allocator uses the pymalloc memory allocator, whereas in the free-threaded build, the default is the mimalloc memory allocator instead.

Aviso

There must be an attached thread state when using these functions.

Alterado na versão 3.6: The default allocator is now pymalloc instead of system `malloc()`.

Alterado na versão 3.13: In the free-threaded build, the default allocator is now mimalloc.

void *PyMem_Malloc(size_t n)

Parte da ABI Estável.

Allocates *n* bytes and returns a pointer of type void* to the allocated memory, or `NULL` if the request fails.

Requesting zero bytes returns a distinct non-`NULL` pointer if possible, as if `PyMem_Malloc(1)` had been called instead. The memory will not have been initialized in any way.

void *PyMem_Calloc(size_t nelem, size_t elsize)

Parte da ABI Estável desde a versão 3.7.

Allocates *nelem* elements each of size *elsize* bytes and returns a pointer of type void* to the allocated memory, or `NULL` if the request fails. The memory is initialized to zeros.

Requesting zero elements or elements of size zero bytes returns a distinct non-`NULL` pointer if possible, as if `PyMem_Calloc(1, 1)` had been called instead.

Adicionado na versão 3.5.

void *PyMem_Realloc(void *p, size_t n)

Parte da ABI Estável.

Resizes the memory block pointed to by *p* to *n* bytes. The contents will be unchanged to the minimum of the old and the new sizes.

If *p* is `NULL`, the call is equivalent to `PyMem_Malloc(n)`; else if *n* is equal to zero, the memory block is resized but is not freed, and the returned pointer is non-`NULL`.

Unless *p* is `NULL`, it must have been returned by a previous call to **`PyMem_Malloc()`**, **`PyMem_Realloc()`** or **`PyMem_Calloc()`**.

If the request fails, **`PyMem_Realloc()`** returns `NULL` and *p* remains a valid pointer to the previous memory area.

void PyMem_Free(void *p)

Parte da ABI Estável.

Frees the memory block pointed to by *p*, which must have been returned by a previous call to **`PyMem_Malloc()`**, **`PyMem_Realloc()`** or **`PyMem_Calloc()`**. Otherwise, or if `PyMem_Free(p)` has been called before, undefined behavior occurs.

If *p* is `NULL`, no operation is performed.

The following type-oriented macros are provided for convenience. Note that *TYPE* refers to any C type.

PyMem_New(TYPE, n)

Same as **`PyMem_Malloc()`**, but allocates `(n * sizeof(TYPE))` bytes of memory. Returns a pointer cast to `TYPE*`. The memory will not have been initialized in any way.

PyMem_Resize(p, TYPE, n)

Same as **`PyMem_Realloc()`**, but the memory block is resized to `(n * sizeof(TYPE))` bytes. Returns a pointer cast to `TYPE*`. On return, *p* will be a pointer to the new memory area, or `NULL` in the event of failure.

This is a C preprocessor macro; *p* is always reassigned. Save the original value of *p* to avoid losing memory when handling errors.

void PyMem_Del(void *p)

Same as **`PyMem_Free()`**.

Apelidos descontinuados

Esses são apelidos suavemente descontinuados para funções e macros existentes. Eles existem unicamente para fins de retrocompatibilidade.

Apelido descontinuado	Corresponding function or macro

Apelido descontinuado	Corresponding function or macro
PyMem_MALLOC(size)	`PyMem_Malloc()`
PyMem_NEW(type, size)	`PyMem_New`
PyMem_REALLOC(ptr, size)	`PyMem_Realloc()`
PyMem_RESIZE(ptr, type, size)	`PyMem_Resize`
PyMem_FREE(ptr)	`PyMem_Free()`
PyMem_DEL(ptr)	`PyMem_Free()`

Alterado na versão 3.4: The macros are now aliases of the corresponding functions and macros. Previously, their behavior was the same, but their use did not necessarily preserve binary compatibility across Python versions.

Descontinuado desde a versão 2.0.

Alocadores de objeto

The following function sets, modeled after the ANSI C standard, but specifying behavior when requesting zero bytes, are available for allocating and releasing memory from the Python heap.

Nota

There is no guarantee that the memory returned by these allocators can be successfully cast to a Python object when intercepting the allocating functions in this domain by the methods described in the Customize Memory Allocators section.

The default object allocator uses the pymalloc memory allocator. In the free-threaded build, the default is the mimalloc memory allocator instead.

Aviso

There must be an attached thread state when using these functions.

void *PyObject_Malloc(size_t n)

Parte da ABI Estável.

Allocates *n* bytes and returns a pointer of type void* to the allocated memory, or `NULL` if the request fails.

Requesting zero bytes returns a distinct non-`NULL` pointer if possible, as if `PyObject_Malloc(1)` had been called instead. The memory will not have been initialized in any way.

void *PyObject_Calloc(size_t nelem, size_t elsize)

Parte da ABI Estável desde a versão 3.7.

Allocates *nelem* elements each of size *elsize* bytes and returns a pointer of type void* to the allocated memory, or `NULL` if the request fails. The memory is initialized to zeros.

Requesting zero elements or elements of size zero bytes returns a distinct non-`NULL` pointer if possible, as if `PyObject_Calloc(1, 1)` had been called instead.

Adicionado na versão 3.5.

void *PyObject_Realloc(void *p, size_t n)

Parte da ABI Estável.

Resizes the memory block pointed to by *p* to *n* bytes. The contents will be unchanged to the minimum of the old and the new sizes.

If *p* is `NULL`, the call is equivalent to `PyObject_Malloc(n)`; else if *n* is equal to zero, the memory block is resized but is not freed, and the returned pointer is non-`NULL`.

Unless *p* is `NULL`, it must have been returned by a previous call to **`PyObject_Malloc()`**, **`PyObject_Realloc()`** or **`PyObject_Calloc()`**.

If the request fails, **`PyObject_Realloc()`** returns `NULL` and *p* remains a valid pointer to the previous memory area.

void PyObject_Free(void *p)

Parte da ABI Estável.

Frees the memory block pointed to by *p*, which must have been returned by a previous call to **`PyObject_Malloc()`**, **`PyObject_Realloc()`** or **`PyObject_Calloc()`**. Otherwise, or if `PyObject_Free(p)` has been called before, undefined behavior occurs.

If *p* is `NULL`, no operation is performed.

Do not call this directly to free an object's memory; call the type's **`tp_free`** slot instead.

Do not use this for memory allocated by **`PyObject_GC_New`** or **`PyObject_GC_NewVar`**; use **`PyObject_GC_Del()`** instead.

Ver também

- `PyObject_GC_Del()` is the equivalent of this function for memory allocated by types that support garbage collection.
- `PyObject_Malloc()`
- `PyObject_Realloc()`
- `PyObject_Calloc()`
- `PyObject_New`
- `PyObject_NewVar`
- `PyType_GenericAlloc()`
- `tp_free`

Alocadores de memória padrão

Alocadores de memória padrão:

Configuração	Nome	PyMem_RawMalloc	PyMem_Malloc	PyObject_Malloc
Release build	`"pymalloc"`	`malloc`	`pymalloc`	`pymalloc`
Debug build	`"pymalloc_debug"`	`malloc` + debug	`pymalloc` + debug	`pymalloc` + debug
Release build, without pymalloc	`"malloc"`	`malloc`	`malloc`	`malloc`
Debug build, without pymalloc	`"malloc_debug"`	`malloc` + debug	`malloc` + debug	`malloc` + debug
Free-threaded build	`"mimalloc"`	`mimalloc`	`mimalloc`	`mimalloc`
Free-threaded debug build	`"mimalloc_debug"`	`mimalloc` + debug	`mimalloc` + debug	`mimalloc` + debug

Legenda:

- Name: value for `PYTHONMALLOC` environment variable.
- `malloc`: system allocators from the standard C library, C functions: **`malloc()`**, **`calloc()`**, **`realloc()`** and **`free()`**.

- `pymalloc`: pymalloc memory allocator.
- `mimalloc`: mimalloc memory allocator.
- "+ debug": with debug hooks on the Python memory allocators.
- "Debug build": Python build in debug mode.

Alocadores de memória

Adicionado na versão 3.4.

type PyMemAllocatorEx

Structure used to describe a memory block allocator. The structure has the following fields:

Campo	Significado
`void *ctx`	user context passed as first argument
`void* malloc(void *ctx, size_t size)`	allocate a memory block
`void* calloc(void *ctx, size_t nelem, size_t elsize)`	allocate a memory block initialized with zeros
`void* realloc(void *ctx, void *ptr, size_t new_size)`	allocate or resize a memory block
`void free(void *ctx, void *ptr)`	free a memory block

Alterado na versão 3.5: The **`PyMemAllocator`** structure was renamed to **`PyMemAllocatorEx`** and a new `calloc` field was added.

type PyMemAllocatorDomain

Enum used to identify an allocator domain. Domains:

PYMEM_DOMAIN_RAW

Funções:

- **`PyMem_RawMalloc()`**
- **`PyMem_RawRealloc()`**
- **`PyMem_RawCalloc()`**
- **`PyMem_RawFree()`**

PYMEM_DOMAIN_MEM

Funções:

- **`PyMem_Malloc()`**,
- **`PyMem_Realloc()`**
- **`PyMem_Calloc()`**
- **`PyMem_Free()`**

PYMEM_DOMAIN_OBJ

Funções:

- `PyObject_Malloc()`
- `PyObject_Realloc()`
- `PyObject_Calloc()`
- `PyObject_Free()`

void PyMem_GetAllocator(PyMemAllocatorDomain domain, PyMemAllocatorEx *allocator)

Get the memory block allocator of the specified domain.

void PyMem_SetAllocator(PyMemAllocatorDomain domain, PyMemAllocatorEx *allocator)

Set the memory block allocator of the specified domain.

The new allocator must return a distinct non-`NULL` pointer when requesting zero bytes.

For the `PYMEM_DOMAIN_RAW` domain, the allocator must be thread-safe: a thread state is not attached when the allocator is called.

For the remaining domains, the allocator must also be thread-safe: the allocator may be called in different interpreters that do not share a GIL.

If the new allocator is not a hook (does not call the previous allocator), the `PyMem_SetupDebugHooks()` function must be called to reinstall the debug hooks on top on the new allocator.

See also `PyPreConfig.allocator` and Preinitialize Python with PyPreConfig.

Aviso

`PyMem_SetAllocator()` does have the following contract:

- It can be called after `Py_PreInitialize()` and before `Py_InitializeFromConfig()` to install a custom memory allocator. There are no restrictions over the installed allocator other than the ones imposed by the domain (for instance, the Raw Domain allows the allocator to be called without an attached thread state). See the section on allocator domains for more information.
- If called after Python has finish initializing (after `Py_InitializeFromConfig()` has been called) the allocator **must** wrap the existing allocator. Substituting the current allocator for some other arbitrary one is **not supported**.

Alterado na versão 3.12: All allocators must be thread-safe.

void PyMem_SetupDebugHooks(void)

Setup debug hooks in the Python memory allocators to detect memory errors.

Debug hooks on the Python memory allocators

When Python is built in debug mode, the **PyMem_SetupDebugHooks()** function is called at the Python preinitialization to setup debug hooks on Python memory allocators to detect memory errors.

The **PYTHONMALLOC** environment variable can be used to install debug hooks on a Python compiled in release mode (ex: `PYTHONMALLOC=debug`).

The **PyMem_SetupDebugHooks()** function can be used to set debug hooks after calling **PyMem_SetAllocator()**.

These debug hooks fill dynamically allocated memory blocks with special, recognizable bit patterns. Newly allocated memory is filled with the byte `0xCD` (`PYMEM_CLEANBYTE`), freed memory is filled with the byte `0xDD` (`PYMEM_DEADBYTE`). Memory blocks are surrounded by "forbidden bytes" filled with the byte `0xFD` (`PYMEM_FORBIDDENBYTE`). Strings of these bytes are unlikely to be valid addresses, floats, or ASCII strings.

Checagens em Tempo de Execução:

- Detect API violations. For example, detect if **PyObject_Free()** is called on a memory block allocated by **PyMem_Malloc()**.
- Detect write before the start of the buffer (buffer underflow).
- Detect write after the end of the buffer (buffer overflow).
- Check that there is an attached thread state when allocator functions of **PYMEM_DOMAIN_OBJ** (ex: **PyObject_Malloc()**) and **PYMEM_DOMAIN_MEM** (ex: **PyMem_Malloc()**) domains are called.

On error, the debug hooks use the **tracemalloc** module to get the traceback where a memory block was allocated. The traceback is only displayed if **tracemalloc** is tracing Python memory allocations and the memory block was traced.

Let *S* = `sizeof(size_t)`. `2*S` bytes are added at each end of each block of *N* bytes requested. The memory layout is like so, where p represents the address returned by a malloc-like or realloc-like function (`p[i:j]` means the slice of bytes from `*(p+i)` inclusive up to `*(p+j)` exclusive; note that the treatment of negative indices differs from a Python slice):

`p[-2*S:-S]`
: Number of bytes originally asked for. This is a size_t, big-endian (easier to read in a memory dump).

`p[-S]`
: API identifier (ASCII character):

  - `'r'` for **PYMEM_DOMAIN_RAW**.
  - `'m'` for **PYMEM_DOMAIN_MEM**.
  - `'o'` for **PYMEM_DOMAIN_OBJ**.

`p[-S+1:0]`
: Copies of PYMEM_FORBIDDENBYTE. Used to catch under- writes and reads.

`p[0:N]`

The requested memory, filled with copies of PYMEM_CLEANBYTE, used to catch reference to uninitialized memory. When a realloc-like function is called requesting a larger memory block, the new excess bytes are also filled with PYMEM_CLEANBYTE. When a free-like function is called, these are overwritten with PYMEM_DEADBYTE, to catch reference to freed memory. When a realloc- like function is called requesting a smaller memory block, the excess old bytes are also filled with PYMEM_DEADBYTE.

`p[N:N+S]`
Copies of PYMEM_FORBIDDENBYTE. Used to catch over- writes and reads.

`p[N+S:N+2*S]`
Only used if the `PYMEM_DEBUG_SERIALNO` macro is defined (not defined by default).

A serial number, incremented by 1 on each call to a malloc-like or realloc-like function. Big-endian **`size_t`**. If "bad memory" is detected later, the serial number gives an excellent way to set a breakpoint on the next run, to capture the instant at which this block was passed out. The static function bumpserialno() in obmalloc.c is the only place the serial number is incremented, and exists so you can set such a breakpoint easily.

A realloc-like or free-like function first checks that the PYMEM_FORBIDDENBYTE bytes at each end are intact. If they've been altered, diagnostic output is written to stderr, and the program is aborted via Py_FatalError(). The other main failure mode is provoking a memory error when a program reads up one of the special bit patterns and tries to use it as an address. If you get in a debugger then and look at the object, you're likely to see that it's entirely filled with PYMEM_DEADBYTE (meaning freed memory is getting used) or PYMEM_CLEANBYTE (meaning uninitialized memory is getting used).

Alterado na versão 3.6: The **`PyMem_SetupDebugHooks()`** function now also works on Python compiled in release mode. On error, the debug hooks now use **`tracemalloc`** to get the traceback where a memory block was allocated. The debug hooks now also check if there is an attached thread state when functions of **`PYMEM_DOMAIN_OBJ`** and **`PYMEM_DOMAIN_MEM`** domains are called.

Alterado na versão 3.8: Byte patterns `0xCB` (`PYMEM_CLEANBYTE`), `0xDB` (`PYMEM_DEADBYTE`) and `0xFB` (`PYMEM_FORBIDDENBYTE`) have been replaced with `0xCD`, `0xDD` and `0xFD` to use the same values than Windows CRT debug `malloc()` and `free()`.

The pymalloc allocator

Python has a *pymalloc* allocator optimized for small objects (smaller or equal to 512 bytes) with a short lifetime. It uses memory mappings called "arenas" with a fixed size of either 256 KiB on 32-bit platforms or 1 MiB on 64-bit platforms. When Python is configured with **`--with-pymalloc-hugepages`**, the arena size on 64-bit platforms is increased to 2 MiB to match the huge page size, and arena allocation will attempt to use huge pages (`MAP_HUGETLB` on Linux, `MEM_LARGE_PAGES` on Windows) with automatic fallback to regular pages. It falls back to **`PyMem_RawMalloc()`** and **`PyMem_RawRealloc()`** for allocations larger than 512 bytes.

pymalloc is the default allocator of the **`PYMEM_DOMAIN_MEM`** (ex: **`PyMem_Malloc()`**) and **`PYMEM_DOMAIN_OBJ`** (ex: **`PyObject_Malloc()`**) domains.

The arena allocator uses the following functions:

- **`VirtualAlloc()`** and **`VirtualFree()`** on Windows,
- **`mmap()`** and **`munmap()`** if available,
- **`malloc()`** e **`free()`** do contrário.

This allocator is disabled if Python is configured with the **`--without-pymalloc`** option. It can also be disabled at runtime using the **`PYTHONMALLOC`** environment variable (ex: `PYTHONMALLOC=malloc`).

Typically, it makes sense to disable the pymalloc allocator when building Python with AddressSanitizer (**`--with-address-sanitizer`**) which helps uncover low level bugs within the C code.

Customize pymalloc Arena Allocator

Adicionado na versão 3.4.

type PyObjectArenaAllocator

Structure used to describe an arena allocator. The structure has three fields:

Campo	Significado
`void *ctx`	user context passed as first argument
`void* alloc(void *ctx, size_t size)`	allocate an arena of size bytes
`void free(void *ctx, void *ptr, size_t size)`	free an arena

void PyObject_GetArenaAllocator(PyObjectArenaAllocator *allocator)

Get the arena allocator.

void PyObject_SetArenaAllocator(PyObjectArenaAllocator *allocator)

Set the arena allocator.

The mimalloc allocator

Adicionado na versão 3.13.

Python supports the mimalloc [https://github.com/microsoft/mimalloc/] allocator when the underlying platform support is available. mimalloc is a general purpose allocator with excellent performance characteristics, initially developed by Daan Leijen for the runtime systems of the Koka and Lean languages.

Unlike pymalloc, which is optimized for small objects (512 bytes or fewer), mimalloc handles allocations of any size.

In the free-threaded build, mimalloc is the default and **required** allocator for the `PYMEM_DOMAIN_MEM` and `PYMEM_DOMAIN_OBJ` domains. It cannot be disabled in free-threaded builds. The free-threaded build uses per-thread mimalloc heaps, which allows allocation and deallocation to proceed without locking in most cases.

In the default (non-free-threaded) build, mimalloc is available but not the default allocator. It can be selected at runtime using `PYTHONMALLOC`=`mimalloc` (or `mimalloc_debug` to include debug hooks). It can be disabled at build time using the `--without-mimalloc` configure option, but this option cannot be combined with `--disable-gil`.

tracemalloc C API

Adicionado na versão 3.7.

int PyTraceMalloc_Track(unsigned int domain, uintptr_t ptr, size_t size)

Track an allocated memory block in the `tracemalloc` module.

Return `0` on success, return `-1` on error (failed to allocate memory to store the trace). Return `-2` if tracemalloc is disabled.

If memory block is already tracked, update the existing trace.

int PyTraceMalloc_Untrack(unsigned int domain, uintptr_t ptr)

Untrack an allocated memory block in the `tracemalloc` module. Do nothing if the block was not tracked.

Return `-2` if tracemalloc is disabled, otherwise return `0`.

Exemplos

Here is the example from section Visão Geral, rewritten so that the I/O buffer is allocated from the Python heap by using the first function set:

```
PyObject *res;
char *buf = (char *) PyMem_Malloc(BUFSIZ); /* for I/O */

if (buf == NULL)
    return PyErr_NoMemory();
/* ...Do some I/O operation involving buf... */
res = PyBytes_FromString(buf);
PyMem_Free(buf); /* allocated with PyMem_Malloc */
return res;
```

The same code using the type-oriented function set:

```
PyObject *res;
char *buf = PyMem_New(char, BUFSIZ); /* for I/O */

if (buf == NULL)
```

```
    return PyErr_NoMemory();
/* ...Do some I/O operation involving buf... */
res = PyBytes_FromString(buf);
PyMem_Free(buf); /* allocated with PyMem_New */
return res;
```

Note that in the two examples above, the buffer is always manipulated via functions belonging to the same set. Indeed, it is required to use the same memory API family for a given memory block, so that the risk of mixing different allocators is reduced to a minimum. The following code sequence contains two errors, one of which is labeled as *fatal* because it mixes two different allocators operating on different heaps.

```
char *buf1 = PyMem_New(char, BUFSIZ);
char *buf2 = (char *) malloc(BUFSIZ);
char *buf3 = (char *) PyMem_Malloc(BUFSIZ);
...
PyMem_Del(buf3);  /* Wrong -- should be PyMem_Free() */
free(buf2);       /* Right -- allocated via malloc() */
free(buf1);       /* Fatal -- should be PyMem_Free()  */
```

In addition to the functions aimed at handling raw memory blocks from the Python heap, objects in Python are allocated and released with **PyObject_New**, **PyObject_NewVar** and **PyObject_Free()**.

These will be explained in the next chapter on defining and implementing new object types in C.

Suporte a implementação de Objetos

Este capítulo descreve as funções, tipos e macros usados ao definir novos tipos de objeto.

- Alocando objetos na heap
  - Apelidos suavemente descontinuados
- Ciclo de vida do objeto
  - Eventos de vida
  - Destruição de isolado cíclico
  - Funções
- Estruturas comuns de objetos
  - Macros e tipos de objetos base
  - Implementing functions and methods
  - Accessing attributes of extension types
    - Member flags
    - Member types
    - Defining Getters and Setters
- Estruturas de objetos tipo
  - Referências rápidas
    - “slots tp”
    - sub-slots
    - slot typedefs
  - PyTypeObject Definition
  - PyObject Slots
  - PyVarObject Slots
  - PyTypeObject Slots
  - Static Types
  - Tipos no heap
  - Number Object Structures
  - Mapping Object Structures
  - Sequence Object Structures
  - Buffer Object Structures

    - Async Object Structures
    - Slot Type typedefs
    - Exemplos
- Suporte a Coleta Cíclica de Lixo
    - Traversal
        - Traversal-safe functions
        - “DuringGC” functions
    - Controlando o estado do coletor de lixo
    - Querying Garbage Collector State

Alocando objetos na heap

PyObject *_PyObject_New(PyTypeObject *type)

Retorna valor: Nova referência.

PyVarObject *_PyObject_NewVar(PyTypeObject *type, Py_ssize_t size)

Retorna valor: Nova referência.

PyObject *PyObject_Init(PyObject *op, PyTypeObject *type)

Retorna valor: Referência emprestada. Parte da ABI Estável.

Inicializa um objeto *op* recém-alocado com seu tipo e referência inicial. Retorna o objeto inicializado. Outros campos do objeto não são inicializados. Apesar do nome, esta função não tem relação com o método `__init__()` do objeto (slot `tp_init`). Especificamente, esta função **não** chama o método `__init__()` do objeto.

Em geral, considere esta função como uma rotina de baixo nível. Use `tp_alloc` sempre que possível. Para implementar `tp_alloc` para o seu tipo, prefira `PyType_GenericAlloc()` ou `PyObject_New()`.

Nota

Esta função inicializa apenas a memória do objeto correspondente à estrutura inicial `PyObject`. Não zera o restante.

PyVarObject *PyObject_InitVar(PyVarObject *op, PyTypeObject *type, Py_ssize_t size)

Retorna valor: Referência emprestada. Parte da ABI Estável.

Isto faz tudo que o `PyObject_Init()` faz e também inicializa a informação de comprimento para um objeto de tamanho variável.

Nota

Esta função inicializa apenas parte da memória do objeto. Não zera o restante.

PyObject_New(TYPE, typeobj)

Aloca um novo objeto Python usando o tipo de estrutura C *TYPE* e o objeto de tipo Python *typeobj* (`PyTypeObject*`) chamando `PyObject_Malloc()` para alocar memória e inicializando-o como `PyObject_Init()`. O chamador possuirá a única referência ao objeto (ou seja, sua contagem de referências será um).

Evite chamar isso diretamente para alocar memória para um objeto; em vez disso, chame o slot `tp_alloc` do tipo.

Ao preencher o slot `tp_alloc` de um tipo, `PyType_GenericAlloc()` é preferível a uma função personalizada que simplesmente chama esta macro.

Esta macro não chama `tp_alloc`, `tp_new` (`__new__()`) ou `tp_init` (`__init__()`).

Isso não pode ser usado para objetos com `Py_TPFLAGS_HAVE_GC` definido em `tp_flags`; use `PyObject_GC_New` em vez disso.

A memória alocada por esta macro deve ser liberada com `PyObject_Free()` (geralmente chamado por meio do slot `tp_free` do objeto).

Nota

Não há garantia de que a memória retornada tenha sido completamente zerada antes de ser inicializada.

Nota

Esta macro não constrói um objeto totalmente inicializado do tipo fornecido; ela apenas aloca memória e a prepara para inicialização

posterior por `tp_init`. Para construir um objeto totalmente inicializado, chame *typeobj*. Por exemplo:

```
PyObject *foo = PyObject_CallNoArgs((PyObject *)&PyFoo_Type
```

Ver também

- `PyObject_Free()`
- `PyObject_GC_New`
- `PyType_GenericAlloc()`
- `tp_alloc`

PyObject_NewVar(TYPE, typeobj, size)

Como `PyObject_New`, exceto que:

- Ela aloca memória suficiente para a estrutura *TYPE* mais os *size* (`Py_ssize_t`) campos do tamanho fornecido pelo campo `tp_itemsize` de *typeobj*.
- A memória é inicializada como `PyObject_InitVar()`.

Isso é útil para implementar objetos como tuplas, que conseguem determinar seu tamanho em tempo de construção. Incorporar o vetor de campos na mesma alocação diminui o número de alocações, melhorando a eficiência do gerenciamento de memória.

Evite chamar isso diretamente para alocar memória para um objeto; em vez disso, chame o slot `tp_alloc` do tipo.

Ao preencher o slot `tp_alloc` de um tipo, `PyType_GenericAlloc()` é preferível a uma função personalizada que simplesmente chama esta macro.

Isso não pode ser usado para objetos com `Py_TPFLAGS_HAVE_GC` definido em `tp_flags`; use `PyObject_GC_NewVar` em vez disso.

A memória alocada por esta função deve ser liberada com `PyObject_Free()` (geralmente chamado por meio do slot `tp_free` do

objeto).

Nota

Não há garantia de que a memória retornada tenha sido completamente zerada antes de ser inicializada.

Nota

Esta macro não constrói um objeto totalmente inicializado do tipo fornecido; ela apenas aloca memória e a prepara para inicialização posterior por `tp_init`. Para construir um objeto totalmente inicializado, chame *typeobj*. Por exemplo:

```
PyObject *list_instance = PyObject_CallNoArgs((PyObject *)&
```

Ver também

- `PyObject_Free()`
- `PyObject_GC_NewVar`
- `PyType_GenericAlloc()`
- `tp_alloc`

PyObject _Py_NoneStruct

Objeto o qual é visível no Python como `None`. Isto só deve ser acessado usando a macro `Py_None`, o qual avalia como um ponteiro para este objeto.

Ver também

Objetos do Módulo

Para alocar e criar módulos de extensão.

Apelidos suavemente descontinuados

Suavemente descontinuado desde a versão 3.15.

Esses são apelidos para funções e macros existentes. Eles existem unicamente para fins de retrocompatibilidade.

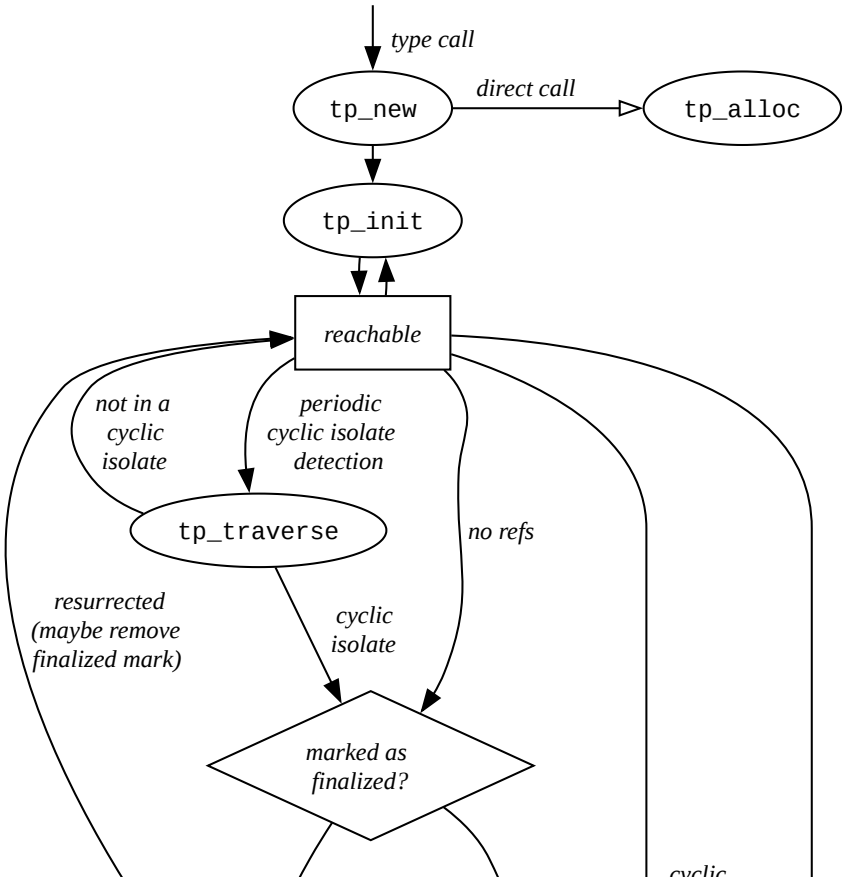
Apelido suavemente descontinuado	Função
PyObject_NEW(type, typeobj)	`PyObject_New`
PyObject_NEW_VAR(type, typeobj, n)	`PyObject_NewVar`
PyObject_INIT(op, typeobj)	`PyObject_Init()`
PyObject_INIT_VAR(op, typeobj, n)	`PyObject_InitVar()`
PyObject_MALLOC(n)	`PyObject_Malloc()`
PyObject_REALLOC(p, n)	`PyObject_Realloc()`
PyObject_FREE(p)	`PyObject_Free()`
PyObject_DEL(p)	`PyObject_Free()`
PyObject_Del(p)	`PyObject_Free()`

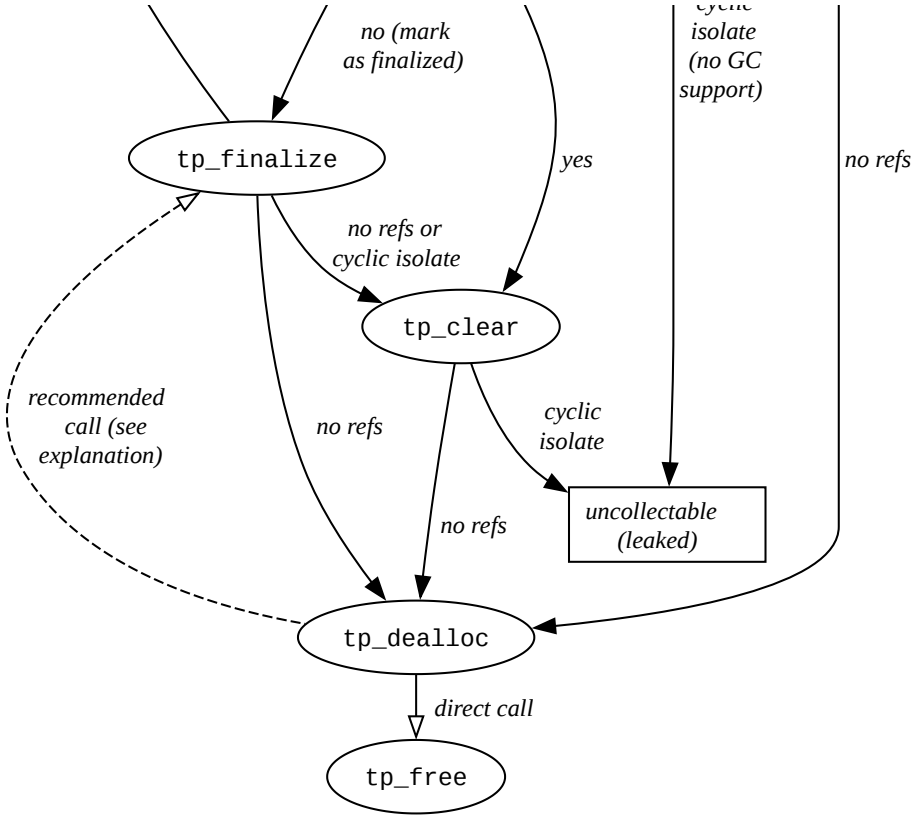
Ciclo de vida do objeto

Esta seção explica como os slots de um tipo se relacionam entre si ao longo da vida de um objeto. Não se destina a ser uma referência canônica completa para os slots; em vez disso, consulte a documentação específica do slot em Estruturas de objetos tipo para obter detalhes sobre um slot específico.

Eventos de vida

A figura abaixo ilustra a ordem dos eventos que podem ocorrer ao longo da vida de um objeto. Uma seta de *A* para *B* indica que o evento *B* pode ocorrer após a ocorrência do evento *A*, com o rótulo da seta indicando a condição que deve ser verdadeira para que *B* ocorra após *A*.





Explicação:

- Quando um novo objeto é construído chamando seu tipo:
  1. `tp_new` é chamado para criar um novo objeto.
  2. `tp_alloc` é chamado diretamente por `tp_new` para alocar a memória para o novo objeto.
  3. `tp_init` inicializa o objeto recém-criado. **`tp_init`** pode ser chamado novamente para reinicializar um objeto, se desejado. A chamada DE **`tp_init`** também pode ser completamente ignorada, por exemplo, com código Python chamando `__new__()`.
- Após a conclusão de **`tp_init`**, o objeto estará pronto para uso.
- Algum tempo após a última referência a um objeto ser removida:
  1. Se um objeto não estiver marcado como *finalizado*, ele poderá ser finalizado marcando-o como *finalizado* e chamando sua função `tp_finalize`. O Python *não* finaliza um objeto quando a última referência a ele é excluída; use `PyObject_CallFinalizerFromDealloc()` para garantir que **`tp_finalize`** seja sempre chamado.

2. Se o objeto estiver marcado como finalizado, `tp_clear` poderá ser chamado pelo coletor de lixo para limpar as referências mantidas pelo objeto. Ele *não* é chamado quando a contagem de referências do objeto chega a zero.
3. `tp_dealloc` é chamado para destruir o objeto. Para evitar duplicação de código, **`tp_dealloc`** normalmente chama `tp_clear` para liberar as referências do objeto.
4. Quando `tp_dealloc` termina a destruição do objeto, ele chama diretamente `tp_free` (geralmente definido como `PyObject_Free()` ou `PyObject_GC_Del()` automaticamente, conforme apropriado para o tipo) para desalocar a memória.

- A função `tp_finalize` tem permissão para adicionar uma referência ao objeto, se desejado. Se isso acontecer, o objeto será *ressuscitado*, impedindo sua destruição pendente. (Somente **`tp_finalize`** tem permissão para ressuscitar um objeto; `tp_clear` e `tp_dealloc` não podem sem chamar **`tp_finalize`**.) Ressuscitar um objeto pode ou não causar a remoção da marca *finalizado* do objeto. Atualmente, o Python não remove a marca *finalizado* de um objeto ressuscitado se ele suportar coleta de lixo (ou seja, o sinalizador `Py_TPFLAGS_HAVE_GC` estiver definido), mas remove a marca se o objeto não suportar coleta de lixo; qualquer um ou ambos os comportamentos podem mudar no futuro.
- `tp_dealloc` pode opcionalmente chamar `tp_finalize` via `PyObject_CallFinalizerFromDealloc()` se desejar reutilizar esse código para auxiliar na destruição de objetos. Isso é recomendado porque garante que **`tp_finalize`** seja sempre chamado antes da destruição. Consulte a documentação de **`tp_dealloc`** para obter um exemplo de código.
- Se o objeto for membro de um isolado cíclico e `tp_clear` não conseguir interromper o ciclo de referência ou o isolado cíclico não for detectado (talvez `gc.disable()` tenha sido chamado ou o sinalizador `Py_TPFLAGS_HAVE_GC` tenha sido omitido erroneamente em um dos tipos envolvidos), os objetos permanecerão indefinidamente não coletáveis (eles “vazam”). Veja `gc.garbage`.

Se o objeto for marcado como compatível com coleta de lixo (o sinalizador `Py_TPFLAGS_HAVE_GC` estiver definido em `tp_flags`), os seguintes eventos

também serão possíveis:

- O coletor de lixo ocasionalmente chama `tp_traverse` para identificar isolados cíclicos.
- Quando o coletor de lixo descobre um isolado cíclico, ele finaliza um dos objetos do grupo marcando-o como *finalizado* e chamando sua função `tp_finalize`, se houver. Isso se repete até que o isolado cíclico não exista mais ou todos os objetos tenham sido finalizados.
- `tp_finalize` tem permissão para ressuscitar o objeto adicionando uma referência externa ao isolado cíclico. A nova referência faz com que o grupo de objetos não forme mais um isolado cíclico (o ciclo de referência ainda pode existir, mas se existir, os objetos não estarão mais isolados).
- Quando o coletor de lixo descobre um isolado cíclico e todos os objetos do grupo já foram marcados como *finalizados*, o coletor de lixo limpa um ou mais objetos não limpos no grupo (possivelmente simultaneamente) chamando a função `tp_clear` de cada um. Isso se repete enquanto o isolado cíclico ainda existir e nem todos os objetos tiverem sido limpos.

Destruição de isolado cíclico

Abaixo estão listados os estágios de vida de um isolado cíclico hipotético que continua a existir após cada objeto membro ser finalizado ou limpo. É um vazamento de memória se um isolado cíclico passar por todos esses estágios; ele deve desaparecer assim que todos os objetos forem limpos, ou até mesmo antes. Um isolado cíclico pode desaparecer porque o ciclo de referência foi quebrado ou porque os objetos não estão mais isolados devido à ressurreição do finalizador (veja `tp_finalize`).

0. **Alcançável** (ainda não é um isolado cíclico): todos os objetos estão em seu estado normal e acessível. Um ciclo de referência pode existir, mas uma referência externa significa que os objetos ainda não estão isolados.
1. **Inalcançável, mas consistente:** a referência final de fora do grupo cíclico de objetos foi removida, causando o isolado dos objetos (criando assim um isolado cíclico). Nenhum dos objetos do grupo foi

finalizado ou limpo ainda. O isolado cíclico permanece neste estágio até alguma execução futura do coletor de lixo (não necessariamente a próxima execução, pois a próxima execução pode não varrer todos os objetos).

2. **Mistura de finalizados e não finalizados:** objetos em um isolado cíclico são finalizados um de cada vez, o que significa que há um período em que o isolado cíclico é composto por uma mistura de objetos finalizados e não finalizados. A ordem de finalização não é especificada, portanto, pode parecer aleatória. Um objeto finalizado deve se comportar de maneira sensata quando objetos não finalizados interagem com ele, e um objeto não finalizado deve ser capaz de tolerar a finalização de um subconjunto arbitrário de seus referentes.
3. **Todos finalizados:** todos os objetos em um isolado cíclico são finalizados antes que qualquer um deles seja limpo.
4. **Combinação de finalizado e limpo:** os objetos podem ser limpos em série ou simultaneamente (mas com a GIL mantida); de qualquer forma, alguns serão concluídos antes de outros. Um objeto finalizado deve ser capaz de tolerar a limpeza de um subconjunto de seus referentes. **PEP 442** [https://peps.python.org/pep-0442/] chama essa etapa de "lixo cíclico".
5. **Vazamento:** se um isolado cíclico ainda existir após todos os objetos do grupo terem sido finalizados e limpos, os objetos permanecerão indefinidamente não coletáveis (consulte `gc.garbage`). É um bug se um isolado cíclico atingir esse estágio — significa que os métodos `tp_clear` dos objetos participantes falharam em interromper o ciclo de referência conforme necessário.

Se `tp_clear` não existisse, o Python não teria como interromper com segurança um ciclo de referência. A simples destruição de um objeto em um isolado cíclico resultaria em um ponteiro pendente, desencadeando um comportamento indefinido quando um objeto que referencia o objeto destruído é destruído. A etapa de limpeza torna a destruição de objetos um processo de duas fases: primeiro, `tp_clear` é chamado para destruir parcialmente os objetos o suficiente para desvinculá-los uns dos outros; em seguida, `tp_dealloc` é chamado para completar a destruição.

Ao contrário da limpeza, a finalização não é uma fase da destruição. Um objeto finalizado ainda deve se comportar corretamente, continuando a cumprir seus contratos de design. O finalizador de um objeto pode executar código Python arbitrário e até mesmo impedir a destruição iminente adicionando uma referência. O finalizador está relacionado à destruição apenas pela ordem de chamada — se for executado, será executado antes da destruição, que começa com `tp_clear` (se chamado) e termina com `tp_dealloc`.

A etapa de finalização não é necessária para recuperar com segurança os objetos em um isolado cíclico, mas sua existência facilita o design de tipos que se comportam de maneira sensata quando os objetos são limpos. Limpar um objeto pode necessariamente deixá-lo em um estado quebrado, parcialmente destruído — pode ser inseguro chamar qualquer um dos métodos do objeto limpo ou acessar qualquer um de seus atributos. Com a finalização, apenas objetos finalizados podem interagir com objetos limpos; objetos não finalizados têm a garantia de interagir apenas com objetos não limpos (mas potencialmente finalizados).

Para resumir as interações possíveis:

- Um objeto não finalizado pode ter referências a ou de objetos não finalizados e finalizados, mas não a ou de objetos limpos.
- Um objeto finalizado pode ter referências a ou de objetos não finalizados, finalizados e limpos.
- Um objeto limpo pode ter referências a ou de objetos finalizados e limpos, mas não a ou de objetos não finalizados.

Sem ciclos de referência, um objeto pode ser simplesmente destruído após a exclusão de sua última referência; as etapas de finalização e limpeza não são necessárias para recuperar objetos não utilizados com segurança. No entanto, pode ser útil chamar automaticamente `tp_finalize` e `tp_clear` antes da destruição, pois o design de tipos é simplificado quando todos os objetos sempre experimentam a mesma série de eventos, independentemente de terem participado ou não de um isolado cíclico. Atualmente, o Python chama **`tp_finalize`** e **`tp_clear`** apenas conforme necessário para destruir um isolado cíclico; isso pode mudar em uma versão futura.

Funções

Para alocar e liberar memória, consulte Alocando objetos na heap.

void PyObject_CallFinalizer(PyObject *op)

Finaliza o objeto conforme descrito em `tp_finalize`. Chame esta função (ou `PyObject_CallFinalizerFromDealloc()`) em vez de chamar `tp_finalize` diretamente, pois esta função pode desduplicar várias chamadas para `tp_finalize`. Atualmente, as chamadas são desduplicadas somente se o tipo oferecer suporte a coleta de lixo (ou seja, se o sinalizador `Py_TPFLAGS_HAVE_GC` estiver definido); isso pode mudar no futuro.

Adicionado na versão 3.4.

int PyObject_CallFinalizerFromDealloc(PyObject *op)

Parte da ABI Estável desde a versão 3.15.

O mesmo que `PyObject_CallFinalizer()`, mas deve ser chamado no início do destrutor do objeto (`tp_dealloc`). Não deve haver nenhuma referência ao objeto. Se o finalizador do objeto ressuscitar o objeto, esta função retornará -1; nenhuma outra destruição deverá ocorrer. Caso contrário, esta função retornará 0 e a destruição pode continuar normalmente.

Adicionado na versão 3.4.

Ver também

`tp_dealloc` para código de exemplo.

Estruturas comuns de objetos

Existe um grande número de estruturas usadas para a definição de tipos objeto para o Python. Esta seção descreve essas estruturas e como são usadas.

Macros e tipos de objetos base

Todos os objetos Python por fim compartilham um pequeno número de campos no começo da representação o objeto na memória. Esses são representados pelos tipos **`PyObject`** e **`PyVarObject`**, que são definidos, por sua vez, pelas expansões de alguns macros também, utilizados, direta ou indiretamente, na definição de todos outros objetos Python. Macros adicionais podem ser encontrados em contagem de referências.

type PyObject

Parte da ABI Estável (see below).

Todos os tipos de objeto são extensões deste tipo. Este é um tipo que contém as informações necessárias para que o Python trate um ponteiro para um objeto como tal. Em uma versão de "lançamento" normal, ele contém apenas a contagem de referências do objeto e um ponteiro para o objeto do tipo correspondente. Nada é declarado como um **`PyObject`**, mas todo ponteiro para um objeto Python pode ser convertido para um PyObject*.

Os membros não devem ser acessados diretamente; em vez disso, use macros como **`Py_REFCNT`** e **`Py_TYPE`**.

In the Stable ABI for Free-Threaded Builds (`abi3t`), this struct is opaque; its size and layout may change between Python versions. In Stable ABI for non-free-threaded builds (`abi3`), the **`ob_refcnt`** and **`ob_type`** fields are available, but using them directly is discouraged.

Py_ssize_t ob_refcnt

Parte da ABI Estável.
A contagem de referências do objeto, conforme retornada por **Py_REFCNT**. Não use este campo diretamente; em vez disso, use funções e macros como **Py_REFCNT**, **Py_INCREF()** e **Py_DecRef()**.

O tipo de campo pode ser diferente de `Py_ssize_t`, dependendo da configuração e da plataforma de construção.

PyTypeObject *ob_type
Parte da ABI Estável.
O tipo do objeto. Não use este campo diretamente; use **Py_TYPE** e **Py_SET_TYPE()** em vez disso.

PyMutex ob_mutex
A per-object lock, present only in the free-threaded build (when **Py_GIL_DISABLED** is defined).

This field is **reserved for use by the critical section API** (**Py_BEGIN_CRITICAL_SECTION** / **Py_END_CRITICAL_SECTION**). Do **not** lock it directly with `PyMutex_Lock`; doing so can cause deadlocks. If you need your own lock, add a separate **PyMutex** field to your object struct.

Adicionado na versão 3.13.

type PyVarObject
Parte da ABI Estável (see below).
Uma extensão de **PyObject** que adiciona o campo **ob_size**. Isso se destina a objetos que possuem alguma noção de *comprimento*.

Assim como com **PyObject**, os membros não devem ser acessados diretamente; em vez disso, use macros como **Py_SIZE**, **Py_REFCNT** e **Py_TYPE**.

In the Stable ABI for Free-Threaded Builds (`abi3t`), this struct is opaque; its size and layout may change between Python versions. In Stable ABI for non-free-threaded builds (`abi3`), the **ob_base** and **ob_size** fields are available, but using them directly is discouraged.

PyObject ob_base

Parte da ABI Estável.

Common object header. Typically, this field is not accessed directly; instead **`PyVarObject`** can be cast to **`PyObject`**.

Py_ssize_t ob_size

Parte da ABI Estável.

Um campo de tamanho, cujo conteúdo deve ser considerado um detalhe de implementação interna do objeto.

Não utilize este campo diretamente; utilize **`Py_SIZE`** em vez disso.

Funções de criação de objetos, como **`PyObject_NewVar()`**, geralmente definem este campo com o tamanho solicitado (número de itens). Após a criação, valores arbitrários podem ser armazenados em **`ob_size`** usando **`Py_SET_SIZE`**.

Para obter o comprimento publicamente exposto de um objeto, conforme retornado pela função Python **`len()`**, use **`PyObject_Length()`** em vez disso.

PyObject_HEAD

Este é um macro usado ao declarar novos tipos que representam objetos sem comprimento variável. O macro PyObject_HEAD se expande para:

```
PyObject ob_base;
```

Veja documentação de **`PyObject`** acima.

PyObject_VAR_HEAD

Esta é uma macro usada ao declarar novos tipos que representam objetos com um comprimento que varia de instância para instância. A macro PyObject_VAR_HEAD se expande para:

```
PyVarObject ob_base;
```

Veja documentação de **`PyVarObject`** acima.

PyTypeObject PyBaseObject_Type

Parte da ABI Estável.
A classe base de todos os outros objetos, equivalente a **`object`** em Python.

int Py_Is(PyObject *x, PyObject *y)

Parte da ABI Estável desde a versão 3.10.
Testa se o objeto *x* é o objeto *y*, o mesmo que `x is y` em Python.

Adicionado na versão 3.10.

int Py_IsNone(PyObject *x)

Parte da ABI Estável desde a versão 3.10.
Test if an object is the `None` singleton, the same as `x is None` in Python.

Adicionado na versão 3.10.

int Py_IsTrue(PyObject *x)

Parte da ABI Estável desde a versão 3.10.
Test if an object is the `True` singleton, the same as `x is True` in Python.

Adicionado na versão 3.10.

int Py_IsFalse(PyObject *x)

Parte da ABI Estável desde a versão 3.10.
Test if an object is the `False` singleton, the same as `x is False` in Python.

Adicionado na versão 3.10.

PyTypeObject *Py_TYPE(PyObject *o)

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.14.
Get the type of the Python object *o*.

The returned reference is borrowed from *o*. Do not release it with **`Py_DECREF()`** or similar.

Alterado na versão 3.11: `Py_TYPE()` is changed to an inline static function. The parameter type is no longer const PyObject*.

int Py_IS_TYPE(PyObject *o, PyTypeObject *type)

Parte da ABI Estável desde a versão 3.15.
Return non-zero if the object *o* type is *type*. Return zero otherwise. Equivalent to: `Py_TYPE(o) == type`.

Adicionado na versão 3.9.

void Py_SET_TYPE(PyObject *o, PyTypeObject *type)

Set the type of object *o* to *type*, without any checking or reference counting.

This is a very low-level operation. Consider instead setting the Python attribute `__class__` using `PyObject_SetAttrString()` or similar.

Note that assigning an incompatible type can lead to undefined behavior.

If *type* is a heap type, the caller must create a new reference to it. Similarly, if the old type of *o* is a heap type, the caller must release a reference to that type.

Adicionado na versão 3.9.

Py_ssize_t Py_SIZE(PyVarObject *o)

Parte da ABI Estável desde a versão 3.15.
Get the `ob_size` field of *o*.

Alterado na versão 3.11: `Py_SIZE()` is changed to an inline static function. The parameter type is no longer const PyVarObject*.

void Py_SET_SIZE(PyVarObject *o, Py_ssize_t size)

Parte da ABI Estável desde a versão 3.15.
Set the `ob_size` field of *o* to *size*.

Adicionado na versão 3.9.

PyObject_HEAD_INIT(type)

This is a macro which expands to initialization values for a new **PyObject** type. This macro expands to:

```
_PyObject_EXTRA_INIT
1, type,
```

PyVarObject_HEAD_INIT(type, size)

This is a macro which expands to initialization values for a new **PyVarObject** type, including the **ob_size** field. This macro expands to:

```
_PyObject_EXTRA_INIT
1, type, size,
```

Implementing functions and methods

type PyCFunction

Parte da ABI Estável.

Type of the functions used to implement most Python callables in C. Functions of this type take two PyObject* parameters and return one such value. If the return value is `NULL`, an exception shall have been set. If not `NULL`, the return value is interpreted as the return value of the function as exposed in Python. The function must return a new reference.

A assinatura da função é:

```
PyObject *PyCFunction(PyObject *self,
                      PyObject *args);
```

type PyCFunctionWithKeywords

Parte da ABI Estável.

Type of the functions used to implement Python callables in C with signature METH_VARARGS | METH_KEYWORDS. The function signature is:

```
PyObject *PyCFunctionWithKeywords(PyObject *self,
                                  PyObject *args,
                                  PyObject *kwargs);
```

type PyCFunctionFast

Parte da ABI Estável desde a versão 3.13.

Type of the functions used to implement Python callables in C with signature `METH_FASTCALL`. The function signature is:

```
PyObject *PyCFunctionFast(PyObject *self,
                          PyObject *const *args,
                          Py_ssize_t nargs);
```

type PyCFunctionFastWithKeywords

Parte da ABI Estável desde a versão 3.13.

Type of the functions used to implement Python callables in C with signature METH_FASTCALL | METH_KEYWORDS. The function signature is:

```
PyObject *PyCFunctionFastWithKeywords(PyObject *self,
                                      PyObject *const *args,
                                      Py_ssize_t nargs,
                                      PyObject *kwnames);
```

type PyCMethod

Type of the functions used to implement Python callables in C with signature METH_METHOD | METH_FASTCALL | METH_KEYWORDS. The function signature is:

```
PyObject *PyCMethod(PyObject *self,
                    PyTypeObject *defining_class,
                    PyObject *const *args,
                    Py_ssize_t nargs,
                    PyObject *kwnames)
```

Adicionado na versão 3.9.

type PyMethodDef

Parte da ABI Estável (incluindo todos os membros).

Structure used to describe a method of an extension type. This structure has four fields:

const char *ml_name
: Nome do método.

  A `NULL` *ml_name* marks the end of a **`PyMethodDef`** array.

PyCFunction ml_meth
: Pointer to the C implementation.

int ml_flags
: Flags bits indicating how the call should be constructed.

const char *ml_doc
: Points to the contents of the docstring.

The **`ml_meth`** is a C function pointer. The functions may be of different types, but they always return PyObject*. If the function is not of the **`PyCFunction`**, the compiler will require a cast in the method table. Even though **`PyCFunction`** defines the first parameter as PyObject*, it is common that the method implementation uses the specific C type of the *self* object.

The **`ml_flags`** field is a bitfield which can include the following flags. The individual flags indicate either a calling convention or a binding convention.

There are these calling conventions:

METH_VARARGS
: *Parte da ABI Estável.*
  This is the typical calling convention, where the methods have the type **`PyCFunction`**. The function expects two PyObject* values. The first one is the *self* object for methods; for module functions, it is the module object. The second parameter (often called *args*) is a tuple object representing all arguments. This parameter is typically processed using **`PyArg_ParseTuple()`** or **`PyArg_UnpackTuple()`**.

METH_KEYWORDS

Can only be used in certain combinations with other flags: METH_VARARGS | METH_KEYWORDS, METH_FASTCALL | METH_KEYWORDS and METH_METHOD | METH_FASTCALL | METH_KEYWORDS.

METH_VARARGS | METH_KEYWORDS

Methods with these flags must be of type **PyCFunctionWithKeywords**. The function expects three parameters: *self*, *args*, *kwargs* where *kwargs* is a dictionary of all the keyword arguments or possibly `NULL` if there are no keyword arguments. The parameters are typically processed using **PyArg_ParseTupleAndKeywords()**.

METH_FASTCALL

Parte da ABI Estável desde a versão 3.10.

Fast calling convention supporting only positional arguments. The methods have the type **PyCFunctionFast**. The first parameter is *self*, the second parameter is a C array of PyObject* values indicating the arguments and the third parameter is the number of arguments (the length of the array).

Adicionado na versão 3.7.

Alterado na versão 3.10: `METH_FASTCALL` is now part of the stable ABI.

METH_FASTCALL | METH_KEYWORDS

Extension of **METH_FASTCALL** supporting also keyword arguments, with methods of type **PyCFunctionFastWithKeywords**. Keyword arguments are passed the same way as in the vectorcall protocol: there is an additional fourth PyObject* parameter which is a tuple representing the names of the keyword arguments (which are guaranteed to be strings) or possibly `NULL` if there are no keywords. The values of the keyword arguments are stored in the *args* array, after the positional arguments.

Adicionado na versão 3.7.

METH_METHOD

Parte da ABI Estável desde a versão 3.7.
Can only be used in the combination with other flags: METH_METHOD | METH_FASTCALL | METH_KEYWORDS.

METH_METHOD | METH_FASTCALL | METH_KEYWORDS

Extension of METH_FASTCALL | METH_KEYWORDS supporting the *defining class*, that is, the class that contains the method in question. The defining class might be a superclass of `Py_TYPE(self)`.

The method needs to be of type **`PyCMethod`**, the same as for `METH_FASTCALL | METH_KEYWORDS` with `defining_class` argument added after `self`.

Adicionado na versão 3.9.

METH_NOARGS

Parte da ABI Estável.
Methods without parameters don't need to check whether arguments are given if they are listed with the **`METH_NOARGS`** flag. They need to be of type **`PyCFunction`**. The first parameter is typically named *self* and will hold a reference to the module or object instance. In all cases the second parameter will be `NULL`.

The function must have 2 parameters. Since the second parameter is unused, **`Py_UNUSED`** can be used to prevent a compiler warning.

METH_O

Parte da ABI Estável.
Methods with a single object argument can be listed with the **`METH_O`** flag, instead of invoking **`PyArg_ParseTuple()`** with a `"O"` argument. They have the type **`PyCFunction`**, with the *self* parameter, and a PyObject* parameter representing the single argument.

These two constants are not used to indicate the calling convention but the binding when used with methods of classes. These may not be used for functions defined for modules. At most one of these flags may be set for any given method.

METH_CLASS

Parte da ABI Estável.

The method will be passed the type object as the first parameter rather than an instance of the type. This is used to create *class methods*, similar to what is created when using the `@classmethod` built-in decorator.

METH_STATIC

Parte da ABI Estável.

The method will be passed `NULL` as the first parameter rather than an instance of the type. This is used to create *static methods*, similar to what is created when using the `@staticmethod` built-in decorator.

One other constant controls whether a method is loaded in place of another definition with the same method name.

METH_COEXIST

Parte da ABI Estável.

The method will be loaded in place of existing definitions. Without *METH_COEXIST*, the default is to skip repeated definitions. Since slot wrappers are loaded before the method table, the existence of a *sq_contains* slot, for example, would generate a wrapped method named `__contains__()` and preclude the loading of a corresponding PyCFunction with the same name. With the flag defined, the PyCFunction will be loaded in place of the wrapper object and will co-exist with the slot. This is helpful because calls to PyCFunctions are optimized more than wrapper object calls.

PyTypeObject PyCMethod_Type

The type object corresponding to Python C method objects. This is available as `types.BuiltinMethodType` in the Python layer.

int PyCMethod_Check(PyObject *op)

Return true if *op* is an instance of the `PyCMethod_Type` type or a subtype of it. This function always succeeds.

int PyCMethod_CheckExact(PyObject *op)

This is the same as **PyCMethod_Check()**, but does not account for subtypes.

PyObject *PyCMethod_New(PyMethodDef *ml, PyObject *self, PyObject *module, PyTypeObject *cls)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.9.

Turn *ml* into a Python callable object. The caller must ensure that *ml* outlives the callable. Typically, *ml* is defined as a static variable.

The *self* parameter will be passed as the *self* argument to the C function in `ml->ml_meth` when invoked. *self* can be `NULL`.

The callable object's `__module__` attribute can be set from the given *module* argument. *module* should be a Python string, which will be used as name of the module the function is defined in. If unavailable, it can be set to **None** or `NULL`.

Ver também

function.__module__

The *cls* parameter will be passed as the *defining_class* argument to the C function. Must be set if **METH_METHOD** is set on `ml->ml_flags`.

Adicionado na versão 3.9.

PyTypeObject PyCFunction_Type

Parte da ABI Estável.

The type object corresponding to Python C function objects. This is available as **types.BuiltinFunctionType** in the Python layer.

int PyCFunction_Check(PyObject *op)

Return true if *op* is an instance of the **PyCFunction_Type** type or a subtype of it. This function always succeeds.

int PyCFunction_CheckExact(PyObject *op)

This is the same as **`PyCFunction_Check()`**, but does not account for subtypes.

PyObject *PyCFunction_NewEx(PyMethodDef *ml, PyObject *self, PyObject *module)

Retorna valor: Nova referência. Parte da ABI Estável.

Equivalente a `PyCMethod_New(ml, self, module, NULL)`.

PyObject *PyCFunction_New(PyMethodDef *ml, PyObject *self)

Retorna valor: Nova referência. Parte da ABI Estável desde a versão 3.4.

Equivalente a `PyCMethod_New(ml, self, NULL, NULL)`.

int PyCFunction_GetFlags(PyObject *func)

Parte da ABI Estável.

Get the function's flags on *func* as they were passed to **`ml_flags`**.

If *func* is not a C function object, this fails with an exception. *func* must not be `NULL`.

This function returns the function's flags on success, and `-1` with an exception set on failure.

int PyCFunction_GET_FLAGS(PyObject *func)

This is the same as **`PyCFunction_GetFlags()`**, but without error or type checking.

PyCFunction PyCFunction_GetFunction(PyObject *func)

Parte da ABI Estável.

Get the function pointer on *func* as it was passed to **`ml_meth`**.

If *func* is not a C function object, this fails with an exception. *func* must not be `NULL`.

This function returns the function pointer on success, and `NULL` with an exception set on failure.

int PyCFunction_GET_FUNCTION(PyObject *func)

This is the same as **PyCFunction_GetFunction()**, but without error or type checking.

PyObject *PyCFunction_GetSelf(PyObject *func)

Parte da ABI Estável.
Get the "self" object on *func*. This is the object that would be passed to the first argument of a **PyCFunction**. For C function objects created through a **PyMethodDef** on a **PyModuleDef**, this is the resulting module object.

If *func* is not a C function object, this fails with an exception. *func* must not be `NULL`.

This function returns a borrowed reference to the "self" object on success, and `NULL` with an exception set on failure.

PyObject *PyCFunction_GET_SELF(PyObject *func)

This is the same as **PyCFunction_GetSelf()**, but without error or type checking.

Accessing attributes of extension types

type PyMemberDef

Parte da ABI Estável (incluindo todos os membros).
Structure which describes an attribute of a type which corresponds to a C struct member. When defining a class, put a NULL-terminated array of these structures in the **tp_members** slot.

Its fields are, in order:

const char *name

Name of the member. A NULL value marks the end of a `PyMemberDef[]` array.

The string should be static, no copy is made of it.

int type

The type of the member in the C struct. See Member types for the possible values.

Py_ssize_t offset

The offset in bytes that the member is located on the type's object struct.

int flags

Zero or more of the Member flags, combined using bitwise OR.

const char *doc

The docstring, or NULL. The string should be static, no copy is made of it. Typically, it is defined using **PyDoc_STR**.

By default (when **flags** is `0`), members allow both read and write access. Use the **Py_READONLY** flag for read-only access. Certain types, like **Py_T_STRING**, imply **Py_READONLY**. Only **Py_T_OBJECT_EX** (and legacy **T_OBJECT**) members can be deleted.

For heap-allocated types (created using **PyType_FromSpec()** or similar), `PyMemberDef` may contain a definition for the special member `"__vectorcalloffset__"`, corresponding to **tp_vectorcall_offset** in type objects. This member must be defined with `Py_T_PYSSIZET`, and either `Py_READONLY` or `Py_READONLY | Py_RELATIVE_OFFSET`. For example:

```
static PyMemberDef spam_type_members[] = {
    {"__vectorcalloffset__", Py_T_PYSSIZET,
     offsetof(Spam_object, vectorcall), Py_READONLY},
    {NULL}  /* Sentinel */
};
```

(You may need to `#include <stddef.h>` for **offsetof()**.)

The legacy offsets **tp_dictoffset** and **tp_weaklistoffset** can be defined similarly using `"__dictoffset__"` and `"__weaklistoffset__"`

members, but extensions are strongly encouraged to use **Py_TPFLAGS_MANAGED_DICT** and **Py_TPFLAGS_MANAGED_WEAKREF** instead.

Alterado na versão 3.12: `PyMemberDef` is always available. Previously, it required including `"structmember.h"`.

Alterado na versão 3.14: **Py_RELATIVE_OFFSET** is now allowed for `"__vectorcalloffset__"`, `"__dictoffset__"` and `"__weaklistoffset__"`.

PyObject *PyMember_GetOne(const char *obj_addr, struct PyMemberDef *m)

Parte da ABI Estável.
Get an attribute belonging to the object at address *obj_addr*. The attribute is described by `PyMemberDef` *m*. Returns `NULL` on error.

Alterado na versão 3.12: `PyMember_GetOne` is always available. Previously, it required including `"structmember.h"`.

int PyMember_SetOne(char *obj_addr, struct PyMemberDef *m, PyObject *o)

Parte da ABI Estável.
Set an attribute belonging to the object at address *obj_addr* to object *o*. The attribute to set is described by `PyMemberDef` *m*. Returns `0` if successful and a negative value on failure.

Alterado na versão 3.12: `PyMember_SetOne` is always available. Previously, it required including `"structmember.h"`.

Member flags

The following flags can be used with **PyMemberDef.flags**:

Py_READONLY

Parte da ABI Estável desde a versão 3.12.
Not writable.

Py_AUDIT_READ

Parte da ABI Estável desde a versão 3.12.

Emit an `object.__getattr__` audit event before reading.

Py_RELATIVE_OFFSET

Parte da ABI Estável desde a versão 3.12.

Indicates that the **`offset`** of this `PyMemberDef` entry indicates an offset from the subclass-specific data, rather than from `PyObject`.

Can only be used as part of the **`Py_tp_members`** **`slot`** when creating a class using negative **`basicsize`**. It is mandatory in that case. When setting **`tp_members`** from the slot during class creation, Python clears the flag and sets **`PyMemberDef.offset`** to the offset from the `PyObject` struct.

Alterado na versão 3.10: The **`RESTRICTED`**, **`READ_RESTRICTED`** and **`WRITE_RESTRICTED`** macros available with `#include "structmember.h"` are deprecated. **`READ_RESTRICTED`** and **`RESTRICTED`** are equivalent to **`Py_AUDIT_READ`**; **`WRITE_RESTRICTED`** does nothing.

Alterado na versão 3.12: The **`READONLY`** macro was renamed to **`Py_READONLY`**. The **`PY_AUDIT_READ`** macro was renamed with the `Py_` prefix. The new names are now always available. Previously, these required `#include "structmember.h"`. The header is still available and it provides the old names.

Member types

`PyMemberDef.type` can be one of the following macros corresponding to various C types. When the member is accessed in Python, it will be converted to the equivalent Python type. When it is set from Python, it will be converted back to the C type. If that is not possible, an exception such as **`TypeError`** or **`ValueError`** is raised.

Unless marked (D), attributes defined this way cannot be deleted using e.g. **`del`** or **`delattr()`**.

Macro name	Tipo em C	Tipo em Python
Py_T_BYTE *Parte da ABI Estável desde a versão 3.12.*	char	`int`
Py_T_SHORT *Parte da ABI Estável desde a versão 3.12.*	short	`int`
Py_T_INT *Parte da ABI Estável desde a versão 3.12.*	int	`int`
Py_T_LONG *Parte da ABI Estável desde a versão 3.12.*	long	`int`
Py_T_LONGLONG *Parte da ABI Estável desde a versão 3.12.*	long long	`int`
Py_T_UBYTE *Parte da ABI Estável desde a versão 3.12.*	unsigned char	`int`
Py_T_UINT *Parte da ABI Estável desde a versão 3.12.*	unsigned int	`int`

Macro name	Tipo em C	Tipo em Python
Py_T_USHORT *Parte da ABI Estável desde a versão 3.12.*	unsigned short	`int`
Py_T_ULONG *Parte da ABI Estável desde a versão 3.12.*	unsigned long	`int`
Py_T_ULONGLONG *Parte da ABI Estável desde a versão 3.12.*	unsigned long long	`int`
Py_T_PYSSIZET *Parte da ABI Estável desde a versão 3.12.*	Py_ssize_t	`int`
Py_T_FLOAT *Parte da ABI Estável desde a versão 3.12.*	float	`float`
Py_T_DOUBLE *Parte da ABI Estável desde a versão 3.12.*	double	`float`
Py_T_BOOL *Parte da ABI Estável desde a versão 3.12.*	char (escrito como 0 ou 1)	`bool`

Macro name	Tipo em C	Tipo em Python
Py_T_STRING *Parte da ABI Estável desde a versão 3.12.*	const char* (*)	`str` (RO)
Py_T_STRING_INPLACE *Parte da ABI Estável desde a versão 3.12.*	const char[] (*)	`str` (RO)
Py_T_CHAR *Parte da ABI Estável desde a versão 3.12.*	char (0-127)	`str` (**)
Py_T_OBJECT_EX *Parte da ABI Estável desde a versão 3.12.*	PyObject*	`object` (D)

(*): Zero-terminated, UTF8-encoded C string. With **`Py_T_STRING`** the C representation is a pointer; with **`Py_T_STRING_INPLACE`** the string is stored directly in the structure.

(**): String of length 1. Only ASCII is accepted.

(RO): implica **`Py_READONLY`**.

(D): pode ser deletado, neste caso o ponteiro é definido para `NULL`. Ler um ponteiro `NULL` levanta uma exceção **`AttributeError`**.

Adicionado na versão 3.12: In previous versions, the macros were only available with `#include "structmember.h"` and were named without the

`Py_` prefix (e.g. as `T_INT`). The header is still available and contains the old names, along with the following deprecated types:

T_OBJECT

Like `Py_T_OBJECT_EX`, but `NULL` is converted to `None`. This results in surprising behavior in Python: deleting the attribute effectively sets it to `None`.

T_NONE

Sempre `None`. Deve ser usado com **`Py_READONLY`**.

Defining Getters and Setters

type PyGetSetDef

Parte da ABI Estável (incluindo todos os membros).
Structure to define property-like access for a type. See also description of the **`PyTypeObject.tp_getset`** slot.

const char *name

attribute name

getter get

C function to get the attribute.

setter set

Optional C function to set or delete the attribute. If `NULL`, the attribute is read-only.

const char *doc

optional docstring

void *closure

Optional user data pointer, providing additional data for getter and setter.

typedef PyObject *(*getter)(PyObject*, void*)

Parte da ABI Estável.

The `get` function takes one PyObject* parameter (the instance) and a user data pointer (the associated `closure`):

It should return a new reference on success or `NULL` with a set exception on failure.

typedef int (*setter)(PyObject*, PyObject*, void*)

Parte da ABI Estável.

`set` functions take two PyObject* parameters (the instance and the value to be set) and a user data pointer (the associated `closure`):

In case the attribute should be deleted the second parameter is `NULL`. Should return `0` on success or `-1` with a set exception on failure.

Estruturas de objetos tipo

Talvez uma das estruturas mais importantes do sistema de objetos Python seja a estrutura que define um novo tipo: a estrutura `PyTypeObject`. Objetos tipo podem ser manipulados usando qualquer uma das funções `PyObject_*` ou `PyType_*`, mas não oferecem muito de interessante para a maioria das aplicações Python. Esses objetos são fundamentais para o comportamento dos objetos, portanto, são muito importantes para o próprio interpretador e para qualquer módulo de extensão que implemente novos tipos.

Os objetos tipo são bastante grandes em comparação com a maioria dos tipos padrão. A razão para o tamanho é que cada objeto de tipo armazena um grande número de valores, principalmente indicadores de função C, cada um dos quais implementa uma pequena parte da funcionalidade do tipo. Os campos do objeto de tipo são examinados em detalhes nesta seção. Os campos serão descritos na ordem em que ocorrem na estrutura.

Além da referência rápida a seguir, a seção Exemplos fornece uma visão geral do significado e uso de `PyTypeObject`.

Referências rápidas

“slots tp”

Slot de PyTypeObject [[1]]	**Tipo**	**métodos/atributos especiais**	**Info [[2]]**			
			O	**T**	**D**	**I**
<R> `tp_name`	const char *	__name__	X	X		
`tp_basicsize`	`Py_ssize_t`		X	X		X

Slot de PyTypeObject [[1]]	Tipo	métodos/atributos especiais	Info [[2]]			
			O	T	D	I
tp_itemsize	Py_ssize_t			X		X
tp_dealloc	destructor		X	X		X
tp_vectorcall_offset	Py_ssize_t			X		X
(tp_getattr)	getattrfunc	__getattribute__, __getattr__				G
(tp_setattr)	setattrfunc	__setattr__, __delattr__				G
tp_as_async	PyAsyncMethods *	sub-slots				%
tp_repr	reprfunc	__repr__	X	X		X
tp_as_number	PyNumberMethods *	sub-slots				%
tp_as_sequence	PySequenceMethods *	sub-slots				%
tp_as_mapping	PyMappingMethods *	sub-slots				%

Slot de PyTypeObject [[1]]	Tipo	métodos/atributos especiais	Info [[2]]			
			O	T	D	I
tp_hash	hashfunc	__hash__	X			G
tp_call	ternaryfunc	__call__		X		X
tp_str	reprfunc	__str__	X			X
tp_getattro	getattrofunc	__getattribute__, __getattr__	X	X		G
tp_setattro	setattrofunc	__setattr__, __delattr__	X	X		G
tp_as_buffer	PyBufferProcs *	sub-slots				%
tp_flags	unsigned long		X	X		?
tp_doc	const char *	__doc__	X	X		
tp_traverse	traverseproc			X		G
tp_clear	inquiry			X		G

Slot de PyTypeObject [[1]]	Tipo	métodos/atributos especiais	Info [[2]]			
			O	T	D	I
tp_richcompare	richcmpfunc	__lt__, __le__, __eq__, __ne__, __gt__, __ge__	X			G
(tp_weaklistoffset)	Py_ssize_t			X		?
tp_iter	getiterfunc	__iter__				X
tp_iternext	iternextfunc	__next__				X
tp_methods	PyMethodDef []		X	X		
tp_members	PyMemberDef []			X		
tp_getset	PyGetSetDef []		X	X		
tp_base	PyTypeObject *	__base__			X	
tp_dict	PyObject *	__dict__			?	
tp_descr_get	descrgetfunc	__get__				X
tp_descr_set	descrsetfunc	__set__, __delete__				X

Slot de PyTypeObject [[1]]	Tipo	métodos/atributos especiais	Info [[2]]			
			O	T	D	I
(tp_dictoffset)	Py_ssize_t			X		?
tp_init	initproc	__init__	X	X		X
tp_alloc	allocfunc		X		?	?
tp_new	newfunc	__new__	X	X	?	?
tp_free	freefunc		X	X	?	?
tp_is_gc	inquiry			X		X
<tp_bases>	PyObject *	__bases__			~	
<tp_mro>	PyObject *	__mro__			~	
[tp_cache]	PyObject *					
[tp_subclasses]	void *	__subclasses__				
[tp_weaklist]	PyObject *					
(tp_del)	destructor					

Slot de PyTypeObject [[1]]	Tipo	métodos/atributos especiais	Info [[2]]			
			O	T	D	I
[`tp_version_tag`]	unsigned int					
`tp_finalize`	`destructor`	__del__				X
`tp_vectorcall`	`vectorcallfunc`					
[`tp_watched`]	unsigned char					

[1]

(): Um nome de slot entre parênteses indica que está (efetivamente) descontinuado.

<>: Names in angle brackets should be initially set to `NULL` and treated as read-only.

[]: Names in square brackets are for internal use only.

<**R**> (as a prefix) means the field is required (must be non-`NULL`).

[2]

Columns:

"O": set on `PyBaseObject_Type`

"T": set on `PyType_Type`

"D": default (if slot is set to `NULL`)

```
X - PyType_Ready sets this value if it is NULL
~ - PyType_Ready always sets this value (it should be NULL)
```

```
? - PyType_Ready may set this value depending on other slots

Also see the inheritance column ("I").
```

“I”: inheritance

```
X - type slot is inherited via *PyType_Ready* if defined with a *NULL
% - the slots of the sub-struct are inherited individually
G - inherited, but only in combination with other slots; see the slot
? - it's complicated; see the slot's description
```

Note that some slots are effectively inherited through the normal attribute lookup chain.

sub-slots

Slot	Tipo	special methods
`am_await`	`unaryfunc`	__await__
`am_aiter`	`unaryfunc`	__aiter__
`am_anext`	`unaryfunc`	__anext__
`am_send`	`sendfunc`	
`nb_add`	`binaryfunc`	__add__ __radd__
`nb_inplace_add`	`binaryfunc`	__iadd__
`nb_subtract`	`binaryfunc`	__sub__ __rsub__

Slot	Tipo	special methods
`nb_inplace_subtract`	`binaryfunc`	__isub__
`nb_multiply`	`binaryfunc`	__mul__ __rmul__
`nb_inplace_multiply`	`binaryfunc`	__imul__
`nb_remainder`	`binaryfunc`	__mod__ __rmod__
`nb_inplace_remainder`	`binaryfunc`	__imod__
`nb_divmod`	`binaryfunc`	__divmod__ __rdivmod__
`nb_power`	`ternaryfunc`	__pow__ __rpow__
`nb_inplace_power`	`ternaryfunc`	__ipow__
`nb_negative`	`unaryfunc`	__neg__
`nb_positive`	`unaryfunc`	__pos__
`nb_absolute`	`unaryfunc`	__abs__
`nb_bool`	`inquiry`	__bool__

Slot	Tipo	special methods
nb_invert	unaryfunc	__invert__
nb_lshift	binaryfunc	__lshift__ __rlshift__
nb_inplace_lshift	binaryfunc	__ilshift__
nb_rshift	binaryfunc	__rshift__ __rrshift__
nb_inplace_rshift	binaryfunc	__irshift__
nb_and	binaryfunc	__and__ __rand__
nb_inplace_and	binaryfunc	__iand__
nb_xor	binaryfunc	__xor__ __rxor__
nb_inplace_xor	binaryfunc	__ixor__
nb_or	binaryfunc	__or__ __ror__
nb_inplace_or	binaryfunc	__ior__

Slot	Tipo	special methods
nb_int	unaryfunc	__int__
nb_reserved	void *	
nb_float	unaryfunc	__float__
nb_floor_divide	binaryfunc	__floordiv__
nb_inplace_floor_divide	binaryfunc	__ifloordiv__
nb_true_divide	binaryfunc	__truediv__
nb_inplace_true_divide	binaryfunc	__itruediv__
nb_index	unaryfunc	__index__
nb_matrix_multiply	binaryfunc	__matmul__ __rmatmul__
nb_inplace_matrix_multiply	binaryfunc	__imatmul__
mp_length	lenfunc	__len__
mp_subscript	binaryfunc	__getitem__

Slot	Tipo	special methods
mp_ass_subscript	objobjargproc	__setitem__, __delitem__
sq_length	lenfunc	__len__
sq_concat	binaryfunc	__add__
sq_repeat	ssizeargfunc	__mul__
sq_item	ssizeargfunc	__getitem__
sq_ass_item	ssizeobjargproc	__setitem__ __delitem__
sq_contains	objobjproc	__contains__
sq_inplace_concat	binaryfunc	__iadd__
sq_inplace_repeat	ssizeargfunc	__imul__
bf_getbuffer	getbufferproc()	__buffer__
bf_releasebuffer	releasebufferproc()	__release_buffer__

slot typedefs

typedef	Parameter Types	Return Type
allocfunc	PyTypeObject * Py_ssize_t	PyObject *
destructor	PyObject *	void
freefunc	void *	void
traverseproc	PyObject * visitproc void *	int
newfunc	PyTypeObject * PyObject * PyObject *	PyObject *
initproc	PyObject * PyObject * PyObject *	int
reprfunc	PyObject *	PyObject *
getattrfunc	PyObject * const char *	PyObject *
setattrfunc	PyObject * const char * PyObject *	int

typedef	Parameter Types	Return Type
getattrofunc	PyObject * PyObject *	PyObject *
setattrofunc	PyObject * PyObject * PyObject *	int
descrgetfunc	PyObject * PyObject * PyObject *	PyObject *
descrsetfunc	PyObject * PyObject * PyObject *	int
hashfunc	PyObject *	Py_hash_t
richcmpfunc	PyObject * PyObject * int	PyObject *
getiterfunc	PyObject *	PyObject *
iternextfunc	PyObject *	PyObject *
lenfunc	PyObject *	Py_ssize_t

typedef	Parameter Types	Return Type
getbufferproc	PyObject * Py_buffer * int	int
releasebufferproc	PyObject * Py_buffer *	void
inquiry	PyObject *	int
unaryfunc	PyObject *	PyObject *
binaryfunc	PyObject * PyObject *	PyObject *
ternaryfunc	PyObject * PyObject * PyObject *	PyObject *
ssizeargfunc	PyObject * Py_ssize_t	PyObject *
ssizeobjargproc	PyObject * Py_ssize_t PyObject *	int
objobjproc	PyObject * PyObject *	int

typedef	Parameter Types	Return Type
objobjargproc	PyObject * PyObject * PyObject *	int

See Slot Type typedefs below for more detail.

PyTypeObject Definition

The structure definition for PyTypeObject can be found in `Include/cpython/object.h`. For convenience of reference, this repeats the definition found there:

```
typedef struct _typeobject {
    PyObject_VAR_HEAD
    const char *tp_name; /* For printing, in format "<module>.<name>"
    Py_ssize_t tp_basicsize, tp_itemsize; /* For allocation */

    /* Methods to implement standard operations */

    destructor tp_dealloc;
    Py_ssize_t tp_vectorcall_offset;
    getattrfunc tp_getattr;
    setattrfunc tp_setattr;
    PyAsyncMethods *tp_as_async; /* formerly known as tp_compare (Pyt
                                    or tp_reserved (Python 3) */
    reprfunc tp_repr;

    /* Method suites for standard classes */

    PyNumberMethods *tp_as_number;
    PySequenceMethods *tp_as_sequence;
    PyMappingMethods *tp_as_mapping;

    /* More standard operations (here for binary compatibility) */

    hashfunc tp_hash;
    ternaryfunc tp_call;
    reprfunc tp_str;
    getattrofunc tp_getattro;
```

```
setattrofunc tp_setattro;

/* Functions to access object as input/output buffer */
PyBufferProcs *tp_as_buffer;

/* Flags to define presence of optional/expanded features */
unsigned long tp_flags;

const char *tp_doc; /* Documentation string */

/* Assigned meaning in release 2.0 */
/* call function for all accessible objects */
traverseproc tp_traverse;

/* delete references to contained objects */
inquiry tp_clear;

/* Assigned meaning in release 2.1 */
/* rich comparisons */
richcmpfunc tp_richcompare;

/* weak reference enabler */
Py_ssize_t tp_weaklistoffset;

/* Iterators */
getiterfunc tp_iter;
iternextfunc tp_iternext;

/* Attribute descriptor and subclassing stuff */
PyMethodDef *tp_methods;
PyMemberDef *tp_members;
PyGetSetDef *tp_getset;
// Strong reference on a heap type, borrowed reference on a stati
PyTypeObject *tp_base;
PyObject *tp_dict;
descrgetfunc tp_descr_get;
descrsetfunc tp_descr_set;
Py_ssize_t tp_dictoffset;
initproc tp_init;
allocfunc tp_alloc;
newfunc tp_new;
freefunc tp_free; /* Low-level free-memory routine */
inquiry tp_is_gc; /* For PyObject_IS_GC */
PyObject *tp_bases;
PyObject *tp_mro; /* method resolution order */
PyObject *tp_cache; /* no longer used */
```

```
    void *tp_subclasses;  /* for static builtin types this is an inde
    PyObject *tp_weaklist; /* not used for static builtin types */
    destructor tp_del;

    /* Type attribute cache version tag. Added in version 2.6.
     * If zero, the cache is invalid and must be initialized.
     */
    unsigned int tp_version_tag;

    destructor tp_finalize;
    vectorcallfunc tp_vectorcall;

    /* bitset of which type-watchers care about this type */
    unsigned char tp_watched;

    /* Number of tp_version_tag values used.
     * Set to _Py_ATTR_CACHE_UNUSED if the attribute cache is
     * disabled for this type (e.g. due to custom MRO entries).
     * Otherwise, limited to MAX_VERSIONS_PER_CLASS (defined elsewher
     */
    uint16_t tp_versions_used;
} PyTypeObject;
```

PyObject Slots

The type object structure extends the **PyVarObject** structure. The **ob_size** field is used for dynamic types (created by **type_new()**, usually called from a class statement). Note that **PyType_Type** (the metatype) initializes **tp_itemsize**, which means that its instances (i.e. type objects) *must* have the **ob_size** field.

PyObject.ob_refcnt

The type object's reference count is initialized to `1` by the `PyObject_HEAD_INIT` macro. Note that for statically allocated type objects, the type's instances (objects whose **ob_type** points back to the type) do *not* count as references. But for dynamically allocated type objects, the instances *do* count as references.

Inheritance:

This field is not inherited by subtypes.

PyObject.ob_type

This is the type's type, in other words its metatype. It is initialized by the argument to the `PyObject_HEAD_INIT` macro, and its value should normally be `&PyType_Type`. However, for dynamically loadable extension modules that must be usable on Windows (at least), the compiler complains that this is not a valid initializer. Therefore, the convention is to pass `NULL` to the `PyObject_HEAD_INIT` macro and to initialize this field explicitly at the start of the module's initialization function, before doing anything else. This is typically done like this:

```
Foo_Type.ob_type = &PyType_Type;
```

This should be done before any instances of the type are created. **PyType_Ready()** checks if **ob_type** is `NULL`, and if so, initializes it to the **ob_type** field of the base class. **PyType_Ready()** will not change this field if it is non-zero.

Inheritance:

This field is inherited by subtypes.

PyVarObject Slots

PyVarObject.ob_size

For statically allocated type objects, this should be initialized to zero. For dynamically allocated type objects, this field has a special internal meaning.

This field should be accessed using the **Py_SIZE()** macro.

Inheritance:

This field is not inherited by subtypes.

PyTypeObject Slots

Each slot has a section describing inheritance. If **PyType_Ready()** may set a value when the field is set to `NULL` then there will also be a "Default" section. (Note that

many fields set on `PyBaseObject_Type` and `PyType_Type` effectively act as defaults.)

const char *PyTypeObject.tp_name

See `Py_tp_name` for the corresponding `Slot ID`.

Pointer to a NUL-terminated string containing the name of the type. For types that are accessible as module globals, the string should be the full module name, followed by a dot, followed by the type name; for built-in types, it should be just the type name. If the module is a submodule of a package, the full package name is part of the full module name. For example, a type named **T** defined in module **M** in subpackage **Q** in package **P** should have the `tp_name` initializer `"P.Q.M.T"`.

For dynamically allocated type objects, this should just be the type name, and the module name explicitly stored in the type dict as the value for key `'__module__'`.

For statically allocated type objects, the *tp_name* field should contain a dot. Everything before the last dot is made accessible as the `__module__` attribute, and everything after the last dot is made accessible as the `__name__` attribute.

If no dot is present, the entire `tp_name` field is made accessible as the `__name__` attribute, and the `__module__` attribute is undefined (unless explicitly set in the dictionary, as explained above). This means your type will be impossible to pickle. Additionally, it will not be listed in module documentations created with pydoc.

This field must not be `NULL`. It is the only required field in `PyTypeObject()` (other than potentially `tp_itemsize`).

Inheritance:

This field is not inherited by subtypes.

Py_ssize_t PyTypeObject.tp_basicsize
Py_ssize_t PyTypeObject.tp_itemsize

These fields allow calculating the size in bytes of instances of the type.

See `Py_tp_basicsize`, `Py_tp_extra_basicsize` and `Py_tp_itemsize` for the corresponding `Slot IDs`.

There are two kinds of types: types with fixed-length instances have a zero **tp_itemsize** field, types with variable-length instances have a non-zero **tp_itemsize** field. For a type with fixed-length instances, all instances have the same size, given in **tp_basicsize**. (Exceptions to this rule can be made using PyUnstable_Object_GC_NewWithExtraData().)

For a type with variable-length instances, the instances must have an ob_size field, and the instance size is **tp_basicsize** plus N times **tp_itemsize**, where N is the "length" of the object.

Functions like PyObject_NewVar() will take the value of N as an argument, and store in the instance's ob_size field. Note that the **ob_size** field may later be used for other purposes. For example, int instances use the bits of **ob_size** in an implementation-defined way; the underlying storage and its size should be accessed using PyLong_Export().

Nota

The ob_size field should be accessed using the Py_SIZE() and Py_SET_SIZE() macros.

Also, the presence of an ob_size field in the instance layout doesn't mean that the instance structure is variable-length. For example, the list type has fixed-length instances, yet those instances have a **ob_size** field. (As with int, avoid reading lists' **ob_size** directly. Call PyList_Size() instead.)

The **tp_basicsize** includes size needed for data of the type's tp_base, plus any extra data needed by each instance.

The correct way to set **tp_basicsize** is to use the `sizeof` operator on the struct used to declare the instance layout. This struct must include the struct used to declare the base type. In other words, **tp_basicsize** must be greater than or equal to the base's **tp_basicsize**.

Since every type is a subtype of object, this struct must include PyObject or PyVarObject (depending on whether ob_size should be included). These are usually defined by the macro PyObject_HEAD or PyObject_VAR_HEAD, respectively.

The basic size does not include the GC header size, as that header is not part of PyObject_HEAD.

For cases where struct used to declare the base type is unknown, see **PyType_Spec.basicsize** and **PyType_FromMetaclass()**.

Notes about alignment:

- **tp_basicsize** must be a multiple of `_Alignof(PyObject)`. When using `sizeof` on a `struct` that includes **PyObject_HEAD**, as recommended, the compiler ensures this. When not using a C `struct`, or when using compiler extensions like `__attribute__((packed))`, it is up to you.
- If the variable items require a particular alignment, **tp_basicsize** and **tp_itemsize** must each be a multiple of that alignment. For example, if a type's variable part stores a `double`, it is your responsibility that both fields are a multiple of `_Alignof(double)`.

Inheritance:

These fields are inherited separately by subtypes. (That is, if the field is set to zero, **PyType_Ready()** will copy the value from the base type, indicating that the instances do not need additional storage.)

If the base type has a non-zero **tp_itemsize**, it is generally not safe to set **tp_itemsize** to a different non-zero value in a subtype (though this depends on the implementation of the base type).

destructor PyTypeObject.tp_dealloc

The corresponding slot ID `Py_tp_dealloc` is part of the ABI Estável.

A pointer to the instance destructor function. The function signature is:

```
void tp_dealloc(PyObject *self);
```

The destructor function should remove all references which the instance owns (e.g., call **Py_CLEAR()**), free all memory buffers owned by the instance, and call the type's **tp_free** function to free the object itself.

If you may call functions that may set the error indicator, you must use **PyErr_GetRaisedException()** and **PyErr_SetRaisedException()** to ensure you don't clobber a preexisting error indicator (the deallocation could have occurred while processing a different error):

```
static void
foo_dealloc(foo_object *self)
```

```
{
    PyObject *et, *ev, *etb;
    PyObject *exc = PyErr_GetRaisedException();
    ...
    PyErr_SetRaisedException(exc);
}
```

The dealloc handler itself must not raise an exception; if it hits an error case it should call `PyErr_FormatUnraisable()` to log (and clear) an unraisable exception.

No guarantees are made about when an object is destroyed, except:

- Python will destroy an object immediately or some time after the final reference to the object is deleted, unless its finalizer (`tp_finalize`) subsequently resurrects the object.
- An object will not be destroyed while it is being automatically finalized (`tp_finalize`) or automatically cleared (`tp_clear`).

CPython currently destroys an object immediately from `Py_DECREF()` when the new reference count is zero, but this may change in a future version.

It is recommended to call `PyObject_CallFinalizerFromDealloc()` at the beginning of **`tp_dealloc`** to guarantee that the object is always finalized before destruction.

If the type supports garbage collection (the `Py_TPFLAGS_HAVE_GC` flag is set), the destructor should call `PyObject_GC_UnTrack()` before clearing any member fields.

It is permissible to call `tp_clear` from **`tp_dealloc`** to reduce code duplication and to guarantee that the object is always cleared before destruction. Beware that **`tp_clear`** might have already been called.

If the type is heap allocated (`Py_TPFLAGS_HEAPTYPE`), the deallocator should release the owned reference to its type object (via `Py_DECREF()`) after calling the type deallocator. See the example code below.:

```
static void
foo_dealloc(PyObject *op)
{
   foo_object *self = (foo_object *) op;
   PyObject_GC_UnTrack(self);
```

```
   Py_CLEAR(self->ref);
   Py_TYPE(self)->tp_free(self);
}
```

tp_dealloc must leave the exception status unchanged. If it needs to call something that might raise an exception, the exception state must be backed up first and restored later (after logging any exceptions with **PyErr_WriteUnraisable()**).

Exemplo:

```
static void
foo_dealloc(PyObject *self)
{
    PyObject *exc = PyErr_GetRaisedException();

    if (PyObject_CallFinalizerFromDealloc(self) < 0) {
        // self was resurrected.
        goto done;
    }

    PyTypeObject *tp = Py_TYPE(self);

    if (tp->tp_flags & Py_TPFLAGS_HAVE_GC) {
        PyObject_GC_UnTrack(self);
    }

    // Optional, but convenient to avoid code duplication.
    if (tp->tp_clear && tp->tp_clear(self) < 0) {
        PyErr_WriteUnraisable(self);
    }

    // Any additional destruction goes here.

    tp->tp_free(self);
    self = NULL;  // In case PyErr_WriteUnraisable() is called bel

    if (tp->tp_flags & Py_TPFLAGS_HEAPTYPE) {
        Py_CLEAR(tp);
    }

done:
    // Optional, if something was called that might have raised ar
    // exception.
    if (PyErr_Occurred()) {
        PyErr_WriteUnraisable(self);
```

```
    }
    PyErr_SetRaisedException(exc);
}
```

tp_dealloc may be called from any Python thread, not just the thread which created the object (if the object becomes part of a refcount cycle, that cycle might be collected by a garbage collection on any thread). This is not a problem for Python API calls, since the thread on which **tp_dealloc** is called with an attached thread state. However, if the object being destroyed in turn destroys objects from some other C library, care should be taken to ensure that destroying those objects on the thread which called **tp_dealloc** will not violate any assumptions of the library.

Inheritance:

This field is inherited by subtypes.

Ver também

Ciclo de vida do objeto for details about how this slot relates to other slots.

Py_ssize_t PyTypeObject.tp_vectorcall_offset

An optional offset to a per-instance function that implements calling the object using the vectorcall protocol, a more efficient alternative of the simpler **tp_call**.

This field is only used if the flag **Py_TPFLAGS_HAVE_VECTORCALL** is set. If so, this must be a positive integer containing the offset in the instance of a **vectorcallfunc** pointer.

The *vectorcallfunc* pointer may be `NULL`, in which case the instance behaves as if **Py_TPFLAGS_HAVE_VECTORCALL** was not set: calling the instance falls back to **tp_call**.

Any class that sets `Py_TPFLAGS_HAVE_VECTORCALL` must also set **tp_call** and make sure its behaviour is consistent with the *vectorcallfunc* function. This can be done by setting *tp_call* to **PyVectorcall_Call()**.

Alterado na versão 3.8: Before version 3.8, this slot was named `tp_print`. In Python 2.x, it was used for printing to a file. In Python 3.0 to 3.7, it was

unused.

Alterado na versão 3.12: Before version 3.12, it was not recommended for mutable heap types to implement the vectorcall protocol. When a user sets `__call__` in Python code, only *tp_call* is updated, likely making it inconsistent with the vectorcall function. Since 3.12, setting `__call__` will disable vectorcall optimization by clearing the `Py_TPFLAGS_HAVE_VECTORCALL` flag.

Inheritance:

This field is always inherited. However, the `Py_TPFLAGS_HAVE_VECTORCALL` flag is not always inherited. If it's not set, then the subclass won't use vectorcall, except when `PyVectorcall_Call()` is explicitly called.

getattrfunc PyTypeObject.tp_getattr

The corresponding slot ID `Py_tp_getattr` is part of the ABI Estável.

An optional pointer to the get-attribute-string function.

This field is deprecated. When it is defined, it should point to a function that acts the same as the `tp_getattro` function, but taking a C string instead of a Python string object to give the attribute name.

Inheritance:

Group: `tp_getattr`, `tp_getattro`

This field is inherited by subtypes together with `tp_getattro`: a subtype inherits both `tp_getattr` and **`tp_getattro`** from its base type when the subtype's **`tp_getattr`** and **`tp_getattro`** are both `NULL`.

setattrfunc PyTypeObject.tp_setattr

The corresponding slot ID `Py_tp_setattr` is part of the ABI Estável.

An optional pointer to the function for setting and deleting attributes.

This field is deprecated. When it is defined, it should point to a function that acts the same as the `tp_setattro` function, but taking a C string instead of a Python string object to give the attribute name.

Inheritance:

Group: **tp_setattr**, **tp_setattro**

This field is inherited by subtypes together with **tp_setattro**: a subtype inherits both **tp_setattr** and **tp_setattro** from its base type when the subtype's **tp_setattr** and **tp_setattro** are both `NULL`.

PyAsyncMethods *PyTypeObject.tp_as_async

Pointer to an additional structure that contains fields relevant only to objects which implement awaitable and asynchronous iterator protocols at the C-level. See Async Object Structures for details.

Adicionado na versão 3.5: Formerly known as `tp_compare` and `tp_reserved`.

Inheritance:

The **tp_as_async** field is not inherited, but the contained fields are inherited individually.

reprfunc PyTypeObject.tp_repr

The corresponding slot ID `Py_tp_repr` is part of the ABI Estável.

An optional pointer to a function that implements the built-in function **repr()**.

The signature is the same as for **PyObject_Repr()**:

```
PyObject *tp_repr(PyObject *self);
```

The function must return a string or a Unicode object. Ideally, this function should return a string that, when passed to **eval()**, given a suitable environment, returns an object with the same value. If this is not feasible, it should return a string starting with `'<'` and ending with `'>'` from which both the type and the value of the object can be deduced.

Inheritance:

This field is inherited by subtypes.

Padrão:

When this field is not set, a string of the form `<%s object at %p>` is returned, where `%s` is replaced by the type name, and `%p` by the object's memory address.

PyNumberMethods *PyTypeObject.tp_as_number

Pointer to an additional structure that contains fields relevant only to objects which implement the number protocol. These fields are documented in Number Object Structures.

Inheritance:

The **`tp_as_number`** field is not inherited, but the contained fields are inherited individually.

PySequenceMethods *PyTypeObject.tp_as_sequence

Pointer to an additional structure that contains fields relevant only to objects which implement the sequence protocol. These fields are documented in Sequence Object Structures.

Inheritance:

The **`tp_as_sequence`** field is not inherited, but the contained fields are inherited individually.

PyMappingMethods *PyTypeObject.tp_as_mapping

Pointer to an additional structure that contains fields relevant only to objects which implement the mapping protocol. These fields are documented in Mapping Object Structures.

Inheritance:

The **`tp_as_mapping`** field is not inherited, but the contained fields are inherited individually.

hashfunc PyTypeObject.tp_hash

The corresponding slot ID `Py_tp_hash` is part of the ABI Estável.

An optional pointer to a function that implements the built-in function **`hash()`**.

The signature is the same as for **`PyObject_Hash()`**:

```
Py_hash_t tp_hash(PyObject *);
```

The value `-1` should not be returned as a normal return value; when an error occurs during the computation of the hash value, the function should set an exception and return `-1`.

When this field is not set (*and* **`tp_richcompare`** is not set), an attempt to take the hash of the object raises **`TypeError`**. This is the same as setting it to **`PyObject_HashNotImplemented()`**.

This field can be set explicitly to **`PyObject_HashNotImplemented()`** to block inheritance of the hash method from a parent type. This is interpreted as the equivalent of `__hash__ = None` at the Python level, causing `isinstance(o, collections.Hashable)` to correctly return `False`. Note that the converse is also true - setting `__hash__ = None` on a class at the Python level will result in the `tp_hash` slot being set to **`PyObject_HashNotImplemented()`**.

Inheritance:

Group: **`tp_hash`**, **`tp_richcompare`**

This field is inherited by subtypes together with **`tp_richcompare`**: a subtype inherits both of **`tp_richcompare`** and **`tp_hash`**, when the subtype's **`tp_richcompare`** and **`tp_hash`** are both `NULL`.

Padrão:

`PyBaseObject_Type` uses **`PyObject_GenericHash()`**.

ternaryfunc PyTypeObject.tp_call

The corresponding slot ID `Py_tp_call` is part of the ABI Estável.

An optional pointer to a function that implements calling the object. This should be `NULL` if the object is not callable. The signature is the same as for **`PyObject_Call()`**:

```
PyObject *tp_call(PyObject *self, PyObject *args, PyObject *kwargs
```

Inheritance:

This field is inherited by subtypes.

reprfunc PyTypeObject.tp_str

The corresponding slot ID `Py_tp_str` is part of the ABI Estável.

An optional pointer to a function that implements the built-in operation **`str()`**. (Note that **`str`** is a type now, and **`str()`** calls the constructor for that type. This constructor calls **`PyObject_Str()`** to do the actual work, and **`PyObject_Str()`** will call this handler.)

The signature is the same as for **`PyObject_Str()`**:

```
PyObject *tp_str(PyObject *self);
```

The function must return a string or a Unicode object. It should be a "friendly" string representation of the object, as this is the representation that will be used, among other things, by the **`print()`** function.

Inheritance:

This field is inherited by subtypes.

Padrão:

When this field is not set, **`PyObject_Repr()`** is called to return a string representation.

getattrofunc PyTypeObject.tp_getattro

The corresponding slot ID `Py_tp_getattro` is part of the ABI Estável.

An optional pointer to the get-attribute function.

The signature is the same as for **`PyObject_GetAttr()`**:

```
PyObject *tp_getattro(PyObject *self, PyObject *attr);
```

It is usually convenient to set this field to **`PyObject_GenericGetAttr()`**, which implements the normal way of looking for object attributes.

Inheritance:

Group: **`tp_getattr`**, **`tp_getattro`**

This field is inherited by subtypes together with **tp_getattr**: a subtype inherits both **tp_getattr** and **tp_getattro** from its base type when the subtype's **tp_getattr** and **tp_getattro** are both NULL.

Padrão:

PyBaseObject_Type uses **PyObject_GenericGetAttr()**.

setattrofunc PyTypeObject.tp_setattro

The corresponding slot ID `Py_tp_setattro` *is part of the ABI Estável.*

An optional pointer to the function for setting and deleting attributes.

The signature is the same as for **PyObject_SetAttr()**:

```
int tp_setattro(PyObject *self, PyObject *attr, PyObject *value);
```

In addition, setting *value* to NULL to delete an attribute must be supported. It is usually convenient to set this field to **PyObject_GenericSetAttr()**, which implements the normal way of setting object attributes.

Inheritance:

Group: **tp_setattr**, **tp_setattro**

This field is inherited by subtypes together with **tp_setattr**: a subtype inherits both **tp_setattr** and **tp_setattro** from its base type when the subtype's **tp_setattr** and **tp_setattro** are both NULL.

Padrão:

PyBaseObject_Type uses **PyObject_GenericSetAttr()**.

PyBufferProcs *PyTypeObject.tp_as_buffer

Pointer to an additional structure that contains fields relevant only to objects which implement the buffer interface. These fields are documented in Buffer Object Structures.

Inheritance:

The **tp_as_buffer** field is not inherited, but the contained fields are inherited individually.

unsigned long PyTypeObject.tp_flags

See `Py_tp_flags` for the corresponding `Slot ID`.

This field is a bit mask of various flags. Some flags indicate variant semantics for certain situations; others are used to indicate that certain fields in the type object (or in the extension structures referenced via `tp_as_number`, `tp_as_sequence`, `tp_as_mapping`, and `tp_as_buffer`) that were historically not always present are valid; if such a flag bit is clear, the type fields it guards must not be accessed and must be considered to have a zero or `NULL` value instead.

Inheritance:

Inheritance of this field is complicated. Most flag bits are inherited individually, i.e. if the base type has a flag bit set, the subtype inherits this flag bit. The flag bits that pertain to extension structures are strictly inherited if the extension structure is inherited, i.e. the base type's value of the flag bit is copied into the subtype together with a pointer to the extension structure. The `Py_TPFLAGS_HAVE_GC` flag bit is inherited together with the `tp_traverse` and `tp_clear` fields, i.e. if the **`Py_TPFLAGS_HAVE_GC`** flag bit is clear in the subtype and the **`tp_traverse`** and **`tp_clear`** fields in the subtype exist and have `NULL` values.

Padrão:

`PyBaseObject_Type` uses `Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE`.

Bit Masks:

The following bit masks are currently defined; these can be ORed together using the `|` operator to form the value of the `tp_flags` field. The macro `PyType_HasFeature()` takes a type and a flags value, *tp* and *f*, and checks whether `tp->tp_flags & f` is non-zero.

Py_TPFLAGS_HEAPTYPE

This bit is set when the type object itself is allocated on the heap, for example, types created dynamically using `PyType_FromSpec()`. In this case, the `ob_type` field of its instances is considered a reference to the type, and the type object is INCREF'ed when a new instance is created, and DECREF'ed when an instance is destroyed (this does not apply to instances of subtypes; only the type referenced by the instance's ob_type

gets INCREF'ed or DECREF'ed). Heap types should also support garbage collection as they can form a reference cycle with their own module object.

Inheritance:

???

Py_TPFLAGS_BASETYPE

Parte da ABI Estável.

This bit is set when the type can be used as the base type of another type. If this bit is clear, the type cannot be subtyped (similar to a "final" class in Java).

Inheritance:

???

Py_TPFLAGS_READY

This bit is set when the type object has been fully initialized by `PyType_Ready()`.

Inheritance:

???

Py_TPFLAGS_READYING

This bit is set while `PyType_Ready()` is in the process of initializing the type object.

Inheritance:

???

Py_TPFLAGS_HAVE_GC

Parte da ABI Estável.

This bit is set when the object supports garbage collection. If this bit is set, memory for new instances (see `tp_alloc`) must be allocated using `PyObject_GC_New` or `PyType_GenericAlloc()` and deallocated (see `tp_free`) using `PyObject_GC_Del()`. More information in section Suporte a Coleta Cíclica de Lixo.

Inheritance:

Group: `Py_TPFLAGS_HAVE_GC`, `tp_traverse`, `tp_clear`

The `Py_TPFLAGS_HAVE_GC` flag bit is inherited together with the `tp_traverse` and `tp_clear` fields, i.e. if the **`Py_TPFLAGS_HAVE_GC`** flag bit is clear in the subtype and the **`tp_traverse`** and **`tp_clear`** fields in the subtype exist and have `NULL` values.

Py_TPFLAGS_DEFAULT

Parte da ABI Estável.

This is a bitmask of all the bits that pertain to the existence of certain fields in the type object and its extension structures. Currently, it includes the following bits: `Py_TPFLAGS_HAVE_STACKLESS_EXTENSION`.

Inheritance:

???

Py_TPFLAGS_METHOD_DESCRIPTOR

Parte da ABI Estável desde a versão 3.8.

This bit indicates that objects behave like unbound methods.

If this flag is set for `type(meth)`, then:

- `meth.__get__(obj, cls)(*args, **kwds)` (with `obj` not None) must be equivalent to `meth(obj, *args, **kwds)`.
- `meth.__get__(None, cls)(*args, **kwds)` must be equivalent to `meth(*args, **kwds)`.

This flag enables an optimization for typical method calls like `obj.meth()`: it avoids creating a temporary “bound method” object for `obj.meth`.

Adicionado na versão 3.8.

Inheritance:

This flag is never inherited by types without the `Py_TPFLAGS_IMMUTABLETYPE` flag set. For extension types, it is inherited whenever `tp_descr_get` is inherited.

Py_TPFLAGS_MANAGED_DICT

This bit indicates that instances of the class have a `__dict__` attribute, and that the space for the dictionary is managed by the VM.

If this flag is set, `Py_TPFLAGS_HAVE_GC` must also be set.

The type traverse function must call `PyObject_VisitManagedDict()` and its clear function must call `PyObject_ClearManagedDict()`.

Adicionado na versão 3.12.

Inheritance:

This flag is inherited unless the `tp_dictoffset` field is set in a superclass.

Py_TPFLAGS_MANAGED_WEAKREF

This bit indicates that instances of the class should be weakly referenceable.

If this flag is set, `Py_TPFLAGS_HAVE_GC` must also be set.

Adicionado na versão 3.12.

Inheritance:

This flag is inherited unless the `tp_weaklistoffset` field is set in a superclass.

Py_TPFLAGS_PREHEADER

These bits indicate that the VM will manage some fields by storing them before the object. Currently, this macro is equivalent to Py_TPFLAGS_MANAGED_DICT | Py_TPFLAGS_MANAGED_WEAKREF.

This macro value relies on the implementation of the VM, so its value is not stable and may change in a future version. Prefer using individual flags instead.

Adicionado na versão 3.12.

Py_TPFLAGS_ITEMS_AT_END

Parte da ABI Estável desde a versão 3.12.
Only usable with variable-size types, i.e. ones with non-zero `tp_itemsize`.

Indicates that the variable-sized portion of an instance of this type is at the end of the instance's memory area, at an offset of `Py_TYPE(obj)->tp_basicsize` (which may be different in each subclass).

When setting this flag, be sure that all superclasses either use this memory layout, or are not variable-sized. Python does not check this.

Adicionado na versão 3.12.

Inheritance:

This flag is inherited.

Py_TPFLAGS_LONG_SUBCLASS

Py_TPFLAGS_LIST_SUBCLASS

Py_TPFLAGS_TUPLE_SUBCLASS

Py_TPFLAGS_BYTES_SUBCLASS

Py_TPFLAGS_UNICODE_SUBCLASS

Py_TPFLAGS_DICT_SUBCLASS

Py_TPFLAGS_BASE_EXC_SUBCLASS

Py_TPFLAGS_TYPE_SUBCLASS
Functions such as `PyLong_Check()` will call `PyType_FastSubclass()` with one of these flags to quickly determine if a type is a subclass of a built-in type; such specific checks are faster than a generic check, like `PyObject_IsInstance()`. Custom types that inherit from built-ins should have their `tp_flags` set appropriately, or the code that interacts with such types will behave differently depending on what kind of check is used.

Py_TPFLAGS_HAVE_FINALIZE
This bit is set when the `tp_finalize` slot is present in the type structure.

Adicionado na versão 3.4.

Descontinuado desde a versão 3.8: This flag isn't necessary anymore, as the interpreter assumes the `tp_finalize` slot is always present in the type structure.

Py_TPFLAGS_HAVE_VECTORCALL

Parte da ABI Estável desde a versão 3.12.
This bit is set when the class implements the vectorcall protocol. See `tp_vectorcall_offset` for details.

Inheritance:

This bit is inherited if `tp_call` is also inherited.

Adicionado na versão 3.8: as `_Py_TPFLAGS_HAVE_VECTORCALL`

Alterado na versão 3.9: Renomeado para o nome atual, sem o sublinhado no início. O nome provisório antigo está suavemente descontinuado.

Alterado na versão 3.12: This flag is now removed from a class when the class's `__call__()` method is reassigned.

This flag can now be inherited by mutable classes.

Py_TPFLAGS_IMMUTABLETYPE

This bit is set for type objects that are immutable: type attributes cannot be set nor deleted.

`PyType_Ready()` automatically applies this flag to static types.

Inheritance:

This flag is not inherited.

Adicionado na versão 3.10.

Py_TPFLAGS_DISALLOW_INSTANTIATION

Disallow creating instances of the type: set `tp_new` to NULL and don't create the `__new__` key in the type dictionary.

The flag must be set before creating the type, not after. For example, it must be set before `PyType_Ready()` is called on the type.

The flag is set automatically on static types if `tp_base` is NULL or `&PyBaseObject_Type` and `tp_new` is NULL.

Inheritance:

This flag is not inherited. However, subclasses will not be instantiable unless they provide a non-NULL `tp_new` (which is only possible via the C API).

Nota

To disallow instantiating a class directly but allow instantiating its subclasses (e.g. for an abstract base class), do not use this flag. Instead, make `tp_new` only succeed for subclasses.

Adicionado na versão 3.10.

Py_TPFLAGS_MAPPING

This bit indicates that instances of the class may match mapping patterns when used as the subject of a `match` block. It is automatically set when registering or subclassing `collections.abc.Mapping`, and unset when registering `collections.abc.Sequence`.

Nota

`Py_TPFLAGS_MAPPING` and `Py_TPFLAGS_SEQUENCE` are mutually exclusive; it is an error to enable both flags simultaneously.

Inheritance:

This flag is inherited by types that do not already set `Py_TPFLAGS_SEQUENCE`.

Ver também

PEP 634 [https://peps.python.org/pep-0634/] – Structural Pattern Matching: Specification

Adicionado na versão 3.10.

Py_TPFLAGS_SEQUENCE

This bit indicates that instances of the class may match sequence patterns when used as the subject of a `match` block. It is automatically set when registering or subclassing `collections.abc.Sequence`, and unset when registering `collections.abc.Mapping`.

Nota

`Py_TPFLAGS_MAPPING` and `Py_TPFLAGS_SEQUENCE` are mutually exclusive; it is an error to enable both flags simultaneously.

Inheritance:

This flag is inherited by types that do not already set `Py_TPFLAGS_MAPPING`.

Ver também

PEP 634 [https://peps.python.org/pep-0634/] – Structural Pattern Matching: Specification

Adicionado na versão 3.10.

Py_TPFLAGS_VALID_VERSION_TAG

Internal. Do not set or unset this flag. To indicate that a class has changed call `PyType_Modified()`

Aviso

This flag is present in header files, but is not be used. It will be removed in a future version of CPython

Py_TPFLAGS_HAVE_VERSION_TAG

This macro does nothing. Historically, this would indicate that the `tp_version_tag` field was available and initialized.

Suavemente descontinuado desde a versão 3.13.

Py_TPFLAGS_INLINE_VALUES

This bit indicates that instances of this type will have an “inline values” array (containing the object’s attributes) placed directly after the end of the

object.

This requires that `Py_TPFLAGS_HAVE_GC` is set.

Inheritance:

This flag is not inherited.

Adicionado na versão 3.13.

Py_TPFLAGS_IS_ABSTRACT

This bit indicates that this is an abstract type and therefore cannot be instantiated.

Inheritance:

This flag is not inherited.

Ver também

`abc`

Py_TPFLAGS_HAVE_STACKLESS_EXTENSION

Internal. Do not set or unset this flag. Historically, this was a reserved flag for use in Stackless Python.

Aviso

This flag is present in header files, but is not be used. This may be removed in a future version of CPython.

const char *PyTypeObject.tp_doc

The corresponding slot ID `Py_tp_doc` is part of the ABI Estável.

An optional pointer to a NUL-terminated C string giving the docstring for this type object. This is exposed as the `__doc__` attribute on the type and instances of the type.

Inheritance:

This field is *not* inherited by subtypes.

traverseproc PyTypeObject.tp_traverse

The corresponding slot ID `Py_tp_traverse` is part of the ABI Estável.

An optional pointer to a traversal function for the garbage collector. This is only used if the **`Py_TPFLAGS_HAVE_GC`** flag bit is set.

See Traversal for documentation.

Inheritance:

Group: **`Py_TPFLAGS_HAVE_GC`**, **`tp_traverse`**, **`tp_clear`**

This field is inherited by subtypes together with **`tp_clear`** and the **`Py_TPFLAGS_HAVE_GC`** flag bit: the flag bit, **`tp_traverse`**, and **`tp_clear`** are all inherited from the base type if they are all zero in the subtype.

inquiry PyTypeObject.tp_clear

The corresponding slot ID `Py_tp_clear` is part of the ABI Estável.

An optional pointer to a clear function. The signature is:

```
int tp_clear(PyObject *);
```

O objetivo desta função é interromper os ciclos de referência que estão causando um isolado cíclico para que os objetos possam ser destruídos com segurança. Um objeto limpo é um objeto parcialmente destruído; o objeto não é obrigado a satisfazer as invariantes de projeto mantidas durante o uso normal.

`tp_clear` does not need to delete references to objects that can't participate in reference cycles, such as Python strings or Python integers. However, it may be convenient to clear all references, and write the type's **`tp_dealloc`** function to invoke **`tp_clear`** to avoid code duplication. (Beware that **`tp_clear`** might have already been called. Prefer calling idempotent functions like **`Py_CLEAR()`**.)

Any non-trivial cleanup should be performed in **`tp_finalize`** instead of **`tp_clear`**.

Nota

Se `tp_clear` não conseguir quebrar um ciclo de referência, os objetos no isolado cíclico poderão permanecer indefinidamente não coletáveis (“vazamento”). Veja `gc.garbage`.

Nota

Referents (direct and indirect) might have already been cleared; they are not guaranteed to be in a consistent state.

Nota

The `tp_clear` function can be called from any thread.

Nota

An object is not guaranteed to be automatically cleared before its destructor (`tp_dealloc`) is called.

This function differs from the destructor (`tp_dealloc`) in the following ways:

- The purpose of clearing an object is to remove references to other objects that might participate in a reference cycle. The purpose of the destructor, on the other hand, is a superset: it must release *all* resources it owns, including references to objects that cannot participate in a reference cycle (e.g., integers) as well as the object’s own memory (by calling `tp_free`).
- When `tp_clear` is called, other objects might still hold references to the object being cleared. Because of this, `tp_clear` must not deallocate the object’s own memory (`tp_free`). The destructor, on the other hand, is only called when no (strong) references exist, and as such, must safely destroy the object itself by deallocating it.
- `tp_clear` pode nunca ser chamado automaticamente. O destrutor de um objeto, por outro lado, será chamado automaticamente algum tempo depois que o objeto se tornar inacessível (ou seja, ou não há referências ao objeto ou o objeto é membro de um isolado cíclico).

No guarantees are made about when, if, or how often Python automatically clears an object, except:

- O Python não limpará automaticamente um objeto se ele for acessível, ou seja, se houver uma referência a ele e ele não for membro de um isolado cíclico.
- Python will not automatically clear an object if it has not been automatically finalized (see **tp_finalize**). (If the finalizer resurrected the object, the object may or may not be automatically finalized again before it is cleared.)
- Se um objeto for membro de um isolado cíclico, o Python não o limpará automaticamente se qualquer membro do isolado cíclico ainda não tiver sido finalizado automaticamente (**tp_finalize**).
- Python will not destroy an object until after any automatic calls to its **tp_clear** function have returned. This ensures that the act of breaking a reference cycle does not invalidate the `self` pointer while **tp_clear** is still executing.
- Python will not automatically call **tp_clear** multiple times concurrently.

Atualmente, o CPython limpa objetos automaticamente apenas quando necessário para quebrar ciclos de referência em um isolado cíclico, mas versões futuras podem limpar objetos regularmente antes de sua destruição.

Taken together, all **tp_clear** functions in the system must combine to break all reference cycles. This is subtle, and if in any doubt supply a **tp_clear** function. For example, the tuple type does not implement a **tp_clear** function, because it's possible to prove that no reference cycle can be composed entirely of tuples. Therefore the **tp_clear** functions of other types are responsible for breaking any cycle containing a tuple. This isn't immediately obvious, and there's rarely a good reason to avoid implementing **tp_clear**.

Implementations of **tp_clear** should drop the instance's references to those of its members that may be Python objects, and set its pointers to those members to `NULL`, as in the following example:

```
static int
local_clear(PyObject *op)
{
    localobject *self = (localobject *) op;
    Py_CLEAR(self->key);
    Py_CLEAR(self->args);
    Py_CLEAR(self->kw);
    Py_CLEAR(self->dict);
```

```
    return 0;
}
```

The **Py_CLEAR()** macro should be used, because clearing references is delicate: the reference to the contained object must not be released (via **Py_DECREF()**) until after the pointer to the contained object is set to `NULL`. This is because releasing the reference may cause the contained object to become trash, triggering a chain of reclamation activity that may include invoking arbitrary Python code (due to finalizers, or weakref callbacks, associated with the contained object). If it's possible for such code to reference *self* again, it's important that the pointer to the contained object be `NULL` at that time, so that *self* knows the contained object can no longer be used. The **Py_CLEAR()** macro performs the operations in a safe order.

If the **Py_TPFLAGS_MANAGED_DICT** bit is set in the **tp_flags** field, the clear function must call **PyObject_ClearManagedDict()** like this:

```
PyObject_ClearManagedDict((PyObject*)self);
```

More information about Python's garbage collection scheme can be found in section Suporte a Coleta Cíclica de Lixo.

Inheritance:

Group: **Py_TPFLAGS_HAVE_GC**, **tp_traverse**, **tp_clear**

This field is inherited by subtypes together with **tp_traverse** and the **Py_TPFLAGS_HAVE_GC** flag bit: the flag bit, **tp_traverse**, and **tp_clear** are all inherited from the base type if they are all zero in the subtype.

Ver também

Ciclo de vida do objeto for details about how this slot relates to other slots.

richcmpfunc PyTypeObject.tp_richcompare

The corresponding slot ID `Py_tp_richcompare` is part of the ABI Estável.

An optional pointer to the rich comparison function, whose signature is:

```
PyObject *tp_richcompare(PyObject *self, PyObject *other, int op);
```

The first parameter is guaranteed to be an instance of the type that is defined by `PyTypeObject`.

The function should return the result of the comparison (usually `Py_True` or `Py_False`). If the comparison is undefined, it must return `Py_NotImplemented`, if another error occurred it must return `NULL` and set an exception condition.

The following constants are defined to be used as the third argument for `tp_richcompare` and for `PyObject_RichCompare()`:

Constante	Comparação
Py_LT	`<`
Py_LE	`<=`
Py_EQ	`==`
Py_NE	`!=`
Py_GT	`>`
Py_GE	`>=`

The following macro is defined to ease writing rich comparison functions:

Py_RETURN_RICHCOMPARE(VAL_A, VAL_B, op)

Return `Py_True` or `Py_False` from the function, depending on the result of a comparison. VAL_A and VAL_B must be orderable by C comparison operators (for example, they may be C ints or floats). The third argument specifies the requested operation, as for `PyObject_RichCompare()`.

The returned value is a new strong reference.

On error, sets an exception and returns `NULL` from the function.

Adicionado na versão 3.7.

Inheritance:

Group: `tp_hash`, `tp_richcompare`

This field is inherited by subtypes together with `tp_hash`: a subtype inherits `tp_richcompare` and **`tp_hash`** when the subtype's **`tp_richcompare`** and **`tp_hash`** are both `NULL`.

Padrão:

`PyBaseObject_Type` provides a `tp_richcompare` implementation, which may be inherited. However, if only `tp_hash` is defined, not even the inherited function is used and instances of the type will not be able to participate in any comparisons.

Py_ssize_t PyTypeObject.tp_weaklistoffset

While this field is still supported, `Py_TPFLAGS_MANAGED_WEAKREF` should be used instead, if at all possible.

If the instances of this type are weakly referenceable, this field is greater than zero and contains the offset in the instance structure of the weak reference list head (ignoring the GC header, if present); this offset is used by `PyObject_ClearWeakRefs()` and the `PyWeakref_*` functions. The instance structure needs to include a field of type PyObject* which is initialized to `NULL`.

Do not confuse this field with `tp_weaklist`; that is the list head for weak references to the type object itself.

It is an error to set both the `Py_TPFLAGS_MANAGED_WEAKREF` bit and `tp_weaklistoffset`.

Inheritance:

This field is inherited by subtypes, but see the rules listed below. A subtype may override this offset; this means that the subtype uses a different weak reference list head than the base type. Since the list head is always found via `tp_weaklistoffset`, this should not be a problem.

Padrão:

If the `Py_TPFLAGS_MANAGED_WEAKREF` bit is set in the `tp_flags` field, then `tp_weaklistoffset` will be set to a negative value, to indicate that it is unsafe

to use this field.

getiterfunc PyTypeObject.tp_iter

The corresponding slot ID `Py_tp_iter` is part of the ABI Estável.

An optional pointer to a function that returns an iterator for the object. Its presence normally signals that the instances of this type are iterable (although sequences may be iterable without this function).

This function has the same signature as **`PyObject_GetIter()`**:

```
PyObject *tp_iter(PyObject *self);
```

Inheritance:

This field is inherited by subtypes.

iternextfunc PyTypeObject.tp_iternext

The corresponding slot ID `Py_tp_iternext` is part of the ABI Estável.

An optional pointer to a function that returns the next item in an iterator. The signature is:

```
PyObject *tp_iternext(PyObject *self);
```

When the iterator is exhausted, the `tp_iternext` function must return `NULL`; a **`StopIteration`** exception may or may not be set. When another error occurs, it must return `NULL` too. The presence of `tp_iternext` signals that the instances of this type are iterators.

Iterator types should also define the **`tp_iter`** function, and that function should return the iterator instance itself (not a new iterator instance).

This function has the same signature as **`PyIter_Next()`**.

Inheritance:

This field is inherited by subtypes.

struct PyMethodDef *PyTypeObject.tp_methods

The corresponding slot ID `Py_tp_methods` is part of the ABI Estável.

An optional pointer to a static `NULL`-terminated array of **`PyMethodDef`** structures, declaring regular methods of this type.

For each entry in the array, an entry is added to the type's dictionary (see **`tp_dict`** below) containing a method descriptor.

Inheritance:

This field is not inherited by subtypes (methods are inherited through a different mechanism).

struct PyMemberDef *PyTypeObject.tp_members

The corresponding slot ID `Py_tp_members` is part of the ABI Estável.

An optional pointer to a static `NULL`-terminated array of **`PyMemberDef`** structures, declaring regular data members (fields or slots) of instances of this type.

For each entry in the array, an entry is added to the type's dictionary (see **`tp_dict`** below) containing a member descriptor.

Inheritance:

This field is not inherited by subtypes (members are inherited through a different mechanism).

struct PyGetSetDef *PyTypeObject.tp_getset

The corresponding slot ID `Py_tp_getset` is part of the ABI Estável.

An optional pointer to a static `NULL`-terminated array of **`PyGetSetDef`** structures, declaring computed attributes of instances of this type.

For each entry in the array, an entry is added to the type's dictionary (see **`tp_dict`** below) containing a getset descriptor.

Inheritance:

This field is not inherited by subtypes (computed attributes are inherited through a different mechanism).

PyTypeObject *PyTypeObject.tp_base

An optional pointer to a base type from which type properties are inherited. At this level, only single inheritance is supported; multiple inheritance require dynamically creating a type object by calling the metatype.

For the corresponding slot ID, see `Py_tp_base`.

Nota

Slot initialization is subject to the rules of initializing globals. C99 requires the initializers to be "address constants". Function designators like `PyType_GenericNew()`, with implicit conversion to a pointer, are valid C99 address constants.

However, the unary '&' operator applied to a non-static variable like `PyBaseObject_Type` is not required to produce an address constant. Compilers may support this (gcc does), MSVC does not. Both compilers are strictly standard conforming in this particular behavior.

Consequently, `tp_base` should be set in the extension module's init function.

Inheritance:

This field is not inherited by subtypes (obviously).

Padrão:

This field defaults to `&PyBaseObject_Type` (which to Python programmers is known as the type `object`).

PyObject *PyTypeObject.tp_dict

The type's dictionary is stored here by `PyType_Ready()`.

This field should normally be initialized to `NULL` before PyType_Ready is called; it may also be initialized to a dictionary containing initial attributes for the type. Once `PyType_Ready()` has initialized the type, extra attributes for the type may be added to this dictionary only if they don't correspond to overloaded operations (like `__add__()`). Once initialization for the type has finished, this field should be treated as read-only.

Some types may not store their dictionary in this slot. Use `PyType_GetDict()` to retrieve the dictionary for an arbitrary type.

Alterado na versão 3.12: Internals detail: For static builtin types, this is always `NULL`. Instead, the dict for such types is stored on `PyInterpreterState`. Use **`PyType_GetDict()`** to get the dict for an arbitrary type.

Inheritance:

This field is not inherited by subtypes (though the attributes defined in here are inherited through a different mechanism).

Padrão:

If this field is `NULL`, **`PyType_Ready()`** will assign a new dictionary to it.

> **Aviso**
> It is not safe to use **`PyDict_SetItem()`** on or otherwise modify **`tp_dict`** with the dictionary C-API.

descrgetfunc PyTypeObject.tp_descr_get

The corresponding slot ID `Py_tp_descr_get` is part of the ABI Estável.

An optional pointer to a "descriptor get" function.

A assinatura da função é:

```
PyObject * tp_descr_get(PyObject *self, PyObject *obj, PyObject *t
```

Inheritance:

This field is inherited by subtypes.

descrsetfunc PyTypeObject.tp_descr_set

The corresponding slot ID `Py_tp_descr_set` is part of the ABI Estável.

An optional pointer to a function for setting and deleting a descriptor's value.

A assinatura da função é:

```
int tp_descr_set(PyObject *self, PyObject *obj, PyObject *value);
```

The *value* argument is set to `NULL` to delete the value.

Inheritance:

This field is inherited by subtypes.

Py_ssize_t PyTypeObject.tp_dictoffset

While this field is still supported, `Py_TPFLAGS_MANAGED_DICT` should be used instead, if at all possible.

If the instances of this type have a dictionary containing instance variables, this field is non-zero and contains the offset in the instances of the type of the instance variable dictionary; this offset is used by `PyObject_GenericGetAttr()`.

Do not confuse this field with `tp_dict`; that is the dictionary for attributes of the type object itself.

The value specifies the offset of the dictionary from the start of the instance structure.

The `tp_dictoffset` should be regarded as write-only. To get the pointer to the dictionary call `PyObject_GenericGetDict()`. Calling `PyObject_GenericGetDict()` may need to allocate memory for the dictionary, so it is may be more efficient to call `PyObject_GetAttr()` when accessing an attribute on the object.

It is an error to set both the `Py_TPFLAGS_MANAGED_DICT` bit and `tp_dictoffset`.

Inheritance:

This field is inherited by subtypes. A subtype should not override this offset; doing so could be unsafe, if C code tries to access the dictionary at the previous offset. To properly support inheritance, use `Py_TPFLAGS_MANAGED_DICT`.

Padrão:

This slot has no default. For static types, if the field is `NULL` then no `__dict__` gets created for instances.

If the **Py_TPFLAGS_MANAGED_DICT** bit is set in the **tp_flags** field, then **tp_dictoffset** will be set to `-1`, to indicate that it is unsafe to use this field.

initproc PyTypeObject.tp_init

The corresponding slot ID `Py_tp_init` is part of the ABI Estável.

An optional pointer to an instance initialization function.

This function corresponds to the **__init__()** method of classes. Like **__init__()**, it is possible to create an instance without calling **__init__()**, and it is possible to reinitialize an instance by calling its **__init__()** method again.

A assinatura da função é:

```
int tp_init(PyObject *self, PyObject *args, PyObject *kwds);
```

The self argument is the instance to be initialized; the *args* and *kwds* arguments represent positional and keyword arguments of the call to **__init__()**.

The **tp_init** function, if not `NULL`, is called when an instance is created normally by calling its type, after the type's **tp_new** function has returned an instance of the type. If the **tp_new** function returns an instance of some other type that is not a subtype of the original type, no **tp_init** function is called; if **tp_new** returns an instance of a subtype of the original type, the subtype's **tp_init** is called.

Returns `0` on success, `-1` and sets an exception on error.

Inheritance:

This field is inherited by subtypes.

Padrão:

For static types this field does not have a default.

allocfunc PyTypeObject.tp_alloc

The corresponding slot ID `Py_tp_alloc` is part of the ABI Estável.

An optional pointer to an instance allocation function.

A assinatura da função é:

```
PyObject *tp_alloc(PyTypeObject *self, Py_ssize_t nitems);
```

Inheritance:

Static subtypes inherit this slot, which will be **`PyType_GenericAlloc()`** if inherited from **`object`**.

Heap subtypes do not inherit this slot.

Padrão:

For heap subtypes, this field is always set to **`PyType_GenericAlloc()`**.

For static subtypes, this slot is inherited (see above).

newfunc PyTypeObject.tp_new

The corresponding slot ID `Py_tp_new` is part of the ABI Estável.

An optional pointer to an instance creation function.

A assinatura da função é:

```
PyObject *tp_new(PyTypeObject *subtype, PyObject *args, PyObject *
```

The *subtype* argument is the type of the object being created; the *args* and *kwds* arguments represent positional and keyword arguments of the call to the type. Note that *subtype* doesn't have to equal the type whose **`tp_new`** function is called; it may be a subtype of that type (but not an unrelated type).

The **`tp_new`** function should call `subtype->tp_alloc(subtype, nitems)` to allocate space for the object, and then do only as much further initialization as is absolutely necessary. Initialization that can safely be ignored or repeated should be placed in the **`tp_init`** handler. A good rule of thumb is that for immutable types, all initialization should take place in **`tp_new`**, while for mutable types, most initialization should be deferred to **`tp_init`**.

Set the **`Py_TPFLAGS_DISALLOW_INSTANTIATION`** flag to disallow creating instances of the type in Python.

Inheritance:

This field is inherited by subtypes, except it is not inherited by static types whose **tp_base** is `NULL` or `&PyBaseObject_Type`.

Padrão:

For static types this field has no default. This means if the slot is defined as `NULL`, the type cannot be called to create new instances; presumably there is some other way to create instances, like a factory function.

freefunc PyTypeObject.tp_free

The corresponding slot ID `Py_tp_free` is part of the ABI Estável.

An optional pointer to an instance deallocation function. Its signature is:

```
void tp_free(void *self);
```

This function must free the memory allocated by **tp_alloc**.

Inheritance:

Static subtypes inherit this slot, which will be **PyObject_Free()** if inherited from **object**. Exception: If the type supports garbage collection (i.e., the **Py_TPFLAGS_HAVE_GC** flag is set in **tp_flags**) and it would inherit **PyObject_Free()**, then this slot is not inherited but instead defaults to **PyObject_GC_Del()**.

Heap subtypes do not inherit this slot.

Padrão:

For heap subtypes, this slot defaults to a deallocator suitable to match **PyType_GenericAlloc()** and the value of the **Py_TPFLAGS_HAVE_GC** flag.

For static subtypes, this slot is inherited (see above).

inquiry PyTypeObject.tp_is_gc

The corresponding slot ID `Py_tp_is_gc` is part of the ABI Estável.

An optional pointer to a function called by the garbage collector.

The garbage collector needs to know whether a particular object is collectible or not. Normally, it is sufficient to look at the object's type's **tp_flags** field,

and check the `Py_TPFLAGS_HAVE_GC` flag bit. But some types have a mixture of statically and dynamically allocated instances, and the statically allocated instances are not collectible. Such types should define this function; it should return `1` for a collectible instance, and `0` for a non-collectible instance. The signature is:

```
int tp_is_gc(PyObject *self);
```

(The only example of this are types themselves. The metatype, `PyType_Type`, defines this function to distinguish between statically and dynamically allocated types.)

Inheritance:

This field is inherited by subtypes.

Padrão:

This slot has no default. If this field is `NULL`, `Py_TPFLAGS_HAVE_GC` is used as the functional equivalent.

PyObject *PyTypeObject.tp_bases

Tuple of base types.

This field should be set to `NULL` and treated as read-only. Python will fill it in when the type is `initialized`.

For the corresponding slot ID, see `Py_tp_bases`.

> **Aviso**
>
> Multiple inheritance does not work well for statically defined types. If you set `tp_bases` to a tuple, Python will not raise an error, but some slots will only be inherited from the first base.

Inheritance:

This field is not inherited.

PyObject *PyTypeObject.tp_mro

Tuple containing the expanded set of base types, starting with the type itself and ending with **object**, in Method Resolution Order.

This field should be set to `NULL` and treated as read-only. Python will fill it in when the type is **initialized**.

Inheritance:

This field is not inherited; it is calculated fresh by **PyType_Ready()**.

PyObject *PyTypeObject.tp_cache

Unused. Internal use only.

Inheritance:

This field is not inherited.

void *PyTypeObject.tp_subclasses

A collection of subclasses. Internal use only. May be an invalid pointer.

To get a list of subclasses, call the Python method **__subclasses__()**.

Alterado na versão 3.12: For some types, this field does not hold a valid PyObject*. The type was changed to void* to indicate this.

Inheritance:

This field is not inherited.

PyObject *PyTypeObject.tp_weaklist

Weak reference list head, for weak references to this type object. Not inherited. Internal use only.

Alterado na versão 3.12: Internals detail: For the static builtin types this is always `NULL`, even if weakrefs are added. Instead, the weakrefs for each are stored on `PyInterpreterState`. Use the public C-API or the internal `_PyObject_GET_WEAKREFS_LISTPTR()` macro to avoid the distinction.

Inheritance:

This field is not inherited.

destructor PyTypeObject.tp_del

The corresponding slot ID `Py_tp_del` is part of the ABI Estável.

This field is deprecated. Use **`tp_finalize`** instead.

unsigned int PyTypeObject.tp_version_tag

Used to index into the method cache. Internal use only.

Inheritance:

This field is not inherited.

destructor PyTypeObject.tp_finalize

The corresponding slot ID `Py_tp_finalize` is part of the ABI Estável desde a versão 3.5.

An optional pointer to an instance finalization function. This is the C implementation of the **`__del__()`** special method. Its signature is:

```
void tp_finalize(PyObject *self);
```

The primary purpose of finalization is to perform any non-trivial cleanup that must be performed before the object is destroyed, while the object and any other objects it directly or indirectly references are still in a consistent state. The finalizer is allowed to execute arbitrary Python code.

Before Python automatically finalizes an object, some of the object's direct or indirect referents might have themselves been automatically finalized. However, none of the referents will have been automatically cleared (**`tp_clear`**) yet.

Other non-finalized objects might still be using a finalized object, so the finalizer must leave the object in a sane state (e.g., invariants are still met).

Nota

After Python automatically finalizes an object, Python might start automatically clearing (**`tp_clear`**) the object and its referents (direct and indirect). Cleared objects are not guaranteed to be in a consistent state; a finalized object must be able to tolerate cleared referents.

Nota

An object is not guaranteed to be automatically finalized before its destructor (`tp_dealloc`) is called. It is recommended to call `PyObject_CallFinalizerFromDealloc()` at the beginning of `tp_dealloc` to guarantee that the object is always finalized before destruction.

Nota

The `tp_finalize` function can be called from any thread, although the GIL will be held.

Nota

The `tp_finalize` function can be called during shutdown, after some global variables have been deleted. See the documentation of the `__del__()` method for details.

When Python finalizes an object, it behaves like the following algorithm:

1. Python might mark the object as *finalized*. Currently, Python always marks objects whose type supports garbage collection (i.e., the `Py_TPFLAGS_HAVE_GC` flag is set in `tp_flags`) and never marks other types of objects; this might change in a future version.
2. If the object is not marked as *finalized* and its `tp_finalize` finalizer function is non-`NULL`, the finalizer function is called.
3. Se a função finalizadora foi chamada e o finalizador tornou o objeto acessível (ou seja, há uma referência ao objeto e ele não é membro de um isolado cíclico), então o finalizador *ressuscitou* o objeto. Não é especificado se o finalizador também pode ressuscitar o objeto adicionando uma nova referência ao objeto que não o torne acessível, ou seja, o objeto (ainda) é membro de um isolado cíclico.
4. If the finalizer resurrected the object, the object's pending destruction is canceled and the object's *finalized* mark might be removed if present. Currently, Python never removes the *finalized* mark; this might change in a future version.

Automatic finalization refers to any finalization performed by Python except via calls to `PyObject_CallFinalizer()` or `PyObject_CallFinalizerFromDealloc()`. No guarantees are made about when, if, or how often an object is automatically finalized, except:

- O Python não finalizará automaticamente um objeto se ele for acessível, ou seja, se houver uma referência a ele e ele não for membro de um isolado cíclico.
- Python will not automatically finalize an object if finalizing it would not mark the object as *finalized*. Currently, this applies to objects whose type does not support garbage collection, i.e., the `Py_TPFLAGS_HAVE_GC` flag is not set. Such objects can still be manually finalized by calling `PyObject_CallFinalizer()` or `PyObject_CallFinalizerFromDealloc()`.
- O Python não finalizará automaticamente quaisquer dois membros de um isolado cíclico simultaneamente.
- Python will not automatically finalize an object after it has automatically cleared (`tp_clear`) the object.
- Se um objeto for membro de um isolado cíclico, o Python não o finalizará automaticamente após limpar automaticamente (veja `tp_clear`) qualquer outro membro.
- O Python finalizará automaticamente cada membro de um isolado cíclico antes de limpar automaticamente (veja `tp_clear`) qualquer um deles.
- If Python is going to automatically clear an object (`tp_clear`), it will automatically finalize the object first.

Atualmente, o Python finaliza automaticamente apenas objetos que são membros de um isolado cíclico, mas versões futuras podem finalizar objetos regularmente antes de sua destruição.

To manually finalize an object, do not call this function directly; call `PyObject_CallFinalizer()` or `PyObject_CallFinalizerFromDealloc()` instead.

`tp_finalize` should leave the current exception status unchanged. The recommended way to write a non-trivial finalizer is to back up the exception at the beginning by calling `PyErr_GetRaisedException()` and restore the exception at the end by calling `PyErr_SetRaisedException()`. If an exception is encountered in the middle of the finalizer, log and clear it with `PyErr_WriteUnraisable()` or `PyErr_FormatUnraisable()`. For example:

```
static void
foo_finalize(PyObject *self)
{
    // Save the current exception, if any.
    PyObject *exc = PyErr_GetRaisedException();
```

```
    // ...

    if (do_something_that_might_raise() != success_indicator) {
        PyErr_WriteUnraisable(self);
        goto done;
    }

done:
    // Restore the saved exception.  This silently discards any ex
    // raised above, so be sure to call PyErr_WriteUnraisable firs
    // necessary.
    PyErr_SetRaisedException(exc);
}
```

Inheritance:

This field is inherited by subtypes.

Adicionado na versão 3.4.

Alterado na versão 3.8: Before version 3.8 it was necessary to set the **`Py_TPFLAGS_HAVE_FINALIZE`** flags bit in order for this field to be used. This is no longer required.

Ver também

- **PEP 442** [https://peps.python.org/pep-0442/]: "Safe object finalization"
- Ciclo de vida do objeto for details about how this slot relates to other slots.
- **`PyObject_CallFinalizer()`**
- **`PyObject_CallFinalizerFromDealloc()`**

vectorcallfunc PyTypeObject.tp_vectorcall

The corresponding slot ID `Py_tp_vectorcall` is part of the ABI Estável desde a versão 3.14.

A vectorcall function to use for calls of this type object (rather than instances). In other words, `tp_vectorcall` can be used to optimize `type.__call__`, which typically returns a new instance of *type*.

As with any vectorcall function, if `tp_vectorcall` is `NULL`, the *tp_call* protocol (`Py_TYPE(type)->tp_call`) is used instead.

Nota

The vectorcall protocol requires that the vectorcall function has the same behavior as the corresponding `tp_call`. This means that `type->tp_vectorcall` must match the behavior of `Py_TYPE(type)->tp_call`.

Specifically, if *type* uses the default metaclass, `type->tp_vectorcall` must behave the same as PyType_Type->tp_call, which:

- calls `type->tp_new`,
- if the result is a subclass of *type*, calls `type->tp_init` on the result of `tp_new`, and
- returns the result of `tp_new`.

Typically, `tp_vectorcall` is overridden to optimize this process for specific **`tp_new`** and **`tp_init`**. When doing this for user-subclassable types, note that both can be overridden (using **`__new__()`** and **`__init__()`**, respectively).

Inheritance:

This field is never inherited.

Adicionado na versão 3.9: (the field exists since 3.8 but it's only used since 3.9)

unsigned char PyTypeObject.tp_watched

Internal. Do not use.

Adicionado na versão 3.12.

Static Types

Traditionally, types defined in C code are *static*, that is, a static **`PyTypeObject`** structure is defined directly in code and initialized using **`PyType_Ready()`**.

This results in types that are limited relative to types defined in Python:

- Static types are limited to one base, i.e. they cannot use multiple inheritance.
- Static type objects (but not necessarily their instances) are immutable. It is not possible to add or modify the type object's attributes from Python.

- Static type objects are shared across sub-interpreters, so they should not include any subinterpreter-specific state.

Also, since **PyTypeObject** is only part of the Limited API as an opaque struct, any extension modules using static types must be compiled for a specific Python minor version.

Tipos no heap

An alternative to static types is *heap-allocated types*, or *heap types* for short, which correspond closely to classes created by Python's `class` statement. Heap types have the **Py_TPFLAGS_HEAPTYPE** flag set.

This is done by filling a **PyType_Spec** structure and calling **PyType_FromSpec()**, **PyType_FromSpecWithBases()**, **PyType_FromModuleAndSpec()**, or **PyType_FromMetaclass()**.

Number Object Structures

type PyNumberMethods

This structure holds pointers to the functions which an object uses to implement the number protocol. Each function is used by the function of similar name documented in the Protocolo de número section.

Here is the structure definition:

```
typedef struct {
     binaryfunc nb_add;
     binaryfunc nb_subtract;
     binaryfunc nb_multiply;
     binaryfunc nb_remainder;
     binaryfunc nb_divmod;
     ternaryfunc nb_power;
     unaryfunc nb_negative;
     unaryfunc nb_positive;
     unaryfunc nb_absolute;
     inquiry nb_bool;
     unaryfunc nb_invert;
     binaryfunc nb_lshift;
     binaryfunc nb_rshift;
     binaryfunc nb_and;
```

```
    binaryfunc nb_xor;
    binaryfunc nb_or;
    unaryfunc nb_int;
    void *nb_reserved;
    unaryfunc nb_float;

    binaryfunc nb_inplace_add;
    binaryfunc nb_inplace_subtract;
    binaryfunc nb_inplace_multiply;
    binaryfunc nb_inplace_remainder;
    ternaryfunc nb_inplace_power;
    binaryfunc nb_inplace_lshift;
    binaryfunc nb_inplace_rshift;
    binaryfunc nb_inplace_and;
    binaryfunc nb_inplace_xor;
    binaryfunc nb_inplace_or;

    binaryfunc nb_floor_divide;
    binaryfunc nb_true_divide;
    binaryfunc nb_inplace_floor_divide;
    binaryfunc nb_inplace_true_divide;

    unaryfunc nb_index;

    binaryfunc nb_matrix_multiply;
    binaryfunc nb_inplace_matrix_multiply;
} PyNumberMethods;
```

Nota

Binary and ternary functions must check the type of all their operands, and implement the necessary conversions (at least one of the operands is an instance of the defined type). If the operation is not defined for the given operands, binary and ternary functions must return `Py_NotImplemented`, if another error occurred they must return `NULL` and set an exception.

Nota

The **`nb_reserved`** field should always be `NULL`. It was previously called **`nb_long`**, and was renamed in Python 3.0.1.

binaryfunc PyNumberMethods.nb_add

The corresponding slot ID `Py_nb_add` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_subtract

The corresponding slot ID `Py_nb_subtract` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_multiply

The corresponding slot ID `Py_nb_multiply` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_remainder

The corresponding slot ID `Py_nb_remainder` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_divmod

The corresponding slot ID `Py_nb_divmod` is part of the ABI Estável.

ternaryfunc PyNumberMethods.nb_power

The corresponding slot ID `Py_nb_power` is part of the ABI Estável.

unaryfunc PyNumberMethods.nb_negative

The corresponding slot ID `Py_nb_negative` is part of the ABI Estável.

unaryfunc PyNumberMethods.nb_positive

The corresponding slot ID `Py_nb_positive` is part of the ABI Estável.

unaryfunc PyNumberMethods.nb_absolute

The corresponding slot ID `Py_nb_absolute` is part of the ABI Estável.

inquiry PyNumberMethods.nb_bool

The corresponding slot ID `Py_nb_bool` is part of the ABI Estável.

unaryfunc PyNumberMethods.nb_invert

The corresponding slot ID `Py_nb_invert` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_lshift

The corresponding slot ID `Py_nb_lshift` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_rshift

The corresponding slot ID `Py_nb_rshift` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_and

The corresponding slot ID `Py_nb_and` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_xor

The corresponding slot ID `Py_nb_xor` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_or

The corresponding slot ID `Py_nb_or` is part of the ABI Estável.

unaryfunc PyNumberMethods.nb_int

The corresponding slot ID `Py_nb_int` is part of the ABI Estável.

void *PyNumberMethods.nb_reserved

unaryfunc PyNumberMethods.nb_float

The corresponding slot ID `Py_nb_float` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_add

The corresponding slot ID `Py_nb_inplace_add` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_subtract

The corresponding slot ID `Py_nb_inplace_subtract` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_multiply

The corresponding slot ID `Py_nb_inplace_multiply` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_remainder

The corresponding slot ID `Py_nb_inplace_remainder` is part of the ABI Estável.

ternaryfunc PyNumberMethods.nb_inplace_power

The corresponding slot ID `Py_nb_inplace_power` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_lshift

The corresponding slot ID `Py_nb_inplace_lshift` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_rshift

The corresponding slot ID `Py_nb_inplace_rshift` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_and

The corresponding slot ID `Py_nb_inplace_and` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_xor

The corresponding slot ID `Py_nb_inplace_xor` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_or

The corresponding slot ID `Py_nb_inplace_or` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_floor_divide

The corresponding slot ID `Py_nb_floor_divide` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_true_divide

The corresponding slot ID `Py_nb_true_divide` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_floor_divide

The corresponding slot ID `Py_nb_inplace_floor_divide` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_inplace_true_divide

The corresponding slot ID `Py_nb_inplace_true_divide` is part of the ABI Estável.

unaryfunc PyNumberMethods.nb_index

The corresponding slot ID `Py_nb_index` is part of the ABI Estável.

binaryfunc PyNumberMethods.nb_matrix_multiply

The corresponding slot ID `Py_nb_matrix_multiply` is part of the ABI Estável desde a versão 3.5.

binaryfunc PyNumberMethods.nb_inplace_matrix_multiply

The corresponding slot ID `Py_nb_inplace_matrix_multiply` is part of the ABI Estável desde a versão 3.5.

Mapping Object Structures

type PyMappingMethods

This structure holds pointers to the functions which an object uses to implement the mapping protocol. It has three members:

lenfunc PyMappingMethods.mp_length

The corresponding slot ID `Py_mp_length` is part of the ABI Estável.

This function is used by `PyMapping_Size()` and `PyObject_Size()`, and has the same signature. This slot may be set to `NULL` if the object has no defined length.

binaryfunc PyMappingMethods.mp_subscript

The corresponding slot ID `Py_mp_subscript` is part of the ABI Estável.

This function is used by `PyObject_GetItem()` and `PySequence_GetSlice()`, and has the same signature as **`PyObject_GetItem()`**. This slot must be filled for the `PyMapping_Check()` function to return `1`, it can be `NULL` otherwise.

objobjargproc PyMappingMethods.mp_ass_subscript

The corresponding slot ID `Py_mp_ass_subscript` is part of the ABI Estável.

This function is used by `PyObject_SetItem()`, `PyObject_DelItem()`, `PySequence_SetSlice()` and `PySequence_DelSlice()`. It has the same signature as **`PyObject_SetItem()`**, but *`v`* can also be set to `NULL` to delete an item. If this slot is `NULL`, the object does not support item assignment and deletion.

Sequence Object Structures

type PySequenceMethods

This structure holds pointers to the functions which an object uses to implement the sequence protocol.

lenfunc PySequenceMethods.sq_length

The corresponding slot ID `Py_sq_length` is part of the ABI Estável.

This function is used by `PySequence_Size()` and `PyObject_Size()`, and has the same signature. It is also used for handling negative indices via the `sq_item` and the `sq_ass_item` slots.

binaryfunc PySequenceMethods.sq_concat

The corresponding slot ID `Py_sq_concat` is part of the ABI Estável.

This function is used by `PySequence_Concat()` and has the same signature. It is also used by the + operator, after trying the numeric addition via the `nb_add` slot.

ssizeargfunc PySequenceMethods.sq_repeat

The corresponding slot ID `Py_sq_repeat` is part of the ABI Estável.

This function is used by **`PySequence_Repeat()`** and has the same signature. It is also used by the `*` operator, after trying numeric multiplication via the **`nb_multiply`** slot.

ssizeargfunc PySequenceMethods.sq_item

The corresponding slot ID `Py_sq_item` is part of the ABI Estável.

This function is used by **`PySequence_GetItem()`** and has the same signature. It is also used by **`PyObject_GetItem()`**, after trying the subscription via the **`mp_subscript`** slot. This slot must be filled for the **`PySequence_Check()`** function to return `1`, it can be `NULL` otherwise.

Negative indexes are handled as follows: if the **`sq_length`** slot is filled, it is called and the sequence length is used to compute a positive index which is passed to **`sq_item`**. If **`sq_length`** is `NULL`, the index is passed as is to the function.

ssizeobjargproc PySequenceMethods.sq_ass_item

The corresponding slot ID `Py_sq_ass_item` is part of the ABI Estável.

This function is used by **`PySequence_SetItem()`** and has the same signature. It is also used by **`PyObject_SetItem()`** and **`PyObject_DelItem()`**, after trying the item assignment and deletion via the **`mp_ass_subscript`** slot. This slot may be left to `NULL` if the object does not support item assignment and deletion.

objobjproc PySequenceMethods.sq_contains

The corresponding slot ID `Py_sq_contains` is part of the ABI Estável.

This function may be used by **`PySequence_Contains()`** and has the same signature. This slot may be left to `NULL`, in this case **`PySequence_Contains()`** simply traverses the sequence until it finds a match.

binaryfunc PySequenceMethods.sq_inplace_concat

The corresponding slot ID `Py_sq_inplace_concat` is part of the ABI Estável.

This function is used by **`PySequence_InPlaceConcat()`** and has the same signature. It should modify its first operand, and return it. This slot may be left

to `NULL`, in this case **`PySequence_InPlaceConcat()`** will fall back to **`PySequence_Concat()`**. It is also used by the augmented assignment `+=`, after trying numeric in-place addition via the **`nb_inplace_add`** slot.

ssizeargfunc PySequenceMethods.sq_inplace_repeat

The corresponding slot ID `Py_sq_inplace_repeat` is part of the ABI Estável.

This function is used by **`PySequence_InPlaceRepeat()`** and has the same signature. It should modify its first operand, and return it. This slot may be left to `NULL`, in this case **`PySequence_InPlaceRepeat()`** will fall back to **`PySequence_Repeat()`**. It is also used by the augmented assignment `*=`, after trying numeric in-place multiplication via the **`nb_inplace_multiply`** slot.

Buffer Object Structures

type PyBufferProcs

This structure holds pointers to the functions required by the Buffer protocol. The protocol defines how an exporter object can expose its internal data to consumer objects.

getbufferproc PyBufferProcs.bf_getbuffer

The corresponding slot ID `Py_bf_getbuffer` is part of the ABI Estável desde a versão 3.11.

The signature of this function is:

```
int (PyObject *exporter, Py_buffer *view, int flags);
```

Handle a request to *exporter* to fill in *view* as specified by *flags*. Except for point (3), an implementation of this function MUST take these steps:

1. Check if the request can be met. If not, raise **`BufferError`**, set `view->obj` to `NULL` and return `-1`.
2. Fill in the requested fields.
3. Increment an internal counter for the number of exports.
4. Set `view->obj` to *exporter* and increment `view->obj`.
5. Retorna `0`.

Thread safety:

In the free-threaded build, implementations must ensure:

- The export counter increment in step (3) is atomic.
- The underlying buffer data remains valid and at a stable memory location for the lifetime of all exports.
- For objects that support resizing or reallocation (such as **bytearray**), the export counter is checked atomically before such operations, and **BufferError** is raised if exports exist.
- The function is safe to call concurrently from multiple threads.

See also Segurança para thread para objetos memoryview for the Python-level thread safety guarantees of **memoryview** objects.

If *exporter* is part of a chain or tree of buffer providers, two main schemes can be used:

- Re-export: Each member of the tree acts as the exporting object and sets `view->obj` to a new reference to itself.
- Redirect: The buffer request is redirected to the root object of the tree. Here, `view->obj` will be a new reference to the root object.

The individual fields of *view* are described in section Buffer structure, the rules how an exporter must react to specific requests are in section Buffer request types.

All memory pointed to in the **Py_buffer** structure belongs to the exporter and must remain valid until there are no consumers left. **format**, **shape**, **strides**, **suboffsets** and **internal** are read-only for the consumer.

PyBuffer_FillInfo() provides an easy way of exposing a simple bytes buffer while dealing correctly with all request types.

PyObject_GetBuffer() is the interface for the consumer that wraps this function.

releasebufferproc PyBufferProcs.bf_releasebuffer

The corresponding slot ID `Py_bf_releasebuffer` is part of the ABI Estável desde a versão 3.11.

The signature of this function is:

```
void (PyObject *exporter, Py_buffer *view);
```

Handle a request to release the resources of the buffer. If no resources need to be released, **`PyBufferProcs.bf_releasebuffer`** may be `NULL`. Otherwise, a standard implementation of this function will take these optional steps:

1. Decrement an internal counter for the number of exports.
2. If the counter is `0`, free all memory associated with *view*.

Thread safety:

In the free-threaded build:

- The export counter decrement in step (1) must be atomic.
- Resource cleanup when the counter reaches zero must be done atomically, as the final release may race with concurrent releases from other threads and deallocation must only happen once.

The exporter MUST use the **`internal`** field to keep track of buffer-specific resources. This field is guaranteed to remain constant, while a consumer MAY pass a copy of the original buffer as the *view* argument.

This function MUST NOT decrement `view->obj`, since that is done automatically in **`PyBuffer_Release()`** (this scheme is useful for breaking reference cycles).

`PyBuffer_Release()` is the interface for the consumer that wraps this function.

Async Object Structures

Adicionado na versão 3.5.

type PyAsyncMethods

This structure holds pointers to the functions required to implement awaitable and asynchronous iterator objects.

Here is the structure definition:

```
typedef struct {
    unaryfunc am_await;
    unaryfunc am_aiter;
    unaryfunc am_anext;
    sendfunc am_send;
} PyAsyncMethods;
```

unaryfunc PyAsyncMethods.am_await

The corresponding slot ID `Py_am_await` is part of the ABI Estável desde a versão 3.5.

The signature of this function is:

```
PyObject *am_await(PyObject *self);
```

The returned object must be an iterator, i.e. **`PyIter_Check()`** must return `1` for it.

This slot may be set to `NULL` if an object is not an awaitable.

unaryfunc PyAsyncMethods.am_aiter

The corresponding slot ID `Py_am_aiter` is part of the ABI Estável desde a versão 3.5.

The signature of this function is:

```
PyObject *am_aiter(PyObject *self);
```

Must return an asynchronous iterator object. See **`__anext__()`** for details.

This slot may be set to `NULL` if an object does not implement asynchronous iteration protocol.

unaryfunc PyAsyncMethods.am_anext

The corresponding slot ID `Py_am_anext` is part of the ABI Estável desde a versão 3.5.

The signature of this function is:

```
PyObject *am_anext(PyObject *self);
```

Must return an awaitable object. See **`__anext__()`** for details. This slot may be set to `NULL`.

sendfunc PyAsyncMethods.am_send

The corresponding slot ID `Py_am_send` is part of the ABI Estável desde a versão 3.10.

The signature of this function is:

```
PySendResult am_send(PyObject *self, PyObject *arg, PyObject **res
```

See **`PyIter_Send()`** for details. This slot may be set to `NULL`.

Adicionado na versão 3.10.

Slot Type typedefs

typedef PyObject *(*allocfunc)(PyTypeObject *cls, Py_ssize_t nitems)

Parte da ABI Estável.

The purpose of this function is to separate memory allocation from memory initialization. It should return a pointer to a block of memory of adequate length for the instance, suitably aligned, and initialized to zeros, but with **`ob_refcnt`** set to `1` and **`ob_type`** set to the type argument. If the type's **`tp_itemsize`** is non-zero, the object's **`ob_size`** field should be initialized to *nitems* and the length of the allocated memory block should be `tp_basicsize + nitems*tp_itemsize`, rounded up to a multiple of `sizeof(void*)`; otherwise, *nitems* is not used and the length of the block should be **`tp_basicsize`**.

This function should not do any other instance initialization, not even to allocate additional memory; that should be done by **`tp_new`**.

typedef void (*destructor)(PyObject*)

Parte da ABI Estável.

typedef void (*freefunc)(void*)

See **`tp_free`**.

typedef PyObject *(*newfunc)(PyTypeObject*, PyObject*, PyObject*)

Parte da ABI Estável.

See **`tp_new`**.

typedef int (*initproc)(PyObject*, PyObject*, PyObject*)

Parte da ABI Estável.

See **`tp_init`**.

typedef PyObject *(*reprfunc)(PyObject*)

Parte da ABI Estável.

See **tp_repr**.

typedef PyObject *(*getattrfunc)(PyObject *self, char *attr)

Parte da ABI Estável.

Return the value of the named attribute for the object.

typedef int (*setattrfunc)(PyObject *self, char *attr, PyObject *value)

Parte da ABI Estável.

Set the value of the named attribute for the object. The value argument is set to `NULL` to delete the attribute.

typedef PyObject *(*getattrofunc)(PyObject *self, PyObject *attr)

Parte da ABI Estável.

Return the value of the named attribute for the object.

See **tp_getattro**.

typedef int (*setattrofunc)(PyObject *self, PyObject *attr, PyObject *value)

Parte da ABI Estável.

Set the value of the named attribute for the object. The value argument is set to `NULL` to delete the attribute.

See **tp_setattro**.

typedef PyObject *(*descrgetfunc)(PyObject*, PyObject*, PyObject*)

Parte da ABI Estável.

See **tp_descr_get**.

typedef int (*descrsetfunc)(PyObject*, PyObject*, PyObject*)

Parte da ABI Estável.

See **tp_descr_set**.

typedef Py_hash_t (*hashfunc)(PyObject*)

Parte da ABI Estável.

See **tp_hash**.

typedef PyObject *(*richcmpfunc)(PyObject*, PyObject*, int)

Parte da ABI Estável.

See **tp_richcompare**.

typedef PyObject *(*getiterfunc)(PyObject*)

Parte da ABI Estável.

See `tp_iter`.

typedef PyObject *(*iternextfunc)(PyObject*)

Parte da ABI Estável.

See `tp_iternext`.

typedef Py_ssize_t (*lenfunc)(PyObject*)

Parte da ABI Estável.

typedef int (*getbufferproc)(PyObject*, Py_buffer*, int)

Parte da ABI Estável desde a versão 3.12.

typedef void (*releasebufferproc)(PyObject*, Py_buffer*)

Parte da ABI Estável desde a versão 3.12.

typedef PyObject *(*unaryfunc)(PyObject*)

Parte da ABI Estável.

typedef PyObject *(*binaryfunc)(PyObject*, PyObject*)

Parte da ABI Estável.

typedef PySendResult (*sendfunc)(PyObject*, PyObject*, PyObject**)

See `am_send`.

typedef PyObject *(*ternaryfunc)(PyObject*, PyObject*, PyObject*)

Parte da ABI Estável.

typedef PyObject *(*ssizeargfunc)(PyObject*, Py_ssize_t)

Parte da ABI Estável.

typedef int (*ssizeobjargproc)(PyObject*, Py_ssize_t, PyObject*)

Parte da ABI Estável.

typedef int (*objobjproc)(PyObject*, PyObject*)

Parte da ABI Estável.

typedef int (*objobjargproc)(PyObject*, PyObject*, PyObject*)

Parte da ABI Estável.

Exemplos

The following are simple examples of Python type definitions. They include common usage you may encounter. Some demonstrate tricky corner cases. For more examples, practical info, and a tutorial, see Definindo Tipos de Extensão: Tutorial and Defining Extension Types: Assorted Topics.

A basic static type:

```
typedef struct {
    PyObject_HEAD
    const char *data;
} MyObject;

static PyTypeObject MyObject_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "mymod.MyObject",
    .tp_basicsize = sizeof(MyObject),
    .tp_doc = PyDoc_STR("My objects"),
    .tp_new = myobj_new,
    .tp_dealloc = (destructor)myobj_dealloc,
    .tp_repr = (reprfunc)myobj_repr,
};
```

You may also find older code (especially in the CPython code base) with a more verbose initializer:

```
static PyTypeObject MyObject_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    "mymod.MyObject",               /* tp_name */
    sizeof(MyObject),               /* tp_basicsize */
    0,                              /* tp_itemsize */
    (destructor)myobj_dealloc,      /* tp_dealloc */
    0,                              /* tp_vectorcall_offset */
    0,                              /* tp_getattr */
    0,                              /* tp_setattr */
    0,                              /* tp_as_async */
    (reprfunc)myobj_repr,           /* tp_repr */
    0,                              /* tp_as_number */
    0,                              /* tp_as_sequence */
    0,                              /* tp_as_mapping */
    0,                              /* tp_hash */
    0,                              /* tp_call */
    0,                              /* tp_str */
```

```
    0,                              /* tp_getattro */
    0,                              /* tp_setattro */
    0,                              /* tp_as_buffer */
    0,                              /* tp_flags */
    PyDoc_STR("My objects"),        /* tp_doc */
    0,                              /* tp_traverse */
    0,                              /* tp_clear */
    0,                              /* tp_richcompare */
    0,                              /* tp_weaklistoffset */
    0,                              /* tp_iter */
    0,                              /* tp_iternext */
    0,                              /* tp_methods */
    0,                              /* tp_members */
    0,                              /* tp_getset */
    0,                              /* tp_base */
    0,                              /* tp_dict */
    0,                              /* tp_descr_get */
    0,                              /* tp_descr_set */
    0,                              /* tp_dictoffset */
    0,                              /* tp_init */
    0,                              /* tp_alloc */
    myobj_new,                      /* tp_new */
};
```

A type that supports weakrefs, instance dicts, and hashing:

```
typedef struct {
    PyObject_HEAD
    const char *data;
} MyObject;

static PyTypeObject MyObject_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "mymod.MyObject",
    .tp_basicsize = sizeof(MyObject),
    .tp_doc = PyDoc_STR("My objects"),
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_BASETYPE |
         Py_TPFLAGS_HAVE_GC | Py_TPFLAGS_MANAGED_DICT |
         Py_TPFLAGS_MANAGED_WEAKREF,
    .tp_new = myobj_new,
    .tp_traverse = (traverseproc)myobj_traverse,
    .tp_clear = (inquiry)myobj_clear,
    .tp_alloc = PyType_GenericNew,
    .tp_dealloc = (destructor)myobj_dealloc,
    .tp_repr = (reprfunc)myobj_repr,
    .tp_hash = (hashfunc)myobj_hash,
```

```
    .tp_richcompare = PyBaseObject_Type.tp_richcompare,
};
```

A str subclass that cannot be subclassed and cannot be called to create instances (e.g. uses a separate factory func) using **Py_TPFLAGS_DISALLOW_INSTANTIATION** flag:

```
typedef struct {
    PyUnicodeObject raw;
    char *extra;
} MyStr;

static PyTypeObject MyStr_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "mymod.MyStr",
    .tp_basicsize = sizeof(MyStr),
    .tp_base = NULL,  // set to &PyUnicode_Type in module init
    .tp_doc = PyDoc_STR("my custom str"),
    .tp_flags = Py_TPFLAGS_DEFAULT | Py_TPFLAGS_DISALLOW_INSTANTIATIC
    .tp_repr = (reprfunc)myobj_repr,
};
```

The simplest static type with fixed-length instances:

```
typedef struct {
    PyObject_HEAD
} MyObject;

static PyTypeObject MyObject_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "mymod.MyObject",
};
```

The simplest static type with variable-length instances:

```
typedef struct {
    PyObject_VAR_HEAD
    const char *data[1];
} MyObject;

static PyTypeObject MyObject_Type = {
    PyVarObject_HEAD_INIT(NULL, 0)
    .tp_name = "mymod.MyObject",
    .tp_basicsize = sizeof(MyObject) - sizeof(char *),
```

```
    .tp_itemsize = sizeof(char *),
};
```

Suporte a Coleta Cíclica de Lixo

O suporte do Python para detectar e coletar o lixo, que envolve referencias circulares, requer suporte dos tipos de objetos que são “contêineres” para outros objetos que também podem ser contêineres. Tipos que não armazenam referências a outros tipos de objetos, ou que apenas armazenam referências a tipos atômicos (como números ou strings), não precisam fornecer nenhum suporte explicito para coleta de lixo.

To create a container type, the `tp_flags` field of the type object must include the `Py_TPFLAGS_HAVE_GC` and provide an implementation of the `tp_traverse` handler. If instances of the type are mutable, a `tp_clear` implementation must also be provided.

`Py_TPFLAGS_HAVE_GC`

Objetos com esse tipo de sinalizador definido devem estar em conformidade com regras documentadas aqui. Por conveniência esses objetos serão referenciados como objetos de contêiner.

Construtores para tipos de contêiner devem obedecer a duas regras:

1. The memory for the object must be allocated using `PyObject_GC_New` or `PyObject_GC_NewVar`.
2. Uma vez que todos os campos que podem conter referências a outros contêineres foram inicializados, deve-se chamar `PyObject_GC_Track()`.

Da mesma forma, o desalocador para o objeto deve estar em conformidade com regras semelhantes:

1. Antes que os campos que fazer referência a outros contêineres sejam invalidados, `PyObject_GC_UnTrack()` deve ser chamado.

2. A memória destinada ao objeto deve ser desalocada usando `PyObject_GC_Del()`.

Aviso

If a type adds the Py_TPFLAGS_HAVE_GC, then it *must* implement at least a `tp_traverse` handler or explicitly use one from its subclass or subclasses.

When calling `PyType_Ready()` or some of the APIs that indirectly call it like `PyType_FromSpecWithBases()` or `PyType_FromSpec()` the interpreter will automatically populate the `tp_flags`, `tp_traverse` and `tp_clear` fields if the type inherits from a class that implements the garbage collector protocol and the child class does *not* include the `Py_TPFLAGS_HAVE_GC` flag.

PyObject_GC_New(TYPE, typeobj)

Analogous to `PyObject_New` but for container objects with the `Py_TPFLAGS_HAVE_GC` flag set.

Do not call this directly to allocate memory for an object; call the type's `tp_alloc` slot instead.

Ao preencher o slot `tp_alloc` de um tipo, `PyType_GenericAlloc()` é preferível a uma função personalizada que simplesmente chama esta macro.

Memory allocated by this macro must be freed with `PyObject_GC_Del()` (usually called via the object's `tp_free` slot).

Ver também

- `PyObject_GC_Del()`
- `PyObject_New`
- `PyType_GenericAlloc()`
- `tp_alloc`

PyObject_GC_NewVar(TYPE, typeobj, size)

Analogous to `PyObject_NewVar` but for container objects with the `Py_TPFLAGS_HAVE_GC` flag set.

Do not call this directly to allocate memory for an object; call the type's `tp_alloc` slot instead.

Ao preencher o slot `tp_alloc` de um tipo, `PyType_GenericAlloc()` é preferível a uma função personalizada que simplesmente chama esta macro.

Memory allocated by this macro must be freed with `PyObject_GC_Del()` (usually called via the object's `tp_free` slot).

Ver também

- `PyObject_GC_Del()`
- `PyObject_NewVar`
- `PyType_GenericAlloc()`
- `tp_alloc`

PyObject *PyUnstable_Object_GC_NewWithExtraData(PyTypeObject *type, size_t extra_size)

Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.

Analogous to `PyObject_GC_New` but allocates *extra_size* bytes at the end of the object (at offset `tp_basicsize`). The allocated memory is initialized to zeros, except for the `Python object header`.

The extra data will be deallocated with the object, but otherwise it is not managed by Python.

Memory allocated by this function must be freed with `PyObject_GC_Del()` (usually called via the object's `tp_free` slot).

Aviso

The function is marked as unstable because the final mechanism for reserving extra data after an instance is not yet decided. For allocating a variable number of fields, prefer using `PyVarObject` and `tp_itemsize` instead.

Adicionado na versão 3.12.

PyObject_GC_Resize(TYPE, op, newsize)

Resize an object allocated by `PyObject_NewVar`. Returns the resized object of type `TYPE*` (refers to any C type) or `NULL` on failure.

op must be of type PyVarObject* and must not be tracked by the collector yet. *newsize* must be of type `Py_ssize_t`.

void PyObject_GC_Track(PyObject *op)

Parte da ABI Estável.

Adds the object *op* to the set of container objects tracked by the collector. The collector can run at unexpected times so objects must be valid while being tracked. This should be called once all the fields followed by the `tp_traverse` handler become valid, usually near the end of the constructor.

int PyObject_IS_GC(PyObject *obj)

Returns non-zero if the object implements the garbage collector protocol, otherwise returns 0.

The object cannot be tracked by the garbage collector if this function returns 0.

int PyObject_GC_IsTracked(PyObject *op)

Parte da ABI Estável desde a versão 3.9.

Returns 1 if the object type of *op* implements the GC protocol and *op* is being currently tracked by the garbage collector and 0 otherwise.

This is analogous to the Python function `gc.is_tracked()`.

Adicionado na versão 3.9.

int PyObject_GC_IsFinalized(PyObject *op)

Parte da ABI Estável desde a versão 3.9.
Returns 1 if the object type of *op* implements the GC protocol and *op* has been already finalized by the garbage collector and 0 otherwise.

This is analogous to the Python function `gc.is_finalized()`.

Adicionado na versão 3.9.

void PyObject_GC_Del(void *op)

Parte da ABI Estável.
Releases memory allocated to an object using `PyObject_GC_New` or `PyObject_GC_NewVar`.

Do not call this directly to free an object's memory; call the type's `tp_free` slot instead.

Do not use this for memory allocated by `PyObject_New`, `PyObject_NewVar`, or related allocation functions; use `PyObject_Free()` instead.

Ver também

- `PyObject_Free()` is the non-GC equivalent of this function.
- `PyObject_GC_New`
- `PyObject_GC_NewVar`
- `PyType_GenericAlloc()`
- `tp_free`

void PyObject_GC_UnTrack(void *op)

Parte da ABI Estável.
Remove the object *op* from the set of container objects tracked by the collector. Note that `PyObject_GC_Track()` can be called again on this object to add it back to the set of tracked objects. The deallocator

(**tp_dealloc** handler) should call this for the object before any of the fields used by the **tp_traverse** handler become invalid.

Alterado na versão 3.8: The **_PyObject_GC_TRACK()** and **_PyObject_GC_UNTRACK()** macros have been removed from the public C API.

The **tp_traverse** handler accepts a function parameter of this type:

typedef int (*visitproc)(PyObject *object, void *arg)

Parte da ABI Estável.

Type of the visitor function passed to the **tp_traverse** handler. The function should be called with an object to traverse as *object* and the third parameter to the **tp_traverse** handler as *arg*. The Python core uses several visitor functions to implement cyclic garbage detection; it's not expected that users will need to write their own visitor functions.

The **tp_clear** handler must be of the **inquiry** type, or `NULL` if the object is immutable.

typedef int (*inquiry)(PyObject *self)

Parte da ABI Estável.

Drop references that may have created reference cycles. Immutable objects do not have to define this method since they can never directly create reference cycles. Note that the object must still be valid after calling this method (don't just call **Py_DECREF()** on a reference). The collector will call this method if it detects that this object is involved in a reference cycle.

Traversal

The **tp_traverse** handler must have the following type:

typedef int (*traverseproc)(PyObject *self, visitproc visit, void *arg)

Parte da ABI Estável.

Traversal function for a garbage-collected object, used by the garbage collector to detect reference cycles. Implementations must call the *visit*

function for each object directly contained by *self*, with the parameters to *visit* being the contained object and the *arg* value passed to the handler. The *visit* function must not be called with a `NULL` object argument. If *visit* returns a non-zero value, that value should be returned immediately.

A typical **`tp_traverse`** function calls the **`Py_VISIT()`** convenience macro on each of the instance's members that are Python objects that the instance owns. For example, this is a (slightly outdated) traversal function for the **`threading.local`** class:

```
static int
local_traverse(PyObject *op, visitproc visit, void *arg)
{
    localobject *self = (localobject *) op;
    Py_VISIT(Py_TYPE(self));
    Py_VISIT(self->args);
    Py_VISIT(self->kw);
    Py_VISIT(self->dict);
    return 0;
}
```

Nota

`Py_VISIT()` requires the *visit* and *arg* parameters to **`local_traverse()`** to have these specific names; don't name them just anything.

Instances of heap-allocated types hold a reference to their type. Their traversal function must therefore visit the type:

```
Py_VISIT(Py_TYPE(self));
```

Alternately, the type may delegate this responsibility by calling `tp_traverse` of a heap-allocated superclass (or another heap-allocated type, if applicable). If they do not, the type object may not be garbage-collected.

If the **`Py_TPFLAGS_MANAGED_DICT`** bit is set in the **`tp_flags`** field, the traverse function must call **`PyObject_VisitManagedDict()`** like this:

```
int err = PyObject_VisitManagedDict((PyObject*)self, visit, a
if (err) {
    return err;
}
```

Only the members that the instance *owns* (by having strong references to them) must be visited. For instance, if an object supports weak references via the `tp_weaklist` slot, the pointer supporting the linked list (what *tp_weaklist* points to) must **not** be visited as the instance does not directly own the weak references to itself.

The traversal function has a limitation:

Aviso

The traversal function must not have any side effects. Implementations may not modify the reference counts of any Python objects nor create or destroy any Python objects, directly or indirectly.

This means that *most* Python C API functions may not be used, since they can raise a new exception, return a new reference to a result object, have internal logic that uses side effects. Also, unless documented otherwise, functions that happen to not have side effects may start having them in future versions, without warning.

For a list of safe functions, see a separate section below.

Nota

The `Py_VISIT()` call may be skipped for those members that provably cannot participate in reference cycles. In the `local_traverse` example above, there is also a `self->key` member, but it can only be `NULL` or a Python string and therefore cannot be part of a reference cycle.

On the other hand, even if you know a member can never be part of a cycle, as a debugging aid you may want to visit it anyway just so the `gc` module's `get_referents()` function will include it.

Nota

The **tp_traverse** function can be called from any thread.

Detalhes da implementação do CPython: Garbage collection is a "stop-the-world" operation: even in free threading builds, only one thread state is attached when **tp_traverse** handlers run.

Alterado na versão 3.9: Heap-allocated types are expected to visit `Py_TYPE(self)` in `tp_traverse`. In earlier versions of Python, due to bug 40217 [https://bugs.python.org/issue40217], doing this may lead to crashes in subclasses.

To simplify writing **tp_traverse** handlers, a **Py_VISIT()** macro is provided. In order to use this macro, the **tp_traverse** implementation must name its arguments exactly *visit* and *arg*:

Py_VISIT(o)

If the PyObject* *o* is not `NULL`, call the *visit* callback, with arguments *o* and *arg*. If *visit* returns a non-zero value, then return it.

This corresponds roughly to:

```
#define Py_VISIT(o)                                \
   if (op) {                                       \
      int visit_result = visit(o, arg);            \
      if (visit_result != 0) {                     \
         return visit_result;                      \
      }                                            \
   }
```

Traversal-safe functions

The following functions and macros are safe to use in a **tp_traverse** handler:

- the *visit* function passed to `tp_traverse`
- **Py_VISIT()**

- **Py_SIZE()**
- **Py_TYPE()**: if called from a **tp_traverse** handler, **Py_TYPE()**'s result will be valid for the duration of the handler call
- **PyObject_VisitManagedDict()**
- **PyObject_TypeCheck()**, **PyType_IsSubtype()**, **PyType_HasFeature()**
- `Py<type>_Check` and `Py<type>_CheckExact` – for example, **PyTuple_Check()**
- "DuringGC" functions

"DuringGC" functions

The following functions should *only* be used in a **tp_traverse** handler; calling them in other contexts may have unintended consequences.

These functions act like their counterparts without the `_DuringGC` suffix, but they are guaranteed to not have side effects, they do not set an exception on failure, and they return/set borrowed references as detailed in the individual documentation.

Note that these functions may fail (return `NULL` or `-1`), but as they do not set an exception, no error information is available. In some cases, failure is not distinguishable from a successful `NULL` result.

void *PyObject_GetTypeData_DuringGC(PyObject *o, PyTypeObject *cls)
void *PyObject_GetItemData_DuringGC(PyObject *o)
void *PyType_GetModuleState_DuringGC(PyTypeObject *type)
void *PyModule_GetState_DuringGC(PyObject *module)
int PyModule_GetToken_DuringGC(PyObject *module, void **result)

Parte da ABI Estável desde a versão 3.15.
See "DuringGC" functions for common information.

Adicionado na versão 3.15.

Ver também

`PyObject_GetTypeData()`, `PyObject_GetItemData()`, `PyType_GetModuleState()`, `PyModule_GetState()`, `PyModule_GetToken()`, `PyType_GetBaseByToken()`

int PyType_GetBaseByToken_DuringGC(PyTypeObject *type, void *tp_token, PyTypeObject **result)

Parte da ABI Estável desde a versão 3.15.
See "DuringGC" functions for common information.

Sets **result* to a borrowed reference rather than a strong one. The reference is valid for the duration of the **`tp_traverse`** handler call.

Adicionado na versão 3.15.

Ver também

`PyType_GetBaseByToken()`

PyObject *PyType_GetModule_DuringGC(PyTypeObject *type)
PyObject *PyType_GetModuleByToken_DuringGC(PyTypeObject *type, const void *mod_token)

Retorna valor: Referência emprestada. Parte da ABI Estável desde a versão 3.15.
See "DuringGC" functions for common information.

These functions return a borrowed reference, which is valid for the duration of the **`tp_traverse`** handler call.

Adicionado na versão 3.15.

Ver também

`PyType_GetModule()`, `PyType_GetModuleByToken()`

Controlando o estado do coletor de lixo

The C-API provides the following functions for controlling garbage collection runs.

Py_ssize_t PyGC_Collect(void)

Parte da ABI Estável.
Perform a full garbage collection, if the garbage collector is enabled. (Note that `gc.collect()` runs it unconditionally.)

Returns the number of collected + unreachable objects which cannot be collected. If the garbage collector is disabled or already collecting, returns `0` immediately. Errors during garbage collection are passed to `sys.unraisablehook`. This function does not raise exceptions.

int PyGC_Enable(void)

Parte da ABI Estável desde a versão 3.10.
Enable the garbage collector: similar to `gc.enable()`. Returns the previous state, 0 for disabled and 1 for enabled.

Adicionado na versão 3.10.

int PyGC_Disable(void)

Parte da ABI Estável desde a versão 3.10.
Disable the garbage collector: similar to `gc.disable()`. Returns the previous state, 0 for disabled and 1 for enabled.

Adicionado na versão 3.10.

int PyGC_IsEnabled(void)

Parte da ABI Estável desde a versão 3.10.
Query the state of the garbage collector: similar to `gc.isenabled()`. Returns the current state, 0 for disabled and 1 for enabled.

Adicionado na versão 3.10.

Querying Garbage Collector State

The C-API provides the following interface for querying information about the garbage collector.

void PyUnstable_GC_VisitObjects(gcvisitobjects_t callback, void *arg)

> *Esta é uma API Instável. Isso pode se alterado sem aviso em lançamentos menores.*

Run supplied *callback* on all live GC-capable objects. *arg* is passed through to all invocations of *callback*.

> **Aviso**
>
> If new objects are (de)allocated by the callback it is undefined if they will be visited.
>
> Garbage collection is disabled during operation. Explicitly running a collection in the callback may lead to undefined behaviour e.g. visiting the same objects multiple times or not at all.

Adicionado na versão 3.12.

typedef int (*gcvisitobjects_t)(PyObject *object, void *arg)

Type of the visitor function to be passed to `PyUnstable_GC_VisitObjects()`. *arg* is the same as the *arg* passed to `PyUnstable_GC_VisitObjects`. Return `1` to continue iteration, return `0` to stop iteration. Other return values are reserved for now so behavior on returning anything else is undefined.

Adicionado na versão 3.12.

API e Versionamento de ABI

Constantes de versão de tempo de construção

O CPython expõe seu número de versão nas seguintes macros. Note que estes correspondem ao código da versão com a qual está **construída**. Veja `Py_Version` para a versão usada em **tempo de execução**.

Veja C API and ABI Stability para uma discussão da estabilidade da API e ABI através das versões.

PY_MAJOR_VERSION
: O `3` em `3.4.1a2`.

PY_MINOR_VERSION
: O `4` em `3.4.1a2`.

PY_MICRO_VERSION
: O `1` em `3.4.1a2`.

PY_RELEASE_LEVEL
: O `a` em `3.4.1a2`. Isto pode ser `0xA` para alfa, `0xB` para beta, `0xC` para o candidato a lançamento ou `0xF` para final.

  Para uma maior clareza, os valores estão disponíveis como macros: **`PY_RELEASE_LEVEL_ALPHA`** (`0xA`), **`PY_RELEASE_LEVEL_BETA`** (`0xB`), **`PY_RELEASE_LEVEL_GAMMA`** (`0xC`) e **`PY_RELEASE_LEVEL_FINAL`** (`0xF`).

PY_RELEASE_SERIAL
: O `2` em `3.4.1a2`. Zero para os lançamentos finais.

PY_VERSION_HEX
: O número da versão do Python codificado em um único inteiro. Consulte **`Py_PACK_FULL_VERSION()`** para obter detalhes sobre a

codificação.

Use isso para comparações numéricas como, por exemplo, `#if PY_VERSION_HEX >= ....`

PY_VERSION
A versão do Python como uma string, por exemplo, `"3.4.1a2"`.

Essas macros estão definidas em Include/patchlevel.h [https://github.com/python/cpython/tree/3.15/Include/patchlevel.h].

Versão de tempo de execução

const unsigned long Py_Version
Parte da ABI Estável desde a versão 3.11.
O número da versão do tempo de execução do Python codificado em um único inteiro constante. Consulte **`Py_PACK_FULL_VERSION()`** para obter detalhes sobre a codificação. Contém a versão do Python usada em tempo de execução.

Use isso para comparações numéricas como, por exemplo, `if (Py_Version >= ...)`.

Adicionado na versão 3.11.

Macros de empacotamento de bits

uint32_t Py_PACK_FULL_VERSION(int major, int minor, int micro, int release_level, int release_serial)
Parte da ABI Estável desde a versão 3.14.
Retorna a versão fornecida, codificada como um único inteiro de 32 bits com a seguinte estrutura:

Argumento	Nº de bits	Máscara de bits	Deslocamento de bits	Exemplo de valores

Argumento	Nº de bits	Máscara de bits	Deslocamento de bits	Exemplo de valores	
				`3.4.1a2`	`3.10.0`
major	8	`0xFF000000`	24	`0x03`	`0x03`
minor	8	`0x00FF0000`	16	`0x04`	`0x0A`
micro	8	`0x0000FF00`	8	`0x01`	`0x00`
release_level	4	`0x000000F0`	4	`0xA`	`0xF`
release_serial	4	`0x0000000F`	0	`0x2`	`0x0`

Por exemplo:

Versão	Argumentos de `Py_PACK_FULL_VERSION`	Versão codificada
`3.4.1a2`	`(3, 4, 1, 0xA, 2)`	`0x030401a2`
`3.10.0`	`(3, 10, 0, 0xF, 0)`	`0x030a00f0`

Bits fora do intervalo nos argumentos são ignorados. Ou seja, a macro pode ser definida como:

```
#ifndef Py_PACK_FULL_VERSION
#define Py_PACK_FULL_VERSION(X, Y, Z, LEVEL, SERIAL) ( \
   (((X) & 0xff) << 24) |                              \
   (((Y) & 0xff) << 16) |                              \
   (((Z) & 0xff) << 8) |                               \
   (((LEVEL) & 0xf) << 4) |                            \
   (((SERIAL) & 0xf) << 0))
#endif
```

`Py_PACK_FULL_VERSION` é principalmente uma macro, destinada ao uso em diretivas `#if`, mas também está disponível como uma função exportada.

Adicionado na versão 3.14.

uint32_t Py_PACK_VERSION(int major, int minor)

Parte da ABI Estável desde a versão 3.14.

Equivalente a `Py_PACK_FULL_VERSION(major, minor, 0, 0, 0)`. O resultado não corresponde a nenhuma versão do Python, mas é útil em comparações numéricas.

Adicionado na versão 3.14.

API C de monitoramento

Adicionada na versão 3.13.

Uma extensão pode precisar interagir com o sistema de monitoramento de eventos. É possível registrar funções de retorno e se inscrever para receber eventos através da API Python exposta em `sys.monitoring`.

Gerando eventos de execução

As funções abaixo permitem que extensões dispararem eventos de monitoramento emulando a execução de código Python. Cada uma dessas funções recebe uma estrutura `PyMonitoringState` que contém informação concisa sobre o estado de ativação de eventos, bem como os argumentos dos eventos, que incluem um `PyObject*` representando o objeto de código, a posição da instrução no bytecode, e, em alguns casos, argumentos adicionais específicos para o evento (veja `sys.monitoring` para detalhes sobre as assinaturas das funções de retorno de diferentes eventos). O argumento `codelike` deve ser uma instância da classe `types.CodeType` ou de um tipo que a emule.

A VM desabilita o rastreamento quando dispara um evento, de forma que não há necessidade do código do usuário fazer isso.

Funções de monitoramento não devem ser chamadas com uma exceção definida, exceto as marcadas abaixo como funções que trabalham com a exceção atual.

type PyMonitoringState

Representação do estado de um tipo de evento. Alocada pelo usuário, enquanto o seu conteúdo é mantido pelas funções da API de monitoramento descritas abaixo.

Todas as funções abaixo retornam 0 para indicar sucesso e -1 (com uma exceção definida) para indicar erro.

Veja `sys.monitoring` para descrições dos eventos.

int PyMonitoring_FirePyStartEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `PY_START`.

int PyMonitoring_FirePyResumeEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `PY_RESUME`.

int PyMonitoring_FirePyReturnEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *retval)

Dispara um evento `PY_RETURN`.

int PyMonitoring_FirePyYieldEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *retval)

Dispara um evento `PY_YIELD`.

int PyMonitoring_FireCallEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *callable, PyObject *arg0)

Dispara um evento `CALL`.

int PyMonitoring_FireLineEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, int lineno)

Dispara um evento `LINE`.

int PyMonitoring_FireJumpEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *target_offset)

Dispara um evento `JUMP`.

int PyMonitoring_FireBranchLeftEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *target_offset)

Dispara um evento `BRANCH_LEFT`.

int PyMonitoring_FireBranchRightEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *target_offset)

Dispara um evento `BRANCH_RIGHT`.

int PyMonitoring_FireCReturnEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *retval)

Dispara um evento `C_RETURN`.

int PyMonitoring_FirePyThrowEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `PY_THROW` com a exceção atual (conforme retornada por **`PyErr_GetRaisedException()`**).

int PyMonitoring_FireRaiseEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `RAISE` com a exceção atual (conforme retornada por **`PyErr_GetRaisedException()`**).

int PyMonitoring_FireCRaiseEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `C_RAISE` com a exceção atual (conforme retornada por **`PyErr_GetRaisedException()`**).

int PyMonitoring_FireReraiseEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `RERAISE` com a exceção atual (conforme retornada por **`PyErr_GetRaisedException()`**).

int PyMonitoring_FireExceptionHandledEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `EXCEPTION_HANDLED` com a exceção atual (conforme retornada por **`PyErr_GetRaisedException()`**).

int PyMonitoring_FirePyUnwindEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset)

Dispara um evento `PY_UNWIND` com a exceção atual (conforme retornada por **`PyErr_GetRaisedException()`**).

int PyMonitoring_FireStopIterationEvent(PyMonitoringState *state, PyObject *codelike, int32_t offset, PyObject *value)

Dispara um evento `STOP_ITERATION`. Se `value` for uma instância de **`StopIteration`**, ele é usado. Caso contrário, uma nova instância de **`StopIteration`** é criada com `value` como o argumento.

Gerenciando o Estado de um Monitoramento

Estados de monitoramento podem ser gerenciados com a ajuda de escopos de monitoramento. Um escopo corresponderia tipicamente a um função Python.

int PyMonitoring_EnterScope(PyMonitoringState *state_array, uint64_t *version, const uint8_t *event_types, Py_ssize_t length)

Insere um escopo monitorado. `event_types` é um vetor de IDs de eventos para eventos que podem ser disparados do escopo. Por exemplo, a ID de um evento `PY_START` é o valor `PY_MONITORING_EVENT_PY_START`, que é numericamente igual ao logaritmo de base 2 de `sys.monitoring.events.PY_START`. `state_array` é um vetor com uma entrada de estado de monitoramento para cada evento em `event_types`, é alocado pelo usuário, mas preenchido por **`PyMonitoring_EnterScope()`** com informações sobre o estado de ativação do evento. O tamanho de `event_types` (e, portanto, também de `state_array`) é fornecido em `length`.

O argumento `version` é um ponteiro para um valor que deve ser alocado pelo usuário junto com `state_array` e inicializado como 0, e então definido somente pelo próprio **`PyMonitoring_EnterScope()`**. Ele permite que esta função determine se os estados de eventos mudaram desde a chamada anterior, e retorne rapidamente se não mudaram.

Os escopos mencionados aqui são escopos lexicais: uma função, classe ou método. **`PyMonitoring_EnterScope()`** deve ser chamada sempre que o escopo lexical for inserido. Os escopos podem ser inseridos novamente, reutilizando o mesmo *state_array* e *version*, em situações como ao emular uma função Python recursiva. Quando a execução de um código semelhante é pausada, como ao emular um gerador, o escopo precisa ser encerrado e inserido novamente.

As macros para *event_types* são:

Macro	Evento
PY_MONITORING_EVENT_BRANCH_LEFT	**`BRANCH_LEFT`**
PY_MONITORING_EVENT_BRANCH_RIGHT	**`BRANCH_RIGHT`**
PY_MONITORING_EVENT_CALL	**`CALL`**
PY_MONITORING_EVENT_C_RAISE	**`C_RAISE`**
PY_MONITORING_EVENT_C_RETURN	**`C_RETURN`**

Macro	Evento
PY_MONITORING_EVENT_EXCEPTION_HANDLED	`EXCEPTION_HANDLED`
PY_MONITORING_EVENT_INSTRUCTION	`INSTRUCTION`
PY_MONITORING_EVENT_JUMP	`JUMP`
PY_MONITORING_EVENT_LINE	`LINE`
PY_MONITORING_EVENT_PY_RESUME	`PY_RESUME`
PY_MONITORING_EVENT_PY_RETURN	`PY_RETURN`
PY_MONITORING_EVENT_PY_START	`PY_START`
PY_MONITORING_EVENT_PY_THROW	`PY_THROW`
PY_MONITORING_EVENT_PY_UNWIND	`PY_UNWIND`
PY_MONITORING_EVENT_PY_YIELD	`PY_YIELD`
PY_MONITORING_EVENT_RAISE	`RAISE`
PY_MONITORING_EVENT_RERAISE	`RERAISE`
PY_MONITORING_EVENT_STOP_ITERATION	`STOP_ITERATION`

int PyMonitoring_ExitScope(void)

Sai do último escopo no qual se entrou com **`PyMonitoring_EnterScope()`**.

int PY_MONITORING_IS_INSTRUMENTED_EVENT(uint8_t ev)

Retorna verdadeiro se o evento correspondente ao ID do evento *ev* for um evento local.

Adicionado na versão 3.13.

Suavemente descontinuado desde a versão 3.14.

Instalando módulos Python

Como um projeto popular de desenvolvimento de código aberto, Python tem uma comunidade de apoio ativa de colaboradores e usuários, que também fazem o seu software disponível para outros desenvolvedores de Python para usar sob os termos da licença de código aberto.

Isso permite aos usuários Python compartilhar e colaborar efetivamente, se beneficiando das soluções que outros já tenham criado para os problemas mais comuns (em alguns casos até mesmo os raros), bem como potencialmente contribuindo com suas próprias soluções para o conjunto de soluções comuns.

Este guia cobre a parte do processo de instalação. Para um guia sobre criar e compartilhar seus próprios projetos Python, confira o Guia de Usuário para Empacotamento de Python [https://packaging.python.org/en/latest/tutorials/packaging-projects/].

Nota

Para corporações e outros usuários institucionais, esteja ciente que muitas organizações têm suas próprias políticas em relação ao uso e contribuição para o software de código aberto. Por favor, leve em conta essas políticas ao usar as ferramentas de distribuição e instalação fornecidas com o Python.

Termos chave

- **pip** é o programa de instalação preferido. Ele é incluído por padrão nos instaladores binários do Python.
- Um *ambiente virtual* é um ambiente Python semi-isolado que permite que pacotes sejam instalados para uso por uma aplicação específica, em vez de serem instaladas em todo o sistema.

- `venv` é a ferramenta padrão para criar ambientes virtuais. O padrão é instalar **pip** em todos os ambientes virtuais criados.
- `virtualenv` é uma alternativa de terceiros (e predecessor) ao `venv`.
- O Python Package Index (PyPI) [https://pypi.org] é um repositório público de pacotes licenciados como código aberto e disponíveis para uso de outros usuários Python.
- O Python Packaging Authority [https://www.pypa.io/] é o grupo de desenvolvedores e autores de documentação responsáveis pela manutenção e evolução das ferramentas de empacotamento padrão e os metadados associados e padrões de formato de arquivo. Eles mantêm uma variedade de ferramentas, documentação e rastreadores de problemas no GitHub [https://github.com/pypa].

Alterado na versão 3.5: O uso de `venv` agora é recomendado para a criação de ambientes virtuais.

Ver também

Guia de Usuário para Empacotamento de Python: Criando e usando ambientes virtuais [https://packaging.python.org/pt-br/latest/tutorials/installing-packages/#creating-virtual-environments]

Uso básico

As ferramentas de empacotamento padrão são todas projetadas para serem usadas na linha de comando.

O comando a seguir instalará a versão mais recente de um módulo e suas dependências do PyPI:

```
python -m pip install AlgumPacote
```

Nota

Para usuários POSIX (incluindo usuários macOS e Linux), os exemplos neste guia presumem o uso de um ambiente virtual.

Para usuários do Windows, os exemplos neste guia presumem que a opção de ajustar a variável de ambiente PATH do sistema foi selecionada durante a instalação do Python.

Também é possível especificar uma versão exata ou mínima diretamente na linha de comando. Ao usar operadores de comparação como >, < ou algum outro caractere especial que é interpretado pelo shell, o nome do pacote e a versão devem ser colocados entre aspas duplas:

```
python -m pip install AlgumPacote==1.0.4    # versão específica
python -m pip install "AlgumPacote>=1.0.4"  # versão mínima
```

Normalmente, se um módulo adequado já estiver instalado, tentar instalá-lo novamente não terá efeito. A atualização de módulos existentes deve ser solicitada explicitamente:

```
python -m pip install --upgrade AlgumPacote
```

Mais informações e recursos sobre o **pip** e seus recursos podem ser encontrados no Python Packaging User Guide [https://packaging.python.org].

A criação de ambientes virtuais é feita através do módulo **venv**. A instalação de pacotes em um ambiente virtual ativo usa os comandos mostrados acima.

Ver também

Guia de Usuário para Empacotamento de Python: Instalando pacotes Python de distribuição [https://packaging.python.org/pt-br/latest/tutorials/installing-packages/]

Como eu …?

Estas são respostas rápidas ou links para algumas tarefas comuns.

… instalo pacotes apenas para o usuário atual?

Passar a opção `--user` para `python -m pip install` irá instalar um pacote apenas para o usuário atual, ao invés de para todos os usuários do sistema.

… instalo pacotes científicos do Python?

Vários pacotes científicos do Python têm dependências binárias complexas e atualmente não são fáceis de instalar usando o **pip** diretamente. Frequentemente será mais fácil para os usuários instalarem esses pacotes por outros meios [https://packaging.python.org/science/] ao invés de tentar instalá-los com **pip**.

Ver também

Guia de Usuário para Empacotamento de Python: Instalando pacotes científicos [https://packaging.python.org/pt-br/latest/guides/installing-scientific-packages//]

… trabalho com várias versões do Python instaladas em paralelo?

No Linux, macOS e outros sistemas POSIX, use os comandos Python com versão em combinação com a opção `-m` para executar a cópia apropriada de **pip**:

```
python3    -m pip install AlgumPacote  # Python 3 padrão
python3.14 -m pip install AlgumPacote  # Python 3.14 especificam
```

Comandos **pip** com versão apropriada também podem estar disponíveis.

No Windows, use o iniciador Python **py** em combinação com a opção `-m`:

```
py -3    -m pip install AlgumPacote  # Python 3 padrão
py -3.14 -m pip install AlgumPacote  # Python 3.14 especificamer
```

Problemas comuns de instalação

Instalando no sistema Python no Linux

Em sistemas Linux, uma instalação Python normalmente será incluída como parte da distribuição. A instalação nesta instalação Python requer acesso root ao sistema, e pode interferir na operação do gerenciador de pacotes do sistema e outros componentes do sistema se um componente for atualizado inesperadamente usando **pip**.

Em tais sistemas, geralmente é melhor usar um ambiente virtual ou uma instalação por usuário ao instalar pacotes com **pip**.

Pip não instalado

É possível que o **pip** não seja instalado por padrão. Uma solução potencial é:

```
python -m ensurepip --default-pip
```

Existem também recursos adicionais para instalar pip. [https://packaging.python.org/en/latest/tutorials/installing-packages/#ensure-pip-setuptools-and-wheel-are-up-to-date].

Instalando extensões binárias

Antigamente, o Python dependia fortemente da distribuição baseada em código-fonte, e esperava-se que os usuários finais compilassem módulos de extensão a partir do código-fonte como parte do processo de instalação.

Com a introdução do formato binário *wheel* e a possibilidade de publicar *wheels* no PyPI, esse problema vem diminuindo, uma vez que os usuários conseguem instalar extensões pré-construídas com mais frequência, em vez de precisarem construí-las eles mesmos.

Algumas das soluções para instalar softwares científicos [https://packaging.python.org/science/] que ainda não estão disponíveis como

arquivos *wheel* pré-construídos também podem ajudar a obter outras extensões binárias sem a necessidade para construí-los localmente.

Ver também

Guia de Usuário para Empacotamento de Python: Extensões binárias
[https://packaging.python.org/pt-br/latest/guides/packaging-binary-extensions/]

Python HOWTOs

Python HOWTOs (COMOFAZER) são documentos que cobrem um tópico específico em profundidade. Modelada na coleção HOWTO do Linux Documentation Project, esta coleção é um esforço para promover uma documentação mais detalhada do que a Referência da Biblioteca do Python.

Geral:

- Uma visão geral conceitual de asyncio
- Boas práticas para anotações
- Tutorial de Argparse
- Guia de descritores
- Enum
- Programação Funcional
- Uma introdução ao módulo ipaddress
- Logging
- Livro de receitas do logging
- Regular expression HOWTO
- Técnicas de ordenação
- Unicode
- Como buscar recursos da Internet usando o pacote urllib

Desenvolvimento avançado:

- Programação em Curses com Python
- Suporte do Python para threads livres
- Suporte a extensões da API C para threads livres
- Isolando módulos de extensão
- Migrating to Stable ABI for free threading (abi3t)
- A Ordem de Resolução de Métodos do Python 2.3
- Programação de Soquetes
- Descritor de arquivo de temporizador
- Portando módulos de extensão para o Python 3

Depuração e perfilação:

- Depurando extensões de API C e internos do CPython com GDB
- Instrumentando o CPython com DTrace e SystemTap
- Python support for the perf map compatible profilers
- Protocolo de anexação de depuração remota

Uma visão geral conceitual de `asyncio`

Este artigo COMOFAZER tem como objetivo ajudá-lo a construir um modelo mental sólido de como **`asyncio`** funciona fundamentalmente, ajudando-lhe a compreender o como e o porquê por trás dos padrões recomendados.

Você pode estar curioso sobre alguns conceitos-chave de **`asyncio`**. Ao final deste artigo, você será capaz de responder confortavelmente a estas perguntas:

- O que acontece nos bastidores quando um objeto é aguardado?
- Como o **`asyncio`** diferencia uma tarefa que não precisa de tempo de CPU (como uma solicitação de rede ou leitura de arquivo) de uma tarefa que precisa (como calcular n-fatorial)?
- Como escrever uma variante assíncrona de uma operação, como uma suspensão assíncrona ou uma solicitação de banco de dados.

Ver também

- O guia [https://github.com/anordin95/a-conceptual-overview-of-asyncio/tree/main] que inspirou este artigo COMOFAZER, por Alexander Nordin.
- Esta série de tutoriais detalhados do YouTube [https://www.youtube.com/watch?v=Xbl7XjFYsN4&list=PLhNSoGM2ik6SIkVGXWBwerucXjgP1rHmB] sobre `asyncio` foi criada pelo membro da equipe principal do Python, Łukasz Langa.
- 500 linhas ou menos: um rastreador da Web com corrotinas asyncio [https://aosabook.org/en/500L/a-web-crawler-with-asyncio-coroutines.html] (em inglês) por A. Jesse Jiryu Davis e Guido van Rossum.

Uma visão geral conceitual parte 1: o alto nível

Na parte 1, abordaremos os principais blocos de construção de alto nível de **`asyncio`**: o laço de eventos, funções de corrotina, objetos de corrotina, tarefas e `await`.

Laço de eventos

Tudo no **`asyncio`** acontece em relação ao laço de eventos (*event loop*). Ele é a estrela do espetáculo, mas prefere atuar nos bastidores, gerenciando e coordenando recursos. É como um maestro de orquestra: alguns poderes lhe são explicitamente conferidos, mas grande parte de sua capacidade de realizar tarefas advém do respeito e da cooperação dos integrantes da orquestra.

Em termos mais técnicos, o laço de eventos contém uma coleção de tarefas a serem executadas. Algumas tarefas são adicionadas diretamente por você e outras indiretamente por **`asyncio`**. O laço de eventos pega uma tarefa do seu backlog de tarefas e a invoca (ou "dá a ela o controle"), semelhante a chamar uma função, e então essa tarefa é executada. Uma vez pausada ou concluída, ela retorna o controle para o laço de eventos. O laço de eventos então seleciona outra tarefa do seu pool e a invoca. Você pode *aproximadamente* pensar na coleção de tarefas como uma fila: as tarefas são adicionadas e processadas uma de cada vez, geralmente (mas nem sempre) em ordem. Esse processo se repete indefinidamente, com o laço de eventos em ciclos infinitos. Se não houver mais tarefas pendentes de execução, o laço de eventos é inteligente o suficiente para descansar e evitar o desperdício desnecessário de ciclos de CPU, e retornará quando houver mais trabalho a ser feito, tal como quando operações E/S são concluídas ou seu tempo expira.

A execução eficaz depende do bom compartilhamento e da cooperação entre as tarefas; uma tarefa gananciosa pode monopolizar o controle e deixar as outras tarefas na miséria, tornando a abordagem geral do laço de eventos inútil.

```
import asyncio

# Isso cria um laço de eventos e percorre indefinidamente
# sua coleção de trabalhos
event_loop = asyncio.new_event_loop()
event_loop.run_forever()
```

Funções assíncronas e corrotinas

Esta é uma função básica e chata do Python:

```
def hello_printer():
    print(
        "Olá, sou uma impressora humilde e simples, embora tenha
        "que preciso na vida: -- \npapel novo e meu querido e am
        "parceiro no crime, o polvo."
    )
```

Chamar uma função regular invoca sua lógica ou corpo:

```
>>> hello_printer()
Olá, sou uma impressora humilde e simples, embora tenha tudo o c
papel novo e meu querido e amado parceiro no crime, o polvo.
```

O [async def](), em oposição a um simples `def`, torna esta uma função assíncrona (ou "função de corrotina"). Chamá-la cria e retorna um objeto [corrotina]().

```
async def loudmouth_penguin(magic_number: int):
    print(
     "Eu sou um pinguim falante superespecial. Muito mais legal
     f"Aliás, meu número da sorte é: {magic_number}."
    )
```

Chamar a função assíncrona, `loudmouth_penguin`, não executa a instrução de impressão; em vez disso, cria um objeto corrotina:

```
>>> loudmouth_penguin(magic_number=3)
<coroutine object loudmouth_penguin at 0x104ed2740>
```

Os termos “função de corrotina” e “objeto corrotina” são frequentemente confundidos com corrotina. Isso pode ser confuso! Neste artigo, corrotina se refere especificamente a um objeto corrotina, ou mais precisamente, a uma instância de `types.CoroutineType` (corrotina nativa). Observe que corrotinas também podem existir como instâncias de `collections.abc.Coroutine` — uma distinção importante para a verificação de tipos.

Uma corrotina representa o corpo ou a lógica da função. Uma corrotina precisa ser iniciada explicitamente; novamente, a mera criação da corrotina não a inicia. Notavelmente, a corrotina pode ser pausada e retomada em vários pontos do corpo da função. Essa capacidade de pausar e retomar é o que permite o comportamento assíncrono!

Corrotinas e funções de corrotina foram criadas aproveitando a funcionalidade de geradores e funções geradoras. Lembre-se: uma função geradora é uma função que executa `yield`, como esta:

```
def get_random_number():
    # Este seria um gerador de número aleatório ruim!
    print("Oi")
    yield 1
    print("Olá")
    yield 7
    print("E aí")
    yield 4
    ...
```

Semelhante a uma função de corrotina, chamar uma função geradora não a executa. Em vez disso, ela cria um objeto gerador:

```
>>> get_random_number()
<generator object get_random_number at 0x1048671c0>
```

Você pode prosseguir para o próximo `yield` de um gerador usando a função embutida `next()`. Em outras palavras, o gerador é executado e, em seguida, pausado. Por exemplo:

```
>>> generator = get_random_number()
>>> next(generator)
```

```
Oi
1
>>> next(generator)
Olá
7
```

Tarefas

Em termos gerais, tarefas são corrotinas (não funções de corrotina) vinculadas a um laço de eventos. Uma tarefa também mantém uma lista de funções de retorno de chamada cuja importância ficará clara em breve, quando discutirmos **`await`**.

A criação de uma tarefa a agenda automaticamente para execução (adicionando um retorno de chamada para executá-la na lista de tarefas do laço de eventos, ou seja, coleção de tarefas). A forma recomendada é criar tarefas por meio de **`asyncio.create_task()`**.

`asyncio` associa automaticamente as tarefas ao laço de eventos. Essa associação automática foi propositalmente incorporada ao **`asyncio`** para simplificar o processo. Sem ela, você teria que controlar o objeto laço de eventos e passá-lo para qualquer função de corrotina que queira criar tarefas, adicionando código redundante ao seu projeto.

```
coroutine = loudmouth_penguin(magic_number=5)
# Isso cria um objeto Task e agenda sua execução por meio do laç
task = asyncio.create_task(coroutine)
```

Anteriormente, criamos manualmente o laço de eventos e o configuramos para ser executado indefinidamente. Na prática, é recomendado (e comum) usar **`asyncio.run()`**, que gerencia o laço de eventos e garante que a corrotina fornecida termine antes de avançar. Por exemplo, muitos programas assíncronos seguem esta configuração:

```
import asyncio

async def main():
    # Faz todo tipo de coisas malucas, selvagens e assíncronas..
    ...
```

```
if __name__ == "__main__":
    asyncio.run(main())
    # O programa não alcançará a seguinte instrução de exibição
    # até que o main() da corrotina seja finalizado.
    print("main() da corrotina concluiu!")
```

É importante estar ciente de que a tarefa em si não é adicionada ao laço de eventos, apenas um retorno de chamada para a tarefa. Isso é importante se o objeto de tarefa que você criou for coletado como lixo antes de ser chamado pelo laço de eventos. Por exemplo, considere este programa:

```
 1 async def hello():
 2     print("hello!")
 3
 4 async def main():
 5     asyncio.create_task(hello())
 6     # Outras instruções assíncronas que são executadas por
 7     # um tempo e cedem o controle ao laço de eventos...
 8     ...
 9
10 asyncio.run(main())
```

Como não há referência ao objeto tarefa criado na linha 5, ele *pode* ser coletado como lixo antes que o laço de eventos o invoque. Instruções posteriores na corrotina `main()` transferem o controle de volta para o laço de eventos para que ele possa invocar outras tarefas. Quando o laço de eventos eventualmente tenta executar a tarefa, pode falhar e descobrir que o objeto task não existe! Isso também pode acontecer mesmo que uma corrotina mantenha uma referência a uma tarefa, mas seja concluída antes que ela termine. Quando a corrotina termina, as variáveis locais saem do escopo e podem estar sujeitas à coleta de lixo. Na prática, `asyncio` e o coletor de lixo do Python trabalham arduamente para garantir que esse tipo de coisa não aconteça. Mas isso não é motivo para ser imprudente!

await

`await` é uma palavra reservada do Python comumente usada de duas maneiras diferentes:

```
await task
await coroutine
```

De maneira crucial, o comportamento de `await` depende do tipo de objeto que está sendo aguardado.

Aguardando por tarefas

Aguardar uma tarefa cederá o controle da tarefa ou corrotina atual para o laço de eventos. No processo de cessão de controle, algumas coisas importantes acontecem. Usaremos o seguinte exemplo de código para ilustrar:

```
async def plant_a_tree():
    dig_the_hole_task = asyncio.create_task(dig_the_hole())
    await dig_the_hole_task

    # Outras instruções associadas com plantar uma árvore.
    ...
```

Neste exemplo, imagine que o laço de eventos passou o controle para o início da corrotina `plant_a_tree()`. Como visto acima, a corrotina cria uma tarefa e a aguarda. A instrução `await dig_the_hole_task` adiciona um retorno de chamada (que retomará `plant_a_tree()`) à lista de retornos de chamada do objeto `dig_the_hole_task`. E então, a instrução cede o controle para o laço de eventos. Algum tempo depois, o laço de eventos passará o controle para `dig_the_hole_task` e a tarefa concluirá o que for necessário. Assim que a tarefa for concluída, ela adicionará seus vários retornos de chamada ao laço de eventos, neste caso, uma chamada para retomar `plant_a_tree()`.

De modo geral, quando a tarefa aguardada termina (`dig_the_hole_task`), a tarefa original ou corrotina (`plant_a_tree()`) é adicionada novamente à lista de tarefas do laço de eventos para ser retomada.

Este é um modelo mental básico, porém confiável. Na prática, as transferências de controle são um pouco mais complexas, mas não muito. Na parte 2, abordaremos os detalhes que tornam isso possível.

Aguardando por corrotinas

Ao contrário de tarefas, aguardar uma corrotina não devolve o controle ao laço de eventos! Envolver uma corrotina em uma tarefa primeiro e depois aguardar isso cederia o controle. O comportamento de `await coroutine` é efetivamente o mesmo que invocar uma função Python síncrona comum. Considere este programa:

```
import asyncio

async def coro_a():
   print("Sou coro_a(). Oi!")

async def coro_b():
   print("Sou coro_b(). Espero que ninguém monopolize o laço de

async def main():
   task_b = asyncio.create_task(coro_b())
   num_repeats = 3
   for _ in range(num_repeats):
      await coro_a()
   await task_b

asyncio.run(main())
```

A primeira instrução na corrotina `main()` cria `task_b` e a agenda para execução via laço de eventos. Em seguida, `coro_a()` é aguardado repetidamente. O controle nunca cede ao laço de eventos, e é por isso que vemos a saída de todas as três invocações de `coro_a()` antes da saída de `coro_b()`:

```
Sou coro_a(). Oi!
Sou coro_a(). Oi!
Sou coro_a(). Oi!
Sou coro_b(). Espero que ninguém monopolize o laço de eventos...
```

Se alterarmos `await coro_a()` para `await asyncio.create_task(coro_a())`, o comportamento muda. A corrotina `main()` cede o controle ao laço de eventos com essa instrução. O laço de

eventos então prossegue com seu backlog de trabalho, chamando `task_b` e, em seguida, a tarefa que encerra `coro_a()` antes de retomar a corrotina `main()`.

```
Sou coro_b(). Espero que ninguém monopolize o laço de eventos...
Sou coro_a(). Oi!
Sou coro_a(). Oi!
Sou coro_a(). Oi!
```

Esse comportamento de `await coroutine` pode confundir muita gente! Este exemplo destaca como usar apenas `await coroutine` pode, involuntariamente, monopolizar o controle de outras tarefas e efetivamente paralisar o laço de eventos. **`asyncio.run()`** pode ajudar a detectar tais ocorrências por meio do sinalizador `debug=True`, que habilita o modo de depuração. Entre outras coisas, ele registrará quaisquer corrotinas que monopolizem a execução por 100 ms ou mais.

O design intencionalmente troca alguma clareza conceitual em torno do uso de `await` por melhor desempenho. Cada vez que uma tarefa é aguardada, o controle precisa ser passado por toda a pilha de chamadas até o laço de eventos. O laço de eventos precisa gerenciar seu estado interno e resolver seu processamento lógico para continuar a próxima tarefas. Isso pode parecer insignificante, mas em um programa grande com muitas instruções `awaits` essa sobrecarga pode representar um significativo prejuízo ao desempenho.

Uma visão geral conceitual, parte 2: os detalhes

A parte 2 detalha os mecanismos que **`asyncio`** usa para gerenciar o fluxo de controle. É aqui que a mágica acontece. Você sairá desta seção sabendo o que `await` faz nos bastidores e como criar seus próprios operadores assíncronos.

O funcionamento interno das corrotinas

`asyncio` utiliza quatro componentes do Python para passar o controle.

coroutine.send(arg) é o método usado para iniciar ou retomar uma corrotina. Se a corrotina foi pausada e agora está sendo retomada, o argumento `arg` será enviado como valor de retorno da instrução `yield` que a pausou originalmente. Se a corrotina estiver sendo usada pela primeira vez (em vez de ser retomada), `arg` deve ser `None`.

```
 1 class Rock:
 2     def __await__(self):
 3         value_sent_in = yield 7
 4         print(f"Rock.__await__ resumindo com o valor: {value_
 5         return value_sent_in
 6
 7 async def main():
 8     print("Iniciando main() da corrotina.")
 9     rock = Rock()
10     print("Aguardando rock...")
11     value_from_rock = await rock
12     print(f"Corrotina recebeu valor: {value_from_rock} de roc
13     return 23
14
15 coroutine = main()
16 intermediate_result = coroutine.send(None)
17 print(f"Corrotina pausou e retornou o valor intermediário: {i
18
19 print(f"Resumindo corrotina e enviando o valor: 42.")
20 try:
21     coroutine.send(42)
22 except StopIteration as e:
23     returned_value = e.value
24 print(f"O main() da corrotina finalizou e forneceu o valor: {
```

yield, como de costume, pausa a execução e retorna o controle ao chamador. No exemplo acima, `yield`, na linha 3, é chamado por `... = await rock` na linha 11. Em termos mais gerais, `await` chama o método **`__await__()`** do objeto fornecido. `await` também faz algo muito especial: ele propaga (ou “repassa”) quaisquer `yield`s que recebe na cadeia de chamadas. Neste caso, voltamos a `... = coroutine.send(None)` na linha 16.

A corrotina é retomada por meio da chamada `coroutine.send(42)` na linha 21. A corrotina continua de onde foi executada (ou pausada) com `yield` na linha 3 e executa as instruções restantes em seu corpo. Quando uma corrotina termina, ela levanta uma exceção **`StopIteration`** com o valor de retorno anexado ao atributo **`value`**.

Esse trecho de código produz esta saída:

```
Iniciando main() da corrotina.
Aguardando rock...
Corrotina pausou e retornou o valor intermediário: 7.
Resumindo coroutine e enviando o valor: 42.
Rock.__await__ resumindo com o valor: 42.
Corrotina recebeu valor: 42 de rock.
O main() da corrotina finalizou e forneceu o valor: 23.
```

Vale a pena parar um momento aqui e certificar-se de que você seguiu as diversas maneiras pelas quais o fluxo de controle e os valores foram passados. Muitas ideias importantes foram abordadas e vale a pena garantir que seu entendimento esteja firme.

A única maneira de ceder (ou efetivamente ceder o controle) de uma corrotina é `await` um objeto que `yield` está em seu método `__await__`. Isso pode parecer estranho para você. Você pode estar pensando:

1. What about a `yield` directly within the coroutine function? The coroutine function becomes an async generator function, a different beast entirely.

2. What about a yield from within the coroutine function to a (plain) generator? That causes the error: `SyntaxError: yield from not allowed in a coroutine`. This was intentionally designed for the sake of simplicity – mandating only one way of using coroutines. Despite that, `yield from` and `await` effectively do the same thing. Initially `yield` was barred as well, but was re-accepted to allow for async generators.

Futuros

Um future é um objeto que representa o status e o resultado de uma computação. O termo é uma referência à ideia de algo que ainda está por vir ou que ainda não aconteceu, e o objeto é uma forma de ficar de olho nesse algo.

Um future possui alguns atributos importantes. Um deles é o seu estado, que pode ser “pending”, “cancelled”, or “done” (“pendente”, “cancelado” ou “concluído”, respectivamente). Outro é o seu resultado, que é definido quando o estado transita para concluído. Ao contrário de uma corrotina, um future não representa a computação real a ser realizada; em vez disso, representa o status e o resultado dessa computação, como uma espécie de luz de status (vermelha, amarela ou verde) ou indicador.

`asyncio.Task` estende `asyncio.Future` para obter esses vários recursos. A seção anterior dizia que as tarefas armazenam uma lista de retornos de chamada, o que não era totalmente preciso. Na verdade, é a classe `Future` que implementa essa lógica, que `Task` herda.

Instruções future também podem ser usados diretamente (não por meio de tarefas). As tarefas se marcam como concluídas quando sua corrotina é concluída. Instruções future são muito mais versáteis e serão marcados como concluídos quando você informar. Dessa forma, eles são a interface flexível para você definir suas próprias condições de espera e retomada.

Um asyncio.sleep caseiro

Veremos um exemplo de como você pode aproveitar um future para criar sua própria variante de suspensão assíncrona (`async_sleep`) que imita `asyncio.sleep()`.

Este trecho de código registra algumas tarefas no laço de eventos e, em seguida, aguarda a tarefa criada por `asyncio.create_task`, que envolve a corrotina `async_sleep(3)`. Queremos que essa tarefa termine somente após três segundos, mas sem impedir a execução de outras tarefas.

```
async def other_work():
    print("Eu gosto de trabalhar. Trabalhar, trabalhar.")
```

```
async def main():
    # Adiciona algumas tarefas ao laço de eventos, de forma que
    # haja algo para fazer durante a suspensão assíncrona.
    work_tasks = [
        asyncio.create_task(other_work()),
        asyncio.create_task(other_work()),
        asyncio.create_task(other_work())
    ]
    print(
        "Começando a suspensão assíncrona no horário: "
        f"{datetime.datetime.now().strftime("%H:%M:%S")}."
    )
    await asyncio.create_task(async_sleep(3))
    print(
        "Concluí a suspensão assíncrona no horário: "
        f"{datetime.datetime.now().strftime("%H:%M:%S")}."
    )
    # asyncio.gather efetivamente espera cada tarefa na coleção.
    await asyncio.gather(*work_tasks)
```

A seguir, usamos um future para permitir o controle personalizado sobre quando essa tarefa será marcada como concluída. Se **`future.set_result()`** (o método responsável por marcar o future como concluído) nunca for chamado, essa tarefa nunca terminará. Também contamos com a ajuda de outra tarefa, que veremos em breve, que monitorará o tempo decorrido e, consequentemente, chamará `future.set_result()`.

```
async def async_sleep(seconds: float):
    future = asyncio.Future()
    time_to_wake = time.time() + seconds
    # Adiciona uma tarefa de monitoramento ao laço de eventos.
    watcher_task = asyncio.create_task(_sleep_watcher(future, ti
    # Bloqueia até future ser marcado como concluído.
    await future
```

A seguir, usamos um objeto `YieldToEventLoop()` bastante simples para `yield` do seu método `__await__`, cedendo o controle ao laço de eventos. Isso é efetivamente o mesmo que chamar `asyncio.sleep(0)`, mas essa abordagem oferece mais clareza, sem mencionar que é um tanto quanto trapaça usar `asyncio.sleep` ao demonstrar como implementá-lo!

Como de costume, o laço de eventos percorre suas tarefas, concedendo-lhes o controle e recebendo-o de volta quando elas pausam ou terminam. A tarefa `watcher_task`, que executa a corrotina `_sleep_watcher(...)`, será invocada uma vez por ciclo completo do laço de eventos. A cada retomada, ela verificará o tempo e, se não tiver decorrido tempo suficiente, pausará novamente e devolverá o controle ao laço de eventos. Assim que o tempo suficiente tiver decorrido, `_sleep_watcher(...)` marcará o futuro como concluído e finalizará, saindo de seu laço `while` infinito. Dado que essa tarefa auxiliar é invocada apenas uma vez por ciclo do laço de eventos, você está correto ao observar que essa suspensão assíncrona durará *pelo menos* três segundos, em vez de exatamente três segundos. Observe que isso também se aplica a `asyncio.sleep`.

```
class YieldToEventLoop:
    def __await__(self):
        yield

async def _sleep_watcher(future, time_to_wake):
    while True:
        if time.time() >= time_to_wake:
            # Isso marca future como concluído.
            future.set_result(None)
            break
        else:
            await YieldToEventLoop()
```

Aqui está a saída completa do programa:

```
$ python custom-async-sleep.py
Começando a suspensão assíncrona no horário: 14:52:22.
Eu gosto de trabalhar. Trabalhar, trabalhar.
Eu gosto de trabalhar. Trabalhar, trabalhar.
Eu gosto de trabalhar. Trabalhar, trabalhar.
Concluí a suspensão assíncrona no horário: 14:52:25.
```

Você pode achar que esta implementação de suspensão assíncrona foi desnecessariamente complexa. E, bem, foi mesmo. O exemplo tinha como objetivo demonstrar a versatilidade dos futures com um exemplo simples que poderia ser replicado para necessidades mais complexas. Para referência, você poderia implementá-lo sem futures, assim:

```
async def simpler_async_sleep(seconds):
    time_to_wake = time.time() + seconds
    while True:
        if time.time() >= time_to_wake:
            return
        else:
            await YieldToEventLoop()
```

Mas por agora é tudo. Espero que agora esteja pronto para mergulhar com mais confiança na programação assíncrona ou consultar tópicos avançados no **restante da documentação**.

Portando módulos de extensão para o Python 3

Recomendamos os seguintes recursos para portar módulos de extensão para o Python 3:

- O capítulo Migrating C extensions [http://python3porting.com/cextensions.html] de *Supporting Python 3: An in-depth guide*, um livro sobre a migração do Python 2 para o Python 3 em geral, orienta o leitor na portabilidade de um módulo de extensão.
- O Porting guide [https://py3c.readthedocs.io/en/latest/guide.html] do projeto *py3c* fornece sugestões opinativas com código de suporte.
- Ferramentas recomendadas de terceiros oferecem abstrações sobre a API C do Python. As extensões geralmente precisam ser reescritas para usar uma delas, mas a biblioteca lida com diferenças entre várias versões e implementações do Python.

Programação em Curses com Python

Resumo

Este documento descreve como usar o módulo de extensão `curses` para controlar visualizações em modo texto.

O que é curses?

A biblioteca curses fornece formas que facilitam a impressão no terminal e o tratamento de entrada via teclado para interfaces de terminal baseadas em texto, tais como VT100s, o console Linux, e emuladores de terminais fornecidos por vários programas. Terminais visuais suportam vários códigos de controle para executar várias operações comuns como mover o cursor, apagar áreas e rolagem de tela. Diferentes terminais usam uma gama de diferentes códigos, e frequentemente têm suas próprias peculiaridades.

No mundo dos displays gráficos, uma pergunta pode vir à tona “por que isso, jovem?”. É verdade que os terminais de exibição de caracteres são uma tecnologia obsoleta, mas há nichos nos quais a capacidade de fazer coisas sofisticadas com eles continua sendo valorizada. Um nicho são os Unixes de baixa pegada de recursos ou embarcados, que não rodam um servidor X, ou seja, não possuem interface gráfica. Outro nicho é o de ferramentas como instaladores de sistemas operacionais e configurações de kernels que podem precisar rodar antes que qualquer suporte a interfaces gráficas esteja disponível.

A biblioteca curses fornece funcionalidades bastante básicas, proporcionando ao programador uma abstração de um monitor contendo janelas de texto não sobrepostas. Os conteúdos de uma janela podem ser

modificados de diversas formas—adicionando texto, apagando-o, modificando sua aparência—e a biblioteca curses irá descobrir quais códigos de controle precisam ser enviados ao terminal para produzir a saída correta. A biblioteca curses não fornece muitos conceitos de interface de usuário como botões, caixas de seleção ou diálogos; se você precisar desses recursos, considere uma biblioteca de interface de usuário como Urwid [https://pypi.org/project/Urwid/].

A biblioteca curses foi originalmente escrita para o BSD Unix; as versões mais recentes do System V do Unix da AT&T adicionaram muitos aprimoramentos e novas funções. A BSD curses não é mais mantida, tendo sido substituída pela ncursess, que é uma implementação de código aberto da interface da AT&T. Se você estiver usando um sistema operacional de código aberto baseado em Unix, tal como Linux ou FreeBSD, seu sistema provavelmente usa ncurses. Uma vez que a maioria das versões comerciais do Unix são baseadas no código do System V, todas as funções descritas aqui provavelmente estarão disponíveis. No entanto, as versões antigas de curses carregadas por alguns Unixes proprietários podem não prover tudo.

A versão do Python para Windows não inclui o módulo **`curses`**. O pacote de terceiros windows-curses [https://pypi.org/project/windows-curses/] fornece a mesma interface no Windows.

O módulo curses de Python

O módulo Python é um invólucro razoavelmente simples sobre as funções C disponibilizadas pela curses; Se você já estiver familiarizado com programação curses em C, é bem fácil transmitir esse conhecimento para o Python. A maior diferença é que a interface Python simplifica as coisas ao juntar diferentes funções C como **`addstr()`**, **`mvaddstr()`** e **`mvwaddstr()`** em um único método **`addstr()`**. Veremos isso mais adiante.

Este documento é uma introdução à escrita de programas em modo texto com curses e Python. Isto não pretende ser um guia completo da API curses; para isso, veja a seção ncurses no guia da biblioteca Python, e o manual de ncurses. Isto, no entanto, lhe dará uma ideia básica.

Começando e terminando uma aplicação curses

Antes de qualquer coisas, a curses precisa ser inicializada. Para isso, chame a função **`initscr()`**, que vai determinar o tipo do terminal, enviar para ele quaisquer códigos de inicialização necessários, e criar várias estruturas de dados internas. Se bem-sucedida, a **`initscr()`** retorna um objeto de janela representando a tela inteira; ele é comumente chamado `stdscr` por causa da variável C correspondente.

```
import curses
stdscr = curses.initscr()
```

Geralmente aplicações curses desativam o envio automático de teclas para a tela, para que seja possível ler teclas e exibi-las somente sob certas circunstâncias. Isto requer a chamada da função **`noecho()`**.

```
curses.noecho()
```

Aplicações também irão comumente precisar reagir a teclas instantaneamente, sem requisitar que a tecla Enter seja pressionada; isto é chamado de modo cbreak, ao contrário do modo de entrada buferizada usual.

```
curses.cbreak()
```

Terminais geralmente retornam teclas especiais, como as teclas de cursor ou de navegação como Page Up e Home, como uma sequência de escape mutibyte. Enquanto você poderia escrever sua aplicação para esperar essas sequências e processá-las de acordo, curses pode fazer isto para você, retornando um valor especial como **`curses.KEY_LEFT`**. Para permitir que curses faça esse trabalho, você precisará habilitar o modo keypad.

```
stdscr.keypad(True)
```

Finalizar uma aplicação curses é mais fácil do que iniciar uma. Você precisará executar:

```
curses.nocbreak()
stdscr.keypad(False)
```

```
curses.echo()
```

para reverter as configurações de terminal usadas pela biblioteca curses. Então, chame a função **endwin()** para restaurar o terminal para seu modo de operação original.

```
curses.endwin()
```

Um problema comum ao depurar uma aplicação curses é deixar seu terminal bagunçado quando a aplicação para sem restaurar o terminal ao seu estado anterior. Em Python isto comumente acontece quando seu código está com problemas e levanta uma exceção não capturada. As teclas não são mais enviadas para a tela quando você as digita, por exemplo, o que torna difícil utilizar o shell.

No Python você pode evitar essas complicações e fazer depurações de forma mais simples importando a função **curses.wrapper()** e utilizando-a desta forma:

```
from curses import wrapper

def main(stdscr):
    # Limpa a tela
    stdscr.clear()

    # Levanta ZeroDivisionError quando i == 10.
    for i in range(0, 11):
        v = i-10
        stdscr.addstr(i, 0, '10 dividido por {} é {}'.format(v,

        stdscr.refresh()
        stdscr.getkey()

wrapper(main)
```

A função **wrapper()** leva um objeto chamável e faz as inicializações descritas acima, inicializando também as cores se o terminal as prover. A **wrapper()** então roda o chamável. Quando ele retornar, a **wrapper()** vai restaurar o estado original do terminal. O chamável é chamado dentro de

um bloco `try`…`except` que captura exceções, restaura o estado do terminal, e então as levanta novamente. Dessa forma, o seu terminal não vai ser deixado em nenhum estado engraçadinho caso haja qualquer erro, e você poderá ler a mensagem e o traceback da exceção.

Janelas e Pads

Janelas são a abstração mais básica em curses. Uma objeto janela representa uma área retangular da tela, e provê métodos para exibir texto, apagá-lo, e permitir ao usuário inserir strings, e assim por diante.

O objeto `stdscr` retornado pela função `initscr()` é um objeto janela que cobre a tela inteira. Muitos programas podem precisar apenas desta janela única, mas você poderia desejar dividir a tela em janelas menores, a fim de redefini-las ou limpá-las separadamente. A função `newwin()` cria uma nova janela de um dado tamanho, retornando o novo objeto janela.

```
begin_x = 20; begin_y = 7
height = 5; width = 40
win = curses.newwin(height, width, begin_y, begin_x)
```

Note que o sistema de coordenadas utilizado na curses é incomum. Coordenadas geralmente são passadas na ordem *y*,*x*, e o canto superior-esquerdo da janela é a coordenada (0,0). Isto quebra a convenção normal para tratar coordenadas onde a coordenada *x* vem primeiro. Isto é uma diferença infeliz da maioria das aplicações computacionais, mas tem sido parte da curses desde que ela foi inicialmente escrita, e agora é tarde demais para mudar isso.

Sua aplicação pode determinar o tamanho da tela usando as variáveis `curses.LINES` e `curses.COLS` para obter os tamanhos *y* e *x*. Coordenadas válidas vão variar de `(0,0)` até `(curses.LINES - 1, curses.COLS - 1)`.

Quando você chamar um método para exibir ou apagar texto, o efeito não é exibido imediatamente na tela. Em vez disso você deve chamar o método `refresh()` dos objetos janela para atualizar a tela.

Isso é porque a curses foi desenvolvida originalmente no contexto de conexões lentas de até 300 Bauds com o terminal; para esses terminais, minimizar o tempo necessário para redesenhar a tela era importantíssimo. No lugar disso, a curses acumula mudanças na tela, e as exibe da forma mais eficiente possível quando você chama **`refresh()`**. Por exemplo, se o seu programa mostra um texto em uma janela e então limpa a janela, não tem porque enviar o texto original já que ele nunca seria visível.

Na prática, ficar pedindo ao curses explicitamente para redesenhar a tela não torna a programação com ele tão complicada assim. A maioria dos programas faz uma enxurrada de atividades, e então pausa esperando uma tecla digitada ou alguma outra ação por parte do usuário. Tudo que você deve fazer é garantir que a tela foi redesenhada antes de pausar esperando pela entrada do usuário, chamando antes a **`stdscr.refresh()`** ou o método **`refresh()`** de alguma outra janela apropriada.

Um pad é um caso especial de janela; ele pode ser mais largo que a tela atual, e apenas uma porção do pad exibido por vez. Criar um pad requer sua altura e largura, enquanto atualizar o pad requer dar as coordenadas da área na tela onde uma subseção do pad será exibida.

```
pad = curses.newpad(100, 100)
# Estes laços preenchem o pad com letras; a addch()
# é explicada na seção a seguir.
for y in range(0, 99):
    for x in range(0, 99):
        pad.addch(y,x, ord('a') + (x*x+y*y) % 26)

# Exibe um pedaço de um pad no meio da tela.
# (0,0) : coordenada do canto superior esquerdo da área do pad a
#         exibida.
# (5,5) : coordenada do canto superior esquerdo da área da janel
#         preenchida com o conteúdo do pad.
# (20, 75) : coordenada inferior direita da área da janela a ser
#          : preenchida com o conteúdo do pad.
pad.refresh( 0,0, 5,5, 20,75)
```

A chamada da **`refresh()`** exibe um pedaço do pad no retângulo que vai da coordenada (5,5) à coordenada (20,75) da tela; o canto superior esquerdo do

pedaço exibido corresponde à coordenada (0,0) do pad. Fora essa diferença, pads são exatamente como janelas comuns, e provêm os mesmos métodos.

Se você tem múltiplas janelas e pads na tela, existe um modo mais eficiente de atualizar a tela que evita as cintilações irritantes que ocorrem à medida que cada parte da tela vai sendo atualizada. O **`refresh()`** na verdade faz duas coisas:

1. Chama o método **`noutrefresh()`** de cada janela para atualizar uma estrutura de dados subjacente representando o estado desejado da tela.
2. Chama a função **`doupdate()`** para modificar a tela física para corresponder com o estado desejado presente na estrutura de dados.

Em vez disso você pode chamar **`noutrefresh()`** em várias janelas para atualizar a estrutura de dados, e então chamar **`doupdate()`** para atualizar a tela.

Exibindo texto

Do ponto de vista de um programador C, a curses à vezes pode parecer como um labirinto sinuoso de funções, todas ligeiramente diferentes. Por exemplo, **`addstr()`** exibe uma string na posição atual do cursor na janela `stdscr`, enquanto **`mvaddstr()`** primeiro move o cursor para uma dada coordenada y,x antes de exibir a string. A **`waddstr()`** é como a **`addstr()`**, mas permite especificar uma janela para utilizar em vez de usar a `stdscr` por padrão. A **`mvwaddstr()`** permite especificar tanto uma janela quanto uma coordenada.

Felizmente, a interface do Python oculta todos estes detalhes. `stdscr` é um objeto de janela como qualquer outro, e métodos como **`addstr()`** aceitam múltiplas formas de argumentos.

Forma	Descrição
str ou *ch*	Mostra a string *str* ou caractere *ch* na posição atual.

Forma	Descrição
str ou *ch*, *attr*	Mostra a string *str* ou caractere *ch*, usando o atributo *attr* na posição atual.
y, *x*, *str* ou *ch*	Move para a posição *y,x* dentro da janela, e exibe *str* ou *ch*
y, *x*, *str* ou *ch*, *attr*	Mover para a posição *y,x* dentro da janela, e exibir *str* ou *ch*, usando o atributo *attr*

Atributos permitem exibir texto de formas destacadas como negrito, sublinhado, código invertido, ou colorido. Elas serão explicadas em mais detalhes na próxima subseção.

The `addstr()` method takes a Python string or bytestring as the value to be displayed. The contents of bytestrings are sent to the terminal as-is. On a build without wide-character support strings are encoded using the value of the window's `encoding` attribute; this defaults to the default system encoding as returned by `locale.getencoding()`.

Os métodos `addch()` pegam um caractere, que pode ser tanto uma string de comprimento 1, um bytestring de comprimento 1, ou um inteiro.

Constants are provided for the characters of the terminal's alternate character set. For example, `ACS_PLMINUS` is a +/- symbol, and `ACS_ULCORNER` is the upper left corner of a box (handy for drawing borders). You can also use the appropriate Unicode character.

Janelas lembram onde o cursor estava após a última operação, então se você omitir as coordenadas *y,x*, a string ou caractere serão exibidos onde quer que a última operação foi deixada. Você também pode mover o cursos com o método `move(y,x)`. Porque alguns terminais sempre exibem um cursos

piscando, você pode querer garantir que o cursor está posicionado em algum local onde ele não será uma distração; pode ser confuso ter o cursor piscando em um local aparentemente aleatório.

If your application doesn't need a blinking cursor at all, you can call `curs_set(False)` to make it invisible. The window method `leaveok()` does something different: when its argument is true, curses leaves the cursor wherever the last update put it, instead of moving it back to the window's cursor position.

Atributos e Cor

Caracteres podem ser exibidos de diferentes formas. Linhas de status em aplicações baseadas em texto são tradicionalmente exibidas em vídeo reverso, ou um visualizador de texto pode precisar destacar certas palavras. Para isso, a curses permite que você especifique um atributo para cada célula na tela.

Um atributo é um inteiro em que cada bit representa um atributo diferente. Você sempre pode tentar exibir texto com múltiplos bits de atributo ligados ao mesmo tempo, mas a curses não garante que todas as combinações possíveis estarão disponíveis, ou que elas são todas visualmente distintas. Isso depende da habilidade do terminal sendo usado, de forma que é mais seguro se ater aos atributos mais comumente disponíveis, listados aqui.

Atributo	Descrição
`A_BLINK`	Texto piscante
`A_BOLD`	Texto em negrito ou brilho extra
`A_DIM`	Texto em brilho médio

Atributo	Descrição
`A_REVERSE`	Texto em vídeo reverso
`A_STANDOUT`	O modo mais destacado disponível
`A_UNDERLINE`	Texto sublinhado

Assim, pra exibir uma linha de status em vídeo reverso na linha no topo da tela, você poderia escrever:

```
stdscr.addstr(0, 0, "Modo atual: Modo de digitação",
              curses.A_REVERSE)
stdscr.refresh()
```

A biblioteca curses também permite o uso de cores nos terminais que as provêm. Destes, o terminal mais comum é provavelmente o console Linux, seguido pelos xterms com cores.

Para usar cores, você deve chamar a função `start_color()` logo depois de chamar `initscr()`, para inicializar a paleta de cores padrão (a função `curses.wrapper()` faz isso automaticamente.) Uma vez feito isso, a função `has_colors()` retorna True se o terminal em uso de fato suportar cores. (Nota: a curses usa a grafia Americana ‘color’ ao invés da grafia Britânica/Canadense ‘colour.’ Se você estiver acostumado com a grafia Britânica, você terá que se resignar a escrever com a grafia trocada ao chamar essas funções.)

A biblioteca curses mantém uma quantidade finita de pares de cores, contendo uma cor de primeiro plano (ou de texto) e uma de fundo. Você pode obter o valor do atributo correspondente a um par de cores com a função `color_pair()`; esse valor pode ser combinado com OU (OR) bit a bit com outros atributos como `A_REVERSE`, mas, novamente, não é garantido que tais combinações vão funcionar em todos os terminais.

Um exemplo, exibindo uma linha de texto com o par de cores 1:

```
stdscr.addstr("Texto colorido", curses.color_pair(1))
stdscr.refresh()
```

Como dito anteriormente, um par de cores consiste de uma cor de primeiro plano e uma cor de fundo. A função `init_pair(n, f, b)` muda a definição do par de cores *n* para a cor de primeiro plano *f* e cor de fundo *b*. O par 0 é pré-definido como branco sobre preto, e não pode ser mudado.

As cores são numeradas, e a `start_color()` inicializa 8 cores básicas ao ativar o modo colorido. São elas: 0:preto, 1:vermelho, 2:verde, 3:amarelo, 4:azul, 5:magenta, 6:ciano e 7:branco. O módulo `curses` define constantes nomeadas para cada uma dessas cores: `curses.COLOR_BLACK`, `curses.COLOR_RED`, e assim por diante.

Vamos juntar tudo isso. Para redefinir a cor 1 como texto vermelho sobre fundo branco, você chamaria:

```
curses.init_pair(1, curses.COLOR_RED, curses.COLOR_WHITE)
```

Quando você redefine um par de cores, qualquer texto que já tenha sido exibido usando esse par de cores vai mudar para a nova definição. Você pode também exibir mais texto nessa nova cor com:

```
stdscr.addstr(0,0, "ALERTA VERMELHO!", curses.color_pair(1))
```

Terminais super sofisticados podem trocar as definições das cores em si para um dado valor RGB. Isso permite que você troque a cor 1, que geralmente é vermelha, para roxo ou azul ou qualquer outra cor que você queira. Infelizmente, o terminal Linux não provê essa funcionalidade, de forma que eu não consigo testá-la para fornecer exemplos. Você pode verificar se o seu terminal tem esse recurso chamando a função `can_change_color()`, que retorna `True` se o recurso existir. Se por ventura o seu terminal for talentoso assim, consulte as páginas man do seu sistema para mais informações.

Entrada de usuário

A biblioteca curses em C oferece somente mecanismos de entrada muito simples. O módulo **curses** do Python adiciona um componente básico de entrada de texto. (Outras bibliotecas como Urwid [https://pypi.org/project/Urwid/] têm coleções mais extensas de componentes.)

There are three methods for getting input from a window:

- **get_wch()** refreshes the screen and then waits for the user to hit a key, displaying the key if **echo()** has been called earlier. You can optionally specify a coordinate to which the cursor should be moved before pausing.
- **getch()** does the same thing but returns the code of the key instead of a character. With ncurses this is a single byte of the key's encoding in the current locale, so a character encoded with several bytes takes several calls, one byte per call.
- **getkey()** does the same as **getch()** but returns a string: an ordinary key as a 1-character string, and a special key as its name, such as `KEY_UP`.

It's possible to not wait for the user using the **nodelay()** window method. After `nodelay(True)`, the reads for the window become non-blocking. To signal that no input is ready, **get_wch()** and **getkey()** raise an exception, and **getch()** returns `-1`. There's also a **halfdelay()** function, which can be used to (in effect) set a timer on each read; if no input becomes available within a specified delay (measured in tenths of a second), the read fails the same way.

Special keys such as Page Up, Home, or the cursor keys are returned by all three as one of the KEY_* constants, all larger than 255. You can compare the value returned to constants such as **curses.KEY_PPAGE**, **curses.KEY_HOME**, or **curses.KEY_LEFT**. The main loop of your program may look something like this:

```
while True:
    c = stdscr.get_wch()
    if c == 'p':
        PrintDocument()
    elif c == 'q':
        break  # Exit the while loop
```

```
    elif c == curses.KEY_HOME:
        x = y = 0
```

O módulo **curses.ascii** fornece funções de teste de pertencimento que levam como argumento tanto inteiros quanto strings de 1 caractere; elas podem ser úteis para escrever testes mais legíveis para tais laços. Ele também fornece funções de conversão que levam como argumento tanto inteiros quanto strings de 1 caractere, e retornam objetos do mesmo tipo. Por exemplo, **curses.ascii.ctrl()** retorna o caractere de controle correspondente ao argumento.

There's also a method to retrieve an entire line, **getstr()**. It isn't used very often, because its functionality is quite limited; the only editing keys available are the erase and kill characters, and the Enter key, which terminates the line. It returns a bytes object, and can optionally be limited to a fixed number of bytes.

```
curses.echo()            # Enable echoing of characters

# Get a line of at most 15 bytes, with the cursor on the top lir
s = stdscr.getstr(0,0, 15)
```

O módulo **curses.textpad** fornece uma caixa de texto que provê um conjunto de atalhos de teclado estilo Emacs. Vários métodos da classe **Textbox** provêm edição com validação de entrada e recuperação dos resultados da edição com ou sem espaços em branco no final.

```
import curses
from curses.textpad import Textbox, rectangle

def main(stdscr):
    stdscr.addstr(
        0, 0,
        "Digite a mensagem instantânea: (aperte Ctrl-G para envi

    editwin = curses.newwin(5,30, 2,1)
    rectangle(stdscr, 1,0, 1+5+1, 1+30+1)
    stdscr.refresh()

    box = Textbox(editwin)
```

```
    # Deixa o usuário editar até apertar Ctrl-G.
    box.edit()

    # Recupera o conteúdo resultante.
    message = box.gather()
```

Consulte a documentação de biblioteca do `curses.textpad` para mais detalhes.

Para mais informações

Este COMOFAZER não cobre alguns tópicos avançados, como ler o conteúdo da tela ou capturar eventos de mouse de uma instância xterm, mas a página de biblioteca Python para o módulo `curses` está agora razoavelmente completa. Ela deve ser o seu próximo destino.

Se estiver em dúvida quanto aos detalhes do comportamento das funções curses, consulte as páginas de manual para a sua implementação curses, seja ela o ncurses ou de algum fornecedor Unix proprietário. As páginas de manual vão documentar quaisquer peculiaridades, e prover listas completas de todas as funções, atributos e caracteres ACS_* disponíveis para você.

Devido à API da curses ser tão grande, algumas funções não são providas pela interface Python. Muitas vezes isso não é por dificuldade de implementação, e sim porque ninguém precisou delas ainda. Além disso, o Python ainda não suporta a biblioteca de menus associada à ncurses. Patches que adicionem suporte a essas funcionalidades seriam bem-vindos; veja o Guia do Desenvolvedor do Python [https://devguide.python.org/] para aprender melhor sobre como submeter patches ao Python.

- Writing Programs with NCURSES [https://invisible-island.net/ncurses/ncurses-intro.html]: um tutorial comprido para programadores C.
- Página man do ncurses [https://linux.die.net/man/3/ncurses]
- FAQ do ncurses [https://invisible-island.net/ncurses/ncurses.faq.html]
- “Use curses… don’t swear” [https://www.youtube.com/watch?v=eN1eZtjLEnU]: vídeo de uma palestra na PyCon 2013 sobre controle de terminais

usando curses ou Urwid.

- "Console Applications with Urwid" [https://pyvideo.org/video/1568/console-applications-with-urwid]: vídeo de uma palestra na PyCon CA 2012 demonstrando algumas aplicações escritas com Urwid.

Guia de descritores

Sumário

- Guia de descritores
  - Introdução
    - Exemplo simples: um descritor que retorna uma constante
    - Pesquisas dinâmicas
    - Atributos gerenciados
    - Nomes personalizados
    - Pensamentos finais
  - Exemplo completamente prático
    - Classe Validator
    - Validadores personalizados
    - Aplicação prática
  - Tutorial técnico
    - Resumo
    - Definição e introdução
    - Protocolo Descriptor
    - Visão geral da invocação do descritor
    - Invocação de uma instância
    - Invocação de uma classe
    - Invocação de super
    - Resumo da lógica de invocação
    - Notificação automática de nome
    - Exemplo de ORM
  - Equivalentes de Python puro
    - Propriedades
    - Funções e métodos
    - Tipos de métodos
    - Métodos estáticos
    - Métodos de classe
    - Member objects and __slots__

Descritores permitem que os objetos personalizem a consulta, o armazenamento e a exclusão de atributos.

Este guia tem quatro seções principais:

1. A “introdução” oferece uma visão geral básica, movendo-se suavemente a partir de exemplos simples, adicionando um recurso de cada vez. Comece aqui se você for novo em descritores.
2. A segunda seção mostra um exemplo de descritor prático completo. Se você já conhece o básico, comece por aí.
3. A terceira seção fornece um tutorial mais técnico que aborda a mecânica detalhada de como os descritores funcionam. A maioria das pessoas não precisa desse nível de detalhe.
4. The last section has pure Python equivalents for built-in descriptors that are written in C. Read this if you’re curious about how functions turn into bound methods or about the implementation of common tools like `@classmethod`, `@staticmethod`, `@property`, and `__slots__`.

Introdução

Nesta introdução, começamos com o exemplo mais básico possível e, em seguida, adicionaremos novos recursos um por um.

Exemplo simples: um descritor que retorna uma constante

A classe `Ten` é um descritor cujo método `__get__()` sempre retorna a constante `10`:

```
class Ten:
    def __get__(self, obj, objtype=None):
        return 10
```

Para usar o descritor, ele deve ser armazenado como uma variável de classe em outra classe:

```
class A:
    x = 5                       # Atributo de classe comum
    y = Ten()                   # Instância de descritor
```

Uma sessão interativa mostra a diferença entre a pesquisa de atributo normal e a pesquisa de descritor:

```
>>> a = A()                        # Cria uma instância da classeA
>>> a.x                            # Pesquisa de atributo normal
5
>>> a.y                            # Pesquisa de descritor
10
```

Na pesquisa de atributo `a.x`, o operador ponto encontra `'x': 5` no dicionário de classe. Na pesquisa `a.y`, o operador ponto encontra uma instância de descritor, reconhecida por seu método `__get__`. Chamar esse método retorna `10`.

Observe que o valor `10` não é armazenado no dicionário da classe ou no dicionário da instância. Em vez disso, o valor `10` é calculado sob demanda.

Este exemplo mostra como funciona um descritor simples, mas não é muito útil. Para recuperar constantes, a pesquisa de atributo normal seria melhor.

Na próxima seção, criaremos algo mais útil, uma pesquisa dinâmica.

Pesquisas dinâmicas

Descritores interessantes normalmente executam cálculos em vez de retornar constantes:

```
import os

class DirectorySize:

    def __get__(self, obj, objtype=None):
        return len(os.listdir(obj.dirname))

class Directory:

    size = DirectorySize()              # Instância de descritor

    def __init__(self, dirname):
        self.dirname = dirname          # Atributo de instância
```

Uma sessão interativa mostra que a pesquisa é dinâmica – calcula respostas diferentes e atualizadas a cada vez:

```
>>> s = Directory('songs')
>>> g = Directory('games')
>>> s.size                              # O jogo de músicas tem
20
>>> g.size                              # O diretório de jogos t
3
>>> os.remove('games/chess')            # Exclui um jogo
>>> g.size                              # Contagem de arquivos é
2
```

Além de mostrar como os descritores podem executar cálculos, este exemplo também revela o propósito dos parâmetros para **`__get__()`**. O parâmetro *self* é *size*, uma instância de *DirectorySize*. O parâmetro *obj* é *g* ou *s*, uma instância de *Directory*. É o parâmetro *obj* que permite ao método **`__get__()`** aprender o diretório de destino. O parâmetro *objtype* é a classe *Directory*.

Atributos gerenciados

Um uso popular para descritores é gerenciar o acesso aos dados da instância. O descritor é atribuído a um atributo público no dicionário da classe, enquanto os dados reais são armazenados como um atributo privado no dicionário da instância. Os métodos **`__get__()`** e **`__set__()`** do descritor são disparados quando o atributo público é acessado.

No exemplo a seguir, *age* é o atributo público e *_age* é o atributo privado. Quando o atributo público é acessado, o descritor registra a pesquisa ou atualização:

```
import logging

logging.basicConfig(level=logging.INFO)

class LoggedAgeAccess:
```

```
    def __get__(self, obj, objtype=None):
        value = obj._age
        logging.info('Accessing %r giving %r', 'age', value)
        return value

    def __set__(self, obj, value):
        logging.info('Updating %r to %r', 'age', value)
        obj._age = value

class Person:

    age = LoggedAgeAccess()             # Instância de descritor

    def __init__(self, name, age):
        self.name = name                # Atributo de instância
        self.age = age                  # Chama __set__()

    def birthday(self):
        self.age += 1                   # Chama __get__() e __se
```

Uma sessão interativa mostra que todo o acesso ao atributo gerenciado *age* é registrado, mas que o atributo regular *name* não é registrado:

```
>>> mary = Person('Mary M', 30)         # A atualização inicial
INFO:root:Updating 'age' to 30
>>> dave = Person('David D', 40)
INFO:root:Updating 'age' to 40

>>> vars(mary)                          # Os dados estão em um a
{'name': 'Mary M', '_age': 30}
>>> vars(dave)
{'name': 'David D', '_age': 40}

>>> mary.age                            # Acessa os dados e regi
INFO:root:Accessing 'age' giving 30
30
>>> mary.birthday()                     # Atualizações são regis
INFO:root:Accessing 'age' giving 30
INFO:root:Updating 'age' to 31

>>> dave.name                           # Pesquisa de atributo r
'David D'
```

```
>>> dave.age                              # Apenas o atributo gere
INFO:root:Accessing 'age' giving 40
40
```

Um grande problema com este exemplo é que o nome privado *_age* está conectado na classe *LoggedAgeAccess*. Isso significa que cada instância pode ter apenas um atributo registrado e que seu nome é imutável. No próximo exemplo, vamos corrigir esse problema.

Nomes personalizados

Quando uma classe usa descritores, ela pode informar a cada descritor sobre qual nome de variável foi usado.

Neste exemplo, a classe **`Person`** tem duas instâncias de descritor, *name* e *age*. Quando a classe **`Person`** é definida, ela faz uma função de retorno para **`__set_name__()`** em *LoggedAccess* para que os nomes dos campos possam ser registrados, dando a cada descritor o seu próprio *public_name* e *private_name*:

```
import logging

logging.basicConfig(level=logging.INFO)

class LoggedAccess:

    def __set_name__(self, owner, name):
        self.public_name = name
        self.private_name = '_' + name

    def __get__(self, obj, objtype=None):
        value = getattr(obj, self.private_name)
        logging.info('Accessing %r giving %r', self.public_name,
        return value

    def __set__(self, obj, value):
        logging.info('Updating %r to %r', self.public_name, valu
        setattr(obj, self.private_name, value)

class Person:
```

```
    name = LoggedAccess()                # Primeira instância de
    age = LoggedAccess()                 # Segunda instância de

    def __init__(self, name, age):
        self.name = name                 # Chama o primeiro desc
        self.age = age                   # Chama o segundo descr

    def birthday(self):
        self.age += 1
```

Uma sessão interativa mostra que a classe **`Person`** chamou `__set_name__()` para que os nomes dos campos fossem registrados. Aqui chamamos `vars()` para pesquisar o descritor sem acioná-lo:

```
>>> vars(vars(Person)['name'])
{'public_name': 'name', 'private_name': '_name'}
>>> vars(vars(Person)['age'])
{'public_name': 'age', 'private_name': '_age'}
```

A nova classe agora registra acesso a *name* e *age*:

```
>>> pete = Person('Peter P', 10)
INFO:root:Updating 'name' to 'Peter P'
INFO:root:Updating 'age' to 10
>>> kate = Person('Catherine C', 20)
INFO:root:Updating 'name' to 'Catherine C'
INFO:root:Updating 'age' to 20
```

As duas instâncias *Person* contêm apenas os nomes privados:

```
>>> vars(pete)
{'_name': 'Peter P', '_age': 10}
>>> vars(kate)
{'_name': 'Catherine C', '_age': 20}
```

Pensamentos finais

Um descritor é o que chamamos de qualquer objeto que define `__get__()`, `__set__()` ou `__delete__()`.

Opcionalmente, os descritores podem ter um método `__set_name__()`. Isso é usado somente em casos em que um descritor precisa saber a classe onde foi criado ou o nome da variável de classe à qual foi atribuído. (Este método, se presente, é chamado mesmo se a classe não for um descritor.)

Descritores são invocados pelo operador ponto durante a pesquisa de atributos. Se um descritor for acessado indiretamente com `vars(some_class)[descriptor_name]`, a instância do descritor é retornada sem invocá-lo.

Descritores só funcionam quando usados como variáveis de classe. Quando colocados em instâncias, eles não têm efeito.

A principal motivação para descritores é fornecer um gancho permitindo que objetos armazenados em variáveis de classe controlem o que acontece durante a pesquisa de atributos.

Tradicionalmente, a classe de chamada controla o que acontece durante a pesquisa. Descritores invertem esse relacionamento e permitem que os dados pesquisados tenham uma palavra a dizer sobre o assunto.

Descriptors are used throughout the language. It is how functions turn into bound methods. Common tools like `@classmethod`, `@staticmethod`, `@property`, and `@functools.cached_property` are all implemented as descriptors.

Exemplo completamente prático

Neste exemplo, criamos uma ferramenta prática e poderosa para localizar bugs de corrupção de dados notoriamente difíceis de encontrar.

Classe Validator

Um validador é um descritor para acesso de atributo gerenciado. Antes de armazenar quaisquer dados, ele verifica se o novo valor atende a várias restrições de tipo e intervalo. Se essas restrições não forem atendidas, ele levanta uma exceção para evitar corrupção de dados em sua origem.

Esta classe **`Validator`** é uma classe base abstrata e um descritor de atributo gerenciado:

```
from abc import ABC, abstractmethod

class Validator(ABC):

    def __set_name__(self, owner, name):
        self.private_name = '_' + name

    def __get__(self, obj, objtype=None):
        return getattr(obj, self.private_name)

    def __set__(self, obj, value):
        self.validate(value)
        setattr(obj, self.private_name, value)

    @abstractmethod
    def validate(self, value):
        pass
```

Validadores personalizados precisam herdar de **`Validator`** e devem fornecer um método **`validate()`** para testar várias restrições conforme necessário.

Validadores personalizados

Vemos aqui três utilitários práticos de validação de dados:

1. **`OneOf`** verifica se um valor é um de um conjunto restrito de opções.
2. **`Number`** verifica se um valor é um **`int`** ou **`float`**. Opcionalmente, ele verifica se um valor está entre um mínimo ou máximo dado.
3. **`String`** verifica se um valor é um **`str`**. Opcionalmente, ele valida um comprimento mínimo ou máximo dado. Ele pode validar um predicado [https://pt.wikipedia.org/wiki/Predicado_(lógica_matemática)] definido pelo usuário também.

```
class OneOf(Validator):

    def __init__(self, *options):
        self.options = set(options)
```

```
    def validate(self, value):
        if value not in self.options:
            raise ValueError(
                f'Expected {value!r} to be one of {self.options!
            )

class Number(Validator):

    def __init__(self, minvalue=None, maxvalue=None):
        self.minvalue = minvalue
        self.maxvalue = maxvalue

    def validate(self, value):
        if not isinstance(value, (int, float)):
            raise TypeError(f'Expected {value!r} to be an int or
        if self.minvalue is not None and value < self.minvalue:
            raise ValueError(
                f'Expected {value!r} to be at least {self.minval
            )
        if self.maxvalue is not None and value > self.maxvalue:
            raise ValueError(
                f'Expected {value!r} to be no more than {self.ma
            )

class String(Validator):

    def __init__(self, minsize=None, maxsize=None, predicate=Nor
        self.minsize = minsize
        self.maxsize = maxsize
        self.predicate = predicate

    def validate(self, value):
        if not isinstance(value, str):
            raise TypeError(f'Expected {value!r} to be a str')
        if self.minsize is not None and len(value) < self.minsiz
            raise ValueError(
                f'Expected {value!r} to be no smaller than {self
            )
        if self.maxsize is not None and len(value) > self.maxsiz
            raise ValueError(
                f'Expected {value!r} to be no bigger than {self.
            )
```

```
        if self.predicate is not None and not self.predicate(val
            raise ValueError(
                f'Expected {self.predicate} to be true for {valu
            )
```

Aplicação prática

Veja como os validadores de dados podem ser usados em uma classe real:

```
class Component:

    name = String(minsize=3, maxsize=10, predicate=str.isupper)
    kind = OneOf('wood', 'metal', 'plastic')
    quantity = Number(minvalue=0)

    def __init__(self, name, kind, quantity):
        self.name = name
        self.kind = kind
        self.quantity = quantity
```

Os descritores impedem que instâncias inválidas sejam criadas:

```
>>> Component('Widget', 'metal', 5)      # Bloqueado: 'Widget' r
Traceback (most recent call last):
    ...
ValueError: Expected <method 'isupper' of 'str' objects> to be t

>>> Component('WIDGET', 'metle', 5)      # Bloqueado: 'metle' co
Traceback (most recent call last):
    ...
ValueError: Expected 'metle' to be one of {'metal', 'plastic', '

>>> Component('WIDGET', 'metal', -5)     # Bloqueado: -5 é negat
Traceback (most recent call last):
    ...
ValueError: Expected -5 to be at least 0

>>> Component('WIDGET', 'metal', 'V')    # Bloqueado: 'V' não é
Traceback (most recent call last):
    ...
TypeError: Expected 'V' to be an int or float
```

```
>>> c = Component('WIDGET', 'metal', 5)  # Permitido:  As entrac
```

Tutorial técnico

O que se segue é um tutorial mais técnico sobre a mecânica e os detalhes de como os descritores funcionam.

Resumo

Define descritores, resume o protocolo e mostra como os descritores são chamados. Fornece um exemplo mostrando como mapeamentos relacionais de objetos funcionam.

Aprender sobre descritores não apenas fornece acesso a um conjunto de ferramentas maior, mas também cria uma compreensão mais profunda de como o Python funciona.

Definição e introdução

Em geral, um descritor é um valor de atributo que tem um dos métodos no protocolo do descritor. Esses métodos são `__get__()`, `__set__()` e `__delete__()`. Se qualquer um desses métodos for definido para um atributo, ele é dito ser um descritor.

O comportamento padrão para acesso a atributos é obter, definir ou excluir o atributo do dicionário de um objeto. Por exemplo, `a.x` tem uma cadeia de pesquisa começando com `a.__dict__['x']`, depois `type(a).__dict__['x']` e continuando pela ordem de resolução de métodos de `type(a)`. Se o valor pesquisado for um objeto que define um dos métodos descritores, o Python pode substituir o comportamento padrão e invocar o método descritor. Onde isso ocorre na cadeia de precedência depende de quais métodos descritores foram definidos.

Descritores são um protocolo poderoso de propósito geral. Eles são o mecanismo por trás de propriedades, métodos, métodos estáticos, métodos

de classe e `super()`. Eles são usados em todo o Python. Descritores simplificam o código C subjacente e oferecem um conjunto flexível de novas ferramentas para programas Python do dia a dia.

Protocolo Descriptor

```
descr.__get__(self, obj, type=None)

descr.__set__(self, obj, value)

descr.__delete__(self, obj)
```

É só isso. Defina qualquer um desses métodos e um objeto é considerado um descritor e pode substituir o comportamento padrão ao ser pesquisado como um atributo.

Se um objeto define `__set__()` ou `__delete__()`, ele é considerado um descritor de dados. Descritores que definem apenas `__get__()` são chamados de descritores não-dados (eles são frequentemente usados para métodos, mas outros usos são possíveis).

Descritores de dados e não dados diferem em como as substituições são calculadas com relação às entradas no dicionário de uma instância. Se o dicionário de uma instância tiver uma entrada com o mesmo nome de um descritor de dados, o descritor de dados terá precedência. Se o dicionário de uma instância tiver uma entrada com o mesmo nome de um descritor não dados, a entrada do dicionário terá precedência.

Para criar um descritor de dados somente leitura, defina `__get__()` e `__set__()` com o **`__set__()`** levantando `AttributeError` quando chamado. Definir o método **`__set__()`** com um espaço reservado levantando uma exceção é o suficiente para torná-lo um descritor de dados.

Visão geral da invocação do descritor

Um descritor pode ser chamado diretamente com `desc.__get__(obj)` ou `desc.__get__(None, cls)`.

Mas é mais comum que um descritor seja invocado automaticamente a partir do acesso ao atributo.

A expressão `obj.x` procura o atributo `x` na cadeia de espaços de nomes para `obj`. Se a busca encontrar um descritor fora da instância `__dict__`, seu método `__get__()` é invocado de acordo com as regras de precedência listadas abaixo.

Os detalhes da invocação dependem se `obj` é um objeto, classe ou instância de super.

Invocação de uma instância

A pesquisa de instância verifica uma cadeia de espaços de nomes, dando aos descritores de dados a maior prioridade, seguidos por variáveis de instância, depois descritores que não são de dados, depois variáveis de classe e, por último, `__getattr__()`, se fornecido.

Se um descritor for encontrado para `a.x`, ele será invocado com: `desc.__get__(a, type(a))`.

A lógica para uma pesquisa pontilhada está em `object.__getattribute__()`. Aqui está um equivalente Python puro:

```
def find_name_in_mro(cls, name, default):
    "Emulate _PyType_Lookup() in Objects/typeobject.c"
    for base in cls.__mro__:
        if name in vars(base):
            return vars(base)[name]
    return default

def object_getattribute(obj, name):
    "Emulate PyObject_GenericGetAttr() in Objects/object.c"
    null = sentinel('null')
    objtype = type(obj)
    cls_var = find_name_in_mro(objtype, name, null)
    descr_get = getattr(type(cls_var), '__get__', null)
    if descr_get is not null:
        if (hasattr(type(cls_var), '__set__')
            or hasattr(type(cls_var), '__delete__')):
```

```
            return descr_get(cls_var, obj, objtype)     # data d
    if hasattr(obj, '__dict__') and name in vars(obj):
        return vars(obj)[name]                          # instar
    if descr_get is not null:
        return descr_get(cls_var, obj, objtype)         # non-da
    if cls_var is not null:
        return cls_var                                  # class
    raise AttributeError(name)
```

Note que não há nenhum gancho **`__getattr__()`** no código **`__getattribute__()`**. É por isso que chamar **`__getattribute__()`** diretamente ou com `super().__getattribute__` ignorará **`__getattr__()`** completamente.

Em vez disso, é o operador ponto e a função **`getattr()`** que são responsáveis por invocar **`__getattr__()`** sempre que **`__getattribute__()`** levanta um **`AttributeError`**. A lógica deles é encapsulada em uma função auxiliar:

```
def getattr_hook(obj, name):
    "Emulate slot_tp_getattr_hook() in Objects/typeobject.c"
    try:
        return obj.__getattribute__(name)
    except AttributeError:
        if not hasattr(type(obj), '__getattr__'):
            raise
    return type(obj).__getattr__(obj, name)             # __geta
```

Invocação de uma classe

A lógica para uma pesquisa pontilhada como `A.x` está em **`type.__getattribute__()`**. Os passos são similares aos de **`object.__getattribute__()`** mas a pesquisa do dicionário de instância é substituída por uma pesquisa através da ordem de resolução de métodos da classe.

Se um descritor for encontrado, ele será invocado com `desc.__get__(None, A)`.

A implementação completa em C pode ser encontrada em **`type_getattro()`** e **`_PyType_Lookup()`** em Objects/typeobject.c [https://github.com/python/cpython/tree/3.15/Objects/typeobject.c].

Invocação de super

A lógica para a pesquisa pontilhada de super está no método **`__getattribute__()`** para o objeto retornado por **`super()`**.

Uma pesquisa pontilhada como `super(A, obj).m` pesquisa `obj.__class__.__mro__` para a classe base `B` imediatamente após `A` e então retorna `B.__dict__['m'].__get__(obj, A)`. Se não for um descritor, `m` é retornado inalterado.

A implementação completa em C pode ser encontrada em **`super_getattro()`** em Objects/typeobject.c [https://github.com/python/cpython/tree/3.15/Objects/typeobject.c]. Um equivalente em Python puro pode ser encontrado no Tutorial do Guido [https://www.python.org/download/releases/2.2.3/descrintro/#cooperation].

Resumo da lógica de invocação

O mecanismo para descritores está incorporado nos métodos **`__getattribute__()`** para **`object`**, **`type`** e **`super()`**.

Os pontos importantes para lembrar são:

- Descritores são invocados pelo método **`__getattribute__()`**.
- As classes herdam esse maquinário de **`object`**, **`type`** ou **`super()`**.
- Substituir **`__getattribute__()`** impede chamadas automáticas do descritor porque toda a lógica do descritor está nesse método.
- **`object.__getattribute__()`** e **`type.__getattribute__()`** fazem chamadas diferentes para **`__get__()`**. O primeiro inclui a instância e pode incluir a classe. O segundo coloca `None` para a instância e sempre inclui a classe.
- Os descritores de dados sempre substituem os dicionários de instância.

- Descritores de não-dados podem ser substituídos pelos dicionários de instância.

Notificação automática de nome

Às vezes, é desejável que um descritor saiba a qual nome de variável de classe ele foi atribuído. Quando uma nova classe é criada, a metaclasse `type` varre o dicionário da nova classe. Se qualquer uma das entradas for descritor e se eles definirem `__set_name__()`, esse método será chamado com dois argumentos. O *owner* é a classe onde o descritor é usado, e o *name* é a variável de classe à qual o descritor foi atribuído.

Os detalhes de implementações estão em `type_new()` e `set_names()` em Objects/typeobject.c [https://github.com/python/cpython/tree/3.15/Objects/typeobject.c].

Como a lógica de atualização está em `type.__new__()`, as notificações só ocorrem no momento da criação da classe. Se descritores forem adicionados à classe posteriormente, `__set_name__()` precisará ser chamado manualmente.

Exemplo de ORM

O código a seguir é um esqueleto simplificado que mostra como os descritores de dados podem ser usados para implementar um mapeamento relacional de objetos [https://pt.wikipedia.org/wiki/Mapeamento_objeto-relacional].

A ideia essencial é que os dados sejam armazenados em um banco de dados externo. As instâncias do Python só guardam chaves para as tabelas do banco de dados. Descritores cuidam de pesquisas ou atualizações:

```
class Field:

    def __set_name__(self, owner, name):
        self.fetch = f'SELECT {name} FROM {owner.table} WHERE {c
        self.store = f'UPDATE {owner.table} SET {name}=? WHERE {

    def __get__(self, obj, objtype=None):
        return conn.execute(self.fetch, [obj.key]).fetchone()[0]
```

```
    def __set__(self, obj, value):
        conn.execute(self.store, [value, obj.key])
        conn.commit()
```

Podemos usar a classe **`Field`** para definir modelos [https://en.wikipedia.org/wiki/Database_model] que descrevem o esquema de cada tabela em um banco de dados:

```
class Movie:
    table = 'Movies'                    # Nome da tabela
    key = 'title'                       # Chave primária
    director = Field()
    year = Field()

    def __init__(self, key):
        self.key = key

class Song:
    table = 'Music'
    key = 'title'
    artist = Field()
    year = Field()
    genre = Field()

    def __init__(self, key):
        self.key = key
```

Para usar os modelos, primeiro conecte ao banco de dados:

```
>>> import sqlite3
>>> conn = sqlite3.connect('entertainment.db')
```

Uma sessão interativa mostra como os dados são recuperados do banco de dados e como eles podem ser atualizados:

```
>>> Movie('Star Wars').director
'George Lucas'
>>> jaws = Movie('Jaws')
>>> f'Released in {jaws.year} by {jaws.director}'
'Released in 1975 by Steven Spielberg'
```

```
>>> Song('Country Roads').artist
'John Denver'

>>> Movie('Star Wars').director = 'J.J. Abrams'
>>> Movie('Star Wars').director
'J.J. Abrams'
```

Equivalentes de Python puro

O protocolo descritor é simples e oferece possibilidades interessantes. Vários casos de uso são tão comuns que foram pré-empacotados em ferramentas embutidas. Propriedades, métodos vinculados, métodos estáticos, métodos de classe e __slots__ são todos baseados no protocolo descritor.

Propriedades

Chamar `property()` é uma maneira sucinta de construir um descritor de dados que dispara uma chamada de função ao acessar um atributo. Sua assinatura é:

```
property(fget=None, fset=None, fdel=None, doc=None) -> property
```

A documentação mostra um uso típico para definir um atributo gerenciado `x`:

```
class C:
    def getx(self): return self.__x
    def setx(self, value): self.__x = value
    def delx(self): del self.__x
    x = property(getx, setx, delx, "I'm the 'x' property.")
```

Para ver como `property()` é implementada em termos do protocolo descritor, aqui está um equivalente Python puro que implementa a maior parte da funcionalidade principal:

```
class Property:
    "Emulate PyProperty_Type() in Objects/descrobject.c"
```

```
    def __init__(self, fget=None, fset=None, fdel=None, doc=None
        self.fget = fget
        self.fset = fset
        self.fdel = fdel
        if doc is None and fget is not None:
            doc = fget.__doc__
        self.__doc__ = doc

    def __set_name__(self, owner, name):
        self.__name__ = name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        if self.fget is None:
            raise AttributeError
        return self.fget(obj)

    def __set__(self, obj, value):
        if self.fset is None:
            raise AttributeError
        self.fset(obj, value)

    def __delete__(self, obj):
        if self.fdel is None:
            raise AttributeError
        self.fdel(obj)

    def getter(self, fget):
        return type(self)(fget, self.fset, self.fdel, self.__doc

    def setter(self, fset):
        return type(self)(self.fget, fset, self.fdel, self.__doc

    def deleter(self, fdel):
        return type(self)(self.fget, self.fset, fdel, self.__doc
```

O recurso embutido `property()` ajuda sempre que uma interface de usuário concede acesso a atributos e alterações subsequentes exigem a intervenção de um método.

Por exemplo, uma classe de planilha pode conceder acesso a um valor de célula por meio de `Cell('b10').value`. Melhorias subsequentes no programa exigem que a célula seja recalculada em cada acesso; no entanto, o programador não quer afetar o código do cliente existente acessando o atributo diretamente. A solução é encapsular o acesso ao atributo de valor em um descritor de dados de propriedade:

```
class Cell:
    ...

    @property
    def value(self):
        "Recalculate the cell before returning value"
        self.recalc()
        return self._value
```

Tanto o **`property()`** embutida quanto nosso equivalente **`Property()`** funcionariam neste exemplo.

Funções e métodos

Os recursos orientados a objetos do Python são construídos sobre um ambiente baseado em funções. Usando descritores de não-dados, os dois são mesclados perfeitamente.

Funções armazenadas em dicionários de classe são transformadas em métodos quando invocadas. Métodos diferem de funções regulares apenas porque a instância do objeto é prefixada aos outros argumentos. Por convenção, a instância é chamada *self*, mas poderia ser chamada *this* ou qualquer outro nome de variável.trabalhar

Os métodos podem ser criados manualmente com **`types.MethodType`**, que é aproximadamente equivalente a:

```
class MethodType:
    "Emulate PyMethod_Type in Objects/classobject.c"

    def __init__(self, func, obj):
        self.__func__ = func
```

```
        self.__self__ = obj

    def __call__(self, *args, **kwargs):
        func = self.__func__
        obj = self.__self__
        return func(obj, *args, **kwargs)

    def __getattribute__(self, name):
        "Emulate method_getset() in Objects/classobject.c"
        if name == '__doc__':
            return self.__func__.__doc__
        return object.__getattribute__(self, name)

    def __getattr__(self, name):
        "Emulate method_getattro() in Objects/classobject.c"
        return getattr(self.__func__, name)

    def __get__(self, obj, objtype=None):
        "Emulate method_descr_get() in Objects/classobject.c"
        return self
```

Para dar suporte à criação automática de métodos, as funções incluem o método **`__get__()`** para vincular métodos durante o acesso ao atributo. Isso significa que as funções são descritores de não-dados que retornam métodos vinculados durante a pesquisa pontilhada de uma instância. Veja como funciona:

```
class Function:
    ...

    def __get__(self, obj, objtype=None):
        "Simulate func_descr_get() in Objects/funcobject.c"
        if obj is None:
            return self
        return MethodType(self, obj)
```

A execução da classe a seguir no interpretador mostra como o descritor de função funciona na prática:

```
class D:
    def f(self):
         return self
```

```
class D2:
    pass
```

A função tem um atributo nome qualificado para dar suporte à introspecção:

```
>>> D.f.__qualname__
'D.f'
```

Acessar a função por meio do dicionário de classes não invoca **`__get__()`**. Em vez disso, ele apenas retorna o objeto da função subjacente:

```
>>> D.__dict__['f']
<function D.f at 0x00C45070>
```

O acesso pontilhado de uma classe chama **`__get__()`** que apenas retorna a função subjacente inalterada:

```
>>> D.f
<function D.f at 0x00C45070>
```

O comportamento interessante ocorre durante o acesso pontuado de uma instância. A pesquisa pontilhada chama **`__get__()`** que retorna um objeto do método vinculado:

```
>>> d = D()
>>> d.f
<bound method D.f of <__main__.D object at 0x00B18C90>>
```

Internamente, o método vinculado armazena a função subjacente e a instância vinculada:

```
>>> d.f.__func__
<function D.f at 0x00C45070>

>>> d.f.__self__
<__main__.D object at 0x00B18C90>
```

Se você já se perguntou de onde vem *self* em métodos regulares ou de onde vem *cls* em métodos de classe, é isso!

Tipos de métodos

Descritores de não-dados fornecem um mecanismo simples para variações nos padrões usuais de vinculação de funções em métodos.

To recap, functions have a `__get__()` method so that they can be converted to a method when accessed as attributes. The non-data descriptor transforms an `obj.f(*args)` call into `f(obj, *args)`. Calling `cls.f(*args)` becomes `f(*args)`.

Este gráfico resume a ligação e suas duas variantes mais úteis:

Transformação	Chamada de um objeto	Chamada de uma classe
função	f(obj, *args)	f(*args)
staticmethod	f(*args)	f(*args)
classmethod	f(type(obj), *args)	f(cls, *args)

Métodos estáticos

Static methods return the underlying function without changes. Calling either `c.f` or `C.f` is the equivalent of a direct lookup into `object.__getattribute__(c, "f")` or `object.__getattribute__(C, "f")`. As a result, the function becomes identically accessible from either an object or a class.

Good candidates for static methods are methods that do not reference the `self` variable.

For instance, a statistics package may include a container class for experimental data. The class provides normal methods for computing the average, mean, median, and other descriptive statistics that depend on the data. However, there may be useful functions which are conceptually related but do not depend on the data. For instance, `erf(x)` is handy conversion routine that comes up in statistical work but does not directly depend on a particular dataset. It can be called either from an object or the class: `s.erf(1.5) --> 0.9332` or `Sample.erf(1.5) --> 0.9332`.

Since static methods return the underlying function with no changes, the example calls are unexciting:

```
class E:
    @staticmethod
    def f(x):
        return x * 10

>>> E.f(3)
30
>>> E().f(3)
30
```

Using the non-data descriptor protocol, a pure Python version of **`@staticmethod`** would look like this:

```
import functools

class StaticMethod:
    "Emulate PyStaticMethod_Type() in Objects/funcobject.c"

    def __init__(self, f):
        self.f = f
        functools.update_wrapper(self, f)

    def __get__(self, obj, objtype=None):
        return self.f

    def __call__(self, *args, **kwds):
        return self.f(*args, **kwds)

    @property
```

```
    def __annotations__(self):
        return self.f.__annotations__
```

The `functools.update_wrapper()` call adds a `__wrapped__` attribute that refers to the underlying function. Also it carries forward the attributes necessary to make the wrapper look like the wrapped function, including `__name__`, `__qualname__`, and `__doc__`.

Métodos de classe

Unlike static methods, class methods prepend the class reference to the argument list before calling the function. This format is the same for whether the caller is an object or a class:

```
class F:
    @classmethod
    def f(cls, x):
        return cls.__name__, x

>>> F.f(3)
('F', 3)
>>> F().f(3)
('F', 3)
```

This behavior is useful whenever the method only needs to have a class reference and does not rely on data stored in a specific instance. One use for class methods is to create alternate class constructors. For example, the classmethod `dict.fromkeys()` creates a new dictionary from a list of keys. The pure Python equivalent is:

```
class Dict(dict):
    @classmethod
    def fromkeys(cls, iterable, value=None):
        "Emulate dict_fromkeys() in Objects/dictobject.c"
        d = cls()
        for key in iterable:
            d[key] = value
        return d
```

Now a new dictionary of unique keys can be constructed like this:

```
>>> d = Dict.fromkeys('abracadabra')
>>> type(d) is Dict
True
>>> d
{'a': None, 'b': None, 'r': None, 'c': None, 'd': None}
```

Using the non-data descriptor protocol, a pure Python version of `@classmethod` would look like this:

```
import functools

class ClassMethod:
    "Emulate PyClassMethod_Type() in Objects/funcobject.c"

    def __init__(self, f):
        self.f = f
        functools.update_wrapper(self, f)

    def __get__(self, obj, cls=None):
        if cls is None:
            cls = type(obj)
        return MethodType(self.f, cls)
```

The `functools.update_wrapper()` call in `ClassMethod` adds a `__wrapped__` attribute that refers to the underlying function. Also it carries forward the attributes necessary to make the wrapper look like the wrapped function: `__name__`, `__qualname__`, `__doc__`, and `__annotations__`.

Member objects and __slots__

When a class defines `__slots__`, it replaces instance dictionaries with a fixed-length array of slot values. From a user point of view that has several effects:

1. Provides immediate detection of bugs due to misspelled attribute assignments. Only attribute names specified in `__slots__` are allowed:

```
class Vehicle:
    __slots__ = ('id_number', 'make', 'model')
```

```
>>> auto = Vehicle()
>>> auto.id_nubmer = 'VYE483814LQEX'
Traceback (most recent call last):
    ...
AttributeError: 'Vehicle' object has no attribute 'id_nubmer'
```

2. Helps create immutable objects where descriptors manage access to private attributes stored in `__slots__`:

```
class Immutable:

    __slots__ = ('_dept', '_name')          # Replace the instar

    def __init__(self, dept, name):
        self._dept = dept                   # Store to private a
        self._name = name                   # Store to private a

    @property                               # Read-only descript
    def dept(self):
        return self._dept

    @property
    def name(self):                         # Read-only descript
        return self._name
```

```
>>> mark = Immutable('Botany', 'Mark Watney')
>>> mark.dept
'Botany'
>>> mark.dept = 'Space Pirate'
Traceback (most recent call last):
    ...
AttributeError: property 'dept' of 'Immutable' object has no set
>>> mark.location = 'Mars'
Traceback (most recent call last):
    ...
AttributeError: 'Immutable' object has no attribute 'location'
```

3. Saves memory. On a 64-bit Linux build, an instance with two attributes takes 48 bytes with `__slots__` and 152 bytes without. This flyweight design pattern [https://en.wikipedia.org/wiki/Flyweight_pattern] likely only matters when a large number of instances are going to be created.

4. Improves speed. Reading instance variables is 35% faster with `__slots__` (as measured with Python 3.10 on an Apple M1 processor).

5. Blocks tools like `@functools.cached_property` which require an instance dictionary to function correctly:

```
from functools import cached_property

class CP:
    __slots__ = ()                          # Eliminates the ins

    @cached_property                        # Requires an instar
    def pi(self):
        return 4 * sum((-1.0)**n / (2.0*n + 1.0)
                       for n in reversed(range(100_000)))
```

```
>>> CP().pi
Traceback (most recent call last):
  ...
TypeError: No '__dict__' attribute on 'CP' instance to cache 'pi
```

It is not possible to create an exact drop-in pure Python version of `__slots__` because it requires direct access to C structures and control over object memory allocation. However, we can build a mostly faithful simulation where the actual C structure for slots is emulated by a private `_slotvalues` list. Reads and writes to that private structure are managed by member descriptors:

```
null = sentinel('null')

class Member:

    def __init__(self, name, clsname, offset):
        'Emulate PyMemberDef in Include/descrobject.h'
        # Also see descr_new() in Objects/descrobject.c
        self.name = name
        self.clsname = clsname
        self.offset = offset

    def __get__(self, obj, objtype=None):
```

```
        'Emulate member_get() in Objects/descrobject.c'
        # Also see PyMember_GetOne() in Python/structmember.c
        if obj is None:
            return self
        value = obj._slotvalues[self.offset]
        if value is null:
            raise AttributeError(self.name)
        return value

    def __set__(self, obj, value):
        'Emulate member_set() in Objects/descrobject.c'
        obj._slotvalues[self.offset] = value

    def __delete__(self, obj):
        'Emulate member_delete() in Objects/descrobject.c'
        value = obj._slotvalues[self.offset]
        if value is null:
            raise AttributeError(self.name)
        obj._slotvalues[self.offset] = null

    def __repr__(self):
        'Emulate member_repr() in Objects/descrobject.c'
        return f'<Member {self.name!r} of {self.clsname!r}>'
```

The **`type.__new__()`** method takes care of adding member objects to class variables:

```
class Type(type):
    'Simulate how the type metaclass adds member objects for slo

    def __new__(mcls, clsname, bases, mapping, **kwargs):
        'Emulate type_new() in Objects/typeobject.c'
        # type_new() calls PyTypeReady() which calls add_methods
        slot_names = mapping.get('slot_names', [])
        for offset, name in enumerate(slot_names):
            mapping[name] = Member(name, clsname, offset)
        return type.__new__(mcls, clsname, bases, mapping, **kwa
```

The **`object.__new__()`** method takes care of creating instances that have slots instead of an instance dictionary. Here is a rough simulation in pure Python:

```
class Object:
    'Simulate how object.__new__() allocates memory for __slots_

    def __new__(cls, *args, **kwargs):
        'Emulate object_new() in Objects/typeobject.c'
        inst = super().__new__(cls)
        if hasattr(cls, 'slot_names'):
            empty_slots = [null] * len(cls.slot_names)
            object.__setattr__(inst, '_slotvalues', empty_slots)
        return inst

    def __setattr__(self, name, value):
        'Emulate _PyObject_GenericSetAttrWithDict() Objects/obje
        cls = type(self)
        if hasattr(cls, 'slot_names') and name not in cls.slot_r
            raise AttributeError(
                f'{cls.__name__!r} object has no attribute {name
            )
        super().__setattr__(name, value)

    def __delattr__(self, name):
        'Emulate _PyObject_GenericSetAttrWithDict() Objects/obje
        cls = type(self)
        if hasattr(cls, 'slot_names') and name not in cls.slot_r
            raise AttributeError(
                f'{cls.__name__!r} object has no attribute {name
            )
        super().__delattr__(name)
```

To use the simulation in a real class, just inherit from **`Object`** and set the metaclass to **`Type`**:

```
class H(Object, metaclass=Type):
    'Instance variables stored in slots'

    slot_names = ['x', 'y']

    def __init__(self, x, y):
        self.x = x
        self.y = y
```

At this point, the metaclass has loaded member objects for *x* and *y*:

```
>>> from pprint import pp
>>> pp(dict(vars(H)))
{'__module__': '__main__',
 '__doc__': 'Instance variables stored in slots',
 'slot_names': ['x', 'y'],
 '__init__': <function H.__init__ at 0x7fb5d302f9d0>,
 'x': <Member 'x' of 'H'>,
 'y': <Member 'y' of 'H'>}
```

When instances are created, they have a `slot_values` list where the attributes are stored:

```
>>> h = H(10, 20)
>>> vars(h)
{'_slotvalues': [10, 20]}
>>> h.x = 55
>>> vars(h)
{'_slotvalues': [55, 20]}
```

Misspelled or unassigned attributes will raise an exception:

```
>>> h.xz
Traceback (most recent call last):
    ...
AttributeError: 'H' object has no attribute 'xz'
```

Depurando extensões de API C e internos do CPython com GDB

Este documento explica como a extensão GDB do Python, `python-gdb.py`, pode ser usada com o depurador GDB para depurar extensões CPython e o próprio interpretador CPython.

Ao depurar problemas de baixo nível, como falhas ou impasses, um depurador de baixo nível, como o GDB, é útil para diagnosticar e corrigir o problema. Por padrão, o GDB (ou qualquer uma de suas interfaces) não oferece suporte a informações de alto nível específicas do interpretador CPython.

A extensão `python-gdb.py` adiciona informações do interpretador CPython ao GDB. A extensão ajuda a inspecionar a pilha de funções Python atualmente em execução. Dado um objeto Python representado por um ponteiro PyObject*, a extensão mostra o tipo e o valor do objeto.

Desenvolvedores que estão trabalhando em extensões CPython ou mexendo com partes do CPython escritas em C podem usar este documento para aprender como usar a extensão `python-gdb.py` com o GDB.

Nota

Este documento pressupõe que você esteja familiarizado com o básico do GDB e da API C do CPython. Ele consolida orientações do devguide [https://devguide.python.org] e da wiki do Python [https://wiki.python.org/moin/DebuggingWithGdb].

Pré-requisitos

Você precisa ter:

- GDB 7 ou posterior. (Para versões anteriores do GDB, consulte `Misc/gdbinit` nas fontes do Python 3.11 ou anterior.)
- Informações de depuração compatíveis com GDB para Python e qualquer extensão que você esteja depurando.
- A extensão `python-gdb.py`.

A extensão é construída com Python, mas pode ser distribuída separadamente ou não ser distribuída. Abaixo, incluímos dicas para alguns sistemas comuns como exemplos. Note que mesmo se as instruções corresponderem ao seu sistema, elas podem estar desatualizadas.

Configuração com Python construído a partir do código-fonte

Quando você compila o CPython a partir do código-fonte, as informações de depuração devem estar disponíveis, e a compilação deve adicionar um arquivo `python-gdb.py` ao diretório raiz do seu repositório.

Para ativar o suporte, você deve adicionar o diretório que contém `python-gdb.py` ao "auto-load-safe-path" do GDB. Se você ainda não fez isso, as versões recentes do GDB imprimirão um aviso com instruções sobre como fazer isso.

Nota

Se você não encontrar instruções para a sua versão do GDB, coloque isso no seu arquivo de configuração (`~/.gdbinit` ou `~/.config/gdb/gdbinit`):

```
add-auto-load-safe-path /caminho/para/cpython
```

Você também pode adicionar vários caminhos, separados por `:`.

Configuração para Python a partir de uma distribuição Linux

A maioria dos sistemas Linux fornece informações de depuração para o Python do sistema em um pacote chamado `python-debuginfo`, `python-dbg`

ou similar. Por exemplo:

- Fedora:

```
sudo dnf install gdb
sudo dnf debuginfo-install python3
```

- Ubuntu:

```
sudo apt install gdb python3-dbg
```

Em vários sistemas Linux recentes, o GDB pode baixar automaticamente símbolos de depuração usando *debuginfod*. No entanto, isso não instalará a extensão `python-gdb.py`; geralmente é necessário instalar separadamente o pacote de informações de depuração.

Usando a compilação de depuração e o modo de desenvolvimento

Para facilitar a depuração, você pode querer:

- Use a compilação de depuração do Python. (Ao compilar a partir do código-fonte, use `configure --with-pydebug`. Em distribuições Linux, instale e execute um pacote como `python-debug` ou `python-dbg`, se disponível.)
- Use o modo de desenvolvimento de tempo de execução (`-X dev`).

Ambos habilitam assertivas extras e desabilitam algumas otimizações. Às vezes isso esconde o bug que você está tentando encontrar, mas na maioria dos casos eles facilitam o processo.

Usando a extensão `python-gdb`

Quando a extensão é carregada, ela fornece duas principais funcionalidades: impressões bonitas para valores Python e comandos adicionais.

Pretty-printers

Este é o aspecto de um backtrace do GDB (truncado) quando esta extensão está habilitada:

```
#0  0x000000000041a6b1 in PyObject_Malloc (nbytes=Cannot access
) at Objects/obmalloc.c:748
#1  0x000000000041b7c0 in _PyObject_DebugMallocApi (id=111 'o',
#2  0x000000000041b717 in _PyObject_DebugMalloc (nbytes=24) at C
#3  0x000000000044060a in _PyUnicode_New (length=11) at Objects/
#4  0x00000000004466aa in PyUnicodeUCS2_DecodeUTF8Stateful (s=0x
    0x0) at Objects/unicodeobject.c:2531
#5  0x0000000000446647 in PyUnicodeUCS2_DecodeUTF8 (s=0x5c2b8d '
    at Objects/unicodeobject.c:2495
#6  0x0000000000440d1b in PyUnicodeUCS2_FromStringAndSize (u=0x5
    at Objects/unicodeobject.c:551
#7  0x0000000000440d94 in PyUnicodeUCS2_FromString (u=0x5c2b8d '
#8  0x0000000000584abd in PyDict_GetItemString (v=
    {'Yuck': <type at remote 0xad4730>, '__builtins__': <module
    0x5c2b8d "__lltrace__") at Objects/dictobject.c:2171
```

Observe como o argumento do dicionário para `PyDict_GetItemString` é exibido como seu `repr()`, em vez de um ponteiro `PyObject *` opaco.

A extensão funciona fornecendo uma rotina de impressão personalizada para valores do tipo `PyObject *`. Se você precisar acessar detalhes de nível inferior de um objeto, então converta o valor para um ponteiro do tipo apropriado. Por exemplo:

```
(gdb) p globals
$1 = {'__builtins__': <module at remote 0x7ffff7fb1868>, '__name
'__main__', 'ctypes': <module at remote 0x7ffff7f14360>, '__doc_
'__package__': None}

(gdb) p *(PyDictObject*)globals
$2 = {ob_refcnt = 3, ob_type = 0x3dbdf85820, ma_fill = 5, ma_use
ma_mask = 7, ma_table = 0x63d0f8, ma_lookup = 0x3dbdc7ea70
<lookdict_string>, ma_smalltable = {{me_hash = 7065186196740147
me_key = '__builtins__', me_value = <module at remote 0x7ffff7fb
{me_hash = -3681813760272919 43, me_key = '__name__',
me_value ='__main__'}, {me_hash = 0, me_key = 0x0, me_value = 0x
{me_hash = 0, me_key = 0x0, me_value = 0x0},
{me_hash = -9177857982131165996, me_key = 'ctypes',
```

```
me_value = <module at remote 0x7ffff7f14360>},
{me_hash = -8518757509529533123, me_key = '__doc__', me_value =
{me_hash = 0, me_key = 0x0, me_value = 0x0}, {
  me_hash = 6614918939584953775, me_key = '__package__', me_valu
```

Observe que os pretty-printers não chamam realmente `repr()`. Para tipos básicos, eles tentam corresponder ao seu resultado de perto.

Uma área que pode ser confusa é que a impressão personalizada para alguns tipos se parece muito com a impressão embutida do GDB para tipos padrão. Por exemplo, a impressora bonita para um `int` do Python (PyLongObject*) fornece uma representação que não é distinguível de um inteiro de nível de máquina regular.

```
(gdb) p some_machine_integer
$3 = 42

(gdb) p some_python_integer
$4 = 42
```

A estrutura interna pode ser revelada com um elenco para PyLongObject*:

```
(gdb) p *(PyLongObject*)algum_inteiro_python $5 = {ob_base = {ob
```

Uma confusão semelhante pode surgir com o tipo `str`, onde a saída se parece muito com a a impressão embutida do gdb para `char *`.

```
(gdb) p ptr_to_python_str
$6 = '__builtins__'
```

O pretty-printer para instâncias de `str` tem como padrão o uso de aspas simples (assim como o `repr` do Python para strings), enquanto o printer padrão para valores de `char *` usa aspas duplas e contém um endereço hexadecimal.

```
(gdb) p ptr_to_char_star
$7 = 0x6d72c0 "hello world"
```

Novamente, os detalhes de implementação podem ser revelados com um chamada a PyUnicodeObject*::.

```
(gdb) p *(PyUnicodeObject*)$6
$8 = {ob_base = {ob_refcnt = 33, ob_type = 0x3dad3a95a0}, length
str = 0x7ffff2128500, hash = 7065186196740147912, state = 1, def
```

py-list

A extensão adiciona um comando `py-list`, que lista o código-fonte Python (se houver) para o quadro atual na thread selecionada. A linha atual é marcada com um “>”:

```
(gdb) py-list
 901        if options.profile:
 902            options.profile = False
 903            profile_me()
 904            return
 905
>906        u = UI()
 907        if not u.quit:
 908            try:
 909                gtk.main()
 910            except KeyboardInterrupt:
 911                # properly quit on a keyboard interrupt
```

Use `py-list START` para listar em um número de linha diferente dentro do código Python, e `py-list START,END` para listar um intervalo específico de linhas dentro do código Python.

py-up e py-down

Os comandos `py-up` e `py-down` são análogos aos comandos regulares `up` e `down` do GDB, mas tentam se mover no nível dos quadros do CPython, em vez dos quadros do C.

GDB nem sempre consegue ler as informações relevantes do quadro, dependendo do nível de otimização com o qual o CPython foi

compilado. Internamente, os comandos procuram por quadros C que estão executando a função de avaliação de quadro padrão (ou seja, o laço do interpretador de bytecode principal dentro do CPython) e procuram o valor do `PyFrameObject *` relacionado.

Eles emitem o número do quadro (no nível C) dentro da thread.

Por exemplo:

```
(gdb) py-up
#37 Frame 0x9420b04, for file /usr/lib/python2.6/site-packag
gnome_sudoku/main.py, line 906, in start_game ()
    u = UI()
(gdb) py-up
#40 Frame 0x948e82c, for file /usr/lib/python2.6/site-packag
gnome_sudoku/gnome_sudoku.py, line 22, in start_game(main=<m
    main.start_game()
(gdb) py-up
Unable to find an older python frame
```

de forma estamos no topo da pilha do Python.

Os números de quadro correspondem aos exibidos pelo comando padrão `backtrace` do GDB. O comando ignora os quadros C que não estão executando código Python.

Voltando para baixo:

```
(gdb) py-down
#37 Frame 0x9420b04, for file /usr/lib/python2.6/site-packag
    u = UI()
(gdb) py-down
#34 (unable to read python frame information)
(gdb) py-down
#23 (unable to read python frame information)
(gdb) py-down
#19 (unable to read python frame information)
(gdb) py-down
#14 Frame 0x99262ac, for file /usr/lib/python2.6/site-packag
            swallower.run_dialog(self.dialog)
```

```
(gdb) py-down
#11 Frame 0x9aead74, for file /usr/lib/python2.6/site-packag
            gtk.main()
(gdb) py-down
#8 (unable to read python frame information)
(gdb) py-down
Unable to find a newer python frame
```

e estamos na parte inferior da pilha do Python.

Observe que no Python 3.12 e versões mais recentes, o mesmo quadro de pilha C pode ser usado para vários quadros de pilha Python. Isso significa que `py-up` e `py-down` podem mover vários quadros Python de uma vez. Por exemplo:

```
(gdb) py-up
#6 Frame 0x7ffff7fb62b0, for file /tmp/rec.py, line 5, in re
   time.sleep(5)
#6 Frame 0x7ffff7fb6240, for file /tmp/rec.py, line 7, in re
   recursive_function(n-1)
#6 Frame 0x7ffff7fb61d0, for file /tmp/rec.py, line 7, in re
   recursive_function(n-1)
#6 Frame 0x7ffff7fb6160, for file /tmp/rec.py, line 7, in re
   recursive_function(n-1)
#6 Frame 0x7ffff7fb60f0, for file /tmp/rec.py, line 7, in re
   recursive_function(n-1)
#6 Frame 0x7ffff7fb6080, for file /tmp/rec.py, line 7, in re
   recursive_function(n-1)
#6 Frame 0x7ffff7fb6020, for file /tmp/rec.py, line 9, in <m
   recursive_function(5)
(gdb) py-up
Unable to find an older python frame
```

py-bt

O comando `py-bt` tenta mostrar uma rastreabilidade em nível Python da thread atual.

Por exemplo:

```
(gdb) py-bt
#8 (unable to read python frame information)
#11 Frame 0x9aead74, for file /usr/lib/python2.6/site-packag
            gtk.main()
#14 Frame 0x99262ac, for file /usr/lib/python2.6/site-packag
            swallower.run_dialog(self.dialog)
#19 (unable to read python frame information)
#23 (unable to read python frame information)
#34 (unable to read python frame information)
#37 Frame 0x9420b04, for file /usr/lib/python2.6/site-packag
    u = UI()
#40 Frame 0x948e82c, for file /usr/lib/python2.6/site-packag
    main.start_game()
```

Os números de quadro correspondem aos exibidos pelo comando padrão `backtrace` do GDB.

py-print

O comando `py-print` procura um nome em Python e tenta imprimi-lo. Ele procura em locais dentro da thread atual, depois em globais e, finalmente, em embutidos:

```
(gdb) py-print self
local 'self' = <SwappableArea(running=<gtk.Dialog at remote
main_page=0) at remote 0x98fa6e4>
(gdb) py-print __name__
global '__name__' = 'gnome_sudoku.dialog_swallower'
(gdb) py-print len
builtin 'len' = <built-in function len>
(gdb) py-print scarlet_pimpernel
'scarlet_pimpernel' not found
```

Se o quadro C atual corresponder a vários quadros Python, `py-print` considera apenas o primeiro.

py-locals

O comando `py-locals` busca todas as variáveis locais do Python no quadro Python atual na thread selecionada e imprime suas representações:

```
(gdb) py-locals
self = <SwappableArea(running=<gtk.Dialog at remote 0x98faa
main_page=0) at remote 0x98fa6e4>
d = <gtk.Dialog at remote 0x98faaa4>
```

Se o quadro C atual corresponder a vários quadros Python, serão mostrados os locais de todos eles:

```
(gdb) py-locals
Locals for recursive_function
n = 0
Locals for recursive_function
n = 1
Locals for recursive_function
n = 2
Locals for recursive_function
n = 3
Locals for recursive_function
n = 4
Locals for recursive_function
n = 5
Locals for <module>
```

Uso com comandos do GDB

Os comandos de extensão complementam os comandos embutidos do GDB. Por exemplo, você pode usar os números de quadro mostrados por `py-bt` com o comando `frame` para ir a um quadro específico dentro da thread selecionada, assim:

```
(gdb) py-bt
(output snipped)
#68 Frame 0xaa4560, for file Lib/test/regrtest.py, line 1548, in
        main()
(gdb) frame 68
#68 0x00000000004cd1e6 in PyEval_EvalFrameEx (f=Frame 0xaa4560,
```

```
2665                            x = call_function(&sp, oparg);
(gdb) py-list
1543        # Run the tests in a context manager that temporary
1544        # temporary and writable directory. If it's not poss
1545        # change the CWD, the original CWD will be used. The
1546        # available from test_support.SAVEDCWD.
1547        with test_support.temp_cwd(TESTCWD, quiet=True):
>1548            main()
```

O comando `info threads` fornecerá uma lista das threads dentro do processo, e você pode usar o comando `thread` para selecionar uma diferente:

```
(gdb) info threads
  105 Thread 0x7fffefa18710 (LWP 10260)  sem_wait () at ../nptl/
  104 Thread 0x7fffdf5fe710 (LWP 10259)  sem_wait () at ../nptl/
* 1 Thread 0x7ffff7fe2700 (LWP 10145)  0x00000038e46d73e3 in sel
```

Você pode usar `thread apply all COMANDO` ou (`t a a COMANDO` para abreviar) para executar um comando em todas as threads. Com `py-bt`, isso permite que você veja o que cada thread está fazendo no nível do Python:

```
(gdb) t a a py-bt

Thread 105 (Thread 0x7fffefa18710 (LWP 10260)):
#5 Frame 0x7fffd00019d0, for file /home/david/coding/python-svn/
        self.__block.acquire()
#8 Frame 0x7fffac001640, for file /home/david/coding/python-svn/
            self._acquire_restore(saved_state)
#12 Frame 0x7fffb8001a10, for file /home/david/coding/python-svn
            cond.wait()
#16 Frame 0x7fffb8001c40, for file /home/david/coding/python-svn
                f()

Thread 104 (Thread 0x7fffdf5fe710 (LWP 10259)):
#5 Frame 0x7fffe4001580, for file /home/david/coding/python-svn/
        self.__block.acquire()
#8 Frame 0x7fffc8002090, for file /home/david/coding/python-svn/
            self._acquire_restore(saved_state)
#12 Frame 0x7fffac001c90, for file /home/david/coding/python-svn
```

```
            cond.wait()
#16 Frame 0x7fffac0011c0, for file /home/david/coding/python-svr
                f()

Thread 1 (Thread 0x7ffff7fe2700 (LWP 10145)):
#5 Frame 0xcb5380, for file /home/david/coding/python-svn/Lib/te
    time.sleep(0.01)
#8 Frame 0x7fffd00024a0, for file /home/david/coding/python-svn/
        _wait()
```

Enum

Uma `Enum` é um conjunto de nomes simbólicos vinculados a valores únicos. São similares a variáveis globais, mas eles oferecem uma `repr()` mais útil, agrupamento, segurança de tipo e alguns outros recursos.

Eles são mais úteis quando você tem uma variável que pode ter uma seleção limitada de valores. Por exemplo, os dias da semana:

```
>>> from enum import Enum
>>> class DiaDaSemana(Enum):
...     SEGUNDA = 1
...     TERÇA = 2
...     QUARTA = 3
...     QUINTA = 4
...     SEXTA = 5
...     SÁBADO = 6
...     DOMINGO = 7
```

Ou talvez as cores primárias RGB:

```
>>> from enum import Enum
>>> class Cor(Enum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3
```

Como você pode ver, criar um `Enum` é tão simples quanto escrever uma classe que herda do próprio `Enum`.

Nota

Maiúsculas em membros de Enums

Como os Enums são usados para representar constantes, e para ajudar a evitar problemas com nomes conflitando entre métodos/atributos de classes mixin e nomes enum, nós fortemente recomendamos o uso de

nomes em UPPER_CASE (em caixa alta) para membros, e usaremos esse estilo em nossos exemplos.

Dependendo da natureza do enum, o valor de um membro pode ou não ser importante. Mas, de qualquer forma, esse valor pode ser usado para obter o membro correspondente:

```
>>> DiaDaSemana(3)
<DiaDaSemana.QUARTA: 3>
```

Como você pode ver, o `repr()` de um membro mostra o nome do enum, o nome do membro e o valor. O `str()` de um membro mostra apenas o nome do enum e o nome do membro:

```
>>> print(DiaDaSemana.QUINTA)
DiaDaSemana.QUINTA
```

O *tipo* de um membro de enumeração é o enum ao qual ele pertence:

```
>>> type(DiaDaSemana.SEGUNDA)
<enum 'DiaDaSemana'>
>>> isinstance(DiaDaSemana.SEXTA, DiaDaSemana)
True
```

Os membros do Enum têm um atributo que contém apenas seu **`name`**:

```
>>> print(DiaDaSemana.TERÇA.name)
TERÇA
```

Da mesma forma, eles têm um atributo para seu **`value`**:

```
>>> DiaDaSemana.QUARTA.value
3
```

Ao contrário de muitas linguagens que tratam enumerações apenas como pares de nome/valor, Enums do Python podem ter comportamento adicionado. Por exemplo, **`datetime.date`** tem dois métodos para retornar o dia da semana: **`weekday()`** e **`isoweekday()`**. A diferença é que um deles conta de 0 a 6 e o outro de 1 a 7. Em vez de tomar conta disso nós mesmos,

podemos adicionar um método ao enum de **`DiaDaSemana`** para extrair o dia da instância de **`date`** e retornar o membro enum correspondente:

```
@classmethod
def from_date(cls, date):
    return cls(date.isoweekday())
```

A enum **`DiaDaSemana`** completa fica então assim:

```
>>> class DiaDaSemana(Enum):
...     SEGUNDA = 1
...     TERÇA = 2
...     QUARTA = 3
...     QUINTA = 4
...     SEXTA = 5
...     SÁBADO = 6
...     DOMINGO = 7
...     #
...     @classmethod
...     def from_date(cls, date):
...         return cls(date.isoweekday())
```

Agora podemos descobrir que dia é hoje! Observe:

```
>>> import datetime as dt
>>> Weekday.from_date(dt.date.today())
<Weekday.TUESDAY: 2>
```

Claro, se você estiver lendo isso em outro dia da semana, você verá esse respectivo dia.

Essa enum **`DiaDaSemana`** é ótima se a nossa variável precisar somente de um único dia, mas e se precisarmos de vários? Talvez estejamos escrevendo uma função para organizar tarefas ao longo da semana, e não queremos usar uma **`list`** – poderíamos ao invés disso usar um tipo diferente de **`Enum`**:

```
>>> from enum import Flag
>>> class DiaDaSemana(Flag):
...     SEGUNDA = 1
...     TERÇA = 2
...     QUARTA = 4
```

```
...     QUINTA = 8
...     SEXTA = 16
...     SÁBADO = 32
...     DOMINGO = 64
```

Nós mudamos duas coisas: estamos herdando de **`Flag`** (sinalizador), e os valores são todos potências de 2.

Assim como a enum **`DiaDaSemana`** original acima, podemos selecionar um item de cada vez:

```
>>> primeiro_dia_útil = DiaDaSemana.SEGUNDA
>>> primeiro_dia_útil
<DiaDaSemana.SEGUNDA: 1>
```

Porém, a **`Flag`** também nos permite combinar vários membros em uma única variável:

```
>>> fim_de_semana = DiaDaSemana.SÁBADO | DiaDaSemana.DOMINGO
>>> fim_de_semana
<DiaDaSemana.SÁBADO|DOMINGO: 96>
```

Você pode até mesmo iterar sobre uma variável **`Flag`**:

```
>>> for dia in fim_de_semana:
...     print(dia)
DiaDaSemana.SÁBADO
DiaDaSemana.DOMINGO
```

Certo, vamos configurar algumas tarefas domésticas:

```
>>> tarefas_do_joão = {
...     'alimentar o gato': DiaDaSemana.SEGUNDA | DiaDaSemana.QU
...     'lavar a louça': DiaDaSemana.TERÇA | DiaDaSemana.QUINTA,
...     'responder perguntas no SO': DiaDaSemana.SÁBADO,
...     }
```

E a função para mostrar as tarefas domésticas para um determinado dia:

```
>>> def mostrar_tarefas(tarefas, dia):
...     for tarefa, dias in tarefas.items():
```

```
...         if dia in dias:
...             print(tarefa)
...
>>> mostrar_tarefas(tarefas_do_joão, DiaDaSemana.SÁBADO)
responder perguntas no SO
```

Quando os valores em si dos membros não forem importantes, você pode economizar trabalho e usar **`auto()`** para eles:

```
>>> from enum import auto
>>> class DiaDaSemana(Flag):
...     SEGUNDA = auto()
...     TERÇA = auto()
...     QUARTA = auto()
...     QUINTA = auto()
...     SEXTA = auto()
...     SÁBADO = auto()
...     DOMINGO = auto()
...     FIM_DE_SEMANA = SÁBADO | DOMINGO
```

Acesso programático aos membros da enumeração e seus atributos.

Em alguns momentos, é util ter acesso aos membros na enumeração de forma programática (ou seja, em situações em que `Cor.RED` não é adequado porque a cor exata não é conhecida no momento da escrita do programa). Classes `Enum` permitem esse tipo de acesso:

```
>>> Cor(1)
<Cor.RED: 1>
>>> Cor(3)
<Cor.BLUE: 3>
```

Se você deseja ter acesso aos membros do enum pelo *nome*, use o acesso por itens:

```
>>> Cor['VERMELHO']
<Cor.VERMELHO: 1>
>>> Color['VERDE']
<Cor.VERDE: 2>
```

Dado um membro de um enum, se você precisar do seu **`name`** ou **`value`**:

```
>>> membro = Cor.VERMELHO
>>> membro.name
'VERMELHO'
>>> membro.value
1
```

Membros e valores duplicados em enums

Ter dois membros de um enum com o mesmo nome é inválido:

```
>>> class Forma(Enum):
...     QUADRADO = 2
...     QUADRADO = 3
...
Traceback (most recent call last):
...
TypeError: 'QUADRADO' already defined as 2
```

Porém, um membro do enum pode ter outros nomes associados a ele. Dados dois membros `A` e `B` com o mesmo valor (e `A` definido primeiro), `B` é um apelido para o membro `A`. A busca pelo membro associado ao valor de `A` retorna o membro `A`. A busca pelo membro com o nome de `A` retorna o membro `A`. A busca pelo membro com o nome de `B` também retorna o membro `A`:

```
>>> class Forma(Enum):
...     QUADRADO = 2
...     DIAMANTE = 1
...     CÍRCULO = 3
...     APELIDO_PARA_O_QUADRADO = 2
...
>>> Forma.QUADRADO
<Forma.QUADRADO: 2>
>>> Forma.APELIDO_PARA_O_QUADRADO
<Forma.QUADRADO: 2>
>>> Forma(2)
<Forma.QUADRADO: 2>
```

Nota

Tentar criar um membro com o mesmo nome de um atributo já definido (outro membro, um método, etc.) ou tentar criar um atributo com o mesmo nome de um membro não é permitido.

Garantindo valores únicos na enumeração

By default, enumerations allow multiple names as aliases for the same value. When this behavior isn't desired, you can use the `@unique` decorator:

```
>>> from enum import Enum, unique
>>> @unique
... class Errado(Enum):
...     UM = 1
...     DOIS = 2
...     TRES = 3
...     QUATRO = 3
...
Traceback (most recent call last):
...
ValueError: duplicate values found in <enum 'Errado'>: QUATRO ->
```

Usando valores automáticos

Se o valor em si não é importante, você pode usar `auto`:

```
>>> from enum import Enum, auto
>>> class Cor(Enum):
...     RED = auto()
...     BLUE = auto()
...     GREEN = auto()
...
>>> [membro.value for membro in Cor]
[1, 2, 3]
```

Os valores são escolhidos pelo `_generate_next_value_()`, que pode ser substituído:

```
>>> class AutoName(Enum):
...     @staticmethod
...     def _generate_next_value_(name, start, count, last_value
...         return name
...
>>> class Ordinal(AutoName):
...     NORTE = auto()
...     SUL = auto()
...     LESTE = auto()
...     OESTE = auto()
...
>>> [membro.value for membro in Ordinal]
['NORTE', 'SUL', 'LESTE', 'OESTE']
```

Nota

O método `_generate_next_value_()` deve ser definido antes de qualquer membro.

Iteração

Iterar sobre os membros de um enum não fornece os apelidos:

```
>>> list(Forma)
[<Forma.QUADRADO: 2>, <Forma.DIAMANTE: 1>, <Forma.CÍRCULO: 3>]
>>> list(DiaDaSemana)
[<DiaDaSemana.SEGUNDA: 1>, <DiaDaSemana.TERÇA: 2>, <DiaDaSemana.
```

Note que os apelidos `Forma.APELIDO_PARA_O_QUADRADO` e `DiaDaSemana.FIM_DE_SEMANA` não são mostrados.

O atributo especial `__members__` é um mapeamento ordenado de somente leitura dos nomes para os membros. Isso inclui todos os nomes definidos na enumeração, incluindo os apelidos:

```
>>> for nome, membroin Forma.__members__.items():
...     nome, membro
...
('QUADRADO', <Forma.QUADRADO: 2>)
('DIAMANTE', <Forma.DIAMANTE: 1>)
('CÍRCULO', <Forma.CÍRCULO: 3>)
('APELIDO_PARA_O_QUADRADO', <Forma.QUADRADO: 2>)
```

O atributo `__members__` pode ser usado para um acesso programático detalhado aos membros da enumeração. Por exemplo, achar todos os apelidos:

```
>>> [nome for nome, membro in Forma.__members__.items() if membr
['APELIDO_PARA_O_QUADRADO']
```

Nota

Apelidos em sinalizadores incluem valores com múltiplos itens ao mesmo tempo, como `3`, e nenhum item definido, isto é, `0`.

Comparações

Membros de uma enumeração são comparados por identidade:

```
>>> Color.VERMELHO is Color.VERMELHO
True
>>> Color.VERMELHO is Color.AZUL
False
>>> Color.VERMELHO is not Color.AZUL
True
```

Comparações ordenadas entre valores de enumerações *não* são suportadas. Membros Enum não são inteiros (porém, veja IntEnum abaixo):

```
>>> Color.VERMELHO < Color.AZUL
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: '<' not supported between instances of 'Color' and 'C
```

Entretanto, comparações de igualdade são definidas:

```
>>> Color.AZUL == Color.VERMELHO
False
>>> Color.AZUL != Color.VERMELHO
True
>>> Color.AZUL == Color.AZUL
True
```

Equality comparisons against non-enumeration values will always return `False` (again, **`IntEnum`** was explicitly designed to behave differently, see below):

```
>>> Color.AZUL == 2
False
```

Aviso

É possivel recarregar módulos – se um módulo recarregado contém enums, eles serão recriados, e os novos membros não podem ser comparados de forma identifica/igual a membros originais.

Membros e atributos permitidos em enumerações

A maioria dos exemplos acima usa inteiros como valores para os enums. Usar inteiros é simples e prático (isso é disponibilizado como padrão pela API funcional), mas não é a única aplicação. Na grande maioria dos caso de uso, não importa o valor de fato que um enum possui. Mas se o valor *é* importante, enums podem ser valores arbitrários.

Enumerações são classes Python, e podem ter métodos e até mesmo métodos especiais como de usual. Se temos essa enumeração:

```
>>> class Mood(Enum):
...     FUNKY = 1
...     HAPPY = 3
...
```

```
...     def describe(self):
...         # self is the member here
...         return self.name, self.value
...
...     def __str__(self):
...         return 'my custom str! {0}'.format(self.value)
...
...     @classmethod
...     def favorite_mood(cls):
...         # cls here is the enumeration
...         return cls.HAPPY
...
```

Então:

```
>>> Mood.favorite_mood()
<Mood.HAPPY: 3>
>>> Mood.HAPPY.describe()
('HAPPY', 3)
>>> str(Mood.FUNKY)
'my custom str! 1'
```

As regras para o que é permitido são: nomes que começam e terminam com underline único são reservados para enum e não podem ser usados; todos os outros atributos definidos dentro de uma enumeração irão se tornar membros dessa enumeração, com exceção dos métodos especiais (`__str__()`, `__add__()`, etc.), descritores (métodos também são descritores), e nomes de variáveis listadas em `_ignore_`.

Nota: se a sua enumeração define `__new__()` e/ou `__init__()`, qualquer valor(es) dado ao membro da enum será passado para esses métodos. Veja Planet, por exemplo.

Nota

O método `__new__()`, se definido, é usado durante a criação dos membros do Enum; em seguida, ele é substituído pelo `__new__()` do Enum, que é usado após a criação da classe para buscar por membros existentes. Consulte When to use __new__() vs. __init__() para mais detalhes.

Herança restrita de Enum

Uma nova classe `Enum` deve ter uma classe enum base, até um tipo de dado concreto, e quantas classes mixin baseadas em `object` forem necessárias. A ordem dessas classes base é:

```
class EnumName([mix-in, ...,] [data-type,] base-enum):
    pass
```

Além disso, criar uma subclasse de uma enumeração é permitido apenas se a enumeração não define nenhum membro. Portanto is,so é proibido:

```
>>> class MoreColor(Color):
...     PINK = 17
...
Traceback (most recent call last):
...
TypeError: <enum 'MoreColor'> cannot extend <enum 'Color'>
```

Mas isso é permitido:

```
>>> class Foo(Enum):
...     def some_behavior(self):
...         pass
...
>>> class Bar(Foo):
...     HAPPY = 1
...     SAD = 2
...
```

Permitir a criação de subclasses de enums que definem membros levaria a violação de alguns invariantes importantes de tipos e instâncias. Por outro lado, faz sentido permitir o compartilhamento de algum comportamento comum entre um grupo de enumerações. (Veja OrderedEnum, por exemplo.)

Suporte a dataclass

Ao herdar de uma **dataclass**, o **__repr__()** omite o nome da classe herdada. Por exemplo:

```
>>> from dataclasses import dataclass, field
>>> @dataclass
... class CreatureDataMixin:
...     size: str
...     legs: int
...     tail: bool = field(repr=False, default=True)
...
>>> class Creature(CreatureDataMixin, Enum):
...     BEETLE = 'small', 6
...     DOG = 'medium', 4
...
>>> Creature.DOG
<Creature.DOG: size='medium', legs=4>
```

Use o argumento `repr=False` de **dataclass()** para utilizar o **repr()** padrão.

Alterado na versão 3.12: Somente os campos da dataclass são exibidos na área de valor, e não o nome da dataclass.

Nota

Adding **@~dataclasses.dataclass** decorator to **Enum** and its subclasses is not supported. It will not raise any errors, but it will produce very strange results at runtime, such as members being equal to each other:

```
>>> @dataclass                # don't do this: it does not make
... class Color(Enum):
...     RED = 1
...     BLUE = 2
...
>>> Color.RED is Color.BLUE
False
>>> Color.RED == Color.BLUE  # problem is here: they should no
True
```

Pickling

Enumerações podem ser serializadas e desserializadas.

```
>>> from test.test_enum import Fruit
>>> from pickle import dumps, loads
>>> Fruit.TOMATO is loads(dumps(Fruit.TOMATO))
True
```

As restrições usuais para serialização se aplicam: enums serializáveis devem ser definidas no nível mais alto de um módulo, já que para fazer a desserialização eles devem ser importados daquele módulo.

Nota

With pickle protocol version 4 it is possible to easily pickle enums nested in other classes.

It is possible to modify how enum members are pickled/unpickled by defining `__reduce_ex__()` in the enumeration class. The default method is by-value, but enums with complicated values may want to use by-name:

```
>>> import enum
>>> class MyEnum(enum.Enum):
...     __reduce_ex__ = enum.pickle_by_enum_name
```

Nota

Using by-name for flags is not recommended, as unnamed aliases will not unpickle.

API funcional

A classe `Enum` é chamável, fornecendo a API funcional a seguir:

```
>>> Animal = Enum('Animal', 'ANT BEE CAT DOG')
>>> Animal
<enum 'Animal'>
>>> Animal.ANT
<Animal.ANT: 1>
>>> list(Animal)
[<Animal.ANT: 1>, <Animal.BEE: 2>, <Animal.CAT: 3>, <Animal.DOG:
```

The semantics of this API resemble **`namedtuple`**. The first argument of the call to **`Enum`** is the name of the enumeration.

The second argument is the *source* of enumeration member names. It can be a whitespace-separated string of names, a sequence of names, a sequence of 2-tuples with key/value pairs, or a mapping (e.g. dictionary) of names to values. The last two options enable assigning arbitrary values to enumerations; the others auto-assign increasing integers starting with 1 (use the `start` parameter to specify a different starting value). A new class derived from **`Enum`** is returned. In other words, the above assignment to **`Animal`** is equivalent to:

```
>>> class Animal(Enum):
...     ANT = 1
...     BEE = 2
...     CAT = 3
...     DOG = 4
...
```

The reason for defaulting to `1` as the starting number and not `0` is that `0` is `False` in a boolean sense, but by default enum members all evaluate to `True`.

Pickling enums created with the functional API can be tricky as frame stack implementation details are used to try and figure out which module the enumeration is being created in (e.g. it will fail if you use a utility function in a separate module, and also may not work on IronPython or Jython). The solution is to specify the module name explicitly as follows:

```
>>> Animal = Enum('Animal', 'ANT BEE CAT DOG', module=__name__)
```

Aviso

If `module` is not supplied, and Enum cannot determine what it is, the new Enum members will not be unpicklable; to keep errors closer to the source, pickling will be disabled.

The new pickle protocol 4 also, in some circumstances, relies on `__qualname__` being set to the location where pickle will be able to find the class. For example, if the class was made available in class SomeData in the global scope:

```
>>> Animal = Enum('Animal', 'ANT BEE CAT DOG', qualname='SomeDat
```

A assinatura completa é:

```
Enum(
    value='NewEnumName',
    names=<...>,
    *,
    module='...',
    qualname='...',
    type=<mixed-in class>,
    start=1,
    )
```

- *value*: What the new enum class will record as its name.
- *names*: The enum members. This can be a whitespace- or comma-separated string (values will start at 1 unless otherwise specified):

  ```
  'RED GREEN BLUE' | 'RED,GREEN,BLUE' | 'RED, GREEN, BLUE'
  ```

  or an iterator of names:

  ```
  ['VERMELHO', 'VERDE', 'AZUL']
  ```

  or an iterator of (name, value) pairs:

```
[('CYAN', 4), ('MAGENTA', 5), ('YELLOW', 6)]
```

or a mapping:

```
{'CHARTREUSE': 7, 'SEA_GREEN': 11, 'ROSEMARY': 42}
```

- *module*: name of module where new enum class can be found.
- *qualname*: where in module new enum class can be found.
- *type*: type to mix in to new enum class.
- *start*: number to start counting at if only names are passed in.

Alterado na versão 3.5: The *start* parameter was added.

Derived Enumerations

IntEnum

The first variation of `Enum` that is provided is also a subclass of `int`. Members of an `IntEnum` can be compared to integers; by extension, integer enumerations of different types can also be compared to each other:

```
>>> from enum import IntEnum
>>> class Shape(IntEnum):
...     CIRCLE = 1
...     SQUARE = 2
...
>>> class Request(IntEnum):
...     POST = 1
...     GET = 2
...
>>> Shape == 1
False
>>> Shape.CIRCLE == 1
True
>>> Shape.CIRCLE == Request.POST
True
```

However, they still can't be compared to standard `Enum` enumerations:

```
>>> class Shape(IntEnum):
...     CIRCLE = 1
...     SQUARE = 2
...
>>> class Color(Enum):
...     RED = 1
...     GREEN = 2
...
>>> Shape.CIRCLE == Color.RED
False
```

`IntEnum` values behave like integers in other ways you'd expect:

```
>>> int(Shape.CIRCLE)
1
>>> ['a', 'b', 'c'][Shape.CIRCLE]
'b'
>>> [i for i in range(Shape.SQUARE)]
[0, 1]
```

StrEnum

The second variation of `Enum` that is provided is also a subclass of `str`. Members of a `StrEnum` can be compared to strings; by extension, string enumerations of different types can also be compared to each other.

Adicionado na versão 3.11.

IntFlag

The next variation of `Enum` provided, `IntFlag`, is also based on `int`. The difference being **`IntFlag`** members can be combined using the bitwise operators (&, |, ^, ~) and the result is still an **`IntFlag`** member, if possible. Like `IntEnum`, **`IntFlag`** members are also integers and can be used wherever an **`int`** is used.

Nota

Any operation on an **IntFlag** member besides the bit-wise operations will lose the **IntFlag** membership.

Bit-wise operations that result in invalid **IntFlag** values will lose the **IntFlag** membership. See **FlagBoundary** for details.

Adicionado na versão 3.6.

Alterado na versão 3.11.

Sample **IntFlag** class:

```
>>> from enum import IntFlag
>>> class Perm(IntFlag):
...     R = 4
...     W = 2
...     X = 1
...
>>> Perm.R | Perm.W
<Perm.R|W: 6>
>>> Perm.R + Perm.W
6
>>> RW = Perm.R | Perm.W
>>> Perm.R in RW
True
```

It is also possible to name the combinations:

```
>>> class Perm(IntFlag):
...     R = 4
...     W = 2
...     X = 1
...     RWX = 7
...
>>> Perm.RWX
<Perm.RWX: 7>
>>> ~Perm.RWX
<Perm: 0>
>>> Perm(7)
<Perm.RWX: 7>
```

Nota

Combinações nomeadas são consideradas apelidos. Apelidos não aparecem durante uma iteração, mas podem ser retornados por pesquisas por valor.

Alterado na versão 3.11.

Another important difference between `IntFlag` and `Enum` is that if no flags are set (the value is 0), its boolean evaluation is `False`:

```
>>> Perm.R & Perm.X
<Perm: 0>
>>> bool(Perm.R & Perm.X)
False
```

Because `IntFlag` members are also subclasses of `int` they can be combined with them (but may lose `IntFlag` membership:

```
>>> Perm.X | 4
<Perm.R|X: 5>

>>> Perm.X + 8
9
```

Nota

The negation operator, ~, always returns an `IntFlag` member with a positive value:

```
>>> (~Perm.X).value == (Perm.R|Perm.W).value == 6
True
```

`IntFlag` members can also be iterated over:

```
>>> list(RW)
[<Perm.R: 4>, <Perm.W: 2>]
```

Adicionado na versão 3.11.

Sinalizador

The last variation is **Flag**. Like **IntFlag**, **Flag** members can be combined using the bitwise operators (&, |, ^, ~). Unlike **IntFlag**, they cannot be combined with, nor compared against, any other **Flag** enumeration, nor **int**. While it is possible to specify the values directly it is recommended to use **auto** as the value and let **Flag** select an appropriate value.

Adicionado na versão 3.6.

Like **IntFlag**, if a combination of **Flag** members results in no flags being set, the boolean evaluation is **False**:

```
>>> from enum import Flag, auto
>>> class Color(Flag):
...     RED = auto()
...     BLUE = auto()
...     GREEN = auto()
...
>>> Color.RED & Color.GREEN
<Color: 0>
>>> bool(Color.RED & Color.GREEN)
False
```

Individual flags should have values that are powers of two (1, 2, 4, 8, …), while combinations of flags will not:

```
>>> class Color(Flag):
...     RED = auto()
...     BLUE = auto()
...     GREEN = auto()
...     WHITE = RED | BLUE | GREEN
...
>>> Color.WHITE
<Color.WHITE: 7>
```

Giving a name to the “no flags set” condition does not change its boolean value:

```
>>> class Color(Flag):
...     BLACK = 0
...     RED = auto()
...     BLUE = auto()
...     GREEN = auto()
...
>>> Color.BLACK
<Color.BLACK: 0>
>>> bool(Color.BLACK)
False
```

Flag members can also be iterated over:

```
>>> purple = Color.RED | Color.BLUE
>>> list(purple)
[<Color.RED: 1>, <Color.BLUE: 2>]
```

Adicionado na versão 3.11.

Nota

For the majority of new code, **Enum** and **Flag** are strongly recommended, since **IntEnum** and **IntFlag** break some semantic promises of an enumeration (by being comparable to integers, and thus by transitivity to other unrelated enumerations). **IntEnum** and **IntFlag** should be used only in cases where **Enum** and **Flag** will not do; for example, when integer constants are replaced with enumerations, or for interoperability with other systems.

Outros

While **IntEnum** is part of the **enum** module, it would be very simple to implement independently:

```
class IntEnum(int, ReprEnum):   # or Enum instead of ReprEnum
    pass
```

This demonstrates how similar derived enumerations can be defined; for example a **`FloatEnum`** that mixes in `float` instead of `int`.

Algumas regras:

1. When subclassing `Enum`, mix-in types must appear before the **`Enum`** class itself in the sequence of bases, as in the `IntEnum` example above.
2. Mix-in types must be subclassable. For example, `bool` and `range` are not subclassable and will throw an error during Enum creation if used as the mix-in type.
3. While `Enum` can have members of any type, once you mix in an additional type, all the members must have values of that type, e.g. `int` above. This restriction does not apply to mix-ins which only add methods and don't specify another type.
4. When another data type is mixed in, the `value` attribute is *not the same* as the enum member itself, although it is equivalent and will compare equal.
5. A `data type` is a mixin that defines `__new__()`, or a `dataclass`
6. %-style formatting: `%s` and `%r` call the `Enum` class's `__str__()` and `__repr__()` respectively; other codes (such as `%i` or `%h` for IntEnum) treat the enum member as its mixed-in type.
7. Formatted string literals, `str.format()`, and `format()` will use the enum's `__str__()` method.

Nota

Because `IntEnum`, `IntFlag`, and `StrEnum` are designed to be drop-in replacements for existing constants, their `__str__()` method has been reset to their data types' **`__str__()`** method.

When to use `__new__()` vs. `__init__()`

`__new__()` must be used whenever you want to customize the actual value of the `Enum` member. Any other modifications may go in either **`__new__()`** or `__init__()`, with **`__init__()`** being preferred.

For example, if you want to pass several items to the constructor, but only want one of them to be the value:

```
>>> class Coordinate(bytes, Enum):
...     """
...     Coordinate with binary codes that can be indexed by the
...     """
...     def __new__(cls, value, label, unit):
...         obj = bytes.__new__(cls, [value])
...         obj._value_ = value
...         obj.label = label
...         obj.unit = unit
...         return obj
...     PX = (0, 'P.X', 'km')
...     PY = (1, 'P.Y', 'km')
...     VX = (2, 'V.X', 'km/s')
...     VY = (3, 'V.Y', 'km/s')
...

>>> print(Coordinate['PY'])
Coordinate.PY

>>> print(Coordinate(3))
Coordinate.VY
```

Aviso

Do not call `super().__new__()`, as the lookup-only `__new__` is the one that is found; instead, use the data type directly.

Finer Points

Supported `__dunder__` and `_sunder_` names

The supported `__dunder__` and `_sunder_` names can be found in the Enum API documentation.

_Private__names

Private names are not converted to enum members, but remain normal attributes.

Alterado na versão 3.11.

`Enum` member type

Enum members are instances of their enum class, and are normally accessed as `EnumClass.member`. In certain situations, such as writing custom enum behavior, being able to access one member directly from another is useful, and is supported; however, in order to avoid name clashes between member names and attributes/methods from mixed-in classes, upper-case names are strongly recommended.

Alterado na versão 3.5.

Creating members that are mixed with other data types

When subclassing other data types, such as `int` or `str`, with an `Enum`, all values after the = are passed to that data type's constructor. For example:

```
>>> class MyEnum(IntEnum):       # help(int) -> int(x, base=10) -
...     example = '11', 16       # so x='11' and base=16
...
>>> MyEnum.example.value         # and hex(11) is...
17
```

Boolean value of `Enum` classes and members

Enum classes that are mixed with non-`Enum` types (such as `int`, `str`, etc.) are evaluated according to the mixed-in type's rules; otherwise, all members evaluate as `True`. To make your own enum's boolean evaluation depend on the member's value add the following to your class:

```
def __bool__(self):
    return bool(self.value)
```

Plain `Enum` classes always evaluate as `True`.

`Enum` classes with methods

If you give your enum subclass extra methods, like the Planet class below, those methods will show up in a `dir()` of the member, but not of the class:

```
>>> dir(Planet)
['EARTH', 'JUPITER', 'MARS', 'MERCURY', 'NEPTUNE', 'SATURN', 'UR
>>> dir(Planet.EARTH)
['__class__', '__doc__', '__module__', 'mass', 'name', 'radius',
```

Combining members of `Flag`

Iterating over a combination of `Flag` members will only return the members that are comprised of a single bit:

```
>>> class Color(Flag):
...     RED = auto()
...     GREEN = auto()
...     BLUE = auto()
...     MAGENTA = RED | BLUE
...     YELLOW = RED | GREEN
...     CYAN = GREEN | BLUE
...
>>> Color(3)  # named combination
<Color.YELLOW: 3>
>>> Color(7)      # not named combination
<Color.RED|GREEN|BLUE: 7>
```

`Flag` and `IntFlag` minutia

Using the following snippet for our examples:

```
>>> class Color(IntFlag):
...     BLACK = 0
```

```
...     RED = 1
...     GREEN = 2
...     BLUE = 4
...     PURPLE = RED | BLUE
...     WHITE = RED | GREEN | BLUE
...
```

the following are true:

- single-bit flags are canonical
- multi-bit and zero-bit flags are aliases
- only canonical flags are returned during iteration:

  ```
  >>> list(Color.WHITE)
  [<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 4>]
  ```

- negating a flag or flag set returns a new flag/flag set with the corresponding positive integer value:

  ```
  >>> Color.BLUE
  <Color.BLUE: 4>

  >>> ~Color.BLUE
  <Color.RED|GREEN: 3>
  ```

- names of pseudo-flags are constructed from their members' names:

  ```
  >>> (Color.RED | Color.GREEN).name
  'RED|GREEN'

  >>> class Perm(IntFlag):
  ...     R = 4
  ...     W = 2
  ...     X = 1
  ...
  >>> (Perm.R & Perm.W).name is None  # effectively Perm(0)
  True
  ```

- multi-bit flags, aka aliases, can be returned from operations:

```
>>> Color.RED | Color.BLUE
<Color.PURPLE: 5>

>>> Color(7)  # or Color(-1)
<Color.WHITE: 7>

>>> Color(0)
<Color.BLACK: 0>
```

- membership / containment checking: zero-valued flags are always considered to be contained:

  ```
  >>> Color.BLACK in Color.WHITE
  True
  ```

  otherwise, only if all bits of one flag are in the other flag will True be returned:

  ```
  >>> Color.PURPLE in Color.WHITE
  True

  >>> Color.GREEN in Color.PURPLE
  False
  ```

There is a new boundary mechanism that controls how out-of-range / invalid bits are handled: `STRICT`, `CONFORM`, `EJECT`, and `KEEP`:

- STRICT –> raises an exception when presented with invalid values
- CONFORM –> discards any invalid bits
- EJECT –> lose Flag status and become a normal int with the given value
- KEEP –> keep the extra bits
  - keeps Flag status and extra bits
  - extra bits do not show up in iteration
  - extra bits do show up in repr() and str()

The default for Flag is `STRICT`, the default for `IntFlag` is `EJECT`, and the default for `_convert_` is `KEEP` (see `ssl.Options` for an example of when `KEEP` is needed).

How are Enums and Flags different?

Enums have a custom metaclass that affects many aspects of both derived **Enum** classes and their instances (members).

Enum Classes

The **EnumType** metaclass is responsible for providing the **__contains__()**, **__dir__()**, **__iter__()** and other methods that allow one to do things with an **Enum** class that fail on a typical class, such as `list(Color)` or `some_enum_var in Color`. **EnumType** is responsible for ensuring that various other methods on the final **Enum** class are correct (such as **__new__()**, **__getnewargs__()**, **__str__()** and **__repr__()**).

Flag Classes

Flags have an expanded view of aliasing: to be canonical, the value of a flag needs to be a power-of-two value, and not a duplicate name. So, in addition to the **Enum** definition of alias, a flag with no value (a.k.a. `0`) or with more than one power-of-two value (e.g. `3`) is considered an alias.

Enum Members (aka instances)

The most interesting thing about enum members is that they are singletons. **EnumType** creates them all while it is creating the enum class itself, and then puts a custom **__new__()** in place to ensure that no new ones are ever instantiated by returning only the existing member instances.

Flag Members

Flag members can be iterated over just like the **Flag** class, and only the canonical members will be returned. For example:

```
>>> list(Color)
[<Color.RED: 1>, <Color.GREEN: 2>, <Color.BLUE: 4>]
```

(Note that `BLACK`, `PURPLE`, and `WHITE` do not show up.)

Inverting a flag member returns the corresponding positive value, rather than a negative value — for example:

```
>>> ~Color.RED
<Color.GREEN|BLUE: 6>
```

Flag members have a length corresponding to the number of power-of-two values they contain. For example:

```
>>> len(Color.PURPLE)
2
```

Enum Cookbook

While `Enum`, `IntEnum`, `StrEnum`, `Flag`, and `IntFlag` are expected to cover the majority of use-cases, they cannot cover them all. Here are recipes for some different types of enumerations that can be used directly, or as examples for creating one's own.

Omitting values

In many use-cases, one doesn't care what the actual value of an enumeration is. There are several ways to define this type of simple enumeration:

- use instances of `auto` for the value
- use instances of `object` as the value
- use a descriptive string as the value
- use a tuple as the value and a custom `__new__()` to replace the tuple with an `int` value

Using any of these methods signifies to the user that these values are not important, and also enables one to add, remove, or reorder members without having to renumber the remaining members.

Using `auto`

Using `auto` would look like:

```
>>> class Color(Enum):
...     RED = auto()
...     BLUE = auto()
...     GREEN = auto()
...
>>> Color.GREEN
<Color.GREEN: 3>
```

Using `object`

Using `object` would look like:

```
>>> class Color(Enum):
...     RED = object()
...     GREEN = object()
...     BLUE = object()
...
>>> Color.GREEN
<Color.GREEN: <object object at 0x...>>
```

This is also a good example of why you might want to write your own `__repr__()`:

```
>>> class Color(Enum):
...     RED = object()
...     GREEN = object()
...     BLUE = object()
...     def __repr__(self):
...         return "<%s.%s>" % (self.__class__.__name__, self._r
...
>>> Color.GREEN
<Color.GREEN>
```

Using a descriptive string

Using a string as the value would look like:

```
>>> class Color(Enum):
...     RED = 'stop'
...     GREEN = 'go'
...     BLUE = 'too fast!'
...
>>> Color.GREEN
<Color.GREEN: 'go'>
```

Using a custom `__new__()`

Using an auto-numbering `__new__()` would look like:

```
>>> class AutoNumber(Enum):
...     def __new__(cls):
...         value = len(cls.__members__) + 1
...         obj = object.__new__(cls)
...         obj._value_ = value
...         return obj
...
>>> class Color(AutoNumber):
...     RED = ()
...     GREEN = ()
...     BLUE = ()
...
>>> Color.GREEN
<Color.GREEN: 2>
```

To make a more general purpose `AutoNumber`, add `*args` to the signature:

```
>>> class AutoNumber(Enum):
...     def __new__(cls, *args):      # this is the only change
...         value = len(cls.__members__) + 1
...         obj = object.__new__(cls)
...         obj._value_ = value
...         return obj
...
```

Then when you inherit from `AutoNumber` you can write your own `__init__` to handle any extra arguments:

```
>>> class Swatch(AutoNumber):
...     def __init__(self, pantone='unknown'):
...         self.pantone = pantone
...     AUBURN = '3497'
...     SEA_GREEN = '1246'
...     BLEACHED_CORAL = () # New color, no Pantone code yet!
...
>>> Swatch.SEA_GREEN
<Swatch.SEA_GREEN: 2>
>>> Swatch.SEA_GREEN.pantone
'1246'
>>> Swatch.BLEACHED_CORAL.pantone
'unknown'
```

Nota

The `__new__()` method, if defined, is used during creation of the Enum members; it is then replaced by Enum's `__new__()` which is used after class creation for lookup of existing members.

Aviso

Do not call `super().__new__()`, as the lookup-only `__new__` is the one that is found; instead, use the data type directly – e.g.:

```
obj = int.__new__(cls, value)
```

OrderedEnum

An ordered enumeration that is not based on `IntEnum` and so maintains the normal `Enum` invariants (such as not being comparable to other enumerations):

```
>>> class OrderedEnum(Enum):
...     def __ge__(self, other):
...         if self.__class__ is other.__class__:
...             return self.value >= other.value
...         return NotImplemented
```

```
...     def __gt__(self, other):
...         if self.__class__ is other.__class__:
...             return self.value > other.value
...         return NotImplemented
...     def __le__(self, other):
...         if self.__class__ is other.__class__:
...             return self.value <= other.value
...         return NotImplemented
...     def __lt__(self, other):
...         if self.__class__ is other.__class__:
...             return self.value < other.value
...         return NotImplemented
...
>>> class Grade(OrderedEnum):
...     A = 5
...     B = 4
...     C = 3
...     D = 2
...     F = 1
...
>>> Grade.C < Grade.A
True
```

DuplicateFreeEnum

Raises an error if a duplicate member value is found instead of creating an alias:

```
>>> class DuplicateFreeEnum(Enum):
...     def __init__(self, *args):
...         cls = self.__class__
...         if any(self.value == e.value for e in cls):
...             a = self.name
...             e = cls(self.value).name
...             raise ValueError(
...                 "aliases not allowed in DuplicateFreeEnum:
...                 % (a, e))
...
>>> class Color(DuplicateFreeEnum):
...     RED = 1
...     GREEN = 2
...     BLUE = 3
```

```
...     GRENE = 2
...
Traceback (most recent call last):
  ...
ValueError: aliases not allowed in DuplicateFreeEnum:  'GRENE' -
```

Nota

This is a useful example for subclassing Enum to add or change other behaviors as well as disallowing aliases. If the only desired change is disallowing aliases, the `unique()` decorator can be used instead.

MultiValueEnum

Supports having more than one value per member:

```
>>> class MultiValueEnum(Enum):
...     def __new__(cls, value, *values):
...         self = object.__new__(cls)
...         self._value_ = value
...         for v in values:
...             self._add_value_alias_(v)
...         return self
...
>>> class DType(MultiValueEnum):
...     float32 = 'f', 8
...     double64 = 'd', 9
...
>>> DType('f')
<DType.float32: 'f'>
>>> DType(9)
<DType.double64: 'd'>
```

Planet

If `__new__()` or `__init__()` is defined, the value of the enum member will be passed to those methods:

```
>>> class Planet(Enum):
...     MERCURY = (3.303e+23, 2.4397e6)
...     VENUS   = (4.869e+24, 6.0518e6)
...     EARTH   = (5.976e+24, 6.37814e6)
...     MARS    = (6.421e+23, 3.3972e6)
...     JUPITER = (1.9e+27,   7.1492e7)
...     SATURN  = (5.688e+26, 6.0268e7)
...     URANUS  = (8.686e+25, 2.5559e7)
...     NEPTUNE = (1.024e+26, 2.4746e7)
...     def __init__(self, mass, radius):
...         self.mass = mass       # in kilograms
...         self.radius = radius   # in meters
...     @property
...     def surface_gravity(self):
...         # universal gravitational constant  (m3 kg-1 s-2)
...         G = 6.67300E-11
...         return G * self.mass / (self.radius * self.radius)
...
>>> Planet.EARTH.value
(5.976e+24, 6378140.0)
>>> Planet.EARTH.surface_gravity
9.802652743337129
```

TimePeriod

An example to show the `_ignore_` attribute in use:

```
>>> import datetime as dt
>>> class Period(dt.timedelta, Enum):
...     "different lengths of time"
...     _ignore_ = 'Period i'
...     Period = vars()
...     for i in range(367):
...         Period['day_%d' % i] = i
...
>>> list(Period)[:2]
[<Period.day_0: datetime.timedelta(0)>, <Period.day_1: datetime.
>>> list(Period)[-2:]
[<Period.day_365: datetime.timedelta(days=365)>, <Period.day_366
```

Subclassing EnumType

While most enum needs can be met by customizing `Enum` subclasses, either with class decorators or custom functions, `EnumType` can be subclassed to provide a different Enum experience.

Programação Funcional

Nesse documento, nós vamos passear pelos recursos do Python que servem para implementar programas de forma funcional. Após uma introdução dos conceitos da programação funcional, nós veremos as propriedades da linguagem como iteradores e geradores e bibliotecas de módulos relevantes como um `itertools` e `functools`.

Introdução

Essa seção explica o conceito básico da programação funcional; se você está interessado em aprender sobre os recursos da linguagem Python, pule essa etapa para a seção Iteradores.

As linguagens de programação permitem decompor problemas de diversas maneiras diferentes:

- A Maioria das linguagens de programação são **procedurais**: os programas são listas de instruções que dizem ao computador o que fazer com as entradas do programa. C, Pascal, e mesmo o Unix shells são linguagens procedurais.
- Em linguagens **declarativas**, você escreve uma especificação que descreve o problema a ser resolvido, e a implementação do idioma descreve como executar a computação de forma eficiente. O SQL é o idioma declarativo com o qual provavelmente você está familiarizado; Uma consulta SQL descreve o conjunto de dados que deseja recuperar e o mecanismo SQL decide se deseja escanear tabelas ou usar índices, quais subcláusulas devem ser realizadas primeiro, etc.
- Os programas **orientados a objetos** manipulam coleções de objetos. Os objetos têm estado interno e métodos de suporte que consultam ou modificam esse estado interno de alguma forma. Smalltalk e Java são linguagens orientadas a objetos. C++ e Python são idiomas que suportam programação orientada a objetos, mas não forçam o uso de recursos de orientação a objetos.

- A programação **funcional** decompõe um problema em um conjunto de funções. Idealmente, as funções apenas recebem entradas e produzem saídas, e não têm nenhum estado interno que afete a saída produzida para uma determinada entrada. Linguagens funcionais bem conhecidos incluem a família ML (Standard ML, OCaml e outras variantes) e Haskell.

Os designers de algumas linguagens de computadores escolhem enfatizar uma abordagem particular da programação. Isso muitas vezes torna difícil escrever programas que usam uma abordagem diferente. Outros idiomas são linguagens com multiparadigmas que suportam várias abordagens diferentes. Lisp, C++ e Python são multiparadigmas; Você pode escrever programas ou bibliotecas que são em grande parte processuais, orientadas a objetos ou funcionais em todos esses idiomas. Em um grande programa, diferentes seções podem ser escritas usando diferentes abordagens; A GUI pode ser orientada a objetos enquanto a lógica de processamento é processual ou funcional, por exemplo.

Em um programa funcional, a entrada flui através de um conjunto de funções. Cada função opera em sua entrada e produz alguma saída. O estilo funcional desencoraja funções com efeitos colaterais que modificam o estado interno ou fazem outras alterações que não são visíveis no valor de retorno da função. As funções que não têm efeitos colaterais são chamadas **puramente funcionais**. Evitar efeitos colaterais significa não usar estruturas de dados que sejam atualizadas à medida que um programa é executado; A saída de cada função só deve depender da sua entrada.

Algumas linguagens são muito estritas a respeito da pureza e não tem nem instruções de atribuição como, por exemplo, `a=3` ou `c = a + b`, mas é difícil evitar todos os efeitos colaterais, como exibir valores na tela ou escrever em um arquivo em disco. Um outro exemplo é a chamada da função `print()` ou da função `time.sleep()`, nenhuma das quais retorna um valor útil. Ambas são chamadas apenas por conta do efeito colateral de mandar algum texto para a tela ou pausar a execução por um segundo.

Os programas Python escritos em estilo funcional geralmente não irão ao extremo de evitar todas as I/O ou todas as atribuições; Em vez disso, eles fornecerão uma interface de aparência funcional, mas usarão recursos não

funcionais internamente. Por exemplo, a implementação de uma função ainda usará atribuições para variáveis locais, mas não modificará variáveis globais ou terá outros efeitos colaterais.

A programação funcional pode ser considerada o oposto da programação orientada a objetos. Os objetos são pequenas cápsulas contendo algum estado interno, juntamente com uma coleção de chamadas de método que permitem modificar este estado, e os programas consistem em fazer o conjunto correto de mudanças de estado. A programação funcional quer evitar as mudanças de estado tanto quanto possível e funciona com o fluxo de dados entre as funções. Em Python você pode combinar as duas abordagens escrevendo funções que levam e retornam instâncias que representam objetos em seu aplicativo (mensagens de e-mail, transações, etc.).

O design funcional pode parecer uma restrição estranha para trabalhar por baixo. Por que você deve evitar objetos e efeitos colaterais? Existem vantagens teóricas e práticas para o estilo funcional:

- Probabilidade formal.
- Modularidade.
- Componibilidade.
- Fácil de depurar e testar.

Probabilidade formal

Um benefício teórico é que é mais fácil construir uma prova matemática de que um programa funcional é correto.

Durante muito tempo os pesquisadores estiveram interessados em encontrar maneiras de provar matematicamente programas corretos. Isso é diferente de testar um programa em inúmeras entradas e concluir que sua saída geralmente é correta, ou ler o código-fonte de um programa e concluir que o código parece certo; O objetivo é uma prova rigorosa de que um programa produz o resultado certo para todas as entradas possíveis.

A técnica usada para comprovar os programas corretos é escrever **invariantes**, propriedades dos dados de entrada e das variáveis do programa

que são sempre verdadeiras. Para cada linha de código, você mostra que se os invariantes X e Y forem verdadeiros **antes** da linha ser executada, os invariantes X’ e Y’ ligeiramente diferentes são verdadeiros **após** a linha ser executada. Isso continua até chegar ao final do programa, em que ponto as invariantes devem corresponder às condições desejadas na saída do programa.

A evitação das tarefas funcionais ocorreu porque as tarefas são difíceis de tratar com esta técnica; As atribuições podem invadir invariantes que eram verdadeiras antes da atribuição sem produzir novos invariantes que possam ser propagados para a frente.

Infelizmente, prpvar que os programas estão corretos são praticamente impraticáveis e não relevantes para o software Python. Mesmo os programas triviais exigem provas de várias páginas; A prova de correção para um programa moderadamente complicado seria enorme, e poucos ou nenhum dos programas que você usa diariamente (o interpretador Python, seu analisador XML, seu navegador) poderia ser comprovado correto. Mesmo que você tenha anotado ou gerado uma prova, então haveria a questão de verificar a prova; Talvez haja um erro nisso, e você acredita erroneamente que você provou o programa corretamente.

Modularidade

Um benefício mais prático da programação funcional é que isso força você a quebrar seu problema em pequenos pedaços. Os programas são mais modulares como resultado. É mais fácil especificar e escrever uma pequena função que faz uma coisa do que uma grande função que realiza uma transformação complicada. Pequenas funções também são mais fáceis de ler e verificar erros.

Fácil de depurar e testar

Testar e depurar um programa de estilo funcional é mais fácil.

A depuração é simplificada porque as funções são geralmente pequenas e claramente especificadas. Quando um programa não funciona, cada função

é um ponto de interface onde você pode verificar se os dados estão corretos. Você pode observar as entradas e saídas intermediárias para isolar rapidamente a função responsável por um erro.

O teste é mais fácil porque cada função é um assunto potencial para um teste unitário. As funções não dependem do estado do sistema que precisa ser replicado antes de executar um teste; Em vez disso, você só precisa sintetizar a entrada certa e depois verificar se o resultado corresponde às expectativas.

Componibilidade

À medida que você trabalha em um programa de estilo funcional, você escreverá várias funções com diferentes entradas e saídas. Algumas dessas funções serão inevitavelmente especializadas em um aplicativo particular, mas outras serão úteis em uma grande variedade de programas. Por exemplo, uma função que leva um caminho de diretório e retorna todos os arquivos XML no diretório, ou uma função que leva um nome de arquivo e retorna seu conteúdo, pode ser aplicada em muitas situações diferentes.

Com o tempo você formará uma biblioteca pessoal de utilitários. Muitas vezes você montará novos programas organizando funções existentes em uma nova configuração e escrevendo algumas funções especializadas para a tarefa atual.

Iteradores

Começarei por olhar para um recurso de linguagem Python que é uma base importante para escrever programas de estilo funcional: iteradores.

Um iterador é um objeto que representa um fluxo de dados; este objeto retorna os dados um elemento por vez. Um iterador Python deve suportar um método chamado `iterator.__next__()` que não leva argumentos e sempre retorna o próximo elemento do fluxo. Se não houver mais elementos no fluxo, `iterator.__next__()` deve aumentar a exceção `StopIteration`. Os iteradores não precisam ser finitos; no entanto, é

perfeitamente razoável escrever um iterador que produza um fluxo infinito de dados.

A função embutida `iter()` leva um objeto arbitrário e tenta retornar um iterador que retornará os conteúdos ou elementos do objeto, caso o contrario retorna `TypeError` se o objeto não suportar a iteração. Vários dos tipos de dados embutidos do Python suportam a iteração, sendo as listas e os dicionários mais comuns. Um objeto é chamado iterable se você pode obter um iterador para ele.

Você pode experimentar a interface de iteração manualmente:

```
>>> L = [1, 2, 3]
>>> it = iter(L)
>>> it
<...iterator object at ...>
>>> it.__next__()  # same as next(it)
1
>>> next(it)
2
>>> next(it)
3
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>>
```

Python espera objetos iteráveis em vários contextos diferentes, sendo o mais importante a instrução `for`. Na declaração `for X in Y`, Y deve ser um iterador ou algum objeto para o qual `iter()` pode criar um iterador. Estas duas declarações são equivalentes:

```
for i in iter(obj):
    print(i)

for i in obj:
    print(i)
```

Iteradores também podem ser materializados como listas ou tuplas, utilizando funções de construção **`list()`** ou **`tuple()`**:

```
>>> L = [1, 2, 3]
>>> iterator = iter(L)
>>> t = tuple(iterator)
>>> t
(1, 2, 3)
```

O desempacotamento de sequência também suporta iteradores: se você sabe que um iterador retornará N elementos, você pode desempacotá-los em uma N-tupla:

```
>>> L = [1, 2, 3]
>>> iterator = iter(L)
>>> a, b, c = iterator
>>> a, b, c
(1, 2, 3)
```

Funções embutidas, tais como **`max()`** e **`min()`** podem ter um único argumento de iterador e retornarão o elemento maior ou menor. Os operadores `"in"` e `"not in"` também aceitam iteradores: `X in iterator` é verdadeiro se X for encontrado no fluxo retornado pelo iterador. Você enfrentará problemas óbvios se o iterador for infinito; **`max()`**, **`min()`** nunca retornará, e se o elemento X nunca aparecer no fluxo, os operadores `"in"` e `"not in"` não retornarão também.

Observe que você só pode avançar em um iterador; não há como obter o elemento anterior, redefinir o iterador ou fazer uma cópia dele. Os objetos iteradores podem opcionalmente fornecer esses recursos adicionais, mas o protocolo do iterador especifica apenas o método **`iterator.__next__()`**. As funções podem, portanto, consumir toda a saída do iterador, e se você precisa fazer algo diferente com o mesmo fluxo, você terá que criar um novo iterador.

Tipos de Dados que Suportam Iteradores

Já vimos como listas e tuplas suportam iteradores. De fato, qualquer tipo de sequência de Python, como strings, suportará automaticamente a criação de

um iterador.

Chamar `iter()` em um dicionário retorna um iterador que irá percorrer as chaves do dicionário:

```
>>> m = {'Jan': 1, 'Fev': 2, 'Mar': 3, 'Abr': 4, 'Mai': 5, 'Jun'
...      'Jul': 7, 'Ago': 8, 'Set': 9, 'Out': 10, 'Nov': 11, 'De
>>> for key in m:
...     print(key, m[key])
Jan 1
Fev 2
Mar 3
Abr 4
Mai 5
Jun 6
Jul 7
Ago 8
Set 9
Out 10
Nov 11
Dez 12
```

Note que a partir de Python 3.7, a ordem de iteração em um dicionário é garantidamente a mesma que a ordem de inserção. Em versões anteriores, o comportamento não era especificado e podia variar entre implementações.

Aplicando `iter()` para um dicionário sempre percorre as teclas, mas os dicionários têm métodos que retornam outros iteradores. Se você deseja iterar sobre valores ou pares de chave/valor, você pode chamar explicitamente os métodos `values()` ou `items()` para obter um iterador apropriado.

O construtor `dict()` pode aceitar um iterador que retorna um fluxo finito de tuplas `(chave, valor)`:

```
>>> L = [('Italy', 'Rome'), ('France', 'Paris'), ('US', 'Washing
>>> dict(iter(L))
{'Italy': 'Rome', 'France': 'Paris', 'US': 'Washington DC'}
```

Os arquivos também suportam a iteração chamando o método **`readline()`** até que não haja mais linhas no arquivo. Isso significa que você pode ler cada linha de um arquivo como este:

```
for line in file:
    # faz algo em cada linha
    ...
```

Os conjuntos podem tirar seus conteúdos de uma iterável e permitir que você faça uma iteração sobre os elementos do conjunto:

```
>>> S = {2, 3, 5, 7, 11, 13}
>>> for i in S:
...     print(i)
2
3
5
7
11
13
```

Expressões do gerador e compreensões de lista

Duas operações comuns na saída de um iterador são 1) executando alguma operação para cada elemento, 2) selecionando um subconjunto de elementos que atendam a alguma condição. Por exemplo, com uma lista de cadeias de caracteres, você pode querer retirar o espaço em branco de cada linha ou extrair todas as sequências de caracteres que contenham uma determinada substring.

As compreensões de lista e as expressões do gerador (forma curta: “listcomps” e “genexps”) são uma notação concisa para tais operações, emprestado da linguagem de programação funcional Haskell (https://www.haskell.org/). Você pode tirar todos os espaços em branco de um fluxo de strings com o seguinte código:

```
>>> line_list = ['  line 1\n', 'line 2  \n', ' \n', '']

>>> # Expressão geradora -- retorna iterador
```

```
>>> stripped_iter = (line.strip() for line in line_list)

>>> # Compreensão de lista -- retorna uma lista
>>> stripped_list = [line.strip() for line in line_list]
```

Você pode selecionar apenas determinados elementos adicionando uma condição `"if"`:

```
>>> stripped_list = [line.strip() for line in line_list
...                  if line != ""]
```

Com uma compreensão de lista, você recebe uma lista Python; `Stripped_list` é uma lista contendo as linhas resultantes, e não um iterador. As expressões do gerador retornam um iterador que calcula os valores conforme necessário, não precisando materializar todos os valores ao mesmo tempo. Isso significa que as compreensões da lista não são úteis se você estiver trabalhando com iteradores que retornam um fluxo infinito ou uma quantidade muito grande de dados. As expressões do gerador são preferíveis nessas situações.

As expressões do gerador são cercadas por parênteses (“()”) e as compreensões da lista são cercadas por colchetes (“[]”). As expressões do gerador têm a forma:

```
( expression for expr in sequence1
             if condition1
             for expr2 in sequence2
             if condition2
             for expr3 in sequence3
             if condition3
             ...
             for exprN in sequenceN
             if conditionN )
```

Novamente, para uma compreensão de lista, apenas os suportes externos são diferentes (colchetes em vez de parênteses).

Os elementos do resultado gerado serão os valores sucessivos de `expression`. As cláusulas `if` são todas opcionais; Se presente, `expression` só é avaliado e adicionado ao resultado quando `condition` é verdadeiro.

As expressões do gerador sempre devem ser escritas dentro de parênteses, mas os parênteses que sinalizam uma chamada de função também contam. Se você quiser criar um iterador que será imediatamente passado para uma função, você pode escrever:

```
obj_total = sum(obj.count for obj in list_all_objects())
```

As cláusulas `for...in` contêm as sequências a serem repetidas. As sequências não precisam ser do mesmo comprimento, porque são iteradas da esquerda para a direita, **não** em paralelo. Para cada elemento em `sequence1`, `sequence2` é enrolado desde o início. `sequence3` é então percorrido para cada par resultante de elementos de `sequence1` e `sequence2`.

Em outras palavras, uma lista de compreensão ou expressão do gerador é equivalente ao seguinte código Python:

```
for expr1 in sequence1:
    if not (condition1):
        continue   # Ignora este elemento
    for expr2 in sequence2:
        if not (condition2):
            continue   # Ignora este elemento
        ...
        for exprN in sequenceN:
            if not (conditionN):
                continue   # Ignora este elemento

            # Emite o valor da expressão,
```

Isso significa que, quando existem várias cláusulas `for...in`, mas não `if`, o comprimento da saída resultante será igual ao produto dos comprimentos de todas as sequências. Se você tiver duas listas de comprimento 3, a lista de saída tem 9 elementos de comprimento:

```
>>> seq1 = 'abc'
>>> seq2 = (1, 2, 3)
>>> [(x, y) for x in seq1 for y in seq2]
[('a', 1), ('a', 2), ('a', 3),
```

```
 ('b', 1), ('b', 2), ('b', 3),
 ('c', 1), ('c', 2), ('c', 3)]
```

Para evitar a introdução de uma ambiguidade na gramática de Python, se `expression` estiver criando uma tupla, ela deve estar cercada de parênteses. A primeira lista de compreensão abaixo é um erro de sintaxe, enquanto o segundo está correto:

```
# Erro de sintaxe
[x, y for x in seq1 for y in seq2]
# Correto
[(x, y) for x in seq1 for y in seq2]
```

Geradores

Os geradores são uma classe especial de funções que simplificam a tarefa de escrever iteradores. As funções convencionais calculam um valor e o retornam, mas os geradores retornam um iterador que retorna um fluxo de valores.

Você está sem dúvida familiarizado com o funcionamento das funções convencionais em Python ou C. Quando você chama uma função, ela recebe um espaço de nome privado onde suas variáveis locais são criadas. Quando a função atinge uma instrução `return`, as variáveis locais são destruídas e o valor retornado ao chamador. Uma chamada posterior para a mesma função cria um novo espaço de nome privado e um novo conjunto de variáveis locais. Mas, e se as variáveis locais não fossem descartadas ao sair de uma função? E se você pudesse retomar a função onde ele deixou? Isto é o que os geradores fornecem; Eles podem ser pensados como funções resumíveis.

Aqui está um exemplo simples de uma função geradora:

```
>>> def generate_ints(N):
...     for i in range(N):
...         yield i
```

Qualquer função contendo um argumento nomeado **`yield`** é uma função de gerador; isso é detectado pelo compilador de bytecode do Python, que

compila a função especialmente como resultado.

Quando você chama uma função de gerador, não retorna um único valor; em vez disso, ele retorna um objeto gerador que suporte o protocolo do iterador. Ao executar a expressão `yield`, o gerador exibe o valor de `i`, semelhante a uma instrução `return`. A grande diferença entre `yield` e uma declaração `return` é que, ao atingir um `yield`, o estado de execução do gerador é suspenso e as variáveis locais são preservadas. Na próxima chamada ao método do gerador **`__next__()`**, a função irá continuar a ser executada.

Aqui está um uso simples do gerador `generate_ints()`

```
>>> gen = generate_ints(3)
>>> gen
<generator object generate_ints at ...>
>>> next(gen)
0
>>> next(gen)
1
>>> next(gen)
2
>>> next(gen)
Traceback (most recent call last):
  File "stdin", line 1, in <module>
  File "stdin", line 2, in generate_ints
StopIteration
```

Você pode igualmente escrever `for i in generate_ints(5)`, ou `a, b, c = generate_ints(3)`.

Dentro de uma função de gerador, `return value` faz com que `StopIteration(value)` seja gerado a partir do método **`__next__()`**. Uma vez que isso acontece, ou a parte inferior da função é alcançada, a procissão dos valores termina e o gerador não pode render mais valores.

Você pode conseguir o efeito de geradores manualmente, escrevendo sua própria classe e armazenando todas as variáveis locais do gerador como variáveis de instância. Por exemplo, retornar uma lista de inteiros pode ser feito configurando `self.count` para 0, e tendo o **`iterator.__next__()`**

incrementar o método `self.count` e devolvê-lo. No entanto, para um gerador moderadamente complicado, escrever uma classe correspondente pode ser muito mais complicado.

O conjunto de teste incluído na biblioteca do Python, lib/test/test_generators.py [https://github.com/python/cpython/tree/3.15/lib/test/test_generators.py], contém vários exemplos mais interessantes. Aqui está um gerador que implementa uma passagem em ordem de uma árvore usando geradores de forma recursiva.

```
# Um gerador recursivo que gera folhas da árvore em ordem.
def inorder(t):
    if t:
        for x in inorder(t.left):
            yield x

        yield t.label

        for x in inorder(t.right):
            yield x
```

Dois outros exemplos no `test_generators.py` produzem soluções para o problema N-Queens (colocando N rainhas em um tabuleiro de xadrez NxN para que nenhuma rainha ameace a outra) e o Passeio do Cavalo (encontrando uma rota que leva um cavalo para cada quadrado de um tabuleiro de xadrez NxN sem visitar nenhum quadrado duas vezes).

Passando valores para um gerador

No Python 2.4 e anteriores, os geradores apenas produziram saída. Uma vez que o código de um gerador foi invocado para criar um iterador, não havia como passar qualquer nova informação na função quando sua execução foi retomada. Você pode cortar essa habilidade fazendo com que o gerador olhe para uma variável global ou passando em algum objeto mutável que os chamadores então modifiquem, mas essas abordagens são desordenadas.

No Python 2.5 há uma maneira simples de passar valores para um gerador. **`yield`** tornou-se uma expressão, retornando um valor que pode ser atribuído a uma variável ou operado de outra forma:

```
val = (yield i)
```

Eu recomendo que você **sempre** coloque parênteses em torno de uma expressão `yield` quando você está fazendo algo com o valor retornado, como no exemplo acima. Os parênteses nem sempre são necessários, mas é mais fácil adicioná-los sempre em vez de ter que lembrar quando são necessários.

(**PEP 342** [https://peps.python.org/pep-0342/] explica as regras exatas, é que uma expressão `yield` deve sempre ser entre parênteses, exceto quando ocorre na expressão de nível superior no lado direito de uma atribuição. Isso significa que você pode escrever `val = yield i`, mas tem que usar parênteses quando há uma operação, como em `val = (yield i) + 12`.)

Os valores são enviados para um gerador chamando seu método **`send(value)`**. Este método retoma o código do gerador e a expressão `yield` retorna o valor especificado. Se o método regular **`__next__()`** for chamado, o `yield` retorna `None`.

Aqui está um contador simples que aumenta em 1 e permite alterar o valor do contador interno.

```
def counter(maximum):
    i = 0
    while i < maximum:
        val = (yield i)
        # Se o valor foi fornecido, altera o contador
        if val is not None:
            i = val
        else:
            i += 1
```

E aqui um exemplo de mudança de contador:

```
>>> it = counter(10)
>>> next(it)
0
>>> next(it)
1
>>> it.send(8)
```

```
8
>>> next(it)
9
>>> next(it)
Traceback (most recent call last):
  File "t.py", line 15, in <module>
    it.next()
StopIteration
```

Porque `yield` retornará frequentemente `None`, você deve verificar sempre este caso. Não use apenas seu valor em expressões, a menos que tenha certeza de que o método **`send()`** será o único método usado para retomar a função do gerador.

Além de **`send()`**, há dois outros métodos em geradores:

- **`throw(value)`** é usado para gerar uma exceção dentro do gerador; a exceção é levantada pela expressão `yield` onde a execução do gerador é pausada.

- **`close()`** envia uma exceção **`GeneratorExit`** ao gerador para encerrar a iteração. Ao receber essa exceção, o código do gerador deve levantar **`GeneratorExit`** ou **`StopIteration`**; capturar a exceção e fazer qualquer outra coisa é ilegal e disparará um **`RuntimeError`**. **`close()`** também será chamado pelo coletor de lixo do Python quando o gerador for coletado como lixo.

  Se você precisar executar um código de limpeza quando ocorrer um **`GeneratorExit`**, sugiro usar um conjunto `try: ... finally:` em vez de capturar **`GeneratorExit`**.

O efeito cumulativo destas mudanças é transformar os geradores de produtores unilaterais de informação em produtores e consumidores.

Geradores também se tornam **corrotinas**, uma forma mais generalizada de subrotinas. As subrotinas são inseridas em um ponto e encerradas em outro (o topo da função e uma instrução `return`), mas as corrotinas podem ser inseridas, encerradas e retomadas em muitos pontos diferentes (as instruções `yield`).

Funções embutidas

Vamos ver com maiores detalhes as funções embutidas que são comumente usadas com iteradores.

Duas das funções embutidas do Python, `map()` e `filter()` duplicam os recursos das expressões geradoras:

`map(f, iterA, iterB, ...)` retorna um iterador sobre a sequência

```
f(iterA[0], iterB[0]), f(iterA[1], iterB[1]), f(iterA[2],
iterB[2]), ....
```

```
>>> def upper(s):
...     return s.upper()
```

```
>>> list(map(upper, ['sentence', 'fragment']))
['SENTENCE', 'FRAGMENT']
>>> [upper(s) for s in ['sentence', 'fragment']]
['SENTENCE', 'FRAGMENT']
```

É claro que você pode alcançar o mesmo efeito com uma compreensão de lista.

`filter(predicate, iter)` retorna um iterador sobre todos os elementos da sequência que atendem a uma determinada condição, e é similarmente duplicada por compreensões de lista. Um **predicate** é uma função que retorna o valor lógico de alguma condição; para uso com **`filter()`**, o predicado deve receber um único valor.

```
>>> def is_even(x):
...     return (x % 2) == 0
```

```
>>> list(filter(is_even, range(10)))
[0, 2, 4, 6, 8]
```

isso também pode ser escrito como uma compreensão de lista:

```
>>> list(x for x in range(10) if is_even(x))
[0, 2, 4, 6, 8]
```

enumerate(iter, start=0) counts off the elements in the iterable returning 2-tuples containing the count (from *start*) and each element.

```
>>> for item in enumerate(['subject', 'verb', 'object']):
...     print(item)
(0, 'subject')
(1, 'verb')
(2, 'object')
```

enumerate() is often used when looping through a list and recording the indexes at which certain conditions are met:

```
f = open('data.txt', 'r')
for i, line in enumerate(f):
    if line.strip() == '':
        print('Blank line at line #%i' % i)
```

sorted(iterable, key=None, reverse=False) collects all the elements of the iterable into a list, sorts the list, and returns the sorted result. The *key* and *reverse* arguments are passed through to the constructed list's **sort()** method.

```
>>> import random
>>> # Generate 8 random numbers between [0, 10000)
>>> rand_list = random.sample(range(10000), 8)
>>> rand_list
[769, 7953, 9828, 6431, 8442, 9878, 6213, 2207]
>>> sorted(rand_list)
[769, 2207, 6213, 6431, 7953, 8442, 9828, 9878]
>>> sorted(rand_list, reverse=True)
[9878, 9828, 8442, 7953, 6431, 6213, 2207, 769]
```

(For a more detailed discussion of sorting, see the Técnicas de ordenação.)

The **any(iter)** and **all(iter)** built-ins look at the truth values of an iterable's contents. **any()** returns `True` if any element in the iterable is a true value, and **all()** returns `True` if all of the elements are true values:

```
>>> any([0, 1, 0])
True
>>> any([0, 0, 0])
```

```
False
>>> any([1, 1, 1])
True
>>> all([0, 1, 0])
False
>>> all([0, 0, 0])
False
>>> all([1, 1, 1])
True
```

`zip(iterA, iterB, ...)` takes one element from each iterable and returns them in a tuple:

```
zip(['a', 'b', 'c'], (1, 2, 3)) =>
  ('a', 1), ('b', 2), ('c', 3)
```

It doesn't construct an in-memory list and exhaust all the input iterators before returning; instead tuples are constructed and returned only if they're requested. (The technical term for this behaviour is lazy evaluation [https://en.wikipedia.org/wiki/Lazy_evaluation].)

This iterator is intended to be used with iterables that are all of the same length. If the iterables are of different lengths, the resulting stream will be the same length as the shortest iterable.

```
zip(['a', 'b'], (1, 2, 3)) =>
  ('a', 1), ('b', 2)
```

You should avoid doing this, though, because an element may be taken from the longer iterators and discarded. This means you can't go on to use the iterators further because you risk skipping a discarded element.

O módulo itertools

The `itertools` module contains a number of commonly used iterators as well as functions for combining several iterators. This section will introduce the module's contents by showing small examples.

The module's functions fall into a few broad classes:

- Funções que criam um novo iterador com base em um iterador existente.
- Funções para tratar os elementos de um iterador como argumentos de funções.
- Funções para selecionar partes da saída de um iterador.
- A function for grouping an iterator's output.

Criando novos iteradores

`itertools.count(start, step)` returns an infinite stream of evenly spaced values. You can optionally supply the starting number, which defaults to 0, and the interval between numbers, which defaults to 1:

```
itertools.count() =>
  0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
itertools.count(10) =>
  10, 11, 12, 13, 14, 15, 16, 17, 18, 19, ...
itertools.count(10, 5) =>
  10, 15, 20, 25, 30, 35, 40, 45, 50, 55, ...
```

`itertools.cycle(iter)` saves a copy of the contents of a provided iterable and returns a new iterator that returns its elements from first to last. The new iterator will repeat these elements infinitely.

```
itertools.cycle([1, 2, 3, 4, 5]) =>
  1, 2, 3, 4, 5, 1, 2, 3, 4, 5, ...
```

`itertools.repeat(elem, [n])` returns the provided element *n* times, or returns the element endlessly if *n* is not provided.

```
itertools.repeat('abc') =>
  abc, abc, abc, abc, abc, abc, abc, abc, abc, abc, ...
itertools.repeat('abc', 5) =>
  abc, abc, abc, abc, abc
```

`itertools.chain(iterA, iterB, ...)` takes an arbitrary number of iterables as input, and returns all the elements of the first iterator, then all the elements of the second, and so on, until all of the iterables have been exhausted.

```
itertools.chain(['a', 'b', 'c'], (1, 2, 3)) =>
  a, b, c, 1, 2, 3
```

`itertools.islice(iter, [start], stop, [step])` returns a stream that's a slice of the iterator. With a single *stop* argument, it will return the first *stop* elements. If you supply a starting index, you'll get *stop-start* elements, and if you supply a value for *step*, elements will be skipped accordingly. Unlike Python's string and list slicing, you can't use negative values for *start*, *stop*, or *step*.

```
itertools.islice(range(10), 8) =>
  0, 1, 2, 3, 4, 5, 6, 7
itertools.islice(range(10), 2, 8) =>
  2, 3, 4, 5, 6, 7
itertools.islice(range(10), 2, 8, 2) =>
  2, 4, 6
```

`itertools.tee(iter, [n])` replicates an iterator; it returns *n* independent iterators that will all return the contents of the source iterator. If you don't supply a value for *n*, the default is 2. Replicating iterators requires saving some of the contents of the source iterator, so this can consume significant memory if the iterator is large and one of the new iterators is consumed more than the others.

```
itertools.tee( itertools.count() ) =>
   iterA, iterB

where iterA ->
   0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...

and   iterB ->
   0, 1, 2, 3, 4, 5, 6, 7, 8, 9, ...
```

Calling functions on elements

The `operator` module contains a set of functions corresponding to Python's operators. Some examples are `operator.add(a, b)` (adds two values), `operator.ne(a, b)` (same as `a != b`), and `operator.attrgetter('id')` (returns a callable that fetches the `.id` attribute).

`itertools.starmap(func, iter)` assumes that the iterable will return a stream of tuples, and calls *func* using these tuples as the arguments:

```
itertools.starmap(os.path.join,
                  [('/bin', 'python'), ('/usr', 'bin', 'java'),
                   ('/usr', 'bin', 'perl'), ('/usr', 'bin', 'rub
=>
  /bin/python, /usr/bin/java, /usr/bin/perl, /usr/bin/ruby
```

Selecionando elementos

Another group of functions chooses a subset of an iterator's elements based on a predicate.

`itertools.filterfalse(predicate, iter)` is the opposite of `filter()`, returning all elements for which the predicate returns false:

```
itertools.filterfalse(is_even, itertools.count()) =>
  1, 3, 5, 7, 9, 11, 13, 15, ...
```

`itertools.takewhile(predicate, iter)` returns elements for as long as the predicate returns true. Once the predicate returns false, the iterator will signal the end of its results.

```
def less_than_10(x):
    return x < 10

itertools.takewhile(less_than_10, itertools.count()) =>
  0, 1, 2, 3, 4, 5, 6, 7, 8, 9

itertools.takewhile(is_even, itertools.count()) =>
  0
```

`itertools.dropwhile(predicate, iter)` discards elements while the predicate returns true, and then returns the rest of the iterable's results.

```
itertools.dropwhile(less_than_10, itertools.count()) =>
  10, 11, 12, 13, 14, 15, 16, 17, 18, 19, ...
```

```
itertools.dropwhile(is_even, itertools.count()) =>
  1, 2, 3, 4, 5, 6, 7, 8, 9, 10, ...
```

`itertools.compress(data, selectors)` takes two iterators and returns only those elements of *data* for which the corresponding element of *selectors* is true, stopping whenever either one is exhausted:

```
itertools.compress([1, 2, 3, 4, 5], [True, True, False, False, T
   1, 2, 5
```

Combinatoric functions

The `itertools.combinations(iterable, r)` returns an iterator giving all possible *r*-tuple combinations of the elements contained in *iterable*.

```
itertools.combinations([1, 2, 3, 4, 5], 2) =>
  (1, 2), (1, 3), (1, 4), (1, 5),
  (2, 3), (2, 4), (2, 5),
  (3, 4), (3, 5),
  (4, 5)

itertools.combinations([1, 2, 3, 4, 5], 3) =>
  (1, 2, 3), (1, 2, 4), (1, 2, 5), (1, 3, 4), (1, 3, 5), (1, 4,
  (2, 3, 4), (2, 3, 5), (2, 4, 5),
  (3, 4, 5)
```

The elements within each tuple remain in the same order as *iterable* returned them. For example, the number 1 is always before 2, 3, 4, or 5 in the examples above. A similar function, `itertools.permutations(iterable, r=None)`, removes this constraint on the order, returning all possible arrangements of length *r*:

```
itertools.permutations([1, 2, 3, 4, 5], 2) =>
  (1, 2), (1, 3), (1, 4), (1, 5),
  (2, 1), (2, 3), (2, 4), (2, 5),
  (3, 1), (3, 2), (3, 4), (3, 5),
  (4, 1), (4, 2), (4, 3), (4, 5),
  (5, 1), (5, 2), (5, 3), (5, 4)
```

```
itertools.permutations([1, 2, 3, 4, 5]) =>
  (1, 2, 3, 4, 5), (1, 2, 3, 5, 4), (1, 2, 4, 3, 5),
  ...
  (5, 4, 3, 2, 1)
```

If you don't supply a value for *r* the length of the iterable is used, meaning that all the elements are permuted.

Note that these functions produce all of the possible combinations by position and don't require that the contents of *iterable* are unique:

```
itertools.permutations('aba', 3) =>
  ('a', 'b', 'a'), ('a', 'a', 'b'), ('b', 'a', 'a'),
  ('b', 'a', 'a'), ('a', 'a', 'b'), ('a', 'b', 'a')
```

The identical tuple `('a', 'a', 'b')` occurs twice, but the two 'a' strings came from different positions.

The **`itertools.combinations_with_replacement(iterable, r)`** function relaxes a different constraint: elements can be repeated within a single tuple. Conceptually an element is selected for the first position of each tuple and then is replaced before the second element is selected.

```
itertools.combinations_with_replacement([1, 2, 3, 4, 5], 2) =>
  (1, 1), (1, 2), (1, 3), (1, 4), (1, 5),
  (2, 2), (2, 3), (2, 4), (2, 5),
  (3, 3), (3, 4), (3, 5),
  (4, 4), (4, 5),
  (5, 5)
```

Agrupando elementos

The last function I'll discuss, **`itertools.groupby(iter, key_func=None)`**, is the most complicated. `key_func(elem)` is a function that can compute a key value for each element returned by the iterable. If you don't supply a key function, the key is simply each element itself.

`groupby()` collects all the consecutive elements from the underlying iterable that have the same key value, and returns a stream of 2-tuples containing a key value and an iterator for the elements with that key.

```
city_list = [('Decatur', 'AL'), ('Huntsville', 'AL'), ('Selma',
             ('Anchorage', 'AK'), ('Nome', 'AK'),
             ('Flagstaff', 'AZ'), ('Phoenix', 'AZ'), ('Tucson',
             ...
            ]

def get_state(city_state):
    return city_state[1]

itertools.groupby(city_list, get_state) =>
  ('AL', iterator-1),
  ('AK', iterator-2),
  ('AZ', iterator-3), ...

where
iterator-1 =>
  ('Decatur', 'AL'), ('Huntsville', 'AL'), ('Selma', 'AL')
iterator-2 =>
  ('Anchorage', 'AK'), ('Nome', 'AK')
iterator-3 =>
  ('Flagstaff', 'AZ'), ('Phoenix', 'AZ'), ('Tucson', 'AZ')
```

`groupby()` assumes that the underlying iterable's contents will already be sorted based on the key. Note that the returned iterators also use the underlying iterable, so you have to consume the results of iterator-1 before requesting iterator-2 and its corresponding key.

O módulo functools

The `functools` module contains some higher-order functions. A **higher-order function** takes one or more functions as input and returns a new function. The most useful tool in this module is the `functools.partial()` function.

For programs written in a functional style, you'll sometimes want to construct variants of existing functions that have some of the parameters filled in. Consider a Python function `f(a, b, c)`; you may wish to create a new function `g(b, c)` that's equivalent to `f(1, b, c)`; you're filling in a

value for one of `f()`'s parameters. This is called "partial function application".

The constructor for **`partial()`** takes the arguments `(function, arg1, arg2, ..., kwarg1=value1, kwarg2=value2)`. The resulting object is callable, so you can just call it to invoke `function` with the filled-in arguments.

Aqui está um pequeno mas bem realístico exemplo:

```
import functools

def log(message, subsystem):
    """Write the contents of 'message' to the specified subsyste
    print('%s: %s' % (subsystem, message))
    ...

server_log = functools.partial(log, subsystem='server')
server_log('Unable to open socket')
```

`functools.reduce(func, iter, [initial value])` cumulatively performs an operation on all the iterable's elements and, therefore, can't be applied to infinite iterables. *func* must be a function that takes two elements and returns a single value. **`functools.reduce()`** takes the first two elements A and B returned by the iterator and calculates `func(A, B)`. It then requests the third element, C, calculates `func(func(A, B), C)`, combines this result with the fourth element returned, and continues until the iterable is exhausted. If the iterable returns no values at all, a **`TypeError`** exception is raised. If the initial value is supplied, it's used as a starting point and `func(initial_value, A)` is the first calculation.

```
>>> import operator, functools
>>> functools.reduce(operator.concat, ['A', 'BB', 'C'])
'ABBC'
>>> functools.reduce(operator.concat, [])
Traceback (most recent call last):
  ...
TypeError: reduce() of empty iterable with no initial value
>>> functools.reduce(operator.mul, [1, 2, 3], 1)
```

```
6
>>> functools.reduce(operator.mul, [], 1)
1
```

If you use `operator.add()` with `functools.reduce()`, you'll add up all the elements of the iterable. This case is so common that there's a special built-in called `sum()` to compute it:

```
>>> import functools, operator
>>> functools.reduce(operator.add, [1, 2, 3, 4], 0)
10
>>> sum([1, 2, 3, 4])
10
>>> sum([])
0
```

For many uses of `functools.reduce()`, though, it can be clearer to just write the obvious `for` loop:

```
import functools
# Instead of:
product = functools.reduce(operator.mul, [1, 2, 3], 1)

# You can write:
product = 1
for i in [1, 2, 3]:
    product *= i
```

A related function is `itertools.accumulate(iterable, func=operator.add)`. It performs the same calculation, but instead of returning only the final result, **`accumulate()`** returns an iterator that also yields each partial result:

```
itertools.accumulate([1, 2, 3, 4, 5]) =>
  1, 3, 6, 10, 15

itertools.accumulate([1, 2, 3, 4, 5], operator.mul) =>
  1, 2, 6, 24, 120
```

O módulo operator

The `operator` module was mentioned earlier. It contains a set of functions corresponding to Python's operators. These functions are often useful in functional-style code because they save you from writing trivial functions that perform a single operation.

Algumas funcionalidades desse módulo são:

- Math operations: `add()`, `sub()`, `mul()`, `floordiv()`, `abs()`, ...
- Logical operations: `not_()`, `truth()`.
- Bitwise operations: `and_()`, `or_()`, `invert()`.
- Comparisons: `eq()`, `ne()`, `lt()`, `le()`, `gt()`, and `ge()`.
- Object identity: `is_()`, `is_not()`.

Consult the operator module's documentation for a complete list.

Pequenas funções e as expressões lambda

When writing functional-style programs, you'll often need little functions that act as predicates or that combine elements in some way.

If there's a Python built-in or a module function that's suitable, you don't need to define a new function at all:

```
stripped_lines = [line.strip() for line in lines]
existing_files = filter(os.path.exists, file_list)
```

If the function you need doesn't exist, you need to write it. One way to write small functions is to use the `lambda` expression. `lambda` takes a number of parameters and an expression combining these parameters, and creates an anonymous function that returns the value of the expression:

```
adder = lambda x, y: x+y

print_assign = lambda name, value: name + '=' + str(value)
```

An alternative is to just use the `def` statement and define a function in the usual way:

```
def adder(x, y):
    return x + y

def print_assign(name, value):
    return name + '=' + str(value)
```

Which alternative is preferable? That's a style question; my usual course is to avoid using `lambda`.

Um dos motivos da minha preferência é que `lambda` é bastante limitado nas funções que pode definir. O resultado deve ser computável como uma única expressão, o que significa que você não pode ter comparações multi-canais `if ... elif ... else` ou `try ... except`. Se você tentar fazer muito em uma declaração `lambda`, você acabará com uma expressão excessivamente complicada que é difícil de ler. Rápido, o que o seguinte código está fazendo?

```
import functools
total = functools.reduce(lambda a, b: (0, a[1] + b[1]), items)[1
```

Você pode descobrir isso, mas leva tempo para desenredar a expressão para descobrir o que está acontecendo. Usar uma breve instrução de `def` aninhada torna as coisas um pouco melhor:

```
import functools
def combine(a, b):
    return 0, a[1] + b[1]

total = functools.reduce(combine, items)[1]
```

Mas seria o melhor de tudo se eu tivesse usado simplesmente um bucle `for`:

```
total = 0
for a, b in items:
    total += b
```

Ou o **`sum()`** embutida e uma expressão do gerador:

```
total = sum(b for a, b in items)
```

Muitas utilizações de `functools.reduce()` são mais claras quando escritas como laços `for`.

Fredrik Lundh sugeriu uma vez o seguinte conjunto de regras para refatoração de usos de `lambda`:

1. Escrever uma função lambda.
2. Escreva um comentário explicando o que o lambda faz.
3. Estude o comentário por um tempo e pense em um nome que capture a essência do comentário.
4. Converta a lambda para uma declaração de definição, usando esse nome.
5. Remover o comentário.

Eu realmente gosto dessas regras, mas você está livre para discordar sobre se esse estilo sem lambda é melhor.

Histórico de Revisão e Reconhecimentos

This HOWTO was originally written by A. M. Kuchling. The author would like to thank the following people for offering suggestions, corrections and assistance with various drafts of this article: Ian Bicking, Nick Coghlan, Nick Efford, Raymond Hettinger, Jim Jewett, Mike Krell, Leandro Lameiro, Jussi Salmela, Collin Winter, Blake Winton.

Versão 0.1: publicado em 30 de junho de 2006.

Versão 0.11: publicado em 1º de julho de 2006. Correções de erros de escrita.

Versão 0.2: publicado em 10 de julho de 2006. Incorporou as seções genexp e listcomp em uma. Correções de erros de escrita.

Versão 0.21: adicionou mais referências sugeridas na lista de correspondência do tutor.

Versão 0.30: Adiciona uma seção no módulo `functional` escrito por Collin Winter; Adiciona seção curta no módulo do operador; Algumas outras

edições.

Referências

Geral

Structure and Interpretation of Computer Programs, by Harold Abelson and Gerald Jay Sussman with Julie Sussman. The book can be found at https://mitpress.mit.edu/sicp. In this classic textbook of computer science, chapters 2 and 3 discuss the use of sequences and streams to organize the data flow inside a program. The book uses Scheme for its examples, but many of the design approaches described in these chapters are applicable to functional-style Python code.

https://defmacro.org/2006/06/19/fp.html: A general introduction to functional programming that uses Java examples and has a lengthy historical introduction.

https://pt.wikipedia.org/wiki/Programação_funcional: Informação geral do Wikipédia para descrever programação funcional.

https://pt.wikipedia.org/wiki/Corrotina: Entrada para corrotinas.

https://en.wikipedia.org/wiki/Partial_application: Entry for the concept of partial function application.

https://pt.wikipedia.org/wiki/Currying: Entrada para o conceito de currying.

Python-specific

https://gnosis.cx/TPiP/: The first chapter of David Mertz's book *Text Processing in Python* discusses functional programming for text processing, in the section titled "Utilizing Higher-Order Functions in Text Processing".

Mertz also wrote a 3-part series of articles on functional programming for IBM's DeveloperWorks site; see part 1 [https://web.archive.org/web/20211006103639/https://developer.ibm.com/articles/l-prog/], part 2

[https://web.archive.org/web/20211205224606/https://developer.ibm.com/tutorials/l-prog2/], and part 3 [https://web.archive.org/web/20211127083846/https://developer.ibm.com/tutorials/l-prog3/].

Documentação do Python

Documentação para o módulo `itertools`.

Documentation for the `functools` module.

Documentação para o módulo `operator`.

PEP 289 [https://peps.python.org/pep-0289/]: “Gerador de Expressões”

PEP 342 [https://peps.python.org/pep-0342/]: “Coroutines via Enhanced Generators” descreve os novos recursos do gerador no Python 2.5.

Logging

Essa página contém informações relativas a tutoriais. Para links de informações de referência e livro de receitas de logging, por favor veja Outros recursos.

Tutorial Básico de Logging

Logging é uma maneira de rastrear eventos que acontecem quando algum software é executado. O desenvolvedor de software adiciona chamadas de logging no código para indicar que determinado evento ocorreu. Um evento é definido por uma mensagem descritiva que pode opcionalmente conter o valor de uma variável (ex.: um dado que é potencialmente diferente pra cada ocorrência do evento). Eventos também possuem importâncias atribuídas pelo desenvolvedor; esta importância pode ser chamada de *nível* ou *severidade*.

Quando usar logging

Você pode usar a funcionalidade de registro de eventos criando um registrador com `logger = logging.getLogger(__name__)` e, em seguida, chamar os métodos `debug()`, `info()`, `warning()`, `error()` e `critical()`. Para determinar a necessidade ou não de usar logging, e escolher o método adequado, consulte a tabela abaixo. Ela indica, para cada conjunto de tarefas comuns, a melhor ferramenta a ser usada.

Tarefa que você deseja executar	A melhor ferramenta para a tarefa
Exibir uma mensagem na saída do console para uso ordinário de um script de linha de comando (CLI) ou programa.	`print()`
Expor eventos que possam ocorrer durante a operação normal de um programa (por exemplo, para monitorar o estado atual de execução ou poder investigar falhas)	Utilize o método `info()` de um registrador (ou o método `debug()`, no caso de mensagens mais detalhadas para diagnóstico e investigação de erros)
Emitir um aviso sobre um evento que ocorre em um ambiente de execução específico	Utilize a função `warnings.warn()` da biblioteca padrão se o problema for evitável, e caso a aplicação deva ser modificada pelo usuário para eliminar o alerta. Utilize o método `warning()` de um registrador, caso não haja nada que o usuário possa fazer na aplicação, mas o evento ainda deva ser registrado

Tarefa que você deseja executar	A melhor ferramenta para a tarefa
Expor um erro sobre um evento em um ambiente de execução específico	Levante uma exceção
Expor a supressão de um erro sem levantar uma exceção (por exemplo, ao utilizar um tratador de erro em um processo de longa execução)	Um dos métodos `error()`, `exception()` ou `critical()` do registrador, conforme apropriado para o erro específico e domínio da aplicação

Os métodos dos registradores são nomeados de acordo com o nível ou grau de severidade dos eventos que são rastreados. Os níveis padrão e sua aplicabilidade são descritos abaixo (em ordem crescente de severidade):

Nível	Quando é usado
`DEBUG`	Informação detalhada, tipicamente utilizável apenas para diagnosticar problemas.
`INFO`	Confirmação de que as coisas estão funcionando como esperado.
`WARNING`	Uma indicação de que algo inesperado aconteceu ou de que poderá haver algum problema em um futuro próximo (por exemplo: ‘pouco espaço em disco’). O software ainda está funcionando como esperado.
`ERROR`	Devido de um problema mais grave, o software não conseguiu executar alguma função.
`CRITICAL`	Um erro grave, indicando que o programa pode não conseguir continuar rodando.

O nível padrão é `WARNING`, o que significa que somente os eventos desse nível de severidade e superiores serão registrados, a menos que o pacote logging esteja configurado para fazer o contrário.

Eventos rastreados podem ser tratados de diferentes formas. O jeito mais simples de lidar com eventos rastreados é exibi-los no console. Outra maneira comum é gravá-los em um arquivo de disco.

Um exemplo simples

Um exemplo bastante simples é:

```
import logging
logging.warning('Cuidado!')  # exibirá uma mensagem no console
logging.info('Eu te avisei')  # não exibirá nada
```

Se você colocar essas linhas no script e executá-lo, você verá:

```
WARNING:root:Cuidado!
```

no console. A mensagem `INFO` não aparece porque o nível padrão de logging é `WARNING` . A mensagem exibida inclui a indicação do nível e a descrição do evento fornecido na chamada de logging. Ou seja, 'Cuidado!'. A saída pode ser formatada de maneira bastante flexível, se necessário; as opções de formatação também serão explicadas mais tarde.

Observe que, neste exemplo, utilizamos funções diretamente do módulo `logging`, como `logging.debug`, em vez de criar uma instância de Logger e chamar funções dela. Estas funções operam no logger raiz, mas pode ser útil chamar **`basicConfig()`** caso ainda não tenha sido chamado, como neste exemplo. No entanto, para programas maiores, você normalmente precisará controlar explicitamente a configuração de logging. Portanto, no geral, é melhor instanciar loggers e chamar seus métodos.

Logging em um arquivo

Uma situação muito comum é a de registrar eventos de log em um arquivo, então vamos dar uma olhada nisso a seguir. Teste o código a seguir em um interpretador Python recém-iniciado. Não continue a partir da sessão usada para o código anterior.

```
import logging
logger = logging.getLogger(__name__)
logging.basicConfig(filename='exemplo.log', encoding='utf-8', level=logging.DEBUG)
logger.debug('Esta mensagem aparecerá no arquivo de log')
logger.info('Esta também')
logger.warning('E também essa')
logger.error('Além de palavras com caracteres não-ASCII, como Øresund e Malmö')
```

Alterado na versão 3.9: O argumento de codificação *encoding* foi adicionado. Em versões anteriores do Python, a codificação utilizada por padrão seria definida em **`open()`**. Embora não seja mostrado no exemplo acima, um argumento de *erro* pode ser utilizado, o que determina como o codificação de erros são tratadas. Para saber mais sobre essas possibilidades, consulte a documentação de **`open()`**.

E agora se nós abrirmos o arquivo e olharmos o que temos, deveremos encontrar essas mensagens de log:

```
DEBUG:__main__:Esta mensagem aparecerá no arquivo de log
INFO:__main__:Esta também
WARNING:__main__:E também essa
ERROR:__main__:Além de palavras com caracteres não-ASCII, como Øresund e Malmö
```

Este exemplo também mostra como você pode configurar o nível do logging , que funcionará como um limite para o rastreamento. Neste caso, como definimos o nível limite como `DEBUG`, todas as mensagens foram exibidas.

Você também pode definir o nível de logging a partir de um parâmetro pela linha de comando (CLI):

```
--log=INFO
```

e se você quiser obter o valor do parâmetro `--log` em algum ponto do código, você pode usar:

```
getattr(logging, loglevel.upper())
```

para obter o valor que você passará para a `basicConfig()` pelo argumento de nível *level*. Isso pode ser útil para verificar erros introduzidos pelo usuário, como no exemplo a seguir:

```
# presumindo que a variável loglevel está atrelada ao valor da string obtida
# pelo argumento de linha de comando; Converter para caixa alta permite ao usuário
# declarar tanto --log=DEBUG quanto--log=debug
nivel_numerico = getattr(logging, loglevel.upper(), None)
if not isinstance(nivel_numerico , int):
    raise ValueError('Nível de log invalido: %s' % loglevel)
logging.basicConfig(level=nivel_numerico, ...)
```

A chamada para `basicConfig()` deve vir *antes* de qualquer chamada para métodos de um logger, como `debug()`, `info()`, etc. Caso contrário, o logging deste evento pode não ser tratado da forma desejada.

Se você executar o script acima diversas vezes, as mensagens das sucessivas execuções serão acrescentadas ao arquivo *exemplo.log*. Se você quiser que cada execução seja iniciada novamente, não guardando as mensagens das execuções anteriores, você pode definir o argumento *filemode*, mudando a chamada no exemplo acima para:

```
logging.basicConfig(filename='exemplo.log', filemode='w', level=logging.DEBUG)
```

A saída será a mesma de antes, mas o arquivo de log não será mais incrementado. Desta forma, as mensagens de execuções anteriores serão perdidas.

Logging de variáveis dinâmicas

Para logar o dado de uma variável, use o formato string para a mensagem descritiva do evento e adicione a variável como argumento. Exemplo:

```
import logging
logging.warning('%s onde você %s', 'Olhe', 'anda!')
```

exibirá:

```
WARNING:root:Olhe onde você anda!
```

Como você pode ver, para combinar uma variável de dados na mensagem descritiva do evento usamos o velho operador %, formatador de strings. Isto é usado para garantir compatibilidade com as versões anteriores: os pacotes logging mais recentes contam com opções de formatação como `str.format()` e `string.Template`. Mas explorar estes métodos está fora do escopo deste tutorial. Veja Using particular formatting styles throughout your application para mais informações.

Alterar o formato das mensagens exibidas

Para mudar o formato usado para exibir mensagens, você precisa especificar o formato que quer usar:

```
import logging
logging.basicConfig(format='%(levelname)s:%(message)s', level=logging.DEBUG)
logging.debug('Esta mensagem aparecerá no console')
```

```
logging.info('Esta também')
logging.warning('E também essa')
```

que vai exibir:

```
DEBUG:Esta mensagem aparecerá no console
INFO:Esta também
WARNING:E também essa
```

Note que a palavra ‘root’ que apareceu nos exemplos anteriores desapareceu. Para todas as configurações que possam aparecer na formatação de strings, você pode consultar a documentação Atributos LogRecord, mas para uso simples, você só precisa do *levelname* (severidade), *message* (descrição do evento, incluindo a variável com dados) e talvez exibir quando o evento ocorreu. Isto esta descrito na próxima seção.

Exibindo data/hora em mensagens

Para exibir a data e hora de um evento, você pode colocar *%(asctime)s* na sua string de formato:

```
import logging
logging.basicConfig(format='%(asctime)s %(message)s')
logging.warning('foi quando este evento foi registrado.')
```

que deve exibir algo assim:

```
2010-12-12 11:41:42,612 foi quando este evento foi registrado.
```

O formato padrão para data/hora (mostrado abaixo) é a ISO8601 ou a **RFC 3339** [https://datatracker.ietf.org/doc/html/rfc3339.html]. Se você precisar de mais controle sobre a formatação de data/hora, defina o argumento *datefmt* na função `basicConfig`, como neste exemplo:

```
import logging
logging.basicConfig(format='%(asctime)s %(message)s', datefmt='%m/%d/%Y %I:%M:%S %p')
logging.warning('foi quando este evento foi registrado.')
```

que deve exibir algo assim:

```
12/12/2010 11:46:36 AM foi quando este evento foi registrado.
```

O formato do argumento *datefmt* é o mesmo suportado por `time.strftime()`.

Próximos Passos

Concluímos aqui o tutorial básico. Isto deve ser o bastante para você começar a trabalhar com logging. Existe muito mais que o pacote de logging pode oferecer, mas para ter o melhor disto, você precisará investir um pouco mais do seu tempo lendo as próximas seções. Se você está pronto para isso, pegue sua bebida favorita e continue.

Se as suas necessidades de logging forem simples, use os exemplos acima para incorporar o registro em seus próprios scripts e, se tiver problemas ou não entender alguma coisa, publique uma pergunta na categoria Help do fórum de discussão Python [https://discuss.python.org/c/help/7]. Você deverá receber ajuda em breve.

Ainda por aqui? Você pode continuar lendo as seções seguintes, que tem um tutorial mais avançado que o básico acima. Depois disso, você pode dar uma olhada no Livro de receitas do logging.

Tutorial Avançado do Logging

A biblioteca de logging tem uma abordagem modular e oferece algumas categorias de componentes: loggers, manipuladores, filtros, e formatadores.

- Os loggers oferecem uma interface para que o código da aplicação possa registrar eventos.
- Manipuladores enviam os registros do evento (criados por loggers) aos destinos apropriados.
- Filtros facilitam granularmente a determinação de quais registros de eventos devem ser exibidos na saída.
- Formatadores especificam a composição visual do log de eventos ao ser exibido.

Um evento, quando registrado, passa por loggers, manipuladores, filtros e formatadores através de uma instância de `LogRecord`

Para executar uma ação de logging, é necessário criar instâncias da classe `Logger` (também chamadas de *loggers*), e invocar seus métodos. Cada instância tem um nome, e elas são conceitualmente organizadas em uma hierarquia de espaço de nomes utilizando pontos como separadores. Por exemplo, um logger nomeado como ‘leitor’ é o pai do logger ‘leitor.texto’, ‘leitor.html’ e ‘leitor.pdf’. Você pode nomear o logger do jeito que preferir, e indicar o trecho da aplicação que origina a mensagem de log.

Uma boa prática ao nomear loggers é criar, em cada módulo que usa logging, um logger definido no mesmo nível do módulo. Como no exemplo abaixo:

```
logger = logging.getLogger(__name__)
```

Isso significa que os nomes dos loggers acompanham a hierarquia de pacotes e módulos, tornando intuitivo saber de onde os eventos foram registrados apenas olhando o nome do logger.

O início da hierarquia de loggers é chamado de logger raiz. É ele que é usado pelas funções `debug()`, `info()`, `warning()`, `error()` e `critical()`, que apenas chamam o método de mesmo nome dentro do logger raiz. Essas funções, e seus métodos correspondentes, têm a mesma assinatura (mesmos parâmetros aceitos). Na saída do log, o nome do logger raiz aparece como ‘root’.

É possível registrar mensagens de log em diferentes destinos. O pacote já traz suporte para enviar logs para arquivos, URLs via HTTP GET/POST, e-mails via SMTP, soquetes genéricos, filas ou mecanismos de log específicos do sistema operacional, como o syslog no Unix/Linux ou o Event Log do Windows NT. Esses destinos são gerenciados por classes chamadas manipuladores. Se precisar de um destino especial que não seja atendido pelos manipuladores embutidos, você poderá criar sua própria classe manipuladora para destinar corretamente o log.

Por padrão, nenhum destino é definido para nenhuma mensagem de registro. É possível especificar um destino (como console, ou arquivo) usando `basicConfig()`, como nos exemplos do tutorial. Se você chamar as funções `debug()` , `info()`, `warning()`, `error()` e `critical()`, elas verificarão se o destino está definido; e, caso não esteja, elas definirão o console como o destino (`sys.stderr`) e definirão o formato utilizando um formato padrão. Só então, enviarão a mensagem ao logger raiz, para que seja registrada na saída.

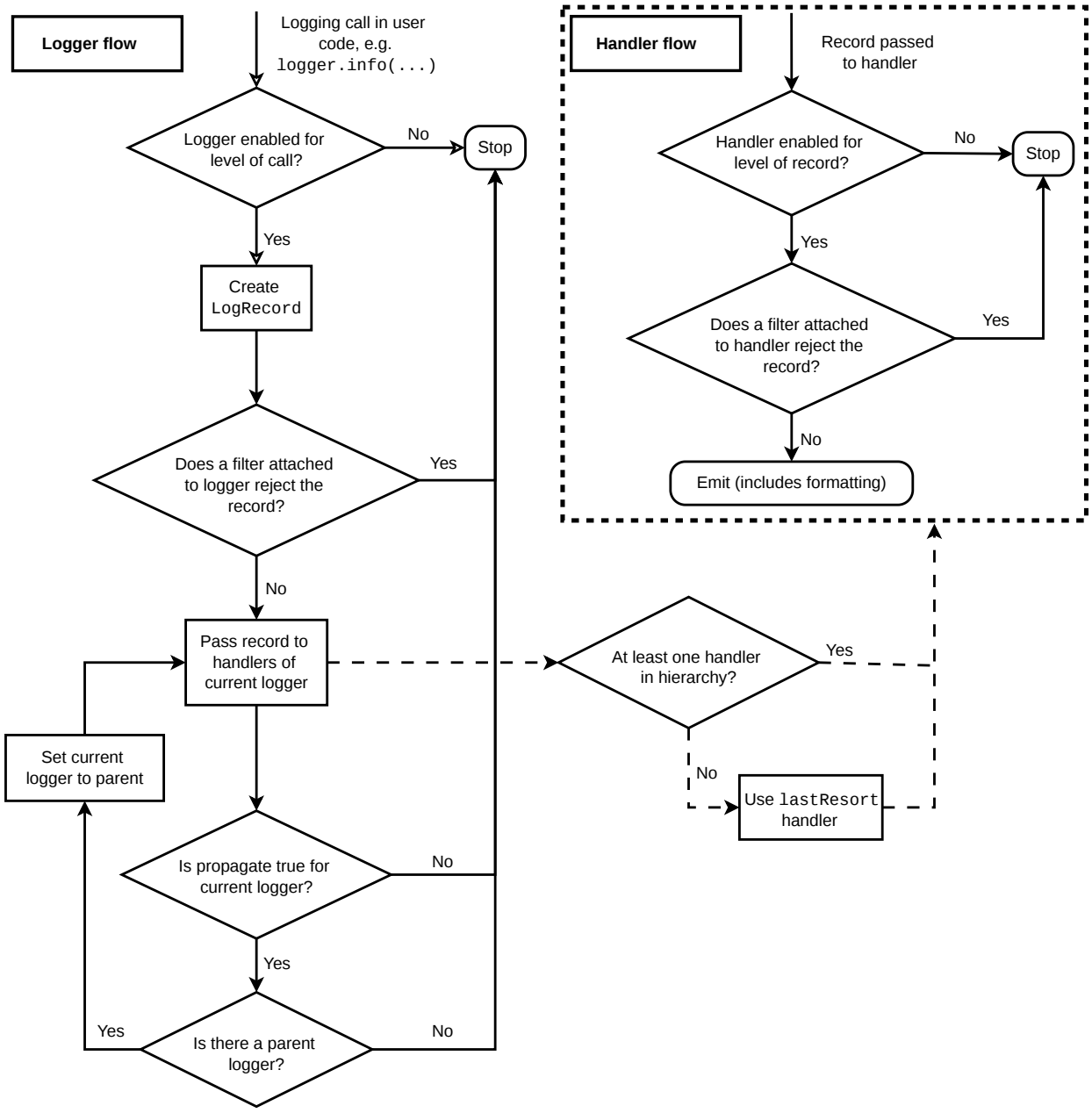
O formato padrão definido por `basicConfig()` para mensagens é:

```
severity:logger name:message
```

Você pode alterar isso passando uma string de formato para `basicConfig()`, através do argumento nomeado *format*. Para sobre como uma string de formato é construída, consulte Formatter Objects.

Fluxo de Logging

O fluxo dos eventos de log em loggers e manipuladores é ilustrado no diagrama a seguir.



Registradores

Objetos `Logger` têm três funções principais. Primeiro, eles expõem vários métodos para que o código da aplicação possa registrar mensagens durante a execução. Segundo, eles decidem quais mensagens de log devem ser processadas, com base no nível de severidade (é o filtro padrão) ou em objetos filtro. Terceiro, eles encaminham as mensagens de log relevantes para todos os manipuladores de log interessados.

Os métodos mais utilizados em objetos logger se enquadram em duas categorias: configuração e envio de mensagem.

Esses são os métodos de configuração mais comuns:

- `Logger.setLevel()` especifica o nível mínimo de severidade que um logger vai lidar. O nível DEBUG é o mais baixo, e o CRITICAL é o mais alto entre os níveis padrão. Por exemplo, se o nível do logger for INFO, ele vai processar apenas mensagens de nível INFO, WARNING, ERROR e CRITICAL, e vai ignorar mensagens DEBUG.
- `Logger.addHandler()` e `Logger.removeHandler()` adicionam e removem objetos manipuladores do objeto logger. Manipuladores serão aprofundados em Manipuladores.
- `Logger.addFilter()` e `Logger.removeFilter()` adicionam e removem objetos filtros do objeto logger. Filtros serão aprofundados em Filter Objects.

Você não precisa sempre invocar esses métodos em cada logger que criar. Consulte os últimos dois paragrafos nessa mesma seção.

Com o objeto logger configurado, os seguintes métodos criam mensagens de log:

- `Logger.debug()`, `Logger.info()`, `Logger.warning()`, `Logger.error()` e `Logger.critical()` criam logs com uma mensagem em um nível correspondente ao respectivo nome do método. A mensagem é uma string de formato, que pode conter a sintaxe padrão de substituições como `%s`, `%d` e `%f`. O restante dos argumentos são uma lista de objetos que correspondem aos campos de substituição da mensagem. No caso do parâmetro `**kwargs`, os métodos de logging cuidam apenas da chave `exc_info`. É essa chave que eles usam para decidir se devem ou não registrar os detalhes de uma exceção.
- `Logger.exception()` cria uma mensagem de log parecida com `Logger.error()`. A diferença é que, além da mensagem, **`Logger.exception()`** também registra o stack trace (situação da pilha de execução). Chame este método apenas a partir de um manipulador de exceção.
- `Logger.log()` recebe o nível de log como um argumento explícito. Isso torna o registro de mensagens um pouco mais verboso do que usar os métodos prontos para cada nível, com os métodos listados anteriormente, mas é uma forma de registrar mensagens em níveis de log personalizados.

`getLogger()` retorna uma referência para uma instância de um logger com o nome especificado, caso informado, ou para o logger raiz (caso nenhum nome seja passado). Os nomes seguem uma estrutura hierárquica separada por pontos. Chamadas repetidas a **`getLogger()`** com o mesmo nome retornam sempre uma referência para a mesma instância do objeto logger. Loggers que aparecem mais embaixo na hierarquia são filhos dos loggers que estão mais acima nessa lista. Por exemplo, dado um logger com nome `foo`, loggers com os nomes `foo.bar`, `foo.bar.baz` e `foo.bam` serão todos descendentes de `foo`.

Loggers têm o conceito de nível efetivo. Se um logger não tiver um nível definido explicitamente, ele herdará o nível do seu logger pai. Caso o pai também não tenha um nível definido, o processo continua subindo na hierarquia, examinando cada ancestral, até encontrar um logger com nível configurado. O

logger raiz sempre terá um nível explícito configurado (por padrão, `WARNING`). Ao decidir se um evento será processado, o logger usa o seu nível efetivo para determinar se o evento deve ser repassado para os manipuladores responsáveis.

Loggers filhos propagam suas mensagens para os manipuladores associados aos seus loggers ancestrais. Por causa disso, não é necessário definir e configurar manipuladores para todos os loggers usados em uma aplicação. Basta configurar os manipuladores em um logger de nível superior, e criar loggers filhos conforme a necessidade. No entanto, é possível desativar essa propagação definindo o atributo *propagate* do logger como `False`.

Manipuladores

Objetos **`Handler`** são responsáveis por enviar as mensagens de log apropriadas (com base na severidade de cada mensagem) para o destino especificado do manipulador. Objetos **`Logger`** podem adicionar um ou mais manipuladores a si mesmos usando o método **`addHandler()`**. Em um cenário hipotético, uma aplicação pode querer "enviar todas as mensagens de log para um arquivo", "enviar apenas mensagens de erro ou mais graves para o *stdout*" e "enviar apenas mensagens críticas para um endereço de e-mail". Esse cenário exigiria três manipuladores diferentes, cada um responsável por encaminhar mensagens de uma determinada gravidade para um destino específico.

A biblioteca padrão possui alguns tipos de manipuladores (veja Handlers úteis); os tutoriais usam principalmente **`StreamHandler`** e **`FileHandler`** em seus exemplos.

Existem pouquíssimos métodos em um manipulador com os quais desenvolvedores de aplicações precisam se preocupar. Para quem está usando manipuladores já inclusos na biblioteca (ou seja, sem criar manipuladores personalizados), os únicos métodos realmente relevantes são os métodos de configuração a seguir:

- O método **`setLevel()`**, assim como acontece nos objetos logger, , define o menor nível de severidade das mensagens que serão encaminhadas para o destino adequado. Mas por que existem dois métodos **`setLevel()`**? Isso acontece porque o nível configurado no logger define quais mensagens ele vai repassar para seus manipuladores. Já o nível configurado em cada manipulador define, entre as mensagens recebidas, quais realmente serão enviadas adiante.
- **`setFormatter()`** define um objeto Formatador para que o manipulador utilize.
- **`addFilter()`** e **`removeFilter()`**, respectivamente, adicionam e removem objetos filtros nos manipuladores.

O código da aplicação não deve instanciar nem usar diretamente objetos da classe **`Handler`**. Na verdade, **`Handler`** é uma classe base que define a interface que todos os manipuladores devem implementar e estabelece um comportamento padrão que pode ser aproveitado (ou sobrescrito) pelas classes filhas.

Formatadores

Objetos Formatadores definem a ordem, a estrutura e o conteúdo final da mensagem de log. Diferentemente da classe base **`logging.Handler`**, o código da aplicação pode instanciar classes de formatadores — embora também seja possível criar subclasses, caso sua aplicação precise de um comportamento específico. O construtor aceita três argumentos opcionais: uma string de formato da mensagem, uma string de formatação da data e um indicador de estilo.

logging.Formatter.__init__(*fmt=None*, *datefmt=None*, *style='%'*)

Se nenhuma string de formato for fornecida na mensagem, o padrão é usar a mensagem bruta. Se nenhuma string de formato de data for definida, o formato de data padrão será:

```
%Y-%m-%d %H:%M:%S
```

com os milissegundos acrescentados ao final. O parâmetro `style` pode ser `'%'`, `'{'` ou `'$'`. Se nenhum deles for especificado, o padrão utilizado será `'%'`.

Se o o parâmetro `style` for `'%'`, a string de formato da mensagem substituirá a string usando o estilo `%(<chave de dicionário>)s`; as chaves possíveis estão documentadas em Atributos LogRecord. Se o estilo for `'{'`, a string de formato é presumida como compatível com **`str.format()`** (usando argumentos nomeados). Já se o estilo for `'$'`, a string de formato deverá seguir o que é esperado por **`string.Template.substitute()`**.

Alterado na versão 3.2: Adicionado o parâmetro `style`.

A string de formato a seguir vai escrever o tempo em um formato legível para humanos, a severidade da mensagem, e o conteúdo da mensagem, nessa ordem:

```
'%(asctime)s - %(levelname)s - %(message)s'
```

Formatadores usam uma função configurável pelo usuário para converter o horário de criação de um registro em uma tupla. Por padrão, é utilizada a função **`time.localtime()`**; Se você quiser alterar isso apenas para uma instância específica de formatador, basta definir o atributo `converter` dessa instância para uma função com a mesma assinatura de **`time.localtime()`** ou **`time.gmtime()`**. Já se a ideia for alterar o comportamento para todos os formatadores (por exemplo, para exibir todos os horários de log em GMT), defina o atributo `converter` diretamente na classe Formatadora (usando, por exemplo, `time.gmtime` para exibição em GMT).

Configurando Logging

Programadores podem configurar logging de três formas:

1. Criando loggers, manipuladores e formatadores de forma explícita, usando código Python que invoca os métodos de configuração mencionados acima.
2. Criando um arquivo de configuração de logging e lendo-o usando a função **`fileConfig()`**.
3. Criando um dicionário com as informações de configuração e passando-o para a função **`dictConfig()`**.

Para a documentação de referência sobre as duas últimas opções, consulte Funções de configuração. O exemplo a seguir configura um logger bem simples, um manipulador de console e um formatador simples utilizando código Python:

```
import logging

# cria o logger
logger = logging.getLogger('exemplo_simples')
logger.setLevel(logging.DEBUG)

# cria o manipulador de console e configura o nível para depuração
```

```
ch = logging.StreamHandler()
ch.setLevel(logging.DEBUG)

# cria o formatador
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelname)s - %(message)s')

# adiciona o formatador ao manipulador
ch.setFormatter(formatter)

# adiciona o manipulador ao logger
logger.addHandler(ch)

# 'código' da aplicação
logger.debug('mensagem de depuração')
logger.info('mensagem informativa')
logger.warning('mensagem de alerta')
logger.error('mensagem de erro')
logger.critical('mensagem crítica')
```

A execução deste módulo pela linha de comando produz a seguinte saída:

```
$ python simple_logging_module.py
2005-03-19 15:10:26,618 - exemplo_simples - DEBUG - mensagem de depuração
2005-03-19 15:10:26,620 - exemplo_simples - INFO - mensagem informativa
2005-03-19 15:10:26,695 - exemplo_simples - WARNING - mensagem de alerta
2005-03-19 15:10:26,697 - exemplo_simples - ERROR - mensagem de erro
2005-03-19 15:10:26,773 - exemplo_simples - CRITICAL - mensagem crítica
```

O módulo Python a seguir cria um logger, um manipulador e um formatador quase idênticos aos do exemplo mostrado acima, sendo a única diferença os nomes dos objetos:

```
import logging
import logging.config

logging.config.fileConfig('logging.conf')

# criação do logger
logger = logging.getLogger('simpleExample')

# código da 'aplicação'
logger.debug('mensagem de debug')
logger.info('mensagem informativa')
logger.warning('mensagem de cuidado')
logger.error('mensagem de erro')
logger.critical('mensagem crítica')
```

Aqui está o arquivo logging.conf:

```
[loggers]
keys=root,exemploSimples

[handlers]
keys=manipuladorConsole

[formatters]
keys=formatadorSimples
```

```
[logger_root]
level=DEBUG
handlers=manipuladorConsole

[logger_exemploSimples]
level=DEBUG
handlers=manipuladorConsole
qualname=exemploSimples
propagate=0

[handler_manipuladorConsole]
class=StreamHandler
level=DEBUG
formatter=formatadorSimples
args=(sys.stdout,)

[formatter_formatadorSimples]
format=%(asctime)s - %(name)s - %(levelname)s - %(message)s
```

A saída é quase idêntica à do exemplo não baseado em arquivo de configuração:

```
$ python simple_logging_config.py
2005-03-19 15:38:55,977 - exemploSimples - DEBUG - mensagem de depuração
2005-03-19 15:38:55,979 - exemploSimples - INFO - mensagem informativa
2005-03-19 15:38:56,054 - exemploSimples - WARNING - mensagem de alerta
2005-03-19 15:38:56,055 - exemploSimples - ERROR - mensagem de erro
2005-03-19 15:38:56,130 - exemploSimples - CRITICAL - mensagem crítica
```

A abordagem de usar um arquivo de configuração oferece algumas vantagens em relação ao uso direto de código Python. Ela permite separar a configuração do código, facilitando a manutenção, e também possibilita que pessoas sem conhecimentos de programação modifiquem facilmente as propriedades de logging apenas editando o arquivo de configuração.

Aviso

A função **`fileConfig()`** possui um parâmetro chamado `disable_existing_loggers`, que por padrão é definido como `True` por questões de retrocompatibilidade. No entanto, esse comportamento pode não ser o desejado, pois todos os loggers que não o raiz que já existiam antes da chamada de **`fileConfig()`** serão desativados, a menos que eles (ou algum de seus ancestrais) sejam explicitamente nomeados na configuração. Caso queira manter os loggers preexistentes ativos, basta definir esse parâmetro como `False`. Para maiores detalhes, consulte a documentação oficial de referência.

O dicionário passado para **`dictConfig()`** também pode especificar um valor booleano com a chave `disable_existing_loggers`. Se essa chave não for definida explicitamente no dicionário, o valor padrão será interpretado como `True`. Isso resulta no mesmo comportamento de desativação de loggers descrito anteriormente, o que pode não ser o desejado. Nesse caso, basta informar a chave no dicionário com o valor `False`.

Observe que os nomes de classe referenciados em arquivos de configuração precisam ser relativos ao módulo logging, ou valores absolutos que possam ser resolvidos pelos mecanismos normais de importação do Python. Por exemplo, você pode usar **`WatchedFileHandler`** (relativo ao módulo

logging) ou `mypackage.mymodule.MyHandler` (para uma classe definida no pacote `mypackage` e no módulo `mymodule`, desde que `mypackage` esteja disponível no caminho de importação do Python).

A partir do Python 3.2, foi introduzida uma nova forma de configurar o logging, utilizando dicionários para armazenar as informações de configuração. Essa abordagem oferece um conjunto ampliado de funcionalidades em relação ao método baseado em arquivos de configuração mencionado anteriormente, sendo, portanto, a forma recomendada de configuração para novas aplicações e implantações. Como a configuração é mantida em um dicionário Python, há maior flexibilidade sobre as opções de como esse dicionário pode ser construído. É possível, por exemplo, utilizar um arquivo em formato JSON ou YAML, construir o dicionário diretamente em código Python, recebê-lo em forma serializada com pickle por meio de um soquete, ou ainda empregar qualquer outro método que faça sentido para a aplicação em questão.

Aqui está um exemplo da mesma configuração acima, em formato YAML, para a nova abordagem baseada em dicionário:

```
version: 1
formatters:
  simple:
    format: '%(asctime)s - %(name)s - %(levelname)s - %(message)s'
handlers:
  console:
    class: logging.StreamHandler
    level: DEBUG
    formatter: simple
    stream: ext://sys.stdout
loggers:
  simpleExample:
    level: DEBUG
    handlers: [console]
    propagate: no
root:
  level: DEBUG
  handlers: [console]
```

Para mais informações sobre logging com dicionário, consulte Funções de configuração.

O que acontece se nenhuma configuração é fornecida

Se nenhuma configuração de logging for fornecida, pode ocorrer uma situação em que um evento de logging precise ser registrado, mas nenhum manipulador esteja disponível para processar e exibir esse evento.

O evento é exibido utilizando um “manipulador de último recurso”, armazenado em **`lastResort`**. Esse manipulador interno não está associado a nenhum logger e funciona como um **`StreamHandler`**, enviando a mensagem de descrição do evento para o valor atual de `sys.stderr` (espeitando, portanto, quaisquer redirecionamentos que estejam em vigor). Nenhuma formatação é aplicada à mensagem — apenas a descrição bruta do evento é impressa. O nível desse manipulador é definido como `WARNING`, de forma que todos os eventos nesse nível ou em níveis mais severos serão exibidos.

Alterado na versão 3.2: Para versões do Python anteriores à 3.2, o comportamento é o seguinte:

- Se **`raiseExceptions`** for `False` (modo em produção), o evento será silenciosamente descartado.

- Se `raiseExceptions` for definido como `True` (modo de desenvolvimento), a mensagem ‘No handlers could be found for logger X.Y.Z’ será exibida uma vez.

Para obter o comportamento pré-versão-3.2, `lastResort` pode ser definido como `None`.

Configurando logging para uma biblioteca

Ao desenvolver um biblioteca que usa logging, é importante documentar como a biblioteca usa o recurso - por exemplo, os nomes dos loggers usados. Também é necessário levar em consideração a configuração de logging. Caso a aplicação que utiliza a biblioteca não faça uso de logging e o código da biblioteca execute chamadas de logging, então os eventos de severidade `WARNING` e níveis mais severos serão impressos em `sys.stderr` .(conforme descrito na seção anterior). Isso garante que mensagens importantes não sejam perdidas, como comportamento padrão, mesmo que a aplicação principal não configure explicitamente o sistema de logging.

Se, por algum motivo, você *não* quiser que essas mensagens sejam exibidas na ausência de qualquer configuração de logging, pode anexar um manipulador que não faz nada ao logger de nível superior da sua biblioteca. Dessa forma, evita-se que a mensagem seja impressa, já que sempre haverá um manipulador associado aos eventos da biblioteca — apenas não produzirá saída. Caso o usuário da biblioteca configure o logging para uso na aplicação, essa configuração provavelmente incluirá alguns manipuladores adicionais e, se os níveis forem configurados adequadamente, as chamadas de logging feitas no código da biblioteca enviarão a saída para esses manipuladores, funcionando normalmente.

Um manipulador-que-não-faz-nada já está incluso no pacote `NullHandler` (desde Python 3.1). Uma instância desse manipulador poderia ser adicionado ao logger de nível superior do espaço de nomes de logging usado pelo biblioteca (*se* você quiser evitar que os eventos registrados do biblioteca sejam enviados para `sys.stderr`, na ausência de configuração de logging). Se todo registro em log de uma biblioteca *foo* for feito usando loggers com nomes correspondente ‘foo.x’, ‘foo.x.y’ etc., então o código:

```
import logging
logging.getLogger('foo').addHandler(logging.NullHandler())
```

deve resultar no efeito desejado. Se uma organização produzir vários bibliotecas, o nome do logger especificado poderá ser ‘orgname.foo’ em vez de apenas ‘foo’.

Nota

É altamente recomendável que você *não registre logs no logger raiz* do seu biblioteca. Em vez disso, use um logger com um nome exclusivo e facilmente identificável, como o `__name__` do nível superior do seu pacote/módulo da biblioteca. O registro em log no logger raiz dificultará, ou mesmo tornará impossível que o desenvolvedor da aplicação personalize o manipulador e a verbosidade de logging.

Nota

É altamente recomendável que você *não adicione outros manipuladores além de* `NullHandler` *nos loggers da sua biblioteca*. Isso porque a configuração de manipuladores é responsabilidade do desenvolvedor da aplicação que utilizará ela. O desenvolvedor sabe o público-alvo da aplicação final, e, portanto, sabe também quais são os manipuladores mais adequados para o caso. Se você

adicionar manipuladores ‘ocultos’, poderá interferir na criação de testes unitários e na entrega de logs úteis.

Níveis de Logging

Os valores numéricos dos níveis de logging estão listados na tabela abaixo. Eles são principalmente de interesse se você quiser definir seus próprios níveis, e precisa deles para definir seus valores específicos relativos aos níveis predefinidos. Se você define um nível com o mesmo valor numérico, ele sobrescreve o valor predefinido; o nome predefinido é perdido.

Nível	Valor numérico
`CRITICAL`	50
`ERROR`	40
`WARNING`	30
`INFO`	20
`DEBUG`	10
`NOTSET`	0

Os níveis também podem ser associados aos registradores, sendo definidos pelo desenvolvedor ou por meio do carregamento de uma configuração salva. Quando um método de logging é invocado em um logger, o logger compara o seu próprio nível com o nível associado à chamada do método. Se o nível do logger for maior do que o da chamada do método, nenhuma mensagem será gerada. Esse é o mecanismo básico que controla a verbosidade da saída de logging.

As mensagens de logging são representadas como instâncias da classe `LogRecord`. Quando um logger decide registrar um evento, uma instância de **`LogRecord`** é criada a partir da mensagem de logging.

As mensagens de logging passam por um mecanismo de despacho que utiliza *manipuladores*, que são instâncias de subclasses da classe `Handler`. Manipuladores são responsáveis por assegurar que uma mensagem registrada em log (na forma de `LogRecord`) seja encaminhada para um determinado destino (ou conjunto de destinos) que serão úteis para o público-alvo daquela mensagem (como usuários finais, equipe de suporte, administradores do sistema e desenvolvedores). Manipuladores transmitem **`LogRecord`** para destinos específicos. Cada logger pode ter zero, um ou mais manipuladores associados à ele (através do método `addHandler()` de `Logger`). Além disso, não são apenas os manipuladores diretamente associados a um logger que processam a mensagem. Por padrão, os manipuladores de

todos os loggers ancestrais também participam do despacho. Essa propagação só é interrompida se o atributo *propagate* do logger estiver definido como `False`.

Assim como os loggers, os manipuladores também podem ter níveis associados. O nível de um manipulador funciona como um filtro, do mesmo modo que o nível de um logger. Quando um manipulador decide processar um evento, ele utiliza o método `emit()` para enviar a mensagem ao seu destino. Na maioria dos casos, subclasses personalizadas de `Handler` precisarão substituir esse método `emit()`.

Níveis personalizados

Criar seus próprios níveis é possível, mas não deve ser necessário, pois os níveis existentes foram escolhidos com base na experiência prática. No entanto, se for estritamente necessário, tome cuidado para fazer. E se você está *desenvolvendo uma biblioteca, não é recomendado que defina níveis personalizados*. Isso porque, se vários autores de biblioteca definirem seus próprios níveis personalizados, as saídas de logging das bibliotecas em conjunto serão orquestradas com maior dificuldade pelo desenvolvedor, pois um determinado valor numérico poderá significar coisas diferentes para diferentes bibliotecas.

Handlers úteis

Em adição à classe base `Handler`, muitas subclasses úteis são fornecidas:

1. `StreamHandler`, quando instanciadas, enviam mensagens para streams (objeto arquivo ou similar).
2. `FileHandler`, quando instanciadas, enviam mensagens para arquivos locais de disco.
3. `BaseRotatingHandler` é a classe base para objetos handler que rotacionam os arquivos de logging em um determinado ponto. Não é feita para ser instanciada diretamente. Em vez disso, use `RotatingFileHandler` ou `TimedRotatingFileHandler`.
4. `RotatingFileHandler`, quando instanciadas, enviam mensagens para arquivos locais de disco. Permite definir o tamanho máximo do arquivo de log e configurar rotação.
5. `TimedRotatingFileHandler`, quando instanciadas, enviam mensagens para arquivos locais de disco, rotacionando o arquivo de log em certos intervalos de tempo.
6. `SocketHandler`, quando instanciadas, enviam mensagens para soquetes TCP/IP. Desde a versão 3.4, os soquetes de domínio Unix também são suportados.
7. `DatagramHandler`, quando instanciadas, enviam mensagens para soquetes UDP. Desde a versão 3.4, os soquetes de domínio Unix também são suportados.
8. `SMTPHandler`, quando instanciadas, enviam mensagens para endereços de e-mail pré-definidos.
9. `SysLogHandler`, quando instanciadas, enviam mensagens para um daemon syslog Unix, normalmente em um computador remoto.
10. `NTEventLogHandler`, quando instanciadas, enviam mensagens para um registro de log do Windows NT/2000/XP.
11. `MemoryHandler`, quando instanciadas, enviam mensagens para um buffer na memória, que é liberado sempre que alguns critérios específicos são atendidos.
12. `HTTPHandler`, quando instanciadas, enviam mensagens para um servidor HTTP, utilizando os métodos `GET` ou `POST`.
13. `WatchedFileHandler`, quando instanciadas, observam o arquivo que estão fazendo logging. Se o arquivo for alterado, ele será fechado e reaberto com o mesmo nome. Este manipulador só é funcional em sistemas Unix ou similares; o Windows não oferece suporte ao mecanismo subjacente utilizado.

14. `QueueHandler`, quando instanciadas, enviam mensagem para uma fila, que pode ser implementada com o módulo `queue` ou com o módulo `multiprocessing`.
15. `NullHandler`, quando instanciadas, não fazem nada com as mensagens de erro. Elas são usadas por desenvolvedores da biblioteca que precisam usar o logging, mas querem evitar o aviso 'No handlers could be found for logger *XXX*', que pode ser exibido quando o logging não é configurado. Veja Configurando logging para uma biblioteca para mais informações.

Adicionado na versão 3.1: A classe `NullHandler`.

Adicionado na versão 3.2: A classe `QueueHandler`.

As subclasses `NullHandler`, `StreamHandler` e `FileHandler` são definidas no pacote base de logging. Os outros manipuladores são definidos no submódulo `logging.handlers`. (Existe também outro submódulo, `logging.config`, para configurar as funcionalidades).

As mensagens de log são formatadas por meio de instâncias da classe `Formatter`. Elas são inicializadas com uma string de formato, que pode ser usada junto com o operador % e um dicionário para substituir os valores na mensagem.

Para formatar várias mensagens em um lote, podem ser usadas instâncias de `BufferingFormatter`. Além de aplicar string de formato em cada mensagem do lote, é possível definir strings de formato para o cabeçalho e o rodapé do lote.

Quando não for suficiente fazer a filtragem baseada no nível do logger e/ou no nível de manipulador, instâncias de `Filter` poderão ser adicionadas às instâncias de `Logger` e `Handler` (por meio do método `addFilter()`). Antes de processar uma mensagem, os loggers e manipuladores consultam todos os seus filtros para obter permissão. Se algum filtro retornar um valor falso, a mensagem não é processada.

A funcionalidade básica do `Filter` permite a filtragem por um nome específico de logger. Se este recurso for usado, as mensagens enviadas para o logger com esse nome, ou seus descendentes, serão permitidas pelo filtro. E todas as outras são descartadas.

Exceções levantadas durante logging

O pacote de logging foi projetado para inibir exceções que possam ocorrer quando os registros forem feitos em produção. Desta forma, a aplicação que está utilizando logging não encerrará abruptamente caso hajam erros nos eventos de logging - devido a problemas como má configuração, rede ou outros.

As exceções `SystemExit` e `KeyboardInterrupt` nunca são inibidas. Outras exceções que possam ocorrem durante o método `emit()` de uma subclasse de `Handler` são passadas para seus respectivos métodos `handleError()`.

A implementação padrão de `handleError()` em `Handler` verifica se uma variável de módulo, `raiseExceptions`, está definida. Caso esteja, um traceback (situação da pilha de execução) é exibido em `sys.stderr`. Se não, a exceção é inibida.

Nota

O valor padrão de `raiseExceptions` é `verdadeiro`. Isso ocorre porque, durante o desenvolvimento, você normalmente desejará ser notificado sobre qualquer exceção que ocorra. É recomendável que

você defina **raiseExceptions** como falso para usos em produção.

Usando objetos arbitrários como mensagens

Nas seções e exemplos anteriores, todas as mensagens criadas ao realizar o logging de um evento foram instanciadas como string. No entanto, essa não é a única possibilidade. Você pode passar um objeto arbitrário como mensagem, e seu método **__str__()** será chamado quando o sistema de logging precisar convertê-lo em uma string. Inclusive, é possível evitar totalmente o uso de uma representação em string - por exemplo, a classe **SocketHandler** emite a mensagem enviando pela rede sua respectiva serialização com pickle.

Optimização

A formatação dos argumentos das mensagem é adiada até o quanto for possível. No entanto, o processamento dos argumentos para o método de logging pode ser custoso, e talvez você queira evitar fazê-lo, se o logging for descartar o evento. Para decidir o que fazer, você pode chamar o método **isEnabledFor()**, que recebe um nível como argumento e retorna verdadeiro apenas se o evento tiver sido criado pelo logger para esse nível em específico. Desta forma, você pode escrever algo como:

```
if logger.isEnabledFor(logging.DEBUG):
    logger.debug('Mensagem com %s, %s', func_custosa1(),
                                        func_custosa2())
```

desta forma, se o limite do logger for definido acima de DEBUG, as chamadas para func_custosa1 e func_custosa2 nunca serão feitas.

Nota

Em alguns casos, o próprio **isEnabledFor()** pode ser mais custoso do que você gostaria (por exemplo, para loggers profundamente aninhados, em que um nível explícito só é definido no logger mais alto da hierarquia). Nesses casos (ou se você quiser evitar chamar um método em um laço), é possível cachear o resultado de uma chamada para **isEnabledFor()** em uma variável local ou instância, e usá-lo em vez de chamar o método todas as vezes. Este valor armazenado só precisaria ser recalculado se a configuração do logger fosse alterada dinamicamente, com a aplicação em execução (o que não é comum).

Existem outras otimizações que podem ser feitas para aplicação específicas que precisam de um controle mais preciso sobre quais informações de logging são registradas. Abaixo encontra-se uma lista de coisas que você pode fazer para evitar o processamento desnecessário para loggings que serão descartados:

O que você não quer registrar	Como evitar o registro

O que você não quer registrar	Como evitar o registro
Informações sobre o ponto em que as chamadas foram feitas.	Defina `logging._srcfile` como `None`. Isso evita chamar `sys._getframe()`, o que pode acelerar seu código em ambientes como o PyPy (que não consegue acelerar códigos que usam **`sys._getframe()`**).
Informações sobre threading.	Defina `logging.logThreads` como `False`.
ID do processo atual (`os.getpid()`)	Defina `logging.logProcesses` como `False`.
Nome do processo atual, ao utilizar o módulo `multiprocessing` para gerenciar múltiplos processamentos.	Defina `logging.logMultiprocessing` como `False`.
Nome da tarefa `asyncio.Task` atual, ao utilizar o módulo `asyncio`.	Defina `logging.logAsyncioTasks` como `False`.

Observe também que o módulo base de logging inclui apenas o manipulador básico. Se você não importar `logging.handlers` e `logging.config`, eles não ocuparão nenhum espaço da memória.

Outros recursos

Ver também

Módulo `logging`
Referência da API para o módulo de logging.

Módulo `logging.config`
API de configuração para o módulo logging.

Módulo `logging.handlers`
Manipuladores úteis incluídos no módulo logging.

Um livro de receitas do logging

Livro de receitas do logging

Esta página contém uma série de receitas relacionadas ao registro de eventos, que foram considerados úteis no passado. Para obter links para tutoriais e outras referências, veja Outros recursos

Usando logging em vários módulos

Várias chamadas para `logging.getLogger('someLogger')` retorna a referência para o mesmo objeto logger. Isso é verdadeiro não apenas dentro do mesmo módulo, mas também entre módulo, desde que esteja no mesmo processo do interpretador Python. Isso é verdadeiro para referências do mesmo objeto; além disso, o código da aplicação pode definir e configurar um logger pai em um módulo e criar (mas não configurar) um logger filho em um módulo separado, e todas as chamadas de logger para o filho passarão para o pai. Aqui está um módulo principal:

```
import logging
import auxiliary_module

# cria objeto logger com 'spam_application'
logger = logging.getLogger('spam_application')
logger.setLevel(logging.DEBUG)
# cria manipulador de arquivo que registra até mesmo mensagens c
fh = logging.FileHandler('spam.log')
fh.setLevel(logging.DEBUG)
# cria manipulador de console com um nível de registro mais altc
ch = logging.StreamHandler()
ch.setLevel(logging.ERROR)
# cria formatador  e adiciona aos manipuladores
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelr
fh.setFormatter(formatter)
ch.setFormatter(formatter)
# adiciona os manipuladores ao logger
logger.addHandler(fh)
logger.addHandler(ch)
```

```
logger.info('creating an instance of auxiliary_module.Auxiliary'
a = auxiliary_module.Auxiliary()
logger.info('created an instance of auxiliary_module.Auxiliary')
logger.info('calling auxiliary_module.Auxiliary.do_something')
a.do_something()
logger.info('finished auxiliary_module.Auxiliary.do_something')
logger.info('calling auxiliary_module.some_function()')
auxiliary_module.some_function()
logger.info('done with auxiliary_module.some_function()')
```

Aqui está o módulo auxiliar:

```
import logging

# cria logger
module_logger = logging.getLogger('spam_application.auxiliary')

class Auxiliary:
    def __init__(self):
        self.logger = logging.getLogger('spam_application.auxili
        self.logger.info('creating an instance of Auxiliary')

    def do_something(self):
        self.logger.info('doing something')
        a = 1 + 1
        self.logger.info('done doing something')

def some_function():
    module_logger.info('received a call to "some_function"')
```

O resultado deve ser algo assim:

```
2005-03-23 23:47:11,663 - spam_application - INFO -
   creating an instance of auxiliary_module.Auxiliary
2005-03-23 23:47:11,665 - spam_application.auxiliary.Auxiliary -
   creating an instance of Auxiliary
2005-03-23 23:47:11,665 - spam_application - INFO -
   created an instance of auxiliary_module.Auxiliary
2005-03-23 23:47:11,668 - spam_application - INFO -
   calling auxiliary_module.Auxiliary.do_something
2005-03-23 23:47:11,668 - spam_application.auxiliary.Auxiliary -
```

```
   doing something
2005-03-23 23:47:11,669 - spam_application.auxiliary.Auxiliary -
   done doing something
2005-03-23 23:47:11,670 - spam_application - INFO -
   finished auxiliary_module.Auxiliary.do_something
2005-03-23 23:47:11,671 - spam_application - INFO -
   calling auxiliary_module.some_function()
2005-03-23 23:47:11,672 - spam_application.auxiliary - INFO -
   received a call to 'some_function'
2005-03-23 23:47:11,673 - spam_application - INFO -
   done with auxiliary_module.some_function()
```

Registro de eventos de várias threads

O registro de eventos de várias threads não requer nenhum esforço especial. O exemplo a seguir mostra o registro de eventos da thread principal (inicial) thread e de outra thread:

```
import logging
import threading
import time

def worker(arg):
    while not arg['stop']:
        logging.debug('Hi from myfunc')
        time.sleep(0.5)

def main():
    logging.basicConfig(level=logging.DEBUG, format='%(relativeC
    info = {'stop': False}
    thread = threading.Thread(target=worker, args=(info,))
    thread.start()
    while True:
        try:
            logging.debug('Hello from main')
            time.sleep(0.75)
        except KeyboardInterrupt:
            info['stop'] = True
            break
    thread.join()
```

```
if __name__ == '__main__':
    main()
```

Quando executado, o script deve exibir algo como o seguinte:

```
   0 Thread-1 Hi from myfunc
   3 MainThread Hello from main
 505 Thread-1 Hi from myfunc
 755 MainThread Hello from main
1007 Thread-1 Hi from myfunc
1507 MainThread Hello from main
1508 Thread-1 Hi from myfunc
2010 Thread-1 Hi from myfunc
2258 MainThread Hello from main
2512 Thread-1 Hi from myfunc
3009 MainThread Hello from main
3013 Thread-1 Hi from myfunc
3515 Thread-1 Hi from myfunc
3761 MainThread Hello from main
4017 Thread-1 Hi from myfunc
4513 MainThread Hello from main
4518 Thread-1 Hi from myfunc
```

Isso mostra a saída de registro intercalada, como seria de se esperar. Essa abordagem funciona para mais thread do que o mostrado aqui, é claro.

Múltiplos manipuladores e formatadores

Registradores (loggers) são objetos Python simples. O método `addHandler()` não tem cota mínima ou máxima para o número de manipuladores que podem ser adicionados. Às vezes, é vantajoso para um aplicação registrar todas as mensagens de todas as gravidades em um arquivo texto e, ao mesmo tempo, registrar erro ou superior no console. Para definir isso, basta configurar os manipuladores apropriados. As chamadas de registro no código da aplicação permanecerão inalteradas. Aqui está uma pequena modificação no exemplo anterior de configuração simples baseada em módulo:

```
import logging

logger = logging.getLogger('simple_example')
logger.setLevel(logging.DEBUG)
# cria manipulador de arquivo que regitra até mensagens de depur
fh = logging.FileHandler('spam.log')
fh.setLevel(logging.DEBUG)
# cria manipulador de console com um nível de gravidade mais alt
ch = logging.StreamHandler()
ch.setLevel(logging.ERROR)
# cria formatador e adiciona-o ao manipulador
formatter = logging.Formatter('%(asctime)s - %(name)s - %(levelr
ch.setFormatter(formatter)
fh.setFormatter(formatter)
# adiciona os manipuladores ao registrador
logger.addHandler(ch)
logger.addHandler(fh)

# código da 'aplicação'
logger.debug('debug message')
logger.info('info message')
logger.warning('warn message')
logger.error('error message')
logger.critical('critical message')
```

Observe que o código da ‘aplicação’ não se preocupa com vários manipuladores. Tudo o que mudou foi a adição e a configuração de um novo manipulador nomeado *fh*.

A capacidade de criar novos manipuladores com filtros de severidade superiores ou inferiores pode ser muito útil ao escrever e testar um aplicação. Em vez de usar muitas instruções instrução para depuração, use `logger.debug`: diferentemente da instrução print, que você terá de excluir ou comentar posteriormente, a instrução logger.debug pode permanecer intacta no código-fonte e ficar inativo até que você precise dela novamente. Nesse momento, a única alteração que precisa ser feita é modificar o nível de severidade do registrador e/ou manipulador para depuração.

Logging to multiple destinations

Let’s say you want to log to console and file with different message formats and in differing circumstances. Say you want to log messages with levels of DEBUG and higher to file, and those messages at level INFO and higher to the console. Let’s also assume that the file should contain timestamps, but the console messages should not. Here’s how you can achieve this:

```
import logging

# set up logging to file - see previous section for more details
logging.basicConfig(level=logging.DEBUG,
                    format='%(asctime)s %(name)-12s %(levelname)
                    datefmt='%m-%d %H:%M',
                    filename='/tmp/myapp.log',
                    filemode='w')
# define a Handler which writes INFO messages or higher to the s
console = logging.StreamHandler()
console.setLevel(logging.INFO)
# set a format which is simpler for console use
formatter = logging.Formatter('%(name)-12s: %(levelname)-8s %(me
# tell the handler to use this format
console.setFormatter(formatter)
# add the handler to the root logger
logging.getLogger().addHandler(console)

# Now, we can log to the root logger, or any other logger. First
logging.info('Jackdaws love my big sphinx of quartz.')

# Now, define a couple of other loggers which might represent ar
# application:

logger1 = logging.getLogger('myapp.area1')
logger2 = logging.getLogger('myapp.area2')

logger1.debug('Quick zephyrs blow, vexing daft Jim.')
logger1.info('How quickly daft jumping zebras vex.')
logger2.warning('Jail zesty vixen who grabbed pay from quack.')
logger2.error('The five boxing wizards jump quickly.')
```

When you run this, on the console you will see

```
root        : INFO     Jackdaws love my big sphinx of quartz.
myapp.area1 : INFO     How quickly daft jumping zebras vex.
myapp.area2 : WARNING  Jail zesty vixen who grabbed pay from qua
myapp.area2 : ERROR    The five boxing wizards jump quickly.
```

and in the file you will see something like

```
10-22 22:19 root         INFO     Jackdaws love my big sphinx of
10-22 22:19 myapp.area1  DEBUG    Quick zephyrs blow, vexing daf
10-22 22:19 myapp.area1  INFO     How quickly daft jumping zebra
10-22 22:19 myapp.area2  WARNING  Jail zesty vixen who grabbed p
10-22 22:19 myapp.area2  ERROR    The five boxing wizards jump c
```

As you can see, the DEBUG message only shows up in the file. The other messages are sent to both destinations.

This example uses console and file handlers, but you can use any number and combination of handlers you choose.

Note that the above choice of log filename `/tmp/myapp.log` implies use of a standard location for temporary files on POSIX systems. On Windows, you may need to choose a different directory name for the log - just ensure that the directory exists and that you have the permissions to create and update files in it.

Custom handling of levels

Sometimes, you might want to do something slightly different from the standard handling of levels in handlers, where all levels above a threshold get processed by a handler. To do this, you need to use filters. Let's look at a scenario where you want to arrange things as follows:

- Send messages of severity `INFO` and `WARNING` to `sys.stdout`
- Send messages of severity `ERROR` and above to `sys.stderr`
- Send messages of severity `DEBUG` and above to file `app.log`

Suppose you configure logging with the following JSON:

```
{
    "version": 1,
    "disable_existing_loggers": false,
    "formatters": {
        "simple": {
            "format": "%(levelname)-8s - %(message)s"
        }
    },
    "handlers": {
        "stdout": {
            "class": "logging.StreamHandler",
            "level": "INFO",
            "formatter": "simple",
            "stream": "ext://sys.stdout"
        },
        "stderr": {
            "class": "logging.StreamHandler",
            "level": "ERROR",
            "formatter": "simple",
            "stream": "ext://sys.stderr"
        },
        "file": {
            "class": "logging.FileHandler",
            "formatter": "simple",
            "filename": "app.log",
            "mode": "w"
        }
    },
    "root": {
        "level": "DEBUG",
        "handlers": [
            "stderr",
            "stdout",
            "file"
        ]
    }
}
```

This configuration does *almost* what we want, except that `sys.stdout` would show messages of severity `ERROR` and only events of this severity and higher will be tracked as well as `INFO` and `WARNING` messages. To prevent this, we can set up a filter which excludes those messages and add it to the

relevant handler. This can be configured by adding a `filters` section parallel to `formatters` and `handlers`:

```
{
    "filters": {
        "warnings_and_below": {
            "()" : "__main__.filter_maker",
            "level": "WARNING"
        }
    }
}
```

and changing the section on the `stdout` handler to add it:

```
{
    "stdout": {
        "class": "logging.StreamHandler",
        "level": "INFO",
        "formatter": "simple",
        "stream": "ext://sys.stdout",
        "filters": ["warnings_and_below"]
    }
}
```

A filter is just a function, so we can define the `filter_maker` (a factory function) as follows:

```
def filter_maker(level):
    level = getattr(logging, level)

    def filter(record):
        return record.levelno <= level

    return filter
```

This converts the string argument passed in to a numeric level, and returns a function which only returns `True` if the level of the passed in record is at or below the specified level. Note that in this example I have defined the `filter_maker` in a test script `main.py` that I run from the command line, so its module will be `__main__` - hence the `__main__.filter_maker` in the

filter configuration. You will need to change that if you define it in a different module.

With the filter added, we can run `main.py`, which in full is:

```
import json
import logging
import logging.config

CONFIG = '''
{
    "version": 1,
    "disable_existing_loggers": false,
    "formatters": {
        "simple": {
            "format": "%(levelname)-8s - %(message)s"
        }
    },
    "filters": {
        "warnings_and_below": {
            "()" : "__main__.filter_maker",
            "level": "WARNING"
        }
    },
    "handlers": {
        "stdout": {
            "class": "logging.StreamHandler",
            "level": "INFO",
            "formatter": "simple",
            "stream": "ext://sys.stdout",
            "filters": ["warnings_and_below"]
        },
        "stderr": {
            "class": "logging.StreamHandler",
            "level": "ERROR",
            "formatter": "simple",
            "stream": "ext://sys.stderr"
        },
        "file": {
            "class": "logging.FileHandler",
            "formatter": "simple",
            "filename": "app.log",
```

```
            "mode": "w"
        }
    },
    "root": {
        "level": "DEBUG",
        "handlers": [
            "stderr",
            "stdout",
            "file"
        ]
    }
}
'''

def filter_maker(level):
    level = getattr(logging, level)

    def filter(record):
        return record.levelno <= level

    return filter

logging.config.dictConfig(json.loads(CONFIG))
logging.debug('A DEBUG message')
logging.info('An INFO message')
logging.warning('A WARNING message')
logging.error('An ERROR message')
logging.critical('A CRITICAL message')
```

And after running it like this:

```
python main.py 2>stderr.log >stdout.log
```

We can see the results are as expected:

```
$ more *.log
::::::::::::::
app.log
::::::::::::::
DEBUG    - A DEBUG message
INFO     - An INFO message
WARNING  - A WARNING message
```

```
ERROR    - An ERROR message
CRITICAL - A CRITICAL message
::::::::::::::
stderr.log
::::::::::::::
ERROR    - An ERROR message
CRITICAL - A CRITICAL message
::::::::::::::
stdout.log
::::::::::::::
INFO     - An INFO message
WARNING  - A WARNING message
```

Configuration server example

Here is an example of a module using the logging configuration server:

```
import logging
import logging.config
import time
import os

# read initial config file
logging.config.fileConfig('logging.conf')

# create and start listener on port 9999
t = logging.config.listen(9999)
t.start()

logger = logging.getLogger('simpleExample')

try:
    # loop through logging calls to see the difference
    # new configurations make, until Ctrl+C is pressed
    while True:
        logger.debug('debug message')
        logger.info('info message')
        logger.warning('warn message')
        logger.error('error message')
        logger.critical('critical message')
        time.sleep(5)
except KeyboardInterrupt:
```

```
    # cleanup
    logging.config.stopListening()
    t.join()
```

And here is a script that takes a filename and sends that file to the server, properly preceded with the binary-encoded length, as the new logging configuration:

```
#!/usr/bin/env python
import socket, sys, struct

with open(sys.argv[1], 'rb') as f:
    data_to_send = f.read()

HOST = 'localhost'
PORT = 9999
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print('connecting...')
s.connect((HOST, PORT))
print('sending config...')
s.send(struct.pack('>L', len(data_to_send)))
s.send(data_to_send)
s.close()
print('complete')
```

Dealing with handlers that block

Sometimes you have to get your logging handlers to do their work without blocking the thread you're logging from. This is common in web applications, though of course it also occurs in other scenarios.

A common culprit which demonstrates sluggish behaviour is the `SMTPHandler`: sending emails can take a long time, for a number of reasons outside the developer's control (for example, a poorly performing mail or network infrastructure). But almost any network-based handler can block: Even a `SocketHandler` operation may do a DNS query under the hood which is too slow (and this query can be deep in the socket library code, below the Python layer, and outside your control).

One solution is to use a two-part approach. For the first part, attach only a **QueueHandler** to those loggers which are accessed from performance-critical threads. They simply write to their queue, which can be sized to a large enough capacity or initialized with no upper bound to their size. The write to the queue will typically be accepted quickly, though you will probably need to catch the **queue.Full** exception as a precaution in your code. If you are a library developer who has performance-critical threads in their code, be sure to document this (together with a suggestion to attach only `QueueHandlers` to your loggers) for the benefit of other developers who will use your code.

The second part of the solution is **QueueListener**, which has been designed as the counterpart to **QueueHandler**. A **QueueListener** is very simple: it's passed a queue and some handlers, and it fires up an internal thread which listens to its queue for LogRecords sent from `QueueHandlers` (or any other source of `LogRecords`, for that matter). The `LogRecords` are removed from the queue and passed to the handlers for processing.

The advantage of having a separate **QueueListener** class is that you can use the same instance to service multiple `QueueHandlers`. This is more resource-friendly than, say, having threaded versions of the existing handler classes, which would eat up one thread per handler for no particular benefit.

An example of using these two classes follows (imports omitted):

```
que = queue.Queue(-1)  # no limit on size
queue_handler = QueueHandler(que)
handler = logging.StreamHandler()
listener = QueueListener(que, handler)
root = logging.getLogger()
root.addHandler(queue_handler)
formatter = logging.Formatter('%(threadName)s: %(message)s')
handler.setFormatter(formatter)
listener.start()
# The log output will display the thread which generated
# the event (the main thread) rather than the internal
# thread which monitors the internal queue. This is what
# you want to happen.
root.warning('Look out!')
listener.stop()
```

which, when run, will produce:

```
MainThread: Look out!
```

Nota

Although the earlier discussion wasn't specifically talking about async code, but rather about slow logging handlers, it should be noted that when logging from async code, network and even file handlers could lead to problems (blocking the event loop) because some logging is done from `asyncio` internals. It might be best, if any async code is used in an application, to use the above approach for logging, so that any blocking code runs only in the `QueueListener` thread.

Alterado na versão 3.5: Prior to Python 3.5, the `QueueListener` always passed every message received from the queue to every handler it was initialized with. (This was because it was assumed that level filtering was all done on the other side, where the queue is filled.) From 3.5 onwards, this behaviour can be changed by passing a keyword argument `respect_handler_level=True` to the listener's constructor. When this is done, the listener compares the level of each message with the handler's level, and only passes a message to a handler if it's appropriate to do so.

Alterado na versão 3.14: The `QueueListener` can be started (and stopped) via the `with` statement. For example:

```
with QueueListener(que, handler) as listener:
    # The queue listener automatically starts
    # when the 'with' block is entered.
    pass
# The queue listener automatically stops once
# the 'with' block is exited.
```

Sending and receiving logging events across a network

Let's say you want to send logging events across a network, and handle them at the receiving end. A simple way of doing this is attaching a **SocketHandler** instance to the root logger at the sending end:

```
import logging, logging.handlers

rootLogger = logging.getLogger()
rootLogger.setLevel(logging.DEBUG)
socketHandler = logging.handlers.SocketHandler('localhost',
                    logging.handlers.DEFAULT_TCP_LOGGING_PORT)
# don't bother with a formatter, since a socket handler sends th
# an unformatted pickle
rootLogger.addHandler(socketHandler)

# Now, we can log to the root logger, or any other logger. First
logging.info('Jackdaws love my big sphinx of quartz.')

# Now, define a couple of other loggers which might represent ar
# application:

logger1 = logging.getLogger('myapp.area1')
logger2 = logging.getLogger('myapp.area2')

logger1.debug('Quick zephyrs blow, vexing daft Jim.')
logger1.info('How quickly daft jumping zebras vex.')
logger2.warning('Jail zesty vixen who grabbed pay from quack.')
logger2.error('The five boxing wizards jump quickly.')
```

At the receiving end, you can set up a receiver using the **socketserver** module. Here is a basic working example:

```
import pickle
import logging
import logging.handlers
import socketserver
import struct


class LogRecordStreamHandler(socketserver.StreamRequestHandler):
    """Handler for a streaming logging request.
```

```
    This basically logs the record using whatever logging policy
    configured locally.
    """

    def handle(self):
        """
        Handle multiple requests - each expected to be a 4-byte
        followed by the LogRecord in pickle format. Logs the rec
        according to whatever policy is configured locally.
        """
        while True:
            chunk = self.connection.recv(4)
            if len(chunk) < 4:
                break
            slen = struct.unpack('>L', chunk)[0]
            chunk = self.connection.recv(slen)
            while len(chunk) < slen:
                chunk = chunk + self.connection.recv(slen - len(
            obj = self.unPickle(chunk)
            record = logging.makeLogRecord(obj)
            self.handleLogRecord(record)

    def unPickle(self, data):
        return pickle.loads(data)

    def handleLogRecord(self, record):
        # if a name is specified, we use the named logger rather
        # implied by the record.
        if self.server.logname is not None:
            name = self.server.logname
        else:
            name = record.name
        logger = logging.getLogger(name)
        # N.B. EVERY record gets logged. This is because Logger.
        # is normally called AFTER logger-level filtering. If yo
        # to do filtering, do it at the client end to save wasti
        # cycles and network bandwidth!
        logger.handle(record)

class LogRecordSocketReceiver(socketserver.ThreadingTCPServer):
    """
    Simple TCP socket-based logging receiver suitable for testir
    """
```

```
    allow_reuse_address = True

    def __init__(self, host='localhost',
                 port=logging.handlers.DEFAULT_TCP_LOGGING_PORT,
                 handler=LogRecordStreamHandler):
        socketserver.ThreadingTCPServer.__init__(self, (host, pc
        self.abort = 0
        self.timeout = 1
        self.logname = None

    def serve_until_stopped(self):
        import select
        abort = 0
        while not abort:
            rd, wr, ex = select.select([self.socket.fileno()],
                                       [], [],
                                       self.timeout)
            if rd:
                self.handle_request()
            abort = self.abort

def main():
    logging.basicConfig(
        format='%(relativeCreated)5d %(name)-15s %(levelname)-8s
    tcpserver = LogRecordSocketReceiver()
    print('About to start TCP server...')
    tcpserver.serve_until_stopped()

if __name__ == '__main__':
    main()
```

First run the server, and then the client. On the client side, nothing is printed on the console; on the server side, you should see something like:

```
About to start TCP server...
   59 root            INFO     Jackdaws love my big sphinx of qu
   59 myapp.area1     DEBUG    Quick zephyrs blow, vexing daft J
   69 myapp.area1     INFO     How quickly daft jumping zebras v
   69 myapp.area2     WARNING  Jail zesty vixen who grabbed pay
   69 myapp.area2     ERROR    The five boxing wizards jump quic
```

Note that there are some security issues with pickle in some scenarios. If these affect you, you can use an alternative serialization scheme by overriding the `makePickle()` method and implementing your alternative there, as well as adapting the above script to use your alternative serialization.

Running a logging socket listener in production

To run a logging listener in production, you may need to use a process-management tool such as Supervisor [http://supervisord.org/]. Here is a Gist [https://gist.github.com/vsajip/4b227eeec43817465ca835ca66f75e2b] which provides the bare-bones files to run the above functionality using Supervisor. It consists of the following files:

Arquivo	Propósito
`prepare.sh`	A Bash script to prepare the environment for testing
`supervisor.conf`	The Supervisor configuration file, which has entries for the listener and a multi-process web application
`ensure_app.sh`	A Bash script to ensure that Supervisor is running with the above configuration
`log_listener.py`	The socket listener program which receives log events and records them to a file
`main.py`	A simple web application which performs logging via a socket connected to the listener

Arquivo	Propósito
`webapp.json`	A JSON configuration file for the web application
`client.py`	A Python script to exercise the web application

The web application uses Gunicorn [https://gunicorn.org/], which is a popular web application server that starts multiple worker processes to handle requests. This example setup shows how the workers can write to the same log file without conflicting with one another — they all go through the socket listener.

To test these files, do the following in a POSIX environment:

1. Download the Gist [https://gist.github.com/vsajip/4b227eeec43817465ca835ca66f75e2b] as a ZIP archive using the *Download ZIP* button.
2. Unzip the above files from the archive into a scratch directory.
3. In the scratch directory, run `bash prepare.sh` to get things ready. This creates a `run` subdirectory to contain Supervisor-related and log files, and a `venv` subdirectory to contain a virtual environment into which `bottle`, `gunicorn` and `supervisor` are installed.
4. Run `bash ensure_app.sh` to ensure that Supervisor is running with the above configuration.
5. Run `venv/bin/python client.py` to exercise the web application, which will lead to records being written to the log.
6. Inspect the log files in the `run` subdirectory. You should see the most recent log lines in files matching the pattern `app.log*`. They won't be in any particular order, since they have been handled concurrently by different worker processes in a non-deterministic way.
7. You can shut down the listener and the web application by running `venv/bin/supervisorctl -c supervisor.conf shutdown`.

You may need to tweak the configuration files in the unlikely event that the configured ports clash with something else in your test environment.

The default configuration uses a TCP socket on port 9020. You can use a Unix Domain socket instead of a TCP socket by doing the following:

1. In `listener.json`, add a `socket` key with the path to the domain socket you want to use. If this key is present, the listener listens on the corresponding domain socket and not on a TCP socket (the `port` key is ignored).
2. In `webapp.json`, change the socket handler configuration dictionary so that the `host` value is the path to the domain socket, and set the `port` value to `null`.

Adding contextual information to your logging output

Sometimes you want logging output to contain contextual information in addition to the parameters passed to the logging call. For example, in a networked application, it may be desirable to log client-specific information in the log (e.g. remote client's username, or IP address). Although you could use the *extra* parameter to achieve this, it's not always convenient to pass the information in this way. While it might be tempting to create **`Logger`** instances on a per-connection basis, this is not a good idea because these instances are not garbage collected. While this is not a problem in practice, when the number of **`Logger`** instances is dependent on the level of granularity you want to use in logging an application, it could be hard to manage if the number of **`Logger`** instances becomes effectively unbounded.

Using LoggerAdapters to impart contextual information

An easy way in which you can pass contextual information to be output along with logging event information is to use the **`LoggerAdapter`** class. This class is designed to look like a **`Logger`**, so that you can call **`debug()`**, **`info()`**, **`warning()`**, **`error()`**, **`exception()`**, **`critical()`** and **`log()`**. These

methods have the same signatures as their counterparts in **Logger**, so you can use the two types of instances interchangeably.

When you create an instance of **LoggerAdapter**, you pass it a **Logger** instance and a dict-like object which contains your contextual information. When you call one of the logging methods on an instance of **LoggerAdapter**, it delegates the call to the underlying instance of **Logger** passed to its constructor, and arranges to pass the contextual information in the delegated call. Here's a snippet from the code of **LoggerAdapter**:

```
def debug(self, msg, /, *args, **kwargs):
    """
    Delegate a debug call to the underlying logger, after adding
    contextual information from this adapter instance.
    """
    msg, kwargs = self.process(msg, kwargs)
    self.logger.debug(msg, *args, **kwargs)
```

The **process()** method of **LoggerAdapter** is where the contextual information is added to the logging output. It's passed the message and keyword arguments of the logging call, and it passes back (potentially) modified versions of these to use in the call to the underlying logger. The default implementation of this method leaves the message alone, but inserts an 'extra' key in the keyword argument whose value is the dict-like object passed to the constructor. Of course, if you had passed an 'extra' keyword argument in the call to the adapter, it will be silently overwritten.

The advantage of using 'extra' is that the values in the dict-like object are merged into the **LogRecord** instance's __dict__, allowing you to use customized strings with your **Formatter** instances which know about the keys of the dict-like object. If you need a different method, e.g. if you want to prepend or append the contextual information to the message string, you just need to subclass **LoggerAdapter** and override **process()** to do what you need. Here is a simple example:

```
class CustomAdapter(logging.LoggerAdapter):
    """
    This example adapter expects the passed in dict-like object
    'connid' key, whose value in brackets is prepended to the lo
```

```
    """
    def process(self, msg, kwargs):
        return '[%s] %s' % (self.extra['connid'], msg), kwargs
```

which you can use like this:

```
logger = logging.getLogger(__name__)
adapter = CustomAdapter(logger, {'connid': some_conn_id})
```

Then any events that you log to the adapter will have the value of `some_conn_id` prepended to the log messages.

Using objects other than dicts to pass contextual information

You don't need to pass an actual dict to a **`LoggerAdapter`** - you could pass an instance of a class which implements `__getitem__` and `__iter__` so that it looks like a dict to logging. This would be useful if you want to generate values dynamically (whereas the values in a dict would be constant).

Using Filters to impart contextual information

You can also add contextual information to log output using a user-defined **`Filter`**. `Filter` instances are allowed to modify the `LogRecords` passed to them, including adding additional attributes which can then be output using a suitable format string, or if needed a custom **`Formatter`**.

For example in a web application, the request being processed (or at least, the interesting parts of it) can be stored in a threadlocal (**`threading.local`**) variable, and then accessed from a `Filter` to add, say, information from the request - say, the remote IP address and remote user's username - to the `LogRecord`, using the attribute names 'ip' and 'user' as in the `LoggerAdapter` example above. In that case, the same format string can be used to get similar output to that shown above. Here's an example script:

```
import logging
from random import choice

class ContextFilter(logging.Filter):
```

```
    """
    This is a filter which injects contextual information into t

    Rather than use actual contextual information, we just use r
    data in this demo.
    """

    USERS = ['jim', 'fred', 'sheila']
    IPS = ['123.231.231.123', '127.0.0.1', '192.168.0.1']

    def filter(self, record):

        record.ip = choice(ContextFilter.IPS)
        record.user = choice(ContextFilter.USERS)
        return True

if __name__ == '__main__':
    levels = (logging.DEBUG, logging.INFO, logging.WARNING, logg
    logging.basicConfig(level=logging.DEBUG,
                        format='%(asctime)-15s %(name)-5s %(leve
    a1 = logging.getLogger('a.b.c')
    a2 = logging.getLogger('d.e.f')

    f = ContextFilter()
    a1.addFilter(f)
    a2.addFilter(f)
    a1.debug('A debug message')
    a1.info('An info message with %s', 'some parameters')
    for x in range(10):
        lvl = choice(levels)
        lvlname = logging.getLevelName(lvl)
        a2.log(lvl, 'A message at %s level with %d %s', lvlname,
```

which, when run, produces something like:

```
2010-09-06 22:38:15,292 a.b.c DEBUG    IP: 123.231.231.123 User:
2010-09-06 22:38:15,300 a.b.c INFO     IP: 192.168.0.1     User:
2010-09-06 22:38:15,300 d.e.f CRITICAL IP: 127.0.0.1       User:
2010-09-06 22:38:15,300 d.e.f ERROR    IP: 127.0.0.1       User:
2010-09-06 22:38:15,300 d.e.f DEBUG    IP: 127.0.0.1       User:
2010-09-06 22:38:15,300 d.e.f ERROR    IP: 123.231.231.123 User:
2010-09-06 22:38:15,300 d.e.f CRITICAL IP: 192.168.0.1     User:
```

```
2010-09-06 22:38:15,300 d.e.f CRITICAL IP: 127.0.0.1       User:
2010-09-06 22:38:15,300 d.e.f DEBUG    IP: 192.168.0.1     User:
2010-09-06 22:38:15,301 d.e.f ERROR    IP: 127.0.0.1       User:
2010-09-06 22:38:15,301 d.e.f DEBUG    IP: 123.231.231.123 User:
2010-09-06 22:38:15,301 d.e.f INFO     IP: 123.231.231.123 User:
```

Use of `contextvars`

Since Python 3.7, the `contextvars` module has provided context-local storage which works for both `threading` and `asyncio` processing needs. This type of storage may thus be generally preferable to thread-locals. The following example shows how, in a multi-threaded environment, logs can populated with contextual information such as, for example, request attributes handled by web applications.

For the purposes of illustration, say that you have different web applications, each independent of the other but running in the same Python process and using a library common to them. How can each of these applications have their own log, where all logging messages from the library (and other request processing code) are directed to the appropriate application's log file, while including in the log additional contextual information such as client IP, HTTP request method and client username?

Let's assume that the library can be simulated by the following code:

```
# webapplib.py
import logging
import time

logger = logging.getLogger(__name__)

def useful():
    # Just a representative event logged from the library
    logger.debug('Hello from webapplib!')
    # Just sleep for a bit so other threads get to run
    time.sleep(0.01)
```

We can simulate the multiple web applications by means of two simple classes, `Request` and `WebApp`. These simulate how real threaded web applications work - each request is handled by a thread:

```
# main.py
import argparse
from contextvars import ContextVar
import logging
import os
from random import choice
import threading
import webapplib

logger = logging.getLogger(__name__)
root = logging.getLogger()
root.setLevel(logging.DEBUG)

class Request:
    """
    A simple dummy request class which just holds dummy HTTP rec
    client IP address and client username
    """
    def __init__(self, method, ip, user):
        self.method = method
        self.ip = ip
        self.user = user

# A dummy set of requests which will be used in the simulation -
# from this list randomly. Note that all GET requests are from 1
# addresses, whereas POST requests are from 192.16.3.XXX address
# are represented in the sample requests.

REQUESTS = [
    Request('GET', '192.168.2.20', 'jim'),
    Request('POST', '192.168.3.20', 'fred'),
    Request('GET', '192.168.2.21', 'sheila'),
    Request('POST', '192.168.3.21', 'jim'),
    Request('GET', '192.168.2.22', 'fred'),
    Request('POST', '192.168.3.22', 'sheila'),
]

# Note that the format string includes references to request cor
```

```
# such as HTTP method, client IP and username

formatter = logging.Formatter('%(threadName)-11s %(appName)s %(r

# Create our context variables. These will be filled at the star
# processing, and used in the logging that happens during that p

ctx_request = ContextVar('request')
ctx_appname = ContextVar('appname')

class InjectingFilter(logging.Filter):
    """
    A filter which injects context-specific information into log
    that only information for a specific webapp is included in i
    """
    def __init__(self, app):
        self.app = app

    def filter(self, record):
        request = ctx_request.get()
        record.method = request.method
        record.ip = request.ip
        record.user = request.user
        record.appName = appName = ctx_appname.get()
        return appName == self.app.name

class WebApp:
    """
    A dummy web application class which has its own handler and
    webapp-specific log.
    """
    def __init__(self, name):
        self.name = name
        handler = logging.FileHandler(name + '.log', 'w')
        f = InjectingFilter(self)
        handler.setFormatter(formatter)
        handler.addFilter(f)
        root.addHandler(handler)
        self.num_requests = 0

    def process_request(self, request):
        """
        This is the dummy method for processing a request. It's
```

```
        different thread for every request. We store the context
        the context vars before doing anything else.
        """
        ctx_request.set(request)
        ctx_appname.set(self.name)
        self.num_requests += 1
        logger.debug('Request processing started')
        webapplib.useful()
        logger.debug('Request processing finished')

def main():
    fn = os.path.splitext(os.path.basename(__file__))[0]
    adhf = argparse.ArgumentDefaultsHelpFormatter
    ap = argparse.ArgumentParser(formatter_class=adhf, prog=fn,
                                 description='Simulate a couple
                                             'applications handl
                                             'requests, showing
                                             'context can be use
                                             'populate logs')
    aa = ap.add_argument
    aa('--count', '-c', type=int, default=100, help='How many re
    options = ap.parse_args()

    # Create the dummy webapps and put them in a list which we c
    # from randomly
    app1 = WebApp('app1')
    app2 = WebApp('app2')
    apps = [app1, app2]
    threads = []
    # Add a common handler which will capture all events
    handler = logging.FileHandler('app.log', 'w')
    handler.setFormatter(formatter)
    root.addHandler(handler)

    # Generate calls to process requests
    for i in range(options.count):
        try:
            # Pick an app at random and a request for it to proc
            app = choice(apps)
            request = choice(REQUESTS)
            # Process the request in its own thread
            t = threading.Thread(target=app.process_request, arg
            threads.append(t)
```

```
            t.start()
        except KeyboardInterrupt:
            break

    # Wait for the threads to terminate
    for t in threads:
        t.join()

    for app in apps:
        print('%s processed %s requests' % (app.name, app.num_re

if __name__ == '__main__':
    main()
```

If you run the above, you should find that roughly half the requests go into `app1.log` and the rest into `app2.log`, and the all the requests are logged to `app.log`. Each webapp-specific log will contain only log entries for only that webapp, and the request information will be displayed consistently in the log (i.e. the information in each dummy request will always appear together in a log line). This is illustrated by the following shell output:

```
~/logging-contextual-webapp$ python main.py
app1 processed 51 requests
app2 processed 49 requests
~/logging-contextual-webapp$ wc -l *.log
  153 app1.log
  147 app2.log
  300 app.log
  600 total
~/logging-contextual-webapp$ head -3 app1.log
Thread-3 (process_request) app1 __main__  jim    192.168.3.21 PO
Thread-3 (process_request) app1 webapplib jim    192.168.3.21 PO
Thread-5 (process_request) app1 __main__  jim    192.168.3.21 PO
~/logging-contextual-webapp$ head -3 app2.log
Thread-1 (process_request) app2 __main__  sheila 192.168.2.21 GE
Thread-1 (process_request) app2 webapplib sheila 192.168.2.21 GE
Thread-2 (process_request) app2 __main__  jim    192.168.2.20 GE
~/logging-contextual-webapp$ head app.log
Thread-1 (process_request) app2 __main__  sheila 192.168.2.21 GE
Thread-1 (process_request) app2 webapplib sheila 192.168.2.21 GE
Thread-2 (process_request) app2 __main__  jim    192.168.2.20 GE
```

```
Thread-3 (process_request) app1 __main__  jim    192.168.3.21 PC
Thread-2 (process_request) app2 webapplib jim    192.168.2.20 GE
Thread-3 (process_request) app1 webapplib jim    192.168.3.21 PC
Thread-4 (process_request) app2 __main__  fred   192.168.2.22 GE
Thread-5 (process_request) app1 __main__  jim    192.168.3.21 PC
Thread-4 (process_request) app2 webapplib fred   192.168.2.22 GE
Thread-6 (process_request) app1 __main__  jim    192.168.3.21 PC
~/logging-contextual-webapp$ grep app1 app1.log | wc -l
153
~/logging-contextual-webapp$ grep app2 app2.log | wc -l
147
~/logging-contextual-webapp$ grep app1 app.log | wc -l
153
~/logging-contextual-webapp$ grep app2 app.log | wc -l
147
```

Imparting contextual information in handlers

Each **Handler** has its own chain of filters. If you want to add contextual information to a **LogRecord** without leaking it to other handlers, you can use a filter that returns a new **LogRecord** instead of modifying it in-place, as shown in the following script:

```
import copy
import logging

def filter(record: logging.LogRecord):
    record = copy.copy(record)
    record.user = 'jim'
    return record

if __name__ == '__main__':
    logger = logging.getLogger()
    logger.setLevel(logging.INFO)
    handler = logging.StreamHandler()
    formatter = logging.Formatter('%(message)s from %(user)-8s')
    handler.setFormatter(formatter)
    handler.addFilter(filter)
    logger.addHandler(handler)
```

```
    logger.info('A log message')
```

Logging to a single file from multiple processes

Although logging is thread-safe, and logging to a single file from multiple threads in a single process *is* supported, logging to a single file from *multiple processes* is *not* supported, because there is no standard way to serialize access to a single file across multiple processes in Python. If you need to log to a single file from multiple processes, one way of doing this is to have all the processes log to a **`SocketHandler`**, and have a separate process which implements a socket server which reads from the socket and logs to file. (If you prefer, you can dedicate one thread in one of the existing processes to perform this function.) This section documents this approach in more detail and includes a working socket receiver which can be used as a starting point for you to adapt in your own applications.

You could also write your own handler which uses the **`Lock`** class from the **`multiprocessing`** module to serialize access to the file from your processes. The stdlib **`FileHandler`** and subclasses do not make use of **`multiprocessing`**.

Alternatively, you can use a `Queue` and a **`QueueHandler`** to send all logging events to one of the processes in your multi-process application. The following example script demonstrates how you can do this; in the example a separate listener process listens for events sent by other processes and logs them according to its own logging configuration. Although the example only demonstrates one way of doing it (for example, you may want to use a listener thread rather than a separate listener process – the implementation would be analogous) it does allow for completely different logging configurations for the listener and the other processes in your application, and can be used as the basis for code meeting your own specific requirements:

```
# You'll need these imports in your own code
import logging
import logging.handlers
```

```
import multiprocessing

# Next two import lines for this demo only
from random import choice, random
import time

#
# Because you'll want to define the logging configurations for l
# listener and worker process functions take a configurer parame
# for configuring logging for that process. These functions are
# which they use for communication.
#
# In practice, you can configure the listener however you want,
# simple example, the listener does not apply level or filter lo
# In practice, you would probably want to do this logic in the w
# sending events which would be filtered out between processes.
#
# The size of the rotated files is made small so you can see the
def listener_configurer():
    root = logging.getLogger()
    h = logging.handlers.RotatingFileHandler('mptest.log', 'a',
    f = logging.Formatter('%(asctime)s %(processName)-10s %(name
    h.setFormatter(f)
    root.addHandler(h)

# This is the listener process top-level loop: wait for logging
# (LogRecords)on the queue and handle them, quit when you get a
# LogRecord.
def listener_process(queue, configurer):
    configurer()
    while True:
        try:
            record = queue.get()
            if record is None:  # We send this as a sentinel to
                break
            logger = logging.getLogger(record.name)
            logger.handle(record)  # No level or filter logic ap
        except Exception:
            import sys, traceback
            print('Whoops! Problem:', file=sys.stderr)
            traceback.print_exc(file=sys.stderr)

# Arrays used for random selections in this demo
```

```
LEVELS = [logging.DEBUG, logging.INFO, logging.WARNING,
          logging.ERROR, logging.CRITICAL]

LOGGERS = ['a.b.c', 'd.e.f']

MESSAGES = [
    'Random message #1',
    'Random message #2',
    'Random message #3',
]

# The worker configuration is done at the start of the worker pr
# Note that on Windows you can't rely on fork semantics, so each
# will run the logging configuration code when it starts.
def worker_configurer(queue):
    h = logging.handlers.QueueHandler(queue)  # Just the one har
    root = logging.getLogger()
    root.addHandler(h)
    # send all messages, for demo; no other level or filter logi
    root.setLevel(logging.DEBUG)

# This is the worker process top-level loop, which just logs ter
# random intervening delays before terminating.
# The print messages are just so you know it's doing something!
def worker_process(queue, configurer):
    configurer(queue)
    name = multiprocessing.current_process().name
    print('Worker started: %s' % name)
    for i in range(10):
        time.sleep(random())
        logger = logging.getLogger(choice(LOGGERS))
        level = choice(LEVELS)
        message = choice(MESSAGES)
        logger.log(level, message)
    print('Worker finished: %s' % name)

# Here's where the demo gets orchestrated. Create the queue, cre
# the listener, create ten workers and start them, wait for them
# then send a None to the queue to tell the listener to finish.
def main():
    queue = multiprocessing.Queue(-1)
    listener = multiprocessing.Process(target=listener_process,
```

```
                                        args=(queue, listener_cor
    listener.start()
    workers = []
    for i in range(10):
        worker = multiprocessing.Process(target=worker_process,
                                         args=(queue, worker_cor
        workers.append(worker)
        worker.start()
    for w in workers:
        w.join()
    queue.put_nowait(None)
    listener.join()

if __name__ == '__main__':
    main()
```

A variant of the above script keeps the logging in the main process, in a separate thread:

```
import logging
import logging.config
import logging.handlers
from multiprocessing import Process, Queue
import random
import threading
import time

def logger_thread(q):
    while True:
        record = q.get()
        if record is None:
            break
        logger = logging.getLogger(record.name)
        logger.handle(record)


def worker_process(q):
    qh = logging.handlers.QueueHandler(q)
    root = logging.getLogger()
    root.setLevel(logging.DEBUG)
    root.addHandler(qh)
    levels = [logging.DEBUG, logging.INFO, logging.WARNING, logg
```

```
                logging.CRITICAL]
    loggers = ['foo', 'foo.bar', 'foo.bar.baz',
                'spam', 'spam.ham', 'spam.ham.eggs']
    for i in range(100):
        lvl = random.choice(levels)
        logger = logging.getLogger(random.choice(loggers))
        logger.log(lvl, 'Message no. %d', i)

if __name__ == '__main__':
    q = Queue()
    d = {
        'version': 1,
        'formatters': {
            'detailed': {
                'class': 'logging.Formatter',
                'format': '%(asctime)s %(name)-15s %(levelname)-
            }
        },
        'handlers': {
            'console': {
                'class': 'logging.StreamHandler',
                'level': 'INFO',
            },
            'file': {
                'class': 'logging.FileHandler',
                'filename': 'mplog.log',
                'mode': 'w',
                'formatter': 'detailed',
            },
            'foofile': {
                'class': 'logging.FileHandler',
                'filename': 'mplog-foo.log',
                'mode': 'w',
                'formatter': 'detailed',
            },
            'errors': {
                'class': 'logging.FileHandler',
                'filename': 'mplog-errors.log',
                'mode': 'w',
                'level': 'ERROR',
                'formatter': 'detailed',
            },
        },
```

```
        'loggers': {
            'foo': {
                'handlers': ['foofile']
            }
        },
        'root': {
            'level': 'DEBUG',
            'handlers': ['console', 'file', 'errors']
        },
    }
    workers = []
    for i in range(5):
        wp = Process(target=worker_process, name='worker %d' % (
        workers.append(wp)
        wp.start()
    logging.config.dictConfig(d)
    lp = threading.Thread(target=logger_thread, args=(q,))
    lp.start()
    # At this point, the main process could do some useful work
    # Once it's done that, it can wait for the workers to termir
    for wp in workers:
        wp.join()
    # And now tell the logging thread to finish up, too
    q.put(None)
    lp.join()
```

This variant shows how you can e.g. apply configuration for particular loggers - e.g. the `foo` logger has a special handler which stores all events in the `foo` subsystem in a file `mplog-foo.log`. This will be used by the logging machinery in the main process (even though the logging events are generated in the worker processes) to direct the messages to the appropriate destinations.

Using concurrent.futures.ProcessPoolExecutor

If you want to use `concurrent.futures.ProcessPoolExecutor` to start your worker processes, you need to create the queue slightly differently. Instead of

```
queue = multiprocessing.Queue(-1)
```

you should use

```
queue = multiprocessing.Manager().Queue(-1)  # also works with t
```

and you can then replace the worker creation from this:

```
workers = []
for i in range(10):
    worker = multiprocessing.Process(target=worker_process,
                                     args=(queue, worker_configu
    workers.append(worker)
    worker.start()
for w in workers:
    w.join()
```

to this (remembering to first import **`concurrent.futures`**):

```
with concurrent.futures.ProcessPoolExecutor(max_workers=10) as e
    for i in range(10):
        executor.submit(worker_process, queue, worker_configurer
```

Deploying web applications using Gunicorn and uWSGI

When deploying web applications using Gunicorn [https://gunicorn.org/] or uWSGI [https://uwsgi-docs.readthedocs.io/en/latest/] (or similar), multiple worker processes are created to handle client requests. In such environments, avoid creating file-based handlers directly in your web application. Instead, use a **`SocketHandler`** to log from the web application to a listener in a separate process. This can be set up using a process management tool such as Supervisor - see Running a logging socket listener in production for more details.

Using file rotation

Sometimes you want to let a log file grow to a certain size, then open a new file and log to that. You may want to keep a certain number of these files, and when that many files have been created, rotate the files so that the

number of files and the size of the files both remain bounded. For this usage pattern, the logging package provides a **RotatingFileHandler**:

```
import glob
import logging
import logging.handlers

LOG_FILENAME = 'logging_rotatingfile_example.out'

# Set up a specific logger with our desired output level
my_logger = logging.getLogger('MyLogger')
my_logger.setLevel(logging.DEBUG)

# Add the log message handler to the logger
handler = logging.handlers.RotatingFileHandler(
              LOG_FILENAME, maxBytes=20, backupCount=5)

my_logger.addHandler(handler)

# Log some messages
for i in range(20):
    my_logger.debug('i = %d' % i)

# See what files are created
logfiles = glob.glob('%s*' % LOG_FILENAME)

for filename in logfiles:
    print(filename)
```

The result should be 6 separate files, each with part of the log history for the application:

```
logging_rotatingfile_example.out
logging_rotatingfile_example.out.1
logging_rotatingfile_example.out.2
logging_rotatingfile_example.out.3
logging_rotatingfile_example.out.4
logging_rotatingfile_example.out.5
```

The most current file is always `logging_rotatingfile_example.out`, and each time it reaches the size limit it is renamed with the suffix `.1`. Each of

the existing backup files is renamed to increment the suffix (`.1` becomes `.2`, etc.) and the `.6` file is erased.

Obviously this example sets the log length much too small as an extreme example. You would want to set *maxBytes* to an appropriate value.

Use of alternative formatting styles

When logging was added to the Python standard library, the only way of formatting messages with variable content was to use the %-formatting method. Since then, Python has gained two new formatting approaches: `string.Template` (added in Python 2.4) and `str.format()` (added in Python 2.6).

Logging (as of 3.2) provides improved support for these two additional formatting styles. The `Formatter` class been enhanced to take an additional, optional keyword parameter named `style`. This defaults to `'%'`, but other possible values are `'{'` and `'$'`, which correspond to the other two formatting styles. Backwards compatibility is maintained by default (as you would expect), but by explicitly specifying a style parameter, you get the ability to specify format strings which work with `str.format()` or `string.Template`. Here's an example console session to show the possibilities:

```
>>> import logging
>>> root = logging.getLogger()
>>> root.setLevel(logging.DEBUG)
>>> handler = logging.StreamHandler()
>>> bf = logging.Formatter('{asctime} {name} {levelname:8s} {mes
...                        style='{')
>>> handler.setFormatter(bf)
>>> root.addHandler(handler)
>>> logger = logging.getLogger('foo.bar')
>>> logger.debug('This is a DEBUG message')
2010-10-28 15:11:55,341 foo.bar DEBUG    This is a DEBUG message
>>> logger.critical('This is a CRITICAL message')
2010-10-28 15:12:11,526 foo.bar CRITICAL This is a CRITICAL mess
>>> df = logging.Formatter('$asctime $name ${levelname} $message
...                        style='$')
```

```
>>> handler.setFormatter(df)
>>> logger.debug('This is a DEBUG message')
2010-10-28 15:13:06,924 foo.bar DEBUG This is a DEBUG message
>>> logger.critical('This is a CRITICAL message')
2010-10-28 15:13:11,494 foo.bar CRITICAL This is a CRITICAL mess
>>>
```

Note that the formatting of logging messages for final output to logs is completely independent of how an individual logging message is constructed. That can still use %-formatting, as shown here:

```
>>> logger.error('This is an%s %s %s', 'other,', 'ERROR,', 'mess
2010-10-28 15:19:29,833 foo.bar ERROR This is another, ERROR, me
>>>
```

Logging calls (`logger.debug()`, `logger.info()` etc.) only take positional parameters for the actual logging message itself, with keyword parameters used only for determining options for how to handle the actual logging call (e.g. the `exc_info` keyword parameter to indicate that traceback information should be logged, or the `extra` keyword parameter to indicate additional contextual information to be added to the log). So you cannot directly make logging calls using **`str.format()`** or **`string.Template`** syntax, because internally the logging package uses %-formatting to merge the format string and the variable arguments. There would be no changing this while preserving backward compatibility, since all logging calls which are out there in existing code will be using %-format strings.

There is, however, a way that you can use {}- and $- formatting to construct your individual log messages. Recall that for a message you can use an arbitrary object as a message format string, and that the logging package will call `str()` on that object to get the actual format string. Consider the following two classes:

```
class BraceMessage:
    def __init__(self, fmt, /, *args, **kwargs):
        self.fmt = fmt
        self.args = args
        self.kwargs = kwargs
```

```
    def __str__(self):
        return self.fmt.format(*self.args, **self.kwargs)

class DollarMessage:
    def __init__(self, fmt, /, **kwargs):
        self.fmt = fmt
        self.kwargs = kwargs

    def __str__(self):
        from string import Template
        return Template(self.fmt).substitute(**self.kwargs)
```

Either of these can be used in place of a format string, to allow {}- or $-formatting to be used to build the actual “message” part which appears in the formatted log output in place of “%(message)s” or “{message}” or “$message”. It’s a little unwieldy to use the class names whenever you want to log something, but it’s quite palatable if you use an alias such as __ (double underscore — not to be confused with _, the single underscore used as a synonym/alias for `gettext.gettext()` or its brethren).

The above classes are not included in Python, though they’re easy enough to copy and paste into your own code. They can be used as follows (assuming that they’re declared in a module called `wherever`):

```
>>> from wherever import BraceMessage as __
>>> print(__('Message with {0} {name}', 2, name='placeholders'))
Message with 2 placeholders
>>> class Point: pass
...
>>> p = Point()
>>> p.x = 0.5
>>> p.y = 0.5
>>> print(__('Message with coordinates: ({point.x:.2f}, {point.y
...       point=p))
Message with coordinates: (0.50, 0.50)
>>> from wherever import DollarMessage as __
>>> print(__('Message with $num $what', num=2, what='placeholder
Message with 2 placeholders
>>>
```

While the above examples use `print()` to show how the formatting works, you would of course use `logger.debug()` or similar to actually log using this approach.

One thing to note is that you pay no significant performance penalty with this approach: the actual formatting happens not when you make the logging call, but when (and if) the logged message is actually about to be output to a log by a handler. So the only slightly unusual thing which might trip you up is that the parentheses go around the format string and the arguments, not just the format string. That's because the __ notation is just syntax sugar for a constructor call to one of the *XXX*`Message` classes.

If you prefer, you can use a **`LoggerAdapter`** to achieve a similar effect to the above, as in the following example:

```
import logging

class Message:
    def __init__(self, fmt, args):
        self.fmt = fmt
        self.args = args

    def __str__(self):
        return self.fmt.format(*self.args)

class StyleAdapter(logging.LoggerAdapter):
    def log(self, level, msg, /, *args, stacklevel=1, **kwargs):
        if self.isEnabledFor(level):
            msg, kwargs = self.process(msg, kwargs)
            self.logger.log(level, Message(msg, args), **kwargs,
                            stacklevel=stacklevel+1)

logger = StyleAdapter(logging.getLogger(__name__))

def main():
    logger.debug('Hello, {}', 'world!')

if __name__ == '__main__':
    logging.basicConfig(level=logging.DEBUG)
    main()
```

The above script should log the message `Hello, world!` when run with Python 3.8 or later.

Personalizando o `LogRecord`

Every logging event is represented by a `LogRecord` instance. When an event is logged and not filtered out by a logger's level, a `LogRecord` is created, populated with information about the event and then passed to the handlers for that logger (and its ancestors, up to and including the logger where further propagation up the hierarchy is disabled). Before Python 3.2, there were only two places where this creation was done:

- `Logger.makeRecord()`, which is called in the normal process of logging an event. This invoked `LogRecord` directly to create an instance.
- `makeLogRecord()`, which is called with a dictionary containing attributes to be added to the LogRecord. This is typically invoked when a suitable dictionary has been received over the network (e.g. in pickle form via a `SocketHandler`, or in JSON form via an `HTTPHandler`).

This has usually meant that if you need to do anything special with a `LogRecord`, you've had to do one of the following.

- Create your own `Logger` subclass, which overrides `Logger.makeRecord()`, and set it using `setLoggerClass()` before any loggers that you care about are instantiated.
- Add a `Filter` to a logger or handler, which does the necessary special manipulation you need when its `filter()` method is called.

The first approach would be a little unwieldy in the scenario where (say) several different libraries wanted to do different things. Each would attempt to set its own `Logger` subclass, and the one which did this last would win.

The second approach works reasonably well for many cases, but does not allow you to e.g. use a specialized subclass of `LogRecord`. Library developers can set a suitable filter on their loggers, but they would have to

remember to do this every time they introduced a new logger (which they would do simply by adding new packages or modules and doing

```
logger = logging.getLogger(__name__)
```

at module level). It's probably one too many things to think about. Developers could also add the filter to a **NullHandler** attached to their top-level logger, but this would not be invoked if an application developer attached a handler to a lower-level library logger — so output from that handler would not reflect the intentions of the library developer.

In Python 3.2 and later, **LogRecord** creation is done through a factory, which you can specify. The factory is just a callable you can set with **setLogRecordFactory()**, and interrogate with **getLogRecordFactory()**. The factory is invoked with the same signature as the **LogRecord** constructor, as **LogRecord** is the default setting for the factory.

This approach allows a custom factory to control all aspects of LogRecord creation. For example, you could return a subclass, or just add some additional attributes to the record once created, using a pattern similar to this:

```
old_factory = logging.getLogRecordFactory()

def record_factory(*args, **kwargs):
    record = old_factory(*args, **kwargs)
    record.custom_attribute = 0xdecafbad
    return record

logging.setLogRecordFactory(record_factory)
```

This pattern allows different libraries to chain factories together, and as long as they don't overwrite each other's attributes or unintentionally overwrite the attributes provided as standard, there should be no surprises. However, it should be borne in mind that each link in the chain adds run-time overhead to all logging operations, and the technique should only be used when the use of a **Filter** does not provide the desired result.

Subclassing QueueHandler and QueueListener- a ZeroMQ example

Subclass `QueueHandler`

You can use a `QueueHandler` subclass to send messages to other kinds of queues, for example a ZeroMQ ‘publish’ socket. In the example below,the socket is created separately and passed to the handler (as its ‘queue’):

```
import zmq   # using pyzmq, the Python binding for ZeroMQ
import json  # for serializing records portably

ctx = zmq.Context()
sock = zmq.Socket(ctx, zmq.PUB)  # or zmq.PUSH, or other suitabl
sock.bind('tcp://*:5556')        # or wherever

class ZeroMQSocketHandler(QueueHandler):
    def enqueue(self, record):
        self.queue.send_json(record.__dict__)


handler = ZeroMQSocketHandler(sock)
```

Of course there are other ways of organizing this, for example passing in the data needed by the handler to create the socket:

```
class ZeroMQSocketHandler(QueueHandler):
    def __init__(self, uri, socktype=zmq.PUB, ctx=None):
        self.ctx = ctx or zmq.Context()
        socket = zmq.Socket(self.ctx, socktype)
        socket.bind(uri)
        super().__init__(socket)

    def enqueue(self, record):
        self.queue.send_json(record.__dict__)

    def close(self):
        self.queue.close()
```

Subclass `QueueListener`

You can also subclass `QueueListener` to get messages from other kinds of queues, for example a ZeroMQ ‘subscribe’ socket. Here’s an example:

```
class ZeroMQSocketListener(QueueListener):
    def __init__(self, uri, /, *handlers, **kwargs):
        self.ctx = kwargs.get('ctx') or zmq.Context()
        socket = zmq.Socket(self.ctx, zmq.SUB)
        socket.setsockopt_string(zmq.SUBSCRIBE, '')  # subscribe
        socket.connect(uri)
        super().__init__(socket, *handlers, **kwargs)

    def dequeue(self):
        msg = self.queue.recv_json()
        return logging.makeLogRecord(msg)
```

Subclassing QueueHandler and QueueListener- a pynng example

In a similar way to the above section, we can implement a listener and handler using pynng [https://pypi.org/project/pynng/], which is a Python binding to NNG [https://nng.nanomsg.org/], billed as a spiritual successor to ZeroMQ. The following snippets illustrate – you can test them in an environment which has pynng installed. Just for variety, we present the listener first.

Subclass `QueueListener`

```
# listener.py
import json
import logging
import logging.handlers

import pynng

DEFAULT_ADDR = "tcp://localhost:13232"

interrupted = False
```

```
class NNGSocketListener(logging.handlers.QueueListener):

    def __init__(self, uri, /, *handlers, **kwargs):
        # Have a timeout for interruptibility, and open a
        # subscriber socket
        socket = pynng.Sub0(listen=uri, recv_timeout=500)
        # The b'' subscription matches all topics
        topics = kwargs.pop('topics', None) or b''
        socket.subscribe(topics)
        # We treat the socket as a queue
        super().__init__(socket, *handlers, **kwargs)

    def dequeue(self, block):
        data = None
        # Keep looping while not interrupted and no data receive
        # socket
        while not interrupted:
            try:
                data = self.queue.recv(block=block)
                break
            except pynng.Timeout:
                pass
            except pynng.Closed:  # sometimes happens when you h
                break
        if data is None:
            return None
        # Get the logging event sent from a publisher
        event = json.loads(data.decode('utf-8'))
        return logging.makeLogRecord(event)

    def enqueue_sentinel(self):
        # Not used in this implementation, as the socket isn't r
        # queue
        pass

logging.getLogger('pynng').propagate = False
listener = NNGSocketListener(DEFAULT_ADDR, logging.StreamHandler
listener.start()
print('Press Ctrl-C to stop.')
try:
    while True:
        pass
```

```
except KeyboardInterrupt:
    interrupted = True
finally:
    listener.stop()
```

Subclass `QueueHandler`

```
# sender.py
import json
import logging
import logging.handlers
import time
import random

import pynng

DEFAULT_ADDR = "tcp://localhost:13232"

class NNGSocketHandler(logging.handlers.QueueHandler):

    def __init__(self, uri):
        socket = pynng.Pub0(dial=uri, send_timeout=500)
        super().__init__(socket)

    def enqueue(self, record):
        # Send the record as UTF-8 encoded JSON
        d = dict(record.__dict__)
        data = json.dumps(d)
        self.queue.send(data.encode('utf-8'))

    def close(self):
        self.queue.close()

logging.getLogger('pynng').propagate = False
handler = NNGSocketHandler(DEFAULT_ADDR)
# Make sure the process ID is in the output
logging.basicConfig(level=logging.DEBUG,
                    handlers=[logging.StreamHandler(), handler],
                    format='%(levelname)-8s %(name)10s %(process
levels = (logging.DEBUG, logging.INFO, logging.WARNING, logging.
          logging.CRITICAL)
```

```
logger_names = ('myapp', 'myapp.lib1', 'myapp.lib2')
msgno = 1
while True:
    # Just randomly select some loggers and levels and log away
    level = random.choice(levels)
    logger = logging.getLogger(random.choice(logger_names))
    logger.log(level, 'Message no. %5d' % msgno)
    msgno += 1
    delay = random.random() * 2 + 0.5
    time.sleep(delay)
```

You can run the above two snippets in separate command shells. If we run the listener in one shell and run the sender in two separate shells, we should see something like the following. In the first sender shell:

```
$ python sender.py
DEBUG         myapp    613 Message no.     1
WARNING  myapp.lib2    613 Message no.     2
CRITICAL myapp.lib2    613 Message no.     3
WARNING  myapp.lib2    613 Message no.     4
CRITICAL myapp.lib1    613 Message no.     5
DEBUG         myapp    613 Message no.     6
CRITICAL myapp.lib1    613 Message no.     7
INFO     myapp.lib1    613 Message no.     8
(and so on)
```

In the second sender shell:

```
$ python sender.py
INFO     myapp.lib2    657 Message no.     1
CRITICAL myapp.lib2    657 Message no.     2
CRITICAL      myapp    657 Message no.     3
CRITICAL myapp.lib1    657 Message no.     4
INFO     myapp.lib1    657 Message no.     5
WARNING  myapp.lib2    657 Message no.     6
CRITICAL      myapp    657 Message no.     7
DEBUG    myapp.lib1    657 Message no.     8
(and so on)
```

In the listener shell:

```
$ python listener.py
Press Ctrl-C to stop.
DEBUG         myapp    613 Message no.     1
WARNING  myapp.lib2    613 Message no.     2
INFO     myapp.lib2    657 Message no.     1
CRITICAL myapp.lib2    613 Message no.     3
CRITICAL myapp.lib2    657 Message no.     2
CRITICAL      myapp    657 Message no.     3
WARNING  myapp.lib2    613 Message no.     4
CRITICAL myapp.lib1    613 Message no.     5
CRITICAL myapp.lib1    657 Message no.     4
INFO     myapp.lib1    657 Message no.     5
DEBUG         myapp    613 Message no.     6
WARNING  myapp.lib2    657 Message no.     6
CRITICAL      myapp    657 Message no.     7
CRITICAL myapp.lib1    613 Message no.     7
INFO     myapp.lib1    613 Message no.     8
DEBUG    myapp.lib1    657 Message no.     8
(and so on)
```

As you can see, the logging from the two sender processes is interleaved in the listener's output.

An example dictionary-based configuration

Below is an example of a logging configuration dictionary - it's taken from the [documentation on the Django project](https://docs.djangoproject.com/en/stable/topics/logging/#configuring-logging) [https://docs.djangoproject.com/en/stable/topics/logging/#configuring-logging]. This dictionary is passed to **`dictConfig()`** to put the configuration into effect:

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'verbose': {
            'format': '{levelname} {asctime} {module} {process:c
            'style': '{',
        },
        'simple': {
            'format': '{levelname} {message}',
            'style': '{',
```

```
        },
    },
    'filters': {
        'special': {
            '()': 'project.logging.SpecialFilter',
            'foo': 'bar',
        },
    },
    'handlers': {
        'console': {
            'level': 'INFO',
            'class': 'logging.StreamHandler',
            'formatter': 'simple',
        },
        'mail_admins': {
            'level': 'ERROR',
            'class': 'django.utils.log.AdminEmailHandler',
            'filters': ['special']
        }
    },
    'loggers': {
        'django': {
            'handlers': ['console'],
            'propagate': True,
        },
        'django.request': {
            'handlers': ['mail_admins'],
            'level': 'ERROR',
            'propagate': False,
        },
        'myproject.custom': {
            'handlers': ['console', 'mail_admins'],
            'level': 'INFO',
            'filters': ['special']
        }
    }
}
```

For more information about this configuration, you can see the relevant section [https://docs.djangoproject.com/en/stable/topics/logging/#configuring-logging] of the Django documentation.

Using a rotator and namer to customize log rotation processing

An example of how you can define a namer and rotator is given in the following runnable script, which shows gzip compression of the log file:

```
import gzip
import logging
import logging.handlers
import os
import shutil

def namer(name):
    return name + ".gz"

def rotator(source, dest):
    with open(source, 'rb') as f_in:
        with gzip.open(dest, 'wb') as f_out:
            shutil.copyfileobj(f_in, f_out)
    os.remove(source)


rh = logging.handlers.RotatingFileHandler('rotated.log', maxByte
rh.rotator = rotator
rh.namer = namer

root = logging.getLogger()
root.setLevel(logging.INFO)
root.addHandler(rh)
f = logging.Formatter('%(asctime)s %(message)s')
rh.setFormatter(f)
for i in range(1000):
    root.info(f'Message no. {i + 1}')
```

After running this, you will see six new files, five of which are compressed:

```
$ ls rotated.log*
rotated.log       rotated.log.2.gz  rotated.log.4.gz
rotated.log.1.gz  rotated.log.3.gz  rotated.log.5.gz
$ zcat rotated.log.1.gz
```

```
2023-01-20 02:28:17,767 Message no. 996
2023-01-20 02:28:17,767 Message no. 997
2023-01-20 02:28:17,767 Message no. 998
```

A more elaborate multiprocessing example

The following working example shows how logging can be used with multiprocessing using configuration files. The configurations are fairly simple, but serve to illustrate how more complex ones could be implemented in a real multiprocessing scenario.

In the example, the main process spawns a listener process and some worker processes. Each of the main process, the listener and the workers have three separate configurations (the workers all share the same configuration). We can see logging in the main process, how the workers log to a QueueHandler and how the listener implements a QueueListener and a more complex logging configuration, and arranges to dispatch events received via the queue to the handlers specified in the configuration. Note that these configurations are purely illustrative, but you should be able to adapt this example to your own scenario.

Here’s the script - the docstrings and the comments hopefully explain how it works:

```
import logging
import logging.config
import logging.handlers
from multiprocessing import Process, Queue, Event, current_proce
import os
import random
import time

class MyHandler:
    """
    A simple handler for logging events. It runs in the listener
    dispatches events to loggers based on the name in the receiv
    which then get dispatched, by the logging system, to the har
    configured for those loggers.
    """
```

```
    def handle(self, record):
        if record.name == "root":
            logger = logging.getLogger()
        else:
            logger = logging.getLogger(record.name)

        if logger.isEnabledFor(record.levelno):
            # The process name is transformed just to show that
            # doing the logging to files and console
            record.processName = '%s (for %s)' % (current_proces
            logger.handle(record)

def listener_process(q, stop_event, config):
    """
    This could be done in the main process, but is just done in
    process for illustrative purposes.

    This initialises logging according to the specified configur
    starts the listener and waits for the main process to signal
    via the event. The listener is then stopped, and the process
    """
    logging.config.dictConfig(config)
    listener = logging.handlers.QueueListener(q, MyHandler())
    listener.start()
    if os.name == 'posix':
        # On POSIX, the setup logger will have been configured i
        # parent process, but should have been disabled followir
        # dictConfig call.
        # On Windows, since fork isn't used, the setup logger wo
        # exist in the child, so it would be created and the mes
        # would appear - hence the "if posix" clause.
        logger = logging.getLogger('setup')
        logger.critical('Should not appear, because of disabled
    stop_event.wait()
    listener.stop()

def worker_process(config):
    """
    A number of these are spawned for the purpose of illustratic
    practice, they could be a heterogeneous bunch of processes r
    ones which are identical to each other.
```

```
    This initialises logging according to the specified configur
    and logs a hundred messages with random levels to randomly s
    loggers.

    A small sleep is added to allow other processes a chance to
    is not strictly needed, but it mixes the output from the dif
    processes a bit more than if it's left out.
    """
    logging.config.dictConfig(config)
    levels = [logging.DEBUG, logging.INFO, logging.WARNING, logg
              logging.CRITICAL]
    loggers = ['foo', 'foo.bar', 'foo.bar.baz',
               'spam', 'spam.ham', 'spam.ham.eggs']
    if os.name == 'posix':
        # On POSIX, the setup logger will have been configured i
        # parent process, but should have been disabled followir
        # dictConfig call.
        # On Windows, since fork isn't used, the setup logger wo
        # exist in the child, so it would be created and the mes
        # would appear - hence the "if posix" clause.
        logger = logging.getLogger('setup')
        logger.critical('Should not appear, because of disabled
    for i in range(100):
        lvl = random.choice(levels)
        logger = logging.getLogger(random.choice(loggers))
        logger.log(lvl, 'Message no. %d', i)
        time.sleep(0.01)

def main():
    q = Queue()
    # The main process gets a simple configuration which prints
    config_initial = {
        'version': 1,
        'handlers': {
            'console': {
                'class': 'logging.StreamHandler',
                'level': 'INFO'
            }
        },
        'root': {
            'handlers': ['console'],
            'level': 'DEBUG'
        }
```

```
}
# The worker process configuration is just a QueueHandler at
# root logger, which allows all messages to be sent to the c
# We disable existing loggers to disable the "setup" logger
# parent process. This is needed on POSIX because the logger
# be there in the child following a fork().
config_worker = {
    'version': 1,
    'disable_existing_loggers': True,
    'handlers': {
        'queue': {
            'class': 'logging.handlers.QueueHandler',
            'queue': q
        }
    },
    'root': {
        'handlers': ['queue'],
        'level': 'DEBUG'
    }
}
# The listener process configuration shows that the full fle
# logging configuration is available to dispatch events to h
# you want.
# We disable existing loggers to disable the "setup" logger
# parent process. This is needed on POSIX because the logger
# be there in the child following a fork().
config_listener = {
    'version': 1,
    'disable_existing_loggers': True,
    'formatters': {
        'detailed': {
            'class': 'logging.Formatter',
            'format': '%(asctime)s %(name)-15s %(levelname)-
        },
        'simple': {
            'class': 'logging.Formatter',
            'format': '%(name)-15s %(levelname)-8s %(process
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'simple',
```

```
            'level': 'INFO'
        },
        'file': {
            'class': 'logging.FileHandler',
            'filename': 'mplog.log',
            'mode': 'w',
            'formatter': 'detailed'
        },
        'foofile': {
            'class': 'logging.FileHandler',
            'filename': 'mplog-foo.log',
            'mode': 'w',
            'formatter': 'detailed'
        },
        'errors': {
            'class': 'logging.FileHandler',
            'filename': 'mplog-errors.log',
            'mode': 'w',
            'formatter': 'detailed',
            'level': 'ERROR'
        }
    },
    'loggers': {
        'foo': {
            'handlers': ['foofile']
        }
    },
    'root': {
        'handlers': ['console', 'file', 'errors'],
        'level': 'DEBUG'
    }
}
# Log some initial events, just to show that logging in the
# normally.
logging.config.dictConfig(config_initial)
logger = logging.getLogger('setup')
logger.info('About to create workers ...')
workers = []
for i in range(5):
    wp = Process(target=worker_process, name='worker %d' % (
                 args=(config_worker,))
    workers.append(wp)
    wp.start()
```

```
        logger.info('Started worker: %s', wp.name)
    logger.info('About to create listener ...')
    stop_event = Event()
    lp = Process(target=listener_process, name='listener',
                 args=(q, stop_event, config_listener))
    lp.start()
    logger.info('Started listener')
    # We now hang around for the workers to finish their work.
    for wp in workers:
        wp.join()
    # Workers all done, listening can now stop.
    # Logging in the parent still works normally.
    logger.info('Telling listener to stop ...')
    stop_event.set()
    lp.join()
    logger.info('All done.')

if __name__ == '__main__':
    main()
```

Inserting a BOM into messages sent to a SysLogHandler

RFC 5424 [https://datatracker.ietf.org/doc/html/rfc5424.html] requires that a Unicode message be sent to a syslog daemon as a set of bytes which have the following structure: an optional pure-ASCII component, followed by a UTF-8 Byte Order Mark (BOM), followed by Unicode encoded using UTF-8. (See the **relevant section of the specification** [https://datatracker.ietf.org/doc/html/rfc5424.html#section-6].)

In Python 3.1, code was added to `SysLogHandler` to insert a BOM into the message, but unfortunately, it was implemented incorrectly, with the BOM appearing at the beginning of the message and hence not allowing any pure-ASCII component to appear before it.

As this behaviour is broken, the incorrect BOM insertion code is being removed from Python 3.2.4 and later. However, it is not being replaced, and if you want to produce **RFC 5424** [https://datatracker.ietf.org/doc/html/rfc5424.html]-

compliant messages which include a BOM, an optional pure-ASCII sequence before it and arbitrary Unicode after it, encoded using UTF-8, then you need to do the following:

1. Attach a `Formatter` instance to your `SysLogHandler` instance, with a format string such as:

   ```
   'ASCII section\ufeffUnicode section'
   ```

   The Unicode code point U+FEFF, when encoded using UTF-8, will be encoded as a UTF-8 BOM – the byte-string `b'\xef\xbb\xbf'`.

2. Replace the ASCII section with whatever placeholders you like, but make sure that the data that appears in there after substitution is always ASCII (that way, it will remain unchanged after UTF-8 encoding).

3. Replace the Unicode section with whatever placeholders you like; if the data which appears there after substitution contains characters outside the ASCII range, that's fine – it will be encoded using UTF-8.

The formatted message *will* be encoded using UTF-8 encoding by `SysLogHandler`. If you follow the above rules, you should be able to produce **RFC 5424** [https://datatracker.ietf.org/doc/html/rfc5424.html]-compliant messages. If you don't, logging may not complain, but your messages will not be RFC 5424-compliant, and your syslog daemon may complain.

Implementing structured logging

Although most logging messages are intended for reading by humans, and thus not readily machine-parseable, there might be circumstances where you want to output messages in a structured format which *is* capable of being parsed by a program (without needing complex regular expressions to parse the log message). This is straightforward to achieve using the logging package. There are a number of ways in which this could be achieved, but the following is a simple approach which uses JSON to serialise the event in a machine-parseable manner:

```
import json
import logging

class StructuredMessage:
    def __init__(self, message, /, **kwargs):
        self.message = message
        self.kwargs = kwargs

    def __str__(self):
        return '%s >>> %s' % (self.message, json.dumps(self.kwar

_ = StructuredMessage   # optional, to improve readability

logging.basicConfig(level=logging.INFO, format='%(message)s')
logging.info(_('message 1', foo='bar', bar='baz', num=123, fnum=
```

If the above script is run, it prints:

```
message 1 >>> {"fnum": 123.456, "num": 123, "bar": "baz", "foo":
```

Note that the order of items might be different according to the version of Python used.

If you need more specialised processing, you can use a custom JSON encoder, as in the following complete example:

```
import json
import logging


class Encoder(json.JSONEncoder):
    def default(self, o):
        if isinstance(o, set):
            return tuple(o)
        elif isinstance(o, str):
            return o.encode('unicode_escape').decode('ascii')
        return super().default(o)

class StructuredMessage:
    def __init__(self, message, /, **kwargs):
```

```
        self.message = message
        self.kwargs = kwargs

    def __str__(self):
        s = Encoder().encode(self.kwargs)
        return '%s >>> %s' % (self.message, s)

_ = StructuredMessage   # optional, to improve readability

def main():
    logging.basicConfig(level=logging.INFO, format='%(message)s'
    logging.info(_('message 1', set_value={1, 2, 3}, snowman='\u

if __name__ == '__main__':
    main()
```

When the above script is run, it prints:

```
message 1 >>> {"snowman": "\u2603", "set_value": [1, 2, 3]}
```

Note that the order of items might be different according to the version of Python used.

Customizing handlers with `dictConfig()`

There are times when you want to customize logging handlers in particular ways, and if you use `dictConfig()` you may be able to do this without subclassing. As an example, consider that you may want to set the ownership of a log file. On POSIX, this is easily done using `shutil.chown()`, but the file handlers in the stdlib don't offer built-in support. You can customize handler creation using a plain function such as:

```
def owned_file_handler(filename, mode='a', encoding=None, owner=
    if owner:
        if not os.path.exists(filename):
            open(filename, 'a').close()
        shutil.chown(filename, *owner)
    return logging.FileHandler(filename, mode, encoding)
```

You can then specify, in a logging configuration passed to **dictConfig()**, that a logging handler be created by calling this function:

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'default': {
            'format': '%(asctime)s %(levelname)s %(name)s %(mess
        },
    },
    'handlers': {
        'file':{
            # The values below are popped from this dictionary a
            # used to create the handler, set the handler's leve
            # its formatter.
            '()': owned_file_handler,
            'level':'DEBUG',
            'formatter': 'default',
            # The values below are passed to the handler creator
            # as keyword arguments.
            'owner': ['pulse', 'pulse'],
            'filename': 'chowntest.log',
            'mode': 'w',
            'encoding': 'utf-8',
        },
    },
    'root': {
        'handlers': ['file'],
        'level': 'DEBUG',
    },
}
```

In this example I am setting the ownership using the `pulse` user and group, just for the purposes of illustration. Putting it together into a working script, `chowntest.py`:

```
import logging, logging.config, os, shutil

def owned_file_handler(filename, mode='a', encoding=None, owner=
    if owner:
```

```
        if not os.path.exists(filename):
            open(filename, 'a').close()
        shutil.chown(filename, *owner)
    return logging.FileHandler(filename, mode, encoding)

LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'default': {
            'format': '%(asctime)s %(levelname)s %(name)s %(mess
        },
    },
    'handlers': {
        'file':{
            # The values below are popped from this dictionary a
            # used to create the handler, set the handler's leve
            # its formatter.
            '()': owned_file_handler,
            'level':'DEBUG',
            'formatter': 'default',
            # The values below are passed to the handler creator
            # as keyword arguments.
            'owner': ['pulse', 'pulse'],
            'filename': 'chowntest.log',
            'mode': 'w',
            'encoding': 'utf-8',
        },
    },
    'root': {
        'handlers': ['file'],
        'level': 'DEBUG',
    },
}

logging.config.dictConfig(LOGGING)
logger = logging.getLogger('mylogger')
logger.debug('A debug message')
```

To run this, you will probably need to run as `root`:

```
$ sudo python3.3 chowntest.py
$ cat chowntest.log
2013-11-05 09:34:51,128 DEBUG mylogger A debug message
$ ls -l chowntest.log
-rw-r--r-- 1 pulse pulse 55 2013-11-05 09:34 chowntest.log
```

Note that this example uses Python 3.3 because that's where `shutil.chown()` makes an appearance. This approach should work with any Python version that supports `dictConfig()` - namely, Python 2.7, 3.2 or later. With pre-3.3 versions, you would need to implement the actual ownership change using e.g. `os.chown()`.

In practice, the handler-creating function may be in a utility module somewhere in your project. Instead of the line in the configuration:

```
'()': owned_file_handler,
```

you could use e.g.:

```
'()': 'ext://project.util.owned_file_handler',
```

where `project.util` can be replaced with the actual name of the package where the function resides. In the above working script, using `'ext://__main__.owned_file_handler'` should work. Here, the actual callable is resolved by `dictConfig()` from the `ext://` specification.

This example hopefully also points the way to how you could implement other types of file change - e.g. setting specific POSIX permission bits - in the same way, using `os.chmod()`.

Of course, the approach could also be extended to types of handler other than a `FileHandler` - for example, one of the rotating file handlers, or a different type of handler altogether.

Using particular formatting styles throughout your application

In Python 3.2, the `Formatter` gained a `style` keyword parameter which, while defaulting to `%` for backward compatibility, allowed the specification of `{` or `$` to support the formatting approaches supported by `str.format()` and `string.Template`. Note that this governs the formatting of logging messages for final output to logs, and is completely orthogonal to how an individual logging message is constructed.

Logging calls (`debug()`, `info()` etc.) only take positional parameters for the actual logging message itself, with keyword parameters used only for determining options for how to handle the logging call (e.g. the `exc_info` keyword parameter to indicate that traceback information should be logged, or the `extra` keyword parameter to indicate additional contextual information to be added to the log). So you cannot directly make logging calls using `str.format()` or `string.Template` syntax, because internally the logging package uses %-formatting to merge the format string and the variable arguments. There would be no changing this while preserving backward compatibility, since all logging calls which are out there in existing code will be using %-format strings.

There have been suggestions to associate format styles with specific loggers, but that approach also runs into backward compatibility problems because any existing code could be using a given logger name and using %-formatting.

For logging to work interoperably between any third-party libraries and your code, decisions about formatting need to be made at the level of the individual logging call. This opens up a couple of ways in which alternative formatting styles can be accommodated.

Using LogRecord factories

In Python 3.2, along with the `Formatter` changes mentioned above, the logging package gained the ability to allow users to set their own `LogRecord` subclasses, using the `setLogRecordFactory()` function. You can use this to set your own subclass of **`LogRecord`**, which does the Right Thing by overriding the `getMessage()` method. The base class implementation of this method is where the `msg % args` formatting happens, and where you

can substitute your alternate formatting; however, you should be careful to support all formatting styles and allow %-formatting as the default, to ensure interoperability with other code. Care should also be taken to call `str(self.msg)`, just as the base implementation does.

Refer to the reference documentation on **`setLogRecordFactory()`** and **`LogRecord`** for more information.

Using custom message objects

There is another, perhaps simpler way that you can use {}- and $-formatting to construct your individual log messages. You may recall (from Usando objetos arbitrários como mensagens) that when logging you can use an arbitrary object as a message format string, and that the logging package will call **`str()`** on that object to get the actual format string. Consider the following two classes:

```
class BraceMessage:
    def __init__(self, fmt, /, *args, **kwargs):
        self.fmt = fmt
        self.args = args
        self.kwargs = kwargs

    def __str__(self):
        return self.fmt.format(*self.args, **self.kwargs)

class DollarMessage:
    def __init__(self, fmt, /, **kwargs):
        self.fmt = fmt
        self.kwargs = kwargs

    def __str__(self):
        from string import Template
        return Template(self.fmt).substitute(**self.kwargs)
```

Either of these can be used in place of a format string, to allow {}- or $-formatting to be used to build the actual “message” part which appears in the formatted log output in place of “%(message)s” or “{message}” or “$message”. If you find it a little unwieldy to use the class names whenever you want to log something, you can make it more palatable if you use an

alias such as `M` or _ for the message (or perhaps __, if you are using _ for localization).

Examples of this approach are given below. Firstly, formatting with `str.format()`:

```
>>> __ = BraceMessage
>>> print(__('Message with {0} {1}', 2, 'placeholders'))
Message with 2 placeholders
>>> class Point: pass
...
>>> p = Point()
>>> p.x = 0.5
>>> p.y = 0.5
>>> print(__('Message with coordinates: ({point.x:.2f}, {point.y
Message with coordinates: (0.50, 0.50)
```

Secondly, formatting with `string.Template`:

```
>>> __ = DollarMessage
>>> print(__('Message with $num $what', num=2, what='placeholder
Message with 2 placeholders
>>>
```

One thing to note is that you pay no significant performance penalty with this approach: the actual formatting happens not when you make the logging call, but when (and if) the logged message is actually about to be output to a log by a handler. So the only slightly unusual thing which might trip you up is that the parentheses go around the format string and the arguments, not just the format string. That's because the __ notation is just syntax sugar for a constructor call to one of the `XXXMessage` classes shown above.

Configuring filters with `dictConfig()`

You *can* configure filters using `dictConfig()`, though it might not be obvious at first glance how to do it (hence this recipe). Since `Filter` is the only filter class included in the standard library, and it is unlikely to cater to

many requirements (it's only there as a base class), you will typically need to define your own **`Filter`** subclass with an overridden **`filter()`** method. To do this, specify the `()` key in the configuration dictionary for the filter, specifying a callable which will be used to create the filter (a class is the most obvious, but you can provide any callable which returns a **`Filter`** instance). Here is a complete example:

```
import logging
import logging.config
import sys

class MyFilter(logging.Filter):
    def __init__(self, param=None):
        self.param = param

    def filter(self, record):
        if self.param is None:
            allow = True
        else:
            allow = self.param not in record.msg
        if allow:
            record.msg = 'changed: ' + record.msg
        return allow

LOGGING = {
    'version': 1,
    'filters': {
        'myfilter': {
            '()': MyFilter,
            'param': 'noshow',
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'filters': ['myfilter']
        }
    },
    'root': {
        'level': 'DEBUG',
        'handlers': ['console']
    },
```

```
}

if __name__ == '__main__':
    logging.config.dictConfig(LOGGING)
    logging.debug('hello')
    logging.debug('hello - noshow')
```

This example shows how you can pass configuration data to the callable which constructs the instance, in the form of keyword parameters. When run, the above script will print:

```
changed: hello
```

which shows that the filter is working as configured.

A couple of extra points to note:

- If you can't refer to the callable directly in the configuration (e.g. if it lives in a different module, and you can't import it directly where the configuration dictionary is), you can use the form `ext://...` as described in Acesso a objetos externos. For example, you could have used the text `'ext://__main__.MyFilter'` instead of `MyFilter` in the above example.
- As well as for filters, this technique can also be used to configure custom handlers and formatters. See Objetos definidos pelo usuário for more information on how logging supports using user-defined objects in its configuration, and see the other cookbook recipe Customizing handlers with dictConfig() above.

Customized exception formatting

There might be times when you want to do customized exception formatting - for argument's sake, let's say you want exactly one line per logged event, even when exception information is present. You can do this with a custom formatter class, as shown in the following example:

```
import logging

class OneLineExceptionFormatter(logging.Formatter):
```

```
    def formatException(self, exc_info):
        """
        Format an exception so that it prints on a single line.
        """
        result = super().formatException(exc_info)
        return repr(result)  # or format into one line however y

    def format(self, record):
        s = super().format(record)
        if record.exc_text:
            s = s.replace('\n', '') + '|'
        return s

def configure_logging():
    fh = logging.FileHandler('output.txt', 'w')
    f = OneLineExceptionFormatter('%(asctime)s|%(levelname)s|%(m
                                  '%d/%m/%Y %H:%M:%S')
    fh.setFormatter(f)
    root = logging.getLogger()
    root.setLevel(logging.DEBUG)
    root.addHandler(fh)

def main():
    configure_logging()
    logging.info('Sample message')
    try:
        x = 1 / 0
    except ZeroDivisionError as e:
        logging.exception('ZeroDivisionError: %s', e)

if __name__ == '__main__':
    main()
```

When run, this produces a file with exactly two lines:

```
28/01/2015 07:21:23|INFO|Sample message|
28/01/2015 07:21:23|ERROR|ZeroDivisionError: division by zero|'T
```

While the above treatment is simplistic, it points the way to how exception information can be formatted to your liking. The `traceback` module may be helpful for more specialized needs.

Speaking logging messages

There might be situations when it is desirable to have logging messages rendered in an audible rather than a visible format. This is easy to do if you have text-to-speech (TTS) functionality available in your system, even if it doesn't have a Python binding. Most TTS systems have a command line program you can run, and this can be invoked from a handler using `subprocess`. It's assumed here that TTS command line programs won't expect to interact with users or take a long time to complete, and that the frequency of logged messages will be not so high as to swamp the user with messages, and that it's acceptable to have the messages spoken one at a time rather than concurrently, The example implementation below waits for one message to be spoken before the next is processed, and this might cause other handlers to be kept waiting. Here is a short example showing the approach, which assumes that the `espeak` TTS package is available:

```
import logging
import subprocess
import sys

class TTSHandler(logging.Handler):
    def emit(self, record):
        msg = self.format(record)
        # Speak slowly in a female English voice
        cmd = ['espeak', '-s150', '-ven+f3', msg]
        p = subprocess.Popen(cmd, stdout=subprocess.PIPE,
                             stderr=subprocess.STDOUT)
        # wait for the program to finish
        p.communicate()

def configure_logging():
    h = TTSHandler()
    root = logging.getLogger()
    root.addHandler(h)
    # the default formatter just returns the message
    root.setLevel(logging.DEBUG)

def main():
    logging.info('Hello')
    logging.debug('Goodbye')
```

```
if __name__ == '__main__':
    configure_logging()
    sys.exit(main())
```

When run, this script should say “Hello” and then “Goodbye” in a female voice.

The above approach can, of course, be adapted to other TTS systems and even other systems altogether which can process messages via external programs run from a command line.

Buffering logging messages and outputting them conditionally

There might be situations where you want to log messages in a temporary area and only output them if a certain condition occurs. For example, you may want to start logging debug events in a function, and if the function completes without errors, you don’t want to clutter the log with the collected debug information, but if there is an error, you want all the debug information to be output as well as the error.

Here is an example which shows how you could do this using a decorator for your functions where you want logging to behave this way. It makes use of the `logging.handlers.MemoryHandler`, which allows buffering of logged events until some condition occurs, at which point the buffered events are `flushed` - passed to another handler (the `target` handler) for processing. By default, the `MemoryHandler` flushed when its buffer gets filled up or an event whose level is greater than or equal to a specified threshold is seen. You can use this recipe with a more specialised subclass of `MemoryHandler` if you want custom flushing behavior.

The example script has a simple function, `foo`, which just cycles through all the logging levels, writing to `sys.stderr` to say what level it’s about to log at, and then actually logging a message at that level. You can pass a parameter to `foo` which, if true, will log at ERROR and CRITICAL levels - otherwise, it only logs at DEBUG, INFO and WARNING levels.

The script just arranges to decorate `foo` with a decorator which will do the conditional logging that's required. The decorator takes a logger as a parameter and attaches a memory handler for the duration of the call to the decorated function. The decorator can be additionally parameterised using a target handler, a level at which flushing should occur, and a capacity for the buffer (number of records buffered). These default to a **`StreamHandler`** which writes to `sys.stderr`, `logging.ERROR` and `100` respectively.

Here's the script:

```
import logging
from logging.handlers import MemoryHandler
import sys

logger = logging.getLogger(__name__)
logger.addHandler(logging.NullHandler())

def log_if_errors(logger, target_handler=None, flush_level=None,
    if target_handler is None:
        target_handler = logging.StreamHandler()
    if flush_level is None:
        flush_level = logging.ERROR
    if capacity is None:
        capacity = 100
    handler = MemoryHandler(capacity, flushLevel=flush_level, ta

    def decorator(fn):
        def wrapper(*args, **kwargs):
            logger.addHandler(handler)
            try:
                return fn(*args, **kwargs)
            except Exception:
                logger.exception('call failed')
                raise
            finally:
                super(MemoryHandler, handler).flush()
                logger.removeHandler(handler)
        return wrapper

    return decorator

def write_line(s):
```

```
    sys.stderr.write('%s\n' % s)

def foo(fail=False):
    write_line('about to log at DEBUG ...')
    logger.debug('Actually logged at DEBUG')
    write_line('about to log at INFO ...')
    logger.info('Actually logged at INFO')
    write_line('about to log at WARNING ...')
    logger.warning('Actually logged at WARNING')
    if fail:
        write_line('about to log at ERROR ...')
        logger.error('Actually logged at ERROR')
        write_line('about to log at CRITICAL ...')
        logger.critical('Actually logged at CRITICAL')
    return fail

decorated_foo = log_if_errors(logger)(foo)

if __name__ == '__main__':
    logger.setLevel(logging.DEBUG)
    write_line('Calling undecorated foo with False')
    assert not foo(False)
    write_line('Calling undecorated foo with True')
    assert foo(True)
    write_line('Calling decorated foo with False')
    assert not decorated_foo(False)
    write_line('Calling decorated foo with True')
    assert decorated_foo(True)
```

When this script is run, the following output should be observed:

```
Calling undecorated foo with False
about to log at DEBUG ...
about to log at INFO ...
about to log at WARNING ...
Calling undecorated foo with True
about to log at DEBUG ...
about to log at INFO ...
about to log at WARNING ...
about to log at ERROR ...
about to log at CRITICAL ...
Calling decorated foo with False
```

```
about to log at DEBUG ...
about to log at INFO ...
about to log at WARNING ...
Calling decorated foo with True
about to log at DEBUG ...
about to log at INFO ...
about to log at WARNING ...
about to log at ERROR ...
Actually logged at DEBUG
Actually logged at INFO
Actually logged at WARNING
Actually logged at ERROR
about to log at CRITICAL ...
Actually logged at CRITICAL
```

As you can see, actual logging output only occurs when an event is logged whose severity is ERROR or greater, but in that case, any previous events at lower severities are also logged.

You can of course use the conventional means of decoration:

```
@log_if_errors(logger)
def foo(fail=False):
    ...
```

Sending logging messages to email, with buffering

To illustrate how you can send log messages via email, so that a set number of messages are sent per email, you can subclass **`BufferingHandler`**. In the following example, which you can adapt to suit your specific needs, a simple test harness is provided which allows you to run the script with command line arguments specifying what you typically need to send things via SMTP. (Run the downloaded script with the `-h` argument to see the required and optional arguments.)

```
import logging
import logging.handlers
import smtplib

class BufferingSMTPHandler(logging.handlers.BufferingHandler):
```

```
    def __init__(self, mailhost, port, username, password, froma
                 subject, capacity):
        logging.handlers.BufferingHandler.__init__(self, capacit
        self.mailhost = mailhost
        self.mailport = port
        self.username = username
        self.password = password
        self.fromaddr = fromaddr
        if isinstance(toaddrs, str):
            toaddrs = [toaddrs]
        self.toaddrs = toaddrs
        self.subject = subject
        self.setFormatter(logging.Formatter("%(asctime)s %(level

    def flush(self):
        if len(self.buffer) > 0:
            try:
                smtp = smtplib.SMTP(self.mailhost, self.mailport
                smtp.starttls()
                smtp.login(self.username, self.password)
                msg = "From: %s\r\nTo: %s\r\nSubject: %s\r\n\r\r
                for record in self.buffer:
                    s = self.format(record)
                    msg = msg + s + "\r\n"
                smtp.sendmail(self.fromaddr, self.toaddrs, msg)
                smtp.quit()
            except Exception:
                if logging.raiseExceptions:
                    raise
            self.buffer = []

if __name__ == '__main__':
    import argparse

    ap = argparse.ArgumentParser()
    aa = ap.add_argument
    aa('host', metavar='HOST', help='SMTP server')
    aa('--port', '-p', type=int, default=587, help='SMTP port')
    aa('user', metavar='USER', help='SMTP username')
    aa('password', metavar='PASSWORD', help='SMTP password')
    aa('to', metavar='TO', help='Addressee for emails')
    aa('sender', metavar='SENDER', help='Sender email address')
    aa('--subject', '-s',
```

```
       default='Test Logging email from Python logging module (b
       help='Subject of email')
    options = ap.parse_args()
    logger = logging.getLogger()
    logger.setLevel(logging.DEBUG)
    h = BufferingSMTPHandler(options.host, options.port, options
                             options.password, options.sender,
                             options.to, options.subject, 10)
    logger.addHandler(h)
    for i in range(102):
        logger.info("Info index = %d", i)
    h.flush()
    h.close()
```

If you run this script and your SMTP server is correctly set up, you should find that it sends eleven emails to the addressee you specify. The first ten emails will each have ten log messages, and the eleventh will have two messages. That makes up 102 messages as specified in the script.

Formatting times using UTC (GMT) via configuration

Sometimes you want to format times using UTC, which can be done using a class such as `UTCFormatter`, shown below:

```
import logging
import time

class UTCFormatter(logging.Formatter):
    converter = time.gmtime
```

and you can then use the `UTCFormatter` in your code instead of **`Formatter`**. If you want to do that via configuration, you can use the **`dictConfig()`** API with an approach illustrated by the following complete example:

```
import logging
import logging.config
import time
```

```
class UTCFormatter(logging.Formatter):
    converter = time.gmtime

LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'utc': {
            '()': UTCFormatter,
            'format': '%(asctime)s %(message)s',
        },
        'local': {
            'format': '%(asctime)s %(message)s',
        }
    },
    'handlers': {
        'console1': {
            'class': 'logging.StreamHandler',
            'formatter': 'utc',
        },
        'console2': {
            'class': 'logging.StreamHandler',
            'formatter': 'local',
        },
    },
    'root': {
        'handlers': ['console1', 'console2'],
   }
}

if __name__ == '__main__':
    logging.config.dictConfig(LOGGING)
    logging.warning('The local time is %s', time.asctime())
```

When this script is run, it should print something like:

```
2015-10-17 12:53:29,501 The local time is Sat Oct 17 13:53:29 20
2015-10-17 13:53:29,501 The local time is Sat Oct 17 13:53:29 20
```

showing how the time is formatted both as local time and UTC, one for each handler.

Using a context manager for selective logging

There are times when it would be useful to temporarily change the logging configuration and revert it back after doing something. For this, a context manager is the most obvious way of saving and restoring the logging context. Here is a simple example of such a context manager, which allows you to optionally change the logging level and add a logging handler purely in the scope of the context manager:

```
import logging
import sys

class LoggingContext:
    def __init__(self, logger, level=None, handler=None, close=T
        self.logger = logger
        self.level = level
        self.handler = handler
        self.close = close

    def __enter__(self):
        if self.level is not None:
            self.old_level = self.logger.level
            self.logger.setLevel(self.level)
        if self.handler:
            self.logger.addHandler(self.handler)

    def __exit__(self, et, ev, tb):
        if self.level is not None:
            self.logger.setLevel(self.old_level)
        if self.handler:
            self.logger.removeHandler(self.handler)
        if self.handler and self.close:
            self.handler.close()
        # implicit return of None => don't swallow exceptions
```

If you specify a level value, the logger’s level is set to that value in the scope of the with block covered by the context manager. If you specify a handler, it is added to the logger on entry to the block and removed on exit

from the block. You can also ask the manager to close the handler for you on block exit - you could do this if you don't need the handler any more.

To illustrate how it works, we can add the following block of code to the above:

```
if __name__ == '__main__':
    logger = logging.getLogger('foo')
    logger.addHandler(logging.StreamHandler())
    logger.setLevel(logging.INFO)
    logger.info('1. This should appear just once on stderr.')
    logger.debug('2. This should not appear.')
    with LoggingContext(logger, level=logging.DEBUG):
        logger.debug('3. This should appear once on stderr.')
    logger.debug('4. This should not appear.')
    h = logging.StreamHandler(sys.stdout)
    with LoggingContext(logger, level=logging.DEBUG, handler=h,
        logger.debug('5. This should appear twice - once on stde
    logger.info('6. This should appear just once on stderr.')
    logger.debug('7. This should not appear.')
```

We initially set the logger's level to `INFO`, so message #1 appears and message #2 doesn't. We then change the level to `DEBUG` temporarily in the following `with` block, and so message #3 appears. After the block exits, the logger's level is restored to `INFO` and so message #4 doesn't appear. In the next `with` block, we set the level to `DEBUG` again but also add a handler writing to `sys.stdout`. Thus, message #5 appears twice on the console (once via `stderr` and once via `stdout`). After the `with` statement's completion, the status is as it was before so message #6 appears (like message #1) whereas message #7 doesn't (just like message #2).

If we run the resulting script, the result is as follows:

```
$ python logctx.py
1. This should appear just once on stderr.
3. This should appear once on stderr.
5. This should appear twice - once on stderr and once on stdout.
5. This should appear twice - once on stderr and once on stdout.
6. This should appear just once on stderr.
```

If we run it again, but pipe `stderr` to `/dev/null`, we see the following, which is the only message written to `stdout`:

```
$ python logctx.py 2>/dev/null
5. This should appear twice - once on stderr and once on stdout.
```

Once again, but piping `stdout` to `/dev/null`, we get:

```
$ python logctx.py >/dev/null
1. This should appear just once on stderr.
3. This should appear once on stderr.
5. This should appear twice - once on stderr and once on stdout.
6. This should appear just once on stderr.
```

In this case, the message #5 printed to `stdout` doesn't appear, as expected.

Of course, the approach described here can be generalised, for example to attach logging filters temporarily. Note that the above code works in Python 2 as well as Python 3.

A CLI application starter template

Here's an example which shows how you can:

- Use a logging level based on command-line arguments
- Dispatch to multiple subcommands in separate files, all logging at the same level in a consistent way
- Make use of simple, minimal configuration

Suppose we have a command-line application whose job is to stop, start or restart some services. This could be organised for the purposes of illustration as a file `app.py` that is the main script for the application, with individual commands implemented in `start.py`, `stop.py` and `restart.py`. Suppose further that we want to control the verbosity of the application via a command-line argument, defaulting to `logging.INFO`. Here's one way that `app.py` could be written:

```
import argparse
import importlib
import logging
import os
import sys

def main(args=None):
    scriptname = os.path.basename(__file__)
    parser = argparse.ArgumentParser(scriptname)
    levels = ('DEBUG', 'INFO', 'WARNING', 'ERROR', 'CRITICAL')
    parser.add_argument('--log-level', default='INFO', choices=l
    subparsers = parser.add_subparsers(dest='command',
                                       help='Available commands:
    start_cmd = subparsers.add_parser('start', help='Start a ser
    start_cmd.add_argument('name', metavar='NAME',
                           help='Name of service to start')
    stop_cmd = subparsers.add_parser('stop',
                                     help='Stop one or more serv
    stop_cmd.add_argument('names', metavar='NAME', nargs='+',
                          help='Name of service to stop')
    restart_cmd = subparsers.add_parser('restart',
                                        help='Restart one or mor
    restart_cmd.add_argument('names', metavar='NAME', nargs='+',
                             help='Name of service to restart')
    options = parser.parse_args()
    # the code to dispatch commands could all be in this file. F
    # of illustration only, we implement each command in a separ
    try:
        mod = importlib.import_module(options.command)
        cmd = getattr(mod, 'command')
    except (ImportError, AttributeError):
        print('Unable to find the code for command \'%s\'' % opt
        return 1
    # Could get fancy here and load configuration from file or c
    logging.basicConfig(level=options.log_level,
                        format='%(levelname)s %(name)s %(message
    cmd(options)

if __name__ == '__main__':
    sys.exit(main())
```

And the `start`, `stop` and `restart` commands can be implemented in separate modules, like so for starting:

```
# start.py
import logging

logger = logging.getLogger(__name__)

def command(options):
    logger.debug('About to start %s', options.name)
    # actually do the command processing here ...
    logger.info('Started the \'%s\' service.', options.name)
```

and thus for stopping:

```
# stop.py
import logging

logger = logging.getLogger(__name__)

def command(options):
    n = len(options.names)
    if n == 1:
        plural = ''
        services = '\'%s\'' % options.names[0]
    else:
        plural = 's'
        services = ', '.join('\'%s\'' % name for name in options
        i = services.rfind(', ')
        services = services[:i] + ' and ' + services[i + 2:]
    logger.debug('About to stop %s', services)
    # actually do the command processing here ...
    logger.info('Stopped the %s service%s.', services, plural)
```

and similarly for restarting:

```
# restart.py
import logging

logger = logging.getLogger(__name__)
```

```
def command(options):
    n = len(options.names)
    if n == 1:
        plural = ''
        services = '\'%s\'' % options.names[0]
    else:
        plural = 's'
        services = ', '.join('\'%s\'' % name for name in options
        i = services.rfind(', ')
        services = services[:i] + ' and ' + services[i + 2:]
    logger.debug('About to restart %s', services)
    # actually do the command processing here ...
    logger.info('Restarted the %s service%s.', services, plural)
```

If we run this application with the default log level, we get output like this:

```
$ python app.py start foo
INFO start Started the 'foo' service.

$ python app.py stop foo bar
INFO stop Stopped the 'foo' and 'bar' services.

$ python app.py restart foo bar baz
INFO restart Restarted the 'foo', 'bar' and 'baz' services.
```

The first word is the logging level, and the second word is the module or package name of the place where the event was logged.

If we change the logging level, then we can change the information sent to the log. For example, if we want more information:

```
$ python app.py --log-level DEBUG start foo
DEBUG start About to start foo
INFO start Started the 'foo' service.

$ python app.py --log-level DEBUG stop foo bar
DEBUG stop About to stop 'foo' and 'bar'
INFO stop Stopped the 'foo' and 'bar' services.

$ python app.py --log-level DEBUG restart foo bar baz
```

```
DEBUG restart About to restart 'foo', 'bar' and 'baz'
INFO restart Restarted the 'foo', 'bar' and 'baz' services.
```

And if we want less:

```
$ python app.py --log-level WARNING start foo
$ python app.py --log-level WARNING stop foo bar
$ python app.py --log-level WARNING restart foo bar baz
```

In this case, the commands don't print anything to the console, since nothing at `WARNING` level or above is logged by them.

A Qt GUI for logging

A question that comes up from time to time is about how to log to a GUI application. The Qt [https://www.qt.io/] framework is a popular cross-platform UI framework with Python bindings using PySide2 [https://pypi.org/project/PySide2/] or PyQt5 [https://pypi.org/project/PyQt5/] libraries.

The following example shows how to log to a Qt GUI. This introduces a simple `QtHandler` class which takes a callable, which should be a slot in the main thread that does GUI updates. A worker thread is also created to show how you can log to the GUI from both the UI itself (via a button for manual logging) as well as a worker thread doing work in the background (here, just logging messages at random levels with random short delays in between).

The worker thread is implemented using Qt's `QThread` class rather than the **`threading`** module, as there are circumstances where one has to use `QThread`, which offers better integration with other `Qt` components.

The code should work with recent releases of any of `PySide6`, `PyQt6`, `PySide2` or `PyQt5`. You should be able to adapt the approach to earlier versions of Qt. Please refer to the comments in the code snippet for more detailed information.

```
import logging
import random
import sys
```

```
import time

# Deal with minor differences between different Qt packages
try:
    from PySide6 import QtCore, QtGui, QtWidgets
    Signal = QtCore.Signal
    Slot = QtCore.Slot
except ImportError:
    try:
        from PyQt6 import QtCore, QtGui, QtWidgets
        Signal = QtCore.pyqtSignal
        Slot = QtCore.pyqtSlot
    except ImportError:
        try:
            from PySide2 import QtCore, QtGui, QtWidgets
            Signal = QtCore.Signal
            Slot = QtCore.Slot
        except ImportError:
            from PyQt5 import QtCore, QtGui, QtWidgets
            Signal = QtCore.pyqtSignal
            Slot = QtCore.pyqtSlot

logger = logging.getLogger(__name__)


#
# Signals need to be contained in a QObject or subclass in order
# initialized.
#
class Signaller(QtCore.QObject):
    signal = Signal(str, logging.LogRecord)

#
# Output to a Qt GUI is only supposed to happen on the main thre
# handler is designed to take a slot function which is set up to
# thread. In this example, the function takes a string argument
# formatted log message, and the log record which generated it.
# string is just a convenience - you could format a string for c
# you like in the slot function itself.
#
# You specify the slot function to do whatever GUI updates you w
# doesn't know or care about specific UI elements.
#
```

```
class QtHandler(logging.Handler):
    def __init__(self, slotfunc, *args, **kwargs):
        super().__init__(*args, **kwargs)
        self.signaller = Signaller()
        self.signaller.signal.connect(slotfunc)

    def emit(self, record):
        s = self.format(record)
        self.signaller.signal.emit(s, record)

#
# This example uses QThreads, which means that the threads at th
# are named something like "Dummy-1". The function below gets th
# current thread.
#
def ctname():
    return QtCore.QThread.currentThread().objectName()


#
# Used to generate random levels for logging.
#
LEVELS = (logging.DEBUG, logging.INFO, logging.WARNING, logging.
          logging.CRITICAL)

#
# This worker class represents work that is done in a thread sep
# main thread. The way the thread is kicked off to do work is vi
# that connects to a slot in the worker.
#
# Because the default threadName value in the LogRecord isn't mu
# a qThreadName which contains the QThread name as computed abov
# value in an "extra" dictionary which is used to update the Log
# QThread name.
#
# This example worker just outputs messages sequentially, inters
# random delays of the order of a few seconds.
#
class Worker(QtCore.QObject):
    @Slot()
    def start(self):
        extra = {'qThreadName': ctname() }
        logger.debug('Started work', extra=extra)
```

```
        i = 1
        # Let the thread run until interrupted. This allows reas
        # thread termination.
        while not QtCore.QThread.currentThread().isInterruptionR
            delay = 0.5 + random.random() * 2
            time.sleep(delay)
            try:
                if random.random() < 0.1:
                    raise ValueError('Exception raised: %d' % i)
                else:
                    level = random.choice(LEVELS)
                    logger.log(level, 'Message after delay of %3
            except ValueError as e:
                logger.exception('Failed: %s', e, extra=extra)
            i += 1

#
# Implement a simple UI for this cookbook example. This contains
#
# * A read-only text edit window which holds formatted log messa
# * A button to start work and log stuff in a separate thread
# * A button to log something from the main thread
# * A button to clear the log window
#
class Window(QtWidgets.QWidget):

    COLORS = {
        logging.DEBUG: 'black',
        logging.INFO: 'blue',
        logging.WARNING: 'orange',
        logging.ERROR: 'red',
        logging.CRITICAL: 'purple',
    }

    def __init__(self, app):
        super().__init__()
        self.app = app
        self.textedit = te = QtWidgets.QPlainTextEdit(self)
        # Set whatever the default monospace font is for the pla
        f = QtGui.QFont('nosuchfont')
        if hasattr(f, 'Monospace'):
            f.setStyleHint(f.Monospace)
        else:
```

```
        f.setStyleHint(f.StyleHint.Monospace)  # for Qt6
    te.setFont(f)
    te.setReadOnly(True)
    PB = QtWidgets.QPushButton
    self.work_button = PB('Start background work', self)
    self.log_button = PB('Log a message at a random level',
    self.clear_button = PB('Clear log window', self)
    self.handler = h = QtHandler(self.update_status)
    # Remember to use qThreadName rather than threadName in
    fs = '%(asctime)s %(qThreadName)-12s %(levelname)-8s %(m
    formatter = logging.Formatter(fs)
    h.setFormatter(formatter)
    logger.addHandler(h)
    # Set up to terminate the QThread when we exit
    app.aboutToQuit.connect(self.force_quit)

    # Lay out all the widgets
    layout = QtWidgets.QVBoxLayout(self)
    layout.addWidget(te)
    layout.addWidget(self.work_button)
    layout.addWidget(self.log_button)
    layout.addWidget(self.clear_button)
    self.setFixedSize(900, 400)

    # Connect the non-worker slots and signals
    self.log_button.clicked.connect(self.manual_update)
    self.clear_button.clicked.connect(self.clear_display)

    # Start a new worker thread and connect the slots for th
    self.start_thread()
    self.work_button.clicked.connect(self.worker.start)
    # Once started, the button should be disabled
    self.work_button.clicked.connect(lambda : self.work_butt

def start_thread(self):
    self.worker = Worker()
    self.worker_thread = QtCore.QThread()
    self.worker.setObjectName('Worker')
    self.worker_thread.setObjectName('WorkerThread')  # for
    self.worker.moveToThread(self.worker_thread)
    # This will start an event loop in the worker thread
    self.worker_thread.start()
```

```
    def kill_thread(self):
        # Just tell the worker to stop, then tell it to quit anc
        # to happen
        self.worker_thread.requestInterruption()
        if self.worker_thread.isRunning():
            self.worker_thread.quit()
            self.worker_thread.wait()
        else:
            print('worker has already exited.')

    def force_quit(self):
        # For use when the window is closed
        if self.worker_thread.isRunning():
            self.kill_thread()

    # The functions below update the UI and run in the main thre
    # that's where the slots are set up

    @Slot(str, logging.LogRecord)
    def update_status(self, status, record):
        color = self.COLORS.get(record.levelno, 'black')
        s = '<pre><font color="%s">%s</font></pre>' % (color, st
        self.textedit.appendHtml(s)

    @Slot()
    def manual_update(self):
        # This function uses the formatted message passed in, bu
        # information from the record to format the message in a
        # color according to its severity (level).
        level = random.choice(LEVELS)
        extra = {'qThreadName': ctname() }
        logger.log(level, 'Manually logged!', extra=extra)

    @Slot()
    def clear_display(self):
        self.textedit.clear()


def main():
    QtCore.QThread.currentThread().setObjectName('MainThread')
    logging.getLogger().setLevel(logging.DEBUG)
    app = QtWidgets.QApplication(sys.argv)
    example = Window(app)
```

```
    example.show()
    if hasattr(app, 'exec'):
        rc = app.exec()
    else:
        rc = app.exec_()
    sys.exit(rc)

if __name__=='__main__':
    main()
```

Logging to syslog with RFC5424 support

Although **RFC 5424** [https://datatracker.ietf.org/doc/html/rfc5424.html] dates from 2009, most syslog servers are configured by default to use the older **RFC 3164** [https://datatracker.ietf.org/doc/html/rfc3164.html], which hails from 2001. When `logging` was added to Python in 2003, it supported the earlier (and only existing) protocol at the time. Since RFC 5424 came out, as there has not been widespread deployment of it in syslog servers, the **`SysLogHandler`** functionality has not been updated.

RFC 5424 contains some useful features such as support for structured data, and if you need to be able to log to a syslog server with support for it, you can do so with a subclassed handler which looks something like this:

```
import datetime as dt
import logging.handlers
import re
import socket
import time

class SysLogHandler5424(logging.handlers.SysLogHandler):

    tz_offset = re.compile(r'([+-]\d{2})(\d{2})$')
    escaped = re.compile(r'([\]"\\])')

    def __init__(self, *args, **kwargs):
        self.msgid = kwargs.pop('msgid', None)
        self.appname = kwargs.pop('appname', None)
        super().__init__(*args, **kwargs)
```

```
def format(self, record):
    version = 1
    asctime = dt.datetime.fromtimestamp(record.created).isof
    m = self.tz_offset.prefixmatch(time.strftime('%z'))
    has_offset = False
    if m and time.timezone:
        hrs, mins = m.groups()
        if int(hrs) or int(mins):
            has_offset = True
    if not has_offset:
        asctime += 'Z'
    else:
        asctime += f'{hrs}:{mins}'
    try:
        hostname = socket.gethostname()
    except Exception:
        hostname = '-'
    appname = self.appname or '-'
    procid = record.process
    msgid = '-'
    msg = super().format(record)
    sdata = '-'
    if hasattr(record, 'structured_data'):
        sd = record.structured_data
        # This should be a dict where the keys are SD-ID anc
        # dict mapping PARAM-NAME to PARAM-VALUE (refer to t
        # mean)
        # There's no error checking here - it's purely for i
        # can adapt this code for use in production environm
        parts = []

        def replacer(m):
            g = m.groups()
            return '\\' + g[0]

        for sdid, dv in sd.items():
            part = f'[{sdid}'
            for k, v in dv.items():
                s = str(v)
                s = self.escaped.sub(replacer, s)
                part += f' {k}="{s}"'
            part += ']'
```

```
                parts.append(part)
            sdata = ''.join(parts)
        return f'{version} {asctime} {hostname} {appname} {proci
```

You'll need to be familiar with RFC 5424 to fully understand the above code, and it may be that you have slightly different needs (e.g. for how you pass structural data to the log). Nevertheless, the above should be adaptable to your specific needs. With the above handler, you'd pass structured data using something like this:

```
sd = {
    'foo@12345': {'bar': 'baz', 'baz': 'bozz', 'fizz': r'buzz'},
    'foo@54321': {'rab': 'baz', 'zab': 'bozz', 'zzif': r'buzz'}
}
extra = {'structured_data': sd}
i = 1
logger.debug('Message %d', i, extra=extra)
```

How to treat a logger like an output stream

Sometimes, you need to interface to a third-party API which expects a file-like object to write to, but you want to direct the API's output to a logger. You can do this using a class which wraps a logger with a file-like API. Here's a short script illustrating such a class:

```
import logging

class LoggerWriter:
    def __init__(self, logger, level):
        self.logger = logger
        self.level = level

    def write(self, message):
        if message != '\n':  # avoid printing bare newlines, if
            self.logger.log(self.level, message)

    def flush(self):
        # doesn't actually do anything, but might be expected of
        # object - so optional depending on your situation
```

```
        pass

    def close(self):
        # doesn't actually do anything, but might be expected of
        # object - so optional depending on your situation. You
        # to set a flag so that later calls to write raise an ex
        pass

def main():
    logging.basicConfig(level=logging.DEBUG)
    logger = logging.getLogger('demo')
    info_fp = LoggerWriter(logger, logging.INFO)
    debug_fp = LoggerWriter(logger, logging.DEBUG)
    print('An INFO message', file=info_fp)
    print('A DEBUG message', file=debug_fp)

if __name__ == "__main__":
    main()
```

When this script is run, it prints

```
INFO:demo:An INFO message
DEBUG:demo:A DEBUG message
```

You could also use `LoggerWriter` to redirect `sys.stdout` and `sys.stderr` by doing something like this:

```
import sys

sys.stdout = LoggerWriter(logger, logging.INFO)
sys.stderr = LoggerWriter(logger, logging.WARNING)
```

You should do this *after* configuring logging for your needs. In the above example, the **`basicConfig()`** call does this (using the `sys.stderr` value *before* it is overwritten by a `LoggerWriter` instance). Then, you'd get this kind of result:

```
>>> print('Foo')
INFO:demo:Foo
>>> print('Bar', file=sys.stderr)
```

```
WARNING:demo:Bar
>>>
```

Of course, the examples above show output according to the format used by **basicConfig()**, but you can use a different formatter when you configure logging.

Note that with the above scheme, you are somewhat at the mercy of buffering and the sequence of write calls which you are intercepting. For example, with the definition of `LoggerWriter` above, if you have the snippet

```
sys.stderr = LoggerWriter(logger, logging.WARNING)
1 / 0
```

then running the script results in

```
WARNING:demo:Traceback (most recent call last):

WARNING:demo:  File "/home/runner/cookbook-loggerwriter/test.py'

WARNING:demo:
WARNING:demo:main()
WARNING:demo:  File "/home/runner/cookbook-loggerwriter/test.py'

WARNING:demo:
WARNING:demo:1 / 0
WARNING:demo:ZeroDivisionError
WARNING:demo::
WARNING:demo:division by zero
```

As you can see, this output isn't ideal. That's because the underlying code which writes to `sys.stderr` makes multiple writes, each of which results in a separate logged line (for example, the last three lines above). To get around this problem, you need to buffer things and only output log lines when newlines are seen. Let's use a slightly better implementation of `LoggerWriter`:

```
class BufferingLoggerWriter(LoggerWriter):
    def __init__(self, logger, level):
```

```
        super().__init__(logger, level)
        self.buffer = ''

    def write(self, message):
        if '\n' not in message:
            self.buffer += message
        else:
            parts = message.split('\n')
            if self.buffer:
                s = self.buffer + parts.pop(0)
                self.logger.log(self.level, s)
            self.buffer = parts.pop()
            for part in parts:
                self.logger.log(self.level, part)
```

This just buffers up stuff until a newline is seen, and then logs complete lines. With this approach, you get better output:

```
WARNING:demo:Traceback (most recent call last):
WARNING:demo:  File "/home/runner/cookbook-loggerwriter/main.py'
WARNING:demo:    main()
WARNING:demo:  File "/home/runner/cookbook-loggerwriter/main.py'
WARNING:demo:    1/0
WARNING:demo:ZeroDivisionError: division by zero
```

How to uniformly handle newlines in logging output

Usually, messages that are logged (say to console or file) consist of a single line of text. However, sometimes there is a need to handle messages with multiple lines - whether because a logging format string contains newlines, or logged data contains newlines. If you want to handle such messages uniformly, so that each line in the logged message appears uniformly formatted as if it was logged separately, you can do this using a handler mixin, as in the following snippet:

```
# Assume this is in a module mymixins.py
import copy
```

```
class MultilineMixin:
    def emit(self, record):
        s = record.getMessage()
        if '\n' not in s:
            super().emit(record)
        else:
            lines = s.splitlines()
            rec = copy.copy(record)
            rec.args = None
            for line in lines:
                rec.msg = line
                super().emit(rec)
```

You can use the mixin as in the following script:

```
import logging

from mymixins import MultilineMixin

logger = logging.getLogger(__name__)

class StreamHandler(MultilineMixin, logging.StreamHandler):
    pass

if __name__ == '__main__':
    logging.basicConfig(level=logging.DEBUG, format='%(asctime)s
                        handlers = [StreamHandler()])
    logger.debug('Single line')
    logger.debug('Multiple lines:\nfool me once ...')
    logger.debug('Another single line')
    logger.debug('Multiple lines:\n%s', 'fool me ...\ncan\'t get
```

The script, when run, prints something like:

```
2025-07-02 13:54:47,234 DEBUG     Single line
2025-07-02 13:54:47,234 DEBUG     Multiple lines:
2025-07-02 13:54:47,234 DEBUG     fool me once ...
2025-07-02 13:54:47,234 DEBUG     Another single line
2025-07-02 13:54:47,234 DEBUG     Multiple lines:
2025-07-02 13:54:47,234 DEBUG     fool me ...
2025-07-02 13:54:47,234 DEBUG     can't get fooled again
```

If, on the other hand, you are concerned about log injection [https://owasp.org/www-community/attacks/Log_Injection], you can use a formatter which escapes newlines, as per the following example:

```
import logging

logger = logging.getLogger(__name__)

class EscapingFormatter(logging.Formatter):
    def format(self, record):
        s = super().format(record)
        return s.replace('\n', r'\n')

if __name__ == '__main__':
    h = logging.StreamHandler()
    h.setFormatter(EscapingFormatter('%(asctime)s %(levelname)-9
    logging.basicConfig(level=logging.DEBUG, handlers = [h])
    logger.debug('Single line')
    logger.debug('Multiple lines:\nfool me once ...')
    logger.debug('Another single line')
    logger.debug('Multiple lines:\n%s', 'fool me ...\ncan\'t get
```

You can, of course, use whatever escaping scheme makes the most sense for you. The script, when run, should produce output like this:

```
2025-07-09 06:47:33,783 DEBUG     Single line
2025-07-09 06:47:33,783 DEBUG     Multiple lines:\nfool me once
2025-07-09 06:47:33,783 DEBUG     Another single line
2025-07-09 06:47:33,783 DEBUG     Multiple lines:\nfool me ...\r
```

Escaping behaviour can’t be the stdlib default , as it would break backwards compatibility.

Patterns to avoid

Although the preceding sections have described ways of doing things you might need to do or deal with, it is worth mentioning some usage patterns which are *unhelpful*, and which should therefore be avoided in most cases. The following sections are in no particular order.

Opening the same log file multiple times

On Windows, you will generally not be able to open the same file multiple times as this will lead to a “file is in use by another process” error. However, on POSIX platforms you’ll not get any errors if you open the same file multiple times. This could be done accidentally, for example by:

- Adding a file handler more than once which references the same file (e.g. by a copy/paste/forget-to-change error).
- Opening two files that look different, as they have different names, but are the same because one is a symbolic link to the other.
- Forking a process, following which both parent and child have a reference to the same file. This might be through use of the `multiprocessing` module, for example.

Opening a file multiple times might *appear* to work most of the time, but can lead to a number of problems in practice:

- Logging output can be garbled because multiple threads or processes try to write to the same file. Although logging guards against concurrent use of the same handler instance by multiple threads, there is no such protection if concurrent writes are attempted by two different threads using two different handler instances which happen to point to the same file.
- An attempt to delete a file (e.g. during file rotation) silently fails, because there is another reference pointing to it. This can lead to confusion and wasted debugging time - log entries end up in unexpected places, or are lost altogether. Or a file that was supposed to be moved remains in place, and grows in size unexpectedly despite size-based rotation being supposedly in place.

Use the techniques outlined in Logging to a single file from multiple processes to circumvent such issues.

Using loggers as attributes in a class or passing them as parameters

While there might be unusual cases where you'll need to do this, in general there is no point because loggers are singletons. Code can always access a given logger instance by name using `logging.getLogger(name)`, so passing instances around and holding them as instance attributes is pointless. Note that in other languages such as Java and C#, loggers are often static class attributes. However, this pattern doesn't make sense in Python, where the module (and not the class) is the unit of software decomposition.

Adding handlers other than `NullHandler` to a logger in a library

Configuring logging by adding handlers, formatters and filters is the responsibility of the application developer, not the library developer. If you are maintaining a library, ensure that you don't add handlers to any of your loggers other than a `NullHandler` instance.

Creating a lot of loggers

Loggers are singletons that are never freed during a script execution, and so creating lots of loggers will use up memory which can't then be freed. Rather than create a logger per e.g. file processed or network connection made, use the existing mechanisms for passing contextual information into your logs and restrict the loggers created to those describing areas within your application (generally modules, but occasionally slightly more fine-grained than that).

Outros recursos

Ver também

Módulo `logging`
Referência da API para o módulo de logging.

Módulo `logging.config`
API de configuração para o módulo logging.

Módulo `logging.handlers`

Manipuladores úteis incluídos no módulo logging.

Tutorial básico

Tutorial avançado

Regular expression HOWTO

Resumo

Este documento é um tutorial introdutório sobre expressões regulares em Python com o módulo `re`. Ele provê uma introdução mais tranquila que a seção correspondente à documentação do módulo.

Introdução

Expressões regulares (chamadas REs, ou regexes ou padrões regex) são essencialmente uma mini linguagem de programação altamente especializada incluída dentro do Python e disponível através do módulo `re`. Usando esta pequena linguagem, você especifica as regras para o conjunto de strings possíveis que você quer corresponder; esse conjunto pode conter sentenças em português, endereços de e-mail, comandos TeX ou qualquer coisa que você queira. Você poderá então perguntar coisas como “Essa string se enquadra dentro do padrão?” ou “Existe alguma parte da string que se enquadra nesse padrão?”. Você também pode usar as REs para modificar uma string ou dividi-la de diversas formas.

Os padrões das expressões regulares são compilados em uma série de bytecodes que são então executadas por um mecanismo de correspondência escrito em C. Para usos avançados, talvez seja necessário prestar atenção em como o mecanismo irá executar uma dada RE, e escrever a RE de forma que os bytecodes executem de forma mais rápida. Otimização é um tema que não será visto neste documento, porque ele requer que você tenha um bom entendimento dos mecanismos de combinação internos.

A linguagem de expressão regular é relativamente pequena e restrita, por isso nem todas as tarefas de processamento de strings possíveis podem ser feitas usando expressões regulares. Existem também tarefas que podem ser feitas com expressões regulares, mas as expressões acabam por se tornarem

muito complicadas. Nestes casos, pode ser melhor para você escrever um código Python para fazer o processamento; embora um código Python seja mais lento do que uma expressão regular elaborada, ele provavelmente será mais compreensível.

Simple patterns

Vamos começar por aprender sobre as expressões regulares mais simples possíveis. Como as expressões regulares são usadas para operar em strings, vamos começar com a tarefa mais comum: correspondência de caracteres.

Para uma explicação detalhada da ciência da computação referente a expressões regulares (autômatos finitos determinísticos e não determinístico), você pode consultara praticamente qualquer livro sobre a escrita de compiladores.

Matching characters

A maioria das letras e caracteres simplesmente irão corresponder entre si. Por exemplo, a expressão regular `teste` irá combinar totalmente com a string `teste`. (Você pode habilitar o modo de maiúsculas e minúsculas que faria a RE corresponder com `Test` ou `TEST` também; veremos mais sobre isso mais adiante.)

Há exceções a essa regra, alguns caracteres são metacaracteres especiais, e não se correspondem. Em vez disso, eles sinalizam que alguma coisa fora do normal deve ser correspondida, ou eles afetam outras partes da RE, repetindo-as ou alterando seus significados. Grande parte deste documento é dedicada à discussão de vários metacaracteres e o que eles fazem.

Aqui está a lista completa de metacaracteres; seus significados serão discutidos ao longo deste documento.

```
. ^ $ * + ? { } [ ] \ | ( )
```

O primeiro metacaractere que vamos estudar é o `[` e o `]`. Eles são usados para especificar uma classe de caracteres, que é um conjunto de caracteres

que você deseja combinar. Caracteres podem ser listados individualmente ou um intervalo de caracteres pode ser indicado fornecendo dois caracteres e separando-os por um `'-'`. Por exemplo, `[abc]` irá encontrar qualquer caractere `a`, `b`, ou `c`; isso é o mesmo que escrever `[a-c]`, que usa um intervalo para expressar o mesmo conjunto de caracteres. Se você deseja encontrar apenas letras minúsculas, sua RE seria `[a-z]`.

Metacaracteres (exceto `\`) não são ativos dentro de classes. Por exemplo, `[akm$]` irá corresponder com qualquer dos caracteres `'a'`, `'k'`, `'m'` ou `'$'`; `'$'` é normalmente um metacaractere, mas dentro de uma classe de caracteres ele perde sua natureza especial.

Você pode combinar os caracteres não listados na classe *complementando* o conjunto. Isso é indicado pela inclusão de um `'^'` como o primeiro caractere da classe. Por exemplo, `[^5]` combinará a qualquer caractere, exceto `'5'`. Se o sinal de intercalação aparecer em outro lugar da classe de caracteres, ele não terá um significado especial. Por exemplo: `[5^]` corresponderá a um `'5'` ou a `'^'`.

Talvez o metacaractere mais importante seja a contrabarra, `\`. Como as strings literais em Python, a barra invertida pode ser seguida por vários caracteres para sinalizar várias sequências especiais. Ela também é usada para *escapar* todos os metacaracteres, e assim, você poder combiná-los em padrões; por exemplo, se você precisa fazer correspondência a um `[` ou `\`, você pode precedê-los com uma barra invertida para remover seu significado especial: `\[` ou `\\`.

Algumas das sequências especiais que começam com `'\'` representam conjuntos de caracteres predefinidos que são frequentemente úteis, como o conjunto de dígitos, o conjunto de letras ou o conjunto de qualquer coisa que não seja espaço em branco.

Vejamos um exemplo: `\w` corresponde a qualquer caractere alfanumérico. Se o padrão regex for expresso em bytes, isso é equivalente à classe `[a-zA-Z0-9_]`. Se o padrão regex for uma string, `\w` combinará todos os caracteres marcados como letras no banco de dados Unicode fornecido pelo módulo `unicodedata`. Você pode usar a definição mais restrita de `\w` em um padrão

de string, fornecendo o sinalizador **`re.ASCII`** ao compilar a expressão regular.

A lista a seguir de sequências especiais não está completa. Para obter uma lista completa das sequências e definições de classe expandidas para padrões de strings Unicode, veja a última parte de Sintaxe de Expressão Regular na referência da Biblioteca Padrão. Em geral, as versões Unicode correspondem a qualquer caractere que esteja na categoria apropriada do banco de dados Unicode.

`\d`
: corresponde a qualquer `dígito decimal`, que é equivalente à classe `[0-9]`.

`\D`
: corresponde a qualquer caractere `não-dígito`, o que é equivalente à classe `[^0-9]`.

`\s`
: corresponde a qualquer caractere `espaço-em-branco`, o que é equivalente à classe `[\t\n\r\f\v]`.

`\S`
: corresponde a qualquer caractere `não-espaço-branco`, o que é equivalente à classe `[^\t\n\r\f\v]`.

`\w`
: corresponde a qualquer caractere `alfanumérico`, o que é equivalente à classe `[azA-Z0-9_]`.

`\W`
: corresponde a qualquer caractere `não-alfanumérico`, o que é equivalente à classe `[^a-zA-Z0-9_]`.

Estas sequências podem ser incluídas dentro de uma classe de caractere. Por exemplo, `[\s,.]` É uma classe de caractere que irá corresponder a qualquer caractere `espaço-em-branco`, ou `,` ou `.`.

O metacaractere final desta seção é o `.`. Ele encontra tudo, exceto um caractere de nova linha, e existe um modo alternativo (`re.DOTALL`), onde ele irá corresponder até mesmo a um caractere de nova linha. `.` é frequentemente usado quando você quer corresponder com “qualquer caractere”.

Repeating things

Ser capaz de corresponder com variados conjuntos de caracteres é a primeira coisa que as expressões regulares podem fazer que ainda não é possível com os métodos disponíveis para strings. No entanto, se essa fosse a única capacidade adicional das expressões regulares, elas não seriam um avanço relevante. Outro recurso que você pode especificar é que partes do RE devem ser repetidas um certo número de vezes.

O primeiro metacaractere para repetir coisas que veremos é `*`. `*` não corresponde ao caractere literal `'*'`; em vez disso, ele especifica que o caractere anterior pode ser correspondido zero ou mais vezes, em vez de exatamente uma vez.

Por exemplo, `ca*t` vai corresponder `'ct'` (0 caracteres `'a'`), `'cat'` (1 `'a'`), `'caaat'` (3 caracteres `'a'`), e assim por diante.

Repetições tais como `*` são gulosas; ao repetir a RE, o motor de correspondência vai tentar repeti-la tantas vezes quanto possível. Se porções posteriores do padrão não corresponderem, o motor de correspondência, em seguida, volta e tenta novamente com menos repetições.

Um exemplo passo a passo fará isso mais óbvio. Vamos considerar a expressão “a[bcd]*b”. Isto corresponde à letra “a”, zero ou mais letras da classe “[bcd]” e, finalmente, termina com “b”. Agora, imagine corresponder este RE com a string “abcbd”.

Passo	Correspondência	Explicação

Passo	Correspondência	Explicação
1	`a`	O caractere `a` na RE tem correspondência.
2	`abcbd`	O motor corresponde com `[bcd]*`, indo tão longe quanto possível, que é o fim do string.
3	*Falha*	O motor tenta corresponder com `b`, mas a posição corrente está no final da string, então ele falha.
4	`abcb`	Voltando, de modo que `[bcd]*` corresponde a um caractere a menos.
5	*Falha*	Tenta `b` novamente, mas a posição corrente é a do último caractere, que é um `'d'`.
6	`abc`	Voltando novamente, de modo que `[bcd]*` está correspondendo com `bc` somente.
7	`abcb`	Tenta `b` novamente. Desta vez, o caractere na posição corrente é `'b'`, por isso sucesso.

O final da RE foi atingido e correspondeu a `'abcb'`. Isso demonstra como o mecanismo de correspondência vai tão longe quanto pode no início e, se nenhuma correspondência for encontrada, ele fará a volta progressivamente e tentará novamente o restante da RE. Ele fará voltas até que tenha tentado zero correspondências para `[bcd]*`, e se isso falhar subsequentemente, o mecanismo concluirá que a string não corresponde a RE de forma alguma.

Outro metacaractere de repetição é o `+`, que corresponde a uma ou mais vezes. Preste muita atenção para a diferença entre `*` e `+`; `*` corresponde com zero ou mais vezes, assim, o que quer que esteja sendo repetido pode não estar presente, enquanto que `+` requer pelo menos uma ocorrência. Para usar um exemplo similar, `ca+t` vai corresponder a `'cat'`, (1 `'a'`), `'caaat'` (3 `'a's`), mas não corresponde com `'ct'`.

Existem mais dois quantificadores ou operadores de repetição. O caractere de ponto de interrogação, `?`, corresponde a uma ou zero vez; você pode pensar nisso como a marcação de algo sendo opcional. Por exemplo, `home-?brew` corresponde tanto a `'homebrew'` quanto a `'home-brew'`.

O qualificador mais complicado é o `{m,n}`, no qual *m* e *n* são números inteiros decimais. Este qualificador significa que deve haver pelo menos *m* repetições, e no máximo *n*. Por exemplo, `a/{1,3}b` irá corresponder a `'a/b'`, `'a//b'` e `'a///b'`. Não vai corresponder a `'ab'`, que não tem barras ou a `'a////b'`, que tem quatro.

Você pode omitir tanto *m* quanto *n*; nesse caso, um valor razoável é presumido para o valor em falta. A omissão de *m* é interpretada como o limite inferior de 0, enquanto a omissão de *n* resulta como limite superior o infinito.

O caso mais simples `{m}` corresponde ao item precedente exatamente *m* vezes. Por exemplo, `a/{2}b` corresponderá somente a `'a//b'`.

Os leitores de uma inclinação reducionista podem notar que os três outros quantificadores podem todos serem expressos utilizando esta notação. `{0,}` é o mesmo que `*`, `{1,}` é equivalente a `+`, e `{0,1}` é o mesmo que `?`. É melhor usar `*`, `+` ou `?` quando puder, simplesmente porque eles são mais curtos e fáceis de ler.

Using regular expressions

Agora que nós vimos algumas expressões regulares simples, como nós realmente as usamos em Python? O módulo `re` fornece uma interface para o

mecanismo de expressão regular, permitindo compilar REs em objetos e, em seguida, executar comparações com eles.

Compiling regular expressions

As expressões regulares são compiladas em objetos padrão, que têm métodos para várias operações, tais como a procura por padrões de correspondência ou realizar substituições de strings.

```
>>> import re
>>> p = re.compile('ab*')
>>> p
re.compile('ab*')
```

`re.compile()` também aceita um argumento opcional *flags*, utilizado para habilitar vários recursos especiais e variações de sintaxe. Nós vamos ver todas as configurações disponíveis mais tarde, mas por agora, um único exemplo vai servir:

```
>>> p = re.compile('ab*', re.IGNORECASE)
```

A RE é passada para `re.compile()` como uma string. REs são tratadas como strings porque as expressões regulares não são parte do núcleo da linguagem Python, e nenhuma sintaxe especial foi criada para expressá-las. (Existem aplicações que não necessitam de REs nenhuma, por isso não há necessidade de inchar a especificação da linguagem, incluindo-as.) Em vez disso, o módulo `re` é simplesmente um módulo de extensão C incluído no Python, assim como os módulos de `socket` ou `zlib`.

Colocando REs em strings mantém a linguagem Python mais simples, mas tem uma desvantagem, que é o tema da próxima seção.

The backslash plague

Como afirmado anteriormente, expressões regulares usam o caractere de contrabarra (\) para indicar formas especiais ou para permitir que caracteres especiais sejam usados sem invocar o seu significado especial. Isso entra

em conflito com o uso pelo Python do mesmo caractere para o mesmo propósito nas strings literais.

Vamos dizer que você quer escrever uma RE que corresponde com a string `\section`, que pode ser encontrada em um arquivo LaTeX. Para descobrir o que escrever no código do programa, comece com a string que se deseja corresponder. Em seguida, você deve preceder qualquer contrabarra e outros metacaracteres com uma contrabarra, tendo como resultado a string `\\section`. A string resultante que deve ser passada para **`re.compile()`** deve ser `\\section`. No entanto, para expressar isso como uma string literal Python, ambas as contrabarras devem ser precedidas com uma contrabarra novamente.

Caracteres	Etapa
`\section`	String de texto a ser correspondida
`\\section`	Preceder com contrabarra para **`re.compile()`**
`"\\\\section"`	Contrabarras precedidas novamente para uma string literal

Em suma, para corresponder com uma contrabarra literal, tem de se escrever `'\\\\'` como a string da RE, porque a expressão regular deve ser `\\`, e cada contrabarra deve ser expressa como `\\` dentro de uma string literal Python normal. Em REs que apresentam contrabarras repetidas vezes, isso leva a um monte de contrabarras repetidas e faz as strings resultantes difíceis de entender.

A solução é usar a notação de string bruta (raw) do Python para expressões regulares; contrabarras não são tratadas de nenhuma forma especial em uma string literal se prefixada com `r`, então `r"\n"` é uma string de dois caracteres contendo `\` e `n`, enquanto `"\n"` é uma string de um único

caractere contendo uma nova linha. As expressões regulares, muitas vezes, são escritas no código Python usando esta notação de string bruta (raw).

In addition, special escape sequences that are valid in regular expressions, but not valid as Python string literals, now result in a `SyntaxWarning` and will eventually become a `SyntaxError`, which means the sequences will be invalid if raw string notation or escaping the backslashes isn't used.

String regular	String bruta
`"ab*"`	`r"ab*"`
`"\\\\section"`	`r"\\section"`
`"\\w+\\s+\\1"`	`r"\w+\s+\1"`

Performing matches

Uma vez que você tem um objeto que representa uma expressão regular compilada, o que você faz com ele? Objetos Pattern têm vários métodos e atributos. Apenas os mais significativos serão vistos aqui; consulte a documentação do módulo `re` para uma lista completa.

Método/Atributo	Propósito
`search()`	Varre toda a string, procurando qualquer local onde esta RE tem correspondência.

Método/Atributo	Propósito
`prefixmatch()`	Determine if the RE matches at the beginning of the string. Previously named match().
`findall()`	Find all substrings where the RE matches, and return them as a list.
`finditer()`	Find all substrings where the RE matches, and return them as an iterator.

`search()` and **`prefixmatch()`** return `None` if no match can be found. If they're successful, a match object instance is returned, containing information about the match: where it starts and ends, the substring it matched, and more.

Você pode aprender mais sobre isso experimentando interativamente com o módulo **`re`**.

Este documento usa o interpretador Python padrão para seus exemplos. Primeiro, execute o interpretador Python, importe o módulo **`re`**, e compile uma RE:

```
>>> import re
>>> p = re.compile('[a-z]+')
>>> p
re.compile('[a-z]+')
```

Now, you can try matching various strings against the RE `[a-z]+`. An empty string shouldn't match at all, since `+` means 'one or more repetitions'. **`search()`** should return `None` in this case, which will cause the interpreter to print no output. You can explicitly print the result of **`search()`** to make this clear.

```
>>> p.search("")
>>> print(p.search(""))
None
```

Now, let's try it on a string that it should match, such as `tempo`. In this case, **`search()`** will return a match object, so you should store the result in a variable for later use.

```
>>> m = p.search('tempo')
>>> m
<re.Match object; span=(0, 5), match='tempo'>
```

Agora você pode consultar o objeto correspondência para obter informações sobre a string correspondente. Instâncias do objeto correspondência também tem vários métodos e atributos; os mais importantes são os seguintes:

Método/Atributo	Propósito
`group()`	Retorna a string que corresponde com a RE
`start()`	Retorna a posição inicial da string correspondente
`end()`	Retorna a posição final da string correspondente
`span()`	Retorna uma tupla contendo as posições (inicial, final) da string correspondente

Experimentando estes métodos teremos seus significado esclarecidos:

```
>>> m.group()
'tempo'
>>> m.start(), m.end()
(0, 5)
```

```
>>> m.span()
(0, 5)
```

group() returns the substring that was matched by the RE. **start()** and **end()** return the starting and ending index of the match. **span()** returns both start and end indexes in a single tuple. The **search()** method of patterns scans through the string, so the match may not start at zero. However, the **prefixmatch()** method only checks if the RE matches at the start of a string, so **start()** will always be zero in that case.

```
>>> m = p.search(':::: message'); print(m)
<re.Match object; span=(4, 11), match='message'>
>>> m.group()
'message'
>>> m.span()
(4, 11)
>>> print(p.prefixmatch(':::: message'))
None
```

Nos programas reais, o estilo mais comum é armazenar o objeto correspondência em uma variável e, em seguida, verificar se ela é `None`. Isso geralmente se parece com:

```
p = re.compile( ... )
m = p.search( 'string goes here' )
if m:
    print('Match found: ', m.group())
else:
    print('No match')
```

Dois métodos padrão retornam todas as correspondências de um padrão. **findall()** retorna uma lista de strings correspondentes:

```
>>> p = re.compile(r'\d+')
>>> p.findall('12 percussionistas tocando tambores, 11 flautista
['12', '11', '10']
```

The `r` prefix, making the literal a raw string literal, is needed in this example because escape sequences in a normal “cooked” string literal that are not recognized by Python, as opposed to regular expressions, now result

in a `SyntaxWarning` and will eventually become a `SyntaxError`. See The backslash plague.

`findall()` tem que criar a lista inteira antes de poder devolvê-la como resultado. O método `finditer()` retorna uma sequência de instâncias objeto correspondência como um iterator:

```
>>> iterator = p.finditer('12 drummers drumming, 11 ... 10 ...')
>>> iterator
<callable_iterator object at 0x...>
>>> for match in iterator:
...     print(match.span())
...
(0, 2)
(22, 24)
(29, 31)
```

Module-level functions

You don't have to create a pattern object and call its methods; the `re` module also provides top-level functions called `search()`, `prefixmatch()`, `findall()`, `sub()`, and so forth. These functions take the same arguments as the corresponding pattern method with the RE string added as the first argument, and still return either `None` or a match object instance.

```
>>> print(re.prefixmatch(r'From\s+', 'Fromage amk'))
None
>>> re.prefixmatch(r'From\s+', 'From amk Thu May 14 19:12:10 199
<re.Match object; span=(0, 5), match='From '>
```

Sob o capô, estas funções simplesmente criam um objeto padrão para você e chamam o método apropriado para ele. Elas também armazenam o objeto compilado em um cache, para que futuras chamadas usando a mesma RE não precisem analisar o padrão de novo e de novo.

Você deve usar essas funções de nível de módulo ou deve obter o padrão e chamar seus métodos você mesmo? Se estiver acessando uma expressão regular dentro de um laço de repetição, pré-compilá-la economizará

algumas chamadas de função. Fora dos laços, não há muita diferença graças ao cache interno.

Compilation flags

Sinalizadores de compilação permitem modificar alguns aspectos de como as expressões regulares funcionam. Sinalizadores estão disponíveis no módulo `re` sob dois nomes, um nome longo, tal como `IGNORECASE` e um curto, na forma de uma letra, como `I`. (Se você estiver familiarizado com o padrão dos modificadores do Perl, o nome curto usa as mesmas letras; o forma abreviada de `re.VERBOSE` é `re.X`, por exemplo). Vários sinalizadores podem ser especificados aplicando um `OU` bit a bit nelas; `re.I | re.M` define os sinalizadores `I` e `M`, por exemplo.

Aqui está uma tabela dos sinalizadores disponíveis, seguida por uma explicação mais detalhada de cada um:

Sinalizador	Significado
`ASCII`, `A`	Faz com que vários escapes como `\w`, `\b`, `\s` e `\d` correspondam apenas a caracteres ASCII com a respectiva propriedade.
`DOTALL`, `S`	Faz o `.` corresponder a qualquer caractere, incluindo novas linhas.
`IGNORECASE`, `I`	Faz combinações sem diferenciar maiúsculo de minúsculo.
`LOCALE`, `L`	Faz uma correspondência considerando a localidade.

Sinalizador	Significado
`MULTILINE`, `M`	Correspondência multilinha, afetando `^` e `$`.
`VERBOSE`, `X` (de ‘extended’; estendido em inglês)	Habilita REs detalhadas, que podem ser organizadas de forma mais clara e compreensível.

re.I
re.IGNORECASE

Faz correspondência sem distinção entre maiúsculas e minúsculas; a classe de caracteres e as strings literais corresponderão às letras ignorando maiúsculas e minúsculas. Por exemplo, `[A-Z]` corresponderá às letras minúsculas também. A correspondência Unicode completa também funciona, a menos que o sinalizador `ASCII` seja usado para desabilitar correspondências não ASCII. Quando os padrões Unicode `[a-z]` ou `[A-Z]` são usados em combinação com o sinalizador `IGNORECASE`, eles corresponderão às 52 letras ASCII e 4 letras não ASCII adicionais: ‘İ’ (U+0130, letra maiúscula latina I com ponto acima), ‘ı’ (U+0131, letra minúscula latina i sem ponto), ‘ſ’ (U+017F, letra minúscula latina s longo) e ‘K’ (U+212A, sinal Kelvin). `Spam` corresponderá a `'Spam'`, `'spam'`, `'spAM'` ou `'ſpam'` (o último é correspondido apenas no modo Unicode). Essa capitalização em minúsculas não leva em conta a localidade atual; levará se você também definir o sinalizador `LOCALE`.

re.L
re.LOCALE

Faz com que `\w`, `\W`, `\b`, `\B` e a correspondência sem diferenciação de maiúsculas e minúsculas dependam da localidade atual em vez do banco de dados Unicode.

Localidades são um recurso da biblioteca C destinado a ajudar na escrita de programas que levam em conta as diferenças de idioma. Por exemplo, se você estiver processando texto codificado em francês, você gostaria de ser capaz de escrever `\w+` para corresponder a palavras, mas `\w` corresponde apenas à classe de caracteres `[A-Za-z]` em padrões de bytes; ele não corresponderá a bytes correspondentes a `é` ou `ç`. Se seu sistema estiver configurado corretamente e uma localidade francesa for selecionada, certas funções C dirão ao programa que o byte correspondente a `é` também deve ser considerado uma letra. Definir o sinalizador **`LOCALE`** ao compilar uma expressão regular fará com que o objeto compilado resultante use essas funções C para `\w`; isso é mais lento, mas também permite que `\w+` corresponda a palavras francesas como você esperaria. O uso deste sinalizador é desencorajado no Python 3, pois o mecanismo de localidade é muito pouco confiável, ele só manipula uma "cultura" por vez e só funciona com localidades de 8 bits. A correspondência Unicode já está habilitada por padrão no Python 3 para padrões Unicode (str), e é capaz de manipular diferentes localidades/idiomas.

re.M
re.MULTILINE

(`^` e `$` ainda não foram explicados, eles serão comentados na seção More metacharacters.)

Normalmente `^` corresponde apenas ao início da string e `$` corresponde apenas ao final da string, e imediatamente antes da nova linha (se existir) no final da string. Quando este sinalizador é especificada, o `^` corresponde ao início da string e ao início de cada linha dentro da string, imediatamente após cada nova linha. Da mesma forma, o metacaractere `$` corresponde tanto ao final da string e ao final de cada linha (imediatamente antes de cada nova linha).

re.S
re.DOTALL

Faz o caractere especial `.` corresponder com qualquer caractere que seja, incluindo uma nova linha; sem este sinalizador, `.` irá corresponder a qualquer coisa, exceto uma nova linha.

re.A
re.ASCII

Faz com que `\w`, `\W`, `\b`, `\B`, `\s` e `\S` executem a correspondência somente ASCII em vez da correspondência Unicode completa. Isso é significativo apenas para padrões Unicode e é ignorado para padrões de bytes.

re.X
re.VERBOSE

This flag allows you to write regular expressions that are more readable by granting you more flexibility in how you can format them. When this flag has been specified, whitespace within the RE string is ignored, except when the whitespace is in a character class or preceded by an unescaped backslash; this lets you organize and indent the RE more clearly. This flag also lets you put comments within a RE that will be ignored by the engine; comments are marked by a `'#'` that's neither in a character class nor preceded by an unescaped backslash.

Por exemplo, aqui está uma RE que usa `re.VERBOSE`; consegue ver o quanto mais fácil de ler é?

```
charref = re.compile(r"""
 &[#]                # Início de uma referência à entidade nu
 (
     0[0-7]+         # Forma octal
   | [0-9]+          # Forma decimal
   | x[0-9a-fA-F]+   # Forma hexadecimal
 )
 ;                   # Ponto e vírgula ao final
""", re.VERBOSE)
```

Sem o "verbose" definido, A RE iria se parecer como isto:

```
charref = re.compile("&#(0[0-7]+"
                     "|[0-9]+"
                     "|x[0-9a-fA-F]+);")
```

No exemplo acima, a concatenação automática de strings literais em Python foi usada para quebrar a RE em partes menores, mas ainda é

mais difícil de entender do que a versão que usa `re.VERBOSE`.

More pattern power

Até agora, cobrimos apenas uma parte dos recursos das expressões regulares. Nesta seção, vamos abordar alguns metacaracteres novos, e como usar grupos para recuperar partes do texto que teve correspondência.

More metacharacters

Existem alguns metacaracteres que nós ainda não vimos. A maioria deles serão referenciados nesta seção.

Alguns dos metacaracteres restantes a serem discutidos são como uma asserção de `largura zero` (zero-width assertions). Eles não fazem com que o mecanismo avance pela string; ao contrário, eles não consomem nenhum caractere, e simplesmente tem sucesso ou falha. Por exemplo, `\b` é uma afirmação de que a posição atual está localizada nas bordas de uma palavra; a posição não é alterada de nenhuma maneira por `\b`. Isto significa que afirmações de `largura zero` nunca devem ser repetidas, porque se elas combinam uma vez em um determinado local, elas podem, obviamente, combinar um número infinito de vezes.

`|`

Alternância, ou operador “or”. Se *A* e *B* são expressões regulares, `A|B` irá corresponder com qualquer string que corresponder com *A* ou *B*. `|` tem uma prioridade muito baixa, a fim de fazê-lo funcionar razoavelmente quando você está alternando entre strings de vários caracteres. `Crow|Servo` irá corresponder tanto com `'Crow'` quanto com `'Servo'`, e não com `'Cro'`, `'w'` ou `'S'`, e `'ervo'`.

Para corresponder com um `'|'` literal, use `\|`, ou coloque ele dentro de uma classe de caracteres, como em `[|]`.

`^`

Corresponde ao início de linha. A menos que o sinalizador `MULTILINE` tenha sido definido, isso só irá corresponder ao início da string. No

modo **MULTILINE**, isso também corresponde imediatamente após cada nova linha de dentro da string.

Por exemplo, para ter correspondência com a palavra `From` apenas no início de uma linha, a RE a ser usada é `^From`.

```
>>> print(re.search('^From', 'From Here to Eternity'))
<re.Match object; span=(0, 4), match='From'>
>>> print(re.search('^From', 'Reciting From Memory'))
None
```

Para corresponder a um `'^'` literal, use `\^`.

`$`

Corresponde ao fim de uma linha, que tanto é definido como o fim de uma string, ou qualquer local seguido por um caractere de nova linha.

```
>>> print(re.search('}$', '{block}'))
<re.Match object; span=(6, 7), match='}'>
>>> print(re.search('}$', '{block} '))
None
>>> print(re.search('}$', '{block}\n'))
<re.Match object; span=(6, 7), match='}'>
```

Para corresponder com um `'$'` literal, use `\$` ou coloque-o dentro de uma classe de caracteres, como em `[$]`.

`\A`

Corresponde apenas com o início da string. Quando não estiver em modo **MULTILINE**, `\A` e `^` são efetivamente a mesma coisa. No modo **MULTILINE**, eles são diferentes: `\A` continua a corresponder apenas com o início da string, mas `^` pode corresponder com qualquer localização de dentro da string, que seja posterior a um caractere nova linha.

`\z`

Corresponde apenas ao final da string.

`\Z`

O mesmo que `\z`. Para compatibilidade com versões mais antigas de Python.

`\b`

Borda de palavra. Esta é uma asserção de largura zero que corresponde apenas ao início ou ao fim de uma palavra. Uma palavra é definida como uma sequência de caracteres alfanuméricos, então o fim de uma palavra é indicado por espaço em branco ou um caractere não alfanumérico.

O exemplo a seguir corresponde a `class` apenas quando é a palavra exata; ele não irá corresponder quando for contido dentro de uma outra palavra.

```
>>> p = re.compile(r'\bclass\b')
>>> print(p.search('no class at all'))
<re.Match object; span=(3, 8), match='class'>
>>> print(p.search('the declassified algorithm'))
None
>>> print(p.search('one subclass is'))
None
```

Há duas sutilezas que você deve lembrar ao usar essa sequência especial. Em primeiro lugar, esta é a pior colisão entre strings literais do Python e sequências de expressão regular. Nas strings literais do Python, `\b` é o caractere backspace, o valor ASCII 8. Se você não estiver usando strings brutas, então Python irá converter o `\b` em um backspace e sua RE não irá funcionar da maneira que você espera. O exemplo a seguir parece igual a nossa RE anterior, mas omite o `'r'` na frente da string RE.

```
>>> p = re.compile('\bclass\b')
>>> print(p.search('no class at all'))
None
>>> print(p.search('\b' + 'class' + '\b'))
<re.Match object; span=(0, 7), match='\x08class\x08'>
```

Além disso, dentro de uma classe de caracteres, onde não há nenhum uso para esta asserção, `\b` representa o caractere backspace, para compatibilidade com strings literais do Python

`\B`

Outra asserção de largura zero; isto é o oposto de `\b`, correspondendo apenas quando a posição corrente não é de uma borda de palavra.

Agrupamento

Frequentemente é necessário obter mais informações do que apenas se a RE teve correspondência ou não. As expressões regulares são muitas vezes utilizadas para dissecar strings escrevendo uma RE dividida em vários subgrupos que correspondem a diferentes componentes de interesse. Por exemplo, uma linha de cabeçalho RFC-822 é dividida em um nome de cabeçalho e um valor, separados por um `':'`, como essa:

```
From: author@example.com
User-Agent: Thunderbird 1.5.0.9 (X11/20061227)
MIME-Version: 1.0
To: editor@example.com
```

Isto pode ser tratado escrevendo uma expressão regular que corresponde com uma linha inteira de cabeçalho, e tem um grupo que corresponde ao nome do cabeçalho, e um outro grupo, que corresponde ao valor do cabeçalho.

Os grupos são marcados pelos metacaracteres `'('` e `')'`. `'('` e `')'` têm muito do mesmo significado que eles têm em expressões matemáticas; eles agrupam as expressões contidas dentro deles, e você pode repetir o conteúdo de um grupo com um qualificador de repetição, como `*`, `+`, `?`, ou `{m,n}`. Por exemplo, `(ab)*` irá corresponder a zero ou mais repetições de `ab`.

```
>>> p = re.compile('(ab)*')
>>> print(p.search('abababababab').span())
(0, 10)
```

Grupos indicados com `'('` e `')'` também capturam o índice inicial e final do texto que eles correspondem; isso pode ser obtido por meio da passagem de um argumento para **`group()`**, **`start()`**, **`end()`** e **`span()`**. Os grupos são numerados começando com 0. O grupo 0 está sempre presente; é toda a RE,

de forma que os métodos de objeto de correspondência têm todos o grupo 0 como seu argumento padrão. Mais tarde veremos como expressar grupos que não capturam a extensão de texto com a qual eles correspondem.

```
>>> p = re.compile('(a)b')
>>> m = p.search('ab')
>>> m.group()
'ab'
>>> m.group(0)
'ab'
```

Subgrupos são numerados a partir da esquerda para a direita, de forma crescente a partir de 1. Os grupos podem ser aninhados; para determinar o número, basta contar os caracteres de abertura de parêntese `(`, indo da esquerda para a direita.

```
>>> p = re.compile('(a(b)c)d')
>>> m = p.search('abcd')
>>> m.group(0)
'abcd'
>>> m.group(1)
'abc'
>>> m.group(2)
'b'
```

`group()` pode receber vários números de grupos de uma vez, e nesse caso ele irá retornar uma tupla contendo os valores correspondentes desses grupos.

```
>>> m.group(2,1,2)
('b', 'abc', 'b')
```

O método `groups()` retorna uma tupla contendo as strings de todos os subgrupos, de 1 até o último.

```
>>> m.groups()
('abc', 'b')
```

Referências anteriores em um padrão permitem que você especifique que o conteúdo de um grupo capturado anteriormente também deve ser

encontrado na posição atual na sequência. Por exemplo, `\1` terá sucesso se o conteúdo exato do grupo 1 puder ser encontrado na posição atual, e falha caso contrário. Lembre-se que as strings literais do Python também usam a contrabarra seguida por números para permitir a inclusão de caracteres arbitrários em uma string, por isso certifique-se de usar strings brutas ao incorporar referências anteriores em uma RE.

Por exemplo, a seguinte RE detecta palavras duplicadas em uma string.

```
>>> p = re.compile(r'\b(\w+)\s+\1\b')
>>> p.search('Paris in the the spring').group()
'the the'
```

Referências anteriores como esta não são, geralmente, muito úteis apenas para fazer pesquisa em uma string — existem alguns formatos de texto que repetem dados dessa forma — mas em breve você irá descobrir que elas são muito úteis para realizar substituições de strings.

Non-capturing and named groups

REs elaboradas podem usar muitos grupos, tanto para capturar substrings de interesse, quanto para agrupar e estruturar a própria RE. Em REs complexas, torna-se difícil manter o controle dos números dos grupos. Existem dois recursos que ajudam a lidar com esse problema. Ambos usam uma sintaxe comum para extensões de expressão regular, então vamos olhar primeiro para isso.

O Perl 5 é bem conhecido por suas poderosas adições às expressões regulares padrão. Para esses novos recursos, os desenvolvedores do Perl não podiam escolher novos metacaracteres de um único caractere ou novas sequências especiais começando com `\` sem tornar as expressões regulares do Perl confusamente diferentes das REs padrão. Se eles escolhessem `&` como um novo metacaractere, por exemplo, as expressões antigas estariam presumindo que `&` era um caractere regular e não teriam escapado dele escrevendo `\&` ou `[&]`.

A solução escolhida pelos desenvolvedores do Perl foi usar `(?...)` como uma sintaxe de extensão. Um `?` imediatamente após um parêntese era um

erro de sintaxe porque o `?` não teria nada a repetir, de modo que isso não introduz quaisquer problemas de compatibilidade. Os caracteres imediatamente após um `?` indicam que a extensão está sendo usada, então `(?=foo)` é uma coisa (uma asserção `lookahead` positiva) e `(?:foo)` é outra coisa (um grupo de não-captura contendo a subexpressão `foo`).

Python oferece suporte a diversas extensões do Perl e adiciona uma sintaxe de extensão à sintaxe de extensão do Perl. Se o primeiro caractere após o ponto de interrogação for um `P`, você sabe que se trata de uma extensão específica do Python.

Agora que vimos a sintaxe geral da extensão, podemos retornar aos recursos que simplificam o trabalho com grupos em REs complexas.

Às vezes você vai querer usar um grupo para representar uma parte de uma expressão regular, mas não está interessado em recuperar o conteúdo do grupo. Você pode tornar isso explícito usando um grupo de não-captura: `(?:...)`, onde você pode substituir o `...` por qualquer outra expressão regular.

```
>>> m = re.search("([abc])+", "abc")
>>> m.groups()
('c',)
>>> m = re.search("(?:[abc])+", "abc")
>>> m.groups()
()
```

Exceto pelo fato de que não é possível recuperar o conteúdo sobre o qual o grupo corresponde, um grupo de não-captura se comporta exatamente da mesma forma que um grupo de captura; você pode colocar qualquer coisa dentro dele, repeti-lo com um metacaractere de repetição, como o `*`, e aninhá-lo dentro de outros grupos (de captura ou não-captura). `(?:...)` é particularmente útil para modificar um padrão existente, já que você pode adicionar novos grupos sem alterar a forma como todos os outros grupos estão numerados. Deve ser mencionado que não há diferença de desempenho na busca entre grupos de captura e grupos de não-captura; uma forma não é mais rápida que outra.

Uma característica mais significativa são os grupos nomeados: em vez de se referir a eles por números, os grupos podem ser referenciados por um nome.

A sintaxe de um grupo nomeado é uma das extensões específicas do Python: `(?P<name>...)`. *name* é, obviamente, o nome do grupo. Os grupos nomeados se comportam exatamente como os grupos de captura, e, adicionalmente, associam um nome a um grupo. Todos os métodos de objeto correspondência que lidam com grupos de captura aceitam tanto inteiros que se referem ao grupo por número ou strings que contêm o nome do grupo desejado. Os grupos nomeados ainda recebem números, então você pode recuperar informações sobre um grupo de duas maneiras:

```
>>> p = re.compile(r'(?P<word>\b\w+\b)')
>>> m = p.search( '(((( Lots of punctuation )))' )
>>> m.group('word')
'Lots'
>>> m.group(1)
'Lots'
```

Além disso, você pode recuperar grupos nomeados como um dicionário com **`groupdict()`**:

```
>>> m = re.search(r'(?P<first>\w+) (?P<last>\w+)', 'Jane Doe')
>>> m.groupdict()
{'first': 'Jane', 'last': 'Doe'}
```

Os grupos nomeados são úteis porque eles permitem que você use nomes de fácil lembrança, em vez de ter que lembrar de números. Aqui está um exemplo de RE usando o módulo **`imaplib`**:

```
InternalDate = re.compile(r'INTERNALDATE "'
        r'(?P<day>[ 123][0-9])-(?P<mon>[A-Z][a-z][a-z])-'
        r'(?P<year>[0-9][0-9][0-9][0-9])'
        r' (?P<hour>[0-9][0-9]):(?P<min>[0-9][0-9]):(?P<sec>[0-9
        r' (?P<zonen>[-+])(?P<zoneh>[0-9][0-9])(?P<zonem>[0-9][0
        r'"')
```

É obviamente muito mais fácil fazer referência a `m.group('zonem')`, do que ter que se lembrar de capturar o grupo 9.

A sintaxe para referências anteriores em uma expressão, tal como `(...)\1`, faz referência ao número do grupo. Existe, naturalmente, uma variante que usa o nome do grupo em vez do número. Isto é outra extensão Python: `(?P=name)` indica que o conteúdo do grupo chamado *name* deve, novamente, ser correspondido no ponto atual. A expressão regular para encontrar palavras duplicadas, `(\b\w+)\s+\1`, também pode ser escrita como `(?P<word>\b\w+)\s+(?P=word)`:

```
>>> p = re.compile(r'\b(?P<word>\w+)\s+(?P=word)\b')
>>> p.search('Paris in the the spring').group()
'the the'
```

Lookahead assertions

Outra asserção de “largura zero” é a asserção lookahead. Asserções lookahead estão disponíveis tanto na forma positiva quanto na negativa, e se parece com isto:

`(?=...)`
: Asserção lookahead positiva. É bem-sucedida se a expressão regular informada, aqui representada por `...`, corresponde com o conteúdo da localização atual, e falha caso contrário. Mas, uma vez que a expressão informada tenha sido testada, o mecanismo de correspondência não faz qualquer avanço; o resto do padrão é tentado no mesmo local de onde a asserção foi iniciada.

`(?!...)`
: Asserção lookahead negativa. É o oposto da asserção positiva; será bem-sucedida se a expressão informada não corresponder com o conteúdo da posição atual na string.

Para tornar isto concreto, vamos olhar para um caso em que uma lookahead é útil. Considere um padrão simples para corresponder com um nome de arquivo e dividi-lo em pedaços, um nome base e uma extensão, separados por um `.`. Por exemplo, em `news.rc`, `news` é o nome base, e `rc` é a extensão do nome de arquivo.

O padrão para corresponder com isso é muito simples:

```
.*[.].*$
```

Observe que o `.` precisa ser tratado de forma especial, pois é um metacaractere e, portanto, está dentro de uma classe de caracteres para corresponder apenas a esse caractere específico. Observe também o `$` ao final; ele é adicionado para garantir que todo o restante da string seja incluído na extensão. Esta expressão regular corresponde a `foo.bar`, `autoexec.bat`, `sendmail.cf` e `printers.conf`.

Agora, considere complicar um pouco o problema; e se você desejar corresponder com nomes de arquivos onde a extensão não é `bat`? Algumas tentativas incorretas:

```
.*[.][^b].*$
```

A primeira tentativa acima tenta excluir `bat`, exigindo que o primeiro caractere da extensão não seja um `b`. Isso é errado, porque o padrão também não corresponde a `foo.bar`.

```
.*[.]([^b]..|.[^a].|..[^t])$
```

A expressão fica mais confusa se você tentar remendar a primeira solução, exigindo que uma das seguintes situações corresponda: o primeiro caractere da extensão não é `b`; o segundo caractere não é `a`; ou o terceiro caractere não é `t`. Isso aceita `foo.bar` e rejeita `autoexec.bat`, mas requer uma extensão de três letras e não aceitará um nome de arquivo com uma extensão de duas letras, tal como `sendmail.cf`. Nós iremos complicar o padrão novamente em um esforço para corrigi-lo.

```
.*[.]([^b].?.?|.[^a]?.?|..?[^t]?)$
```

Na terceira tentativa, a segunda e terceira letras são todas consideradas opcionais, a fim de permitir correspondência com as extensões mais curtas do que três caracteres, tais como `sendmail.cf`.

Agora, o padrão está ficando realmente muito complicado, o que faz com que seja difícil de ler e compreender. Pior ainda, se o problema mudar e

você quiser excluir tanto `bat` quanto `exe` como extensões, o padrão iria ficar ainda mais complicado e confuso.

Uma lookahead negativo elimina toda esta confusão:

```
.*[.](?!bat$)[^.]*$
```

A lookahead negativa significa: se a expressão `bat` não corresponder até este momento, tente o resto do padrão; se `bat$` tem correspondência, todo o padrão irá falhar. O final `$` é necessário para garantir que algo como `sample.batch`, onde a extensão só começa com o `bat`, será permitido.

Excluir uma outra extensão de nome de arquivo agora é fácil; basta fazer a adição de uma alternativa dentro da asserção. O padrão a seguir exclui os nomes de arquivos que terminam com `bat` ou `exe`:

```
.*[.](?!bat$|exe$)[^.]*$
```

Modifying strings

Até este ponto, nós simplesmente realizamos pesquisas em uma string estática. As expressões regulares também são comumente usadas para modificar strings através de várias maneiras, usando os seguintes métodos padrão:

Método/Atributo	Propósito
`split()`	Divide a string em uma lista, dividindo-a onde quer que haja correspondência com a RE
`sub()`	Encontra todas as substrings que correspondem com a RE e faz a substituição por uma string diferente

Método/Atributo	Propósito
`subn()`	Faz a mesma coisa que **`sub()`**, mas retorna a nova string e o número de substituições

Splitting strings

O método **`split()`** de um padrão divide uma string em pedaços onde quer que a RE corresponda, retornando uma lista formada por esses pedaços. É semelhante ao método **`split()`** de strings, mas oferece muito mais generalidade nos delimitadores que pode usar para fazer a divisão; **`split()`** só implementa a divisão por espaço em branco ou por uma string fixa. Como você deve deduzir, existe também uma função a nível de módulo **`re.split()`**.

.split(*string*[, *maxsplit=0*])

> Divide *string* usando a correspondência com uma expressão regular. Se os parênteses de captura forem utilizados na RE, então seu conteúdo também será retornado como parte da lista resultante. Se *maxsplit* é diferente de zero, serão feitas no máximo *maxplit* divisões.

Você pode limitar o número de divisões feitas, passando um valor para *maxsplit*. Quando *maxsplit* é diferente de zero, serão feita no máximo `maxsplit` divisões, e o restante da string é retornado como o elemento final da lista. No exemplo a seguir, o delimitador é qualquer sequência de caracteres não alfanuméricos.

```
>>> p = re.compile(r'\W+')
>>> p.split('This is a test, short and sweet, of split().')
['This', 'is', 'a', 'test', 'short', 'and', 'sweet', 'of', 'spli
>>> p.split('This is a test, short and sweet, of split().', 3)
['This', 'is', 'a', 'test, short and sweet, of split().']
```

Às vezes, você não está interessado apenas no texto que está entre os delimitadores, mas também precisa saber qual o delimitador foi usado. Se os parênteses de captura são utilizados na RE, então os respectivos valores são também retornados como parte da lista. Compare as seguintes chamadas:

```
>>> p = re.compile(r'\W+')
>>> p2 = re.compile(r'(\W+)')
>>> p.split('This... is a test.')
['This', 'is', 'a', 'test', '']
>>> p2.split('This... is a test.')
['This', '... ', 'is', ' ', 'a', ' ', 'test', '.', '']
```

A função de nível de módulo **`re.split()`** adiciona a RE a ser utilizada como o primeiro argumento, mas, fora isso, é a mesma.

```
>>> re.split(r'[\W]+', 'Words, words, words.')
['Words', 'words', 'words', '']
>>> re.split(r'([\W]+)', 'Words, words, words.')
['Words', ', ', 'words', ', ', 'words', '.', '']
>>> re.split(r'[\W]+', 'Words, words, words.', 1)
['Words', 'words, words.']
```

Search and replace

Outra tarefa comum é encontrar todas as combinações para um padrão e substituí-las por uma string diferente. O método **`sub()`** recebe um valor de substituição, que pode ser uma string ou uma função, e a string a ser processada.

.sub(*replacement*, *string*[, *count=0*])

Retorna a string obtida substituindo as ocorrências mais à esquerda não sobrepostas da RE em `string` pelo valor de substituição `replacement`. Se o padrão não for encontrado, a `string` é retornada inalterada.

O argumento opcional *count* é o número máximo de ocorrências do padrão a ser substituído; *count* deve ser um número inteiro não negativo. O valor padrão 0 significa substituir todas as ocorrências.

Aqui está um exemplo simples do uso do método **sub()**. Ele substitui nomes de cores pela palavra `colour`:

```
>>> p = re.compile('(blue|white|red)')
>>> p.sub('colour', 'blue socks and red shoes')
'colour socks and colour shoes'
>>> p.sub('colour', 'blue socks and red shoes', count=1)
'colour socks and red shoes'
```

O método **subn()** faz o mesmo trabalho, mas retorna uma tupla com duas informações: uma string com novo valor e o número de substituições que foram realizadas:

```
>>> p = re.compile('(blue|white|red)')
>>> p.subn('colour', 'blue socks and red shoes')
('colour socks and colour shoes', 2)
>>> p.subn('colour', 'no colours at all')
('no colours at all', 0)
```

Correspondências vazias somente são substituídas quando não estão adjacente (próxima) a uma correspondência vazia anterior.

```
>>> p = re.compile('x*')
>>> p.sub('-', 'abxd')
'-a-b--d-'
```

Se o valor de substituição (*replacement*) é uma string, qualquer barra invertida é interpretada e processada. Isto é, `\n` é convertido a um único caractere de nova linha, `\r` é convertido em um retorno do carro, e assim por diante. Casos desconhecidos, como `\&` são ignorados. Referências anteriores, como `\6`, são substituídas com a substring correspondida pelo grupo correspondente na RE. Isso permite que você incorpore partes do texto original na string de substituição resultante.

Este exemplo corresponde com a palavra `section`, seguida por uma string colocada entre `{`, `}`, e altera `section` para `subsection`:

```
>>> p = re.compile('section{ ( [^}]* ) }', re.VERBOSE)
>>> p.sub(r'subsection{\1}','section{First} section{second}')
'subsection{First} subsection{second}'
```

Há também uma sintaxe para se referir a grupos nomeados como definido pela sintaxe `(?P<name>...)`. `\g<name>` usará a substring correspondida pelo grupo com nome `name` e `\g<number>` utiliza o número do grupo correspondente. `\g<2>` é, portanto, equivalente a `\2`, mas não é ambígua em uma string de substituição, tal como `\g<2>0`. (`\20` seria interpretado como uma referência ao grupo de 20, e não uma referência ao grupo 2 seguido pelo caractere literal `'0'`). As substituições a seguir são todas equivalentes, mas usam todas as três variações da string de substituição.

```
>>> p = re.compile('section{ (?P<name> [^}]* ) }', re.VERBOSE)
>>> p.sub(r'subsection{\1}','section{First}')
'subsection{First}'
>>> p.sub(r'subsection{\g<1>}','section{First}')
'subsection{First}'
>>> p.sub(r'subsection{\g<name>}','section{First}')
'subsection{First}'
```

replacement também pode ser uma função, que lhe dá ainda mais controle. Se *replacement* for uma função, a função será chamada para todas as ocorrências não sobrepostas de *pattern*. Em cada chamada, a função recebe um argumento de objeto Match para a correspondência e pode usar essas informações para calcular a string de substituição desejada e retorná-la.

No exemplo a seguir, a função de substituição traduz decimais em hexadecimal:

```
>>> def hexrepl(match):
...     "Return the hex string for a decimal number"
...     value = int(match.group())
...     return hex(value)
...
>>> p = re.compile(r'\d+')
>>> p.sub(hexrepl, 'Call 65490 for printing, 49152 for user code
'Call 0xffd2 for printing, 0xc000 for user code.'
```

Ao utilizar a função de nível de módulo **`re.sub()`**, o padrão é passado como o primeiro argumento. O padrão pode ser fornecido como um objeto ou como uma string; se você precisa especificar sinalizadores de expressões regulares, você deve usar um objeto padrão como o primeiro parâmetro, ou

usar modificadores embutidos na string padrão, por exemplo, `sub("(?i)b+", "x", "bbbb BBBB")` retorna `'x x'`.

Common problems

Expressões regulares são uma ferramenta poderosa para algumas aplicações, mas de certa forma o seu comportamento não é intuitivo, e às vezes, as RE não se comportam da maneira que você espera que elas se comportem. Esta seção irá apontar algumas das armadilhas mais comuns.

Use string methods

Às vezes, usar o módulo **`re`** é um equívoco. Se você está fazendo correspondência com uma string fixa, ou uma classe de caractere única, e você não está usando nenhum recurso de **`re`** como o sinalizador **`IGNORECASE`**, então pode não ser necessário todo o poder das expressões regulares. Strings possuem vários métodos para executar operações com strings fixas e eles são, geralmente, muito mais rápidos, porque a implementação é um único e pequeno laço de repetição em C que foi otimizado para esse propósito, em vez do grande e mais generalizado mecanismo das expressões regulares.

Um exemplo pode ser a substituição de uma string fixa única por outra; por exemplo, você pode substituir `word` por `deed`. **`re.sub()`** parece ser a função a ser usada para isso, mas considere o método **`replace()`**. Note que **`replace()`** também irá substituir `word` dentro de palavras, transformando `swordfish` em `sdeedfish`, mas uma RE ingênua teria feito isso também. (Para evitar a realização da substituição de partes de palavras, o padrão teria que ser `\bword\b`, a fim de exigir que `word` tenha um delimitador de palavra em ambos os lados. Isso leva a tarefa para além da capacidade de **`replace()`**.)

Outra tarefa comum é apagar todas as ocorrências de um único caractere de uma string ou substituí-lo por um outro caractere único. Você pode fazer isso com algo como `re.sub('\n', ' ', S)`, mas **`translate()`** é capaz de fazer ambas as tarefas e será mais rápida do que qualquer operação de expressão regular pode ser.

Em suma, antes de recorrer ao o módulo `re`, considere se o seu problema pode ser resolvido com um método de string mais rápido e mais simples.

prefixmatch() (aka match) versus search()

`prefixmatch()` was added in Python 3.15 as the preferred name for `match()`. Before this, it was only known as **`match()`** and the distinction with `search()` was often misunderstood.

`prefixmatch()` aka **`match()`** only checks if the RE matches at the beginning of the string while **`search()`** scans forward through the string for a match.

```
>>> print(re.prefixmatch('super', 'superstition').span())
(0, 5)
>>> print(re.prefixmatch('super', 'insuperable'))
None
```

On the other hand, `search()` scans forward through the string, reporting the first match it finds.

```
>>> print(re.search('super', 'superstition').span())
(0, 5)
>>> print(re.search('super', 'insuperable').span())
(2, 7)
```

This distinction is important to remember when using the old `match()` name in code requiring compatibility with older Python versions.

Greedy versus non-greedy

Ao repetir uma expressão regular, como em `a*`, a ação resultante é consumir o tanto do padrão quanto possível. Este fato, muitas vezes derruba você quando você está tentando corresponder com um par de delimitadores balanceados, tal como os colchetes que cercam uma tag HTML. O padrão ingênuo para combinar uma única tag HTML não funciona por causa da natureza gulosa de `.*`.

```
>>> s = '<html><head><title>Title</title>'
>>> len(s)
32
>>> print(re.prefixmatch('<.*>', s).span())
(0, 32)
>>> print(re.prefixmatch('<.*>', s).group())
<html><head><title>Title</title>
```

A RE corresponde a `'<'` em `'<html>'`, e o `.*` consome o resto da string. Existe ainda mais coisa existente na RE, no entanto, e o `>` pode não corresponder com o final da string, de modo que o mecanismo de expressão regular tem que recuar caractere por caractere até encontrar uma correspondência para `>`. A correspondência final se estende do `'<'` em `'<html>'` ao `'>'` em `'</title>'`, que não é o que você quer.

Neste caso, a solução é usar os quantificadores não-gulosos `*?`, `+?`, `??` ou `{m,n}?`, que corresponde com o mínimo de texto possível. No exemplo acima, o `'>'` é tentado imediatamente após a primeira correspondência de `'<'`, e quando ele falhar, o mecanismo avança um caractere de cada vez, experimentado `'>'` a cada passo. Isso produz justamente o resultado correto:

```
>>> print(re.prefixmatch('<.*?>', s).group())
<html>
```

(Note que a análise de HTML ou XML com expressões regulares é dolorosa. Padrões “sujos e rápidos” irão lidar com casos comuns, mas HTML e XML tem casos especiais que irão quebrar expressões regulares óbvias; com o tempo, expressões regulares que você venha a escrever para lidar com todos os casos possíveis, se tornarão um padrão muito complicado. Use um módulo de análise de HTML ou XML para tais tarefas.)

Usando re.VERBOSE

Nesse momento, você provavelmente deve ter notado que as expressões regulares são de uma notação muito compacta, mas não é possível dizer que são legíveis. REs de complexidade moderada podem se tornar longas

coleções de barras invertidas, parênteses e metacaracteres, fazendo com que se tornem difíceis de ler e compreender.

Para tais REs, especificar a flag **`re.VERBOSE`** ao compilar a expressão regular pode ser útil, porque permite que você formate a expressão regular de forma mais clara.

O sinalizador `re.VERBOSE` produz vários efeitos. Espaço em branco na expressão regular que não está dentro de uma classe de caracteres é ignorado. Isto significa que uma expressão como `dog | cat` é equivalente ao menos legível `dog|cat`, mas `[a b]` ainda vai coincidir com os caracteres `a`, `b`, ou um `espaço`. Além disso, você também pode colocar comentários dentro de uma RE, que se estendem de um caractere `#` até a próxima nova linha. Quando usados junto com strings de aspas triplas, isso permite as REs serem formatadas mais ordenadamente:

```
pat = re.compile(r"""
 \s*                 # Ignora espaços em branco no início
 (?P<header>[^:]+)   # Nome do cabeçalho
 \s* :               # Espaço em branco e caractere de dois pont
 (?P<value>.*?)      # O valor do cabeçalho -- o *? costumava
                     # perder o seguinte espaço em branco ao fir
 \s*$                # Espaço em branco no final até o fim da li
""", re.VERBOSE)
```

Isso é muito mais legível do que:

```
pat = re.compile(r"\s*(?P<header>[^:]+)\s*:(?P<value>.*?)\s*$")
```

Comentários

Regular expressions are a complicated topic. Did this document help you understand them? Were there parts that were unclear, or problems you encountered that weren’t covered here? If so, please send suggestions for improvements to the issue tracker.

O livro mais completo sobre expressões regulares é quase certamente o Mastering Regular Expressions de Jeffrey Friedl’s, publicado pela O’Reilly.

Infelizmente, ele se concentra exclusivamente em sabores de expressões regulares do Perl e do Java, e não contém qualquer material relativo a Python, por isso não vai ser útil como uma referência para a programação em Python. (A primeira edição cobre o módulo `regex` agora removido do Python, o que não vai te ajudar muito.) Considere removê-lo de sua biblioteca.

Programação de Soquetes

Resumo

Os soquetes são usados em quase todos os lugares, mas são uma das tecnologias mais mal compreendidas. Esta é uma visão geral contendo 10.000 pontos a respeitos dos soquetes. Não é realmente um tutorial - ainda terás o trabalho para tornar tudo operacionais. Não abrange os pontos finos (e há muitos deles), mas espero que lhe dê uma base suficiente para começar a utilizar de maneira correta.

Soquetes

Só trataremos dos soquetes INET (ou seja, IPv4), no entanto, os mesmos representam ao menos 99% dos soquetes que estão atualmente em uso. Só estudaremos os soquetes STREAM (ou seja, TCP) - a menos que você realmente saiba o que está fazendo (nesse caso, este documento não é para você!), terás um melhor conhecimento sobre o comportamento e o desempenho dos soquetes STREAM do que qualquer outra coisa. Tentarei aclarar o mistério sobre o que realmente é um soquete, bem como, apresentarei dicas de como trabalhar com soquetes bloqueantes e os não bloqueantes. Começaremos o estudo falando da razão pela qual bloquear um soquete. Precisas saber como os mesmos (os soquetes bloqueantes) antes de estudar os não bloqueantes

Parte do problema em compreender o funcionamento se dá pelo fato de que o “soquete” pode ter significados que são sutilmente diferentes, dependendo do contexto. Então, primeiro, vamos fazer uma distinção entre um soquete “cliente” - o ponto final de uma conversa e um soquete “servidor”, que mais parece como um operador de painel. O aplicativo Cliente (seu navegador, por exemplo) usa soquetes “cliente” exclusivos; O servidor Web com o qual conversamos faz uso de soquetes “servidor” e soquetes de “cliente”.

História

Das várias formas de IPC, os soquetes são, de longe, os mais populares. Em qualquer plataforma, muito provável existirão outras formas de IPC que sejam mais rápidas, porém, para a comunicação entre plataformas, os soquetes são sobre o único jogo na cidade.

They were invented in Berkeley as part of the BSD flavor of Unix. They spread like wildfire with the internet. With good reason — the combination of sockets with INET makes talking to arbitrary machines around the world unbelievably easy (at least compared to other schemes).

Criando um Soquete

De forma geral, quando clicastes no link que te trouxe até esta página (documentação do Python), o que o teu navegador fez, foi o seguinte:

```
# create an INET, STREAMing socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# now connect to the web server on port 80 - the normal http por
s.connect(("www.python.org", 80))
```

Quando a `connect` (conexão) foi estabelecida, o soquete `s` pode ser utilizado para enviar uma solicitação de texto para a página. O mesmo soquete é que irá ler a resposta e, em seguida, o mesmo será destruído. Isso mesmo, será destruído. Os soquetes de Clientes normalmente são usados apenas numa única transação (troca) (ou um pequeno conjunto sequencial de transações).

O que acontece no lado do servidor Web é um pouco mais complexo. Primeiro, o Servidor Web cria um “soquete tipo servidor”:

```
# create an INET, STREAMing socket
serversocket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# bind the socket to a public host, and a well-known port
serversocket.bind((socket.gethostname(), 80))
# become a server socket
serversocket.listen(5)
```

Algumas coisas que deves observar: usamos `socket.gethostname()` para que o soquete esteja visível ao mundo exterior. Se tivéssemos usado `s.bind(('localhost', 80))` ou `s.bind(('127.0.0.1', 80))` ainda teríamos um soquete do tipo “servidor”, mas o mesmo só estaria visível dentro do computador em que está sendo executado. `s.bind(('', 80))` determina que o soquete estará acessível por qualquer computador que possuas o endereço IP do computador.

Uma segunda coisa que precisas observar é: as portas baixas, normalmente estão reservadas para serviços “bem conhecidos”, tais como (HTTP, SNMP etc). Caso estejas brincando, utilize um número alto (números contendo 4 dígitos).

Por fim, o argumento “listen” diz à biblioteca de soquetes que queremos enfileirar no máximo 5 requisições de conexão (normalmente o máximo) antes de recusar começar a recusar conexões externas. Caso o resto do código esteja escrito corretamente, isso deverá ser o suficiente.

Agora que temos um soquete tipo “servidor”, que está ouvindo a porta 80, podemos entrar no laço de repetição principal do servidor web:

```
while True:
    # accept connections from outside
    (clientsocket, address) = serversocket.accept()
    # now do something with the clientsocket
    # in this case, we'll pretend this is a threaded server
    ct = make_client_thread(clientsocket)
    ct.start()
```

Na verdade, existem 3 formas gerais nas quais este loop pode ser implementado - despachando um handle que lide com o `clientsocket`, criando um novo processo que funcione com o `clientsocket`, ou reestruture o aplicativo para trabalhar com soquetes não bloqueáveis e multiplexar entre os nossos soquetes tipo “servidor” e qualquer `cliente` ativo que faça uso do `select`. Trataremos mais sobre isso posteriormente. O importante que temos que saber neste momento é isso: isso é *tudo* o que um soquete tipo “Servidor” irá fazer. O mesmo não enviar quaisquer dado. Não

recebe dados. Apenas produz soquetes “cliente”. Cada `clientsocket` será criado em resposta a algum *outro* soquete “cliente” que fará uma `connect()` ao um determinado host numa determinada porta que estamos vinculados. Será dessa forma que criamos aquele “cliente”, voltaremos a ouvir mais conexões. Os dois “clientes” são livres para conversar - eles estão usando uma porta alocada dinamicamente que será descartada quando a conversa terminar.

IPC

Se precisas que o IPC seja rápido entre dois processos numa máquina, deverás procurar por pipes ou compartilhamento de memória. Se decidires usar os soquetes AF_INET, vincule o soquete “servidor” a `'localhost'`. Na maioria das plataformas, isso levará um atalho em torno de algumas camadas de código de rede e funcionará um pouco mais rápido.

Ver também

A módulo `multiprocessing` faz a integração do IPC de forma multiplataforma numa API de nível mais alto.

Usando um Soquete

A primeira coisa que precisamos observar, é que o soquete “cliente” do navegador e o soquete “cliente” do servidor Web são “animais” idênticos. Ou seja, esta é uma conversa “peer to peer”. Ou, em outras palavras, *como designer, terás que decidir quais são as regras da etiqueta para uma conversa*. Normalmente, o soquete `connect`ando inicia a conversa, enviando uma solicitação ou talvez um sinal. Mas essa é uma decisão de design - não é uma regra de soquetes.

Agora, há dois conjuntos de verbos que podemos usar na comunicação. Podemos usar `send` e `recv`, ou podemos transformar o seu soquete cliente numa “besta” semelhante a um arquivo e usar `read` e `write`. O último é o modo como Java apresenta seus soquetes. Não tratarei disso aqui, exceto

para avisar que precisas usar o `flush` com soquetes. Estes são “arquivos” em buffer e um erro comum é “escrever” algo e, em seguida, “ler” uma resposta. Sem que haja um `flush` lá dentro, poderás esperar eternamente pela resposta, porque o pedido ainda poderá estar no buffer de saída.

Agora, chegamos ao principal obstáculo no uso dos soquetes - `send` e `recv` operam nos buffers de rede. Eles não lidam necessariamente com todos os bytes que você os entrega (ou esperam deles), porque o foco principal deles é lidar com os buffers de rede. Em geral, os mesmos retornam quando os buffers de rede associados foram preenchidos (`send`) ou esvaziados (`recv`). Eles então lhe dizem quantos bytes eles trataram. É *sua* responsabilidade chamá-los novamente até que a sua mensagem tenha sido completamente tratada.

Quando um `recv` retornar 0 bytes, significa que o outro lado finalizou a conexão (ou está no processo de fechamento) da conexão. Não receberás mais dados nesta conexão. Pra sempre. Poderás enviar dados com sucesso; Falaremos mais sobre isso posteriormente.

Um protocolo como o HTTP utiliza um soquete para apenas uma transferência. O cliente envia uma solicitação e depois lê uma resposta. É isso aí. O soquete é descartado. Isso significa que um cliente pode detectar o final da resposta ao receber 0 bytes.

But if you plan to reuse your socket for further transfers, you need to realize that *there is no* EOT *on a socket.* I repeat: if a socket `send` or `recv` returns after handling 0 bytes, the connection has been broken. If the connection has *not* been broken, you may wait on a `recv` forever, because the socket will *not* tell you that there’s nothing more to read (for now). Now if you think about that a bit, you’ll come to realize a fundamental truth of sockets: *messages must either be fixed length* (yuck), *or be delimited* (shrug), *or indicate how long they are* (much better), *or end by shutting down the connection*. The choice is entirely yours, (but some ways are righter than others).

Assuming you don’t want to end the connection, the simplest solution is a fixed length message:

```
class MySocket:
    """demonstration class only
      - coded for clarity, not efficiency
    """

    def __init__(self, sock=None):
        if sock is None:
            self.sock = socket.socket(
                            socket.AF_INET, socket.SOCK_STREAM)
        else:
            self.sock = sock

    def connect(self, host, port):
        self.sock.connect((host, port))

    def mysend(self, msg):
        totalsent = 0
        while totalsent < MSGLEN:
            sent = self.sock.send(msg[totalsent:])
            if sent == 0:
                raise RuntimeError("socket connection broken")
            totalsent = totalsent + sent

    def myreceive(self):
        chunks = []
        bytes_recd = 0
        while bytes_recd < MSGLEN:
            chunk = self.sock.recv(min(MSGLEN - bytes_recd, 2048
            if chunk == b'':
                raise RuntimeError("socket connection broken")
            chunks.append(chunk)
            bytes_recd = bytes_recd + len(chunk)
        return b''.join(chunks)
```

The sending code here is usable for almost any messaging scheme - in Python you send strings, and you can use `len()` to determine its length (even if it has embedded `\0` characters). It's mostly the receiving code that gets more complex. (And in C, it's not much worse, except you can't use `strlen` if the message has embedded `\0`s.)

The easiest enhancement is to make the first character of the message an indicator of message type, and have the type determine the length. Now you have two `recvs` - the first to get (at least) that first character so you can look up the length, and the second in a loop to get the rest. If you decide to go the delimited route, you'll be receiving in some arbitrary chunk size, (4096 or 8192 is frequently a good match for network buffer sizes), and scanning what you've received for a delimiter.

Uma complicação a ter em conta: se o seu protocolo de conversa permitir que várias mensagens sejam enviadas de volta para trás (sem algum tipo de resposta) e você passar `recv` um tamanho arbitrário de bloco, você pode acabar lendo o início de uma mensagem seguinte. Você precisará deixar isso de lado e segurá-lo até que seja necessário.

Prefixing the message with its length (say, as 5 numeric characters) gets more complex, because (believe it or not), you may not get all 5 characters in one `recv`. In playing around, you'll get away with it; but in high network loads, your code will very quickly break unless you use two `recv` loops - the first to determine the length, the second to get the data part of the message. Nasty. This is also when you'll discover that `send` does not always manage to get rid of everything in one pass. And despite having read this, you will eventually get bit by it!

In the interests of space, building your character, (and preserving my competitive position), these enhancements are left as an exercise for the reader. Lets move on to cleaning up.

Dados Binários

It is perfectly possible to send binary data over a socket. The major problem is that not all machines use the same formats for binary data. For example, network byte order [https://en.wikipedia.org/wiki/Endianness#Networking] is big-endian, with the most significant byte first, so a 16 bit integer with the value `1` would be the two hex bytes `00 01`. However, most common processors (x86/AMD64, ARM, RISC-V), are little-endian, with the least significant byte first - that same `1` would be `01 00`.

Socket libraries have calls for converting 16 and 32 bit integers - `ntohl, htonl, ntohs, htons` where “n” means *network* and “h” means *host*, “s” means *short* and “l” means *long*. Where network order is host order, these do nothing, but where the machine is byte-reversed, these swap the bytes around appropriately.

In these days of 64-bit machines, the ASCII representation of binary data is frequently smaller than the binary representation. That’s because a surprising amount of the time, most integers have the value 0, or maybe 1. The string `"0"` would be two bytes, while a full 64-bit integer would be 8. Of course, this doesn’t fit well with fixed-length messages. Decisions, decisions.

Desconectando

Strictly speaking, you’re supposed to use `shutdown` on a socket before you `close` it. The `shutdown` is an advisory to the socket at the other end. Depending on the argument you pass it, it can mean “I’m not going to send anymore, but I’ll still listen”, or “I’m not listening, good riddance!”. Most socket libraries, however, are so used to programmers neglecting to use this piece of etiquette that normally a `close` is the same as `shutdown(); close()`. So in most situations, an explicit `shutdown` is not needed.

One way to use `shutdown` effectively is in an HTTP-like exchange. The client sends a request and then does a `shutdown(1)`. This tells the server “This client is done sending, but can still receive.” The server can detect “EOF” by a receive of 0 bytes. It can assume it has the complete request. The server sends a reply. If the `send` completes successfully then, indeed, the client was still receiving.

Python takes the automatic shutdown a step further, and says that when a socket is garbage collected, it will automatically do a `close` if it’s needed. But relying on this is a very bad habit. If your socket just disappears without doing a `close`, the socket at the other end may hang indefinitely, thinking you’re just being slow. *Please* `close` your sockets when you’re done.

Quando o Soquete Morre

Probably the worst thing about using blocking sockets is what happens when the other side comes down hard (without doing a `close`). Your socket is likely to hang. TCP is a reliable protocol, and it will wait a long, long time before giving up on a connection. If you're using threads, the entire thread is essentially dead. There's not much you can do about it. As long as you aren't doing something dumb, like holding a lock while doing a blocking read, the thread isn't really consuming much in the way of resources. Do *not* try to kill the thread - part of the reason that threads are more efficient than processes is that they avoid the overhead associated with the automatic recycling of resources. In other words, if you do manage to kill the thread, your whole process is likely to be screwed up.

Soquetes não-bloqueantes

If you've understood the preceding, you already know most of what you need to know about the mechanics of using sockets. You'll still use the same calls, in much the same ways. It's just that, if you do it right, your app will be almost inside-out.

In Python, you use `socket.setblocking(False)` to make it non-blocking. In C, it's more complex, (for one thing, you'll need to choose between the BSD flavor `O_NONBLOCK` and the almost indistinguishable POSIX flavor `O_NDELAY`, which is completely different from `TCP_NODELAY`), but it's the exact same idea. You do this after creating the socket, but before using it. (Actually, if you're nuts, you can switch back and forth.)

A principal diferença mecânica é que `send`, `recv`, `connect` e `accept` podem retornar sem terem feito nada. Terás (claro) uma série de escolhas. Poderás verificar o código de retorno e os códigos de erro que geralmente nos deixam loucos. Se não acreditas em mim, tente alguma vez. Seu aplicativo vai crescendo, buggy e sugam CPU. Então, vamos ignorar as soluções mortas no cérebro e fazê-lo direito.

Utilize `select`.

In C, coding `select` is fairly complex. In Python, it's a piece of cake, but it's close enough to the C version that if you understand `select` in Python, you'll have little trouble with it in C:

```
ready_to_read, ready_to_write, in_error = \
               select.select(
                  potential_readers,
                  potential_writers,
                  potential_errs,
                  timeout)
```

You pass `select` three lists: the first contains all sockets that you might want to try reading; the second all the sockets you might want to try writing to, and the last (normally left empty) those that you want to check for errors. You should note that a socket can go into more than one list. The `select` call is blocking, but you can give it a timeout. This is generally a sensible thing to do - give it a nice long timeout (say a minute) unless you have good reason to do otherwise.

In return, you will get three lists. They contain the sockets that are actually readable, writable and in error. Each of these lists is a subset (possibly empty) of the corresponding list you passed in.

If a socket is in the output readable list, you can be as-close-to-certain-as-we-ever-get-in-this-business that a `recv` on that socket will return *something*. Same idea for the writable list. You'll be able to send *something*. Maybe not all you want to, but *something* is better than nothing. (Actually, any reasonably healthy socket will return as writable - it just means outbound network buffer space is available.)

If you have a "server" socket, put it in the potential_readers list. If it comes out in the readable list, your `accept` will (almost certainly) work. If you have created a new socket to `connect` to someone else, put it in the potential_writers list. If it shows up in the writable list, you have a decent chance that it has connected.

Actually, `select` can be handy even with blocking sockets. It's one way of determining whether you will block - the socket returns as readable when there's something in the buffers. However, this still doesn't help with the

problem of determining whether the other end is done, or just busy with something else.

Portability alert: On Unix, `select` works both with the sockets and files. Don't try this on Windows. On Windows, `select` works with sockets only. Also note that in C, many of the more advanced socket options are done differently on Windows. In fact, on Windows I usually use threads (which work very, very well) with my sockets.

Técnicas de ordenação

As listas em Python possuem um método embutido `list.sort()` que modifica a lista em si. Há também a função embutida `sorted()` que constrói uma nova lista ordenada à partir de um iterável.

Neste documento, exploramos várias técnicas para ordenar dados utilizando Python.

Básico de ordenação

Uma ordenação ascendente simples é muito fácil: apenas chame a função `sorted()`. Você terá como retorno uma nova lista ordenada:

```
>>> sorted([5, 2, 3, 1, 4])
[1, 2, 3, 4, 5]
```

Você também pode utilizar o método `list.sort()`. Ele modificará a lista localmente (e retornará `None` para evitar confusão). Usualmente este método é menos conveniente que a função `sorted()` - mas, se a lista original não for posteriormente útil, esta maneira será levemente mais eficiente.

```
>>> a = [5, 2, 3, 1, 4]
>>> a.sort()
>>> a
[1, 2, 3, 4, 5]
```

Outra diferença é que o método `list.sort()` é aplicável apenas às listas. Em contrapartida, a função `sorted()` aceita qualquer iterável.

```
>>> sorted({1: 'D', 2: 'B', 3: 'B', 4: 'E', 5: 'A'})
[1, 2, 3, 4, 5]
```

Funções chave

Tanto o método `list.sort()` quando as funções `sorted()`, `min()`, `max()`, `heapq.nsmallest()` e `heapq.nlargest()` possuem o parâmetro *key*, que especifica uma função (ou outro chamável) a ser invocado para cada elemento da lista antes de ser realizada a comparação.

Por exemplo, temos abaixo uma comparação de strings, sem distinção entre caractéres maiúsculos e minúsculos, utilizando `str.casefold()`:

```
>>> sorted("Essa é uma string de teste do Andrew".split(), key=s
['Andrew', 'de', 'do', 'Essa', 'string', 'teste', 'uma', 'é']
```

O valor do parâmetro *key* deve ser uma função (ou outro chamável) que recebe um único argumento e retorna uma chave à ser utilizada na ordenação. Como a função chave é chamada exatamente uma vez para cada elemento, esta técnica é rápida.

Um padrão comum é ordenar objetos complexos utilizando um dos índices do objeto como chave. Por exemplo:

```
>>> tuplas_alunos = [
...     ('joão', 'A', 15),
...     ('joana', 'B', 12),
...     ('davi', 'B', 10),
... ]
>>> sorted(tuplas_alunos, key=lambda aluno: aluno[2])   # ordena
[('davi', 'B', 10), ('joana', 'B', 12), ('joão', 'A', 15)]
```

A mesma técnica funciona com objetos que possuem atributos nomeados. Por exemplo:

```
>>> class Aluno:
...     def __init__(self, nome, nota, idade):
...         self.nome = nome
...         self.nota = nota
...         self.idade = idade
...     def __repr__(self):
...         return repr((self.nome, self.nota, self.idade))

>>> objetos_alunos = [
```

```
...     Aluno('john', 'A', 15),
...     Aluno('jane', 'B', 12),
...     Aluno('dave', 'B', 10),
... ]
>>> sorted(objetos_alunos, key=lambda aluno: aluno.idade)   # or
[('davi', 'B', 10), ('joana', 'B', 12), ('joão', 'A', 15)]
```

Objetos com atríbutos nomeados podem ser criados como uma classe padrão, igual ao exemplo acima, podem ser instâncias de **`dataclass`** ou mesmo uma named tuple.

Operator Module Functions and Partial Function Evaluation

O padrão de função chave mostrado acima é muito comum, por isso, Python provê funções convenientes para tornar as funções de acesso mais fáceis e rápidas. O módulo **`operator`** tem as funções **`itemgetter()`**, **`attrgetter()`** e **`methodcaller()`**

Usando estas funções, os exemplos acima se tornam mais simples e mais rápidos:

```
>>> from operator import itemgetter, attrgetter

>>> sorted(student_tuples, key=itemgetter(2))
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]

>>> sorted(student_objects, key=attrgetter('age'))
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]
```

As funções do módulo operator permite múltiplos níveis de ordenação. Por exemplo, ordenar por *grade* e então por *age*:

```
>>> sorted(student_tuples, key=itemgetter(1,2))
[('john', 'A', 15), ('dave', 'B', 10), ('jane', 'B', 12)]

>>> sorted(student_objects, key=attrgetter('grade', 'age'))
[('john', 'A', 15), ('dave', 'B', 10), ('jane', 'B', 12)]
```

The `functools` module provides another helpful tool for making key-functions. The `partial()` function can reduce the arity [https://en.wikipedia.org/wiki/Arity] of a multi-argument function making it suitable for use as a key-function.

```
>>> from functools import partial
>>> from unicodedata import normalize

>>> names = 'Zoë Åbjørn Núñez Élana Zeke Abe Nubia Eloise'.split

>>> sorted(names, key=partial(normalize, 'NFD'))
['Abe', 'Åbjørn', 'Eloise', 'Élana', 'Nubia', 'Núñez', 'Zeke', '

>>> sorted(names, key=partial(normalize, 'NFC'))
['Abe', 'Eloise', 'Nubia', 'Núñez', 'Zeke', 'Zoë', 'Åbjørn', 'Él
```

Ascendente e descendente

Tanto o método `list.sort()` quanto a função `sorted()` aceitam um valor booleano para o parâmetro *reverse*. Essa flag é utilizada para ordenações descendentes. Por exemplo, para retornar os dados de estudantes pela ordem inversa de *age*:

```
>>> sorted(student_tuples, key=itemgetter(2), reverse=True)
[('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]

>>> sorted(student_objects, key=attrgetter('age'), reverse=True)
[('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]
```

Estabilidade de Ordenação e Ordenações Complexas

Ordenações são garantidas de serem estáveis [https://en.wikipedia.org/wiki/Sorting_algorithm#Stability]. Isso significa que quando múltiplos registros possuem a mesma chave, eles terão sua ordem original preservada.

```
>>> data = [('red', 1), ('blue', 1), ('red', 2), ('blue', 2)]
>>> sorted(data, key=itemgetter(0))
[('blue', 1), ('blue', 2), ('red', 1), ('red', 2)]
```

Observe como os dois registros de *blue* permanecem em sua ordem original de forma que `('blue',1)` é garantido de preceder `('blue',2)`.

Esta maravilhosa propriedade permite que você construa ordenações complexas em uma série de passos de ordenação. Por exemplo, para ordenar os registros de estudante por ordem descendente de *grade* e então ascendente de *age*, primeiro ordene *age* e depois ordene novamente utilizando *grade*:

```
>>> s = sorted(student_objects, key=attrgetter('age'))     # sor
>>> sorted(s, key=attrgetter('grade'), reverse=True)       # now
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]
```

Isso pode ser abstrato no caso das funções invólucros que podem receber uma lista e uma tupla com o campos e então ordená-los em múltiplos passos.

```
>>> def multisort(xs, specs):
...     for key, reverse in reversed(specs):
...         xs.sort(key=attrgetter(key), reverse=reverse)
...     return xs

>>> multisort(list(student_objects), (('grade', True), ('age', F
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]
```

O algoritmo Timsort [https://en.wikipedia.org/wiki/Timsort] utilizado no Python realiza múltiplas ordenações de maneira eficiente, pois se aproveita de qualquer ordenação já presente no conjunto de dados.

Decorate-Sort-Undecorate

Esse item idiomático, chamado de Decorate-Sort-Undecorate, é realizado em três passos:

- Primeiro, a lista inicial é decorada com novos valores que controlarão a ordem em que ocorrerá a ordenação
- Segundo, a lista decorada é ordenada.
- Finalmente, os valores decorados são removidos, criando uma lista que contém apenas os valores iniciais na nova ordenação.

Por exemplo, para ordenar os dados dos estudantes por *grade* usando a abordagem DSU:

```
>>> decorated = [(student.grade, i, student) for i, student in e
>>> decorated.sort()
>>> [student for grade, i, student in decorated]               #
[('john', 'A', 15), ('jane', 'B', 12), ('dave', 'B', 10)]
```

Esse padrão idiomático funciona porque tuplas são comparadas lexicograficamente; os primeiros itens são comparados; se eles são semelhantes, então os segundos itens são comparados e assim sucessivamente.

Não é estritamente necessário incluir o índice *i* em todos os casos de listas decoradas, mas fazer assim traz dois benefícios:

- A ordenação é estável - se dois itens tem a mesma chave, suas ordens serão preservadas na lista ordenada
- Os itens originais não precisarão ser comparados porque a ordenação de tuplas decoradas será determinada por no máximo os primeiros dois itens. Então, por exemplo, a lista original poderia conter números complexos que não poderão ser ordenados diretamente.

Outro nome para este padrão idiomático é Schwartzian transform [https://en.wikipedia.org/wiki/Schwartzian_transform] de Randal L. Schwartz, que popularizou isto entre os programadores Perl.

Agora que a ordenação do Python prevê funções-chave, essa técnica não se faz mais necessária.

Funções de comparação

Unlike key functions that return an absolute value for sorting, a comparison function computes the relative ordering for two inputs.

For example, a balance scale [https://upload.wikimedia.org/wikipedia/commons/1/17/Balance_à_tabac_1850.JPG] compares two samples giving a relative ordering: lighter, equal, or heavier. Likewise, a comparison function such as `cmp(a, b)` will return a negative value for less-than, zero if the inputs are equal, or a positive value for greater-than.

It is common to encounter comparison functions when translating algorithms from other languages. Also, some libraries provide comparison functions as part of their API. For example, `locale.strcoll()` is a comparison function.

To accommodate those situations, Python provides `functools.cmp_to_key` to wrap the comparison function to make it usable as a key function:

```
sorted(words, key=cmp_to_key(strcoll))  # locale-aware sort orde
```

Strategies For Unorderable Types and Values

A number of type and value issues can arise when sorting. Here are some strategies that can help:

- Convert non-comparable input types to strings prior to sorting:

```
>>> data = ['twelve', '11', 10]
>>> sorted(map(str, data))
['10', '11', 'twelve']
```

This is needed because most cross-type comparisons raise a `TypeError`.

- Remove special values prior to sorting:

```
>>> from math import isnan
>>> from itertools import filterfalse
>>> data = [3.3, float('nan'), 1.1, 2.2]
```

```
>>> sorted(filterfalse(isnan, data))
[1.1, 2.2, 3.3]
```

This is needed because the IEEE-754 standard [https://en.wikipedia.org/wiki/IEEE_754] specifies that, “Every NaN shall compare unordered with everything, including itself.”

Likewise, `None` can be stripped from datasets as well:

```
>>> data = [3.3, None, 1.1, 2.2]
>>> sorted(x for x in data if x is not None)
[1.1, 2.2, 3.3]
```

This is needed because `None` is not comparable to other types.

- Convert mapping types into sorted item lists before sorting:

```
>>> data = [{'a': 1}, {'b': 2}]
>>> sorted(data, key=lambda d: sorted(d.items()))
[{'a': 1}, {'b': 2}]
```

This is needed because dict-to-dict comparisons raise a **`TypeError`**.

- Convert set types into sorted lists before sorting:

```
>>> data = [{'a', 'b', 'c'}, {'b', 'c', 'd'}]
>>> sorted(map(sorted, data))
[['a', 'b', 'c'], ['b', 'c', 'd']]
```

This is needed because the elements contained in set types do not have a deterministic order. For example, `list({'a', 'b'})` may produce either `['a', 'b']` or `['b', 'a']`.

Curiosidades e conclusões

- For locale aware sorting, use **`locale.strxfrm()`** for a key function or **`locale.strcoll()`** for a comparison function. This is necessary because “alphabetical” sort orderings can vary across cultures even if the underlying alphabet is the same.

- O parâmetro *reverse* ainda mantém a estabilidade da ordenação (para que os registros com chaves iguais mantenham a ordem original). Curiosamente, esse efeito pode ser simulado sem o parâmetro usando a função embutida `reversed()` duas vezes:

```
>>> data = [('red', 1), ('blue', 1), ('red', 2), ('blue', 2)
>>> standard_way = sorted(data, key=itemgetter(0), reverse=T
>>> double_reversed = list(reversed(sorted(reversed(data), k
>>> assert standard_way == double_reversed
>>> standard_way
[('red', 1), ('red', 2), ('blue', 1), ('blue', 2)]
```

- As rotinas de classificação usam < ao fazer comparações entre dois objetos. Portanto, é fácil adicionar uma ordem de classificação padrão a uma classe definindo um método `__lt__()`:

```
>>> Student.__lt__ = lambda self, other: self.age < other.ag
>>> sorted(student_objects)
[('dave', 'B', 10), ('jane', 'B', 12), ('john', 'A', 15)]
```

  However, note that < can fall back to using `__gt__()` if `__lt__()` is not implemented (see `object.__lt__()` for details on the mechanics). To avoid surprises, **PEP 8** [https://peps.python.org/pep-0008/] recommends that all six comparison methods be implemented. The `@~functools.total_ordering` decorator is provided to make that task easier.

- As funções principais não precisam depender diretamente dos objetos que estão sendo ordenados. Uma função chave também pode acessar recursos externos. Por exemplo, se as notas dos alunos estiverem armazenadas em um dicionário, elas poderão ser usadas para ordenar uma lista separada de nomes de alunos:

```
>>> students = ['dave', 'john', 'jane']
>>> newgrades = {'john': 'F', 'jane':'A', 'dave': 'C'}
>>> sorted(students, key=newgrades.__getitem__)
['jane', 'dave', 'john']
```

Ordenações parciais

Some applications require only some of the data to be ordered. The standard library provides several tools that do less work than a full sort:

- `min()` and `max()` return the smallest and largest values, respectively. These functions make a single pass over the input data and require almost no auxiliary memory.
- `heapq.nsmallest()` and `heapq.nlargest()` return the *n* smallest and largest values, respectively. These functions make a single pass over the data keeping only *n* elements in memory at a time. For values of *n* that are small relative to the number of inputs, these functions make far fewer comparisons than a full sort.
- `heapq.heappush()` and `heapq.heappop()` create and maintain a partially sorted arrangement of data that keeps the smallest element at position `0`. These functions are suitable for implementing priority queues which are commonly used for task scheduling.

Unicode

Este documento fala sobre o suporte do Python para a especificação Unicode de representação de dados textuais e explica diversos problemas que as pessoas costumam encontrar quando tentam trabalhar com Unicode.

Introdução ao Unicode

Definições

Os programas de hoje precisam lidar com uma grande variedade de caracteres. Aplicações são frequentemente internacionalizadas para mostrar mensagens e gerar saídas em uma variedade de idiomas selecionáveis por usuários; o mesmo programa precisar apresentar mensagens de erro em inglês, francês, japonês, hebraico ou russo. Conteúdo da web pode ser escrito em qualquer um desses idiomas e ainda incluir uma variedade de emojis. O tipo string do Python usa o padrão Unicode para representação de caracteres, o que permite aos programas em Python funcionar com todos estes diferentes caracteres.

Unicode (https://www.unicode.org/) é a especificação que visa listar cada caractere utilizado pelos idiomas humanos e dar a cada caractere um código único. As especificações Unicode são continuamente revisadas e atualizadas para adicionar novos idiomas e símbolos.

A **character** is the smallest possible component of a text. ‘A’, ‘B’, ‘C’, etc., are all different characters. So are ‘È’ and ‘Í’. Characters vary depending on the language or context you’re talking about. For example, there’s a character for “Roman Numeral One”, ‘Ⅰ’, that’s separate from the uppercase letter ‘I’. They’ll usually look the same, but these are two different characters that have different meanings.

The Unicode standard describes how characters are represented by **code points**. A code point value is an integer in the range 0 to 0x10FFFF (about

1.1 million values, the actual number assigned [https://www.unicode.org/versions/latest/#Summary] is less than that). In the standard and in this document, a code point is written using the notation `U+265E` to mean the character with value `0x265e` (9,822 in decimal).

O padrão Unicode contém várias tabelas listando caracteres e seus pontos de código:

```
0061    'a'; LATIN SMALL LETTER A
0062    'b'; LATIN SMALL LETTER B
0063    'c'; LATIN SMALL LETTER C
...
007B    '{'; LEFT CURLY BRACKET
...
2167    'Ⅷ'; ROMAN NUMERAL EIGHT
2168    'Ⅸ'; ROMAN NUMERAL NINE
...
265E    '♞'; BLACK CHESS KNIGHT
265F    '♟'; BLACK CHESS PAWN
...
1F600   '😀'; GRINNING FACE
1F609   '😉'; WINKING FACE
...
```

Strictly, these definitions imply that it's meaningless to say 'this is character `U+265E`'. `U+265E` is a code point, which represents some particular character; in this case, it represents the character 'BLACK CHESS KNIGHT', '♞'. In informal contexts, this distinction between code points and characters will sometimes be forgotten.

Um caractere é representado na tela ou no papel como um conjunto de elementos gráficos que é chamado de **glifo**. O glifo para o A maiúsculo, por exemplo, são dois traços diagonais e um traço horizontal, embora os detalhes exatos dependem da fonte utilizada. Na maior parte do código Python não é preciso se preocupar com glifos; descobrir qual o glifo correto a ser mostrado é normalmente parte do trabalho da ferramenta GUI ou do responsável pela renderização de fontes no terminal.

Codificações

To summarize the previous section: a Unicode string is a sequence of code points, which are numbers from 0 through `0x10FFFF` (1,114,111 decimal). This sequence of code points needs to be represented in memory as a set of **code units**, and **code units** are then mapped to 8-bit bytes. The rules for translating a Unicode string into a sequence of bytes are called a **character encoding**, or just an **encoding**.

The first encoding you might think of is using 32-bit integers as the code unit, and then using the CPU's representation of 32-bit integers. In this representation, the string "Python" might look like this:

```
   P           y           t           h           o           r
0x50 00 00 00 79 00 00 00 74 00 00 00 68 00 00 00 6f 00 00 00 6e
   0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20
```

Esta representação é direta, mas usá-la gera uma série de problemas.

1. Ela não é portátil; diferentes processadores ordenam os bytes de forma diferente.
2. Ela gera desperdício de espaço. Na maior parte dos textos, a maioria dos pontos de código são menores que 127 ou menores que 255, então muito do espaço é ocupado por bytes `0x00` . A string acima necessita 24 bytes comparado com os 6 bytes necessários em uma representação ASCII. O aumento de uso da memória RAM normalmente não importa tanto (computadores desktop possuem gigabytes de RAM e strings normalmente não são tão grandes), mas expandir o uso de disco ou de banda por um fator de 4 é inaceitável.
3. Ela não é compatível com as funções de C existentes, como `strlen()`, então uma série de novas funções de string serão necessárias.

Therefore this encoding isn't used very much, and people instead choose other encodings that are more efficient and convenient, such as UTF-8.

UTF-8 is one of the most commonly used encodings, and Python often defaults to using it. UTF stands for "Unicode Transformation Format", and the '8' means that 8-bit values are used in the encoding. (There are also UTF-16 and UTF-32 encodings, but they are less frequently used than UTF-8.) UTF-8 uses the following rules:

1. If the code point is < 128, it's represented by the corresponding byte value.
2. If the code point is >= 128, it's turned into a sequence of two, three, or four bytes, where each byte of the sequence is between 128 and 255.

UTF-8 tem muitas propriedades convenientes:

1. Ela pode lidar com qualquer ponto de código Unicode.
2. A Unicode string is turned into a sequence of bytes that contains embedded zero bytes only where they represent the null character (U+0000). This means that UTF-8 strings can be processed by C functions such as `strcpy()` and sent through protocols that can't handle zero bytes for anything other than end-of-string markers.
3. Uma string de texto ASCII é também um texto UTF-8 válido.
4. UTF-8 is fairly compact; the majority of commonly used characters can be represented with one or two bytes.
5. If bytes are corrupted or lost, it's possible to determine the start of the next UTF-8-encoded code point and resynchronize. It's also unlikely that random 8-bit data will look like valid UTF-8.
6. UTF-8 is a byte oriented encoding. The encoding specifies that each character is represented by a specific sequence of one or more bytes. This avoids the byte-ordering issues that can occur with integer and word oriented encodings, like UTF-16 and UTF-32, where the sequence of bytes varies depending on the hardware on which the string was encoded.

Referências

The Unicode Consortium site [https://www.unicode.org] has character charts, a glossary, and PDF versions of the Unicode specification. Be prepared for some difficult reading. A chronology [https://www.unicode.org/history/] of the origin and development of Unicode is also available on the site.

On the Computerphile Youtube channel, Tom Scott briefly discusses the history of Unicode and UTF-8 [https://www.youtube.com/watch?v=MijmeoH9LT4] (9 minutes 36 seconds).

To help understand the standard, Jukka Korpela has written an introductory guide [https://jkorpela.fi/unicode/guide.html] to reading the Unicode character tables.

Another good introductory article [https://www.joelonsoftware.com/2003/10/08/the-absolute-minimum-every-software-developer-absolutely-positively-must-know-about-unicode-and-character-sets-no-excuses/] was written by Joel Spolsky. If this introduction didn't make things clear to you, you should try reading this alternate article before continuing.

Wikipedia entries are often helpful; see the entries for "character encoding [https://en.wikipedia.org/wiki/Character_encoding]" and UTF-8 [https://en.wikipedia.org/wiki/UTF-8], for example.

Suporte a Unicode no Python

Now that you've learned the rudiments of Unicode, we can look at Python's Unicode features.

O Tipo String

Since Python 3.0, the language's `str` type contains Unicode characters, meaning any string created using `"unicode rocks!"`, `'unicode rocks!'`, or the triple-quoted string syntax is stored as Unicode.

The default encoding for Python source code is UTF-8, so you can simply include a Unicode character in a string literal:

```
try:
    with open('/tmp/input.txt', 'r') as f:
        ...
except OSError:
    # 'File not found' error message.
    print("Fichier non trouvé")
```

Side note: Python 3 also supports using Unicode characters in identifiers:

```
répertoire = "/tmp/records.log"
with open(répertoire, "w") as f:
```

```
    f.write("test\n")
```

If you can't enter a particular character in your editor or want to keep the source code ASCII-only for some reason, you can also use escape sequences in string literals. (Depending on your system, you may see the actual capital-delta glyph instead of a u escape.)

```
>>> "\N{GREEK CAPITAL LETTER DELTA}"  # Using the character name
'\u0394'
>>> "\u0394"                          # Using a 16-bit hex value
'\u0394'
>>> "\U00000394"                      # Using a 32-bit hex value
'\u0394'
```

In addition, one can create a string using the `decode()` method of `bytes`. This method takes an *encoding* argument, such as `UTF-8`, and optionally an *errors* argument.

The *errors* argument specifies the response when the input string can't be converted according to the encoding's rules. Legal values for this argument are `'strict'` (raise a `UnicodeDecodeError` exception), `'replace'` (use `U+FFFD`, `REPLACEMENT CHARACTER`), `'ignore'` (just leave the character out of the Unicode result), or `'backslashreplace'` (inserts a `\xNN` escape sequence). The following examples show the differences:

```
>>> b'\x80abc'.decode("utf-8", "strict")
Traceback (most recent call last):
    ...
UnicodeDecodeError: 'utf-8' codec can't decode byte 0x80 in posi
  invalid start byte
>>> b'\x80abc'.decode("utf-8", "replace")
'\ufffdabc'
>>> b'\x80abc'.decode("utf-8", "backslashreplace")
'\\x80abc'
>>> b'\x80abc'.decode("utf-8", "ignore")
'abc'
```

Encodings are specified as strings containing the encoding's name. Python comes with roughly 100 different encodings; see the Python Library

Reference at Standard Encodings for a list. Some encodings have multiple names; for example, `'latin-1'`, `'iso_8859_1'` and `'8859'` are all synonyms for the same encoding.

One-character Unicode strings can also be created with the **`chr()`** built-in function, which takes integers and returns a Unicode string of length 1 that contains the corresponding code point. The reverse operation is the built-in **`ord()`** function that takes a one-character Unicode string and returns the code point value:

```
>>> chr(57344)
'\ue000'
>>> ord('\ue000')
57344
```

Convertendo para Bytes

The opposite method of **`bytes.decode()`** is **`str.encode()`**, which returns a **`bytes`** representation of the Unicode string, encoded in the requested *encoding*.

The *errors* parameter is the same as the parameter of the **`decode()`** method but supports a few more possible handlers. As well as `'strict'`, `'ignore'`, and `'replace'` (which in this case inserts a question mark instead of the unencodable character), there is also `'xmlcharrefreplace'` (inserts an XML character reference), `backslashreplace` (inserts a `\uNNNN` escape sequence) and `namereplace` (inserts a `\N{...}` escape sequence).

The following example shows the different results:

```
>>> u = chr(40960) + 'abcd' + chr(1972)
>>> u.encode('utf-8')
b'\xea\x80\x80abcd\xde\xb4'
>>> u.encode('ascii')
Traceback (most recent call last):
    ...
UnicodeEncodeError: 'ascii' codec can't encode character '\ua00
  position 0: ordinal not in range(128)
>>> u.encode('ascii', 'ignore')
```

```
b'abcd'
>>> u.encode('ascii', 'replace')
b'?abcd?'
>>> u.encode('ascii', 'xmlcharrefreplace')
b'&#40960;abcd&#1972;'
>>> u.encode('ascii', 'backslashreplace')
b'\\ua000abcd\\u07b4'
>>> u.encode('ascii', 'namereplace')
b'\\N{YI SYLLABLE IT}abcd\\u07b4'
```

The low-level routines for registering and accessing the available encodings are found in the **codecs** module. Implementing new encodings also requires understanding the **codecs** module. However, the encoding and decoding functions returned by this module are usually more low-level than is comfortable, and writing new encodings is a specialized task, so the module won't be covered in this HOWTO.

Unicode Literals in Python Source Code

In Python source code, specific Unicode code points can be written using the `\u` escape sequence, which is followed by four hex digits giving the code point. The `\U` escape sequence is similar, but expects eight hex digits, not four:

```
>>> s = "a\xac\u1234\u20ac\U00008000"
... #     ^^^^ two-digit hex escape
... #         ^^^^^^ four-digit Unicode escape
... #                     ^^^^^^^^^^ eight-digit Unicode escape
>>> [ord(c) for c in s]
[97, 172, 4660, 8364, 32768]
```

Using escape sequences for code points greater than 127 is fine in small doses, but becomes an annoyance if you're using many accented characters, as you would in a program with messages in French or some other accent-using language. You can also assemble strings using the **chr()** built-in function, but this is even more tedious.

Ideally, you'd want to be able to write literals in your language's natural encoding. You could then edit Python source code with your favorite editor

which would display the accented characters naturally, and have the right characters used at runtime.

Python supports writing source code in UTF-8 by default, but you can use almost any encoding if you declare the encoding being used. This is done by including a special comment as either the first or second line of the source file:

```
#!/usr/bin/env python
# -*- coding: latin-1 -*-

u = 'abcdé'
print(ord(u[-1]))
```

The syntax is inspired by Emacs's notation for specifying variables local to a file. Emacs supports many different variables, but Python only supports 'coding'. The `-*-` symbols indicate to Emacs that the comment is special; they have no significance to Python but are a convention. Python looks for `coding: name` or `coding=name` in the comment.

If you don't include such a comment, the default encoding used will be UTF-8 as already mentioned. See also **PEP 263** [https://peps.python.org/pep-0263/] for more information.

Propriedades Unicode

The Unicode specification includes a database of information about code points. For each defined code point, the information includes the character's name, its category, the numeric value if applicable (for characters representing numeric concepts such as the Roman numerals, fractions such as one-third and four-fifths, etc.). There are also display-related properties, such as how to use the code point in bidirectional text.

O programa a seguir exibe alguma informação sobre diversos caracteres e imprime o valor numérico de um caractere em particular:

```
import unicodedata

u = chr(233) + chr(0x0bf2) + chr(3972) + chr(6000) + chr(13231)
```

```
for i, c in enumerate(u):
    print(i, '%04x' % ord(c), unicodedata.category(c), end=" ")
    print(unicodedata.name(c))

# Get numeric value of second character
print(unicodedata.numeric(u[1]))
```

Quando executado, isso imprime:

```
0 00e9 Ll LATIN SMALL LETTER E WITH ACUTE
1 0bf2 No TAMIL NUMBER ONE THOUSAND
2 0f84 Mn TIBETAN MARK HALANTA
3 1770 Lo TAGBANWA LETTER SA
4 33af So SQUARE RAD OVER S SQUARED
1000.0
```

The category codes are abbreviations describing the nature of the character. These are grouped into categories such as “Letter”, “Number”, “Punctuation”, or “Symbol”, which in turn are broken up into subcategories. To take the codes from the above output, `'Ll'` means ‘Letter, lowercase’, `'No'` means “Number, other”, `'Mn'` is “Mark, nonspacing”, and `'So'` is “Symbol, other”. See the General Category Values section of the Unicode Character Database documentation [https://www.unicode.org/reports/tr44/#General_Category_Values] for a list of category codes.

Comparando Strings

Unicode adds some complication to comparing strings, because the same set of characters can be represented by different sequences of code points. For example, a letter like ‘ê’ can be represented as a single code point U+00EA, or as U+0065 U+0302, which is the code point for ‘e’ followed by a code point for ‘COMBINING CIRCUMFLEX ACCENT’. These will produce the same output when printed, but one is a string of length 1 and the other is of length 2.

One tool for a case-insensitive comparison is the **`casefold()`** string method that converts a string to a case-insensitive form following an algorithm

described by the Unicode Standard. This algorithm has special handling for characters such as the German letter ‘ß’ (code point U+00DF), which becomes the pair of lowercase letters ‘ss’.

```
>>> street = 'Gürzenichstraße'
>>> street.casefold()
'gürzenichstrasse'
```

A second tool is the **unicodedata** module’s **normalize()** function that converts strings to one of several normal forms, where letters followed by a combining character are replaced with single characters. **normalize()** can be used to perform string comparisons that won’t falsely report inequality if two strings use combining characters differently:

```
import unicodedata

def compare_strs(s1, s2):
    def NFD(s):
        return unicodedata.normalize('NFD', s)

    return NFD(s1) == NFD(s2)

single_char = 'ê'
multiple_chars = '\N{LATIN SMALL LETTER E}\N{COMBINING CIRCUMFLE
print('length of first string=', len(single_char))
print('length of second string=', len(multiple_chars))
print(compare_strs(single_char, multiple_chars))
```

When run, this outputs:

```
$ python compare-strs.py
length of first string= 1
length of second string= 2
True
```

The first argument to the **normalize()** function is a string giving the desired normalization form, which can be one of ‘NFC’, ‘NFKC’, ‘NFD’, and ‘NFKD’.

The Unicode Standard also specifies how to do caseless comparisons:

```
import unicodedata

def compare_caseless(s1, s2):
    def NFD(s):
        return unicodedata.normalize('NFD', s)

    return NFD(NFD(s1).casefold()) == NFD(NFD(s2).casefold())

# Example usage
single_char = 'ê'
multiple_chars = '\N{LATIN CAPITAL LETTER E}\N{COMBINING CIRCUMF

print(compare_caseless(single_char, multiple_chars))
```

This will print `True`. (Why is **`NFD()`** invoked twice? Because there are a few characters that make **`casefold()`** return a non-normalized string, so the result needs to be normalized again. See section 3.13 of the Unicode Standard for a discussion and an example.)

Expressões Regulares Unicode

The regular expressions supported by the **`re`** module can be provided either as bytes or strings. Some of the special character sequences such as `\d` and `\w` have different meanings depending on whether the pattern is supplied as bytes or a string. For example, `\d` will match the characters `[0-9]` in bytes but in strings will match any character that's in the `'Nd'` category.

The string in this example has the number 57 written in both Thai and Arabic numerals:

```
import re
p = re.compile(r'\d+')

s = "Over \u0e55\u0e57 57 flavours"
m = p.search(s)
print(repr(m.group()))
```

When executed, `\d+` will match the Thai numerals and print them out. If you supply the **`re.ASCII`** flag to **`compile()`**, `\d+` will match the substring

“57” instead.

Similarly, `\w` matches a wide variety of Unicode characters but only `[a-zA-Z0-9_]` in bytes or if **`re.ASCII`** is supplied, and `\s` will match either Unicode whitespace characters or `[ \t\n\r\f\v]`.

Referências

Some good alternative discussions of Python’s Unicode support are:

- Processing Text Files in Python 3 [https://python-notes.curiousefficiency.org/en/latest/python3/text_file_processing.html], by Nick Coghlan.
- Pragmatic Unicode [https://nedbatchelder.com/text/unipain.html], a PyCon 2012 presentation by Ned Batchelder.

The **`str`** type is described in the Python library reference at Tipo sequência de texto — str.

The documentation for the **`unicodedata`** module.

The documentation for the **`codecs`** module.

Marc-André Lemburg gave a presentation titled “Python and Unicode” (PDF slides) [https://downloads.egenix.com/python/Unicode-EPC2002-Talk.pdf] at EuroPython 2002. The slides are an excellent overview of the design of Python 2’s Unicode features (where the Unicode string type is called `unicode` and literals start with `u`).

Reading and Writing Unicode Data

Once you’ve written some code that works with Unicode data, the next problem is input/output. How do you get Unicode strings into your program, and how do you convert Unicode into a form suitable for storage or transmission?

It’s possible that you may not need to do anything depending on your input sources and output destinations; you should check whether the libraries used in your application support Unicode natively. XML parsers often return Unicode data, for example. Many relational databases also support Unicode-valued columns and can return Unicode values from an SQL query.

Unicode data is usually converted to a particular encoding before it gets written to disk or sent over a socket. It’s possible to do all the work yourself: open a file, read an 8-bit bytes object from it, and convert the bytes with `bytes.decode(encoding)`. However, the manual approach is not recommended.

One problem is the multi-byte nature of encodings; one Unicode character can be represented by several bytes. If you want to read the file in arbitrary-sized chunks (say, 1024 or 4096 bytes), you need to write error-handling code to catch the case where only part of the bytes encoding a single Unicode character are read at the end of a chunk. One solution would be to read the entire file into memory and then perform the decoding, but that prevents you from working with files that are extremely large; if you need to read a 2 GiB file, you need 2 GiB of RAM. (More, really, since for at least a moment you’d need to have both the encoded string and its Unicode version in memory.)

The solution would be to use the low-level decoding interface to catch the case of partial coding sequences. The work of implementing this has already been done for you: the built-in **`open()`** function can return a file-like object that assumes the file’s contents are in a specified encoding and accepts Unicode parameters for methods such as **`read()`** and **`write()`**. This works through **`open()`**'s *encoding* and *errors* parameters which are interpreted just like those in **`str.encode()`** and **`bytes.decode()`**.

Reading Unicode from a file is therefore simple:

```
with open('unicode.txt', encoding='utf-8') as f:
    for line in f:
        print(repr(line))
```

It’s also possible to open files in update mode, allowing both reading and writing:

```
with open('test', encoding='utf-8', mode='w+') as f:
    f.write('\u4500 blah blah blah\n')
    f.seek(0)
    print(repr(f.readline()[:1]))
```

The Unicode character `U+FEFF` is used as a byte-order mark (BOM), and is often written as the first character of a file in order to assist with autodetection of the file’s byte ordering. Some encodings, such as UTF-16, expect a BOM to be present at the start of a file; when such an encoding is used, the BOM will be automatically written as the first character and will be silently dropped when the file is read. There are variants of these encodings, such as ‘utf-16-le’ and ‘utf-16-be’ for little-endian and big-endian encodings, that specify one particular byte ordering and don’t skip the BOM.

In some areas, it is also convention to use a “BOM” at the start of UTF-8 encoded files; the name is misleading since UTF-8 is not byte-order dependent. The mark simply announces that the file is encoded in UTF-8. For reading such files, use the ‘utf-8-sig’ codec to automatically skip the mark if present.

Nomes de arquivos Unicode

Most of the operating systems in common use today support filenames that contain arbitrary Unicode characters. Usually this is implemented by converting the Unicode string into some encoding that varies depending on the system. Today Python is converging on using UTF-8: Python on MacOS has used UTF-8 for several versions, and Python 3.6 switched to using UTF-8 on Windows as well. On Unix systems, there will only be a filesystem encoding. if you’ve set the `LANG` or `LC_CTYPE` environment variables; if you haven’t, the default encoding is again UTF-8.

The **`sys.getfilesystemencoding()`** function returns the encoding to use on your current system, in case you want to do the encoding manually, but there’s not much reason to bother. When opening a file for reading or

writing, you can usually just provide the Unicode string as the filename, and it will be automatically converted to the right encoding for you:

```
filename = 'filename\u4500abc'
with open(filename, 'w') as f:
    f.write('blah\n')
```

Functions in the **os** module such as **os.stat()** will also accept Unicode filenames.

The **os.listdir()** function returns filenames, which raises an issue: should it return the Unicode version of filenames, or should it return bytes containing the encoded versions? **os.listdir()** can do both, depending on whether you provided the directory path as bytes or a Unicode string. If you pass a Unicode string as the path, filenames will be decoded using the filesystem's encoding and a list of Unicode strings will be returned, while passing a byte path will return the filenames as bytes. For example, assuming the default filesystem encoding is UTF-8, running the following program:

```
fn = 'filename\u4500abc'
f = open(fn, 'w')
f.close()

import os
print(os.listdir(b'.'))
print(os.listdir('.'))
```

will produce the following output:

```
$ python listdir-test.py
[b'filename\xe4\x94\x80abc', ...]
['filename\u4500abc', ...]
```

The first list contains UTF-8-encoded filenames, and the second list contains the Unicode versions.

Note that on most occasions, you should can just stick with using Unicode with these APIs. The bytes APIs should only be used on systems where

undecodable file names can be present; that's pretty much only Unix systems now.

Tips for Writing Unicode-aware Programs

This section provides some suggestions on writing software that deals with Unicode.

A dica mais importante é:

> Software should only work with Unicode strings internally, decoding the input data as soon as possible and encoding the output only at the end.

If you attempt to write processing functions that accept both Unicode and byte strings, you will find your program vulnerable to bugs wherever you combine the two different kinds of strings. There is no automatic encoding or decoding: if you do e.g. `str + bytes`, a `TypeError` will be raised.

When using data coming from a web browser or some other untrusted source, a common technique is to check for illegal characters in a string before using the string in a generated command line or storing it in a database. If you're doing this, be careful to check the decoded string, not the encoded bytes data; some encodings may have interesting properties, such as not being bijective or not being fully ASCII-compatible. This is especially true if the input data also specifies the encoding, since the attacker can then choose a clever way to hide malicious text in the encoded bytestream.

Converting Between File Encodings

The `StreamRecoder` class can transparently convert between encodings, taking a stream that returns data in encoding #1 and behaving like a stream returning data in encoding #2.

For example, if you have an input file *f* that's in Latin-1, you can wrap it with a `StreamRecoder` to return bytes encoded in UTF-8:

```
new_f = codecs.StreamRecoder(f,
    # en/decoder: used by read() to encode its results and
    # by write() to decode its input.
    codecs.getencoder('utf-8'), codecs.getdecoder('utf-8'),

    # reader/writer: used to read and write to the stream.
    codecs.getreader('latin-1'), codecs.getwriter('latin-1') )
```

Files in an Unknown Encoding

What can you do if you need to make a change to a file, but don't know the file's encoding? If you know the encoding is ASCII-compatible and only want to examine or modify the ASCII parts, you can open the file with the `surrogateescape` error handler:

```
with open(fname, 'r', encoding="ascii", errors="surrogateescape'
    data = f.read()

# make changes to the string 'data'

with open(fname + '.new', 'w',
          encoding="ascii", errors="surrogateescape") as f:
    f.write(data)
```

The `surrogateescape` error handler will decode any non-ASCII bytes as code points in a special range running from U+DC80 to U+DCFF. These code points will then turn back into the same bytes when the `surrogateescape` error handler is used to encode the data and write it back out.

Referências

One section of Mastering Python 3 Input/Output [https://pyvideo.org/video/289/pycon-2010--mastering-python-3-i-o], a PyCon 2010 talk by David Beazley, discusses text processing and binary data handling.

The PDF slides for Marc-André Lemburg's presentation "Writing Unicode-aware Applications in Python" [https://downloads.egenix.com/python/LSM2005-

Developing-Unicode-aware-applications-in-Python.pdf] discuss questions of character encodings as well as how to internationalize and localize an application. These slides cover Python 2.x only.

The Guts of Unicode in Python [https://pyvideo.org/video/1768/the-guts-of-unicode-in-python] is a PyCon 2013 talk by Benjamin Peterson that discusses the internal Unicode representation in Python 3.3.

Reconhecimentos

The initial draft of this document was written by Andrew Kuchling. It has since been revised further by Alexander Belopolsky, Georg Brandl, Andrew Kuchling, and Ezio Melotti.

Thanks to the following people who have noted errors or offered suggestions on this article: Éric Araujo, Nicholas Bastin, Nick Coghlan, Marius Gedminas, Kent Johnson, Ken Krugler, Marc-André Lemburg, Martin von Löwis, Terry J. Reedy, Serhiy Storchaka, Eryk Sun, Chad Whitacre, Graham Wideman.

Como buscar recursos da Internet usando o pacote urllib

Introdução

urllib.request é um modulo Python para buscar URLs (Uniform Resource Locators). Ele oferece uma interface muito simples, na forma da função *urlopen*. Este é capaz de buscar URLs usando uma variedade de diferentes protocolos. Ele também oferece uma interface um pouco mais complexa para lidar com situações comuns - como autenticação básica, cookies, proxies e assim por diante. Estes são fornecidos por objetos chamados handlers (manipuladores) e openers (abridores).

Related Articles

Você também pode achar útil o seguinte artigo na busca de recursos da web com Python:

- Autenticação Básica [https://web.archive.org/web/20201215133350/http://www.voidspace.org.uk/python/articles/authentication.shtml]

  Um tutorial sobre *Autenticação Básica*, com exemplos em Python.

urllib.request suporta o acesso a URLs por meio de vários “esquemas de URL” (identificados pela string antes de `":"` na URL - por exemplo `"ftp"` é o esquema de URL em `"ftp://python.org/"`) usando o protocolo de rede associado a ele (como FTP e HTTP). Este tutorial foca no caso mais comum, HTTP.

Para situações simples “urlopen” é muito fácil de usar. Mas assim que você se depara com erros ou casos não triviais ao abrir URLs HTTP, você vai precisar entender um pouco mais do HyperText Transfer Protocol. A literatura de referência mais reconhecida e compreensível para o HTTP é **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html]. Ela é um documento técnico e não foi feita para ser fácil de ler. Este documento busca ilustrar o

uso de *urllib* com detalhes suficientes sobre HTTP para te permitir seguir adiante. Ele não tem a intenção de substituir a documentação do `urllib.request`, mas é suplementar a ela.

Acessando URLs

O modo mais simples de usar urllib.request é o seguinte:

```
import urllib.request
with urllib.request.urlopen('http://python.org/') as response:
   html = response.read()
```

Se você deseja obter um recurso via URL e guardá-lo em uma localização temporária, você pode fazê-lo com as funções `shutil.copyfileobj()` e `tempfile.NamedTemporaryFile()`:

```
import shutil
import tempfile
import urllib.request

with urllib.request.urlopen('http://python.org/') as response:
    with tempfile.NamedTemporaryFile(delete=False) as tmp_file:
        shutil.copyfileobj(response, tmp_file)

with open(tmp_file.name) as html:
    pass
```

Muitos usos de urllib são simples assim (repare que ao invés de uma URL ‘http:’ nós poderíamos ter usado uma string URL começando com ‘ftp:’, ‘file:’, etc.).. No entanto, o propósito deste tutorial é explicar casos mais complicados, concentrando em HTTP.

HTTP é baseado em solicitações (requests) e respostas (responses) - o cliente faz solicitações e os servidores mandam respostas. urllib.request espelha isto com um objeto `Request` que representa a solicitação HTTP que você está fazendo. Na sua forma mais simples, você cria um objeto Request que especifica a URL que você quer acessar. Chamar `urlopen` com este objeto Request retorna um objeto de resposta para a URL solicitada. Essa

resposta é um objeto arquivo ou similar, o que significa que você pode, por exemplo, chamar `.read()` na resposta:

```
import urllib.request

req = urllib.request.Request('http://python.org/')
with urllib.request.urlopen(req) as response:
   the_page = response.read()
```

Note que urllib.request usa a mesma interface Request para tratar todos os esquemas URL. Por exemplo, você pode fazer uma solicitação FTP da seguinte forma:

```
req = urllib.request.Request('ftp://example.com/')
```

No caso do HTTP, há duas coisas extras que os objetos Request permitem que você faça: primeiro, você pode passar dados a serem enviados ao servidor. Segundo, você pode passar informações extras ("metadados") *sobre* os dados ou sobre a própria solicitação para o servidor — essas informações são enviadas como "cabeçalhos" HTTP. Vamos analisar cada um deles separadamente.

Dados

Às vezes, você deseja enviar dados para uma URL (geralmente a URL se refere a um script CGI (Common Gateway Interface) ou outra aplicação web). Com HTTP, isso geralmente é feito usando o que é conhecido como uma solicitação **POST**. Isso geralmente é o que seu navegador faz quando você envia um formulário HTML preenchido na web. Nem todos os POSTs precisam vir de formulários: você pode usar um POST para transmitir dados arbitrários para sua própria aplicação. No caso comum de formulários HTML, os dados precisam ser codificados de forma padrão e, em seguida, passados para o objeto Request como o argumento `data`. A codificação é feita usando uma função da biblioteca **`urllib.parse`**.

```
import urllib.parse
import urllib.request

url = 'http://www.someserver.com/cgi-bin/register.cgi'
```

```
values = {'name' : 'Michael Foord',
          'location' : 'Northampton',
          'language' : 'Python' }

data = urllib.parse.urlencode(values)
data = data.encode('ascii') # data should be bytes
req = urllib.request.Request(url, data)
with urllib.request.urlopen(req) as response:
   the_page = response.read()
```

Observe que outras codificações às vezes são necessárias (por exemplo, para envio de arquivos de formulários HTML - consulte HTML Specification, Form Submission [https://www.w3.org/TR/REC-html40/interact/forms.html#h-17.13] para mais detalhes).

Se você não passar o argumento `data`, o urllib usará uma requisição **GET**. Uma diferença entre requisições GET e POST é que as requisições POST frequentemente têm "efeitos colaterais": elas alteram o estado do sistema de alguma forma (por exemplo, ao fazer um pedido ao site para que cem libras de spam enlatado sejam entregues em sua porta). Embora o padrão HTTP deixe claro que os POSTs devem *sempre* causar efeitos colaterais, e as requisições GET *nunca* causar efeitos colaterais, nada impede que uma requisição GET tenha efeitos colaterais, nem que uma requisição POST não tenha efeitos colaterais. Dados também podem ser passados em uma requisição HTTP GET codificando-os na própria URL.

Isso é feito como abaixo:

```
>>> import urllib.request
>>> import urllib.parse
>>> data = {}
>>> data['name'] = 'Somebody Here'
>>> data['location'] = 'Northampton'
>>> data['language'] = 'Python'
>>> url_values = urllib.parse.urlencode(data)
>>> print(url_values)  # The order may differ from below.
name=Somebody+Here&language=Python&location=Northampton
>>> url = 'http://www.example.com/example.cgi'
>>> full_url = url + '?' + url_values
>>> data = urllib.request.urlopen(full_url)
```

Observe que o URL completo é criado adicionando um `?` ao URL, seguido pelos valores codificados.

Cabeçalhos

Discutiremos aqui um cabeçalho HTTP específico para ilustrar como adicionar cabeçalhos à sua solicitação HTTP.

Alguns sites [[1]] não gostam de ser navegados por programas ou enviam versões diferentes para navegadores diferentes [[2]]. Por padrão, urllib se identifica como `Python-urllib/x.y` (onde `x` e `y` são os números de versão principal e secundária da versão do Python, por exemplo, `Python-urllib/2.5`), o que pode confundir o site ou simplesmente não funcionar. A forma como um navegador se identifica é através do cabeçalho `User-Agent` [[3]]. Ao criar um objeto Request, você pode passar um dicionário de cabeçalhos. O exemplo a seguir faz a mesma solicitação acima, mas se identifica como uma versão do Internet Explorer [[4]].

```
import urllib.parse
import urllib.request

url = 'http://www.someserver.com/cgi-bin/register.cgi'
user_agent = 'Mozilla/5.0 (Windows NT 6.1; Win64; x64)'
values = {'name': 'Michael Foord',
          'location': 'Northampton',
          'language': 'Python' }
headers = {'User-Agent': user_agent}

data = urllib.parse.urlencode(values)
data = data.encode('ascii')
req = urllib.request.Request(url, data, headers)
with urllib.request.urlopen(req) as response:
   the_page = response.read()
```

A resposta também possui dois métodos úteis. Veja a seção sobre info e geturl, que vem depois de analisarmos o que acontece quando as coisas dão errado.

Tratamento de exceções

urlopen levanta `URLError` quando não consegue tratar uma resposta (embora, como de costume com APIs Python, exceções embutidas como `ValueError`, `TypeError` etc. também possam ser levantadas).

`HTTPError` é a subclasse de `URLError` levantada no caso específico de URLs HTTP.

As classes de exceção são exportadas do módulo `urllib.error`.

URLError

Frequentemente, URLError é levantada porque não há conexão de rede (nenhuma rota para o servidor especificado) ou o servidor especificado não existe. Nesse caso, a exceção gerada terá um atributo “reason”, que é uma tupla contendo um código de erro e uma mensagem de erro em texto.

Por exemplo

```
>>> req = urllib.request.Request('http://www.pretend_server.org'
>>> try: urllib.request.urlopen(req)
... except urllib.error.URLError as e:
...     print(e.reason)
...
(4, 'getaddrinfo failed')
```

HTTPError

Cada resposta HTTP do servidor contém um “código de status” numérico. Às vezes, o código de status indica que o servidor não consegue atender à solicitação. Os manipuladores padrão lidarão com algumas dessas respostas para você (por exemplo, se a resposta for um “redirecionamento” que solicita que o cliente busque o documento de uma URL diferente, o urllib cuidará disso para você). Para aquelas que ele não consegue tratar, o urlopen lançará um `HTTPError`. Erros típicos incluem ‘404’ (página não encontrada), ‘403’ (solicitação proibida) e ‘401’ (autenticação necessária).

Veja a seção 10 de **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html] para uma referência sobre todos os códigos de erro HTTP.

A instância **HTTPError** levantada terá um atributo inteiro ‘code’, que corresponde ao erro enviado pelo servidor.

Códigos de erro

Como os tratadores padrão controlam redirecionamentos (códigos no intervalo 300) e códigos no intervalo 100-299 indicam sucesso, normalmente você verá apenas códigos de erro no intervalo 400-599.

http.server.BaseHTTPRequestHandler.responses é um dicionário útil de códigos de resposta que mostra todos os códigos de resposta usados por **RFC 2616** [https://datatracker.ietf.org/doc/html/rfc2616.html]. Um trecho do dicionário é mostrado abaixo

```
responses = {
    ...
    <HTTPStatus.OK: 200>: ('OK', 'Request fulfilled, document fo
    ...
    <HTTPStatus.FORBIDDEN: 403>: ('Forbidden',
                                  'Request forbidden -- authoriz
                                  'not help'),
    <HTTPStatus.NOT_FOUND: 404>: ('Not Found',
                                  'Nothing matches the given URI
    ...
    <HTTPStatus.IM_A_TEAPOT: 418>: ("I'm a Teapot",
                                    'Server refuses to brew coff
                                    'it is a teapot'),
    ...
    <HTTPStatus.SERVICE_UNAVAILABLE: 503>: ('Service Unavailable
                                            'The server cannot p
                                            'request due to a hi
    ...
    }
```

Quando um erro é levantado, o servidor responde retornando um código de erro HTTP *e* uma página de erro. Você pode usar a instância **HTTPError** como resposta na página retornada. Isso significa que, além do atributo code, ela também possui os métodos read, geturl e info, conforme retornados pelo módulo `urllib.response`:

```
>>> req = urllib.request.Request('http://www.python.org/fish.htm
>>> try:
...     urllib.request.urlopen(req)
... except urllib.error.HTTPError as e:
...     print(e.code)
...     print(e.read())
...
404
b'<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN'
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">\n\r
  ...
  <title>Page Not Found</title>\n
  ...
```

Resumindo

Então, se você quiser se preparar para `HTTPError` *ou* `URLError`, existem duas abordagens básicas. Eu prefiro a segunda.

Número 1

```
from urllib.request import Request, urlopen
from urllib.error import URLError, HTTPError
req = Request(someurl)
try:
    response = urlopen(req)
except HTTPError as e:
    print('The server couldn\'t fulfill the request.')
    print('Error code: ', e.code)
except URLError as e:
    print('We failed to reach a server.')
    print('Reason: ', e.reason)
else:
    # tudo certo
```

Nota

O `except HTTPError` *deve* vir primeiro, caso contrário, `except URLError` *também* capturará uma `HTTPError`.

Número 2

```
from urllib.request import Request, urlopen
from urllib.error import URLError
req = Request(someurl)
try:
    response = urlopen(req)
except URLError as e:
    if hasattr(e, 'reason'):
        print('We failed to reach a server.')
        print('Reason: ', e.reason)
    elif hasattr(e, 'code'):
        print('The server couldn\'t fulfill the request.')
        print('Error code: ', e.code)
else:
    # tudo certo
```

info e geturl

A resposta retornada por urlopen (ou a instância **`HTTPError`**) tem dois métodos úteis **`info()`** e **`geturl()`** e é definida no módulo **`urllib.response`**.

- **geturl -** Isso retorna a URL real da página recuperada. Isso é útil porque `urlopen` (ou o objeto de abertura utilizado) pode ter seguido um redirecionamento. A URL da página recuperada pode não ser a mesma que a URL solicitada.
- **info -** Isso retorna um objeto semelhante a um dicionário que descreve a página recuperada, particularmente os cabeçalhos enviados pelo servidor. Atualmente, é uma instância de **`http.client.HTTPMessage`**.

Cabeçalhos típicos incluem ‘Content-length’, ‘Content-type’ e assim por diante. Consulte a “Referência rápida para cabeçalhos HTTP <https://jkorpela.fi/http.html>” para obter uma lista útil de cabeçalhos HTTP com breves explicações sobre seu significado e uso.

Abridores e tratadores

Ao buscar uma URL, você usa um abridor (uma instância do talvez confuso nome `urllib.request.OpenerDirector`). Normalmente, usamos o abridor padrão - via `urlopen` -, mas você pode criar abridores personalizados. Os abridores usam manipuladores. Todo o "trabalho pesado" é feito pelos manipuladores. Cada manipulador sabe como abrir URLs para um esquema de URL específico (http, ftp, etc.) ou como lidar com um aspecto da abertura de URL, por exemplo, redirecionamentos HTTP ou cookies HTTP.

Você vai querer criar abridores se quiser buscar URLs com manipuladores específicos instalados, por exemplo, para obter um abridor que manipule cookies ou para obter um abridor que não manipule redirecionamentos.

Para criar um abridor, instancie um `OpenerDirector` e então chame `.add_handler(some_handler_instance)` repetidamente.

Como alternativa, você pode usar `build_opener`, que é uma função conveniente para criar objetos de abertura com uma única chamada de função. `build_opener` adiciona vários tratadores por padrão, mas fornece uma maneira rápida de adicionar mais e/ou substituir os tratadores padrão.

Outros tipos de manipuladores que você pode querer podem lidar com proxies, autenticação e outras situações comuns, mas um pouco especializadas.

`install_opener` pode ser usado para tornar um objeto `opener` o abridor padrão (global). Isso significa que chamadas para `urlopen` usarão o abridor que você instalou.

Objetos abridores têm um método `open`, que pode ser chamado diretamente para buscar URLs da mesma forma que a função `urlopen`: não há necessidade de chamar `install_opener`, exceto por conveniência.

Autenticação básica

Para ilustrar a criação e instalação de um manipulador, usaremos o `HTTPBasicAuthHandler`. Para uma discussão mais detalhada sobre este assunto — incluindo uma explicação de como funciona a autenticação

básica — consulte este tutorial de autenticação básica [https://web.archive.org/web/20201215133350/http://www.voidspace.org.uk/python/articles/authentication.shtml].

Quando a autenticação é necessária, o servidor envia um cabeçalho (e o código de erro 401) solicitando autenticação. Isso especifica o esquema de autenticação e um "domínio". O cabeçalho se parece com: `WWW-Authenticate: SCHEME realm="REALM"`.

Por exemplo:

```
WWW-Authenticate: Basic realm="cPanel Users"
```

O cliente deve então tentar a solicitação novamente com o nome e a senha apropriados para o domínio incluídos como cabeçalho na solicitação. Isso é "autenticação básica". Para simplificar esse processo, podemos criar uma instância de `HTTPBasicAuthHandler` e um opener para usar esse manipulador.

O `HTTPBasicAuthHandler` usa um objeto chamado gerenciador de senhas para manipular o mapeamento de URLs e domínios para senhas e nomes de usuário. Se você souber qual é o domínio (a partir do cabeçalho de autenticação enviado pelo servidor), poderá usar um `HTTPPasswordMgr`. Frequentemente, não importa qual seja o domínio. Nesse caso, é conveniente usar `HTTPPasswordMgrWithDefaultRealm`. Isso permite que você especifique um nome de usuário e uma senha padrão para uma URL. Isso será fornecido caso você não forneça uma combinação alternativa para um domínio específico. Indicamos isso fornecendo `None` como argumento de domínio para o método `add_password`.

A URL de nível superior é a primeira URL que requer autenticação. URLs "mais profundas" que a URL que você passa para .add_password() também corresponderão.

```
# cria um gerenciador de senhas
password_mgr = urllib.request.HTTPPasswordMgrWithDefaultRealm()

# Adiciona o nome de usuário e senha.
# Se soubéssemos o realm, poderíamos usá-lo em vez de None.
```

```
top_level_url = "http://example.com/foo/"
password_mgr.add_password(None, top_level_url, username, passwor

handler = urllib.request.HTTPBasicAuthHandler(password_mgr)

# cria um "opener" (OpenerDirector instance)
opener = urllib.request.build_opener(handler)

# usa o opener para obter uma URL
opener.open(a_url)

# Instala o opener.
# Agora, todas as chamadas a urllib.request.urlopen usam nosso c
urllib.request.install_opener(opener)
```

Nota

No exemplo acima, fornecemos apenas nosso `HTTPBasicAuthHandler` para `build_opener`. Por padrão, os “openers” possuem os manipuladores para situações normais: `ProxyHandler` (se uma configuração de proxy, como uma variável de ambiente **`http_proxy`**, estiver definida), `UnknownHandler`, `HTTPHandler`, `HTTPDefaultErrorHandler`, `HTTPRedirectHandler`, `FTPHandler`, `FileHandler`, `DataHandler`, `HTTPErrorProcessor`.

`top_level_url` é, na verdade, *ou* uma URL completa (incluindo o componente do esquema ‘http:’, o nome do host e, opcionalmente, o número da porta), por exemplo, `"http://example.com/"`, *ou* uma “autoridade” (ou seja, o nome do host, incluindo, opcionalmente, o número da porta), por exemplo, `"example.com"` ou `"example.com:8080"` (este último exemplo inclui um número de porta). A autoridade, se presente, NÃO deve conter o componente “userinfo” - por exemplo, `"joe:senha@example.com"` não está correto.

Proxies

urllib detectará automaticamente suas configurações de proxy e as utilizará. Isso ocorre por meio do `ProxyHandler`, que faz parte da cadeia de manipuladores normal quando uma configuração de proxy é detectada. Normalmente, isso é bom, mas há ocasiões em que pode não ser útil [[5]]. Uma maneira de fazer isso é configurar nosso próprio `ProxyHandler`, sem proxies definidos. Isso é feito seguindo etapas semelhantes à configuração de um manipulador de autenticação básica:

```
>>> proxy_support = urllib.request.ProxyHandler({})
>>> opener = urllib.request.build_opener(proxy_support)
>>> urllib.request.install_opener(opener)
```

Nota

Atualmente, `urllib.request` *não* oferece suporte à busca de locais `https` por meio de um proxy. No entanto, isso pode ser habilitado estendendo urllib.request, conforme mostrado na receita [[6]].

Nota

`HTTP_PROXY` será ignorado se uma variável `REQUEST_METHOD` estiver definida; veja a documentação em **`getproxies()`**.

Socekts e camadas

O suporte do Python para buscar recursos web é em camadas. urllib usa a biblioteca **`http.client`**, que por sua vez usa a biblioteca de sockets.

A partir do Python 2.3, você pode especificar quanto tempo um soquete deve aguardar por uma resposta antes de atingir o tempo limite. Isso pode ser útil em aplicações que precisam buscar páginas web. Por padrão, o módulo socket *não tem tempo limite* e pode travar. Atualmente, o tempo limite do soquete não é exposto nos níveis http.client ou urllib.request. No entanto, você pode definir o tempo limite padrão globalmente para todos os soquetes usando

```
import socket
import urllib.request

# tempo limite em secungos
timeout = 10
socket.setdefaulttimeout(timeout)

# isso chamada a urllib.request.urlopen agora usa o tempo limite
# que nós definidos no módulo socket
req = urllib.request.Request('http://www.voidspace.org.uk')
response = urllib.request.urlopen(req)
```

Notas de rodapé

Este documento foi revisado e revisado por John Lee.

[1]

Google, por exemplo.

[2]

A detecção de navegadores é uma prática muito ruim para o design de sites; construir sites usando padrões web é muito mais sensato. Infelizmente, muitos sites ainda enviam versões diferentes para navegadores diferentes.

[3]

O user agent para MSIE 6 é *'Mozilla/4.0 (compatível; MSIE 6.0; Windows NT 5.1; SV1; .NET CLR 1.1.4322)'*

[4]

Para obter detalhes sobre mais cabeçalhos de solicitação HTTP, consulte Referência rápida para cabeçalhos HTTP.

[5]

No meu caso, preciso usar um proxy para acessar a internet no trabalho. Se você tentar buscar URLs *localhost* por meio desse proxy, ele as bloqueia. O IE está configurado para usar o proxy, que o urllib detecta. Para testar scripts com um servidor localhost, preciso impedir que o urllib use o proxy.

[6]

Abridor urllib para proxy SSL (método CONNECT): Receita do livro de receitas ASPN [https://code.activestate.com/recipes/456195-urrlib2-opener-for-ssl-proxy-connect-method/].

Uma introdução ao módulo ipaddress

Visão Geral

Este documento tem como objetivo prover uma suave introdução ao módulo `ipaddress`. Ele é direcionado primeiramente para usuários que ainda não são familiarizados com a terminologia de rede IP, mas também pode ser útil para engenheiros de rede que querem uma visão geral de como `ipaddress` representa os conceitos de endereçamento de rede IP.

Criando objetos de Endereço/Rede/Interface

Como `ipaddress` é um módulo para inspecionar e manipular endereços IP, a primeira coisa que você deseja fazer é criar alguns objetos. Você pode usar `ipaddress` para criar objetos a partir de strings e inteiros.

Uma observação sobre versões do IP

Para leitores que não estão particularmente familiarizados com o endereçamento IP, é importante saber que o Protocolo de Internet (IP) está atualmente em processo de mudança da versão 4 do protocolo para a versão 6. Esta transição está ocorrendo em grande parte porque a versão 4 do protocolo não fornece endereços suficientes para atender às necessidades do mundo inteiro, principalmente devido ao crescente número de dispositivos com conexões diretas à Internet.

Explicar os detalhes das diferenças entre as duas versões do protocolo está além do escopo desta introdução, mas os leitores precisam pelo menos estar cientes de que essas duas versões existem, e às vezes será necessário forçar o uso de uma versão ou do outro.

Endereços IP de host

Os endereços, muitas vezes chamados de “endereços de host”, são a unidade mais básica ao trabalhar com endereçamento IP. A maneira mais simples de criar endereços é usar a função de fábrica `ipaddress.ip_address()`, que determina automaticamente se deve ser criado um endereço IPv4 ou IPv6 com base no valor passado:

```
>>> ipaddress.ip_address('192.0.2.1')
IPv4Address('192.0.2.1')
>>> ipaddress.ip_address('2001:DB8::1')
IPv6Address('2001:db8::1')
```

Os endereços também podem ser criados diretamente a partir de números inteiros. Os valores que cabem em 32 bits são considerados endereços IPv4:

```
>>> ipaddress.ip_address(3221225985)
IPv4Address('192.0.2.1')
>>> ipaddress.ip_address(42540766411282592856903984951653826561)
IPv6Address('2001:db8::1')
```

Para forçar o uso de endereços IPv4 ou IPv6, as classes relevantes podem ser invocadas diretamente. Isto é particularmente útil para forçar a criação de endereços IPv6 para números inteiros pequenos:

```
>>> ipaddress.ip_address(1)
IPv4Address('0.0.0.1')
>>> ipaddress.IPv4Address(1)
IPv4Address('0.0.0.1')
>>> ipaddress.IPv6Address(1)
IPv6Address('::1')
```

Definindo redes

Endereços de host geralmente são agrupados em redes IP, então `ipaddress` fornece uma maneira de criar, inspecionar e manipular definições de rede. Os objetos de rede IP são construídos a partir de strings que definem o intervalo de endereços de host que fazem parte dessa rede. A forma mais simples para essa informação é um par de “endereço de rede/prefixo de

rede”, onde o prefixo define o número de bits iniciais que são comparados para determinar se um endereço faz ou não parte da rede e o endereço de rede define o valor esperado daqueles bits.

Quanto aos endereços, é fornecida uma função de fábrica que determina automaticamente a versão correta do IP:

```
>>> ipaddress.ip_network('192.0.2.0/24')
IPv4Network('192.0.2.0/24')
>>> ipaddress.ip_network('2001:db8::0/96')
IPv6Network('2001:db8::/96')
```

Os objetos de rede não podem ter nenhum bit de host definido. O efeito prático disso é que `192.0.2.1/24` não descreve uma rede. Tais definições são chamadas de objetos interface, uma vez que a notação “ip-em-uma-rede” é comumente usada para descrever interfaces de rede de um computador em uma determinada rede e é descrita mais detalhadamente na próxima seção.

Por padrão, tentar criar um objeto rede com bits de host definidos resultará na exceção **`ValueError`** ser levantada. Para solicitar que os bits adicionais sejam forçados a zero, o sinalizador `strict=False` pode ser passada para o construtor:

```
>>> ipaddress.ip_network('192.0.2.1/24')
Traceback (most recent call last):
   ...
ValueError: 192.0.2.1/24 has host bits set
>>> ipaddress.ip_network('192.0.2.1/24', strict=False)
IPv4Network('192.0.2.0/24')
```

Embora o formato string ofereça significativamente mais flexibilidade, as redes também podem ser definidas com números inteiros, assim como os endereços de host. Neste caso, considera-se que a rede contém apenas o endereço único identificado pelo número inteiro, portanto o prefixo da rede inclui todo o endereço da rede:

```
>>> ipaddress.ip_network(3221225984)
IPv4Network('192.0.2.0/32')
```

```
>>> ipaddress.ip_network(42540766411282592856903984951653826560)
IPv6Network('2001:db8::/128')
```

Tal como acontece com os endereços, a criação de um tipo específico de rede pode ser forçada chamando diretamente o construtor da classe em vez de usar a função de fábrica.

Interfaces do host

Conforme mencionado acima, se você precisar descrever um endereço em uma rede específica, nem o endereço nem as classes de rede serão suficientes. Notação como `192.0.2.1/24` é comumente usada por engenheiros de rede e pessoas que escrevem ferramentas para firewalls e roteadores como uma abreviação para "o host `192.0.2.1` na rede `192.0.2.0/24`", Consequentemente, **`ipaddress`** fornece um conjunto de classes híbridas que associam um endereço a uma rede específica. A interface para criação é idêntica àquela para definição de objetos de rede, exceto que a parte do endereço não está restrita a ser um endereço de rede.

```
>>> ipaddress.ip_interface('192.0.2.1/24')
IPv4Interface('192.0.2.1/24')
>>> ipaddress.ip_interface('2001:db8::1/96')
IPv6Interface('2001:db8::1/96')
```

Entradas de inteiros são aceitas (como acontece com redes), e o uso de uma versão IP específica pode ser forçado chamando diretamente o construtor relevante.

Inspecionando objetos endereço/rede/interface

Você se deu ao trabalho de criar um objeto IPv(4|6)(Address|Network|Interface) e provavelmente deseja obter informações sobre ele. **`ipaddress`** tenta tornar isso fácil e intuitivo.

Extraindo a versão do IP:

```
>>> addr4 = ipaddress.ip_address('192.0.2.1')
>>> addr6 = ipaddress.ip_address('2001:db8::1')
>>> addr6.version
6
>>> addr4.version
4
```

Obtendo a rede de uma interface:

```
>>> host4 = ipaddress.ip_interface('192.0.2.1/24')
>>> host4.network
IPv4Network('192.0.2.0/24')
>>> host6 = ipaddress.ip_interface('2001:db8::1/96')
>>> host6.network
IPv6Network('2001:db8::/96')
```

Descobrindo quantos endereços individuais estão em uma rede:

```
>>> net4 = ipaddress.ip_network('192.0.2.0/24')
>>> net4.num_addresses
256
>>> net6 = ipaddress.ip_network('2001:db8::0/96')
>>> net6.num_addresses
4294967296
```

Iterando através dos endereços “utilizáveis” em uma rede:

```
>>> net4 = ipaddress.ip_network('192.0.2.0/24')
>>> for x in net4.hosts():
...     print(x)
192.0.2.1
192.0.2.2
192.0.2.3
192.0.2.4
...
192.0.2.252
192.0.2.253
192.0.2.254
```

Obtendo a máscara de rede (ou seja, definir bits correspondentes ao prefixo da rede) ou a máscara de host (qualquer bit que não faça parte da máscara de rede):

```
>>> net4 = ipaddress.ip_network('192.0.2.0/24')
>>> net4.netmask
IPv4Address('255.255.255.0')
>>> net4.hostmask
IPv4Address('0.0.0.255')
>>> net6 = ipaddress.ip_network('2001:db8::0/96')
>>> net6.netmask
IPv6Address('ffff:ffff:ffff:ffff:ffff:ffff::')
>>> net6.hostmask
IPv6Address('::ffff:ffff')
```

Explodindo ou compactando o endereço:

```
>>> addr6.exploded
'2001:0db8:0000:0000:0000:0000:0000:0001'
>>> addr6.compressed
'2001:db8::1'
>>> net6.exploded
'2001:0db8:0000:0000:0000:0000:0000:0000/96'
>>> net6.compressed
'2001:db8::/96'
```

Embora o IPv4 não tenha suporte a explosão ou compactação, os objetos associados ainda fornecem as propriedades relevantes para que o código de versão neutra possa facilmente garantir que o formato mais conciso ou mais detalhado seja usado para endereços IPv6, ao mesmo tempo que manipula corretamente os endereços IPv4.

Redes como listas de Addresses

Às vezes é útil tratar redes como listas. Isso significa que é possível indexá-las assim:

```
>>> net4[1]
IPv4Address('192.0.2.1')
>>> net4[-1]
IPv4Address('192.0.2.255')
>>> net6[1]
IPv6Address('2001:db8::1')
```

```
>>> net6[-1]
IPv6Address('2001:db8::ffff:ffff')
```

Isso também significa que os objetos de rede se prestam ao uso da sintaxe de teste de pertinência de lista como esta:

```
if endereço in rede:
    # faz algo
```

O teste de contenção é feito de forma eficiente com base no prefixo de rede:

```
>>> addr4 = ipaddress.ip_address('192.0.2.1')
>>> addr4 in ipaddress.ip_network('192.0.2.0/24')
True
>>> addr4 in ipaddress.ip_network('192.0.3.0/24')
False
```

Comparações

`ipaddress` fornece algumas maneiras simples e, esperançosamente, intuitivas de comparar objetos, onde faz sentido:

```
>>> ipaddress.ip_address('192.0.2.1') < ipaddress.ip_address('19
True
```

Uma exceção `TypeError` é levantada se você tentar comparar objetos de versões ou tipos diferentes.

Usando endereços IP com outros módulos

Outros módulos que usam endereços IP (como `socket`) geralmente não aceitam objetos deste módulo diretamente. Em vez disso, eles devem ser convertidos para um inteiro ou string que o outro módulo aceitará:

```
>>> addr4 = ipaddress.ip_address('192.0.2.1')
>>> str(addr4)
'192.0.2.1'
```

```
>>> int(addr4)
3221225985
```

Obtendo mais detalhes quando a criação da instância falha

Ao criar objetos de endereço/rede/interface usando as funções de fábrica agnósticas de versão, quaisquer erros serão relatados como `ValueError` com uma mensagem de erro genérica que simplesmente diz que o valor passado não foi reconhecido como um objeto daquele tipo. A falta de um erro específico é porque é necessário saber se o valor *supõe* ser IPv4 ou IPv6 para fornecer mais detalhes sobre o motivo de sua rejeição.

Para dar suporte a casos de uso em que é útil ter acesso a esse detalhe adicional, os construtores de classe individuais na verdade geram as subclasses `ValueError` `ipaddress.AddressValueError` e `ipaddress.NetmaskValueError` para indicar exatamente qual parte da definição falhou ao ser analisada corretamente.

As mensagens de erro são significativamente mais detalhadas ao usar os construtores de classe diretamente. Por exemplo:

```
>>> ipaddress.ip_address("192.168.0.256")
Traceback (most recent call last):
  ...
ValueError: '192.168.0.256' does not appear to be an IPv4 or IPv
>>> ipaddress.IPv4Address("192.168.0.256")
Traceback (most recent call last):
  ...
ipaddress.AddressValueError: Octet 256 (> 255) not permitted in

>>> ipaddress.ip_network("192.168.0.1/64")
Traceback (most recent call last):
  ...
ValueError: '192.168.0.1/64' does not appear to be an IPv4 or IP
>>> ipaddress.IPv4Network("192.168.0.1/64")
Traceback (most recent call last):
  ...
ipaddress.NetmaskValueError: '64' is not a valid netmask
```

Entretanto, ambas as exceções específicas do módulo têm `ValueError` como classe pai, então se você não estiver preocupado com o tipo específico de erro, você ainda pode escrever código como o seguinte:

```
try:
    network = ipaddress.IPv4Network(address)
except ValueError:
    print('endereço/máscara de rede é inválida para IPv4:', addr
```

Instrumentando o CPython com DTrace e SystemTap

DTrace e SystemTap são ferramentas de monitoramento, cada uma fornecendo uma maneira de inspecionar o que os processos em um sistema de computador estão fazendo. Ambas usam linguagens específicas de domínio, permitindo que um usuário escreva scripts que:

- filtrar quais processos devem ser observados
- coletem dados dos processos de interesse
- gerem relatórios sobre os dados

A partir do Python 3.6, o CPython pode ser criado com “marcadores” incorporados, também conhecidos como “sondas” (*probes*), que podem ser observados por um script DTrace ou SystemTap, facilitando o monitoramento do que os processos CPython em um sistema estão fazendo.

Os marcadores DTrace são detalhes de implementação do interpretador CPython. Não há garantias sobre a compatibilidade de sondas entre versões do CPython. Os scripts DTrace podem parar de funcionar ou funcionar incorretamente sem aviso ao alterar as versões do CPython.

Habilitando os marcadores estáticos

O macOS vem com suporte embutido para DTrace. No Linux, para construir o CPython com os marcadores incorporados para SystemTap, as ferramentas de desenvolvimento SystemTap devem ser instaladas.

Em uma máquina Linux, isso pode ser feito via:

```
$ yum install systemtap-sdt-devel
```

ou:

```
$ sudo apt-get install systemtap-sdt-dev
```

O CPython deve então ser **configurado com a opção --with-dtrace**:

```
checking for --with-dtrace... yes
```

No macOS, você pode listar as sondas DTrace disponíveis executando um processo Python em segundo plano e listando todas as sondas disponibilizadas pelo provedor Python:

```
$ python3.6 -q &
$ sudo dtrace -l -P python$!  # or: dtrace -l -m python3.6
```

```
   ID   PROVIDER            MODULE                          FUNCTION NAME
29564 python18035        python3.6          _PyEval_EvalFrameDefault function-e
29565 python18035        python3.6             dtrace_function_entry function-e
29566 python18035        python3.6          _PyEval_EvalFrameDefault function-r
29567 python18035        python3.6            dtrace_function_return function-r
29568 python18035        python3.6                           collect gc-done
29569 python18035        python3.6                           collect gc-start
29570 python18035        python3.6          _PyEval_EvalFrameDefault line
29571 python18035        python3.6                 maybe_dtrace_line line
```

No Linux, você pode verificar se os marcadores estáticos do SystemTap estão presentes no binário compilado, observando se ele contém uma seção ".note.stapsdt".

```
$ readelf -S ./python | grep .note.stapsdt
[30] .note.stapsdt        NOTE         0000000000000000 00308d78
```

Se você construiu o Python como uma biblioteca compartilhada (com a opção de configuração `--enable-shared`), você precisa procurar dentro da biblioteca compartilhada. Por exemplo:

```
$ readelf -S libpython3.3dm.so.1.0 | grep .note.stapsdt
[29] .note.stapsdt        NOTE         0000000000000000 00365b68
```

Um readelf moderno o suficiente pode exibir os metadados:

```
$ readelf -n ./python

Displaying notes found at file offset 0x00000254 with length 0x00000020:
    Owner                 Data size          Description
    GNU                  0x00000010          NT_GNU_ABI_TAG (ABI version tag)
        OS: Linux, ABI: 2.6.32

Displaying notes found at file offset 0x00000274 with length 0x00000024:
    Owner                 Data size          Description
    GNU                  0x00000014          NT_GNU_BUILD_ID (unique build ID b
        Build ID: df924a2b08a7e89f6e11251d4602022977af2670

Displaying notes found at file offset 0x002d6c30 with length 0x00000144:
    Owner                 Data size          Description
    stapsdt              0x00000031          NT_STAPSDT (SystemTap probe descri
        Provider: python
        Name: gc__start
        Location: 0x00000000004371c3, Base: 0x0000000000630ce2, Semaphore: 0x00
        Arguments: -4@%ebx
    stapsdt              0x00000030          NT_STAPSDT (SystemTap probe descri
        Provider: python
        Name: gc__done
        Location: 0x00000000004374e1, Base: 0x0000000000630ce2, Semaphore: 0x00
        Arguments: -8@%rax
    stapsdt              0x00000045          NT_STAPSDT (SystemTap probe descri
        Provider: python
```

```
        Name: function__entry
        Location: 0x000000000053db6c, Base: 0x0000000000630ce2, Semaphore: 0x00
        Arguments: 8@%rbp 8@%r12 -4@%eax
    stapsdt              0x00000046          NT_STAPSDT (SystemTap probe descri
        Provider: python
        Name: function__return
        Location: 0x000000000053dba8, Base: 0x0000000000630ce2, Semaphore: 0x00
        Arguments: 8@%rbp 8@%r12 -4@%eax
```

Os metadados acima contêm informações sobre o SystemTap descrevendo como ele pode corrigir instruções de código de máquina estrategicamente posicionadas para habilitar os ganchos de rastreamento usados por um script do SystemTap.

Sondas estáticas do DTtrace

O script DTrace de exemplo a seguir pode ser usado para mostrar a hierarquia de chamada/retorno de um script Python, rastreando apenas dentro da invocação de uma função chamada “start”. Em outras palavras, invocações de função em tempo de importação não serão listadas:

```
self int indent;

python$target:::function-entry
/copyinstr(arg1) == "start"/
{
        self->trace = 1;
}

python$target:::function-entry
/self->trace/
{
        printf("%d\t%*s:", timestamp, 15, probename);
        printf("%*s", self->indent, "");
        printf("%s:%s:%d\n", basename(copyinstr(arg0)), copyinstr(arg1), arg2);
        self->indent++;
}

python$target:::function-return
/self->trace/
{
        self->indent--;
        printf("%d\t%*s:", timestamp, 15, probename);
        printf("%*s", self->indent, "");
        printf("%s:%s:%d\n", basename(copyinstr(arg0)), copyinstr(arg1), arg2);
}

python$target:::function-return
/copyinstr(arg1) == "start"/
{
```

```
        self->trace = 0;
}
```

Pode ser invocado assim:

```
$ sudo dtrace -q -s call_stack.d -c "python3.6 script.py"
```

O resultado deve ser algo assim:

```
156641360502280  function-entry:call_stack.py:start:23
156641360518804  function-entry: call_stack.py:function_1:1
156641360532797  function-entry:  call_stack.py:function_3:9
156641360546807 function-return:  call_stack.py:function_3:10
156641360563367 function-return: call_stack.py:function_1:2
156641360578365  function-entry: call_stack.py:function_2:5
156641360591757  function-entry:  call_stack.py:function_1:1
156641360605556  function-entry:   call_stack.py:function_3:9
156641360617482 function-return:   call_stack.py:function_3:10
156641360629814 function-return:  call_stack.py:function_1:2
156641360642285 function-return: call_stack.py:function_2:6
156641360656770  function-entry: call_stack.py:function_3:9
156641360669707 function-return: call_stack.py:function_3:10
156641360687853  function-entry: call_stack.py:function_4:13
156641360700719 function-return: call_stack.py:function_4:14
156641360719640  function-entry: call_stack.py:function_5:18
156641360732567 function-return: call_stack.py:function_5:21
156641360747370 function-return:call_stack.py:start:28
```

Marcadores estáticos do SystemTap

A maneira de baixo nível de usar a integração do SystemTap é usar os marcadores estáticos diretamente. Isso requer que você declare explicitamente o arquivo binário que os contém.

Por exemplo, este script SystemTap pode ser usado para mostrar a hierarquia de chamada/retorno de um script Python:

```
probe process("python").mark("function__entry") {
     filename = user_string($arg1);
     funcname = user_string($arg2);
     lineno = $arg3;

     printf("%s => %s in %s:%d\\n",
            thread_indent(1), funcname, filename, lineno);
}

probe process("python").mark("function__return") {
    filename = user_string($arg1);
    funcname = user_string($arg2);
    lineno = $arg3;
```

```
    printf("%s <= %s in %s:%d\\n",
           thread_indent(-1), funcname, filename, lineno);
}
```

Pode ser invocado assim:

```
$ stap \
  show-call-hierarchy.stp \
  -c "./python test.py"
```

O resultado deve ser algo assim:

```
11408 python(8274):        => __contains__ in Lib/_abcoll.py:362
11414 python(8274):         => __getitem__ in Lib/os.py:425
11418 python(8274):          => encode in Lib/os.py:490
11424 python(8274):          <= encode in Lib/os.py:493
11428 python(8274):         <= __getitem__ in Lib/os.py:426
11433 python(8274):        <= __contains__ in Lib/_abcoll.py:366
```

sendo as colunas:

- tempo em microssegundos desde o início do script
- nome do executável
- PID do processo

e o restante indica a hierarquia de chamada/retorno conforme o script é executado.

Para uma construção com `--enable-shared` do CPython, os marcadores estão contidos na biblioteca compartilhada libpython, e o caminho pontilhado da sonda precisa refletir isso. Por exemplo, esta linha do exemplo acima:

```
probe process("python").mark("function__entry") {
```

deve ler-se em vez disso:

```
probe process("python").library("libpython3.6dm.so.1.0").mark("function__entry"
```

(presumindo uma construção de depuração do CPython 3.6)

Marcadores estáticos disponíveis

function__entry(str filename, str funcname, int lineno)

Este marcador indica que a execução de uma função Python começou. Ele é acionado somente para funções Python puro (bytecode).

O nome do arquivo, o nome da função e o número da linha são fornecidos de volta ao script de rastreamento como argumentos posicionais, que devem ser acessados usando `$arg1`, `$arg2`, `$arg3`:

- `$arg1` : nome de arquivo como `(const char *)`, acessível usando `user_string($arg1)`
- `$arg2` : nome da função como `(const char *)`, acessível usando `user_string($arg2)`
- `$arg3` : número da linha como `int`

function__return(str filename, str funcname, int lineno)

Este marcador é o inverso de **`function__entry()`**, e indica que a execução de uma função Python terminou (seja via `return`, ou via uma exceção). Ele é acionado somente para funções Python puro (bytecode).

Os argumentos são os mesmos de **`function__entry()`**

line(str filename, str funcname, int lineno)

Este marcador indica que uma linha Python está prestes a ser executada. É o equivalente ao rastreamento linha por linha com um perfilador Python. Ele não é acionado dentro de funções C.

Os argumentos são os mesmos de **`function__entry()`**.

gc__start(int generation)

Dispara quando o interpretador Python inicia um ciclo de coleta de lixo. `arg0` é a geração a ser percurrida, como **`gc.collect()`**.

gc__done(long collected)

Dispara quando o interpretador Python termina um ciclo de coleta de lixo. `arg0` é o número de objetos coletados.

import__find__load__start(str modulename)

Dispara antes de **`importlib`** tentar encontrar e carregar o módulo. `arg0` é o nome do módulo.

Adicionado na versão 3.7.

import__find__load__done(str modulename, int found)

Dispara após a função find_and_load do **`importlib`** ser chamada. `arg0` é o nome do módulo, `arg1` indica se o módulo foi carregado com sucesso.

Adicionado na versão 3.7.

audit(str event, void *tuple)

Dispara quando **`sys.audit()`** ou **`PySys_Audit()`** é chamada. `arg0` é o nome do evento como string C, `arg1` é um ponteiro **`PyObject`** para um objeto tupla.

Adicionado na versão 3.8.

C Entry Points

To simplify triggering of DTrace markers, Python's C API comes with a number of helper functions that mirror each static marker. On builds of Python without DTrace enabled, these do nothing.

In general, it is not necessary to call these yourself, as Python will do it for you.

C API Function	Static Marker	Notas
void PyDTrace_LINE(const char *arg0, const char *arg1, int arg2)	`line()`	
void PyDTrace_FUNCTION_ENTRY(const char *arg0, const char *arg1, int arg2)	`function__entry()`	
void PyDTrace_FUNCTION_RETURN(const char *arg0, const char *arg1, int arg2)	`function__return()`	
void PyDTrace_GC_START(int arg0)	`gc__start()`	
void PyDTrace_GC_DONE(Py_ssize_t arg0)	`gc__done()`	
void PyDTrace_INSTANCE_NEW_START(int arg0)	`instance__new__start()`	Not used by Python
void PyDTrace_INSTANCE_NEW_DONE(int arg0)	`instance__new__done()`	Not used by Python
void PyDTrace_INSTANCE_DELETE_START(int arg0)	`instance__delete__start()`	Not used by Python

C API Function	Static Marker	Notas
void PyDTrace_INSTANCE_DELETE_DONE(int arg0)	`instance__delete__done()`	Not used by Python
void PyDTrace_IMPORT_FIND_LOAD_START(const char *arg0)	`import__find__load__start()`	
void PyDTrace_IMPORT_FIND_LOAD_DONE(const char *arg0, int arg1)	`import__find__load__done()`	
void PyDTrace_AUDIT(const char *arg0, void *arg1)	`audit()`	

C Probing Checks

int PyDTrace_LINE_ENABLED(void)

int PyDTrace_FUNCTION_ENTRY_ENABLED(void)

int PyDTrace_FUNCTION_RETURN_ENABLED(void)

int PyDTrace_GC_START_ENABLED(void)

int PyDTrace_GC_DONE_ENABLED(void)

int PyDTrace_INSTANCE_NEW_START_ENABLED(void)

int PyDTrace_INSTANCE_NEW_DONE_ENABLED(void)

int PyDTrace_INSTANCE_DELETE_START_ENABLED(void)

int PyDTrace_INSTANCE_DELETE_DONE_ENABLED(void)

int PyDTrace_IMPORT_FIND_LOAD_START_ENABLED(void)

int PyDTrace_IMPORT_FIND_LOAD_DONE_ENABLED(void)

int PyDTrace_AUDIT_ENABLED(void)

All calls to `PyDTrace` functions must be guarded by a call to one of these functions. This allows Python to minimize performance impact when probing is disabled.

On builds without DTrace enabled, these functions do nothing and return `0`.

Tapsets de SystemTap

A maneira mais avançada de usar a integração do SystemTap é usar um “tapset”: o equivalente do SystemTap a uma biblioteca, que oculta alguns dos detalhes de nível inferior dos marcadores estáticos.

Aqui está um arquivo tapset, baseado em uma construção não compartilhada do CPython:

```
/*
   Fornece um envólucro de mais alto nível em volta dos marcadores
   function__entry e function__return:
 \*/
probe python.function.entry = process("python").mark("function__entry")
{
    filename = user_string($arg1);
    funcname = user_string($arg2);
    lineno = $arg3;
    frameptr = $arg4
}
probe python.function.return = process("python").mark("function__return")
{
    filename = user_string($arg1);
    funcname = user_string($arg2);
    lineno = $arg3;
    frameptr = $arg4
}
```

Se este arquivo for instalado no diretório de tapsets do SystemTap (por exemplo, `/usr/share/systemtap/tapset`), estes pontos de sondagem adicionais ficarão disponíveis:

python.function.entry(str filename, str funcname, int lineno, frameptr)

Este ponto de sondagem indica que a execução de uma função Python começou. Ele é acionado somente para funções Python puro (bytecode).

python.function.return(str filename, str funcname, int lineno, frameptr)

Este ponto de sondagem é o inverso de `python.function.return`, e indica que a execução de uma função Python terminou (seja via `return`, ou via uma exceção). Ele é acionado somente para funções Python puro (bytecode).

Exemplos

Este script SystemTap usa o tapset acima para implementar de forma mais limpa o exemplo dado acima de rastreamento da hierarquia de chamada de função Python, sem precisar na diretamente

```
probe python.function.entry
{
  printf("%s => %s in %s:%d\n",
         thread_indent(1), funcname, filename, lineno);
}

probe python.function.return
{
  printf("%s <= %s in %s:%d\n",
         thread_indent(-1), funcname, filename, lineno);
}
```

O script a seguir usa o tapset acima para fornecer uma visão geral de todo o código CPython em execução, mostrando os 20 quadros de bytecode mais frequentemente inseridos, a cada segundo, em todo o sistema:

```
global fn_calls;

probe python.function.entry
{
    fn_calls[pid(), filename, funcname, lineno] += 1;
}

probe timer.ms(1000) {
    printf("\033[2J\033[1;1H") /* clear screen \*/
    printf("%6s %80s %6s %30s %6s\n",
           "PID", "FILENAME", "LINE", "FUNCTION", "CALLS")
    foreach ([pid, filename, funcname, lineno] in fn_calls- limit 20) {
        printf("%6d %80s %6d %30s %6d\n",
            pid, filename, lineno, funcname,
            fn_calls[pid, filename, funcname, lineno]);
    }
    delete fn_calls;
}
```

Python support for the `perf map` compatible profilers

The Linux perf profiler [https://perf.wiki.kernel.org] and samply [https://github.com/mstange/samply] are powerful tools that allow you to profile and obtain information about the performance of your application. Both tools have vibrant ecosystems that aid with the analysis of the data they produce.

The main problem with using these profilers with Python applications is that they only get information about native symbols, that is, the names of functions and procedures written in C. This means that the names and file names of Python functions in your code will not appear in the profiler output.

Since Python 3.12, the interpreter can run in a special mode that allows Python functions to appear in the output of compatible profilers. When this mode is enabled, the interpreter will interpose a small piece of code compiled on the fly before the execution of every Python function and it will teach the profiler the relationship between this piece of code and the associated Python function using perf map files.

> **Nota**
>
> Support for profiling is available on Linux and macOS on select architectures. Perf is available on Linux, while samply can be used on both Linux and macOS. samply support on macOS is available starting from Python 3.15. Check the output of the `configure` build step or check the output of `python -m sysconfig | grep HAVE_PERF_TRAMPOLINE` to see if your system is supported.

Por exemplo, considere o seguinte script:

```
def foo(n):
    result = 0
    for _ in range(n):
        result += 1
    return result

def bar(n):
    foo(n)

def baz(n):
    bar(n)

if __name__ == "__main__":
    baz(1000000)
```

Podemos executar `perf` para obter amostras de rastreamentos de pilha da CPU em 9999 hertz:

```
$ perf record -F 9999 -g -o perf.data python meu_script.py
```

Então podemos usar `perf report` para analisar os dados:

```
$ perf report --stdio -n -g

# Children      Self       Samples  Command     Shared Object
# ........  ........  ............  ..........  ................
#
    91.08%     0.00%             0  python.exe  python.exe
            |
            ---_start
            |
                --90.71%--__libc_start_main
                        Py_BytesMain
                        |
                        |--56.88%--pymain_run_python.constprop.0
                        |          |
                        |          |--56.13%--_PyRun_AnyFileObje
                        |          |          _PyRun_SimpleFileO
                        |          |          |
                        |          |          |--55.02%--run_mod
                        |          |          |          |
                        |          |          |           --54.6
```

```
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
                        |           |           |
...
```

Como você pode ver, as funções Python não são mostradas na saída, apenas `_PyEval_EvalFrameDefault` (a função que avalia o bytecode Python) aparece. Infelizmente isso não é muito útil porque todas as funções Python usam a mesma função C para avaliar bytecode, portanto não podemos saber qual função Python corresponde a qual função de avaliação de bytecode.

Em vez disso, se executarmos o mesmo experimento com o suporte `perf` ativado, obteremos:

```
$ perf report --stdio -n -g

# Children      Self       Samples  Command     Shared Object
# ........  ........  ............  ..........  ................
#
    90.58%     0.36%             1  python.exe  python.exe
            |
            ---_start
            |
                --89.86%--__libc_start_main
                        Py_BytesMain
                        |
                        |--55.43%--pymain_run_python.constprop.0
                        |          |
                        |          |--54.71%--_PyRun_AnyFileObje
```

```
                          |           |           _PyRun_SimpleFileO
                          |           |           |
                          |           |           |--53.62%--run_mod
                          |           |           |           |
                          |           |           |            --53.2
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
                          |           |           |
```

Using the samply profiler

samply is a modern profiler that can be used as an alternative to perf. It uses the same perf map files that Python generates, making it compatible with Python's profiling support. samply is particularly useful on macOS where perf is not available.

To use samply with Python, first install it following the instructions at https://github.com/mstange/samply, then run:

```
$ samply record PYTHONPERFSUPPORT=1 python my_script.py
```

This will open a web interface where you can analyze the profiling data interactively. The advantage of samply is that it provides a modern web-

based interface for analyzing profiling data and works on both Linux and macOS.

On macOS, samply support requires Python 3.15 or later. Also on macOS, samply can't profile signed Python executables due to restrictions by macOS. You can profile with Python binaries that you've compiled yourself, or which are unsigned or locally-signed (such as anything installed by Homebrew). In order to attach to running processes on macOS, run `samply setup` once (and every time samply is updated) to self-sign the samply binary.

Como habilitar o suporte a perfilação com `perf`

O suporte à perfilação com `perf` pode ser habilitado desde o início usando a variável de ambiente **`PYTHONPERFSUPPORT`** ou a opção **`-X perf`**, ou dinamicamente usando **`sys.activate_stack_trampoline()`** e **`sys.deactivate_stack_trampoline()`**.

As funções **`sys`** têm precedência sobre a opção **`-X`**, a opção **`-X`** tem precedência sobre a variável de ambiente.

Exemplo usando a variável de ambiente:

```
$ PYTHONPERFSUPPORT=1 perf record -F 9999 -g -o perf.data pythor
$ perf report -g -i perf.data
```

Exemplo usando a opção **`-X`**:

```
$ perf record -F 9999 -g -o perf.data python -X perf meu_script.
$ perf report -g -i perf.data
```

Exemplo usando as APIs de **`sys`** em `example.py`:

```
import sys

sys.activate_stack_trampoline("perf")
do_profiled_stuff()
```

```
sys.deactivate_stack_trampoline()

non_profiled_stuff()
```

… então:

```
$ perf record -F 9999 -g -o perf.data python ./example.py
$ perf report -g -i perf.data
```

Como obter os melhores resultados

For best results, keep frame pointers enabled. On supported GCC-compatible toolchains, CPython builds itself with `-fno-omit-frame-pointer` and similar flags (see `--without-frame-pointers` for details). These flags allow profilers to unwind using only the frame pointer and not on DWARF debug information. This is because as the code that is interposed to allow `perf` support is dynamically generated it doesn't have any DWARF debugging information available.

Você pode verificar se o seu sistema foi compilado com este sinalizador executando:

```
$ python -m sysconfig | grep 'no-omit-frame-pointer'
```

Se você não vir nenhuma saída, significa que seu interpretador não foi compilado com ponteiros de quadro e, portanto, pode não ser capaz de mostrar funções Python na saída de `perf`.

Como trabalhar sem ponteiros de quadro

Se você estiver trabalhando com um interpretador Python que foi compilado sem ponteiros de quadro, você ainda pode usar o perfilador `perf`, mas a sobrecarga será um pouco maior porque o Python precisa gerar informações de desenrolamento para cada chamada de função Python em tempo real. Além disso, `perf` levará mais tempo para processar os dados porque precisará usar as informações de depuração DWARF para desenrolar a pilha e este é um processo lento.

Para habilitar esse modo, você pode usar a variável de ambiente **`PYTHON_PERF_JIT_SUPPORT`** ou a opção **`-X perf_jit`**, que habilitará o modo JIT para o perfilador `perf`.

Nota

Devido a um bug na ferramenta `perf`, apenas versões `perf` superiores à v6.8 funcionarão com o modo JIT. A correção também foi portada para a versão v6.7.2 da ferramenta.

Note que ao verificar a versão da ferramenta `perf` (o que pode ser feito executando `perf version`) você deve levar em conta que algumas distros adicionam alguns números de versão personalizados, incluindo um caractere `-`. Isso significa que `perf 6.7-3` não é necessariamente `perf 6.7.3`.

Ao usar o modo JIT do perf, você precisa de uma etapa extra antes de poder executar `perf report`. Você precisa chamar o comando `perf inject` para injetar as informações JIT no arquivo `perf.data`.:

```
$ perf record -F 9999 -g -k 1 --call-graph dwarf -o perf.data py
$ perf inject -i perf.data --jit --output perf.jit.data
$ perf report -g -i perf.jit.data
```

ou usando a variável de ambiente:

```
$ PYTHON_PERF_JIT_SUPPORT=1 perf record -F 9999 -g --call-graph
$ perf inject -i perf.data --jit --output perf.jit.data
$ perf report -g -i perf.jit.data
```

O comando `perf inject --jit` lerá `perf.data`, pegará automaticamente o arquivo de dump perf que o Python cria (em `/tmp/perf-$PID.dump`) e, em seguida, criará `perf.jit.data` que mescla todas as informações JIT. Ele também deve criar muitos arquivos `jitted-XXXX-N.so` no diretório atual, que são imagens ELF para todos os trampolins JIT que foram criados pelo Python.

Aviso

Ao usar `--call-graph dwarf`, a ferramenta `perf` fará snapshots da pilha do processo que está sendo perfilado e salvará as informações no arquivo `perf.data`. Por padrão, o tamanho do dump da pilha é de 8192 bytes, mas você pode alterar o tamanho passando-o após uma vírgula, como `--call-graph dwarf,16384`.

O tamanho do dump da pilha é importante porque, se for muito pequeno, o `perf` não conseguirá desfazer o descompasso da pilha e a saída será incompleta. Por outro lado, se for muito grande, o `perf` não conseguirá amostrar o processo com a frequência desejada, pois a sobrecarga será maior.

O tamanho da pilha é particularmente importante ao criar perfis de código Python compilado com níveis de otimização baixos (como `-O0`), pois essas construções tendem a ter quadros de pilha maiores. Se você estiver compilando Python com `-O0` e não estiver vendo funções Python na saída do perfil, tente aumentar o tamanho do despejo de pilha para 65528 bytes (o máximo):

```
$ perf record -F 9999 -g -k 1 --call-graph dwarf,65528 -o perf
```

Diferentes sinalizadores de compilação podem impactar significativamente os tamanhos de pilha:

- Construções com `-O0` geralmente têm quadros de pilha muito maiores do que aquelas com `-O1` ou superior
- Adiciona otimizações (`-O1`, `-O2`, etc.) normalmente reduz o tamanho da pilha
- Os ponteiros de quadro (`-fno-omit-frame-pointer`) geralmente fornecem um desenrolamento de pilha mais confiável

Boas práticas para anotações

Resumo

Este documento foi projetado para encapsular as melhores práticas para trabalhar com anotações. Se você escrever um código Python que examina `__annotations__` nos objetos Python, nós o encorajamos a seguir as diretrizes descritas abaixo.

Este documento está dividido em quatro seções: melhores práticas para acessar as anotações de um objeto no Python na versão 3.10 e versões mais recente, melhores práticas para acessar as anotações de um objeto no Python na versão 3.9 e versões mais antiga, outras melhores práticas para `__annotations__` para qualquer versão do Python e peculiaridades do `__annotations__`.

Note que este documento é específico sobre trabalhar com `__annotations__`, não para o uso *de* anotações. Se você está procurando por informações sobre como usar “type hints” no seu código, por favor veja o módulo `typing`

Acessando o dicionário de anotações de um objeto no Python 3.10 e nas versões mais recentes

O Python 3.10 adicionou uma nova função para a biblioteca padrão: `inspect.get_annotations()`. A partir do Python 3.10 até 3.13, chamar esta função é a melhor prática para acessar o dicionário de anotações de qualquer objeto com suporte a anotações. Esta função pode até “destextualizar” anotações textualizadas para você.

No Python 3.14, há um novo módulo `annotationlib` com funcionalidade para trabalhar com anotações. Isso inclui uma função

`annotationlib.get_annotations()`, que substitui `inspect.get_annotations()`.

Se por alguma razão `inspect.get_annotations()` não for viável para o seu caso de uso, você pode acessar o membro de dados `__annotations__` manualmente. As melhores práticas para isto também mudaram no Python 3.10: a partir do Python 3.10, `o.__annotations__` é garantido *sempre* funcionar em funções, classes e módulos Python. Se você tem certeza que o objeto que você está examinando é um desses três *exatos* objetos, pode simplesmente usar `o.__annotations__` para chegar no dicionário de anotações do objeto.

Contudo, outros tipos de chamáveis – por exemplo, chamáveis criados por `functools.partial()` – podem não ter um atributo `__annotations__` definido. Ao acessar o `__annotations__` de um objeto possivelmente desconhecido, as melhores práticas nas versões de Python 3.10 e mais novas é chamar `getattr()` com três argumentos, por exemplo `getattr(o, '__annotations__', None)`.

Antes de Python 3.10, acessar `__annotations__` numa classe que não define anotações mas que possui uma classe base com anotações retorna o `__annotations__` da classe pai. A partir do Python 3.10, a anotação da classe filha será um dicionário vazio.

Acessando o dicionário de anotações de um objeto no Python 3.9 e nas versões mais antigas

Em Python 3.9 e versões mais antigas, acessar o dicionário de anotações de um objeto é muito mais complicado que em versões mais novas. O problema é uma falha de design em versões mais antigas do Python, especificamente a respeito de anotações de classe.

As melhores práticas para acessar os dicionários de anotações de outros objetos, funções, outros chamáveis e módulos - são as mesmas melhores práticas para 3.10, supondo que você não esteja chamando `inspect.get_annotations()`: você deve usar a função `getattr()` com três argumentos para acessar os atributos do objeto `__annotations__`.

Infelizmente, essas não são as melhores práticas para classes. O problema é que, como `__annotations__` é opcional nas classes, e posto que classes podem herdar atributos das suas classes base, acessar o atributo `__annotations__` de uma classe pode inesperadamente retornar o dicionário de anotações de uma *classe base*. Por exemplo:

```
class Base:
    a: int = 3
    b: str = 'abc'

class Derived(Base):
    pass

print(Derived.__annotations__)
```

Isso mostrará o dicionário de anotações de `Base`, não de `Derived`.

Seu código terá que ter um caminho de código separado se o objeto que você está examinando for uma classe (`isinstance(o, type)`). Nesse caso, a melhor prática depende de um detalhe de implementação do Python 3.9 e anteriores: se uma classe tiver anotações definidas, elas serão armazenadas no dicionário **`__dict__`** da classe. Como a classe pode ou não ter anotações definidas, a melhor prática é chamar o método **`get()`** no dict da classe.

Considerando tudo isso, aqui está um exemplo de código que acessa de forma segura o atributo `__annotations__` em um objeto arbitrário em Python 3.9 e anteriores:

```
if isinstance(o, type):
    ann = o.__dict__.get('__annotations__', None)
else:
    ann = getattr(o, '__annotations__', None)
```

Após executar este código, `ann` deve ser um dicionário ou `None`. Você é encorajado a fazer uma checagem dupla do tipo de `ann` utilizando **`isinstance()`** antes de uma análise mais aprofundada.

Observe que alguns objetos de tipos exóticos ou malformados podem não ter atributo um **`__dict__`**, portanto, para maior segurança, talvez você também queira usar **`getattr()`** para acessar **`__dict__`**.

Recuperando manualmente anotações transformadas em strings

Em situações em que as anotações podem ter sido transformadas em strings, e caso queira avaliar essas strings para produzir os valores de Python que elas representam, é melhor chamar `inspect.get_annotations()` para fazer isto por você.

Se estiver usando Python 3.9 ou anterior, ou por algum motivo não puder usar `inspect.get_annotations()`, será necessário replicar sua lógica. Considere examinar a implementação de **`inspect.get_annotations()`** na versão de Python atual e seguir uma abordagem similar.

Resumindo, caso deseje avaliar uma anotação transformada em string em um objeto arbitrário `o`:

- Se `o` for um módulo, use `o.__dict__` como `globals` ao chamar `eval()`.
- Se `o` for uma classe, use `sys.modules[o.__module__].__dict__` como `globals`, e `dict(vars(o))` como `locals`, quando chamar `eval()`.
- Se `o` for um chamável envolto em um invólucro usando `functools.update_wrapper()`, `@functools.wraps` ou `functools.partial()`, desenvolva-o iterativamente acessando `o.__wrapped__` ou `o.func` conforme apropriado, até encontrar a função raiz.
- Caso `o` seja chamável (mas não uma classe), use `o.__globals__` como globals quando chamar `eval()`.

Contudo, nem todas strings usadas como anotações podem ser convertidas em valores de Python utilizando `eval()`. Valores de string poderiam teoricamente conter qualquer string válida, e na prática existem casos válidos para dicas de tipo que requerem anotar com valores de strings que *não* podem ser executados. Por exemplo:

- Tipos de união **PEP 604** [https://peps.python.org/pep-0604/] usando `|`, antes do suporte a isso ser adicionado em Python 3.10.

- Definições que não são necessárias no ambiente de execução, apenas importadas quando `typing.TYPE_CHECKING` é verdadeiro.

Caso `eval()` tente executar tais valores, levantará uma exceção. Então, quando projetar uma biblioteca API que trabalha com anotações, é recomendado que tente executar os valores das strings apenas quando for solicitado explicitamente.

Melhores práticas para `__annotations__` em qualquer versão Python

- Evite atribuir diretamente ao membro `__annotations__` dos objetos. Deixe que Python gerenciar a configuração de `__annotations__`.
- Se você atribuir diretamente ao membro `__annotations__` do objeto, sempre o defina como um objeto `dict`.
- Evita acessar `__annotations__` diretamente em qualquer objeto. Em vez disso, use `annotationlib.get_annotations()` (Python 3.14+) ou `inspect.get_annotations()` (Python 3.10+).
- Caso acesse diretamente o membro `__annotations__` de um objeto, certifique-se de que ele é um dicionário antes de tentar examinar seu conteúdo.
- Evite modificar o dicionário `__annotations__`.
- Evite deletar o atributo `__annotations__` de um objeto.

Peculiaridades de `__annotations__`

Em todas as versões de Python 3, objetos de função criam preguiçosamente um dicionário de anotações caso nenhuma anotação seja definida nesse objeto. Você pode deletar o atributo `__annotations__` utilizando `del fn.__annotations__`, mas ao acessar `fn.__annotations__` o objeto criará um novo dicionário vazio que será armazenado e retornado como suas anotações. Apagar as anotações de uma função antes que ela tenha criado preguiçosamente seu dicionário de anotações irá produzir um `AttributeError`; ao utilizar `del fn.__annotations__` duas vezes em sequência, é garantido que será produzido um `AttributeError`.

O citado no parágrafo acima também se aplica para classes e objetos de módulo em Python 3.10 e posteriores.

Em todas as versões de Python 3, você pode definir `__annotations__` como `None` em um objeto de função. Contudo, acessar em sequência as anotações nesse objeto utilizando `fn.__annotations__` irá criar preguiçosamente um dicionário vazio como descrito no primeiro parágrafo dessa seção. Isso *não* é válido para módulos e classes, em qualquer versão de Python; esses objetos permitem atribuir `__annotations__` a qualquer valor de Python, e armazenam qualquer valor atribuído.

Se Python transformar sua anotação em string (utilizando `from __future__ import annotations`), e você especificar uma string como anotação, essa string será posta entre aspas. Na prática a anotação receberá aspas *duplas*. Por exemplo:

```
from __future__ import annotations
def foo(a: "str"): pass

print(foo.__annotations__)
```

Isso exibe `{'a': "'str'"}`. Não considere isso como uma “peculiaridade”; foi mencionado aqui simplesmente porque pode ser surpreendente.

Se você usar uma classe com uma metaclasse personalizada e acessar `__annotations__` na classe, poderá observar um comportamento inesperado; consulte a **749** [https://peps.python.org/pep-0749/#pep749-metaclasses] para obter alguns exemplos. Você pode evitar essas peculiaridades usando **`annotationlib.get_annotations()`** no Python 3.14+ ou **`inspect.get_annotations()`** no Python 3.10+. Em versões anteriores do Python, você pode evitar esses bugs acessando as anotações do **`__dict__`** da classe (por exemplo, `cls.__dict__.get('__annotations__', None)`).

Em algumas versões do Python, instâncias de classes podem ter um atributo `__annotations__`. No entanto, essa funcionalidade não é suportada. Se precisar das anotações de uma instância, você pode usar **`type()`** para acessar sua classe (por exemplo, `annotationlib.get_annotations(type(myinstance))` no Python 3.14+).

Isolando módulos de extensão

Resumo

Tradicionalmente, o estado que pertence a módulos de extensão do Python era mantido em variáveis `static` em C, que têm escopo em todo o processo. Este documento descreve problemas de tal estado por processo e apresenta um modo mais seguro: o estado por módulo.

O documento também descreve como migrar para o uso do estado por módulo onde for possível. Essa transição envolve alocar espaço para este estado, potencialmente trocar tipos estáticos por tipos no heap, e—talvez o mais importante—acessar o estado por módulo a partir do código.

Quem deveria ler isto

Este guia é escrito para mantenedores de extensões que usam a API C que desejam torná-las mais seguras para o uso em aplicações onde o Python em si é usado como uma biblioteca.

Informações preliminares

Um *interpretador* é o contexto no qual o código Python é executado. Ele contém estado de configuração (por exemplo o caminho de importação) e de tempo de execução (por exemplo o conjunto de módulos importados).

O Python provê suporte para executar múltiplos interpretadores em um processo. Dois casos devem ser considerados—usuários podem executar interpretadores:

- em sequência, com vários ciclos de `Py_InitializeEx()`/`Py_FinalizeEx()`, e

- em paralelo, gerenciando “sub-interpretadores” usando `Py_NewInterpreter()`/`Py_EndInterpreter()`.

Ambos os casos (e combinações deles) são muito úteis ao embutir o Python em uma biblioteca. Bibliotecas geralmente não devem fazer suposições sobre a aplicação que as usa, o que inclui supor um “interpretador Python principal” para o processo inteiro.

Historicamente, módulos de extensão do Python não lidam bem com este caso de uso. Muitos módulos de extensão (e até alguns módulos da biblioteca padrão) usam estado global *por processo*, uma vez que variáveis `static` do C são extremamente fáceis de se usar. Assim, dados que deveriam ser específicos para um interpretador acabam sendo compartilhados entre interpretadores. A menos que o desenvolvedor da extensão tenha cuidado, é muito fácil criar casos particulares que acabam quebrando o processo quando um módulo é carregado em mais de um interpretador no mesmo processo.

Infelizmente, não é fácil fazer o estado por interpretador. Autores de extensões tendem a não ter múltiplos interpretadores em mente ao desenvolver, e no momento é complicado testar este comportamento.

Entra o estado por módulo

Ao invés de focar no estado por interpretador, a API C do Python está evoluindo para melhor suportar o estado *por módulo*, que é mais granular. Isso significa que dados a nível do C devem estar atrelados a um *objeto de módulo*. Cada interpretador cria o seu próprio objeto de módulo, garantindo assim a separação dos dados. Para testar o isolamento, múltiplos objetos de módulo correspondentes a uma única extensão podem até ser carregados em um único interpretador.

O estado por módulo fornece um modo fácil de pensar sobre tempos de vida e posse de recursos: o módulo de extensão será inicializado quando um objeto de módulo for criado, e limpado quando ele for liberado. Nesse sentido, um módulo funciona como qualquer outro PyObject*; não há

ganchos “de desligamento do interpretador” a serem considerados—ou esquecidos.

Note que há casos de uso para diferentes tipos de “objetos globais”: estado por processo, por interpretador, por thread, ou por tarefa. Com o estado por módulo como padrão, as outras formas ainda são possíveis, mas devem ser tratadas como casos excepcionais: se você precisar delas, você deve tomar cuidados adicionais e escrever mais testes. (Note que este guia não cobre tais medidas.)

Objetos de módulo isolados

O ponto chave de se manter em mente ao desenvolver um módulo de extensão é que vários objetos de módulo podem ser criados a partir de uma única biblioteca compartilhada. Por exemplo:

```
>>> import sys
>>> import binascii
>>> old_binascii = binascii
>>> del sys.modules['binascii']
>>> import binascii  # cria um novo objeto do módulo
>>> old_binascii == binascii
False
```

Como regra geral, os dois módulos devem ser completamente independentes. Todos os objetos e o estado específicos do módulo devem ser encapsulados no objeto de módulo, não devem ser compartilhados com outros objetos de módulo, e devem ser limpados quando o objeto de módulo for desalocado. Uma vez que esta é somente uma regra geral, exceções são possíveis (veja Gerenciando estado global), mas elas necessitam mais cuidado e atenção a casos especiais.

Enquanto alguns módulos funcionariam bem com restrições menos rigorosas, isolar os módulos torna mais fácil definir expectativas claras e diretrizes que dão certo em uma variedade de casos de uso.

Casos particulares surpreendentes

Note que módulos isolados criam alguns casos particulares que podem acabar surpreendendo. O mais notável é que, tipicamente, cada objeto de módulo não vai compartilhar as suas classes e exceções com outros módulos similares. Continuando o exemplo acima, note `old_binascii.Error` e `binascii.Error` são objetos separados. No código a seguir, a exceção *não* é capturada:

```
>>> old_binascii.Error == binascii.Error
False
>>> try:
...     old_binascii.unhexlify(b'qwertyuiop')
... except binascii.Error:
...     print('boo')
...
Traceback (most recent call last):
  File "<stdin>", line 2, in <module>
binascii.Error: Non-hexadecimal digit found
```

Isso é esperado. Repare que módulos Python-puro se comportam do mesmo jeito: isso é parte de como o Python funciona.

O objetivo é fazer módulos de extensão seguros no nível do C, e não fazer gambiarras se comportarem de forma intuitiva. Modificar o `sys.modules` "manualmente" conta como uma gambiarra.

Fazendo módulos seguros com múltiplos interpretadores

Gerenciando estado global

Às vezes, o estado associado a um módulo Python não é específico àquele módulo, mas ao processo inteiro (ou a alguma outra coisa "mais global" que um módulo). Por exemplo:

- O módulo `readline` gerencia *o* terminal.
- Um módulo executando em uma placa de circuito quer controlar *o* componente LED.

Nestes casos, o módulo Python deve prover *acesso* ao estado global, ao invés de *possuí-lo*. Se possível, escreva o módulo de forma que múltiplas cópias dele possam acessar o estado independentemente (junto com outras bibliotecas, sejam elas do Python ou de outras linguagens). Se isso não for possível, considere usar travas explícitas.

Se for necessário usar estado global para o processo, o jeito mais simples de evitar problemas com múltiplos interpretadores é prevenir explicitamente que o módulo seja carregado mais de uma vez por processo — veja Exclusão voluntária: limitando a um objeto de módulo por processo.

Gerenciando estado por módulo

Para usar estado por módulo, use inicialização multifásica de módulos de extensão. Assim, você sinaliza que o seu módulo suporta múltiplos interpretadores corretamente.

Defina `PyModuleDef.m_size` como um número positivo *N* para requerer *N* bytes de armazenamento local para o módulo. Geralmente, *N* será o tamanho de alguma `struct` específica para o módulo, a qual pode guardar todo o estado a nível de C do módulo. Em particular, é nela que você deve colocar ponteiros para classes (incluindo exceções, mas excluindo tipos estáticos) e configurações (por exemplo, **`field_size_limit`** no módulo `csv`) que o código C precisa para funcionar.

Nota

Outra opção é guardar estado no `__dict__` do módulo, mas você deve evitar quebrar quando usuários modificarem o `__dict__` a partir do código Python. Isso geralmente significa verificar tipos e erros no nível do C, o que é fácil de ser feito incorretamente e difícil de se testar suficientemente.

Entretanto, se o estado do módulo não for necessário para o código C, guardá-lo somente no `__dict__` é uma boa ideia.

Se o estado do módulo inclui ponteiros para `PyObject`, o objeto de módulo deve conter referências a tais objetos e implementar os ganchos a nível de módulo `m_traverse`, `m_clear` e `m_free`. Eles funcionam como os `tp_traverse`, `tp_clear` e `tp_free` de uma classe. Adicioná-los requer algum trabalho e torna o código mais longo; é o preço de módulos que podem ser descarregados de forma limpa.

Um exemplo de módulo com estado por módulo está disponível atualmente como xxlimited [https://github.com/python/cpython/blob/master/Modules/xxlimited.c]; há um exemplo de inicialização do módulo no final do arquivo.

Exclusão voluntária: limitando a um objeto de módulo por processo

Um `PyModuleDef.m_size` não-negativo sinaliza que um módulo admite múltiplos interpretadores corretamente. Se este ainda não é o caso para o seu módulo, you pode explicitamente torná-lo carregável somente uma vez por processo. Por exemplo:

```
// Um sinalizador para todo o processo
static int loaded = 0;

// Mutex para fornecer segurança para a thread (só necessário pa
static PyMutex modinit_mutex = {0};

static int
exec_module(PyObject* module)
{
    PyMutex_Lock(&modinit_mutex);
    if (loaded) {
        PyMutex_Unlock(&modinit_mutex);
        PyErr_SetString(PyExc_ImportError,
                        "cannot load module more than once per p
        return -1;
    }
    loaded = 1;
    PyMutex_Unlock(&modinit_mutex);
    // ... rest of initialization
}
```

Se a função **PyModuleDef.m_clear** do seu módulo for capaz de se preparar para reinicializações futuras, ela deverá limpar o sinalizador `loaded`. Nesse caso, seu módulo não suportará múltiplas instâncias existentes *simultaneamente*, mas suportará, por exemplo, o carregamento após o desligamento do tempo de execução do Python (**Py_FinalizeEx()**) e a reinicialização (**Py_Initialize()**).

Acesso ao estado de módulo a partir de funções

É trivial acessar o estado a partir de funções a nível do módulo. Funções recebem o objeto de módulo como o primeiro argumento; para extrair o estado, você pode usar `PyModule_GetState`:

```
static PyObject *
func(PyObject *module, PyObject *args)
{
    my_struct *state = (my_struct*)PyModule_GetState(module);
    if (state == NULL) {
        return NULL;
    }
    // ... o resto da lógica
}
```

Nota

`PyModule_GetState` pode retornar `NULL` sem definir uma exceção se não houver estado de módulo, ou seja se `PyModuleDef.m_size` for zero. No seu próprio módulo, você controla o `m_size`, de forma que isso é fácil de prevenir.

Tipos no heap

Tradicionalmente, tipos definidos em C são *estáticos*; isto é, estruturas `static PyTypeObject` definidas diretamente em código e inicializadas usando `PyType_Ready()`.

Tais tipos são necessariamente compartilhados pelo processo inteiro. Compartilhá-los entre objetos de módulo requer atenção a qualquer estado que eles possuam ou acessem. Para limitar potenciais problemas, tipos estáticos são imutáveis a nível do Python: por exemplo, você não pode atribuir `str.meuatributo = 123`.

Não há problema em compartilhar objetos verdadeiramente imutáveis entre interpretadores, desde que através deles não seja possível acessar outros objetos mutáveis. De toda forma, no CPython, todo objeto Python tem um detalhe de implementação mutável: o contador de referências. Mudanças no refcount são protegidas pelo GIL. Logo, todo código que compartilha um objeto Python entre interpretadores depende implicitamente do atual GIL do CPython (que é global a nível de processo).

Por ser imutável e global no processo, um tipo estático não pode acessar o estado do "seu" módulo. Se um método de tal tipo precisar de acesso ao estado do módulo, o tipo precisa ser convertido para um *tipo alocado no heap*, ou, abreviando, *tipo no heap*. Tipos no heap correspondem mais fielmente a classes criadas pela instrução `class` do Python.

Para módulos novos, usar tipos no heap por padrão é uma boa regra geral.

Mudando tipos estáticos para tipos no heap

Tipos estáticos podem ser convertidos para tipos no heap, mas note que a API de tipos no heap não foi projetada para conversão "sem perda" de tipos estáticos—isto é, para criar um tipo que funciona exatamente como um dado tipo estático. Então, ao reescrever a definição de classe em uma nova API, é provável que você altere alguns detalhes sem querer (por exemplo, se o tipo é serializável em pickle ou não, ou slots herdados). Sempre teste os detalhes que são importantes para você.

Fique atento em particular aos dois pontos a seguir (mas note the esta não é uma lista completa):

- Ao contrário de tipos estáticos, tipos no heap são mutáveis por padrão. Use o sinalizador **`Py_TPFLAGS_IMMUTABLETYPE`** para impedir a mutabilidade.

- Tipos no heap herdam `tp_new` por padrão, e portanto eles podem passar a ser instanciáveis a partir de código Python. Você pode impedir isso com o sinalizador `Py_TPFLAGS_DISALLOW_INSTANTIATION`.

Definindo tipos no heap

Tipos no heap podem ser criados preenchendo uma estrutura `PyType_Spec`, uma descrição ou "diagrama" de uma classe, e chamando `PyType_FromModuleAndSpec()` para construir um novo objeto classe.

> **Nota**
>
> Outras funções, como `PyType_FromSpec()`, também podem criar tipos no heap, mas `PyType_FromModuleAndSpec()` associa a classe ao módulo, permitindo acesso ao estado do módulo a partir dos métodos.

A classe deve em geral ser guardada *tanto* no estado do módulo (para acesso seguro a partir do C) *quanto* no `__dict__` do módulo (para acesso a partir de código Python).

Protocolo de recolhimento de lixo

Instâncias de tipos no heap contêm referências aos seus tipos. Isso garante que o tipo não é destruído antes que todas as suas instâncias sejam, mas pode resultar em ciclos de referência que precisam ser quebrados pelo coletor de lixo.

Para evitar vazamentos de memória, instâncias de tipos no heap precisam implementar o protocolo de recolhimento de lixo. Isto é, tipos no heap devem:

- Ter o sinalizador `Py_TPFLAGS_HAVE_GC`.
- Define uma função de travessia usando `Py_tp_traverse`, que visita o tipo (por exemplo, usando `Py_VISIT(Py_TYPE(self))`).

Por favor, veja a documentação de `Py_TPFLAGS_HAVE_GC` e de `tp_traverse` para considerações adicionais.

A API para definir tipos no heap cresceu organicamente, o que resultou em um status quo no qual usá-la pode ser um pouco confuso. As seções a seguir vão lhe guiar pelos problemas mais comuns.

`tp_traverse` no Python 3.8 e anteriores

O requerimento de o `tp_traverse` visitar o tipo foi adicionado no Python 3.9. Se você suporta Python 3.8 e anteriores, a função de travessia *não* deve visitar o tipo, de forma que ela precisa ser mais complicada:

```
static int my_traverse(PyObject *self, visitproc visit, void *ar
{
    if (Py_Version >= 0x03090000) {
        Py_VISIT(Py_TYPE(self));
    }
    return 0;
}
```

Infelizmente, o símbolo `Py_Version` foi adicionado somente no Python 3.11. Para substituí-lo, use:

- `PY_VERSION_HEX`, caso não esteja usando a ABI estável, ou
- `sys.version_info` (via `PySys_GetObject()` e `PyArg_ParseTuple()`).

Delegando a função `tp_traverse`

Se a sua função de travessia delega para a `tp_traverse` da sua classe base (ou de outro tipo), certifique-se de que `Py_TYPE(self)` seja visitado apenas uma vez. Observe que somente tipos no heap devem visitar o tipo em `tp_traverse`.

Por exemplo, se a sua função de travessia incluir:

```
base->tp_traverse(self, visit, arg)
```

…e `base` puder ser um tipo estático, então ela também precisa incluir:

```
if (base->tp_flags & Py_TPFLAGS_HEAPTYPE) {
    // uma tp_traverse do tipo heap já visitou Py_TYPE(self)
} else {
    if (Py_Version >= 0x03090000) {
        Py_VISIT(Py_TYPE(self));
    }
}
```

Não é necessário mexer na contagem de referências do tipo em `tp_new` e `tp_clear`.

Definindo `tp_dealloc`

Se o seu tipo tem uma função `tp_dealloc` customizada, ele precisa:

- chamar `PyObject_GC_UnTrack()` antes que quaisquer campos sejam invalidados, e
- decrementar o contador de referências do tipo.

Para que o tipo permaneça válido durante o `tp_free`, o refcount do tipo precisa ser decrementado *depois* de a instância ser liberada. Por exemplo:

```
static void my_dealloc(PyObject *self)
{
    PyObject_GC_UnTrack(self);
    ...
    PyTypeObject *type = Py_TYPE(self);
    type->tp_free(self);
    Py_DECREF(type);
}
```

A função `tp_dealloc` padrão faz isso, de forma que se o seu tipo *não* a substitui você não precisa se preocupar.

Evitar substituir `tp_free`

O slot `tp_free` de um tipo no heap deve ser `PyObject_GC_Del()`. Este é o padráo; não o substitua.

Evitar `PyObject_New`

Objetos rastreados pelo GC precisam ser alocados usando funções que reconheçam o GC.

Se você usa `PyObject_New()` ou `PyObject_NewVar()`:

- Se possível, chame o slot `tp_alloc` do tipo. Isto é, troque `TYPE *o = PyObject_New(TYPE, typeobj)` por:

  ```
  TYPE *o = typeobj->tp_alloc(typeobj, 0);
  ```

  No lugar de `o = PyObject_NewVar(TYPE, typeobj, size)`, use também a forma acima, mas com `size` ao invés do `0`.

- Se isso não for possível (por exemplo, dentro de um `tp_alloc` customizado), chame `PyObject_GC_New()` or `PyObject_GC_NewVar()`:

  ```
  TYPE *o = PyObject_GC_New(TYPE, typeobj);
  ```

  ```
  TYPE *o = PyObject_GC_NewVar(TYPE, typeobj, size);
  ```

Acessando o estado do módulo a partir de classes

Dado um objeto de tipo definido com `PyType_FromModuleAndSpec()`, você pode chamar `PyType_GetModule()` para acessar o módulo associado, e então `PyModule_GetState()` para acessar o estado do módulo.

Para evitar o tedioso código de tratamento de erros de sempre, você pode combinar essas duas etapas com o `PyType_GetModuleState()` assim:

```
my_struct *state = (my_struct*)PyType_GetModuleState(type);
if (state == NULL) {
    return NULL;
}
```

Acesso ao estado do módulo a partir de métodos regulares

Acessar o estado do módulo a partir de métodos de uma classe já é um pouco mais complicado, mas passou a ser possível graças à API introduzida no Python 3.9. Para conseguir o estado, é necessário primeiro acessar a *classe definidora*, e então obter o estado do módulo a partir dela.

O maior obstáculo é encontrar *a classe na qual um método foi definido*, ou, abreviando, a *classe definidora* desse método. A classe definidora pode guardar uma referência para o módulo do qual ela é parte.

Não confunda a classe definidora com `Py_TYPE(self)`. Se o método for chamado em uma *subclasse* do seu tipo, `Py_TYPE(self)` será uma referência àquela subclasse, a qual pode ter sido definida em um módulo diferente do seu.

Nota

O código Python a seguir ilustra esse conceito. `Base.get_defining_class` retorna `Base` mesmo quando `type(self) == Sub`:

```
class Base:
    def get_type_of_self(self):
        return type(self)

    def get_defining_class(self):
        return __class__

class Sub(Base):
    pass
```

Para um método acessar a sua "classe definidora", ele precisa usar a **`convenção de chamada`** METH_METHOD | METH_FASTCALL | METH_KEYWORDS e a assinatura **`PyCMethod`** correspondente:

```
PyObject *PyCMethod(
    PyObject *self,               // objeto onde o módulo foi ch
```

```
    PyTypeObject *defining_class, // classe definidora
    PyObject *const *args,        // vetor C de argumentos
    Py_ssize_t nargs,             // comprimento de "args"
    PyObject *kwnames)            // NULL, ou dicionário de argu
```

Uma vez que vc tem a classe definidora, chame `PyType_GetModuleState()` para obter o estado do módulo associado a ela.

Por exemplo:

```
static PyObject *
example_method(PyObject *self,
        PyTypeObject *defining_class,
        PyObject *const *args,
        Py_ssize_t nargs,
        PyObject *kwnames)
{
    my_struct *state = (my_struct*)PyType_GetModuleState(definir
    if (state == NULL) {
        return NULL;
    }
    ... // rest of logic
}

PyDoc_STRVAR(example_method_doc, "...");

static PyMethodDef my_methods[] = {
    {"example_method",
      (PyCFunction)(void(*)(void))example_method,
      METH_METHOD|METH_FASTCALL|METH_KEYWORDS,
      example_method_doc}
    {NULL},
}
```

Acesso ao estado do módulo a partir de métodos slot, getters e setters

Nota

Adicionado na versão 3.11

Métodos slot—os métodos rápidos em C equivalentes aos métodos especiais, como `nb_add` para `__add__` ou `tp_new` para inicialização—têm uma API muito simples que não permite passar a classe definidora, ao contrário do `PyCMethod`. O mesmo vale para getters e setters definidos com `PyGetSetDef`.

Para acessar o estado do módulo nesses casos, use a função `PyType_GetModuleByDef()`, e passe a definição do módulo. Uma vez encontrado o módulo, chame `PyModule_GetState()` para obter o estado:

```
PyObject *module = PyType_GetModuleByDef(Py_TYPE(self), &module_
my_struct *state = (my_struct*)PyModule_GetState(module);
if (state == NULL) {
    return NULL;
}
```

Essa função `PyType_GetModuleByDef()` funciona procurando na ordem de resolução de métodos (isto é, todas as superclasses) a primeira superclasse que tem um módulo correspondente.

Nota

Em casos muito exóticos (cadeias hereditárias espalhadas através de múltiplos módulos criados a partir da mesma definição), a `PyType_GetModuleByDef()` pode não retornar o módulo da classe definidora correta. De todo modo, essa função sempre vai retornar um módulo com a mesma definição, garantindo um layout de memória C compatível.

Tempo de vida do estado do módulo

Quando um objeto de módulo é coletado como lixo, o seu estado de módulo é liberado. Para cada ponteiro para o estado do módulo (ou uma parte dele),

é necessário possuir uma referência ao objeto de módulo.

Isso não costuma ser um problema, dado que tipos criados com `PyType_FromModuleAndSpec()`, bem como suas instâncias, guardam referências ao módulo. Mesmo assim, é necessário tomar cuidado com a contagem de referências ao referenciar o estado do módulo a partir de outros lugares, como funções de retorno para bibliotecas externas.

Problemas em aberto

Vários problemas relacionados aos estados por módulo e aos tipos no heap ainda estão em aberto.

Discussões sobre como melhorar a situação são melhor realizadas no fórum de discussão na tag c-api [https://discuss.python.org/c/core-dev/c-api/30].

Escopo por classe

Atualmente (desde o Python 3.11) não é possível anexar estado a *tipos* individuais sem depender de detalhes de implementação do CPython (os quais podem mudar no futuro—talvez, ironicamente, para possibilitar uma solução adequada para o escopo por classe).

Conversão sem perdas para tipos no heap

A API de tipos no heap não foi projetada para conversão “sem perdas” de tipos estáticos. isto é, para criar um tipo que funciona exatamente como um dado tipo estático.

Descritor de arquivo de temporizador

Versão:: 1.13

Este documento discute o suporte do Python para o descritor de arquivo de temporizador do Linux.

Exemplos

O exemplo a seguir mostra como usar um descritor de arquivo de temporizador para executar uma função duas vezes por segundo:

```
# Scripts práticos devem realmente usar um timer sem bloqueio.
# Usamos um timer com bloqueio aqui para simplificar.
import os, time

# Cria um descritor de arquivo de temporizador
fd = os.timerfd_create(time.CLOCK_REALTIME)

# Inicia o temporizador em 1 segundo, com um intervalo de meio s
os.timerfd_settime(fd, initial=1, interval=0.5)

try:
    # Processa eventos do temporizador quatro vezes.
    for _ in range(4):
        # read() ser bloqueado até o temporizador expirar
        _ = os.read(fd, 8)
        print("Timer expired")
finally:
    # Lembre-se de fechar o descritor de arquivo de temporizador
    os.close(fd)
```

Para evitar a perda de precisão causada pelo tipo `float`, os descritores de arquivos de temporizador permitem especificar a expiração inicial e o

intervalo em nanossegundos inteiros com variantes `_ns` das funções.

Este exemplo mostra como **`epoll()`** pode ser usado com descritores de arquivo de temporizador para esperar até que o descritor de arquivo esteja pronto para leitura:

```
import os, time, select, socket, sys

# Cria um objeto epoll
ep = select.epoll()

# Neste exemplo, usa o endereço de loopback para enviar o comanc
#
# $ telnet 127.0.0.1 1234
# Trying 127.0.0.1...
# Connected to 127.0.0.1.
# Escape character is '^]'.
# stop
# Connection closed by foreign host.
#
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(("127.0.0.1", 1234))
sock.setblocking(False)
sock.listen(1)
ep.register(sock, select.EPOLLIN)

# Cria descritores de arquivo de temporizador no modo não bloque
num = 3
fds = []
for _ in range(num):
    fd = os.timerfd_create(time.CLOCK_REALTIME, flags=os.TFD_NON
    fds.append(fd)
    # Registra o descritor de arquivo do temporizador para event
    ep.register(fd, select.EPOLLIN)

# Inicia o temporizador com os.timerfd_settime_ns() em nanossegu
# O temporizador 1 dispara a cada 0,25 segundos; o timer 2 a cac
for i, fd in enumerate(fds, start=1):
    one_sec_in_nsec = 10**9
    i = i * one_sec_in_nsec
    os.timerfd_settime_ns(fd, initial=i//4, interval=i//4)
```

```
timeout = 3
try:
    conn = None
    is_active = True
    while is_active:
        # Aguarda o temporizador expirar por 3 segundos.
        # epoll.poll() retorna uma lista de pares (fd, event).
        # fd é um descritor de arquivo.
        # sock e conn[=valor retornado de socket.accept()] são c
        # Então use sock.fileno() e conn.fileno() para obter os
        events = ep.poll(timeout)

        # Se mais de um descritor de arquivo de timer estiver pr
        # epoll.poll() retorna uma lista de pares (fd, event).
        #
        # Neste exemplo de configurações,
        #    O 1º temporizador dispara a cada 0,25 segundos em 0
        #    O 2º temporizador dispara a cada 0,5 segundos em 0,
        #    O 3º temporizador dispara a cada 0,75 segundos em 0
        #
        #    Em 0,25 segundos, apenas o 1º temporizador dispara.
        #    Em 0,5 segundos, o 1º temporizador e o 2º temporiza
        #    Em 0,75 segundos, o 1º temporizador e o 3º temporiz
        #    Em 1,5 segundos, o 1º temporizador, o 2º temporizac
        #
        # Se um descritor de arquivo de temporizador for sinaliz
        # a última chamada de os.read(), os.read() retornará o r
        # como ordem de host de bytes de classe.
        print(f"Signaled events={events}")
        for fd, event in events:
            if event & select.EPOLLIN:
                if fd == sock.fileno():
                    # Verifica se há uma requisição de conexão.
                    print(f"Accepting connection {fd}")
                    conn, addr = sock.accept()
                    conn.setblocking(False)
                    print(f"Accepted connection {conn} from {adc
                    ep.register(conn, select.EPOLLIN)
                elif conn and fd == conn.fileno():
                    # Verifica se há dados para ler.
                    print(f"Reading data {fd}")
                    data = conn.recv(1024)
                    if data:
```

```
                        # Você deveria capturar exceção UnicodeD
                        cmd = data.decode()
                        if cmd.startswith("stop"):
                            print(f"Stopping server")
                            is_active = False
                        else:
                            print(f"Unknown command: {cmd}")
                    else:
                        # Acabaram os dados, fecha a conexão
                        print(f"Closing connection {fd}")
                        ep.unregister(conn)
                        conn.close()
                        conn = None
                elif fd in fds:
                    print(f"Reading timer {fd}")
                    count = int.from_bytes(os.read(fd, 8), byteo
                    print(f"Timer {fds.index(fd) + 1} expired {c
                else:
                    print(f"Unknown file descriptor {fd}")
finally:
    for fd in fds:
        ep.unregister(fd)
        os.close(fd)
    ep.close()
```

Este exemplo mostra como **select()** pode ser usado com descritores de arquivo de temporizador para esperar até que o descritor de arquivo esteja pronto para leitura:

```
import os, time, select, socket, sys

# Neste exemplo, usa o endereço de loopback para enviar o comanc
#
# $ telnet 127.0.0.1 1234
# Trying 127.0.0.1...
# Connected to 127.0.0.1.
# Escape character is '^]'.
# stop
# Connection closed by foreign host.
#
sock = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
sock.bind(("127.0.0.1", 1234))
```

```
sock.setblocking(False)
sock.listen(1)

# Cria descritores de arquivo de temporizador no modo não bloque
num = 3
fds = [os.timerfd_create(time.CLOCK_REALTIME, flags=os.TFD_NONBL
       for _ in range(num)]
select_fds = fds + [sock]

# Inicia o temporizador com os.timerfd_settime() em segundos.
# O temporizador 1 dispara a cada 0,25 segundos; o timer 2 a cac
for i, fd in enumerate(fds, start=1):
   os.timerfd_settime(fd, initial=i/4, interval=i/4)

timeout = 3
try:
    conn = None
    is_active = True
    while is_active:
       # Aguarda o temporizador expirar por 3 segundos.
       # select.select() retorna uma lista de objetos ou descrit
       rfd, wfd, xfd = select.select(select_fds, select_fds, sel
       for fd in rfd:
           if fd == sock:
               # Verifica se há uma requisição de conexão.
               print(f"Accepting connection {fd}")
               conn, addr = sock.accept()
               conn.setblocking(False)
               print(f"Accepted connection {conn} from {addr}")
               select_fds.append(conn)
           elif conn and fd == conn:
               # Verifica se há dados para ler.
               print(f"Reading data {fd}")
               data = conn.recv(1024)
               if data:
                   # Você deveria capturar exceção UnicodeDecode
                   cmd = data.decode()
                   if cmd.startswith("stop"):
                       print(f"Stopping server")
                       is_active = False
                   else:
                       print(f"Unknown command: {cmd}")
               else:
```

```
                    # Acabaram os dados, fecha a conexão
                    print(f"Closing connection {fd}")
                    select_fds.remove(conn)
                    conn.close()
                    conn = None
            elif fd in fds:
                print(f"Reading timer {fd}")
                count = int.from_bytes(os.read(fd, 8), byteorder=
                print(f"Timer {fds.index(fd) + 1} expired {count}
            else:
                print(f"Unknown file descriptor {fd}")
finally:
    for fd in fds:
       os.close(fd)
    sock.close()
    sock = None
```

A Ordem de Resolução de Métodos do Python 2.3

Nota

Este é um documento histórico, fornecido como apêndice à documentação oficial. A ordem de resolução de métodos discutida aqui foi *introduzida* no Python 2.3, mas ainda é usada em versões posteriores – incluindo o Python 3.

por Michele Simionato [https://github.com/micheles].

Resumo:: *Este documento é destinado a programadores Python que desejam entender a ordem de resolução de métodos C3 usada no Python 2.3. Embora não seja destinado a iniciantes, é bastante pedagógico com muitos exemplos elaborados. Não tenho conhecimento de outros documentos publicamente disponíveis com o mesmo escopo, portanto devem ser úteis.*

Aviso:

Eu doo este documento para a Python Software Foundation, sob a licença Python 2.3. Como de costume nestas circunstâncias, aviso o leitor que o que se segue deve *estar correto, mas não dou nenhuma garantia. Use-o por sua própria conta e risco!*

Reconhecimentos:

Todas as pessoas da lista de discussão do Python que me enviaram seu apoio. Paul Foley, que apontou várias imprecisões e me fez acrescentar a parte sobre ordem de precedência local. David Goodger

pela ajuda com a formatação em reStructuredText. David Mertz pela ajuda com a edição. Finalmente, Guido van Rossum que adicionou com entusiasmo este documento à página inicial oficial do Python 2.3.

O início

Felix qui potuit rerum cognoscere causas – Virgilius

Tudo começou com uma postagem de Samuele Pedroni na lista de discussão de desenvolvimento Python [[1]]. Em sua postagem, Samuele mostrou que a ordem de resolução de métodos do Python 2.2 não é monotônica e propôs substituí-la pela ordem de resolução de métodos C3. Guido concordou com seus argumentos e, portanto, agora o Python 2.3 usa C3. O método C3 em si não tem nada a ver com Python, pois foi inventado por pessoas que trabalharam em Dylan e está descrito em um artigo destinado a lispers [[2]]. O presente artigo fornece uma discussão (espero) legível do algoritmo C3 para Pythonistas que desejam entender os motivos da mudança.

Primeiro de tudo, deixe-me salientar que o que vou dizer se aplica apenas às *classes no novo estilo* introduzidas no Python 2.2: *classes clássicas* mantêm sua antiga ordem de resolução de métodos, profundidade primeiro e depois da esquerda para a direita. Portanto, não há quebra de código antigo para classes clássicas; e mesmo que em princípio pudesse haver quebra de código para novas classes de estilo do Python 2.2, na prática os casos em que a ordem de resolução C3 difere da ordem de resolução de métodos do Python 2.2 são tão raros que nenhuma quebra real de código é esperada. Portanto:

Não tenha medo!

Além disso, a menos que você faça uso intenso de herança múltipla e tenha hierarquias não triviais, você não precisa entender o algoritmo C3 e pode facilmente pular este artigo. Por outro lado, se você realmente deseja saber como funciona a herança múltipla, este artigo é para você. A boa notícia é que as coisas não são tão complicadas quanto você imagina.

Deixe-me começar com algumas definições básicas.

1. Dada uma classe C em uma complicada hierarquia de herança múltipla, não é uma tarefa trivial especificar a ordem na qual os métodos são substituídos, ou seja, especificar a ordem dos ancestrais de C.
2. A lista dos ancestrais de uma classe C, incluindo a própria classe, ordenada do ancestral mais próximo ao mais distante, é chamada de lista de precedência de classe ou *linearização* de C.
3. A *Ordem de Resolução de Métodos* (em inglês Method Resolution Order, MRO) é o conjunto de regras que constroem a linearização. Na literatura Python, a expressão "a MRO de C" também é usada como sinônimo de linearização da classe C.
4. Por exemplo, no caso de hierarquia de herança única, se C é uma subclasse de C1 e C1 é uma subclasse de C2, então a linearização de C é simplesmente a lista [C, C1, C2]. No entanto, com múltiplas hierarquias de herança, a construção da linearização é mais complicada, pois é mais difícil construir uma linearização que respeite a *ordem de precedência local* e a *monotonicidade*.
5. Discutirei a ordem de precedência local mais tarde, mas posso dar aqui a definição de monotonicidade. Uma MRO é monotônico quando o seguinte é verdadeiro: *se C1 precede C2 na linearização de C, então C1 precede C2 na linearização de qualquer subclasse de C*. Caso contrário, a operação inócua de derivar uma nova classe poderia alterar a ordem de resolução dos métodos, potencialmente introduzindo bugs muito sutis. Exemplos onde isso acontece serão mostrados posteriormente.
6. Nem todas as classes admitem uma linearização. Existem casos, em hierarquias complicadas, em que não é possível derivar uma classe tal que a sua linearização respeite todas as propriedades desejadas.

Aqui dou um exemplo desta situação. Considere a hierarquia

```
>>> O = object
>>> class X(O): pass
>>> class Y(O): pass
>>> class A(X,Y): pass
>>> class B(Y,X): pass
```

que pode ser representada com o seguinte grafo de herança, onde denotamos com O a classe `object`, que é o início de qualquer hierarquia para classes de novo estilo:

```
   -----------
  |            |
  |    O       |
  |  /   \     |
   - X    Y  /
     |  / | /
     | /  |/
     A    B
     \   /
       ?
```

Neste caso, não é possível derivar uma nova classe C de A e B, pois X precede Y em A, mas Y precede X em B, portanto a ordem de resolução de métodos seria ambígua em C.

Python 2.3 levanta uma exceção nesta situação (TypeError: MRO conflict among bases Y, X) proibindo o programador ingênuo de criar hierarquias ambíguas. Em vez disso, o Python 2.2 não levanta uma exceção, mas escolhe uma ordem *ad hoc* (CABXYO neste caso).

A ordem de resolução de métodos C3

Deixe-me apresentar algumas notações simples que serão úteis para a discussão a seguir. Usarei a notação de atalho:

```
C1 C2 ... CN
```

para indicar a lista de classes [C1, C2, … , CN].

head da lista é o seu primeiro elemento:

```
head = C1
```

enquanto *tail* é o resto da lista:

```
tail = C2 ... CN.
```

Também usarei a notação:

```
C + (C1 C2 ... CN) = C C1 C2 ... CN
```

para denotar a soma das listas [C] + [C1, C2, …, CN].

Agora posso explicar como funciona a MRO no Python 2.3.

Considere uma classe C em uma hierarquia de herança múltipla, com C herdando das classes base B1, B2, …, BN. Queremos calcular a linearização L[C] da classe C. A regra é a seguinte:

> *a linearização de C é a soma de C mais a mesclagem das linearizações dos pais e da lista dos pais.*

Em notação simbólica:

```
L[C(B1 ... BN)] = C + merge(L[B1] ... L[BN], B1 ... BN)
```

Em particular, se C é a classe `object`, que não tem pais, a linearização é trivial:

```
L[object] = object.
```

Contudo, em geral, deve-se calcular a mesclagem de acordo com a seguinte prescrição:

> *considere o topo da primeira lista, ou seja, L[B1][0]; se esse head não estiver no final de nenhuma das outras listas, então adicione-o à linearização de C e remova-o das listas na mesclagem, caso contrário olhe para o head da próxima lista e pegue-o, se for um bom head. Em seguida, repita a operação até que todas as classes sejam removidas ou seja impossível encontrar boas cabeças. Neste caso, é impossível construir a mesclagem, o Python 2.3 se recusará a criar a classe C e vai levantar uma exceção.*

Esta prescrição garante que a operação de mesclagem *preserva* a ordem, se a ordem puder ser preservada. Por outro lado, se a ordem não puder ser preservada (como no exemplo de desacordo sério sobre ordem discutido acima), então a mesclagem não poderá ser calculada.

O cálculo da mesclagem é trivial se C tiver apenas um pai (herança única); nesse caso:

```
L[C(B)] = C + merge(L[B],B) = C + L[B]
```

No entanto, no caso de herança múltipla, as coisas são mais complicadas e não espero que você consiga entender a regra sem alguns exemplos ;-)

Exemplos

Primeiro exemplo. Considere a seguinte hierarquia:

```
>>> O = object
>>> class F(O): pass
>>> class E(O): pass
>>> class D(O): pass
>>> class C(D,F): pass
>>> class B(D,E): pass
>>> class A(B,C): pass
```

Neste caso, o grafo de herança pode ser desenhado como:

```
                            6
                           ---
  Nível 3                 | O |                  (mais geral)
                        /  ---  \
                       /    |    \                       |
                      /     |     \                      |
                     /      |      \                     |
                    ---    ---    ---                    |
  Nível 2        3 | D | 4| E |  | F | 5                 |
                    ---    ---    ---                    |
                     \  \ _ /       |                   |
                      \    / \ _    |                   |
                       \  /      \  |                   |
```

```
                        ---      ---                    |
Nível 1            1 | B |    | C | 2                  |
                        ---      ---                    |
                          \      /                      |
                           \    /                     \ /
                             ---
Nível 0                    0 | A |                (mais especia
                             ---
```

As linearizações de O,D,E e F são triviais:

```
L[O] = O
L[D] = D O
L[E] = E O
L[F] = F O
```

A linearização de B pode ser calculada como:

```
L[B] = B + merge(DO, EO, DE)
```

Vemos que D é um bom *head*, portanto pegamos ele e ficamos reduzidos a calcular `merge(O,EO,E)`. Agora O não é um bom *head*, pois está no *tail* da sequência EO. Neste caso a regra diz que temos que pular para a próxima sequência. Então vemos que E é um bom *head*; nós pegamos isso e somos reduzidos a calcular `merge(O,O)` que dá O. Portanto:

```
L[B] =  B D E O
```

Usando o mesmo procedimento encontra-se:

```
L[C] = C + merge(DO,FO,DF)
     = C + D + merge(O,FO,F)
     = C + D + F + merge(O,O)
     = C D F O
```

Agora podemos calcular:

```
L[A] = A + merge(BDEO,CDFO,BC)
     = A + B + merge(DEO,CDFO,C)
     = A + B + C + merge(DEO,DFO)
     = A + B + C + D + merge(EO,FO)
```

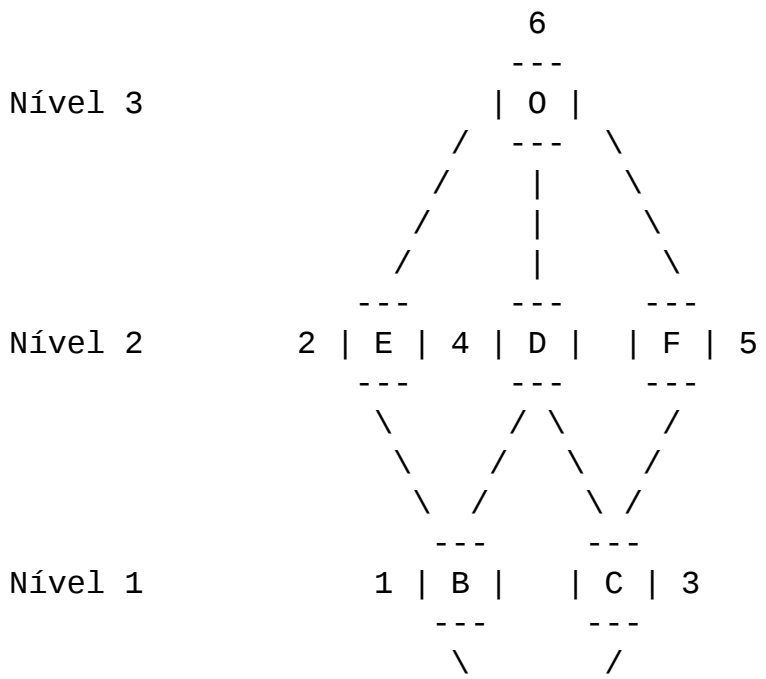
```
= A + B + C + D + E + merge(O,FO)
= A + B + C + D + E + F + merge(O,O)
= A B C D E F O
```

Neste exemplo, a linearização é ordenada de maneira bastante agradável de acordo com o nível de herança, no sentido de que níveis mais baixos (ou seja, classes mais especializadas) têm precedência mais alta (veja o grafo de herança). No entanto, este não é o caso geral.

Deixo como exercício para o leitor calcular a linearização do meu segundo exemplo:

```
>>> O = object
>>> class F(O): pass
>>> class E(O): pass
>>> class D(O): pass
>>> class C(D,F): pass
>>> class B(E,D): pass
>>> class A(B,C): pass
```

A única diferença com o exemplo anterior é a mudança B(D,E) –> B(E,D); no entanto, mesmo uma pequena modificação muda completamente a ordem da hierarquia:



```
                          \     /
                            ---
  Nível 0                 0 | A |
                            ---
```

Observe que a classe E, que está no segundo nível da hierarquia, precede a classe C, que está no primeiro nível da hierarquia, ou seja, E é mais especializado que C, mesmo que esteja em um nível mais alto.

Um programador preguiçoso pode obter a MRO diretamente do Python 2.2, pois neste caso ela coincide com a linearização do Python 2.3. Basta invocar o método `mro()` da classe A:

```
>>> A.mro()
[<class 'A'>, <class 'B'>, <class 'E'>,
<class 'C'>, <class 'D'>, <class 'F'>,
<class 'object'>]
```

Finalmente, deixe-me considerar o exemplo discutido na primeira seção, envolvendo um sério desacordo de ordem. Neste caso, é simples calcular as linearizações de O, X, Y, A e B:

```
  L[O] = O
  L[X] = X O
  L[Y] = Y O
  L[A] = A X Y O
  L[B] = B Y X O
```

Porém, é impossível calcular a linearização para uma classe C que herda de A e B:

```
L[C] = C + merge(AXYO, BYXO, AB)
     = C + A + merge(XYO, BYXO, B)
     = C + A + B + merge(XYO, YXO)
```

Neste ponto não podemos mesclar as listas XYO e YXO, uma vez que X está no *tail* de YXO enquanto Y está no *tail* de XYO: portanto não há bons *head* e o algoritmo C3 para. Python 2.3 levanta um erro e se recusa a criar a classe C.

Ordens de resolução de métodos ruins

Uma MRO é *ruim* quando quebra propriedades fundamentais como ordem de precedência local e monotonicidade. Nesta seção, mostrarei que tanto a MRO para classes clássicas quanto a MRO para classes de novo estilo em Python 2.2 são ruins.

É mais fácil começar com a ordem de precedência local. Considere o seguinte exemplo:

```
>>> F=type('Food',(),{'remember2buy':'spam'})
>>> E=type('Eggs',(F,),{'remember2buy':'eggs'})
>>> G=type('GoodFood',(F,E),{}) # under Python 2.3 this is an er
```

com diagrama de herança

```
                    O
                    |
  (comprar spam)    F
                    | \
                    | E   (comprar ovos)
                    | /
                    G

          (comprar ovos ou spam ?)
```

Vemos que a classe G herda de F e E, com F *antes* de E: portanto, esperaríamos que o atributo *G.remember2buy* fosse herdado por *F.remember2buy* e não por *E.remember2buy*: no entanto, Python 2.2 dá

```
>>> G.remember2buy
'eggs'
```

Isto é uma quebra da ordem de precedência local, uma vez que a ordem na lista de precedência local, ou seja, a lista dos pais de G, não é preservada na linearização de G em Python 2.2:

```
L[G,P22]= G E F object   # F *segue* E
```

Pode-se argumentar que a razão pela qual F segue E na linearização do Python 2.2 é que F é menos especializado que E, uma vez que F é a superclasse de E; no entanto, a quebra da ordem de precedência local é bastante não intuitiva e sujeita a erros. Isto é particularmente verdadeiro porque é diferente das classes de estilo antigo:

```
>>> class F: remember2buy='spam'
>>> class E(F): remember2buy='eggs'
>>> class G(F,E): pass
>>> G.remember2buy
'spam'
```

Neste caso a MRO é GFEF e a ordem de precedência local é preservada.

Como regra geral, hierarquias como a anterior devem ser evitadas, uma vez que não está claro se F deve substituir E ou vice-versa. O Python 2.3 resolve a ambiguidade levantando uma exceção na criação da classe G, impedindo efetivamente o programador de gerar hierarquias ambíguas. A razão para isso é que o algoritmo C3 falha quando a mesclagem:

```
merge(FO,EFO,FE)
```

não puder ser calculada, porque F está no *tail* de EFO e E está no *tail* de FE.

A verdadeira solução é conceber uma hierarquia não ambígua, ou seja, derivar G de E e F (o mais específico primeiro) e não de F e E; neste caso a MRO é GEF, sem dúvida.

```
            O
            |
            F (spam)
          / |
  (ovos)  E |
          \ |
            G
              (ovos, sem dúvida)
```

Python 2.3 força o programador a escrever boas hierarquias (ou, pelo menos, menos propensas a erros).

Falando nisso, deixe-me salientar que o algoritmo Python 2.3 é inteligente o suficiente para reconhecer erros óbvios, como a duplicação de classes na lista de pais:

```
>>> class A(object): pass
>>> class C(A,A): pass # error
Traceback (most recent call last):
  File "<stdin>", line 1, in ?
TypeError: duplicate base class A
```

Python 2.2 (tanto para classes clássicas quanto para classes de novo estilo) nesta situação, não levantaria nenhuma exceção.

Por fim, gostaria de destacar duas lições que aprendemos com este exemplo:

1. apesar do nome, a MRO determina a ordem de resolução dos atributos, não apenas dos métodos;
2. o alimento padrão para Pythonistas é spam! (mas você já sabia disso ;-)

Tendo discutido a questão da ordem de precedência local, deixe-me agora considerar a questão da monotonicidade. Meu objetivo é mostrar que nem a MRO para classes clássicas nem para o novo estilo de classes do Python 2.2 são monotônicas.

Para provar que a MRO para classes clássicas não é monotônica é bastante trivial, basta olhar o diagrama em losango:

```
     C
    / \
   /   \
  A     B
   \   /
    \ /
     D
```

Percebe-se facilmente a inconsistência:

```
L[B,P21] = B C        # B precede C : métodos de B venceram
L[D,P21] = D A C B C  # B segue C  : métodos de B venceram!
```

Por outro lado, não há problemas com as MROs do Python 2.2 e do 2.3, elas fornecem ambos:

```
L[D] = D A B C
```

Guido ressalta em seu ensaio [[3]] que a MRO clássica não é tão ruim na prática, já que normalmente se pode evitar losangos para classes clássicas. Mas todas as classes de novo estilo herdam de `object`, portanto os diamantes são inevitáveis e inconsistências aparecem em cada grafo de herança múltipla.

A MRO do Python 2.2 torna difícil quebrar a monotonicidade, mas não impossível. O exemplo a seguir, fornecido originalmente por Samuele Pedroni, mostra que a MRO do Python 2.2 não é monotônica:

```
>>> class A(object): pass
>>> class B(object): pass
>>> class C(object): pass
>>> class D(object): pass
>>> class E(object): pass
>>> class K1(A,B,C): pass
>>> class K2(D,B,E): pass
>>> class K3(D,A):   pass
>>> class Z(K1,K2,K3): pass
```

Aqui estão as linearizações de acordo com a MRO C3 (o leitor deverá verificar essas linearizações como exercício e desenhar o diagrama de herança ;-)

```
L[A] = A O
L[B] = B O
L[C] = C O
L[D] = D O
L[E] = E O
L[K1]= K1 A B C O
L[K2]= K2 D B E O
L[K3]= K3 D A O
L[Z] = Z K1 K2 K3 D A B C E O
```

Python 2.2 fornece exatamente as mesmas linearizações para A, B, C, D, E, K1, K2 e K3, mas uma linearização diferente para Z:

```
L[Z,P22] = Z K1 K3 A K2 D B C E O
```

É claro que esta linearização está *errada*, uma vez que A vem antes de D, enquanto na linearização de K3 A vem *depois* de D. Em outras palavras, em métodos K3 derivados de D substituem os métodos derivados de A, mas em Z, que ainda é uma subclasse de K3, os métodos derivados de A substituem os métodos derivados de D! Isto é uma violação da monotonicidade. Além disso, a linearização de Z do Python 2.2 também é inconsistente com a ordem de precedência local, uma vez que a lista de precedência local da classe Z é [K1, K2, K3] (K2 precede K3), enquanto na linearização de Z K2 *segue* K3 . Estes problemas explicam porque é que a regra do 2.2 foi rejeitada em favor da regra C3.

O fim

Esta seção é para o leitor impaciente, que passou direto por todas as seções anteriores e pulou imediatamente para o final. Esta seção também é para o programador preguiçoso, que não quer exercitar seu cérebro. Finalmente, é para o programador com alguma arrogância, caso contrário ele/ela não estaria lendo um artigo sobre a ordem de resolução de métodos C3 em múltiplas hierarquias de herança ;-) Essas três virtudes tomadas em conjunto (e *não* separadamente) merecem um prêmio: o prêmio é um pequeno script em Python 2.2 que permite calcular a MRO do 2.3 sem risco para o seu cérebro. Basta alterar a última linha para brincar com os vários exemplos que discuti neste artigo.:

```
#<mro.py>

"""Algoritmo C3 por Samuele Pedroni (com legibilidade melhorada

class __metaclass__(type):
    "Todas as classes são modificadas metamagicamente para serem
    __repr__ = lambda cls: cls.__name__

class ex_2:
```

```
    "Desacordo de ordem grave" #por Guido
    class O: pass
    class X(O): pass
    class Y(O): pass
    class A(X,Y): pass
    class B(Y,X): pass
    try:
        class Z(A,B): pass #cria Z(A,B) no Python 2.2
    except TypeError:
        pass # Z(A,B) não pode ser criado no Python 2.3

class ex_5:
    "Meu primeiro exemplo"
    class O: pass
    class F(O): pass
    class E(O): pass
    class D(O): pass
    class C(D,F): pass
    class B(D,E): pass
    class A(B,C): pass

class ex_6:
    "Meu segundo exemplo"
    class O: pass
    class F(O): pass
    class E(O): pass
    class D(O): pass
    class C(D,F): pass
    class B(E,D): pass
    class A(B,C): pass

class ex_9:
    "Diferença entre MRO do Python 2.2 e o C3" #por Samuele
    class O: pass
    class A(O): pass
    class B(O): pass
    class C(O): pass
    class D(O): pass
    class E(O): pass
    class K1(A,B,C): pass
    class K2(D,B,E): pass
    class K3(D,A): pass
    class Z(K1,K2,K3): pass
```

```
def merge(seqs):
    print '\n\nCPL[%s]=%s' % (seqs[0][0],seqs),
    res = []; i=0
    while 1:
      nonemptyseqs=[seq for seq in seqs if seq]
      if not nonemptyseqs: return res
      i+=1; print '\n',i,'round: candidates...',
      for seq in nonemptyseqs: # encontra candidatos a mesclagem
          cand = seq[0]; print ' ',cand,
          nothead=[s for s in nonemptyseqs if cand in s[1:]]
          if nothead: cand=None #rejeita candidato
          else: break
      if not cand: raise "Hierarquia inconsistente"
      res.append(cand)
      for seq in nonemptyseqs: # remove candidato
          if seq[0] == cand: del seq[0]

def mro(C):
    "Calcula a lista de precedência da classe (mro) conforme C3'
    return merge([[C]]+map(mro,C.__bases__)+[list(C.__bases__)])

def print_mro(C):
    print '\nMRO[%s]=%s' % (C,mro(C))
    print '\nP22 MRO[%s]=%s' % (C,C.mro())

print_mro(ex_9.Z)

#</mro.py>
```

Isso é tudo, pessoal!

Divirtam-se !

Recursos

[1]

O tópico no python-dev iniciado por Samuele Pedroni:
https://mail.python.org/pipermail/python-dev/2002-October/029035.html

[2]

O artigo *Uma Linearização Monotônica de Superclasses para Dylan*, em inglês: https://doi.org/10.1145/236337.236343

[3]

Ensaio de Guido van Rossum, *Unificando tipos e classes em Python 2.2*, em inglês: https://web.archive.org/web/20140210194412/http://www.python.org/download/releases/2.2.2/descrintro

Suporte do Python para threads livres

A partir da versão 3.13, o CPython tem suporte para uma compilação do Python chamada threads livres (em inglês, free threading), onde a trava global do interpretador (GIL) está desabilitada. A execução com threads livres permite a utilização total do poder de processamento disponível, executando threads em paralelo nos núcleos de CPU disponíveis. Embora nem todos os softwares se beneficiem disso automaticamente, os programas projetados com uso de threads em mente serão executados mais rapidamente em hardware com vários núcleos.

Alguns pacotes de terceiros, em particular aqueles com um módulo de extensão, podem não estar prontos para uso em uma construção com threads livres e habilitarão novamente a GIL.

Este documento descreve as implicações de threads livres para o código Python. Veja Suporte a extensões da API C para threads livres para informações sobre como escrever extensões C que oferecem suporte à construção com threads livres.

Ver também

PEP 703 [https://peps.python.org/pep-0703/] – Tornando a trava global do interpretador opcional no CPython para uma descrição geral do Python com threads livres.

Instalação

A partir do Python 3.13, os instaladores oficiais do macOS e do Windows oferecem suporte opcional à instalação de binários Python com threads

livres. Os instaladores estão disponíveis em https://www.python.org/downloads/.

Para obter informações sobre outras plataformas, consulte Instalando um Python com threads livres [https://py-free-threading.github.io/installing_cpython/], um guia de instalação mantido pela comunidade para instalar o Python com threads livres.

Ao construir o CPython a partir do código-fonte, a opção de configuração `--disable-gil` deve ser usada para construir um interpretador Python com threads livres.

Identificando um Python com threads livres

Para verificar se o interpretador atual oferece suporte a threads livres, `python -VV` e `sys.version` contêm “free-threading build”. A nova função `sys._is_gil_enabled()` pode ser usada para verificar se a GIL está realmente desabilitada no processo em execução.

A variável de configuração `sysconfig.get_config_var("Py_GIL_DISABLED")` pode ser usada para determinar se a compilação tem suporte a threads livres. Se a variável for definida como `1`, a compilação tem suporte a threads livres. Este é o mecanismo recomendado para decisões relacionadas à configuração da construção.

A trava global do interpretador no Python com threads livres

Construções com threads livres do CPython oferecem suporte a opcionalmente executar com a GIL habilitada em tempo de execução usando a variável de ambiente `PYTHON_GIL` ou a opção de linha de comando `-X gil`.

A GIL também pode ser habilitada automaticamente ao importar um módulo de extensão da API C que não esteja explicitamente marcado como

oferecendo suporte a threads livres. Um aviso será impresso neste caso.

Além da documentação de pacotes individuais, os seguintes sites rastreiam o status do suporte de pacotes populares para threads livres:

- https://py-free-threading.github.io/tracking/
- https://hugovk.github.io/free-threaded-wheels/

Segurança nas threads

A construção com threads livres do CPython visa fornecer comportamento de segurança às threads semelhante no nível do Python à construção padrão com GIL habilitada. Tipos embutidos como `dict`, `list` e `set` usam travas internas para proteger contra modificações simultâneas de maneiras que se comportam de forma semelhante à GIL. Veja Garantias de segurança de thread para as garantias fornecidas por tipos embutidos.

Nota

É recomendável usar `threading.Lock` ou outras primitivas de sincronização em vez de depender de travas internas de tipos embutidos, quando possível.

Limitações conhecidas

Esta seção descreve as limitações conhecidas da construção do Python com threads livres.

Imortalização

Na construção com threads livres, alguns objetos são imortais. Objetos imortais não são desalocados e têm contagens de referência que nunca são modificadas. Isso é feito para evitar contenção de contagem de referências que impediria o dimensionamento multithread eficiente.

A partir da versão 3.14, a imortalização está limitada a:

- Constantes de código: literais numéricos, literais de string e literais de tupla compostos por outras constantes.
- Strings internadas por `sys.intern()`.

Objetos quadro

Não é seguro acessar `frame.f_locals` a partir de um objeto frame se esse frame estiver sendo executado em outra thread, e fazê-lo pode causar a falha do interpretador.

Iteradores

Geralmente, não é seguro para thread acessar o mesmo objeto iterador a partir de múltiplas threads simultaneamente, e as threads podem encontrar elementos duplicados ou ausentes.

Desempenho com thread única

A construção com threads livres apresenta sobrecarga adicional na execução de código Python em comparação com a construção padrão com GIL habilitado. A quantidade de sobrecarga depende da carga de trabalho e do hardware. No conjunto de benchmarks pyperformance, a sobrecarga média varia de cerca de 1% no macOS aarch64 a 8% em sistemas Linux x86-64.

Mudanças comportamentais

Esta seção descreve as mudanças comportamentais do CPython com a construção com threads livres.

Variáveis de contexto

Na construção com threads livres, o sinalizador `thread_inherit_context` é definido como true por padrão, o que faz com que threads criadas com

`threading.Thread` iniciem com uma cópia de `Context()` do chamador de `start()`. Na construção padrão com GIL habilitada, o sinalizador é definido como false por padrão, então as threads iniciam com um `Context()` vazio.

Filtros de aviso

Na construção com threads livres, o sinalizador `context_aware_warnings` é definido como verdadeiro por padrão. Na construção com GIL habilitada, o sinalizador é definido como falso por padrão. Se o sinalizador for verdadeiro, o gerenciador de contexto `warnings.catch_warnings` usa uma variável de contexto para filtros de aviso. Se o sinalizador for falso, `catch_warnings` modifica a lista de filtros globais, o que não é seguro para thread. Consulte o módulo `warnings` para obter mais detalhes.

Aumento do consumo de memória

A construção com threads livres geralmente consome mais memória do que a construção padrão. Isso ocorre por vários motivos, principalmente devido a decisões de projeto.

Todas as strings internadas são imortais

Nas versões modernas do Python (desde a versão 2.3), internar uma string (por exemplo, com `sys.intern()`) não a torna imortal. Em vez disso, se a última referência a essa string deixar de existir, ela será removida da tabela de strings internadas. Isso não ocorre na construção com threads livres, na qual qualquer string internada torna-se imortal, persistindo até o desligamento do interpretador.

Objetos não-GC têm um cabeçalho de objeto maior

The free-threaded build uses a different `PyObject` structure. Instead of having the GC related information allocated before the `PyObject` structure, like in the default build, the GC related info is part of the normal object header. For example, on the AMD64 platform, `None` uses 32 bytes on the

free-threaded build vs 16 bytes for the default build. GC objects (such as dicts and lists) are the same size for both builds since the free-threaded build does not use additional space for the GC info.

QSBR can delay freeing of memory

In order to safely implement lock-free data structures, a safe memory reclamation (SMR) scheme is used, known as quiescent state-based reclamation (QSBR). This means that the memory backing data structures allowing lock-free access will use QSBR, which defers the free operation, rather than immediately freeing the memory. Two examples of these data structures are the list object and the dictionary keys object. See `InternalDocs/qsbr.md` in the CPython source tree for more details on how QSBR is implemented. Running `gc.collect()` should cause all memory being held by QSBR to be actually freed. Note that even when QSBR frees the memory, the underlying memory allocator may not immediately return that memory to the OS and so the resident set size (RSS) of the process might not decrease.

mimalloc allocator vs pymalloc

The default build will normally use the “pymalloc” memory allocator for small allocations (512 bytes or smaller). The free-threaded build does not use pymalloc and allocates all Python objects using the “mimalloc” allocator. The pymalloc allocator has the following properties that help keep memory usage low: small per-allocated-block overhead, effective memory fragmentation prevention, and quick return of free memory to the operating system. The mimalloc allocator does quite well in these respects as well but can have some more overhead.

In the free-threaded build, mimalloc manages memory in a number of separate heaps (currently four). For example, all GC supporting objects are allocated from their own heap. Using separate heaps means that free memory in one heap cannot be used for an allocation that uses another heap. Also, some heaps are configured to use QSBR (quiescent-state based reclamation) when freeing the memory that backs up the heap (known as “pages” in mimalloc terminology). The use of QSBR creates a delay

between all memory blocks for a page being freed and the memory page being released, either for new allocations or back to the OS.

The mimalloc allocator also defers returning freed memory back to the OS. You can reduce that delay by setting the environment variable **`MIMALLOC_PURGE_DELAY`** to `0`. Note that this will likely reduce the performance of the allocator.

Free-threaded reference counting can cause objects to live longer

In the default build, when an object's reference count reaches zero, it is normally deallocated. The free-threaded build uses "biased reference counting", with a fast-path for objects "owned" by the current thread and a slow path for other objects. See **PEP 703** [https://peps.python.org/pep-0703/] for additional details. Any time an object's reference count ends up in a "queued" state, deallocation can be deferred. The queued state is cleared from the "eval breaker" section of the bytecode evaluator.

The free-threaded build also allows a different mode of reference counting, known as "deferred reference counting". This mode is enabled by setting a flag on a per-object basis. Deferred reference counting is enabled for the following types:

- module objects
- module top-level functions
- class methods defined in the class scope
- descriptor objects
- thread-local objects, created by `threading.local`

When deferred reference counting is enabled, references from Python function stacks are not added to the reference count. This scheme reduces the overhead of reference counting, especially for objects used from multiple threads. Because the stack references are not counted, objects with deferred reference counting are not immediately freed when their internal reference count goes to zero. Instead, they are examined by the next GC run and, if no stack references to them are found, they are freed. This means

these objects are freed by the GC and not when their reference count goes to zero, as is typical.

Per-thread reference counting can delay freeing objects

To avoid contention on the reference count fields of frequently shared objects, the free-threaded build also uses “per-thread reference counting” for a few selected object types. Rather than updating a single shared reference count, each thread maintains its own local reference count array, indexed by a unique id assigned to the object. The true reference count is only computed by summing the per-thread counts when the object’s local count drops to zero. Per-thread reference counting is currently used for:

- heap type objects (classes created in Python)
- code objects
- the `__dict__` of module objects

Because the per-thread counts must be merged back to the object before it can be deallocated, objects using per-thread reference counting are typically freed later than they would be in the default build. In particular, such an object is usually not freed until the thread that referenced it reaches a safe point (for example, in the “eval breaker” section of the bytecode evaluator) or exits. Running `gc.collect()` will merge the per-thread counts and allow these objects to be freed.

Suporte a extensões da API C para threads livres

A partir da versão 3.13, o CPython tem suporte para execução com a trava global do interpretador (GIL) desabilitada em uma configuração chamada threads livres. Este documento descreve como adaptar extensões da API C para ter suporte a threads livres.

Identificando a construção com threads livres em C

A API C do CPython expõe a macro `Py_GIL_DISABLED`: na construção com threads livres ela está definida como `1`, e na construção regular ela não está definida. Você pode usá-la para ativar o código que é executado apenas na construção com threads livres:

```
#ifdef Py_GIL_DISABLED
/* código que só vai ser executado na construção com threads liv
#endif
```

Nota

No Windows, esta macro não é definida automaticamente, mas deve ser especificada ao compilador durante a compilação. A função `sysconfig.get_config_var()` pode ser usada para determinar se o interpretador em execução tinha a macro definida.

Inicialização de módulo

Os módulos de extensão precisam indicar explicitamente que têm suporte à execução com a GIL desabilitada; caso contrário, importar a extensão

levantará um aviso e ativará a GIL em tempo de execução.

Há duas maneiras de indicar que um módulo de extensão tem suporte à execução com a GIL desabilitada, dependendo se a extensão usa inicialização monofásica ou multifásica.

Inicialização multifásica

Extensions that use multi-phase initialization (functions like `PyModuleDef_Init()`, `PyModExport_*` export hook, `PyModule_FromSlotsAndSpec()`) should add a `Py_mod_gil` slot in the module definition. If your extension supports older versions of CPython, you should guard the slot with a `PY_VERSION_HEX` check.

```
static struct PyModuleDef_Slot module_slots[] = {
    ...
#if PY_VERSION_HEX >= 0x030D0000
    {Py_mod_gil, Py_MOD_GIL_NOT_USED},
#endif
    {0, NULL}
};
```

Inicialização monofásica

Extensions that use legacy single-phase initialization (that is, `PyModule_Create()`) should call `PyUnstable_Module_SetGIL()` to indicate that they support running with the GIL disabled. The function is only defined in the free-threaded build, so you should guard the call with `#ifdef Py_GIL_DISABLED` to avoid compilation errors in the regular build.

```
static struct PyModuleDef moduledef = {
    PyModuleDef_HEAD_INIT,
    ...
};

PyMODINIT_FUNC
PyInit_mymodule(void)
{
    PyObject *m = PyModule_Create(&moduledef);
```

```
    if (m == NULL) {
        return NULL;
    }
#ifdef Py_GIL_DISABLED
    PyUnstable_Module_SetGIL(m, Py_MOD_GIL_NOT_USED);
#endif
    return m;
}
```

Diretrizes gerais de API

A maior parte da API C é segura para thread, mas há algumas exceções.

- **Campos de structs**: acessar campos em objetos ou structs da API C do Python diretamente não é seguro para thread se o campo puder ser modificado simultaneamente.
- **Macros**: macros de acesso como `PyList_GET_ITEM`, `PyList_SET_ITEM` e macros como `PySequence_Fast_GET_SIZE` que usam o objeto retornado por `PySequence_Fast()` não realizam nenhuma verificação de erro ou trava. Essas macros não são seguras para thread se o objeto contêiner puder ser modificado simultaneamente.
- **Referências emprestadas**: as funções da API C que retornam referências emprestadas podem não ser seguras para thread se o objeto que as contêm for modificado simultaneamente. Veja a seção sobre referências emprestadas para mais informações.

Segurança de threads de contêineres

Contêineres como `PyListObject`, `PyDictObject` e `PySetObject` executam uma trava interna na construção com threads livres. Por exemplo, `PyList_Append()` irá travar a lista antes de anexar um item.

`PyDict_Next`

Uma exceção notável é `PyDict_Next()`, que não trava o dicionário. Você deve usar `Py_BEGIN_CRITICAL_SECTION` para proteger o dicionário

enquanto itera sobre ele se o dicionário puder ser modificado simultaneamente:

```
Py_BEGIN_CRITICAL_SECTION(dict);
PyObject *key, *value;
Py_ssize_t pos = 0;
while (PyDict_Next(dict, &pos, &key, &value)) {
    ...
}
Py_END_CRITICAL_SECTION();
```

Referências emprestadas

Algumas funções da API C retornam referências emprestadas. Essas APIs não são seguras para thread se o objeto que as contém for modificado simultaneamente. Por exemplo, não é seguro usar `PyList_GetItem()` se a lista puder ser modificada simultaneamente.

A tabela a seguir lista algumas APIs de referências emprestadas e suas substituições que retornam referências fortes.

API de referências emprestadas	API de referências fortes
`PyList_GetItem()`	`PyList_GetItemRef()`
`PyList_GET_ITEM()`	`PyList_GetItemRef()`
`PyDict_GetItem()`	`PyDict_GetItemRef()`
`PyDict_GetItemWithError()`	`PyDict_GetItemRef()`
`PyDict_GetItemString()`	`PyDict_GetItemStringRef()`

API de referências emprestadas	API de referências fortes
`PyDict_SetDefault()`	`PyDict_SetDefaultRef()`
`PyDict_Next()`	nenhuma (veja PyDict_Next)
`PyWeakref_GetObject()`	`PyWeakref_GetRef()`
`PyWeakref_GET_OBJECT()`	`PyWeakref_GetRef()`
`PyImport_AddModule()`	`PyImport_AddModuleRef()`
`PyCell_GET()`	`PyCell_Get()`

Nem todas as APIs que retornam referências emprestadas são problemáticas. Por exemplo, `PyTuple_GetItem()` é segura porque as tuplas são imutáveis. Da mesma forma, nem todos os usos das APIs acima são problemáticos. Por exemplo, `PyDict_GetItem()` é frequentemente usado para analisar dicionários de argumentos nomeados em chamadas de função; esses dicionários de argumentos nomeados efetivamente privados (não acessíveis por outros threads), portanto, usar referências emprestadas nesse contexto é seguro.

Algumas dessas funções foram adicionadas no Python 3.13. Você pode usar o pacote pythoncapi-compat [https://github.com/python/pythoncapi-compat] para fornecer implementações dessas funções para versões mais antigas do Python.

APIs de alocação de memória

A API C de gerenciamento de memória do Python fornece funções em três domínios de alocação diferentes: “raw”, “mem” e “object”. Para segurança de thread, a construção com threads livres requer que apenas objetos Python sejam alocados usando o domínio do objeto e que todos os objetos Python sejam alocados usando esse domínio. Isso difere das versões anteriores do Python, onde era apenas uma melhor prática e não um requisito estrito.

Nota

Procure usos de `PyObject_Malloc()` em sua extensão e verifique se a memória alocada é usada para objetos Python. Use `PyMem_Malloc()` para alocar buffers em vez de `PyObject_Malloc()`.

APIs da GIL e de estado de threads

Python fornece um conjunto de funções e macros para gerenciar o estado de threads e a GIL, como:

- `PyThreadState_Ensure()`, `PyThreadState_EnsureFromView()`, and `PyThreadState_Release()`
- `PyGILState_Ensure()` e `PyGILState_Release()`
- `PyEval_SaveThread()` e `PyEval_RestoreThread()`
- `Py_BEGIN_ALLOW_THREADS` e `Py_END_ALLOW_THREADS`

These functions should still be used in the free-threaded build to manage thread state even when the GIL is disabled. For example, if you create a thread outside of Python, you must call `PyThreadState_Ensure()` before calling into the Python API to ensure that the thread has a valid Python thread state.

Você deve continuar a chamar `PyEval_SaveThread()` ou `Py_BEGIN_ALLOW_THREADS` em torno de operações de travamento, como E/S ou aquisições de trava, para permitir que outras threads executem o coletor de lixo cíclico.

Protegendo o estado interno das extensões

Sua extensão pode ter um estado interno que foi previamente protegido pela GIL. Talvez seja necessário adicionar uma trava para proteger esse estado. A abordagem dependerá da sua extensão, mas alguns padrões comuns incluem:

- **Caches**: caches globais são uma fonte comum de estado compartilhado. Considere usar uma trava para proteger o cache ou desativá-lo na construção com threads livres se o cache não for crítico para o desempenho.
- **Estado global**: o estado global pode precisar ser protegido por uma trava ou movido para o armazenamento local do thread. C11 e C++11 fornecem `thread_local` ou `_Thread_local` para armazenamento local de thread [https://en.cppreference.com/w/c/language/storage_duration].

Seções críticas

Na construção com threads livres, o CPython fornece um mecanismo chamado “critical sections” (em português, “seções críticas”) para proteger dados que, de outra forma, seriam protegidos pela GIL. Embora os autores da extensão possam não interagir diretamente com a implementação da seção crítica interna, entender seu comportamento é crucial ao usar determinadas funções da API C ou gerenciar o estado compartilhado na construção com threads livres.

O que são seções críticas?

Conceitualmente, seções críticas atuam como uma camada de prevenção de impasse construída sobre mutexes simples. Cada thread mantém uma pilha de seções críticas ativas. Quando uma thread precisa adquirir uma trava associada a uma seção crítica (por exemplo, implicitamente ao chamar uma função de API C segura para thread como `PyDict_SetItem()`, ou explicitamente usando macros), ela tenta adquirir o mutex subjacente.

Usando seções críticas

As principais APIs para usar seções críticas são:

- `Py_BEGIN_CRITICAL_SECTION` e `Py_END_CRITICAL_SECTION` - Para travar um único objeto
- `Py_BEGIN_CRITICAL_SECTION2` e `Py_END_CRITICAL_SECTION2` - Para travar dois objetos simultaneamente

Essas macros devem ser usadas em pares correspondentes e devem aparecer no mesmo escopo C, pois estabelecem um novo escopo local. Essas macros são inoperantes em construções com threads livres, portanto, podem ser adicionadas com segurança a códigos que precisam suportar ambos os tipos de construção.

Um uso comum de uma seção crítica seria travar um objeto ao acessar um atributo interno dele. Por exemplo, se um tipo de extensão tiver um campo de contagem interno, você pode usar uma seção crítica ao ler ou escrever nesse campo:

```
// lê a contagem, retorna uma nova referência ao valor da contag
PyObject *result;
Py_BEGIN_CRITICAL_SECTION(obj);
result = Py_NewRef(obj->count);
Py_END_CRITICAL_SECTION();
return result;

// escreve a contagem, consome referência de new_count
Py_BEGIN_CRITICAL_SECTION(obj);
obj->count = new_count;
Py_END_CRITICAL_SECTION();
```

Como seções críticas funcionam

Ao contrário das travas tradicionais, as seções críticas não garantem acesso exclusivo durante toda a sua duração. Se uma thread travar enquanto mantém uma seção crítica (por exemplo, ao adquirir outro bloqueio ou executar E/S), a seção crítica é temporariamente suspensa — todas as travas são liberadas — e então retomada quando a operação de bloqueio é concluída.

Esse comportamento é semelhante ao que ocorre com a GIL quando uma thread faz uma chamada de bloqueio. As principais diferenças são:

- As seções críticas operam por objeto e não globalmente
- As seções críticas seguem uma disciplina de pilha dentro de cada thread (as macros “begin” e “end” impõem isso, pois devem ser pareadas e estar dentro do mesmo escopo)
- As seções críticas liberam e readquirem automaticamente travas em torno de potenciais operações de bloqueio

Prevenção de impasses

Seções críticas ajudam a evitar impasses de duas maneiras:

1. Se uma thread tenta adquirir uma trava que já está sendo mantida por outra thread, ela primeiro suspende todas as suas seções críticas ativas, liberando temporariamente suas travas
2. Quando a operação de bloqueio for concluída, apenas a seção mais crítica será readquirida primeiro

Isso significa que você não pode depender de seções críticas aninhadas para travar vários objetos simultaneamente, pois a seção crítica interna pode suspender as externas. Em vez disso, use `Py_BEGIN_CRITICAL_SECTION2` para travar dois objetos simultaneamente.

Observe que as travas descritas acima são apenas travas baseadas em `PyMutex`. A implementação da seção crítica não conhece ou afeta outros mecanismos de trava que possam estar em uso, como mutexes POSIX. Observe também que, embora o bloqueio em qualquer `PyMutex` cause a suspensão das seções críticas, apenas os mutexes que fazem parte delas são liberados. Se `PyMutex` for usado sem uma seção crítica, ele não será liberado e, portanto, não terá a mesma capacidade de evitar impasses.

Considerações importantes

- Seções críticas podem liberar temporariamente suas travas, permitindo que outras threads modifiquem os dados protegidos. Tenha cuidado ao

fazer suposições sobre o estado dos dados após operações que possam causar bloqueios.

- Como as travas podem ser temporariamente liberadas (suspensas), entrar em uma seção crítica não garante acesso exclusivo ao recurso protegido durante toda a duração da seção. Se o código dentro de uma seção crítica chamar outra função que bloqueie (por exemplo, adquirir outra trava, executar E/S de bloqueio), todas as travas mantidas pela thread por meio de seções críticas serão liberadas. Isso é semelhante à forma como a GIL pode ser liberada durante chamadas de bloqueio.
- Somente a(s) trava(s) associada(s) à seção crítica mais recentemente inserida (superior) tem garantia de ser(em) mantida(s) em um determinado momento. As travas para seções críticas externas e aninhadas podem ter sido suspensas.
- Você pode travar no máximo dois objetos simultaneamente com essas APIs. Se precisar travar mais objetos, será necessário reestruturar seu código.
- Embora seções críticas não entrem em impasse se você tentar travar o mesmo objeto duas vezes, elas são menos eficientes do que travas reentrantes desenvolvidas especificamente para esse caso de uso.
- Ao usar `Py_BEGIN_CRITICAL_SECTION2`, a ordem dos objetos não afeta a correção (a implementação lida com a prevenção de impasse), mas é uma boa prática sempre travar os objetos em uma ordem consistente.
- Lembre-se de que as macros de seção crítica servem principalmente para proteger o acesso a *objetos Python* que podem estar envolvidos em operações internas do CPython suscetíveis aos cenários de impasses descritos acima. Para proteger o estado de extensão puramente interno, mutexes padrão ou outras primitivas de sincronização podem ser mais apropriados.

Per-Object Locks (`ob_mutex`)

In the free-threaded build, each Python object contains a `ob_mutex` field of type `PyMutex`. This mutex is **reserved for use by the critical section API** (`Py_BEGIN_CRITICAL_SECTION` / `Py_END_CRITICAL_SECTION`).

Aviso

Do **not** lock `ob_mutex` directly with `PyMutex_Lock(&obj->ob_mutex)`. Mixing direct `PyMutex_Lock` calls with the critical section API on the same mutex can cause deadlocks.

Even if your own code never uses critical sections on a particular object type, **CPython internals may use the critical section API on any Python object**.

If your extension type needs its own lock, add a separate **`PyMutex`** field (or another synchronization primitive) to your object struct. **`PyMutex`** is very lightweight, so there is negligible cost to having an additional one.

Construindo extensões para a construção com threads livres

As extensões da API C precisam ser construídas especificamente para a construção com threads livres. As wheels, bibliotecas compartilhadas e binários são indicados por um sufixo `t`.

- pypa/manylinux [https://github.com/pypa/manylinux] supports the free-threaded build, with the `t` suffix, such as `python3.14t`.
- pypa/cibuildwheel [https://github.com/pypa/cibuildwheel] supports building wheels for the free-threaded build of Python 3.14 and newer.

API C limitada e ABI estável

A construção com threads livres não tem suporte atualmente à API C limitada ou à ABI estável. Se você usar setuptools [https://setuptools.pypa.io/en/latest/setuptools.html] para construir sua extensão e atualmente definir `py_limited_api=True`, você pode usar `py_limited_api=not sysconfig.get_config_var("Py_GIL_DISABLED")` para não usar a API limitada ao construir com a construção com threads livres.

Nota

Você precisará construir wheels separadas especificamente para a construção com threads livres. Se você usa atualmente a ABI estável, poderá continuar a construir uma única wheel para várias versões do Python sem threads livres.

Windows

Devido a uma limitação do instalador oficial do Windows, você precisará definir manualmente `Py_GIL_DISABLED=1` ao construir extensões a partir do código-fonte.

Ver também

Portando módulos de extensão para oferecer suporte a threads livres [https://py-free-threading.github.io/porting/] (em inglês): Um guia de portabilidade mantido pela comunidade para autores de extensões.

Migrating to Stable ABI for free threading (`abi3t`)

Starting with the 3.15 release, CPython supports a variant of the Stable ABI that supports free-threaded Python: the Stable ABI for Free-Threaded Builds, or `abi3t` for short. This document describes how to adapt C API extensions to support free threading.

Why do this

The typical reason to use the Stable ABI is to reduce the number of artifacts that you need to build and distribute for each version of your library.

Without the Stable ABI, you must build a separate shared library, and typically a *wheel* distribution, for each feature version of CPython you wish to support. For example, each tag in the following table represents a separate library/wheel:

CPython version	Non-free-threaded	Free-threaded
3.12	`cpython-312`	—
3.13	`cpython-313`	`cpython-313t`
3.14	`cpython-314`	`cpython-314t`
3.15	`cpython-315`	`cpython-315t`

CPython version	Non-free-threaded	Free-threaded
3.16	`cpython-316`	`cpython-316t`
Versões posteriores	`cpython-3`*`XX`*	`cpython-3`*`XX`*`t`

That’s a lot of builds, especially when multiplied by the number of supported platforms.

With the Stable ABI (`abi3`, introduced in CPython 3.2), a single extension (per platform) can cover all *non-free-threaded* builds of CPython:

CPython version	Non-free-threaded	Free-threaded
3.12	`abi3`	—
3.13		`cpython-313t`
3.14		`cpython-314t`
3.15		`cpython-315t`
3.16		`cpython-316t`
Versões posteriores		`cpython-3`*`XX`*`t`

The Stable ABI for free-threaded builds (`abi3t`), introduced in CPython 3.15, does the same for free-threaded builds. And it's compatible with non-free-threaded ones as well:

CPython version	Non-free-threaded	Free-threaded
3.12		—
3.13	`abi3` *	`cpython-313t`
3.14		`cpython-314t`
3.15		
3.16	`abi3t`	
Versões posteriores		

* (As above, the `abi3` extension is compatible with all non-free-threaded builds; even the 3.15+ ones that this table "attributes" to `abi3t`.)

Why *not* do this

There are two main downsides to the Stable ABI.

First, your extension may become slower, since the Stable ABI prioritizes compatibility over performance. The difference is usually not noticeable, and often can be mitigated by using the same source to build both a Stable ABI build and a few version-specific ones for "tier 1" CPython versions.

Second, not all of the C API is available. Extensions need to be ported to build for the Stable ABI, which may be difficult or, in rare cases, impossible.

Specifically, `abi3t` requires APIs added in CPython 3.15. If you want to build your extension for older versions of CPython from the same source, you have two main options:

- Use preprocessor conditionals.

  When following this guide, use `#ifdef Py_TARGET_ABI3T` blocks whenever you are told to do a change that breaks the build on CPython versions you care about. Keep the pre-existing code in `#else` blocks.

  For hand-written C extensions, this approach is reasonable down to CPython 3.12, due to additions introduced in **PEP 697** [https://peps.python.org/pep-0697/]. Keeping compatibility with 3.11 and below may be worth it for code generators (for example, Cython).

- Do not port to `abi3t`, and continue building separate extensions for each version of CPython, until you can drop support for the older versions.

  This is a valid approach. Not all extensions need to switch to `abi3t` right now.

Pré-requisitos

This guide assumes that you have an extension written directly in C (or C++), which you want to port to `abi3t`.

If your extension uses a code generator (like Cython) or language binding (like PyO3), it's best to wait until that tool has support for `abi3t`. If you maintain such a tool, you might be able to adapt the instructions here for your tool.

Non-free-threaded Stable ABI

Your extension should support the non-free-threaded Stable ABI (`abi3`). If not, either port it first, or follow this guide but be prepared to fix issues it does not mention.

Free-threading support

While it's technically not a hard prerequisite, you will most likely want to prepare your extension for free threading before you port it to `abi3t`. See Suporte a extensões da API C para threads livres for instructions.

Ver também

Porting Extension Modules to Support Free-Threading [https://py-free-threading.github.io/porting/]: A community-maintained porting guide for extension authors.

Isolating extension modules

Your module should use multi-phase initialization, and it should either be isolated or limit itself to be loaded at most once per process. If it is not your case, follow Isolando módulos de extensão first. (See the opt-out section for a shortcut.)

Avoiding variable-sized types

If your extension defines variable-sized types (using **`Py_tp_itemsize`** or **`PyTypeObject.tp_itemsize`**), it cannot be ported to `abi3t` 3.15.

Setting up the build

If you use a build tool, search its documentation for "`abi3t`", and follow any instructions to select the ABI. For reference, here are direct links for several popular build tools:

- meson-python [https://mesonbuild.com/meson-python/how-to-guides/limited-api.html#the-abi3t-stable-abi]
- scikit-build-core [https://scikit-build-core.readthedocs.io/en/stable/configuration/#customizing-the-output-wheel]
- Maturin [https://www.maturin.rs/bindings#py_limited_apiabi3]

You may want to verify that the tool set the right flag by temporarily adding the following just after `#include <Python.h>`:

```
#if Py_TARGET_ABI3T+0 <= 0x30f0000
#error "abi3t define is not set!"
#endif
```

This should result in a different error than "`abi3t` define is not set".

Ver também

Building and distributing abi3t extensions [https://py-free-threading.github.io/abi3t/]: Build configuration and wheel testing examples in the community-maintained Python Free-Threading Guide.

Nota

If your build tool doesn't support `abi3t` yet, set the following macro before including `Python.h`:

```
#define Py_TARGET_ABI3T 0x30f0000
```

or specify it as a compiler flag, for example:

```
-DPy_TARGET_ABI3T=0x30f0000
```

Once your extension builds with this setting, it will be compatible with CPython 3.15 and above.

If you set this macro manually, you will later need to name and tag the resulting extension manually as well. This is covered in Tagging and distribution below.

This guide will ask you to make a series of changes. After each one, verify that your extension still builds in the original (non-`abi3t`) configuration, and ideally run tests on all Python versions you support. This will ensure that nothing breaks as you are porting.

Gancho de exportação do módulo

Unless you've done this step already, your extension module defines a module initialization function named `PyInit_<modname>` (where `modname` is the name of your module). You will need to port it to a module export hook, `PyModExport_<modname>`, a feature added in CPython 3.15 in **PEP 793** [https://peps.python.org/pep-0793/].

Your existing init function should look like this (with your own names for `<modname>` and `<moddef>`):

```
PyMODINIT_FUNC
PyInit_<modname>(void)
{
    return PyModuleDef_Init(&<moddef>);
}
```

If there is some code before the `return`, move it to a **`Py_mod_create`** or **`Py_mod_exec`** slot function. See the PyInit documentation for related information.

The function references a `PyModuleDef` object (`<moddef>` in the code above). Its definition should be similar to the following, with different values and perhaps some fields unnnamed or left out:

```
static PyModuleDef <moddef> = {
    PyModuleDef_HEAD_INIT,
    .m_name = "my_module",
    .m_doc = "my docstring",
    .m_size = sizeof(my_state_struct),
    .m_methods = my_methods,
    .m_slots = my_slots,
```

```
    .m_traverse = my_traverse,
    .m_clear = my_clear,
    .m_free = my_free,
};
```

Remove this definition and the `PyInit` function (or put them in an `#ifndef Py_TARGET_ABI3T` block, to retain backwards compatibility), and replace them with the following:

```
PyABIInfo_VAR(abi_info);

static PySlot my_slot_array[] = {
    PySlot_STATIC_DATA(Py_mod_abi, &abi_info),
    PySlot_STATIC_DATA(Py_mod_name, "my_module"),
    PySlot_STATIC_DATA(Py_mod_doc, "my docstring"),
    PySlot_SIZE(Py_mod_state_size, sizeof(my_state_struct)),
    PySlot_STATIC_DATA(Py_mod_methods, my_methods),
    PySlot_STATIC_DATA(Py_mod_slots, my_slots),
    PySlot_FUNC(Py_mod_state_traverse, my_traverse),
    PySlot_FUNC(Py_mod_state_clear, my_clear),
    PySlot_FUNC(Py_mod_state_free, my_free),
    PySlot_END
};

PyMODEXPORT_FUNC
PyModExport_<modname>(void)
{
    return my_slot_array;
}
```

Leave out any fields that were missing (except the new **`Py_mod_abi`**), and substitute your own values.

See the **`PySlot`** and export hook documentation for details on this API.

As in the example, your `PyModExport_` function should *only* return a pointer to static data. If you cannot avoid additional code, refer to the caveats in PyModExport documentation.

Nota

When building for Windows using the Setuptools [https://setuptools.pypa.io/] build tool, removing the `PyInit_<modname>` function may result in the linker error `LINK : error LNK2001: unresolved external symbol PyInit_<modname>`. This is caused by Setuptools passing an `/EXPORT` linker flag, which is redundant since Python 3.15 (see gh-141671 [https://github.com/python/cpython/issues/141671]). A workaround is to add a dummy `PyInit_<modname>` function to your code. Python 3.15+ will never call this function if `PyModExport_<modname>` is present, so it can always fail:

```
// Workaround for https://github.com/pypa/distutils/issues/387
PyMODINIT_FUNC
PyInit_<modname>(void)
{
    PyErr_SetString(PyExc_SystemError,
                    "PyInit_* called for module with PyModExpo
    return NULL;
}
```

(This issue is present in Setuptools 84.0.0; it might be fixed in newer versions.)

Existing slots

If you have a `Py_mod_slots` slot, check the array it refers to. It should be a **`PyModuleDef_Slot`** array like the following:

```
static PyObject *create_module(PyObject *spec, PyModuleDef *def)
static int my_first_module_exec(PyObject *module) { ... }
static int my_second_module_exec(PyObject *module) { ... }

static PyModuleDef_Slot my_slots[] = {
   {Py_mod_gil, Py_MOD_GIL_NOT_USED},
   {Py_mod_multiple_interpreters, Py_MOD_PER_INTERPRETER_GIL_SUP
   {Py_mod_create, my_module_create},
   {Py_mod_exec, my_first_module_exec},
   {Py_mod_exec, my_second_module_exec},
   {0, NULL}
};
```

`py_mod_create`

If you have a `Py_mod_create` entry, make sure the function can be called with `NULL` as its second argument (instead of the `PyModuleDef`, which you are removing). Often, this argument isn't used at all; you can check by renaming it:

```
static PyObject *create_module(PyObject *spec, PyModuleDef *_unu
```

If the argument is used, find a different way to pass in the data. Commonly, the information is static and you can refer to it directly. (If you're reusing a single function for several different modules, consider defining several functions instead.)

Multiple `py_mod_exec`

If you have *more than one* `Py_mod_exec` entry, consolidate them: create a new function that calls the others, and replace existing slots with it.

```
static int my_module_exec(PyObject *module) {
   if (my_first_module_exec(module) < 0) return -1;
   if (my_second_module_exec(module) < 0) return -1;
}

static PyModuleDef_Slot my_slots[] = {
   ...
   /* (remove other Py_mod_exec slots) */
   ...
   {Py_mod_exec, my_module_exec},
   {0, NULL}
};
```

If the functions aren't used elsewhere, you can combine their bodies instead.

Merging slot arrays

Optionally, when you break compatibility with Python 3.14, you may clean up the code by moving slots into the **PySlot** array, and converting the definitions to **PySlot_DATA** and **PySlot_FUNC**:

```
static PySlot my_slot_array[] = {
    ...
    PySlot_DATA(Py_mod_gil, Py_MOD_GIL_NOT_USED),
    PySlot_DATA(Py_mod_multiple_interpreters,
         Py_MOD_PER_INTERPRETER_GIL_SUPPORTED)
    PySlot_FUNC(Py_mod_create, my_module_create),
    PySlot_FUNC(Py_mod_exec, my_module_exec),
    PySlot_END
};
```

If you do this, delete the original **PyModuleDef_Slot** array and its `Py_mod_slots` entry.

Associated `PyModuleDef`

Since the new API does not use a **PyModuleDef** structure, a definition will not be associated with the resulting module. This changes the behavior of the following functions:

- **PyModule_GetDef()**
- **PyType_GetModuleByDef()**

Check your code for these. If you do not use them, you can skip this section.

These functions are typically used for two purposes:

1. To get the definition the module was created with. This is no longer possible using the new API. Modules no longer keep a reference to the definition, so you will need to figure out a different way to pass the relevant data around.

2. To check if a given module object is “yours”. This use case is now served by module tokens – opaque pointers that identify a module. To use a token, declare (or reuse) a unique static variable, for example:

```
static char my_token;
```

and add a pointer to it in a new entry to your module's `PySlot` array:

```
static PySlot my_slot_array[] = {
   ...
   PySlot_STATIC_DATA(Py_mod_token, &my_token),
   PySlot_END
}
```

Then, switch from **`PyModule_GetDef()`** calls such as:

```
PyModuleDef *def = PyModule_GetDef(module);
```

to **`PyModule_GetToken()`** (which uses an output argument and may fail with an exception):

```
void *token;
if (PyModule_GetToken(module, &token) < 0) {
    /* handle error */
}
```

and from **`PyType_GetModuleByDef()`** calls such as:

```
PyObject *module = PyType_GetModuleByDef(type, my_def);
/* handle error; use module */
```

to **`PyType_GetModuleByToken()`** (which returns a strong reference):

```
PyObject *module = PyType_GetModuleByToken(type, my_token);
/* handle error; use module */
Py_XDECREF(module);
```

`PyObject` opaqueness

The **`PyObject`** and **`PyVarObject`** structures are opaque in `abi3t`.

Accessing their members is prohibited. If you do this, switch to getter/setter functions mentioned in their documentation:

- `PyObject.ob_type`
- `PyObject.ob_refcnt`
- `PyVarObject.ob_size`

Also, the *size* of the `PyObject` structures is unknown to the compiler. It can – and *does* – change between different CPython builds.

Nota

While the size is available at runtime (for example as `sys.getsizeof(object())` in Python code), you should resist the temptation to calculate pointer offsets from it. The object memory layout is subject to change in future `abi3t` implementations.

Custom type definitions

Since **`PyObject`** is opaque, the traditional way of defining custom types no longer works:

```
typedef struct {
   PyObject_HEAD  // expands to `PyObject ob_base;` which has ur

   int my_data;
} CustomObject;

static PyType_Spec CustomType_spec = {
   ...
   .basicsize = sizeof(CustomObject),
   ...
};
```

Most likely, all your class definitions, *and* all code that accesses your classes' data, will need to be rewritten. This will probably be the biggest change you need to support `abi3t`.

For each such type, instead of defining a `struct` for the entire instance, define one with only the "additional" fields – ones specific to your class,

not its superclasses:

```
typedef struct {
   int my_data;
} CustomObjectData;
```

Change the name. Almost all code that uses the struct will need to change (notably, pointers to the new structure cannot be cast to/from `PyObject*`), and changing the name will highlight the usages as compiler errors. (If you use `typeof`, C++ `auto`, or similar ways to avoid typing the type name, this won't work. Be extra careful, and consider running tools to detect undefined behavior.)

Then, to create the class, use *negative* `basicsize` to indicate "extra" storage space rather than *total* instance size:

```
static PyType_Spec CustomType_spec = {
   ...
   .basicsize = -sizeof(CustomObjectData), /* note the minus sig
   ...
};
```

If you use **`Py_tp_members`**, set the **`Py_RELATIVE_OFFSET`** flag on each member and specify the **`offset`** relative to your new struct.

Custom type data access

Then comes the hard part: in all code that needs to access this struct, you will need an additional **`PyObject_GetTypeData()`** call to retrieve a `CustomObjectData *` pointer from `PyObject *`:

```
PyObject *obj = ...;
CustomObjectData *data = PyObject_GetTypeData(obj, cls);
```

Note that this call requires the *type object* for your class (`cls`).

If your class is not subclassable (that is, it does not use the **`Py_TPFLAGS_BASETYPE`** flag), `cls` will be `Py_TYPE(obj)`. Otherwise, **DO NOT USE** `Py_TYPE` with **`PyObject_GetTypeData()`**: it might return

memory reserved to an unrelated subclass! For example, if a user makes a subclass like this:

```
class Sub(YourCustomClass):
   __slots__ = ('a', 'b')
```

then `Py_TYPE(obj)` is `Sub`, and the underlying memory may look like this:

```
┌─ PyObject *obj
│               ┌─ the pointer you want
│               │                     ┌─ PyObject_GetTypeData(obj,
▼               ▼                     ▼
┌──────────────┬───┬──────────────────┬───┬──────────────────┬───┬──────────
│ PyObject     │...│ CustomTypeData   │...│ PyObject *a      │...│ PyObject
└──────────────┴───┴──────────────────┴───┴──────────────────┴───┴──────────
```

(Ellipses indicate possible padding. Note that this memory layout is not guaranteed: future versions of Python may add different padding or even switch the order of the structures.)

There are two main ways to get the right class:

- In instance methods, your implementation may use the **`PyCMethod`** signature (and the **`METH_METHOD`** bit in **`PyMethodDef.ml_flags`**), and get the class as the `defining_class` argument.

- Otherwise, give your class a unique static token using the **`Py_tp_token`** slot, and use:

  ```
  PyTypeObject cls;
  if (PyType_GetBaseByToken(Py_TYPE(obj), my_tp_token, &cls) <
      /* handle error */
  }
  CustomObjectData *data = PyObject_GetTypeData(obj, cls);
  ```

  Type tokens work similarly to module tokens covered earlier in this guide.

Avoid build-time conditionals

Check your code for API that identifies the version of Python used to *build* your extension. This no longer corresponds to the Python your extension runs on, so code that uses this information often needs changing. The macros to check for are:

- `PY_VERSION_HEX`, `PY_MAJOR_VERSION`, `PY_MINOR_VERSION`:
  - to get the run-time version, use `Py_Version`;
  - to determine what C API is available, use `Py_TARGET_ABI3T`. This macro is set to the minimum supported version.
- `Py_GIL_DISABLED`: under `abi3t`, this macro is always defined. Code that works with free-threaded Python *should* also work with the GIL enabled (since the GIL can be enabled at run time), and usually *does* (unless it, for some reason, requires more than one attached thread state at one time).

Further code changes

If you are still left with compiler errors or warnings, find a way to fix them. Alas, this guide is limited, and cannot cover all possible code changes extensions may need.

If you find a problem that other extension authors might run into, consider reporting an issue (or sending a pull request) for this guide.

It is possible your issue cannot be fixed for the current version of `abi3t`. In that case, reporting it may help it get prioritized for the next version of CPython.

Tagging and distribution

If you are using a build tool with `abi3t` support, your extension is ready, but you might want to check that it was built correctly.

Extensions built with `abi3t` should have the following extension:

- On Windows: `.pyd` (like any other extension);
- Linux, macOS, and other systems that use the `.so` suffix: `.abi3t.so` (**not** `.cpython-315t.so` or `.abi3.so`). Note that both free-threaded and non-free-threaded builds will load `.abi3t.so` extensions;
- Other systems: consult your distributor, and perhaps update this guide.

If you distribute the extension as a *wheel*, use the following tags:

- Python tag: `cp3`*`XX`*, where *XX* is the minimum Python version the extension is built for. (For example, `cp315` if you set `Py_TARGET_ABI3T` to `0x30f0000`. See Compiling for Stable ABI for more values.)
- ABI tag: `abi3.abi3t`. This is a *compressed tag set* that indicates support for both non-free-threaded and free-threaded builds.

For example, the wheel filename may look like this:

```
myproject-1.0-cp315-abi3.abi3t-macosx_11_0_arm64.whl
```

Ver também

Platform Compatibility Tags [https://packaging.python.org/en/latest/specifications/platform-compatibility-tags/] in the PyPA package distribution metadata.

If the filename or tags are incorrect, fix them.

Testing

Note that when you build an extension compatible with multiple versions of CPython, you should always *test* it with each version it supports (for example, 3.15, 3.16, and so on). The Stable ABI only guarantees *ABI* compatibility; there may also be behavior changes – both intentional ones (covered by **PEP 387** [https://peps.python.org/pep-0387/]) and bugs.

Be sure to run tests on both free-threaded and non-free-threaded builds of CPython.

If they pass, congratulations! You have an `abi3t` extension.

Protocolo de anexação de depuração remota

Este protocolo permite que ferramentas externas se conectem a um processo CPython em execução e executem código Python remotamente.

Anexar a outro processo Python pode exigir permissões ou configurações adicionais, dependendo da plataforma.

Disabling remote debugging

To disable remote debugging support, use any of the following:

- Set the `PYTHON_DISABLE_REMOTE_DEBUG` environment variable to `1` before starting the interpreter.
- Use the `-X disable_remote_debug` command-line option.
- Compile Python with the `--without-remote-debug` build flag.

Requisitos de permissão

Attaching to a running Python process for remote debugging requires special configuration on most platforms. The specific requirements and troubleshooting steps depend on your operating system:

Linux

In general, you can debug your own processes, but there are several common configurations that may disable this. Some Linux distributions enable **ptrace restrictions**, aka “Yama,” as a form of system hardening. Recent versions of the `setpriv` command (util-linux 2.41, released June 2025) let you loosen ptrace restrictions on a per-process basis:

```
setpriv --ptracer any python3
```

(This is configured on the process *being debugged*.) You can also turn off ptrace restrictions for all processes until reboot with:

```
echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope
```

This can also be configured persistently, usually in `/etc/sysctl.d`.

Nota

Disabling `ptrace_scope` reduces system hardening and should only be done in low-security environments.

It is also possible that the `ptrace` system call is disabled because of a security filter. In particular, this was common with older versions of some container software. Docker 19.03 or newer (released 2019) and containerd 1.6.7 or newer (released 2022) will automatically allow usage of the `ptrace` system call inside containers, when running on Linux kernel 4.8 or higher. If you cannot upgrade to these versions, you can create your container with an option like `--security-opt seccomp=unconfined` to disable the system call security filter for that container. This weakens the container's isolation and should only be done in low-security environments.

If you need to trace a process that you *do not* own, you will need superuser access or equivalent. This also applies to processes that have changed their security credentials, e.g., set-user-ID or set-group-ID processes (though this is unusual for Python). Try running the debugging command with `sudo -E`.

Nota

The `CAP_SYS_PTRACE` capability is equivalent to superuser access, in that it allows debugging *any* process, not just your own. You may see advice on the internet suggesting using it to work around ptrace restrictions or system call filters. This may work in practice, as would `sudo`, but this

gives the debugging process much more access than it needs and should only be done in low-security environments.

Finally, note that a process can only have one tracer at a time. If you have already attached to a Python process under `strace`, `gdb`, etc., you won't be able to simultaneously use remote debugging. (Superuser access cannot get around this restriction.)

macOS

By default, macOS disables the ability to debug other processes.

You can modify your Python binary to opt in to being debugged by giving it an **ad-hoc code signature** with an **entitlement** enabling it to be debugged. (An ad-hoc "signature" is just a configuration without any actual cryptographic signature or a need for a certificate or anything else such as an Apple developer program membership.)

The following commands will create a file `get-task-allow.plist` with the necessary entitlement and add it to the Python binary:

```
echo '{"com.apple.security.get-task-allow": true}' | plutil -cor
codesign --sign - --entitlements get-task-allow.plist path/to/bi
```

where `path/to/bin/python3` is the path to your Python binary, which you can find by e.g. running `which python3` or evaluating `sys.base_executable` at the Python REPL. (These instructions are for a non-framework build of Python. Framework builds may need to be configured differently.)

You should then be able to debug your own Python processes started with that binary.

Alternatively, much as with Linux, processes with superuser privileges e.g. `sudo` are not subject to this check and can debug any user's process on the

system (though there are additional checks on specific binaries, such as OS-provided commands, due to System Integrity Protection).

Windows

Para se conectar a outro processo, geralmente é necessário executar a ferramenta de depuração com privilégios administrativos. Inicie o prompt de comando ou o terminal como administrador.

Alguns processos ainda podem estar inacessíveis mesmo com direitos de administrador, a menos que você tenha o privilégio `SeDebugPrivilege` ativado.

Para resolver problemas de acesso a arquivos ou pastas, ajuste as permissões de segurança:

1. Clique com o botão direito do mouse no arquivo ou pasta e selecione **Propriedades**.
2. Acesse a aba **Segurança** para visualizar usuários e grupos com acesso.
3. Clique em **Editar** para modificar as permissões.
4. Selecione sua conta de usuário.
5. Em **Permissões**, marque **Leitura** ou **Controle total** conforme necessário.
6. Clique em **Aplicar** e depois em **OK** para confirmar.

Nota

Certifique-se de ter atendido a todos os Requisitos de permissão antes de prosseguir.

Esta seção descreve o protocolo de baixo nível que permite que ferramentas externas injetem e executem um script Python dentro de um processo CPython em execução.

Este mecanismo forma a base da função `sys.remote_exec()`, que instrui um processo Python remoto a executar um arquivo `.py`. No entanto, esta seção não documenta o uso dessa função. Em vez disso, fornece uma explicação detalhada do protocolo subjacente, que recebe como entrada o `pid` de um processo Python de destino e o caminho para um arquivo-fonte Python a ser executado. Essas informações permitem a reimplementação independente do protocolo, independentemente da linguagem de programação.

Aviso

A execução do script injetado depende de o interpretador atingir um ponto de avaliação seguro. Como resultado, a execução pode ser atrasada dependendo do estado de execução do processo de destino.

Uma vez injetado, o script é executado pelo interpretador dentro do processo de destino na próxima vez que um ponto de avaliação seguro for atingido. Essa abordagem permite recursos de execução remota sem modificar o comportamento ou a estrutura da aplicação Python em execução.

As seções subsequentes fornecem uma descrição passo a passo do protocolo, incluindo técnicas para localizar estruturas do interpretador na memória, acessar campos internos com segurança e disparar a execução de código. Variações específicas da plataforma são indicadas quando aplicável, e exemplos de implementação são incluídos para esclarecer cada operação.

Localizando a estrutura de PyRuntime

O CPython coloca a estrutura `PyRuntime` em uma seção binária dedicada para ajudar ferramentas externas a encontrá-la em tempo de execução. O nome e o formato desta seção variam de acordo com a plataforma. Por exemplo, `.PyRuntime` é usado em sistemas ELF, e `__DATA,__PyRuntime` é

usado no macOS. As ferramentas podem encontrar o deslocamento desta estrutura examinando o binário no disco.

A estrutura `PyRuntime` contém o estado global do interpretador do CPython e fornece acesso a outros dados internos, incluindo a lista de interpretadores, estados de thread e campos de suporte do depurador.

Para trabalhar com um processo Python remoto, um depurador precisa primeiro encontrar o endereço de memória da estrutura `PyRuntime` no processo de destino. Este endereço não pode ser codificado ou calculado a partir de um nome de símbolo, pois depende de onde o sistema operacional carregou o binário.

O método para encontrar `PyRuntime` depende da plataforma, mas os passos são os mesmos em geral:

1. Encontrar o endereço base onde o binário Python ou a biblioteca compartilhada foi carregada no processo de destino.
2. Usar o binário no disco para localizar o deslocamento da seção `.PyRuntime`.
3. Adicionar o deslocamento da seção ao endereço base para calcular o endereço na memória.

As seções abaixo explicam como fazer isso em cada plataforma suportada e incluem código de exemplo.

Linux (ELF)

Para encontrar a estrutura de `PyRuntime` no Linux:

1. Lê o mapa de memória do processo (por exemplo, `/proc/<pid>/maps`) para encontrar o endereço onde o executável Python ou `libpython` foi carregado.
2. Analise os cabeçalhos da seção ELF no binário para obter o deslocamento da seção `.PyRuntime`.
3. Adicione esse deslocamento ao endereço base da etapa 1 para obter o endereço de memória de `PyRuntime`.

A seguir está um exemplo de implementação:

```
def find_py_runtime_linux(pid: int) -> int:
    # Etapa 1: tenta encontrar o executável Python na memória
    binary_path, base_address = find_mapped_binary(
        pid, name_contains="python"
    )

    # Etapa 2: recorre à biblioteca compartilhada se não encontr
    if binary_path is None:
        binary_path, base_address = find_mapped_binary(
            pid, name_contains="libpython"
        )

    # Etapa 3: analisa os cabeçalhos ELF para obter o deslocamer
    section_offset = parse_elf_section_offset(
        binary_path, ".PyRuntime"
    )

    # Etapa 4: calcula o endereço de PyRuntime na memória
    return base_address + section_offset
```

Em sistemas Linux, existem duas abordagens principais para ler memória de outro processo. A primeira é através do sistema de arquivos `/proc`, especificamente lendo de `/proc/[pid]/mem`, que fornece acesso direto à memória do processo. Isso requer permissões apropriadas – seja o mesmo usuário do processo alvo ou ter acesso root. A segunda abordagem é usar a chamada de sistema `process_vm_readv()`, que fornece uma maneira mais eficiente de copiar memória entre processos. Embora a operação `PTRACE_PEEKTEXT` do ptrace também possa ser usada para ler memória, ela é significativamente mais lenta, pois lê apenas uma palavra por vez e requer múltiplas trocas de contexto entre os processos rastreador e rastreado.

Para analisar seções ELF, o processo envolve a leitura e a interpretação das estruturas do formato de arquivo ELF do arquivo binário em disco. O cabeçalho ELF contém um ponteiro para a tabela de cabeçalhos de seção. Cada cabeçalho de seção contém metadados sobre uma seção, incluindo seu nome (armazenado em uma tabela de strings separada), deslocamento e tamanho. Para encontrar uma seção específica, como .PyRuntime, você

precisa percorrer esses cabeçalhos e encontrar o nome da seção. O cabeçalho de seção então fornece o deslocamento onde essa seção existe no arquivo, que pode ser usado para calcular seu endereço de tempo de execução quando o binário é carregado na memória.

Você pode ler mais sobre o formato de arquivo ELF na especificação do ELF [https://en.wikipedia.org/wiki/Executable_and_Linkable_Format].

macOS (Mach-O)

Para encontrar a estrutura de `PyRuntime` no macOS:

1. Chame `task_for_pid()` para obter a porta da tarefa `mach_port_t` para o processo de destino. Este identificador é necessário para ler memória usando APIs como `mach_vm_read_overwrite` e `mach_vm_region`.
2. Examine as regiões da memória para encontrar aquela que contém o executável Python ou `libpython`.
3. Carregue o arquivo binário do disco e analise os cabeçalhos do Mach-O para encontrar a seção chamada `PyRuntime` no segmento `__DATA`. No macOS, os nomes dos símbolos são prefixados automaticamente com um sublinhado, de modo que o símbolo `PyRuntime` aparece como `_PyRuntime` na tabela de símbolos, mas o nome da seção não é afetado.

A seguir está um exemplo de implementação:

```
def find_py_runtime_macos(pid: int) -> int:
    # Etapa 1: obtém acesso à memória do processo
    handle = get_memory_access_handle(pid)

    # Etapa 2: tenta encontrar o executável Python na memória
    binary_path, base_address = find_mapped_binary(
        handle, name_contains="python"
    )

    # Etapa 2: recorre à biblioteca compartilhada se não encontr
    if binary_path is None:
        binary_path, base_address = find_mapped_binary(
            handle, name_contains="libpython"
        )
```

```
    # Etapa 4: analisa os cabeçalhos Mach-O para obter o desloca
    section_offset = parse_macho_section_offset(
        binary_path, "__DATA", "__PyRuntime"
    )

    # Etapa 5: calcula o endereço de PyRuntime na memória
    return base_address + section_offset
```

No macOS, acessar a memória de outro processo requer o uso de APIs e formatos de arquivo específicos do Mach-O. O primeiro passo é obter um identificador `task_port` via `task_for_pid()`, que fornece acesso ao espaço de memória do processo de destino. Esse identificador permite operações de memória por meio de APIs como `mach_vm_read_overwrite()`.

A memória do processo pode ser examinada usando `mach_vm_region()` para varrer o espaço da memória virtual, enquanto `proc_regionfilename()` ajuda a identificar quais arquivos binários são carregados em cada região da memória. Quando o binário ou biblioteca Python é encontrado, seus cabeçalhos Mach-O precisam ser analisados para localizar a estrutura `PyRuntime`.

O formato Mach-O organiza código e dados em segmentos e seções. A estrutura `PyRuntime` reside em uma seção chamada `__PyRuntime` dentro do segmento `__DATA`. O cálculo do endereço de tempo de execução envolve encontrar o segmento `__TEXT`, que serve como endereço base do binário, e então localizar o segmento `__DATA` que contém nossa seção de destino. O endereço final é calculado combinando o endereço base com os deslocamentos de seção apropriados dos cabeçalhos do Mach-O.

Observe que acessar a memória de outro processo no macOS normalmente requer privilégios elevados — acesso root ou direitos de segurança especiais concedidos ao processo de depuração.

Windows (PE)

Para encontrar a estrutura de `PyRuntime` no Windows:

1. Use a API ToolHelp para enumerar todos os módulos carregados no processo de destino. Isso é feito usando funções como CreateToolhelp32Snapshot [https://learn.microsoft.com/pt-br/windows/win32/api/tlhelp32/nf-tlhelp32-createtoolhelp32snapshot], Module32First [https://learn.microsoft.com/pt-br/windows/win32/api/tlhelp32/nf-tlhelp32-module32first] e Module32Next [https://learn.microsoft.com/pt-br/windows/win32/api/tlhelp32/nf-tlhelp32-module32next].
2. Identifique o módulo correspondente a `python.exe` ou `python`*`XY`*`.dll`, onde `X` e `Y` são os números de versão principal e secundária do Python, e registre seu endereço base.
3. Localize a seção `PyRuntim`. Devido ao limite de 8 caracteres do formato PE para nomes de seção (definido como `IMAGE_SIZEOF_SHORT_NAME`), o nome original `PyRuntime` está truncado. Esta seção contém a estrutura `PyRuntime`.
4. Recupere o endereço virtual relativo (RVA) da seção e adicione-o ao endereço base do módulo.

A seguir está um exemplo de implementação:

```
def find_py_runtime_windows(pid: int) -> int:
    # Etapa 1: tenta encontrar o executável Python na memória
    binary_path, base_address = find_loaded_module(
        pid, name_contains="python"
    )

    # Etapa 2: recorre à pythonXY.dll compartilhada se não encor
    # o executável
    if binary_path is None:
        binary_path, base_address = find_loaded_module(
            pid, name_contains="python3"
        )

    # Etapa 3: analisa os cabeçalhos da seção PE para obter o R\
    # da seção .PyRuntime. O nome da seção aparece como "PyRunti
    # devido ao limite de 8 caracteres definido pelo formato PE
    # (IMAGE_SIZEOF_SHORT_NAME).
    section_rva = parse_pe_section_offset(binary_path, "PyRuntim

    # Etapa 4: calcula o endereço de PyRuntime na memória
    return base_address + section_rva
```

No Windows, acessar a memória de outro processo requer o uso de funções da API do Windows, como `CreateToolhelp32Snapshot()` e `Module32First()/Module32Next()`, para enumerar os módulos carregados. A função `OpenProcess()` fornece um identificador para acessar o espaço de memória do processo de destino, permitindo operações de memória por meio de `ReadProcessMemory()`.

A memória do processo pode ser examinada enumerando os módulos carregados para encontrar o binário ou DLL Python. Quando encontrados, seus cabeçalhos PE precisam ser analisados para localizar a estrutura `PyRuntime`.

O formato PE organiza código e dados em seções. A estrutura de `PyRuntime` reside em uma seção chamada “PyRuntim” (truncado de “PyRuntime” devido ao limite de 8 caracteres para nomes do PE). O cálculo do endereço de tempo de execução envolve encontrar o endereço base do módulo a partir da entrada do módulo e, em seguida, localizar nossa seção de destino nos cabeçalhos do PE. O endereço final é calculado combinando o endereço base com o endereço virtual da seção, a partir dos cabeçalhos da seção do PE.

Observe que acessar a memória de outro processo no Windows normalmente requer privilégios apropriados — acesso administrativo ou o privilégio de `SeDebugPrivilege` concedidos ao processo de depuração.

Lendo _Py_DebugOffsets

Depois que o endereço da estrutura `PyRuntime` for determinado, o próximo passo é ler a estrutura `_Py_DebugOffsets` localizada no início do bloco `PyRuntime`.

Esta estrutura fornece deslocamentos de campo específicos da versão, necessários para ler com segurança a memória de estado de thread e do interpretador. Esses deslocamentos variam entre as versões do CPython e devem ser verificados antes do uso para garantir sua compatibilidade.

Para ler e verificar os deslocamentos de depuração, siga estas etapas:

1. Lê a memória do processo de destino, começando no endereço `PyRuntime`, cobrindo o mesmo número de bytes que a estrutura `_Py_DebugOffsets`. Essa estrutura está localizada no início do bloco de memória `PyRuntime`. Seu layout é definido nos cabeçalhos internos do CPython e permanece o mesmo em uma determinada versão secundária, mas pode mudar em versões principais.
2. Verifica se a estrutura contém dados válidos:
   - O campo `cookie` deve corresponder ao marcador de depuração esperado.
   - O campo `version` deve corresponder à versão do interpretador Python usado pelo depurador.
   - Se o depurador ou o processo de destino estiver usando uma versão de pré-lançamento (por exemplo, uma versão alfa, beta ou candidata a lançamento), as versões deverão corresponder exatamente.
   - O campo `free_threaded` deve ter o mesmo valor no depurador e no processo de destino.
3. Se a estrutura for válida, os deslocamentos que ela contém podem ser usados para localizar campos na memória. Se alguma verificação falhar, o depurador deve interromper a operação para evitar a leitura da memória no formato errado.

A seguir está um exemplo de implementação que lê e verifica `_Py_DebugOffsets`:

```
def read_debug_offsets(pid: int, py_runtime_addr: int) -> DebugO
    # Etapa 1: lê a memória do processo de destino no endereço P
    data = read_process_memory(
        pid, address=py_runtime_addr, size=DEBUG_OFFSETS_SIZE
    )

    # Etapa 2: desserializa os bytes brutos em uma estrutura _Py
    debug_offsets = parse_debug_offsets(data)

    # Etapa 3: valida o conteúdo da estrutura
    if debug_offsets.cookie != EXPECTED_COOKIE:
        raise RuntimeError("Invalid or missing debug cookie")
```

```
    if debug_offsets.version != LOCAL_PYTHON_VERSION:
        raise RuntimeError(
            "Mismatch between caller and target Python versions'
        )
    if debug_offsets.free_threaded != LOCAL_FREE_THREADED:
        raise RuntimeError("Mismatch in free-threaded configurat

    return debug_offsets
```

Aviso

Suspensão de processo recomendada

Para evitar condições de corrida e garantir a consistência da memória, é altamente recomendável suspender o processo de destino antes de executar qualquer operação que leia ou grave o estado interno do interpretador. O tempo de execução do Python pode, simultaneamente, alterar as estruturas de dados do interpretador — como criar ou destruir threads — durante a execução normal. Isso pode resultar em leituras ou gravações de memória inválidas.

Um depurador pode suspender a execução anexando-se ao processo com `ptrace` ou enviando um sinal `SIGSTOP`. A execução só deve ser retomada após a conclusão das operações de memória do lado do depurador.

Nota

Algumas ferramentas, como perfiladores ou depuradores baseados em amostragem, podem operar em um processo em execução sem suspensão. Nesses casos, as ferramentas devem ser explicitamente projetadas para lidar com memória parcialmente atualizada ou inconsistente. Para a maioria das implementações de depuradores, suspender o processo continua sendo a abordagem mais segura e robusta.

Localizando o estado de thread e do interpretador

Antes que o código possa ser injetado e executado em um processo Python remoto, o depurador deve escolher uma thread para agendar a execução. Isso é necessário porque os campos de controle usados para realizar a injeção remota de código estão localizados na estrutura `_PyRemoteDebuggerSupport`, que está incorporada em um objeto `PyThreadState`. Esses campos são modificados pelo depurador para solicitar a execução dos scripts injetados.

A estrutura `PyThreadState` representa uma thread em execução dentro de um interpretador Python. Ela mantém o contexto de avaliação da thread e contém os campos necessários para a coordenação do depurador. Localizar um `PyThreadState` válido é, portanto, um pré-requisito essencial para acionar a execução remotamente.

Uma thread é normalmente selecionada com base em sua função ou ID. Na maioria dos casos, a thread principal é usada, mas algumas ferramentas podem direcionar uma thread específica por seu ID nativo. Uma vez escolhida a thread de destino, o depurador deve localizar o interpretador e as estruturas de estado de thread associadas na memória.

As estruturas internas relevantes são definidas da seguinte forma:

- `PyInterpreterState` representa uma instância isolada de um interpretador Python. Cada interpretador mantém seu próprio conjunto de módulos importados, estado embutido e lista de estados de thread. Embora a maioria das aplicações Python use um único interpretador, o CPython oferece suporte a múltiplos interpretadores no mesmo processo.
- `PyThreadState` representa uma thread em execução dentro de um interpretador. Ele contém o estado de execução e os campos de controle usados pelo depurador.

Para localizar uma thread:

1. Use o offset `runtime_state.interpreters_head` para obter o endereço do primeiro interpretador na estrutura `PyRuntime`. Este é o ponto de entrada para a lista vinculada de interpretadores ativos.
2. Use o deslocamento `interpreter_state.threads_main` para acessar o estado da thread principal associada ao interpretador selecionado. Normalmente, esta é a thread mais confiável para ser alvo.
3. Opcionalmente, use o deslocamento `interpreter_state.threads_head` para iterar pela lista encadeada de todos os estados de thread. Cada estrutura `PyThreadState` contém um campo `native_thread_id`, que pode ser comparado a um ID da thread de destino para encontrar uma thread específica.
4. Depois que um `PyThreadState` válido for encontrado, seu endereço poderá ser usado em etapas posteriores do protocolo, como escrever campos de controle do depurador e agendar a execução.

A seguir está um exemplo de implementação que localiza o estado da thread principal:

```
def find_main_thread_state(
    pid: int, py_runtime_addr: int, debug_offsets: DebugOffsets,
) -> int:
    # Etapa 1: lê interpreters_head do PyRuntime
    interp_head_ptr = (
        py_runtime_addr + debug_offsets.runtime_state.interprete
    )
    interp_addr = read_pointer(pid, interp_head_ptr)
    if interp_addr == 0:
        raise RuntimeError("No interpreter found in the target p

    # Etapa 2: lê o ponteiro threads_main do interpretador
    threads_main_ptr = (
        interp_addr + debug_offsets.interpreter_state.threads_ma
    )
    thread_state_addr = read_pointer(pid, threads_main_ptr)
    if thread_state_addr == 0:
        raise RuntimeError("Main thread state is not available")

    return thread_state_addr
```

O exemplo a seguir demonstra como localizar uma thread pelo seu ID de thread nativo:

```
def find_thread_by_id(
    pid: int,
    interp_addr: int,
    debug_offsets: DebugOffsets,
    target_tid: int,
) -> int:
    # Começa em threads_head e percorre a lista vinculada
    thread_ptr = read_pointer(
        pid,
        interp_addr + debug_offsets.interpreter_state.threads_he
    )

    while thread_ptr:
        native_tid_ptr = (
            thread_ptr + debug_offsets.thread_state.native_threa
        )
        native_tid = read_int(pid, native_tid_ptr)
        if native_tid == target_tid:
            return thread_ptr
        thread_ptr = read_pointer(
            pid,
            thread_ptr + debug_offsets.thread_state.next
        )

    raise RuntimeError("Thread with the given ID was not found")
```

Depois que um estado de thread válido for localizado, o depurador pode prosseguir com a modificação de seus campos de controle e agendar a execução, conforme descrito na próxima seção.

Escrevendo informações de controle

Uma vez identificada uma estrutura `PyThreadState` válida, o depurador pode modificar campos de controle dentro dela para agendar a execução de um script Python especificado. Esses campos de controle são verificados periodicamente pelo interpretador e, quando definidos corretamente, acionam a execução do código remoto em um ponto seguro no laço de avaliação.

Cada `PyThreadState` contém uma estrutura `_PyRemoteDebuggerSupport` usada para comunicação entre o depurador e o interpretador. As localizações de seus campos são definidas pela estrutura `_Py_DebugOffsets` e incluem o seguinte:

- `debugger_script_path`: Um buffer de tamanho fixo que detém o caminho completo para um arquivo-fonte Python (`.py`). Este arquivo deve estar acessível e legível pelo processo de destino quando a execução for acionada.
- `debugger_pending_call`: Um sinalizador inteiro. Definir como `1` informa ao interpretador que um script está pronto para ser executado.
- `eval_breaker`: Um campo verificado pelo interpretador durante a execução. Definir o bit 5 (`_PY_EVAL_PLEASE_STOP_BIT`, valor `1U << 5`) neste campo faz com que o interpretador pause e verifique a atividade do depurador.

Para concluir a injeção, o depurador deve executar as seguintes etapas:

1. Escreve o caminho completo do script no buffer `debugger_script_path`.
2. Define `debugger_pending_call` com `1`.
3. Lê o valor atual de `eval_breaker`, define o bit 5 (`_PY_EVAL_PLEASE_STOP_BIT`) e grava o valor atualizado de volta. Isso sinaliza ao interpretador para verificar a atividade do depurador.

A seguir está um exemplo de implementação:

```
def inject_script(
    pid: int,
    thread_state_addr: int,
    debug_offsets: DebugOffsets,
    script_path: str
```

```
) -> None:
    # Calcula o deslocamento base de _PyRemoteDebuggerSupport
    support_base = (
        thread_state_addr +
        debug_offsets.debugger_support.remote_debugger_support
    )

    # Etapa 1: Escreve o caminho do script em debugger_script_pa
    script_path_ptr = (
        support_base +
        debug_offsets.debugger_support.debugger_script_path
    )
    write_string(pid, script_path_ptr, script_path)

    # Etapa 2: Define debugger_pending_call com 1
    pending_ptr = (
        support_base +
        debug_offsets.debugger_support.debugger_pending_call
    )
    write_int(pid, pending_ptr, 1)

    # Etapa 3: Define _PY_EVAL_PLEASE_STOP_BIT (bit 5, valor 1 <
    # em eval_breaker
    eval_breaker_ptr = (
        thread_state_addr +
        debug_offsets.debugger_support.eval_breaker
    )
    breaker = read_int(pid, eval_breaker_ptr)
    breaker |= (1 << 5)
    write_int(pid, eval_breaker_ptr, breaker)
```

Após a configuração desses campos, o depurador pode retomar o processo (caso tenha sido suspenso). O interpretador processará a solicitação no próximo ponto de avaliação seguro, carregará o script do disco e o executará.

É responsabilidade do depurador garantir que o arquivo de script permaneça presente e acessível ao processo de destino durante a execução.

Nota

A execução do script é assíncrona. O arquivo de script não pode ser excluído imediatamente após a injeção. O depurador deve aguardar até que o script injetado produza um efeito observável antes de remover o arquivo. Esse efeito depende do objetivo do script. Por exemplo, um depurador pode aguardar até que o processo remoto se conecte novamente a um soquete antes de remover o script. Uma vez observado esse efeito, é seguro presumir que o arquivo não é mais necessário.

Resumo

Para injetar e executar um script Python em um processo remoto:

1. Localize a estrutura `PyRuntime` na memória do processo de destino.
2. Lê e valida a estrutura `_Py_DebugOffsets` no início de `PyRuntime`.
3. Usa os deslocamentos para localizar um `PyThreadState` válido.
4. Escreve o caminho para um script Python em `debugger_script_path`.
5. Define o sinalizador `debugger_pending_call` com `1`.
6. Define `_PY_EVAL_PLEASE_STOP_BIT` no campo `eval_breaker`.
7. Retoma o processo (se suspenso). O script será executado no próximo ponto de avaliação seguro.

Security and threat model

The remote debugging protocol relies on the same operating system primitives used by native debuggers such as GDB and LLDB. Attaching to a process requires the **same privileges** that those debuggers require, for example `ptrace` / Yama LSM on Linux, `task_for_pid` on macOS, and `SeDebugPrivilege` on Windows. Python does not introduce any new privilege escalation path; if an attacker already possesses the permissions needed to attach to a process, they could equally use GDB to read memory or inject code.

The following principles define what is, and is not, considered a security vulnerability in this feature:

Attaching requires OS-level privileges

On every supported platform the operating system gates cross-process memory access behind privilege checks (`CAP_SYS_PTRACE`, root, or administrator rights). A report that demonstrates an issue only after these privileges have already been obtained is **not** a vulnerability in CPython, since the OS security boundary was already crossed.

Crashes or memory errors when reading a compromised process are not vulnerabilities

A tool that reads internal interpreter state from a target process must trust that memory to be well-formed. If the target process has been corrupted or is controlled by an attacker, the debugger or profiler may crash, produce garbage output, or behave unpredictably. This is the same risk accepted by every `ptrace`-based debugger. Bugs in this category (buffer overflows, segmentation faults, or undefined behaviour triggered by reading corrupted state) are **not** treated as security issues, though fixes that improve robustness are welcome.

Vulnerabilities in the target process are not in scope

If the Python process being debugged has already been compromised, the attacker already controls execution in that process. Demonstrating further impact from that starting point does not constitute a vulnerability in the remote debugging protocol.

When to use `PYTHON_DISABLE_REMOTE_DEBUG`

The environment variable **`PYTHON_DISABLE_REMOTE_DEBUG`** (and the equivalent **`-X disable_remote_debug`** flag) allows operators to disable the in-process side of the protocol as a **defence-in-depth** measure. This may be useful in hardened or sandboxed deployment environments where no debugging or profiling of the process is expected and reducing attack surface is a priority, even though the OS-level privilege checks already prevent unprivileged access.

Setting this variable does **not** affect other OS-level debugging interfaces (`ptrace`, `/proc`, `task_for_pid`, etc.), which remain available according to their own permission models.

Perguntas Frequentes Sobre Python

- Python Geral
- FAQ sobre programação
- FAQ sobre design e histórico
- FAQ de Bibliotecas e Extensões
- FAQ sobre Extensão/Incorporação
- Python no Windows
- FAQ da Interface Gráfica do Usuário
- FAD de “Por que o Python está instalado em meu computador?”

Python Geral

Sumário

- Python Geral
  - Informações gerais
    - O que é Python?
    - O que é a Python Software Foundation?
    - Existem restrições de direitos autorais sobre o uso de Python?
    - Em primeiro lugar, por que o Python foi desenvolvido?
    - Para o que Python é excelente?
    - Como funciona o esquema de numeração de versões do Python?
    - Como faço para obter uma cópia dos fonte do Python?
    - Como faço para obter a documentação do Python?
    - Eu nunca programei antes. Existe um tutorial básico do Python?
    - Existe um grupo de discussão ou lista de discussão dedicada ao Python?
    - Como faço para obter uma versão de teste beta do Python?
    - Como eu envio um relatório de erros e correções para o Python?
    - Existem alguns artigos publicados sobre o Python para que eu possa fazer referência?
    - Existem alguns livros sobre o Python?
    - Onde está armazenado o site www.python.org?
    - Por que o nome Python?
    - Eu tenho que gostar de “Monty Python’s Flying Circus”?
  - Python no mundo real
    - Quão estável é o Python?
    - Quantas pessoas usam o Python?
    - Existe algum projeto significativo feito em Python?

- Quais são os novos desenvolvimentos esperados para o Python no futuro?
- É razoável propor mudanças incompatíveis com o Python?
- O Python é uma boa linguagem para quem está começando na programação agora?

Informações gerais

O que é Python?

O Python é uma linguagem de programação interpretada, interativa e orientada a objetos. O mesmo incorporou módulos, exceções, tipagem dinâmica, tipos de dados dinâmicos de alto nível e classes. Há suporte a vários paradigmas de programação além da programação orientada a objetos, tal como programação procedural e funcional. O Python fornece ao desenvolvedor um poder notável aliado a uma sintaxe simples de clara. Possui interfaces para muitas chamadas e bibliotecas do sistema, bem como para vários sistemas de janelas, e é extensível através de linguagem como o C ou C++. Também é utilizado como linguagem de extensão para aplicativos que precisam de uma interface programável. Finalmente, o Python é portátil: o mesmo pode ser executado em várias variantes do Unix, incluindo Linux e Mac, e no Windows.

Para saber mais, inicie pelo nosso tutorial O tutorial do Python. Os links do Beginner's Guide to Python [https://wiki.python.org/moin/BeginnersGuide] para outros tutoriais introdutórios e recursos da linguagem Python.

O que é a Python Software Foundation?

O Python Software Foundation é uma organização independente e sem fins lucrativos que detém os direitos autorais sobre as versões 2.1 do Python e as mais recentes. A missão do PSF é avançar a tecnologia de código aberto relacionada à linguagem de programação Python e divulgar a utilização do Python. A página inicial do PSF pode ser acessada pelo link a seguir https://www.python.org/psf/.

Doações para o PSF estão isentas de impostos nos EUA. Se utilizares o Python e achares útil, contribua através da página de doação da PSF [https://www.python.org/psf/donations/].

Existem restrições de direitos autorais sobre o uso de Python?

Podemos fazer tudo o que quisermos com os fontes, desde que deixemos os direitos autorais e exibamos esses direitos em qualquer documentação sobre o Python que produzirmos. Se honrarmos as regras dos direitos autorais, não há quaisquer problema em utilizar o Python em versões comerciais, vendê-lo, copiá-lo na forma de código-fonte ou o seu binária (modificado ou não modificado), ou para vender produtos que incorporem o Python de alguma forma. Ainda gostaríamos de saber sobre todo o uso comercial de Python, é claro.

Veja a página da licença [https://docs.python.org/3/license.html] para encontrar mais explicações e um link para o texto completo da licença.

O logotipo do Python é marca registrada e, em certos casos, é necessária permissão para usá-la. Consulte a Política de Uso da Marca comercial [https://www.python.org/psf/trademarks/] para obter mais informações.

Em primeiro lugar, por que o Python foi desenvolvido?

Aqui está um resumo *muito* breve de como que tudo começou, escrito por Guido van Rossum:

> Eu tive vasta experiência na implementação de linguagens interpretada no grupo ABC da CWI e, ao trabalhar com esse grupo, aprendi muito sobre o design de linguagens. Esta é a origem de muitos recursos do Python, incluindo o uso do recuo para o agrupamento de instruções e a inclusão de tipos de dados de alto nível (embora existam diversos detalhes diferentes em Python).
>
> Eu tinha uma série de queixas sobre a linguagem ABC, mas também havia gostado de muitos das suas características. Era impossível estender ABC (ou melhorar a implementação) para remediar minhas

queixas – na verdade, a falta de extensibilidade era um dos maiores problemas. Eu tinha alguma experiência com o uso de Modula-2+ e conversei com os designers do Modula-3 e li o relatório do Modula-3. Modula-3 foi a origem da sintaxe e semântica usada nas exceções, e alguns outros recursos do Python.

Eu estava trabalhando no grupo de sistema operacional distribuído da Amoeba na CWI. Precisávamos de uma maneira melhor de administrar o sistema do que escrevendo programas em C ou scripts para a shell Bourne, uma vez que o Amoeba tinha a sua própria interface de chamada do sistema, que não era facilmente acessível a partir do shell Bourne. Minha experiência com o tratamento de erros em Amoeba me conscientizou da importância das exceções como um recurso das linguagens de programação.

Percebi que uma linguagem de script com uma sintaxe semelhante a da ABC, mas com acesso às chamadas do sistema Amoeba, preencheria a necessidade. Percebi também que seria uma boa escrever uma linguagem específica para o Amoeba, então, decidi que precisava de uma linguagem realmente extensível.

Durante as férias do Natal de 1989, tive bastante tempo disponível e então decidi tentar a construção de algo. Durante o ano seguinte, continuei trabalhando em minhas horas vagas, e o Python foi usado no projeto Amoeba com crescente sucesso, e o feedback dos colegas me fez implementar muitas melhorias.

Em fevereiro de 1991, depois de mais de um ano de desenvolvimento, decidi publicar na USENET. O resto está no arquivo `Misc/HISTORY`.

Para o que Python é excelente?

Python é uma linguagem de programação de propósito geral, de alto nível e que pode ser aplicada em muitos tipos diferentes de problemas.

A linguagem vem com uma grande biblioteca padrão que abrange áreas como processamento de strings (expressões regulares, Unicode, cálculo de diferenças entre arquivos), protocolos de internet (HTTP, FTP, SMTP,

XML-RPC, POP, IMAP), engenharia de software (testes de unidade , registro, criação de perfil, análise de código Python) e interfaces de sistema operacional (chamadas de sistema, sistemas de arquivos, soquetes TCP/IP). Veja o sumário de A biblioteca padrão do Python para ter uma ideia do que está disponível. Uma ampla variedade de extensões de terceiros também está disponível. Consulte o Python Package Index [https://pypi.org] para encontrar pacotes de seu interesse.

Como funciona o esquema de numeração de versões do Python?

As versões de Python são enumeradas como “A.B.C” ou “A.B”:

- *A* é o número da versão principal - sendo incrementada apenas em grandes mudanças na linguagem.
- *B* é o número da versão menor - sendo incrementada apenas para mudanças menos estruturais.
- *C* é o número para micro versão – sendo incrementada apenas para lançamento com correção de bugs.

Nem todos as versões são lançamentos de correções de erros. Na corrida por um novo lançamento de funcionalidade, uma série de versões de desenvolvimento são feitas, denotadas como alfa, beta ou candidata. As versões alfa são lançamentos iniciais (early releases) em que as interfaces ainda não estão finalizadas; não é inesperado ver uma mudança de interface entre duas versões alfa. As betas são mais estáveis, preservando as interfaces existentes, mas possivelmente adicionando novos módulos, e as candidatas a lançamento são congeladas, sem alterações, exceto quando necessário para corrigir erros críticos.

As versões alpha, beta e candidata a lançamento possuem um sufixo adicional:

- O sufixo para uma versão alfa é “aN” para algum número pequeno *N*.
- O sufixo para uma versão beta é “bN” para algum número pequeno *N*.
- O sufixo para um lançamento em versão candidata é “rcN” para algum número pequeno *N*.

Em outras palavras, todas as versões rotuladas como *2.0aN* precedem as versões rotuladas como *2.0bN*, que por sua vez precedem versões rotuladas como *2.0rcN*, e *estas* precedem 2.0.

Também podemos encontrar números de versão com um sufixo "+dev", por exemplo, "3.15.0b3+dev". Estas são versões não lançadas, construídas diretamente do repositório de desenvolvimento do CPython. Após o primeiro lançamento beta ser feito, a versão é incrementada para a próxima versão de recursos, que se torna a versão "a0", por exemplo, "3.16.0a0".

Veja o Developer's Guide [https://devguide.python.org/developer-workflow/development-cycle/] para mais informações sobre o ciclo de desenvolvimento, e a **PEP 387** [https://peps.python.org/pep-0387/] para aprender mais sobre a política de compatibilidade com versões anteriores do Python. Veja também a documentação para `sys.version`, `sys.hexversion`, e `sys.version_info`.

Como faço para obter uma cópia dos fonte do Python?

A última distribuição fonte do Python sempre está disponível no python.org, em https://www.python.org/downloads/. As últimas fontes de desenvolvimento podem ser obtidas em https://github.com/python/cpython/.

A distribuição fonte é um arquivo .tar com .gzip contendo o código-fonte C completo, a documentação formatada com o Sphinx, módulos de biblioteca Python, programas de exemplo e várias peças úteis de software livremente distribuível. A fonte compilará e executará sem a necessidade de configurações extras na maioria das plataformas UNIX.

Consulte a seção Introdução do Guia do Desenvolvedor Python [https://devguide.python.org/setup/] para obter mais informações sobre como obter o código-fonte e compilá-lo.

Como faço para obter a documentação do Python?

A documentação padrão para a versão atualmente estável do Python está disponível em https://docs.python.org/3/. Em EPUB, texto simples e

versões HTML para download também estão disponíveis em https://docs.python.org/3/download.html.

A documentação é escrita em reStructuredText e processada pela ferramenta de documentação Sphinx [https://www.sphinx-doc.org/]. Os fonte do reStructuredText para documentação fazem parte da distribuição fonte do Python.

Eu nunca programei antes. Existe um tutorial básico do Python?

Existem inúmeros tutoriais e livros disponíveis. A documentação padrão inclui O tutorial do Python.

Consulte o Guia do Iniciante [https://wiki.python.org/moin/BeginnersGuide] para encontrar informações para quem está começando agora na programação Python, incluindo uma lista com tutoriais.

Existe um grupo de discussão ou lista de discussão dedicada ao Python?

Existe um grupo de notícias *comp.lang.python*, e uma lista de discussão, python-list [https://mail.python.org/mailman/listinfo/python-list]. O grupo notícias e a lista de discussão são conectados um ou outro – se poderes ler as notícias, não será necessário se inscrever na lista de discussão. *comp.lang.python* possui bastante postagem, recebendo centenas de postagens todos os dias, e os leitores do Usenet geralmente são mais capazes de lidar com esse volume.

Os anúncios de novas versões do software e eventos podem ser encontrados em comp.lang.python.announce, uma lista moderada de baixo tráfego que recebe cerca de cinco postagens por dia. Está disponível como a lista de discussão python-announce [https://mail.python.org/mailman3/lists/python-announce-list.python.org/].

Mais informações sobre outras listas de discussão e grupos de notícias podem ser encontradas em https://www.python.org/community/lists/.

Como faço para obter uma versão de teste beta do Python?

As versões alfa e beta estão disponíveis em https://www.python.org/downloads/. Todos os lançamentos são anunciados nos grupos de notícias comp.lang.python e comp.lang.python.announce e na página inicial do Python em https://www.python.org/; um feed RSS de notícias está disponível.

Você também pode acessar a versão de desenvolvimento do Python através do Git. Veja O Guia do Desenvolvedor Python [https://devguide.python.org/] para detalhes.

Como eu envio um relatório de erros e correções para o Python?

Para relatar um bug ou enviar um patch, use o rastreador de problemas em https://github.com/python/cpython/issues.

Para mais informações sobre como o Python é desenvolvido, consulte o Guia do Desenvolvedor Python [https://devguide.python.org/].

Existem alguns artigos publicados sobre o Python para que eu possa fazer referência?

Provavelmente será melhor citar o seu livro favorito sobre o Python.

O primeiro artigo [https://ir.cwi.nl/pub/18204] sobre Python foi escrito em 1991 e atualmente se encontra bastante desatualizado.

> Guido van Rossum e Jelke de Boer, “Interactively Testing Remote Servers Using the Python Programming Language”, CWI Quarterly, Volume 4, Edição 4 (dezembro de 1991), Amsterdam, pp. 283–303.q

Existem alguns livros sobre o Python?

Sim, há muitos publicados e muitos outros que estão sendo nesse momento escritos!! Veja o wiki python.org em

https://wiki.python.org/moin/PythonBooks para obter uma listagem.

Você também pode pesquisar livrarias online sobre “Python” e filtrar as referências a respeito do Monty Python; ou talvez procure por “Python” e “linguagem”.

Onde está armazenado o site www.python.org?

A infraestrutura do projeto Python está localizada em todo o mundo e é gerenciada pela equipe de infraestrutura do Python. Detalhes aqui [https://infra.psf.io].

Por que o nome Python?

Quando o Guido van Rossum começou a implementar o Python, o mesmo também estava lendo os scripts publicados do “Monty Python’s Flying Circus” [https://en.wikipedia.org/wiki/Monty_Python], uma série de comédia da BBC da década de 1970. Van Rossum pensou que precisava de um nome curto, único e ligeiramente misterioso, então resolveu chamar a sua linguagem de Python.

Eu tenho que gostar de “Monty Python’s Flying Circus”?

Não, mas isso ajuda. :)

Python no mundo real

Quão estável é o Python?

Muito estável. Novos lançamentos estáveis são divulgados aproximadamente de 6 a 18 meses desde 1991, e isso provavelmente continuará. A partir da versão 3.9, o Python terá um novo grande lançamento a cada 12 meses (**PEP 602** [https://peps.python.org/pep-0602/]).

Os desenvolvedores lançam versões bugfix de versões mais antigas, então a estabilidade dos lançamentos existentes melhora gradualmente. As

liberações de correções de erros, indicadas por um terceiro componente do número da versão (por exemplo, 3.5.3, 3.6.2), são gerenciadas para estabilidade; somente correções para problemas conhecidos são incluídas em uma versão de correções de erros, e é garantido que as interfaces permanecerão as mesmas durante uma série de liberações de correções de erros.

As últimas versões estáveis podem sempre ser encontradas na página de download do Python [https://www.python.org/downloads/]. O Python 3.x é a versão recomendada e suportada pela maioria das bibliotecas amplamente utilizadas. Python 2.x **não é mais mantido** [https://peps.python.org/pep-0373/].

Quantas pessoas usam o Python?

Provavelmente existem milhões de usuários, embora seja difícil obter uma contagem exata.

O Python está disponível para download gratuito, portanto, não há números de vendas, e o mesmo está disponível em vários diferentes sites e é empacotado em muitas distribuições Linux, portanto, utilizar as estatísticas de downloads não seria a melhor forma para contabilizarmos a base de usuários.

O grupo de notícias comp.lang.python é bastante ativo, mas nem todos os usuários Python postam no grupo ou mesmo o leem regularmente.

Existe algum projeto significativo feito em Python?

Veja a lista em https://www.python.org/about/success para obter uma listagem de projetos que usam o Python. Consultar as conferências passadas do Python [https://www.python.org/community/workshops/] revelará as contribuições de várias empresas e de diferentes organizações.

Os projetos Python de alto perfil incluem o gerenciador de lista de e-mail Mailman [https://www.list.org] e o servidor de aplicativos Zope [https://www.zope.dev]. Várias distribuições Linux, mais notavelmente o Red Hat [https://www.redhat.com], escreveram parte ou a totalidade dos seus instaladores e

software de administração do sistema em Python. Empresas que usam Python internamente incluem Google, Yahoo e Lucasfilm Ltd.

Quais são os novos desenvolvimentos esperados para o Python no futuro?

Consulte https://peps.python.org/ para ver a lista de propostas de aprimoramento do python (PEPs). As PEPs são documentos de design que descrevem novos recursos que foram sugeridos para o Python, fornecendo uma especificação técnica concisa e a sua lógica. Procure uma PEP intitulado de "Python X.Y Release Schedule", onde X.Y é uma versão que ainda não foi lançada publicamente.

Novos desenvolvimentos são discutidos na lista de discussão python-dev [https://mail.python.org/mailman3/lists/python-dev.python.org/].

É razoável propor mudanças incompatíveis com o Python?

Normalmente não. Já existem milhões de linhas de código Python em todo o mundo, de modo que qualquer alteração na linguagem que invalide mais de uma fração muito pequena dos programas existentes deverá ser desaprovada. Mesmo que possamos fornecer um programa de conversão, ainda haverá o problema de atualizar toda a documentação; muitos livros foram escritos sobre o Python, e não queremos invalidá-los todos de uma vez só.

Fornecer um caminho de atualização gradual será necessário se um recurso precisar ser alterado. A **PEP 5** [https://peps.python.org/pep-0005/] descreve o procedimento e em seguida introduz alterações incompatíveis com versões anteriores ao mesmo tempo em que minimiza a interrupção dos usuários.

O Python é uma boa linguagem para quem está começando na programação agora?

Sim.

Ainda é bastante comum que os alunos iniciem com uma linguagem procedimental e estaticamente tipada como Pascal e o C ou um subconjunto do C++ ou do Java. Os alunos podem ser melhor atendidos ao aprender Python como sua primeira linguagem. Python possui uma sintaxe muito simples e consistente e uma grande quantidade de bibliotecas padrão e, o mais importante, o uso do Python em um curso de programação para iniciantes permite aos alunos se concentrarem em habilidades de programação importantes, como a decomposição do problema e o design do tipo de dados. Com Python, os alunos podem ser introduzidos rapidamente em conceitos básicos, como repetições e procedimentos. Provavelmente os mesmos até poderão trabalhar com objetos definidos por ele mesmos logo em seu primeiro curso.

Para um aluno que nunca programou antes, usar uma linguagem estaticamente tipado parece não que não é natural. Isso apresenta uma complexidade adicional que o aluno deverá dominar e geralmente retarda o ritmo do curso. Os alunos estão tentando aprender a pensar como um computador, decompor problemas, projetar interfaces consistentes e encapsular dados. Embora aprender a usar uma linguagem tipicamente estática seja importante a longo prazo, não é necessariamente o melhor tópico a ser abordado no primeiro momento de um curso de programação.

Muitos outros aspectos do Python fazem do mesmo uma excelente linguagem para quem está aprendendo a programar. Como Java, Python possui uma biblioteca padrão grande para que os estudantes possam receber projetos de programação muito cedo no curso e que possam *fazer* trabalhos úteis. As atribuições não estão restritas à calculadora padrão de quatro funções e os programas para verificar o peso. Ao usar a biblioteca padrão, os alunos podem ter a satisfação de trabalhar em aplicações reais à medida que aprendem os fundamentos da programação. O uso da biblioteca padrão também ensina os alunos sobre a reutilização de código. Os módulos de terceiros, como o PyGame, também são úteis para ampliar o alcance dos estudantes.

O interpretador interativo do Python permite aos alunos testarem recursos da linguagem enquanto estão programando. Os mesmos podem manter uma janela com o interpretador executado enquanto digitam o fonte do seu

programa numa outra janela. Se eles não conseguirem se lembrar dos métodos de uma lista, eles podem fazer algo assim:

```
>>> L = []
>>> dir(L)
['__add__', '__class__', '__contains__', '__delattr__', '__delit
'__dir__', '__doc__', '__eq__', '__format__', '__ge__',
'__getattribute__', '__getitem__', '__gt__', '__hash__', '__iadc
'__imul__', '__init__', '__iter__', '__le__', '__len__', '__lt__
'__mul__', '__ne__', '__new__', '__reduce__', '__reduce_ex__',
'__repr__', '__reversed__', '__rmul__', '__setattr__', '__setite
'__sizeof__', '__str__', '__subclasshook__', 'append', 'clear',
'copy', 'count', 'extend', 'index', 'insert', 'pop', 'remove',
'reverse', 'sort']
>>> [d for d in dir(L) if '__' not in d]
['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert'

>>> help(L.append)
Help on built-in function append:

append(...)
    L.append(object) -> None -- append object to end

>>> L.append(1)
>>> L
[1]
```

Com o interpretador, a documentação nunca está longe do aluno quando estão programando.

Há também boas IDEs para o Python. O IDLE é uma IDE multiplataforma para o Python e que foi escrito em Python usando o Tkinter. Os usuários do Emacs estarão felizes em saber que existe um ótimo modo Python para Emacs. Todos esses ambientes de programação fornecem destaque de sintaxe, recuo automático e acesso ao interpretador interativo durante o tempo de desenvolvimento. Consulte o wiki do Python [https://wiki.python.org/moin/PythonEditors] para obter uma lista completa dos ambientes de desenvolvimento para o Python.

Se você quiser discutir o uso do Python na educação, poderás estar interessado em se juntar à lista de discussão edu-sig [https://www.python.org/community/sigs/current/edu-sig].

FAQ sobre programação

Sumário

- FAQ sobre programação
  - General questions
    - Is there a source code-level debugger with breakpoints and single-stepping?
    - Existem ferramentas para ajudar a encontrar bugs ou fazer análise estática de desempenho?
    - Como posso criar um binário independente a partir de um script Python?
    - Existem padrões para a codificação ou um guia de estilo utilizado pela comunidade Python?
  - Core language
    - Porque recebo o erro UnboundLocalError quando a variável possui um valor associado?
    - Quais são as regras para variáveis locais e globais em Python?
    - Por que os lambdas definidos em um laço com valores diferentes retornam o mesmo resultado?
    - Como definir variáveis globais dentro de módulos?
    - Quais são as “melhores práticas” quando fazemos uso da importação de módulos?
    - Por que os valores padrão são compartilhados entre objetos?
    - Como passo parâmetros opcionais ou parâmetros nomeados de uma função para outra?
    - Qual a diferença entre argumentos e parâmetros?
    - Por que ao alterar a lista ‘y’ também altera a lista ‘x’?
    - Como escrevo uma função com parâmetros de saída (chamada por referência)?
    - Como fazer uma função de ordem superior em Python?
    - Como faço para copiar um objeto no Python?

    - Como posso encontrar os métodos ou atributos de um objeto?
    - Como que o meu código pode descobrir o nome de um objeto?
    - O que há com a precedência do operador vírgula?
    - Existe um equivalente ao operador ternário “?:” do C?
    - É possível escrever instruções de uma só linha ofuscadas em Python?
    - O que a barra(/) na lista de parâmetros de uma função significa?
  - Números e strings
    - Como faço para especificar números inteiros hexadecimais e octais?
    - Por que -22 // 10 retorna -3?
    - Como obtenho um atributo de um literal int em vez de SyntaxError?
    - Como faço para converter uma string em um número?
    - Como faço para converter um número em uma string?
    - Como faço para modificar uma string internamente?
    - Como faço para invocar funções/métodos através de strings?
    - Existe um equivalente em Perl `chomp()` para remover linhas novas ao final de strings?
    - Existe uma função `scanf()` ou `sscanf()` ou algo equivalente?
    - O que significa o erro `UnicodeDecodeError` ou `UnicodeEncodeError`?
    - Posso terminar uma string bruta com um número ímpar de contrabarras?
  - Desempenho
    - Meu programa está muito lento. Como faço para melhorar o desempenho?
    - Qual é a maneira mais eficiente de concatenar muitas strings?
  - Sequences (tuples/lists)
    - Como faço para converter tuplas em listas?
    - O que é um índice negativo?

    - Como que eu itero uma sequência na ordem inversa?
    - Como que removo itens duplicados de uma lista?
    - How do you remove multiple items from a list?
    - Como fazer um vetor em Python?
    - Como faço para criar uma lista multidimensional?
    - Como eu aplico um método ou função para uma sequência de objetos?
    - Porque uma tupla[i] += ['item'] levanta uma exceção quando a adição funciona?
    - Quero fazer uma ordenação confusa: você pode fazer uma transformação schwartziana em Python?
    - Como eu posso ordenar uma lista pelos valores de outra lista?
  - Objetos
    - O que é uma classe?
    - O que é um método?
    - O que é o self?
    - Como eu verifico se um objeto é uma instância de uma dada classe ou de uma subclasse dela?
    - O que é delegação?
    - Como eu chamo um método definido em uma classe base a partir de uma classe derivada que a estende?
    - Como eu posso organizar meu código para facilitar a troca da classe base?
    - Como faço para criar dados de classe estáticos e métodos de classe estáticos?
    - Como eu posso sobrecarregar construtores (ou métodos) em Python?
    - Eu tentei usar __spam e recebi um erro sobre _SomeClassName__spam.
    - Minha classe define __del__, mas o mesmo não é chamado quando eu excluo o objeto.
    - Como eu consigo pegar uma lista de todas as instâncias de uma dada classe?
    - Por que o resultado de **`id()`** aparenta não ser único?
    - Quando eu posso depender dos testes de identidade com o operador *is*?

        - Como uma subclasse pode controlar quais dados são armazenados em uma instância imutável?
        - Como faço para armazenar em cache as chamadas de um método?
    - Módulos
        - Como faço para criar um arquivo .pyc?
        - Como encontro o nome do módulo atual?
        - Como posso ter módulos que se importam mutuamente?
        - __import__('x.y.z') retorna <módulo 'x'>; como faço para obter z?
        - Quando eu edito um módulo importado e o reimporto, as mudanças não aparecem. Por que isso acontece?

General questions

Is there a source code-level debugger with breakpoints and single-stepping?

Sim.

Vários depuradores para Python estão descritos abaixo, e a função embutida `breakpoint()` permite que você caia em qualquer um desses pontos.

O módulo pdb é um depurador em modo Console simples, mas adequado, para o Python. Faz parte da biblioteca padrão do Python e está documentado no `manual de referencia da biblioteca`. Você também pode construir do seu próprio depurador usando o código do pdb como um exemplo.

The IDLE interactive development environment, which is part of the standard Python distribution (normally available as `idlelib`), includes a graphical debugger.

O PythonWin é uma IDE feita para o Python que inclui um depurador gráfico baseado no pdb. O depurador do PythonWin colore os pontos de interrupção e tem alguns recursos legais, como a depuração de programas

que não são PythonWin. O PythonWin está disponível como parte do projeto pywin32 [https://github.com/mhammond/pywin32] e como parte da distribuição ActivePython [https://www.activestate.com/products/python/].

Eric [https://eric-ide.python-projects.org/] é uma IDE construída com PyQt e o componente de edição Scintilla.

trepan3k [https://github.com/rocky/python3-trepan/] é um depurador similar ao gdb.

Visual Studio Code [https://code.visualstudio.com/] é uma IDE com ferramentas de depuração integrada com softwares de controle de versão.

Há uma série de IDE comerciais para desenvolvimento com o Python que inclui depuradores gráficos. Dentre tantas temos:

- Wing IDE [https://wingware.com/]
- PyCharm [https://www.jetbrains.com/pycharm/]

Existem ferramentas para ajudar a encontrar bugs ou fazer análise estática de desempenho?

Sim.

Ruff [https://docs.astral.sh/ruff/], Pylint [https://pylint.readthedocs.io/] and Pyflakes [https://github.com/PyCQA/pyflakes] do basic checking that will help you catch bugs sooner.

Static type checkers such as mypy [https://mypy-lang.org/], ty [https://docs.astral.sh/ty/], Pyrefly [https://pyrefly.org/], and pytype [https://github.com/google/pytype] can check type hints in Python source code.

Como posso criar um binário independente a partir de um script Python?

Você não precisa possuir a capacidade de compilar o código Python para C se o que deseja é um programa autônomo que os usuários possam baixar e executar sem ter que instalar a distribuição Python primeiro. Existem várias

ferramentas que determinam o conjunto de módulos exigidos por um programa e vinculam esses módulos ao binário do Python para produzir um único executável.

One is to use the freeze tool, which is included in the Python source tree as Tools/freeze [https://github.com/python/cpython/tree/3.15/Tools/freeze]. It converts Python byte code to C arrays; with a C compiler you can embed all your modules into a new program, which is then linked with the standard Python modules.

A *freeze* trabalha percorrendo seu código recursivamente, procurando por instruções de importação (ambas as formas), e procurando por módulos tanto no caminho padrão do Python, quanto por módulos embutidos no diretório fonte. Ela então transforma o *bytecode* de módulos Python em código C (inicializadores de vetor que podem ser transformados em objetos código usando o módulo *marshal*), e depois cria um arquivo de configurações personalizado que só contém os módulos embutidos usados no programa. A ferramenta então compila os códigos C e os vincula como o resto do interpretador Python, formando um binário autônomo que funciona exatamente como seu *script*.

Os pacotes a seguir podem ajudar com a criação dos executáveis do console e da GUI:

- Nuitka [https://nuitka.net/] (Multiplataforma)
- PyInstaller [https://pyinstaller.org/] (Multiplataforma)
- PyOxidizer [https://pyoxidizer.readthedocs.io/en/stable/] (Multiplataforma)
- cx_Freeze [https://marcelotduarte.github.io/cx_Freeze/] (Multiplataforma)
- py2app [https://github.com/ronaldoussoren/py2app] (somente macOS)
- py2exe [https://www.py2exe.org/] (somente Windows)

Existem padrões para a codificação ou um guia de estilo utilizado pela comunidade Python?

Sim. O guia de estilo esperado para módulos e biblioteca padrão possui o nome de PEP8 e você pode acessar a sua documentação em **PEP 8** [https://peps.python.org/pep-0008/].

Core language

Porque recebo o erro UnboundLocalError quando a variável possui um valor associado?

Pode ser uma surpresa obter a exceção `UnboundLocalError` em um código previamente funcional quando adicionamos uma instrução de atribuição em algum lugar no corpo de uma função.

Este código:

```
>>> x = 10
>>> def bar():
...     print(x)
...
>>> bar()
10
```

funciona, mas este código:

```
>>> x = 10
>>> def foo():
...     print(x)
...     x += 1
```

resulta em uma **`UnboundLocalError`**:

```
>>> foo()
Traceback (most recent call last):
  ...
UnboundLocalError: cannot access local variable 'x' where it is
```

Isso acontece porque, quando atribuímos um valor a uma variável em determinado escopo, essa variável torna-se local desse escopo, acabando por esconder qualquer outra variável de mesmo nome no escopo externo. Como a última instrução em foo atribui um novo valor a `x`, o interpretador a reconhece como uma variável local. Consequentemente, quando o

`print(x)` anterior tentar exibir a variável local não inicializada, um erro aparece.

No exemplo acima, podemos acessar a variável do escopo externo declarando-a como global:

```
>>> x = 10
>>> def foobar():
...     global x
...     print(x)
...     x += 1
...
>>> foobar()
10
```

Esta declaração explícita é necessária para lembrarmos que estamos modificando o valor da variável no escopo externo (ao contrário da situação superficialmente análoga com variáveis de classe e instância):

```
>>> print(x)
11
```

Podemos fazer algo parecido num escopo aninhado usando a palavra reservada **`nonlocal`**:

```
>>> def foo():
...    x = 10
...    def bar():
...        nonlocal x
...        print(x)
...        x += 1
...    bar()
...    print(x)
...
>>> foo()
10
11
```

Quais são as regras para variáveis locais e globais em Python?

Em Python, as variáveis que são apenas utilizadas (referenciadas) dentro de uma função são implicitamente globais. Se uma variável for associada a um valor em qualquer lugar dentro do corpo da função, presume-se que a mesma seja local, a menos que seja explicitamente declarada como global.

Embora um pouco surpreendente no início, um momento de consideração explica isso. Por um lado, exigir `global` para variáveis atribuídas fornece uma barreira contra efeitos colaterais indesejados. Por outro lado, se `global` fosse necessário para todas as referências globais, você estaria usando `global` o tempo todo. Você teria que declarar como global todas as referências a uma função embutida ou a um componente de um módulo importado. Essa desordem anularia a utilidade da declaração de `global` para identificar efeitos colaterais.

Por que os lambdas definidos em um laço com valores diferentes retornam o mesmo resultado?

Assume you use a for loop to define a few different lambdas (or even plain functions), for example:

```
>>> squares = []
>>> for x in range(5):
...     squares.append(lambda: x**2)
```

Isso oferece uma lista que contém 5 lambdas que calculam `x**2`. Você pensar que, quando invocado, os mesmos retornam, respectivamente, `0`, `1`, `4`, `9`, e `16`. No entanto, quando realmente tentar, vai ver que todos retornam `16`:

```
>>> squares[2]()
16
>>> squares[4]()
16
```

This happens because `x` is not local to the lambdas, but is defined in the outer scope, and it is accessed when the lambda is called — not when it is defined. At the end of the loop, the value of `x` is `4`, so all the functions now

return `4**2`, that is `16`. You can also verify this by changing the value of `x` and see how the results of the lambdas change:

```
>>> x = 8
>>> squares[2]()
64
```

Para evitar isso, é necessário salvar os valores nas variáveis locais para os lambdas, para que eles não dependam do valor de `x` global:

```
>>> squares = []
>>> for x in range(5):
...     squares.append(lambda n=x: n**2)
```

Aqui, `n=x` cria uma nova variável `n` local para o lambda e é calculada quando o lambda é definido para que ele tenha o mesmo valor que `x` tem nesse ponto no laço. Isso significa que o valor de `n` será `0` no primeiro "ciclo" do lambda, `1` no segundo "ciclo", `2` no terceiro, e assim por diante. Portanto, cada lambda agora retornará o resultado correto:

```
>>> squares[2]()
4
>>> squares[4]()
16
```

Observe que esse comportamento não é peculiar dos lambdas, o mesmo também ocorre com as funções regulares.

Como definir variáveis globais dentro de módulos?

A maneira canônica de compartilhar informações entre módulos dentro de um único programa é criando um módulo especial (geralmente chamado de config ou cfg). Basta importar o módulo de configuração em todos os módulos da sua aplicação; o módulo ficará disponível como um nome global. Como há apenas uma instância de cada módulo, todas as alterações feitas no objeto do módulo se refletem em todos os lugares. Por exemplo:

config.py:

```
x = 0   # Valor padrão para a configuração de 'x'
```

mod.py:

```
import config
config.x = 1
```

main.py:

```
import config
import mod
print(config.x)
```

Note que usar um módulo também é a base para a implementação do padrão de projeto Singleton, pela mesma razão.

Quais são as "melhores práticas" quando fazemos uso da importação de módulos?

Em geral, não use `from nomemódulo import *`. Isso desorganiza o espaço de nomes do importador e torna muito mais difícil para as ferramentas de análise estática detectarem nomes indefinidos.

Faça a importação de módulos na parte superior do arquivo. Isso deixa claro quais outros módulos nosso código necessita e evita dúvidas sobre, por exemplo, se o nome do módulo está no escopo. Usar uma importação por linha facilita a adição e exclusão de importações de módulos, porém, usar várias importações num única linha, ocupa menos espaço da tela.

É uma boa prática importar os módulos na seguinte ordem:

1. standard library modules – such as `sys`, `os`, `argparse`, `re`
2. third-party library modules (anything installed in Python's site-packages directory) – such as dateutil [https://pypi.org/project/dateutil/], requests [https://pypi.org/project/requests/], tzdata [https://pypi.org/project/tzdata/]
3. módulos desenvolvidos localmente

Às vezes, é necessário transferir as importações para uma função ou classe para evitar problemas com importação circular. Gordon McMillan diz:

As importações circulares vão bem onde ambos os módulos utilizam a forma de importação “import <módulo>”. Elas falham quando o 2º módulo quer pegar um nome do primeiro (“from módulo import nome”) e a importação está no nível superior. Isso porque os nomes no primeiro ainda não estão disponíveis, porque o 1º módulo está ocupado importando o 2º.

Nesse caso, se o segundo módulo for usado apenas numa função, a importação pode ser facilmente movida para dentro do escopo dessa função. No momento em que a importação for chamada, o primeiro módulo terá finalizado a inicialização e o segundo módulo poderá ser importado sem maiores complicações.

Também poderá ser necessário mover as importações para fora do nível superior do código se alguns dos módulos forem específicos de uma determinada plataforma (SO). Nesse caso, talvez nem seja possível importar todos os módulos na parte superior do arquivo. Nessas situações devemos importar os módulos que são específicos de cada plataforma antes de necessitar utilizar os mesmos.

Apenas mova as importações para um escopo local, como dentro da definição de função, se for necessário resolver algum tipo de problema, como, por exemplo, evitar importações circulares ou tentar reduzir o tempo de inicialização do módulo. Esta técnica é especialmente útil se muitas das importações forem desnecessárias, dependendo de como o programa é executado. Também podemos desejar mover as importações para uma função se os módulos forem usados somente nessa função. Note que carregar um módulo pela primeira vez pode ser demorado devido ao tempo de inicialização de cada módulo, no entanto, carregar um módulo várias vezes é praticamente imperceptível, tendo somente o custo de processamento de pesquisas no dicionário de nomes. Mesmo que o nome do módulo tenha saído do escopo, o módulo provavelmente estará disponível em `sys.modules`.

Por que os valores padrão são compartilhados entre objetos?

Este tipo de erro geralmente pega programadores neófitos. Considere esta função:

```
def foo(meudict={}):  # Perigo: referência compartilhada a um di
    ... faz alguma coisa ...
    meudict[chave] = valor
    return meudict
```

Na primeira vez que chamar essa função, `meudict` vai conter um único item. Na segunda vez, `meudict` vai conter dois itens porque, quando `foo()` começar a ser executado, `meudict` começará com um item já existente.

Muitas vezes, espera-se que ao invocar uma função sejam criados novos objetos referente aos valores padrão. Isso não é o que acontece. Os valores padrão são criados exatamente uma vez, quando a função está sendo definida. Se esse objeto for alterado, como o dicionário neste exemplo, as chamadas subsequentes para a essa função se referirão a este objeto alterado.

Por definição, objetos imutáveis, como números, strings, tuplas e o `None`, estão protegidos de sofrerem alteração. Alterações em objetos mutáveis, como dicionários, listas e instâncias de classe, podem levar à confusão.

Por causa desse recurso, é uma boa prática de programação para evitar o uso de objetos mutáveis contendo valores padrão. Em vez disso, utilize `None` como o valor padrão e dentro da função, verifique se o parâmetro é `None` e crie uma nova lista, dicionário ou o que quer que seja. Por exemplo, escreva o seguinte código:

```
def foo(meudict={}):
    ...
```

mas:

```
def foo(meudict=None):
    if meudict is None:
        meudict = {}  # cria um novo dicionário para o espaço de
```

Esse recurso pode ser útil. Quando se tem uma função que consome muito tempo para calcular, uma técnica comum é armazenar em cache os parâmetros e o valor resultante de cada chamada para a função e retornar o valor em cache se o mesmo valor for solicitado novamente. Isso se chama "memoizar", e pode ser implementado da seguinte forma:

```
# Chamadores só podem fornecer dois parâmetros e opcionalmente p
def expensive(arg1, arg2, *, _cache={}):
    if (arg1, arg2) in _cache:
        return _cache[(arg1, arg2)]

    # Cacula o valor
    result = ... cálculo custoso ...
    _cache[(arg1, arg2)] = result           # Armazena o resulta
    return result
```

Pode-se usar uma variável global contendo um dicionário ao invés do valor padrão; isso é uma questão de gosto.

Como passo parâmetros opcionais ou parâmetros nomeados de uma função para outra?

Colete os argumentos usando os especificadores `*` ou `**` na lista de parâmetros da função. Isso faz com que os argumentos posicionais como tupla e os argumentos nomeados sejam passados como um dicionário. Você pode, também, passar esses argumentos ao invocar outra função usando `*` e `**`:

```
def f(x, *args, **kwargs):
    ...
    kwargs['width'] = '14.3c'
    ...
    g(x, *args, **kwargs)
```

Qual a diferença entre argumentos e parâmetros?

Parâmetros são definidos pelos nomes que aparecem na definição da função, enquanto argumentos são os valores que serão passados para a

função no momento em que esta estiver sendo invocada. Os parâmetros irão definir quais os tipos de argumentos que uma função pode receber. Por exemplo, dada a definição da função:

```
def func(foo, bar=None, **kwargs):
    pass
```

foo, *bar* e *kwargs* são parâmetros de `func`. Dessa forma, ao invocar `func`, por exemplo:

```
func(42, bar=314, extra=algumvalor)
```

os valores `42`, `314`, e `algumvalor` são os argumentos.

Por que ao alterar a lista 'y' também altera a lista 'x'?

Se você escreveu um código como:

```
>>> x = []
>>> y = x
>>> y.append(10)
>>> y
[10]
>>> x
[10]
```

pode estar se perguntando por que acrescentar um elemento a `y` também mudou `x`.

Há dois fatores que produzem esse resultado:

1. As variáveis são simplesmente nomes que referem-se a objetos. Usar `y = x` não cria uma cópia da lista. Isso cria uma nova variável `y` que faz referência ao mesmo objeto ao qual `x` está se referindo. Isso significa que existe apenas um objeto (a lista) e que ambos `x` e `y` fazem referência a ele.
2. Listas são objetos mutáveis, o que significa que você pode alterar o seu conteúdo.

Após a chamada para **`append()`**, o conteúdo do objeto mutável mudou de `[]` para `[10]`. Uma vez que ambas as variáveis referem-se ao mesmo objeto, usar qualquer um dos nomes acessará o valor modificado `[10]`.

Se por acaso, atribuímos um objeto imutável a `x`:

```
>>> x = 5  # ints são imutáveis
>>> y = x
>>> x = x + 1  # 5 não pode ser mutado, estamos criando um novo
>>> x
6
>>> y
5
```

podemos ver que nesse caso `x` e `y` não são mais iguais. Isso ocorre porque os números inteiros são imutáveis, e quando fazemos `x = x + 1` não estamos mudando o int `5` e incrementando o seu valor. Em vez disso, estamos criando um novo objeto (o int `6`) e atribuindo-o a `x` (isto é, mudando para o objeto no qual `x` se refere). Após esta atribuição, temos dois objetos (os ints `6` e `5`) e duas variáveis que referem-se a elas (`x` agora se refere a `6`, mas `y` ainda refere-se a `5`).

Algumas operações (por exemplo, `y.append(10)` e `y.sort()`) alteram o objeto, enquanto operações superficialmente semelhantes (por exemplo, `y = y + [10]` e **`sorted(y)`**) criam um novo objeto. Em geral, em Python (e em todos os casos na biblioteca padrão) um método que causa mutação em um objeto retornará `None` para ajudar a evitar confundir os dois tipos de operações. Portanto, se você escrever por engano `y.sort()` pensando que lhe dará uma cópia ordenada de `y`, você terminará com `None`, o que provavelmente fará com que seu programa gere um erro facilmente diagnosticado.

No entanto, há uma classe de operações em que a mesma operação às vezes tem comportamentos diferentes com tipos diferentes: os operadores de atribuição aumentada. Por exemplo, `+=` transforma listas, mas não tuplas ou ints (`uma_lista += [1, 2, 3]` equivale a `uma_lista.extend([1, 2, 3])` a transforma `uma_lista`, sendo que `alguma_tupla += (1, 2, 3)` e `algum_int += 1` cria novos objetos).

Em outras palavras:

- If we have a mutable object (such as `list`, `dict`, `set`), we can use some specific operations to mutate it and all the variables that refer to it will see the change.
- If we have an immutable object (such as `str`, `int`, `tuple`), all the variables that refer to it will always see the same value, but operations that transform that value into a new value always return a new object.

Caso queira saber se duas variáveis fazem referência ao mesmo objeto ou não, pode-se usar o operador `is` ou a função embutida `id()`.

Como escrevo uma função com parâmetros de saída (chamada por referência)?

Remember that arguments are passed by assignment in Python. Since assignment just creates references to objects, there's no alias between an argument name in the caller and callee, and consequently no call-by-reference. You can achieve the desired effect in a number of ways.

1. Retornando um tupla com os resultados:

   ```
   >>> def func1(a, b):
   ...     a = 'valor-novo'       # a e b são nomes locais
   ...     b = b + 1              # atribuídos a novos objetos
   ...     return a, b            # retorna novos valores
   ...
   >>> x, y = 'valor-antigo', 99
   >>> func1(x, y)
   ('valor-novo', 100)
   ```

   Esta é quase sempre a solução mais clara.

2. Utilizando variáveis globais. Essa forma não é segura para thread e, portanto, não é recomendada.

3. Pela passagem de um objeto mutável (que possa ser alterado internamente):

```
>>> def func2(a):
...     a[0] = 'valor-novo'    # 'a' referencia uma lista mu
...     a[1] = a[1] + 1        # altera um objeto compartilh
...
>>> args = ['valor-antigo', 99]
>>> func2(args)
>>> args
['valor-novo', 100]
```

4. Pela passagem de um dicionário que sofra mutação:

```
>>> def func3(args):
...     args['a'] = 'valor-novo'     # args é um dicionário
...     args['b'] = args['b'] + 1   # alteração local, nele
...
>>> args = {'a': 'valor-antigo', 'b': 99}
>>> func3(args)
>>> args
{'a': 'valor-novo', 'b': 100}
```

5. Ou agrupando valores numa instância de classe:

```
>>> class Namespace:
...     def __init__(self, /, **args):
...         for key, value in args.items():
...             setattr(self, key, value)
...
>>> def func4(args):
...     args.a = 'valor-novo'        # args é um Namespace r
...     args.b = args.b + 1         # alteração local do obj
...
>>> args = Namespace(a='valor-antigo', b=99)
>>> func4(args)
>>> vars(args)
{'a': 'valor-novo', 'b': 100}
```

Quase nunca existe uma boa razão para complicar isso.

A sua melhor escolha será retornar uma tupla contendo os múltiplos resultados.

Como fazer uma função de ordem superior em Python?

Existem duas opções: pode-se usar escopos aninhados ou usar objetos chamáveis. Por exemplo, suponha que queira definir `linear(a,b)`, o qual retorna uma função `f(x)` que calcula o valor `a*x+b`. Usando escopos aninhados, temos:

```
def linear(a, b):
    def result(x):
        return a * x + b
    return result
```

Ou utilizando um objeto chamável:

```
class linear:

    def __init__(self, a, b):
        self.a, self.b = a, b

    def __call__(self, x):
        return self.a * x + self.b
```

Em ambos os casos:

```
taxas = linear(0.3, 2)
```

resulta em um objeto chamável, onde `taxes(10e6) == 0.3 * 10e6 + 2`.

A abordagem do objeto chamável tem a desvantagem de que é um pouco mais lenta e resulta num código ligeiramente mais longo. No entanto, note que uma coleção de chamáveis pode compartilhar sua assinatura via herança:

```
class exponential(linear):
    # __init__ herdado
    def __call__(self, x):
        return self.a * (x ** self.b)
```

Objetos podem encapsular o estado para vários métodos:

```
class counter:

    value = 0

    def set(self, x):
        self.value = x

    def up(self):
        self.value = self.value + 1

    def down(self):
        self.value = self.value - 1

count = counter()
inc, dec, reset = count.up, count.down, count.set
```

Aqui `inc()`, `dec()` e `reset()` funcionam como funções que compartilham a mesma variável contadora.

Como faço para copiar um objeto no Python?

Basicamente, tente utilizar a função `copy.copy()` ou a função `copy.deepcopy()` para casos gerais. Nem todos os objetos podem ser copiados, mas a maioria poderá.

Alguns objetos podem ser copiados com mais facilidade. Os dicionários têm um método `copy()`:

```
novodict = antigodict.copy()
```

As sequências podem ser copiadas através do uso de fatiamento:

```
nova_l = l[:]
```

Como posso encontrar os métodos ou atributos de um objeto?

Para uma instância `x` de uma classe definida pelo usuário, `dir(x)` retorna uma lista organizada alfabeticamente dos nomes contidos, os atributos da instância e os métodos e atributos definidos por sua classe.

Como que o meu código pode descobrir o nome de um objeto?

De um modo geral, não pode, porque os objetos realmente não têm nomes. Essencialmente, a atribuição sempre liga um nome a um valor; o mesmo é verdade para as instruções `def` e `class`, mas nesse caso o valor é um chamável. Considere o seguinte código:

```
>>> class A:
...     pass
...
>>> B = A
>>> a = B()
>>> b = a
>>> print(b)
<__main__.A object at 0x16D07CC>
>>> print(a)
<__main__.A object at 0x16D07CC>
```

Provavelmente, a classe tem um nome: mesmo que seja vinculada a dois nomes e invocada através do nome `B`, a instância criada ainda é relatada como uma instância da classe `A`. No entanto, é impossível dizer se o nome da instância é `a` ou `b`, uma vez que ambos os nomes estão vinculados ao mesmo valor.

De um modo geral, não deveria ser necessário que o seu código "conheça os nomes" de valores específicos. A menos que se escreva deliberadamente programas introspectivos, isso geralmente é uma indicação de que uma mudança de abordagem pode ser benéfica.

Em comp.lang.python, Fredrik Lundh deu uma excelente analogia em resposta a esta pergunta:

> Da mesma forma que você pega o nome daquele gato que encontrou na sua varanda: o próprio gato (objeto) não pode lhe dizer o seu nome, e ele realmente não se importa – então, a única maneira de descobrir como ele se chama é perguntar a todos os seus vizinhos (espaços de nomes) se é o gato deles (objeto)…

….e não fique surpreso se você descobrir que é conhecido por muitos nomes, ou até mesmo nenhum nome.

O que há com a precedência do operador vírgula?

A vírgula não é um operador em Python. Considere este código:

```
>>> "a" in "b", "a"
(False, 'a')
```

Uma vez que a vírgula não seja um operador, mas um separador entre as expressões acima, o código será avaliado como se tivéssemos entrado:

```
("a" in "b"), "a"
```

não:

```
"a" in ("b", "a")
```

The same is true of the various assignment operators (=, +=, and so on). They are not truly operators but syntactic delimiters in assignment statements.

Existe um equivalente ao operador ternário “?:” do C?

Sim, existe. A sintaxe é a seguinte:

```
[quando_verdadeiro] if [expressão] else [quando_falso]
```

```
x, y = 50, 25
small = x if x < y else y
```

Antes que essa sintaxe fosse introduzida no Python 2.5, uma expressão comum era usar operadores lógicos:

```
[expressão] and [quando_verdadeiro] or [quando_falso]
```

No entanto, essa forma não é segura, pois pode dar resultados inesperados quando *quando_verdadeiro* tiver um valor booleano falso. Portanto, é

sempre melhor usar a forma `... if ... else ....`

É possível escrever instruções de uma só linha ofuscadas em Python?

Sim. Normalmente, isso é feito aninhando `lambda` dentro de **`lambda`**. Veja os três exemplos a seguir, ligeiramente adaptados de Ulf Bartelt:

```
from functools import reduce

# Primos < 1000
print(list(filter(None,map(lambda y:y*reduce(lambda x,y:x*y!=0,
map(lambda x,y=y:y%x,range(2,int(pow(y,0.5)+1))),1),range(2,1000

# Primeiros 10 números de Fibonacci
print(list(map(lambda x,f=lambda x,f:(f(x-1,f)+f(x-2,f)) if x>1
f(x,f), range(10))))

# Conjunto de Mandelbrot
print((lambda Ru,Ro,Iu,Io,IM,Sx,Sy:reduce(lambda x,y:x+'\n'+y,ma
Iu=Iu,Io=Io,Ru=Ru,Ro=Ro,Sy=Sy,L=lambda yc,Iu=Iu,Io=Io,Ru=Ru,Ro=R
Sx=Sx,Sy=Sy:reduce(lambda x,y:x+y,map(lambda x,xc=Ru,yc=yc,Ru=Ru
i=i,Sx=Sx,F=lambda xc,yc,x,y,k,f=lambda xc,yc,x,y,k,f:(k<=0)or (
>=4.0) or 1+f(xc,yc,x*x-y*y+xc,2.0*x*y+yc,k-1,f):f(xc,yc,x,y,k,f
64+F(Ru+x*(Ro-Ru)/Sx,yc,0,0,i)),range(Sx))):L(Iu+y*(Io-Iu)/Sy),r
))))(-2.1, 0.7, -1.2, 1.2, 30, 80, 24))
#    \___ ___/  \___ ___/  |   |   |__ linhas na tela
#        V          V      |   |______ colunas na tela
#        |          |      |__________ máximo de "interações"
#        |          |_________________ faixa no eixo y
#        |____________________________ faixa no eixo x
```

Não tente isso em casa, crianças!

O que a barra(/) na lista de parâmetros de uma função significa?

Uma barra na lista de argumentos de uma função indica que os parâmetros anteriores a ela são somente-posicionais. Os parâmetros somente-

posicionais são aqueles que não têm nome utilizável externamente. Ao chamar uma função que aceita parâmetros somente-posicionais, os argumentos são mapeados para parâmetros com base apenas em sua posição. Por exemplo, `divmod()` é uma função que aceita parâmetros somente-posicionais. Sua documentação tem esta forma:

```
>>> help(divmod)
Help on built-in function divmod in module builtins:

divmod(x, y, /)
    Return the tuple (x//y, x%y).  Invariant: div*y + mod == x.
```

A barra no final da lista de parâmetros significa que ambos os parâmetros são somente-posicionais. Assim, chamar `divmod()` com argumentos nomeados levaria a um erro:

```
>>> divmod(x=3, y=4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: divmod() takes no keyword arguments
```

Números e strings

Como faço para especificar números inteiros hexadecimais e octais?

Para especificar um dígito no formato octal, preceda o valor octal com um zero e, em seguida, um “o” minúsculo ou maiúsculo. Por exemplo, para definir a variável “a” para o valor octal “10” (8 em decimal), digite:

```
>>> a = 0o10
>>> a
8
```

Hexadecimal é bem fácil. Basta preceder o número hexadecimal com um zero e, em seguida, um “x” minúsculo ou maiúsculo. Os dígitos hexadecimais podem ser especificados em letras maiúsculas e minúsculas. Por exemplo, no interpretador Python:

```
>>> a = 0xa5
>>> a
165
>>> b = 0XB2
>>> b
178
```

Por que -22 // 10 retorna -3?

Esta dúvida é primariamente direcionado pelo desejo de que `i % j` tenha o mesmo sinal que `j`. Se quiser isso, e também quiser:

```
i == (i // j) * j + (i % j)
```

então a divisão inteira deve retornar o piso. C também requer que essa identidade seja mantida, e então os compiladores que truncarem `i // j` precisam fazer com que `i % j` tenham o mesmo sinal que `i`.

Existem poucos casos de uso reais para `i % j` quando `j` é negativo. Quando `j` é positivo, existem muitos, e em virtualmente todos eles é mais útil para `i % j` ser `>= 0`. Se o relógio marca 10 agora, o que dizia há 200 horas? `-190 % 12 == 2` é útil, enquanto `-190 % 12 == -10` é um bug esperando para morder.

Como obtenho um atributo de um literal int em vez de SyntaxError?

Tentar obter um atributo de um literal `int` da maneira normal retorna um `SyntaxError` porque o ponto é visto como um ponto decimal:

```
>>> 1.__class__
  File "<stdin>", line 1
  1.__class__
   ^
SyntaxError: invalid decimal literal
```

A solução é separar o literal do ponto com um espaço ou parênteses.

```
>>> 1 .__class__
<class 'int'>
>>> (1).__class__
<class 'int'>
```

Como faço para converter uma string em um número?

For integers, use the built-in `int()` type constructor, for example, `int('144') == 144`. Similarly, `float()` converts to a floating-point number, for example, `float('144') == 144.0`.

Por padrão, eles interpretam o número como decimal, de modo que `int('0144') == 144` é verdadeiro e `int('0x144')` levanta `ValueError`. `int(string, base)` toma a base para converter como um segundo argumento opcional, então `int( '0x144', 16) == 324`. Se a base for especificada como 0, o número é interpretado usando as regras do Python: um “0o” à esquerda indica octal e “0x” indica um número hexadecimal.

Não use a função embutida `eval()` se tudo que você precisa é converter strings em números. **`eval()`** será significativamente mais lento e apresenta um risco de segurança: alguém pode passar a você uma expressão Python que pode ter efeitos colaterais indesejados. Por exemplo, alguém poderia passar `__import__('os').system("rm -rf $HOME")` que apagaria seu diretório pessoal.

`eval()` also has the effect of interpreting numbers as Python expressions, so that, for example, `eval('09')` gives a syntax error because Python does not allow leading ‘0’ in a decimal number (except ‘0’).

Como faço para converter um número em uma string?

For example, to convert the number `144` to the string `'144'`, use the built-in type constructor `str()`. If you want a hexadecimal or octal representation, use the built-in functions `hex()` or `oct()`. For fancy formatting, see the Literais de strings formatadas and Sintaxe das strings de formato sections. For example, `"{:04d}".format(144)` yields `'0144'` and `"{:.3f}".format(1.0/3.0)` yields `'0.333'`.

Como faço para modificar uma string internamente?

You can't, because strings are immutable. In most situations, you should simply construct a new string from the various parts you want to assemble it from. However, if you need an object with the ability to modify in-place Unicode data, try using an `io.StringIO` object or the `array` module:

```
>>> import io
>>> s = "Hello, world"
>>> sio = io.StringIO(s)
>>> sio.getvalue()
'Hello, world'
>>> sio.seek(7)
7
>>> sio.write("there!")
6
>>> sio.getvalue()
'Hello, there!'

>>> import array
>>> a = array.array('w', s)
>>> print(a)
array('w', 'Hello, world')
>>> a[0] = 'y'
>>> print(a)
array('w', 'yello, world')
>>> a.tounicode()
'yello, world'
```

Como faço para invocar funções/métodos através de strings?

Existem várias técnicas.

- A melhor forma é usar um dicionário que mapeie strings para funções. A principal vantagem desta técnica é que estas strings não precisam corresponder aos nomes das funções. Esta é também a principal técnica utilizada para emular uma construção de instrução estilo *case*:

  ```
  def a():
      pass
  ```

```
def b():
    pass

dispatch = {'go': a, 'stop': b}  # Note a falta de parêntese

dispatch[get_input()]()  # Note os parênteses ao final para
```

- Usar a função embutida `getattr()`:

  ```
  import foo
  getattr(foo, 'bar')()
  ```

  Observe que a função `getattr()` funciona com qualquer objeto, incluindo classes, instâncias de classe, módulos e assim por diante.

  A mesma é usado em vários lugares na biblioteca padrão, como este:

  ```
  class Foo:
      def faz_foo(self):
          ...

      def faz_bar(self):
          ...

  f = getattr(instancia_foo, 'faz_' + opname)
  f()
  ```

- Use `locals()` para determinar o nome da função:

  ```
  def minhaFunc():
      print("hello")

  fname = "minhaFunc"

  f = locals()[fname]
  f()
  ```

Existe um equivalente em Perl `chomp()` para remover linhas novas ao final de strings?

Pode-se utilizar `S.rstrip("\r\n")` para remover todas as ocorrência de qualquer terminador de linha que esteja no final da string `S` sem remover os espaços em branco. Se a string `S` representar mais de uma linha, contendo várias linhas vazias no final, os terminadores de linha de todas linhas em branco serão removidos:

```
>>> linhas = ("linha 1 \r\n"
...           "\r\n"
...           "\r\n")
>>> linhas.rstrip("\n\r")
'linha 1 '
```

Geralmente isso só é desejado ao ler um texto linha por linha, usando `S.rstrip()` dessa maneira funciona bem.

Existe uma função `scanf()` ou `sscanf()` ou algo equivalente?

Não como tal.

Para a análise de entrada simples, a abordagem mais fácil geralmente é dividir a linha em palavras delimitadas por espaços em branco usando o método `split()` de objetos strings e, em seguida, converter as strings decimais para valores numéricos usando a função `int()` ou a função `float()`. O método **`split()`** aceita um parâmetro “sep” opcional que é útil se a linha utilizar algo diferente de espaço em branco como separador.

Para análise de entradas de textos mais complicadas, as expressões regulares são mais poderosas do que a `sscanf` do C e mais adequadas para essa tarefa.

O que significa o erro `UnicodeDecodeError` ou `UnicodeEncodeError`?

Consulte Unicode.

Posso terminar uma string bruta com um número ímpar de contrabarras?

Uma string bruta terminando com um número ímpar de contrabarras vai escapar as aspas da string:

```
>>> r'C:\isso\não\vai\funcionar\'
  File "<stdin>", line 1
    r'C:\isso\não\vai\funcionar\'
    ^
SyntaxError: unterminated string literal (detected at line 1)
```

Há várias soluções alternativas para isso. Uma delas é usar strings regulares e duplicar as contrabarras:

```
>>> 'C:\\isso\\vai\\funcionar\\'
'C:\\isso\\vai\\funcionar\\'
```

Outra é concatenar uma string regular contendo uma contrabarra de escape à string bruta:

```
>>> r'C:\isso\vai\funcionar' '\\'
'C:\\isso\\vai\\funcionar\\'
```

Também é possível usar `os.path.join()` para acrescentar uma contrabarra no Windows:

```
>>> os.path.join(r'C:\isso\vai\funcionar', '')
'C:\\isso\\vai\\funcionar\\'
```

Note que, embora uma contrabarra vai “escapar” uma aspa para fins de determinar onde a string bruta termina, nenhum escape ocorre ao interpretar o valor da string bruta. Ou seja, a contrabarra permanece presente no valor da string bruta:

```
>>> r'contrabarra\'preservada'
"contrabarra\\'preservada"
```

Veja também a especificação na referência da linguagem.

Desempenho

Meu programa está muito lento. Como faço para melhorar o desempenho?

That's a tough one, in general. First, here is a list of things to remember before diving further:

- As características da desempenho podem variar conforme a implementação do Python. Esse FAQ se concentra no CPython.
- O comportamento pode variar em cada sistemas operacionais, especialmente quando estivermos tratando de E/S ou multi-threading.
- Sempre devemos encontrar os hot spots em nosso programa *antes de* tentar otimizar qualquer código (veja o módulo `profile`).
- Escrever scripts de benchmark permitirá iterar rapidamente buscando melhorias (veja o módulo `timeit`).
- É altamente recomendável ter boa cobertura de código (através de testes de unidade ou qualquer outra técnica) antes de potencialmente apresentar regressões escondidas em otimizações sofisticadas.

Dito isto, existem muitos truques para acelerar nossos códigos Python. Aqui estão alguns dos principais tópicos e que geralmente ajudam a atingir níveis de desempenho aceitáveis:

- Fazer seus algoritmos rápidos (ou mudando para mais rápidos) podem produzir benefícios maiores que tentar encaixar várias micro-otimizações no seu código.
- Usar as estruturas de dados corretas. Estude a documentação para Tipos embutidos e o módulo `collections`.
- Quando a biblioteca padrão fornecer um tipo primitivo para fazer algo, é provável (embora não garantido) que isso seja mais rápido do que qualquer alternativa que possa surgir. Isso geralmente é verdade para os tipos primitivos escritos em C, como os embutidos e alguns tipos de extensão. Por exemplo, certifique-se de usar o método embutido `list.sort()` ou a função relacionada `sorted()` para fazer a ordenação (e veja Técnicas de ordenação para exemplos de uso moderadamente avançado).
- As abstrações tendem a criar indireções e forçar o interpretador a trabalhar mais. Se os níveis de indireção superarem a quantidade de

trabalho útil feito, seu programa ficará mais lento. Você deve evitar a abstração excessiva, especialmente sob a forma de pequenas funções ou métodos (que também são muitas vezes prejudiciais à legibilidade).

Se você atingiu o limite do que Python puro pode permitir, existem ferramentas para levá-lo mais longe. Por exemplo, o Cython [https://cython.org] pode compilar uma versão ligeiramente modificada do código Python numa extensão C e pode ser usado em muitas plataformas diferentes. O Cython pode tirar proveito da compilação (e anotações tipo opcional) para tornar o seu código significativamente mais rápido do que quando interpretado. Se você está confiante em suas habilidades de programação C, também pode escrever um módulo de extensão de C.

Ver também

A página wiki dedicada a dicas de desempenho [https://wiki.python.org/moin/PythonSpeed/PerformanceTips].

Qual é a maneira mais eficiente de concatenar muitas strings?

`str` and `bytes` objects are immutable, therefore concatenating many strings together is inefficient as each concatenation creates a new object. In the general case, the total runtime cost is quadratic in the total string length. See Complexidade de tempo de operações em tipos embutidos for more information.

Para juntar vários objetos `str`, a linguagem recomendada colocá-los numa lista e invocar o método `str.join()`:

```
chunks = []
for s in my_strings:
    chunks.append(s)
result = ''.join(chunks)
```

(Another reasonably efficient idiom is to use `io.StringIO`.)

Para juntar vários objetos `bytes`, a forma recomendada é estender uma classe `bytearray` usando a concatenação local (com o operador +=):

```
result = bytearray()
for b in my_bytes_objects:
    result += b
```

Sequences (tuples/lists)

Como faço para converter tuplas em listas?

O construtor de tipo `tuple(seq)` converte qualquer sequência (na verdade, qualquer iterável) numa tupla com os mesmos itens na mesma ordem.

Por exemplo, `tuple([1, 2, 3])` produz `(1, 2, 3)` e `tuple('abc')` produz `('a', 'b', 'c')`. Se o argumento for uma tupla, a mesma não faz uma cópia, mas retorna o mesmo objeto, por isso é barato invocar a função `tuple()` quando você não tiver certeza que determinado objeto já é uma tupla.

O construtor de tipos `list(seq)` converte qualquer sequência ou iterável em uma lista com os mesmos itens na mesma ordem. Por exemplo, `list((1, 2, 3))` produz `[1, 2, 3]` e `list('abc')` produz `['a', 'b', 'c']`. Se o argumento for uma lista, o meso fará uma cópia como em `seq[:]`.

O que é um índice negativo?

Sequências em Python são indexadas com números positivos e números negativos. Para números positivos, 0 é o índice do primeiro elemento, 1 é o índice do segundo elemento e assim por diante. Para números negativos, -1 é índice do último elemento e -2 é o penúltimo (anterior ao último) índice e assim por diante. Pense em `seq[-n]` como o mesmo que `seq[len(seq)-n]`.

Usar índices negativos pode ser muito conveniente. Por exemplo, `S[:-1]` é a string inteira exceto pelo seu último caractere, o que é útil para remover o caractere de nova linha no final de uma string.

Como que eu itero uma sequência na ordem inversa?

Use a função embutida `reversed()`:

```
for x in reversed(sequence):
    ...  # faz alguma coisa com x ...
```

Isso não vai alterar sua sequência original, mas construir uma nova cópia com a ordem inversa para iteração.

Como que removo itens duplicados de uma lista?

Veja o Python Cookbook para uma longa discussão de várias formas de fazer isso:

https://code.activestate.com/recipes/52560/

Se você não se importar em reordenar a lista, ordene-a e depois examine a partir do final da lista, excluindo duplicatas conforme avança:

```
if mylist:
    mylist.sort()
    last = mylist[-1]
    for i in range(len(mylist)-2, -1, -1):
        if last == mylist[i]:
            del mylist[i]
        else:
            last = mylist[i]
```

If all elements of the list may be used as set keys (that is, they are all hashable) this is often faster:

```
mylist = list(set(mylist))
```

Isso converte a lista em um conjunto, deste modo removendo itens duplicados, e depois de volta em uma lista.

How do you remove multiple items from a list?

As with removing duplicates, explicitly iterating in reverse with a delete condition is one possibility. However, it is easier and faster to use slice replacement with an implicit or explicit forward iteration. Here are three variations:

```
mylist[:] = filter(keep_function, mylist)
mylist[:] = (x for x in mylist if keep_condition)
mylist[:] = [x for x in mylist if keep_condition]
```

A compreensão de lista pode ser a mais rápida.

Como fazer um vetor em Python?

Utilize uma lista:

```
["isto", 1, "é", "um", "vetor"]
```

Listas são equivalentes aos vetores de C ou Pascal em termos de complexidade de tempo; a diferença primária é que uma lista em Python pode conter objetos de tipos diferentes.

The `array` module also provides methods for creating arrays of fixed types with compact representations, but they are slower to index than lists. Also note that NumPy [https://numpy.org/] and other third-party packages define array-like structures with various characteristics as well.

Para obter listas ligadas no estilo Lisp, você pode emular *células cons* usando tuplas:

```
lista_lisp = ("como",  ("este",  ("exemplo", None) ) )
```

Se mutabilidade é desejada, você pode usar listas no lugar de tuplas. Aqui o análogo de um *car* Lisp é `lista_lisp[0]` e o análogo de *cdr* é `lista_lisp[1]`. Faça isso somente se você tem certeza que precisa disso, porque isso é usualmente muito mais lento que usar listas Python.

Como faço para criar uma lista multidimensional?

Você provavelmente tentou fazer um vetor multidimensional assim:

```
>>> A = [[None] * 2] * 3
```

Isso parece correto se você imprimir:

```
>>> A
[[None, None], [None, None], [None, None]]
```

Mas quando atribuíres um valor, o mesmo aparecerá em vários lugares:

```
>>> A[0][0] = 5
>>> A
[[5, None], [5, None], [5, None]]
```

A razão é que replicar uma lista com `*` não cria cópias, ela apenas cria referências aos objetos existentes. O `*3` cria uma lista contendo 3 referências para a mesma lista que contém 2 itens cada. Mudanças numa linha serão mostradas em todas as linhas, o que certamente não é o que você deseja.

A abordagem sugerida é criar uma lista com o comprimento desejado primeiro e, em seguida, preencher cada elemento com uma lista recém-criada:

```
A = [None] * 3
for i in range(3):
    A[i] = [None] * 2
```

Isso gera uma lista contendo 3 listas diferentes contendo 2 itens cada. Você também pode usar uma compreensão de lista:

```
w, h = 2, 3
A = [[None] * w for i in range(h)]
```

Ou você pode usar uma extensão que provê um tipo de dados matrix; NumPy [https://numpy.org/] é o mais conhecido.

Como eu aplico um método ou função para uma sequência de objetos?

To call a method or function and accumulate the return values in a list, a list comprehension is an elegant solution:

```
result = [obj.method() for obj in mylist]

result = [function(obj) for obj in mylist]
```

Para apenas chamar o método ou função sem salvar os valores retornados, um laço `for` será o suficiente:

```
for obj in mylist:
    obj.method()

for obj in mylist:
    function(obj)
```

Porque uma_tupla[i] += [‘item’] levanta uma exceção quando a adição funciona?

Isso se deve a uma combinação do fato de que os operadores de atribuição aumentada são operadores de *atribuição* e à diferença entre objetos mutáveis e imutáveis no Python.

Essa discussão se aplica em geral quando operadores de atribuição aumentada são aplicados a elementos de uma tupla que aponta para objetos mutáveis, mas usaremos uma `lista` e `+=` como exemplo.

Se você escrever:

```
>>> a_tuple = (1, 2)
>>> a_tuple[0] += 1
Traceback (most recent call last):
   ...
TypeError: 'tuple' object does not support item assignment
```

O motivo da exceção deve ser imediatamente claro: `1` é adicionado ao objeto que `a_tuple[0]` aponta para (`1`), produzindo o objeto de resultado, `2`, mas quando tentamos atribuir o resultado do cálculo, `2`, ao elemento `0` da

tupla, recebemos um erro porque não podemos alterar o que um elemento de uma tupla aponta.

Por baixo, o que a instrução de atribuição aumentada está fazendo é aproximadamente isso:

```
>>> result = a_tuple[0] + 1
>>> a_tuple[0] = result
Traceback (most recent call last):
  ...
TypeError: 'tuple' object does not support item assignment
```

Essa é a parte da atribuição da operação que produz o erro, já que a tupla é imutável.

Quando você escreve algo como:

```
>>> a_tuple = (['foo'], 'bar')
>>> a_tuple[0] += ['item']
Traceback (most recent call last):
  ...
TypeError: 'tuple' object does not support item assignment
```

A exceção é um pouco mais surpreendente, e ainda mais surpreendente é o fato de que, embora tenha havido um erro, o acréscimo à lista funcionou:

```
>>> a_tuple[0]
['foo', 'item']
```

Para entender por que isso acontece, você precisa saber que (a) se um objeto implementa um método mágico **`__iadd__()`**, ele é chamado quando a atribuição aumentada `+=` é executada, e seu valor de retorno é o que é usado na instrução de atribuição; e (b) para listas, **`__iadd__()`** é equivalente a chamar **`extend()`** na lista e retornar a lista. É por isso que dizemos que, para listas, `+=` é uma “abreviação” para **`list.extend()`**:

```
>>> a_list = []
>>> a_list += [1]
>>> a_list
[1]
```

Isso equivale a:

```
>>> result = a_list.__iadd__([1])
>>> a_list = result
```

O objeto apontado por a_list foi alterado e o ponteiro para o objeto alterado é atribuído novamente a `a_list`. O resultado final da atribuição é um no-op, pois é um ponteiro para o mesmo objeto para o qual `a_list` estava apontando anteriormente, mas a atribuição ainda acontece.

Portanto, em nosso exemplo da tupla, o que está acontecendo é equivalente a:

```
>>> result = a_tuple[0].__iadd__(['item'])
>>> a_tuple[0] = result
Traceback (most recent call last):
  ...
TypeError: 'tuple' object does not support item assignment
```

O **`__iadd__()`** é bem-sucedido e, portanto, a lista é estendida, mas mesmo que `result` aponte para o mesmo objeto para o qual `a_tuple[0]` já aponta, essa atribuição final ainda resulta em um erro, pois as tuplas são imutáveis.

Quero fazer uma ordenação confusa: você pode fazer uma transformação schwartziana em Python?

A técnica, atribuída a Randal Schwartz da comunidade Perl, ordena os elementos de uma lista por uma métrica que mapeia cada elemento para seu "valor de ordem". Em Python, use o argumento `key` para o método **`list.sort()`**:

```
Isorted = L[:]
Isorted.sort(key=lambda s: int(s[10:15]))
```

Como eu posso ordenar uma lista pelos valores de outra lista?

Merge them into an iterator of tuples, sort the resulting list, and then pick out the element you want.

```
>>> list1 = ["what", "I'm", "sorting", "by"]
>>> list2 = ["something", "else", "to", "sort"]
>>> pairs = zip(list1, list2)
>>> pairs = sorted(pairs)
>>> pairs
[("I'm", 'else'), ('by', 'sort'), ('sorting', 'to'), ('what', 's
>>> result = [x[1] for x in pairs]
>>> result
['else', 'sort', 'to', 'something']
```

Objetos

O que é uma classe?

Uma classe é o tipo de objeto específico criado pela execução da instrução class. Os objetos classe são usados como modelos para criar objetos instância, que incorporam os dados (atributos) e o código (métodos) específicos de um tipo de dado.

Uma classe pode ser baseada em uma ou mais outras classes, chamadas de classe(s) base(s), herdando seus atributos e métodos. Isso permite que um modelo de objeto seja sucessivamente refinado por herança. Você pode ter uma classe genérica `Mailbox` que fornece métodos básicos para uma caixa de correio e subclasses como `MboxMailbox`, `MaildirMailbox`, `OutlookMailbox` que manipula vários formatos específicos de caixa de correio.

O que é um método?

Um método é uma função em algum objeto `x` que você normalmente chama com `x.name(arguments...)`. Métodos são definidos como funções dentro da definição da classe:

```
class C:
    def meth(self, arg):
        return arg * 2 + self.attribute
```

O que é o self?

Self é apenas um nome convencional para o primeiro argumento de um método. Um método definido como `meth(self, a, b, c)` deve ser chamado com `x.meth(a, b, c)` para alguma instância `x` da classe em que a definição ocorre; o método chamado pensará que é chamado com `meth(x, a, b, c)`.

Veja também Por que o 'self' deve ser usado explicitamente em definições de método e chamadas?.

Como eu verifico se um objeto é uma instância de uma dada classe ou de uma subclasse dela?

Use the built-in function **`isinstance(obj, cls)`**. You can check if an object is an instance of any of a number of classes by providing a tuple instead of a single class, for example, `isinstance(obj, (class1, class2, ...))`, and can also check whether an object is one of Python's built-in types, for example, `isinstance(obj, str)` or `isinstance(obj, (int, float, complex))`.

Note that **`isinstance()`** also checks for virtual inheritance from an abstract base class. So, the test will return `True` for a registered class even if hasn't directly or indirectly inherited from it. To test for "true inheritance", scan the method resolution order (MRO) of the class:

```
from collections.abc import Mapping

class P:
     pass

class C(P):
    pass

Mapping.register(P)

>>> c = C()
>>> isinstance(c, C)        # direta
True
```

```
>>> isinstance(c, P)        # indireta
True
>>> isinstance(c, Mapping)  # virtual
True

# Cadeira de herança real
>>> type(c).__mro__
(<class 'C'>, <class 'P'>, <class 'object'>)

# Teste pela "herança verdadeira"
>>> Mapping in type(c).__mro__
False
```

Observe que a maioria dos programas não usa **`isinstance()`** em classes definidas pelo usuário com muita frequência. Se você estiver desenvolvendo as classes por conta própria, um estilo orientado a objetos mais adequado é definir métodos nas classes que encapsulam um comportamento específico, em vez de verificar a classe do objeto e fazer uma coisa diferente com base na classe que ele é. Por exemplo, se você tiver uma função que faz algo:

```
def search(obj):
    if isinstance(obj, Mailbox):
        ...  # código para pesquisar uma caixa de correio
    elif isinstance(obj, documento):
        ...  # código para pesquisar um documento
    elif ...
```

Uma abordagem melhor é definir um método `search()` em todas as classes e apenas chamá-lo:

```
class Mailbox:
    def search(self):
        ...  # código para pesquisar uma caixa de correio

class documento:
    def search(self):
        ...  # código para pesquisar um documento

obj.search()
```

O que é delegação?

Delegation is an object-oriented technique (also called a design pattern). Let's say you have an object `x` and want to change the behaviour of just one of its methods. You can create a new class that provides a new implementation of the method you're interested in changing and delegates all other methods to the corresponding method of `x`.

Com Python, você pode implementar delegação facilmente. Por exemplo, o trecho de código a seguir implementa uma classe que se comporta como um arquivo, mas converte todos os dados gravados em letras maiúsculas:

```
class UpperOut:

    def __init__(self, outfile):
        self._outfile = outfile

    def write(self, s):
        self._outfile.write(s.upper())

    def __getattr__(self, name):
        return getattr(self._outfile, name)
```

Aqui, a classe `UpperOut` redefine o método `write()` para converter o argumento string em maiúsculas antes de chamar o método `self._outfile.write()` subjacente. Todos os outros métodos são delegados ao objeto `self._outfile` subjacente. A delegação é realizada por meio do método **`__getattr__()`**; consulte a referência da linguagem para obter mais informações sobre o controle de acesso a atributo.

Observe que, em casos mais gerais, a delegação pode se tornar mais complicada. Quando o atributo precisa ser definido e recuperado, a classe deve definir um método **`__setattr__()`** também, e deve fazê-lo com cuidado. A implementação básica do **`__setattr__()`** é aproximadamente equivalente ao seguinte:

```
class X:
    ...
    def __setattr__(self, name, value):
```

```
        self.__dict__[name] = value
    ...
```

Muitas implementações do método `__setattr__()` chamam o **`object.__setattr__()`** para definir um atributo em si mesmo sem causar recursão infinita:

```
class X:
    def __setattr__(self, name, value):
        # Lógica personalizada aqui...
        object.__setattr__(self, name, value)
```

Como alternativa, é possível definir atributos inserindo entradas em `self.__dict__` diretamente.

Como eu chamo um método definido em uma classe base a partir de uma classe derivada que a estende?

Use a função embutida `super()`:

```
class Derived(Base):
    def meth(self):
        super().meth()  # chama Base.meth
```

No exemplo, `super()` determinará automaticamente a instância da qual foi chamado (o valor de `self`), procura a ordem de resolução de métodos (MRO) com `type(self).__mro__` e então retorna o próximo na linha após `Derived` no MRO: `Base`.

Como eu posso organizar meu código para facilitar a troca da classe base?

You could assign the base class to an alias and derive from the alias. Then all you have to change is the value assigned to the alias. Incidentally, this trick is also handy if you want to decide dynamically (such as depending on availability of resources) which base class to use. Example:

```
class Base:
    ...
```

```
BaseAlias = Base

class Derived(BaseAlias):
    ...
```

Como faço para criar dados de classe estáticos e métodos de classe estáticos?

Tanto dados estáticos quanto métodos estáticos (no sentido de C++ ou Java) são possíveis com Python.

Para dados estáticos, basta definir um atributo de classe. Para atribuir um novo valor ao atributo, você precisa usar explicitamente o nome da classe na atribuição:

```
class C:
    count = 0 # número de vezes que C.__init__ foi chamado

    def __init__(self):
        C.count = C.count + 1

    def getcount(self):
        return C.count # ou return self.count
```

`c.count` também se refere a `C.count` para qualquer `c` de modo que `isinstance(c, C)` seja válido, a menos que seja substituído pelo próprio `c` ou por alguma classe no caminho de busca da classe base de `c.__class__` até `C`.

Cuidado: em um método de C, uma atribuição como `self.count = 42` cria uma instância nova e não-relacionada chamada "count" no próprio dicionário de `self`. A revinculação de um nome de dado de classe estático deve sempre especificar a classe, seja dentro de um método ou não:

```
C.count = 314
```

Métodos estáticos são possíveis:

```
class C:
    @staticmethod
    def static(arg1, arg2, arg3):
        # Sem parâmetro 'self'!
        ...
```

No entanto, uma maneira muito mais direta de obter o efeito de um método estático é por meio de uma simples função em nível de módulo:

```
def getcount():
    return C.count
```

Se o seu código está estruturado de modo a definir uma classe (ou uma hierarquia de classes estreitamente relacionada) por módulo, isso fornecerá o encapsulamento desejado.

Como eu posso sobrecarregar construtores (ou métodos) em Python?

Essa resposta na verdade se aplica para todos os métodos, mas a pergunta normalmente aparece primeiro no contexto de construtores.

In C++ you'd write:

```
class C {
    C() { cout << "Sem argumentos\n"; }
    C(int i) { cout << "Argumento é igual a " << i << "\n"; }
}
```

Em Python, você tem que escrever um único construtor que pega todos os casos usando argumentos padrão. Por exemplo:

```
class C:
    def __init__(self, i=None):
        if i is None:
            print("Sem argumentos")
        else:
            print("Argumento é igual a", i)
```

Isso não é inteiramente equivalente, mas já está bem próximo.

You could also try a variable-length argument list, for example:

```
def __init__(self, *args):
    ...
```

A mesma abordagem funciona para todas as definições de métodos.

Eu tentei usar __spam e recebi um erro sobre _SomeClassName__spam.

Os nomes de variáveis com dois sublinhados à esquerda são “manipulados” para fornecer uma maneira simples, mas eficaz, de definir variáveis privadas de classe. Qualquer identificador no formato `__spam` (pelo menos dois sublinhados à esquerda, no máximo um sublinhado à direita) é textualmente substituído por `_classname__spam`, em que `classname` é o nome da classe atual sem nenhum sublinhado à esquerda.

O identificador pode ser usado normalmente dentro da classe, mas para acessá-lo fora da classe, deve ser usado o nome manipulado:

```
class A:
    def __one(self):
        return 1
    def two(self):
        return 2 * self.__one()

class B(A):
    def three(self):
        return 3 * self._A__one()

four = 4 * A()._A__one()
```

Em particular, isso não garante a privacidade, pois um usuário externo ainda pode acessar deliberadamente o atributo privado; muitas pessoas que usam Python nunca se preocuparam em usar nomes de variáveis privadas.

Ver também

As especificações de desfiguração de nome privado para detalhes e casos especiais.

Minha classe define __del__, mas o mesmo não é chamado quando eu excluo o objeto.

Há várias razões possíveis para isto.

A instrução `del` não necessariamente chama o método `__del__()` - ele simplesmente diminui o contagem de referências do objeto e, se ele chegar a zero, o `__del__()` é chamado.

If your data structures contain circular links (for example, a tree where each child has a parent reference and each parent has a list of children) the reference counts will never go back to zero. Once in a while Python runs an algorithm to detect such cycles, but the garbage collector might run some time after the last reference to your data structure vanishes, so your `__del__()` method may be called at an inconvenient and random time. This is inconvenient if you're trying to reproduce a problem. Worse, the order in which object's `__del__()` methods are executed is arbitrary. You can run `gc.collect()` to force a collection, but there *are* pathological cases where objects will never be collected.

Apesar do coletor de ciclos, ainda é uma boa ideia definir um método `close()` explícito nos objetos a ser chamado sempre que você terminar de usá-los. O método `close()` pode então remover o atributo que se refere a subobjetos. Não chame `__del__()` diretamente - `__del__()` deve chamar `close()` e `close()` deve garantir que ele possa ser chamado mais de uma vez para o mesmo objeto.

Outra forma de evitar referências cíclicas é usar o módulo `weakref`, que permite apontar para objetos sem incrementar o contagem de referências. As estruturas de dados em árvore, por exemplo, devem usar o referência fraca para referenciar seus pais e irmãos (se precisarem deles!).

Por fim, se seu método **`__del__()`** levanta uma exceção, uma mensagem de alerta será enviada para `sys.stderr`.

Como eu consigo pegar uma lista de todas as instâncias de uma dada classe?

Python não mantém o controle de todas as instâncias de uma classe (ou de um tipo embutido). Você pode programar o construtor da classe para manter o controle de todas as instâncias, mantendo uma lista de referências fracas para cada instância.

Por que o resultado de `id()` aparenta não ser único?

A função embutida `id()` retorna um inteiro que é garantido ser único durante a vida útil do objeto. Como em CPython esse número é o endereço de memória do objeto, acontece com frequência que, depois que um objeto é excluído da memória, o próximo objeto recém-criado é alocado na mesma posição na memória. Isso é ilustrado por este exemplo:

```
>>> id(1000)
13901272
>>> id(2000)
13901272
```

Os dois ids pertencem a diferentes objetos inteiros que são criados antes e excluídos imediatamente após a execução da chamada de `id()`. Para ter certeza de que os objetos cujo id você deseja examinar ainda estão vivos, crie outra referencia para o objeto:

```
>>> a = 1000; b = 2000
>>> id(a)
13901272
>>> id(b)
13891296
```

Quando eu posso depender dos testes de identidade com o operador *is*?

O operador `is` testa a identidade do objeto. O teste `a is b` equivale a `id(a) == id(b)`.

A propriedade mais importante de um teste de identidade é que um objeto é sempre idêntico a si mesmo, `a is a` sempre retorna `True`. Testes de identidade são geralmente mais rápidos do que os testes de igualdade. E, ao contrário dos testes de igualdade, teste de identidade garante que retorno seja um booleano `True` ou `False`.

Entretanto, o teste de identidade *somente* pode ser substituído por testes de igualdade quando a identidade do objeto é garantida. Em geral, há três circunstâncias em que a identidade é garantida:

1. Atribuições criam novos nomes, mas não alteram a identidade do objeto. Após a atribuição `new = old`, é garantido que `new is old`.
2. Colocar um objeto em um contêiner que armazena referências de objetos não altera a identidade do objeto. Após a lista atribuição `s[0] = x`, é garantido que `s[0] is x`.
3. Se um objeto for um Singleton, isso significa que só pode existir uma instância desse objeto. Depois de atribuição `a = None` e `b = None`, é garantido que `a is b` porque `None` é um Singleton.

Na maioria das outras circunstâncias, o teste de identidade é desaconselhável e os testes de igualdade são preferíveis. Em particular, teste de identidade não deve ser usado para verificar constantes, como **`int`** e **`str`**, que não têm garantia de serem Singletons:

```
>>> a = 10_000_000
>>> b = 5_000_000
>>> c = b + 5_000_000
>>> a is c
False

>>> a = 'Python'
>>> b = 'Py'
>>> c = b + 'thon'
>>> a is c
False
```

Do mesmo jeito, novas instâncias de contêineres mutáveis nunca são idênticas:

```
>>> a = []
>>> b = []
>>> a is b
False
```

No código da biblioteca padrão, você encontrará vários padrões comuns para usar corretamente o teste de identidade:

1. Conforme recomendado por **PEP 8** [https://peps.python.org/pep-0008/], um teste de identidade é a maneira preferida de verificar `None`. Isso é lido como se fosse inglês simples no código e evita confusão com outros objetos que podem ter valor booleano avaliado para falso.

2. A detecção de argumento opcional pode ser complicada quando `None` é uma valor de entrada válido. Nessas situações, você pode criar um objeto Singleton sinalizador com garantia de ser distinto de outros objetos. Por exemplo, veja como implementar um método que se comporta como **`dict.pop()`**:

   ```
   _sentinel = sentinel('_sentinel')

   def pop(self, key, default=_sentinel):
       if key in self:
           value = self[key]
           del self[key]
           return value
       if default is _sentinel:
           raise KeyError(key)
       return default
   ```

3. Implementações de contêiner às vezes precisam combinar testes de igualdade com testes de identidade. Isso evita que o código seja confundido por objetos como `float('NaN')` que não são iguais a si mesmos.

Por exemplo, aqui está a implementação de **`collections.abc.Sequence.__contains__()`**:

```
def __contains__(self, value):
    for v in self:
        if v is value or v == value:
            return True
    return False
```

Como uma subclasse pode controlar quais dados são armazenados em uma instância imutável?

Quando estender um tipo imutável, sobrescreva o método `__new__()` em vez do método `__init__()`. O último só é executado *depois* que uma instância é criada, o que é tarde demais para alterar os dados em uma instância imutável.

Todas essas classes imutáveis têm um assinatura diferente da sua classe base:

```
import datetime as dt

class FirstOfMonthDate(dt.date):
    "Always choose the first day of the month"
    def __new__(cls, year, month, day):
        return super().__new__(cls, year, month, 1)

class NamedInt(int):
    "Allow text names for some numbers"
    xlat = {'zero': 0, 'one': 1, 'ten': 10}
    def __new__(cls, value):
        value = cls.xlat.get(value, value)
        return super().__new__(cls, value)

class TitleStr(str):
    "Convert str to name suitable for a URL path"
    def __new__(cls, s):
        s = s.lower().replace(' ', '-')
        s = ''.join([c for c in s if c.isalnum() or c == '-'])
        return super().__new__(cls, s)
```

As classes podem ser usadas da seguinte forma:

```
>>> FirstOfMonthDate(2012, 2, 14)
FirstOfMonthDate(2012, 2, 1)
>>> NamedInt('ten')
10
>>> NamedInt(20)
20
>>> TitleStr('Blog: Porque Python domina')
'blog-porque-python-domina'
```

Como faço para armazenar em cache as chamadas de um método?

The two principal tools for caching methods are `@functools.cached_property` and `@functools.lru_cache`. The former stores results at the instance level and the latter at the class level.

The `cached_property` approach only works with methods that do not take any arguments. It does not create a reference to the instance. The cached method result will be kept only as long as the instance is alive.

A vantagem é que, quando um instância não for mais usada, o resultado do método armazenado em cache será liberado imediatamente. A desvantagem é que, se as instâncias se acumularem, os resultados do método também serão acumulados. Eles podem crescer sem limites.

The `lru_cache` approach works with methods that have hashable arguments. It creates a reference to the instance unless special efforts are made to pass in weak references.

A vantagem do algoritmo menos recentemente usado é que o cache é limitado pelo *maxsize* especificado. A desvantagem é que as instâncias são mantidas vivas até que saiam do cache ou até que o cache seja limpo.

Esse exemplo mostra as várias técnicas:

```
class Weather:
    "Procura informações de tempo em sites governamentais"

    def __init__(self, station_id):
```

```
        self._station_id = station_id
        # O _station_id é privado e imutável

    def current_temperature(self):
        "Última observação horária"
        # Não armazena isso em cache porque os dados anteriores
        # podem estar desatualizados.

    @cached_property
    def location(self):
        "Retornas as coordenadas longitude/latitude coordinates
        # Resultado depende apenas de station_id

    @lru_cache(maxsize=20)
    def historic_rainfall(self, date, units='mm'):
        "Precipitação em uma determinada data"
        # Depende de station_id, date, e units.
```

The above example assumes that the *station_id* never changes. If the relevant instance attributes are mutable, the `cached_property` approach can't be made to work because it cannot detect changes to the attributes.

To make the `lru_cache` approach work when the *station_id* is mutable, the class needs to define the **`__eq__()`** and **`__hash__()`** methods so that the cache can detect relevant attribute updates:

```
class Weather:
    "Exemplo com um identificador de estação mutável"

    def __init__(self, station_id):
        self.station_id = station_id

    def change_station(self, station_id):
        self.station_id = station_id

    def __eq__(self, other):
        return self.station_id == other.station_id

    def __hash__(self):
        return hash(self.station_id)
```

```
    @lru_cache(maxsize=20)
    def historic_rainfall(self, date, units='cm'):
        "Precipitação em uma determinada data"
        # Depende de station_id, date, e units.
```

Módulos

Como faço para criar um arquivo .pyc?

Quando um módulo é importado pela primeira vez (ou quando o arquivo de origem foi alterado desde que o arquivo compilado atual foi criado), um arquivo `.pyc` contendo o código compilado deve ser criado em um subdiretório `__pycache__` do diretório que contém o arquivo `.py`. O arquivo `.pyc` terá um nome de arquivo que começa com o mesmo nome do arquivo `.py` e termina com `.pyc`, com um componente intermediário que depende do binário `python` específico que o criou. (Consulte **PEP 3147** [https://peps.python.org/pep-3147/] para obter detalhes.)

Um dos motivos pelos quais um arquivo `.pyc` pode não ser criado é um problema de permissões no diretório que contém o arquivo de origem, o que significa que o subdiretório `__pycache__` não pode ser criado. Isso pode acontecer, por exemplo, se você desenvolver como um usuário, mas executar como outro, como se estivesse testando em um servidor web.

Unless the **`PYTHONDONTWRITEBYTECODE`** environment variable is set, creation of a .pyc file is automatic if you're importing a module and Python has the ability (permissions, free space, and so on) to create a `__pycache__` subdirectory and write the compiled module to that subdirectory.

Running Python on a top-level script is not considered an import and no `.pyc` will be created. For example, if you have a top-level module `foo.py` that imports another module `xyz.py`, when you run `foo` (by typing `python foo.py` as a shell command), a `.pyc` will be created for `xyz` because `xyz` is imported, but no `.pyc` file will be created for `foo` since `foo.py` isn't being imported.

Se você precisar criar um arquivo `.pyc` para `foo`, ou seja, criar um arquivo `.pyc` para um módulo que não é importado, você pode usar os módulos **`py_compile`** e **`compileall`**.

O módulo **`py_compile`** pode compilar manualmente qualquer módulo. Uma maneira é usar interativamente a função `compile()` nesse módulo:

```
>>> import py_compile
>>> py_compile.compile('foo.py')
```

This will write the `.pyc` to a `__pycache__` subdirectory in the same location as `foo.py` (or you can override that with the optional parameter *cfile*).

Você também pode compilar automaticamente todos os arquivos em um diretório ou diretórios usando o módulo **`compileall`**. Você pode fazer isso no console do SO executando `compileall.py` e fornecendo o caminho de um diretório que contenha os arquivos Python a serem compilados:

```
python -m compileall .
```

Como encontro o nome do módulo atual?

Um módulo pode descobrir seu próprio nome consultando a variável global predefinida `__name__` . Se ela tiver o valor `'__main__'` , o programa estará sendo executado como um script. Muitos módulos que são normalmente usados ao serem importados também fornecem uma interface de linha de comando ou um autoteste, e só executam esse código depois de verificar `__name__`:

```
def main():
    print('Executando teste...')
    ...

if __name__ == '__main__':
    main()
```

Como posso ter módulos que se importam mutuamente?

Suponha que tenhas os seguintes módulos:

`foo.py`:

```
from bar importar bar_var
foo_var = 1
```

`bar.py`:

```
from foo import foo_var
bar_var = 2
```

O problema é que o interpretador vai realizar os seguintes passos:

- Programa principal importa `foo`
- São criados globais vazios para `foo`
- `foo` é compilado e começa a ser executado
- `foo` importa `bar`
- São criados globais vazios para `bar`
- `bar` é compilado e começa a ser executado
- `bar` importa `foo` (o que não é executado de fato, pois já existe um módulo chamado `foo`)
- O mecanismo de importação tenta ler `foo_var` do `foo` em globais, para definir `bar.foo_var = foo.foo_var`

A última etapa falha, pois Python ainda não terminou de interpretar `foo` e o dicionário de símbolos global para `foo` ainda está vazio.

O mesmo acontece quando você usa `import foo` e, em seguida, tenta acessar `foo.foo_var` no código global.

Há (pelo menos) três possíveis soluções alternativas para esse problema.

Guido van Rossum recomenda evitar todos os usos de `from <module> import ...` e colocar todo o código dentro de funções. As inicializações de variáveis globais e variáveis de classe devem usar apenas constantes ou funções embutidas. Isso significa que tudo de um módulo importado é referenciado como `<module>.<name>`.

Jim Roskind sugere a execução das etapas na seguinte ordem em cada módulo:

- exportações (globais, função e classes que não precisam de classes base importadas)
- instruções `import`
- código ativo (incluindo globais que são inicializadas de valores importados)

Van Rossum não gosta muito dessa abordagem porque a importação aparece em um lugar estranho, mas ela funciona.

Matthias Urlichs recomenda reestruturar seu código para que importação recursiva não seja necessária em primeiro lugar.

Essas soluções não são mutuamente exclusivas.

__import__('x.y.z') retorna <módulo 'x'>; como faço para obter z?

Em vez disso, considere usar a conveniente função `import_module()` de `importlib`:

```
z = importlib.import_module('x.y.z')
```

Quando eu edito um módulo importado e o reimporto, as mudanças não aparecem. Por que isso acontece?

Por motivos de eficiência e consistência, o Python só lê o arquivo do módulo na primeira vez em que um módulo é importado. Caso contrário, em um programa que consiste em muitos módulos em que cada um importa o mesmo módulo básico, o módulo básico seria analisado e reanalisado várias vezes. Para forçar a releitura de um módulo alterado, faça o seguinte:

```
import importlib
import modname
importlib.reload(modname)
```

Warning: this technique is not 100% fool-proof. In particular, modules containing statements like:

```
from modname import some_objects
```

continuará com a versão antiga dos objetos importados. Se o módulo contiver definições de classe, as instâncias de classe existentes *não* serão atualizadas para usar a nova definição da classe. Isso pode resultar no seguinte comportamento paradoxal:

```
>>> import importlib
>>> import cls
>>> c = cls.C()                # Cria uma instância de C
>>> importlib.reload(cls)
<module 'cls' from 'cls.py'>
>>> isinstance(c, cls.C)       # isinstance é falso?!?
False
```

A natureza do problema fica clara se você exibir a “identidade” dos objetos da classe:

```
>>> hex(id(c.__class__))
'0x7352a0'
>>> hex(id(cls.C))
'0x4198d0'
```

FAQ sobre design e histórico

Sumário

- FAQ sobre design e histórico
  - Por que o Python usa indentação para agrupamento de instruções?
  - Por que eu estou recebendo resultados estranhos com simples operações aritméticas?
  - Por que o cálculo de pontos flutuantes são tão imprecisos?
  - Por que strings do Python são imutáveis?
  - Por que o 'self' deve ser usado explicitamente em definições de método e chamadas?
  - Por que não posso usar uma atribuição em uma expressão?
  - Por que o Python usa métodos para algumas funcionalidades (ex: lista.index()) mas funções para outras (ex: len(lista))?
  - Por que o join() é um método de string em vez de ser um método de lista ou tupla?
  - Quão rápidas são as exceções?
  - Por que não existe uma instrução de switch ou case no Python?
  - Você não pode emular threads no interpretador em vez de confiar em uma implementação de thread específica do sistema operacional?
  - Por que expressões lambda não podem conter instruções?
  - O Python pode ser compilado para linguagem de máquina, C ou alguma outra linguagem?
  - Como o Python gerencia memória?
  - Por que o CPython não usa uma forma mais tradicional de esquema de coleta de lixo?
  - Por que toda memória não é liberada quando o CPython fecha?
  - Por que existem tipos de dados separados para tuplas e listas?
  - Como as listas são implementadas no CPython?
  - Como são os dicionários implementados no CPython?
  - Por que chaves de dicionário devem ser imutáveis?

- Por que lista.sort() não retorna a lista ordenada?
- Como você especifica e aplica um spec de interface no Python?
- Por que não há goto?
- Por que strings brutas (r-strings) não podem terminar com uma contrabarra?
- Por que o Python não tem uma instrução “with” para atribuição de atributos?
- Por que os geradores não suportam a instrução with?
- Por que dois pontos são necessários para as instruções de if/while/def/class?
- Por que o Python permite vírgulas ao final de listas e tuplas?

Por que o Python usa indentação para agrupamento de instruções?

Guido van Rossum acredita que usar indentação para agrupamento é extremamente elegante e contribui muito para a clareza de programa Python mediano. Muitas pessoas aprendem a amar esta ferramenta depois de um tempo.

Uma vez que não há colchetes de início / fim, não pode haver um desacordo entre o agrupamento percebido pelo analisador sintático e pelo leitor humano. Ocasionalmente, programadores C irão encontrar um fragmento de código como este:

```
if (x <= y)
        x++;
        y--;
z++;
```

Somente a instrução `x++` é executada se a condição for verdadeira, mas a indentação leva muitos a acreditarem no contrário. Com frequência, até programadores C experientes a observam fixamente por um longo tempo, perguntando-se por que `y` está sendo decrementada até mesmo para `x > y`.

Como não há chaves de início / fim, o Python é muito menos propenso a conflitos no estilo de codificação. Em C, existem muitas maneiras diferentes de colocar as chaves. Depois de se tornar habitual a leitura e escrita de código usando um estilo específico, é normal sentir-se um pouco receoso ao ler (ou precisar escrever) em um estilo diferente.

Muitos estilos de codificação colocam chaves de início / fim em uma linha sozinhos. Isto torna os programas consideravelmente mais longos e desperdiça espaço valioso na tela, dificultando a obtenção de uma boa visão geral de um programa. Idealmente, uma função deve caber em uma tela (digamos, 20 a 30 linhas). 20 linhas de Python podem fazer muito mais trabalho do que 20 linhas de C. Isso não se deve apenas à falta de colchetes de início/fim – a falta de declarações e os tipos de dados de alto nível também são responsáveis – mas a sintaxe baseada em indentação certamente ajuda.

Por que eu estou recebendo resultados estranhos com simples operações aritméticas?

Veja a próxima questão.

Por que o cálculo de pontos flutuantes são tão imprecisos?

Usuários são frequentemente surpreendidos por resultados como este:

```
>>> 1.2 - 1.0
0.19999999999999996
```

e pensam que isto é um bug do Python. Não é não. Isto tem pouco a ver com o Python, e muito mais a ver com como a estrutura da plataforma lida com números em ponto flutuante.

O tipo `float` no CPython usa um `double` do C para armazenamento. O valor de um objeto **`float`** é armazenado em ponto flutuante binário com uma precisão fixa (normalmente 53 bits) e Python usa operações C, que por

sua vez dependem da implementação de hardware no processador, para realizar operações de ponto flutuante. Isso significa que, no que diz respeito às operações de ponto flutuante, Python se comporta como muitas linguagens populares, incluindo C e Java.

Muitos números podem ser escritos facilmente em notação decimal, mas não podem ser expressados exatamente em ponto flutuante binário. Por exemplo, após:

```
>>> x = 1.2
```

o valor armazenado para `x` é uma (ótima) aproximação para o valor decimal `1.2`, mas não é exatamente igual. Em uma máquina típica, o valor real armazenado é:

```
1.0011001100110011001100110011001100110011001100110011 (binário)
```

que é exatamente:

```
1.1999999999999999555910790149937383830547332763671875 (decimal)
```

A precisão típica de 53 bits fornece floats do Python com precisão de 15 a 16 dígitos decimais.

Para uma explicação mais completa, consulte o capítulo de aritmética de ponto flutuante no tutorial Python.

Por que strings do Python são imutáveis?

Existem várias vantagens.

Uma delas é o desempenho: saber que uma string é imutável significa que podemos alocar espaço para ela no momento da criação, e os requisitos de armazenamento são fixos e imutáveis. Esta é também uma das razões para a distinção entre tuplas e listas.

Outra vantagem é que strings em Python são consideradas tão “elementares” quanto números. Nenhuma atividade alterará o valor 8 para qualquer outra coisa e, em Python, nenhuma atividade alterará a string “oito” para qualquer outra coisa.

Por que o ‘self’ deve ser usado explicitamente em definições de método e chamadas?

A ideia foi emprestada do Modula-3. Acontece dela ser muito útil, por vários motivos.

Primeiro, é mais óbvio que você está usando um método ou atributo de instância em vez de uma variável local. Ler `self.x` ou `self.meth()` deixa absolutamente claro que uma variável de instância ou método é usado mesmo se você não souber a definição da classe de cor. Em C++, você pode perceber pela falta de uma declaração de variável local (presumindo que globais são raras ou facilmente reconhecíveis) – mas no Python não há declarações de variáveis locais, então você teria que procurar a definição de classe para tenha certeza. Alguns padrões de codificação C++ e Java exigem que atributos de instância tenham um prefixo `m_`, portanto, essa explicitação ainda é útil nessas linguagens também.

Segundo, significa que nenhuma sintaxe especial é necessária se você quiser referenciar ou chamar explicitamente o método de uma classe específica. Em C++, se você quiser usar um método de uma classe base que é substituído em uma classe derivada, você deve usar o operador `::` – no Python, você pode escrever `baseclass.methodname(self, <lista de argumentos>)`. Isto é particularmente útil para métodos `__init__()` e, em geral, em casos onde um método de classe derivada deseja estender o método da classe base de mesmo nome e, portanto, precisa chamar o método da classe base de alguma forma.

Finalmente, por exemplo, variáveis, ele resolve um problema sintático com atribuição: uma vez que variáveis locais em Python são (por definição!) aquelas variáveis às quais um valor é atribuído em um corpo de função (e que não são explicitamente declaradas globais), é necessário deve haver alguma forma de dizer ao interpretador que uma atribuição deveria ser

atribuída a uma variável de instância em vez de a uma variável local, e deve ser preferencialmente sintática (por razões de eficiência). C++ faz isso através de declarações, mas Python não possui declarações e seria uma pena ter que introduzi-las apenas para esse fim. Usar o `self.var` explícito resolve isso muito bem. Da mesma forma, para usar variáveis de instância, ter que escrever `self.var` significa que referências a nomes não qualificados dentro de um método não precisam pesquisar nos diretórios da instância. Em outras palavras, variáveis locais e variáveis de instância residem em dois espaço de nomes diferentes, e você precisa informar ao Python qual espaço de nomes usar.

Por que não posso usar uma atribuição em uma expressão?

A partir do Python 3.8, você pode!

Expressões de atribuição usando o operador morsa `:=` atribuem uma variável em uma expressão:

```
while chunk := fp.read(200):
   print(chunk)
```

Veja **PEP 572** [https://peps.python.org/pep-0572/] para mais informações.

Por que o Python usa métodos para algumas funcionalidades (ex: lista.index()) mas funções para outras (ex: len(lista))?

Como Guido disse:

> (a) For some operations, prefix notation just reads better than postfix – prefix (and infix!) operations have a long tradition in mathematics which likes notations where the visuals help the mathematician thinking about a problem. Compare the easy with which we rewrite a formula like x*(a+b) into x*a + x*b to the clumsiness of doing the same thing using a raw OO notation.

> (b) When I read code that says len(x) I *know* that it is asking for the length of something. This tells me two things: the result is an integer, and the argument is some kind of container. To the contrary, when I read x.len(), I have to already know that x is some kind of container implementing an interface or inheriting from a class that has a standard len(). Witness the confusion we occasionally have when a class that is not implementing a mapping has a get() or keys() method, or something that isn't a file has a write() method.
>
> —https://mail.python.org/pipermail/python-3000/2006-November/004643.html

Por que o join() é um método de string em vez de ser um método de lista ou tupla?

Strings se tornaram muito parecidas com outros tipos padrão a partir do Python 1.6, quando métodos que dão a mesma funcionalidade que sempre esteve disponível utilizando as funções do módulo de string foram adicionados. A maior parte desses novos métodos foram amplamente aceitos, mas o que parece deixar alguns programadores desconfortáveis é:

```
", ".join(['1', '2', '4', '8', '16'])
```

que dá o resultado:

```
"1, 2, 4, 8, 16"
```

Existem dois argumentos comuns contra esse uso.

O primeiro segue as linhas de: "Parece muito feio usar um método de uma string literal (constante de string)", para o qual a resposta é que pode, mas uma string literal é apenas um valor fixo. Se os métodos devem ser permitidos em nomes vinculados a strings, não há razão lógica para torná-los indisponíveis em literais.

A segunda objeção é normalmente formulada como: "Na verdade, estou dizendo a uma sequência para unir seus membros com uma constante de

string”. Infelizmente, você não está. Por alguma razão parece haver muito menos dificuldade em ter `split()` como um método string, já que nesse caso é fácil ver que

```
"1, 2, 4, 8, 16".split(", ")
```

é uma instrução para uma string literal para retornar as substrings delimitadas pelo separador fornecido (ou, por padrão, execuções arbitrárias de espaço em branco).

`join()` é um método de string porque ao usá-lo você está dizendo à string separadora para iterar sobre uma sequência de strings e se inserir entre elementos adjacentes. Este método pode ser usado com qualquer argumento que obedeça às regras para objetos sequência, incluindo quaisquer novas classes que você mesmo possa definir. Existem métodos semelhantes para bytes e objetos bytearray.

Quão rápidas são as exceções?

Um bloco `try`/`except` é extremamente eficiente se nenhuma exceção for levantada. Na verdade, capturar uma exceção é caro. Nas versões do Python anteriores à 2.0 era comum usar esta expressão:

```
try:
    value = mydict[key]
except KeyError:
    mydict[key] = getvalue(key)
    value = mydict[key]
```

Isso somente fazia sentido quando você esperava que o dicionário tivesse uma chave quase que toda vez. Se esse não fosse o caso, você escrevia desta maneira:

```
if key in mydict:
    value = mydict[key]
else:
    value = mydict[key] = getvalue(key)
```

Para este caso específico, você também pode usar `value = dict.setdefault(key, getvalue(key))`, mas apenas se a chamada `getvalue()` tiver pouco custosa o suficiente porque é avaliada em todos os casos.

Por que não existe uma instrução de switch ou case no Python?

Em geral, as instruções switch estruturadas executam um bloco de código quando uma expressão possui um valor ou conjunto de valores específico. Desde o Python 3.10 é possível combinar facilmente valores literais, ou constantes dentro de um espaço de nomes, com uma instrução `match ... case`. Veja a especificação e o tutorial para mais informações sobre instruções **`match`**. Uma alternativa mais antiga é uma sequência de `if... elif... elif... else`.

Para casos em que você precisa escolher entre um grande número de possibilidades, você pode criar um dicionário mapeando valores de caso para funções a serem chamadas. Por exemplo:

```
functions = {'a': function_1,
             'b': function_2,
             'c': self.method_1}

func = functions[value]
func()
```

Para chamar métodos em objetos, você pode simplificar ainda mais usando olá função embutida **`getattr()`** para recuperar métodos com um nome específico:

```
class MyVisitor:
    def visit_a(self):
        ...

    def dispatch(self, value):
        method_name = 'visit_' + str(value)
```

```
        method = getattr(self, method_name)
        method()
```

É sugerido que você use um prefixo para os nomes dos métodos, como `visit_` neste exemplo. Sem esse prefixo, se os valores vierem de uma fonte não confiável, um invasor poderá chamar qualquer método no seu objeto.

Imitar o comportamento do switch no C, em que a execução atravessa as instruções de um case para outro caso não seja interrompida, é possível, mais difícil, e menos necessário.

Você não pode emular threads no interpretador em vez de confiar em uma implementação de thread específica do sistema operacional?

Resposta 1: Infelizmente, o interpretador envia pelo menos um quadro de pilha C para cada quadro de pilha Python. Além disso, as extensões podem retornar ao Python em momentos quase aleatórios. Portanto, uma implementação completa de threads requer suporte de thread para C.

Resposta 2: Felizmente, existe o Stackless Python [https://github.com/stackless-dev/stackless/wiki], que tem um laço de interpretador completamente redesenhado que evita a pilha C.

Por que expressões lambda não podem conter instruções?

Expressões lambda no Python não podem conter instruções porque o framework sintático do Python não consegue manipular instruções aninhadas dentro de expressões. No entanto, no Python, isso não é um problema sério. Diferentemente das formas de lambda em outras linguagens, onde elas adicionam funcionalidade, lambdas de Python são apenas notações simplificadas se você tiver muita preguiça de definir uma função.

Funções já são objetos de primeira classe em Python, e podem ser declaradas em um escopo local. Portanto a única vantagem de usar um lambda em vez de uma função definida localmente é que você não precisa inventar um nome para a função – mas esta só é uma variável local para a qual o objeto da função (que é exatamente do mesmo tipo de um objeto que uma expressão lambda carrega) é atribuído!

O Python pode ser compilado para linguagem de máquina, C ou alguma outra linguagem?

O Cython [https://cython.org/] compila uma versão modificada do Python com anotações opcionais em extensões C. Nuitka [https://nuitka.net/] é um compilador emergente de Python em código C++, com o objetivo de oferecer suporte à linguagem Python completa.

Como o Python gerencia memória?

Os detalhes do gerenciamento de memória Python dependem da implementação. A implementação padrão do Python, CPython, usa contagem de referências para detectar objetos inacessíveis, e outro mecanismo para coletar ciclos de referência, executando periodicamente um algoritmo de detecção de ciclo que procura por ciclos inacessíveis e exclui os objetos envolvidos. O módulo **gc** fornece funções para realizar uma coleta de lixo, obter estatísticas de depuração e ajustar os parâmetros do coletor.

Outras implementações (como Jython [https://www.jython.org] ou PyPy [https://pypy.org]), no entanto, podem contar com um mecanismo diferente, como um coletor de lixo maduro. Essa diferença pode causar alguns problemas sutis de portabilidade se o seu código Python depender do comportamento da implementação de contagem de referências.

Em algumas implementações do Python, o código a seguir (que funciona bem no CPython) provavelmente ficará sem descritores de arquivo:

```
for file in very_long_list_of_files:
    f = open(file)
    c = f.read(1)
```

Na verdade, usando o esquema de contagem de referências e destrutor de referências do CPython, cada nova atribuição a `f` fecha o arquivo anterior. Com um GC tradicional, entretanto, esses objetos arquivo só serão coletados (e fechados) em intervalos variados e possivelmente longos.

Se você quiser escrever um código que funcione com qualquer implementação Python, você deve fechar explicitamente o arquivo ou usar a instrução `with`; isso funcionará independentemente do esquema de gerenciamento de memória:

```
for file in very_long_list_of_files:
    with open(file) as f:
        c = f.read(1)
```

Por que o CPython não usa uma forma mais tradicional de esquema de coleta de lixo?

Por um lado, este não é um recurso padrão C e, portanto, não é portátil. (Sim, sabemos sobre a biblioteca Boehm GC. Ela possui pedaços de código assembler para a *maioria* das plataformas comuns, não para todas elas, e embora seja em sua maioria transparente, não é completamente transparente; são necessários patches para obter Python para trabalhar com isso.)

O GC tradicional também se torna um problema quando o Python é incorporado em outros aplicativos. Embora em um Python independente seja bom substituir o padrão `malloc()` e `free()` por versões fornecidas pela biblioteca GC, uma aplicação que incorpora Python pode querer ter seu *próprio* substituto para `malloc()` e `free()`, e podem não querer Python. No momento, CPython funciona com qualquer coisa que implemente `malloc()` e `free()` corretamente.

Por que toda memória não é liberada quando o CPython fecha?

Os objetos referenciados nos espaço de nomes globais dos módulos Python nem sempre são desalocados quando o Python é encerrado. Isso pode acontecer se houver referências circulares. Existem também certos bits de memória alocados pela biblioteca C que são impossíveis de liberar (por exemplo, uma ferramenta como o Purify reclamará disso). Python é, no entanto, agressivo quanto à limpeza de memória na saída e tenta destruir todos os objetos.

Se você quiser forçar o Python a excluir certas coisas na desalocação, use o módulo `atexit` para executar uma função que forçará essas exclusões.

Por que existem tipos de dados separados para tuplas e listas?

Listas e tuplas, embora semelhantes em muitos aspectos, geralmente são usadas de maneiras fundamentalmente diferentes. Tuplas podem ser consideradas semelhantes a `records` de Pascal ou `structs` de C; são pequenas coleções de dados relacionados que podem ser de diferentes tipos e operados como um grupo. Por exemplo, uma coordenada cartesiana é representada apropriadamente como uma tupla de dois ou três números.

As listas, por outro lado, são mais parecidas com arrays em outras linguagens. Eles tendem a conter um número variável de objetos, todos do mesmo tipo e operados um por um. Por exemplo, `os.listdir('.')` retorna uma lista de strings representando os arquivos no diretório atual. Funções que operam nesta saída geralmente não seriam interrompidas se você adicionasse outro arquivo ou dois ao diretório.

Tuplas são imutáveis, o que significa que, uma vez criada uma tupla, não é possível substituir nenhum de seus elementos por um novo valor. Listas são mutáveis, o que significa que você pode sempre alterar os elementos de uma lista. Apenas objetos hasheáveis (que permitem o cálculo de um *hash*) podem ser usados como chaves de dicionário. A maioria dos tipos

imutáveis é hasheável; por isso, tuplas — mas não listas — podem ser usadas como chaves. Observe, no entanto, que uma tupla só é hasheável se todos os seus elementos também o forem.

Como as listas são implementadas no CPython?

As listas do CPython são, na verdade, vetores de comprimento variável, listas vinculadas não no estilo Lisp. A implementação usa um vetor contíguo de referências a outros objetos e mantém um ponteiro para esse vetor e o comprimento de vetor em uma estrutura de cabeçalho de lista.

Isso torna a indexação de uma lista `a[i]` uma operação cujo custo é independente do tamanho da lista ou do valor do índice.

Quando itens são anexados ou inseridos, o vetor de referências é redimensionado. Alguma inteligência é aplicada para melhorar o desempenho de anexar itens repetidamente; quando o vetor precisa ser aumentado, algum espaço extra é alocado para que as próximas vezes não exijam um redimensionamento real.

Consulte Complexidade de tempo de operações em tipos embutidos para os custos das várias operações de lista.

Como são os dicionários implementados no CPython?

Os dicionários do CPython são implementados como tabelas hash redimensionáveis. Em comparação com árvores B, isso oferece melhor desempenho para pesquisa (de longe a operação mais comum) na maioria das circunstâncias, e a implementação é mais simples.

Os dicionários funcionam calculando um código hash para cada chave armazenada no dicionário usando a função embutida **`hash()`**. O código hash varia amplamente dependendo da chave e da semente por processo; por exemplo, `'Python'` poderia fazer hash para `-539294296` enquanto `'python'`, uma string que difere por um único bit, poderia fazer hash para

`1142331976`. O código hash é então usado para calcular um local em um vetor interno onde o valor será armazenado. Supondo que você esteja armazenando chaves com valores de hash diferentes, isso significa que os dicionários levam um tempo constante – $O(1)$, na notação Big-O – para recuperar uma chave.

Consulte Complexidade de tempo de operações em tipos embutidos para os custos das várias operações de dicionário.

Por que chaves de dicionário devem ser imutáveis?

A implementação da tabela hash de dicionários usa um valor hash calculado a partir do valor da chave para encontrar a chave. Se a chave fosse um objeto mutável, seu valor poderia mudar e, portanto, seu hash também poderia mudar. Mas como quem altera o objeto-chave não pode saber que ele estava sendo usado como chave de dicionário, ele não pode mover a entrada no dicionário. Então, quando você tentar procurar o mesmo objeto no dicionário, ele não será encontrado porque seu valor de hash é diferente. Se você tentasse procurar o valor antigo, ele também não seria encontrado, porque o valor do objeto encontrado naquele hash seria diferente.

Se você deseja que um dicionário seja indexado com uma lista, simplesmente converta primeiro a lista em uma tupla; a função `tuple(L)` cria uma tupla com as mesmas entradas da lista `L`. As tuplas são imutáveis e, portanto, podem ser usadas como chaves de dicionário.

Algumas soluções inaceitáveis que foram propostas:

- Listas de hash por endereço (ID do objeto). Isto não funciona porque se você construir uma nova lista com o mesmo valor ela não será encontrada; por exemplo.:

  ```
  mydict = {[1, 2]: '12'}
  print(mydict[[1, 2]])
  ```

levantaria uma exceção `KeyError` porque o id do `[1, 2]` usado na segunda linha difere daquele da primeira linha. Em outras palavras, as chaves de dicionário devem ser comparadas usando `==`, não usando `is`.

- Fazer uma cópia ao usar uma lista como chave. Isso não funciona porque a lista, sendo um objeto mutável, poderia conter uma referência a si mesma e então o código copiado entraria em um laço infinito.

- Permitir listas como chaves, mas dizer ao usuário para não modificá-las. Isso permitiria uma classe de bugs difíceis de rastrear em programas quando você esquecesse ou modificasse uma lista por acidente. Também invalida uma importante invariante dos dicionários: todo valor em `d.keys()` pode ser usado como chave do dicionário.

- Marcar listas como somente leitura quando forem usadas como chave de dicionário. O problema é que não é apenas o objeto de nível superior que pode alterar seu valor; você poderia usar uma tupla contendo uma lista como chave. Inserir qualquer coisa como chave em um dicionário exigiria marcar todos os objetos acessíveis a partir daí como somente leitura – e, novamente, objetos autorreferenciais poderiam causar um laço infinito.

Existe um truque para contornar isso se você precisar, mas use-o por sua própria conta e risco: você pode agrupar uma estrutura mutável dentro de uma instância de classe que tenha um método `__eq__()` e um método `__hash__()`. Você deve então certificar-se de que o valor de hash para todos os objetos invólucros que residem em um dicionário (ou outra estrutura baseada em hash) permaneça fixo enquanto o objeto estiver no dicionário (ou outra estrutura).

```
class ListWrapper:
    def __init__(self, the_list):
        self.the_list = the_list

    def __eq__(self, other):
        return self.the_list == other.the_list

    def __hash__(self):
        l = self.the_list
```

```
        result = 98767 - len(l)*555
        for i, el in enumerate(l):
            try:
                result = result + (hash(el) % 9999999) * 1001 +
            except Exception:
                result = (result % 7777777) + i * 333
        return result
```

Observe que o cálculo do hash é complicado pela possibilidade de que alguns membros da lista possam ser não não-hasheável e também pela possibilidade de estouro aritmético.

Além disso, deve ser sempre o caso que se `o1 == o2` (ou seja, `o1.__eq__(o2) is True`) então `hash(o1) == hash(o2)` (ou seja, `o1.__hash__() == o2.__hash__()`), independentemente de o objeto estar em um dicionário ou não. Se você não cumprir essas restrições, os dicionários e outras estruturas baseadas em hash se comportarão mal.

No caso de **`ListWrapper`**, sempre que o objeto invólucro estiver em um dicionário, a lista agrupada não deve ser alterada para evitar anomalias. Não faça isso a menos que esteja preparado para pensar muito sobre os requisitos e as consequências de não atendê-los corretamente. Considere-se avisado.

Por que lista.sort() não retorna a lista ordenada?

Em situações nas quais desempenho importa, fazer uma cópia da lista só para ordenar seria desperdício. Portanto, **`lista.sort()`** ordena a lista. De forma a lembrá-lo desse fato, isso não retorna a lista ordenada. Desta forma, você não vai ser confundido a acidentalmente sobrescrever uma lista quando você precisar de uma cópia ordenada mas também precisar manter a versão não ordenada.

Se você quiser retornar uma nova lista, use a função embutida **`sorted()`** ao invés. Essa função cria uma nova lista a partir de um iterável provido, o ordena e retorna. Por exemplo, aqui é como se itera em cima das chaves de um dicionário de maneira ordenada:

```
for key in sorted(mydict):
    ...  # use mydict[key] conforme quiser...
```

Como você especifica e aplica um spec de interface no Python?

Uma especificação de interface para um módulo fornecida por linguagens como C++ e Java descreve os protótipos para os métodos e funções do módulo. Muitos acham que a aplicação de especificações de interface em tempo de compilação ajuda na construção de programas grandes.

Python 2.6 adiciona um módulo `abc` que permite definir Classes Base Abstratas (ABCs). Você pode então usar `isinstance()` e `issubclass()` para verificar se uma instância ou classe implementa um ABC específico. O módulo `collections.abc` define um conjunto de ABCs úteis como `Iterable`, `Container` e `MutableMapping`

Para Python, muitas das vantagens das especificações de interface podem ser obtidas por uma disciplina de teste apropriada para componentes.

Um bom conjunto de testes para um módulo pode fornecer um teste de regressão e servir como uma especificação de interface do módulo e um conjunto de exemplos. Muitos módulos Python podem ser executados como um script para fornecer um simples “autoteste”. Mesmo módulos que usam interfaces externas complexas muitas vezes podem ser testados isoladamente usando emulações triviais da interface externa. Os módulos `doctest` e `unittest` ou estruturas de teste de terceiros podem ser usados para construir conjuntos de testes exaustivos que exercitam cada linha de código em um módulo.

Uma disciplina de teste apropriada pode ajudar a construir aplicações grandes e complexas no Python, assim como ter especificações de interface. Na verdade, pode ser melhor porque uma especificação de interface não pode testar certas propriedades de um programa. Por exemplo, espera-se que o método `list.append()` adicione novos elementos ao final de alguma lista interna; uma especificação de interface não pode testar se sua

implementação **`list.append()`** realmente fará isso corretamente, mas é trivial verificar esta propriedade em um conjunto de testes.

Escrever conjuntos de testes é muito útil e você pode querer projetar seu código para torná-lo facilmente testável. Uma técnica cada vez mais popular, o desenvolvimento orientado a testes, exige a escrita de partes do conjunto de testes primeiro, antes de escrever qualquer parte do código real. É claro que o Python permite que você seja desleixado e nem escreva casos de teste.

Por que não há goto?

Na década de 1970, as pessoas perceberam que o goto irrestrito poderia levar a um código "espaguete" confuso, difícil de entender e revisar. Em uma linguagem de alto nível, também é desnecessário, desde que existam maneiras de ramificar (em Python, com instruções **`if`** e **`or`**, **`and`** e expressões **`if`**/**`else`**) e iterar (com instruções **`while`** e **`for`**, possivelmente contendo **`continue`** e **`break`**).

Também é possível usar exceções para fornecer um "goto estruturado" que funcione mesmo em chamadas de função. Muitos acham que as exceções podem convenientemente emular todos os usos razoáveis das construções `go` ou `goto` de C, Fortran e outras linguagens. Por exemplo:

```
class label(Exception): pass  # declara um label

try:
    ...
    if condition: raise label()  # goto label
    ...
except label:  # destino do goto
    pass
...
```

Isso não permite que você pule para o meio de um laço, mas isso geralmente é considerado um abuso de `goto` de qualquer maneira. Use com moderação.

Por que strings brutas (r-strings) não podem terminar com uma contrabarra?

Mais precisamente, eles não podem terminar com um número ímpar de contrabarras: a contrabarra não pareada no final escapa do caractere de aspa de fechamento, deixando uma string não terminada.

Strings brutas foram projetadas para facilitar a criação de entrada para processadores (principalmente mecanismos de expressão regular) que desejam fazer seu próprio processamento de escape de contrabarra. De qualquer forma, esses processadores consideram uma contrabarra incomparável como um erro, portanto, as strings brutas não permitem isso. Em troca, eles permitem que você transmita o caractere de aspas da string escapando dele com uma contrabarra. Essas regras funcionam bem quando r-strings são usadas para a finalidade pretendida.

Se você estiver tentando criar nomes de caminho do Windows, observe que todas as chamadas do sistema do Windows também aceitam barras:

```
f = open("/mydir/file.txt")  # funciona normalmente!
```

Se você estiver tentando construir um nome de caminho para um comando DOS, tente, por exemplo, algum desses

```
dir = r"\this\is\my\dos\dir" "\\"
dir = r"\this\is\my\dos\dir\ "[:-1]
dir = "\\this\\is\\my\\dos\\dir\\"
```

Por que o Python não tem uma instrução “with” para atribuição de atributos?

Python tem uma instrução `with` que envolve a execução de um bloco, chamando o código na entrada e na saída do bloco. Algumas linguagens têm uma construção desta forma:

```
with obj:
    a = 1               # equivalente a obj.a = 1
```

```
    total = total + 1   # obj.total = obj.total + 1
```

Em Python, tal construção seria ambígua.

Outras linguagens, como Object Pascal, Delphi, e C++, usam tipos estáticos, então é possível saber, de maneira não ambígua, que membro está sendo atribuído. Esse é o principal ponto da tipagem estática – o compilador *sempre* sabe o escopo de toda variável em tempo de compilação.

O Python usa tipos dinâmicos. É impossível saber com antecedência que atributo vai ser referenciado em tempo de execução. Atributos membro podem ser adicionados ou removidos de objetos dinamicamente. Isso torna impossível saber, de uma leitura simples, que atributo está sendo referenciado: um atributo local, um atributo global ou um atributo membro?

Por exemplo, pegue o seguinte trecho incompleto:

```
def foo(a):
    with a:
        print(x)
```

O trecho presume que `a` deve ter um atributo de membro chamado `x`. No entanto, não há nada em Python que diga isso ao interpretador. O que deveria acontecer se `a` for, digamos, um número inteiro? Se houver uma variável global chamada `x`, ela será usada dentro do bloco **`with`**? Como você pode ver, a natureza dinâmica do Python torna essas escolhas muito mais difíceis.

O principal benefício de **`with`** e recursos de linguagem semelhantes (redução do volume de código) pode, no entanto, ser facilmente alcançado em Python por atribuição. Em vez de:

```
function(args).mydict[index][index].a = 21
function(args).mydict[index][index].b = 42
function(args).mydict[index][index].c = 63
```

escreva isso:

```
ref = function(args).mydict[index][index]
ref.a = 21
```

```
ref.b = 42
ref.c = 63
```

Isso também tem o efeito colateral de aumentar a velocidade de execução porque as ligações de nome são resolvidas a tempo de execução em Python, e a segunda versão só precisa performar a resolução uma vez.

Propostas semelhantes que introduziriam sintaxe para reduzir ainda mais o volume do código, como o uso de um 'ponto inicial', foram rejeitadas em favor da explicitação (consulte https://mail.python.org/pipermail/python-ideas/2016-May/040070.html).

Por que os geradores não suportam a instrução with?

Por razões técnicas, um gerador usado diretamente como gerenciador de contexto não funcionaria corretamente. Quando, como é mais comum, um gerador é usado como um iterador executado até a conclusão, nenhum fechamento é necessário. Quando estiver, envolva-o como `contextlib.closing(generator)` na instrução `with`.

Por que dois pontos são necessários para as instruções de if/while/def/class?

Os dois pontos são obrigatórios primeiramente para melhorar a leitura (um dos resultados da linguagem experimental ABC). Considere isso:

```
if a == b
    print(a)
```

versus

```
if a == b:
    print(a)
```

Note como a segunda é ligeiramente mais fácil de ler. Note com mais atenção como os dois pontos iniciam o exemplo nessa resposta de FAQ; é

um uso padrão em inglês.

Outro motivo menor é que os dois pontos deixam mais fácil para os editores com realce de sintaxe; eles podem procurar por dois pontos para decidir quando a recuo precisa ser aumentada em vez de precisarem fazer uma análise mais elaborada do texto do programa.

Por que o Python permite vírgulas ao final de listas e tuplas?

O Python deixa você adicionar uma vírgula ao final de listas, tuplas e dicionários:

```
[1, 2, 3,]
('a', 'b', 'c',)
d = {
    "A": [1, 5],
    "B": [6, 7],  # a vírgula ao final é opcional, mas é bom est
}
```

Existem várias razões para permitir isso.

Quando você possui um valor literal para uma lista, tupla, ou dicionário disposta através de múltiplas linhas, é mais fácil adicionar mais elementos porque você não precisa lembrar de adicionar uma vírgula na linha anterior. As linhas também podem ser reordenadas sem criar um erro de sintaxe.

Acidentalmente omitir a vírgula pode levar a erros que são difíceis de diagnosticar. Por exemplo:

```
x = [
  "fee",
  "fie"
  "foo",
  "fum"
]
```

Essa lista parece ter quatro elementos, mas na verdade contém três: “fee”, “fiefoo” e “fum”. Sempre adicionar a vírgula evita essa fonte de erro.

Permitir a vírgula no final também pode deixar a geração de código programático mais fácil.

FAQ de Bibliotecas e Extensões

Sumário

- FAQ de Bibliotecas e Extensões
  - Questões gerais sobre bibliotecas
    - Como encontrar um módulo ou aplicação para realizar uma tarefa X?
    - Onde está o código-fonte do math.py (socket.py, regex.py, etc.)?
    - Como tornar um script Python executável no Unix?
    - Existe um pacote de curses/termcap para Python?
    - Existe a função onexit() equivalente ao C no Python?
    - Por que o meu manipulador de sinal não funciona?
  - Tarefas comuns
    - Como testar um programa ou componente Python?
    - Como faço para criar uma documentação de doc strings?
    - Como faço para pressionar uma tecla de cada vez?
  - Threads
    - Como faço para programar usando threads?
    - Nenhuma de minhas threads parece funcionar, por que?
    - Como distribuo o trabalho entre várias threads de trabalho?
    - Que tipos de variáveis globais mutáveis são seguras para thread?
    - Não podemos remover a Trava Global do interpretador?
  - Entrada e Saída
    - Como faço para excluir um arquivo? (E outras perguntas sobre arquivos)
    - Como eu copio um arquivo?
    - Como leio (ou escrevo) dados binários?
    - Por que não consigo usar os.read() em um encadeamento com os.popen()?
    - Como acesso a porta serial (RS232)?
    - Por que o sys.stdout (stdin, stderr) não fecha?

- Programação Rede / Internet
  - Quais ferramentas para WWW existem no Python?
  - Qual módulo devo usar para ajudar na geração do HTML?
  - Como envio um e-mail de um script Python?
  - Como evito um bloqueio no método connect() de um soquete?
- Base de Dados
  - Existem interfaces para banco de dados em Python?
  - Como você implementa objetos persistentes no Python?
- Matemáticos e Numéricos
  - Como gero número aleatórios no Python?

Questões gerais sobre bibliotecas

Como encontrar um módulo ou aplicação para realizar uma tarefa X?

Verifique a Referência de Bibliotecas para ver se há um módulo relevante da biblioteca padrão. (Eventualmente, você aprenderá o que está na biblioteca padrão e poderá pular esta etapa.)

Para pacotes de terceiros, pesquise no Python Package Index [https://pypi.org] ou tente no Google [https://www.google.com] ou outro buscador na web. Pesquisando por "Python" mais uma ou dois argumentos nomeados do seu tópico de interesse geralmente encontrará algo útil.

Onde está o código-fonte do math.py (socket.py, regex.py, etc.)?

Se você não conseguir encontrar um arquivo de origem para um módulo, ele pode ser um módulo embutido ou carregado dinamicamente, implementado em C, C++ ou outra linguagem compilada. Nesse caso, você pode não ter o arquivo de origem ou pode ser algo como `mathmodule.c`, em algum lugar do diretório de origem C (não no caminho do Python).

Existem (pelo menos) três tipos de módulos no Python:

1. módulos escritos em Python (.py)

2. módulos escritos em C e carregados dinamicamente (.dll, .pyd, .so, .sl, etc.);

3. módulos escritos em C e vinculados ao interpretador; para obter uma dessas listas, digite:

   ```
   import sys
   print(sys.builtin_module_names)
   ```

Como tornar um script Python executável no Unix?

Você precisa fazer duas coisas: o arquivo do script deve ser executável e a primeira linha deve começar com `#!` seguido do caminho do interpretador Python.

Inicialmente, execute o `chmod +x scriptfile` ou, talvez, o `chmod 755 scriptfile`.

A segunda coisa pode ser feita de várias maneiras. A maneira mais direta é escrever

```
#!/usr/local/bin/python
```

como a primeira linha do seu arquivo, usando o endereço do caminho onde o interpretador Python está instalado.

Se você deseja que o script seja independente de onde o interpretador Python mora, você pode usar o programa **env**. Quase todas as variantes do Unix suportam o seguinte, presumindo que o interpretador Python esteja em um diretório no **`PATH`** do usuário:

```
#!/usr/bin/env python
```

Não faça isso para CGI scripts. A variável **`PATH`** para CGI scripts é normalmente muito pequena, portanto, você precisa usar o caminho completo do interpretador.

Ocasionalmente, o ambiente de um usuário está tão cheio que o programa **/usr/bin/env** falha; ou não há nenhum programa env. Nesse caso, você pode tentar o seguinte hack (graças a Alex Rezinsky):

```
#! /bin/sh
""":"
exec python $0 ${1+"$@"}
"""
```

Uma pequena desvantagem é que isso define o script's __doc__ string. Entretanto, você pode corrigir isso adicionando

```
__doc__ = """...Alguma coisa..."""
```

Existe um pacote de curses/termcap para Python?

Para variantes Unix: A distribuição fonte padrão do Python vem com um módulo do curses no subdiretório Modules [https://github.com/python/cpython/tree/3.15/Modules], embora não seja compilado por padrão. (Observe que isso não está disponível na distribuição do Windows – não há módulo curses para o Windows.)

O módulo `curses` provê recursos básicos de curses, bem como muitas funções adicionais de ncurses e curses SYSV, como cor, suporte a conjuntos de caracteres alternativos, pads e suporte a mouse. Isso significa que o módulo não é compatível com sistemas operacionais que possuem apenas maldições BSD, mas não parece haver nenhum sistema operacional mantido atualmente que se enquadre nesta categoria.

Existe a função onexit() equivalente ao C no Python?

O módulo `atexit` fornece uma função de registro similar ao `onexit()` do C.

Por que o meu manipulador de sinal não funciona?

O maior problema é que o manipulador de sinal é declarado com uma lista de argumentos incorretos. Isso é chamado como

```
manipulador(num_sinal, quadro)
```

portanto, isso deve ser declarado com dois parâmetros:

```
def manipulador(num_sinal, quadro):
    ...
```

Tarefas comuns

Como testar um programa ou componente Python?

A Python vem com dois frameworks de teste. O módulo `doctest` busca por exemplos nas docstrings de um módulo e os executa, comparando o resultado com a saída esperada informada na docstring.

O módulo `unittest` é uma estrutura de teste mais sofisticada, modelada nas estruturas de teste do Java e do Smalltalk.

Para facilitar os testes, você deve usar um bom design modular em seu programa. Seu programa deve ter quase todas as funcionalidades encapsuladas em funções ou métodos de classe – e isso às vezes tem o efeito surpreendente e agradável de fazer o programa executar mais rápido (porque os acessos às variáveis locais são mais rápidos que os acessos globais). Além disso, o programa deve evitar depender de variáveis globais mutantes, pois isso torna os testes muito mais difíceis de serem realizados.

A lógica principal do seu programa pode tão simples quanto

```
if __name__ == "__main__":
    main_logic()
```

no botão do módulo principal do seus programa.

Depois que seu programa estiver organizado como uma coleção tratável de comportamentos de funções e classes, você deverá escrever funções de teste que exercitem os comportamentos. Um conjunto de testes que automatiza uma sequência de testes pode ser associado a cada módulo. Parece muito trabalhoso, mas como o Python é tão conciso e flexível, é

surpreendentemente fácil. Você pode tornar a codificação muito mais agradável e divertida escrevendo suas funções de teste em paralelo com o "código de produção", pois isso torna mais fácil encontrar bugs e até mesmo falhas de design mais cedo.

Os "módulos de suporte" que não se destinam a ser o módulo principal de um programa podem incluir um autoteste do módulo.

```
if __name__ == "__main__":
    self_test()
```

Mesmo quando as interfaces externas não estiverem disponíveis, os programas que interagem com interfaces externas complexas podem ser testados usando as interfaces "falsas" implementadas no Python.

Como faço para criar uma documentação de doc strings?

O módulo `pydoc` pode criar HTML a partir das strings de documentos em seu código-fonte Python. Uma alternativa para criar documentação de API puramente a partir de docstrings é epydoc [https://epydoc.sourceforge.net/]. Sphinx [https://www.sphinx-doc.org] também pode incluir conteúdo docstring.

Como faço para pressionar uma tecla de cada vez?

Para variantes do Unix existem várias soluções. Apesar de ser um módulo grande para aprender, é simples fazer isso usando o módulo curses.

Threads

Como faço para programar usando threads?

Certifique-se de usar o módulo `threading` e não o módulo `_thread`. O módulo **`threading`** constrói abstrações convenientes sobre as primitivas de baixo nível fornecidas pelo módulo **`_thread`**.

Nenhuma de minhas threads parece funcionar, por que?

Assim que a thread principal acaba, todas as threads são eliminadas. Sua thread principal está sendo executada tão rápida que não está dando tempo para realizar qualquer trabalho.

Uma solução simples é adicionar um tempo de espera no final do programa até que todos os threads sejam concluídos:

```
import threading, time

def thread_task(name, n):
    for i in range(n):
        print(name, i)

for i in range(10):
    T = threading.Thread(target=thread_task, args=(str(i), i))
    T.start()

time.sleep(10)  # <---------------------------!
```

Mas agora (em muitas plataformas) as threads não funcionam em paralelo, mas parecem funcionar sequencialmente, um de cada vez! O motivo é que o agendador de threads do sistema operacional não inicia uma nova thread até que a thread anterior seja bloqueada.

Uma solução simples é adicionar um pequeno tempo de espera no início da função:

```
def thread_task(name, n):
    time.sleep(0.001)  # <--------------------!
    for i in range(n):
        print(name, i)

for i in range(10):
    T = threading.Thread(target=thread_task, args=(str(i), i))
    T.start()

time.sleep(10)
```

Em vez de tentar adivinhar um bom valor de atraso para `time.sleep()`, é melhor usar algum tipo de mecanismo de semáforo. Uma ideia é usar o módulo `queue` para criar um objeto fila, deixar cada thread anexar um token

à fila quando terminar e deixar a thread principal ler tantos tokens da fila quantos threads houver.

Como distribuo o trabalho entre várias threads de trabalho?

A maneira mais fácil é usar o módulo `concurrent.futures`, especialmente a classe `ThreadPoolExecutor`.

Ou, se quiser um controle preciso sobre o algoritmo de despacho, você pode escrever sua própria lógica manualmente. Use o módulo `queue` para criar uma fila contendo uma lista de tarefas. A classe `Queue` mantém uma lista de objetos e possui um método `.put(obj)` que adiciona itens à fila e um método `.get()` para retorná-los. A classe cuidará da trava necessário para garantir que cada trabalho seja entregue exatamente uma vez.

Aqui está um exemplo simples:

```
import threading, queue, time

# A thread do worker obtém a tarefa da fila. Quando a fila está
# ela presume que não haverá mais tarefas e encerra.
# (Realisticamente, workers trabalharão até serem encerrados.)
def worker():
    print('Running worker')
    time.sleep(0.1)
    while True:
        try:
            arg = q.get(block=False)
        except queue.Empty:
            print('Worker', threading.current_thread(), end=' ')
            print('queue empty')
            break
        else:
            print('Worker', threading.current_thread(), end=' ')
            print('running with argument', arg)
            time.sleep(0.5)

# Cria a fila
q = queue.Queue()
```

```
# Inicia um pool de 5 workers
for i in range(5):
    t = threading.Thread(target=worker, name='worker %i' % (i+1)
    t.start()

# Começa a adicionar tarefa à fila
for i in range(50):
    q.put(i)

# Dá às threads tempo para executar
print('Main thread sleeping')
time.sleep(5)
```

Quando executado, isso produzirá a seguinte saída:

```
Running worker
Running worker
Running worker
Running worker
Running worker
Main thread sleeping
Worker <Thread(worker 1, started 130283832797456)> running with
Worker <Thread(worker 2, started 130283824404752)> running with
Worker <Thread(worker 3, started 130283816012048)> running with
Worker <Thread(worker 4, started 130283807619344)> running with
Worker <Thread(worker 5, started 130283799226640)> running with
Worker <Thread(worker 1, started 130283832797456)> running with
...
```

Consulte a documentação do módulo para mais detalhes; a classe `Queue` fornece uma interface com recursos.

Que tipos de variáveis globais mutáveis são seguras para thread?

Uma trava global do interpretador (GIL) é usada internamente para garantir que apenas um thread seja executado na VM Python por vez. Em geral, Python oferece alternar entre threads apenas entre instruções de bytecode; a frequência com que ele muda pode ser definida via

`sys.setswitchinterval()`. Cada instrução de bytecode e, portanto, todo o código de implementação C alcançado por cada instrução é, portanto, atômico do ponto de vista de um programa Python.

Em teoria, isso significa que uma contabilidade exata requer um entendimento exato da implementação do bytecode PVM. Na prática, isso significa que as operações em variáveis compartilhadas de tipos de dados integrados (inteiros, listas, dicionários, etc.) que "parecem atômicas" realmente são.

Por exemplo, as seguintes operações são todas atômicas (L, L1, L2 são listas, D, D1, D2 são dicionários, x, y são objetos, i, j são inteiros):

```
L.append(x)
L1.extend(L2)
x = L[i]
x = L.pop()
L1[i:j] = L2
L.sort()
x = y
x.field = y
D[x] = y
D1.update(D2)
D.keys()
```

Esses não são:

```
i = i+1
L.append(L[-1])
L[i] = L[j]
D[x] = D[x] + 1
```

Operações que substituem outros objetos podem invocar o método `__del__()` desses outros objetos quando sua contagem de referências chega a zero, e isso pode afetar as coisas. Isto é especialmente verdadeiro para as atualizações em massa de dicionários e listas. Em caso de dúvida, use um mutex!

Não podemos remover a Trava Global do interpretador?

A trava global do interpretador (GIL) é frequentemente vista como um obstáculo para a implantação do Python em máquinas servidoras multiprocessadas de ponta, porque um programa Python multi-threaded efetivamente usa apenas uma CPU, devido à insistência de que (quase) todo código Python só pode ser executado enquanto a GIL é mantida.

Com a aprovação da **PEP 703** [https://peps.python.org/pep-0703/], o trabalho está em andamento para remover a GIL da implementação CPython do Python. Inicialmente, ele será implementado como um sinalizador opcional de compilador ao construir o interpretador, e assim construções separadas estarão disponíveis com e sem a GIL. A longo prazo, a esperança é estabelecer uma única construção, uma vez que as implicações de desempenho da remoção do GIL sejam totalmente compreendidas. O Python 3.13 provavelmente será a primeira versão contendo esse trabalho, embora possa não ser completamente funcional nesta versão.

O trabalho atual para remover a GIL é baseado em um fork do Python 3.9 com a GIL removida [https://github.com/colesbury/nogil] por Sam Gross. Antes disso, na época do Python 1.5, Greg Stein implementou um conjunto abrangente de patches (os patches de "threads livres") que removeu a GIL e a substituiu por um travamento de granulação fina. Adam Olsen fez um experimento semelhante em seu projeto python-safethread [https://code.google.com/archive/p/python-safethread]. Infelizmente, ambos os experimentos iniciais exibiram uma queda acentuada no desempenho de thread único (pelo menos 30% mais lento), devido à quantidade de travamento de granulação fina necessária para compensar a remoção da GIL. O fork do Python 3.9 é a primeira tentativa de remover a GIL com um impacto aceitável no desempenho.

A presença da GIL nas versões atuais do Python não significa que você não pode fazer bom uso do Python em máquinas com várias CPUs! Você só precisa ser criativo ao dividir o trabalho entre vários *processos* em vez de vários *threads*. A classe `ProcessPoolExecutor` no novo módulo `concurrent.futures` fornece uma maneira fácil de fazer isso; o módulo `multiprocessing` fornece uma API de nível inferior, caso você queira mais controle sobre o despacho de tarefas.

O uso criterioso de extensões C também ajudará; se você usar uma extensão C para executar uma tarefa demorada, a extensão poderá liberar a GIL enquanto o thread de execução estiver no código C e permitir que outros threads realizem algum trabalho. Alguns módulos de biblioteca padrão como `zlib` e `hashlib` já fazem isso.

Uma abordagem alternativa para reduzir o impacto da GIL é fazer da GIL uma trava por estado do interpretador em vez de verdadeiramente global. Isso foi implementado pela primeira vez no Python 3.12 e está disponível na API C. Uma interface Python para ele é esperada no Python 3.13. A principal limitação para ele no momento provavelmente são módulos de extensão de terceiros, já que estes devem ser escritos com múltiplos interpretadores em mente para serem utilizáveis, então muitos módulos de extensão mais antigos não serão utilizáveis.

Entrada e Saída

Como faço para excluir um arquivo? (E outras perguntas sobre arquivos)

Use `os.remove(arquivo)` ou `os.unlink(arquivo)`; para documentação, veja o módulo `os`. As duas funções são idênticas; `unlink()` é simplesmente o nome da chamada do sistema para esta função no Unix.

Para remover um diretório, use `os.rmdir()`; use `os.mkdir()` para criar um. `os.makedirs(caminho)` criará quaisquer diretórios intermediários em `path` que não existam. `os.removedirs(caminho)` removerá diretórios intermediários, desde que estejam vazios; se quiser excluir toda uma árvore de diretórios e seu conteúdo, use `shutil.rmtree()`.

Para renomear um arquivos, use `os.rename(caminho_antigo, caminho_novo)`.

Para truncar um arquivo, abra-o usando `f = open(arquivo, "rb+")`, e use `f.truncate(posição)`; a posição tem como padrão a posição atual de busca. Há também `os.ftruncate(fd, posição)` para arquivos abertos com `os.open()`, onde *fd* é o descritor de arquivo (um pequeno inteiro).

O módulo `shutil` também contém uma série de funções para trabalhar em arquivos, incluindo `copyfile()`, `copytree()` e `rmtree()`.

Como eu copio um arquivo?

O módulo `shutil` contém uma função `copyfile()`. Observe que em volumes NTFS do Windows, ele não copia fluxos de dados alternativos [https://en.wikipedia.org/wiki/NTFS#Alternate_data_stream_(ADS)] nem forks de recursos [https://en.wikipedia.org/wiki/Resource_fork] em volumes HFS+ do macOS, embora ambos sejam raramente usados agora. Ele também não copia permissões de arquivo e metadados, embora usar `shutil.copy2()` em vez disso preserve a maioria (embora não todos) deles.

Como leio (ou escrevo) dados binários?

Para ler ou escrever formatos de dados binários complexos, é melhor usar o módulo `struct`. Ele permite que você pegue uma string contendo dados binários (geralmente números) e a converta em objetos Python; e vice-versa.

Por exemplo, o código a seguir lê dois inteiros de 2 bytes e um inteiro de 4 bytes no formato big-endian de um arquivo:

```
import struct

with open(filename, "rb") as f:
    s = f.read(8)
    x, y, z = struct.unpack(">hhl", s)
```

O ‘>’ na string de formato força dados big-endian; a letra ‘h’ lê um “inteiro curto” (2 bytes) e ‘l’ lê um “inteiro longo” (4 bytes) da string.

Para dados mais regulares (por exemplo, uma lista homogênea de ints ou floats), você também pode usar o módulo `array`.

Nota

Para ler e escrever dados binários, é obrigatório abrir o arquivo no modo binário (aqui, passando `"rb"` para `open()`). Se você usar `"r"` em vez disso (o padrão), o arquivo será aberto no modo texto e `f.read()` retornará objetos `str` em vez de objetos `bytes`.

Por que não consigo usar os.read() em um encadeamento com os.popen()?

`os.read()` é uma função de baixo nível que pega um descritor de arquivo, um pequeno inteiro representando o arquivo aberto. `os.popen()` cria um objeto arquivo de alto nível, o mesmo tipo retornado pela função embutida `open()`. Assim, para ler *n* bytes de um pipe *p* criado com `os.popen()`, você precisa usar `p.read(n)`.

Como acesso a porta serial (RS232)?

Para Win32, OSX, Linux, BSD, Jython, IronPython:

pyserial [https://pypi.org/project/pyserial/]

Para Unix, veja uma postagem da Usenet de Mitch Chapman:

https://groups.google.com/groups?selm=34A04430.CF9@ohioee.com

Por que o sys.stdout (stdin, stderr) não fecha?

Os objetos arquivos do Python são uma camada de alto nível de abstração em descritores de arquivo C de baixo nível.

Para a maioria dos objetos arquivo que você cria em Python por meio da função embutida `open()`, `f.close()` marca o objeto arquivo Python como fechado do ponto de vista do Python e também organiza o fechamento do descritor de arquivo C subjacente. Isso também acontece automaticamente no destrutor de `f`, quando `f` se torna lixo.

Mas stdin, stdout e stderr são tratados de forma especial pelo Python, devido ao status especial que também lhes é dado pelo C. Executar `sys.stdout.close()` marca o objeto arquivo de nível Python como fechado, mas *não* fecha o descritor de arquivo C associado.

Para fechar o descritor de arquivo C subjacente para um desses três, você deve primeiro ter certeza de que é isso que você realmente quer fazer (por exemplo, você pode confundir módulos de extensão tentando fazer E/S). Se for, use **`os.close()`**:

```
os.close(stdin.fileno())
os.close(stdout.fileno())
os.close(stderr.fileno())
```

Ou você pode usar as constantes numéricas 0, 1 e 2, respectivamente.

Programação Rede / Internet

Quais ferramentas para WWW existem no Python?

Veja os capítulos intitulados Protocolos de Internet e Suporte e Manuseio de Dados na Internet no Manual de Referência da Biblioteca. Python tem muitos módulos que ajudarão você a construir sistemas web do lado do servidor e do lado do cliente.

Um resumo dos frameworks disponíveis é disponibilizado por Paul Boddie em https://wiki.python.org/moin/WebProgramming.

Qual módulo devo usar para ajudar na geração do HTML?

Você pode encontrar uma coleção de links úteis na página wiki WebProgramming [https://wiki.python.org/moin/WebProgramming].

Como envio um e-mail de um script Python?

Use a biblioteca padrão do módulo **`smtplib`**.

Aqui está um remetente de e-mail interativo muito simples. Este método funcionará em qualquer host que suporte o protocolo SMTP.

```
import sys, smtplib

fromaddr = input("From: ")
toaddrs  = input("To: ").split(',')
print("Enter message, end with ^D:")
msg = ''
while True:
    line = sys.stdin.readline()
    if not line:
        break
    msg += line

# O envio de e-mail real
server = smtplib.SMTP('localhost')
server.sendmail(fromaddr, toaddrs, msg)
server.quit()
```

Uma alternativa somente para Unix usa o sendmail. A localização do programa sendmail varia entre os sistemas; às vezes é `/usr/lib/sendmail`, às vezes `/usr/sbin/sendmail`. A página de manual do sendmail vai ajudar você. Aqui está um código de exemplo:

```
import os

SENDMAIL = "/usr/sbin/sendmail"  # local do sendmail
p = os.popen("%s -t -i" % SENDMAIL, "w")
p.write("To: receiver@example.com\n")
p.write("Subject: teste\n")
p.write("\n")  # linha vazia separando cabeçalho do corpo
p.write("Algum texto\n")
p.write("mais um pouco de texto\n")
sts = p.close()
if sts != 0:
    print("Status de saída do sendmail", sts)
```

Como evito um bloqueio no método connect() de um soquete?

O módulo `select` é normalmente usado para ajudar com E/S assíncrona nos soquetes.

Para evitar que a conexão TCP bloqueie, você pode definir o soquete para o modo sem bloqueio. Então, quando você fizer o `connect()`, você se conectará imediatamente (improvável) ou obterá uma exceção que contém o número de erro como `.errno`. `errno.EINPROGRESS` indica que a conexão está em andamento, mas ainda não terminou. Diferentes sistemas operacionais retornarão valores diferentes, então você terá que verificar o que é retornado no seu sistema.

Você pode usar o método `connect_ex()` para evitar criar uma exceção. Ele retornará apenas o valor de errno. Para pesquisar, você pode chamar **`connect_ex()`** novamente mais tarde – `0` ou `errno.EISCONN` indicam que você está conectado – ou você pode passar este soquete para `select.select()` para verificar se ele é gravável.

Nota

O módulo `asyncio` fornece uma biblioteca assíncrona de thread única e concorrente de propósito geral, que pode ser usada para escrever código de rede não bloqueante. A biblioteca de terceiros Twisted [https://twisted.org/] é uma alternativa popular e rica em recursos.

Base de Dados

Existem interfaces para banco de dados em Python?

Sim.

Interfaces para hashes baseados em disco, como `DBM` e `GDBM` também estão incluídas no Python padrão. Há também o módulo `sqlite3`, que fornece um banco de dados relacional baseado em disco leve.

Suporte para a maioria dos bancos de dados relacionais está disponível. Para mais detalhes, veja a página wiki DatabaseProgramming

[https://wiki.python.org/moin/DatabaseProgramming] para detalhes.

Como você implementa objetos persistentes no Python?

O módulo de biblioteca `pickle` resolve isso de uma maneira muito geral (embora você ainda não possa armazenar coisas como arquivos abertos, soquetes ou janelas), e o módulo de biblioteca `shelve` usa pickle e (g)dbm para criar mapeamentos persistentes contendo objetos Python arbitrários.

Matemáticos e Numéricos

Como gero número aleatórios no Python?

O módulo padrão `random` implementa um gerador de números aleatórios. O uso é simples:

```
import random
random.random()
```

Isso retorna um número flutuante aleatório no intervalo [0, 1).

Existem também muitos outros geradores aleatórios neste módulo, como:

- `randrange(a, b)` escolhe um número inteiro no intervalo entre [a, b).
- `uniform(a, b)` escolhe um número de ponto flutuante no intervalo [a, b).
- `normalvariate(mean, sdev)` gera números pseudoaleatórios que seguem uma distribuição normal (Gaussiana).

Algumas funções de nível elevado operam diretamente em sequencia, como:

- `choice(S)` escolhe um elemento aleatório de uma determinada sequência.
- `shuffle(L)` embaralha uma lista internamente, ou seja permuta seus elementos aleatoriamente.

Existe também uma classe `Random` que você pode instanciar para criar vários geradores de números aleatórios independentes.

FAQ sobre Extensão/Incorporação

Sumário

- FAQ sobre Extensão/Incorporação
  - Posso criar minhas próprias funções em C?
  - Posso criar minhas próprias funções em C++?
  - Escrever C é difícil; há alguma alternativa?
  - Como posso executar instruções arbitrárias de Python a partir de C?
  - Como posso executar e obter o resultado de uma expressão Python arbitrária a partir de C?
  - Como extraio valores em C a partir de um objeto Python?
  - Como posso utilizar Py_BuildValue() para criar uma tupla de comprimento arbitrário?
  - Como eu chamo um método de um objeto a partir do C?
  - Como posso capturar a saída da função PyErr_Print() (ou qualquer outra coisa que escreva para stdout/stderr)?
  - Como faço para acessar a partir do C um módulo escrito em Python?
  - Como posso interagir com objetos C++ a partir do Python?
  - Adicionei um módulo usando o arquivo de Setup e o make falha; por quê?
  - Como eu depuro uma extensão?
  - Quero compilar um módulo Python no meu sistema Linux, mas alguns arquivos estão faltando. Por quê?
  - Como posso distinguir “entrada incompleta” de “entrada inválida”?
  - Como encontro os símbolos __builtin_new ou __pure_virtual não-definidos no g++?
  - Posso criar uma classe de objetos com alguns métodos implementados em C e outros em Python (por exemplo, via herança)?

Posso criar minhas próprias funções em C?

Sim, você pode construir módulos embutidos contendo funções, variáveis, exceções e até mesmo novos tipos em C. Isso é explicado no documento Estendendo e incorporando o interpretador Python.

A maioria dos livros intermediários ou avançados em Python também abordará esse tópico.

Posso criar minhas próprias funções em C++?

Sim, usando recursos de compatibilidade encontrados em C++. Coloque `extern "C" { ... }` em torno dos arquivos de inclusão do Python e coloque `extern "C"` antes de cada função que será chamada pelo interpretador do Python. Objetos globais ou estáticos em C++ com construtores provavelmente não são uma boa ideia.

Escrever C é difícil; há alguma alternativa?

Há diversas alternativas para escrever suas próprias extensões C, dependendo do que você está tentando fazer. Ferramentas recomendadas de terceiros oferecem abordagens mais simples e mais sofisticadas para criar extensões C e C++ para Python.

Como posso executar instruções arbitrárias de Python a partir de C?

A função mais alto-nível para isso é a **`PyRun_SimpleString()`**, que recebe como único argumento uma string a ser executada no contexto do módulo `__main__` e retorna `0` para sucesso e `-1` quando uma exceção ocorrer (incluindo **`SyntaxError`**). Se quiser mais controle, use **`PyRun_String()`**; veja o código-fonte de **`PyRun_SimpleString()`** em `Python/pythonrun.c`.

Como posso executar e obter o resultado de uma expressão Python arbitrária a partir de C?

Chame a função `PyRun_String()` da pergunta anterior passando `Py_eval_input` como o símbolo de início; ela faz a análise sintática de uma expressão, a executa, e retorna o seu valor.

Como extraio valores em C a partir de um objeto Python?

Depende do tipo do objeto. Se for uma tupla, a `PyTuple_Size()` retorna o seu comprimento e a `PyTuple_GetItem()` retorna o item em um determinado índice. Listas têm funções similares, `PyList_Size()` e `PyList_GetItem()`.

Para bytes, a `PyBytes_Size()` retorna o comprimento e a `PyBytes_AsStringAndSize()` fornece um ponteiro para o seu valor e o seu comprimento. Note que objetos bytes em Python podem conter bytes nulos, de forma que a **`strlen()`** do C não deve ser usada.

Para testar o tipo de um objeto, primeiramente se certifique de que ele não é `NULL`, e então use `PyBytes_Check()`, `PyTuple_Check()`, `PyList_Check()`, etc.

Também existe uma API alto-nível para objetos Python fornecida pela chamada interface "abstrata" – leia `Include/abstract.h` para mais detalhes. Ela permite interagir com qualquer tipo de sequência Python usando chamadas como `PySequence_Length()`, `PySequence_GetItem()`, etc, além de vários outros protocolos úteis tais como números (`PyNumber_Index()` e outros) e mapeamentos nas APIs PyMapping.

Como posso utilizar Py_BuildValue() para criar uma tupla de comprimento arbitrário?

Não é possível. Use a função `PyTuple_Pack()` para isso.

Como eu chamo um método de um objeto a partir do C?

A função `PyObject_CallMethod()` pode ser usada para chamar um método arbitrário de um objeto. Os parâmetros são o objeto, o nome do método a ser chamado, uma string de formato como a usada em `Py_BuildValue()`, e os valores dos argumentos:

```
PyObject *
PyObject_CallMethod(PyObject *object, const char *method_name,
                    const char *arg_format, ...);
```

Isso funciona para qualquer objeto que tenha métodos – sejam eles embutidos ou definidos por usuário. Você fica então responsável por chamar `Py_DECREF()` no valor de retorno.

Para chamar, por exemplo, o método “seek” de um objeto arquivo com argumentos 10, 0 (presumindo que “f” é o ponteiro para o objeto arquivo):

```
res = PyObject_CallMethod(f, "seek", "(ii)", 10, 0);
if (res == NULL) {
        ... ocorreu uma exceção ...
}
else {
        Py_DECREF(res);
}
```

Note que a função `PyObject_CallObject()` *sempre* recebe os argumentos da chamada como uma tupla, de forma que para chamar uma função sem argumentos deve-se passar “()” como formato, e para chamar uma função com 1 argumento, coloque-o entre parênteses, por exemplo “(i)”.

Como posso capturar a saída da função PyErr_Print() (ou qualquer outra coisa que escreva para stdout/stderr)?

Com código Python, defina um objeto que suporte o método `write()`. Atribua esse objeto a `sys.stdout` e `sys.stderr`. Chame print_error, or simplesmente deixe o mecanismo padrão de traceback acontecer. Assim, a saída irá para onde quer que o seu método `write()` a envie.

O jeito mais fácil de fazer isso é usar a classe `io.StringIO`:

```
>>> import io, sys
>>> sys.stdout = io.StringIO()
>>> print('foo')
>>> print('hello world!')
>>> sys.stderr.write(sys.stdout.getvalue())
foo
hello world!
```

Um objeto personalizado para fazer a mesma coisa seria esse:

```
>>> import io, sys
>>> class StdoutCatcher(io.TextIOBase):
...     def __init__(self):
...         self.data = []
...     def write(self, stuff):
...         self.data.append(stuff)
...
>>> import sys
>>> sys.stdout = StdoutCatcher()
>>> print('foo')
>>> print('hello world!')
>>> sys.stderr.write(''.join(sys.stdout.data))
foo
hello world!
```

Como faço para acessar a partir do C um módulo escrito em Python?

Você pode obter um pointeiro para o objeto de módulo da seguinte maneira:

```
module = PyImport_ImportModule("<modulename>");
```

Se o módulo ainda não foi importado (isto é, ainda não aparece no `sys.modules`), essa função vai inicializar o módulo; caso contrário, ela vai simplesmente retornar o valor de `sys.modules["<modulename>"]`. Note que ela não adiciona o módulo a nenhum espaço de nomes – ela simplesmente garante que ele foi inicializado e colocado no **`sys.modules`**.

Você pode então acessar os atributos do módulo (isto é, qualquer nome definido no módulo) assim:

```
attr = PyObject_GetAttrString(module, "<attrname>");
```

Chamar `PyObject_SetAttrString()` para definir variáveis no módulo também funciona.

Como posso interagir com objetos C++ a partir do Python?

Dependendo das suas necessidades, há diversas abordagens. Para fazer isso manualmente, comece lendo o documento "Estendendo e Incorporando". Note que, para o sistema Python em tempo de execução, não há muita diferença entre C e C++ – de forma que a estratégia de construir um novo tipo Python ao redor de uma estrutura C (ou ponteiro para uma) também funciona para objetos C++.

Para bibliotecas C++, veja Escrever C é difícil; há alguma alternativa?.

Adicionei um módulo usando o arquivo de Setup e o make falha; por quê?

O Setup deve terminar com uma quebra de linha; se não houver uma quebra de linha no final, o processo de compilação falha. (Consertar isso requer umas gambiarras feias em shell script, e esse bug é tão pequeno que o esforço não parece valer a pena.)

Como eu depuro uma extensão?

Ao usar o GDB com extensões carregadas dinamicamente, você não consegue definir um ponto de interrupção antes da sua extensão ser carregada.

No seu arquivo `.gdbinit` (ou então interativamente), adicione o comando:

```
br _PyImport_LoadDynamicModule
```

Então, ao executar o GDB:

```
$ gdb /local/bin/python
gdb) run myscript.py
gdb) continue # repita até que a sua extensão seja carregada
gdb) finish   # para que a sua extensão seja carregada
gdb) br myfunction.c:50
gdb) continue
```

Quero compilar um módulo Python no meu sistema Linux, mas alguns arquivos estão faltando. Por quê?

A maioria das versões empacotadas de Python omitem alguns arquivos necessários para compilar extensões do Python.

Em sistemas Red Hat, instale o RPM python3-devel para obter os arquivos necessários.

Para Debian, execute `apt-get install python3-dev`.

Como posso distinguir “entrada incompleta” de “entrada inválida”?

Às vezes você quer emular o comportamento do interpretador interativo do Python, que te dá um prompt de continuação quando a entrada está incompleta (por exemplo, você digitou o início de uma instrução “if”, ou

então não fechou os parênteses ou aspas triplas), mas que te dá um mensagem de erro de sintaxe imediatamente se a entrada for inválida.

Em Python você pode usar o módulo `codeop`, que aproxima suficientemente o comportamento do analisador sintático. Por exemplo, o IDLE o usa.

Em C, a forma mais fácil de fazer isso é chamar `PyRun_InteractiveLoop()` (talvez em uma thread separada) e deixar o interpretador do Python tratar a entrada para você. Você também pode apontar o `PyOS_ReadlineFunctionPointer()` para a sua função de entrada personalizada. Consulte `Modules/readline.c` e `Parser/myreadline.c` para mais dicas.

Como encontro os símbolos __builtin_new ou __pure_virtual não-definidos no g++?

Para carregar dinamicamente módulos de extensão feitos com g++, você precisa recompilar o Python, usando o g++ como ligador (mude a constante LINKCC no Makefile dos módulos de extensão do Python), e use o g++ também como ligador do seu módulo (por exemplo, `g++ -shared -o mymodule.so mymodule.o`).

Posso criar uma classe de objetos com alguns métodos implementados em C e outros em Python (por exemplo, via herança)?

Sim, você pode herdar de classes embutidas como `int`, `list`, `dict` etc.

A Boost Python Library (BPL, https://www.boost.org/libs/python/doc/index.html) fornece uma forma de fazer isso a partir do C++ (quer dizer, você consegue herdar de uma classe de extensão escrita em C++ usando a BPL).

Python no Windows

Sumário

- Python no Windows
  - Como faço para executar um programa Python no Windows?
  - Como eu faço para criar programas Python executáveis?
  - Por que Python às vezes demora tanto para iniciar?
  - Como eu faço para criar um executável a partir de um código Python?
  - Um arquivo `*.pyd` é o mesmo que um DLL?
  - Como eu posso embutir Python dentro de uma aplicação do Windows?
  - Como eu impeço editores de adicionarem tabulações na minha source do Python?
  - Como faço para verificar uma tecla pressionada sem bloquear?
  - Como resolvo o erro da api-ms-win-crt-runtime-l1-1-0.dll ausente?

Como faço para executar um programa Python no Windows?

Esta não é necessariamente uma questão direta. Se você já está familiarizado com a execução de programas através das linha de comando do Windows, então tudo parecerá óbvio; caso contrário, poderá precisar de um pouco mais de orientação.

A menos que você use algum tipo de ambiente de desenvolvimento integrado, você vai acabar digitando os comandos do Windows no que é chamado “janela do DOS” ou “janela do prompt de comando”. Geralmente você pode abrir essas janelas procurando na barra de pesquisa por `cmd`. Você

deverá reconhecer quando iniciar porque você verá um “Prompt de Comando do Windows”, que geralmente tem esta forma:

```
C:\>
```

A letra pode ser diferente, e pode haver outras coisas depois, então você facilmente pode ver algo como:

```
D:\SeuNome\Projetos\Python>
```

dependendo de como seu computador foi configurado e o que mais você tem feito com ele recentemente. Uma vez que você tenha iniciado a janela, você estará no caminho para executar os seus programas Python.

Você deve observar que seu código Python deve ser processado por outro programa chamado interpretador. O interpretador lê o seu código, compila em bytecodes, e depois executa os bytecodes para rodar o seu programa. Então, como você pode organizar o interpretador para lidar com seu Python?

Primeiro, você precisa ter certeza de que sua janela de comando reconhece a palavra “py” como uma instrução para iniciar o interpretador. Se você abriu a janela de comando, você deve tentar digitar o comando “py” e o observar o retorno:

```
C:\Users\SeuNome> py
```

Você deve então ver algo como:

```
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900
Type "help", "copyright", "credits" or "license" for more inform
>>>
```

Você iniciou o interpretador no “modo interativo”. Que significa que você pode inserir instruções ou expressões Python interativamente e executá-las ou calculá-las enquanto espera. Esta é uma das características mais fortes do Python. Verifique isso digitando algumas instruções de sua escolha e veja os resultados:

```
>>> print("Olá")
Olá
>>> "Olá" * 3
'OláOláOlá'
```

Muitas pessoas usam o modo interativo como uma calculadora conveniente, mas altamente programável. Quando quiser encerrar sua sessão interativa do Python, chame a função `exit()` ou mantenha pressionada a tecla `Ctrl` enquanto você digita a `Z` e pressione a tecla "`Enter`" para voltar ao prompt de comando do Windows.

Você também pode descobrir que você tem um item no Menu Iniciar como *Iniciar ▸ Programas ▸ Python 3.x ▸ Python (linha de comando)* que resultará em você vendo o prompt `>>>` em uma nova janela. Se acontecer isso, a janela desaparecerá depois que você chamar a função `exit()` ou inserir o caractere `Ctrl-Z`; o Windows está executando um único comando "python" na janela, e fecha quando você termina o interpretador.

Agora que sabemos que o comando `py` é reconhecido, você pode dar seu script Python para ele. Você terá que dar um caminho absoluto ou relativo para o script Python. Vamos dizer que seu script Python está localizado na sua área de trabalho e se chama `oi.py`, e seu prompt de comando está aberto no seu diretório raiz de forma que você está vendo algo similar a:

```
C:\Users\SeuNome>
```

Então, agora você solicitará o comando `py` para fornecer seu script para Python, digitando `py` seguido pelo seu caminho de script:

```
C:\Users\SeuNome> py Desktop\oi.py
oi
```

Como eu faço para criar programas Python executáveis?

No Windows, o instalador padrão do Python já associa a extensão .py com o tipo de arquivo (Python.File) e dá àquele tipo de arquivo um comando aberto que executa o interpretador (`D:\Program`

`Files\Python\python.exe "%1" %*`). Isso é o bastante para fazer scripts executáveis pelo prompt de comando como ‘foo.py’. Se você preferir executar o script simplesmente digitando ‘foo’ sem extensão você precisa adicionar .py à variável de ambiente PATHEXT.

Por que Python às vezes demora tanto para iniciar?

Geralmente, Python inicia muito rapidamente no Windows, mas ocasionalmente há relatos de erros que, de repente, o Python começa a demorar muito tempo para iniciar. Isso é ainda mais intrigante, porque Python funciona bem em outros sistemas Windows que parecem estar configurados de forma idêntica.

O problema pode ser causado por uma desconfiguração de software antivírus na máquina problemática. Alguns antivírus são conhecidos por introduzir sobrecarga de duas ordens de magnitude no início quando estão configurados para monitorar todas as leituras do sistema de arquivos. Tente verificar a configuração do antivírus nos seus sistemas para assegurar que eles estão de fato configurados identicamente. O McAfee, quando configurado para escanear todo a atividade do sistema de arquivos, é um ofensor conhecido.

Como eu faço para criar um executável a partir de um código Python?

Consulte Como posso criar um binário independente a partir de um script Python? para uma lista de ferramentas que podem ser usada para criar executáveis.

Um arquivo `*.pyd` é o mesmo que um DLL?

Sim, os arquivos .pyd são dll, mas existem algumas diferenças. Se você possui uma DLL chamada `foo.pyd`, ela deve ter a função `PyInit_foo()`. Você pode escrever “import foo” do Python, e o Python procurará por

foo.pyd (assim como foo.py, foo.pyc) e, se o encontrar, tentará chamar `PyInit_foo()` para inicializá-lo. Você não vincula seu arquivo .exe ao arquivo foo.lib, pois isso faria com que o Windows exigisse a presença da DLL.

Observe que o caminho de pesquisa para foo.pyd é PYTHONPATH, não o mesmo que o Windows usa para procurar por foo.dll. Além disso, foo.pyd não precisa estar presente para executar seu programa, enquanto que se você vinculou seu programa a uma dll, a dll será necessária. Obviamente, o foo.pyd é necessário se você quiser dizer `import foo`. Em uma DLL, o vínculo é declarado no código-fonte com `__declspec(dllexport)`. Em um .pyd, o vínculo é definido em uma lista de funções disponíveis.

Como eu posso embutir Python dentro de uma aplicação do Windows?

A incorporação do interpretador Python em um aplicativo do Windows pode ser resumida da seguinte forma:

1. **Não** compile o Python diretamente em seu arquivo .exe. No Windows, o Python deve ser uma DLL para manipular os módulos de importação que são eles próprios. (Este é o primeiro fato chave não documentado.) Em vez disso, vincule a `python`*`NN`*`.dll`; normalmente é instalado em `C:\Windows\System`. *NN* é a versão do Python, um número como “33” para o Python 3.3.

   Você pode vincular ao Python de duas maneiras diferentes. A vinculação em tempo de carregamento significa vincular contra `python`*`NN`*`.lib`, enquanto a vinculação em tempo de execução significa vincular a `python`*`NN`*`.dll`. (Nota geral: `python`*`NN`*`.lib` é a chamada “import lib” correspondente a `python`*`NN`*`.dll`. Apenas define símbolos para o ligador.)

   A vinculação em tempo de execução simplifica bastante as opções de vinculação; tudo acontece em tempo de execução. Seu código deve carregar `python`*`NN`*`.dll` usando a rotina `LoadLibraryEx()` do Windows. O código também deve usar rotinas de acesso e dados em

`python*NN*.dll` (ou seja, as APIs C do Python) usando ponteiros obtidos pela rotina `GetProcAddress()` do Windows. As macros podem tornar o uso desses ponteiros transparente para qualquer código C que chama rotinas na API C do Python.

2. Se você usa SWIG, é fácil criar um “módulo de extensão” do Python que disponibilizará os dados e os métodos da aplicação para o Python. O SWIG cuidará de todos os detalhes obscuros para você. O resultado é o código C que você vincula *ao* arquivo.exe (!) Você **não** precisa criar um arquivo DLL, o que também simplifica a vinculação.

3. O SWIG criará uma função init (uma função C) cujo nome depende do nome do módulo de extensão. Por exemplo, se o nome do módulo for leo, a função init será chamada initleo(). Se você usa classes de sombra SWIG, como deveria, a função init será chamada initleoc(). Isso inicializa uma classe auxiliar principalmente oculta usada pela classe shadow.

   O motivo pelo qual você pode vincular o código C na etapa 2 ao seu arquivo .exe é que chamar a função de inicialização equivale a importar o módulo para o Python! (Este é o segundo fato chave não documentado.)

4. Em suma, você pode utilizar o código a seguir para inicializar o interpretador Python com seu módulo de extensão.

```
#include <Python.h>
...
Py_Initialize();  // Inicializa Python.
initmyAppc();  // Inicializa (importa) a classe auxiliar.
PyRun_SimpleString("import myApp");  // Importa a classe sor
```

5. Existem dois problemas com a API C do Python que se tornarão aparentes se você utiliza um compilador que não seja o MSVC, o compilador utilizado no pythonNN.dll.

   Problema 1: As chamadas funções de “Nível Muito Alto” que recebem argumentos `FILE *` não funcionarão em um ambiente com vários

compiladores porque a noção de cada `struct FILE` de um compilador será diferente. Do ponto de vista da implementação, essas são funções de nível muito baixo.

Problema 2: SWIG gera o seguinte código ao gerar invólucros para funções sem retorno:

```
Py_INCREF(Py_None);
_resultobj = Py_None;
return _resultobj;
```

Infelizmente, Py_None é uma macro que se expande para uma referência a uma estrutura de dados complexa chamada _Py_NoneStruct dentro de pythonNN.dll. Novamente, esse código falhará em um ambiente com vários compiladores. Substitua esse código por:

```
return Py_BuildValue("");
```

Pode ser possível usar o comando `%typemap` do SWIG para fazer a alteração automaticamente, embora eu não tenha conseguido fazer isso funcionar (eu sou um completo novato em SWIG).

6. Usar um script de shell do Python para criar uma janela do interpretador Python de dentro da aplicação do Windows não é uma boa ideia; a janela resultante será independente do sistema de janelas da sua aplicação. Em vez disso, você (ou a classe wxPythonWindow) deve criar uma janela “nativa” do interpretador. É fácil conectar essa janela ao interpretador Python. Você pode redirecionar a E/S do Python para qualquer objeto que suporte leitura e gravação; portanto, tudo que você precisa é de um objeto Python (definido no seu módulo de extensão) que contenha métodos read() e write().

Como eu impeço editores de adicionarem tabulações na minha source do Python?

As perguntas frequentes não recomendam a utilização de tabulações, e o guia de estilo Python, :pep:*8*, recomenda 4 espaços para código de Python distribuído; esse também é o padrão do python-mode do Emacs.

Sob qualquer editor, misturar tabulações e espaços é uma má ideia. O MSVC não é diferente nesse aspecto e é facilmente configurado para usar espaços: Selecione *Tools* ‣ *Options* ‣ *Tabs* e, para o tipo de arquivo "Default", defina "Tab size" e "Indent size" para 4 e selecione o botão de opção "Insert spaces".

O Python levanta `IndentationError` ou `TabError` se tabulações e espaços misturados estiverem causando problemas no espaço em branco à esquerda. Você também pode executar o módulo `tabnanny` para verificar uma árvore de diretórios no modo em lote.

Como faço para verificar uma tecla pressionada sem bloquear?

Use o módulo `msvcrt`. Este é um módulo de extensão padrão específico do Windows. Ele define uma função `kbhit()` que verifica se um toque no teclado está presente, e `getch()` que recebe um caractere sem ecoá-lo.

Como resolvo o erro da api-ms-win-crt-runtime-l1-1-0.dll ausente?

Isso pode ocorrer no Python 3.5 e posterior ao usar o Windows 8.1 ou anterior sem que todas as atualizações tenham sido instaladas. Primeiro, certifique-se de que seu sistema operacional seja compatível e esteja atualizado e, se isso não resolver o problema, visite a página de suporte da Microsoft [https://support.microsoft.com/pt-br/help/3118401/] para obter orientação sobre como instalar manualmente a atualização do C Runtime.

FAQ da Interface Gráfica do Usuário

Sumário

- FAQ da Interface Gráfica do Usuário
  - Perguntas Gerais sobre a GUI
  - Quais toolkits de GUI existem para o Python?
  - Perguntas do Tkinter
    - Como eu congelo as aplicações Tkinter?
    - Posso ter eventos Tk manipulados enquanto aguardo pelo E/S?
    - Não consigo fazer as ligações de tecla funcionarem no Tkinter: por que?

Perguntas Gerais sobre a GUI

Quais toolkits de GUI existem para o Python?

As versões padrão do Python incluem uma interface orientada a objetos para o conjunto de widgets Tcl/Tk, chamado tkinter. Este é provavelmente o mais fácil de instalar (uma vez que vem incluído na maioria das distribuições binárias [https://www.python.org/downloads/] do Python) e usar. Para obter mais informações sobre o Tk, incluindo ponteiros para a fonte, consulte a página inicial do Tcl/Tk [https://www.tcl.tk]. Tcl/Tk é totalmente portátil para as plataformas macOS, Windows e Unix.

Dependendo da(s) plataforma(s) que você está visando, também existem várias alternativas. Uma lista de frameworks GUI de plataformas cruzadas [https://wiki.python.org/moin/GuiProgramming#Cross-Platform_Frameworks] e frameworks

GUI específicas de plataforma [https://wiki.python.org/moin/GuiProgramming#Platform-specific_Frameworks] podem ser encontradas na wiki do python.

Perguntas do Tkinter

Como eu congelo as aplicações Tkinter?

Freeze é uma ferramenta para criar aplicativos autônomos. Ao congelar aplicativos Tkinter, os aplicativos não serão verdadeiramente autônomos, pois o aplicativo ainda precisará das bibliotecas Tcl e Tk.

Uma solução é enviar a aplicação com as bibliotecas Tcl e Tk e apontá-las em tempo de execução usando as variáveis de ambiente **`TCL_LIBRARY`** e **`TK_LIBRARY`**.

Várias bibliotecas de congelamento de terceiros, como py2exe e cx_Freeze, possuem manipulação embutida para aplicações Tkinter.

Posso ter eventos Tk manipulados enquanto aguardo pelo E/S?

Em plataformas diferentes do Windows, sim, e você nem precisa de threads! Mas você terá que reestruturar seu código de E/S um pouco. O Tk tem o equivalente à chamada **`XtAddInput()`** do Xt, que permite que você registre uma função de retorno de chamada que será chamada a partir do laço de repetição principal do Tk quando E/S é possível em um descritor de arquivo. Consulte File handlers.

Não consigo fazer as ligações de tecla funcionarem no Tkinter: por que?

Uma queixa frequentemente ouvida é que os manipuladores de eventos vinculados a eventos com o método **`bind()`** não são manipulados mesmo quando a tecla apropriada é pressionada.

A causa mais comum é que o widget para o qual a ligação se aplica não possui “foco no teclado”. Confira a documentação do Tk para o comando

de foco. Normalmente, um widget é dado o foco do teclado clicando nele (mas não para rótulos, veja a opção takefocus).

FAD de “Por que o Python está instalado em meu computador?”

O que é Python?

Python é uma linguagem de programação. É usada para muitas e diversas aplicações. É usada em escolas e faculdades como uma linguagem de programação introdutória, porque Python é fácil de aprender, mas também é usada por desenvolvedores profissionais de software em lugares como Google, Nasa e Lucasfilm Ltd.

Se você quiser aprender mais sobre Python, comece com o Beginner’s Guide to Python [https://wiki.python.org/moin/BeginnersGuide].

Porque Python está instalado em minha máquina?

Se você encontrar Python instalado em seu sistema, mas não se lembra de tê-lo instalado, então podem haver muitas maneiras diferentes dele ter ido parar lá

- Possivelmente outro usuário do computador pretendia aprender programação e o instalou; você terá que descobrir quem estava usando a máquina e pode o ter instalado.
- Um aplicativo de terceiros pode ter sido instalado na máquina e sido escrito em Python e incluído uma instalação do Python. Há muitos desses aplicativos, desde programas com interface gráfica até servidores de rede e scripts administrativos.
- Algumas máquinas Windows já possuem o Python instalado. No presente momento nós temos conhecimento de computadores da Hewlett-Packard e da Compaq que incluem Python. Aparentemente algumas das ferramentas administrativas da HP/Compaq são escritas em Python.

- Muitos sistemas operacionais derivados do Unix, como macOS e algumas distribuições Linux, possuem o python instalado por padrão; está incluído na instalação base.

Eu posso apagar o Python?

Isso depende de como o Python veio.

Se alguém o instalou deliberadamente, você pode removê-lo sem machucar ninguém. No Windows use o Adicionar ou remover programas que se encontra no Painel de Controle.

Se o Python foi instalado por um aplicativo de terceiros, você pode removê-lo mas aquela aplicação não irá mais funcionar. Você deve usar o desinstalador daquele aplicativo para remover o Python diretamente.

Se o Python veio junto com seu sistema operacional, removê-lo não é recomendado. Se você o remover, qualquer ferramenta que tenha sido escrita em Python não vão mais funcionar, e algumas delas podem ser importantes para você. A reinstalação do sistema inteiro seria necessária para consertar as coisas de novo.

Descontinuações

Remoção pendente no Python 3.16

- O sistema de importação:
  - A definição de `__loader__` em um módulo enquanto falha na definição de `__spec__.loader` está descontinuado. No Python 3.16, `__loader__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.
- `array`:
  - O código de formato `'u'` (`wchar_t`) foi descontinuado na documentação desde o Python 3.3 e em tempo de execução desde o Python 3.13. Use o código de formato `'w'` (`Py_UCS4`) para caracteres Unicode.
- `asyncio`:
  - `asyncio.iscoroutinefunction()` foi descontinuado e será removido no Python 3.16, use `inspect.iscoroutinefunction()` em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)
  - O sistema de políticas `asyncio` está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:
    - `asyncio.AbstractEventLoopPolicy`
    - `asyncio.DefaultEventLoopPolicy`
    - `asyncio.WindowsSelectorEventLoopPolicy`
    - `asyncio.WindowsProactorEventLoopPolicy`
    - `asyncio.get_event_loop_policy()`

- `asyncio.set_event_loop_policy()`

Os usuários devem usar `asyncio.run()` ou `asyncio.Runner` com *loop_factory* para usar a implementação de loop de eventos desejada.

por exemplo, para usar `asyncio.SelectorEventLoop` no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventL
```

(Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- `builtins`:
  - A inversão bit a bit em tipos booleanos, `~True` ou `~False` foi descontinuada desde o Python 3.12, pois produz resultados surpreendentes e não intuitivos (`-2` e `-1`). Use `not x` em vez disso para a negação lógica de um Booleano. No caso raro de você precisar da inversão bit a bit do inteiro subjacente, converta para `int` explicitamente (`~int(x)`).
- `functools`:
  - A chamada da implementação Python de `functools.reduce()` com *function* ou *sequence* como argumentos nomeados foi descontinuada desde o Python 3.14.
- `logging`:
  - O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores

com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)

- **mimetypes**:
  - Extensões válidas começam com um '.' ou estão vazias para **mimetypes.MimeTypes.add_type()**. Extensões sem ponto estão descontinuadas e levantarão uma exceção **ValueError** no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)
- **shutil**:
  - A exceção **ExecError** foi descontinuada desde o Python 3.14. Ela não foi usada por nenhuma função em **shutil** desde o Python 3.4, e agora é um alias de **RuntimeError**.
- **symtable**:
  - O método **Class.get_methods** foi descontinuado desde o Python 3.14.
- **sys**:
  - A função **_enablelegacywindowsfsencoding()** foi descontinuada desde o Python 3.13. Use a variável de ambiente **PYTHONLEGACYWINDOWSFSENCODING**.
- **sysconfig**:
  - A função **sysconfig.expand_makefile_vars()** está descontinuada desde o Python 3.14. Em vez disso, use o argumento `vars` de **sysconfig.get_paths()**.
- **tarfile**:
  - O atributo não documentado e não utilizado **TarFile.tarfile** foi descontinuado desde o Python 3.13.

Remoção pendente no Python 3.17

- **`datetime`**:
  - **`strptime()`** chama usando uma string de formato contendo `%e` (dia do mês) sem um ano. Isso está descontinuado desde Python 3.15. (Contribuição de Stan Ulbrych em gh-70647 [https://github.com/python/cpython/issues/70647].)

- **`collections.abc`**:
  - **`collections.abc.ByteString`** está agendado para remoção no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`encodings`**:
  - Passar nomes de *codificação* não ASCII para **`encodings.normalize_encoding()`** está descontinuado e sua

remoção está prevista para o Python 3.17. (Contribuição de Stan Ulbrych em gh-136702 [https://github.com/python/cpython/issues/136702].)

- **`profile`**:
  - O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)
- **`webbrowser`**:
  - **`webbrowser.MacOSXOSAScript`** está descontinuado em favor de **`webbrowser.MacOS`**. (gh-137586 [https://github.com/python/cpython/issues/137586])
- **`typing`**:
  - Antes do Python 3.14, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para compatibilidade com versões anteriores, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como **`typing.get_origin()`** e **`typing.get_args()`**, em vez de depender de detalhes de implementação privada.
  - **`typing.ByteString`**, obsoleto desde o Python 3.9, está programado para ser removido no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

`ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estáticos).

Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`tkinter`**:
  - Os métodos **`trace_variable()`**, **`trace()`** (um apelido de **`trace_variable()`**), **`trace_vdelete()`** e **`trace_vinfo()`** da classe **`tkinter.Variable`**, descontinuados desde o Python 3.14, têm remoção prevista para o Python 3.17. Em vez deles, utilize **`trace_add()`**, **`trace_remove()`** e **`trace_info()`**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

Remoção pendente no Python 3.18

- Não aceita mais um valor booleano quando um descritor de arquivo é esperado. (Contribuição de Serhiy Storchaka em gh-82626 [https://github.com/python/cpython/issues/82626].)

- **`decimal`**:
  - O especificador de formato de **`Decimal`** não padrão e não documentado `'N'`, que só é suportado na implementação C do módulo **`decimal`**, foi descontinuada desde o Python 3.13. (Contribuição de Serhiy Storchaka em gh-89902 [https://github.com/python/cpython/issues/89902].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:
    - Linhas de `import` em arquivos *`name`*`.pth` são silenciosamente ignoradas.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

Remoção pendente no Python 3.19

- **`ctypes`**:
    - Troca implícita para o layout de estrutura compatível com MSVC definindo **`_pack_`**, mas não **`_layout_`** em plataformas não Windows.

- **`hashlib`**:
    - Em construtores de funções de hash, como **`new()`**, ou nos construtores nomeados diretamente pelo algoritmo de hash, como **`md5()`** e **`sha256()`**, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado `data=` ou `string=` em várias implementações do **`hashlib`**.

      O suporte ao nome do argumento nomeado `string` está agora descontinuado e previsto para ser removido no Python 3.19.

      Antes do Python 3.13, o parâmetro nomeado `string` não era suportado corretamente, dependendo da implementação de funções de hash do backend. Dê preferência a passar os dados iniciais como um argumento posicional para garantir a máxima compatibilidade com versões anteriores.

- **`http.cookies`**:
    - **`http.cookies.Morsel.js_output()`** está descontinuado e será removido no Python 3.19.

    - `http.cookies.BaseCookie.js_output()` está descontinuado e será removido no Python 3.19.

- `imaplib`:

    - A alteração de `IMAP4.file` agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade agora não é utilizada e alterar seu valor não fecha automaticamente o arquivo atual.

      Antes do Python 3.14, essa propriedade era usada para implementar os métodos `read()` e `readline()` correspondentes para `IMAP4`, mas isso não ocorre mais desde então.

Remoção pendente no Python 3.20

- Chamar o método `__new__()` de `struct.Struct` sem o argumento *format* está descontinuado e será removido no Python 3.20. Chamar o método `__init__()` em objetos `Struct` já inicializados está descontinuado e será removido no Python 3.20.

  (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize `sys.version_info` em seu lugar.

    - `argparse`
    - `csv`
    - `ctypes`
    - `ctypes.macholib`
    - `decimal` (use `decimal.SPEC_VERSION`)
    - `http.server`
    - `imaplib`
    - `ipaddress`
    - `json`

- **logging** (`__date__` também descontinuado)
- **optparse**
- **pickle**
- **platform**
- **re**
- **socketserver**
- **tabnanny**
- **tarfile**
- **tkinter.font**
- **tkinter.ttk**
- **wsgiref.simple_server**
- **xml.etree.ElementTree**
- **xml.sax.expatreader**
- **xml.sax.handler**
- **zlib**

(Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

  - Avisos são produzidos para linhas `import` encontradas em arquivos *`name`*`.pth`.
  - Arquivos *`name`*`.pth` não são mais decodificados usando a codificação da localidade (*locale encoding*) por padrão. Eles **DEVEM** estar codificados em `utf-8-sig`.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

- **ast**:

  - A criação de instâncias de nós abstratos da AST (como **ast.AST** ou **ast.expr**) está descontinuada e levantará um erro no Python 3.20.

- **typing**:

- A verificação com `isinstance()` e `issubclass()` está descontinuado em classes de protocolo que não foram explicitamente decoradas com **`runtime_checkable()`**, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso levantará em um `TypeError` no Python 3.20.

  (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

Remoção pendente em versões futuras

As APIs a seguir serão removidas no futuro, embora atualmente não haja uma data agendada para sua remoção.

- `argparse`:
  - O aninhamento de grupos de argumentos e o aninhamento de grupos mutuamente exclusivos estão descontinuado.
  - A passagem do argumento nomeado não documentado *prefix_chars* para `add_argument_group()` agora está descontinuado.
  - O conversor de tipo `argparse.FileType` está descontinuado.
- `builtins`:
  - Geradores: a assinatura `throw(type, exc, tb)` e `athrow(type, exc, tb)` está descontinuada: use `throw(exc)` e `athrow(exc)`, a assinatura do argumento único.
  - Atualmente Python aceita literais numéricos imediatamente seguidos por palavras reservadas como, por exemplo, `0in x`, `1or x`, `0if 1else 2`. Ele permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). Um aviso de sintaxe é levantado se o literal numérico for imediatamente seguido por uma das palavras reservadas `and`, `else`, `for`, `if`, `in` , `is` e `or`. Em uma versão futura, será alterado para um erro de sintaxe. (gh-87999 [https://github.com/python/cpython/issues/87999])
  - Suporte para métodos `__index__()` e `__int__()` retornando tipo não-int: esses métodos serão necessários para retornar uma instância de uma subclasse estrita de `int`.

    - Suporte para o método `__float__()` retornando uma subclasse estrita de **`float`**: esses métodos serão necessários para retornar uma instância de **`float`**.
    - Suporte para o método `__complex__()` retornando uma subclasse estrita de **`complex`**: esses métodos serão necessários para retornar uma instância de **`complex`**.
    - Passar um número complexo como argumento *real* ou *imag* no construtor **`complex()`** agora está descontinuado; deve ser passado apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)
- **`calendar`**: as constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por **`calendar.JANUARY`** e **`calendar.FEBRUARY`**. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)
- **`codecs`**: use **`open()`** em vez de **`codecs.open()`**. (gh-133038 [https://github.com/python/cpython/issues/133038])
- **`codeobject.co_lnotab`**: use o método **`codeobject.co_lines()`**.
- **`datetime`**:
    - **`utcnow()`**: use `datetime.datetime.now(tz=datetime.UTC)`.
    - **`utcfromtimestamp()`**: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- **`gettext`**: o valor de plural deve ser um número inteiro.
- **`importlib`**:
    - O parâmetro *debug_override* de **`cache_from_source()`** foi descontinuado: em vez disso, use o parâmetro *optimization*.
- **`importlib.metadata`**:
    - Interface de tupla `EntryPoints`.
    - `None` implícito nos valores de retorno.
- **`logging`**: o método `warn()` foi descontinuado desde o Python 3.3, use **`warning()`**.
- **`mailbox`**: o uso da entrada StringIO e do modo de texto foi descontinuado; em vez disso, use BytesIO e o modo binário.
- **`os`**: chame **`os.register_at_fork()`** em um processo multithread.
- **`os.path`**: **`os.path.commonprefix()`** está descontinuada; use **`os.path.commonpath()`** para prefixos de caminho. A função

os.path.commonprefix() está sendo descontinuada por possuir nome e módulo enganosos. O uso dessa função não é seguro para prefixos de caminho, apesar de ela estar incluída em um módulo voltado à manipulação de caminhos; isso significa que é fácil introduzir acidentalmente vulnerabilidades de *path traversal* (travessia de diretórios) em programas Python ao utilizá-la.

- **pydoc.ErrorDuringImport**: um valor de tupla para o parâmetro *exc_info* foi descontinuado, use uma instância de exceção.
- re: regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)
- shutil: o parâmetro *onerror* de rmtree() foi descontinuado no Python 3.12; use o parâmetro *onexc*.
- Protocolos e opções de ssl
  - ssl.SSLContext sem argumento de protocolo foi descontinuado.
  - ssl.SSLContext: set_npn_protocols() e **selected_npn_protocol()** foram descontinuados: use ALPN.
  - Opções de ssl.OP_NO_SSL*
  - Opções de ssl.OP_NO_TLS*
  - ssl.PROTOCOL_SSLv3
  - ssl.PROTOCOL_TLS
  - ssl.PROTOCOL_TLSv1
  - ssl.PROTOCOL_TLSv1_1
  - ssl.PROTOCOL_TLSv1_2
  - ssl.TLSVersion.SSLv3
  - ssl.TLSVersion.TLSv1
  - ssl.TLSVersion.TLSv1_1
- Métodos de threading:
  - **threading.Condition.notifyAll()**: use notify_all().
  - **threading.Event.isSet()**: use is_set().
  - **threading.Thread.isDaemon()**, threading.Thread.setDaemon(): use o atributo threading.Thread.daemon.

    - **threading.Thread.getName()**, threading.Thread.setName(): use o atributo threading.Thread.name.
    - **threading.currentThread()**: use threading.current_thread().
    - **threading.activeCount()**: use threading.active_count().
- typing.Text (gh-92332 [https://github.com/python/cpython/issues/92332]).
- A classe interna `typing._UnionGenericAlias` não é mais usada para implementar typing.Union. Para preservar a compatibilidade com usuários que utilizam esta classe privada, uma correção de compatibilidade será fornecida pelo menos até a versão 3.17 do Python. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)
- unittest.IsolatedAsyncioTestCase: foi descontinuado retornar um valor que não seja `None` de um caso de teste.
- Funções descontinuadas de urllib.parse: use urlparse()
    - `splitattr()`
    - `splithost()`
    - `splitnport()`
    - `splitpasswd()`
    - `splitport()`
    - `splitquery()`
    - `splittag()`
    - `splittype()`
    - `splituser()`
    - `splitvalue()`
    - `to_bytes()`
- wsgiref: `SimpleHandler.stdout.write()` não deve fazer gravações parciais.
- xml.etree.ElementTree: testar o valor verdade de um Element está descontinuado. Em um lançamento futuro isso sempre retornará `True`. Em vez disso, prefira os testes explícitos `len(elem)` ou `elem is not None`.
- sys._clear_type_cache() está descontinuada: use sys._clear_internal_caches().

descontinuações suave

Não há planos para remover APIs suavemente descontinuadas

- **`re.match()`** e **`re.Pattern.match()`** estão agora suavemente descontinuados em favor das novas APIs **`re.prefixmatch()`** e **`re.Pattern.prefixmatch()`**, adicionadas como nomes alternativos e mais explícitos. Elas visam reduzir a confusão sobre o significado de *match*, seguindo o princípio do Zen do Python: *"Explícito é melhor que implícito"*. A maioria das bibliotecas de expressões regulares de outras linguagens utiliza uma API chamada *match* para a funcionalidade que o Python sempre chamou de *search*.

  Não planejamos remover o nome mais antigo **`match()`**, pois ele tem sido utilizado no código há mais de 30 anos. Códigos que oferecem suporte a versões mais antigas do Python devem continuar utilizando **`match()`**, enquanto novos códigos devem dar preferência a **`prefixmatch()`**. Veja prefixmatch() vs. match().

  (Contribuição de Gregory P. Smith em gh-86519 [https://github.com/python/cpython/issues/86519] e Hugo van Kemenade em gh-148100 [https://github.com/python/cpython/issues/148100].)

- O uso dos códigos de tipo de formato `'F'` e `'D'` do módulo **`struct`** está agora suavemente descontinuado em favor das formas de duas letras `'Zf'` e `'Zd'`. (Contribuição de Sergey B Kirpichev em gh-121249 [https://github.com/python/cpython/issues/121249].)

Descontinuações na API C

Remoção pendente no Python 3.15

- A **`PyImport_ImportModuleNoBlock()`**: Use **`PyImport_ImportModule()`**

- **`PyWeakref_GetObject()`** e **`PyWeakref_GET_OBJECT()`**: Use **`PyWeakref_GetRef()`**. O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pod ser usado para obter **`PyWeakref_GetRef()`** no Python 3.12 e versões mais antigas.

- **`PyUnicode_AsDecodedObject()`**: use **`PyCodec_Decode()`**.

- **`PyUnicode_AsDecodedUnicode()`**: Use **`PyCodec_Decode()`**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **`str`**, tal como **`bytes`**.

- **`PyUnicode_AsEncodedObject()`**: use **`PyCodec_Encode()`**.

- **`PyUnicode_AsEncodedUnicode()`**: Use **`PyCodec_Encode()`**; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de **`bytes`**, tal com **`str`**.

- Funções de inicialização do Python, descontinuadas no Python 3.13:

    - **`Py_GetPath()`**: Use **`PyConfig_Get("module_search_paths")`** (**`sys.path`**).
    - **`Py_GetPrefix()`**: Use **`PyConfig_Get("base_prefix")`** (**`sys.base_prefix`**). Use **`PyConfig_Get("prefix")`** (**`sys.prefix`**) se ambientes virtuais precisarem ser tratados.
    - **`Py_GetExecPrefix()`**: Use **`PyConfig_Get("base_exec_prefix")`** (**`sys.base_exec_prefix`**). Use **`PyConfig_Get("exec_prefix")`** (**`sys.exec_prefix`**) se ambientes virtuais precisarem ser tratados.
    - **`Py_GetProgramFullPath()`**: Use **`PyConfig_Get("executable")`** (**`sys.executable`**).
    - **`Py_GetProgramName()`**: Use **`PyConfig_Get("executable")`** (**`sys.executable`**).
    - **`Py_GetPythonHome()`**: Use **`PyConfig_Get("home")`** ou a variável de ambiente **`PYTHONHOME`**.

  O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter **`PyConfig_Get()`** no Python 3.13 e versões anteriores.

- Funções para configurar a inicialização do Python, descontinuadas no Python 3.11:

    - **`PySys_SetArgvEx()`**: defina **`PyConfig.argv`**.
    - **`PySys_SetArgv()`**: defina **`PyConfig.argv`**.

- **`Py_SetProgramName()`**: defina `PyConfig.program_name`.
- **`Py_SetPythonHome()`**: defina `PyConfig.home`.
- **`PySys_ResetWarnOptions()`**: Limpe `sys.warnoptions` e **`warnings.filters`**.

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig`.

- Variáveis de configuração globais

  - `Py_DebugFlag`: Use `PyConfig.parser_debug` ou `PyConfig_Get("parser_debug")`.
  - `Py_VerboseFlag`: Use `PyConfig.verbose` ou `PyConfig_Get("verbose")`.
  - `Py_QuietFlag`: Use `PyConfig.quiet` ou `PyConfig_Get("quiet")`.
  - `Py_InteractiveFlag`: Use `PyConfig.interactive` ou `PyConfig_Get("interactive")`.
  - `Py_InspectFlag`: Use `PyConfig.inspect` ou `PyConfig_Get("inspect")`.
  - `Py_OptimizeFlag`: Use `PyConfig.optimization_level` ou `PyConfig_Get("optimization_level")`.
  - `Py_NoSiteFlag`: Use `PyConfig.site_import` ou `PyConfig_Get("site_import")`.
  - `Py_BytesWarningFlag`: Use `PyConfig.bytes_warning` ou `PyConfig_Get("bytes_warning")`.
  - `Py_FrozenFlag`: Use `PyConfig.pathconfig_warnings` ou `PyConfig_Get("pathconfig_warnings")`.
  - `Py_IgnoreEnvironmentFlag`: Use `PyConfig.use_environment` ou `PyConfig_Get("use_environment")`.
  - `Py_DontWriteBytecodeFlag`: Use `PyConfig.write_bytecode` ou `PyConfig_Get("write_bytecode")`.
  - `Py_NoUserSiteDirectory`: Use `PyConfig.user_site_directory` ou `PyConfig_Get("user_site_directory")`.
  - `Py_UnbufferedStdioFlag`: Use `PyConfig.buffered_stdio` ou `PyConfig_Get("buffered_stdio")`.
  - `Py_HashRandomizationFlag`: Use `PyConfig.use_hash_seed` e `PyConfig.hash_seed` ou `PyConfig_Get("hash_seed")`.

- `Py_IsolatedFlag`: Use `PyConfig.isolated` ou `PyConfig_Get("isolated")`.
- `Py_LegacyWindowsFSEncodingFlag`: Use `PyPreConfig.legacy_windows_fs_encoding` ou `PyConfig_Get("legacy_windows_fs_encoding")`.
- `Py_LegacyWindowsStdioFlag`: Use `PyConfig.legacy_windows_stdio` ou `PyConfig_Get("legacy_windows_stdio")`.
- **`Py_FileSystemDefaultEncoding`**, **`Py_HasFileSystemDefaultEncoding`**: Use `PyConfig.filesystem_encoding` ou `PyConfig_Get("filesystem_encoding")`.
- **`Py_FileSystemDefaultEncodeErrors`**: Use `PyConfig.filesystem_errors` ou `PyConfig_Get("filesystem_errors")`.
- **`Py_UTF8Mode`**: Use `PyPreConfig.utf8_mode` ou `PyConfig_Get("utf8_mode")`. (veja `Py_PreInitialize()`)

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig` para definir essas opções. Ou `PyConfig_Get()` pode ser usado para obter essas opções em tempo de execução.

Remoção pendente no Python 3.16

- A cópia empacotada do `libmpdec`.

Remoção pendente no Python 3.18

- As seguintes funções privadas estão descontinuado e planejadas para remoção no Python 3.18:

  - **`_PyBytes_Join()`**: use `PyBytes_Join()`.
  - **`_PyDict_GetItemStringWithError()`**: use `PyDict_GetItemStringRef()`.
  - **`_PyDict_Pop()`**: use `PyDict_Pop()`.
  - **`_PyLong_Sign()`**: use `PyLong_GetSign()`.

- **`_PyLong_FromDigits()`** e **`_PyLong_New()`**: use **`PyLongWriter_Create()`**.
- **`_PyThreadState_UncheckedGet()`**: use **`PyThreadState_GetUnchecked()`**.
- **`_PyUnicode_AsString()`**: use **`PyUnicode_AsUTF8()`**.
- **`_PyUnicodeWriter_Init()`**: substitua `_PyUnicodeWriter_Init(&writer)` com **`writer = PyUnicodeWriter_Create(0)`**.
- **`_PyUnicodeWriter_Finish()`**: substitua `_PyUnicodeWriter_Finish(&writer)` por **`PyUnicodeWriter_Finish(writer)`**.
- **`_PyUnicodeWriter_Dealloc()`**: substitua `_PyUnicodeWriter_Dealloc(&writer)` por **`PyUnicodeWriter_Discard(writer)`**.
- **`_PyUnicodeWriter_WriteChar()`**: substituta `_PyUnicodeWriter_WriteChar(&writer, ch)` com **`PyUnicodeWriter_WriteChar(writer, ch)`**.
- **`_PyUnicodeWriter_WriteStr()`**: substitua `_PyUnicodeWriter_WriteStr(&writer, str)` por **`PyUnicodeWriter_WriteStr(writer, str)`**.
- **`_PyUnicodeWriter_WriteSubstring()`**: substitua `_PyUnicodeWriter_WriteSubstring(&writer, str, start, end)` por **`PyUnicodeWriter_WriteSubstring(writer, str, start, end)`**.
- **`_PyUnicodeWriter_WriteASCIIString()`**: substitua `_PyUnicodeWriter_WriteASCIIString(&writer, str)` por **`PyUnicodeWriter_WriteASCII(writer, str)`**.
- **`_PyUnicodeWriter_WriteLatin1String()`**: substitua `_PyUnicodeWriter_WriteLatin1String(&writer, str)` por **`PyUnicodeWriter_WriteUTF8(writer, str)`**.
- **`_PyUnicodeWriter_Prepare()`**: (sem substituto).
- **`_PyUnicodeWriter_PrepareKind()`**: (sem substituto).
- **`_Py_HashPointer()`**: use **`Py_HashPointer()`**.
- **`_Py_fopen_obj()`**: use **`Py_fopen()`**.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas novas funções públicas no Python 3.13

e versões anteriores. (Contribuição de Victor Stinner em gh-128863 [https://github.com/python/cpython/issues/128863].)

Remoção pendente no Python 3.19

- **PEP 456** [https://peps.python.org/pep-0456/] suporte para incorporadores à definição do esquema de hashing de strings.

Remoção pendente no Python 3.20

- **`_PyObject_CallMethodId()`**, **`_PyObject_GetAttrId()`** e **`_PyUnicode_FromId()`** foram descontinuadas desde o 3.15 e serão removidas na versão 3.20. Em vez delas, use **`PyUnicode_InternFromString()`** and armazene em cache o resultado no estado do módulo e chame **`PyObject_CallMethod()`** ou **`PyObject_GetAttr()`**. (Contribuição de Victor Stinner em gh-141049 [https://github.com/python/cpython/issues/141049].)
- O campo `cval` em **`PyComplexObject`** (gh-128813 [https://github.com/python/cpython/issues/128813]). Use **`PyComplex_AsCComplex()`** e **`PyComplex_FromCComplex()`** para converter um número complexo Python de/para a representação C de **`Py_complex`**.
- Macros **`Py_MATH_PIl`** e **`Py_MATH_El`**.

Remoção pendente em versões futuras

As APIs a seguir foram descontinuadas e serão removidas, embora atualmente não haja uma data agendada para sua remoção.

- **`Py_TPFLAGS_HAVE_FINALIZE`**: desnecessária desde o Python 3.8.
- **`PyErr_Fetch()`**: use **`PyErr_GetRaisedException()`**.
- **`PyErr_NormalizeException()`**: use **`PyErr_GetRaisedException()`**.
- **`PyErr_Restore()`**: use **`PyErr_SetRaisedException()`**.
- **`PyModule_GetFilename()`**: use **`PyModule_GetFilenameObject()`**.
- **`PyOS_AfterFork()`**: use **`PyOS_AfterFork_Child()`**.
- **`PySlice_GetIndicesEx()`**: use **`PySlice_Unpack()`** e **`PySlice_AdjustIndices()`**.

- `PyUnicode_READY()`: desnecessário desde o Python 3.12
- `PyErr_Display()`: use `PyErr_DisplayException()`.
- `_PyErr_ChainExceptions()`: use `_PyErr_ChainExceptions1()`.
- O membro `PyBytesObject.ob_shash`: chame `PyObject_Hash()`.
- API do Thread Local Storage (TLS):
  - `PyThread_create_key()`: use `PyThread_tss_alloc()`.
  - `PyThread_delete_key()`: use `PyThread_tss_free()`.
  - `PyThread_set_key_value()`: use `PyThread_tss_set()`.
  - `PyThread_get_key_value()`: use `PyThread_tss_get()`.
  - `PyThread_delete_key_value()`: use `PyThread_tss_delete()`.
  - `PyThread_ReInitTLS()`: desnecessário desde o Python 3.7.

Glossário

>>>

O prompt padrão do console interativo do Python. Normalmente visto em exemplos de código que podem ser executados interativamente no interpretador.

...

Pode se referir a:

- O prompt padrão do console interativo do Python ao inserir o código para um bloco de código recuado, quando dentro de um par de delimitadores correspondentes esquerdo e direito (parênteses, colchetes, chaves ou aspas triplas) ou após especificar um decorador.
- Os três pontos formam o objeto Ellipsis (reticências).

classe base abstrata

Classes bases abstratas (ABCs, do inglês *abstract base class*) complementam a tipagem pato, fornecendo uma maneira de definir interfaces quando outras técnicas, como `hasattr()`, seriam desajeitadas ou sutilmente erradas (por exemplo, com métodos mágicos). ABCs introduzem subclasses virtuais, classes que não herdam de uma classe mas ainda são reconhecidas por `isinstance()` e `issubclass()`; veja a documentação do módulo `abc`. Python vem com muitas ABCs embutidas para estruturas de dados (no módulo `collections.abc`), números (no módulo `numbers`), fluxos (no módulo `io`), localizadores e carregadores de importação (no módulo `importlib.abc`). Você pode criar suas próprias ABCs com o módulo `abc`.

função de anotação

Um chamável que pode ser chamado para recuperar as anotações de um objeto. As funções de anotação são normalmente funções, geradas automaticamente como atributo `__annotate__` de funções, classes e

módulos. As funções de anotação são um subconjunto de funções de avaliação.

anotação

Um rótulo associado a uma variável, um atributo de classe ou um parâmetro de função ou valor de retorno, usado por convenção como dica de tipo.

Anotações de variáveis locais não podem ser acessadas em tempo de execução, mas anotações de variáveis globais, atributos de classe e funções podem ser recuperadas chamando **`annotationlib.get_annotations()`** em módulos, classes e funções, respectivamente.

Consulte anotação de variável, anotação de função, **PEP 484** [https://peps.python.org/pep-0484/], **PEP 526** [https://peps.python.org/pep-0526/] e **PEP 649** [https://peps.python.org/pep-0649/], que descrevem essa funcionalidade. Consulte também Boas práticas para anotações para obter as melhores práticas sobre como trabalhar com anotações.

argumento

Um valor passado para uma função (ou método) ao chamar a função. Existem dois tipos de argumento:

- *argumento nomeado*: um argumento precedido por um identificador (por exemplo, `name=`) na chamada de uma função ou passado como um valor em um dicionário precedido por `**`. Por exemplo, `3` e `5` são ambos argumentos nomeados nas chamadas da função **`complex()`** a seguir:

  ```
  complex(real=3, imag=5)
  complex(**{'real': 3, 'imag': 5})
  ```

- *argumento posicional*: um argumento que não é um argumento nomeado. Argumentos posicionais podem aparecer no início da lista de argumentos e/ou podem ser passados como elementos de um iterável precedidos por `*`. Por exemplo, `3` e `5` são ambos argumentos posicionais nas chamadas a seguir:

```
complex(3, 5)
complex(*(3, 5))
```

Argumentos são atribuídos às variáveis locais nomeadas no corpo da função. Veja a seção Chamadas para as regras de atribuição. Sintaticamente, qualquer expressão pode ser usada para representar um argumento; após a expressão ser avaliada, o valor resultante é atribuído à variável local.

Veja também o termo parâmetro no glossário, a pergunta no FAQ sobre a diferença entre argumentos e parâmetros e **PEP 362** [https://peps.python.org/pep-0362/].

gerenciador de contexto assíncrono

Um objeto que controla o ambiente envolto numa instrução `async with` por meio da definição dos métodos `__aenter__()` e `__aexit__()`. Introduzido pela **PEP 492** [https://peps.python.org/pep-0492/].

gerador assíncrono

Usado informalmente para se referir a uma função geradora assíncrona ou a um iterador gerador assíncrono, dependendo do contexto. Os termos formais função geradora assíncrona e iterador gerador assíncrono são incomuns na prática; “gerador assíncrono”, por si só, é quase sempre suficiente.

função geradora assíncrona

Uma função que retorna um iterador gerador assíncrono. É parecida com uma função de corrotina definida com `async def` exceto pelo fato de conter instruções `yield` para produzir uma série de valores que podem ser usados em um laço `async for`. Veja **PEP 525** [https://peps.python.org/pep-0525/].

Uma função geradora assíncrona pode conter expressões `await` e também as instruções `async for` e `async with`.

iterador gerador assíncrono

Um objeto criado por uma função geradora assíncrona.

Este é um iterador assíncrono que, quando chamado usando o método `__anext__()`, retorna um objeto aguardável que executará o corpo da função geradora assíncrona até a próxima expressão `yield`.

Cada `yield` suspende temporariamente o processamento, lembrando o estado de execução (incluindo variáveis locais e instruções `try` pendentes). Quando o *iterador gerador assíncrono* é efetivamente retomado com outro aguardável retornado por `__anext__()`, ele inicia de onde parou. Veja **PEP 492** [https://peps.python.org/pep-0492/] e **PEP 525** [https://peps.python.org/pep-0525/].

iterável assíncrono

Um objeto que pode ser usado em uma instrução `async for`. Deve retornar um iterador assíncrono do seu método `__aiter__()`. Introduzido por **PEP 492** [https://peps.python.org/pep-0492/].

iterador assíncrono

Um objeto que implementa os métodos `__aiter__()` e `__anext__()`. `__anext__()` deve retornar um objeto aguardável. `async for` resolve os aguardáveis retornados por um método `__anext__()` do iterador assíncrono até que ele levante uma exceção `StopAsyncIteration`. Introduzido pela **PEP 492** [https://peps.python.org/pep-0492/].

operação atômica

Uma operação que parece ser executada como uma única etapa indivisível: nenhuma outra thread pode observá-la pela metade, e os seus efeitos tornam-se visíveis de uma só vez. O Python não garante que as instruções de alto nível sejam atômicas (por exemplo, `x += 1` executa várias operações de bytecode e, portanto, não é atômica). A atomicidade só é garantida quando explicitamente documentada. Veja também condição de corrida e corrida de dados.

estado de thread anexado

Um estado de thread que está ativo para a thread atual do sistema operacional.

Quando um estado de thread é anexado, a thread do sistema operacional tem acesso à API C completa do Python e pode invocar com segurança o interpretador de bytecode.

A menos que uma função indique explicitamente o contrário, tentar chamar a API C sem um estado de thread anexado resultará em um erro fatal ou comportamento indefinido. Um estado de thread pode ser anexado e desanexado explicitamente pelo usuário por meio da API C, ou implicitamente pelo ambiente de execução, inclusive durante o bloqueio de chamadas C e pelo interpretador de bytecode entre as chamadas.

Na maioria das construções do Python, ter um estado de thread anexado implica que o chamador mantém a GIL para o interpretador atual, portanto, apenas uma thread do SO pode ter um estado de thread anexado em um determinado momento. Em construções do Python com threads livres, as threads podem manter simultaneamente um estado de thread anexado, permitindo um verdadeiro paralelismo do interpretador de bytecode.

atributo

Um valor associado a um objeto que é geralmente referenciado pelo nome separado por um ponto. Por exemplo, se um objeto *o* tem um atributo *a* esse seria referenciado como *o.a*.

É possível dar a um objeto um atributo cujo nome não seja um identificador conforme definido por Nomes (identificadores e palavras reservadas), por exemplo usando `setattr()`, se o objeto permitir. Tal atributo não será acessível usando uma expressão pontilhada e, em vez disso, precisaria ser recuperado com `getattr()`.

aguardável

Um objeto que pode ser usado em uma expressão `await`. Pode ser uma corrotina ou um objeto com um método `__await__()`. Veja também a **PEP 492** [https://peps.python.org/pep-0492/].

BDFL

Abreviação da expressão da língua inglesa “Benevolent Dictator for Life” (em português, “Ditador Benevolente Vitalício”), referindo-se a Guido van Rossum [https://gvanrossum.github.io/], criador do Python.

arquivo binário

Um objeto arquivo capaz de ler e gravar em objetos bytes ou similares. Exemplos de arquivos binários são arquivos abertos no modo binário (`'rb'`, `'wb'` ou `'rb+'`), `sys.stdin.buffer`, `sys.stdout.buffer`, e instâncias de `io.BytesIO` e `gzip.GzipFile`.

Veja também arquivo texto para um objeto arquivo capaz de ler e gravar em objetos `str`.

referência emprestada

Na API C do Python, uma referência emprestada é uma referência a um objeto que não é dona da referência. Ela se torna um ponteiro solto se o objeto for destruído. Por exemplo, uma coleta de lixo pode remover a última referência forte para o objeto e assim destruí-lo.

Chamar `Py_INCREF()` na referência emprestada é recomendado para convertê-lo, internamente, em uma referência forte, exceto quando o objeto não pode ser destruído antes do último uso da referência emprestada. A função `Py_NewRef()` pode ser usada para criar uma nova referência forte.

objeto bytes ou similar

Um objeto com suporte ao Protocolo de buffer e que pode exportar um buffer C contíguo. Isso inclui todos os objetos `bytes`, `bytearray` e `array.array`, além de muitos objetos `memoryview` comuns. Objetos bytes ou similares podem ser usados para várias operações que funcionam com dados binários; isso inclui compactação, salvamento em um arquivo binário e envio por um soquete.

Algumas operações precisam que os dados binários sejam mutáveis. A documentação geralmente se refere a eles como “objetos bytes ou similares para leitura e escrita”. Exemplos de objetos de buffer mutável incluem `bytearray` e um `memoryview` de um `bytearray`. Outras

operações exigem que os dados binários sejam armazenados em objetos imutáveis ("objetos bytes ou similares para somente leitura"); exemplos disso incluem **`bytes`** e a **`memoryview`** de um objeto **`bytes`**.

bytecode

O código-fonte Python é compilado para bytecode, a representação interna de um programa em Python no interpretador CPython. O bytecode também é mantido em cache em arquivos `.pyc` e `.pyo`, de forma que executar um mesmo arquivo é mais rápido na segunda vez (a recompilação dos fontes para bytecode não é necessária). Esta "linguagem intermediária" é adequada para execução em uma máquina virtual, que executa o código de máquina correspondente para cada bytecode. Tenha em mente que não se espera que bytecodes sejam executados entre máquinas virtuais Python diferentes, nem que se mantenham estáveis entre versões de Python.

Uma lista de instruções bytecode pode ser encontrada na documentação para o módulo dis.

chamável

Um chamável é um objeto que pode ser chamado, possivelmente com um conjunto de argumentos (veja argumento), com a seguinte sintaxe:

```
chamavel(argumento1, argumento2, argumentoN)
```

Uma função, e por extensão um método, é um chamável. Uma instância de uma classe que implementa o método **`__call__()`** também é um chamável.

função de retorno

Também conhecida como callback, é uma função sub-rotina que é passada como um argumento a ser executado em algum ponto no futuro.

classe

Um modelo para criação de objetos definidos pelo usuário. Definições de classe normalmente contém definições de métodos que operam sobre instâncias da classe.

variável de classe

Uma variável definida em uma classe e destinada a ser modificada apenas no nível da classe (ou seja, não em uma instância da classe).

variável de clausura

Uma variável livre referenciada de um escopo aninhado que é definida em um escopo externo em vez de ser resolvida em tempo de execução a partir dos espaços de nomes embutido ou globais. Pode ser explicitamente definida com a palavra reservada **`nonlocal`** para permitir acesso de gravação, ou implicitamente definida se a variável estiver sendo somente lida.

Por exemplo, na função `interna` no código a seguir, tanto `x` quanto `print` são variáveis livres, mas somente `x` é uma *variável de clausura*:

```
def externa():
    x = 0
    def interna():
        nonlocal x
        x += 1
        print(x)
    return interna
```

Devido ao atributo **`codeobject.co_freevars`** (que, apesar do nome, inclui apenas os nomes das variáveis de clausura em vez de listar todas as variáveis livres referenciadas), o termo mais geral variável livre às vezes é usado mesmo quando o significado pretendido é se referir especificamente às variáveis de clausura.

número complexo

Uma extensão ao familiar sistema de números reais em que todos os números são expressos como uma soma de uma parte real e uma parte imaginária. Números imaginários são múltiplos reais da unidade imaginária (a raiz quadrada de `-1`), normalmente escrita como `i` em matemática ou `j` em engenharia. O Python tem suporte nativo para números complexos, que são escritos com esta última notação; a parte imaginária escrita com um sufixo `j`, p.ex., `3+1j`. Para ter acesso aos equivalentes para números complexos do módulo **`math`**, utilize **`cmath`**. O

uso de números complexos é uma funcionalidade matemática bastante avançada. Se você não sabe se irá precisar deles, é quase certo que você pode ignorá-los sem problemas.

concorrência

A capacidade de um programa de computador executar múltiplas tarefas simultaneamente. Python fornece bibliotecas para escrever programas que utilizam diferentes formas de concorrência. O `asyncio` é uma biblioteca para lidar com tarefas assíncronas e corrotinas. O `threading` fornece acesso às threads do sistema operacional, e o `multiprocessing` a processos. Processadores com vários núcleos podem executar threads e processos em diferentes núcleos da CPU ao mesmo tempo (veja o paralelismo).

modificação concorrente

Quando várias threads modificam dados compartilhados simultaneamente, a modificação concorrente sem a devida sincronização pode causar condições de corrida e também pode causar corrida de dados ou corrupção de dados, ou ambos.

contexto

Este termo tem diferentes significados dependendo de onde e como ele é usado. Alguns significados comuns:

- O estado ou ambiente temporário estabelecido por um gerenciador de contexto por meio de uma instrução `with`.
- A coleção de ligações de chave-valor associadas a um objeto `contextvars.Context` específico e acessadas por meio de objetos `ContextVar`. Veja também variável de contexto.
- Um objeto `contextvars.Context`. Veja também contexto atual.

protocolo de gerenciamento de contexto

Os métodos `__enter__()` e `__exit__()` chamados pela instrução `with`. Veja **PEP 343** [https://peps.python.org/pep-0343/].

gerenciador de contexto

Um objeto que implementa o protocolo de gerenciamento de contexto e controla o ambiente envolto em uma instrução `with`. Veja **PEP 343** [https://peps.python.org/pep-0343/].

variável de contexto

Uma variável cujo valor depende de qual contexto é o contexto atual. Os valores são acessados por meio de objetos `contextvars.ContextVar`. Variáveis de contexto são usadas principalmente para isolar o estado entre tarefas assíncronas simultâneas.

contíguo

Um buffer é considerado contíguo exatamente se for *contíguo C* ou *contíguo Fortran*. Os buffers de dimensão zero são contíguos C e Fortran. Em vetores unidimensionais, os itens devem ser dispostos na memória próximos um do outro, em ordem crescente de índices, começando do zero. Em vetores multidimensionais contíguos C, o último índice varia mais rapidamente ao visitar itens em ordem de endereço de memória. No entanto, nos vetores contíguos do Fortran, o primeiro índice varia mais rapidamente.

corrotina

Corrotinas são uma forma mais generalizada de sub-rotinas. Sub-rotinas tem a entrada iniciada em um ponto, e a saída em outro ponto. Corrotinas podem entrar, sair, e continuar em muitos pontos diferentes. Elas podem ser implementadas com a instrução `async def`. Veja também **PEP 492** [https://peps.python.org/pep-0492/].

função de corrotina

Uma função que retorna um objeto do tipo corrotina. Uma função de corrotina pode ser definida com a instrução `async def`, e pode conter as palavras chaves `await`, `async for`, e `async with`. Isso foi introduzido pela **PEP 492** [https://peps.python.org/pep-0492/].

CPython

A implementação canônica da linguagem de programação Python, como disponibilizada pelo python.org [https://www.python.org]. O termo “CPython” é usado quando necessário distinguir esta implementação de outras como Jython ou IronPython.

contexto atual

O contexto (objeto `contextvars.Context`) que é usado atualmente pelos objetos `ContextVar` para acessar (obter ou definir) os valores de variáveis de contexto. Cada thread tem seu próprio contexto atual. Frameworks para executar tarefas assíncronas (veja `asyncio`) associam cada tarefa a um contexto que se torna o contexto atual sempre que a tarefa inicia ou retoma a execução.

isolado cíclico

Um subgrupo de um ou mais objetos que se referenciam em um ciclo de referência, mas não são referenciados por objetos de fora do grupo. O objetivo do coletor de lixo cíclico é identificar esses grupos e interromper os ciclos de referência para que a memória possa ser recuperada.

corrida de dados

Uma situação em que várias threads acessam simultaneamente o mesmo local da memória, pelo menos um dos acessos é uma gravação e as threads não utilizam qualquer sincronização para controlar o seu acesso. As corridas de dados levam a um comportamento não determinístico e podem causar corrupção de dados. O uso adequado de travas e outras primitivas de sincronização evita corridas de dados. Observe que as corridas de dados só podem ocorrer em código nativo, mas que o código nativo pode estar exposto em uma API Python. Veja também condição de corrida e seguro para threads.

impasse

Uma situação em que duas ou mais tarefas (threads, processos ou corrotinas) esperam indefinidamente umas pelas outras para liberar recursos ou concluir ações, impedindo que qualquer uma delas avance. Por exemplo, se a thread A mantém a trava 1 e espera pela trava 2,

enquanto a thread B mantém a trava 2 e espera pela trava 1, ambas as threads ficarão à espera indefinidamente. Em Python, isto geralmente ocorre devido à aquisição de várias travas em ordens conflitantes ou a dependências circulares de join/await. Os impasses podem ser evitados adquirindo várias travas sempre em uma ordem consistente. Veja também trava e reentrante.

decorador

Uma função que retorna outra função, geralmente aplicada como uma transformação de função usando a sintaxe `@wrapper`. Exemplos comuns para decoradores são **`@classmethod`** e **`@staticmethod`**.

A sintaxe do decorador é meramente um açúcar sintático, as duas definições de funções a seguir são semanticamente equivalentes:

```
def f(arg):
    ...
f = staticmethod(f)

@staticmethod
def f(arg):
    ...
```

O mesmo conceito existe para as classes, mas não é comumente utilizado. Veja a documentação de definições de função e definições de classe para obter mais informações sobre decoradores.

descritor

Qualquer objeto que define os métodos **`__get__()`**, **`__set__()`** ou **`__delete__()`**. Quando um atributo de classe é um descritor, seu comportamento de associação especial é acionado no acesso a um atributo. Normalmente, ao se utilizar *a.b* para se obter, definir ou excluir, um atributo dispara uma busca no objeto chamado *b* no dicionário de classe de *a*, mas se *b* for um descritor, o respectivo método descritor é chamado. Compreender descritores é a chave para um profundo entendimento de Python pois eles são a base de muitas funcionalidades incluindo funções, métodos, propriedades, métodos de classe, métodos estáticos e referências para superclasses.

Para obter mais informações sobre os métodos dos descritores, veja: Implementando descritores ou o Guia de Descritores.

dicionário

Um vetor associativo em que chaves arbitrárias são mapeadas para valores. As chaves podem ser quaisquer objetos que possuam os métodos `__hash__()` e `__eq__()`. Isso é chamado de hash em Perl.

compreensão de dicionário

Uma maneira compacta de processar todos ou parte dos elementos de um iterável e retornar um dicionário com os resultados. `results = {n: n ** 2 for n in range(10)}` gera um dicionário contendo a chave `n` mapeada para o valor `n ** 2`. Veja Compreensões.

visão de dicionário

Os objetos retornados por `dict.keys()`, `dict.values()` e `dict.items()` são chamados de visões de dicionário. Eles fornecem uma visão dinâmica das entradas do dicionário, o que significa que quando o dicionário é alterado, a visão reflete essas alterações. Para forçar a visão de dicionário a se tornar uma lista completa use `list(dictview)`. Veja Objetos de visão de dicionário.

docstring

Abreviatura de “documentation string” (string de documentação). Uma string literal que aparece como primeira expressão numa classe, função ou módulo. Ainda que sejam ignoradas quando a suíte é executada, é reconhecida pelo compilador que a coloca no atributo `__doc__` da classe, função ou módulo que a encapsula. Como ficam disponíveis por meio de introspecção, docstrings são o lugar canônico para documentação do objeto.

tipagem pato

Também conhecida como *duck-typing*, é um estilo de programação que não verifica o tipo do objeto para determinar se ele possui a interface correta; em vez disso, o método ou atributo é simplesmente chamado ou utilizado (“Se se parece com um pato e grasna como um pato, então

deve ser um pato.”) Enfatizando interfaces ao invés de tipos específicos, o código bem desenvolvido aprimora sua flexibilidade por permitir substituição polimórfica. Tipagem pato evita a necessidade de testes que usem `type()` ou `isinstance()`. (Note, porém, que a tipagem pato pode ser complementada pelo uso de classes base abstratas. Em vez disso, normalmente são empregados testes `hasattr()` ou a programação EAFP.

dunder

Uma abreviação informal para “double underscore” (em português, sublinhado duplo), usada ao se referir a um método especial. Por exemplo, `__init__` é frequentemente pronunciado como “dunder init”.

EAFP

Iniciais da expressão em inglês “easier to ask for forgiveness than permission” que significa “é mais fácil pedir perdão que permissão”. Este estilo de codificação comum no Python presume a existência de chaves ou atributos válidos e captura exceções caso essa premissa se prove falsa. Este estilo limpo e rápido se caracteriza pela presença de várias instruções `try` e `except`. A técnica diverge do estilo LBYL, comum em outras linguagens como C, por exemplo.

função de avaliação

Uma função que pode ser chamada para avaliar um atributo com avaliação preguiçosa de um objeto, como o valor de apelidos de tipo criados com a instrução `type`.

esgotado

Diz-se que um iterador que produziu todos os seus valores está *esgotado*. Tentativas subsequentes de obter o próximo valor (por exemplo, chamadas para `next()`) levantam a exceção `StopIteration` (ou `StopAsyncIteration`, no caso de um iterador assíncrono).

expressão

Uma parte da sintaxe que pode ser avaliada para algum valor. Em outras palavras, uma expressão é a acumulação de elementos de expressão

como literais, nomes, atributos de acesso, operadores ou chamadas de funções, todos os quais retornam um valor. Nem todas as construções de linguagem são expressões. Também existem instruções, as quais não podem ser usadas como expressões, como, por exemplo, **`while`**. Atribuições também são instruções, não expressões.

módulo de extensão

Um módulo escrito em C ou C++, usando a API C do Python para interagir tanto com código de usuário quanto do núcleo.

f-string
f-strings

Literais de string prefixados com `f` ou `F` são comumente chamados de "f-strings", abreviação em inglês de literais de strings formatadas. Veja também **PEP 498** [https://peps.python.org/pep-0498/].

objeto arquivo

Um objeto que expõe uma API orientada a arquivos (com métodos tais como **`read()`** ou **`write()`**) para um recurso subjacente. Dependendo da maneira como foi criado, um objeto arquivo pode mediar o acesso a um arquivo real no disco ou outro tipo de dispositivo de armazenamento ou de comunicação (por exemplo a entrada/saída padrão, buffers em memória, soquetes, pipes, etc.). Objetos arquivo também são chamados de *objetos arquivo ou similares* ou *fluxos*.

Atualmente há três categorias de objetos arquivo: arquivos binários brutos, arquivos binários em buffer e arquivos textos. Suas interfaces estão definidas no módulo **`io`**. A forma canônica para criar um objeto arquivo é usando a função **`open()`**.

objeto arquivo ou similar

Um sinônimo do termo objeto arquivo.

tratador de erros e codificação do sistema de arquivos

Tratador de erros e codificação usado pelo Python para decodificar bytes do sistema operacional e codificar Unicode para o sistema

operacional.

A codificação do sistema de arquivos deve garantir a decodificação bem-sucedida de todos os bytes abaixo de 128. Se a codificação do sistema de arquivos falhar em fornecer essa garantia, as funções da API podem levantar `UnicodeError`.

As funções `sys.getfilesystemencoding()` e `sys.getfilesystemencodeerrors()` podem ser usadas para obter o tratador de erros e codificação do sistema de arquivos.

O tratador de erros e codificação do sistema de arquivos são configurados na inicialização do Python pela função `PyConfig_Read()`: veja os membros `filesystem_encoding` e `filesystem_errors` do `PyConfig`.

Veja também codificação da localidade.

localizador

Um objeto que tenta encontrar o carregador para um módulo que está sendo importado.

Existem dois tipos de localizador: localizadores de metacaminho para uso com `sys.meta_path`, e localizadores de entrada de caminho para uso com `sys.path_hooks`.

Veja Localizadores e carregadores e `importlib` para muito mais detalhes.

divisão pelo piso

Divisão matemática que arredonda para baixo para o inteiro mais próximo. O operador de divisão pelo piso é `//`. Por exemplo, a expressão `11 // 4` retorna o valor `2` ao invés de `2.75`, que seria retornado pela divisão de ponto flutuante. Note que `(-11) // 4` é `-3` porque é `-2.75` arredondado *para baixo*. Consulte a **PEP 238** [https://peps.python.org/pep-0238/].

threads livres

Um modelo de threads onde múltiplas threads podem simultaneamente executar bytecode Python no mesmo interpretador. Isso está em contraste com a trava global do interpretador que permite apenas uma thread por vez executar bytecode Python. Veja **PEP 703** [https://peps.python.org/pep-0703/].

construção com threads livres

Uma construção do CPython que oferece suporte a threads livres, configurada usando a opção `--disable-gil` antes da compilação.

Veja Suporte do Python para threads livres.

variável livre

Formalmente, conforme definido no modelo de execução de linguagem, uma variável livre é qualquer variável usada em um espaço de nomes que não seja uma variável local naquele espaço de nomes. Veja variável de clausura para um exemplo. Pragmaticamente, devido ao nome do atributo `codeobject.co_freevars`, o termo também é usado algumas vezes como sinônimo de variável de clausura.

função

Uma série de instruções que retorna algum valor para um chamador. Também pode ser passado zero ou mais argumentos que podem ser usados na execução do corpo. Veja também parâmetro, método e a seção Definições de função.

anotação de função

Uma anotação de um parâmetro de função ou valor de retorno.

Anotações de função são comumente usados por dicas de tipo: por exemplo, essa função espera receber dois argumentos `int` e também é esperado que devolva um valor `int`:

```
def soma_dois_numeros(a: int, b: int) -> int:
   return a + b
```

A sintaxe de anotação de função é explicada na seção Definições de função.

Veja anotação de variável e **PEP 484** [https://peps.python.org/pep-0484/], que descrevem esta funcionalidade. Veja também Boas práticas para anotações para as melhores práticas sobre como trabalhar com anotações.

__future__

A instrução future, `from __future__ import <feature>`, direciona o compilador a compilar o módulo atual usando sintaxe ou semântica que será padrão em uma versão futura de Python. O módulo `__future__` documenta os possíveis valores de *feature*. Importando esse módulo e avaliando suas variáveis, você pode ver quando um novo recurso foi inicialmente adicionado à linguagem e quando será (ou se já é) o padrão:

```
>>> import __future__
>>> __future__.division
_Feature((2, 2, 0, 'alpha', 2), (3, 0, 0, 'alpha', 0), 8192)
```

coleta de lixo

Também conhecido como *garbage collection*, é o processo de liberar a memória quando ela não é mais utilizada. Python executa a liberação da memória através da contagem de referências e um coletor de lixo cíclico que é capaz de detectar e interromper referências cíclicas. O coletor de lixo pode ser controlado usando o módulo `gc`.

gerador

Usado informalmente para se referir a uma função geradora ou a um iterador gerador, dependendo do contexto. Os termos formais função geradora e iterador gerador são incomuns na prática; “gerador”, por si só, é quase sempre suficiente.

função geradora

Uma função que retorna um gerador. É parecida com uma função normal, exceto pelo fato de conter expressões `yield` para produzir uma

série de valores que podem ser usados em um laço **`for`** ou que podem ser obtidos um de cada vez com a função **`next()`**. Veja Expressões yield.

iterador gerador

Um objeto criado por uma função geradora ou uma expressão geradora.

Cada **`yield`** suspende temporariamente o processamento, memorizando o estado da execução (incluindo variáveis locais e instruções try pendentes). Quando o *iterador gerador* retorna, ele se recupera do último ponto onde estava (em contrapartida as funções que iniciam uma nova execução a cada vez que são invocadas).

Iteradores geradores também implementam o método **`send()`** para enviar um valor para o gerador suspenso, e o método **`throw()`** levanta uma exceção no ponto onde o gerador foi pausado. Veja Métodos de iterador gerador.

expressão geradora

Uma expressão que retorna um iterador. Parece uma expressão normal, seguido de uma cláusula **`for`** definindo uma variável de laço, um intervalo, e uma cláusula **`if`** opcional. A expressão combinada gera valores para uma função encapsuladora:

```
>>> sum(i*i for i in range(10))         # soma dos quadrados
285
```

função genérica

Uma função composta por várias funções implementando a mesma operação para diferentes tipos. Qual implementação deverá ser usada durante a execução é determinada pelo algoritmo de despacho.

Veja também a entrada despacho único no glossário, o decorador **`@functools.singledispatch`**, e a **PEP 443** [https://peps.python.org/pep-0443/].

tipo genérico

Um tipo que pode ser parametrizado; tipicamente uma classe contêiner tal como `list` ou `dict`. Usado para dicas de tipo e anotações.

Para mais detalhes, veja tipo apelido genérico, **PEP 483** [https://peps.python.org/pep-0483/], **PEP 484** [https://peps.python.org/pep-0484/], **PEP 585** [https://peps.python.org/pep-0585/], e o módulo `typing`.

GIL

Veja trava global do interpretador.

trava global do interpretador

O mecanismo utilizado pelo interpretador CPython para garantir que apenas uma thread execute o bytecode Python por vez. Isto simplifica a implementação do CPython ao fazer com que o modelo de objetos (incluindo tipos embutidos críticos como o `dict`) ganhem segurança implícita contra acesso concorrente. Travar todo o interpretador facilita que o interpretador em si seja multithread, às custas de muito do paralelismo já provido por máquinas com multiprocessadores.

No entanto, alguns módulos de extensão, tanto da biblioteca padrão quanto de terceiros, são desenvolvidos de forma a liberar a GIL (do inglês, global interpreter lock) ao realizar tarefas computacionalmente muito intensas, como compactação ou cálculos de hash. Além disso, a GIL é sempre liberada nas operações de E/S.

A partir de Python 3.13, a GIL pode ser desabilitada usando a configuração de construção `--disable-gil`. Depois de construir Python com essa opção, o código deve ser executado com a opção `-X gil=0` ou com a variável de ambiente `PYTHON_GIL=0` definida. Esse recurso melhora o desempenho de aplicações com múltiplas threads e torna mais fácil o uso eficiente de CPUs com múltiplos núcleos. Para mais detalhes, veja **PEP 703** [https://peps.python.org/pep-0703/].

Em versões anteriores da API C do Python, uma função podia declarar que precisava que a GIL fosse mantida para ser usada. Isso significa ter um estado de thread anexado.

estado global

Dados que são acessíveis em todo o programa, tais como variáveis ao nível do módulo, variáveis de classe ou variáveis estáticas C em módulos de extensão. Em programas multithread, o estado global partilhado entre threads normalmente requer sincronização para evitar condições de corrida e corridas de dados.

pyc baseado em hash

Um arquivo de cache em bytecode que usa hash ao invés do tempo, no qual o arquivo de código-fonte foi modificado pela última vez, para determinar a sua validade. Veja Invalidação de bytecode em cache.

hasheável

Um objeto é *hasheável* se tem um valor de hash que nunca muda durante seu ciclo de vida (precisa ter um método `__hash__()`) e pode ser comparado com outros objetos (precisa ter um método `__eq__()`). Objetos hasheáveis que são comparados como iguais devem ter o mesmo valor de hash.

A hasheabilidade faz com que um objeto possa ser usado como uma chave de dicionário e como um membro de conjunto, pois estas estruturas de dados utilizam os valores de hash internamente.

A maioria dos objetos embutidos imutáveis do Python é hasheável; contêineres mutáveis (tais como listas ou dicionários) não são; contêineres imutáveis (tais como tuplas e frozensets) são hasheáveis apenas se seus elementos também o são. Objetos que são instâncias de classes definidas pelo usuário são hasheáveis por padrão. Todos eles comparam de forma desigual (exceto entre si mesmos), e o valor de hash é derivado do `id()`.

IDLE

Um ambiente de desenvolvimento e aprendizado integrado para Python. IDLE — Editor e console Python é um editor básico e um ambiente interpretador que vem junto com a distribuição padrão do Python.

imortal

Objetos imortais são um detalhe da implementação do CPython introduzida na **PEP 683** [https://peps.python.org/pep-0683/].

Se um objeto é imortal, sua contagem de referências nunca é modificada e, portanto, nunca é desalocado enquanto o interpretador está em execução. Por exemplo, `True` e `None` são imortais no CPython.

Objetos imortais podem ser identificados via `sys.is_immortal()` ou via `PyUnstable_IsImmortal()` na API C.

imutável

Um objeto com valor fixo. Objetos imutáveis incluem números, strings e tuplos. Este tipo de objeto não pode ser alterado. É necessário criar um novo objeto se for necessário armazenar um valor diferente. Eles desempenham um papel importante em locais onde é necessário um valor hash constante, por exemplo, como uma chave num dicionário. Os objetos imutáveis são inerentemente seguros para threads porque o seu estado não pode ser modificado após a criação, eliminando preocupações com modificações simultâneas sincronizadas incorretamente.

caminho de importação

Uma lista de localizações (ou entradas de caminho) que são buscadas pelo localizador baseado no caminho por módulos para importar. Durante a importação, esta lista de localizações usualmente vem a partir de `sys.path`, mas para subpacotes ela também pode vir do atributo `__path__` de pacotes-pai.

importação

O processo pelo qual o código Python em um módulo é disponibilizado para o código Python em outro módulo.

importador

Um objeto que localiza e carrega um módulo; Tanto um localizador e o objeto carregador.

índice

Um valor numérico que representa a posição de um elemento numa sequência.

Em Python, a indexação começa em zero. Por exemplo, `coisas[0]` nomeia o *primeiro* elemento de `coisas`; `coisas[1]` nomeia o segundo.

Em alguns contextos, o Python permite índices negativos para contar a partir do final de uma sequência e indexação por fatia.

Veja também subscrito.

interativo

Python tem um interpretador interativo, o que significa que você pode digitar instruções e expressões no prompt do interpretador, executá-los imediatamente e ver seus resultados. Apenas execute `python` sem argumentos (possivelmente selecionando-o a partir do menu de aplicações de seu sistema operacional). O interpretador interativo é uma maneira poderosa de testar novas ideias ou aprender mais sobre módulos e pacotes (lembre-se do comando `help(x)`). Para saber mais sobre modo interativo, veja Modo interativo.

interpretado

Python é uma linguagem interpretada, em oposição àquelas que são compiladas, embora esta distinção possa ser nebulosa devido à presença do compilador de bytecode. Isto significa que os arquivos-fontes podem ser executados diretamente sem necessidade explícita de se criar um arquivo executável. Linguagens interpretadas normalmente têm um ciclo de desenvolvimento/depuração mais curto que as linguagens compiladas, apesar de seus programas geralmente serem executados mais lentamente. Veja também interativo.

desligamento do interpretador

Quando solicitado para desligar, o interpretador Python entra em uma fase especial, onde ele gradualmente libera todos os recursos alocados, tais como módulos e várias estruturas internas críticas. Ele também faz diversas chamadas para o coletor de lixo. Isto pode disparar a execução

de código em destrutores definidos pelo usuário ou função de retorno de referência fraca. Código executado durante a fase de desligamento pode encontrar diversas exceções, pois os recursos que ele depende podem não funcionar mais (exemplos comuns são os módulos de bibliotecas, ou os mecanismos de avisos).

A principal razão para o interpretador desligar, é que o módulo `__main__` ou o script sendo executado terminou sua execução.

iterável

Um objeto capaz de retornar seus membros um de cada vez. Exemplos de iteráveis incluem todos os tipos de sequência (tais como **`list`**, **`str`** e **`tuple`**) e alguns tipos de não-sequência, como o **`dict`** e objetos arquivo, além dos objetos de quaisquer classes que você definir com um método **`__iter__()`** ou **`__getitem__()`** que implementam a semântica de sequência.

Iteráveis podem ser usados em um laço **`for`** e em vários outros lugares em que uma sequência é necessária (**`zip()`**, **`map()`**, …). Quando um objeto iterável é passado como argumento para a função embutida **`iter()`**, ela retorna um iterador para o objeto. Este iterador é adequado para se varrer todo o conjunto de valores. Ao usar iteráveis, normalmente não é necessário chamar **`iter()`** ou lidar com os objetos iteradores em si. A instrução **`for`** faz isso automaticamente para você, criando uma variável temporária para armazenar o iterador durante a execução do laço. Veja também iterador, sequência, e gerador.

iterador

Um objeto que representa um fluxo de dados. Repetidas chamadas ao método **`__next__()`** de um iterador (ou passando o objeto para a função embutida **`next()`**) vão retornar itens sucessivos do fluxo. Quando não houver mais dados disponíveis uma exceção **`StopIteration`** será levantada. Neste ponto, o objeto iterador é esgotado e quaisquer chamadas subsequentes a seu método **`__next__()`** vão apenas levantar a exceção **`StopIteration`** novamente. Iteradores precisam ter um método **`__iter__()`** que retorne o objeto iterador em si, de forma que todo iterador também é iterável e pode ser usado na maioria dos lugares em

que um iterável é requerido. Uma notável exceção é código que tenta realizar passagens em múltiplas iterações. Um objeto contêiner (como uma `list`) produz um novo iterador a cada vez que você passá-lo para a função `iter()` ou utilizá-lo em um laço `for`. Tentar isso com o mesmo iterador apenas iria retornar o mesmo objeto iterador esgotado já utilizado na iteração anterior, como se fosse um contêiner vazio.

Mais informações podem ser encontradas em Tipos iteradores.

O CPython não aplica consistentemente a exigência de que um iterador defina `__iter__()`. E também observe que o CPython com threads livres não garante o comportamento seguro para threds das operações do iterador.

chave

Um valor que identifica uma entrada num mapeamento. Ver também subscrito.

função chave

Uma função chave ou função colação é um chamável que retorna um valor usado para ordenação ou classificação. Por exemplo, `locale.strxfrm()` é usada para produzir uma chave de ordenação que leva o locale em consideração para fins de ordenação.

Uma porção de ferramentas no Python aceitam funções chave para controlar como os elementos são ordenados ou agrupados. Algumas delas incluem `min()`, `max()`, `sorted()`, `list.sort()`, `heapq.merge()`, `heapq.nsmallest()`, `heapq.nlargest()` e `itertools.groupby()`.

Existem várias maneiras de criar uma função-chave. Por exemplo, o método `str.casefold()` pode servir como uma função-chave para ordenações que não diferenciam maiúsculas de minúsculas. Alternativamente, uma função-chave pode ser construída a partir de uma expressão `lambda`, como `lambda r: (r[0], r[2])`. Além disso, `operator.attrgetter()`, `operator.itemgetter()` e `operator.methodcaller()` são três construtores de funções-chave.

Consulte Técnicas de ordenação para obter exemplos de como criar e usar funções-chave.

argumento nomeado

Veja argumento.

lambda

Uma função de linha anônima consistindo de uma única expressão, que é avaliada quando a função é chamada. A sintaxe para criar uma função lambda é `lambda [parameters]: expression`

LBYL

Iniciais da expressão em inglês "look before you leap", que significa algo como "olhe antes de pisar". Este estilo de codificação testa as pré-condições explicitamente antes de fazer chamadas ou buscas. Este estilo contrasta com a abordagem EAFP e é caracterizada pela presença de muitas instruções `if`.

Em um ambiente multithread, a abordagem LBYL pode correr o risco de introduzir uma condição de corrida entre "a pesquisa" e "o salto". Por exemplo, o código `if key in mapping: return mapping[key]` pode falhar se outra thread remover *key* de *mapping* após o teste, mas antes da pesquisa. Este problema pode ser resolvido com travas ou usando a abordagem EAFP. Veja também seguro para thread.

analisador léxico

Nome formal para o *tokenizador*; veja token.

lista

Uma sequência embutida no Python. Apesar do seu nome, é mais próximo de um vetor em outras linguagens do que uma lista encadeada, como o acesso aos elementos é da ordem $O(1)$. Veja Complexidade de tempo de operações em tipos embutidos.

compreensão de lista

Uma maneira compacta de processar todos ou parte dos elementos de uma sequência e retornar os resultados em uma lista. `result = ['{:#04x}'.format(x) for x in range(256) if x % 2 == 0]` gera uma lista de strings contendo números hexadecimais (0x..) no intervalo de 0 a 255. A cláusula **`if`** é opcional. Se omitida, todos os elementos no `range(256)` serão processados.

trava

Uma primitiva de sincronização que permite que apenas um segmento de execução acesse um recurso compartilhado de cada vez. Um segmento de execução deve adquirir uma trava antes de acessar o recurso protegido e liberá-lo depois. Se um segmento de execução tentar adquirir uma trava já na posse de outro segmento de execução, ele ficará bloqueado até que a trava fique disponível. O módulo **`threading`** do Python fornece **`Lock`** (uma trava básica) e **`RLock`** (uma trava reentrante). As travas são usadas para evitar condições de corrida e garantir o acesso seguro para threads a dados compartilhados. Existem padrões de projeto alternativos às travas, como filas, padrões produtor/consumidor e estado local da thread. Veja também impasse e reentrante.

sem travas

Uma operação que não adquire nenhuma trava e usa instruções de CPU atômicas para garantir a corretude. Operações sem travas podem ser executadas simultaneamente, sem bloquear umas às outras, e não podem ser bloqueadas por operações com travas. No Python com threads livres, tipos embutidos como **`dict`** e **`list`** fornecem operações de leitura sem travas, o que significa que outras threads podem observar estados intermediários durante modificações em várias etapas, mesmo quando essas modificações mantêm a trava por objeto.

carregador

Um objeto que carrega um módulo. Ele deve definir os métodos **`exec_module()`** e **`create_module()`** para implementar a interface **`Loader`**. Um carregador é normalmente retornado por um localizador. Veja também:

- Localizadores e carregadores

- `importlib.abc.Loader`
- **PEP 302** [https://peps.python.org/pep-0302/]

codificação da localidade

No Unix, é a codificação da localidade do LC_CTYPE, que pode ser definida com `locale.setlocale(locale.LC_CTYPE, new_locale)`.

No Windows, é a página de código ANSI (ex: `"cp1252"`).

No Android e no VxWorks, o Python usa `"utf-8"` como a codificação da localidade.

`locale.getencoding()` pode ser usado para obter a codificação da localidade.

Veja também tratador de erros e codificação do sistema de arquivos.

método mágico

Um sinônimo informal para um método especial.

mapeamento

Um objeto contêiner com suporte a pesquisas de chave arbitrária e que implementa os métodos especificados nas classes base abstratas `collections.abc.Mapping` ou `collections.abc.MutableMapping`. Exemplos incluem `dict`, `collections.defaultdict`, `collections.OrderedDict` e `collections.Counter`.

localizador de metacaminho

Um localizador retornado por uma busca de `sys.meta_path`. Localizadores de metacaminho são relacionados a, mas diferentes de, localizadores de entrada de caminho.

Veja `importlib.abc.MetaPathFinder` para os métodos que localizadores de metacaminho implementam.

metaclasse

A classe de uma classe. Definições de classe criam um nome de classe, um dicionário de classe e uma lista de classes base. A metaclasse é responsável por receber estes três argumentos e criar a classe. A maioria das linguagens de programação orientadas a objetos provê uma implementação default. O que torna o Python especial é o fato de ser possível criar metaclasses personalizadas. A maioria dos usuários nunca vai precisar deste recurso, mas quando houver necessidade, metaclasses possibilitam soluções poderosas e elegantes. Metaclasses têm sido utilizadas para gerar registros de acesso a atributos, para incluir proteção contra acesso concorrente, rastrear a criação de objetos, implementar singletons, dentre muitas outras tarefas.

Mais informações podem ser encontradas em Metaclasses.

método

Uma função que é definida dentro do corpo de uma classe. Se chamada como um atributo de uma instância daquela classe, o método receberá a instância do objeto como seu primeiro argumento (que comumente é chamado de `self`). Veja função e escopo aninhado.

ordem de resolução de métodos

Ordem de resolução de métodos é a ordem em que os membros de uma classe base são buscados durante a pesquisa. Veja A Ordem de Resolução de Métodos do Python 2.3 para detalhes do algoritmo usado pelo interpretador do Python desde a versão 2.3.

módulo

Um objeto que serve como uma unidade organizacional de código Python. Os módulos têm um espaço de nomes contendo objetos Python arbitrários. Os módulos são carregados pelo Python através do processo de importação.

Veja também pacote.

spec de módulo

Um espaço de nomes que contém as informações relacionadas à importação usadas para carregar um módulo. Uma instância de

`importlib.machinery.ModuleSpec`.

Veja também Especificações de módulo.

MRO

Veja ordem de resolução de métodos.

mutável

Um objeto com estado alterável durante a execução do programa. Em programas multithread, objetos mutáveis compartilhados entre threads requerem sincronização cuidadosa para evitar condições de corrida. Veja também imutável, seguro para threads e modificação simultânea.

tupla nomeada

O termo "tupla nomeada" é aplicado a qualquer tipo ou classe que herda de tupla e cujos elementos indexáveis também são acessíveis usando atributos nomeados. O tipo ou classe pode ter outras funcionalidades também.

Diversos tipos embutidos são tuplas nomeadas, incluindo os valores retornados por `time.localtime()` e `os.stat()`. Outro exemplo é `sys.float_info`:

```
>>> sys.float_info[1]                    # acesso indexado
1024
>>> sys.float_info.max_exp               # acesso a campo nome
1024
>>> isinstance(sys.float_info, tuple)    # tipo de tupla
True
```

Algumas tuplas nomeadas são tipos embutidos (tal como os exemplos acima). Alternativamente, uma tupla nomeada pode ser criada a partir de uma definição de classe regular, que herde de `tuple` e que defina campos nomeados. Tal classe pode ser escrita a mão, ou ela pode ser criada herdando `typing.NamedTuple` ou com uma função fábrica `collections.namedtuple()`. As duas últimas técnicas também

adicionam alguns métodos extras, que podem não ser encontrados quando foi escrita manualmente, ou em tuplas nomeadas embutidas.

espaço de nomes

O lugar em que uma variável é armazenada. Espaços de nomes são implementados como dicionários. Existem os espaços de nomes local, global e nativo, bem como espaços de nomes aninhados em objetos (em métodos). Espaços de nomes suportam modularidade ao prevenir conflitos de nomes. Por exemplo, as funções **`__builtin__.open()`** e **`os.open()`** são diferenciadas por seus espaços de nomes. Espaços de nomes também auxiliam na legibilidade e na manutenibilidade ao torar mais claro quais módulos implementam uma função. Escrever **`random.seed()`** ou **`itertools.izip()`**, por exemplo, deixa claro que estas funções são implementadas pelos módulos **`random`** e **`itertools`** respectivamente.

pacote de espaço de nomes

Um pacote que serve apenas como contêiner para subpacotes. Pacotes de espaços de nomes podem não ter representação física, e especificamente não são como um pacote regular porque eles não tem um arquivo `__init__.py`.

Pacotes de espaço de nomes permitem que vários pacotes instaláveis individualmente tenham um pacote pai comum. Caso contrário, é recomendado usar um pacote regular.

Para mais informações, veja **PEP 420** [https://peps.python.org/pep-0420/] e Pacotes de espaço de nomes.

Veja também módulo.

código nativo

Código que é compilado em instruções de máquina e executado diretamente no processador, em oposição ao código que é interpretado ou executado numa máquina virtual. No contexto do Python, código nativo normalmente refere-se ao código C, C++, Rust ou Fortran em

módulos de extensão que podem ser chamados a partir do Python. Veja também módulo de extensão.

escopo aninhado

A habilidade de referir-se a uma variável em uma definição de fechamento. Por exemplo, uma função definida dentro de outra pode referenciar variáveis da função externa. Perceba que escopos aninhados por padrão funcionam apenas por referência e não por atribuição. Variáveis locais podem ler e escrever no escopo mais interno. De forma similar, variáveis globais podem ler e escrever para o espaço de nomes global. O `nonlocal` permite escrita para escopos externos.

classe estilo novo

Antigo nome para o tipo de classes agora usado para todos os objetos de classes. Em versões anteriores do Python, apenas classes estilo novo podiam usar recursos novos e versáteis do Python, tais como `__slots__`, descritores, propriedades, `__getattribute__()`, métodos de classe e métodos estáticos.

não determinístico

Comportamento em que o resultado de um programa pode variar entre execuções com as mesmas entradas. Em programas multithread, o comportamento não determinístico geralmente resulta de condições de corrida, em que o tempo relativo ou o intercalamento de threads afeta o resultado. A sincronização adequada, usando bloqueios e outras primitivas de sincronização, ajuda a garantir um comportamento determinístico.

objeto

Qualquer dado que tenha estado (atributos ou valores) e comportamento definidos (métodos). Também a última classe base de qualquer classe estilo novo.

escopo otimizado

Um escopo no qual os nomes das variáveis locais de destino são conhecidos de forma confiável pelo compilador quando o código é

compilado, permitindo a otimização do acesso de leitura e gravação a esses nomes. Os espaços de nomes locais para funções, geradores, corrotinas, compreensões e expressões geradoras são otimizados desta forma. Nota: a maioria das otimizações de interpretador são aplicadas a todos os escopos, apenas aquelas que dependem de um conjunto conhecido de nomes de variáveis locais e não locais são restritas a escopos otimizados.

módulo opcional

Um módulo de extensão que faz parte da biblioteca padrão, mas pode estar ausente em algumas versões do CPython, geralmente devido à falta de bibliotecas de terceiros ou à não disponibilidade em determinada plataforma.

Consulte Requisitos para módulos opcionais para obter uma lista de módulos opcionais que exigem bibliotecas de terceiros.

pacote

Um módulo Python é capaz de conter submódulos ou recursivamente, subpacotes. Tecnicamente, um pacote é um módulo Python com um atributo `__path__`.

Veja também pacote regular e pacote de espaço de nomes.

paralelismo

Executar várias operações simultaneamente (por exemplo, em vários núcleos de CPU). Em compilações Python com a trava global do interpretador, apenas um segmento executa o bytecode Python de cada vez, portanto, aproveitar vários núcleos de CPU normalmente envolve vários processos (por exemplo, **`multiprocessing`**) ou extensões nativas que liberam a trava global. No Python com threads livres, várias threads podem executar código simultaneamente em diferentes núcleos.

parâmetro

Uma entidade nomeada na definição de uma função (ou método) que específica um argumento (ou em alguns casos, argumentos) que a função pode receber. Existem cinco tipos de parâmetros:

- *posicional-ou-nomeado*: especifica um argumento que pode ser tanto posicional quanto nomeado. Esse é o tipo padrão de parâmetro, por exemplo *foo* e *bar* a seguir:

  ```
  def func(foo, bar=None): ...
  ```

- *somente-posicional*: especifica um argumento que pode ser fornecido apenas por posição. Parâmetros somente-posicionais podem ser definidos incluindo o caractere `/` na lista de parâmetros da definição da função após eles, por exemplo *somentepos1* e *somentepos2* a seguir:

  ```
  def func(somentepos1, somentepos2, /, posicional_ou_nome
  ```

- *somente-nomeado*: especifica um argumento que pode ser passado para a função somente por nome. Parâmetros somente-nomeados podem ser definidos com um simples parâmetro var-posicional ou um `*` antes deles na lista de parâmetros na definição da função, por exemplo *somente_nom1* and *somente_nom2* a seguir:

  ```
  def func(arg, *, somente_nom1, somente_nom2): ...
  ```

- *var-posicional*: especifica que uma sequência arbitrária de argumentos posicionais pode ser fornecida (em adição a qualquer argumento posicional já aceito por outros parâmetros). Tal parâmetro pode ser definido colocando um `*` antes do nome do parâmetro, por exemplo *args* a seguir:

  ```
  def func(*args, **kwargs): ...
  ```

- *var-nomeado*: especifica que, arbitrariamente, muitos argumentos nomeados podem ser fornecidos (em adição a quaisquer argumentos nomeados já aceitos por outros parâmetros). Tal parâmetro pode ser definido colocando-se `**` antes do nome, por exemplo, *kwargs* no exemplo acima.

Parâmetros podem especificar tanto argumentos opcionais quanto obrigatórios, assim como valores padrão para alguns argumentos opcionais.

Veja também o termo argumento no glossário, a pergunta do FAQ sobre a diferença entre argumentos e parâmetros, a classe `inspect.Parameter`, a seção Definições de função e a **PEP 362** [https://peps.python.org/pep-0362/].

trava por objeto

Uma trava associada à instância de um objeto, em vez de uma trava global compartilhada por todos os objetos. No Python com threads livres, tipos embutidos como `dict` e `list` usam a trava por objeto para permitir operações simultâneas em diferentes objetos, enquanto serializam as operações no mesmo objeto. As operações que mantêm o trava por objeto impedem que outras operações com travas sejam executadas no mesmo objeto, mas não bloqueiam operações sem travas.

entrada de caminho

Um local único no caminho de importação que o localizador baseado no caminho consulta para encontrar módulos a serem importados.

localizador de entrada de caminho

Um localizador retornado por um chamável em `sys.path_hooks` (ou seja, um gancho de entrada de caminho) que sabe como localizar os módulos entrada de caminho.

Veja `importlib.abc.PathEntryFinder` para os métodos que localizadores de entrada de caminho implementam.

gancho de entrada de caminho

Um chamável na lista `sys.path_hooks` que retorna um localizador de entrada de caminho caso saiba como localizar módulos em uma entrada de caminho específica.

localizador baseado no caminho

Um dos localizadores de metacaminho padrão que procura por um caminho de importação de módulos.

objeto caminho ou similar

Um objeto representando um caminho de sistema de arquivos. Um objeto caminho ou similar é ou um objeto `str` ou `bytes` representando um caminho, ou um objeto implementando o protocolo `os.PathLike`. Um objeto que suporta o protocolo `os.PathLike` pode ser convertido para um arquivo de caminho do sistema `str` ou `bytes`, através da chamada da função `os.fspath()`; `os.fsdecode()` e `os.fsencode()` podem ser usadas para garantir um `str` ou `bytes` como resultado, respectivamente. Introduzido na **PEP 519** [https://peps.python.org/pep-0519/].

PEP

Proposta de melhoria do Python. Uma PEP é um documento de design que fornece informação para a comunidade Python, ou descreve uma nova funcionalidade para o Python ou seus predecessores ou ambientes. PEPs devem prover uma especificação técnica concisa e um racional para funcionalidades propostas.

PEPs têm a intenção de ser os mecanismos primários para propor novas funcionalidades significativas, para coletar opiniões da comunidade sobre um problema, e para documentar as decisões de design que foram adicionadas ao Python. O autor da PEP é responsável por construir um consenso dentro da comunidade e documentar opiniões dissidentes.

Veja **PEP 1** [https://peps.python.org/pep-0001/].

porção

Um conjunto de arquivos em um único diretório (possivelmente armazenado em um arquivo zip) que contribuem para um pacote de espaço de nomes, conforme definido em **PEP 420** [https://peps.python.org/pep-0420/].

argumento posicional

Veja argumento.

API provisória

Uma API provisória é uma API que foi deliberadamente excluída das bibliotecas padrões com compatibilidade retroativa garantida. Enquanto mudanças maiores para tais interfaces não são esperadas, contanto que

elas sejam marcadas como provisórias, mudanças retroativas incompatíveis (até e incluindo a remoção da interface) podem ocorrer se consideradas necessárias pelos desenvolvedores principais. Tais mudanças não serão feitas gratuitamente – elas irão ocorrer apenas se sérias falhas fundamentais forem descobertas, que foram esquecidas anteriormente a inclusão da API.

Mesmo para APIs provisórias, mudanças retroativas incompatíveis são vistas como uma "solução em último caso" - cada tentativa ainda será feita para encontrar uma resolução retroativa compatível para quaisquer problemas encontrados.

Esse processo permite que a biblioteca padrão continue a evoluir com o passar do tempo, sem se prender em erros de design problemáticos por períodos de tempo prolongados. Veja **PEP 411** [https://peps.python.org/pep-0411/] para mais detalhes.

pacote provisório

Veja API provisória.

Python 3000

Apelido para a linha de lançamento da versão do Python 3.x (cunhada há muito tempo, quando o lançamento da versão 3 era algo em um futuro muito distante.) Esse termo possui a seguinte abreviação: "Py3k".

Pythônico

Uma ideia ou um pedaço de código que segue de perto as formas de escritas mais comuns da linguagem Python, ao invés de implementar códigos usando conceitos comuns a outras linguagens. Por exemplo, um formato comum em Python é fazer um laço sobre todos os elementos de uma iterável usando a instrução `for`. Muitas outras linguagens não têm esse tipo de construção, então as pessoas que não estão familiarizadas com o Python usam um contador numérico:

```
for i in range(len(comida)):
    print(comida[i])
```

Ao contrário do método mais limpo, Pythônico:

```
for parte in comida:
    print(parte)
```

nome qualificado

Um nome pontilhado (quando 2 termos são ligados por um ponto) que mostra o “path” do escopo global de um módulo para uma classe, função ou método definido num determinado módulo, conforme definido pela **PEP 3155** [https://peps.python.org/pep-3155/]. Para funções e classes de nível superior, o nome qualificado é o mesmo que o nome do objeto:

```
>>> class C:
...     class D:
...         def metodo(self):
...             pass
...
>>> C.__qualname__
'C'
>>> C.D.__qualname__
'C.D'
>>> C.D.metodo.__qualname__
'C.D.metodo'
```

Quando usado para se referir a módulos, o *nome totalmente qualificado* significa todo o caminho pontilhado para o módulo, incluindo quaisquer pacotes pai, por exemplo: `email.mime.text`:

```
>>> import email.mime.text
>>> email.mime.text.__name__
'email.mime.text'
```

condição de corrida

Condição de um programa em que o seu comportamento depende do tempo relativo ou da ordem dos eventos, particularmente em programas multithread. As condições de corrida podem levar a um comportamento não determinístico e a bugs difíceis de reproduzir. Uma corrida de dados é um tipo específico de condição de corrida que envolve acesso não sincronizado a uma memória compartilhada. O estilo de codificação LBYL é particularmente suscetível a condições de corrida em código

multithread. O uso de travas e de outras primitivas de sincronização ajuda a evitar condições de corrida.

contagem de referências

O número de referências a um objeto. Quando a contagem de referências de um objeto cai para zero, ele é desalocado. Alguns objetos são imortais e têm contagens de referências que nunca são modificadas e, portanto, os objetos nunca são desalocados. A contagem de referências geralmente não é visível para o código Python, mas é um elemento-chave da implementação do CPython. Os programadores podem chamar a função `sys.getrefcount()` para retornar a contagem de referências para um objeto específico.

Em CPython, as contagens de referências não são consideradas valores estáveis ou bem definidos; o número de referências a um objeto e como esse número é afetado pelo código Python pode ser diferente entre as versões.

pacote regular

Um pacote tradicional, como um diretório contendo um arquivo `__init__.py`.

Veja também pacote de espaço de nomes.

reentrante

Uma propriedade de uma função ou trava que permite que ela seja chamada ou adquirida várias vezes pela mesma thread sem causar erros ou um impasse.

Para funções, reentrância significa que a função pode ser chamada novamente com segurança antes que uma invocação anterior tenha sido concluída, o que é importante quando as funções podem ser chamadas recursivamente ou a partir de manipuladores de sinal. Funções não seguras para threads podem ser não determinísticas se forem chamadas de forma reentrante em um programa multithreaded.

Para travas, o `threading.RLock` (bloqueio reentrante) do Python é reentrante, o que significa que uma thread que já possui a trava pode adquiri-la novamente sem bloquear. Em contrapartida, o `threading.Lock` não é reentrante: tentar adquiri-lo duas vezes na mesma thread causará um impasse.

Consulte também trava e impasse.

REPL

Um acrônimo para “read–eval–print loop”, outro nome para o console interativo do interpretador.

__slots__

Uma declaração dentro de uma classe que economiza memória pré-declarando espaço para atributos de instâncias, e eliminando dicionários de instâncias. Apesar de popular, a técnica é um tanto quanto complicada de acertar, e é melhor se for reservada para casos raros, onde existe uma grande quantidade de instâncias em uma aplicação onde a memória é crítica.

sequência

Um iterável com suporte para acesso eficiente a seus elementos através de índices inteiros via método especial `__getitem__()` e que define o método `__len__()` que devolve o tamanho da sequência. Alguns tipos de sequência embutidos são: `list`, `str`, `tuple`, e `bytes`. Note que `dict` também tem suporte para `__getitem__()` e `__len__()`, mas é considerado um mapeamento e não uma sequência porque a busca usa uma chave hasheável arbitrária em vez de inteiros.

A classe base abstrata `collections.abc.Sequence` define uma interface mais rica que vai além de apenas `__getitem__()` e `__len__()`, adicionando `count()`, `index()`, `__contains__()`, e `__reversed__()`. Tipos que implementam essa interface podem ser explicitamente registrados usando `register()`. Para mais documentação sobre métodos de sequências em geral, veja Operações comuns de sequências.

compreensão de conjunto

Uma maneira compacta de processar todos ou parte dos elementos em iterável e retornar um conjunto com os resultados. `results = {c for c in 'abracadabra' if c not in 'abc'}` gera um conjunto de strings `{'r', 'd'}`. Veja Compreensões.

despacho único

Uma forma de despacho de função genérica onde a implementação é escolhida com base no tipo de um único argumento.

fatia

Um objeto do tipo **`slice`**, usado para descrever uma parte de uma sequência. Um objeto fatia é criado ao usar a forma fatiamento da notação de subscrito, com dois pontos entre colchetes, como em `nome_variavel[1:3:5]`.

suavemente descontinuado

Uma API suavemente descontinuada não deve ser usada em código novo, mas é seguro para código já existente usá-la. A API continua documentada e testada, mas não será melhorada.

A descontinuação suave, diferentemente da descontinuação normal, não planeja remover a API e não emitirá avisos.

Veja PEP 387: Descontinuação suave [https://peps.python.org/pep-0387/#soft-deprecation].

método especial

Um método que é chamado implicitamente pelo Python para executar uma certa operação em um tipo, como uma adição por exemplo. Tais métodos tem nomes iniciando e terminando com dois underscores. Métodos especiais estão documentados em Nomes de métodos especiais.

biblioteca padrão

A coleção de pacotes, módulos e módulos de extensão distribuída como parte do pacote do interpretador Python oficial. A relação exata de

elementos da coleção pode variar de acordo com a plataforma, bibliotecas de sistema disponíveis ou outros critérios. A documentação pode ser encontrada em A biblioteca padrão do Python.

Veja também `sys.stdlib_module_names` para obter uma lista de todos os possíveis nomes de módulos da biblioteca padrão.

instrução

Uma instrução é parte de uma suíte (um "bloco" de código). Uma instrução é ou uma expressão ou uma de várias construções com uma palavra reservada, tal como `if`, `while` ou `for`.

verificador de tipo estático

Uma ferramenta externa que lê código Python e o analisa, procurando por problemas como tipos incorretos. Consulte também dicas de tipo e o módulo `typing`.

stdlib

Uma abreviação para o termo biblioteca padrão em inglês (standard library).

roubar

Na API C do Python, "*roubar*" um argumento significa que a propriedade do argumento é transferida para a função chamada. O chamador da função não deve utilizar essa referência após a chamada. Geralmente, funções que "roubam" um argumento o fazem mesmo que falhem.

Veja Detalhes da contagem de referências para uma explicação completa.

referência forte

Na API C do Python, uma referência forte é uma referência a um objeto que pertence ao código que contém a referência. A referência forte é obtida chamando `Py_INCREF()` quando a referência é criada e liberada com `Py_DECREF()` quando a referência é excluída.

A função `Py_NewRef()` pode ser usada para criar uma referência forte para um objeto. Normalmente, a função `Py_DECREF()` deve ser chamada na referência forte antes de sair do escopo da referência forte, para evitar o vazamento de uma referência.

Veja também referência emprestada.

subscrito

A expressão entre colchetes de uma expressão de subscrição, por exemplo, o `3` em `items[3]`. Normalmente utilizada para selecionar um elemento de um contêiner. Também chamada de chave quando se subscreve um mapeamento, ou um índice quando se subscreve uma sequência.

primitiva de sincronização

Um bloco de construção básico para coordenar (sincronizar) a execução de múltiplas threads, garantindo acesso seguro para threads a recursos compartilhados. O módulo `threading` do Python fornece várias primitivas de sincronização, incluindo `Lock`, `RLock`, `Semaphore`, `Condition`, `Event` e `Barrier`. Além disso, o módulo `queue` fornece filas com vários produtores e consumidores, especialmente úteis em programas multithread. Estas primitivas ajudam a evitar condições de corrida e a coordenar a execução de threads. Veja também trava.

t-string
t-strings

Literais de string prefixados com `t` ou `T` são comumente chamados de "t-strings", que é uma abreviação de literais de strings templates.

codificação do texto

Uma string em Python é uma sequência de pontos de código Unicode (no intervalo `U+0000`–`U+10FFFF`). Para armazenar ou transferir uma string, ela precisa ser serializada como uma sequência de bytes.

A serialização de uma string em uma sequência de bytes é conhecida como "codificação" e a recriação da string a partir de uma sequência de

bytes é conhecida como “decodificação”.

Há uma variedade de diferentes serializações de texto codecs, que são coletivamente chamadas de “codificações do texto”.

arquivo texto

Um objeto arquivo apto a ler e escrever objetos `str`. Geralmente, um arquivo texto, na verdade, acessa um fluxo de dados de bytes e captura a codificação do texto automaticamente. Exemplos de arquivos texto são: arquivos abertos em modo texto (`'r'` or `'w'`), `sys.stdin`, `sys.stdout`, e instâncias de `io.StringIO`.

Veja também arquivo binário para um objeto arquivo apto a ler e escrever objetos bytes ou similares.

estado de thread

As informações usadas pelo ambiente de execução do CPython para executar em uma thread do sistema operacional. Por exemplo, isso inclui a exceção atual, se houver, e o estado do interpretador de bytecode.

Cada estado de thread está vinculado a uma única thread do SO, mas as threads podem ter muitos estados de thread disponíveis. No máximo, um deles pode ser anexado por vez.

Um estado de thread anexado é necessário para chamar a maioria da API C do Python, a menos que uma função documente explicitamente o contrário. O interpretador de bytecode só é executado em um estado de thread anexado.

Cada estado de thread pertence a um único interpretador, mas cada interpretador pode ter vários estados de thread, incluindo vários para a mesma thread do sistema operacional. Estados de thread de vários interpretadores podem ser vinculados à mesma thread, mas apenas um pode ser anexado nessa thread a qualquer momento.

Veja Estado de thread e a trava global do interpretador para mais informações.

seguro para thread

Um módulo, função ou classe que se comporta corretamente quando usado por várias threads simultaneamente. O código seguro para threads usa primitivas de sincronização apropriadas, como travas, para proteger o estado mutável compartilhado, ou é projetado para evitar completamente o estado mutável compartilhado. Na construção com threads livres, tipos embutidos como `dict`, `list` e `set` utilizam travas internas para tornar muitas operações seguras para threads, embora a segurança para threads não seja necessariamente garantida. O código que não é seguro para threads pode sofrer condições de corrida e corridas de dados quando usado em programas multithreads.

token

Uma pequena unidade de código-fonte, gerada pelo analisador léxico (também chamado de *tokenizador*). Nomes, números, strings, operadores, quebras de linha e similares são representados por tokens.

O módulo `tokenize` expõe o analisador léxico do Python. O módulo `token` contém informações sobre os vários tipos de tokens.

aspas triplas

Uma string que está definida com três ocorrências de aspas duplas (”) ou apóstrofos (‘). Enquanto elas não fornecem nenhuma funcionalidade não disponível com strings de aspas simples, elas são úteis para inúmeras razões. Elas permitem que você inclua aspas simples e duplas não escapadas dentro de uma string, e elas podem utilizar múltiplas linhas sem o uso de caractere de continuação, fazendo-as especialmente úteis quando escrevemos documentação em docstrings.

tipo

O tipo de um objeto Python determina qual classe de objeto ele é; cada objeto tem um tipo. Um tipo de objeto é acessível pelo atributo `__class__` ou pode ser recuperado com `type(obj)`.

apelido de tipo

Um sinônimo para um tipo, criado através da atribuição do tipo para um identificador.

Apelidos de tipo são úteis para simplificar dicas de tipo. Por exemplo:

```
def remove_tons_de_cinza(
        cores: list[tuple[int, int, int]]) -> list[tuple[int,
    pass
```

pode tornar-se mais legível desta forma:

```
Cor = tuple[int, int, int]

def remove_tons_de_cinza(cores: list[Cor]) -> list[Cor]:
    pass
```

Veja `typing` e **PEP 484** [https://peps.python.org/pep-0484/], que descreve esta funcionalidade.

dica de tipo

Uma anotação que especifica o tipo esperado de uma variável, de um atributo de classe, de um parâmetro ou de um valor de retorno de uma função.

Dicas de tipo são opcionais e não são forçadas pelo Python, mas são úteis para verificadores de tipo estático. Eles também ajudam IDEs a completar e refatorar código.

Dicas de tipos de variáveis globais, atributos de classes e funções, mas não de variáveis locais, podem ser acessadas usando `typing.get_type_hints()`.

Veja `typing` e **PEP 484** [https://peps.python.org/pep-0484/], que descreve esta funcionalidade.

novas linhas universais

Uma maneira de interpretar fluxos de textos, na qual todos estes são reconhecidos como caracteres de fim de linha: a convenção para fim de

linha no Unix `'\n'`, a convenção no Windows `'\r\n'`, e a antiga convenção no Macintosh `'\r'`. Veja **PEP 278** [https://peps.python.org/pep-0278/] e **PEP 3116** [https://peps.python.org/pep-3116/], bem como **`bytes.splitlines()`** para uso adicional.

anotação de variável

Uma anotação de uma variável ou um atributo de classe.

Ao fazer uma anotação de uma variável ou um atributo de classe, a atribuição é opcional:

```
class C:
    campo: 'anotação'
```

Anotações de variáveis são normalmente usadas para dicas de tipo: por exemplo, espera-se que esta variável receba valores do tipo **`int`**:

```
contagem: int = 0
```

A sintaxe de anotação de variável é explicada na seção instruções de atribuição anotado.

Veja anotação de função, **PEP 484** [https://peps.python.org/pep-0484/] e **PEP 526** [https://peps.python.org/pep-0526/], que descrevem esta funcionalidade. Veja também Boas práticas para anotações para as melhores práticas sobre como trabalhar com anotações.

ambiente virtual

Um ambiente de execução isolado que permite usuários Python e aplicações instalarem e atualizarem pacotes Python sem interferir no comportamento de outras aplicações Python em execução no mesmo sistema.

Veja também **`venv`**.

máquina virtual

Um computador definido inteiramente em software. A máquina virtual de Python executa o bytecode emitido pelo compilador de bytecode.

operador morsa

Uma maneira bem-humorada de se referir ao operador da expressão de atribuição `:=` porque ele se parece um pouco com uma morsa se você virar a cabeça.

Zen do Python

Lista de princípios de projeto e filosofias do Python que são úteis para a compreensão e uso da linguagem. A lista é exibida quando se digita "`import this`" no console interativo.

Sobre esta documentação

A documentação do Python é gerada a partir de fontes reStructuredText [https://docutils.sourceforge.io/rst.html] usando Sphinx [https://www.sphinx-doc.org/], um gerador de documentação criado originalmente para Python e agora mantido como um projeto independente.

O desenvolvimento da documentação e de suas ferramentas é um esforço totalmente voluntário, como Python em si. Se você quer contribuir, por favor dê uma olhada na página Lidando com bugs para informações sobre como fazer. Novos voluntários são sempre bem-vindos!

Agradecimentos especiais para:

- Fred L. Drake, Jr., o criador do primeiro conjunto de ferramentas para documentar Python e autor de boa parte do conteúdo;
- O projeto Docutils [https://docutils.sourceforge.io/] por criar reStructuredText e o pacote Docutils;
- Fredrik Lundh, pelo seu projeto de referência alternativa em Python, do qual Sphinx pegou muitas boas ideias.

Contribuidores da documentação do Python

Muitas pessoas têm contribuído para a linguagem Python, sua biblioteca padrão e sua documentação. Consulte o repositório CPython [https://github.com/python/cpython/graphs/contributors] Para a lista parcial de contribuidores.

Tudo isso só foi possível com o esforço e a contribuição da comunidade Python, por isso temos essa maravilhosa documentação – Obrigado a todos!

Lidando com bugs

O Python é uma linguagem de programação madura que estabeleceu uma reputação pela estabilidade. A fim de manter esta reputação os desenvolvedores gostariam de saber de quaisquer deficiências que você encontre no Python.

Por vezes, pode ser mais rápido corrigir os bugs pessoalmente e contribuir com correções (patches) para o Python, uma vez que simplifica o processo e envolve menos pessoas. Saiba como contribuir.

Bugs na documentação

Se você achar um bug nesta documentação ou gostaria de propor uma melhoria, por favor submeta um relatório no rastreador de problemas. Se você tiver uma sugestão sobre como consertá-lo inclua isso também.

Se o bug ou a melhoria sugerida estiver relacionado à tradução desta documentação, envie o relatório para o repositório de tradução [https://github.com/python/python-docs-pt-br].

Você também pode abrir um item de discussão em nosso fórum Discourse de documentação [https://discuss.python.org/c/documentation/26].

Se você encontrar um bug no tema (HTML/CSS/JavaScript) da documentação, envie um relatório de bug no rastreador de problemas do python-doc-theme [https://github.com/python/python-docs-theme].

Ver também

Bugs na documentação
Uma lista de bugs na documentação que foram submetidos no rastreador de problemas do Python.

Rastreando problemas **[https://devguide.python.org/tracker/]**

Visão geral do processo envolvido em reportar uma melhoria no rastreador.

Ajudando com a documentação **[https://devguide.python.org/docquality/#helping-with-documentation]**

Guia compreensivo para os indivíduos que estão interessados em contribuir com a documentação do Python.

Traduções da documentação **[https://devguide.python.org/documentation/translations/translating/#translation-coordinators]**

Uma lista de páginas no GitHub relacionadas à tradução da documentação e respectivas equipes de coordenação.

Usando o rastreador de problemas do Python

Relatórios de problemas do Python em si devem ser submetidos ao rastreador de problemas do GitHub (https://github.com/python/cpython/issues). O rastreador de problemas do GitHub oferece um formulário web que permite informações pertinentes serem enviadas para os desenvolvedores.

O primeiro passo para preencher um relatório é determinar se o problema já foi reportado. A vantagem em fazê-lo além de poupar o tempo dos desenvolvedores é que você aprende o que foi feito para consertá-lo; pode ser que o problema já foi consertado para o próximo lançamento, ou que informação adicional seja necessária (nesse caso, você é bem-vindo para fornecê-la se você puder!). Para fazer isso, procure no rastreador usando a caixa de busca no topo da página.

Se o problema que você está relatando ainda não estiver na lista, faça login no GitHub. Se você ainda não tiver uma conta do GitHub, crie uma nova conta usando o link “Sign Up”. Não é possível enviar um relatório de bug anonimamente.

Estando agora autenticado, você pode submeter um relatório de problema. Clique no botão “New issue” na barra superior para relatar um novo problema.

O formulário de submissão tem dois campos, “Title” e “Comment”.

Para o campo “Title”, insira uma descrição *muito* curta do problema; menos de dez palavras é bom.

No campo “Comment”, descreva o problema com detalhes, incluindo o que você esperava que acontecesse e o que aconteceu. Certifique-se de incluir se quaisquer módulos de extensão estavam envolvidos, e qual hardware e plataforma de software você está usando (incluindo informações sobre a versão quando apropriado).

Cada relatório de problema será revisado por um desenvolvedor que irá determinar o que é necessário para solucionar o problema. Você receberá atualizações para cada ação tomada para o problema.

Ver também

Como relatar bugs eficientemente **[https://www.chiark.greenend.org.uk/~sgtatham/bugs.html]**

Artigo que entra em algum detalhe sobre como criar um relatório de bug útil. Isto descreve qual tipo de informação é útil e por quê é útil.

Diretrizes para relatar bugs **[https://bugzilla.mozilla.org/page.cgi?id=bug-writing.html]**

Informação sobre como escrever um bom relatório de bug. Um pouco disto é específico do Projeto Mozilla, mas descreve boas práticas em geral.

Começando a contribuir com o Python por conta própria

Além de apenas reportar os bugs que você encontrar, você também é bem-vindo para submeter patches para consertá-los. Você pode achar mais informações sobre como começar a contribuir com patches para o Python no Python Developer's Guide [https://devguide.python.org/]. Se você tem mais perguntas, a lista de discussão core-mentorship [https://mail.python.org/mailman3/lists/core-mentorship.python.org/] é um lugar amigável para obter respostas para qualquer questão relativa ao processo de resolução de problemas no Python.

Direitos autorais

Python e essa documentação é:

Copyright © 2001 Python Software Foundation. Todos os direitos reservados.

Copyright © 2000 BeOpen.com. Todos os direitos reservados.

Copyright © 1995-2000 Corporation for National Research Initiatives. Todos os direitos reservados.

Copyright © 1991-1995 Stichting Mathematisch Centrum. Todos os direitos reservados.

Veja: História e Licença para informações completas de licença e permissões.

História e Licença

História do software

Python foi criado no início dos anos 1990 por Guido van Rossum no Stichting Mathematisch Centrum (CWI, veja https://www.cwi.nl) na Holanda como sucessor de uma linguagem chamada ABC. Guido continua sendo o principal autor do Python, embora inclua muitas contribuições de outros.

Em 1995, Guido continuou seu trabalho em Python na Corporation for National Research Initiatives (CNRI, veja https://www.cnri.reston.va.us) em Reston, Virgínia, onde lançou várias versões do software.

Em maio de 2000, Guido e a equipe de desenvolvimento do núcleo Python mudaram-se para BeOpen.com para formar a equipe BeOpen PythonLabs. Em outubro do mesmo ano, a equipe PythonLabs mudou-se para a Digital Creations, que se tornou Zope Corporation. Em 2001, a Python Software Foundation (PSF, veja https://www.python.org/psf/) foi formada, uma organização sem fins lucrativos criada especificamente para possuir Propriedade Intelectual relacionada ao Python. A Zope Corporation era um membro patrocinador da PSF.

Todas as versões do Python são de código aberto (consulte https://opensource.org para a definição de código aberto). Historicamente, a maioria, mas não todas, versões do Python também são compatíveis com GPL; a tabela abaixo resume os vários lançamentos.

Versão	Derivada de	Ano	Proprietário	Compatível com a GPL? (1)

Versão	Derivada de	Ano	Proprietário	Compatível com a GPL? (1)
0.9.0 a 1.2	n/a	1991-1995	CWI	sim
1.3 a 1.5.2	1.2	1995-1999	CNRI	sim
1.6	1.5.2	2000	CNRI	não
2.0	1.6	2000	BeOpen.com	não
1.6.1	1.6	2001	CNRI	sim (2)
2.1	2.0+1.6.1	2001	PSF	não
2.0.1	2.0+1.6.1	2001	PSF	sim
2.1.1	2.1+2.0.1	2001	PSF	sim
2.1.2	2.1.1	2002	PSF	sim
2.1.3	2.1.2	2002	PSF	sim

Versão	Derivada de	Ano	Proprietário	Compatível com a GPL? (1)
2.2 e acima	2.1.1	2001-agora	PSF	sim

Nota

1. Compatível com a GPL não significa que estamos distribuindo Python sob a GPL. Todas as licenças do Python, ao contrário da GPL, permitem distribuir uma versão modificada sem fazer alterações em código aberto. As licenças compatíveis com a GPL possibilitam combinar o Python com outro software lançado sob a GPL; os outros não.
2. De acordo com Richard Stallman, 1.6.1 não é compatível com GPL, porque sua licença tem uma cláusula de escolha de lei. De acordo com a CNRI, no entanto, o advogado de Stallman disse ao advogado da CNRI que 1.6.1 “não é incompatível” com a GPL.

Graças aos muitos voluntários externos que trabalharam sob a direção de Guido para tornar esses lançamentos possíveis.

Termos e condições para acessar ou usar Python

O software e a documentação do Python são licenciados sob a Python Software Foundation License Versão 2.

A partir do Python 3.8.6, exemplos, receitas e outros códigos na documentação são licenciados duplamente sob o Licença PSF versão 2 e a Licença BSD de Zero Cláusula.

Alguns softwares incorporados ao Python estão sob licenças diferentes. As licenças são listadas com o código abrangido por essa licença. Veja

Licenças e Reconhecimentos para Software Incorporado para uma lista incompleta dessas licenças.

PYTHON SOFTWARE FOUNDATION LICENSE VERSION 2

1. This LICENSE AGREEMENT is between the Python Software Foundat
   the Individual or Organization ("Licensee") accessing and oth
   software ("Python") in source or binary form and its associat

2. Subject to the terms and conditions of this License Agreement
   grants Licensee a nonexclusive, royalty-free, world-wide lice
   analyze, test, perform and/or display publicly, prepare deriv
   distribute, and otherwise use Python alone or in any derivati
   version, provided, however, that PSF's License Agreement and
   copyright, i.e., "Copyright © 2001 Python Software Foundatior
   Reserved" are retained in Python alone or in any derivative v
   prepared by Licensee.

3. In the event Licensee prepares a derivative work that is base
   incorporates Python or any part thereof, and wants to make th
   derivative work available to others as provided herein, then
   agrees to include in any such work a brief summary of the cha

4. PSF is making Python available to Licensee on an "AS IS" basi
   PSF MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIE
   EXAMPLE, BUT NOT LIMITATION, PSF MAKES NO AND DISCLAIMS ANY R
   WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PUR
   USE OF PYTHON WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. PSF SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PYT
   FOR ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS
   MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON, OR ANY DE
   THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a ma
   its terms and conditions.

7. Nothing in this License Agreement shall be deemed to create a
   of agency, partnership, or joint venture between PSF and Lice
   Agreement does not grant permission to use PSF trademarks or

trademark sense to endorse or promote products or services of third party.

8. By copying, installing or otherwise using Python, Licensee ag to be bound by the terms and conditions of this License Agree

ACORDO DE LICENCIAMENTO DA BEOPEN.COM PARA PYTHON 2.0

ACORDO DE LICENCIAMENTO DA BEOPEN DE FONTE ABERTA DO PYTHON VERSÃO 1

1. This LICENSE AGREEMENT is between BeOpen.com ("BeOpen"), havi 160 Saratoga Avenue, Santa Clara, CA 95051, and the Individua ("Licensee") accessing and otherwise using this software in s form and its associated documentation ("the Software").

2. Subject to the terms and conditions of this BeOpen Python Lic BeOpen hereby grants Licensee a non-exclusive, royalty-free, to reproduce, analyze, test, perform and/or display publicly, works, distribute, and otherwise use the Software alone or in version, provided, however, that the BeOpen Python License is Software, alone or in any derivative version prepared by Lice

3. BeOpen is making the Software available to Licensee on an "AS BEOPEN MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMP EXAMPLE, BUT NOT LIMITATION, BEOPEN MAKES NO AND DISCLAIMS AN WARRANTY OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PUR USE OF THE SOFTWARE WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

4. BEOPEN SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS MODIFYING OR DISTRIBUTING THE SOFTWARE, OR ANY DERIVATIVE THE ADVISED OF THE POSSIBILITY THEREOF.

5. This License Agreement will automatically terminate upon a ma its terms and conditions.

6. This License Agreement shall be governed by and interpreted i by the law of the State of California, excluding conflict of Nothing in this License Agreement shall be deemed to create a

agency, partnership, or joint venture between BeOpen and Lice
Agreement does not grant permission to use BeOpen trademarks
trademark sense to endorse or promote products or services of
third party. As an exception, the "BeOpen Python" logos avai
http://www.pythonlabs.com/logos.html may be used according to
granted on that web page.

7. By copying, installing or otherwise using the software, Licer
bound by the terms and conditions of this License Agreement.

CONTRATO DE LICENÇA DA CNRI PARA O PYTHON 1.6.1

1. This LICENSE AGREEMENT is between the Corporation for Nationa
Initiatives, having an office at 1895 Preston White Drive, Re
("CNRI"), and the Individual or Organization ("Licensee") acc
otherwise using Python 1.6.1 software in source or binary for
associated documentation.

2. Subject to the terms and conditions of this License Agreement
grants Licensee a nonexclusive, royalty-free, world-wide lice
analyze, test, perform and/or display publicly, prepare deriv
distribute, and otherwise use Python 1.6.1 alone or in any de
provided, however, that CNRI's License Agreement and CNRI's r
i.e., "Copyright © 1995-2001 Corporation for National Researc
Rights Reserved" are retained in Python 1.6.1 alone or in any
prepared by Licensee. Alternately, in lieu of CNRI's License
Licensee may substitute the following text (omitting the quot
is made available subject to the terms and conditions in CNRI
Agreement. This Agreement together with Python 1.6.1 may be
internet using the following unique, persistent identifier (k
1895.22/1013. This Agreement may also be obtained from a pro
internet using the following URL: http://hdl.handle.net/1895.

3. In the event Licensee prepares a derivative work that is base
incorporates Python 1.6.1 or any part thereof, and wants to m
work available to others as provided herein, then Licensee he
include in any such work a brief summary of the changes made

4. CNRI is making Python 1.6.1 available to Licensee on an "AS I
MAKES NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED.

BUT NOT LIMITATION, CNRI MAKES NO AND DISCLAIMS ANY REPRESENT
OF MERCHANTABILITY OR FITNESS FOR ANY PARTICULAR PURPOSE OR T
PYTHON 1.6.1 WILL NOT INFRINGE ANY THIRD PARTY RIGHTS.

5. CNRI SHALL NOT BE LIABLE TO LICENSEE OR ANY OTHER USERS OF PY
ANY INCIDENTAL, SPECIAL, OR CONSEQUENTIAL DAMAGES OR LOSS AS
MODIFYING, DISTRIBUTING, OR OTHERWISE USING PYTHON 1.6.1, OR
THEREOF, EVEN IF ADVISED OF THE POSSIBILITY THEREOF.

6. This License Agreement will automatically terminate upon a ma
its terms and conditions.

7. This License Agreement shall be governed by the federal intel
law of the United States, including without limitation the fe
law, and, to the extent such U.S. federal law does not apply,
Commonwealth of Virginia, excluding Virginia's conflict of la
Notwithstanding the foregoing, with regard to derivative work
1.6.1 that incorporate non-separable material that was previo
under the GNU General Public License (GPL), the law of the Co
Virginia shall govern this License Agreement only as to issue
with respect to Paragraphs 4, 5, and 7 of this License Agreem
this License Agreement shall be deemed to create any relatior
partnership, or joint venture between CNRI and Licensee. Thi
does not grant permission to use CNRI trademarks or trade nam
sense to endorse or promote products or services of Licensee,
party.

8. By clicking on the "ACCEPT" button where indicated, or by cop
or otherwise using Python 1.6.1, Licensee agrees to be bound
conditions of this License Agreement.

ACORDO DE LICENÇA DA CWI PARA PYTHON 0.9.0 A 1.2

Copyright © 1991 - 1995, Stichting Mathematisch Centrum Amsterda
Netherlands. All rights reserved.

Permission to use, copy, modify, and distribute this software ar
documentation for any purpose and without fee is hereby granted,
the above copyright notice appear in all copies and that both th
notice and this permission notice appear in supporting documenta

```
the name of Stichting Mathematisch Centrum or CWI not be used ir
publicity pertaining to distribution of the software without spe
prior permission.

STICHTING MATHEMATISCH CENTRUM DISCLAIMS ALL WARRANTIES WITH REG
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AN
EVENT SHALL STICHTING MATHEMATISCH CENTRUM BE LIABLE FOR ANY SPE
OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FRO
DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMA
SOFTWARE.
```

ZERO-CLAUSE BSD LICENSE FOR CODE IN THE PYTHON DOCUMENTATION

```
Permission to use, copy, modify, and/or distribute this software
purpose with or without fee is hereby granted.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WA
REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERC
AND FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPEC
INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RES
LOSS OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT,
OTHER TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE
PERFORMANCE OF THIS SOFTWARE.
```

Licenças e Reconhecimentos para Software Incorporado

Esta seção é uma lista incompleta, mas crescente, de licenças e reconhecimentos para softwares de terceiros incorporados na distribuição do Python.

Mersenne Twister

A extensão C `_random` subjacente ao módulo `random` inclui código baseado em um download de http://www.math.sci.hiroshima-u.ac.jp/~m-

mat/MT/MT2002/emt19937ar.html. A seguir estão os comentários literais do código original:

```
A C-program for MT19937, with initialization improved 2002/1/26.
Coded by Takuji Nishimura and Makoto Matsumoto.

Before using, initialize the state by using init_genrand(seed)
or init_by_array(init_key, key_length).

Copyright (C) 1997 - 2002, Makoto Matsumoto and Takuji Nishimura
All rights reserved.

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditic
are met:

  1. Redistributions of source code must retain the above copyrig
     notice, this list of conditions and the following disclaimer

  2. Redistributions in binary form must reproduce the above copy
     notice, this list of conditions and the following disclaimer
     documentation and/or other materials provided with the distr

  3. The names of its contributors may not be used to endorse or
     products derived from this software without specific prior w
     permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUT
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NO
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNES
A PARTICULAR PURPOSE ARE DISCLAIMED.  IN NO EVENT SHALL THE COPY
CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPE
EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED
PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA,
PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THE
LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLU
NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF TH
SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.


Any feedback is very welcome.
```

```
http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/emt.html
email: m-mat @ math.sci.hiroshima-u.ac.jp (remove space)
```

Soquetes

O módulo **`socket`** usa as funções **`getaddrinfo()`** e **`getnameinfo()`**, que são codificadas em arquivos de origem separados do Projeto WIDE, https://www.wide.ad.jp/.

```
Copyright (C) 1995, 1996, 1997, and 1998 WIDE Project.
All rights reserved.

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditic
are met:
1. Redistributions of source code must retain the above copyrigh
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyr
   notice, this list of conditions and the following disclaimer
   documentation and/or other materials provided with the distri
3. Neither the name of the project nor the names of its contribu
   may be used to endorse or promote products derived from this
   without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE PROJECT AND CONTRIBUTORS "AS IS
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICUL
ARE DISCLAIMED.  IN NO EVENT SHALL THE PROJECT OR CONTRIBUTORS E
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUT
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUP
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRA
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING I
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBIL
SUCH DAMAGE.
```

Serviços de soquete assíncrono

Os módulos **`test.support.asynchat`** e **`test.support.asyncore`** contêm o seguinte aviso:

```
Copyright 1996 by Sam Rushing

                         All Rights Reserved

Permission to use, copy, modify, and distribute this software ar
its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of
Rushing not be used in advertising or publicity pertaining to
distribution of the software without specific, written prior
permission.

SAM RUSHING DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS SOFTWAF
INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS,
NO EVENT SHALL SAM RUSHING BE LIABLE FOR ANY SPECIAL, INDIRECT C
CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RESULTING FROM L
OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT,
NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING OUT OF OR IN
CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE.
```

Gerenciamento de cookies

O módulo **`http.cookies`** contém o seguinte aviso:

```
Copyright 2000 by Timothy O'Malley <timo@alum.mit.edu>

                All Rights Reserved

Permission to use, copy, modify, and distribute this software
and its documentation for any purpose and without fee is hereby
granted, provided that the above copyright notice appear in all
copies and that both that copyright notice and this permission
notice appear in supporting documentation, and that the name of
Timothy O'Malley  not be used in advertising or publicity
pertaining to distribution of the software without specific, wri
prior permission.
```

```
Timothy O'Malley DISCLAIMS ALL WARRANTIES WITH REGARD TO THIS
SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILITY
AND FITNESS, IN NO EVENT SHALL Timothy O'Malley BE LIABLE FOR
ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR
PERFORMANCE OF THIS SOFTWARE.
```

Rastreamento de execução

O módulo **trace** contém o seguinte aviso:

```
portions copyright 2001, Autonomous Zones Industries, Inc., all
err...  reserved and offered to the public under the terms of th
Python 2.2 license.
Author: Zooko O'Whielacronx
http://zooko.com/
mailto:zooko@zooko.com

Copyright 2000, Mojam Media, Inc., all rights reserved.
Author: Skip Montanaro

Copyright 1999, Bioreason, Inc., all rights reserved.
Author: Andrew Dalke

Copyright 1995-1997, Automatrix, Inc., all rights reserved.
Author: Skip Montanaro

Copyright 1991-1995, Stichting Mathematisch Centrum, all rights


Permission to use, copy, modify, and distribute this Python soft
its associated documentation for any purpose without fee is here
granted, provided that the above copyright notice appears in all
and that both that copyright notice and this permission notice a
supporting documentation, and that the name of neither Automatri
Bioreason or Mojam Media be used in advertising or publicity per
distribution of the software without specific, written prior per
```

Funções UUencode e UUdecode

O codec uu contém o seguinte aviso:

```
Copyright 1994 by Lance Ellinghouse
Cathedral City, California Republic, United States of America.
                        All Rights Reserved
Permission to use, copy, modify, and distribute this software ar
documentation for any purpose and without fee is hereby granted,
provided that the above copyright notice appear in all copies ar
both that copyright notice and this permission notice appear in
supporting documentation, and that the name of Lance Ellinghouse
not be used in advertising or publicity pertaining to distributi
of the software without specific, written prior permission.
LANCE ELLINGHOUSE DISCLAIMS ALL WARRANTIES WITH REGARD TO
THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANTABILI
FITNESS, IN NO EVENT SHALL LANCE ELLINGHOUSE CENTRUM BE LIABLE
FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR ANY DAMAGE
WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS, WHETHER
ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS ACTION, ARISING
OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF THIS SOFTWARE

Modified by Jack Jansen, CWI, July 1995:
- Use binascii module to do the actual line-by-line conversion
  between ascii and binary. This results in a 1000-fold speedup.
  version is still 5 times faster, though.
- Arguments more compliant with Python standard
```

Chamadas de procedimento remoto XML

O módulo `xmlrpc.client` contém o seguinte aviso:

```
    The XML-RPC client interface is

Copyright (c) 1999-2002 by Secret Labs AB
Copyright (c) 1999-2002 by Fredrik Lundh

By obtaining, using, and/or copying this software and/or its
associated documentation, you agree that you have read, understo
and will comply with the following terms and conditions:
```

Permission to use, copy, modify, and distribute this software ar
its associated documentation for any purpose and without fee is
hereby granted, provided that the above copyright notice appears
all copies, and that both that copyright notice and this permiss
notice appear in supporting documentation, and that the name of
Secret Labs AB or the author not be used in advertising or publi
pertaining to distribution of the software without specific, wri
prior permission.

SECRET LABS AB AND THE AUTHOR DISCLAIMS ALL WARRANTIES WITH REGA
TO THIS SOFTWARE, INCLUDING ALL IMPLIED WARRANTIES OF MERCHANT-
ABILITY AND FITNESS. IN NO EVENT SHALL SECRET LABS AB OR THE AU
BE LIABLE FOR ANY SPECIAL, INDIRECT OR CONSEQUENTIAL DAMAGES OR
DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PROFITS,
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS
ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMA
OF THIS SOFTWARE.

test_epoll

O módulo **`test.test_epoll`** contém o seguinte aviso:

Copyright (c) 2001-2006 Twisted Matrix Laboratories.

Permission is hereby granted, free of charge, to any person obta
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, includ
without limitation the rights to use, copy, modify, merge, publi
distribute, sublicense, and/or sell copies of the Software, and
permit persons to whom the Software is furnished to do so, subje
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLD
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN

```
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNE
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

kqueue de seleção

O módulo `select` contém o seguinte aviso para a interface do kqueue:

```
Copyright (c) 2000 Doug White, 2006 James Knight, 2007 Christiar
All rights reserved.

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditic
are met:
1. Redistributions of source code must retain the above copyrigh
   notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyr
   notice, this list of conditions and the following disclaimer
   documentation and/or other materials provided with the distri

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS "AS IS'
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TC
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICUL
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUT
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUP
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRA
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING I
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBIL
SUCH DAMAGE.
```

SipHash24

O arquivo `Python/pyhash.c` contém a implementação de Marek Majkowski do algoritmo SipHash24 de Dan Bernstein. Contém a seguinte nota:

```
<MIT License>
Copyright (c) 2013  Marek Majkowski <marek@popcount.org>

Permission is hereby granted, free of charge, to any person obta
```

```
of this software and associated documentation files (the "Softwa
in the Software without restriction, including without limitatic
to use, copy, modify, merge, publish, distribute, sublicense, ar
copies of the Software, and to permit persons to whom the Softwa
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be i
all copies or substantial portions of the Software.
</MIT License>

Original location:
   https://github.com/majek/csiphash/

Solution inspired by code from:
   Samuel Neves (supercop/crypto_auth/siphash24/little)
   djb (supercop/crypto_auth/siphash24/little2)
   Jean-Philippe Aumasson (https://131002.net/siphash/siphash24.
```

strtod e dtoa

O arquivo `Python/dtoa.c`, que fornece as funções C dtoa e strtod para conversão de duplas de C para e de strings, é derivado do arquivo com o mesmo nome de David M. Gay, atualmente disponível em https://web.archive.org/web/20220517033456/http://www.netlib.org/fp/dtoa.c. O arquivo original, conforme recuperado em 16 de março de 2009, contém os seguintes avisos de direitos autorais e de licenciamento:

```
/****************************************************************
 *
 * The author of this software is David M. Gay.
 *
 * Copyright (c) 1991, 2000, 2001 by Lucent Technologies.
 *
 * Permission to use, copy, modify, and distribute this software
 * purpose without fee is hereby granted, provided that this ent
 * is included in all copies of any software which is or include
 * or modification of this software and in all copies of the sup
 * documentation for such software.
 *
 * THIS SOFTWARE IS BEING PROVIDED "AS IS", WITHOUT ANY EXPRESS
```

```
 * WARRANTY.  IN PARTICULAR, NEITHER THE AUTHOR NOR LUCENT MAKES
 * REPRESENTATION OR WARRANTY OF ANY KIND CONCERNING THE MERCHAN
 * OF THIS SOFTWARE OR ITS FITNESS FOR ANY PARTICULAR PURPOSE.
 *
 ***************************************************************
```

OpenSSL

Os módulos **hashlib**, **posix** e **ssl** usam a biblioteca OpenSSL para desempenho adicional se forem disponibilizados pelo sistema operacional. Além disso, os instaladores do Windows e do Mac OS X para Python podem incluir uma cópia das bibliotecas do OpenSSL, portanto incluímos uma cópia da licença do OpenSSL aqui: Para o lançamento do OpenSSL 3.0, e lançamentos posteriores derivados deste, se aplica a Apache License v2:

```
                                 Apache License
                           Version 2.0, January 2004
                        https://www.apache.org/licenses/

TERMS AND CONDITIONS FOR USE, REPRODUCTION, AND DISTRIBUTION

1. Definitions.

   "License" shall mean the terms and conditions for use, reproc
   and distribution as defined by Sections 1 through 9 of this c

   "Licensor" shall mean the copyright owner or entity authorize
   the copyright owner that is granting the License.

   "Legal Entity" shall mean the union of the acting entity and
   other entities that control, are controlled by, or are under
   control with that entity. For the purposes of this definitior
   "control" means (i) the power, direct or indirect, to cause t
   direction or management of such entity, whether by contract c
   otherwise, or (ii) ownership of fifty percent (50%) or more c
   outstanding shares, or (iii) beneficial ownership of such ent

   "You" (or "Your") shall mean an individual or Legal Entity
   exercising permissions granted by this License.
```

"Source" form shall mean the preferred form for making modifi including but not limited to software source code, documentat source, and configuration files.

"Object" form shall mean any form resulting from mechanical transformation or translation of a Source form, including but not limited to compiled object code, generated documentation, and conversions to other media types.

"Work" shall mean the work of authorship, whether in Source o Object form, made available under the License, as indicated b copyright notice that is included in or attached to the work (an example is provided in the Appendix below).

"Derivative Works" shall mean any work, whether in Source or form, that is based on (or derived from) the Work and for whi editorial revisions, annotations, elaborations, or other modi represent, as a whole, an original work of authorship. For th of this License, Derivative Works shall not include works tha separable from, or merely link (or bind by name) to the inter the Work and Derivative Works thereof.

"Contribution" shall mean any work of authorship, including the original version of the Work and any modifications or ado to that Work or Derivative Works thereof, that is intentional submitted to Licensor for inclusion in the Work by the copyri or by an individual or Legal Entity authorized to submit on b the copyright owner. For the purposes of this definition, "su means any form of electronic, verbal, or written communicatio to the Licensor or its representatives, including but not lim communication on electronic mailing lists, source code contro and issue tracking systems that are managed by, or on behalf Licensor for the purpose of discussing and improving the Work excluding communication that is conspicuously marked or other designated in writing by the copyright owner as "Not a Contri

"Contributor" shall mean Licensor and any individual or Legal on behalf of whom a Contribution has been received by Licenso subsequently incorporated within the Work.

2. Grant of Copyright License. Subject to the terms and conditio this License, each Contributor hereby grants to You a perpetu

worldwide, non-exclusive, no-charge, royalty-free, irrevocabl
copyright license to reproduce, prepare Derivative Works of,
publicly display, publicly perform, sublicense, and distribut
Work and such Derivative Works in Source or Object form.

3. Grant of Patent License. Subject to the terms and conditions
this License, each Contributor hereby grants to You a perpetu
worldwide, non-exclusive, no-charge, royalty-free, irrevocabl
(except as stated in this section) patent license to make, ha
use, offer to sell, sell, import, and otherwise transfer the
where such license applies only to those patent claims licens
by such Contributor that are necessarily infringed by their
Contribution(s) alone or by combination of their Contributior
with the Work to which such Contribution(s) was submitted. If
institute patent litigation against any entity (including a
cross-claim or counterclaim in a lawsuit) alleging that the W
or a Contribution incorporated within the Work constitutes di
or contributory patent infringement, then any patent licenses
granted to You under this License for that Work shall termina
as of the date such litigation is filed.

4. Redistribution. You may reproduce and distribute copies of th
Work or Derivative Works thereof in any medium, with or witho
modifications, and in Source or Object form, provided that Yo
meet the following conditions:

   (a) You must give any other recipients of the Work or
       Derivative Works a copy of this License; and

   (b) You must cause any modified files to carry prominent noti
       stating that You changed the files; and

   (c) You must retain, in the Source form of any Derivative Wor
       that You distribute, all copyright, patent, trademark, ar
       attribution notices from the Source form of the Work,
       excluding those notices that do not pertain to any part o
       the Derivative Works; and

   (d) If the Work includes a "NOTICE" text file as part of its
       distribution, then any Derivative Works that You distribu
       include a readable copy of the attribution notices contai
       within such NOTICE file, excluding those notices that do
       pertain to any part of the Derivative Works, in at least

of the following places: within a NOTICE text file distri
as part of the Derivative Works; within the Source form c
documentation, if provided along with the Derivative Work
within a display generated by the Derivative Works, if ar
wherever such third-party notices normally appear. The co
of the NOTICE file are for informational purposes only ar
do not modify the License. You may add Your own attributi
notices within Derivative Works that You distribute, alor
or as an addendum to the NOTICE text from the Work, provi
that such additional attribution notices cannot be constr
as modifying the License.

You may add Your own copyright statement to Your modificatior
may provide additional or different license terms and conditi
for use, reproduction, or distribution of Your modifications,
for any such Derivative Works as a whole, provided Your use,
reproduction, and distribution of the Work otherwise complies
the conditions stated in this License.

5. Submission of Contributions. Unless You explicitly state othe
any Contribution intentionally submitted for inclusion in the
by You to the Licensor shall be under the terms and conditior
this License, without any additional terms or conditions.
Notwithstanding the above, nothing herein shall supersede or
the terms of any separate license agreement you may have exec
with Licensor regarding such Contributions.

6. Trademarks. This License does not grant permission to use the
names, trademarks, service marks, or product names of the Lic
except as required for reasonable and customary use in descri
origin of the Work and reproducing the content of the NOTICE

7. Disclaimer of Warranty. Unless required by applicable law or
agreed to in writing, Licensor provides the Work (and each
Contributor provides its Contributions) on an "AS IS" BASIS,
WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express
implied, including, without limitation, any warranties or cor
of TITLE, NON-INFRINGEMENT, MERCHANTABILITY, or FITNESS FOR A
PARTICULAR PURPOSE. You are solely responsible for determinir
appropriateness of using or redistributing the Work and assum
risks associated with Your exercise of permissions under this

8. Limitation of Liability. In no event and under no legal theor

```
   whether in tort (including negligence), contract, or otherwis
   unless required by applicable law (such as deliberate and gro
   negligent acts) or agreed to in writing, shall any Contributo
   liable to You for damages, including any direct, indirect, sp
   incidental, or consequential damages of any character arising
   result of this License or out of the use or inability to use
   Work (including but not limited to damages for loss of goodwi
   work stoppage, computer failure or malfunction, or any and al
   other commercial damages or losses), even if such Contributor
   has been advised of the possibility of such damages.

9. Accepting Warranty or Additional Liability. While redistribut
   the Work or Derivative Works thereof, You may choose to offer
   and charge a fee for, acceptance of support, warranty, indemn
   or other liability obligations and/or rights consistent with
   License. However, in accepting such obligations, You may act
   on Your own behalf and on Your sole responsibility, not on be
   of any other Contributor, and only if You agree to indemnify,
   defend, and hold each Contributor harmless for any liability
   incurred by, or claims asserted against, such Contributor by
   of your accepting any such warranty or additional liability.

END OF TERMS AND CONDITIONS
```

expat

A extensão **pyexpat** é construída usando uma cópia incluída das fontes de expatriadas, a menos que a compilação esteja configurada **--with-system-expat**:

```
Copyright (c) 1998-2000 Thai Open Source Software Center Ltd and
Copyright (c) 2001-2025 Expat maintainers

Permission is hereby granted, free of charge, to any person obta
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, includ
without limitation the rights to use, copy, modify, merge, publi
distribute, sublicense, and/or sell copies of the Software, and
permit persons to whom the Software is furnished to do so, subje
the following conditions:
```

```
The above copyright notice and this permission notice shall be i
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRING
IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR
CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTR
TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH TH
SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
```

libffi

A extensão C **_ctypes** subjacente ao módulo **ctypes** é construída usando uma cópia incluída das fontes do libffi, a menos que a construção esteja configurada com `--with-system-libffi`:

```
Copyright (c) 1996-2008  Red Hat, Inc and others.

Permission is hereby granted, free of charge, to any person obta
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, includ
without limitation the rights to use, copy, modify, merge, publi
distribute, sublicense, and/or sell copies of the Software, and
permit persons to whom the Software is furnished to do so, subje
the following conditions:

The above copyright notice and this permission notice shall be i
in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT.  IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT
HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FRO
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER
DEALINGS IN THE SOFTWARE.
```

zlib

A extensão **zlib** é construída usando uma cópia incluída das fontes zlib se a versão do zlib encontrada no sistema for muito antiga para ser usada na construção:

```
Copyright (C) 1995-2011 Jean-loup Gailly and Mark Adler

This software is provided 'as-is', without any express or implie
warranty.  In no event will the authors be held liable for any c
arising from the use of this software.

Permission is granted to anyone to use this software for any pur
including commercial applications, and to alter it and redistrib
freely, subject to the following restrictions:

1. The origin of this software must not be misrepresented; you m
   claim that you wrote the original software. If you use this s
   in a product, an acknowledgment in the product documentation
   appreciated but is not required.

2. Altered source versions must be plainly marked as such, and m
   misrepresented as being the original software.

3. This notice may not be removed or altered from any source dis

Jean-loup Gailly        Mark Adler
jloup@gzip.org          madler@alumni.caltech.edu
```

cfuhash

A implementação da tabela de hash usada pelo **tracemalloc** é baseada no projeto cfuhash:

```
Copyright (c) 2005 Don Owens
All rights reserved.

This code is released under the BSD license:

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditic
are met:
```

```
  * Redistributions of source code must retain the above copyrig
    notice, this list of conditions and the following disclaimer

  * Redistributions in binary form must reproduce the above
    copyright notice, this list of conditions and the following
    disclaimer in the documentation and/or other materials provi
    with the distribution.

  * Neither the name of the author nor the names of its
    contributors may be used to endorse or promote products deri
    from this software without specific prior written permissior

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUT
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NO
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNES
FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE
COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRE
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES
(INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTIO
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRA
STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVI
OF THE POSSIBILITY OF SUCH DAMAGE.
```

libmpdec

A extensão C **_decimal** subjacente ao módulo **decimal** é construída usando uma cópia incluída da biblioteca libmpdec, a menos que a construção esteja configurada com `--with-system-libmpdec`:

```
Copyright (c) 2008-2020 Stefan Krah. All rights reserved.

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditio
are met:

1. Redistributions of source code must retain the above copyrigh
   notice, this list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyr
```

```
   notice, this list of conditions and the following disclaimer
   documentation and/or other materials provided with the distri

THIS SOFTWARE IS PROVIDED BY THE AUTHOR AND CONTRIBUTORS "AS IS'
ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TC
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICUL
ARE DISCLAIMED.  IN NO EVENT SHALL THE AUTHOR OR CONTRIBUTORS BE
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUT
OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUP
HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRA
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING I
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBIL
SUCH DAMAGE.
```

Conjunto de testes C14N do W3C

O conjunto de testes C14N 2.0 no pacote **`test`** (`Lib/test/xmltestdata/c14n-20/`) foi recuperado do site do W3C em https://www.w3.org/TR/xml-c14n2-testcases/ e é distribuído sob a licença BSD de 3 cláusulas:

```
Copyright (c) 2013 W3C(R) (MIT, ERCIM, Keio, Beihang),
All Rights Reserved.

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditic
are met:

* Redistributions of works must retain the original copyright no
  this list of conditions and the following disclaimer.
* Redistributions in binary form must reproduce the original cop
  notice, this list of conditions and the following disclaimer i
  documentation and/or other materials provided with the distrib
* Neither the name of the W3C nor the names of its contributors
  used to endorse or promote products derived from this work wit
  specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUT
"AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NO
LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNES
```

```
A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYF
OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDE
SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND C
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR T
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF TH
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAM
```

mimalloc

Licença MIT:

```
Copyright (c) 2018-2021 Microsoft Corporation, Daan Leijen

Permission is hereby granted, free of charge, to any person obta
of this software and associated documentation files (the "Softwa
in the Software without restriction, including without limitatic
to use, copy, modify, merge, publish, distribute, sublicense, ar
copies of the Software, and to permit persons to whom the Softwa
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be i
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANT
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVEN
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DE
SOFTWARE.
```

asyncio

Partes do módulo `asyncio` são incorporadas do uvloop 0.16 [https://github.com/MagicStack/uvloop/tree/v0.16.0], que é distribuído sob a licença MIT:

Copyright (c) 2015-2021 MagicStack Inc. http://magic.io

Permission is hereby granted, free of charge, to any person obta
a copy of this software and associated documentation files (the
"Software"), to deal in the Software without restriction, includ
without limitation the rights to use, copy, modify, merge, publi
distribute, sublicense, and/or sell copies of the Software, and
permit persons to whom the Software is furnished to do so, subje
the following conditions:

The above copyright notice and this permission notice shall be
included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND
NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLD
LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN
OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNE
WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Global Unbounded Sequences (GUS)

O arquivo `Python/qsbr.c` é adaptado do esquema de recuperação de memória segura “Global Unbounded Sequences” do FreeBSD em subr_smr.c [https://github.com/freebsd/freebsd-src/blob/main/sys/kern/subr_smr.c]. O arquivo é distribuído sob a licença BSD de 2 cláusulas:

Copyright (c) 2019,2020 Jeffrey Roberson <jeff@FreeBSD.org>

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditio
are met:
1. Redistributions of source code must retain the above copyrigh
   notice unmodified, this list of conditions, and the following
   disclaimer.
2. Redistributions in binary form must reproduce the above copyr
   notice, this list of conditions and the following disclaimer
   documentation and/or other materials provided with the distri

THIS SOFTWARE IS PROVIDED BY THE AUTHOR "AS IS" AND ANY EXPRESS

```
IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED W
OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISC
IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY DIRECT, INDIRECT,
INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUD
NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOS
DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND C
THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR T
(INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF TH
THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE
```

Ligações Zstandard

As ligações de Zstandard em `Modules/_zstd` e `Lib/compression/zstd` são baseadas no código da biblioteca pyzstd [https://github.com/Rogdham/pyzstd/], com direitos autorais de Ma Lin e colaboradores. O código do pyzstd é distribuído sob a Licença BSD de 3 Cláusulas:

```
Copyright (c) 2020-present, Ma Lin and contributors.
All rights reserved.

Redistribution and use in source and binary forms, with or witho
modification, are permitted provided that the following conditic

1. Redistributions of source code must retain the above copyrigh
   list of conditions and the following disclaimer.

2. Redistributions in binary form must reproduce the above copyr
   this list of conditions and the following disclaimer in the c
   and/or other materials provided with the distribution.

3. Neither the name of the copyright holder nor the names of its
   contributors may be used to endorse or promote products deriv
   this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUT
AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICUL
DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTC
FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CON
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUT
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTIC
```

```
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRI
OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY C
OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAM
```

Módulo de perfilamento

O módulo **`profiling`** inclui bibliotecas de terceiros incorporadas em `Lib/profiling/sampling/_vendor/` com as seguintes licenças:

d3-flamegraph

A biblioteca d3-flamegraph é distribuída sob a Licença Apache, Versão 2.0. Consulte a seção OpenSSL acima para obter o texto completo da Licença Apache, Versão 2.0.

d3.js

A biblioteca d3.js contém o seguinte aviso:

```
Copyright 2010-2021 Mike Bostock

Permission to use, copy, modify, and/or distribute this software
with or without fee is hereby granted, provided that the above c
and this permission notice appear in all copies.

THE SOFTWARE IS PROVIDED "AS IS" AND THE AUTHOR DISCLAIMS ALL WA
REGARD TO THIS SOFTWARE INCLUDING ALL IMPLIED WARRANTIES OF MERC
FITNESS. IN NO EVENT SHALL THE AUTHOR BE LIABLE FOR ANY SPECIAL,
INDIRECT, OR CONSEQUENTIAL DAMAGES OR ANY DAMAGES WHATSOEVER RES
OF USE, DATA OR PROFITS, WHETHER IN AN ACTION OF CONTRACT, NEGLI
TORTIOUS ACTION, ARISING OUT OF OR IN CONNECTION WITH THE USE OF
THIS SOFTWARE.
```

Pacotes Pixi

As definições de pacotes Pixi encontradas em `Tools/pixi-packages` derivam de https://github.com/conda-forge/python-feedstock, que contém a seguinte licença:

BSD-3-Clause license
Copyright (c) 2015-2026, conda-forge contributors
All rights reserved.

Redistribution and use in source and binary forms, with or withc
modification, are permitted provided that the following conditic

1. Redistributions of source code must retain the above copyri
   this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above cop
   notice, this list of conditions and the following disclaime
   documentation and/or other materials provided with the dist
3. Neither the name of the copyright holder nor the names of i
   contributors may be used to endorse or promote products der
   this software without specific prior written permission.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUT
AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITE
IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICUL
ARE DISCLAIMED. IN NO EVENT SHALL THE REGENTS OR CONTRIBUTORS BE
ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQU
DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUT
SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTIC
CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRI
LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING I
OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBIL
DAMAGE.

Banco de dados de caracteres Unicode

Um trecho do Unicode Character Database [https://www.unicode.org/ucd/], convertido para um formato interno, é utilizado pelo módulo **`unicodedata`** e para o suporte a Unicode do tipo **`str`**. Os arquivos de dados originais do Unicode são distribuídos sob a Licença Unicode [https://www.unicode.org/license.txt]:

UNICODE LICENSE V3

COPYRIGHT AND PERMISSION NOTICE

Copyright © 1991-2026 Unicode, Inc.

NOTICE TO USER: Carefully read the following legal agreement. BY
DOWNLOADING, INSTALLING, COPYING OR OTHERWISE USING DATA FILES,
SOFTWARE, YOU UNEQUIVOCALLY ACCEPT, AND AGREE TO BE BOUND BY, AL
TERMS AND CONDITIONS OF THIS AGREEMENT. IF YOU DO NOT AGREE, DO
DOWNLOAD, INSTALL, COPY, DISTRIBUTE OR USE THE DATA FILES OR SOF

Permission is hereby granted, free of charge, to any person obta
copy of data files and any associated documentation (the "Data F
software and any associated documentation (the "Software") to de
Data Files or Software without restriction, including without li
the rights to use, copy, modify, merge, publish, distribute, and
copies of the Data Files or Software, and to permit persons to w
Data Files or Software are furnished to do so, provided that eit
this copyright and permission notice appear with all copies of t
Files or Software, or (b) this copyright and permission notice a
associated Documentation.

THE DATA FILES AND SOFTWARE ARE PROVIDED "AS IS", WITHOUT WARRAN
KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRA
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRING
THIRD PARTY RIGHTS.

IN NO EVENT SHALL THE COPYRIGHT HOLDER OR HOLDERS INCLUDED IN TH
BE LIABLE FOR ANY CLAIM, OR ANY SPECIAL INDIRECT OR CONSEQUENTIA
OR ANY DAMAGES WHATSOEVER RESULTING FROM LOSS OF USE, DATA OR PR
WHETHER IN AN ACTION OF CONTRACT, NEGLIGENCE OR OTHER TORTIOUS A
ARISING OUT OF OR IN CONNECTION WITH THE USE OR PERFORMANCE OF T
FILES OR SOFTWARE.

Except as contained in this notice, the name of a copyright hold
not be used in advertising or otherwise to promote the sale, use
dealings in these Data Files or Software without prior written
authorization of the copyright holder.

Índice de Módulos Python

_ | a | b | c | d | e | f | g | h | i | j | k | l | m | n | o | p | q | r | s | t | u | v | w | x | z

_	
`__future__`	*Future statement definitions*
`__main__`	*The environment where top-level code is run. Covers command-line interfaces, import-time behavior, and ``__name__ == '__main__'``.*
`_thread`	*Low-level threading API.*
`_tkinter`	*A binary module that contains the low-level interface to Tcl/Tk.*

a	
`abc`	*Abstract base classes according to :pep:`3119`.*
`aifc`	**Descontinuado:** *Removed in 3.13.*
`annotationlib`	*Functionality for introspecting annotations*
`argparse`	*Command-line option and argument parsing library.*
`array`	*Space efficient arrays of uniformly typed numeric values.*
`ast`	*Abstract Syntax Tree classes and manipulation.*
`asynchat`	**Descontinuado:** *Removed in 3.12.*
`asyncio`	*Asynchronous I/O.*
`asyncore`	**Descontinuado:** *Removed in 3.12.*
`atexit`	*Register and execute cleanup*

	functions.
`audioop`	**Descontinuado:** *Removed in 3.13.*

b	
`base64`	*RFC 4648: Base16, Base32, Base64 Data Encodings; Base85 and Ascii85*
`bdb`	*Debugger framework.*
`binascii`	*Tools for converting between binary and various ASCII-encoded binary representations.*
`bisect`	*Array bisection algorithms for binary searching.*
`builtins`	*The module that provides the built-in namespace.*
`bz2`	*Interfaces for bzip2 compression and decompression.*

c	
`calendar`	*Functions for working with calendars, including some emulation of the Unix cal program.*
`cgi`	**Descontinuado:** *Removed in 3.13.*
`cgitb`	**Descontinuado:** *Removed in 3.13.*
`chunk`	**Descontinuado:** *Removed in 3.13.*
`cmath`	*Mathematical functions for complex numbers.*
`cmd`	*Build line-oriented command interpreters.*
`code`	*Facilities to implement read-eval-print loops.*
`codecs`	*Encode and decode data and streams.*
`codeop`	*Compile (possibly incomplete) Python*

	code.
collections	*Container datatypes*
collections.abc	*Abstract base classes for containers*
colorsys	*Conversion functions between RGB and other color systems.*
compileall	*Tools for byte-compiling all Python source files in a directory tree.*
compression	
compression.zstd	*Low-level interface to compression and decompression routines in the zstd library.*
concurrent	
concurrent.futures	*Execute computations concurrently using threads or processes.*
concurrent.interpreters	*Multiple interpreters in the same process*
configparser	*Configuration file parser.*
contextlib	*Utilities for with-statement contexts.*
contextvars	*Context Variables*
copy	*Shallow and deep copy operations.*
copyreg	*Register pickle support functions.*
crypt	**Descontinuado:** *Removed in 3.13.*
csv	*Write and read tabular data to and from delimited files.*
ctypes	*A foreign function library for Python.*
curses	*An interface to the curses library, providing portable terminal handling.*
curses.ascii	*Constants and set-membership functions for ASCII characters.*
curses.panel	*A panel stack extension that adds depth to curses windows.*
curses.textpad	*Emacs-like input editing in a curses*

	window.
d	
`dataclasses`	*Generate special methods on user-defined classes.*
`datetime`	*Basic date and time types.*
`dbm`	*Interfaces to various Unix "database" formats.*
`dbm.dumb`	*Portable implementation of the simple DBM interface.*
`dbm.gnu`	*GNU database manager*
`dbm.ndbm`	*The New Database Manager*
`dbm.sqlite3`	*SQLite backend for dbm*
`decimal`	*Implementation of the General Decimal Arithmetic Specification.*
`difflib`	*Helpers for computing differences between objects.*
`dis`	*Disassembler for Python bytecode.*
`distutils`	**Descontinuado:** *Removed in 3.12.*
`doctest`	*Test pieces of code within docstrings.*
e	
`email`	*Package supporting the parsing, manipulating, and generating email messages.*
`email.charset`	*Character Sets*
`email.contentmanager`	*Storing and Retrieving Content from MIME Parts*
`email.encoders`	*Encoders for email message payloads.*
`email.errors`	*The exception classes used by the email package.*

email.generator	*Generate flat text email messages from a message structure.*
email.header	*Representing non-ASCII headers*
email.headerregistry	*Automatic Parsing of headers based on the field name*
email.iterators	*Iterate over a message object tree.*
email.message	*The base class representing email messages.*
email.mime	*Build MIME messages.*
email.mime.application	
email.mime.audio	
email.mime.base	
email.mime.image	
email.mime.message	
email.mime.multipart	
email.mime.nonmultipart	
email.mime.text	
email.parser	*Parse flat text email messages to produce a message object structure.*
email.policy	*Controlling the parsing and generating of messages*
email.utils	*Miscellaneous email package utilities.*
encodings	*Encodings package*
encodings.idna	*Internationalized Domain Names implementation*
encodings.mbcs	*Windows ANSI codepage*
encodings.utf_8_sig	*UTF-8 codec with BOM signature*
ensurepip	*Bootstrapping the "pip" installer into an existing Python installation or virtual environment.*
enum	*Implementation of an enumeration*

	class.
errno	*Standard errno system symbols.*

f	
faulthandler	*Dump the Python traceback.*
fcntl	*The fcntl() and ioctl() system calls.*
filecmp	*Compare files efficiently.*
fileinput	*Loop over standard input or a list of files.*
fnmatch	*Unix shell style filename pattern matching.*
fractions	*Rational numbers.*
ftplib	*FTP protocol client (requires sockets).*
functools	*Higher-order functions and operations on callable objects.*

g	
gc	*Interface to the cycle-detecting garbage collector.*
getopt	*Portable parser for command line options; support both short and long option names.*
getpass	*Portable reading of passwords and retrieval of the userid.*
gettext	*Multilingual internationalization services.*
glob	*Unix shell style pathname pattern expansion.*
graphlib	*Functionality to operate with graph-like structures*
grp	*The group database (getgrnam() and friends).*

gzip	*Interfaces for gzip compression and decompression using file objects.*
h	
hashlib	*Secure hash and message digest algorithms.*
heapq	*Heap queue algorithm (a.k.a. priority queue).*
hmac	*Keyed-Hashing for Message Authentication (HMAC) implementation*
html	*Helpers for manipulating HTML.*
html.entities	*Definitions of HTML general entities.*
html.parser	*A simple parser that can handle HTML and XHTML.*
http	*HTTP status codes and messages*
http.client	*HTTP and HTTPS protocol client (requires sockets).*
http.cookiejar	*Classes for automatic handling of HTTP cookies.*
http.cookies	*Support for HTTP state management (cookies).*
http.server	*HTTP server and request handlers.*
i	
idlelib	*Implementation package for the IDLE shell/editor.*
imaplib	*IMAP4 protocol client (requires sockets).*
imghdr	**Descontinuado:** *Removed in 3.13.*
imp	**Descontinuado:** *Removed in 3.12.*
importlib	*The implementation of the import*

	machinery.
`importlib.abc`	*Abstract base classes related to import*
`importlib.machinery`	*Importers and path hooks*
`importlib.metadata`	*Accessing package metadata*
`importlib.resources`	*Package resource reading, opening, and access*
`importlib.resources.abc`	*Abstract base classes for resources*
`importlib.util`	*Utility code for importers*
`inspect`	*Extract information and source code from live objects.*
`io`	*Core tools for working with streams.*
`ipaddress`	*IPv4/IPv6 manipulation library.*
`itertools`	*Functions creating iterators for efficient looping.*

j	
`json`	*Encode and decode the JSON format.*
`json.tool`	*A command-line interface to validate and pretty-print JSON.*

k	
`keyword`	*Test whether a string is a keyword in Python.*

l	
`linecache`	*Provides random access to individual lines from text files.*
`locale`	*Internationalization services.*
`logging`	*Flexible event logging system for applications.*
`logging.config`	*Configuration of the logging module.*

logging.handlers	*Handlers for the logging module.*
lzma	*A Python wrapper for the liblzma compression library.*

m	
mailbox	*Manipulate mailboxes in various formats*
mailcap	**Descontinuado:** *Removed in 3.13.*
marshal	*Convert Python objects to streams of bytes and back (with different constraints).*
math	*Mathematical functions (sin() etc.).*
math.integer	*Integer-specific mathematics functions.*
mimetypes	*Mapping of filename extensions to MIME types.*
mmap	*Interface to memory-mapped files for Unix and Windows.*
modulefinder	*Find modules used by a script.*
msilib	**Descontinuado:** *Removed in 3.13.*
msvcrt	*Miscellaneous useful routines from the MS VC++ runtime.*
multiprocessing	*Process-based parallelism.*
multiprocessing.connection	*API for dealing with sockets.*
multiprocessing.dummy	*Dumb wrapper around threading.*
multiprocessing.managers	*Share data between process with shared objects.*
multiprocessing.pool	*Create pools of processes.*
multiprocessing.shared_memory	*Provides shared memory for direct access across processes.*
multiprocessing.sharedctypes	*Allocate ctypes objects from shared memory.*

n	
netrc	*Loading of .netrc files.*
nis	**Descontinuado:** *Removed in 3.13.*
nntplib	**Descontinuado:** *Removed in 3.13.*
numbers	*Numeric abstract base classes (Complex, Real, Integral, etc.).*

o	
operator	*Functions corresponding to the standard operators.*
optparse	*Command-line option parsing library.*
os	*Miscellaneous operating system interfaces.*
os.path	*Operations on pathnames.*
ossaudiodev	**Descontinuado:** *Removed in 3.13.*

p	
pathlib	*Object-oriented filesystem paths*
pathlib.types	*pathlib types for static type checking*
pdb	*The Python debugger for interactive interpreters.*
pickle	*Convert Python objects to streams of bytes and back.*
pickletools	*Contains extensive comments about the pickle protocols and pickle-machine opcodes, as well as some useful functions.*
pipes	**Descontinuado:** *Removed in 3.13.*
pkgutil	*Utilities for the import system.*
platform	*Retrieves as much platform identifying*

	data as possible.
`plistlib`	*Generate and parse Apple plist files.*
`poplib`	*POP3 protocol client (requires sockets).*
`posix`	*The most common POSIX system calls (normally used via module os).*
`pprint`	*Data pretty printer.*
`profile`	**Descontinuado:** *Pure Python profiler (deprecated).*
`profiling`	*Python profiling tools for performance analysis.*
`profiling.sampling`	*Statistical sampling profiler for Python processes.*
`profiling.tracing`	*Deterministic tracing profiler for Python programs.*
`pstats`	*Statistics object for analyzing profiler output.*
`pty`	*Pseudo-Terminal Handling for Unix.*
`pwd`	*The password database (getpwnam() and friends).*
`py_compile`	*Generate byte-code files from Python source files.*
`pyclbr`	*Supports information extraction for a Python module browser.*
`pydoc`	*Documentation generator and online help system.*

q	
`queue`	*A synchronized queue class.*
`quopri`	*Encode and decode files using the MIME quoted-printable encoding.*

r	
random	*Generate pseudo-random numbers with various common distributions.*
re	*Regular expression operations.*
readline	*GNU readline support for Python.*
reprlib	*Alternate repr() implementation with size limits.*
resource	*An interface to provide resource usage information on the current process.*
rlcompleter	*Python identifier completion, suitable for the GNU readline library.*
runpy	*Locate and run Python modules without importing them first.*

s	
sched	*General purpose event scheduler.*
secrets	*Generate secure random numbers for managing secrets.*
select	*Wait for I/O completion on multiple streams.*
selectors	*High-level I/O multiplexing.*
shelve	*Python object persistence.*
shlex	*Simple lexical analysis for Unix shell-like languages.*
shutil	*High-level file operations, including copying.*
signal	*Set handlers for asynchronous events.*
site	*Module responsible for site-specific configuration.*
sitecustomize	
smtpd	**Descontinuado:** *Removed in 3.12.*
smtplib	*SMTP protocol client (requires*

	sockets).
sndhdr	**Descontinuado:** *Removed in 3.13.*
socket	*Low-level networking interface.*
socketserver	*A framework for network servers.*
spwd	**Descontinuado:** *Removed in 3.13.*
sqlite3	*A DB-API 2.0 implementation using SQLite 3.x.*
ssl	*TLS/SSL wrapper for socket objects*
stat	*Utilities for interpreting the results of os.stat(), os.lstat() and os.fstat().*
statistics	*Mathematical statistics functions*
string	*Common string operations.*
string.templatelib	*Support for template string literals.*
stringprep	*String preparation, as per RFC 3453*
struct	*Interpret bytes as packed binary data.*
subprocess	*Subprocess management.*
sunau	**Descontinuado:** *Removed in 3.13.*
symtable	*Interface to the compiler's internal symbol tables.*
sys	*Access system-specific parameters and functions.*
sys.monitoring	*Access and control event monitoring*
sysconfig	*Python's configuration information*
syslog	*An interface to the Unix syslog library routines.*

t	
tabnanny	*Tool for detecting white space related problems in Python source files in a directory tree.*
tarfile	*Read and write tar-format archive*

	files.
telnetlib	**Descontinuado:** *Removed in 3.13.*
tempfile	*Generate temporary files and directories.*
termios	*POSIX style tty control.*
test	*Regression tests package containing the testing suite for Python.*
test.regrtest	*Drives the regression test suite.*
test.support	*Support for Python's regression test suite.*
test.support.bytecode_helper	*Support tools for testing correct bytecode generation.*
test.support.import_helper	*Support for import tests.*
test.support.os_helper	*Support for os tests.*
test.support.script_helper	*Support for Python's script execution tests.*
test.support.socket_helper	*Support for socket tests.*
test.support.threading_helper	*Support for threading tests.*
test.support.warnings_helper	*Support for warnings tests.*
textwrap	*Text wrapping and filling*
threading	*Thread-based parallelism.*
time	*Time access and conversions.*
timeit	*Measure the execution time of small code snippets.*
tkinter	*Interface to Tcl/Tk for graphical user interfaces*
tkinter.colorchooser	*Color choosing dialog*
tkinter.commondialog	*Tkinter base class for dialogs*
tkinter.dialog	*A simple dialog box built on the classic Tk widgets.*

tkinter.dnd	*Tkinter drag-and-drop interface*
tkinter.filedialog	*Dialog classes for file selection*
tkinter.font	*Tkinter font-wrapping class*
tkinter.messagebox	*Various types of alert dialogs*
tkinter.scrolledtext	*Text widget with a vertical scroll bar.*
tkinter.simpledialog	*Simple dialog windows*
tkinter.ttk	*Tk themed widget set*
token	*Constants representing terminal nodes of the parse tree.*
tokenize	*Lexical scanner for Python source code.*
tomllib	*Parse TOML files.*
trace	*Trace or track Python statement execution.*
traceback	*Print or retrieve a stack traceback.*
tracemalloc	*Trace memory allocations.*
tty	*Utility functions that perform common terminal control operations.*
turtle	*An educational framework for simple graphics applications*
turtledemo	*A viewer for example turtle scripts*
types	*Names for built-in types.*
typing	*Support for type hints (see :pep:`484`).*

u	
unicodedata	*Access the Unicode Database.*
unittest	*Unit testing framework for Python.*
unittest.mock	*Mock object library.*
urllib	
urllib.error	*Exception classes raised by*

	urllib.request.
`urllib.parse`	*Parse URLs into or assemble them from components.*
`urllib.request`	*Extensible library for opening URLs.*
`urllib.response`	*Response classes used by urllib.*
`urllib.robotparser`	*Load a robots.txt file and answer questions about fetchability of other URLs.*
`usercustomize`	
`uu`	**Descontinuado:** *Removed in 3.13.*
`uuid`	*UUID objects (universally unique identifiers) according to RFC 9562*

v	
`venv`	*Creation of virtual environments.*

w	
`warnings`	*Issue warning messages and control their disposition.*
`wave`	*Provide an interface to the WAV sound format.*
`weakref`	*Support for weak references and weak dictionaries.*
`webbrowser`	*Easy-to-use controller for web browsers.*
`winreg`	*Routines and objects for manipulating the Windows registry.*
`winsound`	*Access to the sound-playing machinery for Windows.*
`wsgiref`	*WSGI Utilities and Reference Implementation.*
`wsgiref.handlers`	*WSGI server/gateway base classes.*

wsgiref.headers	*WSGI response header tools.*
wsgiref.simple_server	*A simple WSGI HTTP server.*
wsgiref.types	*WSGI types for static type checking*
wsgiref.util	*WSGI environment utilities.*
wsgiref.validate	*WSGI conformance checker.*

x	
xdrlib	**Descontinuado:** *Removed in 3.13.*
xml	*Package containing XML processing modules*
xml.dom	*Document Object Model API for Python.*
xml.dom.minidom	*Minimal Document Object Model (DOM) implementation.*
xml.dom.pulldom	*Support for building partial DOM trees from SAX events.*
xml.etree.ElementInclude	
xml.etree.ElementTree	*Implementation of the ElementTree API.*
xml.parsers.expat	*An interface to the Expat non-validating XML parser.*
xml.parsers.expat.errors	
xml.parsers.expat.model	
xml.sax	*Package containing SAX2 base classes and convenience functions.*
xml.sax.handler	*Base classes for SAX event handlers.*
xml.sax.saxutils	*Convenience functions and classes for use with SAX.*
xml.sax.xmlreader	*Interface which SAX-compliant XML parsers must implement.*
xmlrpc	*Server and client modules implementing XML-RPC.*

`xmlrpc.client`	*XML-RPC client access.*
`xmlrpc.server`	*Basic XML-RPC server implementations.*

z	
`zipapp`	*Manage executable Python zip archives*
`zipfile`	*Read and write ZIP-format archive files.*
`zipimport`	*Support for importing Python modules from ZIP archives.*
`zlib`	*Low-level interface to compression and decompression routines compatible with gzip.*
`zoneinfo`	*IANA time zone support*

Melhorar uma página da documentação

Sempre estamos interessados em ouvir ideias sobre melhorias à documentação.

Se o bug ou melhoria sugerida está relacionado a tradução desta documentação, abra um relatório de problema ou edite a página no repositório da tradução [https://github.com/python/python-docs-pt-br].

Você tem alguma formas de fazer perguntas ou sugerir mudanças:

- Você pode iniciar uma discussão sobre a página no fórum de discussão do Python. Este link vai iniciar um tópico na categoria Documentation: *New Documentation topic <https://discuss.python.org/new-topic?category=documentation>`_* (“Novo tópico em Documentation”, em inglês).
- Você pode registrar um “issue” no rastreador de problemas do Python no GitHub. Este link vai criar um novo issue com o rótulo “docs”: New docs issue [https://github.com/python/cpython/issues/new?template=documentation.yml] (“Novo problema de documentação”, em inglês).

Melhorar uma página da documentação

Sempre estamos interessados em ouvir ideias sobre melhorias para a documentação.

Você estava lendo “PAGETITLE” em PAGEURL. O código-fonte dessa página está no GitHub [https://github.com/python/cpython/blob/main/Doc/PAGESOURCE?plain=1].

Se o bug ou melhoria sugerida está relacionado a tradução desta documentação, abra um relatório de problema ou edite a página no repositório da tradução [https://github.com/python/python-docs-pt-br].

Você tem alguma formas de fazer perguntas ou sugerir mudanças:

- Você pode iniciar uma discussão sobre a página no fórum de discussão do Python. Este link vai iniciar um tópico pré-preenchido: Question about page “PAGETITLE” [https://discuss.python.org/new-topic?category=documentation&title=Question+about+page+%22PAGETITLE%22&body=About+the+page+at+PAGEURL%3A] (“Pergunta sobre a página”, em inglês).
- Você pode registrar um “issue” no rastreador de problemas do Python no GitHub. Este link vai criar um novo issue pré-preenchido: Docs: problem with page “PAGETITLE” [https://github.com/python/cpython/issues/new?template=documentation.yml&title=Docs%3A+problem+with+page+%22PAGETITLE%22&description=The+page+at+PAGEURL+has+a+problem%3A] (“Docs: problema com a página”, em inglês).
- Você pode editar a página no GitHub [https://github.com/python/cpython/blob/main/Doc/PAGESOURCE?plain=1] para abrir uma pull request e iniciar o processo de contribuição.

Inicialização, finalização e threads

Esta página foi dividida nas seguintes:

- Interpreter initialization and finalization
- Thread states and the global interpreter lock
- Primitivas de sincronização
- Thread-local storage support
- Multiple interpreters in a Python process
- Profiling and tracing

Remoção pendente no Python 3.14

- O campo `ma_version_tag` em `PyDictObject` para módulos de extensão (**PEP 699** [https://peps.python.org/pep-0699/]; gh-101193 [https://github.com/python/cpython/issues/101193]).
- A criação de `tipos imutáveis` com bases mutáveis (gh-95388 [https://github.com/python/cpython/issues/95388]).

Remoção pendente no Python 3.15

- A **`PyImport_ImportModuleNoBlock()`**: Use `PyImport_ImportModule()`

- **`PyWeakref_GetObject()`** e **`PyWeakref_GET_OBJECT()`**: Use `PyWeakref_GetRef()`. O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pod ser usado para obter **`PyWeakref_GetRef()`** no Python 3.12 e versões mais antigas.

- **`PyUnicode_AsDecodedObject()`**: use `PyCodec_Decode()`.

- **`PyUnicode_AsDecodedUnicode()`**: Use `PyCodec_Decode()`; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de `str`, tal como `bytes`.

- **`PyUnicode_AsEncodedObject()`**: use `PyCodec_Encode()`.

- **`PyUnicode_AsEncodedUnicode()`**: Use `PyCodec_Encode()`; Note que alguns codecs (por exemplo, “base64”) podem retornar um tipo diferente de `bytes`, tal com `str`.

- Funções de inicialização do Python, descontinuadas no Python 3.9:

  - **`Py_GetPath()`**: Use `PyConfig_Get("module_search_paths")` (`sys.path`).
  - **`Py_GetPrefix()`**: Use `PyConfig_Get("base_prefix")` (`sys.base_prefix`). Use **`PyConfig_Get("prefix")`** (`sys.prefix`) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetExecPrefix()`**: Use `PyConfig_Get("base_exec_prefix")` (`sys.base_exec_prefix`). Use **`PyConfig_Get("exec_prefix")`** (`sys.exec_prefix`) se ambientes virtuais precisarem ser tratados.
  - **`Py_GetProgramFullPath()`**: Use `PyConfig_Get("executable")` (`sys.executable`).
  - **`Py_GetProgramName()`**: Use `PyConfig_Get("executable")` (`sys.executable`).

- **`Py_GetPythonHome()`**: Use `PyConfig_Get("home")` ou a variável de ambiente `PYTHONHOME`.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter `PyConfig_Get()` no Python 3.13 e versões anteriores.

- Funções para configurar a inicialização do Python, descontinuadas no Python 3.11:

  - **`PySys_SetArgvEx()`**: defina `PyConfig.argv`.
  - **`PySys_SetArgv()`**: defina `PyConfig.argv`.
  - **`Py_SetProgramName()`**: defina `PyConfig.program_name`.
  - **`Py_SetPythonHome()`**: defina `PyConfig.home`.
  - **`PySys_ResetWarnOptions()`**: Limpe `sys.warnoptions` e **`warnings.filters`**.

  Em vez disso, a API `Py_InitializeFromConfig()` deve ser usada com `PyConfig`.

- Variáveis de configuração globais

  - `Py_DebugFlag`: Use `PyConfig.parser_debug` ou `PyConfig_Get("parser_debug")`.
  - `Py_VerboseFlag`: Use `PyConfig.verbose` ou `PyConfig_Get("verbose")`.
  - `Py_QuietFlag`: Use `PyConfig.quiet` ou `PyConfig_Get("quiet")`.
  - `Py_InteractiveFlag`: Use `PyConfig.interactive` ou `PyConfig_Get("interactive")`.
  - `Py_InspectFlag`: Use `PyConfig.inspect` ou `PyConfig_Get("inspect")`.
  - `Py_OptimizeFlag`: Use `PyConfig.optimization_level` ou `PyConfig_Get("optimization_level")`.
  - `Py_NoSiteFlag`: Use `PyConfig.site_import` ou `PyConfig_Get("site_import")`.
  - `Py_BytesWarningFlag`: Use `PyConfig.bytes_warning` ou `PyConfig_Get("bytes_warning")`.

- `Py_FrozenFlag`: Use `PyConfig.pathconfig_warnings` ou `PyConfig_Get("pathconfig_warnings")`.
- `Py_IgnoreEnvironmentFlag`: Use `PyConfig.use_environment` ou `PyConfig_Get("use_environment")`.
- `Py_DontWriteBytecodeFlag`: Use `PyConfig.write_bytecode` ou `PyConfig_Get("write_bytecode")`.
- `Py_NoUserSiteDirectory`: Use `PyConfig.user_site_directory` ou `PyConfig_Get("user_site_directory")`.
- `Py_UnbufferedStdioFlag`: Use `PyConfig.buffered_stdio` ou `PyConfig_Get("buffered_stdio")`.
- `Py_HashRandomizationFlag`: Use `PyConfig.use_hash_seed` e `PyConfig.hash_seed` ou `PyConfig_Get("hash_seed")`.
- `Py_IsolatedFlag`: Use `PyConfig.isolated` ou `PyConfig_Get("isolated")`.
- `Py_LegacyWindowsFSEncodingFlag`: Use `PyPreConfig.legacy_windows_fs_encoding` ou `PyConfig_Get("legacy_windows_fs_encoding")`.
- `Py_LegacyWindowsStdioFlag`: Use `PyConfig.legacy_windows_stdio` ou `PyConfig_Get("legacy_windows_stdio")`.
- **`Py_FileSystemDefaultEncoding`**, **`Py_HasFileSystemDefaultEncoding`**: Use `PyConfig.filesystem_encoding` ou `PyConfig_Get("filesystem_encoding")`.
- **`Py_FileSystemDefaultEncodeErrors`**: Use `PyConfig.filesystem_errors` ou `PyConfig_Get("filesystem_errors")`.
- **`Py_UTF8Mode`**: Use `PyPreConfig.utf8_mode` ou `PyConfig_Get("utf8_mode")`. (veja `Py_PreInitialize()`)

A API `Py_InitializeFromConfig()` deve ser usada com `PyConfig` para definir essas opções. Ou `PyConfig_Get()` pode ser usado para obter essas opções em tempo de execução.

Remoção pendente no Python 3.16

- A cópia empacotada do `libmpdec`.

Remoção pendente no Python 3.18

- As seguintes funções privadas estão descontinuado e planejadas para remoção no Python 3.18:
  - **`_PyBytes_Join()`**: use `PyBytes_Join()`.
  - **`_PyDict_GetItemStringWithError()`**: use `PyDict_GetItemStringRef()`.
  - **`_PyDict_Pop()`**: use `PyDict_Pop()`.
  - **`_PyLong_Sign()`**: use `PyLong_GetSign()`.
  - **`_PyLong_FromDigits()`** e **`_PyLong_New()`**: use `PyLongWriter_Create()`.
  - **`_PyThreadState_UncheckedGet()`**: use `PyThreadState_GetUnchecked()`.
  - **`_PyUnicode_AsString()`**: use `PyUnicode_AsUTF8()`.
  - **`_PyUnicodeWriter_Init()`**: substitua `_PyUnicodeWriter_Init(&writer)` por `writer = PyUnicodeWriter_Create(0)`.
  - **`_PyUnicodeWriter_Finish()`**: substitua `_PyUnicodeWriter_Finish(&writer)` por `PyUnicodeWriter_Finish(writer)`.
  - **`_PyUnicodeWriter_Dealloc()`**: substitua `_PyUnicodeWriter_Dealloc(&writer)` por `PyUnicodeWriter_Discard(writer)`.
  - **`_PyUnicodeWriter_WriteChar()`**: substituta `_PyUnicodeWriter_WriteChar(&writer, ch)` com `PyUnicodeWriter_WriteChar(writer, ch)`.
  - **`_PyUnicodeWriter_WriteStr()`**: substitua `_PyUnicodeWriter_WriteStr(&writer, str)` por `PyUnicodeWriter_WriteStr(writer, str)`.
  - **`_PyUnicodeWriter_WriteSubstring()`**: substitua `_PyUnicodeWriter_WriteSubstring(&writer, str, start, end)` por `PyUnicodeWriter_WriteSubstring(writer, str, start, end)`.

- **`_PyUnicodeWriter_WriteASCIIString()`**: substitua `_PyUnicodeWriter_WriteASCIIString(&writer, str)` por **`PyUnicodeWriter_WriteASCII(writer, str)`**.
- **`_PyUnicodeWriter_WriteLatin1String()`**: substitua `_PyUnicodeWriter_WriteLatin1String(&writer, str)` por **`PyUnicodeWriter_WriteUTF8(writer, str)`**.
- **`_PyUnicodeWriter_Prepare()`**: (sem substituto).
- **`_PyUnicodeWriter_PrepareKind()`**: (sem substituto).
- **`_Py_HashPointer()`**: use **`Py_HashPointer()`**.
- **`_Py_fopen_obj()`**: use **`Py_fopen()`**.

O projeto pythoncapi-compat [https://github.com/python/pythoncapi-compat/] pode ser usado para obter essas novas funções públicas no Python 3.13 e versões anteriores. (Contribuição de Victor Stinner em gh-128863 [https://github.com/python/cpython/issues/128863].)

Remoção pendente no Python 3.19

- **PEP 456** [https://peps.python.org/pep-0456/] suporte para incorporadores à definição do esquema de hashing de strings.

Remoção pendente no Python 3.20

- **`_PyObject_CallMethodId()`**, **`_PyObject_GetAttrId()`** e **`_PyUnicode_FromId()`** foram descontinuadas desde o 3.15 e serão removidas na versão 3.20. Em vez delas, use **`PyUnicode_InternFromString()`** and armazene em cache o resultado no estado do módulo e chame **`PyObject_CallMethod()`** ou **`PyObject_GetAttr()`**. (Contribuição de Victor Stinner em gh-141049 [https://github.com/python/cpython/issues/141049].)
- O campo `cval` em **`PyComplexObject`** (gh-128813 [https://github.com/python/cpython/issues/128813]). Use **`PyComplex_AsCComplex()`** e **`PyComplex_FromCComplex()`** para converter um número complexo Python de/para a representação C de **`Py_complex`**.
- Macros **`Py_MATH_PIl`** e **`Py_MATH_El`**.

Remoção pendente em versões futuras

As APIs a seguir foram descontinuadas e serão removidas, embora atualmente não haja uma data agendada para sua remoção.

- `Py_TPFLAGS_HAVE_FINALIZE`: Desnecessária desde o Python 3.8.
- `PyErr_Fetch()`: Use `PyErr_GetRaisedException()`.
- `PyErr_NormalizeException()`: Use `PyErr_GetRaisedException()`.
- `PyErr_Restore()`: Use `PyErr_SetRaisedException()`.
- `PyModule_GetFilename()`: Use `PyModule_GetFilenameObject()`.
- `PyOS_AfterFork()`: Use `PyOS_AfterFork_Child()`.
- `PySlice_GetIndicesEx()`: Use `PySlice_Unpack()` e `PySlice_AdjustIndices()`.
- `PyUnicode_READY()`: Desnecessário desde o Python 3.12
- `PyErr_Display()`: Use `PyErr_DisplayException()`.
- `_PyErr_ChainExceptions()`: Use `_PyErr_ChainExceptions1()`.
- O membro `PyBytesObject.ob_shash`: chame `PyObject_Hash()`.
- API do Thread Local Storage (TLS):
  - `PyThread_create_key()`: Use `PyThread_tss_alloc()`.
  - `PyThread_delete_key()`: Use `PyThread_tss_free()`.
  - `PyThread_set_key_value()`: Use `PyThread_tss_set()`.
  - `PyThread_get_key_value()`: Use `PyThread_tss_get()`.
  - `PyThread_delete_key_value()`: Use `PyThread_tss_delete()`.
  - `PyThread_ReInitTLS()`: Desnecessário desde o Python 3.7.

Remoção pendente no Python 3.13

Módulos (veja **PEP 594** [https://peps.python.org/pep-0594/]):

- **`aifc`**
- **`audioop`**
- **`cgi`**
- **`cgitb`**
- **`chunk`**
- **`crypt`**
- **`imghdr`**
- **`mailcap`**
- **`msilib`**
- **`nis`**
- **`nntplib`**
- **`ossaudiodev`**
- **`pipes`**
- **`sndhdr`**
- **`spwd`**
- **`sunau`**
- **`telnetlib`**
- **`uu`**
- **`xdrlib`**

Outros módulos:

- **`lib2to3`** e o programa **2to3** (gh-84540 [https://github.com/python/cpython/issues/84540])

APIs:

- **`configparser.LegacyInterpolation`** (gh-90765 [https://github.com/python/cpython/issues/90765])
- `locale.resetlocale()` (gh-90817 [https://github.com/python/cpython/issues/90817])

- **`turtle.RawTurtle.settiltangle()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.findTestCases()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.getTestCaseNames()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.makeSuite()`** (gh-50096 [https://github.com/python/cpython/issues/50096])
- **`unittest.TestProgram.usageExit()`** (gh-67048 [https://github.com/python/cpython/issues/67048])
- **`webbrowser.MacOSX`** (gh-86421 [https://github.com/python/cpython/issues/86421])
- Encadeamento do descritor de **`classmethod`** (gh-89519 [https://github.com/python/cpython/issues/89519])

Remoção pendente no Python 3.14

- **argparse**: Os parâmetros *type*, *choices* e *metavar* de **argparse.BooleanOptionalAction** foram descontinuados e serão removidos na versão 3.14. (Contribuição de Nikita Sobolev em gh-92248 [https://github.com/python/cpython/issues/92248].)

- **ast**: Os seguintes recursos foram descontinuados na documentação desde Python 3.8, agora fazem com que um **DeprecationWarning** seja emitido em tempo de execução quando eles são acessados ou usados, e serão removidos no Python 3.14:

  - **ast.Num**
  - **ast.Str**
  - **ast.Bytes**
  - **ast.NameConstant**
  - **ast.Ellipsis**

  Usa **ast.Constant** em vez disso. (Contribuição de Serhiy Storchaka em gh-90953 [https://github.com/python/cpython/issues/90953].)

- **asyncio**:

  - As classes filhas dos observadores **asyncio.MultiLoopChildWatcher**, **asyncio.FastChildWatcher**, **asyncio.AbstractChildWatcher** e **asyncio.SafeChildWatcher** foram descontinuadas e serão removidas no Python 3.14. (Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
  - **asyncio.set_child_watcher()**, **asyncio.get_child_watcher()**, **asyncio.AbstractEventLoopPolicy.set_child_watcher()** e **asyncio.AbstractEventLoopPolicy.get_child_watcher()** foram descontinuados e serão removidos no Python 3.14.

(Contribuição de Kumar Aditya em gh-94597 [https://github.com/python/cpython/issues/94597].)
    - O método `get_event_loop()` da política de laço de eventos padrão agora emite um `DeprecationWarning` se não houver nenhum laço de eventos atual definido e decidir criar um. (Contribuição de Serhiy Storchaka e Guido van Rossum em gh-100160 [https://github.com/python/cpython/issues/100160].)

- `email`: Descontinua o parâmetro *isdst* em `email.utils.localtime()`. (Contribuição de Alan Williams em gh-72346 [https://github.com/python/cpython/issues/72346].)

- `importlib.abc` descontinuou as classes:

    - **`importlib.abc.ResourceReader`**
    - **`importlib.abc.Traversable`**
    - **`importlib.abc.TraversableResources`**

  Em vez disso, use classes `importlib.resources.abc`:

    - `importlib.resources.abc.Traversable`
    - `importlib.resources.abc.TraversableResources`

  (Contribuição de Jason R. Coombs e Hugo van Kemenade em gh-93963 [https://github.com/python/cpython/issues/93963].)

- `itertools` tinha suporte não documentado, ineficiente, historicamente cheio de bugs e inconsistente para operações de cópia, cópia profunda e serialização com pickle. Isso será removido na versão 3.14 para uma redução significativa no volume de código e na carga de manutenção. (Contribuição de Raymond Hettinger em gh-101588 [https://github.com/python/cpython/issues/101588].)

- `multiprocessing`: O método de inicialização padrão será alterado para um mais seguro no Linux, BSDs e outras plataformas POSIX não-macOS onde `'fork'` é atualmente o padrão (gh-84559 [https://github.com/python/cpython/issues/84559]). Adicionar um aviso de tempo de execução sobre isso foi considerado muito perturbador, pois não se

espera que a maior parte do código se importe. Use as APIs `get_context()` ou `set_start_method()` para especificar explicitamente quando seu código *requer* `'fork'`. Veja Contextos e métodos de inicialização.

- `pathlib`: `is_relative_to()` e `relative_to()`: passar argumentos adicionais foi descontinuado.

- `pkgutil`: **`pkgutil.find_loader()`** e **`pkgutil.get_loader()`** agora levantam `DeprecationWarning`; use `importlib.util.find_spec()`. (Contribuição de Nikita Sobolev em gh-97850 [https://github.com/python/cpython/issues/97850].)

- `pty`:

  - `master_open()`: use `pty.openpty()`.
  - `slave_open()`: use `pty.openpty()`.

- `sqlite3`:

  - **`version`** e **`version_info`**.
  - `execute()` e `executemany()` se espaços reservados nomeados forem usados e *parameters* for uma sequência em vez de um `dict`.

- `urllib`: **`urllib.parse.Quoter`** está obsoleto: não foi planejado para ser uma API pública. (Contribuição de Gregory P. Smith em gh-88168 [https://github.com/python/cpython/issues/88168].)

Remoção pendente no Python 3.15

- O sistema de importação:
  - A definição de `__cached__` em um módulo enquanto falha na definição de `__spec__.cached` está descontinuado. No Python 3.15, `__cached__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
  - A definição de `__package__` em um módulo enquanto falha na definição de `__spec__.parent` está descontinuado. No Python 3.15, `__package__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão. (gh-97879 [https://github.com/python/cpython/issues/97879])
- `ctypes`:
  - A função não documentada `ctypes.SetPointerType()` foi descontinuada desde o Python 3.13.
- `http.server`:
  - A classe obsoleta e raramente usada `CGIHTTPRequestHandler` foi descontinuada desde o Python 3.13. Não existe substituição direta. *Qualquer coisa* é melhor que CGI para fazer a interface de um servidor web com um manipulador de requisição.
  - O sinalizador `--cgi` para a interface de linha de comando **python -m http.server** foi descontinuado desde o Python 3.13.
- `importlib`:
  - Método `load_module()`: use `exec_module()`.
- `pathlib`:
  - `.PurePath.is_reserved()` foi descontinuado desde o Python 3.13. Use `os.path.isreserved()` para detectar caminhos reservados no Windows.
- `platform`:
  - `platform.java_ver()` foi descontinuada desde o Python 3.13. Esta função é útil apenas para suporte Jython, tem uma API confusa e é amplamente não testada.
- `sysconfig`:

    - O argumento *check_home* de `sysconfig.is_python_build()` foi descontinuado desde o Python 3.12.
- `threading`:
    - `RLock()` não aceitará argumentos no Python 3.15. A passagem de quaisquer argumentos foi descontinuada a partir do Python 3.14, uma vez que a versão do Python não permite quaisquer argumentos, mas a versão C permite qualquer número de argumentos posicionais ou nomeados, ignorando todos os argumentos.
- `types`:
    - `types.CodeType`: o acesso a **`codeobject.co_lnotab`** foi descontinuado na **PEP 626** [https://peps.python.org/pep-0626/] desde 3.10 e foi planejado para ser removido em 3.12, mas só recebeu uma `DeprecationWarning` adequada em 3.12. Pode ser removido em 3.15. (Contribuição de Nikita Sobolev em gh-101866 [https://github.com/python/cpython/issues/101866].)
- `typing`:
    - A não-documentada sintaxe de argumento nomeado para criar classes `NamedTuple` (por exemplo, `Point = NamedTuple("Point", x=int, y=int)`) foi descontinuada desde o Python 3.13. Em vez disso, use as sintaxes baseada em classe ou funcional.
    - Ao usar a sintaxe funcional de `TypedDict`s, não passar um valor para o parâmetro *fields* (`TD = TypedDict("TD")`) ou passar `None` (`TD = TypedDict("TD", None)`) foi está descontinuado desde o Python 3.13. Use `class TD(TypedDict): pass` ou `TD = TypedDict("TD", {})` para criar uma classe TypedDict com nenhum campo.
    - A função decoradora **`@typing.no_type_check_decorator`** foi descontinuada desde o Python 3.13. Após oito anos no módulo `typing`, ela ainda não foi suportada por nenhum verificador de tipo importante.
- Módulos **`sre_compile`**, **`sre_constants`** e **`sre_parse`**.
- `wave`:
    - Os métodos `getmark()`, `setmark()` e `getmarkers()` das classes `Wave_read` e `Wave_write` foram descontinuados desde o Python 3.13.

- **`zipimport`**:
  - **`zipimport.zipimporter.load_module()`** está descontinuado desde o Python 3.10. Em vez disso, use **`exec_module()`**. (gh-125746 [https://github.com/python/cpython/issues/125746].)

Remoção pendente no Python 3.16

- O sistema de importação:
  - A definição de `__loader__` em um módulo enquanto falha na definição de `__spec__.loader` está descontinuado. No Python 3.16, `__loader__` deixará de ser definido ou levado em consideração pelo sistema de importação ou pela biblioteca padrão.
- `array`:
  - O código de formato `'u'` (`wchar_t`) foi descontinuado na documentação desde o Python 3.3 e em tempo de execução desde o Python 3.13. Use o código de formato `'w'` (`Py_UCS4`) para caracteres Unicode.
- `asyncio`:
  - `asyncio.iscoroutinefunction()` foi descontinuado e será removido no Python 3.16, use `inspect.iscoroutinefunction()` em vez disso. (Contribuição de Jiahao Li e Kumar Aditya em gh-122875 [https://github.com/python/cpython/issues/122875].)
  - O sistema de políticas `asyncio` está descontinuado e será removido no Python 3.16. Em particular, as seguintes classes e funções estão descontinuadas:
    - `asyncio.AbstractEventLoopPolicy`
    - `asyncio.DefaultEventLoopPolicy`
    - `asyncio.WindowsSelectorEventLoopPolicy`
    - `asyncio.WindowsProactorEventLoopPolicy`
    - `asyncio.get_event_loop_policy()`
    - `asyncio.set_event_loop_policy()`

Os usuários devem usar `asyncio.run()` ou `asyncio.Runner` com *loop_factory* para usar a implementação de laço de eventos desejada.

por exemplo, para usar `asyncio.SelectorEventLoop` no Windows:

```
import asyncio

async def main():
    ...

asyncio.run(main(), loop_factory=asyncio.SelectorEventL
```

(Contribuição de Kumar Aditya em gh-127949 [https://github.com/python/cpython/issues/127949].)

- `builtins`:
  - A inversão bit a bit em tipos booleanos, `~True` ou `~False` foi descontinuada desde o Python 3.12, pois produz resultados surpreendentes e não intuitivos (`-2` e `-1`). Use `not x` em vez disso para a negação lógica de um Booleano. No caso raro de você precisar da inversão bit a bit do inteiro subjacente, converta para `int` explicitamente (`~int(x)`).
- `functools`:
  - A chamada da implementação Python de `functools.reduce()` com *function* ou *sequence* como argumentos nomeados foi descontinuada desde o Python 3.14.
- `logging`:
  - O suporte para manipuladores de registro personalizados com o argumento *strm* foi descontinuado e está programado para ser removido no Python 3.16. Em vez disso, defina manipuladores

com o argumento *stream*. (Contribuição de Mariusz Felisiak em gh-115032 [https://github.com/python/cpython/issues/115032].)

- **mimetypes**:
  - Extensões válidas começam com um '.' ou estão vazias para **mimetypes.MimeTypes.add_type()**. Extensões sem ponto estão descontinuadas e levantarão uma exceção **ValueError** no Python 3.16. (Contribuição de Hugo van Kemenade em gh-75223 [https://github.com/python/cpython/issues/75223].)
- **shutil**:
  - A exceção **ExecError** foi descontinuada desde o Python 3.14. Ela não foi usada por nenhuma função em **shutil** desde o Python 3.4, e agora é um alias de **RuntimeError**.
- **symtable**:
  - O método **Class.get_methods** foi descontinuado desde o Python 3.14.
- **sys**:
  - A função **_enablelegacywindowsfsencoding()** foi descontinuada desde o Python 3.13. Use a variável de ambiente **PYTHONLEGACYWINDOWSFSENCODING**.
- **sysconfig**:
  - A função **sysconfig.expand_makefile_vars()** está descontinuada desde o Python 3.14. Em vez disso, use o argumento `vars` de **sysconfig.get_paths()**.
- **tarfile**:
  - O atributo não documentado e não utilizado **TarFile.tarfile** foi descontinuado desde o Python 3.13.

Remoção pendente no Python 3.17

- **`datetime`**:
  - **`strptime()`** chama usando uma string de formato contendo `%e` (dia do mês) sem um ano. Isso está descontinuado desde Python 3.15. (Contribuição de Stan Ulbrych em gh-70647 [https://github.com/python/cpython/issues/70647].)
- **`collections.abc`**:
  - **`collections.abc.ByteString`** está programado para remoção no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

    `ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estático).

    Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)
- **`encodings`**:

- Passar nomes de *codificação* não ASCII para **`encodings.normalize_encoding()`** está descontinuado e sua remoção está prevista para o Python 3.17. (Contribuição de Stan Ulbrych em gh-136702 [https://github.com/python/cpython/issues/136702].)

- **`profile`**:

  - O módulo **`profile`** está descontinuado e será removido no Python 3.17. Em vez disso, utilize **`profiling.tracing`**, que oferece uma API compatível. Consulte a **PEP 799** [https://peps.python.org/pep-0799/] para obter detalhes. (Contribuição de Pablo Galindo e László Kiss Kollár em gh-138122 [https://github.com/python/cpython/issues/138122].)

- **`webbrowser`**:

  - **`webbrowser.MacOSXOSAScript`** está descontinuado em favor de **`webbrowser.MacOS`**. (gh-137586 [https://github.com/python/cpython/issues/137586])

- **`typing`**:

  - Antes do Python 3.14, as uniões antigas eram implementadas usando a classe privada `typing._UnionGenericAlias`. Essa classe não é mais necessária para a implementação, mas foi mantida para compatibilidade com versões anteriores, com remoção prevista para o Python 3.17. Os usuários devem usar auxiliares de introspecção documentados, como **`typing.get_origin()`** e **`typing.get_args()`**, em vez de depender de detalhes de implementação privada.

  - **`typing.ByteString`**, obsoleto desde o Python 3.9, está programado para ser removido no Python 3.17.

    Use `isinstance(obj, collections.abc.Buffer)` para testar se `obj` implementa o protocolo de buffer em tempo de execução. Para uso em anotações de tipo, use **`Buffer`** ou uma união que especifique explicitamente os tipos suportados pelo seu código (por exemplo, `bytes | bytearray | memoryview`).

`ByteString` foi originalmente concebido para ser uma classe abstrata que serviria como um supertipo de **`bytes`** e **`bytearray`**. No entanto, como o ABC nunca teve métodos, saber que um objeto era uma instância de **`ByteString`** nunca lhe dizia nada de útil sobre o objeto. Outros tipos comuns de buffer, como **`memoryview`**, também nunca foram entendidos como subtipos de **`ByteString`** (seja em tempo de execução ou por verificadores de tipo estático).

Veja **PEP 688** [https://peps.python.org/pep-0688/#current-options] para mais detalhes. (Contribuição de Shantanu Jain em gh-91896 [https://github.com/python/cpython/issues/91896].)

- **`tkinter`**:
  - Os métodos **`trace_variable()`**, **`trace()`** (um apelido de **`trace_variable()`**), **`trace_vdelete()`** e **`trace_vinfo()`** da classe **`tkinter.Variable`**, descontinuados desde o Python 3.14, têm remoção prevista para o Python 3.17. Em vez deles, utilize **`trace_add()`**, **`trace_remove()`** e **`trace_info()`**. (Contribuição de Serhiy Storchaka em gh-120220 [https://github.com/python/cpython/issues/120220].)

Remoção pendente no Python 3.18

- Não aceita mais um valor booleano quando um descritor de arquivo é esperado. (Contribuição de Serhiy Storchaka em gh-82626 [https://github.com/python/cpython/issues/82626].)

- **`decimal`**:

  - O especificador de formato de **`Decimal`** não padrão e não documentado `'N'`, que só é suportado na implementação C do módulo **`decimal`**, foi descontinuada desde o Python 3.13. (Contribuição de Serhiy Storchaka em gh-89902 [https://github.com/python/cpython/issues/89902].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

  - Linhas de `import` em arquivos *`name`*`.pth` são silenciosamente ignoradas.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

Remoção pendente no Python 3.19

- `ctypes`:
  - Troca implícita para o layout de estrutura compatível com MSVC definindo `_pack_`, mas não `_layout_` em plataformas não Windows.
- `hashlib`:
  - Em construtores de funções de hash, como `new()`, ou nos construtores nomeados diretamente pelo algoritmo de hash, como `md5()` e `sha256()`, o parâmetro opcional de dados iniciais também podia ser passado como um argumento nomeado `data=` ou `string=` em várias implementações do **`hashlib`**.

    O suporte ao nome do argumento nomeado `string` está agora descontinuado e previsto para ser removido no Python 3.19.

    Antes do Python 3.13, o parâmetro nomeado `string` não era suportado corretamente, dependendo da implementação de funções de hash do backend. Dê preferência a passar os dados iniciais como um argumento posicional para garantir a máxima retrocompatibilidade.
- `http.cookies`:
  - `http.cookies.Morsel.js_output()` está descontinuado e será removido no Python 3.19.
  - `http.cookies.BaseCookie.js_output()` está descontinuado e será removido no Python 3.19.
- `imaplib`:
  - A alteração de `IMAP4.file` agora está descontinuado e programado para remoção no Python 3.19. Esta propriedade

agora não é utilizada e alterar seu valor não fecha automaticamente o arquivo atual.

Antes do Python 3.14, essa propriedade era usada para implementar os métodos `read()` e `readline()` correspondentes para **`IMAP4`**, mas isso não ocorre mais desde então.

Remoção pendente no Python 3.20

- Chamar o método `__new__()` de `struct.Struct` sem o argumento *format* está descontinuado e será removido no Python 3.20. Chamar o método `__init__()` em objetos `Struct` já inicializados está descontinuado e será removido no Python 3.20.

  (Contribuição de Sergey B Kirpichev e Serhiy Storchaka em gh-143715 [https://github.com/python/cpython/issues/143715].)

- Os atributos `__version__`, `version` e `VERSION` foram descontinuados nestes módulos da biblioteca padrão e serão removidos no Python 3.20. Utilize `sys.version_info` em seu lugar.

  - `argparse`
  - `csv`
  - `ctypes`
  - `ctypes.macholib`
  - `decimal` (use `decimal.SPEC_VERSION`)
  - `http.server`
  - `imaplib`
  - `ipaddress`
  - `json`
  - `logging` (`__date__` também descontinuado)
  - `optparse`
  - `pickle`
  - `platform`
  - `re`
  - `socketserver`
  - `tabnanny`
  - `tarfile`
  - `tkinter.font`
  - `tkinter.ttk`
  - `wsgiref.simple_server`
  - `xml.etree.ElementTree`

  - **`xml.sax.expatreader`**
  - **`xml.sax.handler`**
  - **`zlib`**

  (Contribuição de Hugo van Kemenade e Stan Ulbrych em gh-76007 [https://github.com/python/cpython/issues/76007].)

- Descontinuações definidas por **PEP 829** [https://peps.python.org/pep-0829/]:

  - Avisos são gerados para linhas `import` encontradas em arquivos *`name`*`.pth`.
  - Arquivos *`name`*`.pth` não são mais decodificados usando a codificação da localidade (*locale encoding*) por padrão. Eles **DEVEM** estar codificados em `utf-8-sig`.

  (Contribuição de Barry Warsaw em gh-148641 [https://github.com/python/cpython/issues/148641].)

- **`ast`**:

  - A criação de instâncias de nós abstratos da AST (como **`ast.AST`** ou **`ast.expr`**) está descontinuada e levantará um erro no Python 3.20.

- **`typing`**:

  - É desencorajado realizar verificações com **`isinstance()`** e **`issubclass()`** em classes de protocolo que não foram explicitamente decoradas com **`runtime_checkable()`**, mas que herdam de uma classe de protocolo verificável em tempo de execução. Isso resultará em um **`TypeError`** no Python 3.20.

    (Contribuição de Bartosz Sławecki em gh-132604 [https://github.com/python/cpython/issues/132604].)

Remoção pendente em versões futuras

As APIs a seguir serão removidas no futuro, embora atualmente não haja uma data agendada para sua remoção.

- **argparse**:
  - O aninhamento de grupos de argumentos e o aninhamento de grupos mutuamente exclusivos estão descontinuado.
  - A passagem do argumento nomeado não documentado *prefix_chars* para **add_argument_group()** agora está descontinuado.
  - O conversor de tipo **argparse.FileType** está descontinuado.
- **builtins**:
  - Geradores: A assinatura `throw(type, exc, tb)` e `athrow(type, exc, tb)` está descontinuada: use `throw(exc)` e `athrow(exc)`, a assinatura do argumento único.
  - Atualmente Python aceita literais numéricos imediatamente seguidos por palavras reservadas como, por exemplo, `0in x`, `1or x`, `0if 1else 2`. Ele permite expressões confusas e ambíguas como `[0x1for x in y]` (que pode ser interpretada como `[0x1 for x in y]` ou `[0x1f or x in y]`). Um aviso de sintaxe é levantado se o literal numérico for imediatamente seguido por uma das palavras reservadas **and**, **else**, **for**, **if**, **in** , **is** e **or**. Em uma versão futura, será alterado para um erro de sintaxe. (gh-87999 [https://github.com/python/cpython/issues/87999])
  - Suporte para métodos `__index__()` e `__int__()` retornando tipo não-int: esses métodos serão necessários para retornar uma instância de uma subclasse estrita de **int**.
  - Suporte para o método `__float__()` retornando uma subclasse estrita de **float**: esses métodos serão necessários para retornar uma instância de **float**.
  - Suporte para o método `__complex__()` retornando uma subclasse estrita de **complex**: esses métodos serão necessários para retornar

uma instância de **`complex`**.
    - Passar um número complexo como argumento *real* ou *imag* no construtor **`complex()`** agora está descontinuado; deve ser passado apenas como um único argumento posicional. (Contribuição de Serhiy Storchaka em gh-109218 [https://github.com/python/cpython/issues/109218].)
- **`calendar`**: As constantes `calendar.January` e `calendar.February` foram descontinuadas e substituídas por **`calendar.JANUARY`** e **`calendar.FEBRUARY`**. (Contribuição de Prince Roshan em gh-103636 [https://github.com/python/cpython/issues/103636].)
- **`codecs`**: use **`open()`** em vez de **`codecs.open()`**. (gh-133038 [https://github.com/python/cpython/issues/133038])
- **`codeobject.co_lnotab`**: use o método **`codeobject.co_lines()`**.
- **`datetime`**:
    - **`utcnow()`**: use `datetime.datetime.now(tz=datetime.UTC)`.
    - **`utcfromtimestamp()`**: use `datetime.datetime.fromtimestamp(timestamp, tz=datetime.UTC)`.
- **`gettext`**: O valor de plural deve ser um número inteiro.
- **`importlib`**:
    - O parâmetro *debug_override* de **`cache_from_source()`** foi descontinuado: em vez disso, use o parâmetro *optimization*.
- **`importlib.metadata`**:
    - Interface de tupla `EntryPoints`.
    - `None` implícito nos valores de retorno.
- **`logging`**: o método `warn()` foi descontinuado desde o Python 3.3, use **`warning()`**.
- **`mailbox`**: O uso da entrada StringIO e do modo de texto foi descontinuado; em vez disso, use BytesIO e o modo binário.
- **`os`**: chame **`os.register_at_fork()`** em um processo multithread.
- **`os.path`**: **`os.path.commonprefix()`** está descontinuada; use **`os.path.commonpath()`** para prefixos de caminho. A função **`os.path.commonprefix()`** está sendo descontinuada por possuir nome e módulo enganosos. O uso dessa função não é seguro para prefixos de caminho, apesar de ela estar incluída em um módulo voltado à manipulação de caminhos; isso significa que é fácil introduzir

acidentalmente vulnerabilidades de *path traversal* (travessia de diretórios) em programas Python ao utilizá-la.

- **`pydoc.ErrorDuringImport`**: Um valor de tupla para o parâmetro *exc_info* foi descontinuado, use uma instância de exceção.
- `re`: Regras mais rigorosas agora são aplicadas para referências numéricas de grupos e nomes de grupos em expressões regulares. Apenas a sequência de dígitos ASCII agora é aceita como referência numérica. O nome do grupo em padrões de bytes e strings de substituição agora pode conter apenas letras e dígitos ASCII e sublinhado. (Contribuição de Serhiy Storchaka em gh-91760 [https://github.com/python/cpython/issues/91760].)
- `shutil`: O parâmetro *onerror* de `rmtree()` foi descontinuado no Python 3.12; use o parâmetro *onexc*.
- Protocolos e opções de `ssl`
  - `ssl.SSLContext` sem argumento de protocolo foi descontinuado.
  - `ssl.SSLContext`: `set_npn_protocols()` e **`selected_npn_protocol()`** foram descontinuados: use ALPN.
  - Opções de `ssl.OP_NO_SSL*`
  - Opções de `ssl.OP_NO_TLS*`
  - `ssl.PROTOCOL_SSLv3`
  - `ssl.PROTOCOL_TLS`
  - `ssl.PROTOCOL_TLSv1`
  - `ssl.PROTOCOL_TLSv1_1`
  - `ssl.PROTOCOL_TLSv1_2`
  - `ssl.TLSVersion.SSLv3`
  - `ssl.TLSVersion.TLSv1`
  - `ssl.TLSVersion.TLSv1_1`
- Métodos de `threading`:
  - **`threading.Condition.notifyAll()`**: use `notify_all()`.
  - **`threading.Event.isSet()`**: use `is_set()`.
  - **`threading.Thread.isDaemon()`**, `threading.Thread.setDaemon()`: use o atributo `threading.Thread.daemon`.
  - **`threading.Thread.getName()`**, `threading.Thread.setName()`: use o atributo `threading.Thread.name`.
  - **`threading.currentThread()`**: use `threading.current_thread()`.

    - **`threading.activeCount()`**: use **`threading.active_count()`**.
- **`typing.Text`** (gh-92332 [https://github.com/python/cpython/issues/92332]).
- A classe interna `typing._UnionGenericAlias` não é mais usada para implementar **`typing.Union`**. Para preservar a compatibilidade com usuários que utilizam esta classe privada, uma correção de compatibilidade será fornecida pelo menos até a versão 3.17 do Python. (Contribuição de Jelle Zijlstra em gh-105499 [https://github.com/python/cpython/issues/105499].)
- **`unittest.IsolatedAsyncioTestCase`**: foi descontinuado retornar um valor que não seja `None` de um caso de teste.
- Funções descontinuadas de **`urllib.parse`**: use **`urlparse()`**
    - `splitattr()`
    - `splithost()`
    - `splitnport()`
    - `splitpasswd()`
    - `splitport()`
    - `splitquery()`
    - `splittag()`
    - `splittype()`
    - `splituser()`
    - `splitvalue()`
    - `to_bytes()`
- **`wsgiref`**: `SimpleHandler.stdout.write()` não deve fazer gravações parciais.
- **`xml.etree.ElementTree`**: testar o valor verdade de um **`Element`** está descontinuado. Em um lançamento futuro isso sempre retornará `True`. Em vez disso, prefira os testes explícitos `len(elem)` ou `elem is not None`.
- **`sys._clear_type_cache()`** está descontinuada: use **`sys._clear_internal_caches()`**.

descontinuações suave

Não há planos para remover APIs suavemente descontinuadas

- **`re.match()`** e **`re.Pattern.match()`** estão agora suavemente descontinuados em favor das novas APIs **`re.prefixmatch()`** e **`re.Pattern.prefixmatch()`**, adicionadas como nomes alternativos e mais explícitos. Elas visam reduzir a confusão sobre o significado de *match*, seguindo o princípio do Zen do Python: *"Explícito é melhor que implícito"*. A maioria das bibliotecas de expressões regulares de outras linguagens utiliza uma API chamada *match* para a funcionalidade que o Python sempre chamou de *search*.

  Não planejamos remover o nome mais antigo **`match()`**, pois ele tem sido utilizado no código há mais de 30 anos. Códigos que oferecem suporte a versões mais antigas do Python devem continuar utilizando **`match()`**, enquanto novos códigos devem dar preferência a **`prefixmatch()`**. Veja prefixmatch() vs. match().

  (Contribuição de Gregory P. Smith em gh-86519 [https://github.com/python/cpython/issues/86519] e Hugo van Kemenade em gh-148100 [https://github.com/python/cpython/issues/148100].)

- O uso dos códigos de tipo de formato `'F'` e `'D'` do módulo **`struct`** está agora suavemente descontinuado em favor das formas de duas letras `'Zf'` e `'Zd'`. (Contribuição de Sergey B Kirpichev em gh-121249 [https://github.com/python/cpython/issues/121249].)

Distribuindo módulos Python

Nota

Informações e orientações sobre distribuição de módulos e pacotes Python foram movidas para o Guia de Usuário para Empacotamento de Python [https://packaging.python.org/en/latest/tutorials/packaging-projects/] e o tutorial sobre empacotamento de projetos Python [https://packaging.python.org/].

How-To - Clínica de Argumento

Nota

O How-TO da Clínica de Argumento foi movido para o Python Developer's Guide [https://devguide.python.org/development-tools/clinic/].

Como portar códigos do Python 2 para o Python 3

O Python 2 atingiu seu fim de vida oficial no início de 2020. Isso significa que nenhum novo relatório de bug, correção ou alteração será feito no Python 2 - ele não é mais compatível: consulte a **PEP 373** [https://peps.python.org/pep-0373/] e o status das versões do Python [https://devguide.python.org/versions].

Se você está pensando em portar um módulo de extensão em vez de puro código Python, veja Portando módulos de extensão para o Python 3.

A lista de discussão python-porting [https://mail.python.org/pipermail/python-porting/] arquivada pode conter algumas orientações úteis.

Desde o Python 3.11, o guia de portabilidade original foi descontinuado. Você pode encontrar o guia antigo no arquivo [https://docs.python.org/3.10/howto/pyporting.html].

Guias de terceiros

Existem também vários guias de terceiros que podem ser úteis:

- Guia por Fedora [https://portingguide.readthedocs.io]
- Tutorial da PyCon 2020 [https://www.youtube.com/watch?v=JgIgEjASOlk]
- Guia por DigitalOcean [https://www.digitalocean.com/community/tutorials/how-to-port-python-2-code-to-python-3]
- Guia por ActiveState [https://www.activestate.com/blog/how-to-migrate-python-2-applications-to-python-3]

`aifc` — Lê e escreve arquivos AIFF e AIFC

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `aifc` foi o Python 3.12 [https://docs.python.org/3.12/library/aifc.html].

`asynchat` — Manipulador de comando/resposta de socket assíncrono

Descontinuado desde a versão 3.6, removido na versão 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.6. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

As aplicações devem usar o módulo **`asyncio`** em vez disso.

A última versão do Python que forneceu o módulo **`asynchat`** foi o Python 3.11 [https://docs.python.org/3.11/library/asynchat.html].

`asyncore` — Manipulador de socket assíncrono

Descontinuado desde a versão 3.6, removido na versão 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.6. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

As aplicações devem usar o módulo `asyncio` em vez disso.

A última versão do Python que forneceu o módulo `asyncore` foi o Python 3.11 [https://docs.python.org/3.11/library/asyncore.html].

`audioop` — Manipulando dados de áudio original

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `audioop` foi o Python 3.12 [https://docs.python.org/3.12/library/audioop.html].

`cgi` — Suporte a Common Gateway Interface

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Um fork do módulo no PyPI pode ser usado em vez disso: legacy-cgi [https://pypi.org/project/legacy-cgi/]. Esta é uma cópia do módulo cgi, não mais mantido ou suportado pela equipe core do Python.

A última versão do Python que forneceu o módulo **`cgi`** foi o Python 3.12 [https://docs.python.org/3.12/library/cgi.html].

`cgitb` — Gerenciador de traceback para scripts de CGI

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Um fork do módulo no PyPI agora pode ser usado em vez disso: legacy-cgi [https://pypi.org/project/legacy-cgi/]. Esta é uma cópia do módulo cgi, não mais mantido ou suportado pela equipe core do Python.

A última versão do Python que forneceu o módulo **`cgitb`** foi o Python 3.12 [https://docs.python.org/3.12/library/cgitb.html].

chunk — Lê dados em blocos IFF

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo **chunk** foi o Python 3.12 [https://docs.python.org/3.12/library/chunk.html].

`crypt` — Função para verificar senhas Unix

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

As aplicações podem usar o módulo **`hashlib`** da biblioteca padrão. Outras possíveis substituições são bibliotecas de terceiros do PyPI: legacycrypt [https://pypi.org/project/legacycrypt/], bcrypt [https://pypi.org/project/bcrypt/] ou argon2-cffi [https://pypi.org/project/argon2-cffi/]. Elas não são suportadas nem mantidas pela equipe core do Python.

A última versão do Python que forneceu o módulo **`crypt`** foi o Python 3.12 [https://docs.python.org/3.12/library/crypt.html].

`distutils` — Criação e instalação de módulos do Python

Descontinuado desde a versão 3.10, removido na versão 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.10. A remoção foi decidida na **PEP 632** [https://peps.python.org/pep-0632/], a qual tem um conselho de migração [https://peps.python.org/pep-0632/#migration-advice].

A última versão do Python que forneceu o módulo `distutils` foi o Python 3.11 [https://docs.python.org/3.11/library/distutils.html].

Frameworks de programa

Este capítulo não é mais mantido, e os módulos que ele continha foram movidos para suas respectivas documentações temáticas.

- `cmd` — Bibliotecas de interface de linha de comando
- `shlex` — Serviços específicos do Unix
- `turtle` — Interfaces gráficas de usuário com Tk

`imghdr` — Determina o tipo de uma imagem

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Possíveis substituições são bibliotecas de terceiros do PyPI: filetype [https://pypi.org/project/filetype/], puremagic [https://pypi.org/project/puremagic/] ou python-magic [https://pypi.org/project/python-magic/]. Eles não são suportados ou mantidos pela equipe principal do Python.

A última versão do Python que forneceu o módulo `imghdr` foi o Python 3.12 [https://docs.python.org/3.12/library/imghdr.html].

`imp` — Acessa os componentes internos de importação

Descontinuado desde a versão 3.4, removido na versão 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.4.

O aviso de remoção inclui orientação para migrar código de **`imp`** para **`importlib`**.

A última versão do Python que forneceu o módulo **`imp`** foi o Python 3.11 [https://docs.python.org/3.11/library/imp.html].

`mailcap` — Manipulação de arquivo Mailcap

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `mailcap` foi o Python 3.12 [https://docs.python.org/3.12/library/mailcap.html].

`msilib` — Lê e escreve arquivos do Microsoft Installer

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `msilib` foi o Python 3.12 [https://docs.python.org/3.12/library/msilib.html].

`nis` — Interface para NIS da Sun (Yellow Pages)

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `nis` foi o Python 3.12 [https://docs.python.org/3.12/library/nis.html].

`nntplib` — Cliente de protocolo NNTP

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `nntplib` foi o Python 3.12 [https://docs.python.org/3.12/library/nntplib.html].

`ossaudiodev` — Acesso a dispositivos de áudio compatíveis com OSS

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `ossaudiodev` foi o Python 3.12 [https://docs.python.org/3.12/library/ossaudiodev.html].

`pipes` — Interface para pipelines do shell

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

As aplicações devem usar o módulo `subprocess` em vez disso.

A última versão do Python que forneceu o módulo `pipes` foi o Python 3.12 [https://docs.python.org/3.12/library/pipes.html].

`smtpd` — Serviços SMTP

Descontinuado desde a versão 3.6, removido na versão 3.12.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.12 após ser descontinuado no Python 3.6. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Uma possível substituição é a biblioteca de terceiros aiosmtpd [https://pypi.org/project/aiosmtpd/]. Esta biblioteca não é mantida ou suportada pela equipe principal do Python.

A última versão do Python que forneceu o módulo `smtpd` foi o Python 3.11 [https://docs.python.org/3.11/library/smtpd.html].

`sndhdr` — Determina o tipo de arquivos de som

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Possíveis substituições são módulos de terceiros do PyPI: filetype [https://pypi.org/project/filetype/], puremagic [https://pypi.org/project/puremagic/] ou python-magic [https://pypi.org/project/python-magic/]. Eles não são suportados ou mantidos pela equipe principal do Python.

A última versão do Python que forneceu o módulo `sndhdr` foi o Python 3.12 [https://docs.python.org/3.12/library/sndhdr.html].

`spwd` — O banco de dados de senhas shadow

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Uma possível substituição é a biblioteca de terceiros python-pam [https://pypi.org/project/python-pam/]. Esta biblioteca não é suportada ou mantida pela equipe principal do Python.

A última versão do Python que forneceu o módulo `spwd` foi o Python 3.12 [https://docs.python.org/3.12/library/spwd.html].

sunau — Lê e escreve arquivos AU da Sun

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo **sunau** foi o Python 3.12 [https://docs.python.org/3.12/library/sunau.html].

`telnetlib` — cliente Telnet

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

Possíveis substituições são bibliotecas de terceiros do PyPI: telnetlib3 [https://pypi.org/project/telnetlib3/] ou Exscript [https://pypi.org/project/Exscript/]. Elas não são suportadas ou mantidas pela equipe principal do Python.

A última versão do Python que forneceu o módulo `telnetlib` foi o Python 3.12 [https://docs.python.org/3.12/library/telnetlib.html].

uu — Codifica e decodifica arquivos uuencode

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A codificação e a decodificação no formato uu podem ser realizadas, alternativamente, utilizando `codecs.encode()` e `codecs.decode()`, especificando `"uu"` como a codificação.

A última versão do Python que forneceu o módulo **uu** foi o Python 3.12 [https://docs.python.org/3.12/library/uu.html].

`xdrlib` — Codifica e decodifica dados XDR

Descontinuado desde a versão 3.11, removido na versão 3.13.

Este módulo não faz mais parte da biblioteca padrão do Python. Ele foi removido no Python 3.13 após ser descontinuado no Python 3.11. A remoção foi decidida na **PEP 594** [https://peps.python.org/pep-0594/].

A última versão do Python que forneceu o módulo `xdrlib` foi o Python 3.12 [https://docs.python.org/3.12/library/xdrlib.html].